Vim 中文手册

http://github.com/yianwillis/vimcdoc

January 7, 2020

help.txt For Vim version 8.2. 最近更新：2019年12月

VIM - 主帮助文件

移动：使用光标键，或者用 "h" 向左，"j" 向下，"k" 向上，"l" 向右。

```
      k
  h   l
      j
```

关闭本窗口：使用 ":q<Enter>"。

离开 Vim：使用 ":qa!<Enter>"（当心，所有的改动都会丢失！）。

跳转到一个主题：将光标置于标签（例如 bars ）上然后输入 CTRL-]。

使用鼠标：":set mouse=a" 启动对鼠标的支持（用于 xterm 或 GUI）。
在标签（例如 bars ）上双击。

跳回：键入 CTRL-O。重复则继续向后。

获取特定帮助：在 :help 命令后给出参数可以直接跳转到任何的帮助主题。
还可以通过前缀进一步指定上下文：help-context

类别	前缀	例子
普通模式命令		:help x
可视模式命令	v_	:help v_u
插入模式命令	i_	:help i_<Esc>
命令行模式命令	:	:help :quit
命令行编辑	c_	:help c_<Del>
Vim 命令参数	-	:help -r
选项	'	:help 'textwidth'
正规表达式	/	:help /[

更多上下文及说明参见 help-summary 。

查找帮助：键入 ":help word"，接着键入 CTRL-D 可以看到匹配 "word" 的帮助主题。
也可用 ":helpgrep word"。 :helpgrep

入门：做完 Vim 教程，基本命令 20 分钟的交互式培训，见 vimtutor 。
从头到底读完用户手册： usr_01.txt

Vim 代表 Vi IMproved。Vim 主要由 Bram Moolenaar 编写，但没有许多其他人的帮助是不可能的。请见 credits 。

---

doc-file-list Q_ct

基础：

quickref 总览最常用的命令

tutor	面向初学者的 20 分钟教程
copying	版权信息
iccf	资助乌干达贫困儿童
sponsor	赞助 Vim 的开发，成为一名注册用户
www	互联网上的 Vim
bugs	如何提交漏洞报告

用户手册：这些文件解释如何完成一项编辑任务。

usr_toc.txt	目录

初步知识

usr_01.txt	关于本手册
usr_02.txt	Vim 初步
usr_03.txt	移动
usr_04.txt	做小改动
usr_05.txt	选项设置
usr_06.txt	使用语法高亮
usr_07.txt	编辑多个文件
usr_08.txt	分割窗口
usr_09.txt	使用 GUI 版本
usr_10.txt	做大修改
usr_11.txt	从崩溃中恢复
usr_12.txt	小窍门

高效地编辑

usr_20.txt	快速键入命令行
usr_21.txt	离开和回来
usr_22.txt	寻找要编辑的文件
usr_23.txt	编辑特殊文件
usr_24.txt	快速插入
usr_25.txt	编辑带格式的文本
usr_26.txt	重复
usr_27.txt	查找命令及模式
usr_28.txt	折叠
usr_29.txt	在代码间移动
usr_30.txt	编辑程序
usr_31.txt	利用 GUI
usr_32.txt	撤销树

调节 Vim

usr_40.txt	创建新的命令
usr_41.txt	编写 Vim 脚本
usr_42.txt	添加新的菜单
usr_43.txt	使用文件类型
usr_44.txt	自定义语法高亮
usr_45.txt	选择你的语言

让 Vim 工作

usr_90.txt	安装 Vim

参考手册：这些文件详细地解释 Vim 的每个细节。 reference_toc

一般主题

intro.txt	对 Vim 的一般介绍；帮助文件中所使用的符号
help.txt	总览及快速参考（本文件）
helphelp.txt	关于如何使用帮助文件
index.txt	以字母顺序排列的命令索引
help-tags	可跳转的所有标签（标签索引）
howto.txt	如何完成最常见的编辑任务
tips.txt	Vim 的各种窍门
message.txt	（错误）信息及解释
quotes.txt	Vim 的用户评价
todo.txt	已知的问题以及扩展计划
develop.txt	Vim 的开发
debug.txt	调试 Vim 自身
uganda.txt	Vim 发布条件和怎么花你的 money

基本编辑

starting.txt	启动 Vim，Vim 命令参数，初始化
editing.txt	编辑和写入文件
motion.txt	有关移动的命令
scroll.txt	在窗口中滚动文本
insert.txt	插入模式和替换模式
change.txt	删除和替换文本
undo.txt	撤销和重做
repeat.txt	重复命令，Vim 脚本及调试
visual.txt	使用可视模式（选择一个文本区域）
various.txt	各种其它命令
recover.txt	从崩溃中恢复

高级编辑

cmdline.txt	命令行编辑
options.txt	所有选项的描述
pattern.txt	正则表达式及查找命令
map.txt	键盘映射及缩写
tagsrch.txt	标签及特殊查找
windows.txt	使用多窗口多缓冲区的命令
tabpage.txt	使用多个标签页的命令
spell.txt	拼写检查
diff.txt	编辑同一文本的两到四个版本
autocmd.txt	当事件发生时自动执行命令
eval.txt	表达式求值，条件执行命令
channel.txt	作业、通道、进程间通信
fold.txt	隐藏（折叠）一定范围内的文本

特殊问题

testing.txt	Vim 和 Vim 脚本的测试
print.txt	打印
remote.txt	将 Vim 用作服务器/客户端
term.txt	使用不同终端和鼠标
terminal.txt	终端窗口支持
popup.txt	弹出窗口支持

编程语言支持

indent.txt	C 及其它语言的自动缩进
syntax.txt	语法高亮

textprop.txt 为文本附加属性，用于高亮和其它目的
filetype.txt 对某一类型文件的专有设定
quickfix.txt 支持快速 编辑-编译-修正 循环的命令
ft_ada.txt Ada (编程语言) 支持
ft_rust.txt Rust 文件类型支持
ft_sql.txt SQL 文件类型插件相关

语言支持
digraph.txt 可用的二合字母清单
mbyte.txt 多字节文本支持
mlang.txt 非英语语言支持
rileft.txt 从右向左的编辑模式
arabic.txt 阿拉伯语支持及编辑
farsi.txt Farsi (波斯语) 编辑
hebrew.txt 希伯来语支持及编辑
russian.txt 俄罗斯语支持及编辑
hangulin.txt Hangul (韩语) 输入模式

GUI
gui.txt 图形用户界面 (GUI)
gui_w32.txt Win32 GUI
gui_x11.txt X11 GUI

各种接口
if_cscop.txt 联用 cscope 与 Vim
if_lua.txt Lua 接口
if_mzsch.txt MzScheme 接口
if_perl.txt Perl 接口
if_pyth.txt Python 接口
if_tcl.txt Tcl 接口
if_ole.txt Win32 下的 OLE 接口
if_ruby.txt Ruby 接口
debugger.txt 与调试程序的接口
netbeans.txt NetBeans 外部编辑器接口
sign.txt 调试标号

版本
vi_diff.txt Vim 和 Vi 之间的主要区别
version4.txt Vim 3.0 和 4.x 版本的区别
version5.txt Vim 4.6 和 5.x 版本的区别
version6.txt Vim 5.7 和 6.x 版本的区别
version7.txt Vim 6.4 和 7.x 版本的区别
version8.txt Vim 7.4 和 8.x 版本的区别

sys-file-list
特定系统相关
os_390.txt OS/390 Unix
os_amiga.txt Amiga
os_beos.txt BeOS 和 BeBox
os_dos.txt MS-DOS 和 MS-Windows NT/95 通用的条目
os_mac.txt Macintosh
os_mint.txt Atari MiNT
os_msdos.txt MS-DOS (纯 DOS 及 Windows 内的 DOS 窗口)
os_os2.txt OS/2
os_qnx.txt QNX

os_risc.txt RISC-OS
os_unix.txt Unix
os_vms.txt VMS
os_win32.txt MS-Windows 95/98/NT

standard-plugin-list

标准插件

pi_getscript.txt 下载 Vim 脚本的最新版本
pi_gzip.txt 读写压缩过的文件
pi_logipat.txt 模式的逻辑操作符
pi_netrw.txt 通过网络读写文件
pi_paren.txt 高亮匹配括号
pi_spec.txt 用于 rpm 规格文件的文件类型插件
pi_tar.txt Tar 文件探索器
pi_vimball.txt 创建自安装的 Vim 脚本
pi_zip.txt Zip 归档探索器

本地附加文档

local-additions 请查阅英文文档

---

bars Bars 举例

现在你已经通过 CTRL-] 或双击鼠标跳转到这里了，CTRL-T、CTRL-O、g<RightMouse>、或 <C-RightMouse> 可以用来跳回到你原来的地方。

注意 标签包含在两个 | 字符之间，但打开高亮时这些字符被隐藏，这样更易于命令的阅读。

不过，你可以在任何单词上按 CTRL-]，即使该单词不出现在 | 之间也无妨。Vim 会试图寻找该单词的相关帮助，尤其是单引号之间的选项名，如 'compatible'。

---

vim:tw=78:isk=!-~,^*,^\|,^\":ts=8:noet:ft=help:norl:

usr_toc.txt For Vim version 8.2. 最近更新：2020年1月

VIM 用户手册 - by Bram Moolenaar

目　　录　　user-manual

总览

初步知识

usr_01.txt 关于本手册
usr_02.txt Vim 初步
usr_03.txt 移动
usr_04.txt 做小改动
usr_05.txt 选项设置
usr_06.txt 使用语法高亮
usr_07.txt 编辑多个文件
usr_08.txt 分割窗口
usr_09.txt 使用 GUI 版本
usr_10.txt 做大修改
usr_11.txt 从崩溃中恢复
usr_12.txt 小窍门

高效的编辑

usr_20.txt 快速键入命令行命令
usr_21.txt 离开和回来
usr_22.txt 寻找要编辑的文件
usr_23.txt 编辑特殊文件
usr_24.txt 快速插入
usr_25.txt 编辑已经编排过的文本
usr_26.txt 重复
usr_27.txt 查找命令及模式
usr_28.txt 折叠
usr_29.txt 在代码间移动
usr_30.txt 编辑程序
usr_31.txt 利用 GUI
usr_32.txt 撤销树

调节 Vim

usr_40.txt 创建新的命令
usr_41.txt 编写 Vim 脚本
usr_42.txt 添加新的菜单
usr_43.txt 使用文件类型
usr_44.txt 自定义语法高亮
usr_45.txt 选择你的语言

让 Vim 工作

usr_90.txt 安装 Vim

参考手册

reference_toc 关于所有命令更详细的信息

本手册的英文 HTML 版本和 PDF 版本可以从以下这个地址得到：

http://vimdoc.sf.net

本手册的中文 HTML 版本和 PDF 版本可以从以下这个地址得到:
http://yianwillis.github.io/vimcdoc (HTML)
http://github.com/yianwillis/vimcdoc/releases (PDF)

初步知识

从头至尾阅读这些文档可以学习基本的命令。

usr_01.txt 关于本手册
- 01.1 手册的两个部分
- 01.2 安装了 Vim 之后
- 01.3 教程使用说明
- 01.4 版权声明

usr_02.txt Vim 初步
- 02.1 第一次运行 Vim
- 02.2 插入文本
- 02.3 移动光标
- 02.4 删除字符
- 02.5 撤销与重做
- 02.6 其它编辑命令
- 02.7 退出
- 02.8 寻求帮助

usr_03.txt 移动
- 03.1 词移动
- 03.2 移动到行首或行尾
- 03.3 移动到指定字符
- 03.4 括号匹配
- 03.5 移动到指定的行
- 03.6 确定当前位置
- 03.7 滚屏
- 03.8 简单查找
- 03.9 简单的查找模式
- 03.10 使用标记

usr_04.txt 做小改动
- 04.1 操作符与动作
- 04.2 改变文本
- 04.3 重复一个修改
- 04.4 可视模式
- 04.5 移动文本
- 04.6 拷贝文本
- 04.7 使用剪贴板
- 04.8 文本对象
- 04.9 替换模式
- 04.10 结论

usr_05.txt 选项设置
- 05.1 vimrc 文件

05.2 vimrc 示例解释
05.3 defaults.vim 文件解释
05.4 简单键盘映射
05.5 添加软件包
05.6 添加插件
05.7 添加帮助
05.8 选项窗口
05.9 常用选项

usr_06.txt 使用语法高亮
06.1 功能激活
06.2 颜色显示不出来或者显示出错误的颜色怎么办?
06.3 使用不同颜色
06.4 是否使用颜色
06.5 带颜色打印
06.6 深入阅读

usr_07.txt 编辑多个文件
07.1 编辑另一个文件
07.2 文件列表
07.3 从一个文件中跳到另一个文件
07.4 备份文件
07.5 文件间拷贝文本
07.6 显示文件
07.7 修改文件名

usr_08.txt 分割窗口
08.1 分割窗口
08.2 用另一个文件分割窗口
08.3 窗口大小
08.4 垂直分割
08.5 移动窗口
08.6 对所有窗口执行命令
08.7 用 vimdiff 显示区别
08.8 杂项
08.9 标签页

usr_09.txt 使用 GUI 版本
09.1 GUI 版本的组件
09.2 使用鼠标
09.3 剪贴板
09.4 选择模式

usr_10.txt 做大修改
10.1 记录与回放命令
10.2 替换
10.3 命令范围
10.4 global 命令
10.5 可视块模式
10.6 读、写部分文件内容
10.7 编排文本
10.8 改变大小写
10.9 使用外部程序

usr_11.txt 从崩溃中恢复
11.1 基本恢复
11.2 交换文件在哪
11.3 是不是崩溃了?
11.4 深入阅读

usr_12.txt 小窍门
12.1 单词替换
12.2 把 "Last, First" 改成 "First Last"
12.3 排序
12.4 反排行顺序
12.5 单词统计
12.6 查阅 man 信息
12.7 删除多余空格
12.8 查找单词的使用位置

高效的编辑

可以独立阅读的主题。

usr_20.txt 快速键入命令行
20.1 命令行编辑
20.2 命令行缩写
20.3 命令行补全
20.4 命令行历史
20.5 命令行窗口

usr_21.txt 离开和回来
21.1 挂起和继续
21.2 执行外壳命令
21.3 记忆有关信息; viminfo
21.4 会话
21.5 视图
21.6 模式行

usr_22.txt 寻找要编辑的文件
22.1 文件浏览器
22.2 当前目录
22.3 查找文件
22.4 缓冲区列表

usr_23.txt 编辑特殊文件
23.1 DOS、Mac 和 Unix 文件
23.2 互联网上的文件
23.3 加密
23.4 二进制文件
23.5 压缩文件

usr_24.txt 快速插入
24.1 更正
24.2 显示匹配
24.3 补全
24.4 重复一次插入

24.5 从另一行拷贝
24.6 插入一个寄存器内容
24.7 缩写
24.8 插入特殊字符
24.9 二合字母
24.10 普通模式命令

usr_25.txt 编辑带格式的文本
25.1 断行
25.2 对齐文本
25.3 缩进和制表符
25.4 对长行的处理
25.5 编辑表格

usr_26.txt 重复
26.1 可视模式下的重复
26.2 加与减
26.3 改动多个文件
26.4 在外壳脚本里使用 Vim

usr_27.txt 查找命令及模式
27.1 忽略大小写
27.2 在文件尾折返
27.3 偏移
27.4 匹配重复性模式
27.5 多择一
27.6 字符范围
27.7 字符类
27.8 匹配换行符
27.9 举例

usr_28.txt 折叠
28.1 什么是折叠?
28.2 手动折叠
28.3 对折叠的操作
28.4 存储和恢复折叠
28.5 依缩进折叠
28.6 依标志折叠
28.7 依语法折叠
28.8 依表达式折叠
28.9 折叠未被改动的行
28.10 使用哪种折叠办法呢?

usr_29.txt 在代码间移动
29.1 使用标签
29.2 预览窗口
29.3 在代码间移动
29.4 查找全局标识符
29.5 查找局部标识符

usr_30.txt 编辑程序
30.1 编译
30.2 C 文件缩进
30.3 自动缩进

30.4 其它缩进
30.5 制表符和空格
30.6 排版注释格式

usr_31.txt 使用 GUI
31.1 文件浏览器
31.2 确认
31.3 菜单快捷键
31.4 Vim 窗口位置与大小
31.5 杂项

usr_32.txt 撤销树
32.1 撤销到文件写入时的状态
32.2 为每次改变进行编号
32.3 撤销树内任意跳转
32.4 时间旅行

调节 Vim

告诉 Vim 如何工作。

usr_40.txt 创建新的命令
40.1 键映射
40.2 定义命令行命令
40.3 自动命令

usr_41.txt 编写 Vim 脚本
41.1 简介
41.2 变量
41.3 表达式
41.4 条件语句
41.5 执行一个表达式
41.6 使用函数
41.7 定义一个函数
41.8 列表和字典
41.9 例外
41.10 其它讨论
41.11 编写插件
41.12 编写文件类型插件
41.13 编写编译器插件
41.14 编写快速载入的插件
41.15 编写库脚本
41.16 发布 Vim 脚本

usr_42.txt 添加新的菜单
42.1 简介
42.2 菜单命令
42.3 杂项
42.4 工具栏和弹出菜单

usr_43.txt 使用文件类型
43.1 文件类型插件
43.2 添加一个文件类型

usr_44.txt 自定义语法高亮
44.1 基本语法命令
44.2 关键字
44.3 匹配
44.4 区域
44.5 嵌套项目
44.6 跟随组
44.7 其它参数
44.8 簇
44.9 包含其它语法文件
44.10 同步
44.11 安装语法文件
44.12 可移植的语法文件格式

usr_45.txt 选择你的语言
45.1 消息所用语言
45.2 菜单所用语言
45.3 使用其它种编码
45.4 编辑其它编码的文件
45.5 文本录入

让 Vim 工作

在你使用 Vim 之前。

usr_90.txt 安装 Vim
90.1 Unix
90.2 MS-Windows
90.3 升级
90.4 常见安装问题
90.5 卸载 Vim

版权: 参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

usr_01.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

关于本手册

本章介绍 vim 的手册本身。读者可以通过本章来了解本手册是如何解释 Vim 命令的。

01.1　手册的两个部分
01.2　安装了 Vim 之后
01.3　教程使用说明
01.4　版权声明

下一章：usr_02.txt　　Vim 初步
目录：usr_toc.txt

01.1　手册的两个部分

Vim 的手册分成两个部分：

1. 用户手册
   面向任务的使用说明书，由简入繁，能像书一样从头读到尾。

2. 参考手册
   详细描述 Vim 的每一个命令的详细资料。

本手册使用的符号请参见：notation

跳 转

文本包含两部分内容之间的超级链接，允许你快速地在一个编辑任务的描述和它所涉及的命令和选项的详细说明之间跳转。可以使用如下两个命令实现这个功能：

按 CTRL-] 跳转到当前光标下的单词的相关主题
按 CTRL-O 回跳（重复这个操作可以回跳多次）

大部分链接放在两根竖线之间，如：|bars| 。竖线本身可能被隐藏或不可见；见下。另外，'number' 形式的选项名，用双引号括住的命令 ":write" 或者其它任何单词都可以用作一个超级链接。不妨做如下试验：把光标移动到 CTRL-] 上，并按下 CTRL-]。

其它主题可以用 ":help" 找到；请参见 help.txt 。

竖线和星号在使用 conceal 功能时通常已被隐藏，它们也同时使用 hl-Ignore ，使文本色和背景色相同。要让它们重新可见：

```
:set conceallevel=0
:hi link HelpBar Normal
:hi link HelpStar Normal
```

01.2　安装了 Vim 之后

本手册大部分内容都假定 Vim 已经被正常安装了。如果还没有，或者运行不正常（例如，找不到文件或 GUI 模式下菜单没有显示出来等）。请先阅读关于安装的章节：
usr_90.txt 。

not-compatible
本手册还经常假定 Vim 运行在 Vi 兼容选项关闭的模式下。对大部分命令而言，这无关紧要，但有时却是很重要的。例如，多级撤销（undo）功能就是这样。有一个简单的方法可以保证你的 Vim 设置是正常的 — 拷贝 Vim 自带的 vimrc 范例。在 Vim 里进行操作，你就无需知道这个文件在什么地方。不同系统的操作方法分别如下：

对于 Unix:
```
:!cp -i $VIMRUNTIME/vimrc_example.vim ~/.vimrc
```
对于 MS-Windows:
```
:!copy $VIMRUNTIME/vimrc_example.vim $VIM/_vimrc
```
对于 Amiga:
```
:!copy $VIMRUNTIME/vimrc_example.vim $VIM/.vimrc
```

如果这个文件已经存在，你最好备份一下。

现在启动 Vim，'compatible' 选项应该已经关闭。你可以用以下命令检查一下：

```
:set compatible?
```

如果 Vim 报告 "nocompatible"，则一切正常；如果返回 "compatible" 就有问题。这时你需要检查一下为什么会出现这个问题。一种可能是 Vim 找不到你的配置文件。用如下命令检查一下：

```
:scriptnames
```

如果你的文件不在列表中，检查一下该文件的位置和文件名。如果它在列表中，那么一定是还有某个地方把 'compatible' 选项设回来了。

参考 vimrc 和 compatible-default 可以获得更多信息。

备注:
本手册是关于普通形态的 Vim 的，Vim 还有一种形态叫 "evim"（easy vim），那也是 Vim，不过被设置成 "点击并输入" 风格，就像 Notepad 一样。它总是处于 "插入" 模式，感觉完全不同于通常形态下的 Vim。由于它比较简陋，将不在本手册中描述。详细信息请参考 evim-keys 。

01.3 教程使用说明 tutor vimtutor

除了阅读文字（烦！），你还可以用 vimtutor（Vim 教程）学习基本的 Vim 命令，这是一个 30 分钟的教程，它能教会你大部分基本的 Vim 功能。

在 Unix 中，如果 Vim 安装正常，你可以从命令行上运行以下命令：

```
vimtutor
```

在 MS-Windows 中，你可以在 Program/vim 菜单中找到这个命令，或者在 $VIMRUNTIME 目录中运行 vimtutor.bat 程序。

这个命令会建立一份教程文件的拷贝，你可以任意修改它而不用担心会损坏原始的文件。

这个教材有各种语言的版本。要检查你需要的版本是否可用，使用相应语言的双字母缩写。例如，对于法语版本：

```
vimtutor fr
```

在 Unix 上，如果你更喜欢 Vim 的 GUI 版本，可用 "gvimtutor" 或 "vimtutor -g" 来代替 "vimtutor"。

对于 OpenVMS 而言，如果已经正确安装了 Vim，VMS 提示行上可以启动 vimtutor：

```
@VIM:vimtutor
```

也可以加入可选的双字母语言代码，同上。

在非 Unix 系统中，你还要做些工作：

1. 拷贝教程文件。你可以用 Vim 来完成这个工作 (Vim 知道文件的位置)

```
vim --clean -c 'e $VIMRUNTIME/tutor/tutor' -c 'w! TUTORCOPY' -c 'q'
```

这个命令在当前路径下建立一个 "TUTORCOPY" 的文件。要使用其它语言的版本，在文件名后加上双字母的语言缩写作为扩展名。对于法语：

```
vim --clean -c 'e $VIMRUNTIME/tutor/tutor.fr' -c 'w! TUTORCOPY' -c 'q'
```

2. 用 Vim 编辑这个被拷贝的文件

```
vim --clean -c "set nocp" TUTORCOPY
```

--clean 参数用于保证 Vim 使用良好的缺省值启动。

3. 学习完成后删除临时拷贝文件

```
del TUTORCOPY
```

01.4 版权声明 manual-copyright

Vim 用户手册和参考手册的版权 (1988-2003) 归 Bram Moolenaar 所有。这份材料只可以在符合开放出版物版权协议 (Open Publication License) 1.0 或以后版本的条件下发布。这份协议的最新版本可以在如下地址上找到：
http://opencontent.org/openpub/

对此文档作出贡献的人员必须同意这份声明。

frombook

这份文档的一部分来自 Steve Oualline 的《Vi IMproved - Vim》(New Riders出版，ISBN: 0735710015)。开放出版物版本协议同样适用于这本书。被选入的部分经过剪裁 (例如去掉插图，根据 Vim 6.0 和之后的新特性进行修改，并修正一些错误)。某些部分省略了 frombook 标记，这并不表示这些部分不是来自这本书。

非常感谢 Steve Oualline 和 New Riders 编写并出版了这本书并在 OPL 协议下发行它！它对写这份手册起到了非常大的作用。它不只是提供了文字描述，还确定了这份手册的基调和风格。

如果你通过销售这份手册赚了钱，我们强烈建议你把部分利润捐给乌干达的艾滋病受害者。(参见 iccf)

(译者注：这里是对原文协议的翻译，中文翻译的版权，请参阅 vimcdoc 的 LICENSE 文件)

下一章： usr_02.txt Vim 初步

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

Vim 初 步

本章提供足够的信息使你用够使用 Vim 来做基本的编辑。这里提供的方法不一定是高效快捷的。但起码是有效的。花些时间去练习这些命令，这是后面的知识的基础。

02.1 第一次运行 Vim
02.2 插入文本
02.3 移动光标
02.4 删除字符
02.5 撤销与重做
02.6 其它编辑命令
02.7 退出
02.8 寻求帮助

下一章：usr_03.txt 移动
前一章：usr_01.txt 关于本手册
目录：usr_toc.txt

02.1 第一次运行 Vim

启动 Vim 的命令如下：

```
gvim file.txt
```

在 UNIX 操作系统中，你可以在任意命令提示符下输入这个命令。如果你用的是 Microsoft Windows，启动一个 MS-DOS 窗口，再输入这个命令。

无论哪一种方式，现在 Vim 开始编辑一个名为 file.txt 的文件了。由于这是一个新建文件，你会得到一个空的窗口。屏幕看起来会像下面这样：

```
+----------------------------------------+
|#                                       |
|~                                       |
|~                                       |
|~                                       |
|~                                       |
|"file.txt" [New file]                   |
+----------------------------------------+
        ('#"是当前光标的位置)
```

以波纹线 (~) 开头的行表示该行在文件中不存在。换句话说，如果 Vim 打开的文件不能充满这个显示的屏幕，它就会显示以波纹线开头的行。在屏幕的底部，有一个消息行指示文件名为 file.txt 并且说明这是一个新建的文件。这行信息是临时的，新的信息可以覆盖它。

VIM 命 令

gvim 命令建立一个新窗口用于编辑。如果你用的是这个命令:

```
vim file.txt
```

则编辑在命令窗口内进行。换句话说，如果你在 xterm 中运行，编辑器使用 xterm 窗口。如果你用的是 Microsoft Window 的 MS-DOS 窗口，编辑器使用这个 MS-DOS 窗口。两个版本显示出来的文本看起来是一样的。但如果你用的是 gvim，就会有其他特性，如菜单栏。后面会有更多的描述。

02.2 插入文本

Vim 是一个多模式的编辑器。就是说，在不同模式下，编辑器的响应是不同的。在普通模式下，你敲入的字符只是命令；而在插入模式，你敲入的字符就成为插入的文本了。

当你刚刚进入 Vim，它处在普通模式。通过敲入 "i" 命令 (i 是插入 (Insert) 的缩写) 可以启动插入模式，这样你就可以输入文字了，这些文字将被插入到文件中。不用担心输错了，你随后还能够修正它。要输入下文的程序员打油诗，你可以这样敲:

```
iA very intelligent turtle
Found programming UNIX a hurdle
```

输入 "turtle" 后，按回车开始一个新行。最后，你输入 <Esc> 键退出插入模式而回到普通模式。现在在你的 Vim 窗口中就有两行文字了:

```
+----------------------------------------+
|A very intelligent turtle               |
|Found programming UNIX a hurdle         |
|~                                       |
|~                                       |
|                                        |
+----------------------------------------+
```

我在什么模式?

要看到你在什么模式，输入如下命令:

```
:set showmode
```

你会发现当你敲入冒号后，Vim 把光标移到窗口的最后一行。那里是你输入 "冒号命令" (以冒号开头的命令) 的地方，敲入回车结束这个命令的输入 (所有的冒号命令都用这种方式结束)。

现在，如果你输入 "i" 命令，Vim 会在窗口的底部显示 --INSERT-- (中文模式显示的是 --插入-- 译者注)，这表示你在插入模式。

```
+----------------------------------------+
|A very intelligent turtle               |
|Found programming UNIX a hurdle         |
|~                                       |
|~                                       |
|-- INSERT --                            |
+----------------------------------------+
```

如果你输入 <Esc> 回到普通模式，最后一行又变成空白。

摆 脱 麻 烦

Vim 新手常常遇到的一个问题是不知道自己在什么模式下，可能是因为忘了，也可能是因为不小心敲了一个切换模式的命令。无论你在什么模式，要回到普通模式，只要敲 <Esc> 就可以了。有时你需要敲两次，如果 Vim 发出 "嘀" 一声，就表示你已经在普通模式了。

02.3 移动光标

回到普通模式后，你可以使用如下命令移动光标：

h	左	hjkl
j	下	
k	上	
l	右	

这些命令看起来是随便选的。无论如何，谁听说过用 l 表示右的？但实际上，作这些选择是有理由的：移动光标是最常用的编辑器命令。而这些键位在你的右手本位。也就是说：这种键位的设定使你可以用最快的速度执行移动操作（特别是当你用十指输入的时候）。

> 备注：
> 你还可以用方向键移动光标，但这样会减慢你输入的速度，因为你必须把你的手从字母键移动到方向键。想象一下，你在一个小时内可要这样做几百次，这可要花相当多的时间的。
>
> 而且，有一些键盘是没有方向键的，或者放在一些很特别的地方。所以，知道 hjkl 的用法在这些情况下就很有帮助了。

记住这些命令的一个方法是：h 在左边，l 在右边，j 指着下面。用图表示如下：

```
    k
h   l
  j
```

学习这些命令的最好方法是使用它。用 "i" 命令输入更多的文字。然后用 hjkl 键移动光标并在某些地方输入一些单词。别忘了用 <Esc> 切换回普通模式。 vimtutor 也是一个练习的好办法。

02.4 删除字符

要删除一个字符，把光标移到它上面然后输入 "x"。(这是对以前的打字机的一种回归，那时你通过在字符上输入 xxxx 删除它) 例如，把光标移到行首，然后输入xxxxxxx (七个x) 可以删除 "A very "。结果看起来这样：

```
+---------------------------------------+
|intelligent turtle                     |
|Found programming UNIX a hurdle        |
|~                                      |
|~                                      |
```

```
|                                        |
+----------------------------------------+
```

现在你可以输入新的字符了，例如，通过输入：

```
iA young <Esc>
```

这个命令启动一次插入操作（那个 "i"），并插入 "A young"，然后退出插入模式（最后一个 <Esc>）。结果是：

```
+----------------------------------------+
|A young intelligent turtle              |
|Found programming UNIX a hurdle         |
|~                                       |
|~                                       |
|                                        |
+----------------------------------------+
```

删除一行

要删除一整行，使用 "dd" 命令，后一行会移上来填掉留下的空行：

```
+----------------------------------------+
|Found programming UNIX a hurdle         |
|~                                       |
|~                                       |
|~                                       |
|                                        |
+----------------------------------------+
```

删除一个换行符

在 Vim 中你可以把两行连起来，这意味着删除两行间的换行符。"J" 命令用于完成这个功能。

以下面两行为例：

```
A young intelligent
turtle
```

把光标移到第一行，然后按 "J"：

```
A young intelligent turtle
```

02.5 撤销与重做

假设现在你删得太多了。当然，你可以重新输入需要的内容。不过，你还有一个更简单的选择。"u" 命令撤销上一个编辑操作。看看下面这个操作：先用 "dd" 删除一行，再敲 "u"，该行又回来了。

再给一个例子：把光标移到第一行的 A 上：

```
A young intelligent turtle
```

现在输入 xxxxxxx 删除 "A young"。结果如下：

```
intelligent turtle
```

输入 "u" 撤销最后一个删除操作。那个删除操作删除字符 g，所以撤销命令恢复这个字符：

```
g intelligent turtle
```

下一个 "u" 命令恢复倒数第二个被删除的字符：

```
ng intelligent turtle
```

下一个 "u" 命令恢复 u，如此类推：

```
ung intelligent turtle
oung intelligent turtle
young intelligent turtle
young intelligent turtle
A young intelligent turtle
```

备注：
如果你输入 "u" 两次，你的文本恢复原样，那应该是你的 Vim 被配置在 Vi 兼容模式了。要修正这个问题，看看这里： not-compatible 。
本文假定你工作在 "Vim 的方式"。你可能更喜欢旧的 Vi 的模式，但是你必须小心本文中的一些小区别。

重 做

如果你撤销得太多，你可以输入 CTRL-R (redo) 回退前一个命令。换句话说，它撤销一个撤销。要看执行的例子，输入 CTRL-R 两次。字符 A 和它后面的空格就出现了：

```
young intelligent turtle
```

有一个特殊版本的撤销命令："U" (行撤销)。行撤销命令撤销所有在最近编辑的行上的操作。 输入该命令两次取消前一个 "U"：

```
A very intelligent turtle
  xxxx                  删除 very

A intelligent turtle
              xxxxxx    删除turtle

A intelligent
                        用 "U" 恢复行
A very intelligent turtle
                        用 "u" 撤销 "U"
A intelligent
```

"U" 命令本身就是一个改变操作，"u" 命令撤销该操作，CTRL-R 命令重做该操作。有点乱吧，但不用担心，用 "u" 和 CTRL-R 命令你可以切换到任何你编辑过的状态。更多信息可见 32.2 一节。

02.6 其它编辑命令

Vim 有大量的命令可以修改文本。参见 Q_in 和下文。这里是一些经常用到的:

添 加

"i" 命令在光标所在字符前面插入字符。一般情况下，这就够用了，但如果你刚好想在行尾加东西怎么办? 要解决这个问题，你需要在文本后插入字符。这通过 "a" (append，附加) 命令实现。

例如，要把如下行

```
and that's not saying much for the turtle.
```

改为

```
and that's not saying much for the turtle!!!
```

把光标移到行尾的句号上。然后输入 "x" 删除它。现在光标处于一行的尾部了，现在输入

```
a!!!<Esc>
```

添加三个感叹号到 turtle 的 "e" 后面:

```
and that's not saying much for the turtle!!!
```

开 始 一 个 新 行

"o" 命令在光标下方建立一个新的空行，并把 Vim 切换到插入模式。然后你可以在这个新行内输入文本。

假定你的光标在下面两行中第一行的某个地方:

```
A very intelligent turtle
Found programming UNIX a hurdle
```

如果你现在用 "o" 命令并输入新的文字:

```
oThat liked using Vim<Esc>
```

结果会是:

```
A very intelligent turtle
That liked using Vim
Found programming UNIX a hurdle
```

"O" 命令 (大写) 在光标上方打开一个新行。

指 定 计 数

假定你想向上移动 9 行，你可以输入 "kkkkkkkkk" 或者你可以输入"9k"。实际上，你可以在很多命令前面加一个数字。例如在这章的前面，你通过输入 "a!!!<Esc>" 增加三个感叹号。另一个方法是使用命令 "3a!<Esc>"。计数 3 要求把后面的命令执行三次。同样

的，要删除三个字符，可以使用 "3x"。计数总是放在要被处理多次的命令的前面。

02.7 退出

使用 "ZZ" 命令可以退出。这个命令保存文件并退出。

> 备注：
> 与其他编辑器不一样，Vim 不会自动建立一个备份文件。如果你输入 "ZZ"，你的修改立即生效并且不能恢复。你可以配置 Vim 让它产生一个备份文件；参见 07.4 。

放弃修改

有时你会做了一系列的修改才突然发现还不如编辑之前。不用担心，Vim 有 "放弃修改并退出" 的命令，那就是：

```
:q!
```

别忘了按回车使你的命令生效。

如果你关心细节，此命令有三部分组成：冒号 (:)，它使 Vim 进入命令模式， q 命令，它告诉 Vim 退出，而感叹号是强制命令修饰符。

这里，强制命令修饰符是必要的，它强制性地要求 Vim 放弃修改并退出。如果你只是输入 ":q"，Vim 会显示一个错误信息并拒绝退出：

```
E37: No write since last change (use ! to override)
```

通过指定强制执行，你实际上在告诉 Vim: "我知道我所做的看起来很傻，但我知道自己在做什么。"

如果你放弃修改后还想重新编辑，用 ":e!" 命令可以重新装载原来的文件。

02.8 寻求帮助

所有你想知道的东西，都可以在 Vim 帮助文件中找到答案，随便问！

如果你知道自己想要找什么，用帮助系统里查找通常比 Google 要方便。因为所有主题符合一定的风格指导。

帮助的另一个优点是对应于你特定的 Vim 系统。你不会看到之后加入的命令的帮助。这对你用不上。

要获得一般的帮助，用这个命令：

```
:help
```

你还可以用第一个功能键 <F1>。如果你的键盘上有一个 <Help> 键，可能也有效。

如果你不指定主题，":help" 将命令显示一个总览的帮助窗口。Vim 的作者在帮助系统方面使用了一个很聪明的方案 (也许可以说是很懒惰的方案)：他们用一个普通的编辑窗口来显示帮助。你可以在帮助窗口中使用任何普通的 Vim 命令移动光标。所以，h，j，k 和 l 还是表示左，下，上和右。

要退出帮助窗口，用退出一个普通窗口的命令："ZZ"。这只会退出帮助窗口，而不会退出 Vim。

当你阅读帮助的时候，你会发现有一些文字被一对竖线括起来了（例如 help ）。这表示一个超级链接。如果你把光标移到这两个竖线之间并按 CTRL-]（标签跳转命令），帮助系统会把你引向这个超级链接指向的主题。(由于不是本章的重点，这里不详细讨论，Vim 对超级链接的术语是 "标签"（tag），所以 CTRL-] 实际是跳转到光标所在单词为名的标签所在的位置。)

跳转几次以后，你可能想回到原来的地方。CTRL-T（标签退栈）把你送回前一个跳转点。CTRL-O（跳转到前一个位置）也能完成相同的功能。

在帮助屏幕的顶上，有这样的记号：*help.txt*。"*" 之间的名字被帮助系统用来定义一个标签（也就是超级链接的目标）。

参见 29.1 可以了解更多关于标签的内容。

要获得特定主题的帮助，使用如下命令：

```
:help {主题}
```

例如，要获得 "x" 命令的帮助，输入如下命令：

```
:help x
```

要知道如何删除文本，使用如下命令：

```
:help deleting
```

要获得所有命令的帮助索引，使用如下命令：

```
help index
```

如果你需要获得一个包含控制字符的命令的帮助（例如 CTRL-A），你可以在它前面加上前缀 "CTRL-"。

```
help CTRL-A
```

Vim 有很多模式。在默认情况下，帮助系统显示普通模式的命令。例如，如下命令显示普通模式的 CTRL-H 命令的帮助：

```
:help CTRL-H
```

要表示其他模式，可以使用模式前缀。如果你需要插入模式的命令帮助，使用 "i_" 前缀。例如对于 CTRL-H，你可以用如下命令：

```
:help i_CTRL-H
```

当你启动 Vim，你可以使用一些命令行参数。这些参数以短横线开头（-）。例如知道要 -t 这个参数是干什么用的，可以使用这个命令：

```
:help -t
```

Vim 有大量的选项让你定制这个编辑器。如果你要获得选项的帮助，你需要把它括在一个单引号中。例如，要知道 'number' 这个选项干什么的，使用如下命令：

```
:help 'number'
```

下面有所有模式的前缀列表： help-summary 。

特殊键以尖括号包围。例如，要找到关于插入模式的上箭头键的帮助，用此命令：

```
:help i_<Up>
```

如果你看到一个你不能理解的错误信息，例如：

```
E37: No write since last change (use ! to override) ~
```

你可以使用使用E开头的错误号找关于它的帮助：

```
:help E37
```

小结： help-summary

1) 键入主题后用 Ctrl-D 让 Vim 显示所有的可用主题。也可按 Tab 来补全：
   :help some<Tab>
   关于如何使用 help 的详情：
   :help helphelp

2) 跟随竖杠之间的链接转到相关帮助。可从详细帮助转到用户文档，这里的一些命令解释更加贴近用户，而不过于繁琐。例如：
   :help pattern.txt
   之后，你可以看看用户指南主题 03.9 和 usr_27.txt 相关的介绍。

3) 选项以单引号包围。如要转到 list 选项的帮助主题：
   :help 'list'
   如果你只知道你想找某个选项，也可用：
   :help options.txt
   来打开描述所有选项处理的帮助页面，然后用正规表达式搜索，如 textwidth。
   若干选项有自己的命名空间，例如：
   :help cpo-<letter>
   可查找 'cpoptions' 设置的相关标志位，把 <letter> 替代为特定的标志位，如：
   :help cpo-;
   而要查 guioption 的标志位：
   :help go-<letter>

4) 普通模式命令没有前缀。如要转到 "gt" 命令的帮助页面：
   :help gt

5) 插入模式命令以 i_ 开始。如关于删除单词的帮助：
   :help i_CTRL-W

6) 可视模式命令以 v_ 开始。如跳转到可视区域另一边的帮助：
   :help v_o

7) 命令行编辑和参数以 c_ 开始。如使用命令行参数 % 的帮助：
   :help c_%

8) Ex-命令总是以 ":" 开始，如要转到 ":s" 命令的帮助：
   :help :s

9) 专门用于调试的命令以 ">" 开始。如要转到 "cont" 调试命令的帮助：

```
:help >cont
```

10) 键组合。通常以指示要使用的模式的单个字母开始。例如：

```
:help i_CTRL-X
```

带你到插入模式下的 CTRL-X 命令的家族，可用于自动补全不同的事物。注意，一些特定的键总是以相同的方式书写，如 Control 总写作 CTRL。
普通模式命令没有前缀，相关主题可用 :h CTRL-<Letter> 找到，如

```
:help CTRL-W
```

与之作对比

```
:help c_CTRL-R
```

描述 CTRL-R 在命令行上输入命令时的行为，而

```
:help v_CTRL-A
```

讲述可视模式下数值的增量，而

```
:help g_CTRL-A
```

讲述 "g<C-A>" 命令（例如，你要先按 "g" 再按 <Ctrl-A>）。
这里 "g" 代表普通命令 "g"，它总期待按下第二个键后才能做事，"z" 开始的命令也类似

11) 正规表达式项目总是以 / 开始。如要得到 Vim 正规表达式的 "\+" 量词的帮助：

```
:help /\+
```

如果你需要知道所有关于正规表达式的情况，从这里开始：

```
:help pattern.txt
```

12) 寄存器总是以 "quote" 开始。如要了解特殊的 ":" 寄存器：

```
:help quote:
```

13) Vim 脚本可见

```
:help eval.txt
```

:h expr-X 描述语言的方方面面，其中 "X" 是单个字母。如

```
:help expr-!
```

会带你到描述 VimScript 中 "!"（取非）操作符的主题。
同样重要

```
:help function-list
```

提供所有可用函数的简短描述。Vim 脚本函数的帮助主题总包含 "()"，如：

```
:help append()
```

讲述 Vim 脚本的 append 函数，而不是如何在当前缓冲区内附加文本。

14) 帮助页面 :h map.txt 讲到映射。用

```
:help mapmode-i
```

来查找 :imap 命令。另 :map-topic 可用来查找关于映射的特定子主题，如：

```
:help :map-local
```

说明缓冲区局部的映射，或

```
:help map-bar
```

说明如何在映射中处理 '|'。

15) :h command-topic 讲述命令的定义，所以用

```
:help command-bar
```

可以了解关于自定义命令的 '!' 参数。

16) 窗口管理命令总是以 CTRL-W 开始，用 :h CTRL-W_letter 可以找到相应的帮助。如

```
:help CTRL-W_p
```

说明如何移动到上次访问过的窗口。也可访问

```
:help windows.txt
```

如果你在寻找窗口处理的命令，请仔细阅读。

17) 用 :helpgrep 在所有帮助页面中进行搜索（包括已安装的插件）。用法见 :helpgrep 。要搜索某主题：

```
:helpgrep topic
```

带你到首个匹配处。要到下一个：

```
:cnext
```

quickfix 窗口包含所有的匹配，可以这样打开：

```
:copen
```

先移到你要的匹配，按 Enter 跳转到该帮助。

18) 用户手册。以初学者友好的方式描述帮助主题。 usr_toc.txt 可以找到目录（你可能猜到了），就从这里开始吧：

```
:help usr_toc.txt
```

粗略看看内容，找找感兴趣的主题。如 "二合字母" 和 "插入特殊字符" 项目出现在第 24 章，所以要转到该帮助页面：

```
:help usr_24.txt
```

另外要访问帮助中的特定章节，可以这样直接访问章节号：

```
:help 10.1
```

会转到 usr_10.txt 的第 10.1 章，那里讲述记录宏。

19) 高亮组。总是以 hl-groupname 开始。如

```
:help hl-WarningMsg
```

讲述 WarningMsg 高亮组。

20) 语法高亮使用命名空间 :syn-topic。如

```
:help :syn-conceal
```

讲述 ":syn" 命令的 conceal 参数。

21) quickfix 命令通常以 :c 开始，而位置列表命令通常以 :l 开始

22) 自动命令事件可用其名字查找：

```
:help BufWinLeave
```

要列出所有可能的事件：

```
:help autocommand-events
```

23) 命令行开关总是以 "-" 开始。如关于 Vim 的 -f 命令开关的帮助，可用：

```
:help -f
```

24) 可选的特性总是以 "+" 开始。如了解 conceal 特性，可用：

```
:help +conceal
```

25) 关于系统包含的文件类型的特定功能的文档，通常所在位置的形式是 ft-<filetype>-<functionality>。如

```
:help ft-c-syntax
```

讲述 C 语法文件和它提供的选项。有时，全能补全有自己的段落

```
:help ft-php-omni
```

文件类型插件也是如此，如

```
:help ft-tex-plugin
```

可以参考。

26) 错误和警告代码可在帮助中直接查找。这样

:help E297
准确地带你到交换错误信息的描述，而
:help W10
讲述警告 "Changing a readonly file"。
不过，有时这些错误代码没有相应描述，而是和通常产生该错误的 Vim 命令列在一起。如:
:help E128
带你到 :function 命令

下一章: usr_03.txt 移动

版权: 参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: Nek_in http://vimcdoc.sf.net

移 动

在你插入或者删除之前，你需要移动到合适的位置。Vim 有一大堆命令可以移动光标。本章向你介绍最重要的那些。你可以在 Q_lr 下面找到这些命令的列表。

03.1 词移动
03.2 移动到行首或行尾
03.3 移动到指定的字符
03.4 括号匹配
03.5 移动到指定的行
03.6 确定当前位置
03.7 滚屏
03.8 简单查找
03.9 简单的查找模式
03.10 使用标记

下一章: usr_04.txt 做小改动
前一章: usr_02.txt Vim 初步
目录: usr_toc.txt

03.1 词移动

要移动光标向前跳一个词，可以使用 "w" 命令。像大多数 Vim 命令一样，你可以在命令前加数字前缀表示把这个命令重复多次。例如，"3w" 表示向前移动 3 个单词。用图表示如下:

```
This is a line with example text ~
  --->-->->----------------->
   w  w  w    3w
```

要注意的是，如果光标已经在一个单词的词首，"w" 移动到下一个单词的词首。
"b" 命令向后移动到前一个词的词首:

```
This is a line with example text
<----<--<-<---------<---
   b   b b    2b      b
```

还有一个 "e" 命令可以移到下一个单词的词末，而 "ge" 则移动到前一个单词的末尾:

```
This is a line with example text
   <-   <--- ----->   ---->
   ge    ge     e       e
```

如果你在一行的最后一个单词，"w" 命令将把你带到下一行的第一个单词。这样你可以用这个命令在一段中移动，这比使用 "l" 要快得多。"b" 则在反方向完成这个功能。

一个词以非单词字符结尾，例如 ".", "-" 或者 ")"。要改变 Vim 认为是单词组成部分

的字符，请参见 'iskeyword' 选项。如果你在此帮助文件里直接试验，先复位 'iskeyword'，此例才能工作：

```
:set iskeyword&
```

还可以用空白字符分隔的 "字串" (大写的 WORD) 移动。这不是我们通常意义的 "单词"。这就是为什么使用大写形式的 WORD 的原因。按字串移动的命令也全都是大写的，如下图所示：

```
        ge      b          w                              e
        <-     <-          --->                            --->
This is-a line, with special/separated/words (and some more).
   <----- <-----          ------------------->           ----->
     gE      B                    W                       E
        ge      b          w                              e
```

组合运用这些大写和小写的命令，你可以在段落内快速前后移动。

03.2 移动到行首或行尾

"$" 命令把光标移动到当前行行尾。如果你的键盘上有 <End> 键，也可以完成相同的功能。

"^" 命令把光标移动到一行的第一个非空字符，而 "0" 命令 (零) 则移到一行的第一个字符，<Home> 键也可以完成相同的功能。图示如下：

```
              ^
         <------------
.....This is a line with example text
<-----------------   --------------->
        0                   $
```

(这里 "....." 表示空白字符)

像大多数移动命令一样，"$" 命令接受计数前缀。但是 "移动到一行的行尾 n 次" 没有什么意义，所以它会使光标移动到另一行。例如，"1$" 移动到当前行的行尾，而 "2$" 则移动到下一行的行尾，如此类推。

"0" 命令不能加计数前缀，因为 "0" 本身就是个数字。而且，出人意料地是，"^" 命令使用计数前缀也没有任何效果。

03.3 移动到一个指定的字符

单字符查找命令是最有用的移动命令之一。"fx" 命令向前查找本行中的字符 x。提示："f" 代表 "Find" (寻找)。

例如，假定你在下行行首，而想移动到单词 "human" 的 h 那里。执行命令 "fh" 即可：

```
To err is human.  To really foul up you need a computer.
---------->--------------->
    fh          fy
```

这个例子里同时演示 "fy" 命令移动到了 "really" 的词尾。

你可以在这个命令前面加计数前缀，所以，你可以用 "3fl" 命令移动到 "foul" 的

"l":

```
To err is human.  To really foul up you need a computer.
          -------------------->
                   3fl
```

"F" 命令用于向左查找:

```
To err is human.  To really foul up you need a computer.
          <--------------------
                    Fh
```

"tx" 命令与 "fx" 相似，但它只把光标移动到目标字符的前一个字符上。提示: "t" 表示 "To" (到达)。这个命令的反向版本是 "Tx"。

```
To err is human.  To really foul up you need a computer.
           <------------  ------------>
                Th             tn
```

这四个命令可以通过 ";" 命令重复，"," 命令则用于反向重复。无论用哪个命令，光标永远都不会移出当前行，哪怕这两行是连续的一个句子。

有时你启动了一个查找命令后才发现自己执行了一个错误的命令。例如，你启动了一个 "f" 命令后才发现你本来想用的是 "F"。要放弃这个查找，输入 <Esc>。所以 "f<Esc>" 取消一个向前查找命令而不做任何操作。 备注: <Esc> 可以中止大部分命令，而不仅仅是查找。

03.4 括号匹配

当你写程序的时候，你经常会遇到嵌套的 () 结构。这时，"%" 是一个非常方便的命令: 它能匹配一对括号。如果光标在 "(" 上，它移动到对应的 ")" 上，反之，如果它在 ")" 上，它移动到 "(" 上。

```
                %
           <----->
if (a == (b * c) / d)
   <---------------->
                %
```

这个命令也可适用于 [] 和 {}。(可用 'matchpairs' 选项定义)

当光标不在一个有用的字符上，"%" 会先正向查找找到一个。比如当光标停留在上例中的行首时，"%" 会正向查找到第一个 "("。再按一次会移动到它的匹配处。

```
if (a == (b * c) / d)
---+---------------->
                %
```

03.5 移动到指定的行

如果你是一个 C 或者 C++ 程序员，你对下面这样的错误信息应该非常熟悉:

```
prog.c:33: j   undeclared (first use in this function)
```

这表示你可能要移动到 33 行去作一些修改了。那么怎样找到 33 行？一个办法是执行 "9999k" 命令移到文件头，再执行 "32j" 下移 32 行。这不是一个好办法，但肯定有效。更好的方法是使用 "G" 命令。加上一个计数前缀，这个命令可以把你送到指定的行。例如，"33G" 把你送到 33 行。(要用更好的方法在编译器的错误列表中移动，参见 usr_30.txt 的 ":make" 命令部分。)

如果没有计数前缀，"G" 命令把光标移动到文件末。移动到文件首的命令是 "gg"。"1G" 也能完成这个功能，但稍复杂一点。

```
    |   first line of a file   ^
    |   text text text text    |
    |   text text text text    |  gg
7G  |   text text text text    |
    |   text text text text
    |   text text text text
    V   text text text text    |
        text text text text    |  G
        text text text text    |
        last line of a file    V
```

另一个定位行的方法是使用带计数前缀的 "%" 命令。例如，"50%" 移动到文件的中间，而 "90%" 移到差不多结尾的位置。

前面的描述假定你想移动到文件中的某一行，而不在乎它是否可见。那么如何移动到视野之内的一行呢？下图演示了三个可以使用的命令：

```
                +--------------------------+
H -->   | text sample text          |
        | sample text               |
        | text sample text          |
        | sample text               |
M -->   | text sample text          |
        | sample text               |
        | text sample text          |
        | sample text               |
L -->   | text sample text          |
        +--------------------------+
```

提示：“H” 表示 Home (头)，“M” 表示 Middle (中) 而 “L” 表示 Last (尾)。另外一种记法，“H” 表示 High (高)，“M” 表示 Middle (中) 而 “L” 表示 Low (低)。

03.6 确定当前位置

要确定你在文件中的位置，有三种方法：

1. 使用 CTRL-G 命令，你会获得如下消息 (假定 'ruler' 选项已经被关闭)：

   ```
   "usr_03.txt" line 233 of 650 --35%-- col 45-52
   ```

   这里显示了你正在编辑的文件的名称，你所处的当前行的行号，全文的总行数，光标以前的行占全文的百分比，和你所处的列的列号。

有时你会看到一个分开的两个列号。例如，"col 2-9"。这表示光标处于第二个字符上，但由于使用了制表符，在屏幕上的位置是 9。

2. 置位 'number' 选项。这会在每行的前面加上一个行号：

```
:set number
```

要重新关闭这个选项：

```
:set nonumber
```

由于 'number' 是一个布尔类型的选项，在它前面加上 "no" 表示关闭它。布尔选项只会有两个值，on 或者 off。

Vim 有很多选项，除了布尔类型的，还有数值或者字符串类型的。在用到的时候会给出一些例子的。

3. 置位 'ruler' 选项。这会在 Vim 窗口的右下角显示当前光标的位置：

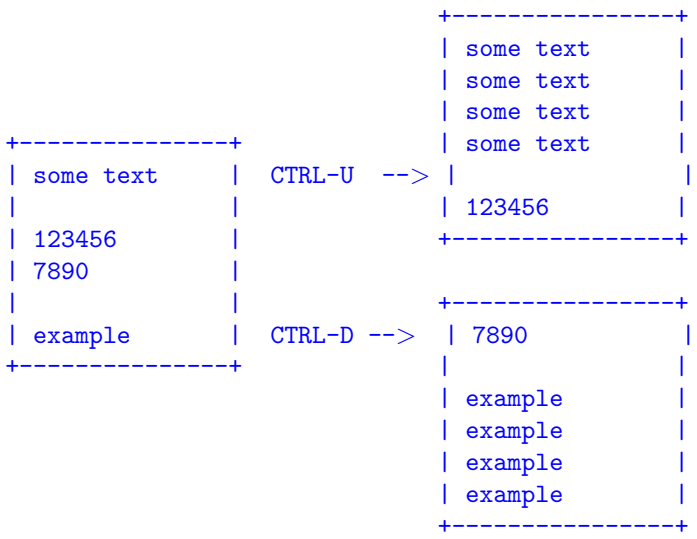
```
:set ruler
```

使用 'ruler' 的好处是它不占多少地方，从而可以留下更多的地方给你的文本。

03.7 滚屏

CTRL-U 命令向下滚动半屏。想象一下通过一个视窗看着你的文本，然后把这个视窗向上移动该窗口的一半高度。这样，窗口移动到当前文字的上面，而文字则移到窗口的下面。不用担心记不住那边是上。很多人都是这样。

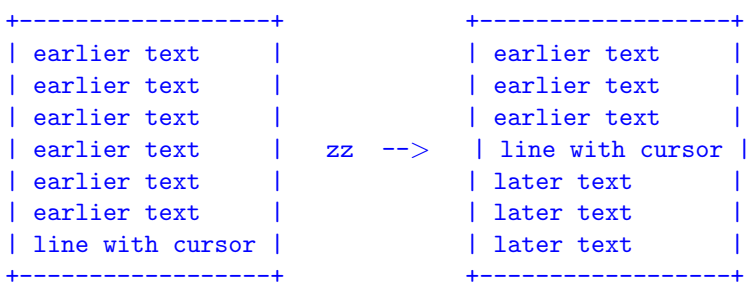
CTRL-D 命令把视窗向下移动半屏，所以把文字向上移动半屏。



每次滚一行的命令是 CTRL-E（上滚）和 CTRL-Y（下滚）。可以把 CTRL-E 想象为是多给你一行（one line Extra）。

正向滚动一整屏的命令是 CTRL-F（减去两行）。反向的命令是 CTRL-B。CTRL-F 是向前（forward）滚动，CTRL-B 是向后（backward）滚动，这比较好记。

移动中的一个常见问题是，当你用 "j" 向下移动的时候，你的光标会处于屏幕的底部，你可能希望，光标所在行处于屏幕的中间。这可以通过 "zz" 命令实现。



"zt" 把光标所在行移动到屏幕的顶部，而 "zb" 则移动到屏幕的底部。Vim 中还有另外一些用于滚动的命令，可以参见 Q_sc 。要使光标上下总保留有几行处于视窗中用作上下文，可以使用 'scrolloff' 选项。

03.8 简单查找

查找命令是 "/String"。例如，要查找单词 "include"，使用如下命令：

```
/include
```

你会注意到，输入 "/" 时，光标移到了 Vim 窗口的最后一行，这与 "冒号命令" 一样，在那里你可以输入要查找的字符串。你可以使用退格键（退格箭头或 <BS>）进行修改，如果需要的时候还可以使用 <Left> 和 <Right> 键。
使用 <Enter> 开始执行这个命令。

> 备注：
> 字符 .*[]^%/\?~$ 有特殊含义。如果你要查找它们，需要在前面加上一个 "\"。请参见下文。

要查找下一个匹配可以使用 "n" 命令。用下面命令查找光标后的第一个 #include：

```
/#include
```

然后输入 "n" 数次。你会移动到其后每一个 #include。如果你知道你想要的是第几个，可以在这个命令前面增加计数前缀。这样，"3n" 表示移动到第三个匹配点。要注意，"/" 不支持计数前缀。

"?" 命令功能与 "/" 的功能类似，但进行反方向查找：

```
?word
```

"N" 命令在反方向重复前一次查找。因此，在 "/" 命令后执行 "N" 命令是反向查找，在 "?" 命令后执行 "N" 命令是正向查找。

忽略大小写

通常，你必须区分大小写地输入你要查找的内容。但如果你不在乎大小写。可以设置 'ignorecase' 选项：

```
:set ignorecase
```

如果你现在要查找 "word"，它将匹配 "word" 和 "WORD"。如果想再次区分大小写：

```
:set noignorecase
```

历 史 记 录

假设你执行了三个查找命令：

```
/one
/two
/three
```

现在，让我们输入 "/" 启动一次查找，但先不按下回车键。现在按 <Up>（上箭头），Vim 把 "/three" 放到你的命令行上。回车就会从当前位置查找 "three"。如果你不回车，继续按 <Up>，Vim 转而显示 "/two"，而下一次 <Up> 变成 "/one"。

你还可以用 <Down> 命令在历史记录中反向查找。

如果你知道前面用过的一个模式以什么开头，而且你想再使用这个模式的话，可以在输入 <Up> 前输入这个开头。继续前面的例子，你可以输入 "/o<Up>"，Vim 就会在命令行上显示 "/one"。

冒号开头的命令也有历史记录。这允许你取回前一个命令并再次执行。这两种历史记录是相互独立的。

在 文 本 中 查 找 一 个 单 词

假设你在文本中看到一个单词 "TheLongFunctionName" 而你想找到下一个相同的单词。你可以输入 "/TheLongFunctionName"，但这要输入很多东西。而且如果输错了，Vim 是不可能找到你要找的单词的。

有一个简单的方法：把光标移到那个单词下面使用 "*" 命令。Vim 会取得光标上的单词并把它作为被查找的字符串。

"#" 命令在反向完成相同的功能。你可以在命令前加一个计数："3*" 查找光标下单词第三次出现的地方。

查 找 整 个 单 词

如果你输入 "/the"，你也可能找到 "there"。要找到以 "the" 结尾的单词，可以用：

```
/the\>
```

"\>" 是一个特殊的记号，表示只匹配单词末尾。类似地，"\<" 只匹配单词的开头。这样，要匹配一个完整的单词 "the"，只需：

```
/\<the\>
```

这不会匹配 "there" 或者 "soothe"。注意 "*" 和 "#" 命令也使用了 "词首" 和 "词尾" 标记来匹配整个单词（要部分匹配，使用 "g*" 和 "g#"）

高亮匹配

当你编辑一个程序的时候，你看见一个变量叫 "nr"。你想查一下它在哪被用到了。你可以把光标移到 "nr" 下用 "*" 命令，然后用 n 命令一个个遍历。

这里还有一种办法。输入这个命令：

```
:set hlsearch
```

现在如果你查找 "nr"，Vim 会高亮显示所有匹配的地方。这是一个很好的确定变量在哪被使用，而不需要输入更多的命令的方法。

要关掉这个功能：

```
:set nohlsearch
```

这样做，下一次查找时你又需要切换回来。如果你只是想去掉高亮显示，用如下命令：

```
:nohlsearch
```

这不会复位 hlsearch 选项。它只是关闭高亮显示。当你执行下一次查找的时候，高亮功能会被再次激活。使用 "n" 和 "N" 命令时也一样。

调节查找方式

有一些选项能改变查找命令的工作方式。其中有几个是最基本的：

```
:set incsearch
```

这个命令使 Vim 在你输入字符串的过程中就显示匹配点。用这个功能可以检查是否会被找到正确的匹配，这时输入 <Enter> 就可以真正地跳到那个地方。否则，继续输入更多的字符可以修改要查找的字符串。

```
:set nowrapscan
```

这个设置使得找到文件结尾后停止查找。或者当你往回查找的时候遇到文件开头停止查找。默认情况下 'wrapscan' 的状态是 "on"。所以在找到文件尾的时候会自动折返到文件头。

插曲

如果你喜欢前面的选项，而且每次用 Vim 都要设置它，那么，你可以把这些命令写到 Vim 的启动文件中。

编辑 not-compatible 中提到的文件，或者用如下命令确定这个文件在什么地方：

```
:scriptnames
```

编辑这个文件，例如，像下面这样：

```
:edit ~/.vimrc
```

然后在文中加一行命令来设置这些选项，就好像你在 Vim 中输入一样，例如：

```
Go:set hlsearch<Esc>
```

"G" 移动到文件的结尾，"o" 开始一个新行，然后你在那里输入 ":set" 命令。
最后你用 <Esc> 结束插入模式。然后用如下命令存盘并关闭文件：

```
ZZ
```

现在如果你重新启动 Vim，'hlsearch' 选项就已经被设置了。

03.9 简单的查找模式

Vim 用正则表达式来定义要查找的对象。正则表达式是一种非常强大和紧凑的定义查找模式的方法。但是非常不幸，这种强大的功能是有代价的，因为使用它需要掌握一些技巧。

本章我们只介绍一些基本的正则表达式。要了解更多的关于查找模式和命令，请参考第 27 章 usr_27.txt 。你还可以在 pattern 中找到正则表达式的完整描述。

行 首 与 行 尾

^ 字符匹配行首。在美式英文键盘上，它在数字键 6 的上面。模式 "include" 匹配一行中任何位置的单词 include。而模式 "^include" 仅匹配在一行开始的 include。

$ 字符匹配行尾。所以，"was$" 仅匹配在行尾的单词 was。

我们在下面的例子中用 "x" 标记出被 "/the" 模式匹配的位置：

```
the solder holding one of the chips melted and the
xxx                       xxx                   xxx
```

用 "/the$" 则匹配如下位置：

```
the solder holding one of the chips melted and the
                                                  xxx
```

而使用 "/^the" 则匹配：

```
the solder holding one of the chips melted and the
xxx
```

你还可以试着用这个模式："/^the$"；它只会匹配仅包括 "the" 的行。并且不包括空格。例如包括 "the " 的行是不会被这个模式匹配的。

匹 配 任 何 单 个 字 符

点 "." 字符匹配任何字符。例如，模式 "c.m" 匹配一个字符串，它的第一个字符是 c，第二个字符是任意字符，而第三个字符是 m。例如：

```
We use a computer that became the cummin winter.
        xxx             xxx      xxx
```

匹 配 特 殊 字 符

如果你确实想匹配点字符，可以在前面加一个反斜杠去消除它的特殊含义。

如果你用 "ter." 模式去查找，会匹配这些地方：

```
We use a computer that became the cummin winter.
               xxxx                          xxxx
```

但如果你查找 "ter\."，只会匹配第二个位置。

03.10 使用标记

当你用 "G" 命令跳到另一个地方，Vim 会记住你从什么地方跳过去的。这个位置成为一个标记，要回到原来的地方，使用如下命令：

```
``
```

` 是反引号，用单引号 ' 也可以。

如果再次执行这个命令你会跳回去原来的地方，这是因为 "``" 命令本身是个跳转，它记住了自己跳转前的位置。

一般，每次你执行一个会将光标移动到本行之外的命令，该移动即被称为一个 "跳转" 。这包括查找命令 "/" 和 "n" (无论跳转到多远的地方)。但不包括 "fx" 和 "tx" 这些行内查找命令或者 "w" 和 "e" 等词移动命令。

另外 "j" 和 "k" 不会被当做是一次 "跳转"，即使你在前面加上计数前缀使之移动到很远的地方也不例外。

"``" 命令可以在两个位置上跳来跳去。而 CTRL-O 命令则跳到一个 "较老" 的地方 (提示： O 表示 older)。CTRL-I 则跳到一个 "较新" 的地方 (提示：在很多常见的键盘布局上，I 在键盘上紧靠着 O)。考虑如下命令序列：

```
33G
/^The
CTRL-O
```

你首先跳到第 33 行，然后查找以 "The" 开头的一行，然后用 CTRL-O 你会跳回到 33 行。再执行 CTRL-O 你会跳到最初的地方。现在你使用 CTRL-I，就又跳到 33 行。而再用一次 CTRL-I 你又会到达找到 "The" 的地方。

```
     |  example text    ^              |
33G  |  example text    |  CTRL-O      | CTRL-I
     |  example text    |              |
     V  line 33 text    ^              V
     |  example text    |              |
/^The |  example text    |  CTRL-O      | CTRL-I
     V  There you are   |              V
        example text
```

备注：
CTRL-I 的功能与 <Tab> 一样。

":jumps" 命令能输出一个你可以跳往的位置的列表。最后一个你使用的标记会用 ">" 符号标记出来。

有 名 字 的 标 记 bookmark

Vim 允许你在文本中放置自定义的标记。命令 "ma" 用 a 标记当前的光标位置。你可以在文本中使用 26 个标记 (a 到 z)。这些标记是不可见的，只是一个由 Vim 记住的位置。

要跳到一个你定义的标记，可以使用命令 `{mark}，这里 {mark} 是指定义标记的那个字母。所以，移到标记 a 的命令是：

```
`a
```

命令 "'mark" (单引号加上一个标记) 会移到标记所在行的行首。这与 "`mark" 命令是不同的，后者同时移到标记标记的列上。

标记在需要处理一个文件的两个相关地方的时候非常有用。假设你在处理文件末的时候需要查看文件首的一些内容。

先移动到文件首，设置一个标记 s (start，开始)：

```
ms
```

然后移动到你需要处理的地方，再设置一个标记 e (end，结束)：

```
me
```

现在你可以随意移动，当你需要看开头的地方，可以使用这个命令移到那里：

```
's
```

然后使用 '' 跳回来。或者用 'e 跳到你正在处理的文件尾部的地方。

这里使用 s 和 e 作标记名没有特别的含义，只是为了好记而已。

你可以用如下命令取得所有的标记的列表：

```
:marks
```

你会注意到有一些特殊的标记，包括：

```
'       跳转前的光标位置
"       最后编辑的光标位置
[       最后修改的开始位置
]       最后修改的结束位置
```

下一章： usr_04.txt 作小改动

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

作小改动

本章介绍几种修正和移动文本的方法，这包括三种修改文本的基本方法：操作符－动作，可视模式以及文本对象。

04.1 操作符与动作
04.2 改变文本
04.3 重复一个修改
04.4 可视模式
04.5 移动文本
04.6 拷贝文本
04.7 使用剪贴板
04.8 文本对象
04.9 替换模式
04.10 结论

下一章： usr_05.txt 选项设置
前一章： usr_03.txt 移动
目录： usr_toc.txt

04.1 操作符与动作

在第二章你已经学过使用 "x" 命令来删除一个字符以及通过计数前缀，例如 "4x" 去删除多个字符。

"dw" 命令删除一个单词。你可能认出来了，"w" 是词移动命令。实际上，"d" 命令后面可以跟任何 "动作" (motion) 命令，它会删除从当前位置到光标移动到的目标位置的全部内容。

例如 "4w" 命令能够向后移动四个单词。所以 "d4w" 命令删除 4 个单词。

```
To err is human. To really foul up you need a computer.
                 ------------------>
                        d4w

To err is human. you need a computer.
```

Vim 只删除从当前位置到 "动作" 把光标移动到的位置的前一个位置。这是因为 Vim 认为你可能不想删掉一个单词的第一个字符。如果你用 "e" 命令作为动作来移动到单词结尾，Vim 这时认为你是想删掉整个单词 (包括最后一个字符)：

```
To err is human. you need a computer.
                 -------->
                    d2e

To err is human. a computer.
```

是否包括光标所在的字符取决于你使用的移动命令。在参考手册中，当不包括这个字符时，称为 "非包含的" (exclusive)，而包括这个字符时，称为 "包含的" (inclusive)。

"$" 命令移动到行尾。所以，"d$" 命令从当前的位置一直删除到本行行尾。这是一个"包含的" 命令，所以，这行的最后一个字符也会被删除：

```
To err is human. a computer.
              ------------>
                   d$

To err is human
```

以上定义了一个命令组合模式：操作符－动作。你首先输入一个操作符命令，例如，"d" 就是一个删除操作符。然后你输入一个动作命令，例如 "4l" 或者 "w"。这种方法使你可以在任何你能越过的文本上执行各种操作。

04.2 修改文本

另一个操作符命令是 "c"，表示修改，change。它的作用方式与 "d" 操作符相似，只是完成后会切换到插入模式。例如，"cw" 修改一个词，更精确的说，它删除一个词，并切换到插入模式。

```
To err is human
   ------->
     c2wbe<Esc>

To be human
```

这里 "c2wbe<Esc>" 包括如下操作：

c	修改操作符
2w	移动两个单词的距离（与操作符合起来，它删除两个单词并进入插入模式）
be	插入 be 这个单词
<Esc>	切换回普通模式

你会发现一个奇怪的地方：human 前面的空格没有被删除。有一句谚语说道：任何问题都有一个简单，清楚但错误的回答。"cw" 命令就属于这种情况。c 操作符在很多地方都和 d 一样，但有一个例外，"cw"。它实际上像 "ce" 一样，删除到单词尾。这样单词后面的空格就不包括在内了。这要追溯到使用 Vi 的旧日子。由于很多人已经习惯了这种方式，这个不一致之处就留在 Vim 里了。

更多的修改命令

像 "dd" 可以删除一行一样，"cc" 修改一整行。但它会保留这一行的缩进（前导空格）。

"d$" 删除到行尾；"c$" 则修改到行尾。这相当于先用 "d$" 删除一行再用 "a" 启动插入模式，以便加入新的文字。

快捷键

有些 "操作符－动作" 命令由于经常被使用，所以被设置为单字符命令：

```
x  表示  dl  (删除当前光标下的字符)
X  表示  dh  (删除光标左边的字符)
D  表示  d$  (删除到行尾)
C  表示  c$  (修改到行尾)
s  表示  cl  (修改一个字符)
S  表示  cc  (修改一整行)
```

在什么地方加入计数前缀

命令 "3dw" 和 "d3w" 都是删除 3 个单词。如果你非要寻根问底，那么："3dw" 表示删除一个单词 3 次，而 "d3w" 表示删除三个单词一次。这是一个没有分别的分别。实际上你可以放两个计数前缀，例如，"3d2w" 删除两个单词三次，共计六个单词。

替换单个字符

"r" 命令不是操作符。它只是等你输入一个字符然后用这个字符替换当前光标上的字符。你可以用 "cl" 命令或者 "s" 命令完成相同的功能，但 "r" 命令不需要使用 <Esc> 退出插入状态：

```
there is somerhing grong here
rT           rt    rw

There is something wrong here
```

通过计数前缀，"r" 命令可以使多个字符被同一个字符替换，例如：

```
There is something wrong here
                  5rx

There is something xxxxx here
```

要用换行符替换一个字符可以用命令 "r<Enter>"。这会删除一个字符并插入一个换行符。在这里使用计数前缀会删除多个字符但只插入一个换行符："4r<Enter>" 用一个换行符替换四个字符。

04.3 重复一个修改

"." 是 Vim 中一个非常简单而有用的命令。它重复最后一次的修改操作。例如，假设你在编辑一个 HTML 文件，你想删除所有的 <B> 标记。你把光标移到第一个 "<" 上，然后用 "df>" 命令删除 <B>。然后你就可以移到 </B> 的 < 上面用 "." 命令删除它。"." 命令执行最后一次的修改命令 (在本例中，就是 "df>")。要删除下一个 <B> 标记，移动到下一个 < 的位置，再执行 "." 命令即可。

```
                          To <B>generate</B> a table of <B>contents
f<   找第一个 <            --->
df>  删除到 >                 -->
f<   找下一个 <                 --------->
.    重复 df>                          --->
f<   找下一个 <                             ------------->
.    重复 df >                                        -->
```

"." 命令重复任何除 "u" (撤销), CTRL-R (重做) 和冒号命令外的修改。

再举一个例子: 你想把 "four" 修改成 "five"。有好几个地方都要作这种修改。你可以用如下命令快速完成这个操作:

```
/four<Enter>     找到第一个 "four"
cwfive<Esc>      修改成 "five"
n                找下一个 "four"
.                重复修改到 "five" 的操作
n                找下一个 "four"
.                重复修改
                 如此类推......
```

04.4 可视模式

要删除一些简单的东西, 用 "操作符－动作" 命令可以完成得很好。但很多情况下, 并不容易确定用什么命令可以移到你想修改的地方。这时候, 你就需要可视模式了。

你可以用 "v" 命令启动可视模式。你可以移动光标到需要的地方。当你这样做的时候, 中间的文本会被高亮显示。最后执行一下 "操作符" 命令即可。

例如, 要从一个单词的中间删除到下一个单词的中间:

```
This is an examination sample of visual mode
          ---------->
            velllld

This is an example of visual mode
```

但你这样做的时候, 你不需要真的算要按 l 多少次, 你可以在按 "d" 前清楚得看到将要被删除的是哪些文本。

如果任何时候你改了主意, 只用按一下 <Esc> 就能退出可视模式。

按 行 选 择

如果你想对整行做操作, 可以使用 "V" 命令来启动可视模式。你会发现在你作任何移动之前, 整行都被高亮显示了。左右移动不会有任何效果。而通过上下移动, 你可以一次选择多行。

例如, 用 "Vjj" 可以选中三行:

```
                +------------------------+
                | text more text         |
              >> | more text more text    | |
选中的行      >> | text text text         | | Vjj
              >> | text more              | V
                | more text more         |
                +------------------------+
```

列 块 选 择

如果你要处理一个矩形块内的文本, 可以使用 CTRL-V 启动可视模式。这在处理表格时非常有用。

```
name            Q1      Q2      Q3
pierre          123     455     234
john            0       90      39
steve           392     63      334
```

要删除中间 "Q2" 这一栏，把光标移动到 "Q2" 的 "Q" 上面。按 CTRL-V 启动列块可视模式。现在用 "3j" 向下移动三行，然后用 "w" 移到下一个单词。你可以看到最后一栏的第一个字符也被包括进来了。要去掉它，用 "h" 命令即可。现在按 "d"，中间一栏就被删除了。

移 动 到 另 一 端

如果你在可视模式下选中了一些文字，然后你又发现你需要改变被选择的文字的另一端，用 "o" 命令即可 (提示: "o" 表示 other end)，光标会移动到被选中文字的另一端，现在你可以移动光标去改变选中文字的开始点了。再按 "o" 光标还会回到另一端。

当使用列块可视模式的时候，你会有四个角，"o" 只是把你移到对角上。而用 "O" 则能移到同一行的另一个角上。

备注: "o" 和 "O" 在可视模式下与在普通模式下的作用有很大的不同；在普通模式下，它们的作用是在光标后或前加入新的一行。

04.5 移动文本

当你用 "d"，"x" 或者其它命令删除文本的时候，这些文字会被存起来。你可以用 p 命令重新粘贴出来 (p 在 Vim 中表示 put，放置)。

看看下面的例子。首先，你会在你要删除的那一行上输入 "dd" 删除一整行，然后移动到你要重新插入这行的地方输入 "p" (put)，这样这一行就会被插入到光标下方。

```
a line          a line          a line
line 2    dd    line 3    p     line 3
line 3                          line 2
```

由于你删除的是一整行，"p" 命令把该行插入到光标下方。如果你删除的是一行的一部分 (例如一个单词)，"p" 命令会把它插入到光标的后面。

```
Some more boring try text to out commands.
                  ---->
                   dw

Some more boring text to out commands.
                  -------->
                     welp

Some more boring text to try out commands.
```

关 于 粘 贴 的 更 多 知 识

"P" 命令像 "p" 一样也是插入字符，但插入点在光标前面。当你用 "dd" 删除一行，"P" 会把它插入到光标所在行的前一行。而当你用 "dw" 删除一个单词，"P" 会把它插入

到光标前面。

你可以执行这个命令多次，每次会插入相同的文本。

"p" 和 "P" 命令接受计数前缀，被插入的文本就会被插入指定的次数。所以 "dd" 后加一个 "3p" 会把删除行的三个拷贝插入到文本中。

交换两个字符

经常发生这样的情况，当你输入字符的时候，你的手指比脑子转得快（或者相反？）。这样的结果是你经常把 "the" 敲成 "teh"。Vim 让你可以很容易得修正这种错误。只要把光标移到 "teh" 的 "e" 上，然后执行 "xp" 即可。这个工作过程是："x" 删除一个字符，保存到寄存器。"p" 把这个被保存的字符插入到光标的后面，也就是在 "h" 的后面了。

```
teh     th     the
 x       p
```

04.6 拷贝文本

要把文本从一个地方拷贝到另一个地方，你可以先删除它，然后用 "u" 命令恢复，再用 "p" 拷到另一个地方。这里还有一种简单的办法：抽出（yank）。"y" 命令可以把文字拷贝到寄存器中。然后用 "p" 命令粘贴到别处。

yanking 是 Vim 中拷贝命令的名字。由于 "c" 已经被用于表示 change 了，所以拷贝（copy）就不能再用 "c" 了。但 "y" 还是可用的。把这个命令称为 "yanking" 是为了更容易记住 "y" 这个键。（译者注：这里只是把原文译出以作参考，"抽出" 文本毕竟是不妥的。后文中将统一使用 "拷贝"。中文可不存在 change 和 copy 的问题。）

由于 "y" 是一个操作符，所以 "yw" 命令就是拷贝一个单词了。当然了，计数前缀也是有效的。要拷贝两个单词，就可以用 "y2w"。例如：

```
let sqr = LongVariable *
          -------------->
                y2w

let sqr = LongVariable *
                        p

let sqr = LongVariable * LongVariable
```

注意："yw" 命令包括单词后面的空白字符。如果你不想要这个字符，改用 "ye" 命令。

"yy" 命令拷贝一整行，就像 "dd" 删除一整行一样。出乎意料地是，"D" 删除到行尾而 "Y" 却是拷贝一整行。要注意这个区别！"y$" 拷贝到行尾。

```
a text line   yy      a text line          a text line
line 2                line 2         p     line 2
last line             last line            a text line
                                           last line
```

04.7 使用剪贴板

如果你使用 Vim 的 GUI 版本 (gvim)，你可以在 "Edit" 菜单中找到 "Copy" 项。你可以先用可视模式选中一些文本，然后使用 Edit/Copy 菜单项目。现在被选中的文本被拷进了剪贴板。你可以把它粘贴到其它程序，或者在 Vim 内部使用。

如果你已经从其它程序中拷贝了一些文字到剪贴板，你可以在 Vim 中用 Edit/Paste 菜单项目粘贴进来，这在普通模式和插入模式中都是有效的。如果在可视模式，被选中的文字会被替换掉。

"Cut" 菜单项会在把文字拷进剪贴板前删除它。"Copy"，"Cut" 和 "Paste" 命令在弹出菜单中也有 (当然了，前提是有弹出式菜单)。如果你的 Vim 有工具条，在工具条上也能找到这些命令。

如果你用的不是 GUI，或者你根本不喜欢用菜单，你只能用其它办法了。你还是可以用普通的 "y" (yank) 和 "p" (put) 命令，但在前面必须加上 "* (一个双引号加一个星号)。例如，要拷贝一行到剪贴板中:

```
"*yy
```

要粘贴回来:

```
"*p
```

这仅在支持剪贴板的 Vim 版本中才能工作。关于剪贴板的更多内容请参见 09.3 和 clipboard 。

04.8 文本对象

如果你在一个单词的中间而又想删掉这个单词，在你用 "dw" 前，你必须先移到这个单词的开始处。这里还有一个更简单的方法: "daw"。

```
this is some example text.
              daw

this is some text.
```

"daw" 的 "d" 是删除操作符。"aw" 是一个文本对象。提示: "aw" 表示 "A Word" (一个单词)，这样，"daw" 就是 "Delete A Word" (删除一个单词)。确切地说，该单词后的空格字符也被删除掉了 (或者如果在行尾的话，单词前的空格字符)。

使用文本对象是 Vim 中执行修改的第三种方法。我们已经有 "操作符－动作" 和可视模式两种方法了。现在我们又有了 "操作符－文本对象"。

这种方法与 "操作符－动作" 很相似，但它不是操作于从当前位置到移动目标间的内容，而是对光标所在位置的 "文本对象" 进行操作。文本对象是作为一个整体来处理的。现在光标在对象中的位置无关紧要。

用 "cis" 可以改变一个句子。看下面的句子:

```
Hello there.  This
is an example.  Just
some text.
```

移动到第二行的开始处。现在使用 "cis":

```
Hello there.    Just
some text.
```

现在你输入新的句子 "Another line.":

```
Hello there.  Another line.  Just
some text.
```

"cis" 包括 "c" (change，修改) 操作符和 "is" 文本对象。这表示 "Inner Sentence" (内含句子)。还有一个文本对象是 "as" ("A Sentence")，区别是 "as" 包括句子后面的空白字符而 "is" 不包括。如果你要删除一个句子，而且你还想同时删除句子后面空白字符，就用 "das"；如果你想保留空白字符而替换一个句子，则使用 "cis"。

你还可以在可视模式下使用文本对象。这样会选中一个文本对象，而且继续留在可视模式，你可以继续多次执行文本对象命令。例如，先用 "v" 启动可视模式，再用 "as" 就可以选中一个句子。现在重复执行 "as"，就会继续选中更多的句子。最后你可以使用一个操作符去处理这些被选中的句子。

你可以在这里找到一个详细的文本对象的列表： text-objects 。

04.9 替换模式

"R" 命令启动替换模式。在这个模式下，你输入的每个字符都会覆盖当前光标上的字符。这会一直持续下去，直到你输入 <Esc>。

在下面的例子中，你在 "text" 的第一个 "t" 上启动替换模式：

```
This is text.
        Rinteresting.<Esc>

This is interesting.
```

你可能会注意到，这是用十二个字符替换一行中的五个字符。如果超出行的范围，"R" 命令自动进行行扩展，而不是替换到下一行。

你可以通过 <Insert> 在插入模式和替换模式间切换。

但当你使用 <BS> (退格键) 进行修正时，你会发现原来被替换的字符又回来了。这就好像一个 "撤销" 命令一样。

04.10 结论

操作符，移动命令和文本对象可以有各种组合。现在你已经知道它们是怎么工作了，你可以用 N 个操作符加上 M 个移动命令，组合出 N*M 个命令！

你可以在这里找到一个操作符的列表： operator 。

还有很多方法可以删除文本。这是一些经常用到的：

x　　删除光标下的字符 ("dl" 的缩写)

X	删除光标前的字符 ("dh" 的缩写)
D	从当前位置删除到行尾 ("d$" 的缩写)
dw	从当前位置删除到下一个单词开头
db	从当前位置删除到前一个单词的开头
diw	删除光标上的单词 (不包括空白字符)
daw	删除光标上的单词 (包括空白字符)
dG	删除到文件末
dgg	删除到文件首

如果你用 "c" 代替 "d"，这会变成修改命令；而改用 "y"，则变成拷贝命令，等等等等。

还有一些常用的进行修改的命令，放在哪一章都不合适，列在这里：

~	修改光标下字符的大小写，并移动到下一个字符。这不是一个操作符 (除非设置了 'tildeop')，所以你不能连接一个动作命令。这个命令在可视模式下也有效，它会改变被选中的所有文本的大小写。
I	移到当前行的第一个非空字符并启动插入模式。
A	移动到行尾并启动插入模式。

下一章： usr_05.txt 设置选项

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in，tocer http://vimcdoc.sf.net

选项设置

Vim 可以按你的需要进行设置。本章告诉你怎样使 Vim 用你指定的选项启动，怎样增加插件以增强 Vim 的功能；以及怎样进行宏定义。

05.1 vimrc 文件
05.2 vimrc 示例解释
05.3 defaults.vim 文件解释
05.4 简单键盘映射
05.5 添加软件包
05.6 添加插件
05.7 添加帮助
05.8 选项窗口
05.9 常用选项

下一章：usr_06.txt 使用语法高亮
前一章：usr_04.txt 做小改动
目录：usr_toc.txt

05.1 vimrc 文件 vimrc-intro

可能你已经厌倦了输入那些经常用到的命令了。要让 Vim 用你习惯的设置启动，你可以把这些设置写到一个叫 vimrc 的文件中。Vim 会在启动的时候执行这个文件里的命令。

如果你已经有 vimrc 文件（例如，系统管理员已经为你配置好了），可以这样编辑它：

```
:edit $MYVIMRC
```

如果你还没有 vimrc 文件，请参考 vimrc 一节看看你应该在什么地方创建该文件。另外 ":version" 命令能告诉你 vim 要查找的 "用户 vimrc 文件" 的名字。

对于 Unix 和 Macintosh 系统，总是使用而且也推荐使用如下文件：

```
~/.vimrc
```

对于 MS-DOS 和 MS-Windows，可以使用下面其中一个文件：

```
$HOME/_vimrc
$VIM/_vimrc
```

对于初学 vimrc 文件的朋友，建议在开头放上这行：

```
source $VIMRUNTIME/defaults.vim
```

这为 Vim 新用户进行了初始化（相对于传统 Vi 用户而言）。详见 defaults.vim 。

vimrc 文件可以包含任何冒号命令。最简单的是设置选项命令。例如，如果你想 Vim 启

动的时候始终开启 'incsearch' 选项，可以在你的 vimrc 文件中加上：

```
set incsearch
```

要使这个命令生效，你需要重启动 Vim。后面我们还会学到如何不退出 Vim 就能让它生效。

这一章只解释最基本的东西。想了解更多关于 Vim 脚本的知识，请参见 usr_41.txt 。

05.2 vimrc 示例解释 vimrc_example.vim

在第一章中，我们曾经介绍过怎样用 vimrc 示例文件（包括在 Vim 发布中）使 Vim 以非 vi 兼容模式启动（参见 not-compatible)。这个文件可以在这里找到：

```
$VIMRUNTIME/vimrc_example.vim
```

我们在这一节中介绍这个文件中用到的一些命令。这会对你自行参数设置有一定的帮助。但我们不会介绍所有的内容。你需要用 ":help" 获得更多的帮助。

```
" 获取多数用户想要的缺省值。
source $VIMRUNTIME/defaults.vim
```

这会载入 $VIMRUNTIME 目录里的 "defaults.vim" 文件。这会按照多数用户的喜好设置 Vim。如果你是少数不喜欢的，注释掉这行。其中的命令解释见下：
defaults.vim-explained

```
if has("vms")
  set nobackup
else
  set backup
  if has('persistent_undo')
    set undofile
  endif
endif
```

这告诉 Vim 当覆盖一个文件的时候保留一个备份。但 VMS 系统除外，因为 VMS 系统会自动产生备份文件。备份文件的名称是在原来的文件名上加上 "~" 字符。参见 07.4

如果可用，也设置 'undofile' 选项。会在一个文件中保存多层撤销信息。效果是，当你改动了文件，退出 Vim，然后再次编辑文件时你可以撤销之前做过的改动。这是很强大很有用的功能，代价是要保存一个文件。详情可见 undo-persistence 。

"if" 命令在设置选项的时候非常有用，它使设置命令在某些条件下才执行。更多的内容请参见 usr_41.txt 。

```
if &t_Co > 2 || has("gui_running")
  set hlsearch
endif
```

它打开 'hlsearch' 选项，告诉 Vim 高亮上次查找模式匹配的地方。

```
augroup vimrcEx
  au!
  autocmd FileType text setlocal textwidth=78
augroup END
```

这使 Vim 在一行长于 78 个字符的时候自动换行，但仅对纯文本文件中有效。这里包括两个部分。其中 "autocmd FileType text" 定义个自动命令，表示当文件类型被设置为 "text" 的时候，后面的命令自动执行。"setlocal textwidth=78" 设置 'textwidth' 选项为 78，但仅对本文件有效。

"augroup vimrcEx" 和 "augroup END" 的封装使 "au!" 命令可以删除自动命令。见 :augroup 。

```
if has('syntax') && has('eval')
  packadd! matchit
endif
```

如果所需的特性可用，这会载入 "matchit" 插件。这使得 % 命令更强大。 matchit-install 有解释。

05.3 defaults.vim 文件解释 defaults.vim-explained

如果用户没有 vimrc 文件，载入 defaults.vim 文件。如要你创建自己新的 vimrc 文件，在顶部附近加入此行，以继续使用：

```
source $VIMRUNTIME/defaults.vim
```

或者使用上面解释过的 vimrc_example.vim 文件。

下面解释 defaults.vim 做的事情。

```
if exists('skip_defaults_vim')
  finish
endif
```

defaults.vim 的载入可以用这个命令关闭：

```
let skip_defaults_vim = 1
```

必须在系统 vimrc 文件完成。见 system-vimrc 。如果有你自己的用户 vimrc，不必如此，因为不会自动载入 defaults.vim。

```
set nocompatible
```

就像第一章所述，本手册解释在改进的方式下工作的 Vim，因此与 Vi 不完全兼容。要关闭 'compatible' 选项，'nocompatible' 可以用于完成这个功能。

```
set backspace=indent,eol,start
```

这指明在插入模式下在哪里允许 <BS> 删除光标前面的字符。逗号分隔的三个值分别指：行首的空白字符，换行符和插入模式开始处之前的字符。见 'backspace'。

```
set history=200
```

这个命令保存 200 个命令和 200 个查找模式的历史。如果你想 Vim 记住多些或者少些命令，可以把这个数改成其它值。见 'history'。

```
set ruler
```

总在 Vim 窗口的右下角显示当前光标位置。见 'ruler'。

```
set showcmd
```

在 Vim 窗口右下角，标尺的右边显示未完成的命令。例如，当你输入 "2f"，Vim 在等你输入要查找的字符并且显示 "2f"。当你再输入 w，"2fw" 命令被执行，"2f" 自动消失。

```
+-----------------------------------------------+
|text in the Vim window                         |
|~                                              |
|~                                              |
|-- VISUAL --                  2f     43,8   17% |
+-----------------------------------------------+
 ^^^^^^^^^^                  ^^^^^^^^ ^^^^^^^^^^
  'showmode'                'showcmd'  'ruler'
```

```
set wildmenu
```

在状态行上显示补全匹配。这是按了 <Tab> 后有多于一个匹配的情况。见 'wildmenu'。

```
set ttimeout
set ttimeoutlen=100
```

这使得按 Esc 的生效更快速。通常 Vim 要等待一秒来看看 Esc 是否是转义序列的开始。如果你使用很慢的远程连接，增加此数值。见 'ttimeout'。

```
set display=truncate
```

如果末行被截短，显示 @@@ 而不是隐藏整行。见 'display'。

```
set incsearch
```

在输入部分查找模式时显示相应的匹配点。见 'incsearch'。

```
set nrformats-=octal
```

不把零开头的数值识别为八进制数。见 'nrformats'。

```
map Q gq
```

这定义一个键映射。下一节会介绍更多的相关内容。这将定义 "Q" 命令用来完成与 "gq" 操作符相同的功能，这是在 Vim 5.0 版前 "Q" 命令的作用。如果没有设置这个映射，"Q" 会启动 Ex 模式，这也许不是你想要的情况。

```
inoremap <C-U> <C-G>u<C-U>
```

插入模式下的 CTRL-U 删除当前行所有输入的文本。使用 CTRL-G u 会先打断撤销，这样你可以在插入换行符后撤销 CTRL-U。用 ":iunmap <C-U>" 还原。

```
if has('mouse')
  set mouse=a
endif
```

如果可用，允许使用鼠标。见 'mouse'。

```
vnoremap _g y:exe "grep /" . escape(@", '\\/') . "/ *.c *.h"<CR>
```

这个映射在可视模式下拷贝已选择的文本并在 C 文件中查找它。你可以看到该映射可以被用来执行相当复杂的操作。但其本质依然是一个命令序列而已，与你直接输入没有什么两样。

```
syntax on
```

打开文件的色彩高亮。见 syntax 。

vimrc-filetype

```
filetype plugin indent on
```

这启动三个非常灵巧的机制：

1. 文件类型探测
   当你开始编辑一个文件的时候，Vim 会试图确定这个文件的类型。当编辑 "main.c" 时，Vim 会根据扩展名 ".c" 认为这是一个 C 源文件。当你编辑一个文件前面是 "#!/bin/sh" 的文件时，Vim 会把它认作 "sh" 文件。文件类型探测用于语法高亮和以下另两项。请参见 filetypes 。

2. 使用文件类型相关的插件
   不同的文件需要不同的选项支持。例如，当你编辑一个 "c" 文件，用 'cindent' 选项来自动缩进就非常有用。这些文件类型相关的选项在 Vim 中是通过文件类型插件来实现的。你也可以加入自己的插件，请参见 write-filetype-plugin 。

3. 使用缩进文件
   当编辑程序的时候，行缩进通常可以被自动决定。Vim 用不同的策略处理不同的文件

类型。请参见 :filetype-indent-on 和 'indentexpr'。

restore-cursor last-position-jump

```
autocmd BufReadPost *
  \ if line("'\"") >= 1 && line("'\"") <= line("$") && &ft !~# 'commit'
  \ |   exe "normal! g`\""
  \ | endif
```

这是又一个自动命令。这回它设置为在读入任何文件之后自动执行。后面那堆复杂的东西检查 '" 标记是否已被定义，如果是，则跳转到该标记。行首的反斜杠用于把所有语句连接成一行。这可以避免一行写得太长，
请参见 line-continuation 。这只在 Vim 脚本文件中有效，在命令行中无效。

```
command DiffOrig vert new | set bt=nofile | r ++edit # | 0d_ | diffthis
         \ | wincmd p | diffthis
```

它加入 ":DiffOrig" 命令。在修改过的缓冲区中应用可以看到和原先载入的文件间的差异。见 diff 和 :DiffOrig 。

```
set nolangremap
```

防止 langmap 选项应用于映射生成的字符上。如果置位（缺省），会破坏插件（但后向兼容）。见 'langremap'。

05.4 简单键盘映射

映射可以使你把一系列 Vim 命令绑定为一个单独的键。假设你要用一个大括号将一个特定的单词括起来。例如，把 "amount" 变成 "{amount}"。用 ":map" 命令，就可以让 F5 来完成这个工作。命令如下：

```
:map <F5> i{<Esc>ea}<Esc>
```

备注：
在输入这个命令时，<F5> 要用四个字符表示。相似地，输入 <Esc> 不是直接按 <Esc> 键，而是输入五个字符。读这份手册时请注意这些区别！

让我们来分解一下这个命令：

<F5> F5 功能键。这是命令的触发器。当这个键被按下时，相应的命令即被执行。

i{<Esc> 插入 { 字符。<Esc> 键用于退出插入模式。

e 移动到词尾。

a}<Esc> 插入 } 到单词尾。

执行 ":map" 命令后，要在单词两端加上 {}，只需要移到单词的第一个字符上并按 F5。

在这个例子中，触发器是单独一个键；它还可以是任何字符串。但若你使用一个已经存在的 Vim 命令，该命令将不再有效。最好避免出现这种情况。

一个可用于映射的键是反斜杠。因为你很可能想定义多个映射，那就加上另一个字符。你可以映射 "\p" 为在单词两端加圆括号，而映射 "\c" 为加花括号，例如：

```
:map \p i(<Esc>ea)<Esc>
:map \c i{<Esc>ea}<Esc>
```

你需要在敲入 \ 后，立即敲入 p，以便 Vim 知道它们组成一个命令。

":map" 命令（无参数）列出当前已定义的映射，至少会包括普通模式下的那些。更多的内容参见 40.1 。

05.5 添加软件包 add-package matchit-install

软件包是一组可加入 Vim 的文件。有两种软件包：可选的和启动时自动载入的。

Vim 发布提供了一些软件包供你选用。例如 matchit 插件。此插件使 "%" 命令跳转到匹配的 HTML 标签、Vim 脚本中的 if/else/endif 等。很有用，但不后向兼容（这是为什么缺省没有打开）。

要开始用 matchit 插件，在 vimrc 文件中加入一行：

```
packadd! matchit
```

就这样！重启 Vim 后可以找到此插件的帮助：

```
:help matchit
```

这之所以能工作，是因为 :packadd 载入插件时，同时把软件包目录加入 'runtimepath'，这样就能找到帮助文件了。

网上有多个地方可以找到软件包。它们通常以归档或版本库形式出现。如果是归档，可以采用以下步骤：

1. 建立软件包目录：
   ```
   mkdir -p ~/.vim/pack/fancy
   ```
   "fancy" 可以是任何你喜欢的名字。要能描述你的软件包。
2. 在该目录里解压归档。假定归档的顶层目录是 "start"：
   ```
   cd ~/.vim/pack/fancy
   unzip /tmp/fancy.zip
   ```
   如果归档的目录结构不同，确保最终的路径像这样：
   ```
   ~/.vim/pack/fancy/start/fancytext/plugin/fancy.vim
   ```
   这里 "fancytext" 是软件包名，这可以是任何其它名字。

关于软件包的详情可见： packages 。

05.6 添加插件 add-plugin plugin

Vim 可以通过插件增强功能。插件其实是一个当 Vim 启动的时候能被自动执行的脚本。简单地把插件放到你 Vim 的 plugin 目录中就可以使它生效。
{仅当 Vim 编译时加入 +eval 特性时才有效}

Vim 中有两种插件：

```
全局插件：用于所有类型的文件
文件类型插件：仅用于特定类型的文件
```

我们将先讨论全局插件，然后涉及文件类型插件 add-filetype-plugin 。

全局插件 standard-plugin

当你启动 Vim，它会自动加载一些插件。你不需要为此做任何事。这些插件增加一些很多人想用的，但由 Vim 脚本实现而非编译进 Vim 中的功能。你可以在帮助索引中找到这些插件： standard-plugin-list 。还可以参照 load-plugins 。

add-global-plugin

你可以加入一个全局插件使得某些功能在你每次使用 Vim 时都被开启。添加一个全局插件只要两步：
1. 获得一个插件的拷贝
2. 把它塞进合适的目录

获得一个全局插件

在什么地方可以找到插件?
- 有一些会总是被载入，你可以在 $VIMRUNTIME/plugin 目录中找到它们。
- 有一些与 Vim 一起发布，你可以在 $VIMRUNTIME/macros 目录或其子目录和 $VIM/vimfiles/pack/dist/opt/ 中找到。
- 从网上下载，http://www.vim.org 收集了很多。
- 在 Vim 的邮件列表中找： maillist 。
- 自己写一个，参见 write-plugin 。

某些插件被打包在 vimball 中，参见 vimball 。
某些插件可以自动更新，参见 getscript 。

使用一个全局插件

首先阅读插件包括的说明文字，看看有没有什么特殊的限制。然后拷贝到你的插件目录：

系统	插件目录
Unix	~/.vim/plugin/
PC	$HOME/vimfiles/plugin 或 $VIM/vimfiles/plugin
Amiga	s:vimfiles/plugin
Macintosh	$VIM:vimfiles:plugin
RISC-OS	Choices:vimfiles.plugin

以 Unix 系统为例 (假设你还没有 plugin 目录):

```
mkdir ~/.vim
mkdir ~/.vim/plugin
cp /tmp/yourplugin.vim ~/.vim/plugin
```

就是这样了！现在你可以用这个插件定义的命令了。

除了把这些插件直接放进 plugin/ 目录以外，还可以更好地组织一下，把它们放进 plugin 的单独的子目录中。例如，可以考虑把所有 Perl 插件放置在 "~/.vim/plugin/perl/*.vim"

文件类型插件 add-filetype-plugin ftplugins

Vim 的发布中包括一套针对不同文件类型的插件。你可以用如下命令启用它们:

```
:filetype plugin on
```

这样就行了！参阅 vimrc-filetype 。

如果你缺少某种文件类型的插件，或者你找到一个更好的，你可以自行添加一个。这也只需两步:
1. 获取一个插件的拷贝
2. 塞到合适的目录。

取得文件类型插件

你可以在找全局插件的相同地方找到文件类型插件。注意一下插件有没有注明文件类型，据此你可以知道这个插件是全局的还是文件类型相关的。在 $VIMRUNTIME/macros 中的是全局插件；文件类型插件在 $VIMRUNTIME/ftplugin 中。

使用文件类型插件 ftplugin-name

你可以通过把插件文件放到合适的目录中来增加一个插件。目录的名字与前面提过的全局插件的位置一样，但最后一级目录是 "ftplugin"。假设你找到一个用于 "stuff" 文件类型的插件，而且你的系统是 Unix。那么，你可以把这个文件用如下命令移入 ftplugin 目录:

```
mv thefile ~/.vim/ftplugin/stuff.vim
```

如果这个文件已经存在，你可以检查一下两个插件有没有冲突。如果没有，你可以用另一个名字:

```
mv thefile ~/.vim/ftplugin/stuff_too.vim
```

这里，下划线用来分开文件类型和其它部分 (这些部分可以由任意字符组成)。但如果你用 "otherstuff.vim" 就不行了。那是用于 "otherstuff" 类型的文件的。

在 MS-DOS 上不能使用长文件名。如果你增加第二个插件，而这个插件超过 6 个字符，你就没法用了。你可以通过使用另一层目录来解决这个问题:

```
mkdir $VIM/vimfiles/ftplugin/fortran
copy thefile $VIM/vimfiles/ftplugin/fortran/too.vim
```

总的来说，一个文件类型相关的插件的名称是:

```
ftplugin/<filetype>.vim
ftplugin/<filetype>_<name>.vim
ftplugin/<filetype>/<name>.vim
```

这里 "<name>" 可以是任何你喜欢的名字。例如，在 Unix 上，"stuff" 文件类型的插件可以是:

```
~/.vim/ftplugin/stuff.vim
~/.vim/ftplugin/stuff_def.vim
~/.vim/ftplugin/stuff/header.vim
```

这里，<filetype> 部分是相应文件类型的名称。只有对应文件类型的文件才会用这个插件内的设置。插件的 <name> 部分则不重要，你可以对同一个文件类型使用多个插件。

注意 插件必须以 ".vim" 结尾。

请进一步阅读：

filetype-plugins	文件类型插件的文档和有关如何避免映射引起的问题。
load-plugins	全局插件的启动时间。
ftplugin-overrule	控制全局文件类型插件的选项。
write-plugin	如何写插件。
plugin-details	关于如何使用插件的信息或者当你的插件不工作的时候如何处理。
new-filetype	如果检测新文件类型。

05.7 添加帮助 add-local-help

如果幸运的话，你安装的插件还会包括帮助文件。我们这里解释如何安装这个帮助文件，以便你能方便地获得新插件的帮助。

我们以 "doit.vim" 插件为例。这个插件有一个文档："doit.txt"。我们先来把该插件拷贝到合适的位置。这次，我们在 Vim 内完成这个工作。(如果某些目录已经存在你可以省略一些 "mkdir" 命令。)

```
:!mkdir ~/.vim
:!mkdir ~/.vim/plugin
:!cp /tmp/doit.vim ~/.vim/plugin
```

"cp" 命令是基于 Unix 的，MS-DOS 上你可以用 "copy"。

现在在某个 'runtimepath' 目录中建立一个 doc 目录。

```
:!mkdir ~/.vim/doc
```

再把帮助文件拷贝进去：

```
:!cp /tmp/doit.txt ~/.vim/doc
```

现在开始玩技巧了，怎样使 Vim 允许你跳转到新的主题上？用 :helptags 命令产生一个本地的 tags 文件即可：

```
:helptags ~/.vim/doc
```

现在，你可以用这个命令

```
:help doit
```

来获得 "doit" 的帮助了。显示你刚才加上的帮助文件。在使用如下命令的时候，可以看见一个新的条目：

```
:help local-additions
```

本地帮助的标题行被自动的加入到该节了。在那里你可以看到 Vim 添加了那些本地的帮助文件。你还可以从这里跳转到新的帮助中。

要写一个本地帮助文件，请参考 write-local-help 。

05.8 选项窗口

如果要找一个选项，你可以在这里寻找帮助： options 。另一个方法是用如下命令：

```
:options
```

这会打开一个新窗口，其中给出一个选项的列表，并对每个选项提供一行解释。这些选项根据种类分组。将光标移到一个主题上然后按 <Enter> 就可以跳转到那里。再按一下 <Enter> 或者 CTRL-O 就可以跳回来。

你可以通过这个窗口改变一个选项的值。例如，移到 "displaying text" 主题。然后把光标下移到这一行：

```
set wrap        nowrap
```

当你在上面键入回车，这行会改变为：

```
set nowrap      wrap
```

现在，这个选项被关闭了。

这行的上方是对这个选项的简要描述。将光标向上移动一行，然后按 <Enter>，你可以跳转到 'wrap' 的完整帮助，再用 CTRL-O 可以跳回来。

对于那些值为数值或者字符串的选项，你可以编辑它的值，然后按 <Enter> 来启用该值。例如，把光标移动到下面这行：

```
set so=0
```

用 $ 移到行尾，再用 "r5" 命令修改为五，然后按 <Enter> 使修改生效。现在如果你移动一下光标，你会发现在你的光标移到窗口边界前，你的文字就开始滚动了。这就是选项 'scrolloff' 完成的功能：它指定在距离边界多远的地方开始滚动文字。

05.9 常用选项

Vim 中有很多选项。大部分你很少用得上。我们在这个介绍一些常用的。别忘了你可以通过 ":help" 命令获得更多的帮助。方法是在选项命令前后加上单引号，例如：

```
:help 'wrap'
```

如果你搞乱了一个选项，你可以通过在选项后加上一个 & 号把它恢复到默认值。例如：

```
:set iskeyword&
```

禁止回绕行

Vim 通常会对长行自动回绕，以便你可以看见所有的文字。但有时最好还是能让文字在一行中显示完。这样，你需要左右移动才能看到一整行。以下命令可以关闭行的回绕：

```
:set nowrap
```

当你移动到那些不能显示的文字上，Vim 会自动向右滚动让你看到后面的文字，要一次滚动十个字符，这样就行了：

```
:set sidescroll=10
```

这个命令不改变文件中的文本，只改变显示方式。

移动命令换行

很多命令只能在一行中移动。你可以通过 'whichwrap' 选项改变它。如下命令把这个选项设为默认值：

```
:set whichwrap=b,s
```

这样，当光标处于行首时用 <BS> 键可以回到前一行的结尾；当处于行尾时用 <Space> 键可以移动到下一行的行首。

要允许 <Left> 和 <Right> 键也能这样，可以用这个命令：

```
set whichwrap=b,s,<,>
```

这只在普通模式中有效，要在插入模式中使 <Left> 和 <Right> 也有效，可以：

```
:set whichwrap=b,s,<,>,[,]
```

还有一些可以用的标志位，参见 'whichwrap'。

显示 TAB 键

文件中有 TAB 键的时候，你是看不见的。要把它显示出来：

```
:set list
```

现在 TAB 键显示为 ^I，而 $显示在每行的结尾，以便你能找到可能会被你忽略的空白字符在哪里。

这样做的一个缺点是在有很多 TAB 的时候看起来很丑。如果你使用一个有颜色的终端，或者使用 GUI 模式，Vim 可以用高亮显示空格和 TAB。使用 'listchars' 选项：

```
:set listchars=tab:>-,trail:-
```

现在，TAB 会被显示成 ">---"（"-" 的个数不定）而行尾多余的空白字符显示成 "-"。看起来好多了，是吧?

关键字

'iskeyword' 选项指定哪些字母可以出现在一个单词中：

```
:set iskeyword
 iskeyword=@,48-57,_,192-255
```

"@" 表示所有字母。"48-57" 表示 ASCII 字符 48-57 ，即数字 0 到 9。"192-255" 是可显示的拉丁字符。

有时你希望横线也是关键字，以便 "w" 命令会把 "upper-case" 看作是一个单词。你可以这样做：

```
:set iskeyword+=-
:set iskeyword
 iskeyword=@,48-57,_,192-255,-
```

看一下新的值，你会发现 Vim 自动在 "-" 前面加了一个逗号。

要从中去掉一个字符，用 "-="。例如要排除下划线：

```
:set iskeyword-=_
:set iskeyword
 iskeyword=@,48-57,192-255,-
```

这回，逗号又自动被删除了。

显 示 消 息 的 空 间

当 Vim 启动的时候，在屏幕底部有一行被用于显示消息。当消息很长的时候，多余的部分会被截断。这样你只能看到一部分。或者文字自动滚动，你要按 <Enter> 来继续。你可以给 'cmdheight' 选项赋一个值，用来设定显示消息所用的行数。例如：

```
:set cmdheight=3
```

这样意味着你用于编辑文字的空间少了，所以这实际上是一种折衷。

下一章： usr_06.txt 使用语法高亮

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

使用语法高亮

黑白的文字让人厌倦了，增加一些色彩能为你的文件带来生气。这不但看起来漂亮，还能够提高你的工作效率。本章介绍如何使用不同颜色显示不同文本并把它打印出来。

06.1 功能激活
06.2 颜色显示不出来或者显示出错误的颜色怎么办?
06.3 使用不同的颜色
06.4 是否使用色彩
06.5 带颜色打印
06.6 深入阅读

下一章： usr_07.txt 编辑多个文件
前一章： usr_05.txt 选项设置
目录： usr_toc.txt

06.1 功能激活

一切从一个简单的命令开始：

```
:syntax enable
```

大多数情况下，这会让你的文件带上颜色。Vim 会自动检测文件的类型，并调用合适的语法高亮。一下子注释变成蓝色，关键字变成褐色，而字符串变成红色了。这使你可以很容易浏览整个文档。很快你就会发现，黑白的文本真的会降低你的效率！

如果你希望总能看到语法高亮，把 "syntax enable" 命令加入到 vimrc 文件中。

如果你想语法高亮只在支持色彩的终端中生效，你可以在 vimrc 文件中这样写：

```
if &t_Co > 1
  syntax enable
endif
```

如果你只想在 GUI 版本中有效，可以把 ":syntax enable" 放入你的 gvimrc 文件。

06.2 颜色显示不出来或者显示出错误的颜色怎么办?

有很多因素会让你看不到颜色：

- 你的终端不支持彩色。
  这种情况下，Vim 会用粗体，斜体和下划线区分不同文字，但这不好看。你可能会希望找一个支持彩色的终端。对于 Unix，我推荐 XFree86 项目的 xterm: xfree-xterm 。

- 你的终端其实支持颜色，可是 Vim 不知道。
  确保你的 $TERM 设置正确。例如，当你使用一个支持彩色的 xterm 终端：

  ```
  setenv TERM xterm-color
  ```

  或者（基于你用的控制台终端）

  ```
  TERM=xterm-color; export TERM
  ```

  终端名必须与你使用的终端一致。如果这还是不行，参考一下 xterm-color ，那里介绍了一些使 Vim 显示彩色的方法（不仅是 xterm）。

- 文件类型无法识别。
  Vim 不可能识别所有文件，而且有时很难说一个文件是什么类型的。试一下这个命令：

  ```
  :set filetype
  ```

  如果结果是 "filetype="，那么问题就是出在文件类型上了。你可以手工指定文件类型：

  ```
  :set filetype=fortran
  ```

  要知道哪些类型是有效的，查看一下 $VIMRUNTIME/syntax 目录。对于 GUI 版本，你还可以使用 Syntax 菜单。设置文件类型也可以通过 modeline ，这种方式使得该文件每次被编辑时都被高亮。例如，下面这行可用于 Makefile（把它放在接近文件首和文件末的地方）

  ```
  # vim: syntax=make
  ```

  你可能知道怎么检测自己的文件类型，通常的方法是检查文件的扩展名（就是点后面的内容）。 new-filetype 说明如何告知 Vim 进行那种文件类型的检查。

- 你的文件类型没有语法高亮定义。
  你可以找一个相似的文件类型并人工设置为那种类型。如果觉得不好，你可以自己写一个，参见 mysyntaxfile 。

或者颜色是错的：

- 彩色的文字难以辨认。
  Vim 自动猜测你使用的背景色。如果是黑的（或者其它深色的色彩），它会用浅色作为前景色。如果是白的（或者其它浅色），它会使用深色作为前景色。如果 Vim 猜错了，文字就很难认了。要解决这个问题，设置一下 'background' 选项。对于深色：

  ```
  :set background=dark
  ```

  而对于浅色：

  ```
  :set background=light
  ```

  这两个命令必须在 ":syntax enable" 命令前调用，否则不起作用。如果要在这之后设置背景，可以再调用一下 ":syntax reset" 使得 Vim 重新进行缺省颜色

的设置。

- 在自下往上滚屏的过程中颜色显示不对。
  Vim 在分析文本的时候不对整个文件进行处理，它只分析你要显示的部分。这样能省不少时间，但也会因此带来错误。一个简单的修正方法是敲 CTRL-L。或者往回滚动一下再回来。要彻底解决这个问题，请参见 :syn-sync 。有些语法定义文件有办法自己找到前面的内容，这可以参见相应的语法定义文件。例如， tex.vim 中可以查到 Tex 语法定义。

06.3 使用不同颜色 :syn-default-override

如果你不喜欢默认的颜色方案，你可以选另一个色彩方案。在 GUI 版本中可以使用 Edit/Color 菜单。你也可以使用这个命令：

```
:colorscheme evening
```

"evening" 是色彩方案的名称。还有几种备选方案可以试一下。在 $VIMRUNTIME/colors 中可以找到这些方案。

等你确定了一种喜欢的色彩方案，可以把 ":colorscheme" 命令加到你的 vimrc 文件中。

你可以自己编写色彩方案，方法如下：

1. 选择一种接近你理想的色彩方案。把这个文件拷贝到你自己的 Vim 目录中。在 Unix 上，可以这样：

```
!mkdir ~/.vim/colors
!cp $VIMRUNTIME/colors/morning.vim ~/.vim/colors/mine.vim
```

   在 Vim 中完成的好处是可以利用 $VIMRUNTIME 变量。

2. 编辑这个色彩方案，常用的有下面的这些条目：

term	黑白终端的属性
cterm	彩色终端的属性
ctermfg	彩色终端的前景色
ctermbg	彩色终端的背景色
gui	GUI 版本属性
guifg	GUI 版本的前景色
guibg	GUI 版本的背景色

   例如，要用绿色显示注释：

```
:highlight Comment ctermfg=green guifg=green
```

   属性是 "bold"（粗体）和 "underline"（下划线）可以用于 "cterm" 和 "gui"。如果你两个都想用，可以用 "bond,underline"。详细信息请参考 :highlight 命令。

3. 告诉 Vim 总使用你这个色彩方案。把如下语句加入你的 vimrc 中：

```
colorscheme mine
```

如果你要测试一下常用的色彩组合，用如下命令：

```
:runtime syntax/colortest.vim
```

这样你会看到不同的颜色组合。你可以很容易的看到哪一种可读性好而且漂亮。

06.4 是否使用色彩

使用色彩显示文本会影响效率。如果你觉得显示得很慢，可以临时关掉这个功能：

```
:syntax clear
```

当你开始编辑另一个文件（或者同一个文件），色彩会重新生效。

如果你要完全关闭这个功能：

```
:syntax off
```

这个命令会停止对所有缓冲区的所有语法高亮。详见 :syntax-off 。

:syn-manual

如果你想只对特定的文件采用语法高亮，可以使用这个命令：

```
:syntax manual
```

这个命令激活语法高亮功能，但不会在你开始编辑一个缓冲区时自动生效（译者注：Vim 中，每个被打开的文件对应一个缓冲区，后面的章节中你会接触到这方面的内容）。要在当前缓冲区中使用高亮，需要设置 'syntax' 选项：

```
:set syntax=ON
```

06.5 带颜色打印

syntax-printing

在 MS-Windows 版本上，你可以用如下命令打印当前文件：

```
:hardcopy
```

这个命令会启动一个常见的打印对话框，你可以通过它选择打印机并作一些必要的设置。如果你使用的是彩色打印机，那么打印出来的色彩将与你在 Vim 中看到的一样。但如果你使用的是深色的背景，它的颜色会被适当调整，以便在白色地打印纸上看起来比较舒服。

下面几个选项可以改变 Vim 的打印行为：

```
'printdevice'
'printheader'
'printfont'
'printoptions'
```

要仅打印一定范围内的行，可以用可视模式选择需要打印的行再执行打印命令，例如：

```
v100j:hardcopy
```

"v" 启动可视模式，"100j" 向下选中 100 行，然后执行 ":hardcopy" 打印这些行。当然，你可以用其它命令选中这 100 行。

如果你有一台 PostScript 打印机，上面的方法也适合 Unix 系统。否则，你必须做一些额外的处理：你需要先把文件转换成 HTML 类型，然后用浏览器打印。

如下命令把当前文件转换成 HTML 格式：

```
:TOhtml
```

如果不行：

```
:source $VIMRUNTIME/syntax/2html.vim
```

你发现它会嘎吱嘎吱执行一阵子，(如果文件很大，这可能要花点时间)。之后，Vim 会打开一个新的窗口并显示 HTML 代码。现在把这个文件存下来 (存在哪都不要紧，反正最后你要删掉它的)：

```
:write main.c.html
```

用你喜欢的浏览器打开这个文件，并通过它打印这个文件。如果一切顺利，这个输出应该与 Vim 中显示的一样。要了解更详细的信息，请参见 2html.vim 。处理完后别忘了删掉那个 HTML 文件。

除了打印，你还可以把这个 HTML 文件，放到 WEB 服务器上，让其他人可以通过彩色文本阅读。

06.6 深入阅读

usr_44.txt	自定义语法高亮
syntax	关于本话题的全部细节

下一章： usr_07.txt 编辑多个文件

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

编辑多个文件

无论你有多少个文件，你都可以同时编辑它们而不需要退出 Vim。本章介绍如何定义一个文件列表，并基于这个列表工作，或者从一个文件跳转到另一个文件，又或者从一个文件中拷贝文字，并写入到另一个文件中。

07.1 编辑另一个文件
07.2 文件列表
07.3 从一个文件中跳到另一个文件
07.4 备份文件
07.5 文件间拷贝
07.6 显示文件
07.7 修改文件名

下一章： usr_08.txt 分割窗口
前一章： usr_06.txt 使用语法高亮
目录： usr_toc.txt

07.1 编辑另一个文件

在本章前，你都是为每一个文件启动一次 Vim 的。实际上还有其它办法。如下命令就可以在 Vim 中打开另一个文件：

```
:edit foo.txt
```

你可以用任何其它文件名取代上面的 "foo.txt"。Vim 会关闭当前文件并打开另一个。但如果当前文件被修改过而没有存盘， Vim 会显示错误信息而不会打开这个新文件：

```
E37: No write since last change (use ! to override)
```

(译者注：在中文状态下显示：

```
E37: 已修改但尚未保存 (可用 ! 强制执行)
```

)

备注：
Vim 在每个错误信息的前面都放了一个错误号。如果你不明白错误信息的意思，可以从帮助系统中获得更详细的说明。对本例而言：

```
:help E37
```

出现上面的情况，你有多个解决方案。首先你可以通过如下命令保存当前文件：

```
:write
```

或者，你可以强制 Vim 放弃当前修改并编辑新的文件。这时应该使用强制修饰符 ! ：

```
:edit! foo.txt
```

如果你想编辑另一个文件，但又不想马上保存当前文件，可以隐藏它：

```
:hide edit foo.txt
```

原来的文件还在那里，只不过你看不见。这将在 " 22.4 : 缓冲区列表" 中解释。

07.2 文件列表

你可以在启动 Vim 的时候指定一堆文件。例如：

```
vim one.c two.c three.c
```

这个命令启动 Vim 并告诉它你要编辑三个文件。Vim 只显示第一个。等你编辑完第一个以后，用如下命令可以编辑第二个：

```
:next
```

如果你在当前文件中有未保存的修改，你会得到一个错误信息而无法编辑下一个文件。这个问题与前一节执行 ":edit" 命令的问题相同。要放弃当前修改：

```
:next!
```

但大多数情况下，你需要保存当前文件再进入下一个。这里有一个特殊的命令：

```
:wnext
```

这相当于执行了两个命令：

```
:write
:next
```

我 在 哪?

要知道当前文件在文件列表中的位置，可以注意一下文件的标题。那里应该显示类似 "(2 of 3)" 的字样。这表示你正在编辑三个文件中的第二个。

如果你要查看整个文件列表，使用如下命令：

```
:args
```

这是 "arguments" (参数) 的缩写。其输出应该像下面这样：

```
one.c [two.c] three.c
```

这里列出所有你启动 Vim 时指定的文件。你正在编辑的那一个，例如，"two.c"，会用中括号括起。

移 动 到 另 一 个 参 数

要回到前一个文件：

```
:previous
```

这个命令与 ":next" 相似，只不过它是向相反的方向移动。同样地，这个命令有一个快捷版本用于 "保存再移动"：

```
:wprevious
```

要移动到列表中的最后一个文件：

```
:last
```

而要移动到列表中的第一个文件：

```
:first
```

不过，可没有 ":wlast" 或者 "wfirst" 这样的命令了。

你可以在 ":next" 和 ":previous" 前面加计数前缀。例如要向后跳两个文件：

```
:2next
```

自 动 保 存

当你在多个文件间跳来跳去进行修改，你要老记着用 ":write" 保存文件。否则你就会得到一个错误信息。 如果你能确定你每次都会将修改存盘的话，你可以让 Vim 自动保存文件：

```
:set autowrite
```

如果你编辑一个你不想自动保存的文件，你可以把功能关闭：

```
:set noautowrite
```

编 辑 另 一 个 文 件 列 表

你可以编辑另一个文件列表而不需要退出 Vim。用如下命令编辑另三个文件：

```
:args five.c six.c seven.h
```

或者使用通配符，就像在控制台上一样：

```
:args *.txt
```

Vim 会跳转到列表中的第一个文件。同样地，如果当前文件没有保存，你需要保存它，或者使用 ":args!" (加了一个 !) 放弃修改。

你 编 辑 了 最 后 一 个 文 件 吗?

arglist-quit

当你使用了文件列表，Vim 假定你想编辑全部文件，为了防止你提前退出，如果你还没有

编辑过最后一个文件。当你退出的时候，Vim 会给如下错误信息：

```
E173: 46 more files to edit
```

如果你确实需要退出，再执行一次这个命令就行了（但如果在两个命令间还执行了其它命令就无效了）。

07.3 从一个文件跳到另一个文件

要在两个文件间快速跳转，按 CTRL-^（美式英语键盘中 ^ 6 的上面）。例如：

```
:args one.c two.c three.c
```

现在你在 one.c。

```
:next
```

现在你在 two.c。现在使用 CTRL-^ 回到 one.c。再按一下 CTRL-^ 则回到 two.c。又按一下 CTRL-^ 你再回到 one.c。如果你现在执行：

```
:next
```

现在你在 three.c。注意 CTRL-^ 不会改变你在文件列表中的位置。只有 ":next" 和 ":previous" 才能做到这点。

你编辑的前一个文件称为 "轮换" 文件。如果你启动 Vim 而 CTRL-^ 不起作用，那可能是因为你没有轮换文件。

预 定 义 标 记

当你跳转到另一个文件后，有两个预定义的标记非常有用：

```
‘"
```

这个标记使你跳转到你上次离开这个文件时的位置。
另一个标记记住你最后一次修改文件的位置：

```
‘.
```

假设你在编辑 "one.txt"，在文件中间某个地方你用 "x" 删除一个字符，接着用 "G" 命令移到文件末尾，然后用 "w" 存盘。然后你又编辑了其它几个文件。你现在用 ":edit one.txt" 回到 "one.txt"。如果现在你用 ‘"，Vim 会跳转到文件的最后一行；而用 ‘. 则跳转到你删除字符的地方。即使你在文件中移动过，但在你修改或者离开文件前，这两个标记都不会改变。

文 件 标 记

在第四章，我们介绍过使用 "mx" 命令在文件中增加标记，那只在一个文件中有效。如果你编辑另一个文件并在那里加了标记，这些标记都是这个文件专用的。这样，每个文件都有一个自己的标记集，并只能在该文件中使用。

到此为止，我们都用小写字母的标记。实际上还可以使用大写字母标记，这种标记是全局的，它们可以在任何文件中使用。例如，你在编辑一个文件 "foo.txt"。在文件的中

间 (50%) 并建立一个 J 标记 (J 表示甲):

```
50%mJ
```

现在编辑文件 "bar.txt" 并在文件的最后一行放一个标记 Y (Y 表示乙):

```
GmY
```

现在你可以使用 "'J" 命令跳回到 foo.txt 的中间。或者在另一个文件中输入 "'Y" 跳回到 bar.txt 的末尾。

文件标记会被一直记住直到被重新定义。这样，你可以在一个文件中留下一个标记，然后任意做一段时间的编辑，最后用这个标记跳回去。

让文件标记符和对应的位置建立一些关系常常是很有用的。例如，用 H 表示头文件 (Head File)，M 表示 Makefile 而 C 表示 C 的代码文件。

要知道一个标记在什么地方，在 ":marks" 命令中加上标记名作为参数即可:

```
:marks M
```

你还可以带多个参数:

```
:marks MCP
```

别忘了你还可以 CTRL-O 和 CTRL-I 在整个跳转序列中前后跳转。

07.4 备份文件

通常 Vim 不会产生备份文件。如果你希望的话，执行如下命令就可以了:

```
:set backup
```

备份文件的文件名是在原始文件的后面加上一个 ~ 。如果你的文件名是 data.txt，则备份文件的文件名就是 data.txt~ 。

如果你不喜欢这个名字，你可以修改扩展名:

```
:set backupext=.bak
```

这会使用 data.txt.bak 而非 data.txt~。

还有一个相关选项是 'backupdir'。它指定备份文件的目录。默认情况是与原始文件的路径一致，这在很多情况下都是合适的。

> 备注:
> 如果 'backup' 选项没有置位而 'writebackup' 选项置了位，Vim 还是会创建备份文件的。但在文件编辑完后，这个备份文件会被自动删除。这个功能用于避免发生异常情况导致没有存盘 (磁盘满是最常见的情况；被雷击也是一种情况，不过很少发生)。

保 留 原 始 文 件

如果你在编辑源程序，你可能想在修改之前保留一个备份。但备份文件会在你存盘的时候被覆盖。这样它只能保留前一个版本，而不是最早的文件。

要让 Vim 保存一个原始的文件，可以设置 'patchmode' 选项。这个选项定义需要改动文件的第一个备份文件的扩展名。通常可以这样设：

```
:set patchmode=.orig
```

这样，当你第一次编辑 data.txt，作了修改并执行存盘，Vim 会保留一个名为 "data.txt.orig" 的原始文件。

如果你接着修改这个文件，Vim 会发现这个原始文件已经存在，并不再覆盖它。进一步的备份就存在 "data.txt~" (或者你设置的 'backupext' 指定的文件) 中。

如果你让 'patchmode' 设为空 (这是默认的情况)，则原始文件不会被保留。

07.5 文件间拷贝文本

本节解释如何在文件间拷贝文本。我们从一个简单的例子开始。编辑一个你要拷贝文本的文件，把光标移到要拷贝的文本的开始处，用 "v" 命令启动可视模式，然后把光标移到要拷贝文本的结尾处，输入 "y" 拷贝文本。

例如，要拷贝上面这段文字，你可以执行：

```
:edit thisfile
/本节解释
vjjj$y
```

现在编辑你要粘贴文本的文件。把光标移到你要插入文本的地方。用 "p" 命令把文本粘贴到那里：

```
:edit otherfile
/There
p
```

当然，你可以用任何命令拷贝文本。例如，用 "V" 命令选中整行的内容。或者用 CTRL-V 选择一个矩形列块。或者使用 "Y" 拷贝一个单行，"yaw" 拷贝一个单词等。

"p" 命令把文本粘贴到光标之后，"P" 命令则粘贴到光标之前。注意，Vim 会记住你拷贝的是一整行还是一个列块，并用相同的方式把文本贴出来。

使用寄存器

当你需要拷贝一个文件的几个地方到另一个文件，用上面的方法，你就得反复在两个文件间跳来跳去。要避免这种情况，你可以把不同的文本拷贝到不同的寄存器中。

寄存器是 Vim 用来保存文本的地方。这里我们使用名称为 a 到 z 的寄存器 (后面我们会发现还有其它寄存器)。让我们拷贝一个句子到 f 寄存器 (f 表示 First)：

```
"fyas
```

"yas" 命令像以前说过的那样拷贝一个句子，而 "f 告诉 Vim 把文本拷贝到寄存器 f 。这必须放在拷贝命令的前面。

现在，拷贝三个整行到寄存器 l (l 表示 line)：

```
"l3Y
```

计数前缀也可以用在 "l 的前面。要拷贝一个文本列块到寄存器 b (代表 block) 中：

```
CTRL-Vjjww"by
```

注意 "b 正好在 "y" 命令的前面，这是必须的。把它放在 "w" 命令的前面就不行。

现在你有了在寄存器 f，l 和 b 有三段文本。编辑另一个文件，并移到要插入文本的地方：

```
"fp
```

同样地，寄存器标识符 "f 必须在 "p" 命令的前面。

你可以用任何顺序粘贴寄存器的内容。并且，这些内容一直存在于寄存器中，直到你拷贝其它文件到这个寄存器中。这样，你可以粘贴任意多次。

删除文本的时候，你也可以指定寄存器。使用这个方法可以移动几处文本。例如，要删除一个单词并写到 w 寄存器中：

```
"wdaw
```

同样地，寄存器标识符必须在删除命令 "d" 的前面。

添 加 到 文 件

当你要在几个文件中收集文本，你可以用这个命令：

```
:write >> logfile
```

这个命令将文本写入到文件的末尾。这样实现了文件添加功能。这样使你免去了拷贝，编辑和拷贝的过程，省了两步。但你只能加到目标文件的末尾。

要只拷贝一部分内容，可以先用可视模式选中这些内容后在执行 ":write"。在第 10 章，你将学会选中一个行范围的办法。

07.6 显示文件

有时，你只是想查看一个文件，而没打算修改它。有一个风险是你想都没想就输入了一个 "w" 命令。要避免这个问题，以只读模式编辑这个文件。

要用只读模式启动 Vim，可以使用这个命令：

```
vim -R file
```

在 Unix，如下命令可以完成相同的功能：

```
view file
```

现在，你就在用只读模式阅读这个文件 "file" 了。但你执行 ":w" 命令的时候，你会得到一个禁止写入的错误信息。

当你试图修改这个文件时，Vim 会给你一个告警提示：

```
W10: Warning: Changing a readonly file
```

即使这样，你的修改还是会被接纳的。有可能你只是想排列这些文本，以便阅读。

如果你确实要改动这个文件，在 write 命令前面加上 ! 可以强制写入。

如果你的确想禁止文件修改，用这个命令：

```
vim -M file
```

现在任何对文件的修改操作都会失败。例如，帮助文件就是这样的。如果你要在上面作修改，你会得到一个错误提示：

```
E21: Cannot make changes, 'modifiable' is off
```

你可以设置 `-M` 参数使 `Vim` 工作在只读模式。这个方式仍然取决于用户的意愿，因为你可以用下面的命令去掉这层保护：

```
:set modifiable
:set write
```

07.7 修改文件名

编辑一个新文件的一个比较聪明的做法是使用一个现存的、其中大部分内容你都需要的文件。例如，你要写一个移动文件的程序，而你已经有一个用于拷贝的程序了，这样可以这样开始：

```
:edit copy.c
```

删除你不要的东西。现在你需要用一个新的文件名保存这个文件。`":saveas"` 命令就是为此设计的：

```
:saveas move.c
```

`Vim` 会用给定的名称保存文件，并开始编辑该文件。这样，下次你用 `":write"`，写入的时候，被写入的就是 `"move.c"`。而 `"copy.c"` 不会被改变。

当你想改变当前文件的文件名，但不想立即保存它，用这个命令：

```
:file move.c
```

`Vim` 会把这个文件标记为 `"未编辑"`。这表示 `Vim` 知道你现在编辑的文件不是原来那个文件了。当你写这个文件的时候，你会得到如下错误信息：

```
E13: File exists (use ! to override)
```

这可以避免你不小心覆盖另一个文件。

下一章： usr_08.txt 分割窗口

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

分割窗口

显示两个不同的文件；或者同时显示一个文件的两个不同地方；又或者并排比较两个文件。这一切都可以通过分割窗口实现。

08.1 分割窗口
08.2 用另一个文件分割窗口
08.3 窗口大小
08.4 垂直分割
08.5 移动窗口
08.6 对所有窗口执行命令
08.7 用 vimdiff 显示文件差异
08.8 杂项
08.9 标签页

下一章：usr_09.txt 使用 GUI 版本
前一章：usr_07.txt 编辑多个文件
目录：usr_toc.txt

08.1 分割窗口

打开新窗口最简单的命令如下：

```
:split
```

这个命令把屏幕分解成两个窗口并把光标置于上面的窗口中：

```
+----------------------------------+
|/* file one.c */                  |
|~                                 |
|~                                 |
 one.c============================
|/* file one.c */                  |
|~                                 |
 one.c============================
|                                  |
+----------------------------------+
```

你可以看到显示同一个文件的两个窗口。带 "====" 的行是状态条，用来显示它上面的窗口的信息。(在实际的屏幕上，状态条用反色显示)

这两个窗口允许你同时显示一个文件的两个部分。例如，你可以让上面的窗口显示变量定义而下面的窗口显示使用这些变量的代码。

CTRL-W w 命令可以用于在窗口间跳转。如果你在上面的窗口，它会跳转到下面的窗口，如果你在下面的窗口，它会跳转到上面的窗口。(CTRL-W CTRL-W 可以完成相同的功能这是为了避免你有时按第二次的时候从 CTRL 键上缩手晚了。)

关 闭 窗 口

以下命令用于关闭窗口：

```
:close
```

实际上，任何退出编辑的命令都可以关闭窗口，像 ":quit" 和 "ZZ" 等。但 "close" 可以避免你在剩下一个窗口的时候不小心退出 Vim 了。

关 闭 所 有 其 它 窗 口

如果你已经打开了一整套窗口，但现在只想编辑其中一个，如下命令可以完成这个功能：

```
:only
```

这个命令关闭除当前窗口外的所有窗口。如果要关闭的窗口中有一个没有存盘，Vim 会显示一个错误信息，并且那个窗口不会被关闭。

08.2 用另一个文件分割窗口

下面命令打开另一个窗口并用该窗口编辑另一个指定的文件：

```
:split two.c
```

如果你在编辑 one.c，则命令执行的结果是：

```
+----------------------------------+
|/* file two.c */                  |
|~                                 |
|~                                 |
 two.c==============================
|/* file one.c */                  |
|~                                 |
 one.c==============================
|                                  |
+----------------------------------+
```

要打开窗口编辑一个新文件，可以使用如下命令：

```
:new
```

你可以重复使用 ":split" 和 ":new" 命令建立任意多的窗口。

08.3 窗口大小

:split 命令可以接受计数前缀。如果指定了这个前缀，这个数将作为窗口的高度。例如如下命令可以打开一个三行的窗口并编辑文件 alpha.c：

```
:3split alpha.c
```

对于已经打开的窗口，你可以用有几种方法改变它的大小。如果你有鼠标，很简单：把鼠标指针移到分割两个窗口的状态栏上，上下拖动即可。

要扩大窗口：

```
CTRL-W +
```

要缩小窗口：

```
CTRL-W -
```

这两个命令接受计数前缀用于指定扩大和缩小的行数。所以 "4 CTRL-W +" 会使窗口增高 4 行。

要把一个窗口设置为指定的高度，可以用这个命令：

```
{height}CTRL-W _
```

就是先输入一个数值，然后输入 CTRL-W 和一个下划线（在美式英语键盘中就是 Shift 加上 "-"）。

要把一个窗口扩展到尽可能大，可以使用无计数前缀的 CTRL-W _ 命令。

使 用 鼠 标

在 Vim 中，你可以用键盘很快完成很多工作。但很不幸，改变窗口大小要敲不少键。在这种情况下，使用鼠标会更快一些。把鼠标指针移到状态条上，按住左键并拖动。状态条会随之移动，这会使一个窗口更大一个更小。

选 项

'winheight' 选项设置最小的期望窗口高度而 'winminheight' 选项设置最小的 "硬性" 高度。

同样，'winwidth' 设置最小期望宽度而 'winminwidth' 设置最小硬性宽度。

'equalalways' 选项使所有的窗口在关闭或者打开新窗口的时候总保持相同大小。

08.4 垂直分割

":split" 命令在当前窗口的上面建立窗口。要在窗口左边打开新窗口，用这个命令：

```
:vsplit
```

或者

```
:vsplit two.c
```

这个命令的结果如下：

```
+-------------------------------------+
|/* file two.c */   |/* file one.c */  |
|~                  |~                 |
```

```
|~                    |~                    |
|~                    |~                    |
 two.c================one.c=============
|                                           |
+-------------------------------------------+
```

实际中，中间的竖线会以反色显示。这称为垂直分割线。它左右分割一个窗口。

还有一个 "vnew" 命令，用于打开一个垂直分割的新窗口。还有一种方法是：

```
:vertical new
```

"vertical" 命令可以放在任何分割窗口的命令的前面。这会在分割窗口的时候用垂直分割取代水平分割。(如果命令不分割窗口，这个前缀不起作用)。

在窗口间跳转

由于你可以用垂直分割和水平分割命令打开任意多的窗口，你就几乎能够任意设置窗口的布局。接着，你可以用下面的命令在窗口之间跳转：

```
CTRL-W h        跳转到左边的窗口
CTRL-W j        跳转到下面的窗口
CTRL-W k        跳转到上面的窗口
CTRL-W l        跳转到右边的窗口

CTRL-W t        跳转到最顶上的窗口
CTRL-W b        跳转到最底下的窗口
```

你可能已经注意到这里使用移动光标一样的命令用于跳转窗口。如果你喜欢，改用方向键也行。

还有其它命令可以跳转到别的窗口，参见： Q_wi 。

08.5 移动窗口

你已经分割了一些窗口，但现在的位置不正确。这时，你需要一个命令用于移动窗口。例如，你已经打开了三个窗口，像这样：

```
+-----------------------------------+
|/* file two.c */                   |
|~                                  |
|~                                  |
 two.c============================
|/* file three.c */                 |
|~                                  |
|~                                  |
 three.c==========================
|/* file one.c */                   |
|~                                  |
 one.c============================
|                                   |
+-----------------------------------+
```

显然，最后一个窗口应该在最上面。移动到那个窗口（用 CTRL-W w）并输入如下命令：

```
CTRL-W K
```

这里使用大写的 K。这样窗口将被移到最上面。你可以注意到，这里又用 K 表示向上移动了。

如果你用的是垂直分割，CTRL-W K 会使当前窗口移动到上面并扩展到整屏的宽度。假设你的布局如下：

```
+-----------------------------------------+
|/* two.c */  |/* three.c */  |/* one.c */  |
|~            |~             |~            |
|~            |~             |~            |
|~            |~             |~            |
|~            |~             |~            |
|~            |~             |~            |
 two.c=========three.c=========one.c========
|                                         |
+-----------------------------------------+
```

当你在中间的窗口（three.c）中使用 CTRL-W K 后，结果会是：

```
+-----------------------------------------+
|/* three.c */                            |
|~                                        |
|~                                        |
 three.c=================================
|/* two.c */           |/* one.c */        |
|~                     |~                 |
 two.c==================one.c=============
|                                         |
+-----------------------------------------+
```

还有三个相似的命令（估计你已经猜出来了）：

```
CTRL-W H        把当前窗口移到最左边
CTRL-W J        把当前窗口移到最下边
CTRL-W L        把当前窗口移到最右边
```

08.6 对所有窗口执行命令

你打开了几个窗口，现在你想退出 Vim，你可以分别关闭每一个窗口。更快的方法是：

```
:qall
```

这表示 "quit all"（全部退出）。如果任何一个窗口没有存盘，Vim 都不会退出。同时光标会自动跳到那个窗口，你可以用 ":write" 命令保存该文件或者 ":quit!" 放弃修改。

如果你知道有窗口被改了，而你想全部保存，则执行如下命令：

```
:wall
```

这表示 "write all"（全部保存）。但实际上，它只会保存修改过的文件。Vim 知道保存

一个没有修改过的文件是没有意义的。
    另外，还有 ":qall" 和 "wall" 的组合命令：

```
:wqall
```

这会保存所有修改过的文件并退出 Vim 。
    最后，下面的命令由于退出 Vim 并放弃所有修改：

```
:qall!
```

注意，这个命令是不能撤销的。

为 所 有 的 参 数 打 开 窗 口

要让 Vim 为每个文件打开一个窗口，可以使用 "-o" 参数：

```
vim -o one.txt two.txt three.txt
```

这个结果会是：

```
+--------------------------------+
|file one.txt                    |
|~                               |
 one.txt=========================
|file two.txt                    |
|~                               |
 two.txt=========================
|file three.txt                  |
|~                               |
 three.txt=======================
|                                |
+--------------------------------+
```

"-O" 参数用于垂直分割窗口。
    如果 Vim 已经启动了，可以使用 ":all" 命令为参数列表中的每个文件打开一个窗口。":vertical all" 以垂直分割的方法打开窗口。

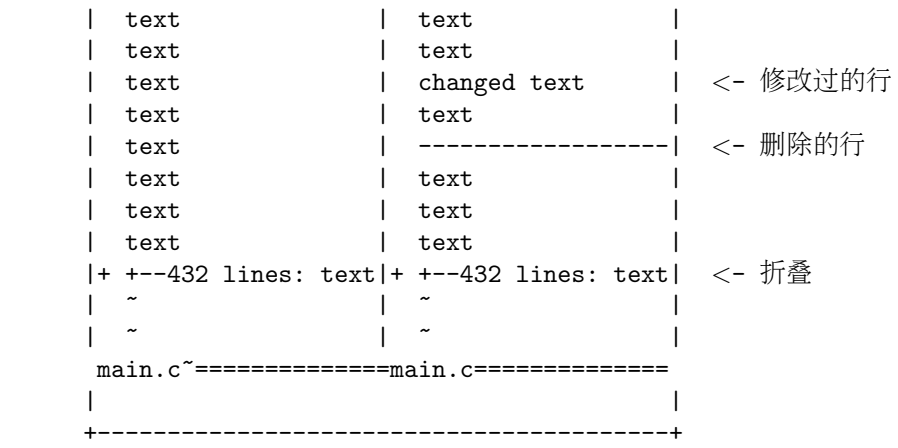
08.7 用 vimdiff 显示文件差异

有一种特殊的启动 Vim 的方法可以用来显示两个文件的差异。让我们打开一个 "main.c" 并插入一些字符。在设置了 'backup' 选项的情况下保存这个文件，以便产生 "main.c~" 备份文件。
    在命令行中输入如下命令：(不是在 Vim 中)

```
vimdiff main.c~ main.c
```

Vim 会用垂直分割的方式打开两个文件。你只能看到你修改过的地方和上下几行的地方。

```
 VV                    VV
+----------------------------------------+
|+ +--123 lines: /* a|+ +--123 lines: /* a|  <- 折叠
|  text              |  text              |
```



(这幅图没有显示出高亮效果，可以使用 vimdiff 命令看到更好的效果)

那些没有修改的行会被折叠成一行，这称为 "关闭的折叠" (closed fold)。上图中由 "<- 折叠" 标记的行就是一个用一行表示 123 行的折叠。这些行在两个文件中完全相同。

标记为 "<- 修改过的行" 被高亮显示，而增加的行被用另一种颜色表示。这可以很清楚地表示出两个文件间的不同。

被删除的行在 main.c 窗口中用 "---" 显示，如图中用 "<- 删除的行" 标记的行。这些字符并不是真的存在。它们只是用于填充 main.c，以便与另一个窗口对齐。

折 叠 栏

每个窗口在左边都有一个颜色略有不同的显示栏，图中标识为 "VV"。你会发现每个折叠在那个位置都有一个加号。把鼠标移到那里并按左键可以打开那个折起，从而让你看到里面的内容。

对于打开的折叠，折叠栏上会出现一个减号。如果你单击那个减号，折叠会被重新关闭。

当然，这只能在你有鼠标的情况下使用。如果你没有，可以用 "zo" 打开一个折叠。关闭使用 "zc"。

用 Vim 做 比 较

启动比较模式的另一种方法从 Vim 内部开始：编辑 "main.c" 文件，然后分割窗口显示区别：

```
:edit main.c
:vertical diffsplit main.c~
```

":vertical" 命令使窗口用垂直的方式分割。如果你不写这个命令，结果会变成水平分割。

如果你有一个当前文件的补丁或者 diff 文件，你可以用第三种方法启动比较模式：先编辑这个文件，然后告诉 Vim 补丁文件的名称：

```
:edit main.c
:vertical diffpatch main.c diff
```

警告：补丁文件中必须仅包括为一个目标文件所做的补丁，否则你可能会得到一大堆错误信息。还可能有些你没打算打补丁的文件也被打了补丁。

补丁功能只改变内存中的文件备份，不会修改你硬盘上的文件（除非你决定写入改动）。

滚动绑定

当文件中有很多改动时，你可以用通常的方式滚动屏幕。Vim 会尽可能保持两个文件对齐，以便你可以并排看到文件的区别。

如果暂时想关闭这个特性，使用如下命令：

```
:set noscrollbind
```

跳转到修改的地方

如果你通过某种方法取消了折叠功能，可能很难找到有改动的地方。使用如下命令可以跳转到下一个修改点：

```
]c
```

反向跳转为：

```
[c
```

加上一个计数前缀可以跳得更远。

消除差异

你可以把文本从一个窗口移到另一个，并以此来消除差异，或者为其中一个文件中增加几行。Vim 有时可能无法及时更新高亮显示。要修正这种问题，使用如下命令：

```
:diffupdate
```

要消除差异，你可以把一个高亮显示的块从一个窗口移动到另一个窗口。以上面的 "main.c" 和 "main.c~" 为例，把光标移到左边的窗口，在另一个窗口中被删除的行的位置，执行如下命令：

```
:dp
```

这将把文字从左边拷到右边，从而消除两边的差异。"dp" 代表 "diff put"。

你也可以反过来做：把光标移到右边的窗口，移到被 "改动" 了的行上，然后执行如下命令：

```
:do
```

这把文本从左边拷到右边，从而消除差异。
由于两个文件已经没有区别了，Vim 会把所有文字全部折叠起来。"do" 代表 "diff obtain"。本来用 "dg"（diff get）会更好。可是它已经有另外的意思了（"dgg" 删除从光标为止到首行的所有文本）。

要了解更多的比较模式的内容，参见 vimdiff 。

08.8 杂项

'laststatus' 选项用于指定什么时候对最后一个窗口显示状态条：

```
0       永远不
1       只有用分割窗口的时候（默认）
2       永远有
```

很多编辑另一个文件的命令都有一个使用分割窗口的变体。对于命令行命令，这通过前置一个 "s" 实现。例如 ":tag" 用来跳到一个标记，"stag" 就会分割出一个新窗口并跳到那个标记。

对于普通模式，前置一个 CTRL-W 可以完成这个功能。例如，CTRL-^ 跳到轮换文件，而 CTRL-W CTRL-^ 打开一个新窗口并编辑轮换文件。

'splitbelow' 选项可以让新的窗口出现在当前窗口的下面。'splitright' 选项让垂直分割的窗口出现在当前窗口的右边。

打开一个新窗口时可以在命令前加上一个修饰符说明新窗口应该出现在什么地方：

```
:leftabove {cmd}        当前窗口的左上方
:aboveleft {cmd}        同上
:rightbelow {cmd}       当前窗口的右下方
:belowright {cmd}       同上
:topleft {cmd}          整个 Vim 窗口的最上面或者最左边
:botright {cmd}         整个 Vim 窗口的最下面或者最右边
```

08.9 标签页

你会注意到窗口永远不会重叠。这意味着屏幕空间很快会用完。这个问题的解决方法叫做标签页。

假设你正在编辑文件 "thisfile"。下面的命令可以建立新的标签页：

```
:tabedit thatfile
```

这会在一个窗口中编辑文件 "thatfile"，这个窗口会占满整个 Vim 窗口。你会注意到在顶部有一个含有两个文件名的横条：

```
+----------------------------------+
| thisfile | /thatfile/ _________X|   (thatfile 用加粗字体出现)
|/* thatfile */                    |
|that                              |
|that                              |
|~                                 |
|~                                 |
|~                                 |
|                                  |
+----------------------------------+
```

现在，你拥有了两个标签页。第一个是文件 "thisfile" 的窗口，第二个是文件 "thatfile" 的窗口。这就像是两张重叠的纸，它们所带的的标签露在外面，显示其文件名。

现在，使用鼠标单击顶端的 "thisfile"。结果是

```
+----------------------------------+
| /thisfile/ | thatfile _________X|   (thisfile 用加粗字体出现)
|/* thisfile */                    |
|this                              |
|this                              |
|~                                 |
|~                                 |
|~                                 |
|                                  |
+----------------------------------+
```

你可以通过单击顶端的标签切换标签页。如果没有鼠标或者不想用它，可以使用 "gt" 命令。助记符：Goto Tab。

现在，让我们通过下面的命令建立另一个标签页：

```
:tab split
```

这会建立一个新的标签页，包含一个窗口，编辑和刚才所在窗口中的缓冲区相同的缓冲区：

```
+-------------------------------------+
| thisfile | /thisfile/ | thatfile __X|  (thisfile 用加粗字体出现)
|/* thisfile */                       |
|this                                 |
|this                                 |
|~                                    |
|~                                    |
|~                                    |
|                                     |
+-------------------------------------+
```

在任何打开窗口的 Ex 命令前面，你都可以放上 ":tab"。这个窗口在新标签页中打开。另一个例子：

```
:tab help gt
```

它将在新的标签页中显示关于 "gt" 的帮助。

使用标签页可以完成更多的工作：

- 在末尾标签后面的空白处单击鼠标
  选择下个标签页，同 "gt"。

- 在右上角的 "X" 处单击鼠标
  关闭当前标签页，除非当前标签页中的改变没有保存。

- 在标签行上双击鼠标

建立新标签页。

- "tabonly" 命令
  关闭除了当前标签页以外的所有标签页，除非其它标签页中的改变没有保存。

关于标签页更多的信息，参见 tab-page 。

下一章： usr_09.txt 使用 GUI 版本

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: Nek_in http://vimcdoc.sf.net

使用 GUI 版本

Vim 能在一般的终端中很好地工作。gVim 则提供了图形用户界面 (GUI)。它可以完成相同，甚至更多的功能。GUI 能提供菜单，工具条，滚动条和其它东西。本章介绍 GUI 这些额外的功能。

09.1 GUI 版本的组件
09.2 使用鼠标
09.3 剪贴板
09.4 选择模式

下一章: usr_10.txt 做大修改
前一章: usr_08.txt 分割窗口
目录: usr_toc.txt

09.1 GUI 版本的组件

你可以在你的桌面上放一个启动 gvim 的图标。此外，下面的任一个命令也可以启动 gvim:

```
gvim file.txt
vim -g file.txt
```

如果这样不行，可能是因为你的 Vim 不支持 GUI 版本特性。你需要先安装一个合适的版本。

执行命令后，Vim 会打开一个窗口，并显示文件 "file.txt"。窗口的样子取决于 Vim 的版本。一般是下面这个样子 (尽可能地用 ASCII 码展示):

```
+------------------------------------------------+
| file.txt + (~/dir) - VIM                    X  |  <- 窗口标题
+------------------------------------------------+
| File  Edit  Tools  Syntax  Buffers  Window  Help  |  <- 菜单栏
+------------------------------------------------+
| aaa  bbb  ccc  ddd  eee  fff  ggg  hhh  iii  jjj  |  <- 工具栏
| aaa  bbb  ccc  ddd  eee  fff  ggg  hhh  iii  jjj  |
+------------------------------------------------+
| file text                                   | ^ |
| ~                                           | # |
| ~                                           | # |  <- 滚动条
| ~                                           | # |
| ~                                           | # |
| ~                                           | # |
|                                             | V |
+------------------------------------------------+
```

最大的一片空间是文件的内容。这部分与终端上看到的是一样的，只是颜色和字体可能有一点差别。

窗 口 标 题

窗口最顶上是窗口标题。这由你的窗口系统绘制。Vim 会在这个标题上显示当前文件的相关信息。首先显示的是文件名，然后是一个特殊字符，最后是用括号括住的目录名。下面是这些特殊字符的含义：

-	文件不能被修改（例如帮助文件）
+	已经被修改过
=	文件只读
=+	文件只读，但仍被修改过

如果没有显示任何特殊字符，表示这是一个普通的，没有改过的文件。

菜 单 栏

你知道菜单是怎么工作的，是吧？ Vim 有些通用的菜单，外加一些特别的。逐个看看，猜测一下这些菜单都可以用来干什么。另一个相关的子菜单是 Edit/Global，你可以在那里找到这些菜单项：

Toggle Toolbar	使工具条可见/不可见
Toggle Bottom Scrollbar	使底部的滚动条可见/不可见
Toggle Left Scrollbar	使左边的滚动条可见/不可见
Toggle Right Scrollbar	使右边的滚动条可见/不可见

在大多数系统里，你可以把菜单 "撕下来"。选中菜单最上面的菜单栏，就是那个看起来像条虚线的。这样你可以得到一个分离的菜单，里面包括了所有菜单项。它会一直挂在那里，直到你关闭它。

工 具 栏

这里包括使用最频繁的操作的图标。希望这些图标功能显而易见。另外，每个图标都支持 "工具提示"（把鼠标移上去停一会儿就能看见这个提示）

"Edit/Global Settings/Toggle Toolbar" 菜单项可以关闭工具条。如果你从来都不使用工具条，可以在 vimrc 文件中加上：

```
:set guioptions-=T
```

这个命令从 'guioptions' 中删除 "T" 标记。其它 GUI 部件也可以通过这种方法激活或关闭。参见这个选项的相关帮助。

滚 动 条

默认情况下，右边会有一个滚动条，它的作用是很明显的。当你分割窗口的时候，每个窗口都会有自己的滚动条。

你可以通过 "Edit/Global Settings/Toggle Bottom Scrollbar" 来启动一个水平滚动条。这在比较模式或没有设置 'wrap' 时非常有用（后面有更多描述）。

在使用垂直分割的时候，只有右边的窗口有滚动条，但当你把光标移到左边的窗口上，右

边的滚动条会对这个窗口起作用，这需要一些时间去适应。

当你使用垂直分割的时候，可以考虑把滚动条放在左边。这可以通过菜单激活，或者使用 'guioptions' 选项：

```
:set guioptions+=l
```

这是在 'guioptions' 中增加 'l' 标志位。

09.2 使用鼠标

标准是好东西。在微软的 Windows 操作系统中，你可以用标准模式选中文本。X Windows 也有一套使用鼠标的标准。非常不幸，这两套标准是不同的。

幸运的是，你可以定制 Vim。你可以让你的鼠标行为像 X Windows 或者像微软 Windows 的鼠标。下面的命令使鼠标用起来像 X Windows：

```
:behave xterm
```

而如下命令使鼠标用起来像微软 Windows：

```
:behave mswin
```

在 UNIX 操作系统中，默认的鼠标行为是 xterm。而默认的微软 Windows 系统的鼠标行为是在安装的时候选定的。要了解这两种行为的详细信息，请参考 :behave 。下面是一些摘要：

XTERM 鼠 标 行 为

左键单击	定位光标
左键拖动	在可视模式下选中文本
中键单击	从剪贴板中粘贴文本
右键单击	把选中的文本扩展到当前的光标位置

微软 Windows 鼠标行为

左键单击	定位光标
左键拖动	在选择模式下选中文本（参见 09.4 ）
按住 Shift，左键单击	把选中的文本扩展到当前的光标位置
中键单击	从剪贴板中粘贴文本
右键单击	显示一个弹出式菜单

可以进一步定制鼠标。请参见下面的选项：

'mouse'	鼠标的使用模式
'mousemodel'	鼠标单击的效果
'mousetime'	双击的间隔允许时间
'mousehide'	输入的时候隐藏鼠标
'selectmode'	鼠标启动可视模式还是选择模式

09.3 剪贴板

04.7 节已经介绍过剪贴板的基本使用了。这里有一个重要的地方要解释一下：对于 X-windows 系统，有两个地方可以在程序间交换文本，而 MS-Windows 不是这样的。

在 X-Windows，有一个 "当前选择区" 的概念。它表示正被选中的文本。在 Vim 中，这表示可视区（假定你正使用默认的设置）。不需要任何其它操作，你就可以把这些文本贴到别的程序中。

例如，你用鼠标在本文中选中一些文本。Vim 会自动切换到可视模式，并高亮这些文本。现在启动另一个 gvim，（由于没有指定文件名，它会显示出一个空窗口）。点击鼠标中键。被选中的文本就会被贴进来。

"当前选择区" 会一直保持有效直到你选中其它文本。在另一个 gvim 中粘贴文本后，在这个窗口中选中一些文字，你会发现上一个窗口中选中的文字显示的方法跟原来有些区别了，这表示这些文字已经不是 "当前选择区" 了。

你不一定要用鼠标来选中文字，用键盘的 "可视" 命令也能达到相同的效果。

"真" 剪 贴 板

对于另一个交换文本的地方，我们称之为 "真" 剪贴板以避免与上面的 "当前选择区" 混淆。通常 "当前选择区" 和 "真" 剪贴板都称为剪贴板，你需要习惯这些名称。

要把文字拷贝到真剪贴板，在一个 gvim 中选中一些文本，然后执行菜单命令 Edit/Copy。这样文字就被拷贝到真剪贴板了。剪贴板的内容是不可见的，除非你使用特别的显示程序，例如 KDE 的 klipper 程序。

现在，切换到另一个 gvim，把光标停在某个位置，然后执行菜单命令 Edit/Paste 菜单。你会看到真剪贴板中的内容被插入到当前的光标位置。

使 用 两 种 剪 贴 板

这种同时使用 "当前选择区" 和 "真剪贴板" 的操作方式听起来很乱。但这是很有用的。我们通过一个例子来说明。用 gvim 打开一个文件并执行如下命令：

- 在可视模式下选中两个词
- 使用 Edit/Copy 菜单把这些词拷到剪贴板
- 再用可视模式选中另一个词
- 执行 Edit/Paste 菜单命令。这样第二次选中的词会被前面剪贴板中的词代替。
- 把鼠标移到另一个地方按中键，你会发现你刚被覆盖的单词被粘贴到新的位置。

如果你小心使用 "当前选择区" 和 "真剪贴板" 两个工具，你可以完成很多很有用的工作。

使 用 键 盘

如果你不喜欢使用鼠标，你可以通过两个寄存器来使用 "当前选择区" 和 "真剪贴板" 两个剪贴板。"* 寄存器用于表示当前选择区。

要使文本变成 "当前选择区"，只要使用可视模式即可。例如，要选中一整行只要输入 "V"。

要拷贝当前选择区的内容：

```
"*P
```

注意 这里 "P" 是大写，表示把文字拷贝到光标的前面。

"+ 寄存器用于真剪贴板。例如，要把当前光标位置到行末的文本拷到真剪贴板：

```
"+y$
```

记得吧，"y" 是 yank，这是 Vim 的拷贝命令。
要把真剪贴板的内容拷到光标前面：

```
"+P
```

这与 "当前选择区" 一样，只是用 (+) 寄存器取代了 (*) 寄存器。

09.4 选择模式

现在介绍一些在 MS-Windows 中比在 X-Windows 中更常被使用的东西 (但在两个系统上都可用)。你已经了解可视模式了。选择模式与可视模式相似，也是用来选中文字的。但有一个显著区别：当输入文本的时候，在选择模式下，被选中的文字将被替换成新输入的文字。

要启用选择模式，先要激活它 (对于 MS-Windows，可能已经激活了，不过多做一次也没什么)：

```
:set selectmode+=mouse
```

现在用鼠标选中一些文本，这些文本会好像可视模式一样被高亮。现在敲入一个字母。被选中的文本被删除，替换成新的字母。现在已经是插入模式了，你可以继续输入。

由于输入普通文本导致选中的文字被删除，这时你不能使用 "hjkl"，"w" 等移动命令。这时可以使用 "Shift" 加功能键。<S-Left> (shift 键加左箭头) 使光标左移。选中的文字像可视模式一样被扩展或者减少。其它箭头起的作用你也可以猜到了，<S-End> 和 <S-Home> 也一样。

你可以通过 'selectmode' 选项修改选择模式的工作方式。

下一章： usr_10.txt 作大修改

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

做大修改

第四章我们已经介绍过作小修改的方法了。本章开始介绍如何重复多次修改和如何改动大量的文字。这将包括使用可视模式处理一些文本块，还有使用一个外部程序去完成非常复杂的功能。

10.1 记录与回放命令
10.2 替换
10.3 命令范围
10.4 global 命令
10.5 可视列块模式
10.6 读、写部分文件内容
10.7 排版文本
10.8 改变大小写
10.9 使用外部程序

下一章： usr_11.txt 从崩溃中恢复
前一章： usr_09.txt 使用 GUI 版本
目录： usr_toc.txt

10.1 记录与回放命令

"." 命令重复前一个修改操作。但如果你需要作一些更复杂的操作它就不行了。这时，记录命令就变得很有效。这需要三个步骤：

1. "q{register}" 命令启动一次击键记录，结果保存到 {register} 指定的寄存器中。寄存器名可以用 a 到 z 中任一个字母表示。
2. 输入你的命令。
3. 键入 q（后面不用跟任何字符）命令结束记录。

现在，你可以用 "@{register}" 命令执行这个宏。

现在看看你可以怎么用这些命令。假设你有如下文件名列表：

```
stdio.h
fcntl.h
unistd.h
stdlib.h
```

而你想把它变成这样：

```
#include "stdio.h"
#include "fcntl.h"
#include "unistd.h"
#include "stdlib.h"
```

先移动到第一行，接着执行如下命令：

qa	启动记录，并使用寄存器 a
^	移到行首
i#include "<Esc>	在行首输入 #include "
$	移到行末
a"<Esc>	在行末加上双引号 (")
j	移到下一行
q	结束记录

现在，你已经完成一次复杂的修改了。你可以通过重复三次 "@a" 完成余下的修改。

"@a" 命令可以通过计数前缀修饰，使操作重复指定的次数。在本例中，你可以输入：

```
3@a
```

移 动 并 执 行

你可能有多个地方需要修改。只要把光标移动到相应的位置并输入 "@a" 命令即可。如果你已经执行过一次，你可以用 "@@" 完成这个操作，这更容易输入一些。例如，你上次使用 "@b" 命令引用了寄存器 b，下一个 "@@" 命令将使用寄存器 b。

如果你对回放命令和 "." 命令作一个比较，你会发现几个区别。首先，"." 只能重复一次改动。而在上例中，"@a" 可以重复多次改动，还能够执行移动操作。第二， "." 只能记住最后一次变更操作。而寄存器执行命令允许你记录任何操作并使用像 "@a" 这样的命令回放这些被记录的操作。最后，你可以使用 26 个寄存器，因此，你可以记录多达 26 个命令序列。

使 用 寄 存 器

用来记录操作的寄存器与你用来拷贝文本的寄存器是相同的。这允许你混合记录操作和其它命令来操作这些寄存器。

假设你在寄存器 n 中记录了一些命令。当你通过 "@n" 执行这些命令时，你发现这些命令有些问题。这时你可以重新录一次，但这样你可能还会犯其它错误。其实，你可以使用如下窍门：

G	移到行尾
o<Esc>	建立一个空行
"np	拷贝 n 寄存器中的文本，你的命令将被拷到整个文件的结尾
{edits}	像修改普通文本一样修改这些命令
0	回到行首
"ny$	把正确的命令拷贝回 n 寄存器
dd	删除临时行

现在你可以通过 "@n" 命令执行正确的命令序列了。(如果你记录的命令包括换行符，请调整上面例子中最后两行的操作来包括所有的行。)

追 加 寄 存 器

到此为止，我们一直使用小写的寄存器名。要附加命令到一个寄存器中，可以使用大写的寄存器名。

假设你在寄存器 c 中已经记录了一个修改一个单词的命令。它可以正常工作，但现在你需要附加一个搜索命令以便找到下一个单词来修改。这可以通过如下命令来完成：

```
qC/word<Enter>q
```

启动 "qC" 命令可以对 c 寄存器追加记录。由此可见，记录到一个大写寄存器表示附加命令到对应的小写寄存器。

这种方法在宏记录，拷贝和删除命令中都有效。例如，你需要把选择一些行到一个寄存器中，可以先这样拷贝第一行：

```
"aY
```

然后移到下一个要拷贝的地方，执行：

```
"AY
```

如此类推。这样在寄存器 a 中就会包括所有你要拷贝的所有行。

10.2 替换 find-replace

":substitute" 命令使你可以在连续的行中执行字符串替换。下面是这个命令的一般形式：

```
:[range]substitute/from/to/[flags]
```

这个命令把 [range] 指定范围中的字符串 "from" 修改为字符串 "to"。例如，你可以把连续几行中的 "Professor" 改为 "Teacher"，方法是：

```
:%substitute/Professor/Teacher/
```

备注：
很少人会把整个 ":substitute" 命令完整敲下来。通常，使用命令的缩写形式 ":s" 就行了。下文我们将使用这个缩写形式。

命令前面的 "%" 表示命令作用于全部行。如果不指定行范围，":s" 命令只作用在当前行上。 10.3 将对 "行范围" 作深入的介绍。

默认情况下，":substitute" 命令只对某一行中的第一个匹配点起作用。例如，前面例子中会把行：

```
Professor Smith criticized Professor Johnson today.
```

修改成：

```
Teacher Smith criticized Professor Johnson today.
```

要对行中所有匹配点起作用，你需要加一个 g (global，全局) 标记。下面命令：

```
:%s/Professor/Teacher/g
```

对上面例子中的句子的作用效果如下：

```
Teacher Smith criticized Teacher Johnson today.
```

":s" 命令还支持其它一些标志位，包括 "p" (print，打印)，用于在命令执行的时候打印出最后一个被修改的行。还有 "c" (confirm，确认) 标记会在每次替换前向你询问是否需要替换。执行如下命令：

```
:%s/Professor/Teacher/c
```

Vim 找到第一个匹配点的时候会向你提示如下：

```
replace with Teacher (y/n/a/q/l/^E/^Y)?
```

(中文翻译如下：

```
替换为 Teacher 么 (y/n/a/q/l/^E/^Y)?
```

)

这种时候，你可以输入如下回答中的一个：

y	Yes，是；执行替换
n	No，否；跳过
a	All，全部；对剩下的匹配点全部执行替换，不需要再确认
q	Quit，退出；不再执行任何替换
l	Last，最后；替换完当前匹配点后退出
CTRL-E	向上滚动一行
CTRL-Y	向下滚动一行

":s" 命令中的 "from" 部分实际上是一个 "匹配模式" (还记得吗？这是我们前面给 pattern 起的名字译者)，这与查找命令一样。例如， 要替换行首的 "the" 可以这样写：

```
:s/^the/these/
```

如果你要在 "from" 或者 "to" 中使用正斜杠，你需要在前面加上一个反斜杠。更简单的方法是用加号代替正斜杠。例如：

```
:s+one/two+one or two+
```

10.3 命令范围

":substitute" 命令和很多其它的 ":" 命令一样，可以作用于选中的一些行。这称为一个 "范围"。

最简单的范围表达形式是 "{number},{number}"。例如：

```
:1,5s/this/that/g
```

这会在 1 到 5 行上执行替换命令。(包括第 5 行)。"范围" 总是放在一个命令的前面。

如果只用一个数值，表示某个指定的行：

```
:54s/President/Fool/
```

有些命令在不指定范围的时候作用于整个文件。要让它只作用于当前行可以用当前行范围

标识 "."。":write" 命令就是这样：不指定范围的时候，它写入整个文件，如果要仅写入当前行，可以这样：

```
:.write otherfile
```

文件的第一行行号总是 1，最后一行又是多少呢？"$" 字符用于解决这个问题。例如，要修改当前行到文件末的全部内容，可以这样：

```
:.,$s/yes/no/
```

我们前面使用的 "%" 就是 "1,$" 的缩写形式，表示从文件首到文件末。

在范围中使用模式

假设你正在编辑一本书中的一章，并且想把所有的 "grey" 修改成 "gray"。但你只想修改这一章，不想影响其它的章节。另外，你知道每章的开头的标志是行首的单词为 "Chapter"。下面的命令会对你有帮助：

```
:?^Chapter?,/^Chapter/s=grey=gray=g
```

你可以看到这里使用了两个查找命令。第一个是 "?^Chapter?"，用于查找前一个行首的 "Chapter"，就是说 "?pattern?" 用于向前查找。同样，"/^Chapter/" 用于向后查找下一章。

为了避免斜杠使用的混淆，在这种情况下，"=" 字符用于代替斜杠。使用斜杠或使用其它字符其实也是可以的。

加减号

上面的方案其实还是有问题的：如果下一章的标题行中包括 "grey"，这个 "grey" 也会被替换掉。如果你正好想这样就最好，可是正好你不想呢？这个时候你需要指定一个偏移。

要查找一个模式，并且使用它的前一行，需要这样：

```
/Chapter/-1
```

你可以用任意数值代替命令中的 1。要定位匹配点下的第二行，要这样：

```
/Chapter/+2
```

偏移还可以用于其它范围指定符。看一下下面这个例子：

```
:.+3,$-5
```

这指定当前行下面第三行到文件末倒数第五行的范围。

使用标记

除了指定行号，(这需要记住并把它敲出来)，你还可以使用标记。

在前面的例子中，你可以用标记指出第三章的位置。例如，用 "mt" 标记开头，再用 "mb" 标记结尾。然后你就可以用标记表示一个范围 (包括标记的那一行)：

```
:'t,'b
```

可视模式和范围

你可以在可视模式中选中一些行。如果你现在输入 ":" 启动冒号命令模式，你会看到：

```
:'<,'>
```

现在，你可以输入剩下的命令，这个命令的作用范围就是可视模式中指定的范围。

> 备注：
> 如果使用可视模式选中行的一部分，或者用 CTRL-V 选中一个文本列块，然后执行冒号命令，命令仍作用于整行，而不只是选中的范围。这可能会在以后的版本中修正。

'< 和 '> 实际上是标记，分别标识可视模式的开始和结尾。这个标记一直有效，直到选中了其它的范围为止。你还可以用标记跳转命令 "'<" 跳转到选中文本的开始处。你还可以把这个标记和其它标记混合，例如：

```
:'>,$
```

这表示从选中部分的结尾到文件末。

指定行数

如果你知道要修改多少行，你可以先输入一个数值再输入冒号。例如，如果你输入 "5:"，你会得到：

```
:.,.+4
```

现在你可以继续你的命令，这个命令将作用于当前行及其后 4 行。

10.4 global 命令

":global" 命令是 Vim 中一个更强大的命令（之一）。它允许你找到一个匹配点并且在那里执行一个命令。它的一般形式是：

```
:[range]global/{pattern}/{command}
```

这有点像 ":substitute" 命令。只是它不替换文本，而是执行 {command} 指定的命令。

> 备注：
> global 中执行的命令只能是冒号命令。普通模式命令不能在这里使用。如果需要，可以使用 :normal 命令。

假设你要把 "foobar" 修改为 "barfoo"，但只需要修改 C++ 风格的注释中的内容。这种注释以 "//" 开头。所以可以使用如下命令：

```
:g+//+s/foobar/barfoo/g
```

这个命令用 ":g" 开头，这是 ":global" 的缩写形式，就像 ":s" 是 ":substitute" 的

缩写形式一样。然后是一个匹配模式，由于模式中包括正斜杠，我们用加号作分隔符，后面是一个把 "foobar" 替换成 "barfoo" 的替换命令。

全局命令的默认范围是整个文件，所以这个例子中没有指定范围。这一点与 ":substitute" 是不同的。后者只作用于一行。

这个命令并非完美。因为 "//" 可能出现在一行的中间，但替换命令会把前后的匹配点都替换了。

像 ":substitute" 一样，这里也可以使用各种各样的匹配模式。当你从后面的章节中学会更多的关于模式的知识，它们都可以用在这里。

10.5 可视列块模式

CTRL-V 命令可以选中一个矩形文本块。有几个命令是专门用来处理这个文本块的。

在可视列块模式中，"$" 命令有些特别。当最后一个移动命令是 "$" 时，整个可视列块将被扩展到每一行的行尾。这种状态在你使用垂直移动命令的时候一直被保持，直到你使用水平移动命令为止。就是说，用 "j" 命令会保持这种状态，而 "h" 会退出。

插 入 文 本

"I{string}<Esc>" 命令把 {string} 插到可视列块的每一行的左边。你用 CTRL-V 进入可视列块模式，然后移动光标定义一个列块。接着输入 I 进入插入模式，并随后输入文本。这时，你输入的内容只出现在第一行。

然后你输入 <Esc> 结束输入，刚才输入的字符串将神奇地出现在每一行的可视区的左边。例如：

```
include one
include two
include three
include four
```

把光标移到第一行 "one" 的 "o"上，输入 CTRL-V。然后用 "3j" 向下移动到 "four"。现在你选中了四行的一个方块。接着输入：

```
Imain.<Esc>
```

结果将是：

```
include main.one
include main.two
include main.three
include main.four
```

如果选中的块经过一个短行，并且这行没有任何内容包括在可视列块中，则新的文本不会被插入到该行中。例如，对于下面的例子，用可视列块选中第一和第三行的 "long"，这样第二行的文本将不会被包括在可视列块中：

```
This is a long line
short
Any other long line
```

```
     ^^^^ 用可视列块选中的部分
```

现在输入 "Ivery <esc>"。结果将是:

```
This is a very long line
short
Any other very long line
```

可以注意到，第二行中没有插入任何文本。

如果插入的文本中包括一个新行，则 "I" 命令的效果与普通插入语句一样，只影响块的第一行。

"A" 命令的效果与 "I" 命令一样，只是把文字插入可视列块的右边，而且在短行中会插入文字。这样，你有在短行中插入文字与否的不同选择。

"A" 在如下情况会有一些特别: 选中一个可视列块然后用 "$" 命令使可视列块扩展到行尾。然后用 "A" 命令插入文本，文件将被插入到 "每一行" 的行尾。

还是用上面的例子，在选中可视列块后输入 "$A XXX<Esc>"，结果将是:

```
This is a long line XXX
short XXX
Any other long line XXX
```

出现这个效果完全是 "$" 命令的作用，Vim 能记住这个命令，如果你用移动命令选中相同的可视列块，是不会有这样的效果的。

修 改 文 本

可视列块中的 "c" 命令会删除整个可视列块并转入 "插入" 模式，使你可以开始文本，这些文本会被插入可视列块经过的每一行。

在上面的例子中，如果仍选中包括所有 "long" 的一个可视列块，然后输入 "c_LONG_<Esc>"，结果会变成:

```
This is a _LONG_ line
short
Any other _LONG_ line
```

与 "I" 命令一样，短行不会发生变化。而且在插入的过程中，你不能断行。

"C" 命令从块的左边界开始删除所有行的后半段，然后状态切换到 "插入" 模式让你输入文本。新的文本被插入到每一行的末尾。

在上面的例子中，如果命令改为 "Cnew text<Esc>"，你将获得这样的结果:

```
This is a new text
short
Any other new text
```

可以注意到，尽管只有 "long" 被选中，它后面的内容也被删除了。所以在这种情况下，块的左边界才是有意义的。

同样，没有包括在块中的行不会受影响。

还有一些命令只影响被选中的字符:

~	交换大小写	(a -> A 而 A -> a)

U	转换成大写	(a -> A 而 A -> A)
u	转换成小写	(a -> a 而 A -> a)

以一个字符填充

要以某一个字符完全填充整个块，可以使用 "r" 命令。再次选中上例中的文本，然后键入 "rx":

```
This is a xxxx line
short
Any other xxxx line
```

备注：
如果你要在可视列块中包括行尾之后的字符，请参考 25 章的 'virtualedit' 特性。

平移

">" 命令把选中的文档向右移动一个 "平移单位"，中间用空白填充。平移的起始点是可视列块的左边界。

还是用上面的例子，">" 命令会导致如下结果：

```
This is a         long line
short
Any other         long line
```

平移的距离由 'shiftwidth' 选项定义。例如，要每次平移 4 个空格，可以用这个命令：

```
:set shiftwidth=4
```

"<" 命令向左移动一个 "平移单位"，但能移动的距离是有限的，因为它左边的不是空白字符的字符会挡住它，这时它移到尽头就不再移动。

连接若干行

"J" 命令连接被选中的行。也就是删除所有的换行符。其实不只是换行符，行前后的多余空白字符会一起被删除而全部用一个空格取代。如果行尾刚好是句尾，就插入两个空格 (参见 'joinspaces' 选项)

还是用那个我们已经非常熟悉的例子，这回的结果将是：

```
This is a long line short Any other long line
```

"J" 命令其实不关心选中了哪些字符，只关心块涉及到哪些行。所以可视列块的效果与 "v" 和 "V" 的效果是完全一样的。

如果你不想改变那些空白字符，可以使用 "gJ" 命令。

当你在写一封 e-mail，你可能想包括另一个文件。这可以通过 ":read {filename}" 命令达到目的。这些文本将被插入到光标的下面。

我们用下面的文本作试验：

```
Hi John,
Here is the diff that fixes the bug:
Bye, Pierre.
```

把光标移到第二行然后输入：

```
:read patch
```

名叫 "patch" 的文件将被插入，成为下面这个样子：

```
Hi John,
Here is the diff that fixes the bug:
2c2
<       for (i = 0; i <= length; ++i)
---
>       for (i = 0; i < length; ++i)
Bye, Pierre.
```

":read" 支持范围前缀。文件将被插入到范围指定的最后一行的下面。所以 ":$r patch" 会把 "patch" 文件插入到当前文件的最后。

如果要插入到文件的最前面怎么办？你可以把文本插入到第 0 行，这一行实际上是不存在的。在普通的命令的范围中如果你用这个行号会出错，但在 "read" 命令中就可以：

```
:0read patch
```

这个命令把 "patch" 文件插入到全文的最前面。

保存部分行

要把一部分行写入到文件，可以使用 ":write" 命令。在没有指定范围的时候它写入全文，而指定范围的时候它只写入范围指定的行：

```
:.,$write tempo
```

这个命令写入当前位置到文件末的全部行到文件 "tempo" 中。如果这个文件已经存在，你会被提示错误。Vim 不会让你直接写入到一个已存在的文件。如果你知道你在干什么而且确实想这样做，就加一个叹号：

```
:.,$write! tempo
```

小心：":!" 必须紧跟着 ":write"，中间不能留有空格。否则这将变成一个过滤器命令，这种命令我们在本章的后面会介绍。

添加内容到文件中

本章开始的时候介绍了怎样把文本添加到寄存器中。你可以对文件作同样的操作。例如，

把当前行写入文件：

```
:.write collection
```

然后移到下一个位置，输入：

```
:.write >>collection
```

">>" 通知 Vim 把内容添加到文件 "collection" 的后面。你可以重复这个操作，直到获得全部你需要收集的文本。

10.7 排版文本

在你输入纯文本时，自动换行自然会是比较吸引的功能。要实现这个功能，可以设置 'textwidth' 选项：

```
:set textwidth=72
```

你可能还记得在示例 vimrc 文件中，这个命令被用于所有的文本文件。所以如果你使用的是那个配置文件，实际上你已经设置这个选项了。检查一下该选项的值：

```
:set textwidth
```

现在每行达到 72 个字符就会自动换行。但如果你只是在行中间输入或者删除一些东西，这个功能就无效了。Vim 不会自动排版这些文本。

要让 Vim 排版当前的段落：

```
gqap
```

这个命令用 "gq" 开始，作为操作符，然后跟着 "ap"，作为文本对象，该对象表示 "一段" (a paragraph)。"一段" 与下一段的分割符是一个空行。

> 备注：
> 只包括空白字符的空白行不能分割 "一段"。这很不容易分辨。

除了用 "ap"，你还可以使用其它 "动作" 或者 "文本对象"。如果你的段落分割正确，你可以用下面命令排版整个文档：

```
gggqG
```

"gg" 跳转到第一行，"gq" 是排版操作符，而 "G" 是跳转到文尾的 "动作" 命令。

如果你没有清楚地区分段落。你可以只排版你手动选中的行。先移到你要排版的行，执行 "gqj"。这会排版当前行和下面一行。如果当前行太短，下面一行会补上来，否则多余的部分会移到下面一行。现在你可以用 "." 命令重复这个操作，直到排版完所有的文本。

10.8 改变大小写

你手头有一个分节标题全部是小写的。你想把全部 "section" 改成大写的。这可以用 "gU" 操作符。先在第一列执行：

```
                          gUw
section header       ---->       SECTION header
```

"gu" 的作用正好相反：

```
                          guw
SECTION header       ---->       section header
```

你还可以用 "g~" 来交换大小写。所有这些命令都是操作符，所以它们可以用于 "动作" 命令，文本对象和可视模式。

要让一个操作符作用于当前行，可以执行这个操作符两次。例如，"d" 是删除操作符，所以删除一行就是 "dd"。相似地，"gugu" 使整一行变成小写。这可以缩成 "guu"。"gUgU" 可以缩成 "gUU" 而 "g~g~" 则是 "g~~"。例如：

```
                              g~~
Some GIRLS have Fun     ---->    sOME girls HAVE fUN
```

10.9 使用外部程序

Vim 有一套功能非常强大的命令，可以完成所有功能。但有些东西外部命令能够完成得更好或者更快。

命令 "!{motion}{program}" 用一个外部程序对一个文本块进行过滤。换句话说，它用一个文本块作为输入，执行一个由 {program} 指定的外部命令，然后用该程序的输出替代选中的文本块。

如果你不熟悉 UNIX 的过滤程序，上面的描述可以说是比较糟糕的。我们这里举个例子来说明一下。sort 命令能对一个文件排序。如果你执行下面的命令，未排序的文件 input.txt 会被排序并写入 output.txt。(这在 UNIX 和 Microsoft Windows 上都有效)

```
sort <input.txt >output.txt
```

现在在 Vim 中完成相同的功能。假设你要对 1 到 5 行排序。你可以先把光标定位在第一行，然后你执行下面的命令：

```
!5G
```

"!" 告诉 Vim 你正在执行一个过滤操作。然后 Vim 编辑器等待一个 "动作" 命令来告诉它要过滤哪部分文本。"5G" 命令告诉 Vim 移到第 5 行。于是，Vim 知道要处理的是第 1 行 (当前行) 到第 5 行间的内容。

由于在执行一个过滤命令，光标被 Vim 移到了屏幕的底部，并显示一个 "!" 作提示符。现在你可以输入过滤程序的名字，在本例中就是 "sort" 了。因此，你整个命令将是：

```
!5Gsort<Enter>
```

这个命令的结果是 sort 程序用前 5 行作为输入执行，程序的输出替换了原来的 5 行。

```
line 55                      line 11
line 33                      line 22
line 11          -->          line 33
line 22                      line 44
line 44                      line 55
last line                    last line
```

"!!" 命令用于对当前行执行过滤命令。在 Unix 上，"date" 命令能打印当前的时间和日期，所以，"!!date<Enter>" 用 "date" 的输出代替当前行。这在为文件加入时间戳的时候非常有用。

如果命令不执行怎么办

启动一个外壳，发送一个命令并捕获它的输出，这需要 Vim 知道这个外壳程序是怎么工作的。如果你要使用过滤程序，你最好需要检查一下下面的选项：

'shell'	指定 Vim 用于执行外部命令的外壳。
'shellcmdflag'	传给外壳的参数
'shellquote'	外壳程序使用的引号（用于引用命令）
'shellxquote'	用于命令和重定向文件名的引号
'shelltype'	外壳程序的类型（仅用于 Amiga）
'shellslash'	在命令中使用正斜杠（仅用于 MS-Windows 和相容系统）
'shellredir'	用于把命令输出写入文件所使用的字符串

在 Unix 上，这几乎不是问题。因为总共只有两种外壳程序："sh" 类的和 "csh" 类的。Vim 会检查选项 'shell'，并根据它的类型自动设置这些参数。

但在 MS-Windows 上，有很多不同的外壳程序，所以你必须修改这些外壳程序以便过滤功能正常执行。详细情况请参考相应选项的帮助。

读入一个命令的输出

要把当前目录的内容读进文件，可以用如下命令：

Unix 上：
```
:read !ls
```
MS-Windows 上：
```
:read !dir
```

"ls" 或者 "dir" 的输出会被捕获并插入到光标下面。这好像读入一个文件一样，但是需要加上一个 "!" 让 Vim 知道后面是一个命令。

这些命令还可以带参数。而且前面还可以带一个范围用于告诉 Vim 把这行放在什么地方：

```
:0read !date -u
```

这将用 UTC 格式把当前的时间插入到文件开头。(当然了，你的 date 命令必须能够接受 -u 选项。) 注意 这与 "!!date" 的区别："!!date" 替代一行，而 ":read !date" 插入一行。

把文本输出到一个命令

Unix 命令 "wc" 用于统计单词数目。要统计当前文件有多少个单词，可以这样：

```
:write !wc
```

这和前面的写入命令一样，但文件名前面改为一个 "!" 用于告诉 Vim 后面是一个要被执行的外部命令。被写入的文本将作为指定命令的标准输入。这个输出将是：

```
       4      47     249
```

"wc" 命令惜字如金。这表示你有 4 行，47 个单词和 249 个字符。

注意 不要错写成：

```
        :write! wc
```

这会强制把当前文件存到当前目录的 "wc" 文件中。在这里空格的位置是非常重要的！

重 画 屏 幕

如果外部程序产生一个错误信息，屏幕显示就会乱掉。Vim 颇重效率，所以它只刷新那些需要刷新的地方。可是它不可能知道其它程序修改了哪些地方。要强制 Vim 重画整个屏幕：

```
        CTRL-L
```

下一章： usr_11.txt 从崩溃中恢复

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: Nek_in http://vimcdoc.sf.net

从崩溃中恢复

你的计算机崩溃过吗? 是不是还正好在你编辑了几个小时以后? 不要惊慌! Vim 已经保存了大部分的信息使你可以恢复你的大多数数据。本章告诉你怎样恢复这些数据并向你介绍 Vim 是如何处理交换文件的。

11.1 基本恢复
11.2 交换文件在哪
11.3 是不是崩溃了?
11.4 深入阅读

下一章: usr_12.txt 小窍门
前一章: usr_10.txt 做大修改
目录: usr_toc.txt

11.1 基本恢复

在大多数情况下, 恢复一个文件相当简单。假设你知道正在编辑的是哪个文件 (并且硬盘没坏的话)。可以用 "-r" 选项启动 Vim:

```
vim -r help.txt
```

Vim 会读取交换文件 (用于保存你的编辑数据的文件) 并且提取原文的编辑碎片。如果 Vim 恢复了你的改变, 你会看到如下文字 (当然了, 文件名会不一样):

```
Using swap file ".help.txt.swp"
Original file "~/vim/runtime/doc/help.txt"
Recovery completed. You should check if everything is OK.
(You might want to write out this file under another name
and run diff with the original file to check for changes)
You may want to delete the .swp file now.

(译者注: 中文情况下是:
使用交换文件 ".help.txt.swp"
原文件 "~/vim/runtime/doc/help.txt"
恢复完成。请确定一切正常。
(你可能想要把这个文件另存为别的文件名,
再执行 diff 与原文件比较以检查是否有改变)
现在可以删除 .swp 文件。
)
```

为了安全起见, 可以用另一个文件名保存这个文件:

```
:write help.txt.recovered
```

可以把这个文件与原文件作一下比较, 看看恢复的效果如何。这方面 Vimdiff 可以帮很大的忙 (参见 08.7)。例如:

```
:write help.txt.recovered
:edit #
:diffsp help.txt
```

注意 用一个比较新的原文件来比较（你在计算机崩溃前最后保存过的文件），并且检查有没有东西丢失了（由于某些问题导致 Vim 无法恢复）。

如果在恢复的过程中 Vim 显示出一些警告信息，注意 小心阅读。这应该是很少见的。

如果恢复产生的文件和文件内容完全一致，你会看到以下消息：

```
Using swap file ".help.txt.swp"
Original file "~/vim/runtime/doc/help.txt"
Recovery completed. Buffer contents equals file contents.
You may want to delete the .swp file now.
```

通常这是因为你已经恢复过改变，或者修改后写入了文件。此时删除交换文件应该安全。

最后所做的一些修改不能恢复是正常的。Vim 在你停止大约 4 秒不输入的时候或者输入大约两百个字符以后才会更新交换文件。这间可以通过 'updatetime' 和 'updatecount' 两个选项来调整。这样，如果系统崩溃前 Vim 没有更新交换文件，最后一次更新后编辑的内容就会丢失。

如果你编辑的时候没有给定文件名，可以用一个空的字符串来表示文件名：

```
vim -r ""
```

你需要进入原来的目录执行这个命令，否则 Vim 是找不到这个交换文件的。

11.2 交换文件在哪

Vim 可以把交换文件保存在几个不同的地方。通常是原文件所在的目录。要知道这一点，进入该目录，然后输入：

```
vim -r
```

Vim 会列出所有它能找到的交换文件。它还会从其它目录寻找本目录文件的交换文件，但它不会寻找其它目录里的交换文件，更不会遍及整个目录树。

这个命令的输出如下：

```
Swap files found:
   In current directory:
1.    .main.c.swp
          owned by: mool   dated: Tue May 29 21:00:25 2001
        file name: ~mool/vim/vim6/src/main.c
         modified: YES
        user name: mool   host name: masaka.moolenaar.net
      process ID: 12525
   In directory ~/tmp:
      -- none --
   In directory /var/tmp:
```

```
      -- none --
   In directory /tmp:
      -- none --
```

(译者：中文的情形如下：

```
找到以下交换文件：
   位于当前目录：
1.    .main.c.swp
            所有者：mool     日期：Tue May 29 21:00:25 2001
            文件名：~mool/vim/vim6/src/main.c
            修改过：是
            用户名：mool      主机名：masaka.moolenaar.net
           进程 ID：12525
   位于目录 ~/tmp：
        -- 无 --
   位于目录 /var/tmp：
        -- 无 --
   位于目录 /tmp：
        -- 无 --
)
```

如果有几个交换文件，其中一个可能是你要的，Vim 会给出一个文件列表，你需要输入一个表示这个文件的数字。小心检查那几个文件的时间，并确定哪一个才是你需要的。

万一你不知道是哪个的话，一个一个试一试。

使 用 指 定 的 交 换 文 件

如果你知道要用哪个文件，你可以指定交换文件的名字。Vim 会找出交换文件所对应的原始文件的名字。

例如：

```
Vim -r .help.txt.swo
```

这个方法在交换文件在一个非预期的目录中时很有用。Vim 知道 *.s[uvw][a-z] 模式的文件是交换文件。

如果这还不行，看看 Vim 报告的文件名是什么，然后根据需要给文件换名。根据 'directory' 选项的值你可以知道 Vim 会把交换文件放到什么地方。

备注：
Vim 在 'dir' 选项指定的目录中寻找名为 "filename.sw?" 的交换文件。如果通配符不能正常工作（例如 'shell' 选项不正确），Vim 转而尝试文件 "filename.swp"。如果仍失败，你就只能通过给定交换文件的名称来恢复原来的文件了。

11.3 是不是崩溃了？ ATTENTION E325

Vim 尽可能保护你不要做傻事。有时你打开一个文件，天真地以为文件的内容会显示出来。可是，Vim 却给出一段很长的信息：

```
         E325: ATTENTION
Found a swap file by the name ".main.c.swp"
```

```
          owned by: mool   dated: Tue May 29 21:09:28 2001
         file name: ~mool/vim/vim6/src/main.c
          modified: no
         user name: mool   host name: masaka.moolenaar.net
        process ID: 12559 (still running)
While opening file "main.c"
             dated: Tue May 29 19:46:12 2001

(1) Another program may be editing the same file.
    If this is the case, be careful not to end up with two
    different instances of the same file when making changes.
    Quit, or continue with caution.

(2) An edit session for this file crashed.
    If this is the case, use ":recover" or "vim -r main.c"
    to recover the changes (see ":help recovery").
    If you did this already, delete the swap file ".main.c.swp"
    to avoid this message.
```

(译者注：翻译成中文如下：

```
        E325: 注意
发现交换文件 "main.c.swp"
     所有者: mool     日期: 2001年5月29日 星期二 21:09:28
     文件名: ~mool/vim/vim6/src/main.c
     修改过: 否
     用户名: mool   主机名: masaka.moolenaar.net
     进程号: 12559 (仍在运行)
正在打开文件 "main.c"
       日期: 2001年5月29日 星期二 19:46:12

(1) 另一个程序可能也在编辑同一个文件。
    如果是这种情况，修改时请注意避免同一个文件产生两个不同的版本。

    退出，或小心地继续。

(2) 上次编辑此文件时崩溃。
    如果是这样，请用 ":recover" 或 "vim -r main.c"
    恢复修改的内容 (请见 ":help recovery")。
    如果你已经进行了恢复，请删除交换文件 ".main.c.swp"
    以避免再看到此消息。

)
```

你遇到这个信息是因为 Vim 发现你编辑的文件的交换文件已经存在。这一定是有什么地方出问题了。可能的原因有两个：

1. 这个文件正在被另一个进程编辑。注意有 "process ID" 那行。它看起来是这样的：

```
        process ID: 12559 (still running)
```

"still running" 表示同一台计算机上有一个进程正在编辑这个文件。在非 Unix 的系统上你不会得到这个信息。而如果你通过网络编辑这个文件，可能也得不到这个信息，因为那个进程不在你的机器上。在这两种情况下，你要自己找到原因。

如果另一个 Vim 正在编辑这个文件，继续编辑会导致同一个文件有两个版本。最后存盘的文件会覆盖前一个版本。这样的结果是一些编辑数据丢失了。这种情况下，你最好退出这个 Vim。

2. 交换文件可能是由于前一次 Vim 或者计算机崩溃导致的。检查提示信息中的日期。如果交换文件比你正在编辑的文件新，而且出现这个信息：

```
modified: YES
```

这就表明你很可能需要恢复了。

如果文件的日期比交换文件新，可能是在崩溃后被修改过了（也许你已经恢复过，只是没有删除交换文件？），也可能文件在崩溃前保存过，但这发生在在最后一次写入该交换文件之后（那你运气了，你根本不需要这个旧的交换文件）。Vim 会用如下语句提醒你：

```
NEWER than swap file!
```

（译者注：意为 "文件比交换文件新"）

备注 在下面情形下，Vim 知道交换文件没有用了，会自动删除它：
- 文件是合法的交换文件（魔术数正确）。
- 文件修改过的标志位未置位。
- 进程不在运行。

无法读取的交换文件

有时下面这样的信息

```
[cannot be read]
```

或 [无法读取]（中文信息，译者）

会出现在交换文件的文件名之下。这可好可坏，依情况而定。

如果上次编辑在作出任何修改前就崩溃了的话，是好事。这样交换文件的长度为 0。你只要删除之然后继续即可。

如果情况是你对交换文件没有读权限，就比较糟糕。你可能得以只读方式浏览该文件。或者退出。在多用户系统中，如果你以别人的身份登录并做了上一次修改，先退出登录然后以那个身份重新登录可能会 "治愈" 该读取错误。不然的话，你得找出是谁做的上一次修改（或正在修改），然后和那个人聊聊。

如果情况是交换文件所在的磁盘物理性地损坏了，就非常糟糕了。幸运的是，这种情况几乎不会发生。
你可能需要以只读方式查看文件（如果允许的话），看看到底有多少改动被 "忘记" 了。
如果你是改动文件的那个人，准备好重做你的改动。

怎么办？ swap-exists-choices

如果 Vim 版本支持对话框，你可以从对话框的六个选择中挑一个：

```
Swap file ".main.c.swp" already exists!
[O]pen Read-Only, (E)dit anyway, (R)ecover, (Q)uit, (A)bort, (D)elete it:
```

(译者：含义是：
交换文件 ".main.c.swp" 已经存在！
以只读方式打开([O])，直接编辑((E))，恢复((R))，退出((Q))，中止((A))，删除交换文件((D))：
)

O 用只读方式打开文件。当你只是想看看文件的内容，而不打算恢复它的时候用这个选项。你可能知道有人在编辑它，但你想看看它的内容，而不会修改它。

E 直接编辑。小心使用这个选择！如果这个文件已经被另一个文件打开，你编辑它会导致它有两个版本。Vim 已经警告过你了，安全比事后说对不起要好。

R 从交换文件中恢复文件。如果你知道交换文件中有新的东西，而你想恢复它，选择这一项。

Q 退出。不再编辑该文件。在有另一个 Vim 编辑该文件的时候选这一项。
如果你刚打开 Vim，这会退出 Vim。当你用多个窗口打开几个文件，Vim 只会在第一个文件遇到交换文件的时候退出。如果你是通过编辑命令打开文件，该文件不会被载入，Vim 会回到原来的文件中。

A 中止。类似 (Q) 退出，但同时中止更多的命令。这在试图加载一个编辑多个文件的脚本 (例如一个多窗口的会话) 时很有用。

D 删除交换文件。当你能确定你不再需要它的时候选这一项。例如，它不包括修改的数据，或者你的文件比交换文件新。
在 Unix 系统上，只有建立这个交换文件的进程不再运行，这个选择才会出现。

如果没有出现对话框 (你使用的 Vim 不支持对话框)，你只能手工处理。要恢复一个文件，使用如下命令：

```
:recover
```

Vim 不是总能检测到一个文件有交换文件的。当另一个会话把交换文件放到别的位置或者在编辑另一台机器的文件的时候，双方使用的交换文件路径不一样都会发生这个问题。所以，不要老是等 Vim 来提醒你。

如果你确实不想看到这个信息，你可以在 'shortmess' 选项中加上 'A' 标志位。不过一般你不需要这样做。

关于加密和交换文件关系的注释，见 :recover-crypt 。
要通过程序访问交换文件，见 swapinfo() 。

11.4 深入阅读

swap-file	解释交换文件在什么地方创建以及名字是什么。
:preserve	手工刷新交换文件
:swapname	查看当前文件的交换文件
'updatecount'	多少个键被敲下后执行一次交换文件刷新
'updatetime'	交换文件刷新后的超时时间
'swapsync'	交换文件刷新后是否执行磁盘同步

'directory'　　列出用于保存交换文件的目录
'maxmem'　　写入交换文件前的内存使用限制
'maxmemtot'　　同上，当用于所有文件

下一章： usr_12.txt　小窍门

版权：参见 manual-copyright　vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 —by Bram Moolenaar
译者：Nek_in http://vimcdoc.sf.net

小窍门

通过组合一些命令，你可以用 Vim 完成几乎所有的工作。本章将介绍一些有用的命令组合。涉及的命令大都是前面章节介绍过的，但也会有一点新命令。

12.1 单词替换
12.2 把 "Last, First" 改成 "First Last"
12.3 排序
12.4 反转行顺序
12.5 单词统计
12.6 查阅 man 信息
12.7 删除多余空格
12.8 查找单词的使用位置

下一章： usr_20.txt 快速键入命令行命令
前一章： usr_11.txt 从崩溃中恢复
目录： usr_toc.txt

12.1 单词替换

替换命令可以在全文中用一个单词替换另一个单词：

```
:%s/four/4/g
```

"%" 范围前缀表示在所有行中执行替换。最后的 "g" 标记表示替换行中的所有匹配点。
如果你有一个像 "thirtyfour" 这样的单词，上面的命令会出错。这种情况下，这个单词会被替换成 "thirty4"。要解决这个问题，用 "\<" 来指定匹配单词开头：

```
:%s/\<four/4/g
```

显然，这样在处理 "fourteen" 的时候还是会出错。用 "\>" 来解决这个问题：

```
:%s/\<four\>/4/g
```

如果你在编码，你可能只想替换注释中的 "four"，而保留代码中的。由于这很难指定，可以在替换命令中加一个 "c" 标记，这样，Vim 会在每次替换前提示你：

```
:%s/\<four\>/4/gc
```

在多个文件中替换

假设你需要替换多个文件中的单词。你的一个选择是打开每一个文件并手工修改。另外，如果使用 "记录－回放" 命令会更快。
假设你有一个包括有 C++ 文件的目录，所有的文件都以 ".cpp" 结尾。有一个叫 "GetResp" 的函数，你需要把它改名为 "GetAnswer"。

`vim *.cpp`	启动 Vim，用当前目录的所有 C++ 文件作为文件参数。启动后你会停在第一个文件上。
`qq`	用 q 作为寄存器启动一次记录。
`:%s/\<GetResp\>/GetAnswer/g`	在第一个文件中执行替换。
`:wnext`	保存文件并移到下一个文件。
`q`	中止记录。
`@q`	回放 q 中的记录。这会执行又一次替换和 ":wnext"。你现在可以检查一下记录有没有错。
`999@q`	对剩下的文件执行 q 中的命令

Vim 会在最后一个文件上报错，因为 ":wnext" 无法移到下一个文件上。这时所有的文件中的操作都完成了。

> 备注：
> 在回放记录的时候，任何错误都会中止回放的过程。所以，要注意保证记录中的命令不会产生错误。

这里有一个陷阱：如果有一个文件不包含 "GetResp"，Vim 会报错，而整个过程会中止，要避免这个问题，可以在替换命令后面加一个标记：

```
:%s/\<GetResp\>/GetAnswer/ge
```

"e" 标记通知 ":substitute" 命令找不到不是错误。

12.2 把 "Last, First" 改成 "First Last"

你有如下样式的一个名字列表：

```
Doe, John
Smith, Peter
```

你想把它改成：

```
John Doe
Peter Smith
```

这可以用一个命令完成：

```
:%s/\([^,]*\), \(.*\)/\2 \1/
```

我们把这个命令分解成几个部分。首先，很明显它是一个替换命令。"%" 是行范围，表示作用于全文。这样替换命令会作用于全文的每一行。

替换命令的参数格式是 "from/to"，正斜杠区分 "from" 模式和 "to" 字符串。所以，"from" 部分是：

```
\([^,]*\), \(.*\)
```

第一对 \(和 \) 之间的部分匹配 "Last"	`\(  \)`
匹配除逗号外的任何东西	`[^,]`
任意多次	`*`
匹配逗号	`,`
第二对 \(和 \) 之间的部分匹配 "First"	`\(  \)`

匹配任意字符 .
任意多次 *

在 "to" 部分，我们有 "\2" 和 "\1"。这些称为 "反向引用"。它们指向前面模式中的 \(和 \) 间的部分。"\2" 指向模式中的第二对 \(和 \) 间的部分，也就是 "First" 名 (译者注：英文中 Last Name 表示姓，即家族名，后面的 First Name 表示名字)。"\1" 指向第一对 \(\)，即 "Last" 名。

你可以在替换部分使用多达 9 个反向引用。"\0" 表示整个匹配部分。还有一些特殊的项可以用在替换命令中。请参阅 sub-replace-special 。

12.3 排序

在你的 Makefile 中常常会有文件列表。例如：

```
OBJS = \
        version.o \
        pch.o \
        getopt.o \
        util.o \
        getopt1.o \
        inp.o \
        patch.o \
        backup.o
```

要对这个文件列表排序可以用一个外部过滤命令：

```
/^OBJS
j
:.,/^$/-1!sort
```

这会先移到 "OBJS" 开头的行，向下移动一行，然后一行行执行过滤，直到遇到一个空行。你也可以先选中所有需要排序的行，然后执行 "!sort"。那更容易一些，但如果有很多行就比较麻烦。

上面操作的结果将是：

```
OBJS = \
        backup.o
        getopt.o \
        getopt1.o \
        inp.o \
        patch.o \
        pch.o \
        util.o \
        version.o \
```

注意，列表中每一行都有一个续行符，但排序后就错掉了！"backup.o" 在列表的最后，不需要续行符，但排序后它被移动了。这时它需要有一个续行符。

最简单的解决方案是用 "A \<Esc>" 补一个续行符。你也可以在最后一行放一个续行符，由于后面有一个空行，这样做是不会有问题的。

12.4 反转行顺序

:global 命令可以和 :move 命令联用，将所有行移动到文件首部。结果是文件被按行反转了次序。命令是：

```
:global/^/m 0
```

缩写：

```
:g/^/m 0
```

正则表达式 "^" 匹配行首（即使该行是一个空行）。 :move 命令将匹配的行移动到那个神秘的第 0 行之后。这样匹配的行就成了文件中的第一行。由于 :global 命令不会被改变了的行号搞混，该命令继续匹配文件中剩余的行并将它们一一变为首行。

这对一个行范围同样有效。先移动到第一行上方并做标记 't（mt）。然后移动到范围的最后一行并键入：

```
:'t+1,.g/^/m 't
```

12.5 单词统计

有时你要写一些有最高字数限制的文字。Vim 可以帮你计算字数。
如果你需要统计的是整个文件的字数，可以用这个命令：

```
g CTRL-G
```

不要在 "g" 后面输入一个空格，这里只是方便阅读。
它的输出是：

```
Col 1 of 0; Line 141 of 157; Word 748 of 774; Byte 4489 of 4976
```

(译者注：中文是：
第 1/0 列；第 141/157 行；第 748/774 个词；第 4489/4976 个字节
)

你可以看到你在第几个单词（748）上以及文件中的单词总数（774）。

如果你要知道的是全文的一部分的字数，你可以移到该文本的开头，输入 "g CTRL-G"，然后移到该段文字的末尾，再输入 "g CTRL-G"，最后心算出结果来。这是一种很好的心算练习，不过不是那么容易。比较方便的办法是使用可视模式，选中你要计算字数的文本，然后输入 "g CTRL-G"，结果将是：

```
Selected 5 of 293 Lines; 70 of 1884 Words; 359 of 10928 Bytes
```

(译者注：中文是：
选择了 5/293 行；70/1884 个词；359/10928 个字节
)

要知道其它计算字数，行数和其它东西总数的方法，可以参见 count-items 。

12.6 查阅 man 信息 find-manpage

编辑一个脚本文件或者 C 程序的时候，有时你会需要从 man 手册中查询某个命令或者函数的用法（使用 Unix 的情况下）。让我们先用一个简单的方法：把鼠标移到对应的单词上然后输入：

```
K
```

Vim 会在对应的单词上执行外部命令：man。如果能找到相应的手册，那个手册页就会被显示出来。它常常用 more 一类的程序显示页面。在手册滚动到文件末并回车，控制就会回到 Vim 中。

这种方法的缺点是你不能同时查看手册和编辑文档。这里有一种办法可以把手册显示到一个 Vim 的窗口中。首先，加载 man 文件类型的外挂：

```
:runtime! ftplugin/man.vim
```

如果你经常用到这种方法，可以把这个命令加到你的 vimrc 文件中。现在你可以用 ":Man" 命令打开一个显示 man 手册的窗口了：

```
:Man csh
```

你可以在这个新的窗口中上下滚动，而手册的本文会用语法高亮的形式显示。这样，你可以找到需要的地方，并用 CTRL-W w 跳转到原来的窗口中继续工作。

要指定手册的章节，可以在手册名称前面指定。例如，要找第三章的 "echo"：

```
:Man 3 echo
```

要跳转到另一个由 "word(1)" 形式定义的手册，只要在上面敲 CTRL-]。无论怎样，":Man" 命令总使用同一个窗口。

要显示当前光标下的单词的手册，这样：

```
\K
```

(如果你重定义了 <Leader>，用那个字符代替上面命令的反斜杠)。
例如，你想知道下面语句中的 "strstr()" 函数的返回值：

```
if ( strstr(input, "aap") == )
```

可以把光标移到 "strstr" 并输入 "\K"。手册使用的窗口会显示 strstr() 的信息。

12.7 删除多余的空格

有些人认为行末的空格是无用，浪费而难看的。要删除这些每行后面多余的空格，可以执行如下命令：

```
:%s/\s\+$//
```

命令前面指明范围是 "%"，所以这会作用于整个文件。"substitute" 命令的匹配模式是 "\s\+$"。这表示行末 ($) 前的一个或者多个 (\+) 空格 (\s)。后面我们会介绍怎样写这样的模式。见 usr_27.txt 。

替换命令的 "to" 部分是空的："//"。这样就会删除那些匹配的空白字符。

另一种没有用的空格是 Tab 前面的字符。通常这可以删除而不影响格式。但并不是总这样！所以，你最好手工删除它。执行如下命令：

```
/
```

你什么都看不见，其实这是一个空格加一个 TAB 键。相当于 "/<Space><Tab>"。现在，你可以用 "x" 删除多余的空格，并保证格式没有改变。接着你可以用 "n" 找到下一个位置并重复这个操作。

12.8 查找单词的使用位置

如果你是一个 UNIX 用户，你可以用 Vim 和 grep 命令的组合来完成编辑包括特定单词的所有文件的工作。这在你编辑一个程序而且想查看和编辑看所有的包括使用某个变量的文件的时候非常有用。

举个例子，假设想编辑所有包括单词 "frame_counter" 的 C 源文件，你可以执行如下命令：

```
vim `grep -l frame_counter *.c`
```

让我们分析一下这个命令。grep 从一组文件中查找特定的单词。由于指定了 -l 参数，grep 只列出文件而不打印匹配点。被查找的单词是 "frame_counter"，其实这可以是任何正则表达式。(注意：grep 所使用的正则表达式与 Vim 使用的不完全一样)。

整个命令用反引号 (`) 包起来，这告诉 UNIX 的外壳使用该命令的输出作为命令行的一部分。于是，grep 命令产生一个文件列表，并作为 Vim 的命令参数。Vim 将编辑 grep 列出来的所有文件。你可以通过 ":next" 和 ":first" 命令一个一个处理这些文件。

找到每一行

上面的命令只是找到包括单词的那个文件。你还需要知道单词在该文件中出现的地方。

Vim 有一个内置的命令用于在一组文件中找一个指定的字符串。例如，如果你想在所有的 C 文件中查找 "error_string"，可以使用如下命令：

```
:grep error_string *.c
```

这会使 Vim 在所有指定的文件 (*.c) 中查找 "error_string"。Vim 会打开第一个匹配的文件并将光标定位在第一个匹配行。要到下一个匹配行 (无论在哪个文件)，可以执行 "cnext" 命令。要回到上一个匹配行，可以用 ":cprev" 命令。使用 "clist" 可以看到所有的匹配点。

":grep" 命令会使用一个外部的程序。可能是 grep (在 Unix 上) 或者 findstr (在 Windows 上)。你可以通过 'grepprg' 选项修改这个设置。

下一章：usr_20.txt 快速键入命令行命令

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Chimin Yen http://vimcdoc.sf.net

快速键入命令行命令

Vim 具备若干基本功能，以简化键入命令的工作。你可以缩写，编辑和重复冒号命令，而补全功能几乎可以用于所有的场合。

20.1 命令行编辑
20.2 命令行缩写
20.3 命令行补全
20.4 命令行历史
20.5 命令行窗口

下一章： usr_21.txt 离开和回来
前一章： usr_12.txt 小窍门
目录： usr_toc.txt

20.1 命令行编辑

当你用冒号 (:) 命令或用 / 或 ? 搜索一个字符串时，Vim 就会把光标置于屏幕下方。你在那儿键入命令或者搜索模式。此处即称为命令行，也是用来输入搜索命令的地方。

最为显而易见的编辑命令的方法是按 <BS> 退格键。按下此键即删去光标前面的字符。如果要删去另一个更早键入的字符，得先用光标方向键把光标移到那儿。

例如，你键入了：

```
:s/col/pig/
```

在你按下回车键之前，你注意到 "col" 应为 "cow"。为了纠正这个错误，你按五次 <Left> 键。现在，光标正好在 "col" 后面。按 <BS> 键，然后键入正确的字符 "w"：

```
:s/cow/pig/
```

现在你可以立刻按 <Enter> 键了。在执行这个命令之前，你无须先把光标移到命令行的末尾。

命令行上移动光标时最常用的键：

<Left>	左移一个字符
<Right>	右移一个字符
<S-Left> 或 <C-Left>	左移一个单词
<S-Right> 或 <C-Right>	右移一个单词
CTRL-B 或 <Home>	命令行行首
CTRL-E 或 <End>	命令行行尾

备注：
<S-Left> (光标左移键和 Shift 键同时按下) 和 <C-Left> (光标左移键和 Control 键同时按下) 并非在所有键盘上都有效。其它 Shift 和 Control 组合键也是这种情况。

你也可以用鼠标来移动光标。

删 除

如前所述，<BS> 键删除光标前一个字符。删除光标前整个单词，则用 CTRL-W。

```
/the fine pig

            CTRL-W

/the fine
```

CTRL-U 删除命令行上全部文字，从而让你从头开始。

替 换

插入键 <Insert> 让你在插入字符和替换字符两种方式之间切换。先键入如下文字：

```
/the fine pig
```

再按两次 <S-Left>（或 <S-Left> 无效时按八次 <Left>），把光标移到 "fine" 起始处。现在，按插入键 <Insert>，切换到替换方式，并键入 "great"：

```
/the greatpig
```

哟，空格没了。现在，别用 <BS> 键，因为那会删除 "t"（这跟插入方式不同）。此时应该按插入键 <Insert>，从替换方式切换到插入方式，并键入空格：

```
/the great pig
```

取 消

你本想执行一个 : 或 / 命令，但却改变了主意。要清除命令行上你已经键入的文字却不执行该命令，按 CTRL-C 或 <Esc>。

> 备注：
> <Esc> 是普遍采用的 "退出" 键。不幸的是，在过去美好的 Vi 版本里，在命令行上按 <Esc> 却是执行命令！由于那会被认为是程序 bug，Vim 采用 <Esc> 来取消命令。但其 'cpoptions' 选项可以使 Vim 跟 Vi 兼容。而且，使用映射时（那可能是为 Vi 而写的），<Esc> 键也总和 Vi 兼容。由此看来，采用 CTRL-C 倒不失为一种永远奏效的方法。

如果你在命令行开始处，那么按 <BS> 将取消整个命令。这就像删除命令行赖以开始的 ":" 或 "/"。

20.2 命令行缩写

有些 ":" 命令确实很长。我们已经提及替代命令 ":substitute" 可以被缩写成 ":s"。这是一个基本机理，即所有 ":" 冒号命令都可以被缩写。

一个命令可以被缩写成多短呢？英文有 26 个字母，而 Vim 却有多得多的命令。例如，":set" 也以 ":s" 开头，但 ":s" 不是启动 ":set" 命令的。":set" 可以被缩写成 ":se"。

把一个命令缩写得更短即可能适用于两个命令，此时，该缩写仍然只能代表其中的一个。而选择哪个却没有任何逻辑，你不得不一个一个地记。最短的有效缩写形式可以在帮助文件里找到。例如：

```
:s[ubstitute]
```

它的意思是替代命令 ":substitute" 的最短缩写形式是 ":s"。随后的字符可以任选。因此 ":su" 和 ":sub" 都有效。

在用户手册里我们有时用命令的全称，有时简称意义明白就用简称。例如，":function" 可以缩写成 ":fu"。但既然大多数读者不明其所指，所以我们采用简称 ":fun"。(Vim 没有命令 ":funny"，否则简称 ":fun" 也会令人迷惑。)

在 Vim 脚本里你最好用命令全称。当你日后改编脚本时，全称读起来比较容易。除非那是一些常用命令如 ":w" (":write") 和 ":r" (":read")。

一个特别令人迷惑的缩写是 ":end"，它可以指 ":endif"，":endwhile" 或 ":endfunction"。所以，遇到这类命令时，最好永远采用全称。

选 项 简 称

在用户手册里，我们采用选项的全称。许多选项还有简称。跟 ":" 冒号命令不一样，有效的选项简称只有一个。例如，'autoindent' 的简称是 'ai'。因而下面两个命令完成同样的动作：

```
:set autoindent
:set ai
```

你可以从这儿找到完整的选项全称和简称的列表： option-list 。

20.3 命令行补全

命令行补全是那些仅仅因为它就值得从 Vi 转到 Vim 的功能之一。一旦你用上了这个功能，你就离不开它了。

假定你有个文件目录，其中存放了这些文件：

```
info.txt
intro.txt
bodyofthepaper.txt
```

你用这个命令来编辑最后那个文件：

```
:edit bodyofthepaper.txt
```

这很容易打错。一个快得多的方法是：

```
:edit b<Tab>
```

其结果是同样的命令。这是怎么回事？制表键 <Tab> 会补全光标前的词。在本例中就是 "b"。Vim 在目录中寻找并找到了唯一的一个以 "b" 开头的文件。那个文件想必是你寻找的，因此 Vim 为你补全了文件名。

现在键入：

```
:edit i<Tab>
```

Vim 会鸣起响铃，并给你这个结果：

```
:edit info.txt
```

响铃的意思是 Vim 找到了不止一个匹配。然后它使用了找到的第一个匹配 (按字母顺序)。如果你再按一次 <Tab>，你得到：

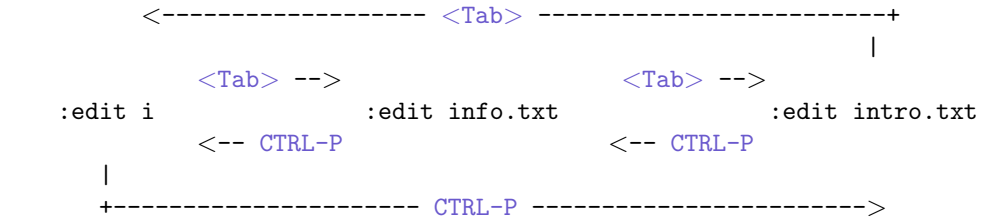
```
:edit intro.txt
```

这样，如果第一次 <Tab> 没给你你要找的文件，你就再按一次。如果还有匹配的文件，你将会看到它们每一个，每按一次，就看到一个。

如果你在最后一个匹配文件名上按 <Tab>，你将会再次看到你起初键入的那个命令：

```
:edit i
```

然后一切再从头开始。这样，Vim 就在匹配列表内周而复始地循环。使用 CTRL-P 依相反方向循环匹配列表：



上 下 文

当你键入 ":set i" 而不是 ":edit i"，并按 <Tab>，你得到的是：

```
:set icon
```

嗨，为什么你没得到 ":set info.txt"？这是由于 Vim 的补全功能是上下文相关的。Vim 寻找的那类词取决于关键词前面的命令。Vim 知道你不可能在命令 ":set" 后面用一个文件名，但却可以用一个选项名。

同样，如果你重复键入 <Tab>，Vim 就会在所有匹配间周而复始地循环。你最好开始时多键入几个字符，否则匹配的选项会很多：

```
:set isk<Tab>
```

结果是：

```
:set iskeyword
```

现在键入 "=" 并按 <Tab>：

```
:set iskeyword=@,48-57,_,192-255
```

在此，Vim 插入的是该选项原来的设定值。现在你可以编辑它了。

按 <Tab> 键补全的乃是 Vim 在命令行那个位置上所期待的。你不妨试试，看它究竟是如何补全的。在某些情形下你会得不到你想要的结果。那或者因为 Vim 不知道你要什么，或者因为 Vim 还没有为该情况实现补全功能。在那个场合，你的 <Tab> 就会被当作字符插入文本（显示为 ^I）。

匹配列表

当匹配有很多时，你可能希望看到一个总览。要看匹配总览，请按 CTRL-D。例如，你键入了以下命令以后按 CTRL-D：

```
:set is
```

结果是：

```
:set is
incsearch  isfname    isident    iskeyword  isprint
:set is
```

Vim 列出了匹配，然后又伴随着你早先键入的命令回到命令行。你现在可以检验匹配列表，找你想要的选项了。如果它不在列表中，你可以用 <BS> 修正那个词。如果匹配太多了，就在按 <Tab> 键补全命令之前再多打几个字符。

如果你仔细观察了，你就会注意到选项 "incsearch" 不是以 "is" 开头。在这个场合，"is" 指的就是 "incsearch" 的简称。（许多选项都有其简称和全称。）Vim 知道你可能要把选项的简称扩展成全称，够聪明的吧。

更多选项

命令 CTRL-L 把词补全为最长无歧义字符串。如果你键入 ":edit i" 而目录内存放着文件 "info.txt" 和 "info_backup.txt"，那么，你将得到 ":edit info"。

选项 'wildmode' 可用来改变补全时采用的方式。
选项 'wildmenu' 可用来取得菜单式的匹配列表。
利用选项 'suffixes' 来指定一些不太重要的文件，并让它们出现在文件列表末尾。
选项 'wildignore' 指定一些根本用不着表列的文件。

欲知更多此类选项详情，参见： cmdline-completion

20.4 命令行历史

我们曾经在第三章简单提到过历史记录功能。其基本用法就是你可以用 <Up> 键调用较早的命令行。而 <Down> 键则让你回到较晚的命令行。

历史记录功能实际上共有四种。我们在这里要提到的是用于 ":" 冒号命令以及用于 "/" 和 "?" 搜索命令的历史记录功能。"/" 和 "?" 命令共享同一历史记录。因为两者同为搜索命令。另外两种历史记录功能用于函数 input() 的表达式和输入行。

cmdline-history

假定你完成了一个 ":set" 命令，又键入了十个冒号命令，然后要再次执行那个 ":set" 命令。你本可以按一个 ":"，然后按十次 <Up>。更快捷的方法是：

```
:se<Up>
```

你现在就回到以前执行过的，以 "se" 开头的命令。没准儿那就是你想找的 ":set" 命令。至少你不至于按很多 <Up> 键（除非你执行的都是 ":set" 命令）。

<Up> 键用你在命令行上已经键入的文本去跟历史记录里的命令作比较。只有匹配的命令才被显示出来。

如果你没找到你要找的命令，用 <Down> 回到你键入的文本作修改。或者用 CTRL-U 从头来过。

显示历史记录里所有的命令：

```
:history
```

那是 ":" 冒号命令的历史记录。搜索历史记录用这个命令来显示：

```
:history /
```

如果你不愿用光标方向键，CTRL-P 作用就跟 <Up> 一样。而 CTRL-N 跟 <Down> 一样。CTRL-P 意指前一个（previous），CTRL-N 意指下一个（next）。

20.5 命令行窗口

在命令行上键入文本跟插入模式下键入文本有所不同。许多修改文本的命令都不能用。对大多数命令而言，那问题不大，但有时你必须键入一个复杂的命令。那样的场合用命令行窗口就非常有用了。

用这个命令来打开命令行窗口：

```
q:
```

Vim 现在就在屏幕底部打开了一个（很小的）窗口。它存放着命令行历史记录，以及一行空行在末尾：

```
+-------------------------------------+
|other window                         |
|~                                    |
 file.txt============================
|:e c                                 |
|:e config.h.in                       |
|:set path=.,/usr/include,,           |
|:set iskeyword=@,48-57,_,192-255     |
|:set is                              |
|:q                                   |
|:                                    |
 command-line========================
|                                     |
+-------------------------------------+
```

现在你处于普通模式下。你可以用 "hjkl" 键来移动光标。例如，用 "5k" 上移至

":e config.h.in" 那一行。键入 "$h" 移到 "in" 的 "i" 字符上，并键入 "cwout"。
现在你把这一行改成了这样：

```
:e config.h.out
```

现在按 <Enter> 执行这个命令。命令行窗口就关上了。
按 <Enter> 键执行光标下的那个命令。这跟 Vim 处于插入模式抑或普通模式无关。
在命令行窗口内所作的修改不会被保存。它们不会导致历史记录被修改。唯一例外就是你执行的命令将被加到历史记录的末尾，跟所有执行过的命令一样。

在你要总览历史记录时，命令行窗口十分有用，查找类似命令，修改一点点，然后执行它。一个搜索命令可用来寻找某些东西。
在前面那个例子中，搜索命令 "?config" 是可以用来寻找先前那个含有 "config" 的命令的。这有点儿特别，因为你是在命令行窗口内用命令行作搜索。在键入搜索命令时，你打不开另一个命令行窗口，命令行窗口是唯一的。

下一章： usr_21.txt 离开和返回

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: Chimin Yen http://vimcdoc.sf.net

离开和回来

本章深入讨论如何让其它程序跟 Vim 混用。或者从 Vim 内部执行别的程序，或者暂时先离开 Vim 而于执行了那个程序后再回来。而且，本章将进一步介绍如何记住 Vim 的当前状态，并在以后把它还原回来。

21.1 挂起和继续
21.2 执行外壳命令
21.3 记忆有关信息; viminfo
21.4 会话
21.5 视图
21.6 模式行

下一章: usr_22.txt 寻找要编辑的文件
前一章: usr_20.txt 快速键入命令行
目录: usr_toc.txt

21.1 挂起和继续

像多数 Unix 程序一样，Vim 可以按 CTRL-Z 挂起。这个动作把 Vim 停下来，并让你回到那个你启动 Vim 的命令外壳里。接着，你可以执行任何别的命令直到你觉得无聊为止。然后再用 "fg" 命令回到 Vim。

```
CTRL-Z
{任何外壳命令序列}
fg
```

你正好回到那个当初你离开 Vim 的地方，什么也没改变。

当你按 CTRL-Z 行不通时，你还可以用命令 ":suspend"。别忘了把 Vim 引导回前台，否则你会失去你所有的修改!

只有 Unix 支持这个功能。在其它系统上 Vim 将为你启动一个外壳。这也让你得以执行外壳命令。可那是一个新外壳，而非你在其中启动 Vim 的那个。

当你运行图像用户界面时，你无法返回那个 Vim 从其中启动的外壳。CTRL-Z 的作用只是把 Vim 窗口最小化。

21.2 执行外壳命令

从 Vim 内部执行单个外壳命令，用 ":!{command}"。例如，要显示目录表:

```
:!ls
:!dir
```

第一行用在 Unix 上，第二行用于微软视窗。

Vim 将执行该程序。当它结束时，你会得到提示，让你击打 <Enter>。这提示允许你

在回去编辑你的文本之前看一看该命令的输出。
字符 "!" 也用在其它有个程序被调用运行的场合。让我们看一看共有哪些：

:!{program}	执行 {program}
:r !{program}	执行 {program} 并读取其输出
:w !{program}	执行 {program} 传送文本至其输入
:[range]!{program}	经由 {program} 过滤文本

注意 "!{program}" 前面那个作用区产生的区别可大了。不附带作用区，这个程序就跟通常一样被执行，而加了这个作用区，作用区内的文本行就经由该程序过滤而出。

用这种方法执行一系列命令也是可以的。但外壳在此却更胜一筹。你可以用这种方法启动一个新外壳：

```
:shell
```

这有点类似于用 CTRL-Z 来挂起 Vim。不同之处在于这种方法启动的是一个新外壳。

使用图像用户界面时，这个外壳利用 Vim 的窗口作为其输入和输出端口。既然 Vim 并非终端仿真器，所以使用中未必尽善近美。要是你遇到麻烦，试试切换 'guipty' 选项。如果这方法仍不奏效，那就只好启动一个新的终端来运行外壳。例如：

```
:!xterm&
```

21.3 记忆有关信息；viminfo

在你编辑了一会儿文件以后，你就会有些文本储存在寄存器内，有些标记指向各种各样文件，还有一些精妙的命令保存在命令行历史记录内。当你退出 Vim，所有这些就全没了。但不用担心，你能够把它们找回来！

信息文件 viminfo 设计用来储存状态信息：

命令行和模式搜索的历史记录
寄存器内文本
各种文件的标记
缓存器列表
全局变量

你每次退出 Vim，它就把此种信息存放在一个文件内。即 viminfo 信息文件。当 Vim 重新启动时，就读取这个信息文件，而那些信息就被还原了。

选项 'viminfo' 的默认设定是还原有限的几种信息。你也许希望设定它记住更多的信息。下面这个命令可以办到：

```
:set viminfo=string
```

其中字符串 "string" 规定了什么要储存。该字符串的语法为一个选项字符跟一个参数。选项和参数组成的对子之间由逗号分隔。
来看一下你可以怎样构建你自己的 viminfo 字符串。首先，选项 ' 用于规定你为多少个文件保存标记 (a-z)。为此选项凑一个整数就行 (比如 1000)。你的命令现在看起来像这样：

```
:set viminfo='1000
```

选项 f 控制是否要储存全局标记 (A-Z 和 0-9)。如果这个选项设为 0，那么什么也不存储。如果设为 1，或你对 f 选项不作规定，那么标记就被存储。你要这个功能，现在你有了：

```
:set viminfo='1000,f1
```

选项 < 控制着每个寄存器内保存几行文本。默认情况下，所有的文本行都被保存。如果设为 0，则什么也不保存。为了避免成千上万行文本被加入你的信息文件 (那些文本可能永远也没用，徒然使 Vim 启动得更慢)，你采用 500 行的上限：

```
:set viminfo='1000,f1,<500
```

你也许用得着的其它选项：

:	保存命令行历史记录内的行数
@	保存输入行历史记录内的行数
/	保存搜索历史记录内的行数
r	可移介质，其上的文件不保存标记 (可用多次)
!	以大写字母开头并且不含有小写字母的全局变量
h	启动时关闭选项 'hlsearch' 高亮显示
%	缓冲区列表 (只有当不带参数启动 Vim 时才用来恢复)
c	用编码 'encoding' 转换文本
n	用于 viminfo 文件的名称 (必须为最后一项选项)

欲知更多详情，参见 'viminfo' 选项以及 viminfo-file 。

当你多次运行 Vim，最后退出的那个就把信息储存起来。这可能导致以前退出的那些 Vim 所存放的信息流失，因为每个条目只能被记住一次。

重返 VIM 中断处

你编辑一个文件到一半，但你得下班去度假了。你退出 Vim 就去享受你的人生，把你那些工作忘得干干净净。两个星期以后你启动 Vim，键入：

```
'0
```

你正好回到当初离开 Vim 的地方。所以你就把你的工作接着做下去。

你每次退出 Vim，它都创建一个标记。最后那个是 '0。原来那个 '0 所指的位置就成了 '1。而原来那个 '1 就成了 '2，依此类推。而原来的标记 '9 就没了。

要发现标记 '0 至 '9 指向何处， :marks 命令很有用。

重返某文件

如果你想回到最近曾经编辑过的文件，但已经退出过 Vim，有一个稍微复杂的方法。你可以这样看到这些文件的列表：

```
:oldfiles
1: ~/.viminfo
2: ~/text/resume.txt
3: /tmp/draft
```

假如你要编辑第二个文件，也就是列表里 "2:" 开始的那个，输入：

```
:e #<2
```

其它接受文件名参数的命令都能替代这里的 ":e"，"#<2" 这种形式可以用在 "%"（当前文件名）和 "#"（轮换文件名）可以出现的地方。由此，你也可以这样来分割窗口来编辑第三个文件：

```
:split #<3
```

#<123 这种东西在你要用来编辑文件时毕竟是太麻烦了。幸运的是有个简单点的办法：

```
:browse oldfiles
1: ~/.viminfo
2: ~/text/resume.txt
3: /tmp/draft
-- More --
```

你可以得到和 :oldfiles 相同的文件。如果要编辑 "resume.txt"，先按 "q" 停止列表，然后会有提示：

```
Type number and <Enter> (empty cancels):
```

输入 "2" 并按 <Enter> 来编辑第二个文件。

详见 :oldfiles 、 v:oldfiles 和 c_#< 。

Vim 间信息移动

在 Vim 仍然运行的情况下，你可以用命令 ":wviminfo" 和 ":rviminfo" 来保存和还原信息。这很方便，比如在两个同时运行的 Vim 之间交换寄存器内容。在第一个 Vim 里执行：

```
:wviminfo! ~/tmp/viminfo
```

而在第二个 Vim 里执行：

```
:rviminfo! ~/tmp/viminfo
```

很明显，字符 "w" 指 "写" 而字符 "r" 指 "读"。

":wviminfo" 用字符 ! 来强制重写一个已存在文件。如果省略而文件却存在，那么这些信息就跟那个文件合并到一起。

用于 ":rviminfo" 的 ! 字符意味着所有的信息都被采用，这可能会重写一些已存在信息。若不用 ! 字符，则只有那些尚未设定的信息才会被采用。

这些命令也可以用来储存信息为将来所用。你可以让一个目录专门存放信息文件，而每一个所包含的信息各有其特殊的目的。

21.4 会话

假定你编着编着，编到那一天要结束了。你想放下手上的工作，而于第二天再从你停下来的那地方继续编下去。你可以做到这一点，只要把你的编辑会话保存起来，第二天再把它还原回来。

Vim 会话存放着所有跟你的编辑相关的信息。这包括诸如文件列表、窗口布局、全局变量、选项、以及其它信息。(究竟什么信息被记住，则由选项 'sessionoptions' 控

制，稍后叙述。）

下面这个命令创建一个会话文件：

```
:mksession vimbook.vim
```

如果你以后要还原这个会话，你可以用这个命令：

```
:source vimbook.vim
```

如果你要启动 Vim 并还原某个特别的会话，你可以用下面这个命令：

```
vim -S vimbook.vim
```

这命令告诉 Vim 在启动时读取一个特定的会话文件。参数 'S' 指会话（实际上，你可以用 -S 运行任何 Vim 脚本，因而，你也不妨指其为运行脚本，"source"）。

那个曾经打开的窗口就还原了，跟以前一样的位置和大小。映射和选项值也像以前一样。

究竟还原了什么取决于 'sessionoptions' 选项。默认值为 "blank,buffers,curdir,folds,help,options,tabpages,winsize,terminal"。

blank	保留空窗口
buffers	所有缓冲区，而非仅仅一个窗口内的
curdir	当前目录
folds	折叠，包括人工创建的
help	帮助窗口
options	所有选项和映射
tabpages	所有标签页
winsize	窗口大小
terminal	包含终端窗口

你爱怎么改就怎么改。例如，除了上述项目以外，你还要还原 Vim 窗口大小：

```
:set sessionoptions+=resize
```

会 话 用 法

最显而易见的会话用法，是在编辑属于不同项目的文件时。假定你把会话文件都储存在目录 "~/.vim" 下。你正编辑着项目 "secret" 的文件，而你必须切换到项目 "boring" 的文件上：

```
:wall
:mksession! ~/.vim/secret.vim
:source ~/.vim/boring.vim
```

首先用命令 ":wall" 把所有修改过的文件存盘。然后用命令 ":mksession!" 保存当前会话。它重写了前一次会话文件。下一次载入 "secret" 会话时，你便可以在原来那一点上接着编辑下去。最后，你载入新的 "boring" 会话。

如果你打开帮助窗口，分割和关闭各种各样窗口，或者一般来说把窗口布局搞砸了，此时你可以恢复上次保存的会话：

```
:source ~/.vim/boring.vim
```

你因而享有充分的控制权，要么把当前会话中的设置保存起来，以便下一次编辑时从现在这个位置接着编下去，要么保留会话文件不变，一切都从那儿开始。

另一种会话用法是你创建了一种你喜欢的窗口布局，并把它保存在一个会话文件中。然后你可以在任何时候恢复这种窗口布局。

例如，这是一个使用起来很不错的布局：

```
+----------------------------------------+
|                  VIM - main help file  |
|                                        |
|Move around:  Use the cursor keys, or "h|
 help.txt================================
|explorer   |                            |
|dir        |~                           |
|dir        |~                           |
|file       |~                           |
|file       |~                           |
|file       |~                           |
|file       |~                           |
 ~/========= [No File]==================|
|                                        |
+----------------------------------------+
```

其顶部有个帮助窗口，使你能够阅读本文。左边那个狭长窗口相当于一个文件浏览器。这是一个 Vim 插件，用来表列一个目录的内容。你可以在那儿挑选文件来编辑。有关这一点，下一章有更多叙述。

从一个刚刚启动的 Vim 创建这样一个布局，请用：

```
:help
CTRL-W w
:vertical split ~/
```

你可以根据你的喜好稍稍改动一点那些窗口的大小。然后保存会话记录：

```
:mksession ~/.vim/mine.vim
```

现在你可以用这个布局启动 Vim：

```
vim -S ~/.vim/mine.vim
```

提示：要在一个空窗口中打开一个表列在浏览器窗口中的文件，请把光标移到文件名上并按 "O"。用鼠标双击也行。

UNIX 和 微 软 视 窗

有些人不得不今天在微软视窗上，而明天则在 Unix 上工作。如果你是其中之一，请考虑把 "slash" 和 "unix" 加入选项 'sessionoptions'。此后存盘的会话文件的格式在两种系统上都适用。请把以下命令加进你的 vimrc 文件内：

```
:set sessionoptions+=unix,slash
```

Vim 将因而使用 Unix 格式，因为微软视窗上的 Vim 能读写 Unix 文件，但 Unix 上的 Vim 却不能读取微软视窗格式的会话文件。类似地，微软视窗上的 Vim 懂得文件路径名称里用以分隔名字的 / ，但 Unix 上的 Vim 却不懂微软视窗上的 \。

会 话 记 录 和 信 息 文 件

会话记录储存了不少东西，但不储存标记位置，寄存器内容以及命令行历史记录。你需要利用 Vim 信息文件 viminfo 储存这些信息。

在大多数情况下，你将需要利用会话记录而非全部信息文件内的信息。这么做可以让你切换到另一个会话记录，但却保留着命令行历史记录。并得以在一个会话期内把文本抄进寄存器，而于另一个会话期把它粘贴出来。

你也许宁可用会话记录保存信息。那么，你必须亲自动手。例如：

```
:mksession! ~/.vim/secret.vim
:wviminfo! ~/.vim/secret.viminfo
```

而再次把它还原：

```
:source ~/.vim/secret.vim
:rviminfo! ~/.vim/secret.viminfo
```

21.5 视图

会话记录储存着整个 Vim 窗口外观。当你只需要为某个窗口储存特性时，得用视图。

视图的用处在于你要以某种特定的方式编辑一个文件。例如，你以 'number' 选项显示了行号，并定义了若干折叠。正如会话记录那样，你可以记住这一视图并在以后还原回来。事实上，当你储存会话时，每个窗口的视图都储存了。

视图有两种基本用法。第一种是让 Vim 为视图文件挑一个文件名。你可以在以后编辑同一文件时还原该视图。为当前窗口储存视图，用：

```
:mkview
```

Vim 将自行决定视图的储存位置。当你以后编辑同一文件时，用这个命令恢复该视图：

```
:loadview
```

这挺容易，不是吗?

现在你要阅读这个文件，阅读时你不要 'number' 选项显示行号，或者你要把所有的折叠都打开。你可以设定这些选项，使窗口看起来就是你要的那个样子。然后储存这个视图：

```
:mkview 1
```

显而易见，你可以用下面的命令把它恢复：

```
:loadview 1
```

现在你可以用 ":loadview" 在这个文件的两个视图之间切换了，一个加参数 "1"，另一个则不加参数。

你可以用这个方法为同一个文件储存视图达十个之多，一个没序号的，而九个则有序号 1 至 9。

命 名 视 图

第二种视图基本用法是把视图储存在一个你选定的文件内。你可以在编辑另外一个文件时载入这个视图。Vim 将接着转而编辑该视图规定的文件。这样你就可以用这种方法迅速换个文件来编辑，其所有的选项设定就跟它们在存盘时一样。

例如，要保存当前文件的视图：

```
:mkview ~/.vim/main.vim
```

你可以用这个命令把它还原：

```
:source ~/.vim/main.vim
```

21.6 模式行

当你编辑一个特定的文件，你也许为该文件设定了特定的选项。每次键入这些命令很无聊。而在许多人共享一个文件时，利用会话和视图来编辑这个文件也无济于事。

解决这个困境的方法是给文件加一个模式行。那是一行文本，它把一些只适用于该文件的选项设定告诉 Vim。

一个典型的例子是在一个 C 程序中你把缩进值设为 4 的倍数。这就要求把选项 'shiftwidth' 设为 4。这个模式行能奏效：

```
/* vim:set shiftwidth=4: */
```

把这一行插入该文件起首五行或结尾五行。编辑这个文件时，你将注意到 'shiftwidth' 选项已经设定为 4。编辑另一个文件时，它再设回默认值 8。

对于有些文件，模式行放在头部挺合适，所以它应该置于该文件的顶部。对于文本文件和其它那些模式行会影响正文阅读的文件，把模式行放在文件结尾处。

选项 'modelines' 规定了要在文件起首和结尾几行之内检查那儿是否包含了模式行。要检查十行：

```
:set modelines=10
```

选项 'modeline' 可以用来撤销这个设定。如果你以超级用户身份 (Unix 的 root 或 MS-Windows 的 Administrator) 工作或者你不信任要编辑的文件时应该如此：

```
:set nomodeline
```

模式行可以用这种格式：

```
any-text vim:set {option}={value} ... : any-text
```

其中 "any-text" 表示你可以在 Vim 实际用到的部分之前和之后加任意文本。这就允许你使它看起来像个注释，正如上例采用了 /* 和 */。

" vim:" 部分使 Vim 识别出这个模式行。在 "vim" 的前面必须有空格，除非 "vim" 置于行首。因此像 "gvim:" 这样用法是不行的。

冒号之间的部分是命令 ":set" 。它的用法就跟键入 ":set" 命令一模一样，只除了在命令包含的任何冒号之前需要插入一个反斜杠 (否则该冒号就会被看成模式行结尾)。

还有一个例子：

```
// vim:set textwidth=72 dir=c\:\tmp:  use c:\tmp here
```

在第一个冒号之前多一个反斜杠，因此它被包括在 ":set" 命令内。第二个冒号后面的文

本则被忽略不计，因此可以在那儿放个注释。

欲知详情，参见 modeline 。

下一章： usr_22.txt 寻找要编辑的文件

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Chimin Yen http://vimcdoc.sf.net

寻找要编辑的文件

到处都是文件，你怎么查找它们呢？Vim 为我们提供了在目录树间浏览的种种方法。还有若干命令让你从一个文件通过文件名跳转到一个文件。而且，Vim 还记着曾经编辑过哪些文件。

22.1 文件浏览器
22.2 当前目录
22.3 查找文件
22.4 缓冲区列表

下一章： usr_23.txt 编辑特殊文件
前一章： usr_21.txt 离开和回来
目录： usr_toc.txt

22.1 文件浏览器

Vim 有个插件可以用来编辑一个目录。试一下这个命令：

```
:edit .
```

借助魔术般的自动命令和 Vim 脚本功能，目录的内容被用来填充窗口。看起来就像这样：

```
" ============================================================================
" Netrw Directory Listing                                        (netrw v109)
"   Sorted by      name
"   Sort sequence: [\/]$,\.h$,\.c$,\.cpp$,*,\.info$,\.swp$,\.o$\.obj$,\.bak$
"   Quick Help: <F1>:help  -:go up dir  D:delete  R:rename  s:sort-by  x:exec
" ============================================================================
../
./
check/
Makefile
autocmd.txt
change.txt
eval.txt~
filetype.txt~
help.txt.info
```

你可以看到这样一些条目：

1. 浏览工具的名称和版本号
2. 浏览目录名
3. 排序方法（可以是名字、时间或大小）
4. 名字如何排序（目录优先，然后是 *.h 文件、*.c 文件，等等）
5. 如何得到帮助（用 <F1> 键），然后是可用命令的概括列表

6. 文件列表，包括 "../"，用户可以由此列出父目录。

如果你启用了语法高亮功能，那么，目录里不同部分就显示成不同颜色，让你比较容易辨认它们。

你可以用普通模式下的 Vim 命令在文本内到处移动。例如，移动光标到一个文件名上并按下 <Enter> 键。你就可以编辑那个文件了。要回到浏览器，再用一次 ":edit ." 或 ":Explore" 即可，按 CTRL-O 也行。

试一下把光标移至某个目录名，按 <Enter> 键。结果，浏览器就进了该目录，并把那里的条目显示出来。对准第一个目录 "../" 按 <Enter> 键， 让你返回父目录。按 "-" 键可达到同样的目的，且无须先把光标移到 "../" 条目上。

你可以按 <F1> 键获取关于 netrw 文件浏览器功能的帮助。帮助文字如下：

```
9. Directory Browsing        netrw-browse   netrw-dir   netrw-list   netrw-help

MAPS                                                                  netrw-maps
     <F1>.............Help.....................................|netrw-help|
     <cr>.............Browsing..................................|netrw-cr|
     <del>............Deleting Files or Directories.............|netrw-delete|
     -................Going Up.................................|netrw--|
     a................Hiding Files or Directories...............|netrw-a|
     mb...............Bookmarking a Directory....................|netrw-mb|
     gb...............Changing to a Bookmarked Directory.........|netrw-gb|
     c................Make Browsing Directory The Current Dir....|netrw-c|
     d................Make A New Directory.......................|netrw-d|
     D................Deleting Files or Directories..............|netrw-D|
     <c-h>............Edit File/Directory Hiding List............|netrw-ctrl-h|
     i................Change Listing Style.......................|netrw-i|
     <c-l>............Refreshing the Listing.....................|netrw-ctrl-l|
     o................Browsing with a Horizontal Split...........|netrw-o|
     p................Use Preview Window.........................|netrw-p|
     P................Edit in Previous Window....................|netrw-p|
     q................Listing Bookmarks and History..............|netrw-qb|
     r................Reversing Sorting Order....................|netrw-r|
    (等等)
```

<F1> 键把你带到 netrw 目录浏览内容的帮助页面。这是一个常规的帮助页面，同样常规的 CTRL-] 跳转到带标签的帮助项目，而 CTRL-O 则返回。

要选择显示和编辑的文件：(光标在某文件名上)

<enter>	在当前窗口打开文件	netrw-cr
o	横向分割窗口并显示文件	netrw-o
v	竖向分割窗口并显示文件	netrw-v
p	使用 preview-window	netrw-p
P	在上次的窗口中编辑	netrw-P
t	在新标签页中打开文件	netrw-t

接下来的普通模式命令用来控制浏览器显示：

i	控制列表风格 (瘦、长、宽和树形)。长列表包含文件大小和日期信息。

s	反复按 s 会改变文件排序的方式；可以按照名字、修改日期或文件大小排序。
r	逆转排列顺序。

略举数例其它的普通模式命令：

c	把当前目录改成显示在窗口中的那个目录。(见 g:netrw_keepdir ，它也控制此行为)
R	为光标下文件改名。Vim 将提示你提供新文件名。
D	删除光标下文件。Vim 将提示你确认删除动作。
mb gb	建立书签/转到书签

还有命令模式；还是一样，只举数例：

:Explore [directory]	浏览指定/当前目录
:NetrwSettings	当前 netrw 设置的综合列表，带有帮助链接。

netrw 浏览器不限于你的本地机器；也可使用 url，如下：　(拖尾的 / 是必需的)

```
:Explore ftp://somehost/path/to/dir/
:e scp://somehost/path/to/dir/
```

详见 netrw-browse 。

22.2 当前目录

正如外壳一样，Vim 也有当前目录的概念。假设你在主目录，并要编辑几个储存在目录 "VeryLongFileName" 下的文件。你可以这样做：

```
:edit VeryLongFileName/file1.txt
:edit VeryLongFileName/file2.txt
:edit VeryLongFileName/file3.txt
```

为了避免太多的键击，你可以这样做：

```
:cd VeryLongFileName
:edit file1.txt
:edit file2.txt
:edit file3.txt
```

":cd" 命令可以用来改变当前目录。你可以用 ":pwd" 命令来查看当前目录使什么：

```
:pwd
/home/Bram/VeryLongFileName
```

Vim 记得最近访问过的那个目录。你可以用 "cd -" 命令回去那儿。例如：

```
:pwd
/home/Bram/VeryLongFileName
:cd /etc
:pwd
/etc
```

```
:cd -
:pwd
/home/Bram/VeryLongFileName
:cd -
:pwd
/etc
```

窗口本地目录

当你把窗口一分为二以后，两个窗口的当前目录是一样的。而当你要在新窗口内编辑几个存放在当前目录以外某处的文件时，你可以让新窗口采用不同的当前目录，同时保持原窗口当前目录不变。新窗口采用的当前目录称为本地目录。

```
:pwd
/home/Bram/VeryLongFileName
:split
:lcd /etc
:pwd
/etc
CTRL-W w
:pwd
/home/Bram/VeryLongFileName
```

只要你不发出 `:lcd` 命令，所有窗口共享同一个当前目录。在一个窗口执行一次 `:cd` 命令，也同时改变其它窗口的当前目录。

执行过 `:lcd` 命令的窗口记得它特有的当前目录。在其它窗口执行 `:cd` 或 `:lcd` 命令对它毫无影响。

在一个采用特有当前目录的窗口执行 `:cd` 命令以后，该窗口就回过头来又采用共享的当前目录了。

标签页本地目录

打开新的标签页时，使用打开时所在的之前标签页窗口的目录。可以用 `:tcd` 命令来改变当前标签页的目录。除了有局部于窗口的目录的窗口以外，同一标签页的所有窗口共享此目录。任何此标签页上打开的新窗口会使用此目录为当前工作目录。在标签页中使用 `:cd` 命令不会改变有局部于标签页目录的标签页的工作目录。在标签页中使用 `:cd` 命令改变全局目录后，也改变了当前标签页的工作目录。

22.3 查找文件

假定现在你在编辑一个 C 程序，该程序有这样一行：

```
#include "inits.h"
```

你想要查看文件 "inits.h" 里有些什么。把光标移到该文件名上，并键入：

```
gf
```

Vim 就会找到并打开这个文件。

那么，如果该文件不在当前目录里怎么办？Vim 将利用 'path' 选项来寻找这个文

件。该选项是一系列目录名，Vim 会在其中为你寻找文件。
假设你把你的头文件存放在目录 "c:/prog/include" 里。下面这个命令把该目录加入 'path' 选项：

```
:set path+=c:/prog/include
```

这个目录名是一绝对路径。不管你在哪儿，它都指向同一目录。如果你已经知道要编辑的文件位于当前文件所在目录下某个子目录里，你该怎么办呢？那样的话，你可以指定一个相对路径。相对路径以 "." 开始：

```
:set path+=./proto
```

这个命令告诉 Vim 到目录 "proto" 里找文件，而 "proto" 则位于你在其中用了 "gf" 命令的文件所在的目录下面。如此一来，对 "inits.h" 用 "gf" 命令，就让 Vim 以当前文件所在目录为起点，寻找 "proto/inits.h"。
如果不加 "./"，只用 "proto"，Vim 就会进入当前目录下的 "proto" 目录寻找。然而当前目录可能不是你在编辑的这个当前文件所在的目录。

'path' 选项还允许你用其它许多方法来指定在其中寻找文件的目录。参见关于 'path' 选项的帮助。
'isfname' 选项用来决定哪些字符可用于文件名，以及哪些不可以 (如上例中的双引号 " 字符)。

当你要找的文件名没出现在你编辑的文件里时，你可以键入这个文件名：

```
:find inits.h
```

然后 Vim 就利用 'path' 选项来确定该文件的位置。这就跟 ":edit" 命令一样，只不过 ":edit" 命令不用 'path' 选项。

要在新窗口内打开那个已经找到的文件，用 CTRL-W f，而不是 "gf"，或用 ":sfind" 而不是 ":find"。

有个好办法可以直接启动 Vim 来编辑 'path' 中的某个文件：

```
vim "+find stdio.h"
```

这会在你的 'path' 中查找 "stdio.h"。双引号是必需的，用来把括起来的当做一个参数 -+c 。

22.4 缓冲区列表

Vim 编辑器使用术语 "缓冲区" 来描述编辑当中的文件。实际上，缓冲区是你编辑的文件的副本。你修改完缓冲区，就把缓冲区的内容写进文件。缓冲区不仅存放文件内容，而且还存放着全部标记，设定，以及其它跟被编辑文件相关的东西。

隐藏的缓冲区

假设你在编辑文件 one.txt，同时又要编辑文件 two.txt。你本来可以简单地用 ":edit two.txt" 来办到，但由于你已经修改了 one.txt，那样做就没用了。而你又不想在此时就把 one.txt 存盘。Vim 可以为你解决这个问题：

```
:hide edit two.txt
```

缓冲区 "one.txt" 从屏幕消失，但 Vim 仍然知道你在编辑这个缓冲区，因而保留着它修改过的文本。这样的缓冲区称为隐藏的缓冲区：缓冲区存放着文本，但你看不见它。

":hide" 命令参数是另一个命令。":hide" 使得那个命令表现得就像 'hidden' 选项已被设定。你也可以不用 ":hide" 命令而设定 'hidden' 选项。其作用是当离开任何缓冲区时，该缓冲区变成隐藏。

小心！当你的隐藏的缓冲区已经改动，千万不要在所有缓冲区存盘之前就退出 Vim。

非激活缓冲区

一个缓冲区一经使用，Vim 就记住了一些有关该缓冲区的信息。即使它既不显示在窗口内，又非隐藏缓冲区，它也仍然在缓冲区列表上。这样的缓冲区称为非激活缓冲区。一般而言，

激活	显示在窗口内，并加载文本
隐藏	不显示在窗口内，但加载文本
非激活	不显示在窗口内，不加载文本

非激活缓冲区不会被遗忘，因为 Vim 保存着关于它们的信息，如标记等。而且记住文件名有个好处，你可以调阅你编辑过的文件名，再次编辑它们。

缓冲区列表

你可以用这个命令查看缓冲区列表：

```
:buffers
```

另一个作用相当的命令，虽然意思不那么明白，但键入时省事多了：

```
:ls
```

其输出可能像这样：

```
1 #h   "help.txt"                    line 62
2 %a + "usr_21.txt"                  line 1
3      "usr_toc.txt"                 line 1
```

第一栏存放缓冲区号。你可以利用它来编辑文件，而不用键入文件名，参见下文。

紧随缓冲区号的是些标志位。然后是文件名，以及光标最后一次停留的行号。

可能出现的标志位有以下这些（自左至右）：

u	列表外缓冲区 unlisted-buffer 。
%	当前缓冲区。
#	轮换缓冲区。
a	激活缓冲区，缓冲区被加载且显示。
h	隐藏缓冲区，缓冲区被加载但不显示。
=	只读缓冲区。
-	不可改缓冲区， 'modifiable' 选项不置位。
+	已修改缓冲区。

编 辑 缓 冲 区

你可以通过指定其缓冲区号来编辑一个缓冲区，而不必键入文件名：

```
:buffer 2
```

但获知缓冲区号的唯一途径是查阅缓冲区列表。如果不用缓冲区号，你可以用文件名，或其部分：

```
:buffer help
```

Vim 将为你键入的文件名找到最佳匹配。如果只有一个缓冲区与之匹配，该缓冲区就被选用。在这个例子中，被选中的就是 "help.txt"。
要在新窗口中打开一个缓冲区：

```
:sbuffer 3
```

这方法也适用于文件名。

使 用 缓 冲 区 列 表

你可以用这些命令在缓冲区列表间移动：

:bnext	编辑下一个缓冲区
:bprevious	编辑前一个缓冲区
:bfirst	编辑第一个缓冲区
:blast	编辑最后一个缓冲区

要从缓冲区列表上删除一个缓冲区，用这个命令：

```
:bdelete 3
```

同样，这命令也适用文件名。
如果你删除了一个激活的缓冲区（显示在窗口中的缓冲区），你也就关闭了该窗口。如果你删除了当前缓冲区，你也就关闭了当前窗口。如果它是最后一个窗口，Vim 将找一个缓冲区来编辑。你无法什么也不编辑！

备注：
即使用 ":bdelete" 命令删除了缓冲区以后，Vim 依然记得它。这个缓冲区实际上成了 "列表外" 缓冲区，它不再出现在 ":buffers" 命令所报告的列表中。不过 ":buffers!" 命令仍会列出 "列表外" 缓冲区（没错，Vim 无所不能）。要让 Vim 彻底忘记一个缓冲区，用 ":bwipe" 命令。另见 'buflisted' 选项。

下一章： usr_23.txt 编辑其它文件

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Chimin Yen http://vimcdoc.sf.net

编辑特殊文件

本章讨论特殊文件的编辑。利用 Vim 你可以编辑经过压缩或加密的文件。有些文件需要在互联网上存取。二进制文件也能有限度地编辑。

23.1 DOS、Mac 和 Unix 文件
23.2 互联网上的文件
23.3 加密
23.4 二进制文件
23.5 压缩文件

下一章： usr_24.txt 快速插入
前一章： usr_22.txt 寻找要编辑的文件
目录： usr_toc.txt

23.1 DOS、Mac 和 Unix 文件

很久以前，老式的电传打字机使用两个字符来另起新行。一个字符把滑动架移回首位（称为回车，<CR>），另一个字符把纸上移一行（称为换行，<LF>）。

当计算机问世以后，存储器曾经非常昂贵。有些人就认定没必要用两个字符来表示行尾。UNIX 开发者决定他们可以用 <Line Feed> 一个字符来表示行尾。Apple 开发者规定了用 <CR>。开发 MS-DOS（以及微软视窗）的那些家伙则决定沿用老式的 <CR><LF>。

那意味着，如果你试图把一个文件从一种系统移到另一种系统，那么你就有换行符方面的麻烦。Vim 编辑器自动识别不同文件格式，并且不劳你操心就把事情给办妥了。

选项 'fileformats' 包含各种各样的格式，Vim 会在编辑一个新文件之初尝试该选项定义的各种格式。例如，下面这个命令告诉 Vim 先尝试用 UNIX 格式，其次，尝试 MS-DOS 格式：

```
:set fileformats=unix,dos
```

编辑一个文件时，你将注意到 Vim 给出的信息消息报中包括文件所用的格式。如果你编辑的是本地格式文件（你编辑的文件格式和所用系统一致），你就不会看到任何格式名。因此在 Unix 系统上编辑一个 Unix 格式文件不会产生任何关于格式的信息。但你若编辑一个 dos 文件，Vim 将这样通知你：

```
"/tmp/test" [dos] 3L, 71C
```

如果是 Mac 文件，你会看到 "[mac]"。

探测到的文件格式会被存入 'fileformat' 选项。执行下面这个命令可以显示你当前使用的文件格式：

```
:set fileformat?
```

Vim 能使用的三种格式如下：

```
unix            <LF>
dos             <CR><LF>
```

```
mac             <CR>
```

使用 MAC 格式

在 Unix 上，<LF> 用于分行。但 <CR> 字符混在文本行中间也非罕见。这种情况碰巧经常发生在 Vi（和 Vim）脚本内。

在采用 <CR> 作为换行符的 Macintosh 上，<LF> 字符也有可能混在文本行中间。

结果，很难 100% 肯定一个同时包含 <CR> 和 <LF> 的文件究竟是 Mac 还是 Unix 格式。所以，Vim 假设你一般不会在 Unix 上编辑一个 Mac 文件，所以干脆对这种文件格式不作检查。果真要检查此种格式，就把 "mac" 加入 'fileformats'：

```
:set fileformats+=mac
```

然后 Vim 就会猜测文件格式。要当心， Vim 可能会猜错的。

强制格式

如果你用往日美好的 Vi 来尝试编辑一个采用 MS-DOS 格式的文件，你将会发现每一行的末尾有个 ^M 字符。(^M 就是 <CR>)。而 Vim 的自动探测功能就避免了这个问题。莫非你确实要按那个样子来编辑这个文件吗？那么你需要强制 Vim 忽略文件格式而使用你指定的格式：

```
:edit ++ff=unix file.txt
```

字符串 "++" 告诉 Vim 后面跟的是选项名，以取代其默认值。但仅作用于这一个命令。"++ff" 用于 'fileformat' 选项。你也可以用 "++ff=mac" 或 "++ff=dos"。

这样用法并非适用于任意选项，目前 Vim 仅仅实现了 "++ff" 和 "++enc"。用全称 "++fileformat" 和 "++encoding" 也行。

转换

你可以用 'fileformat' 选项把文件从一种格式转换为另一种。例如，假定你有个名为 README.TXT 的 MS-DOS 文件，你要把它转换成 UNIX 格式。首先编辑这个采用 MS-DOS 格式的文件：

```
vim README.TXT
```

Vim 将识别出那是一个 dos 格式文件。现在把这个文件的格式改为 UNIX：

```
:set fileformat=unix
:write
```

这个文件就以 Unix 格式存盘了。

23.2 互联网上的文件

有人给你传送了一封电子邮件，其中引用了一个以超链接 URL 表示的文件。例如：

```
You can find the information here:
        ftp://ftp.vim.org/pub/vim/README
```

你当然可以启动一个程序来下载这个文件，把它存入你本地磁盘，然后启动 Vim 来编辑它。

但有一个简单得多的方法。把光标移到那个超链接里任何一个字符上，然后使用这个命令：

```
gf
```

运气好的话，Vim 将确定用哪个程序来下载这个文件，并把文件下载下来让你编辑该副本。在另一个新窗口打开这个文件，则用 CTRL-W f。

如果事情不顺利的话，你会得到出错信息。可能那个链接有错，也可能你没有阅读它的权限，还可能网络连接中断了，等等。不幸的是，很难讲搞清除出错的原因。你也许得尝试以手工方法来下载这个文件。

访问因特网上的文件要安装插件 netrw 才行。目前，下面这些格式的超文本链接可被识别：

```
ftp://          使用 ftp
rcp://          使用 rcp
scp://          使用 scp
http://         使用 wget (只读)
```

Vim 并非亲自与因特网联系。它有赖于你的计算机上安装好的上面提及的程序。大多数 Unix 系统上装有 "ftp" 和 "rcp"。而 "scp" 和 "wget" 也许要另外安装。

无论你用哪个命令开始编辑，Vim 总是会探测一下这些超文本链接。包括如 ":edit" 和 ":split" 这样的命令。存盘命令也行，但除了 http:// 之外。

欲知详情，包括密码问题，参见 netrw 。

23.3 加密

有些信息你希望保留给自己。例如，当你在计算机上写一份给学生用的试卷。你不会愿意在考试开始之前给聪明的学生琢磨出一种偷阅试题的方法。Vim 能为你给文件加密，为你提供一些保护。

要开始编辑一个需要加密的新文件，可以用 "-x" 参数来启动 Vim。例如：

```
vim -x exam.txt
```

Vim 提示你确定一个密码，用于为文件加密和解密：

```
Enter encryption key:

(译者注：中文是
输入密码：
)
```

现在仔细键入那个密码。你所键入的字符将为星号代替，因此你看不到。为了避免由于打字错误引起麻烦，Vim 要求你再输入一次密码：

```
Enter same key again:

(译者注：中文是
请再输入一次：
```

)

现在你可以像平时一样编辑这个文件并把你所有的秘密放进去。当你编完文件要退出 Vim 时，这个文件就被加密存盘了。

当你用 Vim 编辑这个文件时，它就会要求你再输入那同一个密码。你不需要用 "-x" 参数。你也可以用普通的 ":edit" 命令编辑加密的文件。Vim 给这个文件加上特定的 magic 字符串，据以识别那是经过加密的文件。

如果你试图用另一个程序来阅读这个文件，你将读到一堆垃圾。如果你用 Vim 来编辑这个文件，但输入了错误的密码，你也只能得到垃圾。Vim 并不具备检验密码正确性的机理 (这一点使得破译密码更为困难)。

开 / 关 加 密

要给一个文件撤除加密，设定 'key' 选项为空字符串:

```
:set key=
```

你下次把这个文件存盘时，存盘的文件就是未经加密的。

设定 'key' 选项来启用加密是个坏主意。因为密码会被清清楚楚地显示在屏幕上。任何人都可以偷看到你的密码。

为了避免这样的问题，创造了 ":X" 命令。它会像 "-x" 参数向你索取一个密码:

```
:X
Enter encryption key: ******
Enter same key again: ******
```

加 密 的 局 限 性

Vim 采用的加密算法不够强大。它对于防止那种偷窥者是绰绰有余了，但不足以防止一个手上有大量时间的密码专家。交换文件和撤销文件中的文本也是加密的。不过，那是按块进行的，从而可能会缩短破解密码所需的时间。你可以关闭交换文件，但崩溃会丢失您的工作，因为 Vim 会只在内存里保留所有的文本。关闭撤销文件唯一的缺点是缓冲区卸载后不能撤销。

要避免使用交换文件，在命令行上用 -n 参数。例如，要编辑经过加密的文件 "file.txt"，但不用交换文件，请用下面的命令:

```
vim -x -n file.txt
```

如果你已经在编辑这个文件了，那么交换文件 swapfile 可以用下面的命令禁止:

```
:setlocal noswapfile
```

由于没了交换文件，文件复原就不可能了。为了避免失去编辑的成果，要比平时更勤快地存盘你的文件。

文件在内存中以明文形式存在。因此任何具备权限的人都能进入编辑者的内存浏览，从而，发现这个文件的内容。

如果你使用信息文件 viminfo，别忘了文本寄存器的内容也是明明白白写在其中的。

如果你真的要保证一个文件内容的安全，那么，你必须永远只在一个不联网的可携式计算机上编辑这个文件，使用优良的加密工具，并且在不用时，把你的计算机锁进一个大保险箱。

23.4 二进制文件

你可以用 Vim 来编辑二进制文件。Vim 本非为此而设计的，因而有若干局限。但你能读取一个文件，改动一个字符，然后把它存盘。结果是你的文件就只有那一个字符给改了，其它的就跟原来那个一模一样。

要保证 Vim 别把它那些聪明的窍门用错地方，启动 Vim 时加上 "-b" 参数：

```
vim -b datafile
```

这个参数设定了 'binary' 选项。其作用是排除所有的意外副作用。例如，'textwidth' 设为零，免得文本行给擅自排版了。并且，文件一律以 Unix 文件格式读取。

二进制模式可以用来修改某程序的消息报文。小心别插入或删除任何字符，那会让程序运行出问题。用 "R" 命令进入替换模式。

文件里的很多字符都是不可显示的。用 Hex 格式来显示它们的值：

```
:set display=uhex
```

另外，也可以用命令 "ga" 来显示光标下的字符值。当光标位于一个 <Esc> 字符上时，该命令的输出看起来就像这样：

```
<^[>  27,  Hex 1b,  Octal 033
```

文件中也许没那么多换行符。你可以关闭 'wrap' 选项来获得总览的效果：

```
:set nowrap
```

字 节 位 置

要发现你在文件中的当前字节位置，请用这个命令：

```
g CTRL-G
```

其输出十分冗长：

```
Col 9-16 of 9-16; Line 277 of 330; Word 1806 of 2058; Byte 10580 of 12206
```

最后两个数字就是文件中的当前字节位置和文件字节总数。这已经考虑了 'fileformat' 选项导致换行符字节不同的影响。

要移到文件中某个指定的字节，请用 "go" 命令。例如，要移到字节 2345：

```
2345go
```

使 用 XXD

一个真正的二进制编辑器用两种方式来显示文本：二进制和十六进制格式。你可以在 Vim 里通过转换程序 "xxd" 来达到这效果。该程序是随 Vim 一起发布的。

首先以二进制方式编辑这个文件：

```
vim -b datafile
```

现在用 xxd 把这个文件转换成十六进制：

```
:%!xxd
```

文本看起来像这样：

```
0000000: 1f8b 0808 39d7 173b 0203 7474 002b 4e49  ....9..;..tt.+NI
0000010: 4b2c 8660 eb9c ecac c462 eb94 345e 2e30  K,.‘.....b..4^.0
0000020: 373b 2731 0b22 0ca6 c1a2 d669 1035 39d9  7;’1."....i.59.
```

现在你可以随心所欲地阅读和编辑这些文本了。 Vim 把这些信息当作普通文本来对待。修改了十六进制部分并不导致可显示字符部分的改变，反之亦然。

最后，用下面的命令把它转换回来：

```
:%!xxd -r
```

只有十六进制部分的修改才会被采用。右边可显示文本部分的修改忽略不计。

欲知更多详情，参见 xxd 手册。

23.5 压缩文件

这很容易：你可以像编辑任何其它文件一样，来编辑一个经过压缩的文件。插件 "gzip" 负责在你编辑这个文件时把它解压缩，以及在你存盘时再把它压缩起来。

目前支持的压缩方法有下面这些：

```
.Z      compress
.gz     gzip
.bz2    bzip2
```

Vim 调用上面提到的程序来实际完成压缩和解压缩。你也许需要先安装这些程序。

下一章： usr_24.txt 快速插入

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Chimin Yen，tocer http://vimcdoc.sf.net

快速插入

输入文本时，Vim 提供你各种各样的方法来减少键击次数和避免打字错误。你可以利用插入模式下的补全功能来重复先前打过的单词。也可以把长词缩写成短词。甚至可以打出你键盘上没有的字符。

24.1 更正
24.2 显示匹配
24.3 补全
24.4 重复一次插入
24.5 从另一行拷贝
24.6 插入一个寄存器内容
24.7 缩写
24.8 插入特殊字符
24.9 二合字母
24.10 普通模式命令

下一章：usr_25.txt 编辑带格式的文本
前一章：usr_23.txt 编辑特殊文件
目录：usr_toc.txt

24.1 更正

退格键 <BS> 已经在前面提过了。它删除位于光标前一格的字符。而删除键 <Del> 则删除光标下（或者说光标后也可以）的那个字符。

当你把整个词都打错了的时候，用 CTRL-W 来更正：

```
The horse had fallen to the sky
                              CTRL-W
The horse had fallen to the
```

如果你把一行字弄得不可收拾，而要从头来过的话，用 CTRL-U 来删除。这个命令保留了光标之后的文本，也保留了行首的缩进。它只删除了自第一个非空字符至光标位置之间的文本。让光标位于下一行中 "fallen" 的 "f" 上，按 CTRL-U，文本就成了这样：

```
The horse had fallen to the
              CTRL-U
fallen to the
```

当你发现几个词之前有个错误，你需要把光标移到那儿作更正。例如，你打了这样一行：

```
The horse had follen to the ground
```

你要把 "follen" 改成 "fallen"。让光标留在行尾，你输入这个字符串就能更正那错误：

```
                                  <Esc>4blraA
```

```
脱离插入模式                    <Esc>
退回 4 个单词                     4b
移到字母 "o" 上                     l
以字母 "a" 替代                     ra
重新开始插入模式                      A
```

另一种更正这个错误的方法：

```
<C-Left><C-Left><C-Left><C-Left><Right><Del>a<End>
```

```
退回 4 个词                <C-Left><C-Left><C-Left><C-Left>
移到字母 "o" 上                      <Right>
删除字母 "o"                            <Del>
插入字母 "a"                               a
移到行尾                                  <End>
```

这种方法让你留在插入模式下，利用特殊键来移动光标。这类似于你在一个不分模式的编辑器里所采用的操作方法。它比较容易记忆，但比较费事（你必须把你的手从字母键移动到光标方向键，而且，不看键盘难以按准 <End> 键）。

这些特殊键在书写一个停留在插入模式下的映射时非常有用。额外所需的键盘录入也是值得的。

你在插入模式下可以利用的特殊键有以下这些：

```
<C-Home>        移到文件首
<PageUp>        上卷一屏
<Home>          移到行首
<S-Left>        左移一个单词
<C-Left>        左移一个单词
<S-Right>       右移一个单词
<C-Right>       右移一个单词
<End>           移到行尾
<PageDown>      下卷一屏
<C-End>         移到文件尾
```

还有很多键，参见 ins-special-special 。

24.2 显示匹配

你键入一个右括号) 时，如果能知道它匹配哪个左括号 (会很方便。要让 Vim 做到这点，用下面这个命令：

```
:set showmatch
```

现在你键入比如 "(example)" 这样几个字，当你键入右括号) 时，Vim 就把光标闪到匹配的左括号 (上，在那儿停留半秒钟，然后返回原处。

如果匹配的左括号不存在，Vim 就鸣起响铃。这样你就会想起来你可能在哪儿忘了一个左括号，或键入了太多次的右括号。

Vim 也会为 [] 和 {} 这样的括号显示匹配。你不必等待光标返回原处才键入下一个字符，只要下一个字符一键入，光标就会返回，而插入就会跟以前一样继续。

你可以用 'matchtime' 选项改变 Vim 在匹配括号上停留的时间。例如，要让 Vim 等待 1.5 秒：

```
:set matchtime=15
```

指定的时间以 1/10 秒为单位。

24.3 补全

Vim 能自动补全插入的单词。你键入一个单词的开头部分，按 CTRL-P，Vim 就会为你猜测余下的部分。

例如，假定你正在建立一个 C 程序，并要键入以下语句：

```
total = ch_array[0] + ch_array[1] + ch_array[2];
```

你先输入下面这部分：

```
total = ch_array[0] + ch_
```

此时，你用命令 CTRL-P 告诉 Vim 来补全这个词。Vim 就会搜索以光标前字符串开头的词。在这个例子中，就是 "ch_"，与词 ch_array 匹配。所以，键入 CTRL-P 就会得到下面的结果：

```
total = ch_array[0] + ch_array
```

再键入几个字符使这个语句变成这样（结尾是空格）：

```
total = ch_array[0] + ch_array[1] +
```

如果你现在键入 CTRL-P，Vim 将再次搜索以补全光标前的词。由于光标前是空格，它找到的是之前的第一个词，即 "ch_array"。再键入 CTRL-P 给你下一个匹配的词，在本例中就是 "total"。第三次 CTRL-P 搜寻更前面的。如果那儿没其它的了，编辑器就会陷入无词可配状态，所以搜索就返回原处，即那个空格。第四次 CTRL-P 导致编辑器周而复始，又找到 "ch_array"。

往下搜索，用 CTRL-N。由于在文件结尾搜索又绕回开头，CTRL-N 和 CTRL-P 将找到相同的匹配，但顺序不同。提示：CTRL-N 意为下一个匹配，而 CTRL-P 意为前一个匹配。（译者：英文 Next 意为下一个，Previous 意为前一个）

Vim 编辑器会非常努力的来补全不完整的词。默认情况下，它搜索如下一些地方：

1. 当前文件
2. 其它窗口内的文件
3. 其它载入文件（隐藏缓冲区）
4. 未载入文件（非激活缓冲区）
5. 标签文件
6. 被当前文件以 #include 语句包含的所有头文件

选 项

你可以利用 'complete' 选项定制搜索顺序。

还可以使用 'ignorecase' 选项。设定这个选项后，搜寻匹配时大小写的区别就会被忽略。

一个特殊的补全选项是 'infercase'。它的用处是在寻找忽略大小写的匹配时 ('ignorecase' 必须先被设定)，但仍然采用已键入部分的大小写。这样，如果你键入 "For" 而 Vim 找到了匹配 "fortunately"，所产生的结果将是 "Fortunately"。

补全特定文本

如果你知道你要找什么，那么你可以用这些命令来补全某种类型的文本：

CTRL-X CTRL-F	文件名
CTRL-X CTRL-L	整行
CTRL-X CTRL-D	宏定义 (包括包含文件里的)
CTRL-X CTRL-I	当前文件以及所包含的文件
CTRL-X CTRL-K	字典文件内的单词
CTRL-X CTRL-T	同义词词典文件内的单词
CTRL-X CTRL-]	标签
CTRL-X CTRL-V	Vim 命令行

每个命令之后，CTRL-N 可以用来搜索下一个匹配，而 CTRL-P 则用于搜索前一个匹配。
关于每个命令的详细用法，参见： ins-completion 。

补全文件名

我们以 CTRL-X CTRL-F 为例。这个命令将找寻文件名。它在当前目录里搜索文件，并显示每一个与光标前单词匹配的文件名。
例如，假定你在当前目录里有下面这些文件：

```
main.c  sub_count.c  sub_done.c  sub_exit.c
```

现在进入插入模式并开始键入：

```
The exit code is in the file sub
```

就在这点上，你输入 CTRL-X CTRL-F。现在，Vim 通过查看当前目录里的文件来补全当前词 "sub"。最初的匹配是 sub_count.c。这不是你想要的，所以你按 CTRL-N 以匹配下一个文件。这次匹配的是 sub_done.c。再键入 CTRL-N 给了你 sub_exit.c。结果：

```
The exit code is in the file sub_exit.c
```

如果文件名以 / (Unix) 或 C:\ (MS-Windows) 开头，那么你就能搜索文件系统下所有的文件。例如，键入 "/u" 然后 CTRL-X CTRL-F，这将匹配 "/usr" (这是在 Unix 上)：

```
the file is found in /usr/
```

如果你现在按 CTRL-N，你就又回到 "/u"。接受 "/usr/" 并进入下一层目录，再来一次 CTRL-X CTRL-F：

```
the file is found in /usr/X11R6/
```

当然，匹配结果取决于你的文件系统上有什么文件。匹配结果以字母顺序来排列。

补全源代码

源代码文件有良好的结构。这使通过某种智能方式补全成为可能。在 Vim 中，这被称为全能补全。在其他编辑器中，它被称为智能补全(intellisense)，但这是一个注册商标。

全能补全的热键是 CTRL-X CTRL-O。显然，O 在这里代表全能 (Omni)，这样方便我们记忆。让我们以编辑 C 程序为例：

```
{ ~
    struct foo *p; ~
    p-> ~
```

光标在 "p->" 之后。现在键入 CTRL-X CTRL-O。Vim 会给你提供一个可选项的列表，这些可选项为 "struct foo" 所拥有。这和使用 CTRL-P 有很大不同，后者补全任意单词，而我们这里只要求 "struct foo" 的成员。

为使全能补全工作，需要做一些初始化。起码，要保证打开文件类型插件。你的 vimrc 文件应该包含形如：

```
filetype plugin on
```

或：

```
filetype plugin indent on
```

的一行。

对于 C 代码，需要建立标签文件并设置 'tags' 选项。在 ft-c-omni 中有进一步的解释。对于其他文件类型，需要做类似的事情，请查看 compl-omni-filetypes 。补全只对特定文件类型有效。查看 'omnifunc' 选项的值，以便检查补全能否正常工作。

24.4 重复一次插入

如果你按 CTRL-A，编辑器就把你上次在插入模式下输入的文本再输入一次。
比如，假定你有个文件，开头是这样的：

```
"file.h"
/* Main program begins */
```

你在第一行开始处插入 "#include "：

```
#include "file.h"
/* Main program begins */
```

你再用命令 "j^" 往下来到下一行的开始处。现在你开始插入一个新的 "#include" 行。所以你键入：

```
i CTRL-A
```

结果就像下面这样：

```
#include "file.h"
#include /* Main program begins */
```

"#include " 被插入是因为 CTRL-A 会插入上次插入过的文本。现在你键入 "main.h" <Enter> 以结束这一行：

```
#include "file.h"
#include "main.h"
/* Main program begins */
```

CTRL-@ 命令会完成 CTRL-A 的操作后退出插入模式。这是一个快速重复插入一模一样的文本的一个方法。

24.5 从另一行拷贝

CTRL-Y 命令插入光标上方的字符。当你复制前一行文本的时候，这个命令很有用。例如，你有这么一行 C 代码：

```
b_array[i]->s_next = a_array[i]->s_next;
```

现在你需要把这一行再键入一次，并以 "s_prev" 取代 "s_next"。换行以后，按 14 次 CTRL-Y，直到光标位于 "next" 的 "n" 上：

```
b_array[i]->s_next = a_array[i]->s_next;
b_array[i]->s_
```

现在你键入 "prev"：

```
b_array[i]->s_next = a_array[i]->s_next;
b_array[i]->s_prev
```

继续按 CTRL-Y 直到下一个 "next"：

```
b_array[i]->s_next = a_array[i]->s_next;
b_array[i]->s_prev = a_array[i]->s_
```

现在键入 "prev;" 以结束这一行。

CTRL-E 命令操作起来跟 CTRL-Y 一样，只不过它插入光标下方的字符。

24.6 插入一个寄存器内容

命令 CTRL-R {register} 插入寄存器里的内容。它的用处是让你不必键入长词。例如，你要输入下面这些：

```
r = VeryLongFunction(a) + VeryLongFunction(b) + VeryLongFunction(c)
```

这个函数的定义见于另一个文件。编辑那个文件并把光标移到该函数名上，然后把文件名摄入寄存器 v：

```
"vyiw
```

"v 指定寄存器，"yiw" 意思是拷贝一个词，不含空格 (yank-inner-word)。现在编辑那个要插入一行代码的文件，先键入开头几个字符：

```
r =
```

现在用 CTRL-R v 来插入函数名：

```
r = VeryLongFunction
```

你接下来在函数名后面键入其它必要字符，然后再用两次 CTRL-R v。

你也可以用补全功能来完成同样的工作。但当你有好几个词，其开头几个字符都一样的时候，寄存器就有用多了。

如果寄存器存放着诸如 <BS> 或其它特殊字符，这些字符就被解释成好像它们本来是从键盘键入的。如果你不要这样解释（你确实要在文本中插入 <BS>），那么要命令 CTRL-R CTRL-R {register}。

24.7 缩写

缩写是取代一个长词的短词。例如，"ad" 指代 "advertisement"。Vim 让你键入缩写，然后为你自动扩展。

用以下命令告诉 Vim，每当你输入 "ad" 就把它扩展成 "advertisement"：

```
:iabbrev ad advertisement
```

现在，当你键入 "ad"，完整的单词 'advertisement" 就被插入文本。键入一个不可能成为单词一部分的字符，例如一个空格，就会触发缩写功能：

输入的文本	看到的文本
I saw the a	I saw the a
I saw the ad	I saw the ad
I saw the ad<Space>	I saw the advertisement<Space>

当你仅仅键入 "ad" 时，扩展并没发生。它可以被你输入 "add" 这样的词而不被扩展。Vim 只对那些完整的词检查缩写。

多词缩写

为几个词定义一个缩写也是可能的。例如，用下面这个命令，把 "JB" 定义成 "Jack Benny"：

```
:iabbrev JB Jack Benny
```

作为程序员，我使用两个相当不寻常的缩写：

```
:iabbrev #b /*****************************************
:iabbrev #e <Space>*****************************************/
```

它们用于生成大段注释。注释以缩写 #b 开始，划出顶线。接着，我键入注释文字，最后以缩写 #e 划出底线。

注意 缩写 #e 以一个空格开头。换言之，开头两个字符是空格和星号 (*)。通常 Vim 忽略不计位于缩写及其扩展之间的空格。为了避免空格被忽略，我把空格以七个字符表示：<、S、p、a、c、e、>。

备注：
":iabbrev" 有点嫌长。":iab" 作用也一样。缩写命令被缩写了！

更 正 打 字 错 误

我们经常会犯同一个打字错误。例如，把 "the" 打成 "teh"。你可以利用缩写功能来更正这样的错误：

```
:abbreviate teh the
```

你可以加上一系列这样的缩写。每次发现一个常见错误就加一个。

缩 写 列 表

":abbreviate" 命令列出所有缩写：

```
:abbreviate
i  #e             ****************************************/
i  #b            /****************************************
i  JB            Jack Benny
i  ad            advertisement
!  teh           the
```

第一栏的 "i" 表明插入模式。这些缩写仅仅在插入模式下有作用。其它可能的字符：

```
c       命令行模式                                  :cabbrev
!       插入模式和命令行模式                        :abbreviate
```

在命令行模式下使用缩写是不常见的。你主要会在插入模式下用 ":iabbrev" 命令。这样就避免了不必要的扩展，例如，当你键入这样一条命令时，"ad" 就不会被扩展了：

```
:edit ad
```

删 除 缩 写

删除一个缩写，用 ":unabbreviate" 命令。假定你有以下缩写：

```
:abbreviate @f fresh
```

你可以用这个命令删除它：

```
:unabbreviate @f
```

当你键入这个命令的时候，你将注意到 @f 被扩展成 "fresh"。别担心，Vim 明白得很呢(除非当你另有缩写 "fresh"，但那是很偶然的)。

要删除全部缩写：

```
:abclear
```

":unabbreviate" 和 ":abclear" 另有变形，在插入模式下是 "iunabbreviate" 和 ":iabclear"，在命令行模式下是 ":cunabbreviate" 和 "cabclear"。

缩 写 再 映 射

定义缩写时，有一点要注意的：扩展产生的字符串不应当被映射成别的什么。例如：

```
:abbreviate @a adder
:imap dd disk-door
```

现在，你键入 @a，你得到 "adisk-doorer"。那不是你要的结果。为了避免这种事发生，用 ":noreabbrev" 命令。它的作用跟 ":abbreviate" 一样，但却避免了扩展产生的字符串被用于映射：

```
:noreabbrev @a adder
```

现在好了，缩写扩展后的结果不可能被映射了。

24.8 插入特殊字符

CTRL-V 命令用来按本义插入下一个字符。换言之，无论该字符多特殊，其特殊含义都被忽略不计。例如：

```
CTRL-V <Esc>
```

插入 <Esc> 字符。而你并未脱离插入模式。(不要在 CTRL-V 后面键入空格，那个空格仅仅为了方便阅读)。

> 备注：
> 在 MS-Windows 环境下，CTRL-V 用来粘贴文本。所以用 CTRL-Q 代替 CTRL-V。另外，在 Unix 环境下，CTRL-Q 在某些终端上不起作用，因其另有特殊意义。

你也可以用命令 CTRL-V {digits} 来插入一个以若干个十进位数字 {digits} 表示的字符。例如，字符编码 127 是字符 <Del> (但并不一定是 <Del> 键！)。要插入 <Del>，键入：

```
CTRL-V 127
```

你可以用这样的方法输入数值不超过 255 的字符。当你键入一个不足两位的数字时，就要添加一个非数字的字符来结束命令。为了回避这个非数字字符的要求，在它前面加一或两个零以满足三位数的要求。

以下命令都插入一个 <Tab> 字符，然后一个点字符：

```
CTRL-V 9.
CTRL-V 09.
CTRL-V 009.
```

输入一个 16 进制数字，在 CTRL-V 后面，用 "x" 开头：

```
CTRL-V x7f
```

这方法也可以用来输入数值不超过 255 (CTRL-V xff) 的字符。你可以用 "o" 开头输入一个以 8 进制数表示的字符，以及另外两种方法，让你输入多至二进制 16 位和 32 位的数字表示的字符 (例如，Unicode 字符)：

```
CTRL-V o123
```

```
CTRL-V u1234
CTRL-V U12345678
```

24.9 二合字母

有些字符在键盘上找不到。例如，表示版权的字符 © 要在 Vim 里键入这样的字符，你得用二合字母，即以两个字符来表示一个。例如，要键入 ¾ ，你就得按三个键：

```
CTRL-K Co
```

你可以用以下命令来查找有哪些二合字母可供利用：

```
:digraphs
```

Vim 将把二合字母表显示在屏幕上。以下就是其中的三行：

```
AC ~_ 159  NS |  160  !I  161  Ct  162  Pd  163  Cu ¤  164  Ye  165
BB  166  SE §  167  ': ¨  168  Co  169  -a  170  <<  171  NO  172
--  173  Rg  174  'm  175  DG °  176  +- ±  177  2S  178  3S  179
```

这张表告诉你，比如，你键入 CTRL-K Pd 所得到的二合字母是字符 £。该字符编码为 163（十进制）。

Pd 是 Pound（英镑）的简写。大多数二合字母让你一看就猜到两个字符生成什么字符。如果你一个个读下来，便不难理解其中的逻辑。

你可以交换两个字符的顺序，只要那样组合不代表另一个二合字母。因此 CTRL-K dP 也没问题。由于 "dP" 不是二合字母，Vim 会转而搜索表示成 "Pd" 的二合字母。

> 备注：
> 二合字母表取决于 Vim 假定你所使用的字符集。在 MS-DOS 系统上采用的二合字母集不同于 MS-Windows 系统上的。务必用 ":digraphs" 命令来查找当前可供利用的二合字符。

你可以定义你自己的二合字母。例如：

```
:digraph a"
```

以上命令定义了 CTRL-K a" 插入一个 ä 字符。你也可以用十进值数指定这个字符。下面这个命令定义了同一个二合字母：

```
:digraph a" 228
```

更多关于二合字母信息参见： digraphs

另一种插入特殊字符的方法是利用键盘映射。详情参见： 45.5

24.10 普通模式命令

插入模式提供的命令数量有限。在普通模式下，你可用的命令就多得多了。当你要用一个普通模式命令时，你通常用 <Esc> 键来脱离插入模式，执行这个普通模式命令，然后再用 "i" 或 "a" 命令重新进入插入模式。

有一个快捷的方法。用 CTRL-O {command} 你可以在插入模式下执行任何普通模式命令。例如，把光标后面直至行尾的文本删除：

```
CTRL-O D
```

用这个方法，你只能执行一个普通模式命令。但是，你可以指定一个寄存器或一个计数。
请看下面这个更复杂的命令：

```
CTRL-O "g3dw
```

这个命令把光标后面三个单词一起删除，存进了寄存器 g。

下一章： usr_25.txt 编辑带格式的文本

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：snoopyxp http://vimcdoc.sf.net

编辑带格式的文本

很少有那种一行一个句子的文本。这一章我们将要介绍如何为了适合页面而为句子分行以及进行其它的编排操作。当然，针对编辑单行的段落和表格，Vim 也有一些有用的特性。

25.1 断行
25.2 对齐文本
25.3 缩进和制表符
25.4 对长行的处理
25.5 编辑表格

下一章：usr_26.txt 重复
上一章：usr_24.txt 快速插入
目录：usr_toc.txt

25.1 断行

Vim 有一系列功能可以使处理文本变得更容易。在默认情况下，Vim 并不会自动换行，换句话说，你得自己敲 <Enter>。这在你编写程序并想自己决定哪里是行尾时会很有用。但当你想建立一个每行至多有 70 个字符宽的文档时，这就显得没那么好了。

如果你设置了 'textwidth' 选项，Vim 就会自动插入换行符。举个例子，假设您想要一个只有 30 个字符宽的非常窄的栏，你需要执行下面的命令：

```
:set textwidth=30
```

现在开始输入正文（加入了标尺）：

```
          1         2         3
123456789012345678901234567890123456
I taught programming for a whi
```

如果接下来你输入 "l"，这将使此行的长度超出 30 个字符的限制。当 Vim 发现这种情况时，它会自动插入换行符。你将得到下面的结果：

```
          1         2         3
123456789012345678901234567890123456
I taught programming for a
whil
```

接着，你可以输入剩下的段落：

```
          1         2         3
123456789012345678901234567890123456
I taught programming for a
while. One time, I was stopped
by the Fort Worth police,
because my homework wos too
```

```
hard. True story.
```

你不用输入换行符，Vim 会自动插入的。

> 备注：
> 'wrap' 选项会使 Vim 分行显示文本，但并不会在文件中插入换行符。

重 排 文 本：

Vim 编辑器并不是一个字处理器。字处理器在你删除了段落开头的一些东西时会重新调整换行符。(译者注：这意味着后边的文本会向前补进，使换行位置保持不变)。但在 Vim 里并不是这样。因此，当你从第一行删除单词 "programming" 时，你所得到的只是一个短行：

```
          1         2         3
12345678901234567890123456789012345
I taught for a
while. One time, I was stopped
by the Fort Worth police,
because my homework was too
hard. True story.
```

这看起来不大美观。为了保持段落的形状，你要用到 "gq" 操作符。

下面我们首先结合一个可视选择区来使用它。从第一行开始，键入：

```
v4jgq
```

"v" 用来进入可视模式，"4j" 用来移动到段落结尾，接下来是 "gq" 操作符。下面是得到的结果：

```
          1         2         3
12345678901234567890123456789012345
I taught for a while. One
time, I was stopped by the
Fort Worth police, because my
homework was too hard. True
story.
```

备注：有一种方法可以自动地排版特定文本类型的布局，参见 auto-format 。

因为 "gq" 是一个操作符，所以你可以通过下面三种方法之一来选择要操作的文本范围：可视模式，一个移动操作，或是一个文本对象。

上面的例子也可以用 "gq4j" 来完成。这减少了击键次数，但你得知道行数。一个更有用的移动命令是 "}"。使用这个命令可以移动到一个段落的结尾处。因此，"gq}" 将对从当前光标处至当前段段尾所包括的文本进行排版操作。

可以和 "gq" 一起使用的一个很有用的文本对象是段落。试一下这个：

```
gqap
```

"ap" 意味着 "a-paragraph" (译者注：一个段落)。这将对一个段落 (用空行隔开) 的文本进行排版操作。也包括在光标之前的部分。

如果你所有的段落都已经用空行分隔好了，你可以键入以下命令来对整个文件进行格式化操作：

```
gggqG
```

"gg" 用来移动到第一行，"gqG" 用来对文本进行排版操作直到最后一行。
警告：如果你的段落没有被正确的分隔开，它们将被连接在一起。一个常见的错误是文件内包含一些只有空格或制表符的行。这些是空白行，但不是空行。

Vim 并不是只能对纯文本进行排版操作。请参考 fo-table 一节获取更详细的信息。参考 'joinspaces' 选项以了解如何改变在句号之后加入的空格数。
也可以调用外部程序来做排版。这在 Vim 自带的排版功能无法满足你的文本的需要时很有用。参阅 'formatprg' 选项。

25.2 对齐文本

要使一定范围包括的行居中，用以下命令：

```
:{range}center [width]
```

{range} 即是那些通常命令所能指定的范围。 [width] 是一个可选项，用来确定要居中的行的宽度。如果没有指定 [width]，则默认使用 'textwidth' 的值。(如果 'textwidth' 是 0，则默认为 80。)
例如：

```
:1,5center 40
```

将得到下面的结果：

```
      I taught for a while. One
      time, I was stopped by the
    Fort Worth police, because my
     homework was too hard. True
                story.
```

右 对 齐

类似地，":right" 命令将使文本右对齐：

```
:1,5right 37
```

将得到这种结果：

```
          I taught for a while. One
        time, I was stopped by the
    Fort Worth police, because my
      homework was too hard. True
                             story.
```

左 对 齐

我们使用这个命令：

```
:{range}left [margin]
```

和 ":center" ":right" 不同的是，":left" 的参数并不是行的宽度，而是左边的页边距。如果这个参数被省略了，文本将被紧靠在屏幕的左边缘（用一个零页边距参数将得到一样的效果）。如果页边距是 5，文本将缩进五个空格。举个例子，使用以下命令：

```
:1left 5
:2,5left
```

结果会是下面这样：

```
     I taught for a while. One
time, I was stopped by the
Fort Worth police, because my
homework was too hard. True
story.
```

两 端 对 齐

Vim 并没有提供内建的方法来使文本两端对齐。不过，我们可以通过使用一个灵巧的宏来做这项工作。要使用这个包，执行下列命令：

```
:packadd justify
```

或在 .vimrc 里放上之行：

```
packadd! justify
```

这个 Vim 脚本文件定义了一个新的可视命令 "_j"。要使一块文本两端对齐，只需在可视模式中选择该文本块，然后执行 "_j"。

请查阅该文件以获得更详细的解释说明。在文件名 $VIMRUNTIME/pack/dist/opt/justify/plugin/justify.vim 上键入命令 "gf"，就可以打开该文件。

作为另一个选择，你还可以使用一个外部程序来对文本进行过滤。例如：

```
:%!fmt
```

25.3 缩进和制表符

缩进可以被用来突显特定的文本。举个例子，在本手册的示例文本用八个空格或一个制表符来缩进。通常，你可以在每行的开头键入一个制表符来输入下面内容。用这个文本：

```
the first line
the second line
```

这些文本是这样输入的，先输入一个制表符，再输入一些文本，然后键入 <Enter>，接着又是一个制表符，然后输入另一些文本。

置位 'autoindent' 选项可以自动插入缩进：

```
:set autoindent
```

当开始一个新行时，新行会采用和上一行相同的缩进。在上面的例子中，我们再也不需要在 <Enter> 后面输入制表符了。

增 加 缩 进

要增加一行中的缩进量，可以使用 ">" 操作符。一个经常使用的操作是 ">>"，这将为当前行增加缩进。

增加的缩进量是使用 'shiftwidth' 来指定的。默认的值是 8。举例来说，要使 ">>" 插入四个空格宽度的缩进，键入：

```
:set shiftwidth=4
```

当你在示例文本中的第二行上使用时，你会得到下面的结果：

```
the first line
    the second line
```

"4>>" 将为四行增加缩进。

制 表 位

如果你想使缩进量为 4 的倍数，你需把 'shiftwidth' 设置为 4。但是当你敲下 <Tab> 键时，你仍然会得到八个空格宽度的缩进。要改变这种情况，请设置 'softtabstop' 选项。

```
:set softtabstop=4
```

这将使 <Tab> 键插入四个空格宽度值的缩进量。如果已经存在四个空格，就加上一个 <Tab> 字符（在文件中省去了七个字符）。(如果你只想要空格而不想加上 tab 字符，请置位 'expandtab' 选项。)

> 备注：
> 你可以把 'tabstop' 选项设置为 4。尽管如此，如果你在另一个时间再次编辑这个文件时 'tabstop' 的默认值是 8，文件缩进看起来会不对。在其它程序中或者在打印的时候，缩进也将是错的。因此，建议把 'tabstop' 的值始终保持为 8。这是所有地方的标准值。

更 换 制 表 位

当你在 Vim 中编辑一个 tabstop 为 3 的文件时，文件会看起来很难看，因为在 Vim 中 tabstop 的正常值为 8。你可以通过把 'tabstop' 的值设置为 3 来更正它。但是你每一次你编辑这个文件都得做这个更改。

Vim 可以更换你文件使用的制表位。首先，设置 'tabstop' 的值使缩进看起来美观，然后使用 ":retab" 命令：

```
:set tabstop=3
:retab 8
```

":retab" 命令将把 'tabstop' 的值改为 8，同时仍保持文件看起来不变。它把空白区间用制表和空格重新构造。现在你可以写入这个文件。下次你再编辑它的时候缩进将是正确的，你不需要改变任何选项。

警告：当你对一个程序文件使用 ":retab" 命令时，它可能会改变一个字符串常量中的空白。因此，要养成在程序中使用 "\t" 而不是输入一个制表符的好习惯。

25.4 对长行的处理

有时你会编辑一个比窗口列数宽的文件。当发生这种情况时，Vim 将把文件行回绕显示以便适应屏幕显示。

如果你将 'wrap' 选项设置为关闭，文件中的每一行都将在屏幕上仅作为一行显示。这时行尾会超出屏幕右端从而无法看到。

当你移动光标到视野之外的字符时，Vim 将滚动文本来显示它。这就好像在文本的水平方向移动视窗一样。

默认情况下，Vim 并不在 GUI 中显示水平滚动条。如果你想启用它，使用下面的命令：

```
:set guioptions+=b
```

一个水平滚动条将出现在 Vim 窗口的底部。

如果你没有滚动条或者你不想使用它，可以用下面这些命令来滚动文本。光标将停留在同样的地方，但在必要时会回移到可见文本区。

zh	向右滚动
4zh	向右滚动四个字符
zH	向右滚动半个窗口宽度
ze	向右滚动使光标处于行尾
zl	向左滚动
4zl	向左滚动四个字符
zL	向左滚动半个窗口宽度
zs	向左滚动使光标处于行首

让我们试着用一行文本来演示一下。光标停留在 "which" 的 "w" 处。那行上方的 "当前窗口" 标示当前可见的文本。下方的 "窗口" 指示了执行了左边的命令后可见的文本区域。

```
                        |<--     当前窗口     -->|
          some long text, part of which is visible in the window
ze          |<--       窗口       -->|
zH           |<--       窗口       -->|
4zh                |<--       窗口       -->|
zh                   |<--       窗口       -->|
zl                     |<--       窗口       -->|
4zl                       |<--       窗口       -->|
zL                              |<--       窗口       -->|
zs                              |<--       窗口       -->|
```

在关闭折行情况下移动

当 'wrap' 选项被关闭，文本将在水平方向卷动。你可以通过以下命令来使光标移动到你可以看到的一个字符处。这样超出窗口左右两端的文本将被忽略。这些命令不会使文本卷动：

g0	移动到当前行的第一个可见字符
g^	移动到当前行的第一个非空白的可见字符
gm	移动到屏幕行的中点
gM	移动到当前行的文本中点

g$ 移动到当前行的最后一个可见字符

```
        |<--       窗口       -->|
some long    text, part of which is visible in one line
         g0  g^    gm      gM g$
```

断 词 edit-no-break

当你为另一个程序准备文本时，你或许需要使段落没有一处换行。使用 'nowrap' 选项的一个弊端是你看不见你正在处理的整个句子。当 'wrap' 选项开启时，会从单词中间断开，从而难以阅读。

编辑此类段落时，一个好的解决方法是设置 'linebreak' 选项。这样，Vim 将会在一个适当的地方回绕行显示，同时仍保持文件中的文本不变。

没有设置 'linebreak' 选项时的文本看起来可能是这样：

```
+---------------------------------+
|letter generation program for a b|
|ank.  They wanted to send out a s|
|pecial, personalized letter to th|
|eir richest 1000 customers.  Unfo|
|rtunately for the programmer, he |
+---------------------------------+
```

使用如下命令之后：

```
:set linebreak
```

看起来会是这样：

```
+---------------------------------+
|letter generation program for a  |
|bank.  They wanted to send out a |
|special, personalized letter to  |
|their richest 1000 customers.    |
|Unfortunately for the programmer,|
+---------------------------------+
```

相关选项：
'breakat' 指定了可以用来作为插入换行地点的字符。
'showbreak' 指定了一个用于显示在回绕行显示行的行首的字符串。
设置 'textwidth' 的值为零来避免一个段落被拆分。

在 可 见 行 间 移 动

使用 "j" 和 "k" 命令可以移动到下一行和上一行。当作用于一个长行时，这通常意味着要一次移动许多屏幕行。

使用 "gj" 和 "gk" 命令可以只移动一个屏幕行。当一行没有回绕时，它们和 "j" 和 "k" 命令所起的作用一样。当一行回绕时，它们将在屏幕上显示的上一行和下一行的一个字符之间移动。

下面这些移动命令的键映射也许对你有用：

```
:map <Up> gk
:map <Down> gj
```

变 段 为 行 edit-paragraph-join

如果你想把文本导入类似 MS-Word 的程序中，每个段落就要变成一个单行。如果你的段落是由空行分隔开的，下面这个命令就可以要把一个段转化为一个单行：

```
:g/./,/^$/join
```

这看起来挺复杂。让我们把它分解开：

```
:g/./          一个 ":global" 命令，用来搜索至少含一个字符的所有行。
    ,/^$/      一个范围，从当前行开始（非空行）到一个空行。
         join  ":join" 命令把范围内的行连接成一行。
```

从下面这段含有八行，且在第三十列换行的文本开始：

```
+---------------------------------+
|A letter generation program      |
|for a bank.  They wanted to      |
|send out a special,              |
|personalized letter.             |
|                                 |
|To their richest 1000            |
|customers.  Unfortunately for    |
|the programmer,                  |
+---------------------------------+
```

你将得到下面两行：

```
+---------------------------------+
|A letter generation program for a |
|bank.  They wanted to send out a s|
|pecial, personalized letter.     |
|To their richest 1000 customers.  |
|Unfortunately for the programmer, |
+---------------------------------+
```

注意 当分隔段落的行是含有空格和/或制表符的空白行而不是空行时，这行命令将不起作用。下列的命令对于空白行仍起作用：

```
:g/\S/,/^\s*$/join
```

要使最后一段也被连接，这行命令需要文件的结尾仍有一个空白行或空行。

25.5 编辑表格

设想你正在编辑一个含有四列的表格：

```
nice table        test 1        test 2      test 3
input A           0.534
input B           0.913
```

你需要在第三列输入数字。你可以先移动到第二行，键入 "A"，键入一大堆空格然后输入你要输入的文本。

对于这种类型的编辑操作有一个特殊的选项：

```
set virtualedit=all
```

现在你可以把光标移动到没有任何文本的位置。这叫做 "虚拟空间"。通过这种方法，编辑表格变得容易很多。

通过查找最后一列的标题来移动光标：

```
/test 3
```

现在按下 "j"，光标就到了你要输入对应 "input A" 的值的地方了。输入 "0.693"，结果是：

```
nice table      test 1     test 2      test 3
input A         0.534                  0.693
input B         0.913
```

Vim 已经自动为你填充了新文本前面的间隙。现在，要到此列的下一个域，键入 "Bj"。"B" 用来反向移动到空白间隔的单词的开始处。然后，用 "j" 移动到可以输入下一域的地方。

> 备注：
> 你可以把光标移动到窗口的任何地方，也可以超出行尾。但是，Vim 在你未在该位置插入字符之前并不会插入空格。

拷 贝 一 列

你想增加一列，它是第三列的一个拷贝，并且想放在 "test 1" 列的前面。做以下七步：

1. 移动光标到这一列的左上角，例如用 "/test 3"。
2. 按 CTRL-V 来开启可视列块模式。
3. 用 "2j" 使光标向下移动两行。你现在进入了 "虚拟空间": "input B" 行中对应对应列 "test 3" 的地方。
4. 把光标向右移动来选择整列，外加那些你想要的列之间的空格。用 "9l" 就行。
5. 用 "y" 来拷贝矩形选择区的内容。
6. 把光标移动到我们需要插入新列的地方 "test 1"。
7. 按下 "P"。

结果应该是：

```
nice table      test 3     test 1      test 2      test 3
input A         0.693      0.534                  0.693
input B                    0.913
```

需要注意的是整个 "test 1" 列被向右移动了，同时也包括 "test 3" 列中没有文本的行。

要返回到光标的非虚拟移动模式，用：

```
:set virtualedit=
```

虚 拟 替 换 模 式

使用 'virtualedit' 的弊端是你会 "感觉" 不太一样。当你移动光标的时候，你不能分辨超出行尾的制表符或空格。另一种可行的方法是：虚拟替换模式。

设想在表格中有一个包含了制表符和其它字符的行。在第一个制表符上使用 "rx"：

```
inp     0.693   0.534   0.693

        |
   rx   |
        V

inpx0.693   0.534       0.693
```

版面被弄乱了。要避免这种情况，使用 "gr" 命令：

```
inp     0.693   0.534   0.693

        |
  grx   |
        V

inpx    0.693   0.534   0.693
```

这里的情况是 "gr" 命令确保了新字符占据了正确的屏幕空间。额外需要的空格或制表符被插入到间隙中。然而，真正发生的是一个制表符被一个 "x" 代替，然后空白字符被加上来使文本保持它的位置。在这个例子里是插入了一个制表符。

当你需要替换多于一个字符时，使用 "R" 命令来进入替换模式（参看 04.9 ）。这样一来，原来的版面乱套了，而且替换了不该换的字符。

```
inp     0       0.534   0.693

        |
 R0.786 |
        V

inp     0.78634 0.693
```

"gR" 使用虚拟替换模式。这保全了版面布局：

```
inp     0       0.534   0.693

        |
gR0.786 |
        V

inp     0.786   0.534   0.693
```

下一章： usr_26.txt 重复

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者： Chimin Yen http://vimcdoc.sf.net

重复

编辑工作往往不是杂乱无章的。同一个修改动作常常会被重复多次。在本章中，我们将解释几种有用的重复修改动作的方法。

26.1 可视模式下的重复
26.2 加与减
26.3 改动多个文件
26.4 在外壳脚本里使用 Vim

下一章： usr_27.txt 查找命令及模式
前一章： usr_25.txt 编辑带格式的文本
目录： usr_toc.txt

26.1 可视模式下的重复

可视模式在反复修改一段文本方面十分方便。你可以看见高亮的文本，从而确定是否改对了地方。但选择文本也得打几个字。"gv" 命令再次选择上次选过的文本，让你对同一段文本再作别的修改。

假定你有几行文本，你要把其中的 "2001" 改成 "2002"，以及 "2000" 改成 "2001"：

```
The financial results for 2001 are better
than for 2000.  The income increased by 50%,
even though 2001 had more rain than 2000.
                2000            2002
income          45,403          66,234
```

首先把 "2001" 改成 "2002"。在可视模式下选择这几行文本，并执行：

```
:s/2001/2002/g
```

现在用 "gv" 命令再次选择同一文本。光标在哪儿都没关系。然后用 ":s/2000/2001/g" 命令作第二项修改。

很明显，你可以多次重复这些动作。

26.2 加与减

当你反复地把一个数改成另一个时，你常遇到定值的偏移。在上例中，每个年份都加了一。与其为每个要修改的年份都键入一个替换命令，不如利用 CTRL-A 命令。

在上面这段文本中搜索年份：

```
/19[0-9][0-9]\|20[0-9][0-9]
```

现在按 CTRL-A。找到的年份增加了一：

```
The financial results for 2002 are better
than for 2000.  The income increased by 50%,
even though 2001 had more rain than 2000.
                2000            2001
income          45,403          66,234
```

用 "n" 命令找到下一个年份，并按 "." 重复 CTRL-A 命令（键入 "." 会快一点儿）。为所有找到的年份重复 "n" 和 "."。

提示：设定 'hlsearch' 选项以突显那些你要修改的数字，然后你可以提前观察并修改得快些。

你可以在 CTRL-A 之前附加数字来增加一个大于一的数。假定你有这么个列表：

```
1. item four
2. item five
3. item six
```

把光标移到 "1." 上并键入：

```
3 CTRL-A
```

那个 "1." 就变成了 "4."。同样，你可以用 "." 命令对其它数字重复这项修改。

再看一个例子：

```
006     foo bar
007     foo bar
```

在这些数字上执行 CTRL-A 产生了以下结果：

```
007     foo bar
010     foo bar
```

7 加 1 等于 10? 这是因为 Vim 根据首位出现的 "0" 而误以为 "007" 是个八进位数字。这种表示法常用于 C 程序。如果你不要首位为 "0" 的数字被处理成八进位数字，请用这个命令：

```
:set nrformats-=octal
```

CTRL-X 命令用于减数字，用法与加数字类似。

26.3 改动多个文件

假定你有个变量名为 "x_cnt" 而你要把他改为 "x_counter"。这个变量在多个 C 文件都被用到了。你需要在所有文件中作此改动。你得这么做。

把所有相关文件放进参数列表：

```
:args *.c
```

这个命令会找到所有的 C 文件并编辑其中的第一个。现在你可以对所有这些文件执行替代命令：

```
:argdo %s/\<x_cnt\>/x_counter/ge | update
```

命令 ":argdo" 把另一个命令当作其参数。而后者将对参数列表内所有的文件执行。

作为参数的替代命令 "%s" 作用于所有文本行。它用 "\<x_cnt\>" 来查找 "x_cnt"。"\<" 和 "\>" 用来指定仅匹配那些完整的词，而不是 "px_cnt" 或 "x_cnt2"。

替代命令的标志位中包含 "g"，用以置换同一行文本内出现的所有的匹配 "x_cnt"。标志位 "e" 用于避免因文件中找不到 "x_cnt" 而出现错误信息。否则 ":argdo" 命令就会在遇到第一个找不到 "x_cnt" 的文件时中断。

字符 "|" 分隔两条命令。后面的 "update" 命令将那些有改动的文件存盘。如果没有 "x_cnt" 被改成 "x_counter"，这个命令什么也不做。

还有一个 ":windo" 命令，用于在所有窗口内执行其参数所规定的命令。以及 ":bufdo" 命令，对所有缓冲区执行其参数所规定的命令。使用中要小心，因为你在缓冲区列表中的文件数量可能超过你能想像的。请用 ":buffers" 命令 (或 ":ls") 来检查缓冲区列表。

26.4 在外壳脚本里使用 Vim

假定你要在很多文件内把字符串 "-person-" 改成 "Jones"，然后把它们打印出来。你该怎么做？一种方法是键入许许多多命令。另一种是写个外壳脚本来完成这件工作。

作为一个可视的交互式的编辑器，Vim 在执行普通模式命令时表现得极为出色。然而在批量处理时，普通模式命令无法产生简洁的带有注释的命令文件；在此，你该转而利用 Ex 模式。该模式为你提供一种友好的命令行界面，方便你把命令写进一个批处理文件。("Ex 命令" 无非是命令行 (:) 命令的另一个名称。)

以下就是你所需要的 Ex 模式命令:

```
%s/-person-/Jones/g
write tempfile
quit
```

你把这些命令放进文件 "change.vim" 里。现在就用这个外壳脚本在批量模式下运行编辑器:

```
for file in *.txt; do
  vim -e -s $file < change.vim
  lpr -r tempfile
done
```

循环 for-done 是一个外壳结构，用来重复执行循环结构内的两行命令，而变量 $file 则在每次循环时被设成不同的文件名。

脚本第二行的作用是对文件 $file 运行 Vim 且在 Ex 模式下 (参数 -e)，并从文件 "change.vim" 读取命令。参数 -s 告诉 Vim 运行在安静模式下。换言之，不要持续不断的发出 :prompt，或针对那个问题发出的任何别的提示。

命令 "lpr -r tempfile" 打印执行中产生的文件 "tempfile" 并把它删除 (是参数 -r 的作用)。

从标准输入读取

Vim 能从标准输入读取文本。由于通常从那里读取的是命令，你得告诉 Vim 你读的是文本。这需要在通常是文件名的地方传送一个参数 "-"。例如:

```
ls | vim -
```

这个命令让你编辑 "ls" 命令的输出结果，而不必先把那些输出文本存入一个文件。

如果你从标准输入读取文本，那么你可以用参数 "-S" 来读取脚本：

```
producer | vim -S change.vim -
```

普 通 模 式 脚 本

如果你真的要在脚本内利用普通模式命令，你可以这样来用：

```
vim -s script file.txt ..。
```

注意：
当 "-s" 不与 "-e" 一起用时，它的意思是不同的。此时，它的意思是把文件 "script" 里的命令当作普通模式命令来执行。而与 "-e" 一起用时，它的意思是保持安静，并不会把下一个参数视为文件名。

文件 "script" 里的命令就像你键入它们那样得到执行。别忘了换行符被解释成按下回车键 <Enter>。在普通模式下该键把光标移到下一行。

要创建这么一个脚本你可以编辑这个脚本文件，键入那些命令。你得想象每个命令会产生什么样的结果。这可不那么容易。另一种方法是在你手动执行那些命令时把它们记录下来。你可以采用下面的方法：

```
vim -w script file.txt ..。
```

所有键入的字符都将被写进文件 "script"。如果你犯了个小错误，不妨继续输入。但要记得事后更正一下这个脚本文件。

参数 "-w" 会将新键入的命令附加在一个已存在的脚本文件末尾。这在你需要一点儿一点儿记录该脚本时是很不错的。但当你要从零开始重新记录你的脚本时，你则需要用 "-W" 参数。该参数重写任何已存在的文件。

下一章： usr_27.txt 搜索命令和模式

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Chimin Yen http://vimcdoc.sf.net

查找命令及模式

在第三章里，我们曾经提到过几个简单的查找模式 03.9 。Vim 能够胜任复杂得多的查找。本章将解释那些最常用到的模式。详细的说明可以查阅： pattern

27.1 忽略大小写
27.2 在文件尾折返
27.3 偏移
27.4 匹配重复性模式
27.5 多择一
27.6 字符范围
27.7 字符类
27.8 匹配换行符
27.9 举例

下一章： usr_28.txt 折叠
前一章： usr_26.txt 重复
目录： usr_toc.txt

27.1 忽略大小写

默认情况下，Vim 的查找是大小写敏感的。因此，"include"，"INCLUDE"，和 "Include" 为三个各不相同的词，而一次查找将仅仅匹配其中的一个。

现在设定 'ignorecase' 选项：

```
:set ignorecase
```

再查找 "include"，现在它将匹配 "Include"，"INCLUDE" 和 "InClUDe"。(设定 'hlsearch' 选项可以快速查看哪儿有模式的匹配。)

你可以这样撤销这个选项：

```
:set noignorecase
```

让我们保留这个设定，并查找 "INCLUDE"。它匹配的文本就跟查找 "include" 时一模一样。现在我们设定 'smartcase' 选项：

```
:set ignorecase smartcase
```

如果你采用的模式里至少有一个大写字母，查找就成了大小写敏感的。可以这样理解，你不必键入大写字母，也能查找到含有大写字母的词，所以，你若键入大写字母，必然是因为你要大小写敏感的匹配。聪明！

设定了这两个选项，你会找到以下匹配：

模式	匹配
word	word、Word、WORD、WoRd 等。
Word	Word
WORD	WORD

```
WoRd                    WoRd
```

单个模式里的大小写

如果你仅想忽略一个特定的模式里的大小写，那么，在模式前添加 "\c" 字符串就行了。添加 "\C" 将使该模式的匹配大小写敏感。这排除了 'ignorecase' 和 'smartcase' 选项的影响，当 "\c" 或 "\C" 起作用时，它们设成什么值无关紧要。

模式	匹配
\Cword	word
\CWord	Word
\cword	word，Word，WORD，WoRd，等。
\cWord	word，Word，WORD，WoRd，等。

采用 "\c" 和 "\C" 的好处在于它粘附在所用的模式上。因此，重复查找历史记录里的某个模式会产生同样的结果。'ignorecase' 或 'smartcase' 是否更改不影响结果。

备注：
在查找模式中使用以 "\" 开头的项，其效果取决于 'magic' 选项。在本章中，我们将假定 'magic' 为真。这也是标准和建议的设定。如果你把它改了，那么，许多查找模式就会突然变得无效了。

备注：
如果你查了好久，超过了你预计的时间，你可以中断查找，在 Unix 上用 CTRL-C，而在 MS-DOS 和微软视窗上则用 CTRL-Break。

27.2 在文件尾折返

在默认情况下，正向查找从当前光标位置开始，查找特定的字符串。然后它就遇到了文件尾。如果那个时候还没找到那个字符串，它就从头来过，从文件开头一直查到光标处。

记住，当你不断以 "n" 命令查找下一个匹配时，你最终回到第一个匹配。如果你不注意，你将永远找下去！为了提醒你，Vim 显示如下信息：

```
search hit BOTTOM, continuing at TOP
```

如果你使用 "?" 命令按相反方向查找，你得到的信息是这样的：

```
search hit TOP, continuing at BOTTOM
```

可是，你还是不知道你何时回到了第一个匹配。一种办法是设定 'ruler' 选项：

```
:set ruler
```

Vim 将把光标位置显示在窗口的右下角（如果使用了状态条的话，会显示在哪里）。看起来像这样：

```
101,29        84%
```

第一个数字是光标所在的行号。在你开始查找的时候记住行号，那样你就能检查是否越过了该位置。

无 折 返 查 找

要取消查找折返功能，请用以下命令：

```
:set nowrapscan
```

现在当查找遇到文件末尾，一个出错信息就会显示出来：

```
E385: search hit BOTTOM without match for: forever
```

这样，你只要用 "gg" 命令回到文件开头，并一直查到你看到以上信息，你就能找到所有的匹配。

如果你用 "?" 从相反方向查找，你将得到：

```
E384: search hit TOP without match for: forever
```

27.3 偏移

在默认情况下，查找命令让光标停留在匹配的模式的开始。你可以指定一个偏移，告诉 Vim 将光标停留在别的位置上。在正向查找命令 "/" 中指定偏移，就是在模式后面附加一个斜线符 (/) 以及偏移值：

```
/默认/2
```

这个命令查找模式 "默认"。找到后使光标越过匹配的模式而前移两行，并停留在该行的行首。把这个命令用于以上段落中，Vim 在第一行找到词 "默认"。接着光标再往下移两行，落在 "一个" 的 "一" 上。

如果该偏移为一简单数字，那么光标就会被放置在距离匹配那么多行的那一行的行首。该偏移值可为正数或负数。如果它是正数，光标会向前移该数表示的行；若为负数，则往回退该数表示的行。

字 符 偏 移

偏移符 "e" 表示一个偏移从匹配末尾算起。它把光标移到匹配的最后一个字符上。命令：

```
/const/e
```

把光标放到单词 "const" 的 "t" 上。

加一个数字，光标就从该位置再前移该数字指定的那么多个字符。下面这个命令会将光标移到匹配后面第一个字符：

```
/const/e+1
```

一个正数使光标右移，负数使其左移。例如：

```
/const/e-1
```

会把光标移到单词 "const" 的 "s" 字符上。

如果偏移以 "b" 开头，那么光标就移到匹配模式的首位。因为不用 "b" 光标也一样会被移到首位，所以单独使用时没什么意义。在将它与一个加上或减去的数字合起来时，就很有用了。光标就会前移或后移那么多个字符。例如：

```
/const/b+2
```

会把光标移到匹配的首位，再往右移两个字符。因而落在字符 "n" 上。

重 复

当你重复前一次使用过的查找模式，只是偏移不同时，你可以把模式省略了：

```
/that
//e
```

等于：

```
/that/e
```

再以同样的偏移重复查找：

```
/
```

命令 "n" 具有同样的作用。要取消一个以前用过的偏移可以用：

```
//
```

反 向 查 找

命令 "?" 以相同的方式使用偏移，但你必须以 "?" 来分隔模式和偏移，而非 "/"：

```
?const?e-2
```

偏移符 "b" 和 "e" 的用途是一样的。它们并不因为使用了 "?" 而改变方向。

起 始 位 置

查找时，通常从光标位置开始。当你规定的是一个行偏移，这可能造成麻烦。例如：

```
/const/-2
```

这个命令找到下一个单词 "const"，然后上移两行。如果你用命令 "n" 再找，Vim 就从当前位置开始，找到同一个 "const" 匹配。然后再一次在偏移的作用下，回到开始的地方。你给套住了！

还有比这更糟糕的：假定下一行另有一个 "const" 匹配。那么，重复正向查找就会找到这个匹配，并上移两行。这样你实际上把光标往回移了！

当你规定的是一个字符偏移，Vim 将为其作调整。因此，查找会向前或向后跳过几个字符再开始，以便同一个匹配不至于再出现。

27.4 匹配重复性模式

星号项 "*" 规定在它前面的项可以重复任意次。因此：

```
/a*
```

匹配 "a"，"aa"，"aaa"，等等。但也匹配 "" (空字串)，因为零次也包含在内。

星号 "*" 仅仅应用于那个紧邻在它前面的项。因此 "ab*" 匹配 "a"、"ab"、"abb"、"abbb" 等等。如要多次重复整个字符串，那么该字符串必须被组成一个项。组成一项的方法就是在它前面加 "\("，后面加 "\)"。因此这个命令：

```
/\(ab\)*
```

匹配："ab"，"abab"，"ababab"，等等。而且也匹配 ""。

要避免匹配空字串，使用 "\+"。这表示前面一项可以被匹配一次或多次。

```
/ab\+
```

匹配 "ab"、"abb"、"abbb" 等等。它不匹配后面没有跟随 "b" 的 "a"。

要匹配一个可选项，用 "\="。 例如：

```
/folders\=
```

匹配 "folder" 和 "folders"。

指 定 重 复 次 数

要匹配某一项的特定次数重复，使用 "\{n,m}" 这样的形式。其中 "n" 和 "m" 都是数字。在它前面的那个项将被重复 "n" 到 "m" 次 (inclusive 包含 "n" 和 "m")。例如 ：

```
/ab\{3,5}
```

匹配 "abbb"、"abbbb" 以及 "abbbbb"。

当 "n" 省略时，被默认为零。当 "m" 省略时，被默认为无限大。当 ",m" 省略时，就表示重复正好 "n" 次。例如：

模式	匹配次数
\{,4}	0，1，2，3 或 4
\{3,}	3，4，5 等等
\{0,1}	0 或 1，同 \=
\{0,}	0 或更多，同 *
\{1,}	1 或更多，同 \+
\{3}	3

匹配尽可能少的字符

迄今为止，我们所讨论过的都试图匹配尽可能多的字符。若要匹配尽可能少的字符，请用 "\{-n,m}"。它的用法跟 "\{n,m}" 一样，唯一的区别在于，它采用尽可能少的字符。

例如，以下命令：

```
/ab\{-1,3}
```

将匹配 "abbb" 中的 "ab"。实际上，因为没理由匹配更多，所以它永远不会匹配超过一个 b。它需要其它的来强制它超过其下限规定次数，而匹配更多的重复。

这些同样的规则也适用于省略 "n" 和 "m" 的情形。甚至可以把两个都省略，只剩 "\{-}"。这个项匹配其前项的重复，重复次数尽可能少，可以等于或大于零。这个项如单独使用，则总是匹配前项的零次重复。当它跟与其它的模式合起来时，用处就大了。例如：

```
/a.\{-}b
```

这个命令匹配 "axbxb" 中的 "axb"。如果采用了下面这个模式：

```
/a.*b
```

由于 ".*" 匹配尽可能多的字符，整个 "axbxb" 都会被匹配。

27.5 多择一

在一个查找模式中，"或" 运算符是 "\|"。例如：

```
/foo\|bar
```

这个命令匹配了 "foo" 或 "bar"。更多的抉择可以连在后面：

```
/one\|two\|three
```

匹配 "one"、"two" 或 "three"。

如要匹配其多次重复，那么整个抉择结构须置于 "\(" 和 "\)" 之间：

```
/\(foo\|bar\)\+
```

这个命令匹配 "foo"、"foobar"、"foofoo"、"barfoobar" 等等。

再举个例子：

```
/end\(if\|while\|for\)
```

这个命令匹配 "endif"、"endwhile" 和 "endfor"。

一个与此相关的项是 "\&"。它要求两个抉择都与同一位置的文本相符。而最终匹配的则是最后面的那个抉择。例如：

```
/forever\&...
```

这个命令匹配 "forever" 中的 "for"（译者：因为第二抉择要求三个字符）。它将不匹配，比如说 "fortuin"（译者：不符合第一抉择）。

27.6 字符范围

你可以用 "/a\ b\ c" 来匹配 "a"，"b" 或 "c"。当你需要匹配自 "a" 至 "z" 所有的字母时，以这样的方式表达就嫌长了。这里有个比较简短的表达方式：

```
/[a-z]
```

方括号结构 [] 匹配单个字符。你在括号内指定哪些字符可以匹配。你可以把一系列字符包含在内，像这样：

```
/[0123456789abcdef]
```

这个命令将匹配其中的任何一个字符。你可以为一系列连续字符规定一个字符范围。"0-3" 表示 "0123"。"w-z" 表示 "wxyz"。因此，上面那个命令可以缩短为：

```
/[0-9a-f]
```

若要匹配字符 "-" 本身，就得把它放在字符范围的第一或最后的位置上。Vim 会识别下面这些特殊字符，以便在 [] 字符范围里较为方便地使用它们（它们实际上可被用于任何查找模式的任何地方）：

```
\e      <Esc>
\t      <Tab>
\r      <CR>
\b      <BS>
```

还有若干特殊场合用得上 [] 字符范围，参阅 /[] 以了解全部用法。

范 围 求 反

为了避免匹配到一个特定的字符，在字符范围首位使用 "^"。这样方括号项 [] 就会匹配任何括号内不包括的字符。例如：

```
/"[^"]*"

 "        双引号
  [^"]    双引号以外的任何字符
      *   尽可能多个
       "  又一个双引号
```

这个命令匹配 "foo" 和 "3!x"，包含双引号在内。

预 定 义 范 围

有些字符范围使用得很频繁。Vim 为这些字符范围提供了一些快捷方式。例如：

```
/\a
```

这个命令找寻字母字符。这相当于使用 "/[a-zA-Z]"。下面还有几个这样的字符范围：

项	匹配	相当于
\d	数位	[0-9]
\D	非数位	[^0-9]
\x	十六进制数位	[0-9a-fA-F]

\X	非十六进制数位	[^0-9a-fA-F]	
\s	空白字符	[ ]	(<Tab> 和 <Space>)
\S	非空白字符	[^]	(非 <Tab> 和 <Space>)
\l	小写字母	[a-z]	
\L	非小写字母	[^a-z]	
\u	大写字母	[A-Z]	
\U	非大写字母	[^A-Z]	

备注：
使用这些预定义的字符范围要比使用它们所表示的那个字符范围快很多。
这些项不能用于 [] 方括号内。因此 "[\d\l]" 不能用来匹配一个数字或一个小写字母。请换用 "\(\d\|\l\)"。

/\s 包括一个这类范围的完整的列表。

27.7 字符类

一个特定的字符范围匹配一组固定的字符。一个字符类与字符范围相似，不过有个本质的区别：一个字符类代表的那组字符可以重新定义而无须改动查找模式。

例如，查找这个模式：

```
/\f\+
```

其中 "\f" 项表示文件名字符。因而这个命令匹配一个由可用作文件名的字符字符组成的序列。

哪些字符可以用来组成文件名取决于你所使用的系统。在微软视窗上，反斜杠可以，而在 Unix 上却不行。文件名字符由 'isfname' 选项来规定。在 Unix 上，该选项的默认值为：

```
:set isfname
isfname=@,48-57,/,.,-,_,+,,,#,$,%,~,=
```

在其它系统上，其默认值各不相同。因此你可以用 "\f" 组成一个查找模式，以匹配一个文件名。该查找模式将自动调整以适应你所使用的系统。

备注：
实际上，Unix 允许在文件名里使用几乎所有的字符，包括空格字符。把这些字符包括在 'isfname' 里，在理论上是没错的。但那样就无法在文本中发现一个文件名在哪儿结束。因此选项 'isfname' 的默认值是个折衷方案。

字符类有如下这些：

项	匹配	选项
\i	标识符字符	'isident'
\I	类似于 \i，但不包括数字字符	
\k	关键词字符	'iskeyword'
\K	类似于 \k，但不包括数字字符	
\p	可显示字符	'isprint'
\P	类似于 \p，但不包括数字字符	
\f	文件名字符	'isfname'
\F	类似于 \f，但不包括数字字符	

27.8 匹配换行符

Vim 能找寻含有换行符的模式。你需要指定换行符在哪儿出现，因为迄今为止所有我们曾经提到过的项，都不匹配换行符。

用 "\n" 项可以在一个特定的位置查验一个换行符：

```
/one\ntwo
```

这个命令将在一行以 "one" 结尾，而下一行以 "two" 开头的地方找到匹配。如果还要匹配 "one two"，那么你需要匹配一个空格或一个换行符。这可以用 "_s" 表示：

```
/one\_stwo
```

若要允许任意数量的空格：

```
/one\_s\+two
```

这个命令也匹配 "one " 在行尾，及 "   two" 在下一行行首的情形。

"\s" 匹配空格字符，"_s" 匹配空格字符或一个换行符。同理，"\a" 匹配一个字母字符，而 "_a" 匹配一个字母字符或一个换行符。其它字符类和字符范围都可以通过插入一个 "_" 来更改其范围。

很多别的项也可以在前面加 "_" 以匹配一个换行符。例如："_." 匹配任意字符或一个换行符。

> 备注：
> "_.*" 匹配任何字符，直至文件结束。要小心，它会使查找命令执行得非常缓慢。

另一个例子是 "_[]"，一个包含了换行符的字符范围：

```
/"\_[^"]*"
```

这个命令找寻位于一对双引号之间，可能分隔成数行的文本。

27.9 举例

这儿有几个查找模式，你也许会觉得有用。本节向你演示如何综合使用前面提及的那些用法。

寻找一个加州驾驶牌照

有一个驾驶牌照号码为 "1MGU103"。它有一个数字，三个大写字母，然后三位数字。直接把这个号码放入查找模式：

```
/\d\u\u\u\d\d\d
```

另一种方法是用一个计数器来指定其中有三个数字和三个字母：

```
/\d\u\{3}\d\{3}
```

换用 [] 字符范围方法：

```
/[0-9][A-Z]\{3}[0-9]\{3}
```

这些方法之中你应该使用哪一种？挑那种你记得住的。你记得住的简单方法要比你记不住的华丽方法快得多。如果你能把它们都记住，那么避免使用最后那种，因为它要打的字多，而且执行起来慢。

寻找一个标识符

在 C 程序里（以及其它很多计算机程序）一个标识符以字母开头，其余部分由字母和数字组成。下划线字符也可以。这样一个标识符可以用下面的命令找到：

```
/\<\h\w*\>
```

"\<" 和 "\>" 用来寻找那些完整的词。"\h" 表示 "[A-Za-z_]"，而 "\w" 则表示 "[0-9A-Za-z_]"。

备注：
"\<" 和 "\>" 取决于 'iskeyword' 选项。如果这个选项包括 "-" 的话，那么 "ident-" 就不匹配了。在这种情况下，请用：

```
/\w\@<!\h\w*\w\@!
```

这个命令查验是否 "\w" 不匹配标识符之前或之后的字符。
参见 /\@<! 和 /\@! 。

下一章： usr_28.txt 折叠

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: Chimin Yen http://vimcdoc.sf.net

折叠

结构化的文本可以分为许多节。而一节之内可以再分小节。折叠允许你将一节显示为一行，并提供文本的概览。本章将解释各种实现折叠的方法。

28.1 什么是折叠?
28.2 手动折叠
28.3 对折叠的操作
28.4 存储和恢复折叠
28.5 依缩进折叠
28.6 依标志折叠
28.7 依语法折叠
28.8 依表达式折叠
28.9 折叠未被改动的行
28.10 使用哪种折叠办法呢?

下一章: usr_29.txt 在代码间移动
前一章: usr_27.txt 查找命令及模式
目录: usr_toc.txt

28.1 什么是折叠?

折叠用于把缓冲区内某一范围内的文本行显示为屏幕上的一行。就像一张纸，要它缩短些，可以把它折叠起来:

```
+-------------------------+
| 行 1            |
| 行 2            |
| 行 3            |
|------------------------ |
\                          \
 \-------------------------\
 / 被折叠的行               /
/_________________________/
| 行 12           |
| 行 13           |
| 行 14           |
+-------------------------+
```

那些文本仍然在缓冲区内而没有改变。受到折叠影响的只是文本行显示的方式。

折叠的好处是，通过把多行的一节折叠成带有折叠提示的一行，会使你更好地了解对文本的宏观结构。

28.2 手动折叠

试一试：把光标置于某一段落内，并键入：

```
zfap
```

你将会看到该段落被一行高亮的文本所代替。你已经创建了一个折叠了。 zf 是个操作符，而 ap 是一个文本对象。你可以将 zf 操作符跟任何一个移动命令联用，为所经之处的文本创建一个折叠。 zf 也能在可视模式下使用。

若要再阅读那些文本，可以键入以下命令以打开该折叠：

```
zo
```

你还可以用以下命令再关闭该折叠：

```
zc
```

所有的折叠命令都以 'z' 开头。展开你的想像力，这个字母看起来就像一张折叠起来的纸的侧面。而 "z" 后面可用的字母，由于采用了帮助记忆方法选择，很容易记得住：

```
zf      F-old creation (创建折叠)
zo      O-pen a fold (打开折叠)
zc      C-lose a fold (关闭折叠)
```

折叠可以嵌套：一个含有折叠的文本区可以被再次折叠。例如，你可以折叠本节内每一段落，然后折叠本章内所有的节。试试看。你将注意到，打开全章的折叠，会将节的折叠还原得跟以前一样，有些打开，而有些关闭。

假定你已经创建了若干折叠，而现在需要阅览全部文本。你可以移到每个折叠处，并键入 "zo"。若要做得更快，可以用这个命令：

```
zr
```

这将减少 (R-educe) 折叠。相反的操作是：

```
zm
```

这将折叠更多 (M-ore)。你可以重复 "zr" 和 "zm" 来打开和关闭若干层嵌套的折叠。

如果你有一个嵌套了好几层深的折叠，你可以用这个命令把它们全部打开：

```
zR
```

这将减少折叠直至一个也不剩。而用下面这个命令你可以关闭所有的折叠：

```
zM
```

这将增加折叠，直至所有的折叠都关闭了。

你可以用 zn 命令快速禁止折叠功能。然后 zN 恢复原来的折叠。 zi 切换于两者之间。以下步骤是一种实用的操作方法：
- 创建折叠，以获取你的文件的概览
- 移动到你要操作的地方
- 执行 zi 以便一边看着文本，一边编辑

- 再执行 zi 以便移动到另一处

在参考手册中有更多关于手动折叠的内容： fold-manual

28.3 对折叠的操作

当一些折叠关闭时，移动命令，如 "j" 和 "k" 会轻松的移过折叠，就像它是单个空行一样。这允许你快速移过折叠了的文本。

你可以向对待单行一样复制，删除和粘贴折叠。当你要改动某个程序里的函数的先后次序时，这是很实用的。首先，为 'foldmethod' 选择一个正确的折叠方法，以保证每个折叠包含了整个函数（或稍缺一点儿）。然后，用 "dd" 命令删除该函数，移动光标，并用 "p" 命令粘贴。如果函数中某些行在折叠之上，或之下，你可以利用可视模式下的选择方法：
- 把光标置于被移文本的首行
- 击 "V" 键开始可视模式
- 把光标置于被移文本的末行
- 击 "d" 键删除被选中的行。
- 把光标移到新位置，并击 "p" 键把文本粘贴在那儿。

有时候，查看或记住一个折叠在哪儿，挺不容易的。更别说用 zo 命令来打开了。要查看那些已定义的折叠：

```
:set foldcolumn=4
```

这个命令将在窗口左边显示一小栏来标识各个折叠。一个 "+" 表示某个关闭的折叠。一个 "-" 表示每个打开的折叠的开头，而 "|" 则表示该折叠内其余的行。

你可以在折叠栏内用鼠标点击 "+"，以打开一个折叠。点击 "-" ，或在它之下的某个 "|" ，将关闭一个打开的折叠。

打开所有光标行上的折叠用 zO 。
关闭所有光标行上的折叠用 zC 。
删除一个光标行上的折叠用 zd 。
删除所有光标行上的折叠用 zD 。

当你进入插入模式后，光标行上的折叠永远不会关闭。那是要让你看见你打的什么字！

当光标前后跳转至折叠，或在折叠上左右移动时，折叠就会自动打开。例如，零命令 "0" 打开光标下的折叠（假设 'foldopen' 包含 "hor"，即默认设置）。'foldopen' 选项可以修改，为指定的某一类命令打开折叠。如果你要光标遇到折叠，折叠就打开，那么可以这么做：

```
:set foldopen=all
```

警告：你将因此无法移到一个关闭的折叠上。你也许只想临时用一用这个命令，然后把它设回默认值：

```
:set foldopen&
```

你可以在移开折叠时自动关闭折叠：

```
:set foldclose=all
```

这个命令将重新把折叠级别 'foldlevel' 作用到所有的不含光标的折叠。你必须自己试试看你会不会喜欢这个设置。用 zm 增加折叠级别，并用 zr 减少折叠级别（减少折叠的层次）。

折叠是限于本地窗口的。这允许你为同一缓冲区打开两个窗口，一个带折叠，而另一个不带折叠。或者，一个让所有的折叠关闭，而另一个则让所有的折叠打开。

28.4 存储和恢复折叠

当你放弃一个文件时（开始编辑另一个），其折叠状态就丢失了。如果你稍后再回来编辑同一文件，那么，所有手动打开和关闭的折叠，全都恢复到它们的默认状态了。如果折叠是用手动方式创建的，则所有的折叠都消失了！为了保存折叠，可以用 :mkview 命令：

```
:mkview
```

这将储存那些影响文件视图的设定及其它内容。你可以利用 'viewoptions' 选项修改储存的范围。当你稍后回到同一文件时，你可以重新载入这个视图：

```
:loadview
```

你可以为一个文件储存多至十个视图。例如，把当前设置储存为第三个视图，并载入第二个视图：

```
:mkview 3
:loadview 2
```

注意 当你插入或删除一些文本行时，视图可能变得无效。还得检查 'viewdir' 选项，它指定视图文件储存在哪儿。你可能时不时需要删除旧的视图文件。

28.5 依缩进折叠

使用 zf 来定义一个折叠很费事。如果你的文本依循一种结构，以较多的缩进表示较低的层次，那么，你可以采用缩进折叠的方法。这将为每一系列有相同缩进的行创建一个折叠。缩进较多的行将成为嵌套的折叠。缩进折叠适用于许多编程语言。

我们来试试这个方法。先设定 'foldmethod' 选项：

```
:set foldmethod=indent
```

然后你可以用 zm 和 zr 命令增加和减少折叠。在下面这个例文上很容易看明白：

```
本行没有缩进
        本行被缩进一次
                本行被缩进两次
                本行被缩进两次
        本行被缩进一次
本行没有缩进
        本行被缩进一次
        本行被缩进一次
```

注意 缩进多少和折叠深度之间的关系倚赖于 'shiftwidth' 选项。每个 'shiftwidth' 选项规定的缩进宽度，在折叠深度上加一。这被称为一个折叠级别。

当你使用 zr 和 zm 命令时，你实际上是在增加或减少 'foldlevel' 选项。你也可以直接设置它：

```
:set foldlevel=3
```

这意味着，所有缩进等于或大于 'shiftwidth' 三倍的折叠将被关闭。折叠级别设定得越低，越多的折叠将被关闭。当 'foldlevel' 为零时，所有的折叠都将被关闭。 zM 把 'foldlevel' 设为零。相反的命令 zR 把 'foldlevel' 设为文件中最深的折叠级别。

因此，有两种方法开启和关闭折叠：

(A) 设定折叠级别。
    这提供了一种极快的 "缩小" 方法来查看文本结构，移动光标，以及重新 "放大" 到具体的文本。

(B) 利用 zo 和 zc 命令打开和关闭指定的折叠。
    这个方法允许你仅仅打开那些你要打开的折叠，而不影响其它的折叠。

这两种方法可以结合起来用：你可以先用几次 zm 以关闭大多数折叠，然后用 zo 打开一个指定的折叠。或者，用 zR 打开所有的折叠，然后用 zc 关闭指定的折叠。

但是，当折叠方法 'foldmethod' 的值为 "indent" 时，你不能手动定义折叠。因为那样会引起缩进宽度和折叠级别之间的冲突。

在参考手册中有更多关于缩进折叠的内容： fold-indent

28.6 依标志折叠

文本中的标志用于指定一个折叠区的起点和终点。标志折叠可以精确地控制一个折叠究竟包含哪些行文本。缺点是文本需要改动。

试试这个：

```
:set foldmethod=marker
```

以下列 C 程序片段为例：

```
/* foobar () {{{ */
int foobar()
{
        /* return a value {{{ */
        return 42;
        /* }}} */
}
/* }}} */
```

请 注意，折叠行将显示位于标志之前的文字。这正好用来说明该折叠包含了什么。

令人十分困扰的是，当某些文本行移动后，标志不再正确地配对。这种局面可以利用编号标志来避免。例如：

```
/* global variables {{{1 */
int varA, varB;

/* functions {{{1 */
/* funcA() {{{2 */
void funcA() {}

/* funcB() {{{2 */
void funcB() {}
/* }}}1 */
```

每一个编号标志表示一个编号指定级别的折叠的开始。这将使任何较高层次的折叠在此结束。你可以只用编号标志的开始符定义所有的折叠。只有当你要明确地在另一个开始前结束一个折叠时，你才需要加一个标志停止符。

在参考手册中有更多关于标志折叠的内容： fold-marker

28.7 依语法折叠

Vim 为每一种不同的语言使用一个不同的语法文件。语法文件为文件中各种不同语法项定义颜色。如果你正用 Vim 在一个支持色彩的终端上阅读本文，你所看见的色彩就是由语法文件 "help" 定制的。

在语法文件中，你可以加入一些带有 "fold" 参数的语法项。这些语法项将定义折叠区。这要求写一个语法文件，把这些项目加入其中。编写这样一个文件是不容易的。但是一旦写成，所有折叠的创建就变成自动的了。

在此，我们将假定你正使用一个已经写好的语法文件。这样的话，就没更多解释的必要了。你可以像以上解释过的那样打开和关闭折叠。编辑文件时折叠会自动创建和删除。

在参考手册中有更多关于语法折叠的内容： fold-syntax

28.8 依表达式折叠

表达式折叠类似于缩进折叠，但并非利用文本行的缩进，而是调用一个函数来计算一行的折叠级别。当文本的一部分表明那些行属于同一组时，你可以使用这个方法。一个例子是电子邮件，其中引述的文本由行首的 ">" 来表示。要折叠这些引文，可以用以下命令：

```
:set foldmethod=expr
:set foldexpr=strlen(substitute(substitute(getline(v:lnum),'\\s','','\"g\"),'[^>].*','',''
```

你可以在这段文本上试式看：

```
> quoted text he wrote
> quoted text he wrote
> > double quoted text I wrote
> > double quoted text I wrote
```

以下是上例中 'foldexpr' 的解释（自里至外）：

getline(v:lnum)	读取当前行
substitute(...,'\\s','','g')	从当前行删除所有空白字符
substitute(...,'[^>].*','','')	删除行首那些 '>' 之后的任何字符

`strlen(...)` 计算字符串的长度，即 '>' 的个数

注意 在 ":set" 命令中，每一个空格，双引号和反斜线符之前，必须插入一个反斜杠。如果这会把你搞糊涂，那么就执行

```
:set foldexpr
```

来检查所产生的实际值。为了修正一个复杂的表达式，请使用命令行补全：

```
:set foldexpr=<Tab>
```

其中 <Tab> 是一个真实的 Tab 键。Vim 将填入以前的值，然后你可以编辑它。

当该表达式变得相对复杂时，你应当将其放入一个函数。然后设定 'foldexpr' 来调用该函数。

在参考手册中有更多关于表达式折叠的内容： fold-expr

28.9 折叠未被改动的行

当你在同一窗口也设定 'diff' 选项时，这种折叠方法就很有用。 vimdiff 命令为你设定好了使用未改行折叠。例如：

```
:setlocal diff foldmethod=diff scrollbind nowrap foldlevel=1
```

在显示同一文件的不同版本的每个窗口内执行这个命令。你将清楚地看到不同文件间的区别，因为那些没改动的文本都被折叠起来了。

更多细节参见 fold-diff 。

28.10 使用哪种折叠办法呢?

所有这些可能性让你感到纳闷，你究竟应该选择哪种方法。不幸的是，没有放之四海皆准的法则。这里只给出一些提示。

如果存在一个语法文件，其中定义的折叠符合你正在使用的程序语言，那么，语法折叠应该是最好的选择。否则，你也许得试着写一个。这要求你相当的了解关于查找模式知识。这并非易事。但一旦写成，你将不必以手动的方式定义折叠了。

键入命令，手动折叠一个个文本区的方法可用于无结构特点的文本。然后用 :mkview 命令来储存和还原折叠状态。

标志折叠法要求你修改文件。如果你与它人共享这个文件，或不得不遵守公司规定的标准，那么你也许得不到许可给文件加标志。

标志折叠的主要优点是，你可以精确的把标志放在你要的位置。那样就避免了那种在你剪切和粘贴折叠时漏了几行文本的情况。并且，你还可以加个注释，说明该折叠包含些什么。

缩进折叠法是那种在许多文件里都用的着。但并不是每次都能成功的方法。当你无法采用其它方法时，就用这种。然而，缩进折叠在做提纲时特别有用。你必须为每一层嵌套折叠特意的使用同一缩进宽度 'shiftwidth'。

表达式折叠法能够在几乎任何有结构特定的文本中创建折叠。这种方法相当简单，尤其当折叠的开始和结束处能容易地被识别的时候。

如果你用 "expr" 方法来定义折叠而无法得到完全满意的结果。那么你可以切换到手动方法 "manual"。这么做不会删除那些已经定义好了的折叠。之后你便可以手动删除或增加折叠了。

下一章： usr_29.txt 在代码间移动

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：wandys http://vimcdoc.sf.net

在代码间移动

Vim 的创造者是一名计算机程序员，因此这就不奇怪 Vim 中有许多帮助编写程序的功能：跳转到标识符被定义和使用的地方；在另一个窗口中预览有关的声明等等。在下一章中还会介绍更多的功能。

29.1 使用标签
29.2 预览窗口
29.3 在代码间移动
29.4 查找全局标识符
29.5 查找局部标识符

下一章： usr_30.txt 编辑程序
前一章： usr_28.txt 折叠
目录： usr_toc.txt

29.1 使用标签

什么是标签？标签就是一个标识符被定义的地方。一个例子就是 C 或者 C++ 程序中的函数定义。标签列表可以保存在一个标签文件中。Vim 可以通过它来从任何地方跳转到该标签，也就是一个标识符被定义的地方。

在当前目录下为所有的 C 文件生成标签文件，使用下面的这个命令：

```
ctags *.c
```

"ctags" 是一个独立的程序。大多数 Unix 系统上都已经安装了它。如果你还没有安装，可以在这里找到 "Exuberant ctags"：

```
http://ctags.sf.net
```

现在你可以使用下面的命令跳转到一个函数定义的地方：

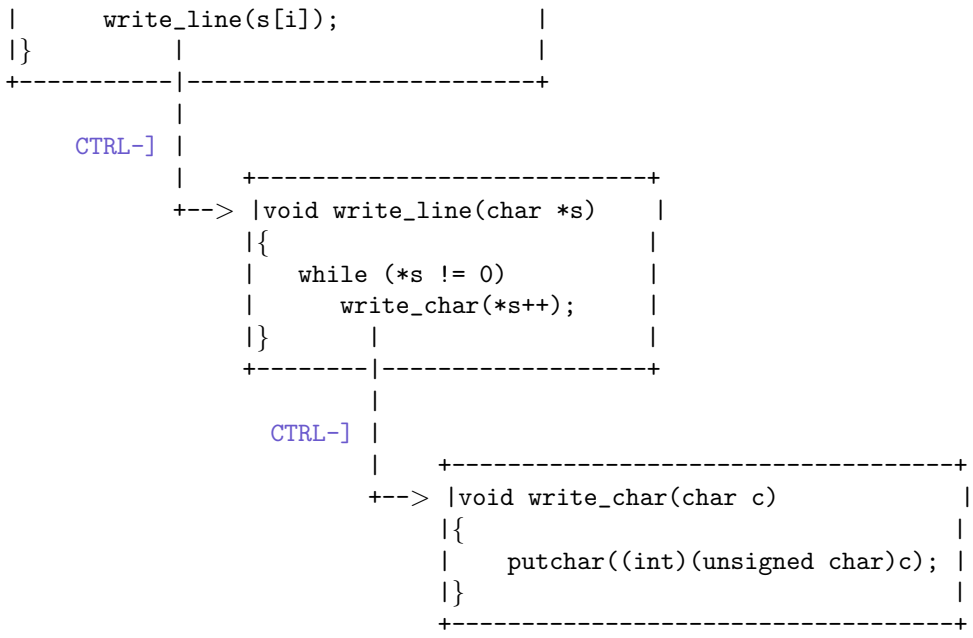
```
:tag startlist
```

这个命令会找到函数 "startlist"，即使该函数是在另一个文件中。

CTRL-] 命令会跳转到当前光标下单词的标签。这样浏览毫无头绪的 C 代码会变得更容易些。举个例子，假设你在函数 "write_block" 中。你可以看到它调用了函数 "write_line"。但 "write_line" 做了什么呢？将光标置于调用 "write_line" 的地方然后按 CTRL-]，你就跳转到了这个函数的定义的地方了。

"write_line" 函数调用了 "write_char"。你需要知道它做了什么。将光标定位到调用 "write_char" 的地方然后按 CTRL-]，你就到了定义 "write_char" 的地方。

```
+-----------------------------------+
|void write_block(char **s; int cnt)  |
|{                                    |
|   int i;                            |
|   for (i = 0; i < cnt; ++i)          |
```



":tags" 命令显示你经过的标签列表：

```
:tags
  # TO tag         FROM line  in file/text
  1  1 write_line          8  write_block.c
  2  1 write_char          7  write_line.c
>
```

现在介绍向回跳转。CTRL-T 命令跳转到上一个标签。在上例中，你会回到 "write_line" 函数调用 "write_char" 的地方。

这个命令接受一个计数参数，用来表示跳转回去的标签个数。你已经向前跳转，现在又跳转了回去。现在我们再一次向前跳转。下面的命令跳转到标签列表中最上面的标签：

```
:tag
```

你可以在前面加上要向前跳转的标签个数。比如："":3tag"。CTRL-T 同样可以加上一个计数参数。

通过这些命令，你可以用 CTRL-] 沿着调用树向前跳转，用 CTRL-T 向回跳转，用 ":tags" 命令显示当前位置。

分 割 窗 口

":tag" 命令会将当前窗口的文件替换为包含新函数的文件。怎样才能同时查看两个文件呢？你可以使用 ":split" 命令将窗口分开然后再用 ":tag" 命令。Vim 有个缩写命令可以做到这些：

```
:stag tagname
```

使用下面的命令可以分割当前窗口并跳转到光标下的标签：

```
CTRL-W ]
```

如果指定了计数参数，新窗口将包含指定的那么多行。

多 个 标 签 文 件

如果在多个目录中都有文件，你可以在每一个目录下创建一个标签文件。Vim 只能跳转到那个目录下的标签。

通过设定 'tags' 选项，你可以使用多个相关的标签文件。比如：

```
:set tags=./tags,./../tags,./*/tags
```

这会使 Vim 找到当前文件所在目录及其父目录和所有子目录下的标签文件。

这已经是不少的标签文件了，但也许仍不够。比如，当编辑 "~/proj/src" 目录下的一个文件时，你无法找到 "~/proj/sub/tags" 目录下的标签文件。对这种情况，Vim 提供了一个查找整个目录树下标签文件的方法，比如：

```
:set tags=~/proj/**/tags
```

单 个 标 签 文 件

当 Vim 在多个地方查找标签文件时，你会听到硬盘在格格作响。这样会有点慢。在这种情况下，你最好将这些时间花在生成一个大的标签文件上。你可以要等一会儿。

这得借助上面提到的 "Exuberant ctags" 程序。它有一个选项可以搜索整个目录树：

```
cd ~/proj
ctags -R .
```

这样做的好处是 "Exuberant ctags" 可以识别多种文件类型，它不仅适用于 C 和 C++ 程序，还适用于 Eiffel 甚至 Vim 脚本。请参考 ctags 文档进行调整所用参数。

现在你只需要告诉 Vim 你的标签文件在何处：

```
:set tags=~/proj/tags
```

多 个 匹 配

当一个函数（或类中的方法）被定义多次，":tags" 命令会跳转到第一处。如果在当前文件中存在匹配，那它将会被首先使用。

你现在可以跳转到同一个标签的其它匹配处：

```
:tnext
```

重复执行这个命令可以找到更多的匹配。如果存在很多匹配，你可以选择要跳转到哪一个：

```
:tselect tagname
```

Vim 会为你展示一个选择列表：

```
  # pri kind tag              file
  1 F   f    mch_init         os_amiga.c
               mch_init()
```

```
  2 F    f    mch_init          os_mac.c
                mch_init()
  3 F    f    mch_init          os_msdos.c
                mch_init(void)
  4 F    f    mch_init          os_riscos.c
                mch_init()
Enter nr of choice (<CR> to abort):
```

你现在可以输入要跳转到的匹配代号（在第一列）。其它列的信息可以让你知道匹配在何处被定义。

可以用这些命令在各匹配的标签间移动：

:tfirst	到第一个匹配
:[count]tprevious	向前 [count] 个匹配
:[count]tnext	向后 [count] 个匹配
:tlast	到最后一个匹配

如果没有指定，[count] 缺省为一。

猜 测 标 签 名

命令行补全是避免输入长标签名的好办法。只需输入开始的一部分然后按 <Tab>：

```
:tag write_<Tab>
```

你会得到第一个匹配。如果这不是你想要的，重复输入 <Tab> 直到你找到正确的匹配。

有时你只知道一个函数名的一部分，或是你有很多以相同字符串开头而结尾不同的标记。这时你可以告诉 Vim 使用一个模式来查找标签。

假设你要跳转到一个包含 "block" 的标签。首先输入：

```
:tag /block
```

现在再利用命令行补全功能：输入 <Tab>。Vim 会找到所有包含 "block" 的标签并使用第一个匹配。

标签名前面的 "/" 告诉 Vim 这不是一个确定的标签名而是一个模式。你可以利用有关查找模式的所有特性。举个列子，假设你要选择所有以 "write_" 开头的标签：

```
:tselect /^write_
```

"^" 指定标签以 "write_" 开头，否则在中间含有 "write_" 的标签名也会被找到。类似地，"$" 指定标签名结尾处的匹配。

标 签 浏 览 器

CTRL-] 可以让你跳转到光标所在标识符的定义处，因此你可以利用标识符的列表来形成一个目录。这里给出一个例子。

首先生成一个标识符列表（需要 Exuberant ctags）：

```
ctags --c-types=f -f functions *.c
```

现在打开 Vim 并在一个垂直分割窗口中编辑这个文件：

```
vim
:vsplit functions
```

窗口中包含一个所有函数的列表。其它的东西可以被忽略。用 ":setlocal ts=99" 命令使其显示得更清晰些。

在这个窗口中，定义一个映射：

```
:nnoremap <buffer> <CR> 0ye<C-W>w:tag <C-R>"<CR>
```

移动光标至要跳转到函数的所在行，输入 <Enter>。Vim 会在另一个窗口中跳转到所选择的函数定义处。

相 关 杂 项

你可以在不改变 'tagcase' 为 "followic" 的情况下置位 'ignorecase' 选项，或者设置 'tagcase' 为 "ignore" 来忽略标签名里的大小写。

'tagbsearch' 选项标明标签文件是否经过排序。缺省是假定为标签文件已排序，这样会使查找更快，但如果文件没有被排序是无法工作的。

'taglength' 选项可用来告诉 Vim 标签的有效字符个数。

Cscope 是一个自由软件。它不仅可以找到一个标识符被声明的地方，还可以找到标识符被使用的地方。请参考 cscope 。

29.2 预览窗口

当编辑含有函数调用的代码时，你需要使用正确的调用参数。要获知所要传递的值，你可以查看这个函数是如何定义的。标签机制对此十分适用。如果定义可在另一个窗口内显示那就更好了。对此我们可以利用预览窗口。

打开一个预览窗口来显示函数 "write_char"：

```
:ptag write_char
```

Vim 会打开一个窗口，跳转到 "write_char" 标签。然后它会回到原来的位置。这样你可以继续输入而不必使用 CTRL-W 命令。

如果函数名出现在文本中，你可以用下面的命令在预览窗口中得到其定义：

```
CTRL-W }
```

有一个脚本可以自动显示光标处的标签定义。请参考 CursorHold-example 。

用下面的命令关闭预览窗口：

```
:pclose
```

要在预览窗口中编辑一个指定的文件，用 ":pedit"。这在编辑头文件时很有用，比如：

```
:pedit defs.h
```

最后，"psearch" 可用来查找当前文件和任何包含文件中的单词并在预览窗口中显示匹配。这在使用没有标签文件的库函数时十分有用。例如：

```
:psearch popen
```

这会在预览窗口中显示含有 popen() 原型的 "stdio.h" 文件：

```
FILE    *popen __P((const char *, const char *));
```

你可以用 'previewheight' 选项指定预览窗口打开时的高度。

29.3 在代码间移动

因为程序代码是结构化的，Vim 可以识别其中的有关项目。一些特定的命令可用来完成相关的移动。

C 程序中经常包含类似下面的代码：

```
#ifdef USE_POPEN
    fd = popen("ls", "r")
#else
    fd = fopen("tmp", "w")
#endif
```

有时会更长，也许还有嵌套。将光标置于 "#ifdef" 处按 %。Vim 会跳转到 "#else"。继续按 % 会跳转到 "#endif"。再次按下 % 又回到原来的 "#ifdef"。

当代码嵌套时，Vim 会找到相匹配的项目。这是检查你是否忘记了一个 "#endif" 的好办法。

当你在一个 "#ifdef" - "#endif" 块内的某个位置，你可以用下面的命令回到开始处：

```
[#
```

如果你的位置不是在 "#if" 或 "#ifdef" 之后，Vim 会鸣音。用下面命令可以跳转到下一个 "#else" 或 "#endif"：

```
]#
```

这两个命令会跳过它所经过的 "#if" - "#endif" 块。
例如：

```
#if defined(HAS_INC_H)
    a = a + inc();
# ifdef USE_THEME
    a += 3;
# endif
    set_width(a);
```

如果光标在最后一行，"[#" 会移动到第一行。中间的 "#ifdef" - "#endif" 块被跳过。

在 代 码 块 内 移 动

C 代码块包含在 {} 中，有时一个代码会很长。要跳转到外部代码块的开始处，用 "[[" 命令。用 "][" 找到结尾处。(前提是 "{" 和 "}" 都在第一列。)

"[{" 命令跳转到当前代码块的开始处。它会跳过同一级别的 {} 对。"]}" 跳转到结尾。

一点概述:

```
                    function(int a)
          +->       {
          |             if (a)
          |       +->    {
      [[  |       |         for (;;)              --+
          |       |     +-> {                       |
          |    [{ |     |      foo(32);             |     --+
          |       |   [{ |      if (bar(a))  --+    | ]}    |
          +--     |     +--        break;     | ]} |       |
                  |           }               <-+    |       | ][
                  +--         foobar(a)              |       |
                        }                      <-+   |       |
                    }                                 <-+
```

当编写 C++ 或 Java 代码时，外部代码块是类，而下一级的 {} 是方法。在类内部用 "[m" 可以找到前一个方法的开始。"]m" 会找到下一个方法的开始。

另外，"[]" 反向移动到前一个函数的结尾，"]]" 正向移动到下一个函数的开始。函数的结尾指的是处在第一列的 "}"。

```
                          int func1(void)
                          {
                                  return 1;
              +---------->  }
              |
        []    |           int func2(void)
              |       +->  {
              |    [[ |           if (flag)
start         +--     +--                 return flag;
              |    ][ |           return 2;
              |       +->  }
        ]]    |
              |           int func3(void)
              +---------->  {
                                  return 3;
                          }
```

不要忘了你还可以用 "%" 在匹配的 ()、{} 和 [] 间移动。这在它们相距很多行时仍然适用。

在括号内移动

"[(" 和 "])" 命令与 "[{" 和 "]}" 类似，只不过它们适用于 () 对而不是 {} 对。

```
                          [(
                <---------------------------------
                        <-------
```

```
if (a == b && (c == d || (e > f)) && x > y)
                    ------------->
               ------------------------------->
                                ])
```

在 注 释 间 移 动

移动到一个注释的开始用 "[/"；向前移动到注释的结尾用 "]/"。这只对 /* - */ 注释有效。

```
  +->     +-> /*
  |    [/ |    * A comment about      --+
[/ |       +--  * wonderful life.        | ]/
  |            */                     <-+
  |
  +--          foo = bar * 3;         --+
                                        | ]/
               /* a short comment */  <-+
```

29.4 查找全局标识符

你在编辑一个 C 程序，想要知道一个变量是被声明为 "int" 还是 "unsigned"。一个快速的方法是使用 "[I" 命令来查找。

假设光标在单词 "column" 处。输入：

```
[I
```

Vim 会列出它所找出的匹配行，不仅在当前文件内查找，还会在所有的包含文件中查找。结果如下所示：

```
structs.h
 1:   29     unsigned     column;     /* column number */
```

相对使用标签文件或预览窗口的好处是包含文件也被搜索。大多数情况下都能找到正确的声明。即使标签文件已经过期或者你没有为包含文件建立标签也不会影响结果。

但是一些准备工作是必要的，否则 "[I" 就没法工作。首先，'include' 选项必须指定文件是如何被包含的。缺省值适用于 C 和 C++。对其它的语言，你需要自己设定。

定 位 包 含 文 件

Vim 会找到 'path' 选项指定路径中的包含文件。如果缺少某个目录，一些包含文件将不会被找到。你可以用这个命令来查看：

```
:checkpath
```

它会列出不能找到的包含文件，以及被找到的包含文件。一个输出样例：

```
--- Included files not found in path ---
<io.h>
vim.h -->
  <functions.h>
  <clib/exec_protos.h>
```

文件 "io.h" 被当前文件包含但无法找到。"vim.h" 可以找到，这样 ":checkpath" 跟进这个文件并检查其中的包含文件。结果显示无法找到 "vim.h" 包含的 "functions.h" 和 "clib/exec_protos.h" 文件。

> 备注：
> Vim 不是一个编译器。它无法识别 "#ifdef" 语句。这就是说所有的 "#include" 语句都会被使用，即使它在 "#if NEVER" 之后。

给 'path' 选项增加一个目录可以修正无法找到文件的错误。一个好的参考是 Makefile。注意那些包括 "-I" 的条目，比如 "-I/usr/local/X11"。要增加这个目录，用：

```
:set path+=/usr/local/X11
```

如果有很多的子目录，你可以用 "*" 通配符。例如：

```
:set path+=/usr/*/include
```

这会找到 "/usr/local/include" 以及 "/usr/X11/include" 目录下的文件。

如果你的工程项目的包含文件都在一个嵌套的目录树下，"**" 就非常有用。它会搜索所有的子目录。例如：

```
:set path+=/projects/invent/**/include
```

这会找到这些目录下的文件：

```
/projects/invent/include
/projects/invent/main/include
/projects/invent/main/os/include
等等
```

还有其它的可能性。更多信息，请查看 'path' 选项。
如果你想查看找到的包含文件，用这个命令：

```
:checkpath!
```

你会得到一个（很长）的包含文件列表。为使它更短些，Vim 会对已经找到的文件显示 "(Already listed)" 而不再重新显示一遍。

跳转到匹配

"[I" 产生一个每项只有一行文本的列表。如果你想要进一步地查看第一项，你可以用这个命令来跳转：

```
[<Tab>
```

你也可以使用 "[CTRL-I"，因为 CTRL-I 和按 <Tab> 效果一样。

"[I" 产生的列表在每行的开头都有一个序号。如果你要跳转到第一项外的其它项，首先输入序号：

```
3[<Tab>
```

会跳转到列表中的第三项。记住你可以用 CTRL-O 跳回到原来的地方。

相 关 命 令

```
[i              只列出第一项匹配
]I              只列出光标下面的项目
]i              只列出光标下面的第一项匹配
```

查 找 宏 定 义 标 识 符

"[I" 命令查找任何标识符。只查找 "#define" 定义的宏，用:

```
[D
```

同样，这会在所有的包含文件中查找。'define' 选项指定 "[D" 所查找的项目定义行的模式。你需要改变它的值来适用于 C 或 C++ 以外的语言。

"[D" 相关命令:

```
[d              只列出第一项匹配
]D              只列出光标下面的项目
]d              只列出光标下面的第一项匹配
```

29.5 查找局部标识符

"[I" 命令查找所有的包含文件。要在当前文件中查找并跳转到光标处单词被首次使用的地方，用:

```
gD
```

提示: Goto Definition。这个命令对查找局部 (C 语言中的 "static") 声明的变量或函数很有用。例如 (光标在 "counter" 处):

```
   +->   static int counter = 0;
   |
   |     int get_counter(void)
gD |     {
   |         ++counter;
   +--       return counter;
         }
```

要进一步的缩小查找范围，只在当前函数内查找，用这个命令:

```
gd
```

这会回到当前函数的开始处寻找光标处单词首次出现的地方。实际上，它是向后找到一个在第一列为 '{' 的上方的空行，然后再从那里开始正向查找标识符。例如 (光标位于 idx 上):

```
int find_entry(char *name)
```

```
        {
   +->      int idx;
   |
gd |        for (idx = 0; idx < table_len; ++idx)
   |            if (strcmp(table[idx].name, name) == 0)
   +--              return idx;
        }
```

下一章: usr_30.txt 编辑程序

版权: 参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：wandys http://vimcdoc.sf.net

编辑程序

Vim 有很多帮助编写程序代码的命令。例如：直接在 Vim 内编译程序并跳转到出错位置；根据语言种类自动设定缩进，还有对注释进行排版。

30.1 编译
30.2 C 文件缩进
30.3 自动缩进
30.4 其它缩进
30.5 制表符和空格
30.6 排版注释格式

下一章： usr_31.txt 使用 GUI
前一章： usr_29.txt 在代码间移动
目录： usr_toc.txt

30.1 编译

Vim 有个 "quickfix" 命令集。通过这些命令，你可在 Vim 内编译程序并能直接跳转到出错位置进行修正。你可以接着重新编译并做修正，直到不再出错为止。

下面的命令运行 "make"（包括你所给出的参数）程序并捕捉其运行结果：

```
:make {arguments}
```

如果有错误产生那么它们将被捕获到。而且光标会停留在出现第一个错误的地方。
让我们看一个 "make" 的例子。当输入 ":make" 后，屏幕出现如下的结果：

```
:!make | &tee /tmp/vim215953.err
gcc -g -Wall -o prog main.c sub.c
main.c: In function 'main':
main.c:6: too many arguments to function 'do_sub'
main.c: At top level:
main.c:10: parse error before '}'
make: *** [prog] Error 1

2 returned
"main.c" 11L, 111C
(3 of 6): too many arguments to function 'do_sub'
Press ENTER or type command to continue
```

你会看到编译 "main.c" 文件（见下）出现的错误。按一下 <Enter>，Vim 会显示文件 "main.c" 并将光标移到第一个出现错误行。你无需指定文件名或具体的行号，Vim 会根据出错信息定位相关位置。

```
                +-------------------------------------------------+
                |int main()                                       |
                |{                                                |
                |        int i=3;                                 |
光标 -> |        do_sub("foo");                           |
                |        ++i;                                     |
                |        return (0);                              |
                |}                                                |
                |}                                                |
                | ~                                               |
                |(3 of 12): too many arguments to function 'do_sub' |
                +-------------------------------------------------+
```

这个命令将光标移到下一个错误所在的行：

```
:cnext
```

光标跳转到文件的最后一行（第十行），那里有一个多余的 '}'。

有时由于空间不够，Vim 会缩短出错信息。如果你想查看详细信息，可执行：

```
:cc
```

你可以通过 ":clist" 命令得到一个有关所有出错信息的概览。输出的内容如下所示：

```
:clist
3 main.c: 6:too many arguments to function 'do_sub'
5 main.c: 10:parse error before '}'
```

只有那些含有文件名或行数的错误信息会被显示。Vim 假定你对其它的信息不敢兴趣。有时你也许需要查看那些并不含文件名或行数的信息（比如连接器输出的有关未定义函数的错误信息）。要查看所有信息，在命令后加上 "!" 就可以：

```
:clist!
1 gcc -g -Wall -o prog main.c sub.c
2 main.c: In function 'main':
3 main.c:6: too many arguments to function 'do_sub'
4 main.c: At top level:
5 main.c:10: parse error before '}'
6 make: *** [prog] Error 1
```

Vim 会高亮显示当前错误。如果你想回到上一个错误，执行：

```
:cprevious
```

以下是一些其它的跳转命令：

```
:cfirst         到第一处错误
:clast          到最后一处错误
:cc 3           到第三处错误
```

指 定 编 译 器

通过 'makeprg' 选项可以指定 ":make" 命令启动的程序名。通常它被设定为 "make"。

但 Visual C++ 的用户需要将它设定为 "nmake":

```
:set makeprg=nmake
```

你可以对这个选项指定参数。如有特殊字符，请用 '\' 来转义。比如:

```
:set makeprg=nmake\ -f\ project.mak
```

你还可以使用特殊的 Vim 内部关键字。比如用 '%' 来代替当前文件名:

```
:set makeprg=make\ %:S
```

这样，当你编辑文件 "main.c" 时，运行 ":make" 命令会运行下面的程序:

```
make main.c
```

这不太有用，所以你需要用 :r (root) 修饰符来稍稍修改一下:

```
:set makeprg=make\ %:r:S.o
```

现在运行的程序就成了:

```
make main.o
```

有关修饰符的更多信息，请查看这里: filename-modifiers 。

旧 的 出 错 信 息 列 表

假设你用 ":make" 编译了一个程序。其中的一个文件里有个警告 (warning) 信息而另一个文件中有一个出错 (error) 信息。你修改了这个错误，并再次运行 ":make" 以参看它是否已被纠正。现在你想查看刚才的那个警告信息。但由于含有警告的那个文件并没有被重新编译，你无法在当前出错信息列表中看到原来的警告信息。在这种情况下，你可以用下面的命令来查看上一个出错信息列表:

```
:colder
```

然后你可以通过 ":clist" 和 ":cc {nr}" 命令来跳转到出现警告的地方。
要查看下一个出错列表:

```
:cnewer
```

Vim 可以记住十个出错信息列表。

更 换 编 译 器

要做到这一点，你需要告诉 Vim 所使用编译器产生的错误信息格式。这可以通过 'errorformat' 来设定。这个选项几乎可以配合任意一个编译器的使用，但它的具体配置却很复杂。请在这里查看详细解释: errorformat 。

你可能会用到多种不同的编译器。每次都设定 'makeprg' 选项，尤其是 'errorformat' 选项是很繁杂的。为此，Vim 提供一个简便的方法。比如: 要使用微软的 Visual C++ 编译器:

```
:compiler msvc
```

这个命令会找到适合 "msvc" 编译器的 Vim 脚本文件并设定相关选项。
你可以为编译器编写自己的脚本文件。请参考 write-compiler-plugin 。

输 出 重 定 向

"make" 命令会将运行结果重定向到一个错误文件中。具体的工作方式取决于很多方面，比如 'shell' 选项。如果你的 ":make" 命令不能捕获输出，请检查 'makeef' 和 'shellpipe' 选项。选项 'shellquote' 和 'shellxquote' 可能也会起作用。

如果你无法利用 ":make" 命令重定向输出，一种变通的方法是在另一个窗口编译程序并将输出重定向到一个文件中。然后你可在 Vim 中查看此文件：

```
:cfile {filename}
```

这样，你就可以像运行 ":make" 命令那样跳转到出错的地方。

30.2 C 风格文件缩进

合理的缩进会使程序更容易理解。Vim 提供了多种方法来简化这项工作。要对 C 或 C 风格如 Java 或 C++ 的程序缩进，请设定 'cindent' 选项。Vim 相当地了解 C 程序，它会尽可能地为你自动缩进。通过 'shiftwidth' 选项，你可以指定下一级的缩进空格数。4 个空格的效果很好。用一个 ":set" 命令就可做到：

```
:set cindent shiftwidth=4
```

设定了这一选项之后，当你输入了一个语句，比如 "if (x)" 后，下一行会自动向下一级缩进。

```
                            if (flag)
自动缩进            --->        do_the_work();
自动取消缩进        <--     if (other_flag) {
自动缩进            --->        do_file();
保持缩进                        do_some_more();
自动取消缩进        <--     }
```

当你在大括号 ({}) 内输入时，语句会在开始处缩进，而在结束处取消缩进。每次输入 '}' 后都会取消缩进，因为 Vim 不知道你下一步将要输入什么。

自动缩进还能帮助你提前发现代码中的错误。比如当你输入了一个 '}' 后，如果发现比预想中的缩进多，那可能缺少了一个 '}'。请用 "%" 命令查找与你输入的 '}' 相匹配的 '{'。
缺少 ')' 和 ';' 同样会引起额外的缩进。当你发现比预料中多空白时，最好检查一下之前的代码。

当你的代码没有被正确地排版，或者你插入或删除了某些行时，你需要重新进行代码缩进。操作符 "=" 能完成这项功能，最简单的格式是：

```
==
```

这会缩进当前行。像其它的操作符一样，有三种方式使用它。可视模式下，"=" 缩进选中

的行。一个有用的文本对象是 "a{"。它会选中当前 {} 区。这样，要重新缩进光标所在代码块：

```
=a{
```

你还可以重新缩进整个文件：

```
gg=G
```

但是，不要对已经手工缩进的文件使用此命令。自动缩进的确做得很好，但在某些情况下你也许确实需要手工缩进。

设 定 缩 进 风 格

不同的人有不同的缩进风格。在默认情况下，Vim 采用了 90% 的程序员都使用的一种方式并能很好地完成工作。但是，如果你想使用其它的风格，你可以通过 'cinoptions' 选项来设定。

'cinoptions' 默认为空，Vim 会使用默认风格。你可以在你希望改变的地方添加相应的项目。例如，要使大括号的缩进如下所示：

```
if (flag)
  {
    i = 8;
    j = 0;
  }
```

请使用这个命令：

```
:set cinoptions+={2
```

还有很多其它的项目可供使用。请参考 cinoptions-values 。

30.3 自动缩进

你无需每次编辑 C 文件时都手工设定 'cindent' 选项。通过下面的命令你可以使它自动化：

```
:filetype indent on
```

实际上，它不仅为 C 文件打开了 'cindent' 选项。首先，它会使 Vim 自动检查文件类型。语法高亮同样需要此功能。

一旦文件类型被识别，Vim 会为此类型的文件查找相关的缩进文件。(Vim 的发行中包含了适合多种不同编程语言的缩进文件。) 该缩进文件将会被用来缩进当前文件。

如果你不喜欢这项功能，可以将它关闭：

```
:filetype indent off
```

如果你不想为某种特定类型的文件进行缩进，你可以这样做：
首先建一个只包括下行的文件：

```
:let b:did_indent = 1
```

然后将其重命名为：

```
{directory}/indent/{filetype}.vim
```

{filetype} 是文件类型的名字，比如 "cpp" 或 "java"。你可以用下面的命令来得到 Vim 识别到的文件类型名：

```
:set filetype
```

对本文件，输出会是：

```
filetype=help
```

这样你就可以用 "help" 来表示 {filetype}。

对 {directory} 部分，你需要根据你的运行时目录来设定。请查看下面命令的输出：

```
set runtimepath
```

请使用第一项（也就是第一个逗号前的名字）。如果上面命令的输出是：

```
runtimepath=~/.vim,/usr/local/share/vim/vim60/runtime,~/.vim/after
```

你需要使用 "~/.vim" 来表示 {directory}。这样最后的文件名就是：

```
~/.vim/indent/help.vim
```

除了关闭缩进选项，你还可以编写自己的缩进文件。请参考 indent-expression 。

30.4 其它缩进

最简单的自动缩进通过 'autoindent' 选项来完成，它会延续上一行的缩进。稍微聪明点的是 'smartindent'，这个选项对那些没有缩进文件可用的编程语言很有用。'smartindent' 选项没有 'cindent' 选项聪明，但要比 'autoindent' 聪明些。

如果 'smartindent' 被设定，会在每个 '{' 处新增一级缩进，并在每个 '}' 处消减。另外，对于 'cinwords' 选项所设定的所有单词也会添加新一级的缩进。所有以 '#' 开始的行都会被特殊处理：所有缩进都被清除。这样做是为了保持所有的预处理命令都在第一列开始。缩进会在下一行中恢复。

修 正 缩 进

当你利用 'autoindent' 和 'smartindent' 选项延续上一行的缩进时，有很多时候你都需要添加或删除一个 'shiftwidth' 宽度的缩进。一个快速的方法是在插入模式下利用 CTRL-D 和 CTRL-T 命令。

比如，当你需要输入以下的外壳脚本时：

```
if test -n a; then
   echo a
   echo "-------"
fi
```

设定了这样的选项：

```
:set autoindent shiftwidth=3
```

你先输入了第一行，按下回车后又输入了第二行的开头：

```
if test -n a; then
echo
```

这时你会发现你需要一个额外的缩进。输入 CTRL-T，结果变为：

```
if test -n a; then
   echo
```

在插入模式下，CTRL-T 命令会加入一个 'shiftwidth' 宽度的缩进，无论光标在当前行的什么位置。

你继续输入第二行，按下回车后又输入了第三行。现在的缩进一切正常。然后你按下回车输入最后一行，现在的情况如下所示：

```
if test -n a; then
   echo a
   echo "-------"
   fi
```

要删除这个多余的缩进，可以在最后一行输入 CTRL-D。这会删除一个 'shiftwidth' 宽度的缩进，无论光标在行中的什么位置。

在普通模式下，你可以用 ">>" 和 "<<" 命令来完成缩进的修正。'>' 和 '<' 是操作符，因此你可以使用通常的那三种方式来指定你要缩进的行。一个有用的组合是：

```
>i{
```

这个命令会缩进当前 {} 区内的行，'{' 和 '}' 本身并不被缩进。">a{" 会包括它们。在下面的例子中，光标停留在 "printf" 上：

```
原文                    ">i{" 之后              ">a{" 之后

if (flag)               if (flag)               if (flag)
{                       {                           {
printf("yes");              printf("yes");          printf("yes");
flag = 0;                   flag = 0;               flag = 0;
}                       }                           }
```

30.5 制表符和空格

'tabstop' 在缺省状态下被设定为 8。尽管你可以改变它，但很快你就会遇到麻烦。其它的程序不知道你用的制表符间隔值是多少，你的文件看起来会一下子改变许多。另外，很多打印机都将制表符间隔值固定为 8。所以最好还是保留 'tabstop' 值不变。(如果你编辑使用其它制表符间隔值的文件，请参考 25.3 来修正。)

如果使用 8 个空格来缩进程序，你很快就会走到窗口的最右端；而用 1 个空格又看不出足够的差别。因此很多人喜欢用 4 个空格。这的确是个很好的折衷。

由于一个制表符 (<Tab>) 是 8 个空格，而你又想使用 4 个空格来缩进，这样你就无法使用制表符来完成缩进。这里有两种解决办法：

1. 混合使用制表符和空格。由于一个制表符占用 8 个空格的位置，你的文件会含有更少的字节数。插入或删除一个制表符也要比 8 个空格快很多。

2. 只用空格。这就避免了那些使用不同制表符间隔值的文件所带来的麻烦。

幸运的是，Vim 能够同时很好地支持这两种方式。

混合使用空格和制表符

如果你使用制表符和空格的组合，你直接按正常情况编辑就行。Vim 缺省状态下，能够很好地处理这些情况。

通过设定 'softtabstop' 可以使工作变得更简便。这个选项能使 <Tab> 看起来像是被设定为 'softtabstop' 所指定的值，但实际上使用的确是制表符和空格的组合。

当你执行下面的命令后，你每次按下 <Tab> 键，光标都会移动到下一个 "4 列" 边界：

```
:set softtabstop=4
```

当你在第一列按下 <Tab> 键后，4 个空格会插入到文本中；再次按下 <Tab> 键，Vim 会先删除那 4 个空格，然后再插入一个制表符。Vim 会尽可能地使用制表符，并辅以空格填补。

删除会以相反的方式进行。<BS> 键总是删除 'softtabstop' 指定的数量。Vim 尽可能地使用制表符，而用空格来填补空隙。

下面的例子显示了多次输入制表符然后使用 <BS> 的情况。"." 代表一个空格而 "------->" 代表制表符。

输入	结果
<Tab>	....
<Tab><Tab>	------->
<Tab><Tab><Tab>	------->....
<Tab><Tab><Tab><BS>	------->
<Tab><Tab><Tab><BS><BS>	....

另一种方法是使用 'smarttab' 选项。当它被设定，Vim 对每个在缩进行中的制表符使用 'shiftwidth'，而对在第一个非空字符后输入的 <Tab> 使用真的制表符。但 <BS> 键不会像在 'softtabstop' 选项下那样工作。

只用空格

如果你不想在文件中出现制表符，可以设定 'expandtab' 选项：

```
:set expandtab
```

当这个选项被设定，<Tab>键会插入一系列的空格。这样你可以获得如同插入一个制表符一样数量的空格。但你的文件中并不包含真正的制表符。

退格键（<BS>）每次只能删除一个空格。这样如果你键入了一个 <Tab>，你需要键入 8 次 <BS> 才能恢复。如果你在调整缩进中，输入 CTRL-D 会更快些。

制表符与空格的相互转换

设定 'expandtab' 选项并不会影响已有的制表符。如果你想将制表符转换为空格，可以

用 ":retab" 命令。使用下面的命令：

```
:set expandtab
:%retab
```

Vim 会在所有缩进中使用空格而非制表符。但是，所有非空字符后的制表符不会受到影响。如果你想要转化这些制表符，需要在命令中加入 !：

```
:%retab!
```

这不大安全。因为它也许会修改字符串内的制表符。要检查这种情况是否存在，可以执行：

```
/"[^"\t]*\t[^"]*"
```

这里建议你不要在字符串中直接使用制表符。请用 "\t" 来替代，麻烦会少些。

将空格转化为制表符的命令则恰好相反：

```
:set noexpandtab
:%retab!
```

30.6 排版注释格式

Vim 最了不起的地方之一就是它理解注释。你可以要求 Vim 排版一段注释。它会做得很出色。

比如，你有下面的一段注释：

```
/*
 * This is a test
 * of the text formatting.
 */
```

你可以要求 Vim 排版这段注释。将光标定位到注释开头，然后输入：

```
gq]/
```

"gq" 是用来排版文本的操作符。"]/" 是移动到注释尾的动作。命令的结果是：

```
/*
 * This is a test of the text formatting.
 */
```

注意 Vim 可以正确处理每行的开头。

另外一种方法是在可视模式下用 "gq" 排版选中的文本。

要在注释中加入新的一行，先将光标移到中间一行，然后按 "o"。结果会如下所示：

```
/*
 * This is a test of the text formatting.
 *
 */
```

Vim 会为你自动添加一个星号和空格，现在你可以输入新的注释。如果一行注释长于 'textwidth'，Vim 会将其自动分开。同样，星号和空格会被自动添加进来：

```
/*
 * This is a test of the text formatting.
 * Typing a lot of text here will make Vim
 * break
 */
```

要使用这些功能，你必须在 'formatoptions' 选项中指定一些标志位：

r	在插入模式下，输入回车时插入星号。
o	在普通模式下，使用 "o" 或 "O" 时插入星号。
c	根据 'textwidth' 将注释分行。

更多标志位请参考 fo-table 。

定义注释

'comments' 选项可以定义注释的样式。Vim 可以分辨单行注释和那些包含开头，中间，结尾三部分的注释。

很多单行注释都是以一个特殊的字符开头。在 C++ 中是 //，在 Makefile 中是 #，在 Vim 脚本中是 "。比如，要使 Vim 理解 C++ 注释：

```
:set comments=://
```

冒号将条目的标志位和用来识别注释的字符分开。'comments' 的一般格式是：

```
{flags}:{text}
```

{flags} 部分可以为空（就像本例）。

不同的条目可以连接在一起，用逗号隔开。这样可以在同时识别多种不同的注释。比如，让我们编辑一个 email 信息。当回复时，别人写的内容会以 "> " 和 "! " 字符开头：

```
:set comments=n:>,n:!
```

这里有两个条目。一个识别以 "> " 开头的注释，一个识别以 "! " 开头的注释。两个都设定了 "n" 标志位，这意味着注释可以嵌套。也就是说，一个以 "> " 开始的行可以在 "> " 之后包括其它的注释符号。这样就可以用来排版下面的文本了：

```
> ! Did you see that site?
> ! It looks really great.
> I don't like it.  The
> colors are terrible.
What is the URL of that
site?
```

试着将 'textwidth' 设定为其它的值，例如 80。在可视模式下选中注释，然后输入 "gq"。结果为：

```
> ! Did you see that site? It looks really great.
> I don't like it.  The colors are terrible.
```

```
What is the URL of that site?
```

你会注意到 Vim 并没有将文本从一种注释移动到另一种注释。因为第一行以 ">!" 开头，而第二行以 ">" 开头，Vim 知道它们是不同的注释，所以第二行的 "I" 并没有移到上一行。

包括三部分的注释

C 语言注释的样式是：以 "/*" 开头，中间含有 "*"，以 "*/" 结尾。我们可以通过 'comments' 选项来这样进行设定：

```
:set comments=s1:/*,mb:*,ex:*/
```

开始部分用 "s1:/*" 定义。"s" 表示三部分注释的开始。冒号将标志位与代表注释的特殊字符 "/*" 分开。这里有一个标志位 "1"。它指明注释的中间部分有一个空格位置的偏移。

"mb:*" 的 "m" 表示这是注释的中间部分。"b" 标志位表示星号后要有空格。否则 Vim 会将形如 "*pointer" 的语句视为某个注释的中间部分。

"ex:*/" 中的 "e" 表示注释的结尾。"x" 标志位表示在 Vim 自动插入星号后，输入 "/" 会删除多余的空格。

要了解更多细节，请参考 format-comments 。

下一章： usr_31.txt 使用 GUI

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：lang2 http://vimcdoc.sf.net

利用 GUI

Vim 可以很好地在终端内工作，但是 GUI 也有其优点。比如：一个使用文件的命令可以利用文件浏览器，需要作选择时可以利用对话框。使用快捷键可以更方便的使用菜单。

31.1 文件浏览器
31.2 确认
31.3 菜单快捷键
31.4 Vim 窗口位置及大小
31.5 杂项

后一章：usr_32.txt 撤销树
前一章：usr_30.txt 编辑程序
目录：usr_toc.txt

31.1 文件浏览器

在使用 File/Open... 菜单的时候你会看到一个文件浏览器。这会使你更容易找到要编辑的文件。但是如果你想把窗口分割并编辑另一个文件呢？没有一个菜单项是做这个的。你当然可以先用 Window/Split 然后再用 File/Open...，但这样适得其反。

大多数时候你使用 Vim 都是键入命令的，同样，打开一个文件浏览器也可以。为了让 split 命令使用文件浏览器，在它前面加上 "browse" 就行了：

```
:browse split
```

选中一个文件后 ":split" 命令就会与其一起被执行。如果你取消文件对话框就什么都不会发生。当前窗口也不会被分割。

你也可以指定一个文件名作为参数。这将告诉浏览器从那里开始查找文件。例如：

```
:browse split /etc
```

文件浏览器将以 "/etc" 作为起始目录。

":browse" 命令几乎可被添加在任何打开文件的命令前。

如果没有指定目录，Vim 会自己决定从哪里开始浏览。缺省情况 Vim 会从上次同样的目录开始。这样当你用 ":browse split" 命令并选中了一个 "/usr/local/share" 里的文件，下次你在用 ":browse" 时，浏览将从 "/usr/local/share" 开始。

用 'browsedir' 选项可以改变浏览的启动目录。可能的选项值包括：

```
last        使用上次的目录 (缺省)
buffer      使用当前缓冲区所在的目录
current     使用当前目录
```

例如，当你在 "/usr" 目录内编辑文件 "/usr/local/share/readme" 时，那么命令：

```
:set browsedir=buffer
:browse edit
```

会从 "/usr/local/share" 开始浏览，类似地：

```
:set browsedir=current
:browse edit
```

会从 "/usr" 开始浏览。

> 备注：
> 为了避免使用鼠标，多数文件浏览器提供键盘操作。这些操作因系统而异，这里不作解释。可能的情况下 Vim 使用系统的标准浏览器。请参阅你所用系统的文档。

当你不是使用 GUI 版本时，你也可以使用文件浏览窗口来选择文件。然而，":browse" 命令就无效了。参阅 netrw-browse 。

31.2 确认

Vim 会保护你的文件不被意外的覆盖或者其它的信息丢失。如果你要作一些可能有危险的事，Vim 会以一个错误信息提醒你并建议在命令后加上 ! 来确认你希望进行该操作。

为了避免重新输入一个带有 ! 的命令，你可以要求 Vim 用一个对话框来向你询问。你就可以选择 "OK" 或 "Cancel" 来告诉 Vim 你的要求。

例如，你正在编辑一个文件并做了一些改动。你要开始编辑另一个文件：

```
:confirm edit foo.txt
```

Vim 会弹出一个类似下面的对话框：

```
+----------------------------------+
|                                  |
|   ?   Save changes to "bar.txt"? |
|                                  |
|   YES   NO             CANCEL    |
+----------------------------------+
```

你可以作选择了。如果你希望保存变动，选择 "YES"。如果你想放弃变动：'NO"。如果你想放弃编辑新文件的操作而返回来看看自己都做了那些改动用 "CANCEL"。 你会回到原来的文件，你所做的改动也都还在。

就像 ":browse" 一样，":confirm" 命令也可以被加在多数编辑其它文件的命令之前。你还可以把它们两个联起来用：

```
:confirm browse edit
```

如果当前缓冲区被改动的话这个命令会产生一个对话框。接着会弹出一个文件浏览器来选择将要编辑的文件。

> 备注：
> 在对话框中你可以使用键盘来作选择操作。通常来说 <Tab> 键和光标键可以改变选项。键入 <Enter> 会确认选项。但这也跟你用的系统有关。

当你不是使用 GUI 版本的时候， ":confirm" 也是有效的。Vim 不会弹出一个对话框，而是将询问显示在 Vim 窗口的底部并提示你键入选择。

```
:confirm edit main.c
Save changes to "Untitled"?
[Y]es, (N)o, (C)ancel:
```

你可以键入单键来作出选择。不用键入 <Enter>，这和其它命令行的输入不同。

31.3 菜单快捷键

所有的 Vim 命令都是用键盘来完成的。在不知道命令名称的情况下，使用菜单会简单些。但是你就得把手从键盘上移开去抓鼠标。

通常菜单用键盘也可以操作。这决定于你所使用的系统，但多数情况下是这样工作的：将 <Alt> 键和菜单项中带下划线的字母连用。例如，<A-w> (<Alt> 加 w) 弹出 Window 菜单。

在 Window 菜单下，"split" 菜单项中的 p 下面划了线。放开 <Alt> 键然后按 p 就可以选中它了。

在用 <Alt> 键选中菜单后，你可以用光标键来在菜单内移动。<Right> 选择一个子菜单，<Left> 关闭之。<Esc> 也用来关闭菜单。<Enter> 选中一个菜单项。

使用 <Alt> 键来操作菜单和使用 <Alt> 键的映射会出现冲突。'winaltkeys' 可以用来告诉 Vim 如何对待 <Alt> 键。

缺省值 "menu" 是一个明智的选择：如果该键组合是一个菜单快捷键那么就不能被作为映射。所有其它的键都可以。

"no" 表示不使用 <Alt> 键组合来操作菜单。这样你就必须使用鼠标。所有 <Alt> 键组合都可以被用作键盘映射。

"yes" 表示 Vim 会使用 <Alt> 键组合来操作菜单。另外的 <Alt> 键组合也可以用作其它用途。

31.4 Vim 窗口位置及大小

要查看当前 Vim 窗口在屏幕上的位置可以用：

```
:winpos
```

这只对 GUI 有效。输出可能是这样的：

```
Window position: X 272, Y 103
```

位置是以屏幕像素为单位的。你可以通过数字来将 Vim 窗口移动到别处。例如，将其向左移动一百个像素：

```
:winpos 172 103
```

备注：
报告的窗口位置和窗口被移动的位置可能会有小的出入。这是由窗口周围的边框引起的。边框是被窗口管理器加上的。

你可以在你的启动脚本中使用这个命令来将窗口定位到一个指定的位置。

Vim 窗口的大小是以字符数目计算的。因此它和所使用的字体的大小有关。你可以查看当

前的窗口大小：

```
:set lines columns
```

要改变窗口大小只要改变 'lines' 和/或 'columns' 选项的值即可：

```
:set lines=50
:set columns=80
```

取得窗口大小的操作在终端和在 GUI 上都可以。但是设定大小在绝大多数终端里都是不可能的。

在启动 X-Window 版本的 gvim 时你可以在命令行指定窗口的位置和大小：

```
gvim -geometry {width}x{height}+{x_offset}+{y_offset}
```

{width} 和 {height} 的单位是字符。{x_offset} 和 {y_offset} 的单位是像素。例：

```
gvim -geometry 80x25+100+300
```

31.5 杂项

你可以使用 gvim 来编辑一个 e-mail。在你的 e-mail 程序里你得选择 gvim 作为其编辑程序。当你尝试这种配置的时候，你会发现行不通：邮件程序认为编辑已经结束了，可实际上 gvim 还在运行着！

这里的情况是由于 gvim 启动时会将自己从外壳分离出来的缘故。如果你是从终端启动 gvim 的话这当然没有问题，你还可以在外壳里作其它事。但如果你的确希望等待 gvim 结束的话，你必须阻止这个分离动作。"-f" 参数就是用来做这个的：

```
gvim -f file.txt
```

"-f" 代表前台 (foreground)。这样 Vim 就会挂起它启动所在的外壳，直到你编辑完毕并退出为止。

推迟启动 GUI

在 Unix 上你可以先在终端里启动 Vim。当你在一个外壳里完成各种不同任务的时候这很有方便。如果当你在编辑某个文件时决定你想使用 GUI，你可以这样做：

```
:gui
```

Vim 就会打开 GUI 窗口并不再使用终端。你可以继续在终端里作其它事情。 "-f" 参数在这里也可以用来将 GUI 放置在前台：":gui -f"。

GVIM 启动文件

当 gvim 启动时，它会读取 gvimrc 文件。这和启动 Vim 时用到的 vimrc 文件类似。gvimrc 文件可以被用来为 GUI 设定专用的选项和命令。例如，你可以设定 'lines' 的值来指定窗口的大小：

```
:set lines=55
```

在终端里使用是没有用的，因为它的大小是固定的（除了那些支持调整大小的 xterm）。
Vim 在 vimrc 文件相同的地方找寻 gvimrc 文件。通常来说在 Unix 上是 "~/.gvimrc"；MS-Windows 上是 "$VIM/_gvimrc"。$MYGVIMRC 环境变量设为该值，这样你可以用下面的命令编辑此文件，如果有的话：

```
:edit $MYGVIMRC
```

如果出于某种原因你不想使用通常的 gvimrc 文件，你可以利用 "-U" 参数来指定另外的一个：

```
gvim -U thisrc ...
```

这使你可以用 gvim 来做另外一类编辑。例如你可以用另外的字体大小。
要完全跳过读取 gvimrc 文件：

```
gvim -U NONE ...
```

下一章： usr_32.txt 撤销树

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：Willis http://vimcdoc.sf.net

撤销树

Vim 提供了多层撤销功能。如果你撤销了一些改变然后又进行了一些新的改变，你就在撤销树里建立了一个分支。本文本讨论如何在分支间来回移动。

32.1 撤销到文件写入时的状态
32.2 为每次改变进行编号
32.3 撤销树内任意跳转
32.4 时间旅行

后一章： usr_40.txt 创建新的命令
前一章： usr_31.txt 利用 GUI
目录： usr_toc.txt

32.1 撤销到文件写入时的状态

有时你做了一些改变，然后发现还是想恢复到最近写入文件时的状态。没问题，用下面的命令就可以：

```
:earlier 1f
```

这里，"f" 代表 "file"（文件）。

可以重复此命令，回到更遥远的过去。使用不同于 1 的计数值可以回去得快一些。

如果回去太久了，可以这样往前：

```
:later 1f
```

注意 这些命令真的是依据时间序列进行的。如果你在撤销一些改变后又进行过改变，这一点很有区别。下一节有所解释。

另外也请 注意 我们说的是文本写入。要把撤销信息写入文件见 undo-persistence 。

32.2 为每次改变进行编号

02.5 一节，我们只讨论了单线的撤销/重做。其实，可以出现分支。当你撤销了一些改变，又进行了一些新的改变，新的改变构成了撤销树的一个分支。

让我们从文本 "one" 开始。第一个要做的改变是附加 " too"。然后移动到第一个 'o' 上并修改为 'w'。这时我们有了两个改变，分别编号为 1 和 2，而文本有三个状态：

```
one
 |
改变 1
```

```
        |
    one too
        |
    改变 2
        |
    one two
```

如果我们撤销一次改变，回到 "one too"，然后把 "one" 换成 "me"，我们就在撤销树里建立了一个分支：

```
          one
           |
       改变 1
           |
       one too
       /     \
  改变 2      改变 3
     |          |
  one two     me too
```

现在你可以用 u 命令来撤销。如果你做两次，你得到的是 "one"。用 CTRL-R 来重做，你会到达 "one too"。多做一次 CTRL-R 又把你带到 "me too"。现在我们看到，撤销/重做使用最近使用的分支，在树内上下移动。

这里重要的是改变发生的顺序。这里说的改变不考虑撤销和重做。每次改变后，你会得到一个新的文本状态。

注意 只有改变被编号，上面显示的文本没有标识符。通常，通过它上方的改变号来引用它。但有时也通过他下方的某个改变之一来引用。特别是在树内往上移动的时候，这样你可以知道哪个改变刚刚被撤销掉。

32.3 撤销树内任意跳转

那么你现在怎么能到 "one two" 呢？你可以用这个命令：

```
:undo 2
```

文本现在成为了 "one two"，因为它在改变号 2 之下。用 :undo 命令可以跳转到树内任何改变之下的文本。

现在再进行一项改变：把 "one" 改成 "not"：

```
          one
           |
       改变 1
           |
       one too
       /     \
  改变 2      改变 3
     |          |
  one two     me too
     |
  改变 4
```

```
    |
not two
```

现在你又改了主意想回到 "me too" 了。用 g- 命令。它在时间点上往后退，也就是说，它不是在树内上下移动，而是回到之前所在的改变。

你可以重复 g- ，这样你会看到文本的变化过程：

```
me too
one two
one too
one
```

用 g+ 时间点上往前进：

```
one
one too
one two
me too
not two
```

:undo 用于你知道你要跳转到哪个改变的场合。 g- 和 g+ 用于你不知道具体要到达的改变号的情况。

你可以在 g- 和 g+ 之前加上计数来重复执行。

32.4 时间旅行

如果你在文本上工作了一段时间，撤销树变得相当大。这时你可能想回到几分钟之前的文本上。

要看撤销树里有什么分支，用：

```
:undolist
number changes  time
     3       2  16 seconds ago
     4       3  5 seconds ago
```

这里你可以看到每个分支上叶结点的编号，还有改变发生的时间。假定我们在改变号 4 下方的 "not two" 那里，你可以这样回到十秒前：

```
:earlier 10s
```

取决于改变发生了多长时间，你回到达树内的某个位置。 :earlier 命令参数可以用 "m" 代表分钟，"h" 代表小时，"d" 代表天。用一个很大的数，你可以一路回到很久以前：

```
:earlier 100d
```

要（再次）进入未来世界，用 :later 命令：

```
:later 1m
```

参数可以用 "s"、"m" 和 "h"，和 :earlier 完全类似。

要看到更多的细节，或者要对信息进行操作，可以用 undotree() 函数。要看看它返回什么：

```
:echo undotree()
```

下一章： usr_40.txt 创建新的命令

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2 http://vimcdoc.sf.net

创建新的命令

Vim 是一个可扩展的编辑器。你可以把一系列你常用的命令组合成一个新的命令。或者重新定义一个现存的命令。各种命令的自动执行可以通过自动命令实现。

40.1 键映射
40.2 定义命令行命令
40.3 自动命令

下一章: usr_41.txt 编写 Vim 脚本
前一章: usr_32.txt 撤销树
目录: usr_toc.txt

40.1 键映射

简单的映射已经在 05.3 介绍过了。基本的概念是将一系列的键输入转换成为另外一个键输入序列。这是一个很简单，但是很有效的机制。

最简单的形式是将一个键输入序列映射到一个键上。由于那些除了 <F1> 外的功能键都没有预先定义的功能，选择它们作为映射对象是很有效的。例如:

```
:map <F2> GoDate: <Esc>:read !date<CR>kJ
```

这显示了如何使用三种不同的运行模式。在用 "G" 移动到最后一行后，"o" 命令开始一个新行并开始插入模式。然后文本 "Date: " 被输入并用 <Esc> 离开插入模式。

注意 在 <> 内使用的特殊键。这叫尖括号记法。你要分别地输入这些字符，而不是键入要表示的键本身。这使得映射更具可读性，而且你也可以方便地拷贝，粘贴文本。

":" 使得 Vim 回到命令行。":read !date" 命令读取 "date" 命令的输出并添加到当前行之下。<CR> 是用来执行该命令的。

到此为止，文本看起来像:

```
Date:
Fri Jun 15 12:54:34 CEST 2001
```

然后 "kJ" 将光标上移并将两行连接起来。

参阅 map-which-keys 可以帮助你决定应该使用哪些键来作映射。

映 射 与 运 行 模 式

":map" 命令定义普通模式的键映射。你也可以为其它运行模式定义映射。例如，":imap" 用来定义插入模式的映射。你可以用它来定义一个插入日期的映射:

```
:imap <F2> <CR>Date: <Esc>:read !date<CR>kJ
```

看起来很像前面为普通模式定义的 <F2> 映射。只是开始的地方有所不同。普通模式下的 <F2> 映射依然有效。这样你就可以在各种模式下为同一映射键定义不同的映射。

应该注意的是，虽然这个映射以插入模式开始，但它却以普通模式结束。如果你希望继续插入模式，可以在最后加上 "a"。

下面是一个映射命令及其生效模式的总览：

:map	普通，可视模式及操作符等待模式
:vmap	可视模式
:nmap	普通模式
:omap	操作符等待模式
:map!	插入和命令行模式
:imap	插入模式
:cmap	命令行模式

操作符等待模式是当你键入一个操作符（比如 "d" 或 "y"）之后，Vim 期待你键入一个动作命令或者文本对象时的状态。比如，当你键入命令 "dw"，那个 "w" 就是在操作符等待模式下键入的。

假定你想定义映射 <F7> 使得命令 d<F7> 删除一个 C 程序块（{} 包括的文本）。类似的 y<F7> 会将程序块拷贝到匿名的寄存器。因此，你所要做的就是定义 <F7> 来选择当前的语法块。你可以用下面的命令做到：

```
:omap <F7> a{
```

这使得 <F7> 在操作符等待模式下选择一个块，就像是你键入了 "a{" 一样。这个映射在你不容易键入 { 时比较有用。

映 射 列 表

要查看当前定义的映射，使用不带参数的 ":map" 命令。或者其它运行模式的变体。输出应该类似于：

```
   _g            :call MyGrep(1)<CR>
v  <F2>           :s/^/> /<CR>:noh<CR>``
n  <F2>           :.,$s/^/> /<CR>:noh<CR>``
   <xHome>        <Home>
   <xEnd>         <End>
```

第一列显示该映射有效的运行模式。"n" 表示普通模式，"i" 表示插入模式，等等。空白表示用 ":map" 命令定义的映射，也就是对普通和可视模式有效。

列出映射的一个比较实用的目的是检查 <> 表示的特殊键是否被识别了（仅当支持多色彩是有效）。例如，当 <Esc> 被用彩色显示时，它表示转义字符。否则，只是 5 个不同的字符。

重 映 射

映射的结果会检查其中包括的其他映射。例如，上面对 <F2> 的映射可以减短为：

```
:map <F2> G<F3>
:imap <F2> <Esc><F3>
:map <F3>  oDate: <Esc>:read !date<CR>kJ
```

在普通模式下 <F2> 被映射为：行进至最后一行，然后输入 <F3>；在插入模式下先键入 <Esc> 后也输入 <F3>。接下来 <F3> 也被映射，执行真正的工作。

假设你几乎不使用 Ex 模式，并想用 "Q" 命令来排版文本（就像旧版本的 Vim 那样）。下面的映射就能做到：

```
:map Q gq
```

但是，你总有需要用到 Ex 模式的时候。我们来将 "gQ" 映射为 Q，这样你仍然可以进入 Ex 模式：

```
:map gQ Q
```

这样一来当你键入 "gQ" 时它被映射为 "Q"。到现在为止一切顺利。但由于 "Q" 被映射为 "gq"， 输入的 "gQ" 被解释成为 "gq"， 你根本就没进入 Ex 模式。

要避免键被再次映射，使用 ":noremap" 命令：

```
:noremap gQ Q
```

现在 Vim 就知道了对 "Q" 不需要检查与之相关的映射。对于每个模式都有一个类似的命令：

:noremap	普通，可视和操作符等待模式
:vnoremap	可视模式
:nnoremap	普通模式
:onoremap	操作符等待模式
:noremap!	插入和命令行模式
:inoremap	插入模式
:cnoremap	命令行模式

递 归 映 射

当一个映射调用它本身的时候，会无限制的运行下去。这可以被用来无限次重复一个操作。

例如，你有一组文件，每个的第一行都包括一个版本号。你用 "vim *.txt" 来编辑它们。你现在正在编辑第一个文件。定义下面的映射：

```
:map ,, :s/5.1/5.2/<CR>:wnext<CR>,,
```

现在当你键入 ",," 时，上面的映射被触发。它把第一行的 "5.1" 替换为 "5.2"。接着执行 ":wnext" 来写入文件并开始编辑下一个。映射以 ",," 结束。这又触发了同一个映射，再次执行替换操作，依此类推。

这个映射会一直进行下去，直至遇到错误为止。在这里可能是查找命令无法匹配到 "5.1" 。你可以自行插入 "5.1" 然后再次键入 ",,"。或者 ":wnext" 因为遇到最后一个文件而失败。

当映射在中途遇到错误时，映射的剩余部分会被放弃。你可用 CTRL-C 中断映射。(在 MS-Windows 上用 CTRL-Break)。

删 除 映 射

要删除一个映射，使用 ":unmap" 命令。同样，删除映射的命令也和运行模式相关：

:unmap	普通，可视和操作符等待模式
:vunmap	可视模式
:nunmap	普通模式
:ounmap	操作符等待模式
:unmap!	插入和命令行模式
:iunmap	插入模式
:cunmap	命令行模式

这里有个小技巧可以定义一个对普通模式和操作符等待模式有效而对可视模式无效的映射：先对三个模式都定义映射，然后将可视模式的那个删除：

```
:map <C-A> /---><CR>
:vunmap <C-A>
```

注意那 5 个字符 "<C-A>" 表示一个键组合 CTRL-A。

要清除所有的映射，使用 :mapclear 命令。现在你应该可以猜到各种模式下的变体了吧。要当心使用这个命令，它不可能被撤销。

特 殊 字 符

在 ":map" 命令后面可以追加另一个命令。需要用 | 字符来将两个命令分开。这也就意味着一个映射中不能直接使用该字符。需要时，可以用 <Bar> (五个字符)。例如：

```
:map <F8> :write <Bar> !checkin %:S<CR>
```

":unmap" 命令有同样的问题，而且你得留意后缀的空白字符。下面两个命令是不同的：

```
:unmap a | unmap b
:unmap a| unmap b
```

第一个命令试图删除映射 "a "，后面带有一个空格。

当要在一个映射内使用空格时，应该用 <Space> (七个字符)：

```
:map <Space> W
```

这使得空格键移动到下一个空白字符分割的单词。

在一个映射后不能直接加注释，因为 " 字符也被当作是映射的一部分。你可以用 |" 绕过这一限制。这实际上是开始一个新的空命令。例如：

```
:map <Space> W|     " Use spacebar to move forward a word
```

映 射 与 缩 写

缩写和插入模式的映射很像。对参数的处理它们是一样的。它们主要的不同在于触发的方式。缩写是由单词之后的非单词字符触发的。而映射由其最后一个字符触发。

另一个区别是你键入的缩写的字符会在你键入的同时被插入到文本内。当缩写被触发时，这些字符会被删除并替换成缩写所对应的字符。当你键入一个映射时，直到你完成所有的映射键而映射被触发时，映射所对应的内容才会被插入。如果你置位 'showcmd' 选项，键入的字符会显示在 Vim 窗口的最后一行。

有一个例外是当映射有歧义的时候。假定你有两个映射：

```
:imap aa foo
:imap aaa bar
```

现在，当你键入 "aa" 时，Vim 不知道是否要使用第一个映射。它会等待另一个键输入。如果是 "a"，第二个映射被执行，结果是 "bar"。如果是其它字符，例如空格，第一个映射被执行，结果是 "foo"， 而且空格字符也会被插入。

另 外 ...

<script> 关键字可以被用来使一个映射仅对当前脚本有效。参见 :map-<script> 。

<buffer> 关键字可以被用来使一个映射仅对当前缓冲区有效。参见 :map-<buffer> 。

<unique> 关键字可以被用来当一个映射已经存在时不允许重新定义。否则的话新的映射会简单的覆盖旧的。参见 :map-<unique> 。

如果要使一个键无效，将之映射至 <Nop> (五个字符)。下面的映射会使 <F7> 什么也干不了：

```
:map <F7> <Nop>| map! <F7> <Nop>
```

注意 <Nop> 之后一定不能有空格。

40.2 定义命令行命令

Vim 编辑器允许你定义你自己的命令。你可以像运行其他命令行命令一样运行你自定义的命令。

要定义一个命令，像下面一样执行 ":command" 命令：

```
:command DeleteFirst 1delete
```

现在当你执行 ":DeleteFirst" 命令时，Vim 执行 ":1delete" 来删除第一行。

> 备注：
> 用户定义的命令必须以大写字母开始，但不能用 ":X"，":Next" 和 ":Print"。也不能用下划线！你可以使用数字，但是不鼓励这么做。

要列出用户定义的命令，执行下面的命令：

```
:command
```

像那些内建的命令一样，用户自定义的命令也可以被缩写。你只需要键入足够区别于其它命令的字符就可以了。命令行补全也有效。

参 数 个 数

自定义命令可以带一系列的参数。参数的数目必须用 -nargs 选项来指定。例如，上面例子中的 :DeleteFirst 命令不带参数，所以你也可以这样来定义：

```
:command -nargs=0 DeleteFirst 1delete
```

不过，因为缺省参数数目为 0，你没有必要加上 "-nargs=0"。其它可用的值是：

-nargs=0	无参数
-nargs=1	一个参数
-nargs=*	任意数目的参数
-nargs=?	没有或一个参数
-nargs=+	一个或更多参数

使 用 参 数

在命令的定义中，<args> 关键字可以用来表示命令带的参数。例如：

```
:command -nargs=+ Say :echo "<args>"
```

现在当你输入

```
:Say Hello World
```

Vim 会显示 "Hello World"。然而如果你加上一个双引号，就不行了。例如：

```
:Say he said "hello"
```

要把特殊字符放到字符串里，必须在它们的前面加上反斜杠，用 "<q-args>" 就可以：

```
:command -nargs=+ Say :echo <q-args>
```

现在上面的 ":Say" 命令会引发下面的命令被执行：

```
:echo "he said \"hello\""
```

关键字 <f-args> 包括与 <args> 一样的信息，不过它将其转换成适用于函数调用的格式。例如：

```
:command -nargs=* DoIt :call AFunction(<f-args>)
:DoIt a b c
```

会执行下面的命令：

```
:call AFunction("a", "b", "c")
```

行 范 围

有些命令需要一个范围作为参数。要告诉 Vim 你需要定义这样的命令，使用 -range 选项。它可能的值如下：

-range	允许范围；缺省为当前行。
-range=%	允许范围；缺省为整个文件。
-range={count}	允许范围；只用该范围最后的行号作为单个数字的参数，其缺省值为 {count}。

当一个范围被指定时，关键字 <line1> 和 <line2> 可以用来取得范围的首行和末行的行号。例如，下面的命令定义一个将指定的范围写入文件 "save_file" 的命令 — SaveIt:

```
:command -range=% SaveIt :<line1>,<line2>write! save_file
```

其 它 选 项

其它的一些选项有：

-count={number}	命令可以带 count 参数，缺省为 {number}。用 <count> 关键字可以访问该参数。
-bang	允许使用 !。若 ! 出现，<bang> 扩展为 !。
-register	你可以指定一个寄存器。(缺省为无名寄存器。)指定的寄存器可通过 <reg> (即 <register>) 来操作。
-complete={type}	给出命令行补全的方式。 :command-completion 列出了所有可用值。
-bar	命令后可用 | 加另外一个命令，或 " 加一个注释。
-buffer	命令仅对当前缓冲区有效。

最后，你还可以使用 <lt> 关键字来代表字符 <。这样可以转义上面提到的 <> 项目的特殊含义。

重 定 义 和 删 除

! 参数可以用来重新定义相同的命令：

```
:command -nargs=+ Say :echo "<args>"
:command! -nargs=+ Say :echo <q-args>
```

要删除自定义命令，使用 ":delcommand"。该命令只带一个参数，那就是自定义命令的名字。例：

```
:delcommand SaveIt
```

要一次删除所有的自定义命令：

```
:comclear
```

要当心！这个命令无法撤销。

关于所有这些内容的更多信息可参阅参考手册： user-commands 。

40.3 自动命令

自动命令是一类特殊的命令。当某些事件，例如文件读入或改变缓冲区等事件发生时，它们会自动被执行。例如，通过自动命令你可以教 Vim 来编辑压缩文件。这个功能被用在 gzip 插件里。

自动命令非常强大。如果你小心使用的话，自动命令可以省去你很多自己敲命令的麻烦。如果不当心的话你就是自找麻烦。

假设你希望在每次写入文件时自动的替换文件尾部的日期戳。先定义一个函数：

```
:function DateInsert()
:  $delete
:  read !date
:endfunction
```

你需要在每次缓冲区写入文件之前想办法调用该函数。下面这一行就能做到：

```
:autocmd BufWritePre *  call DateInsert()
```

"BufWritePre" 是这个自动命令的触发事件：把缓冲区写入文件前 (pre)。"*" 是一个用来匹配文件名的模式。这儿它匹配所有文件。

如果这个命令生效，当你调用 ":write" 时，Vim 检查是否有匹配 BufWritePre 事件的自动命令并执行它们。然后才执行 ":write"。

通用的 :autocmd 命令格式如下：

```
:autocmd [group] {events} {file_pattern} [nested] {command}
```

组名 [group] 是可选的。它被用来管理和调用命令 (后面再讲)。{events} 参数是一个触发事件列表 (用逗号隔开)。

{file_pattern} 是文件命令，通常带有通配符。例如，用 "*.txt" 会使得自动命令对所有文件名以 ".txt" 结尾的文件被调用。选项 [nested] 允许自动命令的嵌套 (见下)。最后，{command} 是要被执行的命令。

事 件

最有用的事件之一是 BufReadPost。它在一个文件被调入编辑之后被触发。常被用来设定相关的选项。例如，你已知 ".gsm" 文件是 GNU 汇编程序源码。为确保使用正确的语法文件，可以定义这样的自动命令：

```
:autocmd BufReadPost *.gsm  set filetype=asm
```

如果 Vim 能够正确的识别文件类型的话，它将为你设定 'filetype' 选项。这会触发 Filetype 事件。你可以利用这个来为某一类型的文件做编辑的准备工作。例如，要为文本文件调入一组缩写：

```
:autocmd Filetype text  source ~/.vim/abbrevs.vim
```

在开始编辑一个新文件时，你可以要求 Vim 插入一个模板：

```
:autocmd BufNewFile *.[ch]  0read ~/skeletons/skel.c
```

在 autocmd-events 可以找到一个完整的事件列表。

匹 配 模 式

那个 {file_pattern} 参数实际上可以是一个以逗号分割开的模式列表。例如："*.c,*.h" 匹配所有文件名以 ".c" 和 ".h" 结尾的文件。

常见的文件通配符都可以使用。这里给出一个最常用的清单：

*	匹配任意字符，任意多次
?	匹配任意字符，一次
[abc]	匹配 a、b 或 c
.	匹配一个点 .
a{b,c}	匹配 "ab" 和 "ac"

当模式包括斜杠 (/) 时 Vim 会比较路径名。否则只有文件名的最后部分才用来作比较。例如，"*.txt" 匹配 "/home/biep/readme.txt"。模式 "/home/biep/*" 也可以匹配那个文件。但是 "home/foo/*.txt" 就不行。

当包括斜杠时，Vim 会试着匹配文件的完整路径 ("/home/biep/readme.txt") 和相对路径 (例如: "biep/readme.txt")。

> 备注:
> 当在使用反斜杠作为文件分隔符的系统 (如 MS-Windows) 上工作时，你也得在自动命令中使用正斜杠。这会使编写匹配模式变得容易些，因为反斜杠有特殊的意义。它同时也使自动命令更具可移植性。

删 除

要删除一个自动命令，使用和定义它一样的命令格式。但不要包括后面的 {command} 部分，而且要加上 !。例如:

```
:autocmd! FileWritePre *
```

这样会删除为 "FileWritePre" 事件定义的匹配 "*" 文件名模式的所有自动命令。

列 表

要列出当前定义的所有自动命令，用这个:

```
:autocmd
```

这个列表可能会相当长，特别是在使用了文件类型检测时。你可以指定组，事件和/或文件名模式来要求仅列出相关的命令。例如，要列出 BufNewFile 事件的所有自动命令:

```
:autocmd BufNewFile
```

列出所有匹配文件名模式 "*.c" 的命令:

```
:autocmd * *.c
```

使用 "*" 作为事件会给出所有事件的列表。要列出 cprograms 组对应的自动命令:

```
:autocmd cprograms
```

组

当定义自动命令时用到 {group} 这一项时，自动命令会被分成组。比如说，这可以被用来删除一个组中的所有命令。

在为某一个组定义数个自动命令时，可以使用 ":augroup" 命令。例如，我们来定义

一些用于 C 程序的自动命令：

```
:augroup cprograms
:  autocmd BufReadPost *.c,*.h :set sw=4 sts=4
:  autocmd BufReadPost *.cpp   :set sw=3 sts=3
:augroup END
```

这和下面的命令有一样的效果：

```
:autocmd cprograms BufReadPost *.c,*.h :set sw=4 sts=4
:autocmd cprograms BufReadPost *.cpp   :set sw=3 sts=3
```

要删除 "cprograms" 组中的所有自动命令：

```
:autocmd! cprograms
```

嵌 套

一般的，某一事件触发的自动命令在被执行时不会再触发其它事件。例如，当因 FileChangedShell 事件而读入一个文件时，那些被定义来设定语法的自动命令就不会被触发。要使那些命令被触发，加上一个 "nested" 参数：

```
:autocmd FileChangedShell * nested  edit
```

执 行 自 动 命 令

Vim 允许你用模拟某一事件发生的办法来触发一个自动命令。这可以在一个自动命令里用来触发另外一个。例如：

```
:autocmd BufReadPost *.new  execute "doautocmd BufReadPost " . expand("<afile>:r")
```

这定义了一个新文件开始编辑之后触发的自动命令。这个文件的文件名必须以 ".new" 结尾。其中的 ":execute" 命令利用表达式求值来组成一个新的命令并执行之。当编辑文件 "tryout.c.new" 时被执行的命令将是：

```
:doautocmd BufReadPost tryout.c
```

expand() 函数的参数是 "<afile>"，用来代表自动命令执行所关联的文件。":r" 指定仅使用其根部分。

":doautocmd" 执行于当前缓冲区。":doautoall" 命令于 "doautocmd" 命令类似但执行于所有缓冲区。

使 用 普 通 模 式 命 令

自动命令所执行的命令是 "命令行" 命令。如果你想在其中执行普通模式命令，可以使用 ":normal" 命令。例如：

```
:autocmd BufReadPost *.log normal G
```

这样，当你编辑 *.log 文件时 Vim 会将光标移动到最后一行。

使用 ":normal" 命令需要点技巧。首先，你要确保其参数是一个包括所有参数的完整命令。当你用 "i" 进入插入模式时，你必须用 <Esc> 离开。如果你用 "/" 来开始查找，你也必须用 <CR> 执行该查找命令。

":normal" 命令会使用其后的所有文本作为将要执行的命令。因此不可能用 | 来后跟另一个命令。有个办法可以绕过这个约束：把 ":normal" 命令放在 ":execute" 命令之内。这个方法同时也方便了不可显示的字符作为参数的传递。例如：

```
:autocmd BufReadPost *.chg execute "normal ONew entry:\<Esc>" |
        \ 1read !date
```

上面的例子还展示了如何用反斜杠来将一个长命令分为几行。这可以用在 Vim 脚本中(不能用在命令行)。

如果你想让你的自动命令作一些复杂的操作，其中涉及在文件间跳转然后回到原来位置，你希望能够恢复文件的视窗位置。 restore-position 有些例子。

忽略事件

有些时候，你并不想触发自动命令。'eventignore' 选项包括了一组会被 Vim 完全忽略的事件。例如，下面的命令会使得进入和离开窗口的事件被忽略掉：

```
:set eventignore=WinEnter,WinLeave
```

要忽略所有的事件，用下面的命令：

```
:set eventignore=all
```

要恢复到正常的状态，把 'eventignore' 设定为空即可：

```
:set eventignore=
```

下一章： usr_41.txt 编写 Vim 脚本

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2、Willis http://vimcdoc.sf.net

编写 Vim 脚本

Vim 脚本语言在很多地方用到，包括 vimrc 文件，语法文件，等等。本章讨论 Vim 脚本相关的知识。这样的内容有很多，所以本章也比较长。

41.1 简介
41.2 变量
41.3 表达式
41.4 条件语句
41.5 执行一个表达式
41.6 使用函数
41.7 定义一个函数
41.8 列表和字典
41.9 例外
41.10 其它讨论
41.11 编写插件
41.12 编写文件类型插件
41.13 编写编译器插件
41.14 编写快速载入的插件
41.15 编写库脚本
41.16 发布 Vim 脚本

下一章: usr_42.txt 添加新的菜单
前一章: usr_40.txt 创建新的命令
目录: usr_toc.txt

41.1 简介 vim-script-intro script

你最初接触到 Vim 脚本是在 vimrc 文件里。当 Vim 启动时它将读取该文件的内容并执行其中的命令。你可以在其中设置选项。你也可以在其中使用任何冒号命令 (以 ":" 开头的命令；这些命令有时也被称作 Ex 命令或命令行命令)。

语法文件其实也是 Vim 脚本。专为某种文件类型设定选项的文件也是。一个很复杂的宏可以被单独的定义在一个 Vim 脚本文件中。你可以自己想到其它的应用。

让我们从一个简单的例子开始:

```
:let i = 1
:while i < 5
:  echo "count is" i
:  let i += 1
:endwhile
```

备注:
那些 ":" 字符并非必须。只有在你键入命令时才需要，在编写 Vim 脚本时可以去掉。在这里用一是为了清楚，二是为了区别于普通模式命令。
备注:

你可以拷贝这里的示例文本，然后用 :@" 执行。

本例的输出是：

```
count is 1
count is 2
count is 3
count is 4
```

第一行的 ":let" 命令给一个变量赋值。通常的用法是：

```
:let {变量} = {表达式}
```

在例子中变量名是 "i" 而表达式是一个简单的数值 1。
":while" 命令开始一个循环。通常的用法是：

```
:while {条件}
:  {语句}
:endwhile
```

只要条件为真，"while" 和 ":endwhile" 包围的语句就会被执行。在例子中使用的条件是表达式 "i < 5"。这个条件在变量 i 小于五时总是真的。

备注：
如果你碰巧写了一个死循环语句，你可以用 CTRL-C 来终止（在 MS-Windows 上使用 CTRL-Break）。

":echo" 命令显示它的参数。在这个例子中的参数是字符串 "count is" 和变量 i 的值。因为开始时 i 的值是 1，所以将会显示：

```
count is 1
```

接着是 ":let i += 1" 命令。该命令相当于 ":let i = i + 1"。在变量 i 上加一并将新的值赋给同一个变量。

给出本例是为了解释命令，不过如果你真的要写这样一个循环，下面的表达更加简洁：

```
:for i in range(1, 4)
:  echo "count is" i
:endfor
```

我们现在不解释 :for 和 range() 如何工作，一会儿再说。如果你没有耐心，点击这些链接。

四 种 数 值

数值可以是十进制，十六进制，八进制或者二进制的。以 "0x" 或 "0X" 开始的数值是十六进制的。例如 "0x1f" 代表十进制 31。以零开始的数值是八进制的。"017" 代表十进制 15。以 "0b" 或 "0B" 开始的数值是二进制的。例如 "0b101" 代表十进制 5。当心：不要在十进制数前添上零，那样该数值将会被作为八进制数对待！
":echo" 命令总以十进制格式显示数值。例：

```
:echo 0x7f 036
127 30
```

在一个数值前加上减号会将其变为负值。十六进制数、八进制数和二进制数亦然。减号也用于减法操作。将下例与前面的比较：

```
:echo 0x7f -036
97
```

表达式中的空白字符将被忽略。然而，为了增加表达式的易读性，建议用来分隔不同的项目。例如，为了不和上面的负号混淆，在减号和之后的数字前加入一个空格：

```
:echo 0x7f - 036
```

41.2 变量

一个变量名可以由 ASCII 字符、数字和下划线组成。但是变量名不能以数字开始。以下是几个有效的变量名：

```
counter
_aap3
very_long_variable_name_with_dashes
FuncLength
LENGTH
```

"foo+bar" 和 "6var" 都是无效的变量名。

这些变量都是全局的。要列出当前定义的所有变量可以用这个命令：

```
:let
```

你可以在任何地方使用全局变量。这同时也意味着：当一个脚本文件使用 "count" 变量时，可能另一个脚本文件也使用了这个变量。这至少会引起混乱，严重时会导致脚本无法正常工作。为避免这种情况发生，你可以在变量名前加上 "s:" 使其变成脚本文件的局部变量。例如，一脚本包含以下代码：

```
:let s:count = 1
:while s:count < 5
:  source other.vim
:  let s:count += 1
:endwhile
```

由于 "s:count" 是局部变量，你可以确信调用 (source) "other.vim" 脚本时不会改变它的值。如果 "other.vim" 也使用一个 "s:count" 变量，该变量将会是仅在脚本内有效的局部变量。更多关于脚本局部变量可以在这里读到： script-variable 。

还有很多其它类型的变量，参阅 internal-variables 。最常用的几类有：

b:name	缓冲区的局部变量
w:name	窗口的局部变量
g:name	全局变量 (也用于函数中)
v:name	Vim 预定义的变量

删 除 变 量

变量不仅仅可以在 ":let" 命令显示，同时也占用内存空间。为了删除一个变量可以使用 ":unlet" 命令。例：

```
:unlet s:count
```

这将删除 "s:count" 这个脚本局部变量并释放其占用的内存。如果你并不确定这个变量是否存在，但并不希望系统在它不存在时报错，可以在命令后添加 !：

```
:unlet! s:count
```

当一个脚本结束时，它使用的局部变量不会自动被删除。下一次脚本被执行时，旧的变量值仍可被使用。例：

```
:if !exists("s:call_count")
:  let s:call_count = 0
:endif
:let s:call_count = s:call_count + 1
:echo "called" s:call_count "times"
```

"exists()" 函数检查一个变量是否已经被定义过了。它的参数是你想检查的变量的名字。而不是变量本身！如果你这样做：

```
:if !exists(s:call_count)
```

那么变量 s:call_count 的值将被用来做检测。你不会得到想的结果。

惊叹号 ! 将一个值取反。当该值为真时，表达式的值为假。当该值为假时，表达式的值为真。你可以把它读作 "not"。这样 "if !exists()" 可以被读作 "if not exists()" (如果-不-存在)。

Vim 把任何非零的值当作真。零代表假。

注意：
如果期待数值类型，Vim 自动把字符串转换为数值。如果使用不以数字开始的字符串，返回的数值为零。所以小心这种代码：

```
:if "true"
```

这里 "true" 会被解读为零，也就是假值！

字符串变量和常量

到目前为止我们只用到了数值作为变量的值。同样的我们可以使用字符串。这两种变量类型是 Vim 支持的基本类型。变量的类型是动态的。每当我们通过 ":let" 语句为变量赋值时，变量的类型才被确定。类型更多的内容见 41.8 。

你需要使用字符串常量来为字符串变量赋值。字符串常量有两种。第一种是由双引号括起来的：

```
:let name = "peter"
:echo name
peter
```

如果你想在这样的字符串内使用双引号，在之前加上反斜杠即可：

```
:let name = "\"peter\""
:echo name
"peter"
```

如果你不想使用反斜杠，也可以用单引号括起字符串：

```
:let name = '"peter"'
:echo name
"peter"
```

所有的字符在单引号内都保持其本来面目。只有单引号本身例外：输入两个你会得到一个单引号。因为反斜杠在其中也被作为其本身来对待，你无法使用它来改变其后的字符的意义。

在双引号括起来的字符串中可以使用特殊字符。这里有一些有用的例子：

```
\t              <Tab>
\n              <NL>，换行
\r              <CR>，<Enter>
\e              <Esc>
\b              <BS>，退格
\"              "
\\              \，反斜杠
\<Esc>           <Esc>
\<C-W>           CTRL-W
```

最后两个只是用来举例子的。"\<name>" 的形式可以被用来表示特殊的键 "name"。

在 expr-quote 中列出了全部的特殊字符。

41.3 表达式

Vim 脚本支持的表达式很丰富，也很容易使用。你可以在这里读到表达式的定义：expression-syntax 。这里我们只看看常用的几个。

已经提到的那些数值，字符串常量和变量都属于表达式。因此任何可以使用表达式的地方，数值，字符串变量和常量都可以使用。其它基本的表达式有：

```
$NAME           环境变量
&name           选项
@r              寄存器
```

例子：

```
:echo "The value of 'tabstop' is" &ts
:echo "Your home directory is" $HOME
:if @a > 5
```

&name 这种形式可以被用来暂时改变一个选项的值。例：

```
:let save_ic = &ic
:set noic
:/The Start/,$delete
:let &ic = save_ic
```

这样既确保了在匹配 "The Start" 模式时 'ignorecase' 选项是关闭的，同时也保留了用户原来的选项值。(另一个方法是在模式里加上 "\C"，见 /\C 。)

算 术

我们把这些基本的东西都混合起来用就更有趣了。先来看看算术运算：

```
a + b           加
a - b           减
a * b           乘
a / b           除
a % b           余
```

先乘除，后加减。例如：

```
:echo 10 + 5 * 2
20
```

括号内的先计算。这也没什么奇怪的。例如：

```
:echo (10 + 5) * 2
30
```

用 "." 可以把两个字符串连接起来。例如：

```
:echo "foo" . "bar"
foobar
```

一般的，当 ":echo" 命令遇到多个参数时，会在它们之间加入空格。但上例中参数是一个表达式，所以不会有空格。

下面的条件表达式很显然是从 C 语言里借来的：

```
a ? b : c
```

如果 "a" 为真用 "b"，否则用 "c"。例如：

```
:let i = 4
:echo i > 5 ? "i is big" : "i is small"
i is small
```

在整个表达式被求值前，结构中的三部分总是先被求值的。因此你可以将其视为：

```
(a) ? (b) : (c)
```

41.4 条件语句

":if" 命令在条件满足的前提下，执行其后直到 ":endif" 的所有语句。常用的形式为：

```
:if {condition}
   {statements}
:endif
```

语句 {statements} 仅当表达式 {condition} 为真（非零）时才被执行。这些语句还必须是有效的。否则 Vim 无法找到相应的 ":endif"。

你也可以使用 ":else"。常用形式为：

```
:if {condition}
   {statements}
:else
   {statements}
:endif
```

第二组 {statements} 仅当条件不满足时被执行。
最后还有 ":elseif":

```
:if {condition}
   {statements}
:elseif {condition}
   {statements}
:endif
```

这种形式就像 ":else" 接着 "if" 一样，但是少出现一个 ":endif"。
下面是一个有用的例子（可以用在你的 vimrc 文件里）：它检查 'term' 选项并根据不同的值做不同的操作：

```
:if &term == "xterm"
:  " Do stuff for xterm
:elseif &term == "vt100"
:  " Do stuff for a vt100 terminal
:else
:  " Do something for other terminals
:endif
```

逻 辑 操 作

实际上我们在前面的几个例子中已经是用到了。下面是几个最常用的形式：

```
a == b          等于
a != b          不等于
a >  b          大于
a >= b          大于等于
a <  b          小于
a <= b          小于等于
```

如果条件满足，结果为 1，否则为 0。例如：

```
:if v:version >= 700
:  echo "祝贺"
:else
:  echo "你在使用旧的版本，升级！"
:endif
```

这里 "v:version" 是 Vim 定义的变量，用来存放 Vim 的版本号。600 意为 6.0 版。6.1 版的值为 601。这对编写可以在不同版本的 Vim 上运行的脚本很有用。参阅 v:version

对数值和字符串都可以做逻辑操作。两个字符串的算术差被用来比较它们的值。这个结果是通过字节值来计算的，对于某些语言，这样做的结果未必正确。
在比较一个字符串和一个数值时，该字符串将先被转换成一个数值。这容易出错，因

为当一个字符串看起来不像数值时，它会被当作 0 对待。例如：

```
:if 0 == "one"
:  echo "yes"
:endif
```

上面的例子将显示 "yes"，因为 "one" 看起来不像一个数值，所以被转换为 0 了。

对于字符串来说还有两种操作：

```
a =~ b          匹配
a !~ b          不匹配
```

左边的 "a" 被当作一个字符串。右边的 "b" 被当作一个匹配模式，正如做查找操作一样。例如：

```
:if str =~ " "
:  echo "字符串包括空格"
:endif
:if str !~ '\.$'
:  echo "字符串不以句号结尾"
:endif
```

注意 在匹配模式中用单引号是很有用的。因为匹配模式中通常有很多反斜杠，而反斜杠在双引号字符串中必须双写才有效。

在做字符串比较时用到 'ignorecase' 选项。如果你不希望使用该选项，可以在比较时加上 "#" 或 "?"。"#" 表示大小写敏感；"?" 表示忽略大小写。因此 "==?" 比较两字符串是否相等，不计大小写。"!~#" 检查一个模式是否被匹配，同时也考虑大小写。
expr-== 有一个完整的字符串比较/匹配操作列表。

循 环 详 述

":while" 命令已经在前面提到了。还有另外两条语句可以在 ":while" 和 ":endwhile" 之间使用。

```
:continue               跳回 while 循环的开始；继续循环
:break                  跳至 ":endwhile"；循环结束
```

例：

```
:while counter < 40
:  call do_something()
:  if skip_flag
:    continue
:  endif
:  if finished_flag
:    break
:  endif
:  sleep 50m
:endwhile
```

":sleep" 命令使 Vim 小憩一下。"50m" 表示休息 50 毫秒。再举一个例子，":sleep 4" 休息 4 秒。

更多循环可以用 ":for" 命令实现，见下面的 41.8 。

41.5 执行一个表达式

到目前为止，脚本内的语句都是由 Vim 直接运行的。用 ":execute" 命令可以执行一个表达式的结果。这是一个创建并执行命令的非常有效的方法。

例如要跳转到一个由变量表示的标签：

```
:execute "tag " . tag_name
```

"." 被用来连接字符串 "tag " 和变量 "tag_name" 的值。假设 "tag_name" 的值为 "get_cmd"，那么被将执行的命令将是：

```
:tag get_cmd
```

":execute" 命令只能用来执行冒号命令。":normal" 命令可以用来执行普通模式命令。然而，它的参数只能是按表面意义解释的命令字符，不能是表达式。例如：

```
:normal gg=G
```

这个命令将跳转到第一行并以 "=" 操作符排版所有行。

为了使 ":normal" 命令也可以带表达式，可以把 ":execute" 与其连起来使用。例：

```
:execute "normal " . normal_commands
```

变量 "normal_commands" 必须包含要执行的普通模式命令。

必须确保 ":normal" 的参数是一个完整的命令。否则，Vim 碰到参数的结尾就会中止其运行。例如，如果你开始了插入模式，你必须也退出插入模式。这样没问题：

```
:execute "normal Inew text \<Esc>"
```

这将在当前行插入 "new text "。注意 这里使用了特殊键 "\<Esc>"。这样就避免了在你的脚本当中键入真正的 <Esc> 字符。

如果你不想执行字符串，而想执行它作为表达式计算的结果，可以用 eval() 函数：

```
:let optname = "path"
:let optval = eval('&' . optname)
```

"&" 被加到 "path" 前面，这样传给 eval() 的参数成为 "&path"。这时得到的返回值就是 'path' 选项的值。

相同的操作可以这样完成：

```
:exe 'let optval = &' . optname
```

41.6 使用函数

Vim 定义了大量的函数并通过这些函数提供了丰富的功能。本节将给出一些例子。你可以

在 functions 找到一个完整的列表。

一个函数可以被 ":call" 命令调用。参数列表要用括号括起来，并用逗号分割。例如：

```
:call search("Date: ", "W")
```

这将以 "Date: " 和 "W" 为参数调用 search() 函数。search() 函数的第一个参数是一个查找模式，第二个是一个标志。标志 "W" 表示查找操作遇到文件尾时不折返。

在一个表达式内也可以调用函数。例如：

```
:let line = getline(".")
:let repl = substitute(line, '\a', "*", "g")
:call setline(".", repl)
```

getline() 函数从当前缓冲区获取一行文本。其参数是行号。在本例中，"." 表示光标所在行。

substitute() 函数的功能和 ":substitute" 命令相似。它的第一个参数是要执行替换操作的源字符串。第二个参数是一个匹配模式，第三个参数是替换字符串。最后一个参数是一个标志位。

setline() 函数将第一个参数表示的行的文本置为第二个参数表示的字符串。本例中光标所在的行被 substitute() 函数的结果所替换。因此这三条语句的效果等同于：

```
:substitute/\a/*/g
```

如果你在调用 substitute() 之前或之后有更多的事情要做的话，用函数的方式就会更有趣了。

函 数 function-list

Vim 提供的函数很多。这里我们以它们的用途分类列出。你可以在 functions 找到一个以字母顺序排列的列表。在函数名上使用 CTRL-] 可以跳转至该函数的详细说明。

字符串操作： string-functions

nr2char()	通过数值码值取得一个字符
list2str()	从数值列表取得字符字符串
char2nr()	取得字符的数值码值
str2list()	从字符串取得数值列表
str2nr()	把字符串转换为数值
str2float()	把字符串转换为浮点数
printf()	根据 % 项目格式化字符串
escape()	将字符串通过 '\' 转义
shellescape()	转义字符串用于外壳命令
fnameescape()	转义 Vim 命令使用的文件名
tr()	把一组字符翻译成另一组
strtrans()	将一个字符串变成可显示的格式
tolower()	将一个字符串转换为小写
toupper()	将一个字符串转换为大写
match()	字符串中的模式匹配处
matchend()	字符串中的模式匹配结束处
matchstr()	在一个字符串中匹配一个模式
matchstrpos()	字符串中满足匹配的模式和位置
matchlist()	类似 matchstr()，同时返回子匹配

stridx()	子串在母串中第一次出现的地方
strridx()	子串在母串中最后一次出现的地方
strlen()	以字节计的字符串长度
strchars()	以字符计的字符串长度
strwidth()	字符串的显示长度
strdisplaywidth()	字符串的显示长度，处理制表
substitute()	用一个字符串替换一个匹配的模式
submatch()	取得 ":s" 和 substitute() 匹配中指定的某个匹配
strpart()	用字节索引取得字符串的子串
strcharpart()	用字符索引获取字符串的子串
strgetchar()	用字符索引获取字符串里的字符
expand()	展开特殊的关键字
expandcmd()	像 :edit 那样扩展命令
iconv()	转换文本编码格式
byteidx()	字符串里字符的字节位置
byteidxcomp()	类似于 byteidx()，但计算组合字符
repeat()	重复字符串多次
eval()	计算字符串表达式
execute()	执行 Ex 命令并获取输出
win_execute()	类似于 execute()，但用于指定窗口
trim()	从字符串中删除字符

列表处理: list-functions

get()	得到项目，错误索引不报错
len()	列表的项目总数
empty()	检查列表是否为空
insert()	在列表某处插入项目
add()	在列表后附加项目
extend()	在列表后附加另一个列表
remove()	删除列表里一或多个项目
copy()	建立列表的浅备份
deepcopy()	建立列表的完整备份
filter()	删除列表的选定项目
map()	改变每个列表项目
sort()	给列表排序
reverse()	反转列表项目的顺序
uniq()	删除重复邻接项目的备份
split()	分割字符串成为列表
join()	合并列表项目成为字符串
range()	返回数值序列的列表
string()	列表的字符串表示形式
call()	调用函数，参数以列表形式提供
index()	列表里某值的索引
max()	列表项目的最大值
min()	列表项目的最小值
count()	计算列表里某值的出现次数
repeat()	重复列表多次

字典处理: dict-functions

get()	得到项目，错误的键不报错
len()	字典项目的总数
has_key()	检查某键是否出现在字典里
empty()	检查字典是否为空
remove()	删除字典的项目

extend()	从一个字典增加项目到另一个字典
filter()	删除字典的选定项目
map()	改变每个字典项目
keys()	得到字典的键列表
values()	得到字典的值列表
items()	得到字典的键-值组对的列表
copy()	建立字典的浅备份
deepcopy()	建立字典的完整备份
string()	字典的字符串表示形式
max()	字典项目的最大值
min()	字典项目的最小值
count()	计算字典里某值的出现次数

浮点数计算: float-functions

float2nr()	把浮点数转换为数值
abs()	绝对值（也适用于数值）
round()	四舍五入
ceil()	向上取整
floor()	向下取整
trunc()	删除小数点后的值
fmod()	除法的余数
exp()	指数
log()	自然对数（以 e 为底的对数）
log10()	以 10 为底的对数
pow()	x 的 y 次方
sqrt()	平方根
sin()	正弦
cos()	余弦
tan()	正切
asin()	反正弦
acos()	反余弦
atan()	反正切
atan2()	反正切
sinh()	双曲正弦
cosh()	双曲余弦
tanh()	双曲正切
isnan()	检查非数

其它计算: bitwise-function

and()	按位与
invert()	按位取反
or()	按位或
xor()	按位异或
sha256()	SHA-256 哈希

变量: var-functions

type()	变量的类型
islocked()	检查变量是否加锁
funcref()	返回指向函数的函数引用
function()	得到函数名对应的函数引用
getbufvar()	取得指定缓冲区中的变量值
setbufvar()	设定指定缓冲区中的变量值
getwinvar()	取得指定窗口的变量值
gettabvar()	取得指定标签页的变量值

gettabwinvar()	取得指定窗口和标签页的变量值
setwinvar()	设定指定窗口的变量值
settabvar()	设定指定标签页的变量值
settabwinvar()	设定指定窗口和标签页的变量值
garbagecollect()	可能情况下释放内存

光标和位置标记位置: cursor-functions mark-functions

col()	光标或位置标记所在的列
virtcol()	光标或位置标记所在的屏幕列
line()	光标或位置标记所在行
wincol()	光标所在窗口列
winline()	光标所在窗口行
cursor()	置光标于 行／列 处
screencol()	得到光标的屏幕列
screenrow()	得到光标的屏幕行
screenpos()	文本字符的屏幕行与列
getcurpos()	得到光标位置
getpos()	得到光标、位置标记等的位置
setpos()	设置光标、位置标记等的位置
byte2line()	取得某字节位置所在行号
line2byte()	取得某行之前的字节数
diff_filler()	得到一行之上的填充行数目
screenattr()	得到屏幕行的属性
screenchar()	得到屏幕行的字符代码
screenchars()	得到屏幕行的多个字符代码
screenstring()	得到屏幕行的字符字符串

操作当前缓冲区的文本: text-functions

getline()	从缓冲区中取一行
setline()	替换缓冲区中的一行
append()	附加行或行的列表到缓冲区
indent()	某行的缩进
cindent()	根据 C 缩进法则的某行的缩进
lispindent()	根据 Lisp 缩进法则的某行的缩进
nextnonblank()	查找下一个非空白行
prevnonblank()	查找前一个非空白行
search()	查找模式的匹配
searchpos()	寻找模式的匹配
searchpair()	查找 start/skip/end 配对的另一端
searchpairpos()	查找 start/skip/end 配对的另一端
searchdecl()	查找名字的声明
getcharsearch()	返回字符搜索信息
setcharsearch()	设置字符搜索信息

操作另一个缓冲区文本:

getbufline()	取得指定缓冲区的行列表
setbufline()	替换指定缓冲区的一行
appendbufline()	给指定缓冲区附加行列表
deletebufline()	从指定缓冲区中删除多行

system-functions file-functions

系统调用及文件操作:

glob()	展开通配符
globpath()	在几个路径中展开通配符

glob2regpat()	转换 glob 模式到搜索模式
findfile()	在目录列表里查找文件
finddir()	在目录列表里查找目录
resolve()	找到一个快捷方式所指
fnamemodify()	改变文件名
pathshorten()	缩短路径里的目录名
simplify()	简化路径，不改变其含义
executable()	检查一个可执行程序是否存在
exepath()	可执行程序的完整路径
filereadable()	检查一个文件可读与否
filewritable()	检查一个文件可写与否
getfperm()	得到文件权限
setfperm()	设置文件权限
getftype()	得到文件类型
isdirectory()	检查一个目录是否存在
getfsize()	取得文件大小
getcwd()	取得当前工作路径
haslocaldir()	检查当前窗口是否使用过 :lcd 或 :tcd
tempname()	取得一个临时文件的名称
mkdir()	建立新目录
chdir()	改变当前目录
delete()	删除文件
rename()	重命名文件
system()	得到字符串形式的外壳命令结果
systemlist()	得到列表形式的外壳命令结果
environ()	得到所有环境变量
getenv()	得到一个环境变量
setenv()	设置一个环境变量
hostname()	系统的名称
readfile()	读入文件到一个行列表
readdir()	从目录得到文件名的列表
writefile()	把一个行列表或 blob 写到文件里

日期和时间：　　date-functions　time-functions

getftime()	得到文件的最近修改时间
localtime()	得到以秒计的当前时间
strftime()	把时间转换为字符串
strptime()	把日期/时间字符串转换为时间
reltime()	得到准确的当前或者已经经过的时间
reltimestr()	把 reltime() 的结果转换为字符串
reltimefloat()	把 reltime() 的结果转换为浮点数

buffer-functions　window-functions　arg-functions

缓冲区，窗口及参数列表：

argc()	参数列表项数
argidx()	参数列表中的当前位置
arglistid()	得到参数列表的编号
argv()	从参数列表中取得一项
bufadd()	给缓冲区列表增加文件
bufexists()	检查缓冲区是否存在
buflisted()	检查缓冲区是否存在并在列表内
bufload()	确保缓冲区已加载
bufloaded()	检查缓冲区是否存在并已加载
bufname()	取得某缓冲区名

bufnr()	取得某缓冲区号
tabpagebuflist()	得到标签页里的缓冲区列表
tabpagenr()	得到标签页号
tabpagewinnr()	类似于特定标签页里的 winnr()
winnr()	取得当前窗口的窗口号
bufwinid()	取得某缓冲区的窗口 ID
bufwinnr()	取得某缓冲区的窗口号
winbufnr()	取得某窗口的缓冲区号
listener_add()	新增回调来监听改动
listener_flush()	调用监听器回调
listener_remove()	删除监听器回调
win_findbuf()	寻找包括某缓冲区的窗口
win_getid()	取得窗口的窗口 ID
win_gotoid()	转到指定 ID 的窗口
win_id2tabwin()	给出窗口 ID 获取标签页号和窗口号
win_id2win()	把窗口 ID 转换为窗口号
getbufinfo()	获取缓冲区信息的列表
gettabinfo()	获取标签页信息的列表
getwininfo()	获取窗口信息的列表
getchangelist()	获取改变列表项目的列表
getjumplist()	获取跳转列表项目的列表
swapinfo()	关于交换文件的信息
swapname()	取得缓冲区的交换文件路径

命令行: command-line-functions

getcmdline()	得到当前命令行
getcmdpos()	得到命令行里的光标位置
setcmdpos()	设置命令行里的光标位置
getcmdtype()	得到当前命令行的类型
getcmdwintype()	返回当前命令行窗口类型
getcompletion()	命令行补全匹配的列表

quickfix 和位置列表: quickfix-functions

getqflist()	quickfix 错误的列表
setqflist()	修改 quickfix 列表
getloclist()	位置列表项目的列表
setloclist()	修改位置列表

插入模式补全: completion-functions

complete()	设定要寻找的匹配
complete_add()	加入要寻找的匹配
complete_check()	检查补全是否被中止
complete_info()	取得当前补全的信息
pumvisible()	检查弹出菜单是否显示
pum_getpos()	如果可见，弹出菜单的位置及大小

折叠: folding-functions

foldclosed()	检查某一行是否被折叠起来
foldclosedend()	类似 foldclosed() 但同时返回最后一行
foldlevel()	检查某行的折叠级别
foldtext()	产生折叠关闭时所显示的行
foldtextresult()	得到关闭折叠显示的文本

语法和高亮: syntax-functions highlighting-functions

clearmatches()　清除 matchadd() 和 :match 诸命令定义的所有匹配

getmatches()　得到 matchadd() 和 :match 诸命令定义的所有匹配

hlexists()　检查高亮组是否存在

hlID()　取得高亮组标示

synID()　取得某位置的语法标示

synIDattr()　取得某语法标示的特定属性

synIDtrans()　取得翻译后的语法标示

synstack()　取得指定位置的语法标示的列表

synconcealed()　取得和隐藏 (conceal) 相关的信息

diff_hlID()　得到比较模式某个位置的高亮标示

matchadd()　定义要高亮的模式 (一个 "匹配")

matchaddpos()　定义要高亮的位置列表

matcharg()　得到 :match 参数的相关信息

matchdelete()　删除 matchadd() 或 :match 诸命令定义的匹配

setmatches()　恢复 getmatches() 保存的匹配列表

拼写: spell-functions

spellbadword()　定位光标所在或之后的错误拼写的单词

spellsuggest()　返回建议的拼写校正列表

soundfold()　返回 "发音相似" 的单词等价形式

历史记录: history-functions

histadd()　在历史记录中加入一项

histdel()　从历史记录中删除一项

histget()　从历史记录中提取一项

histnr()　取得某历史记录的最大索引号

交互: interactive-functions

browse()　显示文件查找器

browsedir()　显示目录查找器

confirm()　让用户作出选择

getchar()　从用户那里取得一个字符输入

getcharmod()　取得最近键入字符的修饰符

getmousepos()　取得最近已知的鼠标位置

feedkeys()　把字符放到预输入队列中

input()　从用户那里取得一行输入

inputlist()　让用户从列表里选择一个项目

inputsecret()　从用户那里取得一行输入，不回显

inputdialog()　从用户那里取得一行输入，使用对话框

inputsave()　保存和清除预输入 (typeahead)

inputrestore()　恢复预输入 (译注：参阅 input())

GUI: gui-functions

getfontname()　得到当前使用的字体名

getwinpos()　Vim 窗口的位置

getwinposx()　Vim 窗口的 X 位置

getwinposy()　Vim 窗口的 Y 位置

balloon_show()　设置气泡的内容

balloon_split()　分割消息用于气泡的显示

balloon_gettext()　取得气泡中的文本

Vim 服务器: server-functions

serverlist()	返回服务器列表
remote_startserver()	启动服务器
remote_send()	向 Vim 服务器发送字符命令
remote_expr()	在 Vim 服务器内对一个表达式求值
server2client()	向一个服务器客户发送应答
remote_peek()	检查一个服务器是否已经应答
remote_read()	从一个服务器读取应答
foreground()	将一个 Vim 窗口移至前台
remote_foreground()	将一个 Vim 服务器窗口移至前台

窗口大小和位置: window-size-functions

winheight()	取得某窗口的高度
winwidth()	取得某窗口的宽度
win_screenpos()	取得某窗口的屏幕位置
winlayout()	取得标签页中窗口的布局
winrestcmd()	恢复窗口大小的返回命令
winsaveview()	得到当前窗口的视图
winrestview()	恢复保存的当前窗口的视图

映射: mapping-functions

hasmapto()	检查映射是否存在
mapcheck()	检查匹配的映射是否存在
maparg()	取得映射的右部 (rhs)
wildmenumode()	检查 wildmode 是否激活

测试: test-functions

assert_equal()	断言两个表达式的值相等
assert_equalfile()	断言两个文件的内容相同
assert_notequal()	断言两个表达式的值不等
assert_inrange()	断言表达式在范围内
assert_match()	断言模式与值匹配
assert_notmatch()	断言模式不与值匹配
assert_false()	断言表达式为假
assert_true()	断言表达式为真
assert_exception()	断言命令抛出例外
assert_beeps()	断言命令会响铃
assert_fails()	断言命令会失败
assert_report()	报告测试失败
test_alloc_fail()	使内存分配失败
test_autochdir()	启动时打开 'autochdir'
test_override()	测试 Vim 内部的覆盖
test_garbagecollect_now()	立即清理内存
test_getvalue()	取得内部变量的值
test_ignore_error()	忽略指定的错误信息
test_null_blob()	返回 null blob
test_null_channel()	返回 null 通道
test_null_dict()	返回 null 字典
test_null_job()	返回 null 作业
test_null_list()	返回 null 列表
test_null_partial()	返回 null 偏函数
test_null_string()	返回 null 字符串
test_settime()	设置 Vim 内部使用的时间
test_setmouse()	设置鼠标位置
test_feedinput()	给输入缓冲区加入键序列

test_option_not_set()	复位指定选项已设的标志位
test_scrollbar()	模拟 GUI 中的滚动条动作

进程间通信: channel-functions

ch_canread()	检查是否有可读的内容
ch_open()	打开通道
ch_close()	关闭通道
ch_close_in()	关闭通道的 in 部分
ch_read()	从通道读取信息
ch_readblob()	从通道读取 blob
ch_readraw()	从通道读取未处理的信息
ch_sendexpr()	从通道读取 JSON 信息
ch_sendraw()	向通道发送未处理的信息
ch_evalexpr()	通过通道计算表达式
ch_evalraw()	通过通道计算未经处理的表达式
ch_status()	获取通道的状态
ch_getbufnr()	获取通道的缓冲区号
ch_getjob()	获取通道相关的作业
ch_info()	获取通道信息
ch_log()	在通道日志文件写下信息
ch_logfile()	设置通道日志文件
ch_setoptions()	设置通道的选项
json_encode()	把表达式编码为 JSON 字符串
json_decode()	把 JSON 字符串解码为 Vim 类型
js_encode()	把表达式编码为 JSON 字符串
js_decode()	把 JSON 字符串解码为 Vim 类型

作业: job-functions

job_start()	启动作业
job_stop()	停止作业
job_status()	获取作业状态
job_getchannel()	获取作业使用的通道
job_info()	获取作业信息
job_setoptions()	设置作业选项

标号: sign-functions

sign_define()	定义或更新标号
sign_getdefined()	取得已定义的标号列表
sign_getplaced()	取得已放置的标号列表
sign_jump()	跳转到标号
sign_place()	放置标号
sign_placelist()	放置一列标号
sign_undefine()	删除标号的定义
sign_unplace()	撤销标号的放置
sign_unplacelist()	撤销一列标号的放置

终端窗口: terminal-functions

term_start()	打开终端窗口并运行作业
term_list()	获取终端缓冲区的列表
term_sendkeys()	给终端发送键击
term_wait()	等待屏幕刷新
term_getjob()	获取终端相关联的作业
term_scrape()	获取终端屏幕的行
term_getline()	获取终端的一行文本行

term_getattr()	获取属性 {what} 的值
term_getcursor()	获取终端的光标位置
term_getscrolled()	获取终端的滚动行数
term_getaltscreen()	获取轮换屏幕标志位
term_getsize()	获取终端大小
term_getstatus()	获取终端状态
term_gettitle()	获取终端标题
term_gettty()	获取终端 tty 名
term_setansicolors()	设置 GUI 使用的 16 种 ANSI 颜色
term_getansicolors()	获取 GUI 使用的 16 种 ANSI 颜色
term_dumpdiff()	显示两份屏幕截图的差异
term_dumpload()	在窗口中载入终端屏幕截图
term_dumpwrite()	把终端屏幕的内容写入文件
term_setkill()	设置停止终端中的作业的信号
term_setrestore()	设置恢复终端的命令
term_setsize()	设置终端的大小

弹出窗口: popup-window-functions

popup_create()	在屏幕中央创建弹出
popup_atcursor()	在光标位置正上方创建弹出，光标移开时关闭
popup_beval()	在 v:beval_ 变量指定的位置，光标移开时关闭
popup_notification()	用三秒钟显示通知
popup_dialog()	创建带填充和边框中间对齐的弹出
popup_menu()	提示从列表中选择一个项目
popup_hide()	临时隐藏弹出
popup_show()	显示之前隐藏的弹出
popup_move()	改变弹出的位置和大小
popup_setoptions()	覆盖弹出的选项
popup_settext()	替换弹出缓冲区的内容
popup_close()	关闭一个弹出
popup_clear()	关闭所有弹出
popup_filter_menu()	从一列项目中选择
popup_filter_yesno()	等待直到按了 'y' 或 'n' 为止
popup_getoptions()	取得弹出的当前选项
popup_getpos()	取得弹出的实际位置和大小

定时器: timer-functions

timer_start()	建立定时器
timer_pause()	暂停或继续定时器
timer_stop()	停止定时器
timer_stopall()	停止所有定时器
timer_info()	获取定时器信息

Tags: tag-functions

taglist()	得到匹配标签的列表
tagfiles()	得到标签文件的列表
gettagstack()	得到窗口的标签栈
settagstack()	修改窗口的标签栈

提示缓冲区: promptbuffer-functions

prompt_setcallback()	设置缓冲区的提示回调
prompt_setinterrupt()	设置缓冲区的中断回调
prompt_setprompt()	设置缓冲区的提示文本

杂项: various-functions

mode()	取得当前编辑状态
visualmode()	最近一次使用过的可视模式
exists()	检查变量，函数等是否存在
has()	检查 Vim 是否支持某特性
changenr()	返回最近的改变号
cscope_connection()	检查有无与 cscope 的连接
did_filetype()	检查某文件类型自动命令是否已经使用
eventhandler()	检查是否在一个事件处理程序内
getpid()	得到 Vim 的进程号
libcall()	调用一个外部库函数
libcallnr()	同上，但返回一个数值
undofile()	得到撤销文件名
undotree()	返回撤销树的状态
getreg()	取得寄存器内容
getregtype()	取得寄存器类型
setreg()	设定寄存器内容及类型
reg_executing()	取得正在执行中的寄存器名
reg_recording()	取得正在记录中的突破口名
shiftwidth()	'shiftwidth' 的有效值
wordcount()	获取缓冲区的字节/单词/字符计数
luaeval()	计算 Lua 表达式
mzeval()	计算 MzScheme 表达式
perleval()	计算 Perl 表达式 (+perl)
py3eval()	计算 Python 表达式 (+python3)
pyeval()	计算 Python 表达式 (+python)
pyxeval()	计算 python_x 表达式
debugbreak()	中断正在调试的程序

41.7 定义一个函数

Vim 允许你定义自己的函数。基本的函数声明如下:

```
:function {name}({var1}, {var2}, ...)
:  {body}
:endfunction
```

注意:
函数名必须以大写字母开始。

让我们来定义一个返回两数中较小者的函数。从下面这一行开始:

```
:function Min(num1, num2)
```

这将告诉 Vim 这个函数名叫 "Min" 并且带两个参数: "num1" 和 "num2"。
你要做的第一件事就是看看哪个数值小一些:

```
:  if a:num1 < a:num2
```

特殊前缀 "a:" 告诉 Vim 该变量是一个函数参数。我们把最小的数值赋给 smaller 变量：

```
:  if a:num1 < a:num2
:    let smaller = a:num1
:  else
:    let smaller = a:num2
:  endif
```

"smaller" 是一个局部变量。一个在函数内部使用的变量，除非被加上类似 "g:"、"a:" 或 "s:" 的前缀，都是局部变量。

> 备注：
> 为了从一个函数内部访问一个全局变量你必须在前面加上 "g:"。因此在一个函数内 "g:today" 表示全局变量 "today"，而 "today" 是另外一个仅用于该函数内的局部变量。

现在你可以使用 ":return" 语句来把最小的数值返回给调用者了。最后，你需要结束这个函数：

```
:  return smaller
:endfunction
```

下面是这个函数完整的定义：

```
:function Min(num1, num2)
:  if a:num1 < a:num2
:    let smaller = a:num1
:  else
:    let smaller = a:num2
:  endif
:  return smaller
:endfunction
```

如果你喜欢简短的函数，下面是等价的形式：

```
:function Min(num1, num2)
:  if a:num1 < a:num2
:    return a:num1
:  endif
:  return a:num2
:endfunction
```

调用用户自定义函数的方式和调用内置函数完全一致。仅仅是函数名不同而已。上面的 Min 函数可以这样来使用：

```
:echo Min(5, 8)
```

只有这时函数才被 Vim 解释并执行。如果函数中有类似未定义的变量之类的错误，你将得到一个错误信息。这些错误在定义函数时是不会被检测到的。

当一个函数执行到 ":endfunction" 或 ":return" 语句没有带参数时，该函数返回零。

如果要重定义一个已经存在的函数，在 "function" 命令后加上 !:

```
:function!  Min(num1, num2, num3)
```

范 围 的 使 用

":call" 命令可以带一个行表示的范围。这可以分成两种情况。当一个函数定义时给出了 "range" 关键字时，表示它会自行处理该范围。

Vim 在调用这样一个函数时给它传递两个参数："a:firstline" 和 "a:lastline"，用来表示该范围所包括的第一行和最后一行。例如：

```
:function Count_words() range
:  let lnum = a:firstline
:  let n = 0
:  while lnum <= a:lastline
:    let n = n + len(split(getline(lnum)))
:    let lnum = lnum + 1
:  endwhile
:  echo "found " . n . " words"
:endfunction
```

你可以这样调用上面的函数：

```
:10,30call Count_words()
```

这个函数将被调用一次并显示字数。

另一种使用范围的方式是在定义函数时不给出 "range" 关键字。Vim 将把光标移动到范围内的每一行，并分别对该行调用此函数。例如：

```
:function  Number()
:  echo "line " . line(".") . " contains: " . getline(".")
:endfunction
```

如果你用下面的方式调用该函数：

```
:10,15call Number()
```

它将被执行六次。

可 变 参 数

Vim 允许你定义参数个数可变的函数。下面的例子给出一个至少有一个参数 (start)，但可以多达 20 个附加参数的函数：

```
:function Show(start, ...)
```

变量 "a:1" 表示第一个可选的参数，"a:2" 表示第二个，如此类推。变量 "a:0" 表示这些参数的个数。

例如：

```
:function Show(start, ...)
```

```
:  echohl Title
:  echo "start is " . a:start
:  echohl None
:  let index = 1
:  while index <= a:0
:    echo "  Arg " . index . " is " . a:{index}
:    let index = index + 1
:  endwhile
:  echo ""
:endfunction
```

上例中 ":echohl" 命令被用来给出接下来的 ":echo" 命令如何高亮输出。
":echohl None" 终止高亮。":echon" 命令除了不输出换行符外，和 ":echo" 一样。

你可以用 a:000 变量，它是所有 "..." 参数的列表。见 a:000 。

函 数 清 单

":function" 命令列出所有用户自定义的函数及其参数:

```
:function
function Show(start, ...)
function GetVimIndent()
function SetSyn(name)
```

如果要查看该函数具体做什么，用该函数名作为 ":function" 命令的参数即可:

```
:function SetSyn
1     if &syntax == ''
2       let &syntax = a:name
3     endif
   endfunction
```

调 试

调试或者遇到错误信息时，行号是很有用的。有关调试模式请参阅 debug-scripts 。
你也可以通过将 'verbose' 选项设为 12 以上来察看所有函数调用。将该参数设为 15 或以上可以查看所有被执行的行。

删 除 函 数

为了删除 Show() 函数:

```
:delfunction Show
```

如果该函数不存在，你会得到一个错误信息。

函 数 引 用

有时使变量指向一个或另一个函数可能有用。要这么做，用 function() 函数。它把函数

名转换为引用：

```
:let result = 0         " 或 1
:function! Right()
:  return 'Right!'
:endfunc
:function! Wrong()
:  return 'Wrong!'
:endfunc
:
:if result == 1
:  let Afunc = function('Right')
:else
:  let Afunc = function('Wrong')
:endif
:echo call(Afunc, [])
Wrong!
```

注意 保存函数引用的变量名必须用大写字母开头，不然和内建函数的名字会引起混淆。

调用变量指向的函数可以用 `call()` 函数。它的第一个参数是函数引用，第二个参数是参数构成的列表。

和字典组合使用函数引用是最常用的，下一节解释。

41.8 列表和字典

到目前为止，我们用了基本类型字符串和数值。Vim 也支持两种复合类型：列表和字典。

列表是事物的有序序列。这里的事物包括各种类型的值。所以你可以建立数值列表、列表列表甚至混合项目的列表。要建立包含三个字符串的列表：

```
:let alist = ['aap', 'mies', 'noot']
```

列表项目用方括号包围，逗号分割。要建立空列表：

```
:let alist = []
```

用 `add()` 函数可以为列表加入项目：

```
:let alist = []
:call add(alist, 'foo')
:call add(alist, 'bar')
:echo alist
['foo', 'bar']
```

列表的连接用 `+` 完成：

```
:echo alist + ['foo', 'bar']
['foo', 'bar', 'foo', 'bar']
```

或者，你可以直接扩展一个列表：

```
:let alist = ['one']
```

```
:call extend(alist, ['two', 'three'])
:echo alist
['one', 'two', 'three']
```

注意 这里如果用 add()，效果不一样：

```
:let alist = ['one']
:call add(alist, ['two', 'three'])
:echo alist
['one', ['two', 'three']]
```

add() 的第二个参数作为单个项目被加入。

FOR 循环

使用列表的一个好处是可以在上面进行叠代：

```
:let alist = ['one', 'two', 'three']
:for n in alist
:  echo n
:endfor
one
two
three
```

这段代码循环遍历列表 "alist" 的每个项目，分别把它们的值赋给变量 "n"。for 循环通用的形式是：

```
:for {varname} in {listexpression}
:  {commands}
:endfor
```

要循环若干次，你需要长度为给定次数的列表。range() 函数建立这样的列表：

```
:for a in range(3)
:  echo a
:endfor
0
1
2
```

注意 range() 产生的列表的第一个项目为零，而最后一个项目比列表的长度小一。
你也可以指定最大值、步进，反向也可以：

```
:for a in range(8, 4, -2)
:  echo a
:endfor
8
6
4
```

更有用的示例，循环遍历缓冲区的所有行：

```
:for line in getline(1, 20)
:  if line =~ "Date: "
:    echo matchstr(line, 'Date: \zs.*')
:  endif
:endfor
```

察看行 1 到 20（包含），并回显那里找到的任何日期。

字 典

字典保存键-值组对。如果知道键，你可以快速查找值。字典用花括号形式建立：

```
:let uk2nl = {'one': 'een', 'two': 'twee', 'three': 'drie'}
```

现在你可以把键放在方括号里以查找单词：

```
:echo uk2nl['two']
twee
```

字典定义的通用形式是：

```
{<key> : <value>, ...}
```

空字典是不包含任何键的字典：

```
{}
```

字典的用途很多。它可用的函数也不少。例如，你可以得到它的键列表并在其上循环：

```
:for key in keys(uk2nl)
:  echo key
:endfor
three
one
two
```

注意 这些键没有排序。你自己可以对返回列表按照特定顺序进行排序：

```
:for key in sort(keys(uk2nl))
:  echo key
:endfor
one
three
two
```

但你永远不能得到项目定义时的顺序。为此目的，只能用列表。列表里的项目被作为有序序列保存。

字 典 函 数

字典项目通常可以用方括号里的索引得到：

```
:echo uk2nl['one']
een
```

完成同样操作且无需那么多标点符号的方法：

```
:echo uk2nl.one
een
```

这只能用于由 ASCII 字母、数位和下划线组成的键。此方式也可以用于赋值：

```
:let uk2nl.four = 'vier'
:echo uk2nl
{'three': 'drie', 'four': 'vier', 'one': 'een', 'two': 'twee'}
```

现在来一些特别的：你可以直接定义函数并把它的引用放在字典里：

```
:function uk2nl.translate(line) dict
:  return join(map(split(a:line), 'get(self, v:val, "???")'))
:endfunction
```

让我们先试试：

```
:echo uk2nl.translate('three two five one')
drie twee ??? een
```

你注意到的第一个特殊之处是 ":function" 一行最后的 "dict"。这标记该函数为某个字典使用。"self" 局部变量这时可以引用该字典。

现在把这个复杂的 return 命令拆开：

```
split(a:line)
```

split() 函数接受字符串，把它分成空白分隔的多个单词，并返回这些单词组成的列表。所以下例返回的是：

```
:echo split('three two five one')
['three', 'two', 'five', 'one']
```

map() 函数的第一个参数是上面这个列表。它然后遍历列表，用它的第二个参数来进行计算，过程中 "v:val" 设为每个项目的值。这相当于 for 循环的快捷方式。命令：

```
:let alist = map(split(a:line), 'get(self, v:val, "???")')
```

等价于：

```
:let alist = split(a:line)
:for idx in range(len(alist))
:  let alist[idx] = get(self, alist[idx], "???")
:endfor
```

get() 函数检查某键是否在字典里存在。如果是，提取它对应的键。如果不是，返回缺省值，此例中缺省值是 '???'。此函数可以很方便地处理键不一定存在而你不想要错误信息的情形。

join() 函数和 split() 刚好相反：它合并列表里的单词，中间放上空格。

split()、map() 和 join() 的组合非常简洁地对单词组成的行进行过滤。

面 向 对 象 编 程

现在你可以把值和函数都放进字典里，实际上，字典已经可以作为对象来使用。

上面我们用了一个字典来把荷兰语翻译为英语。我们可能也想为其他的语言作同样的事。让我们先建立一个对象（也就是字典），它支持 translate 函数，但没有要翻译的单词表：

```
:let transdict = {}
:function transdict.translate(line) dict
:  return join(map(split(a:line), 'get(self.words, v:val, "???")'))
:endfunction
```

和上面的函数稍有不同，这里用 'self.words' 来查找单词的翻译，但我们还没有 self.words。所以你可以把这叫做抽象类。

让我们现在实例化一个荷兰语的翻译对象：

```
:let uk2nl = copy(transdict)
:let uk2nl.words = {'one': 'een', 'two': 'twee', 'three': 'drie'}
:echo uk2nl.translate('three one')
drie een
```

然后来一个德语的翻译器：

```
:let uk2de = copy(transdict)
:let uk2de.words = {'one': 'eins', 'two': 'zwei', 'three': 'drei'}
:echo uk2de.translate('three one')
drei eins
```

你看到 copy() 函数被用来建立 "transdict" 字典的备份，然后修改此备份以加入单词表。当然，原来的字典还是保持原样。

现在你可以再进一步，使用你偏好的翻译器：

```
:if $LANG =~ "de"
:  let trans = uk2de
:else
:  let trans = uk2nl
:endif
:echo trans.translate('one two three')
een twee drie
```

这里 "trans" 指向两个对象（字典）之一，并不涉及到备份的建立。关于列表和字典同一性的更多说明可见 list-identity 和 dict-identity 。

你使用的语言现在可能还不支持。你可以覆盖 translate() 函数，让它什么都不做：

```
:let uk2uk = copy(transdict)
:function! uk2uk.translate(line)
:  return a:line
:endfunction
```

```
:echo uk2uk.translate('three one wladiwostok')
three one wladiwostok
```

注意 使用 ! 会覆盖已有的函数引用。现在，在没找到能够识别的语言的时候，让我们用 "uk2uk":

```
:if $LANG =~ "de"
:  let trans = uk2de
:elseif $LANG =~ "nl"
:  let trans = uk2nl
:else
:  let trans = uk2uk
:endif
:echo trans.translate('one two three')
one two three
```

进一步的阅读可见 Lists 和 Dictionaries 。

41.9 例外

让我们从一个例子开始：

```
:try
:   read ~/templates/pascal.tmpl
:catch /E484:/
:   echo "Sorry, the Pascal template file cannot be found."
:endtry
```

如果该文件不存在的话，":read" 命令就会失败。这段代码可以捕捉到该错误并向用户给出一个友好的信息，而不是一个一般的出错信息。

在 ":try" 和 ":endtry" 之间的命令产生的错误将被转变成为例外。例外以字符串的形式出现。当例外是错误时该字符串就是出错信息。而每一个出错信息都有一个对应的错误码。在上面的例子中，我们捕捉到的错误包括 "E484"。Vim 确保这个错误码始终不变(文字可能会变，例如被翻译)。

当 ":read" 命令引起其它错误时，模式 "E484:" 不会被匹配。因此该例外不会被捕获，结果是一个一般的出错信息。

你可能想这样做：

```
:try
:   read ~/templates/pascal.tmpl
:catch
:   echo "Sorry, the Pascal template file cannot be found."
:endtry
```

这意味着所有的错误都将被捕获。然而这样你就无法得到那些有用的错误信息，比如说 "E21: Cannot make changes, 'modifiable' is off"。

另一个有用的机制是 ":finally" 命令：

```
:let tmp = tempname()
```

```
:try
:   exe ".,$write " . tmp
:   exe "!filter " . tmp
:   .,$delete
:   exe "$read " . tmp
:finally
:   call delete(tmp)
:endtry
```

这个例子将自光标处到文件尾的所有行通过过滤器 "filter"。该程序的参数是文件名。无论在 ":try" 和 ":finally" 之间发生了什么，"call delete(tmp)" 命令始终被执行。这可以确保你不会留下一个临时文件。

关于例外处理更多的讨论可以阅读参考手册： exception-handling 。

41.10 其它的讨论

这里集中了一些和 Vim 脚本相关的讨论。别的地方其实也提到过，这里算做一个整理。

行结束符取决于所在的系统。Unix 系统使用单个的 <NL> 字符。MS-Windows 系列的系统使用 <CR><LF>。对于那些使用 <CR> 的映射而言，这一点很重要。参阅 :source_crnl 。

空 白 字 符

可以使用空白行，但没有作用。

行首的空白字符（空格和制表符）总被忽略。参数间的（例如像下面命令中 "set" 和 "cpoptions" 之间的）空白字符被归约为单个，仅用作分隔符。而最后一个（可见）字符之后的空白字符可能会被忽略也可能不会，视情况而定。见下。

对于一个带有等号 "=" 的 ":set" 命令，如下：

```
:set cpoptions    =aABceFst
```

紧接着等号之前的空白字符会被忽略。然而其后的空白字符是不允许的！

为了在一个选项值内使用空格，必须像下面例子那样使用反斜杠：

```
:set tags=my\ nice\ file
```

如果写成这样：

```
:set tags=my nice file
```

Vim 会给出错误信息，因为它被解释成：

```
:set tags=my
:set nice
:set file
```

注 释

双引号字符 " 标记注释的开始。除了那些不接受注释的命令外（见下例），从双引号起的直到行末的所有字符都将被忽略。注释可以从一行的任意位置开始。

对于某些命令来说，这里有一个小小的 "陷阱"。例如：

```
:abbrev dev development          " shorthand
:map <F3> o#include               " insert include
:execute cmd                     " do it
:!ls *.c                         " list C files
```

缩写 'dev' 会被展开成 'development " shorthand'；<F3> 的键盘映射会是包括 '" insert include' 在内的那一整行；"execute" 命令会给出错误；"!" 命令会将其后的所有字符传给外壳，从而引起一个不匹配 '"' 的错误。

结论是，":map"，":abbreviate"，":execute" 和 "!" 命令之后不能有注释。（另外还有几个命令也是如此）。不过，对于这些命令有一个小窍门：

```
:abbrev dev development|" shorthand
:map <F3> o#include|" insert include
:execute cmd                     |" do it
```

'|' 字符被用来将两个命令分隔开。后一个命令仅仅是一个注释。最后一个命令里，你需要做两件事：:execute 和用 '|'：

```
:exe '!ls *.c'                   |" list C files
```

注意 在缩写和映射后的 '|' 之前没有空格。这是因为对于这些命令，直到行尾或者 '|' 字符为止的内容都是有效的。此行为的后果之一，是你没法总看到这些命令后面包括的空白字符：

```
:map <F4> o#include
```

要发现这个问题，你可以在你的 vimrc 文件内置位 'list' 选项。

Unix 上有一个特殊的办法给一行加注释，从而使得 Vim 脚本可执行：

```
#!/usr/bin/env vim -S
echo "this is a Vim script"
quit
```

"#" 命令本身列出一行并带行号。加上感叹号后使得它什么也不做。从而，你可以在后面加上外壳命令来执行其余的文件。 :#! -S

陷 阱

下面的例子的问题就更大了：

```
:map ,ab o#include
:unmap ,ab
```

这里，unmap 命令是行不通的，因为它试着 unmap ",ab "。而这个映射根本就不存在。因为 'unmap ,ab ' 的末尾的那个空白字符是不可见的，这个错误很难被找出。

在下面这个类似的例子里，'unmap' 后面带有注释：

```
:unmap ,ab     " comment
```

注释将被忽略。然而，Vim 会尝试 unmap 不存在的 ',ab   '。可以重写成：

```
:unmap ,ab|" comment
```

恢复一个视窗位置

有时有你想做一些改动然后回到光标原来的位置。如果能恢复相对位置，把和改动前同样的行置于窗口顶端就更好了。

这里的例子拷贝当前行，粘贴到文件的第一行，然后恢复视窗位置：

```
map ,p ma"aYHmbgg"aP`bzt`a
```

解析：

```
ma"aYHmbgg"aP`bzt`a
ma                      在当前位置做标记 a
  "aY                   将当前行拷贝至寄存器 a
     Hmb                移动到窗口的顶行并做标记 b
        gg              移动到文件首行
          "aP           粘贴拷贝的行到上方
             `b         移动到刚才的顶行
               zt       使窗口出现的文本恢复旧观
                 `a     回到保存的光标位置
```

封装

为了避免你的函数名同其它的函数名发生冲突，使用这样的方法：

- 在函数名前加上独特的字符串。我通常使用一个缩写。例如，"OW_" 被用在 option window 函数上。
- 将你的函数定义放在一个文件内。设置一个全局变量用来表示这些函数是否已经被加载了。当再次 source 这个文件的时候，先将这些函数卸载。

例如：

```
" This is the XXX package

if exists("XXX_loaded")
  delfun XXX_one
  delfun XXX_two
endif

function XXX_one(a)
        ... body of function ...
endfun

function XXX_two(b)
        ... body of function ...
endfun

let XXX_loaded = 1
```

41.11 编写插件 write-plugin

用约定方式编写的脚本能够被除作者外的很多人使用。这样的脚本叫做插件。Vim 用户只要把你写的脚本放在 plugin 目录下就可以立即使用了： add-plugin 。

实际上有两种插件：

全局插件：适用于所有类型的文件。
文件类型插件：仅适用于某种类型的文件。

这一节将介绍第一种。很多的东西也同样适用于编写文件类型插件。仅适用于编写文件类型插件的知识将在下一节 write-filetype-plugin 做介绍。

插 件 名

首先你得给你的插件起个名字。这个名字应该很清楚地表示该插件的用途。同时应该避免同别的插件用同样的名字而用途不同。请将插件名限制在 8 个字符以内，这样可以使得该插件在老的 MS-Windows 系统也能使用。

一个纠正打字错误的插件可能被命名为 "typecorr.vim"。我们将用这个名字来举例。

为了使一个插件能被所有人使用，要注意一些事项。下面我们将一步步的讲解。最后会给出这个插件的完整示例。

插 件 体

让我们从做实际工作的插件体开始：

```
14      iabbrev teh the
15      iabbrev otehr other
16      iabbrev wnat want
17      iabbrev synchronisation
18              \ synchronization
19      let s:count = 4
```

当然，真正的清单会比这长的多。

上面的行号只是为了方便解释，不要把它们也加入到你的插件文件中去！

插 件 头

你很可能对这个插件做新的修改并很快就有了好几个版本。并且当你发布文件的时候，别人也想知道是谁编写了这样好的插件或者给作者提点意见。所以，在你的插件头部加上一些描述性的注释是很必要的：

```
1       " Vim global plugin for correcting typing mistakes
2       " Last Change: 2000 Oct 15
3       " Maintainer: Bram Moolenaar <Bram@vim.org>
```

关于版权和许可：由于插件很有用，而且几乎不值得限制其发行，请考虑对你的插件使用

公共领域 (public domain) 或 Vim 许可 license 。在文件顶部放上说明就行了。例如:

```
4     " License:      This file is placed in the public domain.
```

续行，避免副效应 use-cpo-save

在上面的第 18 行中，用到了续行机制 line-continuation 。那些置位了 'compatible' 选项的用户可能会在这里遇到麻烦。他们会得到一个错误信息。我们不能简单的复位 'compatible' 选项，因为那样会带来很多的副效应。为了避免这些副效应，我们可以将 'cpoptions' 选项设为 Vim 缺省值并在后面恢复之。这将允许续行功能并保证对大多数用户来讲脚本是可用的。就像下面这样:

```
11    let s:save_cpo = &cpo
12    set cpo&vim
..
42    let &cpo = s:save_cpo
43    unlet s:save_cpo
```

我们先将 'cpoptions' 的旧值存在 s:save_cpo 变量中。在插件的最后该值将被恢复。

注意 上面使用了脚本局部变量 s:var 。因为可能已经使用了同名的全局变量。对于仅在脚本内用到的变量总应该使用脚本局部变量。

禁止加载

有可能一个用户并不总希望加载这个插件。或者系统管理员在系统的插件目录中已经把这个插件删除了，而用户希望使用它自己安装的插件。用户应该有机会选择不加载指定的插件。下面的一段代码就是用来实现这个目的的:

```
6     if exists("g:loaded_typecorr")
7       finish
8     endif
9     let g:loaded_typecorr = 1
```

这同时也避免了同一个脚本被加载两次以上。因为那样用户会得到各种各样的错误信息。比如函数被重新定义，自动命令被多次加入等等。

建议使用的名字以 "loaded_" 开头，然后是插件的文件名，按原义输入。之前加上 "g:" 以免错误地在函数中使用该变量 (没有 "g:" 可以是局部于函数的变量)。

"finish" 阻止 Vim 继续读入文件的其余部分，这比用 if-endif 包围整个文件要快得多。

映射

现在让我们把这个插件变得更有趣些: 我们将加入一个映射用来校正当前光标下的单词。我们当然可以任意选一个键组合，但是用户可能已经将其定义为其它的什么功能了。为了使用户能够自己定义在插件中的键盘映射使用的键，我们可以使用 <Leader> 标识:

```
22      map <unique> <Leader>a  <Plug>TypecorrAdd
```

那个 "<Plug>TypecorrAdd" 会做实际的工作，后面我们还会做更多解释。

用户可以将 "mapleader" 变量设为他所希望的开始映射的键组合。比如假设用户这样做：

```
let mapleader = "_"
```

映射将定义为 "_a"。如果用户没有这样做，Vim 将使用缺省值反斜杠。这样就会定义一个映射 - "\a"。

注意 其中用到了 <unique>，这会使得 Vim 在映射已经存在时给出错误信息。
:map-<unique>

但是如果用户希望定义自己的键操作呢？我们可以用下面的方法来解决：

```
21      if !hasmapto('<Plug>TypecorrAdd')
22        map <unique> <Leader>a  <Plug>TypecorrAdd
23      endif
```

我们先检查对 "<Plug>TypecorrAdd" 的映射是否存在。仅当不存在时我们才定义映射 "<Leader>a"。这样用户就可以在他自己的 vimrc 文件中加入：

```
map ,c  <Plug>TypecorrAdd
```

那么键序列就会是 ",c" 而不是 "_a" 或者 "\a" 了。

分 割

如果一个脚本变得相当长，你通常希望将其分割成几部分。常见做法是函数或映射。但同时，你又不希望脚本之间这些函数或映射相互干扰。例如，你定义了一个函数 Add()，但另一个脚本可能也试图定一同名的函数。为了避免这样的情况发生，我们可以在局部函数的前面加上 "s:"。

我们来定义一个用来添加新的错误更正的函数：

```
30      function s:Add(from, correct)
31        let to = input("type the correction for " . a:from . ": ")
32        exe ":iabbrev " . a:from . " " . to
..
36      endfunction
```

这样我们就可以在这个脚本之内调用函数 s:Add()。如果另一个脚本也定义 s:Add()，该函数将只能在其所定义的脚本内部被调用。独立于这两个函数的全局的 Add() 函数（不带 "s:"）也可以存在。

映射则可用 <SID>。它产生一个脚本 ID。在我们的错误更正插件中我们可以做以下的定义：

```
24      noremap <unique> <script> <Plug>TypecorrAdd  <SID>Add
..
28      noremap <SID>Add  :call <SID>Add(expand("<cword>"), 1)<CR>
```

这样当用户键入 "\a" 时，将触发下面的次序：

```
\a  ->  <Plug>TypecorrAdd  ->  <SID>Add  ->  :call <SID>Add()
```

如果另一个脚本也定义了映射 <SID>Add，该脚本将产生另一个脚本 ID。所以它定义的映射也与前面定义的不同。

注意 在这里我们用了 <SID>Add() 而不是 s:Add()。这是因为该映射将被用户键入，因此是从脚本外部调用的。<SID> 被翻译成该脚本的 ID。这样 Vim 就知道在哪个脚本里寻找相应的 Add() 函数了。

这的确是有点复杂，但又是使多个插件一起工作所必需的。基本规则是：在映射中使用 <SID>Add()；在其它地方（该脚本内部，自动命令，用户命令）使用 s:Add()。

我们还可以增加菜单项目来完成映射同样的功能：

```
26      noremenu <script> Plugin.Add\ Correction      <SID>Add
```

建议把插件定义的菜单项都加入到 "Plugin" 菜单下。上面的情况只定义了一个菜单选项。当有多个选项时，可以创建一个子菜单。例如，一个提供 CVS 操作的插件可以添加 "Plugin.CVS" 子菜单，并在其中定义 "Plugin.CVS.checkin"，"Plugin.CVS.checkout" 等菜单项。

注意 为了避免其它映射引起麻烦，在第 28 行使用了 ":noremap"。比如有人可能重新映射了 ":call"。在第 24 也用到了 ":noremap"，但我们又希望重新映射 "<SID>Add"。这就是为什么在这儿要用 "<script>"。它允许只执行局部于脚本的映射。
:map-<script> 同样的道理也适用于第 26 行的 ":noremenu"。 :menu-<script>

<SID> 和 <Plug> using-<Plug>

<SID> 和 <Plug> 都是用来避免映射的键序列和那些仅仅用于其它映射的映射起冲突。
注意 <SID> 和 <Plug> 的区别：

<Plug> 在脚本外部是可见的。它被用来定义那些用户可能定义映射的映射。<Plug> 是无法用键盘输入的特殊代码。
使用结构：<Plug> 脚本名 映射名，可以使得其它插件使用同样次序的字符来定义映射的几率变得非常小。在我们上面的例子中，脚本名是 "Typecorr"，映射名是 "Add"。结果是 "<Plug>TypecorrAdd"。只有脚本名和映射名的第一个字符是大写的，所以我们可以清楚地看到映射名从什么地方开始。

<SID> 是脚本的 ID，用来唯一的代表一个脚本。Vim 在内部将 <SID> 翻译为 "<SNR>123_"，其中 "123" 可以是任何数字。这样一个函数 "<SID>Add()" 可能在一个脚本中被命名为 "<SNR>11_Add()"，而在另一个脚本中被命名为 "<SNR>22_Add()"。如果你用 ":function" 命令来获得系统中的函数列表你就可以看到了。映射中对 <SID> 的翻译是完全一样的。这样你才有可能通过一个映射来调用某个脚本中的局部函数。

用 户 命 令

现在让我们来定义一个用来添加更正的用户命令：

```
38      if !exists(":Correct")
```

```
39        command -nargs=1  Correct  :call s:Add(<q-args>, 0)
40      endif
```

这个用户命令只在系统中没有同样名称的命令时才被定义。否则我们会得到一个错误。用 ":command!" 来覆盖现存的用户命令是个坏主意。这很可能使用户不明白自己定义的命令为什么不起作用。 :command

脚 本 变 量

当一个变量前面带有 "s:" 时，我们将它称为脚本变量。该变量只能在脚本内部被使用。在脚本以外该变量是不可见的。这样就避免了在不同的脚本中使用同一个变量名的麻烦。该变量在 Vim 的运行期间都是可用的。当再次调用 (source) 该脚本时使用的是同一个变量。 s:var

有趣的是这些变量也可以在脚本定义的函数、自动命令和用户命令中使用。在我们的例子中我们可以加入几行用来统计更正的个数：

```
19      let s:count = 4
..
30      function s:Add(from, correct)
..
34        let s:count = s:count + 1
35        echo s:count . " corrections now"
36      endfunction
```

起初 s:count 被脚本初始化为 4。当后来 s:Add() 函数被调用时，其值被增加了。在哪里调用函数无关紧要。只要它是定义在该脚本以内的，就可以使用脚本中的局部变量。

结 果

下面就是完整的例子：

```
 1      " Vim global plugin for correcting typing mistakes
 2      " Last Change: 2000 Oct 15
 3      " Maintainer: Bram Moolenaar <Bram@vim.org>
 4      " License:      This file is placed in the public domain.
 5
 6      if exists("g:loaded_typecorr")
 7        finish
 8      endif
 9      let g:loaded_typecorr = 1
10
11      let s:save_cpo = &cpo
12      set cpo&vim
13
14      iabbrev teh the
15      iabbrev otehr other
16      iabbrev wnat want
17      iabbrev synchronisation
18              \ synchronization
19      let s:count = 4
20
```

```
21      if !hasmapto('<Plug>TypecorrAdd')
22        map <unique> <Leader>a  <Plug>TypecorrAdd
23      endif
24      noremap <unique> <script> <Plug>TypecorrAdd  <SID>Add
25
26      noremenu <script> Plugin.Add\ Correction      <SID>Add
27
28      noremap <SID>Add  :call <SID>Add(expand("<cword>"), 1)<CR>
29
30      function s:Add(from, correct)
31        let to = input("type the correction for " . a:from . ": ")
32        exe ":iabbrev " . a:from . " " . to
33        if a:correct | exe "normal viws\<C-R>\" \b\e" | endif
34        let s:count = s:count + 1
35        echo s:count . " corrections now"
36      endfunction
37
38      if !exists(":Correct")
39        command -nargs=1  Correct  :call s:Add(<q-args>, 0)
40      endif
41
42      let &cpo = s:save_cpo
43      unlet s:save_cpo
```

第 33 行还没有解释过。它将新定义的更正用在当前光标下的单词上。 :normal 被用来使用新的缩写。 注意 虽然这个函数是被一个以 ":noremap" 定义的映射调用的，这里的映射和缩写还是被展开使用了。

推荐对 'fileformat' 选项使用 "unix" 值。这样的 Vim 脚本就可以在所有系统内使用。对 'fileformat' 选项使用 "dos" 的脚本无法正常的在 Unix 上使用。参见 :source_crnl 。为确保该值被正确设置，在写入文件前执行下面的命令：

```
:set fileformat=unix
```

文 档 write-local-help

给你的插件写一些文档是个好主意。特别是当用户可以自定义其中的某些功能时尤为必要。关于如何安装文档，请查阅 add-local-help 。

下面是一个插件帮助文档的简单例子，名叫 "typecorr.txt"：

```
1       *typecorr.txt*  Plugin for correcting typing mistakes
2
3       If you make typing mistakes, this plugin will have them corrected
4       automatically.
5
6       There are currently only a few corrections.  Add your own if you like.
7
8       Mappings:
9       <Leader>a   or   <Plug>TypecorrAdd
10              Add a correction for the word under the cursor.
11
12      Commands:
```

```
13      :Correct {word}
14              Add a correction for {word}.
15
16                                                      *typecorr-settings*
17      This plugin doesn't have any settings.
```

其实只有第一行是文档的格式所必需的。Vim 将从该帮助文件中提取该行并加入到 help.txt 的 "LOCAL ADDITIONS:" local-additions （本地附加文档）一节中。第一个 "*" 一定要在第一行的第一列。加入你的帮助文件之后用 ":help" 来检查一下各项是否很好的对齐了。

你可以为你的帮助文档在 ** 之间加入更多的标签。但请注意不要使用现存的帮助标签。你最好能在标签内使用插件名用以区别，比如上例中的 "typecorr-settings"。

建议使用 || 来引用帮助系统中的其它部分。这可以使用户很容易得找到相关联的帮助。

文件类型检测 plugin-filetype

如果 Vim 还不能检测到你的文件类型，你需要在单独的文件里创立一个文件类型检测的代码段。通常，它的形式是一个自动命令，它在文件名字匹配某模式时设置文件类型。例如：

```
au BufNewFile,BufRead *.foo                     set filetype=foofoo
```

把这个一行的文件写到 'runtimepath' 里第一个目录下的 "ftdetect/foofoo.vim"。Unix 上应该是 "~/.vim/ftdetect/foofoo.vim"。惯例是，使用文件类型的名字作为脚本的名字。

如果你愿意，你可以使用更复杂的检查。比如检查文件的内容以确定使用的语言。另见 new-filetype 。

小结 plugin-special

关于插件的小结：

s:name	脚本的局部变量。
<SID>	脚本 ID，用于局部于脚本的映射和函数。
hasmapto()	用来检测插件定义的映射是否已经存在的函数。
<Leader>	"mapleader" 的值。用户可以通过该变量定义插件所定义映射的起始字符。
:map <unique>	如果一个映射已经存在的话，给出警告信息。
:noremap <script>	在映射右边仅执行脚本的局部映射，而不检查全局映射。
exists(":Cmd")	检查一个用户命令是否存在。

41.12 编写文件类型插件 write-filetype-plugin ftplugin

文件类型插件和全局插件其实很相似。但是它的选项设置和映射等仅对当前缓冲区有效。这类插件的用法请参阅 add-filetype-plugin 。

请先阅读上面 41.11 关于全局插件的叙述。其中所讲的对文件类型插件也都适用。这里只讲述一些不同之处。最根本的区别是文件类型插件只应该对当前缓冲区生效。

禁 用

如果你在编写一个提供很多人使用的文件类型插件，这些用户应该有机会选择不加载该插件。你应该在插件的顶端加上：

```
" Only do this when not done yet for this buffer
if exists("b:did_ftplugin")
  finish
endif
let b:did_ftplugin = 1
```

这同时也避免了同一插件在同一缓冲区内被多次执行的错误（当使用不带参数的 ":edit" 命令时就会发生）。

现在用户只要编写一个如下的一行的文件类型插件就可以完全避免加载缺省的文件类型插件了：

```
let b:did_ftplugin = 1
```

当然这要求该文件类型插件所处的文件类型插件目录在 'runtimepath' 所处的位置在 $VIMRUNTIME 之前！

如果你的确希望使用缺省的插件，但是又想自行支配其中的某一选项，你可以用一个类似下例的插件：

```
set textwidth=70
```

现在将这个文件存入那个 "after" 目录中。这样它就会在调用 Vim 本身的 "vim.vim" 文件类型插件之后被调用 after-directory 。对于 Unix 系统而言，该目录会是 "~/.vim/after/ftplugin/vim.vim"。注意 缺省的插件设置了 "b:did_ftplugin"，但在此脚本应该忽略之。

选 项

为了确保文件类型插件仅仅影响当前缓冲区，应该使用

```
:setlocal
```

命令来设置选项。还要注意 只设定缓冲区的局部选项（查查有关选项的帮助）。当 :setlocal 被用于设置全局选项或者某窗口的局部选项时，会影响到多个缓冲区，这是文件类型插件应该避免的。

当一个选项的值是多个标志位或项目的 "合" 时，考虑使用 "+=" 和 "-="，这样可以保留现有的值。注意 用户可能已经改变了该选项的值了。通常先将选项的值复位成缺省值

再做改动是个好主意。例：

```
:setlocal formatoptions& formatoptions+=ro
```

映 射

为了确保键盘映射只对当前缓冲区生效，应该使用

```
:map <buffer>
```

命令。这还应该和上面讲述的两步映射法连起来使用。下面是一个例子：

```
if !hasmapto('<Plug>JavaImport')
  map <buffer> <unique> <LocalLeader>i <Plug>JavaImport
endif
noremap <buffer> <unique> <Plug>JavaImport oimport ""<Left><Esc>
```

`hasmapto()` 被用来检查用户是否已经定义了一个对 `<Plug>JavaImport` 的映射。如果没有，该文件类型插件就定义缺省的映射。因为缺省映射是以 `<LocalLeader>` 开始，就使得用户可以自己定义映射的起始字符。缺省的是反斜杠。
"`<unique>`" 的用途是当已经存在的了这样的映射或者和已经存在的映射有重叠的时候给出错误信息。
`:noremap` 被用来防止其他用户定义的映射干扰。不过，":noremap `<script>`" 仍然可以允许进行脚本中以 `<SID>` 开头的映射。

一定要给用户保留禁止一个文件类型插件内的映射而不影响其它功能的能力。下面通过一个邮件文件类型插件来演示如何做到这一点：

```
" 增加映射，除非用户反对。
if !exists("no_plugin_maps") && !exists("no_mail_maps")
  " Quote text by inserting "> "
  if !hasmapto('<Plug>MailQuote')
    vmap <buffer> <LocalLeader>q <Plug>MailQuote
    nmap <buffer> <LocalLeader>q <Plug>MailQuote
  endif
  vnoremap <buffer> <Plug>MailQuote :s/^/> /<CR>
  nnoremap <buffer> <Plug>MailQuote :.,$s/^/> /<CR>
endif
```

其中用到了两个全局变量：

`no_plugin_maps`	禁止所有文件类型插件中的映射
`no_mail_maps`	禁止 "mail" 文件类型插件的映射

用 户 命 令

在使用 `:command` 命令时，如果加上 "-buffer" 开关，就可以为某一类型的文件加入一个用户命令，而该命令又只能用于一个缓冲区。例：

```
:command -buffer  Make  make %:r.s
```

变 量

文件类型插件对每一个该类型的文件都会被调用。脚本局部变量 s:var 会被所有的调用共享。如果你想定义一个仅对某个缓冲区生效的变量，使用缓冲区局部变量 b:var 。

函 数

一个函数只需要定义一次就行了。可是文件类型插件会在每次打开相应类型的文件时都被调用。下面的结构可以确保函数只被定义一次：

```
:if !exists("*s:Func")
:  function s:Func(arg)
:    ...
:  endfunction
:endif
```

撤 销 undo_indent undo_ftplugin

当用户执行 ":setfiletype xyz" 时，之前的文件类型命令应该被撤销。在你的文件类型插件中设定 b:undo_ftplugin 变量，用来撤销该插件的各种设置。例如：

```
let b:undo_ftplugin = "setlocal fo< com< tw< commentstring<"
        \ . "| unlet b:match_ignorecase b:match_words b:match_skip"
```

在 ":setlocal" 命令的选项后使用 "<" 会将其值复位为全局值。这可能是最好的复位选项值的方法。

但这也需要把 "C" 标志位从 'cpoptions' 选项中去除，就像上面 use-cpo-save 提到的那样。

要撤销缩进脚本的效果，必须相应地设定 b:undo_indent 变量。

文 件 名

文件类型必须被包括在插件文件名中 ftplugin-name 。可以使用以下三种形式之一：

```
.../ftplugin/stuff.vim
.../ftplugin/stuff_foo.vim
.../ftplugin/stuff/bar.vim
```

"stuff" 是文件类型，"foo" 和 "bar" 是任意名字。

小 结 ftplugin-special

以下是有关文件类型插件一些特殊环节：

<LocalLeader> "maplocalleader" 的值，用户可以通过它来自定义文件类型插件中映射的起始字符。

:map <buffer> 定义一个仅对缓冲区有效的局部映射。

:noremap <script>　　仅重映射脚本中以 <SID> 开始的映射。

:setlocal　　设定仅对当前缓冲区有效的选项。

:command -buffer　　定义一个仅对缓冲区有效的局部命令。

exists("*s:Func")　　查看是否已经定义了某个函数。

参阅所有插件的特殊环节 plugin-special 。

41.13 编写编译器插件 write-compiler-plugin

编译器插件可以用来设定于某一特定编译器相关的选项。用户可以使用 :compiler 命令来加载之。主要是用以设定 'errorformat' 及 'makeprg' 选项。

最简单的方法就是学习一个例子。下面的命令将编辑所有缺省安装的编译器插件：

```
:next $VIMRUNTIME/compiler/*.vim
```

用 :next 可以查阅下一个插件文件。

这类文件有两个特别之处。一是允许用户否决或者增强缺省文件的机制。缺省的文件以下面的代码开始：

```
:if exists("current_compiler")
:  finish
:endif
:let current_compiler = "mine"
```

当你写了编译器文件并把它放到你个人的运行时目录（例如，Unix 上 ~/.vim/compiler）时，你需要设置 "current_compiler" 变量，使得缺省文件不进行设置。

:CompilerSet

第二个特别之处是：用 ":set" 命令来配合 ":compiler!" 而用 ":setlocal" 来配合 ":compiler"。Vim 为此定义了 ":CompilerSet" 用户命令。然而旧版本的 Vim 没有，因此你的插件应该提供该命令。下面是一个例子：

```
if exists(":CompilerSet") != 2
  command -nargs=* CompilerSet setlocal <args>
endif
CompilerSet errorformat&              " use the default 'errorformat'
CompilerSet makeprg=nmake
```

当你为 Vim 发布版本或者整个系统编写编译器插件时，应该使用上面提到的机制。这样当用户插件已经定义了 "current_compiler" 的时候什么也不做。

当你为了自行定义缺省插件的一些设定而编写编译器插件时，不要检查 "current_compiler"。这个插件应该在最后加载，因此其所在目录应该在 'runtimepath' 的最后。对于 Unix 来说可能是 ~/.vim/after/compiler。

41.14 编写快速载入的插件 write-plugin-quickload

插件会不断变大而使代码越来越长。启动时的延迟因而会非常显著，而你可能几乎不用这个插件。这时，就是时候让我们来编写可以快速载入的插件了。

基本的方法是调用插件两次。第一次定义用户命令和映射，提供需要的功能。第二次定义实现这些功能的函数。

听起来很吓人，快速载入意味着载入两次！我们的意思是，第一次载入很快，把脚本的大部分内容延迟到第二次才载入，只有实际使用这些功能时才会这么做。当然，如果你总是用这些功能，实际上更慢了！

注意 从 Vim 7 开始，有一个替代方法：使用 autoload 功能 41.15 。

下例演示如何这是如何完成的：

```
" 演示快速载入的 Vim 全局插件
" Last Change:  2005 Feb 25
" Maintainer:   Bram Moolenaar <Bram@vim.org>
" License:      This file is placed in the public domain.

if !exists("s:did_load")
        command -nargs=* BNRead  call BufNetRead(<f-args>)
        map <F19> :call BufNetWrite('something')<CR>

        let s:did_load = 1
        exe 'au FuncUndefined BufNet* source ' . expand('<sfile>')
        finish
endif

function BufNetRead(...)
        echo 'BufNetRead(' . string(a:000) . ')'
        " 读入功能在此
endfunction

function BufNetWrite(...)
        echo 'BufNetWrite(' . string(a:000) . ')'
        " 写回功能在此
endfunction
```

第一次载入脚本时，没有设置 "s:did_load"。这时执行 "if" 和 "endif" 之间的命令。它以 :finish 命令结束，这样脚本的其余部分不再执行。

第二次载入脚本时，"s:did_load" 已经存在，这时执行 "endif" 之后的命令。这里定义 (可能很长的) BufNetRead() 和 BufNetWrite() 函数。

如果把该脚本放到你的 plugin 目录，Vim 启动时会执行它。下面列出发生事件的序列：

1. 启动期间执行脚本时，定义 "BNRead" 命令并映射 <F19> 键。定义 FuncUndefined 自动命令。":finish" 命令使脚本提前终止。

2. 用户输入 BNRead 命令或者按了 <F19> 键。BufNetRead() 或 BufNetWrite() 函数会被调用。

3. Vim 找不到这些函数并因此激活了 FuncUndefined 自动命令事件。因为模式 "BufNet*" 匹配要调用的函数，执行命令 "source fname"，其中 "fname" 被赋予脚

本的名字，不管它实际在何处都没问题。这是因为该名字来自 "<sfile>" 扩展的结果（见 expand() ）。

4. 再次执行脚本。"s:did_load" 变量已经存在，此时定义函数。

注意 后来载入的函数匹配 FuncUndefined 自动命令的模式。要确信其它插件没有定义匹配此模式的函数。

41.15 编写库脚本 write-library-script

有些功能会在多个地方调用。如果这已经不是一两行的代码，你可能会希望把这些代码放进脚本，然后被许多其它脚本使用。我们把这种脚本称为库脚本。

可以手动载入库脚本，只要你不要重复调用它就行了。用 exists() 函数可以判断。例如：

```
if !exists('*MyLibFunction')
   runtime library/mylibscript.vim
endif
call MyLibFunction(arg)
```

这里你需要知道 MyLibFunction() 在脚本 "library/mylibscript.vim" 里定义，该脚本在 'runtimepath' 的某个目录里。

为了稍稍简化，Vim 提供了自动载入机制。现在，本例看起来像：

```
call mylib#myfunction(arg)
```

简单多了，是不是？Vim 会识别函数名，如果该函数还没有定义，查找 'runtimepath' 里的 "autoload/mylib.vim"。该脚本必须定义 "mylib#myfunction()" 函数。

在 mylib.vim 脚本里可以放上许多其它函数，你可以自由组织库脚本的函数。但必须使函数名 "#" 前面的部分匹配脚本名。否则 Vim 无法知道载入哪个脚本。

如果你真的热情高涨写了很多库脚本，现在可能想要用子目录吧。例如：

```
call netlib#ftp#read('somefile')
```

Unix 上，这里使用的库脚本可以是：

```
~/.vim/autoload/netlib/ftp.vim
```

其中的函数应该如此定义：

```
function netlib#ftp#read(fname)
        "  用 ftp 读入文件 fname
endfunction
```

注意 定义所用的函数名必须和调用的函数名完全相同。最后一个 '#' 之前的部分必须准确匹配子目录和脚本名。

同样的机制可以用来定义变量：

```
let weekdays = dutch#weekdays
```

会载入脚本 "autoload/dutch.vim"，它应该包含这样的内容：

```
let dutch#weekdays = ['zondag', 'maandag', 'dinsdag', 'woensdag',
      \ 'donderdag', 'vrijdag', 'zaterdag']
```

进一步的阅读可见： autoload 。

41.16 发布 Vim 脚本 distribute-script

Vim 用户可以在 Vim 网站上寻找脚本：http://www.vim.org。如果你实现了对别人也有用的功能，让大家一起分享！

Vim 脚本应该可以用于任何系统。它们不一定有 tar 或 gzip 命令。如果你想把文件打包和/或进行压缩，建议使用 "zip" 工具。

最理想的可移植方法是让 Vim 自己给脚本打包，用 Vimball 工具。见 vimball 。

最好你能加入一行内容，实现自动更新。见 glvs-plugins 。

下一章： usr_42.txt 添加新的菜单

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2 http://vimcdoc.sf.net

添加新菜单

现在你应该已经了解到 Vim 是非常灵活的。这也包括在 GUI 中使用的菜单。你可以定义你自己的菜单项来更方便的使用一些命令。这是为那些鼠标爱好者准备的。

42.1 简介
42.2 菜单命令
42.3 杂项
42.4 工具栏和弹出菜单

下一章: usr_43.txt 使用文件类型
前一章: usr_41.txt 编写 Vim 脚本
目录: usr_toc.txt

42.1 简介

Vim 使用的菜单是定义在 "$VIMRUNTIME/menu.vim" 文件里的。如果你想编写自己的菜单，你可以先看看那个文件。

使用 ":menu" 可以定义一个菜单项。这个命令的基本格式如下:

```
:menu {menu-item} {keys}
```

{menu-item} 告诉 Vim 把菜单项放在哪。"File.Save" 是一个典型的例子。它表示 "File" 菜单下的 "Save" 菜单项。那个点被用来分隔菜单的名字。例:

```
:menu File.Save  :update<CR>
```

":update" 命令在文件被修改时写入文件。

你可以再加一层: "Edit.Settings.Shiftwidth" 在 "Edit" 菜单下定义一个 "Settings" 子菜单，然后在其下定义一个菜单项 "Shiftwidth"。你还可以再加更深的层。但别用太多，那样要找到一个菜单层就需要太多的鼠标动作。

":menu" 命令和 :map 命令很像: 左边指定了该菜单项如何被触发，右边定义它将执行的字符串。{keys} 只是一串字符，Vim 会像你键入它们一样使用该字符串。因此在插入模式下，当 {keys} 不包括特殊字符时，Vim 会插入那些字符。

菜 单 快 捷 键

& 字符可以被用来指示一个菜单快捷键。例如，你可以使用 Alt-F 来选择 "File" 然后 S 来选中 "Save"。(不过 'winaltkeys' 选项可以关闭这种操作！)。因此，该菜单项的 {menu-item} 是 "&File.&Save"。快捷键使用的字符在菜单中会被加上下划线。

你必须自己留神不要在同一菜单中使用两个一样的快捷键，那样你就不知道哪个会被选中。Vim 是不会提醒你的。

优 先 级

File.Save 真正的定义是：

```
:menu 10.340 &File.&Save<Tab>:w  :confirm w<CR>
```

数字 10.340 叫优先级数。Vim 用它来决定把该菜单放在哪里。第一个数 (10) 表示该菜单在菜单栏上的位置。优先级数较小的菜单会被放在左边，大数字对应的菜单会被放在右边。下面是标准菜单的优先级：

```
  10    20     40     50      60       70                9999

+-------------------------------------------------------------+
| File  Edit  Tools  Syntax  Buffers  Window            Help |
+-------------------------------------------------------------+
```

Help 菜单被指定了一个很大的值，这样它就会被放置在最右边。

第二个数字 (340) 决定菜单项在下拉菜单中的位置。较小的数字在上边，大的在下边。下面是 File 菜单的优先级：

```
                +-----------------+
   10.310       |Open...          |
   10.320       |Split-Open...    |
   10.325       |New              |
   10.330       |Close            |
   10.335       |---------------- |
   10.340       |Save             |
   10.350       |Save As...       |
   10.400       |---------------- |
   10.410       |Split Diff with  |
   10.420       |Split Patched By |
   10.500       |---------------- |
   10.510       |Print            |
   10.600       |---------------- |
   10.610       |Save-Exit        |
   10.620       |Exit             |
                +-----------------+
```

注意 这些数值不是连续的。那些没有被使用的值可以被用来插入你自己定义的项。(通常情况下，最好不要改动那些标准菜单。如果需要，你可以创建新的菜单)。

当创建子菜单时，你可以加入另一级数字。这样每一个菜单项都有其优先级数。

特 殊 字 符

这里用的 {menu-item} 的例子是 "&File.&Save<Tab>:w"。这里有很重要一点：{menu-item} 必须是一个单词。如果你想加一个点，一个空格或制表符，你要么使用 <> 记法 (例如 <Space> 和 <Tab>) 或使用反斜杠。

```
:menu 10.305 &File.&Do\ It\.\.\. :exit<CR>
```

上面例子中，菜单项名 "Do It..." 包括一个空格，执行的命令是 ":exit<CR>"。

菜单名中的 <Tab> 字符可以被用来分隔菜单定义部分和用户提示部分。<Tab> 之后的部分会被以右对齐格式显示。File.Save 菜单项的定义是 "&File.&Save<Tab>:w"。因此该菜单名是 "File.Save"；提示是 ":w"。

分隔符

分隔符（线）可以用来将相关的菜单放在一起。开始和最后一个字符都是 '-' 的菜单名就会认为是菜单分隔符，例如 "-sep-"。在同一个菜单中使用多个分隔符时，它们的名字必须被区别开，否则其名字不相干。

分隔符所相关的命令永远不会被执行。但你同样得定义一个。用一个冒号就行了。例如：

```
:amenu 20.510 Edit.-sep3- :
```

42.2 菜单命令

你可以为某一特定的运行模式定义专用的菜单。就好像各种 ":map" 命令的变体一样：

:menu	普通、可视和操作符等待模式
:nmenu	普通模式
:vmenu	可视模式
:omenu	操作符等待模式
:menu!	插入和命令行模式
:imenu	插入模式
:cmenu	命令行模式
:tlmenu	终端模式
:amenu	所有模式（除了终端模式外）

为了避免菜单项执行的命令被重映射，使用 ":noremenu"、":nnoremenu"、"anoremenu" 等形式。

使用 :AMENU

":amenu" 命令有些不同。它假定你给出的 {keys} 是被用在普通模式下的。当 Vim 在可视或插入模式下运行并执行这些菜单命令时，Vim 先回到普通模式。":amenu" 会替你插入 CTRL-C 或 CTRL-O。例如，如果你使用这个命令：

```
:amenu  90.100 Mine.Find\ Word  *
```

你得到的菜单命令将会是：

```
Normal mode:            *
Visual mode:            CTRL-C *
Operator-pending mode:  CTRL-C *
Insert mode:            CTRL-O *
Command-line mode:      CTRL-C *
```

在命令行模式下 CTRL-C 会清除已键入的命令。在可视和操作符等待模式下 CTRL-C 会终止该模式。在插入模式下 CTRL-O 会执行该命令然后回到插入模式。

CTRL-O 只能用来执行一个命令。如果你要执行多于两个命令，你可以把它们定义成一个函数然后在菜单定义中调用该函数。例如：

```
:amenu  Mine.Next\ File  :call <SID>NextFile()<CR>
:function <SID>NextFile()
```

```
:  next
:  1/^Code
:endfunction
```

这个菜单项用 ":next" 命令移动到参数列表中的下一个文件，然后查找其中以 "Code" 开始的行。

函数名前的 <SID> 是一个脚本 ID。这使得该函数成为当前 Vim 脚本文件的局部函数。这是用来避免在其他脚本定义了相同名字的函数的麻烦。见 <SID> 。

默 菜 单

Vim 像你键入 {keys} 一样执行菜单命令。对于 ":" 命令这就意味着你将在命令行上看到该命令及其输出。如果命令很长的话，那个令人讨厌的 hit-Enter 提示就会出现。

为了避免这种情况，让那个菜单保持缄默。用 <silent> 参数就可以了。拿前面例子中的 NextFile() 调用来说，当你使用该菜单时，你将在命令行上看到：

```
:call <SNR>34_NextFile()
```

为了避免这些文本被显示，你可以用 "<silent>" 作为第一个参数：

```
:amenu <silent> Mine.Next\ File :call <SID>NextFile()<CR>
```

别滥用 "<silent>"。对于短的命令来说是不需要的。如果你定义一个菜单给其他人使用，让他看到被执行的命令会提示他如何只用键盘来执行这些操作而无需大费周章使用鼠标。

菜 单 列 表

当执行 menu 命令时不带 {keys} 部分，列出已经定义的菜单。你可以给出 {menu-item} 或其中的一部分，来列出指定的菜单。例如：

```
:amenu
```

这将列出所有菜单。太长了！最好指定菜单名来减短一些：

```
:amenu Edit
```

这只列出 "Edit" 菜单下的为所有模式定义的各项。要只列出一个为插入模式定义的菜单项：

```
:imenu Edit.Undo
```

注意 要使用准确的名字。大小写是有区别的。但是 '&' 可以被省略。<Tab> 以及其后的提示也可以被省略。

删 除 菜 单

要删除一个菜单，使用 "unmenu" 命令加上和列出菜单名的命令一样的参数。这样，":menu" 变成 ":unmenu"，":nmenu" 变成 ":nunmenu"，依此类推。要删除为插入模式定义的 "Tools.Make" 菜单项：

```
:iunmenu Tools.Make
```

你也可以用菜单名来删除包括所有子项的整个菜单。例如:

```
:aunmenu Syntax
```

这会删除 Syntax 菜单及其所有菜单项。

42.3 杂项

你可以利用 'guioptions' 选项来改变菜单的外观。缺省情况下除了 "M" 以外，所有的标志位都将被使用。你可以用下面的标志位来选择性去除菜单的某一部分。这些标志位的使用如下:

```
:set guioptions-=m
```

m	当该位被清除时整个菜单都会消失。
M	加入该位时，缺省菜单不会被加载。
g	当该位被清除时那些无效菜单会完全消失而不是仅仅被染灰(并非所有的系统都有效。)
t	当该位被清除时不使用可撕下菜单功能。

一个菜单顶部的那个用点组成的行不是分隔符。当你选中该行时，该菜单会被 "撕下"。它显示在单独的窗口里，亦称撕下菜单。当你经常使用同一菜单的时候，这是非常方便的。

要翻译菜单项，见 :menutrans 。

既然选择菜单项需要使用鼠标，最好在菜单项里使用 ":browse" 命令来选择文件；使用 ":confirm" 来获得一个对话框。这两个也可以联起来用:

```
:amenu File.Open  :browse confirm edit<CR>
```

":browse" 弹出一个文件浏览器用来选择文件。":confirm" 在当前文件改动后会自动弹出一个确认的对话框。你可以选择保存或放弃改动，或者取消命令。

对于更复杂的菜单项，可以使用 confirm() 和 inputdialog() 函数。系统缺省的菜单包含若干实例。

42.4 工具栏和弹出菜单

Vim 中有两个特殊的菜单: ToolBar (工具栏) 和 PopUp (弹出菜单)。以这两个名字开始的菜单不会出现在一般的菜单栏里。

工 具 栏

只有当 'guioptions' 选项中包括 "T" 标志位时，工具栏才会出现。

工具栏使用图标来表示一个命令，而不是文字。例如，名为 "ToolBar.New" 的

{menu-item} 会在工具栏里显示一个 "New" 图标。

Vim 有 28 个内置的图标。这里可以找到一个列表： builtin-tools 。大多数是用在缺省的工具栏里的。你可以重新定义这些工具栏项所执行的操作。(在缺省菜单被设定之后)。

你可以为工具栏项添加另外的位图，或者定义新的工具栏项。例如：

```
:tmenu ToolBar.Compile  Compile the current file
:amenu ToolBar.Compile  :!cc % -o %:r:S<CR>
```

现在你需要创建图标。对于 MS-Windows 图标必须是名为 "Compile.bmp" 的位图格式文件。对于 Unix 系统必须是名为 "Compile.xpm" 的 XPM 格式文件。大小必须是 18 X 18 个像素。对于 MS-Windows 来说也可以使用其它大小的图标，但可能看起来会比较难看。

把位图放到 'runtimepath' 其中一个目录下的 "bitmaps" 子目录内。例如：对于 Unix 系统可以使用 "~/.vim/bitmaps/Compile.xpm" 目录。

你也可以为工具栏项定义一个工具提示。工具提示是解释工具栏项功能的简短文字。例如 "打开文件"。当鼠标移动到该工具栏项上并停留一会时，该提示会被自动显示。当图片的意图不是很明显时，这是很有用的。
例如：

```
:tmenu ToolBar.Make  Run make in the current directory
```

备注：
注意大小写的使用。"Toolbar" 和 "toolbar" 和 "ToolBar" 是不同的！

要去除一个工具提示，可以使用 :tunmenu 命令。

'toolbar' 选项可以被用来在工具栏上显示文本而不是图标，或两者都显示。大多数人只使用位图，因为文本会占用太多空间。

弹 出 菜 单

弹出菜单会被显示在鼠标所在的位置。在 MS-Windows 系统上你点击鼠标右键就可以激活弹出菜单，然后用鼠标左键选择项目。在 Unix 系统上激活该菜单需要按住鼠标右键。

只有当 'mousemodel' 选项被设定为 "popup" 或 "popup_setpos" 时，弹出菜单才有效。两者之间的区别在于 "popup_setpos" 会将光标移动到鼠标指针所在处。当鼠标在一个选择区内时，选中的区域不受影响。当鼠标点击在选中区域外时，该选中的区域会被删除。

对于每一个运行模式分别有一个独立的弹出菜单。所以永远不会有一般菜单中的灰色菜单项。

What is the meaning of life, the universe and everything? 42
Douglas Adams, the only person who knew what this question really was about is now dead, unfortunately. So now you might wonder what the meaning of death is...

生命的意义是什么？宇宙以及一切的一切的意义又是什么？
不幸的是，唯一了解这个问题的人，Douglas Adams 已经死了。所以你现在可能正在冥想死亡的意义是什么...
(译者注：Douglas Adams 写的科幻小说里，超级电脑对这个问题的回答是：42。)

下一章: usr_43.txt 使用文件类型

版权: 参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2 http://vimcdoc.sf.net

使用文件类型

当你在编辑某一类型文件时，例如一个 C 程序或者一个外壳脚本，你通常重复使用同样的设置和键盘映射。很快你就会对每一次都要手动设置这些感到厌烦。这一章就告诉你如果自动化这些设置。

43.1 为一类文件编写的插件
43.2 添加一个文件类型

后一章: usr_44.txt 自定义语法高亮
前一章: usr_42.txt 添加新的菜单
目录: usr_toc.txt

43.1 为一类文件编写的插件 filetype-plugin

如何使用文件类型插件已经在 add-filetype-plugin 这里讨论过了。不过你很可能对缺省的最基本的设定不满意。假定对于所有 C 文件你希望将 'softtabstop' 选项定为 4 并定义一个插入三行注释的键盘映射。下面的两步可以做到:

your-runtime-dir

1. 创建你自己的运行时目录。在 Unix 上通常是 "~/.vim"。在这个目录下创建 "ftplugin" 目录:

```
mkdir ~/.vim
mkdir ~/.vim/ftplugin
```

如果你不是 Unix 用户的话，看看 'runtimepath' 选项的值就知道 Vim 在哪里找 "ftplugin" 目录了:

```
set runtimepath
```

通常你应该使用该列表中的第一个目录名 (第一个逗号之前的那个)。如果你不喜欢缺省值的话，你也可以在 vimrc 文件里把自己的目录名加到 'runtimepath' 选项的最前面。

2. 创建 "~/.vim/ftplugin/c.vim" 文件，并加入以下内容:

```
setlocal softtabstop=4
noremap <buffer> <LocalLeader>c o/**************<CR><CR>/<Esc>
let b:undo_ftplugin = "setl softtabstop< | unmap <buffer> <LocalLeader>c"
```

现在试着编辑一个 C 文件。你就会注意到 'softtabstop' 选项的值已经被设为 4 了。但是当你编辑另外的文件的时候就会被复位到 0。那是因为用了 ":setlocal" 命令。这样，对 'softtabstop' 选项的设置仅对本缓冲区有效。一旦你编辑另外一个文件，该选项的值就会被设定成那个缓冲区的缺省值，或者最近一次被 ":set" 命令设定的值。

同样地，键盘映射 "\c" 在编辑另外一个缓冲区时就不起作用了。":map <buffer>" 命令

建立了一个仅对当前缓冲区有效的映射。其它映射命令 ":map!", ":vmap" 等也是如此。映射中的 <LocalLeader> 被 "maplocalleader" 变量的值所替代。

b:undo_ftplugin 的设定用于文件类型被设成其它值的时候。那种情况下你会想撤销自己的首选项。此 b:undo_ftplugin 变量作为命令执行。小心字符串里有特殊含义的字符，如反斜杠。

在下面这个目录里你可以找到一些文件类型插件的例子：

```
$VIMRUNTIME/ftplugin/
```

进一步关于为某一类文件编写插件的知识可以在这里读到： write-plugin 。

43.2 添加一个文件类型

如果你正在使用一种 Vim 不认识的文件，这一节告诉你怎么将这种文件介绍给 Vim。你需要一个自己的运行时目录。参阅上面的 your-runtime-dir 。

创建一个文件 "filetype.vim" 并加入一个为你的文件类型编写的自动命令。(关于自动命令的阐述在 40.3 。) 例：

```
augroup filetypedetect
au BufNewFile,BufRead *.xyz     setf xyz
augroup END
```

这样所有以 ".xyz" 结尾的文件将被认为 "xyz" 类型的文件。":augroup" 命令将该自动命令加入到 "filetypedetect" 组。这样做的作用是：当用户用 ":filetype off" 命令的命令时，所有文件类型检测的自动命令都被忽略掉。"setf" 命令将 'filetype' 选项设为该命令的参数，除非该选项已经被设置过。该命令保证 'filetype' 不会被重复设定。

你可以使用各种各样的模式来匹配你的文件名。也可以包括目录名。见 autocmd-patterns 。例如，"/usr/share/scripts/" 目录下的文件都是 "ruby" 文件，但没有文件扩展名。加入如下一行就可以了：

```
augroup filetypedetect
au BufNewFile,BufRead *.xyz                     setf xyz
au BufNewFile,BufRead /usr/share/scripts/*      setf ruby
augroup END
```

然而，如果你编辑一个叫做 /usr/share/scripts/README.txt 的文件，那可不是 ruby 文件。使用以 "*" 结尾的模式的不妥之处就在于它会匹配过多的文件。为了避免这种情况，把那个 filetype.vim 文件放到位于 'runtimepath' 最后的那个目录。以 Unix 为例，你可以用 "~/.vim/after/filetype.vim"。

现在你可以把文本文件的检测加入 ~/.vim/filetype.vim：

```
augroup filetypedetect
au BufNewFile,BufRead *.txt                     setf text
augroup END
```

'runtimepath' 首先找到该文件。最后才是 ~/.vim/after/filetype.vim：

```
augroup filetypedetect
```

```
au BufNewFile,BufRead /usr/share/scripts/*      setf ruby
augroup END
```

Vim 会在每一个 'runtimepath' 列出的目录中查找 "filetype.vim" 文件。先是 ~/.vim/filetype.vim。匹配 *.txt 文件的自动命令是在那里定义的。接着 Vim 找到 $VIMRUNTIME 中的 filetype.vim，因为该目录在 'runtimepath' 的中部。最后才找到 ~/.vim/after/filetype.vim，然后其中检测 ruby 文件的自动命令才被添加到系统中。

现在你在编辑 /usr/share/scripts/README.txt 时，自动命令是以其被定义的次序一一检测的。因为匹配 *.txt 模式，这样就会执行 "setf text" 命令，将文件类型设定为 "text"。之后 ruby 文件的模式也匹配了，"setf ruby" 被执行。但是因为 'filetype' 已经被设为 "text" 了，后者就什么作用也不起了。

当你编辑文件 /usr/share/scripts/foobar 同样的自动命令被检测。但只有 ruby 文件的那个匹配，因此 "setf ruby" 命令将 'filetype' 设为 ruby。

依 内 容 而 定

如果你的文件无法以其文件名决定类型，你可以通过其内容来确定。例如，很多脚本文件都是这样开始的：

```
#!/bin/xyz
```

为了认出这个脚本，在你的运行时目录 (和你的 filetype.vim 在同一个目录) 下创建一个 "scripts.vim" 文件。类似这样：

```
if did_filetype()
  finish
endif
if getline(1) =~ '^#!.*[/\\]xyz\>'
  setf xyz
endif
```

先用 did_filetype() 可以避免你无谓的检查已经被检测出文件类型的文件。这样就不必浪费时间了： "setf" 一点作用也没有。

scripts.vim 文件被缺省的 filetype.vim 文件中的一个自动命令调用。因此，检测的次序是：

1. 'runtimepath' 内 $VIMRUNTIME 之前目录中的 filetype.vim 文件
2. $VIMRUNTIME/filetype.vim 的前部
3. 'runtimepath' 内所有的 script.vim 文件
4. $VIMRUNTIME/filetype.vim 余下的部分
5. 'runtimepath' 内 $VIMRUNTIME 之后目录中的 filetype.vim 文件

如果这样对你还是不够的话，加入一个匹配所有文件的自动命令然后调用一个脚本或者执行一个函数来检查文件的内容。

下一章： usr_44.txt 自定义语法高亮

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者：wandys，tocer http://vimcdoc.sf.net

自定义语法高亮

Vim 对上百种文件都有自动语法高亮。如果你所编辑的文件类型没有被包括，通过阅读本章，你可以掌握对这种文件类型自定义语法高亮的方法。同时请参考 :syn-define 。

44.1 基本语法命令
44.2 关键字
44.3 匹配
44.4 区域
44.5 嵌套项目
44.6 跟随组
44.7 其它参数
44.8 簇
44.9 包含其它语法文件
44.10 同步
44.11 安装语法文件
44.12 可移植语法文件格式

后一章： usr_45.txt 选择你的语言
前一章： usr_43.txt 使用文件类型
目录： usr_toc.txt

44.1 基本语法命令

利用一个已经存在的语法文件可以节省很多时间。首先看 $VIMRUNTIME/syntax 目录下是否存在一种类似语言的语法文件。这些文件同时可以让你了解语法文件的一般布局。为了进一步理解，你需要阅读以下的部分。

让我们先从简单的命令开始。在定义新的语法之前，我们需要清除旧的定义：

```
:syntax clear
```

这在最终的语法文件中不是必须的，但在我们试验时却很有用。

本章有很多简化。如果你要编写一个供别人使用的语法文件，请从头到尾阅读本章以通晓有关细节。

列出已定义的项目

要查看现在已定义的项目，用这个命令：

```
:syntax
```

你可以用此命令查看哪些命令确实已被定义。这在调试新的语法文件时很有用。它还会显示每个项目所用的颜色，可以更好的区分彼此。

列出某个特定语法组的项目，用：

```
:syntax list {group-name}
```

这还可以用来列出簇（在 44.8 有解释）。只需要在名字中包含 '@'。

匹配大小写

一些语言是大小写不敏感的，比如 Pascal。另外一些，比如 C，则是大小写敏感的。你需要用以下命令指出你的类型：

```
:syntax case match
:syntax case ignore
```

参数 "match" 意味着 Vim 区分语法元素的大小写，这样 "int" 和 "Int" 与 "INT" 不同。如果指定 "ignore" 参数，"Procedure"、"PROCEDURE" 和 "procedure" 均视为相同。

":syntax case" 命令可以出现在语法文件的任何地方，并对其后的语法定义产生效用。在大多数情况下，你只需要一个 ":syntax case" 命令；但如果你使用一种不常见的语言，这种语言既包括大小写敏感的元素又包括大小写不敏感的元素，你需要在文件中使用多个 ":syntax case" 命令。

44.2 关键字

最基本的语法元素是关键字。要定义一个关键字，用以下命令：

```
:syntax keyword {group} {keyword} ...
```

{group} 是语法组的名字。用 ":highlight" 命令，你可以对 {group} 设定颜色。{keyword} 参数是关键字。这里有一些例子：

```
:syntax keyword xType int long char
:syntax keyword xStatement if then else endif
```

这个例子中用到了两个名为 "xType" 和 "xStatement" 的语法组。按惯例，每个组名都用所定义语言的文件类型作为前缀。本例中是为 x 语言（样例语言（eXample），无特殊含义）定义语法。在为 "csh" 脚本建立的语法文件中，将用到类似 "cshType" 的名字。这样，前缀和 'filetype' 的值相同。

这些命令使 "int"，"long"，"char" 以一种方式被高亮显示，而 "if"、"then"、"else" 和 "endif" 则以另一种方式被高亮显示。现在你需要将 x 组名和标准的 Vim 名字联系起来：

```
:highlight link xType Type
:highlight link xStatement Statement
```

这告诉 Vim 以 "Type" 的方式高亮显示 "xType"，以 "Statement" 的方式高亮显示 "xStatement"。关于标准名，参考 group-name 。

非常见关键字

在关键字里用到的字符必须在 'iskeyword' 选项内。如果你使用其它的字符，关键字将不被匹配。Vim 对此并不给出警告信息。

x 语言在关键字中用到了 '-' 字符。使用的方法如下：

```
:setlocal iskeyword+=-
:syntax keyword xStatement when-not
```

"setlocal" 命令用来指定只对当前缓冲区改变 'iskeyword'。但仍会对 "w" 和 "*" 等命令造成影响。如果这不是所期望的，不要定义关键字而使用匹配（在下一节内解释）。

x 语言允许缩写。比如，"next" 可以缩写成 "n"、"ne" 或者 "nex"。你可以用这个命令来定义：

```
:syntax keyword xStatement n[ext]
```

这并不会匹配 "nextone"，关键字只匹配完整的单词。

44.3 匹配

试想一下定义更复杂的东西。你想要匹配普通的标识符。对此，你定义一个匹配语法项目。它会匹配任意一个只由小写字母组成的单词：

```
:syntax match xIdentifier /\<\l\+\>/
```

备注：
关键字超越其它的语法项目。这样上面的 ":syntax keyword" 命令定义的 "if"，"then" 等关键字将被视为关键字，即使它们匹配 xIdentifier 的模式。

最后一部分是一个模式（类似查找中的模式）。"//" 用来包含模式（像 ":substitute" 命令中的一样）。你也可以用其它的字符，比如加号或者引号。

现在定义一个注释的匹配。在 x 语言中注释是以 '#' 开头的行：

```
:syntax match xComment /#.*/
```

你可以用任意的查找模式。因此通过一个匹配项目，你可以高亮显示非常复杂的项目。关于查找模式，参考 pattern 。

44.4 区域

在 x 样例语言中，字符串用双引号（"）来包含。要高亮一个字符串，你需要定义一个区域。你需要一个区域的开头（双引号）和一个区域的结尾（双引号）。定义如下所示：

```
:syntax region xString start=/"/ end=/"/
```

"start" 和 "end" 指示定义用于寻找区域开头和结尾的模式。但如果一个有这样的字符串该怎么办呢？

```
"A string with a double quote (\") in it"
```

这会带来一个问题：字符串中间的双引号会结束区域。你需要告诉 Vim 忽略任何转义之后的双引号：

```
:syntax region xString start=/"/ skip=/\\"/ end=/"/
```

用两个反斜杠匹配单个反斜杠，因为反斜杠在字符串中是一个特殊字符。

什么时候用区域而不用匹配？两者主要的区别是匹配是单个的模式，它必须作为整体来匹配；而区域是只要区域的 "开始" 匹配便开始匹配，"结尾" 匹配是否找到并不重要。这样当一个项目倚赖于 "结尾" 匹配时，你不能用区域。区域定义起来更简单些，当使用嵌套项目时也更加方便，这将在下一节中解释。

44.5 嵌套项目

看一下这个注释：

```
%Get input  TODO: Skip white space
```

你想将 "TODO" 高亮显示成黄色字符，即使它在一个蓝色高亮显示的注释中。要使 Vim 了解到这些，你需要定义以下的语法组：

```
:syntax keyword xTodo TODO contained
:syntax match xComment /%.*/ contains=xTodo
```

在第一行中，"contained" 参数告诉 Vim 这个关键字只能存在于另一个语法项目中。下一行指定 "contains=xTodo"，这说明 "xTodo" 语法元素存在其中。结果就是注释行作为整体匹配 "xComment" 并被显示成蓝色。而其中的 "TODO" 单词匹配 "xTodo" 将被显示成黄色。

递 归 嵌 套

x 语言用大括号定义代码块。而且一个代码块中可以包含其它的代码块。这可以用以下方式定义：

```
:syntax region xBlock start=/{/ end=/}/ contains=xBlock
```

假设你有这样的代码：

```
while i < b {
        if a {
                b = c;
        }
}
```

首先一个 "xBlock" 开始于第一行的 '{'。第二行中有另一个 '{'。因为我们已经在一个 "xBlock" 之内，因此一个嵌套的 "xBlock" 在此处开始。这样 "b = c" 那一行在第二级的 "xBlock" 区域之内。下一行有一个 '}'，这将和区域的结尾相匹配。它会结束嵌套的 "xBlock"。因为这个 '}' 存在于嵌套的区域，它在第一个 'xBlock' 区域的视野里被隐藏。因此最后一行的 '}' 结束第一个 'xBlock' 区域。

保 留 结 尾

考虑下面的两个语法项目：

```
:syntax region xComment start=/%/ end=/$/ contained
```

```
:syntax region xPreProc start=/#/ end=/$/ contains=xComment
```

你定义注释是以 '%' 开始的任意行，而预处理命令是以 '#' 开始的任意行。因为预处理行可以有注释，预处理的定义包含一个 "contains=xComment" 参数。现在看一下下面的代码会发生什么情况：

```
#define X = Y  % Comment text
int foo = 1;
```

你看到的是第二行也被当作预处理高亮显示。预处理命令应该在行结尾处结束。这也是你要使用 "end=/$/" 的原因。但到底是哪里错了呢?

问题在于被包含的注释。注释以 '%' 开始，在行尾结束。注释结束后，预处理语法继续，这是在行尾已被处理之后进行的，因此下一行也被认为是预处理命令。

为避免这个问题，避免一个包含在内的语法项目吃掉一个必需的行尾，使用 "keepend" 参数。这样就会解决两个行结尾的匹配问题：

```
:syntax region xComment start=/%/ end=/$/ contained
:syntax region xPreProc start=/#/ end=/$/ contains=xComment keepend
```

包含多个项目

你可以用 'contains' 参数来指定任何项目都能被包含。比如：

```
:syntax region xList start=/\[/ end=/\]/ contains=ALL
```

所有的项目都包含在此项中。它也包含它自身，但不能在同一位置（这是为了避免无限循环）。

你可以指定不被包括的组，这样就包含除了指定组之外的所有组：

```
:syntax region xList start=/\[/ end=/\]/ contains=ALLBUT,xString
```

使用 "TOP" 项目，你可以包含所有那些没有 "contained" 参数的项目。"CONTAINED" 用来只包含那些使用了 "contained" 参数的项目。更多信息，参考 :syn-contains 。

44.6 跟随组

x 语言有这种形式的语句：

```
if (condition) then
```

你想对这三种项目使用不同的高亮。尽管 "(condition)" 和 "then" 可能在其它地方有其它的高亮显示方式。这时你需要这样做：

```
:syntax match xIf /if/ nextgroup=xIfCondition skipwhite
:syntax match xIfCondition /([^)]*)/ contained nextgroup=xThen skipwhite
:syntax match xThen /then/ contained
```

"nextgroup" 参数指定跟随的项目。这并不是必须的。如果指定的项目都没找到，什么事情都不会发生。比如，在下面的代码中：

```
if not (condition) then
```

"if" 匹配 "xIf"。"not" 并不匹配指定的跟随组 "xIfCondition"，这样只有 "if" 被高亮显示。

"skipwhite" 参数告诉 Vim 空白字符（空格和制表符）可以出现在项目之间。相似地，"skipnl" 参数允许项目之间存在换行符；"skipempty" 允许存在空行。注意 "skipnl" 并不忽略一个空行，换行符之后必须有能够匹配的内容。

44.7 其它参数

MATCHGROUP

当你定义一个区域时，整个区域将根据指定的组名被高亮显示。比如，要高亮显示 "xInside" 组被小括号 "()" 包含的部分，用下面的命令：

```
:syntax region xInside start=/(/ end=/)/
```

假设，你要以不同方式显示括号。你可以用很多复杂的区域语句来完成，或者你可以使用 "matchgroup" 参数。它将告诉 Vim 用另外一个组的方式来显示区域的开头和结尾（在本例中是 xParen 组）：

```
:syntax region xInside matchgroup=xParen start=/(/ end=/)/
```

"matchgroup" 参数适用于它之后的区域开头或结尾。在前一例中，开头和结尾都以 "xParen" 方式高亮显示。如果要用 "xParenEnd" 显示结尾：

```
:syntax region xInside matchgroup=xParen start=/(/
        \ matchgroup=xParenEnd end=/)/
```

用 "matchgroup" 的一个副作用是被包含的项目不会在区域的开头或结尾匹配。"transparent" 的例子中用到此作用。

TRANSPARENT

在 C 语言文件中你想要以不同方式高亮显示 "while" 和 "for" 后面的小括号。这两者中均会出现嵌套的小括号，它们会以相同的方式显示。你必须保证括号的高亮显示在匹配的 ')' 处结束。下面是一种可用的方法：

```
:syntax region cWhile matchgroup=cWhile start=/while\s*(/ end=/)/
        \ contains=cCondNest
:syntax region cFor matchgroup=cFor start=/for\s*(/ end=/)/
        \ contains=cCondNest
:syntax region cCondNest start=/(/ end=/)/ contained transparent
```

你现在可以用不同方式高亮显示 "cWhile" 和 "cFor"。"cCondNest" 项目可以出现在它们中的任何一个，并且使用所处那个项目的高亮显示。这是 "transparent" 参数产生的效果。

注意 "matchgroup" 参数和项目本身是同一组。为什么要定义它呢？使用 "matchgroup" 的副作用是被包含的项目并不会在开始项目之中寻找匹配。这样就避免了 cCondNest 组匹配 "for" 或 "while" 后面立即跟随的 '('。否则，它会扫描整个文本直到匹配的 ')'，而区域从其后继续。现在 cConNest 只在开始模式匹配后才开始匹配，就是在第一个 '(' 之后。

位 移

假设你要定义一个在 "if" 之后并在 '(' 和 ')' 之间的区域。但是你不想包括 "if" 或者 '(' 和 ')'。你可以为模式定义一个位移。比如:

```
:syntax region xCond start=/if\s*(/ms=e+1 end=/)/me=s-1
```

开始模式的位移是 "ms=e+1"。"ms" 代表匹配开始 (Match Start)。它定义模式开始的偏移量。"e+1" 的意思是匹配在模式结束处 (End) 后再忽略一个字符后开始匹配。

结尾模式的位移是 "me=s-1"。"me" 表示匹配结尾 (Match End)。"s-1" 的意思是在模式开始 (Start) 的前面一个字母处。下面这个代码的结果是:

```
if (foo == bar)
```

只有 "foo == bar" 部分以 "xCond" 方式高亮显示。

关于偏移量的更多信息, 参考: :syn-pattern-offset 。

ONELINE

"oneline" 参数说明区域不超过一行的范围。比如:

```
:syntax region xIfThen start=/if/ end=/then/ oneline
```

这定义了一个开始于 "if" 并在 "then" 结束的区域。但如果在 "if" 后没有 "then", 这个区域就不会被匹配。

备注:
当使用 "oneline" 时, 如果结尾模式没有在同一行被匹配, 那么整个区域就不会匹配。如果不指定 "oneline", Vim 不会 检查结尾模式是否匹配; 即使结尾模式在文件后面的部分没有匹配, 区域也将开始匹配。

续 行 及 避 免

现在事情变得有点复杂。让我们来定义一个预处理行。它以 "#" 开始到行尾结束。一个以 '\' 结束的行使得下一行成为它的续行。这样你可以允许语法项目包括一个续行模式:

```
:syntax region xPreProc start=/^#/ end=/$/ contains=xLineContinue
:syntax match xLineContinue "\\$" contained
```

在本例中, 虽然 'xPreProc' 一般只匹配单个行, 它包含的组 (xLineContinue) 允许它使用多行。比如, 它会匹配下面的两行:

```
#define SPAM  spam spam spam \
                bacon and spam
```

在本例中, 这就是你所需要的。如果它并不是你想要的, 你可以通过给被包含的模式添加 "excludenl" 参数来使区域在一个单行上。比如, 你想高亮显示 "xPreProc" 中只在行结尾处的 "end"。为了避免 "xPreProc" 像 "xLineContinue" 那样扩展到下一行, 使用

"excludenl":

```
:syntax region xPreProc start=/^#/ end=/$/
        \ contains=xLineContinue,xPreProcEnd
:syntax match xPreProcEnd excludenl /end$/ contained
:syntax match xLineContinue "\\$" contained
```

"excludenl" 必须写在模式之前。因为 "xLineContinue" 并没有 "excludenl"，它的匹配会像前面那样扩展 "xPreProc" 到下一行。

44.8 簇

开始编写语法文件时，你会注意到你要产生很多的语法组。Vim 能让你定义一个语法的集合 -- 簇 (cluster)。

假设你有一个语言，它含有 "for" 循环，"if" 语句，"while" 循环和函数。它们都含有相同的语法元素：数字，标识符。你可以这样定义它们：

```
:syntax match xFor /^for.*/ contains=xNumber,xIdent
:syntax match xIf /^if.*/ contains=xNumber,xIdent
:syntax match xWhile /^while.*/ contains=xNumber,xIdent
```

你需要重复使用 "contains" 参数。如果你想添加另一个包含项目，你需要把它加三次。语法簇简化了这些定义，你可以使用一个簇来代表这些语法组。

要为这三个组包含的两个项目定义一个簇，用下面的命令：

```
:syntax cluster xState contains=xNumber,xIdent
```

簇可以像语法组那样用在其它的项目内。它们的名字以 '@' 开头。因此你可以这样定义这三个组：

```
:syntax match xFor /^for.*/ contains=@xState
:syntax match xIf /^if.*/ contains=@xState
:syntax match xWhile /^while.*/ contains=@xState
```

你可以用 "add" 参数给簇添加新组：

```
:syntax cluster xState add=xString
```

你也可以用 "remove" 参数从簇里删除某个组：

```
:syntax cluster xState remove=xNumber
```

44.9 包含其它语法文件

C++ 语言的语法是 C 语言的超集。因为你不想编写两个独立的语法文件，你可以在 C++ 语法文件中首先读入 C 语言的语法文件：

```
:runtime! syntax/c.vim
```

":runtime!" 命令会在 'runtimepath' 里搜索所有的 "syntax/c.vim" 文件。这使得 C++ 语法中的 C 部分定义的方式就像 C 文件里使用的那样。如果你替换了 c.vim 语法

文件，或是在另一个文件中增加了项目，这些都会被载入。

载入 C 语法项目之后，可以定义特定的 C++ 项目。比如，添加一些没在 C 中用到的关键字：

```
:syntax keyword cppStatement    new delete this friend using
```

这会像任何其它的语法文件一样起作用。

现在考虑一下 Perl 语言。Perl 脚本包括两个不同的部分：POD 格式的文档和用 Perl 本身编写的程序。POD 部分以 "=head" 开头并以 "=cut" 结尾。

你想要在一个文件中定义 POD 语法，并在 Perl 语法文件中使用它。":syntax include" 命令读入一个语法文件并将定义的元素储存到一个语法簇中。对 Perl 来说，格式如下所示：

```
:syntax include @Pod <sfile>:p:h/pod.vim
:syntax region perlPOD start=/^=head/ end=/^=cut/ contains=@Pod
```

perlPOD 开始于 Perl 文件中的 "=head"。这个区域包含 @Pod 簇。所有在 pod.vim 语法文件定义的顶层项目均在这里匹配。当找到 "=cut"，区域结束，我们回到在 Perl 文件中定义的项目。

":syntax include" 命令很聪明，它会忽略包含文件中的 ":syntax clear"。并且类似于 "contains=All" 的参数只会包括存在于被包含文件的项目，而不会包括包含文件内的项目。

"<sfine>:p:h/" 部分使用当前文件的名字 (<sfile>)，把它展开成全路径 (:p)，然后取头部分 (:h)。结果就是文件的目录名。这将包含相同目录下的 pod.vim 文件。

44.10 同步

编译器处理起这些来很简单。它们在文件头开始解析以至文件尾。Vim 并没有这么简单。它必须从编辑进行的中间开始。那么怎么确定它的位置?

秘密在于 ":syntax sync" 命令。它告诉 Vim 怎样找出当前位置。比如，下面的命令告诉 Vim 向后扫描 C 风格注释的开头和结尾，并对注释高亮显示：

```
:syntax sync ccomment
```

你可以使用其它的参数。"minlines" 参数告诉 Vim 向后扫描的最小行数；"maxline" 则指定最大行数。

比如，下面的命令告诉 Vim 从屏幕的顶端开始反向扫描至少 10 行：

```
:syntax sync ccomment minlines=10 maxlines=500
```

如果它不能在这个范围内确定位置，它会继续反向查看直至它知道要怎样做。但它最多就查看 500 行。(大的 "maxline" 值会降低速度；而小的值会使同步失败。)

为使同步更快，告诉 Vim 哪些语法项目可以被忽略。对那些只有当确实要显示时才用到的匹配和区域，可以指定 "display" 参数。

缺省情况下，找到的注释会以 Comment 语法组指定的方式显示颜色。如果你想用其它的颜色显示，你可以指定一个不同的语法组：

```
:syntax sync ccomment xAltComment
```

如果你的语言没有 C 风格的注释，你可以用其它的同步方式。最简单的方法是告诉 Vim 跳过若干行并从那里开始定位。下面的命令指定 Vim 反向跳过 150 行，并从那里开始分析：

```
:syntax sync minlines=150
```

一个大的 "minlines" 会使 Vim 变得很慢，尤其是向后卷屏的时候。
最后，你可以用这个命令来指定要查找的语法组：

```
:syntax sync match {sync-group-name}
        \ grouphere {group-name} {pattern}
```

这会告诉 Vim 当它看到 {pattern}，名为 {group-name} 的语法组会在其后开始。{sync-group-name} 用于给这个同步规格说明一个名字。比如，外壳脚本的 if 语句以 "if" 开头，以 "fi" 结尾。

```
if [ --f file.txt ] ; then
        echo "File exists"
fi
```

你可以用下面的命令为这个语法定义一个 "grouphere" 指示：

```
:syntax sync match shIfSync grouphere shIf "\<if\>"
```

"groupthere" 参数告诉 Vim 某个模式结束一个组。比如，if/fi 组的结束如下所示：

```
:syntax sync match shIfSync groupthere NONE "\<fi\>"
```

在这个例子中，NONE 告诉 Vim 你不在任何一个特殊的区域内。特别地，你不在一个 if 代码块内。

你还可以定义没有 "grouphere" 或 "groupthere" 参数的匹配和区域。这些组在同步时会被忽略。比如，下面的命令忽略 {} 内的所有部分，即使它们通常能在其它的同步方法中匹配：

```
:syntax sync match xSpecial /{.*}/
```

有关同步的更多信息，请见参考手册： :syn-sync 。

44.11 安装语法文件

当你新的语法文件要投入使用时，把它放到在 'runtiimepath' 下的一个 "syntax" 目录。对 Unix 来讲，它可以是 "~/.vim/syntax"。
语法文件的名字必须要和文件类型相同并以 ".vim" 做后缀。这样，对 x 语言来讲，这个文件的完整路径是：

```
~/.vim/syntax/x.vim
```

你要确保该文件类型可被识别。参考 43.2 。

如果你的语言文件工作的很好，你也许想要其他的 Vim 用户来使用。首先阅读下一节以确保你的文件能很好的适用于别人。然后将它 email 给 Vim 维护者：<maintainer@vim.org>。并要解释文件类型是怎样识别的。如果幸运的话，你的文件会包括在 Vim 的下一版本中！

向已有文件添加项目

我们前边是假定你要写一个全新的语法文件。如果一个已有的语法文件可以工作但是缺少某些项目，你可以将这些项目添加到另一个独立的文件。这样不用改变发布的语言文件，因为安装 Vim 的新版本时会丢失这些改变。

在你的文件中写入语法命令（也许要用到已有语法的组名）。比如，给 C 语法文件添加新的变量类型：

```
:syntax keyword cType off_t uint
```

文件应使用和原始语法文件一样的名字。在本例中，是 "c.vim"。把它放到 'runtimepath' 里的靠后的目录里。对 Unix 而言，该目录可以是：

```
~/.vim/after/syntax/c.vim
```

44.12 可移植语法文件格式

如果所有的 Vim 用户都能交换语法文件岂不更好？为此，语法文件必须符合一些规则。

首先要有一个说明语法文件目的，维护人，以及更新时间的文件头。但不要包含太多的历史更改信息。比如：

```
" Vim syntax file
" Language:     C
" Maintainer:   Bram Moolenaar <Bram@vim.org>
" Last Change:  2001 Jun 18
" Remark:       Included by the C++ syntax.
```

用和其它语法文件相同的格式。使用一个已有的语法文件作为样例会节约很多时间。

为你的语法文件选择一个好的，描述性强的名字。使用小写字符和数字。不要使用太长的名字，因为它将在很多地方用到：语法文件名 "name.vim"，'filetype'，b:current_syntax 以及组名的开头（例如：nameType、nameStatement、nameStrings 等）。

以一个 "b:current_syntax" 检查开头。如果它被定义，表明其它语法文件（在 'runtimepath' 前部的）已被载入：

```
if exists("b:current_syntax")
  finish
endif
```

为和 Vim 5.8 兼容：

```
if version < 600
  syntax clear
elseif exists("b:current_syntax")
  finish
endif
```

在最后将 "b:current_syntax" 设定为语法的名字。不要忘了被包含的文件也会这样做，当你包含两个文件时，也许要复位 "b:current_syntax"。

如果你想让你的语法文件适用于 Vim 5.x，添加一个对 v:version 检查。参考 Vim 7.2 发布里的某个语法文件。

不要包括任何的用户首选项。不要设定 'tabstop'，'expandtab' 等。这些属于文件类型脚本。

不要包括映射或者缩写。如果确实需要识别关键字，只设定 'iskeyword'。

为使用户能够选择自己喜欢的颜色，请为每一种类型的高亮项目制作不同的高亮组。然后把它们链接到标准高亮组。这使每个色彩方案能正常工作。如果你选择特定的颜色，那样会使一些色彩方案看起来很难看。不要忘了一些人使用不同的背景，或者只能使用八种颜色。

链接可用 "hi def link"，这样用户就可以在语法文件载入前选择不同的高亮。例如：

```
hi def link nameString        String
hi def link nameNumber        Number
hi def link nameCommand       Statement
... 等等 ...
```

给那些在同步时用不到的项目添加 "display" 参数可以提高向后滚屏和 CTRL-L 的速度。

下一章： usr_45.txt 选择你的语言

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2 http://vimcdoc.sf.net

选择你的语言

Vim 中各种的消息可以用多种不同的语言给出。本章就介绍如何改变使用的语言。同时还讲解如何编辑各种语言的文件。

45.1 消息所用语言
45.2 菜单所用语言
45.3 使用其它编码
45.4 编辑其它编码的文件
45.5 文本录入

下一章: usr_90.txt 安装 Vim
前一章: usr_44.txt 自定义语法高亮
目录: usr_toc.txt

45.1 消息所用语言

当你启动 Vim 时，它会检查环境变量从而找出你所使用的语言。大多数情况下这就够了。你会看到你所用语言的消息（如果有的话）。要查看当前语言，使用命令:

```
:language
```

如果你得到 "C" 的输出，表示 Vim 在使用缺省的语言 —— 英语。

备注:
只有当多种语言支持被编译进 Vim 时，你才可能用它来编辑各种不同的语言。要找出 Vim 是否支持多语种，看一下 ":version" 命令的输出中有没有 "+gettext" 和 "+multi_lang"。如果有的话，表示应该没问题了。如果你看到 "-gettext" 或者 "-multi_lang" 的话你得重找一个 Vim。

如果你想要 Vim 的输出消息也用另外一种语言呢? 有几种可能的办法。你应该使用哪一种取决于你所使用的系统。

第一种办法是在启动 Vim 前设置环境变量。以 Unix 为例:

```
env LANG=de_DE.ISO_8859-1  vim
```

只有你的系统支持该语言的时候这才有效。这个办法的好处是所有的 GUI 消息和运行库里的消息都会使用正确的语言。缺点是你必须在启动 Vim 前设定所要用的语言。如果你想在运行 Vim 当中改变所用的语言，你就得使用第二种办法:

```
:language fr_FR.ISO_8859-1
```

这样你可以尝试使用几种不同的语种名。当你所指定的语言系统不支持时你会得到一个错误消息。而当所指定语言的消息文件不存在时，你不会被警告。Vim 只会悄悄的使用缺省的英语。

要找到系统支持的语言，先要找到列出这些语言的目录。在我的系统上，这个目录是 "/usr/share/locale"。有些系统上是 "/usr/lib/locale"。"setlocale" 的手册页应该

告诉你该目录在你使用系统的什么地方。
在键入那些语种名的时候要当心，大小写是有区别的，而且 '-' 和 '_' 字符也很容易被混淆。

你也可以分别为消息，编辑文本和时间等设定所使用的语言。参阅 :language 。

自己动手翻译消息

如果翻译的消息没有包括你的语言，你也可以自己写。你得先拿到 Vim 的源代码和 GNU gettext 软件包。在解压缩了源代码后，先读一下 src/po/README.txt。那里有详细的指导。
翻译本身不是很难的，你不一定要是一个程序员。当然，你必须懂英语和你要翻译成的语言。
当你对你的翻译满意的时候，可以考虑和其它人共享。上载到 vim-online (http://vim.sf.net) 或者 e-mail 给 Vim 的维护者 Vim <maintainer@vim.org>。或者两者都做。

45.2 菜单所用语言

缺省情况下菜单是英语的。要使用你自己的语言，首先它们得被翻译过。通常情况下菜单使用的语言会自动被设置成你所用的语言，就像消息一样。你什么也不用做。但是如果 Vim 不包括你所用语言的菜单的翻译就不行了。
假设你在德国，使用的语言被设为德语。但你更愿意在菜单中看到 "File" 而非 "Datei"。你可以这样将菜单语言换回英语：

```
:set langmenu=none
```

也可以指定某一语言：

```
:set langmenu=nl_NL.ISO_8859-1
```

和上面一样 "-" 和 "_" 是有区别的。然而，大小写在这里可以不分。
'langmenu' 选项必须在菜单被加载前设置才有效。一旦菜单已经被显示了，再改变其值就不会直接生效了。因此，你需要把设置 'langmenu' 的命令放在 vimrc 文件里。
如果你实在想在 Vim 运行当中改变菜单所用的语言，你可以这样做：

```
:source $VIMRUNTIME/delmenu.vim
:set langmenu=de_DE.ISO_8859-1
:source $VIMRUNTIME/menu.vim
```

这样有一个缺陷：所有你自己定义的菜单都不见了。你还得必须重新定义它们。

自己动手翻译菜单

在下面这个目录可以找到那些已有的菜单翻译：

```
$VIMRUNTIME/lang
```

翻译文件被称为 menu_{language}.vim。如果找不到你想使用的语言的翻译，你可以自行翻译。最简单的办法就是拷贝现存的语言文件并在其基础上做改动。
先用 ":language" 命令找出你现在使用的语言的名称。就用这个名称，但名字的所有

字母要小写。将这个文件拷贝到你自己的运行时目录。例如，在 Unix 系统上你应该：

```
:!cp $VIMRUNTIME/lang/menu_ko_kr.euckr.vim ~/.vim/lang/menu_nl_be.iso_8859-1.vim
```

你会在 "$VIMRUNTIME/lang/README.txt" 文件中找到与翻译相关的更多提示。

45.3 使用其它编码

Vim 猜你所要编辑的文件编码和你所用的语言相一致。对于许多欧洲语言而言，编码是 "latin1"。那么每个字节是一个字符。一共可能有 256 个不同的字符。而对于亚洲语言这就远远不够了。这些语言大多使用双字节编码，从而提供超过一万个可能的字符。即使如此，如果一篇文本中包括数种语言的文字，这样还是不够。Unicode 就是来解决这个问题的。其设计允许它包括多种语言中全部的常用字符。它被称为 "替代所有其它编码的超级编码格式"。但它的使用还不很广泛。

幸运的是，Vim 对这三种编码方式都支持，并且有条件的支持使用与系统环境语言不一致的文本编码。

当你的系统语言与你所编辑文件用的编码一致时，缺省设定应该没有问题。所以你什么也不用担心。下面的叙述只是当你想编辑不同的语言时才有关系。

在 GUI 上使用 UNICODE

Unicode 一个很方便的特性就是它可以与其它编码格式相会转换而不丢失信息。当你使 Vim 在内部使用 Unicode，你将可以用任何编码编辑文件。

不过，支持 Unicode 的系统还是比较有限的。因此可能你的语言不使用 Unicode。你得告诉 Vim 你希望使用 Unicode，然后在解决与系统其它部分交互的问题。

我们来启动 GUI 版本的 Vim，这样我们就可以显示 Unicode 字符。这样就行了：

```
:set encoding=utf-8
:set guifont=-misc-fixed-medium-r-normal--18-120-100-100-c-90-iso10646-1
```

'encoding' 选项告诉 Vim 你所用的字符的编码。该选项适用于缓冲区的文本 (你正在编辑的文件)，寄存器，Vim 脚本文件等等。你可以把 'encoding' 选项当作是对 Vim 内部运行机制的设定。

上例假设你的系统安装了指定的字体。例子中的字体名是用于 X-Window 系统的。这种字体是包括在一个用于增强 xterm 的软件包中的。如果你没有这种字体，你可以在这里找到：

```
http://www.cl.cam.ac.uk/~mgk25/download/ucs-fonts.tar.gz
```

对于 MS-Windows，一些字体包含了一些有限的 Unicode 字符。试一下 "Courier New" 字体。你可以通过 Edit/Select Font... 菜单来选择几种字体试试。但是你只能使用定宽的字体。例如：

```
:set guifont=courier_new:h12
```

如果不行的话，试着找个字体包。你可以在下面这个地址找到，如果 Microsoft 没把它移走的话 (译者注：已经不能用了)：

```
http://www.microsoft.com/typography/fonts/default.aspx
```

现在你已经通知了 Vim 在内部使用 Unicode 并用一种 Unicode 字体显示文本。然而，Vim 接收到的输入字符依然来自原来语言的编码字符。这些字符必须被转换成 Unicode。

你可以通过 'termencoding' 选项值来通知 Vim 要转换的语言。像这样：

```
:let &termencoding = &encoding
:set encoding=utf-8
```

这样就在 'encoding' 赋值为 utf-8 之前把 'encoding' 的旧值赋给 'termencoding'。你得自己试试这个办法在你的系统上行不行得通。当你使用一个为亚洲语言专门设计的输入法，同时又想编辑 Unicode 文本，这个办法应该工作的很好。

在 UNICODE 终端内使用 UNICODE

也有直接支持 Unicode 的终端。XFree86 自带的标准 xterm 就是其中之一。我们就用它作为例子。

首先，Unicode 支持要被编译进 xterm。参阅 UTF8-xterm 。

用 "-u8" 参数启动 xterm。你可能还需要指定一个字体。例如：

```
xterm -u8 -fn -misc-fixed-medium-r-normal--18-120-100-100-c-90-iso10646-1
```

现在你就可以在终端内运行 Vim 了。像前面一样将 'encoding' 设定为 "utf-8"。这样就行了。

在普通终端里使用 UNICODE

假定你希望编辑 Unicode 文件，但是手头又没有一个支持 Unicode 的终端。你还是可以用 Vim 这样做，但是那些终端不支持的字符就不能被正确的显示了。文本的编排格式会被保留。

```
:let &termencoding = &encoding
:set encoding=utf-8
```

这和 GUI 的那个例子一样。但内部工作机制不同：Vim 会把显示的字符转换后再送给终端程序。这样屏幕就不会显示一堆毫无意义的字符。

这需要 'termencoding' 和 'encoding' 之间的转换。Vim 会自己处理 latin1 和 Unicode 之间的转化。其它的转换需要编译进来 +iconv 特性。

试着编辑一个含有 Unicode 的文件。你会注意到 Vim 会在那些无法显示的字符处显示问号（或下划线或者其它什么）。将光标移动到问号的地方并使用下面的命令：

```
ga
```

Vim 会显示该字符的代码。这样你就会知道那个字符大概是什么。你可以去查 Unicode 表。如果你有很多很多时间的话，你甚至可以这样来阅读一个文件的内容。

备注：
因为 Vim 对所有的文本都使用 'encoding' 选项。改变该选项的值会使所有的非 ASCII 码字符无效。你在使用寄存器和 'viminfo' 文件（例如，一个被存储的查找模式）时就会注意到了。建议在 vimrc 文件里设置 'encoding' 选项，然后就别再碰它。

45.4 编辑其它编码的文件

假定你已经配置 Vim 使用 Unicode 了，你现在想编辑一个 16 位 Unicode 的文件。听起来很简单，是吗？事实上，Vim 在内部使用 utf-8 编码，所以 16 为的编码必须被转换。因为存在一个字符集 (Unicode) 和编码格式 (utf-8 或 16 位) 的差别。

Vim 会试着用 'fileencodings' 选项中的编码名称检测你在编辑哪种文件。当使用 Unicode 时，缺省的选项值是："ucs-bom,utf-8,latin1"。也就是说 Vim 会检查编辑的文件看看是下面的编码之一：

ucs-bom	文件必须以 Byte Order Mark (BOM) 开始。可以检测 16 位，32 位和 utf-8 Unicode 编码。
utf-8	utf-8 Unicode。如果有在 utf-8 中非法的字节序列，此设定将被拒绝。
latin1	保证有效的经典 8 位编码。

当你开始编辑一个 16 位的 Unicode 编码，而该文件有 BOM 时，Vim 会检测到这些并在读取文件时将其转换为 utf-8 编码。'fileencoding' 选项 (后面没有 s) 被设为检测到的值，在本情况下是 "utf-16le"。表示是 Unicode，双字节以及 little-endian (高位字节在后) 的编码。这样的文件格式在 MS-Windows 上很常用 (例如，注册表文件)。

当写入文件时，Vim 会比较 'fileencoding' 和 'encoding' 的值。如果它们的值不同，文本会被转换。

如果 'fileencoding' 的值是空的，意味着不作任何转换。这样文本就会被以 'encoding' 的值来编码。

如果缺省的 'fileencodings' 值不行的话，你可以将其设成你希望 Vim 尝试使用的编码。只有当一个值无效时下一个才会被使用。将 "latin1" 放到第一位是不行的，因为它永远都不会是无效的。举个例子，要在文件没有 BOM 也不是 utf-8 编码的时候缺省设为简体中文编码 (Unix 和 Windows 上都可以用 chinese 作为编码名)：

```
:set fileencodings=ucs-bom,utf-8,chinese
```

encoding-values 有建议值。其它的值也可能有效。这取决于系统提供的转换功能。

强 制 编 码

如果编码的自动检测对你不起作用的话，你就得告诉 Vim 你要编辑的文件是什么编码。例如：

```
:edit ++enc=koi8-r russian.txt
```

"++enc" 部分指定编辑该文件时应该使用的编码。Vim 会把指定的编码 (在这是俄语) 转换为 'encoding'。'fileencoding' 也会被设为指定的编码，这样当写入文件时就会发生编码的反转换。

在写入文件时也可以用到一样的参数。这样你事实上可以用 Vim 来转换一个文件。例如：

```
:write ++enc=utf-8 russian.txt
```

备注：
转换可能导致字符丢失。在 Unicode 和其他编码之间的相互转换基本上不会有这个麻烦，除非你有非法字符。当文件包括多于一种语言的字符时，从 Unicode 转换为其它编码很可能会丢失信息。

计算机键盘只有那么多键。有些语言包括几千个字符。Unicode 有超过十万。那你怎么才能输入这些字符呢?

首先，如果你不需要很多的特殊字符，你可以用二合字母。这已经在 24.9 讲述过了。

当你使用一种有很多字符的语言，你的键盘应付不了，你就得使用一种输入法 (IM)。这需要学习如何把输入的键变成要输入的字符。你所用的系统很可能已经提供了输入法。Vim 应该可以很好地与其合作。更多的细节请参阅 mbyte-XIM (X-Windows) 和 mbyte-IME (MS-Windows)。

键盘映射表

对于某些语言来说字符集可能不同但是可用的字符数目和拉丁字符集差不多。这样就有可能在字符和键盘之间定义一个一对一的对应。Vim 用键盘映射表来解决这个问题。

假定你想输入希伯来语，你可以这样载入希伯来语的键盘映射表:

```
:set keymap=hebrew
```

Vim 会为你找到一个键盘映射表文件。这取决于 'encodings' 选项的值。如果找不到相应的文件，你会得到一个错误信息。

现在你就可以在插入方式下输入希伯来语了。普通模式下，当你键入 ":", Vim 会自动切换到英语。你也可以用下面这个命令来在希伯来语和英语之间切换:

```
CTRL-^
```

它只在插入模式和命令行模式下才有效。在普通模式下会产生完全不同的效果 (跳转到轮换文件)。

如果你打开 'showmode' 选项的话，键盘映射表的使用情况会在状态信息里指出。在 GUI 上，Vim 会使用不同的光标颜色来指明映射表的使用情况。

你可以用 'iminsert' 和 'imsearch' 选项来改变键盘映射表的用法。

要查看键映射的列表，用这个命令:

```
:lmap
```

要查看系统有哪些键盘映射表文件，在 GUI 上你可以用 Edit/Keymap 菜单。否则你可以用:

```
:echo globpath(&rtp, "keymap/*.vim")
```

(译者注: 中文用户可以试试 pinyin 映射表，可用来输入带四声的韵母。)

自定义键盘映射表

你也可以创建你自己的键盘映射表。不是很难。从一个和你要使用语言类似的映射表开始。将其拷贝到你的运行时目录里的 "keymap" 子目录。例如，在 Unix 上，你可以用 "~/.vim/keymap" 目录。

键盘映射表的文件名必须遵守下面的规则:

```
keymap/{name}.vim
```

或

```
keymap/{name}_{encoding}.vim
```

{name} 是键盘映射表的名字。选择一个容易识别，又和系统中其它映射表不同的名字(除非你希望替换一个现存的映射表)。名字不能包括下划线。使用的编码名是可选的。例如：

```
keymap/hebrew.vim
keymap/hebrew_utf-8.vim
```

文件的内容应该很容易理解。看看随 Vim 一起发布的几个键盘映射表就行了。更详细的，请参阅 mbyte-keymap 。

最 后 的 办 法

如果其它的方法都行不通，你可以用 CTRL-V 来输入任何字符：

编码	键入	范围
8-bit	CTRL-V 123	十进制 0-255
8-bit	CTRL-V x a1	十六进制 00-ff
16-bit	CTRL-V u 013b	十六进制 0000-ffff
31-bit	CTRL-V U 001303a4	十六进制 00000000-7fffffff

注意 不要输入空格。更详细的，参阅 i_CTRL-V_digit 。

下一章： usr_90.txt 安装 Vim

版权：参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

VIM 用户手册 - by Bram Moolenaar
译者: lang2 http://vimcdoc.sf.net

安装 Vim

install

在你开始使用 Vim 之前你必须安装它。根据你的系统不同，安装可能很简单，也可能稍微复杂一点。这一章会给出一些提示，同时也叙述如何升级。

90.1 Unix
90.2 MS-Windows
90.3 升级
90.4 常见安装问题
90.5 卸载 Vim

前一章: usr_45.txt 选择你的语言
目录: usr_toc.txt

90.1 Unix

首先你要决定的是: 要为整个系统安装 Vim 还是为单个用户。安装过程几乎是一样的。但是 Vim 安装的路径不同。

对于系统安装来说基目录常使用 "/usr/local"。但对于你的系统可能也不同。看看其它的软件包是安装在哪里的。

当为单个用户安装时，你可以使用你的主目录作为基目录。Vim 的文件将被放置到 "bin" 和 "shared/vim" 等子目录中。

从一个程序包安装

你可以取得为许多不同的 Unix 系统预先编译好的可执行文件。下面的链接给出一个很长的列表:

http://www.vim.org/binaries.html

这些可执行文件是由志愿者维护的，所以经常会过时。因此通过源代码来自己编译你自己的 UNIX 版本是个更好的办法。同时，从源代码创建 Vim 编辑器允许你控制可选的特性。但这需要一个编译器。

如果你有一个 Linux 的发行版本，其中的 "vi" 程序很可能是一个最小版的 Vim。例如，它可能不支持语法高亮。试试在你的发行版中找另外一个 Vim 程序包，或在网上搜搜。

从源码开始

你需要下面的东西来编译 Vim:

- 一个编译器 (最好用 GCC)
- GZIP 程序 (可以从 from www.gnu.org 得到)
- Vim 源码和运行时归档文件

下面这个文件提供了一个镜像列表，它可以帮助你找到距离你最近的镜像从而提高下载的速度：

```
ftp://ftp.vim.org/pub/vim/MIRRORS
```

如果你觉得够快的话，或者就用主站 ftp.vim.org。进入到 "unix" 目录你就可以找到一组文件。文件名中会含有版本名。最好下载最新的版本。

有两种办法可以下载 Vim 的 Unix 文件：一个包括所有东西的大文档；或者是四个较小的可以装载一张软盘上的小文件。对于 6.1 版那个大文件叫：

```
vim-6.1.tar.bz2
```

你需要 bzip2 程序来将其解压缩。如果你没有，可以下载那四个小些的文件。那些可以用 gzip 程序来解压缩。在 Vim 6.1 中它们叫：

```
vim-6.1-src1.tar.gz
vim-6.1-src2.tar.gz
vim-6.1-rt1.tar.gz
vim-6.1-rt2.tar.gz
```

编 译

先建立一个工作目录，例如：

```
mkdir ~/vim
cd ~/vim
```

在其中解开下载的文档。如果你下载了那个大文档，你可以这样解压缩：

```
bzip2 -d -c path/vim-6.1.tar.bz2 | tar xf -
```

把 "path" 改变为你下载文件所在的路径。

```
gzip -d -c path/vim-6.1-src1.tar.gz | tar xf -
gzip -d -c path/vim-6.1-src2.tar.gz | tar xf -
gzip -d -c path/vim-6.1-rt1.tar.gz | tar xf -
gzip -d -c path/vim-6.1-rt2.tar.gz | tar xf -
```

如果你觉得缺省的特性就够了的话，照下面这样直接编译 Vim 就行了：

```
cd vim61/src
make
```

make 程序会执行 configure 并编译所有的东西。后边我们会介绍如何将不同的特性编译进 Vim。

如果在编译时出现错误，请仔细的查看错误信息。编译程序会给出对于错误的提示。希望你能据此更正错误。你可能需要关闭一些特性。Makefile 也给出了一些可能适合你的特定系统的提示。

测 试

现在你可以试试你的编译成功了没有：

```
make test
```

这将执行一系列的测试脚本来确认 Vim 能正常的工作。测试的过程中 Vim 会多次启动，也会闪过各种各样的文本和信息。如果测试成功的话你最终会看到：

```
test results:
ALL DONE
```

如果你得到 "TEST FAILURE" 的信息，有些测试失败了。如果有一两个测试失败，Vim 可能还可以工作，不过不太完美。但是如果你看到大量的错误信息或者测试无法完成，那一定是有麻烦了。要么尝试自己解决，要么找个能帮助你的人。你可以在 maillist-archive 中查找解决办法。如果实在解决不了，你可以在 maillist 中提问看看有没有人能帮你。

安 装

install-home

如果你想安装在自己的 home 目录，编辑 Makefile 并查找这样的一行：

```
#prefix = $(HOME)
```

把行首的那个 # 去掉。

当你要为整个系统安装的时候，Vim 很可能已经为你选择好了一个合适的安装目录。你也可以照下面讲的自己选择一个，但你必须先改变身份为 root。

要安装 Vim 执行：

```
make install
```

这将把所有相关的文件转移到正确的地方。现在你可以试着运行一下 vim 以确认没有问题。下面两个简单的测试可以检查 Vim 能不能正确地找到那些运行时文件：

```
:help
:syntax enable
```

如果不成功的话，使用下面的命令来查看 Vim 是在哪儿找那些文件的：

```
:echo $VIMRUNTIME
```

你还可以用 "-V" 参数来启动 Vim，那样你可以得到更多的启动信息：

```
vim -V
```

别忘了本手册假设你用某一特定的方式使用 Vim。在安装完成之后，请依照 not-compatible 里面的指示来要求 Vim 以该方式运行。

选 择 特 性

Vim 有多种方法可以选择其特性。一个简单（直接）的办法就是编辑 Makefile。那里有很多的指令和例子。通常你只要去掉一行的注释就可以打开或关闭某一特性。

另外你也可以运行 "configure" 程序。这使得你可以手动指明你所希望的特性。缺点

是你得知道在命令行上输入什么。
　下面列出一些最可能引起你兴趣的 configure 参数。同样的，你也可以通过编辑 Makefile 来设定。

--prefix={directory}	安装 Vim 的顶级目录。
--with-features=tiny	关掉很多特性。
--with-features=small	关掉一些特性。
--with-features=big	打开更多的特性。
--with-features=huge	打开大多数特性。 参阅 +feature-list 察看每一种方式的详细特性列表。
--enable-perlinterp	开启 Perl 接口。类似的还有为 ruby、python 和 tcl 准备的参数。
--disable-gui	不编译 GUI 界面。
--without-x	不编译 X-windows 相关的特性。 当两个同时使用的时候，Vim 不会连接 X 服务器，这会使启动更快些。

察看所有参数的列表使用：

```
./configure --help
```

这里你可以找到各个特性的一些解释，以及指向其它更多信息的链接： feature-list 。
　对于那些喜欢冒险的人，你可以编辑 "feature.h" 文件。你还可以自己改动源代码！

90.2 MS-Windows

有两种方法可以安装 Vim 的 Microsoft Windows 版本。你可以选择自己解压缩几个文档，或者使用一个自动安装程序。多数使用较新的电脑的用户会选择第二种方法。对于第一种方法你需要：

- 包含 Vim 可执行程序的文档。
- Vim 运行时归档文件。
- 解压缩程序。

下面这个文件提供了一个镜像列表，它可以帮助你找到距离你最近的镜像从而提高下载的速度：

```
ftp://ftp.vim.org/pub/vim/MIRRORS
```

如果你觉得够快的话，或者就用主站 ftp.vim.org。进入到 "pc" 目录就可以找到一组文件。文件的名字中含有版本名。最好下载最新的版本。这里我们用 "61" 为例，表示 6.1 版。

gvim61.exe	自动安装程序。

第二种方法你就只需要这个了。执行该程序并依照提示操作就可以了。

第一种方法你需要选择下列的其中一个可执行文档：

gvim61.zip	一般的 MS-Windows GUI 版本。
gvim61ole.zip	带有 OLE 支持的 MS-Windows GUI 版本。 更消耗内存，支持与其他 OLE 程序的接口。
vim61w32.zip	32 位 MS-Windows 控制台版本。适用于 Win NT/2000/XP 命令窗口。不能很好地在 Win 95/98 内运行。
vim61d32.zip	32 位 MS-DOS 版本。适用于 Win 95/98 命令窗口。
vim61d16.zip	16 位 MS-DOS 版本。仅适用于老系统。 不支持长文件名。

你只需要其中的一个就够了。不过你可以同时安装一个 GUI 版本和一个控制台版本。但是包含有运行时文件的归档文件总是必需的。

vim61rt.zip	运行时文件。

使用你的解压缩程序把文件解出来。例如，使用 "unzip" 程序：

```
cd c:\
unzip path\gvim61.zip
unzip path\vim61rt.zip
```

这将把文件解压缩到 "c:\vim\vim61" 目录。如果你已经有一个 "vim" 目录了，你需要进入其上级目录。

现在进入那个 "vim\vim61" 目录，然后执行安装程序：

```
install
```

仔细的察看并选择你想要的。当你最终选择 "do it" 的时候安装程序将根据你的选项执行安装的步骤。

安装程序不会移走运行时文件。它们会被原封不动留在你解压缩的地方。

如果你对可执行文件包含的特性不满意的话，你可以试着自己编译 Vim。从你取得可执行文档同样的地方取得源代码。你需要一个有对应的 makefile 文件的编译器。
可以使用 Microsoft Visual C，但是它很昂贵。免费的 MingW 和 Cygwin 编译器也可以。相关提示要查阅 src/INSTALLpc.txt。

90.3 升级

如果你已经有一个版本的 Vim 并想安装另一个，这里就是你需要做的。

UNIX

当你键入 "make install" 命令时，运行时文件将被拷贝到一个该版本专有的目录，而不会覆盖现存版本的文件。这样就便可以同时运行两个版本的 Vim。

但是可执行文件 "vim" 将覆盖老的版本。如果你不在乎保留老的版本，这样就没问题。你还可以手动删除那些老版本的运行时文件。只要删除掉那个版本所在目录及其下所有文件即可。例：

```
rm -rf /usr/local/share/vim/vim58
```

通常该目录里不会有你改动过的文件。如果你的确改动了，例如，"filetype.vim" 文件，你最好把改动合并到新的版本然后在删除老的文件。

如果你比较小心，希望先试试新的版本，你可以用另外的名字安装新的版本。你需要指定一个 configure 参数。例如：

```
./configure --with-vim-name=vim6
```

在运行 "make install" 之前，你可以使用 "make -n install" 命令来查看安装会不会覆盖现有的重要文件。

当你最终决定启用新版本的时候，你要做的仅仅是将可执行文件改名为 "vim"。例：

```
mv /usr/local/bin/vim6 /usr/local/bin/vim
```

MS-WINDOWS

升级几乎和安装一个新版本一样。把新版本的文件解压缩到上一版本的同样位置。会对于新版本的文件生成一个新的目录。例如：“vim61"。你的运行时文件，vimrc 文件，viminfo 文件等等，会被原样保留。

如果你想同时运行两个版本，你就得做点作些手工活了。别运行安装程序。那样会覆盖老版本的一些文件。用全路径来运行新的可执行文件。程序会自动找到它需要的运行时文件。然而，如果你在别的地方设定了 $VIMRUNTIME 的值，这个法子就不灵了。

如果你对升级满意的话，你可以删除掉前一版本的文件。参阅 90.5 。

90.4 常见安装问题

这一节列举一些安装过程中常见的麻烦并给出一些解决办法。同时也回答一些安装相关的问题。

Q: 我没有 Root 权限。怎么安装 Vim? (Unix)

使用下面的配置命令可以将 Vim 安装到目录 $HOME/vim:

```
./configure --prefix=$HOME
```

这样你可以安装一个个人拷贝。请将 $HOME/bin 加入 path 中，以便执行本编辑器。参阅 install-home 。

Q: 我屏幕上的颜色不对。(Unix)

用下面的命令检查终端的设置：

```
echo $TERM
```

如果列出的终端类型不正确，更正之。查阅 06.2 可以得到更多的提示。另一个解决办法是使用 Vim 的 GUI 版本。名为 gvim。这样你就不需要一个正确的终端设置了。

Q: 我的 Backspace 和 Delete 键工作不正常

对于这两个键 (<BS>、<Del>)，什么键产生它们的键码不是很清楚。首先检查一下你的 $TERM 设置。如果没什么错误，试试这个：

```
:set t_kb=^V<BS>
:set t_kD=^V<Del>
```

第一行你需要键入 CTRL-V 在按 backspace 键。第二行你需要键入 CTRL-V 再按 Delete 键。你可以把这两行加入到你的 vimrc 文件中，参见 05.1 。 这样做的一个缺点是如果你换用另外一个终端程序的话，这些设置可能就不起作用了。其他的解决办法可以在 :fixdel 中找到。

Q: 我使用 RedHat Linux。我能使用系统自带的 Vim 吗?

缺省情况下 RedHat 仅安装一个 Vim 的最小版本。找一个叫 "Vim-enhanced-version.rpm" 的安装包就行了。

Q: 我怎么才能打开语法高亮? 怎么才能使用插件?

使用那个 vimrc 脚本的例子。在这儿你可以找到详细地解释: not-compatible 。

在第 6 章阅读有关语法高亮的解释: usr_06.txt 。

Q: 有没有一个好的 vimrc 文件可以使用?

到 www.vim.org 网站去找找。

Q: 哪儿可以找到好的 Vim 插件?

看看 Vim-online 网站: http://vim.sf.net。很多用户在那里上载了非常有用的 Vim 脚本。

Q: 哪有更多的技巧提示?

看看 Vim-online 网站: http://vim.sf.net。那里收集了很多用户的提示。你也可以在 maillist-archive 里查找。

90.5 卸载 Vim

虽然这几乎不可能发生，但是万一你希望完全卸载 Vim，本节告诉你怎么做。

UNIX

如果你是通过一个软件包安装 Vim 的，看看你的软件包管理器是如何卸载一个软件包的。如果你是通过源代码安装的你可以用这个命令卸载:

```
make uninstall
```

然而，如果你已经将那些文件删除或者你使用的是别人提供的归档，你就没法这样做了。那就只能手动删除那些文件。这里给出一个例子，假设你是以 root 身份将 Vim 安装到

"/usr/local" 下的:

```
rm -rf /usr/local/share/vim/vim61
rm /usr/local/bin/eview
rm /usr/local/bin/evim
rm /usr/local/bin/ex
rm /usr/local/bin/gview
rm /usr/local/bin/gvim
rm /usr/local/bin/gvim
rm /usr/local/bin/gvimdiff
rm /usr/local/bin/rgview
rm /usr/local/bin/rgvim
rm /usr/local/bin/rview
rm /usr/local/bin/rvim
rm /usr/local/bin/rvim
rm /usr/local/bin/view
rm /usr/local/bin/vim
rm /usr/local/bin/vimdiff
rm /usr/local/bin/vimtutor
rm /usr/local/bin/xxd
rm /usr/local/man/man1/eview.1
rm /usr/local/man/man1/evim.1
rm /usr/local/man/man1/ex.1
rm /usr/local/man/man1/gview.1
rm /usr/local/man/man1/gvim.1
rm /usr/local/man/man1/gvimdiff.1
rm /usr/local/man/man1/rgview.1
rm /usr/local/man/man1/rgvim.1
rm /usr/local/man/man1/rview.1
rm /usr/local/man/man1/rvim.1
rm /usr/local/man/man1/view.1
rm /usr/local/man/man1/vim.1
rm /usr/local/man/man1/vimdiff.1
rm /usr/local/man/man1/vimtutor.1
rm /usr/local/man/man1/xxd.1
```

MS-WINDOWS

如果你是用那个自动安装程序安装的话你可以运行 Vim 目录下 (例如 "c:\vim\vim61") 的 "uninstall-gui" 程序。你也可以通过开始菜单来运行。这可以删除大多数的文件菜单以及桌面快捷方式。有些文件可能只有待 Windows 重新启动之后才能被删除。

你可能被提示来删除整个 "vim" 目录。那里面很可能有你的 vimrc 文件以及其他你创建的运行时文件。要当心。

否则，如果你使用了那些 zip 归档安装了 Vim 的话，最好使用 "uninstall" 程序。它就在 "install" 程序所在的目录， 例如，"c:\vim\vim61"。用通常的那个 "install/remove software" 页也一样有效。

然而，这只会清除 Vim 的注册表信息。你得自己删除硬盘上的那些文件。简单的选中 "vim\vim61" 目录并删除就可以了。那里边不应该包含任何你所改动过的文件，不过你可能希望先检查一下。

"vim" 目录很有可能包含有你的 vimrc 文件以及其它你创建的运行时文件。你可能想保留它。

目 录: usr_toc.txt

版权: 参见 manual-copyright vim:tw=78:ts=8:noet:ft=help:norl:

intro.txt　　for Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar

译者：Dasn, tocer
http://vimcdoc.sf.net

介绍 Vim　　ref　reference

1. 简介　intro
2. 互联网上的 Vim　internet
3. 制作群　credits
4. 记法　notation
5. 了解一下模式　vim-modes-intro
6. 从一种模式切换到另一种模式　mode-switching
7. 窗口里的东东　window-contents
8. 术语　definitions

---

1. 简介　intro

Vim 的意思是 Vi IMproved（Vi 改进版）。以前它是 Vi IMitation（Vi 模仿版）的缩写，但是由于做了许多改进，改一下名字还是合适的。Vim 文本编辑器包含了 unix "Vi" 编辑器的几乎所有的命令并且还加入了许多新的命令。这对于编写程序和编辑普通的文本文件是非常有用的。

所有的命令都可以通过键盘来输入。这样的优点在于：你可以通过把手放在键盘上，把眼睛放在屏幕上，集中精力在编辑的任务上。如果你需要，Vim 提供对鼠标的支持，并且还有带有滚动条和菜单的图形界面版本（参见： gui.txt ）。

本手册的概述可以参见 "help.txt" 文件， help.txt 。在 Vim 里，按 <help> 或 <f1> 键就可以访问该文件，使用 :help 命令也可以（只键入 ":help" 即可，没有竖线或引号）。

如果帮助文件不在默认的位置，可以使用选项 'helpfile' 来指定。你可以通过标签跳转到相应的主题：用 CTRL-] 跳转至当前光标下的主题，用 CTRL-T 跳回来。

欲知 Vim 与 Vi 有何不同，请参考 vi_diff.txt 。

本手册说明各种系统上的 Vim。不同的计算机和不同的终端之间可能有一点儿不同。除了本文给出的一些建议，每个支持的系统还有单独的文档，见 sys-file-list 。

pronounce

Vim 的念法是一个词，像 Jim，而不是 vi-ai-em。拼法由大写字母打头，因为它是一个名字，还是像 Jim。

这是一本包含 Vim 所有命令和选项的参考手册。它不是一本介绍如何使用 Vi 或 Vim 的书，从某种意义上讲，它有些复杂了。对于初学者，有一个可供实地演练的 tutor 可以试试。想要学习怎样使用 Vim，请阅读用户手册 usr_toc.txt 。

book　books

许多介绍 Vi 和 Vim 的书都会有一个专门为初学者准备的章节。其它的书花更多笔墨在特定的功能上。这里可以看到 Vim 书的总览：

http://iccf-holland.org/vim_books.html

2. 互联网上的 Vim

internet

www WWW faq FAQ distribution download

Vim 的最新消息可以通过访问其网站页面得到。网站上同样包含指向最新版本的 Vim 的链接。FAQ 是一系列常见问题的解答。如果有问题请先读一下 FAQ。

```
Vim 主页:          https://www.vim.org/
Vim FAQ:           https://vimhelp.org/vim_faq.txt.html
下载:              https://www.vim.org/download.php
```

问问题，找答案：https://vi.stackexchange.com/
"Vi and Vim Stack Exchange is a question and answer site for people using the vi and Vim families of text editors"

讨论 Vim 的新闻组：

news usenet

comp.editors

这个新闻组也讨论其它编辑器。如果你写的是有关 Vim 的话题，要记得讲清楚。
可以在这里访问：
https://groups.google.com/forum/#!topic/comp.editors

mail-list maillist

Vim 有许多邮件列表：

<vim@vim.org>

vim-use vim_use

讨论现存版本的 Vim：有用的映射，提问、回答，以及到哪里得到特定的版本，等等。有相当多的人在关注着这个列表，而且回答问题，包括初学者的问题。这里你可以放心大胆地提问题。

<vim-dev@vim.org>

vim-dev vim_dev vimdev

讨论改进 Vim：新功能，移植，补丁，测试版本等等。

<vim-announce@vim.org>

vim-announce vim_announce

通告 Vim 的新版本；也包括测试版和不同系统的移植版。本邮件列表只读。

<vim-mac@vim.org>

vim-mac vim_mac

讨论如何使用和改进 Macintosh 系统里的 Vim。

访问 http://www.vim.org/maillist.php 以获取最新的信息。

注意：
- 任何人都能看见归档，例如通过 Google groups。如有问题请搜索。
- 你只有在订阅了这些邮件列表之后，才能在上面发布信息。
- 首个信息需要审批，所以可能要几个小时才能登出。
- 为了避免垃圾邮件，你需要用与你订阅该邮件列表相同的邮件地址发送信息。

subscribe-maillist

如果你想加入邮件列表，发 e-mail 至：

<vim-subscribe@vim.org>

确保你的发件人的地址 "From:" 是正确的。邮件列表服务器会回信告诉你如何订阅。

maillist-archive

要得到更多信息和以前的归档，请查看 Vim 邮件列表页面：
http://www.vim.org/maillist.php

报告漏洞 (bug): bugs bug-reports bugreport.vim

有两种报告漏洞的方法，两个都可以：
1. 发送漏洞报告至：Vim Developers <vim-dev@vim.org>
   这是一个邮件列表，你需要先以为成员，很多人会看到邮件。如果某些情况下你不希望如此，例如关于安全方面的问题，发送邮件到 <bugs@vim.org>，只有 Vim 维护者才看得到 (Bram 本人)。
2. 在 GitHub 上提交新议题：https://github.com/vim/vim/issues
   文本会转发到 vim-dev 邮件列表。

请尽量使用精练的语言描述；回答邮件里的问题势必会占用维护者开发 Vim 的时间！要给出导致漏洞的可重复的例子，并且尽可能找出哪个选项或其它东西触发该漏洞。

推荐这样启动 Vim:

```
vim --clean -u reproduce.vim
```

其中 reproduce.vim 是重现问题的一个脚本。如果相关的话，换一台机器试试 (有可能是 MS-Windows 特定的漏洞哦？)。

如果你能解决这个问题，把补丁发给我！如果你在 https://github.com/vim/vim 上创建新的拉取请求，自动检查会运行并报告明显的问题。不过你也可以用 email 发送补丁 (使用附件可以避免空白的变动)。

提供你正在使用的 Vim 版本和配置信息会有帮助。下面这个命令能够帮你收集这些信息：

```
:so $vimruntime/bugreport.vim
```

这条命令会在当前目录下创建一个名为 "bugreport.txt" 的新文件，里面包含了许多与你当前环境有关的信息。在发送之前检查一下，不要泄漏你的机密！

如果 Vim 崩溃了，试着找出哪里出了问题。这里可以提供帮助： debug.txt 。

如果有问题，或者想知道是不是某个问题已经被修正但不知道哪里能找到补丁，加入 vim-dev 邮件列表并在那里发文询问。 maillist

year-2000 Y2K

因为 Vim 在编辑时没有用到日期，所以不必担心 2000 年问题。Vim 使用的时间格式是从 1970 年 1 月 1 日至今的秒数。它被用做被编辑文件和交换文件的时间戳，不是太重要，只多只能引起一些警告。

当一个 32 位的整型数容不下这些秒数的时候，可能会有 2038 年的问题。这取决于你的编译器、连接库以及操作系统。特别是用到 time_t 和 ctime() 函数的时候。time_t 以 4 字节大小存储在交换文件中。但是它只是用来在恢复时显示文件的日期/时间，不会影响到正常的编辑。

Vim 中的 strftime() 函数直接调用系统函数 strftime()。localtime() 调用系统函数 time()。getftime() 使用系统函数 stat() 返回的时间。如果你的系统库函数兼容 2000 年问题，Vim 也就没问题。

用户可能在创建 Vim 脚本时用到了外部命令。它们可能会引入 2000 年问题，但这不是 Vim 本身的问题。

---

3. 制作群 credits author Bram Moolenaar

Vim 的大部分是由 Bram Moolenaar <Bram@vim.org> 开始的。

部分文档来自于由：
W.N. Joy
Alan P.W. Hewett
Mark Horton
编写的多种 Vi 手册。

Vim 编辑器是在 Stevie 的基础上并汲取了其它软件的思想开发出来的。这里提一下为这些软件工作的人。还有许多朋友通过发补丁以及对 Vim 的建议和反馈的方式，给予我很大的帮助，在这里就不一一提及了。

如果没有这些人的帮助，Vim 永远不可能成为现在这个样子。

Ron Aaron	修改 Win32 GUI 版本
Mohsin Ahmed	加密
Zoltan Arpadffy	VMS 版本的移植工作
Tony Andrews	Stevie 的作者
Gert van Antwerpen	对 DJGPP 的 MS-DOS 版本修改
Berkeley DB(3)	交换文件实现的想法
Keith Bostic	Nvi 作者
Walter Briscoe	Makefile 的更新和各种补丁
Ralf Brown	MS-DOS 上的 SPAWNO 库
Robert Colon	提供了许多有用的建议
Marcin Dalecki	GTK+ 图形界面的移植，工具栏的图标，gettext()
Kayhan Demirel	给我发送乌干达的信息
Chris & John Downey	xvi (多窗口的思想)
Henk Elbers	最先移植到 VMS
Daniel Elstner	GTK+ 2 移植
Eric Fischer	Mac 系统移植、'cindent'、其它的改进
Benji Fisher	解答用户问题
Bill Foster	Athena 图形界面移植
Google	让我每星期可以在 Vim 上工作一天
Loic Grenie	xvim (多窗口思想)
Sven Guckes	Vim 促进者和前网站维护者
Darren Hiebert	Exuberant ctags
Jason Hildebrand	GTK+ 2 移植
Bruce Hunsaker	改进了 VMS 移植版本
Andy Kahn	加入了 Cscope 支持，GTK+ 的图形界面移植
Oezguer Kesim	Vim 邮件列表的维护者
Axel Kielhorn	Macintosh 移植
Steve Kirkendall	Elvis 作者
Roger Knobbe	Windows NT 移植
Sergey Laskavy	来自莫斯科的帮助
Felix von Leitner	前任 Vim 邮件列表的维护者
David Leonard	将 Python 扩展移植到 Unix
Avner Lottem	在窗口里从左向右编辑
Flemming Madsen	X11 客户端－服务器，许多功能和补丁
Tony Mechelynck	回答了许多用户的问题
Paul Moore	Python 接口的扩展，许多补丁
Katsuhito Nagano	多字节版本
Sung-Hyun Nam	多字节版本
Vince Negri	加强了 Win32 图形界面和控制台版本
Steve Oualline	第一本介绍 Vim 的书的作者 frombook

Dominique Pelle	Valgrind 报告及许多修正
A.Politz	很多漏洞报告及一些修正
George V. Reilly	最先从事 Win32 及其 GUI 版本的移植工作
Stephen Riehm	bug 收集者
Stefan Roemer	各种补丁以及为用户提供帮助
Ralf Schandl	IBM OS/390 移植
Olaf Seibert	DICE 和 BeBox 版本，强化了正则表达式
Mortaza Shiran	Farsi 补丁
Peter da Silva	termlib
Paul Slootman	OS/2 移植
Henry Spencer	正则表达式
Dany St-Amant	Macintosh 移植
Tim Thompson	Stevie
G. R. (Fred) Walter	Stevie
Sven Verdoolaege	Perl 接口
Robert Webb	命令行补全，GUI 和许多补丁
Ingo Wilken	Tcl 接口
Mike Williams	PostScript 打印
Juergen Weigert	Lattice 版本，AUX 改进，UNIX 和 MS-DOS 移植，及 autoconf
Stefan 'Sec' Zehl	vim.org 的维护者
Yasuhiro Matsumoto	许多 MS-Windows 改进
Ken Takata	修正和功能
Kazunobu Kuriyama	GTK 3
Christian Brabandt	许多修正，功能，用户支持等等
Yegappan Lakshmanan	许多 quickfix 功能

我希望感谢所有向我报告 bug 和给我建议的人，但是人太多在这里不能一一提及。如果没有他们的帮助，Vim 不可能成为现在这个样子：他们是 Vim 的生命之源。
love peace friendship gross-national-happiness

在这篇文档里，有些地方还会提到其它版本的 Vi ：

Vi vi

Vi "原始版本"。如果不提及，这里指 Sun OS 4.x 里出现的 Vi。 ":version" 返回结果 "version 3.7, 6/7/85"。有时也指别的版本的 Vi。只在 Unix 里运行。现在可以在遵循 BSD 风格许可证的条件下得到源代码。Vi 的更多信息：
http://ex-vi.sourceforge.net/

Posix

Posix 来自 IEEE 1003.2 PART 2: Shell and utilities。通常叫做 "Posix"。这篇文本描述了 Vi 应该怎样工作。
参见 posix-compliance 。

Nvi

Nvi "新" vi 的缩写。4.4 BSD 和 FreeBSD 带的 Vi 版本。很好地兼容了原始版本的 Vi，并且增加了一些功能。这里所指的版本是 1.79。 "version" 返回信息是 "version 1.79 (10/23/96)"。虽然有一个 1.81 版正在开发，但是最近几年一直没有新版本放出。
源代码自由发放。

Elvis

Elvis 另一个 Vi 的克隆，作者是 Steve Kirkendall。非常简洁但是不及 Vim 灵活。开发停止了，猫王已经不在了 (译者注：原文是 Elvis has left the building，意为表演结束了，Elvis 这里有双关意思)！源代码自由发放。

Neovim

Neovim 一个 Vim 的克隆。2014 年从 Vim 源代码分叉并朝着不同的方向发展。和

github 绑定很紧且有多许多的依赖，使得开发更复杂且移植性受限。代码已经被重整过，使补丁和 Vim 不能交换使用。支持远程 GUI 和脚本语言的集成。

---

4. 记法 notation

如果你在阅读本文时使用了语法高亮功能，那些不是用来直接输入的文字通常会用特殊的颜色显示。例如在 (), {} 和 <> 里的内容，以及 CTRL-X。

注意 Vim 的命令包括所有可能的字符。有时候 [], {} 和 <> 也需要输入，这些都应该在上下文中有清楚地说明。

[]	方括号里的字符是可选的。
[count]	count [count] 可选的数值，可用在命令前以重复该命令。在没有特殊说明的情况下，如果省略该数值，会认为该数值是 1。注意，在本手册里 [count] 没有出现在命令的描述中，只在解释中有提及。这样做可以使查找命令更简单。如果打开了 'showcmd' 选项，(部分) 输入的计数 (count) 会显示在窗口底部。可以用 <Del> 键删除最后输入的数位 (N<Del>)。
["x]	[quotex] 可选的用于存储文本的寄存器。参见 registers 。x 代表从 'a' 到 'z' 或从 'A' 到 'Z' (大小写英文字母) 以及 '"' 中的一个字符，在某些情况下 (比如在 put 命令中) 还可以是 '0' 到 '9'、'%'、'#' 或其它字符。大写和小写字母指的是同一个寄存器，但是使用小写字母会覆盖原先寄存器中的内容，而使用大写字母会将新的内容追加在原内容之后。若省略 ""x" 或使用 """，则把文本存储到无名寄存器中。
{}	{} 命令中花括号里的内容是必须出现的，但是可以取不同的值。Vim 与 Vi 的不同之处也同样在花括号里给出 (这可以通过上下文清楚地分辨出来)。
{char1-char2}	{char1-char2} 在 char1 到 char2 区间内的一个字符，例如: {a-z} 是一个小写字母。多个区间可以串接，例如: {a-zA-Z0-9} 代表任一数字或字母。
{motion}	{motion} movement 移动光标的命令，或动作。它们在 motion.txt 里有解释。例如: w 移动至下一个单词的首部 b 移动至当前单词的首部 4j 下移四行 /the<cR> 移动至下一个 "The" 这些命令可以跟在一个操作符 operator 后面，以指定操作符所作用的范围。 - 如果动作包含计数，并且操作符中也包含计数，那么最后的效果是两个数值的乘积。例如: "2d3w" 删除六个单词。 - 动作可以是反向的，例如: "db" 删除至当前单词首部。 - 动作也可以是鼠标点击。但不是所有的终端都支持鼠标。 - 命令 ":omap" 可以在操作符等待状态时对字符进行映射。 - 可以使用 ex 命令来移动光标。这可以用来调用函数做一些复杂移

动。无论你使用什么 ":" 命令，这里的动作总是面向字符且非包含 (exclusive) 的。意思是你不可能包括不算换行符的一行的最后一个字符 (除非设置了 'virtualedit' 属性)。
如果 ex 命令改变了操作符开始位置之前的文本或者跳转到别的缓冲区，那么结果不可预测。往后改变文本是可以的。如果当前的缓冲区没有卸载，跳转到别的缓冲区也是可以的。

{Visual}

{visual} 选中的文本区域。用 "v"、"V" 或者 CTRL-V 激活该模式，接着使用移动光标的命令改变选中文本的另一端。该操作在操作符 operator 之前使用，被高亮的文本是操作符的作用范围。
参见 visual-mode 。

<character>

<character> 下面表格中的特殊字符，可以带修饰符，也可以是带有修饰符的单个键 ASCII 字符。

'character'

'c' 单个 ASCII 字符。

CTRL-{char}

CTRL-{char} 作为控制字符输入的 {char}；也就是说，按住 CTRL 键再按 {char}。{char} 是大写字母还是小写字母都没关系；就是说 CTRL-A 和 CTRL-a 是一样的。但是在有些终端里，用 SHIFT 键会生成别的编码，如果是这样，就别这么用。

'option'

'option' 用单引号括起来的是选项，或参数。可以被赋值。参见 options 。

quotecommandquote

"command" 用双引号括起来的是可以输入的命令。
command 新风格的命令，以便和其他用引号括起的文本和字符串相区隔。
(译者注：这里是反引号括起，但显示被隐藏)

key-notation key-codes keycodes

下面这些按键的名称文档里会用到。它们也可以用在 ":map" 命令里 (按下 CTRL-K 再按下你想输入的键就可以输入该键的键名)。

标识符	含义	等价于	十进制数值	
<Nul>	零	CTRL-@	0 (存储为 10)	<Nul>
<BS>	退格键	CTRL-H	8	backspace
<Tab>	制表符	CTRL-I	9	tab Tab
<NL>	换行符	CTRL-J	10 (用作 <Nul>)	linefeed
<FF>	换页符	CTRL-L	12	formfeed
<CR>	回车符	CTRL-M	13	carriage-return
<Return>	同 <CR>			<Return>
<Enter>	同 <CR>			<Enter>
<Esc>	转义	CTRL-[	27	escape <Esc>
<Space>	空格		32	space
<lt>	小于号	<	60	<lt>
<Bslash>	反斜杠	\	92	backslash <Bslash>
<Bar>	竖杠	|	124	<Bar>

<Del>	删除	127	
<CSI>	命令序列引入	ALT-Esc 155	<CSI>
<xCSI>	图形界面的 CSI		<xCSI>
<EOL>	行尾（可以是 <CR>、<LF> 或 <CR><LF>，根据不同的系统和 'fileformat' 而定）		<EOL>
<Up>	光标上移键	cursor-up	cursor_up
<Down>	光标下移键	cursor-down	cursor_down
<Left>	光标左移键	cursor-left	cursor_left
<Right>	光标右移键	cursor-right	cursor_right
<S-Up>	Shift＋光标上移键		
<S-Down>	Shift＋光标下移键		
<S-Left>	Shift＋光标左移键		
<S-Right>	Shift＋光标右移键		
<C-Left>	Control＋光标左移键		
<C-Right>	Control＋光标右移键		
<F1> - <F12>	功能键 1 到 12	function_key	function-key
<S-F1> - <S-F12>	Shift＋功能键 1 到 12	<S-F1>	
<Help>	帮助键		
<Undo>	撤销键		
<Insert>	Insert 键		
<Home>	Home	home	
<End>	End	end	
<PageUp>	Page-up	page_up	page-up
<PageDown>	Page-down	page_down	page-down
<kHome>	小键盘 Home（左上）	keypad-home	
<kEnd>	小键盘 End（左下）	keypad-end	
<kPageUp>	小键盘 Page-up（右上）	keypad-page-up	
<kPageDown>	小键盘 Page-down（右下）	keypad-page-down	
<kPlus>	小键盘 +	keypad-plus	
<kMinus>	小键盘 -	keypad-minus	
<kMultiply>	小键盘 *	keypad-multiply	
<kDivide>	小键盘 /	keypad-divide	
<kEnter>	小键盘 enter	keypad-enter	
<kPoint>	小键盘 小数点	keypad-point	
<k0> - <k9>	小键盘 0 到 9	keypad-0	keypad-9
<S-...>	Shift＋键	shift	<S-
<C-...>	Control＋键	control	ctrl <C-
<M-...>	Alt＋键 或 meta＋键	meta	alt <M-
<A-...>	同 <m-...>	<A-	
<D-...>	Command＋键（只用于苹果机）	<D-	
<t_xx>	termcap 里的 "xx" 入口键		

备注：Shift＋方向键，帮助键和撤销键只在少数终端里有效。在 Amiga 机器上，Shift＋功能键 10 生成命令序列引入码（CSI），也用于键序列。只有再按一个键，该代码才能被识别出来。

备注：Delete 键有两个键码。一般是十进制的 ASCII 码 127，这个总能认出来。但有些 delete 键会发送 termcap 入口项 "kD" 的值。这两个值有同样的效果。
参见 :fixdel 。

备注：小键盘上的按键与其对应的 "正常" 按键一样。例如，<kHome> 与 <Home> 有同样

的效果。如果小键盘上的键发送的原始键码与 "正常" 键相同，那么将被认为是按下了 "正常" 键。例如，如果 <kHome> 发送的键码与 <Home> 相同，当按下 <khome> 的时候，Vim 会认为你按下了 <Home>。<kHome> 的映射此时不会有效。

<>

例子中经常使用 <> 记法。有时这只是用来说明你需要输入什么，但经常它需要照本义键入，例如在 ":map" 命令里。规则是：

1. 任何可显示的字符都可以直接键入，反斜杠和 '<' 除外。
2. 反斜杠用两个反斜杠表示 "\\"，或者用 "<Bslash>"。
3. 真正的 '<' 用 "\<" 或 "<lt>" 表示。只有在没有歧义的时候才可以直接用 '<' 表示。
4. "<key>" 的意思是特殊键。其含义上面的表格都有介绍，下面是一些例子：

<Esc>	Escape 键
<C-G>	CTRL-G
<Up>	光标上移键
<C-LeftMouse>	Control＋鼠标左键点击
<S-F11>	Shift＋功能键 11
<M-a>	Meta- a (第 8 位置位的 'a')
<M-A>	Meta- A (第 8 位置位的 'A')
<t_kd>	termcap 的 "kd" 入口 (光标下移键)

尽管你可以指定 {char} 为多字节字符的 <M-{char}>，Vim 可能不知道那对应什么字节序列，所以不能工作。

如果你想在 Vim 里使用 <> 的完整记法，必须确定 'cpoptions' 里不包括 '<' 标志位 (如果没有置位 'compatible'，默认值就是这样)。

```
:set cpo-=<
```

这里，<> 记法使用 <lt> 来抵消键名的特殊含义。使用反斜杠也可以，但是需要去掉 'cpoptions' 里的 'b' 标志位。

下面的例子是把 CTRL-H 映射成六个字符 "<Home>":

```
:imap <C-H> \<Home>
:imap <C-H> <lt>Home>
```

第一种方法只有在 'cpoptions' 里没有 'B' 标志位才好用。第二种总成立。
要在映射中得到按本义出现的 "<lt>" 这几个字符：

```
:map <c-l> <lt>lt>
```

对于映射，缩写和菜单命令你可以用复制-粘贴直接使用手册里的例子。你也可以手动键入它们，包括 '<' 和 '>'。但是在其它命令里，比如 ":set" 和 "autocmd"，这是 不行的！

---

5. 了解一下模式

vim-modes-intro vim-modes

Vim 有七种基本模式：

Normal Normal-mode command-mode

普通模式	在普通 (normal) 模式里，你可以输入所有的普通编辑命令。启动编辑器之后就处于该模式 (除非你置位了 'insertmode' 选项，下面会有说明)。亦称为命令 (command) 模式。
可视模式	该 (visual) 模式与普通模式相似，但是移动光标的命令会扩展高亮的区域。非移动命令作用于高亮的区域。见 Visual-mode 。

如果置位了 'showmode' 选项，窗口底部会出现 "--VISUAL--" 字样。

选择模式

该 (select) 模式看起来与 MS-Windows 的选择模式很像。键入一个可显示字符会删掉所有被选择的字符并且进入插入模式。见 Select-mode 。
如果置位了 'showmode' 选项，窗口底部会出现 "--SELECT--" 字样。

插入模式

在插入 (insert) 模式下你键入的内容会插入到缓冲区。见 Insert-mode 。
如果置位了 'showmode' 选项，窗口底部会出现 "--INSERT--" 字样。

命令行模式

在命令行 (command-line 或 Cmdline) 模式里你可以在窗口底部输入一行命令。执行 Ex 命令使用 ":"，查找使用 "?" 和 "/"，调用过滤命令使用 "!"。 Cmdline-mode

Ex 模式

与命令行模式类似，但是执行完一条命令后，你会继续停留在 Ex 模式。提供非常有限的命令行编辑。 Ex-mode

终端作业模式

在终端窗口中和作业互动。输入的键传给作业，作业输出在终端窗口中显示。 terminal 说明如何切换到其它模式。

还有七种 附加 模式。这些都是在基本模式的基础上变化而来的:

Operator-pending Operator-pending-mode

操作符等待模式

该 (operator-pending) 模式与普通模式类似，但是在操作符之后，Vim 等待一个动作 {motion} 以确定操作符的作用范围。

替换模式

替换 (replace) 模式是插入模式的特殊情况。你可以做和插入模式里相同的事情，但是你输入的字符会覆盖原先的字符。见 Replace-mode 。
如果置位了 'showmode' 选项，窗口底部会出现 "-- REPLACE --" 字样。

虚拟替换模式

虚拟替换 (virtual replace) 模式类似于替换模式，但你替换的不是文件的字符，而是屏幕上占据的位置。见 Virtual-Replace-mode 。
如果置位了 'showmode' 选项，窗口底部会出现 "-- VREPLACE --" 字样。

插入普通模式

在插入模式里输入 CTRL-O (见 i_CTRL-O) 可进入该 (insert normal) 模式。与普通模式类似，但是只能执行一条命令然后返回到插入模式。
如果置位了 'showmode' 选项，窗口底部会出现 "-- (insert) --" 字样。

终端普通模式

在终端窗口中使用普通模式。不能作任何修改。插入命令如 "a" 或 "i" 会返到终端作业模式。

插入可视模式

在插入模式里进行可视模式的选择，例如，在插入模式里键入

CTRL-O 然后键入 "v" 或 "V" 或 CTRL-V，可进入该 (insert visual) 模式。可视选择结束时，返回到插入模式。
如果置位了 'showmode' 选项，窗口底部会出现 "-- (insert) VISUAL --" 字样。

插入选择模式

在插入模式里启动选择模式可进入该 (insert select) 模式。例如，用鼠标拖拽或 <S-Right>。如果选择模式结束，Vim 返回插入模式。
如果置位了 'showmode' 选项，窗口底部会出现 "-- (insert) SELECT --" 字样。

---

6. 从一种模式切换到另一种模式 mode-switching

如果因为某种原因你不知道处于哪种模式，你总是可以通过按两下 <Esc> 返回普通模式。不过不能在 Ex 模式里这么做，那里，用 ":visual" 返回普通模式。
如果在你按下 <Esc> 后发现屏幕闪烁或者听到响声，这证明你已经回到普通模式。
然而，在插入模式里按下 CTRL-O 后再按下 <Esc> 也会听到喇叭的响声，但是你仍然处于插入模式，再按一下 <Esc> 即可。

i_esc

切换 从 模式	到 模式 普通	可视	选择	插入	替换	命令行	Ex
普通		v V ^V	*4	*1	R gR	: / ? !	Q
可视	*2		^G	c C	--	:	--
选择	*5	^O ^G		*6	--	--	--
插入	<Esc>	--	--		<Insert>	--	--
替换	<Esc>	--	--	<Insert>		--	--
命令行	*3	--	--	:start	--		--
Ex	:vi	--	--	--	--	--	

- 不可用
-- 不可能

*1 从普通模式切换到插入模式可以用这些命令: "i"、"I"、"a"、"A"、"o"、 "O" "c"、"C"、"s" 或 S"。
*2 从可视模式切换到普通模式可以使用一个不移动光标的命令，这样命令执行后就会返回普通模式。也可以使用 <Esc> "v"、"V" 或 "CTRL-V" (见 v_v)，这几个命令可以没有副作用地终止可视模式。
*3 从命令行模式切换至普通模式:
- 敲 <CR> 或 <NL>，使得键入的命令得以执行。
- 删除整行 (例如，用 CTRL-U) 最后再按一下 <BS>。
- 敲 CTRL-C 或 <Esc> 可以退出命令行模式而不执行已经键入的命令。
在最后一种情况里，<Esc> 可能被 'wildchar' 定义，这样 <Esc> 就会作为命令行模式里的命令补全键。你可以再按一下 <Esc> 忽略补全的命令。
*4 从普通模式切换到选择模式:
- 如果 'selectmode' 里包含 "mouse"，当使用鼠标选择文本时即可切换。
- 如果 'selectmode' 里包含 "key"，那么按住 Shift 键然后用不可显示的命令来移动光标。
- 如果 'selectmode' 包含 "cmd"，那么用 "v"、"V" 或 "CTRL-V"
- 使用 "gh"、"gH" 或 "g CTRL-H"
*5 使用不可显示的命令移动光标 (不要按 Shift) 可以从选择模式转到普通模式。

*6 输入一个可显示字符可以从选择模式切换到插入模式。插入该字符的同时删除被选择的部分。

如果打开了 'insertmode' 选项，编辑文件时将直接进入插入模式。

CTRL-_CTRL-N i_CTRL-_CTRL-N c_CTRL-_CTRL-N v_CTRL-_CTRL-N

另外，CTRL-\ CTRL-N 或 <C-\><C-N> 可以用来从其它模式切换到普通模式。这样可以确保 Vim 处于普通模式而不会像 <Esc> 那样产生响声。然而，它不能用在 Ex 模式。当用在一个需要参数的命令，诸如 f 、 m 等之后，等待时间由 'ttimeoutlen' 决定。焦点在终端窗口时，CTRL-\ CTRL-N 只为一个命令切换到普通模式，见 t_CTRL-_CTRL-N 。

CTRL-_CTRL-G i_CTRL-_CTRL-G c_CTRL-_CTRL-G v_CTRL-_CTRL-G

如果置位了 'insertmode'，则 CTRL-\ CTRL-G 或 <C-\><C-G> 切换到插入模式，否则切换至普通模式。这可以让 Vim 处于 'insertmode' 所指定的模式，而不必关心当前的模式。

Q mode-Ex Ex-mode Ex EX E501

Q 切换至 "Ex" 模式。有点像一个接一个的 ":" 命令，除了：
- 你不需要输入 ":"。
- 屏幕不会在每个命令执行完后更新。
- 没有正常的命令行编辑。
- 映射和缩写无效。

实际上，你正在使用一个 "标准的" 行编辑器。(<Del> 或 <BS> 删除一个字符，CTRL-U 删除整行)。
如果使用 "ex" 调用 Vim，Vim 缺省进入该模式。
用 ":vi" 命令 :visual 退出 "Ex" 模式。
注意：老版本的 Vim 里，"Q" 用来排版文本，现在那个功能使用 gq 。但是如果你使用 vimrc_example.vim ，那么 "Q" 会和 "gq" 一样。

gQ

gQ 同 "Q"，切换到 "Ex" 模式，但与一条接一条的 ":" 命令相当。所有的行编辑、补全等功能都可用。
用 ":vi" 命令 :visual 退出 "Ex" 模式。

---

7. 窗口里的东东 window-contents

在普通模式和插入/替换模式里屏幕窗口显示当前缓冲区里的内容：所见即所得。但有两个例外：
- 如果 'cpoptions' 选项里包含 '$'，并且改动在一行之内，被改动的文本没有直接删除，而是在被删除的最后一个字符放上 '$'。

- 当在一个窗口输入文本的时候，监视相同文本的其它窗口不会立即更新，直到插入结束为止。

除非关闭了 'wrap' 选项，否则长度大于窗口宽度的文本会被回绕（见下面）。置位 'linebreak' 选项可以使文本只在空白字符处被回绕。

如果已经到达了缓冲区的最后一行，但是当前窗口还有显示空间，Vim 会在末行之后的第一列显示一个 '~'，像这样：

```
+----------------------+
```

```
|some line              |
|last line              |
|~                      |
|~                      |
+-----------------------+
```

也就是说，'~' 意味着到达了缓冲区的末尾。

如果当前窗口不能完整的显示最后一行，Vim 会在窗口的最后几行的第一列用 '@' 字符表示这一点，就像这样：

```
+-----------------------+
|first line             |
|second line            |
|@                      |
|@                      |
+-----------------------+
```

也就是说，'@' 表示这一行窗口里放不下。

如果标志位 "lastline" 出现在 'display' 选项里，你就不会看到窗口最左边的 '@' 字符。如果最后一行不能完全放到窗口里，就只显示能放下的部分，并且该行的最后三个字符用 "@@@" 代替，像这样：

```
+-----------------------+
|first line             |
|second line            |
|a very long line that d|
|oesn't fit in the wi@@@|
+-----------------------+
```

如果文本中只包含一行，窗口里放不下，这是很特殊的情况。Vim 只会显示光标所在位置周围的部分字符。不会显示特殊字符，这样你可以编辑该行的任何部分。

在 'highlight' 选项里出现的 '@' 可以用来为 '@' 和 '~' 设置特殊的高亮方式。这使它们能够和缓冲区里的真实字符得以区分。

选项 'showbreak' 定义回绕行的前端包含的字符串。

wrap-off

如果关闭了 'wrap' 选项，长行不会被回绕。只显示能放到窗口里的那一部分。如果光标移动到没有显示的部分，屏幕会水平方向滚动。这样做的好处是可以正常显示文本列而窗口里放不下的行也可编辑。缺点是你不能一次看见一行的所有字符。选项 'sidescroll' 可以设定滚动列数的最小值。

所有正常的 ASCII 字符直接显示在屏幕上。<Tab> 用相应数量的空白代替。其余不可显示的字符用 "^{char}" 代替。其中的 {char} 为不可显示字符的值加 64。比如，响铃字符 (7) 显示为 "^G"。ASCII 值在 127 至 160 的字符用 "~{char}" 代替，{char} 是该字符的数值减去 64。这些字符在屏幕上要多占一个位置，而光标只能放在其中的第一个位置上。

如果你置位了 'number' 选项，所有的行前面会加行号。提示：如果你不喜欢回绕的行混淆了行号，可以把 'showbreak' 选项设置成八个空格：

```
:set showbreak=\ \ \ \ \ \ \ \
```

如果你置位了 'list' 选项，<Tab> 字符不会显示成多个空格，而是 "^I"。'$' 字符会出现在每一行的末尾，这样你可以发现尾部的空白字符。

在命令行模式里，只有命令行自己被正确地显示。缓冲区内容的显示从你返回命令模式（即普通模式）的那一刻进行更新。

窗口的最后一行是用来显示状态和其它信息的。状态信息只有在选项打开的时候才可用：

状态信息	选项	缺省	Unix 缺省
当前模式	'showmode'	开	开
命令字符	'showcmd'	开	关
光标位置	'ruler'	关	关

当前模式是 "-- INSERT --" 或 "-- REPLACE --"，见 'showmode'。命令字符是你输入的但还没执行的命令。

如果你用一个慢速终端，你可以关闭状态信息以加速编辑：

```
:set nosc noru nosm
```

如果出错，错误信息会保持至少一秒钟（使用反转显示）。

有些命令会显示有多少行受影响。只有行数超过 'report' 控制（默认是 2）的阈值，这才会发生。

在 Amiga 系统上，Vim 在一个 CLI 窗口里运行。Vim 的名字和当前文件的全名在标题栏里显示。窗口大小改变的时候，Vim 自动重画窗口。你可以把窗口搞的很小，但是如果太小了就连一行文本也装不下了。尽量让窗口保持至少 40 个字符的宽度，这样可以阅读最后一行（状态行）的大多数信息。

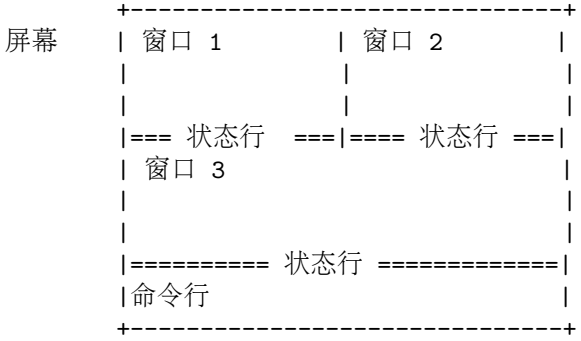
在大多数 Unix 系统上，窗口大小的调整可以正确地被 Vim 识别和处理。

---

8. 术语 definitions

缓冲区	包含文本行，通常从一个文件中读入。
屏幕	Vim 所用的整个工作区域。可以是一个终端模拟窗口。也被叫做 "Vim 窗口"。
窗口	缓冲区视图。同一个缓冲区可有多个窗口。

一个屏幕包含一个或多个窗口，被状态行和屏幕底部的命令行分割。



命令行也用来输出信息。如果命令行没有足够的空间来显示信息，屏幕会上滚。

四种不同类型的 "行":

缓冲 行	缓冲区里的行。与读出/写入文件的行的内容相同。每一行可以包含成千上万个字符。
逻辑 行	应用过折叠的缓冲行。关闭折叠里的缓冲行被折叠成一个逻辑行: "+-- 99 行 折叠"。逻辑行也可以包含成千上万个字符。
窗口 行	窗口里显示的行: 被回绕的逻辑行，换行符，等等，都在这里应用。窗口行的长度不能超过窗口允许的宽度，多出的部分会被回绕或截断。
屏幕 行	Vim 使用的屏幕中的行。包括所有窗口的窗口行、状态行以及命令行。它们的长度受屏幕的宽度的限制。如果命令行里命令的长度大于屏幕的宽度，则自动回绕并滚屏以腾出空间。

```
缓冲 行          逻辑 行          窗口 行          屏幕 行

1. 一            1. 一            1. +-- 折叠      1.  +-- 折叠
2. 二            2. +-- 折叠      2. 五            2.  五
3. 三            3. 五            3. 六            3.  六
4. 四            4. 六            4. 七            4.  七
5. 五            5. 七                             5.  === 状态行 ===
6. 六                                              6.  aaa
7. 七                                              7.  bbb
                                                   8.  ccc ccc c
1. aaa           1. aaa           1. aaa           9.  cc
2. bbb           2. bbb           2. bbb           10. ddd
3. ccc ccc ccc   3. ccc ccc ccc   3. ccc ccc c     11. ~
4. ddd           4. ddd           4. cc            12. === 状态行 ===
                                  5. ddd           13. (命令行)
                                  6. ~
```

---

vim:tw=78:ts=8:noet:ft=help:norl:

helphelp.txt For Vim version 8.2. 最近更新: 2019年12月

VIM 参考手册

by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

帮助文件之帮助 helphelp

1. 帮助命令 online-help
2. 翻译帮助文件 help-translated
3. 编写帮助文件 help-writing

---

1. 帮助命令 online-help

help <Help> :h :help <F1> i_<F1> i_<Help>

<Help> 或
:h[elp]

打开一个窗口并以只读方式显示帮助文件。如果已经打开了一个帮助窗口，就继续使用那个窗口。不然，如果当前窗口占据了屏幕的完整宽度或者至少有 80 个字符宽，帮助窗口会出现在当前窗口的正上方。再不然，新窗口就开在最上方。
如果主帮助文件有多个语言版本，'helplang' 选项选择使用的语言。

{subject} E149 E661

:h[elp] {subject}

类似于 ":help"，但附加跳转到 {subject} 标签上。
例如:

```
:help options
```

{subject} 可以包含 "*"、"?" 和 "[a-z]" 这样的通配符:

```
:help z?      跳到任何包含 "z" 的命令的帮助
:help z.      跳到关于 "z." 的帮助
```

但如果标签存在，按本义接受:

```
:help :?      jump to help for ":?"
```

如果不能完全匹配该模式，或者有多个匹配，那就使用 "最好" 的匹配。这里，有一个相当复杂的算法来排定匹配的优先顺序。它的计算涉及到以下诸方面:
- 大小写完全相同的优先于大小写不完全相同的。
- 开始于非字母数字之后的优先于从单词中间开始的。
- 位于或接近标签开始处的优先于离此距离较远的。
- 匹配的字母数字字符越多越优先。
- 匹配越短的越优先。

如果该 {subject} 有多个语言的帮助，'helplang' 选项用来选择所用的语言。要找到某个标签某个特定的语言版本，附加上 "@ab"，其中 "ab" 是双字母的语言代码。参见 help-translated 。

注意 给出越长的 {subect}，找到的匹配就越少。使用命令行补全功能（在 ":help subject" 之后输入 CTRL-D c_CTRL-D ），你会了解这是如何工作的。

如果有多个匹配，你可以通过敲击 CTRL-D 得到它们的列表。例如：

```
:help cont<Ctrl-D>
```

要找寻 CTRL-V 的帮助不需要打 ":help CTRL-V"，可用：

```
:help ^V
```

这也适用于和其它字符混用的情况。例如寻找 CTRL-V 在插入模式的帮助：

```
:help i^V
```

也可以先 ":help"，然后在帮助窗口里使用 ":tag {pattern}"。这时，":tnext" 命令可以跳转到后一个匹配，"tselect" 列出所有的匹配并让你选择一个。

```
:help index
:tselect /.*mode
```

如果没有参数，你会看到 "help" 的匹配而不是列出所有可能的匹配（那会非常慢）。
显示的匹配个数限于 300 个。

:help 命令可以后面跟一个 '|' 并紧跟另外一个命令。不过，你不需要在 help 命令里转义 '|'。所以下面这些都没问题：

```
:help |
:help k| only
```

注意 如果 '|' 之前有空格，它是 ":help" 参数的一部分。(译者注：前者查找 | 的帮助，后者查找 k 的帮助并执行 only 命令使帮助窗口成为唯一窗口)
你也可以用 <LF> 或 <CR> 来分隔 help 命令和其后的命令。你需要先输入 CTRL-V，再输入 <LF> 或 <CR>。例如：

```
:help so<C-V><CR>only
```

:h[elp]! [subject]　类似于 ":help"，但在非英语帮助文件里，先查找包含和当前文件相同语言的标签的文件。参见 help-translated 。

:helpc :helpclose

:helpc[lose]　如果有的话，关闭一个帮助窗口。
Vim 会试图恢复打开帮助窗口之前原先的窗口布局（包括光标位置）。这可能会触发若干自动命令。

:helpg :helpgrep

:helpg[rep] {pattern}[@xx]
搜索所有的帮助文本并给出一个匹配 {pattern} 行的列表。跳转到第一个匹配。
可选的 [@xx] 指定只寻找 "xx" 语言里的匹配。
你可以用 quickfix 命令来浏览其它的匹配。例如，:cnext 会跳到下一个。在 quickfix 窗口里，也可以用 :cwindow 得到所有的匹配的列表。
{pattern} 视为 Vim 的正规表达式 pattern 。
不使用 'ignorecase'，你可以加上 "\c" 来忽略大小写。
大小写敏感的搜索示例：

```
:helpgrep Uganda
```

大小写不敏感的搜索示例：

```
:helpgrep uganda\c
```

寻找中文帮助的搜索：

```
:helpgrep backspace@cn
```

模式不支持换行符，必须在一行内匹配。为此，可用 :grep 代替，但要得到帮助文件的列表就比较复杂了。
后面不能跟其他的命令。其余部分都被当作模式的一部分。如果需要，可以用 :execute 。
不会在压缩的帮助文件里搜索 (Fedora 压缩帮助文件)。

:lh :lhelpgrep

:lh[elpgrep] {pattern}[@xx]
类似于 ":helpgrep"，除了使用位置列表代替 quickfix 列表之外。如果帮助窗口已经打开，使用该窗口的位置列表。不然，打开新帮助窗口，并设置该窗口的位置列表。当前窗口的位置列表这时不改变。

:exu :exusage

:exu[sage] 显示 Ex 命令的帮助。目的是为了模拟对应的 Nvi 命令。

:viu :viusage

:viu[sage] 显示普通命令的帮助。目的是为了模拟对应的 Nvi 命令。

如果不给出参数，:help 会打开 'helpfile' 选项指定的文件。否则，就会在 'runtimepath' 选项指定的多个路径中所有的 "doc/tags" 文件里查找所要求的标签。

帮助窗口的起始高度可以用 'helpheight' 选项来设置 (缺省是 20)。

标签用来跳转到指定的主题。有两种方法可以选择：
- 在命令或选项之上用 "CTRL-]" 命令。这只有在标签是关键字 (见 'iskeyword') 才行。"<C-Leftmouse>" 和 "g<LeftMouse>" 等价于 "CTRL-]"。
- 用 ":ta {subject}" 命令。这对于包含非关键字字符的标签也适用。

用 CTRL-T 或者 CTRL-O 跳回来。
用 ":q" 关闭帮助窗口。

如果你查找的项目有多个匹配，你可以这样依次跳转到每个匹配：
1. 先打开帮助窗口。
2. 用 ":tag" 命令，标签前加上斜杠。例如：

```
:tag /min
```

3. 用 ":tnext" 跳转到下一个匹配的标签。

你可以为插件或其他项目增加帮助文件。为此，你并不需要修改现有的帮助文件。见 add-local-help 。

关于如何写一个本地的帮助文件，见 write-local-help 。

注意：本地帮助文件的标题行会自动列在帮助文件 "help.txt" 的 "LOCAL ADDITIONS" 一节 local-additions 。只有在 Vim 里实际察看该文件才会这么做，该文件本身并没有被修改。这是通过动态地遍历所有帮助文件并提取每个文件的首行来完成的。其中，跳过 $VIMRUNTINE/doc 里的文件。
(译者注：目前，即使使用经过翻译的帮助，本地帮助文件只能在英文的 help.txt 里看到。用 :help@en 访问。)

help-xterm-window

如果你想在另外一个 xterm 窗口里察看帮助，可以用如下的命令：

```
:!xterm -e vim +help &
```

:helpfind :helpf

:helpf[ind] 和 :help 类似，但用一个对话框来提示输入参数。
这只是为了向后兼容的需要。它现在执行 ToolBar.FindHelp
菜单项而不是内建的对话框。
{仅当编译时加入 +GUI_GTK 特性才有效}

:helpt :helptags
E154 E150 E151 E152 E153 E670

:helpt[ags] [++t] {dir}
为目录 {dir} 生成帮助标签文件 tags。{dir} 为 ALL 时，
使用 'runtimepath' 的所有 "doc" 目录。

扫描该目录及其子目录中所有的 "*.txt" 和 "*.??x" 文件中
帮助标签定义。标签定义出现在星号之间。"*.??x" 文件是经
过翻译的文件。它们相应产生 "tags-??" 文件，参见
help-translated 。所生成的标签文件经过排序。
如果其中有重复项，会给出错误信息。
直接覆盖已有的标签文件，不会有提示。

可选的 "++t" 参数强制加入 "help-tags" 标签。如果 {dir}
等于 $VIMRUNTIME/doc，也会这样做。

例如，要重建运行时目录的帮助标签（需要有相应写权限）：
```
:helptags $VIMRUNTIME/doc
```

---

2. 翻 译 帮 助 文 件 help-translated

除了原始的英语帮助文件外，我们可以添加其他语言的翻译版本。Vim 会在所有
'runtimepath' 的目录的 "doc" 子目录里查找帮助文件。这只有在编译时加入
+multi_lang 特性才会有效。

目前，有以下的翻译可用：

```
中文      - 多位作者
法语      - David Blanchet 翻译
意大利语  - Antonio Colombo 翻译
日文      - 多位作者
波兰语    - Mikolaj Machowski 翻译
俄罗斯语  - Vassily Ragosin 翻译
```

在 Vim 网页上可以找到这些翻译：http://www.vim.org/translations.php

帮助文件的翻译版本包含如下文件：

```
help.abx
howto.abx
...
tags-ab
```

"ab" 是一个双字母的语言代码。这样，中文的文件名是：

```
help.cnx
```

```
howto.cnx
...
tags-cn
```

'helplang' 选项设置若干语言偏好。 缺省值根据当前环境设置。Vim 会先在偏好的语言里查找匹配的标签。如果没有，就使用英语版本。

要查找某一特定的语言的标签，在标签后面加上 "@ab"，其中的 "ab" 是两字节的语言代码。示例：

```
:he user-manual@cn
:he user-manual@en
```

前者查找中文的用户手册，即使 'helplang' 为空。后者查找英语用户手册，即使 'helplang' 设置为 "cn"。

":help" 的命令行补全只会在有多个语言版本的标签时显示 "@en" 后缀。如果只有英语版本，"@en" 就省略。如果首个候选有 "@ab" 后缀而该后缀匹配 'helplang' 的首选语言，则 "@ab" 也被省略。

如果在一个非英语帮助文件里使用 CTRL-] 或者 ":help!"，Vim 会先找相同语言的标签。如果没有，再根据 'helplang' 选择语言。

Help 文件一定要使用 latin1 或 utf-8 编码。Vim 如果发现首行有非 ASCII 的字符，就假设是 utf-8 编码。所以，你至少要翻译头部的 "For Vim version"。

同一个目录里相同语言的帮助文件必须使用相同的编码。不同语言或者相同语言但在不同的目录下可以使用不同的编码。

为译者的提示：
- 不要翻译标签本身。这样才能用 'helplang' 来指定语言偏好。你可以在自己的语言里加入新的标签。
- 如果不想翻译文件的部分内容，用 "tag@en" 的形式标记英语版本的标签。
- 生成一个包，包含所有的帮助和和标签文件，以便下载。用户把它解开到某个 "doc" 目录下就可以开始使用了。请告知 Bram，他可以在 www.vim.org 上给加一个链接。
- 用 :helptags 命令生成标签文件 tags。该命令会在指定目录下找到所有语言的版本。

---

3. 编 写 帮 助 文 件 help-writing

为了方便使用，为插件编写的 Vim 帮助文件应该遵循标准 Vim 帮助文件的格式。如果你在编写新帮助文件，最好从现有的文件复制一份作为模板。

帮助文件的首行的格式应该是这样的：

```
helpfile_name.txt      For Vim version 7.3      Last change: 2010 June 4
```

第一个字段是指向帮助文件名的链接。第二个字段描述所适用的 Vim 版本。最后一个字段给出该文件的最后修改日期。字段之间用制表符分隔。

在帮助文件的底部放上 Vim 的模式行，它设置 'textwidth'、'tabstop' 选项，并把 'filetype' 设为 "help"。请不要在模式行上设置全局选项，否则读取帮助的用户会有不希望的后果。

标 签

要定义帮助标签，把名字放在星号之间（*标签名*）。标签名应该和所有 Vim 帮助标签名不同，最好以 Vim 插件名开头。标签名通常行右对齐。

要引用已有帮助标签并建立一个热链，把名字放在竖线（|）之间，如 |help-writing| 。

要引用 Vim 命令并建立一个热链，把名字放在反引号之间，例如 `:filetype` 里。可见它以命令形式高亮，如同代码块那样（见下）。

要在帮助文件里引用 Vim 选项，可以把选项名在单引号之间，如 'statusline'。

高 亮

要定义栏标题，在行尾加上波浪符。栏标题会使用不同颜色的高亮。例如

栏标题

要分隔同一帮助文件的不同小节，加上一行从首列开始的 '=' 字符序列。小节分隔行会使用不同的高亮。

要不加修饰地引用一段 ex 命令块，在块之前的那行最后加上一个大于号（>）字符，然后在块之后的那行放上一个小于号（<）字符作为该行的第一个非空白字符。任何从第一列开始的行也会隐含地结束之前的 ex 命令块。例如

```
    function Example_Func()
        echo "Example"
    endfunction
```

以下内容在 Vim 帮助文件中采用不同的高亮:
- 使用 <> 记号的特殊键名，如 <PageDown>，或 Ctrl 字符，如 CTRL-X
- 任何 {花括号} 之间的内容，如 {lhs} 和 {rhs}

"Note", "Notes" 和类似的单词会神奇地自动得到独特的高亮，下面的也是:

```
        *Todo   something to do
        *Error  something wrong
```

具体细节可见 $VIMRUNTIME/syntax/help.vim

vim:tw=78:ts=8:noet:ft=help:norl:

quickref.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：lang2 http://vimcdoc.sf.net

快速参考指南

quickref Contents

标签	主题
Q_ct	帮助文件清单
Q_lr	动作：左右
Q_ud	动作：上下
Q_tm	动作：文本对象
Q_pa	动作：模式查找
Q_ma	动作：位置标记
Q_vm	动作：其他
Q_ta	动作：使用标签
Q_sc	屏幕滚动
Q_in	插入：插入文本
Q_ai	插入：普通键
Q_ss	插入：特殊键
Q_di	插入：二合字母
Q_si	插入：特殊插入
Q_de	改动：删除文本
Q_cm	改动：复制与移动
Q_ch	改动：修改文本
Q_co	改动：复杂
Q_vi	可视模式
Q_to	文本对象

标签	主题
Q_re	重复命令
Q_km	键盘映射
Q_ab	缩写
Q_op	选项
Q_ur	撤销/重做命令
Q_et	外部命令
Q_qf	Quickfix 命令
Q_vc	其他命令
Q_ce	Ex：命令行编辑
Q_ra	Ex：范围
Q_ex	Ex：特殊字符
Q_st	启动 Vim
Q_ed	文件编辑
Q_fl	使用参数列表
Q_wq	写入与退出
Q_ac	自动命令
Q_wi	多窗口命令
Q_bu	缓冲区列表命令
Q_sy	语法高亮
Q_gu	GUI 命令
Q_fo	折叠

N 用来（可选）置于命令之前表示该命令执行的计数。

Q_lr 左右动作

h	N	h	左（亦：CTRL-H、<BS> 或 <Left> 键）
l	N	l	右（亦：<Space> 或 <Right> 键）
0		0	至本行第一个字符（亦：<Home> 键）
^		^	至本行第一个非空白字符
$	N	$	至本行（加上 N - 1 后续行）最后一个字符（亦：<End> 键）
g0		g0	至屏幕行第一个字符（当行回绕时和 "0" 不同）
g^		g^	至屏幕行第一个非空白字符（当行回绕时和 "^" 不同）
g$	N	g$	至屏幕行最后一个字符（当行回绕时和 "$" 不同）
gm		gm	至屏幕行中点
gM		gM	至本行中点
bar	N	|	至第 N 列（缺省：1）
f	N	f{char}	至右边第 N 次出现 {char} 之处（find）
F	N	F{char}	至左边第 N 次出现 {char} 之处（Find）
t	N	t{char}	至右边第 N 次出现 {char} 之前（till）
T	N	T{char}	至左边第 N 次出现 {char} 之前（Till）
;	N	;	重复上次 "f"、"F"、"t" 或 "T" 命令 N 次
,	N	,	以相反方向重复上次 "f"、"F"、"t" 或 "T" 命令 N 次

Q_ud　　上下动作

k	N	k	上移 N 行 (亦: CTRL-P 和 <Up>)
j	N	j	下移 N 行 (亦: CTRL-J、CTRL-N、<NL> 和 <Down>)
-	N	-	上移 N 行，至第一个非空白字符
+	N	+	下移 N 行，至第一个非空白字符 (亦: CTRL-M 和 <CR>)
_	N	_	下移 N - 1 行，至第一个非空白字符
G	N	G	至第 N 行 (缺省: 末行) 第一个非空白字符
gg	N	gg	至第 N 行 (缺省: 首行) 第一个非空白字符
N%	N	%	至全文件行数百分之 N 处; 必须给出 N，否则是 % 命令
gk	N	gk	上移 N 屏幕行 (回绕行时不同于 "k")
gj	N	gj	下移 N 屏幕行 (回绕行时不同于 "j")

Q_tm　　文本对象动作

w	N	w	向前 (正向，下同) N 个单词	(word)
W	N	W	向前 N 个空白隔开的字串 WORD	(WORD)
e	N	e	向前至第 N 个单词词尾	(end)
E	N	E	向前至第 N 个空白隔开的字串 WORD 的词尾	(End)
b	N	b	向后 (反向，下同) N 个单词	(backward)
B	N	B	向后至第 N 个空白隔开的字串 WORD 的词尾	(Backward)
ge	N	ge	向后至第 N 个单词词尾	
gE	N	gE	向后至第 N 个空白隔开的字串 WORD 的词尾	
)	N	)	向前 N 个句子	
(	N	(	向后 N 个句子	
}	N	}	向前 N 个段落	
{	N	{	向后 N 个段落	
]]	N	]]	向前 N 个小节，置于小节的开始	
[[	N	[[	向后 N 个小节，置于小节的开始	
][	N	][	向前 N 个小节，置于小节的末尾	
[]	N	[]	向后 N 个小节，置于小节的末尾	
[(	N	[(	向后至第 N 个未闭合的 '('	
[{	N	[{	向后至第 N 个未闭合的 '{'	
[m	N	[m	向后至第 N 个方法 (method) 的开始 (用于 Java)	
[M	N	[M	向后至第 N 个方法的结束 (Method) (用于 Java)	
])	N	])	向前至第 N 个未闭合的 ')'	
]}	N	]}	向前至第 N 个未闭合的 '}'	
]m	N	]m	向前至第 N 个方法 (method) 的开始 (用于 Java)	
]M	N	]M	向前至第 N 个方法的结束 (Method) (用于 Java)	
[#	N	[#	向后至第 N 个未闭合的 "#if" 或 "#else"	
]#	N	]#	向前至第 N 个未闭合的 "#else" 或 "#endif"	
[star	N	[*	向后至第 N 个注释的开始 "/*"	
]star	N	]*	向前至第 N 个注释的结束 "*/"	

Q_pa　　模式查找

/	N	/{pattern}[/[offset]]<CR>	向前查找第 N 次出现 {pattern} 的地方
?	N	?{pattern}[?[offset]]<CR>	向后查找第 N 次出现 {pattern} 的地方

/<CR>	N	/<CR>	向前重复上次查找
?<CR>	N	?<CR>	向后重复上次查找
n	N	n	重复上次查找
N	N	N	相反方向重复上次查找
star	N	*	向前查找光标下的标识符
#	N	#	向后查找光标下的标识符
gstar	N	g*	同 "*"，但也查找部分匹配
g#	N	g#	同 "*"，但也查找部分匹配
gd		gd	至光标下标识符的局部声明 (goto declaration)
gD		gD	至光标下标识符的全局声明 (goto Declaration)

pattern 查找模式中的特殊字符

含义	magic	nomagic
匹配任意单个字符	.	\.
匹配行首	^	^
匹配 <EOL>	$	$
匹配单词开始	\<	\<
匹配单词结束	\>	\>
匹配单个标定范围中的字符	[a-z]	\[a-z]
匹配单个不在标定范围中的字符	[^a-z]	\[^a-z]
匹配一个标识符字符	\i	\i
同上但不包括数字	\I	\I
匹配一个关键字字符	\k	\k
同上但不包括数字	\K	\K
匹配一个文件名字符	\f	\f
同上但不包括数字	\F	\F
匹配一个可显示的字符	\p	\p
同上但不包括数字	\P	\P
匹配一个空白字符	\s	\s
匹配一个非空白字符	\S	\S
匹配 <Esc>	\e	\e
匹配 <Tab>	\t	\t
匹配 <CR>	\r	\r
匹配 <BS>	\b	\b
匹配 0 或多个前面的匹配原	*	*
匹配 1 或多个前面的匹配原	\+	\+
匹配 0 或 1 个前面的匹配原	\=	\=
匹配 2 至 5 个前面的匹配原	\{2,5}	\{2,5}
隔开两种可替换的匹配	|	|
组合模式为单个匹配原	\(\)	\(\)

search-offset 查找命令后可用的偏移量

[num]	向下 [num] 行，位于第一列
+[num]	向下 [num] 行，位于第一列
-[num]	向上 [num] 行，位于第一列
e[+num]	匹配结尾 (end) 右方 [num] 个字符处
e[-num]	匹配结尾 (end) 左方 [num] 个字符处
s[+num]	匹配启始 (start) 右方 [num] 个字符处
s[-num]	匹配启始 (start) 左方 [num] 个字符处
b[+num]	同上 s[+num] (助记: begin)

b[-num]　　同上 s[-num] (助记: begin)
;{search-command}　接着执行 {search-command} 查找命令

Q_ma　　位置标记和相关动作

m	m{a-zA-Z}	用标记 {a-zA-Z} 记录当前位置
‘a	‘{a-z}	至当前文件中的标记 {a-z}
‘A	‘{A-Z}	至任何文件中的标记 {A-Z}
‘0	‘{0-9}	至 Vim 上次退出的位置
‘‘	‘‘	至上次跳转之前的位置
‘quote	‘"	至上次编辑此文件的位置
‘[	‘[	至上次被操作或放置的文本的开始
‘]	‘]	至上次被操作或放置的文本的结尾
‘<	‘<	至 (前次) 可视区域的开始
‘>	‘>	至 (前次) 可视区域的结尾
‘.	‘.	至当前文件最后被改动的位置
’	’{a-zA-Z0-9[]’"<>.}	同 ‘, 但同时移动至该行的第一个非空白字符
:marks	:marks	列出活动的标记
CTRL-O	N CTRL-O	跳转到跳转表中第 N 个较早的位置
CTRL-I	N CTRL-I	跳转到跳转表中第 N 个较晚的位置
:ju	:ju[mps]	列出跳转表

Q_vm　　其他动作

%	%	找到本行中下一个括号、方括号、注释或 "#if"/"#else"/"#endif" 并移动至与其对称的位置
H	N H	至窗口的第 N 行的第一个非空白字符处 (Home)
M	M	至窗口的中间行的第一个非空白字符处 (Middle)
L	N L	至窗口的第 N 行 (从下方算) 的第一个非空白字符处 (Last)

go	N go	至缓冲区的第 N 个字节
:go	:[range]go[to] [off]	至缓冲区的第 [off] 个字节

Q_ta　　使用标签

:ta	:ta[g][!] {tag}	跳转到标签 {tag}
:ta	:[count]ta[g][!]	跳转到标签列表里第 [count] 个较新的标签
CTRL-]	CTRL-]	跳转到光标下的标签, 除非文件被改动
:ts	:ts[elect][!] [tag]	列出匹配的标签并选择其中一个跳转
:tjump	:tj[ump][!] [tag]	跳转到标签 [tag], 当有多个匹配时从列表中选择
:ltag	:lt[ag][!] [tag]	跳转到标签 [tag] 并把匹配的标签加到位置列表
:tags	:tags	显示标签列表
CTRL-T	N CTRL-T	跳转到标签列表中第 N 个较早的标签
:po	:[count]po[p][!]	跳转到标签列表中第 [count] 个较早的标签
:tnext	:[count]tn[ext][!]	跳转到向下第 [count] 个匹配的标签
:tp	:[count]tp[revious][!]	跳转到向上第 [count] 个匹配的标签
:tr	:[count]tr[ewind][!]	跳转到第 [count] 个匹配的标签
:tl	:tl[ast][!]	跳转到上次匹配的标签
:ptag	:pt[ag] {tag}	打开一个显示 {tag} 标签的预览窗口

```
CTRL-W_}      CTRL-W }          同 CTRL-] 但在预览窗口显示标签
:pts      :pts[elect]           同 ":tselect" 但在预览窗口显示标签
:ptjump   :ptj[ump]             同 ":tjump" 但在预览窗口显示标签
:pclose   :pc[lose]             关闭标签预览窗口
CTRL-W_z      CTRL-W z          关闭标签预览窗口
```

```
Q_sc          屏幕滚动

CTRL-E        N  CTRL-E         窗口下滚 N 行 (缺省: 1)          (Extra)
CTRL-D        N  CTRL-D         窗口下滚 N 行 (缺省: 1/2 窗口) (Downwards)
CTRL-F        N  CTRL-F         窗口下滚 N 页                    (Forwards)
CTRL-Y        N  CTRL-Y         窗口上滚 N 行 (缺省: 1)
CTRL-U        N  CTRL-U         窗口上滚 N 行 (缺省: 1/2 窗口) (Upwards)
CTRL-B        N  CTRL-B         窗口上滚 N 页                    (Backwards)
z<CR>             z<CR> 或 zt   重画，当前行置窗口顶端
z.                z.    或 zz   重画，当前行置窗口正中
z-                z-    或 zb   重画，当前行置窗口底端
```

以下仅当 'wrap' 选项关闭时生效:

```
zh            N  zh             屏幕右滚 N 个字符
zl            N  zl             屏幕左滚 N 个字符
zH            N  zH             屏幕右滚半个屏宽
zL            N  zL             屏幕左滚半个屏宽
```

```
Q_in          插入文本

a         N  a    在光标后添加文本 (N 次)
A         N  A    在行末添加文本 (N 次)
i         N  i    在光标前插入文本 (N 次) (亦: <Insert>)
I         N  I    在当前行的第一个非空白字符前插入文本 (N 次)
gI        N  gI   在第一栏中插入文本 (N 次)
o         N  o    在当前行下方开始新行，添加文本 (N 次)
O         N  O    在当前行上方开始新行，添加文本 (N 次)
:startinsert  :star[tinsert][!]  开始插入模式，当使用 [!] 时添加文本
:startreplace :startr[eplace][!] 开始替换模式，当使用 [!] 时从行末开始
```

在可视列块模式下:

```
v_b_I     I   在所有选中的行之前插入相同的文本
v_b_A     A   在所有选中的行之后添加相同的文本
```

```
Q_ai          插入模式键

insert-index  插入模式命令的索引，以字母顺序排列
```

离开插入模式:

```
i_<Esc>       <Esc>             结束插入模式，回到普通模式
i_CTRL-C      CTRL-C            同 <Esc>，但不使用缩写
i_CTRL-O      CTRL-O {command}  执行命令 {command} 并回到插入模式
```

移动:

```
i_<Up>        光标键            左/右/上/下 移动光标
i_<S-Left>    shift-left/right  左/右 移动一个单词
```

```
i_<S-Up>        shift-up/down       后/前 移动一个满屏
i_<End>         <End>                光标至本行的末字符之后
i_<Home>        <Home>               光标至本行的首字符之上
```

```
Q_ss            插入模式特殊键

i_CTRL-V        CTRL-V {char}..    按本义插入字符，或插入十进制数的字节值
i_<NL>           <NL> 或 <CR> 或 CTRL-M 或 CTRL-J
                                    开始新行
i_CTRL-E        CTRL-E              插入光标下方的字符
i_CTRL-Y        CTRL-Y              插入光标上方的字符

i_CTRL-A        CTRL-A              插入上次插入的文本
i_CTRL-@        CTRL-@              插入上次插入的文本并结束插入模式
i_CTRL-R        CTRL-R {register}   插入寄存器的内容
i_CTRL-N        CTRL-N              插入光标前标识符的下一个匹配
i_CTRL-P        CTRL-P              插入光标前标识符的上一个匹配
i_CTRL-X        CTRL-X ...          以各种方式补全光标前的单词

i_<BS>           <BS> 或 CTRL-H     删除光标前的一个字符
i_<Del>          <Del>              删除光标下的一个字符
i_CTRL-W        CTRL-W              删除光标前的一个单词
i_CTRL-U        CTRL-U              删除当前行的所有字符
i_CTRL-T        CTRL-T              在当前行首插入一个 shiftwidth 的缩进
i_CTRL-D        CTRL-D              从当前行首删除一个 shiftwidth 的缩进
i_0_CTRL-D      0 CTRL-D            删除当前行的所有缩进
i_^_CTRL-D      ^ CTRL-D            删除当前行的所有缩进，恢复下一行的缩进
```

```
Q_di            二合字母

:dig        :dig[raphs]             显示当前二合字母列表
:dig        :dig[raphs] {char1}{char2} {number} ...
                                    加入一个或多个二合字母到上述列表中
```

在插入或命令行方式下：

```
i_CTRL-K        CTRL-K {char1} {char2}
                                    键入二合字母
i_digraph       {char1} <BS> {char2}
                                    置位 'digraph' 选项时，键入二合字母
```

```
Q_si            特殊插入

:r          :r [file]           将文件 [file] 的内容插入到光标之下
:r!         :r! {command}       将命令 {command} 的标准输出插入到光标之下
```

```
Q_de            删除文本

x       N  x                删除光标之下及之后的 N 个字符
<Del>   N  <Del>              删除光标之下及之后的 N 个字符
X       N  X                删除光标之前的 N 个字符
d       N  d{motion}        删除动作 {motion} 覆盖的文本
```

```
v_d         {visual}d     删除高亮的文本
dd      N   dd            删除 N 行
D       N   D             删除至行尾 (及 N - 1 后续行)
J       N   J             连接 N - 1 行 (删除 <EOL>)
v_J         {visual}J     连接高亮的行
gJ      N   gJ            同 "J", 但不插入空格
v_gJ        {visual}gJ    同 "{visual}J", 但不插入空格
:d      :[range]d [x]     删除范围 [range] 覆盖的行 [存入寄存器 x]
```

```
Q_cm            复制与移动文本

quote     "{char}         在接下来的删除、抽出或放置命令中使用寄存器 {char}
:reg      :reg            显示所有寄存器的内容
:reg      :reg {arg}      显示寄存器 {arg} 的内容
y         N  y{motion}    将动作 {motion} 覆盖的文本抽出至一个寄存器
v_y           {visual}y   将高亮的文本抽出至一个寄存器
yy        N  yy           抽出 N 行至寄存器
Y         N  Y            抽出 N 行至寄存器
p         N  p            将一个寄存器的内容放置在光标位置后 (N 次)
P         N  P            将一个寄存器的内容放置在光标位置前 (N 次)
]p        N  ]p           同 p, 但调整当前行的缩进
[p        N  [p           同 P, 但调整当前行的缩进
gp        N  gp           同 p, 但将光标留在新文本之后
gP        N  gP           同 P, 但将光标留在新文本之后
```

```
Q_ch            改变文本

r         N  r{char}      以 {char} 替换 N 个字符
gr        N  gr{char}     替换 N 个字符, 但不影响布局
R         N  R            进入替换模式 (重复键入的文本 N 次)
gR        N  gR           进入虚拟替换模式: 同替换模式, 但不影响布局
v_b_r         {visual}r{char}
                          在可视列块模式下: 以 {char} 替换选中文本的每个字符

        (修改 = 删除文本并进入插入模式)
c         N  c{motion}    修改动作 {motion} 覆盖的文本
v_c           {visual}c   修改高亮的文本
cc        N  cc           修改 N 行
S         N  S            修改 N 行
C         N  C            修改至行尾 (及 N - 1 后续行)
s         N  s            修改 N 个字符
v_b_c         {visual}c   在可视列块模式下: 用键入的文本修改选中文本的每一行
v_b_C         {visual}C   在可视列块模式下: 用键入的文本修改选中各行直至行末

~         N  ~            翻转 N 个字符的大小写并前进光标
v_~           {visual}~   翻转高亮文本的大小写
v_u           {visual}u   改高亮的文本为小写
v_U           {visual}U   改高亮的文本为大写
g~            g~{motion}  将动作 {motion} 覆盖的文本翻转大小写
gu            gu{motion}  将动作 {motion} 覆盖的文本改为小写
gU            gU{motion}  将动作 {motion} 覆盖的文本改为大写
v_g?          {visual}g?  对高亮的文本做 rot13 编码
g?            g?{motion}  对动作 {motion} 覆盖的文本做 rot13 编码
```

```
CTRL-A    N  CTRL-A      将光标之上或之后的数值增加 N
CTRL-X    N  CTRL-X      将光标之上或之后的数值减少 N

<          N  <{motion}  将动作 {motion} 覆盖的所有行向左移动 shiftwidth
<<         N  <<          将 N 行向左移动 shiftwidth
>          N  >{motion}  将动作 {motion} 覆盖的所有行向右移动 shiftwidth
>>         N  >>          将 N 行向右移动 shiftwidth
gq         N  gq{motion} 将动作 {motion} 覆盖的所有行进行排版到 'textwidth' 宽
:ce        :[range]ce[nter] [width]
                          将范围 [range] 包括的行居中对齐
:le        :[range]le[ft] [indent]
                          将范围 [range] 中的行靠左对齐 (使用 [indent])
:ri        :[range]ri[ght] [width]
                          将范围 [range] 中的行靠右对齐
```

```
Q_co            复杂改动

!          N  !{motion}{command}<CR>
                          将动作覆盖的行用命令 {command} 过滤
!!         N  !!{command}<CR>
                          将 N 行用命令 {command} 过滤
v_!           {visual}!{command}<CR>
                          将高亮的行用命令 {command} 过滤
:range!    :[range]! {command}<CR>
                          将范围 [range] 包括的行用命令 {command} 过滤
=          N  ={motion}
                          将动作覆盖的行用 'equalprg' 过滤
==         N  ==         将 N 行用 'equalprg' 过滤
v_=           {visual}=
                          将高亮的行用 'equalprg' 过滤
:s         :[range]s[ubstitute]/{pattern}/{string}/[g][c]
                          以 {string} 替代范围 [range] 包括的行中的 {pattern};
                             与 [g] 共用，替代所有的出现的 {pattern};
                             与 [c] 共用，确认每一次替代
:s         :[range]s[ubstitute] [g][c]
                          以新的范围和选项重复上一次的 ":s" 命令
&             &          不带选项在当前行上重复上一次的 ":s" 命令
:ret       :[range]ret[ab][!] [tabstop]
                          置 'tabstop' 为新值并依据此值调整空白字符
```

```
Q_vi            可视模式

visual-index    可视模式命令列表。

v          v              以字符方式开始高亮                  }  移动光标或使用
V          V              以行方式开始高亮                    }  操作符来作用于
CTRL-V     CTRL-V         以列块方式开始高亮                  }  高亮的文本
v_o        o              交换高亮区域的开始处和光标位置
gv         gv             使用上一次的可视区域开始高亮
v_v        v              以字符方式开始高亮或停止高亮
v_V        V              以行方式开始高亮或停止高亮
v_CTRL-V   CTRL-V         以列块方式开始高亮或停止高亮
```

Q_to 文本对象（仅适用于可视模式或在操作符后）

v_aw	N	aw	选择 "一个单词"	(a word)
v_iw	N	iw	选择 "内含单词"	(inner word)
v_aW	N	aW	选择 "一个字串"	(WORD)
v_iW	N	iW	选择 "内含字串"	
v_as	N	as	选择 "一个句子"	(sentence)
v_is	N	is	选择 "内含句子"	
v_ap	N	ap	选择 "一个段落"	(paragraph)
v_ip	N	ip	选择 "内含段落"	
v_ab	N	ab	选择 "一个块" (从 "[(" 至 "])")	(block)
v_ib	N	ib	选择 "内含块" (从 "[(" 到 "])")	
v_aB	N	aB	选择 "一个大块" (从 "[{" 到 "]}")	(Block)
v_iB	N	iB	选择 "内含大块" (从 "[{" 到 "]}")	
v_a>	N	a>	选择 "一个 <> 块"	
v_i>	N	i>	选择 "内含 <> 块"	
v_at	N	at	选择 "一个标签块" (从 <aaa> 到 </aaa>)	(tag)
v_it	N	it	选择 "内含标签块" (从 <aaa> 到 </aaa>)	
v_a'	N	a'	选择 "一个单引号字符串"	
v_i'	N	i'	选择 "内含单引号字符串"	
v_aquote	N	a"	选择 "一个双引号字符串"	
v_iquote	N	i"	选择 "内含双引号字符串"	
v_a`	N	a`	选择 "一个反引号字符串"	
v_i`	N	i`	选择 "内含反引号字符串"	

Q_re 重复命令

.	N	.	重复最近一次改动 (但计数改为 N)
q		q{a-z}	记录键入的字符，存入寄存器 {a-z}
q		q{A-Z}	记录键入的字符，添加进寄存器 {a-z}
q		q	终止记录
@	N	@{a-z}	执行寄存器 {a-z} 的内容 (N 次)
@@	N	@@	重复上次的 @{a-z} 操作 (N 次)
:@	:@{a-z}		将寄存器 {a-z} 的内容当作 Ex 命令来执行
:@@	:@@		重复上次的 :@{a-z} 操作
:g	:[range]g[lobal]/{pattern}/[cmd]		对 [range] 内所有匹配 {pattern} 的行执行 Ex 命令 [cmd] (缺省: ":p")
:g	:[range]g[lobal]!/{pattern}/[cmd]		对 [range] 内所有 不 匹配 {pattern} 的行执行 Ex 命令 [cmd] (缺省: ":p")
:so	:so[urce] {file}		从文件 {file} 读入 Ex 命令
:so	:so[urce]! {file}		从文件 {file} 读入 Vim 命令
:sl	:sl[eep] [sec]		在 [sec] 秒钟内任何事也不做
gs	N	gs	睡 N 秒 (goto sleep)

Q_km 键盘映射

:map	:ma[p] {lhs} {rhs}	在普通及可视模式下将 {rhs} 映射为 {lhs}
:map!	:ma[p]! {lhs} {rhs}	在插入及命令行模式下将 {rhs} 映射为 {lhs}
:noremap	:no[remap][!] {lhs} {rhs}	同 ":map"，但不对 {rhs} 重新映射
:unmap	:unm[ap] {lhs}	清除普通及可视模式下 {lhs} 的映射
:unmap!	:unm[ap]! {lhs}	清除插入及命令行模式下 {lhs} 的映射
:map_l	:ma[p] [lhs]	列出普通及可视模式下以 [lhs] 开始的映射
:map_l!	:ma[p]! [lhs]	列出插入及命令行模式下以 [lhs] 开始的映射
:cmap	:cmap/:cunmap/:cnoremap	同 ":map!"/":unmap!"/":noremap!" 但仅作用于命令行模式 (Command-line)
:imap	:imap/:iunmap/:inoremap	同 ":map!"/":unmap!"/":noremap!" 但仅作用于插入模式 (Insert)
:nmap	:nmap/:nunmap/:nnoremap	同 ":map"/":unmap"/":noremap"， 但仅作用于普通模式 (Normal)
:vmap	:vmap/:vunmap/:vnoremap	同 ":map"/":unmap"/":noremap"， 但仅作用于可视模式 (Visual)
:omap	:omap/:ounmap/:onoremap	同 ":map"/":unmap"/":noremap" 但仅作用于操作符等待模式 (Operator-pending)
:mapc	:mapc[lear]	清除普通及可视模式下的映射
:mapc	:mapc[lear]!	清除插入及命令行模式下的映射
:imapc	:imapc[lear]	清除插入模式下的映射
:vmapc	:vmapc[lear]	清除可视模式下的映射
:omapc	:omapc[lear]	清除操作符等待模式下的映射
:nmapc	:nmapc[lear]	清除普通模式下的映射
:cmapc	:cmapc[lear]	清除命令行模式下的映射
:mkexrc	:mk[exrc][!] [file]	将当前的键盘映射、缩写及设置写入文件 [file] (缺省: ".exrc"; 使用 ! 覆盖文件)
:mkvimrc	:mkv[imrc][!] [file]	同 ":mkexrc"，但缺省为 ".vimrc"
:mksession	:mks[ession][!] [file]	同 ":mkvimrc"，但同时存储当前文件、窗口等信息，使得用户将来可以继续当前对话

Q_ab　　缩写

:abbreviate	:ab[breviate] {lhs} {rhs}	为 {rhs} 加入缩写 {lhs}
:abbreviate	:ab[breviate] {lhs}	显示以 {lhs} 开始的缩写
:abbreviate	:ab[breviate]	显示所有缩写
:unabbreviate	:una[bbreviate] {lhs}	删除 {lhs} 对应的缩写
:noreabbrev	:norea[bbrev] [lhs] [rhs]	同 ":ab"，但不对 [rhs] 重映射
:iabbrev	:iab/:iunab/:inoreab	同 ":ab"，但仅适用于插入模式
:cabbrev	:cab/:cunab/:cnoreab	同 ":ab"，但仅适用于命令行模式
:abclear	:abc[lear]	清除所有缩写
:cabclear	:cabc[lear]	清除所有命令行模式缩写
:iabclear	:iabc[lear]	清除所有插入模式缩写

Q_op	选项	
:set	:se[t]	显示所有被改动的选项
:set	:se[t] all	显示所有非 termcap 选项
:set	:se[t] termcap	显示所有 termcap 选项
:set	:se[t] {option}	置位布尔选项（开启） 显示字符串或数值选项
:set	:se[t] no{option}	复位布尔选项（关闭）
:set	:se[t] inv{option}	逆转布尔选项的值
:set	:se[t] {option}={value}	设置字符串/数值选项的值为 {value}
:set	:se[t] {option}+={value}	将 {value} 附加到字符串选项里，将 {value} 加到数值选项上
:set	:se[t] {option}-={value}	从 {value} 从字符串选项里删除，从数值选项里减去 {value}
:set	:se[t] {option}?	显示 {option} 的值
:set	:se[t] {option}&	重置 {option} 为其缺省值
:setlocal	:setl[ocal]	同 ":set"，但对局部选项只设定局部值
:setglobal	:setg[lobal]	同 ":set"，但对局部选项设定其全局值
:fix	:fix[del]	根据 't_kb' 的值设置 't_kD'
:options	:opt[ions]	打开一个新窗口，用来参看并设置选项，选项们以功能分组，有单行的解释，并有指向帮助的链接

各选项的简短说明：

option-list

'aleph'	'al'	Aleph 字母（希伯来语）的 ASCII 代码
'allowrevins'	'ari'	允许插入和命令行模式的 CTRL-_
'altkeymap'	'akm'	用于波斯语的已废弃选项
'ambiwidth'	'ambw'	如何处理有多种宽度的 Unicode 字符
'antialias'	'anti'	Mac OS X: 用平滑反锯齿的字体
'autochdir'	'acd'	根据当前窗口的文件切换目录
'arabic'	'arab'	使用阿拉伯语为缺省的第二语言
'arabicshape'	'arshape'	阿拉伯语的字型重整
'autoindent'	'ai'	根据上一行决定新行的缩进
'autoread'	'ar'	有 Vim 之外的改动时自动重读文件
'autowrite'	'aw'	有改动时自动写回文件
'autowriteall'	'awa'	类似于 'autowrite'，但适用于更多命令
'background'	'bg'	"dark" 或 "light"，用于色彩的高亮
'backspace'	'bs'	在一行开头按退格键如何处理
'backup'	'bk'	覆盖文件时保留备份文件
'backupcopy'	'bkc'	文件备份使用复制而不是换名
'backupdir'	'bdir'	备份文件使用的目录列表
'backupext'	'bex'	备份文件使用的扩展名
'backupskip'	'bsk'	匹配这些模式的文件不予备份
'balloondelay'	'bdlay'	弹出气泡之前的延迟（以毫秒计）
'balooneval'	'beval'	GUI 上打开气泡表达式求值功能
'balloonevalterm'	'bevalterm'	终端上打开气泡表达式求值功能
'balloonexpr'	'bexpr'	计算气泡显示内容的表达式
'belloff'	'bo'	为这些原因不响铃
'binary'	'bin'	二进制方式读/写/编辑文件
'bioskey'	'biosk'	MS-DOS: 用 bios 调用取得字符输入
'bomb'		文件头附加字节顺序标记（Byte Order Mark）
'breakat'	'brk'	可以在此断行的字符

'breakindent'	'bri'	回绕行重复缩进
'breakindentopt'	'briopt'	'breakindent' 的设置
'browsedir'	'bsdir'	开始浏览文件的目录
'bufhidden'	'bh'	缓冲区不再在窗口显示时的行为
'buflisted'	'bl'	缓冲区是否显示在缓冲区列表中
'buftype'	'bt'	缓冲区的特殊类型
'casemap'	'cmp'	指定字母大小写如何改变
'cdpath'	'cd'	":cd" 搜索的目录列表
'cedit'		打开命令行窗口的键
'charconvert'	'ccv'	完成字符编码转换的表达式
'cindent'	'cin'	实现 C 程序的缩进
'cinkeys'	'cink'	设置 'cindent' 时启动缩进的键
'cinoptions'	'cino'	设置 'cindent' 时如何缩进
'cinwords'	'cinw'	'si' 和 'cin' 在这些词后加入额外的缩进
'clipboard'	'cb'	使用剪贴板作为无名的寄存器
'cmdheight'	'ch'	命令行使用的行数
'cmdwinheight'	'cwh'	命令行窗口的高度
'colorcolumn'	'cc'	高亮指定列
'columns'	'co'	显示屏幕的列数
'comments'	'com'	可以开始注释行的模式
'commentstring'	'cms'	注释的样板；用于折叠的标志
'compatible'	'cp'	尽可能做到与 Vi 兼容
'complete'	'cpt'	指定插入模式的自动补全如何工作
'completefunc'	'cfu'	插入模式补全使用的函数
'completeopt'	'cot'	插入模式补全使用的选项
'completepopup'	'cpp'	插入模式补全 info 弹出使用的选项
'completeslash'	'csl'	类似于 'shellslash'，但用于补全
'concealcursor'	'cocu'	是否隐藏光标所在行的可隐藏文本
'conceallevel'	'cole'	是否显示可隐藏文本
'confirm'	'cf'	询问如何处理未保存/只读的文件
'conskey'	'consk'	直接从控制台读入键击 (只限于 MS-DOS)
'copyindent'	'ci'	使得 'autoindent' 使用已有的缩进结构
'cpoptions'	'cpo'	设定 Vi-兼容的行为
'cryptmethod'	'cm'	文件写入时所用的加密方法
'cscopepathcomp'	'cspc'	显示路径多少部分
'cscopeprg'	'csprg'	执行 cscope 的命令
'cscopequickfix'	'csqf'	用 quickfix 窗口得到 cscope 的结果
'cscoperelative'	'csre'	用 cscope.out 路径目录名作为前缀
'cscopetag'	'cst'	用 cscope 处理标签命令
'cscopetagorder'	'csto'	决定 ":cstag" 的搜索顺序
'cscopeverbose'	'csverb'	增加 cscope 数据库时给出消息
'cursorbind'	'crb'	光标移动时同时在其它窗口移动
'cursorcolumn'	'cuc'	高亮光标所在屏幕列
'cursorline'	'cul'	高亮光标所在屏幕行
'cursorlineopt'	'culopt'	'cursorline' 的设置
'debug'		设为 "msg" 可以看到所有的错误消息
'define'	'def'	查找宏定义所使用的模式
'delcombine'	'deco'	在单独使用时删除组合字符
'dictionary'	'dict'	关键字自动补全所用的文件名
'diff'		当前窗口使用比较模式
'diffexpr'	'dex'	得到比较文件所用的表达式
'diffopt'	'dip'	使用比较模式的选项
'digraph'	'dg'	允许插入模式时输入二合字母
'directory'	'dir'	交换文件所用的目录名列表

'display'	'dy'	如何显示文本的标志位列表
'eadirection'	'ead'	'equalalways' 工作的方向
'edcompatible'	'ed'	切换 ":substitute" 命令的标志位
'emoji'	'emo'	表情字符视作全宽
'encoding'	'enc'	内部使用的编码方式
'endofline'	'eol'	文件最后一行写入换行符 <EOL>
'equalalways'	'ea'	自动使所有窗口大小相同
'equalprg'	'ep'	"=" 命令使用的外部命令
'errorbells'	'eb'	有错误信息时响铃
'errorfile'	'ef'	QuickFix 模式的错误文件名
'errorformat'	'efm'	错误文件行格式的描述
'esckeys'	'ek'	插入模式下识别功能键
'eventignore'	'ei'	忽略的自动命令事件
'expandtab'	'et'	键入 <Tab> 时使用空格
'exrc'	'ex'	在当前目录里读入 .vimrc 和 .exrc
'fileencoding'	'fenc'	多字节文本的文件编码
'fileencodings'	'fencs'	参与自动检测的字符编码
'fileformat'	'ff'	文件输入输出使用的格式
'fileformats'	'ffs'	参与自动检测的 'fileformat' 的格式
'fileignorecase'	'fic'	使用文件名时忽略大小写
'filetype'	'ft'	自动命令使用的文件类型
'fillchars'	'fcs'	显示特殊项目所使用的字符
'fixendofline'	'fixeol'	确保文件的末行有 <EOL>
'fkmap'	'fk'	用于波斯语的已废弃选项
'foldclose'	'fcl'	当光标离开时关闭折叠
'foldcolumn'	'fdc'	设定指示折叠的列宽度
'foldenable'	'fen'	设置为显示所用打开的折叠
'foldexpr'	'fde'	当 'foldmethod' 为 "expr" 时使用的表达式
'foldignore'	'fdi'	当 'foldmethod' 为 "indent" 时忽略的行
'foldlevel'	'fdl'	当折叠级别高于此值时关闭折叠
'foldlevelstart'	'fdls'	开始编辑文件的 'foldlevel'
'foldmarker'	'fmr'	当 'foldmethod' 为 "marker" 时的标志
'foldmethod'	'fdm'	折叠的类型
'foldminlines'	'fml'	折叠关闭所需的最少行数
'foldnestmax'	'fdn'	最大折叠深度
'foldopen'	'fdo'	打开折叠所使用的命令
'foldtext'	'fdt'	显示关闭的折叠所用的表达式
'formatexpr'	'fex'	"gq" 命令使用的表达式
'formatlistpat'	'flp'	识别列表头部的模式
'formatoptions'	'fo'	自动排版完成的方式
'formatprg'	'fp'	"gq" 命令使用的外部程序
'fsync'	'fs'	文件写回后是否激活 fsync()
'gdefault'	'gd'	缺省打开 ":substitute" 的 'g' 标志
'grepformat'	'gfm'	'grepprg' 的输出格式
'grepprg'	'gp'	":grep" 使用的程序
'guicursor'	'gcr'	GUI: 光标形状和闪烁的设置
'guifont'	'gfn'	GUI: 使用的字体名
'guifontset'	'gfs'	GUI: 使用的多字节字体名
'guifontwide'	'gfw'	双倍宽度字符的字体名列表
'guiheadroom'	'ghr'	GUI: 用于窗口装饰的像素空间
'guioptions'	'go'	GUI: 使用的部件和选项
'guipty'		GUI: ":!" 命令尝试仿终端
'guitablabel'	'gtl'	GUI: 标签页定制的标签
'guitabtooltip'	'gtt'	GUI: 标签页定制的工具提示

'helpfile'	'hf'	主帮助文件的完整路径名
'helpheight'	'hh'	新帮助窗口的最小高度
'helplang'	'hlg'	首选帮助语言
'hidden'	'hid'	当缓冲区被放弃 (abandon) 时不卸载之
'highlight'	'hl'	设置若干场合下的高亮模式
'history'	'hi'	记住的命令行的行数
'hkmap'	'hk'	希伯来语的键盘映射
'hkmapp'	'hkp'	希伯来语的音节 (phonetic) 键盘映射
'hlsearch'	'hls'	高亮最近的匹配搜索模式
'icon'		让 Vim 设定窗口图标的文本
'iconstring'		Vim 图标文本所用的字符串
'ignorecase'	'ic'	搜索模式时忽略大小写
'imactivatefunc'	'imaf'	激活/关闭 X 输入方法的函数
'imactivatekey'	'imak'	激活 X 输入方法 (X input method) 的键击
'imcmdline'	'imc'	开始编辑命令行时使用 IM
'imdisable'	'imd'	任何模式下不使用 IM
'iminsert'	'imi'	插入模式下使用 :lmap 或 IM
'imsearch'	'ims'	输入搜索模式时使用 :lmap 或 IM
'imstatusfunc'	'imsf'	获得 X 输入方法的状态的函数
'imstyle'	'imst'	指定输入方法的输入方式
'include'	'inc'	查找包含文件所使用的模式
'includeexpr'	'inex'	处理包含文件行所使用的表达式
'incsearch'	'is'	输入搜索模式时同时高亮部分的匹配
'indentexpr'	'inde'	得到一行的缩进位置的表达式
'indentkeys'	'indk'	使用 'indentexpr' 时启动缩进的键
'infercase'	'inf'	关键字自动补全的匹配调整大小写
'insertmode'	'im'	开始编辑文件时进入插入模式
'isfname'	'isf'	文件和路径名可用的字符
'isident'	'isi'	标识符可用的字符
'iskeyword'	'isk'	关键字可用的字符
'isprint'	'isp'	可显示的字符
'joinspaces'	'js'	连接命令在句号之后加两个空格
'key'		加密密钥
'keymap'	'kmp'	键盘映射名
'keymodel'	'km'	允许用键击开始/停止选择
'keywordprg'	'kp'	"K" 命令所使用的程序
'langmap'	'lmap'	其他语言模式用的字母表字符
'langmenu'	'lm'	菜单使用的语言
'langnoremap'	'lnr'	已映射字符不应用 'langmap'
'langremap'	'lrm'	映射字符适用 'langmap'
'laststatus'	'ls'	当最近的窗口有状态行时提示
'lazyredraw'	'lz'	执行宏时不重画
'linebreak'	'lbr'	在空白处回绕长行
'lines'		显示屏幕的行数
'linespace'	'lsp'	字符之间的像素行数
'lisp'		自动 lisp 缩进
'lispwords'	'lw'	改变 lisp 缩进方式的单词
'list'		显示 <Tab> 和 <EOL>
'listchars'	'lcs'	list 模式下显示用的字符
'loadplugins'	'lpl'	启动时调入插件脚本
'luadll'		Lua 动态库名
'macatsui'		Mac GUI: 使用 ATSUI 文本绘制
'magic'		改变搜索模式所用的特殊字符
'makeef'	'mef'	":make" 所用的错误文件名

'makeencoding'	'menc'	外部 make/grep 命令使用的编码
'makeprg'	'mp'	":make" 命令所用的程序
'matchpairs'	'mps'	"%" 能匹配的字符对
'matchtime'	'mat'	显示匹配括号的时间（以十分之一秒计）
'maxcombine'	'mco'	显示的最大组合字符数
'maxfuncdepth'	'mfd'	用户函数的最大递归深度
'maxmapdepth'	'mmd'	映射的最大递归深度
'maxmem'	'mm'	单个缓冲区可用的最大内存（以千字节计）
'maxmempattern'	'mmp'	模式匹配使用的最大内存（以千字节计）
'maxmemtot'	'mmt'	所有缓冲区可用的最大内存（以千字节计）
'menuitems'	'mis'	菜单可用的最大项目数
'mkspellmem'	'msm'	在 :mkspell 压缩树前可用的内存
'modeline'	'ml'	在文件开头或结尾识别模式行
'modelineexpr'	'mle'	允许模式行上设置表达式选项
'modelines'	'mls'	模式行的检查行数
'modifiable'	'ma'	可否修改文本
'modified'	'mod'	缓冲区已被修改
'more'		全屏显示时暂停列表
'mouse'		允许使用鼠标点击
'mousefocus'	'mousef'	键盘焦点追随鼠标点击
'mousehide'	'mh'	输入时隐藏鼠标指针
'mousemodel'	'mousem'	改变鼠标按钮的含义
'mouseshape'	'mouses'	不同模式下改变鼠标指针的形状
'mousetime'	'mouset'	鼠标双击之间的最大时间
'mzquantum'	'mzq'	MzScheme 线程的轮询间隔
'mzschemedll'		MzScheme 动态库名
'mzschemegcdll'		MzScheme 用于 GC 的动态库名
'nrformats'	'nf'	CTRL-A 命令识别的数字格式
'number'	'nu'	行前显示行号
'numberwidth'	'nuw'	行号使用的列数
'omnifunc'	'ofu'	文件类型特定补全使用的函数
'opendevice'	'odev'	MS-Windows 上允许读/写设备
'operatorfunc'	'opfunc'	g@ 操作符调用的函数
'osfiletype'	'oft'	不再支持
'packpath'	'pp'	软件包使用的目录列表
'paragraphs'	'para'	分隔段落的 nroff 宏
'paste'		允许粘贴文本
'pastetoggle'	'pt'	切换 'paste' 的键盘代码
'patchexpr'	'pex'	用于给文件打补丁的表达式
'patchmode'	'pm'	保留文件最老的版本
'path'	'pa'	"gf" 等命令搜索用的目录列表
'perldll'		Perl 动态库名
'preserveindent'	'pi'	重排时保持原有的缩进结构
'previewheight'	'pvh'	预览窗口的高度
'previewpopup'	'pvp'	使用弹出窗口用于预览
'previewwindow'	'pvw'	标识预览窗口
'printdevice'	'pdev'	用于 :hardcopy 的打印机名
'printencoding'	'penc'	用于打印的编码方式
'printexpr'	'pexpr'	用于 :hardcopy 打印 PostScript 的表达式
'printfont'	'pfn'	用于 :hardcopy 的字体名
'printheader'	'pheader'	用于 :hardcopy 的页眉格式
'printmbcharset'	'pmbcs'	用于 :hardcopy 的 CJK 字符集
'printmbfont'	'pmbfn'	用于 :hardcopy 的 CJK 输出的字体名
'printoptions'	'popt'	控制 :hardcopy 输出格式

'prompt'	'prompt'	Ex 模式打开提示
'pumheight'	'ph'	弹出菜单的最大高度
'pumwidth'	'pw'	弹出菜单的最小宽度
'pythondll'		Python 2 动态库名
'pythonhome'		Python 2 主目录名
'pythonthreedll'		Python 3 动态库名
'pythonthreehome'		Python 3 主目录名
'pyxversion'	'pyx'	pyx* 命令使用的 Python 版本
'quoteescape'	'qe'	字符串里使用的转义字符
'readonly'	'ro'	禁止写入缓冲区
'redrawtime'	'rdt'	'hlsearch' 和 :match 高亮的超时
'regexpengine'	're'	使用的缺省正规表达式引擎
'relativenumber'	'rnu'	每行前显示相对行号
'remap'		允许映射可以递归调用
'renderoptions'	'rop'	Window 上文本渲染选项
'report'		报告行改变的行数下限
'restorescreen'	'rs'	Win32: 退出时恢复屏幕
'revins'	'ri'	字符插入会反向进行
'rightleft'	'rl'	窗口为从右到左模式
'rightleftcmd'	'rlc'	从右到左模式工作的编辑命令
'rubydll'		Ruby 动态库名
'ruler'	'ru'	标尺，在状态行里显示光标的行号和列号
'rulerformat'	'ruf'	定制标尺格式
'runtimepath'	'rtp'	用于运行时文件的目录列表
'scroll'	'scr'	用 CTRL-U 和 CTRL-D 滚动的行数
'scrollbind'	'scb'	其他窗口滚动时滚动当前窗口
'scrollfocus'	'scf'	滚轮用于鼠标指针所在的窗口
'scrolljump'	'sj'	滚动所需的最少行数
'scrolloff'	'so'	光标上下的最少行数
'scrollopt'	'sbo'	'scrollbind' 的行为方式
'sections'	'sect'	分隔小节的 nroff 宏
'secure'		在当前目录下以安全模式读入 .vimrc
'selection'	'sel'	使用何种选择方式
'selectmode'	'slm'	何时使用选择模式而不是可视模式
'sessionoptions'	'ssop'	:mksession 的选项
'shell'	'sh'	使用外部命令的外壳程序名字
'shellcmdflag'	'shcf'	执行命令所使用的外壳的参数
'shellpipe'	'sp'	把 ":make" 结果输出到错误文件的字符串
'shellquote'	'shq'	括起外壳命令的字符
'shellredir'	'srr'	把过滤命令的输出存到暂时文件的字符串
'shellslash'	'ssl'	外壳文件名使用正斜杠
'shelltemp'	'stmp'	外壳命令是否使用临时文件
'shelltype'	'st'	Amiga: 影响如何使用外壳
'shellxescape'	'sxe'	'shellxquote' 为 (时使用的转义字符
'shellxquote'	'sxq'	类似于 'shellquote'，但包括重定向
'shiftround'	'sr'	缩进列数对齐到 shiftwidth 的整数倍
'shiftwidth'	'sw'	(自动) 缩进使用的步进单位，以空白数目计
'shortmess'	'shm'	缩短消息长度的标志位列表
'shortname'	'sn'	非 MS-DOS: 文件名假定为 8.3 字符
'showbreak'	'sbr'	用于提示回绕行开始的字符串
'showcmd'	'sc'	在状态行里显示 (部分) 命令
'showfulltag'	'sft'	自动补全标签时显示完整的标签匹配模式
'showmatch'	'sm'	插入括号时短暂跳转到匹配的括号
'showmode'	'smd'	在状态行上显示当前模式的消息

'showtabline'	'stal'	是否显示标签页行
'sidescroll'	'ss'	横向滚动的最少列数
'sidescrolloff'	'siso'	在光标左右最少出现的列数
'signcolumn'	'scl'	何时显示标号列
'smartcase'	'scs'	模式中有大写字母时不忽略大小写
'smartindent'	'si'	C 程序智能自动缩进
'smarttab'	'sta'	插入 <Tab> 时使用 'shiftwidth'
'softtabstop'	'sts'	编辑时 <Tab> 使用的空格数
'spell'		打开拼写检查
'spellcapcheck'	'spc'	定位句子尾部的模式
'spellfile'	'spf'	zg 和 zw 保存单词的文件
'spelllang'	'spl'	拼写检查使用的语言
'spellsuggest'	'sps'	提供拼写校正建议的方法
'splitbelow'	'sb'	分割窗口时新窗口在当前窗口之下
'splitright'	'spr'	新窗口在当前窗口之右
'startofline'	'sol'	命令移动光标到行的第一个非空白
'statusline'	'stl'	状态行的定制格式
'suffixes'	'su'	多个匹配所忽略的后缀
'suffixesadd'	'sua'	搜索文件时附加的后缀
'swapfile'	'swf'	缓冲区是否使用交换文件
'swapsync'	'sws'	和交换文件同步的方式
'switchbuf'	'swb'	设置切换到别的缓冲区时的行为
'synmaxcol'	'smc'	寻找语法项目的最大列数
'syntax'	'syn'	读入当前缓冲区的语法
'tabline'	'tal'	终端标签页行的定制格式
'tabpagemax'	'tpm'	-p 和 "tab all" 的最大标签页数
'tabstop'	'ts'	<Tab> 在文件里使用的空格数
'tagbsearch'	'tbs'	标签文件里用二分法查找
'tagcase'	'tc'	标签文件查找时如何处理大小写
'tagfunc'	'tfu'	获取标签匹配列表所用的函数
'taglength'	'tl'	标签里的有效字符数
'tagrelative'	'tr'	标签文件里的文件名是相对路径
'tags'	'tag'	标签命令使用的文件名列表
'tagstack'	'tgst'	把标签推入标签栈
'tcldll'		Tcl 动态库名
'term'		终端名
'termbidi'	'tbidi'	终端支持双向文本
'termencoding'	'tenc'	终端使用的编码方式
'termguicolors'	'tgc'	终端使用 GUI 颜色
'termwinkey'	'twk'	终端上引导 Vim 命令的键
'termwinscroll'	'twsl'	终端窗口中回滚的最大行数
'termwinsize'	'tws'	终端窗口的大小
'termwintype'	'twt'	MS-Windows: 终端窗口所用的 pty 类型
'terse'		简化部分消息
'textauto'	'ta'	已废止，用 'fileformats'
'textmode'	'tx'	已废止，用 'fileformat'
'textwidth'	'tw'	插入文本的最大宽度
'thesaurus'	'tsr'	关键字自动补全所用的同义词字典
'tildeop'	'top'	波浪命令 "~" 以操作符方式工作
'timeout'	'to'	映射和键盘代码等待超时
'timeoutlen'	'tm'	超时时间 (以毫秒计)
'title'		让 Vim 设置窗口标题
'titlelen'		用于窗口标题的 'columns' 比例
'titleold'		旧的标题，用于退出时恢复

'titlestring'		用于 Vim 窗口标题的字符串
'toolbar'	'tb'	GUI: 工具栏显示何种项目
'toolbariconsize'	'tbis'	工具栏图标的大小（只适用于 GTK 2）
'ttimeout'		映射等待超时
'ttimeoutlen'	'ttm'	键盘代码超时时间（以毫秒计）
'ttybuiltin'	'tbi'	在外部 termcap 之前先用内建的 termcap
'ttyfast'	'tf'	指示一个快速的终端连接
'ttymouse'	'ttym'	鼠标产生代码的类型
'ttyscroll'	'tsl'	滚动的最大行数
'ttytype'	'tty'	'term' 的别名
'undodir'	'udir'	保存撤销文件的位置
'undofile'	'udf'	把撤销信息写入一个文件里
'undolevels'	'ul'	最多可以撤销的改变个数
'undoreload'	'ur'	保存缓冲区重载撤销的最大行数
'updatecount'	'uc'	刷新交换文件所需的字符数
'updatetime'	'ut'	刷新交换文件所需的毫秒数
'varsofttabstop'	'vsts'	键入 <Tab> 时生成空格数的列表
'vartabstop'	'vts'	用于 <Tab> 对应空格数的列表
'verbose'	'vbs'	给出详细信息
'verbosefile'	'vfile'	消息写入的文件
'viewdir'	'vdir'	:mkview 存储文件的所在目录
'viewoptions'	'vop'	指定 :mkview 保存的内容
'viminfo'	'vi'	启动和退出时使用 .viminfo 文件
'viminfofile'	'vif'	viminfo 文件使用的文件名
'virtualedit'	've'	何时使用虚拟编辑
'visualbell'	'vb'	使用可视铃声而不是响铃
'warn'		当缓冲区改变时，对外壳命令给出警告
'weirdinvert'	'wiv'	用于有特殊反转方法的终端
'whichwrap'	'ww'	允许指定键跨越行边界
'wildchar'	'wc'	用于通配符扩展的命令行字符
'wildcharm'	'wcm'	同 'wildchar'，但对映射情况也适用
'wildignore'	'wig'	匹配这些模式的文件不会参与自动补全
'wildignorecase'	'wic'	匹配文件名时忽略大小写
'wildmenu'	'wmnu'	命令行自动补全所使用的菜单
'wildmode'	'wim'	'wildchar' 命令行扩展所用的模式
'wildoptions'	'wop'	指定如何完成命令行补全
'winaltkeys'	'wak'	何时窗口系统处理 ALT 键
'wincolor'	'wcr'	局部于窗口的高亮
'window'	'wi'	CTRL-F 和 CTRL-B 滚动的行数
'winheight'	'wh'	当前窗口的最少行数
'winfixheight'	'wfh'	打开/关闭窗口时保持窗口高度
'winfixwidth'	'wfw'	打开/关闭窗口时保持窗口宽度
'winminheight'	'wmh'	任何窗口的最少行数
'winminwidth'	'wmw'	任何窗口的最少列数
'winptydll'		winpty 动态库名
'winwidth'	'wiw'	当前窗口的最少列数
'wrap'		长行回绕并在下一行继续
'wrapmargin'	'wm'	使（硬）回绕开始的从右侧起算的字符数
'wrapscan'	'ws'	搜索在文件尾折回文件头
'write'		允许写入文件
'writeany'	'wa'	写入文件不需 "!" 强制
'writebackup'	'wb'	覆盖文件时建立备份
'writedelay'	'wd'	每个字符延迟（以毫秒计）（用于调试）

Q_ur 撤销/重做命令

u	N	u	撤销最近的 N 此改动
CTRL-R	N	CTRL-R	重做最近的 N 个被撤销的改动
U		U	恢复最近被改动的行

Q_et 外部命令

:shell	:sh[ell]	开启一个外壳
:!	:!{command}	通过一个外壳执行命令 {command}
K	K	用 'keywordprg' 程序 (缺省: "man") 查光标下的关键字

Q_qf Quickfix 命令

:cc	:cc [nr]	显示第 [nr] 个错误 (缺省为同一错误)
:cnext	:cn	显示下一个错误
:cprevious	:cp	显示上一个错误
:clist	:cl	列出所有错误
:cfile	:cf	从文件 'errorfile' 读入错误
:cgetbuffer	:cgetb	类似于 :cbuffer 但不跳转到第一个错误
:cgetfile	:cg	类似于 :cfile 但不跳转到第一个错误
:cgetexpr	:cgete	类似于 :cexpr 但不跳转到第一个错误
:caddfile	:caddf	从错误文件加入错误到当前 quickfix 列表
:caddexpr	:cad	从表达式计算结果加入错误到当前 quickfix 列表
:cbuffer	:cb	从缓冲区文本读入错误
:cexpr	:cex	从表达式计算结果读入错误
:cquit	:cq	退出不保存并返回错误代码 (至编译器)
:make	:make [args]	启动 make, 读入错误, 并跳转到第一个错误
:grep	:gr[ep] [args]	执行 'grepprg' 程序以找出匹配并跳转到首个匹配

Q_vc 其他移动

CTRL-L	CTRL-L	清除并重画屏幕
CTRL-G	CTRL-G	显示当前文件名 (包括路径) 和光标位置
ga	ga	以十进制、十六进制和八进制显示光标所在字符的 ASCII 值
g8	g8	用于 utf-8 编码: 以十六进制显示光标所在字符的对应字节序列
g_CTRL-G	g CTRL-G	显示光标所在的列、行、以及字符位置
CTRL-C	CTRL-C	在查找中: 中断查找
dos-CTRL-Break	CTRL-Break	MS-DOS: 在查找中: 中断查找
<Del>	<Del>	输入计数时: 删除最近输入字符
:version	:ve[rsion]	显示版本信息
:mode	:mode N	MS-DOS: 置屏幕模式为 N (数值、C80、C4350 等)
:normal	:norm[al][!] {commands}	执行普通模式命令
Q	Q	切换至 "Ex" 模式

:redir	:redir >{file}	重定向消息至文件 {file}
:silent	:silent[!] {command}	安静地执行 {command} 命令
:confirm	:confirm {command}	退出、写入等有未保存的改动或文件只读时询问
:browse	:browse {command}	使用文件选择对话框打开/读入/写入文件

Q_ce 命令行编辑

c_<Esc>	<Esc>	放弃命令行（如果 'wildchar' 通配符是 <Esc>，键入两次）
c_CTRL-V	CTRL-V {char}	按本义插入 {char}
c_CTRL-V	CTRL-V {number}	输入十进制数表示的字符（可达 3 个数位）
c_CTRL-K	CTRL-K {char1} {char2}	键入二合字母（参见 Q_di ）
c_CTRL-R	CTRL-R {register}	插入指定寄存器的内容
c_<Left>	<Left>/<Right>	光标左移/右移
c_<S-Left>	<S-Left>/<S-Right>	光标左移/右移一个单词
c_CTRL-B	CTRL-B/CTRL-E	光标移动至命令行行首/行尾
c_<BS>	<BS>	删除光标前的字符
c_<Del>	<Del>	删除光标下的字符
c_CTRL-W	CTRL-W	删除光标前的单词
c_CTRL-U	CTRL-U	删除所有字符
c_<Up>	<Up>/<Down>	搜索以当前命令开始的较早/较晚的命令行
c_<S-Up>	<S-Up>/<S-Down>	从命令行历史中重放较早/较晚的命令
c_CTRL-G	CTRL-G	'incsearch' 激活时下次匹配
c_CTRL-T	CTRL-T	'incsearch' 激活时前次匹配
:history	:his[tory]	显示命令行历史

上下文关联的命令行补全：

c_wildchar	'wildchar'	（缺省：<Tab>）根据光标前的命令行补全；如果有多个匹配，响铃并显示第一个；继续 'wildchar' 将显示下一个匹配
c_CTRL-D	CTRL-D	列出匹配光标前的模式的可能的匹配
c_CTRL-A	CTRL-A	插入匹配光标前的模式的所有的匹配
c_CTRL-L	CTRL-L	插入所有匹配当前模式的名字的最长的共同部分
c_CTRL-N	CTRL-N	在 'wildchar' 之后存在多个匹配时：至下一匹配
c_CTRL-P	CTRL-P	在 'wildchar' 之后存在多个匹配时：至上一匹配

Q_ra Ex 范围

:range	,	分隔两个行号
:range	;	同上，但在解释第二个行号之前先移动光标至第一个行号代表的行
:range	{number}	绝对行号
:range	.	当前行
:range	$	文件的最后一行
:range	%	等同于 1,$（整个文件）
:range	*	等同于 '<,'>（可视区域）
:range	't	位置标记 t 的位置
:range	/{pattern}[/]	下一个匹配 {pattern} 的行
:range	?{pattern}[?]	前一个匹配 {pattern} 的行

:range	+[num]	在前面的行号上加 [num] (缺省: 1)
:range	-[num]	从前面的行号里减 [num] (缺省: 1)

Q_ex 特殊 Ex 字符

:bar	|	分隔两个命令 (不适用于 ":global" 及 ":!")
:quote	"	开始注释
:_%	%	当前文件名 (仅当期望文件名时)
:_#	#[num]	第 [num] 个轮换文件名 (仅当期望文件名时)

注: 以下七项以字面意思键入; 并非特殊键!

:<abuf>	<abuf>	缓冲区号, 用于自动命令 (仅当期望文件名时)
:<afile>	<afile>	文件名, 用于自动命令 (仅当期望文件名时)
:<amatch>	<amatch>	匹配模式的内容, 用于自动命令 (仅当期望文件名时)
:<cword>	<cword>	光标下的单词 (仅当期望文件名时)
:<cWORD>	<cWORD>	光标下的字串 (仅当期望文件名时)
:<cfile>	<cfile>	光标下的文件名 (仅当期望文件名时)
:<sfile>	<sfile>	":source" 的文件里该文件的文件名 (仅当期望文件名时)

在 "%"、"#"、"<cfile>"、"<sfile>" 或 "<afile>" 之后

::p	:p	完整路径	(path)
::h	:h	头部 (除去文件名)	(head)
::t	:t	尾部 (仅使用文件名)	(tail)
::r	:r	根部 (除去扩展名)	(root)
::e	:e	扩展名	(extension)
::s	:s/{pat}/{repl}/	以 {repl} 替换 {pat}	

Q_st 启动 Vi

-vim	vim [options]	启动 vim 并开启一个空白缓冲区
-file	vim [options] {file} ..	启动并编辑一个或多个文件
--	vim [options] -	从标准输入读入文件
-tag	vim [options] -t {tag}	编辑与标签 {tag} 关联的文件
-qf	vim [options] -q [fname]	以 QuickFix 模式开始编辑并显示首个错误

最有用的 Vim 参数 (完整列表见 startup-options)

-gui	-g	启动 GUI (同时允许其他选项)
-+	+[num]	置光标于第 [num] 行 (缺省: 末行)
-+c	+{command}	载入文件后执行命令 {command}
-+/	+/{pat} {file} ..	置光标于第一次出现 {pat} 的地方
-v	-v	Vi 模式, 以普通模式启动 ex
-e	-e	Ex 模式, 以 Ex 模式启动 vim
-R	-R	只读模式, 隐含 -n
-m	-m	禁止修改 (复位 'write' 选项)
-d	-d	比较模式 diff
-b	-b	二进制模式
-l	-l	lisp 模式
-A	-A	阿拉伯模式 (置位 'arabic')
-F	-F	波斯模式 (置位 'fkmap' 和 'rightleft')
-H	-H	希伯来模式 (置位 'hkmap' 和 'rightleft')
-V	-V	详细, 给出更多信息

-C	-C	兼容，置位 'compatible' 选项
-N	-N	不兼容，复位 'compatible' 选项
-r	-r	给出交换文件列表
-r	-r {file} ..	恢复中断的编辑
-n	-n	不创建交换文件
-o	-o [num]	打开 [num] 个窗口（缺省：每个文件一个窗口）
-f	-f	GUI：前台进程，不调用 fork Amiga：不打开新窗口启动 Vim（比如，用于 mail）
-s	-s {scriptin}	先从文件 {scriptin} 读入命令
-w	-w {scriptout}	把键入的字符写进文件 {scriptout}（添加）
-W	-W {scriptout}	把键入的字符写进文件 {scriptout}（覆盖）
-T	-T {terminal}	设置终端名
-d	-d {device}	Amiga：打开 {device} 作为控制台使用
-u	-u {vimrc}	从文件 {vimrc} 而非其它文件读入初始化命令
-U	-U {gvimrc}	同上，但用于启动 GUI 时
-i	-i {viminfo}	从文件 {viminfo} 而不是其它文件读入信息
---	--	结束选项，其余的参数都将是文件名
--help	--help	显示参数列表并退出
--version	--version	显示版本信息并退出
--	-	从标准输入读入文件

Q_ed　　文件编辑

无 !：若当前缓冲区已被改动则失败
有 !：放弃任何对当前缓冲区的改动

:edit_f	:e[dit][!] {file}	编辑 {file}
:edit	:e[dit][!]	重新载入当前文件
:enew	:ene[w][!]	编辑一个无名的新缓冲区
:find	:fin[d][!] {file}	在 'path' 当中查找文件 {file} 并编辑之
CTRL-^	N　CTRL-^	编辑第 N 个轮换文件名（同 ":e #N"）
gf	gf　或]f	编辑光标下的文件名对应的文件（goto file）
:pwd	:pwd	显示当前目录名
:cd	:cd [path]	切换当前目录到 [path]
:cd-	:cd -	回到上一次当前目录
:file	:f[ile]	显示当前文件名及光标位置
:file	:f[ile] {name}	置当前文件名为 {name}
:files	:files	显示所有的轮换文件名

Q_fl　　使用参数列表　　argument-list

:args	:ar[gs]	显示参数列表，当前文件显示在 "[]" 中
:all	:all　或 :sall	为参数列表中的每个文件打开一个窗口
:wn	:wn[ext][!]	写入当前文件并编辑下一个文件
:wn	:wn[ext][!] {file}	写到 {file} 并编辑下一个文件，除非文件 {file} 已经存在。使用 !，覆盖已存在的文件
:wN	:wN[ext][!] [file]	写入当前文件并编辑前一个文件

	在当前窗口	新建窗口	
:argument	:argu[ment] N	:sar[gument] N	编辑第 N 个文件
:next	:n[ext]	:sn[ext]	编辑下一个文件
:next_f	:n[ext] {arglist}	:sn[ext] {arglist}	定义新的文件列表并编辑其中的第一个

:Next	:N[ext]	:sN[ext]	编辑前一个文件
:first	:fir[st]	:sfir[st]	编辑第一个文件
:last	:la[st]	:sla[st]	编辑最后一个文件

Q_wq 写入和退出

:w	:[range]w[rite][!]	写入当前文件
:w_f	:[range]w[rite] {file}	写入至文件 {file}，除非其已经存在
:w_f	:[range]w[rite]! {file}	写入至文件 {file}。覆盖已存在的文件
:w_a	:[range]w[rite][!] >>	添加至当前文件
:w_a	:[range]w[rite][!] >> {file}	添加至文件 {file}
:w_c	:[range]w[rite] !{cmd}	执行命令 {cmd}，以 [range] 限定的行作为标准输入
:up	:[range]up[date][!]	如果当前文件被改动则写入
:wall	:wa[ll][!]	写入所有被改动的缓冲区

:q	:q[uit]	退出当前缓冲区，除非已经做了改动。如果没有其它非帮助的缓冲区，退出 Vim
:q	:q[uit]!	强制退出当前缓冲区，放弃所有的改动。如果没有其它非帮助的缓冲区，退出 Vim
:qa	:qa[ll]	退出 Vim，除非作了改动
:qa	:qa[ll]!	退出 Vim，放弃所有改动
:cq	:cq	退出，不写入文件并返回错误代码

:wq	:wq[!]	写入当前文件并退出
:wq	:wq[!] {file}	写入文件 {file} 并退出
:xit	:x[it][!] [file]	同 ":wq" 但是仅当有改动时写入
ZZ	ZZ	同 ":x"
ZQ	ZQ	同 ":q!"
:xall	:xa[ll][!] 或 :wqall[!]	写入所有改动的缓冲区并退出

:stop	:st[op][!]	挂起 Vim 或开始一个新外壳。如果置位了 'aw' 选项并没有给出 [!] 则写入缓冲区
CTRL-Z	CTRL-Z	同 ":stop"

Q_ac 自动命令

viminfo-file 启动时读入寄存器、标记、历史记录，退出时存储这些信息。

:rviminfo	:rv[iminfo] [file]	从文件 [file] 读入 viminfo 信息
:rviminfo	:rv[iminfo]! [file]	同上，但覆盖已存在的信息
:wviminfo	:wv[iminfo] [file]	向文件 [file] 中加入 viminfo 信息
:wviminfo	:wv[iminfo]! [file]	向文件 [file] 中写入 viminfo 信息

modeline 当编辑一个文件时自动设定的选项

modeline	vim:{set-arg}: ..	置于文件的前面或后面数行（参见 'ml' 选项），{set-arg} 为传递给 ":set" 命令的参数

autocommand 特定事件发生时自动执行命令。

:autocmd	:au	列出所有自动命令
:autocmd	:au {event}	列出针对事件 {event} 的所有自动命令
:autocmd	:au {event} {pat}	列出针对事件 {event} 并匹配 {pat} 的所有自动命令
:autocmd	:au {event} {pat} {cmd}	加入针对事件 {event} 及匹配 {pat} 的新自动命令
:autocmd	:au!	清除所有自动命令
:autocmd	:au! {event}	清除所有针对事件 {event} 的自动命令
:autocmd	:au! * {pat}	清除所有匹配 {pat} 的自动命令
:autocmd	:au! {event} {pat}	清除所有针对事件 {event} 及匹配 {pat} 的自动命令
:autocmd	:au! {event} {pat} {cmd}	清除所有针对事件 {event} 及匹配 {pat} 的自动命令并输入新的

Q_wi 多窗口命令

CTRL-W_s	CTRL-W s 或 :split	将窗口分割成两部分 (split)
:split_f	:split {file}	分隔窗口并在其中一个编辑 {file}
:vsplit	:vsplit {file}	同上，但垂直分割
:vertical	:vertical {cmd}	使命令 {cmd} 垂直分割
:sfind	:sf[ind] {file}	分割窗口，从 {path} 中找到文件 {file} 并编辑之
:terminal	:terminal {cmd}	打开新终端窗口
CTRL-W_]	CTRL-W]	分割窗口并跳转到光标下的标签
CTRL-W_f	CTRL-W f	分割窗口并编辑光标下的文件名 (file)
CTRL-W_^	CTRL-W ^	分割窗口并编辑轮换文件
CTRL-W_n	CTRL-W n 或 :new	创建新空白窗口 (new)
CTRL-W_q	CTRL-W q 或 :q[uit]	退出编辑并关闭窗口 (quit)
CTRL-W_c	CTRL-W c 或 :cl[ose]	隐藏当前缓冲区并关闭窗口 (close)
CTRL-W_o	CTRL-W o 或 :on[ly]	使当前窗口成为唯一窗口 (only)
CTRL-W_j	CTRL-W j	跳转到下方窗口
CTRL-W_k	CTRL-W k	跳转到上方窗口
CTRL-W_CTRL-W	CTRL-W CTRL-W	移动光标至下方窗口 (折转) (Wrap)
CTRL-W_W	CTRL-W W	移动光标至上方窗口 (折转) (wrap)
CTRL-W_t	CTRL-W t	跳转到顶端窗口 (top)
CTRL-W_b	CTRL-W b	跳转到底端窗口 (bottom)
CTRL-W_p	CTRL-W p	跳转到上一次激活的窗口 (previous)
CTRL-W_r	CTRL-W r	向下旋转窗口 (rotate)
CTRL-W_R	CTRL-W R	向上旋转窗口 (Rotate)
CTRL-W_x	CTRL-W x	将当前窗口与下一个窗口对调 (eXchange)
CTRL-W_=	CTRL-W =	使所有窗口等高等宽
CTRL-W_-	CTRL-W -	减少当前窗口高度
CTRL-W_+	CTRL-W +	增加当前窗口高度
CTRL-W__	CTRL-W _	设置当前窗口高度 (缺省：很高)
CTRL-W_<	CTRL-W <	减少当前窗口宽度
CTRL-W_>	CTRL-W >	增加当前窗口宽度
CTRL-W_bar	CTRL-W |	设置当前窗口宽度 (缺省：尽可能宽)

Q_bu 缓冲区列表命令

:buffers	:buffers 或 :files	列出所有已知的缓冲区和文件名	
:ball	:ball 或 :sball	编辑所有的参数/缓冲区	
:unhide	:unhide 或 :sunhide	编辑所有载入的缓冲区	
:badd	:badd {fname}	加入文件名 {fname} 到列表中	
:bunload	:bunload[!] [N]	从内存中卸载缓冲区 [N]	
:bdelete	:bdelete[!] [N]	从内存中卸载缓冲区 [N] 并从列表中删除	

	当前窗口内	新窗口内	
:buffer	:[N]buffer [N]	:[N]sbuffer [N]	至参数/缓冲区 N
:bnext	:[N]bnext [N]	:[N]sbnext [N]	往后第 N 个参数/缓冲区
:bNext	:[N]bNext [N]	:[N]sbNext [N]	往前第 N 个参数/缓冲区
:bprevious	:[N]bprevious [N]	:[N]sbprevious [N]	往前第 N 个参数/缓冲区
:bfirst	:bfirst	:sbfirst	至第一个参数/缓冲区
:blast	:blast	:sblast	至最后一个参数/缓冲区
:bmodified	:[N]bmod [N]	:[N]sbmod [N]	第 N 个改动过的缓冲区

Q_sy 语法高亮

:syn-on	:syntax on	开始使用语法高亮
:syn-off	:syntax off	停止使用语法高亮
:syn-keyword	:syntax keyword {group-name} {keyword} ..	添加语法关键字项目
:syn-match	:syntax match {group-name} {pattern} ...	加入语法匹配项目
:syn-region	:syntax region {group-name} {pattern} ...	添加语法区域项目
:syn-sync	:syntax sync [ccomment | lines {N} | ...]	设置语法高亮的同步方式
:syntax	:syntax [list]	列出当前语法项目
:syn-clear	:syntax clear	清除所有语法信息
:highlight	:highlight clear	清除所有高亮信息
:highlight	:highlight {group-name} {key}={arg} ..	为语法组 {group-name} 设置高亮
:filetype	:filetype on	开启文件类型检测，不启用语法高亮
:filetype	:filetype plugin indent on	开启文件类型检测，包括自动缩进及设置

Q_gu GUI 命令

:gui	:gui	UNIX: 启动 GUI
:gui	:gui {fname} ..	同上，并编辑 {fname} ..
:menu	:menu	列出所有菜单
:menu	:menu {mpath}	列出 {mpath} 下的所有菜单
:menu	:menu {mpath} {rhs}	把 {rhs} 加入菜单 {mpath}
:menu	:menu {pri} {mpath} {rhs}	同上，并带有优先权 {pri}

:menu	:menu ToolBar.{name} {rhs}	把 {rhs} 加入工具栏
:tmenu	:tmenu {mpath} {text}	为菜单 {mpath} 加入工具提示
:unmenu	:unmenu {mpath}	删除菜单 {mpath}

Q_fo 折叠

'foldmethod'	set foldmethod=manual	手动折叠
	set foldmethod=indent	按照缩进折叠
	set foldmethod=expr	按照表达式 'foldexpr' 折叠
	set foldmethod=syntax	按照语法区域折叠
	set foldmethod=marker	按照标志 'foldmarker' 折叠
zf	zf{motion}	操作符：手动定义一个折叠 (fold)
:fold	:{range}fold	将范围 {range} 包括的行定义为一个折叠
zd	zd	删除光标下的一个折叠 (delete)
zD	zD	删除光标下的所有折叠 (Delete)
zo	zo	打开光标下的折叠 (open)
zO	zO	打开光标下的所有折叠 (Open)
zc	zc	关闭光标下的一个折叠 (close)
zC	zC	关闭光标下的所有折叠 (Close)
zm	zm	折起更多：减少 'foldlevel' (more)
zM	zM	关闭所有折叠：置 'foldlevel' 为 0
zr	zr	减少折叠：增加 'foldlevel' (reduce)
zR	zR	打开所有折叠：置 'foldlevel' 为最大
zn	zn	不折叠：复位 'foldenable' (none)
zN	zN	正常折叠：置位 'foldenable' (Normal)
zi	zi	反转 'foldenable' (invert)

vim:tw=78:ts=8:noet:ft=help:norl:

index.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar

译者：Willis
http://vimcdoc.sf.net

index

本文件包含各个模式下所有命令的一个完整列表。包含它们的标签和简单说明。列表以 ASCII 值排序。

提示：要找特定功能，使用搜索命令。例如，要找寻如何删除，使用："/删除"。

1. 插入模式　insert-index
2. 普通模式　normal-index
   2.1. 文本对象　objects
   2.2. 窗口命令　CTRL-W
   2.3. 方括号命令　[
   2.4. 'g' 开头的命令　g
   2.5. 'z' 开头的命令　z
   2.6. 操作符等待模式　operator-pending-index
3. 可视模式　visual-index
4. 命令行编辑　ex-edit-index
5. 终端作业模式　terminal-job-index
6. EX 命令　ex-cmd-index

选项总览可见 option-list 。
内建函数总览可见 functions 。
Vim 变量的列表可见 vim-variable 。
帮助项目的完整列表可见 help-tags 。

---

1. 插入模式　insert-index

标签	字符	插入模式动作
i_CTRL-@	CTRL-@	插入最近插入的文本并停止插入
i_CTRL-A	CTRL-A	插入最近插入的文本
	CTRL-B	不用 i_CTRL-B-gone
i_CTRL-C	CTRL-C	退出插入模式，不检查缩写，除非置位了 'insertmode'。
i_CTRL-D	CTRL-D	在当前行删除一个 shiftwidth 的缩进
i_CTRL-E	CTRL-E	插入光标下方的字符
	CTRL-F	不用（但缺省是 'cinkeys' 中的一项，用以重新缩进本行）
i_CTRL-G_j	CTRL-G CTRL-J	往下一行，到插入开始的列
i_CTRL-G_j	CTRL-G j	往下一行，到插入开始的列
i_CTRL-G_j	CTRL-G <Down>	往下一行，到插入开始的列
i_CTRL-G_k	CTRL-G CTRL-K	往上一行，到插入开始的列
i_CTRL-G_k	CTRL-G k	往上一行，到插入开始的列
i_CTRL-G_k	CTRL-G <Up>	往上一行，到插入开始的列
i_CTRL-G_u	CTRL-G u	开始新的可撤销的编辑
i_CTRL-G_U	CTRL-G U	下一个光标移动不打断撤销

```
i_<BS>          <BS>              删除光标前的字符
i_digraph       {char1}<BS>{char2}
                                  输入二合字母（只有打开了 'digraph' 选项才有效）
i_CTRL-H        CTRL-H            等同于 <BS>
i_<Tab>          <Tab>              插入 <Tab> 字符
i_CTRL-I        CTRL-I            等同于 <Tab>
i_<NL>          <NL>                等同于 <CR>
i_CTRL-J        CTRL-J            等同于 <CR>
i_CTRL-K        CTRL-K {char1} {char2}
                                  输入二合字母
i_CTRL-L        CTRL-L            'insertmode' 打开时：离开插入模式
i_<CR>          <CR>                开始新行
i_CTRL-M        CTRL-M            等同于 <CR>
i_CTRL-N        CTRL-N            查找光标前面的关键字的下一个匹配
i_CTRL-O        CTRL-O            执行单个命令然后回到插入模式
i_CTRL-P        CTRL-P            查找光标前面的关键字的前一个匹配
i_CTRL-Q        CTRL-Q            等同于 CTRL-V，除非它用于终端控制流
i_CTRL-R        CTRL-R {register}
                                  插入寄存器内容
i_CTRL-R_CTRL-R  CTRL-R CTRL-R {register}
                                  按本义插入寄存器内容
i_CTRL-R_CTRL-O  CTRL-R CTRL-O {register}
                                  按本义插入寄存器内容，而且不自动缩进
i_CTRL-R_CTRL-P  CTRL-R CTRL-P {register}
                                  按本义插入寄存器内容，而且修正缩进
                CTRL-S            (用于终端控制流)
i_CTRL-T        CTRL-T            在当前行插入一个 shiftwidth 的缩进
i_CTRL-U        CTRL-U            删除当前行的所有输入字符
i_CTRL-V        CTRL-V {char}     按本义插入下一个非数字字符
i_CTRL-V_digit  CTRL-V {number}   插入三位十进制数字代表的单个字节。
i_CTRL-W        CTRL-W            删除光标前的单词
i_CTRL-X        CTRL-X {mode}     进入 CTRL-X 子模式，见  i_CTRL-X_index
i_CTRL-Y        CTRL-Y            插入光标上方的字符
i_CTRL-Z        CTRL-Z            'insertmode' 打开时：暂停 Vim
i_<Esc>          <Esc>              结束插入模式（除非打开了 'insertmode'）
i_CTRL-[        CTRL-[            等同于 <Esc>
i_CTRL-\_CTRL-N  CTRL-\ CTRL-N    进入普通模式
i_CTRL-\_CTRL-G  CTRL-\ CTRL-G    进入 'insertmode' 指定的模式
                CTRL-\ a - z      保留给扩展用
                CTRL-\ 其它       不用
i_CTRL-]        CTRL-]            切换缩写
i_CTRL-^        CTRL-^            切换  :lmap  映射的使用
i_CTRL-_        CTRL-_            'allowrevins' 打开时：改变语言
                                  (希伯来语，波斯语等)
                                  {仅当编译时加入  +rightleft  特性才有效}

                <Space> to '~'  不用，除了 CTRL-D 之前的 '0' 和 '^'

i_0_CTRL-D      0 CTRL-D          删除当前行的所有缩进
i_^_CTRL-D      ^ CTRL-D          删除当前行的所有缩进，在下一行恢复删除的缩进

i_<Del>          <Del>              删除光标所在的字符

                Meta 字符 (0x80 到 0xff，128 到 255)
```

不用

i_<Left>	<Left>	光标左移一个字符
i_<S-Left>	<S-Left>	光标左移一个单词
i_<C-Left>	<C-Left>	光标左移一个单词
i_<Right>	<Right>	光标右移一个字符
i_<S-Right>	<S-Right>	光标右移一个单词
i_<C-Right>	<C-Right>	光标右移一个单词
i_<Up>	<Up>	光标上移一行
i_<S-Up>	<S-Up>	等同于 <PageUp>
i_<Down>	<Down>	光标下移一行
i_<S-Down>	<S-Down>	等同于 <PageDown>
i_<Home>	<Home>	光标移到行首
i_<C-Home>	<C-Home>	光标移到文件首
i_<End>	<End>	光标移到行尾后面
i_<C-End>	<C-End>	光标移到文件尾后面
i_<PageUp>	<PageUp>	满屏上滚
i_<PageDown>	<PageDown>	满屏下滚
i_<F1>	<F1>	等同于 <Help>
i_<Help>	<Help>	停止插入模式显示帮助窗口
i_<Insert>	<Insert>	切换插入/替换模式
i_<LeftMouse>	<LeftMouse>	光标移到鼠标点击处
i_<ScrollWheelDown>	<ScrollWheelDown>	窗口向下滚动三行
i_<S-ScrollWheelDown>	<S-ScrollWheelDown>	窗口向下滚动完整一页
i_<ScrollWheelUp>	<ScrollWheelUp>	窗口向上滚动三行
i_<S-ScrollWheelUp>	<S-ScrollWheelUp>	窗口向上滚动完整一页
i_<ScrollWheelLeft>	<ScrollWheelLeft>	窗口向左滚动六列
i_<S-ScrollWheelLeft>	<S-ScrollWheelLeft>	窗口向左滚动完整一页
i_<ScrollWheelRight>	<ScrollWheelRight>	窗口向右滚动六列
i_<S-ScrollWheelRight>	<S-ScrollWheelRight>	窗口向右滚动完整一页

CTRL-X 子模式的命令 i_CTRL-X_index

i_CTRL-X_CTRL-D	CTRL-X CTRL-D	补全定义过的标识符
i_CTRL-X_CTRL-E	CTRL-X CTRL-E	上滚
i_CTRL-X_CTRL-F	CTRL-X CTRL-F	补全文件名
i_CTRL-X_CTRL-I	CTRL-X CTRL-I	补全标识符
i_CTRL-X_CTRL-K	CTRL-X CTRL-K	从字典补全标识符
i_CTRL-X_CTRL-L	CTRL-X CTRL-L	补全完整的行
i_CTRL-X_CTRL-N	CTRL-X CTRL-N	下一个补全
i_CTRL-X_CTRL-O	CTRL-X CTRL-O	全能 (omni) 补全
i_CTRL-X_CTRL-P	CTRL-X CTRL-P	上一个补全
i_CTRL-X_CTRL-S	CTRL-X CTRL-S	拼写建议
i_CTRL-X_CTRL-T	CTRL-X CTRL-T	从同义词词典补全标识符
i_CTRL-X_CTRL-U	CTRL-X CTRL-U	用 'completefunc' 补全
i_CTRL-X_CTRL-V	CTRL-X CTRL-V	如同 : 命令行那样补全
i_CTRL-X_CTRL-Y	CTRL-X CTRL-Y	下滚
i_CTRL-X_CTRL-]	CTRL-X CTRL-]	补全标签
i_CTRL-X_s	CTRL-X s	拼写建议

补全模式的命令（见 popupmenu-keys ）

complete_CTRL-E	CTRL-E	停止补全并回到原来的文本
complete_CTRL-Y	CTRL-Y	接受选择的匹配并停止补全

CTRL-L	从当前匹配中插入一个字符
<CR>	插入当前选择的匹配
<BS>	删除一个字符，重新搜索
CTRL-H	和 <BS> 相同
<Up>	选择上一个匹配
<Down>	选择下一个匹配
<PageUp>	选择往上几个项目的匹配
<PageDown>	选择往下几个项目的匹配
其它	停止补全并插入键入的字符

---

2. 普通模式 normal-index

CHAR	任何非空白字符
WORD	非空白字符的序列（字串）
N	命令前输入的数值
{motion}	光标移动命令
Nmove	{motion} 移动经过的文本
SECTION	可能以 '}' 而不是 '{' 开始的小节

备注：1 = 光标移动命令；2 = 可以撤销/重做

标签	字符	备注	普通模式的动作
	CTRL-@		不用
CTRL-A	CTRL-A	2	把 N 加到光标所在/之后的数值上
CTRL-B	CTRL-B	1	反向（Back）滚动 N 屏
CTRL-C	CTRL-C		终止当前的（搜索）命令
CTRL-D	CTRL-D		向下（Down）滚动 N 行（缺省：半屏）
CTRL-E	CTRL-E		向上滚动 N 行（附加（Extra）N 行）
CTRL-F	CTRL-F	1	向前（Forward）滚动 N 屏
CTRL-G	CTRL-G		显示当前文件名和位置
<BS>	<BS>	1	等同于 "h"
CTRL-H	CTRL-H	1	等同于 "h"
<Tab>	<Tab>	1	跳转到跳转表里第 N 个较新的项目
CTRL-I	CTRL-I	1	等同于 <Tab>
<NL>	<NL>	1	等同于 "j"
CTRL-J	CTRL-J	1	等同于 "j"
	CTRL-K		不用
CTRL-L	CTRL-L		重画屏幕
<CR>	<CR>	1	光标移到 N 行下面的第一个 CHAR
CTRL-M	CTRL-M	1	等同于 <CR>
CTRL-N	CTRL-N	1	等同于 "j"
CTRL-O	CTRL-O	1	跳转到跳转表里第 N 个较旧的项目
CTRL-P	CTRL-P	1	等同于 "k"
	CTRL-Q		(用于终端控制流)
CTRL-R	CTRL-R	2	重做 'u' 撤销的改变
	CTRL-S		(用于终端控制流)
CTRL-T	CTRL-T		跳转到标签列表的第 N 个较老的标签（Tag）
CTRL-U	CTRL-U		上滚（Upwards）N 行（缺省：半屏）
CTRL-V	CTRL-V		开始面向列块的可视（Visual）模式
CTRL-W	CTRL-W {char}		窗口命令，见 CTRL-W
CTRL-X	CTRL-X	2	从光标所在/之后的数值上减去 N

CTRL-Y	CTRL-Y		下滚 N 行
CTRL-Z	CTRL-Z		暂停程序（或者启动新的外壳）
	CTRL-[<Esc>		不用
CTRL-_CTRL-N	CTRL-\ CTRL-N		进入普通（Normal）模式（空动作）
CTRL-_CTRL-G	CTRL-\ CTRL-G		进入 'insertmode' 指定的模式
	CTRL-\ a - z		保留作扩展用
	CTRL-\ 其它		不用
CTRL-]	CTRL-]		:ta 到光标所在的标识符
CTRL-^	CTRL-^		编辑第 N 个轮换文件（等价于 ":e #N"）
	CTRL-_		不用
<Space>	<Space>	1	等同于 "l"
!	!{motion}{filter}	2	通过 {filter} 命令过滤 Nmove 文本
!!	!!{filter}	2	通过 {filter} 命令过滤 N 行
quote	"{a-zA-Z0-9.%#:-"}		指定下次删除、抽出或放置使用的寄存器 {a-zA-Z0-9.%#:-"}（大写用于添加）（{.%#:} 只能用于放置）
#	#	1	反向搜索第 N 次出现的光标所在的标识符
$	$	1	光标移动到往下第 N 行的行尾之后
%	%	1	查找本行下一个（花/方）括号，并转到其匹配的括号上，或者转到匹配的注释配对上，或者转到匹配的预处理命令上。
N%	{count}%	1	转到文件的百分之 N 的位置上
&	&	2	重复上次 :s
'	'{a-zA-Z0-9}	1	光标移动到位置标记 {a-zA-Z0-9} 所在行的第一个 CHAR 上
''	''	1	光标移动到最近一次跳转之前所在行的第一个 CHAR 上
'(	'(	1	光标移动到当前句子的首部所在行的第一个 CHAR 上
')	')	1	光标移动到当前句子的尾部所在行的第一个 CHAR 上
'<	'<	1	光标移动到当前缓冲区高亮区开始/曾经开始的行的第一个 CHAR 上。
'>	'>	1	光标移动到当前缓冲区高亮区结束/曾经结束的行的第一个 CHAR 上。
'[	'[	1	光标移动到当前缓冲区最近操作/放置的文本的首部所在行的第一个 CHAR 上
']	']	1	光标移动到当前缓冲区最近操作/放置的文本的尾部所在行的第一个 CHAR 上
'{	'{	1	光标移动到当前段落的首部所在行的第一个 CHAR 上
'}	'}	1	光标移动到当前段落的尾部所在行的第一个 CHAR 上
(	(	1	光标回退 N 个句子
)	)	1	光标前进 N 个句子
star	*	1	正向搜索第 N 次出现的光标所在的标识符
+	+	1	等同于 <CR>
,	,	1	反方向重复最近的 f、t、F 或 T N 次
-	-	1	光标移动到上方 N 行的第一个 CHAR 上
.	.	2	重复最近的改变，其计数被 N 取代
/	/{pattern}<CR>	1	正向搜索第 N 次出现的 {pattern}
/<CR>	/<CR>	1	正向搜索最近一次搜索使用的 {pattern}

count	0	1	光标移动到本行的第一个字符上
count	1		附加于命令前，给出计数
count	2		"
count	3		"
count	4		"
count	5		"
count	6		"
count	7		"
count	8		"
count	9		"
:	:	1	开始进入 Ex 命令
N:	{count}:		开始进入 Ex 命令，给出从当前行开始到下方 N-1 行为止的行范围
;	;	1	重复最近的 f、t、F 或 T N 次
<	<{motion}	2	左移 Nmove 文本行一个 'shiftwidth'
<<	<<	2	左移 N 行一个 'shiftwidth'
=	={motion}	2	通过 "indent" 过滤 Nmove 文本行
==	==	2	通过 "indent" 过滤 N 行
>	>{motion}	2	右移 Nmove 文本行一个 'shiftwidth'
>>	>>	2	右移 N 行一个 'shiftwidth'
?	?{pattern}<CR>	1	反向搜索第 N 次出现的 {pattern}
?<CR>	?<CR>	1	反向搜索最近一次搜索使用的 {pattern}
@	@{a-z}	2	执行 {a-z} 寄存器的内容 N 次
@:	@:		重复上次的 ":" 命令 N 次
@@	@@	2	重复上次的 @{a-z} N 次
A	A	2	在行尾附加文本 N 次
B	B	1	光标反向移动 N 个 WORD (字串)
C	["x]C	2	从光标位置到行尾部分，再加上 N-1 行进行修改 [到寄存器 x]；等价于 "c$"
D	["x]D	2	删除位置到行尾部分，再加上 N-1 行的字符 [到寄存器 x]；等价于 "d$"
E	E	1	光标正向移动到第 N 个 WORD 的结束处
F	F{char}	1	光标左移到第 N 次出现的 {char} 上
G	G	1	光标移动到第 N 行，缺省是最后一行
H	H	1	光标移动到屏幕顶部开始算的第 N 行
I	I	2	在本行第一个 CHAR 之前插入文本 N 次
J	J	2	连接 (Join) N 行；缺省为 2
K	K		查找光标所在的关键字 (Keyword)，使用 'keywordprg' 的定义
L	L	1	光标移动到屏幕底部开始算的第 N 行
M	M	1	光标移动到屏幕的中间一行
N	N	1	反方向重复最近的 '/' 或 '?' N 次
O	O	2	在光标上方开始一个新行，并插入文本，重复 N 次
P	["x]P	2	在光标放置文本 [到寄存器 x] N 次
Q	Q		切换到 "Ex" 模式
R	R	2	进入替换模式：覆盖存在的字符，重复输入文本 N-1 次
S	["x]S	2	删除 N 行 [到寄存器 x] 并开始输入；等价于 "cc"
T	T{char}	1	光标移动到左侧第 N 次出现的 {char} 之前
U	U	2	撤销一行内所有最近的改变
V	V		进入面向行的可视 (Visual) 模式
W	W	1	光标前移 N 个 WORD

X	["x]X	2	删除光标前的 N 个字符 [到寄存器 x]
Y	["x]Y		抽出 N 行 [到寄存器 x]；等价于 "yy"
ZZ	ZZ		如果修改过，保存当前文件，然后退出
ZQ	ZQ		无论如何退出当前文件
[	[{char}		方括号命令（见下 [）
	\		不用
]	]{char}		方括号命令（见下] ）
^	^	1	光标移动到本行的第一个 CHAR 上
-	-	1	光标移动到下方第 N - 1 行的第一个 CHAR 上
‘	‘{a-zA-Z0-9}	1	光标移动到位置标记 {a-zA-Z0-9}
‘(	‘(	1	光标移动当前句子的开始处
‘)	‘)	1	光标移动当前句子的结束处
‘<	‘<	1	光标移动高亮区域的开始处
‘>	‘>	1	光标移动高亮区域的结束处
‘[	‘[	1	光标移动最近一次操作/放置的文本的开始处
‘]	‘]	1	光标移动最近一次操作/放置的文本的结束处
‘‘	‘‘	1	光标移动最近一次跳转的位置
‘{	‘{	1	光标移动当前段落的开始处
‘}	‘}	1	光标移动当前段落的结束处
a	a	2	在光标处附加文本 N 次
b	b	1	光标反向移动 N 个单词
c	["x]c{motion}	2	删除 Nmove 文本 [到寄存器 x] 并开始插入
cc	["x]cc	2	删除 N 行 [到寄存器 x] 并开始插入
d	["x]d{motion}	2	删除 Nmove 文本 [到寄存器 x]
dd	["x]dd	2	删除 N 行 [到寄存器 x]
do	do	2	等同于 ":diffget"
dp	dp	2	等同于 ":diffput"
e	e	1	光标正向移动到第 N 个单词的结束处
f	f{char}	1	光标右移到第 N 次出现的 {char} 上
g	g{char}		扩展命令，见下 g
h	h	1	光标左移 N 个字符
i	i	2	在光标前插入文本 N 次
j	j	1	光标下移 N 行
k	k	1	光标上移 N 行
l	l	1	光标右移 N 个字符
m	m{A-Za-z}		在光标位置处设置位置标记 {A-Za-z}
n	n	1	重复最近的 ’/’ 或 ’?’ N 次
o	o	2	在光标下方开始一个新行，并插入文本，重复 N 次
p	["x]p	2	在光标后放置文本 [从寄存器 x] N 次
q	q{0-9a-zA-Z"}		记录输入的字符到命令寄存器 {0-9a-zA-Z"} (大写用于添加)
q	q		(在记录时) 停止记录
q:	q:		在命令行窗口里编辑 : 命令行
q/	q/		在命令行窗口里编辑 / 命令行
q?	q?		在命令行窗口里编辑 ? 命令行
r	r{char}	2	用 {char} 替换 N 个字符
s	["x]s	2	(替换) 删除 N 个字符 [到寄存器 x] 并开始插入
t	t{char}	1	光标移动到右侧第 N 次出现的 {char} 之前
u	u	2	撤销改变
v	v		开始面向字符的可视 (Visual) 模式
w	w	1	光标前移 N 个单词
x	["x]x	2	删除光标开始的 N 个字符 [到寄存器 x]

y	["x]y{motion}		抽出 Nmove 文本 [到寄存器 x]
yy	["x]yy		抽出 N 行 [到寄存器 x]
z	z{char}		'z' 开始的命令，见下 z
{	{	1	光标反向移动 N 个段落
bar	|	1	光标移到第 N 列
}	}	1	光标正向移动 N 个段落
~	~	2	'tildeop' 关闭时：切换光标所在处开始的 N 个字符的大小写，并右移光标 N 个字符
~	~{motion}		'tildeop' 打开时：切换 Nmove 文本的大小写
<C-End>	<C-End>	1	等同于 "G"
<C-Home>	<C-Home>	1	等同于 "gg"
<C-Left>	<C-Left>	1	等同于 "b"
<C-LeftMouse>	<C-LeftMouse>		":ta" 到鼠标点击所在的单词上
<C-Right>	<C-Right>	1	等同于 "w"
<C-RightMouse>	<C-RightMouse>		等同于 "CTRL-T"
<Del>	["x]<Del>	2	等同于 "x"
N<Del>	{count}<Del>		删除 {count} 最后的一位数字
<Down>	<Down>	1	等同于 "j"
<End>	<End>	1	等同于 "$"
<F1>	<F1>		等同于 <Help>
<Help>	<Help>		打开帮助窗口
<Home>	<Home>	1	等同于 "0"
<Insert>	<Insert>	2	等同于 "i"
<Left>	<Left>	1	等同于 "h"
<LeftMouse>	<LeftMouse>	1	移动光标到鼠标点击处
<MiddleMouse>	<MiddleMouse>	2	等同于在鼠标点击处 "gP"
<PageDown>	<PageDown>		等同于 CTRL-F
<PageUp>	<PageUp>		等同于 CTRL-B
<Right>	<Right>	1	等同于 "l"
<RightMouse>	<RightMouse>		开始可视模式，移动光标到鼠标点击处
<S-Down>	<S-Down>	1	等同于 CTRL-F
<S-Left>	<S-Left>	1	等同于 "b"
<S-LeftMouse>	<S-LeftMouse>		等同于在鼠标点击处 "*"
<S-Right>	<S-Right>	1	等同于 "w"
<S-RightMouse>	<S-RightMouse>		等同于在鼠标点击处 "#"
<S-Up>	<S-Up>	1	等同于 CTRL-B
<Undo>	<Undo>	2	等同于 "u"
<Up>	<Up>	1	等同于 "k"
<ScrollWheelDown>	<ScrollWheelDown>		窗口向下滚动三行
<S-ScrollWheelDown>	<S-ScrollWheelDown>		窗口向下滚动完整一页
<ScrollWheelUp>	<ScrollWheelUp>		窗口向上滚动三行
<S-ScrollWheelUp>	<S-ScrollWheelUp>		窗口向上滚动完整一页
<ScrollWheelLeft>	<ScrollWheelLeft>		窗口向左滚动六列
<S-ScrollWheelLeft>	<S-ScrollWheelLeft>		窗口向左滚动完整一页
<ScrollWheelRight>	<ScrollWheelRight>		窗口向右滚动六列
<S-ScrollWheelRight>	<S-ScrollWheelRight>		窗口向右滚动完整一页

2.1 文本对象 objects

他们可以用在操作符之后，或者在可视模式里选择一个对象。

标签 命令 操作符等待和可视模式下的动作

v_aquote	a"	双引号字符串
v_a'	a'	单引号字符串
v_a(	a(	等同于 ab
v_a)	a)	等同于 ab
v_a<	a<	"一个 <> 块" 从 '<' 到匹配的 '>'
v_a>	a>	等同于 a<
v_aB	aB	"一个大块" 从 "[{" 到 "]}" (带上括号)
v_aW	aW	"一个字串" (带上空白)
v_a[	a[	"一个 [] 块" 从 '[' 到匹配的 ']'
v_a]	a]	等同于 a[
v_a`	a`	反引号字符串
v_ab	ab	"一个块" 从 "[(" 到 "])" (带上括号)
v_ap	ap	"一个段落" (带上空白)
v_as	as	"一个句子" (带上空白)
v_at	at	"一个标签块" (带上空白)
v_aw	aw	"一个单词" (带上空白)
v_a{	a{	等同于 aB
v_a}	a}	等同于 aB
v_iquote	i"	双引号字符串，不带引号
v_i'	i'	单引号字符串，不带引号
v_i(	i(	等同于 ib
v_i)	i)	等同于 ib
v_i<	i<	"内含 <> 块" 从 '<' 到匹配的 '>'
v_i>	i>	等同于 i<
v_iB	iB	"内含大块" 从 "[{" 到 "]}"
v_iW	iW	"内含字串"
v_i[	i[	"内含 [] 块" 从 '[' 到匹配的 ']'
v_i]	i]	等同于 i[
v_i`	i`	反引号字符串，不带反引号
v_ib	ib	"内含块" 从 "[(" 到 "])"
v_ip	ip	"内含段落"
v_is	is	"内含句子"
v_it	it	"内含标签块"
v_iw	iw	"内含单词"
v_i{	i{	等同于 iB
v_i}	i}	等同于 iB

2.2 窗口命令

CTRL-W

标签	命令	普通模式下的动作
CTRL-W_CTRL-B	CTRL-W CTRL-B	等同于 "CTRL-W b"
CTRL-W_CTRL-C	CTRL-W CTRL-C	等同于 "CTRL-W c"
CTRL-W_CTRL-D	CTRL-W CTRL-D	等同于 "CTRL-W d"
CTRL-W_CTRL-F	CTRL-W CTRL-F	等同于 "CTRL-W f"
	CTRL-W CTRL-G	等同于 "CTRL-W g .."
CTRL-W_CTRL-H	CTRL-W CTRL-H	等同于 "CTRL-W h"
CTRL-W_CTRL-I	CTRL-W CTRL-I	等同于 "CTRL-W i"
CTRL-W_CTRL-J	CTRL-W CTRL-J	等同于 "CTRL-W j"
CTRL-W_CTRL-K	CTRL-W CTRL-K	等同于 "CTRL-W k"
CTRL-W_CTRL-L	CTRL-W CTRL-L	等同于 "CTRL-W l"

CTRL-W_CTRL-N	CTRL-W CTRL-N	等同于 "CTRL-W n"
CTRL-W_CTRL-O	CTRL-W CTRL-O	等同于 "CTRL-W o"
CTRL-W_CTRL-P	CTRL-W CTRL-P	等同于 "CTRL-W p"
CTRL-W_CTRL-Q	CTRL-W CTRL-Q	等同于 "CTRL-W q"
CTRL-W_CTRL-R	CTRL-W CTRL-R	等同于 "CTRL-W r"
CTRL-W_CTRL-S	CTRL-W CTRL-S	等同于 "CTRL-W s"
CTRL-W_CTRL-T	CTRL-W CTRL-T	等同于 "CTRL-W t"
CTRL-W_CTRL-V	CTRL-W CTRL-V	等同于 "CTRL-W v"
CTRL-W_CTRL-W	CTRL-W CTRL-W	等同于 "CTRL-W w"
CTRL-W_CTRL-X	CTRL-W CTRL-X	等同于 "CTRL-W x"
CTRL-W_CTRL-Z	CTRL-W CTRL-Z	等同于 "CTRL-W z"
CTRL-W_CTRL-]	CTRL-W CTRL-]	等同于 "CTRL-W]"
CTRL-W_CTRL-^	CTRL-W CTRL-^	等同于 "CTRL-W ^"
CTRL-W_CTRL-_	CTRL-W CTRL-_	等同于 "CTRL-W _"
CTRL-W_+	CTRL-W +	增加当前窗口高度 N 行
CTRL-W_-	CTRL-W -	减少当前窗口高度 N 行
CTRL-W_:	CTRL-W :	等同于 : ，编辑命令行
CTRL-W_<	CTRL-W <	减少当前窗口宽度 N 列
CTRL-W_=	CTRL-W =	使得所有窗口同宽同高
CTRL-W_>	CTRL-W >	增加当前窗口宽度 N 列
CTRL-W_H	CTRL-W H	移动当前窗口到最左端
CTRL-W_J	CTRL-W J	移动当前窗口到最底部
CTRL-W_K	CTRL-W K	移动当前窗口到最顶部
CTRL-W_L	CTRL-W L	移动当前窗口到最右端
CTRL-W_P	CTRL-W P	跳转到预览窗口
CTRL-W_R	CTRL-W R	向上旋转窗口 N 次
CTRL-W_S	CTRL-W S	等同于 "CTRL-W s"
CTRL-W_T	CTRL-W T	移动当前窗口到新标签页
CTRL-W_W	CTRL-W W	跳转到前面第 N 个窗口（回绕）
CTRL-W_]	CTRL-W]	分割窗口并跳转到光标所在的标签上
CTRL-W_^	CTRL-W ^	分割窗口并编辑轮换文件 N
CTRL-W__	CTRL-W _	设置当前窗口高度为 N（缺省：尽量大）
CTRL-W_b	CTRL-W b	跳转到底部的窗口
CTRL-W_c	CTRL-W c	关闭当前窗口（类似于 :close ）
CTRL-W_d	CTRL-W d	分割窗口并跳转到光标所在的定义上
CTRL-W_f	CTRL-W f	分割窗口并跳转到光标所在的文件名上
CTRL-W_F	CTRL-W F	分割窗口并编辑光标所在的文件名，然后跳转到文件名后给出的行号上。
CTRL-W_g_CTRL-]	CTRL-W g CTRL-]	分割窗口并在光标所在的标签上执行 :tjump
CTRL-W_g]	CTRL-W g]	分割窗口并在光标所在的标签上执行 :tselect
CTRL-W_g}	CTRL-W g }	在光标所在的标签上执行 :ptjump
CTRL-W_gf	CTRL-W g f	在新的标签页里编辑光标所在的文件名
CTRL-W_gF	CTRL-W g F	在新的标签页里编辑光标所在的文件名然后跳转到文件名后给出的行号上。
CTRL-W_gt	CTRL-W g t	等同于 gt : 跳转到下一个标签页
CTRL-W_gT	CTRL-W g T	等同于 gT : 跳转到上一个标签页
CTRL-W_h	CTRL-W h	跳转到左面第 N 个的窗口（在首个窗口上停止）
CTRL-W_i	CTRL-W i	分割窗口并跳转到光标所在的标识符的声明上
CTRL-W_j	CTRL-W j	跳转到下面第 N 个的窗口（在最后窗口上停止）
CTRL-W_k	CTRL-W k	跳转到上面第 N 个的窗口（在首个窗口上停止）
CTRL-W_l	CTRL-W l	跳转到右面第 N 个的窗口（在最后窗口上停止）
CTRL-W_n	CTRL-W n	打开新窗口，N 行高
CTRL-W_o	CTRL-W o	关闭除了当前窗口以外的所有窗口（类似于 :only ）

CTRL-W_p	CTRL-W p	转到前面一个（最近访问的）窗口
CTRL-W_q	CTRL-W q	退出当前窗口（类似于 :quit ）
CTRL-W_r	CTRL-W r	向下旋转窗口 N 次
CTRL-W_s	CTRL-W s	分割当前窗口为两部分，新的窗口 N 行高
CTRL-W_t	CTRL-W t	跳转到顶部窗口
CTRL-W_v	CTRL-W v	垂直分割当前窗口，新的窗口 N 列宽
CTRL-W_w	CTRL-W w	转到后面第 N 个窗口（回绕）
CTRL-W_x	CTRL-W x	交换当前和第 N 个窗口（缺省：下一个窗口）
CTRL-W_z	CTRL-W z	关闭预览窗口
CTRL-W_bar	CTRL-W |	设置窗口宽度为 N 列
CTRL-W_}	CTRL-W }	在预览窗口里显示光标所在的标签
CTRL-W_<Down>	CTRL-W <Down>	等同于 "CTRL-W j"
CTRL-W_<Up>	CTRL-W <Up>	等同于 "CTRL-W k"
CTRL-W_<Left>	CTRL-W <Left>	等同于 "CTRL-W h"
CTRL-W_<Right>	CTRL-W <Right>	等同于 "CTRL-W l"

2.3 方括号命令 [ ]

标签	字符	备注	普通模式下的动作
[_CTRL-D	[CTRL-D		跳转到第一个在当前文件和它包含的头文件里匹配光标所在的单词的 #define，从当前文件的头部开始
[_CTRL-I	[CTRL-I		跳转到第一个在当前文件和它包含的头文件里匹配光标所在单词的位置，从当前文件头部开始
[#	[#	1	光标移动到前面第 N 个未匹配的 #if、#else 或者 #ifdef
['	['	1	光标移动到前面的小写位置标记，定位在它所在行的第一个非空白字符上
[(	[(	1	光标移动到前面第 N 个未匹配的 '('
[star	[*	1	等同于 "[/"
[`	[`	1	光标移动到前一个小写位置标记
[/	[/	1	光标移动到前面第 N 个 C 注释的开始处
[D	[D		列出在当前文件和它包含的头文件里匹配光标所在的单词的所有定义，从当前文件的头部开始
[I	[I		列出在当前文件和它包含的头文件里匹配光标所在单词的所有位置，从当前文件的头部开始
[P	[P	2	等同于 "[p"
[[	[[	1	光标后退 N 个小节
[]	[]	1	光标后退 N 个 SECTION
[c	[c	1	光标后退 N 个更改开始的地方
[d	[d		显示第一个在当前文件和它包含的头文件里匹配光标所在的单词的 #define，从当前文件的头部开始
[f	[f		等同于 "gf"
[i	[i		显示第一个在当前文件和它包含的头文件里匹配光标所在单词的位置，从当前文件的头部开始
[m	[m	1	光标后退 N 个成员函数的开始
[p	[p	2	类似于 "P"，但调整当前行的缩进
[s	[s	1	移动到前一个拼写错误的单词
[z	[z	1	移动到打开的折叠的开始
[{	[{	1	光标后退 N 个未匹配的 '{'

[<MiddleMouse> [<MiddleMouse> 2 等同于 "[p"

]_CTRL-D	] CTRL-D		跳转到第一个在当前文件和它包含的头文件里匹配光标所在的单词的 #define，从光标位置开始
]_CTRL-I	] CTRL-I		跳转到第一个在当前文件和它包含的头文件里匹配光标所在单词的位置，从光标位置开始
]#	]#	1	光标移动到后面第 N 个未匹配的 #endif、#else
]'	]'	1	光标移动到后面的小写位置标记，定位在它所在行的第一个非空白字符上
])	])	1	光标移动到后面第 N 个未匹配的 ')'
]star	]*	1	等同于 "]/"
]`	]`	1	光标移动到后一个小写位置标记
]/	]/	1	光标移动到后面第 N 个 C 注释的结尾处
]D	]D		列出在当前文件和它包含的头文件里匹配光标所在的单词的所有定义，从光标位置开始
]I	]I		列出在当前文件和它包含的头文件里匹配光标所在单词的所有位置，从光标位置开始
]P	]P	2	等同于 "[p"
][	][	1	光标前进 N 个小节
]]	]]	1	光标前进 N 个 SECTION
]c	]c	1	光标前进 N 个更改开始的地方
]d	]d		显示第一个在当前文件和它包含的头文件里匹配光标所在的单词的 #define，从光标位置开始
]f	]f		等同于 "gf"
]i	]i		显示第一个在当前文件和它包含的头文件里匹配光标所在单词的位置，从光标位置开始
]m	]m	1	光标前进 N 个成员函数的结尾
]p	]p	2	类似于 "p"，但调整当前行的缩进
]s	]s	1	移动到后一个拼写错误的单词
]z	]z	1	移动到打开的折叠的结尾处
]}	]}	1	光标前进 N 个未匹配的 '}'
]<MiddleMouse>	]<MiddleMouse> 2 等同于 "]p"		

2.4 'g' 开始的命令

g

标签　　字符　　备注　　普通模式下的动作

---

g_CTRL-A	g CTRL-A		只有在编译时加入了 MEM_PROFILE 时才有定义：转储内存内容
g_CTRL-G	g CTRL-G		显示当前光标位置
g_CTRL-H	g CTRL-H		启动选择列块模式
g_CTRL-]	g CTRL-]		:tjump 到光标所在的标签上
g#	g#	1	类似于 "#"，但不使用 "\<" 和 "\>"
g$	g$	1	'wrap' 关闭时转到当前行最右侧屏幕上可见的字符 'wrap' 打开时转到当前屏幕行最右侧的字符
g&	g&	2	在所有行上重复上次的 ":s"
g'	g'{mark}	1	类似于 ' ，但不改变跳转表
g`	g`{mark}	1	类似于 ` ，但不改表跳转表
gstar	g*	1	类似于 "*"，但不使用 "\<" 和 "\>"
g+	g+		转到第 N 个更新的文本状态
g,	g,	1	转到改变表第 N 个更新的位置

g-	g-		转到第 N 个更旧的文本状态
g0	g0	1	'wrap' 关闭时转到当前行最左侧屏幕上可见的字符 'wrap' 打开时转到当前屏幕行最左侧的字符
g8	g8		显示光标所在的 UTF-8 字符的十六进制字节码
g;	g;	1	转到改变表第 N 个更旧的位置
g<	g<		显示上次的命令输出
g?	g?	2	Rot13 编码操作符
g?g?	g??	2	Rot13 编码当前行
g?g?	g?g?	2	Rot13 编码当前行
gD	gD	1	转到当前文件的光标所在单词的定义
gE	gE	1	反向转移到上一个 WORD 的结尾处
gH	gH		启动选择行模式
gI	gI	2	类似于 "I"，但总是从第一列开始
gJ	gJ	2	连接行，不插入空格
gN	gN	1,2	使用前次搜索模式搜索上一个匹配，并可视选择之
gP	["x]gP	2	在光标签放置文本 [从寄存器 x] N 次，光标留在插入文本之后
gQ	gQ		Vim 编辑时进入 "Ex" 模式
gR	gR	2	进入虚拟替换模式
gT	gT		转到前一个标签页
gU	gU{motion}	2	使得 Nmove 文本全部大写
gV	gV		在执行选择模式下的映射或者菜单时，不要重复选择过去的可视区域
g]	g]		:tselect 到光标所在的标签上
g^	g^	1	'wrap' 关闭时转到当前行最左侧屏幕上可见的非空白字符 'wrap' 打开时转到当前屏幕行最左侧的非空白字符
g_	g_	1	移动光标到下方 N - 1 行的最后 CHAR
ga	ga		打印光标所在处的字符的 ascii 值
gd	gd	1	转到当前函数里当前光标所在的单词的定义
ge	ge	1	反向转到前一个单词的结尾处
gf	gf		开始编辑当前光标所在的名字对应的文件
gF	gF		开始编辑光标所在的文件名并跳转到文件名后的行号上。
gg	gg	1	光标移动到第 N 行，缺省是第一行
gh	gh		开始选择模式
gi	gi	2	类似于 "i"，但先移动到 '^ 位置标记
gj	gj	1	类似于 "j"，但 'wrap' 打开时往下 N 个屏幕行
gk	gk	1	类似于 "k"，但 'wrap' 打开时往上 N 个屏幕行
gm	gm	1	转到屏幕行正中间的那个字符
gM	gM	1	转到文本行正中间的那个字符
gn	gn	1,2	使用前次搜索模式搜索下一个匹配，并可视选择之
go	go	1	光标移动到缓冲区的第 N 个字节
gp	["x]gp	2	在光标后面放置文本 [从寄存器 x] N 次，光标留在后面
gq	gq{motion}	2	排版 Nmove 文本
gr	gr{char}	2	虚拟替换 N 个字符为 {char}
gs	gs		睡眠 N 秒 (缺省 1)
gt	gt		转到下一个标签页
gu	gu{motion}	2	使得 Nmove 文本全部小写

gv	gv		重新选择上次的可视区域
gw	gw{motion}	2	排版 Nmove 文本并保持光标位置
netrw-gx	gx		执行光标下的文件名对应的应用程序（仅限于 netrw 插件）
g@	g@{motion}		调用 'operatorfunc'
g~	g~{motion}	2	改变 Nmove 文本的大小写
g<Down>	g<Down>	1	等同于 "gj"
g<End>	g<End>	1	等同于 "g$"
g<Home>	g<Home>	1	等同于 "g0"
g<LeftMouse>	g<LeftMouse>		等同于 <C-LeftMouse>
	g<MiddleMouse>		等同于 <C-MiddleMouse>
g<RightMouse>	g<RightMouse>		等同于 <C-RightMouse>
g<Up>	g<Up>	1	等同于 "gk"

2.5 'z' 开始的命令 z

标签	字符	备注	普通模式下的动作
z<CR>	z<CR>		重画，光标移动到窗口的顶行的第一个非空白字符
zN<CR>	z{height}<CR>		重画，使得窗口高度为 {height} 行
z+	z+		光标移动到第 N 行（缺省为窗口之后的第一行），其它同 "z<CR>"
z-	z-		重画，光标移动到窗口末行的第一个非空白字符
z.	z.		重画，光标移动到窗口的中间行的第一个非空白字符
z=	z=		给出拼写建议
zA	zA		递归打开一个关闭的折叠或关闭一个打开的折叠
zC	zC		递归关闭折叠
zD	zD		递归删除折叠
zE	zE		除去所有折叠
zF	zF		为 N 行建立折叠
zG	zG		把单词标记为拼写正确（good）
zH	zH		'wrap' 关闭时向右滚屏半个屏幕宽度
zL	zL		'wrap' 关闭时向左滚屏半个屏幕宽度
zM	zM		设置 'foldlevel' 为零
zN	zN		置位 'foldenable'
zO	zO		递归打开折叠
zR	zR		设置 'foldlevel' 为最大折叠级别
zW	zW		把单词标记为拼写错误（wrong）
zX	zX		重新应用 'foldlevel'
z^	z^		光标移动到第 N 行（缺省为窗口之前的一行），其它同 "z-"
za	za		打开关闭的折叠，或关闭打开的折叠
zb	zb		重画，光标在窗口底行
zc	zc		关闭折叠
zd	zd		删除折叠
ze	ze		'wrap' 关闭时，水平滚动使得光标定位到屏幕的尾部（最右侧）
zf	zf{motion}		为 Nmove 文本建立折叠
zg	zg		把单词标记为拼写正确（good）
zh	zh		'wrap' 关闭时，水平右滚屏幕 N 个字符

zi	zi		切换 'foldenable'
zj	zj	1	移动到下一个折叠的开始处
zk	zk	1	移动到前一个折叠的结束处
zl	zl		'wrap' 关闭时，水平左滚屏幕 N 个字符
zm	zm		从 'foldlevel' 减一
zn	zn		复位 'foldenable'
zo	zo		打开折叠
zr	zr		给 'foldlevel' 加一
zs	zs		'wrap' 关闭时，水平滚动使得光标定位到屏幕的起始处（最左侧）
zt	zt		重画，光标移动到窗口的顶部
zuw	zuw		撤销 zw
zug	zug		撤销 zg
zuW	zuW		撤销 zW
zuG	zuG		撤销 zG
zw	zw		把单词标记为拼写错误 (wrong)
zv	zv		打开足够的折叠，使得当前行可见
zx	zx		重新应用 'foldlevel' 然后执行 "zv"
zz	zz		重画，光标移动到窗口的中间行
z<Left>	z<Left>		等同于 "zh"
z<Right>	z<Right>		等同于 "zl"

2.6 操作符等待模式 operator-pending-index

它们在操作符后但在键入 {motion} 之前使用。

标签	字符	操作符等待模式下的动作
o_v	v	强制操作符面向字符
o_V	V	强制操作符面向行
o_CTRL-V	CTRL-V	强制操作符面向列块

3. 可视模式 visual-index

可视模式下，大多数命令同普通模式的相同。以下列出的是不同的部分。

标签	字符	备注	可视模式下的动作
v_CTRL-_CTRL-N	CTRL-\ CTRL-N		结束可视模式
v_CTRL-_CTRL-G	CTRL-\ CTRL-G		转到 'insertmode' 指定的模式
v_CTRL-A	CTRL-A	2	给高亮文本中的数值加上 N
v_CTRL-C	CTRL-C		结束可视模式
v_CTRL-G	CTRL-G		在可视模式和选择模式间切换
v_<BS>	<BS>	2	选择模式：删除高亮区域
v_CTRL-H	CTRL-H	2	等同于 <BS>
v_CTRL-O	CTRL-O		从选择模式切换到可视模式，只限于下个命令
v_CTRL-V	CTRL-V		使得可视模式面向列块，或者退出可视模式
v_CTRL-X	CTRL-X	2	给高亮文本中的数值减去 N
v_<Esc>	<Esc>		结束可视模式
v_CTRL-]	CTRL-]		跳转到高亮的标签上
v_!	!{filter}	2	通过外部命令 {filter} 过滤高亮行

v_:	:		使用高亮行作为范围，开始一个命令行
v_<	<	2	左移高亮行一个 'shiftwidth'
v_=	=	2	用 {equalprg} 选项指定的外部程序过滤高亮行
v_>	>	2	右移高亮行一个 'shiftwidth'
v_b_A	A	2	列块模式下：在高亮区域的所有行后附加相同的文本
v_C	C	2	删除高亮行，并开始插入
v_D	D	2	删除高亮行
v_b_I	I	2	列块模式：在所有高亮行之前插入相同的文本
v_J	J	2	连接高亮行
v_K	K		在高亮区域上运行 'keywordprg'
v_O	O		水平移动到区域的另外一侧
	Q		不启动 Ex 模式
v_R	R	2	删除高亮行并开始插入
v_S	S	2	删除高亮行并开始插入
v_U	U	2	使得高亮区域全变大写
v_V	V		使得可视区域面向行，或者退出可视模式
v_X	X	2	删除高亮区域
v_Y	Y		抽出高亮行
v_aquote	a"		扩展高亮区域，使包含一个双引号字符串
v_a'	a'		扩展高亮区域，使包含一个单引号字符串
v_a(	a(		等同于 ab
v_a)	a)		等同于 ab
v_a<	a<		扩展高亮区域，使包含一个 <> 块
v_a>	a>		等同于 a<
v_aB	aB		扩展高亮区域，使包含一个 {} 块
v_aW	aW		扩展高亮区域，使包含 "一个 WORD"
v_a[	a[		扩展高亮区域，使包含一个 [] 块
v_a]	a]		等同于 a[
v_a`	a`		扩展高亮区域，使包含一个反引号字符串
v_ab	ab		扩展高亮区域，使包含一个 () 块
v_ap	ap		扩展高亮区域，使包含一个段落
v_as	as		扩展高亮区域，使包含一个句子
v_at	at		扩展高亮区域，使包含一个标签块
v_aw	aw		扩展高亮区域，使包含 "一个单词"
v_a{	a{		等同于 aB
v_a}	a}		等同于 aB
v_c	c	2	删除高亮区域，并开始插入
v_d	d	2	删除高亮区域
v_g_CTRL-A	g CTRL-A	2	给高亮文本中的数值加上 N
v_g_CTRL-X	g CTRL-X	2	给高亮文本中的数值减去 N
v_gJ	gJ	2	连接高亮行，不插入空格
v_gq	gq	2	排版高亮行
v_gv	gv		交换现在和以前的高亮区域
v_iquote	i"		扩展高亮区域，使包含一个双引号字符串 (但不含引号)
v_i'	i'		扩展高亮区域，使包含一个单引号字符串 (但不含引号)
v_i(	i(		等同于 ib
v_i)	i)		等同于 ib
v_i<	i<		扩展高亮区域，使包含内含 <> 块
v_i>	i>		等同于 i<
v_iB	iB		扩展高亮区域，使包含内含 {} 块
v_iW	iW		扩展高亮区域，使包含 "内含 WORD"

v_i[	i[		扩展高亮区域，使包含内含 [] 块
v_i]	i]		等同于 i[
v_i‘	i‘		扩展高亮区域，使包含一个反引号字符串 (但不含反引号)
v_ib	ib		扩展高亮区域，使包含内含 () 块
v_ip	ip		扩展高亮区域，使包含内含段落
v_is	is		扩展高亮区域，使包含内含句子
v_it	it		扩展高亮区域，使包含内含标签块
v_iw	iw		扩展高亮区域，使包含 "内含单词"
v_i{	i{		等同于 iB
v_i}	i}		等同于 iB
v_o	o		移动光标到区域的另一侧
v_r	r	2	用字符替换高亮区域
v_s	s	2	删除高亮区域，并开始插入
v_u	u	2	使得高亮区域全变小写
v_v	v		使得可视模式面向字符，或者退出可视模式
v_x	x	2	删除高亮区域
v_y	y		抽出高亮区域
v_~	~	2	改变高亮区域的大小写

---

4. 命令行编辑 ex-edit-index

使用 ':'、'!'、'/' 或 '?' 命令进入命令行编辑。
输入普通字符会在当前光标位置插入该字符。
下面的 "补全" 指上下文相关的补全。如果可能，会补全文件名、标签、命令等。

标签	字符	命令行编辑模式下的动作
	CTRL-@	不用
c_CTRL-A	CTRL-A	使用光标前面的模式进行补全，并插入所有的匹配
c_CTRL-B	CTRL-B	光标移动到命令行开始的地方
c_CTRL-C	CTRL-C	等同于 <Esc>
c_CTRL-D	CTRL-D	列出光标前面模式的所有补全
c_CTRL-E	CTRL-E	光标移动到命令行结束的地方
'cedit'	CTRL-F	'cedit' 的缺省值：打开命令行窗口；否则不用
c_CTRL-G	CTRL-G	'incsearch' 激活时，下个匹配
c_<BS>	<BS>	删除光标前面的字符
c_digraph	{char1} <BS> {char2}	'digraph' 打开时，输入二合字母
c_CTRL-H	CTRL-H	等同于 <BS>
c_<Tab>	<Tab>	如果 'wildchar' 是 <Tab>：补全光标前面的模式
c_<S-Tab>	<S-Tab>	等同于 CTRL-P
c_wildchar	'wildchar'	补全光标前面的模式 (缺省：<Tab>)
c_CTRL-I	CTRL-I	等同于 <Tab>
c_<NL>	<NL>	等同于 <CR>
c_CTRL-J	CTRL-J	等同于 <CR>
c_CTRL-K	CTRL-K {char1} {char2}	输入二合字母
c_CTRL-L	CTRL-L	补全光标前的模式，并插入最长的公共子串
c_<CR>	<CR>	执行输入的命令
c_CTRL-M	CTRL-M	等同于 <CR>
c_CTRL-N	CTRL-N	使用 'wildchar' 并有多个匹配：转到下一个匹配，

		不然：从历史中调入后一个命令行
	CTRL-O	不同
c_CTRL-P	CTRL-P	使用 'wildchar' 并有多个匹配：转到上一个匹配，不然：从历史中调入前一个命令行
c_CTRL-Q	CTRL-Q	等同于 CTRL-V，除非用于终端控制流
c_CTRL-R	CTRL-R {regname}	插入寄存器的内容或特殊对象到光标所在的地方，如同直接输入那样
c_CTRL-R_CTRL-R	CTRL-R CTRL-R {regname}	
c_CTRL-R_CTRL-O	CTRL-R CTRL-O {regname}	按本义插入寄存器的内容或特殊对象到光标所在的地方
	CTRL-S	(用于终端控制流)
c_CTRL-T	CTRL-T	'incsearch' 激活时，前个匹配
c_CTRL-U	CTRL-U	删除所有字符
c_CTRL-V	CTRL-V	按本义插入下一个非数字字符，或者插入其后三位数字所代表的单个字节。
c_CTRL-W	CTRL-W	删除光标前面的单词
	CTRL-X	不用（保留作补全用）
	CTRL-Y	复制（抽出）无模式的选择
	CTRL-Z	不用（保留作暂停用）
c_<Esc>	<Esc>	放弃命令行，不执行
c_CTRL-[	CTRL-[	等同于 <Esc>
c_CTRL-_CTRL-N	CTRL-\ CTRL-N	转到普通模式，放弃命令行
c_CTRL-_CTRL-G	CTRL-\ CTRL-G	转到 'insertmode' 指定的模式，放弃命令行
	CTRL-\ a - d	保留给扩展用
c_CTRL-_e	CTRL-\ e {expr}	以 {expr} 的结果替换命令行
	CTRL-\ f - z	保留给扩展用
	CTRL-\ 其它	不用
c_CTRL-]	CTRL-]	激活缩写
c_CTRL-^	CTRL-^	切换 :lmap 映射的使用
c_CTRL-_	CTRL-_	'allowrevins' 打开时：改变语言（希伯来，波斯）
c_<Del>	<Del>	删除光标所在的字符
c_<Left>	<Left>	光标左移
c_<S-Left>	<S-Left>	光标左移一个单词
c_<C-Left>	<C-Left>	光标左移一个单词
c_<Right>	<Right>	光标右移
c_<S-Right>	<S-Right>	光标右移一个单词
c_<C-Right>	<C-Right>	光标右移一个单词
c_<Up>	<Up>	从历史中调入匹配光标前模式的前一个命令行
c_<S-Up>	<S-Up>	从历史中调入前一个命令行
c_<Down>	<Down>	从历史中调入匹配光标前模式的后一个命令行
c_<S-Down>	<S-Down>	从历史中调入后一个命令行
c_<Home>	<Home>	光标移动到命令行开始的地方
c_<End>	<End>	光标移动到命令行结束的地方
c_<PageDown>	<PageDown>	等同于 <S-Down>
c_<PageUp>	<PageUp>	等同于 <S-Up>
c_<Insert>	<Insert>	切换插入/替换模式
c_<LeftMouse>	<LeftMouse>	光标移动到鼠标点击的地方

5．终端作业模式 terminal-job-index

除了窗口命令（ CTRL-W ）以外，绝大多数普通模式命令不能直接应用在终端窗口。要用这些命令，先切换到终端普通模式。
这里假定未设置 'termwinkey'。

标签	字符	终端作业模式下的动作
t_CTRL-_CTRL-N	CTRL-\ CTRL-N	切换到终端普通模式
t_CTRL-W_N	CTRL-W N	切换到终端普通模式
t_CTRL-W_:	CTRL-W :	输入 Ex 命令
t_CTRL-W_.	CTRL-W .	在终端中输入 CTRL-W
	CTRL-W CTRL-\	给终端中的作业发送 CTRL-\
t_CTRL-W_quote	CTRL-W " {0-9a-z"%#*:=}	在终端中粘贴寄存器
t_CTRL-W_CTRL-C	CTRL-W CTRL-C	强制结束作业
t_CTRL-W_CTRL-W	CTRL-W CTRL-W	移动焦点到下个窗口
t_CTRL-W_gt	CTRL-W gt	转到下一个标签页，等同于 gt
t_CTRL-W_gT	CTRL-W gT	转到上一个标签页，等同于 gT

亚瑟王，你找到（圣杯）了！ holy-grail :smile

6. EX 命令 ex-cmd-index :index

这里是一个简短但完整的 ":" 命令的列表。这里不讨论参数。命令可选的部分用 [] 括起。命令按照名字非可选部分排序。

标签	字符	动作
:	:	什么都不作
:range	:{range}	转到 {range} 的末行
:!	:!	过滤行，或可通过外部命令执行之
:!!	:!!	重复上次的 ":!" 命令
:#	:#	等同于 ":number"
:&	:&	重复上次的 ":substitute"
:star	:*	执行寄存器的内容
:<	:<	将行左移一个 'shiftwidth'
:=	:=	显示光标所在的行号
:>	:>	将行右移一个 'shiftwidth'
:@	:@	执行寄存器的内容
:@@	:@@	重复上次的 ":@"
:Next	:N[ext]	转到参数列表的上一个文件
:Print	:P[rint]	显示行
:X	:X	请求加密密钥
:append	:a[ppend]	附加文本
:abbreviate	:ab[breviate]	输入缩写
:abclear	:abc[lear]	删除所有的缩写
:aboveleft	:abo[veleft]	使得分割的窗口出现在左侧或者上方
:all	:al[l]	为参数列表的每个文件打开一个窗口
:amenu	:am[enu]	为所有模式输入菜单项
:anoremenu	:an[oremenu]	为所有模式输入菜单项，不会被重新映射
:args	:ar[gs]	显示参数列表
:argadd	:arga[dd]	增加项目到参数列表中

:argdelete	:argd[elete]	从参数列表中删除项目
:argedit	:arge[dit]	增加项目到参数列表中并编辑之
:argdo	:argdo	在参数列表上的所有项目上执行一个命令
:argglobal	:argg[lobal]	定义全局参数列表
:arglocal	:argl[ocal]	定义本地参数列表
:argument	:argu[ment]	转到参数列表一个指定的文件
:ascii	:as[cii]	显示光标所在的字符的 ascii 值
:autocmd	:au[tocmd]	输入或者显示自动命令
:augroup	:aug[roup]	选择要用的自动命令组
:aunmenu	:aun[menu]	删除所有模式下的菜单
:buffer	:b[uffer]	转到缓冲区列表里的特定缓冲区
:bNext	:bN[ext]	转到缓冲区列表里的上一个缓冲区
:ball	:ba[ll]	为缓冲区列表里的每个缓冲区打开一个窗口
:badd	:bad[d]	增加缓冲区到缓冲区列表
:bdelete	:bd[elete]	从缓冲区列表删除缓冲区
:behave	:be[have]	设置鼠标和选择行为
:belowright	:bel[owright]	使得分割的窗口出现在右侧或者下方
:bfirst	:bf[irst]	转到缓冲区列表里的第一个缓冲区
:blast	:bl[ast]	转到缓冲区列表里的最后一个缓冲区
:bmodified	:bm[odified]	转到缓冲区列表里的下一个修改过的缓冲区
:bnext	:bn[ext]	转到缓冲区列表里的下一个缓冲区
:botright	:bo[tright]	使得分割的窗口出现在最右侧或者下方
:bprevious	:bp[revious]	转到缓冲区列表里的上一个缓冲区
:brewind	:br[ewind]	转到缓冲区列表里的第一个缓冲区
:break	:brea[k]	跳出 while 循环
:breakadd	:breaka[dd]	增加调试器断点
:breakdel	:breakd[el]	删除调试器断点
:breaklist	:breakl[ist]	列出调试器断点
:browse	:bro[wse]	使用文件选择对话框
:bufdo	:bufdo	为每个列表内缓冲区执行命令
:buffers	:buffers	列出缓冲区列表里的所有文件
:bunload	:bun[load]	卸载指定的缓冲区
:bwipeout	:bw[ipeout]	真正删除一个缓冲区
:change	:c[hange]	替换一行或若干行
:cNext	:cN[ext]	跳转到上一个错误
:cNfile	:cNf[ile]	跳转到前一个文件的最后一个错误
:cabbrev	:ca[bbrev]	类似于 ":abbreviate"，但用于命令行模式
:cabclear	:cabc[lear]	清除所有缩写，用于命令行模式
:cabove	:cabo[ve]	转到当前行之上的错误
:caddbuffer	:cad[dbuffer]	通过缓冲区补充错误
:caddexpr	:cadde[xpr]	通过表达式补充错误
:caddfile	:caddf[ile]	加入错误信息到当前 quickfix 列表
:cafter	:caf[ter]	转到当前光标之后的错误
:call	:cal[l]	调用函数
:catch	:cat[ch]	:try 命令的一部分
:cbefore	:cbef[ore]	转到当前光标之前的错误
:cbelow	:cbel[ow]	转到当前行之下的错误
:cbottom	:cbo[ttom]	滚动到 quickfox 窗口底部
:cbuffer	:cb[uffer]	分析错误信息并跳转到第一个
:cc	:cc	跳转到特定的错误
:cclose	:ccl[ose]	关闭 quickfix 窗口
:cd	:cd	改变目录
:cdo	:cdo	为每个合法的错误列表项目执行命令
:cfdo	:cfd[o]	为每个错误列表的文件执行命令

:center	:ce[nter]	排版行，使其居中对齐
:cexpr	:cex[pr]	从表达式里读取错误并跳转到第一个
:cfile	:cf[ile]	读入含有错误信息的文件，并跳转到第一个
:cfirst	:cfir[st]	跳转到指定的错误上，缺省为第一个
:cgetbuffer	:cgetb[uffer]	从缓冲区里得到错误
:cgetexpr	:cgete[xpr]	从表达实里得到错误
:cgetfile	:cg[etfile]	读入含有错误信息的文件
:changes	:changes	显示修改的列表
:chdir	:chd[ir]	改变当前目录
:checkpath	:che[ckpath]	列出包含的头文件
:checktime	:checkt[ime]	检查载入缓冲区的时间标签
:chistory	:chi[story]	列出所有的错误列表
:clast	:cla[st]	跳转到指定的错误上，缺省为最后一个
:clearjumps	:cle[arjumps	清除跳转列表
:clist	:cl[ist]	列出所有错误
:close	:clo[se]	改变当前窗口
:cmap	:cm[ap]	类似于 ":map"，但用于命令行模式
:cmapclear	:cmapc[lear]	清除所有映射，用于命令行模式
:cmenu	:cme[nu]	为命令行模式增加菜单
:cnext	:cn[ext]	跳转到下一个错误
:cnewer	:cnew[er]	跳转到下一个较新的错误列表
:cnfile	:cnf[ile]	跳转到下一个文件的第一个错误
:cnoremap	:cno[remap]	类似于 ":noremap"，但用于命令行模式
:cnoreabbrev	:cnorea[bbrev]	类似于 ":noreabbrev"，但用于命令行模式
:cnoremenu	:cnoreme[nu]	类似于 ":noremenu"，但用于命令行模式
:copy	:co[py]	复制行
:colder	:col[der]	跳转到上一个较老的错误列表
:colorscheme	:colo[rscheme]	载入特定的色彩方案
:command	:com[mand]	建立用户自定的命令
:comclear	:comc[lear]	清除所有用户自定的命令
:compiler	:comp[iler]	为特定编译器设定选项
:continue	:con[tinue]	回到 :while 开始处
:confirm	:conf[irm]	需要确认时，提示用户
:const	:cons[t]	建立常量变量
:copen	:cope[n]	打开 quickfix 窗口
:cprevious	:cp[revious]	跳转到上一个错误
:cpfile	:cpf[ile]	跳转到上一个文件的最后一个错误
:cquit	:cq[uit]	退出 Vim，带错误码
:crewind	:cr[ewind]	跳到指定的错误，缺省为第一个
:cscope	:cs[cope]	执行 cscope 命令
:cstag	:cst[ag]	使用 cscope 来跳转到标签上
:cunmap	:cu[nmap]	类似于 ":unmap"，但用于命令行模式
:cunabbrev	:cuna[bbrev]	类似于 ":unabbrev"，但用于命令行模式
:cunmenu	:cunme[nu]	删除命令行的菜单
:cwindow	:cw[indow]	打开或者关闭 quickfix 窗口
:delete	:d[elete]	删除行
:delmarks	:delm[arks]	删除位置标记
:debug	:deb[ug]	在调试模式执行命令
:debuggreedy	:debugg[reedy]	从正常的输入那里读取调试模式命令
:delcommand	:delc[ommand]	删除用户自定的命令
:delfunction	:delf[unction]	删除用户函数
:diffupdate	:dif[fupdate]	更新 'diff' 缓冲区
:diffget	:diffg[et]	删除当前缓冲区的所有差异
:diffoff	:diffo[ff]	关闭比较模式

:diffpatch	:diffp[atch]	应用补丁然后显示差异
:diffput	:diffpu[t]	删除另外一个缓冲区的差异
:diffsplit	:diffs[plit]	显示和另外一个文件的差异
:diffthis	:diffthis	使得当前窗口成为 diff 窗口
:digraphs	:dig[raphs]	显示或者输入二合字母
:display	:di[splay]	显示寄存器
:dl	:dl	带 'l' 标志位 :delete 的简写
:del	:del[ete]l	带 'l' 标志位 :delete 的简写
:djump	:dj[ump]	跳转到 #define
:dlist	:dli[st]	列出所有的 #define
:doautocmd	:do[autocmd]	在当前缓冲区应用自动命令
:doautoall	:doautoa[ll]	在所有载入的缓冲区上应用自动命令
:dp	:d[elete]p	带 'p' 标志位 :delete 的简写
:drop	:dr[op]	跳到编辑某文件的窗口，不成功则在当前窗口编辑之
:dsearch	:ds[earch]	列出一个 #define
:dsplit	:dsp[lit]	分割窗口并跳转到 #define
:edit	:e[dit]	编辑文件
:earlier	:ea[rlier]	回到旧的改变，撤销
:echo	:ec[ho]	回显表达式结果
:echoerr	:echoe[rr]	类似于 :echo，如同错误一样显示和使用历史
:echohl	:echoh[l]	为 echo 等命令设置高亮
:echomsg	:echom[sg]	等同于 :echo，在历史里放置消息
:echon	:echon	等同于 :echo，但不留 <EOL>
:else	:el[se]	:if 命令的一部分
:elseif	:elsei[f]	:if 命令的一部分
:emenu	:em[enu]	按名字执行菜单
:endif	:en[dif]	结束最近的 :if
:endfor	:endfo[r]	结束最近的 :for
:endfunction	:endf[unction]	结束用户函数
:endtry	:endt[ry]	结束最近的 :try
:endwhile	:endw[hile]	结束最近的 :while
:enew	:ene[w]	编辑新的无名缓冲区
:ex	:ex	等同于 ":edit"
:execute	:exe[cute]	执行表达式的结果产生的字符串
:exit	:exi[t]	等同于 ":xit"
:exusage	:exu[sage]	Ex 命令总览
:file	:f[ile]	显示或者设置当前文件名
:files	:files	列出缓冲区列表的所有文件
:filetype	:filet[ype]	切换文件类型检测的打开/关闭
:filter	:filt[er]	过滤跟随的命令的输出结果
:find	:fin[d]	在 'path' 里找寻文件并编辑之
:finally	:fina[lly]	:try 命令的一部分
:finish	:fini[sh]	退出 Vim 脚本的执行
:first	:fir[st]	转到参数列表的第一个文件
:fixdel	:fix[del]	设置 <Del> 的键码
:fold	:fo[ld]	创建折叠
:foldclose	:foldc[lose]	关闭折叠
:folddoopen	:foldd[oopen]	对所有不在关闭折叠的行执行命令
:folddoclosed	:folddoc[losed]	对所有在关闭折叠的行执行命令
:foldopen	:foldo[pen]	打开折叠
:for	:for	for 循环
:function	:fu[nction]	定义用户函数
:global	:g[lobal]	对匹配的行执行命令
:goto	:go[to]	转到缓冲区某字节处

:grep	:gr[ep]	执行 'grepprg' 并转到第一个匹配处
:grepadd	:grepa[dd]	类似 :grep，但附加到当前列表后
:gui	:gu[i]	启动 GUI
:gvim	:gv[im]	启动 GUI
:hardcopy	:ha[rdcopy]	发送文本到打印机
:help	:h[elp]	打开帮助窗口
:helpclose	:helpc[lose]	关闭帮助窗口
:helpfind	:helpf[ind]	打开显示帮助窗口的对话框
:helpgrep	:helpg[rep]	类似于 ":grep"，但搜索帮助文件
:helptags	:helpt[ags]	为一个目录产生帮助标签
:highlight	:hi[ghlight]	指定高亮方法
:hide	:hid[e]	为一个命令隐藏当前缓冲区
:history	:his[tory]	显示历史列表
:insert	:i[nsert]	插入文本
:iabbrev	:ia[bbrev]	类似于 ":abbrev"，但用于插入模式
:iabclear	:iabc[lear]	类似于 ":abclear"，但用于插入模式
:if	:if	条件满足时执行命令
:ijump	:ij[ump]	跳转到标识符的定义
:ilist	:il[ist]	列出匹配标识符的行
:imap	:im[ap]	类似于 ":map"，但用于插入模式
:imapclear	:imapc[lear]	类似于 ":map"，但用于插入模式
:imenu	:ime[nu]	为插入模式增加菜单
:inoremap	:ino[remap]	类似于 ":noremap"，但用于插入模式
:inoreabbrev	:inorea[bbrev]	类似于 ":noreabbrev"，但用于插入模式
:inoremenu	:inoreme[nu]	类似于 ":noremenu"，但用于插入模式
:intro	:int[ro]	显示介绍文字
:isearch	:is[earch]	列出匹配标识符的一行
:isplit	:isp[lit]	分割窗口，并跳转到标识符的定义
:iunmap	:iu[nmap]	类似于 ":unmap"，但用于插入模式
:iunabbrev	:iuna[bbrev]	类似于 ":unabbrev"，但用于插入模式
:iunmenu	:iunme[nu]	删除插入模式的菜单
:join	:j[oin]	连接行
:jumps	:ju[mps]	显示跳转表
:k	:k	设置位置标记
:keepalt	:keepa[lt]	跟随的命令保持轮换文件不变
:keepmarks	:kee[pmarks]	跟随的命令保持位置标记不变
:keepjumps	:keepj[umps]	跟随的命令保持跳转表和位置标记不变
:keeppatterns	:keepp[atterns]	跟随的命令保持搜索模式历史不变
:lNext	:lN[ext]	转到位置列表的上一个位置
:lNfile	:lNf[ile]	转到上个文件的最近一个位置
:list	:l[ist]	显示行
:labove	:lab[ove]	转到当前行之上的位置
:laddexpr	:lad[dexpr]	通过表达式补充位置
:laddbuffer	:laddb[uffer]	通过缓冲区补充位置
:laddfile	:laddf[ile]	把位置加到当前位置列表里
:lafter	:laf[ter]	转到当前光标之后的位置
:last	:la[st]	转到参数列表的最后一个文件
:language	:lan[guage]	设置语言 (locale)
:later	:lat[er]	转到新的改变，重做
:lbefore	:lbef[ore]	转到当前光标之前的位置
:lbelow	:lbel[ow]	转到当前行之下的位置
:lbottom	:lbo[ttom]	滚动到位置窗口的底部
:lbuffer	:lb[uffer]	分析位置并跳转到第一个位置
:lcd	:lc[d]	改变本地的目录

:lchdir	:lch[dir]	改变本地的目录
:lclose	:lcl[ose]	关闭位置窗口
:lcscope	:lcs[cope]	类似于 ":cscope"，但使用位置列表
:ldo	:ld[o]	为每个合法位置列表项目执行命令
:lfdo	:lfd[o]	为每个位置列表的文件执行命令
:left	:le[ft]	左对齐行
:leftabove	:lefta[bove]	使得分割的窗口出现在左侧或者上方
:let	:let	给变量或选项赋值
:lexpr	:lex[pr]	从表达式里读取位置并跳转到第一个
:lfile	:lf[ile]	从文件里读取位置并跳转到第一个
:lfirst	:lfir[st]	转到指定的位置上，缺省第一个
:lgetbuffer	:lgetb[uffer]	从缓冲区里读取位置
:lgetexpr	:lgete[xpr]	从表达式里读取位置
:lgetfile	:lg[etfile]	从文件里读取位置
:lgrep	:lgr[ep]	运行 'grepprg' 并跳转到第一个匹配
:lgrepadd	:lgrepa[dd]	类似于 :grep，但附加到当前列表上
:lhelpgrep	:lh[elpgrep]	类似于 ":helpgrep"，但使用位置列表
:lhistory	:lhi[story]	列出所有位置列表
:ll	:ll	转到指定位置
:llast	:lla[st]	转到指定位置，缺省最后一个
:llist	:lli[st]	列出所有的位置
:lmake	:lmak[e]	执行外部命令 'makeprg' 并分析错误信息
:lmap	:lm[ap]	类似于 ":map!"，但包括 Lang-Arg 模式
:lmapclear	:lmapc[lear]	类似于 ":mapclear!"，但包括 Lang-Arg 模式
:lnext	:lne[xt]	跳到下一个位置
:lnewer	:lnew[er]	跳到较新的位置列表
:lnfile	:lnf[ile]	跳到下一个文件的第一个位置
:lnoremap	:ln[oremap]	类似于 ":noremap!"，但包括 Lang-Arg 模式
:loadkeymap	:loadk[eymap]	载入后面的键盘映射表，直到 EOF 为止
:loadview	:lo[adview]	从文件里为当前窗口载入视图
:lockmarks	:loc[kmarks]	跟随的命令保持位置标记不变
:lockvar	:lockv[ar]	给变量加锁
:lolder	:lol[der]	转到较旧的位置列表
:lopen	:lope[n]	打开位置窗口
:lprevious	:lp[revious]	转到前一个位置
:lpfile	:lpf[ile]	转到上一个文件的最近的位置
:lrewind	:lr[ewind]	转到指定的位置，缺省第一个
:ls	:ls	列出所有的缓冲区
:ltag	:lt[ag]	跳转到标签，并把匹配的标签加到位置列表上
:lunmap	:lu[nmap]	类似于 ":unmap!"，但包括 Lang-Arg 模式
:lua	:lua	执行 Lua 命令
:luado	:luad[o]	对每行都执行 Lua 命令
:luafile	:luaf[ile]	执行 Lua 脚本文件
:lvimgrep	:lv[imgrep]	在若干文件里搜索模式
:lvimgrepadd	:lvimgrepa[dd]	类似于 :vimgrep，但附加到当前列表
:lwindow	:lw[indow]	打开或关闭位置窗口
:move	:m[ove]	移动行
:mark	:ma[rk]	设置位置标记
:make	:mak[e]	执行外部命令 'makeprg' 并分析其错误信息
:map	:map	显示或者输入映射
:mapclear	:mapc[lear]	清除所有普通和可视模式的映射
:marks	:marks	列出所有的位置标记
:match	:mat[ch]	定义要高亮显示的匹配的模式
:menu	:me[nu]	输入新的菜单项

:menutranslate	:menut[ranslate]	增加菜单翻译项目
:messages	:mes[sages]	显示以前显示的消息
:mkexrc	:mk[exrc]	写入当前映射和设置到文件
:mksession	:mks[ession]	写入会话信息到文件
:mkspell	:mksp[ell]	生成 .spl 拼写文件
:mkvimrc	:mkv[imrc]	写入当前映射和设置到文件
:mkview	:mkvie[w]	写入当前窗口的视图到文件
:mode	:mod[e]	显示或者改变屏幕模式
:mzscheme	:mz[scheme]	执行 MzScheme 命令
:mzfile	:mzf[ile]	执行 MzScheme 脚本文件
:nbclose	:nbc[lose]	关闭当前 Netbeans 会话
:nbkey	:nb[key]	传递一个键给 Netbeans
:nbstart	:nbs[art]	打开一个新的 Netbeans 会话
:next	:n[ext]	跳转到参数列表的下一个文件
:new	:new	创建一个新的空白窗口
:nmap	:nm[ap]	类似于 ":map"，但使用普通模式
:nmapclear	:nmapc[lear]	清除普通模式的所有映射
:nmenu	:nme[nu]	为普通模式增加菜单
:nnoremap	:nn[oremap]	类似于 ":noremap"，但使用普通模式
:nnoremenu	:nnoreme[nu]	类似于 ":noremenu"，但使用普通模式
:noautocmd	:noa[utocmd]	跟随的若干命令不激活自动命令
:noremap	:no[remap]	输入不会被重新映射的映射
:nohlsearch	:noh[lsearch]	暂停 'hlsearch' 高亮
:noreabbrev	:norea[bbrev]	输入不会被重新映射的缩写
:noremenu	:noreme[nu]	输入不会被重新映射的菜单
:normal	:norm[al]	执行普通模式的命令
:noswapfile	:nos[wapfile]	跟随的若干命令不创建交换文件
:number	:nu[mber]	显示文本行时给出行号
:nunmap	:nun[map]	类似于 ":unmap"，但使用普通模式
:nunmenu	:nunme[nu]	删除普通模式的菜单
:oldfiles	:ol[dfiles]	列出 viminfo 文件中包含位置标记的文件
:open	:o[pen]	启动打开模式（没有实现）
:omap	:om[ap]	类似于 ":map"，但使用操作符等待模式
:omapclear	:omapc[lear]	删除操作符等待模式的所有映射
:omenu	:ome[nu]	为操作符等待模式增加菜单
:only	:on[ly]	关闭当前窗口，除了当前的窗口以外
:onoremap	:ono[remap]	类似于 ":noremap"，但使用操作符等待模式
:onoremenu	:onoreme[nu]	类似于 ":noremenu"，但使用操作符等待模式
:options	:opt[ions]	打开选项窗口
:ounmap	:ou[nmap]	类似于 ":unmap"，但使用操作符等待模式
:ounmenu	:ounme[nu]	删除操作符等待模式的菜单
:ownsyntax	:ow[nsyntax]	为本窗口设置新的局部语法高亮
:packadd	:pa[ckadd]	从 'packpath' 载入插件
:packloadall	:packl[oadall]	从 'packpath' 载入所有包
:pclose	:pc[lose]	关闭预览窗口
:pedit	:ped[it]	在预览窗口编辑文件
:perl	:pe[rl]	执行 Perl 命令
:print	:p[rint]	显示行
:profdel	:profd[el]	停止剖视 (profile) 函数或脚本
:profile	:prof[ile]	剖视 (profile) 函数或脚本
:promptfind	:pro[mptfind]	打开 GUI 对话框进行搜索
:promptrepl	:promptr[epl]	打开 GUI 对话框进行搜索/替代
:perldo	:perld[o]	对每行都执行 Perl 命令
:pop	:po[p]	跳转到标签栈较旧的项目上

:popup	:popu[p]	根据名字弹出菜单
:ppop	:pp[op]	在预览窗口 ":pop"
:preserve	:pre[serve]	写入所有文本到交换文件
:previous	:prev[ious]	跳转到参数列表里的上一个文件
:psearch	:ps[earch]	类似于 ":ijump"，但在预览窗口显示匹配
:ptag	:pt[ag]	在预览窗口显示标签
:ptNext	:ptN[ext]	在预览窗口 :tNext
:ptfirst	:ptf[irst]	在预览窗口 :trewind
:ptjump	:ptj[ump]	在预览窗口 :tjump 和显示标签
:ptlast	:ptl[ast]	在预览窗口 :tlast
:ptnext	:ptn[ext]	在预览窗口 :tnext
:ptprevious	:ptp[revious]	在预览窗口 :tprevious
:ptrewind	:ptr[ewind]	在预览窗口 :trewind
:ptselect	:pts[elect]	在预览窗口 :tselect 和显示标签
:put	:pu[t]	插入寄存器的内容到文本
:pwd	:pw[d]	显示当前目录
:py3	:py3	执行 Python 3 命令
:python3	:python3	同 :py3
:py3do	:py3d[o]	对每行都执行 Python 3 命令
:py3file	:py3f[ile]	执行 Python 3 脚本文件
:python	:py[thon]	执行 Python 命令
:pydo	:pyd[o]	对每行都执行 Python 命令
:pyfile	:pyf[ile]	执行 Python 脚本文件
:pyx	:pyx	执行 python_x 命令
:pythonx	:pythonx	等同于 :pyx
:pyxdo	:pyxd[o]	对每行都执行 python_x 命令
:pyxfile	:pyxf[ile]	执行 python_x 脚本文件
:quit	:q[uit]	退出当前窗口（如果只有一窗口，则退出 Vim）
:quitall	:quita[ll]	退出 Vim
:qall	:qa[ll]	退出 Vim
:read	:r[ead]	读入文件到文本
:recover	:rec[over]	从交换文件里恢复文件
:redo	:red[o]	重做一次撤销的改变
:redir	:redi[r]	重定向消息到文件或者寄存器
:redraw	:redr[aw]	强迫显示的重画
:redrawstatus	:redraws[tatus]	强迫状态行的重画
:redrawtabline	:redrawt[abline]	强迫标签行的重画
:registers	:reg[isters]	显示寄存器的内容
:resize	:res[ize]	改变当前窗口的高度
:retab	:ret[ab]	改变制表大小
:return	:retu[rn]	从用户函数返回
:rewind	:rew[ind]	转到参数列表的第一个文件
:right	:ri[ght]	右对齐文本
:rightbelow	:rightb[elow]	使得分割窗口出现在右侧或者下方
:ruby	:rub[y]	执行 Ruby 命令
:rubydo	:rubyd[o]	对每行执行 Ruby 命令
:rubyfile	:rubyf[ile]	执行 Ruby 脚本文件
:rundo	:rund[o]	从文件里读入撤销信息
:runtime	:ru[ntime]	执行 'runtimepath' 里的 vim 脚本
:rviminfo	:rv[iminfo]	读取 viminfo 文件
:substitute	:s[ubstitute]	查找和替代文本
:sNext	:sN[ext]	分割窗口并转到参数列表的前一个文件
:sandbox	:san[dbox]	在沙盘里执行命令
:sargument	:sa[rgument]	分割窗口并转到参数列表的某一个文件

:sall	:sal[l]	为参数列表的每个文件打开窗口
:saveas	:sav[eas]	用别的名字保存文件。
:sbuffer	:sb[uffer]	分割窗口并转到缓冲区列表的某一个文件
:sbNext	:sbN[ext]	分割窗口并转到缓冲区列表的前一个文件
:sball	:sba[ll]	为缓冲区列表的每个文件打开窗口
:sbfirst	:sbf[irst]	分割窗口并转到缓冲区列表的第一个文件
:sblast	:sbl[ast]	分割窗口并转到缓冲区列表的最后一个文件
:sbmodified	:sbm[odified]	分割窗口并转到缓冲区列表的修改过的文件
:sbnext	:sbn[ext]	分割窗口并转到缓冲区列表的后一个文件
:sbprevious	:sbp[revious]	分割窗口并转到缓冲区列表的前一个文件
:sbrewind	:sbr[ewind]	分割窗口并转到缓冲区列表的第一个文件
:scriptnames	:scr[iptnames]	列出所有执行过的 Vim 脚本的名字
:scriptencoding	:scripte[ncoding]	执行过的 Vim 脚本使用的编码
:scriptversion	:scriptv[ersion]	Vim 脚本使用的版本
:scscope	:scs[cope]	分割窗口并执行 cscope 命令
:set	:se[t]	显示或设置选项
:setfiletype	:setf[iletype]	设置 'filetype'，除非已经设置过
:setglobal	:setg[lobal]	显示或者设置全局选项值
:setlocal	:setl[ocal]	显示或者设置本地选项值
:sfind	:sf[ind]	分割当前窗口并编辑 'path' 的文件
:sfirst	:sfir[st]	分割窗口并转到参数列表的第一个文件
:shell	:sh[ell]	退到外壳程序
:simalt	:sim[alt]	Win32 GUI: 模拟 Windows ALT 键
:sign	:sig[n]	操作标号
:silent	:sil[ent]	安静地运行命令
:sleep	:sl[eep]	几秒内不作任何事
:slast	:sla[st]	分割窗口并转到参数列表的最后一个文件
:smagic	:sm[agic]	带 'magic' 的 :substitute
:smap	:smap	类似于 ":map"，但用于选择模式
:smapclear	:smapc[lear]	删除选择模式所有的映射
:smenu	:sme[nu]	为选择模式增加菜单
:smile	:smi[le]	让用户高兴一下
:snext	:sn[ext]	分割窗口并转到参数列表的下一个文件
:snomagic	:sno[magic]	带 'nomagic' 的 :substitute
:snoremap	:snor[emap]	类似于 ":noremap"，但用于选择模式
:snoremenu	:snoreme[nu]	类似于 ":noremenu"，但用于选择模式
:sort	:sor[t]	给行排序
:source	:so[urce]	从文件里读取 Vim 或 Ex 命令
:spelldump	:spelld[ump]	分割窗口并用正确的单词填充
:spellgood	:spe[llgood]	增加拼写的好词
:spellinfo	:spelli[nfo]	显示载入的拼写文件的信息
:spellrare	:spellra[re]	增加拼写的罕见词
:spellrepall	:spellr[epall]	像上次 z= 那样，但替换所有的坏词
:spellundo	:spellu[ndo]	删除好词或坏词
:spellwrong	:spellw[rong]	增加拼写错误
:split	:sp[lit]	分割当前窗口
:sprevious	:spr[evious]	分割窗口并转到参数列表的前一个文件
:srewind	:sre[wind]	分割窗口并转到参数列表的第一个文件
:stop	:st[op]	暂停编辑器并退回外壳
:stag	:sta[g]	分割窗口并跳转到标签上
:startgreplace	:startg[replace]	开始虚拟替换模式
:startreplace	:startr[eplace]	开始替换模式
:startinsert	:star[tinsert]	开始插入模式
:stopinsert	:stopi[nsert]	退出插入模式

:stjump	:stj[ump]	执行 ":tjump" 并分割窗口
:stselect	:sts[elect]	执行 ":tselect" 并分割窗口
:sunhide	:sun[hide]	等同于 ":unhide"
:sunmap	:sunm[ap]	类似于 ":unmap"，但用于选择模式
:sunmenu	:sunme[nu]	删除选择模式的菜单
:suspend	:sus[pend]	等同于 ":stop"
:sview	:sv[iew]	分割窗口并以只读模式编辑文件
:swapname	:sw[apname]	显示当前交换文件的名字
:syntax	:sy[ntax]	语法高亮
:syntime	:synti[me]	测量语法高亮速度
:syncbind	:sync[bind]	同步滚动绑定
:t	:t	等同于 ":copy"
:tNext	:tN[ext]	跳转到上一个匹配的标签
:tabNext	:tabN[ext]	跳到上一个标签页
:tabclose	:tabc[lose]	关闭当前标签页
:tabdo	:tabdo	为每个标签页执行命令
:tabedit	:tabe[dit]	在新标签页里编辑文件
:tabfind	:tabf[ind]	寻找 'path' 里的文件，在新标签页里编辑之
:tabfirst	:tabfir[st]	转到第一个标签页
:tablast	:tabl[ast]	转到最后一个标签页
:tabmove	:tabm[ove]	把标签页移到别的位置
:tabnew	:tabnew	在新标签页里编辑文件
:tabnext	:tabn[ext]	转到下一个标签页
:tabonly	:tabo[nly]	关闭所有除了当前标签页以外的所有标签页
:tabprevious	:tabp[revious]	转到前一个标签页
:tabrewind	:tabr[ewind]	转到第一个标签页
:tabs	:tabs	列出标签页以及里面包含的内容
:tab	:tab	打开新窗口时建立新的标签页
:tag	:ta[g]	跳转到标签上
:tags	:tags	显示标签栈的内容
:tcd	:tcd	切换标签页的目录
:tchdir	:tch[dir]	切换标签页的目录
:tcl	:tc[l]	执行 Tcl 命令
:tcldo	:tcld[o]	为每行执行 Tcl 命令
:tclfile	:tclf[ile]	执行 Tcl 脚本文件
:tearoff	:te[aroff]	撕下一个菜单
:terminal	:ter[minal]	打开终端窗口
:tfirst	:tf[irst]	转到第一个匹配的标签
:throw	:th[row]	抛出例外
:tjump	:tj[ump]	类似于 ":tselect"，只有一个匹配时直接跳转之
:tlast	:tl[ast]	跳转到最后一个匹配的标签
:tlmenu	:tlm[enu]	为终端作业模式增加菜单
:tlnoremenu	:tln[oremenu]	类似于 ":noremenu"，但用于终端作业模式
:tlunmenu	:tlu[nmenu]	删除终端作业模式的菜单
:tmapclear	:tmapc[lear]	删除所有终端作业模式的映射
:tmap	:tma[p]	类似于 ":map" 但用于终端作业模式
:tmenu	:tm[enu]	定义菜单的工具提示
:tnext	:tn[ext]	跳转到下一个匹配的标签
:tnoremap	:tno[remap]	类似于 ":noremap" 但用于终端作业模式
:topleft	:to[pleft]	使得分割的窗口出现在顶部或者最左端
:tprevious	:tp[revious]	跳转到上一个匹配的标签
:trewind	:tr[ewind]	跳转到第一个匹配的标签
:try	:try	执行命令，错误或者例外时终止
:tselect	:ts[elect]	列出匹配的标签并从中选择一个

:tunmap	:tunma[p]	类似于 ":unmap" 但用于终端作业模式
:tunmenu	:tu[nmenu]	删除菜单的工具提示
:undo	:u[ndo]	撤销最近的改变
:undojoin	:undoj[oin]	将下一个改变和上一个撤销块合并
:undolist	:undol[ist]	列出撤销树的叶结点
:unabbreviate	:una[bbreviate]	删除缩写
:unhide	:unh[ide]	为缓冲区列表里每个载入的文件打开窗口
:unlet	:unl[et]	删除变量
:unlockvar	:unlo[ckvar]	给变量解锁
:unmap	:unm[ap]	删除映射
:unmenu	:unme[nu]	删除菜单
:unsilent	:uns[ilent]	非安静模式执行命令
:update	:up[date]	如果有修改，写回缓冲区
:vglobal	:v[global]	对不匹配的行，执行命令
:version	:ve[rsion]	显示版本号和其它信息
:verbose	:verb[ose]	执行命令，过程中设置 'verbose'
:vertical	:vert[ical]	使得跟随的命令垂直分割
:vimgrep	:vim[grep]	在若干文件里查找模式
:vimgrepadd	:vimgrepa[dd]	类似于 :vimgrep，但附加到当前列表
:visual	:vi[sual]	等同于 ":edit"，但关掉 "Ex" 模式
:viusage	:viu[sage]	普通模式命令的总览
:view	:vie[w]	以只读方式编辑文件
:vmap	:vm[ap]	类似于 ":map"，但使用可视+选择模式
:vmapclear	:vmapc[lear]	删除可视+选择模式所有的映射
:vmenu	:vme[nu]	为可视+选择模式增加菜单
:vnew	:vne[w]	建立新的空白窗口，垂直分割
:vnoremap	:vn[oremap]	类似于 ":noremap"，但使用可视+选择模式
:vnoremenu	:vnoreme[nu]	类似于 ":noremenu"，但使用可视+选择模式
:vsplit	:vs[plit]	垂直分割当前窗口
:vunmap	:vu[nmap]	类似于 ":unmap"，但使用可视+选择模式
:vunmenu	:vunme[nu]	删除可视+选择模式的菜单
:windo	:windo	为每个窗口执行命令
:write	:w[rite]	写回文件
:wNext	:wN[ext]	写回文件，然后跳转到参数列表的前一个文件
:wall	:wa[ll]	写回所有（改变的）缓冲区
:while	:wh[ile]	只要条件满足，执行循环体
:winsize	:wi[nsize]	取得或者设置窗口大小（已废弃）
:wincmd	:winc[md]	执行窗口（CTRL-W）命令
:winpos	:winp[os]	取得或者设置窗口位置
:wnext	:wn[ext]	写回文件，然后跳转到参数列表的下一个文件
:wprevious	:wp[revious]	写回文件，然后跳转到参数列表的前一个文件
:wq	:wq	写回文件，然后退出窗口或者 Vim
:wqall	:wqa[ll]	写回所有（改变的）缓冲区然后退出 Vim
:wundo	:wu[ndo]	把撤销信息写入文件
:wviminfo	:wv[iminfo]	写入 viminfo 文件
:xit	:x[it]	如果缓冲区被改动，写入之。然后退出窗口或者 Vim
:xall	:xa[ll]	等同于 ":wqall"
:xmapclear	:xmapc[lear]	删除可视模式的所有映射
:xmap	:xm[ap]	类似于 ":map"，但使用可视模式
:xmenu	:xme[nu]	增加可视模式的菜单
:xrestore	:xr[estore]	恢复 X 服务器的连接
:xnoremap	:xn[oremap]	类似于 ":noremap"，但使用可视模式
:xnoremenu	:xnoreme[nu]	类似于 ":noremenu"，但使用可视模式
:xunmap	:xu[nmap]	类似于 ":unmap"，但使用可视模式

:xunmenu	:xunme[nu]	删除可视模式的菜单
:yank	:y[ank]	抽出行到寄存器
:z	:z	显示一些行
:~	:~	重复上次 ":substitute"

vim:tw=78:ts=8:noet:ft=help:norl:

howto.txt For Vim version 8.2. 最近更新：2006年5月

VIM 参考手册 by Bram Moolenaar
译者：iCrazy <icrazy@ustc.edu> http://vimcdoc.sf.net

如何…… howdoi how-do-i howto how-to

tutor	入门
:quit	退出？我被困住了，请帮帮我！
initialization	初始化 Vim
vimrc-intro	写一个 Vim 脚本文件 (vimrc)
suspend	挂起 Vim
usr_11.txt	在一次崩溃后恢复
07.4	在覆盖我的文件时保留一个备份
usr_07.txt	编辑文件
23.4	编辑二进制文件
usr_24.txt	插入文本
deleting	删除文本
usr_04.txt	改变文本
04.5	复制和移动文本
usr_25.txt	给文本排版
30.6	给注释排版
30.2	缩进 C 程序
25.3	自动设置缩进
usr_26.txt	重复执行命令
02.5	撤销和重做
usr_03.txt	在文本中移动
word-motions	基于单词的移动
left-right-motions	左右移动
up-down-motions	上下移动
object-motions	基于文本对象的移动
various-motions	其它移动方式
object-select	基于文本对象的选择
'whichwrap'	移动时跳过换行符
'virtualedit'	移动到没有文本的地方
usr_27.txt	指定查找的模式
tags-and-searches	进行标签和特殊查找
29.4	在包含文件中查找变量、函数或宏
K	查找光标处关键词的手册页
03.7	滚屏
'sidescroll'	水平/竖直方向滚屏
'scrolloff'	设置可见的上下文行数
mode-switching	切换模式
04.4	使用可视模式
'insertmode'	以插入模式启动 Vim
40.1	映射热键

24.7 创建缩写

ins-expandtab 在插入模式下用空格扩展制表符
i_CTRL-R 在插入模式下插入寄存器的内容
24.3 在插入模式下自动补全单词
25.1 断开过长的行

20.1 编辑命令行
20.3 命令行自动补全
'cmdheight' 增加命令行的高度
10.3 指定命令行范围
40.3 指定在 读/写 或 进入/离开 缓冲区/窗口 之前/之后
自动执行命令

'autowrite' 自动存盘
30.1 加速 编辑-编译-编辑 循环，在 Vim 里面编译和修正错误

options 设置选项
auto-setting 自动设置选项
term-dependent-settings 设置依赖于终端名称的选项
save-settings 保存设置
:quote 对我的 .vim 文件进行注释
'helpheight' 调整默认的帮助高度
'highlight' 设置各种高亮模式
'title' 设置窗口标题
'icon' 设置窗口图标标题
'report' 避免看到每一行上的改变信息
'shortmess' 避免按回车 hit-enter 提示

mouse-using 在 Vim 中使用鼠标
usr_08.txt 管理多窗口和缓冲区
gui.txt 使用 GUI

|您（还）不能！| 用 Vim 洗盘子 :)

usr_06.txt 打开语法高亮
2html.vim 把一个彩色的文件转换成 HTML
less 用 Vim 模拟 less 或 more，但带有语法高亮

vim:tw=78:ts=8:noet:ft=help:norl:

tips.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar
译者：iCrazy <icrazy@ustc.edu>, tocer http://vimcdoc.sf.net

使用 Vim 的技巧　　tips

这里只是一小部分我们认为会对很多用户有用的技巧。更多的技巧都在 wiki 上。URL 可以从这里找到：
http://www.vim.org

别忘记浏览用户手册，里面也有很多实用的技巧 usr_toc.txt 。

编辑 C 程序　　C-editing
查找使用标识符的地方　　ident-search
在 xterm 中切换屏幕　　xterm-screens
在插入模式下滚屏　　scroll-insert
平滑的滚屏　　scroll-smooth
纠正普通的录入错误　　type-mistakes
统计单词数、行数等　　count-items
恢复光标位置　　restore-position
文件更名　　rename-files
更改多个文件中的一个名字　　change-name
加速外部命令的执行　　speed-up
一些有用的映射　　useful-mappings
压缩帮助文件　　gzip-helpfile
在一个窗口中执行外壳命令　　shell-window
十六进制编辑　　hex-editing
在自动命令中使用 <> 记法　　autocmd-<>
高亮匹配括号　　match-parens

编辑 C 程序　　C-editing

Vim 里面有不少功能可以帮助你们编辑 C 程序。以下是一个概括，你们可以使用标签跳转到具体的内容中去：

usr_29.txt	用户手册中关于在程序的不同部分间移动的内容。
usr_30.txt	用户手册中关于编辑程序的内容。
C-indenting	输入时自动设置每行的缩进。
=	重新缩进一些行。
format-comments	对注释进行排版。
:checkpath	显示所有被（嵌套）包含的文件。
[i	在当前和被包含的文件中查找光标所在的的标识符。
[_CTRL-I	跳到 "[i" 的匹配。
[I	列出在当前和被包含的文件中匹配光标所在的标识符的行。
[d	在当前和被包含的文件中查找光标所在的标识符的宏定义。
CTRL-]	跳到光标所在的标签处（例如：函数的定义）。
CTRL-T	跳回执行 CTRL-] 命令前的地方。
:tselect	从一连串匹配的标签中选出一个。

gd	跳到光标所在的局部变量的声明处。
gD	跳到光标所在的全局变量的声明处。
gf	跳到光标所在的文件名表示的文件。
%	跳到匹配的 ()、{}、[]、/* */、#if、#else、#endif 处。
[/	跳到上一个注释开始的位置。
]/	跳到下一个注释结束的位置。
[#	反向到未闭合的 #if、#ifdef 或 #else 处。
]#	正向到未闭合的 #else 或 #endif 处。
[(	反向到未闭合的 '(' 处。
])	正向到未闭合的 ')' 处。
[{	反向到未闭合的 '{' 处。
]}	正向到未闭合的 '}' 处。
v_ab	选中 "一个块" ("a block"), 从 "[(" 至 "])", 含括号
v_ib	选中 "内含块" ("inner block"), 从 "[(" 至 "])"
v_aB	选中 "一个大块" ("a Block"), 从 "[{" 至 "]}", 含括号
v_iB	选中 "内含大块" ("inner Block"), 从 "[{" 至 "]}"

查找使用标识符的地方 ident-search

你应该已经知道 tags 可以被用来跳转到定义函数和变量的地方。但是有时你希望跳转到使用函数和变量的地方。可以用以下两种方法实现:
1. 使用 :grep 命令。这个应该可以在大多数 Unix 系统上工作, 但是速度会比较慢(因为它读取所有的文件), 并且只能在一个目录中搜索。
2. 使用 ID 工具集。这个速度比较快而且可以搜索多个目录。它使用一个数据库来存放定位信息。你需要一些额外的程序来使它得以工作, 并且你必须使数据库不断保持更新。

使用 GNU id 工具集 (id-tools):

你所需要的:
- 安装 GNU id 工具集 (mkid 是用来创建 ID 的, lid 是用来运行宏的)
- 一个在当前目录下名为 "ID" 的标识符数据库。你可以用外壳命令 "mkid file1 file2 .." 来创建它。

把这些添加写你的 .vimrc 文件中:

```
map _u :call ID_search()<Bar>execute "/\\<" . g:word . "\\>"<CR>
map _n :n<Bar>execute "/\\<" . g:word . "\\>"<CR>

function! ID_search()
  let g:word = expand("<cword>")
  let x = system("lid --key=none ". g:word)
  let x = substitute(x, "\n", " ", "g")
  execute "next " . x
endfun
```

使用的时候, 把光标放在一个单词上, 敲入 "_u", 然后 vim 会读入含有这个单词的文件。使用 "n" 查找这个单词在相同文件中下一次的出现的地方。使用 "_n" 可以跳转到下一个文件。

这些操作已经使用 id-utils-3.2 (这是位于距你最近的 gnu ftp 镜像服务器上的档案

名称）测试通过了。

[这个的想法来自于 Andreas Kutschera]

在 xterm 中切换屏幕

xterm-screens xterm-save-screen

(来源：comp.editors，作者：Juergen Weigert，回答一个问题的时候)

```
:> 另一个问题就是退出 vim 后，屏幕内容就被留在那儿了，也就是说：我刚刚正在查看
:> (或编辑) 的内容就被留在屏幕上了。而我此前执行的命令 (例如："ls") 的输出却不
:> 见了，换而言之在滚屏缓冲区里不存在了。我知道有个办法可以在退出 vim 或其他仿
:> vi 的编辑器的时候恢复屏幕内容，但是我不知道该如何操作。请帮助我，谢谢！
:
: 我认为可能有人可以回答这个问题。我认为 vim 和 vi 在某个特定的 xterm 安装下和
: 别的程序一样工作。
```

他们并不一定完全相同，因为这牵扯到一个 termcap 对 terminfo 的问题。你们应该知道针对一种特定的终端，有两种数据库可以用来描述它的属性：termcap 和 terminfo。当它们中的条目不同，而且以上问题中的一个程序使用 terminfo，另一个使用 termcap 时，两个程序会出现差异 (请参见： +terminfo)。

在你的问题中，你可能需要以下的控制序列：^[[?47h 和 ^[[?47l。他们用来在 xterm 备用和主屏幕缓冲区中切换。一个快速的临时解决方案使用如下的命令序列

```
echo -n "^[[?47h"; vim ... ; echo -n "^[[?47l"
```

这可能就是你所需要的。(我用符号 ^[表示 ESC 字符，往后你还会看到数据库使用 \E 来表示它)。

在启动的时候，vim 把 termcap 中变量 ti (terminfo:smcup) 的值回显在终端上。退出的时候，它回显 te (terminfo: rmcup) 的值。这样一来，这两个变量正好处于以上所述的控制序列应该执行的位置。

把你的 xterm termcap 条目 (在 /etc/termcap 中) 和 xterm terminfo 条目 (用 "infocmp -C xterm" 得到) 比较一下。两者应该都会有与下面类似的条目：

```
:te=\E[2J\E[?47l\E8:ti=\E7\E[?47h:
```

附：如果你发现了任何差异，那么最好让某人 (或许是你的系统管理员) 彻底地检查一下 termcap 和 terminfo 的一致性。

备注 1：如果你在 feature.h 中定义了 FEAT_XTERM_SAVE 之后又重新编译了 Vim，那么内置的 xterm 会有上述的 "te" 和 "ti" 条目。

备注 2：如果你希望禁止屏幕切换，并且不希望改变你的 termcap，你可以在 .vimrc 文件中加入这一行：

```
:set t_ti= t_te=
```

在插入模式下滚屏

scroll-insert

如果你处于插入模式下并且希望看一些恰好在屏幕范围以外的东西，你可以使用 CTRL-X CTRL-E 和 CTRL-X CTRL-Y 来滚屏。

i_CTRL-X_CTRL-E

为了使这个简单些，你可以用这些映射：

```
:inoremap <C-E> <C-X><C-E>
:inoremap <C-Y> <C-X><C-Y>
```

(逐字敲入这些字符，并确认 '<' 标志位不在 'cpoptions' 选项中)。
不过这样你就不能使用从光标上一行/下一行拷贝文字的功能了 i_CTRL-E 。

你还可以考虑把 'scrolloff' 设置得大一些，这样你就总能看到光标附近的上下文了。
如果 'scrolloff' 的值大于窗口高度的一半，在向上或向下移动光标时，文字会上下卷动，但是光标会始终停留在屏幕中间的位置。

平滑的滚屏 scroll-smooth

如果你希望你的滚屏更加平滑一些，你可以使用以下的映射：

```
:map <C-U> <C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y><C-Y>
:map <C-D> <C-E><C-E><C-E><C-E><C-E><C-E><C-E><C-E><C-E><C-E><C-E><C-E>
```

(逐字敲入这些字符，并确认 '<' 标志位不在 'cpoptions' 选项中)。

纠正普通的录入错误 type-mistakes

如果有一些单词你总是敲错，你可以使用缩写来改正它们。例如：

```
:ab teh the
:ab fro for
```

统计单词数、行数等 count-items

如果需要统计缓冲区中一个模式 (pattern) 出现的频率，使用 substitute 命令并加入 'n' 标志位以避免实际的替代。Vim 报告的已替代的数目就是模式出现的次数。例：

```
:%s/./&/gn              字符数
:%s/\i\+/&/gn           单词数
:%s/^//n                行数
:%s/the/&/gn            任何地方出现的 "the"
:%s/\<the\>/&/gn         作为单词出现的 "the"
```

你可能希望复位 'hlsearch' 或者使用 ":nohlsearch"。
如果你希望没有匹配的时候不显示错误，那么增加标志位 'e'。

另一种办法就是在可视模式下使用 v_g_CTRL-G 。

如果你想在多个文件中查找匹配，用 :vimgrep 。

count-bytes

如果你想数字节数，你可以这样做：

选中那些字符 (列块选择也可以)
使用 "y" 来拷贝这些字符
使用 strlen() 函数：

```
:echo strlen(@")
```

一个换行符被当做一个字节。

恢复光标位置 restore-position

有时你希望写一个映射，让它在文件中的其他地方做一些修改然后恢复光标的位置（不滚动文本）。例如，修改一个文件中的日期标记：

```
:map <F2> msHmtgg/Last [cC]hange:\s*/e+1<CR>"_D"=strftime("%Y %b %d")<CR>p'tzt`s
```

分解出保存位置的命令：

ms	把光标位置存放在位置标记 's' 中
H	跳转到窗口的顶端
mt	把这个位置存放在位置标记 't' 中

分解出恢复位置的命令：

't	跳转到先前位于窗口顶端的那一行
zt	滚屏，使这一行位于窗口的顶端
`s	跳转到最初光标的位置

更高级的功能见 winsaveview() 和 winrestview() 。

文件更名 rename-files

假如我有一个目录，里面有如下的文件（目录是随机选取的）：

```
buffer.c
charset.c
digraph.c
...
```

现在我希望把 *.c 更名为 *.bla。我可以这样做：

```
$ vim
:r !ls *.c
:%s/\(.*\).c/mv & \1.bla
:w !sh
:q!
```

更改多个文件中的一个名字 change-name

使用脚本文件在若干文件中更改一个名字的示例：

创建文件 "subs.vim"，包含替代命令和 :update 命令：

```
:%s/Jones/Smith/g
:%s/Allen/Peter/g
:update
```

在所有你要改动的文件上执行 Vim，然后为每个参数执行脚本：

```
vim *.let
argdo source subs.vim
```

参见 :argdo 。

加速外部命令的执行 speed-up

在一些情况下，外部命令执行起来非常地慢。而且还会减慢 Unix 上的通配符扩展。这儿有一些建议可以加快速度：

如果你的 .cshrc 文件（根据你使用的外壳，文件名可能不同）非常地长，你应该把它分割成两节：需要和用户交互的、无需和用户交互的（经常被称作二级外壳）。当你在 Vim 中执行一个类似 ":!ls" 的命令时，你就不需要和用户交互的那一部分（例如：设置提示符）。把那些不必要的部分放到下面这些行后面去：

```
if ($?prompt == 0) then
        exit 0
endif
```

另一个办法是在 'shell' 选项中包含 "-f" 参数，例如：

```
:set shell=csh\ -f
```

(这儿的反斜杠是必需的，这样才能在选项中表示一个空格)。
这样就会使 csh 完全跳过 .cshrc 文件。不过这样可能会造成一些程序不能正常运行。

一些有用的映射 useful-mappings

这里有一些人们喜欢使用的映射。

map-backtick

```
:map ' `
```

使得单引号和 ' 一样工作。把光标移动到一个位置标记所在的列，而不是那一行的第一个非空白字符。

emacs-keys

要在命令行上实现 Emacs 风格的编辑操作：

```
" 至行首
:cnoremap <C-A>         <Home>
" 后退一个字符
:cnoremap <C-B>         <Left>
" 删除光标所在的字符
:cnoremap <C-D>         <Del>
" 至行尾
:cnoremap <C-E>         <End>
" 前进一个字符
:cnoremap <C-F>         <Right>
" 取回较新的命令行
:cnoremap <C-N>         <Down>
" 取回以前（较旧的）命令行
:cnoremap <C-P>         <Up>
" 后退一个单词
:cnoremap <Esc><C-B>    <S-Left>
" 前进一个单词
```

```
:cnoremap <Esc><C-F>    <S-Right>
```

备注：前提条件是 '<' 标志位不在 'cpoptions' 选项中。 <>

format-bullet-list

这个映射可以排版任何带符号的 (bullet) 列表，不过它需要在每一个条目的上下都各有一个空行。使用表达式命令，以便对映射的部分进行注释：

```
:let m =     ":map _f  :set ai<CR>"   " 需要置位 'autoindent'
:let m = m . "{O<Esc>"                " 在项目上面加入空行
:let m = m . "}{)^W"                 " 跳转到 bullet 之后的文本
:let m = m . "i     <CR>     <Esc>"   " 为缩进加空格
:let m = m . "gq}"                   " 排版 bullet 之后的文本
:let m = m . "{dd"                   " 删除空行
:let m = m . "5lDJ"                  " 把文本放到 bullet 之后
:execute m                           |" 定义这个 mapping
```

(<> 记法 <> 。注意 必须按照字面逐个输入。^W 是 "^" 和 "W"，而不是 CTRL-W。如果 '<' 标志位不在 'cpoptions' 选项中，你可以把这些拷贝/粘贴代码给 Vim 执行。)

注意 最后一个注释以 |" 开始，因为 ":execute" 不能直接识别一个注释。

你还需要把 'textwidth' 设置成一个非 0 值，例如：

```
:set tw=70
```

以下这个映射可以达到同样的效果，不过它从第一行获得列表的缩进（备注：这个映射其实只有一行，其中有很多空格)：

```
:map _f :set ai<CR>}{a                                          <Esc>WWm
```

collapse

这两个映射可以把一连串的空行 (;b) 或空白行 (;n) 压缩到一行

```
:map ;b   GoZ<Esc>:g/^$/.,/./-j<CR>Gdd
:map ;n   GoZ<Esc>:g/^[ <Tab>]*$/.,/[^ <Tab>]/-j<CR>Gdd
```

压缩帮助文件 gzip-helpfile

对于那些磁盘空间极度紧张的人来说，你们可以压缩帮助文件。这样会使得查看帮助文件时稍微慢一点，并且需要 "gzip" 这个程序的支持。

(1) 压缩所有帮助文件："gzip doc/*.txt"。

(2) 编辑文件 "doc/tags"，用 ".txt.gz" 替换 ".txt"：

```
:%s=\(\t.*\.txt\)\t=\1.gz\t=
```

(3) 把这一行加入到你的 vimrc 文件中：

```
set helpfile={dirname}/help.txt.gz
```

这儿 {dirname} 是存放帮助文件的目录。 gzip 这个插件会负责解压缩这些文件的。如果其它 Vim 文件所在的位置和存放压缩帮助的 "doc" 目录不一致，你必须肯定已经把 $VIMRUNTIME 设置成存放 Vim 文件的目录。参见： $VIMRUNTIME 。

在一个窗口中执行外壳命令 shell-window

见 terminal 。

另一个方法是用 "splitvt" 程序把你的终端屏幕或显示窗口进行分割。在一些 ftp 服务器上你可以找到这个工具。Sam Lantinga <slouken@cs.ucdavis.edu> 对此了解颇多。

另一种办法是使用在 BSD Unix 系统上出现的 "window" 命令，它支持多个重叠的窗口。或者使用最先出现在 www.uni-erlangen.de 上的 "screen" 程序，它支持窗口栈。

十六进制编辑 hex-editing using-xxd

请看用户手册的第 23.4 节。

如果你用一些专门的扩展名来命名二进制文件（诸如 exe，bin 等等），你会发现以下在 <.vimrc> 文件中使用的一些命令在自动处理这些文件时非常有用。你可以用你希望编辑的文件扩展名（用逗号分隔）替换以下的 "*.bin":

```
" vim -b : edit binary using xxd-format!
augroup Binary
  au!
  au BufReadPre  *.bin let &bin=1
  au BufReadPost *.bin if &bin | %!xxd
  au BufReadPost *.bin set ft=xxd | endif
  au BufWritePre *.bin if &bin | %!xxd -r
  au BufWritePre *.bin endif
  au BufWritePost *.bin if &bin | %!xxd
  au BufWritePost *.bin set nomod | endif
augroup END
```

在自动命令中使用 <> 记法 autocmd-<>

在自动命令的参数中，不能识别 <> 记法。为避免使用特殊的字符，你可以使用一个可以自我毁灭的映射来得到 <> 记法，然后从自动命令中调用这个映射。举例如下：

map-self-destroy

```
" 此命令自动把文件名加入到菜单列表中。
" 它使用了一个可以自我毁灭的映射！
" 1. 用缓冲区中的一行把文件名中的点（'dots'）转换成 \.
" 2. 把该行存放在寄存器 '"' 中
" 3. 把该名字加入缓冲区菜单列表
" 警 告：这会有些副作用，比如：覆盖当前的寄存器内容和删除任何 "i" 命令的映射
"
autocmd BufNewFile,BufReadPre * nmap i :nunmap i<CR>O<C-R>%<Esc>:.g/\./s/\./\\./g<CR>0"9y$u:m
autocmd BufNewFile,BufReadPre * normal i
```

另一个或许更好一些的办法就是使用 ":execute" 命令。在字符串中，你可以通过在 <> 记法前面加一个反斜杠的方法来使用它。别忘了加倍已经存在的反斜杠的数目以及在 '"' 前面放一个反斜杠。

```
autocmd BufNewFile,BufReadPre * exe "normal O\<C-R>%\<Esc>:.g/\\./s/\\./\\\\./g\<CR>0\"9y$u:me
```

为了建立一个真正的缓冲区菜单，需要用到用户函数（参见 :function），不过那里不使用 <> 记法，所以失去了在这里举例的意义。

高亮匹配括号 match-parens

本例演示一些高级技巧的用法：
- 使用 CursorMoved 自动命令事件
- 使用 searchpairpos() 查找匹配括号
- 使用 synID() 检测光标是否在字符串或注释上
- 使用 :match 高亮一些内容
- 使用 pattern 匹配文件的特定位置。

它应该被放在 Vim 脚本里，因为使用了局部于脚本的变量。它跳过字符串或注释里的匹配，除非光标起始于字符串或者注释的内部。这需要语法高亮的支持。

matchparen 插件使用稍稍改进的版本。

```
let s:paren_hl_on = 0
function s:Highlight_Matching_Paren()
  if s:paren_hl_on
    match none
    let s:paren_hl_on = 0
  endif

  let c_lnum = line('.')
  let c_col = col('.')

  let c = getline(c_lnum)[c_col - 1]
  let plist = split(&matchpairs, ':\|,')
  let i = index(plist, c)
  if i < 0
    return
  endif
  if i % 2 == 0
    let s_flags = 'nW'
    let c2 = plist[i + 1]
  else
    let s_flags = 'nbW'
    let c2 = c
    let c = plist[i - 1]
  endif
  if c == '['
    let c = '\['
    let c2 = '\]'
  endif
  let s_skip ='synIDattr(synID(line("."), col("."), 0), "name") ' .
        \ '=~?  "string\\|comment"'
  execute 'if' s_skip '| let s_skip = 0 | endif'

  let [m_lnum, m_col] = searchpairpos(c, '', c2, s_flags, s_skip)

  if m_lnum > 0 && m_lnum >= line('w0') && m_lnum <= line('w$')
```

```
      exe 'match Search /\(\%' . c_lnum . 'l\%' . c_col .
            \ 'c\)\|\(\%' . m_lnum . 'l\%' . m_col . 'c\)/'
      let s:paren_hl_on = 1
    endif
  endfunction

  autocmd CursorMoved,CursorMovedI * call s:Highlight_Matching_Paren()
  autocmd InsertEnter * match none


vim:tw=78:ts=8:noet:ft=help:norl:
```

message.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis，tocer
http://vimcdoc.sf.net

本文件包含一个 Vim 产生的消息和错误信息按字母排序的列表。如果你不理解消息的含义，可以在这里查找。不过，本列表未必完整。

1. 以前的消息 :messages
2. 错误信息 error-messages
3. 消息 messages

---

1. 以前的消息 :messages :mes message-history

:messages" 命令可以查看以前给出的消息。根据 'shortmess' 选项的设置，消息可能被覆盖或者截断。这时该命令尤其有用。

:messages	显示所有消息。
:{count}messages	显示 {count} 个最近的消息。
:messages clear	清除所有消息。
:{count}messages clear	清除消息，只保留 {count} 个最近的消息。

使用微小 (tiny) 特性集的版本，记住的消息数目固定是 20。其它版本是 200。

g<
这个命令用于察看前一个命令输出的最后一页。如果你在提示按回车的时候不小心按了空格 <Space> 键，那么该命令尤其有用。此时你将回到输入回车的提示画面，并可重新回滚。
注意：如果在提示显示更多信息时使用 "q" 停止输出，那么只显示已输出的那部分显示。当另一个命令输出结果时，会清空前一个命令的输出。
不重定向 "g<" 的输出。

如果使用翻译的消息，显示的第一行会告诉你谁维护这个消息翻译。如果你发现问题，可以联系该维护者。

如果你想要查找特定 (错误) 消息，使用消息开始的 ID 号。比如，要得到下面消息的帮助：

```
E72: Close error on swap file
```

或者 (经翻译的)：

```
E72: Errore durante chiusura swap file
```

使用：

```
:help E72
```

如果你很忙，不按 shift 键也可以：

```
:help e72
```

2．错误信息 error-messages errors

显示的错误信息，如果在你没读到之前就已经消失了，你可以这样重新看到：

```
:echo errmsg
```

或者查看最近的消息列表：

```
:messages
```

见上 :messages 。

消 息 列 表

E222 E228 E232 E256 E293 E298 E304 E317
E318 E356 E438 E439 E440 E316 E320 E322
E323 E341 E473 E570 E685 E950

```
Add to read buffer
makemap: Illegal mode
Cannot create BalloonEval with both message and callback
Hangul automata ERROR
block was not locked
Didn't get block nr {N}?
ml_upd_block0(): Didn't get block 0??
pointer block id wrong {N}
Updated too many blocks?
get_varp ERROR
u_undo: line numbers wrong
undo list corrupt
undo line missing
ml_get: cannot find line {N}
cannot find line {N}
line number out of range: {N} past the end
line count wrong in block {N}
Internal error
Internal error: {function}
fatal error in cs_manage_matches
Invalid count for del_bytes(): {N}
```

内部错误。如果你能够使之重现，请发送漏洞报告。 bugs

```
ATTENTION
Found a swap file by the name ...
```

见 ATTENTION 。

E92

```
Buffer {N} not found
```

你请求的缓冲区不存在。在你清除了一个缓冲区，而它包含一个位置标记或者有其它途径还被引用的时候，也会出现这个错误。 :bwipeout

E95

Buffer with this name already exists

你不能有两个同名的缓冲区。

E72

Close error on swap file

用来保存编辑文本的备份的 swap-file 不能被正确关闭。大多数情况下这不会有什么问题。

E169

Command too recursive

当 Ex 命令执行了一个 Ex 命令，后者又执行一个 Ex 命令，如此反复的时候，就可能发生。最多能有 200 层或 'maxfuncdepth' 的值的较大者。如果超过，很有可能是无穷循环。可能涉及的命令是 :execute 或 :source 。

E254

Cannot allocate color {name}

颜色名 {name} 未知。多数系统中可用的颜色列表可以参见 gui-colors 。

E458

Cannot allocate colormap entry, some colors may be incorrect

这意味着 Vim 没有足够的颜色可以分配。虽然 Vim 还能运行，但是有的颜色可能不会以指定的颜色出现。试试停止其它使用很多颜色的应用程序，或者在启动 gvim 之后启动它们。已知很多浏览器会消耗很多颜色。要避免这个问题，你可以告诉 netscape 使用自己的颜色映射表:

```
netscape -install
```

或者告诉它只使用有限种的颜色 (64 应该足够了):

```
netscape -ncols 64
```

这也可以通过 Xdefaults 文件设置:

```
Netscape*installColormap: Yes
```

或者

```
Netscape*maxImageColors:  64
```

E79

Cannot expand wildcards

文件名包含特殊的字符组合，Vim 因而试图进行通配符扩展但不成功。这 不 意味着没有匹配的文件名，只是说该模式是非法的。

E459

Cannot go back to previous directory

在扩展文件名的时候，Vim 不能回到最近用过的目录名。所有正在使用的文件名现在都可能成为非法！你需要得到当前目录的可执行 (遍历) 权限。

E190 E212

Cannot open "{filename}" for writing
Can't open file for writing

由于某种原因，你要写入的文件不能建立或者覆盖。这可能是因为你没有目录的写权限，或者该文件名不合法。

E166

Can't open linked file for writing

你试图写入文件，但该文件无法被覆盖，且该文件是链接类型（硬链接或者符号链接）。其实，包含该链接或文件的目录可写的时候并非不可写入。只不过 Vim 不知道你是要删除该链接并在同样的位置写入实际文件呢，还是要删除链接到的文件并用新文件覆盖之。如果你确实需要用这个名字写入文件，请手动删除这个链接或者文件，或者改变权限，使得 Vim 可以对它进行覆盖。

E46

Cannot change read-only variable "{name}"

你试图给一个函数参数 a:var 或者 Vim 的内部只读变量 v:var 赋值。

E90

Cannot unload last buffer

Vim 要求必须有一个载入的缓冲区，不然窗口中就没有东西可以显示了。

E40

Can't open errorfile <filename>

使用 ":make" 或者 ":grep" 命令时：不能打开用于保存错误信息或者 grep 的输出结果的文件。可能原因如下：
- 'shellredir' 的值不正确。
- 外壳改变了目录，使得错误文件写到别的目录里。这可以通过改变 'makeef' 修正，但是 make 命令还是在错误的目录下执行。
- 'makeef' 的值不正确。
- 'grepprg' 或者 'makeprg' 不能执行。这一点不一定总能检测到（尤其在 MS-Windows 上）。检查你的 $PATH。

Can't open file C:\TEMP\VIoD243.TMP

MS-Windows 上，期待读入外部命令输出的结果但是命令没有正确执行时会显示此信息。这可能有多种原因。检查 'shell'、'shellquote'、'shellxquote'、 'shellslash' 和相关的选项。这个消息也有可能是由找不到外部命令引起的。但错误信息无法区别。

E12

Command not allowed from exrc/vimrc in current dir or tag search

因为安全原因，有的命令不允许被执行。这些命令通常来自当前目录的 .exrc 或者 .vimrc 文件或标签文件。另见 'secure'。

E74

Command too complex

某个映射产生了太长的命令行字符串。可能是因为一个映射间接的引用了自己。

CONVERSION ERROR

写入文件时，这个错误消息的出现意味着部分内容在从内部 UTF-8 格式转化到文件本身格式的过程中丢失。文件因此不会被标为未修改。如果你关心丢失的信息，把 'fileencoding' 选项设为别的能够处理缓冲区的字符的值并重新写一次。如果你觉得无所谓，直接放弃缓冲区或者复位 'modified' 选项。

E302

Could not rename swap file

如果文件名改变，Vim 试图对 swap-file 也进行换名。如果不成功，那么依然使用旧的交换文件。一般情况下这没有什么问题。

E43 E44

Damaged match string
Corrupted regexp program

Vim 的内部问题并因而产生了一个不合法的正规表达式。如果你知道如何重现，请报告。 bugs

E208 E209 E210

Error writing to "{filename}"
Error closing "{filename}"
Error reading "{filename}"

如果 Vim 试图给一个文件换名，但是简单的文件名更名不能工作时，会尝试进行文件复制，这些错误消息出现在这一步也失败的时候。结果可能是，原始文件和目标文件都存在，但是目标文件不完整。

Vim: Error reading input, exiting...

Vim 在需要输入却不能读入键入的字符时，就会报错。Vim 没法继续，除了退出以外别无选择。这可能发生在标准输入和标准错误 (stderr) 都被重定向，而一个自身不会退出 Vim 的脚本被执行的情况。

E47

Error while reading errorfile

不能读入错误文件。此消息的发生 不 是因为错误信息不能识别。

E80

Error while writing

文件的写入不能成功完成，该文件可能不完整。

E13 E189

File exists (add ! to override)
"{filename}" exists (add ! to override)

系统为了保护你，不让你不小心覆盖已存在文件。如有必要，使用相同的命令但在其后加 "!"。例如:

```
:w /tmp/test
```

可以改成:

```
:w! /tmp/test
```

E768

```
Swap file exists: {filename} (:silent! overrides)
```

系统能够保护你不去覆盖一个正在被 Vim 编辑的文件。在使用 ":w! filename" 并且发现一个交换文件时会出现这种情况。
- 如果交换文件是由以前某个崩溃的编辑对话留下的，也许你希望删除这个交换文件。编辑 {filename} 来找出关于该交换文件的信息。
- 如果你希望强行保存，那么在命令前加 ":silent!"。例如:
  ```
  :silent! w! /tmp/test
  ```
  之所以需要这个特殊命令，是因为你已经用 ! 来覆盖已经存在的文件了。

E139

```
File is loaded in another buffer
```

你试图写入文件，但是别的缓冲区正在使用你写入的名字。不这么做会导致相同文件的两个版本。

E142

```
File not written: Writing is disabled by 'write' option
```

'write' 选项被关闭。这使得所有试图写入文件的命令都会产生这个消息。这可能由 -m 命令行参数引起。你可以用 ":set write" 打开 'write' 选项。

E25

```
GUI cannot be used: Not enabled at compile time
```

你运行的 Vim 版本不支持 GUI 代码。所以 "gvim" 和 ":gui" 都不工作。

E49

```
Invalid scroll size
```

这是由 'scroll'、'scrolljump' 或者 'scrolloff' 选项的值不正确引起的。

E17

```
"{filename}" is a directory
```

你试图写入一个和目录名字相同的文件。这是不可以的。你可能想要附加一个文件名。

E19

```
Mark has invalid line number
```

你使用一个位置标记，但它所指的行号不存在。这可能是因为你使用了指向别的文件的位置标记，而别的程序删除了其中的一些行引起的。

E219 E220

```
Missing {.
Missing }.
```

在文件名中使用了 {} 构造，但只有 { 而没有匹配的 } 或者相反。该构造的使用方法是: 用 {foo,bar} 匹配 "foo" 和 "bar"。

E315

ml_get: invalid lnum: {number}

内部错误。如果你知道如何重现该问题，请提交漏洞报告。 bugreport.vim 。

E173

{number} more files to edit

你试图退出，但是参数列表里的末项还没有被编辑到。这保护你在还有文件要编辑的时候意外的退出。见 argument-list 。如果你坚持想退出，再试一次就行了。

E23 E194

No alternate file
No alternate file name to substitute for '#'

轮换文件还没有定义。见 alternate-file 。

E32

No file name

当前的缓冲区还没有名字。要写入，可用 ":w fname"。或者用 ":file fname" 先给出缓冲区的名字。

E141

No file name for buffer {number}

一个被改变的缓冲区没有文件名。所以不能被写入。你需要先给缓冲区一个文件名：

```
:buffer {number}
:file {filename}
```

E33

No previous substitute regular expression

在 ":substitute" 命令里，模式里使用的 '~' 字符会被替换成上次用到的模式。如果这个命令还没用过，就会失败。见 /~ 。如果使用 ":s/pat/%/"，也会发生类似问题，其中的 "%" 表示上次的替代字符串。

E35

No previous regular expression

使用空搜索模式会引用上次用到的模式。如果没有上一次搜索，那么就会失败。

E24

No such abbreviation

你使用了 ":unabbreviate" 命令，但是所带的参数不是一个已有的缩写。这个命令的各种变形 ":cunabbrev"、":iunabbrev" 等都会给出相同的消息。检查一下尾部的空白。

/dev/dsp: No such file or directory

只有在带 Gnome 支持的 GTK GUI 才会给出。Gnome 试图使用音频设备但是它不存在。你可以忽略这个错误。

E31

No such mapping

你使用了 ":unmap" 命令，但是所带的参数不是已有的映射。这个命令的各种变形 ":cunmap"、":unmap!" 等都会给出相同的消息。一些提示：
- 检查一下尾部的空白。
- 如果是局部于缓冲区的映射，需要使用 ":unmap <buffer>"。
  :map-<buffer>

E37 E89

No write since last change (add ! to override)
No write since last change for buffer {N} (add ! to override)

你试图放弃 abandon 一个有过改动的文件。Vim 不让你这么做，以免丢失你的工作。你可以把改动的部分用 ":w" 写回，如果你坚持要放弃而不介意并丢失所有的改变，可以在使用的命令之后加上 '!' 字符。例如：

```
:e other_file
```

可以改为：

```
:e! other_file
```

E162

No write since last change for buffer "{name}"

如果你退出 Vim 时有的缓冲区已经被改变了就会发生这个错误。你或者可以写入改动过的缓冲区 (用 :w)，或者可以用一个强迫放弃缓冲区的命令，比如，":qa!"。要小心，不要丢掉你想保留的改变。有些缓冲区可能已经被忘记了，尤其是置位了 'hidden' 的时候。

[No write since last change]

如果你执行 Shell 命令时至少一个缓冲区被改变了，就会发生这个错误。要不想显示这个信息，可以复位 'warn' 选项。

E38

Null argument

Vim 内部出错，产生一个 NULL 指针。如果你知道如何重现，请报告。 bugs

E41 E82 E83 E342

Out of memory!
Out of memory! (allocating {number} bytes)
Cannot allocate any buffer, exiting...
Cannot allocate buffer, using other one...

噢，噢。你可能作了一些特别复杂的操作，或者有其它程序正在占用你的内存。小心！Vim 没有完全准备好内存溢出的情况。首先确保所有的改变被保存。然后试图解决内存的短缺问题。要确保安全无虞，先退出 Vim，再重新开始。

如果这发生在 Vim 还在初始化时，文件编辑很可能不行，所以 Vim 会以值 123 退出。

缓冲区只是部分保存在内存里，所以编辑一个很大的文件不太可能导致内存溢出错误。撤销信息则是完全保存在内存里，可以用以下选项减少内存的占用：
- 'undolevels' 设为较小的值，或 -1 以完全关闭之。对影响所有行的改变这会很有帮助。

- 'undoreload' 设为零以关闭之。

E339

Pattern too long

这发生在 16 位整数的系统上：编译的正规表达式模式超过了 65000 个字节。试试短一些的模式。
这也可能发生在某个规则的偏移量超出了可用的空间。试试对模式进行简化。

E45

'readonly' option is set (add ! to override)

你试图写入一个标为只读的文件。如果要坚持这么做，或者复位 'readonly' 选项，或者在你用的命令后面增加 '!' 字符。例如：

```
:w
```

可以改为：

```
:w!
```

E294 E295 E301

Read error in swap file
Seek error in swap file read
Oops, lost the swap file!!!

Vim 试图从 swap-file 里读取文本时出了问题。相关缓冲区的文本现在可能已经没法用了！在写入缓冲区之前仔细检查。你可以先写到别的文件上，然后比较两者的差异。

E192

Recursive use of :normal too deep

你在使用 ":normal" 命令，而它的参数又递归使用 ":normal" 命令。'maxmapdepth' 限制最高的递归层数。下例显示如何得到这个消息：

```
:map gq :normal gq<CR>
```

如果你输入 "gq"，就会执行这个映射，它又会再次调用 "gq"。

E22

Scripts nested too deep

脚本可以通过 "-s" 命令行参数或者 ":source" 命令读取。脚本又可以读取别的脚本。这可以继续大概 14 层。如果需要更多的嵌套，Vim 假设其中出现了递归的循环，所以用此错误信息停止。

E319

Sorry, the command is not available in this version

你使用了目前使用的 Vim 版本不存在的命令。在编译 Vim 的时候，许多不同的特性可以打开或者关闭。这取决于编译时选择要多大的 Vim，还有具体操作系统。能用的特性可见 +feature-list 。 :version 命令显示 Vim 编译时所带的特性。

E300

Swap file already exists (symlink attack?)

当 Vim 试图打开交换文件并发现它已经存在，而且是一个符号链接时，显示该错误。这不应该发生，因为 Vim 已经检查过该文件不存在。除非别人同时打开了相同的文件（非常不太可能）或者有人试图一个符号链接攻击（如果编辑 /tmp 下的文件或者

'directory' 以 "/tmp" 开始时可能发生。这是一个坏的选择)。

E432

Tags file not sorted: {file name}

Vim (和 Vi) 期待标签文件以 ASCII 顺序排序。然后可以使用二分法搜索，这比线性查找快很多。如果你的标签文件没有适当的排序，复位 'tagbsearch' 选项。
这个消息只有在 Vim 在搜索标签时发生问题的时候才会给出。有的时候即使标签文件没有适当排序，这个消息也不会给出。

E424

Too many different highlighting attributes in use

Vim 只能处理 223 种不同的高亮。如果遇到了这个限制，你一定用了太多的带不同参数的 :highlight 命令。":highlight link" 不计算在内。

E77

Too many file names

在扩展文件名的时候，找到了多于一个的匹配。而使用的命令只支持单一匹配的参数。

E303

Unable to open swap file for "{filename}", recovery impossible

Vim 不能建立交换文件。你可以继续编辑该文件，但如果 Vim 不正常地退出，你所有的改变就丢失了。同时，Vim 可能在编辑大文件时消耗大量的内存。要避免这个错误，你可能想改变 'directory' 选项。当 'directory' 为空时不给出此错误。见 swap-file 。

E140

Use ! to write partial buffer

将区域对应缓冲区的部分内容写到一个已有的文件是不常见的。这可能是个错误 (比如，如果在可视模式下使用 ":w" 的时候)，所以 Vim 需要在命令后使用 !，比如: ":3,10w!"。

Warning: Cannot convert string "<Key>Escape,_Key_Cancel" to type VirtualBinding

启动时的时候可能出现这样的错误。这不是 Vim 的问题，而是你的 X11 配置有问题。你可以在这里看到如何解决这个问题的提示:
http://groups.yahoo.com/group/solarisonintel/message/12179。
[此 URL 已不可用]

W10

Warning: Changing a readonly file

文件是只读的，而你坚持要修改它。你可以用 FileChangedRO 自动命令事件来避免这个消息 (该自动命令必须复位 'readonly' 选项)。'modifiable' 可以完全禁止修改文件。
只有 'readonly' 置位后第一次作改变时，才显示这条消息。

W13

Warning: File "{filename}" has been created after editing started

你在 Vim 里开始编辑文件时它并不存在，但是现在存在了。你可以决定保留 Vim 的版本还是新建的文件。这个消息在 'buftype' 非空的时候不会给出。

W11

```
Warning: File "{filename}" has changed since editing started
```

你已经开始编辑的文件现在有了别的时间戳（修改时间），而且内容也发生了改变（更精确地说，使用当前选项设置和自动命令重新读入文件会给你不同的文本）。这可能是因为别的程序修改了文件。你需要自己研究发生了什么事，并决定保留哪个版本的内容。如果你想自动处理，参见 'autoread' 选项。如果 'buftype' 不为空，该消息不会给出。

有一种情况即使没发生任何错误也会得到这个消息：在 Windows 上，你在夏令时开始的那天保存一个文件。有几种方法可以修正之：
- 在你的 autoexec.bat 增加这行：

```
SET TZ=-1
```

  根据你的时区调整 "-1" 为其它值。
- 关闭 "根据夏令时时间自动调整时钟"。
- 第二天再次写入文件。或者把时钟设为第二天，写入文件两次，再把时钟调回来。

如果你不停收到 W11，可能需要关闭 "Acronis Active Protection" 或把 Vim 登记为受信任的服务/应用。

W12

```
Warning: File "{filename}" has changed and the buffer was changed in Vim as well
```

和上面类似，而这个 Vim 里这个文件的缓冲区也改变了。你必须决定要保留这个 Vim 的版本还是磁盘上的。这个消息在 'buftype' 非空时不会给出。

W16

```
Warning: Mode of file "{filename}" has changed since editing started
```

缓冲区的时间戳发生了改变时，内容保持不变但是模式（许可权限）发生了改变。这通常是从版本控制系统里取出文件而复位了只读位引起的。重读文件应该是安全的选择。置位 'autoread' 可以自动重读文件。

E211

```
File "{filename}" no longer available
```

你已经开始编辑的文件现在不再存在或不能访问。确定你把缓冲区写到某处，以免丢失改动。如果 'buftype' 非空，该消息不会给出。

W14

```
Warning: List of file names overflow
```

你一定用了太多的缓冲区了。可能两个缓冲区使用相同的号码，这会引起问题。你可能要退出 Vim 并重启。

E931

```
Buffer cannot be registered
```

内存溢出，或者重复的缓冲区号。可能在 W14 之后出现。查找缓冲区不总是有效，最好重启 Vim。

E296 E297

Seek error in swap file write
Write error in swap file

这通常是因为磁盘空间满引起的。Vim 不能把文本写入 swap-file 。它本身并无害，但如果 Vim 意外退出，有些文本可能丢失，而无法恢复。如果这个问题持续存在，Vim 可能会内存溢出。

connection-refused

Xlib: connection to "<machine-name:0.0" refused by server

当 Vim 试图连接到 X 服务器，但是 X 服务器不允许连接的时候，就会发生这个错误。只有建立 X 服务器的连接才能恢复标题和得到 xterm 的剪贴板的支持。不幸的是，这个错误信息无法避免，除非关闭 +xterm_clipboard 和 +X11 特性。

E10

\\ should be followed by /, ? or &

命令行以反斜杠开始，或者命令的范围包含了在错误位置出现的反斜杠。通常，这是因为命令行的续行功能被关闭了。从 'cpoptions' 选项里去掉 'C' 标志位就可以打开这一功能。使用 ":set nocp" 也可。

E471

Argument required

这是由 Ex 命令执行时需要的参数没有提供引起的。

E474 E475 E983

Invalid argument
Invalid argument: {arg}
Duplicate argument: {arg}

执行 Ex 命令或函数时使用了非法的参数。

E488

Trailing characters

在不允许有参数的 Ex 命令后带了参数。

E477 E478

No ! allowed
Don't panic!

给不支持带 ! 后缀的 Ex 命令加了 "!" 命令。

E481

No range allowed

在不支持范围参数的 Ex 命令前加了范围。见 cmdline-ranges 。

E482 E483

Can't create file {filename}
Can't get temp file name

Vim 不能建立临时文件。

E484 E485

```
Can't open file {filename}
Can't read file {filename}
```

Vim 不能读临时文件。特别是在 Windows 上，这可能由错误转义 cmd.exe 的中特殊字符引起。 7.3.443 补丁对转义的方法有所改变。对 system() 的所有外壳参数应有 shellescape() 进行转义，或直接用 ^ 转义。另见 'shellxquote' 和 'shellxescape'。

E464

```
Ambiguous use of user-defined command
```

有两个用户定义的命令，它们有相同的前缀，而你用命令行补全来执行其中的一个。 user-cmd-ambiguous
示例:

```
:command MyCommand1 echo "one"
:command MyCommand2 echo "two"
:MyCommand
```

E492

```
Not an editor command
```

你试图执行一个既不是 Ex 命令也不是用户定义的命令。

E943

```
Command table needs to be updated, run 'make cmdidxs'
```

只可能在修改了源代码，在 src/ex_cmds.h 中增加了新命令时发生。此时需要更新查找表，运行:

```
make cmdidxs
```

---

3. 消息 messages

这是一个（不完全）Vim 给出的各种消息的总览:

hit-enter press-enter hit-return
press-return hit-enter-prompt

```
Press ENTER or type command to continue
```

如果屏幕上有东西要你阅读，而屏幕即将刷新的时候，就会给出该消息:
- 执行完外部命令以后（例如，":!ls" 和 "="）。
- 状态行显示的内容超过了窗口的宽度或者跨越到 'showcmd' 或者 'ruler' 的输出位置。

-> 按 <Enter> 或者 <Space> 可以重画屏幕并继续，除此以外就像该键没按过一样。
-> 按 ":" 或者其它普通模式的命令字符开始执行该命令。
-> 按 'k'、<Up>、'u'、'b' 或 'g' 可以使消息回滚。这和 more-prompt 中的工作方式一样。只有关闭 'compatible' 并且打开 'more' 时才有效。
-> 消息的滚动离开屏幕的顶部时，按 'j'、'f'、'd' 或 <Down> 没有效果，'compatible' 关闭且 'more' 打开。这是为了避免按 'j' 或 'f' 太多而使得消息完

全消失。
-> 按 <C-Y> 复制（抽出）一个无模式的选择到剪贴板寄存器中。
-> 使用菜单。应用命令行模式定义的字符。
-> 如果 'mouse' 包含 'r' 标志位，按鼠标左键和按 <Space> 效果相同。不过这使得选择文本成为不可能。
-> GUI 模式上，在末行按鼠标左键和按 <Space> 效果相同。

如果你不小心按了 <Enter> 或 <Space>，而你还想看显示的消息，可以使用 g< 。这个命令只有 'more' 置位时才有效。

要减少 hit-enter 提示的次数:
- 把 'cmdheight' 设为 2 或者更高。
- 在 'shortmess' 里增加标志位。
- 复位 'showcmd' 和/或 'ruler'。
- 确保 :echo 文本不长于 v:echospace 屏幕单元格。

如果脚本导致出现按回车的提示而你不明所以， v:scrollstart 变量可能很有用。

另见 'mouse'。hit-enter 消息使用 hl-Question 高亮组。

more-prompt pager

```
-- More --
-- More -- SPACE/d/j: screen/page/line down, b/u/k: up, q: quit
```

如果屏幕被消息全部占据，本消息就可能出现。只有在 'more' 选项打开的时候它才会出现。它的显示使用 hl-MoreMsg 高亮组。

输入	效果
<CR> 或 <NL> 或 j 或 <Down>	下一行
d	下一页（半屏）
<Space> 或 f 或 <PageDown>	下一屏
G	一直往下，直到出现按回车的提示
<BS> 或 k 或 <Up>	往回一行
u	往上一页（半屏）
b 或 <PageUp>	上一屏
g	回到开始处
q、<Esc> 或 CTRL-C	终止列表
:	终止列表并进入命令行
<C-Y>	抽出（复制）无模式的选择 modeless-selection 到剪贴板 ("* 和 "+ 寄存器)
{menu-entry}	命令行模式里定义的菜单项。
<LeftMouse>	下页 (*)

所有其它的键保留它们本来的含义。

(*) 鼠标左键的点击只有在以下情况下才生效:
- GUI: 屏幕的最后一行。
- 'mouse' 里定义了 'r'（不过这时文本选择不会工作）。

注意：键的输入是直接从终端取得的。它不通过映射，预输入也被忽略。

g< 命令用于查看前一个命令输出的最后一页。如果在提示按回车的时候不小心按了空格 <Space> 键，那么该命令尤其有用。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

quotes

以下是一些我从新闻组和邮件收集的关于 Vim 的一些很好的评价 (英语)。

vim (vim) noun - Ebullient vitality and energy. [Latin, accusative of vis, strength] (Dictionary)

Vim is so much better than vi that a great many of my old vi :map's became immediately obsolete! (Tony Nugent, Australia)

Coming with a very GUI mindset from Windows, I always thought of people using Vi as some kind of outer space alien in human clothes. Once I tried I really got addicted by its power and now I found myself typing Vim keypresses in the oddest places! That's why I would like to see Vim embedded in every application which deals with text editing. (José Fonseca)

I was a 12-year emacs user who switched to Vim about a year ago after finally giving up on the multiple incompatible versions, flaky contributed packages, disorganized keystrokes, etc. And it was one of the best moves I ever made. (Joel Burton)

Although all of the programs were used during the preparation of the new and revised material, most of the editing was done with Vim versions 4.5 and 5.0 under GNU-Linux (Redhat 4.2). (Arnold Robbins, Israel, author of "Learning the Vi editor")

Out of all the open software i've ever seen and used, and i've seen a lot, Vim is the best, most useful and highest quality to work with, second only to the linux kernel itself. (Peter Jay Salzman)

It's well worth noting that the _entirety_ of SourceForge was written using Vim and its nifty PHP syntax highlighting. I think the entire SF.net tech staff uses Vim and we're all excited to have you aboard! (Tim Perdue)

Vim is one of a select bunch of tools for which I have no substitute. It is a brilliant piece of work! (Biju Chacko)

A previous girlfriend of mine switched to emacs. Needless to say, the relationship went nowhere. (Geoffrey Mann)

I rarely think about Vim, in the same way that I guess a fish rarely thinks about water. It's the environment in which everything else happens. I'm a fairly busy system administrator working on a lot of different platforms. Vim is the only thing that's consistent across all my systems, and it's just about the only thing that doesn't break from time to time. When a new system comes in the door without Vim, I install it right away. Great to have a tool that's

the same everywhere, that's completely reliable, so I can ignore it and think about other things. (Pete Schaeffer)

Having recently succeeded in running Vim via telnet through a Nokia Communicator, I can now report that it works nicely on a Palm Pilot too. (Allan Kelly, Scotland)

You've done a tremendous job with 'VIM', Bram! The more I use it, the more impressed I get (I am an old 'vi' die hard who once started out with early versions of 'emacs' in the late 1970's and was relieved by finding 'vi' in the first UNIX I came across in 1983). In my opinion, it's about time 'VIM' replace 'emacs' as the standard for top editors. (Bo Thide', Sweden)

I love and use Vim heavily too. (Larry Wall)

Vi is like a Ferrari, if you're a beginner, it handles like a bitch, but once you get the hang of it, its small, powerful and FAST! (Unknown)
Vim is like a new model Ferrari, and sounds like one too - "VIIIIIIMMM!" (Stephen Riehm, Germany)

Schon bei Nutzung eines Bruchteils der Vim-Funktionen wird der Benutzer recht schnell die Vorzuege dieses Editors kennen- und schaetzenlernen.
Translated: Even when only using a fraction of Vim-functions, the user will quickly get used to and appreciate the advantages of this editor. (Garry Glendown, conclusion of an article on Vim in iX magazine 9/1998)

I've recently acquired the O'Reilly book on Vi (it also discusses Vim in-depth), and I'm amazed at just how powerful this application is. (Jeffrey Rankin)

This guide was written using the Windows 9.x distribution of gvim, which is quite possibly the greatest thing to come along since God created the naked girl. (Michael DiBernardo)

Boy, I thought I knew almost everything about Vim, but every time I browse the online documentation, I hit upon a minor but cool aspect of a Vim feature that I didn't know before! I must say the documentation is one the finest I've ever seen in a product -- even better than most commercial products. (Gautam Mudunuri)

Vim 4.5 is really a fantastic editor. It has sooooo many features and more importantly, the defaults are so well thought out that you really don't have to change anything!! Words cannot express my amazement and gratitude to the creators of Vim. Keep it up. (Vikas, USA)

I wonder how long it will be before people will refer to other Vi editors as Vim clones? (Darren Hiebert)

I read about [auto-positioning-in-file-based-on-the-errors-from-make] in one of those "Perfect Programmer's Editor" threads and was delighted to discover that Vim already supports it. (Brendan Macmillan, Australia)

I just discovered Vim (5.0) and I'm telling everyone I know about it!
I tell them Vim stands for Vi for the new (M)illenium. Thanks so much!

(Matt F. Valentine)

I think from now on "vi" should be called "Vim Imitation", not the other way around. (Rungun Ramanathan)

The Law of Vim:
For each member b of the possible behaviour space B of program P, there exists a finite time t before which at least one user u in the total user space U of program P will request b becomes a member of the allowed behaviour space B' (B' <= B).
In other words: Sooner or later everyone wants everything as an option. (Negri)

Whenever I move to a new computing platform, the first thing I do is to port Vim. Lately, I am simply stunned by its ease of compilation using the configure facility. (A.M. Sabuncu, Turkey)

The options are really excellent and very powerful. (Anish Maharaj)

The Spring user-interface designs are in, and word from the boutiques is that 80x24 text-only mode is back with a *vengeance! Vi editor clone Vim burst onto March desk-tops with a dazzling show of pastel syntax highlights for its 5.0 look. Strident and customizable, Vim raises eyebrows with its interpretation of the classic Vi single-key macro collection.
http://www.ntk.net/index.cgi?back=archive98/now0327.txt&line=179#1

I just wanted to take this opportunity to let you know that Vim 5 ROCKS! Syntax highlighting: how did I survive without it?! Thank you for creating mankind's best editor! (Mun Johl, USA)

Thanks again for Vim. I use it every day on Linux. (Eric Foster-Johnson, author of the book "UNIX Programming Tools")

The BEST EDITOR EVER (Stuart Woolford)

I have used most of Vim's fancy features at least once, many frequently, and I can honestly say that I couldn't live with anything less anymore. My productivity has easily doubled compared to what it was when I used vi. (Sitaram Chamarty)

I luv Vim. It is incredible. I'm naming my first-born Vimberly. (Jose Unpingco, USA)

Hint: "Vim" is "vi improved" - much better! (Sven Guckes, Germany)

I use Vim every day. I spend more time in Vim than in any other program... It's the best vi clone there is. I think it's great. (Craig Sanders, Australia)

I strongly advise using Vim--its infinite undo/redo saved me much grief. (Terry Brown)

Thanks very much for writing what in my opinion is the finest text editor on the planet. If I were to get another cat, I would name it "Vim".

(Bob Sheehan, USA)

I typed :set all and the screen FILLED up with options. A whole screen of things to be set and unset. I saw some of my old friends like wrapmargin, modelines and showmode, but the screen was FILLED with new friends! I love them all! I love Vim! I'm so happy that I've found this editor! I feel like how I once felt when I started using vi after a couple of years of using ed. I never thought I'd forsake my beloved ed, but vi ... oh god, vi was great. And now, Vim. (Peter Jay Salzman, USA)

I am really happy with such a wonderful software package. Much better than almost any expensive, off the shelf program. (Jeff Walker)

Whenever I reread the Vim documentation I'm overcome with excitement at the power of the editor. (William Edward Webber, Australia)

Hurrah for Vim!! It is "at your fingertips" like vi, and has the extensions that vi sorely needs: highlighting for executing commands on blocks, an easily navigable and digestible help screen, and more. (Paul Pax)

The reason WHY I don't have this amazingly useful macro any more, is that I now use Vim - and this is built in!! (Stephen Riehm, Germany)

I am a user of Vim and I love it. I use it to do all my programming, C, C++, HTML what ever. (Tim Allwine)

I discovered Vim after years of struggling with the original vi, and I just can't live without it any more. (Emmanuel Mogenet, USA)

Emacs has not a bit of chance to survive so long as Vim is around. Besides, it also has the most detailed software documentation I have ever seen---much better than most commercial software! (Leiming Qian)

This version of Vim will just blow people apart when they discover just how fantastic it is! (Tony Nugent, Australia)

I took your advice & finally got Vim & I'm really impressed. Instant convert. (Patrick Killelea, USA)

Vim is by far my favorite piece of shareware and I have been particularly pleased with version 3.0. This is really a solid piece of work. (Robert Colon, USA)

Vim is a joy to use, it is so well thought and practical that I wonder why anybody would use visual development tools. Vim is powerful and elegant, it looks deceptively simple but is almost as complex as a 747 (especially when I look at my growing .vimrc), keep up that wonderful job, Vim is a centerpiece of the free software world. (Louis-David Mitterand, USA)

I cannot believe how great it is to use Vim. I think the guys at work are getting tired of hearing me bragging about it. Others eyes are lighting up. (Rick Croote)

Emacs takes way too much time to start up and run, it is too big and bulky for

effective use and the interface is more confusing than it is of any help. Vim however is short, it is fast, it is powerful, it has a good interface and it is all purpose. (Paal Ditlefsen Ekran)

From the first time I got Vim3.0, I was very enthusiastic. It has almost no problems. The swapfile handling and the backup possibilities are robust, also the protection against editing one file twice. It is very compatible to the real VI (and that is a MUST, because my brain is trained over years in using it). (Gert van Antwerpen, Holland)

Visual mode in Vim is a very powerful thing! (Tony Nugent, Australia)

I have to say that Vim is =THE= single greatest piece of source code to ever come across the net (Jim Battle, USA).

In fact, if you do want to get a new vi I'd suggest Vim-3.0. This is, by far, the best version of vi I've ever seen (Albert W. Schueller).

I should mention that Vim is a very good editor and can compete with anything (Ilya Beloozerov).

To tell the truth sometimes I used elvis, vile, xvi, calvin, etc. And this is the reason that I can state that Vim is the best! (Ferenc Deak, Hungary)

Vim is by far the best editor that I have used in a long time, and I have looked at just about every thing that is available for every platform that I use. Vim is the best on all of them. (Guy L. Oliver)

Vim is the greatest editor since the stone chisel. (Jose Unpingco, USA)

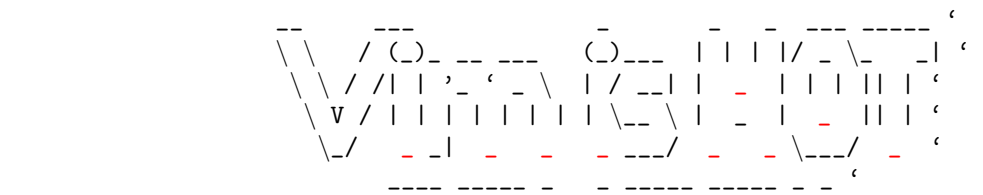
I would like to say that with Vim I am finally making the 'emacs to vi' transition - as an Editor it is so much better in many ways: keyboard layout, memory usage, text alteration to name 3. (Mark Adam)

In fact, now if I want to know what a particular setting does in vi, I fire up Vim and check out its help! (Nikhil Patel, USA)

As a vi user, Vim has made working with text a far more pleasant task than before I encountered this program. (Steinar Knutsen, Norway)

I use Vim since version 3.0. Since that time, it is the ONLY editor I use, with Solaris, Linux and OS/2 Warp. I suggest all my friends to use Vim, they try, and they continue using it. Vim is really the best software I have ever downloaded from the Internet, and the best editor I know of. (Marco Eccettuato, Italy)

一言而蔽之:



翻译: Vim 真是个好东西!

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

Vim 待完成任务列表 todo

这是一个很长。。。的已知漏洞，目前工作和未来希望的改进的列表（英文）。为了使其能稍微好找一点，旧项目由主题编组。每行的第一列是标明准备时候去做的分类，以便按部就班，依次解决。

优先级分类:
9 下一个小发行
8 下一个主要发行
7 尽快
6 很快
5 应该会加入
4 能有更好
3 考虑更好
2 也许不太会
1 可能不会
- 未分类

关于开发的计划，参见 develop.txt 。如果你赞助 Vim 开发，你可以投票决定哪个项目值得开发。请见 sponsor 。 votes-for-changes

可以在线提供新问题: https://github.com/vim/vim/issues
只用于漏洞报告，不是提问题！那应该去邮件组。其中更新的部分会被转发到 vim_dev 邮件列表。该处已经提供的问题就不在下面重复了，除非有额外的信息。

#1234 这样的号码指 github 的议题或拉取请求。要在浏览器上查看，可用:
https://github.com/vim/vim/issues/1234

(把 1234 替换为实际的议题/拉取请求号)

---------- 已知漏洞和当前工作 ---------- known-bugs

Include ipv6 syntax changes? (DJ Lucas, #5360)

Popup windows:
- Make it possible to put a terminal window in a popup. Would always grab key input? Sort-of possible by creating a hidden terminal and opening a popup with that buffer: #4063.
- Use popup (or popup menu) for command line completion
- Implement flip option
- Make redrawing more efficient and avoid flicker:
  - put popup menu also in popup_mask?
- Any other commands to disable in a popup window?
  Use ERROR_IF_POPUP_WINDOW for these.

- Figure out the size and position better if wrapping inserts indent

Text properties:
- Implement prop_find() issue #4970
- Text properties spanning more than one line
- See remarks at top of src/textprop.c

'incsearch' with :s:
- :s/foo using CTRL-G moves to another line, should not happen, or use the
  correct line (it uses the last but one line) (Lifepillar, Aug 18, #3345)
- :s@pat/tern@ doesn't include "/" in the pattern. (Takahiro Yoshihara, #3637)
   pass delim to do_search() ?
- Also support range: :/foo/,/bar/delete
- Also support for user command, e.g. Cfilter
- :%s/foo should take the first match below the cursor line, unless there
  isn't one?
  Then :%s?foo should take the first match above the cursor line.

Prompt buffer:
- Add a command line history, using up/down keys. issue #5010
- delay next prompt until plugin gives OK?
- add prompt_addtext({buf}, {expr})   none   add text to a prompt buffer

Terminal debugger:
- Make prompt-buffer variant work better.
- Add option to not open the program window. It's not used when attaching to
  an already running program. (M. Kelly)
- When only gdb window exists, on "quit" edit another buffer.
- Use a sign group
- Termdebug does not work when Vim was build with mzscheme: gdb hangs just
  after "run". Everything else works, including communication channel. Not
  initializing mzscheme avoid the problem, thus it's not some #ifdef.
- Add support for lldb? issue #3565
- Could we do something similar for debugging Vim scripts? At least see the
  script while stepping through it. Simple version would use an extra window.
  More complete solution would actually run Vim in a Terminal and control it
  with another Vim instance.

Terminal emulator window:
- When the job in the terminal doesn't use mouse events, let the scroll wheel
  scroll the scrollback, like a terminal does at the shell prompt. #2490
  And use modeless selection. #2962
- Use CTRL-W CTRL-K to enter a digraph? #5371
- When Vim runs in the terminal and changes the title, the statusline needs to
  be redrawn.
- GUI: When using ":set go+=!" a system() call causes the hit-enter prompt.
  (#3327)
- GUI: hang until key typed. (#3530)
- Allow for specifying the directory, with ++cwd={dir}.
- With a vertical split only one window is updated. (Linwei, 2018 Jun 2,
  #2977)
- When pasting should call vterm_keyboard_start_paste(), e.g. when using
  K_MIDDLEMOUSE, calling insert_reg().
- Users expect parsing the :term argument like a shell does, also support

  single quotes. E.g. with: :term grep 'alice says "hello"' (#1999)
- Win32: Redirecting input does not work, half of Test_terminal_redir_file()
  is disabled.
- Win32: Redirecting output works but includes escape sequences.
- Win32: Make terminal used for :!cmd in the GUI work better. Allow for
  redirection.
- When the job only outputs lines, we could handle resizing the terminal
  better: store lines separated by line breaks, instead of screen lines,
  then when the window is resized redraw those lines.
- Redrawing is slow with Athena and Motif. (Ramel Eshed)
- For the GUI fill termios with default values, perhaps like pangoterm:
  http://bazaar.launchpad.net/~leonerd/pangoterm/trunk/view/head:/main.c#L134
- When 'encoding' is not utf-8, or the job is using another encoding, setup
  conversions.

Error numbers available:
E290, E292, E362, E366, E450, E451, E452,
E453, E454, E460, E489, E491, E565, E578, E610, E611, E653,
E654, E856, E857, E860, E861, E863, E889, E900

Patch to move duplicated code to a function. (Yegappan Lakshmanan, #5330)

Patch to use more FOR_ALL_ macros and use them. (Yegappan Lakshmanan, #5339)

Running test_gui and test_gui_init with Motif sometimes kills the window
manager. Problem with Motif? Now test_gui crashes in submenu_change().
Athena is OK.
Motif: Build on Ubuntu can't enter any text in dialog text fields.

Patch to properly break CJK lines: Anton Kochkov, #3875
Flag in 'formatoptions' is not used in the tests.

Patch to add 'vtp' option. (#5344)
Needs better docs. Is there a better name?

undo result wrong: Masato Nishihata, #4798

Undo puts cursor in wrong line after "cG<Esc>" undo.

:unmap <c-n> gives error but does remove the mapping. (Antony Scriven, 2019
Dec 19)

Sound: support on Mac? Or does libcanberra work there?

Patch to fix session file when using multiple tab pages. (Jason Franklin, 2019
May 20)
Also put :argadd commands at the start for all buffers, so that their order
remains equal? Then %argdel to clean it up. Do try this with 'hidden' set.
Also #4994: window-local options not always restored, related to using :badd.
Also #5326: netrw buffers are not restored.

Completion mixes results from the current buffer with tags and other files.
Happens when typing CTRL-N while still searching for results. E.g., type "b_"
in terminal.c and then CTRL-N twice.

Should do current file first and not split it up when more results are found.
(Also #1890)

Adding "10" to 'spellsuggest' causes spell suggestions to become very slow.
(#4087)

Instead of using a syntax error, use #error to fail early when something is
wrong. E.g. in vim.h "Error:", and blowfish.c "error!". (#5299)

":helptags ALL" should skip directories where "tags" cannot be written.
(Matěj Cepl, #5026)

":bnext" in a help buffer is supposed to go to the next help buffer, but it
goes to any buffer, and then :bnext skips help buffers, since they are
unlisted. (#4478)

Enable 'termbidi' if $VTE_VERSION >= 5703 ?

Universal solution to detect if t_RS is working, using cursor position.
Koichi Iwamoto, #2126

The :syntax cchar value can only be a single character. It would be useful to
support combining characters. (Charles Campbell) Also #4687

Include Haiku port. (Adrien Destugues, Siarzhuk Zharski, 2013 Oct 24)
It can replace the BeOS code, which is likely not used anymore.
Now on github: #1856. Updated Oct 2017
Got permission to include this under the Vim license.

Result of synID() sometimes wrong in help files. (#5252)

Problem showing a line if the number column width changes when using "o".
(Mateusz Morusiewicz, #4245)

When using :packadd for a replacement language plugin, it is loaded after the
default one. #4698

When using :packadd files under "later" are not used, which is inconsistent
with packages under "start". (xtal8, #1994)

Patch to add new motion](and]{. (Yasuhiro Matsumoto, #5320)

Modeless selection doesn't work in gvim. (#4783)
Caused by patch 8.1.1534.

Visual highlight not removed when 'dipslay' is "lastline" and line doesn't
fit. (Kevin Lawler, #4457)

Patch to add per-tabpage and per-window previous directory: "lcd -" and "tcd
-". (Yegappan Lakshmanan, #4362)

Does not build with MinGW out of the box:
- _stat64 is not defined, need to use "struct stat" in vim.h
- WINVER conflict, should use 0x0600 by default?

Crash when mixing matchadd and substitute()? (Max Christian Pohle, 2018 May 13, #2910) Can't reproduce?

Patch to configure BUILD_DATE for reproducible builds. (James McCoy, #513)

Patch to add MODIFIED_BY to MSVC build file. (Chen Lei, 2016 Nov 24, #1275)

Patch to support "0o" for octal numbers. (Ken Takata, #5304)

When getting a focus event halfway a mapping this aborts the mapping. E.g. when "qq" is mapped and after the first "q" the mouse is moved outside of the gvim window (with focus follows mouse), then the K_FOCUSLOST key is put in the input buffer. (#5302)

xterm should be able to pass focus changes to Vim, so that Vim can check for buffers that changed. Perhaps in misc.c, function selectwindow().
Xterm 224 supports it!
Patch to make FocusGained and FocusLost work in modern terminals. (Hayaki Saito, 2013 Apr 24) Update 2016 Aug 12.
Also see issue #609.
We could add the enable/disable sequences to t_ti/t_te or t_ks/t_ke.

Check_external_diff() is used too often. (Daniel Hahler, #4800)

Win32: after "[I" showing matches, scroll wheel messes up screen. (Tsakiridis, 2007 Feb 18)
Patch by Alex Dobrynin, 2007 Jun 3. Also fixes other scroll wheel problems.

Add a WindowScrolled event. Trigger around the same time as CursorMoved.
Can be used to update highlighting. #3127 #5181

Patch for Template string: #4491. New pull: #4634
Implementation is too inefficient, avoid using lambda.

Incorrect formatting with autoindent. (Sebastian Gniazdowski, #4909)

Patch to add the :bvimgrep command. (Christian Brabandt, 2014 Nov 12)
Updated 2016 Jun 10, #858 Update 2017 Mar 28: use <buffer>

Errors found with random data:
    heap-buffer-overflow in alist_add (#2472)

Patch to support CamelCase for spell checking: See a lower-to-upper case change as a word boundary. (btucker-MPCData, 2016 Nov 6, #1235)
patch for 'spellcamelcase' option: spellcheck each CamelCased word.
(Ben Tucker, 2016 Dec 2)

Patch to add {skip} argument to search(). (Christian Brabandt, 2016 Feb 24)
Update 2016 Jun 10, #861

Patch to add "cmdline" completion to getcompletion(). (Shougo, Oct 1, #1140)

Improve fallback for menu translations, to avoid having to create lots of

files that source the actual file. E.g. menu_da_de -> menu_da
Include part of #3242?

When a terminal exit_cb closes the window, a following typed key is lost, if it's in a mapping. (2018 Oct 6, #2302, #3522)

Patch to have text objects defined by arbitrary single characters. (Daniel Thau, 2013 Nov 20, 2014 Jan 29, 2014 Jan 31)
Added tests (James McCoy, 2016 Aug 3, #958). Still needs more work.

Patch for multi-byte characters in langmap and applying a mapping on them. (Christian Brabandt, 2015 Jun 12, update July 25)
Is this the right solution? Need to cleanup langmap behavior:
- in vgetorpeek() apply langmap to the typeahead buffer and put the result in a copy-buffer, only when langmap is appropriate for the current mode. Then check for mapping and let gotchars() work on the copy-buffer.
- Remove LANGMAP_ADJUST() in other parts of the code. Make sure the mode is covered by the above change.

So that replaying the register doesn't use keymap/langmap and still does the same thing. Remarks on PR #543 (Roland Puntaier).
Also see #737: langmap not applied to replaying recording.

Would be nice to set tab-local values for 'diffexpr' and 'diffopt'. Use t:diffexpr_option t:diffopt_option? (#4782)

v:register isn't reset early enough, may be used by next command.
(Andy Massimino, #5294, possible fix in #5305)

Internal diff doesn't handle binary file like external diff does. (Mike Williams, 2018 Oct 30)

'[mark in wrong column after put. (#4776)

"exepath('bin/cmd')" does not work while ":!bin/cmd" does work.
(Daniel Hahler, #4710) and executable('bin/cmd') returns 1

Error drawing the number column when 'cursorline' is set. (#3893)

Problem with :tlmenu: Detach item added with all modes? Issue #3563.

Add an argument to expandcmd() to expand like ":next" does.

When both "a" and "l" is in 'formatoptions' then auto-formatting also happens in a long line. #5189

The quoting of the [command] argument of :terminal is not clearly documented. Give a few examples. (#4288)

Opening a file with --remote-tab-silent that matches 'wildignore' does not work, results in (E479: No match". (#4610)

Patch for this (Tristan Konolige, #1011, only adds the option, no implem.):
7 Add an option to add one pixel column to the character width? Lucida Console italic is wider than the normal font ("d" overlaps with next char).

Opposite of 'linespace': 'columnspace'.

Bug: script written with "-W scriptout" contains Key codes, while the script
read with "-s scriptin" expects escape codes. Probably "scriptout" needs to
be adjusted. (Daniel Steinberg, 2019 Feb 24, #4041)

Add a string to the 'display' option ("smoothscroll" ?) to make CTRL-E and
CTRL-Y scroll one screen line, also if this means the first line doesn't start
with the first character (like what happens with a last line that doesn't
fit). Display "<<<" at the start of the first visible line (like "@@@" is
displayed in the last line). (Arseny Nasokin, #5154)

Patch for ambiguous width characters in libvterm on MS-Windows 10.
(Nobuhiro Takasaki, #4411)

Window size changes after closing a tab. (#4741)

Problem with colors in terminal window. (Jason Franklin, 2019 May 12)

Lifepillar: Updated/cleaned up color schemes:
https://github.com/lifepillar/vim8-colorschemes.

Include a few color schemes, based on popularity:
http://www.vim.org/scripts/script_search_results.php?keywords=&script_type=color+scheme&order_by=r
http://vimawesome.com/?q=tag:color-scheme
Use names that indicate their appearance (Christian Brabandt, 2017 Aug 3)
- monokai - Xia Crusoe (2017 Aug 4)
- seoul256 - Christian Brabandt (2017 Aug 3)
- gruvbox - Christian Brabandt (2017 Aug 3) (simplified version from
    Lifepillar, 2018 Jan 22, #2573)
- janah - Marco Hinz (2017 Aug 4)
- apprentice - Romain Lafourcade (2017 Aug 6) remarks about help file #1964
Suggested by Hiroki Kokubun:
- [Iceberg](https://github.com/cocopon/iceberg.vim) (my one)
- [hybrid](https://github.com/w0ng/vim-hybrid)
Include solarized color scheme?, it does not support termguicolors.
- Sanitized version of pablo (Lifepillar, 2017 Nov 21)

Bug: "vipgw" does not put cursor back where it belongs. (Jason Franklin, 2019
Mar 5)

Many users don't like 'thesaurus' to match so many words. (#4667, #1611)
- when the match with the first word is partial, don't add other words on that
  line.
- support other file formats, such as comma separated. Indicate by a prefix
  to the file in the 'thesaurus' option.
- 'threglookexp' option: only match with first word in thesaurus file.
  (Jakson A. Aquino, 2006 Jun 14)

Some composing characters actually add a cell width to the character they are
on top off, making the whole thing two characters wide. (#4526)

Should we include some part of pull request #4505, not increment changedtick
in some cases? E.g. for ":write" when the changed flag was already off, the

buffer didn't change at all.

Patch to add getreginfo() and setreg() with an option to set the unnamed
register "", So that registers can be saved and fully restored.
(Andy Massimino, 2018 Aug 24, #3370)

Line numbers in profile are off when function was defined with ":execute".
(Daniel Hahler, #4511)

Add a way to create an empty, hidden buffer. Like doing ":new|hide".
":let buf = bufcreate('name')

Session file contains absolute paths when "curdir" is removed form
'sessionoptions', making it impossible to have a session with a relative path.
(#4450)

When using a timer callback vgetc_busy is reset, allowing for using input().
But in a channel callback this does not happen. We need to do something
similar to check_due_timer(). Also see #3809.

C syntax: {} inside () causes following {} to be highlighted as error.
(Michalis Giannakidis, 2006 Jun 1)

Support setting the character displayed below the last line? Neovim uses
"eob:X" in 'fillchars'.

Check: __attribute__((format(printf, on semsg() and siemsg(). Where was this
added?

Patch to add a flatten() function. #3676. Check that the doc explains the
maxdepth argument (applies to the input "recursiveness").

Add test for urxvt mouse codes. Also test that mouse coordinates can be
negative. (see #4326)

'cmdheight' has a tab-local value, but it cannot be obtained with
`:echo gettabwinvar(2, 1, '&cmdheight')` returns the value for the _current_
tab page. (Ingo Karkat, #4324)
:call settabwinvar(1, 1, '&cmdheight', 2) also doesn't work well.

This modeline throws unexpected errors: (#4165)
    vim: syn=nosyntax

":doau SomeEvent" gives "No matching autocommands". This message doesn't give
a hint about how to fix it. (#4300)

Make balloon_show() work outside of 'balloonexpr'? Users expect it to work:
#2948. (related to #1512?)
On Win32 it stops showing, because showState is already ShS_SHOWING.
balloon_show() does not work properly in the terminal. (Ben Jackson, 2017 Dec
20, #2481)
Also see #2352, want better control over balloon, perhaps set the position.
Should also be possible to add highlighting, like in the status line?
balloonexpr() on MS-Windows GUI doesn't handle accented chars? (nivaemail,

2018 Sep 14)
Another request: #3811.

More warnings from static analysis:
https://lgtm.com/projects/g/vim/vim/alerts/?mode=list

Still a E315 error when using terminal. (Masato Nishihata, #3959)

Not existing directory in CDPATH leads to two shell calls. (#4525)

Use dict_iterate_start() / dict_iterate_next() instead of relying on the
internals of the dict structure.

nvo-mode mapping works on Windows, not on Linux. (#3678)

Missing tests for:
- add_termcap_entry()

Redo only remembers the last change. Could use "{count}g." to redo an older
change. How does the user know which change? At least have a way to list
them: ":repeats". Add to history, like search history and command line history.

Incsearch test fails when locale is "C". (Dominique Pelle, #3986)
Also run all tests with C locale?

"vat" doesn't work well on XML when the closing > is on another line.
#3927

Pasting foo} causes Vim to behave weird. (John Little, 2018 Jun 17)
Related to bracketed paste. I cannot reproduce it.

When 'confirm' is set a "silent q" doesn't show the prompt. It should in this
case. (Nate Peterson, 2019 Jan 31, #3892)
For "silent! q" it should not prompt and just fail.

Add <aevent>, which expands to the currently triggered autocommand event name.
(Daniel Hahler, #4232) Or add it to v:event (easier to use but slightly more
expensive).

Some xterm responses are not properly handled: (Markus Gmmel, 2019 Apr 1)
    DCS 0 $ r Pt ST   should be ignored.
    DCS 0 + r/Pt/ ST already ignored?

Using CTRL-L to add a character to the search string that contains \v,
punctuation is repeated. (Smylers, 2018 Nov 17, #3621)

ml_get error: (Israel Chauca Fuentes, 2018 Oct 17, #3550).

Patch to add more info to OptionSet. Should mention what triggered the change
":set", ":setlocal", ":setglobal", "modeline"; and the old global value.
#4118. Proposed implementation: 2019 Mar 27.
Updated 2019 May 25.

Using single wide base character with double wide composing character gives

drawing errors. Fill up the base character? (Dominique, #4328)

Problem with two buffers with the same name a/b, if it didn't exist before and is created outside of Vim. (dskloetg, 2018 Jul 16, #3219)

When 'sidescrolloff' is set, using "zl" to go to the end of the line, suddenly scrolls back. Should allow for this scrolling, like 'scrolloff' does when using CTRL-E. (Yee Cheng Chin, #3721)

Invalid memory access with old regexp engine. (Dominique Pelle, 2018 Sep 3, #3405) Introduced by 8.0.1517, which was fixing another memory access error. (Sep 8)

Add function to make use of internal diff, working on two lists and returning unified diff (list of lines).

When splitting a window with few text lines, the relative cursor position is kept, which means part of the text isn't displayed. Better show all the text when possible. (Dylan Lloyd, #3973)

Make ":interactive !cmd" stop termcap mode, also when used in an autocommand. (#3692)

Add buffer argument to undotree(). (#4001)

```
Using uninitialized value in test_gn
Using uninitialized value in test_crypt.
memory leak in test_paste
Memory leak in test_terminal:
==23530==    by 0x2640D7: alloc (misc2.c:874)
==23530==    by 0x2646D6: vim_strsave (misc2.c:1315)
==23530==    by 0x25841D: FullName_save (misc1.c:5443)
==23530==    by 0x17CB4F: fix_fname (buffer.c:4794)
==23530==    by 0x17CB9A: fname_expand (buffer.c:4838)
==23530==    by 0x1759AB: buflist_new (buffer.c:1889)
==23530==    by 0x35C923: term_start (terminal.c:421)
==23530==    by 0x2AFF30: mch_call_shell_terminal (os_unix.c:4377)
==23530==    by 0x2B16BE: mch_call_shell (os_unix.c:5383)
TODO: be able to run all parts of test_alot with valgrind separately
Memory leak in test_alot with pyeval() (allocating partial)
Memory leak in test_alot with expand()
Memory leaks in test_channel? (or is it because of fork())
```

gethostbyname() is old, use getaddrinfo() if available. (#3227)

matchaddpos() gets slow with many matches. Proposal by Rick Howe, 2018 Jul 19.

```
Should make 'listchars' global-local.  Local to window or to buffer?
Probably window.  #5206
Add something like 'fillchars' local to window, but allow for specifying a
highlight name.  Esp. for the statusline.
And "extends" and "precedes" are also useful without 'list' set.  Also in
'fillchars' or another option?
```

Sourceforge Vim pages still have content, redirect from empty page.
Check for PHP errors. (Wayne Davison, 2018 Oct 26)

Patch to support ":tag <tagkind> <tagname>". (emmrk, 2018 May 7, #2871)
Use something like ":tag {kind}/{tagname}".
Not ready to include.

Problem with Visual yank when 'linebreak' and 'showbreak' are set.
Patch with tests, but it's not clear how it is supposed to work. (tommm, 2018
Nov 17) Asked about this, Dec 22. Christian will have a look.

Patch for larger icons in installer. (#978) Still not good.

Patch to fix that using "5gj" starting inside a closed fold does not work on
screen lines but on text lines. (Julius Hulsmann, #4095) Lacks a test.

Patch to implement 'diffref' option. (#3535)
  Easier to use a 'diffmaster' option, is the extra complexity needed?
  Not ready to include.

Patch to specify color for cterm=underline and cterm=undercurl, like "guisp".
Patch #2405 does something like this, but in the wrong way.

home_replace() uses $HOME instead of "homedir". (Cesar Martins, 2018 Aug 9)

When the status line uses term_gettitle(), it does not get updated when the
terminal title changes. (Josh Triplett, 2018 Sep 9, #3418)
How would we know that the status line needs to be updated?

Adjust windows installer explanation of behavior. (scootergrisen, #3310)

Update for xim-input-style help (Tony Mechelynck, 2019 Jan 10).
Feedback from someone who uses this?

ml_get error. (Dominique Pelle, 2018 Sep 14, #3434)

Only output t_Cs when t_Ce is also set. do not use Cs and Ce termcap entries. (Daniel Hahler, 20
Add t_cS and t_cR for cursor color select and reset. Use Cs and Cr terminfo
values.

Further xdiff changes:
- More options, e.g. different kind of whitespace diff.
- when editing text, update the surrounding diff blocks.
- omit diff.exe from distribution
- Can we make this show differences within a line?
- add option to use external diff above a certain size.

Difference between two regexp engines: #3373

Patch to add ch_listen() (Yasuhiro Matsumoto, 2018 Nov 26, #3639)
What is the practical use for this?

When the last line wraps, selecting with the mouse below that line only

includes the first screen line. (2018 Aug 23, #3368)

Refactored HTML indent file. (Michael Lee, #1821)
Asked to write a test.

MS-Windows: .lnk file not resolved properly when 'encoding' is set.
(lkintact, 2018 Sep 22, #3473)

Merge checking for 'cursorline' and 'concealcursor', see neovim #9492.

Display error when a conceal match uses '\%>11'. (#4854)

Add a windowID argument to placing a sign, so that it only shows up in one
window for the buffer.

Request to add sign_setlist() to make it faster to add a lot of signs, e.g.
when adding a sign for every quickfix entry. (#4557)

Win32 key codes are messy. Mike Williams tried to fix that, but now old
mappings no longer work. Create a new terminal for the better solution?

Patch to "fix" 'visualbell'. Add option to set delay? (#1789)

Script generated by :mksession does not work well if there are windows with
modified buffers
  change "silent only" into "silent only!"
  change "edit fname" of first buffer to "hide edit fname"
  skip "badd fname" if "fname" is already in the buffer list
  remove remark about unloading buffers from documentation

Compiler warnings (geeknik, 2017 Oct 26):
- signed integer overflow in do_sub() (#2249)
- signed integer overflow in get_address() (#2248)
- signed integer overflow in getdecchrs() (#2254)
- undefined left shift in get_string_tv() (#2250)

Win32 console: <F11> and <F12> typed in Insert mode don't result in normal
characters. (#3246)

'foldtext' is evaluated too often. (Daniel Hahler, #2773)

Add Native language protocol server (LSP) support. (Yegappan Lakshmanan, 2018
Oct 28)

ml_get error when using a Python script. (Yggdroot, 2017 Jun 1, #1737)
Lemonboy can reproduce (2017 Jun 5)
Also reproduced by Benjamin Doherty, 2018 Oct 4.
Simpler way: Ken Takata, Oct 6.

Add more testing of the GTK GUI.
- gtk_test_widget_click() can be used to simulate a click in a widget.

Tests failing for "make testgui" with GTK:
- Test_setbufvar_options()

- Test_exit_callback_interval()

Patch to be able to use hex numbers with :digraph. (Lcd, 2015 Sep 6)
Update Sep 7. Update by Christian Brabandt, 2015 Sep 8, 2016 Feb 1.
Patch to be able to disable default digraphs (incomplete) (Eric Pruitt, 2018
Nov 22).

Patch to list user digraphs. (Christian Brabandt, 2012 Apr 14)

Patch to add digraph() function. (Christian Brabandt, 2013 Aug 22, update Aug
24)

Try out background make plugin:
  https://github.com/AndrewVos/vim-make-background
or asyncmake:
  https://github.com/yegappan/asyncmake

Add a ModeChanged autocommand that has an argument indicating the old and new
mode, as what's returned from mode(). Also used for switching Terminal mode.

Add an option with file patterns, to be used when unloading a buffer: If there
is a match, remove entries for the buffer from marks, jumplist, etc. To be
used for git temp files.

Cursor in wrong position when line wraps. (#2540)

Setting 'columns' in a BufEnter autocommand causes a second tab width to
behave strangely, as if there is a gap and a vertical window separator.
(Michael Soyka, 2018 Sep 23, #3477)

Make {skip} argument of searchpair() consistent with other places where we
pass an expression to evaluate. Allow passing zero for "never skip".

Add an option similar to 'lazyredraw' to skip redrawing while executing a
script or function.

When using a menu item while the "more" prompt is displayed doesn't work well.
E.g. after using help->version. Have a key that ends the "more" prompt and
does nothing otherwise?

MS-Windows: write may fail if another program is reading the file.
If 'readonly' is not set but the file appears to be readonly later, try again
(wait a little while).
CreateFile() returns ERROR_SHARING_VIOLATION (Linwei, 2018 May 5)

Using --remote to open a file in which a # appears does not work on
MS-Windows. Perhaps in \# the \ is seen as a path separator. (Axel Bender,
2017 Feb 9) Can we expand wildcards first and send the path literally to the
receiving Vim? Or make an exception for #, it's not useful remotely.

Should add a test for every command line argument. Check coverage for what is
missing: --nofork, -A , -b, -h, etc.

":au * * command" should not be allowed, only use * for event when listing or

deleting autocmds, not when adding them.

Alternative manpager.vim. (Enno, 2018 Jan 5, #2529)

Delete all the specific stuff for the Borland compiler? (#3374)
Patch in #3377 (Thomas Dziedzic)

With 'foldmethod' "indent" and appending an empty line, what follows isn't included in the existing fold. Deleting the empty line and undo fixes it. (Oleg Koshovetc, 2018 Jul 15, #3214)

Patch to support "xxd -ps". (Erik Auerswald, 2018 May 1)
Lacks a test.

Column number is wrong when using 'linebreak' and 'wrap'. (Keith Smiley, 2018 Jan 15, #2555)

":bufdo e" disabled syntax HL in windows other than the current. (BPJ)

Check argument of systemlist(). (Pavlov)

No maintainer for Vietnamese translations.
No maintainer for Simplified Chinese translations.

Python indenting: alternative way to indent arguments:
http://orchistro.tistory.com/236
Should be supported with a flag.

Starting job with cwd option, when the directory does not exist, gives a confusing error message. (Wang Shidong, 2018 Jan 2, #2519)

Add the debug command line history to viminfo.

Add Makefiles to the runtime/spell directory tree, since nobody uses Aap.
Will have to explain the manual steps (downloading the .aff and .dic files, applying the diff, etc.

Pasting a register in Visual mode cannot be repeated. (Mahmoud Al-Qudsi, 2018 Apr 26, #2849)

User dictionary ~/.vim/spell/lang.utf-8.add not used for spell checking until a word is re-added to it. (Matej Cepl, 2018 Feb 6)

Fold at end of the buffer behaves inconsistently. (James McCoy, 2017 Oct 9)

With foldmethod=syntax and nofoldenable comment highlighting isn't removed. (Marcin Szewczyk, 2017 Apr 26)

Using 'wildignore' also applies to literally entered file name. Also with :drop (remote commands).

Inserting a line in a CompleteDone autocommand may confuse undo. (micbou, 2018 Jun 18, #3027)

```
Implement option_save() and option_restore():
option_restore({list})                                    option_restore()
                Restore options previously saved by option_save().
                When buffer-local options have been saved, this function must
                be called when the same buffer is the current buffer.
                When window-local options have been saved, this function must
                be called when the same window is the current window.
                When in the wrong buffer and/or window an error is given and
                the local options won't be restored.

option_save({list})                                          option_save()
                Saves the options named in {list}.  The returned value can be
                passed to option_restore().  Example:
                        let s:saved_options = option_save([
                            \ 'ignorecase',
                            \ 'iskeyword',
                            \ ])
                         au <buffer> BufLeave *
                            \ call option_restore(s:saved_options)
                The advantage over using  :let  is that global and local
                values are handled and the script ID is restored, so that
                `:verbose set` will show where the option was originally set,
                not where it was restored.
  Alternatively: save and restore ALL options.  Implementation needs to use
        copy-on-write.  Return an ID from option_save(), when
        option_restore(ID) is called give an error if another option_save()
        was called in the meantime, they must be balanced.

"gvim --remote" from a directory with non-word characters changes the current
directory (Paulo Marcel Coelho Arabic, 2017 Oct 30, #2266)
Also see #1689.

crash when removing an element while inside map(). (Nikolai Pavlov, 2018 Feb
17, #2652)

When 'virtualedit' is "all" and 'cursorcolumn' is set, the wrong column may be
highlighted. (van-de-bugger, 2018 Jan 23, #2576)

":file" does not show anything when 'shortmess' contains 'F'. (#3070)

No profile information for function that executes ":quit". (Daniel Hahler,
2017 Dec 26, #2501)

A function on a dictionary is not profiled. (ZyX, 2010 Dec 25)

Add script number to profile?  (#3330 breaks tests).

A function defined locally and lambda's are not easily recognized.
Mention where they were defined somewhere.

Invalid range error when using BufWinLeave for closing terminal.
(Gabriel Barta, 2017 Nov 15, #2339)

ml_get errors with buggy script. (Dominique, 2017 Apr 30)
```

Error in emsg with buggy script. (Dominique, 2017 Apr 30)

Join truncates xml comment. (Dmitrii Tcyganok, 2017 Dec 24, #2494)
Requires 'formatoptions' to include "j". (Gary Johnson, 2017 Dec 24)

Patch to support hunspell. (Matej Cepl, Jan 2018, #2500) Based on older patch in #846)
Doesn't work on Windows yet. Not ready to included, hard coded paths.

Win32 GUI: when running a fast timer, the cursor no longer blinks.
Was reported: cursor blinks in terminal on widows with a timer. (xtal8, #2142)

When a timer is running and typing CTRL-R on the command line, it is not redrawn properly. (xtal8, 2017 Oct 23, #2241)

In an optional package the "after" directory is not scanned?
(Renato Fabbri, 2018 Feb 22)

Patch for Neovim concerning restoring when closing help window. (glacambre neovim #7431)

Patch for improving detecting Ruby on Mac in configure. (Ilya Mikhaltsou, 2017 Nov 21)

When t_Co is changed from termresponse, the OptionSet autocmmand event isn't triggered. Use the code from the end of set_num_option() in set_color_count().

Add another autocommand like TermResponse that is fired for the other terminal responses, such as bg and fg. Use "bg", "fg", "blink", etc. for the name.

When using command line window, CmdlineLeave is triggered without CmdlineEnter. (xtal8, 2017 Oct 30, #2263)
Add some way to get the nested state. Although CmdwinEnter is obviously always nested.

matchit hasn't been maintained for a long time. #955.

Problem with 'delcombine'. (agguser, 2017 Nov 10, #2313)

MS-Windows: buffer completion doesn't work when using backslash (or slash) for a path separator. (xtal8, #2201)

Would be nice for Insert mode completion to highlight the text that was added (and may change when picking another completion).

Test runtime files.
Start with filetype detection: testdir/test_filetype.vim

Window not closed when deleting buffer. (Harm te Hennepe, 2017 Aug 27, #2029)

Add options_default() / options_restore() to set several options to Vim defaults for a plugin. Comments from Zyx, 2017 May 10.

Perhaps use a vimcontext / endvimcontext command block.

After using :noautocmd CursorMoved may still trigger. (Andy Stewart, 2017 Sep 13, #2084). Set old position after the command.

When bracketed paste is used, pasting at the ":append" prompt does not get the line breaks. (Ken Takata, 2017 Aug 22)

The ":move" command does not honor closed folds. (Ryan Lue, #2351)

Patch to fix increment/decrement not working properly when 'virtualedit' is set. (Hirohito Higashi, 2016 Aug 1, #923)

Cannot copy modeless selection when cursor is inside it. (lkintact, #2300)

Test_writefile_fails_conversion failure on Solaris because if different iconv behavior. Skip when "uname" returns "SunOS"? (Pavel Heimlich, #1872)

'tagrelative' is broken in specific situation. (xaizek, 2017 Oct 19, #2221)

The ++ options for the :edit command are also useful on the Vim command line.

When recovering a file, put the swap file name in b:recovered_swapfile. Then a command can delete it.

Overlong utf-8 sequence is displayed wrong. (Harm te Hennepe, 2017 Sep 14, #2089) Patch with possible solution by Bjrn Linse.

The change list index is local to a buffer, but it doesn't make sense using it for another buffer. (lacygoll) Copy w_changelistidx to wininfo_S and back.

X11: Putting more than about 262040 characters of text on the clipboard and pasting it in another Vim doesn't work. (Dominique Pelle, 2008 Aug 21-23)
clip_x11_request_selection_cb() is called with zero value and length.
Also: Get an error message from free() in the process that owns the selection.
Seems to happen when the selection is requested the second time, but before clip_x11_convert_selection_cb() is invoked, thus in X library code.
Kazunobu Kuriyama is working on a proper fix. (2017 Jul 25)

Problem with three-piece comment. (Michael Lee, 2017 May 11, #1696)

Creating a partial with an autoload function is confused about the "self" attribute of the function. For an unknown function assume "self" and make that optiona? (Bjorn Linse, 2017 Aug 5)

Cindent: returning a structure has more indent for the second item.
(Sam Pagenkopf, 2017 Sep 14, #2090)

Patch from Christian Brabandt to preserve upper case marks when wiping out a buffer. (2013 Dec 9)
Also fixes #2166?

Patch to add argument to :cquit. (Thinca, 2014 Oct 12)

Python: After "import vim" error messages only show the first line of the stack trace. (Yggdroot, 2017 Jul 28, #1887)

Profile of a dict function is lost when the dict is deleted. Would it be possible to collect this? (Daniel Hahler, #2350)

When checking if a bufref is valid, also check the buffer number, to catch the case of :bwipe followed by :new.

Patch to skip writing a temp file for diffing if the buffer is equal to the existing file. (Akria Sheng, 2017 Jul 22)
Could also skip writing lines that are the same.

Patch with Files for Latvian language. (Vitolins, 2017 May 3, #1675)

MS-Windows: Opening same file in a second gvim hangs. (Sven Bruggemann, 2017 Jul 4)

Setting 'clipboard' to "unnamed" makes a global command very slow (Daniel Drucker, 2017 May 8).
This was supposed to be fixed, did it break again somehow?
Christian cannot reproduce it.

Using composing char in mapping does not work properly. maparg() shows the wrong thing. (Nikolai Pavlov, 2017 Jul 8, #1827)
Or is this not an actual problem?

Better TeX indent file. (Christian Brabandt, 2017 May 3)

Patch to use a separate code for BS on Windows. (Linwei, #1823)

Use gvimext.dll from the nightly build? (Issue #249)

'synmaxcol' works with bytes instead of screen cells. (Llandon, 2017 May 31, #1736)

Problem with using :cd when remotely editing a file. (Gerd Wachsmuth, 2017 May 8, #1690)

Bogus characters inserted when triggering indent while changing text.
(Vitor Antunes, 2016 Nov 22, #1269)

Using "wviminfo /tmp/viminfo" does not store file marks that Vim knows about, it only works when merging with an existing file. (Shougo, 2017 Jun 19, #1781)

Segmentation fault with complete(). (Lifepillar, 2017 Apr 29, #1668)
Check for "pat" to be NULL in search_for_exact_line()?
How did it get NULL? Comment by Christian, Apr 30.

Is it possible to keep the complete menu open when calling complete()?
(Prabir Shrestha, 2017 May 19, #1713)

Memory leak in test97? The string is actually freed. Weird.

assert_fails() can only check for the first error. Make it possible to have
it catch multiple errors and check all of them.

Add a toolbar in the terminal. Can be global, above all windows, or specific
for one window.

Make maparg() also return the raw rhs, so that it doesn't depend on 'cpo'.
(Brett Stahlman, 2017 May 23)
Even better: add a way to disable a mapping temporarily and re-enable it
later. This is for a sub-mode that is active for a short while (one buffer).
Still need maplist() to find the mappings. What can we use to identify a
mapping? Something unique would be better than the LHS.
Perhaps simpler: actually delete the mappings. Use maplist() to list matching
mappings (with a lhs prefix, like maparg()), mapdelete() to delete,
maprestore() to restore (using the output of maplist()).

Add an argument to :mkvimrc (or add another command) to skip mappings from
plugins (source is a Vim script). No need to put these in a .vimrc, they will
be defined when the plugin is loaded.

Use tb_set(winid, [{'text': 'stop', 'cb': callback, 'hi': 'Green'}])
    tb_highlight(winid, 'ToolBar')
    tb_get(winid)

json_encode(): should convert to utf-8. (Nikolai Pavlov, 2016 Jan 23)
What if there is an invalid character?

Json string with trailing \u should be an error. (Lcd)

import can't be used in define option when include matches too.
(Romain Lafourcade, 2017 Jun 18, #1519)

When session file has name in argument list but the buffer was deleted, the
buffer is not deleted when using the session file. (#1393)
Should add the buffer in hidden state.

Wrong diff highlighting with three files. (2016 Oct 20, #1186)
Also get E749 on exit.
Another example in #1309

When deleting a mark or register, leave a tombstone, so that it's also deleted
when writing viminfo (and the delete was the most recent action). #1339

Suggestion to improve pt-br spell checking. (Marcelo D Montu, 2016 Dec 15,
#1330)

Error in test_startup_utf8 on Solaris. (Danek Duvall, 2016 Aug 17)

Completion for :!cmd shows each match twice. #1435

GTK: When adding a timer from 'balloonexpr' it won't fire, because
g_main_context_iteration() doesn't return. Need to trigger an event when the
timer expires.

Screen update bug related to matchparen. (Chris Heath, 2017 Mar 4, #1532)

Rule to use "^" for statusline does not work if a space is defined with
highlighting for both stl and stlnc. Patch by Ken Hamada (itchyny, 2016 Dec 11)

8 "stl" and "stlnc" in 'fillchars' don't work for multi-byte characters.
   Patch by Christian Wellenbrock, 2013 Jul 5.

Using CTRL-G_U in InsertCharPre causes trouble for redo. (Israel Chauca
Fuentes, 2017 Feb 12, #1470)

Add a "keytrans()" function, which turns the internal byte representation of a
key into a form that can be used for :map. E.g.
```
    let xx = "\<C-Home>"
    echo keytrans(xx)
    <C-Home>
```

Check for errors E704 and E705 only does VAR_FUNC, should also do VAR_PARTIAL.
(Nikolai Pavlov, 2017 Mar 13, #1557)
Make a function to check for function-like type?

Screen updated delayed when using CTRL-O u in Insert mode.
(Barlik, #1191) Perhaps because status message?

Implement optional arguments for functions.
```
    func Foo(start, count = 1 all = 1)
    call Foo(12)
    call Foo(12, all = 0)
    call Foo(12, 15, 0)
```

Add a command to take a range of lines, filter them and put the output
somewhere else. :{range}copy {dest} !cmd

Patch to fix that empty first tab is not in session.
(Hirohito Higashi, 2016 Nov 25, #1282)

Patch to fix escaping of job arguments. (Yasuhiro Matsumoto, 2016 Oct 5)
Update Oct 14: https://gist.github.com/mattn/d47e7d3bfe5ade4be86062b565a4bfca
Update Aug 2017: #1954

The TermResponse event is not triggered when a plugin has set 'eventignore' to
"all". Netrw does this. (Gary Johnson, 2017 Jan 24)
Postpone the event until 'eventignore' is reset.

Expanding /**/ is slow. Idea by Luc Hermitte, 2017 Apr 14.

Once .exe with updated installer is available: Add remark to download page
about /S and /D options (Ken Takata, 2016 Apr 13)
Or point to nightly builds: https://github.com/vim/vim-win32-installer/releases

Problem passing non-UTF-8 strings to Python 3. (Bjrn Linse, 2016 Sep 11,
#1053) With patch, does it work?

":sbr" docs state it respects 'switchbuf', but "vsplit" does not cause a

vertical split. (Haldean Brown, 2017 Mar 1)

Use ADDR_OTHER instead of ADDR_LINES for many more commands.
E.g. all the location list commands use a count.
Add tests for using number larger than number of lines in buffer.

Might be useful to have isreadonly(), like we have islocked().
Avoids exceptions, e.g. when using the b: namespace as a dict.

Patch to make v:shell_error writable. (Christian Brabandt, 2016 Sep 27)
Useful to restore it. Is there another solution?

"ci[" does not look for next [like ci" does look for next ".
(J.F. 2017 Jan 7)

Patch for wrong cursor position on wrapped line, involving breakindent.
(Ozaki Kiichi, 2016 Nov 25)
Does this also fix #1408 ?

'cursorline' and match interfere. (Ozaki Kiichi, 2017 Jun 23, #1792)

Patch for 'cursorlinenr' option. (Ozaki Kiichi, 2016 Nov 30)

When 'completeopt' has "noselect" does not insert a newline. (Lifepillar, 2017 Apr 23, #1653)

Window resizing with 'winfixheight': With a vertical split the height changes anyway. (Tommy allen, 2017 Feb 21, #1502)

Invalid behavior with NULL list. (Nikolai Pavlov, #768)
E.g. deepcopy(test_null_list())

Patch to make it possible to extend a list with itself.
(Nikolai Pavlov, 2016 Sep 23)

Patch to add Zstandard compressed file support. (Nick Terrell, 2016 Oct 24)

On Windows buffer completion sees backslash as escape char instead of path separator. (Toffanim, 2016 Nov 24, #1274)

min() and max() spawn lots of error messages if sorted list/dictionary contains invalid data (Nikolay Pavlov, 2016 Sep 4, #1039)

Should :vmap in matchit.vim be :xmap? (Tony Mechelynck)

Problem with whitespace in errorformat. (Gerd Wachsmuth, 2016 May 15, #807)

Undo problem: "g-" doesn't go back, gets stuck. (Bjrn Linse, 2016 Jul 18)

Add "unicode true" to NSIS installer. Doesn't work with Windows 95, which we no longer support.

sort() is not stable when using numeric/float sort (Nikolay Pavlov, 2016 Sep 4#1038)

+channel:
- Add a in_cb, invoked when the write buffer has become empty. (Matteo Landi)
- Add ch_readlines(): for a channel in NL mode, reads as many lines as are available. Should be more efficient than looping over ch_read() with ch_status() to check for more.
- Add a separate timeout for opening a socket. Currently it's fixed at 50 msec, which is too small for a remote connection. (tverniquet, #2130)
- Problem with stderr on Windows? (Vincent Rischmann, 2016 Aug 31, #1026)
- Writing raw mode to a buffer should still handle NL characters as line breaks. (Dmitry Zotikov, 2017 Aug 16)
- When out_cb executes :sleep, the close_cb may be invoked. (Daniel Hahler, 2016 Dec 11, #1320)
- Implement job-term ?
- Calling a function when receiving a "call" on a channel, using feedkeys() does not work. It does work from a timer. (Qiming Zhao, #3852)
- Channel test fails with Motif. Sometimes kills the X11 server.
- When a message in the queue but there is no callback, drop it after a while? Add timestamp to queued messages and callbacks with ID, remove after a minute. Option to set the droptime.
- Add an option to drop text of very long lines? Default to 1 Mbyte.
- Add remark about undo sync, is there a way to force it?
- When starting a job, have an option to open the server socket, so we know the port, and pass it to the command with --socket-fd {nr}. (Olaf Dabrunz, Feb 9) How to do this on MS-Windows?
- For connection to server, a "keep open" flag would be useful. Retry connecting in the main loop with zero timeout.
- job_start(): run job in a newly opened terminal (not a terminal window).
  With xterm could use -S{pty}.
  Although user could use "xterm -e 'cmd arg'".

正规表达式问题:
- NFA engine can be slow for some patterns. Dominique found out that most time is spent in addstate_here() copying the threads. Instead of copying, let each thread point to the next one (by offset, the list is reallocated). (Dominique Pelle, 2019 Feb 18)
- Old engine: using 'incsearch' /\Zabc does not highlight the "c" if it has a composing character. New engine is OK. (Tony Mechelynck, 2019 May 5)
- When search pattern has the base character both with and without combining character, search fails. E.g. "" in "". (agguser, #2312)
- [:space:] only matches ASCII spaces. Add [:white:] for all space-like characters, esp. including 0xa0. Use character class zero.
- Since 7.4.704 the old regex engine fails to match [[:print:]] in 0xf6. (Manuel Ortega, 2016 Apr 24)
  Test fails on Mac. Avoid using isalpha(), isalnum(), etc? Depends on LC_CTYPE
- The old engine does not find a match for "/\%#=1\(\)\{80}", the new engine matches everywhere.
- Using win_linetabsize() can still be slow. Cache the result, store col and vcol. Reset them when moving to another line.
- Very slow with a long line and Ruby highlighting. (John Whitley, 2014 Dec 4)
- Bug with pattern: '\vblock (\d+)\.\n.*\d+%(\1)@<!\.$'
  (Lech Lorens, 2014 Feb 3)
- Issue 164: freeze on regexp search.

- Ignorecase not handled properly for multi-byte characters. (Axel Bender, 2013 Dec 11)
- Using \@> and \?. (Brett Stahlman, 2013 Dec 21) Remark from Marcin Szamotulski; Remark from Brett 2014 Jan 6 and 7.
- NFA regexp doesn't handle \%<v correctly. (Ingo Karkat, 2014 May 12)
- Does not work with NFA regexp engine:
  \%u, \%x, \%o, \%d followed by a composing character
- Search for \%d0\+ may fail with E363. (Christian Brabandt, 2016 Oct 4)
- \%'[does not work. '%'] does work. (Masaaki Nakamura, 2016 Apr 4)
- Bug relating to back references. (Ingo Karkat, 2014 Jul 24)
- New RE does not give an error for empty group: "\(\)\{2}" (Dominique Pelle, 2015 Feb 7)
- Using back reference before the capturing group sometimes works with the old engine, can we do this with the new engine? E.g. with "/\%(<\1>\)\@<=.*\%(<\/\(\w\+\)>\)\@=" matching text inside HTML tags. This problem is probably the same: "\%(^\1.*$\n\)\@<=\(\d\+\).*$". (guotuofeng, 2015 Jun 22)
- Strange matching with "\(Hello\n\)\@<=A". (Anas Syed, 2015 Feb 12)
- Problem with \v(A)@<=b+\1c. (Issue 334)
- Diff highlighting can be very slow. (Issue 309)
- Using %> for a virtual column has a check based on 'tabsize'. Better would be to cache the result of win_linetabsize(col), storing both col and vcol, and use them to decide whether win_linetabsize() needs to be called. Reset col and vcol when moving to another line.
- this doesn't work: "syntax match ErrorMsg /.\%9l\%>20c\&\%<28c/". Leaving out the \& works. Seems any column check after \& fails.
- Difference between two engines: ".*\zs\/\@>\/" on text "///"
  (Chris Paul, 2016 Nov 13) New engine not greedy enough?
  Another one: echom matchstr(" sdfsfsf\n sfdsdfsdf",'[^\n]*')
  (2017 May 15, #1252)

Feature request: Complete members of a dictionary. (Luc Hermitte, 2017 Jan 4, #1350)

Undo message is not always properly displayed. Patch by Ken Takata, 2013 oct 3. Doesn't work properly according to Yukihiro Nakadaira.
Also see #1635.

Patch for systemlist(), add empty item. (thinca, Sep 30, #1135)
Add an argument to choose binary or non-binary (like readfile()), when omitted use the current behavior.
Include the test.

When 'keywordprg' starts with ":" the argument is still escaped as a shell command argument. (Romain Lafourcade, 2016 Oct 16, #1175)

Idea from Sven: record sequence of keys. Useful to show others what they are doing (look over the shoulder), and also to see what happened.
Probably list of keystrokes, with some annotations for mode changes.
Could store in logfile to be able to analyse it with an external command.
E.g. to see when's the last time a plugin command was used.

execute() cannot be used with command completion. (Daniel Hahler, 2016 Oct 1, #1141)

cmap using execute() has side effects. (Killthemule, 2016 Aug 17, #983)

:map X may print invalid data. (Nikolay Pavlov, 2017 Jul 3, #1816)

Patch to order results from taglist(). (Duncan McDougall, 2016 Oct 25)

When using ":diffput" through a mapping, undo in the target buffer isn't
synced. (Ryan Carney, 2016 Sep 14)

Syntax highlighting for messages with RFC3339 timestamp (#946)
Did maintainer reply?

Patch to avoid problem with special characters in file name.
(Shougo, 2016 Sept 19, #1099) Not finished?

ml_get errors when reloading file. (Chris Desjardins, 2016 Apr 19)
Also with latest version.

Cannot delete a file with square brackets with delete(). (#696)

Patch to add ":syn foldlevel" to use fold level further down the line.
(Brad King, 2016 Oct 19, update 2017 Jan 30)

Completion for input() does not expand environment variables. (chdiza, 2016
Jul 25, #948)

Patch to add 'systemencoding', convert between 'encoding' and this for file
names, shell commands and the like. (Kikuchan, 2010 Oct 14)
Assume the system converts between the actual encoding of the filesystem to
the system encoding (usually utf-8).

Using ":tab drop file" does not trigger BufEnter or TabEnter events.
(Andy Stewart, 2017 Apr 27, #1660)
Autocommands blocked in do_arg_all(). Supposed to happen later?

'hlsearch' interferes with a Conceal match. (Rom Grk, 2016 Aug 9)

MS-Windows: use WS_HIDE instead of SW_SHOWMINNOACTIVE in os_win32.c?
Otherwise task flickers in taskbar.

Should make ":@r" handle line continuation. (Cesar Romani, 2016 Jun 26)
Also for ":@.".

Repeating 'opfunc' in a function only works once. (Tarmean, 2016 Jul 15, #925)

Have a way to get the call stack, in a function and from an exception.
#1125

Second problem in #966: ins_compl_add_tv() uses get_dict_string() multiple
times, overwrites the one buffer. (Nikolay Pavlov, 2016 Aug 5)

Possibly wrong value for seq_cur. (Florent Fayolle, 2016 May 15, #806)

Filetype plugin for awk. (Doug Kearns, 2016 Sep 5)

Patch to improve map documentation. Issue #799.

Patch for syntax folding optimization. (Shougo, 2016 Sep 6, #1045)

We can use '. to go to the last change in the current buffer, but how about
the last change in any buffer? Can we use ', (, is next to .)?

Ramel Eshed: system() is much slower than job_start(), why? (Aug 26)

When generating the Unicode tables with runtime/tools/unicode.vim the
emoji_width table has only one entry.

It's possible to add ",," to 'wildignore', an empty entry. Causes problems.
Reject the value? #710.

When doing "vi buf.md" a BufNew autocommand for *.md is not triggered.
Because of using the initial buffer? (Dun Peal, 2016 May 12)

Add redrawtabline command. (Naruhiko Nishino, 2016 Jun 11)

Neovim patch for utfc_ptr2char_len() https://github.com/neovim/neovim/pull/4574
No test, needs some work to include.

Patch to improve indenting for C++ constructor with initializer list.
(Hirohito Higashi, 2016 Mar 31)

Zero-out krypt key information when no longer in use. (Ben Fritz, 2017 May 15)

Add stronger encryption. Could use libsodium (NaCl).
https://github.com/jedisct1/libsodium/
Possibly include the needed code so that it can be build everywhere.

Add a way to restart a timer. It's similar to timer_stop() and timer_start(),
but the reference remains valid.

Need to try out instructions in INSTALLpc.txt about how to install all
interfaces and how to build Vim with them.
Appveyor build with self-installing executable, includes getting most
interfaces: https://github.com/k-takata/vim/tree/chrisbra-appveyor-build
result: https://ci.appveyor.com/project/k-takata/vim/history

Problem that a previous silent ":throw" causes a following try/catch not to
work. (ZyX, 2013 Sep 28) With examples: (Malcolm Rowe, 2015 Dec 24)

Problem using ":try" inside ":execute". (ZyX, 2013 Sep 15)

Patch to make tests pass with EBCDIC. (Owen Leibman, 2016 Apr 10)

Add ":read :command", to insert the output of an Ex command?
Can already do it with ":$put =execute('command')".

When repeating the 'confirm' dialog one needs to press Enter. (ds26gte, 2016

Apr 17) #762

exists(":tearoff") does not tell you if the command is implemented. (Tony Mechelynck) Perhaps use exists("::tearoff") to check?

Use vim.vim syntax highlighting for help file examples, but without ":" in 'iskeyword' for syntax.

Patch to make "%:h:h" return "." instead of the full path.
(Coot, 2016 Jan 24, #592)

Remove SPACE_IN_FILENAME ? What could possibly go wrong?

When command names are very long :command output is difficult to read. Use a maximum for the column width? (#871)
Patcy by varmanishant, 2016 Jun 18, #876

Installation of .desktop files does not work everywhere.
It's now fixed, but the target directory probably isn't right.
Add configure check?
Should use /usr/local/share/applications or /usr/share/applications.
Or use $XDG_DATA_DIRS.
Also need to run update-desktop-database (Kuriyama Kazunobu, 2015 Nov 4)

Test object i{ and it do not behave the same. #1379
Do not include the linebreak at the start?

Feature request: add the "al" text object, to manipulate a screen line.
Especially useful when using 'linebreak'

":cd C:\Windows\System32\drivers\etc*" does not work, even though the directory exists. (Sergio Gallelli, 2013 Dec 29)

In debug mode one can inspect variables, but not the function parameters (starting with a:). (Luc Hermitte, 2017 Jan 4, #1352)

If ":bd" also closes a Tab page then the " mark is not set. (Harm te Hennepe, 2016 Apr 25, #780)

Patch to avoid redrawing tabline when the popup menu is visible.
(Christian Brabandt, 2016 Jan 28)

Patch to show search statistics. (Christian Brabandt, 2016 Jul 22)

When the CursorMovedI event triggers, and CTRL-X was typed, a script cannot restore the mode properly. (Andrew Stewart, 2016 Apr 20)
Do not trigger the event?

Using ":windo" to set options in all windows has the side effect that it changes the window layout and the current window. Make a variant that saves and restores. Use in the matchparen plugin.
Perhaps we can use ":windo <restore> {cmd}"?
Patch to add <restore> to :windo, :bufdo, etc. (Christian Brabandt, 2015 Jan 6, 2nd message)

Alternative: ":keeppos" command modifier: ":keeppos windo {cmd}".

Patch to fix display of listchars on the cursorline. (Nayuri Aohime, 2013)
Update suggested by Yasuhiro Matsumoto, 2014 Nov 25:
https://gist.github.com/presuku/d3d6b230b9b6dcfc0477

Patch to make the behavior of "w" more straightforward, but not Vi compatible.
With a 'cpo' flag. (Christian Brabandt, 2016 Feb 8)

Patch to add optionproperties(). (Anton Lindqvist, 2016 Mar 27, update Apr 13)

Patch to add TagNotFound autocommand. (Anton Lindqvist, 2016 Feb 3)

Patch to add Error autocommand. (Anton Lindqvist, 2016 Feb 17)
Only remembers one error.

GVim: when both Tab and CTRL-I are mapped, use CTRL-I not for Tab.

Unexpected delay when using CTRL-O u. It's not timeoutlen.
(Gary Johnson, 2015 Aug 28)

Instead of separately uploading patches to the ftp site, we can get them from
github with a URL like this:
   https://github.com/vim/vim/compare/v7.4.920%5E...v7.4.920.diff
Diff for version.c contains more context, can't skip a patch.

Python: ":py raw_input('prompt')" doesn't work. (Manu Hack)

Comparing nested structures with "==" uses a different comparator than when
comparing individual items.
Also, "'' == 0" evaluates to true, which isn't nice.
Add "===" to have a strict comparison (type and value match).
Add "==*" (?) to have a value match, but no automatic conversion, and v:true
equals 1 and 1.0, v:false equals 0 and 0.0.?

Using uninitialized memory. (Dominique Pelle, 2015 Nov 4)

MS-Windows: When editing a file with a leading space, writing it uses the
wrong name. (Aram, 2014 Nov 7) Vim 7.4.

Can't recognize the $ProgramFiles(x86) environment variable. Recognize it
specifically? First try with the parens, then without.

Patch to add :mapgroup, put mappings in a group like augroup.
(Yasuhiro Matsumoto, 2016 Feb 19)

Value returned by virtcol() changes depending on how lines wrap. This is
inconsistent with the documentation.

Value of virtcol() for '[and '] depend on multi-byte character.
(Luchr, #277)

Can we cache the syntax attributes, so that updates for 'relativenumber' and
'cursorline'/'cursorcolumn' are a lot faster? Thus store the attributes

before combining them.

C highlighting: modern C allows: /* comment */ #ifdef
and also line continuation after #include.
I can't recommend it though.

Build with Python on Mac does not always use the right library.
(Kazunobu Kuriyama, 2015 Mar 28)

To support Thai (and other languages) word boundaries, include the ICU
library: http://userguide.icu-project.org/boundaryanalysis

When complete() first argument is before where insert started and 'backspace'
is Vi compatible, the completion fails. (Hirohito Higashi, 2015 Feb 19)

Patch to use two highlight groups for relative numbers. (Shaun Brady, 2016 Jan
30)

MS-Windows: Crash opening very long file name starting with "\\".
(Christian Brock, 2012 Jun 29)

The OptionSet autocommand event is not always triggered. (Rick Howe, 2015 Sep
24): :diffthis, :diffoff.

":set all&" still does not handle all side effects. Centralize handling side
effects for when set by the user, on init and when reset to default.

":tag" does not jump to the right entry of a :tselect. (James Speros, 2015 Oct
9)

The argument for "-S" is not taken literally, the ":so" command expands
wildcards. Add a ":nowild" command modifier? (ZyX, 2015 March 4)

Proposal to make options.txt easier to read. (Arnaud Decara, 2015 Aug 5)
Update Aug 14.

When using --remote-tab on MS-Windows 'encoding' hasn't been initialized yet,
the file name ends up encoded wrong. (Raul Coronado, 2015 Dec 21)

Example in editing.txt uses $HOME with the expectation that it ends in a
slash. For me it does, but perhaps not for everybody. Add a function that
inserts a slash when needed? pathconcat(dir, path) (Thilo Six, 2015 Aug 12)

ml_updatechunk() is slow when retrying for another encoding. (John Little,
2014 Sep 11)

Patch to fix checking global option value when not using it.
(Arnaud Decara, 2015 Jul 23)

When 'showbreak' is set repeating a Visual operation counts the size of the
'showbreak' text as part of the operation. (Axel Bender, 2015 Jul 20)

Patch to add grepfile(). (Scott Prager, 2015 May 26)
Work in progress.

Would be useful to have a treemap() or deepmap() function. Like map() but
when an item is a list or dict would recurse into it.

Patch for global-local options consistency. (Arnaud Decara, 2015 Jul 22)
Is this right?

Patch to make getregtype() return the right size for non-linux systems.
(Yasuhiro Matsumoto, 2014 Jul 8)
Breaks test_eval. Inefficient, can we only compute y_width when needed?

Patch to use different terminal mode settings for system(). (Hayaki Saito)
Does this work for everybody?

Patch for man.vim. (SungHyun Nam, 2015 May 20)
Doesn't work completely (Dominique Orban)

Patch to add a "literal" argument to bufnr(). (Olaf Dabrunz, 2015 Aug 4)

When a session file is created and there are "nofile" buffers, these are not
filled. Need to trigger BufReadCmd autocommands. Also handle deleting the
initial empty buffer better. (ZyX, 2015 March 8)

Extended file attributes lost on write (backupcopy=no). Issue 306.

Patch to add :lockjumps. (Carlo Baldassi, 2015 May 25)
OK to not block marks?

Mixup of highlighting when there is a match and SpellBad. (ZyX, 2015 Jan 1)

Patch on Issue 72: 'autochdir' causes problems for :vimgrep.

When two SIGWINCH arrive very quickly, the second one may be lost.
(Josh Triplett, 2015 Sep 17)

Make comments in the test Makefile silent. (Kartik Agaram, 2014 Sep 24)

Result of systemlist() does not show whether text ended in line break.
(Bjorn Linse, 2014 Nov 27)

When in 'comments' "n:x" follows after three-part comment directly it repeats
any one-character from the previous line. (Kartik Agaram, 2014 Sep 19)

Syntax highlighting slow (hangs) in SASS file. (Niek Bosch, 2013 Aug 21)

Adding "~" to 'cdpath' doesn't work for completion? (Davido, 2013 Aug 19)

Patch: Let rare word highlighting overrule good word highlighting.
(Jakson A. Aquino, 2010 Jul 30, again 2011 Jul 2)

Patch to add digits argument to round(). (Yasuhiro Matsumoto, 2015 Apr 26)

Can assign to s:type when a function s:type has been defined.
Also the other way around: define a function while a variable with that name

was already defined.
(Yasuhiro Matsumoto, 2014 Nov 3)

Patch for ordered dict. (Ozaki Kiichi, 2015 May 7)

Patch to make closed folds line up. (Charles Campbell, 2014 Sep 12)
Remark from Roland Eggner: does it cause crashes? (2014 Dec 12)
Updated patch by Roland Eggner, Dec 16
Updated patch from Charles, 2016 Jul 2

Patch for building a 32bit Vim with 64bit MingW compiler.
(Michael Soyka, 2014 Oct 15)

Patch: On MS-Windows shellescape() may have to triple double quotes.
(Ingo Karkat, 2015 Jan 16)

Patch for glob(), adding slash to normal files. (Ingo Karkat, 2014 Dec 22)

When entering and leaving the preview window autocommands are triggered, but these may not work well. Perhaps set a flag to indicate that the preview window is involved? (John Otter, 2015 Oct 27)

Using "." to repeat an Ex command puts that command in history. Probably should not happen. If the command is the result of a mapping it's not put in history either. (Jacob Niehus, 2014 Nov 2)
Patch from Jacob, Nov 2.

"hi link" does not respect groups with GUI settings only. (Mark Lodato, 2014 Jun 8)

Bug: Autocompleting ":tag/pat" replaces "/pat" with a match but does not insert a space. (Micha Mos, 2014 Nov 7)

No error for missing endwhile. (ZyX, 2014 Mar 20)

Patch to make extend() fail early when it might fail at some point.
(Olaf Dabrunz, 2015 May 2) Makes extend() slower, do we still want it?
Perhaps only the checks that can be done without looping over the dict or arguments.

Problem with transparent and matchgroup. Issue #475

Patch to add :arglocal and :arglists. (Marcin Szamotulski, 2014 Aug 6)

Spell files use a latin single quote. Unicode also has another single quote: 0x2019. (Ron Aaron, 2014 Apr 4)
New OpenOffice spell files support this with ICONV. But they are not compatible with Vim spell files. The old files can no longer be downloaded.

Spell checking: Add a feature to only consider two spaces after a dot to start a new sentence. Don't give the capitalization error when there is one space.

Idea: For a window in the middle (has window above and below it), use right-mouse-drag on the status line to move a window up/down without changing

its height? It's like dragging the status bar above it at the same time.

Patch to add a :domodeline command. (Christian Brabandt, 2014 Oct 21)

This does not give an error: (Andre Sihera, 2014 Mar 21)
    vim -u NONE 1 2 3 -c 'bufdo if 1 | echo 1'
This neither: (ZyX)
    vim -u NONE 1 2 3 -c 'bufdo while 1 | echo 1'

'viewdir' default on MS-Windows is not a good choice, it's a system directory.
Change 'viewdir' to "$HOME/vimfiles/view" and use 'viewdiralt' to also read
from?

Problem with upwards search on Windows (works OK on Linux). (Brett Stahlman,
2014 Jun 8)

Include a plugin manager with Vim? Neobundle seems to be the best currently.
Also Vundle: https://github.com/gmarik/vundle
Long message about this from ZyX, 2014 Mar 23. And following replies.
Also see http://vim-wiki.mawercer.de/wiki/topic/vim%20plugin%20managment.html
User view:
- Support multiple sources, basically any http:// URL. Or a central place that
  will work for everybody (github? redirects from vim.org?).
  Be able to look into the files before deciding to install.
- Be able to try out a plugin and remove it again with (almost) no traces.
- Each plugin needs a "manifest" file that has the version, dependencies
  (including Vim version and features), conflicts, list of files, etc.
  Updater uses that to decide what/how to update.
  Dependencies can use a URL for specific versions, or short name for scripts
  on vim.org.
- Once a plugin is installed it remembers where it came from, updater checks
  there. Can manually update when really needed.
- Must be possible to install for one user. Also system wide?
- Can edit plugin config with Vim. Can temporarily disable a plugin.
- Run the update manually, find latest version and install.
- Be able to download without special tools, must work for 95% of users.
Implementation:
- Avoid the 'runtimepath' getting long. Need some other way to keep each
  plugin separate.
- When installing or updating, first figure out what needs to be done. This
  may involve recursively fetching manifest files for dependencies. Then show
  the user what's going to change and ask for OK.
- Scripts on Vim.org must be able to consist of several files. Is zip format
  sufficient? Upload the manifest? Or refer to a site that has the manifest?
- Best is to fetch individual files or use a Vimball. Reduces dependency on
  tools that might be missing and allows inspection of the files before
  installing.
Out of scope:
- Overview of plugins, ratings, comments, etc. That's another world.
- Development work on plugins (although diff with distributed version would be
  useful).

Setting the spell file in a session only reads the local additions, not the
normal spell file. (Enno Nagel, 2014 Mar 29)

When typing the first character of a command, e.g. "f", then using a menu, the menu item doesn't work. Clear typeahead when using a menu?

Editing an ascii file as ucs-2 or ucs-4 causes display errors.
(ZyX, 2014 Mar 30)

":Next 1 some-arg" does not complain about trailing argument. Also for various other commands. (ZyX, 2014 Mar 30)

Patch to skip sort if no line matches the expression.
(Christian Brabandt, 2014 Jun 25)

VMS: Select() doesn't work properly, typing ESC may hang Vim. Use sys$qiow instead. (Samuel Ferencik, 2013 Sep 28)

Patch for XDG base directory support. (Jean Franois Bignolles, 2014 Mar 4)
Remark on the docs. Should not be a compile time feature. But then what?

Completion of ":e" is ":earlier", should be ":edit". Complete to the matching command instead of doing this alphabetically. (Mikel Jorgensen)

Patch to define macros for hardcoded values. (Elias Diem, 2013 Dec 14)

Several syntax file match "^\s*" which may get underlined if that's in the highlight group. Add a "\zs" after it?

The undo file name can get too long. (Issue 346)
For the path use a hash instead of dir%dir%dir%name hash%name.

Patch to add ":undorecover", get as much text out of the undo file as possible. (Christian Brabandt, 2014 Mar 12, update Aug 22)

Updated spec ftplugin. (Mat ě j Cepl, 2013 Oct 16)

Patch to handle integer overflow. (Aaron Burrow, 2013 Dec 12)

7 Windows XP: When using "ClearType" for text smoothing, a column of yellow
   pixels remains when typing spaces in front of a "D" ('guifont' set to
   "lucida_console:h8").
Patch by Thomas Tuegel, also for GTK, 2013 Nov 24

:help gives example for z?, but it does not work. m? and t? do work.

Discussion about canonicalization of Hebrew. (Ron Aaron, 2011 April 10)

Checking runtime scripts: Thilo Six, 2012 Jun 6.

When evaluating expression in backticks, autoload doesn't work.
(Andy Wokula, 2013 Dec 14)

Using <nr>ifoobar<esc> can slow down Vim. Patch by Christian Brabandt, 2013 Dec 13. Only helps a bit, 10000ii<Esc> is still too slow.

GTK: problem with 'L' in 'guioptions' changing the window width.
(Aaron Cornelius, 2012 Feb 6)

Patch to add option that tells whether small deletes go into the numbered
registers. (Aryeh Leib Taurog, 2013 Nov 18)

Javascript file where indent gets stuck on: GalaxyMaster, 2012 May 3.

The BufUnload event is triggered when re-using the empty buffer.
(Pokey Rule, 2013 Jul 22)
Patch by Marcin Szamotulski, 2013 Jul 22.

The CompleteDone autocommand needs some info passed to it:
- The word that was selected (empty if abandoned complete)
- Type of completion: tag, omnifunc, user func.

Patch to allow more types in remote_expr(). (Lech Lorens, 2014 Jan 5)
Doesn't work for string in list. Other way to pass all types of variables
reliably?

Patch to add {lhs} to :mapclear: clear all maps starting with {lhs}.
(Christian Brabandt, 2013 Dec 9)

Exception caused by argument of return is not caught by try/catch.
(David Barnett, 2013 Nov 19)

Patch to fix that 'cedit' is recognized after :normal. (Christian Brabandt,
2013 Mar 19, later message)

Patch to view coverage of the tests. (Nazri Ramliy, 2013 Feb 15)

Patch to invert characters differently in GTK. (Yukihiro Nakadaira, 2013 May
5)

Patch to add "Q" and "A" responses to interactive :substitute. They are
carried over when using :global. (Christian Brabandt, 2013 Jun 19)

Bug with 'cursorline' in diff mode. Line being scrolled into view gets
highlighted as the cursor line. (Alessandro Ivaldi, 2013 Jun 4)

Two highlighting bugs. (ZyX, 2013 Aug 18)

Patch to support 'u' in interactive substitute. (Christian Brabandt, 2012 Sep
28) With tests: Oct 9.

Dialog is too big on Linux too. (David Fishburn, 2013 Sep 2)

Patch to make fold updates much faster. (Christian Brabandt, 2012 Dec)

-   Add regex for 'paragraphs' and 'sections': 'parare' and 'sectre'. Combine
    the two into a regex for searching. (Ned Konz)
Patch by Christian Brabandt, 2013 Apr 20, unfinished.

Bug: findfile("any", "file:///tmp;") does not work.

In the ATTENTION message about an existing swap file, mention the name of the process that is running. It might actually be some other program, e.g. after a reboot.

patch to add "combine" flag to syntax commands. (so8res, 2012 Dec 6)

Syntax update problem in one buffer opened in two windows, bottom window is not correctly updated. (Paul Harris, 2012 Feb 27)

Patch to add getsid(). (Tyru, 2011 Oct 2) Do we want this? Update Oct 4. Or use expand('<sid>')?

Patch to make confirm() display colors. (Christian Brabandt, 2012 Nov 9)

Problem with refresh:always in completion. (Tyler Wade, 2013 Mar 17)

b:undo_ftplugin cannot call a script-local function. (Boris Danilov, 2013 Jan 7)

Win32: The Python interface only works with one version of Python, selected at compile time. Can this be made to work with version 2.1 and 2.2 dynamically?

Python: Be able to define a Python function that can be called directly from Vim script. Requires converting the arguments and return value, like with vim.bindeval().

Patch for :tabcloseleft, after closing a tab go to left tab. (William Bowers, 2012 Aug 4)

Patch to improve equivalence classes in regexp patterns.
(Christian Brabandt, 2013 Jan 16, update Jan 17)

Patch with suggestions for starting.txt. (Tony Mechelynck, 2012 Oct 24)
But use Gnome instead of GTK?

Should be possible to enable/disable matchparen per window or buffer.
Add a check for b:no_match_paren in Highlight_matching_Pair() (Marcin Szamotulski, 2012 Nov 8)

Session file creation: 'autochdir' causes trouble. Keep it off until after loading all files.

MS-Windows resizing problems:
- Windows window on screen positioning: Patch by Yukihiro Nakadaira, 2012 Jun 20. Uses getWindowRect() instead of GetWindowPlacement()
- Win32: When the taskbar is at the top of the screen creating the tabbar causes the window to move unnecessarily. (William E. Skeith III, 2012 Jan 12) Patch: 2012 Jan 13 Needs more work (2012 Feb 2)

'iminsert' global value set when using ":setlocal iminsert"? (Wu, 2012 Jun 23)

Patch to append regexp to tag commands to make it possible to select one out of many matches. (Cody Cutler, 2013 Mar 28)

The input map for CTRL-O in mswin.vim causes problems after CTRL-X CTRL-O.
Suggestion for another map. (Philip Mat, 2012 Jun 18)
But use "gi" instead of "a". Or use CTRL-\ CTRL-O.

When there are no command line arguments ":next" and ":argu" give E163, which
is confusing. Should say "the argument list is empty".

URXVT:
- will get stuck if byte sequence does not contain the expected semicolon.
- Use urxvt mouse support also in xterm. Explanations:
  http://www.midnight-commander.org/ticket/2662

Patch to have the fold and sign column and at the last line of the buffer.
(Marco Hinz, 2014 Sep 25)
Alternate suggestion: let all columns continue, also the number column.

Patch to add tests for if_xcmdsrv.c., Jul 8, need some more work. (Brian Burns)
New tests Jul 13. Update Jul 17. Discussion Jul 18.

When running Vim in silent ex mode, an existing swapfile causes Vim to wait
for a user action without a prompt. (Maarten Billemont, 2012 Feb 3)
Do give the prompt? Quit with an error?

Patch for input method status. (Hirohito Higashi, 2012 Apr 18)

Update Vim app icon (for Gnome). (Jakub Steiner, 2013 Dec 6)

Patch to use .png icons for the toolbar on MS-Windows. (Martin Gieseking, 2013
Apr 18)

Patch for has('unnamedplus') docs. (Tony Mechelynck, 2011 Sep 27)
And one for gui_x11.txt.

":cd" doesn't work when current directory path contains "**".
finddir() has the same problem. (Yukihiro Nakadaira, 2012 Jan 10)
Requires a rewrite of the file_file_in_path code.

Should use has("browsefilter") in ftplugins. Requires patch 7.3.593.

Update for vim2html.pl. (Tyru, 2013 Feb 22)

Patch to sort functions starting with '<' after others. Omit dict functions,
they can't be called. (Yasuhiro Matsumoto, 2011 Oct 11)

Patch to pass list to or(), and() and xor(). (Yasuhiro Matsumoto, 2012 Feb 8)

Patch to improve "it" and "at" text object matching. (Christian Brabandt, 2011
Nov 20)

Patch to improve GUI find/replace dialog. (Christian Brabandt, 2012 May 26)
Update Jun 2.

`] moves to character after insert, instead of the last inserted character.

(Yukihiro Nakadaira, 2011 Dec 9)

Plugin for Modeleasy. (Massimiliano Tripoli, 2011 Nov 29)

BufWinLeave triggers too late when quitting last window in a tab page. (Lech Lorens, 2012 Feb 21)

Patch for 'transparency' option. (Sergiu Dotenco, 2011 Sep 17)
Only for MS-Windows. No documentation. Do we want this?

Patch to support cursor shape in Cygwin console. (Ben bgold, 2011 Dec 27)

On MS-Windows a temp dir with a & init causes system() to fail. (Ben Fritz, 2012 Jun 19)

'cursorline' is displayed too short when there are concealed characters and 'list' is set. (Dennis Preiser)
Patch 7.3.116 was the wrong solution.
Christian Brabandt has another incomplete patch. (2011 Jul 13)

With concealed text mouse click doesn't put the cursor in the right position. (Herb Sitz) Fix by Christian Brabandt, 2011 Jun 16. Doesn't work properly, need to make the change in where RET_WIN_BUF_CHARTABSIZE() is called.

Syntax region with 'concealends' and a 'cchar' value, 'conceallevel' set to 2, only one of the two ends gets the cchar displayed. (Brett Stahlman, 2010 Aug 21, Ben Fritz, 2010 Sep 14)

Win32: Patch to use task dialogs when available. (Sergiu Dotenco, 2011 Sep 17)
New feature, requires testing. Made some remarks.

Win32: Patch for alpha-blended icons and toolbar height. (Sergiu Dotenco, 2011 Sep 17) Asked for feedback from others.

Win32: Cannot cd into a directory that starts with a space. (Andy Wokula, 2012 Jan 19)

Need to escape $HOME on Windows for fnameescape()? (ZyX, 2011 Jul 21, discussion 2013 Jul 4) Can't simply use a backslash, \$HOME has a different meaning already. Would be possible to use $$HOME where $HOME is to be used.

"2" in 'formatoptions' not working in comments. (Christian Corneliussen, 2011 Oct 26)

Bug in repeating Visual "u". (Lawrence Kesteloot, 2010 Dec 20)

With "unamedplus" in 'clipboard' pasting in Visual mode causes error for empty register. (Michael Seiwald, 2011 Jun 28) I can't reproduce it.

Windows keys not set properly on Windows 7? (cncyber, 2010 Aug 26)

When using a Vim server, a # in the path causes an error message.
(Jeff Lanzarotta, 2011 Feb 17)

When there is a ">" in a line that "gq" wraps to the start of the next line, then the following line will pick it up as a leader. Should get the leader from the first line, not a wrapped line. (Matt Ackeret, 2012 Feb 27)

Using ":break" or something else that stops executing commands inside a ":finally" does not rethrow a previously uncaught exception. (ZyX, 2010 Oct 15)

Vim using lots of memory when joining lines. (John Little, 2010 Dec 3)

BT regexp engine: After trying a \@> match and failing, submatches are not cleared. See test64.

Patch to make "z=" work when 'spell' is off. Does this have nasty side effects? (Christian Brabandt, 2012 Aug 5, Update 2013 Aug 12)
Would also need to do this for spellbadword() and spellsuggest().
https://github.com/chrisbra/vim-mq-patches/blob/master/enable_spellchecking

On 64 bit MS-Windows "long" is only 32 bits, but we sometimes need to store a 64 bits value. Change all number options to use nropt_T and define it to the right type.

string() can't parse back "inf" and "nan". Fix documentation or fix code? (ZyX, 2010 Aug 23)

When doing "redir => s:foo" in a script and then "redir END" somewhere else (e.g. in a function) it can't find s:foo.
When a script contains "redir => s:foo" but doesn't end redirection, a following "redir" command gives an error for not being able to access s:foo. (ZyX, 2011 Mar 27)

When setqflist() uses a filename that triggers a BufReadCmd autocommand Vim doesn't jump to the correct line with :cfirst. (ZyX, 2011 Sep 18)

Behavior of i" and a" text objects isn't logical. (Ben Fritz, 2013 Nov 19)

maparg() does not show the <script> flag. When temporarily changing a mapping, how to restore the script ID?

Bug in try/catch: return with invalid compare throws error that isn't caught. (ZyX, 2011 Jan 26)

When setting a local option value from the global value, add a script ID that indicates this, so that ":verbose set" can give a hint. Check with options in the help file.

After patch 7.3.097 still get E15. (Yukihiro Nakadaira, 2011 Jan 18)
Also for another example (ZyX, 2011 Jan 24)

Build problem with small features on Mac OS X 10.6. (Rainer, 2011 Jan 24)

"0g@$" puts '] on last byte of multi-byte. (ZyX, 2011 Jan 22)

Patch for :tabrecently. (Hirokazu Yoshida, 2012 Jan 30)

Problem with "syn sync grouphere". (Gustavo Niemeyer, 2011 Jan 27)

Loading autoload script even when usage is inside "if 0". (Christian Brabandt, 2010 Dec 18)

With a filler line in diff mode, it isn't displayed in the column with line number, but it is in the sign column. Doesn't look right. (ZyX 2011 Jun 5)
Patch by Christian Brabandt, 2011 Jun 5. Introduces new problems.

Add jump() function. (Marcin Szamotulski, 2013 Aug 29)
Is this needed? CTRL-O and CTRL-I do the same, just more difficult to use.

8 Add a command to jump to the next character highlighted with "Error".
Patch by Christian Brabandt, uses]e [e]t and [t. 2011 Aug 9.

Add event for when the text scrolls. A bit like CursorMoved. Also a similar one for insert mode. Use the event in matchparen to update the highlight if the match scrolls into view.

7 Use "++--", "+++--" for different levels instead of "+---" "+----".
Patch by Christian Brabandt, 2011 Jul 27.
Update by Ben Fritz, with fix for TOhtml. (2011 Jul 30)

9 Add %F to 'errorformat': file name without spaces. Useful on Unix to
avoid matching something up to a time 11:22:33.
Patch by Christian Brabandt, 2011 Jul 27.

Patch to add up to 99 match groups. (Christian Brabandt, 2010 Dec 22)
Also add named groups: \%{name}(re) and \%{name}g

In the sandbox it's not allowed to do many things, but it's possible to change or set variables. Add a way to prevent variables from being changed in the sandbox? E.g.: ":protect g:restore_settings".

GTK: drawing a double-width combining character over single-width characters doesn't look right. (Dominique Pelle, 2010 Aug 8)

GTK: tear-off menu does not work. (Kurt Sonnenmoser, 2010 Oct 25)

Win32: tear-off menu does not work when menu language is German. (Markus Bossler, 2011 Mar 2) Fixed by 7.3.095?

Wish for NetBeans commands:
- make it possible to have 'defineAnnoType' also handle terminal colors.

Version of netbeans.c for use with MacVim. (Kazuki Sakamoto, 2010 Nov 18)

7.3.014 changed how backslash at end of line works, but still get a NUL when there is one backslash. (Ray Frush, 2010 Nov 18) What does the original ex do?

Searching mixed with Visual mode doesn't redraw properly. (James Vega, 2010 Nov 22)

New esperanto spell file can't be processed. (Dominique Pelle, 2011 Jan 30)
- move compflags to separate growarray?
- instead of a regexp use a hashtable. Expand '?', '*', '+'. What would be the maximum repeat for * and +?

"L'Italie" noted as a spell error at start of the sentence. (Dominique Pelle, 2011 Feb 27)

Editing a file with a ^M with 'ff' set to "mac", opening a help file, then the ^M is displayed as ^J sometimes. Getting 'ff' value from wrong window/buffer?

When Vim is put in the background (SIGTSTP) and then gets a SIGHUP it doesn't exit. It exists as soon as back in the foreground. (Stephen Liang, 2011 Jan 9) Caused by vim_handle_signal(SIGNAL_BLOCK); in ui.c.

g` not working correctly when using :edit. It works OK when editing a file on the command line. (Ingo Karkat, 2011 Jan 25)

Since patch 7.2.46 Yankring plugin has become very slow, eventually make Vim crash? (Raiwil, 2010 Nov 17)

Patch to add FoldedLineNr highlighting: different highlighting for the line number of a closed fold. (eXerigumo Clanjor, 2013 Jul 15)

Regexp engine performance:
- Profiling:
```
        ./vim -u NONE -s ~/vim/test/ruby.vim
        ./vim -u NONE -s ~/vim/test/loop.vim
        ./vim -u NONE -s ~/vim/test/alsa.vim
        ./vim -s ~/vim/test/todo.vim
        ./vim -s ~/vim/test/xml.vim
```
    Dominique Pelle: xmlSyncDT is particularly slow (Jun 7)
- More test files from the src/pkg/regexp/testdata directory in the Go repo.
- Performance tests:
  - Using asciidoc syntax. (Marek Schimara, 2013 Jun 6)
  - ~/vim/text/FeiqCfg.xml (file from Netjune)
  - ~/vim/text/edl.svg (also XML)
  - glts has five tests. (May 25)
  - ~/vim/test/slowsearch
  - ~/vim/test/rgb.vim
  - search for a.*e*exn in the vim executable. Go to last line to use 'hlsearch'.
  - Slow combination of folding and PHP syntax highlighting. Script to reproduce it. Caused by "syntax sync fromstart" in combination with patch 7.2.274. (Christian Brabandt, 2010 May 27) Generally, folding with 'foldmethod' set to "syntax" is slow. Do profiling to find out why.

Problem producing tags file when hebrew.frx is present. It has a BOM. Results in E670. (Tony Mechelynck, 2010 May 2)

'beval' option should be global-local.

Ruby: ":ruby print $buffer.number" returns zero.

setpos() does not restore cursor position after :normal. (Tyru, 2010 Aug 11)

With "tw=55 fo+=a" typing space before) doesn't work well. (Scott Mcdermott, 2010 Oct 24)

Messages in message.txt are highlighted as examples.

When using cp850 the NBSP (0xff) is not drawn correctly. (Brett Stahlman, 2010 Oct 22) 'isprint' is set to "@,161-255".

":echo "\x85" =~# '[\u0085]'" returns 1 instead of 0. (ZyX, 2010 Oct 3)

'cindent' not correct when 'list' is set. (Zdravi Korusef, 2010 Apr 15)

C-indenting: A matching { in a comment is ignored, but intermediate { are not checked to be in a comment. Implement FM_SKIPCOMM flag of findmatchlimit(). Issue 46.

Mac with X11: clipboard doesn't work properly. (Raf, 2010 Aug 16)

Using CompilerSet doesn't record where an option was set from. E.g., in the gcc compiler plugin. (Gary Johnson, 2010 Dec 13)

":helpgrep" does not put the cursor in the correct column when preceded by accented character. (Tony Mechelynck, 2010 Apr 15)

Don't call check_restricted() for histadd(), setbufvar(), settabvar(), setwinvar().

Patch for gVimExt to show an icon. (Dominik Riebeling, 2010 Nov 7)

When 'lines' is 25 and 'scrolloff' is 12, "j" scrolls zero or two lines instead of one. (Constantin Pan, 2010 Sep 10)

Gui menu edit/paste in block mode insert only inserts in one line (Bjorn Winckler, 2011 May 11)
Requires a map mode for Insert mode started from blockwise Visual mode.

Writing nested List and Dict in viminfo gives error message and can't be read back. (Yukihiro Nakadaira, 2010 Nov 13)

Problem with cursor in the wrong column. (SungHyun Nam, 2010 Mar 11)
Additional info by Dominique Pelle. (also on 2010 Apr 10)

CreateFile and CreateFileW are used without sharing, filewritable() fails when the file was already open (e.g. script is being sourced). Add FILE_SHARE_READ| FILE_SHARE_WRITE in mch_access()? (Phillippe Vaucher, 2010 Nov 2)

Is ~/bin (literally) in $PATH supposed to work? (Paul, 2010 March 29)
Looks like only bash can do it. (Yakov Lerner)

Cscope "cs add" stopped working somewhat before 7.2.438. (Gary Johnson, 2010 Jun 29) Caused by 7.2.433?

Jumplist doesn't work properly in Insert mode? (Jean Johner, 2010 Mar 20)

Problem with transparent cmdline. Also: Terminal title is wrong with non-ASCII character. (Lily White, 2010 Mar 7)

iconv() doesn't fail on an illegal character, as documented. (Yongwei Wu, 2009 Nov 15, example Nov 26) Add argument to specify whether iconv() should fail or replace with a character and continue?

Add local time at start of --startuptime output.
Requires configure check for localtime().
Use format year-month-day hr:min:sec.

Patch to add "combine" to :syntax, combines highlight attributes. (Nate Soares, 2012 Dec 3)

Patch to make ":hi link" also take arguments. (Nate Soares, 2012 Dec 4)

Shell not recognized properly if it ends in "csh -f". (James Vega, 2009 Nov 3) Find tail? Might have a / in argument. Find space? Might have space in path.

Test 51 fails when language set to German. (Marco, 2011 Jan 9)
Dominique can't reproduce it.

'ambiwidth' should be global-local.

":function f(x) keepjumps" creates a function where every command is executed like it has ":keepjumps" before it.

Coverity: Check if there are new reported defects:
https://scan.coverity.com/projects/241

Patch to support :undo absolute jump to file save number. (Christian Brabandt, 2010 Nov 5)

Patch to use 'foldnestmax' also for "marker" foldmethod. (Arnaud Lacombe, 2011 Jan 7)

Problem with editing file in binary mode. (Ingo Krabbe, 2009 Oct 8)

With 'wildmode' set to "longest:full,full" and pressing Tab once the first entry in wildmenu is highlighted, that shouldn't happen. (Yuki Watanabe, 2011 Feb 12)

Display error when 'tabline' that includes a file name with double-width characters. (2010 Aug 14, bootleq)

Problem with stop directory in findfile(). (Adam Simpkins, 2009 Aug 26)

Using ']' as the end of a range in a pattern requires double escaping:
```
    /[@-\\]]  (Andy Wokula, 2011 Jun 28)
```

Syntax priority problem. (Charles Campbell, 2011 Sep 15)

When completion inserts the first match, it may trigger the line to be folded.
Disable updating folds while completion is active? (Peter Odding, 2010 Jun 9)

When a:base in 'completefunc' starts with a number it's passed as a number,
not a string. (Sean Ma) Need to add flag to call_func_retlist() to force a
string value.

For running gvim on a USB stick: avoid the OLE registration. Use a command
line argument -noregister.

When using an expression in 'statusline' leading white space sometimes goes
missing (but not always). (ZyX, 2010 Nov 1)

When a mapping exists both for insert mode and lang-insert mode, the last one
doesn't work. (Tyru, 2010 May 6) Or is this intended?

Still a problem with ":make" in the wrong directory. Caused by ":bufdo".
(Ajit Thakkar, 2009 Jul 1) More information Jul 9, Jul 15.
Caused by "doautoall syntaxset BufEnter *" in syntax/nosyntax.vim ?
There also is a BufLeave/BufEnter aucmd to save/restore view.
Does the patch to save/restore globaldir work?

":bufdo normal gg" while 'hidden' is set leaves buffers without syntax
highlighting. Don't disable Syntax autocommands then? Or add a flag/modifier
to avoid changing 'eventignore'?

Patch for displaying 0x200c and 0x200d. (Ali Gholami Rudi, 2009 May 6)
Probably needs a bit of work.

List of encoding aliases. (Takao Fujiwara, 2009 Jul 18)
Are they all OK? Update Jul 22.

Win32: Improved Makefile for MSVC. (Leonardo Valeri Manera, 2010 Aug 18)

Win32: Expanding 'path' runs into a maximum size limit. (bgold12, 2009 Nov 15)

Win32: Patch for enabling quick edit mode in console. (Craig Barkhouse, 2010
Sep 1)

Win32: Patch for using .png files for icons. (Charles Peacech, 2012 Feb 5)

Putting a Visual block while 'visualedit' is "all" does not leave the cursor
on the first character. (John Beckett, 2010 Aug 7)

Setting 'tags' to "tagsdir/*" does not find "tagsdir/tags". (Steven K. Wong,
2009 Jul 18)

Patch to add "focusonly" to 'scrollopt', so that scrollbind also applies in
window that doesn't have focus. (Jonathon Mah, 2009 Jan 12)
Needs more work.

Problem with <script> mappings (Andy Wokula, 2009 Mar 8)

When starting Vim with "gvim -f -u non_existent_file > foo.txt" there are a few control characters in the output. (Dale Wiles, 2009 May 28)

'cmdwinheight' is only used in last window when 'winheight' is a large value. (Tony Mechelynck, 2009 Apr 15)

Status line containing winnr() isn't updated when splitting the window (Clark J. Wang, 2009 Mar 31)

When $VIMRUNTIME is set in .vimrc, need to reload lang files. Already done for GTK, how about others? (Ron Aaron, 2010 Apr 10)

Patch for GTK buttons X1Mouse and X2Mouse. (Christian J. Robinson, 2010 Aug 9)

":tab split fname" doesn't set the alternate file in the original window, because win_valid() always returns FALSE. Below win_new_tabpage() in ex_docmd.c.

Space before comma in function definition not allowed: "function x(a , b)" Give a more appropriate error message. Add a remark to the docs.

string_convert() should be able to convert between utf-8 and utf-16le. Used for GTK clipboard. Avoid requirement for iconv.

Now that colnr_T is int instead of unsigned, more type casts can be removed.

'delcombine' does not work for the command line. (Tony Mechelynck, 2009 Jul 20)

Don't load macmap.vim on startup, turn it into a plugin. (Ron Aaron, 2009 Apr 7) Reminder Apr 14.

Add "no_hlsearch" to winsaveview().

Cursorline highlighting combines with Search ('hlsearch') but not with SpellBad. (Jim Karsten, 2009 Mar 18)

When 'foldmethod' is "indent", adding an empty line below a fold and then indented text, creates a new fold instead of joining it with the previous one. (Evan Laforge, 2009 Oct 17)

Bug: When reloading a buffer changed outside of Vim, BufRead autocommands are applied to the wrong buffer/window. (Ben Fritz, 2009 Apr 2, May 11)
Ignore window options when not in the right window?
Perhaps we need to use a hidden window for applying autocommands to a buffer that doesn't have a window.

When using "ab foo bar" and mapping <Tab> to <Esc>, pressing <Tab> after foo doesn't trigger the abbreviation like <Esc> would. (Ramana Kumar, 2009 Sep 6)

getbufvar() to get a window-local option value for a buffer that's not displayed in a window should return the value that's stored for that buffer.

":he ctrl_u" can be auto-corrected to ":he ctrl-u".

There should be a way after an abbreviation has expanded to go back to what was typed. CTRL-G h ? Would also undo last word or line break inserted perhaps. And undo CTRL-W. CTRL-G l would redo.

Diff mode out of sync. (Gary Johnson, 2010 Aug 4)

Win32 GUI: last message from startup doesn't show up when there is an echoerr command. (Cyril Slobin, 2009 Mar 13)

Win32: completion of file name ":e c:\!test" results in ":e c:\\!test", which does not work. (Nieko Maatjes, 2009 Jan 8, Ingo Karkat, 2009 Jan 22)

opening/closing window causes other window with 'winfixheight' to change height. Also happens when there is another window in the frame, if it's not very high. (Yegappan Lakshmanan, 2010 Jul 22, Michael Peeters, 2010 Jul 22)

Directory wrong in session file, caused by ":lcd" in BufEnter autocommand. (Felix Kater, 2009 Mar 3)

Session file generates error upon loading, cause by --remote-silent-tab. (7tommm (ytommm) 2010 Nov 24)

Using ~ works OK on 'a' with composing char, but not on 0x0418 with composing char 0x0301. (Tony Mechelynck, 2009 Mar 4)

Searching for composing char works, but not when inside []. (ZyX, Benjamin R. Haskell, 2010 Aug 24)

This does not work yet: "a\(%C\)" (get composing characters into a submatch).

Inconsistent: starting with $LANG set to es_ES.utf-8 gives Spanish messages, even though locale is not supported. But ":lang messages es_ES.utf-8" gives an error and doesn't switch messages. (Dominique Pelle, 2009 Jan 26)

When $HOME contains special characters, such as a comma, escape them when used in an option. (Michael Hordijk, 2009 May 5)
Turn "esc" argument of expand_env_esc() into string of chars to be escaped.

Should make 'ignorecase' global-local, so that it makes sense setting it from a modeline.

Add cscope target to Makefile. (Tony Mechelynck, 2009 Jun 18, replies by Sergey Khorev)

Consider making YankRing or something else that keeps a list of yanked text part of standard Vim. The "1 to "9 registers are not sufficient.

After doing "su" $HOME can be the old user's home, thus ~root/file is not correct. Don't use it in the swap file.

Completion for ":buf" doesn't work properly on Win32 when 'shellslash' is off.

(Henrik Ohman, 2009, Jan 29)

shellescape() depends on 'shellslash' for quoting. That doesn't work when 'shellslash' is set but using cmd.exe. (Ben Fritz)
Use a different option or let it depend on whether 'shell' looks like a unix-like shell?

Bug: in Ex mode (after "Q") backslash before line break, when yanked into a register and executed, results in <Nul>: instead of line break.
(Konrad Schwarz, 2010 Apr 16)

Have a look at patch for utf-8 line breaking. (Yongwei Wu, 2008 Mar 1, Mar 23)
Now at: http://vimgadgets.sourceforge.net/liblinebreak/

Greek sigma character should be lower cased depending on the context. Can we make this work? (Dominique Pelle, 2009 Sep 24)

When changing 'encoding' convert all the swap file names, so that we can still delete them. Also convert all buffer file names?

"gqip" in Insert mode has an off-by-one error, causing it to reflow text.
(Raul Coronado, 2009 Nov 2)

Something wrong with session that has "cd" commands and "badd", in such a way that Vim doesn't find the edited file in the buffer list, causing the ATTENTION message? (Tony Mechelynck, 2008 Dec 1)
Also: swap files are in ~/tmp/ One has relative file name ".mozilla/...".

MS-Windows: editing the first, empty buffer, 'ffs' set to "unix,dos", ":enew" doesn't set 'ff' to "unix". (Ben Fritz, 2008 Dec 5) Reusing the old buffer probably causes this.

'scrollbind' is not respected when deleting lines or undo. (Milan Vancura, 2009 Jan 16)

Document that default font in Athena can be set with resources:
```
   XtDefaultFont: "9x15"
   XtDefaultFontSet: "9x15"
```
(Richard Sherman, 2009 Apr 12)

Having "Syntax" in 'eventignore' for :bufdo may cause problems, e.g. for ":bufdo e" when buffers are open in windows. ex_listdo(eap) could set the option only for when jumping to another buffer, not when the command argument is executed.

":pedit %" with a BufReadPre autocommand causes the cursor to move to the first line. (Ingo Karkat, 2008 Jul 1) Ian Kelling is working on this.
Similar problem with ":e". (Marc Montu, 2014 Apr 22)

Wildmenu not deleted: "gvim -u NONE", ":set nocp wildmenu cmdheight=3 laststatus=2", CTRL-D CTRL-H CTRL-H CTRL-H. (A.Politz, 2008 April 1)
Works OK with Vim in an xterm.

Cursor line moves in other window when using CTRL-W J that doesn't change

anything. (Dasn, 2009 Apr 7)

On Unix "glob('does not exist~')" returns the string. Without the "~" it doesn't. (John Little, 2008 Nov 9)
Shell expansion returns unexpanded string?
Don't use shell when "~" is not at the start?

When using ":e ++enc=foo file" and the file is already loaded with 'fileencoding' set to "bar", then do_ecmd() uses that buffer, even though the fileencoding differs. Reload the buffer in this situation? Need to check for the buffer to be unmodified.
Unfinished patch by Ian Kelling, 2008 Jul 11. Followup Jul 14, need to have another look at it.

c.vim: XXX in a comment is colored yellow, but not when it's after "#if 0". (Ilya Dogolazky, 2009 Aug 7)

You can type ":w ++bad=x fname", but the ++bad argument is ignored. Give an error message? Or is this easy to implement? (Nathan Stratton Treadway, 2008 Aug 20) This is in ucs2bytes(), search for 0xBF. Using the ++bad argument is at the other match for 0xBF.

When adding "-complete=file" to a user command this also changes how the argument is processed for <f-args>. (Ivan Tishchenko, 2008 Aug 19)

Win32: associating a type with Vim doesn't take care of space after a backslash? (Robert Vibrant, 2008 Jun 5)

When 'rightleft' is set, cursorcolumn isn't highlighted after the end of a line. It's also wrong in folds. (Dominique Pelle, 2010 Aug 21)

Using an insert mode expression mapping, cursor is not in the expected position. (ZyX, 2010 Aug 29)

After using <Tab> for command line completion after ":ta blah" and getting E33 (no tags file), further editing the command to e.g., ":echo 'blah'", the command is not executed. Fix by Ian Kelling?

":help s/~" jumps to *s/\~*, while ":help s/\~" doesn't find anything. (Tim Chase) Fix by Ian Kelling, 2008 Jul 14.

When mapping : to ; and ; to :, @; doesn't work like @: and @: doesn't work either. Matt Wozniski: nv_at() calls do_execreg() which uses put_in_typebuf(). Char mapped twice?

Despite adding save_subexpr() this still doesn't work properly:
Regexp: matchlist('12a4aaa', '^\(.\{-}\)\(\%5c\@<=a\+\)\(.\+\)\?')
Returns ['12a4', 'aaa', '4aaa'], should be ['12a4', 'aaa', '']
Backreference not cleared when retrying after \@<= fails?
(Brett Stahlman, 2008 March 8)

Problem with remote_send(). (Charles Campbell, 2008 Aug 12)

ftplugin for help file should set 'isk' to help file value.

Win32: remote editing fails when the current directory name contains "[".
(Ivan Tishchenko, Liu Yubao) Suggested patch by Chris Lubinski: Avoid
escaping characters where the backslash is not removed later. Asked Chris for
an alternate solution, also for src/ex_getln.c.
This also fails when the file or directory name contains "%". (Thoml, 2008
July 7)
Using --remote-silent while the current directory has a # in the name does not
work, the # needs to be escaped. (Tramblay Bruno, 2012 Sep 15)

When using remote-silent the -R flag is not passed on. (Axel Bender, 2012 May
31)

Win32: A --remote command that has a directory name starting with a (doesn't
work, the backslash is removed, assuming that it escapes the (. (Valery
Kondakoff, 2009 May 13)

Win32: Using "gvim --remote-tab-silent eluti.txt" doesn't work, the
multi-byte character isn't passed and edits elsuti.txt.
(Raúl Núez de Arenas Coronado, 2015 Dec 18)

Problem with 'langmap' being used on the rhs of a mapping. (Nikolai Weibull,
2008 May 14)
Possibly related problem: Alexey Muranov, 2015 Apr 2

Problem with CTRL-F. (Charles Campbell, 2008 March 21)
Only happens with "gvim -geometry "160x26+4+27" -u NONE -U NONE prop.c".
'lines' is 54. (2008 March 27)

Problem with pointer wrapping around in getvcol(). (Wolfgang Kroworsch, 2008
Oct 19) Check for "col" being "MAXCOL" separately?

Unexpectedly inserting a double quote. (Anton Woellert, 2008 Mar 23)
Works OK when 'cmdheight' is 2.

8 Use a mechanism similar to omni completion to figure out the kind of tab
for CTRL-] and jump to the appropriate matching tag (if there are
several).
Alternative: be able to define a function that takes the tag name and uses
taglist() to find the right location. With indication of using CTRL-] so
that the context can be taken into account. (Robert Webb)
Patch by Christian Brabandt, 2013 May 31.

The utf class table is missing some entries:
0x2212, minus sign
0x2217, star
0x2500, bar
0x26ab, circle

Visual line mode doesn't highlight properly when 'showbreak' is used and the
line doesn't fit. (Dasn, 2008 May 1)

GUI: In Normal mode can't yank the modeless selection. Make "gy" do this?
Works like CTRL-Y in Command line mode.

Mac: Move Carbon todo items to os_mac.txt. Note that this version is frozen, try the Cocoa version.

Mac: After a ":vsplit" the left scrollbar doesn't appear until 'columns' is changed or the window is resized.

GTK: when setting 'columns' in a startup script and doing ":vertical diffsplit" the window isn't redrawn properly, see two vertical bars.

Mac: Patch for configure: remove arch from ruby link args. (Knezevic, 2008 Mar 5) Alternative: Kazuki Sakamoto, Mar 7.

Mac: trouble compiling with Motif, requires --disable-darwin. (Raf, 2008 Aug 1) Reply by Ben Schmidt.

C't: On utf-8 system, editing file with umlaut through Gnome results in URL with %nn%nn, which is taken as two characters instead of one.
Try to reproduce at work.

Patch for default choice in file changed dialog. (Bjorn Winckler, 2008 Oct 19)
Is there a way to list all the files first?

When 'smartcase' is set and using CTRL-L to add to the search pattern it may result in no matches. Convert chars to lower case? (Erik Wognsen, 2009 Apr 16)

Fail to edit file after failed register access. Error flag remains set? (Lech Lorens, 2010 Aug 30)

Patch for redo register. (Ben Schmidt, 2007 Oct 19)
Await response to question to make the register writable.

Problem with 'ts' set to 9 and 'showbreak' to ">>>". (Matthew Winn, 2007 Oct 1)

In the swapfile dialog, add a H(elp) option that gives more info about what each choice does. Similar to ":help swap-exists-choices"

try/catch not working for argument of return. (Matt Wozniski, 2008 Sep 15)

try/catch not working when inside a for loop. (ZyX, 2011 Jan 25)

":tab help" always opens a new tab, while ":help" re-uses an existing window. Would be more consistent when an existing tab is re-used. (Tony Mechelynck)

Add ":nofold". Range will apply without expanding to closed fold.

Using Aap to build Vim: add remarks about how to set personal preferences.
Example on http://www.calmar.ws/tmp/aap.html

Syntax highlighting wrong for transparent region. (Doug Kearns, 2007 Feb 26)
Bug in using a transparent syntax region. (Hanlen in vim-dev maillist, 2007 Jul 31)

When 'diffopt' has "context:0" a single deleted line causes two folds to merge and mess up syncing. (Austin Jennings, 2008 Jan 31)

Gnome improvements: Edward Catmur, 2007 Jan 7
    Also use Save/Discard for other GUIs

New PHP syntax file, use it? (Peter Hodge)

":echoe" in catch block stops processing, while this doesn't happen outside of a catch block. (ZyX, 2011 Jun 2)

'foldcolumn' in modeline applied to wrong window when using a session. (Teemu Likonen, March 19)

Test 54 uses shell commands, that doesn't work on non-Unix systems. Use some other way to test buffer-local autocommands.

The documentation mentions the priority for ":2match" and ":3match", but it appears the last one wins. (John Beckett, 2008 Jul 22) Caused by adding matchadd()? Suggested patch by John, 2008 Jul 24.

When 'encoding' is utf-8 the command line is redrawn as a whole on every character typed. (Tyler Spivey, 2008 Sep 3) Only redraw cmdline for 'arabicshape' when there is a character on the command line for which (ARABIC_CHAR(u8c)) is TRUE.

Cheng Fang made javacomplete. (2007 Aug 11)
Asked about latest version: 0.77.1 is on www.vim.org.

More AmigaOS4 patches. (Peter Bengtsson, Nov 9)

Amiga patches with vbcc. (Adrien Destugues, 2010 Aug 30)
http://pulkomandy.ath.cx/drop/vim73_vbcc_amiga.diff

Insert mode completion: When editing the text and pressing CTRL-N again goes back to originally completed text, edited text is gone. (Peng Yu, 2008 Jul 24)
Suggestion by Ben Schmidt, 2008 Aug 6.

Problem with compound words? (Bert, 2008 May 6)
No warning for when flags are defined after they are used in an affix.

Screen redrawing when continuously updating the buffer and resizing the terminal. (Yakov Lerner, 2006 Sept 7)

Add option settings to help ftplugin. (David Eggum, 2006 Dec 18)

Autoconf problem: when checking for iconv library we may add -L/usr/local/lib, but when compiling further tests -liconv is added without the -L argument, that may fail (e.g., sizeof(int)). (Blaine, 2007 Aug 21)

Problem with ".add" files when using two languages and restarting Vim. (Raul Coronado, 2008 Oct 30)

Popup menu redraw: Instead of first redrawing the text and then drawing the popup menu over it, first draw the new popup menu, remember its position and size and then redraw the text, skipping the characters under the popup menu. This should avoid flicker. Other solution by A.Politz, 2007 Aug 22.

When a register contains illegal bytes, writing viminfo in utf-8 and reading it back doesn't result in utf-8. (Devin Bayer)

Command line completion: Scanning for tags doesn't check for typed key now and then? Hangs for about 5 seconds. Appears to be caused by finding include files with "foo/**" in 'path'. (Kalisiak, 2006 July 15)
Additional info: When using the wildcards ** globing, vim hangs indefinitely on lots of directories. The file-searching globing, like in ":set path=/**" does not hang as often as with globing with wildcards , like in ":1find /**/file". This is for files that unix "find" can find very quickly. Merging the 2 kinds of globing might make this an easier fix. (Ian Kelling, 2008 July 4)

When the file name has parenthesis, e.g., "foo (bar).txt", ":!ls '%'" has the parenthesis escaped but not the space. That's inconsistent. Either escape neither or both. No escaping might be best, because it doesn't depend on particularities of the shell. (Zvi Har'El, 2007 Nov 10) (Teemu Likonen, 2008 Jun 3)
However, for backwards compatibility escaping might be necessary. Check if the user put quotes around the expanded item?

A throw in a function causes missing an endif below the call. (Spiros Bousbouras, 2011 May 16)

Error E324 can be given when a cron script has wiped out our temp directory. Give a clear error message about this (and tell them not to wipe out /tmp).

Color for cUserLabel should differ from case label, so that a mistake in a switch list is noticed:

```
    switch (i)
    {
    case 1:
    foobar:
    }
```

Look at http://www.gtk-server.org/ . It has a Vim script implementation.

Netbeans problem. Use "nc -l 127.0.0.1 55555" for the server, then run gvim with "gvim -nb:localhost:55555:foo". From nc do: '1:editFile!0 "foo"'. Then go to Insert mode and add a few lines. Then backspacing every other time moves the cursor instead of deleting. (Chris Kaiser, 2007 Sep 25)

Windows installer could add a "open in new tab of existing Vim" menu entry.
GvimExt: patch to add "Edit with single Vim &tabbed" menu entry.
Just have two choices, always using one Vim and selecting between using an argument list or opening each file in a separate tab.
(Erik Falor, 2008 May 21, 2008 Jun 26)

Windows installer: licence text should not use indent, causes bad word wrap.

(Benjamin Fritz, 2010 Aug 16)

Dos uninstal may delete vim.bat from the wrong directory (e.g., when someone makes his own wrapper). Add a magic string with the version number to the .bat file and check for it in the uninstaller. E.g.
        # uninstall key: vim8.1*

Changes for Win32 makefile. (Mike Williams, 2007 Jan 22, Alexei Alexandrov, 2007 Feb 8)

Win32: Can't complete shell command names. Why is setting xp_context in set_one_cmd_context() inside #ifndef BACKSLASH_IN_FILENAME?

Win32: Patch for cscope external command. (Mike Williams, 2007 Aug 7)

Win32: XPM support only works with path without spaces. Patch by Mathias Michaelis, 2006 Jun 9. Another patch for more path names, 2006 May 31.
New version: http://members.tcnet.ch/michaelis/vim/patches.zip (also for other patches by Mathias, see mail Feb 22)

Win32: compiling with normal features and OLE fails. Patch by Mathias Michaelis, 2006 Jun 4.

Win32: using CTRL-S in Insert mode doesn't remove the "+" from the tab pages label. (Tsakiridis, 2007 Feb 18) Patch from Ian Kelling, 2008 Aug 6.

Win32: using "gvim --remote-tab-silent fname" sometimes gives an empty screen with the more prompt. Caused by setting the guitablabel? (Thomas Michael Engelke, 2007 Dec 20 - 2008 Jan 17)

Win32: patch for fullscreen mode. (Liushaolin, 2008 April 17)

Win32: When 'shell' is bash shellescape() doesn't always do the right thing. Depends on 'shellslash', 'shellquote' and 'shellxquote', but shellescape() only takes 'shellslash' into account.

Menu item that does "xxd -r" doesn't work when 'fileencoding' is utf-16. Check for this and use iconv? (Edward L. Fox, 2007 Sep 12)
Does the conversion in the other direction work when 'fileencodings' is set properly?

Cursor displayed in the wrong position when using 'numberwidth'. (James Vega, 2007 Jun 21)

When $VAR contains a backslash expand('$VAR') removes it. (Teemu Likonen, 2008 Jun 18)

If the variable "g:x#y#z" exists completion after ":echo g:x#" doesn't work.

Feature request: Command to go to previous tab, like what CTRL-W p does for windows. (Adam George)

In debug mode, using CTRL-R = to evaluate a function causes stepping through the function. (Hari Krishna Dara, 2006 Jun 28)

C++ indenting wrong with "=". (James Kanze, 2007 Jan 26)

":lockvar" should use copyID to avoid endless loop.

When using --remote-silent and the file name matches 'wildignore' get an E479 error. without --remote-silent it works fine. (Ben Fritz, 2008 Jun 20)

GVim: dialog for closing Vim should check if Vim is busy writing a file. Then use a different dialog: "busy saving, really quit? yes / no".

Check other interfaces for changing curbuf in a wrong way. Patch like for if_ruby.c.

":helpgrep" should use the directory from 'helpfile'.

The need_fileinfo flag is messy. Instead make the message right away and put it in keep_msg?

Editing a file remotely that matches 'wildignore' results in a "no match" error. Should only happen when there are wildcards, not when giving the file name literally, and esp. if there is only one name.

Test 61 fails sometimes. This is a timing problem: "sleep 2" sometimes takes longer than 2 seconds.

Using ":au CursorMoved * cmd" invokes mch_FullName(), which can be slow.
Can this be avoided? (Thomas Waba, 2008 Aug 24)
Also for ":w" without a file name.
The buffer has the full path in ffname, should pass this to the autocommand.

"vim -C" often has 'nocompatible', because it's set in some startup script.
Set 'compatible' after startup is done? Patch by James Vega, 2008 Feb 7.

VMS: while editing a file found in complex, Vim will save file into the first directory of the path and not to the original location of the file.
(Zoltan Arpadffy)

VMS: VFC files are in some cases truncated during reading (Zoltan Arpadffy)

input() completion should not insert a backslash to escape a space in a file name?

Ruby completion is insecure. Can this be fixed?

When 'backupskip' is set from $TEMP special characters need to be escaped.
(patch by Grembowietz, 2007 Feb 26, not quite right)
Another problem is that file_pat_to_reg_pat() doesn't recognize "\\", so "\\(" will be seen as a path separator plus "\(".

gvim d:\path\path\(FILE).xml should not remove the \ before the (.
This also fails with --remote.

When doing ":quit" the Netbeans "killed" event isn't sent. (Xavier de Gaye,

2008 Nov 10) call netbeans_file_closed() at the end of buf_freeall(), or in all places where buf_freeall() is called?

aucmd_prepbuf() should also use a window in another tab page.

When unloading a buffer in a BufHidden autocommand the hidden flag is reset? (Bob Hiestand, 2008 Aug 26, Aug 27)

Substituting an area with a line break with almost the same area does change the Visual area. Can this be fixed? (James Vega, 2006 Sept 15)

GUI: When combining fg en bg make sure they are not equal.

Spell checking: Add a way to specify punctuation characters. Add the superscript numbers by default: 0x2070, 0xb9, 0xb2, 0xb3, 0x2074 - 0x2079.

Spell checking in popup menu: If the only problem is the case of the first character, don't offer "ignore" and "add to word list".

Use different pt_br dictionary for spell checking. (Jackson A. Aquino, 2006 Jun 5)

Use different romanian dictionary for spell checking. (Andrei Popescu, Nov 2008) Use http://downloads.sourceforge.net/rospell/ro_RO.3.2.zip
Or the hunspell-ro.3.2.tar.gz file, it also has a iso-8859-2 list.

In a C file with spell checking, in "% integer" "nteger" is seen as an error, but "]s" doesn't find it. "nteger" by itself is found. (Ralf Wildenhues, 2008 Jul 22)

There should be something about spell checking in the user manual.

Spell menu: When using the Popup menu to select a replacement word, ":spellrepeat" doesn't work. SpellReplace() uses setline(). Can it use "z=" somehow? Or use a new function.

Mac: Using gvim: netrw window disappears. (Nick Lo, 2006 Jun 21)

Add an option to specify the character to use when a double-width character is moved to the next line. Default '>', set to a space to blank it out. Check that char is single width when it's set (compare with 'listchars').

The generated vim.bat can avoid the loop for NT. (Carl Zmola, 2006 Sep 3)

When showing a diff between a non-existent file and an existing one, with the cursor in the empty buffer, the other buffer only shows the last line. Change the "insert" into a change from one line to many? (Yakov Lerner, 2008 May 27)

These two abbreviations don't give the same result:

```
        let asdfasdf = "xyz\<Left>"
        cabbr XXX <C-R>=asdfasdf<CR>
        cabbr YYY xyz<Left>
```

Michael Dietrich: maximized gvim sometimes displays output of external command

partly. (2006 Dec 7)

In FileChangedShell command it's no longer allowed to switch to another buffer. But the changed buffer may differ from the current buffer, how to reload it then?

For Aap: include a config.arg.example file with hints how to use config.arg.

Command line completion when 'cmdheight' is maximum and 'wildmenu' is set, only one buffer line displayed, causes display errors.

Completing with 'wildmenu' and using <Up> and <Down> to move through directory tree stops unexpectedly when using ":cd " and entering a directory that doesn't contain other directories.

Default for 'background' is wrong when using xterm with 256 colors.
Table with estimates from Matteo Cavalleri, 2014 Jan 10.

Setting 'background' resets the Normal background color:
```
   highlight Normal ctermbg=DarkGray
   set background=dark
```
This is undesired, 'background' is supposed to tell Vim what the background color is, not reset it.

Completion menu: For a wrapping line, completing a long file name, only the start of the path is shown in the menu. Should move the menu to the right to show more text of the completions. Shorten the items that don't fit in the middle?

Accessing file#var in a function should not need the g: prepended.

When exiting detects a modified buffer, instead of opening the buffer in the current tab, use an existing tab, if possible. Like finding a window where the buffer is displayed. (Antonios Tsakiridis)

When ":cn" moves to an error in the same line the message isn't shortened. Only skip shortening for ":cc"?

Problem with ":call" and dictionary function. Hari Krishna Dara, Charles Campbell 2006 Jul 06.

Syntax HL error caused by "containedin". (Peter Hodge, 2006 Oct 6)

A custom completion function in a ":command" cannot be a Funcref. (Andy Wokula, 2007 Aug 25)

Problem with using :redir in user command completion function? (Hari Krishna Dara, 2006 June 21)

Another resizing problem when setting 'columns' and 'lines' to a very large number. (Tony Mechelynck, 2007 Feb 6)

After starting Vim, using 'O to jump somewhere in a file, ":sp" doesn't center the cursor line. It works OK after some other commands.

Win32: Is it possible to have both postscript and Win32 printing?

Problem with 'cdpath' on MS-Windows when a directory is equal to $HOME. (2006 Jul 26, Gary Johnson)

Using UTF-8 character with ":command" does not work properly. (Matt Wozniski, 2008 Sep 29)

In the Netbeans interface add a "vimeval" function, so that the other side can check the result of has("patch13").

Cursor line at bottom of window instead of halfway after saving view and restoring. Only with 'nowrap'. (Robert Webb, 2008 Aug 25)

Netrw has trouble executing autocommands only for a directory. Add <isdir> and <notisdir> to autocommand patterns? Also <isfile>?

Add command modifier that skips wildcard expansion, so that you don't need to put backslashes before special chars, only for white space.

Syntax HL: open two windows on the same C code, delete a ")" in one window, resulting in highlighted "{" in that window, not in the other.

In mswin.vim: Instead of mapping <C-V> for Insert mode in a complicated way, can it be done like ":imap <C-V> <MiddleMouse>" without negative side effects?

GTK: when the Tab pages bar appears or disappears while the window is maximized the window is no longer maximized. Patch that has some idea but doesn't work from Geoffrey Antos, 2008 May 5.
Also: the window may no longer fit on the screen, thus the command line is not visible.

When right after "vim file", "M" then CTRL-W v the windows are scrolled differently and unexpectedly. Caused by patch 7.2.398?

The magic clipboard format "VimClipboard2" appears in several places. Should be only one.

Win32, NTFS: When editing a specific infostream directly and 'backupcopy' is "auto" should detect this situation and work like 'backupcopy' is "yes". File name is something like "c:\path\foo.txt:bar", includes a colon. (Alex Jakushev, 2008 Feb 1)

Small problem displaying diff filler line when opening windows with a script. (David Luyer, 2007 Mar 1 ~/Mail/oldmail/mool/in.15872)

Is it allowed that 'backupext' is empty? Problems when backup is in same dir as original file? If it's OK don't compare with 'patchmode'. (Thierry Closen)

Patch for adding ":lscscope". (Navdeep Parhar, 2007 Apr 26; update 2008 Apr 23)

":mkview" isn't called with the right buffer argument. Happens when using

tabs and the autocommand "autocmd BufWinLeave * mkview". (James Vega, 2007 Jun 18)

When completing from another file that uses a different encoding completion text has the wrong encoding. E.g., when 'encoding' is utf-8 and file is latin1. Example from Gombault Damien, 2007 Mar 24.

Syntax HL: When using "nextgroup" and the group has an empty match, there is no search at that position for another match. (Lukas Mai, 2008 April 11)

In gvim the backspace key produces a backspace character, but on Linux the VERASE key is Delete. Set VERASE to Backspace? (patch by Stephane Chazelas, 2007 Oct 16)

TermResponse autocommand isn't always triggered when using vimdiff. (Aron Griffis, 2007 Sep 19)

Create a gvimtutor.1 file and change Makefiles to install it.

When 'encoding' is utf-8 typing text at the end of the line causes previously typed characters to be redrawn. Caused by patch 7.1.329. (Tyler Spivey, 2008 Sep 3, 11)

":vimgrep" does not recognize a recursive symlink. Is it possible to detect this, at least for Unix (using device/inode)?

When switching between windows the cursor is often put in the middle. Remember the relative position and restore that, just like lnum and col are restored. (Luc St-Louis)

Patch to support horizontal scroll wheel in GTK. Untested. (Bjorn Winckler, 2010 Jun 30)

Add an option for a minimal text length before inserting a line break for 'textwidth'. Avoids very short lines when a very long word follows. (Kartik Agaram)

更好的插件支持（不是插件管理器，去别处找吧）：
- Avoid use of feedkeys, add eval functions where needed:
  - manipulating the Visual selection?
- Add createmark(): add a mark like mM, but return a unique ID. Need some way to clean them up again... Use a name + the script ID.
  Add createmark(, 'c') to track inserts/deletes before the column.
- Plugins need to make a lot of effort, lots of mappings, to know what happened before pressing the key that triggers a plugin action. How about keeping the last N pressed keys, so that they do not need to be mapped?
- equivalent of netbeans_beval_cb(). With an autocommand?
- Add something to enable debugging when a remote message is received.

更多补丁：
- Another patch for Javascript indenting. (Hari Kumar, 2010 Jul 11)
  Needs a few tests.

- Add 'cscopeignorecase' option. (Liang Wenzhi, 2006 Sept 3)
- Extra argument to strtrans() to translate special keys to their name (Eric Arnold, 2006 May 22)
- Mac: indicate whether a buffer was modified. (Nicolas Weber, 2006 Jun 30)
- Allow negative 'nrwidth' for left aligning. (Nathan Laredo, 2006 Aug 16)
- ml_append_string(): efficiently append to an existing line. (Brad Beveridge, 2006 Aug 26) Use in some situations, e.g., when pasting a character at a time?
- recognize hex numbers better. (Mark Manning, 2006 Sep 13)
- Add <AbbrExpand> key, to expand an abbreviation in a mapping. (Kana Natsuno, 2008 Jul 17)
- Add 'wspara' option, also accept blank lines like empty lines for "{" and "}". (Mark Lundquist, 2008 Jul 18)
- Patch to add CTRL-T to delete part of a path on cmdline. (Adek, 2008 Jul 21)
- Instead of creating a copy of the tutor in all the shell scripts, do it in vimtutor.vim. (Jan Minar, 2008 Jul 20)
- When fsync() fails there is no hint about what went wrong. Patch by Ben Schmidt, 2008 Jul 22.
- testdir/Make_dos_sh.mak for running tests with MingW. (Bill Mccarthy, 2008 Sep 13)
- Replace ccomplete.vim by cppcomplete.vim from www.vim.org? script 1520 by Vissale Neang. (Martin Stubenschrott) Asked Vissale to make the scripts more friendly for the Vim distribution.
  New version received 2008 Jan 6.
  No maintenance in two years...
- Patch to open dropped files in new tabs. (Michael Trim, 2010 Aug 3)

等待补丁信息的更新:

9 Mac unicode patch (Da Woon Jung, Eckehard Berns):
  8 Add patch from Muraoka Taro (Mar 16) to support input method on Mac?
    New patch 2004 Jun 16
  - selecting proportional font breaks display
  - UTF-8 text causes display problems. Font replacement causes this.
  - Command-key mappings do not work. (Alan Schmitt)
  - With 'nopaste' pasting is wrong, with 'paste' Command-V doesn't work. (Alan Schmitt)
  - remove 'macatsui' option when this has been fixed.
  - when 'macatsui' is off should we always convert to "macroman" and ignore 'termencoding'?
9 HTML indenting can be slow. Caused by using searchpair(). Can search() be used instead? A.Politz is looking into a solution.
8 Win32: Add minidump generation. (George Reilly, 2006 Apr 24)
7 Completion of network shares, patch by Yasuhiro Matsumoto.
  Update 2004 Sep 6.
  How does this work? Missing comments.
8 Add a few more command names to the menus. Patch from Jiri Brezina (28 feb 2002). Will mess the translations...
7 ATTENTION dialog choices are more logical when "Delete it" appears before "Quit". Patch by Robert Webb, 2004 May 3.
- Include flipcase patch: ~/vim/patches/wall.flipcase2 ? Make it work for multi-byte characters.
- Win32: add options to print dialog. Patch from Vipin Aravind.
- Patch to add highlighting for whitespace. (Tom Schumm, 2003 Jul 5)

use the patch that keeps using HLF_8 if HLF_WS has not
been given values.
Add section in help files for these highlight groups?
7 Add "DefaultFG" and "DefaultBG" for the colors of the menu. (Marcin
Dalecki has a patch for Motif and Carbon)
- Add possibility to highlight specific columns (for Fortran). Or put a
line in between columns (e.g., for 'textwidth').
Patch to add 'hlcolumn' from Vit Stradal, 2004 May 20.
8 Add functions:
gettext() Translate a message. (Patch from Yasuhiro Matsumoto)
Update 2004 Sep 10
Another patch from Edward L. Fox (2005 Nov 24)
Search in 'runtimepath'?
More docs needed about how to use this.
How to get the messages into the .po files?
confirm() add "flags" argument, with 'v' for vertical
layout and 'c' for console dialog. (Haegg)
Flemming Madsen has a patch for the 'c' flag
(2003 May 13)
raisewin() raise gvim window (see HierAssist patch for
Tcl implementation ~/vim/HierAssist/)
taglist() add argument to specify maximum number of matches.
useful for interactive things or completion.
col('^') column of first non-white character.
Can use "len(substitute(getline('.'), '\S.*', '', ''))
+ 1", but that's ugly.
7 Add patch from Benoit Cerrina to integrate Vim and Perl functions
better. Now also works for Ruby (2001 Nov 10)
7 When 'rightleft' is set, the search pattern should be displayed right
to left as well? See patch of Dec 26. (Nadim Shaikli)
8 Option to lock all used memory so that it doesn't get swapped to disk
(uncrypted). Patch by Jason Holt, 2003 May 23. Uses mlock.
7 Add ! register, for shell commands. (patch from Grenie)
8 In the gzip plugin, also recognize *.gz.orig, *.gz.bak, etc. Like it's
done for filetype detection. Patch from Walter Briscoe, 2003 Jul 1.
7 Add a "-@ filelist" argument: read file names from a file. (David
Kotchan has a patch for it)
7 Add ":justify" command. Patch from Vit Stradal 2002 Nov 25.
- findmatch() should be adjusted for Lisp. See remark at
get_lisp_indent(). Esp. \(and \) should be skipped. (Dorai Sitaram,
incomplete patch Mar 18)
- For GUI Find/Replace dialog support using a regexp. Patch for Motif
and GTK by degreneir (nov 10 and nov 18).
- Patch for "paranoid mode" by Kevin Collins, March 7. Needs much more work.

更好的 'rightleft' 或 BIDI 支持:
- Minimal Vi with bidi support: https://github.com/aligrudi/neatvi
By Ali Gholami Rudi, also worked on arabic.c

Quickfix/位置列表:
- Window size is wrong when using quickfix window. (Lifepillar, 2018 Aug 24,
#2999)
- When using CTRL-W CR in the quickfix window, the jumplist in the opened
window is cleared, to avoid going back to the list of errors buffer (would

have two windows with it). Can we just remove the jump list entries for the quickfix buffer?

- Quickfix window height is not kept with a vertical split. (Lifepillar, 2018 Jun 10, #2998)
- When an item in the quickfix list has a file name that does not exist, behave like the item was not a match for :cnext.
- When adding an item to a new quickfix list make ":cnext" jump to that item. Make a difference being at the first item and not having used :cnext at all. (Afanasiy Fet, 2017 Jan 3)
- This does not work: :set cscopequickfix=a- (Linewi, 2015 Jul 12, #914)
- When opening quickfix window, disable spell checking?
- Patch for supporting count before CR in quickfix window. (AOYAMA Shotaro, 2007 Jan 1)
- Patch from Herculano de Lima Einloft Neto for better formatting of the quickfix window (2004 dec 2)
- Amiga: When using quickfix with the Manx compiler we only get the first 25 errors. How do we get the rest?

8 The quickfix file is read without conversion, thus in 'encoding'. Add an option to specify the encoding of the errorfile and convert it. Also for ":grep" and ":helpgrep".
More generic solution: support a filter (e.g., by calling a function).

7 Add a command that goes back to the position from before jumping to the first quickfix location. ":cbefore"?

Vi 不兼容性:

- Try new POSIX tests, made after my comments. (Geoff Clare, 2005 April 7) Version 1.5 is in ~/src/posix/1.5. (Lynne Canal)

8 With undo/redo only marks in the changed lines should be changed. Other marks should be kept. Vi keeps each mark at the same text, even when it is deleted or restored. (Webb)
Also: A mark is lost after: make change, undo, redo and undo.
Example: "{d''" then "u" then "d''": deletes an extra line, because the '' position is one line down. (Veselinovic)

8 When stdin is not a tty, and Vim reads commands from it, an error should make Vim exit.

7 Unix Vim (not gvim): Typing CTRL-C in Ex mode should finish the line (currently you can continue typing, but it's truncated later anyway). Requires a way to make CTRL-C interrupt select() when in cooked input.

8 When loading a file in the .exrc, Vi loads the argument anyway. Vim skips loading the argument if there is a file already. When no file argument given, Vi starts with an empty buffer, Vim keeps the loaded file. (Bearded)

6 In Insert mode, when using <BS> or <Del>, don't wipe out the text, but only move back the cursor. Behaves like '$' in 'cpoptions'. Use a flag in 'cpoptions' to switch this on/off.

8 When editing a file which is a symbolic link, and then opening another symbolic link on the same file, Vim uses the name of the first one. Adjust the file name in the buffer to the last one used? Use several file names in one buffer???
Also: When first editing file "test", which is symlink to "test2", and then editing "test2", you end up editing buffer "test" again. It's not logical that the name that was first used sticks with the buffer.

7 The ":undo" command works differently in Ex mode. Edit a file, make some changes, "Q", "undo" and _all_ changes are undone, like the ":visual"

command was one command.
On the other hand, an ":undo" command in an Ex script only undoes the last change (e.g., use two :append commands, then :undo).

7 The ":map" command output overwrites the command. Perhaps it should keep the ":map" when it's used without arguments?

7 CTRL-L is not the end of a section? It is for Posix! Make it an option.

7 Implement 'prompt' option. Init to off when stdin is not a tty.

7 Add a way to send an email for a crashed edit session. Create a file when making changes (containing name of the swap file), delete it when writing the file. Supply a program that can check for crashed sessions (either all, for a system startup, or for one user, for in a .login file).

7 Vi doesn't do autoindenting when input is not from a tty (in Ex mode).

7 "z3<CR>" should still use the whole window, but only redisplay 3 lines.

7 ":tag xx" should move the cursor to the first non-blank. Or should it go to the match with the tag? Option?

7 Implement 'autoprint'/'ap' option.

7 Add flag in 'cpoptions' that makes <BS> after a count work like <Del> (Sayre).

7 Add flag in 'cpoptions' that makes operator (yank, filter) not move the cursor, at least when cancelled. (default Vi compatible).

7 This Vi-trick doesn't work: "Q" to go to Ex mode, then "g/pattern/visual". In Vi you can edit in visual mode, and when doing "Q" you jump to the next match. Nvi can do it too.

7 Support '\' for line continuation in Ex mode for these commands: (Luebking)

```
    g/./a\              g/pattern1/ s/pattern2/rep1\\
    line 1\             line 2\\
    line 2\             line 3\\
    .                   line4/
```

6 ":e /tmp/$tty" doesn't work. ":e $uid" does. Is $tty not set because of the way the shell is started?

6 Vi compatibility (optional): make "ia<CR><ESC>10." do the same strange thing. (only repeat insert for the first line).

GTK+ GUI 已知的漏洞:

9 Crash with X command server over ssh. (Ciaran McCreesh, 2006 Feb 6)

8 GTK 2: Combining UTF-8 characters not displayed properly in menus (Mikolaj Machowski) They are displayed as separate characters. Problem in creating a label?

8 GTK 2: Combining UTF-8 characters are sometimes not drawn properly. Depends on the font size, "monospace 13" has the problem. Vim seems to do everything right, must be a GTK bug. Is there a way to work around it?

9 Can't paste a Visual selection from GTK-gvim to vim in xterm or Motif gvim when it is longer than 4000 characters. Works OK from gvim to gvim and vim to vim. Pasting through xterm (using the shift key) also works. It starts working after GTK gvim loses the selection and gains it again.

- Gnome2: When moving the toolbar out of the dock, so that it becomes floating, it can no longer be moved. Therefore making it float has been blocked for now.

Win32 GUI 已知的漏洞:

- Win32: tearoff menu window should have a scrollbar when it's taller than the screen.

8 The -P argument doesn't work very well with many MDI applications.
The last argument of CreateWindowEx() should be used, see MSDN docs.
Tutorial: http://win32assembly.online.fr/tut32.html
6 Win32 GUI: With "-u NONE -U NONE" and doing "CTRL-W v" "CTRL-W o", the ":"
of ":only" is highlighted like the cursor. (Lipelis)
8 When 'encoding' is "utf-8", should use 'guifont' for both normal and wide
characters to make Asian languages work. Win32 fonts contain both
type of characters.
7 When font smoothing is enabled, redrawing can become very slow. The reason
appears to be drawing with a transparent background. Would it be possible
to use an opaque background in most places?
7 The cursor color indicating IME mode doesn't work properly. (Shizhu Pan,
2004 May 9)
8 Win32: When clicking on the gvim title bar, which gives it focus, produces
a file-changed dialog, after clicking on a button in that dialog the gvim
window follows the mouse. The button-up event is lost. Only with
MS-Windows 98?
Try this: ":set sw ts", get enter-prompt, then change the file in a
console, go back to Vim and click "reload" in the dialog for the changed
file: Window moves with the cursor!
Put focus event in input buffer and let generic Vim code handle it?
8 Win32 GUI: With maximized window, ":set go-=r" doesn't use the space that
comes available. (Poucet) It works OK on Win 98 but doesn't work on Win
NT 4.0. Leaves a grey area where the scrollbar was. ":set go+=r" also
doesn't work properly.
8 When Vim is minimized and when maximizing it a file-changed dialog pops
up, Vim isn't maximized. It should be done before the dialog, so that it
appears in the right position. (Webb)
9 When selecting at the more-prompt or hit-enter-prompt, the right mouse
button doesn't give popup menu.
At the hit-enter prompt CTRL-Y doesn't work to copy the modeless
selection.
On the command line, don't get a popup menu for the right mouse button.
Let the middle button paste selected text (not the clipboard but the
non-Visual selection)? Otherwise CTRL-Y has to be used to copy the text.
8 When 'grepprg' doesn't execute, the error only flashes by, the
user can hardly see what is wrong. (Moore)
Could use vimrun with an "-nowait" argument to only wait when an error
occurs, but "command.com" doesn't return an error code.
8 When the 'shell' cannot be executed, should give an appropriate error msg.
Esp. for a filter command, currently it only complains the file could not
be read.
7 At the hit-enter prompt scrolling now no longer works. Need to use the
keyboard to get around this. Pretend <CR> was hit when the user tries to
scroll?
7 Scrollbar width doesn't change when selecting other windows appearance.
Also background color of Toolbar and rectangle below vert. scrollbar.
6 Drawing text transparently doesn't seem to work (when drawing part cursor).
8 CTRL key doesn't always work in combination with ALT key. It does work
for function keys, not for alphabetic characters. Perhaps this is because
CTRL-ALT is used by Windows as AltGr?
8 CTRL-- doesn't work for AZERTY, because it's CTRL-[for QWERTY. How do we
know which keyboard is being used?
7 When scrolling, and a background color is dithered, the dither pattern

doesn't always join correctly between the scrolled area and the new drawn area (Koloseike).

8 When gui_init_font() is called with "*", p_guifont is freed while it might still be used somewhere. This is too tricky, do the font selection first, then set the new font by name (requires putting all logfont parameters in the font name).

Athena 和 Motif:

6 New Motif toolbar button from Marcin Dalecki:
- When the mouse pointer is over an Agide button the red becomes black. Something with the way colors are specified in the .xpm file.
- The pixmap is two pixels smaller than it should be. The gap is filled with grey instead of the current toolbar background color.

9 Can configure be changed to disable netbeans if the Xpm library is required and it's missing?

8 When using the resource "Vim*borderwidth 2" the widgets are positioned wrong.

9 XIM is disabled by default for SGI/IRIX. Fix XIM so that 'imdisable' can be off by default.

9 XIM doesn't work properly for Athena/Motif. (Yasuhiro Matsumoto) For now, keep XIM active at all times when the input method has the preediting flag.

8 X11: A menu that contains an umlaut is truncated at that character. Happens when the locale is "C", which uses ASCII instead of ISO-8859-1. Is there a way to use latin1 by default? Gnome_init() seems to do this.

8 Perhaps use fontsets for everything?

6 When starting in English and switching the language to Japanese, setting the locale with ":lang", 'guifontset' and "hi menu font=", deleting all menus and setting them again, the menus don't use the new font. Most of the tooltips work though...

7 Motif: when using a file selection dialog, the specified file name is not always used (when specifying a filter or another directory).

8 When 'encoding' is different from the current locale (e.g., utf-8) the menu strings don't work. Requires conversion from 'encoding' to the current locale. Workaround: set 'langmenu'.

Athena GUI:

9 The first event for any button in the menu or toolbar appears to get lost. The second click on a menu does work.

9 When dragging the scrollbar thumb very fast, focus is only obtained in the scrollbar itself. And the thumb is no longer updated when moving through files.

7 The file selector is not resizable. With a big font it is difficult to read long file names. (Schroeder)

4 Re-write the widget attachments and code so that we will not have to go through and calculate the absolute position of every widget every time the window is refreshed/changes size. This will help the "flashing-widgets" problem during a refresh.

5 When starting gvim with all the default colors and then typing ":hi Menu guibg=cyan", the menus change color but the background of the pullright pixmap doesn't change colors.
If you type ":hi Menu guibg=cyan font=anyfont", then the pixmap changes

```
    colors as it should.
    Allocating a new pixmap and setting the resource doesn't change the
    pullright pixmap's colors.  Why?  Possible Athena bug?


Motif GUI:
-   gui_mch_browsedir() is missing, browsedir() doesn't work nicely.
7   Use XmStringCreateLocalized() instead of XmStringCreateSimple()?
    David Harrison says it's OK (it exists in Motif 1.2).
8   Lesstif: When deleting a menu that's torn off, the torn off menu becomes
    very small instead of disappearing.  When closing it, Vim crashes.
    (Phillipps)


GUI:
9   On Solaris, creating the popup menu causes the right mouse button no
    longer to work for extending the selection. (Halevy)
9   When running an external program, it can't always be killed with CTRL-C.
    e.g., on Solaris 5.5, when using "K" (Keech).  Other 'guipty' problems on
    Solaris 2.6. (Marley)
9   On Solaris: Using a "-geometry" argument, bigger than the window where Vim
    is started from, causes empty lines below the cmdline. (raf)
8   X11 GUI: When menu is disabled by excluding 'm' from 'guioptions', ALT key
    should not be used to trigger a menu (like the Win32 version).
8   When setting 'langmenu', it should be effective immediately.  Store both
    the English and the translated text in the menu structure.  Re-generate
    the translation when 'langmenu' has changed.
8   Basic flaw in the GUI code: NextScreen is updated before calling
    gui_write(), but the GUI code relies on NextScreen to represent the state
    of where it is processing the output.
    Need better separation of Vim core and GUI code.
8   When fontset support is enabled, setting 'guifont' to a single font
    doesn't work.
8   Menu priority for sub-menus for: Amiga.
8   When translating menus ignore the part after the Tab, the shortcut.  So
    that the same menu item with a different shortcut (e.g., for the Mac) are
    still translated.
8   Add menu separators for Amiga.
8   Add way to specify the file filter for the browse dialog.  At least for
    browse().
8   Add dialog for search/replace to other GUIs?  Tk has something for this,
    use that code?  Or use console dialog.
8   When selecting a font with the font dialog and the font is invalid, the
    error message disappears too quick.
7   More features in the find/replace dialog:
    - regexp on/off
    - search in selection/buffer/all buffers/directory
       when all buffers/directory is used:
        - filter for file name
       when directory is used:
        - subdirectory on/off
        - top directory browser
8   gui_check_colors() is not called at the right moment.  Do it much later,
    to avoid problems.
```

8 gui_update_cursor() is called for a cursor shape change, even when there are mappings to be processed. Only do something when going to wait for input. Or maybe every 100 ms?
8 X11: When the window size is reduced to fit on screen, there are blank lines below the text and bottom scrollbar. "gvim -geometry 80x78+0+0". When the "+0+0" is omitted it works.
8 When starting an external command, and 'guipty' set, BS and DEL are mixed up. Set erase character somehow?
8 A dead circumflex followed by a space should give the '^' character (Rommel). Look how xterm does this.
Also: Bednar has some code for dead key handling.
Also: Nedit 5.0.2 with USE_XMIM does it right. (Gaya)
8 The compose key doesn't work properly (Cepas). Both for Win32 and X11.
7 The cursor in an inactive window should be hollow. Currently it's not visible.
7 GUI on Solaris 2.5.1, using /usr/dt/..: When gvim starts, cursor is hollow, after window lowered/raised it's OK. (Godfrey)
7 When starting GUI with ":gui", and window is made smaller because it doesn't fit on the screen, there is an extra redraw.
8 When setting font with .Xdefaults, there is an extra empty line at the bottom, which disappears when using ":set guifont=<Tab>". (Chadzelek)
8 When font shape changes, but not the size, doing ":set font=" does not redraw the screen with the new font. Also for Win32.
When the size changes, on Solaris 2.5 there isn't a redraw for the remaining part of the window (Phillipps).
- Flashes really badly in certain cases when running remotely from a Sun.
4 Re-write the code so that the highlighting isn't changed multiple times when doing a ":hi clear". The color changes happen three or more times currently. This is very obvious on a 66Mhz 486.

Win32 控制台:
8 Should $USERPROFILE be preferred above $HOMEDRIVE/$HOMEPATH? No, but it's a good fallback, thus use:
```
$HOME
$HOMEDRIVE$HOMEPATH
SHGetSpecialFolderPath(NULL, lpzsPath, CSIDL_APPDATA, FALSE);
$USERPROFILE
SHGetSpecialFolderPath(NULL, lpzsPath, CSIDL_COMMON_APPDATA, FALSE);
$ALLUSERSPROFILE
$SYSTEMDRIVE\
C:\
```
8 Win32 console: <M-Up> and <M-Down> don't work. (Geddes) We don't have special keys for these. Should use modifier + key.
8 Win32 console: caps-lock makes non-alpha keys work like with shift. Should work like in the GUI version.
8 Environment variables in DOS are not case sensitive. Make a define for STRCMP_ENV(), and use it when comparing environment var names.
8 Setting 'shellslash' has no immediate effect. Change all file names when it is set/reset? Or only use it when actually executing a shell command?
8 When editing a file on a Samba server, case might matter. ":e file" followed by ":e FILE" will edit "file" again, even though "FILE" might be another one. Set last used name in buflist_new()? Fix do_ecmd(), etc.
8 When a buffer is editing a file like "ftp://mach/file", which is not going

to be used like a normal file name, don't change the slashes to backslashes. (Ronald Hoellwarth)

Win32 控制台:

9 When editing a file by its short file name, it should be expanded into its long file name, to avoid problems like these: (Mccollister)
1) Create a file called ".bashrc" using some other editor.
2) Drag that file onto a shortcut or the actual executable.
3) Note that the file name is something like BASHRC~1
4) Go to File->Save As menu item and type ".bashrc" as the file name.
5) Press "Yes" to indicate that I want to overwrite the file.
6) Note that the message "File exists (add ! to override)" is displayed and the file is not saved.
Use FindFirstFile() to expand a file name and directory in the path to its long name.

8 Also implement 'conskey' option for the Win32 console version? Look at how Xvi does console I/O under Windows NT.

7 Re-install the use of $TERM and support the use of different terminals, besides the console.

8 Use of <altgr> modifier doesn't work? 5.3 was OK. (Garcia-Suarez/Guckes)

9 Mapping <C-S-Tab> doesn't work correctly. How to see the difference with <C-S-i>?

9 tmpnam() uses file in root of file system: "\asdf". That doesn't work on a Netware network drive. Use same function as for Win32 GUI?

8 In os_win32.h, HAVE_STRICMP and HAVE_STRNICMP are defined only if __GNUC__ is not defined. Shouldn't that be the other way around?

Amiga:

8 In mch_inchar() should use convert_input_safe() to handle incomplete byte sequences.

9 In mch_expandpath() a "*" is to be expanded, but "*" isn't. Remove backslashes in result.

8 Executing a shell, only one option for 'shell' is separated. Should do all options, using white space separation.

Macintosh:

- GUI: gui_mch_browsedir() is missing.

7 Loading the Perl library only works on OS/X 10.2 or 10.3, never on both. Load the Perl library dynamically see Python sources file dynload_mac (Jack)
dynamic linking: http://developer.apple.com/technotes/tn2002/tn2064.html

8 inputdialog() doesn't resize when giving more text lines. (David Fishburn, 2006 Sept 28)

8 Define vim_mkdir() for Macintosh.

8 Define mch_writable() for Macintosh.

9 When DiskLock is running, using a swap file causes a crash. Appears to be a problem with writing a file that starts with a dot. (Giacalone)

9 In mac_expandpath() check that handling of backslashes is done properly.

"小" 问题:

- Can't disable terminal flow control, to enable the use of CTRL-S and CTRL-Q. Add an option for it?
- When using e_secure in do_one_cmd() mention the command being executed, otherwise it's not clear where it comes from.
- When the quickfix window is open and executing ":echo 'hello'" using the Command-line window, the text is immediately removed by the redrawing. (Michael Henry, 2008 Nov 1)
  Generic solution: When redrawing while there is a message on the cmdline, don't erase the display but draw over the existing text.
  Other solution, redraw after closing the cmdline window, before executing the command.

9 For Turkish vim_tolower() and vim_toupper() also need to use utf_ functions for characters below 0x80. (Sertacyildiz)

9 When the last edited file is a help file, using '0 in a new Vim doesn't edit the file as a help file. 'filetype' is OK, but 'iskeyword' isn't, file isn't readonly, etc.

8 When an ":edit" is inside a try command and the ATTENTION prompt is used, the :catch commands are always executed, also when the file is edited normally. Should reset did_emsg and undo side effects. Also make sure the ATTENTION message shows up. Servatius Brandt works on this.

7 Vimtutor leaves escape sequence in terminal. This is the xterm response to requesting the version number. (Yasuhiro Matsumoto)

8 When redirecting and using ":silent" the current column for displaying and redirection can be different. Use a separate variable to hold the column for redirection.

7 The messages for "vim --help" and "vim --version" don't use 'termencoding'.

- Could the hit-enter prompt be avoided when a message only overlaps the 'showcmd' area? Clear that area when the next cmd is typed.

8 When 'scrollbind' is set, a window won't scroll horizontally if the cursor line is too short. Add a word in 'scrollopt' to allow moving the cursor to longer line that is visible. A similar thing is done for the GUI when using the horizontal scrollbar.

7 VisVim can only open one file. Hard to solve: each opened file is passed with a separate invocation, would need to use timestamps to know the invocations belong together.

8 When giving a ":bwipeout" command a file-changed dialog may popup for this buffer, which is pointless. (Mike Williams)

8 On MS-Windows ":make" doesn't show output while it is working. Use the tee.exe from http://unxutils.sourceforge.net/ ? About 16 Kbyte in the UnxUtils.zip archive.
  Is it better than what we have in src/tee?

8 When doing Insert mode completion a mapping cannot recursively call edit(), because the completion information is global. Put everything in an allocated structure?

8 Command line completion: buffers "foo.txt" and "../b/foo.txt", completing ":buf foo<Tab>" doesn't find the second one. (George V. Reilly)

7 mb_off2cells() doesn't work correctly on the tail byte of a double-byte character. (Yasuhiro Matsumoto) It should return 1 when used on a tail byte, like for utf-8. Store second byte of double-byte in ScreenLines2[] (like for DBCS_JPNU) and put a zero in the second byte (like for UTF-8).

7 Inside a function with "perl <<EOF" a line with "$i++" is recognized as an ":insert" command, causing the following "endfunction" not to be found. Add skipping this perl construction inside function definitions.

7 When 'ttimeoutlen' is 10 and 'timeoutlen' is 1000, there is a keycode "<Esc>a" and a mapping <Esc>x", when typing "<Esc>a" with half a second delay should not be interpreted as a keycode. (Hans Ginzel)

7 ":botright 1 new" twice causes all window heights to be changed. Make the bottom window only bigger as much as needed.

7 The Cygwin and MingW makefiles define "PC", but it's not used anywhere. Remove? (Dan Sharp)

9 User commands use the context of the script they were defined in. This causes a "s:var" argument to unexpectedly use a variable in the defining script, not the calling script. Add an argument to ":command": "-keepcontext". Do replace <SID>, so that a function in the defining script can be called.

8 The Japanese message translations for MS-Windows are called ja.sjis.po, but they use encoding cp932. Rename the file and check that it still works.

8 A very long message in confirm() can't be quit. Make this possible with CTRL-C.

8 "gf" always excludes trailing punctuation characters. file_name_in_line() is currently fixed to use ".,:;!". Add an option to make this configurable?

8 'hkmap' should probably be global-local.

8 Using ":s" in a function changes the previous replacement string. Save "old_sub" in save_search_patterns()?

8 Should allow multi-byte characters for the delimiter: ":s+a+b+" where "+" is a multi-byte character.

8 When appending to a file and 'patchmode' isn't empty, a backup file is always written, even when the original file already exists.

9 When getting focus while writing a large file, could warn for this file being changed outside of Vim. Avoid checking this while the file is being written.

7 The message in bt_dontwrite_msg() could be clearer.

8 The script ID that is stored with an option and displayed with ":verbose set" isn't reset when the option is set internally. For example when 'foldlevel' is set from 'foldlevelstart'.

8 Also store the line number with the script ID and use it for ":verbose", so that "set nocompatible" is found when it changes other option values. When an option is set indirectly mention the command? E.g. when ":diffsplit" sets 'foldmethod'.

8 In the fileformat dialog, "Cancel" isn't translated. Add a global variable for this. (Eduardo Fernandez)

9 When editing a file with 'readonly' set, there is no check for an existing swap file. Then using ":write" (without making any changes) doesn't give a warning either. Should check for an existing swap file without creating one. Unfinished patch by Ian Kelling, 2008 July 14.

7 When 'showbreak' is set, the amount of space a Tab occupies changes. Should work like 'showbreak' is inserted without changing the Tabs.

7 When 'mousefocus' is set and switching to another window with a typed command, the mouse pointer may be moved to a part of the window that's covered by another window and we lose focus. Only move in the y direction, not horizontally?

8 ":hardcopy":
- Using the cterm_color[] table is wrong when t_colors is > 16.
- Need to handle unprintable characters.
- Win32: On a B&W printer syntax highlighting isn't visible. Perform

dithering to make grey text?
- Add a flag in 'printoptions' to add an empty page to make the total number even. "addempty"? (Mike Williams)
- Respect 'linebreak'. Perhaps also 'showbreak'?
- Should interpret CTRL-L as a page break.
- Grey line numbers are not always readable. Add field in 'printoptions'. Default to black when no syntax highlighting.
- Be able to print a window in diff mode.
- Be able to specify a colorscheme to use for printing. And a separate one for B&W printing (if that can be detected).

8 When 'virtualedit' is "block,insert" and encoding is "utf-8", selecting a block of one double-wide character, then "d" deletes only half of it.

8 When 'virtualedit' is set, should "I" in blockwise visual mode also insert in lines that don't extend into the block?

8 With 'virtualedit' set, in Insert mode just after the end of line, CTRL-O yh does not yank the last character of the line. (Pavel Papushev)
Doing "hl" first appears to make it work.

8 With 'virtualedit' set it's possible to move into the blank area from 'linebreak'.

8 With 'virtualedit' set and 'selection' "exclusive", a Visual selection that ends in or after a tab, "d" doesn't delete (part of) the tab. (Helmut Stiegler)

9 When jumping to a tag, the search pattern is put in the history. When 'magic' is on, the pattern may not work. Translate the pattern depending on p_magic when putting it in the history? Alternative: Store value of 'magic' in history. (Margo)

9 optwin.vim: Restoring a mapping for <Space> or <CR> is not correct for ":noremap". Add "mapcmd({string}, {mode})? Use code from ":mkexrc".

9 term_console is used before it is set (msdos, Amiga).

9 Get out-of-memory for ":g/^/,$s//@/" on 1000 lines, this is not handled correctly. Get many error messages while redrawing the screen, which cause another redraw, etc.

8 [<C-I> doesn't work when '*' is in 'iskeyword'. find_pattern_in_path() must escape special characters in the pattern.

8 Vim can overwrite a read-only file with ":w!". ":w" can't overwrite an existing file, "w!" can, but perhaps not a read-only file? Then use ":w!!" for that.
Or ask for permission to overwrite it (if file can be made writable) and restore file to readonly afterwards.
Overwriting a file for which a swap file exists is similar issue.

7 When compiled with "xterm_clipboard", startup can be slower and might get error message for invalid $DISPLAY. Try connecting to the X server in the background (forked), so that Vim starts up quicker? Connect as soon as the clipboard is to be used (Visual select mode starts, paste from clipboard)

7 X11: Some people prefer to use CLIPBOARD instead of PRIMARY for the normal selection. Add an "xclipboard" argument to the 'clipboard' option? (Mark Waggoner)

6 When the xterm reports the number of colors, a redraw occurs. This is annoying on a slow connection. Wait for the xterm to report the number of colors before drawing the screen. With a timeout.

8 When the builtin xterm termcap contains codes that are not wanted, need a way to avoid using the builtin termcap.

8 Xterm sends ^[[H for <Home> and ^[[F for <End> in some mode. Also

recognize these keys? Mostly useful for xterm simulators, like gnometerm. See http://dickey.his.com/xterm/xterm.faq.html#xterm_pc_style.

8 '[and '] should be set to start/end of line when using a linewise operator (e.g., ":w").

8 CTRL-A can't handle big "long" numbers, they become negative. Check for "-" character, if not present, use unsigned long.

8 Add suspending with CTRL-Z at the "more" prompt, and when executing a long script in do_cmdline().

8 When using 'hidden', many swap files will be open. When Vim runs into the maximum number of open files, error messages will appear. Detect that this problem is present, and close any hidden files that don't have changes.

8 With 'viminfo' set such that the ".viminfo" file is written on a FAT filesystem, an illegal file name may be created: ".vim".

8 For each buffer that is opened, the viminfo file is opened and read to check for file marks. This can be slow.

8 Core dump within signal function: gdb doesn't show stack backtrace! Option to skip catch_signals()?

9 Repeating a "cw" with "." doesn't work if the text was pasted from the clipboard. (Thomas Jones) It's because the menu/toolbar item exits Insert mode and uses "gP". How to fix this without breaking inserting a block of text?

8 In Replace mode pasting from the clipboard (using menu or toolbar) inserts all the text. Add ":rmenu"?

8 Pasting with the mouse in Replace mode inserts the text, instead of overwriting, when it is more than one line. Same for using <C-R>.

9 CTRL-E and CTRL-Y don't work in small window when 'so' is 4 and lines are wrapping (Acevedo/in.226). E.g., when using CTRL-E, window height 7, window might actually scroll down when last line of buffer is displayed. --> Remember if the previous command was "cursor follows screen" or "screen follow cursor" and use this in cursupdate().

7 tilde_replace() can only handle "~/", should also do "~user/".
Get the list of home directories (from /etc/passwd? Use getpwent()) and use some clever algorithm to match a path with that. Find common strings in the list?

8 When dragging status line with mouse, sometimes a jump when first clicking on the status line (caused by 'winheight'). Select window on button up, instead of on button down.

8 Dragging the status line doesn't scroll but redraw.

9 Evaluating 'statusline' in build_stl_str_hl() does not properly check for reaching the end of the available buffer.
Patch to dynamically allocate the buffer for % items. (Eric Arnold, 2006 May 14)

8 When performing incremental search, should abort searching as soon as a character is typed.

8 When the value of $MAKE contains a path, configure can't handle this. It's an autoconf bug. Remove the path from $MAKE to work around it.

8 How to set VIMRC_FILE to \"something\" for configure? Why does this not work: CFLAGS='-DVIMRC_FILE=\"/mydir/myfile\"' ./configure

8 The temporary file is sometimes not writable. Check for this, and use an alternate name when it isn't. Or add the 'temptemplate' option: template for the temp file name ":set temptemplate=/usr/tmp/?????.tmp".
Also: Win32 version uses Windows temp directory, which might not work for cygwin bash.

7 Get error "*, \+ or \(operand could be empty" for pattern "\(.\)\1\{3}". Remember flags for backreferences.
7 When switching to Daylight Saving Time, Vim complains that a file has been changed since last read. Can we use a function that uses GMT?
7 When completing an environment variable after a '$', check for file names that contain a '$' after all have been found.
8 When "cm" termcap entry is missing, starting gvim shouldn't complain about it. (Lohner) Try out with "vt100" entry, cm replaced with cX.
7 When an include file starts with "../", the check for already visiting this file doesn't work. Need to simplify the file name.
7 The names and comments for the arguments of do_browse() are confusing. "dflt" isn't the default file name when "initdir" is not NULL and "initdir" is the default path to be used.
7 When 'scrolloff' is exactly half the window height, "j" causes a scroll of two lines at a time. "k" doesn't do this. (Cory T. Echols)
8 When write_viminfo() is used while there are many orphaned viminfo tempfiles writing the viminfo file fails. Give a clear error message so that the user knows he has to delete the files.
7 It's possible to redefine a script-local function with ":func <SNR>123_Test()". (Krishna) Disallow this.

我无法重现这些（如果你可以，请告知！）：
9 NT 4.0 on NTFS file system: Editing ".bashrc" (drag and drop), file disappears. Editing ".xyz" is OK. Also, drag&drop only works for three files. (McCollister)

（可能）不会解决的问题：
- GTK: when using the popup menu with spelling suggestions and releasing the right mouse button before the menu appears selecting an item with the right mouse button has no effect. GTK does not produce an event for this.
- GTK 2: Cannot use the file selector. When using it many things become slow. This is caused by some code in GTK that writes ~/.recently-used.xbel every time an event is handled. It assumes the main loop is never quit, which is a wrong assumption. Also, it overwrites the file with different file permissions, which is a privacy issue. This needs to be fixed in GTK. A solution in Vim would be really complicated. (2008 Jul 31) This appears to be fixed in Vim 7.3.
- xterm title: The following scenario may occur (esp. when running the Vim test script): Vim 1 sets the title to "file1", then restores the title to "xterm" with an ESC sequence when exiting. Vim 2 obtains the old title with an X library call, this may result in "file1", because the window manager hasn't processed the "xterm" title yet. Can apparently only be worked around with a delay.
- In a terminal with 'mouse' set such that the mouse is active when entering a command line, after executing a shell command that scrolls up the display and then pressing ":": Selecting text with the mouse works like the display wasn't scrolled. Vim doesn't know how much the external command scrolled up the display. Use Shift to select text.
- X windows: When $DISPLAY points to a X server where there is no access permission, trying to connect to the X server causes an error message. XtOpenDisplay() prints this directly, there is no way to avoid it.
- X windows: Setting 'guifontset' to an illegal value sometimes crashes Vim.

This is caused by a fault in a X library function, can't be solved in Vim.
- Win32 tcl: has("tcl") hangs when the tcl84.dll is from cygwin.
- Motif: When adding a menu item "Find this &Symbol", the "s" in "this" will be underlined, instead of in "Symbol". Motif doesn't let us specify which character gets the highlighting.
- Moving the cursor removes color in color-xterm. This is a color-xterm problem! color-xterm ver. 6.1 beta 3 and later work properly.
- In zsh, "gvim&" changes the terminal settings. This is a zsh problem. (Jennings)
- Problem with HPterm under X: old contents of window is lost (Cosentino).
- Amiga: The ":cq" command does not always abort the Manx compiler. Why?
- Linux: A file with protection r--rw-rw- is seen readonly for others. The access() function in GNU libc is probably wrong.
- When doing a CTRL-Z and typing a command for the shell, while Vim is busy (e.g. writing a file), the command for the shell is sometimes eaten by Vim, because the terminal mode is changed from RAW to CBREAK.
- An old version of GNU tgoto can't handle the terminfo code for "AF". The "%p1" is interpreted as "%p" and "1", causing color not to be working. Fix: Change the "%p1" in the "AF" and "AB" terminfo entries to "%p". (Benzinger).
- When running an external command from the GUI, typeahead is going to that program, not to Vim. It looks like the shell eats the characters, Vim can't get back what the external command didn't use.
- Win32 GUI: Error code from external command not returned in shell_error. It appears that cmd.exe and command.com don't return an error code.
- Win32 GUI: The Toolbar is a bit too high when the flat style is being used. We don't have control over the height of the Toolbar.
- Win32: All files created on the day of switching from winter to summer time cause "changed since editing started" messages. It goes away when the file is written again the next day, or the timezone is adjusted. DJGPP version is OK. (Zaimi) Looks like a problem with the Win32 library. Rebooting doesn't help. Time stamps look OK in directory. (Penn) Is this on FAT (stores wall clock time) or NTFS (stores UTS)?
- Win32, MS-Windows XP: $HOME uses the wrong drive when the user profiles are not on the boot disk. This is caused by a wrong value of $HOMEDRIVE. This is a bug in XP, see MSKB article 818134.
- Win32, MS-Windows: expanding plugin/**/*.vim also picks up dir/ctags.vim,v. This is because the short file name is something like "ctags~1.vim" and that matches the pattern.
- SunOS 5.5.1 with Motif: The file open dialog does not have a horizontal scroll bar for the "files" selection. This is a problem in the Motif libraries, get a patch from Sun.
- Solaris 2.6 with GTK and Perl: gvim crashes when started. Problem with X input method called from GDK code. Without Perl it doesn't crash.
- VMS: Vimdiff doesn't work with the VMS diff, because the output looks different. This makes test 47 fail. Install a Unix-compatible diff.
- Win32 GUI: mouse wheel always scrolls rightmost window. The events arrive in Vim as if the rightmost scrollbar was used.
- GTK with Gnome: Produces an error message when starting up:
  Gdk-WARNING **: locale not supported by C library
  This is caused by the gnome library gnome_init() setting $LC_CTYPE to "en_US". Not all systems support this locale name, thus causing the error. Hopefully a newer version of GTK/Gnome fixes this problem.
- GTK 2: With this mapping the hit-enter prompt is _sometimes_ below the

screen, at other times there is a grey area below the command line:
:nmap <F11> :if &guioptions=~'m' \| set guioptions-=m \| else \| set guioptions+=m \| end
- GTK: When pasting a selection from Vim to xclipboard gvim crashes with a ABRT signal. Probably an error in the file gdkselection.c, the assert always fails when XmbTextListToTextProperty() fails. (Tom Allard)
- GTK 2: gives an assertion error for every non-builtin icon in the toolbar. This is a GTK 2.4.x bug, fixed in GTK 2.4.2. (Thomas de Grenier de Latour)
- When using an xterm that supports the termresponse feature, and the 't_Co' termcap option was wrong when Vim started, it will be corrected when the termresponse is received. Since the number of colors changes, the highlighting needs to be initialized again. This may cause colors defined in the vimrc file to be lost.
- On Windows NT 4.0 the number of files passed to Vim with drag&drop and "Edit with Vim" is limited. The maximum command line length is 255 chars.

------------------------- 扩展及改进 ----------------------------
extensions-improvements

修正所有漏洞之后要新增的最有意思的功能:
- Using ":exe edit fname" has escaping problems. Use ":edit ++(fname)". Thus use "++=" to give arguments as expressions, comma separated as if calling a function.
With options: ":edit ++(['!', '++enc=abc'], ['+/pat'], fname)".
Alternative: Make a function for Ex commands: cmd_edit().
- Add COLUMN NUMBERS to ":" commands ":line1,line2[col1,col2]cmd". Block can be selected with CTRL-V. Allow '$' (end of line) for col2. (issue #3292)
- ECLIPSE plugin. Problem is: the interface is very complicated. Need to implement part in Java and then connect to Vim. Some hints from Alexandru Roman, 2004 Dec 15. Should then also work with Oracle Jdeveloper, see JSR 198 standard http://www.jcp.org/en/jsr/detail?id=198.
Eclim does it: http://eclim.sourceforge.net/ (Eric Van Dewoestine)
Plugin that uses a terminal emulator: http://vimplugin.sf.net
And another one: http://www.satokar.com/viplugin/index.php
- STICKY CURSOR: Add a way of scrolling that leaves the cursor where it is. Especially when using the scrollbar. Typing a cursor-movement command scrolls back to where the cursor is.
- Scroll commands by screen line. g CTRL-E and g CTRL-Y ? Requires the first line to be able to start halfway.
8 Add a command to jump to a certain kind of tag. Allow the user to specify values for the optional fields. E.g., ":tag size type=m".
Also allow specifying the file and command, so that the result of taglist() can be used.
- X11: Make it possible to run Vim inside a window of another program. This can be done with XReparentWindow(). But how exactly?

文档:
8 List of Vim runtime directories. dotvim.txt from Charles Campbell, 2007 Feb 20.
8 The GUI help should explain the Find and Find/Replace dialogs. Add a link to it from ":promptrepl" and ":promptfind".
8 List of options should mention whether environment variables are expanded or not.

8 Extend usr_27.txt a bit. (Adam Seyfarth)
9 Make the Reference Manual more precise. For each command mention:
- change to cursor position and curswant
- if it can be undone (u/CTRL-R) and redone (.)
- how it works for folded lines
- how it works with multi-byte characters
9 In change.txt, remark about Javadoc isn't right. Right alignment would work too.
8 Spread the windows commands over the other files. For example, ":stag" should be with ":tag". Cross-link with tags to avoid too much double text.
8 Add tags for all features, e.g. "gui_running".
7 MS-Windows: When a wrong command is typed with an ALT key, give a hint to look at the help for 'winaltkeys'.
7 Add a help.vim plugin that maps <Tab> to jump to the next tag in || and <C-Tab> (and <S-Tab>) to the previous tag.
Patch by Balazs Kezes, 2007 Dec 30. Remark from A. Politz.
- Check text editor compendium for vi and Vim remarks.

帮助:
- First try using the ":help" argument literally, before using it as a pattern. And then match it as part of a tag.
- When a help item has multiple matches make it possible to use ":tn" to go to the other matches.
- Support a way to view (and edit) .info files.
- Implement a "sticky" help window, some help text lines that are always displayed in a window with fixed height. (Guckes) Use "~/.vimhelp" file, user can edit it to insert his favorite commands, new account can contain a default contents.
- Make 'winminheight' a local option, so that the user can set a minimal height for the help window (and other windows).
- ":help :s^I" should expand to ":help :substitute".
- Make the help key (<F1>) context sensitive?
- Learn mode: show short help while typing commands.

对用户更友好:
8 Windows install with install.exe: Use .exe instead of .bat files for links, so that command line arguments are passed on unmodified? (Walter Briscoe)
8 Windows install: Be able to associate Vim with a selection of file types?
8 Windows uninstall: Have uninstal.c delete the vimfiles directories that dosinst.c creates. List the contents of the directory (recursively) if the user asks for it. Requires an implementation of "rm -rf".
8 Remember the name of the vimrc file that was used (~/.vimrc, $VIM/_vimrc, $HOME/_vimrc, etc.) and add "edit vimrc" to the File menu.
- Add a way to save local settings and mappings into a new plugin file. ":mkplugin <file>"?
- Add mappings local to a window: ":map <window> ..."?
9 Add buffer-local menu. Should offer a choice between removing the menu or disabling it. Be careful that tear-offs don't disappear (keep one empty item?).
Alternative: use BufEnter and BufLeave autocommands.

8 make a vimtutor script for Amiga and other systems.
7 When Vim detects a file is being edited elsewhere and it's a gvim session of the same user it should offer a "Raise" button, so that the other gvim window can be displayed. (Eduard)
8 Support saving and restoring session for X windows? It should work to do ":mksession" and use "-S fname" for the restart command. The gui_x11_wm_protocol_handler() already takes care of the rest. global_event_filter() for GTK.

标签页:
9 GUI implementation for the tab pages line for other systems.
7 GUI: Control over the appearance of the text in the labels (bold, color, font, etc.)
8 Make GUI menu in tab pages line configurable. Like the popup menu.
8 balloons for the tab page labels that are shortened to show the full path.
7 :tabdup duplicate the tab with all its windows.
7 Option to put tab line at the left or right? Need an option to specify its width. It's like a separate window with ":tabs" output.
8 Add local options for each tab page? E.g., 'diffopt' could differ between tab pages.
7 Add local highlighting for each tab page?

拼写检查:
- Support more regions? Caolan McNamara argues it's needed for es_XX. https://bugzilla.redhat.com/bugzilla/show_bug.cgi?id=219777
- Unicode defines another quote character: 0x2019. Use it as an equivalent of a single quote, thus use it as a word character like a quote and match with words, replacing the curly quote with a single quote.
- Could filter é things for HTML before doing spell checking. Similarly for TeX.
- The Hungarian spell file uses four extra characters in the FOL/UPP/LOW items than other spell files with the ISO-8859-2 encoding, that causes problem when changing 'spelllang'. There is no obvious way to fix this.
- Considering Hunspell 1.1.4:
  What does MAXNGRAMSUGS do?
  Is COMPLEXPREFIXES necessary when we have flags for affixes?
- Support spelling words in CamelCase as if they were two separate words. Requires some option to enable it. (Timothy Knox)
- There is no Finnish spell checking file. For openoffice Voikko is now used, which is based on Malaga: http://home.arcor.de/bjoern-beutel/malaga/ (Teemu Likonen)
8 ":mkspell" still takes much too long in Hungarian dictionary from hunspell. Only solution appears to be to postpone secondary suffixes.
8 Handle postponed prefix with COMPOUNDPERMITFLAG or COMPOUNDFORBIDFLAG. WFP_COMPPERMIT and WFP_COMPFORBID
8 implement use of <compoptions> in .spl file:
  implement CHECKCOMPOUNDREP: when a compound word seems to be OK apply REP items and check if the result is a valid word.
  implement CHECKCOMPOUNDDUP
  implement CHECKCOMPOUNDTRIPLE
  Add CHECKCOMPOUNDCASE: when compounding make leading capital lower case. How is it supposed to work?

- Add a command the repeats]s and z=, showing the misspelled word in its context. Thus to spell-check a whole file.
- suggestion for "KG" to "kg" when it's keepcase.
- For flags on affixes: Use a "AFFCOMPSET" flag; means the compound flags of the word are not used.
- Support breakpoint character ? 0xb7 and ignore it? Makes it possible to use same wordlist for hyphenation.
- Compound word is accepted if nr of words is <= COMPOUNDWORDMAX OR nr of syllables <= COMPOUNDSYLMAX. Specify using AND in the affix file?
- NEEDCOMPOUND also used for affix? Or is this called ONLYINCOMPOUND now? Or is ONLYINCOMPOUND only for inside a compound, not at start or end?
- Do we need a flag for the rule that when compounding is done the following word doesn't have a capital after a word character, even for Onecap words?
- New hunspell home page: http://hunspell.sourceforge.net/
  - Version 1.1.0 is out now, look into that.
  - Lots of code depends on LANG, that isn't right. Enable each mechanism in the affix file separately.
  - Example with compounding dash is bad, gets in the way of setting COMPOUNDMIN and COMPOUNDWORDMAX to a reasonable value.
  - PSEUDOROOT == NEEDAFFIX
  - COMPOUNDROOT -> COMPOUNDED? For a word that already is a compound word
    Or use COMPOUNDED2, COMPOUNDED3, etc.
- CIRCUMFIX: when a word uses a prefix marked with the CIRCUMFIX flag, then the word must also have a suffix marked with the CIRCUMFIX flag. It's a bit primitive, since only one flag is used, which doesn't allow matching specific prefixes with suffixes.
  Alternative:
  PSFX {flag} {pchop} {padd} {pcond} {schop} {sadd}[/flags] {scond}
  We might not need this at all, you can use the NEEDAFFIX flag and the affix which is required.
- When a suffix has more than one syllable, it may count as a word for COMPOUNDWORDMAX.
- Add flags to count extra syllables in a word. SYLLABLEADD1 SYLLABLEADD2, etc.? Or make it possible to specify the syllable count of a word directly, e.g., after another slash: /abc/3
- MORPHO item in affix file: ignore TAB and morphological field after word/flags and affix.
- Implement multiple flags for compound words and CMP item?
  Await comments from other spell checking authors.
- Also see tklspell: http://tkltrans.sourceforge.net/

8 Charles Campbell asks for method to add "contained" groups to existing syntax items (to add @Spell).
Add ":syntax contains {pattern} add=@Spell" command? A bit like ":syn cluster" but change the contains list directly for matching syntax items.

- References: MySpell library (in OpenOffice.org).
  http://spellchecker.mozdev.org/source.html
  http://whiteboard.openoffice.org/source/browse/whiteboard/lingucomponent/source/spellcheck
  author: Kevin Hendricks <kevin.hendricks@sympatico.ca>

8 It is currently not possible to mark "can not" as rare, because "can" and "not" are good words. Find a way to let "rare" overrule "good"?

8 Make "en-rare" spell file? Ask Charles Campbell.

8 The English dictionaries for different regions are not consistent in their use of words with a dash.

7 Insert mode completion mechanism that uses the spell word lists.

8 Add hl groups to 'spelllang'?
    :set spelllang=en_us,en-rare/SpellRare,en-math/SpellMath
More complicated: Regions with different languages? E.g., comments
in English, strings in German (po file).

Diff 模式:

9 When making small changes, e.g. deleting a character, update the diff.
Possibly without running diff.
8 Also show difference with the file when editing started? Should show what
can be undone. (Tom Popovich)

折叠:

(commands still available: zI zJ zK zp zP zq zQ zV zy zY;
secondary: zB zS zT zZ, z=)
8 Vertical folds: looks like vertically split windows, but the cursor moves
through the vertical separator, separator moves when scrolling.
8 Add "z/" and "z?" for searching in not folded text only. Or use a regexp
item, so that it can be used in any pattern.
8 When a closed fold is displayed open because of 'foldminlines', the
behavior of commands is still like the fold is closed. How to make the
user aware of this?
8 Add an option 'foldskip' with values like 'foldopen' that specifies which
commands skip over a closed fold.
8 "H" and "L" count buffer lines instead of window lines. (Servatius Brandt)
8 Add a way to add fold-plugins. Johannes Zellner has one for VB.
7 When using manual folding, the undo command should also restore folds.
- Allow completely hiding a closed fold. E.g., by setting 'foldtext' to an
empty string. Require showing a character in 'foldcolumn' to avoid the
missing line goes unnoticed.
How to implement this?
- When pressing the down arrow of a scrollbar, a closed fold doesn't scroll
until after a long time. How to make scrolling with closed folds
smoother?
- When creating a session, also store folds for buffers in the buffer list,
using the wininfo in wi_folds.
- When currently editing the first file in the argument list the session
file can contain:
    args version.c main.c
    edit version.c
Can editing version.c twice be avoided?
- 'foldmethod' "textobject": fold on sections and paragraph text objects.
- "zuf": undo change in manual fold. "zUf" redo change in manual fold. How
to implement this?
- "zJ" command: add the line or fold below the fold in the fold under the
cursor.
- 'foldmethod' "syntax": "fold=3" argument: set fold level for a region or
match.
- Apply a new foldlevel to a range of lines. (Steve Litt)

多字节字符处理:

- When editing a file with both utf-8 and latin1 text Vim always falls back
to latin1. Add a command to convert the latin1 characters to utf-8?
    :unmix utf-8,latin1 filename

Would only work when 'encoding' is utf-8.

9 When the tail byte of a double-byte character is illegal (e.g., a CR), the display is messed up (Yasuhiro Matsumoto). Should check for illegal double-byte characters and display them differently (display each single byte).

9 'fenc' in modeline problem: add option to reload the file when 'fenc' is set to a different value in a modeline? Option can be default on. Could it be done with an autocommand?

8 Add an item in 'fileencodings' to check the first lines of a file for the encoding. See Python PEP: http://www.python.org/peps/pep-0263.html. To avoid getting a wrong encoding only accept something Emacs-like: "-*- coding: enc-na_me.foo -*-" and "-*- coding= enc-na_me.foo -*-" Match with "-*-\s*coding[:=]\s*\([::word::-_.]\+\)\s*-*-" and use first item.

8 Add an item in 'fileencodings' to check the first line of an XML file for the encoding. <?xml version="1.0" encoding="UTF-8"?> Or "charset=UTF-8"? For HTML look for "charset=utf-8".

8 When a file was converted from 'fileencoding' to 'encoding', a tag search should also do this on the search pattern. (Andrzej M. Ostruszka)

8 When filtering changes the encoding 'fileencoding' may not work. E.g., when using xxd and 'fileencoding' is "utf-16". Add an option to set a different fileencoding for filter output?

7 When converting a file fails, mention which byte could not be converted, so that the user can fix the problem.

8 Add configure option to be able to disable using the iconv library. (Udo Schweigert)

9 'aleph' should be set to 1488 for Unicode. (Zvi Har'El)

8 Should add test for using various commands with multi-byte characters.

8 'infercase' doesn't work with multi-byte characters.

8 toupper() function doesn't handle byte count changes.

7 Searching and composing characters:
When searching, should order of composing characters be ignored?
Add a special item to match with a composing character, so that composing characters can be manipulated.

8 Should implement 'delcombine' for command line editing.

8 Detect overlong UTF-8 sequences and handle them like illegal bytes.

8 ":s/x/\u\1/" doesn't work, making uppercase isn't done for multi-byte characters.

8 UTF-8: "r" in Visual mode doesn't take composing characters.

8 UTF-8: When there is a precomposed character in the font, use it instead of a character and a composing character. See xterm for an example.

7 When a character can't be displayed, display its digraph instead. 'display' option to specify this.

7 Use ideas for nl_langinfo() from Markus Kuhn in enc_default(): (www.cl.cam.ac.uk/~mgk25/ucs/langinfo.c)

- GTK and Win32: Allow selecting fonts for 'guifontset' with the fontselector somehow.

- GTK and Win32: make it possible to set the font for the menu to make it possible to have 'encoding' different from the current locale.

- dbcs_class() only works for Japanese and Korean. Implement this for other encodings. The "euc-jp" and "euc-kr" choices might be wrong.

- Find some way to automatically select the right GUI font or fontset, depending on the default value of 'encoding'.
Irrelevant in the GTK+ 2 GUI so long as UTF-8 is used.

For Windows, the charset_pairs[] table could be used. But how do we know if a font exists?
- Do keyboard conversion from 'termencoding' to 'encoding' with convert_input() for Mac GUI.
- Add mnemonics from RFC1345 longer than two characters. Support CTRL-K _{mnemonic}_
- Make 'breakat' accept multi-byte characters. Problem: can't use a lookup table anymore (breakat_flags[]). Simplistic solution: when 'formatoptions' contains "m" also break a line at a multi-byte character >= 0x100.
- Add the possibility to enter mappings which are used whenever normal text could be entered. E.g., for "f" command. But not in Normal mode. Sort of opposite of 'langmap'. Use ":amap" command?
- When breaking a line, take properties of multi-byte characters into account. The "linebreak" program from Bruno Haible can do it: ftp://ftp.ilog.fr/pub/Users/haible/gnu/linebreak-0.1.tar.gz But it's very complicated...

打印:

7 Implement "undercurl" for printing.
- Add "page width" to wrap long lines.
- Win32: use a font dialog for setting 'printfont'. Can reuse the code for the 'guifont' dialog, put the common code in a separate function.
- Add the file timestamp to the page header (with an option). (George Reilly)
- Win32: when 'printfont' is empty use 'guifont'.
- Unix: Use some dialog box to do the obvious settings (paper size, printer name, portrait/landscape, etc).
- PostScript: Only works for text that can be converted to an 8-bit character set. How to support Unicode fully?
- Allow specifying the paper size, instead of using a standard size. Same units as for the margins.
- Support right-to-left text?

8 Make the foreground color darkening function preserve the hue of the color.

语法高亮:

8 Make ":syn off" use 'runtimepath' instead of $VIMRUNTIME. (Gary Johnson) Should do the same for ":syn on" and ":syn manual".

8 Support "containedin" argument for ":syn include", so that the defined cluster can be added to existing syntax items.

8 C syntax: Don't highlight {} as errors inside () when used like this: "({ something })", often used in GCC code.

7 Add a "startgroup" to a region. Used like "nextgroup" inside the region, preferred item at the start of the region. (Charles Campbell)

8 When editing a new file without a name and giving it a name (by writing it) and 'filetype' is not set, detect the filetype. Avoid doing it for ":wq file".

7 For "nextgroup" we have skipwhite, skipnl and skipempty. It would be really nice to be able to skip with a pattern. Or skip with a syntax group. (Nikolai Weibull, 2007 Feb 27)

8 Make conversion to HTML faster (Write it in C or pre-compile the script).

9 There is still a redraw bug somewhere. Probably because a cached state is used in a wrong way. I can't reproduce it...

7 Be able to change only the background highlighting. Useful for Diff* and Search highlighting.

7 When 'number' is set highlight the number of the current line.
Must be enabled with an option, because it slows down display updating.

8 Allow the user to add items to the Syntax menu sorted, without having to change this for each release.

8 Add a "matchcontains" for regions: items contained in the start or end pattern, but not in the body.

8 Add a "keepend-contained" argument: Don't change the end of an item this one is contained in. Like "keepend" but specified on the contained item, instead of the containing item.

8 cpp.vim: In C++ it's allowed to use {} inside ().

8 Some syntax files set 'iskeyword', they should use "syn iskeyword".
Also need a separate 'iskeyword' for the command line, e.g., in a help window ":e /asdf/asdf/" CTRL-W works different.

8 Add specific syntax item to match with parens/braces that don't have a "%" match. :syntax nomatch cMatchError (,{,[,),},] [contained]

8 Highlight the text between two matching parens (e.g., with a grey background) when on one of the parens or in between them.
Option for the matchparen plugin?

8 When using a cterm, and no ctermfg or ctermbg are defined, use start/stop sequences. Add remark in docs that :if 'term' == "term-name" should be used.

8 Add @spell cluster to String and Comment groups for many languages. Will allow spell checking. (Fleiner)

8 When listing syntax items, try to sort the keywords alphabetically. And re-insert the [] if possible.

8 Make it possible to use color of text for Visual highlight group (like for the Cursor).

8 It would be useful to make the highlight group name an expression. Then when there is a match, the expression would be evaluated to find out what highlight group to use. Could be used to check if the shell used in a password file appears in /etc/shells. (Nikolai Weibull)
```
    syn match =s:checkShell(v:match) contained 'pattern'
```

8 Make it possible to only highlight a sub-expression of a match. Like using "\1" in a ":s" command.

8 Support for deleting syntax items:
```
:syn keyword cTodo remove this
:syn match cTodo remove "pattern"
:syn region cString remove start="this" end="that"
```

8 Add possibility to sync on something else, when the syncing in one way doesn't find match. For HTML: When no {script} is found, try looking for a '<'. (Fleiner)

7 Replace the synchronizing method with a state machine specification? Should be able to start at any line in the file, search forwards or backwards, and use the result of matching a pattern.

7 Use parsing like awk, so that e.g., a (without a matching) can be detected.

8 Make it possible to use "inverted" highlighting, invert the original character. For Visual mode. (xterm-selection already does this).

8 Highlight non-printable characters with "SpecialChar", linked to "Special". Display them with the digraph characters, if possible.

8 Highlight the clipboard-selection with a highlight group.
8 Be able to reset highlighting to its original (default) values.
7 Be able to write current highlighting to a file as commands, similar to
":mkvimrc".
8 Improve c.vim:
- Add check for unterminated strings, with a variable to switch it on:
"c_strict_ansi".
- Detect unbalanced "#endif". Requires looking back a long way...
8 Add an option to restrict the updating of syntax highlighting to the
current line while in Insert mode.
8 When guessing value of 'background', the syntax file has already been
loaded (from the .gvimrc). After changing 'background', load it again?
8 Add ":syn resync" command, to re-parse the whole file until the current
display position.
8 Should support "me" offset for a region start pattern. To be used to
allow searching for the end pattern inside the match of the end pattern.
Example: syn region pikeXX start="([^{]" end=")" should work on "()".
8 When using a regexp for "contains=", should delay matching with it until
redrawing happens. Set a flag when a group is added, check this flag when
highlighting starts.
7 It's possible for an item to be transparent, so that the colors of an item
lower on the stack is used. Also do this with highlighting, so that the
user can set transparent highlighting? E.g. a number in a C comment would
get the color of a comment, a number in an assignment Normal. (Nikolai
Weibull)
7 Add "semitrans": Add highlighting. E.g., make the text bold, but keep the
colors. And add colors, so that Green+Red becomes Yellow.
E.g. for this html:
<B> bold text <I> italic+bold text </B> italic text </I>
7 CTRL-] checks the highlight group for finding out what the tag is.
7 Add an explanation how a list of words can be used to highlight misspelled
words.
8 Add more command line completion for :syntax.
8 Add more command line completion for :highlight.
7 Should find a better way to parse the :syntax and :highlight commands.
Use tables or lists that can be shared by parsing for execution and
completion?
8 Add ColorSchemePost autocommand event, so that scripts can set up their
highlighting. (Salman Halim)
7 Add a few sets of colors (e.g. Borland Turbo C one). With a menu to
select one of the sets.
8 Add offsets to sub-matches: "\(a*\) *"he=e1-1
'e' is end of match 'e1' is end of sub-match 1, 's2' is start of submatch
2, etc.
8 In Insert mode, when there are typeahead characters, postpone the
highlighting (for "." command).
8 Syncing on comments isn't 100% correct when / / lines mix with / * and * /.
For example: What about a line that starts with / / and contains * /?
8 Ignore / * and * / inside strings, when syncing.
7 Build a few more syntax files from the file "/usr/share/misc/vgrindefs":
ISP, LDL, Icon, ratfor. And check "nedit/source/highlight.c".
6 Add possibility to have background color continue until the right edge of
the window. Useful for comment blocks and function headings. (Rogall)
- Make it possible to add "contains" items for all items in a group. Useful

when extending an already existing syntax file.
- Add line-continuation pattern for non-syncing items too?
- Add possibility to highlight the whole line, including the right margin (for comment blocks).
- Add 'hlmatch' option: List of flags:
'c': highlight match for character under the cursor.
'b': highlight the previous (, and its match.
'a': highlight all text from the previous (until its match.
Also for {}, <>, etc.?
'e': highlight all braces without a match (slow?)
OR: add an argument "cursor" to the syntax command, which means that the region/match/keyword is only highlighted when the cursor is on it. (Campbell)
Or do it like Elvis: define text objects and how to highlight them around the cursor. (Iain Truskett)

7 Make it possible to use all words in the tags files as Keyword.
Can also be done with a script (but it's slow).

7 Make it possible to call a ":" command when a match is found. Should allow for adding keywords from the text (e.g. variables that are set). And allows for sections with different highlighting.

7 Add highlight group for commandline: "Commandline". Make sure it highlights the command line while typing a command, and any output from messages. And external commands?

8 Make a version that works like less, but with highlighting: read stdin for text, exit at end of file, don't allow editing, etc. moreim? lessim?

7 SpecialKey highlighting overrules syntax highlighting. Can't give an unprintable char another color. Would be useful for ^M at end of line.

Vim 脚本语言:

8 Make the filename and line number available to script functions, so that they can give useful debugging info. The whole call stack would be ideal. At least use this for error messages.

7 Execute a function with standard option values. No need to save and restore option values. Especially useful for new options. Problem: how to avoid a performance penalty (esp. for string options)?

8 Add referring to key options with "&t_xx". Both for "echo &t_xx" and ":let &t_xx =". Useful for making portable mappings.

- range for ":exec", pass it on to the executed command. (Webb)

8 ":{range}source": source the lines from the current file.
You can already yank lines and use :@" to execute them.
Most of do_source() would not be used, need a new function.
It's easy when not doing breakpoints or profiling.
Requires copying the lines into a list and then creating a function to execute lines from the list. Similar to getnextac().

7 ":include" command: just like ":source" but doesn't start a new scriptID? Will be tricky for the list of script names.

8 Have a look at VSEL. Would it be useful to include? (Bigham)

8 Have a prefix for a function to make it unique. When using packages it can be the plugin name.
Perhaps also have a way to remove everything that the package added? including autocommands.

7 Pre-parse or compile Vim scripts into a bytecode.
1. Put the bytecode with the original script, with an ":if

has('bytecode')" around it, so that it's only used with a Vim that supports it. Update the code with a command, can be used in an autocommand.

2. Use a ".vic" file (like Python use .pyc). Create it when writing a .vim file. Problem: distribution.
3. Use a cache directory for each user. How to recognize which cached file belongs to a sourced script?

7 Add argument to winwidth() to subtract the space taken by 'foldcolumn', signs and/or 'number'.

6 Add ++ and -- operators? They only work on variables (lvals), how to implement this?

8 Add functions:

has(":command")	Check if ":command" works. compare function with "ex_ni". E.g. for ":simalt".
escape()	Add argument to specify what to escape with.
modestack()	Instead of just the current mode return the stack of Insert / CTRL-O / :normal things.
realname()	Get user name (first, last, full) user_fullname() patch by Nikolai Weibull, Nov 3 2002 Only add this when also implemented for non-Unix systems, otherwise a shell cmd could be used. get_user_name() gets login name.
menuprop({name}, {idx}, {what})	Get menu property of menu {name} item {idx}. menuprop("", 1, "name") returns "File". menuprop("File", 1, "n") returns "nmenu File.Open..." argument. Patch by Ilya Sher, 2004 Apr 22 Return a list of menus and/or a dictionary with properties instead.
mapname({idx}, mode)	return the name of the idx'th mapping. Patch by Ilya Sher, 2004 Mar 4. Return a list instead.
char2hex()	convert char string to hex string.
crypt()	encrypt string
decrypt()	decrypt string
base64enc()	base 64 encoding
base64dec()	base 64 decoding
attributes()	return file protection flags "drwxrwxrwx"
filecopy(from, to)	Copy a file
shorten(fname)	shorten a file name, like home_replace()
perl(cmd)	call Perl and return string
inputrl()	like input() but right-to-left
typed()	return the characters typed and consumed (to find out what happened)
virtualmode()	add argument to obtain whether "$" was used in Visual block mode.
getacp()	Win32: get codepage (Glenn Maynard)
libcall()	Allow more than one argument.
libcallext()	Like libcall(), but using a callback function to allow the library to execute a command or evaluate an expression.

```
7   Make bufname("'0") return the buffer name from mark '0.  How to get the
    column and line number?  col("'0") currently returns zero.
8   argc() returns 0 when using "vim -t tag".  How to detect that no file was
    specified in any way?  To be able to jump to the last edited file.
8   Pass the command line arguments to Vim scripts in some way.  As v:args
    List?  Or extra parameter to argv()?
8   Add command arguments with three dashes, passed on to Vim scripts.
9   Add optional arguments to user functions:
        :func myFunc(arg1, arg2, arg3 = "blah", arg4 = 17)
6   User functions: Functions local to buffer "b:func()"?
8   For Strings add ":let var[{expr}] = {expr}".  When past the end of "var"
    just ignore.
8   The "= register should be writable, if followed by the name of a variable,
    option or environment variable.
8   ":let &option" should list the value of the option.
8   ":let Func().foo = value" should work, also when "foo" doesn't exist.
    Also: ":let Func()[foo] = value" should work.  Same for a List.
7   Add synIDlist(), making the whole list of syntax items on the syntax stack
    available as a List.
8   Add autocommand-event for when a variable is changed:
        :au VarChanged {varname} {commands}
8   Add "has("gui_capable")", to check if the GUI can be started.
8   Add possibility to use variables like registers: characterwise (default),
    linewise (when ending in '\n'), blockwise (when ending in '\001').  reg0,
    rega, reg%, etc.  Add functions linewise({expr}), blockwise({expr}) and
    charwise({expr}).
7   Make it possible to do any command on a string variable (make a buffer
    with one line, containing the string).  Maybe add an (invisible) scratch
    buffer for this?
        result = scratch(string, command)
        result = apply(string, command)
        result = execute(string, command)
    "command" would use <> notation.
    Does scratch buffer have a number?  Or re-use same number?
7   Add function to generate unique number (date in milliseconds).
```

健壮性:

```
6   Add file locking.  Lock a file when starting to edit it with flock() or
    fcntl().  This patch has advisory file locking while reading/writing
    the file for Vim 5.4: ~/vim/patches/kahn_file_locking .
    The patch is incomplete (needs support for more systems, autoconf).
    Andy doesn't have time to work on it.
    Disadvantage: Need to find ways to gracefully handle failure to obtain a
    lock.  When to release a lock: When buffer is unloaded?
```

性能:

```
7   For string variables up to 3 bytes don't allocate memory, use v_list
    itself as a character array.  Use VAR_SSTRING (short string).
7   Add 'lazysize' option: Above this size Vim doesn't load everything before
    starting to edit a file.  Things like 'fileencodings' only work up to this
    size, modelines only work at the top.  Useful for large log files where
    you only want to look at the first few pages.  Use zero to disable it.
```

8 move_lines() copies every line into allocated memory, making reloading a buffer a lot slower than re-editing the file. Can the memline be locked so that we don't need to make a copy? Or avoid invoking ml_updatechunk(), that is taking a lot of time. (Ralf Wildenhues, 2008 Jul 7)
With a patch, but does it work?
8 Turn b_syn_ic and b_syn_containedin into b_syn_flags.
9 Loading menu.vim still takes quite a bit of time. How to make it faster?
8 in_id_list() takes much time for syntax highlighting. Cache the result?
7 setpcmark() shifts the jumplist, this takes quite a bit of time when jumping around. Instead use an index for the start?
8 When displaying a space with only foreground highlighting, it's the same as a space without attributes. Avoid displaying spaces for the "~" lines when starting up in a color terminal.
8 Avoid alloc() for scratch buffer use, esp. in syntax.c. It's very slow on Win16.
8 Profiling shows that in_id_list() is used very often for C code. Can this function be improved?
8 For an existing file, the page size of the swap file is always the default, instead of using the block size of the device, because the swap file is created only after setting the block size in mf_open(). How can this be improved?
8 Set default for 'ttyscroll' to half a screen height? Should speed up MS-DOS version. (Negri)
7 C syntax highlighting gets a lot slower after ":set foldmethod=syntax". (Charles Campbell) Inserting a "{" is very slow. (dman)
7 HTML syntax highlighting is slow for long lines. Try displaying http://www.theregister.co.uk/content/4/22908.html. (Andre Pang)
7 Check how performance of loading the wordlist can be improved (adding a lot of abbreviations).
7 Compile Ex commands to byte codes. Store byte codes in a vim script file at the end, after "compiled:. Make it look like a single comment line for old Vim versions. Insert first line "Vim script compiled <timestamp>.
Only used compiled code when timestamp matches the file stat.
Add command to compile a vim script and add it to the file in-place.
Split Ex command executing into a parsing and executing phase.
Use compiled code for functions, while loops, etc.
8 When defining autocommands (e.g., from $VIMRUNTIME/filetype.vim), need to compare each pattern with all existing patterns. Use a hash code to avoid using strcmp() too often?
7 Include turbo_loader patches, speeding up reading a file?
Speed up reading a file by reading it into a fixed-size buffer, creating the list of indexes in another buffer, and then copying the result into a memfile block with two copies. Then read the next block into another fixed-size buffer, create the second list of indexes and copy text from the two blocks to the memfile block.
7 do_cmdline(): Avoid that the command line is copied to allocated memory and freed again later all the time. For while loops, and for when called with an argument that can be messed with.
Generic solution: Make a struct that contains a pointer and a flag that indicates if the pointer should be freed when replaced.
7 Check that the file size is not more than "sizeof(long)".
- Further improve finding mappings in maphash[] in vgetorpeek()
8 Syntax highlighting is slow when deleting lines. Try in $VIMRUNTIME/filetype.vim.

- "out of memory" after deleting (1,$d) and changing (:%s/^/> /) a lot of lines (27000) a few times. Memory fragmentation?
- Have a look at how pdksh does memory allocation (alloc.c). (Dalecki)
- Do profiling on:
  - :g/pat/normal cmd
  - deleting 10Mbyte worth of lines (netscape binary)
  - "[i" and "[d" (Yegappan Lakshmanan)
  - ":g/^/m0" on a 450Kbyte file. And the "u".
  - highlighting "~/vim/test/longline.tex", "~/vim/test/scwoop.tcl" and "~/vim/test/lockup.pl".
  - loading a syntax file to highlight all words not from a dictionary.
  - editing a Vim script with syntax highlighting on (loading vim.vim).

7 Screen updating can be further improved by only redrawing lines that were changed (and lines after them, when syntax highlighting was used, and it changed).
  - On each change, remember start and end of the change.
  - When inserting/deleting lines, remember begin, end, and line count.

- Use macros/duarte/capicua for profiling. Nvi 1.71 is the fastest!
- When using a file with one long line (1Mbyte), then do "$hhhh", is still very slow. Avoid calling getvcol() for each "h"?
- Executing a register, e.g. "10000@@" is slow, because ins_typebuf has to move the previous commands forward each time. Pass count from normal_cmd() down to do_execreg().
- Avoid calls to plines() for cursor line, use w_cline_height.
- After ":set nowrap" remove superfluous redraw with wrong hor. offset if cursor is right of the screen.

8 Make CTRL-C on Unix generate a signal, avoid using select() to check for a CTRL-C (it's slow).

代码大小:

8 GUI: When NO_CONSOLE is defined, more code can be excluded.
- Put getline() and cookie in a struct, so only one argument has to be passed to do_cmdline() and other functions.

8 Make a GUI-only version for Unix?
8 In buf_write _() isn't needed when setting errmsg, do it once when using it.
7 When compiling with a GUI-only version, the code for cterm colors can be left out.
8 When compiled with a GUI-only version, the termcap entries for terminals can be removed.
8 Can the check for libelf in configure.ac be removed?

消息:

8 When using ":q" in a changed file, the error says to "add !". Add the command so that beginners understand it: "use :q!".
8 For 'verbose' level 12 prints commands from source'ed files. How to skip lines that aren't executed? Perhaps move the echoing to do_cmdline()?
8 Use 'report' for ":bdel"? (Krishna) To avoid these messages when using a script.
- Delete message after new command has been entered and have waited for key. Perhaps after ten seconds?
- Make message history available in "msg" variables: msg1, msg2, .. msg9.

8 When reading from stdin allow suppressing the "reading from stdin" message.
9 Check handling of overwriting of messages and delays:
Very wrong: errors while redrawing cause endless loop.
When switching to another file and screen scrolls because of the long message and return must be typed, don't scroll the screen back before redrawing.
8 When address range is wrong you only get "Invalid range". Be a bit more specific: Negative, beyond last line, reverse range? Include the text.
8 Make it possible to ignore errors for a moment ('errorignore'?). Another option to switch off giving error messages ('errorquiet'?). Also an option not to give any messages ('quiet')? Or ":quiet on", ":quiet off".
Careful: For a severe error (out of memory), and when the user starts typing, error messages must be switched back on.
Also a flag to ignore error messages for shell commands (for mappings).
- Option to set time for emsg() sleep. Interrupt sleep when key is typed?
Sleep before second message?
8 In Ex silent mode or when reading commands from a file, what exactly is not printed and what is? Check ":print", ":set all", ":args", ":vers", etc. At least there should be no prompt. (Smulders) And don't clear the screen when reading commands from stdin. (Kendall)
--> Make a difference between informative messages, prompts, etc. and error messages, printing text, etc.
8 Window should be redrawn when resizing at the hit-enter prompt.
Also at the ":tselect" prompt. Find a generic solution for redrawing when a prompt is present (with a callback function?).

屏幕刷新:
- screen_line():
- insert/delete character stuff.
- improve delete rest of line (spaces at end of line).
- When moving or resizing window, try to avoid a complete redraw (esp. when dragging the status line with the mouse).
- When 'lazyredraw' set, don't echo :ex commands? Need a flag to redraw when waiting for a character.
8 Add a ":refresh [winnr]" command, to force updating a window. Useful from an event handler where ":normal" can't be used. Also useful when 'lazyredraw' is set in a mapping.

滚动:
8 Add "zy" command: scroll horizontally to put the cursor in the middle.
6 Add option to set the overlap for CTRL-F and CTRL-B. (Garhi)
- extend 'scrollbind' option: 'scrollopt' words "search", "relative", etc..
Also 'e'xecute some commands (search, vertical movements) in all bound windows.
7 Add 'scrollbind' feature to make the offset of one window with the next one equal to the window height. When editing one file in both windows it looks like each window displays a page of the buffer.
- Allow scrolling by dragging with the mouse (grab a character and move it up/down). Like the "hand" in Acrobat reader. Use Alt-LeftMouse for this? (Goldfarb)
- Add command to execute some commands (search, vertical movements) in all bound windows.

- Add 'search' option to 'scrollopt' to allow 'scrollbind' windows to be bound by regexp searches
- Add "z>" and "z<": scroll sideways one screenful. (Campbell)
- Add option to set the number of lines when not to scroll, instead of the fixed number used now (for terminals that scroll slow with a large number of lines but not with a single line).

Autoconf:

8 Should use acconfig.h to define prototypes that are used by autoheader.

8 Some compilers don't give an error for "-OPT:Olimit" but a warning. (Webb) Add a check for the warning, so that "Olimit" can be added automatically?

- Autoconf: Use @datadir@ for the system independent files. Make sure the system dependent and system independent files are separated. (Leitner).
- Add autoconf check for waitpid()/wait4().
- Remove fcntl() from autoconf, all systems have it?
- Set default for 'dictionary', add search for dictionary to autoconf.

Perl 接口:

8 Rename typemap file to something else?

7 Make buffers accessed as Perl arrays. (Clark)

7 Make it possible to compile with non-ANSI C?

6 Tcl/Tk has the "load" command: load a shared library (.so or .dll).

共享库:

8 libcall() can keep the library around instead of always calling dlclose(). (Jason Felice, 2018 Mar 20)

6 Add support for loading shared libraries, and calling functions in it.

```
:libload internal-name libname
:libunload internal-name
:liblist
:libcall internal-name function(arg1, arg2, ...)
:libcall function(arg1, arg2, ...)
```

libcall() can have only one integer or String argument at the moment.

6 Have a look on how Perl handles loading dynamic libraries.

标签:

9 With ":set tags=./tags,../tags" and a tag appears in both tags files it is added twice. Requires figuring out the actual file name for each found match. Remove tag_fname from the match and combine it with the fname in the match (without expanding or other things that take time). When 'tagrelative' is off tag_fname isn't needed at all.

8 For 'tags' wildcard in the file name is not supported, only in the path. This is due to it using file-searching . Suboptimal solution would be to make the filename or the whole option use wildcards globing, better would be to merge the 2 kinds of globing. originally (Erik Falor, 2008 April 18), updated (Ian Kelling, 2008 July 4)

7 Can CTRL-] (jump to tag) include a following "." and "->" to restrict the number of possible matches? Check tags file for an item that has members. (Flemming Madsen)

8 Scope arguments for ":tag", e.g.: ":tag class:cPage open", like Elvis.

8 When output of ":tselect" is long, getting the more-prompt, should be able to type the tag number directly.
7 Add the possibility to use the "-t {tag}" argument multiple times. Open a window for each tag.
7 Make output of ":tselect" a bit nicer. Use highlighting?
7 Highlight the "tag 1 of >2" message. New highlight group, or same as "hit bottom" search message.
7 When using ":tag" at the top of the tag stack, should add another entry, so CTRL-T can bring you back to where you are now AND to where you were before the previous ":tag" command. (Webb)
- When doing "[^I" or "[^D" add position to tag stack.
- Add command to put current position to tag stack: ":tpush".
- Add functions to save and restore the tag stack? Or a command to switch to another tag stack? So that you can do something else and come back to what you were working on.
7 When using CTRL-] on someClass::someMethod, separate class from method and use ":ta class:someClass someMethod".
Include C++ tags changes (Bertin). Change "class::func" tag into "func" with "class=class"? Docs in oldmail/bertin/in.xxx.
7 Add ":tagargs", to set values for fields:
:tagargs class:someclass file:version.c
:tagargs clear
These are then the default values (changes the order of priority in tag matching).
7 Support for "gtags" and "global"? With ":rtag" command?
There is an example for how to do this in Nvi.
Or do it like Elvis: 'tagprg' and 'tagprgonce' options. (Yamaguchi)
The Elvis method is far more flexible, do it that way.
7 Support "col:99" extra field, to position the cursor in that column. With a flag in 'cpoptions' to switch it off again.
7 Better support for jumping to where a function or variable is used. Use the id-utils, with a connection to "gid" (Emacs can do it too). Add ":idselect", which uses an "ID" database (made by "mkid") like "tselect".

Win32 GUI:
8 Make debug mode work while starting up (vim -D). Open console window for the message and input?
7 GvimExt: when there are several existing Vims, move the list to a submenu. (Mike McCollister)
8 When using "Edit with Vim" for one file it changes directory, when several files are selected and using "Edit with single Vim" the directory isn't changed. At least change directory when the path is the same for all files. Perhaps just use the path of the first file or use the longest common part of the path.
8 Add font argument to set the lfCharSet. (Bobcik)
8 Somehow automatically detect the system language and set $LANG, so that gettext and menus work.
8 Could keep console open to run multiple commands, to avoid the need to hit return in every console.
Also: Look at how Emacs does run external commands:
http://www.cs.washington.edu/homes/voelker/ntemacs.html.
8 Need a separate PopUp menu for modeless selection. Need two new commands: Copy selection to clipboard, Paste selection (as typed text).

8 Support copy/paste for other file formats. At least HTML, perhaps RTF.
Add "copy special" and "paste special" commands?
7 Use different default colors, to match the current Windows color scheme.
Sys_WindowText, Sys_Window, etc. (Lionel Schaffhauser)
7 Use <C-Tab> to cycle through open windows (e.g., the find dialog).
7 <Esc> should close a dialog.
7 Keep the console for external commands open. Don't wait for a key to be
hit. Re-open it when the user has closed it anyway. Or use a prepended
command: ":nowait {cmd}", or ":quiet", which executes {cmd} without any
prompts.
7 Should be able to set an option so that when you double click a file that
is associated with Vim, you can either get a new instance of Vim, or have
the file added into an already running Vim.
7 The "-P" argument only works for the current codepage. Use wide
functions to find the window title.

GUI:
8 Make inputdialog() work for Photon, Amiga.
- <C--> cannot be mapped. Should be possible to recognize this as a
normal "-" with the Ctrl modifier.
7 Implement ":popup" for other systems than Windows.
8 Implement ":tearoff" for other systems than Win32 GUI.
6 Implement ":untearoff": hide a torn-off menu.
8 When using the scrollbar to scroll, don't move the cursor position. When
moving the cursor: scroll to the cursor position.
9 Make <S-Insert> paste from the clipboard by default. (Kunze)
7 Menu local to a buffer, like mappings. Or local to a filetype?
8 In Buffers menu, add a choice whether selecting a buffer opens it in the
current window, splits the window or uses ":hide".
8 Dragging the mouse pointer outside of a Vim Window should make the text
scroll. Return a value from gui_send_mouse_event() to the machine
specific code to indicate the time in which the event should be repeated.
8 Make it possible to ignore a mouse click when it's used to give Vim (gvim)
window focus. Also when a mouse click is used to bring a window to front.
8 Make the split into system independent code and system specific code more
explicit. There are too many #ifdefs in gui.c.
If possible, separate the Vim code completely from the GUI code, to allow
running them in separate processes.
7 X11: Support cursorColor resource and "-cr" argument.
8 X11 (and others): CTRL-; is not different from ';'. Set the modifier mask
to include CTRL for keys where CTRL produces the same ASCII code.
7 Add some code to handle proportional fonts on more systems? Need to draw
each character separately (like xterm). Also for when a double-width font
is not exactly double-width. (Maeda)
8 Should take font from xterm where gvim was started (if no other default).
8 Selecting font names in X11 is difficult, make a script or something to
select one.
8 Visual highlighting should keep the same font (bold, italic, etc.).
8 Add flag to 'guioptions' to not put anything in the clipboard at all?
8 Should support a way to use keys that we don't recognize yet. Add a
command that adds entries to special_keys somehow. How do we make this
portable (X11, Win32, ..)?
7 Add a flag to 'guioptions' that tells not to remove inactive menu items.

For systems where greying-out or removing menu items is very slow. The menu items would remain visibly normally, but not do anything.

7 Add ":minimize" and ":maximize", which iconize the window and back. Useful when using gvim to run a script (e.g. 2html.vim).

7 X11: Is it possible to free allocated colors, so that other programs can use them again? Otherwise, allow disabling allocating the default colors. Or allocate an own colormap (check UAE). With an option to use it. For the commandline, "-install" is mostly used for X11 programs.

7 Should support multi-column menus.

- Should add option for where to put the "Help" menu: like Motif at the far right, or with the other menus (but still at the right).

- Add menu item to "Keep Insert mode".

8 ":mkgvimrc" command, that includes menus.

6 Big change: Move GUI to separate program "vimgui", to make startup of vim a lot faster, but still be able to do "vim -g" or ":gui".

7 More explicit mouse button binding instead of 'mousemodel'?

7 Add option to set the position of the window on the screen. 'windowpos', which has a value of "123,456": <x>,<y>.
Or add a command, like ":winsize"?

7 Add toolbar for more GUIs.

8 Make it possible to use "amenu icon=BuiltIn##", so that the toolbar item name can be chosen free.

7 Make it possible to put the toolbar on top, left, right and/or bottom of the window? Allows for softkey-like use.

6 Separate the part of Vim that does the editing from the part that runs the GUI. Communicate through a pseudo-tty. Vim starts up, creates a pty that is connected to the terminal. When the GUI starts, the pty is reconnected to the GUI process. When the GUI stops, it is connected to the terminal again. Also use the pty for external processes, it looks like a vt100 terminal to them. Vim uses extra commands to communicate GUI things.

7 Motif: For a confirm() dialog <Enter> should be ignored when no default button selected, <Esc> should close the dialog.

7 When using a pseudo-tty Vim should behave like some terminal (vt52 looks simple enough). Terminal codes to/from shell should be translated.

- Would it be useful to be able to quit the GUI and go back to the terminal where it was started from?

7 Support "-visual <type>" command line argument.

自动命令:

9 Rework the code from FEAT_OSFILETYPE for autocmd-osfiletypes to use 'filetype'. Only for when the current buffer is known.

- Put autocommand event names in a hashtable for faster lookup?

8 When the SwapExists event is triggered, provide information about the swap file, e.g., whether the process is running, file was modified, etc. Must be possible to check the situation that it's probably OK to delete the swap file. (Marc Merlin)

8 When all the patterns for an event are "*" there is no need to expand buffer names to a full path. This can be slow for NFS.

7 For autocommand events that trigger multiple times per buffer (e.g., CursorHold), go through the list once and cache the result for a specific buffer. Invalidate the cache when adding/deleting autocommands or changing the buffer name.

```
7   Add TagJump event: do something after jumping to a tag.
8   Add "TagJumpFile" autocommand: When jumping to another file for a tag.
    Can be used to open "main.c.gz" when "main.c" isn't found.
8   Use another option than 'updatetime' for the CursorHold event.  The two
    things are unrelated for the user (but the implementation is more
    difficult).
7   Add autocommand event for when a buffer cannot be abandoned.  So that the
    user can define the action taking (autowrite, dialog, fail) based on the
    kind of file. (Yakov Lerner)  Or is BufLeave sufficient?
8   Autocommand for when modified files have been found, when getting input
    focus again (e.g., FileChangedFocus).
    Check when: getting focus, jumping to another buffer, ...
8   Autocommands should not change registers.  And marks?  And the jumplist?
    And anything else?  Add a command to save and restore these things.
8   Add autocommands, user functions and user commands to ":mkvimrc".
6   Add KeymapChanged event, so that the effects of a different keymap can be
    handled (e.g., other font) (Ron Aaron)
7   When trying to open a directory, trigger an OpenDirectory event.
7   Add file type in front of file pattern: <d> for directory, <l> for link,
    <x> for executable, etc.  With commas to separate alternatives.  The
    autocommand is only executed when both the file type AND the file pattern
    match. (Leonard)
5   Add option that specifies extensions which are to be discarded from the
    file name.  E.g. 'ausuffix', with ".gz,.orig".  Such that file.c.gz will
    trigger the "*.c" autocommands.  (Belabas)
7   Add something to break the autocommands for the current event, and for
    what follows.  Useful for a "BufWritePre" that wants to avoid writing the
    file.
8   When editing "tt.gz", which is in DOS format, 'fileformat' stays at
    "unix", thus writing the file changes it.  Somehow detect that the read
    command used dos fileformat.  Same for 'fileencoding'.
-   Add events to autocommands:
    Error           - When an error happens
    ModeChange      - after changing mode (before waiting for a char)
    VimLeaveCheck    - Before Vim decides to exit, so that it can be cancelled
                       when exiting isn't a good idea.
    CursorHoldC     - CursorHold while command-line editing
    WinMoved        - when windows have been moved around, e.g, ":wincmd J"
    SearchPost      - After doing a search command (e.g. to do "M")
    PreDirChanged/PostDirChanged
                    - Before/after ":cd" has been used (for changing the
                      window title)
    ShutDown        - when the system is about to shut down
    InsertCharPost  - user typed a character in Insert mode, after inserting
                      the char.
    BufModified     - When a buffer becomes modified, or unmodified (for
                      putting a [+] in the window title or checking out the
                      file from CVS).
    BufFirstChange  - When making a change, when 'modified' is set.  Can be
                      used to do a :preserve for remote files.
    BufChange       - after a change was made.  Set some variables to indicate
                      the position and number of inserted/deleted lines, so
                      that marks can be updated.  HierAssist has patch to add
                      BufChangePre, BufChangePost and RevertBuf. (Shah)
```

```
    ViewChanged     - triggered when the text scrolls and when the window size
                      changes.
    WinResized      - After a window has been resized
    WinClose        - Just before closing a window
-   Write the file now and then ('autosave'):
                                  'autosave'   'as'   'noautosave'   'noas'
    'autosave' 'as' number  (default 0)
            Automatically write the current buffer to file N seconds after the
            last change has been made and when  'modified'  is still set.
            Default: 0 = do not autosave the buffer.
    Alternative: have 'autosave' use 'updatetime' and 'updatecount' but make
    them save the file itself besides the swapfile.
```

全能 (omni) 补全:

```
-   Add a flag to 'complete' to be able to do omni completion with CTRL-N (and
    mix it with other kinds of completion).
-   Ideas from the Vim 7 BOF at SANE:
        - For interpreted languages, use the interpreter to obtain information.
          Should work for Java (Eclipse does this), Python, Tcl, etc.
          Richard Emberson mentioned working on an interface to Java.
        - Check Readline for its completion interface.
-   Ideas from others:
        http://www.wholetomato.com/
        http://www.vim.org/scripts/script.php?script_id=747
            http://sourceforge.net/projects/insenvim
                or http://insenvim.sourceforge.net
            Java, XML, HTML, C++, JSP, SQL, C#
            MS-Windows only, lots of dependencies (e.g. Perl, Internet
                explorer), uses .dll shared libraries.
            For C++ uses $INCLUDE environment var.
            Uses Perl for C++.
            Uses ctags to find the info:
                ctags -f $allTagsFile --fields=+aiKmnsSz --language-force=C++ --C++-kinds=+cefgmnp
        www.vim.org script 1213 (Java Development Environment) (Fuchuan Wang)
        IComplete: http://www.vim.org/scripts/script.php?script_id=1265
            and http://stud4.tuwien.ac.at/~e0125672/icomplete/
        http://cedet.sourceforge.net/intellisense.shtml (for Emacs)
        Ivan Villanueva has something for Java.
        Emacs: http://www.xref-tech.com/xrefactory/more_c_completion.html
        Completion in .NET framework SharpDevelop: http://www.icsharpcode.net
-   Pre-expand abbreviations, show which abbrevs would match?
```

插入模式自动补全/扩展:

```
-   GUI implementation of the popup menu.
7   When searching in other files the name flash by, too fast to read.  Only
    display a name every second or so, like with ":vimgrep".
7   When expanding file names with an environment variable, add the match with
    the unexpanded var.  So $HOME/tm expands to "/home/guy/tmp" and
    "$HOME/tmp"
8   When there is no word before the cursor but something like "sys." complete
    with "sys.".  Works well for C and similar languages.
9   ^X^L completion doesn't repeat correctly.  It uses the first match with
```

the last added line, instead of continuing where the last match ended. (Webb)

8 Add option to set different behavior for Insert mode completion:
- ignore/match case
- different characters than 'iskeyword'

8 Add option 'isexpand', containing characters when doing expansion (so that "." and "\" can be included, without changing 'iskeyword'). (Goldfarb)
Also: 'istagword': characters used for CTRL-].
When 'isexpand' or 'istagword' are empty, use 'iskeyword'.
Alternative: Use a pattern so that start and end of a keyword can be defined, only allow dash in the middle, etc.

8 Add a command to undo the completion, go back to the original text.

7 Completion of an abbreviation: Can leave letters out, like what Instant text does: www.textware.com

8 Use the class information in the tags file to do context-sensitive completion. After "foo." complete all member functions/variables of "foo". Need to search backwards for the class definition of foo.
Should work for C++ and Java.
Even more context would be nice: "import java.^N" -> "io", "lang", etc.

7 When expanding $HOME/dir with ^X^F keep the $HOME (with an option?).

7 Add CTRL-X command in Insert mode like CTRL-X CTRL-N, that completes WORDS instead of words.

8 Add CTRL-X CTRL-R: complete words from register contents.

8 Add completion of previously inserted texts (like what CTRL-A does). Requires remembering a number of insertions.

8 Add 'f' flag to 'complete': Expand file names.
Also apply 'complete' to whole line completion.

- Add a flag to 'complete' to only scan local header files, not system header files. (Andri Moell)

- Make it possible to search include files in several places. Use the 'path' option? Can this be done with the dictionary completion (use wildcards in the file name)?

- Make CTRL-X CTRL-K do a binary search in the dictionary (if it's sorted).

- Speed up CTRL-X CTRL-K dictionary searching (don't use a regexp?).

- Set a mark at the position where the match was found (file mark, could be in another file).

- Add CTRL-A command in CTRL-X mode: show all matches.

- Make CTRL-X CTRL-L use the 'complete' option?

- Add command in CTRL-X mode to add following words to the completed string (e.g. to complete "Pointer->element" with CTRL-X CTRL-P CTRL-W CTRL-W)

- CTRL-X CTRL-F: Use 'path' to find completions.

- CTRL-X CTRL-F: Option to use forward slashes on MS-Windows?

- CTRL-X CTRL-F: Don't replace "$VIM" with the actual value. (Kelly)

- Allow listing all matches in some way (and picking one from the list).

命令行编辑:

7 Add commands (keys) to delete from the cursor to the end of the command line.

8 Custom completion of user commands can't use the standard completion functions. Add a hook to invoke a user function that returns the type of completion to be done: "file", "tag", "custom", etc.

- Add flags to 'whichwrap' for command line editing (cursor right at end of lines wraps to start of line).

- Make editing the command line work like Insert mode in a single-line view on a buffer that contains the command line history. But this has many disadvantages, only implement it when these can be solved. Elvis has run into these, see remarks from Steve (~/Mail/oldmail/kirkendall/in.00012).
  - Going back in history and editing a line there would change the history. Would still need to keep a copy of the history elsewhere. Like the cmdwin does now already.
  - Use CTRL-O to execute one Normal mode command. How to switch to normal mode for more commands? <Esc> should cancel the command line. CTRL-T?
  - To allow "/" and "= need to recursively call getcmdline(), overwrite the cmdline. But then we are editing a command-line again. How to avoid that the user gets confused by the stack of command lines?
  - Use edit() for normal cmdline editing? Would have to integrate getcmdline() into edit(). Need to solve conflicts between Insert mode and Command-line mode commands. Make it work like Korn shell and tcsh. Problems:
    - Insert: completion with 'wildchar'
    - Insert: use cmdline abbreviations
    - Insert: CTRL-D deletes indent instead of listing matches
    - Normal: no CTRL-W commands
    - Normal: no ":" commands?
    - Normal: allow Visual mode only within one line.
  - where to show insert/normal mode message? Change highlighting of character in first column?
  - Implementation ideas:
    - Set "curwin" and "curbuf" to the command line window and buffer.
    - curwin->w_topline is always equal to curwin->w_cursor.lnum.
    - never set 'number', no folding, etc. No status line.
    - sync undo after entering a command line?
    - use NV_NOCL flag for commands that are not allowed in Command-line Mode.

命令行自动补全:

8 Change expand_interactively into a flag that is passed as an argument.

8 With command line completion after '%' and '#', expand current/alternate file name, so it can be edited. Also with modifiers, such as "%:h".

8 When completing command names, either sort them on the long name, or list them with the optional part inside [].

8 Add an option to ignore case when doing interactive completion. So that ":e file<Tab>" also lists "Filelist" (sorted after matching case matches).

7 Completion of ":map x ": fill in the current mapping, so that it can be edited. (Sven Guckes)

- For 'wildmenu': Simplify "../bar" when possible.

- When using <Up> in wildmenu mode for a submenu, should go back to the current menu, not the first one. E.g., ":emenu File.Save<Up>".

8 When using backtick expansion, the external command may write a greeting message. Add an option or commands to remove lines that match a regexp?

7 When listing matches of files, display the common path separately from the file names, if this makes the listing shorter. (Webb)

- Add command line completion for ":ilist" and friends, show matching identifiers (Webb).

8 Add command line completion for "old value" of a command. ":args <key>" would result in the current list of arguments, which you can then edit.

7 Add command line completion with CTRL-X, just like Insert mode completion. Useful for ":s/word/xx/".
- Add command to go back to the text as it was before completion started. Also to be used for <Up> in the command line.
- Add 'wildlongest' option: Key to use to find longest common match for command line completion (default CTRL-L), like 'wildchar'. (Cregut) Also: when there are several matches, show them line a CTRL-D.

命令行历史:
- Add "KeyWasTyped" flag: It's reset before each command and set when a character from the keyboard is consumed. Value is used to decide to put a command line in history or not. Put line in history if it didn't completely result from one mapping.
- When using ":browse", also put the resulting edit command in the history, so that it can be repeated. (Demirel)

插入模式:
9 When 'autoindent' is set, hitting <CR> twice, while there is text after the cursor, doesn't delete the autoindent in the resulting blank line. (Rich Wales) This is Vi compatible, but it looks like a bug.
8 When using CTRL-O in Insert mode, then executing an insert command "a" or "i", should we return to Insert mode after <Esc>? (Eggink) Perhaps it can be allowed a single time, to be able to do "<C-O>10axyz<Esc>". Nesting this further is confusing. ":map <F2> 5aabc<Esc>" works only once from Insert mode.
8 When using CTRL-G CTRL-O do like CTRL-\ CTRL-O, but when returning with the cursor in the same position and the text didn't change continue the same change, so that "." repeats the whole insert.
7 Use CTRL-G <count> to repeat what follows. Useful for inserting a character multiple times or repeating CTRL-Y.
- Make 'revins' work in Replace mode.
7 Use 'matchpairs' for 'showmatch': When inserting a character check if it appears in the rhs of 'matchpairs'.
- In Insert mode (and command line editing?): Allow undo of the last typed character. This is useful for CTRL-U, CTRL-W, delete and backspace, and also for characters that wrap to the next line.
Also: be able to undo CTRL-R (insert register).
Possibly use 'backspace'="whole" for a mode where at least a <CR> that inserts autoindent is undone by a single <BS>.
- Use CTRL-G in Insert mode for an extra range of commands, like "g" in Normal mode.
- Make 'paste' work without resetting other options, but override their value. Avoids problems when changing files and modelines or autocommands are used.
- When typing CTRL-V and a digit higher than 2, only expect two digits.
- Insert binary numbers with CTRL-V b.
- Make it possible to undo <BS>, <C-W> and <C-U>. Bash uses CTRL-Y.

'cindent'、'smartindent':
9 Wrapping a variable initialization should have extra indent:
```
    char * veryLongName =
```

```
            "very long string"
    Also check if "cino=+10" is used correctly.
8   Lisp indenting: "\\" confuses the indenter. (Dorai Sitaram, 2006 May 17)
8   Why are continuation lines outside of a {} block not indented?  E.g.:
        long_type foo =
        value;
8   Java: Inside an anonymous class, after an "else" or "try" the indent is
    too small. (Vincent Bergbauer)
    Problem of using {} inside (), 'cindent' doesn't work then.
8   In C++ it's possible to have {} inside (): (Kirshna)
            func(
                    new String[] {
                        "asdf",
                        "asdf"
                    }
                );
8   In C++ a function isn't recognized inside a namespace:
    (Chow Loong Jin)
        namespace {
            int
                 func(int arg) {
                 }
        }
6   Add 'cino' flag for this function argument layout: (Spencer Collyer)
            func( arg1
                , arg2
                , arg3
                );
7   Add separate "(0" option into inside/outside a function (Zellner):
        func(
           int x)       // indent like "(4"
        {
           if (a
               && b)    // indent like "(0"
9   Using "{" in a comment: (Helmut Stiegler)
    if (a)
    {
            if (b)
            {
                    // {
            }
            } <-- this is indented incorrect
    Problem is that find_start_brace() checks for the matching brace to be in
    a comment, but not braces in between.  Requires adding a comment check to
    findmatchlimit().
-   Make smartindenting configurable.  Add 'sioptions', e.g. '#' setting the
    indent to 0 should be switched on/off.
7   Support ANSI style function header, with each argument on its own line.
-   "[p" and "]p" should use 'cindent' code if it's on (only for the first
    line).
-   Add option to 'cindent' to set indent for comments outside of {}?
-   Make a command to line up a comment after a code line with a previous
    comment after a code line.  Can 'cindent' do this automatically?
-   When 'cindent'ing a '}', showmatch is done before fixing the indent.  It
```

looks better when the indent is fixed before the showmatch. (Webb)
- Add option to make indenting work in comments too (for commented-out code), unless the line starts with "*".
- Don't use 'cindent' when doing formatting with "gq"?
- When formatting a comment after some text, insert the '*' for the new line (indent is correct if 'cindent' is set, but '*' doesn't get inserted).
8 When 'comments' has both "s1:/*,mb:*,ex:*/" and "s1:(*,mb:*,ex:*)", the 'x' flag always uses the first match. Need to continue looking for more matches of "*" and remember all characters that could end the comment.
- For smartindent: When typing 'else' line it up with matching 'if'.
- 'smartindent': allow patterns in 'cinwords', for e.g. TeX files, where lines start with "\item".
- Support this style of comments (with an option): (Brown)

```
/* here is a comment that
is just autoindented, and
nothing else */
```

- Add words to 'cinwords' to reduce the indent, e.g., "end" or "fi".
7 Use Tabs for the indent of starting lines, pad with spaces for continuation lines. Allows changing 'tabstop' without messing up the indents.
Patch by Lech Lorens, 2010 Mar. Update by James McCoy, 2014 Mar 15.

Java:
8 Can have {} constructs inside parens. Include changes from Steve Odendahl?
8 Recognize "import java.util.Vector" and use $CLASSPATH to find files for "[i" commands and friends.
- For files found with 'include': handle "*" in included name, for Java. (Jason)
- How to make a "package java.util" cause all classes in the package to be searched? Also for "import java.util.*". (Mark Brophy)

'comments':
8 When formatting C comments that are after code, the "*" isn't repeated like it's done when there is no code. And there is no automatic wrapping. Recognize comments that come after code. Should insert the comment leader when it's "#" or "//".
Other way around: when a C command starts with "* 4" the "*" is repeated while it should not. Use syntax HL comment recognition?
7 When using "comments=fg:--", Vim inserts three spaces for a new line. When hitting a TAB, these spaces could be removed.
7 The 'n'esting flag doesn't do the indenting of the last (rightmost) item.
6 Make strings in 'comments' option a RE, to be able to match more complicated things. (Phillipps) Use a special flag to indicate that a regexp is used.
8 Make the 'comments' option with "/* * */" lines only repeat the "*" line when there is a "/*" before it? Or include this in 'cindent'?

虚拟编辑:
8 Make the horizontal scrollbar work to move the text further left.
7 Allow specifying it separately for Tabs and beyond end-of-line?

文本对象：

8 Add text object for fold, so that it can be yanked when it's open.

8 Add test script for text object commands "aw", "iW", etc.

8 Add text object for part of a CamelHumpedWord and under_scored_word. (Scott Graham) "ac" and "au"?

8 Add a text object for any kind of quoting, also with multi-byte characters. Option to specify what quotes are recognized (default: all) use "aq" and "iq". Use 'quotepairs' to define pairs of quotes, like 'matchpairs'?

8 Add text object for any kind of parens, also multi-byte ones.

8 Add a way to make an ":omap" for a user-defined text object. Requires changing the starting position in oap->start.

8 Add "gp" and "gP" commands: insert text and make sure there is a single space before it, unless at the start of the line, and after it, unless at the end of the line or before a ".".

7 Add objects with backwards extension? Use "I" and "A". Thus "2dAs" deletes the current and previous sentence. (Jens Paulus)

7 Add "g{" and "g}" to move to the first/last character of a paragraph (instead of the line just before/after a paragraph as with "{" and "}").

6 Ignore comment leaders for objects. Make "das" work in reply-email.

5 Make it possible to use syntax group matches as a text object. For example, define a "ccItem" group, then do "da<ccItem>" to delete one. Or, maybe just define "dai", delete-an-item, to delete the syntax item the cursor is on.

选择模式：

8 In blockwise mode, typed characters are inserted in front of the block, backspace deletes a column before the block. (Steve Hall)

7 Alt-leftmouse starts block mode selection in MS Word.
See http://vim.wikia.com/wiki/Use_Alt-Mouse_to_select_blockwise.

7 Add Cmdline-select mode. Like Select mode, but used on the command line.
- Change gui_send_mouse_event() to pass on mouse events when 'mouse' contains 'C' or 'A'.
- Catch mouse events in ex_getln.c. Also shift-cursor, etc., like in normal_cmd().
- remember start and end of selection in cmdline_info.
- Typing text replaces the selection.

可视模式：

8 Support using "." in Visual mode. Use the operator applied to the Visual selection, if possible.

- When dragging the Visual selection with the mouse and 'scrolloff' is zero, behave like 'scrolloff' is one, so that the text scrolls when the pointer is in the top line.

- Displaying size of Visual area: use 24-33 column display.
When selecting multiple lines, up to about a screenful, also count the characters.

8 When using "I" or "A" in Visual block mode, short lines do not get the new text. Make it possible to add the text to short lines too, with padding where needed.

```
7   With a Visual block selected, "2x" deletes a block of double the width,
    "3y" yanks a block of triple width, etc.
7   When selecting linewise, using "itext" should insert "text" at the start
    of each selected line.
8   What is "R" supposed to do in Visual mode?
8   Make Visual mode local to the buffer.  Allow changing to another buffer.
    When starting a new Visual selection, remove the Visual selection in any
    other buffer. (Ron Aaron)
8   Support dragging the Visual area to drop it somewhere else. (Ron Aaron,
    Ben Godfrey)
7   Support dragging the Visual area to drop it in another program, and
    receive dropped text from another program. (Ben Godfrey)
7   With blockwise Visual mode and "c", "C", "I", "A", etc., allow the use of
    a <CR>.  The entered lines are repeated over the Visual area.
7   Filtering a block should only apply to the block, not to the whole lines.
    When the number of lines is increased, add lines. When decreased, pad with
    spaces or delete? Use ": <, >" on the command line.
8   After filtering the Visual area, make "gv" select the filtered text?
    Currently "gv" only selects a single line, not useful.
7   Don't move the cursor when scrolling?  Needed when the selection should
    stay the same.  Scroll to the cursor at any movement command.  With an
    option!
7   In Visual block mode, need to be able to define a corner on a position
    that doesn't have text?  Also: when using the mouse, be able to select
    part of a TAB.  Even more: Add a mode where the cursor can be on a screen
    position where there is no text.  When typing, add spaces to fill the gap.
    Other solution: Always use curswant, so that you can move the cursor to
    the right column, and then use up/down movements to select the line,
    without changing the column.
6   ":left" and ":right" should work in Visual block mode.
7   CTRL-I and CTRL-O should work in Visual mode, but only jump to marks in the
    current buffer.
6   In non-Block mode, "I" should insert the same text in front of each line,
    before the first non-blank, "gI" in column 1.
6   In non-Block mode, "A" should append the same text after each line.
6   When in blockwise visual selection (CTRL-V), allow cursor to be placed
    right of the line.  Could also allow cursor to be placed anywhere on a TAB
    or other special character.
6   Add commands to move selected text, without deselecting.
```

更多高级的重复命令:

```
-   Add "." command for visual mode: redo last visual command (e.g. ":fmt").
-   Add command to repeat last movement.  Including count.
-   Add "." command after operator: repeat last command of same operator.  E.g.
    "c." will repeat last change, also when "x" used since then (Webb).
    "y." will repeat last yank.
    "c2." will repeat the last but one change?
    Also: keep history of Normal mode commands, add command to list the history
    and/or pick an older command.
-   History stack for . command?  Use "g." command.
```

映射和缩写:

8 When "0" is mapped (it is a movement command) this mapping should not be used after typing another number, e.g. "20l". (Charles Campbell) Is this possible without disabling the mapping of the following command?
8 Should mapping <C-A> and <C-S-A> both work?
7 ":abbr b byte", append "b " to an existing word still expands to "byte". This is Vi compatible, but can we avoid it anyway?
8 To make a mapping work with a prepended "x to select a register, store the last _typed_ register name and access it with "&.
8 Add ":amap", like ":amenu".
7 Add a mapping that works always, for remapping the keyboard.
8 Add ":cab!", abbreviations that only apply to Command-line mode and not to entering search strings.
8 Add a flag to ":abbrev" to eat the character that triggers the abbreviation. Thus "abb ab xxx" and typing "ab<Space>" inserts "xxx" and not the <Space>.
8 Give a warning when using CTRL-C in the lhs of a mapping. It will never (?) work.
7 Add <0x8f> (hex), <033> (octal) and <123> (decimal) to <> notation?
7 When someone tries to unmap with a trailing space, and it fails, try unmapping without the trailing space. Helps for ":unmap xx | unmap yy".
6 Context-sensitive abbreviations: Specify syntax group(s) in which the abbreviations are to be used.
- Add mappings that take arguments. Could work like the ":s" command. For example, for a mouse escape sequence:

```
    :mapexp  <Esc>{\([0-9]*\),\([0-9]*\);   H\1j\2l
```

- Add optional <Number> argument for mappings:

```
:map <Number>q           ^W^W<Number>G
:map <Number>q<Number>t  ^W^W<Number1-1>G<Number2>l
:map q<Char>            :s/<Char>/\u\0/g
Or implicit:
:map q                      <Register>d<Number>$
```

- Add command to repeat a whole mapping ("." only repeats the last change in a mapping). Also: Repeat a whole insert command, including any mappings that it included. Sort-of automatic recording?
- Include an option (or flag to 'cpoptions') that makes errors in mappings not flush the rest of the mapping (like nvi does).
- Use context sensitiveness of completion to switch abbreviations and mappings off for :unab and :unmap.
6 When using mappings in Insert mode, insert characters for incomplete mappings first, then remove them again when a mapping matches. Avoids that characters that are the start of some mapping are not shown until you hit another character.
- Add mappings for replace mode: ":rmap". How do we then enter mappings for non-replace Insert mode?
- Add separate mappings for Visual-character/block/line mode?
- Add 'mapstop' command, to stop recursive mappings.
- List mappings that have a raw escape sequence both with the name of the key for that escape sequence (if there is one) and the sequence itself.
- List mappings: Once with special keys listed as <>, once with meta chars as <M-a>, once with the byte values (octal?). Sort of "spell mapping" command?
- When entering mappings: Add the possibility to enter meta keys like they are displayed, within <>: <M-a>, <^@> or <|a>.
- Allow multiple arguments to :unmap.
- Command to show keys that are not used and available for mapping

":freekeys".
- Allow any character except white space in abbreviations lhs (Riehm).

增量搜索:
- Wrong scrolling when using incsearch. Patch by Christian Brabandt, 2014 Dec 4. Is this a good solution?
- Temporarily open folds to show where the search ends up. Restore the folds when going to another line.
Patch to open folds for 'incsearch'. (Christian Brabandt, 2015 Jan 6)
- Bug with 'incsearch' going to wrong line. (Wolfram Kresse, 2009 Aug 17)
Only with "vim -u NONE".
- When no match is found and the user types more, the screen is redrawn anyway. Could skip that. Esp. if the line wraps and the text is scrolled up every time.
- When incsearch used and hitting return, no need to search again in many cases, saves a lot of time in big files. (Slootman wants to work on this?)
When not using special characters, can continue search from the last match (or not at all, when there was no match). See oldmail/webb/in.872.
9 incsearch is incorrect for "/that/<Return>/this/;//" (last search pattern isn't updated).

搜索:
9 Should have an option for :vimgrep to find lines without a match.
8 Add "g/" and "gb" to search for a pattern in the Visually selected text? "g?" is already used for rot13.
The vis.vim script has a ":S" command that does something like this.
Can use "g/" in Normal mode, uses the '< to '> area.
Use "&/" for searching the text in the Visual area?
9 Add "v" offset: "/pat/v": search for pattern and start Visual mode on the matching text.
8 Add a modifier to interpret a space like "_s\+" to make it much easier to search for a phrase.
8 Add a mechanism for recursiveness: "\@(([^()]*\@g[^()]*)\)". \@g stands for "go recursive here" and \@(\) marks the recursive part.
Perl does it this way:
```
        $paren = qr/ \(( [^()] | (??{ $paren }) )* \) /x;
```
Here $paren is evaluated when it's encountered. This is like a regexp inside a regexp. In the above terms it would be:
```
        \@((\([^()]\|\@g\)*)\)
```
8 Show the progress every second. Could use the code that checks for CTRL-C to find out how much time has passed. Or use SIGALRM. Where to show the number?
7 Support for approximate-regexps to find similar words (agrep http://www.tgries.de/agrep/ tre: http://laurikari.net/tre/index.html).
8 Add an item for a big character range, so that one can search for a chinese character: \z[234-1234] or \z[XX-YY] or \z[0x23-0x234].
7 Add an item stack to allow matching (). One side is "push X on the stack if previous atom matched". Other side is "match with top of stack, pop it when it matches". Use "\@pX" and "\@m"?
Example: \((\@p).\{-}\@m\)*
7 Add a flag to "/pat/" to discard an error. Useful to continue a mapping when a search fails. Could be "/pat/E" (e is already used for end offset).

7 Add pattern item to use properties of Unicode characters. In Perl it's "\p{L}" for a letter. See Regular Expression Pocket Reference.

8 Would it be possible to allow ":23,45/pat/flags" to search for "pat" in lines 23 to 45? Or does this conflict with Ex range syntax?

8 Allow identical pairs in 'matchpairs'. Restrict the search to the current line.

7 Allow longer pairs in 'matchpairs'. Use matchit.vim as an example.

8 Make it possible to define the character that "%" checks for in #if/#endif. For nmake it's !if/!endif.

- For "%" command: set hierarchy for which things include other things that should be ignored (like "*/" or "#endif" inside /* */).
Also: use "%" to jump from start to end of syntax region and back.
Alternative: use matchit.vim

8 A pattern like "\([^a]\+\)\+" takes an awful long time. Recognize that the recursive "\+" is meaningless and optimize for it.
This one is also very slow on "/* some comment */": "^\/*\(.*[^/]\)*$".

7 Recognize "[a-z]", "[0-9]", etc. and replace them with the faster "\l" and "\d".

7 Add a way to specify characters in <C-M> or <Key> form. Could be \%<C-M>.

8 Add an argument after ":s/pat/str/" for a range of matches. For example, ":s/pat/str/#3-4" to replace only the third and fourth "pat" in a line.

8 When 'iskeyword' is changed the matches from 'hlsearch' may change. (Benji Fisher) redraw if some options are set while 'hlsearch' is set?

8 Add an option not to use 'hlsearch' highlighting for ":s" and ":g" commands. (Kahn) It would work like ":noh" is used after that command.
Also: An extra flag to do this once, and a flag to keep the existing search pattern.

- Make 'hlsearch' a local/global option, so that it can be disabled in some of the windows.

- Add \%h{group-name}; to search for a specific highlight group.
Add \%s{syntax-group}; to search for a specific syntax group.

- Support Perl regexp. Use PCRE (Perl Compatible RE) package. (Shade)
Or translate the pattern to a Vim one.
Don't switch on with an option for typed commands/mappings/functions, it's too confusing. Use "\@@" in the pattern, to avoid incompatibilities.

8 Add a way to access the last substitute text, what is used for ":s//~/".
Can't use the ~ register, it's already used for drag & drop.

- Remember flags for backreferenced items, so that "*" can be used after it.
Check with "\(\S\)\1\{3}". (Hemmerling)

8 Flags that apply to the whole pattern.
This works for all places where a regexp is used.
Add "\q" to not store this pattern as the last search pattern?

- Add flags to search command (also for ":s"?):
i ignore case
I use case
p use Perl regexp syntax (or POSIX?)
v use Vi regexp syntax
f forget pattern, don't keep it for "n" command
F remember pattern, keep it for "n" command
Perl uses these too:
e evaluate the right side as an expression (Perl only)
m multiple line expression (we don't need it)

o compile only once (Perl only)
s single line expression (we don't need it)
x extended regexp (we don't need it)
When used after ":g" command, backslash needed to avoid confusion with the following command.
Add 'searchflags' for default flags (replaces 'gdefault').

- Add command to display the last used substitute pattern and last used pattern. (Margo) Maybe make it accessible through a register (like "/ for search string)?

7 Use T-search algorithm, to speed up searching for strings without special characters. See C't article, August 1997.

- Add 'fuzzycase' option, so that case doesn't matter, and '-' and '_' are equivalent (for Unix filenames).
- Add 'v' flag to search command: enter Visual mode, with the matching text as Visual area. (variation on idea from Bertin)
- Searching: "/this//that/" should find "that" after "this".
- Add global search commands: Instead of wrapping at the end of the buffer, they continue in another buffer. Use flag after search pattern:
a for the next file in the argument list
f for file in the buffer list
w for file edited in a window.
e.g. "/pat/f". Then "n" and "N" work through files too. "f" flag also for ":s/pat/foo/f"??? Then when 'autowrite' and 'hidden' are both not set, ask before saving files: "Save modified buffer "/path/file"? (Yes/Hide/No Save-all/hide-All/Quit) ".
- ":s/pat/foo/3": find 3rd match of "pat", like sed. (Thomas Koehler)

7 When searching with 'n' give message when getting back where the search first started. Remember start of search in '/ mark.

7 Add option that scrolls screen to put cursor in middle of screen after search always/when off-screen/never. And after a ":tag" command. Maybe specify how many lines below the screen causes a redraw with the cursor in the middle (default would be half a screen, zero means always).

6 Support multiple search buffers, so macros can be made without side effects.

7 From xvim: Allow a newline in search patterns (also for :s, can delete newline). Add BOW, EOW, NEWL, NLORANY, NLBUTANY, magic 'n' and 'r', etc. [not in xvim:] Add option to switch on matches crossing ONE line boundary.

7 Add ":iselect", a combination of ":ilist" and ":tselect". (Aaron) (Zellner) Also ":dselect".

撤销:

9 ":gundo" command: global undo. Undoes changes spread over multiple files in the order they were made. Also ":gredo". Both with a count. Useful when tests fail after making changes and you forgot in which files.

9 After undo/redo, in the message show whether the buffer is modified or not.

8 Search for pattern in undo tree, showing when it happened and the text state, so that you can jump to it.

8 Undo tree: visually show the tree somehow (Damian Conway)
Show only the leaves, indicating how many changed from the branch and the timestamp?
Put branch with most recent change on the left, older changes get more indent?

- Make it possible to undo all the commands from a mapping, including a trailing unfinished command, e.g. for ":map K iX^[r".
- When accidentally hitting "R" instead of Ctrl-R, further Ctrl-R is not possible, even when typing <Esc> immediately. (Grahn) Also for "i", "a", etc. Postpone saving for undo until something is really inserted?

8 When Inserting a lot of text, it can only be undone as a whole. Make undo sync points at every line or word. Could recognize the start of a new word (white space and then non-white space) and backspacing.
Can already use CTRL-G u, but that requires remapping a lot of things.

8 Make undo more memory-efficient: Compare text before and after change, only remember the lines that really changed.

7 Add undo for a range of lines. Can change these back to a previous version without changing the rest of the file. Stop doing this when a change includes only some of these lines and changes the line count. Need to store these undo actions as a separate change that can be undone.

- For u_save() include the column number. This can be used to set '[and ']. And in the future the undo can be made more efficient (Webb).
- In out-of-memory situations: Free allocated space in undo, and reduce the number of undo levels (with confirmation).
- Instead of [+], give the number of changes since the last write: [+123]. When undoing to before the last write, change this to a negative number: [-99].
- With undo with simple line delete/insert: optimize screen updating.
- When executing macro's: Save each line for undo only once.
- When doing a global substitute, causing almost all lines to be changed, undo info becomes very big. Put undo info in swap file??

缓冲区列表:

7 Command to execute a command in another buffer: ":inbuf {bufname} {cmd}". Also for other windows: ":inwin {winnr} {cmd}". How to make sure that this works properly for all commands, and still be able to return to the current buffer/window? E.g.: ":inbuf xxx only".

8 Add File.{recent_files} menu entries: Recently edited files.
Ron Aaron has a plugin for this: mru.vim.

8 Unix: Check all uses of fnamecmp() and fnamencmp() if they should check inode too.

7 Add another number for a buffer, which is visible for the user. When creating a new buffer, use the lowest number not in use (or the highest number in use plus one?).

7 Offer some buffer selection from the command line? Like using ":ls" and asking for a buffer number. (Zachmann)

- When starting to edit a file that is already in the buffer list, use the file name argument for the new short file name. (Webb)
- Add an option to make ":bnext" and ":bprev" wrap around the end of the buffer list. Also for ":next" and ":prev"?

7 Add argument to ":ls" which is a pattern for buffers to list.
E.g. ":ls *.c". (Thompson)

7 Add expansion of buffer names, so that "*.c" is expanded to all buffer names. Needed for ":bdel *.c", ":bunload *.c", etc.

8 Support for <afile> where a buffer name is expected.

7 Add an option to mostly use slashes in file names. Separately for internal use and for when executing an external program?

8 Some file systems are case-sensitive, some are not. Besides

'wildignorecase' there might be more parts inside
CASE_INSENSITIVE_FILENAME that are useful on Unix.

交换 (.swp) 文件:

8 If writing to the swap file fails, should try to open one in another directory from 'dir'. Useful in case the file system is full and when there are short file name problems.

8 Also use the code to try using a short file name for the backup and swap file for the Win32 and Dos 32 bit versions.

8 When a file is edited by root, add $LOGNAME to know who did su.

8 When the edited file is a symlink, try to put the swap file in the same dir as the actual file. Adjust FullName(). Avoids editing the same file twice (e.g. when using quickfix). Also try to make the name of the backup file the same as the actual file?
Use the code for resolve()?

7 When using 64 bit inode numbers, also store the top 32 bits. Add another field for this, using part of bo_fname[], to keep it compatible.

7 When editing a file on removable media, should put swap file somewhere else. Use something like 'r' flag in 'viminfo'. 'diravoid'?
Also: Be able to specify minimum disk space, skip directory when not enough room.

7 Add a configure check for which directory should be used: /tmp, /var/tmp or /var/preserve.

- Add an option to create a swap file only when making the first change to the buffer. (Liang) Or only when the buffer is not read-only.

- Add option to set "umask" for backup files and swap files (Antwerpen). 'backupumask' and 'swapumask'? Or 'umaskbackup' and 'umaskswap'?

- When editing a readonly file, don't use a swap file but read parts from the original file. Also do this when the file is huge (>'maxmem'). We do need to load the file once to count the number of lines? Perhaps keep a cached list of which line is where.

Viminfo:

7 Can probably remove the code that checks for a writable viminfo file, because we now do the chown() for root, and others can't overwrite someone else's viminfo file.

8 When there is no .viminfo file and someone does "su", runs Vim, a root-owned .viminfo file is created. Is there a good way to avoid this? Perhaps check the owner of the directory. Only when root?

8 Add argument to keep the list of buffers when Vim is started with a file name. (Schild)

8 Keep the last used directory of the file browser (File/Open menu).

8 Remember the last used register for "@@".

8 Remember the redo buffer, so that "." works after restarting.

8 Remember a list of last accessed files. To be used in the "File.Open Recent" menu. Default is to remember 10 files or so.
Also remember which files have been read and written. How to display this?

7 Also store the ". register (last inserted text).

7 Make it possible to store buffer names in viminfo file relative to some directory, to make them portable over a network. (Aaron)

6 Store a snapshot of the currently opened windows. So that when quitting

Vim, and then starting again (without a file name argument), you see the same files in the windows. Use ":mksession" code?
- Make marks present in .viminfo usable as file marks: Display a list of "last visited files" and select one to jump to.

模式行:
8 Before trying to execute a modeline, check that it looks like one (valid option names). If it's very wrong, silently ignore it.
Ignore a line that starts with "Subject: ".
- Add an option to whitelist options that are allowed in a modeline. This would allow careful users to use modelines, e.g., only allowing 'shiftwidth'.
- Add an option to let modelines only set local options, not global ones such as 'encoding'.
- When an option value is coming from a modeline, do not carry it over to another edited file? Would need to remember the value from before the modeline setting.
- Allow setting a variable from a modeline? Only allow using fixed strings, no function calls, to avoid a security problem.
- Allow ":doauto BufRead x.cpp" in modelines, to execute autocommands for .cpp files.
- Support the "abbreviate" command in modelines (Kearns). Careful for characters after <Esc>, that is a security leak.
- Add option setting to ask user if he wants to have the modelines executed or not. Same for .exrc in local dir.

会话:
8 DOS/Windows: ":mksession" generates a "cd" command where "aa\#bb" means directory "#bb" in "aa", but it's used as "aa#bb". (Ronald Hoellwarth)
7 When there is a "help.txt" window in a session file, restoring that session will not get the "LOCAL ADDITIONS" back.
8 With ":mksession" always store the 'sessionoptions' option, even when "options" isn't in it. (St-Amant)
8 When using ":mksession", also store a command to reset all options to their default value, before setting the options that are not at their default value.
7 With ":mksession" also store the tag stack and jump history. (Michal Malecki)
7 Persistent variables: "p:var"; stored in viminfo file and sessions files.

选项:
7 ":with option=value | command": temporarily set an option value and restore it after the command has executed.
8 Make "old" number options that really give a number of effects into string options that are a comma separated list. The old number values should also be supported.
8 Add commands to save and restore an option, which also preserves the flag that marks if the option was set. Useful to keep the effect of setting 'compatible' after ":syntax on" has been used.
7 There is 'titleold', why is there no 'iconold'? (Chazelas)

外部命令:

8 When filtering text, redirect stderr so that it can't mess up the screen and Vim doesn't need to redraw it. Also for ":r !cmd".

4 Set separate shell for ":sh", piping "range!filter", reading text "r !ls" and writing text "w !wc". (Deutsche) Allow arguments for fast start (e.g. -f).

4 Allow direct execution, without using a shell.

4 Run an external command in the background. But how about I/O in the GUI? Careful: don't turn Vim into a shell!

4 Add feature to disable using a shell or external commands.

多重窗口:

7 "vim -oO file ..." use both horizontal and vertical splits.

8 Add CTRL-W T: go to the top window in the column of the current window. And CTRL-W B: go to bottom window.

7 Use CTRL-W <Tab>, like alt-tab, to switch between buffers. Repeat <Tab> to select another buffer (only loaded ones?), <BS> to go back, <Enter> to select buffer, <Esc> to go back to original buffer.

7 Make it possible to edit a new buffer in the preview window. A script can then fill it with something. ":popen"?

7 Add a 'tool' window: behaves like a preview window but there can be several. Don't count it in only_one_window(). (Alexei Alexandrov)

6 Add an option to resize the shell when splitting and/or closing a window. ":vsp" would make the shell wider by as many columns as needed for the new window. Specify a maximum size (or use the screen size). ":close" would shrink the shell by as many columns as come available. (Demirel)

7 When starting Vim several times, instantiate a Vim server, that allows communication between the different Vims. Feels like one Vim running with multiple top-level windows. Esp. useful when Vim is started from an IDE too. Requires some form of inter process communication.

- Support a connection to an external viewer. Could call the viewer automatically after some seconds of non-activity, or with a command. Allow some way of reporting scrolling and cursor positioning in the viewer to Vim, so that the link between the viewed and edited text can be made.

位置标记:

8 Add ten marks for last changed files: ':0, ':1, etc. One mark per file.

8 When cursor is first moved because of scrolling, set a mark at this position. (Rimon Barr) Use '-.

8 Add a command to jump to a mark and make the motion inclusive. g'm and g`m?

8 The '" mark is set to the first line, even when doing ":next" a few times. Only set the '" mark when the cursor was really moved in a file.

8 Make `` and '', which would position the new cursor position in the middle of the window, restore the old topline (or relative position) from when the mark was set.

7 Make a list of file marks in a separate window. For listing all buffers, matching tags, errors, etc. Normal commands to move around. Add commands to jump to the mark (in current window or new window). Start it with ":browse marks"?

6 Add a menu that lists the Marks like ":marks". (Amerige)

7 For ":jumps", ":tags" and ":marks", for not loaded buffers, remember the

text at the mark. Highlight the column with the mark.
7 Highlight each mark in some way (With "Mark" highlight group).
Or display marks in a separate column, like 'number' does.
7 Use d"m to delete rectangular area from cursor to mark m (like Vile's \m command).
7 Try to keep marks in the same position when:
- replacing with a line break, like in ":s/pat/^M/", move marks after the line break column to the next line. (Acevedo)
- inserting/deleting characters in a line.
5 Include marks for start/end of the current word and section. Useful in mappings.
6 Add "unnamed mark" feature: Like marks for the ":g" command, but place and unplace them with commands before doing something with the lines.
Highlight the marked lines somehow.

二合字母:

7 Make "ga" show the keymap for a character, if it exists.
Also show the code of the character after conversion to 'fileencoding'.
- Use digraph table to tell Vim about the collating sequence of special characters?
8 Add command to remove one or more (all) digraphs. (Brown)
7 Support different sets of digraphs (depending on the character set?). At least Latin1/Unicode, Latin-2, MS-DOS (esp. for Win32).

文件写入:

- In vim_rename(), should lock "from" file when deleting "to" file for systems other than Amiga. Avoids problems with unexpected longname to shortname conversion.
8 write mch_isdevice() for Amiga, Mac, VMS, etc.
8 When appending to a file, Vim should also make a backup and a 'patchmode' file.
8 'backupskip' doesn't write a backup file at all, a bit dangerous for some applications. Add 'backupelsewhere' to write a backup file in another directory? Or add a flag to 'backupdir'?
6 Add an option to write a new, numbered, backup file each time. Like 'patchmode', e.g., 'backupmode'.
6 Make it possible to write 'patchmode' files to a different directory. E.g., ":set patchmode=~/backups/*.orig". (Thomas)
6 Add an option to prepend something to the backup file name. E.g., "#". Or maybe allow a function to modify the backup file name?
8 Only make a backup when overwriting a file for the first time. Avoids losing the original when writing twice. (Slootman)
7 On non-Unix machines, also overwrite the original file in some situations (file system full, it's a link on an NFS partition).
7 When editing a file, check that it has been change outside of Vim more often, not only when writing over it. E.g., at the time the swap file is flushed. Or every ten seconds or so (use the time of day, check it before waiting for a character to be typed).
8 When a file was changed since editing started, show this in the status line of the window, like "[time]".
Make it easier to reload all outdated files that don't have changes. Automatic and/or with a command.

替换:

8 Substitute with hex/unicode number "\%xff" and "\%uabcd". Just like "\%uabcd" in search pattern.
8 Make it easier to replace in all files in the argument list. E.g.: ":argsub/oldword/newword/". Works like ":argdo %s/oldword/newword/g|w".
- :s///p prints the line after a substitution.
- With :s///c replace \&, ~, etc. when showing the replacement pattern.
8 With :s///c allow scrolling horizontally when 'nowrap' is effective. Also allow a count before the scrolling keys.
- Add number option to ":s//2": replace second occurrence of string? Or: :s///N substitutes N times.
- Add answers to ":substitute" with 'c' flag, used in a ":global", e.g.: ":g/pat1/s/pat2/pat3/cg": 'A' do all remaining replacements, 'Q' don't do any replacements, 'u' undo last substitution.
7 Substitute in a block of text. Use {line}.{column} notation in an Ex range, e.g.: ":1.3,$.5s" means to substitute from line 1 column 3 to the last line column 5.
5 Add commands to bookmark lines, display bookmarks, remove bookmarks, operate on lines with bookmarks, etc. Like ":global" but with the possibility to keep the bookmarks and use them with several commands. (Stanislav Sitar)

鼠标支持:

8 Add 'o' flag to 'mouse'?
7 Be able to set a 'mouseshape' for the popup menu.
8 Add 'mouse' flag, which sets a behavior like Visual mode, but automatic yanking at the button-up event. Or like Select mode, but typing gets you out of Select mode, instead of replacing the text. (Bhaskar)
- Implement mouse support for the Amiga console.
- Using right mouse button to extend a blockwise selection should attach to the nearest corner of the rectangle (four possible corners).
- Precede mouse click by a number to simulate double clicks?!?
- When mouse click after 'r' command, get character that was pointed to.

参数列表:

6 Add command to put all filenames from the tag files in the argument list. When given an argument, only use the files where that argument matches (like `grep -l ident`) and jump to the first match.
6 Add command to form an args list from all the buffers?

寄存器:

8 Don't display empty registers with ":display". (Etienne)
8 Add put command that overwrites existing text. Should also work for blocks. Useful to move text around in a table. Works like using "R ^R r" for every line.
6 When yanking into the unnamed registers several times, somehow make the previous contents also available (like it's done for deleting). What register names to use? g"1, g"2, etc.?
- When appending to a register, also report the total resulting number of

lines. Or just say "99 more lines yanked", add the "more".
- When inserting a register in Insert mode with CTRL-R, don't insert comment leader when line wraps?
- The ":@r" commands should take a range and execute the register for each line in the range.
- Add "P" command to insert contents of unnamed register, move selected text to position of previous deleted (to swap foo and bar in " + foo")
8 Should be able to yank and delete into the "/ register.
How to take care of the flags (offset, magic)?

调试模式:
8 Add breakpoints for setting an option
8 Add breakpoints for assigning to a variable.
7 Store the history from debug mode in viminfo.
7 Make the debug mode history available with histget() et al.

其他改进:
7 Add plugins for formatting? Should be able to make a choice depending on the language of a file (English/Korean/Japanese/etc.).
Setting the 'langformat' option to "chinese" would load the "format/chinese.vim" plugin.
The plugin would set 'formatexpr' and define the function being called.
Edward L. Fox explains how it should be done for most Asian languages. (2005 Nov 24)
Alternative: patch for utf-8 line breaking. (Yongwei Wu, 2008 Feb 23)
7 [t to move to previous xml/html tag (like "vatov"),]t to move to next ("vatv").
7 [< to move to previous xml/html tag, e.g., previous <li>.]< to move to next <li>,]< to next , [< to previous .
8 Add ":rename" command: rename the file of the current buffer and rename the buffer. Buffer may be modified.
7 Instead of filtering errors with a shell script it should be possible to do this with Vim script. A function that filters the raw text that comes from the 'makeprg'?
- Add %b to 'errorformat': buffer number. (Yegappan Lakshmanan / Suresh Govindachar)
7 Allow a window not to have a statusline. Makes it possible to use a window as a buffer-tab selection.
8 Allow non-active windows to have a different statusline. (Yakov Lerner)
7 Add an invisible buffer which can be edited. For use in scripts that want to manipulate text without changing the window layout.
8 Add a command to revert to the saved version of file; undo or redo until all changes are gone.
6 "vim -q -" should read the list of errors from stdin. (Gautam Mudunuri)
8 Add "--remote-fail": When contacting the server fails, exit Vim.
Add "--remote-self": When contacting the server fails, do it in this Vim.
Overrules the default of "--remote-send" to fail and "--remote" to do it in this Vim.
8 When Vim was started without a server, make it possible to start one, as if the "--servername" argument was given. ":startserver <name>"?
8 No address range can be used before the command modifiers. This makes them difficult to use in a menu for Visual mode. Accept the range and have it apply to the following command.

8 Add the possibility to set 'fileformats' to force a format and strip other CR characters. For example, for "dos" files remove CR characters at the end of the line, so that a file with mixed line endings is cleaned up. To just not display the CR characters: Add a flag to 'display'?

7 Some compilers give error messages in which the file name does not have a path. Be able to specify that 'path' is used for these files.

7 Xterm sends <Esc>O3F for <M-End>. Similarly for other <M-Home>, <M-Left>, etc. Combinations of Alt, Ctrl and Shift are also possible. Recognize these to avoid inserting the raw byte sequence, handle like the key without modifier (unless mapped).

6 Add "gG": like what "gj" is to "j": go to the N'th window line.

8 Add command like ":normal" that accepts <Key> notation like ":map".

9 Support ACLs on more systems.

7 Add ModeMsgVisual, ModeMsgInsert, etc. so that each mode message can be highlighted differently.

7 Add a message area for the user. Set some option to reserve space (above the command line?). Use an ":echouser" command to display the message (truncated to fit in the space).

7 Add %s to 'keywordprg': replace with word under the cursor. (Zellner)

8 Support printing on Unix. Can use "lpansi.c" as an example. (Bookout)

8 Add put command that replaces the text under it. Esp. for blockwise Visual mode.

7 Enhance termresponse stuff: Add t_CV(?): pattern of term response, use regexp: "\e\[[>?][0-9;]*c", but only check just after sending t_RV.

7 Add "g|" command: move to N'th column from the left margin (after wrapping and applying 'leftcol'). Works as "|" like what "g0" is to "0".

7 Support setting 'equalprg' to a user function name.

7 Highlight the characters after the end-of-line differently.

7 When 'whichwrap' contains "l", "$dl" should join lines?

8 Add an argument to configure to use $CFLAGS and not modify it? (Mooney)

8 Enabling features is a mix of configure arguments and defines in feature.h. How to make this consistent? Feature.h is required for non-unix systems. Perhaps let configure define CONF_XXX, and use #ifdef CONF_XXX in feature.h? Then what should min-features and max-features do?

8 Add "g^E" and "g^Y", to scroll a screen-full line up and down.

8 Add ":confirm" handling in open_exfile(), for when file already exists.

8 When quitting with changed files, make the dialog list the changed file and allow "write all", "discard all", "write some". The last one would then ask "write" or "discard" for each changed file. Patch in HierAssist does something like this. (Shah)

7 Use growarray for replace stack.

7 Have a look at viH (Hellenic or Greek version of Vim). But a solution outside of Vim might be satisfactory (Haritsis).

3 Make "2d%" work like "d%d%" instead of "d2%"?

7 "g CTRL-O" jumps back to last used buffer. Skip CTRL-O jumps in the same buffer. Make jumplist remember the last ten accessed buffers?

7 Make it possible to set the size of the jumplist (also to a smaller number than the default). (Nikolai Weibull)

- Add code to disable the CAPS key when going from Insert to Normal mode.

- Set date/protection/etc. of the patchfile the same as the original file.

- Use growarray for termcodes[] in term.c

- Add <window-99>, like <cword> but use filename of 99'th window.

7 Add a way to change an operator to always work characterwise-inclusive (like "v" makes the operator characterwise-exclusive). "x" could be used.

- Make a set of operations on list of names: expand wildcards, replace home dir, append a string, delete a string, etc.
- Remove using mktemp() and use tmpname() only? Ctags does this.
- When replacing environment variables, and there is one that is not set, turn it into an empty string? Only when expanding options? (Hiebert)
- Option to set command to be executed instead of producing a beep (e.g. to call "play newbeep.au").
- Add option to show the current function name in the status line. More or less what you find with "[[k", like how 'cindent' recognizes a function. (Bhatt).
- "[r" and "]r": like "p" and "P", but replace instead of insert (esp. for blockwise registers).
- Add 'timecheck' option, on by default. Makes it possible to switch off the timestamp warning and question. (Dodt).
- Add an option to set the time after which Vim should check the timestamps of the files. Only check when an event occurs (e.g., character typed, mouse moved). Useful for non-GUI versions where keyboard focus isn't noticeable.
- Make 'smartcase' work even though 'ic' isn't set (Webb).

7 When formatting text, allow to break the line at a number of characters. Use an option for this: 'breakchars'? Useful for formatting Fortran code.

- Add flag to 'formatoptions' to be able to format book-style paragraphs (first line of paragraph has larger indent, no empty lines between paragraphs). Complements the '2' flag. Use '>' flag when larger indent starts a new paragraph, use '<' flag when smaller indent starts a new paragraph. Both start a new paragraph on any indent change.

8 The 'a' flag in 'formatoptions' is too dangerous. In some way only do auto-formatting in specific regions, e.g. defined by syntax highlighting.

8 Allow using a trailing space to signal a paragraph that continues on the next line (MIME text/plain; format=flowed, RFC 2646). Can be used for continuous formatting. Could use 'autoformat' option, which specifies a regexp which triggers auto-formatting (for one line).
":set autoformat=\\s$".

- Be able to redefine where a sentence stops. Use a regexp pattern?
- Support multi-byte characters for sentences. Example from Ben Peterson.

7 Add command "g)" to go to the end of a sentence, "g(" to go back to the end of a sentence. (Servatius Brandt)

- Be able to redefine where a paragraph starts. For "[[" where the '{' is not in column 1.

6 Add ":cdprev": go back to the previous directory. Need to remember a stack of previous directories. We also need ":cdnext".

7 Should ":cd" for MS-DOS go to $HOME, when it's defined?

- Make "gq<CR>" work on the last line in the file. Maybe for every operator?
- Add more redirecting of Ex commands:

```
    :redir #>  bufname
    :redir #>> bufname   (append)
```

- Give error message when starting :redir: twice or using END when no redirection was active.
- Setting of options, specifically for a buffer or window, with ":set window.option" or ":set buffer.option=val". Or use ":buffer.set". Also: "buffer.map <F1> quit".

6 Would it be possible to change the color of the cursor in the Win32 console? (Klaus Hast)

- Add :delcr command:

:delcr

:[range]delcr[!] Check [range] lines (default: whole buffer) for lines ending in <CR>. If all lines end in <CR>, or [!] is used, remove the <CR> at the end of lines in [range]. A CTRL-Z at the end of the file is removed. If [range] is omitted, or it is the whole file, and all lines end in <CR> 'textmode' is set.

- Should integrate addstar() and file_pat_to_reg_pat().
- When working over a serial line with 7 bit characters, remove meta characters from 'isprint'.
- Use fchdir() in init_homedir(), like in FullName().
- In win_update(), when the GUI is active, always use the scrolling area. Avoid that the last status line is deleted and needs to be redrawn.
- That "cTx" fails when the cursor is just after 'x' is Vi compatible, but may not be what you expect. Add a flag in 'cpoptions' for this? More general: Add an option to allow "c" to work with a null motion.
- Give better error messages by using errno (strerror()).
- Give "Usage:" error message when command used with wrong arguments (like Nvi).
- Make 'restorescreen' option also work for xterm (and others), replaces the SAVE_XTERM_SCREEN define.

7 Support for ":winpos" In xterm: report the current window position.

- Give warning message when using ":set t_xx=asdf" for a termcap code that Vim doesn't know about. Add flag in 'shortmess'?

6 Add ":che <file>", list all the include paths which lead to this file.

- For a commandline that has several commands (:s, :d, etc.) summarize the changes all together instead of for each command (e.g. for the rot13 macro).
- Add command like "[I" that also shows the tree of included files.
- ":set sm^L" results in ":set s", because short names of options are also expanded. Is there a better way to do this?
- Add ":@!" command, to ":@" like what ":source!" is to ":source".

8 Add ":@:!": repeat last command with forceit set.

- Add 't_normal': Used whenever t_me, t_se, t_ue or t_Zr is empty.
- ":cab map test ^V| je", ":cunab map" doesn't work. This is vi compatible!
- CTRL-W CTRL-E and CTRL-W CTRL-Y should move the current window up or down if it is not the first or last window.
- Include-file-search commands should look in the loaded buffer of a file (if there is one) instead of the file itself.

7 Change 'nrformats' to include the leader for each format. Example:

```
nrformats=hex:$,binary:b,octal:0
```

Add setting of 'nrformats' to syntax files.

- 'path' can become very long, don't use NameBuff for expansion.
- When unhiding a hidden buffer, put the same line at top of the window as the one before hiding it. Or: keep the same relative cursor position (so many percent down the windows).
- Make it possible for the 'showbreak' to be displayed at the end of the line. Use a comma to separate the part at the end and the start of the line? Highlight the linebreak characters, add flag in 'highlight'. Make 'showbreak' local to a window.
- Some string options should be expanded if they have wildcards, e.g. 'dictionary' when it is "*.h".
- Use a specific type for number and boolean options, making it possible to change it for specific machines (e.g. when a long is 64 bit).

- Add option for <Insert> in replace mode going to normal mode. (Nugent)
- Add a next/previous possibility to "[^I" and friends.
- Add possibility to change the HOME directory. Use the directory from the passwd file? (Antwerpen)

8 Add commands to push and pop all or individual options. ":setpush tw", ":setpop tw", ":setpush all". Maybe pushing/popping all options is sufficient. ":setflush" resets the option stack?
How to handle an aborted mapping? Remember position in tag stack when mapping starts, restore it when an error aborts the mapping?

- Change ":fixdel" into option 'fixdel', t_del will be adjusted each time t_bs is set? (Webb)
- "gc": goto character, move absolute character positions forward, also counting newlines. "gC" goes backwards (Weigert).
- When doing CTRL-^, redraw buffer with the same topline. (Demirel) Store cursor row and window height to redraw cursor at same percentage of window (Webb).
- Besides remembering the last used line number of a file, also remember the column. Use it with CTRL-^ et. al.
- Check for non-digits when setting a number option (careful when entering hex codes like 0xff).
- Add option to make "." redo the "@r" command, instead of the last command executed by it. Also to make "." redo the whole mapping. Basically: redo the last TYPED command.
- Support URL links for ^X^F in Insert mode, like for "gf".
- Support %name% expansion for "gf" on Windows.
- Make "gf" work on "file://c:/path/name". "file:/c:/" and "file:///c:/" should also work?
- Add 'urlpath', used like 'path' for when "gf" used on a URL?

8 When using "gf" on an absolute file name, while editing a remote file (starts with scp:// or http://) should prepend the method and machine name.

- When finding a URL or file name, and it doesn't exist, try removing a trailing '.'.
- Add ":path" command modifier. Should work for every command that takes a file name argument, to search for the file name in 'path'. Use find_file_in_path().
- Highlight control characters on the screen: Shows the difference between CTRL-X and "^" followed by "X" (Colon).
- Integrate parsing of cmdline command and parsing for expansion.
- Create a program that can translate a .swp file from any machine into a form usable by Vim on the current machine.
- Add ":noro" command: Reset 'ro' flag for all buffers, except ones that have a readonly file. ":noro!" will reset all 'ro' flags.
- Add a variant of CTRL-V that stops interpretation of more than one character. For entering mappings on the command line where a key contains several special characters, e.g. a trailing newline.
- Make '2' option in 'formatoptions' also work inside comments.
- Add 's' flag to 'formatoptions': Do not break when inside a string. (Dodt)
- When window size changed (with the mouse) and made too small, set it back to the minimal size.
- Add "]>" and "[<", shift comment at end of line (command; /* comment */).
- Should not call cursorcmd() for each vgetc() in getcmdline().
- ":split file1 file2" adds two more windows (Webb).
- Don't give message "Incomplete last line" when editing binary file.

- Add ":a", ":i" for preloading of named buffers.
- When entering text, keep other windows on same buffer updated (when a line entered)?
- Check out how screen does output optimizing. Apparently this is possible as an output filter.
- In dosub() regexec is called twice for the same line. Try to avoid this.
- Window updating from memline.c: insert/delete/replace line.
- Optimize ml_append() for speed, esp. for reading a file.
- V..c should keep indent when 'ai' is set, just like [count]cc.
- Updatescript() can be done faster with a string instead of a char.
- Screen updating is inefficient with CTRL-F and CTRL-B when there are long lines.
- Uppercase characters in Ex commands can be made lowercase?

8 Add option to show characters in text not as "|A" but as decimal ("^129"), hex ("\x81") or octal ("\201") or meta (M-x). Nvi has the 'octal' option to switch from hex to octal. Vile can show unprintable characters in hex or in octal.

7 Tighter integration with xxd to edit binary files. Make it more easy/obvious to use. Command line argument?

- How does vi detect whether a filter has messed up the screen? Check source. After ":w !command" a wait_return?
- Improve screen updating code for doput() (use s_ins()).
- With 'p' command on last line: scroll screen up (also for terminals without insert line command).
- Use insert/delete char when terminal supports it.
- Optimize screen redraw for slow terminals.
- Optimize "dw" for long row of spaces (say, 30000).
- Add "-d null" for editing from a script file without displaying.
- In Insert mode: Remember the characters that were removed with backspace and re-insert them one at a time with <key1>, all together with <key2>.
- Amiga: Add possibility to set a keymap. The code in amiga.c does not work yet.
- Implement 'redraw' option.
- Add special code to 'sections' option to define something else but '{' or '}' as the start of a section (e.g. one shiftwidth to the right).

7 Allow using Vim in a pipe: "ls | vim -u xxx.vim - | yyy". Only needs implementing ":w" to stdout in the buffer that was read from stdin. Perhaps writing to stdout will work, since stderr is used for the terminal I/O.

8 Allow opening an unnamed buffer with ":e !cmd" and ":sp !cmd". Vile can do it.

- Add commands like]] and [[that do not include the line jumped to.
- When :unab without matching "from" part and several matching "to" parts, delete the entry that was used last, instead of the first in the list.
- Add text justification option.
- Set boolean options on/off with ":set paste=off", ":set paste=on".
- After "inv"ing an option show the value: ":set invpaste" gives "paste is off".
- Check handling of CTRL-V and '\' for ":" commands that do not have TRLBAR.
- When a file cannot be opened but does exist, give error message.
- Amiga: When 'r' protection bit is not set, file can still be opened but gives read errors. Check protection before opening.
- When writing check for file exists but no permission, "Permission denied".
- If file does not exist, check if directory exists.

- Settings edit mode: make file with ":set opt=xx", edit it, parse it as ex commands.
- ":set -w all": list one option per line.
- Amiga: test for 'w' flag when reading a file.
- :table command (Webb)
- Add new operator: clear, make area white (replace with spaces): "g ".
- Add command to ":read" a file at a certain column (blockwise read?).
- Add sort of replace mode where case is taken from the old text (Goldfarb).
- Allow multiple arguments for ":read", read all the files.
- Support for tabs in specific columns: ":set tabcol=8,20,34,56" (Demirel).
- Add 'searchdir' option: Directories to search for file name being edited (Demirel).
- Modifier for the put command: Change to linewise, charwise, blockwise, etc.
- Add commands for saving and restoring options ":set save" "set restore", for use in macro's and the like.
- Keep output from listings in a window, so you can have a look at it while working in another window. Put cmdline in a separate window?
- Add possibility to put output of Ex commands in a buffer or file, e.g. for ":set all". ":r :set all"?
- When the 'equalalways' option is set, creating a new window should not result in windows to become bigger. Deleting a window should not result in a window to become smaller (Webb).
- When resizing the whole Vim window, the windows inside should be resized proportionally (Webb).
- Include options directly in option table, no indirect pointers. Use mkopttab to make option table?
- When doing ":w dir", where "dir" is a directory name, write the current file into that directory, with the current file name (without the path)?
- Support for 'dictionary's that are sorted, makes access a lot faster (Haritsis).
- Add "^Vrx" on the command line, replace with contents of register x. Used instead of CTRL-R to make repeating possible. (Marinichev)
- Add "^Vb" on the command line, replace with word before or under the cursor?
- Support mapping for replace mode and "r" command (Vi doesn't do this)?

8 Sorting of filenames for completion is wrong on systems that ignore case of filenames. Add 'ignorefncase' option. When set, case in filenames is ignored for sorting them. Patch by Mike Williams: ~/vim/patches/ignorefncase. Also change what matches? Or use another option name.

8 Should be able to compile Vim in another directory, with $(srcdir) set to where the sources are. Add $(srcdir) in the Makefile in a lot of places. (Netherton)

6 Make it configurable when "J" inserts a space or not. Should not add a space after "(", for example.

5 When inserting spaces after the end-of-line for 'virtualedit', use tabs when the user wants this (e.g., add a "tab" field to 'virtualedit'). (Servatius Brandt)

从 Elvis 那里:

- Use "instman.sh" to install manpages?
- Add ":alias" command.
- Search patterns:

```
      \@        match word under cursor.
    but do:
      \@w       match the word under the cursor?
      \@W       match the WORD under the cursor?
8   ":window" command:
    :win +      next window (up)
    :win ++     idem, wrapping
    :win -      previous window (down)
    :win --     idem, wrapping
    :win nr     to window number "nr"
    :win name   to window editing buffer "name"
7   ":cc" compiles a single file (default: current one).  'ccprg'   option is
    program to use with ":cc".  Use ":compile" instead of ":cc"?
```

从 xvi 那里:

```
-   CTRL-_ : swap 8th bit of character.
-   Add egrep-like regex type, like xvi (Ned Konz) or Perl (Emmanuel Mogenet)
```

从 vile 那里:

```
-   When horizontal scrolling, use '>' for lines continuing right of a window.
-   Support putting .swp files in /tmp: Command in rc.local to move .swp files
    from /tmp to some directory before deleting files.
```

遥远的将来和 "重大" 的扩展:

```
-   Instead of using a Makefile and autoconf, use a simple shell script to
    find the C compiler and do everything with C code.  Translate something
    like an Aap recipe and configure.ac to C.  Avoids depending on Python,
    thus will work everywhere.  With batch file to find the C compiler it
    would also work on MS-Windows.
-   Make it easy to setup Vim for groups of users: novice vi users, novice
    Vim users, C programmers, xterm users, GUI users,...
-   Change layout of blocks in swap file: Text at the start, with '\n' in
    between lines (just load the file without changes, except for Mac).
    Indexes for lines are from the end of the block backwards.  It's the
    current layout mirrored.
-   Make it possible to edit a register, in a window, like a buffer.
-   Add stuff to syntax highlighting to change the text (upper-case keywords,
    set indent, define other highlighting, etc.).
-   Mode to keep C-code formatted all the time (sort of on-line indent).
-   Several top-level windows in one Vim session.  Be able to use a different
    font in each top-level window.
-   Allow editing above start and below end of buffer (flag in 'virtualedit').
-   Smart cut/paste: recognize words and adjust spaces before/after them.
-   Add open mode, use it when terminal has no cursor positioning.
-   Special "drawing mode": a line is drawn where the cursor is moved to.
    Backspace deletes along the line (from jvim).
-   Support for underlining (underscore-BS-char), bold (char-BS-char) and other
    standout modes switched on/off with , 'overstrike' option (Reiter).
-   Add vertical mode (Paul Jury, Demirel): "5vdw" deletes a word in five
    lines, "3vitextESC" will insert "text" in three lines, etc..
4   Recognize l, #, p as 'flags' to EX commands:
```

:g/RE/#l shall print lines with line numbers and in list format.
:g/RE/dp shall print lines that are deleted.
POSIX: Commands where flags shall apply to all lines written: list,
number, open, print, substitute, visual, &, z. For other commands, flags
shall apply to the current line after the command completes. Examples:
:7,10j #l Join the lines 7-10 and print the result in list
- Allow two or more users to edit the same file at the same time. Changes
are reflected in each Vim immediately. Could work with local files but
also over the internet. See http://www.codingmonkeys.de/subethaedit/.

vim:tw=78:sw=4:sts=4:ts=8:noet:ft=help:norl:
vim: set fo+=n :

develop.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：jwdu，iCrazy，tocer (http://vimcdoc.sf.net)

Vim 的开发 development

这一节的内容对那些想进一步开发 Vim 的人来说是很重要的。

1. 设计目标 design-goals
2. 编程风格 coding-style
3. 设计决定 design-decisions
4. 假定 design-assumptions

请先看一下 "src" 目录下的 README.txt 以便对源代码有一个总体的了解。

Vim 是开源软件，鼓励所有人参与并改进 Vim。如果你想递交 patch，请使用 "diff -u" 来产生合并 (unified) 格式的 diff 文件。
另见 http://vim.wikia.com/wiki/How_to_make_and_submit_a_patch。

---

1. 设计目标 design-goals

首先说最重要的内容 (概略的说一下)。

注意 许多项目是互相冲突的，但这是故意的。对于它们，你必须自己平衡一下。

VIM 是 ... 和 VI 兼 容 的 design-compatible

首先，必须使 Vim 能够代替 Vi。如果用户愿意，他可以在兼容模式下使用 Vim 而且很难发现它和原先的 Vi 有什么不同。

例外：
- 在 Vim 中不再有明显的 Vi 错误。
- 有很多不同的 Vi 版本。我用 3.7 (6/7/85) 版作为参考。但是如果可能，也会支持其它的版本。POSIX 中的 Vi 部分不是最主要的来源。
- Vim 中有一些新的指令，你不要因为某个命令在 Vi 中不存在，就指望它会失败。
- Vim 有许多特性是 Vi 所没有的。从 Vim 回到 Vi ，将会出现一些不可避免的问题。
- 有一些指令可能是很少用到 (open 模式、在崩溃时发送一封邮件等等)。只有你能说明为什么应当加入这些功能，并且不会增加多少代码量时，它们才会被加入。
- 对一些命令是否仍需兼容 Vi 是一个有争议的问题。对这些，会使用一个选项标志位。

VIM 是 ... 改 进 过 的 design-improved

Vim 的改良使它成为一个更好的 Vi，而没有变成一个完全不同的编译器。对它的扩展是本着 "Vi 精神" 完成的。
- 尽可能多的使用键盘，鼠标需要第三只手来操作，而我们没有第三只手。而且，许多终端没有鼠标。
- 当使用鼠标操作时，尽量少回到键盘操作，避免鼠标和键盘的混合操作。
- 用一致的方式增加命令和选项。否则，人们将会很难发现并记住它们。请记住：以后会加入更多的命令和选项。

- 一个人们不知道的特性是一个无用的特性。不要添加不明显的特性，或者至少在文档中说明它们的存在。
- 尽量少用 CTRL 和其它修饰符，因为它们很难输入。
- 因为对许多人来说，Vim 是他们刚接触或不熟悉的，所以请尽量使 Vim 容易上手。
- 特性是可以无限增加的，对它们的选择基于以下方面：(1) 使用者需要什么 (2) 实现它需要多少工作量 (3) 的确有人会去实现它。

VIM 是 ... 多 平 台 的 design-multi-platform

Vim 设法支持尽可能多的平台，从而让尽可能多的人能够使用它。
- 支持多种终端。最小的需求是定位鼠标和清屏。指令最好用多数键盘都具有的那些键来执行。在映射键盘时，支持键盘上所有的键。
- 支持多种平台。必要条件是有人想在此平台上开发 Vim，而且不要把代码搞乱。
- 支持多种编译器和库。并不是每个人都能够安装另一个编译器或图形库。
- 人们经常从一个平台转移到另一个平台，从图形界面转移到终端模式。特性应在每个版本中都加入，至少应该在尽可能多的版本中出现。尽量避免使用者为了有效的完成他们的工作而需要在不同平台间转移。
- 也许有些特性在某些平台上没有，或者只在某个平台上有，但这不意味着它不应当被实现。[这个和上一个原则明显的冲突了，但我们必须在它们之间做个平衡。]

VIM 是 ... 具 有 详 尽 文 档 的 design-documented

- 一个没有文档的特性是一个无用的特性。针对新特性的补丁必须包含它的文档。
- 文档必须全面且容易理解。最好举例说明问题。
- 请使文件尽可能的短，短文档能使得找到它容易一些。

VIM 是 ... 速 度 快 、 体 积 小 的 design-speed-size

Vim 不能大量消耗系统资源，必须使它体积小且速度快。
- 计算机每年都在变得更快、更大。Vim 也可以同样的变化，但是速度不能超过计算机的变化速度。要使 Vim 在老的计算机上也可以使用。
- 许多使用者经常从外壳启动 Vim。启用时间必须尽量短。
- 指令必须有效的工作，它们消耗的时间要尽可能的短。有用的指令可以多消耗一点时间。
- 注意到许多人在慢速线路上使用 Vim，因此，要使通信的开销尽可能的少。
- 如果一个显著增加 Vim 体积的功能不被大多数人使用，应该可以关闭该功能。
- Vim 只是其它许多组件中的一个，不要把它弄成一个笨重的应用程序，倒要使它能够和其它程序一起很好的工作。

VIM 是 ... 可 维 护 的 design-maintain

- 源代码不应该变成一团糟。它应该是可靠的代码。
- 所有的文件应使用相同的风格，使得它们易读性强 coding-style 。
- 根据需要有效的使用注释！ 不要 引用函数和参数名。要解释它们的意义。
- 移植到另一个平台应该很容易，而不必改变太多独立于平台的代码。
- 使用面向对象的思想：把数据和代码放在一起，波及到代码其它部分的知识要尽可能的少。

VIM 是 ... 灵 活 的 design-flexible

Vim 应该让使用它的人很容易在他们喜欢的风格下工作，而不强迫他们使用一个特定的方式工作。这个对于那些有巨大影响的选项（如 'compatible' 选项）和其他细节都是一样。缺省值是经过认真挑选的，以便大多数使用者都乐意在默认设置下使用 Vim。用户可以用命令和选项来调整 Vim 来满足他们和环境的需要。

VIM 不是 ... design-not

- Vim 不是一个外壳或操作系统。它确实提供自己的终端窗口，可在其中运行外壳或调试，以便于在 ssh 会话中应用。但如果不需要文本编辑，这些功能就超出了它的应用范围（请改用 screen 或 tmux 这样的工具）。
  这里有一个讽刺的说法："Vim 不像 Emacs 那样会包含除厨房中的水槽以外的每件事物，但有人说如果你需要的话，却可以用它来洗盘子。;-)"
  要在 Vim 中使用 gdb，参见：http://www.agide.org 和 http://clewn.sf.net。
- Vim 不是一个为了看起来漂亮而牺牲多平台一致性的华而不实的 GUI 编辑器。但我们依然欢迎有效的 GUI 特性。

---

2. 编程风格 coding-style

这些是改变 Vim 源代码的时候必须遵循的准则。为了保持源代码的可读性和可维护性，请坚持使用这些原则。

这个列表是不完全的，请查看源代码以获得更多的例子。

修改代码 style-changes

修改代码的基本步骤：
1. 从 github 获取代码。方便你的版本随时和主代码库保持同步（你的修改可能需要一段时间才会被接受）。需要花点时间学习 git，它的用户界面不是最直观。
2. 首先调整文档，这样做将使你对你的更改如何影响使用者有一个印象。
3. 改变源代码。
4. 检查 ../doc/todo.txt，看看所作的修改是否影响其中的项目。
5. 用 "git diff" 做一个补丁。也可以在 github 上创建拉取请求，但主要是 diff 的部分。
6. 写一个修改了哪些地方的记录，最好提到相关问题和解决方案。给 vim-dev 邮件组发信包含这个解释和 diff 本身。也可以在 github 上创建拉取请求。

C COMPILER style-compiler ANSI-C C89 C99

支持的 C 编译器最低版本是 C89，也即 ANSI C。之后的标准，如 C99，还没有得到广泛的支持，或至少没有得到 100% 的支持。因此我们只使用部分 C99 特性而禁止其它的一些特性（至少目前如此）。

请不要在任何地方修改代码来应用 C99 特性，这会给现有补丁的合并带来困难。C99 特性仅限于新增或需要改动的代码。

注释

Vim 传统上使用 /* 注释 */。我们打算继续在文件和函数头部，还有大块代码处使用这种风格，如：

```
        /*
```

```
         * The "foo" argument does something useful.
         * Return OK or FAIL.
         */
```

新代码或需要改动的代码上建议使用 // 注释，特别是在代码之后的注释：

```
        int some_var;  // 这里可用单行注释
```

枚举

枚举的最后一项可有拖尾的逗号。C89 过去不允许。

类型

允许使用 "long long"，并应假定为 64 位。printf 格式中对应可用 %lld。另外，%llu 可用于 "long long unsigned"。

不使用

不使用以下 C99 特性，因为支持的编译器尚不广泛：
- 语句之后的声明 (MSVC 2012 不支持)。所有的声明必须在块的开始处。
- 可变长度数组 (即便在 C11 中也是一个可选的特性)。
- _Bool 和 _Complex 类型。
- "inline" (也几乎用不到，让优化器自己去做吧)
- 灵活数组成员 (Flexible Array Member): HP-UX C 编译器不支持 (John Marriott)

普 通 函 数 的 使 用 style-functions

一些普遍使用的函数都有一个特殊的 Vim 版本。它们的引入是有原因的，因此，一般应考虑使用这些 Vim 版本。

通用名	VIM 函数名	VIM 版本的不同
free()	vim_free()	检查释放 NULL 的情况
malloc()	alloc()	检查内存不足的情况
malloc()	lalloc()	类似于 alloc()，但是支持长参数
strcpy()	STRCPY()	对于 char_u* 类型的参数，会自动将其转换为 (char*) 类型
strchr()	vim_strchr()	接受特殊字符
strrchr()	vim_strrchr()	接受特殊字符
isspace()	vim_isspace()	可以处理 ASCII 码 > 128 的字符
iswhite()	vim_iswhite()	仅对 Tab 和空格返回真值
memcpy()	mch_memmove()	能处理范围重叠的复制
bcopy()	mch_memmove()	处理重叠的复制
memset()	vim_memset()	对所有系统通用

名 称 style-names

函数名不能超过 31 个字符的长度 (因为 VMS 的缘故)。

不要使用 "delete" 或 "this" 作为变量名称，C++ 不喜欢这样。

因为 Vim 需要在尽可能多的平台上运行，所以我们需要避免那些已经被系统定义过的名称。这里是一个已知会造成麻烦的名称列表 (使用正规表达式模式给出)。

is.*()	POSIX，ctype.h

to.*()	POSIX，ctype.h
d_.*	POSIX，dirent.h
l_.*	POSIX，fcntl.h
gr_.*	POSIX，grp.h
pw_.*	POSIX，pwd.h
sa_.*	POSIX，signal.h
mem.*	POSIX，string.h
str.*	POSIX，string.h
wcs.*	POSIX，string.h
st_.*	POSIX，stat.h
tms_.*	POSIX，times.h
tm_.*	POSIX，time.h
c_.*	POSIX，termios.h
MAX.*	POSIX，limits.h
__.*	POSIX，system
_[A-Z].*	POSIX，system
E[A-Z0-9]*	POSIX，errno.h
.*_t	POSIX，用于 typedefs。请用 .*_T 代替。
wait	不要使用它作为函数的参数，和 types.h 冲突
index	屏蔽全局声明
time	屏蔽全局声明
new	C++ 保留关键字
clear	Mac curses.h
echo	Mac curses.h
instr	Mac curses.h
meta	Mac curses.h
newwin	Mac curses.h
nl	Mac curses.h
overwrite	Mac curses.h
refresh	Mac curses.h
scroll	Mac curses.h
typeahead	Mac curses.h
basename()	GNU 串函数
dirname()	GNU 串函数
get_env_value()	Linux 系统函数

杂 项 style-various

类型定义 (typedef) 的名称应该以 "_T" 结尾:

```
typedef int some_T;
```

宏 (define) 的名称应该全部是大写:

```
#define SOME_THING
```

特性 (feature) 应该总是以 "FEAT_" 开头:

```
#define FEAT_FOO
```

不要使用 '\"'，一些编译器不能处理它。用 '"' 比较合适。

不要使用:

```
    #if HAVE_SOME
```
一些编译器不能处理它并报告说 "HAVE_SOME" 没有被定义。
请使用
```
    #ifdef HAVE_SOME
```
或
```
    #if defined(HAVE_SOME)
```

风 格 style-examples

一般原则：每行一条语句。

```
错误:       if (cond) a = 1;

正确:       if (cond)
                a = 1;

错误:       while (cond);

正确:       while (cond)
                ;

错误:       do a = 1; while (cond);

正确:       do
                a = 1;
            while (cond);

错误:       if (cond) {
               cmd;
               cmd;
            } else {
               cmd;
               cmd;
            }

正确:       if (cond)
            {
               cmd;
               cmd;
            }
            else
            {
               cmd;
               cmd;
            }
```

单行的块可以省略花括号。if/else 一个单行的块有括号时，另一个块最好也有括号：

```
正确:       if (cond)
               cmd;
            else
               cmd;

正确:       if (cond)
```

```
        {
            cmd;
        }
        else
        {
            cmd;
            cmd;
        }
```

使用 ANSI (新风格) 函数声明，返回类型在单独缩进的一行上。

```
错误:   int function_name(int arg1, int arg2)

正确:   /*
         * 解释函数的功能。
         *
         * 解释返回值。
         */
            int
        function_name(
            int         arg1,           // arg1 的简短注释
            int         arg2)           // arg2 的简短注释
        {
            int         local;          // local 的注释

            local = arg1 * arg2;
```

空 格 和 标 点 style-spaces

不要在函数名和括号间留空格:

```
错误:   func (arg);
正确:   func(arg);
```

但请在 if, while, switch 等之后留一个空格:

```
错误:   if(arg)         for(;;)
正确:   if (arg)        for (;;)
```

在逗号和分号后留一个空格:

```
错误:   func(arg1,arg2);        for (i = 0;i < 2;++i)
正确:   func(arg1, arg2);       for (i = 0; i < 2; ++i)
```

在 '=', '+', '/' 等的前后各留一个空格:

```
错误:   var=a*5;
正确:   var = a * 5;
```

一般地：使用空行来把代码分组。在每组代码的上面写上注释，这样就可以快速的知道这些代码是用来干什么的。

正确：
```
/* Prepare for building the table. */
get_first_item();
table_idx = 0;

/* Build the table */
while (has_item())
    table[table_idx++] = next_item();

/* Finish up. */
cleanup_items();
generate_hash(table);
```

---

3. 设计决定 design-decisions

折叠

对同一个缓冲区可以有多种折叠形式。例如，一个窗口显示函数体折叠后的文本，另一个窗口显示函数体。

折叠是一种显示文本的方式。它不应该改变文本自身。所以折叠是被当作缓冲区文本（缓冲区行）和窗口内显示的文本（逻辑行）之间的过滤器来实现的。

为窗口命名

我们一般用单词 "窗口" 来称呼很多东西：屏幕上的一个窗口、xterm 窗口、Vim 中一个用来查看缓冲区内容的窗口。
为避免混淆，我们给其他有时被称作窗口的项目起了其他的名称。这些是相关项目的概述：

screen	整个屏幕。对于 GUI 来说，就是类似于 1024x768 像素之类的东西。Vim 外壳可以使用整个屏幕（例如，在控制台上运行时）或它的一部分（xterm 或 GUI）。
shell	Vim 这个应用程序。它可以覆盖整个屏幕（例如，在控制台上运行时）或它的一部分（xterm 或 GUI）。
window	用来查看缓冲区内容的地方。Vim 中可以包括很多窗口，还有命令行、菜单栏、工具栏等。它们被整合到外壳中。

拼写检查 develop-spell

当要把拼写检查加进 Vim 时，考察了一些可用的拼写检查库和程序。不幸的是，没有一个能提供足够的能力使其能够作为 Vim 的拼写检查引擎，原因很多：

- 缺乏对多字节编码的支持。至少要支持 UTF-8，以便同一文件能使用多种语言。实时转换未必总是可行的（这需要 iconv 支持）。
- 对于那些程序和库：如果不作任何改动，需要和 Vim 分开单独安装才能使用。一般来说，这不是不可以，但总是个问题。
- 性能：测试表明这些程序和库可以像语法高亮那样支持（重画时）实时的拼写检查，但那些代码所用的机制太慢了。例如使用哈希表的 Myspell。大部分的拼写检查程序使用字缀压缩（affix compression）算法也会使其速度减慢不少。
- 使用像 aspell 这样的外部程序需要设置通信机制。这会使可移植性变得复杂（只考虑 Unix 系统相对简单点，但还远远不够）。而且性能也是个问题（大量的进程切换）。

- 缺乏对包含非单词字符的单词的支持，如 "Etten-Leur" 和 "et al."。需要把词中每个部分都标识为好词，但这样就降低了检测的可靠性。
- 缺乏对区域或方言的支持。例如，接受所有英语单词并且单独高亮出非加拿大的单词是很困难的。
- 缺乏对生僻词的支持。有很多词拼写正确但几乎很少使用，却有可能是拼写错误的常用词。
- 对于拼写建议，速度不太重要，要安装其他的程序或者库也是可以接受的。但是这些程序或库提供的单词列表也许与拼写检查程序使用的不一致，给出的建议可能是拼写检查程序检测出的拼写错误的单词。

拼写建议 develop-spell-suggestions

对于实现拼写建议，有两种基本的方法：

1. 尝试轻微改变错误的单词，然后检查能否和正确的单词匹配。或者浏览正确单词列表，轻微改变它们，看能否和错误的单词匹配。这里所指的改动包括：删除一个字符，插入一个字符，交换两个字符等。
2. 比较坏词和好词列表的发音来寻找匹配。可能也需要像第一种方法那样对单词做少许改变。

第一种方法的优势在于查找录入错误。在使用哈希表试验和参考了其他拼写检查程序的方案后，得出结论：trie（一种树结构）是理想方案。既可以减少内存占用，又可以尝试合理的改变。例如，只有在能够拼成正确单词的情况下才插入字符。其他方式（使用哈希表）需要在单词的所有位置尝试所有可能的字符。而且，哈希表需要单独标识单词的边界。而 trie 就不需要这样做。这样会简化很多。

当我们知道单词如何发音却不知道如何拼写时，按发音折叠匹配就很有用了。例如，单词 "dictionary" 可能写成 "daktonerie"。第一种方法需要尝试的次数非常多，很难找到正确的单词。按发音匹配后，这两个单词变成了 "tkxnry" 和 "tktnr"，两者仅有两个字符的差异。

为了通过相似的发音来查找单词（寻找按发音折叠等价的单词），我们需要所有发音相似的单词列表。为此，我们作了一些试验来寻找最好的方法。可供选择的方案是：

1. 在寻找拼写建议时，实时进行发音匹配。这意味着浏览 trie 树以寻找正确单词，匹配每个单词发音，并检查它与错误单词的不同。这种方式内存占用少，但花费时间多。在一台比较快的电脑上，对于英文需要几秒，这对于交互式使用还可以接受。但是对于某些语言，则超过 10 秒（如：德语、加泰罗尼亚语），这就慢得无法忍受了。对于成批处理（自动更正），所有的语言处理起来都很慢。
2. 把 trie 用于发音匹配的单词，这样查找时就可以和没有用发音折叠的方法一模一样。但这需要对每个相近的发音记住所有好词的列表。这种方式搜索匹配很快，但是要占用大量内存，其级别大约在 1M 到 10M 之间。对于某些语言，甚至会大于原单词列表。
3. 类似于第二种方案，但用字缀压缩（affix compression）并且只保存发音相近的单词基本部分（字根）来减少内存开销。aspell 就是这么做的。缺点是，在按发音进行匹配前，需要从错误的单词上分开字缀，这意味着在单词头和/或尾出现的错误会导致本方案失败。而且当错误的单词和正确的单词间有很大不同时，本方案会变得很慢。

我们作出的选择是使用第二种方案，并使用单独的文件。这样，有足够内存的用户可以得到很好的拼写建议，而内存不足或者只想要拼写检查而不需要建议的用户就可以不用那么多内存。

单词频率

对建议进行排序时，知道单词常见与否很有帮助。理论上，我们可以在字典中对应单词保存一个单词频率。但是每个单词都需要计数。这会使单词树的压缩效率大为降低。而且对所有的语言都要维护单词频率，将是个繁重的任务。另外，最好优先考虑文本里已经出现的单词。这样在特定文本里出现的单词可以被建议列表优先列出。

现已实现的是统计已经显示的单词。哈希表用于快速查找单词的计数。字缀文件的 COMMON 项目中列出的单词会先有初始计数，这样，即使新编辑文件时也能马上用此功能。

这还不够理想，因为 Vim 运行的时间越长，单词计数就会越大。但实际应用中，这仍然比不使用单词计数的情形要改进不少。

---

4. 假定 design-assumptions

变量的大小：

char	8 位有符号数
char_u	8 位无符号数
int	32 或 64 位有符号数 (16 位是可能的，但功能会有限制)
unsigned	32 或 64 位无符号数 (16 位情况同 int)
long	32 或 64 位有符号数，可具有指针

注意：某些编译器不能处理过长的源码行或者字符串。C89 标准规定限制在 509 个字符以内。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

调试 Vim debug-vim

本文档是为了调试 Vim 自身而准备的，如果 Vim 不能正确工作，请看这里。
如果要调试 Vim 脚本和函数等，见 debug-scripts 。

1. 用 gcc 和 gdb 定位崩溃 debug-gcc
2. 定位内存泄漏 debug-leaks
3. Windows 漏洞报告 debug-win32

---

1. 用 gcc 和 gdb 定位崩溃 debug-gcc gdb

如果 Vim 崩溃于某测试文件而使用的编译器是 gcc 的话，下面给出具体步骤，你可以用它来发现 Vim 崩溃的准确位置。MingW 工具同样适用。

1. 用 "-g" 选项编译 Vim (src/Makefile 有一行用于此目的，只要去掉注释即可)。同时保证不使用 "strip" (不要安装，或者用 "STRIP = /bin/true")。

2. 执行命令 (把 "11" 换成实际失败的那个测试):
```
cd testdir
gdb ../vim
run -u unix.vim -U NONE -s dotest.in test11.in
```

3. 检查 Vim 在哪里崩溃，gdb 会给出这方面的消息。

4. 此 gdb 命令可以得到堆栈记录:
```
where
```
此命令可以查看不同的堆栈记录位置:
```
frame 3
```
把 "3" 换成堆栈记录其中一个位置。

---

2. 定位内存泄漏 debug-leaks valgrind

如果你怀疑 Vim 泄漏内存并且使用的是 Linux，valgrind 是定位内存泄漏很有用的工具。

首先，编译 Vim 时定义 EXITFREE。在 MAKEFILE 里查找之并去掉注释。

用以下的命令启动 Vim:

```
valgrind --log-file=valgrind.log --leak-check=full ./vim
```

备注: Vim 的运行会变慢许多。如果 .vimrc 很大或者有多个插件，启动时请耐心，也可使用 "--clean" 参数运行来跳过这些。

常有一些库函数发生泄漏，例如 getpwuid() 和 XtVaAppCreateShell()。没法避免这种情况，但泄漏的字节数应该很少，不超过一千字节。

---

3. Windows 漏洞报告 debug-win32

如果 Vim 的 Windows 版本崩溃且崩溃可以重现，通过采取若干步骤，你可以提供有用的漏洞报告。

3.1 通用

首先，你需要可执行文件的调试器符号文件 (PDB): gvim.pdb 对应 gvim.exe，vim.pdb 对应 vim.exe。在获得可执行文件的相同位置应该可以下载 PDB。确信使用和 EXE 匹配的 PDB 文件 (日期相同)。

如果你自己用 Microsoft Visual C++ 编译器编译可执行文件，PDB 应该和 EXE 在同一个生成目录。

如果你有 Visual Studio，用它来代替 VC Toolkit 和 WinDbg。

如果是使用其它编译器，你应该用相应的调试器: Cygwin 和 MinGW 编译器使用 gdb (见上 debug-gcc)。

3.2 用 Visual Studio 2005/Visual C++ 2005 Express 调试 Vim 的崩溃

debug-vs2005

首先启动 vim.exe 或 gvim.exe，然后启动 Visual Studio。(如果你没有 Visual Studio，按照 get-ms-debuggers 的指示获取免费版本 Visual C++ 2005 Express Edition。)

在 Tools 菜单上点击 Attach to Process (连接到进程)。选择 Vim 的进程。

在 Vim 里，重现崩溃。Visual Studio 这时会给出对话框，告诉你 Vim 进程出现了未处理的例外。点击 Break，中断该进程并进入调试。

Visual Studio 这时弹出另一个对话框，告诉你无法载入符号文件所以不能显示源代码。点击 OK。

这时会打开若干窗口。在 Call Stack (调用栈) 窗口按鼠标右键。然后选择 Load Symbols (载入符号)。Find Symbols (寻找符号) 对话框会打开以寻找 (g)vim.pdb。导航到你准备好的包含 PDB 文件的目录，然后选择 Open。

此时，你应该有了完整的调用栈，其中包含了 vim 的函数名和行号。双击任何一行会给出 Find Source (寻找源代码) 对话框。导航到包含 Vim 源代码的目录 (如果有的话)。

如果你不知道如何继续调试，按照 ":help bug-reports" 的步骤进行。把调用栈粘贴到漏洞报告里。

如果有 Visual Studio 的付费版本，你可以用 Debug 菜单保存一个 minidump 并在漏洞报告里一并发送。minidump 是一个小文件 (<100KB)，里面包含进程的状态信息。

Visual C++ 2005 Express Edition 不能保存 minidump 也不能安装成为 just-in-time (即时) 调试器。如果你需要保存 minidump 或者需要即时 (或 postmortem，事后) 调试器的话，使用 WinDbg， debug-windbg 。

debug-windbg

3.3 用 WinDbg 调试 Vim 的崩溃

get-ms-debuggers 说明如何得到 WinDbg。

和 Visual Studio IDE 一样，你可以连接 WinDbg 到运行中的 Vim 进程。你也可以让系统自动激活 WinDbg，从而使之成为事后调试器。要设置 WinDbg 成为一个事后调试器，运行 "windbg -I"。

要连接 WinDbg 到运行中的 Vim 进程，启动 WinDbg，在 File 菜单上点击 Attach to Process (连接到进程)。选择 Vim 进程然后点击 OK。

这个时候，选择 File 菜单的 Symbol File Path (符号文件路径)，然后把包含 Vim PDB 文件的目录加入 sympath。如果你有 Vim 源代码，使用 File 菜单的 Source File Path (源文件路径)。WinDbg 里可以打开源文件，如果你愿意，也可以设置断点。现在重现崩溃。WinDbg 应该会打开源文件并定位于崩溃所在的位置。使用 View 菜单，你可以检查调用栈、局部变量、观察窗口 (watch window) 和其它。

如果把 WinDbg 用作事后调试器，你不需要把 WinDbg 连接到 Vim 进程上。只要简单地重现崩溃，WinDbg 就会自动启动。用上面相同的方法设置符号文件路径和源文件路径。

要保存 minidump，在 WinDbg 命令行上输入:

```
.dump vim.dmp
```

debug-minidump

3.4 打开 Minidump

如果你有 minidump 文件，你可以用 Visual Studio 或 WinDbg 打开它。

Visual Studio 2005 里: File 菜单上选择 Open，然后 Project/Solution。导航到该 .dmp 文件然后将它打开。现在按 F5 打开调试器。按照 debug-vs2005 的指示来设置符号文件路径。

WinDbg 里: 选择 File 菜单的 Open Crash Dump，按照 debug-windbg 的指示来设置符号文件路径。

get-ms-debuggers

3.5 获得 Microsoft 调试工具

Debugging Tools for Windows (Windows 调试工具) 可以在这里下载

```
http://www.microsoft.com/whdc/devtools/debugging/default.mspx
```

它包括 WinDbg 调试器。

Visual C++ 2005 Express Edition 可以在这里免费下载:

```
http://msdn.microsoft.com/vstudio/express/visualC/default.aspx
```

---

vim:tw=78:ts=8:noet:ft=help:norl:

uganda.txt For Vim version 8.2. 最近更新：2013年8月

VIM 参考手册 by Bram Moolenaar
译者：Dasn
http://vimcdoc.sf.net

uganda Uganda copying copyright license

简介：

iccf ICCF

Vim 是慈善软件，您可以任意地使用和复制，但还是希望您能够捐助那些需要帮助的乌干达儿童。请看下方的 kcc ，或者通过下面几个镜像地址访问 ICCF 的主页以获取更多地信息：

```
http://iccf-holland.org/
http://www.vim.org/iccf/
http://www.iccf.nl/
```

您也可以资助 Vim 的开发。Vim 的资助者可以对功能进行投票选举，见 sponsor 。无论哪种方式，钱都会送往乌干达的。

Vim 的文档是以开放式发行许可证 (Open Publication License) 发布的，见 manual-copyright 。

=== 许可证开始 ===
(译者注：本翻译只供参考。我们不承担任何错漏之处引起的法律责任。VIM 许可证必须以英文形式完整出现。如有疑义，一律以英文原文为准。)

VIM 许可证

I) 可以任意发布没有修改的 Vim 的拷贝，但是必须保证包含本许可证。您也可以发布未经修改的部分 Vim，同样也必须包含这份许可证。发布由未经修改的 Vim 源代码所编译出的 Vim 可执行文件，外加您自己的应用实例和 Vim 脚本也是允许的。

II) 在满足以下全部四个条件的前提下，您可以发布经过修改 (或扩充) 的 Vim 版本，包括可执行文件 和/或 源代码：
    1) 本许可证必须包含在内，并且不能被修改。
    2) 经过修改的 Vim 必须以下述五种方式之一发布：
      a) 如果您本人对 Vim 做了改动，您必须在发布版本里清楚地说明如何与您联系。当 Vim 的维护者 (以任何方式) 向您索取您所发布的 Vim 时，您必须把所做的改动包括源代码无偿地提供出来。维护者保留把这些改动加入 Vim 正式版本的权利。至于维护者怎样处理这些改动，以及用什么许可证发布，可以协商。如果没有协商，那么，本许可证，或者它更新的版本，同样适用于您做出的改动。Vim 现在的维护者是 Bram Moolenaar <Bram@vim.org>。如果维护者发生变动，会在合适的地方 (很可能是 vim.sf.net、www.vim.org 和/或 comp.editors) 公布，当完全不能与维护者联系时，发送变更的约定自动终止。一旦维护者确认收到了您所做的修改，您就不必再次发送了。

      b) 如果您得到的是一个修改过的 Vim，并且它是在条件 a) 下发布的，那么您可以不加改动地在条件 I) 下发布它；如果您又做了额外的改动，则这些改动受到 a) 款条文的约束。

      c) 在您发布的经过修改的 Vim 的每一份拷贝里，提供所有的变更部分，包括源代码。提供的形式可以采用上下文风格的差异比较记录 (context diff)。您可以

为添加的新代码选择许可证，但是这些更改和为其选择的许可证不能限制他人对 Vim 正式版本作出自己的改动。

d) 在满足以下全部三个条件的前提下，您可以继续发布带有条件 c) 所提及之变更的经过修改的 Vim，而不必在发布时提供更改部分的源代码：
- 这些变更所附带的许可证允许您把这些变更无偿地并且没有任何限制地提供给 Vim 的维护者，而且允许 Vim 的维护者无偿地并且没有任何限制地把这些更改加入到 Vim 的正式版本中。
- 从您最后一次发布更改的 Vim 之日起，您要保存这些改动至少三年时间。在这期间，维护者或别人 (以任何方式) 向您要求提供这些变更时，您必须提供给他。
- 您要在发布版本中清楚地说明如何与您联系，这个联系方式必须保证自最后一次发布相应的经过修改的 Vim 之日起至少三年有效，或尽可能长。

e) 当这些变更以 GPL (GNU General Public License，GNU 通用公共许可证) 发布时，您可以在 GPL 版本 2，或更高版本的 GPL 下发布修改过的 Vim。

3) 必须添加一条改动的信息。至少要放在 "version" 命令的输出和启动画面里，好让用户知道自己用的是一个修改过的 Vim。当以 2)e) 条件发布时，只有不与变更适用的许可证冲突，这个信息的添加才是必要的。

4) 在 2)a) 和 2)d) 条件里要求的联系方式不能随便更改或删除，除非是作者自己作出的更正。

III) 如果您发布一个更改过的 Vim，强烈建议您对变更部分使用 Vim 的许可证，并且对维护者提供变更部分并开放源代码。最好的方式是通过电子邮件或者把文件放到服务器上，通过电子邮件传送 URL。如果只修改了很少的部分 (例如，只是一个修改过的 Makefile)，那么传送一个上下文风格的差异比较记录 (context diff) 就可以了。电子邮件的地址是 <maintainer@vim.org>

IV) 不允许从 Vim 的源代码的发行版本或其中部分的源代码里删除本许可证，即使来自更过过的版本也是如此。您可能想用这份许可证代替以前版本的 Vim 里的许可证，这可以由您自行决定。

=== 许可证结束 ===

备注：

- 如果您在 Vim 中找到了乐趣，请阅读剩下的部分，并且考虑一下是不是可以帮助乌干达的儿童。用这种方式来表达您的喜悦之情。

- 如果您想更好地支持 Vim 的开发工作，请考虑成为一名 sponsor 。无论哪种方式，钱都会送往乌干达的。

- Richard Stallman 说，Vim 的许可证与 GNU GPL 兼容。在他看过之后，我做过很少改动，但这不应该有什么影响。

- 如果您在连接时所用的库是以 GPL 发放的，那么即使您没有修改 Vim，未来的发布版本也必须采用 GPL 许可证。

- 一旦一项变更以 GPL 发放，那么这使得以后所有的改动都以 GPL 或与 GPL 兼容的许可证发放。

- 如果您发布一个经过修改的 Vim，您可以在 "configure" 的 "--with-modified-by" 参数里加入您的名字和联系方式，或者直接修改宏 MODIFIED_BY。

Kibaale 儿童中心

kcc Kibaale charity

Kibaale 儿童中心 (Kibaale Children's Centre 简称 KCC) 位于 Kibaale。Kibaale 是东非乌干达南部的一个小镇，靠近坦桑尼亚。这里属于 Rakai 地区。大多数人是农民。虽然人们很穷，但是食物很充足。这是世界上受艾滋病影响最严重的地区，有人说这里是艾滋病的发源地。据估计，有 10% — 30% 的乌干达人是 HIV (人类免疫缺陷病毒，即艾滋病病毒。译者) 的携带者。由于父母双亡，这里有许多孤儿。在这个仅有 35 万人口地区，却有大约 6 万儿童失去了双亲或单亲，而且这种状况还在继续。

孩子们需要很多帮助。KCC 一直很努力地提供一些帮助，比如：食物、医疗和教育。食物和医疗使他们保持健康，而教育可以让他们在以后的生活里照顾自己。KCC 是一个以基督教为基础的组织，但是他们为任何宗教信仰的孩子提供帮助。

解决问题的关键是这一地区的教育。然而在 Idi Amin 总统执政期间以及后来的国内战争的几年时间里，这一问题被忽视了。现在政局已经稳定，孩子们和他们的父母有必要学会怎样照顾自己，以及如何避免感染。饥饿的人和生病的人也同样需要帮助，但我们最主要的目的是让他们远离疾病，教他们怎样种植健康的食物。

大多数孤儿都住在亲戚家里，由叔叔和稍大一点的姐姐照顾他们。因为家庭很大，所以收入 (即使有的话也) 很少。一个孩子如果能得到健康的食物就很幸运了。衣服、医疗、受教育，这些都是奢望。于是，为了帮助这些可怜的孩子，一套捐助程序启动了。一个孩子可以通过人们的捐助来收养。每月捐助几美元，由 KCC 来管理，就可以给孩子们生活必需品，保持健康，以及让他们上学。KCC 还会做所有其它对孩子和抚养他们的家庭有必要的事情。

除了直接帮助孩子外，孩子们的成长环境也有待改善。KCC 帮助学校改进他们的教学方法。在中心有一个示范学校，老师们可以在那里得到培训。同时，也进行医疗健康的人员培训，展开卫生教育活动，每个家庭还被鼓励建造合适的洗手间。我帮忙建设了生产水泥板的地方，水泥板可以用来做一个很好的洗手间。它们的出售价格低于成本价。

我们还建了一个小诊所，可以给孩子和他们的家庭提供一些医疗帮助，如果需要，提供病人转往医院的交通工具。接种疫苗的程序也已经启动，传染病肆虐的时候也可以提供帮助 (麻疹和霍乱曾经是一个问题)。

donate

从 1994 年夏天到 1995 年夏天，我作为志愿者在中心工作了整整一年的时间。其间，我帮助参与了中心的扩建，并从事水和卫生设备方面的工作。我觉得 KCC 所提供的这些帮助真的是很有用。我回到荷兰后，还想继续支持 KCC。于是我筹措资金组织了这个捐助项目。请考虑下面几种捐助方式：

1、捐助一个读小学的孩子：每月 17 欧元 (或更多)。
2、捐助一个读中学的孩子：每月 25 欧元 (或更多)。
3、捐助诊所：每月或每季度，数额不限。
4、一次性捐赠。

相对于其它资助儿童的组织，这笔钱的数额相对很低。这是因为款项直接进入中心，其中不到 5% 的捐款用于管理，对于一个小型的志愿者组织来说，做到这一点并不难。如果您想捐助一名儿童，那么您应该有至少捐助一年的准备。

您怎么知道这些钱用到了该用的地方？首先，以我个人 (Vim 的作者) 的名义向您保证。我信任在中心工作的人，并且对他们个人非常了解。再者，中心是世界宣明会 (World Vision)，拯救儿童基金会 (Save the Children Fund) 合作捐资建成的，并受到过它们的审视，现在由 Pacific Academy Outreach Society 监管。我们几乎每年都自费到中心去查看进展状况，从 1993 年开始，我本人先后访问过中心很多次，在 ICCF 的网站上有相关报道。

如果您还有别的问题，请发送电邮至：<Bram@vim.org>。

中心的地址：

Kibaale Children's Centre
p.o. box 1658
Masaka, Uganda, East Africa

捐资地址：iccf-donations

查看 ICCF 的网站以获取最新的信息！ICCF 的 URL 可以参见： iccf

美国：可以使用下面提到的方法。
很不幸，寄支票给 Nehemiah Group Outreach Society (NGOS) 不再可行了。我们在寻找其它能给你 IRS 纳税收据的方法。
如要捐助儿童，请洽加拿大的 KCF（见下），可寄美国支票到他们那里，以减少转账的开销。

加拿大：联系位于加拿大萨里 (Surrey) 市的 Kibaale 儿童基金会 (Kibaale Children's Fund，简称 KCF)。
他们会接待 Kibaale 儿童在加拿大的捐助者。KCF 把 100% 的钱汇到乌干达的项目中。您可以直接给他们汇款一次性的捐赠。捐赠之后请通知我，好让我知道什么样的捐赠与 Vim 有关。请向 KCF 询问捐助的相关事宜。

Kibaale Children's Fund c/o Pacific Academy
10238-168 Street
Surrey, B.C. V4N 1Z4
Canada
Phone: 604-581-5353

如果您向 KCF 捐赠，您会收到一份纳税收据，可以在报税的时候把它和报税单一并交上。

荷兰：请汇款至在 Lisse 的 "Stichting ICCF Holland" 的账户，如果您住在荷兰，可以减税。

Postbank, nr. 4548774
IBAN: NL95 INGB 0004 5487 74

德国：可以在捐赠时得到税款返还。查阅 ICCF 网站获取更多的信息：

http://iccf-holland.org/germany.html

世界其它地方：通过邮政汇票。这应该在任何国家都可以，主要从邮局开出。我的名字（护照上的名字）是："Abraham Moolenaar"。如果有可能，请兑换成欧元。

欧洲的其它国家：请尽量通过银行汇款，您的银行应有用于此目的的表格。swift 代码和 IBAN 号见下方的 "其它" 一节。
其它方式应该也可以，请询问提供捐助信息的单位。

信用卡：您可以通过 PayPal 来使用信用卡进行捐资，这是 Internet 上应用最广的一个结帐系统，用起来很简单。下面的链接提供详情：

https://www.paypal.com/en_US/mrb/pal=XAC62PML3GF8Q

捐款的电邮地址：

Bram@iccf-holland.org

如果多于 400 欧元 (500 美元)，建议使用支票。

其它：　　请汇款至下面几个帐户之一：

```
    Postbank, account 4548774
                Swift code: INGB NL 2A
                IBAN: NL95 INGB 0004 5487 74
        收款人的名字是 "stichting ICCF Holland", Lisse
```

如果不行，可用：

```
    Rabobank Lisse, account 3765.05.117
                Swift code: RABO NL 2U
        收款人的名字是 "Bram Moolenaar", Lisse
```

否则，请用支票的形式 (美元或欧元) 汇到下面的地址。最小数额：70 美元 (抱歉，我的银行不接收更小数额的国外支票)。

支票请汇到下述地址：

```
    Bram Moolenaar
    Finsterruetihof 1
    8134 Adliswil
    Switzerland
```

该地址应该长期有效。

```
vim:tw=78:ts=8:noet:ft=help:norl:
```

sponsor.txt For Vim version 8.2. 最近更新：2008年4月

VIM 参考手册 by Bram Moolenaar
译者：Dasn, tocer
http://vimcdoc.sf.net

赞助 VIM 的开发 sponsor

修正程序中的漏洞和给程序增加新的功能需要付出很多时间和辛劳。为了表明你对这份努力的感谢，而且激励 Bram 和其他人继续开发 Vim，请您考虑资助一下。

自从 Bram 恢复为全职工作以来，捐款将用于帮助乌干达的儿童。参见 uganda 。但同时，捐款将激励 Bram 继续开发 Vim。

有关赞助的最新消息请访问 Vim 网站：

http://www.vim.org/sponsor/

更多的说明可以在 sponsor-faq 里找到。

VIM 的注册用户 register

您可以通过资助至少 10 欧元的方式成为 Vim 的注册用户。这与赞助 Vim 类似，参见上面的 sponsor 。增加注册这种方式主要是考虑到您的老板或会计可能更愿意去注册一个软件，而不喜欢 "赞助" 和 "捐赠" 的方式。

更多的说明可以在 register-faq 里找到。

投票选出您希望实现的"功能" vote-for-features

相对于普通用户，Vim 的注册用户和资助者的优势在于他们可以投票选出 Bram 下一步应该解决的问题。投票是如何进行的呢?

1. 资助至少 10 欧元。参见下面的几种资助方式 send-money 。
2. 您会收到一封带注册码的电邮。在 Vim 网站您的帐号页面里输入这个注册码。如果您没有帐户，可以注册一个，过程很简单。
3. 这样您就可以在投票页面上投票了。在您输入注册码之后，您的帐户页面上会有一个链接（指向投票页面）。选票的有效期是两年。
4. 投票的结果出现在以下页面，这是大家都能读到的：
   http://www.vim.org/sponsor/vote_results.php

另外，一旦您资助的总额超过了 100 欧元，您的名字就会出现在 "Vim 英雄堂" (Vim hall of honour) 里：http://www.vim.org/sponsor/hall_of_honour.php，不过您需要先在帐户页面里启动这项设置。

如何资助 send-money

信用卡 使用 PayPal，参见 PayPal 网站：
https://www.paypal.com/en_US/mrb/pal=XAC62PML3GF8Q
接收赞助的电邮地址是：
donate@vim.org
接受注册的电邮地址是：
register@vim.org
建议使用欧元，但也接受其它币种。
欧洲国家建议使用银行转帐，成本稍低。

其它方法 见 iccf-donations 。
请在转帐的备注信息里注明 "Vim sponsor" 或 "Vim registration"。
如果您想要进行功能投票或者向别人说明自己是一名注册的 Vim 用户或赞助者，那么请给我发电邮说明您资助的数额。

现金 少量现金可以通过普通的信件邮寄。把钱用东西包好，让别人从外面看不出来。如果您想要进行功能投票或者向别人说明自己是一名注册的 Vim 用户或赞助者，那么请同时写下您的电邮地址。

您可以使用下面的永久地址：

Bram Moolenaar
Finsterruetihof 1
8134 Adliswil
Switzerland

常见问题解答 sponsor-faq register-faq

为什么我应该资助?

如果没有您对 Vim 工作的欣赏，Bram 不会这么有动力修正漏洞并增加新功能。他会考虑干点别的。

我应该资助多少数额?

这取决于您。您多给一些，就可以救助更多的儿童。下面的建议可供参考，个人用户在家里应用：每年 10 欧元。专业应用：每年 30 欧元。至少要资助 10 欧元才能进行功能投票。

我能得到什么回报?

每一个资助了至少 10 欧元的注册用户和赞助者，都可以为自己需要的功能投票。选出的这些功能会被优先考虑在 Vim 中实现。投票的有效期是两年。资助的数额越多，您的选票的点数越多 votes-counted 。

如果助的数额总数超过了 100 欧元，您的名字会出现在 Vim 网站的 "Vim 英雄堂"。但是必须在您的帐户页面上启用这项功能。您也可以选择您的资助数额是否可见。

我怎样成为 Vim 的赞助者或 Vim 的 注册用户?

通过上面 send-money 解释的方法进行资助，并写上您的电邮地址。钱款收到之后，您会收到一个唯一的注册码。这个注册码可以被用来激活您在 Vim 网站上的帐户的投票功能。您将会得到一个额外的用来投票的页面，并且可以选择您的捐助是否可见。在 "我的帐户" ("My Account") 里有一个链接指向那个页面。

赞助和注册有什么不同?

名字不同。如果您的老板不喜欢 "赞助" 或 "捐赠"，那就用 "注册" 的形式。结果是一样的。

我应当如何汇款?

参见: send-money 。关注网站以获得最新消息:
http://www.vim.org/sponsor/

为什么你不用 SourceForge 的捐赠系统?

SourceForge 会把 5% 的捐赠留给他们自己。如果您想支持 SourceForge，可以直接资助他们。

我付不起钱，可以继续使用 Vim 么?

可以。

我没有注册 Vim，我能使用全部的功能吗?

可以。

我发现了一个漏洞 (bug)，我必须注册才能报告么?

不需要。您永远可以提供改进 Vim 的建议。发表改进的意见请使用开发者邮件列表 maillist ，报告漏洞参见 bugs 。

选票是如何统计的? votes-counted

当您资助了 10 欧元或更多时，您可以投票。您至多可以投十票。可以在一个选项上多次投票，以便给它更多的点数。您还可以投三次反对票，以负点数计。

当您资助的数额多于 30 欧元，您的点数会翻番。超过 100 欧元，点数是原来的四倍，超过 300 欧元变成六倍，1000 欧元以上十倍。

我能更改自己的投票吗?

在您资助之后的两年时间里，您可以在任何您喜欢的时间更改您的选票。选票的点数将立即生效。

我能增加投票的项目么?

您无法直接增加。您可以向 Bram 建议。他将考虑把您的提议加进去。

关于慈善软件?

目前，Vim 的资助总是到乌干达 uganda 。不论您赞助 Vim 还是 ICCF 都是一样。不过赞助 Vim，您同时可以对新特性投票。

我已经捐了 $$$，那么请加入 XYZ 新功能!

Bram 的工作和您的捐助没有直接的关系。否则，就变成了您为 Bram 的工作付钱，而我们应该为这些捐助缴税。如果您想雇 Bram 做一些别的工作，直接和他联系，请不要使用捐助系统。

捐助可以免税么?

这取决于您的国家。赞助乌干达 Uganda 的儿童在荷兰、德国、加拿大，还有美国是免税的。见 ICCF 的网站 http://iccf-holland.org/donate.html。您可以发送电邮告知 Bram，让他知道您的捐献是因为使用 Vim 的结果。

你能寄给我发票么?

对不起，因为您捐助的金钱和我们完成的工作并没有关联。但寄收据是可以的。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册

by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

启动 Vim starting

1. Vim 参数 vim-arguments
2. Amiga 上的 Vim starting-amiga
3. 运行 eVim evim-keys
4. 初始化 initialization
5. $VIM 和 $VIMRUNTIME $VIM
6. 暂停 suspend
7. 退出 exiting
8. 保存设置 save-settings
9. 视图和会话 views-sessions
10. viminfo 文件 viminfo-file

---

1. Vim 参数 vim-arguments

最常见的是启动 Vim 编辑单个文件。使用的命令是

```
vim filename
```
-vim

更一般的，Vim 是这么启动的：

```
vim [option | filename] ..
```

选项参数和文件名参数可以混合使用，使用的数目也没有限制。不过，小心带参数的选项。

关于各个 Vi 版本的兼容性，见 cmdline-arguments 。

开始编辑时，以下五项可以且只能选一个。

-file ---

文件名　一个或多个文件名。其中的第一个将成为当前文件并读入缓冲区。光标会定位在缓冲区的第一行。
要避免以 '-' 开始的文件名被解释成选项，在文件名列表前加上 "--"。例如：

```
vim -- -filename
```

在 "--" 之后的所有参数被解释为文件名。也就是说，后面不能再跟选项或者 "+command" 参数。
关于 MS-Windows 上引号的行为，见 win32-quotes 。

--

\-　该参数有两个含义，取决于是否使用 Ex 模式。

以普通模式启动：

```
vim -
```

```
ex -v -
```
开始编辑新缓冲区，并以标准输入读取的文本填充之。通常从标准输入读入的命令现在从标准错误 (stderr) 读入。例如：
```
find . -name "*.c" -print | vim -
```
缓冲区不会被标为已修改，以方便退出。如果不想意外地丢失，可以小心地把缓冲区标记成已修改。例如：
```
ls | view -
```

以 Ex 模式启动：
```
ex -
vim -e -
exim -
vim -E
```
开始编辑时处于安静模式。见 -s-ex 。

-t {tag} -t -tag

一个标签。从标签文件里查找 "tag"。相关的文件成为当前文件，并执行相关的命令。通常用于 C 程序，此时 "tag" 通常是函数名。其效果是，包含该函数的文件称为当前文件，而光标定位在函数的开始处 (见 tags)。

-q [errorfile] -q -qf

QuickFix 模式。名字为 {errorfile} 的文件被读入并显示第一个错误。 见 quickfix 。
如果不给出 [errorfile]，使用 'errorfile' 选项给定的文件名。其缺省值见 'errorfile'。

(无)

不带上面四种参数的任何一种，Vim 开始编辑一个新的缓冲区。它是空白的，也不带文件名。

启动模式也可以通过使用程序的名字 (替代 "vim") 来指定。具体来说，不同的名字等价于给出不同的选项。见下：

ex	vim -e	以 Ex 模式启动 (见 Ex-mode)。	ex
exim	vim -E	以增强的 Ex 模式启动 (见 Ex-mode)。 (通常并没有安装)	exim
view	vim -R	以只读模式启动 (见 -R)。	view
gvim	vim -g	以 GUI 启动 (见 gui)。	gvim
gex	vim -eg	以 Ex 模式启动 GUI。	gex
gview	vim -Rg	以只读模式启动 GUI.	gview
rvim	vim -Z	和 "vim" 类似，但以受限模式启动 (见 -Z)	rvim
rview	vim -RZ	和 "view" 类似，但以受限模式启动。	rview
rgvim	vim -gZ	和 "gvim" 类似，但以受限模式启动。	rgvim
rgview	vim -RgZ	和 "gview" 类似，但以受限模式启动。	rgview
evim	vim -y	简易 Vim: 置位 'insertmode' (见 -y)	evim
eview	vim -yR	和 "evim" 类似，启用只读模式	eview
vimdiff	vim -d	以比较模式启动 diff-mode	
gvimdiff	vim -gd	以比较模式启动 GUI diff-mode	

程序名字后面还可以有附加的字符，它们都被忽略。例如，你可以用 "gvim-5" 来启动 GUI。当然，你必须有这个名字的可执行文件。

Unix 上，你通常有一个可执行文件叫 Vim，并链接该可执行文件到不同的启动名字。如果你的系统不支持链接，但是你不想保留该可执行文件的多个备份，可以使用别名。例

如:

```
alias view   vim -R
alias gvim   vim -g
```

startup-options

选项参数可以任何顺序给出。单个字母的选项可以在一个连字号后一起给出。在 "--" 参数之后，就不能再有任何选项参数了。

VMS 上，所有的选项参数假定都是小写的，除非前面有一个正斜杠。这样，"-R" 应该意味着恢复而 "-/R" 表示只读。

--help -h --help -?
-?
-h 给出使用说明（帮助）并退出。
关于捕获文本，见 info-message 。

--version
--version 显示版本信息并退出。和 :version 命令的输出结果相同。
关于捕获文本，见 info-message 。

--noplugin
--noplugin 跳过插件的载入。复位 'loadplugins' 选项。
注意 -u 参数可能会屏蔽载入的插件:

参数	载入: vimrc 文件	插件	defaults.vim
(无)	是	是	是
-u NONE	否	否	否
-u DEFAULTS	否	否	是
-u NORC	否	是	否
--noplugin	是	否	是

--startuptime {fname} --startuptime
启动时把计时信息写入文件 {fname}。可用于分析载入 .vimrc、插件和打开首个文件的过程中时哪一步最耗时，
如果 {fname} 已存在，在后面附加新消息。
{仅当编译时带有 +startuptime 特性才有效}

--literal
--literal 按本义取文件名，不扩展通配符。在 Unix 上不需要，因为那里 Vim 总是按本义取文件名的（外壳程序扩展通配符）。
适用于所有名字，包括本参数之前的那些名字。

-+
+[num] 光标在编辑第一个文件时会定位在第 "num" 行。如果不给出 "num"，光标会定位在最后一行。

-+/
+/{pat} 光标在编辑第一个文件时会定位在第一个包含 "pat" 模式的行（可用的搜索模式可见 pattern ）。搜索从光标位置开始，这可以是第一行或者来自 viminfo 的上次使用的光标位置。要强制从第一行开始搜索，用 "+1 +/pat"。

+{command} -+c -c
-c {command} 在读入第一个文件后执行命令 {command}（在自动命令和文件的模式行处理之后）。"command" 按 Ex 命令来解释。如果 "command" 包含空

格，应该用双引号括起该命令（取决于使用的外壳）。
示例：

```
vim  "+set si"  main.c
vim  "+find stdio.h"
vim  -c "set ff=dos"  -c wq  mine.mak
```

注意：在一个 Vim 命令里，你可以使用不超过 10 个 "+" 或者 "-c" 参数。它们的执行按给定的顺序进行。"-S" 参数也被当作 "-c" 参数那样对待。

--cmd {command}　　--cmd

在处理任何 vimrc 文件之前执行命令 {command}。除此以外，和 -c {command} 类似。你可以使用不超过 10 个本命令，不占用 "-c" 命令的数目限制。

-S　　-S

-S {file}　在第一个文件读入之后，执行脚本 {file}。这是下面这条等价的命令的一个简便方式：

```
-c "source {file}"
```

它可以和 "-c" 参数混合使用，也和 "-c" 那样可以使用多次。"-c" 参数的 10 个的限制也应用于此。
{file} 不能以 "-" 开始。

不要用此形式来执行脚本做事并退出 Vim，你不会看到错误信息。使用 -u 来代替。

-S　　和 "-S Session.vim" 类似。只有在用作最后一个参数或者其后有另外一个 "-" 选项时才会被这么理解。

-r　　-r

-r　　恢复模式。如果没有文件名参数，使用已有的所有交换文件。如果给出文件名，读入相应的交换文件以恢复以前崩溃的编辑会话。见 crash-recovery 。

-L　　-L

-L　　和 -r 相同。

-R　　-R

-R　　只读模式。为所有编辑的文件置位 'readonly' 选项。你仍然可以编辑缓冲区，但你不能实际覆盖文件。如果你忘记了你处于只读 (View) 模式而试图进行改动，仍然可以在 Ex 命令后面附加感叹号来覆盖文件，比如 ":w!"。'readonly' 选项可以用 ":set noro" 复位（见选项一章， options ）。
后来开始的编辑并不自动进入只读模式。调用可执行的 "view" 和给出 -R 参数有相同的效果。
'updatecount' 选项设为 10000，意味着交换文件不会很经常的自动更新。
-M 可用来禁止修改。

-m　　-m

-m　　不允许写入修改的部分。复位 'write' 选项，从而屏蔽了文件的写入。不过，之后可以重新置位 'write' 选项来打开写文件的功能。

-M

-M　　不允许修改。复位 'modifiable' 选项，从而禁止任何改动。也复位 'write' 选项，从而写入文件也被屏蔽。不过，'modifiable' 和 'write' 选项都可以被重新设置来允许修改和写入文件。

-Z　　-Z restricted-mode E145 E981
受限模式。使用外部外壳的任何命令都被禁止。这包括使用 CTRL-Z 暂停、":sh"、过滤程序，system() 函数，反引号扩展 和 libcall()。
也禁止 delete()、rename()、mkdir()、 job_start() 等等。
也禁止 Python、Ruby 和 Lua 接口，因为它们可用来执行外壳命令。
Perl 使用安全模块。
注意 用户仍然可能找到空子来执行外壳命令，这越来越困难了。

-g　　-g
启动 Vim 并进入 GUI 模式。见 gui 。相反的功能可见 -v 。

-v　　-v
启动 Ex 进入 Vi 模式。只有在可执行文件名字叫 "ex" 或者 "gvim" 的时候才有意义。对于 gvim 而言，即使能够，也不启动 GUI。

-e　　-e
启动 Vim 进入 Ex 模式 Q ，只有在可执行文件不叫 "ex" 的时候才有意义。

-E　　-E
启动 Vim 进入增强的 Ex 模式 gQ 。只有在可执行文件不叫 "exim" 的时候才有意义。

-s　　-s-ex
安静或批处理模式。只有在 Vim 使用 "ex" 名字或者前面以 "-e" 参数启动的时候才有意义。否则的话，见 -s ，后者接受参数，但是本参数不需要。Vim 用来从文件而不是终端读取 Ex 命令并执行之。关闭大多数提示和信息消息，也包括警告和错误消息。
这些命令的输出仍然显示 (到标准输出 (stdout)):

```
:print
:list
:number
:set       显示选项值。
```

如果 'verbose' 非零，则消息也会显示 (用于调试，到标准错误 (stderr))。
不使用 'term' 和 $TERM。
如果 Vim 看起来没有动静，试试输入 "qa!<Enter>"。你不会得到提示，所以你没法看到 Vim 在等待你输入。
跳过初始化 (除了通过 "-u" 参数给出的那些)。
示例:

```
vim -e -s  < thefilter  thefile
```

相反地，要看到脚本的错误，带 -u 标志位执行文件:

```
vim -u thefilter thefile
```

-b　　-b
二进制模式。文件 I/O 只会用 <NL> 来分隔行。'expandtab' 选项被复位。'textwidth' 选项设为 0。'modeline' 被复位。'binary' 选项被置位。这发生在 vimrc/exrc 文件被读入以后，但在读入文件参数列表之前。另见 edit-binary 。

-l

-l　　Lisp 模式。置位 'lisp' 和 'showmatch' 选项。

-A

-A　　阿拉伯模式。置位 'arabic' 选项。
{仅当编译时加入 +arabic 特性才有效（这也包含 +rightleft ）。否则 Vim 给出错误信息并退出}

-F

-F　　曾用于波斯模式，此支持已删除。见 farsi.txt 。

-H

-H　　希伯来模式。置位 'hkmap' 和 'rightleft' 选项。
{仅当编译时加入 +rightleft 特性才有效。否则 Vim 给出错误信息并退出}

-V verbose

-V[N]　　详细。设置 'verbose' 选项为 [N]（缺省：10）。对每个被执行的脚本和读入或写入的 viminfo 文件都会给出消息。可以用来了解启动和退出时发生了什么事情。
示例：

```
vim -V8 foobar
```

-V[N]{filename}

类似于 -V 并同时设置 'verbosefile' 为 {filename}。结果是，消息不显示，而是写到文件 {filename}。{filename} 不能以数字开始。
示例：

```
vim -V20vimlog foobar
```

-D

-D　　调试。在执行脚本的第一个命令的时候进入调试模式。 debug-mode
{仅当编译时加入 +eval 特性才有效}

-C

-C　　兼容模式。置位 'compatible' 选项。你可以用此参数进入 'compatible' 模式，即使存在 .vimrc 文件也是如此。
要 注意 有些插件或启动脚本里的命令 ":set nocompatible" 总可以超越之，所以你最后还是用了 "nocompatible"。要找出原因：

```
:verbose set compatible?
```

有些插件在置位 'compatible' 时不能工作，你需要在启动后这样置位之：

```
vim "+set cp" filename
```

另见 compatible-default 。

-N

-N　　不兼容模式。复位 'compatible' 选项。你可以用此参数设置 'nocompatible'，即使不存在 .vimrc 文件或使用 "-u NONE" 时也是如此。
另见 compatible-default 。

-y easy

-y　　简易模式。 evim 和 eview 隐含本参数。开始时置位 'insertmode'，并表现的像点击-输入的编辑器那样。设置该参数会执行脚本 $VIMRUNTIME/evim.vim。设置相应的映射，使之和多数点击-输

入的编辑器行为类似。见 evim-keys 。如果可能，启动 GUI。

-n

-n

不使用交换文件。在崩溃后因而无法恢复。如果你只想浏览或者在很慢的媒介上（例如，软盘上）编辑文件，这也许有用。
也可以通过 ":set updatecount=0" 完成。你可以通过给 'updatecount' 选项一些值来重新打开，例如 ":set uc=100"。
注意：不用混用 -n 和 -b，使之成为 -nb，这是另一个意思： -nb 。
执行 vimrc 文件的命令 后 ，'updatecount' 才置为 0，但这发生在 GUI 初始化之前。因此，该选项会覆盖 vimrc 文件里对 'updatecount' 的设置，但不会覆盖 gvimrc 的。见 startup 。
如果你想减少对磁盘的访问（例如在笔记本上），不要使用 "-n"，而是设置 'updatetime' 和 'updatecount' 为很大的值，然后在需要保存你的工作的时候输入 ":preserve"。这使得你可以从崩溃中恢复。

-o[N]

-o

打开 N 个窗口，水平分割。如果没有给出 [N]，为每个文件参数打开一个窗口。如果没有足够的空间，只打开前面数项。如果给出的窗口数超过参数的数目，后面的窗口会编辑空文件。

-O[N]

-O

打开 N 个窗口，垂直分割。此外和 -o 类似。如果命令行上同时给出 -o 和 -O 选项，较后的那个决定窗口如何分割。

-p[N]

-p

打开 N 个标签页。如果没有给出 [N]，为每个文件参数打开一个标签页。最大标签页数由 'tabpagemax' 决定（缺省为 10）。如果有比参数更多的标签页，后面的标签页会编辑空文件。另见 tabpage 。

-T {terminal}

-T

设置终端名为 "terminal"。这影响 Vim 发送给你终端的代码。通常不需要，因为 Vim 能够找到你使用的终端类型（见 terminal-info ）。

--not-a-term

--not-a-term

告知 Vim 用户确定输入和/或输出不连接到终端。可用于避免相关警告和因此发生的两秒延迟。
同时也避免了 "Reading from stdin..." 信息。
同时也避免了 "N files to edit" 信息。

--ttyfail

--ttyfail

标准输入或标准输出不是终端（tty）时立即退出。

-d

-d

以比较模式启动，和 vimdiff 类似。
{仅当编译时加入 +diff 特性才有效}

-d {device}

只出现在 Amiga 上，并且编译时没有加入 +diff 特性才适用。
和 "-dev" 类同。

-dev {device}

-dev

只出现在 Amiga 上，打开 {device} 设备，并用来编辑。
通常你会使用该参数来设置窗口位置和大小："-d con:x/y/width/height"，比如 "-d con:30/10/600/150"。但是你可以用之在别的设备上启动编辑，比如，AUX:。

-f

-f

GUI: 不要从启动 Vim 的程序那里断开。'f' 代表 "foreground" (前台)。如果不提供，GUI 分叉 (fork) 出一个新进程，并结束当前的进程。 "-f" 应该用于 gvim 被别的程序启动，并且需等待编辑会话完成的场合 (例如，mail 或者 readnews)。如果你希望 gvim 永远不要分叉，在你的 gvimrc 里给 'guioptions' 加上 'f'。
小心: 你可以使用 "-gf" 在前台启动 GUI，但是 "-fg" 是用来指定前台颜色的。 gui-fork

Amiga: 不要在新窗口上启动 Vim。该选项应该用于 Vim 被别的程序启动，并且需等待编辑会话完成的场合 (例如，mail 或者 readnews)。见 amiga-window 。

MS-Windows: 不支持此选项。但用已安装的 vim.bat 或 gvim.bat 运行 Vim 时就可以如此工作。

--nofork

--nofork

GUI: 不分叉。和 -f 相同。

-u E282

-u {vimrc}

读入文件 {vimrc} 执行初始化。跳过多数其他的初始化步骤；见 initialization 。

这可以用来启动 Vim 进入特殊模式，使用特殊的映射和设置。可以用外壳映射来方便使用之。例如:

```
alias vimc vim -u ~/.c_vimrc !*
```

也可以考虑自动命令；见 autocommand 。

如果 {vimrc} 等于 "NONE" (全是大写)，所有文件和环境变量的初始化都被跳过，这包括 GUI 开始时候 gvimrc 文件的读入。插件的载入也被跳过。

如果 {vimrc} 等于 "NORC" (全是大写)，那么和 "NONE" 基本相同，但不跳过插件的载入。

如果 {vimrc} 等于 "DEFAULTS" (全是大写)，那么和 "NONE" 基本相同，但载入 defaults.vim 脚本，其中置位 'nocompatible'。另见 --clean 。

除了 DEFAULTS 以外，使用 "-u" 参数的副作用是 'compatible' 选项缺省是打开的。这可能有意想不到的效果。见 'compatible' 。

-U E230

-U {gvimrc}

在 GUI 启动的时候读入 {gvimrc} 文件并执行初始化。跳过其它的 GUI 初始化。如果 {gvimrc} 等于 "NONE"，GUI 初始化过程不读入任何文件。 gui-init
特例: 系统菜单文件总会读入。

-i

-i {viminfo}

使用 "viminfo" 文件而不是缺省的 viminfo 文件。如果使用名字 "NONE" (全是大写)，则即使设置了 'viminfo' 或者使用 ":rv" 或 ":wv" 的时候，也不读写 viminfo 文件。另见 viminfo-file 。

--clean

--clean

类似于 "-u DEFAULTS -U NONE -i NONE":
- 跳过文件和环境变量的初始化

- 设置 'runtimepath' 和 'packpath' 把主目录项目排除在外 (-u DEFAULTS 并不发生)。
- 载入 defaults.vim 脚本，隐含 'nocompatible': 使用 Vim 缺省值
- 不载入 gvimrc 脚本
- 不读也不写 viminfo 文件

-x

-x 读写文件都使用加密。会提示密钥，该密钥存在 'key' 选项里。所有的写操作使用该密钥加密文本。 '-x' 参数在读文件的时候是不需要的，因为待读取的文件是否经过加密总是会检查的。Vim 会自动询问密钥。 encryption

-X

-X 不试图连接到 X 服务器以取得当前窗口的标题，也不使用 X 剪贴板进行复制/粘贴。这避免了在终端模拟器里运行 Vim 的时候因为慢速的 X 服务器连接而导致的漫长的启动时间。
--startuptime 可以帮助你发现导致缓慢的原因。
只有在 Unix 或 VMS 上并编译时使用 +X11 特性才有意义。不然该参数被忽略。
如果要为特定的终端禁止连接，见 'clipboard' 选项。
如果内建了 X11 会话管理协议 (XSMP) 的处理程序，-X 选项也关闭到它的连接，以减少意想不到的延迟。
如果后来又希望恢复连接 (比如，客户-服务器间的消息)，调用 serverlist() 函数。不过，这并不会重新打开 XSMP 的处理。

-s

-s {scriptin} 读入脚本文件 "scriptin"。文件里的字符就如同你输入那样被解释。这和命令 ":source! {scriptin}" 相同。如果在编辑器结束之前就遇到文件尾，后续的字符从键盘读入。只有在不是 Ex 模式启动的时候才工作。见 -s-ex 。另见 complex-repeat 。

-w_nr

-w {number}
-w{number} 设置 'window' 选项为 {number}。

-w

-w {scriptout} 所有你输入的字符记录在 "scriptout" 文件里，直到 Vim 结束为止。这可用于创建一个脚本文件，以后可以用 "vim -s" 或 ":source!" 重新执行之。如果 "scriptout" 文件已经存在，新的字符会附加在其后。另见 complex-repeat 。
{scriptout} 不能以数字开始。

-W

-W {scriptout} 和 -w 类似，但不是附加，而是覆盖已经存在的文件。

--remote [+{cmd}] {file} ...
在另外一个作为服务器的 Vim 里打开 {file}。任何非文件的参数必须在此之前出现。
见 --remote 。

--remote-silent [+{cmd}] {file} ...
和 --remote 类似，但如果找不到服务器，不会抱怨。
见 --remote-silent 。

--remote-wait [+{cmd}] {file} ...
和 --remote 类似，但等待服务器结束文件的编辑。
见 --remote-wait 。

--remote-wait-silent [+{cmd}] {file} ...
和 --remote-wait 类似，但如果找不到服务器，不会抱怨。
见 --remote-wait-silent 。

--servername {name}
指定 Vim 服务器的名字，用以向它发送，会用于自己建立之。
见 --servername 。

--remote-send {keys}
向 Vim 服务器发送键码序列 {keys} 并退出。
见 --remote-send 。

--remote-expr {expr}
在另外一个作为服务器的 Vim 里执行表达式 {expr}。结果在标准输出里显示。
见 --remote-expr 。

--serverlist
输出 Vim 服务器名字的列表并退出。
见 --serverlist 。

--socketid {id} --socketid
只限于 GTK+ GUI Vim。gvim 会试图使用 GtkPlug 机制，这样它就可别的窗口里面运行。 详情见 gui-gtk-socketid 。

--windowid {id} --windowid
只限于 Win32 GUI Vim。gvim 会尝试使用窗口 {id} 作为父窗口，这样它就在该窗口内运行。详见 gui-w32-windowid 。

--echo-wid --echo-wid
只限于 GTK+ GUI Vim。gvim 会在标准输出显示窗口 ID。可以用之使得 gvim 在 kpart 窗口组件里运行。输出格式是:

```
WID: 12345\n
```

--role {role} --role
只限于 GTK+ 2 GUI。设置主窗口的角色为 {role}。窗口管理器使用窗口角色来唯一表示窗口，从而可以恢复窗口布置等。登录时恢复对话时会自动传递 --role 参数。见 gui-gnome-session 。

-P {parent-title} -P MDI E671 E672
只限于 Win32: 指定父应用程序的标题。如果可能，Vim 会找到该程序，并在该应用程序里以 MDI 窗口运行。
{parent-title} 必须出现在父应用程序的窗口标题里。确信它足够特殊。
注意 现在的实现还很原始。不能适用于所有的应用程序，而且也不能使用菜单。

-nb -nb
-nb={fname}
-nb:{hostname}:{addr}:{password}

试图连接到 Netbeans 并成为它的一个编辑器服务器。第二种形式指定一个文件，从该文件里可以读取连接的信息。第三种形式指定 Netbeans 需要的机器名、地址和密码信息。 netbeans-run {仅当编译时加入 +netbeans_intg 特性才有效；如果没有 -nb 会使 Vim 退出}

如果可执行文件叫 "view"，Vim 会以可读模式启动。如果你可以创建从 "vim" 到 "view" 的硬链接或者符号链接，就可以这么用。
也可以用 "vim -R" 启动只读的 Vim。

如果可执行文件叫 "ex"，Vim 会以 "Ex" 模式启动。这意味着它只接受 ":" 命令。不过如果给出 "-v" 参数，Vim 仍然会以普通模式启动。

在 unix 系统上如果编译时加入 X11 GUI 支持，还可以使用其它参数。见 gui-resources 。

---

2. Amiga 上的 Vim starting-amiga

从 Workbench 启动 Vim workbench

Vim 可以从 Workbench 通过双击其图标启动。这时，启动时使用空白缓冲区。

Vim 可以通过 "Project" 图标开始编辑一到多个文件。图标的 "Default Tool" 应该是 Vim 可执行文件的完整路径。".info" 文件名应该和文本文件的名字相同。通过双击图标，Vim 启动时编辑该文件名，并作为当前文件名，读入缓冲区 (如果该文件存在的话)。你可以通过点击图标时按住 Shift 键，并在最后一个文件上双击来编辑多个文件。所有这些图标的 "Default Tool" 都应该一致。

除了文件名以外，通过 workbench 不可能给 Vim 传递别的参数。

Vim 窗口 amiga-window

Vim 在启动的 CLI 窗口里运行。如果 Vim 以 "run" 或 "runback" 命令启动，或者 Vim 通过 workbench 启动的话，它会给自己打开一个窗口。

技术细节:
要打开一个新窗口，要用一点小技巧。一旦 Vim 认识到它不可能在正常的 CLI 窗口里运行，它会在 "t:" 建立一个脚本文件。该脚本文件包含 Vim 启动时的相同命令，还有 "endcli" 命令。然后，该脚本通过 "newcli" 命令执行 (这需要 "c:run" 和 "c:newcli" 命令)。脚本文件会保留，直到重启或者你删除它的时候为止。该方法需要 ":sh" 和 ":!" 命令能够正确工作。但如果 Vim 以 -f 选项 (前台模式) 启动，就不使用这个方法。原因是，一个程序以 -f 选项启动 Vim 时，它等待 Vim 返回。使用这个脚本的技巧，调用程序没法知道 Vim 什么时候真的退出。比如，mail 程序可以使用 -f 选项来启动 Vim，然后等待编辑会话的结束。受其影响，":sh" 和 ":!" 命令在应用 -f 选项的时候不能使用。

Vim 会自动识别窗口大小，并对窗口的大小调整做出反应。在 Amiga DOS 1.3 里面，建议使用 fastfonts 程序 "FF"，来加快显示的重画速度。

---

3. 运行 eVim evim-keys

EVim 运行的 Vim 使用了点击-输入的编辑器风格。这和原来的 Vi 的设计大有分别。但是对不常用 Vim 的人学习 Vim 命令有帮助。希望他们能发现使用普通模式下的命令会使得编辑更为有效。

Evim 里，以下选项被改变，不同于缺省值：

:set nocompatible	使用 Vim 的改进
:set insertmode	大部分时间里保持在插入模式里
:set hidden	使得不可见的缓冲区保持载入
:set backup	保持备份文件 (VMS 没有)
:set backspace=2	退格可以删除任何东西
:set autoindent	自动缩进新行
:set history=50	保持 50 行的 Ex 命令历史
:set ruler	显示光标位置
:set incsearch	输入模式过程中动态显示部分模式对应的匹配
:set mouse=a	在所有模式下使用鼠标
:set hlsearch	高亮搜索模式的所有匹配
:set whichwrap+=<,>,[,]	<Left> 和 <Right> 在换行符上回绕
:set guioptions-=a	只限于非 Unix: 不要自动选择

键盘映射：

<Down>	在屏幕行而不是文件行间移动
<Up>	同上
Q	等同于 "gq"，排版，而不是 Ex 模式
<BS>	可视模式：删除选择
CTRL-X	可视模式：剪切到剪贴板
<S-Del>	同上
CTRL-C	可视模式：复制到剪贴板
<C-Insert>	同上
CTRL-V	粘贴到剪贴板 (在任何模式下都可以)
<S-Insert>	同上
CTRL-Q	做以前 CTRL-V 做的事
CTRL-Z	撤销
CTRL-Y	重做
<M-Space>	系统菜单
CTRL-A	选择所有
<C-Tab>	下一个窗口，CTRL-W w
<C-F4>	关闭窗口，CTRL-W c

另外：
- 使用 ":behave mswin" :behave
- 打开语法高亮
- 使用文件类型检测。也使用文件类型插件和缩进
- 在文本文件里，'textwidth' 设为 78

一个提示：如果你想进入普通模式并可以输入一串命令，使用 CTRL-L。 i_CTRL-L

---

4. 初始化 initialization startup

本节是关于 Vim 的非 GUI 版本的。关于启动 GUI 的额外初始化步骤，见 gui-fork 。

启动时，Vim 检查环境变量和文件，并相应设置值。Vim 按以下顺序处理：

1. 设置 'shell' 和 'term' 选项 SHELL COMSPEC TERM
   环境变量 SHELL，如果存在的话，用来设置 'shell' 选项。MS-DOS 和 Win32 上，如果没有设置 SHELL 的话，使用 COMSPEC 变量。
   环境变量 TERM，如果存在的话，用来设置 'term' 选项。不过，后来在启动 GUI 时会改变 'term'（下面的第 8 步）。

2. 处理参数
   Vim 启动时命令行上给出的选项和文件名被检查。为所有的文件创建缓冲区（但这时还没有载入）。
   -V 参数可以用来显示或记录下一步发生了什么，用于对初始化进行调试。

3. 根据环境变量和/或文件执行 Ex 命令
   读入某个环境变量，作为 Ex 命令执行之，多个命令可以用 '|' 或 "<NL>" 分隔。

   vimrc exrc
   包含初始化命令的文件叫做 "vimrc" 文件。vimrc 文件的每一行都作为 Ex 命令执行。有时，它也被叫做 "exrc" 文件。它们是同一种类型文件。但是 Vi 一直用 "exrc"。而 "vimrc" 是 Vim 专用的名字。另见 vimrc-intro 。

   保存你个人初始化的地方是：

Unix	$HOME/.vimrc 或 $HOME/.vim/vimrc
MS-Windows	$HOME/_vimrc、$HOME/vimfiles/vimrc 或 $VIM/_vimrc
Amiga	s:.vimrc、home:.vimrc、home:vimfiles:vimrc 或 $VIM/.vimrc

   按上述顺序依次寻找，只读入首个成功找到的文件。

   推 荐：把所有的 Vim 配置放在 $HOME/.vim 目录底下（MS-Windows 则是 $HOME/vimfiles）。这样方便复制到别的系统上。

   如果 Vim 启动时使用了 "-u filename" 参数，则使用文件 "filename"。直到第四步以前的所有初始化都被跳过。不设置 $MYVIMRC。
   "vim -u NORC" 可以用来跳过这些初始化，而且不读入任何文件。
   "vim -u NONE" 同时也跳过插件的载入。 -u

   如果 Vim 启动时使用了 "-s" 参数，所有直到第四步以前的所有初始化都被跳过。只有 "-u" 选项会被解释。

   evim.vim
   a. 如果 vim 以 evim 或 eview 之名启动，或带上 -y 参数，载入 $VIMRUNTIME/evim.vim 脚本。

   system-vimrc
   b. 对 Unix、MS-Windows、VMS、Macintosh、RISC-OS 和 Amiga，读入系统 vimrc 文件执行初始化。该文件的路径可见于 ":version" 命令。多数情况下，它是 "$VIM/vimrc"。
   注意 该文件 总 是以 'compatible' 模式读入的，因为自动复位 'compatible' 只有可能在以后发生。如果你喜欢，加上 ":set nocp" 命令。
   对 Macintosh，读入 $VIMRUNTIME/macmap.vim。

   VIMINIT .vimrc _vimrc EXINIT .exrc _exrc $MYVIMRC
   c. 在五个地方依次搜索初始化命令。第一个找到的地方被使用，余者被忽略。
   $MYVIMRC 环境变量被设为最先找到的文件，除非 $MYVIMRC 已经被设置和

(译者注：或？) 使用 VIMINIT。

I 环境变量 VIMINIT (另见 compatible-default) (*)
$VIMINIT 的值作为 Ex 命令行来使用。
II 用户 vimrc 文件:
"$HOME/.vimrc" (用于 Unix) (*)
"$HOME/.vim/vimrc" (用于 Unix) (*)
"s:.vimrc (用于 Amiga) (*)
"home:.vimrc" (用于 Amiga) (*)
"home:vimfiles:vimrc" (用于 Amiga) (*)
"$VIM/.vimrc" (用于 Amiga) (*)
"$HOME/_vimrc" (用于 Win32) (*)
"$HOME/vimfiles/vimrc" (用于 Win32) (*)
"$VIM/_vimrc" (用于 Win32) (*)
注意：对 Unix 和 Amiga 而言，如果 ".vimrc" 不存在，则尝试 "_vimrc"。因为有可能使用了 MS-DOS 兼容的文件系统。
对 MS-Windows 而言，".vimrc" 在 "_vimrc" 之后检查，因为可能可以使用长文件名。
注意：对 Win32 而言，先检查 "$HOME"。如果那里没有 "_vimrc" 或 ".vimrc"，再找 "$VIM"。如果 $VIM 没有设置，见 $VIM 。
III 环境变量 EXINIT。
$EXINIT 的值作为 Ex 命令行来使用。
IV 用户 exrc 文件。和用户 vimrc 文件一样，只不过 "vimrc" 被 "exrc" 代替而已。但只会使用 ".exrc" 或 "_exec" 两者之一，取决于不同系统。而且不适用 (*)!
V 缺省 vimrc 文件，$VIMRUNTIME/defaults.vim。它设置选项值，并执行 "syntax on" 和 "filetype on" 命令，这是绝大多数新用户所需要的。见 defaults.vim 。

d. 如果打开 'exrc' 选项 (并非 缺省)，在当前目录搜索三个文件。最先找到的被使用，其余的忽略。
- 文件 ".vimrc" (用于 Unix、Amiga) (*)
"_vimrc" (用于 Win32) (*)
- 文件 "_vimrc" (用于 Unix、Amiga) (*)
".vimrc" (用于 Win32) (*)
- 文件 ".exrc" (用于 Unix、Amiga)
"_exrc" (用于 Win32)

(*) 如果使用该文件或环境变量，'compatible' 缺省被关闭。见 compatible-default 。

备注： mzscheme 界面在 vimrc 文件载入后初始化。此后再改变 'mzschemedll' 不会有效果。

4. 载入插件脚本。 load-plugins
相当于执行如下命令:
```
:runtime! plugin/**/*.vim
```
结果是，'runtimepath' 选项下的所有目录会搜索 "plugin" 子目录，而其中所有 ".vim" 结尾的文件会被执行 (每个目录下，按字母顺序依次执行)，包括所有子目录的子目录 (依此类推)。
不过，这里跳过 'runtimepath' 里以 "after" 结尾的目录，它们只在软件包之后载入，详情下述。
以下情况下，不载入插件:

- vimrc 文件里复位 'loadplugins' 选项。
- 使用 --noplugin 命令行参数。
- 使用 --clean 命令行参数。
- 使用 "-u NONE" 命令行参数 -u 。
- 编译 Vim 时不带 +eval 特性。

注意 使用 "-c set noloadplugins" 没有用，因为命令行的命令这时还没有执行。可以用 "--cmd 'set noloadplugins'" 或 "--cmd 'set loadplugins'" --cmd 。

载入软件包。如上所述找到插件，但在 'packpath' 的每个项目的 "start" 目录里寻找。每个找到的插件目录被加入 'runtimepath' 并执行找到的插件。见 packages 。

如上所述载入插件脚本，但现在只使用 "after" 结尾的目录。注意 如果找到软件包，'runtimepath' 会被改变，但不会加入 "after" 结尾的目录。

5. 设置 'shellpipe' 和 'shellredir'
   除非已经设置过，'shellpipe' 和 'shellredir' 选项根据 'shell' 选项的值取值。这意味着除非你自己已经选好了，Vim 会为你找到 'shellpipe' 和 'shellredir' 的值。

6. 如果使用 "-n" 命令行参数，设置 'updatecount' 为零

7. 置位 binary 选项
   如果给 Vim "-b" 标志，现在会置位二进制编辑的选项。见 -b 。

8. 进行 GUI 初始化
   只有启动 "gvim" 的时候，才会进行 GUI 的初始化。见 gui-init 。

9. 读入 viminfo 文件
   如果 'viminfo' 选项不为空，读入 viminfo 文件。见 viminfo-file 。

10. 读入 quickfix 文件
    如果给 Vim "-q" 标志，读入 quickfix 文件。如果失败，Vim 会退出。

11. 打开所有窗口
    如果给出 -o 标志，打开所有窗口（此时还没显示）。
    如果给出 -p 标志，创建所有标签页（此时还没显示）。
    屏幕的切换现在开始。启动重画。
    如果给出 "-q" 标志，跳到第一个错误处。
    所有窗口的缓冲区会被载入，不触发 BufAdd 自动命令。

12. 执行启动命令
    如果给 Vim "-t" 标志，跳转到标签处。
    -c 和 +cmd 参数给出的命令现在执行。
    如果置位了 'insertmode' 选项，进入插入模式。
    复位启动标志位，has("vim_starting") 现在返回零。
    v:vim_did_enter 变量被设为 1。
    执行 VimEnter 自动命令。

$MYVIMRC 或 $MYGVIMRC 文件（译者注：应为变量）会被设为首个找到的 vimrc 和/或 gvimrc 文件。

初始化过程的一些提示

标准安装:
创建 vimrc 文件，为你所有的编辑会话建立缺省的设置和映射。把它放到 3b 指出的地方:

```
~/.vimrc        (Unix)
s:.vimrc        (Amiga)
$VIM\_vimrc     (Win32)
```

注意 创建 vimrc 文件会使得 'compatible' 选项缺省被关闭。见 compatible-default 。

本地安装:
把所有在某个目录编辑所需要的命令放到 vimrc 文件里，并把它放在那个目录下并命名 ".vimrc" (MS-DOS 和 Win32 使用 "_vimrc")。注意: 要使得 Vim 查找这些特殊文件，你需要打开 'exrc' 选项。另见 trojan-horse 。

系统安装:
这只适用于你管理整个多个用户的 Unix 系统，而希望为所有用户设置缺省值的场合。建立建立缺省设置和映射的命令的 vimrc 文件，并把它放在 ":version" 命令指定的目录下。

保存当前 Vim 的状态到文件

无论你何时改变选项的值或者建立新映射，你都可能想保存它们到 vimrc 文件里以备后用。关于如何保存当前设置的状态到文件，见 save-settings 。

Vi 用户如何避免安装问题

Vi 使用 EXINIT 变量和 "~/.exrc" 文件。所以，如果你不想影响 Vi，使用 VIMINIT 变量和 "vimrc" 文件。

Amiga 环境变量

Amiga 上，有两种环境变量。使用 DOS 1.3 (或以后版本) setenv 命令设置的那些可以识别。见 AmigaDos 1.3 手册。使用旧的 Manx Set 命令设置的环境变量 (5.0 版本以前) 不能识别。

MS-DOS 行分隔符

在 MS-Windows 上，Vim 假设所有的 vimrc 文件使用 <CR> <NL> 分隔行。如果你使用只用 <NL> 的文件，而且有类似于 ":map xx yy^M" 的行，会有问题。因为结尾的 ^M 被忽略。

Vi 兼容缺省值

compatible-default

Vim 启动时，'compatible' 选项是打开的。Vim 开始初始化的时候使用该设置。不过一旦:
- 找到用户 vimrc 文件，或者

- 找到当前目录下的 vimrc 文件，或者
- 设置了 "VIMINIT" 环境变量，或者
- 给出 "-N" 命令行参数，或者
- 给出 "--clean" 命令行参数，或者
- 载入了 defaults.vim 脚本，或者
- 找到 gvimrc 文件，

就会把该选项设为 'nocompatible'。

注意 这并 不 适用于找到系统范围的 vimrc 文件。

这么做的副作用是会置位或复位其它选项（见 'compatible'），但只限于那些还没有设置或复位过的选项。这和启动 Vim 时采用 'compatible' 的新设置效果相同。

下述情况下 不 复位 'compatible' 也不载入 defaults.vim：
- Vim 启动时用 -u 命令行参数，尤其是 "-u NONE"，或者
- 启动时用 -C 命令行参数，或者
- 可执行文件的名字以 "ex" 结尾时（这使得 Vim 以 "ex" 的名字启动的时候，其行为和 "ex" 类同）

在找到 .vimrc 文件的这一刻置位或复位 'compatible' 有副作用：映射在遭遇的时候就被解释。因此，使用像 "<CR>" 这样的值就会受到此影响。如果映射依赖于使用 'compatible' 某个特定值，在给出映射前先置位或复位它。

没有 .vimrc 文件里的缺省值

defaults.vim

如果 Vim 正常启动且没找到用户 vimrc 文件，载入 $VIMRUNTIME/defaults.vim 脚本。这会关闭 'compatible'，打开语法高亮和一些更多的设置。详见该脚本。注意：这从 Vim 8.0 开始，不是 Vim 7.4。(准确地说，由 7.4.2111 补丁加入)。

这方便了新 Vim 用户。如果创建了自己 .vimrc，建议在顶部附近加入这些行：

```
unlet! skip_defaults_vim
source $VIMRUNTIME/defaults.vim
```

如果你有 .vimrc，这样 Vim 就像之前一样地工作了。一个方法是复制 $VIMRUNTIME/vimrc_example。此外，也可以复制 defaults.vim 到 .vimrc 并自行修改(但下次它更动时，你不会得到更新)。

如果你不喜欢部分缺省设置，仍然可以先执行 defaults.vim，然后回滚个别设置。参见 defaults.vim 文件中关于如何回滚每个项目的提示。

skip_defaults_vim

如果使用系统范围的 vimrc 但不想用 defaults.vim 来改变设置，置位 "skip_defaults_vim" 变量。如果它已经置位，而从 .vimrc 里想载入 defaults.vim，先 unlet skip_defaults_vim，如上例所示。

避免特洛伊木马

trojan-horse

读入当前目录下的 "vimrc" 或 "exrc" 文件时，为了安全原因，通过置位 'secure' 选项可以屏蔽某些命令。执行标签文件里的命令的时候，则总是如此。否则，你可能会不小心使用别人创建的 vimrc 或者标签文件，从而执行了可怕的命令。屏蔽的包括那些需要启动外壳的命令、写入文件的命令还有自动命令。此外会回显 ":map" 命令，从而看到那些键被映射了。

如果你想 Vim 执行本地 vimrc 文件的所有命令，可以使用 EXINIT 或者 VIMINIT 环境变量或者在全局 "exec" 或 "vimrc" 文件里屏蔽 'secure' 选项。显然，

不可以在当前目录下的 "vimrc" 或 "exrc" 里这么做。

在 Unix 系统上，只有你不是 vimrc 文件的拥有者的时候这才发生。警告：如果你解压归档文件而且其中含有 vimrc 或者 exrc 文件的时候，它们也会属于你，从而你不再有安全保护。启动 Vim 的时候检查当前目录 vimrc 文件，或者干脆复位 'exec' 选项。有的 Unix 系统允许用户给文件 "chown"（更换拥有者）。这使得别的用户可以创建有问题的 vimrc，并使你成为拥有者。要小心！

使用标签搜索命令的时候，搜索命令的执行（标签文件行里最后一部分）总是在安全模式下执行的。这和在当前目录执行 vimrc/exrc 的命令的情况完全类似。

如果 Vim 启动太慢

slow-start

如果 Vim 启动时间过长，用 --startuptime 参数可以知道为什么。可能有几个原因：

- 如果 Unix 版本编译时加入 GUI 和/或 X11（检查 ":version" 的输出是否含有 "+GUI" 和 "+X11"），它可能需要载入共享库并连接到 X11 服务器上。尝试编译一个不带有 GUI 和 X11 的版本。这也使得可执行文件小一些。
  使用 -X 命令行参数可以避免在终端运行时连接到 X 服务器上。
- 如果你打开 "viminfo"，viminfo 文件的载入可能需要一点时间。通过暂时屏蔽 viminfo，你可以判断这是否是个问题（使用 Vim 参数 "-i NONE"， -i ）。尝试减少寄存器可存贮的行数：":set viminfo='20,<50,s10"。 viminfo-file 。

:intro

如果 Vim 启动的时候没有给出文件名，它会显示一段介绍文字（为了帮助那些不知道 Vim 是什么的人）。一旦因为任何原因显示重画，它就被删除了。要再次看到这段文字，使用 ":intro" 命令（如果空间不够，你只会看到部分内容）。

要避免启动时的介绍文字，在 'shortmess' 里加上 'I' 标志位。

简介消息

info-message

--help 和 --version 参数使得 Vim 显示消息，并退出。通常，该消息发送到标准输出上。因此，它可以这样被重定向到文件里：

```
vim --help >file
```

在 Vim 里面：

```
:read !vim --help
```

使用 gvim 的时候，会检测到因为从桌面启动，无法使用终端来显示消息。这是通过检测到标准输出（stdout）和标准错误（stderr）不全是 tty 完成的。这也使得上述例子里的 ":read" 命令不能工作。要使其能够工作，把 'shellredir' 设为 ">" 而不是缺省的 ">&"：

```
:set shellredir=>
:read !gvim --help
```

不过，在 gvim 完全无法使用标准输出的系统中，这仍然无法工作。

---

5. $VIM 和 $VIMRUNTIME

$VIM

环境变量 "$VIM" 用于定位许多 Vim 使用的用户文件，例如用户启动脚本 ".vimrc"。不同的系统有所不同，见 startup 。

为了省得每个用户都设置 $VIM 环境变量，Vim 试图根据以下顺序取得 $VIM 的值：
1. $VIM 环境变量定义的值。你可以这样使得 Vim 在特定目录下找到它的支持文件。例如：
```
setenv VIM /home/paul/vim
```
2. 使用 'helpfile' 的路径，除非它本身也包含其它的环境变量（因为其缺省值是 "$VIMRUNTIME/doc/help.txt"：鸡生蛋的问题）。文件名（"help.txt" 或者别的）被去掉。结尾的几项目录名也被分别去掉。其顺序是："doc"、"runtime" 和 "vim{version}"（例如，"vim54"）。
3. 对 Win32 而言，Vim 试图使用可执行文件的目录名。如果以 "/src" 结尾，则去掉该部分。这用于以下场合：你把 .zip 文件解压到某个目录，然后调整搜索路径以找到其中的 vim 可执行文件。结尾的目录名被去掉。其顺序是："runtime" 和 "vim{version}"（例如，"vim54"）。
4. Unix 上，使用编译时定义的安装目录（见 ":version" 的输出结果）。

一旦 Vim 完成以上操作，它据此设置 $VIM 环境变量。要在其后修改，可以这样使用 ":let" 命令：
```
:let $VIM = "/home/paul/vim/"
```

$VIMRUNTIME

环境变量 "$VIMRUNTIME" 用于定位若干支持文件，比如在线的帮助文档和语法高亮文件。例如，主帮助文件正常情况下应该是 "$VIMRUNTIME/doc/help.txt"。
你通常不需要自己设置 $VIMRUNTIME，而应该让 Vim 自己去找。以下是查找 $VIMRUNTIME 的值所用的顺序：
1. 如果设置了环境变量 $VIMRUNTIME，则使用之。如果运行时文件在不常见的位置，可以在这里定义。
2. 如果 "$VIM/vim{version}" 存在，则使用之。{version} 是 Vim 的版本号，不包括任何的 '-' 或 '.'。例如："$VIM/vim54"。这是正常情况下 $VIMRUNTIME 的值。
3. 如果 "$VIM/runtime" 存在，使用之。
4. 使用 $VIM 的值。这和较老的版本后向兼容。
5. 如果设置了 'helpfile' 选项，并且它不包括 '$'，则使用它的值，去掉后面的 "doc/help.txt" 部分。

在 Unix 上，有一个 $VIMRUNTIME 编译时的缺省值（检查 ":version" 的输出），跳过步骤 2、3 和 4，而在第五步之后使用编译时的缺省值。这意味着编译时的缺省值否决了 $VIM 的值。这可以用于 $VIM 为 "/etc" 而运行时文件在 "/usr/share/vim/vim54" 的情况。

一旦 Vim 完成以下操作，它据此设置 $VIMRUNTIME 环境变量。要在其后修改之，这样使用 ":let" 命令：
```
:let $VIMRUNTIME = "/home/piet/vim/vim54"
```

如果你需要在外壳里得到 $VIMRUNTIME 的值（例如，在帮助文件里 grep 脚本），你可以这么用：
```
VIMRUNTIME=`vim -e -T dumb --cmd 'exe "set t_cm=\<C-M>"|echo $VIMRUNTIME|quit' | tr -d '\
```

不要设置 $VIMRUNTIME 为空值，有些东西会停止工作。

---

6. 暂停 suspend

iconize iconise CTRL-Z v_CTRL-Z

CTRL-Z 暂停 Vim，和 ":stop" 类似。

适用于普通和可视模式。在插入和命令行模式里，CTRL-Z 作为普通的字符插入。在可视模式下，Vim 回到普通模式。
注意：如果 CTRL-Z 撤销改变，见 mswin.vim 。

:sus[pend][!] 或 :st[op][!]

:sus :suspend :st :stop

暂停 Vim。
如果没有给出 '!' 并且置位 'autowrite'，那么每个修改过且有文件名的缓冲区都被写回。如果给出 '!' 或 'autowrite' 没有置位，修改过的缓冲区不会写回。
迟些时候，不要忘记把 Vim 带回前台！

在 GUI 上，暂停是通过使 Vim 图标化完成的。在 Windows 95/NT 上，gvim 被最小化。

在许多 Unix 系统中，可以使用 CTRL-Z 暂停 Vim。这只适用于普通和可视模式（见下一章， vim-modes ）。如果你把 Vim 放回前台，它会继续工作。在其它系统中，CTRL-Z 会启动新的外壳。这和 ":sh" 命令相同。在退出外壳后，Vim 会继续。

在 X-windows 里，Vim 暂停时不再拥有选择区。这意味着你不能在其它应用粘贴之（因为 Vim 进入睡眠，试图得到选择区会使得程序挂起）。

---

7. 退出 exiting

有几种方式可退出 Vim:
- 用 :quit 关闭最后一个窗口。仅当没有改动时有效。
- 用 :quit! 关闭最后一个窗口。即使有改动也可。
- 用 :qall 关闭所有窗口。仅当没有改动时有效。
- 用 :qall! 关闭所有窗口。即使有改动也可。
- 用 :cquit 。即使有改动也可。

使用 :cquit 或有错误信息时，Vim 退出时退出代码为 1。用 :silent! 或 :catch 可避免错误。

---

8. 保存设置 save-settings

多数情况下，你会手动编辑你的 vimrc 文件。这给你最大程度的灵活性。有一些命令可以自动产生 vimrc 文件。你可以直接使用生成的文件，也可以复制/粘贴其中的一些行到其它的 vimrc 文件里。

:mk :mkexrc

:mk[exrc] [file] 写入当前的键盘映射和修改过的选项到 [file] 里（缺省是当前目录下的 ".exrc"），除非它已经存在。

:mk[exrc]! [file] 总是写入当前的键盘映射和修改过的选项到 [file] 里（缺省是当前目录下的 ".exrc"）。

:mkv :mkvi :mkvimrc

:mkv[imrc][!] [file] 和 ":mkexrc" 类似，但是缺省值是当前目录下的 ".vimrc"。同时，也把 ":version" 写入文件里。

这些命令会把 ":map" 和 ":set" 命令写入到文件里，使得这些命令执行时，当前的键盘

映射和选项会设成和现在相同的值。选项 'columns'、'endofline'、'fileformat'、'key'、'lines'、'modified'、'scroll'、'term'、'textmode'、'ttyfast' 和 'ttymouse' 不含在内，因为它们和终端和文件有关。注意 选项 'binary'、'paste' 和 'readonly' 是包含在内的，有时这并不是你所希望的。

如果映射使用了特殊键，'cpoptions' 选项会暂时设为 Vim 的缺省值，以免映射被错误的解释。这使得该文件和 Vi 不兼容，但可以确保它能在不同的终端环境上使用。

只有全局映射被保存，局部于缓冲区的不会。

常见的方法是使用缺省的 ".vimrc" 文件，修改一些 ":map" 和 ":set" 命令，然后写回修改过的文件。先用命令读入缺省的 ".vimrc" 文件，像这样 ":source ~piet/.vimrc.Cprogs"，修改设置，然后保存到当前目录: ":mkvimrc!"。若果你想使之成为缺省的 .vimrc，把该文件移动到主目录 (Unix 上)，s: (Amiga) 或者 $VIM 目录 (MS-DOS)。你也可以使用自动命令 autocommand 和/或模式行 modeline 。

vimrc-option-example

如果你只想增加一个选项设置到 vimrc 里，可以使用这些步骤:

1. 用 Vim 编辑你的 vimrc 文件。
2. 测试选项值直到正确为止。比如，试试 'guifont' 不同的值。
3. 附加一行设置选项的值。使用表达式寄存器 '=' 来输入值。比如，对 'guifont' 选项:

   o:set guifont=<C-R>=&guifont<CR><Esc>

   [<C-R> 是 CTRL-R，<CR> 是回车，<Esc> 是 escape 键]

   你需要转义特殊字符，特别是空格。

注意 你建立 .vimrc 文件本身会影响 'compatible' 选项，这有若干副作用。见 'compatible' 。":mkvimrc"、":mkexrc" 和 ":mksession" 首先写入置位或者复位 'compatible' 选项的命令到输出文件里，就是因为有这些副作用的存在。

---

9. 视图和会话

views-sessions

在用户手册的 21.4 和 21.5 两节已有相关介绍。

View view-file

视图是应用于一个窗口的设置的集合。你可以保存视图，并在以后恢复之，从而使文本以同样的方式显示。本窗口的选项和映射也会同时恢复，这样你可以从上次保存的视图开始继续编辑。

Session session-file

会话保存所有窗口的视图，外加全局设置。你可以保存会话，并在以后恢复之，使得窗口的布局看起来一样。你可以创建会话在不同的项目之间快速切换，切换时自动调入你上次在那个项目上工作的文件。

视图和会话是 viminfo 文件的一个很好的补充，后者用来记住所有视图和会话都使用的信息 viminfo-file 。

通过 -S 参数，你可以使用以前保存的视图或者会话快速启动编辑:

```
vim -S Session.vim
```

{仅当编译时加入 +mksession 特性才有效}。

:mks :mksession

:mks[ession][!] [file] 写入一个 Vim 脚本，该脚本能够恢复当前的编辑会话。
如果包含 [!]，已有的文件会被覆盖。如果不提供 [file]，
使用 "Session.vim"。

":mksession" 的输出结果类似于 ":mkvimrc"，但文件里附加更多的命令。这取决于
'sessionoptions' 选项的值。产生的文件在用 ":source" 命令执行时：

1. 如果 'sessionoptions' 包含 "options" 的话，恢复全局映射和选项。局部于脚本的映射不会被写入。
2. 如果 'sessionoptions' 包含 "globals" 的话，恢复大写字母开始并至少包含一个小写字母的全局变量。
3. 卸载当前所有载入的缓冲区。
4. 如果 'sessionoptions' 包含 "curdir" 的话，恢复当前目录。或者如果 'sessionoptions' 包含 "sesdir" 的话，设置当前目录为会话文件所在的位置。
5. 如果 'sessionoptions' 包含 "winpos" 的话，恢复 GUI Vim 的窗口位置。
6. 如果 'sessionoptions' 包含 "resize" 的话，恢复屏幕大小。
7. 恢复缓冲区列表，还有它们最后的光标位置。如果 'sessionoptions' 包含 "buffers" 的话，恢复所有的缓冲区，包括隐藏和未载入的。不然，只有窗口里打开的缓冲区才被恢复。
8. 恢复所有窗口并使用和过去相同的布局。如果 'sessionoptions' 包含 "help" 的话，帮助窗口也被恢复。如果 'sessionoptions' 包含 "blank" 的话，编辑无名缓冲区的窗口也被恢复。如果 'sessionoptions' 包含 "winsize" 并且没有任何（帮助/空白）窗口被放弃的话，则恢复窗口大小（相对于屏幕大小）。否则，只为窗口给定合适的大小。
9. 如同 :mkview 那样，恢复各个窗口的所有视图。但使用 'sessionoptions' 而不是 'viewoptions'。
10. 如果存在和会话文件同名，但以 "x.vim"（x 代表 "附加" eXtra）结尾的文件，也同时执行之。你可以用 *x.vim 文件来指定和给出会话相关的附加设置和动作，比如在 GUI 版本里创立菜单项目。

在恢复会话后，当前会话的完整文件名可以通过内部变量 "v:this_session" 得到
this_session-variable 。映射示例：

```
:nmap <F2> :wa<Bar>exe "mksession! " . v:this_session<CR>:so ~/sessions/
```

这会保存当前会话，并启动载入另外一个会话的命令。

会话包含所有标签页，除非从 'sessionoptions' 里删除了 'tabpages'。 tab-page

SessionLoadPost 自动命令事件在会话文件载入/执行后被激活。

SessionLoad-variable

会话文件载入期间，SessionLoad 全局变量被设为 1。用此变量，插件可以推迟部分工作，直到激活 SessionLoadPost 事件时才执行。

:mkvie :mkview

:mkvie[w][!] [file] 写入一个 Vim 脚本，该脚本能够恢复当前的窗口内容。
如果包含 [!]，已有的文件会被覆盖。如果不提供 [file] 或者它是从 1 到 9 的数字，则生成一个名字并在前面加上 'viewdir' 的值。如果 'viewdir' 的最后一个路径部分不存在，则顺便建立该目录。例如 'viewdir' 为 "$VIM/vimfiles/view" 时，在 "$VIM/vimfiles" 下建立 "view"。
如果该文件已存在，这时它总会被覆盖。使用 :loadview 来重新载入该视图。
如果 [file] 是文件名（不考虑 'viewdir'），则编辑该文件

的命令加入到生成的文件里。

":mkview" 的输出包含如下项目:
1. 该窗口使用的参数列表。如果使用全局参数列表，它被重设为全局列表的值。参数列表的索引位置也被保存。
2. 窗口编辑的文件。如果没有文件，该窗口被清空。
3. 如果 'viewoptions' 包含 "options" 或者 "localoptions" 的话，恢复映射、缩写和局部于窗口的选项。一般，只有局部于当前缓冲区和窗口的选项值被恢复。如果视图作为会话的一部分被保存，并且 'sessionoptions' 包含 "options" 的话，局部选项的全局值也被保存。
4. 如果使用手动折叠并且 'viewoptions' 包含 "folds" 的话，恢复折叠。也恢复手动打开和关闭的折叠。
5. 滚动条的位置和文件中的光标位置。如果有关闭的折叠，这不能很好的工作。
6. 本地的当前目录，如果和全局当前目录不同且 'viewoptions' 包含 "curdir" 的话。

注意 视图和会话并不完善:
- 它们并不会恢复所有的信息。比如，定义的函数、自动命令和 ":syntax on" 并不包括在内。包括寄存器内容、命令行历史包含在 viminfo 里，而不是会话或者视图里。
- 全局选项值只有在和缺省值不同时才会被设置。如果当前值不同于缺省值，载入会话不会把它设为缺省值。不过，局部选项会设为它们的缺省值。
- 已有的映射没有预警的被覆盖。其它已有的映射可能会因为这些二义性引起错误。
- 保存手动折叠和手动打开/关闭的折叠时，保存和载入时的文件内容差异会引起折叠恢复的混乱。
- Vim 脚本效率不高。不过比你自己输入这些命令总会快点。

:lo :loadview

:lo[adview] [nr] 为当前文件载入视图。如果没有给出 [nr]，":mkview" 保存的视图被恢复。如果指定 [nr]，":mkview [nr]" 保存的视图被恢复。

":mkview" 和 ":loadview" 的组合可以用来保存同一文件不超过十个的不同视图。它们被保存于 'viewdir' 选项指定的目录下。使用该文件名保存视图。如果文件被换名或者通过 (符号) 链接访问，就找不回你的视图了。

时不时，你也许要想要清除 'viewdir' 目录的内容。

要自动保存和恢复 *.c 文件的视图:

```
au BufWinLeave *.c mkview
au BufWinEnter *.c silent loadview
```

---

10. viminfo 文件

viminfo viminfo-file E136 E575 E576 E577

如果你退出 Vim 以后又再次启动，通常你会丢失许多信息。viminfo 文件可以用来记住这些信息，从而允许你继续上次退出的编辑。

用户手册 21.3 一节有所介绍。

viminfo 文件用来保存:
- 命令行历史。
- 搜索字符串历史。
- 输入行历史。
- 非空的寄存器内容。
- 多个文件的位置标记。

- 文件标记，指向文件的位置。
- 最近搜索/替换的模式（适用于 'n' 和 '&'）。
- 缓冲区列表。
- 全局变量。

如果 +viminfo 特性在编译时关闭，不支持 viminfo 文件。

与此同时，你可以使用会话文件。差别是，viminfo 文件并不依赖于你工作的内容。通常，只有一个 viminfo 文件。而会话文件用来保存某个编辑会话的状态。你可以保存多个会话文件，每个针对一个你工作的项目。viminfo 和会话文件一起可以用来有效的进入 Vim 并直接从你希望的设置开始工作。 session-file

viminfo-read
如果 Vim 启动时 'viminfo' 选项非空，viminfo 文件的内容被读入，而其中的信息在适当的地方被应用。填入 v:oldfiles 变量。启动时并不读入位置标记（但是文件标记被读入）。关于如何在启动时设置 'viminfo' 选项，见 initialization 。

viminfo-write
如果 Vim 退出时 'viminfo' 非空，相关信息被保存在 viminfo 文件里（实际上，如果该文件存在的话，和已有的内容合并）。'viminfo' 选项是一个定义需要保存什么信息的字符串，也包含这些信息需要保存多少的数量限制（见 'viminfo'）。

合并以两种方式发生。多数项目保存当前 Vim 会话中被改变或设置的值，没有变动的则从原先的 viminfo 文件里的值填充。例如：
- Vim 会话 A 读入 viminfo，其中包含变量 START。
- Vim 会话 B 做同样的事
- Vim 会话 A 设置变量 AAA 和 BOTH 然后退出
- Vim 会话 B 设置变量 BBB 和 BOTH 然后退出

现在 viminfo 将会包含：
```
START - 原先在 viminfo 里，会话 A 或 B 都未改变
AAA   - 来自会话 A，会话 B 保留
BBB   - 来自会话 B
BOTH  - 来自会话 B，会话 A 的值丢失
```

viminfo-timestamp
有些项目使用时间戳来保留最近改动的版本。这和 Vim 会话退出的顺序无关，总是保存最新的项目（可有多项）。用于：
- 命令行历史。
- 搜索字符串历史。
- 输入行历史。
- 非空寄存器内容。
- 跳转表
- 文件标记

时间戳功能在 Vim 8.0 之前被加入。从 7.4.1131 开始的稍老的 Vim 版本，保存项目时记录时间戳，但不使用。因此使用旧版本和新版本的 Vim 时都会保存最近的数据。

Unix 上的 注意 事项：
- viminfo 的文件保护会被设置，使得其它用户不能够读取。因为它可能包含任何文本或者工作所需的命令。
- 如果你需要和其它用户共享 viminfo 文件（例如，当你 "su" 到另外一个用户的时候），你可以使该文件对同组甚至所有人可写。Vim 在替换 viminfo 文件的时候会保留该文件的权限。要小心，不要不加考虑的允许任何人读写你的 viminfo 文件！
- Vim 不会覆盖当前 "实际" 用户不能写入的 viminfo 文件。这对使用 "su" 成为 root 但是 $HOME 仍然指向一个普通用户的主目录的用户有用。不然 Vim 会建立一个 root

拥有的 viminfo 文件，使得别人都无法读取。
- viminfo 文件不能是符号链接。为了避免安全问题。

可以为每个文件分别存贮位置标记。文件读入时，如果 'viminfo' 非空，该文件的位置标记会从 viminfo 文件里读取。注意：位置标记只有在退出 Vim 的时候才会保存。因为对所有你在当前编辑会话打开的文件都保存位置标记，这应该没有问题。除非使用过 ":bdel"。如果你想保存准备用 ":bdel" 放弃的文件的位置标记，使用 ":wv"。
'[' 和 ']' 位置标记不会保存，但 '"' 位置标记会。'"' 位置标记用于跳转到文件最近退出时的光标位置。用 'viminfo' 的 "r" 标志位指定的字符串开始的任何文件不会保存其位置标记。这可以用来防止为可移动媒介上的文件保存位置标记 (MS-DOS 上你可以用 "ra:,rb:"，Amiga 上 "rdf0:,rdf1:,rdf2:")。
v:oldfiles 变量填入 viminfo 文件中有位置标记的那些文件名。

viminfo-file-marks

大写位置标记 ('A 到 'Z) 在写入 viminfo 文件时被保存。数字标记 ('0 到 '9) 则有点特殊。在写入 viminfo 文件时 (退出或者使用 ":wviminfo" 命令)，'0 设置为当前光标的位置和所在的文件。旧的 '0 被移到 '1，'1 到 '2，等等。这和 "1 到 "9 删除用的寄存器的处理类似。如果当前光标位置已经在 '0 到 '9 里，它被移动到 '0，以免保存相同的位置两次。结果是，使用 "'0" 可以跳转回你退出 Vim 时所在的文件和行号。要立刻执行之，试用以下命令：

```
vim -c "normal '0"
```

在 csh 兼容的外壳里，你可为此建立一个别名：

```
alias lvim vim -c '"'normal "'"0'"'
```

在 bash 兼容的外壳里：

```
alias lvim='vim -c "normal '\''0"'
```

'viminfo' 里可以使用 "r" 标志位来指定什么文件不需要保存位置标记。

viminfo 文件名

viminfo-file-name

- 在 Unix 上，viminfo 文件的缺省文件名是 "$HOME/.viminfo"。Amiga 是 "s:.viminfo"，Win32 则是 "$HOME_viminfo"。对于后两者，如果没有设置 $HOME，使用 "$VIM_viminfo"。如果 $VIM 也没有设置，使用 "c:_viminfo"。
- 'viminfo' 选项里的 'n' 标志位可以用来指定另外一个 viminfo 文件名 'viminfo'。
- Vim 的 "-i" 参数可以用来设置另外一个文件名 -i。如果给出的文件名是 "NONE" (全是大写)，不读写任何 viminfo 文件。下面给出的命令也不会！
- 'viminfofile' 选项和 "-i" 参数用法相同。事实上，"-i" 参数的值保存在 'viminfofile' 选项里。
- 下面给出的命令里，可以给定另外一个文件名。它覆盖缺省文件名和 'viminfo' 或者 "-i" 给定的文件名 (除非它是 NONE)。

字符编码

viminfo-encoding

viminfo 文件的文本使用 'encoding' 选项指定的编码。通常你总是使用相同的 'encoding' 值，这没有问题。不过，如果使用不同于写入时使用的 'encoding' 的值读取 viminfo 文件，有些文本 (非 ASCII 字符) 可能不正确。如果不能接受，你可以在

'viminfo' 选项里加上 'c' 标志位:

```
:set viminfo+=c
```

Vim 会试图从 viminfo 文件写入时所使用的 'encoding' 值转换到当前的 'encoding' 值。这需要 Vim 编译时带上 +iconv 特性。文件名不会被转换。

手 动 读 写 viminfo-read-write

两个命令可以用来手动读写 viminfo 文件。这可以用来在两个运行中的 Vim 程序之间交换寄存器的值: 先在其中一个输入 ":wv", 再在另外一个用 ":rv"。注意 如果寄存器里已经有内容, 那么需要 ":rv!" 才行。也要 注意 这意味着所有的内容都被第一个 Vim 实例的信息所覆盖, 包括命令行历史等。

viminfo 文件本身也可以手动修改, 不过我们建议你从已有的 viminfo 开始, 保证格式没有错误。一旦你熟悉了, 该文件格式很容易理解。这可以用来建立另一个 viminfo 文件, 比如说 "~/.my_viminfo", 包含每次启动 Vim 你想要的设置。比如, 你可以用某些数据预先载入寄存器, 或者把特定的命令放在命令行历史里。.vimrc 文件这样的一行

```
:rviminfo! ~/.my_viminfo
```

可以用来载入这些信息。你甚至可以为不同的文件类型应用不同的 viminfo (比如 C 代码) 上。通过使用 ":autocmd" 命令 (见 :autocmd) 就可以根据文件名自动载入。

viminfo-errors

一旦 Vim 在读入 viminfo 文件时检测到错误, 以后就不会覆盖那个文件。如果有超过 10 个错误, Vim 停止该 viminfo 文件的读取。这可以防止在给错 viminfo 文件名字的情况下, 不小心覆盖该文件。比如, 这可能发生于以下场合: 输入 "vim -i file" 但实际想输入 "vim -R file" (是的, 有人不小心这么干过)。如果你仍然想覆盖有错误的 viminfo 文件, 或者修正这个错误, 或者先删除这个文件 (在 Vim 运行的时候。这样大部分的信息仍然可以恢复)。

:rv :rviminfo E195

:rv[iminfo][!] [file] 读入 viminfo 文件 [file] (缺省: 见上)。
如果给出 [!], 那么任何已经给出的信息 (寄存器、位置标记、 v:oldfiles , 等等) 都会被覆盖。

:wv :wviminfo E137 E138 E574 E886 E929

:wv[iminfo][!] [file] 写到 viminfo 文件 [file] (缺省: 见上)。
该文件的信息先被读入, 以合并原有和新的信息。如果使用 [!], 旧的信息不会先读取, 而是只写入系统自己内部的信息。如果 'viminfo' 为空, 会写入不超过 100 个文件的位置标记。
如果你得到错误 "E929: Too many viminfo temp files", 检查
有没有旧的临时文件没删除 (比如 ~/.viminf*) 还有你是否可以写入 .viminfo 文件所在的目录。

:ol :oldfiles

:ol[dfiles] 列出 viminfo 文件中存储位置标记的文件。该列表在启动时读入, 其后只有 :rviminfo! 能改变它。另见 v:oldfiles 。
其中的编号可用于 c_#< 。
输出结果可用 :filter 过滤, 例如:

```
filter /\.vim/ oldfiles
```

过滤作用于文件名。

{仅当编译时加入 +eval 特性才有效}

:bro[wse] ol[dfiles][!]
像 :oldfiles 那样列出文件名，然后提示要求输入编号。
如果编号合法，编辑列表里指定的文件。
如果得到 press-enter 提示时你可以按 "q" 键并且还能得到输入文件编号的提示。
! 用来放弃已修改的缓冲区。 abandon
{仅当编译时使用 tiny 或 small 特性才有效}

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册

by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

编辑文件 edit-files

1. 简介 edit-intro
2. 编辑单个文件 edit-a-file
3. 参数列表 argument-list
4. 写入 writing
5. 写入并退出 write-quit
6. 对话框 edit-dialogs
7. 当前目录 current-directory
8. 编辑二进制文件 edit-binary
9. 加密 encryption
10. 修改时间 timestamps
11. 文件搜索 file-searching

---

1. 简介 edit-intro

用 Vim 编辑一个文件意味着：

1. 把该文件读到缓冲区
2. 用编辑器命令修改缓冲区
3. 把缓冲区内容写回文件

current-file
只要你不保存缓冲区，原来的文件保持不变。如果你开始编辑一个文件（把它的内容读到缓冲区里），该文件名被认作 "当前文件名"，或者说当前缓冲区的名字。命令行上，可以用 "%" 来引用它 :_% 。

alternate-file
如果已经有了另外一个当前文件名，它会成为轮换文件名。命令行上，可以用 "#" 引用它 :_# 。你还可以用 CTRL-^ 命令在当前和轮换文件之间切换。不过，轮换文件名在使用 :keepalt 期间不予改变。
每个窗口分别记住自己的轮换文件名。

:keepalt :keepa
:keepalt {cmd} 执行 {cmd}，在此期间保持当前轮换文件名不变。注意 间接调用的命令（比如通过函数）仍然可以设置轮换文件名。

所有文件名都会被记录在缓冲区列表里。在你输入文件名以便编辑（例如，用 ":e 文件名"）或者写回（例如，用 ":w 文件名"）时，此文件名被加入该列表。你可以用缓冲区列表来记住编辑过什么文件并用 CTRL-^ 命令快速的从一个文件切换到另一个（例如，以便复制文本）。先敲入文件的编号再敲入 CTRL-^。

CTRL-G 或 CTRL-G :f :fi :file
:f[ile] 显示当前文件名（如同输入的那样，除非使用过 ":cd"）、光

标位置（除非置位了 'ruler' 选项）和文件状态（只读、已修改、读入错误、新文件）。参见 'shortmess' 选项，了解如何可以简化该消息。

:f[ile]!　和 :file 类似，但即使 'shortmess' 要求截短文件名，也不这么做。

{count}CTRL-G　和 CTRL-G 类似，但显示当前文件名的完整路径。如果计数大于 1，则同时给出当前缓冲区号。

g_CTRL-G　word-count　byte-count

g CTRL-G　显示当前光标位置，以五种方式表达：列、行、单词、字符和字节计数。如果字符数和字节数相同，省略字符位置。
如果当前行有字符占据多于一个位置（<Tab> 或其他特殊字符），"实际" 列和屏幕列同时显示，以连字号分隔。参见 'ruler' 选项和 wordcount() 函数。

v_g_CTRL-G

{Visual}g CTRL-G　和 "g CTRL-G" 类似，但显示可视选择区域的单词、字符、行和字节计数。在面向列块模式里，列数也同时显示。(关于 {Visual} 见 Visual-mode 。)

:file_f

:f[ile][!] {name}　设置当前文件名为 {name}。可选的 ! 避免截短消息，就像 :file 那样。
如果缓冲区已有名字，该名字成为 alternate-file 轮换文件名。新建一个列表外缓冲区来记住这个旧名字。

:0file

:0f[ile][!]　删除当前缓冲区的名字。可选的 ! 避免截短消息，就像 :file 那样。

:buffers
:files
:ls　列出当前已知的文件名。见 'windows.txt' :files :buffers :ls 。

vim 会记住你输入文件名的完整路径。大多数情况下，文件名只以你输入的方式显示。但 ":cd" 命令 :cd 使用后，就会看到完整路径名。

home-replace

如果设置了环境变量 $HOME，而文件名以该字符串开始，它的显示会把 HOME 替换成 "~"。这么做是为了使文件名的显示较为简洁。在读写文件时仍然使用完整路径名，"~" 只供显示文件名用。如果要替换名字实质上只有 "~" 的文件，会使用 "~/" (为了避免设为 $HOME 的选项和设为 '~' 时的 'backupext' 混淆)。

保存缓冲区时，缺省使用当前文件名。这样，如果你给出 "ZZ" 或者 ":wq" 命令，原有的文件被覆盖。如果你不想如此，给 ":write" 命令给出一个文件名参数，你可以把缓冲区写到另外一个文件里。例如：

```
vim testfile
[用编辑器命令修改缓冲区]
:w newfile
:q
```

这会建立文件 "newfile"，它是 "testfile" 经过修改的版本。"testfile" 文件保持不变。无论如何，如果置位 'backup' 选项，Vim 会在原来的文件被覆盖之前改名或复制。如果你以后发现你需要原来的版本，你可以用这个备份文件。也可参见 'patchmode' 选项。备份文件名通常是原来的文件加上 'backupext' 后缀。缺省的 "~" 比较奇特，目的是为了防止意外覆盖现存的文件。如果你喜欢 ".bak"，修改 'backupext' 选项。在 MS-DOS 机器上，当 Vim 检测到正在使用的是 MS-DOS 或类似的文件系统（例如，messydos 或者 crossdos）或者打开了 'shortname' 选项，多余的点号被替换成 '_'。通过设置 'backupdir'，备份文件也可存在别的目录下。

auto-shortname

技术上：在 Amiga 上文件名可以长达 30 个字符。但在 MS-DOS 兼容的文件系统上只能用 8+3 个字符。Vim 试图在创建 .swp 文件的时候检测文件系统的类型。如果怀疑是一个类似 MS-DOS 的文件系统，则置位一个标志位，它和设置 'shortname' 选项有相同的效果。一旦你开始编辑一个新文件，该标志位被复位。在根据当前文件名创建 ".swp" 和 ".~" 文件时，该标志位起作用。但如果你在一个正常的文件系统上编辑文件，而写到一个类似于 MS-DOS 的文件系统，该标志位不会被置位。这种情况下，创建 ".~" 文件可能会失败，你会得到一个错误消息。这种情况下，用 'shortname' 选项。

当你开始编辑而不给出文件名，会给出 "No File" 消息。如果 ":write" 命令带一个文件名参数，当前文件名被设为那个名字。这只有在 'cpoptions' 里带 'F' 标志位（缺省的确是如此）时才会发生 cpo-F 。这对在空缓冲区里编辑文本后写入到文件有用。如果 'cpoptions' 里包含 'f' 标志位（缺省 不 包含） cpo-f ，在执行 ":read file" 时，文件名被设置。这可用来先不带参数启动 Vim 再用 ":read file" 编辑文件。
如果设置了文件名而 'filetype' 为空，激活文件类型检测的自动事件。

not-edited

因为设置文件名并不是真正编辑该文件，你不会覆盖它。这可由设置 "notedited" 标志位来完成。你可由 CTRL-O 或 ":file" 命令看到该标志位是否设置。如果 "notedited" 标志位已设置，它会包括 "[Not edited]"。当将缓冲区写回当前文件名（用 ":w!") 时，"notedited" 标志位被重设。

abandon

Vim 记住你是否修改了缓冲区。你所作的修改不会不小心丢失。如果试图不写回而退出，或开始编辑别的文件，Vim 会拒绝。要绕过这重保护，在命令后加上 '!'。这样，修改就会丢失。例如：":q" 在缓冲区修改后不会工作，但 ":q!" 会。要知道缓冲区是否修改，用 "CTRL-G" 命令。如果缓冲区被修改，消息里会有字符串 "[Modified]" 或者 "+"，后者如果 'shortmess' 有 'm' 标志位。

如果你想不经提示而自动写回改动，打开 'autowriteall' 选项。'autowrite' 是相关的 Vi-兼容选项，它并非对所有命令有效。

如果想保留修改了的缓冲区但不写回，打开 'hidden' 选项。参见 hidden-buffer 。有些命令即便 'hidden' 没有置位也会如此，具体须参见各个命令的帮助。

---

2. 编辑单个文件

edit-a-file

:e :edit reload

:e[dit] [++opt] [+cmd] 编辑当前文件。可用于在它被 Vim 之外的程序所改变的时候重新编辑当前文件。 如果当前缓冲区已经被修改而且没有置位 'autowriteall'，或者文件不能被写入，本命令失败。另见 ++opt 和 +cmd 。

:edit! discard

:e[dit]! [++opt] [+cmd]
总是编辑当前文件。放弃当前缓冲区任何已有的改变。如果你想从头来过，这就会很有用。
另见 ++opt 和 +cmd 。

:edit_f
:e[dit] [++opt] [+cmd] {file}
编辑文件 {file}。
如果当前缓冲区已被修改，这会失败，除非置位了 'hidden' 或者 'autowriteall' 并且文件能够写入。
另见 ++opt 和 +cmd 。

:edit!_f
:e[dit]! [++opt] [+cmd] {file}
总是编辑文件 {file}。放弃当前缓冲区任何已有的改变。
另见 ++opt 和 +cmd 。

:e[dit] [++opt] [+cmd] #[count]
编辑第 [count] 个缓冲区（根据 :files 显示的结果）。
该命令和 [count] CTRL-^ 一样，但是 ":e #" 在轮换缓冲区没有文件名时不能工作，而 CTRL-^ 则可以。
另见 ++opt 和 +cmd 。

:ene :enew
:ene[w]
编辑一个新的无名缓冲区。如果当前缓冲区已被修改，这会失败，除非置位了 'hidden' 或者 'autowriteall' 并且文件能够写入。
如果 'fileformats' 非空，其给出的第一个格式将用于新的缓冲区上。如果 'fileformats' 为空，则使用当前缓冲区的 'fileformat'。

:ene! :enew!
:ene[w]!
编辑新的无名缓冲区。放弃当前缓冲区任何已有的改变。
'fileformat' 的设置类似于 :enew 。

:fin :find
:fin[d][!] [++opt] [+cmd] {file}
在 'path' 里找到 {file}，然后编辑 :edit 它。
{仅当编译时加入 +file_in_path 特性才有效}

:{count}fin[d][!] [++opt] [+cmd] {file}
和 ":find" 类似，但使用 'path' 的第 {count} 个匹配。这样，":2find file" 会找到 'path' 里的第二个 "file"。如果 'path' 里文件的匹配不足所需之数，你会得到一个错误消息。

:ex
:ex [++opt] [+cmd] [file]
和 :edit 相同。

:vi :visual
:vi[sual][!] [++opt] [+cmd] [file]
用于 Ex 模式时：退出 Ex-mode ，回到普通模式。否则和 :edit 相同。

:vie :view

:vie[w][!] [++opt] [+cmd] file
用于 Ex 模式时：退出 Ex-mode，回到普通模式。否则和 :edit 相同，但为本缓冲区置位 'readonly' 选项。

CTRL-^ CTRL-6

CTRL-^
编辑轮换文件。
多数情况下，轮换文件就是上次编辑的文件。这是切换两个文件的一个快速方法。这和 ":e #" 等价，但也能用在没有文件名的情况。

如果 'autowrite' 或 'autowriteall' 选项打开并且缓冲区被修改，则写入文件。
大多数情况下，^ 字符在键 6 之上，同时按 CTRL 和 6 会给出我们叫 CTRL-^ 的键码。但在有的非-US 的键盘上 CTRL-^ 须以不同的方式生成。

[count]CTRL-^
编辑缓冲区列表里第 [count] 个文件（和 ":e #[count]" 等价）。这是切换文件的一个快速方法。
详情见上面 CTRL-^。

[count]]f]f [f
[count][f
和 "gf" 一样。它本身已被淘汰。

gf E446 E447

[count]gf
编辑光标之上或之后的文件名。助记："goto file"。
使用 'isfname' 选项来决定组成文件名的字符。拖尾的标点符号 ".,:;!" 被忽略。空格的转义 "\ " 缩为一个空格。
在 'path' 选项组成的目录名列表查找文件。关于相对路径和通配符，可见 'path' 选项。
设置 'suffixesadd' 选项可以用来查找文件所要附加的后缀。如果文件找不到，用 'includeexpr' 来修改名字，然后再试一次。
如果给出 [count]，'path' 里找到的第 count 个文件被编辑。
如果 Vim 拒绝放弃 abandon 当前文件，该命令失败。
如果你想在新窗口里编辑文件，用 CTRL-W_CTRL-F。
如果你想编辑新文件，用：

```
:e <cfile>
```

想使得 gf 总是如此工作，可以如此：

```
:map gf :e <cfile><CR>
```

如果名字是一个超文本链接，也就是看来像 "type://machine/path" 这样的，你需要 netrw 插件。
对 Unix 而言，'~' 字符被扩展，比如在 "~user/file" 的场合。环境变量也得到扩展 expand-env。
{仅在编译时加入 +file_in_path 特性才有效}

v_gf
{Visual}[count]gf
和 "gf" 相同，但是高亮文本被用来作为要编辑的文件名。'isfname' 被忽略。开头的空白被忽略，除此以外，所有的空白和特殊字符被包括在文件名里。
({Visual} 可参见 Visual-mode。)

gF

[count]gF　　和 "gf" 相同，但如果文件名后面跟随着数值，那么光标将定位在该文件名的该行上。文件名和数值之间必须以一个非文件名（见 'isfname'）和非数字字符分隔。文件名、分隔符和数值之间的空白被忽略。
例如：

```
eval.c:10
eval.c @ 20
eval.c (30)
eval.c 40
```

v_gF

{Visual}[count]gF　　和 "v_gf" 相同。

这些命令用来开始编辑单独一个文件。这意味着该文件被读入缓冲区里并设置当前文件名。打开哪个文件和当前目录有关，见 :cd 。

关于文件读入以后给出的消息，参见 read-messages 。

如果你把缓冲区内容弄乱了想从头开始编辑，你可以用 ":e!" 命令。":e" 命令只有在你改变当前文件名的时候有用。

:filename {file}

除了这里说明的事项以外， cmdline-special 也提到更多特殊项目，它们在期待文件名时可用。

注意 除了 Unix 以外的系统：在使用接受单个文件名的名字时（如 ":edit file"），文件名里可以有空格，但是拖尾的空格被忽略。这可用于允许文件名有内含的空格的系统（如 MS-Windows 和 Amiga）。示例：命令 ":e Long File Name " 会编辑文件 "Long File Name"。在使用能接受多于一个文件名的命令时（如 ":next file1 file2"），内含的空格必须以反斜杠转义。

wildcard wildcards

{file} 里的通配符被扩展，但和文件补全一样，也适用 'wildignore' 和 'suffixes'。具体支持什么通配符由系统决定。以下是一些通用的字符：

```
?       匹配一个字符
*       匹配任何东西，包括什么都没有
**      匹配任何东西，包括什么都没有，递归进入目录
[abc]   匹配 'a'、'b' 或 'c'
```

要避免通配符的特殊含义，在前面加上反斜杠。不过，在 MS-Windows 上，反斜杠是路径分隔符，"path\[abc]" 在 'isfname' 选项里有 "[" 的时候还是被看作通配符。一个简单避免该问题的方法是用 "path\[[]abc]"。这样这匹配文件名 "path[abc]"。

starstar-wildcard

在 Unix、Win32、Mac OS/X 和一些其它系统上，可以扩展 "**"。它允许你在目录树里搜索。最大深度可以到 100 层目录。
注意 有些命令的工作方式略有不同。见 file-searching 。
例如：

```
:n **/*.txt
```

找到文件：

```
aaa.txt
subdir/bbb.txt
a/b/c/d/ccc.txt
```

紧接着 "**" 之前或之后使用的非通配字符，只匹配第一层目录，而不用于匹配树结构其下的目录。例如：

```
:n /usr/inc**/types.h
```

找到文件：

```
/usr/include/types.h
/usr/include/sys/types.h
/usr/inc/old/types.h
```

注意 之所以带 "/sys" 的路径，是因为那一层的名字不需要匹配 "/inc"。也就是说，这相当于匹配 "/usr/inc*/*/*..."，而不是 "/usr/inc*/inc*/inc*"。

backtick-expansion `-expansion

在 Unix 和一些其它系统上你可以用反引号代替文件名参数。例如：

```
:next `find . -name ver\\*.c -print`
:view `ls -t *.patch  \| head -n1`
```

Vim 会用 'shell' 来执行反引号内的命令，其标准输出作为给定 Vim 命令的参数（但外壳命令的错误信息会被丢弃）。
要看到 Vim 执行的外壳命令，设置 'verbose' 选项为 4。如果外壳命令返回非零退出码，会显示错误信息并异常中止 Vim 命令。如果不想如此，须保证外壳命令永远返回零值。可以这样：

```
:next `find . -name ver\\*.c -print \|\| true`
```

星号之前需要有反斜杠，以防 "ver*.c" 在执行 find 程序之前被外壳先扩展。在外壳管道符号 "|" 之前的反斜杠使 Vim 不会把它解析为命令的终止。
该功能对于多数其他系统也使用，但要求反引号必须包围整个项目。你不能在第一个之前和最后一个反引号之后有别的文本。

`=

在第一个反引号之后紧跟等号，可以使得反引号用来作 Vim 表达式，而非外部命令来扩展。例如

```
:e `=tempname()`
```

该表达式里可以包含任何东西，因而可以用来避免 '"'、'|'、'%' 和 '#' 的特殊含义。不过，和其它通配符一样，'wildignore' 也适用。

表达式里的环境变量在执行表达式前先扩展，所以可以这样：

```
:e `=$HOME . '/.vimrc'`
```

此处 $HOME 如果在字符串里，会被按本义使用，所以不能这样：

```
:e `='$HOME' . '/.vimrc'`
```

如果表达式结果为字符串，名字以换行符分隔。如果结果是 List，每个项目用作一个名字。换行符也分隔名字。
注意 这样的表达式只有在 Ex 命令期待文件名参数的地方才有效。

++opt [++opt]

[++opt] 参数可以用来为某个命令强制指定 'fileformat'、'fileencoding' 或 'binary' 的值，并指定遇到坏字符的行为。其形式是：

```
++{optname}
```

或：

```
++{optname}={value}
```

其中 {optname} 是以下之一： ++ff ++enc ++bin ++nobin ++edit

ff	或	fileformat	超越 'fileformat' 之值
enc	或	encoding	超越 'fileencoding' 之值
bin	或	binary	置位 'binary'

nobin 或 nobinary　　复位 'binary'
bad　　指定坏字符的行为
edit　　只用于 :read : 保持选项的值，就像编辑文件那样

{value} 不能包含空白。它可以是这些选项任何可取之值。例如:

```
:e ++ff=unix
```

再次编辑相同的文件，但指定 'fileformat' 为 "unix"。

```
:w ++enc=latin1 newfile
```

把当前缓冲区写入到 "newfile"，以 latin1 编码。

可以用多个 ++opt 参数，以空格分隔。它们都必须在任何 +cmd 参数之前。

++bad

参数 "++bad=" 指定遇到不能转化或者包含非法字节的字符时的行为。它可以是以下三种形式之一:

++bad=X　　单字节的字符，替代每个坏字符。
++bad=keep　　保留坏字符，不予转化。注意 这可能使得你的文本里出现非法字节!
++bad=drop　　删除坏字符。

缺省就像用了 "++bad=?" 一样: 每个坏字符被问号代替。有些地方会使用一个倒转的问号 (0xBF)。

注意 不是所有的命令都接受 ++bad 参数，但也不会报错，例如 :write 。

注意 在读入文件时，'fileformat' 和 'fileencoding' 选项会被设为实际所用的格式。在写回时，这不会发生。因此下一次写会使用选项的旧值。'binary' 选项也同样如此。

+cmd [+cmd]

[+cmd] 参数可以用来在新打开的文件定位光标或执行任何其他命令:

+　　从最后一行开始。
+{num}　　从第 {num} 行开始。
+/{pat}　　从匹配 {pat} 的第一行开始。
+{command}　　打开新文件以后执行 {command}。
{command} 可以是任何 Ex 命令。

要在 {pat} 或 {command} 里包含空白，在它之前加上反斜杠。反斜杠本身则要加倍。

```
:edit  +/The\ book          file
:edit  +/dir\ dirname\\     file
:edit  +set\ dir=c:\\\\temp  file
```

注意 最后一个例子里反斜杠减半两次: 一次用于 "+cmd" 参数，一次用于 ":set" 命令。

file-formats

'fileformat' 选项指定文件里的换行符 (<EOL>) 风格:

'fileformat'	字符	名字	
"dos"	<CR><NL> 或 <NL>	DOS 格式	DOS-format
"unix"	<NL>	Unix 格式	Unix-format
"mac"	<CR>	Mac 格式	Mac-format

以前还使用 'textmode' 。现在已经废弃了。

在读入文件时，以上列出的字符被解释为换行符。DOS 格式 (Win32 缺省值) 把 <CR><NL> 和 <NL> 都解释为换行符。 注意 在以 DOS 格式写入文件时，会在每个单独的

<NL> 之后加上 <CR> 字符。另见 file-read 。

当写入文件时，换行符用以上列出的字符写入。DOS 格式使用 <CR><NL>。另见 DOS-format-write 。

你可以从 DOS 格式读入文件再以 Unix 格式写回。这样所有的 <CR><NL> 对会被换成 <NL> (假设 'fileformats' 包括 "dos"):

```
:e file
:set fileformat=unix
:w
```

反之，如果你从 Unix 格式读入文件再以 DOS 格式写回。所有的 <NL> 字符会被换成 <CR><NL> (假设 'fileformats' 包括 "unix"):

```
:e file
:set fileformat=dos
:w
```

如果你开始编辑新文件而 'fileformats' 选项非空 (缺省)，Vim 会测试文件以何种格式分隔行。如果设为 "unix,dos"，Vim 会检查每行是带 <NL> (Unix 和 Amiga) 还是带 <CR><NL> 对 (MS-DOS)。只有当 所有 行都以 <CR><NL> 结尾，'fileformat' 才被设为
"dos"，否则，它被设为 "unix"。如果 'fileformats' 包含 "mac" 而且文件不含 <NL> 字符，'fileformat' 设为 "mac"。

如果 'fileformat' 选项在非 MS-DOS 的系统上设为 "dos"，消息 "[dos format]" 会提示你发生了不同寻常的事情。 在 MS-DOS 系统上，如果 'fileformat' 设为 "unix"，你也会得到消息 "[unix format]"。除了 Macintosh 以外的所有的系统中，如果 'fileformat' 设为 "mac"，你会得到消息 "[mac format]"。

如果 'fileformats' 选项为空而使用了 DOS 格式，但是在读入文件时发现某些行不以 <CR><NL> 结尾，"[CR missing]" 会在文件消息里列出。
如果 'fileformats' 选项为空而使用了 Mac 格式，但是在读入文件时发现某些行有 <NL>，"[NL missing]" 会在文件消息里列出。

如果新文件不存在，则当 'fileformats' 为空时使用当前缓冲区的 'fileformat'，否则使用 'fileformats' 的第一个格式。

在开始编辑二进制、可执行或 Vim 脚本文件时，你应该置位 'binary' 选项。一个简单的方式是以 "-b" 选项启动 Vim。该选项可以避免使用 'fileformat'。如果不设，你会有单个 <NL> 字符被莫名其妙地换成 <CR><NL> 的风险。

你可以用 'key' 选项在写回时给文件加密。它提供了相当安全性，防止别人读到你的文件。 encryption

---

3. 参数列表 argument-list arglist

如果你在启动 Vim 的时候给出多个文件名，这些文件将被记住，即参数列表。你可以跳转到该列表里的任何一个文件。

不要把它和 :buffers 命令里的缓冲区列表混淆。参数列表在 Vi 里已经存在，但缓冲区列表是 Vim 新加入的。参数列表里的文件名也会在缓冲区列表里存在 (除非你用 :bdel 或 :bwipe 将其删除)，但不在参数列表而出现在缓冲区列表的名字很常见。

该主题在用户手册的 07.2 一节已有介绍。

只有一个全局参数列表，所有窗口缺省都使用它。但可以创建局部于窗口的新参数列表，见 :arglocal 。

下面的命令使用参数列表，还有表达式函数 argc() 和 argv() 。他们均作用于当前窗口的参数列表。

:ar :arg :args

:ar[gs] 显示参数列表，当前文件以方括号表示。

:ar[gs] [++opt] [+cmd] {arglist} :args_f
定义 {arglist} 为新的参数列表并编辑其中的第一个。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。
另见 ++opt 和 +cmd 。

:ar[gs]! [++opt] [+cmd] {arglist} :args_f!
定义 {arglist} 为新的参数列表并编辑其中的第一个。忽略任何对当前缓冲区的改动。
另见 ++opt 和 +cmd 。

:[count]arge[dit][!] [++opt] [+cmd] {name} .. :arge :argedit
将一或多个 {name} 加到参数列表里，并编辑之。
如果 {name} 已经在参数列表里存在，只编辑之。
这和用 :argadd 然后 :edit 类似。
文件名里的空格需用 "\" 转义。
[count] 的用法和 :argadd 一样。
如果当前文件不能被放弃 abandon ，{name} 等仍会被加入到参数列表中，但不进行编辑。不检查重复项。
另见 ++opt 和 +cmd 。

:[count]arga[dd] {name} .. :arga :argadd E479
:[count]arga[dd]
将若干文件名 {name} 等加到参数列表里。如果 {name} 省略，把当前缓冲区名字加入参数列表。
如果忽略 [count]， {name} 等加入到参数列表的当前项之后。不然，加到第 [count] 个文件之后。如果参数列表是 "a b c"，而 "b" 是当前参数，那么以下命令会导致：

命令	新的参数列表
:argadd x	a b x c
:0argadd x	x a b c
:1argadd x	a x b c
:$argadd x	a b c x

在末项之后：

:+2argadd y	a b c x y

这里不会检查重复项，因此一个文件可能加入参数列表两次。
不改变当前编辑的文件。
注意：你也可以使用这个方法：

```
:args ## x
```

这会加入 "x" 项并对新的列表排序。

:argd[elete] {pattern} .. :argd :argdelete E480
从参数列表里删除匹配 {pattern} (可有多个) 的文件。
{pattern} 为文件模式，见 file-pattern 。"%" 可以用来

删除当前入口项。
即使该命令从参数列表里删除了当前编辑文件，该文件保持被编辑。
例如：

```
:argdel *.obj
```

:[range]argd[elete] 从参数列表里删除 [range] 范围里的文件。
例如：

```
:10,$argdel
```

删除第 10 个和其后的参数，保留 1-9。

```
:$argd
```

只删除最后一个。

```
:argd
:.argd
```

删除当前参数。

```
:%argd
```

删除参数列表里所有文件。
如果该范围的最后一个数字过大，删除直到最后一个参数为止。

:argu :argument
:[count]argu[ment] [count] [++opt] [+cmd]
编辑参数列表里的第 [count] 个文件。在忽略 [count] 的时候，编辑当前入口项。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。
另见 ++opt 和 +cmd 。

:[count]argu[ment]! [count] [++opt] [+cmd]
编辑参数列表里的第 [count] 个文件。忽略任何对当前缓冲区的已有修改。在忽略 [count] 的时候，编辑当前入口项。
另见 ++opt 和 +cmd 。

:[count]n[ext] [++opt] [+cmd] :n :ne :next E165 E163
编辑向后第 [count] 个文件。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。另见 ++opt 和 +cmd 。

:[count]n[ext]! [++opt] [+cmd]
编辑向后第 [count] 个文件。忽略任何对当前缓冲区的已有修改。另见 ++opt 和 +cmd 。

:n[ext] [++opt] [+cmd] {arglist} :next_f
和 :args_f 相同。

:n[ext]! [++opt] [+cmd] {arglist}
和 :args_f! 相同。

:[count]N[ext] [count] [++opt] [+cmd] :Next :N E164
编辑向前第 [count] 个文件。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。另见 ++opt 和 +cmd 。

:[count]N[ext]! [count] [++opt] [+cmd]
编辑向前第 [count] 个文件。忽略任何对当前缓冲区的已有

修改。另见 ++opt 和 +cmd 。

:[count]prev[ious] [count] [++opt] [+cmd] :prev :previous
和 :Next 相同。另见 ++opt 和 +cmd 。

:rew :rewind
:rew[ind] [++opt] [+cmd]
开始编辑参数列表的第一个文件。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。另见 ++opt 和 +cmd 。

:rew[ind]! [++opt] [+cmd]
开始编辑参数列表的第一个文件。忽略任何对当前缓冲区的已有修改。另见 ++opt 和 +cmd 。

:fir :first
:fir[st][!] [++opt] [+cmd]
":rewind" 的别名。

:la :last
:la[st] [++opt] [+cmd]
开始编辑参数列表的最后一个文件。如果已经作了修改而 vim 不能放弃 abandon 当前的缓冲区，该命令失败。另见 ++opt 和 +cmd 。

:la[st]! [++opt] [+cmd]
开始编辑参数列表的最后一个文件。忽略任何对当前缓冲区的已有修改。另见 ++opt 和 +cmd 。

:wn :wnext
:[count]wn[ext] [++opt]
写回当前文件并开始编辑向后第 [count] 个文件。另见 ++opt 和 +cmd 。

:[count]wn[ext] [++opt] {file}
写入当前文件到 {file} 并开始编辑向后第 [count] 个文件，除非 {file} 已经存在并且 'writeany' 选项被关闭。另见 ++opt 和 +cmd 。

:[count]wn[ext]! [++opt] {file}
写入当前文件到 {file} 并开始编辑向后第 [count] 个文件。另见 ++opt 和 +cmd 。

:[count]wN[ext][!] [++opt] [file] :wN :wNext
:[count]wp[revious][!] [++opt] [file] :wp :wprevious
和 :wnext 相同，向前编辑而不是向后。

以上命令的 [count] 缺省为一。对有些命令可以使用两个计数。后者（右面那个）被使用。

如果没有给出 [+cmd] 参数，光标定位于文件最后一个已知的位置。如果置位了 'startofline'，光标会定位于该行第一个非空白字符，不然，则使用最后一个已知列。如果没有最后已知的光标位置，光标会定位在第一行（Ex 模式下则是最后一行）。

{arglist}
参数列表里的通配符会扩展，文件名会排序。这样你可以用命令 "vim *.c" 编辑所有的 C 文件。在 Vim 里面，":n *.c" 命令会做同样的事。

空白用来分隔文件名。在空格或制表前加上反斜杠可以在文件名里使用它们。例如，要编辑单个文件 "foo bar":

```
:next foo\ bar
```

在 Unix 和一些其它系统上你还可以使用反引号，例如:

```
:next `find . -name \\*.c -print`
```

星号之前的反斜杠是必要的，它用来防止 "*.c" 在执行 find 程序之前被外壳扩展。

arglist-position
如果有参数列表，你可以在窗口的标题上看到正在编辑哪个文件（如果有一个窗口，而且 'title' 被置位），或者通过 "CTRL-G" 命令，在文件消息里看到。你会看到类似于

```
(file 4 of 11)
```

的东西。如果 'shortmess' 包含 'f'，它会成为

```
(4 of 11)
```

如果你还没有编辑参数列表当前位置的文件，它会是

```
(file (4) of 11)
```

这表示你在参数列表的第四个位置，不过还没有开始编辑之。这发生于你刚做 ":e file" 的时候。

局部参数列表

:arglocal

:argl[ocal] 复制一个全局参数列表的局部备份。并不开始编辑另外一个文件。

:argl[ocal][!] [++opt] [+cmd] {arglist}
定义一个新的局部于当前窗口的参数列表。除此以外，和 :args_f 列表。

:argglobal

:argg[lobal] 当前窗口使用全局参数列表。并不开始编辑另外一个文件。

:argg[lobal][!] [++opt] [+cmd] {arglist}
当前窗口使用全局参数列表。和 :args_f 一样，定义一个新的全局参数列表。所有使用全局参数列表的窗口都会看到这个新的列表。

可以有多个参数列表。他们可以在窗口之间共享。如果他们共享，在其中一个窗口修改参数列表就会改变另一个窗口的列表。

如果一个窗口被分割，新的窗口从原来的窗口继承参数列表。两个窗口因而共享该列表，直到其中一个窗口使用 :arglocal 或 :argglobal 来使用另外一个参数列表。

使用参数列表

:argdo

:[range]argdo[!] {cmd} 对参数列表里的每个文件执行 {cmd}。如果给出 [range]，只对给出范围内的参数执行。

它的工作方式大致如下：

```
:rewind
:{cmd}
:next
:{cmd}
等等
```

如果当前文件不能被放弃 abandon 而且不存在 [!]，该命令失败。
如果一个文件检测到错误，参数列表里的其余文件将不再被操作。
参数列表里的最后一个文件（或发生错误的那个）成为当前文件。
{cmd} 可以包含 '|'，从而连接多个命令。
{cmd} 不可修改参数列表。
注意：当命令执行时，Syntax 自动命令事件被加到 'eventignore' 里，从而被屏蔽。这样显著提高了编辑每个文件的速度。
另见 :windo 、 :tabdo 、 :bufdo 、 :cdo 、 :ldo 、 :cfdo 和 :lfdo

示例：

```
:args *.c
:argdo set ff=unix | update
```

它把 'fileformat' 选项设为 "unix"，并写回现在已改变的所有的 *.c 文件。

示例：

```
:args *.[ch]
:argdo %s/\<my_foo\>/My_Foo/ge | update
```

它把所有 *.c 和 *.h 文件里的单词 "my_foo" 换成 "My_Foo"。"e" 标志位使得 ":substitute" 命令对没有使用 "my_foo" 的文件不提示错误。":update" 只有在发生改变的时候才写入文件。

---

4. 写入 writing save-file

注意：如果 'write' 选项关闭，你不能写入任何文件。

:w :write
E502 E503 E504 E505
E512 E514 E667 E796 E949

:w[rite] [++opt]
将整个缓冲区写入当前文件。这是保存文件更动最普通的方式。如果置位了 'readonly' 选项或者其他原因不能写入文件，它会失败。
关于 ++opt 可见 ++opt ，但只有 ++bin、++nobin、++ff 和 ++enc 有效。

:w[rite]! [++opt]
和 ":write" 类似，但即使 'readonly' 已置位或者有其他原因写入被拒绝，还是强制写入。
注意：这可能会改变文件的权限和所有者，或者破坏（符号）连接。在 'cpoptions' 里加上 'W' 标志位可以避免这一点。

:[range]w[rite][!] [++opt]
将指定行写入当前文件。这是很特别的，因为文件里将不会包

含缓冲区的所有行。

:w_f :write_f

:[range]w[rite] [++opt] {file}
将指定行写入文件 {file}，除非该文件已存在并且未置位 'writeany' 选项。

:w!

:[range]w[rite]! [++opt] {file}
将指定行写入文件 {file}。覆盖已存在的文件。

:w_a :write_a E494

:[range]w[rite][!] [++opt] >>
将指定行附加到当前文件后。

:[range]w[rite][!] [++opt] >> {file}
将指定行附加到文件 {file} 之后。'!' 强制写入，即使该文件还不存在。

:w_c :write_c

:[range]w[rite] [++opt] !{cmd}
执行命令 {cmd}，以 [range] 指定的行作为它的标准输入。(注意 ! 之前的空格)。{cmd} 以 ":!{cmd}" 类似的方式被执行，任何 '!' 被替换成前一个命令 :! 。

":w" 命令缺省的 [range] 是整个缓冲区 (1,$)。如果写回整个缓冲区，缓冲区不再被认为已改变。用 ":w somefile" 写到别的文件的情形则取决于 'cpoptions' 的 "+" 标志位。如果包含，即便缓冲区本身和其关联的文件可能已不同，该写命令还是复位 "modified" 标志位。

如果 ":w" 给出一个文件名，它成为轮换文件。这可以用来，比如说，当写入失败而你想再迟些再试一次: ":w #"。该功能可以通过从 'cpoptions' 选项里去掉 'A' 标志位来关闭。

注意 'fsync' 选项如果置位，写操作会变慢 (但更安全)。

:sav :saveas

:sav[eas][!] [++opt] {file}
用文件名 {file} 保存当前缓冲区，并设置当前缓冲区的文件名为 {file}。前一个名字用作轮换文件名。[!] 用以覆盖已存在的文件。
如果 'filetype' 为空，在写入文件前用新名字进行文件类型检测。
如果写操作成功，复位 'readonly'。

:up :update

:[range]up[date][!] [++opt] [>>] [file]
和 ":write" 类似，但只有在缓冲区已修改的时候才写入。

写 入 多 个 缓 冲 区 buffer-write

:wa :wall

:wa[ll] 保存所有已修改的缓冲区。没有文件名的缓冲区会报错。只读

的缓冲区不会被写入。

:wa[ll]!　　保存所有已修改的缓冲区，包括只读的。没有文件名的缓冲区不会写入并报错。

如果你试图覆盖在别的地方修改的文件，Vim 会警告你。参见 timestamp 。

backup E207 E506 E507 E508 E509 E510

如果你写入已存在的文件（但非附加）而打开了 'backup'、'writebackup' 或者 'patchmode' 选项，原来文件的一个备份会被建立。该文件或者由复制，或者由换名得到（见 'backupcopy'）。在文件被成功写入后，当置位 'writebackup' 选项而没有置位 'backup' 时，该备份文件被删除。当 'patchmode' 选项打开时，备份文件可能被改名。

backup-table

'backup'	'writebackup'	动作
关	关	没有备份
关	开	备份当前文件，写入后删除（缺省）
开	关	删除旧的备份，备份当前文件
开	开	删除旧的备份，备份当前文件

当 'backupskip' 模式匹配被写入的文件名，备份文件不会被创建。此时，'backup' 和 'writebackup' 被忽略。

如果 'backup' 选项置位，旧的备份文件（与新的备份文件同名）被删除。如果没有置位 'backup' 但置位了 'writebackup'，已有的备份文件不会被删除。文件写入时使用的是另外一个文件名。

有些文件系统的崩溃可能导致备份文件和新写入的文件同时丢失（可能还在但包含不正确数据）。这种情况下可以尝试修复功能 :recover ，因为交换文件在盘上保持一个同步的备份，可能还有用。

'backupdir' 选项给出的目录用于存放备份文件（缺省在写入文件的相同目录）。

不管备份文件是否新建，它是否原始文件复制还是换名得来由 'backupcopy' 选项决定。该选项的描述解释了何时使用复制，何时使用换名。

如果备份文件创建失败，写入不会进行。除非命令中加上 '!'。

write-permissions

写入新文件时其权限设为可读写。unix 上的掩码是 0666 外加 umask。写入已经读入的文件时，Vim 保留原有权限，但清除 s 位。

write-readonly

如果 'cpoptions' 选项里包含 'W'，Vim 拒绝覆盖只读文件。如果没有该标志位，":w!" 可以用来覆盖只读文件， 如果系统支持的话（目录本身必须是可写的）。

write-fail

如果新文件写入失败，要小心不要同时丢失了你的修改内容 和 原始的文件。如果没有备份文件而写入新文件失败的话，原始的文件已经没有了！除 非 你 写 了 文 件，不 要 退 出 VIM！如果创建了备份，它保存了原始文件的内容（如果可能的话）。如果你退出 Vim 并丢失了修改的部分，原始的文件很有可能还在。如果原始文件的保存也失败的话，你会得到一个错误信息，提示丢失了原始文件。

DOS-format-write
如果 'fileformat' 为 "dos"，<CR><NL> 用作换行符 <EOL>。这是 Win32 的缺省行为。在其他系统中，"[dos format]" 信息会提示你使用了不同寻常的 <EOL>。

Unix-format-write
如果 'fileformat' 为 "unix"，<NL> 用作换行符 <EOL>。在 Win32 上会显示 "[unix format]"。

Mac-format-write
如果 'fileformat' 为 "mac"，<CR> 用作换行符 <EOL>。在非 Mac 的系统上会显示 "[mac format]" 消息。

另见 file-formats 和 'fileformat' 以及 'fileformats' 选项。

ACL
ACL 代表访问控制表 (Access Control List)。这是控制文件访问权限一个高级方法。在新的 MS-Windows 和 Unix 系统上，如果有文件系统的支持，可以使用之。
Vim 试图在写入文件时保留 ACL 信息。备份文件会用和原始文件相同的 ACL 信息。
ACL 信息也用来检查文件是否只读 (在打开文件时)。

read-only-share
当 MS-Windows 共享网络驱动器时，可以指定其为只读。这意味着即使文件本身没有只读属性，你也不能写入该文件。Win32 平台上的 Vim 会检测只读的网络驱动器，并标记其上的文件为只读。你不能用 :write 修改。

write-device
如果文件名其实是设备名，Vim 不会建立备份 (这也不可能)。你需要使用 "!"，因为设备已经存在。Unix 上的示例:

```
:w! /dev/lpt0
```

MS-DOS 或 MS-Windows 上:

```
:w! lpt0
```

Unix 上当文件名不引用一个普通文件或者目录时，检测为设备。fifo 和命令管道对 Vim 而言视同设备。MS-DOS 和 MS-Windows 上以下名字检测为设备:

```
AUX
CON
CLOCK$
NUL
PRN
COMn    n=1,2,3... 等
LPTn    n=1,2,3... 等
```

名字大小写均可。

---

5. 写入并退出 write-quit

:q :quit

:q[uit]	退出当前窗口。如果是最后的一个，退出 Vim。如果已经有修改而且 Vim 拒绝放弃 abandon 当前的缓冲区，或者如果参数列表的最后一个文件还没有被编辑，该操作失败。如果有其它标签页，并且退出的是当前标签页的最后一个窗口，关闭当前标签页 tab-page 。 激活 QuitPre 自动命令事件。 参见 CTRL-W_q 用来退出其它窗口。
:conf[irm] q[uit]	退出，但如果已经有修改或者参数列表的最后一个文件还未被编辑，给出提示。参见 :confirm 和 'confirm'。

:q[uit]!　　退出不保存，即使当前缓冲区发生了修改也是。缓冲区被卸载，包括置位 'hidden' 了的。
如果是最后一个窗口并有修改过的隐藏缓冲区，放弃当前缓冲区，第一个修改过的隐藏缓冲区成为当前缓冲区。
为了保证总能退出，用 ":qall!"。

:cq[uit]　　在任何情形下，退出不保存，并返回一个错误代码。参见 :cq 。此功能可用于 Manx 的 QuickFix 模式 (参见 quickfix)。

:wq　　:wq
:wq [++opt]　　写回当前的文件并且退出。如果文件只读或者该缓冲区无名，写回操作将失败。如果参数列表的最后一个文件还没有编辑，那么退出操作将会失败。

:wq! [++opt]　　写回当前的文件并且退出。如果当前缓冲区无名，则写回操作将失败。

:wq [++opt] {file}　　写入到 {file} 并且退出。 如果参数列表的最后一个文件还没有编辑， 那么退出操作将会失败。

:wq! [++opt] {file}　　写入到 {file} 并且退出。

:[range]wq[!] [++opt] [file]
　　同上，但只写入 [range] 界定的那些行。

:x　　:xit
:[range]x[it][!] [++opt] [file]
　　和 ":wq" 类似， 但只有文件已修改时写入才会实际进行。
如果 'hidden' 被设置并且还有其他窗口，当前缓冲区会在写入后被隐藏。

:exi　　:exit
:[range]exi[t][!] [++opt] [file]
　　同 :xit。

ZZ　　ZZ
ZZ　　如果文件被修改，写回当前文件。然后退出。(和 :x 相同)。(注意：如果有当前文件有多个窗口，该文件在被修改时会被写回，当前窗口会被关闭)。

ZQ　　ZQ
ZQ　　退出，不检查是否发生了修改 (等同于 ":q!")。

多 个 窗 口 和 缓 冲 区　　window-exit

:qa　　:qall
:qa[ll]　　退出 Vim，除非存在修改过的缓冲区。(可以使用 ":bmod" 跳转到下一个修改过的缓冲区)。如果置位了 'autowriteall'，所有被修改的缓冲区将被保存，一如调用了 :wqall 。

:conf[irm] qa[ll]
　　退出 Vim。如果存在修改过的缓冲区，给出提示。参见 :confirm 。

:qa[ll]!　　退出 Vim，不会保存任何的修改。
另见 :cquit ，功能相同但使用非零的退出值。

:quita　:quitall
:quita[ll][!]　　等同于 ":qall"。

:wqa[ll] [++opt]　:wqa　:wqall　:xa　:xall
:xa[ll]　　保存所有修改过的缓冲区并退出 Vim。如果其中有无名的、只读的、其他原因写入失败的缓冲区，或者有还在运行作业的终端，Vim 不会退出。

:conf[irm] wqa[ll] [++opt]
:conf[irm] xa[ll]
保存所有修改过的缓冲区并退出 Vim。如果其中有只读或者其他原因写入失败的缓冲区，给出提示。参看 :confirm 。

:wqa[ll]! [++opt]
:xa[ll]!　　保存所有修改过的缓冲区，甚至包括只读的，然后退出 Vim。不过，如果有无名或者其他原因写入失败的缓冲区，Vim 仍然不会退出。

---

6. 对话框　edit-dialogs

:confirm　:conf
:conf[irm] {command}　　执行 {command}，如果有操作需要确认，显示对话框。可用于 ":q"、":qa" 和 ":w" 命令（后者用于超越只读的设定），和其他会以类似方式失败的命令，如 :only 、 :buffer 、 :bdelete 等。

示例：

:confirm w foo
　　如果 "foo" 存在请求确认。
:confirm q
　　如果有改变请求确认。
:confirm qa
　　如果有任何修改过的未保存的缓冲区存在，对其中的每个缓冲区提示是保存还是放弃。还可以选择 "save all"（保存所有）或 "abandon all"（放弃所有）。

如果你希望总是用 ":confirm"，置位 'confirm' 选项。

:browse　:bro　E338　E614　E615　E616
:bro[wse] {command}　　为 {command} 的参数显示文件选择对话框。目前这可用于 :e 、 :w 、 :wall 、 :wq 、 :wqall 、 :x 、 :xall 、 :exit 、 :view 、 :sview 、 :r 、 :saveas 、 :sp 、 :mkexrc 、 :mkvimrc 、 :mksession 、 :mkview 、 :split 、 :vsplit 、 :tabe 、 :tabnew 、 :cfile 、 :cgetfile 、 :caddfile 、 :lfile 、 :lgetfile 、 :laddfile 、 :diffsplit 、 :diffpatch 、 :open 、 :pedit 、 :redir 、 :source 、 :update 、 :visual 、 :vsplit ，还有如果置位 'confirm' 的话，:qall 。

{仅对 Win32、Athena、Motif、GTK 和 Mac GUI 有效}
如果不可用 ":browse"，你会得到错误消息。如果 +browse

特性不存在或者 {command} 不支持该（浏览）功能，不显示对话框，如常执行 {command}。
":browse set" 和 :options 类同。
关于 ":browse oldfiles"，另见 :oldfiles 。

该语法最好通过若干范例说明之：

:browse e $vim/foo
在 $vim/foo 目录里打开浏览器并编辑选择的文件。

:browse e
在 'browsedir' 指定的目录里打开浏览器并编辑选择的文件。

:browse w
在当前缓冲区所在的目录里打开浏览器。缺省使用当前缓冲区的文件名。把缓冲区写入到指定的文件。

:browse w C:/bar
在 C:/bar 目录里打开浏览器，缺省使用当前缓冲区的文件名，把缓冲区写入到指定的文件。

另见 'browsedir' 选项。
如果所用的 Vim 版本不支持浏览，该命令照原样执行。

browsefilter

对 MS Windows 和 GTK 而言，你可以修改浏览对话框的过滤（filter）。设置 g:browsefilter 或 b:browsefilter 变量，你可以修改全局或局部于缓冲区的过滤设置。该变量应设为符合如下格式的字符串 "{过滤标签}\t{pattern};{pattern}\n"，其中的 {过滤标签} 是出现 "Files of Type" 组合框里应出现的文字，而 {pattern} 是过滤文件名的模式。可以给出多个模式，以 ";" 分隔。

Motif 使用相同的格式，但只使用第一个模式（Motif 只提供一个模式，但可以编辑）。

例如，要使得对话框里只显示 Vim 文件，你可以用以下命令：

```
let g:browsefilter = "Vim Scripts\t*.vim\nVim Startup Files\t*vimrc\n"
```

你可以通过在某个缓冲区上设置 b:browsefilter 变量超越全局过滤的设置。最有可能的情况是在文件类型插件里设置 b:browsefilter，以便浏览对话框只显示和当前编辑的文件所属类型相关的项目。缺点是：无法编辑一个新的不同类型的文件。要克服这一点，你可以加上 "All Files\t*.*\n" 作为最后的过滤。用户从而拥有访问任何类型文件的选择。

如果 Vim 不能实际支持时不想设置 browsefilter，可以测试 has("browsefilter")：

```
if has("browsefilter")
   let g:browsefilter = "你要的值"
endif
```

---

7. 当前目录 current-directory

你也可以用 :cd 、:tcd 和 :lcd 命令换到别的目录，所以你可以不用总是在文件名前输入目录名。这也影响到外部命令的执行，例如 ":!ls"。

如果当前缓冲区被修改、'cpoptions' 里有 '.' 标志位而命令又不使用 "!"，那么改变目录会失败。

:cd E747 E472

:cd[!]

在非 Unix 系统上：显示当前目录名。在 Unix 系统上：改变当前目录到主 (home) 目录。在所有的系统上，用 :pwd 可以显示当前目录。
在 Unix 系统上：清除任何局部于窗口的目录。

:cd[!] {path}

切换当前目录为 {path}。如果 {path} 是相对路径，则在 'cdpath' 列出的目录搜索之。
清除任何局部于窗口的目录。
这不会改变已经打开的文件，因为记住的是它们的完整路径。
但 arglist 里的文件则可能会改变！
MS-DOS 上这会同时改变当前的驱动器。
要改变到当前文件的目录：

```
:cd %:h
```

:cd- E186

:cd[!] -

切换到上一个当前目录 (在上个 ":cd {path}" 命令之前)。

:chd :chdir

:chd[ir][!] [path]

和 :cd 相同。

:tcd

:tcd[!] {path}

和 :cd 类似，但只为当前标签页设置当前目录。其它标签页上的窗口和当前标签页上有自己局部于窗口的目录的窗口的当前目录保持不变。

:tch :tchdir

:tch[dir][!]

和 :tcd 相同。

:lc :lcd

:lc[d][!] {path}

和 :cd 类似，但只在光标在当前窗口时设置当前目录。别的窗口的当前目录保持不变，切换到其它窗口会停止使用 {path}。

:lch :lchdir

:lch[dir][!]

和 :lcd 相同。

:pw :pwd E187

:pw[d]

显示当前目录名。
另见 getcwd() 。

只要没有用过 :lcd 或 :tcd 命令，所有窗口共享同一个当前目录。切换到别的窗口的命令因此不会改变当前目录。

当某个窗口用了 :lcd 命令，所指定的目录成为该窗口的当前目录。没用过 :lcd 的窗口还是和全局或局部于标签页的当前目录同步。跳转到另一个窗口时，当前目录成为上次指定的本地当前目录。如果没有指定过，则使用全局或局部于标签页的当前目录。

当某个标签页用了 :tcd 命令，所指定的目录成为当前标签页和当前窗口的当前目录。其它标签页上的当前目录不受影响。跳转到另一个标签页时，当前目录成为上次指定的局部于那个标签页的当前目录。如果没有指定过，则使用全局的当前目录。

如果使用 :cd 命令，当前窗口和标签页会丢失它的本地当前目录，并从此以后使用全局当前目录。

:cd 的完整路径名用于其后的文件读写。在某些网络文件系统中，这可能会引起麻烦。使用完整路径名的结果是当前使用的文件名还是指向相同的文件。例如：如果你有文件 a:test 和 目录 a:vim。命令 ":e test" ":cd vim" ":w" 会覆盖文件 a:test 而不是 a:vim/test。但如果你使用 ":w test" 文件 a:vim/test 会被覆盖，因为你给出一个新的文件名，因而与 ":cd" 之前所指向的文件业已不同。

---

8. 编辑二进制文件 edit-binary

尽管 Vim 是用来编辑文本文件的，它也可用来编辑二进制文件。Vim 的 -b 参数 (b 代表二进制 (binary)) 使 Vim 以二进制模式进行文件读写操作，并设置二进制文件编辑相关的一些选项 ('binary' 打开，'textwidth' 为 0，'modeline' 关闭，'expandtab' 关闭)。置位 'binary' 选项效果相同。不要忘记应在读入文件之前进行该操作。

编辑二进制文件时要注意以下几点：
- 在编辑可执行文件时，不要修改字符数。只用 "R" 或 "r" 命令来修改文本。不要用 "x" 或退格键删除字符。
- 设置 'textwidth' 选项为 0。不然，行会被意外的截为两段。
- 如果 <EOL> 符不多，行会很长。如果你希望编辑屏幕显示不下的行，重设 'wrap' 选项，使用横向滚动。如果一行过长 (Amiga 上超过 32767 个字符，32 位系统中则大得多，见 limits)，你不能编辑该行。该行在读入时会被截断。也可能读入发生 "out of memory" 错误。
- 确保在读入文件 前 置位 'binary' 选项。不然，<CR><NL> 和 <NL> 都被认为行尾，从而在写回时，<NL> 会被 <CR><NL> 取代。
- <Nul> 字符会在屏幕上显示为 ^@。你可以用 "CTRL-V CTRL-@" 或 "CTRL-V 000" 输入之
- 插入 <NL> 字符会截断一行。在将缓冲区写入到文件时，<NL> 会作为 <EOL> 被写入。
- 如果文件尾没有 <EOL>，Vim 通常会自己加上。置位 'binary' 选项会禁止这一行为。你也可以读取该选项的值来知道最后一行是否有 <EOL> (文本上你是看不到的)。

---

9. 加密 encryption

Vim 可用加密的方式写入文件并读回。没有正确的密钥，加密文本不能被读取。
{仅当编译时加入 +cryptv 功能才有效} E833

交换文件和撤销文件的文本也被加密。 E843
不过，这是分块进行的，这样可能缩短破解密钥的时间。可以关闭交换文件，但任何崩溃会丢失你之前的成果。关闭撤销文件但没有什么大损失。

```
:set noundofile
:noswapfile edit secrets
```

注意：内存里的文本是不加密的。系统管理员因此可以在你编辑时看到你的文本。用 ":!filter" 或者 ":w !command" 过滤的文本也是不加密的，这些都可能造成文本泄露。'viminfo' 文件也不加密。

要编辑极度机密的文本：

```
:set noundofile viminfo=
:noswapfile edit secrets.txt
```

牢记没有交换文件意味着你冒崩溃、停电等事件时丢失工作成果的风险。

警告 ：如果在输入密钥时发生错误而且写入文件后退出，这些文本就永远丢失了。

使用加密的通常方法是用 ":X" 命令，它会提示你输入一个密钥。其后的一个写入命令会使用该密钥加密文件。如果你以后编辑该文件，Vim 会要求你输入密钥。如果你的输入正确，文件是可读的，不然，文本会显示为乱码。

:X

:X　　提示加密密钥。输入时不会反显输入的内容。别人即使看到屏幕，也不能看见你的密钥。输入的字符保存在 'key' 选项里，它的值用来在写入文件时加密该文件。在写入之前，该文件保持不变。另见 -x 。

'key' 选项里的值在文件写入时使用。如果该选项非空，写入的文件会用它的值作为密钥加密。加密过的文件的头部有个魔术数字，使得 Vim 可以识别这是加密文件。

要关闭加密，复位 'key' 选项为空值：

```
:set key=
```

使用 'cryptmethod' 选项来选择加密方法为以下之一：

```
:setlocal cm=zip        " 较弱的加密方法，后向兼容
:setlocal cm=blowfish   " 有漏洞的方法
:setlocal cm=blowfish2  " 中等强度的方法
```

必须在写入文件前进行。读入加密文件时，自动设置该选项为文件写入时选择的方法。写入前改变 'cryptmethod' 就可以改变使用的加密方法。

要设置用于新文件的缺省方法，在 vimrc 文件中放入：

```
set cm=blowfish2
```

强烈推荐使用 "blowfish2"。只有在不支持它的旧版本的 Vim 上才用其它方法。

读写文件给出的信息会显示 "[crypted]" 对应 zip，"[blowfish]" 对应 blowfish，等等。

写入撤销文件时，撤销文件的文本使用相同的密钥和方法。 persistent-undo 。

要测试有无 blowfish 支持，使用以下条件：

```
has('crypt-blowfish')
has('crypt-blowfish2')
```

尽管更早时就有 blowfish 支持，这只可用于 Vim 7.4.1099 开始的版本。所以此条件的失败不意味着不支持 blowfish。可以这么测试 blowfish：

```
v:version >= 703
```

对 blowfish2 则可以用：

```
v:version > 704 || (v:version == 704 && has('patch401'))
```

如果确信你的 Vim 包含了 patch 7.4.237，更简单的检查是：

```
has('patch-7.4.401')
```

E817 E818 E819 E820

如果加密过程有错，文本可以写到文件里，但再不能正确读回来了。为此准备了一项测试以确定加密顺利完成。如果看到这些错误其中之一，不要以加密方式写入文件！要修正之，你需要重新编译 Vim 程序。

E831 这是一个内部错误，"不应该发生"。如果你能重现该错误，请向开发者报告。

读入加密文件时如果 'key' 选项非空，它被用作解密的密钥。如果该值为空，你会得到提示，输入密钥。如果你不输密钥或者密钥不符，该文件以未解密的形式被编辑。使用错误的密钥不会有警告 (以增加暴力破解密钥的难度)。

如果要以不同的密钥读入别的文件，设置 'key' 选项为空串，从而让 Vim 给出提示输入新值。不要用 ":set" 命令输入该值，否则别人在你身后就能看到了。

因为 'key' 选项的值应该是个秘密，永远不要看它的值，也不要在 vimrc 文件里设置。

如果你把如下诸行加到 "/etc/magic"、"/usr/share/misc/magic" 或者你系统支持的 "magic" 文件里，你的加密文件可以被 "file" 命令识别：

```
0  string  VimCrypt~       Vim encrypted file
>9 string  01      - "zip" cryptmethod
>9 string  02      - "blowfish" cryptmethod
>9 string  03      - "blowfish2" cryptmethod
```

注意：
- 如果用 'charconvert' 转换，不能实施加密。
- 你复制或者删除的文本保存在编号的寄存器中。寄存器可以保存在 .viminfo 文件里，从而可以读到。修改你的 'viminfo' 选项以确保安全。
- 如果你走开一会儿，别人可能在 Vim 里输入命令，他不可能得到你的密钥。
- 如果你输入密钥时发生错误，你就无法取回你的文本了！
- 如果你用 ":set key=value" 命令输入密钥，它会保存在历史里，从而在 viminfo 文件里显示 'key' 的值。
- 没有 100% 的安全。Vim 的加密算法没有经过健壮性的严格测试。
- 用于 'cryptmethod' 的 "zip" 算法是可以破解的。一个 4 字符的密钥需要大概一个小时，6 字符的密钥需要一天（在一台 Pentinum 133 PC 上）。这需要你知道文件里一定会出现的若干文本。
  专家可以依此方法破解任何密钥，如果文本已经被解密，则密钥可以被推算出，而其他以相同密钥加密的文件也能被解出。
- Pkzip 使用和 'cryptmethod' 的 "zip" 相同的加密方法，美国政府对其出口不持异议。Pkzip 的公共文件 APPNOTE.TXT 详细描述了这一算法。
- 'cryptmethod' 的 "blowfish" 实现有漏洞。可以破解文件的前 64 个字节，在有些情况下，甚至更多。不推荐，但 Vim 7.3 和 7.4 里还是最强的方法。"zip" 方法更弱。
- Vim 起源于荷兰。这是源代码的出处。因此，加密代码并非从美国出口。

---

10. 修改时间 timestamp timestamps

Vim 记住文件开始编辑时的修改时间、模式和大小。这用来防止（在你自己知道之前）你有同一个文件的不同版本。

在执行了外壳命令（ :!cmd suspend :read! K ）以后，Vim 会对窗口里的所有缓冲区的修改时间、模式和大小进行比较。Vim 会对在此期间被修改的文件执行相关的 FileChangedShell 自动命令或者显示警告。GUI 上，在 Vim 取回输入焦点的时候也会发生。

E321 E462

如果你想自动重新载入在 Vim 之外被修改的文件的新内容，置位 'autoread' 选项。如果你已经在 Vim 里修改过这个文件，这不会工作。

ignore-timestamp

如果你不想被询问，也不想自动载入文件，可以这样：

```
set buftype=nofile
```

或者，从外壳启动 gvim：

```
gvim file.log -c "set buftype=nofile"
```

注意 如果定义了 FileChangedShell 自动命令，你不会得到警告信息或者提示。该自动命令应负责处理。

对目录的修改也不会有警告（例如用 netrw-browse ），除非你开始编辑一个新文件，但它在外部被建立为一个目录。

当 Vim 注意到一个文件的修改时间被修改并且该文件正在缓冲区里等待编辑但还未经修改，会先检查文件内容是否相同，这是通过再次读入该文件（到一个隐藏缓冲区，该缓冲区事后被立即删除），并比较两个缓冲区的文本完成的。如果相同，你不会得到警告。

如果你觉得警告不够及时或不够充分，你可以使用如下命令。

:checkt :checktime

:checkt[ime] 检查是否有缓冲区在 Vim 之外被改变。 该命令检查你是否有文件的两个版本并给出警告。
如果它在自动命令、":global" 命令或者任何非键盘输入中调用，实际的检查会延迟，直到副作用（重新读入文件）不会有任何问题。
每个已读入的缓冲区会检查相关的文件是否被改变。如果是，则 Vim 会采取行动。如果缓冲区没有修改过而且置位了 'autoread'，缓冲区被重新读入。否则，你会被询问是否重新读入文件。如果文件被删除，你会得到一个错误信息。如果文件过去不存在但现在已经存在，你会得到一个警告。
一旦检查过某个文件，他的修改时间被重置。因而，你不会被重复警告。

:[N]checkt[ime] {filename}
:[N]checkt[ime] [N]
检查某个特定缓冲区的修改时间。该缓冲区可以用名字、编号或者模式指定。

E813 E814

如果你指定 Vim 重载缓冲区，如果某可见窗口包含了该缓冲区，重载会在该窗口的上下文中进行。否则，会使用一个特殊窗口，这使得大多数的自动命令可以执行。你不能关闭这个窗口，还有一些其它特殊的限制。最好不要涉及到本缓冲区之外的对象。例如，设置局部于窗口的选项可能会选错窗口。分割窗口来做一些操作，然后关闭新分割的窗口应该没问题（假定其它自动命令没有副作用）。但关闭不相关的窗口和缓冲区就不好了。

在你写入文件的时候，修改时间被检查。如果发生了改变，Vim 会提示你是否要真的覆盖该文件：

```
WARNING: The file has been changed since reading it!!!
Do you really want to write to it (y/n)?
```

如果你按 'y'，Vim 会继续写入文件。如果你按 'n'，写入操作被取消。如果你用 ":wq" 或者 "ZZ"，Vim 因而不会退出。从而你还有机会写入文件。

通常，该消息意味着有人在编辑会话开始后，写入到该文件里。这可能是别人，这时你可能想检查你的修改和别人的修改是否可以合并。将文件写到别的名字然后比较两者的差异（可以用 "diff" 程序）。

当然也有可能你自己在另外一个编辑会话中或者用别的命令（例如一个过滤命令）修改了该文件。这样，你自己知道希望保留文件的哪个版本。

有一个情况即使在没有任何错误的时候也会得到消息：Win32 系统在夏令时开始的时候。Win32 库的一些细节使得 Vim 对小时的变化有些混淆。第二天问题就没有了。

---

11. 文件搜索 file-searching

{仅当编译时加入 +path_extra 特性才有效}

文件搜索目前用于 'path'、'cdpath' 和 'tags' 选项和 finddir() 和 findfile() 。其他命令使用 wildcards ，则略有不同。

有三种不同类型的搜索：

1) 向下搜索： starstar
向下搜索使用 '*'、'**' 或其他操作系统支持的通配符。'*' 和 '**' 是由 Vim 内部处理的，以适用于所有操作系统。注意 "**" 只在文件名开始处才被当作特殊的通配符。

'*' 的使用很简单：匹配 0 个或更多字符。如果用搜索模式，应该是 ".*"。但注意文件搜索不使用 "." 元字符。

'**' 则复杂的多：
- 只 匹配目录。
- 匹配缺省最多可达 30 层的目录。所以你可以用它整个目录树里查找
- 最大匹配的层数可以通过在 '**' 之后给出数字限定。
  这样，'/usr/**2' 可以匹配：
  ```
  /usr
  /usr/include
  /usr/include/sys
  /usr/include/g++
  /usr/lib
  /usr/lib/X11
  ....
  ```
  它 不 匹配 '/usr/include/g++/std'，因为这已到了第三层。
  允许的数字范围是 0 ('**0' 被直接去掉) 到 100。如果给定的数字小于 0，则缺省为 30。如果大于 100，则使用 100。系统本身还有路径长度的上限，通常是 256 或 1024 字节。
- '**' 只能用于路径的尾部，或者后面紧跟一个路径分隔符，或者后面紧跟一个数字加一个路径分隔符。

你可以用任何顺序混合 '*' 和 '**'：
```
/usr/*/sys/**
/usr/*tory/sys/**
/usr/**2/sys/*
```

2) 向上搜索：
这里，你可以给出一个目录，然后沿着目录树向上搜索一个文件。你可以给出若干终止目录来限制搜索的范围。终止目录附加于路径 (对 'path' 选项而言) 或文件名 (对 'tags' 选项而言) 之后，中间以 ";" 分隔。如果需要多个终止目录，一一列出，以 ';' 分隔。如果你不想要终止目录 (搜索直到根目录)，只要用 ";"。
```
/usr/include/sys;/usr
```
会在以下位置搜索：

```
/usr/include/sys
/usr/include
/usr
```

如果你用相对路径，则向上搜索始于 Vim 的当前目录或当前文件的目录（如果相对路径始于 "./" 并且 'cpoptions' 里不含 'd' 的话）。

如果 Vim 的当前路径是 /u/user_x/work/release 而你用

```
:set path=include;/u/user_x
```

然后用 gf 搜索文件，则文件在如下位置搜索：

```
/u/user_x/work/release/include
/u/user_x/work/include
/u/user_x/include
```

3) 混合向上/向下搜索

如果 Vim 当前路径是 /u/user_x/work/release 而你用

```
set path=**;/u/user_x
```

然后用 gf 搜索文件，则文件在如下位置搜索：

```
/u/user_x/work/release/**
/u/user_x/work/**
/u/user_x/**
```

要小心！这会消耗大量时间，因为对 '/u/user_x/**' 的搜索包括 '/u/user_x/work/**' 和 '/u/user_x/work/release/**'。因此 '/u/user_x/work/release/**' 被搜索三遍而 '/u/user_x/work/**' 被搜索两遍。

因而，在上例中你可能想如此设置 path：

```
:set path=**,/u/user_x/**
```

这会搜索：

```
/u/user_x/work/release/**
/u/user_x/**
```

它会搜索相同的目录群，但顺序不同。

注意 ":find"、":sfind" 和 ":tabfind" 命令的补全目前不接受包含 URL、使用带深度限定的双星记号（/usr/**2）或向上搜索（;）记号的 'path' 项目。

vim:tw=78:ts=8:noet:ft=help:norl:

motion.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　　by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

光标移动　　cursor-motions　navigation

这些命令用于移动光标。如果新的位置离开了当前显示范围，屏幕将滚动到合适的位置，使得光标可见（参见 'scrolljump' 和 'scrolloff' 选项）。

1. 动作和操作符　　operator
2. 左右动作　　left-right-motions
3. 上下动作　　up-down-motions
4. 单词动作　　word-motions
5. 文本对象动作　　object-motions
6. 文本对象选择　　object-select
7. 位置标记　　mark-motions
8. 跳转　　jump-motions
9. 其他动作　　various-motions

概论：

如果你想知道在当前文件所在的位置，可以用 "CTRL-G" 命令　CTRL-G　或者 "g CTRL-G" 命令　g_CTRL-G 。如果你置位了 'ruler' 选项，光标位置会在状态行上保持更新（Vim 会因此稍慢一些）。

有经验的用户更喜欢 hjkl 键，因为它们就在自己的手指下面。初学者则倾向于光标移动键，因为他们不知道 hjkl 键做什么。看看键盘的布局就很容易记住 hjkl 的意义。把 j 想象成一个向下的箭头就可以了。

置位 'virtualedit' 选项使得光标可以移动到还没有字符或者半个字符的位置。

---

1. 动作和操作符　　operator

动作命令出现在操作符之后，从而使操作符作用于被该动作所跨越的文本之上。也就是，在动作之前和之后的光标位置之间的文本。一般的，操作符用来删除或者改变文本。下面列出所有的操作符：

c	c	修改（change）
d	d	删除（delete）
y	y	抽出（yank）到寄存器（不改变文本）
~	~	变换大小写（只有当 'tildeop' 置位时有效）
g~	g~	变换大小写
gu	gu	变为小写
gU	gU	变为大写
!	!	通过外部程序过滤
=	=	通过 'equalprg'（若为空，C-indenting）过滤
gq	gq	文本排版
gw	gw	文本排版，不移动光标
g?	g?	ROT13 编码

>	>	右移
<	<	左移
zf	zf	定义折叠
g@	g@	调用 'operatorfunc' 选项定义的函数

如果动作包括一个次数而操作符之前也有一个的话，两者相乘。因此，"2d3w" 删除六个单词。

大多数情况下，光标在应用操作符后停在被操作的文本的起始处。例如，"yfe" 不移动光标，而 "yFe" 则向左移动光标到抽出的文本的起始的那个 "e" 上。

linewise characterwise

操作符或者影响开始和结束位置之间的整行或者字符区间。一般说来，在行间移动的动作影响整行（或者说面向行的），而在行内移动的动作影响字符区间（或者说面向字符的）。但是有例外。

exclusive inclusive

面向字符的动作或者是闭的，或者是开的。闭动作的开始和结束位置包含在操作范围里。开动作中，靠近缓冲区尾端的最后一个字符不被包含在内。行的动作总包含开始和结尾的位置。

下面的命令里总会提到动作是行的、开的、或是闭的。不过，有两个一般的特例：

1. 如果一个开动作的结尾应在第一列，那么，它会前移到上一行的结尾处，并成为一个闭动作。例如，"}" 移动到一个段落之后的第一行。但 "d}" 不会包含那一行。

exclusive-linewise

2. 如果一个开动作的结尾应在第一列，而它的开始在一行的第一个非空白字符之前或之上，那么它会转换成一个行动作。例如，如果一个段落以若干空白开始，而你在第一个非空白字符上执行 "d}"，那么该段落的所有行都被删除，包括之前的空白。如果你再放置 (put)，那么删除的行将插在光标之下。

请 注意 如果操作符处于等待状态（键入了操作符命令但还没有键入动作），将应用一组特别的映射命令。参见 :omap 。

除了先给出操作符再给出动作的方式以外，你还可以用可视模式：先用 "v" 标记文本的开始处，移动光标到文本的末尾，然后再输入操作符。开始和当前光标位置之间的文本以高亮显示，从而你可以直观地看到要操作的文本。这种方式提供了更多的自由，但代价是更多的键击而且重做不易。参见可视模式的章节 Visual-mode 。

你可以用 ":" 命令定义一个动作。例如，"d:call FindEnd()"。
不过如果命令多于一行，该操作不能用 "." 重复。例如，可以重复：

```
d:call search("f")<CR>
```

但不能重复：

```
d:if 1<CR>
   call search("f")<CR>
endif<CR>
```

注意 ":" 的使用使任何动作变成面向字符。

forced-motion

强制一个动作面向行、面向字符或者面向可视列块

如果一个动作不是你希望使用的类型，你可以在操作符后用 "v"、"V" 或者 CTRL-V 来强制转换类型。
示例：

dj
删除两行
dvj
删除当前光标位置到光标下方字符之间字符
d<C-V>j
删除光标所在和光标下方的字符。

把面向行的动作强制为面向字符或者列块时要注意，列不一定总有定义。

v　在操作符后和动作命令之前应用：即使该动作是面向行的，也强制该操作面向字符。如果它应是面向行的，则变成开动作。如果已经是面向字符的，则在开/闭间切换。这可以使一个开动作成为闭的，或者使一个闭动作成为开的。　o_v

V　在操作符后和动作命令之前应用：即使该动作是面向字符的，也强制该操作面向行。　o_V

CTRL-V　在操作符后和动作命令之前应用：强制该动作面向列块。这类似于可视列块模式的选择，由动作之前和之后的光标位置定义两个角落。　o_CTRL-V

---

2. 左右动作　left-right-motions

这些命令移动光标到当前行的指定列。除了 "$" 之外（有可能跨越多行），它们最多停在该行的第一列或者行尾。'whichwrap' 选项可以使其中的一些命令跨越行的边界。

h 或 <Left> 或 CTRL-H 或 <BS>　h <Left> CTRL-H <BS>
向左 [count] 个字符。 exclusive 开动作。
注意：如果你希望 <BS> 删除字符，使用如下映射：
:map CTRL-V<BS> X
(要输入 "CTRL-V<BS>" 先敲 CTRL-V 键，紧跟一个 <BS> 键)
如果 <BS> 和希望的操作不符，查阅 :fixdel 。

l 或 <Right> 或 <Space>　l <Right> <Space>
向右 [count] 个字符。 exclusive 开动作。
关于调整行尾的行为，参见 'whichwrap' 选项

0　到行的第一个字符。 exclusive 开动作。　0

<Home>　到行的第一个字符。 exclusive 开动作。上下移动时，停在相同的文本列（如果可能的话），与之对照，多数其它命令则保持在相同的 屏幕 列上。和 "1|" 类似。如果一行以 <Tab> 开头，则和 "0" 有区别。　<Home> <kHome>

^　到行的第一个非空白字符。 exclusive 开动作。　^

$ 或 <End>　　$　<End>　<kEnd>
到行尾。如果给出 [count]，则先往下走 [count-1] 行。
inclusive 闭动作。
可视模式下，光标移到紧贴该行最后一个字符之后的位置。
如果置位了 'virtualedit'，"$" 可从行尾之后的空白后退到行尾。

g_　　g_
往下 [count-1] 行并到该行的最后一个非空白字符。
inclusive 闭动作。

g0 或 g<Home>　　g0　g<Home>
如果设置了行回绕（'wrap' 打开）：到屏幕行的第一个字符。
exclusive 开动作。如果一文本行不能在屏幕上完全显示，它和 "0" 就会有不同。
如果没有设置（'wrap' 关闭）：到该行屏幕显示范围的最左面的字符。如果该行第一个字符不在屏幕上，则它和 "0" 就有不同。

g^　　g^
如果设置了行回绕（'wrap' 打开）：到屏幕行的第一个非空白字符。 exclusive 开动作。如果一文本行不能在屏幕上完全显示，它和 "^" 就会有区别。
如果没有设置（'wrap' 关闭）：到该行屏幕显示范围的最左面的非空白字符。如果该行第一个字符不在屏幕上，则它和 "^" 就有区别。

gm　　gm
和 "g0" 类似，但（尽可能）向右移到屏幕显示宽度的中间位置。

gM　　gM
和 "g0" 类似，但到行文本的中间位置。
带计数时：行文本的此百分比位置。
因而，"10gM" 是接近文本的开始处，而 "90gM" 是接近文本的结尾处。

g$　或　g<End>　　g$　g<End>
如果设置了行回绕（'wrap' 打开）：往下 [count-1] 屏幕行并到该屏幕行的行尾。 inclusive 闭动作。如果一文本行不能在屏幕上完全显示，它和 "$" 就会有不同。
如果没有设置（'wrap' 关闭）：到该行屏幕显示范围的最右面的字符。如果该行的行尾字符不可见，则它和 "$" 就有不同。另外，垂直移动保持所在的列，而不是移到行尾。
打开 'virtualedit' 时移动到屏幕行尾。

|　　bar
到当前行的 [count] 屏幕列。 exclusive 开动作。
Ceci n'est pas une pipe（译者注：法语 这不是一只烟斗，这是名句，pipe 也做管道讲，有双关义）。

f{char}　　f
到右侧第 [count] 次出现的字符 {char}。光标放在 {char}

上。 inclusive 闭动作。
{char} 可以输入二合字母 digraph-arg 。
如果 'encoding' 设为 Unicode，则可以输入组合字符，参见 utf-8-char-arg 。
{char} 要进行 :lmap 映射。插入模式下的 CTRL-^ 命令可以用来切换这一点 i_CTRL-^ 。

F{char} F
到左侧第 [count] 次出现的字符 {char}。
光标放在 {char} 上。 exclusive 开动作。
{char} 可以和 f 命令相同的方式输入。

t{char} t
直到右侧第 [count] 次出现的字符 {char} 之前。光标放在 {char} 左边的位置。 inclusive 闭动作。
{char} 可以和 f 命令相同的方式输入。

T{char} T
直到左侧第 [count] 次出现的字符 {char} 之后。光标放在 {char} 右侧的位置。 exclusive 开动作。
{char} 可以和 f 命令相同的方式输入。

; ;
重复上次的 f、t、F 或者 T 命令 [count] 次。见 cpo-;

, ,
反方向重复上次的 f、t、F 或者 T 命令 [count] 次。另见 cpo-;

---

3. 上下动作 up-down-motions

k 或 <Up> 或 CTRL-P k <Up> CTRL-P
向上 [count] 行。 linewise 行动作。

j 或 <Down> 或 CTRL-J 或 <NL> 或 CTRL-N j <Down> CTRL-J <NL> CTRL-N
向下 [count] 行。 linewise 行动作。

gk 或 g<Up> gk g<Up>
向上 [count] 显示行。 exclusive 开动作。
在行回绕和操作符之后与 'k' 不同，因为后者不是面向行的。

gj 或 g<Down> gj g<Down>
向下 [count] 显示行。 exclusive 开动作。
在行回绕和操作符之后与 'j' 不同，因为后者不是面向行的。

- <minus> -
向上 [count] 行，停在第一个非空白字符上。 linewise

行动作。

+ 或 CTRL-M 或 <CR>	+ CTRL-M <CR> 向下 [count] 行，停在第一个非空白字符上。 linewise 行动作。
_ <underscore>	_ 向下 [count] - 1 行，停在第一个非空白字符上。 linewise 行动作。
G	G 到第 [count] 行，缺省是最后一行， linewise 行动作。 如果 'startofline' 没有置位，保持在相同的列上，不然，停在第一个非空白字符上。 G 是 jump-motions 之一。
<C-End>	<C-End> 到第 [count] 行，缺省是最后一行；并停在最后一个字符上。 inclusive 闭动作。
<C-Home> 或 gg	gg <C-Home> 到第 [count] 行，缺省是第一行， linewise 行动作。 如果 'startofline' 没有置位，保持在相同的列上，不然，停在第一个非空白字符上。
:[range]	:[range] 把光标移到 [range] 的最后一行。[range] 也可以是单独一个行号，如 ":1" 或 ":'m"。 和 G 不同，此命令不修改 jumplist 。
{count}%	N% 到文件的 {count} 百分比处并停在行的第一个非空白字符上， linewise 行动作。新的行号计算方法如下： ({count} * 总行数 + 99) / 100 另见 'startofline'。
:[range]go[to] [count] [count]go	:go :goto go 到缓冲区的第 [count] 个字节。缺省的 [count] 是 1，即文件开始处。如果给定 [range]，则最后的一个数字用作字节的序号。'fileformat' 的当前设置决定如何计算换行符的个数。 另见 line2byte() 函数和 'statusline' 的 'o' 选项。 {仅当编译时加入 +byte_offset 特性才有效}

这些命令移动到特定的行上，最多，它们移动到第一行或者最后一行。开始两个命令的光标将停留在最后一个改变列号命令所指定的列上（如果可能的话）。"$" 命令除外，这时，光标总会停在一行的行尾。

如果 "k"、"-" 或 CTRL-P 使用时给出 [count]，而光标上方的行数小于 [count] 且 'cpo' 选项包含了 "-" 标志位，报错 cpo-- 。

---

4. 单词动作 word-motions

<S-Right>　或　<S-Right>　w
w　正向 [count] 个单词。 exclusive 开动作。

<C-Right>　或　<C-Right>　W
W　正向 [count] 个字串。 exclusive 开动作。

e
e　正向到第 [count] 个单词的尾部。 inclusive 闭动作。
不停留在空行上。

E
E　正向到第 [count] 个字串的尾部。 inclusive 闭动作。
不停留在空行上。

<S-Left>　或　<S-Left>　b
b　反向 [count] 个单词。 exclusive 开动作。

<C-Left>　或　<C-Left>　B
B　反向 [count] 个字串。 exclusive 开动作。

ge
ge　反向到第 [count] 个单词的尾部。 inclusive 闭动作。

gE
gE　反向到第 [count] 个字串的尾部。 inclusive 闭动作。

这些命令在单词或字串间移动。

word
一个单词由字符、数字和下划线序列或者其他的非空白字符的序列组成。单词间可以空白字符（空格、制表、换行）分隔。这一规则可以用 'iskeyword' 选项改变。空行也被认作单词。

WORD
一个字串由非空白字符序列组成。字串以空白分隔。空行也被认作字串。

已折叠的行序列被认作由单个字符组成的单词。"w" 和 "W"、"e" 和 "E" 移动到折叠行之后的第一个单词或字串的开始/结尾处。"b" 和 "B" 移动到折叠之前的第一个单词的开始处。

特例：如果光标在非空白字符上，"cw" 和 "cW" 等价于 "ce" 和 "cE"。这是因为 "cw" 被诠释为 修改-单词 (change-word)，而单词并不包括后续的空格。

另外一个特例：如果 "w" 动作带操作符并且该动作的最后一个单词在行尾，则该操作范围结束于行尾而非下一行的第一个单词。

原始 Vi 的 "e" 实现有问题。例如，如果前一行为空而光标停在后一行的第一个字符的话，"e" 就会卡在那里，但是你用 "2e" 就很正常。Vim 里就不是，"ee" 和 "2e" 有相同的行为，这更合乎逻辑。不过，这造成了 Vi 和 Vim 小小的不兼容。

---

5. 文本对象动作　object-motions

(
(　反向 [count] 个句子。 exclusive 开动作。

)　　正向 [count] 个句子。 exclusive 开动作。

{　　反向 [count] 个段落。 exclusive 开动作。

}　　正向 [count] 个段落。 exclusive 开动作。

]]　　正向 [count] 个小节或到出现在首列的 '{'。如果带操作符，则同时停留在首列的 '}' 的下方。 exclusive 开动作。 注意 exclusive-linewise 常常适用。

][　　正向 [count] 个小节或到出现在首列的 '}'。 exclusive 开动作。
注意 exclusive-linewise 常常适用。

[[　　反向 [count] 个小节或到出现在首列的 '{'。 exclusive 开动作。
注意 exclusive-linewise 常常适用。

[]　　反向 [count] 个小节或到出现在首列的 '}'。 exclusive 开动作。
注意 exclusive-linewise 常常适用。

这些命令在三类文本对象上移动，见下。

sentence
一个句子以 '.'、'!' 或者 '?' 结尾并紧随着一个换行符、空格或者制表符。标点和空白字符之间可以出现任何数量的闭括号和引号: ')'、']'、'"' 和 '''。另，段落和小节的边界也视为句子的边界。
如果 'cpoptions' 包含 'J' 标志位，那么标点之后的空格至少要出现两个，而且制表符不被视为空白字符。

paragraph
一个段落从空行或某一个段落宏命令开始，段落宏由 'paragraphs' 选项里成对出现的字符所定义。它的缺省值为 "IPLPPPQPP TPHPLIPpLpItpplpipbp"，也就是宏 ".IP"、".LP" 等 (这些是 nroff 宏，所以句号一定要出现在第一列)。小节边界也被视为段落边界。
注意 空白行 (只包含空白) 不是 段落边界。
也要 注意: 这不包括首列出现的 '{' 或 '}'。如果 'cpoptions' 里包含 '{' 标志位，那么首列的 '{' 用作段落边界 posix 。

section
一个小节从首列出现的换页符 (<C-L>) 或某一个小节宏命令开始。小节宏由 'sections' 选项里成对出现的字符所定义。它的缺省值是 "SHNHH HUnhsh"，也就是说小节可以从如下的 nroff 宏开始: ".SH"、".NH"、".H"、".HU"、".nh" 和 ".sh"。

"]" 和 "[" 命令也停在首列出现的 '{' 或 '}' 上。这有助于在 C 程序里找到函数的开始和结束位置。 注意: 命令的第一个字符决定搜索的方向，第二个字符决定要找到的括

号。

如果你的 '{' 或 '}' 不在第一列但是你还是希望用 "[[" 和 "]]" 来找它们，试试这些映射：

```
:map [[ ?{<CR>w99[{
:map ][ /}<CR>b99]}
:map ]] j0[[%/{<CR>
:map [] k$][%?}<CR>
```

[照文本直接输入，参见 <>]

---

6. 文本对象选择

object-select text-objects
v_a v_i

这里是一系列只能在可视模式或操作符后使用的命令。这些命令或以 "a" 打头，代表选择一个 ("a"n) 包含空白的对象；或以 "i" 带头，代表选择内含 ("i"nner) 对象：它们不包含空白。另外，空白本身也是内含对象。这样，"内含" 对象总比 "一个" 对象选择较少的文本。

这些命令只有在编译时加入 +textobjects 特性后才有效。
另见 gn 和 gN，操作对象是前次搜索模式。

v_aw aw

aw "一个单词"，选择 [count] 个单词（见 word）。
包括开头或拖尾的空白，但不单独计算。在可视面向行的模式下，"aw" 切换到可视面向字符的模式。

v_iw iw

iw "内含单词"，选择 [count] 个单词（见 word）。
单词之间的空白也被算为一个单词。在可视面向行的模式下，"iw" 切换到可视面向字符的模式。

v_aW aW

aW "一个字串"，选择 [count] 个字串（见 WORD）。
包括开头或拖尾的空白，但不单独计算。在可视面向行的模式下，"aW" 切换到可视面向字符的模式。

v_iW iW

iW "内含字串"，选择 [count] 个字串（见 WORD）。
字串之间的空白也被算为一个字串。在可视面向行的模式下，"iW" 切换到可视面向字符的模式。

v_as as

as "一个句子"，选择 [count] 个句子（见 sentence）。
可视模式下它切换为面向字符的模式。

v_is is

is "内含句子"，选择 [count] 个句子（见 sentence）。
可视模式下它切换为面向字符的模式。

v_ap ap

ap "一个段落"，选择 [count] 个段落（见 paragraph）。
特例：空白行（只包含空白的行）也被视为段落边界。
可视模式下它切换为面向行的模式。

v_ip ip

ip "内含段落"，选择 [count] 个段落（见 paragraph ）。
特例：空白行（只包含空白的行）也被视为段落边界。
可视模式下它切换为面向行的模式。

v_a] v_a[a] a[

a] "一个 [] 块"，选择 [count] 层 '[' ']' 块。为此，先反向
a[查找第 [count] 个未匹配的 '['，然后查找其相应的 ']'。
两者之间的文本，包括 '[' 和 ']'，都被选择。
可视模式下它切换为面向字符的模式。

v_i] v_i[i] i[

i] "内含 [] 块"，选择 [count] 层 '[' ']' 块。为此，先反向
i[查找第 [count] 个未匹配的 '['，然后查找其相应的 ']'。
两者之间的文本，但不包括 '[' 和 ']'，被选择。
可视模式下它切换为面向字符的模式。

v_a) a) a(

a) vab v_ab v_a(ab
a(
ab "一个块"，选择 [count] 层块，从 "[count] [(" 到其相应
的 ')'，包括 '(' 和 ')'（见 [(）。它并不包括括号之
外的空白。
可视模式下它切换为面向字符的模式。

v_i) i) i(

i) vib v_ib v_i(ib
i(
ib "内含块"，选择 [count] 层块，从 "[count] [(" 到其相应
的 ')'，但不包括 '(' 和 ')'（见 [(）。
可视模式下它切换为面向字符的模式。

v_a> v_a< a> a<

a> "一个 <> 块"，选择 [count] 层 <> 块，从反向第 [count]
a< 个未匹配的 '<' 到其匹配的 '>'，包括 '<' 和 '>'。
可视模式下它切换为面向字符的模式。

v_i> v_i< i> i<

i> "内含 <> 块"，选择 [count] 层 <> 块，从反向第 [count]
i< 个未匹配的 '<' 到其匹配的 '>'，但不包括 '<' 和 '>'。
可视模式下它切换为面向字符的模式。

v_at at

at "一个标签块"，选择 [count] 层标签块。从反向第 [count]
个未匹配的 "<aaa>" 到其匹配的 "</aaa>"，包括 "<aaa>"
和 "</aaa>"。
详情见 tag-blocks 。
可视模式下它切换为面向字符的模式。

v_it it

it "内部标签块"，选择 [count] 层标签块。从反向第 [count]
个未匹配的 "<aaa>" 到其匹配的 "</aaa>"，但不包括
"<aaa>" 和 "</aaa>"。
详情见 tag-blocks 。
可视模式下它切换为面向字符的模式。

a}　　v_a}　a}　a{
a{　　v_aB　v_a{　aB
aB　"一个大块"，选择 [count] 层大块，从 "[count] [{" 到其相应的 '}'，包括 '{' 和 '}'（见 [{ ）。
可视模式下它切换为面向字符的模式。

i}　　v_i}　i}　i{
i{　　v_iB　v_i{　iB
iB　"内含大块"，选择 [count] 层大块，从 "[count] [{" 到其相应的 '}'，但不包括 '{' 和 '}'（见 [{ ）。
可视模式下它切换为面向字符的模式。

a"　　v_aquote　aquote
a'　　v_a'　a'
a`　　v_a`　a`
"一个引号字符串"。选择上一个引号开始到下一个引号结束的文本。'quoteescape' 选项用于跳过转义的引号。
只在同一行内有效。
如果开始时光标在引号上，Vim 会从该行行首开始搜索，以决定哪个引号对构成字符串。
包含任何拖尾的空白，如果没有拖尾的，也包含开头的空白。
可视模式下它切换为面向字符的模式。
在可视模式下重复此对象会包含另一个字符串。目前不使用计数。

i"　　v_iquote　iquote
i'　　v_i'　i'
i`　　v_i`　i`
类似于 a"、a' 和 a`，但不包括引号，而重复也不会扩展可视选择区。
特例：计数为 2 时包含引号，但不包括 a"/a'/a` 包含的额外的空白。

在操作符之后：
非块对象：
对于 "一个" 命令：操作符作用于对象与其后的空白。如果其后没有空白或者光标位于对象之前的空白上的话，那么也包括对象之前的空白。
对于 "内含" 命令：如果光标在对象之上，那么操作符作用于该对象。如果光标在空白上，那么操作符作用于空白。
块对象：
操作符作用于光标所在位置所在（包括光标在括号上的特殊情况）的块。对于 "内含" 命令，不包含包围的括号。而对于 "一个" 命令，则包含之。

在可视模式下：
如果可视区域的起始和结束点在相同位置（刚输入 "v"）：
选择一个区域，就和使用操作符一样。
如果可视区域的起始和结束点不同：
若不是块对象，该区域被对象或者下一个对象之前空白所扩展，对于 "一个" 命令，则两者都包含。扩展的方向决定于可视区域和光标的相对位置。
若是块对象，该块向外扩展一层。

让我们用一系列删除命令来说明，删除的范围从小到大。请 注意 对于单个字符和整行的操作我们用了已有的 Vi 移动命令。
"dl"　删除字符（缩写："x"）　dl

"diw"	删除内含单词	diw
"daw"	删除一个单词	daw
"diW"	删除内含字串（见 WORD ）	diW
"daW"	删除一个字串（见 WORD ）	daW
"dgn"	删除下一个前次搜索模式的匹配	dgn
"dd"	删除一行	dd
"dis"	删除内含句子	dis
"das"	删除一个句子	das
"dib"	删除内含 '(' ')' 块	dib
"dab"	删除一个 '(' ')' 块	dab
"dip"	删除内含段落	dip
"dap"	删除一个段落	dap
"diB"	删除内含 '{' '}' 大块	diB
"daB"	删除一个 '{' '}' 大块	daB

请 注意 移动命令和文本对象的区别。移动命令作用于这里（光标当前位置）到移动后的位置。而对象的使用则会作用于整个对象，而和光标在对象的何处无关。例如，我们可以比较 "dw" 和 "daw": "dw" 删除光标位置到下一个单词的起始处，"daw" 删除光标所在的整个单词和其后或其前的空白。

标签块 tag-blocks

"it" 和 "at" 文本对象尝试选择 HTML 和 XML 的匹配标签之间的块。但因为它们并不完全兼容，有一些限制。

通常的方法是选择 <tag> 直到匹配的 </tag> 为止的内容。"at" 包含标签，"it" 不包含。不过重复 "it" 时标签仍然会包含（不然不会有任何改变）。此外，在没有内容的标签块上使用 "it" 会单独选择引导标签。

跳过 "<aaa/>" 项目。忽略大小写，即使对大小写应该敏感的 XML 也是如此。

HTML 里可以有
 或 <meta ...> 这样没有匹配结束标签的标签。它们被忽略。

这些文本对象能够容忍错误。单独出现的结束标签被忽略。

---

7. 位置标记 mark-motions E20 E78

跳转到一个位置标记有两种方法：
1. 用 ` (反引号)： 光标放在指定的位置， exclusive 开动作。
2. 用 ' (单引号)： 光标放在指定位置所在行的第一个非空字符上， linewise 行动作。

m m mark Mark

m{a-zA-Z} 把位置标记 {a-zA-Z} 设在当前光标位置（不移动光标，这不是动作命令）。

m' m`

m' 或 m` 设置前次上下文标记。以后可以用 "''" 或者 "``" 跳转到这个位置（不移动光标，这不是动作命令）。

m[m]

m[或 m] 设置 '[或者 '] 标记。可以用在执行多个命令的操作符

的定义中。（不移动光标，这不是动作命令）。

m< m>

m< 或 m> 设置 '< 或者 '> 标记。可以用在改变 gv 命令选择的范围。（不移动光标，这不是动作命令）。
注意 不能设置可视模式，只能设置开始和结束的位置。

:ma :mark E191

:[range]ma[rk] {a-zA-Z'}
把位置标记 {a-zA-Z'} 设在 [range] 的最后一行，第 0 列。缺省的 [range] 是当前行。

:k

:[range]k{a-zA-Z'} 和 :mark 相同，但是标记名之前的空格可以省略。

' 'a ` `a

'{a-z} `{a-z} 跳转到当前缓冲区的位置标记 {a-z}。

'A '0 `A `0

'{A-Z0-9} `{A-Z0-9} 跳转到设置位置标记 {A-Z0-9} 的文件所在的标记位置（如果切换到另外一个文件，这就不是动作命令）

g' g'a g` g`a

g'{mark} g`{mark}
跳转到指定的位置标记 {mark}，但在当前缓冲区内跳转时，不改变跳转表。示例：

```
g`"
```

跳转到当前文件最近的位置。参看 $VIMRUNTIME/vimrc_example.vim。另见 :keepjumps 。

:marks

:marks 列出所有的位置标记（这不是动作命令），但不包括 '(，') ，'{ 和 '} 标记。第一列的编号为零。

E283

:marks {arg} 列出所有 {arg} 包含的位置标记（这不是动作命令）。例如，

```
:marks aB
```

列出位置标记 'a' 和 'B'。

:delm :delmarks

:delm[arks] {marks} 删除指定的位置标记。可以删除的位置标记也包括 A-Z 或 0-9。不能删除 ' 位置标记。
指定的方式包括给出位置标记名的列表和使用连字符分隔的范围。忽略空格。例如：

```
:delmarks a         删除位置标记 a
:delmarks a b 1     删除位置标记 a，b 和 1
:delmarks Aa        删除位置标记 A 和 a
:delmarks p-z       删除位置标记 p 到 z
:delmarks ^.[]      删除位置标记 ^ . [ ]
:delmarks \"        删除位置标记 "
```

:delm[arks]! 删除当前缓冲区所有的位置标记，不包括 A-Z 或 0-9 位置标记。

位置标记在任何情况下都是不可见的。它只是文件中一个被记住的位置。不要和命名的寄存器混淆，两者毫不相干。

'a - 'z	小写位置标记，在每个文件内有效。
'A - 'Z	大写位置标记，也叫做文件标记，在文件间都有效。
'0 - '9	数字位置标记，在 .viminfo 文件里设置。

只要文件还在缓冲区列表里，小写位置标记 'a 到 'z 就被记住。换而言之，如果在缓冲区列表里删除一个文件，它的位置标记就消失了。如果删除包含某个位置标记的文本行，这个位置标记也就随之消失。

小写位置标记可以和操作符合并使用。例如，"d't" 删除从光标位置到包含 't 标记的文本行。提示：用 't' 标记代表顶部 (Top)，'b' 标记代表底部 (Bottom)，等等。小写位置标记在撤销/重做时会被复原。

大写位置标记 'A 到 'Z 包含了所在的文件名。可以用这些标记在文件间跳转。要和操作符合并使用的大写位置标记必须在当前文件里。即使插入/删除一些行或者同时编辑别的文件，这种标记的行号总是正确的。如果 'viminfo' 选项不为空，大写位置标记由 .viminfo 文件保存。 参阅 viminfo-file-marks 。

数字位置标记 '0 到 '9 很不一样。它们不是直接设置，而只能从 viminfo 文件 viminfo-file 中获取。简单的说，'0 是你上次离开 Vim 时的光标位置。'1 是再上一次，等等。'viminfo' 的 'r' 标志可以指定不记录数字位置标记的文件。参见 viminfo-file-marks 。

'[`[到上次改变或者抽出的文本的第一个字符。

'] `] 到上次改变或者抽出文本的最后一个字符。

执行完一个操作符后，光标放在操作文本的开始。执行完一个放置命令 ("p" 或者 "P")，光标有时放在第一个被插入的位置，有时放在最后一个。上述四个命令可以把光标放在两端的任何一端。例如，在抽出 10 行后，你想要到跳转到最后一行：`"10Y']"`。在用 "p" 插入数行后你想跳到最底下的那行：`"p']"`。这对插入的文本也可以。

请 注意：在删除文本后，除非是面向列块的可视模式，开始和结束的位置是重合的。而如果没有任何改变，这四个命令不会有任何作用。

'< `< 到上次当前缓冲区选择的可视区域首行或第一个字符。对于列块模式而言，可能也是第一行的最后一个字符 (为了能定义列块)。

'> `> 到上次当前缓冲区选择的可视区域末行或最后一个字符。对于列块模式而言，可能也是末行的第一个字符 (为了能定义列块)。注意 这里适用 'selection'，该位置可能是刚刚在可视区域之后的位置。

'' `` 到上次跳转的位置，或者上次 "m'" 或者 "m`" 命令给出的位

置。如果使用了 :keepjumps 命令修饰符，这个前次上下文标记不会被改变。也可参考 restore-position 。

'quote `quote

'" `" 到上次离开本缓冲区时的光标位置。缺省是第一行的第一个字符。 参见 last-position-jump 以了解如何对所有刚打开的文件使用这个标记。每个缓冲区，而不是每个窗口，只有一个记住的位置。只要这个缓冲区在某一个窗口可见，这个标记是不会改变的。

'^ `^ 到上次插入模式关闭的光标位置。 gi 命令用到这个标记。如果使用了 :keepjumps 命令修饰符，这个标记不会改变。

'. `. 到上次改变的位置。这个位置位于或者接近于改变开始的地方。有些命令引起几个改变，那么这个位置应该是接近于这个命令最后改变的地方。例如，插入一个单词的时候，这个位置是在最后一个的字符上。
要跳转到较旧的改变，用 g; 。

'(`(到当前句子的开始处，就像 (命令。

') `) 到当前句子的结尾处，就像) 命令。

'{ `{ 到当前段落的开始处，就像 { 命令。

'} `} 到当前段落的结尾处，就像 } 命令。

这些命令不是标记本身，而是实现对某个标记的跳转：

]' [count] 次到当前行之后下一个包含小写位置标记的行的第一个非空白字符。

]` [count] 次下一个小写位置标记。

[' [count] 次到当前行之前上一个包含小写位置标记的行的第一个非空白字符。

[` [count] 次上一个小写位置标记。

:loc[kmarks] {command} :loc :lock :lockmarks
执行命令 {command}，并且不调整位置标记。这可以用于不影响行的数目的文本改变。如果行数发生变化，那么改变之后的

标记将仍然保持它们的行号，从而实际上移到了别的行上。

以下各项对于插入/删除行不会被调整：
- 小写位置标记 'a - 'z
- 大写位置标记 'A - 'Z
- 数字位置标记 '0 - '9
- 上次插入位置 '^
- 上次改变位置 '.
- 上次影响的文本区域 '[和 ']
- 可视选择区域 '< 和 '>
- 已设定的标号 signs 的行号
- quickfix 位置的行号
- 跳转表 jumplist 里的位置
- 标签堆栈 tagstack 里的位置

以下各项则总会被调整：
- 前次上下文标记 ''
- 光标位置
- 窗口或者缓冲区的视图
- 折叠 folding
- 差异视图 diff

:kee[pmarks] {command} :kee :keep :keepmarks

目前，这只对过滤命令 :range! 有效：
- 如果过滤后的行数不少于过滤之前，那么所有的过滤行中的标记保持行号不变。
- 如果行数减少，那么消失了的行中的标记就被删除掉。

在任何情况下，过滤文本之后的标记需要调整行号，以保持和文本同步。

如果 'cpoptions' 中没有 'R' 标志，那么直接执行过滤命令和使用 ":keepmarks" 的效果一样。

:keepj :keepjumps

:keepj[umps] {command}

在 {command} 的移动不改变 '' ， '. 和 '^ 标记，跳转表 jumplist 或是改变表 changelist 。这对于自动改变或者插入文本而不需要用户跳转到那个位置有用。例如，当更新 "最近改变" 的时间标签：

```
:let lnum = line(".")
:keepjumps normal gg
:call SetLastChange()
:keepjumps exe "normal " . lnum . "G"
```

注意 ":keepjumps" 必须在每个命令里使用。如果在命令里调用了函数，则该函数里的命令仍然可能改变跳转表。另外，":keepjumps exe 'command '" 里的 "command" 不会保护跳转表。应该用：":exe 'keepjumps command'"

---

8. 跳转 jump-motions

"跳转" 是通常能移动光标数行之外的命令。如果你使光标 "跳转"，那么跳转之前的光标位置会被记住。除非包含该位置的行被改变或者删除，你可以用 "''" 和 "``" 命令返回这个位置。以下命令是 "跳转" 命令："'"、"`"、"G"、"/"、"?"、"n"、"N"、"%"、

"("、")"、 "[["、"]]"、"{"、"}"、":s"、":tag"、"L"、"M"、"H" 和开始编辑新文件的命令。

CTRL-O

CTRL-O 转到跳转表里第 [count] 个较旧的光标位置(不是动作命令)。
{仅当编译时加入 +jumplist 特性才有效}

<Tab> 或 CTRL-I <Tab>

CTRL-I 转到跳转表里第 [count] 个较新的光标位置(不是动作命令)。
{仅当编译时加入 +jumplist 特性才有效}

:ju :jumps

:ju[mps] 打出跳转表(不是动作命令)。
{仅当编译时加入 +jumplist 特性才有效}

:cle :clearjumps

:cle[arjumps] 清除当前窗口的跳转表。
{仅当编译时加入 +jumplist 特性才有效}

jumplist

跳转表用来记住跳转的位置。用 CTRL-O 和 CTRL-I 命令，你可以跳回到较早前的跳转位置，然后在跳回来。这样，你就可以在这个列表上下移动。每个窗口有独立的跳转表，每个表最多能存储 100 项。
{仅当编译时加入 +jumplist 特性才有效}

例如，在三个跳转命令之后你有如下的跳转表:

```
  jump line  col file/text
    3     1    0 some text
    2    70    0 another line
    1  1154   23 end.
>
```

"file/text" 列显示文件名，如果是当前文件，则显示跳转所在的文本(为了能在窗口里显示，去掉开头的缩排空白并截断过长的行)。

你现在位于第 1167 行。如果你用 CTRL-O 命令，光标会到第 1154 行。结果如下:

```
  jump line  col file/text
    2     1    0 some text
    1    70    0 another line
>   0  1154   23 end.
    1  1167    0 foo bar
```

指针会指向上一次跳转的位置。下一个 CTRL-O 命令会使用更上面的那项。而下一个 CTRL-I 命令则会使用下面的那项。如果指针在最后一项之下，那说明你还没用过 CTRL-I 或 CTRL-O 命令。此时，CTRL-O 命令会把光标位置加到跳转表里，这样你以后就可以回到用 CTRL-O 之前的位置。在本例中，这是第 1167 行。

更多的 CTRL-O 命令会使你分别到第 70 和第 1 行。如果你用 CTRL-I，你又可以回到第 1154 和第 1167 行。 注意: "jump" 列的数字指示你用 CTRL-O 或 CTRL-I 到该位置所需的次数。

如果你用跳转命令，当前的行号被插到跳转表的最后。如果相同的行已经在跳转表里，那会被删除。结果是，CTRL-O 就会直接回到该行之前的位置。

如果用了 :keepjumps 命令修饰符，跳转就不会被保存在跳转表里。一些其它场合也不保存跳转，例如在 :global 命令的里面。你可以用 "m'" 设置 ' 位置标记来显式加入跳转。注意 调用 setpos() 做不到这一点。

在 CTRL-O 命令之后，你到了第 1154 行。如果你给出另外一个跳转命令（比如，"G"），那么跳转表就会成为：

```
  jump line  col file/text
    4     1    0 一些文字
    3    70    0 另外一行
    2  1167    0 foo bar
    1  1154   23 end.
>
```

删除和插入行以后，行号会得到调整。不过，如果你不保存文件而放弃编辑，例如 ":n!"，该调整会失败。

如果你分割一个窗口，跳转表会复制到新的窗口里。

如果在 'viminfo' 选项里有 ' 项，跳转表会保存在 viminfo 文件里。这样，重新启动 Vim 就会恢复跳转表。

改变表跳转 changelist change-list-jumps E664

在改变文本以后，光标的位置被记住。每个改变都会记住一个位置，从而可以使该操作能被撤销。除非，这个位置和上一个改变很接近。可以用两个命令跳转到改变所在的位置，包括那些已被撤销的：

g; g; E662
转到改变表里第 [count] 个较旧的位置。
如果 [count] 比所有的较旧的改变位置都多，回到最老的那个。
如果没有较旧的改变，给出一个错误消息。
(不是动作命令)
{仅当编译时加入 +jumplist 特性才有效}

g, g, E663
转到改变表里第 [count] 个较新的位置。
和 g; 类似，但朝向相反的方向。
(不是动作命令)
{仅当编译时加入 +jumplist 特性才有效}

使用次数的时候你向后或向前跳转到尽可能接近该次数的位置。这样，你可以用 "999g;" 跳到还能记住的最初的改变所在的位置。改变的最大项目数和跳转表 jumplist 一致。

当两个可以撤销的改变在同一行，并且所在的列差小于 'textwidth'，只有后一个会被记住。这可以避免同一行里一系列很小的改变，例如 "xxxxx"，在改变表里占据很多位置。如果 'textwidth' 为 0，则使用 'wrapmargin'。如果它也没有设置，就用固定的数目 79。细节：考虑到速度的影响，计算用的是字节数而不是字符数（这只对多字节编码有意

义)。

请 注意 如果后来有插入和删除文本，尤其删除整行的时候，光标位置和当时改变的位置可能稍有不同。

如果用到 :keepjumps 命令修饰符，改变的位置不会被记住。

:changes

:changes

打出改变表。 ">" 字符指示当前的位置。在一个改变之后，它在最新的一项之后，这意味着 g; 会带你到最新一项的位置。第一列则指示要到该位置需要的次数。例如:

```
change line  col text
    3     9    8 bla bla bla
    2    11   57 foo is a bar
    1    14   54 最新改变的行
```

'3g;' 命令会带你到第 9 行。这时，':changes' 的结果是:

```
change line  col text ~
>   0     9    8 bla bla bla
    1    11   57 foo is a bar
    2    14   54 最新改变的行
```

现在你可以用 "g," 到第 11 行，"2g," 到第 14 行。

---

9. 其他动作

various-motions

%

%

找到本行的光标所在或其后的下一个项目，并跳转到它的匹配。
inclusive 闭动作。
项目可以是:

([{}])	小括号或者 (花/方) 括号。 (这可以用 'matchpairs' 选项改变)
/* */	C-风格的注释的开始或结尾
#if、#ifdef、#else、#elif、#endif	C 预处理条件宏 (光标在 # 上，或者其后没有 ([{ 的时候)

要匹配其它的项目可以用 matchit 插件，参见 matchit-install 。此插件也可用于跳过注释中的匹配。

在 'cpoptions' 包含 "M" 时， cpo-M 括号之前的反斜杠被忽略。若不然，那么括号之前的反斜杠的数目很重要。带偶数数目的不能匹配带奇数数目的。这样，在 "(\))" 和 "\((\)" 里，第一个和最后一个小括号互相匹配。

如果 'cpoptions' 里没有 '%' 字符 cpo-% ，那么双引号里的括号被忽略，除非一行里的括号数目不对称，而且该行和前一行不以反斜杠结尾。'('、'{'、'['、']'、'}' 和 ')' 也被忽略 (单引号里的括号)。 注意 这对 C 适用，但对 Perl

就不行。Perl 里单引号用来括起字符串。

对注释中的匹配并无特殊处理。可以用 matchit 插件
matchit-install 或者用引号括起匹配。

这里不能用计数，{count}% 跳转到文件里百分之 {count} 的
那行 N% 。
在 #if/#else/#endif 上用 % 使该动作面向行。

[(
[(反向第 [count] 个的未匹配的 '('。
exclusive 开动作。

[{
[{ 反向第 [count] 个的未匹配的 '{'。
exclusive 开动作。

])
]) 正向第 [count] 个的未匹配的 ')'。
exclusive 开动作。

]}
]} 正向第 [count] 个的未匹配的 '}'。
exclusive 开动作。

以上四个命令用于转到当前代码块的开始或者结尾位置。这和在 '('、')'、'{' 或 '}'
上用 "%" 类似，但你可以在代码块的任何位置这么做，这对 C 程序很有用。例如：在
"case x:" 上用 "[{" 会把你带回到 switch 语句上。

]m
]m 正向第 [count] 个方法 (method) 的开始处 (适用于 Java
或类似结构的语言)。如果不在某个方法开始处之前，则跳转
到类的开始或结束处。如果光标之后已无 '{'，会有一个错
误。
exclusive 开动作。

]M
]M 正向第 [count] 个方法的结束处 (适用于 Java 或类似结构
的语言)。如果不在某个方法开始处之前，则跳转到类的开始
或结束处。如果光标之后已无 '}'，会有一个错误。
exclusive 开动作。

[m
[m 反向第 [count] 个方法的开始处 (适用于 Java 或类似结构
的语言)。如果不在某个方法开始处之前，则跳转到类的开始
或结束处。如果光标之前已无 '{'，会有一个错误。
exclusive 开动作。

[M
[M 反向第 [count] 个方法的结束处 (适用于 Java 或类似结构
的语言)。如果不在某个方法开始处之前，则跳转到类的开始
或结束处。如果光标之前已无 '}'，会有一个错误。
exclusive 开动作。

以上这些命令假设文件包含一个有方法的类定义。类定义用 '{' 和 '}' 包围，而方法定
义亦然。Java 语言便是如此。源文件看起来应像这样：

```
// 注释
class foo {
        int method_one() {
                body_one();
        }
        int method_two() {
                body_two();
        }
}
```

开始，光标在 "body_two()" 上，用 "[m" 会跳转到 "method_two()" 开始的那个 '{' (显然，如果方法定义很长，这就更有用！)。用 "2[m" 会跳转到 "method_one()" 的开始处。用 "3[m" 会跳转到类的开始处。

[#

[# 反向第 [count] 个未匹配的 "#if" 或 "#else"。 exclusive 开动作。

]#

]# 正向第 [count] 个未匹配的 "#else" 或 "#endif"。 exclusive 开动作。

这两个命令对有 #if/#else/#endif 结构的 C 程序很有用。它们把你带到当前行所在的 #if/#else/#endif 的开始或结束处。然后，你可以用 "%" 找到相匹配的行。

[star [/

[* 或 [/ 反向第 [count] 个 C 注释的开始 "/*"。 exclusive 开动作。

]star]/

]* 或]/ 正向第 [count] 个 C 注释的结束 "*/"。 exclusive 开动作。

H

H 到窗口从顶部 (Home) 算第 [count] 行 (缺省: 窗口的第一行) 并停在第一个非空白字符上。 linewise 行动作。参见 'startofline' 选项。光标还要根据 'scrolloff' 调整，除非某个操作符处于等待状态，那时文本可能会滚动。例如 "yH" 从第一个可见行抽起，直到光标所在行为止 (闭区间)。

M

M 到窗口的中间 (Middle) 一行并停在第一个非空白字符。 linewise 行动作。参见 'startofline' 选项。

L

L 到窗口从底部 (Last) 算第 [count] 行 (缺省: 窗口的最后一行) 并停在第一个非空白字符上。 linewise 行动作。参见 'startofline' 选项。光标还要根据 'scrolloff' 调整，除非某个操作符处于等待状态，那时文本可能会滚动。例如 "yL" 从光标所在行抽起，直到最末的可见行为止 (包含)。

<LeftMouse> 到屏幕上鼠标点击的位置。 exclusive 开动作。参见 <LeftMouse> 。如果鼠标在状态行上，则所属的窗口被激活但光标位置不改变。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar

译者：iCrazy <icrazy@ustc.edu>，tocer http://vimcdoc.sf.net

滚屏 scrolling

这些命令是用来移动窗口中的内容的。如果光标的位置移出了窗口，那么它会带动窗口的内容移动几行（行数由 'scrolloff' 选项指定），使光标回到窗口里。这里，我们定义一页为窗口的行数减 2。下面这些命令的助记符甚易混淆。请记住这些命令所指的是视窗（缓冲区中您可以看到的那一部分）在缓冲区里向上移动还是向下移动。当窗口在缓冲区里向上移动的时候，窗口中的文字朝您屏幕的下方移动。

请参阅用户手册的第 03.7 节，那里有一些介绍。

1. 向下滚屏 scroll-down
2. 向上滚屏 scroll-up
3. 相对于光标滚屏 scroll-cursor
4. 水平滚动 scroll-horizontal
5. 同步滚屏 scroll-binding
6. 用鼠标滚轮滚屏 scroll-mouse-wheel

---

1. 向下滚屏 scroll-down

下面的这些命令使编辑视窗（缓冲区中您可以看到的那一部分）向下移动（这样您就可以看到缓冲区更多下方的文本行）：

CTRL-E 使窗口在缓冲区中向下滚动 [count] 行。
文本朝屏幕上方移动。
助记：额外（Extra）的行。

CTRL-D 使窗口在缓冲区中向下滚动。滚动的行数由 'scroll' 选项指定（默认设置为半个屏幕）。如果指定了 [count]，则会先把 'scroll' 选项设为 [count]。同时，光标试图在文件中向下移动相同的行数（有行回绕或者碰到文件结束时行数会有差异）。当光标处于缓冲区最后一行时，这个命令无效，并且系统会发出一声蜂鸣。
参看 'startofline' 选项。
助记：Downwards。

<S-Down> 或 <PageDown> 或 CTRL-F 使窗口在缓冲区中向下滚动 [count] 页。
参看 'startofline' 选项。
如果只有一个窗口，可能使用 'window' 选项的值。
助记：Forwards。

z+ 不指定 [count]：重绘，在窗口中显示下一页的内容（使当前

窗口之下的一行显示在窗口最上方)。同时把光标放在那一行第一个非空白字符上。
指定 [count]：和 "z<CR>" 一样。

2．向上滚屏 scroll-up

下面的这些命令使编辑视窗（缓冲区中您可以看到的那一部分）向上移动（这样您就可以看到缓冲区更多上方的文本行）：

CTRL-Y

CTRL-Y 使窗口在缓冲区中向上滚动 [count] 行。
文本朝屏幕下方移动。
备注：如果您使用 MS-Windows，组合键 CTRL-Y 会被重新映射为重做功能。

CTRL-U

CTRL-U 使窗口在缓冲区中向上滚动。滚动的行数由 'scroll' 选项指定（默认设置为半个屏幕）。如果指定了 [count]，则会先把 'scroll' 选项设为 [count]。同时，光标试图在文件中向上移动相同的行数（有行回绕或者碰到文件结束时（译者注：当为文件开始之误）行数会有差异）。当光标处于缓冲区第一行时，这个命令无效，并且系统会发出一声蜂鸣。
参看 'startofline' 选项。
助记：Upwards。

<S-Up> 或 <PageUp> 或 CTRL-B

<S-Up> <kPageUp> <PageUp> CTRL-B

使窗口在缓冲区中向上滚动 [count] 页。
参看 'startofline' 选项。
如果只有一个窗口，可能使用 'window' 选项的值。
助记：Backwards。

z^

不指定 [count]：重绘，在窗口中显示上一页的内容（使当前窗口之上的一行显示在窗口最下方)。同时把光标放在那一行第一个非空白字符上。
指定 [count]：先滚动文本，使得缓冲区第 [count] 行显示在窗口底部，然后重绘，使得当时处于窗口顶端的文本显示在窗口的底部。同时把光标放在那一行第一个非空白字符上。

3．相对于光标滚屏 scroll-cursor

以下的命令重新定位编辑视窗（缓冲区中您可以看到的那一部分），但仍维持光标留在当前行。注意 'scrolloff' 选项可用使得上下文行在光标上下显示。

z<CR>

z<CR> 重绘，使第 [count] 行显示在窗口顶端（默认为光标所在的行)。光标停留在那一行第一个非空白字符上。

zt

zt 类似 "z<CR>"，不过光标停留在相同的列上。

zN<CR>

z{height}<CR>　重绘，使得窗口有 {height} 行高。当屏幕刷新速度很慢的时候，这对减少行数很有用。不能使窗口的高度超过屏幕的实际高度。

z.

重绘，使第 [count] 行显示在窗口中部（默认为光标所在的行）。光标停留在那一行第一个非空白字符上。

zz

类似 "z."，不过光标停留在相同的列上。注意：如果打开了 caps-lock，这个命令就变成了 "ZZ"：写缓冲区并退出！

z-

重绘，使第 [count] 行显示在窗口底部（默认为光标所在的行）。光标停留在那一行第一个非空白字符上。

zb

类似 "z-"，不过光标停留在相同的列上。

---

4. 水平滚动　scroll-horizontal

使用以下四个命令时，鼠标会跟随屏幕而移动。如果光标移出了屏幕，它就会落在屏幕中距它最近的那个字符上。忽略 'sidescroll' 选项的值。

z<Right>　或　zl　z<Right>

使文本视图向右移动 [count] 个字符，也就是使文本向左滚动 [count] 个字符。仅当 'wrap' 为关时有效。

z<Left>　或　zh　z<Left>

使文本视图向左移动 [count] 个字符，也就是使文本向右滚动 [count] 个字符。仅当 'wrap' 为关时有效。

zL

使文本视图向右移动半个屏幕宽度，也就是使文本向左滚动半个屏幕宽度。仅当 'wrap' 为关时有效。

zH

使文本视图向左移动半个屏幕宽度，也就是使文本向右滚动半个屏幕宽度。仅当 'wrap' 为关时有效。

以下两个命令仅仅在屏幕上滚动文本，光标不会移动。

zs

水平滚动文本，使光标置于屏幕的开始处（最左侧）。仅当 'wrap' 为关时有效。

ze

水平滚动文本，使光标置于屏幕的结束处（最右侧）。仅当 'wrap' 为关时有效。

---

5. 同步滚屏 scroll-binding

有时候，我们希望将两个或多个窗口绑在一起，这样，在一个窗口滚屏时，其他的窗口都会跟着一起滚屏。在 Vim 中，可以通过设置针对不同窗口的 'scrollbind' 选项来达到这个目的。当一个置位 'scrollbind' 选项的窗口滚屏时，其它置位 'scrollbind' 选项的窗口会同步滚屏（如果可能的话）。选项 'scrollbind' 的行为通过 'scrollopt' 选项可以修改。

使用滚动条时进行滚动时，只有带焦点的窗口（光标所在的窗口）才会执行绑定。因而，不用复位选项，您也可以在一段时间内避免滚动绑定的发生。

如果一个窗口还置位了 'diff' 选项，则滚屏绑定利用 2 个缓冲区中的差异来精确同步它们的位置。否则，将会使用下面的方式：

scrollbind-relative

每一个置位了 'scrollbind' 的窗口都会记录它的 "相对偏移"，可以把它看作当前窗口和另一个窗口垂直滚屏位置上的差异。当一个置位了 'scrollbind' 的窗口被要求进行超越其文本的开始和结束极限的滚屏时，这个窗口不再继续滚屏，但是它会记住超过极限的数量。窗口记住了这些信息是为了将来能维持同样的相对偏移，不管是否曾经被要求过进行超过其极限的滚屏。

但是，当一个置位了 'scrollbind' 且具有一个超过其缓冲区极限的相对偏移的窗口获得了光标焦点的时候，其他置位了 'scrollbind' 的窗口就必须根据当前窗口的相对偏移，跳转到相应的位置。这种行为可以通过清除 'scrollopt' 选项中的 "jump" 标志位来改变。

syncbind :syncbind :sync

:syncbind	强制所有 'scrollbind' 窗口具有相同的相对偏移。即：当一个窗口滚屏到了它缓冲区的顶部时，所有的 'scrollbind' 窗口都会滚屏到它们缓冲区的顶部。

scrollbind-quickadj

当使用键盘命令来垂直的滚动一个窗口，或用垂直滚动条来滚动一个具有光标焦点的窗口时，'scrollbind' 标志是起作用的。但是，当用垂直滚动条来滚动一个不具有光标焦点的窗口时，'scrollbind' 标志被忽略。这样就可以对 'scrollbind' 窗口的相对偏移量进行快速的调整。

---

6. 用鼠标滚轮滚屏 scroll-mouse-wheel

当您的鼠标有一个滚轮时，它在 GUI 方式下的 Vim 中应该可以正常工作。它怎么工作取决于您的系统。它在 xterm 中也可能可以工作，参见： xterm-mouse-wheel 。缺省只支持垂直滚轮，但有些 GUI 也支持横向滚轮。

Win32 GUI 的滚动功能是通过硬编码实现的。它的效果就像在拖曳当前窗口的滚动条。具体滚多少行取决于您的鼠标驱动程序。如果滚屏功能造成了输入焦点的问题，请看： intellimouse-wheel-problems 。

而对 X11 GUI（Motif、Athena 和 GTK）而言，滚动滚轮产生 <ScrollWheelDown>、<ScrollWheelUp>、<ScrollWheelLeft> 和 <ScrollWheelRight> 的按键动作。这些键的
默认行为如下：

<ScrollWheelUp>	向上滚动三行	<ScrollWheelUp>
<S-ScrollWheelUp>	向上滚动一页	<S-ScrollWheelUp>

<C-ScrollWheelUp>	向上滚动一页	<C-ScrollWheelUp>
<ScrollWheelDown>	向下滚动三行	<ScrollWheelDown>
<S-ScrollWheelDown>	向下滚动一页	<S-ScrollWheelDown>
<C-ScrollWheelDown>	向下滚动一页	<C-ScrollWheelDown>
<ScrollWheelLeft>	向左滚动六列	<ScrollWheelLeft>
<S-ScrollWheelLeft>	向左滚动一页	<S-ScrollWheelLeft>
<C-ScrollWheelLeft>	向左滚动一页	<C-ScrollWheelLeft>
<ScrollWheelRight>	向右滚动六列	<ScrollWheelRight>
<S-ScrollWheelRight>	向右滚动一页	<S-ScrollWheelRight>
<C-ScrollWheelRight>	向右滚动一页	<C-ScrollWheelRight>

它们应该适用于所有模式，编辑命令行时除外。

注意 仅当 'nowrap' 置位时横向滚动可用。此外，除非 'guioptions' 设置了 "h" 标志位，如果光标行将要滚动到屏幕之外，移动光标到最长的可见行 (和横向滚动条的工作方式类似)。

您可以通过键盘映射来改变缺省行为。例如，要使得普通模式下滚轮一次滚动一行或半页：

```
:map <ScrollWheelUp> <C-Y>
:map <S-ScrollWheelUp> <C-U>
:map <ScrollWheelDown> <C-E>
:map <S-ScrollWheelDown> <C-D>
```

您也可以使用 Alt 和 Ctrl 修饰符。

自然，这些功能仅在 Vim 可以获取滚动事件时才可以使用。您可以使用 "xev" 程序来确定这些是否功能有效。

当您使用 XFree86 时，文件 /etc/XF86Config 中必须有匹配您使用的鼠标的正确条目。就 FreeBSD 来说，这个条目适用于一款罗技滚轮鼠标：

```
Protocol     "MouseMan"
Device       "/dev/psm0"
ZAxisMapping 4 5
```

欲了解更多的信息，请看 XFree86 的文档。

<MouseDown> <MouseUp> <MouseDown> 和 <MouseUp> 键已废弃。用 <ScrollWheelUp> 来代替 <MouseDown>，<ScrollWheelDown> 来代替 <MouseUp>。

xterm-mouse-wheel

如前所述，您仅需要使滚轮鼠标在您的 Xserver 上工作正常，它就可以在一个新的 xterm 中工作。

为了在一个老的 xterm 中使用您的滚轮，您需要做以下工作：

1. 如前所述，使它在您的 Xserver 上工作正常。
2. 给 xterm 添加转换，使得 xterm 可以用转义序列 (escape sequence) 来传递一个滚屏事件。
3. 给 Vim 添加映射，来把这些转义序列解释为 <ScrollWheelDown>、<ScrollWheelUp> 键。

把以下代码添加到您的 ~.Xdefaults 文件 (或者其他保存您 X 资源的文件) 中来进行转换：

```
XTerm*VT100.Translations:            #override \n\
             s<Btn4Down>: string("0x9b") string("[64~") \n\
```

```
s<Btn5Down>: string("0x9b") string("[65~") \n\
<Btn4Down>: string("0x9b") string("[62~") \n\
<Btn5Down>: string("0x9b") string("[63~") \n\
<Btn4Up>: \n\
<Btn5Up>:
```

把这些映射加到您的 vimrc 文件:

```
:map <M-Esc>[62~ <ScrollWheelUp>
:map! <M-Esc>[62~ <ScrollWheelUp>
:map <M-Esc>[63~ <ScrollWheelDown>
:map! <M-Esc>[63~ <ScrollWheelDown>
:map <M-Esc>[64~ <S-ScrollWheelUp>
:map! <M-Esc>[64~ <S-ScrollWheelUp>
:map <M-Esc>[65~ <S-ScrollWheelDown>
:map! <M-Esc>[65~ <S-ScrollWheelDown>
```

vim:tw=78:ts=8:noet:ft=help:norl:

insert.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　　by Bram Moolenaar
译者：Willis，tocer
http://vimcdoc.sf.net

插入和替换文本　　Insert　Insert-mode　mode-ins-repl

本文件主要讨论插入和替换模式。最后讨论一些其它方式插入文本的命令。

最常用的命令的总览可以在用户手册第 24 章　usr_24.txt　找到。

1. 特殊键　　ins-special-keys
2. 特殊的特殊键　　ins-special-special
3. 'textwidth' 和 'wrapmargin' 选项　　ins-textwidth
4. 'expandtab'、'smarttab' 和 'softtabstop' 选项　　ins-expandtab
5. 替换模式　　Replace-mode
6. 虚拟替换模式　　Virtual-Replace-mode
7. 插入模式补全　　ins-completion
8. 插入模式命令　　inserting
9. Ex 插入命令　　inserting-ex
10. 插入文件　　inserting-file

关于如何移动光标到没有字符的位置，另见 'virtualedit'。对编辑表格有用。

---

1. 特殊键　　ins-special-keys

在插入和替换模式里，以下字符有特殊含义；其它字符被直接插入。要插入这些特殊字符到缓冲区里，在前面加上 CTRL-V。要插入 <Nul> 字符，使用 "CTRL-V CTRL-@" 或者 "CTRL-V 000"。在有的系统上，你必须使用 "CTRL-V 003" 来插入 CTRL-C。注意：如果 CTRL-V 被映射，你也许会经常使用 CTRL-Q 来代替　i_CTRL-Q　。

如果插入时你在特殊的语言模式下工作，参见 'langmap' 选项　'langmap'　了解如何避免反复进出这些模式。

如果置位了 'insertmode'，<Esc> 和一些其它的键有另外的含义。见　'insertmode'　。

字符　　动作

---

i_CTRL-[　i_<Esc>
<Esc> 或 CTRL-[　结束插入或替换模式，回到普通模式。结束缩写。
注意：如果你很难在键盘上敲上 <Esc> 键，训练自己使用 CTRL-[。如果 Esc 不能用而你使用的是 Mac，试试 CTRL-Esc。或者在辅助功能里关闭收听。

i_CTRL-C
CTRL-C　　退出插入模式，回到普通模式。不检查缩写。不激活　InsertLeave　自动命令事件。

i_CTRL-@

CTRL-@ 插入最近插入的文本，并停止插入。

i_CTRL-A

CTRL-A 插入最近插入的文本。

i_CTRL-H i_<BS> i_BS

<BS> 或 CTRL-H 删除光标前的字符（关于连接行，见 i_backspacing ）。
如果你的 <BS> 键不正确，见 :fixdel 。

i_<Del> i_DEL

<Del> 删除光标下的字符。如果光标在行尾，并且 'backspace' 选项包括 "eol"，删除 <EOL>；下一行就此附加于当前行之后。
如果你的 <Del> 键不正确，见 :fixdel 。

i_CTRL-W

CTRL-W 删除光标前的单词（关于连接行，见 i_backspacing ）。关于单词的定义，见 word-motions 关于 "单词动作" 的定义。

i_CTRL-U

CTRL-U 删除当前行上光标前的所有输入字符。如果没有新输入的字符而 'backspace' 不为空，删除当前行上光标前的所有字符。
关于连接行，见 i_backspacing 。

i_CTRL-I i_<Tab> i_Tab

<Tab> 或 CTRL-I 插入制表。如果打开 'expandtab' 选项，等价数目的空格被插入（使用 CTRL-V <Tab> 避免这种扩展：如果 CTRL-V 被映射，可以使用 CTRL-Q <Tab>。 i_CTRL-Q ）。另见 'smarttab' 选项和 ins-expandtab 。

i_CTRL-J i_<NL>

<NL> 或 CTRL-J 开始新行。

i_CTRL-M i_<CR>

<CR> 或 CTRL-M 开始新行。

i_CTRL-K

CTRL-K {char1} [char2]
输入二合字母（见 digraphs ）。当 {char1} 为特殊字符时，该键的键码以 <> 形式插入。例如字符串 "<S-Space>" 可以这样输入：<C-K><S-Space>（两个键）。两个键都不考虑映射。

CTRL-N 查找下一个关键字（见 i_CTRL-N ）。
CTRL-P 查找上一个关键字（见 i_CTRL-P ）。

CTRL-R {register} i_CTRL-R
插入寄存器内容。在输入 CTRL-R 和第二个字符之间，'"' 会显示出来，以提示你需要输入寄存器的名字。文本插入方式和直接输入相同，但不使用映射和缩写。如果设置了 'textwidth'、'formatoptions' 或 'autoindent'，插入的结果会受到影响。这和使用 "p" 命令和用鼠标粘贴文本不同。
特殊寄存器：

'"'	无名寄存器，包含最近删除或抽出的文本
'%'	当前文件名
'#'	轮换文件名
'*'	剪贴板内容（X11：主选择）
'+'	剪贴板内容
'/'	最近的搜索模式
':'	最近的命令行
'.'	最近插入的文本
'-'	最近的行内（少于一行）删除

i_CTRL-R_=

'='　　表达式寄存器；你会被提示输入一个表达式（见 expression ）
注意 0x80（十进制 128）用于特殊键。例如，你可以这样移动光标向上：
CTRL-R ="\<Up>"
用 CTRL-R CTRL-R 可以按本义插入文本。
如果结果是 List ，里面的项目被看作行，之间以换行符连接。
如果结果是浮点数，自动转化为字符串。
调用 append() 或 setline() 会打断撤销序列。
关于寄存器见 registers 。

CTRL-R CTRL-R {register}　　i_CTRL-R_CTRL-R
插入寄存器内容。和单个 CTRL-R 类似，但是文本按本义插入，而不是像键盘输入那样。这意味着如果寄存器包含 <BS> 这样的字符，结果会不同。例如，如果寄存器包含 "ab^Hc":
CTRL-R a　　产生 "ac"。
CTRL-R CTRL-R a　　产生 "ab^Hc"。
'textwidth'、'formatoptions' 等等选项仍然适用。如果你连这些都想避免，使用 CTRL-R CTRL-O，见下。
'.' 寄存器（最近插入的文本）依然如同键盘输入那样的方式插入。

CTRL-R CTRL-O {register}　　i_CTRL-R_CTRL-O
按本义插入寄存器内容，并且不进行自动缩进。和鼠标粘贴文本相同 <MiddleMouse> 。如果寄存器是面向行的，在当前行之上插入文本，类似于 P。
不会替换字符！
'.' 寄存器（最近插入的文本）依然如同键盘输入那样的方式插入。

CTRL-R CTRL-P {register}　　i_CTRL-R_CTRL-P
按本义插入寄存器内容，修正缩进，和 [<MiddleMouse> 类似。
不会替换字符！
'.' 寄存器（最近插入的文本）依然如同键盘输入那样的方式插入。

i_CTRL-T
CTRL-T　　在当前行开始处插入一个 shiftwidth 的缩进。缩进总是取整到 'shiftwidth' 的倍数（这是 vi 兼容的）。

i_CTRL-D
CTRL-D　　在当前行开始处删除一个 shiftwidth 的缩进。缩进总是取整到 'shiftwidth' 的倍数（这是 vi 兼容的）。

i_0_CTRL-D
0 CTRL-D　　删除所有当前行的缩进。

i_^_CTRL-D
^ CTRL-D　　删除当前行的所有缩进。缩进在下一行上恢复。这可以用于插入卷标。

i_CTRL-V
CTRL-V　　如果下一个是非数字，按本义插入。对特殊键而言，插入其终端代码。不然，输入的是字符的十、八或十六进制值。 i_CTRL-V_digit 。
CTRL-V 之后紧接着输入的字符不经过映射。
注意：当 CTRL-V 被映射时（例如，用来粘贴文本），你可能经常需要使用 CTRL-Q 来代替 i_CTRL-Q 。
如果打开 modifyOtherKeys ，特殊转义序列被转换回没有 modifyOtherKeys 时的样子，除非同时按住 Shift 键。

i_CTRL-SHIFT-V

CTRL-SHIFT-V 与 CTRL-V 类似，除非激活了 modifyOtherKeys ，此时插入带修饰符的键的转义序列。

i_CTRL-Q

CTRL-Q 等同于 CTRL-V。
注意：有的终端连接会吃掉 CTRL-Q，导致该快捷键无效。在 GUI 版本里就不会出现这样的问题。

CTRL-X 进入 CTRL-X 模式，一个子模式。那里你可以给出命令来补全单词或者滚动窗口。见 i_CTRL-X 和 ins-completion 。

i_CTRL-E

CTRL-E 插入光标下面的字符。

i_CTRL-Y

CTRL-Y 插入光标上面的字符。
注意 CTRL-E 和 CTRL-Y 不使用 'textwidth'，从而可以从长行里复制字符。

i_CTRL-_

CTRL-_ 切换语言，如下：
- 在从右到左的窗口里，切换 revins 和 nohkmap，因为在这种情况下英语的输入可能就是倒过来的。
- 在非从右到左的窗口里，切换 revins 和 hkmap。因为希伯来语等语种可能是倒过来输入的。

CTRL-_ 移动光标到输入文本的尾部。

该命令只有在 'allowrevins' 选项置位的时候才有效。
请参考 rileft.txt ，那里可以了解到更多有关从右到左模式的信息。
只有在编译时加入 +rightleft 特性才有效。

i_CTRL-^

CTRL-^ 切换语言字符输入的使用方式。
如果定义了语言映射 :lmap :
- 'iminsert' 为 1 (使用 langmap 映射) 时变成 0 (不使用 langmap 映射)。
- 'iminsert' 为其它值时变成 1，从而打开 langmap 映射。

如果没有定义语言映射：
- 'iminsert' 为 2 (使用输入方法 (Input Method)) 时变成 0 (不使用输入方法)。
- 'iminsert' 为其它值时变成 2，从而打开输入方法。

如果 为 1，"b:keymap_name" 变量的值、'keymap' 选项或者 "<lang>" 出现在状态行上。
这些语言映射通常用来输入不同于键盘上能直接产生的字符。'keymap' 选项用来安装若干完整的这样的映射表。

i_CTRL-]

CTRL-] 切换缩写，不插入字符。

i_<Insert>

<Insert>　　切换插入和替换模式。

---

i_backspacing

<BS>、CTRL-W 和 CTRL-U 的效果决定于 'backspace' 选项（除非置位了 'revins'）。这时一个逗号分隔的项目列表：

项目	动作
indent	允许退格删除自动缩进
eol	允许退格删除换行符（连接行）
start	允许退格删除插入开始之前的位置；CTRL-W 和 CTRL-U 在开始位置停止

如果 'backspace' 为空，则使用 Vi 兼容的退格方式。不能退格删除自动缩进、回到第一列之前、或者超过插入开始的地方。

为了后向兼容起见，取值 "0"、"1" 和 "2" 也是允许的，见 'backspace' 。

如果 'backspace' 选项的确包含 "eol"，光标在第一列，而使用了这三个键中的一个，当前行会和上一行连接。这实际上删除了光标之前的 <EOL>。

i_CTRL-V_digit

使用 CTRL-V，字符的十、八、十六进制值可以直接输入。这样你可以输入任何字符，除了换行符（<NL>，其值为 10）。有五个方法可以输入字符值：

第一个字符	模式	最大字符数	最大值	
(无)	十进制	3	255	
o 或 O	八进制	3	377	(255)
x 或 X	十六进制	2	ff	(255)
u	十六进制	4	ffff	(65535)
U	十六进制	8	7fffffff	(2147483647)

通常你会输入最大数目的字符数。这样，要输入空格（值为 32），你需要键入 <C-V>032。你可以忽略开头的零，这时，数字之后的字符必须不能再是数字。其它模式下也一样：一旦你输入在该模式下不合法的字符，那么这之前的值就会被使用，而 "非法" 的这个字符以正常的方式继续处理。

如果你输入的值为 10，在文件中最后会以 0 出现。10 是 <NL>，内部被用来代表 <Nul> 字符。在写入文件时，<NL> 字符被翻译成 <Nul>。而在每行的最后写入 <NL>。所以，如果你想在文件中插入 <NL> 字符，你需要使用换行。

i_CTRL-X　insert_expand

CTRL-X 进入一个子模式，那里可以使用若干命令。绝大多数命令执行关键字补全；见 ins-completion 。

有两个命令可以在不退出插入模式的前提下上下滚动窗口：

i_CTRL-X_CTRL-E

CTRL-X CTRL-E　　窗口滚动上移一行。
补全时看这里：　complete_CTRL-E

i_CTRL-X_CTRL-Y

CTRL-X CTRL-Y　　窗口滚动下移一行。
补全时看这里：　complete_CTRL-Y

按了 CTRL-X 以后，每个 CTRL-E（CTRL-Y）滚动窗口上（下）移一行，除非这使得光标不得不离开当前文件中所在的位置。一旦按了另外一个键，CTRL-X 模式就会退出，而回到插入模式下解释该键。

---

2. 特殊的特殊键 ins-special-special

一些的键是特殊的。它们停止当前的插入，做一些事情，然后重新插入。这意味着你可以不脱离插入模式的情况下做一些事情。这适合于经常使用插入模式的用户，就像编辑器没有单独的普通模式一样。这时，也可以设置 'backspace' 选项为 "indent,eol,start" 还有置位 'insertmode' 选项。如果你想给功能键映射到一个命令，你可以使用 CTRL-O。

这些键前后的改动（插入或者删除字符）可以分别撤销。只有最后的改动可以重做，而其行为和 "i" 命令相当。

字符	动作	
<Up>	光标上移一行	i_<Up>
<Down>	光标下移一行	i_<Down>
CTRL-G <Up>	光标上移一行，到插入开始时所在的列	i_CTRL-G_<Up>
CTRL-G k	光标上移一行，到插入开始时所在的列	i_CTRL-G_k
CTRL-G CTRL-K	光标上移一行，到插入开始时所在的列	i_CTRL-G_CTRL-K
CTRL-G <Down>	光标下移一行，到插入开始时所在的列	i_CTRL-G_<Down>
CTRL-G j	光标下移一行，到插入开始时所在的列	i_CTRL-G_j
CTRL-G CTRL-J	光标下移一行，到插入开始时所在的列	i_CTRL-G_CTRL-J
<Left>	光标左移一个字符	i_<Left>
<Right>	光标右移一个字符	i_<Right>
<S-Left>	光标反向一个单词（像 "b" 命令那样）	i_<S-Left>
<C-Left>	光标反向一个单词（像 "b" 命令那样）	i_<C-Left>
<S-Right>	光标正向一个单词（像 "w" 命令那样）	i_<S-Right>
<C-Right>	光标正向一个单词（像 "w" 命令那样）	i_<C-Right>
<Home>	光标移到该行第一个字符	i_<Home>
<End>	光标移到该行最后一个字符	i_<End>
<C-Home>	光标移到该文件第一个字符	i_<C-Home>
<C-End>	光标移到该文件最后一个字符	i_<C-End>
<LeftMouse>	光标移动鼠标点击处	i_<LeftMouse>
<S-Up>	上翻窗口一页	i_<S-Up>
<PageUp>	上翻窗口一页	i_<PageUp>
<S-Down>	下翻窗口一页	i_<S-Down>
<PageDown>	下翻窗口一页	i_<PageDown>
<ScrollWheelDown>	窗口向下滚动三行	i_<ScrollWheelDown>
<S-ScrollWheelDown>	窗口向下滚动一个整页	i_<S-ScrollWheelDown>
<ScrollWheelUp>	窗口向上滚动三行	i_<ScrollWheelUp>
<S-ScrollWheelUp>	窗口向上滚动一个整页	i_<S-ScrollWheelUp>
<ScrollWheelLeft>	窗口向左滚动六列	i_<ScrollWheelLeft>
<S-ScrollWheelLeft>	窗口向左滚动一个整页	i_<S-ScrollWheelLeft>
<ScrollWheelRight>	窗口向右滚动六列	i_<ScrollWheelRight>
<S-ScrollWheelRight>	窗口向右滚动一个整页	i_<S-ScrollWheelRight>
CTRL-O	执行命令，然后返回到插入模式	i_CTRL-O
CTRL-\ CTRL-O	类似于 CTRL-O，但不移动光标	i_CTRL-_CTRL-O
CTRL-L	置位 'insertmode' 时：转到普通模式	i_CTRL-L

CTRL-G u	打断撤销序列，开始新的改变	i_CTRL-G_u
CTRL-G U	光标停在同一行时，下个左/右光标移动不打断撤销	i_CTRL-G_U

注意：如果光标键把你带出插入模式，查查 'noesckeys' 选项。

CTRL-O 命令有时有副作用：如果光标在行尾之后，它会先被移动该行最后一个字符上。在映射里，通常更好的方法是使用 <Esc> （先在文本中放一个 "x"，<Esc> 这时总会把光标放到它的上面）。或者使用 CTRL-\ CTRL-O，不过这时要注意光标可能移到行尾之外的位置。注意 CTRL-\ CTRL-O 之后的命令仍然可以移动光标，光标不会自动恢复到原来的位置。

CTRL-O 命令带你到普通模式，此时你如使用进入插入模式的命令，插入模式通常不会嵌套。因而，输入完 "a<C-O>a" 然后按 <Esc>，你会回到普通模式，不需要按 <Esc> 两次。此规则有例外，如果不是键入此命令，也即，在执行映射或执行脚本时，就会嵌套。这样，短暂切换到插入模式的映射才可以工作。

不是在所有的终端上都能用 Shift + 光标键。

另外一个副作用是 "i" 或 "a" 命令之前指定的计数会被忽略。这是因为要实现 CTRL-O 之后的命令的重复执行太复杂了。

一个使用 CTRL-G u 的例子：

```
:inoremap <C-H> <C-G>u<C-H>
```

它重定义退格键开始新的撤销序列。你可以撤销退格键的效果，而不会改变你之前输入的内容，就像 CTRL-O u 那样。另一例：

```
:inoremap <CR> <C-]><C-G>u<CR>
```

这样，每行之后就不能撤销。在此之前也进行缩写的扩展。

一个使用 CTRL-G U 的例子：

```
inoremap <Left>  <C-G>U<Left>
inoremap <Right> <C-G>U<Right>
inoremap <expr> <Home> col('.') == match(getline('.'), '\S') + 1 ?
 \ repeat('<C-G>U<Left>', col('.') - 1) :
 \ (col('.') < match(getline('.'), '\S') ?
 \     repeat('<C-G>U<Right>', match(getline('.'), '\S') + 0) :
 \     repeat('<C-G>U<Left>', col('.') - 1 - match(getline('.'), '\S')))
inoremap <expr> <End> repeat('<C-G>U<Right>', col('$') - col('.'))
inoremap ( ()<C-G>U<Left>
```

这样，可以在插入模式下使用光标键而不会打断撤销序列，从而 .（重做）命令就能正常工作。
另外，输入以下的文本（使用上述的 "(" 映射）：

```
Lorem ipsum (dolor
```

可以用 . 重复直到出现期待结果为止

```
   Lorem ipsum (dolor)
```

CTRL-O 的使用分割撤销：之前输入的文本和之后的被分别撤销。如果不想如此（比如用在映射里），你可以用 CTRL-R = i_CTRL-R 。例如，要调用函数：

```
        :imap <F2> <C-R>=MyFunc()<CR>
```

如果正确设置 'whichwrap' 选项，在一行的第一个/最后一个字符上按 <Left> 和 <Right> 键使得光标回绕到上一行/下一行。

CTRL-G j 和 CTRL-G k 命令可以用来在某一列前插入文本。例如：

```
   int i;
   int j;
```

把光标定位在第一个 "int" 上，输入 "istatic <C-G>j   "。结果是：

```
   static int i;
          int j;
```

要把相同的文本插入在每行的某列之前，使用可视列块命令 "I" v_b_I 。

---

3. _92protect'textwidth' 和 _92protect'wrapmargin' 选项 ins-textwidth

'textwidth' 选项可以用来在行变得很长之前自动断行。设置 'textwidth' 选项为希望的最大行长。如果你输入更多字符（不是空格或者制表），最后一个单词会放在一个新行上（除非这是该行唯一一个单词）。如果你设置 'textwidth' 为 0，关闭该特性。

'wrapmargin' 选项做的事情基本相同。区别在于 'textwidth' 是一个固定的宽度，而 'wrapmargin' 根据屏幕的宽度设置。如果设置 'wrapmargin'，这等价于 'textwidth' 设为 (columns - 'wrapmargin')，其中 columns 是屏幕的宽度。

如果同时设置 'textwidth' 和 'wrapmargin'，使用 'textwidth'。

如果你并不真的想断开行，而只是想文本行在合适的位置回绕，见 'linebreak' 选项。

文本行只有在插入模式下或者附加到行后的时候才会自动断开。在替换模式下，只要行的长度没有变，就不会断行。

长行在你输入一个出现在边界之后的非空白字符的时候断开。什么时候断行的限制还可以通过在 'formatoptions' 选项增加如下字母决定：

"l" 断行只有在插入开始时文本行的长度不超过 'textwidth' 才会发生。
"v" 只有在当前插入命令中输入的空白字符上才会断行。这是和 Vi 最兼容的行为。
"lv" 断行只有在插入开始时文本行的长度不超过 'textwidth' 并且在当前插入命令中输入的空白字符上才会发生。和 "l" 唯一的不同在超过 'textwidth' 边界之后输入非空白字符的时候。

通常使用内部函数来决定哪里断行。如果你想使用不同的方法，设置 'formatexpr' 选项为一个表达式，它处理换行的行为。

如果你想排版文本块，可以使用 "gq" 操作符。输入 "gq" 和可以移动光标到块尾的移动命令。在许多情况下，命令 "gq}" 会做你想要做的事情（排版直到段落尾部）。或者，你可以使用 "gqap"。它会排版整个段落，和当前光标所在的位置无关。或者，你可以使用可视模式：敲击 "v"，移动到块尾，然后输入 "gq"。另见 gq 。

---

4. _92protect'expandtab'、_92protect'smarttab' 和 _92protect'softtabstop' 选项 ins-expandt

如果打开 'expandtab' 选项，空格可以用来填充制表键的空白位置。如果你需要输入一个真正的 <Tab>，先输入 CTRL-V（用 CTRL-Q 如果 CTRL-V 被映射的话 i_CTRL-Q ）。缺省 'expandtab' 选项是关闭的。注意 在替换模式里，一个字符被多个空格字符所代替。结果是，行内的字符数会增加。退格键只会一次删一个空格键。原来的字符只有在一个空格（最后一个）上退格才能得回来。

ins-smarttab

当 'smarttab' 选项打开时，<Tab> 在行首插入 'shiftwidth' 个位置，而在其它地方插入 'tabstop' 个。这意味着经常会插入空格而不是 <Tab> 字符。当 'smarttab' 关闭时，<Tab> 总是插入 'tabstop' 个位置，而 'shiftwidth' 只有在 ">>" 和类似的命令中才会用到。

ins-softtabstop

如果 'softtabstop' 选项不为零，<Tab> 插入 'softtabstop' 个位置，而过去用来删除空格的 <BS>，现在会删除 'softtabstop' 个位置。感觉上，'tabstop' 被设成了 'softtabstop' 的值，但实际上一个真正的 <Tab> 字符还是占据 'tabstop' 个位置。从而，你的文件在别的应用程序里看起来还是正确的。

如果 'softtabstop' 不为零，<BS> 会试图删除尽量多的空白，以便能够回到往前 'softtabstop' 的位置，除非前面一个插入的字符正好就是一个空格，这时它只会删除光标前的那个字符。否则，你不一定总能删除光标前的一个字符。你需要先删除 'softabstop' 个字符，然后再输入额外的空格，以到达你想要的地方。

---

5. 替换模式 Replace Replace-mode mode-replace

在普通模式里输入 "R" 命令进入替换模式。

在替换模式里，行内的单个字符在你输入字符的时候被删除。如果没有字符可以删了（在行尾），输入的字符被附加（和插入模式一样）。这样，一行内的字符数保持不变，直到你到达行尾为止。如果输入了 <NL>，插入一个换行符，但不会删除任何字符。

要小心 <Tab> 字符。如果你输入一个正常的打印字符在它上面，字符数仍然一样，但是列数看起来少了。

如果你在替换模式下删除字符（用 <BS>、CTRL-W 或 CTRL-U），实际发生的事是你删除了改变。被替换的字符被复原了。如果你的输入超过已有的部分，新增的字符被删除了。实际上，这可以看作是一次一个字符的撤销。

如果打开了 'expandtab' 选项，<Tab> 会用多个空格替换一个字符。结果是，行内的字符数增加了。退格键只能一次删一个空格。原来的字符只有在一个空格（最后一个）上退格才能得回来。

---

6. 虚拟替换模式 vreplace-mode Virtual-Replace-mode

在普通模式里输入 "gR" 命令进入虚拟替换模式。
{仅当编译时加入 +vreplace 特性才会有效}

虚拟替换模式和替换模式类似，但不是替换文件里的实际字符，而是替换屏幕的领地。这样，文件里的字符看起来不会移动。

所以，如果你输入了 <Tab>，它会替换多个普通的字符，而如果你在 <Tab> 上输入字母，它可能什么都没有代替，因为 <Tab> 还是会占据相同的位置。

输入 <NL> 不是导致文件后面的字符看起来在移动。结果是，当前行的后面部分被 <NL> 所替换（也就是，它们被删除），而替换继续在下一行进行。新行 不 会被插入，除非你到达文件尾部之后。

输入 CTRL-T 和 CTRL-D 会看到有趣的效果。光标前面的字符向一边移动，跟平常一样，但是光标后面的字符保持不动。CTRL-T 会隐藏一些被移动字符遮盖的旧行，但是 CTRL-D 会重新让它们显现出来。

和替换模式一样，使用 <BS> 等会恢复被替换的字符。即使和 'smartindent'、CTRL-T 和 CTRL-D、'expandtab'、'smarttab'、'softtabstop' 等一起使用的效果也是如此。

在 'list' 模式下，虚拟替换模式的行为和不在 'list' 模式下一样，除非 'cpoptions' 里设置了 "L"。

注意 此模式下，不在光标位置上的字符看起来在移动的可能性仅有如下情况：列表模式 'list'，置位 'wrap' 时偶尔也会出现（这时行改变长度，使得比屏幕宽度更窄或者更宽）。其他情况下会插入空格以防止之后的字符移动。

该模式对编辑 <Tab> 分隔表格列的时候很有用，因为输入新的数据的时候同时还能保持所有的列对齐。

---

7. 插入模式补全 ins-completion

在插入和替换模式下，有若干命令可以补全输入的部分关键字或者行。这可以用于使用复杂关键字的场合（例如，函数名里有大写字母或者下划线）。

补全可以是针对：

1. 整行	i_CTRL-X_CTRL-L
2. 当前文件内的关键字	i_CTRL-X_CTRL-N
3. 'dictionary' 的关键字	i_CTRL-X_CTRL-K
4. 'thesaurus' 的关键字，同义词风格	i_CTRL-X_CTRL-T
5. 当前文件及其头文件内的关键字	i_CTRL-X_CTRL-I
6. 标签	i_CTRL-X_CTRL-]
7. 文件名	i_CTRL-X_CTRL-F
8. 定义或宏	i_CTRL-X_CTRL-D
9. Vim 命令	i_CTRL-X_CTRL-V
10. 用户定义的补全	i_CTRL-X_CTRL-U
11. 全能（omni）补全	i_CTRL-X_CTRL-O
12. 拼写建议	i_CTRL-X_s
13. 'complete' 的关键字	i_CTRL-N i_CTRL-P

所有这些，除了 CTRL-N 和 CTRL-P 以外，都通过 CTRL-X 模式完成。这是插入和替换模式的一个子模式。你可以键入 CTRL-X 和一个 CTRL-X 命令进入 CTRL-X 模式。要退出，输入不合法的 CTRL-X 模式的命令。合法的键包括 CTRL-X 命令本身，CTRL-N（下一个）和 CTRL-P（前一个）。

想得到当前补全的信息，可用 complete_info()。
如果你想调整匹配的大小写，另见 'infercase' 选项。

complete_CTRL-E

如果补全处于激活状态，可以用 CTRL-E 来停止补全并回到原来录入的文字。CTRL-E 本身不会被插入。

complete_CTRL-Y

如果已经弹出菜单，可以使用 CTRL-Y 停止补全并接受当前的选择项。CTRL-Y 本身不会被插入。键入空格、回车或者其他不可显示字符将离开补全模式并插入键入的字符。

弹出菜单显示时，有一些特殊键可用，见 popupmenu-keys 。

注意：CTRL-X 模式下合法的键不经过映射。这使得 ":map ^F ^X^F" 能够工作（其中 ^F 是 CTRL-F 而 ^X 是 CTRL-X）。能够使得 CTRL-X 模式退出的键（任何不是合法 CTRL-X 模式命令的键）则经过映射。
另外，通过 'complete' 的补全也使用映射。

注意：补全激活时，不能递归使用插入模式。以某种方式调用 ":normal i.." 的映射将产生 E523 错误。

建议使用以下映射来使得输入补全命令简单一点（不过它们可能屏蔽其它的命令）：

```
:inoremap ^] ^X^]
:inoremap ^F ^X^F
:inoremap ^D ^X^D
:inoremap ^L ^X^L
```

一个特例是，执行寄存器插入的 CTRL-R（见 i_CTRL-R ）不会退出 CTRL-X 模式。这主要是为了允许通过使用 '=' 寄存器来调用若干函数来决定下一个操作。如果该寄存器的内容（或者 '=' 寄存器计算的结果）不是合法的 CTRL-X 模式键，那么就会退出 CTRL-X 模式，如同键盘输入这些内容一样。

例如，下面的程序会如此映射 <Tab>：如果当前行只有空白，就插入 <Tab>，不然就开始或继续 CTRL-N 补全操作：

```
function! CleverTab()
   if strpart( getline('.'), 0, col('.')-1 ) =~ '^\s*$'
      return "\<Tab>"
   else
      return "\<C-N>"
   endif
endfunction
inoremap <Tab> <C-R>=CleverTab()<CR>
```

补全整行 compl-whole-line

i_CTRL-X_CTRL-L

CTRL-X CTRL-L 反向搜索和当前行光标前字符序列完全相同的行。忽略缩进。找到的行插入在光标的前面。
'complete' 选项用来决定匹配在哪个缓冲区里搜索，已载入和未载入的缓冲区都被使用。

CTRL-L 或
CTRL-P 反向搜索前一个匹配行。替换上一次匹配的行。

CTRL-N 正向搜索下一个匹配行。替换上一次匹配的行。

CTRL-X CTRL-L　在扩展一行以后，你可以通过接着输入 CTRL-X CTRL-L 得到紧接着匹配行之后的行，直到见到两个 CTRL-X 为止。只能用于已载入的缓冲区。

补全当前文件内的关键字　compl-current　i_CTRL-X_CTRL-P　i_CTRL-X_CTRL-N

CTRL-X CTRL-N　正向搜索以光标前面的关键字开始的单词。找到的关键字插入在光标的前面。

CTRL-X CTRL-P　反向搜索以光标前面的关键字开始的单词。找到的关键字插入在光标的前面。

CTRL-N　正向搜索下一个匹配的关键字。替换上一次匹配的关键字。

CTRL-P　反向搜索前一个匹配的关键字。替换上一次匹配的关键字。

CTRL-X CTRL-N 或
CTRL-X CTRL-P　继续使用 CTRL-X CTRL-N 或 CTRL-X CTRL-P 会复制上次本类型补全在其它上下文里扩展的单词之后紧跟的单词，直到见到两个 CTRL-X 为止。

如果在光标的前面有一个关键字（由字母字符和 'iskeyword' 指定的字符组成的名字），它的前面再加上 "\<"（含义：单词开始）就被用作搜索模式。否则 "\<\k\k" 被用作搜索模式（任何包含至少两个字符的关键字的开始）。

在替换模式里，替换的字符数目决定于匹配字符串的长度。这和直接在替换模式下键盘输入经过替换的字符串类似。

如果光标前面不是一个合法的关键字字符，则匹配任何至少有两个字符的关键字。
例如，要得到：
```
printf("(%g, %g, %g)", vector[0], vector[1], vector[2]);
```
只需输入：
```
printf("(%g, %g, %g)", vector[0], ^P[1], ^P[2]);
```

搜索会在文件末尾回绕，这里不使用 'wrapscan' 的值。

相同的补全内容多次重复会被跳过；这样每次 CTRL-N 和 CTRL-P 都会插入不同的匹配（除非只有一个匹配的关键字）。

永远不会得到单个字符的匹配，因为它们通常不是你真想要的。
例如，要得到：
```
printf("name = %s\n", name);
```
或者：
```
printf("name = %s\n", n^P);
```
甚至：
```
printf("name = %s\n", ^P);
```
'\n' 中的 'n' 被跳过。

在扩展完一个词后，你可以使用 CTRL-X CTRL-P 或 CTRL-X CTRL-N 得到紧跟在扩展词之后的单词。这些序列搜索刚刚扩展的文本，并且继续扩展之，使之包括另外一个词。这可以用于你需要重复一系列复杂的单词的场合。尽管 CTRL-P 和 CTRL-N 只找至少两个字

符的字符串，CTRL-X CTRL-P 和 CTRL-X CTRL-N 可以用来扩展只有一个字符的单词。
例如，要得到：

```
M&eacute;xico
```

你可以输入：

```
M^N^P^X^P^X^P
```

CTRL-N 开始一个扩展，而 CTRL-P 回到单个字符 "M"，然后后面的两个 CTRL-X CTRL-P 分别得到 "&eacute" 和 ";xico"。

如果上次的扩展因为超过 'textwidth' 被分割，则只会使用当前行的文本。

如果匹配在行尾，那么下一行的第一个单词会被插入，而且显示消息 "word from next line"。如果该词被接受，那么下个 CTRL-X CTRL-P 或者 CTRL-X CTRL-N 会搜索那些以这个单词开始的行。

补全 'dictionary' 的关键字 compl-dictionary

i_CTRL-X_CTRL-K

CTRL-X CTRL-K 根据 'dictionary' 选项给出的文件搜索光标前关键字开始的单词。这和 CTRL-N 类似，只不过搜索的是字典文件，而不是当前文件。找到的关键字插入在光标之前。这可能很慢，因为在第一个匹配用到之前，所有的匹配都会被找到。缺省，'dictionary' 选项为空。
要得到哪里能找单词列表的建议，见 'dictionary' 选项。

CTRL-K 或
CTRL-N 正向搜索下一个匹配的关键字。替换上一次匹配的关键字。

CTRL-P 反向搜索前一个匹配的关键字。替换上一次匹配的关键字。

i_CTRL-X_CTRL-T

CTRL-X CTRL-T 和 CTRL-X CTRL-K 类似，但稍有不同。它使用 'thesaurus' 选项，而不是 'dictionary'。如果匹配在同义词字典里找到，同一行里其余单词也在匹配里列出，即使它们并不真的匹配。这样一个单词可以被完全替换。

举一个例子，假想 'thesaurus' 文件有一行形如：

```
angry furious mad enraged
```

把光标放在字母 "ang" 之后并输入 CTRL-X CTRL-T 会匹配单词 "angry"；继续按会把单词改为 "furious"、"mad" 等等。其它的使用包括两种语言之间的翻译，或者用关键字给 API 函数归类。

CTRL-T 或
CTRL-N 正向搜索下一个匹配的关键字。替换上一次匹配的关键字。

CTRL-P 反向搜索前一个匹配的关键字。替换上一次匹配的关键字。

补全当前文件及其头文件内的关键字 compl-keyword

'include' 选项指定如何找到含有头文件名字的行。'path' 选项用来搜索头文件。

i_CTRL-X_CTRL-I

CTRL-X CTRL-I　　搜索当前文件及其头文件里第一个以光标前面的字母序列开始的关键字。找到的关键字插入在光标的前面。

CTRL-N　　正向搜索下一个匹配的关键字。替换上一次匹配的关键字。

注意：CTRL-I 和 <Tab> 相同，而这可能会在成功的补全之后输入，因此不使用 CTRL-I 来搜索下一个匹配。

CTRL-P　　反向搜索前一个匹配的关键字。替换上一次匹配的关键字。

CTRL-X CTRL-I　　继续使用 CTRL-X CTRL-I 会复制上次本类型补全在其它上下文里扩展的单词之后紧跟的单词，直到见到两个 CTRL-X 为止。

补全标签　　compl-tag i_CTRL-X_CTRL-]

CTRL-X CTRL-]　　搜索第一个以光标前面的字母序列开始的标签。匹配的标签插在光标前面。标签名可以包含字母字符和由 'iskeyword' 决定的字符（和关键字相同）。另见 CTRL-] 。'showfulltag' 选项可以用来增加标签定义前后的上下文。

CTRL-] 或 CTRL-N　　正向搜索下一个匹配的标签。替换上一次匹配的标签。

CTRL-P　　反向搜索前一个匹配的标签。替换上一次匹配的标签。

补全文件名　　compl-filename i_CTRL-X_CTRL-F

CTRL-X CTRL-F　　搜索第一个以光标前面的字母序列开始的文件。匹配的文件插在光标前面。标签名可以包含字母字符和由 'isfname' 决定的字符（和关键字相同）。注意，（目前）这里不使用 'path' 选项。

CTRL-F 或 CTRL-N　　正向搜索下一个匹配的文件名。替换上一次匹配的文件名。

CTRL-P　　反向搜索前一个匹配的文件名。替换上一次匹配的文件名。

补全定义或宏　　compl-define

'define' 选项用来指定包含定义的行。'include' 选项用来指定包含头文件名的行。'path' 选项用来搜索头文件。

i_CTRL-X_CTRL-D

CTRL-X CTRL-D　　搜索当前和头文件里第一个以光标前面的字母序列开始的定义（或宏）。找到的定义名插入在光标的前面。

CTRL-D 或 CTRL-N　　正向搜索下一个匹配的定义。替换上一次匹配的定义。

CTRL-P　　反向搜索前一个匹配的定义。替换上一次匹配的定义。

CTRL-X CTRL-D　　继续使用 CTRL-X CTRL-D 会复制上次本类型补全在其它上下文里扩展的单词之后紧跟的单词，直到见到两个 CTRL-X 为止。

补全 Vim 命令 compl-vim

这里，补全是上下文敏感的，和命令行上的情况相似。它既能补全 Ex 命令，又能补全它的参数。可用于编写 Vim 脚本。

i_CTRL-X_CTRL-V

CTRL-X CTRL-V 猜测光标前的项目的条目，并找到第一个匹配。
注意：如果 CTRL-V 被映射，你通常可以用 CTRL-Q 来代替 i_CTRL-Q 。

CTRL-V 或
CTRL-N 正向搜索下一个匹配。替换上一次匹配。

CTRL-P 反向搜索前一个匹配。替换上一次匹配。

CTRL-X CTRL-V 继续使用 CTRL-X CTRL-V 和 CTRL-V 一样。这允许映射键来执行 Vim 命令补全，例如：

```
:imap <Tab> <C-X><C-V>
```

用户定义补全 compl-function

命令补全可以由用户通过 'completefunc' 选项自定义一个函数来完成。下面说明如何调用此函数，并提供示例 complete-functions 。

i_CTRL-X_CTRL-U

CTRL-X CTRL-U 猜测光标前面项目的类型，并寻找它的第一个匹配。
CTRL-U 或
CTRL-N 正向搜索下一个匹配。替换上一次匹配。

CTRL-P 反向搜索前一个匹配。替换上一次匹配。

全能 (omni) 补全 compl-omni

命令补全可以由用户通过 'omnifunc' 选项自定义一个函数来完成。这通常用于特定文件类型的补全。

下面说明如何调用此函数，并提供示例 complete-functions 。
关于特定文件类型的说明，见 compl-omni-filetypes 。
将来会有更多补全脚本，欢迎查阅 www.vim.org。目前已经有了 C++ 的首个版本。

i_CTRL-X_CTRL-O

CTRL-X CTRL-O 猜测光标前面项目的类型，并寻找它的第一个匹配。
CTRL-O 或
CTRL-N 正向搜索下一个匹配。替换上一次匹配。

CTRL-P 反向搜索前一个匹配。替换上一次匹配。

拼写建议 compl-spelling

定位光标所在或之前的单词，然后提出正确拼写的单词作为建议进行替代。如果该行里有一个错误拼写的单词在光标之前或之下，移动光标到它后面。否则，使用刚刚在光标之前

的那个单词来提出建议，即使该单词没有拼写错误。

NOTE: 很多 Unix 终端上，CTRL-S 暂停显示。这时用 's' 可以代替。如果显示暂停，输入 CTRL-Q 会继续显示。

i_CTRL-X_CTRL-S i_CTRL-X_s

CTRL-X CTRL-S 或
CTRL-X s 定位光标之前的单词，并寻找它的第一个拼写建议。

CTRL-S 或
CTRL-N 正向搜索下一个建议。替换上一次的建议。 注意 这里不能用 's'。

CTRL-P 反向搜索前一个建议。替换上一次的建议。

从不同的来源补全关键字 compl-generic

i_CTRL-N
CTRL-N 在 'complete' 选项给出的地方搜索下一个以光标前面的关键字开始的单词。找到的关键字名插入在光标的前面。

i_CTRL-P
CTRL-P 在 'complete' 选项给出的地方搜索上一个以光标前面的关键字开始的单词。找到的关键字名插入在光标的前面。

CTRL-N 正向搜索下一个匹配的关键字。替换上一次匹配的关键字。

CTRL-P 反向搜索前一个匹配的关键字。替换上一次匹配的关键字。

CTRL-X CTRL-N 或
CTRL-X CTRL-P 继续使用 CTRL-X CTRL-N 或 CTRL-X CTRL-P 会复制上次本类型补全在其它上下文里扩展的单词之后紧跟的单词，直到见到两个 CTRL-X 为止。

寻找补全的函数 complete-functions

这里指 'completefunc' 和 'omnifunc'。

函数被调用两次，使用不同的方式：
- 首先，调用函数以寻找补全文本的开始位置。
- 然后，调用函数以寻找实际的匹配。

第一次调用时，参数是：
```
a:findstart  1
a:base       空
```

函数必须返回补全开始位置的列数，这个数字必须在零到光标所在列 "col('.')" 之间。过程应该检查光标之前的字符，并包含那些可能成为补全项一部分的字符。该列到光标列之间的文本将来会被匹配结果替换。如果返回值大于光标列，则使用光标列。

负返回值：
```
-2   安静的撤销，留在补全模式。
-3   安静的撤销，退出补全模式。
```

其它负值：补全从光标列开始

第二次调用时，参数是：

```
a:findstart  0
a:base       补全必须匹配的文本；即第一次调用定位的文本（可以为空）
```

函数必须返回匹配单词的列表。这些匹配通常包含 "a:base" 文本。如果没有匹配，返回空列表。

要知道关于匹配词的信息，返回包含匹配列表的一个字典。字典包含以下项目：

```
        words           包含匹配单词的列表（必需）。
        refresh         用于控制函数重新启动的字符串（可选）。
                        目前唯一支持的值是 "always"。效果是每次前导文本改变时
                        都会调用函数。
```

如果你要抑制空返回值引起的警告信息，可返回 v:none 。这可用于通过 complete() 实现异步补全。

忽略其他项目。

要在补全结束之后有所行动，参见 CompleteDone 自动命令事件。

例如，函数可以这么做：

```
        let matches = ... 单词列表 ...
        return {'words': matches, 'refresh': 'always'}
```

complete-items

每个列表项可以是字符串或者字典类型。如果是字符串，直接用作补全文本。如果是字典，可以包含以下各项：

```
        word            需要插入的文本，必需
        abbr            "word" 的缩写；如果非空，菜单里使用它而不是 "word"
        menu            用于弹出菜单的补充文本，在 "word" 或 "abbr" 之后显示
        info            关于补全项的更多信息，能够在预览或弹出窗口显示
        kind            代表补全类型的单个字母
        icase           如果非零，比较项目是否等同时忽略大小写；如果省略就假定
                        为零，这时可以同时加入只有大小写有差异的匹配项
        equal           如果非零，总是在比较时把此项当成相等。这意味着，
                        "equal=1" 关闭此项的过滤。
        dup             如果非零，那么即使和此匹配包含相同单词的匹配项已经存在
                        也无妨。
        empty           如果非零，匹配项即使是空字符串也会加入
        user_data       和项目相关联的定制数据，可从 v:completed_item 得到
```

除了 "icase"、"equal"、"dup" 和 "empty" 以外，其它各项必须是字符串。如果有一项不合要求，报错，而列表的其余项目也不再使用。你可以在返回列表中混用字符串和字典项目。

"menu" 项目用于弹出菜单且可能被截短，所以它应该尽量简短。"info" 项目可以稍长。如果在 'completeopt' 中包含 "preview"，使用预览窗口来显示该项信息，如果在 'completeopt' 中包含 "popup"，则使用弹出窗口。在预览窗口中关闭弹出菜单后，"info" 项目将保留显示，这对录入函数参数很有用。用单个空格设置 "info" 可以清除预览窗口现存的文本。预览窗口的大小缺省是三行，但在 'previewheight' 为 1 或 2 时使用其值。

complete-popup

如果 'completeopt' 中有 "popup"，使用弹出窗口显示 "info"。这时 'completepopup' 选项指定弹出的属性，在 info 弹出建立时使用。此选项是逗号分隔的值列表：

height	弹出的最大高度
width	弹出的最大宽度
highlight	弹出的高亮组（缺省是 PmenuSel）
align	"item"（缺省）或 "menu"
border	"on"（缺省）或 "off"

例如：

```
:set completepopup=height:10,width:60,highlight:InfoPopup
```

如果 "align" 值是 "item"，弹出的位置靠近选择的项目。改变选择也会移动弹出。如果 "align" 是 "menu"，如果菜单在文本之下，弹出和菜单顶部对齐，否则和菜单底部对齐。

info 弹出被创建后，可在 popup_findinfo() 里找到，其属性可由 popup_setoptions() 改变。

complete-popuphidden

如果是通过异步获取弹出的信息，在 'completeopt' 中用 "popuphidden"。info 弹出会先被隐藏，一旦 info 被填充，必须调用 popup_show() 。可通过 CompleteChanged 自动命令完成，就像这样：

```
set completeopt+=popuphidden
au CompleteChanged * call UpdateCompleteInfo()
func UpdateCompleteInfo()
  " 取消任何进行中的 info 获取
  let item = v:event.completed_item
  " 开始获取项目的 info，然后调用 ShowCompleteInfo(info)
endfunc
func ShowCompleteInfo(info)
  let id = popup_findinfo()
  if id
    call popup_settext(id, 'async info: ' .. a:info)
    call popup_show(id)
  endif
endfunc
```

complete-item-kind

"kind" 项目用单一字母表示补全类型。用它可以指定补全的不同显示方式（不同颜色或者图标）。目前，可用如下类型：

v	变量
f	函数或方法
m	结构或类成员
t	typedef
d	#define 或宏

如果搜索匹配耗时较长，可以调用 complete_add() 向总列表中增加每个匹配。不要在返回的列表里包含这些匹配！搜索匹配的同时，时不时地调用 complete_check() 来使得用户仍然可以按键。如果该函数返回非零，搜索停止。

E839 E840

该函数可以移动光标，结束后光标会恢复。
该函数不允许移动到其他窗口，也不允许删除文本。

一个补全月份名的示例：

```
fun! CompleteMonths(findstart, base)
  if a:findstart
    " 定位单词的开始处
    let line = getline('.')
    let start = col('.') - 1
    while start > 0 && line[start - 1] =~ '\a'
      let start -= 1
    endwhile
    return start
  else
    " 寻找匹配 "a:base" 的月份
    let res = []
    for m in split("Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec")
      if m =~ '^' . a:base
        call add(res, m)
      endif
    endfor
    return res
  endif
endfun
set completefunc=CompleteMonths
```

功能同上，但是现在假设搜索比较慢：

```
fun! CompleteMonths(findstart, base)
  if a:findstart
    " 定位单词的开始处
    let line = getline('.')
    let start = col('.') - 1
    while start > 0 && line[start - 1] =~ '\a'
      let start -= 1
    endwhile
    return start
  else
    " 寻找匹配 "a:base" 的月份
    for m in split("Jan Feb Mar Apr May Jun Jul Aug Sep Oct Nov Dec")
      if m =~ '^' . a:base
        call complete_add(m)
      endif
      sleep 300m        " simulate searching for next match
      if complete_check()
        break
      endif
    endfor
    return []
  endif
endfun
set completefunc=CompleteMonths
```

插 入 补 全 弹 出 菜 单

ins-completion-menu
popupmenu-completion

Vim 可以用更简单的弹出菜单来显示匹配。

当下面条件符合时使用弹出菜单：
- 'completeopt' 选项包含 "menu" 或 "menuone"。
- 显示终端至少支持 8 色。
- 至少有两条匹配项。如果使用 "menuone"，一条匹配也可以。

选项 'pumheight' 用于设置最大高度。默认值是使用全部有效空间。
选项 'pumwidth' 用于设置最小宽度。默认值是 15 个字符。

有三个状态：
1. 插入了完整的匹配，例如在 CTRL-N 或 CTRL-P 之后。
2. 用光标键选择其它匹配项。此时不插入该匹配项，只在弹出菜单中高亮选中的条目。
3. 只插入了部分匹配文本，并且已经输入字符或者使用了退格键，这时匹配项列表根据光标前的内容进行调整。

开始时你通常处于状态一并插入第一个匹配。如果 'completeopt' 包含了 "longest" 而且有多个匹配项，那么开始于状态三。

如果选择其它匹配项，例如键入 CTRL-N 或 CTRL-P，就进入了状态一。这不会改变匹配项列表。

如果退回到原文，就会处于状态三。要立即进入该状态，可以使用快捷键映射方法，该映射在补全开始后立即使用 CTRL-P：

```
:imap <F7> <C-N><C-P>
```

popupmenu-keys

状态一下，这些键有特别的含义：

<BS> 和 CTRL-H	删除一个字符，查找光标前单词的匹配项。这会减少列表中匹配项的数目，常常到只有一个项目，然后切换到状态二。
其它非特殊字符：	停止补全，不改变匹配，然后插入输入的字符。

状态二和状态三下，这些键有特别的含义：

<BS> 和 CTRL-H	删除一个字符，并查找光标前变短的单词的匹配项。这可能会发现更多的匹配项。
CTRL-L	从当前匹配项中增加一个字符，可能会减少匹配项的数量。
任何可显示的非空白字符：	加入该字符，减少匹配项的数量。

全部三个状态中，可以使用这些键：

CTRL-Y	是 (Yes)：接受当前选择的匹配项并停止补全。
CTRL-E	结束 (End) 补全，回退到选择匹配前原有的内容 (原先输入的或者最长的公共字符串)。
<PageUp>	反向若干项选择一个匹配项，但不插入。
<PageDown>	正向若干项选择一个匹配项，但不插入。
<Up>	选择前一个匹配，同 CTRL-P，但不插入。
<Down>	选择下一个匹配，同 CTRL-N，但不插入。
<Space> 或 <Tab>	停止补全，不改变匹配，插入键入的该字符

<Enter> 键的行为取决于你现在所处的状态：

状态一：	使用现有的文本，然后插入换行符。
状态二：	插入当前选择项。
状态三：	使用现有的文本，然后插入换行符。

换句话说：如果你只使用光标键在匹配项列表中选择其它条目，按 <Enter> 键将插入该

匹配。如果键入其它字符，按 <Enter> 键将插入换行符。

下面的高亮组能够改变菜单颜色：

Pmenu	普通项	hl-Pmenu
PmenuSel	选中项	hl-PmenuSel
PmenuSbar	滚动条	hl-PmenuSbar
PmenuThumb	滚动条拇指（thumb）	hl-PmenuThumb

显示弹出窗口时，没有专门的映射。但你可以使用插入模式映射并检查 pumvisible() 函数的返回值以进行不同的处理。例如：

```
:inoremap <Down> <C-R>=pumvisible() ? "\<lt>C-N>" : "\<lt>Down>"<CR>
```

映射中用 <expr> 可以在键入某字符或者满足某些条件时弹出菜单。例如，键入 '.'：

```
inoremap <expr> . MayComplete()
func MayComplete()
    if (can complete)
      return ".\<C-X>\<C-O>"
    endif
    return '.'
endfunc
```

详见 :map-<expr> 。

特定文件类型全能补全的补充说明 compl-omni-filetypes

文件类型 {filetype} 使用的是 'runtimepath' 的 autoload/{filetype}complete.vim 文件。比如对于 "java"，就是文件 autoload/javacomplete.vim。

C ft-c-omni

C 代码补全需要标签文件。你应该使用 Exuberant ctags 软件，因为它会加入补全所需要的额外信息。你可以在这里找到它： http://ctags.sourceforge.net/ 推荐使用 5.6 或以后版本。

对于 5.5.4 版本，你应该打上增加 "typename:" 字段的补丁：

```
ftp://ftp.vim.org/pub/vim/unstable/patches/ctags-5.5.4.patch
```

可以在这里找到 MS-Windows 上已经编译好的可执行版本：

```
http://ctags.sourceforge.net/
https://github.com/universal-ctags/ctags-win32
```

如果你想补全系统函数，可以用 ctags 生成包含所有系统头文件的标签文件：

```
% ctags -R -f ~/.vim/systags /usr/include /usr/local/include
```

在 vimrc 文件中，把这个标签文件增加到 'tags' 选项中：

```
set tags+=~/.vim/systags
```

如果在不包含 "." 或 "=" 的名字后面用 CTRL-X CTRL-O，会直接从标签文件中补全。这适用于任何标识符，包括函数名。如果你想补全标签文件没有的局部变量名，用 CTRL-P 代替。

如果在包含 "." 或 "=" 的名字后面用 CTRL-X CTRL-O，Vim 会试图识别变量类型并指出它所含的成员。这意味着只会列出该变量的有效成员。

如果成员名字已经完整，CTRL-X CTRL-O 会为复合类型加上 "." 或 "->"。

Vim 没有包含 C 编译器，它只能识别使用明确格式的声明。预处理器指令可能会引起混淆。如果在多个位置里定义了同样的结构名，所有可能的结构成员都会被包括。

CSS ft-css-omni

遵循 CSS 2.1 标准来补全属性和相应的值。

HTML ft-html-omni
XHTML ft-xhtml-omni

CTRL-X CTRL-O 能够补全 (X)HTML 文件的各种元素。它是为了支持编写 XHTML 1.0 Strict 文件而设计的，但也可用于其它 HTML 版本。特性：

- "<" 之后，根据上下文补全标签名 (在标签内部，不给出 div 标签建议)；'/>' 表明空标签
- 在标签中，补全合适的属性 (不包括标签的 width 属性)；同时显示属性类型；'*' 表明必需的属性
- 如果属性只有有限的几个可能值，用它们来补全
- 补全实体 (entity) 名
- 根据 <style> 标签和包含的 CSS 文件里提取的数据，补全 "class" 和 "id" 属性值
- 对 "style" 属性值或在 "style" 标签内部补全时，切换到 ft-css-omni 补全
- 对事件属性值或在 "script" 标签内部补全时，切换到 ft-javascript-omni 补全
- "</" 之后，CTRL-X CTRL-O 会关闭最近打开的标签

备注：如果是第一次使用，补全菜单的显示会有少许延迟--这点时间用于数据文件载入。

备注：补全可能会在错误格式的文档中失效。在这种情况下，请尝试运行 :make 命令检查格式问题。

HTML 类型 html-flavor

默认的 HTML 补全机制依赖于文件类型：HTML 文件使用 HTML 4.01 Transitional 标准 ('filetype' 是 'html')，XHTML 使用 XHTML 1.0 Strict 标准 ('filetype' 是 'xhtml')。

如果在任何标签的外部进行补全，你可以选择 DOCTYPE，然后载入合适的数据文件，并用于后面所有的补全操作。

关于数据文件格式更多的信息见 xml-omni-datafile 。vim 的线上站点 (www) 可以找到一些数据文件。

注意 b:html_omni_flavor 可以指向任何使用 XML 数据的文件。这就使混合 PHP (ft-php-omni) 和任何 XML 方言的补全成为可能 (假设你有相应的数据文件)。如果该变量没有设置，使用 XHTML 1.0 Strict 标准。

JAVASCRIPT ft-javascript-omni

补全绝大部分 JavaScript 语言和 DOM 元素。

补全:

- 变量
- 函数名；显示函数参数
- 函数参数
- 变量属性，试图检测变量类型
- 根据上下文补全 DOM 对象和属性
- 语言中的关键字

补全机制可以用于单独的 JavaScript 文件 (&ft==javascript)、(X)HTML 的 <script> 标签内部和事件的属性值 (包含对外部文件的扫描)。

DOM 兼容性

当前 (2006 年初) 有两种主要浏览器 - MS Internet Explorer 和 Mozilla Firefox。这两个软件市场占有率达到 90% 以上。理论上，W3C 组织 (http://www.w3c.org) 建立标准，但是这些标准并没有完全遵守和实现。

IE	FF	W3C	全能补全
+/-	+/-	+	+
+	+	-	+
+	-	-	-
-	+	-	-

不管浏览器的实现进度如何，补全插件总是把标准定义的元素放入建议列表。两个主要浏览器都实现但标准没有覆盖的元素也作为建议。而其它的元素则不出现于建议列表。

PHP ft-php-omni

PHP 代码的补全需要标签文件才能对外部文件的数据和类进行补全。应该使用 Exuberant ctags 5.5.4 版本或更新的版本。在这里可以找到它: http://ctags.sourceforge.net/

脚本对以下项目进行补全:

- $ 变量名之后
  - 如果变量声明为对象，加上 "->"，如果标签文件有效，显示类名
  - 在 "->" 之后，只补全给定类中的函数和变量名。为了查找类的位置和内容，需要标签文件。因为 PHP 不是强类型的语言，用户可以使用 @var 标签来声明类:

    ```
    /* @var $myVar myClass */
    $myVar->
    ```

    不过，要找到 myClass 的内容，仍然需要标签文件。

- 带有附加信息的函数名:
  - 如果是内建函数，列出可能的参数，在 | 之后列出函数返回的数据类型
  - 如果是用户函数，列出参数和函数定义所在的文件名 (如果不是当前文件)

- 常量名
- 在 "new" 声明之后的类名

注意：如果第一次调用补全功能，Vim 会把所有需要的数据载入内存。这可能需要几秒钟。下次补全时，就几乎感觉不到延迟了。

脚本检测光标是否在 <?php ?> 标签内。如果不是，会自动切换到 HTML/CSS/JavaScript 补全。注意：和原始 HTML 文件不同，标签补全（也仅对标签补全而言）和上下文无关。

RUBY ft-ruby-omni

Ruby 代码的补全需要 Vim 编译时带 +ruby 特性。

Ruby 补全会因需分析你的缓冲区以提供补全列表。它会从 'require' 载入的和当前缓冲区定义的模块里提取补全。

CTRL-X CTRL-O 提供的补全是上下文相关的：

上下文	提供的补全
1. 不在类定义中	类名、常量和全局变量
2. 类定义中	这个类所定义的方法或常量
3. '.'、'::' 或者 ':' 之后	被解除参照的对象所适用的方法
4. ':' 或者 ':foo' 之后	符号名 ('foo' 起始的那些)

备注：

- Vim 会载入/执行程序代码，以便提供补全。这可能会导致部分代码被执行。这一点也许值得关注。缺省行为不再打开这一功能。如果需要此特性，加上
  ```
  let g:rubycomplete_buffer_loading = 1
  ```
- 在第 1 点中，Vim 可以解析整个缓冲区以获得用作补全结果的类名列表。默认关闭此功能。要使其生效，在 vimrc 里加入
  ```
  let g:rubycomplete_classes_in_global = 1
  ```
- 在第 2 点中，不支持匿名类。
- 在第 3 点中，Vim 会试图判断对象所支持的方法。
- Vim 可以检测和载入 Rails 环境并用于 rails 项目的文件。默认关闭此特性。要使其生效，在 vimrc 里加入
  ```
  let g:rubycomplete_rails = 1
  ```

SYNTAX ft-syntax-omni

Vim 能够对将近 500 种语言进行语法高亮。高亮支持的一部分是需要知道构成语言的关键字。许多文件类型已经有专门的补全脚本，而对其它的文件类型而言，syntaxcomplete 脚本可以提供基本的补全。实现的方法是用 Vim 知道如何色彩高亮的那些文本来构造全能补全列表。它适用于任何文件类型，并可以提供基本但和语言相关的补全机制。

要打开语法打开补全：

```
setlocal omnifunc=syntaxcomplete#Complete
```

你可以把下列语句放到 .vimrc 中（要在任何 ":filetype" 命令之后），使其自动生效：

```
if has("autocmd") && exists("+omnifunc")
    autocmd Filetype *
                \   if &omnifunc == "" |
                \           setlocal omnifunc=syntaxcomplete#Complete |
                \   endif
endif
```

只有在针对特定文件类型的插件不存在的情况下，上述语句才对脚本设置补全操作。

每个文件类型可能有很多语法项目。此插件允许你定制从列表里包含或排除哪些语法组。让我们看看 PHP 文件类型如何处理。

如果你正在编辑一个 index.php 文件，运行如下命令：

```
syntax list
```

首先你将看到有许多不同的语法组。PHP 语言可以包含来自不同语言的元素，比如 HTML、 JavaScript 和许多其它语言。这种情况下，syntax 插件只包含由文件类型 "php" 开头的语法组。例如，缺省 PHP 包含这些语法组：phpEnvVar、phpIntVar、phpFunctions。

如果希望包括非本文件类型指定的其他语法项目，可用正规表达式语法 (autoload/syntaxcomplete.vim 的 13.0 版本加入) 来加入项目。编辑 PHP 文件时查看 ":syntax list" 的输出，我可以看到如下一些项目：

```
htmlArg,htmlTag,htmlTagName,javaScriptStatement,javaScriptGlobalObjects
```

要在编辑 PHP 文件时时使用 Javascript 和 HTML 关键字语法高亮组，可以指定三个正规表达式，每种语言分别一个。也可以简单限制包含组为特写值，而无需正规表达式：

```
let g:omni_syntax_group_include_php = 'php\w\+,javaScript\w\+,html\w\+'
let g:omni_syntax_group_include_php = 'phpFunctions,phpMethods'
```

这个变量的基本形式是：

```
let g:omni_syntax_group_include_{filetype} = '逗号,分隔的,列表'
```

PHP 语言可以进行语法高亮的项目非常多，而这些项目在全能补全列表里都会出现。

有些人可能觉得这个列表不实用或者只对某些项感兴趣。(如有必要) 有两种方法可以裁剪这个列表。如果不想显示特定语法组，有两种不同的方法来标识这些组。第一个具体列出这些语法组的名字。第二个使用正规表达式来指定所有想要的语法组。在 vimrc 中增加两者之一即可：

```
let g:omni_syntax_group_exclude_php = 'phpCoreConstant,phpConstant'
let g:omni_syntax_group_exclude_php = 'php\w*Constant'
```

该列表可以加入多个语法组，以逗号分隔。这个变量的基本形式是：

```
let g:omni_syntax_group_exclude_{filetype} = '逗号,分隔的,列表'
```

可以建立任意多个这些变量，只要变量名尾部的文件类型不同就行了。

此插件使用 isKeyword 选项来决定用于语法项目的单词边界。例如，Scheme 语言的补全应该包含 "-"，call-with-output-file。取决于你的文件类型，此方法未必能提供你期待的单词。设置 g:omni_syntax_use_iskeyword 选项为 0 会强制语法插件在单词字符上断开。在 vimrc 可以加如下行进行控制：

```
let g:omni_syntax_use_iskeyword = 0
```

为了插件开发者着想，本插件提供了公共函数 OmniSyntaxList。本函数可用于提供语法

项目的列表。编辑 SQL 文件时 (:e syntax.sql)，可以用 ":syntax list" 命令看到各种组和语法项目。示例：

```
syntax list
```

返回的数据类似如此：

```
sqlOperator    xxx some prior all like and any escape exists in is not
                   or intersect minus between distinct
                   links to Operator
sqlType        xxx varbit varchar nvarchar bigint int uniqueidentifier
                   date money long tinyint unsigned xml text smalldate
                   double datetime nchar smallint numeric time bit char
                   varbinary binary smallmoney
                   image float integer timestamp real decimal
```

这里列出两个语法组：sqlOperator 和 sqlType。要得到语法项目的列表，可用几种不同的方式调用 OminiSyntaxList。要得到不分组的所有语法项目：

```
echo OmniSyntaxList( [] )
```

要得到 sqlOperator 语法组的所有语法项目：

```
echo OmniSyntaxList( ['sqlOperator'] )
```

要得到 sqlOperator 和 sqlType 两个组的所有语法项目：

```
echo OmniSyntaxList( ['sqlOperator', 'sqlType'] )
```

也可用正规表达式：

```
echo OmniSyntaxList( ['sql\w\+'] )
```

在插件中，通常可以把输出结果赋给一个列表：

```
let myKeywords = []
let myKeywords = OmniSyntaxList( ['sqlKeyword'] )
```

SQL ft-sql-omni

SQL 语言的补全包括语句、函数和关键字。还可以动态地补全表、过程、视图和列的列表，此时数据直接从数据库里提取。详细的指令和教程见 omni-sql-completion 。

SQL 补全插件可以和其它补全插件一起使用。例如，PHP 文件类型有它自己的补全插件。因为 PHP 常用来生成访问数据库的动态网站，也可以同时打开 SQL 补全插件。这样就可以同时补全 PHP 代码和 SQL 代码。

XML ft-xml-omni

Vim 7 为 XML 文件中提供上下文相关的补全机制。它依赖于特殊的数据文件 xml-omni-datafile 和两个命令： :XMLns 和 :XMLent 。特性如下：

- "<" 之后，根据上下文补全标签名
- 标签内部补全合适的属性
- 如果属性只有有限的几个可能值，用它们来补全
- 补全实体 (entity) 名 (xml-omni-datafile 里的定义加上当前文件 "<!ENTITY" 的声明)
- "</" 之后，CTRL-X CTRL-O 会关闭最后打开的标签

XML 数据文件的格式 xml-omni-datafile

XML 数据文件保存在 'runtimepath' 下的 "autoload/xml" 目录中。Vim 发布在 "$VIMRUNTIME/autoload/xml" 目录下提供了示例数据文件。这些文件名有特别含义，命令里使用会使用这些名字。文件名应该唯一，否则以后会产生冲突。例如，xhtml10s.vim 代表 XHTML 1.0 Strict 标准的数据文件。

每个文件包含一个名字形如 g:xmldata_xhtml10s 的变量。它由两个部分组成：

1. "g:xmldata_" 通用前缀，所有数据文件都是如此
2. "xhtml10s" 文件名，描述 XML 的方言；会用作 :XMLns 命令的参数

第二部分必须和文件名完全一样。

该变量为字典 Dictionary 类型。键是标签名，而值是两个元素的 List 。列表中第一个元素也是列表，包含可能的子元素名称，第二个元素是字典 Dictionary ，键是属性名，而值是属性的可能值。例如：

```
let g:xmldata_crippled = {
\ "vimxmlentities": ["amp", "lt", "gt", "apos", "quot"],
\ 'vimxmlroot': ['tag1'],
\ 'tag1':
\ [ ['childoftag1a', 'childoftag1b'], {'attroftag1a': [],
\ 'attroftag1b': ['valueofattr1', 'valueofattr2']}],
\ 'childoftag1a':
\ [ [], {'attrofchild': ['attrofchild']}],
\ 'childoftag1b':
\ [ ['childoftag1a'], {'attrofchild': []}],
\ "vimxmltaginfo": {
\ 'tag1': ['Menu info', 'Long information visible in preview window']},
\ 'vimxmlattrinfo': {
\ 'attrofchild': ['Menu info', 'Long information visible in preview window']}}
```

该例应放到 "autoload/xml/crippled.vim" 文件中。可用于编写下述文件：

```
<tag1 attroftag1b="valueofattr1">
    <childoftag1a attrofchild>
            &amp; &lt;
    </childoftag1a>
    <childoftag1b attrofchild="5">
        <childoftag1a>
            &gt; &apos; &quot;
        </childoftag1a>
    </childoftag1b>
</tag1>
```

从该例中，我们可以看到四种特殊元素：

1. "vimxmlentities" - 特殊键，包含此 XML 方言的所有实体的列表。
2. 如果这个包含属性可能值的列表只有一个元素，而该元素和属性名一样，那么该属性被看作逻辑值，插入时使用 'attrname' 而不是 'attrname="'。
3. "vimxmltaginfo" - 特殊键，包含键为标签名、值为两元素列表的字典。值列表包含附加的菜单信息和长描述。

4. "vimxmlattrinfo" - 特殊键，包含键为属性名、值为两元素列表的字典。值列表包含附加的菜单信息和长描述。

注意：数据文件里的标签名 必须 不能包含命名空间的描述。示例见 xsl.vim。
注意：所有的数据和函数都作为全局变量/函数可以在任何地方访问，所以它们可以用于个人编辑用的函数。

DTD -> Vim dtd2vim

www 上有个脚本 dtd2vim ，能够解析 DTD 并为 Vim XML 全能补全建立 XML 数据文件。

```
dtd2vim: http://www.vim.org/scripts/script.php?script_id=1462
```

查看文件开始部分的详细用例。
该脚本需要 Perl 和:

```
perlSGML: http://savannah.nongnu.org/projects/perlsgml
```

命令

:XMLns {name} [{namespace}] :XMLns

Vim 需要知道要使用的数据文件和命名空间。 :XMLns 命令可以载入数据文件并把数据连接到合适的命名空间。第一个（必需的）参数是数据名（xhtml10s、xsl）。第二个参数是命名空间编码（h，xsl）。如果不使用第二个参数，那么将使用默认值--不声明命名空间。例如在 .xsl 文件中使用 XML 补全:

```
:XMLns xhtml10s
:XMLns xsl xsl
```

:XMLent {name} :XMLent

缺省，根据默认命名空间的数据文件补全实体（entity）。如果没有默认命名空间，应该用 XMLent 命令:

```
:XMLent xhtml10s
```

用法

在下述情况下（在前一部分的声明之后，| 代表当前光标位置):

```
<|
```

补全合适的 XHTML 标签。而:

```
<xsl:|
```

补全合适的 XSL 标签。

由 autoload 机制提供的 xmlcomplete.vim 脚本定义了函数 xmlcomplete#GetLastOpenTag()，在 XML 文件中，这个函数可用于取得最后打开的标签名（下例必须先定义 b:unaryTagsStack）：

```
:echo xmlcomplete#GetLastOpenTag("b:unaryTagsStack")
```

---

8. 插入模式命令 inserting

下列命令可以用来在缓冲区里插入新的文本。它们都可以撤销，也可以通过 "." 命令重复。

a
在光标后附加文本 [count] 次。如果光标在空行的第一列，启动插入模式。但在置位了 'virtualedit' 以后就不是！

A
在行尾附加文本 [count] 次。

<insert> 或 i insert <Insert>
在光标前插入文本 [count] 次。在插入模式里使用 CTRL-O 的时候，i_CTRL-O 不支持计数。

I
在本行第一个非空白字符之前插入文本 [count] 次。
如果 'cpoptions' 里有 'H' 标志位而本行只有空白，在最后一个空白前插入。

gI
在第一列插入文本 [count] 次。

gi
在当前缓冲区最近一次插入模式停止的位置继续插入文本。
该位置记在 '^ 位置标记里。如果标记在行末之后，和 "`^i" 有所差异。
该位置在插入/删除行时会自动修正。但 不 在插入/删除字符时被修正。
使用 :keepjumps 命令修饰符时，不改变 '^ 位置标记。

o
在光标下方开启新行，并插入文本，重复 [count] 次。
如果 'cpoptions' 里有 '#' 标志位，忽略计数。

O
在光标上方开启新行，并插入文本，重复 [count] 次。
如果 'cpoptions' 里有 '#' 标志位，忽略计数。

这些命令用以开始插入文本。你可以用 <Esc> 退出插入模式。关于插入模式里的其它特殊字符，见 mode-ins-repl 。[count] 的效果只有在退出插入模式以后才会发生。

如果打开 'autoindent'，新行的缩进从上一行得到。打开 'smartindent' 或 'cindent' 时，行的缩进根据 C 程序的要求自动调整。

'textwidth' 可以设置一行的最大宽度。如果一行过长，在添加字符时会自动添加换行符。

---

9. Ex 插入命令 inserting-ex

:{range}a[ppend][!] :a :append

在指定行下方添加若干行。如果没有给出 {range}，文本会在当前行之后插入。
加入 [!] 切换此命令执行时的 'autoindent'。

:{range}i[nsert][!] :i :in :insert

在指定行上方添加若干行。如果没有给出 {range}，文本会在当前行之前插入。
加入 [!] 切换此命令执行时的 'autoindent'。

这两个命令会继续要求行，直到你输入了只包含 "." 的一行。小心反斜杠开始的行，见 line-continuation 。

Ex 模式（见 -e ）下，行尾的反斜杠可用来插入 NUL 字符。所以要使一行以反斜杠结尾，用两个反斜杠。也就是说仅在行尾情况下，反斜杠数目减半。

注意：这些命令不能和 :global 或 :vglobal 一起使用。":append" 和 ":insert" 在 ":if" 和 ":endif"、":for" 和 ":endfor" 还有 ":while" 和 ":endwhile" 之间不能很好的工作。

:star[tinsert][!] :start :startinsert

在执行完本命令后，启动插入模式。和普通模式下输入 "i" 类似。如果包含 !，和 "A" 类似，附加到行后。否则，就从光标当前位置开始插入。
注意 在函数或者脚本里使用本命令时，插入只会在函数和脚本结束的时候才会开始。
此命令不能在 :normal 里使用。

:stopi[nsert] :stopi :stopinsert

尽快停止插入模式。和在插入模式时输入 <Esc> 类似。可以用在自动命令里。示例：

```
:au BufEnter scratch stopinsert
```

:startr[eplace][!] replacing-ex :startreplace

在执行完本命令后，启动替换模式。和普通模式下输入 "R" 类似。如果包含 !，和 "$R" 类似（也就是，从行尾开始替换模式）。否则，从光标当前位置开始替换。
注意 在函数或者脚本里使用本命令时，替换只会在函数和脚本结束的时候才会开始。

:startg[replace][!] :startgreplace

和 :startreplace 完全类似，用虚拟替换模式，和使用

gR 类似。

---

10. 插入文件 inserting-file

:r :re :read

:r[ead] [++opt] [name]
在光标下方插入文件 [name] (缺省: 当前文件)。
++opt 说明 [++opt] 可能的取值。

:{range}r[ead] [++opt] [name]
在指定行下方插入文件 [name] (缺省: 当前文件)。
++opt 说明 [++opt] 可能的取值。

:r! :read!

:[range]r[ead] [++opt] !{cmd}
执行 {cmd} 并把它的标准输出插入到光标下方。临时文件会建立来保存命令输出的结果，并被读到缓冲区里。
'shellredir' 用来保存命令的输出结果，它可以设置是否包含标准错误的输出。{cmd} 的执行和 ":!{cmd}" 类似，任何的 '!' 会被替换成以前的命令 :! 。
++opt 说明 [++opt] 可能的取值。

这些命令插入文件的内容，或者命令的输出结果到缓冲区里。两者都可以撤销。但不能用 "." 命令重复。它们是基于行工作的，插入从光标所在行或指定行的下方开始。要在第一行之上插入文本，使用命令 ":0r {name}"。

在 ":read" 命令之后，光标留在第一个新行的第一个非空白处。和 Ex 模式不一样。那里光标留在最后一个新行上 (对不起，那是为了和 Vi 兼容)。

如果文件名字通过 ":r" 给出，它成为轮换文件。这可以用来，比如说，你想编辑那个文件的时候: ":e! #"。该特性可以通过删除 'cpoptions' 选项里的 'a' 标志位来关闭。

[++opt] 参数里，有一个是 ":read" 专用的: ++edit 参数。当 ":read" 命令就像编辑文件一样把文件读入到缓冲区时，这个参数很有用。在空缓冲区上使用如下命令:

```
:read ++edit filename
```

效果是 'fileformat'、'fileencoding'、'bomb' 等选项根据 "filename" 的检测进行设置。注意 会留下一行空行，你也许想把它删掉。

file-read

'fileformat' 选项设置文件的 <EOL> 风格:

'fileformat'	字符	名称
"dos"	<CR><NL> 或 <NL>	DOS 格式
"unix"	<NL>	Unix 格式
"mac"	<CR>	Mac 格式

以前使用 'textmode'。现在已经废弃了。

如果 'fileformat' 为 "dos"，在 <NL> 之前的 <CR> 被忽略，而在文件尾部的 CTRL-Z 被忽略。

如果 'fileformat' 为 "mac"，文件里的 <NL> 被内部表示为 <CR>。这是为了避免和用来表示 <NUL> 的 <NL> 引起混淆。见 CR-used-for-NL 。

如果 'fileformats' 选项不为空，Vim 试图识别 <EOL> 的类型 (见 file-formats)。

不过，'fileformat' 选项的值不会被改变，检测到的格式只会在读入文件时使用。
'fileencodings' 与此情形类似。

在非 MS-DOS 和 Win32 系统上，消息 "[dos format]" 会在读入 DOS 格式的文件时给出，以提醒你发生了不寻常的事情。
在 Macintosh 和 Win32 系统上，消息 "[unix format]" 会在读入 Unix 格式的文件时给出。
在非 Macintosh 的系统上，消息 "[Mac format]" 会在读入 Mac 格式的文件时给出。

关于如何使用 ":r !" 的一个例子:

```
:r !uuencode binfile binfile
```

该命令读入 "binfile"，用 uuencode 进行编码，并读入当前缓冲区。可以用于编辑包含附带的二进制的文件的 e-mail。

read-messages

在读入文件时，Vim 会显示消息，显示读入文件的相关信息。以下的表格给出一些项目的解释。其它的项目都不言自明。使用长格式还是短格式取决于 'shortmess' 选项的设置。

长	短	含义
[readonly]	{RO}	文件被写保护
[fifo/socket]		使用流
[fifo]		使用 fifo 流
[socket]		使用套接字 (socket) 流
[CR missing]		使用 "dos" 'fileformat' 读入文件的时候出现没有前导的 CR 的 NL
[NL found]		使用 "mac" 'fileformat' 读入文件的时候出现 NL (可能是 "unix" 格式)
[long lines split]		至少一行以上被分割
[NOT converted]		期待从 'fileencoding' 到 'encoding' 的转换但是做不到
[converted]		从 'fileencoding' 到 'encoding' 的转换完成
[crypted]		文件被解密
[READ ERRORS]		不是文件所有部分都被成功读入

vim:tw=78:ts=8:noet:ft=help:norl:

change.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar

译者：Willis
http://vimcdoc.sf.net

本文档描述删除和修改文本的命令。在本文中，修改文本意味着用单个命令删除部分文本并将其替换成其他文本。所有本文描述的命令都可以被撤销。其中非命令行 (Ex) 的命令可以用 "." 命令重复。

1. 删除文本　deleting
2. 删除并插入　delete-insert
3. 简单修改　simple-change　changing
4. 复杂修改　complex-change
   4.1 过滤命令　filter
   4.2 替代　:substitute
   4.3 搜索与替代　search-replace
   4.4 调整制表　change-tabs
5. 复制并移动文本　copy-move
6. 文本排版　formatting
7. 文本排序　sorting

关于插入文本部分，参阅 insert.txt 。

---

1. 删除文本　deleting　E470

["x]<Del>　或　<Del>　x　dl

["x]x　删除 [count] 个光标之下和之后的字符 [到寄存器 x 里] (非 linewise 行动作)。和 "dl" 相同。
<Del> 键不使用 [count] 计数。相反，如果有计数，它删除该计数的最后一位。
如果 <Del> 键的操作与你预期的不符，参见 :fixdel 。关于如何删除换行符 (连接行)，参见 'whichwrap' 。

X　dh

["x]X　删除 [count] 个光标之前的字符 [到寄存器 x 里] (非 linewise 行动作)。和 "dh" 相同。另见 'whichwrap' 。

d

["x]d{motion}　删除 {motion} 动作跨越的文本 [到寄存器 x 里]。例外见下。

dd

["x]dd　删除 [count] 行 [到寄存器 x 里]， linewise 行动作。

D

["x]D　删除光标所在的字符到行尾及其后的 [count]-1 行 [到寄存器 x 里] (非 linewise 行动作)。和 "d$" 同义。
如果 'cpoptions' 里有 '#' 标志位，忽略计数。

{Visual}["x]x　或　v_x　v_d　v_<Del>

{Visual}["x]d 或
{Visual}["x]<Del> 删除高亮文本 [到寄存器 x 里] (关于 {Visual} 见 Visual-mode)。

{Visual}["x]CTRL-H 或 v_CTRL-H v_<BS>
{Visual}["x]<BS> 在选择模式时，删除高亮文本 [到寄存器 x 里]。

{Visual}["x]X 或 v_X v_D v_b_D
{Visual}["x]D 删除高亮行 [到寄存器 x 里] (关于 {Visual} 见 Visual-mode)。在可视列块模式里，"D" 删除高亮的文本直到行尾。

:d :de :del :delete :dl :dp
:[range]d[elete] [x] 删除 [range] 范围的行 (缺省: 当前行) [到寄存器 x 里]。

注意 这些奇异的缩写:

:dl	删除并列出 (list)
:dell	同上
:delel	同上
:deletl	同上
:deletel	同上
:dp	删除并显示 (print)
:dep	同上
:delp	同上
:delep	同上
:deletp	同上
:deletep	同上

:[range]d[elete] [x] {count}
从 [range] 指定的范围开始，删除 {count} 行 (缺省: 当前行 cmdline-ranges) [到寄存器 x 里]。

这些命令删除文本。你可以用 . 命令重复它们 (除了 :d)，也可以撤销它们。用可视模式可以删除文本列块。关于寄存器的解释，参见 registers 。

d{motion} 命令的一个特例: 如果动作不面向行，动作的开始和结束处不在同一行，开始位置之前只有空白，并且结束位置之后没有非空白的话，该删除会转为行动作。这意味着删除命令可能删除你还想保留的空白行。 o_v 操作符强制该动作面向字符。

如果 'cpoptions' 包括 'E' 标志位，删除文本里的空区域 (例如，在首列上 "d0") 会报错。

J
J 连接 [count] 行，但至少包含两行。删除缩进，插入不多于两个的空格 (见下)。在缓冲区末行此操作会失败。[count] 如果过多，会自行缩减至余下可用的行。

v_J
{Visual}J 连接高亮行，但至少包含两行。删除缩进，插入不多于两个的空格 (见下)。

gJ
gJ 连接 [count] 行，但至少包含两行。不插入或删除任何空格。

v_gJ

{Visual}gJ 连接高亮行，但至少包含两行。不插入或删除任何空格。

:j :join

[range]j[oin][!] [flags]
连接 [range] 范围的行。和 "J" 相同，但如有 [!] 时，连接不插入或删除任何空格。如果 [range] 包括相同的开始和结束行，该命令不做任何事。缺省行为是连接当前行与下一行。
[flags] 部分可参见 ex-flags 。

:[range]j[oin][!] {count} [flags]
连接 [range] 开始的 {count} 行（缺省：当前行 cmdline-ranges ）。和 "J" 相同，但如有 [!] 时，连接不插入或删除任何空格。
[flags] 部分可参见 ex-flags 。

这些命令删除行间的 <EOL>，即换行符，从而实际上使多行连接成一行。除了 :j 之外，你可以重复这些命令或者撤销之。

这些命令，除了 "gJ" 以外，插入一个空格以代替 <EOL>，除非当前行的结尾已经有空白或者下一行以 ')' 开始。这些命令，除了 "gJ" 之外，也删除下一行所有开头的空白。如果 'joinspaces' 选项打开，这些命令在 '.'、'!' 或者 '?' 之后插入两个空格（但如果 'cpoptions' 包括 'j' 标志位，只有在 '.' 之后插入两个空格）。
'formatoptions' 里的 'B' 和 'M' 标志位改变在多字节字符之前或者之后插入空格的行为 fo-table 。

'[标记指向被连接的首行之原行尾处，'] 则指向合并后的行尾。

---

2. 删除并插入 delete-insert replacing

R

R 进入替换模式：每个输入的字符替代一个现有的字符，从光标所在的位置开始。重复输入的文本 [count] - 1 次。参见 Replace-mode 以了解相关详情。

gR

gR 进入虚拟替换模式：每个输入的字符替代屏幕位置一个现有的字符。因而，一个 <Tab> 可以一次替换若干个字符。重复输入的文本 [count]-1 次。参见 Virtual-Replace-mode 以了解相关详情。

c

["x]c{motion} 将 {motion} 跨过的文本删除 [到寄存器 x 里] 并开始插入。如果 'cpoptions' 包括 'E' 标志位并且没有任何文本实际需要删除时（例如，"cTx" 而光标刚在 'x' 之后的时候），报错并拒绝开始插入模式（这与 Vi 兼容）。如果 'cpoptions' 没有 'E'，"c" 命令总是启动插入模式，即使没有文本要删除也是如此。

cc

["x]cc 删除 [count] 行 [到寄存器 x 里] 并开始插入。

linewise 行动作。如果置位 'autoindent'，保留首行的缩进。

C
["x]C 从当前光标位置删除到行尾，再删除 [count]-1 行 [到寄存器 x 里]，并开始插入。和 c$ 等价（非 linewise 行动作）。

s
["x]s 删除 [count] 个字符 [到寄存器 x 里]，并开始插入。(s 代表替代 (Substitute))。和 "cl" 等价（非 linewise 行动作）。

S
["x]S 删除 [count] 行 [到寄存器 x 里] 并开始插入。和 "cc" 等价 linewise 行动作。

v_c v_s
{Visual}["x]c or
{Visual}["x]s 删除高亮文本 [到寄存器 x 里]，并开始插入（关于 {Visual} 参见 Visual-mode ）。

v_r
{Visual}["x]r{char} 把所有高亮的字符替换成 {char}。

v_C
{Visual}["x]C 删除高亮行 [到寄存器 x 里] 并开始插入。在可视列块模式下有所不同 v_b_C 。

v_S
{Visual}["x]S 删除高亮行 [到寄存器 x 里] 并开始插入（关于 {Visual} 参见 Visual-mode ）。

v_R
{Visual}["x]R 现在和 {Visual}["x]S 相同。以后的版本可能会有所改变。

注意：
- 你可以用 <Esc> 退出插入和替换模式。
- 参见 "插入和替换模式" 一节 mode-ins-repl 以了解其他这些模式下的特殊字符。
- [count] 的效果只有在 Vim 退出插入或替换模式时才会体现。
- 当 'cpoptions' 选项包含 '$' 并且所做的修改在一行之内，Vim 继续显示被删除的文本，并在最后一个被删除的字符之后加上一个 '$'。

关于寄存器的解释，参见 registers 。

替换模式和插入模式类似，除了每输入一个字符同时删除一个字符以外。如果到达行尾，Vim 把其后的字符插入在行尾（和插入模式相同）。在替换模式下，退格键恢复原来的文本（如果有的话）。(参见 "插入和替换模式" 一节 mode-ins-repl)。

cw cW
特殊情况：如果光标在单词内部，"cw" 和 "cW" 不包含单词后的空白而只修改到词尾。这是因为 Vim 把 "cw" 解释为 修改-单词，而单词并不包括其后的空白。
{Vi: 在其后还有空白的空白字符上 "cw" 只修改第一个空白；这也许是一个漏洞，因为 "dw" 删除所有空白；用 'cpoptions' 里的 'w' 标志位来设置类似于 Vi 的工作方式}

如果你希望 "cw" 包括单词之后的空格，使用如下的映射：

```
:map cw dwi
```

或者，用 "caw"（见 aw ）。

:c :ch :change

:{range}c[hange][!] 用另外的文本替换若干文本行。输入只包含 "." 的行结束替换。如果没有 {range}，该命令只置换当前行。
加上 [!] 后，在本命令的执行期间切换 'autoindent'。

---

3. 简单修改

simple-change

r

r{char} 把光标下的字符换成 {char}。如果 {char} 是 <CR> 或者 <NL>，则把该字符变成换行符。要换成一个真正的 <CR>，使用 CTRL-V <CR>。CTRL-V <NL> 则换成 <Nul>。

若 {char} 为 CTRL-E 或 CTRL-Y，使用下一行或上一行的字符来代替，就像 i_CTRL-E 和 i_CTRL-Y 一样。这里计数也适用，所以 10r<C-E> 从下一行复制 10 个字符。

如果你给出 [count]，Vim 替换 [count] 个字符为 [count] 个 {char}。不过如果 {char} 为 <CR> 或者 <NL>，Vim 只插入一个换行符：\"5r<CR>\" 替换五个字符为一个换行符。

当 {char} 为 <CR> 或者 <NL>，Vim 会自动缩进。这和先执行删除要替换的字符在执行 "i<CR><Esc>" 类似。

{char} 可以输入二合字母 digraph-arg 。

:lmap 映射会应用在 {char} 上。插入模式下的 CTRL-^ 命令可以切换这一功能 i_CTRL-^ 。

参见 utf-8-char-arg 以了解在 'encoding' 为 Unicode 编码时如果使用组合字符。

gr

gr{char} 替换光标下的虚拟字符。替换发生在屏幕位置而不是文件位置上。参见 gR 和 Virtual-Replace-mode 以了解相关详情。和 r 一样，可以使用计数。{char} 输入的方式也相同。

digraph-arg

普通模式下的命令，像 r 和 t ，需要一个单字符的参数。如果 'cpo' 不包含 'D' 标志位，该字符可以用 digraphs 那样的方式输入：先键入 CTRL-K 然后输入二合字母的两个字符 {仅当编译时加入 +digraphs 特性有效}。

case

下面的命令根据当前的 locale 改变字母的大小写。参见 :language 。这里会用到 LC_CTYPE 的值。

~

~ 'notildeop' 选项：切换光标下字符的大小写，并把光标向右移。如果给出 [count]，应用在那么多数目的字符上。

~{motion} 'tildeop' 选项：切换 {motion} 跨越的文本的大小写。

g~{motion}　　切换 {motion} 跨越的文本的大小写。　　g~

g~g~
g~~　　切换当前行的大小写。　　g~g~　g~~

{Visual}~　　切换高亮文本的大小写（关于 {Visual} 见 Visual-mode ）。　　v_~

{Visual}U　　使高亮文本成为大写（关于 {Visual} 见 Visual-mode ）。　　v_U

gU{motion}　　使 {motion} 跨越的文本成为大写。　　gU　uppercase
例如：

```
:map! <C-F> <Esc>gUiw`]a
```

可以用在插入模式下：按 CTRL-F 使光标之前的单词成为大写。这使得输入大写单词很方便，只要输入小写单词再一次转换就行了。

gUgU
gUU　　使得当前行成为大写。　　gUgU　gUU

{Visual}u　　使高亮文本成为小写（关于 {Visual} 见 Visual-mode ）。　　v_u

gu{motion}　　使 {motion} 跨越的文本成为小写。　　gu　lowercase

gugu
guu　　使得当前行成为小写。　　gugu　guu

g?{motion}　　用 Rot13 对 {motion} 跨越的文本进行编码。　　g?　rot13

{Visual}g?　　用 Rot13 对高亮文本进行编码（关于 {Visual} 见 Visual-mode ）。　　v_g?

g?g?
g??　　用 Rot13 对当前行进行编码。　　g?g?　g??

要以标题大写（title cap）方式改写一行，亦即使每个单词的首字母大写：

```
:s/\v<(.)(\w*)/\u\1\L\2/g
```

递增与递减

CTRL-A　　把当前光标之上或之后的数值或者字母加上 [count]。　　CTRL-A

v_CTRL-A

{Visual}CTRL-A　　给高亮文本内的数值或者字母加上 [count]。

v_g_CTRL-A
{Visual}g CTRL-A　　给高亮文本内的数值或者字母加上 [count]。如果高亮超过一行，每行会加上额外的 [count] (即生成 [count] 为增量的递增数列)。
例如，假定有如下数字的列表：

```
1.
1.
1.
1.
```

移动至第二个 "1."，可视地选择后三行，按 g CTRL-A 会生生成：

```
1.
2.
3.
4.
```

CTRL-X
CTRL-X　　把当前光标之上或之后的数值或者字母减去 [count]。

v_CTRL-X
{Visual}CTRL-X　　给高亮文本内的数值或者字母减去 [count]。

MS-Windows 上，此键被映射到剪切可视文本 dos-standard-mappings 。要屏蔽该映射，可用：

```
silent! vunmap <C-X>
```

v_g_CTRL-X
{Visual}g CTRL-X　　给高亮文本内的数值或者字母减去 [count]。如果高亮超过一行，每行会减去额外的 [count] (即生成 [count] 为增量的递减数列)。

CTRL-A 和 CTRL-X 命令可用于：
- 带符号或无符号十进制数
- 无符号二进制、八进制和十六进制数
- 字母

这取决于 'nrformats' 选项：
- 当 'nrformats' 包括 "bin" 时，Vim 假设 '0b' 或 '0B' 开始的数值为二进制。
- 当 'nrformats' 包括 "octal" 时，Vim 假设 '0' 开始的数值为八进制，除非该数值里包含 '8' 或 '9'。其他的数值为十进制，并可以在开始带一个可选的负号。
  如果光标已经在数值上，命令应用于该数值；否则，应用于光标右侧的数值。
- 当 'nrformats' 包括 "alpha" 时，Vim 会改变光标之上或之后的字母。这可用于构造字母编号的列表。
- 当 'nrformats' 包括 "hex" 时，Vim 假设 '0x' 或者 '0X' 开始的数值为十六进制。这个数值最右端的字母决定所产生十六进制数值的大小写。如果当前数值没有字母，Vim 使用上一次检测到的大小写。

增减操作会考虑十进制开始的负号。二进制、八进制和十六进制值则不会。要忽略正负号，可视地选择数值本身。然后再应用 CTRL-A 或 CTRL-X。

对零开头的数值 (包括八进制和十六进制的)，Vim 尽可能保留相同数量的字符。CTRL-A 在 "0077" 上产生 "0100"，CTRL-X 在 "0x100" 上产生 "0x0ff"。

有一个例外：在发现某数值以零开始但不是八进制（包含 '8' 或 '9'），而 'noformats' 却包含 "octal" 的时候，引导的零会被删除，以免结果被误认为八进制。

注意 如果 'nrformats' 包括 "octal"，开头有零的十进制数会产生错误，因为会和八进制数产生混淆。

同样类似的要 注意，如果 'nrformats' 包括 "bin"，开头有 '0x' 或 '0X' 的二进制数被认作十六进制，因为 '0b' 是合法的十六进制数字。

CTRL-A 命令在宏命令里很有用。例如：使用以下的步骤构造一个数字编号的列表。

1. 建立第一个列表项。确保它以数字开始。
2. qa - 用寄存器 'a' 开始记录
3. Y - 抽出这个列表项
4. p - 把该项的一个副本放置在下一行上
5. CTRL-A - 增加计数
6. q - 停止记录
7. <count>@a - 重复抽出、放置和增加计数操作 <count> 次

将文本左移或右移 shift-left-right

<{motion} 将 {motion} 跨越的行左移 'shiftwidth' 列。 <

如果打开 'vartabstop' 特性且 'shiftwidth' 选项设为零，缩进量在行中第一个非空白字符处计算。

<< 将 [count] 行左移 'shiftwidth' 列。 <<

{Visual}[count]< 将高亮行左移 [count] 个 'shiftwidth' 列（关于 {Visual} 见 Visual-mode ）。 v_<

>{motion} 将 {motion} 跨越的行右移 'shiftwidth' 列。 >

如果打开 'vartabstop' 特性且 'shiftwidth' 选项设为零，缩进量在行中第一个非空白字符处计算。

>> 将 [count] 行右移 'shiftwidth' 列。 >>

{Visual}[count]> 将高亮行右移 [count] 个 'shiftwidth' 列（关于 {Visual} 见 Visual-mode ）。 v_>

:[range]< 将 [range] 指定的行左移 'shiftwidth' 列。多个 '<' 左移多个 'shiftwidth' 列。 :<

:[range]< {count} 左移 [range] 开始的 {count} 行 'shiftwidth' 列（缺省从当前行 cmdline-ranges ）。多个 '<' 左移多个 'shiftwidth' 列。

:[range]le[ft] [indent] 左对齐 [range] 指定的行。设置缩进距离为 [indent] (缺省为 0)。

:[range]> [flags] :>
将 [range] 指定的行右移 'shiftwidth' 列。多个 '>' 右移多个 'shiftwidth' 列。
[flags] 部分可参见 ex-flags 。

:[range]> {count} [flags]
右移 [range] 开始的 {count} 行 'shiftwidth' 列 (缺省从当前行 cmdline-ranges 开始)。多个 '>' 右移多个 'shiftwidth' 列。
[flags] 部分可参见 ex-flags 。

">" 和 "<" 命令可以用来方便地调整程序的缩进。使用 'shiftwidth' 选项设置这些命令增加或者减少的空白的数量。通常，'shiftwidth' 选项是 8，但你也可以设置为，比如说，3，使得缩进更小些。如果已经没有缩进，左移命令会停止。另一方面，右移命令则不会影响空白行。

如果 'shiftround' 选项打开，缩进距离被取整到 'shiftwidth' 的倍数。

如果 'smartindent' 选项打开，或者 'cindent' 打开并且 'cinkeys' 包含带零值的 '#'，右移不影响 '#' 开始的行 (这些应该是 C 预处理行，它们应该保持在第一列)。用 'cino' 选项可改变此行为，参见 cino-# 。

如果 'expandtab' 选项关闭 (这是缺省值)， Vim 尽可能使用 <Tab> 来构成缩进。你可以用 ">>><<" 来把缩进尽可能从空格替换成同样缩进距离的 <Tab> (如有需要，还有少量的空格)。如果 'expandtab' 选项打开，Vim 只使用空格。这样你可以使用 ">>><<" 把 <Tab> 替换成空格 (或者可以用 :retab!)。

要移动一行多个 'shiftwidth' 列，使用可视模式或者 : 命令。例如:

```
Vjj4>          右移三行四个缩进位
:<<<           左移当前行三个缩进位
:>> 5          右移五行两个缩进位
:5>>           右移第五行两个缩进位
```

---

4. 复杂修改 complex-change

4.1 过滤命令 filter

过滤程序是一个接受文本作为标准输入，作某些修改，并把结果放到标准输出的程序。你可以用下面的命令把若干文本发送给过滤程序，然后用过滤的输出结果替换。一个过滤程序的例子是 "sort"，按字母顺序给行排序；还有 "indent"，排版 C 程序文件 (你需要一个能以过滤程序方式工作的版本，并非所有的版本都可以)。'shell' 选项指定 Vim 使用的外壳程序，用以执行过滤程序 (另见 'shelltype' 选项)。你可以用 "." 重复过滤命令。Vim 不会识别 :! 命令之后的注释 (用 '"' 开始)。

!{motion}{filter} !
将 {motion} 跨越的行用外部程序 {filter} 过滤。

!!{filter} !!
将 [count] 行用外部程序 {filter} 过滤。

v_!
{Visual}!{filter}　　将高亮行用外部程序 {filter} 过滤。(关于 {Visual} 见 Visual-mode)。

:{range}![!]{filter} [!][arg]　　:range!
将 {range} 指定的行用外部程序 {filter} 过滤。Vim 把可选的感叹号替换成最后一次使用的命令，并附加上可选的参数 [arg]。Vim 把过滤命令的输出保存到临时文件，并把文件内容读到一个缓冲区里 tempfile 。Vim 使用 'shellredir' 选项把过滤程序的结果重定向到临时文件。不过，如果关闭了 'shelltemp' 选项且可以的话，使用管道机制 (Unix 上)。
如果 'cpoptions' 包含 'R' 标志位，过滤行里的位置标记被删除，除非使用了 :keepmarks 命令。例如:

```
:keepmarks '<,'>!sort
```

如果过滤后的行数变少，删去的行里的位置标记无论如何不会保存。

=
=={motion}　　把 {motion} 跨越的行用 'equalprg' 选项指定的外部程序过滤。如果 'equalprg' 选项为空 (缺省)，使用内部的排版机制 C-indenting 和 'lisp' ，除非 'indentexpr' 非空，此时使用 indent-expression 机制来代替。反之，如果 Vim 编译时没有包含内部排版机制，最后手段是使用外部的 "indent" 程序。

==
==　　和 ={motion} 类似，过滤 [count] 行。

v_=
{Visual}=　　和 ={motion} 类似，过滤高亮行。

tempfile　setuid
Vim 使用临时文件来处理过滤、执行比较和用于 tempname()。Unix 上，该文件会放在一个私人的目录里 (只能被当前用户访问)，以防堵安全上的漏洞 (例如，符号链接攻击或文件被其他用户读取等)。Vim 退出时，自动删除该目录及之下的所有文件。如果 Vim 本身设置了 setuid 位，这样或许会有问题。临时文件由 setuid 用户拥有，但过滤程序以原来用户的权限执行。
MS-Windows 上，使用以下目录中首个可用者: $TMP、$TEMP、c:\TMP、c:\TEMP。
Unix 上，使用的目录依次为: $TMPDIR、/tmp、当前目录、$HOME。
MS-Windows 上，使用 GetTempFileName() 系统函数。
其它系统上使用 tmpnam() 库函数。

4.2 替代

:substitute
:s　:su
:[range]s[ubstitute]/{pattern}/{string}/[flags] [count]
对 [range] 指定的行把 {pattern} 的匹配替代成 {string}。
关于 {pattern}，参见 pattern 。
{string} 可以是按本义出现的字符串，也可包含特殊字符。参见 sub-replace-special 。
如果不指定 [range] 和 [count]，仅在当前行进行替代。

如果指定 [count]，在 [range] 最后一行开始的 [count] 行进行替代。如果不指定 [range] ，则从当前行开始。
E939
[count] 必须为正数。另见 cmdline-ranges 。

关于 [flags]，参见 :s_flags 。

:[range]s[ubstitute] [flags] [count]
:[range]&[&][flags] [count] :&
使用最后一次 :substitute 相同的模式和替代字符串，但不包括相同的标志位。你可以另加 [flags] (见 :s_flags)。
注意 在 :substitute 之后，不能使用 '&' 标志位。它被认为是一个模式分隔符。
:substitute 和 'c'、'g'、'i'、'I' 和 'r' 标志位之间的空格不是必需的，但在脚本里为了避免混淆起见，最好保留它。

:[range]~[&][flags] [count] :~
使用最后一次 :substitute 相同的替代字符串，但使用最近使用的搜索模式作为匹配模式。这类似于 :&r 。
关于 [flags]，参见 :s_flags 。

& &
等价于 :s (重复上次的 substitute)。注意 这里不记住标志位，所以实际工作方式可能不尽相同。你可以用 :&& 来保持相同的标志位。

g& g&
等价于 :%s//~/& (在所有行上重复上次的 substitute 并使用相同的标志位，但使用前次的匹配模式)。
例如，如果先用了替代 :s/pattern/repl/flags 然后 /search 搜索了其他模式，g& 会执行 :%s/search/repl/flags 。
助记：全局 (global) 替代 (substitute)。

:[range]sno[magic] ... :snomagic :sno
和 :substitute 相同，但总使用 'nomagic'。

:[range]sm[agic] ... :smagic :sm
和 :substitute 相同，但总使用 'magic'。

:s_flags
你可以在 substitute 命令里使用以下标志位：

[&] :&&
必须是首个使用的标志位：保留和上次 substitute 相同的标志位。例如：

```
:&&
:s/this/that/&
```

注意 :s 和 :& 不保留标志位。

[c] 确认每个替代。Vim 高亮匹配的字符串 (如果使用了 hl-IncSearch)。你可以输入：
:s_c
'y' 来替代这次匹配

'l'　来替代这次匹配并退出（助记："last"，最后）
'n'　来跳过这次匹配
<Esc>　来退出替代过程
'a'　来替代这次和以后所有的匹配
'q'　来退出替代过程
CTRL-E　来上卷屏幕
CTRL-Y　来下卷屏幕

如果 'edcompatible' 选项打开，Vim 记住 [c] 标志位并在每次使用它时切换是否确认，但在给出一个新的匹配模式时复位。

:s_e

[e]　如果模式搜索不成功，不给出错误信息。因为没产生错误，映射的过程得以继续。这主要用来防止在映射执行过程的无匹配（"No match"）错误中断映射。不过，Vim 不会抑制以下的错误信息：

Regular expressions can't be delimited by letters（正规表达式不能以字母分隔）
\ should be followed by /, ? or &（\ 必须后面跟 /、? 或者 &）
No previous substitute regular expression（没有上次的替代正规表达式）
Trailing characters（结尾有多余的字符）
Interrupted（中断）

:s_g

[g]　对行内所有的匹配进行替代。如果没有这个参数，替代只对每行的第一个匹配进行。如果 'edcompatible' 选项打开，Vim 记住这个标志并在你每次使用该标志时切换，但在给出一个新的匹配模式时复位。如果 'gdefault' 选项打开，这个标志缺省打开，而 [g] 参数关闭之。

:s_i

[i]　忽略模式的大小写。不使用 'ignorecase' 和 'smartcase' 选项。

:s_I

[I]　不忽略模式的大小写。不使用 'ignorecase' 和 'smartcase' 选项。

:s_n

[n]　报告匹配的次数，并不实际进行替代。忽略 [c] 标志位。匹配报告的发生就像 'report' 总是为零那样。可用于 count-items 。
如果使用 \= sub-replace-expression ，该表达式对每一个匹配的执行都在 sandbox 里进行。

[p]　显示包含最后一次替代的行。 :s_p

[#]　类似 [p]，且在前面加上行号。 :s_#

[l]　类似 [p]，但显示的方式类似于 :list 。 :s_l

:s_r

[r]　仅对于不带参数的 :& 和 :s 有用。:&r 和 :~ 工作的方式相同：如果匹配模式为空，使用上一次使用的搜索模式，而不是上一次的 substitute 或者 :global 所使用的模式。如果最近一次使用搜索的命令就是 substitute 或者 :global ，那就没有区别了。如果最近的命令的是 "/" 那样的搜索命令，使用那个命令的搜索模式。
带参数的 :s ，则不论如何，总是如此：

```
:s/blue/red/
```

```
/green
:s//red/   或  :~   或  :&r
```
最后的命令把 "green" 替代成 "red"。
```
:s/blue/red/
/green
:&
```
最后的命令把 "blue" 替代成 "red"。

注意 这里没有标志位可以改变模式的 "魔术性" (magicness)。你可以用别的命令或 /\v 之类。原因是标志位只能在跳过模式之后才能找到，而要跳过模式，必须先知道模式的 "魔术性"。第二十二条军规！

如果 substitute 命令所用的 {pattern} 为空，该命令使用上次的 substitute 或者 :global 命令用过的模式。如果不存在，但有上次搜索模式，使用之。如果有 [r] 标志位，该命令使用上次 substitute、 :global 或者搜索命令使用的模式。

如果 {string} 省略，替代命令假定它为空。这样就把匹配文本删除了。这时，{pattern} 之后的分隔符也可省略。例如:
```
:%s/TESTING
```
删除所有行上的 "TESTING"，但每行只删一个。

为了和 Vi 的兼容性，有两个例外:
"\/{string}/" 和 "\?{string}?" 等同于 "//{string}/r"。
"\&{string}&" 等同于 "//{string}/"。

E146
除了用 '/' 来包围模式和替代字符串之外，你可以使用任何其他的单字节字符，除了字母、数字、'\'、'"' 或 '|' 之外。这可以用于 '/' 是搜索模式或替代字符串一部分的场合。例如:
```
:s+/+//+
```

关于模式的定义，参见 pattern 。在可视列块模式下，如果模式中使用了 /\%V ，那么替代只在列块内进行，否则它作用于整行。

sub-replace-special :s\=
{string} 以 "\=" 开始时，它被作为表达式来执行，参见 sub-replace-expression 。你可以用此功能实现复杂替换，也可用来使用特殊字符。

否则，{string} 字符串里的字符有如下的特殊含义:

:s%
如果 {string} 等于 "%"，且 'cpoptions' 选项包含 '/'，重复使用上次替代命令的 {string}，见 cpo-/

magic	nomagic	动作	
&	\&	替代为完整的匹配	s/\&
\&	&	替代为 &	
	\0	替代为完整的匹配	\0 s/\0
	\1	替代为匹配的第一个 () 里面的内容	s/\1
	\2	替代为匹配的第二个 () 里面的内容	s/\2
	..	..	s/\3
	\9	替代为匹配的第九个 () 里面的内容	s/\9
~	\~	替代为前一个 substitute 的替代字符串	s~
\~	~	替代为 ~	s/\~
	\u	下一个字符成为大写	s/\u
	\U	其后字符成为大写，直到 \E 出现	s/\U

\l	下一个字符成为小写	s/\l
\L	其后字符成为小写，直到 \E 出现	s/\L
\e	结束 \u、\U、\l 和 \L（注意：不是 <Esc>!）	s/\e
\E	结束 \u、\U、\l 和 \L	s/\E
<CR>	把该行在此位置一分为二 (<CR> 以 CTRL-V <Enter> 方式输入)	s<CR>
\r	同上	s/\r
\<CR>	插入一个回车 (CTRL-M) (<CR> 以 CTRL-V <Enter> 方式输入)	s/\<CR>
\n	插入一个 <NL> (文件里的 <NUL>) (此处并不是换行)	s/\n
\b	插入一个 <BS>	s/\b
\t	插入一个 <Tab>	s/\t
\\	插入单个反斜杠	s/\\
\x	其中 x 是上面没提到的任何一个字符： 保留作将来的扩展	

这里的特殊含义也用于 substitute() 函数的第三个参数 {sub}，除了若干例外：
- % 总是插入百分号，与 'cpoptions' 无关。
- 总是假定有魔术性，与 'magic' 无关。
- ~ 总是插入波浪符。
- <CR> 和 \r 插入回车 (CTRL-M)。
- \<CR> 没有特殊意义，只是 \x 之一。

示例：

:s/a|b/xxx\0xxx/g	修改 "a b"	为	"xxxaxxx xxxbxxx"
:s/\([abc]\)\([efg]\)/\2\1/g	修改 "af fa bg"	为	"fa fa gb"
:s/abcde/abc^Mde/	修改 "abcde"	为	"abc"、"de" (两行)
:s/$/\^M/	修改 "abcde"	为	"abcde^M"
:s/\w\+/\u\0/g	修改 "bla bla"	为	"Bla Bla"
:s/\w\+/\L\u\0/g	修改 "BLA bla"	为	"Bla Bla"

注意："\L\u" 可用于使单词的首个字母变大写。此行为和 Vi 以及旧版 Vim 不兼容，那里 "\u" 会抵消 "\L"。"\U\l" 亦然。

注意：在以前的版本里，CTRL-V 以特殊的方式处理。因为和 Vi 不兼容，该功能已经被去掉了，现在用反斜杠来替代。

命令	文本	结果
:s/aa/a^Ma/	aa	a<line-break>a
:s/aa/a\^Ma/	aa	a^Ma
:s/aa/a\\^Ma/	aa	a\<line-break>a

(你需要输入 CTRL-V <CR> 来得到这里的 ^M)

"\1"，"\2" 等里的数字是基于模式里 "\(" 出现的顺序 (从左到右)。如果一个括号组匹配多次，最后一次的匹配被使用在 "\1"，"2" 等里。例如：

:s/\(\(a[a-d] \)*\)/\2/ 修改 "aa ab x" 为 "ab x"

"\2" 对应 "\(a[a-d] \)"。第一次匹配 "aa "，第二次匹配 "ab "。

如果括号和 '|' 组合使用，如 \([ab]\)\|\([cd]\)，两者有一个会不匹配，所以 \1 或者 \2 会为空。例如：

:s/\([ab]\)\|\([cd]\)/\1x/g 修改 "a b c d" 为 "ax bx x x"

:sc :sce :scg :sci :scI :scl :scp :sg :sgc
:sge :sgi :sgI :sgl :sgn :sgp :sgr :sI :si
:sic :sIc :sie :sIe :sIg :sIl :sin :sIn :sIp
:sip :sIr :sir :sr :src :srg :sri :srI :srl
:srn :srp

2-字母和 3-字母的 :substitute 命令

:substitute 命令列表

	c	e	g	i	I	n	p	l	r
c	:sc	:sce	:scg	:sci	:scI	:scn	:scp	:scl	---
e									
g	:sgc	:sge	:sg	:sgi	:sgI	:sgn	:sgp	:sgl	:sgr
i	:sic	:sie	---	:si	:siI	:sin	:sip	---	:sir
I	:sIc	:sIe	:sIg	:sIi	:sI	:sIn	:sIp	:sIl	:sIr
n									
p									
l									
r	:src	---	:srg	:sri	:srI	:srn	:srp	:srl	:sr

例外:

:scr 是 :scriptnames
:se 是 :set
:sig 是 :sign
:sil 是 :silent
:sn 是 :snext
:sp 是 :split
:sl 是 :sleep
:sre 是 :srewind

以表达式方式替代 sub-replace-expression
sub-replace-\= s/\=

当替代字符串以 "\=" 开始时，其余部分被解释为一个表达式。

除了 "<CR>" 以外， sub-replace-special 提到的字符的特殊含义这里不适用。<NL> 字符被用作换行符，双引号字符串 "\n" 也可以。在 <NL> 之前加上反斜杠得到一个真正的 <NL> 字符（在文件里成为 NUL）。

"\=" 记法也可用于 substitute() 函数的第三个参数 {sub} 之内，这时，sub-replace-special 指出的特殊含义完全不适用。具体说，<CR> 和 <NL> 不用作换行，而分别是回车和新行。

如果结果是 List ，连接其中的项目并以换行符分隔。这样，每个项目单独成为一行，当然本身就包含换行符的除外。

submatch() 函数可用来获取匹配文本。完整的匹配可以用 "submatch(0)" 得到。首个括号里的匹配可以用 "submatch(1)"，余者类似。

小心：分隔符不能出现在表达式里！可以考虑使用 "@" 或者 ":" 那样的字符。执行结果里出现分隔符不成问题。

例如：

```
:s@\n@\="\r" . expand("$HOME") . "\r"@
```
把换行符替代为包含 $HOME 值的一个新行。

```
s/E/\="\<Char-0x20ac>"/g
```
把所有的 'E' 字符替代为欧元符号。详见 <Char-> 。

4.3 搜索与替代 search-replace

:pro :promptfind

:promptf[ind] [string]
弹出搜索对话框。如果给出 [string]，它被用作初始的搜索字符串。
{仅适用于 Win32、Motif 和 GTK GUI 环境}

:promptr :promptrepl

:promptr[epl] [string]
弹出搜索/替代对话框。如果给出 [string]，它被用作初始的搜索字符串。
{仅适用于 Win32、Motif 和 GTK GUI 环境}

4.4 改变制表 change-tabs

:ret :retab :retab!

:[range]ret[ab][!] [new_tabstop]
把所有包含 <Tab> 的空白序列替代成由新的制表位 [new_tabstop] 确定的空白序列。如果你不指定新的制表位，或者它为 0，Vim 使用原来的制表位 'tabstop'。
已有的 Tab 的宽度总是用 'tabstop' 的当前值来计算。
如果有 !，Vim 也在合适的时候，把只包含正常空格的字符串换成 Tab。
如果置位了 'expandtab'，Vim 把所有的 Tab 换成相当的空格。
该命令把 'tabstop' 设为新值。如果按照缺省的情况，在全文件上进行处理，视觉上应该不会有任何改变。
小心：该命令修改 C 程序中的字符串里的任何 <Tab> 字符。要避免这一点，用 "\t" (无论如何，应该养成这个好的习惯)。
:retab! 也把空格序列换成 <Tab>，这可能会使 printf() 引起混淆。
如果打开 +vartabs 特性，可用以逗号分隔的制表宽度的列表来代替单个制表位。列表的每个值代表一个制表位的宽度。末值除外，该值适用于其后所有的制表位。

retab-example

下面的例子使用自动命令和 ":retab" 来编辑使用制表位为 8 的文件，但在编辑时制表位设置为 4。警告：字符串里的空格会被改变。另见 'softtabstop' 选项。

```
:auto BufReadPost     *.xx    retab! 4
:auto BufWritePre     *.xx    retab! 8
:auto BufWritePost    *.xx    retab! 4
:auto BufNewFile      *.xx    set ts=4
```

---

5. 复制并移动文本 copy-move

quote

"{a-zA-Z0-9.%#:-"}　指定下次的删除、抽出和放置命令使用的寄存器 {a-zA-Z0-9.%#:-"}（大写字符使得删除和抽出命令附加到该寄存器）（{.%#:} 只能用于放置命令）。

:reg　:registers

:reg[isters]　显示所有编号和命名寄存器的类型和内容。但不列出用于 :redir 目的地的寄存器。
类型可以是以下之一：
"c"　用于 characterwise 文本
"l"　用于 linewise 文本
"b"　用于 blockwise-visual 文本

:reg[isters] {arg}　显示 {arg} 里提到的编号和命名寄存器的内容。例如：

```
:reg 1a
```

显示寄存器 '1' 和 'a'。{arg} 里可以用空格。

:di　:display

:di[splay] [arg]　和 :registers 相同。

y　yank

["x]y{motion}　抽出 {motion} 跨越的文本 [到寄存器 x]。如果没有字符被抽出（例如，在第一列执行 "y0"）并且 'cpoptions' 里包括 'E' 标志位，这是一个错误。

yy

["x]yy　抽出 [count] 行 [到寄存器 x] linewise 行动作。

Y

["x]Y　抽出 [count] 行 [到寄存器 x]（等同于 yy，linewise 行动作）。如果你想要 "Y" 执行从光标到行尾的操作（更合乎逻辑，但是与 Vi 不兼容），用 ":map Y y$"。

v_y

{Visual}["x]y　抽出高亮文本 [到寄存器 x]（关于 {Visual} 见 Visual-mode ）。

v_Y

{Visual}["x]Y　抽出高亮行 [到寄存器 x]（关于 {Visual} 见 Visual-mode ）。

:y　:yank　E850

:[range]y[ank] [x]　抽出 [range] 所指定的行 [到寄存器 x]。仅当包含 +clipboard 特性时才可以抽出到 "* 或 "+ 寄存器。

:[range]y[ank] [x] {count}
从 [range] 的最后一行开始（缺省：当前行 cmdline-ranges ）抽出 {count} 行 [到寄存器 x]。

p　put　E353

["x]p　放置文本 [从寄存器 x] 在光标之后 [count] 次。

P

["x]P　　放置文本 [从寄存器 x] 在光标之前 [count] 次。

<MiddleMouse>

["x]<MiddleMouse>　　从一个寄存器放置文本在光标之前 [count] 次。除非另外指定，否则用 "* 寄存器。
光标停留在新文本的尾部。
只有在 'mouse' 包含 'n' 或者 'a' 时鼠标才会工作。
如果你有滚轮鼠标而且经常不小心粘贴了文本，你可以使用以下映射来关闭鼠标中键粘贴的功能：

```
:map <MiddleMouse> <Nop>
:imap <MiddleMouse> <Nop>
```

你也许还想要关闭多键击的功能。参见 double-click 。

gp

["x]gp　　如同 "p"，但光标停留在新文本之后。

gP

["x]gP　　如同 "P"，但光标停留在新文本之后。

:pu　:put

:[line]pu[t] [x]　　放置文本 [从寄存器 x] 在行号 [line] (缺省为当前行) 之后。它总是 linewise 行动作，因而这个命令可以用来把抽出的块放置在新行上。
如果未指定寄存器，则取决于 'cb' 选项：如果 'cb' 包含 "unnamedplus"，则从 + 寄存器 quoteplus 取出，否则如果 'cb' 包含 "unnamed"，则从 * 寄存器 quotestar 取出，否则，从无名寄存器 quote_quote 取出。
寄存器也可以是 '='，跟随一个可选的表达式。表达式继续到该命令结束为止。你需要在 '|' 和 '"' 字符前加上反斜杠不让它们终止你的命令行。例如：

```
:put ='path' . \",/test\"
```

如果 '=' 之后没有表达式，Vim 使用前一个表达式。用 ":dis =" 你可以看到它。

:[line]pu[t]! [x]　　放置文本 [从寄存器 x] 在行号 [line] (缺省为当前行) 之前。

]p　]<MiddleMouse>

["x]]p　或　["x]]<MiddleMouse>　　类似 "p"，但调整当前行的缩进。只有在 'mouse' 包含 'n' 或者 'a' 时鼠标才会工作。

[P

["x][P　或
]P

["x]]P　或
[p

["x][p　或　["x][<MiddleMouse>　　类似 "P"，但调整当前行的缩进。只有在 'mouse' 包含 'n' 或者 'a' 时鼠标才会工作。

你可以用这些命令把文本从一个地方复制到另一个地方。首先，把文本用抽出、删除或者修改命令取到一个寄存器里，然后用放置命令把寄存器的内容插入。你可以用这些命令把文本从一个文件移动到另一个文件，因为 Vim 在切换缓冲区时保留所有的寄存器 (CTRL-^ 命令可用来快捷地切换文件)。

linewise-register　characterwise-register

除了 :put 以外，你可以用 "." 重复放置命令，你也可以撤销它们。如果保存到寄存器

文本的命令是 linewise 行动作，Vim 把文本插入在光标所在的行之下 ("p") 或之上 ("P")。不然，Vim 把文本插入在光标的之后 ("p") 或之前 ("P")。":put" 命令使得 Vim 总是把文本放在下一行。你可以用命令序列 "xp" 来交换两个字母。你可以用 "ddp" 命令序列来交换两行。你还可以用 "deep" 命令交换两个单词 (光标在第一个单词之前的空白上)。你更可以在放置命令之后用 "']" 或者 "`]" 命令把光标移动到插入文本之后，或者用 "'[" 或 "`[" 把光标移动到文本的开始处。

put-Visual-mode v_p v_P
在可视模式下使用如 p 或者 P 之类的放置命令时，Vim 试图把选择的文本替换成寄存器的内容。是否工作如你所愿决定于选择的类型和寄存器里文本的类型。对于列块选择而言，它也决定于列块的大小，和头尾是否在已存在的字符上。(实现细节：实际上，它先把寄存器的内容放到选择区之后，再删除选择区的内容。)
前次选择的文本被放在无名寄存器中。如果希望把相同的文本重复放入可视选择区多次，请选用其它寄存器。例如，先抽出要复制的文本，再可视化选择要替换的文本，然后使用 "0p。重复任意多次均可，而每次无名寄存器都会被改变。

应用面向列块的可视模式命令抽出单行到寄存器时，粘贴该寄存器到某一可视选择的区域会把该行复制到选择的每一行上 (也即，把选择的列块区域替代为被复制行的列块)。

blockwise-register
如果你使用面向列块的可视模式命令把文本存到寄存器里，文本列块会被插入在当前和其后的行的当前列之前 ("P") 或之后 ("p")。Vim 使得整个文本列块从同一列开始。这样，插入的文本看上去和抽出或删除时看起来一样。为了使之可能，Vim 可能需要把一些 <Tab> 字符替换成空格。不过，如果列块的宽度不是 <Tab> 宽度的整数倍并且插入列块之后文本里包含 <Tab> 的时候，那些文本可能就不会那么整齐。

注意 在一个面向字符的抽出命令之后，Vim 把光标停留在最接近缓冲区开头的第一个被抽出的字符之上。这意味着 "yl" 不会移动光标，但是 "yh" 把光标向左移一格。
逻辑：在 Vi 里 "y" 命令跟一个反向的动作有时不会把光标移动到第一个被抽出的字符之上，因为屏幕没有刷新。Vim 符合 Posix 的规范，总是把光标移动到第一个字符之上。
在一个面向行的抽出命令之后，光标放在第一行上，但是列保持不变，所以不必然在第一个抽出的字符上。

共有十种类型的寄存器： registers {register} E354
1. 无名寄存器 ""
2. 10 个编号寄存器 "0 到 "9
3. 行内删除寄存器 "-
4. 26 个命名的寄存器 "a 到 "z 或者 "A 到 "Z
5. 三个只读寄存器 ":、". 和 "%
6. 轮换缓冲区寄存器 "#
7. 表达式寄存器 "=
8. 选择和拖放寄存器 "*、"+ 和 "~
9. 黑洞寄存器寄存器 "_
10. 最近搜索模式寄存器 "/

1. 无名寄存器 "" quote_quote quotequote
用 "d"、"c"、"s"、"x" 等命令删除或者用 "y" 等抽出命令复制的文本都被 Vim 用来填充该寄存器，不管是否用到别的专门寄存器 (例如 "xdd)。这就好像无名寄存器是指向最近使用的那个寄存器的指针一样。使用大写寄存器进行添加时，无名寄存器包含和命名寄存器相同的文本。'_' 寄存器是一个特例。"_dd 不把删除的文本存在任何寄存器里。
不指定寄存器的放置命令 (p 或 P) 使用无名寄存器的内容。你也可以用 '"' 作为名字来访问该寄存器。这意味着要输入两个连续的双引号。写入 "" 寄存器将实际写到寄存器 "0 上。

{Vi: 寄存器内容在更换文件时丢失，没有 '""'}

2. 编号寄存器 "0 到 "9 quote_number quote0 quote1 quote2 quote3 quote4 quote9

Vim 把抽出和删除命令的文本保存在这些寄存器里。

编号寄存器 0 包含最近抽出的文本，除非该命令用 ["x] 指定了别的寄存器。

编号寄存器 1 包含了最近删除或者修改的文本，除非该命令用 ["x] 指定了别的寄存器或者该文本小于一行（该情况下使用行内删除寄存器）。以下移动命令在应用删除操作符时例外： % 、 (、) 、 ` 、 / 、 ? 、 n 、 N 、 { 和 } 。它们总是使用寄存器 "1（为了和 Vi 兼容）。如果文本小于一行，同时使用 "- 寄存器。

注意 这些字符可能会被映射。例如，matchit 插件会映射 % 字符。

每来一次新的删除和修改，Vim 把前一次的寄存器 1 的内容复制到寄存器 2，2 到 3，依此类推。而寄存器 9 的内容就丢失了。

{Vi: 编号寄存器的内容在更换文件时丢失；寄存器 0 不存在}

3. 行内删除寄存器 "- quote_- quote-

该寄存器保存删除不到一行内容的命令的文本，除非该命令用 ["x] 指定了寄存器。

4. 命名寄存器 "a 到 "z 或者 "A 到 "Z quote_alpha quotea

Vim 只有在你指定的时候才使用这些寄存器。指定为小写字母时替换原来的内容，指定为大写字母时附加到原来的内容。如果 'cpoptions' 里有 '>' 标志位，在附加文本前插入一个换行符。

5. 只读寄存器 ":、". 和 "%

它们是 '%'、'#'（译者注：原文如此）、':' 和 '.'。你只能在 "p"、"P"、":put" 命令和 CTRL-R 的时候使用它们。

quote_. quote. E29

". 包含最近插入的文本（和插入模式命令 CTRL-A 和 CTRL-@ 插入的一样）。注意：它不适用于 CTRL-R 在命令行上的操作。它的工作方式稍有不同，例如，文本是被插入而不是放置的（'textwidth' 和其它选项影响插入的内容）。

quote_% quote%

"% 包含当前文件名。

quote_: quote: E30

": 包含最近执行过的命令行。例如：用 "@:" 重复上次执行过的命令行命令。只有当一个命令行中至少一个字符是键入的，该寄存器才会保存命令行的内容。所以，如果命令行完全来自映射，该寄存器保持不变。{仅当编译时加入 +cmdline_hist 特性才有效}

quote_# quote#

6. 轮换文件寄存器 "#

包含当前窗口轮换文件的名字。它影响 CTRL-^ 的命令的工作方式。

该寄存器是可写的，主要用来帮助插件在改变其值后恢复原值。接受缓冲区号：

```
let altbuf = bufnr(@#)
...
let @# = altbuf
```

如果传递了不存在的缓冲区号，会报错 E86 。

也可接受现有缓冲区名的匹配：

```
let @# = 'buffer_name'
```

如果有多于一个缓冲区匹配该名，报错 E93 ，如果没有缓冲区匹配该名，也会报错 E94 。

7. 表达式寄存器 "= quote_= quote= @=

其实并没有这么一个寄存器可以储存文本，但是这是用来在使用寄存器的命令中使用表达式的一个方式。表达式寄存器是可读写的。

" 之后输入 '=' 或者输入 CTRL-R 时，光标移到命令行上，这时你可以输入任何的表达式（见 expression ）。所有普通的命令行编辑命令都可以使用，还有一个表达式专门的历史表。当你按回车结束命令行时，Vim 计算表达式的结果。如果你用 <Esc> 结束，Vim 终止表达式。如果你不输入表达式，Vim 使用最近的表达式（和 "/" 命令的处理相似）。

表达式的计算结果必须是一个字符串。数值结果会自动转化为字符串。对 "p" 和 ":put" 命令而言，浮点数结果也会被转化为字符串。如果结果为列表，每个成员被转成字符串，然后被单独放在一行上。字典、函数引用类型的结果则产生错误信息（通过 string() 转化）。

如果 "= 寄存器被 "p" 命令使用，该字符串会在 <NL> 字符处断开。如果该字符串以 <NL> 结尾，则它被视为一个面向行的寄存器。

8. 选择和拖放寄存器 "*、"+ 和 "~
用这些寄存器来保存和取得 GUI 界面选择的文本。参见 quotestar 和 quoteplus 。如果剪贴板不存在或者不工作，使用无名寄存器。Unix 上，仅当 +xterm_clipboard 特性存在时剪贴板才可用。

注意 "* 和 "+ 在 X11 系统上有分别。关于该差别的解释，参见 x11-selection 。在 MS-Windows 上，"* 和 "+ 的使用实际上是等价的，可见 gui-clipboard 。

quote_~ quote~ <Drop>
只读的 "~ 寄存器保存最近一次拖放操作放下的文本。如果有什么东西被放到 Vim 上，"~ 寄存器被填充，<Drop> 虚拟键被激活。如果你需要，你可以重定义该键的映射；缺省的动作（适用于所有模式）是把 "~ 寄存器的内容插入到当前光标位置。
{仅当编译时加入 +dnd 特性才有效，当前只适用于 GTK GUI 版本}

注意: "~ 寄存器仅用于在 Vim 上拖放普通文本。拖放 URI 列表在内部处理。

9. 黑洞寄存器 "_ quote_
当写到这个寄存器时，什么都不会发生。这可以用来删除文本，而不影响任何正常的寄存器。从该寄存器读时，什么都不会返回。

10. 最近搜索模式寄存器 "/ quote_/ quote/
含有最近搜索的模式。它被 "n" 和 'hlsearch' 使用。可以用 :let 来修改。你可以改变它使得 'hlsearch' 不经过实际的搜索直接高亮某些匹配。你不能把抽出或者删除命令的内容放到该寄存器上。搜索方向则可从 v:searchforward 得到。
注意 从函数返回时该值被复原 function-search-undo 。

@/
你可以用 :let 命令写到一个寄存器 :let-@ 。例如:

```
:let @/ = "the"
```

如果你用放置命令而不指定寄存器，Vim 使用上次填充的寄存器（这也是无名寄存器的内容）。如果你弄糊涂了，用 :dis 命令看看 Vim 会放置什么内容（该命令显示所有的有名和无名的寄存器；无名寄存器被标为 '"'）。

下面三个命令总是针对整行工作。

:[range]co[py] {address} :co :copy
把 [range] 指定的行复制到 {address} 给出的行之下。

:t

和 :copy 等价。

:[range]m[ove] {address}

把 [range] 指定的行移动到 {address} 给出的行之下。

---

6. 文本排版

formatting

:[range]ce[nter] [width]

在 [range] 指定范围的行和 [width] 限定的列（缺省为 'textwidth'，如其为 0，则取 80）范围内的文本居中对齐。

:[range]ri[ght] [width]

在 [range] 指定范围的行和 [width] 限定的列（缺省为 'textwidth'，如其为 0，则取 80）范围内的文本靠右对齐。

:[range]le[ft] [indent]

在 [range] 指定范围的行的文本靠左对齐。其缩进的距离可由 [indent] 设置（缺省为 0）。

gq{motion}

对 {motion} 动作跨越的行进行排版。
排版使用如下三种方式之一：
1. 如果 'formatexpr' 不为空，计算该表达式。每个缓冲区的值可以不同。
2. 如果 'formatprg' 不为空，使用外部程序。
3. 否则，使用内部排版机制。

第三种方式使用 'textwidth' 选项控制所有待排版行的行宽（见下）。
如果 'textwidth' 选项为 0，则排版行宽设为屏幕的宽度（但最大宽度不超过 79）。
'formatoptions' 选项控制排版的方式 fo-table 。
光标停留在排版行末行的第一个非空白处。
注意：'Q' 命令以前执行此项功能。如果你还想继续用 "Q" 来排版，执行如下映射命令：

```
:nnoremap Q gq
```

gqgq
gqq

排版当前行。如带计数排版相应多行。

{Visual}gq

排版高亮文本（{Visual} 的部分参见 Visual-mode ）。

gw{motion}

对 {motion} 动作跨越的行进行排版。和 gq 类似，但排版后光标恢复原位。而且不使用 'formatprg' 和 'formatexpr' 选项。

gwgw
gww

对当前行排版，其它和 "gw" 类同。

{Visual}gw 排版高亮文本，其它和 "gw" 类同。({Visual} 可见 v_gw Visual-mode)。

例如：要排版当前段落，可用： gqap

```
gqap
```

"gq" 命令后，光标随着指定的移动命令移动。这使得用 "." 进行连续排版成为可能。例如，"gqj"（对当前行和下一行排版）和 "gq}"（排版到段落尾）便可如此。注意：如果设置了 'formatprg'，"gq" 把光标留在排版后的首行（如同过滤命令那般）。

如果你想对当前段落排版后留在原来的位置，可用：

```
gwap
```

如果你想使段落自动排版，可以在 'formatoptions' 里加入 'a' 标志位。参见 auto-format 。

如果 'autoindent' 选项打开，Vim 用首行的缩进距离排版其后的所有行。

排版不会改变空行（但会改变只有空白字符的行！）。

如果有连接行的操作，应用 'joinspaces' 选项。

你可以设置 'formatexpr' 选项为某表达式或设置 'formatprg' 选项为外部程序名，Vim 会用之进行文本排版。'textwidth' 等选项对外部程序的排版不起作用。

format-formatexpr

'formatexpr' 选项可设为对缓冲区重新排版的 Vim 脚本函数。此功能常用于 ftplugin ，因为排版方式高度依赖于文件类型。这里，建议使用 autoload 脚本，这样只有在有需要时才会载入对应的脚本，而该脚本应命名为 <filetype>format.vim。

例如，随 Vim 发布的在 $VIMRUNTIME 目录里的 XML 文件类型插件把 'formatexpr' 选项设为：

```
setlocal formatexpr=xmlformat#Format()
```

这意味着，在以下目录你会找到相应的脚本，其中定义了 xmlformat#Format() 函数：
$VIMRUNTIME/autoload/xmlformat.vim

下例脚本删除选定文本的拖尾空白。把它放在你的 autoload 目录，如 ~/.vim/autoload/format.vim 里：

```
func! format#Format()
  " 只在显式的 gq 命令时重新排版
  if mode() != 'n'
    " 缺省使用 Vim 内部的重新排版方式
    return 1
  endif
  let lines = getline(v:lnum, v:lnum + v:count - 1)
  call map(lines, {key, val -> substitute(val, '\s\+$', '', 'g')})
  call setline('.', lines)

  " 不使用内部的排版方式！
  return 0
```

```
endfunc
```

然后，如此打开此种排版方式：

```
setlocal formatexpr=format#Format()
```

注意：此函数在插入模式下（一般这意味着文本的插入超出了 'textwidth' 限制）显式返回非零。这样，Vim 会使用缺省的内部排版方法。

反之，该 gq 命令会用于重新排版文本，此时，函数获取所选的行，删除这些行拖尾的空白，并在原地放置。如果你需要把单行分割为多行，小心不要覆盖额外的部分。

如果在插入或替换模式下希望重排文本，分外小心，因为此时函数可能会被递归调用。调试可用 'debug' 选项。

right-justify

Vim 没有靠右控制对齐（right justify）文本的命令。你可以用 "par" 这样的外部命令来实现（例如，"!}par" 对到段落尾的文本排版），或者设置 'formatprg' 为 "par"。

format-comments

用户手册的 30.6 一节给出注释排版的总览。

Vim 可以自动对注释的插入和排版进行特殊处理。Vim 把以特定字符串开头（忽略空白）的行识别为注释。以下是三种不同类型的注释：

- 在每行的开头都出现的注释字符串。例如，外壳脚本使用的注释行都以 "#" 开头。
- 只在首行出现的注释字符串。使用连字符 "-" 的本列表就是这样的一个例子，
- 由三部分组成的注释，包括起始字符串，结尾字符串，和两者之间可选的行的起始字符串。三种字符串可分别指定。如下 C 风格的注释就是如此：

```
/*
 * this is a C comment
 */
```

'comments' 选项是一个由逗号分隔的列表。每个部分定义一种类型的注释字符串。每个部分的组成方式是：

{flags}:{string}

{string} 是必须出现的字符串（不作转义）。

{flags}:

n 可嵌套的注释：允许多个部分间的嵌套。例如，'comments' 为 "n:),n:>"。则以 ">) >" 开始的行视为注释。

b 在 {string} 之后必须有空白字符（<Space>、<Tab> 或 <EOL>）。

f 该注释字符串只在首行出现。下一行不重复注释，但保留相同的缩进（例如，带符号的列表（bullet-list））。

s 三段式注释的起始字符串

m 三段式注释的中间字符串

e 三段式注释的结尾字符串

l 左对齐。和 's' 或 'e' 一起使用。起始或结尾字符串的最左字符和中间字符串

的最左字符对齐。
这是缺省值，可以省略。详情见下。

r　　右对齐。同上，不过不是最左字符而是最右字符。详情见下。

O　　"O" 命令不把它当作注释。

x　　允许三段式注释用以下方式结束：在中间字符串自动提供的情况下，在新行第一个动作是输入结尾字符串的最后一个字符。详情见下。

{digits}
在和 's' 或 'e' 一起使用时：对自动插入的中间或结尾字符串增加 {digit} 单位的位移。该位移以左对齐的位置为基准。详情见下。

-{digits}
和 {digits} 类似，但减少缩进。这只对开始或结尾部分至少有相当数量的缩进时才有效。

如果一个字符串没有 'f'、's'、'm' 或 'e' 任何一个标志位，Vim 假设注释字符串在每行都重复出现。标志位部分可以为空。

在 {string} 之前或之后的任何空白都是 {string} 的一部分，所以不要随便留出开头或结尾的空白，除非空白的确是注释字符串的一部分。

如果某个注释字符串是另一个字符串的一部分，先指定完整的，再指定部分的。例如，要包含 "-" 和 "->"，用

```
:set comments=f:->,f:-
```

三段式注释必须以开始、中间和结尾三部分给出，而且不能间杂其它的部分。一个三段式 C 注释的例子是

```
sr:/*,mb:*,ex:*/
```

为了避免形如 "*ptr" 的内容被识别为注释，这里的中间字符串包含了 'b' 标志位。对三段式注释而言，Vim 从开始和中间字符串之后查找结尾字符串。如果找到，注释就不会再从下一行继续。三段式注释必须要有一个中间字符串，不然 Vim 无法识别中间的那些行。

注意　上述三段式注释定义里的 "x" 标志位。当你在一个 C-注释里按回车时，Vim 会自动在新行后插入中间字符串：" * "。要结束注释，你只需要在新行里首先键入 "/"。这样就把中间字符串替换成结尾字符串并应用指定的对齐方式，成为了 " */"。从而省却了先按退格键的麻烦。

如果同时匹配中间部分和结尾部分但结尾部分更长，使用结尾部分。这样 C 风格注释就不需要中间部分后加空格了。

这里是一个关于用于对齐的标志位的例子，使得注释看起来很突出（也很像一个 1 字）。考虑注释字符串：

```
:set comments=sr:/***,m:**,ex-2:******/
```

```
                                /***
                                  **<--右对齐，来自 "r" 标志位
                                  **
  2 个单位的位移，来自 "2" 标志位--->**
                                ******/
```

该例中，第一行注释是键入的，然后按回车四次，最后按 "/" 以结束注释。

这里是三部分注释的一些细节。有三个时间点应用对齐和位移标志位：输入起始字符串后开启新行、在结尾字符串前开启新行、三部分注释的自动结束。而结尾部分的对齐标志位要反过来看；这样，相同的对齐标志位在分别用于 "s" 和 "e" 时才会使起始和结尾部分有相同的缩进。每个注释部分只应用一种对齐方式，但位移优先于 "r" 和 "l" 标志位。

打开 'cindent' 在很多情况下会覆盖对齐标志位。使用其它方法，如 gq 或 = 来重新缩进也不会参考对齐标志位。这些其它的排版选择可以定义类似的行为。一个担心是 'cindent' 虽然有众多附加的选项来对注释进行基于上下文的缩进，但不能再现三部分注释的缩进对齐。但 'indentexpr' 应该有能力很好地处理三部分注释。

其它示例：

"b:*"	包含 "*" 开头的行，但 "*" 后面不是空白的除外。这样避免指针取值操作 "*str" 被识别为注释。
"n:>"	包含形如 ">"、">>"、">>>" 等开头的行。
"fb:-"	包含 "- " 开头的列表，可用来自动排版。

该选项的缺省值是 "b:#"。这意味着 "#include" 开头的行不会被认为是注释，但 "# define" 是。这是个不得不然的妥协。

fo-table

你可以使用 'formatoptions' 选项来控制 Vim 如何对文本进行排版。'formatoptions' 是一个字符串，它可以包含下列字符。缺省设置是 "tcq"。为了提高可读性，你可以用逗号分隔选项字符。

字符	在 'formatoptions' 里代表的含义
t	使用 'textwidth' 自动回绕文本
c	使用 'textwidth' 自动回绕注释，自动插入当前注释前导符。
r	在插入模式按回车时，自动插入当前注释前导符。
o	在普通模式按 'o' 或者 'O' 时，自动插入当前注释前导符。
q	允许 "gq" 排版时排版注释。 注意 排版不会影响空行或者只有注释前导符的行。这样的行开启一个新段落，注释前导符的改变也是如此。
w	拖尾的空格指示下一行继续同一个段落。而以非空白字符结束的行结束一个段落。
a	自动排版段落。每当文本被插入或者删除时，段落都会自动进行排版。参见 auto-format 。 如果 'c' 标志位存在，该设置只对识别的注释有效。
n	在对文本排版时，识别编号的列表。实际上，这里使用了 'formatlistpat' 选项，所以可以使用任何类型的列表。出现在数字之后的文本缩进距离被应用到后面的行。数字之后可以有可选的 '.'、 ':'、')'、']' 或者 '}'。 注意 'autoindent' 也必须置位。不要和 "2" 一起使用，效果不好。 示例： 1. 第一项 回绕文字 2. 第二项
2	在对文本排版时，将段落第二行而非第一行的缩进距离应用到其后的行上。这适用于第一行有特殊缩进需要的段落。注意 'autoindent' 也必须置位。 示例： first line of a paragraph second line of the same paragraph third line.

也适用于注释内部，并忽略注释前导符。

v　Vi-兼容的插入模式自动回绕：只有在当前输入命令键入的空白上才会分行。（注意：这并不能 100% Vi 兼容。Vi 在这方面有些 "意想不到的特性"，换而言之，漏洞。它使用屏幕列而非实际的列。）

b　和 'v' 类似，但只有在键入空白时还没抵达或者刚到回绕边界的时候才会自动回绕。如果一行在开始插入之前已经超过 'textwidth' 指定的长度，或者在到达 'textwidth' 之时没有输入过空白，Vim 不会自动回绕。

l　插入模式不分行：当一行已经超过 'textwidth' 时，插入命令不会自动排版。

m　可以在任何值高于 255 的多字节字符上分行。这对亚洲文本尤其有用，因为每个字符都是单独的单位。

M　在连接行时，不要在多字节字符之前或之后插入空格。优先于 'B' 标志位。

B　在连接行时，不要在两个多字节字符之间插入空格。有 'M' 标志位时无效。

1　不要在单字母单词后分行。如有可能，在它之前分行。

j　在合适的场合，连接行时删除注释前导符。例如，连接：

```
int i;   // the index
         // in the list
```

结果是：

```
int i;   // the index in the list
```

p　不在句号后的单个空白上断行。它的目的是和 'joinspaces' 和 cpo-J 相补充，用于两个空格分隔的日常语句。例如，如果 'textwidth' 设为 28：

```
Surely you're joking, Mr. Feynman!
```

结果是：

```
Surely you're joking,
Mr. Feynman!
```

而不是：

```
Surely you're joking, Mr.
Feynman!
```

't' 和 'c' 不同组合方式决定 Vim 何时进行自动回绕：

值	行为
""	没有自动排版（你可以用 "gq" 进行手工排版）
"t"	自动排版文本，不包括注释
"c"	自动排版注释，不包括文本（对 C 程序适用）
"tc"	自动排版文本和注释

注意 如果 'textwidth' 为 0，Vim 不会做任何自动排版（但是会根据 'comments' 选项自动插入注释前导符）。但 'a' 标志位存在时有例外。 auto-format

注意 如果 'paste' 打开，Vim 也不会做任何排版。

注意 即使 Vim 不做自动回绕，仍然可以把 'textwidth' 设为非零。'textwidth' 对 "gq" 的排版依然有用。

如果 'comments' 选项包含 "/*"、"*" 和/或 "*/"，Vim 有一套处理这些注释更加聪明的内嵌方法。在 "/*" 或 "*/" 之前或之后开始一个新行（在 'formatoptions' 里有 'r' 或者 'o' 的情况下），会自动给出正确的开始部分。排版或者自动回绕也会有相同的处理。在以 "/*" 或者 "*" 开始并包含 "*/" 的行之后开启新行，就不会插入注释前导符，而且新行的缩进由注释起始行决定。

例如：

```
/* ~
 * 你的任何注释。
 */ ~
```

该行的缩进和以上注释的起始行相同。

上述这些应该已经十分够用了，尤其在与新的 :autocmd 命令协同使用时，可以为不同文件类型提供不同的设置。

一些例子：
适用于 C 代码（只对注释排版）：
```
:set fo=croq
```
适用于邮件/新闻（排版所有文本，"o" 命令不开始注释）：
```
:set fo=tcrq
```

自动排版 auto-format autoformat

如果 'formatoptions' 里包含 'a' 标志位，在插入或者删除文本时会自动进行排版。这对编辑文本段落很好用。以下对如何使用这一功能提供一些提示：

- 你需要正确定义何谓段落。最简单的方式是以空行分隔的为段落。如果没有分隔的空行，考虑用 'w' 标志位并在段落中除了最后一行以外，每行结尾加上一个空格。

- 你可以根据文件类型 filetype ， 或者用 modeline 指定特定文件的方式，设置不同的 'formatoptions'。

- 将 'formatoptions' 设为 "aw2tq" 使得文本以如下方式缩进：

```
    bla bla foobar bla
bla foobar bla foobar bla
    bla bla foobar bla
bla foobar bla bla foobar
```

- 如果只想自动排版注释，加上 'c' 标志位。可用于源代码。

- 设置 'textwidth' 为你希望的宽度。如果为零，使用 79 或屏幕宽度中较小的那个。

还有一些警告：

- 如果段落没有正确分隔，任何改变都会使得所有相连的文本自动排版。考虑

```
:set fo-=a
```

- 如果用 'w' 标志位（行尾的空格意味着段落的继续）并且用 dd 删除了段落的末行，这段落自动和下一个段落合并为一个段落。

- 改变的文本被保存以备可能的撤销之用。排版也是改变的一种。所以每次排版都会为撤销保存文本。这会有相当的内存开销。

- 排版一个很长的段落和/或复杂的缩进也许会相当慢。

---

7. 文本排序 sorting

Vim 有排序函数和排序命令。排序函数可见： sort() ， uniq() 。

:sor :sort

:[range]sor[t][!] [b][f][i][n][o][r][u][x] [/{pattern}/]

给 [range] 里的行排序。如果没有给出行范围，给所有行排序。

带 [!] 则反向排序。

带 [i] 则忽略大小写。

选项 [n][f][x][o][b] 互斥。

带 [n] 则排序基于每行的第一个十进制数（在 {pattern} 匹配之后或之内）。
数值包含前导的 '-'。

带 [f] 则排序基于每行内的浮点数。浮点值相当于在文本（{pattern} 匹配的之后或之内）上调用 str2float() 函数的结果。仅当 Vim 编译时加入浮点数支持时该标志位才有效。

带 [x] 则排序基于每行的第一个十六进制数（在 {pattern} 匹配之后或之内）。忽略引导的 "0x" 或 "0X"。
数值包含前导的 '-'。

带 [o] 则排序基于每行的第一个八进制数（在 {pattern} 匹配之后或之内）。

带 [b] 则排序基于每行的第一个二进制数（在 {pattern} 匹配之后或之内）。

带 [u]（u 代表 unique 唯一）则只保留完全相同的行的第一行（如果带 [i]，忽略大小写的区别）。
没有这个标志位，完全相同的行的序列会按照它们原来的顺序被保留下来。
注意 引导和拖尾的空白差异会导致不相同的行。

如果指定 /{pattern}/ 并且没有 [r] 标志位，跳过匹配 {pattern} 的文本，使得排序在匹配之后的内容上进行。
除了斜杠以外，任何非字母的字符都可以。
例如，要按第二个逗号分隔的字段排序：

```
:sort /[^,]*,/
```

按虚拟第 10 列的文本排序（从而忽略制表和空格的区别）：

```
:sort /.*\%10v/
```

按每行的第一个数值排序，不管它前面有什么：

```
:sort /.\{-}\ze\d/
```

（说明："."\{-}" 匹配任何文本，"\ze" 设置匹配结束位置，\d 匹配单个数位。）
带 [r] 则排序在匹配 {pattern} 的文本上进行，而不是如上所述的用它之后的文本。
例如，要按每行的前三个字母排序且只看这三个字母：

```
:sort /\a\a\a/ r
```

如果使用 {pattern}，不匹配 {pattern} 的行按照它们原来的顺序被保留，但和匹配 {pattern} 的行分开。如果反向排序，它们会以反向顺序出现，并在排序好的行之后。否则它们会以原有顺序出现，在排序好的行之前。

如果 {pattern} 为空（例如指定 //），则使用最近使用的搜索模式。这样你可以先试好模式，然后在执行。

注意 :sort 和 :global 一起使用并不能对匹配的行进行排序，这样做没有意义。

排序的细节取决于使用的库函数。不能保证排序依赖于当前的 locale。你要自己试试才知道。Vim 保证 "稳定" 排序。

排序可以被中断，但在整个过程里如果中断得太晚，最后可能会出现重复的行。这取决于使用的系统库函数。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者: jwdu, tocer http://vimcdoc.sf.net

撤销和重做 undo-redo

在用户手册的 02.5 节对它们的基本操作有解释。

1. 撤销和重做操作的命令 undo-commands
2. 撤销操作的两种方法 undo-two-ways
3. 撤销块 undo-blocks
4. 撤销分支 undo-branches
5. 撤销的持久性 undo-persistence
6. 撤销操作的解释 undo-remarks

---

1. 撤销和重做命令 undo-commands

<Undo> 或 u — 撤销 [count] 次更改。 undo <Undo> u

:u[ndo] — 撤销一次更改。 :u :un :undo

:u[ndo] {N} — 转到改变号 {N} 之后。{N} 的含义参见 undo-branches 。 E830

CTRL-R — 重做 [count] 次被撤销的更改。 CTRL-R

:red[o] — 重做一个被撤销的更改。 :red :redo redo

U — 撤销最近对特定行所作的一系列更改，也就是最近作改变的那行。 U 自己也算一次改变，因此 U 会撤销之前的 U 。

由于最近所作的更改是被记住的，因此，你可以应用上面的撤销和重做命令把文件倒转到你所作每次更改以前的状态。当然，你也可以重新应用这些修改，把文件重做到执行撤销操作以前的状态。

对于撤销/重做 (undo/redo) 来说，"U" 命令和其他命令是同等对待的。所以 "u" 可以撤销一个 "U" 命令、'CTRL-R' 也可以重做对 "U" 命令的撤销。当 "U"、"u" 和 'CTRL-R' 三个命令混合使用时，你将注意到 "U" 命令决意将被该行还原到前一个 "U" 命令之前的状态。这或许会令你困惑。多试试，你会习惯的。
"U" 命令总是把缓冲区标记为已改变。即使它把修改过的缓冲区重做到未改变的状态也是如此。为此，只有使用 "u" 命令，才能令缓冲区正确地回复到未改变的标记。

---

2. 撤销操作的两种方法 undo-two-ways

撤销和重做的工作模式依赖于 'cpoptions' 中 'u' 标志位。这里有两种模式，Vim 模式（不包含 'u'）和 Vi-兼容模式（包含 'u'）。在 Vim 模式下，"uu" 命令撤销两次改变，在 Vi-兼容模式下，"uu" 命令什么也不做（撤销第一次撤销）。

不包含 'u' 的 Vim 模式：
及时的撤销命令可以回到先前的状态。你也可以使用重做命令再次前进。不过，如果在撤销命令后你做了一个新的改变，重做上次的撤销便不再可能。

包含 'u' 的 Vi-兼容模式：
撤销命令撤销包含以前的撤销命令在内的任何更改。重做命令重复前一个撤销命令。但它 不 用来重复改变命令。为此请用 "." 命令。

举例	Vim 模式	Vi-兼容模式
"uu"	两次撤销	空操作
"u CTRL-R"	空操作	两次撤销

原理：　Nvi 使用 "." 命令而不是 CTRL-R。不幸的是，这不是 Vi 兼容的。例如 "dwdwu." 命令，在 Vi 中删除两个词，但在 Nvi 中，它什么也不做。

---

3. 撤销块　　undo-blocks

单个 undo 命令通常撤掉一个输入的命令，不论这个命令造成多少改变。这个可以撤销的改变序列构成了一个撤销块。所以如果键入的命令调用一个函数，那么在这个函数中的所有命令全部被撤销。

如果你想编写一个函数或脚本，它不包含新的可撤销之改变，而合并到之前的改变中，用以下命令：

:undoj　:undojoin　E790

:undoj[oin]　把其后的改变和以前的撤销块进行合并。
警告：小心使用。它会阻止用户合理地撤销改变。不要在撤销或重做后使用。

这最适用于在改变的过程中需要提示用户的场合。例如调用 getchar() 的函数。确保在这之前有一个相关的必须与之合并的改变。

这条命令不能单独工作，因为下一个键击会再次造成新改变。但你可以这么做：

```
:undojoin | delete
```

在此之后，"u" 命令会同时撤销 delete 命令和前一次改变。

要反过来，把一个改动分成两个撤销块，在插入模式下用 CTRL-G u。可用于使某个插入命令部分可撤销。譬如可分拆到每个句子。 i_CTRL-G_u
设置 'undolevels' 的值也会打断撤销。即使新值和旧值相同。

---

4. 撤销分支　　undo-branches　undo-tree

上面我们只讨论了单线的撤销/重做。但你也可以进行分支。考虑你先撤销了若干改变，然后又进行了一些其它的改变。此时，被撤销的改变就成为一个分支。下面的命令使你能

够到达那个分支。

这在用户手册中有解释： usr_32.txt 。

:undol :undolist

:undol[ist] 列出改变树的所有叶结点。例如：

```
number changes  when               saved
    88      88  2010/01/04 14:25:53
   108     107  08/07 12:47:51
   136      46  13:33:01             7
   166     164  3 seconds ago
```

"number" 列是改变号。这个编号持续增加，用于标识特定可撤销的改变，参见 :undo 。
"changes" 列是树的根结点到此叶结点所需的改变数目。
"when" 列是此改变发生的日期时间。四种可能的格式是：

N seconds ago	
HH:MM:SS	时分秒
MM/DD HH:MM:SS	同上，还有月日
YYYY/MM/DD HH:MM:SS	同上，还有年

"save" 列给出此改变是否已写入硬盘和第几次写入文件。可用于 :later 和 :earlier 命令。
要更详细的信息可用 undotree() 函数。

g-

g- 转到较早的文本状态。如果带计数，重复那么多次。

:ea :earlier

:earlier {count}	转到 {count} 次较早的文本状态。
:earlier {N}s	转到大约 {N} 秒钟之前的较早的文本状态。
:earlier {N}m	转到大约 {N} 分钟之前的较早的文本状态。
:earlier {N}h	转到大约 {N} 小时之前的较早的文本状态。
:earlier {N}d	转到大约 {N} 天之前的较早的文本状态。

:earlier {N}f 转到 {N} 次文件写入之前的较早的文本状态。
如果上次写入之后有改动，":earlier 1f" 会恢复文本到上次写入时的状态。否则会转到再上一次写入时的状态。
如果在第一次文件写入的状态，甚至于从未写入过文件，":earlier 1f" 会转到首次改变之前的状态。

g+

g+ 转到较新的文本状态。如果带计数，重复那么多次。

:lat :later

:later {count}	转到 {count} 次较新的文本状态。
:later {N}s	转到大约 {N} 秒钟之后的较新的文本状态。
:later {N}m	转到大约 {N} 分钟之后的较新的文本状态。
:later {N}h	转到大约 {N} 小时之后的较新的文本状态。
:later {N}d	转到大约 {N} 天之后的较新的文本状态。

:later {N}f 转到 {N} 次文件写入之后较新的文本状态。
若在在最后一次文件写入的状态，":later 1f" 会转到最新的文本状态。

注意 如果由于 'undolevels' 选项，撤销信息被清空，那么文本状态将无法访问。

在不同时间点上移动的时候，不要奇怪一个时间点会同时发生多个改变。通过撤销树跳转然后又做了新的改变后就会发生这种情况。

示 例

从这一行开始：

```
one two three
```

按三次 "x" 删除第一个单词：

```
ne two three
e two three
 two three
```

现在按 "u" 三次撤销：

```
e two three
ne two three
one two three
```

按三次 "x" 删除第二个单词：

```
one wo three
one o three
one  three
```

现在按 "g-" 三次撤销：

```
one o three
one wo three
 two three
```

现在，回到第一个撤销分支，也就是在删除 "one" 之后。重复 "g-" 会回到原始文本：

```
e two three
ne two three
one two three
```

使用 ":later 1h" 跳到最后一次改变：

```
one  three
```

使用 ":earlier 1h" 再次回到开始：

```
one two three
```

注意 使用 "u" 和 CTRL-R 无法得到全部可能的文本状态，但是重复 "g-" 和 "g+" 却可以。

5．撤销的持久性 undo-persistence persistent-undo

卸载缓冲区时，Vim 通常会删除该缓冲区建立的撤销树。通过设置 'undofile' 选项，Vim 会在写入文件时自动保存撤销历史，而重新编辑文件时，恢复撤销历史。

'undofile' 选项在写入文件之后检查，而发生在 BufWritePost 自动命令之前。要控制哪些文件需要保存撤销信息，可以使用 BufWritePre 自动命令：

```
au BufWritePre /tmp/* setlocal noundofile
```

Vim 把撤销树保存在一个独立的撤销文件里，每个编辑的文件对应一个，使用一个简单的

方案来对应文件系统的路径到撤销文件名。Vim 会检测是否某个撤销文件不再和写它时的那个文件同步（使用文件内容的哈希值），如果文件内容在撤销文件写入后有改动，忽略撤销文件，以防止文件遭破坏。如果撤销文件的拥有者和正在编辑文件的不同，也忽略之，除非撤销文件的拥有者是当前用户。设置 'verbose' 可获取关于打开文件的消息。

撤销文件通常保存在文件本身相同的目录里。这可以用 'undodir' 选项改变。

如果文件被加密，撤销文件的文件也会加密。使用相同的密钥和方法。 encryption

备注 撤销文件不保存文本属性。只要缓冲区载入了就可以恢复文本属性，但你不能从撤销文件中恢复。理据：这需要和之前以完全相同的方式定义相关联的文本属性类型，而这不能保证。

你也可以用 ":wundo" 和 ":rundo" 来相应地保存和恢复撤销历史：

:wundo :rundo

:wundo[!] {file}
把撤销历史写入 {file}。
如果 {file} 已存在而看起来不像撤销文件（文件头部的魔术数字不符），此命令失败。除非加上 !。
如果文件存在且看起来像撤销文件，覆盖之。如果没有撤销历史，不写入任何东西。
实现细节：覆盖操作如此执行，先删除已有的文件，然后建立同名的文件。所以，不能在写保护目录里覆盖已有的撤销文件。

:rundo {file} 从 {file} 读出撤销历史。

你可以在自动命令中用这些命令来显式指定历史文件名。例如：

```
au BufReadPost * call ReadUndo()
au BufWritePost * call WriteUndo()
func ReadUndo()
  if filereadable(expand('%:h'). '/UNDO/' . expand('%:t'))
    rundo %:h/UNDO/%:t
  endif
endfunc
func WriteUndo()
  let dirname = expand('%:h') . '/UNDO'
  if !isdirectory(dirname)
    call mkdir(dirname)
  endif
  wundo %:h/UNDO/%:t
endfunc
```

此时，应该关闭 'undofile'，否则每次写入文件时会有两个撤销文件。

可以用 undofile() 函数来确定 Vim 会使用的文件名。

注意：读写文件时，如果设置了 'undofile'，大部分的错误信息会被屏蔽，除非设置了 'verbose'。而 :wundo 和 :rundo 会得到更到的错误信息，如文件不能读或写等。

注意：Vim 从不删除撤销文件。你需要自己进行清除。

读出已经存在的撤销文件可能会有以下的失败原因：
E822 不能打开，文件许可权限不许可。

E823 文件头的魔术数字不符。通常这意味着这不是一个撤销文件。
E824 撤销文件的版本号说明它是由更新的 Vim 版本写入的。需要更新的版本来读。如果你想保留该文件中的撤销信息，不要写入缓冲区。
"File contents changed, cannot use undo info"
文件文本和撤销文件写入时的不同。这意味着不再能使用该撤销文件，否则会破坏文本。这也可能是因为 'encoding' 和撤销文件写入时的不同。
E825 撤销文件没有合法的内容，不能使用。
E826 撤销文件经过加密而解密失败。
E827 撤销文件经过加密而此版本的 Vim 不支持加密。用别的 Vim 打开该文件。
E832 撤销文件经过加密而 'key' 没有设置，文本文件本身没有加密。这可能是因为文本文件本来是由 Vim 用加密方式写入的，后来又用非加密方式进行了覆盖。最好删除该撤销文件。
"Not reading undo file, owner differs"
撤销文件的拥有者和文本文件的拥有者不同。因为安全原因，不使用该撤销文件。

写入撤销文件可能会有以下的失败原因：
E828 待写入的文件不能建立。可能你没有该目录的写入权限。
"Cannot write undo file in any directory in 'undodir'"
'undodir' 里没有一个可用的目录。
"Will not overwrite with undo file, cannot read"
待写入的撤销文件已经存在，但不能读回。你需要先删除或改名。
"Will not overwrite, this is not an undo file"
待写入的撤销文件已经存在，但开头没有正确的魔术数字。你需要先删除或改名。
"Skipping undo file write, nothing to undo"
没有可写的撤销信息，没有任何改变，或者 'undolevels' 为负。
E829 写入撤销文件时有错。可以多试几次。

---

6. 撤销操作的解释 undo-remarks

能记忆的最大改变次数由 'undolevels' 选项决定。如果它的值是零，我们总是运行在 Vi-兼容模式。如果它的值是负的，任何撤销都是不可能的。这只有在内存紧张的时候适用。

clear-undo
如果设置 'undolevels' 为 -1，撤销信息不会立即清除。当有新改变时才会。要强迫清除撤销信息，可以用：

```
:let old_undolevels = &undolevels
:set undolevels=-1
:exe "normal a \<BS>\<Esc>"
:let &undolevels = old_undolevels
:unlet old_undolevels
```

当前缓冲区的位置标记 ('a 至 'z) 随文本一起被保存和复原。

当所有的改变都被撤销时，缓冲区被标记为未改变。这时可以使用 ":q" 而不一定是 ":q!" 退出 Vim 。注意 未改变是相对文件的最后写入而言的。在写入 ":w" 后紧跟的撤销 "u"，实际上改变了被写入以后缓冲区的状态。因此，此时缓冲区应被视为已改变。

当使用手动 folding 时，折叠不会被保存和复原。只有完全发生折叠内部的改变才不会影响该折叠，因为它开头和最后一行没有改变。

数字编号的寄存器也可以被用作撤销删除操作。你每一次删除文本，该文本被放在 "1 寄存器中。同样的，"1 寄存器的内容被移到 "2 寄存器，依次类推。"9 寄存器的内容则会丢失。现在，你可以通过命令 '"1P' 得到最近删除的文本。(如果被删除的内容来自最近的删除或复制操作，'P' 或 'p' 同样也可得到你要的结果，因为他们会复制未命名寄存器的内容)。使用 '"3P' 可以得到三个删除操作之前的文本。

redo-register

如果你想得到多于一处被删除文本的内容，你可以使用重复命令 "." 一个特殊的特性：它会递增所使用寄存器内的序号。所以，你如果先使用 ""1P" ，那么接下来的 "." 就意味着 '"2P 。重复这样的操作，所有编号的寄存器都会被插入。

例如：如果你用 'dd....' 删除了内容 (五行文本)，那么用 '"1P....' 可以恢复之。

如果你不知道被删除的内容在哪一个寄存器，你可以用 :display 命令。还有一个方法就是先试第一个寄存器 '"1P' ，如果不对，用 'u.' 命令。这将会移走第一次放进的内容，然后在第二个寄存器上重复 put 命令。重复使用 'u.' 直到你得到想要的内容为止。

vim:tw=78:ts=8:noet:ft=help:norl:

repeat.txt For Vim version 8.2. 最近更新: 2020年1月

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

重复命令，Vim 脚本和调试 repeating

用户手册第 26 章 usr_26.txt 介绍了重复命令。

1. 单次重复 single-repeat
2. 多次重复 multi-repeat
3. 重复组合操作 complex-repeat
4. 使用 Vim 脚本 using-scripts
5. 使用 Vim 软件包 packages
6. 创建 Vim 软件包 package-create
7. 调试脚本 debug-scripts
8. 刨视 profiling

---

1. 单次重复 single-repeat

. 重复上次改变，有计数前缀的重复 [count] 次。
如果 'cpoptions' 包括 'y' 标志， 也可以用来重复一个抽出 (yank) 操作。不能重复命令行命令。

简单的改变操作可以用 "." 重复。如果没有数字前缀，使用上次的改变的重复次数。如果输入一个重复次数，就用它取代上次的计数。设置 v:count 和 v:count1。

如果上次改变的操作使用数字编号的寄存器，则逐次递增使用寄存器的序号。一个使用的示例可以参见 redo-register 。

注意，如果重复的命令涉及可视的选择区域，使用相同 大小 的区域。参见 visual-repeat 。

@: 重复上次命令行 [count] 次。
{仅当编译时加入 +cmdline_hist 特性才可用}

---

2. 多次重复 multi-repeat

:g :global E148

:[range]g[lobal]/{pattern}/[cmd]
在 [range] 界定的匹配模式 {pattern} 的文本行上执行 Ex 命令 [cmd] (缺省是 ":p")。

:[range]g[lobal]!/{pattern}/[cmd]
在 [range] 界定的 不 匹配模式 {pattern} 的文本行上执

行 Ex 命令 [cmd] (缺省是 ":p")。

:v :vglobal

:[range]v[global]/{pattern}/[cmd]
等同于 :g!。

除了用 '/' 来包围 {pattern} 以外，你可以用任何其它单字节字符，但不包括字母、'\'、'"' 或 '|'。这对你想在搜索模式或者替代字符串里包含 '/' 很有用。

模式的定义见 pattern 。

注意 [cmd] 可能包含范围；示例见 collapse 和 edit-paragraph-join。

global 命令先扫描 [range] 界定的行，然后对每个匹配的行进行标记 (对于一个跨行的匹配，则只针对开始的一行)。
第二次扫描对每个标记的行，就像光标在那行那样，执行指定的 [cmd] 命令。 ":v" 和 ":g!" 命令针对未标记的行进行。如果一行被删除，其标记也自动消失。
缺省的 [range] 是整个缓冲区 (1,$)。用 "CTRL-C" 中断该命令。如果某行的执行给出错误，该行的执行被中断，但 global 命令仍从下一个被标记或未被标记的行继续。

E147
命令递归使用时，只能作用于一行。此时不能给出范围。可用于找到所有匹配某模式，但不匹配另一模式的行:

```
:g/found/v/notfound/{cmd}
```

这会先找到所有包含 "found" 的行，但只当不匹配 "notfound" 时才执行 {cmd}。

要执行一个非 Ex 的命令， 你可以使用 ":normal" 命令:

```
:g/pat/normal {commands}
```

要确保 {commands} 以完整的命令结束，不然 Vim 会等待你输入该命令余下的部分，对每次匹配都将如此。由于屏幕此时还未更新，你恐怕会因此不知所措。参见 :normal 。

撤销/重做命令将会针对整个 global 命令。换而言之，前次上下文标记只有在命令开头会设一次 (用 "''" 可以回到 global 之前的光标位置)。

global 命令也设置最近使用的搜索模式和最近使用的替换模式 (后者是 vi 兼容的)，以便于全局替换一个字符串:

```
:g/pat/s//PAT/g
```

该命令替换所有的 "pat" 为 "PAT"。同样的功能也可以用:

```
:%s/pat/PAT/g
```

实现。这样省下了两个字符!

Ex 模式下执行 "global" 时，使用 ":visual" 命令是特例。该命令移动到匹配行，进入普通模式并在该模式下执行命令，直到你用 Q 回到 Ex 模式为止。对每个匹配行重复此操作。此时不能再用 ":global"。要中止，按 CTRL-C 两次。

---

3. 重复组合操作 complex-repeat

q q recording

q{0-9a-zA-Z"} 在寄存器 {0-9a-zA-Z"} 里记录键入的字符 (大写名字的寄存器表示附加键入的内容)。'q' 命令不能在执行寄存器时使用。同样，在映射和 :normal 里也不能。

注意: 如果用作记录的寄存器用于 y 和 p，其结果多数不如你的预想，因为 p 会粘贴记录的宏，而 y 会覆盖记录的宏。

q　　停止记录。(注意 实现细节：停止记录的 'q' 不会被保存在寄存器里，除非它是映射的结果)

@{0-9a-z".=*+}　　执行寄存器 {0-9a-z".=*+} 的内容 [count] 次。注意 不能用寄存器 '%'（当前文件名）和 '#'（轮换文件名）。
寄存器的执行方式与映射类同，'wildchar' 和 'wildcharm' 的差异适用此处。
"@=" 则会提示你输入一个表达式。这个表达式的结果会被执行。
参见 @: 。

@@　　执行上次执行的 @{0-9a-z":*} [count] 次。

:[addr]*{0-9a-z".=+}
:[addr]@{0-9a-z".=*+}　　把寄存器 {0-9a-z".=*+} 里的内容看成一个 Ex 命令执行。
首先，把光标放在 [addr] 行（缺省是当前行）上。如果 'cpoptions' 设定了 'e' 标志位而寄存器的内容最后一行没有 <CR>，会自动加上。
注意 ":*" 命令只有在 'cpoptions' 设置 '*' 标志位时才会识别。如果用 'nocompatible'，这 并 非缺省。
":@=" 使用最近使用的表达式。表达式执行的结果被当作 Ex 命令执行。
这些命令里，不识别映射。
将来：会对地址范围里每一行执行寄存器的命令。

:[addr]@:　　先把光标放在 [addr] 行（缺省是当前行）上，然后重复上次的命令行。

:[addr]@
:[addr]@@　　先把光标放在 [addr] 行（缺省是当前行）上，然后重复上次的 :@{0-9a-z"}。

---

4. 使用 Vim 脚本　　using-scripts

参阅用户手册第 41 章 usr_41.txt 了解如何写 Vim 脚本。

:so[urce] {file}　　从 {file} 里读取 Ex 命令，即 ":" 开头的命令，并执行。
激活 SourcePre 自动命令。

:so[urce]! {file}　　从 {file} 里读取 Vim 命令，就像你在普通模式下键入的命令一样。
如果要执行的命令在 :global 、 :argdo 、 :windo 或 :bufdo 之后、在循环体内、或者有另外一个命令紧跟其后，那么执行时不会更新屏幕显示。
不能用于 sandbox 。

:ru :runtime

:ru[ntime][!] [where] {file} ..
从 'runtimepath' 和/或 'packpath' 指定的目录里查找 {file}。从匹配的文件里读取 Ex 命令。如果没有匹配的文件，不报错。

例如：

```
:runtime syntax/c.vim
```

可以指定多个以空格分隔的 {file} 参数。每个 {file} 都从 'runtimepath' 指定的第一个目录开始查找，然后是第二个、第三个，等等。{file} 里可以通过加反斜杠来包含空格（不过，为了避免麻烦，最好不要在文件名里使用空格）。

如果使用了 [!]，所有找到的文件都被执行。否则，只执行第一个找到的文件。

如果 [where] 省略，只使用 'runtimepath'。其它可用值是：

START	搜索 'packpath' 里的 "start"
OPT	搜索 'packpath' 里的 "opt"
PACK	搜索 'packpath' 里的 "start" 和 "opt"
ALL	先用 'runtimepath'，然后搜索 'packpath' 里的 "start" 和 "opt"

如果 {file} 包含通配符，它被扩展为所有的匹配文件名。例如：

```
:runtime! plugin/**/*.vim
```

这是 Vim 启动的时候启动插件所使用的命令。类似的：

```
:runtime plugin/**/*.vim
```

只会执行其中的第一个文件。

当 'verbose' 至少为一时，如果没有文件找到，会显示信息。
当 'verbose' 至少为二时，对每个搜索到的文件都会提示信息。

:pa :packadd E919

:pa[ckadd][!] {name}
在 'packpath' 里搜索可选的插件目录，然后执行找到的插件文件。目录必须匹配：

```
pack/*/opt/{name}
```

如果目录还不在 'runtimepath'，加入之。
如果目录 pack/*/opt/{name}/after 存在，在 'runtimepath' 的尾部加入之。

如果之前跳过 "pack/*/start" 的软件包载入，则先搜索此目录：

```
pack/*/start/{name}
```

注意 {name} 是目录名，而不是 .vim 文件名。所有匹配模式的

```
pack/*/opt/{name}/plugin/**/*.vim
```

文件都被执行。这样可以允许 "plugin" 下使用子目录，就像 'runtimepath' 里的插件一样。

如果还未打开文件类型检测（这通常是在 .vimrc 里的 "syntax enable" 或 "filetype on" 命令完成的），也会寻找 "{name}/ftdetect/*.vim" 这样的文件。

如果加入可选的 !，不载入插件文件或 ftdetect 脚本，只把匹配的目录加入 'runtimepath'。这可用于 .vimrc。插件会随后在初始化时载入，见 load-plugins 。

另见 pack-add 。
{仅当编译时加入 +eval 特性才有功能}

:packl :packloadall

:packl[oadall][!] 载入 'packpath' 每个项目的 "start" 目录的所有软件包。

首先把所有找到的目录加入 'runtimepath'，然后执行所有目录里找到的插件。这允许一个插件依赖于另一个插件的某个部分，例如 "autoload" 目录。 packload-two-steps 说明此顺序所起的作用。

通常在启动时在载入 .vimrc 文件后自动完成此步骤。直接用本命令，可以在更早时执行这一步。

软件包只载入一次。再次使用 :packloadall 没有效果。但如果加入可选的 !，此命令会再次载入已经载入过的软件包。

注意 在 vimrc 文件中使用 :packloadall 时，刷新 'runtimepath' 选项，之后会载入 'runtimepath' 中所有的插件，这意味着它们会被再次载入。插件应能对此正确处理。

执行脚本发生错误时终止错误发生的脚本，继续载入后续的插件。见 packages 。
{仅当编译时加入 +eval 特性才有功能}

:scripte[ncoding] [encoding] :scripte :scriptencoding E167

指定脚本使用的字符编码。后续以 [encoding] 编码的脚本行会被转换成 'encoding' 选项所指定的编码，如果两者不同的话。如:

```
scriptencoding iso-8859-5
scriptencoding cp932
```

如果 [encoding] 为空，则不作任何转换。这可以用来避免对一些行进行转换:

```
scriptencoding euc-jp
... 被转换的行 ...
scriptencoding
... 不被转换的行 ...
```

如果系统不支持所需的转换，不会有错误信息，但转换也不会发生。如果不能转换某一行，不报错，保留原来的行。

不要用 "ucs-2" 或者 "ucs-4"。Vim 不能用这些编码的脚本（因为其中会有 NUL 字节）。如果一个待执行的脚本以一个 BOM (Byte Order Mark 字节顺序标记) 开头，Vim 会识别其为 utf-8 编码，从而无须指定 ":scriptencoding utf-8"。

如果在 .vimrc 里设置 'encoding' 选项，
:scriptencoding 必须在放在它后面。例如:

```
set encoding=utf-8
scriptencoding utf-8
```

:scriptv[ersion] {version} :scriptv :scriptversion E999 E984

指定同一文件中其后的行所使用的 Vim 版本。只适用于被执行脚本的顶层，不适用于函数体内。

如果 {version} 高于当前 Vim 版本所能支持的，给出 E999 错误。你可能要重写脚本以支持旧版本的 Vim 或更新 Vim 版本。版本间的变化见 vimscript-version 。

:scr[iptnames] :scr :scriptnames

列出所有执行过的脚本名字，以它们初次执行之顺序排列。排列的次序号码被用作相应的脚本 ID <SID> 。
{仅当编译时加入 +eval 特性才有功能}

:scr[iptnames][!] {scriptId} :script

编辑脚本 {scriptId}。尽管 ":scriptnames name" 也可以，建议使用 ":script name"。
如果当前缓冲区不能被 abandon 且 ! 不给出，本命令失败。

:fini[sh] :fini :finish E168

停止执行脚本。只能用在 Vim 脚本中，来快速跳过脚本的其余内容。如果出现在 :try 之后但在相应的 :finally (如果存在的话) 之前，":finally" 到 :endtry 的内容还会执行。执行完所有嵌套的 ":try" 层的 ":finally" 代码后，最外层的 ":endtry" 才会最终真正停止脚本的执行。

所有的命令和命令序列可以通过把它们放在命名的寄存器里执行来重复调用。有两个方法可以把命令放在寄存器里:

- 用记录命令 "q"。可以键入一串命令，在执行的同时它们被存入一个寄存器里。这很简明，因为你能看见你所做的事情。如果你敲错了，用 "p" 把寄存器的内容 "放置" 在一个文件里，然后编辑这个命令序列，把它们再次放回到 (比如，用删除命令) 寄存器里。你也可以用大写名字的寄存器名来附加命令，从而继续上次的纪录。
- 删除 (delete) 或者抽出 (yank) 命令序列到寄存器。

常用的命令序列可以用 ":map" 命令映射到一个功能键上。

另外一个办法则是把命令写到一个文件里，用 ":source!" 命令执行之。这对很长的命令序列有用。你甚至可以把它和 ':map' 命令混合使用，从而用一个功能键来储存复杂的功能。

':source' 命令从文件里逐行读取 Ex 命令。如果其间需要键盘输入，你需要自己键入。'source!' 命令从脚本里逐字读取命令，就像你自己敲入每一个字符一样。

示例: 如果你给出一个 ':!ls' 命令，你得到一个 hit-enter 提示。如果你用 ':source' 执行包含 '!ls' 一行的文件，你必须按一个回车。不过如果你用 ':source!' 来执行包含 ':!ls' 的文件，其后的字符会依次读入，直到遇到一个 <CR> 为止。你不需

要通过键盘键入这个 <CR>，除非 ":!ls" 是文件的最后一行。

在脚本里当然也可以有 ':source[!]' 命令，从而可以建立一个自顶而下的脚本调用树。':source' 命令允许的嵌套深度由同时打开的最大的文件数目决定（大概 15 个左右），':source!' 命令许可的嵌套深度则最多为 15 层。

在被执行的文件里，你可以在需要文件名的地方用 "<sfile>" 字符串（直接的文本，不是一个特殊的键）。它会被被执行的文件的名字来代替。例如，如果你在 ".vimrc" 文件相同的目录里有一个 "other.vimrc" 文件，你可以在 ".vimrc" 里如此调用它：

```
:source <sfile>:h/other.vimrc
```

在脚本文件里，依赖于终端的键码由不依赖于终端的两个字符码代表。这样，他们就能在不同的终端里代表相同的意义。这里，第一个字符码是 0x80 或者 128，屏幕上显示为 "~@"。第二个字符可以在 key-notation 列表里找到。这些编码也可以用 CTRL-V 加上一个三位数字的十进制码来键入。这个方法 不 适用于 <t_xx> termcap 代码，它们只能用在映射里。

:source_crnl W15

Win32: 用 ":source" 执行的文件通常每行以 <CR><NL> 结尾。这没有问题。在 'fileformats' 非空并且第一行不以 <CR> 结尾的时候，用 <NL> 的行结尾（比如，Unix 编写的文件）会被识别。不过，如果第一行里有 ":map <F1> :help^M" 这样的内容（其中 ^M 是一个 <CR>），这个机制会失败。如果第一行以 <CR> 结尾，但其后的行不是，那你会得到错误信息，因为第一行里的 <CR> 会丢失。

Mac Classic: 用 ":source" 执行的文件通常每行以 <CR> 结尾。这没有问题。在 'fileformats' 非空并且第一行不以 <CR> 结尾的时候，用 <NL> 的行结尾（比如，Unix 编写的文件）会被识别。如果用 <NL> 行结尾的时候，要小心第一行不要有 <CR>。

在别的系统上，Vim 期待 ":source" 执行的文件以 <NL> 行结尾。这没有问题。如果你的文件用 <CR><NL> 结束行（比如，MS-DOS 编写的文件），所有的行都会有一个附尾的 <CR>。有些命令（比如映射命令）会因此有问题。这里不使用自动换行符识别机制，因为第一行出现以 <CR> 结尾的映射命令很常见，自动机制这时容易出错。

line-continuation

":source" 执行的 Ex 命令脚本里的长行可以用通过在下一行的开始插入续行符 "\"（反斜杠）来分开。反斜杠之前可以出现空格，它们将被忽略。

示例：如下几行

```
:set comments=sr:/*,mb:*,el:*/,
                \://,
                \b:#,
                \:%,
                \n:>,
                \fb:-
```

会被解读为一行：

```
:set comments=sr:/*,mb:*,el:*/,://,b:#,:%,n:>,fb:-
```

每行反斜杠之前的所有引导空白字符会被忽略。注意：在此之前的一行的行尾的空格可能不能随便添加；这由命令分开的位置决定，那里也许可以也许不可以有额外的空格。

需要空格时最好放在反斜杠之后。行尾的空格很难注意到，很可能会被意外删除掉。

```
:syn match Comment
        \ "very long regexp"
        \ keepend
```

在用 ":append" 和 ":insert" 命令时，有一个问题：

```
:1append
\asdf
.
```

反斜杠被看作续行符，所以这等价于以下命令：

```
:1appendasdf
.
```

要避免这一点，在 'cpoptions' 选项里加上 'C' 标志位：

```
:set cpo+=C
:1append
\asdf
.
:set cpo-=C
```

要 注意 在函数里的命令里用到这些命令时，你需要在定义函数的时候加上 'C' 标志位，而不是执行函数时。

```
:set cpo+=C
:function Foo()
:1append
\asdf
.
:endfunction
:set cpo-=C
```

line-continuation-comment

要在行与行间加入注释，以 '"\ ' 开头。留心反斜杠之后的空格。例如：

```
        let array = [
                "\ first entry comment
                \ 'first',
                "\ second entry comment
                \ 'second',
                \ ]
```

原理：

许多程序用行尾的反斜杠来指示行要继续。如果这样，Vim 会和 Vi 不兼容。例如下面的 Vi 映射：

```
:map xx  asdf\
```

因此，只能用特殊形式的出现在行首的反斜杠。

以注释开始某继续行会使得所有其它的继续行都成为注释的一部分。这样的行为已经很长时间了，虽然可能在继续行序列半中间加入注释，但过去不能使用 \"，因为这不是合法的继续行。'"\ ' 是最接近的，虽然看来有点古怪。在反斜杠后需要空格，是因为这样就很难和正常的注释行混淆。

---

5. 使用 Vim 软件包 packages

Vim 软件包是包含一个或多个插件的目录。和普通插件相比，它的优点是：

- 软件包可作为归档下载，并可解压到单独的目录。这样文件就不会和其它插件的文件混杂。这样方便更新和删除。
- 软件包可以用 git、mercurial 等版本库。这样更新尤其方便。
- 软件包可包含相互依赖的多个插件。
- 软件包可包含启动时自动载入的插件和通过 :packadd 加入仅当需要时才载入的插件。

使用软件包并自动载入

让我们假定，你的 Vim 文件在 "~/.vim" 目录里，而你想加入来自 zip 归档 "/tmp/foopack.zip" 的软件包：

```
% mkdir -p ~/.vim/pack/foo
% cd ~/.vim/pack/foo
% unzip /tmp/foopack.zip
```

目录名 "foo" 是任意的，可以选任何名字。

现在在 ~/.vim 里就有这些文件：

```
pack/foo/README.txt
pack/foo/start/foobar/plugin/foo.vim
pack/foo/start/foobar/syntax/some.vim
pack/foo/opt/foodebug/plugin/debugger.vim
```

Vim 启动时，处理完 .vimrc 后，扫描 'packpath' 里的所有目录，寻找 "pack/*/start" 目录里的插件。首先把这些目录加入 'runtimepath'，然后载入所有插件。 packload-two-steps 说明此顺序所起的作用。

此例中，Vim 会找到 "pack/foo/start/foobar/plugin/foo.vim" 并把 "~/.vim/pack/foo/start/foobar" 加入 'runtimepath'。

如果 "foobar" 插件把 'filetype' 设为 "some"，Vim 会找到 syntax/some.vim 文件，因为此目录在 'runtimepath' 里。

如果有的话，Vim 也会载入 ftdetect 文件。

注意 不自动载入 "pack/foo/opt" 里的文件，只载入 "pack/foo/start" 里的。下面的 pack-add 说明 "opt" 目录的用法。

如果关闭了插件的载入，不会自动载入软件包，见 load-plugins 。

要再早些载入软件包，以便更新 'runtimepath'：

```
:packloadall
```

这里即使关闭了插件载入也可以。自动载入只会发生一次。

如果软件包有 "after" 目录，此目录加入 'runtimepath' 的尾部，这样那里的内容会较后载入。

使用单个插件并自动载入

如果没有软件包但有单个插件，需要建立额外的目录结构：

```
% mkdir -p ~/.vim/pack/foo/start/foobar
% cd ~/.vim/pack/foo/start/foobar
% unzip /tmp/someplugin.zip
```

现在有这些文件：

```
pack/foo/start/foobar/plugin/foo.vim
pack/foo/start/foobar/syntax/some.vim
```

此后，工作方式就和上面一样。

可选插件

pack-add

要从软件包里载入可选插件，用 :packadd 命令：

```
:packadd foodebug
```

从 'packpath' 里搜索 "pack/*/opt/foodebug"，找到 ~/.vim/pack/foo/opt/foodebug/plugin/debugger.vim 并执行之。

可以只在满足某些条件时才这样做。例如，取决于 Vim 是否支持某特性或缺少某依赖关系。

也可以在启动时载入可选插件，在 .vimrc 里放上：

```
:packadd! foodebug
```

额外的 "!" 确保 Vim 以 --noplugin 启动时不载入插件。

软件包只包含 "opt" 目录的文件是完全正常的。每个插件只有在需要用的时候才载入。

什么放在哪里

因为通过 :colorscheme 载入的色彩方案可在 "pack/*/start" 和 "pack/*/opt" 下找到，你可以把它们放在任何地方。我们建议你放在 "pack/*/opt" 底下，例如 ".vim/pack/mycolors/opt/dark/colors/very_dark.vim"。

文件类型插件应该在 "pack/*/start" 下，这样总能找到它们。除非某个文件类型有多于一个插件，而你想用 :packadd 选择载入哪个。例如，取决于编译器版本：

```
if foo_compiler_version > 34
  packadd foo_new
else
  packadd foo_old
endif
```

软件包内最没用的是 "after" 目录。但不禁止。

---

6. 建立 Vim 软件包

package-create

这里假定你在编写一个或多个插件，并想以软件包形式分发。

两个无关的插件要用两个软件包，这样 Vim 用户可以选择要装什么。也可以决定用带可选插件的软件包，然后告知用户用 :packadd 加入他们想要的。

你要决定你如何分发软件包。可以建立一个归档或使用版本库。归档可以被更多的用户使用，但更新版本要困难一些。版本库可以方便的保持更新，但需要 "git" 之类的程序。也可以两者都用，github 可以自动为发布版建立归档。

目录布局应该是这样：

```
start/foobar/plugin/foo.vim          " 总是载入，定义命令
start/foobar/plugin/bar.vim          " 总是载入，定义命令
start/foobar/autoload/foo.vim        " 使用 foo 命令时载入
start/foobar/doc/foo.txt             " foo.vim 的帮助
start/foobar/doc/tags                " 帮助标签
opt/fooextra/plugin/extra.vim        " 可选插件，定义命令
```

```
opt/fooextra/autoload/extra.vim      " 使用 extra 命令时载入
opt/fooextra/doc/extra.txt           " extra.vim 的帮助
opt/fooextra/doc/tags                " 帮助标签
```

这样用户就可以:

```
mkdir ~/.vim/pack/myfoobar
cd ~/.vim/pack/myfoobar
git clone https://github.com/you/foobar.git
```

这里 "myfoobar" 是用户可选的名字，唯一的条件是和其它软件包的名字不同。

在你的文档里，你要解释插件的用途，并告知用户如何载入可选插件:

```
:packadd! fooextra
```

可以在你提供的插件之一加入此 packadd 命令，在需要可选插件时执行。

运行 :helptags 命令生成文档/标签文件。把此生成文件包含在软件包内，用户就可以把软件包放在自己的 pack 目录里，立即可以使用 help 命令。不要忘记改变插件帮助后重新运行此命令:

```
:helptags path/start/foobar/doc
:helptags path/opt/fooextra/doc
```

插件间的依赖关系

packload-two-steps

假定你有两个插件，它们都依赖于同一个功能。可以把公共的功能放在 autoload 目录里，以便自动找到。你的软件包里有以下文件:

```
pack/foo/start/one/plugin/one.vim
        call foolib#getit()
pack/foo/start/two/plugin/two.vim
        call foolib#getit()
pack/foo/start/lib/autoload/foolib.vim
        func foolib#getit()
```

这样可行，因为软件包在载入时先把所有找到的目录加入 'runtimepath'，然后再执行这些插件。

---

7. 调试脚本

debug-scripts

除了在自己的脚本里提示明显的消息，Vim 提供了调试模式来让你了解自己的代码在做什么。你可以单步执行脚本文件和函数和设置断点。

请 注意: 调试模式远未完善。调试程序会对 Vim 的工作产生副作用。你不能用它调试一切细节。例如，调试信息会弄乱屏幕的显示。

另外一个办法是设置 'verbose' 选项。设置一个比较大的数字，你会得到 Vim 在做什么的更详尽的信息。

启动调试模式

debug-mode

以下方法可以进入调试模式:

1. 用 -D 参数启动 Vim:
   ```
   vim -D file.txt
   ```
   调试会在执行第一个 vimrc 文件的时候开始。这有助于了解 Vim 启动的时候干了些什么。一个副作用是 Vim 会在初始化完成之前切换终端模式，这会有意想不到的后果。
   对只用 GUI 的版本 (Windows、Macintosh) 调试会在 GUI 窗口打开的一刻开始。要提早进入调试，在 vimrc 文件里加上 ":gui" 命令。

:debug

2. 执行命令前加上 ":debug" 前缀。这样，调试只对这个命令进行。这对调试某一个特定的脚本和用户函数，或者 autocommands 用到的脚本和函数有用。例如:
   ```
   :debug edit test.txt.gz
   ```

3. 在一个执行的文件和用户函数里设置断点。你可以在命令行里这么做:
   ```
   vim -c "breakadd file */explorer.vim" .
   ```
   这会启动 Vim 并在 "explorer.vim" 脚本的第一行停下。进入调试模式后也可以设置断点。

在调试模式里，每个命令都会在执行前被显示。注释行、空行和其他不执行的行会被跳过。如果一行里有两个 "|" 分隔的命令，它们被分别显示。

调 试 模 式

进入调试模式以后，可以使用通常的 Ex 命令。比如，要检查某变量的值:
```
echo idx
```
在一个用户函数里，这会显示局域变量 "idx" 的值。在变量前加上 "g:" 可以得到全局变量的值:
```
echo g:idx
```
所有的命令都在当前函数或脚本的上下文下执行。你可以设置选项，比如设置或者重设 'verbose' 来显示当前发生的事情，但是你需要在执行你感兴趣的行之前设置它们:
```
:set verbose=20
```

要避免更新屏幕的命令，因为直到退出调试模式之前，无法看到它们的效果。例如，帮助命令:
```
:help
```
不会很有帮助。

调试模式有自己的命令行历史。

函数行的行号是相对于函数开始的地方的。如果你要知道你在哪里，在另外一个 Vim 里编辑包含这个函数的文件，先找到函数的开始处，然后用 "99j"，其中的 "99" 用实际的行号代替。

另外，可以使用如下的命令:

>cont

cont　　继续执行到下一个断点。

>quit

quit　　终止执行。这和 CTRL-C 类似，但是还是有要执行的东西，并不是所有的都中止。在遇到断点时还是会停下来。

>next

next　　执行一个命令，并在它结束时返回调试模式。步过用户函数调用和被执行的脚本。

>step

step　　执行一个命令，并在它结束时返回调试模式。步入用户函数

调用和被执行的脚本。

>interrupt

interrupt 类似于 CTRL-C，但不同于 ">quit"，这会在执行下个命令前返回到调试模式。它有助于测试 :finally 和 :catch 对中断例外的处理。

>finish

finish 结束当前的脚本或者用户函数，并在调用该脚本或函数的行的下一行之前返回到调试模式。

>bt
>backtrace
>where

backtrace 显示当前调试会话的调用堆栈回溯。
bt
where

>frame

frame N 转到 N 层堆栈回溯。+ 和 - 标记进行相对移动。例如，":frame +3" 往上三层堆栈。

>up

up 向上一层调用堆栈回溯。

>down

down 向下一层调用堆栈回溯。

关于调试模式下的附加命令：
- 没有它们的命令行自动补全。补全只对一般的 Ex 命令有效。
- 除非有多于一个命令以相同字母开始，可以用一个字符来简写。"f" 代表 "finish"，"fr" 代表 "frame"。
- 按 <CR> 会重复上一个调试命令。在执行另一个命令之后，这个功能会被关掉（因为不知道你要重复什么）。
- 如果你想调用相同的名字的 Ex 命令，在前面附加一个冒号：":cont"、":next"、":finish"（或者它们的缩写）。

堆栈回溯显示函数调用的层级，例如：

```
>bt
  3 function One[3]
  2 Two[3]
->1 Three[3]
  0 Four
line 1: let four = 4
```

"->" 指向当前堆栈。用 "up"、"down" 和 "frame N" 可选择另一层堆栈。

在当前栈中可以计算本地函数变量。还没有办法看到当前行所在的命令。

定义断点

:breaka :breakadd

:breaka[dd] func [lnum] {name}
在函数上设置断点。例如：

```
:breakadd func Explore
```

它不会检查函数名的合法性。这样，断点可以在函数定义之前设置。

:breaka[dd] file [lnum] {name}
在一个脚本文件上设置断点。例如：

```
:breakadd file 43 .vimrc
```

:breaka[dd] here
在当前文件的当前行上设置断点。类似于:

```
:breakadd file <cursor-line> <current-file>
```

注意 只能用于执行此文件时会执行的命令，不适用于该文件此处定义的函数。

:breaka[dd] expr {expression}
设置断点，任何时候 {expression} 的计算值发生改变时中断。如:

```
:breakadd expr g:lnum
```

会在任何时候全局变量 lnum 改变时中断。
注意 如果观察 script-variable ，则脚本切换时会发生中断，因为脚本变量只在其定义所在的脚本里有效。如果其它脚本调用此脚本，则任何该变量变成可见或不可见的时候，中断都会发生。

其中 [lnum] 是断点所在的行号。Vim 会在它或它之后的行上停止。如果省略，就用行号 1。

:debug-name
{name} 是用来匹配文件名或者函数名的模式。这和 autocommands 所有的模式类似。它必须是一个完整匹配 (如同它以 "^" 开头和 "$" 结尾一样)。"*" 匹配任何字符序列。它不用 'ignorecase' 选项，但是模式里可以用 "\c" 来忽略大小写 /\c 。不要给函数名加上 ()!

对脚本文件的匹配给予它的完整文件名。例如:

```
breakadd file explorer
```

不会匹配。因为没有给出路径。

```
breakadd file *explorer.vim
```

匹配 ".../plugin/explorer.vim" 和 ".../plugin/iexplorer.vim"。

```
breakadd file */explorer.vim
```

只匹配 ".../plugin/explorer.vim"。

对函数名的匹配基于 ":function" 输出结果里它的显示方式。对于局部函数，这意味着函数名前会附加上类似 "<SNR>99_" 的字符串。

注意 函数先被载入然后再执行。载入时，检查 "file" 断点，而执行时检查 "func" 断点。

删除断点

:breakd :breakdel E161

:breakd[el] {nr}
删除断点 {nr}。用 :breaklist 可以看到每个断点的编号 {nr}。

:breakd[el] *
删除所有断点。

:breakd[el] func [lnum] {name}
删除函数断点。

:breakd[el] file [lnum] {name}
删除脚本断点。

:breakd[el] here
删除当前文件的当前行上的断点。

如果 [lnum] 省略，删除该函数或者文件的第一个断点。
{name} 必须和 ":breakadd" 所匹配的名字完全相同。"explorer"、"*explorer.vim" 和 "*explorer*" 是不一样的。

列出断点

:breakl :breaklist

:breakl[ist]
列出所有的断点。

不常用的

:debugg :debuggreedy

:debugg[reedy]
从正常的输入流而不是直接从用户输入里读取调试模式命令。这只对测试脚本有用。例如:

```
echo 'q^Mq' | vim -e -s -c debuggreedy -c 'breakadd file script.vim' -S script.v
```

:0debugg[reedy]
撤销 ":debuggreedy": 从用户那里直接读取调试模式命令，而不为调试命令预读取。

---

8. 刨视

profile profiling

进行刨视意味着 Vim 测量执行函数和/或脚本时使用的时间。为此，需要 +profile 特性。只有 Vim 编译时使用了 "huge" (巨大) 特性包才会包含此特性。

你也可以用 reltime() 函数来测量时间。这只需要 +reltime 特性，它更常出现。

关于语法高亮的刨视可见 :syntime 。

例如，要刨视 one_script.vim 脚本文件:

```
:profile start /tmp/one_script_profile
:profile file one_script.vim
:source one_script.vim
:exit
```

:prof[ile] start {fname} :prof :profile E750
启动刨视，退出时把结果写到 {fname} 文件。
扩展 {fname} 中的 "~/" 和环境变量。
如果 {fname} 已存在，安静地覆盖它。
变量 v:profiling 设为一。

:prof[ile] pause
直到下个 ":profile continue" 为止，停止刨视。可用来执行一些不需计算的操作 (例如，执行外部命令)。不能嵌套。

:prof[ile] continue
在 ":profile pause" 之后出现，恢复刨视。

:prof[ile] func {pattern}
刨视匹配模式 {pattern} 的函数。
:debug-name 说明如何使用 {pattern}。

:prof[ile][!] file {pattern}
刨视匹配模式 {pattern} 的脚本。
:debug-name 说明如何使用 {pattern}。
只刨视脚本自身，不包括其中定义的函数。
如果加入 [!]，同时刨视脚本里定义的所有函数。
注意 只有在此命令后载入脚本完成时才会开始刨视。脚本自身不可以启动 :profile 命令。

:profd[el] ... :profd :profdel
停止对指定参数的刨视，:breakdel 说明其中的参数。

刨视总是用 ":profile start fname" 开始。Vim 退出时写入结果文件。这里是输出的示例，加上行号是为了方便解释：

```
 1 FUNCTION  Test2()
 2 Called 1 time
 3 Total time:   0.155251
 4  Self time:   0.002006
 5
 6 count  total (s)   self (s)
 7     9              0.000096   for i in range(8)
 8     8   0.153655   0.000410     call Test3()
 9     8              0.000070   endfor
10                               " Ask a question
11     1              0.001341   echo input("give me an answer: ")
```

头部（行号 1-4）给出整个函数的时间。"Total" 时间是函数执行时使用的时间。"Self" 时间是 "Total" 时间减去用于下面事项的时间：
- 其它用户定义的函数
- 执行脚本
- 执行自动命令
- 外部（外壳）命令

行号 7-11 显示每个执行行花费的时间。不计算不执行的行。所以注释行从不参与计算。

Count 列显示每行执行的次数。注意 到第 7 行的 "for" 命令执行次数比后面的行要多一。这是因为该行也被用来检测循环结束。

Vim 花费在等待用户输入的时间完全不参与计算。所以 input() 提示后你的响应时间不相关。

刨视给出关于时间花费在哪里的很好的指示，但要记住很多因素会影响输出结果：

- 测量时间的准确度取决于 gettimeofday() 系统函数。精确度可能是 1/100 秒，但显示的时间仍以微秒计。

- 测量的是实际流逝的时间。如果其它进程繁忙，可能造成的延迟无法预测。你可能需要多次运行刨视，并使用时间最少的结果。

- 如果一行内执行多个命令，你只会得到一个时间。要看到个别命令的时间，把行分开。

- 各行时间的加总几乎总会比整个函数的时间要少。中间有少许开销。

- Vim 退出前删除的函数不会产生刨视信息。如果需要，你可以检查 v:profiling 变量:

```
:if !v:profiling
:   delfunc MyFunc
:endif
```

- 在多处理器系统中，因为睡眠模式或者为省电而减低处理器频率等因素影响，刨视结果可能不正常。

- "self" 时间对递归调用的函数是不正确的。

vim:tw=78:ts=8:noet:ft=help:norl:

visual.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者： quhongjun http://vimcdoc.sf.net

可视模式 Visual Visual-mode visual-mode

可视模式提供一种灵活易用的方法选择一块文本供操作符使用。它也是选择一个文本列块的唯一方法。

关于本章的介绍包含在用户手册的 04.4 节中。

1. 使用可视模式 visual-use
2. 进入和退出可视模式 visual-start
3. 改变可视区域 visual-change
4. 操作可视区域 visual-operators
5. 列块操作 blockwise-operators
6. 重复 visual-repeat
7. 示例 visual-examples
8. 选择模式 Select-mode

{仅当编译时打开 +visual 特性才有效}
{Vim 7.4.200 开始，总是包含 +visual 特性}

---

1. 使用可视模式 visual-use

使用可视模式包含三个部分：
1. 用 "v"、"V" 或 CTRL-V 标记文本的开始。
   当前光标下的字符将被作为标记的起始点。
2. 把光标移到要标记的文本末尾。
   起始点和光标之间的文本（包括当前光标下的字符）将被高亮显示。
3. 键入操作符命令。
   操作符命令将被应用到高亮显示的字符。

'highlight' 选项用来设置可视模式下的高亮显示方式。
'virtualedit' 选项用来设置是否允许光标停留在没有字符的位置。

通常高亮显示的文本包括当前光标下的字符。
然而，如果 'selection' 选项被设置为 "exclusive"，并且当前光标位于可视区域之后，那么光标下的字符将不被包括。

使用 "v" 命令，被高亮的文本将不包括起始点之前和结束点之后的字符。然而，所有的大写命令和非字母命令（"~" 和 "U" 除外）仍将作用在整行上。请参考下面的操作命令列表。

visual-block
使用 CTRL-V（列块可视模式），高亮区域是从开始点到光标之间的一个矩形区域。然而，有些操作符仍将作用在整行上（参见下面的列表）。修改（c）和替代（s）操作将删除高亮区域的文本，然后从左上角开始插入。

---

2. 进入和退出可视模式 visual-start

v characterwise-visual

[count]v 进入可视模式，以字符为单位选择。
若带 [count]，选择的字符数或行数和上次的可视操作使用的一样，但乘以 [count]，且从当前光标位置开始。
如果上次可视模式是基于块的，块宽度与高度同时乘以 [count]。
如果没有上次可视操作，选择 [count] 个字符，就像光标向右移动 N * [count] (译者注: [count]?) 字符那样。如果 'selection' 不是 "exclusive"，少算一个字符。

V linewise-visual

V 进入可视模式，以行为单位选择。
若带 [count]，从当前光标位置开始，选择上次可视操作使用的相同行数乘以 [count]。如果没有前次可视操作，选择 [count] 行。

CTRL-V blockwise-visual

[count]CTRL-V 进入列块可视模式。备注: 在 Windows 环境中，CTRL-V 可能被映射为粘贴操作，所以可能无法用来进入列块可视模式。参见 CTRL-V-alternative 。
[count] 用法同上述 v 。

在可视模式下，按 <Esc>、点击鼠标左键或键入任意一个跳转到另外一个缓冲区的命令，都会取消当前选择区的高亮显示，不影响任何文本。同时，在字符可视模式按 "v"、列块可视模式按 "CTRL-V" 以及行可视模式按 "V" 将起到同样效果。按 CTRL-Z 键将终止可视高亮显示，然后挂起当前编辑器或者启动新的外壳。 CTRL-Z

当键入 "v"、"CTRL-V" 和 "V" 时的模式转换: v_v v_CTRL-V v_V

原有模式	"v"	"CTRL-V"	"V"
普通	可视	列块可视	行可视
可视	普通	列块可视	行可视
列块可视	可视	普通	行可视
行可视	可视	列块可视	普通

gv v_gv reselect-Visual

gv 启动可视模式，重新选择上一次的选择区和相同模式。
在可视模式下，该命令将交换当前和上一次选择区。
在可视模式下使用 "p" 或 "P" 后，本命令选择被粘贴的文本。

gn v_gn

gn 使用前次搜索模式正向搜索，就像 n 一样，并启动可视模式来选中匹配。
如果光标在匹配上，可视地选择之。
如果有操作符等待，在匹配上进行操作。例如，"dgn" 删除下一个匹配的文本。
如果可视模式已激活，选择范围扩展到下一个匹配处的尾部。
应用 'wrapscan'
注意: 和 n 不同，搜索方向不取决于前次搜索命令。

gN v_gN

gN　　类似于 gn 但反向搜索，就像 N 那样。

<LeftMouse>　　设置当前光标位置。如果处在可视模式，则退出可视模式。此功能只有在 'mouse' 选项包含 'n' 或 'a' 才可用。如果点击位置与屏幕最后一行的距离在 'so' 选项设置的范围之内，屏幕文本将会上滚。同样，如果点击位置与屏幕第一行的距离在 'so' 选项设置的范围之内，屏幕文本将会下滚。

<RightMouse>　　如果当前不是可视模式，则进入可视模式。光标位置和鼠标点击位置之间的文本将被高亮显示。如果当前已经处于可视模式，文本高亮选择区的起点或终点之一将被置为鼠标点击的位置。起点或终点的选择取决于谁离点击位置更近。此功能只有在 'mouse' 选项包含 'n' 或 'a' 时才可用。

备注：当 'mousemodel' 被设为 "popup"，本操作将引发 <S-LeftMouse> 动作，而不是 <RightMouse>。

<LeftRelease>　　如果 <LeftRelease> 的位置与 <LeftMouse> 不同，那么它的

作用等同于 <LeftMouse>。在旧版本的 xterm 中，鼠标释放之前是看不到所选择的区域的，除非可以访问 xterm 运行的显示设置（通过 DISPLAY 环境变量或 -display 参数）。此功能只有在 'mouse' 选项包含 'n' 或 'a' 时才可用。

如果可视模式没有激活，同时 "v"、"V" 或 CTRL-V 命令之前包含一个计数（count），那么上一次选择区的大小将被作为本次选择操作的参考值。然后你可以移动选择区的结束点并对选择区进行操作。选择区类型与上次相同（字符选择，行选择，或列块选择）。
- 行选择模式：行的数目与计数相乘。
- 列块模式：行和列的数目分别与计数相乘。
- 普通可视模式，选择区在一行以内：选择区的字符数与计数相乘。
- 普通可视模式，选择区包含多行：行的数目与计数相乘，最后一行的字符数目与上一次选择区相同。

文本的起始点在光标位置。如果上一次的命令中包含了 "$" 命令来扩展高亮文本，那么选择区将扩展到最长行的最后一列。

如果你想选取和上一次完全一样的选择区，请使用 "gv" gv v_gv 命令。

<Esc>　　可视模式下：停止可视模式。

CTRL-C　　可视模式下：停止可视模式。如果插入模式等待中（模式信息显示为 "--（插入）可视 --"），插入模式同时被停止。

---

3. 改变可视区域　　visual-change

o　　跳到高亮文本的另一端（Other）：当前光标位置变成高亮文本的起点，光标移动到高亮文本的另一端。高亮区域保持不变。

v_O

O　　跳到高亮文本的另一端。同 "o" 命令相似，但是在列块模式下，光标移动到水平方向的另一个角。如果这个角上的字符占据了多个屏幕位置（例如一个 <Tab>），那么高亮的文本区域会被改变。

v_$

如果 "$" 命令和列块模式同时使用，那么高亮文本区域的右边界将取决于高亮区域中最长的行。如果遇到一个不是直上直下的移动命令，那么这个规则将终止。

有很多命令可以移动块的结束点。但是不能使用 Ex 命令，以及其它可能改变或放弃文件的命令。下面这些命令（或者以它们开始的命令）：".", "&", CTRL-^, "Z", CTRL-], CTRL-T, CTRL-R, CTRL-I 和 CTRL-O 将会引起铃声，但可视模式保持不变。

当切换到同一缓冲区的另外一个窗口时，目标窗口中的光标位置会被调整，以使相同的可视区域被选中。这一点特别适合在一个窗口查看可视区域的起点，而在另一个窗口查看它的终点。这时候可以用鼠标右键 <RightMouse>（或者，当 'mousemodel' 选项置为 "popup" 时，用 <S-LeftMouse>）来拖拽可视区域两端之一。

---

4. 操作可视区域　　visual-operators

对可视区域可以使用的操作符包括：

~	切换大小写	v_~
d	删除	v_d
c	修改 (4)	v_c
y	抽出	v_y
>	右移 (4)	v_>
<	左移 (4)	v_<
!	通过外部命令过滤 (1)	v_!
=	通过 'equalprg' 选项的命令过滤 (1)	v_=
gq	按照 'textwidth' 指定的宽度排版行 (1)	v_gq

可用的对象包括：

aw	一个单词（包括空格）	v_aw
iw	内含单词	v_iw
aW	一个字串（包括空格）	v_aW
iW	内含字串	v_iW
as	一个句子（包括空格）	v_as
is	内含句子	v_is
ap	一个段落（包括空格）	v_ap
ip	内含段落	v_ip
ab	一个 () 块（包括小括号）	v_ab
ib	内含 () 块	v_ib
aB	一个 {} 块（包括大括号）	v_aB
iB	内含 {} 块	v_iB
at	一个 <tag> </tag> 块（包含 tag）	v_at
it	内含 <tag> </tag> 块	v_it
a<	一个 <> 块（包括 <>）	v_a<
i<	内含 <> 块	v_i<
a[	一个 [] 块（包括 []）	v_a[
i[	内含 [] 块	v_i[
a"	一个双引号字符串（包括引号）	v_aquote
i"	内含双引号字符串	v_iquote

a’	一个单引号字符串（包括引号）	v_a’
i’	内含单引号字符串	v_i’
a‘	一个反引号字符串（包括反引号）	v_a‘
i‘	内含反引号字符串	v_i‘

其它可以使用的命令：

:	为高亮的行启动 Ex 命令	v_:
r	修改 (4)	v_r
s	修改	v_s
C	修改 (2) (4)	v_C
S	修改 (2)	v_S
R	修改 (2)	v_R
x	删除	v_x
D	删除 (3)	v_D
X	删除 (2)	v_X
Y	抽出 (2)	v_Y
p	放置	v_p
J	连接 (1)	v_J
U	变成大写	v_U
u	变成小写	v_u
^]	查找标签	v_CTRL-]
I	列块插入	v_b_I
A	列块添加	v_b_A

(1)：总是应用到整行，见 :visual_example 。
(2)：如果不是使用 CTRL-V，应用到整行。
(3)：如果不是使用 CTRL-V，应用到整行。如果使用 CTRL-V，删除到行尾。
(4)：如果使用 CTRL-V，只对列块操作。

注意 ":vmap" 可以进行可视模式下的键盘映射。例如，如果你不想让 "/" 命令扩展选择区，而是将高亮的文本作为关键字进行搜索，可以这样映射：

```
:vmap / y/<C-R>"<CR>
```

(<> 记法要求逐字键入它的内容 <> ，同时必须在 'cpoptions' 选项中移去 'B' 和 '<' 标志位)

如果你想用 """ 命令指定一个寄存器，只须将寄存器命令加在操作符之前：
"v{move-around}"xd"

如果你想给命令加上计数，只须将数字加在操作符之前："v{move-around}3>"（向右缩进 3 次）。

{move-around}
{move-around} 是任何移动命令的组合。注意 它与 {motion} 的区别，后者只代表 _一个 _移动命令。

对可视区域的另一种操作方法是用模式中的 /\%V 项。例如，要在可视区域中把 '(' 全部替代为 '#'：

```
:'<,'>s/\%V(/#/g
```

注意 在可视模式中按 ":" 时 "'<,'>" 部分会自动出现。

---

5. 列块操作 blockwise-operators

提醒：尝试打开 'virtualedit' 选项，以便列块选择的起点和终点可以超越一行的结尾或位于制表符的中间。

列块插入 v_b_I
对一个列块可视模式下的选择区，I{string}<ESC> 命令将会从选择区开始的位置在每一行插入字符串 {string}。但是只有那些和选择区有交集的行才会被插入，因此过短的行将不受影响。如果选择区包含制表符，制表符将会被分开，以维持选择区的可视列不变。只适用于给一行增加文本，而不是删除。见 v_b_I_example 。

列块添加 v_b_A
对一个列块可视模式下的选择区，A{string}<ESC> 命令将会从选择区结束处开始在每一行插入字符串 {string}。当行的长短不同的时候，将导致选择区的右边界不是直线，这时候列块添加操作的行为就会有所不同：
1. 如果列块是用 <C-v>$ 命令产生的：
   这种情况下字符串被附加到每一行的结尾。
2. 如果列块是用 <C-v>{move-around} 命令产生的：
   这种情况下字符串将被附加到每一行列块的结尾。对于比较短的行，行尾和列块结尾列之间将以空白填充。

见 v_b_A_example 。
备注：对于那些由于过短而没有扩展到选择区的行，"I" 和 "A" 的操作结果是不同的。这样的设计是有意的，以便你能够选择自己所希望的操作。
只适用于给一行增加文本，而不是删除。见 v_b_I_example 。

列块修改 (c) v_b_c
选择区的所有文本将被相同的字符串代替。当使用 "c" 命令的时候选择区的文本将被删除，然后进入插入模式。这时候你可以键入不带回车的文本。当你按 <Esc> 的时候所键入的文本将被插入选择区的每一行。

列块修改 (C) v_b_C
同 "c" 命令，但是选择区将扩展到每一行的结尾。

v_b_<
列块平移 v_b_>
列块按照 'shiftwidth' 指定的单位移动。与块的右边界无关。块的左边界决定了从哪一点开始右移，填充的空白根据 'ts' 和 'et' 设置的宽度最优化制表符的使用方式。而块的左边界决定了左移的终止点。
见 v_b_>_example 。
见 v_b_<_example 。

列块替换 v_b_r
高亮区域的每个字符被同一个字符代替。制表符将被分割，虚拟空白字符也将被替换，以保持屏幕布局。
见 v_b_r_example 。

---

6. 重复 visual-repeat

当重复一个可视模式操作时，该操作将被应用到与上次操作相同数量的文本上：
- 行可视模式：相同数量的行。
- 列块可视模式：相同数量的行和列。
- 选择区在一行之内的普通可视模式：相同数量的字符。

- 跨越多行的普通可视模式：相同数量的行，最后一行的字符数与上次相同。
重复操作的起始点在当前光标位置。如果重复的命令中包含 "$" 命令来扩展选择区，那么重复操作将应用到最长行的最右一列。

---

7. 示例 visual-examples

:visual_example
目前 ":" 命令只能作用在一整行。当你选择某一行的一部分，执行一个类似 ":!date" 的命令，命令输出将替换当前行的全部。如果你只想让它替换行的一部分，则必须做一个映射。在将来的版本中，":" 可能可以作用于行的一部分。

下面这个示例能够用 "date" 命令的输出替换所选择的文本：

```
:vmap _a <Esc>‘>a<CR><Esc>‘<i<CR><Esc>!!date<CR>kJJ
```

(<> 记法要求逐字键入它的内容 <> ，同时必须在 'cpoptions' 选项中移去 'B' 和 '<' 标志位)

它所做的是：

<Esc>	停止可视模式
‘>	移动到可视区域的结尾
a<CR><Esc>	在可视区域的结尾断行
‘<	跳到可视区域的开始
i<CR><Esc>	在可视区域的起点断行
!!date<CR>	通过 date 命令过滤可视文本
kJJ	把行重新连接起来

visual-search
下面这个键盘映射能够将当前的选择区作为关键字进行搜索：

```
:vmap X y/<C-R>"<CR>
```

(<> 记法要求逐字键入它的内容 <> ，同时必须在 'cpoptions' 选项中移去 'B' 和 '<' 标志位)

注意 某些特殊字符 (例如 '.' 和 '*') 可能会引起问题。

列块可视模式示例 blockwise-examples
在下面的示例中，我将说明列块操作命令以及命令所执行的结果。在所有的示例中，光标起始点位于测试文本第一行的 'a' 字符上。
假定使用模式行的设置为：":ts=8:sw=4:"。

另外，进行这样的设置将有助于显示操作细节：

```
:set hls
/<TAB>
```

这里的 <TAB> 是一个真正的 TAB (即按下 Tab 键)。

测试文本是：

```
abcdefghijklmnopqrstuvwxyz
abc             defghijklmnopqrstuvwxyz
abcdef  ghi             jklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyz
```

1. fo<C-v>3jISTRING<ESC> v_b_I_example

```
abcdefghijklmnSTRINGopqrstuvwxyz
abc           STRING  defghijklmnopqrstuvwxyz
abcdef  ghi   STRING    jklmnopqrstuvwxyz
abcdefghijklmnSTRINGopqrstuvwxyz
```

2. fo<C-v>3j$ASTRING<ESC> v_b_A_example

```
abcdefghijklmnopqrstuvwxyzSTRING
abc             defghijklmnopqrstuvwxyzSTRING
abcdef  ghi             jklmnopqrstuvwxyzSTRING
abcdefghijklmnopqrstuvwxyzSTRING
```

3. fo<C-v>3j3l<.. v_b_<_example

```
abcdefghijklmnopqrstuvwxyz
abc           defghijklmnopqrstuvwxyz
abcdef  ghi   jklmnopqrstuvwxyz
abcdefghijklmnopqrstuvwxyz
```

4. fo<C-v>3j>.. v_b_>_example

```
abcdefghijklmn            opqrstuvwxyz
abc                         defghijklmnopqrstuvwxyz
abcdef  ghi                         jklmnopqrstuvwxyz
abcdefghijklmn            opqrstuvwxyz
```

5. fo<C-v>5l3jrX v_b_r_example

```
abcdefghijklmnXXXXXXuvwxyz
abc           XXXXXXhijklmnopqrstuvwxyz
abcdef  ghi   XXXXXX    jklmnopqrstuvwxyz
abcdefghijklmnXXXXXXuvwxyz
```

---

8. 选择模式 Select Select-mode

选择模式看起来与可视模式非常相似，但是在选择模式所接受的命令与可视模式有很大不同。它类似于 Microsoft Windows 上的选择模式。如果 'showmode' 选项打开，处于选择模式的时候在屏幕底端会显示 "-- 选择 --"。

进入选择模式的几种方法：
- 使用鼠标选择一片区域，同时 'selectmode' 选项包含 "mouse"。'mouse' 必须包含对应当前模式的标志位。
- 在按下 Shift 键的同时，键入不可显示的光标移动命令，同时 'selectmode' 选项包含 "key"。例如：<S-Left> 和 <S-End>。'keymodel' 选项也必须包含 "startsel"。
- 键入 "v"、"V" 或 CTRL-V 命令，同时 'selectmode' 选项包含 "cmd"。
- 在普通模式键入 "gh"、"gH" 或 "g_CTRL-H" 命令。
- 在可视模式，键入 CTRL-G。 v_CTRL-G

选择模式的命令：
- 所有可显示字符、<NL> 和 <CR> 将删除当前的选择区，然后进入插入模式。插入键入的字符。

- 键入不可显示移动命令的同时，按下 Shift 键，将扩展当前的选择区。
  'keymodel' 选项必须包含 "startsel"。
- 键入不可显示移动命令的同时，没有按下 Shift 键，将退出选择模式。
  'keymodel' 选项必须包含 "stopsel"。
- 按 ESC 键退出选择模式。
- CTRL-O 临时切换到可视模式执行一个命令。 v_CTRL-O
- CTRL-G 切换到可视模式。

除此之外，所有键入字符的处理方式与可视模式相同。

如果选择模式是按行选择，那么在选择模式使用操作符时，所选中的行将被作为操作对象，但是作用的方式与按字符选择相同。举例来说：如果删除了一整行，之后它可以被粘贴到另一行的中间。

选择模式中的键盘映射和菜单 Select-mode-mapping

:vmap 或 :vmenu 命令定义的映射和菜单可以同时用于可视模式和选择模式。如果用于选择模式，Vim 自动切换到可视模式，以便执行与可视模式相同的操作。如果你不希望这样，可以使用 :xmap 或者 :smap 。

用户一般期待可显示字符会替代选择区域。所以，要避免选择模式下映射可显示字符。也可以在 :map 和 :vmap 之后用 :sunmap 来撤销选择模式下的映射。

当键盘映射或菜单命令执行完毕，Vim 回到选择模式，选择区被重新激活。除非选择区域被删除，切换到另外一个缓冲区，或者窗口布局被改变。

当键入一个字符，将引起当前选择区被删除从而进入插入模式。插入模式的键盘映射将应用到这个刚刚键入的字符。这可能会引起一些混淆，因为它意味着在选择模式输入的字符使用了插入模式的映射。语言的映射与此相同。

gV gV v_gV
当一个选择模式的键盘映射或菜单命令被执行完毕后，不再自动选择可视区域。把 gV 放在键盘映射或菜单命令的结尾即可。或者至少应该在任何对选择区操作的命令之后。

gh gh
启动字符方式的选择模式。同 "v" 命令类似，但是启动的是选择模式，而不是可视模式。
助记："get highlighted"。

gH gH
启动按行选择的选择模式。同 "V" 命令类似，但是启动的是选择模式，而不是可视模式。
助记："get Highlighted"。

g CTRL-H g_CTRL-H
启动列块方式的选择模式。同 CTRL-V 命令类似，但是启动的是选择模式，而不是可视模式。
助记："get Highlighted"。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

各种其他命令 various

1. 其他命令 various-cmds
2. 用 Vim 代替 less 或 more less

---

1. 其他命令 various-cmds

CTRL-L
清屏并重画屏幕。重画可能稍后发生，在处理完预读入之后。

:redr :redraw
:redr[aw][!]
立即重画屏幕。如果有 !，先清屏。
这对在执行脚本或函数的中间要刷新屏幕，或者在置位了 'lazyredraw' 而执行映射的过程中间都有用。

:redraws :redrawstatus
:redraws[tatus][!]
重画当前窗口的状态行。如果有 !，重画所有的状态行。它适用于 'statusline' 包含有不能自动更新项目的时候。

:redrawt :redrawtabline
:redrawt[abline]
重画标签页行。用于 'tabline' 包含不能触发自动更新的项目的时候来更新标签页行。

N<Del>
<Del>
输入数字的时候：删除上一位数字。 注意：如果你想要 <BS> 做同样的事，在 .vimrc 里加入如下的映射：
:map CTRL-V <BS> CTRL-V <Del>
如果你的 <Del> 键工作不正常，可参见 :fixdel 。

ga :as :ascii
:as[cii] 或
ga
显示光标下字符的 ascii 值，分别以十进制、十六进制和八进制显示。
助记：得到 (Get) Ascii 值。

例如，如果光标下是 'R'：
<R> 82, Hex 52, Octal 122
如果该字符不是标准的 ASCII 字符，但据 'isprint' 选项可以显示，则另给出不可显示的版本。

如果该字符大于 127，还显示 <M-X> 形式的结果。例如：
<~A> <M-^A> 129, Hex 81, Octal 201
<p> <|~> <M-~> 254, Hex fe, Octal 376
(其中的 <p> 是某个特殊字符)
文件里的 <Nul> 字符内部以 <NL> 存储，但显示为：
<^@> 0, Hex 00, Octal 000

如果字符有组合字符，同时显示它们。'maxcombine' 的值这时不起作用。

如果该字符可用二合字母插入，也输出可用于创建该字符的两个字符：

```
    <> 246, Hex 00f6, Oct 366, Digr o:
```

显示可以按 CTRL-K o : 来插入 。

g8

显示光标下字符（假定其为 UTF-8 编码）的各字节的十六进制值。它会同时显示组合字符。'maxcombine' 的值这时不起作用。

一个带三个组合字符的字符示例：

```
        e0 b8 81 + e0 b8 b9 + e0 b9 89
```

8g8

寻找光标所在或之后的非法 UTF-8 字节序列。两种情形下这可以工作：

1. 如果 'encoding' 是任何 8 位编码
2. 如果 'encoding' 是 "utf-8" 而 'fileencoding' 是任何 8 位编码

所以它可以用来编辑本来应该是 UTF-8 但因为包含了非法字节而读入时被当作 8 位编码的文件。

文件尾不回绕。

注意 如果光标在非法字节上或者光标在多字节字符的中间，此命令不移动光标。

:[range]p[rint][flags]

显示 [range] 范围里的行（缺省是当前行）。

注意：如果你要打印你的文本到纸张上，见 :hardcopy 。GUI 上可以使用 File.Print 菜单项。

ex-flags 说明 [flags]。

:filter 命令可用来只显示匹配模式的行。

:[range]p[rint] {count} [flags]

显示 {count} 行，从 [range] 指定的范围开始（缺省是当前行 cmdline-ranges ）。

ex-flags 说明 [flags]。

:[range]P[rint] [count] [flags]

和 ":print" 相同。Vi 里加入这个功能，很明显地，是给那些按 Shift 键太久的人用的 ...

备注：用户命令可以覆盖此命令。

ex-flags 说明 [flags]。

:[range]l[ist] [count] [flags]

和 :print 相同，但是不可显示字符用 '^' 代替，行尾加上 $。这可进一步用 'listchars' 选项改变。

ex-flags 说明 [flags]。

:nu :number

:[range]nu[mber] [count] [flags]
和 :print 相同，但每行之前显示行号（也参见 'highlight' 和 'numberwidth' 选项）。
ex-flags 说明 [flags]。

:#

:[range]# [count] [flags]
和 :number 相同。

:#!

:#!{anything}
忽略，从而你可以用下面的方式开始 Vim 脚本：

```
#!vim -S
echo "this is a Vim script"
quit
```

:z E144

:{range}z[+-^.=]{count} 显示包围 {range} 指定行的若干行文本，如果没有 {range}，显示当前行上下的文本。如果有 {count}，它指定你能看到的行数。否则，如果只有一个窗口，使用 'scroll' 选项值的两倍。再不然，使用当前的窗口大小减 3。

如果提供了 {count}，'window' 选项会被设为该值。

:z 可以单独使用，也可以后跟数个标点符号中的一个。他们有如下的效果：

标记	首行	末行	新光标行
+	当前行	向前 1 屏	向前 1 屏
-	向后 1 屏	当前行	当前行
^	向后 2 屏	向后 1 屏	向后 1 屏
.	向后 1/2 屏	向前 1/2 屏	向前 1/2 屏
=	向后 1/2 屏	向前 1/2 屏	当前行

不指定任何标记相当于指定 "+"。如果标记是 "="，当前行会被连字符组成的两行包围。

:{range}z#[+-^.=]{count} :z#
和 ":z" 类似，但显示行号。

:=

:= [flags]
显示末行的行号。
ex-flags 说明 [flags]。

:{range}= [flags]
显示 {range} 范围内最后一行的行号。例如，打印当前行号：

```
:.=
```

ex-flags 说明 [flags]。

:norm[al][!] {commands} :norm :normal
执行若干普通模式命令 {commands}。这使得在命令行上执行普通模式命令成为可能。{commands} 的执行就像从键盘输入那样。撤销命令会撤销所有的命令。

如果发生错误，结束执行。

如果给出 [!]，不使用映射。
如果没有，如果一个不可重新映射的映射 (:noremap) 调用该命令，它的参数就因而还是有可能被重新映射。

{commands} 必须是完整的命令。如果 {commands} 没有完整给出一个命令，未完成的最后那个命令会被终止，如同键入了 <Esc> 或 <C-C> 一样。
这意味着插入命令也必须完整 (要开始插入模式，见 :startinsert)。":" 命令也必须完整。而且你不能用 "Q" 或 "gQ" 启动 Ex 模式。

":normal" 还在工作的时候，显示不会被刷新。

{commands} 不能以空格开始，如果要，前面加个计数 1 (一)。"1 " 就是一个空格。

{commands} 里忽略 'insertmode' 选项。

该命令后面不能跟其他命令。因为任何 '|' 都被认为是命令的一部分。

该命令可以递归使用，但是深度不能超过 'maxmapdepth'。

一个替代的方法是 :execute ，它可以用表达式作为参数。这样就可使用可显示字符来代表特殊字符。

例如:

```
:exe "normal \<c-w>\<c-w>"
```

:{range}norm[al][!] {commands} :normal-range

在 {range} 指定的范围中的每行执行若干普通模式命令 {commands}。在执行 {commands} 之前，光标先定位在该行的第一列。除此以外，和没有给出范围的 ":normal" 命令并无差别。

:sh[ell] :sh :shell E371

该命令启动一个外壳。外壳退出后 (执行完 "exit" 命令)，你会回到 Vim。外壳命令的名字来自 'shell' 选项。

E360

注意: 当 Vim 在 Amiga 上通过编译器的 QuickFix 模式时启动时，该命令不能使用。这是因为编译器把标准输入设为非交互模式。

:!{cmd} :!cmd :! E34

在外壳里执行 {cmd} 命令。也可参见 'shell' 和 'shelltype' 选项。

{cmd} 里的任何 '!' 都被前一个外部命令所取代 (参见 'cpoptions')，除非之前有反斜杠。这时反斜杠被去掉。例如: "!ls" 之后的 "!echo ! \! \\!" 实际执行 "echo ls ! \!"。

命令执行之后，检查当前文件的修改时间 timestamp 。
{cmd} 里的 '|' 会传给外壳，你不能用来附加 Vim 命令。
见 :bar 。

如果 {cmd} 里包含 "%"，被扩展为当前文件名。不转义特殊字符，用引号以避免特殊含义：

```
:!ls "%"
```

如果文件名包含 "$"，用单引号较好（但一个单引号会有麻烦）：

```
:!ls '%'
```

总是可以这样做，但要多打点字：

```
:exe "!ls " . shellescape(expand("%"))
```

{cmd} 以换行符结尾，其后的内容被理解为另外一个 ":" 命令。不过，如果换行之前有反斜杠，该反斜杠被去掉，{cmd} 继续接受参数。无论之前有多少个反斜杠都是如此。只有一个会被去掉。

Unix 上命令通常在非交互的外壳里运行。如果你希望用交互外壳（比如要用别名机制 aliases），设置 'shellcmdflags' 为 "-ic"。Win32 上可参见 :!start 。

命令执行后，检查当前文件的时间戳和大小 timestamp 。

Vim 在命令结束以后重画屏幕，因为命令可能显示任何文本。因此，你会有一个按回车的提示，以便你看到执行结果。要避免这一点：

```
:silent !{cmd}
```

这样屏幕就不会重画。如果确实有输出，你需要用 CTRL-L 或 ":redraw!" 来刷新。
参见 shell-window 。

:!!

:!! 重复上次的 ":!{cmd}"。

:ve :ver :version

:ve[rsion] 显示编辑器的版本号。如果编译器能解释 "__DATE__"，也显示编译的日期。不然，显示固定的发布日期。
然后，显示 Vim 编译时所带特性的信息。如果某特性之前有 '+'，该特性被包含。如果之前有 '-'，该特性没有包含。要改变编译时带的特性，修改 feature.h 并重新编译 Vim。
要在表达式里确定是否包含某特性，参见 has() 。
以下是特性的总览。
第一列显示包含该特性的最小版本。

T	tiny（微小）（总是存在）
S	small（小）
N	normal（普通）
B	big（大）
H	huge（巨大）
m	手动设定或者依赖于其他特性
-	永不会，此特性被删除
（无）	依赖于系统

这样，如果一个特性标记为 "N"，它包含在 Vim 的 normal、big 和 huge 版本里。

+feature-list

	+acl	包含 ACL 支持
	+ARP	仅对 Amiga 有效：支持 ARP
B	+arabic	支持阿拉伯语 Arabic
T	+autocmd	:autocmd ，自动命令
H	+autoservername	自动打开 clientserver
m	+balloon_eval	GUI 上气泡表达式支持 balloon-eval 。如果编译时包含支持的 GUI (Motif、GTK、GUI) 同时还有 Netbeans/Sun Workshop 集成或者 +eval 特性之一就包含此特性。
H	+balloon_eval_term	终端上气泡表达式支持 balloon-eval ，'balloonevalterm'
N	+browse	:browse 命令
N	+builtin_terms	一些内建终端项 builtin-terms
B	++builtin_terms	最大内建终端项支持 builtin-terms
N	+byte_offset	支持 'statusline' 选项的 'o' 标志位、"go" 和 ":goto" 命令。
m	+channel	进程间通信 channel
N	+cindent	'cindent' ，C 缩进
N	+clientserver	Unix 和 Win32：远程调用 clientserver
	+clipboard	编译时加入了剪贴板支持 clipboard
	+clipboard_working	编译时加入了剪贴板支持 clipboard 且可用
T	+cmdline_compl	命令行补全 cmdline-completion
S	+cmdline_hist	命令行历史 cmdline-history
N	+cmdline_info	'showcmd' 和 'ruler'
T	+comments	支持 'comments'
B	+conceal	"conceal" 支持，见 conceal :syn-conceal 等。
N	+cryptv	加密支持 encryption
B	+cscope	支持 cscope
T	+cursorbind	'cursorbind' 支持
m	+cursorshape	termcap-cursor-shape 支持
m	+debug	带调试的编译。
N	+dialog_gui	支持 :confirm 显示 GUI 对话框。
N	+dialog_con	支持 :confirm 显示控制台对话框。
N	+dialog_con_gui	支持 :confirm 显示 GUI 或控制台对话框。
N	+diff	vimdiff 和 'diff'
N	+digraphs	digraphs E196
	+directx	只对 Win32 GUI 有效：DirectX 和 'renderoptions'
	+dnd	"~ 寄存器的拖放支持 quote_~ 。
B	+emacs_tags	emacs-tags 文件
N	+eval	表达式计算 eval.txt
N	+ex_extra	现在总是打开，用于 Vim 附加的 Ex 命令
N	+extra_search	'hlsearch' 和 'incsearch' 选项。
-	+farsi	已删除： farsi 波斯语言
N	+file_in_path	gf 、 CTRL-W_f 和 <cfile>
N	+find_in_path	包含文件搜索： [I 、 :isearch 、 CTRL-W_CTRL-I 、 :checkpath ，等等。
N	+folding	折叠 folding
	+footer	信息页脚 gui-footer
	+fork	只对 Unix 有效： fork 外壳命令
	+float	浮点数支持
N	+gettext	消息翻译 multi-lang
	+GUI_Athena	只对 Unix 有效：Athena GUI
	+GUI_neXtaw	只对 Unix 有效：neXtaw GUI
	+GUI_GTK	只对 Unix 有效：GTK+ GUI

	+GUI_Motif	只对 Unix 有效: Motif GUI
	+GUI_Photon	只对 QNX 有效: Photon GUI
m	+hangul_input	韩语 (Hangul) 输入支持 hangul
	+iconv	编译时有 iconv() 函数
	+iconv/dyn	类似 iconv-dynamic /dyn
T	+insert_expand	insert_expand 插入模式补全
m	+job	启动和终止作业 job
S	+jumplist	跳转表 jumplist
B	+keymap	键盘映射表 'keymap'
N	+lambda	lambda 和 closure
B	+langmap	语言映射表 'langmap'
N	+libcall	函数库调用 libcall()
N	+linebreak	'linebreak' 、 'breakat' 和 'showbreak'
N	+lispindent	'lisp'
T	+listcmds	Vim 的缓冲区列表相关命令 buffer-hidden 和参数列表 :argdelete
T	+localmap	支持缓冲区的局部映射 :map-local
m	+lua	Lua 接口
m	+lua/dyn	Lua 接口 /dyn
N	+menu	菜单 :menu
N	+mksession	创建会话 :mksession
T	+modify_fname	文件名修饰符 filename-modifiers
T	+mouse	鼠标处理 mouse-using
N	+mouseshape	鼠标形状 'mouseshape'
B	+mouse_dec	只对 Unix 有效: Dec 终端鼠标处理 dec-mouse
N	+mouse_gpm	只对 Unix 有效: Linux 终端鼠标处理 gpm-mouse
N	+mouse_jsbterm	JSB 鼠标处理 jsbterm-mouse
B	+mouse_netterm	只对 Unix 有效: netterm 终端鼠标处理 netterm-mouse
N	+mouse_pterm	只对 QNX 有效: pterm 终端鼠标处理 qnx-terminal
N	+mouse_sysmouse	只对 Unix 有效: *BSD 控制台鼠标处理 sysmouse
B	+mouse_sgr	只对 Unix 有效: sgr 鼠标处理 sgr-mouse
B	+mouse_urxvt	只对 Unix 有效: urxvt 鼠标处理 urxvt-mouse
N	+mouse_xterm	只对 Unix 有效: xterm 终端鼠标处理 xterm-mouse
N	+multi_byte	16 和 32 位字符 multibyte
	+multi_byte_ime	Win32 多字节字符的输入法 multibyte-ime
N	+multi_lang	非英语语言支持 multi-lang
m	+mzscheme	Mzscheme 接口 mzscheme
m	+mzscheme/dyn	Mzscheme 接口 mzscheme-dynamic /dyn
m	+netbeans_intg	netbeans
	+num64	64-位数值支持 Number
m	+ole	只对 Win32 GUI 有效: ole-interface
N	+packages	载入 packages
N	+path_extra	'path' 和 'tags' 的向上和向下搜索
m	+perl	Perl 接口 perl
m	+perl/dyn	Perl 接口 perl-dynamic /dyn
N	+persistent_undo	撤销的持久性 undo-persistence
N	+popupwin	弹出窗口 popup-window
	+postscript	:hardcopy 可以写 PostScript 文件
N	+printer	:hardcopy 命令
H	+profile	:profile 命令
m	+python	Python 2 接口 python
m	+python/dyn	Python 2 接口 python-dynamic /dyn
m	+python3	Python 3 接口 python
m	+python3/dyn	Python 3 接口 python-dynamic /dyn

N	+quickfix	:make 和 quickfix 命令
N	+reltime	reltime() 函数，'hlsearch'/'incsearch' 超时，'redrawtime' 选项
B	+rightleft	从右到左输入 'rightleft'
m	+ruby	Ruby 接口 ruby
m	+ruby/dyn	Ruby 接口 ruby-dynamic /dyn
T	+scrollbind	滚动绑定 'scrollbind'
B	+signs	标号 :sign
N	+smartindent	'smartindent'
B	+sound	sound_playevent() 、 sound_playfile() 函数等等
N	+spell	拼写检查支持，见 spell
N	+startuptime	--startuptime 参数
N	+statusline	'statusline'、'rulerformat' 选项和 'titlestring' 和 'iconstring' 的特殊格式
-	+sun_workshop	已删除： workshop
N	+syntax	语法高亮 syntax
	+system()	只对 Unix 有效： +fork 的相反特性
T	+tag_binary	标签文件的二分查找 tag-binary-search
-	+tag_old_static	已删除：支持静态标签的旧方法 tag-old-static
-	+tag_any_white	已删除：标签文件里允许空白 tag-any-white
m	+tcl	Tcl 接口 tcl
m	+tcl/dyn	Tcl 接口 tcl-dynamic /dyn
m	+terminal	终端窗口支持 terminal
	+terminfo	用 terminfo 代替 termcap
N	+termresponse	支持 t_RV 和 v:termresponse
B	+termguicolors	xterm 兼容的终端 24 位色彩支持
N	+textobjects	文本对象 text-objects 选择
N	+textprop	text-properties
	+tgetent	只对非 Unix 有效：可以使用外部 termcap
N	+timers	timer_start() 函数
N	+title	设置窗口 'title' 和 'icon'
N	+toolbar	GUI 工具栏 gui-toolbar
T	+user_commands	用户定义命令。 user-commands 8.1.1210 开始总是打开。
B	+vartabs	可变宽度制表位。 'vartabstop'
N	+viminfo	'viminfo'
	+vertsplit	垂直分割窗口 :vsplit ； 8.0.1118 开始总是打开。
N	+virtualedit	'virtualedit'
T	+visual	可视模式 Visual-mode 7.4.200 开始总是打开。
T	+visualextra	额外的可视模式命令 blockwise-operators
T	+vreplace	gR 和 gr
	+vtp	MS-Windows 控制台上：'termguicolors' 的支持
N	+wildignore	忽略文件通配 'wildignore'
N	+wildmenu	匹配菜单 'wildmenu'
T	+windows	多于一个窗口；8.0.1118 开始总是打开。
m	+writebackup	'writebackup' 缺省打开
m	+xim	X 输入法 xim
	+xfontset	X 字体集支持 xfontset
	+xpm	pixmap 支持
m	+xpm_w32	只对 Win32 GUI 有效：pixmap 支持 w32-xpm-support
	+xsmp	XSMP (X 会话管理) 支持
	+xsmp_interact	交互 XSMP (X 会话管理) 支持
N	+xterm_clipboard	只对 Unix 有效：xterm 剪贴板处理
m	+xterm_save	保存和恢复 xterm 屏幕 xterm-screens

N +X11 只对 Unix 有效：可以恢复窗口标题 X11

/dyn E370 E448
有些特性可能会显示 "/dyn"，表明该特性只有在动态调入相关库时才可用。

:ve[rsion] {nr} 现在已被忽略。以前用来检查 .vimrc 文件的版本号。现在取消的原因是因为你可以用 ":if" 命令区别不同版本的不同行为。

:redi :redir
:redi[r][!] > {file} 重定向消息到文件 {file}。将命令的输出消息写到该文件，直到重定向结束。消息也同时显示在屏幕上。如果有 [!]，覆盖已存在的文件。如果没有但文件 {file} 存在，该命令会失败。

一次只能激活一个 ":redir"。也就是，调用 ":redir" 开始重定向到新的目标之前，任何已激活的重定向都会被关闭。要递归调用，请查看 execute() 。

如果不想在屏幕上显示消息或者命令的回显，在函数里调用命令，以 ":silent call Function()" 方式调用该函数。
一个替代方案是使用 'verbosefile' 选项，它也可以和 ":redir" 组合使用。

:redi[r] >> {file} 重定向消息到文件 {file}，如果文件 {file} 已存在，附加在其后。

:redi[r] @{a-zA-Z}
:redi[r] @{a-zA-Z}> 重定向消息到寄存器 {a-z}。如果给出的名字是大写 {A-Z}，附加到已有内容之后。寄存器名后的 '>' 可选。
:redi[r] @{a-z}>> 附加消息到寄存器 {a-z}。

:redi[r] @*>
:redi[r] @+> 重定向消息到选择区或剪贴板。为了后向兼容，寄存器名后的 '>' 可以省略。见 quotestar 和 quoteplus 。
:redi[r] @*>>
:redi[r] @+>> 附加消息到选择区或剪贴板。

:redi[r] @"> 重定向消息到无名寄存器。为了后向兼容，寄存器名后的 '>' 可以省略。
:redi[r] @">> 附加消息到无名寄存器。

:redi[r] => {var} 重定向消息到变量。如果变量不存在，建立之。如果变量已经存在，它被初始化为空字符串，而且保持为空，直到重定向结束为止。只能使用字符串变量。重定向开始后，如果变量被删除或加锁或类型被改变，那么将来的命令输出消息会报错。
要得到一个命令的输出，可用 execute() 函数。

:redi[r] =>> {var} 附加消息到已有的变量。只能使用字符串变量。

:redi[r] END 结束消息的重定向。

:filt :filter

:filt[er][!] {pat} {command}
:filt[er][!] /{pat}/ {command}

找到 {command} 的输出里匹配 {pat} 的行。例如，要只列出 xml 文件：

```
:filter /\.xml$/ oldfiles
```

如果给出 [!]，只限 {command} 的输出里 不 匹配 {pat} 的行。

{pat} 是 Vim 搜索模式。不一定要用 / 包围，可以用任何非 ID 字符（见 'isident'），只要不在 {pat} 里出现。如果不使用包围字符，模式里不能包含竖杠字符。

模式匹配输出的相关部分，不一定是整行。只有部分命令支持过滤，先试试看能不能用。一些支持过滤的命令：

:#	- 过滤整行
:clist	- 按文件名或模块名过滤
:command	- 按命令名过滤
:files	- 按文件名过滤
:highlight	- 按高亮组过滤
:jumps	- 按文件名过滤
:let	- 按变量名过滤
:list	- 过滤整行
:llist	- 按文件名或模块名过滤
:marks	- 按当前文件的文本或其它文件的文件名过滤
:oldfiles	- 按文件名过滤
:registers	- 按寄存器内容过滤（不能用于多行）
:set	- 按变量名过滤

只过滤正常信息，不过滤错误信息。

:sil :silent :silent!

:sil[ent][!] {command} 安静地执行命令 {command}。不显示正常的消息，也不会把它加进消息历史。
如果有 [!]，同时跳过错误消息，检测到错误时，命令和映射也也不会被终止。但还是会相应设置 v:errmsg 。
如果没有 [!]，错误信息会使得其后的所有消息都被正常显示。
:redir 开始的重定向会继续工作，但可能有些细微差异。命令输出仍然重定向，但屏幕上看不到。例如：

```
:redir >/tmp/foobar
:silent g/Aap/p
:redir END
```

要安静地执行普通模式命令，用 :normal 命令。例如，要搜索字符串且不给出任何消息：

```
:silent exe "normal /path\<CR>"
```

":silent!" 可以用来执行命令，不管该命令是否失败都继续执行。
例如：

```
:let v:errmsg = ""
:silent! /^begin
:if v:errmsg != ""
: ... 模式没找到
```

":silent" 还可以避免敲入回车 (hit-enter) 的提示。执行

外部命令时，这可能会使屏幕显示混乱。这时，用 CTRL-L 可以清理。
":silent menu ..." 定义一个不会回显命令行命令的菜单。但该命令仍然产生消息。如果连这也不要，在命令本身前用 ":silent": ":silent menu .... :silent command"。

:uns :unsilent

:uns[ilent] {command} 非安静模式执行命令 {command}。仅当在用了 :silent 之后才有意义。
用此命令可以在用了 :silent 之后还是可以给出消息。下例中， :silent 用于避免读入文件时给出消息，而 :unsilent 用于列出每个文件的第一行。

```
:silent argdo unsilent echo expand('%') . ": " . getline(1)
```

:verb :verbose

:[count]verb[ose] {command}
执行命令 {command}，执行期间 'verbose' 设为 [count]。如果忽略 [count]，缺省为 1。":0verbose" 可以用来设置 'verbose' 为零。

在此基础之上再应用 ":silent"，可以产生相应详细度的消息但不显示。":silent" 和 ":verbose" 的组合因而可以只在内部产生消息，然后用 v:statusmsg 或相关命令进行检查。例如：

```
:let v:statusmsg = ""
:silent verbose runtime foobar.vim
:if v:statusmsg != ""
:  " foobar.vim 找不到
:endif
```

如果和别的命令相连接，":verbose" 只适用于第一个命令：

```
:4verbose set verbose | set verbose
  verbose=4
  verbose=0
```

要记录 verbose 消息到文件，用 'verbosefile' 选项。

:verbose-cmd

如果 'verbose' 非零，列出 Vim 选项、键映射、缩写、用户定义的函数或命令、高亮组或者自动命令的值会同时显示它最近被定义的位置。如果手动定义，不会有这个 "Last set" 消息。如果在执行函数、用户命令或者自动命令时定义，报告它定义所在的脚本。
{仅当编译时加入 +eval 特性才有效}

K

K 运行程序，查找光标下的关键字。该程序的名字由 'keywordprg' (kp) 选项给出 (缺省是 "man")。该关键字由字母、数值和 'iskeyword' 里的字符组成，它出现在光标所在或右侧的位置。以下命令可以达到同样的效果

```
:!{program} {keyword}
```

在 Vim 的 tools 目录下有一个示例程序。它叫作 "ref"，做简单的拼写检查。
特例：
- 如果 'keywordprg' 以 ":" 开始，作为带 [count] 的 Vim Ex 命令进行调用。
- 如果 'keywordprg' 为空，使用 ":help" 命令。此时，如

果要找到更多帮助，应该在 'iskeyword' 里加入更多的字符。
- 如果 'keywordprg' 是 "man" 或以 ":" 开始，K 之前的 [count] 会被插在 keywordprg 之后和关键字之前。例如，如果光标在 "mkdir" 之上用 "2K"，结果是:
```
!man 2 mkdir
```
- 如果 'keywordprg' 是 "man -s"，K 之前的计数会插在 "-s" 之后。如果没有计数，则去掉 "-s"。

{Visual}K — 和 "K" 类似，但用可视的高亮文本代替关键字。只在高亮文本不超过一行时有效。 v_K

[N]gs
:[N]sl[eep] [N] [m] — 停止（睡眠）[N] 秒。如果有 [m]，停止 [N] 毫秒。"gs" 前的计数总是表示秒数。缺省值是 1 秒。 gs :sl :sleep
```
:sleep          "睡眠一秒
:5sleep         "睡眠五秒
:sleep 100m     "睡眠一百毫秒
10gs            "睡眠十秒
```
可以被 CTRL-C (MS-DOS 上的 CTRL-Break) 中断。
"gs" 代表 "goto sleep"。
睡眠时如果在可见位置的话，光标定位于文本处。
也处理接收到的 netbeans 信息。{仅当编译时加入 +netbeans_intg 特性才有效}

:xr[estore] [display] — 重新初始化 X11 服务器的连接。X 服务器重启后有用，例如在 screen/tmx 内运行 Vim 很长时间，并连接到不同机器的时候。 :xrestore :xr
[display] 应为 $DISPLAY 环境变量使用的格式（如 "localhost:10.0"）
如果省略 [display]，使用之前运行此命令时使用的值来重新初始化 X11 服务器的连接。如果该值从未提供过，使用 Vim 启动时 $DISPLAY 环境变量的值。
{仅当编译时加入 +clipboard 特性才有效}

g CTRL-A — 只有在 Vim 编译时定义了 MEM_PROFILING 才有效（这很少见）：显示内存用量的统计。只对调试 Vim 有用。 g_CTRL-A
关于可视模式下的增量，见 v_g_CTRL-A 。

---

2. 用 Vim 代替 less 或 more less

如果你用 less 或 more 程序来查阅文件，你不会有语法高亮。所以，你也许会更喜欢用 Vim。你可以用外壳脚本 "$VIMRUNTIME/macros/less.sh" 来做到这一点。

这个外壳脚本用到 Vim 脚本 "$VIMRUNTIME/macros/less.vim"。它设置了一些映射来模拟 less 支持的命令。除此以外，你可以正常使用 Vim 命令。

这还没有尽善尽美。例如，即使查阅短的文件，Vim 还是会使用整屏。但对大多数应用而言，这已经足够好了。何况你还有语法高亮呢。

"h" 键给你一个可用命令的简短小结。

如果你想设置和 less 不同的选项，在 vimrc 里定义 LessInitFunc，例如：

```
func LessInitFunc()
  set nocursorcolumn nocursorline
endfunc
```

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者：iCrazy<icrazy@ustc.edu> http://vimcdoc.sf.net
更新：tocer

从崩溃中恢复 crash-recovery

您经历过这样的事情么？您录入一个文档，录入了很长时间，眼看就要完成的时候，灾难来了：您的电脑崩溃了！

别 担 心 ！

您可以从 Vim 提供的文件里恢复大部分的改动，Vim 用这些文件保存文件的内容。一般情况下，用单个命令就可以恢复您的文档：

```
vim -r filename
```

1. 交换文件 swap-file
2. 恢复 recovery

---

1. 交换文件 swap-file

Vim 把您对文件的改动保存在一个交换文件中。多数情况下，用修改前的原文件加上 Vim 的交换文件，您就可以恢复对文档所做的改动了。

您可以用以下的命令看到当前正在使用的交换文件的文件名：

```
:sw[apname]                                    :sw  :swapname
```

也可使用 swapname() 函数，也可以看到其它缓冲区的交换文件名。

正常情况下，交换文件的文件名和您正在编辑的文件相同，只是扩展名换成了 ".swp"。
- 在 Unix 系统上，交换文件名以 '.' 开始，这样它们就不会在列目录文件时被显示出来。
- 在 MS-DOS 系统上，如果 'shortname' 选项是打开的，原文件名中的 '.' 会被 '_' 替代。
- 如果同名交换文件已经存在（例如：当您从一个崩溃中恢复时），Vim 会显示一个警告信息，并且会使用另外一个诸如 ".swo"，".swn" ……这样的文件扩展名。
- 现存的文件永远不会因此被 Vim 所覆盖。
- 当 Vim 停止编辑文件的时候，交换文件会被立即删除。

技术上：用 '_' 替代 '.' 是为了避免在与 MS-DOS 兼容的文件系统中（crossdos、multidos 等）出现问题。如果 Vim 检测到您所编辑的文件存放在一个与 MS-DOS 兼容的文件系统上，Vim 会建立一个标志位，实现和 'shortname' 选项相同的功能。当您开始编辑其他文件时，这个标志位会被重置。

E326

如果扩展名是 ".swp" 的文件已经存在，逐次递减最后的字母，直到找到没有现存文件的名字为止。如果过了 ".saa" 还找不到，不建立交换文件。

通过设置 'directory' 这个选项，您可以把交换文件放到其他目录中去，而不是默认地和您所编辑的文件放在同一目录中。

优点：
- 您不会因为 ".swp" 文件的存在弄乱您的目录。
- 当您把选项 'directory' 的值设置到另一个分区的时候，可以减少（在一次系统崩溃中）文件所在分区文件系统毁坏的危险。

缺点：
- 如果您在不同目录下面编辑同名文件，您会碰到文件重名问题（尽管 Vim 会比较文件路径而尽力避免此问题的发生）。这会导致 Vim 给出虚假的 ATTENTION 警告信息。
- 如果您使用您的宿主（home）目录，而别人试图编辑同一个文件的时候，他不能访问您的交换文件，也就不会出现 ATTENTION 警告信息。

在 Amiga 系统上，您还可以使用一种可以恢复数据的 ram disk，但是不能保证它可以 100% 工作正常。把交换文件放在普通的 ram disk（就像 Amiga 系统中的内存那样）或者放在重启后所有内容都会被清除的地方（就像 Unix 上的 /tmp 目录）是没有意义的，在一次计算机崩溃中您会丢失所有的交换文件。

如果您希望把交换文件放在一个固定的地方，把类似于下面这些的命令写到您的 .vimrc 文件中去：

```
:set dir=dh2:tmp        (针对 Amiga)
:set dir=~/tmp          (针对 Unix)
:set dir=c:\\tmp        (针对 MS-DOS 和 Win32)
```

这对于编辑软盘上的文件是十分便利的。当然您必须先自己建立这个 "tmp" 目录。

对于那些只读的文件，Vim 不会产生交换文件，除非文件特别大，使得内存使用超出了 'maxmem' 或 'maxmemtot' 的设置值。当然，如果对只读的文件进行了修改，交换文件文件还是会被自动建立。

可以复位 'swapfile' 这个选项的值以避免产生交换文件。:noswapfile 修饰符可用来使新缓冲区不建立交换文件。

:nos[wapfile] {command} :nos :noswapfile
执行 {command}。如果包含载入新缓冲区的命令，缓冲区载入时不建立交换文件，并复位 'swapfile' 选项。如果缓冲区已经有了交换文件则不删除之，也不复位 'swapfile'。

检测已经存在的交换文件

您可以在用户手册中找到这些信息，参阅章节 11.3 。

更新交换文件

在您录入 200 个字符后或者 4 秒内没有录入任何字符时，交换文件会被更新。只有在缓冲区被修改的情况下，才会有这样的更新，仅仅移动光标是不会更新交换文件的。为什么不进行实时的更新？原因在于这样频繁的更新会极大的拖慢其他正常工作的速度。您可以在 'updatecount' 选项中修改 200 个字符的设置，还可以在 'updatetime' 选项中修改 4 秒的设置（时间以毫秒为单位给出）。Vim 更新交换文件后，会把文件同步到磁盘上去。这会消耗一定的时间，在忙碌的 Unix 系统上尤其如此。如果不希望这样，您可以把 'swapsync' 选项置成空串。不过，这样会使文件丢失的几率增大。在一些非 Unix 系统（MS-DOS、Amiga）上，交换文件根本不会被写到磁盘上。

如果不希望把修改写到交换文件中，可以通过把 'updatecount' 选项设成 0 来避免这一点，用 "-n" 选项启动 Vim 时也可以达到相同的效果。而把 'updatecount' 选项设成非 0 值可以恢复对交换文件的写入。这种情况下，所有缓冲区的交换文件都会被建立。但是当把 'updatecount' 选项设成 0 时，已经存在的交换文件不会被删除，只有设置之后打

开的文件才受到影响。

如果您想确定您对文件所作的修改已经被写到交换文件中了，使用这个命令：

:pre :preserve E313 E314

:pre[serve] 把缓冲区中的所有文本写入交换文件中。这样在文件恢复时就不再需要原文件了。
本命令设置了当前缓冲区的一个标志位。如果 'cpoptions' 里有 '&' 标志位而且 Vim 退出时如果此缓冲区还处于载入状态，不删除此缓冲区的交换文件 cpo-& 。

Vim 会通过文件开始的 6 个字符 "b0VIM " 来识别它的交换文件。Vim 的版本号（如：3.0）会紧跟在那些字符之后。

链接和符号链接

Unix 上同一文件可以有两个名字。这可以通过硬链接和符号链接（symlink）完成。

对于硬链接，Vim 不知道文件另外的名字。所以，交换文件名基于你用来编辑的文件名。也不会检查是否编辑了使用别的名字的相同文件，因为 Vim 无法找到另外那个交换文件（除非在你整个硬盘上搜索，这太慢了）。

对于符号链接，Vim 解析链接以找到实际文件的名字。交换文件名基于那个实际的名字。所以不管你用哪个名字编辑，交换文件名通常总是相同的。但有若干例外：
- 如果实际文件所在的目录不可写，交换文件会放在别处。
- 如果符号链接因为某些原因形成了循环，你会得到 E773 错误消息，而使用未经修改的文件名。你不能在以后正常地保存文件。

---

2. 恢复 recovery E308 E311

基本的文件恢复已经在用户手册中解释了： usr_11.txt 。

另一种恢复方式是在启动 Vim 后使用 ":recover" 命令。当您用 Vim 编辑一个文件并且得到一个 "注意：已找到一个交换文件 ..." 这样的提示时，这样比较方便。这种情况下，一个简单的 ":recover" 命令就可以完成所有的工作。您也可以在使用 ":recover" 命令时指定文件名或者交换文件名：

:rec :recover E305 E306 E307

:rec[over] [file] 试图从交换文件中恢复 [file]。如果没有指定 [file]，使用当前缓冲区的文件名。当前缓冲区中的内容会丢失。如果缓冲区已经被修改，此命令失效。

:rec[over]! [file] 类似于 ":recover"，但当前缓冲区中的任何改变都会丢失。

E312 E309 E310

在交换文件被损坏时，Vim 仿佛有些智能，知道接下来该怎么做。如果 Vim 对于找到的文件有所怀疑，它会给出一个出错信息，并且在文本中插入含有 "???" 的行。如果您在恢复的时候看到了出错信息，请在文件中查找 "???" 并确定到底哪里出了问题。或许您仍然可以通过剪切和粘贴来取回可用的信息。

最常见的信息是 "???LINES MISSING"。这表示 Vim 无法从原文件中读取文本。通常在系统崩溃而导致部分原文件无法及时写入磁盘时会出现这样的情况。

在覆盖原文件并删除交换文件之前，请务必确定您所作的恢复是成功的。先把恢复出来的文件存到别的地方，然后运行 'diff' 来确定是否已经所需要的改动，是个好习惯。也可使用 :DiffOrig 。

当您确定恢复成功后，请删除交换文件。否则，您会继续得到 ".swp" 文件已经存在的警告信息。

加 密 和 交 换 文 件 :recover-crypt

如果文本文件经过加密，交换文件也同样要加密。这使得恢复稍微复杂一些。从交换文件恢复而又用到加密时，你需要输入一到两次加密密钥。

如果文本文件不再存在，只需要输入加密密钥一次来解密交换文件。

如果文本文件存在，其加密方式可能与交换文件不同。你需要输入加密密钥两次：

```
Need encryption key for "/tmp/tt"
Enter encryption key: ******
"/tmp/tt" [crypted] 23200L, 522129C
Using swap file "/tmp/.tt.swp"
Original file "/tmp/tt"
Swap file is encrypted: "/tmp/.tt.swp"
If you entered a new crypt key but did not write the text file,
enter the new crypt key.
If you wrote the text file after changing the crypt key press enter
to use the same key for text file and swap file
Enter encryption key:
```

有两种可能的情况：

1. 加密密钥没有改变，或者在改变后才写入文本文件。你需要输入加密密钥两次。第二次，按回车即可。这样就会应用相同的密钥来解密文本文件和交换文件。
2. 输入了新加密密钥，但之后没有保存文本文件。Vim 会使用新密钥来加密交换文件，但文本文件仍然会使用旧密钥来加密。第二次提示时要输入新密钥。

注意 恢复后，交换文件的密钥会用于文本文件。写入文本文件时，需要使用新密钥。

vim:tw=78:ts=8:noet:ft=help:norl:

cmdline.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Dasn, tocer http://vimcdoc.sf.net

命令行模式 Cmdline-mode Command-line-mode Cmdline Command-line mode-cmdline :

命令行模式可以用来输入 Ex 命令 (":")，搜索符合模式的字符串 ("/" 和 "?")，和过滤命令 ("!")。

一些基本的、与命令行编辑相关的内容在用户手册的第 20 章里有所描述，见：usr_20.txt 。

1. 命令行编辑 cmdline-editing
2. 命令行补全 cmdline-completion
3. Ex 命令行 cmdline-lines
4. Ex 命令行的范围 cmdline-ranges
5. Ex 命令行的标志位 ex-flags
6. Ex 特殊字符 cmdline-special
7. 命令行窗口 cmdline-window

---

1. 命令行编辑 cmdline-editing

通常，字符会在光标位置前插入。你可以用键盘上的左右键来移动光标的位置。还可以通过 <Insert> 键，在 "插入" 和 "替换" 两种模式之间切换。

注意，如果你的键盘上没有可以移动光标的键或其它特殊键，你可以用 ":cnoremap" 定义另外一个键来代替它们。例如，定义一组 tcsh 风格的键： tcsh-style

```
:cnoremap <C-A> <Home>
:cnoremap <C-F> <Right>
:cnoremap <C-B> <Left>
:cnoremap <Esc>b <S-Left>
:cnoremap <Esc>f <S-Right>
```

(<> 记法，见 <> ；上面的例子要一字不落地输入)

cmdline-too-long
当命令行指令长度大于屏幕的宽度时，只有合适的部分被显示出来。光标只能在命令的可视部分移动，所以你不能编辑其他部分。

cmdline-history history
在命令行里输入的命令会存储在历史记录中，你可以使用上下箭头键来翻阅它们。有五个历史表：
- 一个是为 ':' 命令准备的
- 一个是为搜索字符串准备的
- 一个是为表达式准备的
- 一个是为输入行准备的，用来输入 input() 函数。
- 一个是为调试模式下的命令准备的

这些记录是完全分开的。每一种输入模式只能访问与之相关的一个记录。
选项 'history' 用来设定保存历史记录的行数 (默认值：50) 。

注意：
- 如果你输入了一条命令与历史记录中已有的命令完全相同，那么历史记录中的这条旧命令会被删除（这样做是为了避免历史列表被大量的重复指令占据）。
- 只有键入的命令才会记录下来。完全来自映射的命令不会。
- 所有的搜索操作都会放到与搜索相关的历史记录中，包括通过 "*" 和 "#" 进行的搜索。但是对于映射，只有最近一次的搜索会被记录下来（这样是为了避免很长的映射把历史记录搞乱）。

{仅当编译的时候加入 +cmdline_hist 特性才可用}

命令行的自动匹配功能；见 cmdline-completion 。

c_CTRL-V

CTRL-V 下一个字符如果是非数字字符，按本义插入。不然，不超过三个的数字字符用于输入单个字节的十进制编码。非数字和三个数字字符不会被映射。这与插入模式下的工作方式相同（见 i_CTRL-V ）。
注意：由于在 MS-Windows 系统上，CTRL-V 被映射为 "粘贴文本"，这时我们要用 CTRL-Q 代替 CTRL-V。
如果打开 modifyOtherKeys ，特殊转义序列被转换回没有 modifyOtherKeys 时的样子，除非同时按住 Shift 键。

c_CTRL-Q

CTRL-Q 与 CTRL-V 相同。但是有些终端把它定义为控制字符，那样就无法使用了。

CTRL-SHIFT-V c_CTRL-SHIFT-V c_CTRL-SHIFT-Q
CTRL-SHIFT-Q 与 CTRL-V 类似，除非激活了 modifyOtherKeys ，此时插入带修饰符的键的转义序列。

c_<Left> c_Left

<Left> 光标左移

c_<Right> c_Right

<Right> 光标右移

c_<S-Left>
<S-Left> 或 <C-Left> c_<C-Left>
光标左移一个字串 (WORD)

c_<S-Right>
<S-Right> 或 <C-Right> c_<C-Right>
光标右移一个字串 (WORD)

CTRL-B 或 <Home> c_CTRL-B c_<Home> c_Home
光标移动至命令的起始

CTRL-E 或 <End> c_CTRL-E c_<End> c_End
光标移动至命令的末尾

c_<LeftMouse>

<LeftMouse> 将光标置于点击鼠标的地方。

c_<MiddleMouse>

<MiddleMouse> 粘贴剪贴板的内容 (X11 而言是 PRIMARY 选择区)。和 CTRL-R * 类似，但行间不插入 CR 字符。

CTRL-H c_<BS> c_CTRL-H c_BS
<BS> 删除光标前面的字符（如果你的 <BS> 键不能正常工作，请参阅 :fixdel ）。

c_<Del> c_Del

<Del>　　删除光标下的字符（如果光标处于行的末尾，则删除光标前的字符）(如果你的 <Del> 键不能正常工作，请参阅 :fixdel)。

c_CTRL-W
CTRL-W　　删除光标前的单词 (word)。单词范围受 'iskeyword' 选项影响。

c_CTRL-U
CTRL-U　　删除光标所在位置与行首之间的所有字符。先前一些版本的 vim 删除整行的所有字符。如果你喜欢原先的这种方式，可以把下面的命令加入到你的 .vimrc :

```
:cnoremap <C-U> <C-E><C-U>
```

c_<Insert>　c_Insert
<Insert>　　在插入和替换之间切换。

{char1} <BS> {char2}　或　c_digraph
CTRL-K {char1} {char2}　c_CTRL-K
输入二合字母（见 digraphs)。如果 {char1} 是一个特殊键，插入该键 <> 形式的编码。

CTRL-R {register}　c_CTRL-R　c_<C-R>
插入寄存器里的内容。当按下 CTRL-R 时，屏幕会显示一个 '"' 字符，提示你输入一个寄存器的名字。
插入的文字就像是你自己输入的，但是映射和缩写不会生效。同样也不会触发 'wildchar' 的补全功能。并且那些可以结束命令的字符会被按字面插入 (<Esc>, <CR>, <NL>, <C-C>)。但 <BS> 或 CTRL-W 仍旧可
能结束命令行模式，并使剩余的字符被接下来其他的模式解释，这可能并非你的本意。
特殊寄存器:

'"'	无名寄存器，包含最近一次删除或复制的内容
'%'	当前文件名
'#'	轮换文件名
'*'	剪贴板的内容 (X11: 主选择区)
'+'	剪贴板的内容
'/'	最近一次的查找模式
':'	最近一次在命令行输入的命令
'-'	最近一次小的（少于一行）删除
'.'	最近插入的文本

c_CTRL-R_=

'='	表达式寄存器：会提示你输入一个表达式。(见 expression) (在表达式提示中无效；不允许改变缓冲区或者当前窗口这样的操作，以避免副作用) 如果结果为 List 列表，其中的每一项当作一行。当然其中内容也可包含换行符。如果结果为浮点数，自动转换为字符串。

与寄存器相关的内容请参见 registers 。
实现细节：如果使用 expression 寄存器并且调用 setcmdpos()，那么在插入返回的文本之前设置光标位置。使用 CTRL-R CTRL-R 在插入返回的文本之后设置光标位置。

CTRL-R CTRL-F　c_CTRL-R_CTRL-F　c_<C-R>_<C-F>
CTRL-R CTRL-P　c_CTRL-R_CTRL-P　c_<C-R>_<C-P>
CTRL-R CTRL-W　c_CTRL-R_CTRL-W　c_<C-R>_<C-W>

CTRL-R CTRL-A　　c_CTRL-R_CTRL-A　c_<C-R>_<C-A>
CTRL-R CTRL-L　　c_CTRL-R_CTRL-L　c_<C-R>_<C-L>

插入光标下的对象：

CTRL-F	光标下的文件名
CTRL-P	用 'path' 扩展的光标下的文件名。与 gf 中的类似。
CTRL-W	光标下的单词
CTRL-A	光标下的字串；参见： WORD
CTRL-L	光标下的文本行

如果置位 'incsearch'，使用当前显示匹配文本的末尾作为光标位置。
使用 CTRL-W 时，已经录入的单词部分不再重复插入。

CTRL-F 和 CTRL-P: {仅当包含 +file_in_path 特性才可用}

c_CTRL-R_CTRL-R　c_<C-R>_<C-R>
c_CTRL-R_CTRL-O　c_<C-R>_<C-O>

CTRL-R CTRL-R {register CTRL-F CTRL-P CTRL-W CTRL-A CTRL-L}
CTRL-R CTRL-O {register CTRL-F CTRL-P CTRL-W CTRL-A CTRL-L}

插入寄存器或光标下的对象。类似于 c_CTRL-R ，但是该命令按本义插入文本。例如，如果寄存器 a 的内容为 "xy^Hz" (^H 是退格键)，那么 "CTRL-R a" 会插入 "xz"，而 "CTRL-R CTRL-R a" 会插入 "xy^Hz"。

CTRL-\ e {expr}　　c_CTRL-_e

计算表达式 {expr} 并用结果替换整个命令行。程序会提示你输入表达式，用 <Enter> 键提交表达式。这项功能在映射里十分有用。参见 expression 。
欲了解插入表达式结果的相关信息，请参阅 c_CTRL-R_= 。
getcmdline() 、 getcmdline() 和 getcmdpos() 函数在这里很有用。
缺省，光标的位置不会改变，除非光标正位于行末，那么它还会停留在行末。
不过，可以用 setcmdpos() 来自行设置光标的位置。
使用沙盘 sandbox 计算表达式以避免讨厌的副作用。
例如：

```
:cmap <F7> <C-\>eAppendSome()<CR>
:func AppendSome()
   :let cmd = getcmdline() . " Some()"
   :" place the cursor on the )
   :call setcmdpos(strlen(cmd))
   :return cmd
:endfunc
```

它无法递归操作，所以如果正在编辑一个表达式，它将无效。但可在映射中使用。

c_CTRL-Y
CTRL-Y　当存在无模式选择时，复制该选择区域的内容至剪贴板。
参考 无模式选择 modeless-selection 。
如果没有选择，那么 CTRL-Y 会被当作字符直接插入。

CTRL-M 或 CTRL-J　　c_CTRL-M　c_CTRL-J　c_<NL>　c_<CR>　c_CR
<CR> 或 <NL>　执行输入的命令

CTRL-[c_CTRL-[c_<Esc> c_Esc
<Esc> 如果 'cpoptions' 选项中不包括 'x'，那么输入该键会退出命令行模式，而不执行命令。若 'cpoptions' 里包含 'x'，或者该键出现在宏命令里，则执行输入的命令。
注意：如果键盘上输入 <Esc> 不方便，请训练一下用 CTRL-[吧。
c_CTRL-C
CTRL-C 不执行命令，退出命令行模式

c_<Up> c_Up
<Up> 在历史记录中查找开始部分与当前输入匹配的前一条命令。(见下面)
{仅当编译时加入 +cmdline_hist 特性才可用}
c_<Down> c_Down
<Down> 在历史记录中查找开始部分与当前输入匹配的下一条命令。(见下面)
{仅当编译时加入 +cmdline_hist 特性才可用}

c_<S-Up> c_<PageUp>
<S-Up> 或 <PageUp>
取回历史记录中的上一条命令
{仅当编译时加入 +cmdline_hist 特性才可用}
c_<S-Down> c_<PageDown>
<S-Down> 或 <PageDown>
取回历史记录中的下一条命令
{仅当编译时加入 +cmdline_hist 特性才可用}

CTRL-D 命令行补全 (见 cmdline-completion)
'wildchar' 选项
命令行补全 (见 cmdline-completion)
CTRL-N 命令行补全 (见 cmdline-completion)
CTRL-P 命令行补全 (见 cmdline-completion)
CTRL-A 命令行补全 (见 cmdline-completion)
CTRL-L 命令行补全 (见 cmdline-completion)

c_CTRL-_
CTRL-_ a - 在希伯来语和英语的键盘模式之间切换，这只是针对命令行模式，与 hkmap 无关。如果命令行、搜索、缩写，等等需要输入希伯来文的话，这一功能就很有用了。只有在 Vim 编译时加入了 +rightleft ，并且置位了 'allowrevins' 选项后，该功能才可用。
参见 rileft.txt 。

b - 在波斯语和英语的键盘模式之间切换，这只是针对命令行模式，与 fkmap 无关。在波斯语键盘模式下，字符的插入方式与平时相反 (即从右至左)。如果命令行、搜索、缩写，等等需要输入波斯语的话，这一功能就很有用了。只有在 Vim 编译时加入了 +farsi ，该功能才可用。
参见 farsi.txt 。

c_CTRL-^
CTRL-^ 语言映射 :lmap 和/或 输入法的开关。
如果输入搜索模式且 'imsearch' 的值不是 -1 ，VAL 是 'imsearch' 的值，否则，VAL 是 'iminsert' 的值。
如果定义了语言映射:
- VAL 为 1 (使用 langmap 映射) 时变成 0 (不使用 langmap 映射)。
- VAL 不为 1 时变成 1，从而打开 langmap 映射。

如果没有定义语言映射:
- VAL 为 2 (使用输入法) 时变成 0 (不使用输入法)
- VAL 为其它值时变成 2，从而打开输入法。
这些语言映射通常用来输入不同于键盘上能直接产生的字符。'keymap' 选项用来安装若干完整的这样的映射表。
命令行输入时，langmap 映射关闭，因为假定你想输入命令。即使你用 CTRL-^ 打开了映射，新的状态也不会被下一条命令或搜索模式再次使用。
(译者注: 此处的语言映射或 langmap 映射与 'langmap' 选项是不同的概念，后者只适用于普通模式。)

c_CTRL-]
CTRL-]　　激活缩写，但不插入任何字符。

对于 Emacs 风格的命令行编辑请参阅 emacs-keys 。

<Up> 和 <Down> 键使用当前的命令行作为搜索的字符串。前面或后面的命令的起始部分与该字符串做比较。最先匹配的作为新的命令。若连续重复这两个键，则使用同一字符串进行再次匹配。例如，寻找前一个替换命令: 输入 ":s" 然后 <Up>。同样的效果也可以用重复输入一连串 <S-Up> 直至找到你需要的命令为止达成。(注意: Shift+箭头键在某些终端里不可用)

:his　:history
:his[tory]　　列出最近执行的命令。
{仅当编译时加入 +cmdline_hist 特性才可用}

:his[tory] [{name}] [{first}][, [{last}]]
列出 {name} 历史记录的内容，{name} 可以是:

c[md]	或 :	命令行的历史记录
s[earch]	或 / 或 ?	查找字符串的历史记录
e[xpr]	或 =	表达式寄存器的历史记录
i[nput]	或 @	输入行的历史记录
d[ebug]	或 >	调试命令的历史记录
a[ll]		所有上述记录

如果给定了数字 {first} 和 {last}，那么就会列出指定范围内的历史记录条目。这些数字可以是下面的格式:

:history-indexing
正数表示历史记录的绝对索引，也就是 :history 命令列出的第一列数字。即使历史记录中的其它条目被删除了，该数字也会保持不变。

负数表示与某一记录的相对位置。以最新的一条记录 (索引号为 -1) 为基准向后算起。

例子:
列出搜索历史记录中的第 6 到第 12 条记录:

```
:history / 6,12
```

列出所有历史记录中倒数第二条记录录:

```
:history all -2
```

列出所有历史记录中最近的两条记录:

```
:history all -2,
```

:keepp[atterns] {command}　　:keepp　:keeppatterns
执行 {command}，不加入搜索历史

---

2. 命令行补全　　cmdline-completion

在编辑命令行的时候，有一些命令可以用来补全光标前面的单词。此功能可用于下述情况：

- 命令名称：在命令行的起始处。
- 标签：只在 ":tag" 命令后。
- 文件名：只在可以接受文件名的命令或可以接受文件名的选项之后。也被称作文件名补全。
- 外壳命令名：在 ":!cmd"、":r !cmd" 和 ":w !cmd" 之后。使用 $PATH。
- 选项：仅在 ":set" 命令之后。
- 映射：仅在 ":map" 或与之类似的命令之后。
- 变量和函数名：仅在 ":if"，":call" 或类似的命令之后。

帮助条目的匹配数目有上限（目前是 300 个），以避免过多匹配造成的延迟问题。

以下是可以使用的命令：

c_CTRL-D
CTRL-D　列出与光标前面的模式所匹配的名字。
在列文件名的时候，目录名会被高亮（见 'highlight' 选项）。扩展名与 'suffixes' 匹配的文件名放到列表的最后。
要列出匹配标签所在的文件，可以设置选项 'wildoptions' 为 "tagfile"。

c_CTRL-I　c_wildchar　c_<Tab>
'wildchar' 选项
用光标前的模式完成匹配。匹配的项目（如果有多项匹配，则选第一项）会插入并取代模式。（注意：该功能不能用于宏，因为在大多数情况下 <Tab> 或 <Esc> 被用做 'wildchar'，而这些键在某些宏里面有特殊的意义。）如果再次输入，并且有多项匹配时，则插入下一项匹配。在最后一项匹配之后，会再次匹配第一项（循环使用）。
该行为可以通过设置 'wildmode' 选项来改变。

c_<S-Tab>
<S-Tab>　和 'wildchar' 或 <Tab> 类似，但从最末匹配开始，依次选择前一项匹配。
<S-Tab> 不是所有系统都可用。

c_CTRL-N
CTRL-N　在使用 'wildchar' 之后，如果得到了多项匹配，则选择下一项匹配。否则选择命令历史列表中的下一条命令。

c_CTRL-P
CTRL-P　在使用 'wildchar' 之后，如果得到了多项匹配，则选择前一项匹配。否则选择命令历史列表中的上一条命令。

c_CTRL-A
CTRL-A　将所有与模式匹配的项全部插入到光标前面。

c_CTRL-L
CTRL-L　用光标前的模式完成匹配。如果有匹配的项目，则插入并取代当前的模式。如果有多个项目匹配，那么取这些项目中尽可能长的共同部分插入以取代模式。若结果比模式短，则匹配不成功。

/_CTRL-L

如果置位 'incsearch'，用 "/" 或 "?" 录入搜索模式而显示当前的匹配时，CTRL-L 会从当前匹配的末尾一次加一个字符。如果置位了 'ignorecase' 和 'smartcase' 而命令行没有大写字符，加入的字符会转换为小写。

c_CTRL-G /_CTRL-G

CTRL-G 如果置位 'incsearch'，用 "/" 或 "?" 录入搜索模式而显示当前的匹配时，CTRL-G 会移动到下一个匹配（不考虑 search-offset）。CTRL-T 移动到前一个匹配。提示：在常规键盘上 T 在 G 上方。

c_CTRL-T /_CTRL-T

CTRL-T 如果置位 'incsearch'，用 "/" 或 "?" 录入搜索模式而显示当前的匹配时，CTRL-T 会移动到前一个匹配（不考虑 search-offset）。CTRL-G 移动到下一个匹配。提示：在常规键盘上 T 在 G 上方。

选项 'wildchar' 的默认值是 <Tab>（在 Vi 兼容模式里用的是 CTRL-E；在以前的版本里用的是 <Esc>）。如果用来匹配文件名，在匹配模式里可以使用标准的通配符 '*' 和 '?'。'*' 匹配任意字符串，'?' 匹配一个字符。

重复使用 'wildchar' 或 CTRL-N 会循环选择匹配，并最终转回到正在输入的内容。如果首个匹配不如所愿，可以用 <S-Tab> 或 CTRL-P 直接回到刚输入的内容。

置位选项 'wildignorecase' 可用于忽略文件名的大小写。

'wildmenu' 选项可用于设置在命令行正上方显示可能的匹配。

如果你喜欢 tcsh 的自动列表匹配，可以使用这个映射：

```
:cnoremap X <C-L><C-D>
```

(X 是命令键，<C-L> 是 CTRL-L，<C-D> 是 CTRL-D) 这样就可以找到最长的匹配并列出所有匹配的文件。

如果你喜欢 tcsh 的自动列表匹配，你可以使用 'wildmode' 选项来模拟它。例如，这与 autolist=ambiguous 极其类似：

```
:set wildmode=longest,list
```

这将在你输入第一个 'wildchar' 之后，找到最长的匹配，接着列出所有匹配的文件。

suffixes

对于文件名的匹配你可以用 'suffixes' 选项在相似的文件名之间设定优先级。如果有多个文件匹配，并且 'suffixes' 选项中包含这些文件的扩展名，那么这些文件会被忽略。'suffixes' 的默认值是 ".bak,~,.o,.h,.info,.swp,.obj"。其含义是：以 ".bak"，"~"，".o"，".h"，".info"，".swp" 和 ".obj" 结尾的文件名有时被忽略。

空项目，即两个连续的逗号，匹配不包含 "." 的文件名，因为它们没有扩展名。可用来忽略 "prog" 而选择 "prog.c"。

例如：

模式：	文件：	匹配：
test*	test.c test.h test.o	test.c
test*	test.h test.o	test.h and test.o
test*	test.i test.h test.c	test.i and test.c

无法忽略包括两个句号的后缀名。

如果有多个文件匹配（在忽略了与 'suffixes' 匹配的扩展名之后），则第一个匹配的文件名被插入。当你按两次 'wildchar' 时，你可以看到只有一个匹配并且完整的匹配保持

不变。你可以再输入一次 'wildchar'，CTRL-N，或 CTRL-P 得到另一个匹配。包括所有的文件，也包括扩展名与 'suffixes' 匹配的文件。

要完全忽略有某些扩展名的文件，请使用 'wildignore' 选项。

要只匹配以输入文本结尾的文件，附加 "$" 即可。例如，要匹配 ".c" 结尾的文件：

```
:e *.c$
```

这样就不会匹配 ".cpp" 结尾的文件。如果没有 "$" 就会。

一个选项的旧值可以通过在 '=' 后面输入 'wildchar' 来获得。例如，在 ":set dir=" 后输入 'wildchar'，就会插入当前 'dir' 的值。对于选项值是文件名的选项，这一行为优先于文件名匹配。

若你喜欢在 xterm 里使用 <S-Tab> 代替 CTRL-P，把这条命令放到你的 .cshrc 里：

```
xmodmap -e "keysym Tab = Tab Find"
```

把这条命令放到你的 .vimrc 里：

```
:cmap <Esc>[1~ <C-P>
```

---

3. Ex 命令行 cmdline-lines

Ex 命令有几个特别的地方：

:quote :comment

如果 '"' 出现在一行的开始，那么该行被忽略。如果 '"' 出现在命令之后，则使得该行的剩余部分被忽略。这可以被用于添加注释。例如：

```
:set ai         "set 'autoindent' option
```

不能给外壳命令 (":!cmd") 或 ":map" 和其它的一些的命令 (主要是期待表达式的命令) 添加注释，因为它们把 '"' 视为命令的一部分。

```
:argdo
:autocmd
:bufdo
:cexpr (诸如此类)
:cdo (诸如此类)
:command
:cscope (诸如此类)
:debug
:display
:echo (诸如此类)
:elseif
:execute
:folddoopen
:folddoclosed
:for
:grep (诸如此类)
:help (诸如此类)
:if
:let
:make
:map (诸如此类，包括 :abbrev 等命令)
:menu (诸如此类)
:mkspell
:normal
```

:ownsyntax
:popup
:promptfind (诸如此类)
:registers
:return
:sort
:syntax
:tabdo
:tearoff
:vimgrep (诸如此类)
:while
:windo

:bar :\bar

'|' 可以用来分割命令，所以你可以在一行里输入多个命令。如果你想在参数里使用 '|'，应该在前面加上 '\'。

下面这些命令把 '|' 当作它们的参数，所以后面不能跟别的 Vim 命令:

:argdo
:autocmd
:bufdo
:cdo
:cfdo
:command
:cscope
:debug
:folddoopen
:folddoclosed
:function
:global
:help
:helpfind
:lcscope
:ldo
:lfdo
:make
:normal
:perl
:perldo
:promptfind
:promptrepl
:pyfile
:python
:registers
:read !
:scscope
:sign
:tcl
:tcldo
:tclfile
:vglobal
:windo
:write !
:[range]!

没有参数 "-bar" 的用户自定义命令 :command

注意：在 ":g" 中 '|' 包含在命令里（作为参数），但 ":s" 却不是。这的确有点乱（继承于 Vi）。

为确保总能执行另一个命令，请使用 ":execute" 命令。
例如（读入 "ls" 的输出，并且跳到第一行）：

```
:execute 'r !ls' | '[
```

有一个例外：如果 'b' 出现在 'cpoptions' 选项中，那么在 ":map" 和 ":abbr" 之类的命令里需要用 CTRL-V 代替 '\'。你也可以用 "<Bar>" 来代替。参见 map_bar 。

例如：

```
:!ls | wc               查看两个命令的输出
:r !ls | wc             将相同的输出插入文本
:%g/foo/p|>             将所有匹配的行移动一个 shiftwidth
:%s/foo/bar/|>          将一行移动一个 shiftwidth
:map q 10^V|            将 "q" 映射为 "10|"
:map q 10\| map \ l     将 "q" 映射为 "10\" 和将 "\" 映射为 "l"
                                (如果 'cpoptions' 选项中包含 'b')
```

用与 '|' 相同的方法，你也可以用 <NL> 分割多个命令。用 CTRL-V CTRL-J 插入一个 <NL>。Vim 会显示一个 "^@"。应该说，使用 '|' 是一个更好的方法。但是对于外部命令，就必须使用 <NL>，因为外部命令会包含 '|'。如果要避免 <NL> 的特殊含义，就必须在它前面加反斜杠。例如：

```
:r !date<NL>-join
```

将当前时间读入文件，并且与上一行文本衔接。

注意：如果 '|' 前面的命令产生了错误，那么其后面的命令不会执行。

为了兼容 Vi，下面几个奇怪的命令 Vim 也支持：

```
:|                      打印当前行（同 ":p"）
:3|                     打印第 3 行（同 ":3p"）
:3                      前进至第 3 行
```

在范围与命令之间允许有一个冒号。这个冒号被忽略（与 Vi 兼容）。例如：

```
:1,$:s/pat/string
```

当字符 '%' 或 '#' 被用在一个需要文件名的地方时，会被扩展为当前的文件名或轮换文件名（参见 "编辑文件" 的章节 :_% :_# ）。

在 Amiga 系统上，如果文件名要作为参数使用，那么文件名里是允许包含空格符的。通常末尾的空格符会被忽略，除非使用转义字符反斜杠或 CTRL-V 使它转义。注意，":next" 命令使用空格符分割文件名。所以要在该命令里使用带空格符的文件名，就需要将空格符转义。例如：

```
:next foo\ bar goes\ to school\
```

开始编辑 "foo bar"，"goes to" 和 "school " 三个文件。

当你想在命令里使用特殊字符 '"' 或 '|'，或者想在文件名里使用 '%' 或 '#'，就需要在这些字符前加一个反斜杠。在范围和 ":substitute" 命令里不需要使用反斜杠。
另见 `= 。

:_!

在 Ex 命令之后的 '!' (叹号) 使得命令以不同的方式执行。'!' 应该紧跟在命令之后，中间没有任何空白字符。如果插入了空白字符，'!' 则被视为该命令的参数，其含义是不同的。例如:

:w! name	将当前缓冲区内容写入文件 "name"。若该文件存在，则覆盖之。
:w !name	将当前缓冲区内容作为标准输入发送给 "name" 命令。

---

4. Ex 命令行的范围 cmdline-ranges [range] E16

有些 Ex 命令前面可以接受一个范围限定，这被称作 [range] (范围)。它由一个或多个被 ',' 或 ';' 分割的行限定符组成。

一些基本的内容在用户手册的 10.3 里解释过。

:, :;

当范围被 ';' 分割时，在解释下一个行限定符之前，光标的位置会被设置为前一个行限定符确定的行值。这种情况不会发生在 ',' 上。例如:

4,/this line/
从第 4 行到匹配当前光标后的一个 "this line" 为止的范围。

5;/that line/
从第 5 行到匹配第 5 行之后的一个 "that line" 为止的范围。

对大多数命令而言，默认的行限定符就是光标的位置，但是 ":write" 和 ":global" 命令的默认值是整个文件 (1,$)。

若行限定符的个数多于命令需要的数量，则第一个 (或前几个) 限定符被忽略。

行号可以用下列符号限定: :range {address}

{number}	行号	
.	当前行	:.
$	文件的最后一行	:$
%	相当于 1,$ (整个文件)	:%
't	标记 t 的位置 (小写)	:'
'T	标记 T 的位置 (大写)；如果标记存在于另一个文件中，则不能在范围里应用。	
/{pattern}[/]	下一个 匹配 {pattern} 的行	:/
?{pattern}[?]	前一个 匹配 {pattern} 的行	:?
\/	下一个 与前次搜索模式匹配的行	
\?	前一个 与前次搜索模式匹配的行	
\&	下一个 与前次替代模式匹配的行	

这些符号后面可以跟 (一个或多个) '+' 或 '-' 和一个可选的数字。该数字会与前面的行号相加或相减。若该数字省略，则认为是 1。

在 {pattern} 之后需要由 "/" 或 "?" 来分隔匹配模式与其之后的内容。

"/" 和 "?" 前面可能有另一个地址。查找就从那里开始。和使用分号的方式不同之处是: 光标不会被移动。例如:

/pat1//pat2/	从包含 "pat1" 的行开始，向后寻找直至找到包含 "pat2" 的行为止，不移动光标位置。
7;/pat2/	从第 7 行开始，向后寻找直至找到包含 "pat2" 的行为止。光标留在第 7 行。

{number} 必须在 0 和文件的总行数之间。当使用 0（零）的时候，大多数命令会把它解释成 1 。把它用作计数器的命令会将它解释为零（ :tag ， :pop 等）。有些命令会把零解释为 "在第一行之前"（ :read ，搜索模式等）。

例如：

```
.+3             光标下面三行
/that/+1        下一个包含 "that" 的行的下一行
.,$             从当前行直至文件末尾
0;/that         第一个包含 "that" 的行，也能匹配第一行
1;/that         第一行之后（从第二行开始）包含 "that" 的那一行
```

有些命令后面可以跟一个数量。这个数量被用作从上一个行限定符（默认值是当前光标所在的行）开始计算的行数。可以使用范围但是不使用文件名作为参数的命令可以接受这种数量参数（因为文件名也可能是一个数字）。

例如：

```
:s/x/X/g 5      用 'X' 替换当前行和下面四行里的 'x'。
:23d 4          删除 23，24，25 和 26 行。
```

折叠和范围

如果激活了折叠功能，行号会被取整以包括整个折叠。见 fold-behavior 。

反向范围 E493

在范围里，小的数字应该出现在前面。如果不是这样，Vim 会询问你是否交换。

```
Backwards range given, OK to swap
```

但全局命令 ":g" 里不会这么做。

在命令前用 ":silent" 可以避免此问题，这时范围总是进行交换。

计数和范围 N:

如果在进入 ":" 之前输入了计数前缀，这种情况会被解释成：

```
:.,.+(count - 1)
```

简单来说就是：包括当前行，向后共计 'count' 行。例如，删除三行：

```
3:d<CR>          被翻译成：.,.+2d<CR>
```

可视模式和范围 v_:

{Visual}: 将可视模式中选择的区域用作命令行模式的范围。 :'<,'> 被用来表示这种范围。这样就可以在不同的选择区域上，从命令历史列表里选择相似的命令来重复执行操作。
如果已经退出可视模式，更简短的方法是用 :* 来得到可视区域的范围。需要 'cpo' 里不出现 "*"，参见 cpo-star 。否则，只能通过输入 :'<,'> 得到。

5. Ex 命令行的标志位 ex-flags

部分 Ex 命令支持以下标志位。它们显示光标在执行完命令之后所在的行：

```
l       像 :list 那样输出
#       加上行号
p       像 :print 那样输出
```

可以联合使用这些标志位，这样 "l#" 表示使用行号并且使用 :list 风格输出。

---

6. Ex 里的特殊字符 cmdline-special

注意：这些是要执行的 Ex 命令行中的特殊字符。如果你想要在录入时插入特殊的内容，可以使用 CTRL-R 命令。例如，"%" 表示当前文件名，而 CTRL-R % 则立即插入当前文件名。见 c_CTRL-R 。

备注：在 Vim 脚本中要避免特殊字符的效果，可用 fnameescape() 。另见 ‘= 。

在 Ex 命令中，下列字符在可以使用文件名的地方有特殊的意义。它们同样可以用到表达式函数 expand() 中。

%	被当前文件名取代。	:_% c_%
#	被轮换文件名取代。	:_# c_#
	每个窗口都会分别记住。	
#n	(n 是一个数字) 被第 n 个缓冲区的文件名取代。	:_#0 :_#n
	"#0" 与 "#" 相同。	c_#n
##	被参数列表里的所有文件名 (使用空格符分隔) 取代。	:_## c_##
	文件名中的空格符前面会加一个反斜杠。	
#<n	(n 是一个正数) 被第 n 个旧文件名取代。	:_#< c_#<
	要得到这个数值，见 :oldfiles 或 v:oldfiles 。	E809
	{仅当编译时加入 +eval 和 +viminfo 特性才可用}	

注意：除了 "#<n" 以外，它们只给出输入时使用的文件名。如果需要绝对路径 (当从另一个目录使用文件名的时候)，你需要加 ":p"。见 filename-modifiers 。

"#<n" 项返回绝对路径，但在主目录下的文件名会以 "~/" 开始。

注意：空格符前面会插入反斜杠，以便 Vim 命令会正确地解释文件名。但是外壳命令不会如此。为此，你可以使用引号 (但对包含引号和通配符的文件名会有问题)：

```
:!ls "%"
:r !spell "%"
```

要避免 '%' 和 '#' 的特殊含义，可以在它前面插入一个反斜杠。细节：只要有反斜杠在这些字符前面，它们就不具备特殊含义，不管有多少个反斜杠。

你输入：	结果
#	轮换文件名
\#	#
\\#	\#

另见 ‘= 。

:<cword> <cword> :<cWORD> <cWORD>
:<cexpr> <cexpr> :<cfile> <cfile>

:<afile> <afile> :<abuf> <abuf>
:<amatch> <amatch>
:<sfile> <sfile> :<slnum> <slnum>
:<sflnum> <sflnum> E499 E500

注意：下面这些不是特殊键，需要逐个字符输入！

<cword>　被当前光标下的单词代替（类似于 star ）

<cWORD>　被当前光标下的字串代替（见 WORD ）

<cexpr>　被当前光标下的单词，必要时包含更多内容以构成的完整 C 表达式所代替。如，当光标在 "ptr->arg" 的 "arg" 上，结果是 "ptr->arg"；如光标在 "list[idx]" 的 "]" 上，则结果是 "list[idx]"。用于 v:beval_text 。

<cfile>　被当前光标下的路径名代替（类似于 gf 用的方法）

<afile>　在执行自动命令时，被所操作的缓冲区或用来读或写的文件的文件名取代。 E495

<abuf>　在执行自动命令时，被当前有效的缓冲区序号取代（对于 ":r file" 和 ":so file" 来说是当前的缓冲区，因为被读取/执行的文件不在缓冲区里）。 E496

<amatch>　在执行自动命令时，被激发自动命令的实际匹配代替。 E497
只有在文件名不用于匹配（对于 FileType、Syntax 和 SpellFileMissing 事件）时，才与 <afile> 不同。

<sfile>　执行 ":source" 命令时，替换为正在被 "source" 执行的文件名。 E498
执行函数时，替换为：
"function {function-name}[{lnum}]"
其中，函数嵌套调用表示为：
"function {function-name1}[{lnum}]..{function-name2}[{lnum}]"
注意，如果 <sfile> 用在函数里，那么文件名修饰符 (filename-modifiers) 就没用了。

<slnum>　执行 ":source" 命令时，替换为行号。 E842
执行函数时，这是相对函数开始处的行号。

<sflnum>　执行脚本时，替换为行号。
和 <slnum> 不同在于不管什么情况，<sflnum> 都会被替换为脚本的行号。 E961

filename-modifiers
:_%: ::8 ::p ::. ::~ ::h ::t ::r ::e ::s ::gs ::S
%:8 %:p %:. %:~ %:h %:t %:r %:e %:s %:gs %:S

可以在 "%"，"#"，"#n"，"<cfile>"，"<sfile>"，"<afile>" 或 "<abuf>" 之后使用文
件名修饰符。这些修饰符也可以和 fnamemodify() 函数一起用。

此类修饰符必须按如下顺序给出：

:p　给出文件名的全路径。必须是第一个修饰符。也会将 "~/" (Unix 和 VMS 系统里的 "~user/") 变成指向用户主目录的路径。若被修饰的名字是一个目录，则在后面加一个路径分隔符。如果文件名不存在，或其绝对路径不存在，那么结果不可预料。MS-Windows 上 8.3 文件名被扩展为长文件名。

:8　将路径转换为 8.3 短格式（目前仅适用于 MS-Windows）。转换的结果尽可能与现存路径一致。

:~　如果可能，路径名缩减为基于主目录的相对路径。若文件不在主目录下，则文件名不会被改变。

:.　如果可能，路径名缩减为基于当前目录的相对路径。若文件不在当前目录下，则文件名不会被改变，但在 MS-Windows 上，如果和当前驱动器

匹配，会删除驱动器部分。
要想尽可能缩减，可使用 ":~:." 。

:h　文件名的头部（除去文件名的最后一部分以及路径分隔符）。
不能与 :e，:r 或 :t 一起使用。
可以被重复使用来删除后面的多个部分。
如果文件名以一个路径分隔符结尾，仅删除该分隔符。这样一个目录名的 ":p:h" 结果是目录名本身（没有后面的斜杠）。
如果文件名是一个绝对路径（Unix 上以 "/" 开始；WIN32 以 "x:\" 开始；Amiga 以 "drive:" 开始），这些部分不会被删除。如果没有文件头（路径是相对于当前目录的），则结果为空。

:t　文件名的尾部（文件名的最后一部分）。必须在 :r 或 :e 之前。

:r　文件名的根部（除去最后的扩展名）。如果只有扩展名（文件名以 '.' 开始，例如，".vimrc"），则不会被删除。可以重复使用，以删除多个扩展名（最后一个先被删除）。

:e　扩展名。只有单独使用时才有意义。
如果没有扩展名，那结果为空。
如果文件名只是一个扩展名（以 '.' 开始的文件名），则结果为空。
可以重复使用来包含更多的扩展名。如果没有足够的扩展名（但是至少有一个），那么就尽可能多的包含。

:s?pat?sub?
用 "sub" 代替第一次出现的 "pat"。这类似于 :s 命令。"pat" 是一个正则表达式。
可以用其它字符代替 '?'，只要该字符不出现在 "pat" 或 "sub" 里。
在此之后，上述的修饰符可以再次使用。例如 ":p"，是替换之后的完整路径。

:gs?pat?sub?
用 "sub" 替换所有的 "pat"。其余和 ":s" 一样。

:S　转义所有外壳命令使用的特殊字符（见 shellescape() ）。必须是最后一个修饰符。示例：

```
:!dir <cfile>:S
:call system('chmod +w -- ' . expand('%:S'))
```

例如，如果文件名是 "src/version.c"，当前目录是 "/home/mool/vim":

```
:p                  /home/mool/vim/src/version.c
:p:.                           src/version.c
:p:~                       ~/vim/src/version.c
:h                             src
:p:h                /home/mool/vim/src
:p:h:h              /home/mool/vim
:t                                 version.c
:p:t                               version.c
:r                             src/version
:p:r                /home/mool/vim/src/version
:t:r                               version
:e                                       c
:s?version?main?                src/main.c
:s?version?main?:p  /home/mool/vim/src/main.c
:p:gs?/?\\?         \home\mool\vim\src\version.c
```

又例，如果文件名是 "src/version.c.gz":

```
:p                  /home/mool/vim/src/version.c.gz
:e                                             gz
:e:e                                         c.gz
:e:e:e                                       c.gz
```

```
:e:e:r                                          c
:r                                    src/version.c
:r:e                                            c
:r:r                                  src/version
:r:r:r                                src/version
```

extension-removal :_%<

如果 "<" 跟在 "%", "#", "#n" 或 "CTRL-V p" 后面，那么文件的扩展名 ('.' 之后的内容，包括 '.') 会被删除。包含这样的功能是为了与 3.0 版本兼容，现在推荐使用 ":r" 的形式。例如:

```
%               当前文件名
%<              当前主文件名 (无扩展名)
#               当前窗口的轮换文件名
#<              同上，但无扩展名
#31             序号为 31 的轮换文件名
#31<            同上，但无扩展名
<cword>         光标下的单词
<cWORD>         光标下的字串 (见 |WORD|)
<cfile>         光标下的路径名
<cfile><        同上，但无扩展名
```

注意: 在需要文件名的地方，就会完成文件名的通配符扩展。在 Unix 上，除非可以在内部实现 (为了运行速度考虑)，此功能会通过外壳来完成。
除非在 restricted-mode 中，也可以使用反引号，比如在

```
:n `echo *.c`
```

但是扩展仅在 '%', '#' 等字符扩展之前就出现通配符的时候发生。这可以避免对文件名中的通配符字符进行扩展。如果你想扩展 <cfile> 的结果，可以给它加一个通配字符。例如: (轮换文件名是 "?readme?")

```
命令            扩展为
:e #            :e ?readme?
:e `ls #`       :e {匹配 "?readme?" 的文件}
:e #.*          :e {匹配 "?readme?.*" 的文件}
:cd <cfile>      :cd {光标下的文件}
:cd <cfile>*     :cd {光标下的文件名加上 "*" 然后进行扩展}
```

另见 `= 。

当被扩展的参数包含一个 "!" 并且在外壳命令里 (":!cmd", ":r !cmd" 或 ":w !cmd") 使用时，"!" 被反斜杠转义，以免扩展为前次使用的命令。如果 'shell' 选项包含 "sh"，会执行两次转义，以避免外壳扩展 "!"。

filename-backslash

对于文件系统使用反斜杠作为目录分隔符的操作系统 (MS-Windows) 来说，确认反斜杠是否为转义字符就有点儿困难了。一般来说: 如果反斜杠后面跟一个普通的字符，那么就没有特殊意义。因此 "\file\foo" 是合法文件名，没有必要输入两次反斜杠。

一个例外是 '$' 符号。在文件名里它是一个合法的字符。但是为了避免像 "$home" 这样的文件名被当作环境变量解释，则需要在前面加一个反斜杠。因此你需要用 "/\$home" 表示根目录下的 "$home" 文件。几个例子:

```
文件名          被解释为
$home           扩展为环境变量 $home 的值
\$home          当前目录里的 "$home" 文件
```

```
/\$home         根目录下的 "$home" 文件
\\$home         名为 "\\" 加上扩展后的 $home 的文件
```

另见 ‘= 。

---

7. 命令行窗口 cmdline-window cmdwin command-line-window

在命令行窗口里，可以像编辑文本一样来编辑命令。这是一类特殊的窗口，你不能把它当作普通窗口使用。
{仅当编译时加入 +cmdline_hist 特性才可用}

开 启 c_CTRL-F q: q/ q?

有两种方法可以打开命令行窗口：
1. 在命令行模式里，用 'cedit' 选项所指定的键。
   在没有置位 'compatible' 的情况下，默认值是 CTRL-F 。
2. 在普通模式里，用 "q:"，"q/" 或 "q?" 命令。
   它们分别开始编辑 Ex 命令 ("q:") 或搜索字符串 ("q/" 或 "q?")。 注意，处于记录状态时不能使用 ("q" 会结束记录)。

窗口里的内容是命令行的历史记录。最后一行是最近输入的命令。左面一列字符表示命令行的类型。见： cmdwin-char 。

如果没有置位 'insertmode' 选项，Vim 在打开之后会进入普通模式。

窗口的高度可以通过 'cmdwinheight' 选项设定 (如果没有足够的空间，窗口会缩小一点)。窗口永远都是以最大宽度出现，并且被放在命令行的上面 (紧靠命令行)。

编 辑

现在你可以在窗口里移动或者编辑里面的文本了。可以在普通模式和插入模式中进行这些操作。

可以用 ":"，"/" 和其他命令使用命令行，但却不能再打开另一个命令行窗口。这里不允许嵌套。

E11

命令行窗口不是普通的窗口。你不能移动到别的窗口或编辑别的缓冲区。在命令行窗口里，所有此类操作都被禁止。当然你 可以 执行在命令行窗口里输入的任何命令。关闭窗口时，其它的文本编辑都被舍弃。

关 闭 E199

有很多方法可以退出命令行窗口：

<CR>	执行光标下的命令。插入和普通模式里都可以。
CTRL-C	继续回到命令行模式。光标下的命令被用做命令行的命令。插入和普通模式里都可以。不会重画窗口，这样命令行窗口仍然可见。
:quit	放弃命令行，回到普通模式。也可以使用 ":close"、":exit"，":xit" 和 CTRL-\ CTRL-N 命令。

:qall　　除非改动了某些缓冲区，否则退出 Vim。
:qall!　　退出 Vim，放弃所做的改动。

一旦命令行窗口关闭，旧的窗口就会立刻恢复原先的大小。待执行的命令会作用于调用该命令行窗口或缓冲区，就好像没有没有用过命令行窗口一样（除了会有一次额外的屏幕刷新以外）。命令行窗口用过的缓冲区会被删除。对其中的命令所做的改动（如果没有用 <CR> 执行过）会全部丢失。

如果你想执行光标所在的命令后再次打开命令行窗口，可用以下映射：

```
:autocmd CmdwinEnter * map <buffer> <F5> <CR>q:
```

其 它

命令行窗口不能用在：
- 已经存在一个命令行窗口的时候（不能嵌套）
- 输入密钥或使用 inputsecret() 函数的时候

打开命令行窗口的时候，会自动设置下面的选项：
'filetype'　　当编辑 Ex 命令时，会设置为 "vim"；如果打开高亮的话，这样就会启用对 Vim 脚本的语法高亮。
'rightleft'　　关
'modifiable'　　开
'buftype'　　"nofile"
'swapfile'　　关

将命令窗口缓冲区的内容写到文件是允许的。这也是一个保存命令历史记录以便日后取回的好办法。

如果 'wildchar' 选项设置为 <Tab>，并且命令行窗口用于 Ex 命令，那么会在命令行窗口里加入两个映射用于命令补全，如下：

```
:imap <buffer> <Tab> <C-X><C-V>
:nmap <buffer> <Tab> a<C-X><C-V>
```

注意，在普通模式里敲 <Tab> 键会在下一个字符上补全。这样的话，就可以在行尾进行补全了。
如果你不想要这些映射，可用下面的命令将它们禁用：

```
au CmdwinEnter [:>] iunmap <Tab>
au CmdwinEnter [:>] nunmap <Tab>
```

你可以把这几行命令放到你的 vimrc 文件中。

在命令行窗口里，你不能用鼠标把光标放到另一个窗口里，或者拽动其它窗口的状态栏。你可以拽动命令行窗口自己的状态栏以及命令行窗口之上的状态栏来改变命令行窗口的大小，但对其它窗口无效。

getcmdwintype() 函数返回正在编辑的命令行的种类，其值见 cmdwin-char 。

自 动 命 令

使用两种自动命令事件： CmdwinEnter 和 CmdwinLeave 。你可以使用这些 Cmdwin 事件对命令行窗口进行特殊的设置。小心不要引起副作用！
例如：

```
:au CmdwinEnter :  let b:cpt_save = &cpt | set cpt=.
```

```
:au CmdwinLeave :  let &cpt = b:cpt_save
```
设置 'complete' 选项，以便在当前窗口用 i_CTRL-N 进行匹配。
另一个例子:
```
:au CmdwinEnter [/?]  startinsert
```
将使 Vim 在打开命令行窗口时切换到插入模式。

cmdwin-char

模式使用的下列字符用于指示命令行的种类:

:	普通的 Ex 命令
>	调试模式的命令 debug-mode
/	正向查找字符串
?	反向查找字符串
=	"= 表达式 expr-register
@	input() 字符串
-	:insert 或 :append 的文本

vim:tw=78:ts=8:noet:ft=help:norl:

options.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

选项 options

1. 设置选项 set-option
2. 自动设置选项 auto-setting
3. 选项总结 option-summary

选项的总览可见 quickref.txt option-list 。

Vim 有很多内部变量和开关，可以达到特殊的效果。这些选项有三种形式：

布尔型	可以打开或关闭	boolean toggle
数值型	值为数值	
字符串	值为字符串	

---

1. 设置选项 set-option E764

:se :set

:se[t]	显示所有不同于缺省值的选项。
:se[t] all	显示除了终端设置以外的所有选项。
:se[t] termcap	显示所有的终端选项。注意 在 GUI 里，不会显示键码，因为它们是内部生成的，无法改变。而且 GUI 里即使修改终端代码也没用 ...
:se[t] {option}?	显示 {option} 的值。 E518 E519
:se[t] {option}	切换选项：置位，打开。 数值选项：显示其值。 字符串选项：显示其值。
:se[t] no{option}	切换选项：复位，关闭。
:se[t] {option}! 或 :se[t] inv{option}	切换选项：反转其值。 :set-! :set-inv
:se[t] {option}& :se[t] {option}&vi :se[t] {option}&vim	复位选项为其缺省值。可能和 'compatible' 的现值有关。 复位选项为 Vi 的缺省值。 复位选项为 Vim 的缺省值。 :set-default :set-& :set-&vi :set-&vim
:se[t] all&	设置所有选项为其缺省值。以下选项的值不会改变： 所有 t_ 开始的终端选项 'columns'

'cryptmethod'
'encoding'
'key'
'lines'
'term'
'ttymouse'
'ttytype'
警告：这有很多副作用。

:set-args E487 E521

:se[t] {option}={value} 或
:se[t] {option}:{value}

设置字符串或数值选项的值为 {value}。
数值选项的值可以用十进制、十六进制（0x 开头）或八进制（'0' 开头）。
可以用 'wildchar' 输入原来的值（缺省是 <Tab>，但如果置位 'compatible'，是 CTRL-E）。见 cmdline-completion 。
{option} 和 '=' 之间可以有空白，但会被忽略。'=' 和 {value} 之间不能有空白。
option-backslash 说明如何在 {value} 里使用空白和反斜杠。

:se[t] {option}+={value} :set+=

把 {value} 加到数值选项里，或者附加到字符串选项之后。
如果选项是逗号分隔的列表，除非原来的值为空，会加上一个逗号。
如果选项是标志位的列表，删除多余的标志位。如果加入已经存在的标志位，选项值不变。
另见上面的 :set-args 。

:se[t] {option}^={value} :set^=

把 {value} 乘到数值选项里，或者附加到字符串选项之前。
如果选项是逗号分隔的列表，除非原来的值为空，会加上一个逗号。
另见上面的 :set-args 。

:se[t] {option}-={value} :set-=

把 {value} 从数值选项里减去，或者从字符串选项里删除，如果该值原来存在的话。如果不存在，不会有错误或者警告。
如果选项是逗号分隔的列表，除非新值为空，删除一个逗号。
如果选项是标志位的列表，{value} 必须和选项里出现的顺序完全相同。一个一个地分别删除标志位可以解决这个问题。
另见上面的 :set-args 。

":set" 可以设置多个 {option} 参数。例如：

```
:set ai nosi sw=3 ts=3
```

如果其中一个参数有错，给出错误信息，而且其后的参数也不再处理。

:set-verbose

如果 'verbose' 非零，显示选项值的同时会告诉你上次在哪里设置。例如：

```
:verbose set shiftwidth cindent?
  shiftwidth=4
        Last set from modeline line 1
  cindent
```

Last set from /usr/local/share/vim/vim60/ftplugin/c.vim line 30
只有请求特定选项的值时才会这么做，":verbose set all" 或者没有参数的
":verbose set" 都不会。
如果该选项是手动设置的，"Last set" 消息不会给出。
如果执行函数、用户命名或者自动命令时设定选项，报告的位置是定义它们的脚本。
注意 选项的设置也可能是来自设置 'compatible' 的副作用。
一些特殊的文字：

Last set from modeline line 1
选项最后在 modeline 里设置。
Last set from --cmd argument
选项最后由命令行参数 --cmd 或 + (译者注：应该不含 +) 设置。
Last set from -c argument
选项最后由命令行参数 -c 、+、 -S 或 -q 设置。
Last set from environment variable
选项最后由环境变量、$VIMINIT、$GVIMINIT 或 $EXINIT 设置。
Last set from error handler
选项计算时出错，因而被清除。

{仅当编译时加入 +eval 特性才有效}

:set-termcap E522
需要 {option} 的地方，可以使用 "t_xx" 形式来设置终端选项。这些选项覆盖相应的
termcap 值。设置后，可以用于映射。如果 "xx" 包含特殊字符，须用 <t_xx> 形式：

```
:set <t_#4>=^[Ot
```

也可用来翻译普通键的特殊键码。例如，如果 Alt-b 产生 <Esc>b，可用：

```
:set <M-b>=^[b
```

(这里 ^[是真正的 <Esc>，用 CTRL-V <Esc> 来输入)
这个方法优于映射之处在于它能适用于所有情况。

你可以定义任何键码，如：

```
:set t_xy=^[foo;
```

如果使用不能识别的名字，没有警告。你可以随便定义映射：

```
:map <t_xy> something
```

E846
还没有设置的键码会被当作不存在。试图得到其值会报错：

```
:set t_kb=
:set t_kb
E846: Key code not set: t_kb
```

为了安全原因，t_xx 选项不能在 modeline 或者 sandbox 里设置。

":set" 给出的列表看起来和 Vi 不同。长字符串选项在列表底部。而选项的数量也相当
多。"set all" 的输出可能一整屏都放不下，此时 Vim 会给出 more-prompt 。

option-backslash
在字符串选项值中，如果要包含空白，一定要用反斜杠前导。如果要包含单个反斜杠，一
定要输入两个。换而言之，选项值里的反斜杠数目会减半 (往下取整)。
这里有一些例子：

```
:set tags=tags\ /usr/tags       产生 "tags /usr/tags"
:set tags=tags\\,file           产生 "tags\,file"
:set tags=tags\\\ file          产生 "tags\ file"
```

"|" 字符把 ":set" 命令和下一个命令隔开。要在选项值里包含 "|"，用 "\|" 代替。
下例把 'titlestring' 选项设为 "hi|there":

```
:set titlestring=hi\|there
```
而要把 'titlestring' 选项设为 "hi" 并把 'iconstring' 设为 "there":
```
:set titlestring=hi|set iconstring=there
```

双引号字符开始注释。类似地，要在选项值里包含 '"'，用 '\"' 代替。下例设置 'titlestring' 选项为 'hi "there"':
```
:set titlestring=hi\ \"there\"
```

MS-DOS 和 WIN32 基本上不删除文件名里的反斜杠。更准确地说：对于期待文件名的选项而言（那些需要扩展环境变量的），不删除普通文件名字符之前的反斜杠。但特殊字符之前的反斜杠（空格、反斜杠、逗号等）的使用仍如上所述。
有一个特例，如果该值以 "\\" 开始:
```
:set dir=\\machine\path          产生 "\\machine\path"
:set dir=\\\\machine\\path       产生 "\\machine\path"
:set dir=\\path\\file            产生 "\\path\file" (错了！)
```
第一个例子的开始部分被保持，但第二个的反斜杠被减半。这样可以确保在你期望反斜杠减半和反斜杠保持原样的两种情况都能工作。第三个的结果也许不是你想要的。避免这种用法。

add-option-flags remove-option-flags
E539 E550 E551 E552

有些选项是标志位的列表。如果你想给这样的选项加入标志位而不影响已有的，可以:
```
:set guioptions+=a
```
从选项里删除标志位可以用:
```
:set guioptions-=a
```
这样就删除 'guioptions' 里的 'a' 标志位。
注意 一次只应该只加减一个标志位。如果 'guioptions' 的值是 "ab"，不能用 "set guioptions-=ba"，因为字符串 "ba" 没有出现。

:set_env expand-env expand-environment-var

若干字符串选项扩展环境变量。如果某环境变量存在，'$' 后跟该环境变量的名字被替换为它的值。如果不存在，'$' 和该名字不会被修改。任何非标识符的字符（不是字母，数字或 '_'）都可以跟在环境变量名字的后面。该字符和其后的内容附加于环境变量的值之后。比如:
```
:set term=$TERM.new
:set path=/usr/$INCLUDE,$HOME/include,.
```
如果用 ":set opt-=val" 或 ":set opt+=val" 从选项里加减字符串，该扩展在加入或删除操作之前完成。

局部选项的处理 local-options

有的选项只适用于单个窗口或缓冲区。每个窗口或缓冲区有该选项的单独版本，因而分别拥有不同的值。这样便可以在一个窗口置位 'list'，而另一个不置位。又或者在一个缓冲区设置 'shiftwidth' 为 3 而另一个设置为 4。

下面解释在一些特定情形下局部选项如何进行操作。其实，你无需了解所有的细节，因为 Vim 基本上依照你的想法使用这些选项值。不幸的是，要满足用户的要求并不简单 ...

分割窗口时，局部选项复制到新窗口。因而分割后，两个窗口的内容看起来一模一样。

编辑新缓冲区时需要初始化局部选项值。因为当前缓冲区的局部选项可能为它自己度身定制，我们不能使用这些设置。相反，每个缓冲区局部选项都存在一个全局值，它们被用于新缓冲区。":set" 同时修改局部和全局值。"setlocal" 只修改局部值而不影响将来编辑

新缓冲区时选项的设置。

如果要编辑的缓冲区过去曾经编辑过，重新应用它最近关闭的窗口选项：如果该缓冲区曾经在本窗口编辑过，使用本窗口当时的值。否则使用该缓冲区上次编辑所在的最后关闭的窗口用过的值。

一种类型的缓冲区可能有专门的窗口局部选项。同一个窗口要切换编辑的缓冲区时，你可能因而不希望维持这些局部选项。为此，Vim 维护窗口局部选项的全局值，切换编辑的缓冲区时使用全局值。每个窗口都有全局值的独立版本。因而这些值局部于窗口，但全局于该窗口的所有缓冲区。这样，你可以用：

```
:e one
:set list
:e two
```

现在 'list' 选项也会在 "two" 里置位，因为 ":set list" 命令同时置位全局值。

```
:set nolist
:e one
:setlocal list
:e two
```

现在 'list' 选项不被置位，因为 ":set nolist" 复位全局值，而 ":setlocal list" 只改变局部值，但 ":e two" 又使用全局值。注意 如果你接下来这么做：

```
:e one
```

你会得到上次编辑 "one" 时的 'list' 值。因为每个缓冲区会分别记住局部于窗口的选项。这也适用于未载入的缓冲区，但真正删除缓冲区 :bwipe 时此信息就丢失了。

:setl :setlocal

:setl[ocal] ... 类似于 ":set"，但只设置局部于当前缓冲区或者窗口的值。不是所有的选项都有局部值的。如果该选项没有局部值，那么那么设置全局值。
如果用 "all" 参数：显示所有局部选项的局部值。
如果没有参数：显示所有不同于缺省的局部选项的局部值。
显示局部选项时，显示它的局部值。对全局/局部布尔变量，如果使用了全局值，在选项名前会显示 "--"。
全局选项则显示全局值 (将来或许会有所改变)。

:setl[ocal] {option}< 通过复制全局值，把 {option} 的局部值设回它的全局值。

:se[t] {option}< 对 global-local 选项：撤销 {option} 的局部值，从而使用它的全局值。

:setg :setglobal

:setg[lobal] ... 类似于 ":set"，但只设置局部选项的全局值，而不改变其局部值。
显示选项时，显示它的全局值。
如果用 "all" 参数：显示所有局部选项的全局值。
如果没有参数：显示所有不同于缺省的局部选项的全局值。

对缓冲区和窗口局部选项而言：

命令	全局值	局部值
:set option=value	设置	设置
:setlocal option=value	-	设置
:setglobal option=value	设置	-
:set option?	-	显示
:setlocal option?	-	显示
:setglobal option?	显示	-

有局部值的全局选项 global-local

选项之所以定义为全局类型，是因为多数情况下它们在所有缓冲区和窗口里只需设置一个值。但对有些全局选项而言，定义不同的局部值可能有用。你可以使用 ":setlocal" 来设置这些全局选项的局部值。此时，该缓冲区或窗口使用局部值，而其它缓冲区和窗口继续使用全局值。

例如，你有两个窗口，都在编辑 C 源代码，也都使用全局的 'makeprg' 选项。如果其中一个这么做：

```
:set makeprg=gmake
```

那么另一个窗口也会切换到新值。无需再为那个 C 源代码窗口重复设置一遍 'makeprg' 选项。
不过，如果你在新窗口里编辑 Perl 脚本而想让它使用别的 'makeprg'，但你又不想改变 C 源代码使用的值。可用这个命令：

```
:setlocal makeprg=perlmake
```

只要把局部值设为空，你就可以切换回全局值：

```
:setlocal makeprg=
```

这只适用于字符串选项。数值或布尔型选项需要使用 "<" 标志：

```
:setlocal autoread<
```

要 注意，对非布尔型和非数值的选项使用 "<" 会把全局值复制给局部值，而不是切换到直接使用全局值的情形（如果后来又改变全局值，就能看出两者的区别了）。也可用：

```
:set path<
```

这样做会使 'path' 的局部值为空并从而使用它的全局值。它和下行的效果相同：

```
:setlocal path=
```

注意：更多的全局选项将来可能会成为全局-局部类型。那时，在全局选项上使用 ":setlocal" 的效果能会有不同。

设置文件类型

:setf[iletype] [FALLBACK] {filetype} :setf :setfiletype
把 'filetype' 选项设为 {filetype}。但如果已经在（嵌套）自动命令序列中设置过，就不再进行。
等价于下面代码的缩写：

```
:if !did_filetype()
:  setlocal filetype={filetype}
:endif
```

在一个 filetype.vim 里使用该命令可避免设置 'filetype' 选项两次，导致不同的设置和语法文件被载入。

如果给出可选的 FALLBACK 参数，之后的 :setfiletype 命令可覆盖 'filetype'。这用于仅仅是猜测的文件类型检测。此命令后 did_filetype() 为假。

option-window optwin
:bro[wse] se[t] :set-browse :browse-set :opt :options
:opt[ions]
打开窗口，阅读和设置所有的选项。选项以功能分组。
提供每个选项简短的帮助。在简短帮助上按 <CR>，会打开帮助窗口来提供该选项更多的帮助。
修改选项的值，然后在 "set" 行上按 <CR> 就会设置新值。

对于窗口和缓冲区特定的选项，设置的是最近访问窗口里的选项值。除非它是帮助窗口，这时使用帮助窗口下方的窗口（选项窗口本身不算）。
{仅当编译时加入 +eval 特性才有效}

$HOME
使用 "~" 如同使用 "$HOME"，但只有在选项开头和空格或逗号之后才会识别。

Unix 系统上也可以用 "~user"。它被用户 "user" 的主目录代替。例如：
```
:set path=~mool/include,/usr/include,.
```

Unix 系统上还可以用 "${HOME}" 形式。这时，{} 之间的名字可以包含非标识符字符。
注意 如果你想为 "gf" 命令使用这种形式，'{' 和 '}' 字符需要加到 'isfname' 里。

注意：环境变量和 "~/" 的扩展只在 ":set" 命令里进行，给选项赋值的 ":let" 不会。

$HOME-windows
MS-Windows 上，如果没定义 $HOME 环境变量，运行时 Vim 把它设为 $HOMEDRIVE$HOMEPATH 扩展后的值。
如果 $HOMEDRIVE 无定义则使用 $USERPROFILE。

此扩展后的值不导回到环境中，运行外部命令时就看这里的区别：
```
:echo system('set | findstr ^HOME=')
```
和
```
:echo luaeval('os.getenv("HOME")')
```
应该什么都不返回（空串），尽管 exists('$HOME') 返回真值。如果 $HOME 设置为非空字符串，它应会被导出到子进程里。

注意 扩展后的选项的最大长度有限制。具体多少与系统有关，一般是 256 或 1024 个字符这样的数目。

:fix :fixdel
:fix[del] 设置 't_kD' 的值，如果：

't_kb' 是	't_kD' 就变成
CTRL-?	CTRL-H
非 CTRL-?	CTRL-?

（CTRL-? 是 0177 八进制、0x7f 十六进制）

如果删除键的终端代码不对，但退格键的代码是对的，在你的 .vimrc 里放上：
```
:fixdel
```
不管退格键的实际代码是什么，它都能工作。

如果退格键的终端代码不对，可以用：
```
:if &term == "termname"
:  set t_kb=^V<BS>
:  fixdel
:endif
```
这里 "^V" 是 CTRL-V 而 "<BS>" 是退格键（不要输入四个字符！）。把 "termname" 换成你的终端名字。

如果 <Delete> 键发送一个奇怪的键序列（不是 CTRL-? 或

CTRL-H)，不要用 ":fixdel"，而应该：

```
:if &term == "termname"
:  set t_kD=^V<Delete>
:endif
```

这里 "^V" 是 CTRL-V 而 "<Delete>" 是删除键（不要输入八个字符！）。把 "termname" 换成你的终端名字。

Linux-backspace

Linux 的 备注：退格键缺省产生 CTRL-?，这是错的。在 rc.local 里放上这行可以修正：

```
echo "keycode 14 = BackSpace" | loadkeys
```

NetBSD-backspace

NetBSD 的 备注：如果退格键产生错误的键码，可试用：

```
xmodmap -e "keycode 22 = BackSpace"
```

如果可以，在 .Xmodmap 文件里加入：

```
keysym 22 = BackSpace
```

要使之生效，你需要重启。

2．自动设置选项 auto-setting

除了用 ":set" 命令设置选项以外，还有三个方法可以自动设置一个或多个文件的选项：

1．启动 Vim 时，在若干地方可以进行初始化，见 initialization 。多适用于所有编辑会话。有些则取决于 Vim 启动时所在目录。你可以用 :mkvimrc 、 :mkview 和 :mksession 创建初始化设置。
2．开始编辑新文件时，会执行自动命令。这里可以为匹配一定模式的文件设置选项和做其它的事，见 autocommand 。
3．开始编辑新文件并且打开 'modeline' 选项时，在文件开始和结束处的一些行上检查模式行。下面解释此机制。

modeline vim: vi: ex: E520

有两种模式行。第一种形式：

```
[text{white}]{vi: vim: ex:}[white]{options}
```

[text{white}]	空，或任何后跟不少于一个的空白字符（<Space> 或 <Tab>）的文本；"ex:" 总是需要至少一个空白字符
{vi: vim: ex:}	字符串 "vi:"、"vim:" 或 "ex:"
[white]	可选的空白字符
{options}	选项设置的列表，用空格或 ':' 分隔。每个 ':' 之间的部分成为一个 ":set" 命令的参数（可为空）

例如：

```
vi:noai:sw=3 ts=6
vim: tw=77
```

第二种形式（和部分版本的 Vi 兼容）：

```
[text]{white}{vi: vim: Vim:|ex:}[white]se[t] {options}:[text]
```

[text]	任何文本，可以为空

{white}	不少于一个的空白字符（<Space> 或 <Tab>）
{vi: vim: Vim:|ex:}	字符串 "vi:"、"vim:"、"Vim:" 或 "ex:"
[white]	可选的空白字符
se[t]	字符串 "set " 或 "se "（注意 空格）；使用 "Vim" 时必须是 "set"。
{options}	选项列表，以空白分隔。每个部分成为一个 ":set" 命令的参数
:	冒号
[text]	任何文本，可以为空

例如：

```
/* vim: set ai tw=75: */
/* Vim: set ai tw=75: */
```

{vi: vim: Vim:|ex:} 之前的空格是必要的。这样可以减少错用比如 "lex:" 这样的普通单词的可能性。有一个特例："vi:" 和 "vim:" 也可以出现在行首（为和 3.0 版本兼容）。在行首使用 "ex:" 会被忽略（这可以是 "example:" 的缩写）。

modeline-local

选项的设置类似于 ":setlocal"：新值只适用于包含该文件的缓冲区和窗口。尽管可以从模式行里设置全局值，这非常罕见。如果你打开两个窗口，而其中的两个文件为同一个全局值设置不同的值，结果取决于哪个文件后打开。

编辑已经载入的文件时，只使用模式行里局部于窗口的选项。因而，如果打开文件后你手动修改局部于缓冲区的选项，在别的窗口里编辑相同的缓冲区不会改变这些选项。但局部于窗口的选项会被设置。

modeline-version

如果模式行只用于某些版本的 Vim，版本号应在使用 "vim:" 或 "Vim:" 时指定：

vim{vers}:	{vers} 版本或更高
vim<{vers}:	{vers} 之前的版本
vim={vers}:	{vers} 版本
vim>{vers}:	{vers} 之后的版本

对于 Vim 7.0，{vers} 是 700（主版本号乘一百加副版本号）。
例如，要设置 Vim 7.0 或以后版本使用的模式行：

```
/* vim700: set foldmethod=marker */
```

要为 Vim 7.2 以前的版本设置模式行：

```
/* vim>702: set cole=2: */
```

"vim" 和 ":" 之间不能有空格。

检查的行数用 'modelines' 选项设置。如果关闭 'modeline' 或者 'modelines' 为零，不检查模式行。

注意 第一种形式使用该行的其余部分。所以这行：

```
/* vi:ts=4: */
```

会指出拖尾的 "*/" 是一个错误。这样就可以：

```
/* vi:set ts=4: */
```

如果检测到错误，忽略该行的其余部分。

如果你想在 set 命令里包含 ':'，在冒号之前加上 '\'。处理时，':' 之前的反斜杠会被删除。例如：

```
/* vi:set fillchars=stl\:^,vert\:\|: */
```

会设置 'fillchars' 选项为 "stl:^,vert:\|"。':' 之前只删除一个反斜杠。因此，要包含 "\:" 你需要指定 "\\:"。

E992

为了安全原因，除了 "set" 以外，不支持别的命令（有人可能会用模式行创建一个文本的特洛伊木马文件）。而且也不是所有选项都可设置。对有些选项而言，会设置一个标志位来激活 sandbox 。有的选项只能在 'modelineexpr' 置位时（缺省是关闭）才能从模式行设置。

即便如此，使用模式行还是有些微风险。例如，如果有些开玩笑的人设置 'textwidth' 为 5，你的所有行都会意想不到地被回绕。因此，在编辑一些不信任的文本时不妨关闭模式行。例如，mail 文件类型插件就这么做。

提示：除了设置选项以外，如果你需要做别的事情，可以定义自动命令并检查文件是否包含特定字符串。例如：

```
au BufReadPost * if getline(1) =~ "VAR" | call SetVar() | endif
```

并且定义函数 SetVar()，为包含 "VAR" 行的文件做一些事。

---

3. 选项总结 option-summary

下表给出所有选项，包括全名和，如果有的话，它们的缩写。可任意使用两种形式。

本文档里，布尔型选项的 "置位" 意味着输入 ":set option"。选项的 "复位" 意味着使用 ":set nooption"。

有些选项有两个缺省值："Vim 缺省" 在没有置位 'compatible' 时使用，而 "Vi 缺省" 在置位 'compatible' 时使用。

多数选项适用于所有的窗口和缓冲区。有一些专用于设置窗口里如何显示文本。每个窗口可以为它们设置不同的值。比如 'list' 选项，可以在一个窗口置位而在显示相同文本的另一个窗口复位，这样你就可以同时看到两种视图。有一些选项专用于特定的文件。它们可以为每个文件或者缓冲区设置不同的值。比如 'textwidth' 选项可以在普通的文本文件里设为 78，而在 C 程序里设为 0。

全局	所有缓冲区和窗口使用同一个选项值
局部于窗口	每个窗口有自己的选项值
局部于缓冲区	每个缓冲区有自己的选项值

如果创建一个新窗口，使用当前活动的窗口的选项值作为窗口专用选项的缺省值。缓冲区专用的选项取决于 'cpoptions' 选项的 's' 和 'S' 标志位。如果包含 's'（缺省），那么缓冲区在第一次进入时，从当前活动的缓冲区复制选项值。如果包含 'S'，每次进入缓冲区都会复制选项的值。这样，和全局选项几乎没有区别。如果 's' 和 'S' 都不存在，缓冲区在建立时从当前活动的缓冲区复制选项。

隐藏选项 hidden-options

不是所有版本都支持所有的选项。这取决于哪些特性得到支持，有时和系统也有关。下面在花括号里提供这方面的评注。一个选项即使不支持也可以进行设置，这时并不会报错。这些选项称为隐藏选项。不过，你无法取回隐藏选项的值，因为设置的值不会被保存。

要测试选项 "foo" 是否可用 ":set" 设置：

```
if exists('&foo')
```

这里，隐藏选项也返回真。要测试是否真的支持选项 "foo"：

```
if exists('+foo')
```

E355

Q_op 提供包含简短描述的选项列表，可用它进行跳转。

'aleph' 'al' 'aleph' 'al' aleph Aleph

'aleph' 'al' 数值型 (MS-DOS 的缺省为 128，否则为 224)
全局
{仅当编译时加入 +rightleft 特性才有效}
希伯来字母表的第一个字母的 ASCII 码。希伯来模式下的键盘映射例程，包括插入模式（如果置位 'hkmap'）和命令行模式（如果按了 CTRL-_），都使用 [aleph..aleph+26] 范围输出希伯来字符。
aleph=128 适用于 PC 代码，而 aleph=224 适用于 ISO 8859-8。
见 rileft.txt 。

'allowrevins' 'ari' 'noallowrevins' 'noari'

'allowrevins' 'ari' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +rightleft 特性才有效}
允许插入和命令行模式使用 CTRL-_。缺省是关闭的，以免用户想输入 SHIFT-_ 时不小心输入 CTRL-_ 而进入反向插入模式又不知道怎么退出。见 'revins'。
注意：如果置位 'compatible'，本选项被复位。

'altkeymap' 'akm' 'noaltkeymap' 'noakm'

'altkeymap' 'akm' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +farsi 特性才有效}
此选项曾用于支持波斯语，此支持已删除。见 farsi.txt 。

'ambiwidth' 'ambw'

'ambiwidth' 'ambw' 字符串 (缺省: "single")
全局
只有在 'encoding' 为 "utf-8" 或别的 Unicode 编码时才有效。告诉 Vim 怎么处理东亚二义性宽度字符类 (East Asian Width Class Ambiguous) (例如欧元符号、注册记号、版权记号、希腊字母、西里尔字母等等)。

目前有两个可能的选择:
"single": 使用和 US-ASCII 字符相同的宽度。多数用户希望如此。
"double": 使用 US-ASCII 字符两倍的宽度。
E834 E835
如果 'listchars' 或 'fillchars' 包含会是双倍宽度的字符，不能使用 "double" 值。

在一些 CJK 字体里，这些字符的字形宽度完全由它们在传统的 CJK 编码里占据字节的数目决定。那些编码中，欧元、注册记号、希腊/西里尔字母等占据两个字节，因而它们在这些字体里用 "宽" 字形显示。这也包括文本文件里制表用的一些画线字符。因此如果 GUI Vim 使用 CJK 字体、或者在使用 CJK 字体的终端 (模拟器) (包括带有 "-cjkwidth" 选项的 xterm) 里运行 Vim，应把本选项设为 "double"，这样可以匹配这些字体里 Vim 实际看到相关字形的宽度。 CJK MS-Windows 2k/XP 上，如果系统 locale 为 CJK locale，也应把本选项设为 "double"。见 Unicode Standard Annex #11 (http://www.unicode.org/reports/tr11)。

如果编译时加入 +termresponse 特性并且 t_u7 设为获取光标位置报告的转义序列，Vim 可以在启动时自动设置此选项。该报告的反馈值可从

v:termu7resp 中找到。

'antialias' 'anti' 'noantialias' 'noanti'
'antialias' 'anti' 布尔型（缺省：关闭）
全局
{仅当编译时加入 Mac OS X 的 GUI 支持才有效}
只有在 Mac OS X v10.2 或以后版本的 Vim 的 GUI 版本上，本选项才有效。如果打开，Vim 使用平滑（"反锯齿"）字体。在特定显示上的特定大小的字体，可能会较易阅读。有时，如果 'guifont' 设为缺省值（空字符串），置位本选项会引起问题。
注意：如果置位 'compatible'，本选项被复位。

'autochdir' 'acd' 'noautochdir' 'noacd'
'autochdir' 'acd' 布尔型（缺省关闭）
全局
{仅当编译时加入此特性才有效，用 exists("+autochdir") 可以检查}
如果打开，Vim 会在你打开文件、切换缓冲区、删除缓冲区或者打开/关闭窗口时改变当前工作目录的值。具体来说，就是打开或者选择的文件所在的目录。
注意：打开此选项会使得某些插件无法工作。

'arabic' 'arab' 'noarabic' 'noarab'
'arabic' 'arab' 布尔型（缺省关闭）
局部于窗口
{仅当编译时加入 +arabic 特性才有效}
可以置位本选项来开始编辑阿拉伯文本。
置位本选项会：
- 置位 'rightleft' 选项，除非置位 'termbidi'。
- 置位 'arabicshape' 选项，除非置位 'termbidi'。
- 设置 'keymap' 选项为 "arabic"；插入模式下 CTRL-^ 会切换英语和阿拉伯键盘映射。
- 置位 'delcombine' 选项。
注意 要使用阿拉伯文本，'encoding' 必须是 "utf-8"。

复位本选项会：
- 复位 'rightleft' 选项。
- 关闭 'keymap' 的使用（但不改变其值）。
注意 这里不复位 'arabicshape' 和 'delcombine'（这些是全局选项）。
注意：如果置位 'compatible'，本选项被复位。
另见 arabic.txt 。

'arabicshape' 'arshape'
'noarabicshape' 'noarshape'
'arabicshape' 'arshape' 布尔型（缺省打开）
全局
{仅当编译时加入 +arabic 特性才有效}
如果打开且 'termbidi' 关闭，启动必须的可见字符的校正，以正确显示阿拉伯语言。本质上它打开了字型重整；这是一个广义的术语，它包括：
a) 根据词内位置（开头、中间、结尾和单独出现）改变/调整字符的形状。
b) 启动组合字符的能力
c) 启动一些字符所需的组合
如果关闭，屏幕用每个字符原来单独显示的方式显示。
阿拉伯语是一个复杂的语言，还需要其他的设置。详见 arabic.txt 。

注意：如果置位 'compatible'，本选项被复位。

'autoindent' 'ai' 'noautoindent' 'noai'
'autoindent' 'ai' 布尔型 (缺省关闭)
局部于缓冲区
开启新行时 (插入模式下输入 <CR>，或者使用 "o" 或 "O" 命令)，从当前行复制缩进距离。如果你在新行除了 <BS> 或 CTRL-D 以外不输入任何东西，然后输入 <Esc>、CTRL-O 或 <CR>，缩进又被删除。移动光标到其它行也有同样的效果，除非 'cpoptions' 里包含 'I' 标志位。
如果打开自动缩进，排版 (用 "gq" 命令或者插入模式下到达了 'textwidth') 使用第一行的缩进距离。
打开 'smartindent' 或 'cindent' 时，缩进的修改方式有所不同。
置位 'paste' 选项时，'autoindent' 选项被复位。复位 'paste' 时，恢复此选项。

'autoread' 'ar' 'noautoread' 'noar'
'autoread' 'ar' 布尔型 (缺省关闭)
全局或局部于缓冲区 global-local
如果发现文件在 Vim 之外修改过而在 Vim 里面没有的话，自动重新读入。
如果文件在外部被删除，不会这么做，这样你还有删除前的文本内容。再次发生的话就会真去读了。 timestamp
如果本选项有局部值，使用下面的命令切换回直接使用全局值：
:set autoread<

'autowrite' 'aw' 'noautowrite' 'noaw'
'autowrite' 'aw' 布尔型 (缺省关闭)
全局
自动把内容写回文件：如果文件被修改过，在每个 :next、:rewind、:last、:first、:previous、:stop、:suspend、:tag、:!、:make、CTRL-] 和 CTRL-^ 命令时进行；用 :buffer、CTRL-O、CTRL-I、'{A-Z0-9} 或 `{A-Z0-9} 命令转到别的文件时亦然。
注意 有些命令不使用 'autowrite' 选项。为此目的，可用 'autowriteall'。
有的缓冲区不能写入，具体地说，当 'buftype' 为 "nowrite"、"nofile"、"terminal" 或 "prompt" 时。

'autowriteall' 'awa' 'noautowriteall' 'noawa'
'autowriteall' 'awa' 布尔型 (缺省关闭)
全局
和 'autowrite' 类似，但也适用于 ":edit"、":enew"、":quit"、":qall"、":exit"、":xit"、":recover" 和关闭 Vim 窗口。
置位本选项也意味着 Vim 的行为就像打开 'autowrite' 一样。

'background' 'bg'
'background' 'bg' 字符串 (缺省为 "dark" 或 "light"，见下)
全局
设为 "dark" 时，Vim 试图使用深色背景上看起来舒服的颜色。如果设为 "light"，Vim 会试图使用在浅色背景上看起来舒服的颜色。其它的值都是非法的。Vim 试图根据你使用的终端确定其缺省值，但不见得总做的对。
设置本选项不改变背景色，它告诉 Vim 背景颜色应该看起来怎么样。要改变背景色，见 :hi-normal 。

如果设置 'background'，Vim 会根据新值调整缺省的颜色组。但语法高亮使用的颜色不会改变。 g:colors_name
已经载入色彩方案时 ("g:colors_name" 变量已设置)，设置 'background' 会

重新载入色彩方案。如果色彩方案根据 'background' 调整，这不错。但如果颜色方案自己设置 'background'，实际效果可能就被撤销了。如果需要的话，先删除 "g:colors_name" 变量。

如果这样复原 'background' 为其缺省值：

```
:set background&
```

Vim 会猜测其值。GUI 里应该没有问题，其它情况下 Vim 可能不一定能猜对。如果 GUI 支持深色主题，可以在 'guioptions' 中使用 "d" 标志位，见 'go-d'。

设置 t_RB 选项时，Vim 使用它从终端请求背景色。如果返回的 RGB 色是深/浅而 'background' 不是深/浅时，设置 'background' 并重画屏幕。可能有副作用，如果怀疑有问题，在 .vimrc 里复位 t_BG。 t_RB 的反馈值可从 v:termrbgresp 中获取。

启动 GUI 时，'background' 的缺省值是 "light"。如果 .gvimrc 里没有设置该值，而 Vim 检测到背景实际上很深，'background' 会设为 "dark"。但这在 .gvimrc 读入 之后 才发生 (因为窗口需要被打开才能发现实际的背景色)。要避免这一点，在 .gvimrc 里放上 ":gui" 命令以强迫打开 GUI 窗口。它应在使用 'background' 的值的命令之前 (比如，在 ":syntax on" 之前)。

MS-Windows 缺省值是 "dark"。
其他系统下，当 'term' 为 "linux"、"screen.linux"、"cygwin" 或 "putty" 时，或当 $COLORFGBG 建议使用深色时，使用 "dark"，否则为 "light"。

:terminal 命令和 term_start() 函数使用 'background' 值来决定终端窗口启动时是白色还是黑色背景。

通常，本选项应在 .vimrc 文件里设置。可以依赖于终端名。比如：

```
:if &term == "pcterm"
:  set background=dark
:endif
```

如果设置本选项，高亮组的缺省设置会发生改变。要进行其它设置，在设置 'background' 选项 之后 才使用 ":highlight" 命令。
本选项也在 "$VIMRUNTIME/syntax/syntax.vim" 文件里使用，用来选择语法高亮的颜色。在改变此选项后，你必须再次载入 syntax.vim 才能看到效果。这可以通过 ":syntax on" 完成。

'backspace' 'bs'
'backspace' 'bs'　　字符串 (缺省 ""， defaults.vim 里设为 "indent,eol,start")
全局

影响 <BS>、<Del>、CTRL-W 和 CTRL-U 在插入模式下的工作方式。它是逗号分隔的项目列表。每个项目允许一种退格删除的内容：

值	效果
indent	允许在自动缩进上退格
eol	允许在换行符上退格 (连接行)
start	允许在插入开始的位置上退格；CTRL-W 和 CTRL-U 到达插入开始的位置时停留一次。

如果该值为空，使用 Vi 兼容的退格方式。

为了和 5.4 及更早的版本后向兼容：

值	效果

0 等同于 ":set backspace=" (Vi 兼容)
1 等同于 ":set backspace=indent,eol"
2 等同于 ":set backspace=indent,eol,start"

如果你的 <BS> 或 <Del> 键不合你的期望，见 :fixdel 。
注意：如果置位 'compatible'，本选项被设为 ""。

'backup' 'bk' 'nobackup' 'nobk'
'backup' 'bk' 布尔型 (缺省关闭)
全局
覆盖文件前创建一个备份。文件成功写入后保留该备份。如果你不想保留备份文件，但希望写入期间能有备份，复位本选项，置位 'writebackup' 选项 (这是缺省行为)。如果你完全不想要备份文件，同时复位两个选项 (如果你的文件系统差不多满了，这会有用)。更多的解释可见 backup-table 。
如果匹配 'backupskip' 模式，无论如何都不会建立备份。
如果设置 'patchmode'，备份文件会换名成为文件的旧版本。
注意：如果置位 'compatible'，本选项被复位。

'backupcopy' 'bkc'
'backupcopy' 'bkc' 字符串 (Vi 在 Unix 上的缺省："yes"，否则："auto")
全局或局部于缓冲区 global-local
写回文件并建立备份时，本选项指定它应如何完成。这是逗号分隔的单词列表。

主要的值是：
"yes" 复制文件，然后覆盖原来的文件
"no" 给文件换名，然后写入一个新文件
"auto" 两者哪个更好就用哪个

可以和上面的一起使用的附加设置包括：
"breaksymlink" 写入时总是断开符号链接
"breakhardlink" 写入时总是断开硬链接

复制文件并覆盖原来的文件：
- 需要额外的时间复制文件。
+ 如果文件有特殊属性，是 (硬 / 符号) 链接或者有资源分叉 (resource fork)，都能保存下来。
- 如果文件是一个链接，备份文件名使用链接的名字，而不是实际文件的。

给文件换名并写入新文件：
+ 很快。
- 有时不是所有的文件属性都能被复制到新文件里。
- 如果文件是链接，新文件将不再是链接。

"auto" 值是一个折衷：如果 Vim 看到文件换名可以没有副作用 (属性可以被传递而文件也不是链接)，就使用换名方式。如果遇到问题，就使用复制方式。

"breaksymlink" 和 "breakhardlink" 值可以和 "yes"、"no" 以及 "auto" 中的任何一个混合使用。如果包含，它们迫使 Vim 总是使用 "no" 选项完全相同的步骤断开符号链接或者硬链接：给原来的文件换名并成为备份文件，在原来文件的位置写入新的文件内容。这可以用于，比如说，所有的文件或者是符号链接或者是硬链接的源代码树。这样，任何的改变都应该会保留在本地源代码树中，而不影响原来的代码。

crontab
一种情形下 "no" 和 "auto" 可能会有问题：一个程序打开文件、启用 Vim 编

辑该文件、然后测试已打开的文件是否发生改变（通过文件描述符）。该测试总是会检查备份文件而不是新建立的文件。"crontab -e" 就是这样一个例子。

用复制方式时，原来的文件会被清空，然后填入新的文本。这意味着保护位、拥有者和原来文件的符号链接都保持不变。但备份文件是一个新的文件，拥有者是编辑文件的人，所在的组是原来文件所在的组。如果这不行，组的保护位设得和其它人（非用户和组成员）的保护位相同。

如果文件被换名，事情就刚好相反：备份文件有着和原来文件相同的属性，而新写的文件由当前用户拥有。如果文件曾是（硬/符号）链接，新文件不再是了！这就是为什么 "auto" 值不为链接文件换名的原因。新写入文件的拥有者和所属组会设得和原来的文件相同。但系统可能拒绝这么做，这种情形下，"auto" 值仍然会选择不用换名方式。

注意：'compatible' 置位时此选项设为 Vi 缺省值，'compatible' 复位时此选项设为 Vim 缺省值。

'backupdir' 'bdir' 'backupdir' 'bdir'　　字符串　(Amiga 的缺省：".,t:",
MS-DOS 和 Win32: ".,$TEMP,c:/tmp,c:/temp"
Unix: ".,~/tmp,~/")
全局

逗号分隔的备份文件的目录名列表。

- 备份文件会在第一个可能的目录里建立。该目录必须已经存在，Vim 不会帮你建立该目录。
- 空意味着不会建立备份文件（'patchmode' 是不会工作的！）。写入也可能因此而失败。
- 目录 "." 意味着把备份文件放在被编辑文件的相同目录里。
- "./"（或 MS-DOS 等环境的 ".\"）开始的目录意味着把备份文件放在被编辑文件所在的相对位置。开头的 "." 由被编辑文件的路径名替换。
  （目录中间的 "." 没有特殊含义）。
- 逗号之后的空格被忽略，其它的空格视为目录名的一部分。要在目录命令开头包含空格，在它之前加入反斜杠。
- 要在目录名包含逗号，在它之前加入反斜杠。
- 目录名可以用 '/' 结尾。
- Unix 和 Win32 上，如果目录以两个路径分隔符 "//" 结尾，备份文件名会由文件的完整路径把所有的路径分隔符替换为百分号 '%' 来构造。此方法保证了备份目录中文件名的唯一性。
  Win32 上，也可以用 "\\" 结尾。不如，如果后面有逗号，必须使用 "//"，因为 "\\" 会把逗号当做文件名的一部分。所以，建议使用 '//' 而不是 '\\'。
- 环境变量被扩展 :set_env 。
- 小心使用 '\' 字符，在空格前输入一个，真正的反斜杠前要输入两个（见 option-backslash ）。例如：

```
:set bdir=c:\\tmp,\ dir\\,with\\,commas,\\\ dir\ with\ spaces
```

- 为了和 Vim 3.0 版本后向兼容，本选项开头的 '>' 被删除。

另见 'backup' 和 'writebackup' 选项。
如果你需要在 Unix 上隐藏备份文件，考虑这样的值：

```
:set backupdir=./.backup,~/.backup,.,/tmp
```

要使之正确工作，你需要在每个目录和你的主目录里建立 ".backup" 目录。
建议使用 :set+= 和 :set-= 来从列表里加减目录。这可以避免未来版本使用其它缺省值出现的问题。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'backupext' 'bex' E589

'backupext' 'bex' 字符串 (缺省为 "~", VMS 上: "_")
全局
附加到文件名后面的字符串，以构造备份文件的名字。缺省值很特别，因为它可以避免不小心覆盖已有的且带有备份的文件。你可能喜欢使用 ".bak"，但要确保不存在已经带有 ".bak" 而且你想保留的文件。
只能使用普通的文件名字符。"/*?[|<>" 都不合法。

如果你喜欢保留很多备份，可以用 BufWritePre 自动命令，使之在写入文件前改变 'backupext'，以包含写入的时间。

```
:au BufWritePre * let &bex = '-' . strftime("%Y%b%d%X") . '~'
```

'backupdir' 可用来把备份放在不同的目录里。

'backupskip' 'bsk'

'backupskip' 'bsk' 字符串 (缺省: "$TMPDIR/*,$TMP/*,$TEMP/*"
Unix: "/tmp/*,$TMPDIR/*,$TMP/*,$TEMP/*"
Mac: "/private/tmp/*,$TMPDIR/*,$TMP/*,$TEMP/*")
全局
{仅当编译时加入 +wildignore 特性才有效}
文件名模式列表。如果其中某个模式匹配要写入的文件名，不建立它的备份文件。同时测试指定的文件名和它的完整路径名。模式的使用方式和 :autocmd 类同，见 autocmd-patterns 。
小心特殊字符，见 option-backslash 。
如果没有定义 $TMPDIR、$TMP 或 $TEMP，缺省值就不包含它们。"/tmp/*" 只用于 Unix。

警告: 关闭此选项意味着如果不能正确保存缓冲区到文件，则 Vim 退出 (不管何种原因) 时，你同时丢失了原来的文件和最新的改动。只有在你真不介意丢失整个文件时才可以关闭备份。

注意 这里不扩展环境变量。如果你想要用 $HOME，必须显式地对其扩展，例如:

```
:let &backupskip = escape(expand('$HOME'), '\') . '/tmp/*'
```

注意 缺省值也确保 "crontab -e" 能工作 (如果给原来文件换名来建立备份，crontab 不会看到新建立的文件)。另见 'backupcopy' 和 crontab 。

'balloondelay' 'bdlay'

'balloondelay' 'bdlay' 数值型 (缺省: 600)
全局
{仅当编译时加入 +balloon_eval 特性才有效}
弹出气泡之前以毫秒计的延迟。见 balloon-eval 。

'ballooneval' 'beval' 'noballooneval' 'nobeval'

'ballooneval' 'beval' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +balloon_eval 特性才有效}
为 GUI 打开 balloon-eval 功能。

'balloonevalterm' 'bevalterm' 'noballoonevalterm' 'nobevalterm'

'balloonevalterm' 'bevalterm' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +balloon_eval_term 特性才有效}

为终端打开 balloon-eval 功能。

'balloonexpr' 'bexpr'
'balloonexpr' 'bexpr' 字符串 (缺省 "")
全局或局部于缓冲区 global-local
{仅当编译时加入 +balloon_eval 特性才有效}
计算气泡显示文本的表达式。只在 'balooneval' 或 'balloonevalterm' 打开时才使用。它用到以下变量:

v:beval_bufnr	要显示气泡的缓冲区号
v:beval_winnr	窗口编号
v:beval_winid	窗口 ID
v:beval_lnum	行号
v:beval_col	列号 (字节位置)
v:beval_text	鼠标指针所在或之后的单词

气泡显示仅限于普通文本。你考虑使用弹出窗口代替，见 popup_beval_example 。弹出窗口可以使用高亮并显示边框。

表达式的计算不能有副作用!
例如:

```
function MyBalloonExpr()
    return 'Cursor is at line ' . v:beval_lnum .
            \', column ' . v:beval_col .
            \ ' of file ' .  bufname(v:beval_bufnr) .
            \ ' on word "' . v:beval_text . '"'
endfunction
set bexpr=MyBalloonExpr()
set balooneval
```

另见 balloon_show() ，可用于气泡内容的异步获取。此时 'balloonexpr' 的计算应返回空串。如果你看见气泡只有 "0" 的话，你的函数可能什么都没有返回。

注意: 只有光标在文本字符上的时候才会显示气泡。如果 'balloonexpr' 的计算结果非空，Vim 不会试图发送消息给外部调试器 (Netbeans 或 Sun Workshop)。

如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

计算 'balloonexpr' 时，不能改变文本或跳到别的窗口，见 textlock 。

要检查气泡文本是否可以包含换行符:

```
if has("balloon_multiline")
```

如果支持，"\n" 字符开启新行。如果表达式计算结果为 List ，这相当于把每个列表项目当作字符串，然后之间用 "\n" 连接。
注意: 如果置位 'compatible'，本选项设为 ""。

'belloff' 'bo'
'belloff' 'bo' 字符串 (缺省 "")
全局
指定什么事件不会响铃。逗号分隔的项目列表。每个指定的项目都会关闭响铃。

对插入模式指定事件保持安静最有用。

项目	存在时的意义
all	所有事件。
backspace	按 <BS> 或 <Del> 和删除产生错误。
cursor	Insert-mode 时移动光标键或 <PageUp>/<PageDown> 失败。
complete	i_CTRL-X_CTRL-K 或 i_CTRL-X_CTRL-T 产生错误。
copy	插入模式下用 i_CTRL-Y 或 i_CTRL-E 不能复制字符。
ctrlg	插入模式下 <C-G> 之后的字符不能识别。
error	其它错误发生（例如试图对末行进行连接）（多用于 Normal-mode 或 Cmdline-mode ）。
esc	Normal-mode 按 <Esc>。
ex	Visual-mode 按 Q 产生错误。
hangul	忽略。
insertmode	'insertmode' 按 <Esc>。
lang	调用 Lua/Mzscheme/TCL 的 beep 模块。
mess	g< 没有输出可用。
showmatch	'showmatch' 函数发生错误。
operator	cpo-E 空区域出错。
register	Insert-mode <C-R> 之后的寄存器不能识别。
shell	外壳输出 :! 的铃声。
spell	拼写检查出错。
wildmode	cmdline-completion 有更多的匹配可用（取决于 'wildmode' 设置）。

最常用于调节插入模式响铃的行为。普通模式和 Ex 命令的响铃通常指示有错误发生。此时通过 "error" 关键字可以关闭响铃。

'binary' 'bin' 'nobinary' 'nobin'
'binary' 'bin' 布尔型 （缺省关闭）
局部于缓冲区

本选项应该在编辑二进制文件之前设置。你也可以用 Vim 的 -b 参数。如果打开本选项，一些选项的值会发生改变（即使它们原本是打开的）：

'textwidth' 会设为 0
'wrapmargin' 会设为 0
'modeline' 会被关闭
'expandtab' 会被关闭

此外，也不会使用 'fileformat' 和 'fileformats' 选项。文件的读写如同 'fileformat' 设为 "unix" 那样（单个 <NL> 分隔行）。
不会使用 'fileencoding' 和 'fileencodings' 选项。文件读取不经过转换。
注意：如果 'bin' 选项已经打开而你开始编辑（另）一个文件，自动命令的设置可能会再次改变这些相关的设置（比如，'textwidth'），使得编辑出现麻烦。因而，载入文件时，你可以再次置位 'bin'。
这些选项以前的值被记住，'bin' 从开到关时会复原它们。每个缓冲区都有一组保存的选项值。
要使用 'binary' 编辑文件，你可以使用 ++bin 参数。这样你不用输入 ":set bin"，而后者会影响所有你编辑的文件。
写回文件时，只有原来文件本就包含时才写入末行的 <EOL> （通常，如果末行没有 <EOL>，Vim 会附加一个；这样文件会变长）。见 'endofline' 选项。

'bioskey' 'biosk' 'nobioskey' 'nobiosk'
'bioskey' 'biosk' 布尔型 （缺省打开）
全局
{仅适用于 MS-DOS}

为 MS-DOS 设计，不再支持。

'bomb' 'nobomb'
'bomb' 布尔型 (缺省关闭)
局部于缓冲区
写入文件时，如果满足以下条件，在文件头部写入 BOM (Byte Order Mark，字节顺序标记)：
- 打开本选项
- 关闭 'binary' 选项
- 'fileencoding' 是 "utf-8"、"ucs-2"、"ucs-4" 或它们的 little/big endian (高位在右侧/左侧的字节序) 变种。

有些应用程序使用 BOM 识别文件的编码。通常用于 MS-Windows 上的 UCS-2 文件。对别的程序而言，这会造成麻烦，比如："cat file1 file2" 使得 file2 的 BOM 出现在结果文件的中间。gcc 不接受 BOM。
读入文件时，如果 'fileencodings' 以 "ucs-bom" 开头，Vim 会检查 BOM 是否存在，并相应地设置 'bomb'。
除非置位 'binary'，BOM 会从首行上移去，所以编辑时你看不到它。如果你没有改变相应设置，写回文件时恢复 BOM。

'breakat' 'brk'
'breakat' 'brk' 字符串 (缺省为 " ^I!@*-+;:,./?")
全局
{仅当编译时加入 +linebreak 特性才有效}
如果打开 'linebreak'，本选项让你选择可以在什么字符上换行。只能用 ASCII 字符，但如果 'encoding' 是 8 位编码，也可以用 8 位的字符。

'breakindent' 'bri' 'nobreakindent' 'nobri'
'breakindent' 'bri' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +linebreak 特性才有效}
回绕行保持视觉上的缩进 (和该行开始处相同的空白数目)，从而保留文本的水平块。
注意：如果置位 'compatible'，本选项被复位。

'breakindentopt' 'briopt'
'breakindentopt' 'briopt' 字符串 (缺省为空)
局部于窗口
{仅当编译时加入 +linebreak 特性才有效}
'breakindent' 的相关设置。有以下可选部分组成，以逗号分隔：
min:{n} 应用 'breakindent' 后保留的最小文本宽度。哪怕缩进后的文本窄于此宽度也是如此。这防止文本在断开后，占据太多垂直空白，从而缩进到太靠窗口右边界的位置。
shift:{n} 应用 'breakindent' 后，回绕行的开始处右移指定数目的字符。可以实现动态法式段落缩进 (负位移) 或强调行缩进 (正位移)。
sbr 应用额外缩进后，显示 'showbreak' 值。
缺省 min 值为 20 和 shift 值为 0。

'browsedir' 'bsdir'
'browsedir' 'bsdir' 字符串 (缺省为 "last")
全局
{仅适用于 Motif、Athena、GTK、Mac 和 Win32 GUI}
文件浏览器使用的目录：
last 使用文件浏览器最近打开或保存文件时相同的访问目录。

buffer　　使用相关缓冲区的目录。
current　　使用当前目录。
{path}　　使用指定目录。

'bufhidden'　'bh'
'bufhidden' 'bh'　　字符串 (缺省: "")
局部于缓冲区
本选项指定缓冲区不再出现于窗口时的行为:
<empty>　　跟随全局 'hidden' 选项
hide　　隐藏缓冲区 (不卸载), 即使没有置位 'hidden' 也如此
unload　　卸载缓冲区, 即使置位 'hidden' 或者使用 :hide 时也如此
delete　　从缓冲区列表里删除缓冲区, 即使置位 'hidden' 或者使用 :hide 时也如此, 类似于 :bdelete
wipe　　从缓冲区列表里真正删除缓冲区, 即使置位 'hidden' 或者使用 :hide 时也如此, 类似于 :bwipeout

小心: 使用 "unload"、"delete" 或 "wipe" 后, 缓冲区的修改会丢失而且没有警告。另外, 这些值也会使临时切换缓冲区的自动命令不能工作。
本选项和 'buftype' 和 'swapfile' 一起使用, 指定特殊的缓冲区的类型。见 special-buffers 。

'buflisted'　'bl'　'nobuflisted'　'nobl'　E85
'buflisted' 'bl'　　布尔型 (缺省: 打开)
局部于缓冲区
如果置位本选项, 缓冲区在缓冲区列表里显示。如果复位, 缓冲区不能用 ":bnext"、"ls"、Buffers 菜单等访问。
Vim 为只用来记住文件名或位置标记的缓冲区复位本选项。Vim 在开始编辑缓冲区时置位本选项, 但用 ":buffer" 移动到缓冲区时不会。

'buftype'　'bt'　E382
'buftype' 'bt'　　字符串 (缺省: "")
局部于缓冲区
本选项的值指定缓冲区的类型:
<empty>　　普通缓冲区
nofile　　和文件不相关且不会写回的缓冲区
nowrite　　不会写回的缓冲区
acwrite　　缓冲区总是用 BufWriteCmd 自动命令写回。
quickfix　　quickfix 缓冲区, 包含错误列表 :cwindow 或位置列表 :lwindow
help　　帮助缓冲区 (你不应该手动设置本值)
terminal　　用于 terminal 的缓冲区 (你不应该手动设置本值)
prompt　　只有末行可以编辑的缓冲区, 用于插件, 见 prompt-buffer {仅当编译时加入 +channel 特性才有效}
popup　　弹出窗口使用的缓冲区, 见 popup 。{仅当编译时加入 +textprop 特性才有效}

本选项和 'bufhidden' 和 'swapfile' 一起使用, 指定特殊的缓冲区类型。见 special-buffers 。

修改本选项要小心。它有很多副作用!

"quickfix" 缓冲区只用于错误列表和位置列表。用 :cwindow 和 :lwindow

命令设置本值，不要直接修改它。

"nofile" 和 "nowrite" 缓冲区很类似:
都是: 缓冲区不写回磁盘。":w" 不能工作 (":w filename" 仍然可以)。
都是: 缓冲区永远不会认为是 'modified' (修改过的)。没有警告说改变会丢失，即使你退出 Vim 时也是如此。
都是: 交换文件只有在使用超量内存时才会建立 (如果复位 'swapfile'，无论如何也不会有交换文件)。
只限 nofile: 缓冲区名是固定的，不被视为文件名来处理。 :cd 命令也不会修改它。
都是: 用 ":e bufname" 且 "bufname" 已被编辑时，清空缓冲区，并和正常 :edit 一样激发自动事件。
E676

"acwrite" 隐含意味着缓冲区名和文件不相关，类似于 "nofile"，但它会被写回。所以，和 "nofile" 与 "nowrite" 不同，可以 ":w" 而且修改过的缓冲区没有保存不能被放弃。写入操作要有匹配的 BufWriteCmd 、 FileWriteCmd 和 FileAppendCmd 自动命令。

'casemap' 'cmp'
'casemap' 'cmp' 字符串 (缺省: "internal,keepascii")
全局
指定改变字母大小写的细节。它包含以下单词，以逗号分隔:
internal 使用内部大小写映射函数，当前的 locale 不改变大小写映射。这只影响 'encoding' 为 Unicode 编码、"latin1" 或 "iso-8859-15" 的情况。如果没有 "internal"，使用系统库函数 (如果有的话) towupper() 和 towlower()。
keepascii ASCII 字符 (0x00 到 0x7f)，使用美国的大小写映射，当前 locale 不起作用。这可能只影响土耳其语。

'cdpath' 'cd' E344 E346
'cdpath' 'cd' 字符串 (缺省: 等价于 $CDPATH 或 ",,")
全局
{仅当编译时加入 +file_in_path 特性才有效}
这是一个目录列表。使用 :cd 、 :tcd 和 :lcd 命令时，如果查找的是相对路径，而不是以 "/"、"./" 或 "../" 开始的绝对路径，在这里搜索目录。绝对路径不使用 'cdpath' 选项。
'cdpath' 选项的值和 'path' 有相同的形式和语义。另见 file-searching 。
缺省值取自 $CDPATH，并在前面附加 ",", 以便先搜索当前目录。
如果取自 $CDPATH 的缺省值不是你想要的，在 vimrc 文件里加入下面命令的修改版本以覆盖之:

```
:let &cdpath = ',' . substitute(substitute($CDPATH, '[, ]', '\\\0', 'g'), ':', ',', 'g')
```

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。
('cdpath' 的内容可传递给外壳以便扩展文件名)。

'cedit'
'cedit' 字符串 (Vi 缺省: "", Vim 缺省: CTRL-F)
全局
用于命令行模式打开命令行窗口的键。
'compatible' 关闭时，缺省值为 CTRL-F。
只允许不可显示的键。
该键可以用单个字符的形式指定，但不易输入，建议使用 <> 记法。例如:

```
:exe "set cedit=\<C-Y>"
```

```
:exe "set cedit=\<Esc>"
```

Nvi 也有本选项，但只使用第一个字符。
见 cmdwin 。
注意：如果复位 'compatible'，此选项设为 Vim 缺省值。

'charconvert' 'ccv' E202 E214 E513

'charconvert' 'ccv' 字符串 (缺省为 "")
全局
{仅当编译时加入 +eval 特性才有效}

用于进行字符编码转换的表达式。文件读写时如果和期望的编码不同，计算本表达式。
如果内部支持 iconv() 函数并且它能完成本次转换，不使用 'charconvert'。iconv() 更好，因为它快得多。
读入标准输入 -- 时不使用 'charconvert'，因为没有可以文件可以进行转换。你需要先把文本保存到文件里。
表达式必须返回零或空字符串表示成功，非零表示失败。
可能遇到的编码名字来自 'encoding'。另外，也使用 'fileencodings' 和 'fileencoding' 里的名字。
"latin1"、"unicode"、"ucs-2"、"ucs-4" 和 "utf-8" 之间的转换由 Vim 内部完成，'charconvert' 不用于此种转换。
如果 'viminfo' 包含 'c' 标志位，'charconvert' 也被用来转换 viminfo 文件。它也用于 Unicode 转换。
示例：

```
set charconvert=CharConvert()
fun CharConvert()
  system("recode "
        \ . v:charconvert_from . ".." . v:charconvert_to
        \ . " <" . v:fname_in . " >" v:fname_out)
  return v:shell_error
endfun
```

相关的 Vim 变量是：

v:charconvert_from	当前的编码名
v:charconvert_to	期待的编码名
v:fname_in	输入文件名
v:fname_out	输出文件名

注意 v:fname_in 和 v:fname_out 永远不会相同。
注意 v:charconvert_from 和 v:charconvert_to 可能与 'encoding' 不同。Vim 内部使用 UTF-8 代替 UCS-2 或 UCS-4。
使用 'charconvert' 时，Vim 不进行加密。如果你希望转换后加密文件，'charconvert' 需要自己负责这一点。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'cindent' 'cin' 'nocindent' 'nocin'

'cindent' 'cin' 布尔型 (缺省关闭)
局部于缓冲区
{仅当编译时加入 +cindent 特性才有效}

打开自动 C 程序缩进。'cinkeys' 说明如何设置插入模式下启动重新缩进的热键，'cinoptions' 说明如何设置你喜欢的缩进风格。
如果 'indentexpr' 非空，它否决 'cindent' 的设置。
如果没有打开 'lisp'，而 'indentexpr' 和 'equalprg' 都为空，"=" 操作符使用本算法缩进，而不调用外部程序。
见 C-indenting 。
如果你不喜欢 'cindent' 的工作方式，可以试试 'smartindent' 选项或者

'indentexpr'。
如果置位 'paste'，不使用本选项。
注意：如果置位 'compatible'，本选项被复位。

'cinkeys' 'cink'

'cinkeys' 'cink'　　字符串　(缺省为 "0{,0},0),0],:,0#,!^F,o,O,e")
局部于缓冲区
{仅当编译时加入 +cindent 特性才有效}
键列表，在插入模式输入这些键时，当前行被重新缩进。只有打开 'cindent' 而且 'indentexpr' 为空时才会用到。
本选项的格式见 cinkeys-format 。
见 C-indenting 。

'cinoptions' 'cino'

'cinoptions' 'cino'　　字符串　(缺省为 "")
局部于缓冲区
{仅当编译时加入 +cindent 特性才有效}
'cinoptions' 影响 'cindent' 重新缩进 C 程序行的方式。
cinoptions-values 说明如何设置本选项的值，而 C-indenting 提供总体的 C 缩进方式的信息。

'cinwords' 'cinw'

'cinwords' 'cinw'　　字符串　(缺省为 "if,else,while,do,for,switch")
局部于缓冲区
{仅当编译时同时加入 +cindent 和 +smartindent 特性才有效}
如果使用 'smartindent' 或 'cindent'，这些关键字在下一行开启额外的缩进。对于 'cindent'，只有在合适的地方才会这么做（在 {} 里）。
注意 'ignorecase' 对 'cinwords' 没有影响。如果希望忽略大小写，包含关键字所有的大小写："if,If,IF"。

'clipboard' 'cb'

'clipboard' 'cb'　　字符串　(X-windows 的缺省为
"autoselect,exclude:cons\|linux"，
其它为 "")
全局
{仅有在 GUI 版本或者包含 +xterm_clipboard 特性时才有效}
本选项是逗号分隔的名字列表。
识别以下名字：

clipboard-unnamed

unnamed　　如果包含，Vim 使用剪贴板寄存器 '*' 来执行所有的抽出、删除、修改和放置操作，而不是通常使用的无名寄存器。除非显式指定了寄存器，这时不管 'clipboard' 是不是包含 "unnamed" 都使用指定的寄存器。剪贴板寄存器总是可以显式地用 "* 记法来访问。另见 gui-clipboard 。

clipboard-unnamedplus

unnamedplus　　"unnamed" 标志位的变种，对所有通常使用无名寄存器来抽出、删除、修改和放置操作，以剪贴板 '+' 寄存器 (quoteplus) 代替 '*' 寄存器。如也包含 "unnamed"，抽

出动作（但不包括删除、修改和放置）会同时把文本放入 '*' 寄存器。
仅当 +X11 特性存在时才可用。
以下方法可检查该选项是否可用：

```
if has('unnamedplus')
```

clipboard-autoselect

autoselect 和 'guioptions' 的 'a' 标志位工作方法类似：如果存在，无论什么时候启动可视模式或者可视区域扩展，Vim 都试图成为窗口系统全局选择区的拥有者，或者把选择的文本放置到选择区寄存器 "* 使用的剪贴板上。详见 'go-a' 和 quotestar 。如果激活 GUI，使用 'guioptions' 的 'a' 标志位。如果没有激活 GUI，使用本 "autoselect" 标志位。
也适用于无模式的选择。

clipboard-autoselectplus

autoselectplus 类似于 "autoselect"，但用 + 寄存器代替 * 寄存器。可以和 'guioptions' 的 'P' 标志位相比拟。

clipboard-autoselectml

autoselectml 类似于 "autoselect"，但只适用于无模式的选择。可以和 'guioptions' 的 'A' 标志位相比拟。

clipboard-html

html 当剪贴板包含 HTML 时，粘贴时使用 HTML 格式。放置文本到剪贴板时，标识为 HTML。可用于从 Firefox 复制 HTML 内容并粘贴原始 HTML 码到 Vim，或从 Vim 选择 HTML 码，然后粘贴到 Firefox 的富文本编辑框 (rich edit)。
你可能只想暂时包含本选项，例如通过 BufEnter 自动命令。
只支持 GTK 版本 2 或以上。

clipboard-exclude

exclude:{pattern}
定义用来匹配终端名 'term' 的模式。匹配的终端不连接到 X 服务器。可用于以下这种情形：
- 在控制台上运行 Vim，
- $DISPLAY 的设置使得应用程序在另一个显示上打开，
- 不希望在控制台上连接到 X 服务器，但希望在终端模拟器上如此。

如果永远不想连接到 X 服务器，使用：

```
exclude:.*
```

这和 -X 参数的效果相同。
注意 如果没有到 X 服务器的连接，窗口标题不会复原，而剪贴板也无法访问。
忽略 'magic' 的值，{pattern} 的解释总是假定 'magic' 打开。
选项值的其余部分用于 {pattern}，本项目必须是最后一个项目。

'cmdheight' 'ch'

'cmdheight' 'ch' 数值型 (缺省为 1)
全局
命令行使用的屏幕行数。有助于避免 hit-enter 提示。
此选项值保存在每个标签页里，从而每个标签页可有不同的值。

'cmdwinheight' 'cwh'

'cmdwinheight' 'cwh'　　数值型　(缺省为 7)
　　全局
　　命令行窗口的屏幕行数。 cmdwin

'colorcolumn' 'cc'

'colorcolumn' 'cc'　　字符串　(缺省为 "")
　　局部于窗口
　　{仅当编译时加入 +syntax 特性才有效}
　　'colorcolumn' 是逗号分隔的屏幕列的列表，这些列会用 ColorColumn hl-ColorColumn 高亮。可用于对齐文本。会使屏幕刷新变慢。
　　屏幕列可以是绝对值，也可以在数值之前带有 '+' 或 '-' ，后者会采用从 'textwidth' 加上或减去该值之后的值。

```
:set cc=+1  " 高亮 'textwidth' 之后的列
:set cc=+1,+2,+3  " 高亮 'textwidth' 之后的三列
:hi ColorColumn ctermbg=lightgrey guibg=lightgrey
```

　　'textwidth' 为零时不使用带 '-' 和 '+' 的项目。
　　最多高亮 256 列。

'columns' 'co' E594

'columns' 'co'　　数值型　(缺省为 80 或终端宽度)
　　全局
　　屏幕的列数。通常，它由终端初始化代码设置，不需要手动完成。另见 posix-screen-size 。
　　如果 Vim 运行 GUI 或者在可改变大小的窗口里，设置此选项可以改变窗口的大小。如果你只想设置 GUI 的大小，在 gvimrc 文件里放入设置命令。
　　如果你设置此选项而 Vim 无法改变物理的显示列数，显示可能会混乱。GUI 没有这种约束，但 Vim 限制能在屏幕上显示的行数。你可以用此命令得到可用的最大窗口宽度:

```
:set columns=9999
```

　　最小值为 12，最大值为 10000。

'comments' 'com' E524 E525

'comments' 'com'　　字符串　(缺省 "s1:/*,mb:*,ex:*/,://,b:#,:%,:XCOMM,n:>,fb:-")
　　局部于缓冲区
　　逗号分隔的可以开始注释行的字符串列表。见 format-comments 。 option-backslash 说明如何包含空格和使用反斜杠。

'commentstring' 'cms' E537

'commentstring' 'cms'　　字符串　(缺省为 "/*%s*/")
　　局部于缓冲区
　　{仅当编译时加入 +folding 特性才有效}
　　注释行的样板。该值里面的 "%s" 被注释文本替换。目前，只用于为折叠增加标志。见 fold-marker 。

'compatible' 'cp' 'nocompatible' 'nocp'

'compatible' 'cp'　　布尔型　(缺省打开，如果找到 vimrc 或 gvimrc 文件则为关闭，重设用 defaults.vim)
　　全局
　　本选项的效果是使得 Vim 或者更兼容 Vi，或者以更有用的方式工作。

这是一个特殊的选项，因为它的置位与复位影响到其他的设置。
注意：置位或者复位本选项会产生很多意想不到的效果：映射以不同的方式进行解释，撤销的行为也有所不同，不一而足。如果你在 vimrc 文件里设置此选项，最好放在相当靠前的位置。

缺省打开本选项，而各种选项使用 Vi 的缺省。本缺省值是为那些希望像 Vi 那样使用 Vim 的人准备的，他们可能都不（想）知道 'compatible' 选项。
如果 Vim 启动时找到 vimrc 或 gvimrc 文件，关闭本选项，而没有修改的选项都设置为 Vim 的缺省。这意味着如果存在 vimrc 或 gvimrc 文件，Vim 使用 Vim 的缺省，否则使用 Vi 的缺省。（注意：这不适用于系统范围的 vimrc 或 gvimrc 文件，也不适用 -u 参数给定的文件）。另见 compatible-default 和 posix-compliance 。
你也可以用 "-C" 参数置位此选项，或用 "-N" 复位。见 -C 和 -N 。
'cpoptions' 提供更多 Vi 兼容性的调整设置。

如果置位此选项，设置若干其它选项使得 Vim 尽量和 Vi 兼容。如果复位此选项，设置若干选项使 Vim 更有用。下表列出所有受影响的选项。
{?} 列指示选项何时受影响。
+ 意味着 'compatible' 置位时，选项设为 {设置值} 给出的值。
& 意味着 'compatible' 置位时，选项设为 {设置值} 给出的值，而且在 'compatible' 复位时设为 Vim 缺省值。
- 意味着 'compatible' 置位时，选项值不改变，但在 'compatible' 复位时设为 Vim 缺省值。
{效果} 栏总结 'compatible' 置位时的改变。

选项	+	设置值	效果
'allowrevins'	+	关闭	没有 CTRL-_ 命令
'antialias'	+	关闭	不使用反锯齿字体
'arabic'	+	关闭	复位阿拉伯语相关的选项
'arabicshape'	+	打开	字型校正
'backspace'	+	""	普通的退格
'backup'	+	关闭	没有备份文件
'backupcopy'	&	Unix: "yes"	备份文件由复制产生
		其它: "auto"	复制或者换名产生备份文件
'balloonexpr'	+	""	计算气泡中显示的文本
'breakindent'	+	关闭	回绕行时不缩进
'cedit'	-	{不改变}	{仅当复位 'cp' 时设为 vim 缺省值}
'cindent'	+	关闭	不打开 C 代码的缩进
'compatible'	-	{不改变}	{仅当复位 'cp' 时设为 vim 缺省值}
'copyindent'	+	关闭	不复制缩进结构
'cpoptions'	&	(所有标志位)	Vi 兼容的标志位
'cscopepathcomp'	+	0	标签列表中不显示目录
'cscoperelative'	+	关闭	不使用路径的基础名作为前缀
'cscopetag'	+	关闭	":tag" 不使用 cscope
'cscopetagorder'	+	0	见 cscopetagorder
'cscopeverbose'	+	关闭	见 cscopeverbose
'delcombine'	+	关闭	unicode: 删除整个字符组合
'digraph'	+	关闭	没有二合字母
'esckeys'	&	关闭	插入模式不使用 <Esc> 开始键码的键
'expandtab'	+	关闭	制表不扩展为空格
'fileformats'	&	""	没有自动的文件格式检测

		"dos,unix"	除了 MS-Windows 外
'formatexpr'	+	""	自动排版时使用 'formatprg'
'formatoptions'	&	"vt"	Vi 兼容的排版选项
'gdefault'	+	关闭	":s" 没有缺省的 'g' 标志位
'history'	&	0	没有命令行历史
'hkmap'	+	关闭	没有希伯来语键盘映射
'hkmapp'	+	关闭	没有根据读音的希伯来语键盘映射
'hlsearch'	+	关闭	没有搜索匹配的高亮
'incsearch'	+	关闭	没有增量搜索
'indentexpr'	+	""	没有基于表达式的缩进
'insertmode'	+	关闭	不以插入模式启动
'iskeyword'	&	"@,48-57,_"	关键字包含字母数字和 '_'
'joinspaces'	+	打开	句号后插入两个空格
'modeline'	&	关闭	没有模式行
'more'	&	关闭	列表不暂停
'mzquantum'	-	{不改变}	{仅当复位 'cp' 时设为 vim 缺省值}
'numberwidth'	&	8	行号的最少列数
'preserveindent'	+	关闭	改变缩进时不保留当前缩进结构
'revins'	+	关闭	没有反向插入
'ruler'	+	关闭	没有标尺
'scrolljump'	+	1	滚动不跳跃
'scrolloff'	+	0	没有滚动偏移
'shelltemp'	-	{不改变}	{仅当复位 'cp' 时设为 vim 缺省值}
'shiftround'	+	关闭	缩进不取整到 shiftwidth 的倍数
'shortmess'	&	"S"	不缩短信息
'showcmd'	&	关闭	不显示命令字符
'showmode'	&	关闭	不显示当前模式
'sidescrolloff'	+	0	滚动时光标移动到屏幕边缘
'smartcase'	+	关闭	没有忽略大小写的自动切换
'smartindent'	+	关闭	没有智能缩进
'smarttab'	+	关闭	没有智能制表大小
'softtabstop'	+	0	制表总是使用 'tabstop' 个位置
'startofline'	+	打开	一些命令到行首
'tagcase'	&	"followic"	搜索标签文件时 'ignorecase'
'tagrelative'	&	关闭	标签文件名不是相对的
'termguicolors'	+	关闭	不使用高亮-(guifg|guibg)
'textauto'	&	关闭	没有自动文本模式检测
'textwidth'	+	0	没有自动行回绕
'tildeop'	+	关闭	波浪符不是操作符
'ttimeout'	+	关闭	没有终端超时
'undofile'	+	关闭	不使用撤销文件
'viminfo'	-	{不改变}	{仅当复位 'cp' 时设为 vim 缺省值}
'virtualedit'	+	""	光标只能放在字符上
'whichwrap'	&	""	左右移动不回绕
'wildchar'	&	CTRL-E	只有在当前值为 <Tab> 时，使用 CTRL-E 用于命令行补全
'writebackup'	+	打开或关闭	取决于 +writebackup 特性

'complete' 'cpt' E535

'complete' 'cpt' 字符串 (缺省: ".,w,b,u,t,i")
局部于缓冲区

本选项控制关键字补全 ins-completion 在使用 CTRL-P 或 CTRL-N 时如何工作，也用于整行的补全 i_CTRL-X_CTRL-L 。它指示补全的类型和需要扫描的位置。这是一个逗号分隔的标志位的列表：

. 扫描当前缓冲区（忽略 'wrapscan'）
w 扫描其它窗口的缓冲区
b 扫描其它缓冲区列表里载入的缓冲区
u 扫描缓冲区列表里卸载的缓冲区
U 扫描不在缓冲区列表里的缓冲区
k 扫描 'dictionary' 选项给出的文件
kspell 使用当前激活的拼写检查 spell
k{dict} 扫描文件 {dict}。可以给出多个 "k" 标志位，也可以使用模式。比如：

```
:set cpt=k/usr/dict/*,k~/spanish
```

s 扫描 'thesaurus' 选项给出的文件
s{tsr} 扫描文件 {tsr}。可以给出多个 "s" 标志位，也可以使用模式。
i 扫描当前文件和包含的头文件
d 扫描当前文件和包含的头文件里的定义名或宏 i_CTRL-X_CTRL-D
] 标签补全
t 等同于 "]"

卸载的缓冲区不会载入，因而它们的自动命令 :autocmd 也不会执行，这使得有些文件的补全效果出人意表（比如 gzipped 压缩的文件）。整行补全也不会扫描卸载的缓冲区。

缺省是 ".,w,b,u,t,i"，它意味着扫描：
1. 当前缓冲区
2. 其它窗口的缓冲区
3. 其它载入的缓冲区
4. 卸载的缓冲区
5. 标签
6. 头文件

如你所知，CTRL-N 和 CTRL-P 可用于完成任何基于 'iskeyword' 的扩展（比如字典 i_CTRL-X_CTRL-K 、头文件里的模式 i_CTRL-X_CTRL-I 、标签 i_CTRL-X_CTRL-] 和普通的扩展）。

'completefunc' 'cfu'
'completefunc' 'cfu' 字符串 （缺省：空）
局部于缓冲区
{仅当编译时加入 +eval 特性才有效}
本选项指定插入模式补全 CTRL-X CTRL-U 使用的函数。 i_CTRL-X_CTRL-U complete-functions 解释如何调用该函数以及它应该返回何值。

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'completeslash' 'csl'
'completeslash' 'csl' 字符串 （缺省："")
局部于缓冲区
{仅适用于 MS-Windows}
此选项设置时，在补全的时候否决 'shellslash'：
- 如果此选项设为 "slash"，正斜杠用于插入模式时的路径补全。这可用于 MS-Windows 上 'noshellslash' 置位时编辑 HTML 标签或 Makefile。
- 如果此选项设为 "backslash"，使用反斜杠。这可用于 MS-Windows 上 'shellslash' 置位时编辑批处理文件。
- 如果此选项为空，使用和 'shellslash' 相同的字符。

插入模式的补全使用缓冲区局部变量。命令行补全使用全局值。

'completeopt' 'cot'

'completeopt' 'cot'	字符串 (缺省: "menu,preview")
全局

逗号分隔的选项列表，用于插入模式的补全 ins-completion 。支持的值是:

menu	使用弹出菜单来显示可能的补全。只有匹配多于一个且有足够的颜色时才显示该菜单。 ins-completion-menu

menuone	即使只有一个匹配，也使用弹出菜单。可用于匹配有附加信息的情况，例如，它来自的文件。

longest	只插入匹配的最长公共文本。如果显示菜单，用 CTRL-L 来插入更多的字符。是否忽略大小写取决于补全的类型。缓冲区文本使用 'ignorecase' 选项。

preview	在预览窗口里显示当前选择的补全的额外信息。只能和 "menu" 或 "menuone" 搭配使用。

popup	在弹出窗口里显示当前选择的补全的额外信息。只能和 "menu" 或 "menuone" 搭配使用。否决 "preview"。
指定相关属性见 'completepopup' 。
{仅当编译时加入 +textprop 特性才有效}

popuphidden
类似于 "popup"，但弹出先被弹藏。使用 CompleteChanged 自动命令来获取 info，一旦弹出被填充，再调用 popup_show() 。例见 complete-popuphidden 。
{仅当编译时加入 +textprop 特性才有效}

noinsert	不插入任何匹配的文本，直到用户从菜单里选择一个匹配为止。只能用于和 "menu" 和 "menuone" 的组合。如果 "longest" 存在，没有效果。

noselect	不选择菜单里的匹配，强迫用户从菜单中选择。只能用于和 "menu" 和 "menuone" 的组合。

'completepopup' 'cpp'

'completepopup' 'cpp'	字符串 (缺省: 空)
全局
{仅当编译时加入 +textprop 或 +quickfix 特性才有效}

当 'completeopt' 包含 "popup" 时，此选项用于 info 弹出建立时的属性。你也可用 popup_findinfo() 然后用 popup_setoptions() 来设置已有的 info 弹出的属性。见 complete-popup 。

'concealcursor' 'cocu'

'concealcursor' 'cocu'	字符串 (缺省: "")
局部于窗口
{仅当编译时加入 +conceal 特性才有效}

设置也能隐藏光标所在行的文本的模式。
如果当前模式在此处列出，光标行的隐藏行为和其它行一样。

n	普通模式
v	可视模式

i 插入模式
c 命令行编辑模式，用于 'incsearch'

'v' 适用于可视区域的所有行，不局限于光标所在行。
一个有用的值为 "nc"。可用于帮助文件。如果你只来回移动，文本会保持隐藏。但一旦你开始插入文本或选择一个可视区域，隐藏文本被显示，以便你看清你在干什么。
记住光标位置并不总是它所显示的位置，例如，上下移动时，所在列可能改变。

'conceallevel' 'cole'

'conceallevel' 'cole' 数值型（缺省为 0）
局部于窗口
{仅当编译时加入 +conceal 特性才有效}
指定如何显示使用 "conceal" 语法属性 :syn-conceal 的文本：

值 效果
0 正常显示文本
1 每处隐藏文本块被一个字符替代。如果语法项目没有定义定制的替代字符（见 :syn-cchar ），使用 'listchars' 定义的字符（缺省是空格），
使用 "Conceal" 高亮组高亮。
2 完全不显示隐藏文本，除非它定义了自己的替换字符（见 :syn-cchar ）。
3 完全不显示隐藏文本。

注意：光标所在行的隐藏文本总是会被显示，以便你编辑和复制文本。但可用 'concealcursor' 选项改变此行为。

'confirm' 'cf' 'noconfirm' 'nocf'

'confirm' 'cf' 布尔型（缺省关闭）
全局
如果 'confirm' 打开，一些通常因为缓冲区有未保存的改变而失败的操作，比如 ":q" 和 ":e"，会弹出对话框 dialog ，询问你是否想保存当前（一个或多个）文件。你仍然可以使用 ! 来无条件放弃 abandon 缓冲区。
如果 'confirm' 关闭，你仍然可以通过 :confirm 命令为单个命令启动确认机制（常用于映射）。
另见 confirm() 函数和 'guioptions' 的 'v' 标志位。

'conskey' 'consk' 'noconskey' 'noconsk'

'conskey' 'consk' 布尔型 （缺省关闭）
全局
为 MS-DOS 设计，不再支持。

'copyindent' 'ci' 'nocopyindent' 'noci'

'copyindent' 'ci' 布尔型 （缺省关闭）
局部于缓冲区
自动缩进新行时，复制已有的行的缩进结构。通常，用连续的制表，必要时后面再跟若干空格，重新构造新缩进（除非打开 'expandtab' ，这时只用空格）。打开此选项使得新行准确地复制已有行中用来缩进的所有字符。'expandtab' 对这些字符没有影响，制表还是制表。如果新的缩进比已有行的缩进要多，多余部分的空白使用平常的填充方式。
注意：如果置位 'compatible'，本选项被复位。
另见 'preserveindent'。

'cpoptions' 'cpo' cpo 'cpoptions' 'cpo'

'cpoptions' 'cpo' 字符串 (Vim 缺省: "aABceFs",
Vi 缺省: 所有的标志位)
全局

单字符标志位的序列。每个字符的存在指示一种 Vi 兼容的行为。可用于改变常见或被认为是更好的 Vi 不兼容的行为。
'cpoptions' 代表兼容性选项 "(compatible-options)"。
为了可读性起见，可以加入逗号。
建议使用 ":set" 的 "+=" 和 "-=" 特性，这可以避免未来版本增加新的标志位时出现的问题 add-option-flags 。

注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

注意: 如果应该使用 Vi 的缺省值且 $VIM_POSIX 环境变量存在时，本选项设为 POSIX 的缺省值 posix 。这意味着 Vim 尽量符合 POSIX 规范的要求。

包含	行为
a	cpo-a 如果包含，带文件名参数的 ":read" 命令会为当前窗口设置轮换文件名。
A	cpo-A 如果包含，带文件名参数的 ":write" 命令会为当前窗口设置轮换文件名。
b	cpo-b ":map" 命令里识别 "|" 为 map 命令的结束。'\' 被包含在映射里，'|' 之后的文本被解释为别的命令。使用 CTRL-V 代替反斜杠使 '|' 也包含在映射里。适用于所有的映射、缩写、菜单和自动命令。 另见 map_bar 。
B	cpo-B 映射、缩写、用户命令和菜单命令的 "to" 部分里反斜杠没有特殊含义。删除本标志位使得反斜杠的效果类似于 CTRL-V。 例如，命令 ":map X \<Esc>" 使得 X 被映射为: 包含 'B' 时: "\^[" (^[是真正的 <Esc>) 排除 'B' 时: "<Esc>" (5 个字符) (两种情况下都假定 '<' 标志位被排除)
c	cpo-c 搜索从任何光标所在的匹配的结束处继续，但不超过下一行的开始处。如果没有此标志位，搜索从光标位置之后的一个字符开始。使用 'c'，"abababababab" 重复用 "/abab" 模式时只有三个匹配，没有 'c' 的话有五个。
C	cpo-C 不串接反斜杠开始的被 source 的行。见 line-continuation 。
d	cpo-d 'tags' 选项使用 "./" 不意味着使用相对于当前文件的标签文件，而是当前目录下的标签文件。
D	cpo-D 带有字符参数的普通模式命令不使用 CTRL-K 输入二合字母，比如 r 、 f 和 t 。
e	cpo-e 用 ":@r" 执行寄存器时，总是在最后一行加入 <CR>，即使寄存器不是面向行的也如此。如果本标志位不存在、寄存器非面

向行且末行不以 <CR> 结尾的话，那么末行仍留在命令行上，在按 <CR> 之前可以继续编辑。

cpo-E

E 在空区域上使用 "y"、"d"、"c"、"g~"、"gu" 或 "gU" 是个错误。这些操作符至少需要操作一个字符。比如："y0" 在第一列会失败。

cpo-f

f 如果包含并且当前缓冲区还没有文件名，带文件名参数的 ":read" 命令会为当前缓冲区设置文件名。

cpo-F

F 如果包含并且当前缓冲区还没有文件名，带文件名参数的 ":write" 命令会为当前缓冲区设置文件名。另见 cpo-P 。

cpo-g

g 没有参数的 ":edit" 会转到第一行。

cpo-H

H 如果在只有空白的行使用 "I"，在最后一个空白前插入。没有此标志位，在最后一个空白后插入。

cpo-i

i 如果包含，读入时被中断的文件会使其状态成为已修改。

cpo-I

I 'autoindent' 插入缩进后上下移动光标不删除缩进。

cpo-j

j 连接行时，只有在 '.' 之后加入两个空格，'!' 或 '?' 之后不会。另见 'joinspaces'。

cpo-J

J sentence 必须在 '.'，'!' 或 '?' 有两个空格才算。<Tab> 不识别为空白。

cpo-k

k 映射、缩写和菜单命令的 "to" 部分不识别原始键盘代码。比如，如果 <Key> 发送 ^[OA (其中的 ^[是 <Esc>)，命令 ":map X ^[OA" 会使得 X 被映射为：

包含 'k' 时： "^[OA" (3 个字符)

排除 'k' 时： "<Key>" (单个键码)

另见下面的 '<' 标志位。

cpo-K

K 如果映射只完成键码的一部分，不等待其完成。这会使映射 <F1><F1> 在只读入第二个 <F1> 的一部分时无法工作。不过，这意味着用 <F1><Esc> 就可以中止该映射。

cpo-l

l 搜索模式里的 [] 范围内的反斜杠按本义出现，只有 "\]"、"\^"、"\-" 和 "\\" 是特别的。见 /[]

包含 'l' 时："/[\t]" 找到 <Space>、'\' 和 't'

排除 'l' 时："/[\t]" 找到 <Space> 和 <Tab>

另见 cpo-\ 。

cpo-L

L 如果置位 'list' 选项，'wrapmargin'、'textwidth'、'softtabstop' 和虚拟替换模式 (见 gR) 把 <Tab> 看成两个字符，和 <Tab> 的普通行为不同。

cpo-m

m 如果包含，showmatch 总会等到半秒。如果不包含，showmatch 等待半秒或者到有字符输入为止。 'showmatch'

cpo-M

M 如果排除，"%" 匹配会考虑反斜杠。所以 "(\( )" 和 "\((\)" 里，外层的括号会匹配。如果包含，"%" 忽略反斜

杠，这是 Vi 兼容行为。

cpo-n

n 如果包含，'number' 和 'relativenumber' 所用的列也用于文本的回绕行。

cpo-o

o 不记住搜索命令的行偏移并将它用于下一个搜索。

cpo-O

O 如果文件被覆盖，即使开始编辑时该文件还不存在，也不会抱怨。这项措施保护你不会意外地覆盖别人建立的文件。Vi 并不会为此抱怨。

cpo-p

p Vi 兼容的 Lisp 缩进。如果不包含，使用稍好一些的算法。

cpo-P

P 如果包含，当当前缓冲区还没有文件名并且同时包含 'F' 标志位 cpo-F 时，附加到文件后的 ":write" 命令会设置当前缓冲区的文件名。

cpo-q

q 连接多行时，把光标留在连接两行时应该在的位置。

cpo-r

r 重复 ("." 命令) 使用 "/" 来重复搜索命令，而不一定使用实际用过的搜索字符串。

cpo-R

R 删除过滤行的位置标记。没有该标志位，标记会被保存，就像用了 :keepmarks 那样。

cpo-s

s 第一次进入缓冲区时，设置缓冲区选项。这和 Vim 3.0 版本的行为相同，现在这也是缺省。如果不存在，选项在缓冲区建立时设置。

cpo-S

S 每次进入缓冲区都设置缓冲区选项 (除了 'readonly'、'fileformat'、'filetype' 和 'syntax' 以外)。这是 (最) 符合 Vi 兼容性的设定。
这些选项将被设为当前缓冲区的值。如果你改变选项并切换到别的缓冲区，该值会被复制。这实际意味着，缓冲区选项全局于所有缓冲区。

's'	'S'	缓冲区选项的复制在
否	否	建立缓冲区时
是	否	第一次进入缓冲区时 (缺省)
均可	是	每次进入缓冲区时 (vi 兼容)

cpo-t

t "n" 命令记住标签命令的搜索模式。否则，Vim 只把该模式放到搜索模式的历史里，但不改变最近使用的搜索模式。

cpo-u

u 撤销的行为和 Vi 兼容。见 undo-two-ways 。

cpo-v

v 插入模式里，退格删除的字符在屏幕上保持可见。没有本标志位，该字符从屏幕上立即删除。有了本标志位，新输入的文本覆盖屏幕上退格经过的字符。

cpo-w

w 在空白字符上使用 "cw" 只改变一个字符，而不是所有到下一个单词之前的空白。

cpo-W

W 不覆盖只读文件。如果不提供，":w!" 在可能的情况下覆盖只

读文件。

cpo-x

x 命令行上输入 <Esc> 执行该命令行。
Vim 的缺省行为是放弃命令行。因为 <Esc> 通常的含义是中止命令。 c_<Esc>

cpo-X

X 使用计数时，"R" 只删除一次替换的文本。用带计数的 "." 来重复 "R" 也是如此。

cpo-y

y 抽出 (yank) 命令可以用 "." 重复。真想用此标志位时要三思，它可能会破坏插件，因为大多数人期待 "." 只重复改动。

cpo-Z

Z 置位 'readonly' 选项时，使用 "w!" 不复位 'readonly'。

cpo-!

! 重复过滤命令时，使用最后使用的外部命令，不管它是什么。否则，使用最近的 -过滤- 命令。

cpo-$

$ 对一行进行修改 (change) 时，不重显该行，但在修改文本的尾部放上 '$'。被修改的文本会被新输入的文本覆盖。如果你输入任何把光标移开插入点的命令，该行被重显。

cpo-%

% "%" 命令执行 Vi 兼容的匹配行为。
不识别 "#if"、"#endif" 等。
不识别 "/*" 和 "*/"。
单引号和双引号里的括号也被计算在内，这使得包含括号的字符串常数会影响匹配。比如，在 "if (strcmp("foo(", s))" 这样的行里，第一个括号不匹配最后一个。如果不包含本标志位，单引号和双引号里的括号被特殊处理。如果匹配引号之外的括号，引号之内的所有内容被忽略。如果匹配引号之内的括号，会找到它的匹配 (如果有的话)。这对 C 程序很管用。
此标志位还用于其它特性，例如 C 缩进。

cpo--

- 如果包含，到首行之上或末行之下的垂直移动的命令会失败。
如果不包含，光标移到首行或末行，除非本来已经在该行上。
适用于命令 "-"、"k"、CTRL-P、"+"、"j"、CTRL-N、CTRL-J 和 ":1234"。

cpo-+

+ 如果包含，":write file" 命令会复位缓冲区的 'modified' 标志位，即使缓冲区本身可能和对应的文件不同。

cpo-star

* ":*" 和 ":@" 相同。如果不包含，":*" 是 ":'<,'>" 的别名，选择可视区域。

cpo-<

< 在映射、缩写和菜单命令的 "to" 部分不识别 <> 形式的特别键码。比如，命令 ":map X <Tab>" 使得 X 被映射为:
包含 '<' 时: "<Tab>" (5 个字符)
排除 '<' 时: "^I" (^I 是真正的 <Tab>)
另见上面的 'k' 标志位。

cpo->

> 附加到寄存器时，在附加文本之前加上换行符。

cpo-;

; 用 , 或 ; 重复上次 t 搜索时，如果光标正在搜索的字符之上，光标不移动。如果不包含，光标会跳过此字符并

跳到下一个匹配之上。

POSIX 标志位。Vi 缺省值里没有，除非启动时设置 $VIM_POSIX。 posix

包含	行为
#	cpo-# "D"、"o" 和 "O" 之前的计数没有效果。
&	cpo-& 使用 ":preserve" 时，正常退出但缓冲区仍然载入时保持交换文件。退出时测试此标志位。
\	cpo-\ 搜索模式里的 [] 范围内的反斜杠按本义出现，只有 "\]" 是特别的。见 /[] 包含 '\' 时: "/[\-]" 找到 <Space>、'\' 和 '-' 排除 '\' 时: "/[\-]" 找到 <Space> 和 '-' 另见 cpo-l 。
/	cpo-/ 使用 "%" 作为 :s 命令的替代字符串时，使用最近的替代字符串。 :s%
{	cpo-{ { 和 } 命令也停在行首的 "{" 字符上。
.	cpo-. ":chdir" 和 ":cd" 命令在当前缓冲区被修改时失败，除非指定 !。Vim 并不需要这样，因为它记住打开文件的完整路径。
|	cpo-bar $LINES 和 $COLUMNS 环境变量的值否决通过系统特定的函数得到的终端大小的值。

'cryptmethod' 'cm'

'cryptmethod' 'cm' 字符串 (缺省为 "blowfish2")
全局或局部于缓冲区 global-local
把缓冲区写入到文件时所用的加密方法:

zip	pkzip PkZip 兼容方法。加密较弱。和 Vim 7.2 和之前版本兼容。
blowfish	blowfish Blowfish 方法。中等强度的加密，但有实现漏洞。需要 Vim 7.3 或之后版本，文件 不 能用 Vim 7.2 或之前版本读出。文件中包含一个加密用的 "种子"，所以每次写入文件时加密后的字节都会不相同。
blowfish2	blowfish2 Blowfish 方法。中等强度的加密。需要 Vim 7.4.401 或之后版本，文件 不 能用 Vim 7.3 或之前版本读出。文件中包含一个加密用的 "种子"，所以每次写入文件时加密后的字节都会不相同。对整个撤销文件进行加密，而不只是文本片断。

应该使用 "blowfish2"，同时应对旧文件重新加密。

读入加密文件时，'cryptmethod' 会自动设为检测到的读入文件所用之方法。这样在写入文件时，如果不手动改变 'cryptmethod'，会采用之前的方法。
改变 'cryptmethod' 并不会自动使文件标为已改动，你需要显式写入文件才会生效。除非有其它修改，你不会得到警告。另见 :X 。

如果设置全局值为空串，等同 "blowfish2"。如果设置局部值为空串，使用全局

值。

如果新版本 Vim 加入其它加密方法而当前版本不支持的话，会得到 E821 。
需要使用更新的 Vim 版本来编辑该文件。

'cscopepathcomp' 'cspc'
'cscopepathcomp' 'cspc' 数值型 (缺省为 0)
全局
{仅当编译时加入 +cscope 特性才有效}
决定标签列表里显示多少路径部分。
见 cscopepathcomp 。
注意：如果置位 'compatible'，本选项被设为 0。

'cscopeprg' 'csprg'
'cscopeprg' 'csprg' 字符串 (缺省为 "cscope")
全局
{仅当编译时加入 +cscope 特性才有效}
指定执行 cscope 的命令。见 cscopeprg 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'cscopequickfix' 'csqf'
'cscopequickfix' 'csqf' 字符串 (缺省为 "")
全局
{仅当编译时加入 +cscope 和 +quickfix 特性才有效}
设置是否使用 quickfix 窗口来显示 cscope 的结果。
见 cscopequickfix 。

'cscoperelative' 'csre' 'nocscoperelative' 'nocsre'
'cscoperelative' 'csre' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +cscope 特性才有效}
cscope 没有提供前缀 (-P) 时，置位此选项会使用 cscope.out 路径的目录名作为前缀。
见 cscoperelative 。
注意：如果置位 'compatible'，本选项被复位。

'cscopetag' 'cst' 'nocscopetag' 'nocst'
'cscopetag' 'cst' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +cscope 特性才有效}
让标签命令使用 cscope。见 cscope-options 。
注意：如果置位 'compatible'，本选项被复位。

'cscopetagorder' 'csto'
'cscopetagorder' 'csto' 数值型 (缺省为 0)
全局
{仅当编译时加入 +cscope 特性才有效}
决定 ":cstag" 执行查找的顺序。见 cscopetagorder 。
注意：如果置位 'compatible'，本选项被设为 0。

'cscopeverbose' 'csverb'
'nocscopeverbose' 'nocsverb'

'cscopeverbose' 'csverb' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +cscope 特性才有效}
增加 cscope 数据库时，给出消息。见 cscopeverbose 。
注意：如果置位 'compatible'，本选项被复位。

'cursorbind' 'crb' 'nocursorbind' 'nocrb'
'cursorbind' 'crb' 布尔型 (缺省关闭)
局部于窗口
如果置位此选项，光标在当前窗口移动时，其它光标绑定的窗口 (置位本选项的窗口) 也会移动他们的光标到对应的行和列。此选项可用于浏览文件不同版本的异同 (见 'diff')；比较模式下，已插入和删除的行 (但不包括行内删除的字符) 已经考虑在内。

'cursorcolumn' 'cuc' 'nocursorcolumn' 'nocuc'
'cursorcolumn' 'cuc' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +syntax 特性才有效}
用 CursorColumn hl-CursorColumn 高亮光标所在的屏幕列。用于对齐文本。屏幕刷新会变慢。
如果你只想为当前窗口高亮，可以使用这些自动命令：

```
au WinLeave * set nocursorline nocursorcolumn
au WinEnter * set cursorline cursorcolumn
```

'cursorline' 'cul' 'nocursorline' 'nocul'
'cursorline' 'cul' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +syntax 特性才有效}
用 CursorLine hl-CursorLine 高亮光标所在的文本行。用于方便定位光标。屏幕刷新会变慢。
激活可视模式时，为了容易看到选择的文本，不使用此高亮。

'cursorlineopt' 'culopt'
'cursorlineopt' 'culopt' 字符串 (缺省: "number,line")
局部于窗口
{仅当编译时加入 +syntax 特性才有效}
关于 'cursorline' 如何显示的逗号分隔的设置列表。
合法值:

"line"	用 CursorLine hl-CursorLine 高亮光标所在的文本行。
"screenline"	用 CursorLine hl-CursorLine 高亮光标所在的屏幕行。
"number"	用 CursorLineNr hl-CursorLineNr 高亮光标所在的行号。

特殊值:

"both"	"line,number" 值的别名。

"line" 和 "screenline" 不能同时使用。

'debug'
'debug' 字符串 (缺省为 "")

全局
可以使用这些值:
msg　　别的情况下会忽略的错误信息此时都会给出。
throw　别的情况下会忽略的错误信息此时都会给出，同时抛出例外并设置 v:errmsg 。
beep　　别的情况下会给出铃声的消息此时都会给出。
这些值可以合并，用逗号分隔。
"msg" 和 "throw" 可用于调试 'foldexpr'、'formatexpr' 或 'indentexpr'。

'define' 'def'
'define' 'def'　　字符串 (缺省为 "^\s*#\s*define")
全局或局部于缓冲区 global-local
用以找到宏定义的模式。这是一个搜索模式，和 "/" 命令使用的相同。本选项用于 "[i" 和 "[d" 等命令 include-search 。'isident' 选项在该匹配之后用于识别定义名本身:
{'define' 的匹配}{非标识符字符}{定义名}{非标识符字符}
option-backslash 说明如何插入反斜杠以包含空格和反斜杠。
缺省值是为 C 程序设计的。C++ 也可用，但如果也要包含常数类型定义的话:
^\(#\s*define\|[a-z]*\s*const\s*[a-z]*\)
也可在名字前用 "\ze" 并继续模式来检查后跟的内容。例如，对 Javascript，如果函数用 "func_name = function(args)" 来定义:
^\s*\ze\i\+\s*=\s*function(
如果函数是用 "func_name : function() {..." 定义的话:
^\s*\ze\i\+\s*[:]\s*(*function\s*(
用 ":set" 命令设置时，要加倍反引号！要避免之，用 :let 时带单引号字符串:
let &l:define = '^\s*\ze\k\+\s*=\s*function('

'delcombine' 'deco' 'nodelcombine' 'nodeco'
'delcombine' 'deco'　　布尔型 (缺省关闭)
全局
如果编辑 Unicode 时打开本选项，退格和普通模式的 "x" 删除单个组合字符。如果关闭 (缺省值) 每个字符连同它的组合字符一并删除。
注意: 置位 'delcombine' 时，"xx" 可能和 "2x" 不一样！

这对阿拉伯语、希伯来语和许多其他语言有用，那里需要在基础字符的上方使用组合字符，而有时只想删除组合的部分。
注意: 如果置位 'compatible'，本选项被复位。

'dictionary' 'dict'
'dictionary' 'dict'　　字符串 (缺省为 "")
全局或局部于缓冲区 global-local
逗号分隔的文件名列表，用于为关键字补全命令查找单词 i_CTRL-X_CTRL-K 。每个文件由单词的列表构成，可以每行一个单词，也可以每行多个，以非关键字字符分隔 (建议使用空白)。最大的行长为 510 个字节。

如果此选项为空或者给出项目 "spell"，并且打开拼写检查，使用当前激活的 'spelllang' 对应的单词列表中的单词。见 spell

要在文件名里包含逗号，在它之前加上反斜杠。逗号之后的空格被忽略，其它情况下空格视为文件的一部分。 option-backslash 说明如何使用反斜杠。
这和 Dictionary 变量类型完全无关。
什么地方可以找到单词列表呢?

- FreeBSD 上，有文件 "/usr/share/dict/words"。
- Simtel 归档里，在 "msdos/linguist" 目录里找找。
- GNU 工具集里的 "miscfiles"。

建议使用 :set+= 和 :set-= 来从列表里加减目录。这可以避免未来版本使用其它缺省值出现的问题。
为了安全原因，反引号不能用于此选项里。

'diff' 'nodiff'

'diff' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +diff 特性才有效}

加入本窗口到显示文件差异的窗口组里。见 vimdiff 。

'dex' 'diffexpr'

'diffexpr' 'dex' 字符串 (缺省为 "")
全局
{仅当编译时加入 +diff 特性才有效}

用于计算文件两个不同版本的差异文件 (ed 风格或合并风格) 的表达式。见 diff-diffexpr 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'dip' 'diffopt'

'diffopt' 'dip' 字符串 (缺省为 "internal,filler,closeoff")
全局
{仅当编译时加入 +diff 特性才有效}

比较模式的设置选项。由以下几个项目组成。全都是可选的。项目之间由逗号分隔。

filler 显示填充行，使得文本和在相同位置多出文本的其它窗口同步。多用于并排并且置位 'scrollbind' 的窗口。

context:{n} 在更改和包含未更改行的折叠之间保留的上下文行数 {n} (译者注：即差异文周围不被折叠的行数)。如果省略，使用六行上下文。如果为零，实际上下文其实是一，因为折叠需要之间至少有一行，也适用于删除行。
见 fold-diff 。

iblank 忽略全空白行的改动。如果 'diffexpr' 为空，给 "diff" 命令加上 "-B" 标志位。查阅 "diff" 文档了解该标志位具体的行为。
注意：差异窗口会失去同步，因为不计算空白行间的区别。

icase 忽略文本大小写的更改。"a" 和 "A" 视为等同。如果 'diffexpr' 为空，在 "diff" 命令里加入 "-i" 标志位。

iwhite 忽略空白字符数目的更改。如果 'diffexpr' 为空，在 "diff" 命令里加入 "-b" 标志位。查阅 "diff" 文档了解该标志位具体的行为。比如，它应该忽略行尾的空白，但不忽略行首的。

iwhiteall　　忽略所有空白字符的更改。如果 'diffexpr' 为空，在 "diff" 命令里加入 "-w" 标志位。查阅 "diff" 文档了解该标志位具体的行为。

iwhiteeol　　忽略行尾空白字符的更改。如果 'diffexpr' 为空，在 "diff" 命令里加入 "-Z" 标志位。查阅 "diff" 文档了解该标志位具体的行为。

horizontal　　以水平分割启动比较模式（除非显式指定不要如此）。

vertical　　以垂直分割启动比较模式（除非显式指定不要如此）。

closeoff　　置位 'diff' 的窗口关闭时，如果同一个标签页中只剩下一个置位 'diff' 的窗口时，在那个窗口中执行 :diffoff 。这会撤销 :diffsplit 命令的效果。

hiddenoff　　不对隐藏缓冲区使用比较模式。

foldcolumn:{n}　　启动比较模式时设置 'foldcolumn' 选项为 {n}。如果不存在，用 2。

internal　　使用内部比较库。设置 'diffexpr' 时忽略此项目。 E960
写入缓冲区时如果内存溢出，在涉及该缓冲区的比较中忽略此项目。要了解这在什么时候发生过，见 'verbose' 选项。

indent-heuristic
　　内部比较库使用缩进启发。

algorithm:{text}　内部比较引擎使用指定的比较算法。目前支持的算法是：

myers	缺省算法
minimal	花费额外时间来生成最小的可能差异
patience	patience 比较算法
histogram	histogram 比较算法

示例：

```
:set diffopt=internal,filler,context:4
:set diffopt=
:set diffopt=internal,filler,foldcolumn:3
:set diffopt-=internal  " _不_使用内部比较解析器
```

'digraph' 'dg' 'nodigraph' 'nodg'
'digraph' 'dg'　　布尔型 （缺省关闭）
全局
{仅当编译时加入 +digraphs 特性才有效}
允许插入模式下使用 {char1} <BS> {char2} 输入二合字母。见 digraphs 。
注意：如果置位 'compatible'，本选项被复位。

'directory' 'dir'
'directory' 'dir'　　字符串 （Amiga 的缺省为："."，t:"，
MS-DOS 和 Win32: ".,$TEMP,c:\tmp,c:\temp"

Unix: ".,~/tmp,/var/tmp,/tmp")
全局

逗号分隔的交换文件的目录名列表。
- 交换文件会在第一个可能的目录里建立。
- 空意味着不会建立交换文件（不能恢复！）且不给出 E303 错误。
- 目录 "." 意味着把交换文件放在和被编辑文件相同的目录里。Unix 上，文件名字前面会附加句号，使得目录列表里不出现该文件。MS-Windows 上，置位 "hidden" 属性并在可能的情况下附加句号。
- "./"（或 MS-DOS 等环境的 ".\"）开始的目录意味着把交换文件放在被编辑文件所在的相对位置。开头的 "." 用被编辑文件的路径名替换。
- Unix 和 Win32 上，如果目录以两个路径分隔符 "//" 结尾，交换文件名会由文件的完整路径把所有的路径分隔符替换为百分号 '%' 来构造。此方法保证了备份目录中文件名的唯一性。
  Win32 上，也可以用 "\\" 结尾。不如，如果后面有逗号，必须使用 "//"，因为 "\\" 会把逗号当做文件名的一部分。所以，建议使用 '//' 而不是 '\\'。
- 逗号之后的空格被忽略，其它的空格视为目录名的一部分。要在目录命令开头包含空格，在它之前加入反斜杠。
- 要在目录名包含逗号，在它之前加入反斜杠。
- 目录名可以用 ':' 或 '/' 结尾。
- 环境变量被扩展 :set_env 。
- 小心使用 '\' 字符，在空格前输入一个，真正的反斜杠前要输入两个（见 option-backslash)。例如:
  :set dir=c:\\tmp,\ dir\\,with\\,commas,\\\ dir\ with\ spaces
- 为了和 Vim 3.0 版本后向兼容，本选项开头的 '>' 被删除。

建议列表里的第一个是 "."。这意味着编辑相同文件两次会得到警告。Unix 上不鼓励使用 "/tmp": 系统崩溃时交换文件会丢失。
重启时，通常不会清除 "/var/tmp"，这样它比 "/tmp" 安全一些。但可能那里文件太多，而你的交换文件会迷失于其中。所以，建议先尝试你自己主目录下面的 "tmp" 目录。
建议使用 :set+= 和 :set-= 来从列表里加减目录。这可以避免未来版本使用其它缺省值出现的问题。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'display' 'dy'
'display' 'dy' 字符串 (缺省为 "", defaults.vim 里设为 "truncate")
全局

改变文件显示的方式。这是逗号分隔的标志位的列表:

lastline	如果包含，显示窗口末行尽量多的内容。如果不包含，不能在窗口完全放入的末行会以 "@" 行代替。在屏幕最后一行的最后几列会放上 "@@@"，以指示该行的其余部分不再显示。
truncate	类似于 "lastline"，但 "@@@" 显示在屏幕最后一行的首列。优先于 "lastline"。
uhex	显示不可显示字符的十六进制值，形如 <xx>，而不是用 ^C 和 ~C 这样的形式。

如果既没有 "lastline" 也没有 "truncate"，不能完全包含的末行会以 "@" 行代替。

'eadirection' 'ead'
'eadirection' 'ead' 字符串 (缺省为 "both")
全局

说明如何应用 'equalalways' 选项:

ver	垂直方向，窗口宽度不受影响

hor　　水平方向，窗口高度不受影响
both　　窗口宽度和高度都受影响

'ed'　'edcompatible'　'noed'　'noedcompatible'
'edcompatible' 'ed'　　布尔型　(缺省关闭)
全局
使得 ":substitute" 命令的 'g' 和 'c' 标志位在每次使用时切换。见 complex-change 。另见 'gdefault' 选项。
打开此选项可能会破坏插件的使用！

'emoji'　'emo'　'noemoji'　'noemo'
'emoji' 'emo'　布尔型 (缺省: 打开)
全局
如果打开，所有的 Unicode 表情字符作为全宽字符处理。

'encoding'　'enc'　E543
'encoding' 'enc'　　字符串 (缺省: "latin1" 或者 $LANG 的值)
全局
设置 Vim 内部使用的字符编码。它应用于缓冲区、寄存器、表达式所用的字符串、viminfo 保存的等等各种文本。本选项设置 Vim 可以工作的字符类型。可用的值见 encoding-names 。

注意：改变此选项不会改变 Vim 已有文本的编码。但可能会使非 ASCII 文本变成非法。通常，应该保持其为缺省值，或者只有在 Vim 启动时设置之。见 multibyte 。要重新载入菜单，见 :menutrans 。

不能从 modeline 设置本选项。否则很有可能破坏文本。

注意：GTK+ 2 或更新版本上高度建议设置 'encoding' 为 "utf-8"。尽管已经付出一定努力使得设置别的 'encoding' 值成为可能，"utf-8" 是该环境下的自然选择，而且可以避免不必要的转换开销。"utf-8" 之所以没有成为缺省，是为了避免 GUI 和终端的行为不同步，也避免在你不知情的情况下改变新建立文件的编码 (如果 'fileencodings' 为空的话)。

文件的字符编码可以和 'encoding' 不同。这由 'fileencoding' 指定。转换通过 iconv() 完成或者通过 'charconvert' 指定。

如果你需要知道 'encoding' 指定的是否是多字节编码，可用:
```
if has("multi_byte_encoding")
```

通常，'encoding' 应该等同于你现有的 locale。如果 Vim 能识别环境变量的设定，这应是本选项的缺省值。如果 'encoding' 没有设为你现有的 locale，必须设定 'termencoding' 来转换输入和显示的文本。见 encoding-table 。

设置此选项激活 EncodingChanged 自动命令事件，使得你可以在必要时设置字体。

设置此选项时，其值被转成小写。所以，你也可以用大写的值来设置。下划线被换成 '-' 符号。
如果该编码能够识别，它被转换为标准值。比如 "Latin-1" 成为 "latin1"，"ISO_88592" 成为 "iso-8859-2"，还有 "utf8" 成为 "utf-8"。

注意：如果无法识别该编码，总是使用 "latin1"。只有编辑的文件实际使用相

同的编码才不会有问题！如果实际的字符集不是 latin1，确保 'fileencoding' 和 'fileencodings' 为空。如果转换是必要的，切换该值为 utf-8。

如果使用 "unicode"、"ucs-2" 或 "ucs-4"，Vim 内部总是使用 utf-8。编辑时你可能没有注意到这一点，但使用 viminfo-file 就有关系了。还有，Vim 此时假定你的终端使用 utf-8。所以，设置 'encoding' 为这里某个非 utf-8 的值唯一的效果是指定 'fileencoding' 为空时文件使用的编码。

如果设置 'encoding' 为 Unicode 编码并且还没有设置过 'fileencodings'，'fileencodings' 的缺省值相应改变。

'endofline' 'eol' 'noendofline' 'noeol'
'endofline' 'eol' 布尔型 (缺省打开)
局部于缓冲区
写入文件时如果关闭本选项并且打开 'binary' 选项，或者关闭 'fixeol' 选项，文件的末行不写入 <EOL>。编辑新文件时自动打开或复位本选项，取决于该文件的末行原本有没有 <EOL>。通常你不需要自己置位或者复位本选项。
如果 'binary' 关闭并且 'fixeol' 打开，写入文件时不使用本值。如果 'binary' 打开或 'fixeol' 关闭，本值被用于记住文件末行是否存在 <EOL>，这样在写回文件时，就不会改变原来文件的使用方式。不过如果你愿意改变之，可自便。

'equalalways' 'ea' 'noequalalways' 'noea'
'equalalways' 'ea' 布尔型 (缺省打开)
全局
如果打开，所有窗口在分割或关闭某窗口时自动使用相同的尺寸。即使此选项已经打开也会如此。如果关闭，分割窗口会减少当前窗口的尺寸，并保持其它窗口不变。如果关闭窗口，额外的空间分配给紧邻的窗口 (取决于 'splitbelow' 和 'splitright')。
如果混合垂直和水平分割窗口，计算最小的尺寸，如果有空间的话，适当增加某些窗口的大小。 'eadirection' 选项设定在哪个方向上改变尺寸。要防止某个窗口高度和宽度被改变，分别设置 'winfixheight' 和 'winfixwidth'。
建立新窗口时如果指定窗口大小，目前不试图使尺寸相同 (比较复杂，将来也许会实现)。

'equalprg' 'ep'
'equalprg' 'ep' 字符串 (缺省为 "")
全局或局部于缓冲区 global-local
"=" 命令使用的外部程序。如果本选项为空，使用内部的排版函数: 'lisp'、'cindent' 或 'indentexpr'。如果 Vim 编译时未带内部的排版功能，使用外部的 "indent" 程序。
环境变量被扩展 :set_env 。 option-backslash 说明如何包含空格和反斜杠。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'errorbells' 'eb' 'noerrorbells' 'noeb'
'errorbells' 'eb' 布尔型 (缺省关闭)
全局
错误信息响铃 (鸣叫或屏幕闪烁)。只有对错误信息有意义。很多没有消息的错误也会使用该响铃 (比如，普通模式里按 <Esc>)。'visualbell' 说明如何设置响铃的行为: 鸣叫、屏幕闪烁或什么都不做。'belloff' 可以细调响铃行为。

'errorfile' 'ef'
'errorfile' 'ef' 字符串 (Amiga 缺省: "AztecC.Err",

其它: "errors.err")
全局
{仅当编译时加入 +quickfix 特性才有效}
quickFix 模式使用的错误文件名（见 :cf ）。
如果使用 "-q" 命令行参数，'errorfile' 设为它其后的参数。见 -q 。
不 用于 ":make" 命令。为此目的，见 'makeef'。
环境变量被扩展 :set_env 。
option-backslash 说明如何包含空格和反斜杠。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'errorformat' 'efm'
'errorformat' 'efm' 字符串 （缺省很长）
全局或局部于缓冲区 global-local
{仅当编译时加入 +quickfix 特性才有效}
类似于 scanf 风格的错误文件的行格式的描述（见 errorformat ）。

'esckeys' 'ek' 'noesckeys' 'noek'
'esckeys' 'ek' 布尔型 （Vim 缺省: 打开，Vi 缺省: 关闭）
全局
插入模式识别 <Esc> 开始的功能键。如果关闭此选项，光标和功能键，如果它们以 <Esc> 开头的话，不能用于插入模式。它的优点是，单个 <Esc> 可以被马上识别，而不用等待一秒。除了复位此选项以外，你也可以改变 'timeoutlen' 和 'ttimeoutlen' 的值。注意 如果 'esckeys' 关闭，你仍然可以建立相关的映射，但缺省情况下无法使用光标键。
注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。
备注: 此选项如果关闭，在插入模式会屏蔽 modifyOtherKeys 功能，以免每个带修饰符的键都会退出插入模式。

'eventignore' 'ei'
'eventignore' 'ei' 字符串 （缺省为 ""）
全局
自动命令事件名的列表，它们将被忽略。
如果设为 "all" 或者其中一个项目是 "all"，忽略所有的自动命令事件，自动命令将不会执行。
否则，此为逗号分隔的事件名列表。例如:
```
:set ei=WinEnter,WinLeave
```

'expandtab' 'et' 'noexpandtab' 'noet'
'expandtab' 'et' 布尔型 （缺省关闭）
局部于缓冲区
插入模式里: 插入 <Tab> 时使用合适数量的空格。如果 'autoindent' 打开，'>' 和 '<' 命令使用空格进行缩进。'expandtab' 打开时，要插入实际的制表，可用 CTRL-V<Tab>。另见 :retab 和 ins-expandtab 。
'paste' 选项置位时复位本选项，'paste' 选项复位时恢复本选项。
注意: 如果置位 'compatible'，本选项被复位。

'exrc' 'ex' 'noexrc' 'noex'
'exrc' 'ex' 布尔型 （缺省关闭）
全局
允许读入当前目录的 .vimrc，.exrc 和 .gvimrc。

设置此选项是个潜在的安全漏洞。例如，考虑解压文件包或从 github 获取文件

时，那里如果有 .vimrc，可以放置有特洛伊木马。 最好不要用本选项 ！
一个替代方案是，在 .vimrc 里定义自动命令，为匹配的目录设置选项。

如果打开此选项，你需要考虑同时置位 'secure' 选项 (见
initialization)。
另见 .vimrc 和 gui-init 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'fileencoding' 'fenc' E213
'fileencoding' 'fenc' 字符串 (缺省: "")
局部于缓冲区
设置此缓冲区所在文件的字符编码。
如果 'fileencoding' 不同于 'encoding'，写文件时需要进行转换。读文件时见下。
如果 'fileencoding' 为空，使用 'encoding' 相同的值 (而读写文件也不需要转换)。
设置此值时不会报错，只有在使用的时候，写入文件的时候才会。
如果 'encoding' 和 'fileencoding' 都是某种 Unicode 编码而且 'fileencoding' 不是 utf-8，会进行转换。这是因为 Unicode 在内部总是使用 utf-8 编码进行存储。
警 告: 转换可能导致信息的丢失！如果 'encoding' 为 "utf-8" 或其它 Unicode 编码，那么转换的结果通过逆转换很有可能产生相同的文本。相反，如果 'encoding' 不是 "utf-8"，一些字符可能会丢失！
'encoding' 说明本选项可用的值。另外，也许还有其它转换器能够处理的值可用，见 mbyte-conversion 。
读入文件时，'fileencoding' 会根据 'fileencodings' 进行设置。
要用特定编码读入文件，不能设置 'fileencoding'，用 ++enc 参数。但有一个例外: 如果 'fileencodings' 为空，则使用 'fileencoding'。
新文件使用全局的 'fileencoding' 值。
这里，在前面附加 "8bit-" 和 "2byte-" 没有意义，它们也被忽略。
设置本选项时其值被转换为小写。所以，你设置大写的值也没关系。
'_' 字符被替换成 '-'。如果是一个 'encoding' 列表里能识别的名字，它被替换为标准名。比如 "ISO8859-2" 成为 "iso-8859-2"。
开始编辑文件后，如果设置本选项，'modified' 选项被置位，因为文件被认为已经和当初写入时不同。
记住从模式行设置 'fenc' 发生的时刻是在文本读入 之后 ，所以它只影响文件的写入。如果你确实要在模式行上设置 'fenc'，最后同时设置 'nomodified'，否则会不能 ":q"。

本选项在 'modifiable' 关闭时不能改变。

'fe'
注意: 在 6.0 版本之前，本选项设定整个 Vim 所用的编码，这是一个错误。现在用 'encoding' 代替。旧的简称是 'fe'，现已不再使用。

'fileencodings' 'fencs'
'fileencodings' 'fencs' 字符串 (缺省: "ucs-bom",
如果 'encoding' 为某个 Unicode 的值,
"ucs-bom,utf-8,default,latin1")
全局
这是一个字符编码的列表，开始编辑已存在的文件时，参考此选项。如果文件被读入，Vim 尝试使用本列表第一个字符编码。如果检测到错误，使用列表的下一个。如果找到一个能用的编码，设置 'fileencoding' 为该值。如果全都失败，'fileencoding' 设为空字符串，这意味着使用 'encoding' 的值。

警 告：转换可能导致信息的丢失！如果 'encoding' 为 "utf-8"（或某个其它的 Unicode 变种），那么转换的结果通过逆转换很有可能产生相同的文本。相反，如果 'encoding' 不是 "utf-8"，一些非 ASCII 的字符可能会丢失！你可用 ++bad 参数指定转换失败时的行为。

对于空文件或者只含 ASCII 字符的文件，多数编码都可以工作，因而会使用 'fileencodings' 的第一项（除了 "ucs-bom" 以外，它需要 BOM）。如果你更愿意别的编码，用 BufReadPost 自动命令事件来测试是否需要使用你所喜欢的编码。比如：

```
au BufReadPost * if search('\S', 'w') == 0 |
        \ set fenc=iso-2022-jp | endif
```

如果文件没有非空白字符，该代码设置 'fileencoding' 为 "iso-2022-jp"。如果用 ++enc 参数，不使用 'fileencodings' 的值。

注意 'fileencodings' 不用于新文件。此时，使用 'fileencoding' 的全局值。你可以这样设置：

```
:setglobal fenc=iso-8859-2
```

这意味着不存在的文件可能和空文件得到不同的编码。

特殊值 "ucs-bom" 可用来检查文件开始处的 Unicode 的 BOM（Byte Order Mark 字节顺序标记）。要使之能正常工作，不能把 "utf-8" 或别的 Unicode 编码放在它的前面。

8 位编码的项目（比如，"latin1"）应该放在最后，因为 Vim 不会检测出错，因而总会接受该编码。

特殊值 "default" 可以用来指定环境使用的编码。这是 'encoding' 的缺省值。可用于把 'encoding' 设为 "utf-8" 而环境使用非 latin1 编码的场合，例如俄语。

如果 'encoding' 为 "utf-8" 而文件包含非法的字节序列，它不会被识别为 UTF-8。可用 8g8 命令寻找非法的字节序列。

错误的值:	错在哪里:
latin1,utf-8	总是使用 "latin1"
utf-8,ucs-bom,latin1	utf-8 文件里的 BOM 不会被识别
cp1250,latin1	总是使用 "cp1250"

如果 'fileencodings' 为空，不修改 'fileencoding'。

'fileencoding' 说明本选项可能的值。

直到下一次读入文件时，此选项的设置才会有效。

'fileformat' 'ff'

字符串 (MS-Windows 的缺省: "dos", Unix、macOS 的缺省: "unix")

局部于缓冲区

给出当前缓冲区的 <EOL> 格式，用于从文件读入缓冲区和把缓冲区写回文件：

dos	<CR> <NL>
unix	<NL>
mac	<CR>

如果使用 "dos"，忽略文件尾的 CTRL-Z。

见 file-formats 和 file-read 。

文件使用的字符编码见 'fileencoding'。

如果置位 'binary'，忽略 'fileformat' 的值。文件输入/输出如同它被设为 "unix" 那样。

文件开始编辑时，如果 'fileformats' 非空而 'binary' 关闭，本选项被自动设置。

开始编辑文件后，如果设置本选项，'modified' 选项被置位，因为文件被认为和当初写入时已经不同。

本选项在 'modifiable' 关闭时不能改变。

为了后向兼容：如果本选项设为 "dos"，置位 'textmode'，否则，复位之。

'fileformats' 'ffs'

'fileformats' 'ffs' 字符串 (缺省:
Vim+Vi MS-Windows: "dos,unix",
Vim Unix、macOS: "unix,dos",
Vi Cygwin: "unix,dos",
Vi 其它: "")
全局

给出换行符 (<EOL>) 的格式，开始编辑新缓冲区或者读入文件到已有的缓冲区时，尝试这些格式:
- 如果为空，总是使用 'fileformat' 定义的格式。不自动设置该选项。
- 如果设为一个名字，总是为打开的新缓冲区使用该格式，也为该缓冲区相应地设置 'fileformat'。文件读入已有的缓冲区时，使用 'fileformats' 给出的名字，不管该缓冲区设定的 'fileformat' 是什么。
- 如果给出多于一个以逗号分隔的名字，读入文件时会进行自动 <EOL> 检测。开始编辑文件时，这样检查 <EOL>:
  1. 如果所有行都以 <CR><NL> 结尾，而 'fileformats' 包含 "dos" 的话，'fileformat' 设为 "dos"。
  2. 如果找到一个 <NL> 而 'fileformats' 包含 "unix" 的话，'fileformat' 设为 "unix"。注意 如果找到的 <NL> 没有前导 <CR>，"unix" 比 "dos" 优先。
  3. 如果 'fileformat' 没有设置，且文件里有 <CR>，且 'fileformats' 包含 "mac"，'fileformat' 设为 "mac"。
     这意味着 "mac" 只有在:
     没有给出 "unix" 或者在文件里没有找到 <NL>，并且
     没有给出 "dos" 或者没有在文件里找到 <CR><NL> 时
     才会使用。
     例外: 如果先选择 "unix"，但在第一个出现的 <NL> 之前有 <CR>，而文件的头几行中 <CR> 比 <NL> 看起来多的话，也使用 "mac"。
  4. 如果还是不能设置 'fileformat'，使用 'fileformats' 的第一个名字。
  读入文件到已有的缓冲区时，完成相同的步骤，但如同 'fileformat' 已经为该文件合适地设置过，不改变该选项。

如果置位 'binary'，不使用 'fileformats' 的值。

Vim 启动时使用的空缓冲区使用这里的首个项目。可以在 .vimrc 中设置 'fileformat' 进行改变。

对于使用 DOS 类型的 <EOL> (<CR><NL>) 的系统来说，读入待执行的脚本 (":source") 或者 vimrc 文件时，可能进行自动 <EOL> 的检测:
- 如果 'fileformats' 为空，没有自动检测。使用 DOS 格式。
- 如果 'fileformats' 设为一到多个名字，进行自动检测。它基于文件中的第一个 <NL>: 如果在它之前有一个 <CR>，使用 DOS 格式，不然，使用 Unix 格式。

另见 file-formats 。
为了后向兼容: 如果设置此选项为空字符串或者单一格式 (没有包含逗号)，复位 'textauto'，否则置位 'textauto'。
注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'fileignorecase' 'fic' 'nofileignorecase' 'nofic'

'fileignorecase' 'fic' 布尔型 (缺省在一般忽略文件名大小写的系统上打开)
全局

置位时，使用文件名和目录时忽略大小写。

若只要被全时忽略大小写，参见 'wildignorecase'。

'filetype' 'ft'

'filetype' 'ft' 字符串 (缺省: "")
局部于缓冲区
如果设置此选项，激活 FileType 自动命令事件。执行所有匹配本选项值的自动命令。这时，使用 'filetype' 的值匹配而不是文件名。
否则，本选项不总是反映当前的文件类型。
通常，在检测到文件类型时设置此选项。要打开这一功能，使用 ":filetype on" 命令。 :filetype
对于不能自动识别文件类型的文件而言，在模式行里设置此选项是最有用的。比如对于 IDL 文件:

```
/* vim: set filetype=idl : */
```

FileType filetypes
如果选项值里有句号，它分隔两个名字。例如:

```
/* vim: set filetype=c.doxygen : */
```

先使用 "c" 文件类型，再使用 "doxygen" 文件类型。这既适用于文件类型插件，也适用于语法文件。句号可以多于一个。
不管 'cpoptions' 里是否包含 's' 或 'S' 标志位，本选项不会复制到别的缓冲区。
只能使用普通的文件名字符。"/*?[|<>" 都不合法。

'fillchars' 'fcs'

'fillchars' 'fcs' 字符串 (缺省为 "vert:|,fold:-")
全局
{仅当编译时加入 +folding 特性才有效}
填充状态行和垂直分割线的字符。
它是逗号分隔的项目列表:

项目	缺省	用于
stl:c	' ' 或 '^'	当前窗口的状态行
stlnc:c	' ' 或 '='	非当前窗口的状态行
vert:c	'|'	垂直分割线 :vsplit
fold:c	'-'	填充 'foldtext'
diff:c	'-'	'diff' 选项里的被删除行

任何不提供的项目使用其缺省值。"stl" 和 "stlnc" 在有高亮的地方使用空格，否则使用 '^' 或 '='。

例如:

```
:set fillchars=stl:^,stlnc:=,vert:\|,fold:-,diff:-
```

除了在高亮时也使用那些字符以外，这类似于缺省值。

"stl" 和 "stlnc" 只支持单字节的值。

这些项目使用的高亮:

项目	高亮组	
stl:c	StatusLine	hl-StatusLine
stlnc:c	StatusLineNC	hl-StatusLineNC
vert:c	VertSplit	hl-VertSplit
fold:c	Folded	hl-Folded
diff:c	DiffDelete	hl-DiffDelete

'fixendofline' 'fixeol' 'nofixendofline' 'nofixeol'

'fixendofline' 'fixeol' 布尔型 (缺省打开)
局部于缓冲区
写入文件且本选项打开时，恢复文件尾缺失的 <EOL>。如果你要保留文件原本的样子，关闭本选项。
'binary' 选项置位时，忽略本选项。
见 'endofline' 选项。

'fkmap' 'fk' 'nofkmap' 'nofk'
'fkmap' 'fk' 布尔型 (缺省关闭) E198
全局
{仅当编译时加入 +rightleft 特性才有效}
此选项曾用于支持波斯语，此支持已删除。见 farsi.txt 。

'foldclose' 'fcl'
'foldclose' 'fcl' 字符串 (缺省为 "")
全局
{仅当编译时加入 +folding 特性才有效}
如果设为 "all"，不包含光标且级别高于 'foldlevel' 的折叠被关闭。用于光标移出后，自动关闭折叠。

'foldcolumn' 'fdc'
'foldcolumn' 'fdc' 数值型 (缺省为 0)
局部于窗口
{仅当编译时加入 +folding 特性才有效}
如果非零，指定宽度的列在窗口的一侧显示，指示折叠的打开和关闭。最大值为 12。
见 folding 。

'foldenable' 'fen' 'nofoldenable' 'nofen'
'foldenable' 'fen' 布尔型 (缺省打开)
局部于窗口
{仅当编译时加入 +folding 特性才有效}
如果关闭，所有的折叠都被打开。本选项用于在文本显示的完全打开折叠和保留折叠之间 (包括手动打开或关闭的折叠) 快速切换。 zi 命令切换本选项。
如果 'foldenable' 关闭，'foldcolumn' 会保持空白。
建立新折叠或者关闭折叠的命令置位本选项。见 folding 。

'foldexpr' 'fde'
'foldexpr' 'fde' 字符串 (缺省: "0")
局部于窗口
{仅当编译时加入 +folding 和 +eval 特性才有效}
'foldmethod' 为 "expr" 时使用的表达式。每行使用它计算折叠级别。见 fold-expr 。

如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
如果 'diff' 选项置位或 'modelineexpr' 选项关闭，不能从 modeline 中设置。

计算 'foldexpr' 时不允许修改文本或者跳到其它窗口 textlock 。

'foldignore' 'fdi'
'foldignore' 'fdi' 字符串 (缺省: "#")
局部于窗口

{仅当编译时加入 +folding 特性才有效}
只有 'foldmethod' 为 "indent" 时才用到。以 'foldignore' 里的字符开始的行会从包围它的行里取得折叠级别。检查字符时，跳过空白。缺省的 "#" 对于 C 程序非常适用。见 fold-indent 。

'foldlevel' 'fdl'

'foldlevel' 'fdl' 数值型 (缺省: 0)
局部于窗口
{仅当编译时加入 +folding 特性才有效}
设置折叠级别：高于此级别的折叠会被关闭。
设置此选项为零关闭所有的折叠。更高的数字关闭更少的折叠。
zm 、 zM 和 zR 等命令设置此选项。
见 fold-foldlevel 。

'foldlevelstart' 'fdls'

'foldlevelstart' 'fdls' 数值型 (缺省: -1)
全局
{仅当编译时加入 +folding 特性才有效}
在窗口里开始编辑一个缓冲区时设置 'foldlevel'。用于开始编辑时总是关闭所有的折叠 (值为零)，关闭某些折叠 (1) 或者没有折叠 (99)。
它在读入任何模式行之前进行，因此模式行里的设置可以否决本选项的设定。开始编辑 diff-mode 的文件也忽略本选项，并关闭所有折叠。
它也在 BufReadPre 自动命令之前完成，使得自动命令可以为特定文件重新设定 'foldlevel' 的值。
如果值为负，不使用本选项。

'foldmarker' 'fmr' E536

'foldmarker' 'fmr' 字符串 (缺省: "{{{,}}}")
局部于窗口
{仅当编译时加入 +folding 特性才有效}
'foldmethod' 为 'marker' 时使用的开始和结束标志。必须有一个逗号分隔开始和结束标志。标志是一个按本义出现的字符串 (正规表达式太慢了)。
见 fold-marker 。

'foldmethod' 'fdm'

'foldmethod' 'fdm' 字符串 (缺省: "manual")
局部于窗口
{仅当编译时加入 +folding 特性才有效}
当前窗口使用的折叠方式。可能的值是:

fold-manual	manual	手动建立折叠。
fold-indent	indent	相同缩进距离的行构成折叠。
fold-expr	expr	'foldexpr' 给出每行的折叠级别。
fold-marker	marker	标志用于指定折叠。
fold-syntax	syntax	语法高亮项目指定折叠。
fold-diff	diff	没有改变的文本构成折叠。

'foldminlines' 'fml'

'foldminlines' 'fml' 数值型 (缺省: 1)
局部于窗口
{仅当编译时加入 +folding 特性才有效}
设置屏幕行数，超过该值的折叠可以关闭。也适用于手动关闭的折叠。
缺省值 1 意味着折叠只有在占据两行或更多时才能关闭。
设为零则可以关闭只占据一个屏幕行的折叠。
注意 这只对显示的效果有影响。使用 "zc" 关闭折叠以后，如果该折叠比

'foldminlines' 小，会显示为打开。但接下来的 "zc" 就可能会关闭包含该折叠的折叠。

'foldnestmax' 'fdn'

'foldnestmax' 'fdn' 数值型（缺省：20）
局部于窗口
{仅当编译时加入 +folding 特性才有效}

设置 "indent" 和 "syntax" 方法的最大折叠嵌套层数。它避免建立过多的折叠。本值不能超过 20，因为内部的限制就是 20 层。

'foldopen' 'fdo'

'foldopen' 'fdo' 字符串（缺省："block,hor,mark,percent,quickfix,search,tag,undo"）
全局
{仅当编译时加入 +folding 特性才有效}

如果某个命令移动光标到关闭的折叠，本选项指定该命令是什么类型时会打开折叠。这是一个逗号分隔的项目列表。

备注：当命令是映射一部分时不使用本选项。在映射中加入 zv 命令可以达到相同的效果。
(原因：映射本身可能想要控制折叠的打开)

项目	命令
all	任何
block	"("、"{"、"[["、"[{" 等。
hor	水平移动："l"、"w"、"fx" 等。
insert	任何插入模式下的命令
jump	远距离跳转："G"、"gg" 等。
mark	跳转到位置标记："'m"、CTRL-O 等。
percent	"%"
quickfix	":cn"、":crew"、":make" 等。
search	模式搜索："/"、"n"、"*"、"gd" 等。 (不适用于 ":" 命令里的模式搜索) 也用于 [s 和]s 。
tag	跳转到标签：":ta"、CTRL-T 等。
undo	撤销或重做："u" 和 CTRL-R

如果移动命令用作操作符（比如，"dl" 或 "y%"），不使用本选项。这意味着操作符会包含整个关闭的折叠。

注意 这里不包括垂直移动命令，否则移过关闭的折叠会非常困难。
插入模式下，插入文本时总会打开光标所在的折叠。
要关闭折叠，你可以用 zx 命令重新应用 'foldlevel'，或者把 'foldclose' 选项设为 "all"。

'foldtext' 'fdt'

'foldtext' 'fdt' 字符串（缺省："foldtext()"）
局部于窗口
{仅当编译时加入 +folding 特性才有效}

用来指定替代关闭折叠的显示文本的表达式。见 fold-foldtext 。

如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

计算 'foldtext' 时不允许修改文本或者跳到其它窗口 textlock 。

'formatexpr' 'fex'

'formatexpr' 'fex'　　字符串 (缺省 "")
局部于缓冲区
{仅当编译时加入 +eval 特性才有效}

gq 操作符或自动排版时 (见 'formatoptions') 计算如何对行范围排版的表达式。如果此选项为空，使用 'formatprg'。

v:lnum　变量保存要排版的第一行。
v:count　变量保存要排版的行数。
v:char　变量如果因为自动排版而计算该表达式，保存将要插入的字符。可以为空，这时还不要插入！

例如:

```
:set formatexpr=mylang#Format()
```

调用 mylang#Format() 函数，它在 'runtimepath' 里的 autoload/myland.vim 文件里定义。 autoload

如果设置 'textwidth' 并且新加的文本超过此界限，也计算本表达式。这和内部排版发生的条件一样。确保光标到达相对于文本而言相同的位置！ mode() 函数在此情形下会返回 "i" 或 "R"。

函数如果返回非零，Vim 依旧使用内部的排版机制。

模式行中设置时会在沙盘 sandbox 里计算此表达式。见 sandbox-option 。不过那里本选项无法达到需要的效果，因为你不能改变缓冲区的文本。'modelineexpr' 关闭时，不能在模式行中设置此选项。

注意: 如果置位 'compatible'，本选项被设为 ""。

'formatlistpat' 'flp'

'formatlistpat' 'flp'　　字符串 (缺省: "^\s*\d\+[\]:.)}\t]\s*")
局部于缓冲区

识别列表头部的模式。用于 'formatoptions' 里的 'n' 标志位。
该模式必须和下面的行要出现的缩进完全匹配。你可以用 /\ze 来标记匹配的尾部，即使其后还有更多的字符要检查。模式之后必须要有一个字符，如果匹配整行，就像没有匹配那样。
缺省设置匹配数值，后跟可选的字符和空白。

'formatoptions' 'fo'

'formatoptions' 'fo'　　字符串 (Vim 缺省: "tcq", Vi 缺省: "vt")
局部于缓冲区

描述自动排版如何进行的字母序列。见 fo-table 。如果置位 'paste' 选项，不进行排版 (如同 'formatoptions' 为空)。可以插入逗号，以增加可读性。建议使用 ":set" 的 "+=" 和 "-=" 特性，这可以避免未来版本增加新的标志位时出现的问题 add-option-flags 。

注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'formatprg' 'fp'

'formatprg' 'fp'　　字符串 (缺省为 "")
全局或局部于缓冲区 global-local

gq 命令排版时，在选择的行上使用的外部程序的名字。该程序必须能读取标准输入并在标准输出上返回结果。Unix 程序 "fmt" 是这样的一个程序。

如果 'formatexpr' 选项非空，用它而不是本选项。否则，如果本选项为空字符串，使用内部的排版函数 C-indenting 。
环境变量被扩展 :set_env 。 option-backslash 说明如何包含空格和反斜杠。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'fsync' 'fs' 'nofsync' 'nofs'
'fsync' 'fs' 布尔型 (缺省打开)
全局
如果打开，写入文件后调用库函数 fsync()。它会把文件刷新到磁盘上，确保文件即使在只做元数据 (metadata) 日志 (journaling) 的文件系统上也会被安全写入。Linux 系统的笔记本模式下，它会强迫硬盘转上一会儿，有时候你不想这样。但要警告你，关闭此选项增加崩溃时数据丢失的机率。没有 fsync() 实现的系统上，此选项总是关闭的。
另见 'swapsync'，说明如何控制交换文件的 fsync()。
'fsync' 也适用于 writefile() ，除非使用了否决它的标志位。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'gdefault' 'gd' 'nogdefault' 'nogd'
'gdefault' 'gd' 布尔型 (缺省关闭)
全局
如果打开，":substitute" 的 'g' 标志位缺省打开。这意味着所有的行内匹配都被替换，而不是只有一个。这样，":substitute" 命令里使用一个 'g' 标志位会切换一次替代使用的所有/单个匹配方式。见 complex-change 。

命令	'gdefault' 打开	'gdefault' 关闭
:s///	替代所有	替代单个
:s///g	替代单个	替代所有
:s///gg	替代所有	替代单个

注意：如果置位 'compatible'，本选项被复位。
已废弃：设置此选项可能会破坏不 注意 到此选项的插件。另外，许多用户会觉得 /g 标志位的作用和正常效果相反很混淆。

'grepformat' 'gfm'
'grepformat' 'gfm' 字符串 (缺省为 "%f:%l:%m,%f:%l%m,%f %l%m")
全局
识别 ":grep" 命令输出的格式。
这是类似于 scanf 格式的字符串，和 'errorformat' 选项使用的格式相同: errorformat 。

'grepprg' 'gp'
'grepprg' 'gp' 字符串 (缺省为 "grep -n ",
Unix: "grep -n $* /dev/null",
Win32: "findstr /n" 或 "grep -n",
VMS: "SEARCH/NUMBERS ")
全局或局部于缓冲区 global-local
:grep 命令使用的程序。本选项可以包含 '%' 和 '#' 字符，其扩展方法和命令行上的相同。占位符 "$*" 指定包含参数的位置。环境变量被扩展 :set_env 。 option-backslash 说明如何包含空格和反斜杠。
如果你的 "grep" 接受 "-H" 参数，如下的设置使得 ":grep" 也能接受单个文件:

```
:set grepprg=grep\ -nH
```

特殊值：若 'grepprg' 设为 "internal"， :grep 命令相当于 :vimgrep ，:lgrep 相当于 :lvimgrep ， :grepadd 相当于 :vimgrepadd 而 :lgrepadd 相当于 :lvimgrepadd 。
另见 :make_makeprg 一节，因为多数那里的说明也同样适用于 'grepprg'。
Win32 上，如果能找到 "findstr.exe" 的话，缺省值为 "findstr /n"。否则，缺省值为 "grep -n"。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'guicursor' 'gcr' E545 E546 E548 E549

'guicursor' 'gcr' 字符串 (缺省为 "n-v-c:block-Cursor/lCursor,
ve:ver35-Cursor,
o:hor50-Cursor,
i-ci:ver25-Cursor/lCursor,
r-cr:hor20-Cursor/lCursor,
sm:block-Cursor
-blinkwait175-blinkoff150-blinkon175",
MS-DOS 和 Win32 终端:
"n-v-c:block,o:hor50,i-ci:hor15,
r-cr:hor30,sm:block")
全局
{仅适用于 GUI 和 MS-DOS 还有 Win32 控制台}
本选项告诉 Vim 光标在不同模式的外观。GUI 里完全支持。MSDOS 或者 Win32 控制台上，只能改变光标高度，这可以通过指定块光标、或者带百分比的垂直和水平光标完成。
控制台上使用 't_SI'、't_SR' 和 't_EI' 转义序列。

本选项是逗号分隔的部分列表。每个部分由模式/位置列表和参数列表组成：
模式列表:参数列表,模式列表:参数列表,..
"模式列表" 是指连字符分隔的下面这些模式/位置的列表：
n 普通模式
v 可视模式
ve 可视模式，仅限于 'selection' 为 "exclusive" 时 (如果不是，和 'v' 相同)
o 操作符等待模式
i 插入模式
r 替换模式
c 在命令行附加
ci 在命令行插入
cr 在命令行替换
sm 插入模式下的显示匹配 ('showmatch')
a 所有模式
参数列表是连字符分隔的下面参数的列表：
hor{N} 水平线，字符高度的百分之 {N}
ver{N} 垂直线，字符宽度的百分之 {N}
block 块光标，填充整个字符
[以上三者只能且必须择一]
blinkwait{N} cursor-blinking
blinkon{N}
blinkoff{N}
光标的闪烁时间：blinkwait 指定光标开始闪烁前的延迟，blinkon 指定光标显示的时间而 blinkoff 指定光标不显示的时间。这些时间均以毫秒计。如果任何一个数值为零，就不会有闪烁的效果。缺省为：
"blinkwait700-blinkon400-blinkoff250"。这些数值用于没

有提供项目的缺省值。这意味着缺省打开闪烁。要关闭闪烁，可用 "blinkon0"。光标只有在 Vim 等待输入时才会闪烁，执行命令时不会。
要使得光标在 xterm 上闪烁，见 xterm-blink 。

{group-name}
一个高亮组名，设置光标的颜色和字体

{group-name}/{group-name}
一对高亮组名。没有语言映射时用第一个，否则用另一个。
language-mapping

"部分" 的例子：

n-c-v:block-nCursor 在普通、命令行和可视模式里，使用块光标和 "nCursor" 高亮组的颜色

i-ci:ver30-iCursor-blinkwait300-blinkon200-blinkoff150
在插入和命令行插入模式里，使用 30% 的垂直线光标和 "iCursor" 高亮组的颜色。闪烁也加快一点。

'a' 模式有所不同。它给所有的模式设置相同的参数列表。没有出现的参数并不复位为缺省值。可用于给所有模式增加一个通用的设置。例如，关闭闪烁：
"a:blinkon0"

光标高亮的例子：

```
:highlight Cursor gui=reverse guifg=NONE guibg=NONE
:highlight Cursor gui=NONE guifg=bg guibg=fg
```

'guifont' 'gfn'
E235 E596

'guifont' 'gfn' 字符串 (缺省为 "")
全局
{仅当编译时加入 GUI 的支持才有效}
这是 Vim 的 GUI 版本使用的字体列表。
最简单的形式是单个字体名。
详见 gui-font 。

'guifontset' 'gfs'
E250 E252 E234 E597 E598

'guifontset' 'gfs' 字符串 (缺省为 "")
全局
{仅当编译时打开 GUI 支持并加入 +xfontset 特性才有效}
{对 GTK+ GUI 无效}
如果非空，指定两个 (或更多) 的字体。第一个为普通的英语而设，第二个为你的特殊语言而设。见 xfontset 。

'guifontwide' 'gfw' E231 E533 E534

'guifontwide' 'gfw' 字符串 (缺省为 "")
全局
{仅当编译时加入 GUI 的支持才有效}
如果非空，指定逗号分隔的用于双宽字符的字体列表。使用第一个能载入的字体。见 gui-fontwide 。

'guiheadroom' 'ghr'

'guiheadroom' 'ghr' 数值型 (缺省为 50)
全局

{仅有 GTK 和 X11 GUI 才支持}
为了在屏幕上使 GUI 窗口合身 (fit) 而从屏幕高度减去的像素点数。在 GUI 启动前设置，比如在你的 gvimrc 文件里。如果为零，整个屏幕的高度都被窗口使用。如果为正，指定数目的像素行会被减去，以用于窗口的装饰和屏幕的其它项目。设为负值使得窗口比屏幕更高。

'guioptions' 'go' 字符串 (缺省为 "egmrLtT" (MS-Windows, defaults.vim 里去除了 "t"), "aegimrLtT" (GTK、Motif 和 Athena),)
全局
{仅当编译时加入 GUI 的支持才有效}
本选项只有在 Vim 的 GUI 版本才有效。它是字母的序列，分别描述 GUI 应该使用的部件和选项。
建议使用 ":set" 的 "+=" 和 "-=" 特性，这可以避免未来版本增加新的标志位时出现的问题 add-option-flags 。

下面是合法的标志位字符:

'go-!'
'!' 在终端窗口中执行外部命令。如果无此标志位，MS-Windows GUI 会打开一个控制台窗口来执行命令。Unix GUI 会模拟一个哑终端来列印命令的输出。
终端窗口会定位在底部，需要时向上扩展。

'go-a'
'a' 自动选择: 如果存在，无论什么时候启动可视模式或者扩展可视区域，Vim 都试图成为窗口系统全局选择区的拥有者。这意味着可以粘贴高亮的可视文本到其它应用程序甚至 Vim 自身。如果因为在文本之上进行了操作，或者应用程序需要粘贴选择区等原因使得可视区域被终止，高亮文本会被自动抽出到 "* 选择寄存器里。这样，即使在可视模式结束以后，选择区仍然可以被粘贴到别的应用程序。
如果不存在，Vim 不会成为窗口系统的全局选择区，除非使用 "* 寄存器进行抽出和删除操作，这时该选择区被显式地占有。
同样适用无模式的选择。

'go-P'
'P' 类似于自动选择，但使用 "+ 寄存器代替 "* 寄存器。

'go-A'
'A' 自动选择无模式的选择。类似于 'a'，但仅限于无模式的选择。

'guioptions'	自动选择可视	自动选择无模式
""	--	--
"a"	是	是
"A"	--	是
"aA"	是	是

'go-c'
'c' 简单的选择使用控制台对话框而不是弹出式对话框。

'go-d'
'd' 如果可能，使用深色主题变体。目前仅能用于 GTK+ GUI。

'go-e'
'e' 'showtabline' 要求时，加入标签页。
'guitablabel' 可用来改变标签文本。
如果没有 'e'，可能使用非 GUI 标签页行。
只有一些系统支持 GUI 标签页，现在包括 GTK、Motif、Mac OS/X 和

MS-Windows。

'go-f'

'f' 前台：不用 fork() 从启动外壳分叉出本 GUI 进程。用于需要等待编辑器完成的程序（例如，e-mail 程序）。你也可以用 "gvim -f" 或 ":gui -f" 来在前台启动 GUI。 gui-fork
注意：在 vimrc 文件里设置本选项。读入 gvimrc 文件时分叉操作可能已经发生。

'go-i'

'i' 使用 Vim 图标。GTK 和 KDE 上它出现在窗口的左上角。在非 GTK 的环境上，因为 X11 的限制，它是黑白的。要得到彩色图标，见 X11-icon 。

'go-m'

'm' 使用菜单栏。

'go-M'

'M' 不执行系统菜单脚本 "$VIMRUNTIME/menu.vim"。注意 本标志位必须在 .vimrc 文件里加入，在打开语法或文件类型识别之前（执行 gvimrc 文件时，系统菜单已经载入；而 ':syntax on' 和 ':filetype on' 命令同样会载入菜单）。

'go-g'

'g' 灰色菜单项：使得不活动的菜单项变灰。如果没有包含 'g'，不活动的菜单项完全不显示。
特例：Athena 总会使用灰色的菜单项。

'go-t'

't' 包含可撕下的菜单项。目前只用于 Win32、GTK+ 和 Motif 1.2 GUI。

'go-T'

'T' 包含工具栏。目前只用于 Win32、GTK+、Motif、Photon 和 Athena GUI。

'go-r'

'r' 右边的滚动条总是存在。

'go-R'

'R' 如有垂直分割的窗口，右边的滚动条总是存在。

'go-l'

'l' 左边的滚动条总是存在。

'go-L'

'L' 如有垂直分割的窗口，左边的滚动条总是存在。

'go-b'

'b' 底部的（水平）滚动条总是存在。它的大小取决于最长的可见行，或者如果包含 'h' 标志位的话，光标所在的行。 gui-horiz-scroll

'go-h'

'h' 限制水平滚动条的大小为光标所在行的长度，以减少计算量。 gui-horiz-scroll

是的，如果你真的想要，左右两边都可以有滚动条。:-)。详情可见 gui-scrollbars 。

'go-v'

'v' 对话框使用垂直的按钮布局。如果不包含，倾向使用水平布局，但如果空间不够，还是用垂直的布局。

'go-p'

'p' 使用 X11 GUI 的指针回调。有些窗口管理器需要。如果光标不闪烁或者在一定场合下变空，考虑增加此标志位。必须在启动 GUI 之前完成。在你的 gvimrc 里设置。在 GUI 启动后增加或删除不会有任何效果。

'go-F'

'F' 增加信息页脚。只适用于 Motif。见 gui-footer 。

'go-k'

'k' 加入/删除滚动条、工具条、标签页条等的时候保持 GUI 窗口大小不变。行为和最大化窗口时类似，自动调整 'lines' 和 'columns' 来适应窗口的大小。相反地，无 'k' 标志位时 Vim 在添减 GUI 组件时会试图保持 'lines' 和 'columns' 不变。

'guipty' 'noguipty'

'guipty' 布尔型 (缺省打开)
全局
{仅当编译时加入 GUI 的支持才有效}
只适用于 GUI: 如果打开，到/从外壳命令的输入/出会尝试打开伪终端 (psuedo-tty)。见 gui-pty 。

'guitablabel' 'gtl'

'guitablabel' 'gtl' 字符串 (缺省为空)
全局
{仅当编译时加入 GUI 的支持才有效}
如果非空，描述 GUI 标签页行使用的标签的文本。如果为空或结果为空，Vim 使用缺省的标签。详情见 setting-guitablabel 。

此选项的格式类似于 'statusline'。
'guitabtooltip' 用于工具提示，见下。
如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

只有显示 GUI 标签页的时候才用到。'guioptions' 必须包含 'e'。非 GUI 的标签页行使用 'tabline'。

'guitabtooltip' 'gtt'

'guitabtooltip' 'gtt' 字符串 (缺省为空)
全局
{仅当编译时加入 GUI 的支持才有效}
如果非空，描述 GUI 标签页行使用的工具提示文本。如果为空，Vim 使用缺省的工具提示。
此选项的其它方面类似于上面的 'guitablabel'。
可以包含换行符。最简单的方法是用 :let :

```
:let &guitabtooltip = "line one\nline two"
```

'helpfile' 'hf'

'helpfile' 'hf' 字符串 (缺省为 (MSDOS) "$VIMRUNTIME\doc\help.txt"
(其它) "$VIMRUNTIME/doc/help.txt")
全局
主帮助文件名。所有发布的帮助文件都应该放在一个目录里。另外，也使用所有 'runtimepath' 里的 "doc" 目录。
环境变量被扩展 :set_env 。例如: "$VIMRUNTIME/doc/help.txt"。如果没有设置 $VIMRUNTIME，也会尝试 $VIM。另见 $VIMRUNTIME 。关于如何包含空格和反斜杠，见 option-backslash 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'helpheight' 'hh'

'helpheight' 'hh'　　数值型　(缺省为 20)
全局
用 ":help" 命令打开帮助文件的最小的初始高度。帮助文件的初始高度是当前窗口的一半，或者（打开 'ea' 选项时）和其它窗口一样。如果高度小于 'helpheight'，高度设为 'helpheight'。设为零关闭此功能。

'helplang' 'hlg'

'helplang' 'hlg'　　字符串　(缺省：消息语言或为空)
全局
{仅当编译时加入 +multi_lang 特性才有效}
逗号分隔的语言列表。Vim 使用希望得到的帮助的第一个能找到的语言版本。英语帮助总是最后使用。你可以把 "en" 来把英语放在其它语言之前，但那只会找到在那个语言里存在的标签，而不是英语的帮助。
例如：

```
:set helplang=de,it
```

会先寻找德语，然后是意大利语，最后才是英语。
如果在非英语的帮助文件里使用 CTRL-] 和 ":help!"，Vim 会先寻找该语言版本的那个标签，然后才使用本选项。见 help-translated 。

'hidden' 'hid' 'nohidden' 'nohid'

'hidden' 'hid'　　布尔型　(缺省关闭)
全局
如果关闭， abandon 放弃时卸载缓冲区。如果打开， abandon 放弃时隐藏缓冲区。当然，如果缓冲区仍然在别的窗口里显示，它不会被隐藏。
在缓冲区列表里移动的命令有时会隐藏缓冲区，即使关闭 'hidden' 选项也是如此，条件是：缓冲区被修改过、'autowrite' 关闭或者不能写入、并且使用 '!' 标志位。另见 windows.txt 。
如果只想隐藏一个缓冲区，使用 'bufhidden' 选项。
":hide {command}" 为单个命令关闭本选项 :hide 。
警 告：对隐藏缓冲区的修改很容易忘记。":q!" 或 ":qa!" 时三思而后行！

'highlight' 'hl'

'highlight' 'hl'　　字符串　(缺省为（单个字符串）：

```
"8:SpecialKey,~:EndOfBuffer,@:NonText,
d:Directory,e:ErrorMsg,i:IncSearch,
l:Search,m:MoreMsg,M:ModeMsg,n:LineNr,
a:LineNrAbove,b:LineNrBelow,
N:CursorLineNr,r:Question,s:StatusLine,
S:StatusLineNC,c:VertSplit,t:Title,
v:Visual,V:VisualNOS,w:WarningMsg,
W:WildMenu,f:Folded,F:FoldColumn,
A:DiffAdd,C:DiffChange,D:DiffDelete,
T:DiffText,>:SignColumn,-:Conceal,
B:SpellBad,P:SpellCap,R:SpellRare,
L:SpellLocal,+:Pmenu,=:PmenuSel,
x:PmenuSbar,X:PmenuThumb,*:TabLine,
#:TabLineSel,_:TabLineFill,!:CursorColumn,
.:CursorLine,o:ColorColumn,q:QuickFixLine,
z:StatusLineTerm,Z:StatusLineTermNC")
```

全局
本选项可以用来为若干场合设置高亮模式。它是逗号分隔的字符对的列表。每对字符中，第一个字符给出适用的场合，第二个字符设定该场合使用的模式。这些场合是：
hl-SpecialKey　8　":map" 列出的 Meta 和特殊键

hl-EndOfBuffer	~	缓冲区末行之后的行
hl-NonText	@	窗口末尾的 '@' 和 'showbreak' 里的字符
hl-Directory	d	CTRL-D 列出的目录和列表里其它特殊的东西
hl-ErrorMsg	e	错误信息
	h	(已废弃，可以忽略)
hl-IncSearch	i	'incsearch' 高亮
hl-Search	l	最近搜索模式高亮 (见 'hlsearch')
hl-MoreMsg	m	more-prompt
hl-ModeMsg	M	模式 (例如，"-- INSERT --")
hl-LineNr	n	":number" 和 ":#" 命令的行号，也用于 'number' 和 'relativenumber' 选项置位时的行号。
hl-LineNrAbove	a	'relativenumber' 选项置位时光标上方的行号。
hl-LineNrBelow	b	'relativenumber' 选项置位时光标下方的行号。
hl-CursorLineNr	N	类似 n，用于 'cursorline' 或 'relativenumber'。
hl-Question	r	hit-enter 提示和 yes/no 问题
hl-StatusLine	s	当前窗口的状态行 status-line
hl-StatusLineNC	S	非当前窗口的状态行
hl-Title	t	":set all"、":autocmd" 等输出的标题。
hl-VertSplit	c	用来分隔垂直分割窗口的列
hl-Visual	v	可视模式
hl-VisualNOS	V	Vim 不是 "选择区拥有者" 时的可视模式。只限于 X11 GUI 的 gui-x11 和 xterm-clipboard 。
hl-WarningMsg	w	警告消息
hl-WildMenu	W	'wildmenu' 显示的通配符匹配
hl-Folded	f	关闭的折叠所用的行
hl-FoldColumn	F	'foldcolumn'
hl-DiffAdd	A	比较模式增加的行
hl-DiffChange	C	比较模式改变的行
hl-DiffDelete	D	比较模式删除的行
hl-DiffText	T	比较模式插入的文本
hl-SignColumn	>	signs 所用的列
hl-Conceal	-	用于隐藏字符的填充字符 (见 'conceallevel')
hl-SpellBad	B	拼写错误的单词 spell
hl-SpellCap	P	应该大写开头的单词 spell
hl-SpellRare	R	偏僻词 spell
hl-SpellLocal	L	来自其它区域的单词 spell
hl-Pmenu	+	弹出菜单正常行
hl-PmenuSel	=	弹出菜单选中行
hl-PmenuSbar	x	弹出菜单滚动条
hl-PmenuThumb	X	弹出菜单滚动条拇指 (thumb)

显示模式是:

r	反转	(termcap 项目 "mr" 和 "me")
i	斜体	(termcap 项目 "ZH" 和 "ZR")
b	粗体	(termcap 项目 "md" 和 "me")
s	突出	(termcap 项目 "so" 和 "se")
u	下划线	(termcap 项目 "us" 和 "ue")
t	删除线	(termcap 项目 "Ts" 和 "Te")
c	下曲线	(termcap 项目 "Cs" 和 "Ce")
n	没有高亮	
-	没有高亮	
:	使用高亮组	

没有包含在内的场合使用缺省值。
如果你想改变显示模式时所做的事，见 dos-colors 里的例子。

如果使用 ':' 显示模式，后面必须跟一个高亮组的名字。高亮组可以用来定义任何高亮的类型，包括使用颜色。 :highlight 说明如何定义之。缺省为每个场合定义一个不同的高亮组。
highlight-default 说明缺省的高亮组。

'history' 'hi'
'history' 'hi' 数值型 (Vim 缺省: 50，Vi 缺省: 0， defaults.vim 里设为 200)
全局
":" 命令的历史和最近的搜索模式的历史会被记住。本选项决定每个历史分别记住多少项（见 cmdline-editing ）。
最大值为 10000。
注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'hkmap' 'hk' 'nohkmap' 'nohk'
'hkmap' 'hk' 布尔型（缺省关闭）
全局
{仅当编译时加入 +rightleft 特性才有效}
如果打开，键盘被映射为希伯来字符集。通常，你会置位 'allowrevins' 并在插入模式里用 CTRL-_ 打开本模式。见 rileft.txt 。
注意：如果置位 'compatible'，本选项被复位。

'hkmapp' 'hkp' 'nohkmapp' 'nohkp'
'hkmapp' 'hkp' 布尔型（缺省关闭）
全局
{仅当编译时加入 +rightleft 特性才有效}
如果打开，把键盘映射设成 '根据读音的希伯来语' (phonetic)。'hkmap' 必须也打开。可用于非希伯来语的键盘。
见 rileft.txt 。
注意：如果置位 'compatible'，本选项被复位。

'hlsearch' 'hls' 'nohlsearch' 'nohls'
'hlsearch' 'hls' 布尔型 (缺省关闭)
全局
{仅当编译时加入 +extra_search 特性才有效}
如果有上一个搜索模式，高亮它的所有匹配。使用高亮的类型可以用 'highlight' 选项的 'l' 位设置。缺省，使用 "Search" 高亮组。注意 只有匹配的文本被高亮，位移此处不予考虑。
另见: 'incsearch' 和 :match 。
如果你厌倦总是看到高亮匹配，用 :nohlsearch 可以暂时关闭。它并不改变此选项的值。一旦使用搜索命令，高亮会重新出现。
'redrawtime' 指定寻找匹配会花费的最大时间。
如果搜索模式可以匹配换行符，Vim 会试图高亮所有的匹配文本。不过，这依赖于搜索从哪里开始，这会是窗口的第一行，或者关闭折叠之下的第一行，从不会显示的上一行开始的匹配可能不会在新显示的行上继续。
'viminfo' 中的 'h' 标志位 viminfo-h 可以指定启动时是否恢复高亮状态。
注意：如果置位 'compatible'，本选项被复位。

'icon' 'noicon'
'icon' 布尔型 (缺省关闭，如果标题可以恢复则打开)
全局
{仅当编译时加入 +title 特性才有效}

如果打开，窗口的图标文本会被设为 'iconstring' 的值（如果非空），否则，设为当前编辑的文件名，只使用名字的最后一部分。
可以用 'iconstring' 选项覆盖。
只适用于支持设置窗口图标的终端（目前，只有 X11 GUI 版本和带有非空的 't_ts' 选项的终端 - 缺省情况下，它们是 Unix xterm 和 iris-ansi，其中 't_ts' 从内建的 termcap 读取）。
如果 Vim 编译时定义 HAVE_X11，在可能的情况下恢复原来的图标 X11 。
X11-icon 说明如何改变 X11 的图标。
MS-Windows 可以改变图标，见 windows-icon 。

'iconstring'
'iconstring' 字符串 （缺省为 ""）
全局
{仅当编译时加入 +title 特性才有效}
如果此选项非空，它被用于窗口的图标文本。只有 'icon' 选项打开才会如此。
只适用于支持设置窗口图标的终端（目前，只有 X11 GUI 版本和带有非空的 't_ts' 选项的终端）。
不支持 MS Windows。
如果 Vim 编译时定义 HAVE_X11，在可能的情况下恢复原来的图标 X11 。如果此选项包含 printf 风格的 '%' 项目，它们根据 'statusline' 使用的规则进行扩展。'titlestring' 给出示例设置。
'modelineexpr' 关闭时，不能在模式行中设置此选项。
{仅当编译时加入 +statusline 特性才有效}

'ignorecase' 'ic' 'noignorecase' 'noic'
'ignorecase' 'ic' 布尔型 （缺省关闭）
全局
搜索模式里忽略大小写。也用于标签文件的查找。
另见 'smartcase' 和 'tagcase'。
可以在模式里使用 "\c" 或 "\C" 否决这里的设置。见 /ignorecase 。

'imactivatefunc' 'imaf'
'imactivatefunc' 'imaf' 字符串 （缺省 ""）
全局
此选项指定的函数会在输入法激活或关闭时调用。
不用于 GUI。
如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。

示例：

```
function ImActivateFunc(active)
  if a:active
    ... 做点什么
  else
    ... 做点什么
  endif
  " 返回值被忽略
endfunction
set imactivatefunc=ImActivateFunc
```

'imactivatekey' 'imak'
'imactivatekey' 'imak' 字符串 （缺省为 ""）
全局
{仅当编译时加入 +vim 和

+GUI_GTK 特性才有效} E599

指定激活 X-Window 窗口输入方法 (Input Method) 的热键。如果正确指定，vim 可以用 'imcmdline'、'iminsert' 和 'imsearch' 来完整控制 IM。
你不能用本选项来改变激活键。这个选项只是告知 Vim 这个键是什么。
格式:

```
[MODIFIER_FLAG-]KEY_STRING
```

MODIFIER_FLAG (忽略大小写) 可以使用以下字符:

```
S       Shift 键
L       Lock 键
C       Control 键
1       Mod1 键
2       Mod2 键
3       Mod3 键
4       Mod4 键
5       Mod5 键
```

接受组合，比如 "S-C-space" 或 "SC-space" 都代表 Shift+Ctrl+空格。
KEY_STRING 的格式见 <X11/keysymdef.h> 和 XStringToKeysym。

例如:

```
:set imactivatekey=S-space
```

"S-space" 代表 Shift+空格。这是 kinput2+canna (日语)，ami (韩语) (译者注: 还有 scim) 的激活键。

'imcmdline' 'imc' 'noimcmdline' 'noimc'
'imcmdline' 'imc' 布尔型 (缺省关闭)
全局
如果置位，开始编辑命令行时总是打开输入方法 (IM)，除了输入搜索模式以外 (此时适用 'imsearch')。
如果你的输入方法允许直接输入英语字符，可以置位此选项。比如，它可以使用死键 (dead key) 来输入带重音的字符。

'imdisable' 'imd' 'noimdisable' 'noimd'
'imdisable' 'imd' 布尔型 (缺省关闭，有的系统打开 (SGI))
全局
如果置位，用于关闭输入方法。IM 工作不正常时可用于关闭之。目前，SGI/IRIX 机器上缺省打开本选项。将来的版本也许会有变化。

'iminsert' 'imi'
'iminsert' 'imi' 数值型 (缺省为 0)
局部于缓冲区
指定插入模式里使用 :lmap 还是输入方法 (IM)。合法的值是:

```
0       :lmap 关闭而 IM 关闭
1       :lmap 打开而 IM 关闭
2       :lmap 关闭而 IM 打开
```

要在用 <Esc> 退出插入模式时总把本选项复位为 0，可以用:

```
:inoremap <ESC> <ESC>:set iminsert=0<CR>
```

这使得 :lmap 和 IM 在离开插入模式时自动关闭。
注意 插入模式下使用 CTRL-^ 时此选项也发生改变。 i_CTRL-^ 。
如果把 'keymap' 设为合法的键盘映射表名字，此值设为 1。
它也用于像 "r" 和 "f" 这样的命令的参数。
在 Athena 和 Motif 上，一些 XIM 方法可能使值 0 不能正确工作。这时，用 'imdisable' 关闭 XIM。

即使 Vim 编译时没有加入 +xim 、 +multi_byte_ime 或 global-ime 特性，也可用 'imactivatefunc' 和 'imstatusfunc' 来通过外部命令处理 IME/XIM。

'imsearch' 'ims' 数值型 (缺省为 -1)
局部于缓冲区
指定输入搜索模式时使用 :lmap 还是输入方法 (IM)。合法的值是:
-1 使用 'iminsert' 的值，就像 'iminsert' 同时用于搜索模式的输入一样
0 :lmap 关闭而 IM 关闭
1 :lmap 打开而 IM 关闭
2 :lmap 关闭而 IM 打开
注意 命令行模式下使用 CTRL-^ 时此选项也发生改变。 i_CTRL-^ 。
如果此值不等于 -1，在 'keymap' 设为合法的键盘映射表名时，此值设为 1。
在 Athena 和 Motif 上，一些 XIM 方法可能使值 0 不能正确工作。这时，用 'imdisable' 关闭 XIM。

'imstatusfunc' 'imsf' 字符串 (缺省 "")
全局
此选项指定的函数会被调用来得到输入法的状态。IME 激活时应返回一个正数。
不用于 GUI。

示例:

```
function ImStatusFunc()
  let is_active = ...做点什么
  return is_active ? 1 : 0
endfunction
set imstatusfunc=ImStatusFunc
```

备注: 此函数调用非常频繁。必须保持快速。
如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。

'imstyle' 'imst' 数值型 (缺省 1)
全局
{仅当编译时加入 +xim 和 +GUI_GTK 特性才有效}
此选项指定输入法的输入方式:
0 使用 on-the-spot 输入方式
1 over-the-spot 输入方式
见: xim-input-style

很长一段时间里 vim 的 GTK 版本使用 on-the-spot 方式。已知这会给映射 single-repeat 等造成麻烦。所以现在 over-the-spot 风格成为缺省。对多数人这应该没问题，如果对你造成了不便，考虑切换回 on-th-spot 风格。
如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。

'include' 'inc' 字符串 (缺省为 "^\s*#\s*include")
全局或局部于缓冲区 global-local

{仅当编译时加入 +find_in_path 特性才有效}
用来查找包含命令的模式。这是一个搜索模式，和 "/" 命令使用的相同（见 pattern ）。缺省值是为 C 程序设置的。本选项用于命令 "[i"、"]I"、"[d" 等等。
通常 'isfname' 选项用于识别跟随在此匹配模式之后的文件。但如果 "\zs" 出现在模式里，那么匹配文本从 "\zs" 到最后或如有 "\ze" 的话到 "\ze" 为止的文本用作文件名。这样可以包含比如空格这样不在 'isfname' 里的字符。
然后你可以用 'includeexpr' 来处理匹配的文本。
option-backslash 说明如何包含空格和反斜杠。

'includeexpr' 'inex'
'includeexpr' 'inex' 字符串 (缺省为 "")
局部于缓冲区
{仅当编译时加入 +find_in_path 和 +eval 特性才有效}
把 'include' 选项找到的字符串转换为文件名的表达式。Java 里，多用此将 "." 换成 "/":

```
:set includeexpr=substitute(v:fname,'\\.','/','g')
```

"v:fname" 变量会被设为检测到的文件。

如果 gf 命令找不到未经修改的文件名，也用此选项。这样，可以在 'include' 语句之后的名字上执行 "gf"。
也用于 <cfile> 。

如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

计算 'includeexpr' 时不允许修改文本或者跳到其它窗口 textlock 。

'incsearch' 'is' 'noincsearch' 'nois'
'incsearch' 'is' 布尔型 (缺省关闭，如果支持 +reltime 特性，defaults.vim 中置位)
全局
{仅当编译时加入 +extra_search 特性才有效}
输入搜索命令时，显示目前输入的模式的匹配位置。匹配的字符串被高亮。如果该模式不合法或者没有匹配，不显示任何东西。屏幕会经常刷新，所以只有对快速终端，这才有意义。
也适用于以下命令中的模式:

```
:global
:lvimgrep
:lvimgrepadd
:smagic
:snomagic
:sort
:substitute
:vglobal
:vimgrep
:vimgrepadd
```

注意 会显示匹配，但如果找不到匹配和按 <Esc> 的时候，光标会回到原来的位置。你仍然需要用 <Enter> 完成搜索命令才会移动光标到匹配位置。
用 CTRL-G 和 CTRL-T 键可以移到下个和上个匹配。 c_CTRL-G c_CTRL-T
编译时加入 +reltime 特性时，Vim 只会搜索大概半秒钟。如果模式太过复杂和/或有很多文本存在，不一定能找到匹配。这是为了避免 Vim 在输入模式的时候挂起。

可以用 'highlight' 的 'i' 标志位设置高亮模式。
如果也打开了 'hlsearch'，输入搜索命令时也会高亮所有匹配的字符串。另见：'hlsearch'。
如果不想打开 'hlsearch' 但搜索时想高亮所有的匹配，可以用 autocmd 打开和关闭 'hlsearch'。
示例：

```
augroup vimrc-incsearch-highlight
  autocmd!
  autocmd CmdlineEnter /,\? :set hlsearch
  autocmd CmdlineLeave /,\? :set nohlsearch
augroup END
```

CTRL-L 可用来在命令行上给当前匹配之后增加一个字符。如果置位了 'ignorecase' 和 'smartcase' 而命令行上没有大写字符，加入的字符被转换为小写。
CTRL-R CTRL-W 可用来在当前匹配的尾部增加单词，排除已经输入的部分。
注意：如果置位 'compatible'，本选项被复位。

'indentexpr' 'inde' 字符串 (缺省为 "")
局部于缓冲区
{仅当编译时加入 +cindent 和 +eval 特性才有效}
计算一行所需的缩进距离的表达式。在建立新行、 = 操作符和插入模式下输入 'indentkeys' 选项指定的热键时使用。
如果此选项非空，它否决 'cindent' 和 'smartindent' 的缩进计算。
如果置位 'lisp'，则本选项被自带的 Lisp 缩进算法否决。
如果置位 'paste'，不使用此选项来计算缩进。
计算此表达式时， v:lnum 设为要计算缩进的行号。光标在计算表达式时也在该行上 (但此后可以移开)。
表达式必须返回缩进对应的空格数目。它可以返回 "-1"，保持当前的缩进 (这意味着该缩进使用 'autoindent' 的计算)。
计算缩进过程中，有用的函数包括 indent() 、 cindent() 和 lispindent() 。
该表达式的计算不能有副作用！它不能改变文本，跳到别的窗口，等等。此后，光标的位置总是会被复原，所以光标可以移动。
通常，设置本选项为一个函数的调用：

```
:set indentexpr=GetMyIndent()
```

除非 'debug' 选项包含 "msg"，错误信息会被压抑。
见 indent-expression 。
注意：如果置位 'compatible'，本选项被设为 ""。

如果从模式行中设置，此表达式的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

计算 'indentexpr' 时不允许修改文本或者跳到其它窗口 textlock 。

'indentkeys' 'indk' 字符串 (缺省为 "0{,0},0),0],:,0#,!^F,o,O,e")
局部于缓冲区
{仅当编译时加入 +cindent 特性才有效}
热键的列表。在插入模式输入时，重新缩进当前行。只有在 'indentexpr' 不为空时才会发生。

格式和 'cinkeys' 完全相同。见 indentkeys-format 。
见 C-indenting 和 indent-expression 。

'infercase' 'inf' 'noinfercase' 'noinf'
'infercase' 'inf' 布尔型 (缺省关闭)
局部于缓冲区
插入模式里补全关键字时 ins-completion ，如果同时打开 'ignorecase'，匹配的大小写会根据输入进行调整。如果输入的文本包含小写字母而在那里匹配串是大写字母，补全部分被换成小写。如果输入文本没有小写字母，而且某个字母之后有大写字母而在那里匹配串是小写字母，补全部分被换成大写。
如用 'noinfercase'，照原样给出匹配。

'insertmode' 'im' 'noinsertmode' 'noim'
'insertmode' 'im' 布尔型 (缺省关闭)
全局
改变 Vim 的工作方式，使得插入模式是缺省模式。Vim 因而可以作为无模式的编辑器来使用，用于 evim 。
下面的插入模式命令会有用:
- 使用光标键移动。
- 使用 CTRL-O 执行一个单个普通模式的命令 i_CTRL-O 。如果这是映射，执行时假设 'insertmode' 是关闭的。普通模式保持激活，直到映射结束为止。
- 使用 CTRL-L 执行一系列普通模式，然后用 <Esc> 返回插入模式。注意 CTRL-L 向左移动光标，和没有置位 'insertmode' 时 <Esc> 所做的类似。 i_CTRL-L

置位 'insertmode' 时，以下项目被改变:
- 开始编辑文件时，Vim 进入插入模式。
- 插入模式里的 <Esc> 是空操作，会响铃。
- 普通模式里的 <Esc> 会使得 Vim 退回到插入模式。
- 插入模式里的 CTRL-L 是一个命令，不会被插入。
- 插入模式里的 CTRL-Z 暂停 Vim，见 CTRL-Z 。 i_CTRL-Z
不过，如果在映射里使用 <Esc>，它的行为和没有置位 'insertmode' 一样。这样，无论 'insertmode' 置位与否，都可以使用相同的映射。
:normal 执行命令时，不使用 'insertmode'。

注意: 如果置位 'compatible'，本选项被复位。

'isfname' 'isf'
'isfname' 'isf' 字符串 (Win32 缺省为:
"@,48-57,/,\,.,-,_,+,,,#,$,%,{,},[,],:,@-@,!,~,="
AMIGA: "@,48-57,/,.,-,_,+,,,$,:"
VMS: "@,48-57,/,.,-,_,+,,,#,$,%,<,>,[,],:,;,~"
OS/390: "@,240-249,/,.,-,_,+,,,#,$,%,~,="
否则: "@,48-57,/,.,-,_,+,,,#,$,%,~,=")
全局
文件名和路径名由此选项指定的字符组成。使用文件名的命令包括 "gf"，"[i" 还有在标签文件里的那些。它也用于 pattern 里的 "\f"。
多字节字符，即 256 及更高的字符，总是包含在内，只有 255 以内的字符需要在此选项里指定。
对于 UTF-8 而言，也会包含 0xa0 到 0xff 的字符。
给此选项加入空白字符时要三思而行。虽然空格可以出现在文件名中，这么做的后果使得 Vim 在补全时无法知道文件名何处开始，何处结束。'isfname' 没有空白字符也许更好些。

注意 在使用反斜杠作为路径分隔符的系统上，Vim 竭尽所能使得它如你期望的那样工作。但需要点技巧，因为 Vi 原来使用反斜杠来转义特殊字符。在这些系统上，Vim 不会删除普通文件名字符之前的反斜杠，但在 Unix 和类似的系统上会。缺省不包含 '&' 和 '^'，因为它们对 cmd.exe 有特殊意义。

本选项的格式为逗号分隔的部分的列表。每个部分是单个字符数值或者一个范围。范围包括两个字符数值，中间以 '-' 相连。字符数值可以是一个 0 到 255 的十进制数，或者是 ASCII 字符自身（不包括数字字符）。例如：

```
"_,-,128-140,#-43"      (包含 '_' 和 '-' 和 128 到 140 和 '#' 到 43)
```

如果一个部分以 '^' 开始，则后面的字符数值或范围从选项里被排除。选项的解释从左到右。排除的字符应放在包含该字符的范围之后。要包含 '^' 自身，让它成为选项的最后一个字符，或者成为范围的结尾。比如：

```
"^a-z,#,^"      (不包括 'a' 到 'z'，包括 '#' 和 '^')
```

如果字符是 '@'，所有 isalpha() 为真的字符都包含在内。通常这包括 a 到 z 和 A 到 Z，加上它们带重音的变形。要包含 '@' 自身，使用 "@-@"。例如：

```
"@,^a-z"        所有除了小写 ASCII 字母以外的字母表字符。
"a-z,A-Z,@-@"   所有字母加上 '@' 字符。
```

要包含逗号，在期待使用字符数字的地方使用它。比如：

```
"48-57,,,_"     数字、逗号和下划线。
```

逗号可以在它之前加 '^' 来排除。比如：

```
" -~,^,,9"      从空格到 '~' 的所有字符，排除逗号，加上 <Tab>。
```

option-backslash 说明如何包含空格和反斜杠。

'isident' 'isi' 字符串 (Win32 缺省为: "@,48-57,_,128-167,224-235" 否则: "@,48-57,_,192-255") 全局

标识符由此选项给出的字符组成。标识符用于识别环境变量和 'define' 选项匹配之后的名字，也用于 pattern 里的 "\i"。'isfname' 描述本选项的格式。小心：如果改变此选项，环境变量的扩展可能会失败。例如，如果包含 '/'，Vim 会试图扩展 "$HOME/.viminfo"。也许你想改变的是 'iskeyword'。

'iskeyword' 'isk' 字符串 (Vim Win32 缺省: "@,48-57,_,128-167,224-235" 其它: "@,48-57,_,192-255" Vi 缺省: "@,48-57,_") 局部于缓冲区

搜索和许多命令识别关键字："w"、"*"、"[i" 等等。它也用于 pattern 里的 "\k"。'isfname' 描述本选项的格式。对于 C 程序来说，你应该使用 "a-z,A-Z,48-57,_,.,-,>"。
对帮助文件，它设置为除了 '*'、'"' 和 '|' 以外所有非空白的可显示字符（这样在命令上使用 CTRL-] 可以找到该命令的帮助）。
如果打开 'lisp' 选项，总是包含 '-' 字符。
此选项影响语法高亮，除非语法使用了 :syn-iskeyword 。
注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'isprint' 'isp' 字符串 (Win32 和 Macintosh 缺省为: "@,~-255";

否则："@,161-255")
全局
此选项给出的字符在屏幕上能直接显示。 pattern 里的 "\p" 也使用此选项。从空格 (ASCII 32) 到 '~' (ASCII 126) 的字符总是可以直接显示的，即使它们没有包含在 'isprint' 里，甚至被排除也是如此。此选项的格式可见 'isfname' 处的描述。

不可显示字符使用两个字符显示：

```
  0 -  31       "^@" - "^_"
 32 - 126       总是单个字符
   127          "^?"
128 - 159       "~@" - "~_"
160 - 254       "| " - "|~"
  255           "~?"
```

如果 'encoding' 是某个 Unicode 编码，128 到 255 的非法字符显示为 <xx>，其中给出该字节的十六进制值。
如果 'display' 包含 "uhex"，所有的不可显示字符都显示为 <xx>。
不可显示字符使用 SpecialKey 高亮。 hl-SpecialKey

多字节字符，即 256 及更高的字符，总是包含在内，只有 255 以内的字符才需要在此选项里指定。如果字符可显示但当前字体没有包含该字符，显示一个替代字符。
不可显示并且零宽度的 Unicode 字符都显示为 <xxxx>。没有指定这些字符的选项。

'joinspaces' 'js' 'nojoinspaces' 'nojs'
'joinspaces' 'js' 布尔型 (缺省打开)
全局
使用连接命令时，在 '.'、'?' 和 '!' 之后插入两个空格。如果 'cpoptions' 包含 'j' 标志位，只在 '.' 之后插入空格。
如果不置位，只插入一个空格。
注意：置位 'compatible' 时，也置位此选项。

'key'
'key' 字符串 (缺省为 "")
局部于缓冲区
{仅当编译时加入 +cryptv 特性才有效}
用于加密和解密当前缓冲区的密钥。见 encryption 和 'cryptmethod'。
小心：不要手动设置 key 的值，别人也许看到你输入的密钥。用 :X 命令。
不过你可以手动设 'key' 为空：

```
:set key=
```

不能用 ":set key" 或 "echo &key" 得到本选项的值。以防本来不应该知道的人能够看到。这也意味着一旦设置以后，你自己也无法看到，小心不要输错！
在表达式中可用 "&key" 来判断加密是否已打开。如果 'key' 已设置，该值返回 "*****" (五颗星)。

'keymap' 'kmp' E544
'keymap' 'kmp' 字符串 (缺省为 "")
局部于缓冲区
{仅当编译时加入 +keymap 特性才有效}
键盘映射表的名字。见 mbyte-keymap 。
设置此选项为合法的键盘映射表名也会设置 'iminsert' 为 1，使得该键盘映射表生效。除非 'imsearch' 原来为 -1，它也被设为 1。
只能使用普通的文件名字符。"/*?[|<>" 都不合法。

'keymodel' 'km'

'keymodel' 'km' 字符串 (缺省为 "")
全局
逗号分隔的单词列表，打开一些特殊的键功能。可以使用下面的值：
startsel 使用 Shift + 特殊键可以开始选择 (选择模式或可视模式，取决于 'selectmode' 里有没有 "key")。
stopsel 使用非 Shift + 特殊键停止选择。
这里，特殊键指光标键、<End>、<Home>、<PageUp> 和 <PageDown>。
:behave 命令设置 'keymodel' 选项。

'keywordprg' 'kp'

'keywordprg' 'kp' 字符串 (缺省为 "man" 或 "man -s"，DOS: ":help"，VMS: "help")
全局或局部于缓冲区 global-local
K 命令使用的程序。环境变量被扩展 :set_env 。设为 ":help" 可以访问 Vim 的内部帮助。(注意 过去设置全局选项为空值可以做到这一点，现在该方法已过时。)
如果首个字符为 ":"，作为 Vim Ex 命令执行命令，前面带 [count]。
如果使用 "man"、"man -s" 或 Ex 命令，Vim 自动翻译 "K" 命令的计数，传为首个参数。"man -s" 如果没有计数就删掉 "-s"。
option-backslash 说明如何包含空格和反斜杠。
示例：

```
:set keywordprg=man\ -s
```

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'langmap' 'lmap' E357 E358

'langmap' 'lmap' 字符串 (缺省为 "")
全局
{仅当编译时加入 +langmap 特性才有效}
本选项允许你切换键盘到一种特殊的语言模式。插入模式下输入文本时直接插入这些字符。在普通模式下，'langmap' 选项把这些特殊的字符翻译成本键的原始含义。这意味着你不需要改变键盘模式就可以执行普通模式的命令。
此选项和 'keymap' 选项相反，那里，映射的是插入模式下的字符。
也要考虑复位 'langremap' 来避免 'langmap' 应用于映射产生的字符。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

示例 (希腊文，UTF-8)：
greek

```
:set langmap=ΑA,ΒB,ΨC,ΔD,ΕE,ΦF,ΓG,ΗH,ΙI,ΞJ,ΚK,ΛL,ΜM,ΝN,ΟO,ΠP,QQ,ΡR,ΣS,
```

示例 (交换命令键 z 和 y 的意义)：

```
:set langmap=zy,yz,ZY,YZ
```

'langmap' 选项是一个逗号分隔的部分列表。每个部分是两种形式之一：
1. 组对的列表。每个组对包括 "from" 字符和后面立即跟随的 "to" 字符。比如："aA"，"aAbBcC"。
2. "from" 字符的列表，分号，然后是 "to" 字符的列表。例如："abc;ABC"
例如："aA,fgh;FGH,cCdDeE"
特殊字符需要在前面加上反斜杠。它们包括 ";"、',' 和反斜杠自身。

这使你可以激活 vim 动作，而无须在语言之间来回切换。你的语言字符可以在下述情况下被理解为普通的 vim 英语字符 (根据本 langmap 的映射)：
- o 普通/可视模式 (命令，缓冲区/寄存器名，用户映射)
- o 插入/替换模式：CTRL-R 之后的寄存器名
- o 插入/替换模式：映射

命令行模式输入的字符 不 受本选项影响。 注意 任何时刻，可以改变本选项来切换不同语言/编码的映射。使用映射以免每次都要重新输入一遍！
(译者注：本选项和 :lmap 的语言映射是两码事。)

'langmenu' 'lm'

'langmenu' 'lm' 字符串 (缺省为 "")
全局
{仅当编译时加入 +menu 和 +multi_lang 特性才有效}
用于菜单翻译的语言。它指出从 'runtimepath' 里的 "lang" 目录里载入的文件是：

```
"lang/menu_" . &langmenu . ".vim"
```

(不包含空格)。例如，要使用荷兰语菜单，无论 $LANG 如何设置：

```
:set langmenu=nl_NL.ISO_8859-1
```

如果 'langmenu' 为空，使用 v:lang 代替。
只能使用普通的文件名字符。"/*?[|<>" 都不合法。
如果你的 $LANG 设为非英语，但你希望使用英语的菜单：

```
:set langmenu=none
```

本选项必须在载入菜单、打开文件类型检测或语法高亮之前设置。一旦定义了菜单，本选项就不再有效果。但你可以这么做：

```
:source $VIMRUNTIME/delmenu.vim
:set langmenu=de_DE.ISO_8859-1
:source $VIMRUNTIME/menu.vim
```

警告：这样会删除你自己定义的所有菜单！

'langnoremap' 'lnr' 'nolangnoremap' 'nolnr'

'langnoremap' 'lnr' 布尔型 (缺省关闭， defaults.vim 里置位)
全局
{仅当编译时加入 +langmap 特性才有效}
类似于 'langremap'，但反转其值。只为了后向兼容性而保留。置位 'langremap' 时， 'langnoremap' 设为其相反值，反之亦然。

'langremap' 'lrm' 'nolangremap' 'nolrm'

'langremap' 'lrm' 布尔型 (缺省打开， defaults.vim 里复位)
全局
{仅当编译时加入 +langmap 特性才有效}
关闭时，'langmap' 的设置不作用于映射产生的字符。这基本上意味着，如果你 注意 到设置 'langmap' 时屏蔽了一些你的映射，复位本选项。
为了后向兼容，此选项缺省为打开。如果可以，关闭此选项以避免破坏映射。

'laststatus' 'ls'

'laststatus' 'ls' 数值型 (缺省为 1)
全局
本选项的值影响最后一个窗口何时有状态行：

0: 永不
1: 只有在有至少两个窗口时
2: 总是

如果你有多个窗口，有状态行会使屏幕看起来好一些，但它会占据一个屏幕行。
status-line

'lazyredraw' 'lz' 'nolazyredraw' 'nolz'

'lazyredraw' 'lz' 布尔型 (缺省关闭)
全局
如果置位本选项，执行宏、寄存器和其它不通过输入的命令时屏幕不会重画。另

外，窗口标题的刷新也被推迟。要强迫刷新，使用 :redraw 。

'linebreak' 'lbr' 'nolinebreak' 'nolbr'

'linebreak' 'lbr' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +linebreak 特性才有效}

如果打开，Vim 会在 'breakat' 里的字符上，而不是在屏幕上可以显示的最后一个字符上回绕长行。和 'wrapmargin' 和 'textwidth' 不同，此处不会插入 <EOL>，它只影响文件的显示方式，而不是其内容。
如果置位 'breakindent'，视觉上对行缩进。此时，'showbreak' 的值会出现在回绕行的前面。如果 'wrap' 选项关闭，不使用本选项。
注意 <EOL> (屏幕上的) 之后的 <Tab> 字符在多数情况下，不会显示正确数量的空格。

'lines' E593

'lines' 数值型 (缺省为 24 或者终端高度)
全局

Vim 窗口的行数。
通常，你不需要设置此选项。终端初始化代码会自动完成。另见 posix-screen-size 。
如果 Vim 运行 GUI 或者在可改变大小的窗口里，设置此选项可以改变窗口的大小。如果你只想设置 GUI 的大小，在 gvimrc 文件里放入命令。
Vim 限制能在屏幕上显示的行数。你可以用此命令得到可用的最大窗口高度:

```
:set lines=999
```

最小值为 2，最大值为 1000。
如果你得到的行数少于预期，检查 'guiheadroom' 选项。
如果你设置此选项而 Vim 无法改变物理的显示行数，显示可能会混乱。

'linespace' 'lsp'

'linespace' 'lsp' 数值型 (缺省为 0，Win32 GUI 为 1)
全局
{仅有 GUI 里才支持}

字符之间插入的像素行数。如果所用字体占用全部字符单元的高度，使得行与行之间互相接触到，这会有用。
如果非零，有下划线的空间。
有些字体的行间距可能又过大 (为了给 ascent 和 descent 留出空间)。这时给 'linespace' 赋负值可能有意义。不过这样显示可能会有问题!

'lisp' 'nolisp'

'lisp' 布尔型 (缺省关闭)
局部于缓冲区
{仅当编译时加入 +lispindent 特性才有效}

Lisp 模式: 如果在插入模式下输入 <Enter>，根据 Lisp 标准 (大致上) 设置下一行缩进。也用于 "cc" 或 "S"。
必须打开 'autoindent' 才能使此选项工作。
'cpoptions' 里的 'p' 标志位改变缩进的方法: Vi 兼容或者更好的方法。
另见 'lispwords'。
在关键字可用的字符里 ('iskeyword') 加入 '-' 字符。
重定义 "=" 操作符，使用这里描述的缩进算法，而不是调用外部程序。
如果置位 'paste'，不使用本选项。

'lispwords' 'lw'

'lispwords' 'lw' 字符串 (缺省很长)
全局或局部于缓冲区 global-local

{仅当编译时加入 +lispindent 特性才有效}
逗号分隔单词的列表，影响 Lisp 的缩进方式。 'lisp'

'list' 'nolist'
'list' 布尔型 (缺省关闭)
局部于窗口
列出方式：像显示 CTRL-I 那样显示制表，行尾之后显示 $。用于看到制表和空格的区别以及拖尾的空白。'listchars' 选项可以进一步指定改变。

光标显示在制表占据的空白的开始位置，而不是像通常在普通模式那样显示在最后位置。要用空格来显示制表并同时把光标放在空白开始位置，可用：

```
:set list lcs=tab:\ \
```

注意 如果 'cpoptions' 里包含 'L'，本选项也影响排版 (用 'textwidth' 或 'wrapmargin' 方法设置的)。'listchars' 说明如何改变制表显示的方式。

'listchars' 'lcs'
'listchars' 'lcs' 字符串 (缺省为 "eol:$")
全局
'list' 模式和 :list 命令使用的字符串。这是逗号分隔的字符串设置的列表。

lcs-eol
eol:c 行尾显示的字符。如果忽略，行尾没有额外的字符。

lcs-tab
tab:xy[z] 用于显示制表的两或三个字符。第三个字符是可选的。

tab:xy 总是使用 'x'，然后使用 'y' 多次以填充。所以 "tab:>-" 显示：

```
>-
>--
等等
```

tab:xyz 总是使用 'z'，然后前面加上 'x'，然后使用 'y' 多次以填充。所以 "tab:<->" 显示：

```
<>
<->
<-->
等等
```

如果省略了 "tab:"，制表显示为 ^I。

lcs-space
space:c 用于显示空格的字符。如果忽略，空格显示为空白。

lcs-trail
trail:c 拖尾空白显示的字符。如果忽略，拖尾空白显示为空格。对拖尾的空白，此设置优先于 "space" 设置。

lcs-extends
extends:c 如果 'wrap' 关闭，而行在屏幕右边界之后还要继续时，最后一列显示的字符。

lcs-precedes
precedes:c 物理行在可视范围第一列之前还有内容时，第一个可见列显示的字符。

lcs-conceal

conceal:c　　用于替代隐藏文本的字符，用于 'conceallevel' 为 1 时。
lcs-nbsp
nbsp:c　　显示不可间断空格的字符（字符 0xA0（十进制 160）和 U+202F）。如果忽略，留出一个空格。

不应该使用字符 ':' 和 ','。如果 'encoding' 为 "utf-8"，可以使用 UTF-8 字符，不然只能使用可显示的字符。所有字符必须使用单倍宽度。

示例：

```
:set lcs=tab:>-,trail:-
:set lcs=tab:>-,eol:<,nbsp:%
:set lcs=extends:>,precedes:<
```

"NonText" 高亮会用于 "eol"、"extends" 和 "precedes"。"SpecialKey" 用于 "nbsp"、"space"、"tab" 和 "trail"。
hl-NonText　hl-SpecialKey

'lpl'　'nolpl'　'loadplugins'　'noloadplugins'
'loadplugins' 'lpl'　　布尔型　（缺省打开）
全局
如果打开，启动时载入插件脚本　load-plugins　。
你可以在　vimrc　文件里复位本选项，禁止载入插件。
注意　"-u NONE"、"-u DEFAULTS" 和 "--noplugin" 命令行参数复位本选项。见　-u　和　--noplugin　。

'luadll'
'luadll'　　字符串　（缺省取决于编译设置）
全局
{仅当编译时加入　+lua/dyn　特性才有效}
指定 Lua 共享库的名字。缺省是编译时指定的 DYNAMIC_LUA_DLL。
环境变量被扩展　:set_env　。
为了安全原因，本选项不能在　modeline　或　sandbox　里设置。

'macatsui'　'nomacatsui'
'macatsui'　　布尔型　（缺省打开）
全局
{仅用于 Mac GUI 版本}
这是绘制不能正确工作时的一个临时解决方法。如果置位并且编译时加入多字节支持，使用 ATSUI 文本绘制。如果不置位，不使用 ATSUI 文本绘制。如果你遇到绘制的问题，关闭此选项。将来的版本可能会解决这个问题，那时此选项就会被废弃。如果这样，用此方法来进行复位：

```
if exists('&macatsui')
   set nomacatsui
endif
```

另一个可以检查是否有绘制问题的选项是 'termencoding'。

'magic'　'nomagic'
'magic'　　布尔型　（缺省打开）
全局
改变搜索模式使用的特殊字符。见　pattern　。
警告：关闭此选项很有可能会破坏插件！这是因为很多模式假定此选项打开，而它关闭时会失败。只有在旧的 Vi 脚本里才需要把它关闭。任何别的情况下，使用能在 'magic' 打开状态下工作的模式。如果你一定要的话，包含 "\M" /\M　。

'makeef' 'mef'

'makeef' 'mef' 字符串 (缺省: "")
全局
{仅当编译时加入 +quickfix 特性才有效}
:make 和 :grep 命令的错误文件的名字 (见 :make_makeprg)。
如果为空，使用内部生成的临时文件。
如果包含 "##"，它被一个数值代替，使完整的名字唯一。这确保 ":make" 命令不会覆盖已有的文件。
不 用于 ":cf" 命令。那里用到的是 'errorfile'。
环境变量被扩展 :set_env 。
option-backslash 说明如何包含空格和反斜杠。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'makeencoding' 'menc'

'makeencoding' 'menc' 字符串 (缺省为 "")
全局或局部于缓冲区 global-local
读入外部命令的输出时使用的编码。如果为空，不转换编码。
用于 :make 、 :lmake 、 :grep 、 :lgrep 、 :grepadd 、 :lgrepadd 、 :cfile 、 :cgetfile 、 :caddfile 、 :lfile 、 :lgetfile 和 :laddfile 。

最常用于 MS-Windows 上设置 'encoding' 为 "utf-8" 的情况。如果打开 +iconv 且使用 GNU libiconv，设置 'makeencoding' 为 "char" 和使用系统语言编码有相同的效果。例如:

```
:set encoding=utf-8
:set makeencoding=char  " 使用系统语言
```

'makeprg' 'mp'

'makeprg' 'mp' 字符串 (缺省为 "make", VMS: "MMS")
全局或局部于缓冲区 global-local
":make" 命令使用的程序。见 :make_makeprg 。
本选项可包含 '%' 和 '#' 字符 (见 :_% 和 :_#)，分别扩展为当前和轮换文件名。如果文件名有特殊字符， ::S 可用来转义文件名。
环境变量被扩展 :set_env 。 option-backslash 说明如何包含空格和反斜杠。
注意 '|' 必须转义两次: 一次为了 ":set"，一次为了命令的解释。如果你使用名为 "myfilter" 的过滤程序，这么做:

```
:set makeprg=gmake\ \\\|\ myfilter
```

可以给出占位符 "$*" (甚至多次) 来指示包含参数的位置。例如:

```
:set makeprg=latex\ \\\\nonstopmode\ \\\\input\\{$*}
```

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'matchpairs' 'mps'

'matchpairs' 'mps' 字符串 (缺省为 "(:),{:},[:]")
局部于缓冲区
形成配对的字符。 % 命令从其中一个跳转到另一个。
可以接受字符配对的必须不同，例如，不能在两个双引号间跳转。
字符使用冒号分隔。
多个配对之间使用逗号分隔。包含 '<' 和 '>' 的例子 (HTML):

```
:set mps+=<:>
```

更奇特的例子，在赋值语句的 '=' 和 ';' 来回跳转，用于 C 和 Java 这样的语言:

```
:au FileType c,cpp,java set mps+==:;
```

更高级的 "%" 的使用方式，可见 $VIMRUNTIME/pack/dist/opt/matchit 目录的 matchit.vim 插件。 add-local-help

'matchtime' 'mat'

'matchtime' 'mat'　　数值型　(缺省为 5)
全局
如果置位 'showmatch'，显示配对括号的十分之一秒数。 注意 这不是毫秒数，和其它设置时间的选项不同。这是为了和 Nvi 兼容。

'maxcombine' 'mco'

'maxcombine' 'mco'　　数值型 (缺省为 2)
全局
最大支持的显示用组合字符的数目。
只用于 'encoding' 为 "utf-8" 的时候。
缺省值对多数语言足够了。希伯来语肯定要 4 个。
最大值为 6。
即使本选项设为 2，你仍然可以编辑有更多组合字符的文本，只不过你看不到而已。用 g8 或 ga 。见 mbyte-combining 。

'maxfuncdepth' 'mfd'

'maxfuncdepth' 'mfd'　　数值型　(缺省为 100)
全局
{仅当编译时加入 +eval 特性才有效}
用户函数的最大函数调用层数。通常用于捕获无限递归。如果使用需要更多深度的递归函数，把 'maxfuncdepth' 设大一点。不过这意味着需要更多内存，从而需要冒内存不足的风险。
增加此限制超过 200，也会改变 Ex 命令嵌套的最大层数，见 E169 。
另见 :function 。

'maxmapdepth' 'mmd' E223

'maxmapdepth' 'mmd'　　数值型　(缺省为 1000)
全局
最大次数的映射次数，在此期间必须使用一个字符。通常用于捕获无限映射。比如 ":map x y" 又 ":map y x"。它仍然不能捕获 ":map g wg"，因为在下一个映射完成之前，已经使用了 'w'。另见 key-mapping 。

'maxmem' 'mm'

'maxmem' 'mm'　　数值型　(缺省在 256 和 5120 之间 (依赖于系统) 或者可用内存的一半)
全局
单个缓冲区可用的内存的上限 (以千字节计)。如果到达此限度，为缓冲区再分配额外的内存会释放其它内存。
最大可用值大约为 2000000，此值表示不再有限制。
'swapfile' 关闭时忽略此值。
另见 'maxmemtot'。

'maxmempattern' 'mmp'

'maxmempattern' 'mmp'　　数值型　(缺省为 1000)
全局
用于模式搜索的最大内存量 (以千字节计)。最大值大约为 2000000，此值表示不再有限制。
E363
如果 Vim 到达此限额，给出错误信息，其行为基本上和按 CTRL-C 类似。

到达此限额通常意味着该模式效率非常低或者太过复杂。在很长的行上，用模式 "\(.\)*" 可能就会发生此情况。".*" 就好很多。
重画时语法规则会试图匹配复杂的文本结构，这也可能发生。
Vim 可能在到达 'maxmempattern' 限制之前就用完内存，那时你会得到 "Out of memory" 的错误。

'maxmemtot' 'mmt' 数值型 (缺省在 2048 和 10240 之间 (依赖于系统) 或者可用内存的一半)
全局
所有缓冲区可用的内存以千字节计的上限。最大可用值大约为 2000000，此值表示不再有限制。
在 64 位机器里可能可以用更高的值。嗨，你真的需要超过 2G 字节进行文本编辑吗？记住文本在交换文件里保存，所以可以编辑超过 2 Gbyte 的文件。需要足够内存来保存撤销信息。
即使关闭了 'swapfile' 的缓冲区也还是算在总内容用量里。
另见 'maxmem'。

'menuitems' 'mis' 数值型 (缺省为 25)
全局
{仅当编译时加入 +menu 特性才有效}
菜单可用的最大项目数量。用于从项目列表生成的菜单，比如 Buffers 菜单。改变此选项不会立即生效，刷新菜单后才可以。

'mkspellmem' 'msm' 字符串 (缺省 "460000,2000,500")
全局
{仅当编译时加入 +syntax 特性才有效}
:mkspell 的参数。调节何时开始压缩单词树。如果单词很多，压缩可能很慢，但为了不耗尽内存，还是有必要的。每个单词需要的内存取决于单词间的相似程度，这就是为什么此调节非常复杂的原因。

有三个数值，以逗号分隔:
{start},{inc},{added}

多数语言下，不压缩的单词树能放进内存。{start} 给出进行任何压缩前可以使用的千字节计的内存量。应该比 Vim 可用的内存量要稍小一些。

超出 {start} 限制时，{inc} 参数指定千字节计在下一次压缩进行之前要分配的内存量。数值越小意味着在加入较少的单词后就要进行压缩，从而会变慢。数值越大意味着需要分配更多内存。

压缩完成后，可以在忽略 {inc} 限制之前加入 {added} 乘以 1024 个单词，如果需要额外的内存，进行压缩。数值越小意味着到达 {inc} 限制的机率越小，使用更少的内存，但速度也更慢。

这些数值有重要意义的语言包括意大利语和匈牙利语，缺省值在你有 512 兆字节的内存比较适合。如果你有 1G 字节，可以用:
```
:set mkspellmem=900000,3000,800
```
如果你的内存不足 512 兆字节，:mkspell 可能对某些语言会失败，不管 'mkspellmem' 如何设置。

本选项不能在 modeline 或 sandbox 里设置。

'modeline' 'ml' 'nomodeline' 'noml'

'modeline' 'ml' 布尔型 (Vim 缺省: 打开 (root 用户则为关闭)
Vi 缺省: 关闭)
局部于缓冲区

如果 'modeline' 打开，'modelines' 给出检查 set 等命令的行数目。如果 'modeline' 关闭或者 'modelines' 为零，不检查任何行。见 modeline 。

'modelineexpr' 'mle' 'nomodelineexpr' 'nomle'

'modelineexpr' 'mle' 布尔型 (缺省关闭)
全局

如果打开，允许模式行上设置某些表达式选项。要看是否受 'modelineexpr' 影响，查看选项。另见 modeline 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'modelines' 'mls'

'modelines' 'mls' 数值型 (缺省为 5)
全局

如果 'modeline' 打开，'modelines' 给出检查 set 等命令的行数目。如果 'modeline' 关闭或者 'modelines' 为零，不检查任何行。见 modeline 。
注意: 如果置位 'compatible'，'modeline' 被设为 Vi 的缺省值。相反，如果复位 'compatible'，它被设为 Vim 的缺省值。

'modifiable' 'ma' 'nomodifiable' 'noma'
E21

'modifiable' 'ma' 布尔型 (缺省打开)
局部于缓冲区

如果关闭，缓冲区的内容不能修改。'fileformat' 和 'fileencoding' 选项也不能修改。
启动时用 -M 命令行参数可以复位。

'modified' 'mod' 'nomodified' 'nomod'

'modified' 'mod' 布尔型 (缺省关闭)
局部于缓冲区

如果打开，缓冲区被认为已修改。本选项在以下情形下被置位:

1. 上次写入后，对文本进行修改。使用 undo 命令回到原始的文本就会复位本选项。但撤销写回缓冲区之前的修改又会打开本选项，因为文本和写入时已经不同。
2. 'fileformat' 和 'fileencoding' 不同于原值。缓冲区读入或写回时设回原值。":set nomodified" 命令则把 "原值" 设为当前值，并复位 'modified' 选项。
   'eol' 和 'bomb' 也类似。

因为 BufNewFile、BufRead/BufReadPost、BufWritePost、FileAppendPost 或 VimLeave 自动命令事件而导致的缓冲区改变不置位此选项。相关解释见 gzip-example 。
'buftype' 为 "nowrite" 或 "nofile" 时，本选项也可置位，但会被忽略。
注意 文本可能实际上相同，例如在 "A" 上使用 "rA" 仍会置位 'modified'。

'more' 'nomore'

'more' 布尔型 (Vim 缺省: 打开，Vi 缺省: 关闭)
全局

如果打开，列表在全屏填满时暂停，并得到 more-prompt 。如果此选项关闭就不会有暂停，列表会继续进行直到结束为止。

注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'mouse' E538
'mouse' 字符串 (缺省为 ""，GUI、MS-DOS 和 Win32 为 "a"，在 defaults.vim 里设为 "a" 或 "nvi")
全局
允许使用鼠标。可用于大多数终端 (xterm、MS-DOS、Win32 win32-mouse 、QNX pterm、带 sysmouse 的 *BSD 控制台 和带 gpm 的 Linux 控制台)。关于 GUI 里如何使用鼠标，见 gui-mouse 。
可以为不同模式分别打开鼠标：

n	普通模式和终端模式
v	可视模式
i	插入模式
c	命令行模式
h	编辑帮助文件时，所有前面的模式
a	所有前面的模式
r	hit-enter 和 more-prompt 提示时

通常你会这样为所有五个模式打开鼠标：

```
:set mouse=a
```

如果你的终端不能支配发送给应用的鼠标事件，用：

```
:set mouse=nvi
```

此时你可以按 ":"，为系统选择文本，然后按 Esc 回到 Vim 来使用鼠标事件。

defaults.vim 中如果 'term' 选项不匹配 "xterm"，使用 "nvi"。

如果鼠标没有打开，GUI 仍然使用鼠标进行无模式的选择。此行为不会移动文本光标。

见 mouse-using 。另见 'clipboard' 。

注意：在终端里打开鼠标时，如果能连接到 X 服务器，复制/粘贴会使用 * 寄存器。按住 Shift 键仍然可以使用 xterm 对鼠标按钮的处理。
另见 'clipboard' 选项。

'mousefocus' 'mousef' 'nomousefocus' 'nomousef'
'mousefocus' 'mousef' 布尔型 (缺省关闭)
全局
{仅适用于 GUI}
自动激活鼠标指针所在的窗口。如果改变窗口布局或用别的方式改变窗口焦点，鼠标指针移动到带有键盘焦点的窗口。缺省是关闭的，因为它使得下拉菜单操作不太舒服，不小心移动鼠标会意外地激活窗口。
MS-Windows: 关于使用鼠标滚轮时滚动哪个窗口。另见 'scrollfocus'。

'mousehide' 'mh' 'nomousehide' 'nomh'
'mousehide' 'mh' 布尔型 (缺省打开)
全局
{仅适用于 GUI}
如果打开，输入字符时隐藏鼠标指针。鼠标移动时恢复鼠标指针。

'mousemodel' 'mousem'
'mousemodel' 'mousem' 字符串 (缺省为 "extend"，MS-DOS 和 Win32 为 "popup")
全局
设置鼠标使用的模型。这些名字主要用来设定鼠标右键的用途：

extend　　鼠标右键扩展选择区。这种工作方式和 xterm 类似。
popup　　鼠标右键弹出菜单。Shift + 鼠标左键扩展选择区。这种工作方式和 Microsoft Windows 类似。
popup_setpos　类似于 "popup"，但光标移动到鼠标点击的地方，因而菜单选择的操作会根据点击的对象而定。如果在选择区内点击，那么使用选择区进行操作而没有光标移动。当然这也意味着，在选择区之外点击右键会结束可视模式。

在不同模型下，按钮工作方式的总览：

鼠标	extend	popup(_setpos)
左键点击	放置光标	放置光标
左键拖动	开始选择	开始选择
Shift + 左键	搜索单词	扩展选择
右键点击	扩展选择	弹出菜单（放置光标）
右键拖动	扩展选择	-
中键点击	粘贴	粘贴

在 "popup" 模型里，鼠标右键产生一个弹出式菜单。你需要先定义此菜单，见 popup-menu 。

注意　你可以接着用映射重定义按钮的含义。见 gui-mouse-mapping 。但映射不用于无模式的选择（因为这是 GUI 代码直接处理的）。

:behave 命令设置 'mousemodel' 选项。

'mouseshape' 'mouses' E547

'mouseshape' 'mouses'　字符串　(缺省为 "i-r:beam,s:updown,sd:udsizing,
vs:leftright,vd:lrsizing,m:no,
ml:up-arrow,v:rightup-arrow")
全局
{仅当编译时加入 +mouseshape 特性才有效}

本选项指示 Vim 鼠标指针在不同的模式里应有的外观。它是逗号分隔的部分列表，和 'guicursor' 所用的类似。每个部分由模式/位置列表和参数列表组成：
模式列表:外型,模式列表:外型,..
"模式列表" 是指连字符分隔的下面这些模式/位置的列表：

普通窗口里：
n　普通模式
v　可视模式
ve　可视模式，仅限于 'selection' 为 "exclusive" 时（如果不是，和 'v' 相同）
o　操作符等待模式
i　插入模式
r　替换模式

其它：
c　在命令行上附加
ci　在命令行上插入
cr　在命令行上替换
m　在 'Hit ENTER' 或 'More' 等提示上
ml　同上，但光标在最后一行
e　任何模式，指针在最后一个窗口之下
s　任何模式，指针在状态行上
sd　任何模式，拖动状态行时
vs　任何模式，指针在垂直分割线上
vd　任何模式，拖动垂直分割线时

a 所有地方

外型可从下表选一:

可用	名字	看起来像
w x	arrow	普通鼠标箭头
w x	blank	没有指针 (小心使用！)
w x	beam	I 型横梁
w x	updown	上下改变大小的指针
w x	leftright	左右改变大小的指针
w x	busy	系统通常的忙指针
w x	no	系统通常的 '没有输入' 指针
x	udsizing	指示上下改变大小
x	lrsizing	指示左右改变大小
x	crosshair	像一个大而细的 +
x	hand1	黑色的手
x	hand2	白色的手
x	pencil	你用来书写的东西
x	question	大的 ?
x	rightup-arrow	指向右上方的箭头
w x	up-arrow	指向上方的箭头
x	<number>	X11 指针号 (见 X11/cursorfont.h)

"可用" 列包含 'w' 意味该外型可用于 Win32，x 代表 X11。
任何没有指定的模式或者不可用的外型都会使得普通鼠标指针被使用。

例如:

```
:set mouseshape=s:udsizing,m:no
```

会使得鼠标在状态行上变成上下改变大小的箭头，在 hit-enter 提示显示时指示没有输入 (因为此状态下，点击鼠标没有作用。)

'mousetime' 'mouset'

'mousetime' 'mouset' 数值型 (缺省为 500)
全局

只限于 GUI、MS-DOS、Win32 和使用 xterm 的 Unix。定义两个鼠标点击之间最大的使第二击被识别为多击的毫秒数。

'mzschemedll'

'mzschemedll' 字符串 (缺省取决于编译选项)
全局
{仅当编译时加入 +mzscheme/dyn 特性才有效}

指定 MzScheme 共享库名。缺省是编译时指定的 DYNAMIC_MZSCH_DLL。
环境变量被扩展 :set_env 。
本值必须在 vimrc 脚本或之前设定。在启动阶段 load-plugins 这步之前。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'mzschemegcdll'

'mzschemegcdll' 字符串 (缺省取决于编译选项)
全局
{仅当编译时加入 +mzscheme/dyn 特性才有效}

指定 MzScheme GC 共享库名。缺省是编译时指定的 DYNAMIC_MZGC_DLL。
如果 'mzschemedll' 包含 GC 代码，两者可以使用相同的库。
环境变量被扩展 :set_env 。
本值必须在 vimrc 脚本或之前设定。在启动阶段 load-plugins 这步

之前。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'mzquantum' 'mzq'
'mzquantum' 'mzq'　　数值型 (缺省为 100)
全局
{仅当编译时加入 +mzscheme 特性才有效}
轮询 MzScheme 线程的毫秒计的时间间隔。负数或零意味着没有线程调度。
注意：如果复位 'compatible'，本选项被设为 Vim 缺省值。

'nrformats' 'nf'
'nrformats' 'nf'　　字符串 (缺省为 "bin,octal,hex"，defaults.vim 里设为 "bin,hex")
局部于缓冲区
本选项定义 Vim 使用 CTRL-A 和 CTRL-X 命令分别加减数值时使用的数值的基底；关于这些命令的更多的信息，见 CTRL-A 。
alpha　如果包含，单个字母会被递增和递减。这可以用于使用字母索引的列表 a)、b)，等等。 octal-nrformats
octal　如果包含，零开始的数值认为是八进制。例如：在 "007" 上用 CTRL-A 得到 "010"。
hex　如果包含，"0x" 或 "0X" 开始的数值认为是十六进制。例如："0x100" 上用 CTRL-X 得到 "0x0ff"。
bin　如果包含，"0b" 或 "0B" 开始的数值认为是二进制。例如："0b1000" 上用 CTRL-X 得到 "0b0111"。
简单的以 1-9 的数字开始的数值总被认为是十进制的。这也包括不能识别为八进制或十六进制的数值。

'number' 'nu' 'nonumber' 'nonu'
'number' 'nu'　　布尔型 (缺省关闭)
局部于窗口
在每行前面显示行号。如果 'cpoptions' 里没有 'n' 选项，回绕的行不使用行号所在的列 (如果没有置位 'compatible'，这是缺省)。
'numberwidth' 选项可以用来设置行号使用的空间。
如果长的回绕的行不从第一个字符开始，'-' 字符出现在行号前面。
hl-LineNr 和 hl-CursorLineNr 说明行号使用的高亮。
number_relativenumber
'relativenumber' 选项影响相对光标的的显示数字。和 'number' 合用，有以下四个不同组合 (光标在第三行)：

'nonu' 'nornu'	'nu' 'nornu'	'nonu' 'rnu'	'nu' 'rnu'
|apple	1 apple	2 apple	2 apple
|pear	2 pear	1 pear	1 pear
|nobody	3 nobody	0 nobody	|3 nobody
|there	4 there	1 there	1 there

'numberwidth' 'nuw'
'numberwidth' 'nuw'　　数值型 (Vim 缺省：4 Vi 缺省：8)
局部于窗口
{仅当编译时加入 +linebreak 特性才有效}
行号使用的最小列数。只有置位 'number' 或 'relativenumber' 选项或者带行号打印时才有意义。因为行号和文本之间总有一个空格，实际的字符数比该数值小一。

该值是最小的宽度。取决于使用 'number' 还是 'relativenumber'，如果为适应缓冲区最大的行号或窗口的行数而有必要的话，会使用更大的宽度。这样，Vim 缺省值 4 可以容纳最大行号为 999。若缓冲区有 1000 行，会使用五列。最小值为 1，最大值为 20。

注意：如果置位 'compatible'，本选项被设为 Vi 缺省值，而复位 'compatible' 时，设为 Vim 缺省值。

'omnifunc' 'ofu'

'omnifunc' 'ofu' 字符串 (缺省：空)
局部于缓冲区
{仅当编译时加入 +eval 特性才有效}

指定插入模式用 CTRL-X CTRL-O 进行全能 (omni) 补全使用的函数 i_CTRL-X_CTRL-O 。
complete-functions 解释如何调用该函数以及它应该返回何值。
本选项通常在文件类型插件里设置：:filetype-plugin-on
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'opendevice' 'odev' 'noopendevice' 'noodev'

'opendevice' 'odev' 布尔型 (缺省关闭)
全局
{仅限于 MS-Windows}

允许从设备中读写。这可能会使 Vim 在一个可以打开但不能实际完成 I/O 的设备上死机。因此缺省关闭此选项。

注意 在 MS-Windows 上编辑 "aux.h"、"lpt1.txt" 诸如此类也相当于对设备进行编辑。

'operatorfunc' 'opfunc'

'operatorfunc' 'opfunc' 字符串 (缺省：空)
全局

指定 g@ 操作符调用的函数。
详情和示例见 :map-operator 。

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'osfiletype' 'oft'

'osfiletype' 'oft' 字符串 (缺省："")
局部于缓冲区

此选项用来支持 RISC OS，后者已被移除。

'packpath' 'pp'

'packpath' 'pp' 字符串 (缺省：见 'runtimepath')

寻找软件包的目录。见 packages 。

'paragraphs' 'para'

'paragraphs' 'para' 字符串 (缺省为 "IPLPPPQPP TPHPLIPpLpItpplpipbp")
全局

指定用于分隔段落的 nroff 宏的列表，它们各是两个字符构成的组对 (见 object-motions)。

'paste' 'nopaste'

'paste' 布尔型 (缺省关闭)
全局

将 Vim 切换到粘贴模式。可用于从一个窗口剪切或复制文本并粘贴到 Vim。它的使用会避免一些意想不到的效果。
设置此选项可用于终端上运行的 Vim，因为那里 Vim 没法区别输入和粘贴的文本。在 GUI 里，Vim 知道何者来自粘贴，即使不打开 'paste' 也基本上不会做错。对 Vim 能自己处理鼠标点击的终端也是如此。
启动 GUI 时复位本选项。所以如果你在 .vimrc 里置位它，可以使它在终端里工作，但不是在 GUI 里。在 GUI 里置位 'paste' 有副作用：例如，Paste 工具栏按钮在插入模式下不能工作，因为它使用了映射。
打开 'paste' 选项时 (包括它本来就是打开的也会如此)：

- 屏蔽插入模式和命令行模式的映射
- 屏蔽缩写
- 'autoindent' 被复位
- 'expandtab' 被复位
- 'formatoptions' 的使用方式就像它为空一样
- 'revins' 被复位
- 'ruler' 被复位
- 'showmatch' 被复位
- 'smartindent' 被复位
- 'softtabstop' 设为 0
- 'textwidth' 设为 0
- 'wrapmargin' 设为 0

下面这些选项保持原来的值，但没有实际的效果：

- 'cindent'
- 'indentexpr'
- 'lisp'

注意：打开 'paste' 选项时如果你开始编辑别的文件，模式行或者自动命令可能会再次改变这些设置，从而在粘贴文本时造成麻烦。这时，你可能需要再次置位 'paste' 选项。
'paste' 选项复位时，上面提到的选项被复原为上次 'paste' 从关到开时的设置。置位 'paste' 之前复位 'paste' 没有任何效果。因为 'paste' 激活时不能用映射，'pastetoggle' 选项可以用来指定切换 'paste' 选项的热键。

'pastetoggle' 'pt'

'pastetoggle' 'pt' 字符串 (缺省为 "")
全局

如果非空，指定切换 'paste' 选项的键序列。类似于设定如下的映射：

```
:map {keys} :set invpaste<CR>
```

其中的 {keys} 是 'pastetoggle' 的值。
区别是，即使 'paste' 置位以后，它仍然工作。'pastetoggle' 可用于插入模式和普通模式，但不能用于命令行模式。
映射先检查，它可能会覆盖 'pastetoggle'。不过，打开 'paste' 时，插入模式里的映射被忽略，所以你可以这么做：

```
:map <F10> :set paste<CR>
:map <F11> :set nopaste<CR>
:imap <F10> <C-O>:set paste<CR>
:imap <F11> <nop>
:set pastetoggle=<F11>
```

这使得 <F10> 开始粘贴模式而 <F11> 终止粘贴模式。
注意 在粘贴模式里输入 <F10> 会插入 "<F10>"。因为在粘贴模式里，除了

'pastetoggle' 键序列以外，一切都是按本义插入的。
如果该值需要若干字节，适用 'ttimeoutlen'。

'pex' 'patchexpr'
'patchexpr' 'pex' 字符串 (缺省为 "")
全局
{仅当编译时加入 +diff 特性才有效}
用来给文件打补丁并生成文件新版本的表达式。见 diff-patchexpr 。

'patchmode' 'pm' E205 E206
'patchmode' 'pm' 字符串 (缺省为 "")
全局
如果非空，文件的旧版本被保存。在源代码发布中可以用来修改文件并保持文件的旧版本。只有第一次写入文件时，才保存原始文件的备份。备份的名字是原始文件的名字加上 'patchmode' 选项的值。本选项必须是以句号开始的字符串，形如 ".orig" 或 ".org"。需要 'backupdir' 不为空 (细节：新文件成功写入后，备份文件被换名为 patchmode 文件。这就是为什么必须可以写备份文件的原因)。如果没有文件需要备份 (原来文件不存在)，建立空文件。
如果匹配 'backupskip' 模式，不建立 patchmode 文件。
在压缩文件上使用 'patchmode' 会把后缀附加在压缩文件名的后面 (比如，"file.gz.orig")，产生的文件名不一定能被识别为压缩文件。
只能使用普通的文件名字符。"/*?[|<>" 都不合法。

'path' 'pa' E343 E345 E347 E854
'path' 'pa' 字符串 (缺省在 Unix 上：".,/usr/include,,"
其它系统：".,,")
全局或局部于缓冲区 global-local
这是一个目录列表， gf 、[f、]f、^Wf、 :find 、 :sfind 、 :tabfind 和其它命令的文件搜索如果使用相对路径 (不以 "/"、"./" 或 "../" 开始) 的话，会在这里进行。'path' 选项里的目录可以是相对也可以是绝对路径。
- 使用逗号分隔目录名：
```
:set path=.,/usr/local/include,/usr/include
```
- 空格可以用来分隔目录名 (为了和 3.0 版本后向兼容)。要在目录名里使用空格，在它前面加额外的反斜杠，然后转义空格：
```
:set path=.,/dir/with\\ space
```
- 要在目录名里加上空格，在它前面加上额外的反斜杠：
```
:set path=.,/dir/with\\,comma
```
- 要搜索相对于当前文件所在的目录，使用：
```
:set path=.
```
- 要在当前目录下搜索，在两个逗号之间使用空字符串：
```
:set path=,,
```
- 目录名可以用 ':' 或 '/' 结尾。
- 环境变量被扩展 :set_env 。
- 如果使用 netrw.vim ，可以使用 URL。比如，加入 "http://www.vim.org" 会使 ":find index.html" 能够工作。
- 使用 "*"、"**" 和 ";" 在目录树里往上和往下搜索。相关信息和语法见 file-searching 。
{仅当编译时加入 +path_extra 特性才有效}
- 小心使用 '\' 字符，在选项里输入两个得到一个：
```
:set path=.,c:\\include
```
也可使用 '/' 代替：
```
:set path=.,c:/include
```
不要忘记 "."，否则甚至不会在文件的同一目录里搜索文件！
最大的长度有限制。具体多少由系统决定，一般是 256 或 1024 个字符。

你可以检查是不是所有的头文件都能用 'path' 的值找到，见 :checkpath 。
建议使用 :set+= 和 :set-= 来从列表里加减目录。这可以避免未来版本使用其它缺省值出现的问题。要删除当前目录，可以用：

```
:set path-=
```

要增加当前目录，使用：

```
:set path+=
```

环境变量的使用可能意味着需要更换分隔符。这里是一个加上 $INCL 的例子，该变量里的路径名以分号分隔：

```
:let &path = &path . "," . substitute($INCL, ';', ',', 'g')
```

把 ';' 换成 ':' 或者任何你使用的分隔符。注意 如果 $INCL 里包含逗号或者空格，这不会工作。

'perldll'
'perldll' 字符串 (缺省取决于编译设置)
全局
{仅当编译时加入 +perl/dyn 特性才有效}
指定 Perl 共享库的名字。缺省是编译时指定的 DYNAMIC_PERL_DLL。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'preserveindent' 'pi' 'nopreserveindent' 'nopi'
'preserveindent' 'pi' 布尔型 (缺省关闭)
局部于缓冲区
改变当前行缩进时，尽量保持缩进结构。通常，缩进被连续的制表，必要时后面再跟若干空格，来替换 (除非打开 'expandtab' ，这时只使用空格)。打开此选项意味着缩进会保留尽可能多的已有字符，只有必要时才加入附加的制表或空格。
对保持的空白不应用 'expandtab'，制表还是制表。
注意：多次使用 ">>" 产生的缩进是制表和空格的混合。你可能不喜欢这样。
注意：如果置位 'compatible'，本选项被复位。
另见 'copyindent'。
使用 :retab 来清理空白。

'previewheight' 'pvh'
'previewheight' 'pvh' 数值型 (缺省为 12)
全局
{仅当编译时加入 +quickfix 特性才有效}
预览窗口的缺省高度。用于 :ptag 和相关的命令，也用于不带计数的 CTRL-W_} 。'previewpopup' 设置时不使用。

'previewpopup' 'pvp'
'previewpopup' 'pvp' 字符串 (缺省为空)
全局
{仅当编译时加入 +textprop 或 +quickfix 特性才有效}
如果非空，本会打开预览窗口的命令会使用弹出窗口。见 preview-popup 。
不用于插入模式补全的 info，为此在 'completeopt' 中加入 "popup"。

'previewwindow' 'nopreviewwindow'
'pvw' 'nopvw' E590
'previewwindow' 'pvw' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +quickfix 特性才有效}
标识预览窗口。只有一个窗口可以置位此选项。本选项通常不直接设置，而通过

:ptag 、 :pedit 等命令进行。

'printdevice' 'pdev'
'printdevice' 'pdev' 字符串 (缺省为空)
全局
{仅当编译时加入 +printer 特性才有效}
:hardcopy 使用的打印机名。
见 pdev-option 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'printencoding' 'penc'
'printencoding' 'penc' 字符串 (缺省为空，有些系统除外)
全局
{仅当编译时加入 +printer 和 +postscript 特性才有效}
设置打印时使用的字符编码。
见 penc-option 。

'printexpr' 'pexpr'
'printexpr' 'pexpr' 字符串 (缺省：见下)
全局
{仅当编译时加入 +printer 和 +postscript 特性才有效}
用于打印 ":hardcopy" 产生的 PostScript 文件的表达式。
见 pexpr-option 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'printfont' 'pfn'
'printfont' 'pfn' 字符串 (缺省为 "courier")
全局
{仅当编译时加入 +printer 特性才有效}
:hardcopy 命令使用的字体名。
见 pfn-option 。

'printheader' 'pheader'
'printheader' 'pheader' 字符串 (缺省为 "%<%f%h%m%=Page %N")
全局
{仅当编译时加入 +printer 特性才有效}
定义 :hardcopy 输出的页眉格式。
见 pheader-option 。

'printmbcharset' 'pmbcs'
'printmbcharset' 'pmbcs' 字符串 (缺省为 "")
全局
{仅当编译时加入 +printer 和 +postscript 特性才有效}
:hardcopy 产生的 CJK 输出使用的 CJK 字符集。
见 pmbcs-option 。

'printmbfont' 'pmbfn'
'printmbfont' 'pmbfn' 字符串 (缺省为 "")
全局
{仅当编译时加入 +printer 和 +postscript 特性才有效}

:hardcopy 产生的 CJK 输出使用的 CJK 字体名列表。
见 pmbfn-option 。

'printoptions' 'popt'
'printoptions' 'popt' 字符串 (缺省为 "")
全局
{仅当编译时加入 +printer 特性才有效}
控制 :hardcopy 的输出格式的项目列表。
见 popt-option 。

'prompt' 'noprompt'
'prompt' 布尔型 (缺省打开)
全局
如果打开，Ex 模式使用 ":" 提示。

'pumheight' 'ph'
'pumheight' 'ph' 数值型 (缺省为 0)
全局
决定用于插入模式补全的弹出菜单显示项目的最大数目。如果为零，有多少空间就用多少。
ins-completion-menu 。

'pumwidth' 'pw'
'pumwidth' 'pw' 数值型 (缺省为 15)
全局
决定用于插入模式补全的弹出菜单的最小宽度。 ins-completion-menu 。

'pythondll'
'pythondll' 字符串 (缺省取决于编译设置)
全局
{仅当编译时加入 +python/dyn 特性才有效}
指定 Python 2.x 共享库的名字。缺省是编译时指定的 DYNAMIC_PYTHON_DLL。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'pythonhome'
'pythonhome' 字符串 (缺省为 "")
全局
{仅当编译时加入 +python/dyn 特性才有效}
指定 Python 2.x 的主目录名。如果 'pythonhome' 和 PYTHONHOME 环境变量都未设定，使用编译时指定的 PYTHON_HOME 作为 Python 2.x 主目录。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'pythonthreedll'
'pythonthreedll' 字符串 (缺省取决于编译设置)
全局
{仅当编译时加入 +python3/dyn 特性才有效}
指定 Python 3 共享库的名字。缺省是编译时指定的 DYNAMIC_PYTHON3_DLL。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'pythonthreehome'
'pythonthreehome' 字符串 (缺省为 "")

全局
{仅当编译时加入 +python3/dyn 特性才有效}
指定 Python 3 的主目录名。如果 'pythonthreehome' 和 PYTHONHOME 环境变量都未设定，使用编译时指定的 PYTHON3_HOME 作为 Python 3 主目录。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'pyxversion' 'pyx'
'pyxversion' 'pyx' 数值型 (缺省取决于编译设置)
全局
{仅当编译时加入 +python 或 +python3 特性才有效}
指定 pyx* 函数和 python_x 命令使用的 python 版本。缺省值如下:

编译时带有	缺省
+python 和 +python3	0
仅有 +python	2
仅有 +python3	3

可用值为 0、2 和 3。
如果 'pyxversion' 为 0，在首次执行任何 python2/3 命令或函数时被设为 2 或 3。例如， :py 设为 2， :py3 设为 3。 sets to 3. 如果 Python 3 可用， :pyx 设为 3，否则如果 Python 2 可用，设为 2。
另见: has-pythonx

如果 Vim 编译时只带有 +python 或 +python3 ，设置 'pyxversion' 没有作用。pyx* 函数和命令总是和编译时的版本相同。

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'quoteescape' 'qe'
'quoteescape' 'qe' 字符串 (缺省为 "\")
局部于缓冲区
字符串里用以转义引号的字符。用于 a'、a" 和 a‘ a’ 这样的文本对象。
如果字符串里找到本选项里任何一个字符，跳过下一个字符。缺省值使得文本 "foo\"bar\\" 被认为是单个字符串。

'readonly' 'ro' 'noreadonly' 'noro'
'readonly' 'ro' 布尔型 (缺省关闭)
局部于缓冲区
如果打开，除非使用 '!'，不允许写入。它保护你不小心覆盖文件。如果 Vim 以只读模式启动 ("vim -R") 或者可执行文件的名字叫 "view"，缺省打开。
用 ":w!" 以后，复位当前缓冲区的 'readonly' 选项，除非 'cpoptions' 里包含 'Z' 标志位。
如果使用 ":view" 命令，新编辑的缓冲区也置位 'readonly' 选项。
关于如何禁止改动缓冲区，参见 'modifiable'。

'redrawtime' 'rdt'
'redrawtime' 'rdt' 数值型 (缺省为 2000)
全局
{仅当编译时加入 +reltime 特性才有效}
重画屏幕以毫秒计的时间。应用于 'hlsearch' 的模式搜索、 :match 高亮和语法高亮。
如果重画需时超过这么多的毫秒数，不再继续高亮匹配。用于避免 Vim 在使用非常复杂的模式时挂起。

对于语法高亮，为每个窗口分别计算时间。超过限制时，暂停语法高亮，直到按 CTRL-L 为止。
用于避免使用非常复杂的模式时 Vim 的挂起。

'regexpengine' 're'

'regexpengine' 're' 数值置 (缺省为 0)
全局
选择缺省正规表达式引擎。 two-engines
可用值为:

0	自动选择
1	旧引擎
2	NFA 引擎

注意 使用 NFA 引擎时如果模式包含不支持的部分，整个模式会不匹配。本选项只用于调试正规表达式引擎。
如果缺省的引擎变得太昂贵，例如如果 NFA 引擎使用了太多状态的话，使用自动选择允许 Vim 进行引擎切换。这可用于避免复杂模式和长文本的组合时 Vim 的挂起。

'relativenumber' 'rnu' 'norelativenumber' 'nornu'

'relativenumber' 'rnu' 布尔型 (缺省关闭)
局部于窗口
在每行前显示相对于光标所在的行的行号。相对行号帮助你给出某些垂直移动命令 (如 j k + -) 时需要的计数值 count ，省得你自己去计算。和其它若干命令组合使用时会更有用 (如 y d c < > gq gw =)。
如果 'cpoptions' 排除 'n' 选项，回绕行不使用行号所占的列 (如果没有置位 'compatible'，这是缺省情况)。
可用 'numberwidth' 选项设置行号所需的空间。
经回绕的长行如果不是从首个字符开始，行号之前会加上 '-' 字符。
行号使用的高亮见 hl-LineNr 和 hl-CursorLineNr 。

光标行之前的数字也取决于 'number' 的值，关于两个选项的可能组合，参见 number_relativenumber 。

'remap' 'noremap'

'remap' 布尔型 (缺省打开)
全局
允许递归映射。如果对单个项目你不想这样，使用 :noremap[!] 命令。
注意: 为了避免使用模式的移植性问题，永远把本选项设为缺省 "打开" 状态。只有在旧的 Vi 脚本里才需要把它关闭。

'renderoptions' 'rop'

'renderoptions' 'rop' 字符串 (缺省: 空白)
全局
{仅当编译时加入 GUI 和 MS-Windows 的 DIRECTX 才有效}
选择文本渲染器并设置其选项。选项取决于渲染器。

语法:

```
set rop=type:{renderer}(,{name}:{value})*
```

目前，只有一个可选的渲染器。

渲染器 行为
directx Vim 使用 DirectX (DirectWrite) 绘制文本。比缺省 GDI 能画出更好

看的字形。
要求 'encoding' 为 "utf-8"，并且只有 MS-Windows Vista 或更新的版本才能工作。

选项：

名字	含义	类型	值
gamma	gamma	浮点	1.0 - 2.2 (可能)
contrast	enhancedContrast	浮点	(未知)
level	clearTypeLevel	浮点	(未知)
geom	pixelGeometry	整数	0 - 2 (见下)
renmode	renderingMode	整数	0 - 6 (见下)
taamode	textAntialiasMode	整数	0 - 3 (见下)
scrlines	Scroll Lines	整数	(已废弃)

详情可见此 URL (除了 scrlines 外)：
https://msdn.microsoft.com/en-us/library/dd368190.aspx

关于 geom：设备像素结构。
0 - DWRITE_PIXEL_GEOMETRY_FLAT
1 - DWRITE_PIXEL_GEOMETRY_RGB
2 - DWRITE_PIXEL_GEOMETRY_BGR

详情可见此 URL：
https://msdn.microsoft.com/en-us/library/dd368114.aspx

关于 renmode：渲染字形的方法。
0 - DWRITE_RENDERING_MODE_DEFAULT
1 - DWRITE_RENDERING_MODE_ALIASED
2 - DWRITE_RENDERING_MODE_GDI_CLASSIC
3 - DWRITE_RENDERING_MODE_GDI_NATURAL
4 - DWRITE_RENDERING_MODE_NATURAL
5 - DWRITE_RENDERING_MODE_NATURAL_SYMMETRIC
6 - DWRITE_RENDERING_MODE_OUTLINE

详情可见此 URL：
https://msdn.microsoft.com/en-us/library/dd368118.aspx

关于 taamode：绘制文本的反锯齿方法。
0 - D2D1_TEXT_ANTIALIAS_MODE_DEFAULT
1 - D2D1_TEXT_ANTIALIAS_MODE_CLEARTYPE
2 - D2D1_TEXT_ANTIALIAS_MODE_GRAYSCALE
3 - D2D1_TEXT_ANTIALIAS_MODE_ALIASED

详情可见此 URL：
https://msdn.microsoft.com/en-us/library/dd368170.aspx

关于 scrlines：
过去用于优化滚动的行为，不过现在已废弃。即使指定了也会直接忽略。

示例：

```
set encoding=utf-8
set gfn=Ricty_Diminished:h12
set rop=type:directx
```

如果 'guifont' 选择了光栅字体 (Courier、Terminal 或 FixedSys，文件名有 ".fon" 扩展名)，使用后备的 GDI 绘制。

备注：已知有些字体的选项的组合会在字形绘制时出问题。

- 'renmode:5' 和 'renmode:6' 不适用于某些特制的字体 (只包含位图字形的 True-Type 字体)。
- 'taamode:3' 不适用于某些矢量字体。

备注：用此选项时，在 Windows 8.1 或之后版本可以显示色彩表情图标 (emoticon)。要显示色彩表情图标，要留心一些适用条件。

- 如果你的字体已经包含非彩色的表情图标，会用之。
- 如果你的字体不包含表情图标，系统会选用备用符号字体。Windows 10 上使用 "Segoe UI Emoji"。
- 如果备用字体没有固定宽度字形，表情图标的渲染可能会超出绘制单元的边界框。

目前不支持其它的渲染器类型。

'report'
'report'　　数值型　(缺省为 2)
全局
报告改变行数的阈值。如果多于 'report' 的行发生改变，多数 ":" 命令会给出消息。如果你总是想得到这个消息，设定 'report' 为零。
对于 ":substitute" 命令而言，使用替代的数目，而不是行数。

'restorescreen'　'rs'　'norestorescreen'　'nors'
'restorescreen' 'rs'　　布尔型　(缺省打开)
全局
{仅适用于 Windows 95/NT 控制台版本}
如果置位，屏幕内容在退出 Vim 后恢复。也适用于执行外部命令时。

对于非 Windows 的 Vim: 你可以在 .vimrc 里置位和复位 't_ti' 和 't_te' 选项。要禁止复原:

```
set t_ti= t_te=
```

要允许复原 (如果是 xterm):

```
set t_ti=^[7^[[r^[[?47h t_te=^[[?47l^[8
```

(其中 ^[是一个 <Esc>，输入 CTRL-V <Esc> 可以插入)

'revins'　'ri'　'norevins'　'nori'
'revins' 'ri'　　布尔型　(缺省关闭)
全局
{仅当编译时加入 +rightleft 特性才有效}
插入模式字符的输入反向工作。见 "反向输入" ins-reverse 。如果置位 'allowrevins'，本选项可以用插入模式下的 CTRL-_ 命令切换。
注意：如果置位 'compatible'，本选项被复位。
'paste' 置位时复位本选项，复位时恢复本选项。

'rightleft'　'rl'　'norightleft'　'norl'
'rightleft' 'rl'　　布尔型　(缺省关闭)
局部于窗口
{仅当编译时加入 +rightleft 特性才有效}

如果打开，显示的方向变为从右到左，也就是，存贮在文件里的字符看起来从右到左出现。
使用本选项可以编辑从右到左书写的语言，比如希伯来语和阿拉伯语。
本选项局部于窗口，所以可以同时编辑不同朝向的文件，或者用不同的方向阅读同一文件（可用于有混合从右到左和从左到右字符串文本的文件，以便两个方向在不同的窗口里都能正确显示）。另见 rileft.txt 。

'rightleftcmd' 'rlc'
'rightleftcmd' 'rlc' 字符串 (缺省为 "search")
局部于窗口
{仅当编译时加入 +rightleft 特性才有效}
此选项里的每个单词允许命令行编辑的一组命令在从右到左模式下工作：

search "/" 和 "?" 命令

可用于希伯来语、阿拉伯语和波斯语等语言。
要使 'rightleftcmd' 生效，必须置位 'rightleft' 选项。

'rubydll'
'rubydll' 字符串 (缺省取决于编译设置)
全局
{仅当编译时加入 +ruby/dyn 特性才有效}
指定 Ruby 共享库的名字。缺省是编译时指定的 DYNAMIC_RUBY_DLL。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'ruler' 'ru' 'noruler' 'noru'
'ruler' 'ru' 布尔型 (缺省关闭，defaults.vim 里置位)
全局
{仅当编译时加入 +cmdline_info 特性才有效}
标尺。显示光标位置的行号和列号，逗号分隔。如果还有空间，在最右端显示文本在文件中的相对位置：

Top	首行可见
Bot	末行可见
All	首末两行都可见
45%	文件中的相对位置

如果设置 'rulerformat'，它决定标尺的内容。
每个窗口都有自己的标尺。如果窗口有状态行，标尺在那里显示。否则，它显示在屏幕的最后一行上。如果通过 'statusline' 给出状态行（亦即，非空），该选项优先于 'ruler' 和 'rulerformat'。
如果显示的字符数不同于文本的字节数（比如，TAB 或者多字节字符），同时显示文本列号（字节数）和屏幕列号，以连字符分隔。
空行显示 "0-1"。
空缓冲区的行号也为零："0,0-1"。
'paste' 置位时复位本选项。复位时恢复本选项。
如果你不想一直看到标尺但想知道现在在哪里，使用 "g CTRL-G" g_CTRL-G 。
注意：如果置位 'compatible'，本选项被复位。

'rulerformat' 'ruf'
'rulerformat' 'ruf' 字符串 (缺省为空)
全局
{仅当编译时加入 +statusline 特性才有效}
如果本选项非空，它决定 'ruler' 选项要显示的标尺字符串的内容。
本选项的格式和 'statusline' 类同。

'modelineexpr' 关闭时，不能在模式行中设置此选项。

缺省的标尺宽度是 17 个字符。要使标尺 15 个字符宽，在开始加上 "%15("，在最后加入 "%)"。
例如：

```
:set rulerformat=%15(%c%V\ %p%%)
```

'runtimepath' 'rtp' vimfiles

'runtimepath' 'rtp' 字符串 (缺省：

```
Unix: "$HOME/.vim,
        $VIM/vimfiles,
        $VIMRUNTIME,
        $VIM/vimfiles/after,
        $HOME/.vim/after"
Amiga: "home:vimfiles,
        $VIM/vimfiles,
        $VIMRUNTIME,
        $VIM/vimfiles/after,
        home:vimfiles/after"
PC:    "$HOME/vimfiles,
        $VIM/vimfiles,
        $VIMRUNTIME,
        $VIM/vimfiles/after,
        $HOME/vimfiles/after"
Macintosh: "$VIM:vimfiles,
        $VIMRUNTIME,
        $VIM:vimfiles:after"
RISC-OS: "Choices:vimfiles,
        $VIMRUNTIME,
        Choices:vimfiles/after"
VMS: "sys$login:vimfiles,
        $VIM/vimfiles,
        $VIMRUNTIME,
        $VIM/vimfiles/after,
        sys$login:vimfiles/after")
```

全局
搜索运行时文件所用的目录：

filetype.vim	根据文件名决定文件类型 new-filetype
scripts.vim	根据文件内容决定文件类型 new-filetype-scripts
autoload/	自动载入的脚本 autoload-functions
colors/	色彩方案文件 :colorscheme
compiler/	编辑器文件 :compiler
doc/	文档 write-local-help
ftplugin/	文件类型插件 write-filetype-plugin
indent/	缩进脚本 indent-expression
keymap/	键盘映射表文件 mbyte-keymap
lang/	菜单翻译 :menutrans
menu.vim	GUI 菜单 menu.vim
pack/	软件包 :packadd
plugin/	插件脚本 write-plugin
print/	打印所需的文件 postscript-print-encoding
spell/	拼写检查文件 spell
syntax/	语法文件 mysyntaxfile
tutor/	vimtutor 所需的文件 tutor

还有其它用 :runtime 命令搜索的文件。

多数系统上，设置的缺省值搜索五个位置：
1. 你的主目录，里面有你的个人偏好。
2. 系统范围的 Vim 目录，系统管理员的设置。
3. $VIMRUNTIME 里，Vim 发布的文件。

after-directory

4. 系统范围 Vim 目录的 "after" 目录。为了系统管理员能够修改发布的缺省值或增加设置（很少用到）。
5. 你的主目录下的 "after" 目录。为了你能够修改发布或者系统范围的设置或增加个人偏好（很少用到）。

用 packages 会增加更多项目。如果太长了，‘:set rtp‘ 会被截短，‘:echo &rtp‘ 可以看到完整的字符串。

注意，和 'path' 不同，这里不允许 "**" 等通配符。普通的通配符是可以的，但这样做显著地减慢运行时文件的搜索速度。为效率着想，尽可能少使用项目并避免通配符。
见 :runtime 。
例如：

```
:set runtimepath=~/vimruntime,/mygroup/vim,$VIMRUNTIME
```

先使用目录 "~/vimruntime"（包含你的个人 Vim 运行时文件），然后是 "/mygroup/vim"（在组间共享），最后是 "$VIMRUNTIME"（发布的运行时文件）。为使用发布的运行时文件，你可能总是要在某处包含 $VIMRUNTIME。你可以在 $VIMRUNTIME 之前放置目录，以定位替换发布的运行时文件的文件。你也可以在 $VIMRUNTIME 之后放置目录，以定位改进发布的运行时文件的文件。
如果 Vim 启动时带了 --clean ，跳过主目录下的项目。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'scroll' 'scr'

'scroll' 'scr' 数值型 （缺省：窗口高度的一半）
局部于窗口

使用 CTRL-U 和 CTRL-D 滚动的行数。如果改变窗口大小，本选项设为窗口新行数的一半。如果 CTRL-U 或 CTRL-D 命令给出计数，该计数会成为 'scroll' 的新值。":set scroll=0" 可以复位到缺省情况：使用窗口高度的一半。

'scrollbind' 'scb' 'noscrollbind' 'noscb'

'scrollbind' 'scb' 布尔型 （缺省关闭）
局部于窗口

另见 scroll-binding 。如果置位此选项，当前窗口随着其它的 scrollbind 窗口（同样置位此选项的窗口）一起滚动。此选项可用于查看文件两个版本的差异，见 'diff'。
'scrollopt' 解释决定本选项如何解释的相关选项。
分割窗口以编辑其它文件时，多复位此选项。这意味着 ":split | edit file" 产生两个置位 scrollbind 的窗口，但 ":split file" 不会。

'scrollfocus' 'scf' 'noscrollfocus' 'noscf'

'scrollfocus' 'scf' 布尔型 （缺省关闭）
全局
{仅用于 MS-Windows GUI}

使用滚轮并且置位此选项时，滚动鼠标所在的窗口。如果关闭此选项则滚动当前窗口。
MS-Windows 之外的系统的行为总像此选项打开那样。

'scrolljump' 'sj'

'scrolljump' 'sj'　　数值型 (缺省为 1)
全局

光标离开屏幕时 (比如用 "j"), 最少的滚动行数。不用于滚动命令 (比如 CTRL-E、CTRL-D)。如果屏幕滚动很慢, 会有帮助。
如果设为 -1 到 -100 的负数, 用作窗口高度的百分比。这样 -50 滚动到窗口高度的一半。
注意: 如果置位 'compatible', 本选项被设为 1。

'scrolloff' 'so'

'scrolloff' 'so'　　数值型 (缺省为 0, defaults.vim 里设为 5)
全局或局部于窗口 global-local

光标上下两侧最少保留的屏幕行数。这使你工作时总有一些可见的上下文。
如果你设置此选项为很大的值 (比如 999), 光标所在的行将总定位在窗口的中间位置 (除非你非常靠近文件开始或者结束的地方, 或者有长行回绕)。
使用局部值后, 可用以下两者之一来回到全局值:

```
setlocal scrolloff<
setlocal scrolloff=-1
```

水平滚动见 'sidescrolloff'。
注意: 如果置位 'compatible', 本选项被设为 0。

'scrollopt' 'sbo'

'scrollopt' 'sbo'　　字符串 (缺省为 "ver,jump")
全局

这是逗号分隔的单词列表, 指定 'scrollbind' 窗口的行为。'sbo' 代表 ScrollBind Options (滚动绑定选项)。
可用以下的单词:

ver　　'scrollbind' 窗口绑定垂直滚动

hor　　'scrollbind' 窗口绑定水平滚动

jump　　应用两个垂直滚动窗口之间的相对偏移。该偏移是两个绑定窗口的首个显示行之间的行号差。如果在一个窗口里移动, 另一个 'scrollbind' 的窗口可能会到达缓冲区开始之前或结束之后的位置。不过, 该偏移没有改变。如果再往相反方向滚动, 那个 'scrollbind' 窗口会试图滚动到根据该偏移它应该到达的位置, 如有可能。
如果现在使得该窗口成为当前窗口, 相对偏移会有两种情况:
1. 如果不包含 "jump", 相对偏移根据新窗口的滚动位置进行调整。回到原来那个窗口时, 会使用新的相对偏移。
2. 如果包含 "jump", 别的窗口会根据相同的相对偏移进行滚动, 回到原来那个窗口时, 仍然使用原有的相对偏移。

另见 scroll-binding 。
激活 'diff' 模式时, 总有垂直滚动绑定。即使这里没有 "ver" 亦然。

'sections' 'sect'

'sections' 'sect'　　字符串 (缺省为 "SHNHH HUnhsh")
全局

指定用于分隔小节的 nroff 宏的列表, 它们各是两个字符构成的组对 (见 object-motions)。缺省值使得小节用以下的 nroff 宏开始: ".SH"、".NH"、".H"、".HU"、".nh" 和 ".sh"。

'secure' 'nosecure' E523

'secure'　　布尔型 (缺省关闭)
全局

如果打开，当前目录下的 ".vimrc" 和 ".exrc" 不允许使用 ":autocmd"、外壳和写入命令，映射命令被显示。只有你确信不会有问题，或者 'exrc' 选项被关闭，才可以关闭此选项。Unix 上，只有在 ".vimrc" 或 ".exrc" 不为你所拥有时才用到此选项。如果系统允许用户使用 "chown"，这仍然可能有危险。最好在你的 ~/.vimrc 的最后给 'secure' 置位。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'selection' 'sel'

'selection' 'sel' 字符串 (缺省为 "inclusive")
全局
本选项定义选择区的行为。只有可视和选择模式使用本选项。
可能的值:

值	越行	闭区间
old	否	是
inclusive	是	是
exclusive	是	否

"越行" 意味着光标是否允许定位在行后一个字符的地方。
"闭区间" 意味着选择区的最后一个字符包含在操作范围之内。比如，用 "x" 删除选择区时。
如果使用了 "old" 而 'virtualedit' 允许光标移动过行尾，则还是不包括换行符。
注意 如果从普通模式开始、'virtualedit' 为空、使用 "exclusive" (开区间) 而从行尾开始反向选择，你无法包含行尾的字符。

:behave 命令设置 'selection' 选项。

'selectmode' 'slm'

'selectmode' 'slm' 字符串 (缺省为 "")
全局
这是逗号分隔的单词列表，指定什么场合开始选择时启动选择模式而不是可视模式。
可能的值:

mouse	使用鼠标时
key	使用 Shift + 特殊键时
cmd	使用 "v"、"V" 或 CTRL-V 时

见 Select-mode 。
:behave 命令设置 'selectmode' 选项。

'sessionoptions' 'ssop'

'sessionoptions' 'ssop' 字符串 (缺省: "blank,buffers,curdir,folds,
help,options,tabpages,winsize,terminal")
全局
{仅当编译时加入 +mksession 特性才有效}
改变 :mksession 命令的效果。这是逗号分隔的单词列表。每个单词允许保存和恢复某种设置:

单词	保存和恢复
blank	空窗口
buffers	隐藏和卸载的缓冲区，不光是那些在窗口里的
curdir	当前目录
folds	手动建立的折叠、打开/关闭的折叠和局部折叠选项
globals	大写字母开头且包含至少一个小写字母的全局变量。只保存字符串和数值类型。
help	帮助窗口
localoptions	局部于窗口或缓冲区的选项和映射 (不是局部选项的全局值)

options	所有的选项和映射（也包括局部选项的全局值）
resize	Vim 窗口的大小：'lines' 和 'columns'
sesdir	会话文件所在的目录会成为当前目录（用于通过网络访问别的系统的项目）
slash	文件名里的反斜杠被替换成正斜杠
tabpages	所有标签页；如果不包含，只恢复当前标签页。这样你可以为每个标签页分别保存一个会话
terminal	包含终端窗口，可以恢复其中的命令
unix	使用 Unix 换行格式（单个 <NL>），即使在 Windows 或 DOS 上也是如此
winpos	整个 Vim 窗口的位置
winsize	窗口大小

不要同时包含 "curdir" 和 "sesdir"。
如果既没有包含 "curdir" 也没有包含 "sesdir"，使用绝对路径保存文件名。
"slash" 和 "unix" 可用于 Windows 系统中，如果要和 Unix 共享会话文件的话。Unix 版本的 Vim 不能执行 dos 格式的脚本，但是 Windows 版本的 Vim 可以执行 unix 版本的脚本。

'shell' 'sh' 字符串 （缺省为 $SHELL 或 "sh"，Win32: "cmd.exe"）
全局
用于 ! 和 :! 命令的外壳名。如果改变此值，同时检查以下选项：
'shelltype'、'shellpipe'、'shellslash'、'shellredir'、'shellquote'、'shellxquote' 和 'shellcmdflag'。
可以给出命令所需的参数，比如 "csh -f"。
option-backslash 说明如何包含空格和反斜杠。
环境变量被扩展 :set_env 。

如果外壳的名字包含空格，你需要把它放在引号里或把空格转义。带引号的例子：

```
:set shell=\"c:\program\ files\unix\sh.exe\"\ -f
```

注意 每个引号（避免作为注释的开始）和每个空格（避免作为选项值的终止）之前的反斜杠。还要 注意 "-f" 不在引号里，因为它不是命令名本身的一部分。Vim 自动地（神奇地）识别作为路径分隔符的反斜杠。
带转义空格的例子（Vim 从 $SHELL 初始化本选项时会这样做）：

```
:set shell=/bin/with\\\ space/sh
```

'shell' 的最终值是 "/bin/with\ space/sh"， :set 会用掉两个反斜杠。

MS-Windows 上，如果执行文件以 ".com" 结尾，必须把它包含在内。这样，设置外壳为 "command.com" 或 "4dos.com" 没问题，但 "command" 和 "4dos" 就不能适用于所有命令（比如，过滤命令）。
因为未知的原因，使用 "4dos.com" 时，当前目录被改为 "C:\"。要避免这个问题，设置 'shell' 为：

```
:set shell=command.com\ /c\ 4dos
```

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'shellcmdflag' 'shcf' 字符串 （缺省："-c"；
MS-DOS 和 Win32，如果 'shell' 不包含 "sh" 的话："/c"）
全局
执行 "!" 和 ":!" 命令时传递给外壳的参数；比如，"bash.exe -c ls" 或

"command.com /c dir"。对于 MS-DOS 系列的系统，缺省的设置根据 'shell' 的值而定，以减少用户自行设置此选项的需要。
Unix 上可有多个值。空格分隔的每个部分被分别作为一个参数传递给外壳。
option-backslash 说明如何包含空格和反斜杠。
MS-DOS 和 MS-Windows 上另见 dos-shell 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'shellpipe' 'sp'

'shellpipe' 'sp' 字符串 (缺省为 ">"、"| tee"、"|& tee" 或 "2>&1| tee")
全局
{仅当编译时加入 +quickfix 特性才有效}
用于把 ":make" 命令输出存到错误文件的字符串。另见 :make_makeprg 。
option-backslash 说明如何包含空格和反斜杠。
如有需要，临时文件名可以用 "%s" 表示 (如果选项值里没有 %s，该文件名自动添加在最后)。
Amiga 和 MS-DOS 上，缺省为 ">"。输出直接存到文件里，不在屏幕上回显。
Unix 上，缺省是 "| tee"。编译器的标准输出存到文件里，并在屏幕上回显。
如果初始化后，'shell' 选项是 "csh" 或 "tcsh"，缺省值变为 "|& tee"。如果 'shell' 选项为 "sh"、"ksh"、"mksh"、"pdksh"、"zsh"、"zsh-beta"、"bash" 或 "fish"，缺省值为 "2>&1| tee"。这意味着标准错误 (stderr) 也被包含在内。使用 "shell" 选项时排除路径部分，也就是 "/bin/sh" 被当成 "sh"。
此选项的初始化在读入 ".vimrc" 和其它初始化步骤之后完成，所以如果那里设置 'shell' 选项，'shellpipe' 选项被自动更改，除非之前已经被显式地设置过。
如果 'shellpipe' 设为空字符串，不进行 ":make" 输出的重定向。这可用于自己会写入 'makeef' 的 'makeprg' 程序。如果你不用管道，但要加入 'makeef' 作为参数，设置 'shellpipe' 为单个空格。不要忘了在空格之前加上反斜杠: ":set sp=\ "。
将来，管道可能用于过滤程序，而此选项可能会被废弃 (至少对 Unix 如此)。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'shellquote' 'shq'

'shellquote' 'shq' 字符串 (缺省: ""; MS-DOS 和 Win32，如果 'shell' 包含 "sh" 的话: "\"")
全局
(一或多个) 引用字符，用来包围 "!" 和 ":!" 命令里传递给外壳的命令。重定向在引用之外。要把重定向包含在内，见 'shellxquote'。可能没有必要同时设置这两个选项。
如果其值为 '(，则在之后附加 ')'。如果值为 '"(，则在之后附加)"'。
如果其值为 '(，另见 'shellxescape'。
多数系统上，缺省是一个空字符串。已知只有对 MS-DOS 系列的系统上有用，或者用 cmd.exe，自动去掉命令的最先和最后的引号，或者用第三方外壳，比如 MKS Korn Shell 或 bash，那里本选项应是 "\""。缺省值会根据 'shell' 的值调整，以减少用户自行设置本选项的需要。见 dos-shell 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'shellredir' 'srr'

'shellredir' 'srr' 字符串 (缺省为 ">"，">&" 或 ">%s 2>&1")
全局
用于把过滤命令输出存到临时文件的字符串。另见 :! 。
option-backslash 说明如何包含空格和反斜杠。
如有需要，临时文件名可以用 "%s" 表示 (如果选项值里没有 %s，该文件名自动添加在最后)。

缺省为 ">"。Unix 上，如果初始化后，'shell' 选项是 "csh" 或 "tcsh"，缺省值为 ">&"。如果 'shell' 选项为 "sh"、"ksh"、"mksh"、"pdksh"、"zsh"、"zsh-beta"、"bash" 或 "fish"，缺省值则为 ">%s 2>&1"。这意味着标准错误 (stderr) 也被包含在内。
Win32 上，进行 Unix 同样的检查，另外还检查 "cmd"，缺省是 ">%s 2>&1"。此外，检查相同但有 ".exe" 后缀的名字。
此选项的初始化在读入 ".vimrc" 和其它初始化步骤之后完成，所以如果那里设置 'shell' 选项，'shellredir' 选项被自动更改，除非之前已经被显式地设置过。
将来，管道可能用于过滤程序，而此选项可能会被废弃 (至少对 Unix 如此)。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'shellslash' 'ssl' 'noshellslash' 'nossl'
'shellslash' 'ssl' 布尔型 (缺省关闭)
全局
{仅适用于 MS-Windows}
如果置位，扩展文件名时使用正斜杠。如果使用 Unix 风格的外壳代替 command.com 或 cmd.exe，这会有用。你仍然可以输入反斜杠，但 Vim 会把它们换成正斜杠。
注意 置位或复位本选项对已有的文件名没有影响，所以为了最佳的效果，本选项需要在打开任何文件之前使用。将来此行为或许会有改变。
'shellslash' 只用于使用反斜杠作为路径分隔符的系统。要看是否如此：
```
if exists('+shellslash')
```
另见 'completeslash'。

'shelltemp' 'stmp' 'noshelltemp' 'nostmp'
'shelltemp' 'stmp' 布尔型 (Vi 缺省关闭，Vim 缺省打开)
全局
如果打开，外壳命令使用临时文件。如果关闭，使用管道。
如果不能使用管道，无论如何总使用临时文件。
目前，只有 Unix 和 MS-Windows 2K 及更新版本支持管道。你可以这样检查：
```
:if has("filterpipe")
```
使用管道的好处是没人能够读到临时文件，而 'shell' 命令不须支持重定向。
临时文件的好处可以检测文件类型和编码。
FilterReadPre 、 FilterReadPost 、 FilterWritePre 、 FilterWritePost 自动命令事件在 'shelltemp' 关闭时不会激活。
system() 函数不理会此选项，总是使用临时文件。
注意：如果复位 'compatible'，本选项被设为 Vim 缺省值。

'shelltype' 'st'
'shelltype' 'st' 数值型 (缺省为 0)
全局
{仅适用于 Amiga}
Amiga 上，此选项影响使用外壳的命令的工作方式。
0 和 1：总是用外壳
2 和 3：只用外壳过滤行
4 和 5：只用外壳执行 ':sh' 命令
如果不使用外壳，命令被直接执行。

0 和 2：使用 "shell 'shellcmdflag' cmd" 启动外部命令
1 和 3：使用 "shell cmd" 启动外部命令
(译者注：4 和 5 应该也 分别 包含在上面两种情况里)

'shellxescape' 'sxe'

'shellxescape' 'sxe' 字符串 (缺省: "";
MS-DOS 和 MS-Windows: "\"&|<>()@^")
全局

'shellxquote' 如果设为 "(", 此选项列出的字符会用 '^' 字符转义。这梓可以用 cmd.exe 执行大部分外部命令。

'shellxquote' 'sxq'

'shellxquote' 'sxq' 字符串 (缺省: "";
Win32 如果 'shell' 是 cmd.exe: "("
Win32 如果 'shell' 包含 "sh": "\""
Unix 如果使用 system() 的话: "\"")
全局

(一或多个) 引用字符, 用来包围 "!" 和 ":"!" 命令里传递给外壳的命令。
重定向包含在引用之内。要把重定向排除在外, 见 'shellquote'。可能没有必要同时设置两个选项。
缺省是一个空字符串。已知只有对 MS-DOS 系列的系统上的第三方外壳有用, 比如 MKS Korn Shell 或 bash, 那里本选项应是 "\""。缺省值会根据 'shell' 的值调整, 以减少用户自行设置本选项的需要。见 dos-shell 。
为了安全原因, 本选项不能在 modeline 或 sandbox 里设置。

'shiftround' 'sr' 'noshiftround' 'nosr'

'shiftround' 'sr' 布尔型 (缺省关闭)
全局

缩进取整到 'shiftwidth' 的倍数。应用于 > 和 < 命令。插入模式里的 CTRL-T 和 CTRL-D 总是把缩进取整到 'shiftwidth' 的倍数 (和 Vi 兼容)。
注意: 如果置位 'compatible', 本选项被复位。

'shiftwidth' 'sw'

'shiftwidth' 'sw' 数值型 (缺省为 8)
局部于缓冲区

(自动) 缩进每一步使用的空白数目。用于 'cindent' 、 >> 、 << 等。如果为零, 使用 'ts' 的值。 shiftwidth() 函数可用来得到有效的 shiftwidth 值。

'shortmess' 'shm'

'shortmess' 'shm' 字符串 (Vim 缺省 "filnxtToOS", Vi 缺省: "S",
POSIX 缺省: "AS")
全局

本选项有助于避免文件信息的所有 hit-enter 提示, 比如用 CTRL-G 的时候。它还用于避免或减少一些其它消息。以下是标志位的列表:

标志位	存在时的意义
f	用 "(3 of 5)" 代替 "(file 3 of 5)"
i	用 "[noeol]" 代替 "[Incomplete last line]"
l	用 "999L, 888C" 代替 "999 lines, 888 characters"
m	用 "[+]" 代替 "[Modified]"
n	用 "[New]" 代替 "[New File]"
r	用 "[RO]" 代替 "[readonly]"
w	用 "[w]" 代替文件写入消息的 "written", 还有用 "[a]" 代替 ':w >> file' 命令的 "appended"
x	用 "[dos]" 代替 "[dos format]"、 用 "[unix]" 代替 "[unix format]"、 还有用 "[mac]" 代替 "[mac format]"。
a	所有以上的缩写。

o 后续文件的读入信息覆盖文件的写回消息 (":wn" 或打开 'autowrite' 时有用)。
O 文件的读入信息覆盖任何前面消息，包括 quickfix 消息 (比如，":cn")。
s 不给出 "search hit BOTTOM, continuing at TOP" 或 "search hit TOP, continuing at BOTTOM" 消息；使用搜索计数时不在计数信息后显示 "W" (见下 S)
t 如果文件消息不能完整放在命令行上，从开头截短该消息。最左列给出 "<"。Ex 模式忽略此标志位。
T 如果其它的消息不能完整放在命令行上，从中间截短该消息。中间给出 "<"。Ex 模式忽略此标志位。
W 写入文件时，不给出 "written" 或 "[w]"。
A 如果发现交换文件已存在，不给出 "ATTENTION" 消息。
I 启动 Vim 时不给出介绍消息 :intro 。
c 不给出 ins-completion-menu 信息。例如，"-- XXX completion (YYY)"、"match 1 of 2"、"The only match"、"Pattern not found"、"Back at original" 等等。
q 用 "recording" 代替 "recording @a"
F 编辑文件时不给出文件信息，就像给命令用了 :silent 一样；注意 这也会影响自动命令的信息
S 搜索时不显示搜索计数，如 "[1/5]"

这给你一个机会，避免在缓冲区之间切换要按 <Enter>，但仍然能够在可用的空间里得到有用的消息。要得到 'shm' 为空时的完整消息，使用 ":file!"。
有用的值:

shm=	没有消息缩写。
shm=a	有缩写，但不丢失消息。
shm=at	有缩写，必要时截短消息。

注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'shortname' 'sn' 'noshortname' 'nosn'
'shortname' 'sn' 布尔型 (缺省关闭)
局部于缓冲区
假定文件名为 8 字符加 3 字符的扩展。文件名不能有多个句号。如果打开此选项，在附加扩展名时，文件名里的句号被下划线替换 (".~" 或 ".swp")。此选项在 MS-DOS 上不可用，因为那里它总应该是打开的。此选项用于编辑 MS-DOS 兼容的文件系统的文件，比如，messydos 或 crossdos。运行 Win32s 上的 Win32 GUI 版本时，缺省总是打开此选项。

'showbreak' 'sbr' E595
'showbreak' 'sbr' 字符串 (缺省为 "")
全局或局部于窗口 global-local
{仅当编译时加入 +linebreak 特性才有效}
回绕行放置在开头的字符串。有用的值如 "> " 或 "+++ ":

```
:set showbreak=>\
```

注意 要用反斜杠来转义拖尾的空格。下面的就容易些:

```
:let &showbreak = '+++ '
```

只允许可显示且只占用一个单元位置的字符，不包括 <Tab> 和逗号 (将来的版本里，逗号可能用来分隔行首和行尾显示的不同部分)。
这些字符的高亮由 'highlight' 的 '@' 标志位决定。

注意 showbreak 之后的制表使用不同的显示方式。
如果要 'showbreak' 出现在行号之间，在 'cpoptions' 里加上 "n" 标志位。

窗口局部值会否决全局值。如果设置了全局值而当前窗口想要空值，用 NONE:

```
:setlocal showbreak=NONE
```

'showcmd' 'sc' 'noshowcmd' 'nosc'
'showcmd' 'sc' 布尔型 (Vim 缺省: 打开，Unix 关闭，Vi 缺省: 关闭，defaults.vim 里置位)
全局
{仅当编译时加入 +cmdline_info 特性才有效}
在屏幕最后一行显示 (部分的) 命令。如果你的终端很慢，关闭此选项。
可视模式里，显示选择区域的大小，即:
- 在行内选择若干字符时，字符数。如果字节数不同，同时显示字节数: "2-6" 代表 2 个字符和 6 个字节。
- 选择多于一行时，行数。
- 选择可视列块时，屏幕字符的数目: {lines} 乘以 {columns}。

注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'showfulltag' 'sft' 'noshowfulltag' 'nosft'
'showfulltag' 'sft' 布尔型 (缺省关闭)
全局
插入模式里，用标签文件补全单词时 (见 ins-completion)，同时显示标签名和查找模式 (如果有的话) 的整齐格式作为可能的匹配。这样，如果匹配 C 函数，你会看到一个样板，从中了解到需要什么样的参数 (允许指定编码风格)。
注意 如果 'completeopt' 有 "longest"，这不会很好地工作，因为搜索模式的补全未必会匹配输入的文本。

'showmatch' 'sm' 'noshowmatch' 'nosm'
'showmatch' 'sm' 布尔型 (缺省关闭)
全局
插入括号时，短暂地跳转到匹配的对应括号。只有在屏幕上能看到匹配时才会进行跳转。显示匹配的时间用 'matchtime' 设置。
如果没有匹配会响铃 (和匹配能否看到无关)。
'paste' 置位时复位本选项，复位时恢复本选项。
如果 'cpoptions' 里没有 'm' 标志位，接着输入字符会立即把光标移动到它应该在的位置。
'guicursor' 的 "sm" 域说明显示匹配时，如何设置光标形状和闪烁。
'matchpairs' 选项可指定显示匹配所用的字符。用 'rightleft' 和 'revins' 查找反方向的匹配。
移动时要高亮匹配，另见 matchparen 插件 pi_paren.txt 。
备注: 要使用短形式，建议加强家长指导 (译者注: :-))。

'showmode' 'smd' 'noshowmode' 'nosmd'
'showmode' 'smd' 布尔型 (Vim 缺省: 打开，Vi 缺省: 关闭)
全局
在插入、替换和可视模式里，在最后一行提供消息。
'highlight' 的 'M' 标志位设置此消息的高亮类型。
如果可能使用 XIM ，消息里会包含 "XIM"。但这不意味着实际激活了 XIM，尤其是可能还没有设置 'imactivatekey'。
注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位

'compatible'，本选项被设为 Vim 的缺省值。

'showtabline' 'stal'

'showtabline' 'stal'　　数值型　(缺省为 1)
全局

本选项的值指定何时显示带有标签页标签的行：
0：永远不会
1：至少有两个标签页时才会
2：永远会

标签页行有 GUI 和非 GUI 两种实现。
tab-page　解释标签页的更多信息。

'sidescroll' 'ss'

'sidescroll' 'ss'　　数值型　(缺省为 0)
全局

水平滚动时滚动的最少列数。只用于 'wrap' 选项关闭且光标移出屏幕范围时。如果为零，把光标定位在屏幕的中间。在慢速的终端里，设大一些或者为 0。如果使用快速的终端，设小一些或者为 1。不用于 "zh" 和 "zl" 命令。

'sidescrolloff' 'siso'

'sidescrolloff' 'siso'　数值型 (缺省为 0)
全局或局部于窗口　global-local

如果设置 'nowrap'，光标左右两侧保留的最少屏幕列数。设置此选项为大于零的值，同时使 'sidescroll' 也为非零值，可以使得水平滚动的行上总有一些可见的上下文 (除了在行首之外)。设置此选项为很大的值 (比如 999) 可以保持光标总定位在窗口水平中央，除非你非常靠近行首。
使用局部值后，可用以下两者之一来回到全局值：

```
setlocal sidescrolloff<
setlocal sidescrolloff=-1
```

注意：如果置位 'compatible'，本选项被设为 0。

示例：　试试和 'sidescroll' 以及 'listchars' 一起使用，参考下面的例子，不让光标移到 "extends" 字符上：

```
:set nowrap sidescroll=1 listchars=extends:>,precedes:<
:set sidescrolloff=1
```

'signcolumn' 'scl'

'signcolumn' 'scl'　　字符串　(缺省 "auto")
局部于窗口
{仅当编译时加入　+signs　特性才有效}

是否绘制标号列。合法值是：

"auto"	仅当有标号要显示时
"no"	从不
"yes"	永远
"number"	在 'number' 列上标号。如果没有 number 列，相当于 "auto"。

'smartcase' 'scs' 'nosmartcase' 'noscs'

'smartcase' 'scs'　　布尔型　(缺省关闭)
全局

如果搜索模式包含大写字符，不使用 'ignorecase' 选项。只有在输入搜索模式并且打开 'ignorecase' 选项时才会使用。用于 "/"、"?"、"n"、"N"、 ":g"

和 ":s" 命令。不用于 "*"、"#"、"gd"、标签查找等。在 "*" 和 "#" 之后，可以通过 "/" 命令从历史里回忆搜索模式，然后按回车使用 'smartcase'。
注意：如果置位 'compatible'，本选项被复位。

'smartindent' 'si' 'nosmartindent' 'nosi'
'smartindent' 'si' 布尔型 (缺省关闭)
局部于缓冲区
{仅当编译时加入 +smartindent 特性才有效}
开启新行时使用智能自动缩进。适用于 C 这样的程序，但或许也能用于其它语言。'cindent' 类似，它多数情况下更好，但更严格，见 C-indenting 。如果打开 'cindent' 或设置了 'indentexpr'，置位 'si' 没有效果。
'indentexpr' 是更高级的替代方案。
通常，使用 'smartindent' 时也应该打开 'autoindent'。
在这些情况下自动插入缩进：
- '{' 结束的行后。
- 'cinwords' 中的某个关键字开始的行后。
- '}' 开始的行前 (只有使用 "O" 命令才会)。
在新行第一个输入的字符如果是 '}'，该行使用匹配的 '{' 相同的缩进。
在新行第一个输入的字符如果是 '#'，该行的缩进被删除，'#' 被放到第一列。下一行上，恢复原来缩进。如果你不想这么做，使用下面的映射：
":inoremap # X^H#"，其中的 ^H 用 CTRL-V CTRL-H 输入。
使用 ">>" 命令时，'#' 开始的行不右移。
注意：如果置位 'compatible'，本选项被复位。
'paste' 置位时复位本选项，复位时恢复本选项。

'smarttab' 'sta' 'nosmarttab' 'nosta'
'smarttab' 'sta' 布尔型 (缺省关闭)
全局
如果打开，行首的 <Tab> 根据 'shiftwidth' 插入空白。'tabstop' 或 'softtabstop' 用在别的地方。<BS> 删除行首 'shiftwidth' 那么多的空白。
如果关闭，<Tab> 总是根据 'tabstop' 或 'softtabstop' 决定插入空白的数目。'shiftwidth' 只用于文本左移或右移 shift-left-right 。
插入空白的具体方式 (制表还是空格) 取决于 'expandtab' 选项。另见 ins-expandtab 。如果没有置位 'expandtab'，通过使用 <Tab>，使空格数目减到最小。
'paste' 置位时复位本选项，复位时恢复本选项。
注意：如果置位 'compatible'，本选项被复位。

'softtabstop' 'sts'
'softtabstop' 'sts' 数值型 (缺省为 0)
局部于缓冲区
执行编辑操作，如插入 <Tab> 或者使用 <BS> 时，把 <Tab> 算作空格的数目。
"感觉上" 你就像使用单个 <Tab> 一样，而实际上使用的是空格和 <Tab> 的混合。这可以用来维持 'ts' 的设置为标准值 8 不变，但编辑时感觉就像它被设为 'sts' 那样。不过，"x" 这样的命令仍然会在实际的字符上操作。
如果 'sts' 为零，关闭此特性。
如果 'sts' 为负，使用 'shiftwidth' 的值。
如果置位 'paste' 选项，'softtabstop' 被设为 0。'paste' 复位时恢复本选项。
另见 ins-expandtab 。如果没有置位 'expandtab'，通过使用 <Tab>，使空格数目减到最小。
'cpoptions' 里的 'L' 标志位改变制表在 'list' 置位时的使用方式。

注意：如果置位 'compatible'，本选项被设为 0。

如果 Vim 编译时带了 +vartabs 特性且 'varsofttabstop' 不是空串，忽略 'softtabstop' 的值。

'spell' 'nospell'
'spell' 布尔型 (缺省关闭)
局部于窗口
{仅当编译时加入 +syntax 特性才有效}
如果打开，进行拼写检查。见 spell 。
用 'spelllang' 指定使用的语言。

'spellcapcheck' 'spc'
'spellcapcheck' 'spc' 字符串 (缺省为 "[.?!]_[\])'" \t]\+")
局部于缓冲区
{仅当编译时加入 +syntax 特性才有效}
定位句子结束位置的模式。检查匹配文本之后的下个单词是否以大写字母开头。
如果不是，用 SpellCap 高亮 hl-SpellCap (除非该词同时包含拼写错误)。
如果不希望有这项检查，置本选项为空。
只有置位 'spell' 时才使用。
小心特殊字符， option-backslash 说明如何包含空格和反斜杠。
要根据语言自动设置此选项，见 set-spc-auto 。

'spellfile' 'spf'
'spellfile' 'spf' 字符串 (缺省为空)
局部于缓冲区
{仅当编译时加入 +syntax 特性才有效}
单词列表文件名，此文件用于 zg 和 zw 命令加入单词。它必须以
".{encoding}.add" 结尾。路径是必要的，否则该文件就放到当前目录。
E765
它也可以是逗号分隔的名字列表。 zg 和 zw 命令前的计数用来指定需要的名字。这可用于，比方说，分别指定个人单词列表和项目单词列表文件。
如果此选项在加入单词时为空，Vim 会为你进行如下设置：使用第一个可写的 'runtimepath' 目录；如果那里还没有 "spell" 目录，先建立之；然后使用 'spelllang' 里的出现的第一个语言名作为文件名，忽略区域部分。
生成的 ".spl" 文件将用于拼写检查，它不需要在 'spelllang' 里出现。
通常所有区域使用同一个文件，如果你想加入区域名也可以。但要记住，只有 'spellfile' 设置为该名字时才会使用区域名。使用 'spelllang' 项目的缺省方法只会寻找不带区域名的文件。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'spelllang' 'spl'
'spelllang' 'spl' 字符串 (缺省为 "en")
局部于缓冲区
{仅当编译时加入 +syntax 特性才有效}
逗号分隔的单词列表名字的列表。打开 'spell' 选项时，为这些语言进行拼写检查。例如：

```
set spelllang=en_us,nl,medical
```

意味着识别美国英语、荷兰语和医疗用语。高亮不能识别的单词。
单词列表名由字母、数字、连字符或下划线组成。不能包含逗号或点号。建议使用连字符来分隔两字母的语言名和规格说明。这样，"en-rare" 代表罕见的英语单词。
区域名必须出现在名字的最后，它的形式是 "_xx"，其中 "xx" 是两字母小写的区域名。你可以使用多于一个区域，一一列出便可："en_us,en_ca" 支持美国和

加拿大英语，但不支持澳大利亚、新西兰或英国专有的单词。(备注：目前 en_au 和 en_nz 字典比 en_ca、en_gb 和 en_us 的要老)。
如果包含名字 "cjk"，拼写检查排除东亚字符。对编辑同时包含亚洲单词的文本有用。

E757
一个特例是可以直接给出一个 .spl 文件名。删除名字中第一个 "_xx" 部分，把删除部分用作区域名 (_xx 是下划线和两个字母，它后面要跟一个非字母)，主要为测试用。你必须确保使用正确的编码，Vim 不做此检查。
设置 'encoding' 时，会重新载入单词列表。因而，最好在设置 'encoding' 之后设置 'spelllang'，避免重复载入文件。
如何找到相关的拼写文件的解释可见： spell-load 。

如果激活 spellfile.vim 插件而你使用 'runtimepath' 里找不到 .spl 的语言名，该插件会询问你是否要下载此文件。

成功设置此选项后，Vim 会执行 'runtimepath' 的 "spell/LANG.vim" 文件。其中 "LANG" 是 'spelllang' 第一个非 ASCII 字母、数字或连字符的字符之前的值。另见 set-spc-auto 。

'spellsuggest' 'sps'

'spellsuggest' 'sps' 字符串 (缺省为 "best")
全局
{仅当编译时加入 +syntax 特性才有效}
拼写检查使用的方法。用于 z= 命令和 spellsuggest() 函数。它是逗号分隔的项目列表：

best 内部方法，最适用于英语。搜寻需要的改动的方式类似于 "fast"，还用到少许基于发音相近的计分机制，以改进排列的顺序。

double 内部方法，使用两个方法并混合其结果。第一个方法是 "fast"，另一个是计算建议单词和坏词发音相近程度。只能用于能按发音折叠的语言。可能较慢，且结果不一定更好。

fast 内部方法，只检查简单的改动：字符插入/删除/交换。对简单的拼写错误效果不错。

{number} z= 列出的最大建议数目。不用于 spellsuggest() 。建议数目永远不会比 'lines' 的值减 2 更多。

file:{filename} 读入文件 {filename}，必须包含斜杠分隔的两列。第一列包含坏词，第二列包含建议的好词。
例如：

```
theribal/terrible
```

用于没有出现在内部方法提供的建议列表顶部的常见错误。
忽略没有斜杠的行，这可用于注释。
第二列的单词必须正确，否则不予使用。如果单词目前被标记为拼写错误，该单词被加到一个 ".add" 文件里。
此文件用于所有语言。

expr:{expr} 计算表达式 {expr}。使用函数可以避免空格的麻烦。 v:val 是拼写错误的单词。该表达式必须返回列表的列表，每个项目包含建议和评分两项。

例如：

```
[['the', 33], ['that', 44]]
```

设置 'verbose' 并用 z= 可以看到内部方法使用的评分。分数越小越好。

如果你暂时从 'spellsuggest' 里排除 "expr:" 部分，也可以调用 spellsuggest() 。

安静地忽略错误，除非你设置 'verbose' 选项为非零值。

只能使用 "best"、"double" 或 "fast" 中的一个。其它项目可以出现多次，任何顺序都可以。例如：

```
:set sps=file:~/.vim/sugg,best,expr:MySuggest()
```

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'splitbelow' 'sb' 'nosplitbelow' 'nosb'
'splitbelow' 'sb' 布尔型 (缺省关闭)
全局

如果打开，窗口的分割会把新窗口放到当前窗口之下。 :split

'splitright' 'spr' 'nosplitright' 'nospr'
'splitright' 'spr' 布尔型 (缺省关闭)
全局

如果打开，窗口的分割会把新窗口放到当前窗口之右。 :split

'startofline' 'sol' 'nostartofline' 'nosol'
'startofline' 'sol' 布尔型 (缺省打开)
全局

如果打开，下面列出的命令把光标移动到行首的第一个非空白。如果关闭，光标保持在同一列上 (如果可能的话)。这适用于以下命令：
CTRL-D、CTRL-U、CTRL-B、CTRL-F、"G"、"H"、"M"、"L"，gg，以及使用面向行操作符的 "d"、"<<" 和 ">>"，还有带计数的 "%" 和缓冲区改变命令 (CTRL-^、 :bnext、:bNext，等等)，最后包括只有单个行号的 Ex 命令，比如 ":25" 或 ":+"。
在缓冲区改变命令里，光标定位在上次缓冲区编辑时它所在的列上。
注意：如果置位 'compatible'，本选项也被置位。

'statusline' 'stl' E540 E542
'statusline' 'stl' 字符串 (缺省为空)
全局或局部于窗口 global-local
{仅当编译时加入 +statusline 特性才有效}

如果非空，本选项决定状态行的内容。另见 status-line 。

此选项包含 printf 风格的 '%' 项目，中间可以间杂普通文本。每个状态行项目有如下形式：

```
%-0{minwid}.{maxwid}{item}
```

除了 {item} 以外，每个字段都是可选的。单个百分号可以用 "%%" 给出。最多可给出 80 个项目。 E541

如果此选项以 "%!" 开始，它用作表达式。计算此表达式的结果用作选项值。例如：

```
:set statusline=%!MyStatusLine()
```

g:statusline_winid 变量会设为状态行所属的窗口的 window-ID 。
返回值可以包含 %{} 项目，它还会被继续计算下去。

注意 "%!" 表达式计算的上下文是当前窗口和缓冲区，而 %{} 项目计算的上下文是状态行所属的窗口。

如果计算选项时有错误，会把它清空以防将来继续出错。否则屏幕更新会陷入循环。

注意 如果设置本选项（并且 'laststatus' 为 2 的话），'ruler' 的唯一效果是控制 CTRL-G 的输出。

域	含义
-	左对齐项目。如果 minwid 大于项目的长度，缺省是右对齐。
0	数值项目前面用零填补。'-' 更优先。
minwid	项目的最小宽度，以 '-' 和 '0' 补空。该值不能超过 50。
maxwid	项目的最大宽度。如果超过，在文本项目的左侧截短，以 '<' 代替。数值项目则往下移到 maxwid-2 个数位，然后跟 '>'number，其中的 number 是丢失的数位，这非常类似于指数记法。
item	单个字符的代码，下面给出描述。

下面是可能状态行项目的描述。其中，"项目" 的第二个字符代表类型：

N 代表数值型
S 代表字符串型
F 代表下面描述的标志位
- 不适用

项目	含义
f S	缓冲区的文件路径，保持输入的形式或相对于当前目录。
F S	缓冲区的文件完整路径。
t S	缓冲区的文件的文件名（尾部）。
m F	修改标志位，文本是 "[+]"；若 'modifiable' 关闭则是 "[-]"。
M F	修改标志位，文本是 ",+" 或 ",-"。
r F	只读标志位，文本是 "[RO]"。
R F	只读标志位，文本是 ",RO"。
h F	帮助缓冲区标志位，文本是 "[help]"。
H F	帮助缓冲区标志位，文本是 ",HLP"。
w F	预览窗口标志位，文本是 "[Preview]"。
W F	预览窗口标志位，文本是 ",PRV"。
y F	缓冲区的文件类型，如 "[vim]"。见 'filetype'。
Y F	缓冲区的文件类型，如 ",VIM"。见 'filetype'。
q S	"[Quickfix List]"、"[Location List]" 或空。
k S	"b:keymap_name" 的值或使用 :lmap 映射时的 'keymap': "<keymap>"。
n N	缓冲区号。
b N	光标所在字符的值。
B N	同上，以十六进制表示。
o N	光标所在字节在文件中的字节偏移，第一个字节为 1。助记：从文件开始的偏移（Offset）（加上 1）{仅当编译时加入 +byte_offset 特性才有效}
O N	同上，以十六进制表示。
N N	打印机页号。（只用于 'printheader' 选项。）
l N	行号。
L N	缓冲区里的行数。
c N	列号。
v N	虚拟列号。
V N	虚拟列号，表示为 -{num}。如果等于 'c' 的值，不显示。

p N　行数计算在文件位置的百分比，如同 CTRL-G 给出的那样。
P S　显示窗口在文件位置的百分比，类似于 'ruler' 描述的百分比。长度总是为 3，除非经过翻译。
a S　参数列表状态，就像缺省标题里的那样。({current} of {max})
如果参数列表里的文件数为零或一，空字符串。
{ NF　计算 '%{' 和 '}' 之间的表达式，并返回其结果替代。注意 结束的 '}' 之前没有 '%'。表达式不能包含 '}' 字符，要绕过这点，请调用函数。
见下 stl-%{ 。
(-　项目组的开始。可以用来为某组项目设置宽度和对齐。后面某处必须有 %)。
) -　项目组的结束。不能指定宽度域。
T N　用于 'tabline': 标签页 N 标签的开始。最后一个标签之后用 %T。鼠标点击时用此信息。
X N　用于 'tabline': 关闭标签页 N 标签的开始。%X 则用来结束当前标签。例如: %3Xclose%X，用 %999X 来代表 "关闭当前标签页" 那个符号。鼠标点击时用此信息。
< -　如果行过长，在什么地方截短。缺省是在开头。不能指定宽度域。
= -　左对齐和右对齐项目之间的分割点。不能指定宽度域。
-　设置高亮组。必须后面跟名字，然后又是 #。这样，%#HLname# 代表高亮组 HLname。包括非当前窗口的状态行都使用相同的高亮组。
* -　设置高亮组为 User{N}，其中的 {N} 取自 minwid 域，比如 %1*。用 %* 或者 %*0 可以恢复 normal 高亮。User{N} 和 StatusLine 的区别也会应用到非当前窗口的状态行使用的 StatusLineNC 上。
数字 N 必须从 1 到 9。见 hl-User1..9

显示标志位时，如果它紧跟在普通文本之后，Vim 删除之前的前导逗号。这使得下面例子里使用的标志位显示看起来很舒服。

如果组内的所有项目都是空字符串（比如，标志位没有设置）而该组没有设置 minwid，整个组成为空字符串。这使得下面这样的组完全从状态行上消失，如果没有标志位被置位的话。

```
:set statusline=...%(\ [%M%R%H]%)...
```

要小心，每次显示状态行时都要计算此表达式。

stl-%{ g:actual_curbuf g:actual_curwin

计算 %{} 时，当前缓冲区和当前窗口会临时设为目前要显示的状态行所属的窗口（缓冲区），而表达式会使用此上下文计算。
变量 "g:actual_curbuf" 被设为实际的当前缓冲区的 'bufnr()' 号，而 "g:actual_curwin" 被设为实际的当前窗口的 window-ID 。这些值均为字符串类型。

如果从模式行中设置，'statusline' 选项的计算在沙盘 sandbox 里进行，见 sandbox-option 。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

计算 'statusline' 时不允许修改文本或者跳到其它窗口 textlock 。

如果状态行在你希望时没有更新（如在设置完表达式里使用的某变量之后），设置选项可以强制进行更新而无须更改其值。例如:

```
:let &ro = &ro
```

如果结果全是数字，用作显示时把它作为数值处理。否则结果作为文本，并应用上面描述的规则。

小心表达式里的错误。它们可能使 Vim 不可用！

如果你被困住，按住 ':' 或 'Q' 来得到提示，然后退出并用 "vim --clean" 来编辑 .vimrc 或者别的什么地方，以修正问题。

示例:
模拟 'ruler' 设置的标准状态行
```
:set statusline=%<%f\ %h%m%r%=%-14.(%l,%c%V%)\ %P
```
类似，但加上光标所在字符的 ASCII 值（类似于 "ga"）
```
:set statusline=%<%f%h%m%r%=%b\ 0x%B\ \ %l,%c%V\ %P
```
显示字节偏移和字节值，用红色标记已修改标志位。
```
:set statusline=%<%f%=\ [%1*%M%*%n%R%H]\ %-19(%3l,%02c%03V%)%O'%02b'
:hi User1 term=inverse,bold cterm=inverse,bold ctermfg=red
```
如果载入的是压缩文件，显示 ,GZ 标志
```
:set statusline=...%r%{VarExists('b:gzflag','\ [GZ]')}%h...
```
并在 :autocmd 里:
```
:let b:gzflag = 1
```
或:
```
:unlet b:gzflag
```
还要定义此函数:
```
:function VarExists(var, val)
:    if exists(a:var) | return a:val | else | return '' | endif
:endfunction
```

'suffixes' 'su'
'suffixes' 'su'　　字符串　(缺省为 ".bak,~,.o,.h,.info,.swp,.obj")
全局
带这些后缀名的文件在通配符能匹配多个文件时使用较低的优先级。见 'suffixes'。可以用逗号分隔这些后缀，逗号之后的空格被忽略。句号也被看作是后缀的开始。要避免句号或逗号被识别为分隔符，在它们之前加上反斜杠（option-backslash 说明如何包含空格和反斜杠）。
'wildignore' 说明如何完全忽略某些文件。
建议使用 :set+= 和 :set-= 来从列表里加减后缀。这可以避免未来版本使用其它缺省值出现的问题。

'suffixesadd' 'sua'
'suffixesadd' 'sua'　　字符串　(缺省为 "")
局部于缓冲区
{仅当编译时加入 +file_in_path 特性才有效}
逗号分隔的后缀名列表。"gf"、"[I" 等命令搜索文件时使用它们。例如:
```
:set suffixesadd=.java
```

'swapfile' 'swf' 'noswapfile' 'noswf'
'swapfile' 'swf'　　布尔型 (缺省打开)
局部于缓冲区
缓冲区使用交换文件。如果不想为特定缓冲区使用交换文件，可以复位本选项。例如，包含即使 root 也不应得到的机密信息。要小心: 所有的文本都在内存:
- 不要在大文件里使用。
- 无法恢复!

交换文件只有在 'updatecount' 不为零并且置位 'swapfile' 时才会存在。复位 'swapfile' 时，立即删除当前缓冲区的交换文件。如果置位 'swapfile' 并且 'updatecount' 非零，立即建立交换文件。
另见 swap-file 和 'swapsync'。
如果你要打开新缓冲区而不想为之建立交换文件，用 :noswapfile 修饰符。
'directory' 说明交换文件创建的位置。

此选项可以和 'bufhidden' 和 'buftype' 一起使用，指定特殊类型的缓冲区。见 special-buffers 。

'swapsync' 'sws'

'swapsync' 'sws' 字符串 (缺省为 "fsync")
全局
如果此选项非空，写入交换文件后同步到磁盘上。这需要一点时间，尤其在繁忙的 unix 系统上。
如果此选项为空，交换文件的部分内容可能在内存里，还没写回磁盘上。如果系统崩溃，你可能会丢失更多的工作。
Unix 上，系统时不时进行同步，无须 Vim 请求。所以关闭此选项的缺点很少。有的系统上，交换文件完全不会被写入。在 Unix 系统上，设置它为 "sync" 会使用 sync() 调用而不是缺省的 fsync()，在有的系统上这样做可能效果更好。
'fsync' 选项用于实际文件上。

'switchbuf' 'swb'

'switchbuf' 'swb' 字符串 (缺省为 "")
全局
本选项控制缓冲区切换的行为。可能的值是 (逗号分隔的列表):

useopen	如果包含，跳到第一个打开的包含指定缓冲区的窗口 (如果有的话)。 否则: 不检查其它窗口。 quickfix 命令在跳转到错误时 (":cc"、":cn"、"cp"，等等) 检查此设置。所有缓冲区相关的分割命令也会，比如 ":sbuffer"、":sbnext" 或 ":sbrewind"。
usetab	类似于 "useopen"，但也考虑其它标签页里的窗口。
split	如果包含，在载入用于在 quickfix 命令里显示错误的缓冲区之前分割当前窗口。不然: 不分割，使用当前窗口 (如果已在 quickfix 窗口中: 上次使用的窗口，如果没有其它窗口，还是会分割)。
vsplit	类似于 "split"，但水平分割。
newtab	类似于 "split"，但打开新标签页。如果和 "split" 同时存在，本值优先。
uselast	如果包含，当用 quickfix 命令跳转到错误时，跳到上次使用的窗口。

'synmaxcol' 'smc'

'synmaxcol' 'smc' 数值型 (缺省为 3000)
局部于缓冲区
{仅当编译时加入 +syntax 特性才有效}
搜索语法项目的最大列数。长行里超过此列数的文本不再高亮，后续行也不一定能正确高亮。因为语法状态被清除。
有助于避免单个长行的 XML 文件的重画非常缓慢的问题。
设为零取消此限制。

'syntax' 'syn'

'syntax' 'syn' 字符串 (缺省为空)
局部于缓冲区
{仅当编译时加入 +syntax 特性才有效}
如果设置此选项，载入该名字对应的语法，除了 ":syntax off" 用来关闭语法高亮以外。
否则，此选项并不总是反映当前的语法 (b:current_syntax 变量才是)。
对于不能自动识别语法的文件而言，在模式行里设置此选项是最有用的。比如对于 IDL 文件:

```
/* vim: set syntax=idl : */
```

如果选项值里有句号，它分隔两个文件类型名字。例如：

```
/* vim: set syntax=c.doxygen : */
```

先使用 "c" 语法，再使用 "doxygen" 语法。注意 第二个语法须准备好作为附加部分被载入，否则它会被跳过。句号可以多于一个。

要关闭当前文件的语法高亮，用：

```
:set syntax=OFF
```

要根据当前的 'filetype' 选项的值打开语法高亮：

```
:set syntax=ON
```

设置 'syntax' 选项的实际效果是激活以该值为参数的 Syntax 自动命令事件。

不管 'cpoptions' 里是否包含 's' 或 'S' 标志位，本选项不会复制到别的缓冲区。

只能使用普通的文件名字符。"/*?[|<>" 都不合法。

'tabline' 'tal'

'tabline' 'tal' 字符串 (缺省为空)
全局

如果非空，此选项决定 Vim 窗口顶部标签页行的内容。如果为空，Vim 使用缺省的标签页行。详见 setting-tabline 。

只有符合 'showtabline' 选项的要求而且没有 GUI 标签页行时才显示这里的标签页行。如果 'guioptions' 里包含 'e' 且 GUI 支持标签页行，用 'guioptions' 代替。

此值的计算类似于 'statusline'。你可以用 tabpagenr() 、tabpagewinnr() 和 tabpagebuflist() 来决定要显示的文本。用 "%1T" 来指定第一个标签，"%2T" 第二个，依此类推。用 "%X" 项目指定关闭的标签。

当 'tabline' 使用的某些值被改变而没有触发其更新时，使用 :redrawtabline 。

'modelineexpr' 关闭时，不能在模式行中设置此选项。

记住只有一个标签页是当前标签页，其它标签页都是不可见的，你不能调到它们的窗口里。

'tabpagemax' 'tpm'

'tabpagemax' 'tpm' 数值型 (缺省为 10)
全局

-p 命令行参数或 ":tab all" 命令打开的最大标签页数。 tabpage

'tabstop' 'ts'

'tabstop' 'ts' 数值型 (缺省为 8)
局部于缓冲区

文件里的 <Tab> 代表的空格数。另见 :retab 命令和 'softtabstop' 选项。

注意：设置 'tabstop' 为不同于 8 的值可能使你的文件在很多地方看起来不正确 (比如，打印时)。

Vim 里有四个主要的使用制表的方法：

1. 总是保持 'tabstop' 为 8，设置 'softtabstop' 和 'shiftwidth' 为 4 (或 3 或任何你想要的) 然后用 'noexpandtab'。这时，Vim 使用制表和空

格的混合，但输入 <Tab> 或 <BS> 键就像每个制表占用 4（或 3）个字符一样。
2. 设置 'tabstop' 和 'shiftwidth' 为想要的任何值，然后用 'expandtab'。这样，你总是插入空格。改变 'tabstop' 时绝不会影响排版。
3. 设置 'tabstop' 和 'shiftwidth' 为想要的任何值，然后用 modeline，再次编辑时就会重新设置这些值。这只适用于总是使用 Vim 进行文件编辑的情况。
4. 永远把 'tabstop' 和 'shiftwidth' 设为相同的值，并用 'noexpandtab'。这样，就可以（只适用于行首的缩进）使用任何别人的制表位设置。不过，如果你这么做，最好在第一个非空白字符之后想插入制表时以空格代替。否则，改变 'tabstop' 时，注释等的对齐会不正确。

如果 Vim 编译时带了 +vartabs 特性且 'vartabstop' 不是空串，忽略 'tabstop' 的值。

'tagbsearch' 'tbs' 'notagbsearch' 'notbs'
'tagbsearch' 'tbs' 布尔型 (缺省打开)
全局

查找标签时（比如 :ta 命令），Vim 可以在标签文件里使用二分法查找或者线性查找。二分法查找使标签的查找过程会快得多，但如果标签文件没有进行适当的排序，线性查找会找到更多的标签。
Vim 通常假设你的标签文件已经排序，或者明确指示说它们没有排序。只有不符合这些情况，才需要关闭 'tagbsearch' 选项。

如果打开 'tagbsearch'，先在标签文件里使用二分法查找。在特定场合下，Vim 会为特定文件代之以线性查找，或在使用线性查找重试所有的文件。如果关闭 'tagbsearch'，只使用线性查找。

如果 Vim 发现文件开头的行里指示该文件没有排序，总是对该文件进行线性查找：

```
!_TAG_FILE_SORTED	0	/一些注释/
```

['0' 之前和之后的空白必须是单个 <Tab>]

如果二分法查找进行完毕但在所有 'tags' 列出的文件中找不到匹配，而且忽略大小写或者使用的是模式而不是普通的标签名，用线性查找重试。没有排序的标签文件里的标签和大小写不同的匹配，只有在重试里才能找到。

如果标签文件指示它按大小写合并排序并且忽略大小写时，可以避免两次线性查找。在 "!_TAG_FILE_SORTED" 行中使用值 '2'。标签文件可以用多数 unix 系统上存在的 "sort" 程序的 -f 选项完成大小写合并的排序，类似于下面的命令："sort -f -o tags tags"。对于 "Exuberant ctags" 5.x 或更高的版本（至少 5.5），--sort=foldcase 选项也可用于此目的。注意 要使之工作，大小写必须都被合并到大写字母上。

缺省，标签查找是大小写敏感的。如果 'ignorecase' 置位并且 'tagcase' 为 "followic"，或 'tagcase' 为 "ignore" 时，忽略大小写。
'tagcase' 为 "followscs" 并且 'smartcase' 置位，或者 'tagcase' 为 "smart" 并且模式只包含小写字符时，也是如此。

如果 'tagbsearch' 关闭，标签查找慢得多，但找不到完整的匹配比找到完整匹配的更慢。没有排序的标签文件里的标签只能在 'tagbsearch' 关闭时找到。
如果标签文件没有排序，或者排序方法不正确（不按照 ASCII 字节值排序），'tagbsearch' 必须关闭，或至少上面给出的行应该包含在标签文件里。
本选项不影响查找所有匹配标签的命令（例如，命令行补全和 ":help"）。

'tagcase' 'tc'

'tagcase' 'tc'　　字符串　(缺省为 "followic")
　　全局或局部于缓冲区　global-local
此选项指定标签文件查找时如何处理大小写:
　　followic　　依照 'ignorecase' 选项
　　followscs　　依照 'smartcase' 和 'ignorecase' 选项
　　ignore　　忽略大小写
　　match　　匹配大小写
　　smart　　除非使用大写字母，忽略大小写
注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'tagfunc' 'tfu'

'tagfunc' 'tfu'　　字符串　(缺省: 空)
　　局部于缓冲区
　　{仅当编译时加入 +eval 特性才有效}
此选项指定函数，用于执行标签搜索。此函数取得标签模式并应返回匹配标签的列表。见 tag-function 说明如何写此函数并给出一例。

'taglength' 'tl'

'taglength' 'tl'　　数值型　(缺省为 0)
　　全局
如果非零，每个标签的有效字符不超过此数目。

'tagrelative' 'tr' 'notagrelative' 'notr'

'tagrelative' 'tr'　　布尔型　(Vim 缺省: 打开，Vi 缺省: 关闭)
　　全局
如果打开并使用别的目录下的标签文件，那个标签文件的文件名相对于标签文件所在的目录。
注意: 如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'tags' 'tag' E433

'tags' 'tag'　　字符串　(缺省为 "./tags,tags"，如果编译时加入 +emacs_tags : "./tags,./TAGS,tags,TAGS")
　　全局或局部于缓冲区　global-local
标签命令所用的多个文件名，空格或逗号分隔。要在文件名里包含空格或逗号，在它们之前加上反斜杠 (option-backslash 说明如何包含空格和反斜杠)。如果文件名以 "./" 开始，'.' 被替换为当前文件的路径。但只在 'cpoptions' 里没有包含 'd' 标志位时才会如此。环境变量被扩展 :set_env 。另见 tags-option 。
可以用 "*"、"**" 和其它通配符来搜索目录树下的标签文件。见 file-searching 。例如，"/lib/**/tags" 会找到 "/lib" 下面所有名为 "tags" 的文件。文件名本身不能包含通配符，会按原样使用。例如 "/lib/**/tags?" 会找到名为 "tags?" 的文件
{仅当编译时加入 +path_extra 特性才有效}
可以用 tagfiles() 函数来得到实际使用的文件名列表。
如果 Vim 编译时加入 +emacs_tags 特性，也支持 Emacs 风格的标签文件。它们被自动识别。缺省值这时变成 "./tags,./TAGS,tags,TAGS"，除非大小写被忽略 (MS-Windows)。 emacs-tags
建议使用 :set+= 和 :set-= 来从列表里加减文件名。这可以避免未来版本使用其它缺省值出现的问题。

'tagstack' 'tgst' 'notagstack' 'notgst'
'tagstack' 'tgst' 布尔型 (缺省打开)
全局
如果打开，正常使用 tagstack 。如果关闭，带参数的 ":tag" 或 ":tselect" 命令不会把标签推入标签栈中。后续的不带参数的 ":tag"、":pop" 命令或者其它使用的标签栈的命令会使用没有改动过的标签栈，但会改变活动项目的指针。复位本选项，可在映射里使用 ":tag" 命令而不期望它去改变标签栈。

'tcldll'
'tcldll' 字符串 (缺省取决于编译设置)
全局
{仅当编译时加入 +tcl/dyn 特性才有效}
指定 Tcl 共享库的名字。缺省是编译时指定的 DYNAMIC_TCL_DLL。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'term' E529 E530 E531
'term' 字符串 (缺省为 $TERM，如果失败:
GUI: "builtin_gui"
Amiga: "amiga"
BeOS: "beos-ansi"
Mac: "mac-ansi"
MiNT: "vt52"
MS-DOS: "pcterm"
Unix: "ansi"
VMS: "ansi"
Win 32: "win32")
全局
终端的名字。用于选择终端控制字符。环境变量被扩展 :set_env 。
例如:

```
:set term=$TERM
```

见 termcap 。

'termbidi' 'tbidi'
'notermbidi' 'notbidi'
'termbidi' 'tbidi' 布尔型 (缺省关闭，"mlterm" 打开)
全局
{仅当编译时加入 +arabic 特性才有效}
终端负责双向显示文本 (根据 Unicode 的定义)。也期待终端完成一些语言 (比如阿拉伯语) 所需的字型重整。
置位本选项意味着置位 'arabic' 时不会置位 'rightleft'，而且也忽略 'arabicshape' 的值。
注意 置位 'termbidi' 会立即忽略 'arabicshape'，但 'rightleft' 不会自动进行修改。
GUI 启动时复位本选项。
详见 arabic.txt 。

'termencoding' 'tenc'
'termencoding' 'tenc' 字符串 (缺省为 ""; GTK+ GUI: "utf-8";
Macintosh GUI: "macroman")
全局
终端使用的编码。它指定键盘产生的和显示能识别的字符编码。对 GUI 而言，这只适用于键盘 ('encoding' 用于显示)。Mac 上如果 'macatsui' 关闭时例

外，此时 'termencoding' 应是 "macroman"。

E617

注意：不适用于 GTK+ GUI。那里，GUI 成功初始化后，'termencoding' 被强制设为 "utf-8"。任何试图设置不同值的努力都被拒绝，而且会给出错误消息。
Win32 GUI 和控制台版本不使用 'termencoding'，因为 Win32 系统总是传入 Unicode 字符。
如果为空，使用和 'encoding' 选项相同的编码。这是正常情况。
不是所有的 'termencoding' 和 'encoding' 的组合都是可以的。见 encoding-table 。
此选项的值必须被内部转换机制或 iconv() 支持。如果这不可能，不会进行任何转换，非 ASCII 字符可能会遇到问题。
例如：你在 euc-jp（日本语）locale 的设置下工作，但需要编辑 UTF-8 文件：

```
:let &termencoding = &encoding
:set encoding=utf-8
```

如果你的系统没有 UTF-8 的 locale 支持，需要这么做。

'termguicolors' 'tgc' E954

'termguicolors' 'tgc' 布尔型（缺省关闭）
全局
{仅当编译时加入 +termguicolors 特性才有效}

打开时，在终端上使用 highlight-guifg 和 highlight-guibg 属性（因而使用 24-位彩色）。

需要 ISO-8613-3 兼容终端。如果置位本选项无效（产生无色的 UI），阅读 xterm-true-color 可能有帮助。

Win32 控制台需要 Windows 10 1703 (Creators Updae) 或之后版本。可以这样确定：

```
if has('vcon')
```

这需要 Vim 编译时带上 +vtp 特性。

注意 仍然使用 "cterm" 属性，而不是 "gui" 那些。

注意：如果置位 'compatible'，本选项被复位。

'termwinkey' 'twk'

'termwinkey' 'twk' 字符串 （缺省 ""）
局部于窗口

终端窗口中引导 CTRL-W 系列命令的键。其它键发送给窗口中运行中的作业。
可以使用 <> 记号，如：

```
:set termwinkey=<C-L>
```

字符串必须对应一个键击，但可以有多个字节。
如果不指定则使用 CTRL-W，这样 CTRL-W : 把你带到命令行上。如果 'termwinkey' 设为 CTRL-L，则 CTRL-L : 会把你带到命令行上。

'termwinscroll' 'twsl'

'termwinscroll' 'twsl' 数值型 （缺省 10000）
局部于缓冲区
{仅当编译时加入 +terminal 特性才有效}

保留的回滚行数。超过此限制后删除保留回滚行的前 10% 部分。这只是为了减少内容消耗。见 Terminal-Normal 。

'termwinsize' 'tws'

'termwinsize' 'tws'　　字符串　(缺省 "")
局部于窗口
terminal　窗口大小。格式：{rows}x{columns} 或 {rows}*{columns}。
- 如为空，终端使用窗口的大小。
- 如用 "x" 格式（如 "24x80"），终端大小不随窗口大小调整。窗口变小后只显示左上部分。
- 如用 "*" 格式（如 "10*50"），终端大小随着窗口大小调整，但不小于指定的行数和/或列数。
- 行数为零时使用窗口高度。
- 列数为零时使用窗口宽度。
- "0x0" 或 "0*0" 等同于空串。

示例：
"30x0" 使用 30 行和当前窗口宽度。
"20*0" 使用至少 20 行和当前窗口宽度。
"0*40" 使用当前窗口高度和至少 40 列。
注意 终端中运行的命令还可能改变终端大小。此时只要可能，会调整 Vim 窗口到该大小。

'termwintype' 'twt'

'termwintype' 'twt'　　字符串　(缺省 "")
全局
{仅当 MS-Windows 上编译时加入 +terminal 特性才有效}
指定打开终端窗口时使用的虚拟控制台 (pty)。

可能值是：
""　　如果 ConPTY 稳定则使用之，返回使用 winpty
"winpty"　　使用 winpty，如果不支持则失败
"conpty"　　使用 ConPTY，如果不支持则失败

ConPTY 的支持取决于系统。Windows 10 October 2018 更新是首个支持 ConPTY 的版本，不过还是不够稳定。Windows 10 的下个发布 ConPTY 会开始稳定。winpty 的支持需要安装。如果两者都没有，你不能打开终端窗口。

'terse' 'noterse'

'terse'　　布尔型　(缺省关闭)
全局
如果置位：在 'shortmess' 选项里加入 's' 标志位（这使得遭遇文件首部和尾部的搜索不会给出消息）。如果复位：从 'shortmess' 选项里删除 's' 标志位。

'textauto' 'ta' 'notextauto' 'nota'

'textauto' 'ta'　　布尔型　(Vim 缺省：打开，Vi 缺省：关闭)
全局
本选项已废弃。使用 'fileformats'。
为了后向兼容，如果置位 'textauto'，'fileformats' 被设为当前系统的缺省值。如果复位 'textauto'，'fileformats' 被清空。
注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'textmode' 'tx' 'notextmode' 'notx'

'textmode' 'tx'　　布尔型　(Win32：缺省打开，
其它：缺省关闭)

局部于缓冲区
本选项已废弃。使用 'fileformat'。
为了后向兼容，如果置位 'textmode'，'fileformat' 设为 "dos"。如果复位 'textmode'，'fileformat' 设为 "unix"。

'textwidth' 'tw'
'textwidth' 'tw'　　数值型 (缺省为 0)
局部于缓冲区
插入文本的最大宽度。更长的行会在空白之后截断，以达到此宽度。设为零关闭此项功能。
'paste' 选项置位时 'textwidth' 被设为 0。复位时恢复本选项。
如果 'textwidth' 为零，可以使用 'wrapmargin'。另见 'formatoptions' 和 ins-textwidth 。
如果设置了 'formatexpr'，它用来设置如何断开行。
备注：'compatible' 置位时本选项被设为 0。

'thesaurus' 'tsr'
'thesaurus' 'tsr'　　字符串 (缺省为 "")
全局或局部于缓冲区 global-local
逗号分隔的文件名列表，用于为同义词补全命令查找单词 i_CTRL-X_CTRL-T 。

文件里的每行应该包含相近意思的单词，以非关键字字符分隔 (建议使用空白)。最大的行长为 510 个字节。

此 github 议题附加了英语单词列表:
https://github.com/vim/vim/issues/629#issuecomment-443293282
解包 thesaurus_pkg.zip，把 thesaurus.txt 文件放在某处，如 ~/.vim/thesaurus/english.txt，然后把 'thesaurus' 选项设为此文件名。

要在文件名里包含逗号，在它之前加上反斜杠。逗号之后的空格被忽略，其它情况下空格视为文件的一部分。 option-backslash 说明如何使用反斜杠。
建议使用 :set+= 和 :set-= 来从列表里加减目录。这可以避免未来版本使用其它缺省值出现的问题。
为了安全原因，反引号不能用于此选项里。

'tildeop' 'top' 'notildeop' 'notop'
'tildeop' 'top'　　布尔型 (缺省关闭)
全局
如果打开：波浪符命令 "~" 的行为类似于操作符。
注意：如果置位 'compatible'，本选项被复位。

'timeout' 'to' 'notimeout' 'noto'
'timeout' 'to'　　布尔型 (缺省打开)
全局
'ttimeout' 'nottimeout'
'ttimeout'　　布尔型 (缺省关闭， defaults.vim 里置位)
全局
这两个选项一起决定收到部分映射的键序列或键码时的行为：

'timeout'	'ttimeout'	动作
关闭	关闭	没有超时
打开	打开或关闭	为这些打开超时：映射和键码
关闭	打开	为键码打开超时

如果两个选项都关闭，Vim 会一直等待，直到收到完整的映射或键序列为止，如果收到的字符没有对应的映射或者键序列，清除状态。比如：如果映射 "vl" 而 Vim 收到 'v'，需要下一个字符来决定 'v' 后面跟的是不是 'l'。如果打开这里的一个选项，Vim 会为下个字符等待一秒。在此之后，已经收到的字符被解释为单个字符。等待的时间可以用 'timeoutlen' 选项改变。
在慢速的终端或者很繁忙的系统中，超时可能使得光标键操作不正常。如果两个选项都关闭，Vim 在输入 <Esc> 后会永远等待下去，如果键码以 <Esc> 开始，你需要输入 <Esc> 两次。如果键码没有问题而你希望 :map 映射的键序列不等待 1 秒钟超时，置位 'ttimeout' 但关闭 'timeout' 选项。

注意：如果置位 'compatible'，'ttimeout' 被复位。

'timeoutlen' 'tm'
'timeoutlen' 'tm'　　数值型　(缺省为 1000)
全局
{仅有部分 Vi 版本支持}

'ttimeoutlen' 'ttm'
'ttimeoutlen' 'ttm'　　数值型　(缺省为 -1，defaults.vim 里设为 100)
全局

毫秒计的等待键码或者映射的键序列完成的时间。也用于 CTRL-\ CTRL-N 和 CTRL-\ CTRL-G 命令的一部分已经输入的场合。
通常只使用 'timeoutlen'，而 'ttimeoutlen' 设为 -1 就可以。如果希望为键码使用不同的超时的值，设置 'ttimeoutlen' 为非负数。

ttimeoutlen	映射延迟	键码延迟
< 0	'timeoutlen'	'timeoutlen'
>= 0	'timeoutlen'	'ttimeoutlen'

只有 'timeout' 和 'ttimeout' 选项要求时，超时才会发生。可用的设置

```
:set timeout timeoutlen=3000 ttimeoutlen=100
```

(映射上的超时在三秒以后发生，键码上的超时在十分之一秒后发生)。

'title' 'notitle'
'title'　　布尔型　(缺省关闭，标题能恢复时打开)
全局
{仅当编译时加入 +title 特性才有效}

如果打开，窗口标题会被设为 'titlestring' 的值 (如果非空的话)，否则：

```
filename [+=-] (path) - VIM
```

其中：

filename	编辑的文件名
-	指示文件不能修改，'ma' 关闭
+	指示文件已被修改
=	指示文件只读
=+	指示文件只读并且被修改
(path)	被编辑文件的路径
- VIM	服务器名 v:servername 或为 "VIM"

只有终端支持设置窗口标题时才可用 (目前有 Amiga 控制台、Win32 控制台、所有的 GUI 版本和带有非空的 't_ts' 选项的终端 - 缺省情况下，它们是 Unix xterm 和 iris-ansi，其中 't_ts' 从内建的 termcap 读取)。

X11
如果 Vim 编译时定义 HAVE_X11，在可能的情况下会恢复原先的标题。
":version" 的输出会在 HAVE_X11 有定义时包含 "+X11"，否则将是 "-X11"。
这也适用于图标名字 'icon'。
不过：如果 Vim 启动时带 -X 参数，标题无法恢复 (除了 GUI 以外)。如果

标题无法恢复，它被设为 'titleold' 的值。那时，你可能希望在 Vim 之外恢复标题。
如果在远程的机器上使用 xterm，你可以使用命令：

```
rsh machine_name xterm -display $DISPLAY &
```

那么应该会继承 WINDOWID 环境变量，而窗口的标题在退出 Vim 之后应该会换回原先的值。

'titlelen'

'titlelen'　　数值型　(缺省为 85)
全局
{仅当编译时加入　+title　特性才有效}

给出 'columns' 的一个百分比，用于窗口标题的长度。如果标题较之更长，只有路径的尾部会被显示。路径名字前面的 '<' 字符用于指示这种情况。使用百分比使得这和窗口的宽度相适应。但这并不完美，因为实际的字符数也取决于使用的字体还有标题栏的其它部分。如果 'titlelen' 为零，使用完整的路径。不然，可以使用百分之 1 到 30000 的值。
'titlelen' 也用于 'titlestring' 选项。

'titleold'

'titleold'　　字符串　(缺省为 "Thanks for flying Vim")
全局
{仅当编译时加入　+title　特性才有效}

退出 Vim 时如果无法恢复原来的标题，使用本选项。这只会在 'title' 打开或 'titlestring' 不为空时发生。
为了安全原因，本选项不能在　modeline　或　sandbox　里设置。

'titlestring'

'titlestring'　　字符串　(缺省为 "")
全局
{仅当编译时加入　+title　特性才有效}

如果此选项不为空，用来设置窗口的标题。只有在 'title' 选项打开时才会发生。
只有终端支持设置窗口标题时才可用（目前有 Amiga 控制台、Win32 控制台、所有的 GUI 版本和带有非空的 't_ts' 选项的终端）。
如果 Vim 编译时定义 HAVE_X11，在可能的情况下恢复原来的标题，见　X11　。
如果本选项包含 printf 风格的 '%' 项目，依照 'statusline' 使用的规则进行扩展。
'modelineexpr' 关闭时，不能在模式行中设置此选项。

例如：

```
:auto BufEnter * let &titlestring = hostname() . "/" . expand("%:p")
:set title titlestring=%<%F%=%l/%L-%P titlelen=70
```

'titlelen' 的值用来在可用空间的中间或右侧对齐项目。
有的人喜欢文件名放在前面：

```
:set titlestring=%t%(\ %M%)%(\ (%{expand(\"%:~:.:h\")})%)%(\ %a%)
```

注意　"%{ }" 的使用，以及用于得到不含文件名的文件名路径的表达式。只有在必要时，"%(%)" 构造才会加入分隔的空格。
注意：'titlestring' 使用特殊字符可能会使显示引起混乱（比如，如果它包含 CR 或者 NL 字符的话）。
{仅当编译时加入　+statusline　特性才有效}

'toolbar' 'tb'

'toolbar' 'tb'　　字符串　(缺省为 "icons,tooltips")

全局
{仅适用于 +GUI_GTK 、 +GUI_Athena 、 +GUI_Motif 和 +GUI_Photon }

本选项的内容控制若干工具栏的设置。可能的值是:

icons	工具栏按钮使用图标显示。
text	工具栏按钮使用文字显示。
horiz	工具栏按钮使用水平排放的图标和文本。 {仅适用于 GTK+ 2 GUI}
tooltips	激活工具栏按钮的工具提示。

工具提示指的是鼠标光标在工具栏按钮上停留短暂的时间后弹出的帮助文本。

如果你希望工具栏同时显示图标和文本，这么做:

```
:set tb=icons,text
```

Motif 和 Athena 不能同时显示图标和文本。如果两者都被请求，只显示图标。

如果 'toolbar' 指定的字符串都不合法或者 'toolbar' 为空，本选项被忽略。
如果你想关闭工具栏，需要设置 'guioptions' 选项。例如:

```
:set guioptions-=T
```

另见 gui-toolbar 。

'toolbariconsize' 'tbis'
'toolbariconsize' 'tbis' 字符串 (缺省为 "small")
全局
{仅用于 GTK+ GUI}

控制工具栏图标的大小。可能的值是:

tiny	使用微小工具栏图标。
small	使用小工具栏图标 (缺省)。
medium	使用中等大小的工具栏图标。
large	使用大型的工具栏图标。
huge	使用更大型的工具栏图标。
giant	使用非常大的工具栏图标。

准确的以像素点计算的尺寸取决于当前使用的主题。常见的大小为 giant=48x48、huge=32x32、large=24x24、medium=24x24、small=20x20 和 tiny=16x16。

如果 'toolbariconsize' 为空，使用用户偏好里设定的全局缺省大小，或者应用当前的主题。

'ttybuiltin' 'tbi' 'nottybuiltin' 'notbi'
'ttybuiltin' 'tbi' 布尔型 (缺省打开)
全局

如果打开，内建的 termcap 在外部的之前搜索。
如果关闭，内建的 termcap 在外部的之后搜索。
如果本选项被改变，你需要接着设置 'term' 选项，使之能够生效。例如:

```
:set notbi term=$TERM
```

另见 termcap 。
原理: 此选项的缺省值为 "打开"，因为内建的 termcap 项目通常更好 (许多系统的 xterm 项目都有问题...)。

'ttyfast' 'tf' 'nottyfast' 'notf'
'ttyfast' 'tf' 布尔型 (缺省关闭，如果 'term' 为 xterm、hpterm、sun-cmd、screen、rxvt、dtterm 或 iris-ansi 则打开；在 DOS 控制台上运行 Vim 时也打开)

全局
表明使用的是快速终端连接。重画时可以发送给屏幕多个字符，而不使用插入/删除行命令。有多个窗口和终端不支持滚动区域时，会提高重画的平滑度。同时打开需要回绕行的屏幕行的行尾的额外的字符写入。这有助于在 xterm 和其它终端上使用鼠标来进行复制/粘贴。

'ttymouse' 'ttym'
'ttymouse' 'ttym' 字符串 (缺省取决于 'term')
全局
{仅适用于 Unix 和 VMS，不适用于 GUI；只有在编译时加入 +mouse 才有效}
鼠标代码能够识别的终端类型名。目前有以下合法的字符串：

xterm-mouse
xterm 方式的鼠标处理。鼠标产生 "<Esc>[Mscr"，其中 "scr" 是三个字节：
"s" = 按钮状态
"c" = 列号加 33
"r" = 行号加 33
只有不超过 223 列才能工作！可能的解决方法可见 "dec"、"urxvt" 和 "sgr"。

xterm2
和 "xterm" 类似，但 xterm 报告鼠标拖动时的鼠标位置。这样更快也更准确。你的 xterm 必须在补丁号 88 / XFree 3.3.3 或以上才可以。稍后，说明 Vim 的自动识别机制。

netterm-mouse
netterm
NetTerm 鼠标处理。鼠标左键点击产生 "<Esc>}r,c<CR>"，其中 "r,c" 是两个代表行和列的十进制数。不支持其它鼠标事件。

dec-mouse
dec
DEC 终端处理。鼠标产生相当复杂的序列，以 "<Esc>[" 开始。
也可用于 Xterm，如果它在 configure 时带上参数 "--enable-dec-locator"。

jsbterm-mouse
jsbterm
JSB 终端鼠标处理。

pterm-mouse
pterm
QNX 终端鼠标处理。

urxvt-mouse
urxvt
urxvt (rxvt-unicode) 终端鼠标处理。
只有终端支持此编码风格鼠标才能工作，但不像 "xterm" 或 "xterm2" 那样有 223 列的限制。

sgr-mouse
sgr
对产生 SGR 风格鼠标报告的终端鼠标处理。鼠标在 223 以外的列上也能工作。此选项与 "xterm2" 后向兼容，因为它也能解码 "xterm2" 风格的鼠标代码。

鼠标处理的支持必须在编译时打开 +mouse_xterm +mouse_dec +mouse_netterm +mouse_jsbterm +mouse_urxvt +mouse_sgr 。
只有 "xterm"(2) 是总能识别的。如果编译时打开，总能识别 NetTerm 鼠标代码。如果编译时打开，且 'ttymouse' 不为 "xterm"、 "xterm2"、"urxvt" 或 "sgr" 时，能识别 DEC 终端的鼠标代码 (这是因为 dec 的鼠标代码和它们有冲突)。
如果 'term' 选项设为以 "xterm"、"mlterm"、"screen"、"tmux"、"st" (只能完整匹配)、"st-" 或 "stterm" 开始的名字并且 'ttymouse' 还未设的话，本选项自动设为 "xterm"。

此外，在编译时加入 +termresponse 特性并且 t_RV 设为请求 xterm 版本号的转义序列以后，会运行更加智能的检测过程。
如果报告的 xterm 版本是 95 到 276，本值设为 "xterm2"。如果Vim 检测到 Mac Terminal.app、iTerm2 或 mintty，并且（译者注：原文如此，似应为或）报告的 xterm 版本是 277 或更高，本值设为 "sgr"。
如果你不想 'ttymouse' 被自动设为 "xterm2" 或 "sgr"，把 t_RV 置为空字符串：

```
:set t_RV=
```

'ttyscroll' 'tsl'
'ttyscroll' 'tsl' 数值型 （缺省为 999）
全局
屏幕滚动的最大行数。如果有更多行需要滚动，重画窗口。对于滚动很慢但刷新不慢的终端，它应该设为较小的值，比如 3，以加快显示速度。

'ttytype' 'tty'
'ttytype' 'tty' 字符串 （缺省取决于 $TERM）
全局
'term' 的别名，见上。

'undodir' 'udir'
'undodir' 'udir' 字符串 （缺省 "."）
全局
{仅当编译时加入 +persistent_undo 特性才有效}
撤销文件使用的目录名列表，以逗号分隔。
所用格式的详情见 'backupdir' 。
"." 代表文件所在的目录。此时，"file.txt" 对应的撤销文件名是 ".file.txt.un~"。
如使用其他目录，撤销文件名是被编辑文件的完整路径，但路径分隔符以 "%" 代替。
写入时：使用第一个存在的目录。因为 "." 总是可用， "." 之后的目录不会用来写入。
读入时，查找所有的项目来寻找对应的撤销文件。使用第一个找到的撤销文件。如果该文件不可读，报错，但不再继续查找。
见 undo-persistence 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'undofile' 'noundofile' 'udf' 'noudf'
'undofile' 'udf' 布尔型 （缺省关闭）
局部于缓冲区
{仅当编译时加入 +persistent_undo 特性才有效}
打开时，Vim 在把缓冲区写入文件时，自动保存撤销历史到撤销文件。读入文件到缓冲区时，自动恢复撤销历史。
撤销文件写入的目录由 'undodir' 指定。
关于此功能，可详见 undo-persistence 。
如果 'undoreload' 导致重载前为了可撤销而保存缓冲区，不读入撤销文件。
关闭 'undofile' 时并 不 删除撤销文件。
注意：如果置位 'compatible'，本选项被复位。

'undolevels' 'ul'
'undolevels' 'ul' 数值型 （缺省为 100，Unix、VMS、Win32 为 1000）
全局或局部于缓冲区 global-local
可以撤销的最大改变次数。因为撤销需要的信息保存在内存里，更大的数值导致更多的内存被消耗。不过，单个改变本来就可能使用不受限制的内存。

设为 0 和 Vi 兼容：一层撤销，而 "u" 可以撤销自身：

```
set ul=0
```

你可以用在 'cpoptions' 里加入 'u' 标志位得到 Vi 的兼容性，同时可以使用 CTRL-R 来进行多次撤销。
另见 undo-two-ways 。
设为 -1 完全关闭撤销功能。也许只想对当前缓冲区如此设置：

```
setlocal ul=-1
```

如果单个改变也能使你内存溢出，这会有帮助。

如果使用全局值，局部值设为 -123456。

另见 clear-undo 。

'undoreload' 'ur'

'undoreload' 'ur' 数值型 (缺省 10000)
全局
重载缓冲区时为了可撤销，保存整个缓冲区。适用于 ":e!" 命令和在 Vim 之外改变缓冲区时的情况。 FileChangedShell
仅当本选项的值为负或行数小于此选项值时，进行保存。
设置此选项为零关闭重载的撤销。

为重载保存撤销时，不读入撤销文件。

注意 这会导致整个缓冲区被保存到内存。如果发生了内存溢出，设置此选项为较小的值。

'updatecount' 'uc'

'updatecount' 'uc' 数值型 (缺省: 200)
全局
输入这么多个字符以后，把交换文件写入磁盘。如果为零，连交换文件都不会建立（见灾难恢复的章节 crash-recovery ）。以 "-n" 选项启动 Vim 会把 'updatecount' 设为零，见 startup 。在只读模式编辑时，本选项初始化为 10000。
交换文件可以用 'swapfile' 为每个缓冲区分别关闭。
如果 'updatecount' 从零改设为非零，所有置位 'swapfile' 的缓冲区会建立交换文件。如果 'updatecount' 改设为零，已有的交换文件不会删除。
另见 'swapsync' 。
对于 'buftype' 为 "nofile" 或 "nowrite" 的缓冲区，本选项没有作用。

'updatetime' 'ut'

'updatetime' 'ut' 数值型 (缺省为 4000)
全局
如果过了这么多毫秒数以后还没有任何输入，把交换文件写入磁盘（见 crash-recovery ）。也用于 CursorHold 自动命令事件。

'varsofttabstop' 'vsts'

'varsofttabstop' 'vsts' 字符串 (缺省 "")
局部于缓冲区
{仅当编译时加入 +vartabs 特性才有效}
在编辑时，比如插入 <Tab> 或按 <BS> 在时候，把 <Tab> 算作的空格数的一个
列表。"感觉上" 就像插入了可变宽度的 <Tab> 的那样，而事实上是空格和 <Tab> 的混合。制表宽度以逗号分隔，末值应用于所有其后的制表位。

例如，编辑汇编语言文件时，语句从第 9 列开始，而注释从第 41 列开始，可以这么设置：

```
:set varsofttabstop=8,32,8
```

会设定软制表位分别在 8 和 8 + 32 空格位，而其后每列多 8 个空格位。

注意 设置 'varsofttabstop' 时忽略 'softtabstop' 的值。

'vartabstop' 'vts'

'vartabstop' 'vts' 字符串 (缺省 "")
局部于缓冲区
{仅当编译时加入 +vartabs 特性才有效}

在文件中 <Tab> 所代表的空格数一个列表，以逗号分隔。每个值对应一个制表位，末值应用于所有其后的制表位。例如：

```
:set vartabstop=4,20,10,8
```

会设定首个制表位 4 个空格宽，第二个 20 个空格宽，第三个 10 个空格宽，其后每个制表位 8 个空格宽。

注意 设置 'vartabstop' 时忽略 'tabstop' 的值。

'verbose' 'vbs'

'verbose' 'vbs' 数值型 (缺省为 0)
全局

如果大于零，Vim 给出目前进行操作的相关消息。
目前，给出以下消息：

>= 1	viminfo 文件何时进行读写。
>= 2	文件何时被 ":source"。
>= 5	每个被搜索的标签文件和头文件。
>= 8	执行的自动命令组所操作的文件。
>= 9	每个执行的自动命令。
>= 12	每个执行的函数。
>= 13	何时例外被抛出、捕获、结束处理或者放弃。
>= 14	任何在 ":finally" 子句里暂停的部分。
>= 15	每个执行的 Ex 命令 (截短到 200 个字符)。

本选项可以用 "-V" 参数设置。见 -V 。
:verbose 命令也可以设置此选项。

如果设置了 'verbosefile' 选项，不显示 verbose 消息。

'verbosefile' 'vfile'

'verbosefile' 'vfile' 字符串 (缺省为空)
全局

如果非空，写入所有消息到此文件。如果文件已存在，附加消息于其后。
Vim 退出或 'verbosefile' 被清空时，结束文件的写入。写入要通过缓冲，因此不一定能马上看得到写入的内容。
设置 'verbosefile' 为新值就像先把它清空然后再设置一样。
和 :redir 的区别是，在设置 'verbosefile' 时不再显示 verbose 消息。

'viewdir' 'vdir'

'viewdir' 'vdir' 字符串 (Amiga 和 Win32 的缺省:
"$VIM/vimfiles/view",
Unix: "~/.vim/view",
Macintosh: "$VIM:vimfiles:view"
VMS: "sys$login:vimfiles/view"

RiscOS: "Choices:vimfiles/view")
全局
{仅当编译时加入 +mksession 特性才有效}
储存 :mkview 文件的目录名。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'viewoptions' 'vop'
'viewoptions' 'vop' 字符串 (缺省: "folds,options,cursor,curdir")
全局
{仅当编译时加入 +mksession 特性才有效}
修改 :mkview 命令的效果。本选项是逗号分隔的单词列表。每个单词允许保存和恢复某种设置:

单词	保存和恢复
cursor	文件和窗口的光标位置
folds	手动建立的折叠、打开/关闭的折叠和局部折叠选项
options	局部于窗口或缓冲区的选项和映射 (不是局部选项的全局值)
localoptions	等同于 "options"
slash	文件名的反斜杠被替换成正斜杠
unix	使用 Unix 换行格式 (单个 <NL>)，即使在 Windows 上也是如此
curdir	如果用 :lcd 设置过，局部于窗口的目录

"slash" 和 "unix" 可在 MS-Windows 上使用，使得生成的视图文件可以和 Unix 的共享。Unix 版本的 Vim 不能执行 dos 格式的脚本，但 MS-Windows 版本的 Vim 却可以执行 unix 格式的脚本。

'viminfo' 'vi' E526 E527 E528
'viminfo' 'vi' 字符串 (Vi 缺省: "", Vim
MS-Windows 的缺省: '100,<50,s10,h,rA:,rB:,
Amiga: '100,<50,s10,h,rdf0:,rdf1:,rdf2:
其它: '100,<50,s10,h)
全局
{仅当编译时加入 +viminfo 特性才有效}
如果非空，初始化时读入 viminfo 文件，退出 Vim 时写回 (见 viminfo-file)。除非 'viminfofile' 为 "NONE"。
该字符串应该是逗号分隔的参数，每个参数由单个字符组成，标识特定的参数名，后面跟着标识该参数的值的一个数值或字符串。如果某个字符没有列出，则使用该参数的缺省值。下表是能识别的字符和它们值对应的效果。

字符	值

viminfo-!
! 如果包含，保存和恢复大写字母开头，并且不包含小写字母的全局变量。这样，保存 "KEEPTHIS" 和 "K_L_M"，但不保存 "KeepThis" 和 "_K_L_M"。不能正确读回嵌套的列表和字典值，它们变成空值。

viminfo-quote
" 每个寄存器最大保存的行数。'<' 项目的旧名，缺点是你需要在 " 之前加上反斜杠，不然它被识别为注释的开始!

viminfo-%
% 如果包含，保存和恢复缓冲区列表。如果 Vim 启动时指定文件名参数，缓冲区列表不予恢复。如果 Vim 启动时没有指定文件名参数，缓冲区列表从 viminfo 文件里恢复。没有文件名的缓冲区和帮助文件的缓冲区不会写入 viminfo 文件。不保存 quickfix ('buftype')、未列出 ('buflisted')、无名和在可移动媒介上 (viminfo-r) 的缓冲区。
如果后跟数值，该数值指定保存的缓冲区的最大个数。如果没有此数，

保存所有的缓冲区。

viminfo-'

'　　编辑过的文件的最大数目，为它们记住位置标记。如果 'viminfo' 非空，必须包含本参数。
包含本项目意味着 jumplist 和 changelist 也保存在 viminfo 文件里。

viminfo-/

/　　保存的搜索模式历史的最大项目数目。如果非零，那么也保存前次搜索和替代模式。如果不包含，使用 'history' 的值。

viminfo-:

:　　保存的命令行历史的最大项目数目。如果不包含，使用 'history' 的值。

viminfo-<

<　　每个寄存器最大保存的行数。如果为零，不保存寄存器。如果不包含，所有的行都被保存。'"' 是本项目的旧名。
另见下面的 's' 项目：千字节指定的限制。

viminfo-@

@　　保存的输入行历史的最大项目数目。如果不包含，使用 'history' 的值。

viminfo-c

c　　如果包含，把 viminfo 里的文本从写入时使用的 'encoding' 转换为当前的 'encoding'。见 viminfo-encoding 。

viminfo-f

f　　是否保存文件位置标记。如果为零，不保存文件位置标记 ('0 到 '9, 'A 到 'Z)。如果不存在或者非零，它们都被保存。'0 记住光标的当前位置 (退出或者执行 ":wviminfo" 时)。

viminfo-h

h　　载入 viminfo 文件时，关闭 'hlsearch' 的效果。如果不包含，取决于在最近的搜索命令之后是否使用过 ":nohlsearch"。

viminfo-n

n　　viminfo 文件的名字。该名字必须立即跟随在 'n' 之后，而且这必须是选项最后一个！如果设置了 'viminfofile' 选项，那个文件名覆盖 'viminfo' 在这里给出的。环境变量在文件打开时被扩展，而不是设置选项时。

viminfo-r

r　　可移动的媒介。参数是字符串 (直到下一个 ',' 为止)，本参数可以多次给出。每个参数指定路径的开始部分，符合者不保存位置标记。这被用于避免可移动的媒介。MS-DOS 上你可以用 "ra:,rb:"，Amiga 则可用 "rdf0:,rdf1:,rdf2:"。你也可用于指定临时文件的路径，比如 Unix 上："r/tmp"。大小写忽略。每个 'r' 参数的最大长度是 50 个字符。

viminfo-s

s　　每个项目千字节计的最大长度。如果为零，不保存寄存器。目前，只适用于寄存器。缺省的 "s10" 会忽略包含超过 10 千字节文本的寄存器。
另见上面的 '<' 项目：行数限制。

例如：

```
:set viminfo='50,<1000,s100,:0,n~/vim/viminfo
```

'50　　记住最近 50 个你编辑的文件的位置标记。
<1000　　记住寄存器的内容 (每个不超过 1000 行)。
s100　　跳过超过 100 千字节文本的寄存器。
:0　　不保存命令行历史。

n~/vim/viminfo　使用的文件名是 "~/vim/viminfo"。
no /　因为没有指定 '/'，使用缺省。也就是，保存所有的搜索历史和前次搜索和替代模式。
no %　不保存也不读入缓冲区列表。
no h　恢复 'hlsearch' 高亮。

'viminfo' 从空值改设为非空值以后，你可以用 :rviminfo 读入文件的内容，但这不是自动进行的。

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。
注意：如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'viminfofile' 'vif'
'viminfofile' 'vif'　字符串 (缺省: "")
全局
{仅当编译时加入 +viminfo 特性才有效}
如果非空，覆盖 viminfo 使用的文件名。
如果为 "NONE"，不读取或写入 viminfo 文件。
可用 -i 命令行标志位设置此选项。 --clean 命令行标志位设置它为 "NONE"。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'virtualedit' 've'
'virtualedit' 've'　字符串 (缺省为 "")
全局
{仅当编译时加入 +virtualedit 特性才有效}
逗号分隔的下面单词的列表:
block　允许可视列块模式的虚拟编辑。
insert　允许插入模式的虚拟编辑。
all　允许所有模式的虚拟编辑。
onemore　允许光标移动到刚刚超过行尾的位置

虚拟编辑意味着光标可以定位在没有实际字符的地方。这可以是制表的中间，也可以是行尾之后的位置。可用于在可视模式下选择一个方块，还有表格的编辑。"onemore" 不同，它只允许光标移动到刚刚超过行尾字符之后的位置。这使得有些命令更加一致。以前，在空行上光标总是刚刚超过行尾。但这和 Vi 远远不兼容，而且也有可能使一些插件或 Vim 脚本不能工作，比如因为 l 可以移动光标到行尾字符之后。小心使用！
$ 命令总是移动到行尾字符上，而不是超过它。这使光标可能实际向左移动！
g$ 命令则移动到屏幕行尾。
组合使用 "all" 和 "onemore" 没有意义，但你不会因此得到警告。
注意：如果置位 'compatible'，本选项被设为 ""。

'visualbell' 'vb' 'novisualbell' 'novb' beep
'visualbell' 'vb'　布尔型 (缺省关闭)
全局
使用可视响铃代替鸣叫。显示可视响铃的终端代码由 't_vb' 给出。如果既不想要响铃也不想要闪烁，使用:
:set vb t_vb=
如果想要短闪烁，在很多终端上可用:
:set vb t_vb=[?5h$<100>[?5l
这里 $<100> 指定时间，较小或较大的值会得到更短或更长的闪烁。

注意：Vim 会限制响铃不超过半秒一次。这避免在重复键击等造成的多次响铃会

导致长时间的等待。即使不置位 'visualbell' 也是如此。

在 GUI 里，'t_vb' 缺省为 "<Esc>|f"，反转显示 20 毫秒。如果你想使用别的时间，可设 "<Esc>|40f"，其中 40 是毫秒计的时间。

注意：GUI 启动时，'t_vb' 复位为缺省值。你可能想在 gvimrc 里再次设置之。

在 Amiga 上没有用，你总会得到屏幕闪烁。
另见 'errorbells'。

'warn' 'nowarn'
'warn' 布尔型 (缺省打开)
全局
在已修改的缓冲区上使用外壳命令会给出警告信息。

'weirdinvert' 'wiv' 'noweirdinvert' 'nowiv'
'weirdinvert' 'wiv' 布尔型 (缺省关闭)
全局
本选项和 't_xs' 终端选项的效果相同。
为了和 4.x 版本后向兼容而提供本选项。
置位 'weirdinvert' 的效果是使得 't_xs' 非空，反之亦然。如果运行 GUI，就没有任何效果。

'whichwrap' 'ww'
'whichwrap' 'ww' 字符串 (Vim 缺省: "b,s"，Vi 缺省: "")
全局
使指定的左右移动光标的键在行首或行尾可以移到前一行或者后一行。连接这些字符，可以让所有指定的键都有此功能：

字符	键	模式
b	<BS>	普通和可视
s	<Space>	普通和可视
h	"h"	普通和可视 (不建议)
l	"l"	普通和可视 (不建议)
<	<Left>	普通和可视
>	<Right>	普通和可视
~	"~"	普通
[	<Left>	插入和替换
]	<Right>	插入和替换

例如：

```
:set ww=<,>,[,]
```

只允许光标键进行回绕。
如果移动键和删除和改变操作符一起使用时，<EOL> 也被看作一个字符。这使得光标跨过行尾时，"3h" 和 "3dh" 效果不同。这也适用于 "x" 和 "X"，因为它们分别和 "dl" 以及 "dh" 相同。如果这么用，你可能想用映射 ":map <BS> X" 来使退格键删除光标之前的字符。
如果包含 'l'，位于 (非空) 行尾时如果在操作符之后使用它，不会移动到下一行。这样 "dl"、"cl"、"yl" 等都能正常工作。
注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'wildchar' 'wc'
'wildchar' 'wc' 数值型 (Vim 缺省: <Tab>，Vi 缺省: CTRL-E)

全局
在命令行上，用于启动通配符扩展的输入字符。其方式由 'wildmode' 指定。
详情可见： cmdline-completion 。
不在宏里识别该字符。为此，见 'wildcharm'。
尽管 'wc' 是一个数值选项，你可以设置它为一个特殊键：

```
:set wc=<Esc>
```

注意：如果置位 'compatible'，本选项被设为 Vi 的缺省值。相反，如果复位 'compatible'，本选项被设为 Vim 的缺省值。

'wildcharm' 'wcm'
'wildcharm' 'wcm' 数值型 (缺省：无 (0))
全局
'wildcharm' 和 'wildchar' 完全类似，除了在宏内识别该字符以外。
ex-edit-index 里可以找到命令行上 "空余" 的键用于此选项。通常你不会直接输入 'wildcharm'，而只是在映射里用它来自动启动补全模式，例如：

```
:set wcm=<C-Z>
:cnoremap ss so $vim/sessions/*.vim<C-Z>
```

这样，输入 :ss 以后，你可以使用 CTRL-P 和 CTRL-N。

'wildignore' 'wig'
'wildignore' 'wig' 字符串 (缺省为 "")
全局
{仅当编译时加入 +wildignore 特性才有效}
文件模式的列表。扩展 wildcards 和文件和目录名的补全中忽略符合其中任何模式的文件，它也同时影响 expand() 、 glob() 和 globpath() ，除非指定标志位来关闭之。
这些模式的应用方式类似于 :autocmd ，见 autocmd-patterns 。
另见 'suffixes'。
例如：

```
:set wildignore=*.o,*.obj
```

建议使用 :set+= 和 :set-= 来从列表里加减模式。这可以避免未来版本使用其它缺省值出现的问题。

'wildignorecase' 'wic' 'nowildignorecase' 'nowic'
'wildignorecase' 'wic' 布尔型 (缺省关闭)
全局
置位时对文件名和目录的补全忽略大小写。'fileignorecase' 置位时没有作用。
不适用于用外壳扩展通配符的情况，当有特殊字符时这会发生。

'wildmenu' 'wmnu' 'nowildmenu' 'nowmnu'
'wildmenu' 'wmnu' 布尔型 (缺省关闭， defaults.vim 里置位)
全局
{仅当编译时加入 +wildmenu 特性才有效}
'wildmenu' 打开时，命令行补全以增强模式运行。按下 'wildchar' (通常是 <Tab>) 启动补全。这时，在命令行的上方显示可能的匹配，然后高亮首个匹配 (如果有状态行，覆盖之)。显示前项/后项匹配的热键，如 <Tab> 或 CTRL-P/CTRL-N，移动高亮到合适的匹配上。
使用 'wildmode' 时，指定 "full" 会启用 "wildmenu" 模式。"longest" 和 "list" 则不会启动 "wildmenu" 模式。
可用 wildmenumode() 查看当前模式。
如果多个匹配超过一行，在右侧显示 ">" 和/或在左侧显示 "<"。需要的话，滚动状态行。

按下不用于选择补全项的键时，放弃 "wildmenu" 模式。
"wildmenu" 激活时，下面的键有特殊含义：

<Left> <Right> - 选择前项/后项匹配（类似于 CTRL-P/CTRL-N）
<Down> - 文件名/菜单名补全中：移进子目录和子菜单。
<CR> - 菜单补全中，如果光标在句号之后：移进子菜单。
<Up> - 文件名/菜单名补全中：上移到父目录或父菜单。

这使得控制台上有菜单可用 console-menus 。

如果你喜欢 <Left> 和 <Right> 键移动光标而不是选择不同的匹配，用：

```
:cnoremap <Left> <Space><BS><Left>
:cnoremap <Right> <Space><BS><Right>
```

"WildMenu" 高亮用于显示当前匹配 hl-WildMenu 。

'wildmode' 'wim'

'wildmode' 'wim' 字符串 (Vim 缺省: "full")
全局
用 'widechar' 指定的字符所用的补全模式。它是逗号分隔的不超过四部分的列表。每个部分指定每次连续使用的 'widechar' 要完成的动作。第一部分指定第一次使用时的行为。第二部分指定第二次使用时的，等等。
每个部分的可能值如下：

""	只用第一个匹配补全。
"full"	用下一个完整的匹配补全。在最后一次匹配之后，使用原来的字符串，然后又是第一个匹配。
"longest"	用最长的公共子串补全。如果结果没有变长，使用下一个部分。
"longest:full"	类似于 "longest"，但如果 'wildmenu' 打开的话，使用之。
"list"	如果有多于一个补全，列出所有的匹配。
"list:full"	如果有多于一个补全，列出所有的匹配并先用第一个补全。
"list:longest"	如果有多于一个补全，列出所有的匹配并先用最长的子串补全。
"list:lastused"	如果有多于一个缓冲区匹配，给缓冲区按最近使用的时间排序(除当前缓冲区外)。

如果只有一个匹配，所有情况下都完整补全。

例如：

```
:set wildmode=full
```

用第一个完整的匹配补全，然后下一个匹配，依此类推（缺省）

```
:set wildmode=longest,full
```

用最长的子串补全，然后是每个完整的匹配

```
:set wildmode=list:full
```

列出所有的匹配并用每个完整的匹配补全

```
:set wildmode=list,full
```

列出所有的匹配而不补全，然后是每个完整的匹配

```
:set wildmode=longest,list
```

用最长的子串补全，然后列出所有的可能性。
详情可见： cmdline-completion 。

'wildoptions' 'wop'

'wildoptions' 'wop' 字符串 (缺省为 "")
全局
{仅当编译时加入 +wildignore 特性才有效}

单词列表，改变命令行补全的行为。
目前只有一个可以使用的单词：

tagfile	如果用 CTRL-D 列出匹配的标签，列出标签的类型和文件。每行只显示一个匹配。常用的标签类型是：
	d #define
	f 函数

另见 cmdline-completion 。

'winaltkeys' 'wak' 'winaltkeys' 'wak' 字符串 (缺省为 "menu")
全局
{仅适用于 Win32、Motif、GTK 和 Photon GUI}
有的 GUI 版本允许用 ALT 键和菜单中下划线标出的字符的组合来访问菜单项。这和使用 ALT 键进行映射和输入特殊字符冲突。本选项指定此时如何做：

no	不用 ALT 键访问菜单。ALT 键组合可以被映射，但不会自动处理。之后可以用 :simalt 命令来完成。
yes	ALT 键的处理由窗口系统完成。ALT 键的组合不能被映射。
menu	使用 ALT 和菜单快捷键字符的组合会被窗口系统处理。其余的键仍然可以被映射。

如果用 'guioptions' 里排除 'm' 的方式来屏蔽菜单，ALT 键决不会被菜单所使用。
此选项不用于 <F10>；在 Win32 和 GTK 上，<F10> 会选择菜单，除非它被映射。

'wincolor' 'wcr' 'wincolor' 'wcr' 字符串 (缺省为空)
局部于窗口
用于此窗口取代 Normal 颜色 hl-Normal 的高亮组名。

'window' 'wi' 'window' 'wi' 数值型 (缺省为屏幕高度 - 1)
全局
窗口高度。不要和 Vim 窗口的高度混淆。那里要用 'lines'。
CTRL-F 和 CTRL-B 执行时，如果只有一个窗口且本选项的值小于 'lines' 减 1，用到本选项。此时，屏幕滚动的行数为 'window' 减两行，但至少一行。
如果 'window' 等于 'lines' 减 1，CTRL-F 和 CTRL-B 的滚动更智能一些，它们会照顾到回绕行。
改变 Vim 窗口大小时，本值如果小于 1 或者大于等于 'lines'，它被设为 'lines' 减 1。

'winheight' 'wh' E591
'winheight' 'wh' 数值型 (缺省为 1)
全局
当前窗口的最少行数。这不是绝对的最小值，如果没有足够的空间，Vim 仍会使用更少的行。
焦点移到比本选项小的窗口时，增大其高度，但以减少其它窗口的高度为代价。
普通的编辑应设置 'winheight' 为一个较小的值。
设之为 999 使得当前窗口填充几乎整个屏幕 ，其它窗口只有 'winminheight' 高。不过这有缺点，":all" 只会建立两个窗口。要避免 "vim -o 1 2 3 4" 只建立两个窗口的情况，用 VimEnter 事件在启动后再设置选项：

```
au VimEnter * set winheight=999
```

最小的值为 1。
改变当前窗口的高度的命令不会调整此高度。
'winheight' 应用于当前窗口。'winminheight' 设置其它窗口的最小高度。

'winfixheight' 'wfh' 'nowinfixheight' 'nowfh'
'winfixheight' 'wfh' 布尔型 (缺省关闭)
局部于窗口
窗口打开或关闭并且置位 'equalalways' 时，保持窗口的高度。另见 CTRL-W_= 。 preview-window 和 quickfix-window 中缺省打开。
不过，如果空间不够，高度仍然会改变。

'winfixwidth' 'wfw' 'nowinfixwidth' 'nowfw'
'winfixwidth' 'wfw' 布尔型 (缺省关闭)
局部于窗口
窗口打开或关闭并且置位 'equalalways' 的时候，保持窗口的宽度。另见 CTRL-W_= 。
不过，如果空间不够，宽度仍然会改变。

'winminheight' 'wmh'
'winminheight' 'wmh' 数值型 (缺省为 1)
全局
非当前窗口的最小高度。
这是一个绝对的最小值，窗口决不会比这更小。
如果设为零，在需要的话，窗口可能被 "挤压" 为零行 (也就是，只有状态行)。激活它们时，至少给出一行 (因为光标总要在什么地方显示的)。
使用 'winheight' 设置当前窗口的最小高度。
本选项只有在窗口会变的更小时才会检查。不要使用很大的值。不然，如果打开窗口变多，会报错。设为 0 到 3 的值是合理的。

'winminwidth' 'wmw'
'winminwidth' 'wmw' 数值型 (缺省为 1)
全局
非当前窗口的最小宽度。
这是一个绝对的最小值，窗口决不会比这更小。
如果设为零，在需要的话，窗口可能被 "挤压" 为零列 (也就是，只有垂直分割符)。激活它们时，至少给出一列 (因为光标总要在什么地方显示的)。
使用 'winwidth' 设置当前窗口的最小宽度。
本选项只有在窗口会变的更小时才会检查。不要使用很大的值。不然，如果打开窗口变多，会报错。设为 0 到 12 的值是合理的。

'winptydll'
'winptydll' 字符串 (缺省 "winpty32.dll" 或 "winpty64.dll")
全局
{仅当编译时加入 +terminal 特性才有效}
指定用于 :terminal 命令的 winpty 共享库名。缺省取决于编译目标的可执行程序是 32 位还是 64 位。如果没找到，作为后备会尝试 "winpty.dll"。
环境变量被扩展 :set_env 。
为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

'winwidth' 'wiw' E592
'winwidth' 'wiw' 数值型 (缺省为 20)
全局
当前窗口的最少列数。这不是绝对的最小值，如果没有足够的空间，Vim 仍会使用更少的列。如果当前窗口比本选项小，增大之，但以减少其它窗口的宽度为代价。设之为 999 使得当前窗口总是填充整个屏幕。普通的编辑应设置它为一个较小的值。
改变当前窗口的宽度的命令不会调整此宽度。

'winwidth' 应用于当前窗口。'winminwidth' 设置其它窗口的最小宽度。

'wrap' 'nowrap'

'wrap' 布尔型 (缺省打开)
局部于窗口

本选项改变文本显示的方式。它不改变缓冲区里的文本，'textwidth' 会。
如果打开，超过窗口宽度的行会回绕，并在下一行继续显示。如果关闭，行不会回绕。长行只有一部分会显示。如果光标移动到不显示的部分，屏幕会水平滚动。
如果需要的话，行会在单词的中间断开。'linebreak' 说明如何在单词边界上断行。
要使得水平滚动稍微更有用些，试设：

```
:set sidescroll=5
:set listchars+=precedes:<,extends:>
```

见 'sidescroll'、'listchars' 和 wrap-off 。
如果 'diff' 选项置位，不能从 modeline 中设置。

'wrapmargin' 'wm'

'wrapmargin' 'wm' 数值型 (缺省为 0)
局部于缓冲区

使 (硬) 回绕开始的从右窗口边界起算的字符数。如果输入的文本超过此限制，<EOL> 会被插入，而插入从下一行继续。
增加边距的选项，比如 'number' 和 'foldcolumn'，会使得文本宽度被进一步限缩。这是 Vi 兼容行为。
如果 'textwidth' 非零，不使用本选项。
'paste' 选项置位时本选项被设为 0，复位时恢复本选项。
另见 'formatoptions' 和 ins-textwidth 。

'wrapscan' 'ws' 'nowrapscan' 'nows'

'wrapscan' 'ws' 布尔型 (缺省打开) E384 E385
全局

搜索在文件末回绕回文件首。也用于]s 和 [s ，搜索拼写错误。

'write' 'nowrite'

'write' 布尔型 (缺省打开)
全局

允许写入文件。如果不置位，不允许写入文件。可以用来设置只读但允许在里面修改文本的模式。可以用命令行参数 -m 或 -M 复位。仍然可以过滤文本，即使这需要写入临时文件。

'writeany' 'wa' 'nowriteany' 'nowa'

'writeany' 'wa' 布尔型 (缺省关闭)
全局

允许写入文件，而无需 "!" 覆盖缺省行为。

'writebackup' 'wb' 'nowritebackup' 'nowb'

'writebackup' 'wb' 布尔型 (有 +writebackup 特性时缺省打开，否则缺省关闭)
全局

覆盖文件前建立备份。文件成功写入后，除非 'backup' 选项也被打开，删除该备份。
警告：关闭此选项意味着如果不能正确保存缓冲区到文件，则 Vim 退出 (不管何种原因) 时，你同时丢失了原来的文件和最新的改动。只有在你的文件系统几乎已满以致无法写入文件时，才复位此选项 (确保不要退出 Vim，直到写入操作

能成功为止)。
backup-table 还有另外的解释。
如果 'backupskip' 模式匹配，无论如何都不会建立备份。
取决于 'backupcopy'，备份或是一个新文件，或是换名后的原始文件 (同时新文件被写入)。
注意：如果置位 'compatible'，本选项被设为缺省值。

'writedelay' 'wd'

'writedelay' 'wd' 数值型 (缺省为 0)
全局
等待每个字符发送到屏幕的毫秒数。如果非零，字符被一个一个地发送到终端。不适用于 MS-DOS pcterm。此选项为调试所设。

vim:tw=78:ts=8:noet:ft=help:norl:

pattern.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：lang2 http://vimcdoc.sf.net

模式及查找命令 pattern-searches

最基本的东西可以在用户手册的 03.9 节中找到。在 usr_27.txt 也有一些解释。

1. 查找命令 search-commands
2. 匹配模式的定义 search-pattern
3. 魔术 /magic
4. 模式项总览 pattern-overview
5. 倍数项 pattern-multi-items
6. 普通匹配原 pattern-atoms
7. 忽略大小写 /ignorecase
8. 组合字符 patterns-composing
9. 与 Perl 匹配模式的比较 perl-patterns
10. 高亮显示匹配 match-highlight

---

1. 查找命令 search-commands

/

/{pattern}[/]<CR> 正向查找第 [count] 次出现 {pattern} 的地方 (exclusive 开动作)。

/{pattern}/{offset}<CR> 正向查找第 [count] 次出现 {pattern} 的地方并向上或下移动 {offset} 行。 linewise 行动作。

/<CR>

/<CR> 正向查找第 [count] 次出现处，使用上次的模式 last-pattern 以及上次的 {offset} 。

//{offset}<CR> 正向查找第 [count] 次出现处，使用上次的模式 last-pattern 以及新偏移 {offset} 。若 {offset} 为空，不使用任何偏移。

?

?{pattern}[?]<CR> 反向查找第 [count] 次出现 {pattern} 的地方 (exclusive 开动作)

?{pattern}?{offset}<CR> 反向查找第 [count] 次出现 {pattern} 的地方并向上或下移动 {offset} 行 linewise 行动作。

?<CR>

?<CR> 反向查找第 [count] 次出现处，使用上次的模式 last-pattern 以及上次的 {offset} 。

??{offset}<CR> 反向查找第 [count] 次出现处，使用上次的模式 last-pattern 以及新偏移 {offset} 。若 {offset} 为空，不使用任何偏移。

n

n

将上次的 "/" 或 "?" 重复 [count] 次。
如果光标不移动，搜索重复 count + 1 次。
last-pattern

N

N

将上次的 "/" 或 "?" 在相反方向上重复 [count] 次。
last-pattern

*

star E348 E349

正向查找第 [count] 次出现距离当前光标最近的单词的地方。用于查找的单词是以下最先符合条件的:

1. 光标所在的关键字 'iskeyword'
2. 当前行内的光标后的第一个关键字。
3. 光标所在的非空白单词
4. 当前行内的光标后的第一个非空白单词

只查找整字，就像使用了命令 "/\<keyword\>" 一样。
(exclusive 开动作)
使用 'ignorecase'，不使用 'smartcase'。

#

#

同 "*"，但反向查找。也可以用磅符号 (字符 163)。如果 "#" 的功能和退格键一样，试着在启动 Vim 前使用 "stty erase <BS>" (<BS> 是 CTRL-H 或真的退格键)。

g*

gstar

同 "*"，但不使用 "\<" 和 "\>"。这样查找就会匹配非整字的地方。

g#

g#

同 "#"，但不使用 "\<" 和 "\>"。这样查找就会匹配非整字的地方。

gd

gd

跳转 (goto) 至局部声明 (declaration)。当光标下是一个局部变量时，此命令会跳转到该变量被声明的地方。
首先，Vim 会查找当前函数的起始处，就像使用 "[[" 一样。如果找不到会停止在第一行。如果找到， Vim 会反向搜索直到找到一个空行，然后从这里开始正向查找光标所在的关键字，就像使用了 "*" 一样，但看起来像注释的行会被忽略。(见 'comments' 选项)。
注意 这并非总有效。Vim 不会做语法分析。它仅仅查找关键字的匹配。如果头文件也需要搜索的话，可以使用 include-search 列出的命令。
这个命令之后，可以使用 n 正向查找下一个匹配 (不能反向)。

gD

gD

跳转 (goto) 至全局声明 (Declaration)。当光标下是一个全局变量时，此命令会跳转到该变量被声明的地方。此命令和 "gd" 命令一样，不过查找总是从第一行开始。

1gd

1gd	同 "gd"，但忽略光标位置前结束的 {} 块里的匹配。
1gD	同 "gD"，但忽略光标位置前结束的 {} 块里的匹配。
CTRL-C	中止当前的（搜索）命令。在 MS-DOS 上用 CTRL-Break dos-CTRL-Break 。 在普通模式下，任何等待的命令将被终止。
:noh[lsearch]	停止 'hlsearch' 选项的高亮显示。如果再执行查找命令或者设定 'hlsearch' 选项，高亮会自动打开。此命令无法在自动命令里使用，因为高亮的状态会在自动命令执行期间自动保存和恢复 autocmd-searchpat 。 执行用户函数时也是如此。

如果打开 'incsearch' 选项，键入查找模式时，即时显示当前的匹配。不过，你还得用 <CR> 来结束查找命令并将光标定位到显示的匹配。也可以用 <ESC> 来放弃查找。

如果打开 'hlsearch'，最后一次查找命令的所有匹配点都会被高亮。用 :nohlsearch 命令来暂停。

如果 'shortmess' 不包含 "S" 标志位，Vim 会自动显示光标所在的索引。看起来像这样：

[1/5]	光标在 5 个匹配的第一个。
[1/>99]	光标在多于 99 个匹配的第一个。
[>99/>99]	光标在多于 99 个匹配的第 99 个之后。
[?/??]	存在的匹配数未知，因为搜索超时，统计的生成被中断。

注意：计数不考虑偏移。

如果没找到匹配，会报错：E486 Pattern not found
注意 为 Vi 兼容起见，:global 命令会把此错误当作一个普通的消息。:s 命令的 "e" 标志位可用来避免此错误 :s_flags 。

search-offset {offset}
这些命令查找一个指定的模式。而 "/" 和 "?" 命令还可以指定一个额外的偏移量。有两种偏移：行偏移和字符偏移。

偏移用来指定光标相对于找到的匹配的位置：

[num]	[num] 行向下，第一列
+[num]	[num] 行向下，第一列
-[num]	[num] 行向上，第一列
e[+num]	[num] 字符向右（从匹配结束（end）处算起）
e[-num]	[num] 字符向左（从匹配结束（end）处算起）
s[+num]	[num] 字符向右（从匹配开始（start）处算起）
s[-num]	[num] 字符向左（从匹配开始（start）处算起）
b[+num]	[num] 等价于 s[+num]（从匹配开始（begin）处算起）
b[-num]	[num] 等价于 s[-num]（从匹配开始（begin）处算起）
;{pattern}	开始另一项搜索，见 //;

如果给出了 '-' 或 '+' 但是省略了 [num]，会使用缺省的 1。

如果使用了 'e' 偏移，查找会成为 inclusive 闭动作（光标要到达的字符本身会包含在操作里）。

示例：

模式	光标位置
/test/+1	"test" 下一行，第一列
/test/e	"test" 的后一个 't'
/test/s+2	"test" 中的 's'
/test/b-3	"test" 前的第三个字符

如果把这些命令之一和一个操作符连用，影响的是查找之前和之后光标位置之间的字符。然而，如果使用了行偏移，影响的是两光标位置之间的所有整行。

一个演示如何查找一个模式并用另一个单词来更改匹配的例子：

```
/foo<CR>        查找 "foo"
c//e<CR>        修改从当前位置到匹配结尾的部分
bar<Esc>        输入替换文本
//<CR>          跳转到下一个相同匹配的开始
c//e<CR>        修改从当前位置到匹配结尾的部分
beep<Esc>       输入替换文本
                等等。
```

//; E386

一个很特别的偏移是 ';' 后面接另一个查找命令。例如：

```
/test 1/;/test
/test.*/+1;?ing?
```

其中第一例会先找到下一个出现 "test 1" 的地方，然后寻找在它之后 "test" 的第一次出现之处。

这就好像接连执行两次查找命令，不同之处在于：
- 它可以被当作操作符之后的单个移动命令。
- 以后的 "n" 或 "N" 命令基于第一个查找操作。
- 当有错误时光标原地不动。

last-pattern

最近使用的模式和偏移会被记住。它们可以用来重复查找，而且查找的方向和计数都可以修改。注意 Vim 会记住两个模式：一个是普通模式下查找命令用的，另一个是替代命令 ":s" 用的。每次给出一个空白的模式时，都将用到前次使用模式。不过，如果没有前次搜索模式，如果可以，使用前次替代模式。

'magic' 选项的值是和最近使用模式本身绑定的。如果你改变了 'magic'，最近使用模式的解释方法不会改变。'ignorecase' 选项就不同。当 'ignorecase' 的值改变时，该模式会匹配另外的文本。

如果你设定了 'hlsearch' 选项，所有最近使用的查找模式的匹配点都会高亮。

要清除最近使用的查找模式：

```
:let @/ = ""
```

这不会把该模式设置为空串，因为那样会匹配所有的东西。该模式是真的清除了，就像 Vim 刚刚启动一样。

查找通常会跳过那些不移动光标位置的匹配。下一次匹配是开始于下一个字符还是在跳过那些匹配之后取决于 'cpoptions' 选项中的 'c' 标志位。见 cpo-c 。

带 'c' 标志位: "/..." 前进 1 至 3 个字符
不带 'c' 标志位: "/..." 前进 1 个字符

带 'c' 标志位对应结果难以完全确定，因为查找可以从第一列开始，忽略任何匹配，直到找到超过光标位置的匹配为止。

如果反向寻找，要是如上所述 'cpoptions' 里包含了 'c' 标志位的话，从行首开始搜索，使用光标位置之前的最后一个匹配。

在 Vi 里 ":tag" 命令会将最近查找模式设定为要搜索的标签。除非 'cpoptions' 选项中包括 't' 标志位，Vim 不会这样做，前次查找模式仍然被记住。查找模式总被存入查找历史记录。

如果 'wrapscan' 选项的值为真（缺省）的话，查找会在缓冲区的结尾折返到首部。如果不为真，反向查找会在开始处停止；正向查找会在结尾处停止。如果该选项为真但找不到该模式，会给出一个错误信息: "pattern not found"，光标原地不动。如果不为真，取决于查找的方向是正向还是反向，给出的信息是: "search hit BOTTOM without match" 或者 "search hit TOP without match"。如果该选项为真，当查找折返时会分别显示: "search hit TOP, continuing at BOTTOM" 或 "search hit BOTTOM, continuing at TOP"。此时，消息可以用设定 'shortmess' 选项里的 's' 标志位来关闭。该信息的显示会使用 'w' 的高亮方法（缺省: 突出）。

search-range
用 \%>l 项目，你可以限定 "/" 命令查找的范围。例如，要在行 199 之下行 300 之上搜索单词 "limit":

```
/\%>199l\%<300llimit
```

另见 /\%>l 。

另一个方法是使用 ":substitute" 命令并与 'c' 联用。例:

```
:.,300s/Pattern//gc
```

这个命令会从光标所在处开始查找 "Pattern"，直到第 300 行。在匹配处，你会被提示键入一个字符。键入 'q' 终止；键入 'n' 找下一个匹配。

"*", "#", "g*" 和 "g#" 命令依照以下的次序查找在光标附近的一个单词，使用以下最先找到的那个:
- 光标所在的关键字。
- 同行的光标右侧的第一个关键字。
- 光标下的第一个字串（WORD）。
- 同行的光标右侧的第一个字串（WORD）。

关键字只能包括字母和 'iskeyword' 中的字符。字串（WORD）可以包括任何非空白（<Tab> 和／或 <Space>）。

备注 如果你用十个指头打字的话，这些字符是很容易记住的:
"#" 在你的左手中指上（向左上查找）; "*" 在你的右手中指上（向右下查找）。
（不过，这取决于你的键盘布局）。

E956
在极罕见的情况下会递归使用正规表达式。可能发生的情形是执行一个模式花很长的时间，同时在通道上检查消息时所调用的回调中也使用了模式或者激活了自动命令。多数情况下，这没问题，但如果一个模式在使用时，再一次被用会失败。通常这意味模式本身有些问题。

---

2. 模式的定义 search-pattern pattern [pattern]

regular-expression regexp Pattern
E76 E383 E476

头一道菜已经在用户手册的第 27 章 usr_27.txt 上过了。

/bar /\bar /pattern

1. 一个模式 (pattern) 是由 "\|" 分隔开的一个或多个分支 (branch)。它可以匹配其中的任何一个分支匹配的文本。例如: "foo\|beep" 可以匹配 "foo" 或 "beep"。如果超过一个分支可以匹配，选用其中的第一个。

```
pattern ::=     branch
            或  branch \| branch
            或  branch \| branch \| branch
            等。
```

/branch /\&

2. 一个分支是一个或多个被 "\&" 分隔的邻接。它匹配最后一个邻接，但仅仅当前面所的邻接也在同样位置匹配。例:
   "foobeep\&..." 匹配 "foobeep" 中的 "foo"。
   ".*Peter\&.*Bob" 匹配同时包括 "Peter" 和 "Bob" 的一行

```
branch ::=      concat
            或  concat \& concat
            或  concat \& concat \& concat
            等。
```

/concat

3. 一个邻接 (concat) 是一或多个相邻的组件 (piece)。它匹配第一个组件，后接第二个组件的匹配，等等。例如: "f[0-9]b"，先匹配 "f"，然后是一个数位，接着 "b"。

```
concat  ::=     piece
            或  piece piece
            或  piece piece piece
            等。
```

/piece

4. 一个组件是一个匹配原 (atom)，后面可能带一个倍数 (multi)，用来表示该匹配原匹配的次数。例如: "a*" 匹配任意个 "a" 组成的序列: ""，"a"，"aa" 等。参见 /multi 。

```
piece   ::=     atom
            或  atom  multi
```

/atom

5. 一个匹配原可以是一个很长的条目列表中的一个。许多匹配原用来匹配文本中的一个字符，通常是一个普通的字符或字符类。也可以用圆括号来将一个模式变成一个匹配原。下面的 "\z(\)" 构造仅用在语法高亮中。

```
atom    ::=     ordinary-atom           /ordinary-atom
            或  \( pattern \)           /\(
            或  \%( pattern \)          /\%(
            或  \z( pattern \)          /\z(
```

/\%#= two-engines NFA

Vim 包含两个正规表达式引擎：
1. 老式的基于回溯的引擎，支持所有功能。
2. 新式的 NFA 引擎，对某些模式会快很多，对某些模式可能会变慢。

Vim 会自动为你选择合适的引擎。不过，如果你遇过问题或者想特别指定某一个引擎，可以在模式开头处加入：

\%#=0	强制自动选择。仅在 'regexpengine' 设为非零时才有效果。
\%#=1	强制使用旧引擎。
\%#=2	强制使用 NFA 引擎。

也可用 'regexpengine' 选项改变缺省机制。

E864 E868 E874 E875 E876 E877 E878

选择 NFA 引擎时如果遇到未实现的模式部分时，整个模式会不匹配。只用于调试 Vim。

---

3. 魔术 /magic

某些字符在模式中是按本义出现的。它们匹配自身。然而，当前面有一个反斜杠时，这些字符具有特殊的含义。

另外一些字符即使没有反斜杠也代表特殊的意思。它们反而需要一个反斜杠来匹配按本义出现的自身。

一个字符是否按本义出现取决于 'magic' 选项以及下面将解释的条目。

/\m /\M

使用 "\m" 会使得其后的模式的解释方式就如同设定了 'magic' 选项一样。而且将忽略 'magic' 选项的实际值。
使用 "\M" 会使得其后的模式的解释方式就如同设定了 'nomagic' 选项一样。

/\v /\V

使用 "\v" 会使得在它之后，所有 '0'-'9'，'a'-'z'，'A'-'Z' 和 '_' 之外的字符都当作特殊字符解释："very magic"

使用 "\V" 会使得在它之后，只有反斜杠和终止字符（通常是 / 或 ?）有特殊的意义："very nomagic"

示例：

在这之后：	\v	\m 'magic'	\M 'nomagic'	\V	匹配
	$	$	$	\$	匹配行尾
	.	.	\.	\.	匹配任何字符
	*	*	*	*	前面匹配原的任意次重复
	~	~	\~	\~	最近替代字符串
	()	\(\)	\(\)	\(\)	组成为单个匹配原
	|	|	|	|	分隔可选分支
	\a	\a	\a	\a	字母字符
	\\	\\	\\	\\	反斜杠（按本义）
	\.	\.	.	.	英文句号（按本义）
	\{	{	{	{	'{'（按本义）
	a	a	a	a	'a'（按本义）

{仅 Vim 支持 \m，\M，\v 和 \V}

建议始终将 'magic' 选项保持在缺省值 — 'magic'。这可以避免移植性的麻烦。要使模式不受该选项值的影响，在模式前面加上 "\m" 或 "\M"。

---

4. 模式项总览

pattern-overview
E865 E866 E867 E869

倍数总览。 /multi E61 E62 E64 E871
更多解释和示例见下，点击链接即可。

	倍数 'magic'	倍数 'nomagic'	匹配前面的匹配原	
/star	*	*	0 或更多	尽可能多
/\+	\+	\+	1 或更多	尽可能多
/\=	\=	\=	0 或 1	尽可能多
/\?	\?	\?	0 或 1	尽可能多
/\{	\{n,m}	\{n,m}	n 到 m	尽可能多
	\{n}	\{n}	n	准确
	\{n,}	\{n,}	最少 n	尽可能多
	\{,m}	\{,m}	0 到 m	尽可能多
	\{}	\{}	0 或更多	尽可能多 (和 * 相同)
/\{-	\{-n,m}	\{-n,m}	n 到 m	尽可能少
	\{-n}	\{-n}	n	准确
	\{-n,}	\{-n,}	最少 n	尽可能少
	\{-,m}	\{-,m}	0 到 m	尽可能少
	\{-}	\{-}	0 或更多	尽可能少

E59

/\@>	\@>	\@>	1，要求完整的模式匹配 (不能回溯)
/\@=	\@=	\@=	无，但要求匹配 /zero-width
/\@!	\@!	\@!	无，但要求 不 匹配 /zero-width
/\@<=	\@<=	\@<=	无，但要求反向匹配 /zero-width
/\@<!	\@<!	\@<!	无，但要其反向 不 匹配 /zero-width

普通匹配原总览。 /ordinary-atom
更多解释和示例见下，点击链接即可。

	普通匹配原 magic	普通匹配原 nomagic	匹配
/^	^	^	行首 (在模式起始) /zero-width
/\^	\^	\^	按本义出现的 '^'
/_^	_^	_^	行首 (用于任何地方) /zero-width
/$	$	$	行尾 (在模式结尾) /zero-width
/\$	\$	\$	按本义出现的 '$'
/_$	_$	_$	行尾 (用于任何地方) /zero-width
/.	.	\.	任何单个字符 (不包括换行)
/_.	_.	_.	任何单个字符，包括换行
/\<	\<	\<	单词的起始 /zero-width
/\>	\>	\>	单词的结尾 /zero-width
/\zs	\zs	\zs	任何字符，设定匹配部分起始

/\ze	\ze	\ze	任何字符，设定匹配部分结束
/\%^	\%^	\%^	文件首 /zero-width E71
/\%$	\%$	\%$	文件尾 /zero-width
/\%V	\%V	\%V	可视区域内 /zero-width
/\%#	\%#	\%#	光标位置 /zero-width
/\%'m	\%'m	\%'m	位置标记 m 的位置 /zero-width
/\%l	\%23l	\%23l	在第 23 行 /zero-width
/\%c	\%23c	\%23c	在第 23 列 /zero-width
/\%v	\%23v	\%23v	在虚拟第 23 列 /zero-width

字符类: /character-classes

magic	nomagic	匹配
/\i	\i	\i	标识符字符（见 'isident' 选项）
/\I	\I	\I	同 "\i"，但不包括数字字符
/\k	\k	\k	关键字字符（见 'iskeyword' 选项）
/\K	\K	\K	同 "\k"，但不包括数字字符
/\f	\f	\f	文件名字符（见 'isfname' 选项）
/\F	\F	\F	同 "\f"，但不包括数字字符
/\p	\p	\p	可显示字符（见 'isprint' 选项）
/\P	\P	\P	同 "\p"，但不包括数字字符
/\s	\s	\s	空白字符；<Space> 和 <Tab>
/\S	\S	\S	非空白字符：\s 之反
/\d	\d	\d	数位：[0-9]
/\D	\D	\D	非数位：[^0-9]
/\x	\x	\x	十六进制数位：[0-9A-Fa-f]
/\X	\X	\X	非十六进制数位：[^0-9A-Fa-f]
/\o	\o	\o	八进制数位：[0-7]
/\O	\O	\O	非八进制数位：[^0-7]
/\w	\w	\w	单词字符：[0-9A-Za-z_]
/\W	\W	\W	非单词字符：[^0-9A-Za-z_]
/\h	\h	\h	单词首字符：[A-Za-z_]
/\H	\H	\H	非单词首字符：[^A-Za-z_]
/\a	\a	\a	英文字母字符：[A-Za-z]
/\A	\A	\A	非英文字母字符：[^A-Za-z]
/\l	\l	\l	小写字符：[a-z]
/\L	\L	\L	非小写字符：[^a-z]
/\u	\u	\u	大写字符：[A-Z]
/\U	\U	\U	非大写字符 [^A-Z]
/_	_x	_x	其中 x 可以是以上任意一个字符：对应字符类加上换行

(字符类结束)

magic	nomagic	匹配
/\e	\e	\e	<Esc>
/\t	\t	\t	<Tab>
/\r	\r	\r	<CR>
/\b	\b	\b	<BS>
/\n	\n	\n	行尾符
/~	~	\~	上次给出的替代字符串
/\1	\1	\1	第一个 \(\) 匹配的字符
/\2	\2	\2	如 "\1"，但用第二个 \(\)
	...		
/\9	\9	\9	如 "\1"，但用第九个 \(\)
			E68
/\z1	\z1	\z1	仅用于语法高亮，见 :syn-ext-match

```
          ...
/\z1    \z9     \z9     仅用于语法高亮，见 :syn-ext-match

        x       x       一个没有特殊含义的字符匹配其自身

/[]     []      \[]     [] 内指定的任何字符之一
/\%[]   \%[]    \%[]    一个可选匹配原的序列

/\c     \c      \c      忽略大小写，不使用 'ignorecase' 选项
/\C     \C      \C      匹配大小写，不使用 'ignorecase' 选项
/\Z     \Z      \Z      忽略 Unicode 中的 "组合字符" 的区别，对于搜索带元音的
                        希伯来和阿拉伯文本有用。

     magic   nomagic    匹配
/\m     \m      \m      对之后模式中字符打开 'magic' 选项
/\M     \M      \M      对之后模式中字符关闭 'magic' 选项
/\v     \v      \v      对之后模式中字符设定 'magic' 选项为 "very magic"
/\V     \V      \V      对之后模式中字符设定 'magic' 选项为 "very nomagic"
/\%#=    \%#=1   \%#=1   选择正规表达式引擎 /zero-width

/\%d    \%d     \%d     匹配用十进制指定的字符 (如 \%d123)
/\%x    \%x     \%x     匹配用十六进制指定的字符 (如 \%x2a)
/\%o    \%o     \%o     匹配用八进制指定的字符 (如 \%o040)
/\%u    \%u     \%u     匹配指定的多字节字符 (如 \%u20ac)
/\%U    \%U     \%U     匹配指定的巨大多字节字符 (如 \%U12345678)
/\%C    \%C     \%C     匹配任何组合字符

示例                    匹配
\<\I\i*         或
\<\h\w*
\<[a-zA-Z_][a-zA-Z0-9_]*
                        一个标识符 (例如，在一个 C 程序里)。

\(\.$\|\. \)            一个英文句号后跟 <EOL> 或一个空格。

[.!?][])"']*\($\|[ ]\)  一个匹配英文句子结尾的模式。几乎和 ")" 的定义相同。

cat\Z                   匹配 "cat" 和 "cat" ("a" 后跟 0x0300)
                        不匹配 "cà t" (字符 0x00e0)，即使它们看起来可能一样。
```

---

5. 倍数项 pattern-multi-items

一个匹配原后面可以跟一个表示该匹配原匹配次数和方式的修饰。这个修饰称为倍数。
在 /multi 可以看到一个综述。

/star /\star

```
*       (在没有设定 'magic' 时使用 \*)
        匹配 0 或更多个前面的匹配原，尽可能多地匹配。
        示例      'nomagic'     匹配
        a*          a\*         "" 、"a" 、"aa" 、"aaa" 等。
        .*          \.\*        任意，包括空串，不包括行尾符
        \_.*        \_.\*       匹配至缓冲区结束
```

_.*END　　_.*END　　匹配至缓冲区中最后一次出现 "END" 处，包括该 "END"

特例：当 "*" 用在模式的开头或者紧跟在 "^" 之后时，它匹配星号字符。

要当心，重复匹配 "_." 会包括大量的文本，因而可能会花很长时间。例如，"_.*END" 匹配从当前位置开始到文件中最后一次出现 "END" 的地方。因为 "*" 会尽可能多的匹配，它会先跳过到文件结束前的所有行，然后一次退一个字符以查找 "END"。

/\+

\+　匹配一个或更多前面的匹配原。尽可能多。

例	匹配
^.\+$	任意空行
\s\+	一个以上的空白字符

/\=

\=　匹配 0 或 1 个前面的匹配原。尽可能多。

例	匹配
foo\=	"fo" 和 "foo"

/\?

\?　和 \= 一样。不能和反向查找的 "?" 命令中使用。

/\{　E60　E554　E870

\{n,m}　匹配 n 至 m 个前面的匹配原。尽可能多
\{n}　匹配 n 个前面的匹配原
\{n,}　匹配至少 n 个前面的匹配原。尽可能多
\{,m}　匹配 0 至 m 个前面的匹配原。尽可能多
\{}　匹配 0 个以上前面的匹配原。尽可能多 (和 * 等价)

/\{-

\{-n,m}　匹配 n 至 m 个前面的匹配原。尽可能少
\{-n}　匹配 n 个前面的匹配原
\{-n,}　匹配至少 n 个前面的匹配原。尽可能少
\{-,m}　匹配 0 至 m 个前面的匹配原。尽可能少
\{-}　匹配 0 个以上前面的匹配原。尽可能少

n 和 m 是正的十进制数或零

non-greedy

如果一个 "-" 紧接在 "{" 之后，那么最短匹配优先算法将启用 (见下面的例子)。特别的，"\{-}" 和 "*" 一样，但使用最短匹配优先算法。 不过 ， 更早开始的匹配比更短的匹配优先: "a\{-}b" 匹配 "xaaab" 中的 "aaab"。

示例	匹配
ab\{2,3}c	"abbc" 或 "abbbc"
a\{5}	"aaaaa"
ab\{2,}c	"abbc"、"abbbc"、"abbbbc" 等
ab\{,3}c	"ac"、"abc"、"abbc" 或 "abbbc"
a[bc]\{3}d	"abbbd"、"abbcd"、"acbcd"、"acccd" 等。
a\(bc\)\{1,2}d	"abcd" 或 "abcbcd"
a[bc]\{-}[cd]	"abcd" 中的 "abc"
a[bc]*[cd]	"abcd" 中的 "abcd"

} 之前可以加 (也可以不加) 一个反斜杠: \{n,m\}。

/\@=

\@=　零宽度匹配前面的匹配原。
类似于 Perl 中的 "(?=pattern)"。

示例	匹配
foo\(bar\)\@=	"foobar" 中的 "foo"
foo\(bar\)\@=foo	空

/zero-width

"\@=" (或 "^", "$", "\<", "\>") 的匹配不包括任何字符。这些字符仅仅是用来检查匹配是否存在。这很容易搞错，因为后续的项会在同样位置作匹配。上面的最后一个例子不会匹配 "foobarfoo"，因为 Vim 会尝试在 "bar" 匹配的同样地方匹配 "foo"。

注意　使用 "\&" 同使用 "\@=" 是一样的: "foo\&.." 等于 "\(foo\)\@=.."。但用 "\&" 容易些，你可以省了那些括号。

/\@!

\@!　零宽度否匹配，亦即，要求前面的匹配原在当前位置 不 匹配。 /zero-width
类似于 Perl 中的 "(?!pattern)"。

示例	匹配
foo\(bar\)\@!	任意后面不带 "bar" 的 "foo"
a.\{-}p\@!	"a", "ap", "app", "appp" 等。后面不带 "p"
if \(\(then\)\@!.\)*$	"if " 后面无跟随 "then"

使用 "\@!" 要小心，因为很多地方一个模式可以不匹配。"a.*p\@!" 会匹配一个 "a" 直到行尾，因为 ".*" 匹配所有行内的字符，而 "p" 不在行尾匹配。"a.\{-}p\@!" 则可以匹配 "a", "ap", "app" 等中的一项，只要它们不再紧跟另外一个 "p"，这是因为 "." 可以匹配 "p" 而 "p\@!" 不匹配这个 "p" 的后面。

你不能用 "\@!" 去寻找匹配所在位置之前的否匹配。"\(foo\)\@!bar" 会匹配 "foobar" 中的 "bar"，因为在 "bar" 匹配的位置上，"foo" 并不匹配。如果不想匹配 "foobar"，你可以尝试 "\(foo\)\@!...bar"，但它不能匹配在行首出现的 "bar"。最好的方法是用 "\(foo\)\@<!bar"。

有用的例子: 要找到包含 "foo" 但没有 "bar" 的行:

```
/^\%(.*bar\)\@!.*\zsfoo
```

此模式先检查某行中没有一个位置可以匹配 "bar"。如果 ".*bar" 匹配，\@! 会拒绝本模式。否则，继续寻找 "foo"。"\zs" 使得匹配从 "foo" 处开始。

/\@<=

\@<=　零宽度反向匹配。要求前面的匹配原紧贴在后面的模式之前匹配。
/zero-width
类似于 Perl 的 "(?<=pattern)"，但是 Vim 允许使用非固定宽度的模式。

示例	匹配
\(an_s\+\)\@<=file	在 "an" 和若干个空白或者行尾符之后的 "file"。

为了提高性能，最好避免这个倍数项。可以用 "\zs" 替代 /\zs 。要匹配上述的示例:

```
an\_s\+\zsfile
```

反向匹配时至少要设一个限制范围，见下。

"\@<=" 和 "\@<!" 检查刚好在其后模式匹配点之前出现的匹配文本。理论上，这些匹配可以从当前位置之前的任何地方开始，为了限制所需的时间，只有其后

匹配所在的行和它上一行（如果有上一行的话）会被检索。这对大多数的应用来说已经足够，而且也不会减慢速度太多。

在旧的正规表达式引擎里，在 "\@<=" 和 "\@<!" 之后的模式部分其实先被匹配，所以不能用 "\1" 之类的结构来引用零宽度匹配原中的 \(\)。反过来倒是可以的。

坏示例	匹配
\%#=1\1\@<=,\([a-z]\+\)	"abc,abc" 里的 ",abc"

不过，新的正规表达式引擎工作方式不同，最好不要依赖此行为，如果可以避免，不要用 \@<=:

示例	匹配
\([a-z]\+\)\zs,\1	"abc,abc" 里的 ",abc"

\@123<=
类似于 "\@<="，但只往回看 123 个字节。这样可以避免去匹配很多已知不会成功的但使模式很慢的尝试。例如，要检查紧接 "span" 之前要否有 "<":

```
/<\@1<=span
```

这里只须对 "span" 之前一个字节试图匹配 "<"，本来这也是唯一可能的位置。如果跨过行边界，此限制相对于该行的行尾。这样，匹配所在行的行首部分不计入限制范围（只是为了简单起见）。
零相当于无限制。

/\@<!

\@<!　零宽度反向否匹配，要求前面的匹配原 不 紧贴在其后的模式之前匹配。换而言之，如果在当前行或之前的一行中没有任何位置这个匹配原可以匹配而且它的匹配文本刚好紧贴在其后模式的匹配点之前。 /zero-width
类似于 Perl 的 "(?<!pattern)"，但是 Vim 允许使用非固定宽度的模式。
该倍数所引用的匹配原必须紧贴在其后匹配之前，所以这个匹配原可以用 ".*" 结尾。警告：该倍数项可能会很慢（因为在匹配之前的许多位置都要被检查）。如有可能，请使用限制范围，见下。

示例	匹配
\(foo\)\@<!bar	任何不在 "foobar" 中的 "bar"
\(\/\/.*\)\@<!in	不在 "//" 之后（中间可以有间隔）的 "in"

\@123<!
类似于 "\@<!"，但只往回看 123 个字节。这样可以避免去匹配很多已知不会成功的但使模式很慢的尝试。

/\@>

\@>　匹配前面的匹配原时要求完整匹配（不能回溯）。
类似于 Perl 中的 "(?>pattern)"。

示例	匹配
\(a*\)\@>a	无（"a*" 用尽了所有的 "a"，后面不可能再有）

试图匹配前面的匹配原，就如同匹配一个完整的模式那样。如果以后无法匹配，不会用较短的子模式或者任何其它的方法来重试该匹配原。 注意 观察以下两者的不同： "a*b" 和 "a*ab" 都匹配 "aaab"，但是后者中的 "a*" 仅仅匹配前两个 "a"。"\(a*\)\@>ab" 不会匹配 "aaab"，因为 "a*" 匹配了 "aaa"（尽可能多的 "a"），导致 "ab" 无法匹配。

---

6. 普通匹配原　　pattern-atoms

一个普通匹配原可能是：

/^

^ 在模式起点或在 "\|"，"\("，"\%("，"\n" 之后：匹配行首；在其它位置匹配按本义出现的 '^' 字符。 /zero-width

示例 匹配

^beep(C 函数 "beep" 开始的地方（很可能）。

/\^

\^ 匹配按本义出现的 '^' 字符。可以用于任何位置。

/_^

_^ 匹配行首。 /zero-width 可以用于任何位置。

示例 匹配

_s*_^foo 空白字符及空行，后接行首的 "foo"。

/$

$ 在模式终点或 "\|"、"\)" 或 "\n"（打开 'magic' 的话）之前：匹配行尾 <EOL>；其它位置，匹配按本义出现的 '$' 字符。 /zero-width

/\$

\$ 匹配按本义出现的 '$' 字符。可以用于任何位置。

/_$

$ 匹配行尾。 /zero-width 可以用于任何位置。注意：'a$b" 永远无法匹配，因为 "b" 不能匹配行尾符。可以使用 "a\nb" 代替 /\n 。

示例 匹配

foo_$_s* "foo" 在行尾，后接空白字符及空行

/. /\.

. ('nomagic' 的情形：\.)
匹配任何单个字符，不包括行尾符。

/_.

. 匹配任何单个字符或行尾符。当心：".*" 会一直匹配到缓冲区结尾！

/\<

\< 匹配单词起点：下一个字符是单词的首字符。'iskeyword' 选项指定哪些是组成单词的字符。 /zero-width

/\>

\> 匹配单词终点：前一个字符是单词的尾字符。'iskeyword' 选项指定哪些是组成单词的字符。 /zero-width

/\zs

\zs 匹配任何位置，并将匹配起始处置于该处：下一个字符将是整个匹配的第一个字符。 /zero-width
例如：

```
"^\s*\zsif"
```

匹配行首的 "if"，忽略前导的空白字符。
可以使用多次，但实际使用最后一次在匹配的分支里遇到的。例如：

```
/\(.\{-}\zsFab\)\{3}
```

找到第三次出现的 "Fab"。
不能后跟倍数。 E888

{仅当编译时加入 +syntax 特性才可用}

/\ze

\ze 匹配任何位置，并将匹配结尾处置于该处：前一个字符将是整个匹配的最后一个字符。 /zero-width
可以使用多次，但实际使用最后一次在匹配的分支里遇到的。
例如："end\ze\(if\|for\)" 匹配 "endif" 和 "endfor" 中的 "end"。
不能后跟倍数。 E888
{仅当编译时加入 +syntax 特性才可用}

/\%^ start-of-file

\%^ 匹配文件头。当用于一个字符串时，匹配字符串起始处。
例如，要查找一个文件中首次出现 "VIM" 的地方：

```
/\%^\_.\{-}\zsVIM
```

/\%$ end-of-file

\%$ 匹配文件尾。当用于一个字符串时，匹配字符串结束处。
注意 下面的模式不匹配文件中的最后一个 "VIM"：

```
/VIM\_.\{-}\%$
```

它会找到下一个 VIM，因为其后的部分总会匹配。这样才能匹配文件中的最后一个 "VIM"：

```
/VIM\ze\(\(VIM\)\@!\_.\)*\%$
```

这里使用了 /\@! 来确定在首个 "VIM" 之后 无法 再次匹配 "VIM"。
从文件尾部反向搜索会容易些！

/\%V

\%V 在可视区域内匹配。可视模式停止后，在 gv 将会重新选择的区域内匹配。
这是一个零宽度 /zero-width 匹配。要确保整个模式都在可视区域里，把本匹配原放在匹配模式的开始和刚好结尾之前处，例如：

```
/\%Vfoo.*ba\%Vr
```

这在可视选择只包含 "foo bar" 时，才会匹配。而：

```
/\%Vfoo.*bar\%V
```

这在可视选择区在 "r" 之后继续时，才会匹配 "foo bar"。
只能用于当前缓冲区。

/\%# cursor-position

\%# 匹配光标位置。仅对显示于窗口内的缓冲区有效。
警 告：如果使用了该模式之后光标被移动，结果不再有效，Vim 不会自动更新匹配。这对语法高亮和 'hlsearch' 很重要。换言之：在光标移动时显示并不因此而更新。Vim 只对行被修改（整行被更新）或用户使用 CTRL-L 命令时（整个屏幕被更新）进行更新。例如，要高亮显示光标下的单词：

```
/\k*\%#\k*
```

当设定 'hlsearch' 时，移动光标并作若干更改，你就能清楚地看到更新在何时进行。

/\%'m /\%<'m /\%>'m

\%'m 匹配位置标记 m 的位置。
\%<'m 匹配位置标记 m 之前的位置。
\%>'m 匹配位置标记 m 之后的位置。
例如，高亮位置标记 's 到 'e 的文本：

```
/.\%>'s.*\%<'e..
```

注意 在匹配里需要两个句号来包含位置标记 'e。这是因为 "\%<'e" 匹配 'e 位置标记之前的字符，而既然它是零宽度匹配 /zero-width，它并不真正包含该字符。

警 告：如果使用模式后移动了位置标记，结果不再有效。Vim 并不自动更新匹配。这和 "\%#" 里移动光标的情形类似 /\%# 。

/\%l /\%>l /\%<l E951

\%23l 匹配指定的行。
\%<23l 匹配指定行的上方（行号更小）。
\%>23l 匹配指定行的下方（行号更大）。
这三个匹配原可以用来匹配缓冲区内的特定行。"23" 可以是任何行号。第一行为 1。
警 告：当插入和删除行时 Vim 不会自动更新匹配。这意味着语法高亮很快就会有问题。
例如，要高亮显示光标所在行：

```
:exe '/\%' . line(".") . 'l.*'
```

当设定 'hlsearch' 时，移动光标并作若干更改，你就能清楚地看到更新在何时进行。

/\%c /\%>c /\%<c

\%23c 匹配指定的列。
\%<23c 匹配指定列之前。
\%>23c 匹配指定列之后。
这三个匹配原可以用来匹配缓冲区或字符串中的特定列。"23" 可以是任何列号。第一列为 1。事实上，列以字节计算（因此对多字节字符来说是不准确的）。
警 告：当插入和删除字符时 Vim 并不自动更新匹配。这意味着语法高亮很快就会有问题。
例如，要高亮显示光标所在列：

```
:exe '/\%' . col(".") . 'c'
```

当设定 'hlsearch' 时，移动光标并作若干更改，你就能清楚地看到更新在何时进行。
一个匹配第 44 列单个字节的例子：

```
/\%>43c.\%<46c
```

注意：\"\%<46c" 匹配第 45 列，而 "." 匹配 44 列的一个字节。

/\%v /\%>v /\%<v

\%23v 匹配指定虚拟列。
\%<23v 匹配指定虚拟列之前。
\%>23v 匹配指定虚拟列之后。
这三个匹配原可以用来匹配缓冲区或字符串中指定的虚拟列。如果不是在匹配窗口内的缓冲区，使用当前窗口的选项值（如 'tabstop'）。
"23" 可以是任何列号。第一列为 1。
注意 某些虚拟列位置永远无法被匹配，因为它们在制表符或其它占用超过一个屏幕位置的特殊字符的中间某处。
警 告：当插入和删除字符时 Vim 并不自动更新高亮的匹配。这意味着语法高亮很快就会有问题。
例如，要高亮显示所有位于虚拟第 72 列之后的字符：

```
/\%>72v.*
```

当设定 'hlsearch' 时，移动光标并作若干更改，你就能清楚地看到更新在何时进行。
要匹配直到第 17 列的文本：

```
/^.*\%17v
```

这里不包括第 17 列，因为这是 /zero-width 匹配。要包含该列，应用：

```
/^.*\%17v.
```

下例同上，但如果第 17 列没有字符也同样匹配：

```
/^.*\%<18v.
```

注意 如果没有 "^" 来锚住首列的匹配，第 17 列也能被高亮：

```
/.*\%17v
```

'hlsearch' 高亮第 17 列，因为存在另外一个匹配，其中 ".*" 匹配零个字符。

字符类：

\i	标识符字符（见 'isident' 选项）	/\i
\I	同 "\i"，但不包括数字字符	/\I
\k	关键字字符（见 'iskeyword' 选项）	/\k
\K	同 "\k"，但不包括数字字符	/\K
\f	文件名字符（见 'isfname' 选项）	/\f
\F	同 "\f"，但不包括数字字符	/\F
\p	可显示字符（见 'isprint' 选项）	/\p
\P	同 "\p"，但不包括数字字符	/\P

备注：上面这些适用于多字节字符，而下面只匹配 ASCII 字符，因为它们由范围限定。

			whitespace white-space
\s	空白字符；<Space> 和 <Tab>		/\s
\S	非空白字符：\s 之反		/\S
\d	数位：	[0-9]	/\d
\D	非数位：	[^0-9]	/\D
\x	十六进制数位：	[0-9A-Fa-f]	/\x
\X	非十六进制数位：	[^0-9A-Fa-f]	/\X
\o	八进制数位：	[0-7]	/\o
\O	非八进制数位：	[^0-7]	/\O
\w	单词字符：	[0-9A-Za-z_]	/\w
\W	非单词字符：	[^0-9A-Za-z_]	/\W
\h	单词首字符：	[A-Za-z_]	/\h
\H	非单词首字符：	[^A-Za-z_]	/\H
\a	英文字母字符：	[A-Za-z]	/\a
\A	非英文字母字符：	[^A-Za-z]	/\A
\l	小写字符：	[a-z]	/\l
\L	非小写字符：	[^a-z]	/\L
\u	大写字符：	[A-Z]	/\u
\U	非大写字符：	[^A-Z]	/\U

备注：使用匹配原比使用 [] 构造要快。

备注：字符类不使用 'ignorecase'，"\c" 和 "\C" 的设定。

```
/\_   E63   /\_i   /\_I   /\_k   /\_K   /\_f   /\_F
/\_p   /\_P   /\_s   /\_S   /\_d   /\_D   /\_x   /\_X
/\_o   /\_O   /\_w   /\_W   /\_h   /\_H   /\_a   /\_A
/\_l   /\_L   /\_u   /\_U
```

_x	其中 x 为上面列出的可用字符之一：结果为对应字符类加上行尾符（字符类结束）	

\e	匹配 <Esc>	/\e
\t	匹配 <Tab>	/\t
\r	匹配 <CR>	/\r
\b	匹配 <BS>	/\b

\n　匹配行尾符　/\n
当对字符串而不是缓冲区进行匹配时，匹配一个按本义出现的换行字符。

~　匹配上次给出的替代字符串　/~　/\~

\(\)　一个由转义的括号括起来的模式。　/\(　/\(\)　/\)
例: "\(^a\)" 匹配行首的 'a'。
E51　E54　E55　E872　E873

\1　匹配和第一个 \(和 \) 之间的子表达式的匹配文本　/\1　E65
相同的字符串。
例如: "\([a-z]\).\1" 匹配 "ata", "ehe", "tot" 等。

\2　类似于 "\1"，但使用第二子表达式，　/\2
...　/\3
\9　类似于 "\1"，但使用第三子表达式。　/\9
备注: 各组的标号是基于哪一个 "\(" 先出现 (自左至右)，而 非 哪一个先匹配。

\%(\)　一个由转义的括号括起来的模式。　/\%(\)　/\%(　E53
类似 \(\)，但不算作一个子表达式。这样做允许使用更多的群组，并且处理时会稍快些。

x　一个没有特殊含义的单个字符，匹配其自身

/\　/\\
\x　一个反斜杠带一个没有特殊含义的单个字符，保留作将来的扩展

[]　('nomagic' 的情形: \[])　/[]　/\[]　/_[]　/collection
_[]
一个集合。一组由方括号括起来的字符。匹配集合中的任何一个字符。

示例	匹配
[xyz]	'x'、'y' 或 'z' 之一
[a-zA-Z]$	出现在行尾的字母字符
\c[a-z]$	同上
[-]	俄语字符集 (包括 utf-8 和 cp1251)

/[\n]
如果在集合前加上 "_" 的话，同时匹配行尾符，这等价于给集合加上 "\n"。即使该模式以 "^" 开始，也能匹配行尾符！因此 "_[^ab]" 将匹配行尾符以及所有非 "a" 或 "b" 的字符。
这使其与 Vi 兼容: 没有 "_" 或 "\n" 时，集合不匹配行尾符。

E769
如果没有 ']'，Vim 不会给出错误信息，而是假设不使用集合。可用于直接搜索 '['。不过内部搜索会得到 E769。同时要注意在 ":substitute" 命令里，整个命令作为模式处理，例如 ":s/[/x/" 搜索 "[/x" 并替换为空，而不是搜索 "[" 并替换为 "x"！

E944　E945
如果集合以 "^" 开始，它匹配所有 不在 集合内的字符: "[^xyz]" 匹配任何不是 'x'、'y' 和 'z' 的字符。
- 如果集合中的两个字符以 '-' 隔开，表示它们之间的所有 ASCII 字符。例如，"[0-9]" 匹配任何十进制数位。如果开始字符大于结束字符，如 [c-a]，报错 E944。可以使用非 ASCII 字符，但在旧的正规表达式引擎中字符值的距

离不能超过 256。例如设置 re=1 后搜索 [\u3000-\u4000]，会报错 E945。在模式之前加上 \%#=2 可以解决这个问题。

- 字符类表达式被解释为该字符类代表的字符集合。支持以下字符类：

名称	函数	内容
[:alnum:]	[:alnum:]	isalnum	字母和数位
[:alpha:]	[:alpha:]	isalpha	字母
[:blank:]	[:blank:]		空格和制表
[:cntrl:]	[:cntrl:]	iscntrl	控制字符
[:digit:]	[:digit:]		十进制数位 '0' 到 '9'
[:graph:]	[:graph:]	isgraph	ASCII 可显示字符，不包括空格
[:lower:]	[:lower:]	(1)	小写字母 (使用 'ignorecase' 时代表所有字母)
[:print:]	[:print:]	(2)	可显示字符，包括空格
[:punct:]	[:punct:]	ispunct	ASCII 标点字符
[:space:]	[:space:]		空白字符：空格、制表、CR、NL、垂直制表、换页
[:upper:]	[:upper:]	(3)	大写字母 (使用 'ignorecase' 时代表所有字母)
[:xdigit:]	[:xdigit:]		十六进制数位：0-9、a-f、A-F
[:return:]	[:return:]		<CR> 字符
[:tab:]	[:tab:]		<Tab> 字符
[:escape:]	[:escape:]		<Esc> 字符
[:backspace:]	[:backspace:]		<BS> 字符
[:ident:]	[:ident:]		标识符字符 (同 "\i")
[:keyword:]	[:keyword:]		关键字字符 (同 "\k")
[:fname:]	[:fname:]		文件名字符 (同 "\f")

方括号括起的字符类表达式是对于方括号定界的集合的扩充。例如，下面的模式是一个 UNIX 文件名："[-./[:alnum:]_~]\+"。即至少有一个字符的字符列表，其中字符可以是 '-'，'.'，'/'，字母，数位，'_' 或 '~' 中的一个。这些项仅对 8 位字符有效，但在新的正规表达式引擎里，[:lower:] 和 [:upper:] 也可用于多字节字符。见 two-engines 。将来，其它项目可能也可用于多字节字符。现在，要得到所有的 "alpha" 字符，可用：[[:lower:][:upper:]]。

"函数" 列显示使用的库函数。具体实现取决于系统。否则：
(1) ASCII 使用 islower()，其它字符使用 Vim 内建规则。
(2) 使用 Vim 内建规则
(3) 用 (1) 但使用 isupper()

/[[= [==]

- 等价类 (equivalence class)。匹配的字符和被比较的字符 "几乎" 相同，但重音的区别可以忽略。目前支持 Unicode、latin1 和 latin9。形式是：

  [=a=]

/[[. [..]

- 排序规则元素 (collation element)。目前只能简单地接受如下形式的单个字符：

  [.a.]

/\]

- 要在集合之内包括字符 ']'，^'，'-' 或 '\'，只需在该字符前加上反斜杠 "[xyz\]]"，"[\^xyz]"，"[xy\-z]" 和 "[xyz\\]"。
  (备注：POSIX 不支持这种反斜杠的用法)。
  对于 ']' 你也可以将其作为第一个字符 (但可能在 "^" 之后) 而无须转义："[]xyz]" 或 "[^]xyz]"。
  对于 '-' 你也可以将其作为第一个或最后一个字符："[-xyz]"，"[^-xyz]" 或 "[xyz-]"。

对于 '\' 你也可以将其放置在除了 "^]-\bdertnoUux" 以外的任何一个字符之前而无须转义：匹配 '\'，'x'，'y' 及 'z'。但是用 "\\" 好些，因为将来的扩展可能会在 '\' 之后使用其它字符。

- 省略结尾的] 不被视为错误。"[]" 等同于 "[]]"，匹配 ']' 字符。
- 当 'cpoptions' 中不包括 'l' 标志位时，以下转义可用：

```
\e      <Esc>
\t      <Tab>
\r      <CR>    ( 不是 行尾符！)
\b      <BS>
\n      换行符，见上 /[\n]
\d123   字符的十进制表示
\o40    字符的八进制表示，最大值为 0377
\x20    字符的十六进制表示，最大值为 0xff
\u20AC  多字节字符的十六进制表示，最大值为 0xffff
\U1234  多字节字符的十六进制表示，最大值为 0xffffffff
```

备注：其它曾在前面提到的反斜杠转义码（译者注：如 \s、\d 等）在方括号 [] 之内是不起作用的！

- 用集合来匹配可能会很慢。因为每个字符都要和集合中的每一个字符作比较。尽可能使用上面提到的其它匹配原。例如：同样是匹配一个数位，"\d" 要比 "[0-9]" 快的多。不过，新的 NFA 正规表达式引擎比旧的引擎这方面要处理地好得多。

/\%[] E69 E70 E369

\%[] 一个可选匹配原的序列。它总能匹配，但试图匹配尽量多的列出的匹配原，而在第一个不匹配的地方终止。例如：

```
/r\%[ead]
```

匹配 "r"，"re"，"rea" 或者 "read"。采用最长可能的匹配。下面例子可以用来匹配 Ex 命令 "function"，其中 "fu" 是必须的而 "nction" 是可选的：

```
/\<fu\%[nction]\>
```

这里用到了单词结尾匹配原 "\>" 来避免匹配 "full" 中的 "fu"。
当要匹配的不是普通字符时，问题就变得更复杂了。你不一定会经常这样，但可以这么做。例如：

```
/\<r\%[[eo]ad]\>
```

匹配 "r"，"re"，"ro"，"rea"，"roa"，"read" 及 "road"。
[] 里不能有 \(\)、\%(\) 或 \z(\) 项，\%[] 也不能嵌套。
要包含 "["，用 "[[]" 而 "]" 则可用 "[]]"，例如：

```
/index\%[[[]0[]]]
```

匹配 "index"、"index["、"index[0" 和 "index[0]"。
{仅当编译时加入 +syntax 特性才有效}

/\%d /\%x /\%o /\%u /\%U E678

\%d123 匹配十进制指定的字符。后面必须跟着非数位。
\%o40 匹配八进制指定的字符，至多 0377。
小于 040 的数必须后面跟着非八进制的数位或非数位。
\%x2a 匹配不超过两位十六进制数位指定的字符。
\%u20AC 匹配不超过四位十六进制数位指定的字符。
\%U1234abcd 匹配不超过八位十六进制数位指定的字符，直到 0x7fffffff

---

7. 忽略大小写 /ignorecase

如果选项 'ignorecase' 打开的话，忽略正常字母的大小写。设定 'smartcase' 会在模

式只包含小写字母时才忽略大小写。

/\c /\C
如果模式的任何位置出现了 "\c"，整个模式会像打开 'ignorecase' 一样处理。真正的 'ignorecase' 和 'smartcase' 的值会被忽略。"\C" 的效果恰好相反：强制对整个模式匹配大小写。
{仅 Vim 支持 \c 和 \C}
注意 'ignorecase'，"\c" 和 "\C" 对字符类无效。

例如：

模式	'ignorecase'	'smartcase'	匹配
foo	关	-	foo
foo	开	-	foo Foo FOO
Foo	开	关	foo Foo FOO
Foo	开	开	Foo
\cfoo	-	-	foo Foo FOO
foo\C	-	-	foo

技术细节：
NL-used-for-Nul
文件中的 <Nul> 字符在内存中存储为 <NL>。显示为 "^@"。翻译是在读写文件时完成的。为了在查找模式中匹配 <Nul> 你可以使用 CTRL-@ 或 "CTRL-V 000"。这应该不出你的所料。在内部，该字符被替换为 <NL>。不寻常的是键入 CTRL-V CTRL-J 同样插入一个 <NL>，因此也搜索文件中的 <Nul>。

CR-used-for-NL
当 'fileformat' 为 "mac" 时，<NL> 字符在内部储存为 <CR>。在文本里显示为 "^J"。除此以外，和 <NL> 作为 <Nul> 的用法类似。

在作表达式求值时，模式中的 <NL> 字符匹配字符串中的 <NL>。用 "\n"（反斜杠 n）来匹配 <NL> 在这里是行不通的，它仅对缓冲区内的文本有效。

pattern-multi-byte
模式匹配对于多字节字符同样适用。大体上使用并无区别，不过要注意无效字节可能引起的问题，一个包含无效字节的模式永远不会得到匹配。

---

8. 组合字符

patterns-composing

/\Z
如果模式的任何位置出现了 "\Z"，所有的组合字符将被忽略。这时，只比较基本字符，组合字符可以不同，其数量也可以不同。这一功能只在 'encoding' 为 'utf-8' 时有意义。
例外：如果模式以一或多个组合字符开始，它们必须匹配。

/\%C
"\%C" 可用来跳过任何组合字符。例如，模式 "a" 不匹配 "cat"（这里 a 有组合字符 0x0300），但 "a\%C" 可以。注意 不匹配 "cát"（这里 á 是字符 0xe1，没有组合字符）。但匹配 "cat"（这里 a 就是简单的 a）。

如果一个组合字符出现在模式的开始处或在不包含组合字符的项目之后出现，它匹配任何包含该组合字符的字符。

句号和组合字符联用和组合字符本身的效果相同，但是你不用担心该字符前面出现的是什么项目。

组合字符出现的顺序无关重要。另外，文本可以使用比模式更多的组合字符，它仍然匹配。但模式中组合字符必须全数出现在文本中。

假定 B 是基本字符，x 和 y 是组合字符：

模式	文本	匹配与否
Bxy	Bxy	是（完全匹配）
Bxy	Byx	是（忽略顺序）
Bxy	By	否（缺失 x）
Bxy	Bx	否（缺失 y）
Bx	Bx	是（完全匹配）
Bx	By	否（缺失 x）
Bx	Bxy	是（忽略多余的 y）
Bx	Byx	是（忽略多余的 y）

---

9. 与 Perl 模式的比较 perl-patterns

Vim 中的正则表达式和 Perl 的在功能上来说非常相似。它们之间的区别基本上只是在表示方法上。这里给出一个它们之间的区别的总结：

功能	Vim 方言	Perl 方言
强制不区分大小写	\c	(?i)
强制区分大小写	\C	(?-i)
不须被引用的群组	\%(atom\)	(?:atom)
保守的倍数描述	\{-n,m}	*?, +?, ??, {}?
零宽度匹配	atom\@=	(?=atom)
零宽度否匹配	atom\@!	(?!atom)
零宽度反向匹配	atom\@<=	(?<=atom)
零宽度反向否匹配	atom\@<!	(?<!atom)
无重试匹配	atom\@>	(?>atom)

(译注：常见的其它译法

不须被引用的群组	非捕获分组
保守的倍数描述	非贪婪匹配
零宽度匹配	正向预搜索匹配
零宽度否匹配	正向预搜索不匹配
零宽度反向匹配	反向预搜索匹配
零宽度反向否匹配	反向预搜索不匹配

)

Vim 和 Perl 匹配一个字符串中的换行符时有所不同：

在 Perl 里，^ 和 $ 缺省只匹配文本的开始和结尾。但是你也可以设定 'm' 标志位来使它们匹配内嵌的换行符。你还可以设定 's' 标志位使句号 (.) 可以也用来匹配换行符。(顺便提一下，使用类似上面 i 标志位的用法，可以在在模式内部改变这两个标志位。)

另一方面，Vim 中的 ^ 和 $ 永远匹配内嵌的换行符。但另外有两个匹配原： \%^ 和 \%$，可以分别用来匹配 (且只匹配) 文本的起始和结尾。对第二个问题，Vim 用 _ "修饰符" 来解决：把它放在一个句号或字符类之前，对应的模式就可以同时匹配换行字符。

最后，以下这些结构是 Perl 独有的：
- 在正则表达式内执行任意代码： (?{perl 代码})

- 条件表达式： (?(条件)真值表达式|假值表达式)

... 而这些是 Vim 独有的:
- 改变模式的 'magic' 属性： \v \V \m \M (对于避免反斜杠非常有用)
- 可选匹配原的序列： \%[atoms]
- \& (\& 之于 \| 就像 "与" 之于 "或"；它强迫多个模式在同一个位置匹配)
- 通过行/列号匹配： \%5l \%5c \%5v
- 设置匹配的开始和结束位置： \zs \ze

---

10. 高亮显示匹配 match-highlight

:mat :match

:mat[ch] {group} /{pattern}/

定义一个在当前窗口高亮显示的模式。它会以 {group} 高亮。例:

```
:highlight MyGroup ctermbg=green guibg=green
:match MyGroup /TODO/
```

除了 // 以外，任何字符都可以用来标识一个 {pattern} 的开始和结束。要当心某些特殊的字符，例如 '"' 和 '|'。

该命令执行时，{group} 必须已经定义。

对于 'hlsearch' 要高亮的字符，仍然适用 {group} 高亮，因为匹配高亮的优先级比 'hlsearch' 要高。同样的，语法高亮 (见 'syntax') 也被匹配否决。

注意 使用 'hlsearch' 高亮显示最近使用的查找模式对所有窗口都有效，而以 ":match" 定义的模式仅仅对当前窗口生效。当切换至另一个缓冲区时，该设定被保留。

'ignorecase' 不起作用。在模式中使用 /\c 来忽略大小写，否则，大小写总是区分的。

'redrawtime' 定义搜索模式匹配的最大时间。

匹配行尾而 Vim 只重绘部分显示时，结果可能会令你意想不到。这是因为 Vim 在重绘开始的行上寻找匹配的缘故。

另见 matcharg() 和 getmatches()。前者返回上次 :match 命令使用的高亮组和模式。后者返回 matchadd() 和 :match 两者定义的所有高亮组和模式的列表。

:match 定义的高亮匹配仅限于三个 (除了 :match 以外，还有 :2match 和 :3match 可用)。而 matchadd() 无此限制，而且还能排定匹配的优先级。

另一个例子，高亮显示虚拟第 72 列之后的所有文本:

```
:highlight rightMargin term=bold ctermfg=blue guifg=blue
:match rightMargin /.\%>72v/
```

要高亮显示所有位于虚拟第 7 列的文本:

```
:highlight col8 ctermbg=grey guibg=grey
:match col8 /\%<8v.\%>7v/
```

注意 用两个项目是为了匹配那些占用了超过一个虚拟列的字符，例如

TAB。

:mat[ch]
:mat[ch] none
清除上次定义的匹配模式。

:2mat[ch] {group} /{pattern}/ :2match
:2mat[ch]
:2mat[ch] none
:3mat[ch] {group} /{pattern}/ :3match
:3mat[ch]
:3mat[ch] none
和上面的 :match 完全类似，但设置不同的匹配。这样，同时可以有三组激活的匹配。如果不同匹配在相同位置出现，最小编号的匹配优先。
":3match" 命令用于 matchparen 插件。建议使用 ":match" 进行手动的搜索，而 ":2match" 用于其它的插件。

vim:tw=78:ts=8:noet:ft=help:norl:

map.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　　by Bram Moolenaar
译者：　con<con@netease.com>
http://vimcdoc.sf.net

键映射、缩写和用户定义的命令。

本主题在用户手册　05.3　，　24.7　和　40.1　中有过介绍。

1. 键映射　　key-mapping
　1.1 映 射 命 令　　:map-commands
　1.2 特殊参数　　:map-arguments
　1.3 映射与运行模式　　:map-modes
　1.4 列出映射　　map-listing
　1.5 映射特殊键　　:map-special-keys
　1.6 特殊字符　　:map-special-chars
　1.7 映射哪些键　　map-which-keys
　1.8 示例　　map-examples
　1.9 使用映射　　map-typing
　1.10 映射 ALT 键　　:map-alt-keys
　1.11 modifyOtherKeys 模式映射　　modifyOtherKeys
　1.12 映射操作符　　:map-operator
2. 缩写　　abbreviations
3. 局部映射和函数　　script-local
4. 用户定义的命令　　user-commands

---

1. 键映射　　key-mapping　mapping　macro

键映射用于改变输入键的含义。最常见的用途是把功能键定义为一系列的命令。比如：

```
:map <F2> a<C-R>=strftime("%c")<CR><Esc>
```

这个映射会在光标之后追加当前的日期和时间（用 <> 记法　<>）。

1.1 映 射 命 令　　:map-commands

有很多命令用于定义新的映射，删除映射和列出当前的映射。可以从　map-overview　参考 "映射" 命令的不同形式及其与模式的关系。

{lhs}　表示左手边　　{lhs}
{rhs}　表示右手边　　{rhs}

:map　{lhs} {rhs}	mapmode-nvo	:map	
:nm[ap] {lhs} {rhs}	mapmode-n	:nm	:nmap
:vm[ap] {lhs} {rhs}	mapmode-v	:vm	:vmap
:xm[ap] {lhs} {rhs}	mapmode-x	:xm	:xmap
:smap　{lhs} {rhs}	mapmode-s	:smap	
:om[ap] {lhs} {rhs}	mapmode-o	:om	:omap
:map!　{lhs} {rhs}	mapmode-ic	:map!	

:im[ap] {lhs} {rhs}	mapmode-i	:im	:imap
:lm[ap] {lhs} {rhs}	mapmode-l	:lm	:lma :lmap
:cm[ap] {lhs} {rhs}	mapmode-c	:cm	:cmap
:tma[p] {lhs} {rhs}	mapmode-t	:tma	:tmap

在映射命令作用的模式中把键系列 {lhs} 映射为 {rhs}。并且映射后的 {rhs} 也被进行映射扫描。这个特性可以用来进行映射的嵌套和递归。

		:nore	:norem	
:no[remap] {lhs} {rhs}	mapmode-nvo	:no	:noremap	:nor
:nn[oremap] {lhs} {rhs}	mapmode-n	:nn	:nnoremap	
:vn[oremap] {lhs} {rhs}	mapmode-v	:vn	:vnoremap	
:xn[oremap] {lhs} {rhs}	mapmode-x	:xn	:xnoremap	
:snor[emap] {lhs} {rhs}	mapmode-s	:snor	:snore	:snoremap
:ono[remap] {lhs} {rhs}	mapmode-o	:ono	:onoremap	
:no[remap]! {lhs} {rhs}	mapmode-ic	:no!	:noremap!	
:ino[remap] {lhs} {rhs}	mapmode-i	:ino	:inor	:inoremap
:ln[oremap] {lhs} {rhs}	mapmode-l	:ln	:lnoremap	
:cno[remap] {lhs} {rhs}	mapmode-c	:cno	:cnor	:cnoremap
:tno[remap] {lhs} {rhs}	mapmode-t	:tno	:tnoremap	

在映射命令作用的模式中把键序列 {lhs} 映射为 {rhs} 。禁止对映射后的 {rhs} 进行映射扫描。这个特性可以避免映射的嵌套和递归。通常用于重定义一个命令。

:unm[ap] {lhs}	mapmode-nvo	:unm	:unmap
:nun[map] {lhs}	mapmode-n	:nun	:nunmap
:vu[nmap] {lhs}	mapmode-v	:vu	:vunmap
:xu[nmap] {lhs}	mapmode-x	:xu	:xunmap
:sunm[ap] {lhs}	mapmode-s	:sunm	:sunmap
:ou[nmap] {lhs}	mapmode-o	:ou	:ounmap
:unm[ap]! {lhs}	mapmode-ic	:unm!	:unmap!
:iu[nmap] {lhs}	mapmode-i	:iu	:iunmap
:lu[nmap] {lhs}	mapmode-l	:lu	:lunmap
:cu[nmap] {lhs}	mapmode-c	:cu	:cun :cunmap
:tunma[p] {lhs}	mapmode-t	:tunma	:tunmap

在映射命令作用的模式中删除 {lhs} 的映射。该映射仍然可以在其它模式中保留其定义。

备注：{lhs} 包含末尾的空格。该映射取消操作 不会 生效：

```
:map @@ foo
:unmap @@ | print
```

:mapc[lear]	mapmode-nvo	:mapc	:mapclear
:nmapc[lear]	mapmode-n	:nmapc	:nmapclear
:vmapc[lear]	mapmode-v	:vmapc	:vmapclear
:xmapc[lear]	mapmode-x	:xmapc	:xmapclear
:smapc[lear]	mapmode-s	:smapc	:smapclear
:omapc[lear]	mapmode-o	:omapc	:omapclear
:mapc[lear]!	mapmode-ic	:mapc!	:mapclear!
:imapc[lear]	mapmode-i	:imapc	:imapclear
:lmapc[lear]	mapmode-l	:lmapc	:lmapclear
:cmapc[lear]	mapmode-c	:cmapc	:cmapclear
:tmapc[lear]	mapmode-t	:tmapc	:tmapclear

在映射命令作用的模式中删除 所有 的映射。

<buffer> 参数可用来删除所的有缓冲区局部映射
:map-<buffer>
警告：同时也会删除缺省的映射。

:map	mapmode-nvo
:nm[ap]	mapmode-n
:vm[ap]	mapmode-v
:xm[ap]	mapmode-x
:sm[ap]	mapmode-s
:om[ap]	mapmode-o
:map!	mapmode-ic
:im[ap]	mapmode-i
:lm[ap]	mapmode-l
:cm[ap]	mapmode-c
:tma[p]	mapmode-t

在映射命令作用的模式中列出所有的键映射。注意 ":map" 和 ":map!" 是最常用的，因为它们包括其它模式。

:map {lhs}	mapmode-nvo	:map_l
:nm[ap] {lhs}	mapmode-n	:nmap_l
:vm[ap] {lhs}	mapmode-v	:vmap_l
:xm[ap] {lhs}	mapmode-x	:xmap_l
:sm[ap] {lhs}	mapmode-s	:smap_l
:om[ap] {lhs}	mapmode-o	:omap_l
:map! {lhs}	mapmode-ic	:map_l!
:im[ap] {lhs}	mapmode-i	:imap_l
:lm[ap] {lhs}	mapmode-l	:lmap_l
:cm[ap] {lhs}	mapmode-c	:cmap_l
:tma[p] {lhs}	mapmode-t	:tmap_l

在映射命令作用的模式中列出以 {lhs} 开头的键映射的键系列。

这些命令用于把一个键或键系列映射成一个字符串。可以用来在功能键里放置一系列命令，把一个键转换成另一个，等等。如何保存和恢复当前映射可以参考 :mkexrc 。

map-ambiguous
当两个映射以相同的字符顺序开始，它们是有二义性的。例如：

```
:imap aa foo
:imap aaa bar
```

当 Vim 读入 "aa" 后，它需要取得另外一个字符才能决定应该映射 "aa" 还是 "aaa"。这意味着输入 "aa" 后映射还不会展开，Vim 还在等待另一个字符。如果你接着输入一个空格，那么将插入 "foo" 加上空格。如果你输入一个 "a"，那么将插入 "bar"。

1.2 特 殊 参 数

:map-arguments

"<buffer>"，"<nowait>"，"<silent>"，"<special>"、"<script>"、"<expr>" 和 "<unique>" 可以按任意顺序使用。它们必须紧跟在命令的后边，而在其它任何参数的前边。

:map-local :map-<buffer> E224 E225
如果这些命令的第一个参数是 "<buffer>"，映射将只局限于当前的缓冲区内。例如：

```
:map <buffer> ,w /[.,;]<CR>
```

然后你可以在另一个缓冲区内把 ",w" 作另外的映射：

```
:map <buffer>  ,w  /[#&!]<CR>
```
局部缓冲区映射在全局映射之前被应用。下面介绍的 <nowait> 可以使一个较短的局部映射在较长的全局映射存在时不起作用 (译者注：此处或有误)。
"<buffer>" 参数也可以用于清除映射：
```
:unmap <buffer> ,w
:mapclear <buffer>
```
当一个缓冲区被删除时局部映射也会被清除，但是在它被卸载时不会。就像局部选项值的情况一样。
另见 map-precedence 。

:map-<nowait> :map-nowait

定义局部于缓冲区的映射 "," 时，可能有另一个全局映射也以 "," 开始。这时你需要键入另一个字符，Vim 才能知道是用 "," 映射还是更长的那个。要避免这个问题，加入 <nowait> 参数。这样映射一旦匹配就会被使用，Vim 不会等待更多字符的输入。但如果那些字符已经输入了，还是会使用的。

:map-<silent> :map-silent

要在定义一个映射时不在命令行上回显该映射，可以使用 "<silent>" 作为第一个参数，例如：
```
:map <silent> ,h /Header<CR>
```
在使用这个映射时搜索字串将不回显。不过被执行命令的信息仍然会。要把它也关掉，可以在执行的命令里加入一个 ":silent"：
```
:map <silent> ,h :exe ":silent normal /Header\r"<CR>
```
仍然会给出提示，比如使用 inputdialog() 的时候。
在缩写上使用 "<silent>" 是可以的，但它的作用是使命令行不进行重绘。

:map-<special> :map-special

定义映射时，特殊键可用 <> 记法，即使 'cpoptions' 包含了 "<" 标志位也没问题。这可用于不希望看到设置 'cpoptions' 时出现的副作用的场合。例如：
```
:map <special> <F12> /Header<CR>
```

:map-<script> :map-script

如果给用于定义新映射或缩写的命令的第一个参数是 "<script>"，该映射只使用通过以 "<SID>" 开头来定义的的脚本局部映射来重映射 {rhs} 中的字符。这可以用于避免来自外部的脚本的干扰 (举例来说，在 mswin.vim 中 CTRL-V 被重新映射的时候就是如此)，但是又需要使用该脚本中定义的其它映射的情形。
备注：":map <script>" 和 ":noremap <script>" 做同样的事情。这里 "<script>" 超越命令名。不过，更推荐使用 ":noremap <script>"，因为它更清晰地表示了重映射已被 (大多数时候) 禁止。

:map-<unique> E226 E227

如果给用于定义新映射或缩写的命令的第一个参数是 "<unique>" 并且它该映射或缩写已经存在，则该命令会失败。例如：
```
:map <unique> ,w  /[#&!]<CR>
```
定义一个局部映射时，同时也会检查是否已存在了一个相同的全局映射。
这个例子将失败：
```
:map ,w  /[#&!]<CR>
:map <buffer> <unique> ,w  /[.,;]<CR>
```
如果你想给键进行映射，但同时又想执行原来映射的内容，参见 maparg() 。

:map-<expr> :map-expression

如果给用于定义新映射或缩写的命令的第一个参数是 "<expr>"，那么参数会作为表达式来进行计算，结果作为实际使用的 {rhs}。例如：
```
:inoremap <expr> . InsertDot()
```

会插入 InsertDot() 函数的返回值。这可以用来检查光标之前的文本并在一定条件下启动全能 (omni) 补全。

对于缩写， v:char 设为激活缩写的那个输入字符。你可以用它来决定如何扩展 {lhs}。不能修改或插入 v:char。

要非常小心副作用！计算表达式的同时正在获取字符，因此很有可能你使得该命令不再可用。为此原因禁止以下行为：
- 改变缓冲区文本 textlock
- 编辑其它缓冲区
- :normal 命令
- 可以移动光标，但事后光标会被恢复

如果你希望通过映射来完成这些操作，让返回的字符做这些事情。

你可以使用 getchar()，丢弃可能有的预输入。例如，如果有以下的映射：

```
inoremap <expr> <C-L> nr2char(getchar())
inoremap <expr> <C-L>x "foo"
```

此时你如果输入 CTRL-L，什么都不会发生。Vim 需要下一个字符来决定采用哪个映射。如果键入 'x'，采用第二个映射，插入 "foo"。如果键入其他字符，采用第一个映射，getchar() 得到键入的字符并返回之。

这里是插入递增的列表编号的例子：

```
let counter = 0
inoremap <expr> <C-L> ListItem()
inoremap <expr> <C-R> ListReset()

func ListItem()
  let g:counter += 1
  return g:counter . '. '
endfunc

func ListReset()
  let g:counter = 0
  return ''
endfunc
```

CTRL-L 插入下一个数值，CTRL-R 复位计数且返回空字符串，这样就不会插入任何内容。

注意 要使特殊键工作并转义文本中的 CSI 字节需要一些特殊处理。 :map 命令已经做好了，所以你应该避免做重复的操作。这样不行：

```
:imap <expr> <F3> "<Char-0x611B>"
```

因为 <Char- 序列作为 :imap 的参数被转义，而 <expr> 又做一次。这样就可以：

```
:imap <expr> <F3> "\u611B"
```

在其它文本之前使用单个字节出现的 0x80 是不行的。它会被看作一个特殊键。

1.3 映 射 与 运 行 模 式 :map-modes

mapmode-nvo mapmode-n mapmode-v mapmode-o

有六种映射存在
- 用于普通模式：输入命令时。
- 用于可视模式：可视区域高亮并输入命令时。
- 用于选择模式：类似于可视模式，但键入的字符对选择区进行替换。
- 用于操作符等待模式：操作符等待中 ("d", "y", "c" 等等之后)。

见下： omap-info 。
- 用于插入模式：也用于替换模式。
- 用于命令行模式：输入 ":" 或 "/" 命令时。

特殊情况：当在普通模式里为一个命令输入一个计数时，对 0 的映射会被禁用。这样在输入一个带有 0 的计数时不会受到对 0 键映射的干扰。

map-overview map-modes

关于每个映射命令对应的工作模式的概况。详情见下。

命 令			模 式
:map	:noremap	:unmap	普通、可视、选择、操作符等待
:nmap	:nnoremap	:nunmap	普通
:vmap	:vnoremap	:vunmap	可视与选择
:smap	:snoremap	:sunmap	选择
:xmap	:xnoremap	:xunmap	可视
:omap	:onoremap	:ounmap	操作符等待
:map!	:noremap!	:unmap!	插入与命令行
:imap	:inoremap	:iunmap	插入
:lmap	:lnoremap	:lunmap	插入、命令行、Lang-Arg
:cmap	:cnoremap	:cunmap	命令行
:tmap	:tnoremap	:tunmap	终端作业

命 令				模 式		
				普通	可视+选择	操作符等待
:map	:noremap	:unmap	:mapclear	是	是	是
:nmap	:nnoremap	:nunmap	:nmapclear	是	-	-
:vmap	:vnoremap	:vunmap	:vmapclear	-	是	-
:omap	:onoremap	:ounmap	:omapclear	-	-	是

修道院之外也有 :nunmap (译者注：nun，修女)。

mapmode-x mapmode-s

有的命令能同时用于可视和选择模式，有的只能用于其中一个。注意 很常见的情况是提到 "可视" 的时候实际同时适用可视和选择两种模式。 Select-mode-mapping

备注：在选择模式映射可显示字符容易引起用户的混淆。最好直接用 :xmap 和 :smap 来映射可显示字符。或者在定义映射后使用 :sunmap。

命 令				模 式	
				可视	选择
:vmap	:vnoremap	:vunmap	:vmapclear	是	是
:xmap	:xnoremap	:xunmap	:xmapclear	是	-
:smap	:snoremap	:sunmap	:smapclear	-	是

mapmode-ic mapmode-i mapmode-c mapmode-l

有的命令同时支持插入模式和命令行模式，有的不是：

命 令				模 式		
				插入	命令行	Lang-Arg
:map!	:noremap!	:unmap!	:mapclear!	是	是	-
:imap	:inoremap	:iunmap	:imapclear	是	-	-
:cmap	:cnoremap	:cunmap	:cmapclear	-	是	-
:lmap	:lnoremap	:lunmap	:lmapclear	是*	是*	是*

如果 'iminsert' 为 1，见下面的 language-mapping 。

原来的 Vi 没有针对普通/可视/操作符等待模式和针对插入/命令行模式的独立映射。因此 ":map" 和 ":map!" 命令为多个模式定义和回显映射。在 Vim 中你可以使用 ":nmap"、":vmap"、:omap"、":cmap" 和 ":imap" 命令来对每个不同的模式分别定义映射。

mapmode-t

终端映射用于终端窗口中，其中输入的键用于终端中运行的作业。见 terminal-typing 。

omap-info

操作符等待映射可以用来定义和任何操作符一起使用的移动命令。简单例子：

```
:omap { w
```

会使 "y{" 等同于 "yw"，"d{" 也等同于 "dw"。

要忽略光标原来所在的位置并选择另外的文本，你可以使 omap 进入可视模式来选择要操作的文本。例如，要在位于当前行的函数名上操作：

```
onoremap <silent> F :<C-U>normal! 0f(hviw<CR>
```

CTRL-U (<C-U>) 用于删除命令行上 Vim 可能插入的范围。普通模式命令寻找第一个 '(' 字符并选择之前的第一个单词。通常那就是函数名了。

要为普通和可视模式但不包括操作符等待模式输入一个映射，首先在所有的三个模式中定义该映射，然后在操作符等待模式中取消该映射：

```
:map    xx something-difficult
:ounmap xx
```

对于一个同时用于可视和操作符等待模式、或同时用于普通和操作符等待模式的映射也可照此办理。

language-mapping

":lmap" 定义一个应用于以下情况的映射：
- 插入模式
- 命令行模式
- 输入一个搜索模式时
- 接受一个文本字符作为参数的命令，比如 "r" 和 "f"
- 对于 input() 行

更一般地：任何输入的字符是缓冲区文本的一部分而非一个 Vim 命令字符的时候。"Lang-Arg" 不是真正的另外一个模式，它仅用来表示这些情况的存在。

载入一个相关语言映射集合的最简单的方法是通过使用 'keymap' 选项。参考 45.5 。

在插入模式和命令行模式中可用 CTRL-^ 命令来关闭映射 i_CTRL-^ c_CTRL-^ 。这些命令改变 'iminsert' 选项的值。普通命令行 (非模式搜索) 开始输入时，映射被关闭直到输入 CTRL-^ 为止。而插入模式和模式搜索却会分别记住上次使用的状态。需要输入一个字符作为参数的命令，如 "f" 或 "t" 之类，也使用插入模式的状态。

语言映射永远不能应用于已经映射的字符上。它们仅用于键入的字符上。这意味着输入映射时，语言映射已经完成。

1.4 列出映射

map-listing

当列出映射时，前面两栏的字符表示 (可有多个)：

字符	模式
<Space>	普通、可视、选择和操作符等待
n	普通

v	可视和选择
s	选择
x	可视
o	操作符等待
!	插入和命令行
i	插入
l	插入、命令行和 Lang-Arg 模式的 ":lmap" 映射
c	命令行
t	终端作业

{rhs} 之前可能显示一个特殊字符:

*	表示它不可重映射
&	表示仅脚本的局部映射可以被重映射
@	表示缓冲区的局部映射

从 {lhs} 以后的第一个非空字符到行的末尾 (或 '|') 都被认为是 {rhs} 的一部分。这允许 {rhs} 以一个空格结尾。

注意: 在可视模式里使用映射时, 你可以使用 "'<" 位置标记, 它表示当前缓冲区中最后被选中的可视区域的开始 '< 。

:filter 命令可用于选择列出哪种映射。模式匹配 {lhs} 和 {rhs} 原始文本。

:map-verbose

如果 'verbose' 非零, 列出键映射的同时可以显示它在哪里定义。例如:

```
:verbose map <C-W>*
n  <C-W>*      * <C-W><C-S>*
        Last set from /home/abcd/.vimrc
```

:verbose-cmd 说明详情。

1.5 映 射 特 殊 键 :map-special-keys

有三种方法来映射一个特殊键:

1. Vi 兼容的方法: 对键码进行映射。通常这是一个以 <Esc> 开头的序列。要输入一个这样的映射先输入 ":map " 然后再敲入功能键之前得先输入一个 CTRL-V。注意 如果键码在 termcap (t_ 开头的选项) 里, 它会被自动转换到内码并变成映射的第二种方法 (除非 'cpoptions' 里包括了 'k' 标志位)。
2. 第二种方法是使用功能键的内码。要输入这样的映射输入 CTRL-K 并敲要映射的功能键, 或者使用 "#1"、"#2"、.. "#9"、"#0"、"<Up>"、"<S-Down>"、"<S-F7>" 等等的
   形式 (参考键表 key-notation , 所有从 <Up> 开始的键都可以使用)。头十个功能键能以两种方式被定义: 仅用数字, 比如 "#2"; 或者使用 "<F>", 如 "<F2>"。两种都代表功能键 F2。"#0" 表示功能键 F10, 由选项 't_f10' 定义, 它在某些键盘上可能是 F0。<> 的形式在 'cpoptions' 包含 '<' 标志位时不能使用。
3. 使用 termcap 条目, 以 <t_xx> 的形式出现, 这里 "xx" 是 termcap 条目的名字。可以使用任何字符串条目。例如:
   ```
   :map <t_F3> G
   ```
   把功能键 13 映射成 "G"。'cpoptions' 包括 '<' 标志位时不能使用这种方式。

第二种和第三种方法的优点是不加修改就可以在不同的终端上使用 (功能键会被转换成相同的内码或实际的键码, 不论使用何种终端都是如此。termcap 必须正确才能正常工作,

并且必须使用相同的映射)。

细节: Vim 首先检查是否从键盘输入的序列是否已被映射。否的话将试图使用终端键码(参考 terminal-options)。如果找到终端编码,它会被替换成内码。然后再次检查一个映射是否已完成(因此你也能把一个内码映射成其它东西)。在脚本文件中写入什么东西取决于何者被识别。如果终端键码被识别为映射,写入键码本身;如果它被识别为一个终端编码,则在脚本中写入内码。

1.6 特殊字符

:map-special-chars
map_backslash map-backslash

注意 这里仅提及 CTRL-V 可以作为用于映射和缩写的特殊字符。当 'cpoptions' 不包含 'B' 时,反斜杠也可起到 CTRL-V 一样的作用,这时可以完全地使用 <> 记法 <> 。但你不能期望 "<C-V>" 像 CTRL-V 那样转换后来者的特殊含义。

要映射反斜杠,或者在 {rhs} 中使用按本义出现的反斜杠,可以使用特殊字符序列 "<Bslash>" 。这可以避免在使用嵌套映射时使用双反斜杠的需要。

map_CTRL-C map-CTRL-C
{lhs} 里可以使用 CTRL-C,但只有在 Vim 等待输入键时才可以,Vim 忙着做别的事情的时候不行。如果 Vim 在忙,CTRL-C 总是中断/打断该命令。
使用 MS-Windows 上的 GUI 版本时 CTRL-C 能被映射以允许复制到剪贴板的命令。使用 CTRL-Break 来中断 Vim。

map_space_in_lhs map-space_in_lhs
要在 {lhs} 中包含一个空格,在前面输入一个 CTRL-V (每个空格之前实际要输入两个 CTRL-V)。

map_space_in_rhs map-space_in_rhs
如果你需要 {rhs} 以空格开头,使用 "<Space>"。要与 Vi 完全兼容(但不可读),不要使用 <> 记法,在 {rhs} 前面先输入一个单独的 CTRL-V (你必须输入 CTRL-V 两次)。

map_empty_rhs map-empty-rhs
你可以通过在一个单独的 CTRL-V (你必须输入 CTRL-V 两次) 后面什么也不输入来建立一个空的 {rhs}。不幸的是在 vimrc 文件中你不能使用这种方式。

<Nop>
得到什么都不做的映射的更容易的方法是在 {rhs} 中使用 "<Nop>"。仅当 <> 记法允许时这种方法才生效。例如确保功能键 F8 什么事情都不做:

```
:map  <F8>  <Nop>
:map! <F8>  <Nop>
```

map-multibyte
可以对多字节字符映射,但只能是整个字符。不能仅映射第一个字节。这是为了避免下面场景中的问题:

```
:set encoding=latin1
:imap <M-C> foo
:set encoding=utf-8
```

<M-C> 的映射是在 latin1 编码中被定义的,结果是一个 0xc3 字节。如果你在 UTF-8 解码中输入 á (0xe1 <M-a>) 它是双字节 0xc3 0xa1。这个时候你不希望 0xc3 字节被映射,否则的话将不能输入 á 字符了。

<Leader> mapleader
要定义一个使用 "mapleader" 变量的映射,可以使用特殊字串 "<Leader>"。它会被 "mapleader" 的字串值所替换。如果 "mapleader" 未设置或为空,则用反斜杠代替,例

如：

```
:map <Leader>A  oanother line<Esc>
```

和下面一样：

```
:map \A  oanother line<Esc>
```

但是当：

```
:let mapleader = ","
```

时，又相当于：

```
:map ,A  oanother line<Esc>
```

注意 "mapleader" 的值仅当定义映射时被使用。后来改变的 "mapleader" 不会影响已定义的映射。

<LocalLeader> maplocalleader

<LocalLeader> 和 <Leader> 类似，除了它使用 "maplocalleader" 而非 "mapleader" 以外。<LocalLeader> 用于局部于缓冲区的映射，例如：

```
:map <buffer> <LocalLeader>A  oanother line<Esc>
```

在一个全局插件里应该使用 <Leader> 而在一个文件类型插件里应该用 <LocalLeader>。"mapleader" 和 "maplocalleader" 可以是相同的。尽管如此，如果你把它们设为不同，全局插件和文件类型插件的映射冲突的机会是不是会小一点呢？例如，你可以保持把 "mapleader" 设置为缺省的反斜杠，而设置 "maplocalleader" 为下划线。

map-<SID>

在一个脚本中有一个特殊关键字叫 "<SID>" 能被用来定义一个局部于脚本中的映射。具体细节请参考 <SID> 。

<Plug>

叫做 "<Plug>" 的特殊关键字可以用于一个内部映射，它不与任何键的序列匹配。这在插件中有用 using-<Plug> 。

<Char> <Char->

要根据一个字符的十进制，八进制或十六进制数字形式进行映射，可以使用 <Char> 来构造：

```
<Char-123>      字符 123
<Char-033>      字符 27
<Char-0x7f>     字符 127
<S-Char-114>    字符 114 ('r') 加上 Shift ('R')
```

它可以用来在一个 'keymap' 文件里指定一个（多字节）字符。大小写的区别此处不计。

map-comments

在这些命令的后面不可能放置注释，因为 '"' 字符被认为是 {lhs} 或 {rhs} 的一部分。不过，你可以用 |"，它开启了带注释的新空白命令。

map_bar map-bar

因为字符 '|' 用来分隔映射命令和后面的命令，所以包括 '|' 的 {rhs} 要做一些特殊的处理，有三种方法：

使用	可用于	示例
<Bar>	'<' 不在 'cpoptions' 里	:map _l :!ls <Bar> more^M
|	'b' 不在 'cpoptions' 里	:map _l :!ls | more^M
^V|	总可以，Vim 和 Vi 都行	:map _l :!ls ^V| more^M

(这里 ^V 表示 CTRL-V；要输入一个 CTRL-V 你必须按键两次；在这里不能使用 <> 记法 "<C-V>")。

当你使用 'cpoptions' 的缺省设置时三种方式都可以正常工作。

当 'b' 出现在 'cpoptions' 中时，"\|" 会被认为是一个映射的结束，后面的是另一个命令。这是为了和 Vi 兼容，但是和其它命令比较时有点不合常理。

map_return map-return
当你的映射包含 Ex 命令时，你需要在其后放置行终结符才能让它执行。在这里推荐使用 <CR> （参考 <> ）。例如：

```
:map _ls :!ls -l %:S<CR>:echo "the end"<CR>
```

在插入或命令行模式中输入时要避免字符被映射，可以先输入一个 CTRL-V。在插入模式中如果 'paste' 选项被打开的话，映射也会被禁止。

map-error
注意 当遇到错误时（会导致一个错误信息或蜂鸣）剩下的映射将不会被执行。这是为了保持和 Vi 兼容。

注意 @zZtTfF[]rm'`"v 和 CTRL-X 命令的第二个字符（参数）不被映射。这样做是为了能够使用所有的命名寄存器和位置标记，即使同名的命令被映射时也是如此。

1.7 映 射 哪 些 键 map-which-keys

如果你要做一些映射，你得选择在 {lhs} 中要用哪些键。你应该避免使用 Vim 命令所使用的那些键。否则你将不能再使用这些命令了。下面是一些建议：

- 功能键 <F2>、<F3> 等；Shift 加功能键 <S-F1>、<S-F2> 等等。注意 <F1> 已经用作
  帮助命令。
- 带 Meta 的键（和 ALT 键一起按下）。取决于你的键盘，也可以用带重音的字符。
  :map-alt-keys
- 使用 '_' 或 ',' 字符然后加上任何其它的字符。"_" 和 "," 命令在 Vim 中是存在的（参考 _ 和 , ），但你也许永远不会用到它们。
- 使用和其它命令的同义的热键。例如：CTRL-P 和 CTRL-N。使用一个附加的字符可以定义更多的映射。
- <Leader> 定义的键加上一到多个其它键。尤其对脚本有用。 mapleader

参考文件 "index" 可以知道哪些键没有被使用，从而使映射不会覆盖任何内建的功能。也可使用 ":help {key}^D" 来找出是否一个键已经用于某些命令。（{key} 用于指定你要寻找的键，^D 是 CTRL-D）。

1.8 示 例 map-examples

以下是一些例子（照字面输入，如 "<CR>" 需要输入四个字符；为此 'cpoptions' 中不应出现 '<' 标志位）。

```
:map <F3>  o#include
:map <M-g> /foo<CR>cwbar<Esc>
:map _x    d/END/e<CR>
:map! qq   quadrillion questions
```

计数相乘

如果你在激活映射前输入计数，实际效果就像是该计数在 {lhs} (译者注：疑为 {rhs}) 之前输入一样。例如，对下面的映射：

```
:map <F4>  3w
```

输入 2<F4> 会得到 "23w"。不是移动 2 * 3 个单词，而是 23 个单词。
如果你希望得到计数相乘的效果，可使用表达式寄存器：

```
:map <F4>  @='3w'<CR>
```

引号之间的部分是待执行的表达式。 @=

1.9 使 用 映 射 map-typing

当你输入一个被映射序列的头部时 Vim 开始比较你的输入。如果匹配尚不完全，它会等待更多的字符输入直到可以确定是否匹配。例如：如果你映射了 map! "qq"，然后你输入的第一个 'q' 将不会显示在屏幕上，直到你输入另一个 'q' 或其它字符。如果打开了 'timeout' 选项 (这是缺省选项) Vim 仅会等待一秒钟 (或任何 'timeoutlen' 指定的时间)。之后，它假定 'q' 已经不会再被输入。如果你的输入很慢，或者你的系统很慢，复位 'timeout' 选项。这时，你可能还需要是否置位 'ttimeout' 选项。

map-precedence

缓冲区局部映射 (:map-<buffer> 所定义的) 优先于全局映射。如果缓冲区局部映射和全局映射完全相同，Vim 使用缓冲区局部映射。另外如果映射以 <nowait> 定义的话，Vim 立即使用完整出现的映射，即便有一个更长的映射使用相同的前缀。例如，给定以下的映射：

```
:map <buffer> <nowait> \a   :echo "Local \a"<CR>
:map                   \abc :echo "Global \abc"<CR>
```

键入 \a 后会立即使用那个缓冲区局部映射。Vim 不再等待更多的字符来判断用户是否想输入 \abc。

map-keys-fails

有若干情况键码可能不被识别：
- Vim 仅能读取部分的键码。通常仅仅是第一个字符。在某些 Unix 版本的 xterm 上有这种情况。
- 键码在已经映射的字符之后。举例来说，"<F1><F1>" 或 "g<F1>"。

其结果是在这种情况下键码不会被识别，所以映射失败。有两种方法可以避免此问题：

- 从 'cpoptions' 中删除 'K' 标志位。这会使 Vim 等待功能键的其余部分。
- 使用 <F1> 到 <F4> 时，实际产生的键码可能是 <xF1> 到 <xF4>。存在 <xF1> 到 <F1>，<xF2> 到 <F2> 的映射等，但是在映射的后一半的那些依然不会被识别。确认从 <F1> 到 <F4> 的键码是正确的：

```
:set <F1>=<type CTRL-V><type F1>
```

以四个字符输入 <F1>。"=" 号后面的部分必须输入实际的字符而不是字面的文本。
另一种解决方法是在映射中为第二个特殊键使用实际的键码：

```
:map <F1><Esc>OP :echo "yes"<CR>
```

不要输入一个真正的 <Esc>，总之 Vim 将识别键码并把它替换为 <F1>。

另一个问题可能是保持 ALT 或 Meta 的时候，终端在前面附加 ESC 而不是给第 8 位置位。见 :map-alt-keys 。

recursive_mapping

如果 {rhs} 中包括了 {lhs}，那么你定义了一个递归映射。当 {lhs} 被输入，它会被替换成 {rhs}。当遇到 {rhs} 中包含的 {lhs} 又会被替换成 {rhs}，依此类推。
这可用来使一个命令重复无数次。这种情况唯一的问题是出错是停止它的唯一方法。解决迷宫的宏会用到这个，去那里找找例子吧。有一个例外：如果 {rhs} 以 {lhs} 开始，第

一个字符不会被再次映射（这与 Vi 兼容）。
例如：

```
:map ab abcd
```

将执行 "a" 命令并且在文本中插入 "bcd"。{rhs} 中的 "ab" 不会被再次映射。

如果你要交换两个键的含义，应该使用 :noremap 命令。例如：

```
:noremap k j
:noremap j k
```

这会交换光标上移和光标下移命令。

如果使用普通 :map 命令，并且 'remap' 选项被打开，映射一直进行直到文本不再是某个 {lhs} 的一部分。例如，如果你用：

```
:map x y
:map y x
```

Vim 将把 x 替换成 y，并把 y 替换成 x，等等。这种情况会发生 'maxmapdepth' 次（缺省为 1000），然后 Vim 会给出错误信息 "recursive mapping"（递归映射）。

:map-undo

如果你在一个被映射的序列中包含了一个 undo 命令，将会把文本带回宏执行前的状态。这和原始的 Vi 是兼容的，只要被映射的序列仅包含一个 undo 命令（原始的 Vi 中被映射的序列有两个 undo 命令是无意义的，你会得到第一个 undo 之前的文本）。

1.10 映 射 ALT 键 :map-alt-keys

GUI 上，Vim 自己处理 Alt 键，所以用 ALT 键的映射应该总没有问题。但在终端上，Vim 得到字节的序列，它必须自己判断是不是按了 ALT 键。

如果终端支持 modifyOtherKeys 模式且打开，Vim 可以识别更多的键组合，见下 modifyOtherKeys 。

Vim 缺省假设按下 ALT 键等于置位输入字符的第 8 位。多数正常的终端如此工作，包括 xterm、aterm 和 rxvt。假如你的 <A-k> 映射不能工作，可能的原因是你的终端用在字符前加上 ESC 前缀的方法。但是你本来也可能在字符前输入 ESC，这时 Vim 就不知道到底发生了什么（只能检查字符间的延迟，但这并不可靠）。

在此文写作时，有些主流的终端，如 gnome-terminal 和 konsole，使用 ESC 前缀。没有办法让它们用置位第 8 位来代替。Xterm 缺省应该没有问题。Aterm 和 rxvt 启动时如果使用 "--meta8" 参数也可以如此。你也可以修改资源来达到目的：
"metaSendsEscape"、"eightBitInput" 和 "eightBitOutput"。

Linux 控制台上，可以用 "setmetamode" 命令切换此行为。记住不使用 ESC 前缀可能和其它程序发生冲突。确保你的 bash 把 "convert-meta" 选项设为 "on"，确保 Meta 键盘绑定仍然工作（这是缺省的 readline 行为，除非你的系统配置专门作了改变）。为此，你需要加入这行：

```
set convert-meta on
```

到你的 ~/.inputrc 文件。如果你新建此文件，可能想把：

```
$include /etc/inputrc
```

放在第一行，如果此文件在你的系统中存在的话。这样可以保持全局的选项设置。不过，这可能会使 umlaut 这样的特殊字符的输入有问题。这时，输入字符前用 CTRL-V 前导。

要知道有报告说 convert-meta 使得 UTF-8 locale 的使用有问题。在 xterm 这样的终端里，可以在 "Main Options" 菜单里随时切换 "metaSendsEscape" 资源，或者终端上按 Ctrl-LeftClick 也可以；如果你需要给 Vim 之外的其它应用程序发送 ESC，这是最后应急的方法。

1.11 modifyOtherKeys 模式映射 modifyOtherKeys

Xterm 和一些其它的终端可以进入一种模式，其中带修饰符的键可以使用特殊的转义代码发送。Vim 识别这些代码，所以可以区别 CTRL-H 和退格键，尽管退格键也发送字符 8。还有许许多特殊键。

xterm 在内建的 termcap 项目中打开了 modifyOtherKeys。如果没有用，可以在 vimrc 中打开 modifyOtherKeys:

```
let &t_TI = "\<Esc>[>4;2m"
let &t_TE = "\<Esc>[>4;m"
```

万一 modifyOtherKeys 模式造成问题，可以关闭之:

```
let &t_TI = ""
let &t_TE = ""
```

这不会立即生效。要避免重启 Vim 但要使之生效，执行外壳命令，如: !ls 或把它们加入 vimrc .

modifyOtherKeys 打开时可以映射 <C-[> 和 <C-S-{>:

```
imap <C-[> [[[
imap <C-S-{> {{{
```

如果没有 modifyOtherKeys，<C-[> 和 <C-S-{> 与 Esc 无法区分。

已知的一个副作用是在插入模式中在 CTRL-V 键后插入的是原始转义序列。这可用来检查是否打开了 modifyOtherKeys。插入模式下按 CTRL-V CTRL-V，如果得到一个字节那么 modifyOtherKeys 是关闭的，如果得到 <1b>27;5;118~ 那么它是打开的。

'esckeys' 选项如果关闭，在插入模式会屏蔽 modifyOtherKeys，以免每个带修饰符的键都会退出插入模式。

1.12 映射操作符 :map-operator

操作符应用于 {motion} 命令之前。要定义你自己的操作符，你需要先创建映射来设置 'operatorfunc' 选项，然后调用 g@ 操作符。这样用户输入 {motion} 命令后，会调用指定的函数。

g@ E774 E775

g@{motion} 调用 'operatorfunc' 选项设置的函数。
'[位置标记定位在 {motino} 跨越的文本的开始处，而 ']
位置标记在此文本的结束处。
函数调用时，带一个字符串参数:

参数		如果
"line"	{motion}	本是 linewise
"char"	{motion}	本是 characterwise
"block"	{motion}	本是 blockwise-visual

不过，"block" 很少出现，因为它只能来自可视模式，那里 "g@" 不是很有用。

{仅当编译时加入 +eval 特性才有效}

这里是一例，<F4> 来计算空格数目：

```
nmap <silent> <F4> :set opfunc=CountSpaces<CR>g@
vmap <silent> <F4> :<C-U>call CountSpaces(visualmode(), 1)<CR>

function! CountSpaces(type, ...)
  let sel_save = &selection
  let &selection = "inclusive"
  let reg_save = @@

  if a:0  " 在可视模式里调用，使用 gv 命令。
    silent exe "normal! gvy"
  elseif a:type == 'line'
    silent exe "normal! '[V']y"
  else
    silent exe "normal! `[v`]y"
  endif

  echomsg strlen(substitute(@@, '[^ ]', '', 'g'))

  let &selection = sel_save
  let @@ = reg_save
endfunction
```

注意 'selection' 选项暂时设为 "inclusive"，以便可视模式下用 '[到 '] 位置标记可以抽出正确的文本。

也要 注意 这里为可视模式提供了专用的映射。它先删除 ":" 在可视模式里插入的 "'<,'>" 范围，然后调用函数，调用时使用了 visualmode() 和一个额外的参数。

---

2. 缩写 abbreviations Abbreviations

缩写在插入，替换和命令行模式中使用。如果你输入一个是缩写的单词，它会被替换成所表示的东西。这可以在经常输入的长单词时节省键击。并且能用它来自动更正经常犯的拼写错误。例如：

```
:iab ms Microsoft
:iab tihs this
```

有三种类型的缩写：

full-id "full-id" 类型完全由关键字字符组成（字母和 'iskeyword' 选项的字符）。这是最普通的缩写。

例如："foo"，"g3"，"-1"

end-id "end-id" 类型以一个关键字字符结尾，但所有其它字符都不是关键字字符。

例如："#i"，"..f"，"$/7"

non-id "non-id" 类型以一个非关键字字符结尾，其它字符可以是任意类型，除了空

格和制表。{Vi 不支持这种类型}

例如: "def#", "4/7$"

不能被缩写的字串例子: "a.b", "#def", "a b", "_$r"

仅当你输入一个非关键字字符时缩写才会被识别，这也包括用 <Esc> 退出插入模式或用 <CR> 结束一个命令的情形。结束缩写的非关键字字符被插入到缩写的扩展后面。一个例外是字符 <C-]>，它用来扩展一个缩写，但不插入任何附加字符。

例如:

```
:ab hh      hello
        "hh<Space>" 被扩展为 "hello<Space>"
        "hh<C-]>" 被扩展为 "hello"
```

光标前的字符必需和缩写匹配。每种类型还有附加规则:

full-id　匹配的前面是一个非关键字字符，或者是在行或插入的开始。例外: 当缩写仅有一个字符时，如果它前面有一个非关键字字符则不会被识别，除非那是空格和制表。不过，命令行上忽略 "'<,'>" (或其它标记) 部分，就像命令行从那之后开始那样。

end-id　匹配的前面是一个关键字字符，或者空格或制表，或者行或插入的开始。

non-id　匹配的前面是一个空格、制表或者行或插入的开始。

例如: ({CURSOR} 是你输入一个非关键字字符的地方)

```
:ab foo   four old otters
            " foo{CURSOR}"    被扩展为 " four old otters"
            " foobar{CURSOR}" 不被扩展
            "barfoo{CURSOR}"  不被扩展

:ab #i #include
            "#i{CURSOR}"      被扩展为 "#include"
            ">#i{CURSOR}"     不被扩展

:ab ;; <endofline>
            "test;;"          不被扩展
            "test ;;"         被扩展为 "test <endofline>"
```

要在插入模式中避免缩写: 在会启动缩写的的字符之前面输入 CTRL-V，例如 CTRL-V <Space>。或者先输入缩写的部分，以 <Esc> 退出插入模式，再用 'a' 重新进入插入模式并输入剩下的部分。

要在命令行模式中避免缩写: 在缩写的某处输入 CTRL-V 两次来避免它被替换。不然，一个普通字符前面的 CTRL-V 通常会被忽略。

缩写进行之后移动光标是可能的:

```
:iab if if ()<Left>
```

如果 'cpoptions' 里面包含 '<' 标志位时，这不能正常工作。 <>

你甚至可以做更复杂的事情。例如，要消灭一个缩写后面输入的空格:

```
func Eatchar(pat)
   let c = nr2char(getchar(0))
```

```
    return (c =~ a:pat) ? '' : c
  endfunc
  iabbr <silent> if if ()<Left><C-R>=Eatchar('\s')<CR>
```

没有缺省的缩写。

缩写永远不会递归。你可以设置 ":ab f f-o-o" 而不会有任何问题。但是缩写能被映射。{一些版本的 Vi 支持递归缩写，这毫无道理}

'paste' 选项打开时，缩写被禁止。

:abbreviate-local :abbreviate-<buffer>

和映射一样，缩写可以被局部于一个缓冲区之内。这经常用于 filetype-plugin 文件。一个 C 插件文件的例子：

```
:abb <buffer> FF  for (i = 0; i < ; ++i)
```

:ab :abbreviate

:ab[breviate] 列出所有的缩写。第一栏中的字符表示该缩写作用的模式：'i' 指插入模式，'c' 指命令行模式，'!' 指两种模式都有。这和映射的相同，参看 map-listing 。

:abbreviate-verbose

如果 'verbose' 非零，缩写列出的同时显示它最近定义的位置。例如：

```
:verbose abbreviate
!  teh           the
        Last set from /home/abcd/vim/abbr.vim
```

:verbose-cmd 说明详情。

:ab[breviate] {lhs} 列出以 {lhs} 开头的缩写

:ab[breviate] [<expr>] [<buffer>] {lhs} {rhs}
增加一个从 {lhs} 到 {rhs} 的缩写。如果 {lhs} 已经存在则它会被替换成新的 {rhs}。{rhs} 可包含空格。
:map-<expr> 说明可选的 <expr> 参数。
:map-<buffer> 说明可选的 <buffer> 参数。

:una :unabbreviate

:una[bbreviate] [<buffer>] {lhs}
从列表中删除 {lhs} 的缩写。如果找不到，删除 {rhs} 匹配这里的 {lhs} 参数的缩写。这是为了方便你删除扩展后的缩写。要避免扩展，插入 CTRL-V (记住输入两次)。

:norea :noreabbrev

:norea[bbrev] [<expr>] [<buffer>] [lhs] [rhs]
与 ":ab" 一样，但 {rhs} 不进行重映射。

:ca :cab :cabbrev

:ca[bbrev] [<expr>] [<buffer>] [lhs] [rhs]
与 ":ab" 一样，但仅在命令行模式中使用。

:cuna :cunabbrev

:cuna[bbrev] {lhs} 与 ":una" 一样，但仅在命令行模式中使用。

:cnorea :cnoreabbrev

:cnorea[bbrev] [<expr>] [<buffer>] [lhs] [rhs]
与 ":ab" 一样，但仅在命令行模式中使用并且 {rhs} 不进行重映射。

:ia :iabbrev

:ia[bbrev] [<expr>] [<buffer>] [lhs] [rhs]
与 ":ab" 一样，但仅在插入模式中使用。

:iuna :iunabbrev

:iuna[bbrev] {lhs} 与 ":una" 一样，但仅在插入模式中使用。

:inorea :inoreabbrev

:inorea[bbrev] [<expr>] [<buffer>] [lhs] [rhs]
与 ":ab" 一样，但仅在插入模式中使用并且 {rhs} 不进行重映射。

:abc :abclear

:abc[lear] [<buffer>] 删除所有的缩写。

:iabc :iabclear

:iabc[lear] [<buffer>] 为插入模式删除所有的缩写。

:cabc :cabclear

:cabc[lear] [<buffer>] 为命令行模式删除所有的缩写。

using_CTRL-V

在一个缩写的 {rhs} 中使用特殊字符是可能的。CTRL-V 可以用来避免多数不可显示字符的特殊含义。需要输入多少个 CTRL-V 取决于你如何输入缩写。此处讨论同样适用于映射。这里使用一个例子说明。

假设你需要把 "esc" 缩写为输入一个 <Esc> 字符。当你在 Vim 中输入 ":ab" 命令，你必需这样输入：(这里 ^V 是一个 CTRL-V 并且 ^[is <Esc>)

你输入： ab esc ^V^V^V^V^V^[

所有的键盘输入都经过 ^V 引用解释，所以第一个，第三个，和第五个 ^V 字符只是为了把第二个、第四个 ^V 和 ^[输入到命令行里。

你看到： ab esc ^V^V^[

命令行里在 ^[之前包含两个实际的 ^V。如果你采用这种方法，这是该行在你的 .exrc 文件应该出现的样子。第一个 ^V 作为引用第二个 ^V 的字符：这是因为 :ab 命令使用 ^V 作为它自己的引用字符，以便你能在缩写中包含被引用的空白字符或 | 字符。:ab 命令对 ^[字符并不做特殊的事情，所以它不需要被引用。(尽管引用也没有害处；因而输入 7 个 [8 个不行！] ^V 也会工作。)

被保存为： esc ^V^[

解析后，该缩写的简短形式 ("esc") 和扩展形式 (两字符 "^V^[") 被保存在缩写表中。如果输入不带参数的 :ab 命令，这是该缩写被显示的形式。

然后当用户输入单词 "esc" 而扩展该缩写时，扩展形式服从和一般键盘输入同样形式的 ^V 解释。所以 ^V 保护 ^[字符不被解释为 "退出插入模式" 的字符，而把 ^[插入到文本里。

扩展为：^[

[Steve Kirkendall 提供示例]

---

3. 局部映射和函数 script-local

当使用多个 Vim 脚本文件时，一个脚本和另一个脚本使用同样名字的映射和函数是危险的。为了避免这种情况，它们可以局部在脚本。

<SID> <SNR> E81

字串 "<SID>" 能用于映射或菜单。这要求 'cpoptions' 中没有 '<' 标志位。

当执行映射命令时，Vim 将把 "<SID>" 替换成特殊键码 <SNR>，后跟一个每个脚本唯一的数字编号，和一个下划线。例如：

```
:map <SID>Add
```

会定义一个映射 "<SNR>23_Add"。

当在一个脚本中定义一个函数的时候，可以在名字的前面用一个 "s:" 来使它局部于脚本中。但当一个映射（译者注：似应为函数）从脚本外面被执行时，它不知道该函数在哪个脚本中被定义。为了避免这种情况，使用 "<SID>" 来代替 "s:"。映射也做同样的变换。这使得在映射里可以定义一个函数调用。

当一个局部函数被执行时，它在定义脚本的上下文中运行。这意味着，它定义的新函数和映射也可以使用 "s:" 或 "<SID>"，并且使用和函数本身定义时相同的唯一数字编号。此外，也能用 "s:var" 脚本局部变量。

当执行一个自动命令或一个用户命令时，它将在定义脚本的上下文中运行。这使得该命令可以调用一个局部函数或者使用一个局部映射。

除此以外，在脚本上下文之外使用 "<SID>" 是错误的。

如果你需要在一个复杂的脚本中取得脚本的数字编号，使用此函数：

```
function s:SID()
  return matchstr(expand('<sfile>'), '<SNR>\zs\d\+\ze_SID$')
endfun
```

列出函数和映射时会显示 "<SNR>"。可以用来它们在哪里被定义。

命令 :scriptnames 可以用来查看哪些脚本已经被读入以及它们的 <SNR> 数字编号。

这些都是 {仅当编译时加入 +eval 特性才有效}。

---

4. 用户定义的命令 user-commands

可以定义你自己的 Ex 命令。用户自定义命令可以和内建命令一样运行（它可以有范围或参数，参数可以是自动补全的文件名或缓冲区名，等等），除了当该命令执行时，它会被转换成一个普通的 Ex 命令然后再被执行以外。

对于初学者来说：参考用户手册中的 40.2 。

E183 E841 user-cmd-ambiguous

所有用户定义的命令都必须以大写字母开头，来避免与内建命令的冲突。以下内建命令是例外：

```
:Next
:X
```

它们不能用于用户自定义命令。":Print" 也是已定义的命令，但已废弃，可以被覆盖。

用户命令的其它字符可以是大写字母，小写字母或数字。当使用数字时，小心会和其它以数字作为参数的命令混淆。例如，命令 ":Cc2" 可能是不带参数的用户命令 ":Cc2"，也可能是参数为 "2" 的命令 "Cc"。建议在命令名和参数之间放置一个空格来避免这些问题。

当使用一个用户定义的命令时，该命令可以缩写。但是，如果缩写不唯一，会发生错误。此外，内建命令总是优先执行。

例如：

```
:command Rename ..。
:command Renumber ..。
:Rena                          " 意味着 "Rename"
:Renu                          " 意味着 "Renumber"
:Ren                           " 错误 - 有二义性
:command Paste ..。
:P                             " 内建的 :Print
```

建议在脚本中使用用户自定义命令的全名。

:com[mand] :com :command

列出所有用户自定义命令。在列出命令时，首栏的字符表示

- ! 命令有 -bang 属性
- " 命令有 -register 属性
- | 命令有 -bar 属性
- b 命令局部于当前缓冲区

(下面给出属性的详细描述)

此列表可用 :filter 命令过滤，例如，要列出所有名字带 "Pyth" 的命令：

```
filter Pyth command
```

:com[mand] {cmd} 列出以 {cmd} 开头的用户命令

:command-verbose

如果 'verbose' 非零，命令列出的同时显示它最近定义的位置。例如：

```
:verbose command TOhtml
    Name        Args Range Complete  Definition
    TOhtml      0    %               :call Convert2HTML(<line1>, <line2>)
        Last set from /usr/share/vim/vim-7.0/plugin/tohtml.vim
```

:verbose-cmd 介绍详情。

E174 E182

:com[mand][!] [{attr}...] {cmd} {rep}

定义一个用户命令。命令的名字是 {cmd}，而替换的文本是 {rep}。该命令的属性（参考下面）是 {attr}。如果该命令已存在，报错，除非已经指定了一个 !，这种情况下命令被重定义。
有一个特例：再次执行脚本时，该脚本中之前定义的命令会悄然地被替代。

:delc[ommand] {cmd}　　:delc　:delcommand　E184
删除用户定义命令 {cmd}。

:comc[lear]　　:comc　:comclear
删除所有用户定义命令。

命令属性

Vim 和任何其它 Ex 命令一样对待用户自定义命令。它能有参数，也可以指定范围。参数可以进行文件名，缓冲区等补全。具体的工作方式取决于命令的属性，属性在命令被定义时被指定。

属性可分四大类：参数处理、补全行为、范围处理和特殊情况。下面分类描述之。

参数处理

E175　E176　:command-nargs

缺省时，用户自定义命令不接受参数（如果使用了任何参数会报错）。但通过使用 -nargs 属性可以允许命令接受参数。有效的值为：

-nargs=0	不允许有参数（缺省情况）
-nargs=1	要求一个参数，包括空格
-nargs=*	允许任何数目的参数（0，1 或更多），以空格分隔
-nargs=?	允许 0 或 1 个参数
-nargs=+	必需给出参数，但是数目任意

此上下文中，（未转义的）空格或制表用来分隔参数，除非指定只有一个参数，此时空格认为是参数的一部分。

注意 参数被作为文本使用，不是表达式。特别是，"s:var" 会使用定义命令的脚本的局部变量，不是执行时的！例如：

```
script1.vim:
    :let s:error = "None"
    :command -nargs=1 Error echoerr <args>
script2.vim:
    :source script1.vim
    :let s:error = "Wrong!"
    :Error s:error
```

执行 script2.vim 会回显 "None"，不是你想要的！解决方法可以通过调用函数实现。

自动补全行为

:command-completion　E179　E180　E181
:command-complete

缺省时，用户定义命令的参数不进行自动补全。但是，通过指定以下的一个或多个属性后，参数可以进行自动补全：

-complete=arglist	参数列表中的文件名

-complete=augroup	自动命令组
-complete=buffer	缓冲区名
-complete=behave	:behave 子选项
-complete=color	颜色方案
-complete=command	Ex 命令（及其参数）
-complete=compiler	编译器
-complete=cscope	:cscope 子选项
-complete=dir	目录名
-complete=environment	环境变量名
-complete=event	自动命令事件
-complete=expression	Vim 表达式
-complete=file	文件和目录名
-complete=file_in_path	'path' 中的文件和目录名
-complete=filetype	文件类型名 'filetype'
-complete=function	函数名
-complete=help	帮助主题
-complete=highlight	高亮组
-complete=history	:history 子选项
-complete=locale	locale 名（和 locale -a 给出的相同）
-complete=mapclear	缓冲区参数
-complete=mapping	映射名
-complete=menu	菜单
-complete=messages	:messages 子选项
-complete=option	选项
-complete=packadd	可选包 pack-add 名
-complete=shellcmd	外壳命令
-complete=sign	:sign 子选项
-complete=syntax	语法文件名 'syntax'
-complete=syntime	:syntime 子选项
-complete=tag	标签
-complete=tag_listfiles	标签，但敲入 CTRL-D 时显示文件名
-complete=user	用户名
-complete=var	用户变量
-complete=custom,{func}	用户定制的自动补全，通过 {func} 来定义
-complete=customlist,{func}	用户定制的自动补全，通过 {func} 来定义

备注：部分补全方法可能会扩展环境变量。

用户定制的自动补全

:command-completion-custom
:command-completion-customlist E467 E468

通过 "custom,{func}" 或 "customlist,{func}" 自动补全参数可以定义定制的自动补全方案。其中 {func} 是有如下声明的函数：

```
:function {func}(ArgLead, CmdLine, CursorPos)
```

该函数不需要使用所有的这些参数，它应该提供自动补全候选作为返回值，

对于 "custom" 参数，函数应该返回字符串，每行一个候选，用换行符分隔。

对于 "customlist" 参数，函数应该返回 Vim 列表形式的补全候选。忽略列表里的非字符串项目。

该函数的参数是：

ArgLead	当前自动补全的前导参数
CmdLine	完整的命令行
CursorPos	里面的光标位置（字节位置）

该函数可能要根据这些来判别上下文。对 "custom" 参数，它无须用 ArgLead（里面的隐式规则）来过滤候选。在函数返回时 Vim 将用它的正则表达式引擎来进行过滤，这种方式在大多数情况下效率更高。对于 "customlist" 参数，Vim 不会过滤返回的补全候选，用户提供的函数应该自己过滤候选。

以下的例子为列出 Finger 命令的用户名

```
:com -complete=custom,ListUsers -nargs=1 Finger !finger <args>
:fun ListUsers(A,L,P)
:    return system("cut -d: -f1 /etc/passwd")
:endfun
```

下例从 'path' 选项指定的目录补全文件名：

```
:com -nargs=1 -bang -complete=customlist,EditFileComplete
                    \ EditFile edit<bang> <args>
:fun EditFileComplete(A,L,P)
:    return split(globpath(&path, a:A), "\n")
:endfun
```

此例不适用于带空格的文件名！

范围处理

E177 E178 :command-range :command-count

缺省时，用户定义的命令不接受一个行号范围。不过，可以使命令接受一个范围（-range 属性），或者接受一个任意的数量值，该数量可以出现在指定行号的位置（-range=N，类似于 :split 命令的风格），也可以来自一个 "count" 参数（-count=N，类似于 :Next 命令的风格）。此时计数可以用 <count> 从参数里得到。

可能的属性有：

-range	允许使用范围，缺省为当前行
-range=%	允许使用范围，缺省是整个文件（1,$）
-range=N	出现在行号位置的一个数量（缺省是 N）（类似于 :split ）；允许行号为零。
-count=N	出现在行号位置或者作为首个参数的一个数量（缺省是 N）（类似于 :Next ）。
-count	等价于 -count=0

注意 -range=N 和 -count=N 是互斥的，只应该指定其中的一个。

:command-addr

范围中的特殊字符如 .、$ 或 % 缺省对应当前行、末行和整个缓冲区，但可使之对应参数列表、（已载入的）缓冲区、窗口或标签页。

可能值有（第二列是列表使用的短名）：

-addr=lines		行的范围（这是缺省）
-addr=arguments	arg	参数的范围
-addr=buffers	buf	缓冲区的范围（也包括未载入的缓冲区）
-addr=loaded_buffers	load	载入缓冲区的范围
-addr=windows	win	窗口的范围

-addr=tabs	tab	标签页的范围
-addr=quickfix	qf	quickfix 项的范围
-addr=other	?	其它范围；可用 "."、"$" 和 "%" 还有 "lines" (这是 -count 的缺省)

特殊情况

:command-bang :command-bar
:command-register :command-buffer

有如下特殊情况：

-bang	这些命令可以使用一个 ! 修饰符 (和 :q 或 :w 类似)
-bar	这些命令可以跟随一个 "|" 和其它命令。那么命令参数中就不允许有 "|" 。用一个 " 可以开始一个注释。
-register	给这些命令的第一个参数可以是一个可选的寄存器名 (和 :del，:put，:yank 类似)。
-buffer	这些命令仅在当前缓冲区里有效。

-count 和 -register 属性的情况，如果提供了可选的参数，它会被从参数列表中删除，并且和替换文本分别处理。

注意 这些参数可以简写，但这是已淘汰的功能，新脚本里请用全名。

替换文本

用户自定义命令的替换文本扫描使用 <...> 记法的特殊转义序列。命令行输入的值中，转义序列被替换，其它文本不变。最终字符串被作为 Ex 命令来执行。要避免替换，使用 <lt> 代替初始的 <。这样，要按本义包含 "<bang>"，请使用 "<lt>bang>"。

有效的转义序列有

<line1>
<line1> 命令处理范围的开始行。

<line2>
<line2> 命令处理范围的末尾行。

<range>
<range> 命令处理范围的项目数：0、1 或 2

<count>
<count> 提供的数量 (在 '-range' 和 '-count' 属性中描述)。

<bang>
<bang> (参考 '-bang' 属性) 如果命令执行时带了 ! 修饰符，扩展为 !，否则什么也不扩展。

<mods>
<mods> 如果指定，命令修饰符。否则不扩展。支持以下修饰符
:aboveleft 、 :belowright 、 :botright 、 :browse 、
:confirm 、 :hide 、 :keepalt 、 :keepjumps 、
:keepmarks 、 :keeppatterns 、 :leftabove 、 :lockmarks 、
:noswapfile :rightbelow 、 :silent 、 :tab 、 :topleft 、
:verbose 和 :vertical 。
注意 现在还不支持： :noautocmd 、 :sandbox 和 :unsilent 。
示例：

```
command! -nargs=+ -complete=file MyEdit
            \ for f in expand(<q-args>, 0, 1) |
            \ exe '<mods> split ' . f |
            \ endfor
```

```
function! SpecialEdit(files, mods)
    for f in expand(a:files, 0, 1)
        exe a:mods . ' split ' . f
    endfor
endfunction
command! -nargs=+ -complete=file Sedit
            \ call SpecialEdit(<q-args>, <q-mods>)
```

<reg> <register>

<reg> （参考 '-register' 属性）如果命令行上指定，可选的寄存器名。否则什么也不扩展。<register> 是它的一个同义词。

<args>

<args> 命令的参数，和实际提供的完全相同（但正如上面提到过的，数量或寄存器会消耗若干参数，它们不再是 <args> 的一部分）。

<lt> 一个单独的 '<'（小于号）字符。在扩展中可用于使以上转义序列按本义出现。例如，要获得 <bang>，使用 <lt>bang>。

<q-args>

如果一个转义序列的最前两个字符是 "q-"（例如，<q-args>）那么该值用引号括起，使之在表达式里使用时成为合法的值。这种方式把参数当做单个值。如果没有参数，<q-args> 是空字符串。

<f-args>

要允许命令把参数传送给用户定义的函数，有一种特殊的形式 <f-args>（"function args"，函数参数）。它在空格和制表处分割命令行参数，每个参数分别用引号括起，然后把 <f-args> 序列替换为括起参数用逗号分隔的列表。参考下面的 Mycmd 示例。没有参数时，<f-args> 被删除。

要在 <f-args> 的参数中嵌入空白字符，在前面加上反斜杠。<f-args> 把每对反斜杠（\\）用单个反斜杠替代。反斜杠后如跟非空白或反斜杠字符，保持不变。总览如下：

```
命令              <f-args>
XX ab             'ab'
XX a\b            'a\b'
XX a\ b           'a b'
XX a\  b          'a ', 'b'
XX a\\b           'a\b'
XX a\\ b          'a\', 'b'
XX a\\\b          'a\\b'
XX a\\\ b         'a\ b'
XX a\\\\b         'a\\b'
XX a\\\\ b        'a\\', 'b'
```

示例

```
" 删除从这里到末尾的所有东西
:com Ddel +,$d

" 把当前缓冲区改名
:com -nargs=1 -bang -complete=file Ren f <args>|w<bang>

" 用一个文件的内容来替换某个范围内的内容
" (请用一行输入本命令)
:com -range -nargs=1 -complete=file
      Replace <line1>-pu_|<line1>,<line2>d|r <args>|<line1>d
```

```
" 计算范围内的行数
:com! -range -nargs=0 Lines  echo <line2> - <line1> + 1 "lines"

" 调用一个用户函数 (<f-args> 的示例)
:com -nargs=* Mycmd call Myfunc(<f-args>)
```

当执行:

```
:Mycmd arg1 arg2
```

时，它将调用:

```
:call Myfunc("arg1","arg2")
```

```
:" 一个更实用的例子
:function Allargs(command)
:    let i = 0
:    while i < argc()
:       if filereadable(argv(i))
:        execute "e " . argv(i)
:         execute a:command
:      endif
:      let i = i + 1
:   endwhile
:endfunction
:command -nargs=+ -complete=command Allargs call Allargs(<q-args>)
```

命令 Allargs 接受任意 Vim 命令作为参数并在参数列表里的所有文件上执行。使用示例(注意使用 "e" 标志位来忽略错误，以及用 "update" 命令来刷新修改过的缓冲区):

```
:Allargs %s/foo/bar/ge|update
```

它将调用:

```
:call Allargs("%s/foo/bar/ge|update")
```

在脚本里定义用户命令时，它可以调用局部于脚本中的函数和使用局部于脚本的映射。用户调用用户命令时，该命令将运行在定义它的脚本的上下文里，如果一个命令中使用了 <SID> ，这一点很重要。

vim:tw=78:ts=8:noet:ft=help:norl:

tagsrch.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

标签和特殊搜索 tags-and-searches

用户手册 29.1 有这部分内容的介绍。

1. 跳转到标签 tag-commands
2. 标签栈 tag-stack
3. 标签匹配表 tag-matchlist
4. 标签细节 tag-details
5. 标签文件格式 tags-file-format
6. 头文件搜索 include-search
7. 使用 'tagfunc' tag-function

---

1. 跳转到标签 tag-commands

tag tags
标签是出现在 "tags" 文件的一个标识符。它是一种能够跳转的标记。例如，在 C 程序里，每个函数名都可以是一个标签。在使用标签文件之前，"tags" 文件由 ctags 这样的程序生成。

使用 ":tag" 命令移动光标到标签上。使用 CTRL-] 命令，光标所在的关键字就可用作标签。如果光标不在关键字上，则使用最右侧的一个关键字。

":tag" 命令对 C 程序十分有效。如果你看到一个函数的调用而想知道该函数的作用，把光标移动到函数名上，并按 CTRL-]。它会把你带到该函数的定义上。简单的，按 CTRL-T 就可以回去。同时，也请阅读下面关于标签栈的描述。

:ta :tag E426 E429

:[count]ta[g][!] {name}
根据 tags 文件的信息，跳转到 {name} 的定义上。
把 {name} 放在堆栈上。[!] 见 tag-! 。
{name} 可以是正规表达式。见 tag-regexp 。
如果有 {name} 的多个匹配，则跳转到它们中的第 [count] 个。如果 [count] 被省略，则跳转到第一个。要跳转到其它匹配的标签，参见 tag-matchlist 。

g<LeftMouse> g<LeftMouse>
<C-LeftMouse> <C-LeftMouse> CTRL-]
CTRL-]
跳转到光标所在的关键字的定义。和 ":tag {name}" 相同，只不过 {name} 是光标之下或者之后的关键字。
如果有 {name} 的多个匹配，则跳转到它们中的第 [count] 个。如果没有 [count]，跳转到第一个。 要跳转到其它匹配的标签，参见 tag-matchlist 。

v_CTRL-]
{Visual}CTRL-]
和 ":tag {name}" 相同，其中 {name} 是高亮的文本。

telnet-CTRL-]

CTRL-] 是缺省的 telnet 转义键。在你按 CTRL-] 进行标签跳转的时候，你会得到 telnet 的提示符。许多版本的 telnet 允许你改变或者屏蔽缺省的转义键。见 telnet 的手册页。你可以用 'telnet -E {Hostname}' 来屏蔽转义键，或者用 'telnet -e {EscapeCharacter} {Hostname}' 来指定别的转义键。如果可能，用 "ssh" 来代替 "telnet" 也可解决这个问题。

tag-priority

如果有标签的多个匹配，使用以下的优先级：
1. "FSC" 当前文件的完整的匹配静态标签。
2. "F C" 当前文件的完整匹配全局标签。
3. "F " 其他文件的完整匹配全局标签。
4. "FS " 其他文件的完整匹配静态标签。
5. " SC" 当前文件的忽略大小写的匹配静态标签。
6. " C" 当前文件的忽略大小写匹配全局标签。
7. "   " 其他文件的忽略大小写匹配全局标签。
8. " S " 其他文件的忽略大小写匹配静态标签。

注意 当前文件改变时，优先顺序多数情况下不会改变，以免使 ":tnext" 的使用引起混淆。在使用 ":tag {name}" 时，优先顺序才会改变。

以下情况下，":tag" 命令不会查找忽略大小写的匹配：
- 'tagcase' 为 "followic" 且 'ignorecase' 选项关闭
- 'tagcase' 为 "followscs" 且 'ignorecase' 选项关闭且 'smartcase' 选项关闭或模式包含大写字符。(译者注：原文如此，似不确)
- 'tagcase' 为 "match"
- 'tagcase' 为 "smart" 且模式包含大写字符。

以下情况下，查找忽略大小写的匹配：
- 使用模式 (以 "/" 开始)
- ":tselect"
- 'tagcase' 为 "followic" 且 'ignorecase' 打开
- 'tagcase' 为 "followscs" 且 'ignorecase' 打开或 'smartcase' 选项打开且模式不包含大写字符 (译者注：原文如此，似不确)
- 'tagcase' 为 "ignore"
- 'tagcase' 为 "smart" 且模式不包含大写字符

注意 使用忽略大小写查找就无法应用 tags 文件的二分法查找，从而减慢了查找速度。这可以用大小写合并排序的标签文件来解决。具体解释见 'tagbsearch' 选项。

---

2. 标签栈 tag-stack tagstack E425

标签栈记住你跳转过的标签，还有从哪里跳转到这些标签。只有在设置 'tagstack' 选项的时候，标签才被压到堆栈里。

g<RightMouse> g<RightMouse>
<C-RightMouse> <C-RightMouse> CTRL-T
CTRL-T 跳到标签栈上第 [count] 个较早的项目 (缺省为 1)。

:po :pop E555 E556
:[count]po[p][!] 跳到标签栈上第 [count] 个较早的项目 (缺省为 1)。
[!] 见 tag-! 。

:[count]ta[g][!]　　跳到标签栈上第 [count] 个较新的项目（缺省为 1）。[!] 见 tag-! 。

:tags　　显示标签栈的内容。激活的项目以 '>' 标明。

":tags" 的输出结果类似于:

```
  # TO tag       FROM line in file/text
  1  1 main              1  harddisk2:text/vim/test
> 2  2 FuncA            58  i = FuncA(10);
  3  1 FuncC           357  harddisk2:text/vim/src/amiga.c
```

该列表显示你跳转到的标签和跳转前光标的位置。较早的项目在顶部，较新的在底部。

'>' 指向活动的项目。这是下一个 ":tag" 命令将使用的标签。CTRL-T 和 ":pop" 使用活动项目之上的那项的位置。

"TO" 之下的那列显示匹配表的当前匹配的数目。注意 这不会因为使用 ":pop" 或者 ":tag" 而改变。

记住文件名和行号可以使你回到 tag 命令之前的位置。即使你删除或者插入行，行号也会保持正确，除非你用别的程序（比如，另外一个 Vim 的实例）做这些事。

对于当前文件，"file/text" 列显示该位置的文本。缩进被删除，长行也被截短，以便能放进窗口里显示。

有若干命令可以让你跳到之前用过的标签。一些例子包括:

":pop" 或 CTRL-T	到上一个标签之前的位置
{count}CTRL-T	到第 {count} 个较早的标签之前的位置
":tag"	到更新的标签
":0tag"	到前一个使用过的标签

最明显的应用就是浏览程序的调用图。考虑下面的调用图:

```
main  --->  FuncA  --->  FuncC
      --->  FuncB
```

(解释: main 调用 FuncA 和 FuncB; FuncA 调用 FuncC)。
从 main 到 FuncA，你可以在调用 FuncA 的行上用 CTRL-] 。然后再用 CTRL-] 跳到 FuncC。如果你现在想跳回到 main，你只需要按 CTRL-T 两次。然后你可以用 CTRL-] 到 FuncB。

如果你使用 ":ta {name}" 或者 CTRL-] 命令，该标签被加在栈的当前位置。如果栈已满（最多可以容纳 20 项），最早的项目被删除，较早的项目则依次前移（索引号分别减 1）。如果最近使用的项目不在底部，它其后的项目都被删除。这意味着调用图上的旧的分支丢失了。在上面解释的命令执行以后，标签栈看起来像这样:

```
  # TO tag     FROM line in file/text
  1  1 main            1  harddisk2:text/vim/test
  2  1 FuncB          59  harddisk2:text/vim/src/main.c
```

gettagstack() 函数返回指定窗口的标签栈。 settagstack() 函数修改窗口的标签栈。

E73
如果你试图使用标签栈，而其中没有任何内容，你会得到一个错误信息。

---

3. 标签匹配表 tag-matchlist E427 E428

如果有多个匹配的标签，这些命令可以用来在它们之间跳转。 注意 这些命令不影响标签栈，它保留原来的项目。

:ts :tselect
:ts[elect][!] [name] 根据 tags 文件的信息，列出匹配 [name] 的标签。
如果 [name] 没有给出，使用标签栈上最后的标签名。
关于 [!] 见 tag-! 。
第一列的 '>' 指示列表的当前位置（如果有的话）。
[name] 可以是正规表达式，见 tag-regexp 。
列表使用的优先级顺序见 tag-priority 。
示例输出：

```
 nr pri kind tag               file
  1 F    f    mch_delay         os_amiga.c
                mch_delay(msec, ignoreinput)
> 2 F    f    mch_delay         os_msdos.c
                mch_delay(msec, ignoreinput)
  3 F    f    mch_delay         os_unix.c
                mch_delay(msec, ignoreinput)
Enter nr of choice (<CR> to abort):
```

"pri" 列见 tag-priority 。 注意 这决定于当前文件，因而 ":tselect xxx" 可能会输出不同的结果。
如果 tags 文件给出标签的类型。"kind" 列输出此信息。
"info" 列给出 tags 文件找到的信息。这决定于产生 tags 文件的程序。
如果列表很长，你可能会看到 more-prompt 。如果你已经看到要用的标签，按 'q' 并输入那个数字。

:sts :stselect
:sts[elect][!] [name] ":tselect[!] [name]" 并分割窗口以显示选择的标签。

g]
g] 类似于 CTRL-]，但使用 ":tselect" 而不是 ":tag"。

v_g]
{Visual}g] 和 "g]" 相同，但使用高亮文本作为标识符。

:tj :tjump
:tj[ump][!] [name] 类似于 ":tselect"，但如果只有一个匹配，直接跳转之。

:stj :stjump
:stj[ump][!] [name] ":tjump[!] [name]" 并分割窗口以显示选择的标签。

g_CTRL-]
g CTRL-]　　类似于 CTRL-]，但使用 ":tjump" 而不是 ":tag"。

v_g_CTRL-]
{Visual}g CTRL-]　　和 "g CTRL-]" 相同，但使用高亮文本作为标识符。

:tn　:tnext
:[count]tn[ext][!]　　跳转到向下 [count] 个匹配的标签（缺省为 1）。[!] 见 tag-! 。

:tp　:tprevious
:[count]tp[revious][!]　　跳转到向上 [count] 个匹配的标签（缺省为 1）。[!] 见 tag-! 。

:tN　:tNext
:[count]tN[ext][!]　　和 ":tprevious" 相同。

:tr　:trewind
:[count]tr[ewind][!]　　跳转到第一个匹配的标签。如果给出 [count]，跳到第 [count] 个匹配的标签。[!] 见 tag-! 。

:tf　:tfirst
:[count]tf[irst][!]　　和 ":trewind" 相同。

:tl　:tlast
:tl[ast][!]　　跳到最后匹配的标签。[!] 见 tag-! 。

:lt　:ltag
:lt[ag][!] [name]　　跳转到标签 [name] 上并在当前窗口的新位置列表中加入匹配的标签。
[name] 可以是正规表达式，见 tag-regexp 。如果 [name] 没有给出，使用标签栈上最后的标签名。查找标签行的搜索模式会加上 "\V" 前缀，转义所有的特殊字符 (very nomagic)。显示匹配标签的位置列表独立于标签栈。 [!] 见 tag-! 。

如果没有其他消息，Vim 显示跳到的是哪个匹配的标签，还有匹配标签的总数：

```
tag 1 of 3 or more
```

" or more" 用于说明 Vim 并没有尝试完所有的 tags 文件。多用几次 ":tnext" 或者用 ":tlast" 也许会找到更多。

如果因为有其他的消息使得你没看到这个消息，或者你想知道现在在哪里，以下这个命令会再次显示本消息（并跳转到最近一次的相同的标签）：

```
:0tn
```

tag-skip-file
如果找到一个匹配的标签，但是相应的文件却不存在，该匹配被跳过，而使用下一个匹配。Vim 报告这一点，提醒你文件的丢失。在匹配列表的结束处，会给出错误。

tag-preview
标签匹配表也可用在预览窗口。命令和上面一样，除了前面有个 "p"。
{仅当编译时加入 +quickfix 特性才有效}

:pts　:ptselect

:pts[elect][!] [name]　":tselect[!] [name]" 并在预览窗口显示新的标签。详情见 :ptag 。

:ptj[ump][!] [name]　":tjump[!] [name]" 并在预览窗口显示新的标签。详情见 :ptag 。 :ptj :ptjump

:[count]ptn[ext][!]　在预览窗口里 ":tnext"。见 :ptag 。 :ptn :ptnext

:[count]ptp[revious][!]　在预览窗口里 ":tprevious"。见 :ptag 。 :ptp :ptprevious

:[count]ptN[ext][!]　等价于 ":ptprevious"。 :ptN :ptNext

:[count]ptr[ewind][!]　在预览窗口中 ":trewind"。见 :ptag 。 :ptr :ptrewind

:[count]ptf[irst][!]　等价于 ":ptrewind"。 :ptf :ptfirst

:ptl[ast][!]　在预览窗口中见 ":tlast"。见 :ptag 。 :ptl :ptlast

---

4. 标签细节 tag-details

static-tag

静态标签是为特定文件定义的标签。在 C 程序里，这可能是一个静态函数。

Vi 里跳转某个标签的同时设置当前搜索模式。这意味着在标签跳转后，"n" 命令不再查找跳转前最后使用的模式了。Vim 没有这么做，因为我们认为这是一个错误。你仍然可以从搜索历史里查找到标签的搜索模式。如果你真的想要从前 Vi 那样的行为，在 'cpoptions' 里设置 't' 标志位。

tag-binary-search

Vim 使用二分法查找标签文件，以快速查找到希望找到的标签 (如果编译时打开 +tag_binary 特性的话)。不过这只有在 tags 文件按 ASCII 字节值排序才有效。因此，如果没有匹配，Vim 会按照线性查找再来一次。如果你只想要线性查找。复位 'tagbsearch' 选项。或者更好的: 对 tags 文件排序!

注意 如果不查找特定名字，标签的查找不使用二分法查找。这包括忽略大小写的查找或不以固定字符串开始的正规表达式的查找。这样的标签搜索可能会慢很多。前者可以用大小写合并排序来解决。详情见 'tagbsearch'。

tag-regexp

":tag" 和 ":tselect" 命令接受正规表达式参数。 pattern 可以看到如何使用特别的字符。如果参数以 '/' 开始，它被认为是一个模式。如果不是，它被认为是按本义出现的完整的标签名。
示例:

```
:tag main
```
跳转到最高优先级的 "main" 标签。
```
:tag /^get
```

跳转到最后优先级的 "get" 开始的标签。

:tag /norm

列出所有包含 "norm" 的标签，例如 "id_norm"。

如果参数既能按本义匹配，也能按正规表达式匹配，本义匹配有更高的优先级。比如，":tag /open" 在 "open_file" 和 "file_open" 之前匹配 "open"。
如果使用模式，那么忽略大小写。如果你想在模式中匹配大小写，使用 "\C"。

tag-!

如果当前文件里有需要的标签，就没有问题。不然，采取的行动决定于当前文件是否发生了改变，还有是否命令之后加了 !，还有 'autowrite' 选项：

标签在当前文件里	文件是否改变	!	autowrite 选项	行动
是	x	x	x	转到标签
否	否	x	x	读入其他文件，转到标签
否	是	是	x	放弃当前文件，读入其他文件，转到标签
否	是	否	开	写入当前文件，读入其他文件，转到标签
否	是	否	关	失败

- 如果标签在当前文件里，该命令总能工作。
- 如果标签在另外的文件里而当前文件没有改变。那个文件成为当前文件，并读入到缓冲区里。
- 如果标签在另外的文件里而当前文件发生了改变。而命令后加了 !，那么当前文件的改变被放弃，那个文件成为当前文件，并读入缓冲区。
- 如果标签在另外的文件里而当前文件发生了改变。而 'autowrite' 选项被打开，那么当前文件的改变被保存，那个文件成为当前文件，并读入缓冲区。
- 如果标签在另外的文件里而当前文件发生了改变。而 'autowrite' 选项被关闭，那么该命令失败。此时，如果你想保存改变，先用 ":w" 命令，再用不带参数的 ":tag"。这是因为标签已经在堆栈上了。如果你想放弃改变，你可以用 ":tag!" 命令。

tag-security

注意 Vim 为了安全原因禁止某些命令。这类似于在当前目录下的 exrc/vimrc 文件里使用了 'secure' 选项。见 trojan-horse 和 sandbox 。
如果 {tagaddress} 改变了缓冲区，你会得到警告：

"WARNING: tag command changed a buffer!!!"

在将来的版本里，也许会禁止对缓冲区的改变。这些都是为了安全原因：有的人可能在 tags 文件里隐藏了恶意的命令。如果不这么做，你也许不能看到这一点。例如：

:$d|/tag-function-name/

Vi 里跳转某个标签的同时设置当前搜索模式。Vim 没有这么做，以前的搜索模式还是记住的。除非 'cpoptions' 里设置了 't' 标志位。搜索历史里包含了这个搜索模式，所以在搜索失败的时候你可以修改这个模式。

emacs-tags emacs_tags E430

Emacs 风格的 tag 文件只有在 Vim 编译的时候加入 +emacs_tags 特性时才有效。对不起，这里没有 Emacs 标签文件的解释，支持它只是为了后向兼容。:-)。

Emacs 标签文件的行可能很长。Vim 只处理不超过 510 字节的行。要看到哪些行被忽略，设置 'verbose' 为 5 或更高。非 Emacs 的标签文件行接受任何长度。

tags-option

'tags' 选项是文件名的列表。其中的每个文件都用来搜索标签。这可以用来指定不同于缺省的 "tags" 文件的标签文件。它也可以用来访问一个公用的标签文件。

下列情况下，不再查找列表中的下个文件：
- 找到了当前缓冲区的匹配的静态标签。
- 找到了全局标签。
同时也取决于是否忽略大小写。以下条件下忽略大小写：
- 'tagcase' 为 "followic" 并且置位 'ignorecase'
- 'tagcase' 为 "ignore"
- 'tagcase' 为 "smart" 且模式只包含小写字符。
- 'tagcase' 为 "followscs" 且置位 'smartcase' 且模式只包含小写字符。
如果不忽略大小写，而当前标签文件找到大小写不符合的匹配，那么就会查找下一个文件。如果没有匹配大小写的标签，则使用第一个大小写不符合的匹配。如果忽略大小写，而找到全局的匹配（不论大小写），则立即使用之，而无须继续查找下一个文件。

如果标签文件以 "./" 开始，则 '.' 会被替代成当前的文件所在的路径。这样，你可以使用当前文件所在的目录下的标签文件（无论当前目录在哪里）。使用 "./" 的逻辑是，你可以定义哪个标签文件最先查找：先当前目录（"tags,./tags"），还是先当前文件所处的目录（"./tags,tags"）。

例如：
```
:set tags=./tags,tags,/home/user/commontags
```

在这个例子里，先查找当前文件所处目录的 "tags" 文件。再查找当前目录下的 "tags" 文件。如果没有，再查找 "/home/user/commontags"。

要 Vi 兼容，可以在 'cpoptions' 里包含 'd' 标志位。这样，"./tags" 会是当前目录下的 tags 文件，而不是当前文件所在的目录下的。

除了逗号以外，也可以用空格。而在字符串值的选项里，空格需要用反斜杠转义：
```
:set tags=tags\ /home/user/commontags
```

这样，要在文件名里包含空格，需要三个反斜杠。要在文件名里包含逗号，需要两个反斜杠。例如，可以用：
```
:set tags=tag\\\ file,/home/user/common\\,tags
```

来指定文件 "tag file" 和 "/home/user/common,tags"。'tags' 选项会包含值 "tag\ file,/home/user/common\,tags"。

如果打开 'tagrelative' 选项（这是缺省值）而使用别的目录的标签文件，该标签文件所引用的文件名相对于标签文件本身所处的目录。

---

5. 标签文件格式 tags-file-format E431

ctags jtags

标签文件是外部程序创建的，比如 "ctags"。对每个函数，它会包含一个标签。有些版本的 "ctags" 会为每个 "#define" 宏、类型等价定义（typedef）、枚举（enum）等也创建标签。

能产生标签文件的程序有：

ctags	大多数 Unix 系统都能找到。只支持 C。只有基本的功能。
	Exuberant_ctags
exuberant ctags	这是很好的一个。它支持 C、C++、Java、Fortran、Eiffel

	等等。它能为很多项目产生标签。见 http://ctags.sourceforge.net。
etags	和 Emacs 有关。支持很多语言。
JTags	用 Java 编写，为 Java 而设计。可以在这里找到: http://www.fleiner.com/jtags/。
ptags.py	用 Python 编写，为 Python 而设计。它位于你的 Python 源代码目录 Tools/scripts/ptags.py。
ptags	用 Perl 编写，为 Perl 而设计。可以在这里找到: http://www.eleves.ens.fr:8080/home/nthiery/Tags/。
gnatxref	为 Ada 而设计。见 http://www.gnuada.org/。gnatxref 是 gnat 包的一部分。

标签文件的行应是以下两种格式之一:

1. {tagname} {TAB} {tagfile} {TAB} {tagaddress}
2. {tagname} {TAB} {tagfile} {TAB} {tagaddress} {term} {field} ..

之前支持一种旧格式。见 tag-old-static 。

第一种格式是普通的标签，和 Vi 完全兼容。传统的 ctags 实现只能产生这种格式。通常，这用于被别的文件所引用的全局函数，

标签文件行可以用 <LF> 或者 <CR><LF> 结尾。在 Macintosh 上，<CR> 也能工作。<CR> 和 <NL> 字符不应该在行内出现。

第二个格式是新出现的。它在每行尾部的可选字段上包含了附加的信息。它和 Vi 后向兼容。只有新的 ctags 版本 (比如 Exuberant ctags) 才支持这种格式。

{tagname}	标识符。通常是函数的名字，但可以是任何标识符。不能包含 <Tab>。
{TAB}	单个 <Tab> 字符。注意: 以前的版本允许任意多个空白字符。现在不支持，因为 {tagfile} 里可能包含空格。
{tagfile}	包含 {tagname} 定义的文件。可以是绝对路径或者相对路径。还可以包含环境变量甚至通配符 (尽管使用通配符很值得怀疑)。它不能包含 <Tab>。
{tagaddress}	可以定位到 tag 的光标位置的 Ex 命令。可以是任何 Ex 命令，尽管有一些限制 (见 tag-security)。Posix 只允许行号和搜索命令，这也是最常用的。
{term}	;" 分号和双引号两个字符。这被 Vi 解释为注释的开始，从而忽略其后的部分。这样，就能和 Vi 后向兼容，因为 Vi 忽略后面的字段。例如: APP file /^static int APP;$/;" v 如果 {tagaddress} 不是行号或搜索模式，{term} 必须是 |;"。这里的竖线结束命令 (不包含竖线)，而 ;" 使 Vi 忽略行的其它部分。例如: APP file.c call cursor(3, 4)|;" v
{field} ..	可选的字段列表。每个字段有以下形式: <Tab>{fieldname}:{value} {fieldname} 标识字段，并且只能包含字母 [a-zA-Z]。 {value} 可以使任何字符串，但不能包含 <Tab>。 以下字符有特殊含义:

"\t" 代表 <Tab>
"\r" 代表 <CR>
"\n" 代表 <NL>
"\\" 代表单个 '\' 字符

有一个字段不需要 ':'。那就是标签的类型。它的处理方式就像前面有 "kind:" 开头那样。ctags 能产生的类型见它的文档。

唯一的 Vim 能识别的其他字段是 "file:" (带一个空值)。用于静态变量。

标签文件的开头可以包含这样开始的行:

```
!_TAG_
```

排序以后，他们一般都排在最前面。只有很少的以 "!" 开头的标签能排在他们前面。Vim 识别两项。第一个是指示文件是否排序的标志。如果找到这么一行，Vim 使用二分法查找标签文件:

```
!_TAG_FILE_SORTED<Tab>1<Tab>{anything}
```

标签文件可以按大小写合并排序，以免忽略大小写时只能进行线性查找 ('ignorecase' 置位且 'tagcase' 为 "followic" 或 'tagcase' 为 "ignore" 时忽略大小写)。详情见 'tagbsearch'。这时可以用值 2:

```
!_TAG_FILE_SORTED<Tab>2<Tab>{anything}
```

另一个 Vim 识别的标签指定标签文件的编码:

```
!_TAG_FILE_ENCODING<Tab>utf-8<Tab>{anything}
```

这里 "utf-8" 是标签们所用的编码。Vim 会把查找的标签从 'encoding' 转为标签文件的编码。当列出标签时，则反之。如果转换失败，则直接使用未经转换的标签。

tag-search

标签搜索命令可以是任何 Ex 命令，但通常就是普通的搜索命令。比如:

```
tag1    file1   /^main(argc, argv)/
tag2    file2   108
```

该命令执行时总假设不设定 'magic'。搜索模式里仅有的特别的字符是 "^" (行首) 和 "$" (<EOL>)。见 pattern 。注意，你必须在搜索文本里的每个反斜杠前加上反斜杠。这和 Vi 反向兼容。

E434 E435

如果该命令是一个普通的搜索命令 (也就是，以 "/" 或 "?" 开头和结尾)，会进行一些特殊处理:

- 搜索从文件第 1 行开始。
  如果用 "/"，搜索方向为正向，"?" 则为反向。
  注意 'wrapscan' 没有影响，总是搜索整个文件。
- 如果搜索失败，忽略大小写再重新尝试一次。如果还不行，搜索:

  ```
  "^tagname[ \t]*("
  ```

  (实际的标签前面加上 '^' 而后面加上 "[\t]*(")。应用函数名的时候，它会找到在第 0 列的函数名。这会有助于找到该函数，即使它的参数和生成 tags 文件时有差异。如果还不行，进行另外一个搜索:

  ```
  "^[#a-zA-Z_].*\<tagname[ \t]*("
  ```

  这意味着: 以 '#' 或者一个标识符开始的一行，包含标签本身，后面紧跟空白和一个 '('。这会找到宏名和前面有返回类型的函数名。

tag-old-static

直到 2019 年 3 月 (8.1.1092 补丁) 为止，曾经支持过另一种过时的格式：
    {tagfile}:{tagname} {TAB} {tagfile} {TAB} {tagaddress}

这种格式只限于静态标签。现在已经废弃了，被第二种格式取而代之。只有 Elvis 1.x，旧版的 Vim 还有很少的 ctags 版本支持。静态变量用于本地函数，只有文件 {tagfile} 才引用。注意 对静态变量而言，{tagfile} 的两次出现必须完全一致。关于如何使用静态变量，见下面的 tags-option 。

删除了此支持，因为你既然可以更新 Vim 版本，也应可以更新 ctags 以支持第二种格式。

---

6. 头文件搜索

include-search definition-search
E387 E388 E389

这些命令递归地查找当前文件和所有遇到的头文件的一个字符串。这可以用来查找变量的定义，函数或者宏。如果只想查找当前缓冲区，用 pattern-searches 列出来的命令。

这些命令只有在编译时加入 +find_in_path 特性才会有效。

如果一个文件的某行包含另一个文件，那么那个文件在当前缓冲区继续之前先进行搜索。头文件里包含的头文件也会被搜索。当头文件找不到时，它被悄悄地忽略掉。用 :checkpath 命令检查哪些文件可能找不到。是不是因为你的 'path' 选项没有设对。注意：被搜索的是头文件本身，不是可能正在编辑它的缓冲区。只有当前的文件除外，它使用缓冲区里的行。

字符串可能是任何的关键字或者定义的宏。对于关键字，任何匹配都会找到。对于定义的宏，只有用匹配 'define' 选项的行才会被找到。它的缺省值是 "^#\s*define"。这对 C 程序适用，对于其它语言你可能想改变之。C++ 的一个例子见 'define'。该字符串不能包含换行符，只能匹配一行之内的文本。

如果找到一个定义的宏的匹配，并且该行以反斜杠结尾的话。行的显示在下一行继续。

"[" 开头的命令从当前文件开始查找。"]" 开始的命令从当前光标位置开始。

'include' 选项用于定义可以包含另一个文件的行。缺省是 "\^#\s*include"。它适用于 C 程序。注意：Vim 并不试图识别 C 语法。如果 'include' 选项在 #ifdef/#endif 之间或者注释内的一行上匹配，它也会被搜索到。'isfname' 选项用于识别匹配之后的文件名。

'path' 选项用于查找不含绝对路径的头文件所在的目录。

'comments' 选项用来显示或者跳转到一行的命令。如果定义的模式可能包含注释，这些行用来跳过搜索，除非使用 [!]。一个特例：如果一行匹配模式 "^# *define"，它不会被认为是注释。

如果你想列出所有的匹配，然后选择一个并跳转之，你可以使用一个映射来做这件事。这里是一个例子：

```
:map <F4> [I:let nr = input("Which one: ")<Bar>exe "normal " . nr ."[\t"<CR>
```

[i 显示第一个包含光标所在的关键字的行。搜索从文件开头开

始。看起来像注释的行被忽略（见 'comments' 选项）。如果给出一个计数，跳转到第 count 个匹配的行，注释行不忽略。

]i
]i 类似于 "[i"，但是从当前光标位置开始。

:is :isearch
:[range]is[earch][!] [count] [/]pattern[/]
类似于 "[i" 和 "]i"，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

[I
[I 显示包含光标下的关键字的所有的行。找到的每行都显示文件名和行号。搜索从文件头开始。

]I
]I 类似于 "[I"，但是从当前光标位置开始。

:il :ilist
:[range]il[ist][!] [/]pattern[/]
类似于 "[I" 和 "]I"，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

[_CTRL-I
[CTRL-I 跳转到第一个包含光标所在的关键字的行。搜索从文件开头开始。看起来像注释的行被忽略（见 'comments' 选项）。如果给出一个计数，跳转到第 count 个匹配的行，注释行不忽略。

]_CTRL-I
] CTRL-I 类似于 "[CTRL-I"，但是从当前光标位置开始。

:ij :ijump
:[range]ij[ump][!] [count] [/]pattern[/]
类似于 "[CTRL-I" 和 "] CTRL-I"，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

CTRL-W CTRL-I CTRL-W_CTRL-I CTRL-W_i
CTRL-W i 打开一个新窗口，把光标放在第一个包含光标所在的关键字的行。搜索从文件开头开始。看起来像注释的行被忽略（见 'comments' 选项）。如果给出一个计数，跳转到第 count 个匹配的行，注释行不忽略。

:isp :isplit
:[range]isp[lit][!] [count] [/]pattern[/]
类似于 "CTRL-W i" 和 "CTRL-W i"，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

[d

[d 显示第一个包含光标所在的关键字的宏定义。搜索从文件头开始。如果给出一个计数，跳转到第 count 个匹配的行。

]d

]d 类似于 "[d"，但是从当前光标位置开始。

:ds :dsearch

:[range]ds[earch][!] [count] [/]string[/]
类似于 "[d" 和 "]d"，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

[D

[D 显示包含光标下的宏的所有的宏定义。找到的每行都显示文件名和行号。搜索从文件头开始。

]D

]D 类似于 "[D"，但是从当前光标位置开始。

:dli :dlist

:[range]dli[st][!] [/]string[/]
类似于 [D 和]D ，但在 [range] 行范围搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。
注意 :dl 功能等同带 "l" 标志位的 :delete ，而不是 :dlist 。

[_CTRL-D

[CTRL-D 跳转到第一个包含光标所在的关键字的宏定义。搜索从文件头开始。如果给出一个计数，跳转到第 count 个匹配的行。

]_CTRL-D

] CTRL-D 类似于 "[CTRL-D"，但是从当前光标开始。

:dj :djump

:[range]dj[ump][!] [count] [/]string[/]
类似于 "[CTRL-D" 和 "] CTRL-D"，但是在 [range] 行范围内搜索（缺省：整个文件）。
关于 [/] 和 [!] 见 :search-args 。

CTRL-W CTRL-D CTRL-W_CTRL-D CTRL-W_d

CTRL-W d 打开一个新窗口，把光标放在第一个包含光标所在的关键字的宏定义行。搜索从文件头开始。如果给出一个计数，跳转到第 count 个匹配的行。

:dsp :dsplit

:[range]dsp[lit][!] [count] [/]string[/]
类似于 "CTRL-W d"，但是在 [range] 行范围内查找

(缺省：整个文件)。
关于 [/] 和 [!] 见 :search-args

:che :chec :check :checkpath

:che[ckpath] 列出所有找不到的头文件。

:che[ckpath]! 列出所有的头文件。

:search-args

以上命令通用的参数：

[!] 如果包含，查找包括识别为注释的匹配。如果不包含，则被识别为注释（根据 'comments' ）或者是一个 C 注释（"//" 之后或者 /* */ 之间）的行不参加匹配。注意 如果一行被识别为注释，但是注释在行中间结束，匹配可能丢失。另外，如果一行是注释，但是没有被识别（根据 'comments'），还是可能从中找到匹配。例子：

```
/* comment
   foobar */
```

还是可以找到 "foobar" 的匹配，因为该行没有被识别为注释（即使语法高亮能够识别）。

注意：因为宏定义一般看起来不像注释，[!] 对 ":dlist"、":dsearch" 和 ":djump" 没有影响。

[/] 模式可以被 '/' 包围。没有 '/' 的话，只有完整单词的匹配，就像使用模式 "\<pattern\>" 那样。只有在第二个 '/' 之后，下一个命令才可以通过 '|' 附加在后。例如：

```
:isearch /string/ | echo "the last one"
```

对于 ":djump"、":dsplit"、":dlist" 和 ":dsearch" 命令，模式是按本义理解的字符串，而不是搜索模式。

---

7. 使用 _92protect'tagfunc' tag-function

可以给 Vim 提供函数，生成 :tag 、 :tselect 等命令和 CTRL-] 这样的普通模式标签命令所使用的标签列表。

生成标签列表的函数通过设置 'tagfunc' 选项指定。此函数调用时带三个参数：

a:pattern 标签搜索时使用的标签标识符。

a:flags 控制函数行为的标志位列表。

a:info 包含以下项目的字典：

- buf_ffname 可用于优先级的完整文件名。
- user_data 如有，之前通过 tagfunc 存在标签栈里的定制数据字符串。

目前可以给标签函数传递两个标志位：

'c' 函数是处理中的普通命令调用的
(助记：标签函数可用光标附近的上下文（context）来更好地生成标签列表。)

'i' 插入模式中，用户在补全标签（用 i_CTRL-X_CTRL-] ）。

注意 设置 'tagfunc' 时， tag-priority 描述的标签优先级不适用。相反，优先级完全由函数返回的列表元素的顺序来决定。

E987

函数应返回字典项目的列表。每个字典至少包含以下项，所有值都是字符串：

name 标签名。

filename	标签定义所在的文件名。可以是相对于当前目录的相对路径，也可以是完整路径。
cmd	用于在文件中定位标签的 Ex 命令。可以是 Ex 搜索模式或是行号。

注意 格式和 taglist() 类似，这样可以使用后者的输出来生成结果。

下列域是可选的：

kind	标签的类型。
user_data	存于标签栈中的定制数据，可用于区别不同操作间的标签。

如果函数返回 v:null 而不是列表，执行标准的标签查找。

在 'tagfunc' 内不允许改动标签栈。 E986

下面是一个用于 'tagfunc' 函数的假想例。它使用 taglist() 的输出来生成结果：按文件名的逆序生成标签列表。

```
function! TagFunc(pattern, flags, info)
  function! CompareFilenames(item1, item2)
    let f1 = a:item1['filename']
    let f2 = a:item2['filename']
    return f1 >=# f2 ?
                \ -1 : f1 <=# f2 ? 1 : 0
  endfunction

  let result = taglist(a:pattern)
  call sort(result, "CompareFilenames")

  return result
endfunc
set tagfunc=TagFunc
```

vim:tw=78:ts=8:noet:ft=help:norl:

windows.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Dasn
http://vimcdoc.sf.net

使用多个窗口和缓冲区进行编辑 windows buffers

本章我们看一下操作多个窗口和缓冲区的命令。另外有些命令在操作多个窗口的时候不同于单个窗口，这些命令也会在本章被介绍。

一些基本的内容在用户手册的第 7、8 两个章节里 usr_07.txt usr_08.txt 已经介绍过了。

1. 简介 windows-intro
2. 启动 Vim windows-starting
3. 打开和关闭窗口 opening-window
4. 把光标移动到另一个窗口 window-move-cursor
5. 移动窗口 window-moving
6. 改变窗口大小 window-resize
7. 参数和缓冲区列表的命令 buffer-list
8. 在所有的缓冲区和窗口里执行命令 list-repeat
9. 当前光标处的标签和文件名 window-tag
10. 预览窗口 preview-window
11. 使用隐藏缓冲区 buffer-hidden
12. 特殊类型缓冲区 special-buffers

{仅当编译时加入 +windows 特性才能使用多窗口功能}

---

1. 简介 windows-intro window

小结：
缓冲区是内存中的文件文本。
窗口是缓冲区的视窗。
标签页是窗口的集合。

"窗口" 被用来查看缓冲区里的内容。你可以用多个窗口观察同一个缓冲区，也可以用多个窗口观察不同的缓冲区。

"缓冲区" 是一块内存区域，里面存储着正在编辑的文件。如果没有把缓冲区里的文件存盘，那么原始文件不会被更改。

缓冲区的状态可以是下述三种情况之一：

active-buffer
激活： 缓冲区的内容在窗口里显示。如果该缓冲区是针对某一个文件创建的，那么该文件已经被读入缓冲区。如果缓冲区被更改过，其内容便会与原文件不同。

hidden-buffer
隐藏： 缓冲区的内容不被显示。如果该缓冲区是针对某一个文件创建的，那么该文件已经被读入缓冲区。除此以外，与激活状态一样，只是你看不见它。

inactive-buffer

非激活： 缓冲区的内容不被显示，也不包含任何数据。如果曾经载入过文件，与该缓冲区相关的选项会被记住。它可以包括来自 viminfo 文件的位置标记，但是这种缓冲区不包含文本。

用一个表来说明：

状态	在窗口显示	文件载入	":buffers" 命令显示
激活	是	是	'a'
隐藏	否	是	'h'
非激活	否	否	' '

备注：所有的 CTRL-W 命令都可以用 :wincmd 执行，当我们不能输入普通模式命令，或者输入不方便时可以试一下。

主 Vim 窗口可以容纳多个分割的窗口。此外还有标签页 tab-page ，每个标签页能容纳多个窗口。

window-ID winid windowid

每个窗口有唯一的标识符，称为窗口 ID。此标识符在同一 Vim 会话中不会改变。 win_getid() 和 win_id2tabwin() 函数可用于在窗口/标签页号和此标识符间转换。同时也有窗口号，它在每次窗口打开或关闭时会改变，见 winnr() 。
窗口号只在特定的标签页中有效。窗口 ID 跨标签页都是合法的。多数函数接受窗口 ID 或窗口号，窗口号只适用于当前标签页，而窗口 ID 可引用任何标签页中的窗口。

每个缓冲区有唯一的编号而在同一 Vim 会话中此编号不会改变。 bufnr() 和 bufname() 函数可用于在缓冲区名和缓冲区号间转换。

---

2. 启动 Vim windows-starting

在默认情况下，与 Vi 类似，Vim 启动后只打开一个窗口。

参数 "-o" 和 "-O" 可以让 Vim 为参数列表里的每一个文件打开一个窗口。参数 "-o" 水平分割窗口；参数 "-O" 垂直分割窗口。如果 "-o" 和 "-O" 都用了，那么最后一个参数决定分割的方向。例如，下面的例子打开三个水平分割的窗口：

```
vim -o file1 file2 file3
```

参数 "-oN"，这里的 N 是一个十进制数，用这个参数可以打开 N 个水平分割的窗口。如果文件名的个数多于窗口数目，则只打开 N 个窗口，这样有一些文件就得不到窗口来显示。如果指定的窗口数多于文件的个数，那么后面的几个窗口会编辑空的缓冲区。同样， "-ON" 打开 N 个垂直分割的窗口，相应的规则与 "-oN" 相同。

如果你打开了很多的文件，每个窗口都会便得很小。你可能需要设置 'winheight' 和 'winwidth' 选项来创建一个适合工作的环境。

Buf/Win Enter/Leave 等自动命令 autocommand 不会在打开窗口或读取文件时运行，只有在真正进入缓冲区后才会执行。

status-line

状态行用来分割窗口。选项 'laststatus' 用来设置最后一个窗口在什么时候可以有状态栏：

'laststatus' = 0	永远不会有状态行
'laststatus' = 1	窗口数多于一个的时候
'laststatus' = 2	总是显示状态行

你可以通过设置 'statusline' 选项来改变状态行的内容。此选项可以局部于窗口，所以你可以为每个窗口设置不同的状态行。

通常状态行用反色显示。你可以通过修改 'highlight' 选项中的 's' 字符来改变。例如，"sb" 设置为粗体字。如果状态行没有启用高亮 ("sn")，那么字符 '^' 表示当前窗口，字符 '=' 表示其它窗口。如果支持鼠标并且已经通过设置 'mouse' 选项使之启动，那么你可以用鼠标拖动状态行以改变窗口的大小。

备注：如果你想让状态行以反色显示，但是没有效果。查看一下 'highlight' 选项是否包含了 "si"。在 3.0 版本里，这意味着逆转 (invert) 状态行。但是现在应该使用 "sr"，反转 (reverse) 状态行，因为 "si" 现在表示斜体 (italic)！如果你的终端不支持斜体，那么状态行还是会以反色显示；这种问题只会出现在有斜体的 termcap 码的终端上。

---

3. 打开和关闭窗口 opening-window E36

CTRL-W s CTRL-W_s
CTRL-W S CTRL-W_S
CTRL-W CTRL-S CTRL-W_CTRL-S
:[N]sp[lit] [++opt] [+cmd] [file] :sp :split
把当前窗口分割成两个，结果是两个窗口显示同一个文件。

新打开的窗口高度为 N (默认值是当前窗口高度的一半)。减少当前窗口的高度，空出地方给新的窗口 (如果你置位了 'equalalways' 选项，并且 'eadirection' 没有设置为 "hor"，并且其它窗口中的一个高于当前或新窗口的话，其它窗口也被影响)。

如果给出 [file]，在新窗口中编辑该文件。如果还未在任何缓冲区中载入，先读入该文件。否则新窗口会使用已载入的缓冲区。

备注：不是在所有的终端都能使用 CTRL-S，并且可能导致不能继续输入，遇到这种情况，用 CTRL-Q 继续。参见 ++opt 和 +cmd 。 E242
在自动命令中分割窗口要小心，如果在其它窗口布局改动的同时发生，可能会搞乱窗口布局。

CTRL-W CTRL-V CTRL-W_CTRL-V
CTRL-W v CTRL-W_v
:[N]vs[plit] [++opt] [+cmd] [file] :vs :vsplit
与 :split 类似，但垂直分割窗口。满足以下条件的话，窗口会被均匀地水平铺开：
1. 没有指定宽度，
2. 打开了 'equalalways' 选项，
3. 'eadirection' 不是 "ver"，且
4. 其它窗口中的一个宽于当前或新窗口。
备注：在其它地方 CTRL-Q 与 CTRL-V 相同，但是在这里不是！

CTRL-W n CTRL-W_n
CTRL-W CTRL_N CTRL-W_CTRL-N
:[N]new [++opt] [+cmd] :new
创建一个新窗口并且开始编辑一个空文件。新窗口的高度为 N (默认值为现存高度的一半)。减少当前窗口的高度，为新窗口留出空间 (如果

你打开了 'equalalways' 选项，并且 'eadirection' 没有设置为 "hor"，其它窗口也会被影响)。
参见： ++opt 和 +cmd 。
如果设置了 'fileformats' 选项，那么新缓冲区会使用其中的第一个格式。如果 'fileformats' 为空，那么使用当前缓冲区的 'fileformat' 值。这个值可以被 ++opt 参数覆盖。
自动命令以如下顺序被执行：
1. 在当前窗口执行 WinLeave
2. 在新窗口里执行 WinEnter
3. 在当前缓冲区执行 BufLeave
4. 在新缓冲区执行 BufEnter
这跟先执行 ":split"，再执行一个 ":enew" 命令的效果差不多。

:[N]vne[w] [++opt] [+cmd] [file] :vne :vnew
与 :new 命令相似，但是它垂直分割窗口，如果打开了 'equalalways' 选项，并且选项 'eadirection' 没有设置为 "ver"，窗口会被均匀地水平铺开，除非你给出了指定窗口宽度的参数。

:[N]new [++opt] [+cmd] {file}
:[N]sp[lit] [++opt] [+cmd] {file} :split_f
创建一个新的窗口，并且开始编辑文件 {file}。这跟先执行 ":split"，再执行一个 ":e" 命令的效果差不多。
如果使用了 [+cmd] 参数，文件加载完毕后会执行命令 +cmd 。
也参见： ++opt 。
新窗口的高度为 N (默认值为现存高度的一半)。减少当前窗口的高度，为新窗口留出空间 (如果你打开了 'equalalways' 选项，其它窗口也会被影响)。

:[N]sv[iew] [++opt] [+cmd] {file} :sv :sview splitview
与 ":split" 命令相同，但是会给缓冲区置位 'readonly' 选项。

:[N]sf[ind] [++opt] [+cmd] {file} :sf :sfi :sfind splitfind
与 ":split" 命令相同，但是会在 'path' 里寻找 {file}，就像 :find 那样。如果找不到，就不会分割窗口。

CTRL-W CTRL-^ CTRL-W_CTRL-^ CTRL-W_^
CTRL-W ^ 把当前窗口分成两个，并且编辑轮换文件。如果指定了计数，分割窗口，并且编辑第 N 个缓冲区。类似于 ":sp #" 和 ":sp #N"，但支持另一缓冲区为无名缓冲区。此命令匹配 CTRL-^ 的行为，但先分割窗口。

CTRL-W_:
CTRL-W : 和 : - 输入命令行相同。可用于终端窗口中，那里所有的 Vim 命令必须以 CTRL-W 或 'termwinkey' 前导。

注意 'splitbelow' 和 'splitright' 选项会影响新窗口的位置。

:vert :vertical
:vert[ical] {cmd}
执行 {cmd}。如果包含一个分割窗口的命令，那么将垂直分割窗口。不能用于 :execute 和 :normal 。

:lefta[bove] {cmd} :lefta :leftabove
:abo[veleft] {cmd} :abo :aboveleft

执行 {cmd}。如果包含一个分割窗口的命令，那么将从当前窗口的左（垂直分割）或者上方（水平分割）分割窗口，而忽略 'splitbelow' 和 'splitright' 的设置。
不能用于 :execute 和 :normal 。

:rightb[elow] {cmd} :rightb :rightbelow
:bel[owright] {cmd} :bel :belowright
执行 {cmd}。如果包含一个分割窗口的命令，那么将从当前窗口的右（垂直分割）或者下方（水平分割）分割窗口，而忽略 'splitbelow' 和 'splitright' 的设置。
不能用于 :execute 和 :normal 。

:topleft E442
:to[pleft] {cmd}
执行 {cmd}。如果包含一个分割窗口的命令，那么该窗口出现在顶部，并且占据最大宽度。当垂直分割窗口时，窗口出现在最左边，并且占据最大高度。
不能用于 :execute 和 :normal 。

:bo :botright
:bo[tright] {cmd}
执行 {cmd}。如果包含一个分割窗口的命令，那么该窗口出现在底部，并且占据最大宽度。当垂直分割窗口时，窗口出现在最右边，并且占据最大高度。
不能用于 :execute 和 :normal 。

这些修饰符可以结合使用来打开一个垂直分割的并且占据最大高度的窗口：

```
:vertical topleft split tags
```

在 Vim 窗口最左边打开一个垂直分割的、最大高度的窗口，用来编辑 tags 文件。

关闭窗口

:q[uit]
:{count}q[uit]
CTRL-W q CTRL-W_q
CTRL-W CTRL-Q CTRL-W_CTRL-Q
没有 {count}：退出当前窗口。如果给出 {count}，退出第 {count} 个窗口。

如果退出的是最后一个窗口（不包括帮助窗口），就会退出 Vim。

如果置位了 'hidden' 并且只有此一个窗口监视当前缓冲区，那么该缓冲区被隐藏。如果没有置位 'hidden'，并且只有当前这一个窗口监视该缓冲区，而且缓冲区被修改了，那么该命令失败。

（注意：CTRL-Q 不是在所有的终端下都好用）。

如果 [count] 大于最后一个窗口号，关闭最后一个窗口：

```
:1quit  " 退出第一个窗口
:$quit  " 退出最后一个窗口
:9quit  " 如果打开少于 9 个窗口，退出最后一个窗口
:-quit  " 退出上一个窗口
```

```
:+quit   " 退出下一个窗口
:+2quit " 退出下面第二个窗口
```

关闭帮助窗口时，Vim 会试图恢复上次的窗口布局 :helpclose 。

:q[uit]!
:{count}q[uit]!
没有 {count}：退出当前窗口。如果给出 {count}，退出第 {count} 个窗口。

如果这是缓冲区的最后一个窗口，那么对于此缓冲区的改动将全部丢失。如果退出的是最后一个窗口（不包括帮助窗口），就会退出 Vim。缓冲区里的全部内容将丢失。即使置位了 'hidden' 选项也是如此。

:clo[se][!]
:{count}clo[se][!]
CTRL-W c CTRL-W_c :clo :close
没有 {count}：退出当前窗口。如果给出 {count}，退出第 {count} 个窗口。

如果置位了 'hidden'，或者改动了缓冲区但是命令里用了 [!]，缓冲区会被隐藏（除非还有一个窗口在编辑该缓冲区）。

如果当前标签页只有一个窗口而且有其它标签页，关闭当前标签页。 tab-page 。

该命令在下述情况下会失败： E444
- 屏幕上只剩下最后一个窗口的时候。
- 当 'hidden' 没有置位，也没有用 [!]，并且缓冲区也已经被修改，而且也没有其它窗口监视该缓冲区的时候。

对缓冲区的改动既不写入文件，也不会丢失，所以这是个 "安全" 的命令。

CTRL-W CTRL-C CTRL-W_CTRL-C
你可能会认为 CTRL-W CTRL-C 关闭当前窗口，但是这不行，因为 CTRL-C 会撤销整个命令。

:hide
:hid[e]
:{count}hid[e]
没有 {count}：除非是屏幕上的最后一个窗口，退出当前窗口。
如果给出 {count}：退出第 {count} 个窗口。

缓冲区被隐藏起来（除非还有另一个窗口正在编辑它，或者 'bufhidden' 设置为 "unload"、"delete" 或 "wipe"）。
如果这是当前标签页的最后一个窗口，关闭此标签页。 tab-page

此命令不受 'hidden' 的值影响。对缓冲区的改动既不写入文件，也不会丢失，所以这是个 "安全" 的命令。

:hid[e] {cmd} 执行 {cmd}，其间置位 'hidden' 选项。{cmd} 执行完毕后，先前的 'hidden' 选项会被恢复。
例如：

```
:hide edit Makefile
```

编辑 "Makefile" 文件，如果当前缓冲区有改动的话，隐藏之。

:on[ly][!]
:{count}on[ly][!]
CTRL-W o　　CTRL-W_o　E445
CTRL-W CTRL-O　　CTRL-W_CTRL-O　:on　:only
使当前窗口成为屏幕上唯一的窗口。其它窗口都关闭。关于 {count} 可见 :quit 命令。

如果置位了 'hidden' 选项，被关闭窗口里的所有缓冲区变成隐藏。

如果没有置位 'hidden'，但是置位了 'autowrite' 选项，那么已经更改的缓冲区被写入文件。否则，除非使用 [!] 强制关闭，那些包含修改过的缓冲区的窗口不被删除，它们转入隐藏状态。因为修改过的缓冲区永远不会被放弃，缓冲区的改动也永远不会丢失。

---

4. 把光标移动到另一个窗口　　window-move-cursor

CTRL-W <Down>　　CTRL-W_<Down>
CTRL-W CTRL-J　　CTRL-W_CTRL-J　CTRL-W_j
CTRL-W j　　把光标向下移动 N 个窗口。用光标的位置在若干窗口之间做出选择。

CTRL-W <Up>　　CTRL-W_<Up>
CTRL-W CTRL-K　　CTRL-W_CTRL-K　CTRL-W_k
CTRL-W k　　把光标向上移动 N 个窗口。用光标的位置在若干窗口之间做出选择。

CTRL-W <Left>　　CTRL-W_<Left>
CTRL-W CTRL-H　　CTRL-W_CTRL-H
CTRL-W <BS>　　CTRL-W_<BS>　CTRL-W_h
CTRL-W h　　把光标向左移动 N 个窗口。用光标的位置在若干窗口之间做出选择。

CTRL-W <Right>　　CTRL-W_<Right>
CTRL-W CTRL-L　　CTRL-W_CTRL-L　CTRL-W_l
CTRL-W l　　把光标向右移动 N 个窗口。用光标的位置在若干窗口之间做出选择。

CTRL-W w　　CTRL-W_w　CTRL-W_CTRL-W
CTRL-W CTRL-W　　如果没有添加计数，则将光标移动到当前窗口的右/下方的窗口。如果右/下方没有窗口，就移动到左上角的窗口。
要是添加了计数，就移动到第 N 个窗口 (从左上方到右下方依次计数)。窗口号可以通过 bufwinnr() 和 winnr() 来获得。如果 N 大于窗口总数，跳到最后一个窗口。

CTRL-W_W
CTRL-W W　　如果没有添加计数，则将光标移动到当前窗口的左/上方的窗口。如果左/上方没有窗口，就移动到右下角的窗口。
要是添加了计数，就移动到第 N 个窗口，类似于 CTRL-W w。

CTRL-W t　　CTRL-W_t　CTRL-W_CTRL-T
CTRL-W CTRL-T　　把光标移动到左上角的窗口。

CTRL-W b　　CTRL-W_b　CTRL-W_CTRL-B
CTRL-W CTRL-B　　把光标移动到右下角的窗口。

CTRL-W p　　CTRL-W_p　　CTRL-W_CTRL-P
CTRL-W CTRL-P　　移动到前一个 (previous) (上次访问的) 窗口。

CTRL-W_P　　E441
CTRL-W P　　移动到预览窗口。如果没有预览窗口就会出错。
{仅当编译时加入 +quickfix 特性才有效}

在可视模式下，如果新窗口编辑的是同一个缓冲区，则设置光标位置使得相同的区域被选择。如果不是同一个缓冲区，则终止可视模式。

:winc　　:wincmd
这些命令也可以用 ":wincmd" 来执行:

:[count]winc[md] {arg}
与 CTRL-W [count] {arg} 相同。例如:
:wincmd j
把光标移动到当前窗口下面的窗口。
这个命令在普通模式不可用的时候 (比如在 CursorHold 自动命令执行的时候)，或者在启用普通模式不方便时，特别好用。
计数也可以是一个窗口号。例如:
:exe nr . "wincmd w"
这会切换到窗口 "nr"。

---

5. 移动窗口　　window-moving

CTRL-W r　　CTRL-W_r　　CTRL-W_CTRL-R　　E443
CTRL-W CTRL-R　　向右/下方向旋转窗口。第一个窗口变成第二个，第二个变成第三个……最后一个变成第一个。光标保留在相同窗口里。
本操作只在和当前窗口同行或同列的窗口间移动。

CTRL-W_R
CTRL-W R　　向左/上方向旋转窗口。第二个窗口变成第一个，第三个变成第二个……第一个变成最后一个。光标保留在相同窗口里。
本操作只在和当前窗口同行或同列的窗口间移动。

CTRL-W x　　CTRL-W_x　　CTRL-W_CTRL-X
CTRL-W CTRL-X　　若没有计数: 交换当前窗口与下一个窗口。如果没有下一个窗口，则与前一个窗口交换。
若有计数: 交换当前窗口与第 N 个窗口 (第一个窗口为 1)。光标被放置在那个窗口里。
如果既有垂直分割又有水平分割，那么只能与当前窗口同行或者同列的窗口进行交换。

下面的几个命令可以被用来改变窗口的布局。例如，CTRL-W K 可以把两个垂直分割的窗口变成水平分割的窗口。CTRL-W H 则正好相反。

CTRL-W_K
CTRL-W K　　把当前窗口放到最顶端，并且是最大宽度。这很像在关闭当前窗口之后，再用 ":topleft split" 打开一个新的窗口，不过新窗口会包含当前窗口的内容。

CTRL-W_J
CTRL-W J　　把当前窗口放到最底部，并且是最大宽度。这很像在关闭当前窗口之

后，再用 ":botright split" 打开一个新的窗口，不过新窗口会包含当前窗口的内容。

CTRL-W_H
CTRL-W H　　把当前窗口放到最左边，并且是最大高度。这很像在关闭当前窗口之后，再用 ":vert topleft split" 打开一个新的窗口，不过新窗口会包含当前窗口的内容。

CTRL-W_L
CTRL-W L　　把当前窗口放到最右边，并且是最大高度。这很像在关闭当前窗口之后，再用 ":vert botright split" 打开一个新的窗口，不过新窗口会包含当前窗口的内容。

CTRL-W_T
CTRL-W T　　把当前窗口移到一个新的标签页上。如果当前标签页只有一个窗口，此命令失败。
如果给出计数，打开的新标签页会出现在该索引给出的标签页之前。否则，它出现在当前标签页之后。

---

6. 改变窗口大小　　window-resize

CTRL-W_=
CTRL-W =　　使得所有窗口（几乎）等宽、等高，但当前窗口使用 'winheight' 和 'winwidth'。
置位 'winfixheight' 的窗口保持它们的高度，而置位 'winfixwidth' 的窗口保持它们的宽度。

:res[ize] -N　　:res　:resize　CTRL-W_-
CTRL-W -　　使得当前窗口高度减 N（默认值是 1）。
如果在 'vertical' 之后使用，则使得宽度减 N。

:res[ize] +N　　CTRL-W_+
CTRL-W +　　使得当前窗口高度加 N（默认值是 1）。
如果在 'vertical' 之后使用，则使得宽度加 N。

:res[ize] [N]
CTRL-W CTRL-_　　CTRL-W_CTRL-_　CTRL-W__
CTRL-W _　　设置当前窗口的高度为 N（默认值为最大可能高度）。

:{winnr}res[ize] [+-]N
类似于上面的 :resize，但改变窗口 {winnr} 的大小，而不是当前窗口。

z{nr}<CR>　　设置当前窗口的高度为 {nr}。

CTRL-W_<
CTRL-W <　　使得当前窗口宽度减 N（默认值是 1）。

CTRL-W_>
CTRL-W >　　使得当前窗口宽度加 N（默认值是 1）。

:vertical res[ize] [N]　　:vertical-resize　CTRL-W_bar
CTRL-W |　　设置当前窗口的宽度为 N（默认值为最大可能宽度）。

你也可以用鼠标上下拖动状态行来改变窗口的高度，或者左右拖动垂直分割线来改变窗口宽度。这些功能只有在支持鼠标的版本里，并且设置了 'mouse' 选项才可以用。

选项 'winheight' ('wh') 用来设置当前窗口的最小高度。每当其它窗口变为当前窗口的时候，这一选项就会生效。如果设置为 0，则禁用。把 'winheight' 设一个很大的值，例如，'9999'，就使得当前窗口永远占据尽可能大的空间。尽量设置成一个编辑时比较舒服的值，比如说，'10' 就是一个合理的值。

同样，'winwidth' ('wiw') 选项设定了当前窗口的最小宽度。

如果置位了 'equalalways' ('ea') 选项，在分割或者关闭窗口之后，所有窗口都会变为相同大小。如果你不置位这个选项，分割窗口时会减少当前窗口的大小，而其它窗口保持不变。当关闭窗口时，多出来的行会被上面的窗口占用。

选项 'eadirection' 用来限制 'equalalways' 的作用方向。默认值是 "both"，两个方向都改变。当设置为 "ver" 时，只限制所有窗口的高度必须相同，这样你可以自己调节窗口的宽度。而垂直分割窗口的大小如果经过手工调整，其宽度也不会被破坏。同样，"hor" 使得窗口的宽度都相等。

选项 'cmdheight' ('ch') 用来设置命令行的高度。如果你对 hit-enter 的提示信息感到厌烦，可以将它设置为 2 或 3。

如果只有一个窗口，那么改变该窗口大小的同时也会改变命令行的高度。如果有多个窗口，那么改变当前窗口的大小也会改变它下面的窗口（有时是上面的窗口）的高度。

一个窗口的最小高度和最小宽度可以用 'winminheight' 和 'winminwidth' 设定。这是硬性值，一旦设定，窗口的高度（或宽度）将不能小于该值。

7．参数和缓冲区列表的命令 buffer-list

参数列表	缓冲区列表	含义
1. :[N]argument [N]	11. :[N]buffer [N]	切换至第 N 号参数/缓冲区
2. :[N]next [file ..]	12. :[N]bnext [N]	切换至向后第 N 个参数/缓冲区
3. :[N]Next [N]	13. :[N]bNext [N]	切换至向前第 N 个参数/缓冲区
4. :[N]previous [N]	14. :[N]bprevious [N]	切换至向前第 N 个参数/缓冲区
5. :rewind / :first	15. :brewind / :bfirst	切换至第一个参数/缓冲区
6. :last	16. :blast	切换至最后一个参数/缓冲区
7. :all	17. :ball	编辑所有的参数/缓冲区
	18. :unhide	编辑所有已载入的缓冲区
	19. :[N]bmod [N]	切换至第 N 个已修改的缓冲区

分割 & 参数列表	分割 & 缓冲区列表	含义
21. :[N]sargument [N]	31. :[N]sbuffer [N]	分割并至第 N 号参数/缓冲区
22. :[N]snext [file ..]	32. :[N]sbnext [N]	分割并至向后第 N 个参数/缓冲区
23. :[N]sNext [N]	33. :[N]sbNext [N]	分割并至向前第 N 个参数/缓冲区
24. :[N]sprevious [N]	34. :[N]sbprevious [N]	分割并至向前第 N 个参数/缓冲区
25. :srewind / :sfirst	35. :sbrewind / :sbfirst	分割并至第一个参数/缓冲区
26. :slast	36. :sblast	分割并至最后一个参数/缓冲区
27. :sall	37: :sball	编辑所有的参数/缓冲区
	38. :sunhide	编辑所有已载入的缓冲区
	39. :[N]sbmod [N]	分割并至第 N 个已修改的缓冲区

40. :args　　显示参数列表
41. :buffers　　显示缓冲区列表

上述命令中的 [N] 的含义取决于你使用的命令:
[N] 在命令 2/12/22/32、3/13/23/33 和 4/14/24/34 中是向前/向后的缓冲区个数。
[N] 在命令 1 和 21　　中是参数号，默认值是当前参数。
[N] 在命令 11 和 31　　中是缓冲区号，默认值是当前缓冲区。
[N] 在命令 19 和 39　　中是计数。

备注: ":next" 是一个例外，因为它必须以文件名列表为参数，这是为了和 Vi 兼容。

参数列表和多窗口

每一个窗口所编辑的文件可能位于参数列表的不同位置。记住！用 ":e file" 编辑文件后，你在参数列表中的位置没有变化，但你并不是在编辑处于那个位置的文件。为了说明这一点，文件消息 (和标题栏，如果有的话) 会显示 "(file (N) of M)"，"(N)" 是当前文件在列表中的位置，"M" 是列表中文件的数量。

所有参数列表中的条目都被加载到缓冲区列表中，这样你也可以用缓冲区列表的命令进行操作，比如 ":bnext"。

:[N]al[l][!] [N]　　:al　:all　:sal　:sall
:[N]sal[l][!] [N]
重新整理屏幕，为每一个参数打开一个窗口。其它窗口统统关闭。如果使用了计数，则其数值为窗口数目的最大值。
如果带 :tab 修饰符，为每个参数打开一个标签页。如果参数数目多于 'tabpagemax'，多余参数成为最后一个标签页的分割窗口。
如果设置了 'hidden'，所有要关闭的窗口变为隐藏。
如果没有设置 'hidden'，但是设置了 'autowrite'，那么所有改动的缓冲区被写入文件。否则，包含更改过的缓冲区的窗口不会被删除，除非你用 [!] 使它们隐藏。更改过的缓冲区永远不会被丢弃，所以改动不会丢失。
[N] 是最大可以打开的窗口数。'winheight' 也限制打开的窗口数。(如果前加了 :vertical 的话 'winwidth')。
Buf/Win Enter/Leave 自动命令不会被这里的新窗口执行，只有在它们真正进入时才会执行。
如果此命令在忙时，自动命令改变了窗口布局，报错。 E249

:[N]sa[rgument][!] [++opt] [+cmd] [N]　　:sa　:sargument
这是 ":split | argument [N]" 命令的简写。分割窗口并切换至第 N 号参数。但是如果该参数不存在，窗口不会分割。参见 ++opt 和 +cmd 。

:[N]sn[ext][!] [++opt] [+cmd] [file ..]　　:sn　:snext
这是 ":split | [N]next" 命令的简写。分割窗口并切换到向后第 N 个参数。但是如果该参数不存在，窗口不会分割。参见 ++opt 和 +cmd 。

:[N]spr[evious][!] [++opt] [+cmd] [N]　　:spr　:sprevious
:[N]sN[ext][!] [++opt] [+cmd] [N]　　:sN　:sNext
这是 ":split | [N]Next" 命令的简写。分割窗口并切换到向前第 N 个参数。但是如果该参数不存在，窗口不会分割。参见 ++opt 和

+cmd 。

:sre :srewind

:sre[wind][!] [++opt] [+cmd]
这是 ":split | rewind" 命令的简写。分割窗口并切换到第一个参数。但是如果没有参数列表，窗口不会分割。参见 ++opt 和 +cmd 。

:sfir :sfirst

:sfir[st] [++opt] [+cmd]
与 ":srewind" 相同。

:sla :slast

:sla[st][!] [++opt] [+cmd]
这是 ":split | last" 命令的简写。分割窗口并切换到最后一个参数。但是如果没有参数列表，窗口不会分割。参见 ++opt 和 +cmd 。

:dr :drop

:dr[op] [++opt] [+cmd] {file} ..
在一个窗口内编辑第一个 {file}。
- 如果该文件已经被打开，切换至该文件所在窗口。
- 如果该文件尚未被打开，在当前窗口内打开该文件。如果当前窗口无法被放弃 abandon ，窗口会先被分割。
- 如果可能的话，关闭不在参数列表里或不是全宽的窗口。

参数列表 argument-list 会被像使用 :next 命令一样被设定。此命令的目的在于：允许调试器之类的程序里让 Vim 编辑另一个文件。
如果使用 :tab 修饰符，为每个参数打开一个标签页。如果为空，使用最后一个窗口。
另见 ++opt 和 +cmd 。

---

8. 在所有的缓冲区或窗口里执行命令 list-repeat

:windo

:[range]windo {cmd} 在每一个窗口里执行 {cmd}，如果给出 [range]，只对窗口号在 [range] 内的窗口执行。这就像是：

```
CTRL-W t
:{cmd}
CTRL-W w
:{cmd}
等等……
```

只在当前标签页进行操作。
如果在一个窗口里检测到错误，余下的窗口不再访问。
最后一个窗口（或者出错的那个窗口）成为当前窗口。
{cmd} 可以包含 '|' 来连接多个命令。
{cmd} 不能打开或关闭窗口，也不能更换它们的顺序。

也参见 :tabdo 、 :argdo 、 :bufdo 、 :cdo 、 :ldo 、 :cfdo 和 :lfdo

:bufdo

:[range]bufdo[!] {cmd} 在缓冲区列表的每一个缓冲区里执行 {cmd}，如果给出 [range]，只对缓冲区号在 [range] 内的缓冲区执行。

就像是:

```
:bfirst
:{cmd}
:bnext
:{cmd}
等等……
```

如果当前文件不能被丢弃 abandon ，并且没有使用 [!]，命令失败。
当在一个缓冲区里检测到错误，剩下的缓冲区不再访问。
跳过列表外缓冲区。
最后一个缓冲区（或者出错的那个缓冲区）成为当前缓冲区。
{cmd} 可以包含 '|' 来连接多个命令。
{cmd} 不能删除或添加缓冲区到缓冲区列表。
备注：此命令执行时，通过把 Syntax 加入 'eventignore' 来屏蔽此自动命令事件。这样大大加快了每个缓冲区的编辑速度。

也参见 :tabdo 、 :argdo 、 :windo 、 :cdo 、 :ldo 、 :cfdo 和 :lfdo

例子:

```
:windo set nolist nofoldcolumn | normal zn
```

这个命令会复位 'list' 选项并在所有的窗口里关闭折叠功能。

```
:bufdo set fileencoding= | update
```

此命令在每个缓冲区里复位 'fileencoding' 选项的值，并将改动的缓冲区存盘。结果是所有的缓冲区都会用 'encoding' 编码（如果能够正确转换的话）。

---

9. 当前光标处的标签和文件名 window-tag

:sta :stag

:sta[g][!] [tagname]
执行 ":tag[!] [tagname]" 并且分割窗口打开新的标签。参见 :tag 。

CTRL-W] CTRL-W_] CTRL-W_CTRL-]
CTRL-W CTRL-]
把当前窗口一分为二，把光标所在的标识符当作标签，并在上面的新窗口跳转至标签所指向的地方。
可视模式下把可视选择的文本当作标签。
新窗口的高度为 N。

CTRL-W_g]
CTRL-W g]
把当前窗口一分为二，把光标所在的标识符当作标签，并在上面的新窗口里执行 ":tselect"。
可视模式下把可视选择的文本当作标签。
新窗口的高度为 N。

CTRL-W_g_CTRL-]
CTRL-W g CTRL-]
把当前窗口一分为二，把光标所在的标识符当作标签，并在上面的新窗

口里执行 ":tjump"。
可视模式下把可视选择的文本当作标签。
新窗口的高度为 N。

CTRL-W f　　CTRL-W_f　CTRL-W_CTRL-F
CTRL-W CTRL-F　把当前窗口一分为二。编辑光标下的文件名。
类似 ":split gf"，但是找不到文件或者文件不存在时，不会分割窗口。
此操作在 'path' 变量所定义的目录和当前文件所在的目录里寻找文件。
如果是一个类似 "type://machine/path" 的超链接，那么只使用 "/path"。
如果指定了计数 N，那么编辑第 N 个匹配的文件。
{仅在编译时加入 +file_in_path 特性才有效}

CTRL-W F　　CTRL-W_F
把当前窗口一分为二。编辑光标下的文件名并跳转到文件名之后的行号上。 gF 详细说明该行号如何获取。
{仅在编译时加入 +file_in_path 特性才有效}

CTRL-W gf　　CTRL-W_gf
打开新标签页，并编辑光标下的文件名。类似于 "tab split" 和 "gf"，但如果文件不存在，不建立新标签页。
{仅在编译时加入 +file_in_path 特性才有效}

CTRL-W gF　　CTRL-W_gF
打开新标签页，编辑光标下的文件名，并跳转到文件名之后的行号上。类似于 "tab split" 然后 "gF"，但如果文件不存在，不建立新标签页。
{仅在编译时加入 +file_in_path 特性才有效}

CTRL-W gt　　CTRL-W_gt
转到下个标签页，等同于 gt 。

CTRL-W gT　　CTRL-W_gT
转到上个标签页，等同于 gT 。

另见 CTRL-W_CTRL-I : 为包含当前光标下的关键字的头文件打开窗口。

---

10. 预览窗口　　preview-window

预览窗口是显示（预览）其它文件的特殊窗口。它通常是一个用来查看包含文件或者函数定义的小窗口。{仅当编译时加入 +quickfix 特性才有效}

(每个标签页) 只能有一个预览窗口，可以用如下命令之一创建。另外，'previewheight' 选项可以用来指定预览窗口的高度；预览窗口设定 'previewwindow' 选项以便区分。'winfixheight' 选项也被设定用来保持该窗口的高度，不受其它窗口开关的影响。

　　preview-popup
另一个方案是通过设置 'previewpopup' 来使用弹出窗口。如果设置，它否决 'previewwindow' 和 'previewheight' 设置。该选项是逗号分隔的值列表：

height	弹出的最大高度
width	弹出的最大宽度
highlight	弹出的高亮组（缺省为 Pmenu）

示例：

```
:set previewpopup=height:10,width:60
```

一些特殊之处：
- 如果文件已在某缓冲区中，会重用该缓冲区。这样文件在弹出窗口中可见的同时可以进行编辑。
- 不使用 ATTENTION 对话框，因为你不能编辑弹出窗口中的文件。如果之后在普通窗口中打开了相同的缓冲区，可能不会注意到它在其它地方被编辑。此时再使用 ":edit" 会触发 ATTENTION，按 "A" 放弃时，预览窗口会变空。

:pta :ptag

:pta[g][!] [tagname]
执行 ":tag[!] [tagname]" 并且在预览窗口里显示标签所指向的内容，不改变当前缓冲区和光标的位置。如果预览窗口已经打开，则会被重用（类似于帮助窗口）。新预览窗口的高度由 'previewheight' 决定。另见 :tag 。
参考下面的例子。 CursorHold-example
与 :tag 有细小的差别：当 [tagname] 与当前显示的标签相同时，该命令不会重新设置标签匹配表的当前位置。这使 :ptnext 执行完后， CursorHold-example 仍可以运行。

CTRL-W z CTRL-W_z
CTRL-W CTRL-Z CTRL-W_CTRL-Z :pc :pclose
:pc[lose][!] 关闭当前打开的预览窗口。如果置位了 'hidden'，或者更改了缓冲区并且使用了 [!]，则缓冲区变为隐藏（除非还有其它窗口正在编辑该缓冲区）。如果预览窗口的缓冲区不能关闭，此命令失败。也参见 :close 。

:pp :ppop

:[count]pp[op][!]
在预览窗口执行 ":[count]pop[!]"。参见 :pop 和 :ptag 。

CTRL-W } CTRL-W_}
使用当前光标所处位置的标识符作为标签，执行 :ptag。生成预览窗口（如果需要的话），窗口的高度为 N，如果没指定高度，就使用 'previewheight' 的值。

CTRL-W g } CTRL-W_g}
使用当前光标所处位置的标识符作为标签，执行 :ptjump。生成预览窗口（如果需要的话），窗口的高度为 N，如果没指定高度，就使用 'previewheight' 的值。

:ped :pedit

:ped[it][!] [++opt] [+cmd] {file}
在预览窗口里编辑文件 {file}。预览窗口打开的方式和 :ptag 类似。当前窗口和光标的位置都不改变。可用实例：

```
:pedit +/fputc /usr/include/stdio.h
```

:ps :psearch

:[range]ps[earch][!] [count] [/]pattern[/]
与 :ijump 命令很像，只是找到的匹配在预览窗口显示。预览窗口用 :ptag 打开。当前窗口和光标的位置都不改变。有用的例子：

```
:psearch popen
```

这很像 :ptag 命令，你可以用它自动显示光标所在单词的相关信

息。这个命令虽然不如 :ptag 强大，但是你可以不使用标签文件，而同样可以找到系统的头文件。例如：

```
:au! CursorHold *.[ch] nested exe "silent! psearch " . expand("<cword>")
```

警告：可能会很慢。

例子 CursorHold-example

```
:au! CursorHold *.[ch] nested exe "silent! ptag " . expand("<cword>")
```

如果光标停留达到至少 'updatetime' 指定的时间，这条命令以当前光标位置处的关键字为参数，执行 ":ptag" 命令。"nested" 使得其它自动命令得以执行，这样可以在预览窗口里启用语法高亮。"silent!" 屏蔽了在找不到标签时产生的错误信息。也参见 CursorHold 。要禁用此功能：

```
:au! CursorHold
```

更好的改进是能高亮找到的标签，光标处没有单词时不执行 ":ptag"，还能处理一些其它细节，见下：

```
:au! CursorHold *.[ch] nested call PreviewWord()
:func PreviewWord()
:  if &previewwindow                    " 不要在预览窗口里执行
:    return
:  endif
:  let w = expand("<cword>")             " 在当前光标位置抓词
:  if w =~ '\a'                         " 如果该单词包括一个字母
:
:    " 在显示下一个标签之前，删除所有现存的语法高亮
:    silent! wincmd P                   " 跳转至预览窗口
:    if &previewwindow                  " 如果确实转到了预览窗口……
:      match none                       " 删除语法高亮
:      wincmd p                         " 回到原来的窗口
:    endif
:
:    " 试着显示当前光标处匹配的标签
:    try
:       exe "ptag " . w
:    catch
:      return
:    endtry
:
:    silent! wincmd P                   " 跳转至预览窗口
:    if &previewwindow                  " 如果确实转到了预览窗口……
:      if has("folding")
:        silent! .foldopen              " 展开折叠的行
:      endif
:      call search("$", "b")            " 到前一行的行尾
:      let w = substitute(w, '\\', '\\\\', "")
:      call search('\<\V' . w . '\>') " 定位光标在匹配的单词上
:      " 给在此位置的单词加上匹配高亮
:      hi previewWord term=bold ctermbg=green guibg=green
:      exe 'match previewWord "\%' . line(".") . 'l\%' . col(".") . 'c\k*"'
:      wincmd p                         " 返回原来的窗口
:    endif
```

```
  :  endif
:endfun
```

---

11. 使用隐藏缓冲区 buffer-hidden

隐藏的缓冲区不会在窗口里显示，但仍然载入内存，这使得 Vim 可以在文件间切换，而无需每次在一个窗口里读入另一个缓冲区时读写文件。

:buffer-!

如果置位了 'hidden' ('hid') 选项，":edit"、":next"、":tag" 等编辑其它文件的命令会保留被丢弃的缓冲区。有时虽然没有置位 'hidden'，一些在缓冲区列表中穿梭的命令还是会把当前窗口变为隐藏。这种情况发生在：你试图用 '!' 从窗口里强制删除修改过的缓冲区。但 'autowrite' 选项关闭或者缓冲区无法写回，

你可以用任何启动编辑的命令使隐藏的缓冲区不再隐藏。或者用 ":bdelete" 命令将其删除。

'hidden' 是全局选项，它作用于所有的缓冲区。'bufhidden' 选项可以作用于指定的缓冲区，该选项可以是下面的值：

<empty>	使用全局 'hidden' 的值。
hide	隐藏该缓冲区，'hidden' 没有置位也是如此。
unload	不隐藏，而是卸载该缓冲区，'hidden' 置位也是如此。
delete	删除该缓冲区。

hidden-quit

如果存在一个隐藏且修改过的缓冲区，退出 Vim 会得到一条错误信息，并且 Vim 会把该缓冲区变为当前的缓冲区。你可以决定是保存 (":wq") 还是退出 (":q!")。注意：可能不止一个这样的缓冲区！

缓冲区也可能是列表外的 (unlisted)，这意味着此缓冲区存在，但是从缓冲区列表中看不到。 unlisted-buffer

:files[!] [flags] :files
:buffers[!] [flags] :buffers :ls
:ls[!] [flags]

显示所有缓冲区。示例：

```
1 #h   "/test/text"          line 1
2u     "asdf"                line 0
3 %a + "version.c"           line 1
```

如果使用了 [!]，将显示所有的缓冲区，包括列表外缓冲区 (此时，"列表外" (unlisted) 这一术语多少有点儿别扭……)。

每一个缓冲区都有唯一的号码与之对应。这个编号是不会改变的，所以你可以一直使用 ":buffer N" 或 "N CTRL-^" (N 是缓冲区编号) 来选择特定的缓冲区。

文件名使用以下的特殊值：

[Prompt] prompt-buffer
[Popup] popup-window 的缓冲区

[Scratch] 'buftype' 为 "nofile"
[No Name] 没有指定文件名
terminal-window 缓冲区使用状态。

标识符号（同一列上的符号不可能同时出现）：
u 列表外缓冲区（只有使用 [!] 才能看到） unlisted-buffer
% 当前窗口里的缓冲区
# 用 ":e #" 或 CTRL-^ 可切换到的轮换缓冲区
a 激活缓冲区：已载入内存并且可见
h 隐藏缓冲区：已载入内存但是没有窗口显示它
hidden-buffer
\- 不可更改的缓冲区，选项 'modifiable' 被关闭。
= 只读缓冲区
R 作业运行中的终端缓冲区
F 作业已完成的终端缓冲区
? 没有作业的缓冲区缓冲区：‘:terminal NONE‘
\+ 已经更改的缓冲区
x 有读错误的缓冲区

[flags] 可以是下面字符的组合，用来限制列出的缓冲区：
\+ 修改过的缓冲区
\- 'modifiable' 关闭的缓冲区
= 只读缓冲区
a 激活的缓冲区
u 列表外缓冲区（覆盖 "!"）
h 隐藏缓冲区
x 有读入错误的缓冲区
% 当前缓冲区
# 轮换缓冲区
R 作业运行中的终端缓冲区
F 作业已完成的终端缓冲区
? 没有作业的缓冲区缓冲区：‘:terminal NONE‘
t 显示最近使用的时间并给缓冲区排序
标志位的组合意味着它们被 "与" 在一起，例如：
h+ 修改过的隐藏缓冲区
a+ 修改过的激活的缓冲区

:filter 的模式匹配显示的缓冲区名，例如：
filter /\.vim/ ls

:bad :badd
:bad[d] [+lnum] {fname}
在缓冲区列表里添加文件名 {fname}，但是不把该文件载入内存。
如果指定了 "lnum"，那么第一次进入缓冲区时，光标会停留在那一行。注意 其它 + 后面的命令将被忽略。

:[N]bd[elete][!] :bd :bdel :bdelete E516
:bd[elete][!] [N]
卸载缓冲区 [N]（默认：当前缓冲区）并且从缓冲区列表里删除该缓冲区。如果缓冲区被改动过，那么该命令将失败，除非使用 [!]，但是这样所有的改动都会丢失，文件不受影响。该缓冲区的所有窗口都会关闭。如果缓冲区 [N] 是当前的缓冲区，那么 Vim 会显示另一个缓冲区来代替。具体来说，Vim 会选择跳转表里最新且已经载入的缓冲区。
实际上，该缓冲区还没有被完全删除，但是缓冲区列表把它除名

unlisted-buffer ，而且该缓冲区中选项的值、变量、映射和缩写被清空。例如：

```
:.,$-bdelete    " 删除从当前开始到倒数第二个的缓冲区
:%bdelete       " 删除所有缓冲区
```

:bdelete[!] {bufname} E93 E94

类似 ":bdelete[!] [N]"，但是使用名字指定缓冲区，见 {bufname} 。

:bdelete[!] N1 N2 ...

对缓冲区 N1、N2、…… 执行 ":bdelete[!]"。参数可以是缓冲区名或号(但不能是本身为数字的缓冲区名)。如果名字里含有空格，在空格前插入反斜杠。

:N,Mbdelete[!] 对 N 到 M 号（包含 N 和 M inclusive ）缓冲区执行 ":bdelete[!]"。

:[N]bw[ipeout][!] :bw :bwipe :bwipeout E517

:bw[ipeout][!] {bufname}

:N,Mbw[ipeout][!]

:bw[ipeout][!] N1 N2 ...

类似 ":bdelete"，但真正删除缓冲区。所有和缓冲区相关的信息都丢失了。所有位置标记失效，选项丢失，诸如此类。除非你确切地知道自己正在做什么，否则不要使用这个命令。例如：

```
:.+,$bwipeout   " 真正删除当前缓冲区之后的所有缓冲区
:%bwipeout      " 真正删除所有缓冲区
```

:[N]bun[load][!] :bun :bunload E515

:bun[load][!] [N]

卸载缓冲区 [N]（默认：当前缓冲区）。原来占用的内存将被释放。该缓冲区还保留在缓冲区列表里。

如果缓冲区被改动过，那么该命令将失败，除非使用 [!]，但是这样所有的改动都会丢失。

该缓冲区的所有窗口都会关闭。如果缓冲区 [N] 是当前的缓冲区，那么 Vim 会显示另一个缓冲区来代替。具体来说，Vim 会选择跳转表里最新且已经载入的缓冲区。

:bunload[!] {bufname}

类似 ":bunload[!] [N]"，但是使用名字指定缓冲区。亦见 {bufname} 。

:N,Mbunload[!] 对 N 到 M 号（包含 N 和 M inclusive ）缓冲区执行 ":bunload[!]" 。

:bunload[!] N1 N2 ...

对缓冲区 N1、N2、…… 执行 ":bunload[!]"。参数可以是缓冲区名或号(但不能是本身为数字的缓冲区名)。如果名字里含有空格，在空格前插入反斜杠。

:[N]b[uffer][!] [+cmd] [N] :b :bu :buf :buffer E86

编辑缓冲区列表中第 [N] 个缓冲区，如果没指定 [N]，就编辑当前的缓冲区。关于 [!] 参见 :buffer-! 。这个命令同样可以编辑不在列

表里的缓冲区，'buflisted' 不会因而被置位。
另见 +cmd 。

:[N]b[uffer][!] [+cmd] {bufname} {bufname}
编辑缓冲区列表中 {bufname} 对应的缓冲区。名字的部分也可以，只要在缓冲区列表中是唯一的。
注意 如果缓冲区名字是一个数字，不能使用名字来访问；而应用其编号代替。
如果名字里含有空格，在空格前插入反斜杠。
关于 [!] 参见 :buffer-! 。
这个命令同样可以编辑不在列表里的缓冲区， 'buflisted' 不会因而被置位。
另见 +cmd 。

:[N]sb[uffer] [+cmd] [N] :sb :sbuffer
分割窗口并编辑缓冲区列表中第 [N] 个缓冲区，如果没指定 [N]，就编辑当前的缓冲区。分割时，适用 'switchbuf' 的 "useopen" 设置。
这个命令同样可以编辑不在列表里的缓冲区，'buflisted' 不会因而被置位。
另见 +cmd 。

:[N]sb[uffer] [+cmd] {bufname}
分割窗口并编辑缓冲区列表中 {bufname} 对应的缓冲区。这个命令同样可以编辑不在列表里的缓冲区，'buflisted' 不会因而被置位。
备注：如果你想做的是分割缓冲区并在另一个名字下建立备份，可以这么做：

```
:w foobar | sp #
```

另见 +cmd 。

:[N]bn[ext][!] [+cmd] [N] :bn :bnext E87
至缓冲区列表里往后第 [N] 个缓冲区。[N] 默认是 1。如果到了缓冲区列表的末尾，则从头循环。
关于 [!] 参见 :buffer-! 。
另见 +cmd 。
如果你在帮助缓冲区里，这条命令让你跳到后面的帮助缓冲区（如果有的话）。同样，你在一个正常（非帮助）缓冲区里，就会使你跳到后面的正常缓冲区。这样是为了在打开帮助的时候不会影响到浏览正常的代码或文本。下面三个命令都是如此。

:sbn :sbnext
:[N]sbn[ext] [+cmd] [N]
分割窗口并至缓冲区列表往后第 [N] 个缓冲区。如果到了缓冲区列表的末尾，则从头循环。使用 'switchbuf'
另见 +cmd 。

:[N]bN[ext][!] [+cmd] [N] :bN :bNext :bp :bprevious E88
:[N]bp[revious][!] [+cmd] [N]
至缓冲区列表往前第 [N] 个缓冲区。[N] 默认是 1。如果到了缓冲区列表的顶端，则从末尾往前循环。
关于 [!] 参见 :buffer-! 和 'switchbuf'。
另见 +cmd 。

:[N]sbN[ext] [+cmd] [N] :sbN :sbNext :sbp :sbprevious

:[N]sbp[revious] [+cmd] [N]
分割窗口并至缓冲区列表往前第 [N] 个缓冲区。如果到了缓冲区列表的顶端，则从末尾往前循环。
使用 'switchbuf'
另见 +cmd 。

:br[ewind][!] [+cmd] :br :bre :brewind
至缓冲区列表中的第一个缓冲区。如果列表为空，则转到第一个列表外缓冲区。
对于 [!] 参见 :buffer-!

:bf[irst] [+cmd] :bf :bfirst
与 ":brewind" 相同。
另见 +cmd 。

:sbr[ewind] [+cmd] :sbr :sbrewind
分割窗口并至缓冲区列表中的第一个缓冲区。如果列表为空，则转到第一个列表外缓冲区。
还需考虑 'switchbuf' 选项。
另见 +cmd 。

:sbf[irst] [+cmd] :sbf :sbfirst
与 ":sbrewind" 相同。

:bl[ast][!] [+cmd] :bl :blast
至缓冲区列表中的最后一个缓冲区。如果列表为空，则转到最后一个列表外缓冲区。
对于 [!] 参见 :buffer-!

:sbl[ast] [+cmd] :sbl :sblast
分割窗口并至缓冲区列表中的最后一个缓冲区。如果列表为空，则转到最后一个列表外缓冲区。
还需考虑 'switchbuf' 选项。

:[N]bm[odified][!] [+cmd] [N] :bm :bmodified E84
至往后第 [N] 个修改过的缓冲区。备注：这一命令同样也能找到列表外缓冲区。如果没有修改的缓冲区，此命令失败。

:[N]sbm[odified] [+cmd] [N] :sbm :sbmodified
分割窗口并至往后第 [N] 个修改过的缓冲区。
还需考虑 'switchbuf' 选项。
备注：这个命令也能找到不在列表里的缓冲区。

:[N]unh[ide] [N] :unh :unhide :sun :sunhide
:[N]sun[hide] [N]
重新排列屏幕，给缓冲区列表中的每一个载入的缓冲区打开一个窗口。如果给出计数，则计数为打开窗口的最多数目。

:[N]ba[ll] [N] :ba :ball :sba :sball
:[N]sba[ll] [N] 重新排列屏幕，给缓冲区列表中的每一个缓冲区打开一个窗口。如果给出计数，则计数为打开窗口的最多数目。''winheight' 也限制了打开窗口的数目（当前缀 :vertical 时则为 'winwidth'）。
Buf/Win Enter/Leave 不会在新窗口执行，只有真正进入这些窗口时才会。

如果使用 :tab 修饰符，新窗口在新标签页上打开，直到用完 'tabpagemax' 个标签页为止。

备注：上面所有开始编辑另一个缓冲区的命令维持 'readonly' 的原值。这和 ":edit" 命令不同，它每次读入文件时都会设置 'readonly' 标志位。

---

12. 特殊类型缓冲区 special-buffers

除了用来存放文本，缓冲区也可以用于其它目的。一些选项可以改变缓冲区的行为：

'bufhidden'	缓冲区不再在窗口里显示时的行为
'buftype'	缓冲区的类型
'swapfile'	缓冲区是否需要交换文件
'buflisted'	缓冲区是否在缓冲区列表中出现

有用的缓冲区类型：

quickfix 用来存放错误列表和位置列表。参见 :cwindow 和 :lwindow 。这些命令设置 'buftype' 的值为 "quickfix"。你不应该修改它！'swapfile' 为关。

help 包含帮助文件。只能用 :help 命令创建，缓冲区的帮助标志是内置的，不能被更改。选项 'buflisted' 在帮助缓冲区里被复位。

terminal 终端窗口缓冲区，见 terminal 。直到作业完成为止，内容不可读或改变。

directory 显示目录内容。可用于文件探索器插件。该缓冲区由以下设置创建：

```
:setlocal buftype=nowrite
:setlocal bufhidden=delete
:setlocal noswapfile
```

缓冲区的名字就是目录的名字，并且在使用 :cd 命令时会作出相应的调整。

scratch 该缓冲区包含的文本可以被随时扔掉。在关闭窗口时，缓冲区被保存下来，要删除缓冲区必须显式进行。
设置如下：

```
:setlocal buftype=nofile
:setlocal bufhidden=hide
:setlocal noswapfile
```

如果你能起个有意义的名字，缓冲区的名字可以用来标识该缓冲区。

unlisted-buffer

unlisted 该缓冲区不在缓冲区列表里。它不能用来编辑，但是可以显示帮助文件、记住文件名或标记。":bdelete" 命令也会设置这个选项，这样就不会完全地删除该缓冲区。设置如下：

```
:setlocal nobuflisted
```

vim:tw=78:ts=8:noet:ft=help:norl:

tabpage.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

多标签页窗口编辑 tab-page tabpage

这里解释新加入的处理多标签页的命令。另外，也提供因为和多于一个标签页组合使用而行为不同的命令的解释。

1. 简介 tab-page-intro
2. 命令 tab-page-commands
3. 其它项目 tab-page-other
4. 设置 'tabline' setting-tabline
5. 设置 'guitablabel' setting-guitablabel

{仅当编译时加入 +windows 特性才能使用多标签页}

---

1. 简介 tab-page-intro

一个标签页容纳一或多个窗口。标签页间很容易切换，这种方式使你可以拥有多个窗口集合，每个集合做不同的工作。

通常你会在 Vim 窗口顶部看到一组标签的列表，每个标签页一个。用鼠标点击标签可以跳转到对应的标签页。下面还介绍在其它标签页间移动的方式。

多数命令只能用于当前标签页，其中包括 CTRL-W 命令、:windo 、:all 和 :ball (除非用 :tab 修饰符)。
下面会列出那些能意识到有其它标签页存在的命令。

标签页也是临时编辑某个缓冲区而不用修改当前窗口布局的好方法。只要打开新标签页，做你想做的事，然后关闭该标签页就可以了。

---

2. 命令 tab-page-commands

打开新标签页:

启动 Vim 时用 "vim -p filename ..." 为每个文件参数打开一个新标签页 (标签页个数不能超过 'tabpagemax')。见 -p

非 GUI 标签页行上双击鼠标打开新的空白标签页。该标签页出现在点击位置的左侧。第一击可能会先选择不同的标签页，从而导致额外的屏幕刷新。

在一些 GUI 版本上，特别是 Win32 和 Motif，也可以这么用。但必须在所有标签右侧点击。

GUI 标签页行可以用鼠标右键打开菜单。 tabline-menu 。

相关的自动命令可见 tabnew-autocmd 。

:[count]tabe[dit] :tabe :tabedit :tabnew
:[count]tabnew
在当前标签页之后打开带空窗口的新标签页。
如果给出 [count]，新标签页出现在第 [count] 个标签页之后，否则新标签页出现在当前页之后。

```
:tabnew     " 在当前页之后打开标签页
:.tabnew    " 同上
:+tabnew    " 在下页之后打开标签页
            " 备注: 这比 :tabnew 要后一个位置
:-tabnew    " 在当前页之前打开标签页
:0tabnew    " 在首页之前打开标签页
:$tabnew    " 在末页之后打开标签页
```

:[count]tabe[dit] [++opt] [+cmd] {file}
:[count]tabnew [++opt] [+cmd] {file}
打开新标签页并编辑 {file}，其余和 :edit 类同。关于 [count] 见上面的 :tabnew 。

:[count]tabf[ind] [++opt] [+cmd] {file} :tabf :tabfind
打开新标签页并编辑 'path' 里的 {file}，其余和 :find 类同。关于 [count] 见上面的 :tabnew 。
{仅当编译时加入 +file_in_path 特性才有效}

:[count]tab {cmd} :tab
执行 {cmd}，如果它打开新窗口，使用新标签页代替。不能用于 :diffsplit 、 :diffpatch 、 :execute 和 :normal 。
如果给出 [count]，新标签页出现在第 [count] 个标签页之后，否则新标签页出现在当前页之后。
例如:

```
:tab split      " 在新标签页上打开当前缓冲区
:tab help gt    " 打开标签页显示 "gt" 的帮助
:.tab help gt   " 同上
:+tab help      " 在下页之后打开标签页显示帮助
:-tab help      " 在当前页之前打开标签页显示帮助
:0tab help      " 在首页之前打开标签页显示帮助
:$tab help      " 在末页之后打开标签页显示帮助
```

CTRL-W gf 打开新标签页并编辑光标所在的文件名。见 CTRL-W_gf 。

CTRL-W gF 打开新标签页，编辑光标所在的文件名并转到文件名后的行号上。见 CTRL-W_gF 。

关闭标签页:

关闭标签页的最后一个窗口同时也关闭该标签页，除非它是唯一的标签页。

使用鼠标: 如果显示标签页行，点击右上角的 "X" 可以关闭当前标签页。如果使用自定义的 'tabline' ，显示内容可能有所不同。

:tabc :tabclose
:tabc[lose][!] 关闭当前标签页。
如下情形使该命令失败:

- 屏幕上只有一个标签页。 E784
- 如果 'hidden' 没有置位，没有 [!]，缓冲区有修改，而且该缓冲区没有其它窗口。

缓冲区的修改不会被写入，也不会丢失。所以这可以说是个 "安全的" 命令。

```
:tabclose       " 关闭当前标签页
```

:tabc[lose][!] {count}
关闭第 {count} 个标签页。失败条件和上面的 :tabclose 相同。

```
:-tabclose      " 关闭上个标签页
:+tabclose      " 关闭下个标签页
:1tabclose      " 关闭首个标签页
:$tabclose      " 关闭末个标签页
:tabclose -2    " 关闭往前第二个标签页
:tabclose +     " 关闭下个标签页
:tabclose 3     " 关闭第三个标签页
:tabclose $     " 关闭末个标签页
```

:tabo :tabonly

:tabo[nly][!] 关闭所有其它的标签页。
如果置位 'hidden' 选项，所有关闭窗口的缓冲区成为隐藏。
如果没有置位 'hidden' 但置位 'autowrite' 选项，写回修改过的缓冲区。否则，包含修改过的缓冲区的窗口不会被删除。但如果给出 [!]，这些窗口成为隐藏。修改过的缓冲区永远不会被放弃，所以永远不被丢失这些改动。

```
:tabonly        " 关闭除了当前页的所有标签页
```

:{count}tabo[nly][!]
:tabo[nly][!] {count}
关闭除了 {count} 以外的所有的标签页。

```
:.tabonly       " 同上
:-tabonly       " 关闭除了上一页之外的所有标签页
:+tabonly       " 关闭除了下一页之外的所有标签页
:1tabonly       " 关闭除了首页之外的所有标签页
:$tabonly       " 关闭除了末页之外的所有标签页
:tabonly -      " 关闭除了上一页之外的所有标签页
:tabonly +2     " 关闭除了往后第二页之外的所有标签页
:tabonly 1      " 关闭除了首页之外的所有标签页
:tabonly $      " 关闭除了末页之外的所有标签页
```

切换到另一个标签页:

使用鼠标：如果显示标签页行，某个标签页标签上单击可以切换到该标签页。没有标签的地方点击则转到下一个标签页。 'tabline'

:tabn[ext] :tabn :tabnext gt <C-PageDown> CTRL-<PageDown> <C-PageDown> gt i_CTRL-<PageDown> i_<C-PageDown>
转到下一个标签页。最后一个标签页则回绕到第一个。

:{count}tabn[ext]
:tabn[ext] {count}

转到第 {count} 个标签页。首个标签页编号为一。

```
:-tabnext   " 转到上个标签页
:+tabnext   " 转到下个标签页
:+2tabnext  " 转到往后第二个标签页
:1tabnext   " 转到首个标签页
:$tabnext   " 转到末个标签页
:tabnext $  " 同上
:tabnext -  " 转到上个标签页
:tabnext -1 " 同上
:tabnext +  " 转到下个标签页
:tabnext +1 " 同上
```

{count}<C-PageDown>
{count}gt 转到第 {count} 个标签页。首个标签页编号为一。

:tabp[revious] :tabp :tabprevious gT :tabN
:tabN[ext] :tabNext CTRL-<PageUp>
<C-PageUp> <C-PageUp> i_CTRL-<PageUp> i_<C-PageUp>
gT 转到前一个标签页。首个标签页则回绕到最后一个。

:tabp[revious] {count}
:tabN[ext] {count}
{count}<C-PageUp>
{count}gT 转到前面 {count} 个标签页。首个标签页则回绕到最后一个。注意 {count} 的用法和 :tabnext 不同，后者它用作标签页号。

:tabr[ewind] :tabfir :tabfirst :tabr :tabrewind
:tabfir[st] 转到第一个标签页。

:tabl :tablast
:tabl[ast] 转到最后一个标签页。

其它命令:

:tabs
:tabs 列出标签页和它们包含的窗口。
当前窗口显示 ">"。
修改过的缓冲区显示 "+"。
例如:

```
Tab page 1
  + tabpage.txt
    ex_docmd.c
Tab page 2
>   main.c
```

重新排列标签页:

:tabm[ove] [N] :tabm :tabmove
:[N]tabm[ove]
把当前标签页移到第 N 个标签页之后。用 0 使当前标签页成为首个标签页。计算 N 是在移动前，所以如果第二个标签页是当前标签页，

‘:tabmove 1‘ 和 ‘:tabmove 2‘ 没有效果。
如果没有 N，当前标签页成为最后一个。

```
:.tabmove    " 什么都不做
:-tabmove    " 往左移动当前标签页
:+tabmove    " 往右移动当前标签页
:0tabmove    " 移动当前标签页到标签页之首
:tabmove 0   " 同上
:tabmove     " 移动当前标签页到标签页之末
:$tabmove    " 同上
:tabmove $   " 同上
```

:tabm[ove] +[N]
:tabm[ove] -[N]
移动当前标签页向右（用 + 的话）或向左（用 - 的话）N 个位置。

```
:tabmove -   " 往左移动当前标签页
:tabmove -1  " 同上
:tabmove +   " 往右移动当前标签页
:tabmove +1  " 同上
```

注意 可以用 :Ntabmove 来移动标签页到第 N 个标签页之后，也可用 :+Ntabmove 来移动 N 个位置。在此情形下 +N 的实际含义可见 [range] 处的解释。

循环遍历所有标签页：

:tabd :tabdo

:[range]tabd[o] {cmd}
对每个标签页执行 {cmd}，如果给出 [range]，只对页号在 [range] 内的标签页执行。
大致相当于：

```
:tabfirst
:{cmd}
:tabnext
:{cmd}
等等
```

只对每个标签页的当前窗口进行操作。如果某个标签页出现错误，不再继续访问其余的标签页。
最后一个标签页（或任何出错的标签页）成为当前标签页。
{cmd} 可以包含 '|' 以连接多个命令。
{cmd} 不能打开标签页、关闭标签页或重排标签页。
另见 :windo 、 :argdo 、 :bufdo 、 :cdo 、 :ldo 、 :cfdo 和 :lfdo

---

3. 其它项目 tab-page-other

tabline-menu

GUI 标签页行提供弹出菜单。用右键点击可以访问。它包含的项目有：

Close	关闭鼠标指针所在的标签页。如果鼠标指针不在任何标签上，关闭当前标签页。
New Tab	打开新标签页并编辑空缓冲区。该标签页出现在鼠标指针的左侧。
Open Tab...	类似于 "New Tab"，但同时提供文件选择器以选择要编辑的文件。

每个标签页对比较模式进行分别的处理。你可以在一个标签页上比较若干文件。而在另一个标签页上比较不同的文件。

局部于标签页的变量以 "t:" 开头。 tabpage-variable

目前只有一个局部于标签页的选项：'cmdheight'。

tabnew-autocmd

TabLeave 和 TabEnter 自动命令事件可以在标签页切换时执行任务。准确的顺序取决于你做的事情。建立新标签页相当于先建立工作于同一缓冲区的新窗口，再在其上编辑别的缓冲区。所以，":tabnew" 依次激活：

WinLeave	离开当前窗口
TabLeave	离开当前标签页
WinEnter	进入新标签页的窗口
TabEnter	进入新标签页
BufLeave	离开当前缓冲区
BufEnter	进入新的空白缓冲区

而切换到另一个标签页的相应顺序是：

```
BufLeave
WinLeave
TabLeave
TabEnter
WinEnter
BufEnter
```

---

4. 设置 _92protect'tabline'

setting-tabline

'tabline' 选项指定标签页行的外观。只有没有 GUI 标签页行时才使用该选项。

用 'showtabline' 选项可以指定何时显示标签页行：从不、多于一个标签页才显示、总是。

标签页行的高亮用 TabLine、TabLineSel 和 TabLineFill 分别指定。 hl-TabLine hl-TabLineSel hl-TabLineFill

包含修改过的窗口的标签页会显示 "+"。同时也显示该标签页里包含的窗口数。所以，"3+" 意味着有三个窗口，其中一个有修改过的缓冲区。

'tabline' 选项允许你定义自己喜欢的标签页标签显示方式。实现并不简单，所以这里给出一个例子。

基本知识见 'statusline' 选项。'tabline' 选项可以使用相同的项目。此外，还可用 tabpagebuflist() 、 tabpagenr() 和 tabpagewinnr() 函数。

因为标签页标签的编号可以不同，你可以指定整个选项为一个函数。就像这样：

```
:set tabline=%!MyTabLine()
```

然后定义 MyTabLine() 函数以列出所有标签页标签。一个方便的方式是把该函数分成两部分：首先遍历所有的标签页并分别定义它们的标签。然后得到每个标签页的标签。

```
function MyTabLine()
```

```
  let s = ''
  for i in range(tabpagenr('$'))
    " 选择高亮
    if i + 1 == tabpagenr()
      let s .= '%#TabLineSel#'
    else
      let s .= '%#TabLine#'
    endif

    " 设置标签页号 (用于鼠标点击)
    let s .= '%' . (i + 1) . 'T'

    " MyTabLabel() 提供标签
    let s .= ' %{MyTabLabel(' . (i + 1) . ')} '
  endfor

  " 最后一个标签页之后用 TabLineFill 填充并复位标签页号
  let s .= '%#TabLineFill#%T'

  " 右对齐用于关闭当前标签页的标签
  if tabpagenr('$') > 1
    let s .= '%=%#TabLine#%999Xclose'
  endif

  return s
endfunction
```

现在为每个标签页调用 MyTabLabel() 函数以得到它们的标签。

```
function MyTabLabel(n)
  let buflist = tabpagebuflist(a:n)
  let winnr = tabpagewinnr(a:n)
  return bufname(buflist[winnr - 1])
endfunction
```

这只是一个产生和缺省值类似的标签页行的简单例子，和缺省的区别是修改过的缓冲区没有加上 + 号，而名字也没有被截短。如果空间不足，你需要用聪明一些的方法来缩短标签宽度。 'columns' 选项可以查看可用的空间。

---

5. 设置 _92protect'guitablabel' setting-guitablabel

如果显示 GUI 的标签页行，'guitablabel' 可以用来指定每个标签页显示的标签。和 'tabline' 不同，后者一次指定整个标签页行，而 'guitablabel' 分别指定每个标签。

'guitabtooltip' 与之非常类似，但用来设置同一标签的工具提示。因为只有在鼠标指针在标签上逗留时它才会出现，所以通常可以长一些。不过，只有部分系统支持。

'statusline' 选项说明本选项值的格式。

用 "%N" 项目可以得到当前的标签页号。计算此选项时， v:lnum 变量也设置为该值。使用文件名的项目引用当前标签页的当前窗口。

注意 此选项不使用语法高亮。也忽略 %T 和 %X 项目。

简单示例，在标签页标签上显示标签页号和缓冲区名：

```
:set guitablabel=%N\ %f
```

模拟 'guitablabel' 缺省值的示例：显示标签页里的窗口数目，如果有修改过的缓冲区，显示 '+'：

```
function GuiTabLabel()
  let label = ''
  let bufnrlist = tabpagebuflist(v:lnum)

  " 如果标签页里有修改过的缓冲区，加上 '+'
  for bufnr in bufnrlist
    if getbufvar(bufnr, "&modified")
      let label = '+'
      break
    endif
  endfor

  " 如果标签页里的窗口数目多于一，加上窗口数目
  let wincount = tabpagewinnr(v:lnum, '$')
  if wincount > 1
    let label .= wincount
  endif
  if label != ''
    let label .= ' '
  endif

  " 附加缓冲区名
  return label . bufname(bufnrlist[tabpagewinnr(v:lnum) - 1])
endfunction

set guitablabel=%{GuiTabLabel()}
```

注意 设置选项值前就必须已经定义好函数。否则会报错，提示函数未知。

如果你希望回到缺省的标签，返回空字符串。

如果你希望显示标签页特定的内容，可以使用局部于标签页的变量。 t:var

vim:tw=78:ts=8:noet:ft=help:norl:

spell.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

拼写检查　　spell

1. 快速入门　　spell-quickstart
2. 拼写检查评注　　spell-remarks
3. 生成拼写文件　　spell-mkspell
4. 拼写文件格式　　spell-file-format

{仅当编译时加入 +syntax 特性才有效}

备注：另有一个 vimspell 插件。如果你安装了，通过 ":help vimspell" 可以了解它。不过你可能更想把这个插件删掉而用 'spell' 选项。后者更好。

---

1. 快速入门　　spell-quickstart　E756

此命令打开拼写检查：

```
:setlocal spell spelllang=en_us
```

它打开 'spell' 选项并指定检查美国英语。

有问题的单词使用下面的高亮：

SpellBad	不能识别的单词（坏词）	hl-SpellBad
SpellCap	开头没有大写的单词	hl-SpellCap
SpellRare	偏僻词	hl-SpellRare
SpellLocal	不符合所选区域的拼写方式	hl-SpellLocal

Vim 只检查单词的拼写，不做语法检查。

如果 'mousemodel' 选项设为 "popup" 并且光标在拼写错误的单词上，或者它被设为 "popup_setpos" 而鼠标指针在拼写错误的单词上，那么弹出的菜单包含一个子菜单用来替换这个坏词。注意：这减慢了弹出菜单出现的速度。GTK 的 备注：直到菜单出现为止不要释放鼠标右键，否则会有问题。

要搜索下一个拼写有问题的单词：

]s　　]s
移动到光标之后下一个拼写有问题的单词。命令前的计数可以用来重复。适用 'wrapscan'。

[s　　[s
类似于 "]s" 但反向搜索。寻找光标之前拼写有问题的单词。不能识别分散在两行的单词，因而可能停在不被高亮为坏词的单词上。但不会停在行首没有大写开头的单词上。

]S
]S 类似于 "]s" 但只检查坏词，不检查偏僻词或其它区域的词。

[S
[S 类似于 "]S" 但反向搜索。

要加入词汇到自定义的单词列表：

zg
zg 把光标所在的单词作为一个好（good）词加入到 'spellfile' 的第一个名字对应的文件。命令前加上计数指示使用 'spellfile' 的第几个项目。计数 2 使用第二个项目。

可视模式下把选择的字符序列作为一个单词（包括空白！）。
如果光标在标为坏词的文本上，使用标注的文本。
否则使用非单词字符分隔的光标所在的单词。

如果该单词在其它拼写文件里被显式标注为坏词，结果无法预测。

zG
zG 同 "zg" 但加入单词到内部单词列表 internal-wordlist 。

zw
zw 类似于 "zg" 但标记单词为坏（错误：wrong）词。
如果单词已经在 'spellfile' 里出现，把它变为注释行。
spellfile-cleanup 说明如何清理这些行。

zW
zW 同 "zw" 但加入单词到内部单词列表 internal-wordlist 。

zuw
zug
zug 撤销 zw 和 zg ，从 'spellfile' 里删除单词。计数的使用方法同 zg 。

zuW
zuG
zuG 撤销 zW 和 zG ，从内部单词列表里删除单词。计数的使用方法同 zg 。

:spe :spellgood
:[count]spe[llgood] {word}
把 {word} 作为一个好词加入到 'spellfile'，类似 zg 。如果没有计数，使用第一个名字项目，计数为 2 使用第二个项目，等等。

:spe[llgood]! {word} 把 {word} 作为一个好词加入到内部单词列表。类似 zG 。

:spellw :spellwrong
:[count]spellw[rong] {word}
把 {word} 作为一个坏（错误：wrong）词加入到 'spellfile'，类似 zw 。如果没有计数，使用第一个名字项目，计数为 2 使用第二个项目，等等。

:spellw[rong]! {word}　把 {word} 作为一个坏（错误：wrong）词加入到内部单词列表。类似 zW 。

:spellra :spellrare

:[count]spellr[are] {word}
把 {word} 作为一个罕见词加入到 'spellfile'，类似 zw 。如果没有计数，使用第一个名字项目，计数为 2 使用第二个项目，等等。

没有普通模式命令来标记单词为罕见词，因为这是很不常用的命令，而所有直观的命令已经被用掉了。如果你要，可以自动增加映射，如：

```
nnoremap z?  :exe ':spellrare  ' . expand('<cWORD>')<CR>
nnoremap z/  :exe ':spellrare! ' . expand('<cWORD>')<CR>
```

:spellundo 、 zuw 或 zuW 可用来撤销此行为。

:spellr[rare]! {word}　把 {word} 作为一个罕见词加入内部单词列表，类似 zW 。

:[count]spellu[ndo] {word} :spellu :spellundo
类似于 zuw ，[count] 的使用同 :spellgood 。

:spellu[ndo]! {word}　类似于 zuW ，[count] 的使用同 :spellgood 。

用上面的命令加入单词到 'spellfile' 后，相关联的 ".spl" 文件会自动更新并重新载入。如果手动修改 'spellfile'，你需要用 :mkspell 命令。以下命令多数情况下应该能够工作：

```
:edit <'spellfile' 里的文件>
(修改拼写文件)
:mkspell! %
```

有关 'spellfile' 格式的详情见 spell-wordlist-format 。

internal-wordlist

置位 'spell' 时，所有缓冲区使用同一个内部单词列表。它不会被保存，退出 Vim 时就会丢失。设置 'encoding' 时它也被清空。

要列出坏词的拼写建议：

z=
z=　为光标所在/之后的单词提供正确拼写单词的建议。即使单词没有被高亮为坏词，它也可用来寻找替代的单词，比如当该单词之后的单词是坏词。
可视模式下使用高亮文本作为要被替代的单词。
结果按和被替代的单词的相似度排序。
可能需时很久。如果等不及，你可以按 CTRL-C。

如果命令没有给出计数，列出所有替代，可以输入编号进行选择，如果不想替换，输入 <Enter>。你可以用鼠标点击你希望的选择（鼠标必须可用于普通模式，而且不能有回绕行）。点击第一行（标题）取消操作。

通常，建议列表用来替换高亮的坏词。有时它们也包含其它文本，这种情况也同时列出替换文本，出现在一个 "<" 之后。

如果使用计数，直接使用该建议，不再提示。例如，"1z=" 总是使用第一个建议。

如果 'verbose' 非零，除了建议以外，同时显示指示和坏词相似程度的评分（分数愈高差别越大）。
如果单词被替换，重复命令 "." 会重复单词的替换。这相当于 "ciw"，跟好词，然后跟 <Esc>。 不 能用于泰语和其它不用空格分隔单词的语言。

:spellr[epall] 在当前窗口重复 z= 进行的替换，即把所有的匹配用替换过的那个单词替换。 :spellr :spellrepall E752 E753

插入模式下，如果光标在坏词之后，用 CTRL-X s 可以寻找建议。这使用插入模式补全的工作方式。CTRL-N 得到下一个建议，CTRL-P 往回走。 i_CTRL-X_s

'spellsuggest' 选项影响建议列表的生成和排序方式。见 'spellsuggest' 。

'spellcapcheck' 选项用来检查句子第一个单词是否以大写字母开头。这不用于文件的首个单词。如果句子之后立即换行，下一行的高亮可能被延迟。如果需要的话，用 CTRL-L 。另见 set-spc-auto 了解如何在设置 'spelllang' 时自动设置本选项。

Vim 计算好词的使用次数并用来对建议进行排序：出现过的单词有较小的奖励，经常见到的单词有较大的奖励。字缀文件里 COMMON 项目也用来定义常用词，这样新文件或短文件都能使用此机制 spell-COMMON 。

---

2. 拼写建议的评注 spell-remarks

性 能

Vim 进行在线拼写检查。要快速工作就需要把单词列表载入内存。这需要很大的内存（1 兆字节或更多）。载入单词列表产生的延迟也很显著，这个过程发生在置位 'spell' 和已经置位了 'spell' 再设置 'spelllang' 的时候。要最大程度减少延迟，每个单词列表只载入一次。'spelllang' 清空或者 'spell' 复位时都不删除它。但设置 'encoding' 时会重载所有的单词列表，这时你也会注意到明显的延迟。

区 域

不同的区域可能使用同一单词不同的拼写形式。例如，英语（至少）有以下变种：

en	所有区域
en_au	澳大利亚
en_ca	加拿大
en_gb	英国
en_nz	新西兰
en_us	美国

不用于指定区域但用于其它区域的单词使用 SpellLocal 高亮 hl-SpellLocal 。

语言和区域名总用小写。

用 zg 和其它命令加入单词时总为所有区域加入单词。如果不想这样，你需要手动修改 'spellfile'。见 spell-wordlist-format 。注意 只有所有 'spelllang' 所有项目 (不包括指定 .spl 名字的文件) 都指定相同区域时，才使用 'spellfile' 文件指定的区域。

spell-german

特例：德语使用如下特殊区域：

de	接受所有的德语单词
de_de	旧式和新式拼写
de_19	旧式拼写
de_20	新式拼写
de_at	奥地利
de_ch	瑞士

spell-russian

特例：俄语使用如下特殊区域：

ru	接受所有的俄语单词
ru_ru	"IE" 字母拼写
ru_yo	"YO" 字母拼写

spell-yiddish

意第绪语 (yiddish) 需要 "utf-8" 编码，因为它使用了特殊字符。如果使用 latin1，Vim 会使用转译的 (罗马化的) 意第绪语。如果你想使用 utf-8 的转译意第绪语，用 "yi-tr"。见下表：

'encoding'	'spelllang'	
utf-8	yi	意第绪语
latin1	yi	转译的意第绪语
utf-8	yi-tr	转译的意第绪语

spell-cjk

中文、日文或其它东亚语字符通常被标为错误，因为拼写检查不支持这些语种。如果 'spelllang' 包含 "cjk"，不把它们标为错误。这可用于编辑带有若干亚洲单词的文本的拼写检查。

拼 写 文 件

spell-load

Vim 在 'runtimepath' 里的 "spell" 子目录查找拼写文件。名字规则是：LL.EEE.spl，其中：

LL	语言名
EEE	'encoding' 值

"LL" 的值来自 'spelllang'，但不包括区域名。例如：

'spelllang'	LL
en_us	en
en-rare	en-rare
medical_ca	medical

只载入 'runtimepath' 里第一个可用的文件。如果成功，载入文件名为 LL.EEE.add.spl 的附加文件。所有找到的都被使用。

如果找不到拼写文件，激活 SpellFileMissing 自动命令事件。它可能会启动 spellfile.vim 插件提供你机会来下载拼写文件。

另外，载入和 'spellfile' 的名字相关的文件，也就是 zg 和 zw 用来加入好词和坏词的文件。

例外：
- 如果 'encoding' 是 "iso-8859-15"，Vim 使用 "latin1"。对拼写来说，欧元符号没有作用。
- 如果 'encoding' 没有对应的拼写文件，Vim 尝试用 "ascii" 代替。这只能用于单词几乎全是 ASCII 字符构成的语言，例如英语。可用 'encoding' 不是 "latin1"，如 iso-8859-2，但要编辑英文文本的场合。 ".add" 文件使用和找到的主拼写文件相同的名字。

例如，如果设置如下：

```
'runtimepath' 是 "~/.vim,/usr/share/vim81,~/.vim/after"
'encoding'    是 "iso-8859-2"
'spelllang'   是 "pl"
```

Vim 会寻找：

```
1. ~/.vim/spell/pl.iso-8859-2.spl
2. /usr/share/vim81/spell/pl.iso-8859-2.spl
3. ~/.vim/spell/pl.iso-8859-2.add.spl
4. /usr/share/vim81/spell/pl.iso-8859-2.add.spl
5. ~/.vim/after/spell/pl.iso-8859-2.add.spl
```

这里假设没找到 1 但找到了 2。

如果 'encoding' 为 "latin1"，Vim 会寻找：

```
1. ~/.vim/spell/pl.latin1.spl
2. /usr/share/vim81/spell/pl.latin1.spl
3. ~/.vim/after/spell/pl.latin1.spl
4. ~/.vim/spell/pl.ascii.spl
5. /usr/share/vim81/spell/pl.ascii.spl
6. ~/.vim/after/spell/pl.ascii.spl
```

这里假设以上文件都找不到（如果去掉所有的非 ASCII 字符，波兰语无法阅读）。

目前不支持 EBCDIC 编码的拼写检查。

当前的 'encoding' 下不一定有可用的拼写文件。 spell-mkspell 说明如何建立拼写文件。用 "iconv" 转换拼写文件 不能 工作！

备注：VMS 上 ".{enc}.spl" 被换名为 "_{enc}.spl"，否则文件名会有问题。

spell-sug-file E781

如果存在和 ".spl" 同名但以 ".sug" 结尾的文件，使用该文件来给出更好的建议。直到需要建议的时候才载入该文件，以减少内存开销。

E758 E759 E778 E779 E780 E782

载入拼写文件时，Vim 检查它的格式是否正确。如果有错，该文件可能已经被清空或修改过，也可能它本来是为其它 Vim 版本设计的。

清 理 拼 写 文 件 spellfile-cleanup

zw 命令会把 'spellfile' 的已有项目变成注释行。这避免了每次都写入一个新文件，

但文件因而只会增长而不会缩短。要清理所有 ".add" 拼写文件的注释行，执行：

```
:runtime spell/cleanadd.vim
```

它删除所有除了 "##" 开头的注释行。"##" 行可以用来假如你想保留的注释。

你可以随时调用此脚本。系统提供了一个变量，用于跳过对最近刚修改过的文件的清理。它设置文件修改以后隔多长时间才需要清理，以秒计。例如，如果只要清理那些最近一个小时没有修改过的文件：

```
let g:spell_clean_limit = 60 * 60
```

缺省值为一秒。

单 词

Vim 使用固定的方法来识别单词，和 'iskeyword' 相互独立。所以该方法适用于帮助文件和 'iskeyword' 里包含 '-' 等字符的语言。单词字符仍然和 'encoding' 有关。

单词字符表格保存在主 .spl 文件里。所以它和生成文件时的当前 locale 有关！不过，.add.spl 文件不包含单词列表。

数位开头的单词总是忽略该数位，除非能够识别整个单词。这样，如果 "3D" 是单词而 "D" 不是，那么识别 "3D" 为一个单词，但如果 "3D" 不是单词，只把 "D" 标识为坏词。能识别 0x12ab 和 0X12AB 形式的十六进制数。

单 词 组 合

拼写检查的单词可以包含空格。这可以用来识别不能单独使用的单词，例如 "et al."。也可以用来识别 "the the" 这样的错误并高亮之。

空格的个数无关紧要。多数情况下换行符也可以出现。不过，这使得判断从哪里开始检查拼写错误出现困难。如果你修改了一行而且只有该行被重画，Vim 不会去看上一行，所以如果 "et" 出现在上一行的尾部，"al." 会被标记为错误。如果你输入 "the<CR>the"，直到第一行被重画，高亮并不出现。要立即重画，用 CTRL-L 。"[s" 也会停在带换行符的单词组合上。

如果遇到换行符，Vim 跳过之后的 '*'、'>' 和 '"' 这样的字符。这使得拼写检查可以在 C、外壳脚本和 Vim 代码里的注释进行。

语 法 高 亮 spell-syntax

使用语法高亮的文件可以指定应在哪里进行拼写检查：

1.	所有地方	缺省
2.	在特定项目里	使用 "contains=@Spell"
3.	除了特定项目的所有地方	使用 "contains=@NoSpell"

第二个方法里，加入 @NoSpell 簇的项目又会屏蔽拼写检查。这可用来，比如说，给程序注释加上 @Spell，而给不应检查拼写的项目加上 @NoSpell。
另见 :syn-spell 了解如何处理不在语法项目里的文本。

VIM 脚 本

如果你想编写和拼写有关的 Vim 脚本，以下函数可能有用：

spellbadword()	寻找光标所在的坏词
spellsuggest()	得到拼写建议列表
soundfold()	得到单词按发音折叠的等价形式

自 动 设 置 'spellcapcheck' set-spc-auto

一旦 'spelllang' 选项被成功设置，Vim 会执行 'runtimepath' 的 "spell/LANG.vim" 文件。"LANG" 是 'spelllang' 中第一个逗号、句号或下划线之前的值。这可用来设置语言特定的选项，尤其是 'spellcapcheck'。

发布中包含了若干这样的文件。用此命令可以看看它们做了什么：

```
:next $VIMRUNTIME/spell/*.vim
```

注意 如果 'spellcapcheck' 被修改而不同于缺省值，这些脚本不对它进行修改。这时假设用户已经有其它的偏好值。

双 重 评 分 spell-double-scoring

'spellsuggest' 选项可用来选择 "双重" 评分。该机制基于以下原则：有两种不同类型的拼写错误：

1. 你知道如何拼写单词，但有些输入错误。这种类型会导致较小的编辑距离（交换/省略/插入的字符数）和一个发音可能完全不同的单词。

2. 你不知道如何拼写单词，所以输入发音类似的形式。这时编辑距离可能很大，但经过按发音折叠以后单词可以很相似。

既然两种错误的评分可以非常不同，我们为每种错误使用一个列表，并把它们合并。

按发音折叠较慢，而通晓语言的用户也不太会犯第二种错误。所以设置 'spellsuggest' 可以让用户选择自己偏好的建议评分方法。

---

3. 生成拼写文件 spell-mkspell

Vim 的拼写文件使用二进制文件格式。这大大加快了单词列表的载入，并减小了文件的大小。

.aff .dic Myspell

你可以用 Myspell 使用的 .aff 和 .dic 文件构造 Vim 的拼写文件。OpenOffice.org 和 Mozilla 使用 Myspell。OpenOffice .oxt 文件是包含 .aff 和 .dic 文件的 zip 压缩包。你可以在此找到那些文件：

```
http://extensions.services.openoffice.org/dictionary
```

如果不行，这里可以找到较旧的 OpenOffice 2 文件：

```
http://wiki.services.openoffice.org/wiki/Dictionaries
```

你也可以使用简单单词列表列表。效果相同，这取决于你能找到什么格式的单词列表。

如果你安装 Aap（来自 www.a-a-p.org），可以利用 runtime/spell/??/ 目录的若干构造文件（recipe）。Aap 会自己下载文件，应用 Vim 必需的补丁并构造 .spl 文件。

确信你正确设置了当前 locale，否则 Vim 不知道什么字符是大写字母，什么字符是小写的。如果不知道 locale (例如，在 Unix 上使用 MS-Windows 代码页)，在 .aff 文件里增加表格 spell-affix-chars 。如果 .aff 文件没有定义表格，使用当前激活的拼写的单词表格。如果拼写也没有激活，Vim 只好试图进行一些猜测。

:mksp :mkspell

:mksp[ell][!] [-ascii] {outname} {inname} ...
从单词列表生成 Vim 的拼写文件。例如:

```
:mkspell /tmp/nl nl_NL.words
```

E751

如果 {outname} 以 ".spl" 结尾，它被用作文件名。否则，它指定诸如 "en" 这样的语言名，不带区域名。要写入的文件名是 "{outname}.{encoding}.spl"，其中 {encoding} 是 'encoding' 选项的值。

如果输出文件已存在，必须用 [!] 才能进行覆盖。

如果给出 [-ascii] 参数，跳过包含非 ASCII 字符的单词。产生的文件名以 "ascii.spl" 结尾。

输入可以是 Myspell 格式的文件 {inname}.aff 和 {inname}.dic。如果 {inname}.aff 不存在，输入也可以用文件名为 {inname} 的普通单词列表文件。

可以给出多个 {inname} 参数，把多个区域合并到一个 Vim 拼写文件。例如:

```
:mkspell ~/.vim/spell/en /tmp/en_US /tmp/en_CA /tmp/en_AU
```

合并美加澳三地的英语单词列表成为单个 en.spl 文件。
至多可以合并八个区域。 E754 E755
使用第一个包含 REP 和 SAL 项目的 .aff 文件的这些项目。
spell-REP spell-SAL

E845

该命令需要大量内存，用于寻找最优单词树 (波兰语、意大利语和匈牙利语需要几百兆字节)。最终结果因为使用压缩的缘故会小得多。为了避免内存溢出，压缩会时不时进行。使用 'mkspellmem' 选项可以对此行为进行调节。

拼写文件写回后，在缓冲区使用时会自动重新载入。

:mksp[ell] [-ascii] {name}.{enc}.add
类似于上面的 ":mkspell"，使用 {name}.{enc}.add 作为输入文件，并在相同目录产生在后面附加 ".spl" 的输出文件。

:mksp[ell] [-ascii] {name}
类似于上面的 ":mkspell"，使用 {name} 作为输入文件，并在相同目录产生在后面附加 ".{enc}.spl" 的输出文件。

Vim 会报告重复单词的个数。这可能是单词列表里的错误，但有时一个基本字可以加上不同的前缀和后缀以免把它们合并在一起 (例如，捷克语就这么用)。如果你想让 Vim 报告所有的重复单词，设置 'verbose' 选项。

如果为了 Vim 的使用，你想修改 Myspell 单词列表，我们建议以下步骤:

1. 从 Myspell 获取 xx_YY.aff 和 xx_YY.dic 文件。
2. 把这些文件备份为 xx_YY.orig.aff 和 xx_YY.orig.dic。
3. 修改 xx_YY.aff 和 xx_YY.dic 文件以删除坏词、增加新词、定义 FOL/LOW/UPP 的单词字符，等等。可以考虑使用发布中的 "*.diff" 文件。
4. 用正确的 locale 启动 Vim，然后用 :mkspell 生成 Vim 拼写文件。
5. 如果拼写文件被写到 'runtimepath' 里的 spell 目录，这个命令可以测试该文件: ":set spell spelllang=xx"。如果要写到别处，用 ":set spelllang=xx.enc.spl"。

如果 Myspell 的文件有更新，你可以把差异合并进来:
1. 取得新的 Myspell 文件，假设名为 xx_YY.new.aff 和 xx_UU.new.dic。
2. 用 Vimdiff 来对改变的内容进行比较:

```
vimdiff xx_YY.orig.dic xx_YY.new.dic
```

3. 选择 xx_YY.dic 里你喜欢的改变。xx_YY.aff 可能也需要修改。
4. 把 xx_YY.new.dic 换名为 xx_YY.orig.dic，xx_YY.new.aff 换为 xx_YY.orig.aff。

拼 写 文 件 版 本

E770 E771 E772

拼写检查是 Vim 较新的一个特性，因此将来 .spl 文件格式可能会修改以支持更多的语言。Vim 会检查拼写文件的合法性并报告可能的错误。

E771: Old spell file, needs to be updated
拼写文件比你的 Vim 版本要旧。你需要更新 .spl 文件。

E772: Spell file is for newer version of Vim
拼写文件比你的 Vim 版本要新。你需要更新 Vim。

E770: Unsupported section in spell file
这意味着拼写文件是为 Vim 的后来版本设计的，因为它包含了拼写文件需要的（但目前不支持的）一个小节。这种情况下最好升级 Vim。

拼 写 文 件 转 储

如果为了某些原因你需要检查当前使用的拼写文件支持哪些单词，用此命令:

:spelldump :spelld

:spelld[ump] 打开新窗口，用当前所有的合法单词填充。不包含复合词。
注意: 对某些语言结果可能很大，甚至会使 Vim 溢出内存。

:spelld[ump]! 类似于 ":spelldump"，同时包含单词计数，也就是在更新屏幕时找到每个单词的个数。COMMON 项目里的单词的初始计数为 10。

使用单词列表格式 spell-wordlist-format 。你可以用 ":mkspell" 来阅读它并生成包含所有单词的 .spl 文件。

如果 'spelllang' 里的所有项目使用相同区域或者不使用任何区域，转储的单词里包含区域信息。否则，只包含当前区域里的单词，并且不生成 "/regions" 行。

该 .spl 文件里，带有 .spl 文件名的注释行头部出现在所有生成的单词之前。

拼 写 文 件 丢 失

spell-SpellFileMissing spellfile.vim

如果使用语言的拼写文件不存在，报错。但如果激活了 "spellfile.vim" 插件，它会提供你下载拼写文件的机会。你只要按照指示去做就可以了，它会告诉你把文件写到哪里（为此，'runtimepath' 中必须要有一个可写的目录）。

包含在该插件里用于寻找拼写文件的缺省位置使用 Vim 的 ftp 服务器。如果你想用其它位置或者其它协议，设置 g:spellfile_URL 变量为保存拼写文件目录的位置。 netrw 插件用来得到该文件，那里提供了 URL 的格式说明。例如：

```
let g:spellfile_URL = 'http://ftp.vim.org/vim/runtime/spell'
```

你可能需要转义字符。

关于下载语言，该插件只会询问你一次。如果你想再做一次，重启 Vim 或者设置 g:spellfile_URL 为其它值（比如，在前面加一个空格）。

如果不想使用 "spellfile.vim" 插件，在 vimrc 文件里加入：

```
let loaded_spellfile_plugin = 1
```

除了用插件以外，定义 SpellFileMissing 自动命令也可以让你自己处理丢失的文件。你可以这么做：

```
:au SpellFileMissing * call Download_spell_file(expand('<amatch>'))
```

这里 <amatch> 项目包含语言名。另一个重要的值是 'encoding'，因为每个编码都有自己的拼写文件。有两个例外：

- ISO-8859-15 (latin9) 使用名字 "latin1"（两者的区别只在于一些不在字典单词里的字符上）。
- 某些大多数单词里只使用 ASCII 字符的语言也可以使用名字 "ascii"。

缺省 "spellfile.vim" 插件使用此自动命令，如果在它之后你要定义自动命令，用 ":au! SpellFileMissing" 可以否决它的设置。如果你在插件之前定义自动命令，该插件会 注意 到这一点而不做任何事。

E797

注意 SpellFileMissing 自动命令不能改变或破坏用户正在编辑的缓冲区。

---

4. 拼写文件格式 spell-file-format

这里包含用户为建立并维护单词列表使用的文件格式。

注意 我们这里避免使用单词 "字典"。这是因为拼写检查的目标和编写（成册的）字典有所不同。对于拼写，我们需要正确单词的列表，这些单词因而不会被高亮。字典里不会包含人名和公司名，但单词列表里会。有些古老的词汇现在已经罕用，而它们往往是常用词的错误拼写。它们在字典出现，但不在单词列表里。

有两种格式：直接的单词列表和使用字缀压缩（affix compression）的列表。Myspell（Mozilla 和 OpenOffice.Org）使用字缀压缩的文件。它需要两个文件，分别带 .aff 和 .dic 扩展名。

直接单词列表格式 spell-wordlist-format

每行一个单词。就需要这么多。

此外，还识别以下项目：

- 忽略空行和空白行。

```
# comment
```

- 忽略 # 开头的行（注释行）。

```
/encoding=utf-8
```

- 在任何单词之前的以 "/encoding=" 开头的行指定文件的编码。'=' 之后是编码名。它告诉 Vim 如何设定从指定编码到 'encoding' 的转换。这样，同一单词列表可用于多个目标编码。

```
/regions=usca
```

- "/regions=" 开头的行指定支持的一到多个区域名。每个区域名必须是两个 ASCII 字符。第一个区域是区域 1。所以 "/regions=usca" 包含区域 1 "us" 和区域 2 "ca"。附加单词列表的区域名必须和主单词列表的相同！

- 其它 '/' 开头的行保留将来使用。忽略不识别的行。但你会得到警告，让你知道发生了一些事。

- 单词后面可以跟 "/" 加以下的项目：

=	大小写必须准确匹配。
?	偏僻词。
!	坏（错误）词。
1 到 9	包含该单词的合法区域。如果不指定区域，该词在所有区域均合法。

示例：

# 这是示例单词列表	注释
/encoding=latin1	文件编码
/regions=uscagb	区域 "us"、"ca" 和 "gb"
example	所有区域可用的单词
blah/12	区域 "us" 和 "ca" 可用的单词
vim/!	坏词
Campbell/?3	区域 3 "gb" 的偏僻词
's mornings/=	保持大小写的单词

注意 如果使用 "/="，不能接受再次出现的同一单词但使用全大写的形式。这和混合大小写的单词不同，后者自动标注为保持大小写，但可以用全大写的形式出现。

.AFF 和 .DIC 文件格式 aff-dic-format

有两个文件：基本单词列表和字缀文件。字缀文件指定语言的设定，也可以包含字缀。字缀用于修改基本单词来得到完整的单词列表。这样显著地减少了单词的数目，对波兰语这样的语言尤其如此。这被称为字缀压缩。

基本单词列表和字缀文件用 ":mkspell" 命令合并以产生二进制的拼写文件。因为已经经过预处理，此文件的载入很快。二进制文件格式在源代码中描述（src/spell.c）。但只有开发者需要知道它的信息。

预处理过程也使得我们在构造 Vim 单词列表之前可以接受 Myspell 语言文件并进行一定修改。这个工具可以在 "src/spell" 目录里找到。

字缀和单词列表文件基于 Myspell 使用的文件 (Mozilla 和 OpenOffice.org 的拼写检查器)。这里给出它们的描述:

http://lingucomponent.openoffice.org/affix.readme

注意 字缀是大小写敏感的，上面的描述里对此并不明确。

Vim 支持不少附加成分。下面给出描述 spell-affix-vim 。已经作出努力，使文件格式和其它拼写检查器使用的兼容，因此常常可以直接使用相同的文件。比 Myspell 提供更多功能的另一开发项目是 Hunspell (http://hunspell.sf.net)。

单词列表格式 spell-dic-format

简单例子，附注行号:

```
1       1234
2       aan
3       Als
4       Etten-Leur
5       et al.
6       's-Gravenhage
7       's-Gravenhaags
8       # 区域间有差异的词
9       kado/1
10      cadeau/2
11      TCP,IP
12      /使用 S 字缀可以加上 's'
13      bedel/S
```

第一行包含单词个数。Vim 忽略，但如果没有会报错。 E760

其后，每个单词一行。忽略行尾的空格，其它地方出现的空格仍然有意义。字缀文件指定编码 spell-SET 。

注释行以 '#' 或 '/' 开始。参见示例行 8 和 12。注意 不能 在单词之后加上注释:

```
someword    # 这里的注释会出错!
```

单词之后有一个可选的斜杠与标志位。多数标志位是用来指示该词可以使用哪些字缀的字母。.aff 文件的 SFX 和 PFX 说明它们的用法，见 spell-SFX 和 spell-PFX 。Vim 也允许使用由字缀文件的 FLAG 项目里指定的其它标志位类型 spell-FLAG 。

如果单词只有小写字母，同时匹配第一个字母大写的形式。

如果单词里包含大写字母，相应位置必须大写。不能匹配同一位置使用小写形式的相同单词。同样，也不能匹配将其它位置 (本是小写) 的字母变成大写的形式。

总能接受全部大写的单词形式。

单词列表	匹配	不匹配
als	als Als ALS	ALs AlS aLs aLS
Als	Als ALS	als ALs AlS aLs aLS
ALS	ALS	als Als ALs AlS aLs aLS
AlS	AlS ALS	als Als ALs aLs aLS

可以用 KEEPCASE 的字缀标识符来指定某个单词只能接受大小写完全匹配的形式，见下 spell-KEEPCASE 。

注意：行 5 和 7 使用了非单词字符。你可以在单词里包含任何字符。检查文本时，单词只有在前后都是非单词字符的时候才可能匹配。Myspell 也许不能用非单词字符开始的单词。

行 12 (译者注：应为行 11) 定义单词 "TCP/IP"。因为斜杠有特殊含义，这里使用逗号代替。它定义在字缀文件的 SLASH 项目里，见 spell-SLASH 。注意 没有这个 SLASH 项目，匹配的单词将是 "TCP,IP"。

字缀文件格式

spell-aff-format spell-affix-vim

spell-affix-comment

.aff 文件的注释行以 '#' 开始：

```
# 注释行
```

带有固定数目参数的项目之后可以有注释，但不能有包含空白的参数。注释必须以 "#" 字符开始。例如：

```
KEEPCASE =  # fix case for words with this flag
```

编码

spell-SET

字缀文件可以使用任何 "iconv" 支持的编码。但在有些情况下，调用 :mkspell 时必须设置正确的当前 locale。加入 FOL/LOW/UPP 行可以取消这个限制 spell-FOL 。

编码应该在任何依赖于编码的内容之前指定。此编码将同时应用于字缀文件和字典文件。用 SET 行设置编码：

```
SET utf-8
```

此编码可以和 ":mkspell" 使用时的 'encoding' 选项值不同。Vim 这时会把所有内容转换为 'encoding' 并生成适用于 'encoding' 的拼写文件。如果有些使用的字符不能转换为 'encoding'，报错。

spell-affix-mbyte

如果使用多字节编码，可以使用更多不同的字缀标志位。Myspell 不支持这些，因而你也许又不愿意这样用。为了兼容性起见，使用 8 位编码。

信息

字缀文件的这些项目可以用来为拼写文件加入信息。这里没有格式的限制，但正确的编码仍然是必要的。

spell-NAME spell-VERSION spell-HOME
spell-AUTHOR spell-EMAIL spell-COPYRIGHT

```
NAME            语言名
VERSION         1.0.1  with fixes
HOME            http://www.myhome.eu
```

```
AUTHOR          John Doe
EMAIL           john AT Doe DOT net
COPYRIGHT       LGPL
```

这些字段被按原样放到 .spl 文件里。 :spellinfo 命令可以查看这些信息。

:spellinfo :spelli

:spelli[nfo] 显示当前缓冲区使用的拼写文件的信息。

字 符 表 格

spell-affix-chars

如果使用 8 位编码，字缀文件应该定义哪些字符是单词字符。这是因为使用 ":mkspell" 的系统可能不支持此编码的 locale，因而 isalpha() 不一定工作。Unix 上用 "cp1250" 编码就是如此。

E761 E762 spell-FOL spell-LOW spell-UPP

字缀文件需要如下三行。简单的例子：

```
FOL  á
LOW  á
UPP
```

这三行必须使用完全相同的字符数。

"FOL" 行指定大小写合并字母。在忽略大小写的情况下，它们用来比较单词。多数编码里和小写字母行完全相同。

"LOW" 行指定小写字母。多数情况下和 "FOL" 行等同。

"UPP" 行指定大写字母。也就是，如果一个字符和 "FOL" 相同位置的字符不同，它就是大写字母。

德语的 sharp s 是个特例。它的大写形式是 "SS"。在 FOL/LOW/UPP 行里应该包含它以识别为单词字符，但在所有三行上，用 字符代替。

这里应该省略 ASCII 字符。不管怎么指定，Vim 总是以相同的方式处理它们。
如果编码为 UTF-8，不需要指定单词字符。

E763

Vim 允许在相同文件里为多种语言进行拼写检查。它们在 'spelllang' 选项里列出。为此，相同编码的所有拼写文件必须使用相同的单词字符，否则不能保证它们合并使用而不产生错误。

如果你得到 E763 警告说单词表格不一致，可能需要更新 .spl 文件。如果你是下载得到的文件，下载所有拼写文件的最新版本。如果只用一种语言，如德语，请同时下载最近的英语拼写文件。否则重新用 :mkspell 生成 .spl 文件。如果还有错误，请检查所用的 .aff 文件里的 FOL、LOW 和 UPP 三行。

用 "-ascii" 参数生成的 XX.ascii.spl 拼写文件不包含字符表格，所以它可以和任何编码的拼写文件合并使用。.add.spl 文件同样如此。

词中字符

spell-midword

有些字符只有在两个平常的单词字符之间才被认为是单词字符。单引号是一例：我们经常把文本放在单引号里，所以它不应该被认为是单词字符，但出现在单词字符之间的单引号又应该算。这样做的目的是为了识别诸如 they'are 这样的拼写错误。正确的形式应该是 they're，但既然 "they" 和 "are" 本身都是单词，不这么做就发现不了此错误。

.aff 文件里用 MIDWORD 定义这些字符。例如：

```
MIDWORD '-
```

标 志 位 类 型

spell-FLAG

标志位用来指定单词可用的字缀和其它属性。通常使用单字符的标志位。这限制了可用的标志位数，尤其是对 8 位编码而言。如果需要更多的字缀，可使用 FLAG 项目。可能的值包括：

FLAG long	使用两字符标志位
FLAG num	使用数值，从 1 到 65000
FLAG caplong	使用不含 A-Z 的单字符标志位和 A-Z 开始的两字符标志位

如用 "FLAG num"，代表字缀的数值列表需要用逗号分隔："234,2143,1435"。此方法效率不高，但方便程序自动生成文件。

如用 "caplong"，两字符标志位都以大写字母开头："Aa"、"B1"、"BB" 等。一般用法是单字符标志位用于常用项目，而两字符标志位用于不常用的项目。

注意：如果使用 utf-8，标志位只能使用不超过 65000 的字符值。

注意：即使使用 "num" 或 "long"，复合词和前缀的标志位限定在大约 250 个。

后 缀

spell-PFX spell-SFX

支持常见的 PFX（前缀）和 SFX（后缀）行（见 Myspell 文档或 Aspell 手册：http://aspell.net/man-html/Affix-Compression.html）。

小结：

```
SFX L Y 2
SFX L 0 re [^x]
SFX L 0 ro x
```

第一行是标题行，包含四个字段：

```
SFX {flag} {combine} {count}
```

{flag} 后缀使用的名字。多数是单个字符，但其它形式也是可能的，见 spell-FLAG 。

{combine} 可为 'Y' 或 'N'。如果为 'Y'，加上后缀的单词可以有前缀。如果为 'N'，不再允许前缀。

{count} 后面跟随的行数。如果不正确，报错。

PFX 使用的字段完全相同。

后面跟随的行使用的基本格式是：

```
SFX {flag} {strip} {add} {condition} {extra}
```

{flag}　必须和首行使用的 {flag} 相同。

{strip}　从基本字删除的字符序列。并不检查是否真有这些字符，只使用其长度（以字节计）。最好和 {condition} 匹配，否则会发生奇怪的行为。如果 {strip} 长度等于或大于基本字的长度，不能使用该后缀。如果 {strip} 为 0（零），不删除任何字符。

{add}　加入基本字的字符序列，发生在删除 {strip} 之后。可选后跟 '/' 带标志位。这些标志位应用于加上字缀的单词。见 spell-affix-flags

{condition}　简单模式。基本字只有匹配此模式才能使用该后缀。常用于不同结尾的单词使用单个后缀字符时使用不同的 {add} 和 {strip} 字段的情况。如果 {condition} 为一个 .（句号），没有任何条件。
此模式可以包含：
- 按本义出现的字符。
- [] 里的字符集。[abc] 匹配 a、b 和 c。可使用连字符表示范围 [a-c]，但只用于 Vim。
- ^ 开始的字符集取反指定的字符。[^abc] 匹配任何除了 a、b 和 c 以外的字符。

{extra}　可选的附加文本：

# 注释	忽略注释
-	Hunspell 使用，忽略

PFX 使用相同的字段，但 {strip}、{add} 和 {condition} 应用于单词的头部。

注意：Myspell 忽略相关信息之后的任何附加文本。Vim 需要这些文本之前有 "#"，这样不会错过可能的错误。例如：

```
SFX F 0 in   [^i]n      # Spion > Spionin
SFX F 0 nen  in         # Bauerin > Bauerinnen
```

不过要避免为 Myspell 写成的 affix 文件产生许多错误，可加上 IGNOREEXTA 标志位。

Myspell 看来允许同一个字缀名使用多于一次。因为这种使用方法可能是因为不小心造成的错误，Vim 检查（标题行上）额外的 "S" 标志。Myspell 使用此特性的字缀文件看来都有此标志位。例如：

```
SFX a Y 1 S
SFX a 0 an .

SFX a Y 2 S
SFX a 0 en .
SFX a 0 on .
```

字缀标志位　spell-affix-flags

此特性来自 Hunspell：字缀可以指定标志位，和基本字标志位的使用方法类似。这些标志位适用于基本字加上字缀后的单词（但有限制）。例如：

```
SFX S Y 1
SFX S 0 s .

SFX A Y 1
SFX A 0 able/S .
```

如果字典文件包含 "drink/AS"，以下单词是合法的：

```
drink
drinks          使用 S 后缀
drinkable       使用 A 后缀
drinkables      使用 A 后缀然后 S 后缀
```

一般来说，后缀标志位和基本字标志位都应用于加上后缀后的单词。但只能用基本字标志位指定一个字缀，唯一的特例是可以同时使用一个前缀和一个后缀，条件是两者都支持组合 (combine)。

特别地，字缀的标志位可用于：
- 后缀的后缀，见上例。只能用一次，这样一个单词可用两个后缀 (加上一个前缀)。
- 使带该字缀的单词成为偏僻词。 spell-RARE 标志位。
- 禁止带该字缀的单词进行复合。 spell-COMPOUNDFORBIDFLAG 标志位。
- 允许带该字缀的单词在字缀的这边进行复合。 spell-COMPOUNDPERMITFLAG 标志位。
- 使用 NEEDCOMPOUND 标志位：加上字缀的单词只能用作复合词的一部分。 spell-NEEDCOMPOUND
- 复合词标志位：加上字缀的单词可已出现在复合词的尾部、中间、头部等位置。这些标志位和基本字标志位合并。 spell-compound
- NEEDAFFIX：需要另一个字缀来构成一个合法的单词。
- CIRCUMFIX，下面立即解释。

IGNOREEXTRA spell-IGNOREEXTRA

通常 Vim 对不以 '#' 开始的额外字段报错，以确保不会错过任何错误。不过，有些为 Myspell 或 Hunspell 而生成的文件可能带有不少有额外字段的项目。用 IGNOREEXTRA 标志位可避免过多的错误。

CIRCUMFIX spell-CIRCUMFIX

CIRCUMFIX (环缀) 标志位意味着必须同时加入一个前缀和一个后缀。如果使用有 CIRCUMFIX 标志位的前缀，只能加入带有 CIRCUMFIX 标志位的后缀。反之亦然。(译者注：CIRCUMFIX 的使用方法和 KEEPCASE spell-KEEPCASE 等相同。)
一个替代的方法是只指定后缀，并为该后缀指定两个标志位：所需前缀和 NEEDAFFIX 标志位。 spell-NEEDAFFIX

PFXPOSTPONE spell-PFXPOSTPONE

如果字缀文件有许多前缀，而它们又应用于许多单词，你无法在内存里构造完整的单词列表。希伯来语正是如此 (包含所有单词的列表将超过 1 Gbyte)。这种情况下必须延迟前缀的应用。此方法使拼写检查变慢。.aff 文件用此关键字指示使用此方法：

```
PFXPOSTPONE
```

只能延迟没有删除字符串并且没有标志位的前缀。带有删除字符串或标志位的前缀仍然会被包含在单词列表里。但如果该删除字符串只有一个字符并等于附加字符串的最后一个字符但以小写形式出现的话例外。此时如果使用该删除字符串，后面的单词可以用大写字母开始。

带斜杠的单词 spell-SLASH

`.dic` 文件使用斜杠分隔基本字与字缀字符以及其它标志位。不幸的是，这意味着你不能在单词里使用斜杠。这样，`"TCP/IP"` 不是一个单词，而是 `"TCP"` 并带标志位 `"IP"`。要在单词里包含斜杠，在它之前加上反斜杠。如果罕见地你想在单词里用反斜杠，用两个反斜杠就可以。其它反斜杠的使用方式为将来的扩展保留。
(译者注：此处似乎不需要特别的 `SLASH` 项目，但 spell-dic-format 有相关暗示)

保持大小写的单词 spell-KEEPCASE

字缀文件可用 `KEEPCASE` 行定义使单词保持大小写的字缀名。例如：

```
KEEPCASE =
```

`Myspell` 不支持此标志位。此标志位的出现意味着大小写相关。
这可用于句首出现的单词不需要首字母大写的情况。例如：

单词列表	匹配	不匹配
`'s morgens/=`	`'s morgens`	`'S morgens 's Morgens 'S MORGENS`
`'s Morgens`	`'s Morgens 'S MORGENS`	`'S morgens 's morgens`

此标志位也可用来避免匹配单词的全大写字母形式。

偏僻词 spell-RARE

字缀文件可用 `RARE` 行来定义偏僻词使用的字缀名。例如：

```
RARE ?
```

偏僻词和坏词的高亮方法不同。它用来标识对语言而言正确但几乎没人用的单词，因而这可能是其它单词输入错误的结果。如果相同单词已经被认为是好词，它不再被高亮为偏僻词。

此标志位也用于字缀，这时，基本字不罕见，但加上字缀后却是 spell-affix-flags 。不过，如果其它方式已经使该词以好词出现（例如，属于其它区域），它不会被标记为偏僻词。

坏词 spell-BAD

字缀文件可以使用 `BAD` 行来定义坏词使用的字缀名。例如：

```
BAD !
```

它可以用来排除本来是好词的单词。例如，`.dic` 文件里的 `"the the"`：

```
the the/!
```

一旦一个单词被标为坏词，即使遇到相同的词被标为好词的情况，也不能再撤销。

此标志位也可用于带字缀的单词，因而可以用来标注一系列相关的词为坏词。

spell-FORBIDDENWORD

FORBIDDENWORD 等同于 BAD。为了保持和 Hunspell 的兼容而设。

spell-NEEDAFFIX

NEEDAFFIX 标志位用来指定单词必须使用字缀。该单词本身不被认为是好词 (除非有空字缀)。例如:

```
NEEDAFFIX +
```

复 合 词 spell-compound

复合词是通过连接 .dic 文件里出现的单词而成的更长的词。要指定那些单词可以连接，要用一个字符。该字符放在单词之后的字缀列表里。这里我们把这个字符叫做标志位。显然，这些标志位必须不能和任何字缀标识符冲突。

spell-COMPOUNDFLAG

Myspell 兼容方法使用 COMPOUNDFLAG 指定的标志位。带该标志位的所有单词可以用任何顺序进行合并。这样无法实现哪个单词必须出现在前等控制。例如:

```
COMPOUNDFLAG c
```

spell-COMPOUNDRULE

更高级的方式是用带有多种标志位的多种项目来指定如何构成复合词。这和 Myspell 3.0 不兼容。让我们从下例开始:

```
COMPOUNDRULE c+
COMPOUNDRULE se
```

第一行定义带有 "c" 标志位的单词可以用任何顺序进行来连接。第二行定义由一个带有 "s" 标志位和一个带有 "e" 标志位的单词可以构成复合词。假定使用下面的字典:

```
bork/c
onion/s
soup/e
```

你可以构造以下单词:

```
bork
borkbork
borkborkbork
(等等)
onion
soup
onionsoup
```

COMPOUNDRULE 项目可以出现多次。它的参数包含一或多个组。每组可以是:

一个标志位	例如，c
[] 里多选一的标志位	例如，[abc]

可选地，后面可跟:

*	该组出现零或多次，例如，sm*e
+	该组出现一或多次，例如，c+

?　　该组出现零或一次，例如，x?

这和正规表达式的语法相似（但不完全相同！）。一些例子如下，这里同时列出它们所要求的标志位序列：

```
COMPOUNDRULE x+         x xx xxx 等
COMPOUNDRULE yz         yz
COMPOUNDRULE x+z        xz xxz xxxz 等
COMPOUNDRULE yx+        yx yxx yxxx 等
COMPOUNDRULE xy?z       xz xyz

COMPOUNDRULE [abc]z     az bz cz
COMPOUNDRULE [abc]+z    az aaz abaz bz baz bcbz cz caz cbaz 等
COMPOUNDRULE a[xyz]+    ax axx axyz ay ayx ayzz az azy azxy 等
COMPOUNDRULE sm*e       se sme smme smmme 等
COMPOUNDRULE s[xyz]*e   se sxe sxye sxyxe sye syze sze szye szyxe  等
```

特别例子：复合词需要由两个单词和一个连字符构成：

.aff 文件：

```
COMPOUNDRULE sde
NEEDAFFIX x
COMPOUNDWORDMAX 3
COMPOUNDMIN 1
```

.dic 文件：

```
start/s
end/e
-/xd
```

这样，单词 "start-end" 可以接受但 "startend" 不行。

还有一个隐含的规则，如果没有更多标志位，带前缀的单词不能在另一个单词之后复合，同样，带有后缀的单词不能在另一个单词之前复合。也就是，字缀不能出现在复合词的中间。这一规则用 spell-COMPOUNDPERMITFLAG 可以改变。

spell-NEEDCOMPOUND

NEEDCOMPOUND 标志位要求单词必须用作复合词的一部分。该单词本身不作为好词。例如：

```
NEEDCOMPOUND &
```

spell-ONLYINCOMPOUND

ONLYINCOMPOUND 等同于 NEEDCOMPOUND。为保持和 Hunspell 兼容性而设。

spell-COMPOUNDMIN

用于复合词的最短字符长度用 COMPOUNDMIN 指定。例如：

```
COMPOUNDMIN 5
```

如果省略，没有最短长度。显然，不给短词加上复合用的标志位就可以了。保留此特性是为了和 Myspell 兼容。

spell-COMPOUNDWORDMAX

用于复合词的最大单词长度用 COMPOUNDMAX 指定。例如：

```
COMPOUNDWORDMAX 3
```

如果省略，没有最大长度。适用于所有的复合词。

要设置带有特定标志位的单词的限制，确信在 COMPOUNDRULE 出现的项目不允许太多的单词。

spell-COMPOUNDSYLMAX

复合词包含的最大音节数用 COMPOUNDSYLMAX 指定。例如：

```
COMPOUNDSYLMAX 6
```

如果没有 SYLLABLE 项目，没有任何效果。如果没有 COMPOUNDSYLMAX，没有音节数的限制。

如果同时定义了 COMPOUNDWORDMAX 和 COMPOUNDSYLMAX，接受满足其中任何一个条件的单词，也就是说，或者不超过 COMPOUNDWORDMAX 个单词，或者不超过 COMPOUNDSYLMAX 个音节。

spell-COMPOUNDFORBIDFLAG

COMPOUNDFORBIDFLAG 指定可用于字缀的标志位。它意味着加上字缀的单词不能用于复合词。例如：

字缀文件：

```
COMPOUNDFLAG c
COMPOUNDFORBIDFLAG x
SFX a Y 2
SFX a 0 s    .
SFX a 0 ize/x .
```

字典：

```
word/c
util/ac
```

允许 "wordutil" 和 "wordutils"，但不允许 "wordutilize"。

备注：这还不能用于延迟前缀。

spell-COMPOUNDPERMITFLAG

COMPOUNDPERMITFLAG 指定可用于字缀的标志位。它意味着加上字缀的单词可以用于复合词，其中该字缀可以出现在单词中间。没有此标志位，这是不允许的。

备注：这还不能用于延迟前缀。

spell-COMPOUNDROOT

COMPOUNDROOT 标志位用来指定字典里已经是复合词的单词。检查复合词规则时，它可以被计算为两个单词。也可用于字缀，使得该字缀成为一个复合词。

spell-CHECKCOMPOUNDPATTERN

CHECKCOMPOUNDPATTERN 用于定义这样的模式，在两个词复合的位置匹配时禁止该复合词。示例：

```
CHECKCOMPOUNDPATTERN o e
```

如果第一个词以 "o" 结尾而第二个词以 "e" 开始，禁止它们的复合。

虽然名字叫模式，参数必须是普通的文本而不是什么模式。总是忽略大小写。

不支持 Hunspell 带三个参数和标志位的功能。

spell-NOCOMPOUNDSUGS

此项目指定不使用复合词作为提示。这可用于复合词用于很短甚至单个字符的单词上。如从数位中提取数值的场合。没有此标志位，提示的建立会花大部分时间在尝试各种奇奇怪怪的复合词上。

```
NOCOMPOUNDSUGS
```

spell-SYLLABLE

SYLLABLE 项目定义用于计算单词音节数的字符或字符序列。例如：

```
SYLLABLE aáeéiíoóuúüy/aa/au/ea/ee/ei/ie/oa/oe/oo/ou/uu/ui
```

第一个斜杠之前是当作一个音节的字符的集合，即使它们被重复和混合也是如此，直到下一个字符不属于此集合为止。该斜杠之后列出当作一个音节的字符序列。它们比使用集合里的字符优先。用上面这个示例，"ideeen" 有三个音节，分别包含 "i"、"ee" 和 "e"。

这里只需要包含大小写合并字母。

上面已经提到了另一个限制复合的方法：为字缀加上 spell-COMPOUNDFORBIDFLAG 标志位，这样所有带有该字缀的单词都不用于复合。

无限复合

spell-NOBREAK

有些语言，比如泰语，使用的单词之间没有空格。这就像所有单词都进行了复合。要指定此行为，使用字缀文件的 NOBREAK 项目，没有参数：

```
NOBREAK
```

Vim 会试图判断哪里单词结束而下一个单词开始。如果有拼写错误，这不一定能做得很好。

spell-COMMON

可用 COMMON 项目指定常用词。编辑短文件时，它会提供更好的建议。例如：

```
COMMON  the of to and a in is it you that he was for on are
```

这些单词必须用空格分隔，一行至多 25 个。
如果 ":mkspell" 命令指定多个区域，所有区域的常用词进行合并并用于所有区域。

spell-NOSPLITSUGS

此项目指示不要分割单词来提供建议。分割单词的建议只有在相近词很少时才会出现。

```
NOSPLITSUGS
```

spell-NOSUGGEST

NOSUGGEST 指定的标志位可用来不参与建议的单词。可用于不洁词汇。

```
NOSUGGEST %
```

替换

spell-REP

字缀文件可用 REP 项目来定义常见错误。用于提供拼写建议。这些项目定义 "从" 一个文本替换 "到" 的另一个文本。例如：

```
REP 4
REP f ph
REP ph f
```

```
REP k ch
REP ch k
```

第一行指定后面跟随的 REP 行数。Vim 忽略该计数，但此行仍然要存在（为了和 Myspell 兼容）。

不要包含简单的单字符替换或交换。但你要指定的话 Vim 还是会去做。如果你喜欢，可以包含整个单词，但用 'spellsuggest' 里的 "file:" 项目可能更好。

下划线可用来包含空格：

```
REP the_the the
```

相近字符 spell-MAP E783

字缀文件可用 MAP 项目定义非常相近的字母。多用于带有不同重音的同一字母。替换这些字母的建议会得到更高的优先级。例如：

```
MAP 2
MAP eéêè
MAP uüùú
```

第一行指定后面跟随的 MAP 行数。Vim 忽略该计数，但该行仍然要存在。

每个字母只能在一个 MAP 项目里出现。如果第一个字母是 ASCII 或至少是不带重音的那个，效率会稍稍提高。

.SUG 文件 spell-NOSUGFILE

如果字缀文件指定按发音折叠，":mkspell" 通常会和 .spl 文件一起产生 .sug 文件。该文件用于快速寻找发音相近形式的建议。代价是大量的内存开销（开销取决于单词数，:mkspell 完成时会显示这方面大概的估计）。

如果不想生成 .sug 文件，在字缀文件里使用项目：

```
NOSUGFILE
```

如果只是不想用，用户可以简单地忽略 .sug 文件。

SOUND-A-LIKE spell-SAL

字缀文件可用 SAL（sound-a-like，发音相近）项目来定义要使用的发音相近机制。主要项目定义 "从" 一个文本替换 "到" 的另一个文本。简单的例子：

```
SAL CIA                 X
SAL CH                  X
SAL C                   K
SAL K                   K
```

这里有一些规则，它们可能会变得很复杂。具体如何工作的解释可见 Aspell 手册：
http://aspell.net/man-html/Phonetic-Code.html

还有一些特殊项目：

```
SAL followup            true
SAL collapse_result     true
SAL remove_accents      true
```

"1" 的含义和 "true" 相同。其它值代表 "false"。

简单按发音折叠 spell-SOFOFROM spell-SOFOTO

SAL 机制复杂又费时。一个稍简单的机制是把所有的字符映射为其它字符，这样发音相近的字符被映射为同一字符。同时也可用于大小写合并。你不能同时使用 SAL 项目和简单按发音折叠项目。

这个机制需要两个项目：一个指定被映射的字符，一个指定映射到的字符。它们必须包含相同数目的字符。例如：

```
SOFOFROM abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
SOFOTO   ebctefghejklnnepkrstevvkesebctefghejklnnepkrstevvkes
```

这个例子里所有元音被映射为同一个字符 'e'。另一个方法是完全省略所有的元音。一些发音几乎相同的字符经常会被混用，比如 'm' 和 'n'，它们也被映射到同一个字符上。不要过于极端，否则所有单词看起来都差不多了。

不出现在 SOFOFROM 里的字符被省略，除了所有的空白被为单个空格替换以外。SOFOFORM 里相同字符的序列被替换为一个字符。

你可以用 soundfold() 函数测试结果。也可以设置 'verbose' 选项来看看 z= 命令输出的评分。

不支持的项目 spell-affix-not-supported

这些项目出现在其它拼写检查器使用的字缀文件里。Vim 里忽略它们，不支持也不用其它方法定义。

ACCENT (Hunspell) spell-ACCENT
用 MAP 代替。 spell-MAP

BREAK (Hunspell) spell-BREAK
定义分切点。还不清楚这是怎么工作的。
不支持。

CHECKCOMPOUNDCASE (Hunspell) spell-CHECKCOMPOUNDCASE
不允许复合词边界上有大写字母。不支持。

CHECKCOMPOUNDDUP (Hunspell) spell-CHECKCOMPOUNDDUP
不允许复合词里使用同一单词两次。不支持。

CHECKCOMPOUNDREP (Hunspell) spell-CHECKCOMPOUNDREP
有关 REP 项目和复合词。不支持。

CHECKCOMPOUNDTRIPLE (Hunspell) spell-CHECKCOMPOUNDTRIPLE
禁止复合时出现三个相同的字符。不支持。

CHECKSHARPS (Hunspell)) spell-CHECKSHARPS
大写（德语）单词中的 SS 字母对可以是大写的 sharp s ()。不支持。

COMPLEXPREFIXES (Hunspell) spell-COMPLEXPREFIXES
允许使用两个前缀。不支持。

COMPOUND (Hunspell) spell-COMPOUND
先是包含 COMPOUND 项目总数的一行，然后跟随那么多数目带有模式的 COMPOUND 行。
删除带有计数的第一行，把其它行换名为 COMPOUNDRULE。
spell-COMPOUNDRULE

COMPOUNDFIRST (Hunspell) spell-COMPOUNDFIRST
用 COMPOUNDRULE 代替。 spell-COMPOUNDRULE

COMPOUNDBEGIN (Hunspell) spell-COMPOUNDBEGIN
COMPOUNDBEGIN 签字的单词可能是复合词的起始元素。
用 COMPOUNDRULE 代替。 spell-COMPOUNDRULE

COMPOUNDLAST (Hunspell) spell-COMPOUNDLAST
COMPOUNDLAST 签字的单词可能是复合词的结尾元素。
用 COMPOUNDRULE 代替。 spell-COMPOUNDRULE

COMPOUNDEND (Hunspell) spell-COMPOUNDEND
大概和 COMPOUNDLAST 相同

COMPOUNDMIDDLE (Hunspell) spell-COMPOUNDMIDDLE
COMPOUNDMIDDLE 签字的单词可能是复合词的中间元素。
用 COMPOUNDRULE 代替。 spell-COMPOUNDRULE

COMPOUNDRULES (Hunspell) spell-COMPOUNDRULES
紧跟其后的 COMPOUNDRULE 行的数目。忽略，但参数必须是数值。

COMPOUNDSYLLABLE (Hunspell) spell-COMPOUNDSYLLABLE
用 SYLLABLE 和 COMPOUNDSYLMAX 代替。 spell-SYLLABLE
spell-COMPOUNDSYLMAX

KEY (Hunspell) spell-KEY
定义键盘上互相靠近的字符。
用于给出更好的建议。不支持。

LANG (Hunspell) spell-LANG
指定语言特定的行为。实际上把部分有关语言的知识放进了程序里。因而 Vim 不支持。必须分别指定每个语言属性。

LEMMA_PRESENT (Hunspell) spell-LEMMA_PRESENT
只用于词态学分析。

MAXNGRAMSUGS (Hunspell) spell-MAXNGRAMSUGS
设置 n-元组建议的数目。不支持。

PSEUDOROOT (Hunspell) spell-PSEUDOROOT
用 NEEDAFFIX 代替。 spell-NEEDAFFIX

SUGSWITHDOTS (Hunspell) spell-SUGSWITHDOTS
给建议加上句号。Vim 不需要。

SYLLABLENUM (Hunspell) spell-SYLLABLENUM
不支持。

TRY (Myspell、Hunspell 以及其它) spell-TRY
Vim 不使用 TRY 项目并忽略它们。要提供建议，使用单词里的实际字符，这样效率高很多。

WORDCHARS (Hunspell) spell-WORDCHARS
用于识别单词。Vim 不需要，因为没有检查单词之前分隔单词的必要(我们用 trie 树代替了哈希表)。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

diff.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar

译者：Zimin<cranecai@users.sf.net>, tocer
http://vimcdoc.sf.net

diff　vimdiff　gvimdiff　diff-mode

这篇文章描述 +diff 特性：比较同一文件的两到八个版本间的差异。

用户手册 08.7 节说明了一些比较基本的用法。

1. 进入比较模式　start-vimdiff
2. 查看比较结果　view-diffs
3. 跳转到差异文　jumpto-diffs
4. 复制差异文本　copy-diffs
5. diff 选项　diff-options

---

1. 进入比较模式　start-vimdiff

进入比较模式编辑的最简单方法就是用 "vimdiff" 命令。它如常启动 Vim，但附加一些设置，以便于查看输入参数所指定的文件间的差异：

```
vimdiff file1 file2 [file3 [file4]]
```

这等同于：

```
vim -d file1 file2 [file3 [file4]]
```

你也可以使用 "gvimdiff" 或 "vim -d -g" 以启动 GUI 的版本。
又或者，使用 "viewdiff" 或 "gviewdiff" 以启动只读模式。
"r" 可以附加在这些名字前面，用来进入受限模式（参考 -Z ）。

第二个及其后的参数也可以是目录名。Vim 将依据第一个参数所指定的文件名在指定目录中查找另外的文件名。

缺省使用内部 diff 库。如果设置了 'diffopt' 或 'difexpr'，则使用外部的 "diff" 命令。只有这样的 diff 程序可用的时候才行。

窗口比较局部于当前标签页 tab-page 中。你不能看到某窗口和别的标签页中的窗口的差异。这样，可以同时打开多组比较窗口，每组差异在单独的标签页中。

Vim 将为每个文件打开一个窗口，并且就像使用 -O 参数一样，使用垂直分割。如果你要水平分割，加上 -o 参数：

```
vimdiff -o file1 file2 [file3 [file4]]
```

如果你喜欢总使用水平分割，在 'diffopt' 里包含 "horizontal"。

在每一个被编辑的文件中，以下选项被设定：

```
'diff'　　打开
```

'scrollbind'	打开
'cursorbind'	打开
'scrollopt'	包含 "hor"
'wrap'	关闭
'foldmethod'	"diff"
'foldcolumn'	来自 'diffopt' 的值，缺省为 2

这些参数仅设置于当前窗口。当编辑其它文件时，以上选项被重设回全局值。
不过，重新编辑文件时，仍然可以用模式行修改这些选项。但 'diff' 置位时，
'foldmethod' 和 'wrap' 不受模式行改变。
:diffoff 提供了反转这样选项的简便办法。

比较基于缓冲区的内容。因而，如果在载入文件后你做过改动，这些改动也将参加比较。
不过，你也许要不时地使用 "diffupdate"。因为并非所有的改动的结果都能自动更新，
尤其是使用外部 diff 命令的时候。

在 .vimrc 文件中，你可以用以下的结构，对比较模式做专门的设定：

```
if &diff
   setup for diff mode
else
   setup for non-diff mode
endif
```

如果已在 Vim 中，你可以用三种方式进入比较模式。

E98
:diffs[plit] {filename} :diffs :diffsplit
对 {filename} 开一个新窗口。当前的和新开的窗口将设定和
"vimdiff" 一样的参数。另见 'diffexpr'。

:difft :diffthis
:difft[his] 使当前窗口参与比较。本命令设定和 "vimdiff" 同样的选项。

:diffp[atch] {patchfile} E816 :diffp :diffpatch
使用当前的缓冲区，用 {patchfile} 给它打上补丁并打开一个缓冲区
显示结果。两个缓冲区都将设定为和 "vimdiff" 同样的参数。
{patchfile} 可以是任何一种 "patch" 程序认识的或 'patchexpr' 可
以处理的格式。
注意：{patchfile} 只能包含一个文件的比较结果：当前文件。如果
{patchfile} 也包含了其他文件的比较，结果将不可预知。Vim 改变当
前目录到 /tmp， 以避免当前目录下的文件被意外的打补丁。但它仍不
能避免若干 ".rej" 文件产生。当绝对路径名出现时，这些文件也不能
避免被打补丁。

要垂直分割窗口，在前面加上 :vertical 。例如：

```
:vert diffsplit main.c~
:vert diffpatch /tmp/diff
```

如果你偏好垂直分割，可以在 'diffopt' 中包含 "vertical"。

E96
最多可以对八个缓冲区设置 'diff' 模式。

因为选项的值是由缓冲区记录的，在编辑另一个文件后又回到此文件时，你仍会处于比较模式。

:diffo :diffoff

:diffo[ff] 为当前窗口关闭比较模式。即使 'diff' 没有置位，也复位相关选项。

:diffo[ff]! 为当前窗口和当前标签页所有置位 'diff' 的窗口关闭比较模式。只在在置位 'diff' 的窗口中才复位相关选项，如果当前窗口没有置位 'diff'，不改变其任何选项。
隐藏缓冲区也从参于比较的缓冲区列表中删除。

:diffoff 命令复位相关选项到启用 :diffsplit，:diffpatch，:diffthis 之前或以比较模式启动 Vim 时的值。
使用 :diffoff 两次恢复再上次的保存值。
否则复位其默认值如下：

'diff'	关闭
'scrollbind'	关闭
'cursorbind'	关闭
'scrollopt'	没有 "hor"
'wrap'	打开
'foldmethod'	"manual"
'foldcolumn'	0

---

2. 查看比较结果 view-diffs

比较产生的结果是，不同比较窗口显示同一文件不同版本，高亮标出差异的部分。当滚动文本时，'scrollbind' 选项将使得另一窗口的相应文本也随着滚动。垂直分割时，文本会恰当地对齐。

不过，以下情况将使文本对齐出现问题：
- 当设置了 'wrap' 后，一些行将被自动换行，在屏幕上占用两行以上。
- 一个缓冲区打开了折叠，另一个却没有。
- 'scrollbind' 被关闭
- 文本做过改动
- 'diffopt' 中没有 "filler"，插入或删除的行会使对齐出错

所有打开 'diff' 选项并在某一个窗口编辑的缓冲区都将参与比较。即使对于一个隐藏的缓冲区也是如此，如果它曾在一个窗口内被编辑过的话。要去除隐藏缓冲区，用 :diffoff! 。

:DiffOrig diff-original-file

因为 'diff' 是一个局部于窗口的选项，所以同一缓冲区可以在某一个窗口里是比较模式，而在另一个窗口里则是普通模式。你也可能想查看对一缓冲区在读入文件后所做过的改动。既然 Vim 不允许同一个文件有 2 个缓冲区，你需要另一个缓冲区。可用下面的命令：

```
command DiffOrig vert new | set bt=nofile | r ++edit # | 0d_
        \ | diffthis | wincmd p | diffthis
```

(defaults.vim 里有该示例)。之后，使用 ":DiffOrig" 就可以查看当前缓冲区和它刚从文件里载入时的不同之处。

要 注意：已经卸载的缓冲区不能参与比较。但一个隐藏的缓冲区是可以的。你可以使用

":hide" 来关闭但不卸载窗口。如果你不想缓冲区继续参与比较，可以在隐藏前先执行 ":set nodiff"。

:dif :diff :diffupdate

:dif[fupdate][!] 刷新比较结果的高亮和折叠。

在改变文本时，Vim 试图更新比较的结果。插入或删除行一般会得到正确的更新。但行内或者更复杂的改动将无法如此。要强制刷新比较结果，使用：

```
:diffupdate
```

包含 ! 时，Vim 会检查文件是否被外部改变而需要重新载入。对每个被改变的文件给出提示，就像用了 :checktime 那样。

一些行在本窗口没有，但却在一个窗口里出现。它们或者是在那个文件里新插入的，或者是在本文件里刚删除的。对这些行，除非 'diffopt' 选项中不包含 "filler"，Vim 将在本窗口对应的位置显示填充行。

折叠可以用来隐藏那些没有被更改过的文字。要知道所有用于折叠的命令，可以参考 folding 。

'diffopt' 选项还可以用来设置在差异文之前不被折叠的行数（上下文）。如要把上下文设为 3 行：

```
:set diffopt=filler,context:3
```

以下的语法高亮群组可以用来显示比较的结果：

hl-DiffAdd	DiffAdd	添加（插入）的行。这些行只存在于本缓冲区里。
hl-DiffChange	DiffChange	改动过的行。
hl-DiffText	DiffText	在改动行中被更改的文本。Vim 找到第一个和最后一个不同的字符（从行末开始搜起）之间的文字，包括其实没被改动的部分，都被高亮。这里只受到 'diffopt' 中的 "iwhite" 和 "icase" 标志位的影响。
hl-DiffDelete	DiffDelete	被删除的行。也称为填充行，因为在本缓冲区里这些行并不真正存在。

---

3. 跳转到差异文 jumpto-diffs

有两条命令可用于在跳转到差异文所在的位置：

[c

[c 反向跳转至上一处更改的开始。计数前缀使之重复执行相应次。

]c

]c 正向跳转至下一个更改的开始。计数前缀使之重复执行相应次。

如果不存在光标可以跳转到的更改，将产生错误。

---

4. 复制差异文本 copy-diffs E99 E100 E101 E102 E103 merge

有两个命令可用来在两个缓冲区之间复制文本。相应的结果是，在一定的范围内两缓冲区的内容被统一。

:diffg :diffget

:[range]diffg[et] [bufspec]
用另一个缓冲区来修改当前的缓冲区，消除不同之处。除非只有另外一个比较模式下的缓冲区，[bufspec] 必须存在并指定那个缓冲区。
如果 [bufspec] 指定的是当前缓冲区，则为空动作。
[range] 可以参考下面。

:diffpu :diffput E793

:[range]diffpu[t] [bufspec]
用当前缓冲区来修改另一个缓冲区，消除不同之处。不同于 ":diffget" 之处仅在于被更改的是另一个缓冲区。
如果忽略 [bufspec]，而多于一个缓冲区处于比较模式并置位 'modifiable' 的话，此命令失败。
[range] 可以参考下面。

do

[count]do 同 ":diffget"，但没有范围。"o" 表示 "obtain"（不能用 "dg"，因为那可能是 "dgg" 的开始！）。注意 不适用于可视模式。
给出的 [count] 用作 ":diffget" 的 [bufspec] 参数。

dp

dp 同 ":diffput"，但没有范围。注意 不适用于可视模式。
给出的 [count] 用作 ":diffput" 的 [bufspec] 参数。

当没有给定 [range] 时，受影响的仅是当前光标所处位置或其紧上方的差异文本。
当指定 [range] 时，Vim 试图仅改动它指定的行。不过，当有被删除的行时，这不总有效。

可能在最后一行之后有删除的行。要从另一个缓冲区中取得那些行，可使用最后一行的行号加一来实现。以下命令从另一个缓冲区中得到所有的差异文本：

```
:1,$+1diffget
```

注意：被删除的行会被显示，但不作为文本行看待。你也不能将光标移至其中。要用另一个缓冲区来填充被删除的行，可在其下一行用 ":diffget"。

E787

如果要修改的缓冲区只读，而 FileChangedRO 激活的自动命令要修改缓冲区，该命令会失败。此时，自动命令不能修改缓冲区。

参数 [bufspec] 可以是缓冲区的序号，匹配缓冲区名称或缓冲区名称的一部分的模式。例如：

:diffget	使用另一个进入比较模式的缓冲区
:diffget 3	使用 3 号缓冲区
:diffget v2	使用名字同 "v2" 匹配的缓冲区，并进入比较模式（例如，"file.c.v2"）

5. diff 选项 diff-options

也可参考 'diffopt' 和 'fillchars' 中的 "diff" 项。

diff-slow diff_translations
比较语法高亮对很长的行可能会较慢，尤其是因为它试图匹配各种本地化的文本。要关闭本地化并提高语法高亮速度，设置全局变量 g:diff_translations 为零：

```
let g:diff_translations = 0
```

设置此变量后，重新载入语法脚本：

```
set syntax=diff
```

查找不同 diff-diffexpr

'diffexpr' 选项可以用来设定非标准的 "diff" 程序，用来比较文件间的异同。
E959

当 'diffexpr' 为空的时候，Vim 使用以下命令在 file1 和 file2 中查找不同之处：

```
diff file1 file2 > outfile
```

其中的 ">" 会用 'shellredir' 的值替换。

"diff" 的输出必须是普通的 "ed" 风格或合并风格 (unified) 的 diff。不要使用上下文 (context) 风格的 diff。下面就是一个 Vim 所期望的 "ed" 风格的范例：

```
1a2
> bbb
4d4
< 111
7c7
< GGG
---
> ggg
```

"1a2" 项添加了 "bbb" 行。
"4d4" 项删除了 "111" 行。
"7c7" 项用 "ggg" 行替代了 "GGG" 行。

当 'diffexpr' 不为空时，Vim 执行它以得到一个满足上述格式的 diff 文件。在执行过程中，以下的变量会被设置为须用到的文件名：

v:fname_in	原始文件
v:fname_new	同一文件的新版
v:fname_out	产生的 diff 文件

另外，'diffexpr' 应负责实现 'diffopt' 选项中的 "icase" 和 "iwhite"。
而且，'diffexpr' 不能更改 'lines' 和 'columns' 的值。

示例 (大致相当于 'diffexpr' 为空时的行为)：

```
set diffexpr=MyDiff()
function MyDiff()
   let opt = ""
   if &diffopt =~ "icase"
     let opt = opt . "-i "
   endif
   if &diffopt =~ "iwhite"
     let opt = opt . "-b "
   endif
   silent execute "!diff -a --binary " . opt . v:fname_in . " " . v:fname_new .
        \  " > " . v:fname_out
   redraw!
endfunction
```

其中，"-a" 参数被用来强制将文件作为文本来比较，二进制的比较没有什么意义。
"--binary" 参数使得文件以二进制模式读入，这样在 DOS 上 CTRL-Z 就不会结束文本。

redraw! 命令不一定必要，取决于执行外壳命令时是否会在屏幕上显示东西。

E810 E97

Vim 将测试 diff 的输出看上去是否完全正确。如果不正确，你将得到一个错误信息。可能是因为：
- "diff" 程序无法执行。
- "diff" 程序无法产生普通 "ed" 风格的 diff 文件（参考上面）。
- 'shell' 和相关选项没有正确设置。试试类似 ":!sort" 过滤运行是否正确。
- 你设置的 'diffexpr' 可能不正确。

如果问题出在哪里不是很清楚，可以设置 'verbose' 选项为一或更高的值，以获得更多的信息。

MS-Windows 自安装的 Vim 包含了 diff 程序。如果没有，您可以自行下载 diff.exe。例如从这里
http://gnuwin32.sourceforge.net/packages/diffutils.htm。

使 用 补 丁

diff-patchexpr

选项 'patchexpr' 可以用来设定非标准的 "patch" 程序。

当 'patchexpr' 为空时，Vim 将这样调用 "patch" 程序：

```
patch -o outfile origfile < patchfile
```

对于大多数的 "patch" 程序版本，这都可以正确工作。 注意 ：在一行中间的 CR 可能产生问题。它被当做一个换行符。

如果默认值无法使工作，设定 'patchexpr' 使之有以上所述的同样的效果。它被执行时，以下的变量会被设定为相关的文件名：

v:fname_in	原始文件
v:fname_diff	补丁文件
v:fname_out	要生成的打过补丁的文件

示例 (对应 'patchexpr' 为空时的行为):

```
set patchexpr=MyPatch()
function MyPatch()
   :call system("patch -o " . v:fname_out . " " . v:fname_in .
   \  " < " . v:fname_diff)
endfunction
```

请确定使用 "patch" 程序时不会有不想要的副作用。比如，要留心那些生成的额外的文件，用完之后应该将其删除。该程序应该仅仅给文件打补丁而没有别的作用。

使用 'patchexpr' 的值之前， Vim 将改变当前目录到 "/tmp" 或别的临时目录。这是为了让当前目录下的文件不被意外的打补丁。Vim 也将删除以 v:fname_in 开始以 ".rej" 和 ".orig" 结尾的文件。

vim:tw=78:ts=8:noet:ft=help:norl:

autocmd.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

自动命令 autocommand autocommands

一个基本的介绍可以在用户手册的 40.3 章节找到。

1. 简介 autocmd-intro
2. 定义自动命令 autocmd-define
3. 删除自动命令 autocmd-remove
4. 列出自动命令 autocmd-list
5. 事件 autocmd-events
6. 模式 autocmd-patterns
7. 局部于缓冲区的自动命令 autocmd-buflocal
8. 组 autocmd-groups
9. 执行自动命令 autocmd-execute
10. 自动命令的使用 autocmd-use
11. 屏蔽自动命令 autocmd-disable

---

1. 简介 autocmd-intro

在文件读写，缓冲区或窗口进出，甚至 Vim 退出等时刻，你都可以指定要自动执行的命令。例如，你可以创建一个自动命令，对匹配 *.c 的文件自动置位 'cindent' 选项。你还可以用自动命令来实现诸如编辑压缩文件（见 gzip-example ）等的高级特性。一般来说，自动命令在 .vimrc 或 .exrc 文件里设置。

E203 E204 E143 E855 E937 E952

警告：自动命令功能强大，甚至会导致意想不到的副作用。小心你的文本不要遭到破坏。
- 最好能先在一个能够牺牲的副本上进行测试。
  例如：如果你使用自动命令在文件开始编辑时进行解压，应确保写回时能正确执行压缩的自动命令。
- 准备好中途出现的错误（例如，磁盘没有空间）。Vim 通常能够撤销缓冲区里的改动，但文件的其他方面改动需要你手动清理（例如，压缩被解压的文件）。
- 如果 BufRead* 等事件允许你编辑一个压缩文件，FileRead* 等事件应该完成同样的操作（这使得在一些特殊情况下内容可以恢复）。如果可能，尽量用相同的自动命令处理 File* 和 Buf* 事件。

---

2. 定义自动命令 autocmd-define

:au :autocmd

:au[tocmd] [group] {event} {pat} [++once] [++nested] {cmd}
把 {cmd} 加到 Vim 在匹配 {pat} 模式的文件执行 {event} 事件时自动执行的命令列表。见 autocmd-patterns 。
注意：引号视为 :autocmd 的参数而非注释的引导。
Vim 总把 {cmd} 加到已有的自动命令之后，这样保证自动命

令的执行顺序与其定义的顺序相同。
关于 [++nested]，参见 autocmd-nested 。为了后向兼容，也可用 "nested" (没有 ++)。
autocmd-once
如果给出 [++once]，命令执行一次，然后删除 ("一次性")。

特殊模式 <buffer> 或 <buffer=N> 定义局部于缓冲区的自动命令。见 autocmd-buflocal 。

注意：要在 ":autocmd" 命令之后紧跟其他命令，'|' 必须在 {cmd} 之前。可以这样：

```
:augroup mine | au! BufRead | augroup END
```

但下列命令会把 "augroup" 视为所定义命令的一部分：

```
:augroup mine | au! BufRead * | augroup END
:augroup mine | au BufRead * set tw=70 | augroup END
```

正确的方法应把组名放在命令里：

```
:au! mine BufRead *
:au mine BufRead * set tw=70
```

或者用 :execute :

```
:augroup mine | exe "au! BufRead *" | augroup END
:augroup mine | exe "au BufRead * set tw=70" | augroup END
```

注意 ":autocmd" 的参数里的特殊字符 (例如，"%"、"<cword>") 在定义时不会被扩展，而是在事件发生并执行 {cmd} 的时候才进行。唯一的例外是 "<sfile>" 在定义时扩展。例如：

```
:au BufNewFile,BufRead *.html so <sfile>:h/html.vim
```

这里 Vim 把 <sfile> 扩展为该行所在的文件名。

即使已经执行过，:autocmd 仍会把同一命令加入自动命令列表。如果你的 .vimrc 被执行两次，自动命令就会出现两次。要避免这个问题，在一个组里定义自动命令，这样方便清除之前的命令：

```
augroup vimrc
  " 清除 vimcrc 组全部的的自动命令
  autocmd!
  au BufNewFile,BufRead *.html so <sfile>:h/html.vim
augroup END
```

如果你不想删除所有的自动命令，可以用变量来确保 Vim 只定义自动命令一次：

```
:if !exists("autocommands_loaded")
:  let autocommands_loaded = 1
:  au ...
:endif
```

如果没有给定 {group} 参数，Vim 使用当前组 (由 ":augroup" 定义)；不然，Vim 使用 {group} 定义的组。注意 该 [group] 必须事先定义。定义新组不能用 ":au group ..." 而应该用 ":augroup"。

测试自动命令时，你也许会发现 'verbose' 选项很有用：

```
:set verbose=9
```

这个设置使得 Vim 在执行自动命令时回显之。

在脚本里定义自动命令时，可以调用局部于脚本的函数或者使用局部于脚本的映射。在事件激活并执行相应命令时，该命令在定义脚本的上下文内执行。如果命令里用到 <SID> ，这一点很重要。

执行命令时，某个命令的消息覆盖前一个的。这和手动执行命令不同。通常，屏幕不会滚动，所以也不会有按-回车的提示。但当一个命令输出两条消息时，这仍会发生。

---

3. 删除自动命令 autocmd-remove

:au[tocmd]! [group] {event} {pat} [++once] [++nested] {cmd}
删除所有和 {event} 事件和 {pat} 模式相关联的自动命令，然后加入命令 {cmd}。
关于 [++once]，参见 autocmd-once 。
关于 [++nested]，参见 autocmd-nested 。

:au[tocmd]! [group] {event} {pat}
删除所有和 {event} 事件和 {pat} 模式相关联的自动命令。

:au[tocmd]! [group] * {pat}
删除所有和 {pat} 模式相关联的自动命令。

:au[tocmd]! [group] {event}
删除所有和 {event} 事件相关联的自动命令。
警告：没有给出组时，不要轻易用本命令对 BufRead 和其它常用事件进行操作，可能会对插件、语法高亮等产生破坏。

:au[tocmd]! [group]
删除所有的自动命令。
注意：引号视为 :autocmd 的参数而非注释的引导。
警告：没有给出组时，不要轻易用本命令，会对插件、语法高亮等产生破坏。

如果没有给出 {group} 参数，Vim 使用当前组（由 ":augroup" 定义）；不然，Vim 使用 {group} 定义的组。

---

4. 列出自动命令 autocmd-list

:au[tocmd] [group] {event} {pat}
显示所有和 {event} 事件和 {pat} 模式相关联的自动命令。

:au[tocmd] [group] * {pat}
显示所有和 {pat} 模式相关联的自动命令。

:au[tocmd] [group] {event}
显示所有和 {event} 事件相关联的自动命令。

:au[tocmd] [group]
显示所有自动命令。

如果给出 {group} 参数，Vim 只列出 [group] 里的自动命令；不然，Vim 列出 所有 组里的自动命令。 注意 此处该参数的行为和定义与删除自动命令时的不同。

要列出局部于缓冲区的自动命令，使用如下形式的模式 <buffer> 或 <buffer=N>。见 autocmd-buflocal 。

:autocmd-verbose

如果 'verbose' 非空，列出自动命令同时显示该自动命令最近修改的位置。例如：

```
:verbose autocmd BufEnter
FileExplorer  BufEnter
    *         call s:LocalBrowse(expand("<amatch>"))
        Last set from /usr/share/vim/vim-7.0/plugin/NetrwPlugin.vim
```

详见 :verbose-cmd 。

---

5. 事件 autocmd-events E215 E216

你可以指定逗号分隔的事件名列表。此列表中不能有空格。该命令应用于列表里的所有事件。

文件读入 时，有四类可能的事件：

BufNewFile		开始编辑一个还不存在的文件
BufReadPre	BufReadPost	开始编辑一个已经存在的文件
FilterReadPre	FilterReadPost	读取过滤程序输出的暂存文件
FileReadPre	FileReadPost	任何别的文件读入

Vim 读入文件时，只用这四类中的一类。"Pre" 和 "Post" 事件被双双激活，分别在文件读取的前后。

注意 *ReadPre 和所有的过滤事件的自动命令不能更换当前缓冲区 (如果你这么做，会得到错误信息)。可用来防止文件被读入错误的缓冲区。

注意 执行完 BufReadPost 和 BufNewFile 自动命令 之后 复位 'modified' 标志位，但如果自动命令已经置位了 'modified' 选项除外。

你可以使用 'eventignore' 选项来忽略若干甚至全部事件。

autocommand-events {event}

Vim 识别以下事件，事件名的大小写被忽略 (例如，你可以使用 "BUFread" 或者 "bufread" 来取代 "BufRead")。

这里首先提供功能的总览并提供简短说明。然后，按字母顺序列出它们的详细解释 autocmd-events-abc 。

名字	激活条件
读入	
BufNewFile	开始编辑尚不存在的文件
BufReadPre	开始编辑新缓冲区，读入文件前
BufRead	开始编辑新缓冲区，读入文件后
BufReadPost	开始编辑新缓冲区，读入文件后
BufReadCmd	开始编辑新缓冲区前 Cmd-event

FileReadPre	用 ":read" 命令读入文件前
FileReadPost	用 ":read" 命令读入文件后
FileReadCmd	用 ":read" 命令读入文件前 Cmd-event
FilterReadPre	用过滤命令读入文件前
FilterReadPost	用过滤命令读入文件后
StdinReadPre	从标准输入读入缓冲区前
StdinReadPost	从标准输入读入缓冲区后

写回

BufWrite	开始把整个缓冲区写回到文件
BufWritePre	开始把整个缓冲区写回到文件
BufWritePost	把整个缓冲区写回到文件后
BufWriteCmd	把整个缓冲区写回到文件前 Cmd-event
FileWritePre	开始把缓冲区部分内容写回到文件
FileWritePost	把缓冲区部分内容写回到文件后
FileWriteCmd	把缓冲区部分内容写回到文件前 Cmd-event
FileAppendPre	开始附加到文件
FileAppendPost	附加到文件后
FileAppendCmd	附加到文件前 Cmd-event
FilterWritePre	开始为过滤命令或 diff 写到文件
FilterWritePost	为过滤命令或 diff 写到文件后

缓冲区

BufAdd	刚把缓冲区附加到缓冲区列表后
BufCreate	刚把缓冲区附加到缓冲区列表后
BufDelete	从缓冲区列表删除缓冲区前
BufWipeout	从缓冲区列表完全删除缓冲区前
BufFilePre	改变当前缓冲区名字前
BufFilePost	改变当前缓冲区名字后
BufEnter	进入缓冲区后
BufLeave	转到其它缓冲区前
BufWinEnter	在窗口显示缓冲区前
BufWinLeave	从窗口删除缓冲区前
BufUnload	卸载缓冲区前
BufHidden	刚把缓冲区变为隐藏后
BufNew	刚建立新缓冲区后
SwapExists	检测到交换文件已经存在

选项

FileType	设置 'filetype' 选项时
Syntax	设置 'syntax' 选项时
EncodingChanged	'encoding' 选项改变后
TermChanged	'term' 的值改变后
OptionSet	设置任何选项后

启动和退出

VimEnter	完成所有的初始化步骤后
GUIEnter	成功启动 GUI 后
GUIFailed	启动 GUI 失败之后
TermResponse	收到 t_RV 的终端应答后
QuitPre	用 :quit 时，决定是否退出之前
ExitPre	用可使 Vim 退出的命令时
VimLeavePre	退出 Vim 前，在写入 viminfo 文件之前
VimLeave	退出 Vim 前，在写入 viminfo 文件之后

终端

TerminalOpen	建立终端缓冲区后
TerminalWinOpen	在新窗口建立终端缓冲区后

杂项

FileChangedShell	Vim 注意到文件在编辑开始后被改变
FileChangedShellPost	对在编辑开始后被改变的文件的处理完成后
FileChangedRO	对只读文件进行第一次修改前
DiffUpdated	刷新比较结果后
DirChanged	工作目录改变后
ShellCmdPost	执行外壳命令后
ShellFilterPost	用外壳命令执行完过滤后
CmdUndefined	调用没有定义的用户命令
FuncUndefined	调用没有定义的用户函数
SpellFileMissing	使用不存在的拼写文件
SourcePre	执行 Vim 脚本之前
SourcePost	执行 Vim 脚本之后
SourceCmd	执行 Vim 脚本之前 Cmd-event
VimResized	Vim 窗口大小改变后
FocusGained	Vim 得到输入焦点
FocusLost	Vim 失去输入焦点
CursorHold	用户有一段时间没有按键
CursorHoldI	在插入模式下，用户有一段时间没有按键
CursorMoved	普通模式下移动了光标
CursorMovedI	插入模式下移动了光标
WinNew	创建新窗口后
TabNew	创建新标签页后
TabClosed	关闭标签页后
WinEnter	进入其它窗口后
WinLeave	离开窗口前
TabEnter	进入其它标签页后
TabLeave	离开标签页前
CmdwinEnter	进入命令行窗口后
CmdwinLeave	离开命令行窗口前
CmdlineChanged	命令行文本发生改变后
CmdlineEnter	光标移到命令行后
CmdlineLeave	光标离开命令行前

InsertEnter	开始插入模式前
InsertChange	在插入或替换模式下输入 <Insert> 时
InsertLeave	离开插入模式时
InsertCharPre	插入模式输入每个字符前
TextChanged	普通模式中对文本进行改变后
TextChangedI	弹出菜单不可见时，插入模式中对文本进行改变后
TextChangedP	弹出菜单可见时，插入模式中对文本进行改变后
TextYankPost	文本抽出或删除后
SafeState	没有任何待定字符，等待用户键入字符
SafeStateAgain	重复出现 的 SafeState
ColorSchemePre	载入色彩方案前
ColorScheme	载入色彩方案后
RemoteReply	得到了 Vim 服务器的应答
QuickFixCmdPre	执行 quickfix 命令前
QuickFixCmdPost	执行 quickfix 命令后
SessionLoadPost	载入会话文件后
MenuPopup	刚要显示弹出菜单前
CompleteChanged	插入模式补全菜单被改变后
CompleteDone	插入模式补全结束之后
User	和 ":doautocmd" 一起使用

自动命令事件按字母顺序排列的列表 autocmd-events-abc

BufCreate BufAdd

BufAdd 或 BufCreate

缓冲区列表加入缓冲区后。可以是刚建立的新缓冲区或者是已有的缓冲区。
也在缓冲区列表的某个缓冲区换名之后发生。
启动时初始缓冲区的建立不触发。
BufCreate 的名称有其历史原因。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被建立的缓冲区 "<afile>" 不同。

BufDelete

BufDelete

缓冲区列表删除缓冲区前。可能先调用 BufUnload 事件 (如果该缓冲区已经载入的话)。
也在缓冲区列表的某个缓冲区换名之前发生。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被删除的缓冲区 "<afile>" 及 "<afile>" 不同。此处不可切换到其它缓冲区，否则会引起问题。

BufEnter

BufEnter

进入缓冲区后。可用来设定有关文件类型的选项。也在开始编辑缓冲区时执行，它发生在 BufReadPost 自动命令之后。

BufFilePost

BufFilePost
":file" 或 ":saveas" 命令改变当前缓冲区的名字后。
BufFilePre

BufFilePre
":file" 或 ":saveas" 命令改变当前缓冲区的名字前。
BufHidden

BufHidden
缓冲区刚被隐藏前。也就是说，没有窗口显示该缓冲区，但是它没有被卸载或者删除。":qa" 或者 ":q" 退出 Vim 时不会激活该事件。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被隐藏的缓冲区 "<afile>" 不同。
BufLeave

BufLeave
转到别的缓冲区前、或离开/关闭当前窗口并且新的当前窗口编辑的不是相同的缓冲区前，
":qa" 或 ":q" 退出 Vim 时不会激活此事件。
BufNew

BufNew
刚建立新缓冲区或给缓冲区换名后。缓冲区被加到缓冲区列表时，同时会激活 BufAdd 事件。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被建立的缓冲区 "<afile>" 不同。
BufNewFile

BufNewFile
开始编辑尚未存在的文件时。可用来读入骨架文件。
BufRead BufReadPost

BufRead 或 BufReadPost
开始编辑新的缓冲区并把文件读入缓冲区后，执行模式行之前。模式行之后的事件，见 BufWinEnter 。
不适用于 ":r file"，也不适用于文件还不存在的情况。但在成功修复文件之后会激活该事件。
也在 ":filetype detect" 执行时，激活 filetypedetect 自动命令组中的本事件。还有，写回未命名缓冲区时如使缓冲区取得名字，也会激活。
BufReadCmd

BufReadCmd
开始编辑新的缓冲区前。应执行把文件读入缓冲区的操作。 Cmd-event
BufReadPre E200 E201

BufReadPre
开始编辑新的缓冲区并把文件读入缓冲区前。如果文件还不存在，不会有此事件。
BufUnload

BufUnload
缓冲区卸载前。此时，缓冲区里的文本将要被释放。可在 BufWritePost 后和 BufDelete 前发生。 Vim 即将退出时，每个载入的缓冲区也会收到该事件。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被卸载的缓冲区 "<afile>" 不同。
此处不可切换到其它缓冲区或窗口，否则会引起问题。
退出时如果 v:dying 至少为 2，不触发此事件。
BufWinEnter

BufWinEnter
窗口显示缓冲区后。可以是新缓冲区载入（处理完模式行之后）或者隐藏缓冲区在窗口开始显示（从而不再隐藏）。
不带参数的 :split 不激活此事件，因为你继续编辑的是同一个缓冲区，":split" 已在某窗口打开的文件也是如此，同样因为它重用已有的缓冲区。但用当前缓冲区名来 ":split" 却会激活本事件，因为此

时该缓冲区被重新载入。
终端窗口不适用，因为以终端-作业模式启动时，普通模式不可用。可用 TerminalOpen 代替。

BufWinLeave

BufWinLeave
窗口删除缓冲区前。除非它在别的窗口仍然可见。
系统退出时也会激活。在 BufUnload 或 BufHidden 之前激活。
注意：当此自动命令被执行时，当前缓冲区 "%" 可能会和被卸载的缓冲区 "<afile>" 不同。
退出时如果 v:dying 至少为 2，不触发此事件。

BufWipeout

BufWipeout
完全删除缓冲区前。可能先调用 BufUnload 和 BufDelete 事件（如果缓冲区已经载入并且在缓冲区列表之中的话）。
也在不在缓冲区列表的某个缓冲区换名之前发生。
注意：执行此自动命令时，当前缓冲区 "%" 可能会和被删除的缓冲区 "<afile>" 不同。
此处不可切换到其它缓冲区，否则会引起问题。

BufWrite 或 BufWritePre

BufWrite 或 BufWritePre
把整个缓冲区写回到文件前。

BufWriteCmd

BufWriteCmd
把整个缓冲区写回到文件前。应执行把文件写回的操作并在成功后复位 'modified' 标志，除非 'cpo' 里包含了 '+' 并且写到另一个文件里 cpo-+ 。它不应改动缓冲区的内容。
此操作复位 'modified' 时，调整撤销信息，把之前的撤销状态标记为 'modified'，这和 :write 的行为一致。
Cmd-event

BufWritePost

BufWritePost
把整个缓冲区写回到文件后（应该撤销 BufWritePre 的相关命令）。

CmdUndefined

CmdUndefined
调用未定义的用户命令时。可以用来实现在实际调用时才提供动态定义的命令。模式匹配的是命令的名字。 <amatch> 和 <afile> 都被设为该命令的名字。
注意：在命令未定义前，自动补全不能工作。一个替代方法是总是定义好用户命令，使之调用自动载入函数。见 autoload 。

CmdlineChanged

CmdlineChanged
命令行文本发生改变后。小心不要改动命令行本身，不然可能会使 Vim 死锁。
<afile> 设为单个字符，指示命令行的类型。
cmdwin-char

CmdlineEnter

CmdlineEnter
光标移到命令行后，此时用户可输入命令或搜索字符串。
<afile> 设为单个字符，指示命令行的类型。
cmdwin-char

CmdlineLeave

CmdlineLeave
离开命令行前。也用于按 CTRL-C 或 <Esc> 之后放弃命令行时。

此处命令出错时，命令行本身依旧执行。
<afile> 设为单个字符，指示命令行的类型。
cmdwin-char

CmdwinEnter

CmdwinEnter
进入命令行窗口后。可用来对此特殊类型的窗口进行设置。
<afile> 设为单个字符，指示命令行的类型。
cmdwin-char

CmdwinLeave

CmdwinLeave
退出命令行窗口前。可用来清除任何 CmdwinEnter 所做的全局设置。
<afile> 设为单个字符，指示命令行的类型。
cmdwin-char

ColorScheme

ColorScheme
载入色彩方案后。 :colorscheme
模式匹配的是色彩方案名。<afile> 可获得设置该选项时的实际文件名。<amatch> 为新色彩方案名。

ColorSchemePre

ColorSchemePre
载入色彩方案前。 :colorscheme
可用于在下个色彩方案载入之前，删除前一个色彩方案的若干设置，

CompleteChanged

CompleteChanged
每次插入模式补全菜单被改动后。不为弹出式菜单的隐藏触发，为此可用 CompleteDone 。绝不会递归触发。

设置以下这些 v:event 键:

completed_item	见 complete-items 。
height	可见项目数目
width	屏幕单元格数
row	屏幕顶行号
col	屏幕最左列号
size	项目总数
scrollbar	如果可见则为 TRUE

不能改变文本 textlock 。

也可通过 pum_getpos() 得到弹出式菜单的大小和位置。

CompleteDone

CompleteDone
插入模式补全结束之后。不管补会是否成功完成或是被放弃都会激活本事件。 ins-completion
v:completed_item 变量包含补全项目的信息。

CursorHold

CursorHold
用户在 'updatetime' 指定的时间里没有按键时。如果用户还没有按键，该事件不会再次激活 (就是说，如果你离开 Vim 去煮杯咖啡，该事件不会每 'updateime' 毫秒就发生一次。:)
CursorHold-example 提供了预览标签的范例。
该事件只有在普通模式才会激活。等待输入命令参数或操作符之后的动作命令时，该事件不会发生。
记录时，不激活 CursorHold 事件。 q

<CursorHold>
在内部实现里，<CursorHold> 键用来激活此自动命令。此字符在 getchar() 表达式映射中可见。

注意：该事件的处理不能使用交互的命令，不会有按 - 回车 (hit-enter) 的提示。
注意：将来设定时间可能有别的选项。
提示：要强制更新状态行，用：
:let &ro = &ro
{仅在 Amiga、Unix、Win32、MSDOS 和所有的 GUI 版本上有效}

CursorHoldI
CursorHoldI
类似于 CursorHold，但用于插入模式。
等待其它键时不激活。譬如在 CTRL-V 之后。也不适用于 CTRL-X 模式 insert_expand 。

CursorMoved
CursorMoved
在普通或可视模式下移动光标后。也用于光标行的文本被改变时，例如，使用 "x"、"rx" 或 "p"。
如果有预输入或在等待操作符中，不激活之。
示例可见 match-parens 。
备注： :noautocmd 不会跳过此事件。
小心：本事件发生非常频繁，不要做任何用户意想不到或需时很久的事情。

CursorMovedI
CursorMovedI
在插入模式下移动光标后。但有弹出菜单时不激活。
其余细节和 CursorMoved 相同。

EncodingChanged
EncodingChanged
改变 'encoding' 选项后激活。可用于设定字体。

FileAppendCmd
FileAppendCmd
附加到文件前。应执行附加到文件的操作。用 '[和 '] 位置标记来定位行范围。 Cmd-event

FileAppendPost
FileAppendPost
附加到文件后。

FileAppendPre
FileAppendPre
附加到文件前。用 '[和 '] 位置标记来定位行范围。

FileChangedRO
FileChangedRO
刚开始修改只读文件前。可以用来从源文件控制系统里更新文件。但如果该修改由自动命令产生，该事件不会发生。
该事件在缓冲区的第一次修改或者 'readonly' 置位后的第一次修改时激活，就在文本刚刚要被修改前发生。
警告：如果在自动命令里移动了光标，此修改的效果无法预测。

E788
这里不能切换到别的缓冲区。你可以重新载入本缓冲区，但不能用来编辑别的文件。

E881
如果行数改变，撤销的保存可能会失败，相关改动也会被放弃。

DiffUpdated

DiffUpdated
刷新比较结果后。取决于使用什么类型的比较（内部或外部），触发可能在每次改动时，也可能在 :diffupdate 后。

DirChanged
因 :cd 、:tcd 或 :lcd 命令，或 'autochdir' 选项而引起的工作目录改变时。
模式可以是:
"window" 由 :lcd 触发
"tabpage" 由 :tcd 触发
"global" 由 :cd 触发
"auto" 由 'autochdir' 触发。
"drop" 由文件编辑触发
<afile> 设为新目录名。

ExitPre
用 :quit 、:wq 等命令如使得 Vim 退出，或用 :qall 时，此事件紧跟在 QuitPre 之后。可用于关闭非紧要的窗口。如果有已修改的缓冲区且未自动保存的话，退出行为仍然可被终止。真正退出可用 VimLeavePre 。

FileChangedShell
Vim 注意 到文件的修改时间不同于编辑开始的时间或者文件属性或文件大小发生改变时。 timestamp
该事件最有可能在执行外壳命令后发生，也可以在执行 :checktime 命令或 gvim 重新获得输入焦点后发生。
该自动命令对每个发生改变的文件进行，但不包括置位了 'autoread' 且（译者注：似乎应为或）没发生改变而的缓冲区。如果指定了 FileChangedShell 自动命令，不会给出相应的警告消息和提示。
v:fcs_reason 变量被设置，以指示发生了什么事，而 v:fcs_choice 则可用来告知 Vim 下一步该做什么。
注意：当此自动命令执行时，当前缓冲区 "%" 可能和 "<afile>" 指定的被改变的缓冲区不同。
注意：执行的命令必须不能修改当前缓冲区，跳转到别的缓冲区，或者删除任何一个缓冲区。
E246 E811
注意：该事件不会嵌套而引起无限循环。这意味着，FileChangedShell 事件所执行的命令不会引起别的 FileChangedShell 事件。

FileChangedShellPost
对外部改变的文件的处理完成后。可用于更新状态行。

FileEncoding
已废弃。仍可用并等价于 EncodingChanged 。

FileReadCmd
":read" 命令读入文件前。应执行把文件读入缓冲区的操作。 Cmd-event

FileReadPost
":read" 命令读入文件后。
注意 Vim 设置 '[' 和 ']' 位置标记为读入的首行和末行。它们可以用来操作刚读入的行范围。

FileReadPre

FileReadPre
":read" 命令读入文件前。

FileType
FileType
设置 'filetype' 选项时。模式匹配的是文件类型。<afile> 可以用来取得设置该选项的文件名，而 <amatch> 则是 'filetype' 的新值。不允许切换到其它窗口或缓冲区。
参见 filetypes 。

FileWriteCmd
FileWriteCmd
写入文件前，但不包括写入整个缓冲区。应执行写入文件的操作。不应改动缓冲区的内容。用 '[和 '] 位置标记来定位行范围。
Cmd-event

FileWritePost
FileWritePost
写入文件后，但不包括写入整个缓冲区。

FileWritePre
FileWritePre
写入文件前，但不包括写入整个缓冲区。用 '[和 '] 位置标记来定位行范围。

FilterReadPost
FilterReadPost
过滤命令读入文件后。Vim 用当前缓冲区的名字来匹配模式，正如 FilterReadPre 那样。
如果 'shelltemp' 关闭，不激活此事件。

FilterReadPre E135
FilterReadPre
过滤命令读入文件前。Vim 用当前缓冲区的名字来匹配模式，而不是过滤命令输出的临时文件名字。
如果 'shelltemp' 关闭，不激活此事件。

FilterWritePost
FilterWritePost
过滤命令写入文件或用外部 diff 产生 diff 文件后(内部 diff 可见 DiffUpdated)。Vim 用当前缓冲区的名字来匹配模式，正如 FilterWritePre 那样。
如果 'shelltemp' 关闭，不激活此事件。

FilterWritePre
FilterWritePre
过滤程序写入文件或用外部 diff 产生 diff 文件前。Vim 用当前缓冲区的名字来匹配模式，而不是过滤命令输出的临时文件名字。
如果 'shelltemp' 关闭，不激活此事件。

FocusGained
FocusGained
Vim 取得输入焦点时。只有 GUI 和少数几个控制台版本能检测该事件。

FocusLost
FocusLost
Vim 失去输入焦点时。只有 GUI 和少数几个控制台版本能检测该事件。也有可能在弹出对话框时发生。

FuncUndefined
FuncUndefined
调用未定义的用户函数时。可以用来实现在实际调用时才提供动态定义的函数。模式匹配的是函数的名字。 <amatch> 和 <afile> 都被设为该函数的名字。
注意: 写 Vim 脚本时，最好采用自动调入函数。见 autoload-functions 。

GUIEnter
GUIEnter
成功启动 GUI 并打开窗口后。用 gvim 的时候，它在 VimEnter 之前发生。在 .gvimrc 里可用它来定位窗口:

```
:autocmd GUIEnter * winpos 100 50
```

GUIFailed

GUIFailed 启动 GUI 失败后。如果可能，Vim 会继续在终端模式下运行（仅当在 Unix 相容系统上连接 X 服务器失败后）。如果此时你想退出 Vim:

```
:autocmd GUIFailed * qall
```

InsertChange

InsertChange 在插入或替换模式下输入 <Insert> 时。
v:insertmode 变量指明新模式。
小心：不要移动光标或做任何用户意想不到的事情。

InsertCharPre

InsertCharPre 插入模式输入每个字符前。
v:char 变量指向正在输入的字符，事件处理时可以改变此变量，从而更改插入的字符。
如果 v:char 被设为多于一个字符，按本义插入。
文本锁激活时不能改变文本 textlock 。
'paste' 置位时不激活本事件。
{仅当编译时加入 +eval 特性才有效}

InsertEnter

InsertEnter 刚开始插入模式之前。也适用于替换模式和虚拟替换模式。 v:insertmode 变量指明该模式。
小心：不要做任何用户意想不到的事情。
光标位置事后会恢复。如果不想如此，设置 v:char 为某个非空的字符串。

InsertLeave

InsertLeave 离开插入模式时。也用于 CTRL-O i_CTRL-O ，但不用于 i_CTRL-C 。

MenuPopup

MenuPopup 刚要显示弹出菜单前（鼠标右键下）。用于根据光标或鼠标指针下的内容调整菜单。
对应的模式匹配代表模式的一或两个字符：

n	普通
v	可视
o	操作符等待
i	插入
c	命令行
tl	终端

OptionSet

OptionSet 设置选项时。模式匹配的是长选项名。
<amatch> 指出选项是否被置位。

v:option_type 说明是全局还是局部值，
v:option_command 说明使用的 set/let 命令的类型（跟随标签可见对应表）。
v:option_new 给出新设选项值，
v:option_oldlocal 给出旧局部选项值，
v:option_oldglobal 给出旧全局选项值，
v:option_old 给出旧选项值，

只有用 :set 、:setlocal 或 modeline 设置选项时才会设置 v:option_oldlocal 。相似地，只有用 :set 或 :setglobal 时才设置 v:option_oldglobal 。

注意 用 :set 设置 global-local 字符串选项时， v:option_old 是旧的全局值。但对其它类型

的选项（局部字符串选项、全局-局部数值选项，等等），它是旧的局部值。

启动时不激活 OptionSet，'key' 选项亦然。其理甚明。

示例：检查 'backupdir' 和 'undodir' 选项使用的目录是否存在，不存在时建立目录。

注意：在此自动命令里建议不要复位选项，可能会对插件破坏。用 :noa 可避免执行此自动命令。

自动命令中使用 :set 时，此事件不再触发。

QuickFixCmdPre

QuickFixCmdPre 开始执行 quickfix 命令前（:make、:lmake、:grep、:lgrep、:grepadd、:lgrepadd、:vimgrep、:lvimgrep、:vimgrepadd、:lvimgrepadd、:cscope、:cfile、:cgetfile、:caddfile、:lfile、:lgetfile、:laddfile、:helpgrep、:lhelpgrep、:cexpr、:cgetexpr、:caddexpr、:cbuffer、:cgetbuffer、:caddbuffer）。
对应的模式匹配执行的命令。如果使用 :grep 但 'grepprg' 设为 "internal"，仍然匹配 "grep"。
该命令不能用于设置 'makeprg' 和 'grepprg' 变量。
如果该命令出错，不执行 quickfix 命令。

QuickFixCmdPost

QuickFixCmdPost 类似于 QuickFixCmdPre，但在执行 quickfix 命令后，跳转到第一个位置之前。:cfile 和 :lfile 等命令则在读入错误文件后，跳转到第一个位置之前执行。
见 QuickFixCmdPost-example 。

QuitPre

QuitPre :quit、:wq 或 :qall 调用时，在决定是否关闭当前窗口或退出 Vim 之前。可用于在当前窗口是最后一个普通窗口时关闭辅助的窗口。
另见 ExitPre 。

RemoteReply

RemoteReply Vim 作为服务器时收到应答时 server2client() 。模式匹配的是 {serverid}。<amatch> 是发出应答的机器的 {serverid}，而 <afile> 是实际的应答字符串。
注意 即使定义了自动命令，还是要用 remote_read() 来取走应答。

SafeState

SafeState 没有任何待定字符，等待用户键入字符时。
以下情形之一不触发：
- 操作符等待中
- 用 "r 输入了寄存器时
- 命令执行到一半时
- 执行映射时
- 有预输入（typeahead）时

- 插入模式补全激活时
- 命令行模式补全激活时
mode() 可用来找到 Vim 所处的状态。可以是:
- 可视模式
- 普通模式
- 插入模式
- 命令行模式
取决于你想做什么，也可用 state() 检查更多信息，如是否因消息导致了屏幕滚动等。

SafeStateAgain

SafeStateAgain 类似于 SafeState，但在处理完任何信息和回调调用后。此事件可能会经常触发，不要做耗时的事情。

SessionLoadPost

SessionLoadPost 载入 :mksession 命令建立的会话文件后。

ShellCmdPost

ShellCmdPost 执行用 :!cmd 、 :shell 、 :make 和 :grep 指定的外壳命令后。可用于检查任何文件的改变。

ShellFilterPost

ShellFilterPost 执行用 ":{range}!cmd"、":w !cmd" 或 ":r !cmd" 指定的外壳命令后。可用于检查任何文件的改变。

SourcePre

SourcePre 执行 Vim 脚本前。 :source <afile> 是待执行的文件名。

SourcePost

SourcePost 执行 Vim 脚本后。 :source <afile> 是待执行的文件名。
脚本执行被中断后不触发。
SourceCmd 自动命令触发后也会触发此事件。

SourceCmd

SourceCmd 执行 Vim 脚本时。 :source <afile> 是待执行的文件名。自动命令应实现执行该文件的操作。
Cmd-event

SpellFileMissing

SpellFileMissing 试图载入拼写检查文件，但找不到该文件时。模式匹配的是语言名。 <amatch> 指定语言，和 'encoding' 也有关。见
spell-SpellFileMissing 。

StdinReadPost

StdinReadPost 从标准输入读取输入到缓冲区后，执行模式行前。仅用于 Vim 启动时使用了 "-" 参数时发生 -- 。

StdinReadPre

StdinReadPre 从标准输入读取输入到缓冲区前。仅用于 Vim 启动时使用了 "-" 参数时发生 -- 。

SwapExists

SwapExists 开始编辑文件时检测到交换文件已存在。只有此时，可以选择处理此情形的方法，也在此时，Vim 可能会询问用户应该如何做。
v:swapname 变量保存找到的交换文件名。<afile> 则是待编辑的文件。 v:swapcommand 可以包含该文件打开后执行的命令。
此事件的命令应该设置 v:swapchoice 变量为包含单字符的字符串，指示 Vim 下一步应该做什么:
'o' 以只读方式打开

'e'　　仍然编辑文件
'r'　　恢复
'd'　　删除交换文件
'q'　　退出，不编辑文件
'a'　　中止，就像按了 CTRL-C 一样
如果设为空串，则询问用户，就像没有 SwapExists 自动命令那样。

E812
此处不允许切换到其它缓冲区、为缓冲区换名或者更改目录。
{仅当编译时加入 +eval 特性才有效}

Syntax
Syntax　　设置 'syntax' 选项时。模式匹配的是语法名。<afile> 可以用来取得设置该选项的文件名，而 <amatch> 则是 'syntax' 的新值。参见 :syn-on 。

TabClosed
TabClosed　　关闭标签页后。

TabEnter
TabEnter　　刚进入标签页后。 tab-page
在激活 WinEnter 事件之后和 BufEnter 事件之前发生。

TabLeave
TabLeave　　刚要离开标签页前。 tab-page
WinLeave 事件在此之前已经激活。

TabNew
TabNew　　创建标签页时。 tab-page
在此前，WinEnter 事件首先激活，TabEnter 其次。

TermChanged
TermChanged　　'term' 的值发生改变后。可用来重新读入语法文件，更新色彩、字体和其他终端相关的设置。对所有已载入的缓冲区执行。

TerminalOpen
TerminalOpen　　用 :terminal 或 term_start() 建立终端缓冲区后。即使以 ++hidden 选项创建无窗口的缓冲区也会触发此事件。

TerminalWinOpen
TerminalWinOpen　　用 :terminal 或 term_start() 建立终端缓冲区后。仅当缓冲区创立时带了窗口才会触发此事件。可用于设置终端窗口的局部于窗口的选项。

TermResponse
TermResponse　　收到终端对 t_RV 的应答后。
可用 v:termresponse 的值判别终端版本。
注意 本事件可能在另一事件执行半途中激发，特别是在文件 I/O，外壳命令等耗时的操作进行时尤有可能。

TextChanged
TextChanged　　普通模式修改当前缓冲区文本之后。即 b:changedtick 改变 (TextChanged 自动命令定义之前发生的也算) 之后。
有预输入或有操作符等待时不激活。
注意: :noautocmd 不会跳过此事件。
小心：本事件发生非常频繁，不要做任何用户意想不

到或需时很久的事情。

TextChangedI

TextChangedI 插入模式修改当前缓冲区文本之后。
弹出菜单可见时不激活。其他事项同 TextChanged。

TextChangedP

TextChangedP 插入模式修改当前缓冲区文本之后。
只适用于弹出菜单可见时。其他事项同 TextChanged。

TextYankPost

TextYankPost 当前缓冲区的文本被抽出或删除后。
v:event 的值可用于决定触发此自动命令的操作，如下所示：

operator	执行的操作。
regcontents	寄存器里以行列表形式保存的文本，相当于： getreg(r, 1, 1)
regname	register 名，无名寄存器返回空串。
regtype	寄存器类型，参见 getregtype() 。

用 quote_ 时不触发，递归调用时也是。
此事件不允许改变缓冲区文本，参见 textlock 。
{仅当编译时加入 +eval 特性才有效}

User

User 不会自动执行。只有用 ":doautocmd" 执行自动命令时才会调用。
注意 使用 ‘:doautocmd User MyEvent‘ 如果没有匹配的自动命令，会报错。如果不想，自己定义一个空自动命令。

UserGettingBored

UserGettingBored 用户按同一个键 42 遍的时候。开玩笑的！

VimEnter

VimEnter 做完所有启动任务后，包括载入 .vimrc 文件，执行 "-c cmd" 参数，创建所有的窗口并在其中载入所有缓冲区。
在此事件激活之前，置位 v:vim_did_enter 变量，所以你可以做：

```
if v:vim_did_enter
  call s:init()
else
  au VimEnter * call s:init()
endif
```

VimLeave

VimLeave 退出 Vim 前，刚写入 .viminfo 文件之后。和 VimLeavePre 一样，只执行一次。
要检测非正常的退出，使用 v:dying 。
退出时如果 v:dying 至少为 2，不触发此事件。

VimLeavePre

VimLeavePre 退出 Vim 时刚写入 .viminfo 文件之前。如果匹配退出时当前缓冲区的名字匹配才会激活。只执行一次。通常指定 "*" 模式。

```
:autocmd VimLeavePre * call CleanupStuff()
```

要检测非正常的退出，使用 v:dying 。
退出时如果 v:dying 至少为 2，不触发此事件。

VimResized

VimResized
在 Vim 窗口的大小改变后，因而 'lines' 和/或 'columns' 也已随之改变。不过，启动时不用。

WinEnter

WinEnter
进入别的窗口后。不包括 Vim 启动时的第一个窗口。
可用来设定窗口的高度。
如果该窗口显示缓冲区，Vim 在 WinEnter 自动命令之后执行 BufEnter 自动命令。
注意：对分割和标签页诸命令来说，WinEnter 事件在这些分割或标签命令之后但在文件载入之前发生。

WinLeave

WinLeave
离开某窗口前。如果将要进入的窗口要显示的是别的缓冲区，Vim 在 WinLeave 自动命令前先执行 BufLeave 自动命令（但不包括 ":new"）。
":qa" 或 ":q" 退出 Vim 时不会激活此事件。

WinNew

WinNew
创建新窗口时。不用于 Vim 启动时的首个窗口。在 WinEnter 事件之前激活。

---

6. 模式 autocmd-patterns {pat}

{pat} 参数可以是逗号分隔的列表，相当于对每个模式分别给出该命令。因而：

```
:autocmd BufRead *.txt,*.info set et
```

等价于：

```
:autocmd BufRead *.txt set et
:autocmd BufRead *.info set et
```

文件模式 {pat} 用以下两种方式之一匹配文件名：
1. 如果模式里没有 '/'，只匹配文件名的尾部（不包括它之前的目录路径）。
2. 如果模式里有 '/'，既匹配短本件名（你输入的），也匹配完整文件名（扩展为完整路径并进行完符号链接的解析以后）。

特殊模式 <buffer> 或 <buffer=N> 用于局部于缓冲区的自动命令 autocmd-buflocal 。该模式不是用来匹配缓冲区的名字的。

例如：

```
:autocmd BufRead *.txt          set et
```

为所有的文本文件置位 'et' 选项。

```
:autocmd BufRead /vim/src/*.c   set cindent
```

为所有 /vim/src 目录下的 C 文件置位 'cindent' 选项。

```
:autocmd BufRead /tmp/*.c       set ts=5
```

如果你有一个从 "/tmp/test.c" 到 "/home/nobody/src/test.c" 的链接并且开始编辑 "/tmp/test.c"，该自动命令会匹配。

注意：要匹配部分路径而不从根目录开始指定，第一个字符用 '*'。例如：

```
:autocmd BufRead */doc/*.txt    set tw=78
```

该自动命令会在例如 "/tmp/doc/xx.txt" 和 "/usr/home/piet/doc/yy.txt" 上执行。目录的层次此处无关紧要。

模式匹配的是通配符扩展后的文件名。这样：

```
:e $ROOTDIR/main.$EXT
```

的参数会在匹配自动命令模式前先被扩展成：

```
/usr/root/main.py
```

小心 FileReadCmd 这样的事件使用的 <amatch> 的值未必如你所料。

模式里可以指定环境变量：

```
:autocmd BufRead $VIMRUNTIME/doc/*.txt  set expandtab
```

而 ~ 也可以指定主目录（如果定义了 $HOME）：

```
:autocmd BufWritePost ~/.vimrc   so ~/.vimrc
:autocmd BufRead ~archive/*      set readonly
```

在自动命令的定义时扩展环境变量，而不是在执行时进行。这和命令的处理不同！

file-pattern

这里，模式的解释和文件名里的模式大致相同：

*	匹配任何字符序列
?	匹配任何单个字符；特别的是，也包括路径分隔符
\?	匹配 '?'
.	匹配 '.'
~	匹配 '~'
,	分隔模式
\,	匹配 ','
{ }	类似于 pattern 里的 \(\)
,	在 { } 里：类似于 pattern 里的 |
\}	本义的 }
\{	本义的 {
\\\{n,m\}	类似于 pattern 里的 \{n,m}
\	类似于 pattern 里的特殊含义
[ch]	匹配 'c' 或 'h'
[^ch]	匹配除了 'c' 和 'h' 的任何字符

注意 在任何系统上，'/' 字符都被用作路径分隔符（即使 MS-Windows 也是如此）。如此做是因为反斜杠在模式里很难使用，而且也为了自动命令能在不同系统间可以相互移植。

可以使用普通模式的项目，但请见上述的特定转译，可能和你的预期有所落差。

autocmd-changes

模式的匹配是在事件激活时进行的。即使某个自动命令改变了缓冲区名字甚至删除了缓冲区，也不会改变执行的是哪个自动命令。例如：

```
au BufEnter *.foo  bdel
au BufEnter *.foo  set modified
```

会删除当前缓冲区，并置位取代当前缓冲区的新缓冲区的 'modified' 标志。Vim 不管 "*.foo" 此时已经不匹配该缓冲区名字。"*.foo" 在该事件被激活时匹配当时的缓冲区名字。

不过，局部于缓冲区的自动命令在用 :bwipe 彻底删除的缓冲区上不会执行。如果 用 :bdel 删除缓冲区，该缓冲区其实还是存在的（它只是不被列出），因而这些自动命令还会执行。

7. 局部于缓冲区的自动命令 autocmd-buflocal autocmd-buffer-local <buffer=N> <buffer=abuf> E680

局部于缓冲区的自动命令和特定缓冲区相联系。它们可用于没有名字或者名字不匹配特定模式的缓冲区。但这也意味着必须为每个缓冲区显式地加入这些自动命令。

局部于缓冲区的自动命令不用模式，而用如下的形式：

<buffer>	当前缓冲区
<buffer=99>	缓冲区号 99
<buffer=abuf>	用 <abuf> (只当执行自动命令时适用) <abuf>

示例：

```
:au CursorHold <buffer>  echo 'hold'
:au CursorHold <buffer=33>  echo 'hold'
:au BufNewFile * au CursorHold <buffer=abuf>  echo 'hold'
```

所有自动命令的命令都可用于局部于缓冲区的自动命令，只要简单地用这些特殊字符串来替代模式就行了。例如：

```
:au! * <buffer>                  " 删除当前缓冲区的局部于缓冲区的自动命令
:au! * <buffer=33>               " 删除缓冲区 #33 的局部于缓冲区的自动命令
:bufdo :au! CursorHold <buffer>  " 删除所有缓冲区里给定事件的自动命令
:au * <buffer>                   " 列出当前缓冲区的局部于缓冲区的自动命令
```

注意 如果为当前缓冲区定义自动命令，保存时记住的是它的缓冲区号。这里用的形式是 "<buffer=12>"，其中 12 是当前缓冲区的编号。例如，列出自动命令时你看到的就是这种形式。

要测试局部于缓冲区的自动命令是否存在，用 exists() 函数：

```
:if exists("#CursorHold#<buffer=12>") | ... | endif
:if exists("#CursorHold#<buffer>") | ... | endif    " 指定当前缓冲区
```

如果缓冲区被彻底删除，其局部于缓冲区的自动命令当然也没有了。注意 缓冲区如果被删除，比如用 ":bdel"，它只是不被列出而已，其自动命令还是存在的。要观察局部于缓冲区的自动命令的删除情况：

```
:set verbose=6
```

不能为还不存在的缓冲区定义局部于缓冲区的自动命令。

---

8. 组 autocmd-groups

自动命令可以被一起放在一个组里。这可用于删除或者执行一组自动命令。例如，所有有关语法高亮的自动命令被放在 "highlight" 组里，这样在 GUI 启动时可以一并执行 ":doautoall highlight BufRead"。

如果没有指定特殊的组名，Vim 使用缺省组。缺省组没有名字。你不能单独执行缺省组的所有自动命令；只有在执行所有组里的自动命令时才会执行它们。

正常情况下，在自动执行自动命令时，Vim 使用所有组的自动命令。组只有在用 ":doautocmd" 或 ":doautoall" 执行自动命令或者在定义或删除自动命令时才用的上。

组名可以包含任何非空白字符。但组名 "end" 保留 (包括大写形式)。

组名是区分大小写的。注意 这和事件名不同！

:aug :augroup

:aug[roup] {name} 定义其后的 ":autocmd" 命令使用的自动命令组名。名字 "end" 或者 "END" 选择缺省组。
为了避免混淆，此处不要用和已有 {event} 名重名的名字，很可能和你想做的不同。

:augroup-delete E367 W19 E936

:aug[roup]! {name} 删除自动命令组 {name}。如果还有自动命令使用该组，不要这么做！不然，系统会警告你。而且如果该组是当前组，会报错 E936。

要为某个组输入自动命令，使用如下方法：
1. 用 ":augroup {name}" 选择组。
2. 用 ":au!" 删除所有旧的自动命令。
3. 定义自动命令。
4. 用 "augroup END" 回到缺省组。

例如：

```
:augroup uncompress
:  au!
:  au BufEnter *.gz     %!gunzip
:augroup END
```

这样可以防止自动命令被多次定义（例如，再次执行 .vimrc 文件）。

---

9. 执行自动命令 autocmd-execute

Vim 也可以非自动地执行自动命令。如果你修改了自动命令或者 Vim 执行了不正确的自动命令（例如文件模式的匹配不正确），这也许会有用。

注意 'eventignore' 选项也适用于此。不会为该选项列出的事件执行任何命令。

:do :doau :doaut :doautocmd E217

:do[autocmd] [<nomodeline>] [group] {event} [fname]
应用匹配 [fname]（缺省是当前文件名）和针对当前缓冲区的 {event} 事件的自动命令。如果当前文件不匹配正确的模式，修改完设置，或者想手动执行某一特定的事件的自动命令的时候，都可以使用该命令。
自动命令中也可以使用，这样你可以用基于一个扩展名的自动命令来应用于另一个扩展名。例如：

```
:au BufEnter *.cpp so ~/.vimrc_cpp
:au BufEnter *.cpp doau BufEnter x.c
```

要小心避免死循环，参见 autocmd-nested 。

如果没有给出 [group] 参数，Vim 执行所有组里的自动命令。如果给出 [group] 参数，Vim 只执行该组里匹配的自动命令。注意：如果你使用未定义的组名，Vim 会报错。

<nomodeline>
应用完自动命令后，会执行模式行。其中的设置可以否决自动命令里的设置，一如编辑文件时那样。给出 <nomodeline> 参数时，跳过这一步骤。对那些不用于缓冲区载入时的事件，如

User ，这一参数较为有用。
如果没有匹配的自动命令可执行，也不会处理模式行。

:doautoa :doautoall

:doautoa[ll] [<nomodeline>] [group] {event} [fname]
类似于 ":doautocmd"，但对每个已载入的缓冲区应用自动命令。注意 [fname] 用于选择自动命令，而不是其应用的缓冲区。
要小心：不要用这个命令执行删除缓冲区、切换到别的缓冲区或者修改缓冲区内容的自动命令；否则结果不可预测。该命令是设计用来执行类似于设置选项、修改高亮等任务的自动命令的。

---

10. 自动命令的使用 autocmd-use

对于文件的 写入 ，有四组可能的事件。Vim 对一个写入命令只会执行其中的一组：

BufWriteCmd	BufWritePre	BufWritePost	写回整个缓冲区
	FilterWritePre	FilterWritePost	写入过滤程序的临时文件
FileAppendCmd	FileAppendPre	FileAppendPost	附加到文件
FileWriteCmd	FileWritePre	FileWritePost	其他的文件写入

如果定义了 "*Cmd" 自动命令，它应该完成相应写入的操作。因而，其他的写入操作不会进行，其他的事件也不会被激活。 Cmd-event

注意 *WritePost 命令应该撤销 *WritePre 命令对缓冲区所做的任何改动；否则，文件的写入会有不应该有的修改缓冲区的副作用。

开始执行自动命令前，写入的行所在的缓冲区暂时成为当前缓冲区。除非自动命令修改了当前缓冲区或者删除了先前的那个当前缓冲区，先前的那个又会重新成为当前的。

*WritePre 和 *AppendPre 自动命令不能删除写入的行所在的那个缓冲区。

'[和 '] 位置标记有特殊的位置：
- 在 *ReadPre 事件之前， '[标记设为新行将要插入的位置上方的那行。
- 在 *ReadPost 事件之前，'[标记设为新读入的内容的第一行，'] 则为其最后一行。
- 开始执行 *WriteCmd、*WritePre 和 *AppendPre 自动命令前，'[标记设为要写入的内容的第一行，'] 则为其最后一行。

小心：'[和 '] 指定的位置在使用修改缓冲区的命令时会改变。

在期待文件名的命令里，你可以使用 "<afile>" 指定被读入的文件名 :<afile> （你可以用 "%" 指定当前文件名）。"<abuf>" 指定当前有效的缓冲区的缓冲区号。它可以用于没有名字的缓冲区，但不适用于没有缓冲区的文件（例如，用 ":r file"）。

gzip-example

读写压缩文件的示例：

```
:augroup gzip
:  autocmd!
:  autocmd BufReadPre,FileReadPre     *.gz set bin
:  autocmd BufReadPost,FileReadPost   *.gz '[,']!gunzip
:  autocmd BufReadPost,FileReadPost   *.gz set nobin
:  autocmd BufReadPost,FileReadPost   *.gz execute ":doautocmd BufReadPost " . expand("%:r")
```

```
  :  autocmd BufWritePost,FileWritePost *.gz !mv <afile> <afile>:r
  :  autocmd BufWritePost,FileWritePost *.gz !gzip <afile>:r

  :  autocmd FileAppendPre              *.gz !gunzip <afile>
  :  autocmd FileAppendPre              *.gz !mv <afile>:r <afile>
  :  autocmd FileAppendPost             *.gz !mv <afile> <afile>:r
  :  autocmd FileAppendPost             *.gz !gzip <afile>:r
  :augroup END
```

我们用 "gzip" 组执行 ":autocmd!"，从而能在脚本文件被执行两次时，删除已经定义的自动命令。

("<afile>:r" 是去掉扩展名的文件名，参见 :_%:)

BufNewFile、BufRead/BufReadPost、BufWritePost、FileAppendPost 和 VimLeave 事件执行的自动命令不置位或复位缓冲区的修改标志。当你用 BufReadPost 自动命令解压缓冲区时，你还可以用 ":q" 直接退出。当你在 BufWritePost 里用 ":undo" 撤销 BufWritePre 命令所做的改变时，你也可以用 ":q" (所以，也可以用 "ZZ")。如果你想使缓冲区设为修改过的，置位 'modified' 选项。

要在自动命令里执行普通模式的命令，用 ":normal" 命令。要小心：如果普通模式命令没能结束，用户需要键入字符 (例如，":normal m" 之后需要输入一个位置标记名)。

如果你在缓冲区修改后想使之成为未修改状态，复位 'modified' 选项。这使得用 ":q" 退出缓冲区 (而不用 ":q!") 成为可能。

autocmd-nested E218

自动命令缺省不会嵌套。例如，如果你在自动命令里用 ":e" 或者 ":w"，Vim 不会执行这些命令相应的 BufRead 和 BufWrite 自动命令。如果你需要这么做，在需要嵌套的命令的定义里加上 "nested" 标志位。例如：

```
  :autocmd FileChangedShell *.c ++nested e!
```

为了防止递归循环，嵌套限定为 10 层。

自动命令里可以用 ":au" 命令。甚至可以用来实现自我修改的命令。这适用于只执行一次的自动命令。

要想为单个命令跳过自动命令，使用 :noautocmd 命令修饰符，或者 'eventignore' 选项。

注意：读入文件时 (用 ":read file" 或者过滤命令)，如果文件的最后一行没有换行符 <EOL>，Vim 记住这一点。下一次写 (用 ":write file" 或者过滤命令) 的时候，如果最后一行不变 而且 置位了 'binary'，Vim 不会自己提供 <EOL>。这使得在刚读入的行上的过滤命令写入相同的文件时写的内容和读入的完全一致，也使得在刚过滤过的行上的写入命令写入相同文件时写的和从过滤程序读取的完全相同。例如，另一个写压缩文件的方法是：

```
  :autocmd FileWritePre *.gz   set bin|'[,']!gzip
  :autocmd FileWritePost *.gz  undo|set nobin
```

autocommand-pattern

你可以指定逗号分隔的多个模式。以下是一些示例。

```
  :autocmd BufRead   *          set tw=79 nocin ic infercase fo=2croq
  :autocmd BufRead   .letter    set tw=72 fo=2tcrq
```

```
:autocmd BufEnter  .letter    set dict=/usr/lib/dict/words
:autocmd BufLeave  .letter    set dict=
:autocmd BufRead,BufNewFile   *.c,*.h set tw=0 cin noic
:autocmd BufEnter  *.c,*.h    abbr FOR for (i = 0; i < 3; ++i)<CR>{<CR>}<Esc>O
:autocmd BufLeave  *.c,*.h    unabbr FOR
```

要指定 makefiles (makefile、Makefile、imakefile、makefile.unix 等等):

```
:autocmd BufEnter  ?akefile*  set include=^s\=include
:autocmd BufLeave  ?akefile*  set include&
```

要使得 C 程序的编辑从第一个函数开始:

```
:autocmd BufRead   *.c,*.h    1;/^{
```

上面如果没有 "1;", 搜索会从文件进入的位置开始, 而不是文件的开始处。

skeleton template

要开始编辑新文件时读入一个骨架 (样板) 文件:

```
:autocmd BufNewFile  *.c      0r ~/vim/skeleton.c
:autocmd BufNewFile  *.h      0r ~/vim/skeleton.h
:autocmd BufNewFile  *.java   0r ~/vim/skeleton.java
```

要在写入一个 *.html 文件时插入当前日期和时间:

```
:autocmd BufWritePre,FileWritePre *.html   ks|call LastMod()|'s
:fun LastMod()
:  if line("$") > 20
:    let l = 20
:  else
:    let l = line("$")
:  endif
:  exe "1," . l . "g/Last modified: /s/Last modified: .*/Last modified: " .
:  \ strftime("%Y %b %d")
:endfun
```

要这段代码工作, 你需要在文件开始的 20 行里有这行 "Last modified: <date time>"。 Vim 把 <date time> (包括该行其后的任何内容) 替换为当前的日期和时间。
解释:

```
ks              保存当前位置到 's' 标记
call LastMod()  调用 LastMod() 函数完成工作
's              光标回到旧的位置
```

LastMode() 函数先检查文件是否少于 20 行, 然后用 ":g" 命令查找包含 "Last Modified:" 的行。在这些行上执行 ":s" 命令实现从已有的时间到当前时间的替换。 ":execute" 命令使 ":g" 和 ":s" 命令可以使用表达式。日期用 strftime() 函数取得。它可以用别的参数取得不同格式的日期字符串。

在命令行上输入 :autocmd 的时候, 事件和命令名字在可能的情况可以用 <Tab>, CTRL-D 等进行自动补全。

Vim 根据你定义的顺序执行所有匹配的自动命令。建议第一个自动命令使用 "*" 作为文件模式, 从而使之适用于所有文件。这意味着你可以在这里设定任何选项的缺省值, 如果有别的匹配的自动命令, 可以把这些缺省值覆盖。但如果没有, 至少你的缺省设置得到保

证（如果从另一个能够匹配自动命令的文件进入这个文件）。注意 "*" 也会匹配以 "." 开始的文件，这一点和 Unix 外壳不同。

autocmd-searchpat
自动命令不会改变当前的搜索模式。Vim 在执行自动命令前保存当前的搜索模式，在完成后恢复之。这意味着自动命令不会影响 'hlsearch' 选项指定的高亮字符串。自动命令里你可以正常的使用模式搜索。例如，用 "n" 命令。如果你想要自动命令设置在命令完成后仍然可用的搜索模式，用 ":let @/ =" 命令。自动命令里不能用 ":nohlsearch" 关闭高亮部分。不过，在启动 Vim 的时候，可以用 'viminfo' 选项里的 'h' 标志位关闭搜索高亮功能。

Cmd-event
在使用 "*Cmd" 事件之一时，匹配的自动命令应该负责执行文件读取、写入或脚本执行操作。这可以用以操作特殊的文件，例如在远程文件系统上。
小心：如果你不正确使用这些事件，造成的效果是你无法读写匹配的文件！确保你小心的测试过这些自动命令。最好使用的是不会匹配正常文件的模式，例如 "ftp://*"。

定义 BufReadCmd 以后，Vim 很难从崩溃的编辑会话恢复。从原始文件恢复的时候，Vim 只会读取交换文件里不存在的部分。因为这用 BufReadCmd 不可能做到，用 :preserve 可以保证恢复的时候不需要原始的文件。应该只有在文件被修改的时候你才想这么做。

对于文件读写命令，v:cmdarg 变量保存当前有效的 "++enc=" 和 "++ff=" 参数。在读写文件的命令里应该用到这些参数。用 "!" 后缀时，v:cmdbang 参数为 1，不然其为 0。

示例参见 $VIMRUNTIME/plugin/netrwPlugin.vim。

---

11. 屏蔽自动命令

autocmd-disable

要在一段时间里屏蔽自动命令，使用 'eventignore' 选项。注意 这可能会导致意料不到的效果。确信在此之后恢复 'eventignore'，可用带 :finally 的 :try 块。

:noautocmd :noa
要为单个命令屏蔽自动命令，使用 ":noautocmd" 命令修饰符。它会在下一个命令的执行期间把 'eventignore' 设为 "all"。例如：

```
:noautocmd w fname.gz
```

这样，可以写入文件而不激活 gzip 插件定义的自动命令。

注意 有些自动事件不会立即激活，而会在之后发生。特别适用于 CursorMoved 和 TextChanged 。

vim:tw=78:ts=8:noet:ft=help:norl:

eval.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

表达式求值　　expression　expr　E15　eval

用户手册第 41 章　usr_41.txt　有使用表达式的介绍。

注意：表达式求值可以在编译时关闭。如果你这么做，本文档介绍的特性就不复存在。见 +eval　和　no-eval-feature　。

1.　变量　　variables
　1.1 变量类型
　1.2 函数引用　　Funcref
　1.3 列表　　Lists
　1.4 字典　　Dictionaries
　1.5 blob　　Blobs
　1.6 变量的更多细节　　more-variables
2.　表达式语法　　expression-syntax
3.　内部变量　　internal-variables
4.　内建函数　　functions
5.　定义函数　　user-functions
6.　花括号名字　　curly-braces-names
7.　命令　　expression-commands
8.　例外处理　　exception-handling
9.　示例　　eval-examples
10. Vim 脚本版本　　vimscript-version
11. 不包含 +eval 特性　　no-eval-feature
12. 沙盘 (sandbox)　　eval-sandbox
13. 文本锁　　textlock

测试支持的文档可见　testing.txt　。
剖视的文档可见　profiling　。

---

1. 变量　　variables

1.1 变量类型

E712　E896　E897　E899

有十种类型的变量：

Number　Integer

数值　　32 位或 64 位带符号整数。　expr-number
64-位整数只有在编译时带　+num64　特性时才有效。
示例：　-123　0x10　0177　0b1011

浮点数　　带小数的数值。　floating-point-format　Float
{仅当编译时加入　+float　特性才有效}
示例：123.456　1.15e-6　-1.1e3

字符串　　NUL 结尾的 8 位无符号字符（即字节）的串。 E928 expr-string
示例: "ab\txx\"--" 'x-z''a,c'

列表　　项目的有序的序列，详见 List 。
示例: [1, 2, ['a', 'b']]

字典　　关联的无序数组: 每个项目包含一个键和一个值。 Dictionary
示例:
{'blue': "#0000ff", 'red': "#ff0000"}
#{blue: "#0000ff", red: "#ff0000"}

函数引用　　指向一个函数的引用 Funcref 。
示例: function("strlen")
可以绑定到字典或参数上，这样就类似于一个偏函数。
示例: function("Callback", [arg], myDict)

特殊　　v:false 、 v:true 、 v:none 和 v:null 。 Special

作业　　用于作业，见 job_start() 。 Job Jobs

通道　　用于通道，见 ch_open() 。 Channel Channels

blob　　二进制大对象 (Binary Large Object)。存储任意字符序列。详见 Blob
示例: 0zFF00ED015DAF
0z 是空 blob。

数值和字符串类型之间会根据使用的情况自动转换。

数值到字符串的转换使用数值的 ASCII 表示。例如:

```
数值 123        -->     字符串 "123"
数值 0          -->     字符串 "0"
数值 -1         -->     字符串 "-1"
```

octal
字符串到数值的转换则把字符串开头的一系列数字位转换成数值。可以识别十六进制 "0xf9"、八进制 "017" 和二进制 "0b10" 形式的数值（注意: scriptversion-4 不识别八进制）。如果字符串不以数字开始，则结果为零。
例如:

```
字符串 "456"    -->     数值 456
字符串 "6bar"   -->     数值 6
字符串 "foo"    -->     数值 0
字符串 "0xf1"   -->     数值 241
字符串 "0100"   -->     数值 64
字符串 "0b101"  -->     数值 5
字符串 "-8"     -->     数值 -8
字符串 "+8"     -->     数值 0
```

要强制从字符串转换到数值，给它加零:

```
:echo "0100" + 0
64
```

要避免开头的零导致八进制的转换，或者想换不同的基底，用 str2nr() 。

TRUE FALSE Boolean
布尔型的操作使用数值类型。零代表假值 (FALSE)，非零代表真值 (TRUE)。也可用 v:false 和 v:true 。函数返回 TRUE 时相当于数值一，FALSE 相当于数值零。

注意 在命令：

```
:if "foo"
:" _不_ 执行
```

里，"foo" 被转换成 0，也就是假值。如果字符串以非零数字开始，则代表真值：

```
:if "8foo"
:" 执行
```

要测试字符串非空，应该使用 empty()：

```
:if !empty("foo")
```

non-zero-arg
函数参数和 TRUE 行为通常略有差异：如果参数存在且其值为非零数值、 v:true 或非空字符串，则视之为真值。
注意 " " 和 "0" 也是非空字符串，亦应视为真值。列表、字典和浮点数不是数值或字符串，因而视为假值。

E745 E728 E703 E729 E730 E731 E908 E910 E913
E974 E975 E976
List 、 Dictionary 、 Funcref 、 Job 、 Channel 和 Blob 类型不会自动进行转换。

E805 E806 E808
混合数值和浮点数的计算时，数值转换为浮点数。否则没有自动到浮点数的转换。用 str2float() 可转换字符串到浮点数，printf() 从浮点数到字符串，float2nr() 则从浮点数到数值。

E891 E892 E893 E894 E907 E911 E914
期待浮点数的地方也可用数值代替，但其它都不行。

no-type-checking
试图改变变量类型不会报错。

1.2 函数引用

Funcref E695 E718
函数引用变量可以通过 function() 函数、 funcref() 函数或者 expr-lambda 匿名表达式得到。可以在表达式里用它来代替函数名，在围绕参数的括号之前，以调用它引用的函数。例如：

```
:let Fn = function("MyFunc")
:echo Fn()
```

E704 E705 E707
函数引用变量必须以大写字母、"s:"、"w:"、"t:" 或 "b:" 开始。可以用 "g:"，但后面的名字必须以大写开始。函数引用变量不能和任何函数重名。

特例是可以定义函数并直接把它的函数引用赋给字典的一个项目。例如：

```
:function dict.init() dict
:   let self.val = 0
:endfunction
```

该字典的键可以用小写字母开始。这里不用实际的函数名。另见 numbered-function 。

函数引用可以用 :call 命令调用:

```
:call Fn()
:call dict.init()
```

所引用的函数的名字可以用 string() 得到,

```
:let func = string(Fn)
```

你可以用 call() 来调用函数引用并用一个列表变量来传递参数:

```
:let r = call(Fn, mylist)
```

Partial

函数引用可以选择绑定字典和/或参数,也叫偏函数。通过向 function() 或 funcref() 提供字典和/或参数完成。调用函数时,该字典和/或参数被传入函数。例如:

```
let Cb = function('Callback', ['foo'], myDict)
call Cb('bar')
```

这类似于如下方式地调用函数:

```
call myDict.Callback('foo', 'bar')
```

对 ch_open() 这样传递函数作为参数的情况,这很有用。

注意 把函数作为字典的一个成员时,也会绑定函数到字典:

```
let myDict.myFunction = MyFunction
call myDict.myFunction()
```

这里,调用 "myFunction" 成员时,MyFunction() 会通过 "self" 得到 myDict。如果把 "myFunction" 赋给 otherDict 并调用之,则相应绑定到 otherDict 上:

```
let otherDict.myFunction = myDict.myFunction
call otherDict.myFunction()
```

现在 "self" 变成了 "otherDict"。但字典如果是显式绑定,这不会发生:

```
let myDict.myFunction = function(MyFunction, myDict)
let otherDict.myFunction = myDict.myFunction
call otherDict.myFunction()
```

这里 "self" 还是 "myDict",因为是通过显式绑定的。

1.3 列表

list List Lists E686

列表是项目的有序序列。项目可以是任何类型,用索引号可以进行访问。可以在序列的任何位置上增加或者删除项目。

列表建立

E696 E697

列表用方括号里逗号分隔的项目序列建立。例如:

```
:let mylist = [1, two, 3, "four"]
```

```
:let emptylist = []
```

项目可以是任何表达式。用列表作为项目就能建立列表的列表：

```
:let nestlist = [[11, 12], [21, 22], [31, 32]]
```

忽略末项之后额外的逗号。

列表索引

list-index E684

在列表之后的方括号中放上索引号可以访问列表项目。索引从零开始，也就是说，第一个项目的索引值为零。

```
:let item = mylist[0]           " 得到第一个项目：1
:let item = mylist[2]           " 得到第三个项目：3
```

如果返回的项目本身是列表，可以重复这样的操作：

```
:let item = nestlist[0][1]      " 得到第一个列表的第二个项目：12
```

负索引从尾端开始计算。索引 -1 指向列表的最后一个项目，-2 指向倒数第二个项目，依此类推。

```
:let last = mylist[-1]          " 得到最后一个项目："four"
```

要避免非法索引值产生的错误，用 get() 函数。如果项目不存在，它返回零或者你指定的缺省值：

```
:echo get(mylist, idx)
:echo get(mylist, idx, "NONE")
```

列表连接

两个列表可以用 "+" 操作符连接：

```
:let longlist = mylist + [5, 6]
:let mylist += [7, 8]
```

要在前面或后面附加项目，在项目外面加上 [] 从而把它变为一个列表。要改变列表内部的值，见下 list-modification 。

子列表

sublist

列表的一部分可以通过指定首末两个索引获得，方括号内以冒号分隔两者：

```
:let shortlist = mylist[2:-1]   " 得到列表 [3, "four"]
```

首索引的省略类似于用 0。末索引的省略类似于用 -1。

```
:let endlist = mylist[2:]       " 从项目 2 到结束：[3, "four"]
:let shortlist = mylist[2:2]    " 单个项目的列表：[3]
:let otherlist = mylist[:]      " 复制列表
```

如果首索引在列表末项之后或者末索引小于首索引，返回空列表。没有错误信息。

如果末索引大于等于列表的长度，使用列表长度减一：

```
:let mylist = [0, 1, 2, 3]
:echo mylist[2:8]               " 返回：[2, 3]
```

注意：mylist[s:e] 意味着用变量 "s:e" 作为索引。在 ":" 之前用单个字母作为变量要小心。需要的话加上空格：mylist[s : e]。

列表同一

list-identity

如果变量 "aa" 是列表，把它赋给另一个变量 "bb" 后，两个变量指向同一列表。因此，对列表 "aa" 的修改也同时修改了 "bb"：

```
:let aa = [1, 2, 3]
:let bb = aa
:call add(aa, 4)
:echo bb
[1, 2, 3, 4]
```

copy() 函数可以复制列表。如上所述，用 [:] 也可。这种方式建立列表的浅备份：改变列表中的列表项目仍然会修改复制列表的相应项目：

```
:let aa = [[1, 'a'], 2, 3]
:let bb = copy(aa)
:call add(aa, 4)
:let aa[0][1] = 'aaa'
:echo aa
[[1, aaa], 2, 3, 4]
:echo bb
[[1, aaa], 2, 3]
```

要建立一个完全独立的列表，用 deepcopy() 。它递归地建立列表值的备份。最深可达 100 层。

可用操作符 "is" 检查两个变量是否指向同一个列表。"isnot" 刚好相反。与此对照，"==" 比较两个列表的值是否相同。

```
:let alist = [1, 2, 3]
:let blist = [1, 2, 3]
:echo alist is blist
0
:echo alist == blist
1
```

比较列表时 注意：如果长度相同，所有项目用 "==" 的比较的结果也相同，两个列表就认为相同。有一个例外：数值和字符串总被认为不相同。这里不进行自动类型转换，而在变量间直接用 "==" 却不是如此。例如：

```
echo 4 == "4"
1
echo [4] == ["4"]
0
```

因此可以说，列表的比较比数值和字符串的比较更严格。你同样可以用这种方式比较简单类型的值，把它们放到列表里就行了：

```
:let a = 5
:let b = "5"
:echo a == b
1
:echo [a] == [b]
0
```

列表解包

要给列表项目解包，即把它们分别存入单独的变量，用方括号把变量括起来，如同把它们当作列表项目：

```
:let [var1, var2] = mylist
```

如果变量和列表的项目数量不同，报错。要处理列表中所有额外的项目，加上 ";" 和单个变量：

```
:let [var1, var2; rest] = mylist
```

它的工作方式就像：

```
:let var1 = mylist[0]
:let var2 = mylist[1]
:let rest = mylist[2:]
```

如果只有两个项目，不会报错。这时 "rest" 成为空表。

列表修改

list-modification

要修改列表的指定项目，用 :let ：

```
:let list[4] = "four"
:let listlist[0][3] = item
```

要修改列表的一部分，可以指定要修改的首末项目。提供的值的个数必须不少于该范围内的项目数：

```
:let list[3:5] = [3, 4, 5]
```

给列表增加和删除项目可以通过函数完成。一些例子如下：

```
:call insert(list, 'a')          " 在最前面插入 'a'
:call insert(list, 'a', 3)       " 在 list[3] 前插入项目 'a'
:call add(list, "new")           " 在最后附加字符串项目
:call add(list, [1, 2])          " 在最后附加新的列表项目
:call extend(list, [1, 2])       " 在最后扩展列表，使之多包含两个项目
:let i = remove(list, 3)         " 删除项目 3
:unlet list[3]                   " 同上
:let l = remove(list, 3, -1)     " 从项目 3 删除到最后
:unlet list[3 : ]                " 同上
:call filter(list, 'v:val !~ "x"')  " 删除有 'x' 的项目
```

改变列表项目的顺序：

```
:call sort(list)                 " 按字母给列表排序
:call reverse(list)              " 反转项目的顺序
:call uniq(sort(list))           " 排序并删除重复项
```

For 循环

:for 循环为每个列表项目执行命令。一个变量被依次设为每个列表项目。例如：

```
:for item in mylist
:   call Doit(item)
:endfor
```

它的工作方式就像：

```
:let index = 0
:while index < len(mylist)
:   let item = mylist[index]
:   :call Doit(item)
:   let index = index + 1
:endwhile
```

如果你只是想要修改每个列表项目，`map()` 函数比 `for` 循环简单得多。

就像 `:let` 命令，`:for` 也可以接受变量的列表。这需要参数是列表的列表。

```
:for [lnum, col] in [[1, 3], [2, 8], [3, 0]]
:   call Doit(lnum, col)
:endfor
```

这就像对列表的每个项目使用了 `:let` 命令。重复一次，类型必须相同，否则会报错。

也可以用变量保存列表变量的其余项目：

```
:for [i, j; rest] in listlist
:   call Doit(i, j)
:   if !empty(rest)
:      echo "remainder: " . string(rest)
:   endif
:endfor
```

列表的相关函数

E714

可用于列表的函数：

```
:let r = call(funcname, list)    " 调用带参数列表的函数
:if empty(list)                  " 检查 list 是否为空
:let l = len(list)               " list 项目的数量
:let big = max(list)             " list 项目的最大值
:let small = min(list)           " list 项目的最小值
:let xs = count(list, 'x')       " 计算 list 里 'x' 出现的次数
:let i = index(list, 'x')        " list 第一个 'x' 的位置
:let lines = getline(1, 10)      " 得到缓冲区十行文本行
:call append('$', lines)         " 附加若干文本行到缓冲区尾部
:let list = split("a b c")       " 用字符串中的项目建立列表
:let string = join(list, ', ')   " 用 list 项目构造字符串
:let s = string(list)            " list 的字符串表示
:call map(list, '">> " . v:val')  " 在每个项目前加上 ">> "
```

不要忘记组合使用不同功能可以简化任务。例如，要计算列表中所有数值的总和：

```
:exe 'let sum = ' . join(nrlist, '+')
```

1.4 字典

dict Dict Dictionaries Dictionary

字典是关联数组：每个项目有一个键和一个值。用键可以定位项目，而项目的存储不能确定任何特定顺序。

字典建立

E720 E721 E722 E723

字典通过花括号里逗号分隔的项目列表建立。每个项目包含以冒号分隔的键和值。一个键只能出现一次。例如：

```
:let mydict = {1: 'one', 2: 'two', 3: 'three'}
:let emptydict = {}
```

E713 E716 E717

键必须是字符串。用数值也可以，但它总被自动转换为字符串。所以字符串 '4' 和数值 4 总会找到相同的项目。 注意 字符串 '04' 和数值 04 是不一样的，因为后者被转换成字符串 '4'。空字符串也可用作键。

literal-Dict #{}

要不想给每个键都围上引号，可以用 #{} 形式。这要求键只包含 ASCII 字母、数位、'-' 和 '_'。例如：

```
:let mydict = #{zero: 0, one_key: 1, two-key: 2, 333: 3}
```

注意 这里的 333 代表字符串 "333"。空键用 #{} 是不可能的。

值可以是任何表达式。如果值本身是字典，就可以建立嵌套的字典：

```
:let nestdict = {1: {11: 'a', 12: 'b'}, 2: {21: 'c'}}
```

忽略末项之后的逗号。

访问项目

常见的访问项目的方式是把键放入方括号：

```
:let val = mydict["one"]
:let mydict["four"] = 4
```

用这种方式可以给已存在的字典增加新项目，这和列表不同。

如果键只包含字母、数字和下划线，可以使用如下形式 expr-entry ：

```
:let val = mydict.one
:let mydict.four = 4
```

因为项目可以是包括列表和字典的任何类型，你可以反复使用索引和键进行访问：

```
:echo dict.key[idx].key
```

字典到列表的转换

你可以循环遍历字典的所有项目。为此，你需要把字典转为列表，然后把它传递给 :for 。

通常，你期望遍历所有的键，用 keys() 函数就可以了：

```
:for key in keys(mydict)
:   echo key . ': ' . mydict[key]
:endfor
```

键列表没有经过排序。你可能希望先进行排序：

```
:for key in sort(keys(mydict))
```

要遍历所有的值，用 values() 函数：

```
:for v in values(mydict)
```

```
:   echo "value: " . v
:endfor
```

如果你想同时得到键和值，用 items() 函数。它返回一个列表，其中每个项目是两个项目的列表：键和值：

```
:for [key, value] in items(mydict)
:   echo key . ': ' . value
:endfor
```

字典同一

dict-identity

就像列表那样，你需要用 copy() 和 deepcopy() 来构造字典的备份。否则，赋值产生的结果会引用同一个字典：

```
:let onedict = {'a': 1, 'b': 2}
:let adict = onedict
:let adict['a'] = 11
:echo onedict['a']
11
```

如果所有的键-值组对的比较结果相同，两个字典比较的结果也相同。详情见 list-identity 。

字典修改

dict-modification

要修改字典已经存在的项目或者增加新的项目，用 :let :

```
:let dict[4] = "four"
:let dict['one'] = item
```

从字典里删除项目可以通过 remove() 或 :unlet 完成。
从 dict 里删除键 "aaa" 的项目有三种方法：

```
:let i = remove(dict, 'aaa')
:unlet dict.aaa
:unlet dict['aaa']
```

两个字典的合并可以用 extend() :

```
:call extend(adict, bdict)
```

这使得 adict 得到扩展，加入所有的 bdict 项目。对于重复的键，adict 项目被覆盖。可选的第三个参数可以改变这一点。
注意 这不影响字典项目的顺序，不要希望 ":echo adict" 会先显示原有 adict 项目，然后再显示 bdict 的项目。

从字典里删除多个项目可以用 filter() 完成：

```
:call filter(dict, 'v:val =~ "x"')
```

删除 "dict" 里所有值不匹配 "x" 的项目。

字典函数

Dictionary-function self E725 E862

定义函数时，如果带有 "dict" 属性，可以以一种特殊方式使用字典。例如：

```
:function Mylen() dict
:   return len(self.data)
:endfunction
```

```
:let mydict = {'data': [0, 1, 2, 3], 'len': function("Mylen")}
:echo mydict.len()
```

这类似于面向对象编程的方法。字典项目用作 Funcref 。局部变量 "self" 引用函数所在的字典。

字典里也可以加入指向没有 "dict" 属性的函数的函数引用，不过这时无法使用 "self" 变量。

numbered-function anonymous-function

要避免额外的函数名，可以定义时直接赋给字典：

```
:let mydict = {'data': [0, 1, 2, 3]}
:function mydict.len()
:   return len(self.data)
:endfunction
:echo mydict.len()
```

该函数会得到一个编号，而 dict.len 的值是指向此函数的 Funcref 。该函数只能通过 Funcref 访问。如果没有任何 Funcref 引用，它会被自动删除。

编号函数不一定要有 "dict" 属性。

如果你的编号函数有错，可以用一个技巧知道它是什么内容。假定函数是 42，命令为：

```
:function {42}
```

字典相关函数

E715

可以用于字典的函数：

```
:if has_key(dict, 'foo')        " 如果 dict 有带 "foo" 键的项目则为真
:if empty(dict)                 " 如果 dict 为空则为真
:let l = len(dict)              " dict 项目的数量
:let big = max(dict)            " dict 项目的最大值
:let small = min(dict)          " dict 项目的最小值
:let xs = count(dict, 'x')      " 统计 dict 里 'x' 出现的数目
:let s = string(dict)           " dict 的字符串表示
:call map(dict, '">> " . v:val')  " 在每个项目前加上 ">> "
```

1.5 blob

blob Blob Blobs E978

blob 是一个二进制对象。例如，可用来读取文件的映像并通过通道发送。

blob 基本类似于数值的 List ，其中每个数值是个 8 位字节的值，从 0 到 255。

blob 建立

可用 blob-literal 建立 blob：

```
:let b = 0zFF00ED015DAF
```

字节（一对十六进制字符）之间可以插入点号以提高可读性。不改变值本身：

```
:let b = 0zFF00.ED01.5DAF
```

用 readfile() 可从文件里读取 blob，其中 {type} 参数设为 "B"。例如：

```
:let b = readfile('image.png', 'B')
```

用 ch_readblob() 函数可从通道读取 blob。

blob 索引

blob-index E979

blob 之后用方括号放上索引可以访问 blob 中的字节。索引从零开始，因而首个字节的索引为零。

```
:let myblob = 0z00112233
:let byte = myblob[0]           " 读取首个字节: 0x00
:let byte = myblob[2]           " 读取第三个字节: 0x22
```

负数索引从末尾开始计算。索引 -1 引用 blob 最后一个字节，-2 是倒数第二个字节，等等。

```
:let last = myblob[-1]          " 读取最后一个字节: 0x33
```

用 get() 函数可以避免索引值非法的错误，如果索引不可用，返回 -1 或你指定的缺省值:

```
:echo get(myblob, idx)
:echo get(myblob, idx, 999)
```

blob 遍历

:for 循环为 blob 的每个字节执行命令。其中循环变量设为 blob 的每个字节。例如:

```
:for byte in 0z112233
:   call Doit(byte)
:endfor
```

会分别用 0x11、0x22 和 0x33 调用 Doit()。

blob 连接

用 "+" 操作符可以连接两个 blob:

```
:let longblob = myblob + 0z4455
:let myblob += 0z6677
```

要在原位修改 blob，可见下述的 blob-modification 。

blob 部分

blob 的一部分可用在方括号内指定以冒号分隔的开始处和结束处的索引来获得:

```
:let myblob = 0z00112233
:let shortblob = myblob[1:2]    " 返回 0z1122
:let shortblob = myblob[2:-1]   " 返回 0z2233
```

开始索引要是省略相当于零。结束索引要是省略相当于 -1。

```
:let endblob = myblob[2:]       " 从项目 2 到尾: 0z2233
:let shortblob = myblob[2:2]    " 一个字节的 blob: 0z22
:let otherblob = myblob[:]      " 建立 blob 的备份
```

如果开始索引超过 blob 的最后字节，或者结束索引在开始索引之前，返回空 blob。不报错。

如果结束索引等于或大于列表长度，使用长度减一：

```
:echo myblob[2:8]                " 返回: 0z2233
```

blob 修改

blob-modification

要修改 blob 的特定字节，可以这么用 :let :

```
:let blob[4] = 0x44
```

如果索引是 blob 最后索引加一，附加到尾部。再大的索引会报错。

要改变字节序列，可用 [:] 记法：

```
let blob[1:3] = 0z445566
```

被替换字节序列的长度必须和提供的值相等。 E972

要修改 blob 的部分，指定要修改的开始和结束字节。新值必须和范围的字节数相同：

```
:let blob[3:5] = 0z334455
```

也可用 add() 、 remove() 和 insert() 函数。

blob 同一

blob 间可以比较是否值相等：

```
if blob == 0z001122
```

也可以比较是否同一引用：

```
if blob is otherblob
```

blob-identity E977

当 "aa" 变量是 blob 而把它赋值给另一个变量 "bb" 时，两个变量指向同一个 blob。因而 "is" 操作符返回真值。

用 [:] 或 copy() 创建备份时值相等，但是不同的引用：

```
:let blob = 0z112233
:let blob2 = blob
:echo blob == blob2
1
:echo blob is blob2
1
:let blob3 = blob[:]
:echo blob == blob3
1
:echo blob is blob3
0
```

blob 建立备份可通过 copy() 函数。用 [:] 也可以，已有上述。

1.6 变量的更多细节

more-variables

如果你需要知道变量或表达式的类型，使用 type() 函数。

如果 'viminfo' 选项包含 '!' 标志位，大写开头且不包含小写字母的全局变量被保存在 viminfo 文件里 viminfo-file 。

如果 'sessionoptions' 选项包含 "global"，大写开头且包含至少一个小写字母的全局变量被保存在会话文件里 session-file 。

变量名	可以保存的位置
my_var_6	无
My_Var_6	会话文件
MY_VAR_6	viminfo 文件

可以使用花括号来构造变量名，见 curly-braces-names 。

---

2. 表达式语法 expression-syntax

表达式语法小结，优先级从低到高排列：

```
expr1  expr2
       expr2 ? expr1 : expr1   if-then-else

expr2  expr3
       expr3 || expr3 ...      逻辑或

expr3  expr4
       expr4 && expr4 ...      逻辑与

expr4  expr5
       expr5 == expr5          等于
       expr5 != expr5          不等于
       expr5 >  expr5          大于
       expr5 >= expr5          大于等于
       expr5 <  expr5          小于
       expr5 <= expr5          小于等于
       expr5 =~ expr5          匹配正规表达式
       expr5 !~ expr5          不匹配正规表达式

       expr5 ==? expr5         等于，忽略大小写
       expr5 ==# expr5         等于，匹配大小写
       等等                    如上，? 忽略大小写，# 则匹配之

       expr5 is expr5          相同的 List 、 Dictionary 或 Blob 实例
       expr5 isnot expr5       不同的 List 、 Dictionary 或 Blob 实例

expr5  expr6
       expr6 +  expr6 ...      数值加法、列表或 blob 连接
       expr6 -  expr6 ...      数值减法
       expr6 .  expr6 ...      字符串连接
       expr6 .. expr6 ...      字符串连接

expr6  expr7
       expr7 *  expr7 ...      数值乘法
       expr7 /  expr7 ...      数值除法
```

	expr7 % expr7 ...	数值求余
expr7	expr8	
	! expr7	逻辑非
	- expr7	一元减法：取反
	+ expr7	一元加法：原值
expr8	expr9	
	expr8[expr1]	字符串里的字节或者 List 的项目
	expr8[expr1 : expr1]	字符串子串或 List 的子列表
	expr8.name	Dictionary 的项目
	expr8(expr1, ...)	使用 Funcref 变量的函数调用
	expr8->name(expr1, ...)	method 调用
expr9	number	数值常数
	"string"	字符串常数，反斜杠有特殊含义
	'string'	字符串常数，' 加倍
	[expr1, ...]	List
	{expr1: expr1, ...}	Dictionary
	#{key: expr1, ...}	Dictionary
	&option	选项值
	(expr1)	嵌套表达式
	variable	内部变量
	va{ria}ble	带花括号的内部变量
	$VAR	环境变量
	@r	寄存器 'r' 的值
	function(expr1, ...)	函数调用
	func{ti}on(expr1, ...)	带花括号的函数调用
	{args -> expr1}	匿名函数表达式

"..." 标明这一层上的操作可以连接。比如：

```
&nu || &list && &shell == "csh"
```

同一层的表达式从左到右进行分析。

expr1 expr1 E109

---

expr2 ? expr1 : expr1

'?' 之前的表达式作为数值求值。如果结果为 TRUE，最终的结果是 '?' 和 ':' 之间的表达式的值，不然最终的结果是 ':' 之后的表达式的值。例如：

```
:echo lnum == 1 ? "top" : lnum
```

因为第一个表达式是 "expr2"，它不能包含另一个 ?:。另外两个表达式则没有这个限制，从而使得递归使用 ?: 成为可能。例如：

```
:echo lnum == 1 ? "top" : lnum == 1000 ? "last" : lnum
```

要使之可读，建议使用续行符 line-continuation ：

```
:echo lnum == 1
:\      ? "top"
:\      : lnum == 1000
```

```
:\              ? "last"
:\              : lnum
```

在 ':' 前，你总是应该加上空格，否则它可能被错误用在如 "a:1" 这样的变量里。

expr2 和 expr3

expr2 expr3

---

expr3 || expr3 ..	逻辑或	expr-barbar
expr4 && expr4 ..	逻辑与	expr-&&

"||" 和 "&&" 操作符左右两边各接受一个参数。参数是（或转化为）数值。运算结果是：

输入		输出	
n1	n2	n1 || n2	n1 && n2
FALSE	FALSE	FALSE	FALSE
FALSE	TRUE	TRUE	FALSE
TRUE	FALSE	TRUE	FALSE
TRUE	TRUE	TRUE	TRUE

操作符可以连接。比如：

```
&nu || &list && &shell == "csh"
```

注意 "&&" 比 "||" 优先级高，所以这等价于：

```
&nu || (&list && &shell == "csh")
```

一旦结果可以确定，表达式使用 "短路" 计算，也就是，不再计算后面的参数，这和 C 的情形类似。比如：

```
let a = 1
echo a || b
```

这是合法的，即使没有叫 "b" 的变量也是如此。因为 "a" 已经是 TRUE ，结果必然是 TRUE 。下面的情形类似：

```
echo exists("b") && b == "yes"
```

无论 "b" 定义与否，这是合法的。第二个子句只有在 "b" 定义的时候才会被计算。

expr4

expr4

---

expr5 {cmp} expr5

比较两个 expr 表达式，如果结果为假，返回 0，如果结果为真，返回 1。

expr-==	expr-!=	expr->	expr->=
expr-<	expr-<=	expr-=~	expr-!~
expr-==#	expr-!=#	expr->#	expr->=#
expr-<#	expr-<=#	expr-=~#	expr-!~#

	expr-==?	expr-!=?	expr->?	expr->=?
	expr-<?	expr-<=?	expr-=~?	expr-!~?
	expr-is	expr-isnot	expr-is#	expr-isnot#
	expr-is?	expr-isnot?		
使用 'ignorecase'		匹配大小写	忽略大小写	
等于	==	==#	==?	
不等于	!=	!=#	!=?	
大于	>	>#	>?	
大于等于	>=	>=#	>=?	
小于	<	<#	<?	
小于等于	<=	<=#	<=?	
匹配正规表达式	=~	=~#	=~?	
不匹配正规表达式	!~	!~#	!~?	
相同实例	is	is#	is?	
不同实例	isnot	isnot#	isnot?	

示例:
"abc" ==# "Abc" 结果为 0
"abc" ==? "Abc" 结果为 1
"abc" == "Abc" 如果置位了 'ignorecase'，结果为 1，不然结果为 0

E691 E692

List 只能和 List 比较，而且只能用 "等于"、"不等于"、"is" 和 "isnot"。比较针对列表的值，递归进行。忽略大小写意味着比较项目的值时忽略大小写。

E735 E736

Dictionary 只能和 Dictionary 比较，而且只能用 "等于"、"不等于"、"is" 和 "isnot"。比较针对 Dictionary 的键/值，递归进行。忽略大小写意味着比较项目的值时忽略大小写。

E694

Funcref 只能和 Funcref 比较，而且只能用 "等于"、"不等于"、"is" 和 "isnot"。这里永不忽略大小写。参数或字典是否绑定（即偏函数的情况）是有关系的。绑定的字典必须相等（或 "is" 的情况下，相同实例），参数亦然。

要比较函数引用是否指向相同的函数但忽略绑定的字典和参数，用 get() 来取得函数名:

```
if get(Part1, 'name') == get(Part2, 'name')
   " Part1 and Part2 refer to the same function
```

List 、 Dictionary 或 Blob 用 "is" 或 "isnot" 时，检查表达式是否指向同一个 List 、 Dictionary 或 Blob 实例。一个 List 的备份和原来的 List 不同。如果不是 List 、 Dictionary 或 Blob ，用 "is" 等价于用 "等于"，而 "isnot" 等价于 "不等于"，有一点区别：不同的类型总认为有不同的值:

```
echo 4 == '4'
1
echo 4 is '4'
0
echo 0 is []
0
```

"is#"/"isnot#" 和 "is?"/"isnot?" 用于强制匹配和忽略大小写。

如果比较字符串和数值，字符串被转化成数值，而比较是在数值之间进行的。这意味着:

```
echo 0 == 'x'
1
```
因为 'x' 转化的数值为零。不过：
```
echo [0] == ['x']
0
```
在列表或字典里不进行这种转换。

如果比较两个字符串，使用 strcmp() 或 stricmp()。因而，比较的是数学上的差异（比较字节码），而不必然是本地语言的字母的差异。

如果操作符后带上 '#'，或者 'ignorecase' 关闭时使用无 '#' 的版本时，比较使用 strcmp(): 大小写相关。

如果操作符后带上 '?'，或者 'ignorecase' 打开时使用无 '?' 的版本时，比较使用 stricmp(): 大小写无关。

这里 'smartcase' 不适用。

"=~" 和 "!~" 操作符使用右边的参数作为模式来匹配左边的参数。模式的定义见 pattern 。匹配进行时，总是假设置位了 'magic' 并且 'cpoptions' 为空，无论 'magic' 或 'cpoptions' 实际的值为何。这使得脚本可移植。要避免在正规表达式里使用的反斜杠需要加倍的问题，可以使用单引号的字符串，见 literal-string 。
既然字符串假定为单行，多行的模式（包含 \n，即反斜杠-n）不会被匹配。不过，按本义出现的单个 NL 字符可以像普通字符一样匹配。比如：
```
"foo\nbar" =~ "\n"      结果为 1
"foo\nbar" =~ "\\n"     结果为 0
```

expr5 和 expr6 expr5 expr6

---

expr6 + expr6	数值加法、List 或 Blob 连接	expr-+
expr6 - expr6	数值减法	expr--
expr6 . expr6	字符串连接	expr-.
expr6 .. expr6	字符串连接	expr-..

Lists 只能用 "+"，而且两个 expr6 必须都是列表。返回两者连接以后的新列表。

字符串连接建议使用 ".."，因为 "." 有二义性，也用于 Dict 的成员访问和浮点数。
当 vimscript-version 为 2 或更高时，不允许使用 "."。

expr7 * expr7	数值乘法	expr-star
expr7 / expr7	数值除法	expr-/
expr7 % expr7	数值求余	expr-%

除了 "." 和 ".." 以外，这里所有的操作都把字符串转化成数值。
按位操作运算见 and() 、 or() 和 xor() 。

注意 "+" 和 "." 的差异：
```
"123" + "456" = 579
"123" . "456" = "123456"
```

因为 '.' 和 '+' 与 '-' 的优先级相同，你需要把：
```
1 . 90 + 90.0
```
看作：

```
(1 . 90) + 90.0
```
这没问题，因为字符串 "190" 被自动转换为数值 190，然后和浮点数 90.0 相加。不过：
```
1 . 90 * 90.0
```
应被看作：
```
1 . (90 * 90.0)
```
因为 '.' 的优先级比 '*' 低，这 不能 工作，因为它试图连接浮点数和字符串。

数值除以零时，结果取决于该值：
```
 0 / 0  = -0x80000000  (类似于浮点数的 NaN)
>0 / 0  =  0x7fffffff  (类似于正无穷大)
<0 / 0  = -0x7fffffff  (类似于负无穷大)
(Vim 7.2 之前，总是返回 0x7fffffff)
```

启用 64-位数值支持时：
```
 0 / 0  = -0x8000000000000000  (类似于浮点数的 NaN)
>0 / 0  =  0x7fffffffffffffff  (类似于正无穷大)
<0 / 0  = -0x7fffffffffffffff  (类似于负无穷大)
```

如果 '%' 的右边为零，结果为 0。

这些操作不适用于 Funcref 。

而 % 也不适用于浮点数。 E804

expr7 expr7

! expr7	逻辑非	expr-!
- expr7	一元减法：取反	expr-unary--
+ expr7	一元加法：原值	expr-unary-+

'!' 把 TRUE 变为 FALSE ， FALSE 变为 TRUE (一)。
'-' 改变数值的符号。
'+' 保持原值。

字符串会先转化为数值。

可以重复和混合这三种运算。例如：
```
!-1         == 0
!!8         == 1
--9         == 9
```

expr8 expr8

此处表达式或是 expr9 ，或是任意顺序的下面各项之一的序列。例如这些都可以：
```
expr8[expr1].name
expr8.name[expr1]
expr8(expr1, ...)[expr1].name
expr8->(expr1, ...)[expr1]
```
计算总是从左到右。

expr8[expr1]　　字符串或 List 的项目　　expr-[] E111 E909 subscript

如果 expr8 是数值或字符串，结果是字符串，包含 expr8 里第 expr1 个字节。expr8 视作字符串，expr1 视作数值。这里不识别多字节编码，但可考虑使用 byteidx()，或用 split() 把字符串变为字符的列表。

索引 0 给出第一个字节。这和 C 类同。要小心：文本列号可是从 1 开始的！例如，要得到光标所在的字节：

```
:let c = getline(line("."))[col(".") - 1]
```

如果字符串的长度小于索引值，结果为空字符串。负索引总是给出空字符串（原因：反向兼容）。用 [-1:] 得到最后一个字节。

如果 expr8 是 List，返回索引值为 expr1 的项目。可用的索引值见 list-index。如果索引越界，产生错误。例如：

```
:let item = mylist[-1]          " 得到最后一个项目
```

一般的，如果 List 索引大于等于 List 的长度，或者比 List 的长度更负，产生错误。

expr8[expr1a : expr1b]　子字符串或子列表　　expr-[:]

如果 expr8 是数值或字符串，结果是子字符串，包含第 expr1a 到第 expr1b（包含）个字节。expr8 视作字符串，expr1a 和 expr1b 视作数值。这里不识别多字节编码，需用 byteidx() 来计算索引值。

如果省略 expr1a，用零。如果省略 expr1b，用字符串的长度减一。

可以用负数来从字符串尾部开始计算位置。-1 代表最后一个字符，-2 倒数第二个，依此类推。

如果索引越界，忽略这些字符。如果 expr1b 小于 expr1a，结果是空字符串。

例如：

```
:let c = name[-1:]              " 字符串最后一个字节
:let c = name[-2:-2]            " 字符串倒数第二个字节
:let s = line(".")[4:]          " 从第五个字节到最后
:let s = s[:-3]                 " 删除最后两个字节
```

slice

如果 expr8 是 List，结果是新的 List，包含 expr1 和 expr1b 索引指定的项目。和上面描述的字符串情形类似。另见下面的 sublist。例如：

```
:let l = mylist[:3]             " 前四个项目
:let l = mylist[4:4]            " 单个项目的列表
:let l = mylist[:]              " 列表的浅备份
```

如果 expr8 是 Blob，结果是新的 Blob，包含从索引 expr1a 到 expr1b 的字节，闭区间。例如：

```
:let b = 0zDEADBEEF
:let bs = b[1:2]                " 0zADBE
:let bs = b[:]                  " 0zDEADBEEF 的备份
```

在 Funcref 上用 expr8[expr1] 或 expr8[expr1a : expr1b] 出错。

小心命名空间和变量后加冒号的子列表用法引起的混淆：

```
mylist[n:]      " 使用变量 n
mylist[s:]      " 使用命名空间 s:，报错！
```

expr8.name　　Dictionary 的项目　　expr-entry

如果 expr8 是一个 Dictionary 且后跟句号再跟一个名字，该名字用作 Dictionary 的键。这相当于：expr8[name]。

该名字必须由字母数字字符组成。这和变量名一样，不过这里可以用数字开始。但不能用花括号。

句号前后不能用空白。

例如：

```
:let dict = {"one": 1, 2: "two"}
:echo dict.one          " 显示 "1"
:echo dict.2            " 显示 "two"
:echo dict .2           " 因为点号前有空格，报错
```

注意 句号也用于字符串连接。要避免混淆，用于字符串连接的句号前后加上空白。

expr8(expr1, ...)　　Funcref 函数调用

如果 expr8 是 Funcref 类型的变量，调用它指向的函数。

expr8->name([args])　　方法调用　　method　->
expr8->{lambda}([args])

E276

对同时作为全局函数存在的方法而言，这等价于：

```
name(expr8 [, args])
```

也有专用于 "expr8" 类型的方法。

可用于链式调用，把一个方法的结果传给下一个方法：

```
mylist->filter(filterexpr)->map(mapexpr)->sort()->join()
```

使用匿名函数的示例：

```
GetPercentage()->{x -> x * 100}()->printf('%d%%')
```

使用 -> 时，先应用 expr7 操作符，所以：

```
-1.234->string()
```

等价于：

```
(-1.234)->string()
```

而 不是 ：

```
-(1.234->string())
```

E274

"->name(" 内部不能包含空白。"->" 之前和 "(" 之后可有空白，所以可以这样断行：

```
mylist
\ ->filter(filterexpr)
```

```
\ ->map(mapexpr)
\ ->sort()
\ ->join()
```

使用匿名函数形式时，} 和 (之间不能有空白。

expr9

number

---

number 数值常数 expr-number
hex-number octal-number binary-number

十进制、十六进制 (0x 或 0X 开始)、二进制 (0b 或 0B 开始) 和 八进制 (0 开始)。

floating-point-format

浮点数可用两种形式给出：

```
[-+]{N}.{M}
[-+]{N}.{M}[eE][-+]{exp}
```

{N} 和 {M} 都是数值。{N} 和 {M} 都必须存在，且只能包含数位。
[-+] 意味着有一个可选的正负号。
{exp} 是指数部分，以 10 为基。
只能接受小数点，逗号不行。这和当前的 locale 无关。
{仅当编译时加入 +float 特性才有效}

示例：

```
123.456
+0.0001
55.0
-0.123
1.234e03
1.0E-6
-3.1416e+88
```

下面的形式是 非法的 ：

```
3.              {M} 为空
1e40            {M} 为空
```

理据：
浮点数引入之前，文本 "123.456" 被解释为两个数值 "123" 和 "456"，转换为字符串，然后进行连接而生成字符串 "123456"。这被认为没有意义，也没有找到有意使用此特性的 Vim 脚本，因此我们采纳了这种普遍的浮点数记法，而接受其后向不兼容性。

float-pi float-e

可以复制-粘贴的一些常用值：

```
:let pi = 3.14159265359
:let e  = 2.71828182846
```

或者，如果不要直接写浮点常量，可用相应的函数，就像这样：

```
:let pi = acos(-1.0)
:let e  = exp(1.0)
```

floating-point-precision

浮点数的精度和取值范围取决于 Vim 编译时使用的库如何理解 "double"。运行时无法改变。

浮点数 Float 的显示缺省使用 6 位十进制位，类似于 printf("%g", f)。使用 printf() 函数时可以指定其它位数。例如：

```
:echo printf('%.15e', atan(1))
7.853981633974483e-01
```

string string String expr-string E114

"string" 字符串常数 expr-quote

注意 使用的是双引号。

字符串常数接受以下特殊字符：

\...	三位八进制数（例如，"\316"）
\..	两位八进制数（必须后跟非数字）
\.	一位八进制数（必须后跟非数字）
\x..	两位十六进制数指定的字节（例如，"\x1f"）
\x.	一位十六进制数指定的字节（必须后跟非十六进制数字）
\X..	等同于 \x..
\X.	等同于 \x.
\u....	四位十六进制指定的字符。根据 'encoding' 的当前值决定的编码进行存贮（例如，"\u02a4"）
\U....	等同于 \u 但接受多达 8 位十六进制数。
\b	退格 <BS>
\e	escape <Esc>
\f	换页 <FF>
\n	换行 <NL>
\r	回车 <CR>
\t	制表 <Tab>
\\	反斜杠
\"	双引号
\<xxx>	"xxx" 命名的特殊字符，例如 "\<C-W>" 代表 CTRL-W。用于映射，0x80 字节被转义。 双引号必须转义："<M-\">"。 不要用 <Char-xxxx> 来得到 utf-8 字符，用上面提到的 \uxxxxx。

注意 "\xff" 保存为字节 255，在某些编码中它是不合法的。使用 "\u00ff" 可以按照 'encoding' 的当前值保存字符 255。

注意 "\000" 和 "\x00" 强制字符串结束。

blob-literal blob-literal E973

0z 或 0Z 开头的十六进制序列，字符数目不限。序列必须是偶数位的十六进制字符。例如：

```
:let b = 0zFF00ED015DAF
```

literal-string literal-string E115

'string' 字符串常数 expr-'

注意 使用的是单引号。

字符串这里按原义出现。不去掉反斜杠，它也没有特殊含义。唯一的特例是两个单引号代表一个单引号。

单引号字符串有助于模式的使用，因为反斜杠不再需要加倍。以下两个命令等价：

```
if a =~ "\\s*"
if a =~ '\s*'
```

option expr-option E112 E113

&option 选项值，如有存在，使用局部值
&g:option 全局选项值
&l:option 局部选项值

例如：

```
echo "tabstop is " . &tabstop
if &insertmode
```

这里可以使用任何选项值。见 options 。如果指定要使用局部值，但不存在局部于缓冲区或局部于窗口的选项，则还是使用全局值。

register expr-register @r

@r 寄存器 'r' 的值

结果是命名寄存器的内容，以单个字符串表达。换行符在需要时会被插入。要得到无名寄存器的内容，使用 @" 或 @@。可用寄存器的相关解释可见 registers 。

如果用 '=' 寄存器，你得到表达式自身，而不是它计算的结果。用 eval() 来进行计算。

nesting expr-nesting E110

(expr1) 嵌套表达式

environment variable expr-env

$VAR 环境变量

任何环境变量的字符串值。如果该环境变量没有定义，结果为空字符串。

也可用 getenv() 和 setenv() 函数，它们支持非字母数字的环境变量名。
可用 environ() 函数得到包含所有环境变量的字典。

expr-env-expand

注意 直接使用 $VAR 和使用 expand("$VAR") 有区别。直接使用的形式只能扩展当前 Vim 会话所知的环境变量。使用 expand() 会先尝试当前 Vim 会话所知的环境变量，如果不成功，则使用外壳扩展该变量。这会变慢，但可以用来扩展只有外壳知道的变量。
例如:

```
:echo $shell
:echo expand("$shell")
```

前者可能不会回显任何内容，后者会回显 $shell 变量 (如果你的外壳支持的话)。

internal variable expr-variable

---

variable 内部变量
见下面的 internal-variables 。

function call expr-function E116 E118 E119 E120

---

function(expr1, ...) 函数调用
见下面的 functions 。

lambda expression expr-lambda lambda

{args -> expr1} 匿名函数表达式

匿名函数表达式创建一个新的无名函数，返回 expr1 的计算结果。匿名函数表达式和 user-functions 的区别如下:

1. 匿名函数表达式的本体是 expr1 而不是 Ex 命令序列。
2. 前缀 "a:" 不用于参数。例如:

```
:let F = {arg1, arg2 -> arg1 - arg2}
:echo F(5, 2)
3
```

参数可选。例如:

```
:let F = {-> 'error function'}
:echo F()
error function
```

closure

匿名函数表达式可能访问外层变量和参数。这通常被称为闭包。下例中匿名函数可以使用已存在于函数作用域的 "i" 和 "a:arg"。在函数返回后它们依然有效:

```
:function Foo(arg)
:  let i = 3
:  return {x -> x + i - a:arg}
:endfunction
:let Bar = Foo(4)
:echo Bar(6)
```

```
5
```

注意 变量必须在 lambda 定义之前已存在于外部作用域。另见 :func-closure 。

可以这样来检查匿名函数和闭包的支持：

```
if has('lambda')
```

示例如何在 sort() 、 map() 和 filter() 中使用匿名函数：

```
:echo map([1, 2, 3], {idx, val -> val + 1})
[2, 3, 4]
:echo sort([3,7,2,1,4], {a, b -> a - b})
[1, 2, 3, 4, 7]
```

匿名函数表达式也可用于通道、作业和定时器中：

```
:let timer = timer_start(500,
                \ {-> execute("echo 'Handler called'", "")},
                \ {'repeat': 3})
Handler called
Handler called
Handler called
```

注意 execute() 是如何用来执行 Ex 命令的。这不怎样好看。

匿名函数表达式使用的内部名形如 '<lambda>42'。如果有关于某匿名函数的错误，可以用下述命令查看它的定义：

```
:function {'<lambda>42'}
```

另见： numbered-function

---

3. 内部变量 internal-variables E461

内部变量的名字由字母、数字和 '_' 组成。但不能由数字开始。可以使用花括号，见 curly-braces-names 。

内部变量通过 ":let" 命令建立 :let 。
内部变量通过 ":unlet" 命令显式删除 :unlet 。
使用非内部变量的名字或引用已经删除的内部变量会产生错误。

变量有不同的命名空间，根据附加的前缀决定：

	(无)	函数内：局部于函数；否则：全局
buffer-variable	b:	局部于当前缓冲区。
window-variable	w:	局部于当前窗口。
tabpage-variable	t:	局部于当前标签页。
global-variable	g:	全局。
local-variable	l:	局部于函数。
script-variable	s:	局部于 :source 的 Vim 脚本。
function-argument	a:	函数参数（只限于函数内使用）。
vim-variable	v:	Vim 预定义的全局变量。

作用域本身可以用作 Dictionary 。例如，要删除所有局部于脚本的变量：

```
:for k in keys(s:)
:    unlet s:[k]
:endfor
```

buffer-variable b:var b:

"b:" 开头的变量名局部于当前缓冲区。这样，你可以为每个缓冲区定义不同的 "b:foo" 变量。这种变量在缓冲区被删除时（:bwipeout 或 :bdelete :bdelete ）同时被删除。

预定义了如下的缓冲区局部变量：

b:changedtick changetick

b:changedtick 当前缓冲区的改变次数。每次改变都会递增。撤销命令在此情形下也被视作一次改变。写入缓冲区后复位 'modified' 也算。
这可用来在缓冲区发生改变时执行一些动作。比如：

```
:if my_changedtick != b:changedtick
: let my_changedtick = b:changedtick
: call My_Update()
:endif
```

不可改变或删除 b:changedtick 变量。

window-variable w:var w:

"w:" 开头的变量名局部于当前窗口。窗口关闭时被删除。

tabpage-variable t:var t:

"t" 开始的变量名局部于当前标签页。标签页关闭时，这些变量被删除。
{仅当编译时加入 +windows 特性才有效}

global-variable g:var g:

函数内部，全局变量可以通过 "g:" 访问。如果不提供前缀，会使用函数的局部变量。在其他地方，如果你想的话。也可以使用 "g:"。

local-variable l:var l:

访问函数的局部变量无需任何前缀。但如果你想要，可以使用 "l:"。不过，如果没有 "l:" 前缀，你可能会和保留的变量名冲突。例如 "count"。它本身指代 "v:count"。但使用了 "l:count" 你就可以使用同名的局部变量。

script-variable s:var

Vim 脚本里，可以使用 "s:" 开头的变量。它们不能在脚本之外访问，因而可以称为局部于脚本的变量。

它们可以用于：
- 载入脚本时执行的命令
- 脚本定义的函数
- 脚本定义的自动命令
- 脚本定义的函数和自动命令里定义的函数和自动命令（递归）
- 脚本里定义的用户定义命令

但不能用在：
- 该脚本载入的其它脚本
- 映射
- 菜单
- 等等

脚本变量可以用来防止和全局变量名的冲突。看看这个例子：

```
let s:counter = 0
function MyCounter()
  let s:counter = s:counter + 1
```

```
  echo s:counter
endfunction
command Tick call MyCounter()
```

你可以从任何脚本里启动 "Tick"，但那个脚本里的 "s:counter" 变量不会被改变，只有在 "Tick" 定义所在脚本的 "s:counter" 才会。

另一个完成相同功能的例子：

```
let s:counter = 0
command Tick let s:counter = s:counter + 1 | echo s:counter
```

如果调用函数或者启动用户定义命令，脚本变量的上下文设置为函数和命令定义所在的脚本。

脚本变量也可用于脚本里定义的函数里定义的函数。例如：

```
let s:counter = 0
function StartCounting(incr)
  if a:incr
    function MyCounter()
      let s:counter = s:counter + 1
    endfunction
  else
    function MyCounter()
      let s:counter = s:counter - 1
    endfunction
  endif
endfunction
```

调用 StartCounting() 时，定义 MyCounter() 函数或者递增或者递减计数器。不管 StartCounting() 在哪里调用，s:counter 变量总可以在 MyCounter() 里访问。

如果相同的脚本多次执行，使用的是同一个脚本变量。只要 Vim 还在运行，就保持有效。这可以用于维护计数：

```
if !exists("s:counter")
  let s:counter = 1
  echo "脚本首次执行"
else
  let s:counter = s:counter + 1
  echo "脚本现在执行了 " . s:counter . " 次"
endif
```

注意 这意味着 filetype 插件不能为每个缓冲区提供不同的脚本变量。这时应使用缓冲区的局部变量 b:var 。

预定义的 VIM 变量: vim-variable v:var v: E963

有些变量可以被用户设置，但不能改变类型。

v:argv 调用 Vim 时的命令行参数。这是字符串列表。首项是 Vim 命令。 v:argv argv-variable

v:beval_col　　v:beval_col　beval_col-variable
鼠标指针所在的列号，即 v:beval_lnum 行中的字节位置。
仅当计算 'balloonexpr' 选项时有效。

v:beval_bufnr　　v:beval_bufnr　beval_bufnr-variable
鼠标指针所在的缓冲区号。仅当计算 'balloonexpr' 选项时有效。

v:beval_lnum　　v:beval_lnum　beval_lnum-variable
鼠标指针所在的行号。仅当计算 'balloonexpr' 选项时有效。

v:beval_text　　v:beval_text　beval_text-variable
鼠标指针所在或之后的文本。通常是一个单词，可用于调试 C 程序。
此处用到 'iskeyword'，但也包括此位置之前的句号和 "->"。如果在 ']' 上，使用它之前的文本，包括匹配的 '[' 和它之前的单词。如果在单行的可视区域上，使用高亮文本。另见 <cexpr> 。
仅当计算 'balloonexpr' 选项时有效。

v:beval_winnr　　v:beval_winnr　beval_winnr-variable
鼠标指针所在的窗口号。仅当计算 'balloonexpr' 选项时有效。首个窗口的编号为零 (这和多数需要窗口编号的地方不同)。

v:beval_winid　　v:beval_winid　beval_winid-variable
鼠标指针所在的窗口 ID window-ID 。其它类同于 v:beval_winnr。

v:char　　v:char　char-variable
计算 'formatexpr' 时使用的参数和用于带 <expr> 的缩写中输入的字符 :map-<expr> 。
也用于 InsertCharPre 和 InsertEnter 事件。

v:charconvert_from　　v:charconvert_from　charconvert_from-variable
要转换的文件字符编码名。只在计算 'charconvert' 选项时有效。

v:charconvert_to　　v:charconvert_to　charconvert_to-variable
转换后的文件字符编码名。只在计算 'charconvert' 选项时有效。

v:cmdarg　　v:cmdarg　cmdarg-variable
该变量有两个目的:
1. 文件读写命令的额外参数。目前，它们包括 "++enc=" 和 "++ff="。该变量在文件读写命令的自动命令事件激活之前设置。开头有一个空格，以便直接把该变量附加到读写命令之后。注意：这里不包括 "+cmd" 参数，因为它总要被执行的。
2. 使用 ":hardcopy" 打印 PostScript 文件时，":hardcopy" 命令的参数。在 'printexpr' 里用得到。

v:cmdbang　　v:cmdbang　cmdbang-variable
文件读写命令时，和 v:cmdarg 设置的时间类似。如果使用了 "!"，其值为 1，不然为 0。注意 它只能用于自动命令。用户命令里可以用 <bang> 。

v:completed_item　　v:completed_item　completed_item-variable

v:completed_item
包含 complete-items 的 Dictionary，用于得到 CompleteDone 之后的最近补全。如果补全失败， Dictionary 为空。

v:count count-variable
v:count 最近的普通模式命令使用的计数。在映射前可用于得到计数。只读。
例如:

```
:map _x :<C-U>echo "计数为 " . v:count<CR>
```

注意: <C-U> 是必要的，它删除紧跟在计数之后 ':' 所给出的行范围。
如果有两个计数，如 "3d2w"，它们进行相乘，如同命令行实际发生的那样，等同于 "d6w"。
也用于计算 'formatexpr' 选项。
为了后向兼容，这里也可以用 "count"，除非 scriptversion 为 3 或更高。

v:count1 count1-variable
v:count1 类似于 "v:count"，但没有给出计数时，缺省为 1。

v:ctype ctype-variable
v:ctype 运行环境当前的字符 locale 设置。它使得 Vim 脚本能得到当前的 locale 编码。技术细节: 这就是 LC_CTYPE 的值。如果没有使用 locale，其值为 "C"。
该变量不能直接设置，请使用 :language 命令。
见 multi-lang 。

v:dying dying-variable
v:dying 通常为零。如果捕获到某个 "致命" 的 signal，设为 1。如果同时捕获到多个 signal，其值相应增加。在自动命令里可以用来检查 Vim 是否被异常终止。{仅限于 Unix}
例如:

```
:au VimLeave * if v:dying | echo "\nAAAAaaaarrrgggghhhh!!!\n" | endif
```

备注: 如果 v:dying 为一而同时又捕捉到另一个致命的 signal，不执行 VimLeave 自动命令。

v:echospace echospace-variable
v:echospace 在屏幕行末行上 :echo 消息不引发 hit-enter-prompt 最多可用的屏幕单元格数目。取决于 'showcmd'、'ruler' 和 'columns'。需要检查 'cmdheight' 看看在末行之上是否有全宽的行存在。

v:errmsg errmsg-variable
v:errmsg 最近给出的错误信息。该变量可以设置。
例如:

```
:let v:errmsg = ""
:silent! next
:if v:errmsg != ""
:  ... handle error
```

为了后向兼容，这里也可以用 "errmsg"，除非 scriptversion 为 3 或更高。

v:errors errors-variable assert-return
v:errors assert 函数找到的错误，如 assert_true() 。
是一个字符串列表。

assert 函数在 assert 失败后附加项目。
返回值说明这一点：如果有项目加入 v:errors，返回一，不然返回零。
要清空旧的结果：

```
:let v:errors = []
```

如果用非列表来设置 v:errors，assert 函数会把它变成空列表。

v:event *event-variable*

v:event 包含当前 autocommand 信息的字典。autocommand 结束后该字典清空。请参见 dict-identity 来了解如何取得字典独立的备份。

v:exception *exception-variable*

v:exception 最近捕获且没有完成的例外的值。见 v:throwpoint 和 throw-variables 。
例如：

```
:try
:  throw "oops"
:catch /.*/
:  echo "caught" .. v:exception
:endtry
```

输出："caught oops"。

v:false *false-variable*

v:false 取值为零的数值。用于在 JSON 里填入 "false"。见 json_encode() 。
用于字符串时会返回 "v:false"。

```
echo v:false
v:false
```

这样 eval() 可以把该字符串解析回相同的值。只读。

v:fcs_reason *fcs_reason-variable*

v:fcs_reason 激活 FileChangedShell 事件的原因。
可以在自动命令里用来决定该做什么和/或如何设置 v:fcs_choice。可能的值是：

deleted	文件不再存在
conflict	文件内容、模式或修改时间被改变，而缓冲区同时被修改
changed	文件内容被改变
mode	文件模式被改变
time	文件修改时间被改变

v:fcs_choice *fcs_choice-variable*

v:fcs_choice FileChangedShell 事件激活后该做什么。可以在自动命令里用来告诉 Vim 如何处理涉及的缓冲区：

reload	重新载入缓冲区（如果文件已删除，不能工作）。
ask	询问用户该做什么，就像没有自动命令一样。不过，如果只有修改时间被改变，不做任何事。
<空>	不做任何事。自动命令应该已经处理完毕。

缺省为空。如果使用别的（非法的）值，Vim 的行为就像它为空一样。不会有警告信息。

v:fname_in *fname_in-variable*

v:fname_in　　输入文件名。在计算以下选项时合法：

选项	用于
'charconvert'	要转换的文件
'diffexpr'	原始文件
'patchexpr'	原始文件
'printexpr'	要打印的文件

SwapExists 里设为交换文件名。

v:fname_out　　fname_out-variable

v:fname_out　　输出文件名。只有在计算以下选项时才合法：

选项	用于
'charconvert'	生成的转换完成的文件 (*)
'diffexpr'	diff 的结果
'patchexpr'	产生的补丁文件

(*) 如果用于为写入命令进行转换 (比如，":w file")，等价于 v:fname_in。如果用于为读入命令进行转换 (比如，":e file")，它是一个临时文件名，和 v:fname_in 不同。

v:fname_new　　fname_new-variable

v:fname_new　　文件新版本的名字。只有在计算 'diffexpr' 的时候才有效。

v:fname_diff　　fname_diff-variable

v:fname_diff　　比较结果 (或补丁) 的文件名。只有在计算 'patchexpr' 的时候才有效。

v:folddashes　　folddashes-variable

v:folddashes　　用于 'foldtext'：反映关闭的折叠的折叠级别的连字符。
sandbox 里只读。 fold-foldtext

v:foldlevel　　foldlevel-variable

v:foldlevel　　用于 'foldtext'：关闭的折叠的折叠级别。
sandbox 里只读。 fold-foldtext

v:foldend　　foldend-variable

v:foldend　　用于 'foldtext'：关闭的折叠的最后一行。
sandbox 里只读。 fold-foldtext

v:foldstart　　foldstart-variable

v:foldstart　　用于 'foldtext'：关闭的折叠的第一行。
sandbox 里只读。 fold-foldtext

v:hlsearch　　hlsearch-variable

v:hlsearch　　用于指定搜索高亮是否打开的变量。只有在启动 'hlsearch' 时它的设置才有意义，这需要 +extra_search 。设置该变量为零相当 :nohlsearch 命令，设置为一则相当于

```
let &hlsearch = &hlsearch
```

备注 函数返回时复原其值。 function-search-undo 。

v:insertmode　　insertmode-variable

v:insertmode　　用于 InsertEnter 和 InsertChange 自动命令事件。取值：

i	插入模式
r	替换模式
v	虚拟替换模式

v:key　　v:key　key-variable

v:key　Dictionary　里当前项目的键。只有在　map()　和　filter()　里计算表达式时有效。
只读。

v:lang　　v:lang　lang-variable

v:lang　运行环境当前的消息 locale 设置。它使得 Vim 脚本能得到当前使用的语言。技术细节：这就是 LC_MESSAGES 的值。该值和系统有关。
该变量不能直接设置，请使用　:language　命令。
它和　v:ctype　不同，因为消息可能使用不同于字符编码的语言。见　multi-lang　。

v:lc_time　　v:lc_time　lc_time-variable

v:lc_time　运行环境当前的时间消息 locale 设置。它使得 Vim 脚本能得到当前使用的语言。技术细节：这就是 LC_TIME 的值。
该变量不能直接设置，请使用　:language　命令。见　multi-lang　。

v:lnum　　v:lnum　lnum-variable

v:lnum　'foldexpr'　fold-expr　、'formatexpr' 和 'indentexpr' 表达式中的行号和 'guitablevel' 和 'guitabtooltip' 中的标签页号。只有在计算这些表达式时才合法。在　sandbox　里时只读。

v:mouse_win　　v:mouse_win　mouse_win-variable

v:mouse_win　用　getchar()　得到鼠标点击时所在的窗口号。首个窗口的编号为 1 ，就像　winnr()　那样。如果那时没有鼠标点击，该值为零。

v:mouse_winid　　v:mouse_winid　mouse_winid-variable

v:mouse_winid　用　getchar()　得到鼠标点击时所在的窗口 ID。如果那时没有鼠标点击，该值为零。

v:mouse_lnum　　v:mouse_lnum　mouse_lnum-variable

v:mouse_lnum　用　getchar()　得到鼠标点击时所在的行号。这是文本行号，不是屏幕行号。如果那时没有鼠标点击，该值为零。

v:mouse_col　　v:mouse_col　mouse_col-variable

v:mouse_col　用　getchar()　得到鼠标点击时所在的列号。这是屏幕列号，就像　virtcol()　那样。如果那时没有鼠标点击，该值为零。

v:none　　v:none　none-variable　None

v:none　空字符串。用于在 JSON 里填入空项目。见　json_encode()　。
用于数值时返回零。
用于字符串时会返回 "v:none"。

```
echo v:none
v:none
```

这样 eval() 可以把该字符串解析回相同的值。只读。

v:null　　v:null　null-variable

v:null　空字符串。用于在 JSON 里填入 "null"。见　json_encode()　。
用于数值时返回零。
用于字符串时会返回 "v:null"。

```
echo v:null
v:null
```

这样 eval() 可以把该字符串解析回相同的值。只读。

v:oldfiles　　oldfiles-variable

v:oldfiles　　启动时从 viminfo 文件载入的文件名列表。Vim 记住的位置标记所在的就是这些文件。列表长度的上限由 'viminfo' 选项的 ' 参数定义(缺省是 100)。
如果不用 viminfo，该列表为空。
另见 :oldfiles 和 c_#< 。
此列表可修改，但并不影响之后 viminfo 文件保存什么。同时，如果使用非字符串的值，会有问题。
{仅当编译时加入 +viminfo 特性才有效}

v:option_new

v:option_new　　选项的新值。执行 OptionSet 自动命令时有效。

v:option_old

v:option_old　　选项的旧值。执行 OptionSet 自动命令时有效。取决于用来设置的命令和选项的类型，这可以是旧的局部值，也可以是旧的全局值。

v:option_oldlocal

v:option_oldlocal
　　选项的旧的局部值。执行 OptionSet 自动命令时有效。

v:option_oldglobal

v:option_oldglobal
　　选项的旧的全部值。执行 OptionSet 自动命令时有效。

v:option_type

v:option_type　　设置命令的作用域。执行 OptionSet 自动命令时有效。可能为 "global" 或 "local"

v:option_command

v:option_command
　　用于设置选项的命令。执行 OptionSet 自动命令时有效。

值	选项设置方法
"setlocal"	:setlocal 或 ":let l:xxx"
"setglobal"	:setglobal 或 ":let g:xxx"
"set"	:set 或 :let
"modeline"	modeline

v:operator　　operator-variable

v:operator　　普通模式给出的最近的操作符。除了 <g> 或 <z> 开始的命令是两个字符外，这是单个字符。最好和 v:prevcount 和 v:register 一起使用。常常，先中止操作符等待模式，然后使用操作符，例如:

```
:omap O <Esc>:call MyMotion(v:operator)<CR>
```

直到输入下一个操作符之前，该值保持不变。因此不要期待该值会为空。
:delete 、 :yank 或其它 Ex 命令不改变 v:operator。
只读。

v:prevcount　　prevcount-variable

v:prevcount　　倒数第二次的普通模式命令使用的计数，也就是再上一个命令用的 v:count 的值。可以用来先中止可视模式或操作符等待模式，然后使用计数。

```
:vmap % <Esc>:call MyFilter(v:prevcount)<CR>
```

只读。

v:profiling　　profiling-variable

v:profiling　　通常为零。开始用 ":profile start" 之后设为一。见 profiling 。

v:progname　　progname-variable

v:progname　　包含 Vim 启动时使用的名字 (路径已被去掉)。可以用来为 view 、

evim 等符号链接到 Vim 的名字提供特殊的设置。
只读。

v:progpath　　v:progpath　progpath-variable
包含 Vim 启动时使用的命令，使用的形式如果传递给外壳的话会保证和当前进程运行相同的 Vim 可执行程序（如果 $PATH 没变的话）。可用于通过 --remote-expr 给 Vim 服务器发消息。
要得到完整路径：

```
echo exepath(v:progpath)
```

如果命令使用相对路径，这样就会把相对路径扩展为完整路径，从而可在 :cd 后使用。假定用 "./vim" 启动，返回 "/home/user/path/to/vim/src/vim"。
Linux 和其它系统上总是使用完整路径。
Mac 上可能只是 "vim"，如上所述使用 exepath() 可用来得到完整路径。
MS-Windows 上的可执行文件可能叫做 "vim.exe"，但 v:progpath 中不会带上 ".exe"。
只读。

v:register　　v:register　register-variable
当前的普通模式命令使用的寄存器名字（不管该命令是否使用寄存器），也用于当前执行的普通模式的映射（用于其中的接受寄存器的自定义命令）。
如果没有特殊指定，使用缺省寄存器 '"'。除非 'clipboard' 包含 "unamed" 或 "unamedplus"，此时它为 '*' 或 '+'。
另见 getreg() 和 setreg()

v:scrollstart　　v:scrollstart　scrollstart-variable
指示使屏幕上滚的脚本或函数的字符串。只有在原来为空时才设置，因此只记住第一个原因。如果来自输入的命令，设为 "Unknown"。
可以用来发现你的脚本为什么产生 hit-enter 提示。

v:servername　　v:servername　servername-variable
如果有的话，注册过的 client-server-name 名字。
只读。

v:searchforward　　v:searchforward　searchforward-variable
搜索方向：正向搜索后为 1，反向搜索后为 0。直接设置最近搜索模式会复位此值为正向，见 quote/ 。
注意 从函数返回时该值被复原 function-search-undo 。
可读写。

v:shell_error　　v:shell_error　shell_error-variable
最近一次外壳命令的返回值。如果非零，最近一次外壳命令有错。如果为零，则该命令成功返回。这只有在外壳把错误代码返回给 Vim 的时候才工作。-1 通常用来告知该命令无法执行。只读。
例如：

```
:!mv foo bar
:if v:shell_error
:  echo '不能把 "foo" 换名为 "bar"!'
:endif
```

为了后向兼容，这里也可以用 "shell_error"，除非 scriptversion 为 3 或更高。

v:statusmsg　　v:statusmsg　statusmsg-variable

最近给出的状态消息。可以设置该变量。

v:swapname　　v:swapname　swapname-variable

只有在执行 SwapExists 自动命令时才合法：找到的交换文件名。只读。

v:swapchoice　　v:swapchoice　swapchoice-variable

SwapExists 自动命令可以设置此值，以选择如何处理已有交换文件：

```
'o'     以只读方式打开
'e'     仍然编辑
'r'     恢复
'd'     删除交换文件
'q'     退出
'a'     中止
```

该值应是单个字符的字符串。如果为空，用户会被询问，就像没有 SwapExists 自动命令那样。缺省为空。

v:swapcommand　　v:swapcommand　swapcommand-variable

打开文件后执行的普通模式命令。可以用于 SwapExists 自动命令，用以让另一个 Vim 打开文件并跳转到合适的位置。例如，要跳转到某标签，用的值是 ":tag tagname\r"。":edit +cmd file" 用的值是 ":cmd\r"。

v:t_bool　　v:t_TYPE　v:t_bool　t_bool-variable

Boolean 的类型值。只读。见： type()

v:t_channel　　v:t_channel　t_channel-variable

Channel 的类型值。只读。见： type()

v:t_dict　　v:t_dict　t_dict-variable

Dictionary 的类型值。只读。见： type()

v:t_float　　v:t_float　t_float-variable

Float 的类型值。只读。见： type()

v:t_func　　v:t_func　t_func-variable

Funcref 的类型值。只读。见： type()

v:t_job　　v:t_job　t_job-variable

Job 的类型值。只读。见： type()

v:t_list　　v:t_list　t_list-variable

List 的类型值。只读。见： type()

v:t_none　　v:t_none　t_none-variable

None 的类型值。只读。见： type()

v:t_number　　v:t_number　t_number-variable

Number 的类型值。只读。见： type()

v:t_string　　v:t_string　t_string-variable

String 的类型值。只读。见： type()

v:t_blob　　v:t_blob　t_blob-variable

Blob 的类型值。只读。见： type()

v:termresponse　　v:termresponse　termresponse-variable

使用 t_RV termcap 项目返回的终端的转义序列。Vim 收到 ESC [或者 CSI 开始，然后是 '>' 或 '?'，并以一个 'c' 结束，并且其间只包含数字和 ';' 的转义序列的时候，会设置该值。
如果设置该选项，会激活 TermResponse 自动命令事件，这样你就可以对终端的应答做出反应。

新的 xterm 的应答是: "<Esc>[> Pp ; Pv ; Pc c"。 Pp 是终端类型: 0 代表 vt100，而 1 代表 vt220。 Pv 是补丁号 (因为这是 patch 95 引入的，补丁号应该总是 95 会更高)。Pc 总是零。
{仅当编译时加入 +termresponse 特性才有效}

v:termblinkresp
v:termblinkresp 终端用于 t_RC termcap 项目的转义序列。用于找出终端光标是否闪烁。用于 term_getcursor() 。

v:termstyleresp
v:termstyleresp 终端用于 t_RS termcap 项目的转义序列。用于找出终端光标的形状。用于 term_getcursor() 。

v:termrbgresp
v:termrbgresp 终端用于 t_RB termcap 项目的转义序列。用于找出终端的背景色，见 'background'。

v:termrfgresp
v:termrfgresp 终端用于 t_RF termcap 项目的转义序列。用于找出终端的前景色。

v:termu7resp
v:termu7resp 终端用于 t_u7 termcap 项目的转义序列。用于找出终端如何处理二义性宽度字符，见 'ambiwidth'。

v:testing testing-variable
v:testing 必须在 test_garbagecollect_now() 之前设置。
另外，设置时，在 2 秒内不会显示特定的错误信息 (例如 "'dictionary' option is empty")

v:this_session this_session-variable
v:this_session 最近载入或者保存的会话文件的文件名 :mksession 。可以设置该变量。如果没有保存过会话文件，该变量为空。
为了后向兼容，这里也可以用 "this_session"，除非 scriptversion 为 3 或更高。

v:throwpoint throwpoint-variable
v:throwpoint 最近捕获且未完成的例外的抛出位置。输入的命令不会设置此变量。另见 v:exception 和 throw-variables 。
例如:

```
:try
:  throw "oops"
:catch /.*/
:  echo "Exception from" v:throwpoint
:endtry
```

输出: "Exception from test.vim, line 2"

v:true true-variable
v:true 取值为一的数值。用于在 JSON 里填入 "true"。见 json_encode() 。
用于字符串时会返回 "v:true"。

```
echo v:true
v:true
```

这样 eval() 可以把该字符串解析回相同的值。只读。

v:val　　List 或 Dictionary 当前项目的值。只有在计算 map() 和 filter() 里的表达式时才有效。只读。　　v:val　val-variable

v:version　　Vim 的版本号：主版本号乘以 100 加上副版本号。5.0 版本对应的是 500。5.1 版本则是 501。只读。为了后向兼容，这里也可以用 "version"，除非 scriptversion 为 3 或者更高。
用 has() 可以检查是否包含某补丁，例如：

```
if has("patch-7.4.123")
```

注意 补丁号和版本有关，5.0 和 5.1 版本都有补丁号 123，但完全不同。　　v:version　version-variable

v:versionlong　　类似于 v:version，但末四位数字包含补丁号。8.1 版本带补丁 123 的值是 8010123。可以这样用：

```
if v:versionlong >= 8010123
```

不过，如果包含的补丁列表有空档这样不太完美。比如为了安全原因，有时旧的版本会打上新近的补丁。用 has() 函数可以确定是否包含某补丁。　　v:versionlong　versionlong-variable

v:vim_did_enter　　直到绝大部分的初始化工作做完之前保持为零。在 VimEnter 自动命令刚刚激活之前，设为一。　　v:vim_did_enter　vim_did_enter-variable

v:warningmsg　　最近给出的警告消息。该变量可以设置。　　v:warningmsg　warningmsg-variable

v:windowid　　运行基于 X11 的任何 GUI，或者在终端运行且 Vim 连接到 X 服务器 (-X) 时，给出窗口号。
运行 MS-Windows GUI 时，给出窗口的句柄。
否则该值为零。
注意：Vim 内部的窗口请用 winnr() 或 win_getid()，见 window-ID 。　　v:windowid　windowid-variable

---

4. 内建函数　　functions

function-list 提供了按功能分组的一个函数列表。

(在函数名上使用 CTRL-] 跳转到完整的功能说明。)

用法	结果	描述
abs({expr})	浮点或数值	{expr} 的绝对值
acos({expr})	浮点	{expr} 的反余弦值
add({object}, {item})	列表或 blob	在 {object} 最后附加 {item}
and({expr}, {expr})	数值	按位与
append({lnum}, {text})	数值	在第 {lnum} 行下附加 {text}
appendbufline({expr}, {lnum}, {text})	数值	在缓冲区 {expr} 第 {lnum} 行下附加 {text}

argc([{winid}]) 数值 参数列表的文件数目
argidx() 数值 参数列表的当前索引
arglistid([{winnr} [, {tabnr}]]) 数值 参数列表的 id
argv({nr} [, {winid}]) 字符串 参数列表第 {nr} 个参数
argv([-1, {winid}]) 列表 参数列表
assert_beeps({cmd}) 数值 断言 {cmd} 产生铃声
assert_equal({exp}, {act} [, {msg}])
  数值 断言 {exp} 等于 {act}
assert_exception({error} [, {msg}])
  数值 断言 v:exception 中有 {error}
assert_fails({cmd} [, {error} [, {msg}]])
  数值 断言 {cmd} 失败
assert_false({actual} [, {msg}])
  数值 断言 {actual} 为假
assert_inrange({lower}, {upper}, {actual} [, {msg}])
  数值 断言 {actual} 在指定范围内
assert_match({pat}, {text} [, {msg}])
  数值 断言 {pat} 匹配 {text}
assert_notequal({exp}, {act} [, {msg}])
  数值 断言 {exp} 不等于 {act}
assert_notmatch({pat}, {text} [, {msg}])
  数值 断言 {pat} 不匹配 {text}
assert_report({msg}) 数值 报告一个测试失败
assert_true({actual} [, {msg}]) 数值 断言 {actual} 为真
asin({expr}) 浮点 {expr} 的反正弦值
atan({expr}) 浮点 {expr} 的反正切值
atan2(‘{expr}, {expr}) 浮点 {expr1} / {expr2} 的反正切值
balloon_gettext() 字符串 气泡的当前文本
balloon_show({expr}) 空 在气泡内显示{expr}
balloon_split({msg}) 列表 分割用于的 {msg}
browse({save}, {title}, {initdir}, {default})
  字符串 启动文件请求窗口
browsedir({title}, {initdir}) 字符串 启动目录请求窗口
bufadd({name}) 数值 在缓冲区列表里加入缓冲区
bufexists({expr}) 数值 如果缓冲区 {expr} 存在则为 TRUE
buflisted({expr}) 数值 如果缓冲区 {expr} 在列表内则为 TRUE
bufload({expr}) 数值 如果还未载入，载入缓冲区 {expr}
bufloaded({expr}) 数值 如果缓冲区 {expr} 被载入则为 TRUE
bufname([{expr}]) 字符串 缓冲区 {expr} 的名字
bufnr([{expr} [, {create}]]) 数值 缓冲区 {expr} 的编号
bufwinid({expr}) 数值 缓冲区 {expr} 的窗口 ID
bufwinnr({expr}) 数值 缓冲区 {expr} 的窗口号
byte2line({byte}) 数值 第 {byte} 个字节所在的行号
byteidx({expr}, {nr}) 数值 {expr} 里第 {nr} 个字符的字节位置
byteidxcomp({expr}, {nr}) 数值 {expr} 里第 {nr} 个字符的字节位置
call({func}, {arglist} [, {dict}])
  可变 调用函数 {func}，使用参数 {arglist}
ceil({expr}) 浮点 {expr} 向上取整
ch_canread({handle}) 数值 检查是否有可读的内容
ch_close({handle}) 空 关闭 {handle}
ch_close_in({handle}) 空 关闭 {handle} 的 in 部分
ch_evalexpr({handle}, {expr} [, {options}])
  可变 在 JSON {handle} 上执行 {expr}
ch_evalraw({handle}, {string} [, {options}])

	可变	在原始 {handle} 上执行 {string}
ch_getbufnr({handle}, {what})	数值	获得 {handle}/{what} 的缓冲区号
ch_getjob({channel})	作业	获得 {channel} 的相关作业
ch_info({handle})	字符串	有关通道 {handle} 的信息
ch_log({msg} [, {handle}])	空	在通道日志文件中写入 {msg}
ch_logfile({fname} [, {mode}])	空	开始记录通道活动
ch_open({address} [, {options}])		
	通道	打开到 {address} 的通道
ch_read({handle} [, {options}])	字符串	从 {handle} 读取
ch_readblob({handle} [, {options}])		
	blob	从 {handle} 读取 blob
ch_readraw({handle} [, {options}])		
	字符串	从 {handle} 读取原始格式
ch_sendexpr({handle}, {expr} [, {options}])		
	可变	在 JSON {handle} 上发送 {expr}
ch_sendraw({handle}, {expr} [, {options}])		
	可变	在原始 {handle} 上发送 {expr}
ch_setoptions({handle}, {options})		
	空	设置 {handle} 的选项
ch_status({handle} [, {options}])		
	字符串	通道 {handle} 的状态
changenr()	数值	当前改变号
char2nr({expr} [, {utf8}])	数值	{expr} 里第一个字符串的 ASCII/UTF8 值
chdir({dir})	数值	改变当前目录
cindent({lnum})	数值	第 {lnum} 行的 C 缩进
clearmatches([{win}])	无	清除所有的匹配
col({expr})	数值	光标或位置标记的列号
complete({startcol}, {matches})	无	设置插入模式补全
complete_add({expr})	数值	增加补全匹配
complete_check()	数值	补全时检查输入的键
complete_info([{what}])	字典	获得当前补全信息
confirm({msg} [, {choices} [, {default} [, {type}]]])		
	数值	用户选择的序号
copy({expr})	可变	提供 {expr} 的浅备份
cos({expr})	浮点	{expr} 的余弦值
cosh({expr})	浮点	{expr} 的双曲余弦值
count({comp}, {expr} [, {ic} [, {start}]])		
	数值	计算 {comp} 里有多少个 {expr}
cscope_connection([{num} , {dbpath} [, {prepend}]])		
	数值	检查 cscope 连接是否存在
cursor({lnum}, {col} [, {off}])		
	数值	移动光标到 {lnum}, {col}, {off}
debugbreak({pid})	数值	中断待调试的进程
cursor({list})	数值	移动光标到 {list} 里的位置
deepcopy({expr} [, {noref}])	可变	提供 {expr} 的完整备份
delete({fname} [, {flags}])	数值	删除文件或目录 {fname}
deletebufline({expr}, {first} [, {last}])		
	数值	删除缓冲区 {expr} 的多行
did_filetype()	数值	用过 FileType 自动命令事件则为 TRUE
diff_filler({lnum})	数值	{lnum} 行之上的 diff 填充行数
diff_hlID({lnum}, {col})	数值	{lnum}/{col} 位置的 diff 高亮
empty({expr})	数值	如果 {expr} 为空则为 TRUE
environ()	字典	返回所有环境变量
escape({string}, {chars})	字符串	在 {string} 里用 '\' 转义 {chars}

```
eval({string})                  可变     计算 {string}，返回结果
eventhandler()                  数值     如果在事件处理中则为 TRUE
executable({expr})              数值     如果可执行文件 {expr} 存在则为 1
execute({command})              字符串   执行 {command} 并取得输出结果
exepath({expr})                 字符串   命令 {expr} 的完整路径
exists({expr})                  数值     如果 {expr} 存在则为 TRUE
extend({expr1}, {expr2} [, {expr3}])
                                列表/字典 把 {expr2} 里的项目插入 {expr1}
exp({expr})                     浮点     {expr} 的指数函数值
                                         (译者注: 以 e 为底)
expand({expr} [, {nosuf} [, {list}]])
                                可变     扩展 {expr} 里的特殊关键字
expandcmd({expr})               字符串   像 :edit 那样扩展 {expr}
feedkeys({string} [, {mode}])   数值     给预输入缓冲区加入键序列
filereadable({file})            数值     如果 {file} 是个可读文件则为 TRUE
filewritable({file})            数值     如果 {file} 是个可写文件则为 TRUE
filter({expr1}, {expr2})        列表/字典 删除 {expr1} 里 {expr2} 为 0 的项目
finddir({name} [, {path} [, {count}]])
                                字符串   在 {path} 里寻找目录 {name}
findfile({name} [, {path} [, {count}]])
                                字符串   在 {path} 里寻找文件 {name}
float2nr({expr})                数值     转换浮点数 {expr} 为数值
floor({expr})                   浮点     {expr} 向下取整
fmod({expr1}, {expr2})          浮点     {expr1} / {expr2} 的浮点余数
fnameescape({fname})            字符串   转义 {fname} 中的特殊字符
fnamemodify({fname}, {mods})    字符串   修改文件名
foldclosed({lnum})              数值     {lnum} 所在折叠的首行，如果是关闭的话
foldclosedend({lnum})           数值     {lnum} 所在折叠的末行，如果是关闭的话
foldlevel({lnum})               数值     {lnum} 的折叠级别
foldtext()                      字符串   关闭的折叠显示的行
foldtextresult({lnum})          字符串   {lnum} 所在的关闭的折叠的文本
foreground()                    数值     把 Vim 窗口带到前台
funcref({name} [, {arglist}] [, {dict}])
                                函数引用  函数 {name} 的引用
function({name} [, {arglist}] [, {dict}])
                                函数引用  函数 {name} 的命名引用
garbagecollect([{atexit}])      无       释放内存，打破循环引用
get({list}, {idx} [, {def}])    可变     得到 {list} 或 {def} 的项目 {idx}
get({dict}, {key} [, {def}])    可变     得到 {dict} 或 {def} 的项目 {idx}
get({func}, {what})             可变     得到函数引用/偏函数 {func} 的属性
getbufinfo([{expr}])            列表     缓冲区信息
getbufline({expr}, {lnum} [, {end}])
                                列表     缓冲区 {expr} 第 {lnum} 到 {end} 行
getbufvar({expr}, {varname} [, {def}])
                                可变     缓冲区 {expr} 的变量 {varname}
getchangelist([{expr}])         列表     改变列表项目的列表
getchar([expr])                 数值     让用户输入一个字符
getcharmod()                    数值     最近输入字符的修饰符
getcharsearch()                 字典     最近字符搜索选项
getcmdline()                    字符串   返回当前命令行
getcmdpos()                     数值     返回命令行的光标位置
getcmdtype()                    字符串   返回当前命令行类型
getcmdwintype()                 字符串   返回当前命令行窗口类型
getcompletion({pat}, {type} [, {filtered}])
```

```
                                    列表     命令行补全匹配列表
getcurpos()                         列表     光标位置
getcwd([{winnr} [, {tabnr}]])       字符串   当前工作目录
getenv({name})                      字符串   返回环境变量
getfontname([{name}])               字符串   使用的字体名
getfperm({fname})                   字符串   文件 {fname} 的文件权限
getfsize({fname})                   数值     字节计算的文件 {fname} 大小
getftime({fname})                   数值     文件的最新修改时间
getftype({fname})                   字符串   文件 {fname} 类型的描述
getimstatus()                       数值     如果 IME 处于激活状态则为 TRUE
getjumplist([{winnr} [, {tabnr}]])
                                    列表     跳转列表项目的列表
getline({lnum})                     字符串   当前缓冲区的第 {lnum} 行
getline({lnum}, {end})              列表     当前缓冲区第 {lnum} 到 {end} 行
getloclist({nr} [, {what}])         列表     位置列表项目的列表
getmatches([{win}])                 列表     当前匹配的列表
getmousepos()                       字典     最近已知的鼠标位置
getpid()                            数值     Vim 的进程号
getpos({expr})                      列表     光标、位置标记等的位置
getqflist([{what}])                 列表     quickfix 项目的列表
getreg([{regname} [, 1 [, {list}]]])
                                    字符串/列表      寄存器内容
getregtype([{regname}])             字符串   寄存器类型
gettabinfo([{expr}])                列表     标签页列表
gettabvar({nr}, {varname} [, {def}])
                                    可变     {tabnr} 标签页的 {varname} 变量
gettabwinvar({tabnr}, {winnr}, {name} [, {def}])
                                    可变     {tabnr} 标签页 {winnr} 窗口的 {name}
gettagstack([{nr}])                 字典     获取窗口 {nr} 的标签栈
getwininfo([{winid}])               列表     每个窗口信息的列表
getwinpos([{timeout}])              列表     Vim 窗口以像素计的 X 和 Y 坐标
getwinposx()                        数值     Vim 窗口以像素计的 X 坐标
getwinposy()                        数值     Vim 窗口以像素计的 Y 坐标
getwinvar({nr}, {varname} [, {def}])
                                    可变     窗口 {expr} 的变量 {varname}
glob({expr} [, {nosuf} [, {list} [, {alllinks}]]])
                                    可变     扩展 {expr} 里的文件通配符
glob2regpat({expr})                 字符串   转化 glob 模式为搜索模式
globpath({path}, {expr} [, {nosuf} [, {list} [, {alllinks}]]])
                                    字符串   在 {path} 所有目录下执行 glob({expr})
has({feature})                      数值     如果支持特性 {feature} 则为 TRUE
has_key({dict}, {key})              数值     如果 {dict} 有项目 {key} 则为 TRUE
haslocaldir([{winnr} [, {tabnr}]])
                                    数值     如果当前窗口执行过 :lcd 则为 TRUE
hasmapto({what} [, {mode} [, {abbr}]])
                                    数值     如果 {what} 的映射存在则为 TRUE
histadd({history},{item})           字符串   在历史里增加项目
histdel({history} [, {item}])       字符串   从历史里删除项目
histget({history} [, {index}])      字符串   得到历史的第 {index} 项
histnr({history})                   数值     历史里最高的项目号
hlexists({name})                    数值     如果高亮组 {name} 存在则为 TRUE
hlID({name})                        数值     高亮组 {name} 的语法 ID
hostname()                          字符串   Vim 运行的机器名字
iconv({expr}, {from}, {to})         字符串   转换 {expr} 的编码
```

indent({lnum}) 数值 第 {lnum} 行的缩进
index({object}, {expr} [, {start} [, {ic}]]) 数值 {object} 里出现 {expr} 的项目的索引
input({prompt} [, {text} [, {completion}]]) 字符串 从用户得到输入
inputdialog({prompt} [, {text} [, {completion}]]) 字符串 类似于 input()，但使用 GUI 对话框
inputlist({textlist}) 数值 让用户从选择列表里挑选
inputrestore() 数值 恢复预输入
inputsave() 数值 保存和清除预输入
inputsecret({prompt} [, {text}]) 字符串 类似于 input()，但隐藏文本
insert({object}, {item} [, {idx}]) 列表 在 {object} 里插入 {item} [{idx} 之前]
interrupt() 无 中断脚本执行
invert({expr}) 数值 按位取反
isdirectory({directory}) 数值 如果 {directory} 是目录则为 TRUE
islocked({expr}) 数值 如果 {expr} 被锁住则为 TRUE
isinf({expr}) 数值 决定 {expr} 是否为无限大（正或负）
isnan({expr}) 数值 如果 {expr} 为 NaN 则为 TRUE
items({dict}) 列表 {dict} 里的键-值组对
job_getchannel({job}) 通道 获取 {job} 的通道句柄
job_info([{job}]) 字典 获取 {job} 的信息
job_setoptions({job}, {options}) 无 设置 {job} 选项
job_start({command} [, {options}]) 作业 启动作业
job_status({job}) 字符串 获取 {job} 的状态
job_stop({job} [, {how}]) 数值 停止 {job}
join({list} [, {sep}]) 字符串 连接 {list} 的项目成为一个字符串
js_decode({string}) 可变 解码 JS 风格的 JSON
js_encode({expr}) 字符串 编码 JS 风格的 JSON
json_decode({string}) 可变 解码 JSON
json_encode({expr}) 字符串 编码 JSON
keys({dict}) 列表 {dict} 的所有键
len({expr}) 数值 {expr} 的长度
libcall({lib}, {func}, {arg}) 字符串 调用库 {lib} 的函数 {func}，使用参数 {arg}
libcallnr({lib}, {func}, {arg}) 数值 同上，但返回数值
line({expr} [, {winid}]) 数值 光标所在、末行或者位置标记所在的行号
line2byte({lnum}) 数值 行 {lnum} 的字节位置
lispindent({lnum}) 数值 行 {lnum} 的 Lisp 缩进
list2str({list} [, {utf8}]) 字符串 把 {list} 中的数值转成字符串
listener_add({callback} [, {buf}]) 数值 add a callback to listen to changes
listener_flush([{buf}]) 无 激活监听器回调
listener_remove({id}) 无 删除监听器回调
localtime() 数值 当前时间
log({expr}) 浮点 {expr} 的自然对数（以 e 为底）
log10({expr}) 浮点 {expr} 以 10 为底的对数
luaeval({expr} [, {expr}]) 可变 执行 Lua 表达式
map({expr1}, {expr2}) 列表/字典 {expr1} 的每个项目改变为 {expr2}
maparg({name} [, {mode} [, {abbr} [, {dict}]]]) 字符串/字典 模式 {mode} 的映射 {name} 的右手边
mapcheck({name} [, {mode} [, {abbr}]]) 字符串 检查匹配 {name} 的映射

```
match({expr}, {pat} [, {start} [, {count}]])
                                数值    {expr} 里 {pat} 的匹配位置
matchadd({group}, {pattern} [, {priority} [, {id} [, {dict}]]])
                                数值    用 {group} 高亮 {pattern}
matchaddpos({group}, {pos} [, {priority} [, {id} [, {dict}]]])
                                数值    {group} 的高亮位置
matcharg({nr})                  列表     :match  的参数
matchdelete({id} [, {win}])     数值    删除 {id} 指定的匹配
matchend({expr}, {pat} [, {start} [, {count}]])
                                数值    {expr} 里 {pat} 的结束位置
matchlist({expr}, {pat} [, {start} [, {count}]])
                                列表    {expr} 里 {pat} 的匹配和子匹配
matchstr({expr}, {pat} [, {start} [, {count}]])
                                字符串  {expr} 里 {pat} 的第 {count} 个匹配文
                                        本
matchstrpos({expr}, {pat} [, {start} [, {count}]])
                                列表    {expr} 里 {pat} 的第 {count} 个匹配
max({expr})                     数值    {expr} 的项目的最大值
min({expr})                     数值    {expr} 的项目的最小值
mkdir({name} [, {path} [, {prot}]])
                                数值    建立目录 {name}
mode([expr])                    字符串  当前编辑模式
mzeval({expr})                  可变    计算  MzScheme  表达式
nextnonblank({lnum})            数值    第一个 >= {lnum} 的非空白行的行号
nr2char({expr} [, {utf8}])      字符串  ASCII/UTF8 值为 {expr} 的单个字符
or({expr}, {expr})              数值    按位或
pathshorten({expr})             字符串  缩短路径里的目录名
perleval({expr})                可变    计算  Perl  表达式
popup_atcursor({what}, {options}) 数值  在光标附近创建弹出窗口
popup_beval({what}, {options})  数值    为 'balooneval' 创建弹出窗口
popup_clear()                   无      关闭所有弹出窗口
popup_close({id} [, {result}])  无      关闭弹出窗口 {id}
popup_create({what}, {options}) 数值    创建弹出窗口
popup_dialog({what}, {options}) 数值    创建弹出窗口用作对话框
popup_filter_menu({id}, {key})  数值    菜单弹出窗口的过滤
popup_filter_yesno({id}, {key}) 数值    对话框弹出窗口的过滤
popup_findinfo()                数值    获取信息弹出窗口的窗口 ID
popup_findpreview()             数值    获取预览弹出窗口的窗口 ID
popup_getoptions({id})          字典    获取弹出窗口 {id} 的选项
popup_getpos({id})              字典    获取弹出窗口 {id} 的位置
popup_hide({id})                无      隐藏弹出窗口 {id}
popup_menu({what}, {options})   数值    创建弹出窗口用作菜单
popup_move({id}, {options})     无      设置弹出窗口 {id} 的位置
popup_notification({what}, {options})
                                数值    创建通知弹出窗口
popup_show({id})                无      撤销弹出窗口 {id} 的隐藏
popup_setoptions({id}, {options})
                                无      设置弹出窗口 {id} 的选项
popup_settext({id}, {text})     无      设置弹出窗口 {id} 的文本
pow({x}, {y})                   浮点    {x} 的 {y} 次方
prevnonblank({lnum})            数值    最后一个 <= {lnum} 的非空白行的行号
printf({fmt}, {expr1}...)       字符串  排版文本
prompt_setcallback({buf}, {expr}) 无    设置提示回调函数
prompt_setinterrupt({buf}, {text}) 无   设置提示中断函数
```

```
prompt_setprompt({buf}, {text}) 无       设置提示文本
prop_add({lnum}, {col}, {props})  无     新增文本属性
prop_clear({lnum} [, {lnum-end} [, {props}]])
                                 无       删除所有文本属性
prop_find({props} [, {direction}])
                                 字典     查找文本属性
prop_list({lnum} [, {props})     列表     {lnum} 的文本属性
prop_remove({props} [, {lnum} [, {lnum-end}]])
                                 数值     删除一个文本属性
prop_type_add({name}, {props})   无       定义新的文本属性类型
prop_type_change({name}, {props})
                                 无       改变已有的文本属性类型
prop_type_delete({name} [, {props}])
                                 无       删除文本属性类型
prop_type_get([{name} [, {props}])
                                 字典     获取文本属性类型的值
prop_type_list([{props}])        列表     获取文本属性类型的列表
pum_getpos()                     字典     如果可见，pum 的位置和大小
pumvisible()                     数值     弹出窗口是否可见
pyeval({expr})                   可变     计算 Python 表达式
py3eval({expr})                  可变     计算 python3 表达式
pyxeval({expr})                  可变     计算 python_x 表达式
rand([{expr}])                   数值     得到伪随机数
range({expr} [, {max} [, {stride}]])
                                 列表     从 {expr} 到 {max} 的序列
readdir({dir} [, {expr}])        列表     {dir} 中 {expr} 所选择的文件名
readfile({fname} [, {type} [, {max}]])
                                 列表     得到文件 {fname} 的行列表
reg_executing()                  字符串   得到执行中的寄存器名
reg_recording()                  字符串   得到记录中寄存器名
reltime([{start} [, {end}]])     列表     得到时间值
reltimefloat({time})             浮点数   把时间值转化为浮点数
reltimestr({time})               字符串   把时间值转化为字符串
remote_expr({server}, {string} [, {idvar} [, {timeout}]])
                                 字符串   发送表达式
remote_foreground({server})      数值     把 Vim 服务器带到前台
remote_peek({serverid} [, {retvar}])
                                 数值     检查应答字符串
remote_read({serverid} [, {timeout}])
                                 字符串   读入应答字符串
remote_send({server}, {string} [, {idvar}])
                                 字符串   发送键序列
remote_startserver({name})       空       成为服务器 {name}
remove({list}, {idx} [, {end}]) 可变/列表
                                          从 {list} 里删除项目 {idx}-{end}
remove({blob}, {idx} [, {end}]) 可变/blob
                                          从 {blob} 里删除字节 {idx}-{end}
remove({dict}, {key})            可变     从 {dict} 里删除项目 {key}
rename({from}, {to})             数值     换名（移动）文件，从 {from} 到 {to}
repeat({expr}, {count})          字符串   重复 {expr} {count} 次
resolve({filename})              字符串   解析快捷方式对应的文件名
reverse({list})                  列表     反转 {list}，直接修改 {list}
round({expr})                    浮点     {expr} 四舍五入
rubyeval({expr})                 可变     计算 Ruby 表达式
```

```
screenattr({row}, {col})          数值    当前光标所在的属性
screenchar({row}, {col})          数值    当前光标所在的字符
screenchars({row}, {col})         列表    当前光标所在的字符列表
screencol()                       数值    当前光标列
screenpos({winid}, {lnum}, {col}) 字典    文本字符的屏幕行与列
screenrow()                       数值    当前光标行
screenstring({row}, {col})        字符串  当前光标所在的字符串
search({pattern} [, {flags} [, {stopline} [, {timeout}]]])
                                  数值    搜索 {pattern}
searchdecl({name} [, {global} [, {thisblock}]])
                                  数值    搜索变量声明
searchpair({start}, {middle}, {end} [, {flags} [, {skip} [...]]])
                                  数值    搜索 start/end 对的另一侧
searchpairpos({start}, {middle}, {end} [, {flags} [, {skip} [...]]])
                                  列表    搜索 start/end 队的另一侧
searchpos({pattern} [, {flags} [, {stopline} [, {timeout}]]])
                                  列表    搜索 {pattern}
server2client({clientid}, {string})
                                  数值    发送应答字符串
serverlist()                      字符串  得到可用的服务器列表
setbufline({expr}, {lnum}, {text})
                                  数值    设置缓冲区 {expr} 第 {lnum} 行为
                                          {text}
setbufvar({expr}, {varname}, {val})
                                  空      设置缓冲区 {expr} 的 {varname} 为
                                          {val}
setcharsearch({dict})             字典    从 {dict} 设置字符搜索选项
setcmdpos({pos})                  数值    设置命令行的光标位置
setenv({name}, {val})             无      设置环境变量
setfperm({fname}, {mode})         数值    设置 {fname} 文件权限为 {mode}
setline({lnum}, {line})           数值    设置第 {lnum} 行的内容为 {line}
setloclist({nr}, {list} [, {action} [, {what}]])
                                  数值    用 {list} 修改位置列表
setmatches({list} [, {win}])      数值    还原匹配列表
setpos({expr}, {list})            数值    设置 {expr} 的位置为 {list}
setqflist({list} [, {action} [, {what}]])
                                  数值    用 {list} 修改 quickfix 列表
setreg({n}, {v} [, {opt}])        数值    设置寄存器的值和类型
settabvar({nr}, {varname}, {val}) 无      设置标签页 {nr} 的 {varname} 变量为
                                          {val}
settabwinvar({tabnr}, {winnr}, {varname}, {val})
                                  无      设置标签页 {tabnr} 窗口 {winnr} 的
                                          {varname} 变量为 {val}
settagstack({nr}, {dict} [, {action}])
                                  数值    用 {dict} 修改标签栈
setwinvar({nr}, {varname}, {val}) 空      设置窗口 {expr} 的 {varname} 为 {val}
sha256({string})                  字符串  {string} 的 SHA256 校验码
shellescape({string} [, {special}])
                                  字符串  转义 {string} 以便用作外壳命令的参数
shiftwidth([{col}])               数值    'shiftwidth' 的有效值
sign_define({name} [, {dict}])    数值    定义或更新标号
sign_define({list})               列表    定义或更新一列标号
sign_getdefined([{name}])         列表    得到已定义的标号的列表
sign_getplaced([{expr} [, {dict}]])
```

	列表	得到已放置的标号的列表
sign_jump({id}, {group}, {expr})	数值	跳转到标号
sign_place({id}, {group}, {name}, {expr} [, {dict}])	数值	放置标号
sign_placelist({list})	列表	放置一列标号
sign_undefine([{name}])	数值	撤销标号的定义
sign_undefine({list})	列表	撤销一列标号的定义
sign_unplace({group} [, {dict}])	数值	撤销标号的放置
sign_unplacelist({list})	列表	撤销一列标号的放置
simplify({filename})	字符串	尽可能简化文件名
sin({expr})	浮点	{expr} 的正弦值
sinh({expr})	浮点	{expr} 的双曲正弦值
sort({list} [, {func} [, {dict}]])	列表	排序 {list}，用比较函数 {func}
sound_clear()	无	停止播放所有声音
sound_playevent({name} [, {callback}])	数值	播放事件声音
sound_playfile({path} [, {callback}])	数值	播放声音文件 {path}
sound_stop({id})	无	停止播放声音 {id}
soundfold({word})	字符串	按发音折叠 {word}
spellbadword()	字符串	光标所在的拼写错误的单词
spellsuggest({word} [, {max} [, {capital}]])	列表	拼写建议
split({expr} [, {pat} [, {keepempty}]])	列表	从 {pat} 分割的 {expr} 里构造 List
sqrt({expr})	浮点	{expr} 的平方根
srand([{expr}])	列表	取得 rand() 的种子
state([{what}])	字符串	Vim 的当前状态
str2float({expr})	浮点	转换字符串为浮点数
str2list({expr} [, {utf8}])	列表	把 {expr} 的每个字符转换为 ASCII/UTF8 值
str2nr({expr} [, {base} [, {quoted}]])	数值	把字符串转换为数值
strchars({expr} [, {skipcc}])	数值	{expr} 字符串的字符长度
strcharpart({str}, {start} [, {len}])	字符串	{str} 从 {start} 开始的 {len} 个字符
strdisplaywidth({expr} [, {col}])	数值	{expr} 字符串的显示长度
strftime({format} [, {time}])	字符串	使用指定格式排版时间
strgetchar({str}, {index})	数值	从 {str} 取得字符 {index}
stridx({haystack}, {needle} [, {start}])	数值	{haystack} 里 {needle} 的位置
string({expr})	字符串	{expr} 值得字符串表示
strlen({expr})	数值	字符串 {expr} 的长度
strpart({str}, {start} [, {len}])	字符串	{str} 从 {start} 开始的 {len} 个字节
strridx({haystack}, {needle} [, {start}])	数值	{haystack} 里最后一个 {needle} 的位置
strptime({format}, {timestring})	数值	把 {timestring} 转换为 unix 时间戳
strtrans({expr})	字符串	翻译字符串，使之可以显示
swapinfo({fname})	字典	关于交换文件 {fname} 的信息

```
swapname({expr})                字符串  缓冲区 {expr} 的交换文件
strwidth({expr})                数值    {expr} 字符串的显示单元长度
submatch({nr} [, {list}])       字符串/列表
                                        ":s" 或 substitute() 的特定匹配
substitute({expr}, {pat}, {sub}, {flags})
                                字符串  {expr} 里的所有 {pat} 被 {sub} 替代
synID({lnum}, {col}, {trans})   数值    {lnum} 行 {col} 列所在的语法 ID
synIDattr({synID}, {what} [, {mode}])
                                字符串  syntax ID {synID} 的 {what} 属性
synIDtrans({synID})             数值    {synID} 经过翻译的语法 ID
synconcealed({lnum}, {col})     列表    关于隐藏的信息
synstack({lnum}, {col})         列表    {lnum} 行 {col} 列所在的语法 ID 堆栈
system({expr} [, {input}])      字符串  外壳命令/过滤 {expr} 的输出
systemlist({expr} [, {input}])  列表    外壳命令/过滤 {expr} 的输出
tabpagebuflist([{arg}])         列表    标签页里的缓冲区号列表
tabpagenr([{arg}])              数值    当前或最后标签页的编号
tabpagewinnr({tabarg} [, {arg}]) 数值   标签页里当前窗口的编号
taglist({expr} [, {filename}])  列表    匹配 {expr} 的标签列表
tagfiles()                      列表    使用的标签文件
tan({expr})                     浮点    {expr} 的正切值
tanh({expr})                    浮点    {expr} 的双曲正切值
tempname()                      字符串  临时文件的文件名
term_dumpdiff({filename}, {filename} [, {options}])
                                数值    显示两份截图的差异
term_dumpload({filename} [, {options}])
                                数值    显示一份屏幕截图
term_dumpwrite({buf}, {filename} [, {options}])
                                无      为终端窗口的内容进行截图
term_getaltscreen({buf})        数值    取得备用屏幕标志位
term_getansicolors({buf})       列表    取得 GUI 色彩模式的 ANSI 调色板
term_getattr({attr}, {what})    数值    取得 {what} 属性的值
term_getcursor({buf})           列表    取得终端光标位置
term_getjob({buf})              作业    取得终端关联的作业
term_getline({buf}, {row})      字符串  从终端取得一行文本行
term_getscrolled({buf})         数值    取得终端的滚动数目
term_getsize({buf})             列表    取得终端的大小
term_getstatus({buf})           字符串  取得终端的状态
term_gettitle({buf})            字符串  取得终端的标题
term_gettty({buf}, [{input}])   字符串  取得终端的 tty 名
term_list()                     列表    取得终端缓冲区的列表
term_scrape({buf}, {row})       列表    取得终端屏幕的某行
term_sendkeys({buf}, {keys})    无      给终端发送键序列
term_setapi({buf}, {expr})      无      设置 terminal-api 函数名前缀
term_setansicolors({buf}, {colors})
                                无      设置 GUI 色彩模式的 ANSI 调色板
term_setkill({buf}, {how})      无      设置停止终端作业的信号值
term_setrestore({buf}, {command}) 无    设置恢复终端的命令
term_setsize({buf}, {rows}, {cols})
                                无      设置终端的大小
term_start({cmd} [, {options}]) 数值    打开终端窗口并执行作业
term_wait({buf} [, {time}])     数值    等待屏幕刷新
test_alloc_fail({id}, {countdown}, {repeat})
                                空      使内存分配失败
test_autochdir()                空      系统启动中启动 'autochdir'
```

test_feedinput({string})	空	给输入缓冲区增加键序列
test_garbagecollect_now()	空	为测试用，立即释放内存
test_garbagecollect_soon()	空	为测试用，尽快释放内存
test_getvalue({string})	可变	得到内部变量的值
test_ignore_error({expr})	空	忽略特定的错误
test_null_blob()	blob	用作测试的空值
test_null_channel()	通道	用作测试的空值
test_null_dict()	字典	用作测试的空值
test_null_job()	作业	用作测试的空值
test_null_list()	列表	用作测试的空值
test_null_partial()	函数引用	用作测试的空值
test_null_string()	字符串	用作测试的空值
test_option_not_set({name})	无	复位指示选项已设置的标志位
test_override({expr}, {val})	空	Vim 内部覆盖用作测试
test_refcount({expr})	数值	得到 {expr} 的引用计数
test_scrollbar({which}, {value}, {dragging})	无	为测试用，在 GUI 内滚动
test_setmouse({row}, {col})	无	为测试用，设置鼠标位置
test_settime({expr})	空	用作测试的当前时间
timer_info([{id}])	列表	定时器信息
timer_pause({id}, {pause})	空	暂停或继续定时器
timer_start({time}, {callback} [, {options}])	数值	新建定时器
timer_stop({timer})	空	停止定时器
timer_stopall()	空	停止所有定时器
tolower({expr})	字符串	字符串 {expr} 变为小写
toupper({expr})	字符串	字符串 {expr} 变为大写
tr({src}, {fromstr}, {tostr})	字符串	把 {src} 里的 {fromstr} 字符翻译为 {tostr} 字符
trim({text} [, {mask}])	字符串	从 {text} 中删除 {mask} 中的字符
trunc({expr})	浮点	浮点数 {expr} 截断小数点
type({name})	数值	变量 {name} 的类型
undofile({name})	字符串	对应 {name} 的撤销文件名
undotree()	列表	撤销文件树
uniq({list} [, {func} [, {dict}]])	列表	从列表中删除相邻的重复项
values({dict})	列表	{dict} 的所有值
virtcol({expr})	数值	光标或位置标记的屏幕列
visualmode([expr])	字符串	最近使用的可视模式
wildmenumode()	数值	'wildmenu' 模式是否激活
win_execute({id}, {command} [, {silent}])	字符串	在窗口 {id} 执行 {command}
win_findbuf({bufnr})	列表	寻找包含 {bufnr} 的窗口
win_getid([{win} [, {tab}]])	数值	得到 {tab} 中 {win} 的窗口 ID
win_gotoid({expr})	数值	转到 ID 为 {expr} 的窗口
win_id2tabwin({expr})	列表	从窗口 ID 获得标签页与窗口
win_id2win({expr})	数值	从窗口 ID 获得窗口号
win_screenpos({nr})	列表	得到窗口 {nr} 的屏幕位置
win_splitmove({nr}, {target} [, {options}])	数值	移动窗口 {nr} 成为 {target} 的分割窗口
winbufnr({nr})	数值	窗口 {nr} 的缓冲区号
wincol()	数值	光标所在的窗口列
winheight({nr})	数值	窗口 {nr} 的高度
winlayout([{tabnr}])	列表	标签页 {tabnr} 的窗口布局

winline()	数值	光标所在的窗口行
winnr([{expr}])	数值	当前窗口的编号
winrestcmd()	字符串	返回恢复窗口大小的命令
winrestview({dict})	无	恢复当前窗口的视图
winsaveview()	字典	保存当前窗口的视图
winwidth({nr})	数值	窗口 {nr} 的宽度
wordcount()	字典	字节/字符/单词统计
writefile({object}, {fname} [, {flags}])	数值	把行的 Blob 或 List 写到文件 {fname}
xor({expr}, {expr})	数值	按位异或

abs({expr}) abs()

返回 {expr} 的绝对值。如果 {expr} 计算结果为浮点数，abs() 返回浮点数。如果 {expr} 可以转换为数值，abs() 返回数值。否则报错并返回 -1。

示例：

```
echo abs(1.456)
1.456
echo abs(-5.456)
5.456
echo abs(-4)
4
```

也可用作 method：

```
Compute()->abs()
```

{仅当编译时加入 +float 特性才有效}

acos({expr}) acos()

返回以弧度表示的 {expr} 的反余弦值，返回值为 [0, pi] 区间内的浮点数。

{expr} 的计算结果必须是 [-1, 1] 区间内的浮点数或数值。

示例：

```
:echo acos(0)
1.570796
:echo acos(-0.5)
2.094395
```

也可用作 method：

```
Compute()->acos()
```

{仅当编译时加入 +float 特性才有效}

add({object}, {expr}) add()

在 List 或 Blob {object} 最后附加项目 {expr}。返回新产生的 List 或 Blob。例如：

```
:let alist = add([1, 2, 3], item)
:call add(mylist, "woodstock")
```

注意 如果 {expr} 是 List，它被作为单个项目附加进去。extend() 可以用来连接 List。

如果 {object} 是 Blob，{expr} 必须是个数值。

insert()　可以用来把一个项目加到其它的位置上。

也可用作 method :

```
mylist->add(val1)->add(val2)
```

and({expr}, {expr})　and()

对两个参数进行按位与。参数须转换为数值。列表、字典或浮点数参数会报错。

示例:

```
:let flag = and(bits, 0x80)
```

也可用作 method :

```
:let flag = bits->and(0x80)
```

append({lnum}, {text})　append()

当 {text} 为 List : 把每个 List 项目作为文本行，附加到当前缓冲区第 {lnum} 行之下。

否则，把 {text} 作为单个文本行，附加于当前缓冲区第 {lnum} 行之下。

{lnum} 可以为零，用于在第一行前插入一行。如果失败 ({lnum} 越界)，返回 1，成功则返回 0。例如:

```
:let failed = append(line('$'), "# THE END")
:let failed = append(0, ["Chapter 1", "the beginning"])
```

也可用作列表的 method :

```
mylist->append(lnum)
```

appendbufline({expr}, {lnum}, {text})　appendbufline()

类似于 append() ，但在缓冲区 {expr} 中附加文本。

此函数只能用于已载入的缓冲区。有必要的话先调用 bufload() 。

{expr} 的用法可见 bufname() 。

{lnum} 的用法和 append() 类似。注意 使用 line() 会用到当前缓冲区而不是正在进行附加的那个。

"$" 用来附加于缓冲区的末尾。

成功时返回 0，失败则返回 1。

如果 {expr} 不是合法的缓冲区或 {lnum} 不合法，报错。例如:

```
:let failed = appendbufline(13, 0, "# THE START")
```

也可用作列表的 method :

```
mylist->appendbufline(buf, lnum)
```

argc([{winid}])　argc()

返回参数列表的文件数目。见 arglist 。

如果 {winid} 未给出，使用当前窗口的参数列表。

如果 {winid} 是 -1，使用全局参数列表。

否则，{winid} 指定参数列表所在的窗口: 窗口号或窗口 ID 均可。

如果 {winid} 参数不合法，返回 -1。

argidx()
argidx()
返回参数列表的当前索引。0 是第一个文件。argc() - 1 是最后一个。见 arglist 。

arglistid()
arglistid([{winnr} [, {tabnr}]])
返回参数列表 ID，这是一个用来标识正在使用的参数列表的数值。零代表全局参数列表。见 arglist 。
如果参数非法，返回 -1。

无参数时，使用当前窗口。
只提供 {winnr} 时，使用当前标签页的指定窗口。
同时提供 {winnr} 和 {tabnr} 时，使用指定标签页的指定窗口。
{winnr} 可以是窗口号或 window-ID 。

argv()
argv([{nr} [, {winid}])
返回参数列表第 {nr} 个参数。见 arglist 。"argv(0)" 是第一个参数。例如:

```
:let i = 0
:while i < argc()
:  let f = escape(fnameescape(argv(i)), '. ')
:  exe 'amenu Arg.' . f . ' :e ' . f . '<CR>'
:  let i = i + 1
:endwhile
```

如果没有 {nr} 参数或 {nr} 为 -1，返回完整的 {arglist} 的 List 。

{winid} 参数指定窗口 ID，见 argc() 。
关于 Vim 的命令行参数，可见 v:argv 。

assert_ 函数文档在这里: assert-functions-details

asin({expr})
asin()
返回以弧度表示的 {expr} 的反正弦值，返回值为 [-pi/2, pi/2] 区间内的浮点数。
{expr} 的计算结果必须是 [-1, 1] 区间内的浮点数或数值。
示例:

```
:echo asin(0.8)
0.927295
:echo asin(-0.5)
-0.523599
```

也可用作 method :

```
Compute()->asin()
```

{仅当编译时加入 +float 特性才有效}

atan({expr})
atan()

返回 {expr} 反正切的主值，返回值为 [-pi/2, +pi/2] 区间内的弧度值浮点数。
{expr} 计算结果必须为浮点数或数值。
示例：

```
:echo atan(100)
1.560797
:echo atan(-4.01)
-1.326405
```

也可用作 method：

```
Compute()->atan()
```

{仅当编译时加入 +float 特性才有效}

atan2({expr1}, {expr2}) atan2()
返回 {expr1} / {expr2} 的反正切值，以弧度计算，返回值为 [-pi, +pi] 区间内的浮点数。
{expr1} 和 {expr2} 计算结果必须为浮点数或数值。
示例：

```
:echo atan2(-1, 1)
-0.785398
:echo atan2(1, -1)
2.356194
```

也可用作 method：

```
Compute()->atan(1) (译者注：原文如此)
```

{仅当编译时加入 +float 特性才有效}

balloon_gettext() balloon_gettext()
返回气泡的当前文本。只用于字符串，不用于列表。

balloon_show({expr}) balloon_show()
在气泡中显示 {expr}。GUI 上 {expr} 用作字符串。而终端上 {expr} 可以是多行的列表。如果 {expr} 不是列表，会用 balloon_split() 进行分割。
如果 {expr} 为空，删除任何已存在的气泡。

示例：

```
func GetBalloonContent()
   " ... 启动内容的获取过程
   return ''
endfunc
set balloonexpr=GetBalloonContent()

func BalloonCallback(result)
  call balloon_show(a:result)
endfunc
```

也可用作 method：

```
GetText()->balloon_show()
```

期待的用法是从 'balloonexpr' 启动气泡内容的获取过程。它应调用异步方法，然后在其回调中执行 balloon_show()。'balloonexpr' 自身可返回空字符串或任何占位符。

如果不能显示气泡，不做任何事，不会报错。
{仅当编译时加入 +balloon_eval 或 balloon_eval_term 特性才有效}

balloon_split({msg}) balloon_split()
把 {msg} 分割为用于气泡中显示的多行。分割依据的是当前窗口大小，并进行优化以显示调试器的输出。
返回分割后行的 List 。
也可用作 method :

```
GetText()->balloon_split()->balloon_show()
```

{仅当编译时加入 balloon_eval_term 特性才有效}

browse() browse()
browse({save}, {title}, {initdir}, {default})
启动文件请求窗口。只有在 "has("browse")" 返回 TRUE 时（只有在一些 GUI 版本里）才可以。
输入的字段包括：

{save}	TRUE 时，选择要写入的文件
{title}	请求窗口的标题
{initdir}	开始浏览的目录
{default}	缺省文件名

如果按了 "Cancel" 按钮、出错、或者无法浏览，返回空字符串。

browsedir() browsedir()
browsedir({title}, {initdir})
启动目录请求窗口。只有在 "has("browse")" 返回 TRUE 时（只有在一些 GUI 版本里）才能工作。
有的系统上不支持目录浏览器，这时使用文件浏览器。此时：选择要用的目录里的文件。
输入的字段包括：

{title}	请求窗口的标题
{initdir}	开始浏览的目录

如果按了 "Cancel" 按钮、出错、或者无法浏览，返回空字符串。

bufadd({name}) bufadd()
在缓冲区列表中加入名为 {name} 的缓冲区。
如果对应文件 {name} 的缓冲区已经存在，返回该缓冲区号。否则返回新建缓冲区的号。{name} 为空串时，总是建立新缓冲区。
该缓冲区不置位 'buflisted' 也还未载入。要在缓冲区中增加文件：

```
let bufnr = bufadd('someName')
call bufload(bufnr)
call setbufline(bufnr, 1, ['some', 'text'])
```

也可用作 method :

```
let bufnr = 'somename'->bufadd()
```

bufexists({expr}) bufexists()
返回数值，如果名为 {expr} 的缓冲区存在的话，返回 TRUE 。
{expr} 参数是数值时，指定缓冲区号。数值零指定当前窗口的轮换缓冲区。

{expr} 参数是字符串时，必须完全匹配指定缓冲区名。该名可以是：
- 相对于当前目录。

- 完整路径。
- 'buftype' 设为 "nofile" 的缓冲区名。
- URL 名。
列表外缓冲区也会被找到。
注意 帮助文件在 :buffers 里列出的是它们的短名字。但 bufexists() 需要它们的长名字才能找到它们。
bufexists() 可能报告缓冲区存在，但要使其名字可用于 :buffer 命令，可能需用到 expand() 。尤其 MS-Windows 8.3 名字可能形如 "c:\DOCUME~1"。
使用 "bufexists(0)" 可以测试是否存在轮换文件名。

也可用作 method :
```
let exists = 'somename'->bufexists()
```

已废弃的名字: buffer_exists()。 buffer_exists()

buflisted({expr}) buflisted()
返回数值，如果名为 {expr} 的缓冲区在列表内的话 (置位了 'buflisted' 选项)，返回 TRUE 。
{expr} 参数用法同 bufexists() 。

也可用作 method :
```
let listed = 'somename'->buflisted()
```

bufload({expr}) bufload()
确保缓冲区 {expr} 已载入。如果缓冲区名指向已存在的文件，读入该文件。否则缓冲区会为空。如果缓冲区已载入，则没有改变。
如果缓冲区的文件已有交换文件，不会弹出对话框，还是会载入缓冲区。
{expr} 参数的用法类似于 bufexists() 。

也可用作 method :
```
eval 'somename'->bufload()
```

bufloaded({expr}) bufloaded()
返回数值，如果名为 {expr} 的缓冲区存在且已载入的话 (在窗口显示，或者至少被隐藏)，返回 TRUE 。
{expr} 参数用法同 bufexists() 。

也可用作 method :
```
let loaded = 'somename'->bufloaded()
```

bufname([{expr}]) bufname()
返回缓冲区的名字，如同 ":ls" 命令显示的那样。
如果 {expr} 省略使用当前缓冲区。
如果 {expr} 参数是数值，指定缓冲区号。数值零代表当前窗口的轮换缓冲区。
如果 {expr} 参数是字符串，它用作 file-pattern 来匹配缓冲区名字。这里总假设置位 'magic' 而 'cpoptions' 为空。如果有超过一个匹配，返回空字符串。
"" 或 "%" 可用来指定当前缓冲区，"#" 指定轮换缓冲区。
完整的匹配优先，如果没有，也接受在缓冲区名的开始，结束和中间的匹配。如果你只愿意接受完整的匹配，在模式的开始放上 "^"，在结尾

放上 "$"。
先查找列表内缓冲区。如果列出缓冲区有唯一的匹配，返回之。不然，再查找列表外的缓冲区。
如果 {expr} 是字符串，但你想用作缓冲区号，给它加零可以强制转化为数值型：

```
:echo bufname("3" + 0)
```

也可用作 method ：

```
echo bufnr->bufname()
```

如果缓冲区不存在，或者没有名字，返回空字符串。

```
bufname("#")            轮换缓冲区名
bufname(3)              缓冲区 3 的名字
bufname("%")            当前缓冲区名
bufname("file2")        匹配 "file2" 的缓冲区名。
```

buffer_name()
已废弃的名字：buffer_name()。

bufnr()

bufnr([{expr} [, {create}]])
返回缓冲区的编号，如同 ":ls" 命令显示的那样。关于 {expr} 的使用，见上 bufname() 。
如果不存在符合的缓冲区，返回 -1。或者，如果提供了 {create} 参数而且非零，建立一个新的列表外缓冲区，并返回其编号。例如：

```
let newbuf = bufnr('Scratch001', 1)
```

这里空名会使用当前缓冲区。要创建新缓冲区可用 bufadd() 带空名。

bufnr("$") 是最后一个缓冲区：

```
:let last_buffer = bufnr("$")
```

返回数值，即最大的已有的缓冲区的编号。注意 较小的编号不一定都对应存在的缓冲区，因为 ":bwipeout" 可能永久地删除了部分的缓冲区。用 bufexists() 可以测试缓冲区是否存在。

也可用作 method ：

```
echo bufref->bufnr()
```

已废弃的名字：buffer_number()。 buffer_number()
last_buffer_nr()
bufnr("$") 已废弃的名字：last_buffer_nr()。

bufwinid({expr}) bufwinid()
返回数值，带缓冲区 {expr} 相关的第一个窗口的 window-ID 。{expr} 的用法见上述 bufname() 。 如果缓冲区 {expr} 不存在或没有对应窗口，返回 -1。示例：

```
echo "A window containing buffer 1 is " . (bufwinid(1))
```

只处理当前标签页。

也可用作 method ：

```
FindBuffer()->bufwinid()
```

bufwinnr({expr}) bufwinnr()
和 bufwinid() 类似，但返回窗口号而不是 window-ID 。

如果缓冲区 {expr} 不存在或者没有对应的窗口，返回 -1。例如：

```
echo "包含缓冲区 1 的窗口是 " . (bufwinnr(1))
```

该编号可用于 CTRL-W_w 和 ":wincmd w" :wincmd 。

也可用作 method ：

```
FindBuffer()->bufwinnr()
```

byte2line({byte}) byte2line()

返回当前缓冲区第 {byte} 个字节所在的行号。取决于当前缓冲区的 'fileformat' 选项，这可以包括不同的换行符。第一个字符的字节编号为 1。
另见 line2byte() 、 go 和 :goto 。

也可用作 method ：

```
GetOffset()->byte2line()
```

{仅当编译时加入 +byte_offset 特性才有效}

byteidx({expr}, {nr}) byteidx()

返回字符串 {expr} 里第 {nr} 个字符的字节位置。零代表第一个字符，此时返回零。
该函数只对多字节字符有用，不然返回值总是等于 {nr}。
组合字符不单独计数，其字节长度加到其前导的基础字符上。要将组合字符分别计数，见下 byteidxcomp() 。例如：

```
echo matchstr(str, ".", byteidx(str, 3))
```

显示第四个字符。另一个方法也能达到相同的效果：

```
let s = strpart(str, byteidx(str, 3))
echo strpart(s, 0, byteidx(s, 1))
```

另见 strgetchar() 和 strcharpart() 。

如果字符数不足 {nr}，返回 -1。
如果刚好 {nr} 个字符，返回字符串的字节长度。

也可用作 method ：

```
GetName()->byteidx(idx)
```

byteidxcomp({expr}, {nr}) byteidxcomp()

类似于 byteidx()，但组合字符作为一个单独的字符计算。示例：

```
let s = 'e' . nr2char(0x301)
echo byteidx(s, 1)
echo byteidxcomp(s, 1)
echo byteidxcomp(s, 2)
```

第一个和第三个 echo 返回 3 ('e' 加上组合字符是 3 个字节)，第二个 echo 返回 1 ('e' 是一个字节)。
只有当 'encoding' 设为某种 Unicode 编码时，才会和 byteidx() 有所不同。

也可用作 method ：

```
GetName()->byteidxcomp(idx)
```

call({func}, {arglist} [, {dict}]) call() E699

调用函数 {func}，使用 List {arglist} 项目作为参数。

{func} 可以是 Funcref 或函数名。
a:firstline 和 a:lastline 设为当前行。
返回调用的函数的结果。
{dict} 用于为函数指定 "dict" 属性。其目的是设置局部变量 "self"。 Dictionary-function

也可用作 method :

```
GetFunc()->call([arg, arg], dict)
```

ceil({expr}) ceil()

返回浮点数，即大于等于 {expr} 的最小整数（向上取整）。
{expr} 的计算结果必须是浮点数或数值。
示例：

```
echo ceil(1.456)
2.0
echo ceil(-5.456)
-5.0
echo ceil(4.0)
4.0
```

也可用作 method :

```
Compute()->ceil()
```

{仅当编译时加入 +float 特性才有效}

ch_ 函数文档在这里： channel-functions-details

changenr() changenr()

返回最近改变的编号。这和 :undolist 显示的编号相同，可用于 :undo 命令。
如果发生了改变，返回的是此改变的编号。在重做以后，返回的是重做的改变的编号。在撤销以后，返回撤销的改变的编号减一。

char2nr({expr} [, {utf8}]) char2nr()

返回 {expr} 第一个字符的数值结果。例如：

```
char2nr(" ")        返回 32
char2nr("ABC")      返回 65
```

如果 {utf8} 省略或为零，使用当前的 'encoding'。比如对 "utf-8" 来说：

```
char2nr("")         返回 225
char2nr(""[0])      返回 195
```

如果 {utf8} 为 1，则总视为 utf-8 字符。
组合字符是一个单独的字符。
nr2char() 是它的逆操作。
要把字符串转换为字符数值的列表：

```
let str = "ABC"
let list = map(split(str, '\zs'), {_, val -> char2nr(val)})
```

结果是：[65, 66, 67]

也可用作 method :

```
GetChar()->char2nr()
```

chdir({dir}) chdir()

切换当前工作目录为 {dir}。目录改变的作用域取决于当前窗口的目

录：

- 如果当然窗口有局部于窗口的目录（ :lcd ），则改变局部于窗口的目录。
- 否则，如果当前标签页有局部目录（ :tcd ），则改变局部于标签页的目录。
- 否则，返回全局目录。

如果成功，返回上次的工作目录。把它传给另一个 chdir() 会恢复上次目录。
如果失败，返回空串。

示例：

```
let save_dir = chdir(newdir)
if save_dir != ""
   " ... 做点事
   call chdir(save_dir)
endif
```

也可用作 method ：

```
GetDir()->chdir()
```

cindent({lnum}) cindent()

得到第 {lnum} 行根据 C 缩进规则应有的缩进距离，见 'cindent'。
缩进的计算以空格计，因而和 'tabstop' 的值是有关系的。{lnum} 的使用方式和 getline() 相同。
如果 {lnum} 非法或者 Vim 编译时不带 +cindent 特性，返回 -1。
见 C-indenting 。

也可用作 method ：

```
GetLnum()->cindent()
```

clearmatches([{win}]) clearmatches()

清除之前 matchadd() 和 :match 命令为当前窗口定义的匹配。
如果给出 {win}，使用指定窗口号或窗口 ID 的窗口而不是当前窗口。

也可用作 method ：

```
GetWin()->clearmatches()
```

col() col({expr})

返回数值，即 {expr} 给定的列位置的字节索引。可接受的位置是：

.	光标位置
$	光标行的行尾（返回光标行的字节数加 1）
'x	位置标记 x 的位置（如果该位置标记没有设置，返回 0）
v	可视模式下：可视区域的开始位置（光标是结束位置）。如果不在可视模式下，返回当前光标位置。和 '< 不同的是，会被立即更新。

另外，{expr} 可以是 [lnum, col]: 包含行号和列号的 List 。常用于指定列号为 "$" 以得到特定行的末列列号。如果 "lnum" 或 "col" 超出范围，col() 返回零。
要得到行号用 line() 。两者都要用 getpos() 。
要得到屏幕列的位置，用 virtcol() 。
注意 只能使用当前文件的位置标记。
例如：

```
col(".")                光标所在列
col("$")                光标行的长度加 1
```

```
        col("'t")               位置标记 t 的列号
        col("'" . markname)     等于 markname 的位置标记的列号
```

第一列为 1。0 用来返回错误。
大写位置标记的对应列可能在另一个缓冲区中。
如果激活了 'virtualedit' 并且如果光标在行尾之后的话，计算光标位置得到的列号比实际的多 1。可用来在插入模式得到列数：

```
        :imap <F2> <C-O>:let save_ve = &ve<CR>
                \<C-O>:set ve=all<CR>
                \<C-O>:echo col(".") . "\n" <Bar>
                \let &ve = save_ve<CR>
```

也可用作 method：

```
        GetPos()->col()
```

complete({startcol}, {matches}) complete() E785

设置插入模式补全的匹配。
只能用于插入模式。需要使用 CTRL-R = 的映射（见 i_CTRL-R）。不能在 CTRL-O 之后或者在表达式映射里使用。
{startcol} 是行内待补全文本开始的字节偏移。直到光标为止的文本就是原始的文本，它将要被匹配所替代。用 col('.') 会得到空串。而 "col('.') - 1" 将用匹配替换单个字符。
{matches} 必须是 List。每个 List 项目是一个匹配。
complete-items 说明可能的项目类型。
注意 调用此函数后，你不应插入任何使补全停止的内容。
用 CTRL-N 和 CTRL-P 选择匹配，就像普通的插入模式补全那样。如果指定，会出现弹出菜单，见 ins-completion-menu。
示例：

```
inoremap <F5> <C-R>=ListMonths()<CR>

func! ListMonths()
  call complete(col('.'), ['January', 'February', 'March',
        \ 'April', 'May', 'June', 'July', 'August', 'September',
        \ 'October', 'November', 'December'])
  return ''
endfunc
```

此例并不很有用，但可以说明功能。注意这里返回空串，以免插入零。

也可用作 method，基是作为第二个参数传递的：

```
        GetMatches()->complete(col('.'))
```

complete_add({expr}) complete_add()

把 {expr} 加到匹配的列表里。只能用于 'completefunc' 选项指定的函数里。
返回 0 代表失败（空字符串或者内存不足），1 代表加入了匹配，2 代表匹配已经在列表里。
complete-functions 解释 {expr}。它和 'omnifunc' 应该返回的列表中单个项目相同。

也可用作 method：

```
        GetMoreMatches()->complete_add()
```

complete_info()

complete_info([{what}])

返回关于插入模式补全信息的字典。见 ins-completion 。
项目是:

mode	当前补全模式名的字符串。 可用值见 complete_info_mode 。
pum_visible	如果弹出菜单可见则为 TRUE 。 见 pumvisible() 。
items	补全匹配的列表。每个项目是包含项目 "word"、"abbr"、"menu"、"kind"、"info" 和 "user_data" 的字典。 见 complete-items 。
selected	选中项目的索引。首个索引为零。如果没有选中的项目则索引为 -1 (只显示输入的文本)
inserted	插入的字符串。[尚未实现]

complete_info_mode

mode 值是:

""	不在补全模式中
"keyword"	关键字补全 i_CTRL-X_CTRL-N
"ctrl_x"	则按了 CTRL-X i_CTRL-X
"whole_line"	整行 i_CTRL-X_CTRL-L
"files"	文件名 i_CTRL-X_CTRL-F
"tags"	标签 i_CTRL-X_CTRL-]
"path_defines"	定义补全 i_CTRL-X_CTRL-D
"path_patterns"	头文件补全 i_CTRL-X_CTRL-I
"dictionary"	字典 i_CTRL-X_CTRL-K
"thesaurus"	同义词词典 i_CTRL-X_CTRL-T
"cmdline"	Vim 命令行 i_CTRL-X_CTRL-V
"function"	用户定义补全 i_CTRL-X_CTRL-U
"omni"	全能补全 i_CTRL-X_CTRL-O
"spell"	拼写建议 i_CTRL-X_s
"eval"	complete() 补全
"unknown"	其它内部模式

如果提供可选的 {what} 列表参数，只返回 {what} 中列出的项目。
{what} 中不支持的项目被安静地跳过。

要得到弹出菜单的位置和大小，可见 pum_getpos() 。也可通过 CompleteChanged 事件中的 v:event 得到。

示例:

```
" 取得所有项目
call complete_info()
" 只要 'mode'
call complete_info(['mode'])
" 只要 'mode' 和 'pum_visible'
call complete_info(['mode', 'pum_visible'])
```

也可用作 method :

```
GetItems()->complete_info()
```

complete_check() complete_check()

寻找补全匹配时，检查输入的键。如果寻找匹配需要一定时间时有用。
如果匹配的搜索被中止，返回 TRUE 。否则返回零。

只能用于 'completefunc' 选项指定的函数里。

confirm()

confirm({msg} [, {choices} [, {default} [, {type}]]])

confirm() 提供用户一个对话框，从中可以作出选择。返回选择的序号。第一个选择为 1。

注意：confirm() 只有在编译时加入对话框支持才存在，见 +dialog_con 和 +dialog_gui 。

在 dialog 里显示 {msg} 消息，并提供可能的选择 {choices}。如果 {choices} 不存在或者为空，使用 "&OK"（经过翻译）。

{msg} 是字符串，'\n' 用来包含换行符。在有些系统上该字符串在放不下时被回绕，但并非所有系统都如此。

{choices} 是一个字符串，用 '\n' 分隔各个选择，例如

```
confirm("Save changes?", "&Yes\n&No\n&Cancel")
```

'&' 之后的字符提供该选择的快捷键。这样，你可以输入 'c' 来选择 "Cancel"。快捷键不一定是第一个字符：

```
confirm("file has been modified", "&Save\nSave &All")
```

控制台里，每个选择的第一个字符用作缺省的快捷键。

可选的 {default} 参数是用户按 <CR> 使用的选择号。设定 1 使得第一个选项成为缺省，如果是 0，则不设定任何缺省。如果不提供 {default}，假设为 1。

可选的 {type} 参数指定对话框的类型。只有在 GTK、Mac、Motif 和 Win32 GUI 上才用得上，它用以指定图标。可以取的值是："Error"、"Question"、"Info"、"Warning" 或 "Generic"。只有第一个字符是重要的。如果忽略 {type}，使用 "Generic"。

如果用户用 <Esc>、CTRL-C 或者别的合法的中断键中止对话框，confirm() 返回 0。

一个例子：

```
:let choice = confirm("你要吃什么？", "&苹果\n&桔子\n&香蕉", 2)
:if choice == 0
:    echo "快下定决心！"
:elseif choice == 3
:    echo "好吃"
:else
:    echo "我本人喜欢香蕉。"
:endif
```

GUI 的对话框使用按钮。按钮的排放方式取决于 'guioptions' 里的 'v' 标志位。如果包含该标志位，按钮总是竖排的。不然，confirm() 试图把按钮放在一行里。如果放不下，那么还是使用竖排的方式。在有的系统上，无论如何总是使用横排。

也可用作 method：

```
BuildMessage()->confirm("&Yes\n&No")
```

copy()

copy({expr}) 构造 {expr} 的备份。对数值和字符串而言，这和直接使用 {expr} 并无不同。

如果 {expr} 是 List，建立一个浅备份。这意味着，原来的 List 可以被改变，而不会影响新建的备份。反之亦然。不过，其中的项目只有一份，所以修改项目同时修改两个 List 的内容。

Dictionary 的复制和 List 类似。

另见 deepcopy() 。
也可用作 method :
```
mylist->copy()
```

cos({expr}) cos()
返回浮点数，即以弧度测量的 {expr} 的余弦值。
{expr} 的计算结果必须是浮点数或数值。
示例:
```
:echo cos(100)
0.862319
:echo cos(-4.01)
-0.646043
```

也可用作 method :
```
Compute()->cos()
```

{仅当编译时加入 +float 特性才有效}

cosh({expr}) cosh()
返回 {expr} 的双曲余弦值，返回值为 [1, inf] 区间内的浮点数。
{expr} 的计算结果必须是浮点数或数值。
示例:
```
:echo cosh(0.5)
1.127626
:echo cosh(-0.5)
-1.127626
```

也可用作 method :
```
Compute()->cosh()
```

{仅当编译时加入 +float 特性才有效}

count({comp}, {expr} [, {ic} [, {start}]]) count()
返回 String 、 List 或 Dictionary {comp} 里值为 {expr} 的项目出现的次数。

如果给出 {start}，从该索引指定的项目开始。{start} 只能用于 List 。

如果给出 {ic} 并且为 TRUE ，忽略大小写。

{comp} 为字符串时返回 {expr} 不重叠的出现次数。{expr} 为空串时返回零。

也可用作 method :
```
mylist->count(val)
```

cscope_connection() cscope_connection()
cscope_connection([{num} , {dbpath} [, {prepend}]])
检查 cscope 连接是否存在。如果没有参数，则函数返回:
0，如果 cscope 不存在 (编译没有带该特性)，或者不存在 cscope 连接;

1，如果至少有一个 cscope 连接。

如果指定了参数，那么 {num} 的值指定如何检查 cscope 连接存在与否的方式：

{num}	存在检查的方式描述
0	等同于无参数（例如，"cscope_connection()"）。
1	忽略 {prepend}，使用 {dbpath} 的字符串部分匹配。
2	忽略 {prepend}，使用 {dbpath} 的字符串完整匹配。
3	使用 {prepend}，使用 {dbpath} 和 {prepend} 的字符串部分匹配。
4	使用 {prepend}，使用 {dbpath} 和 {prepend} 的字符串完整匹配。

注意：所有的字符串比较都对大小写敏感！

示例。假定我们有如下设置（":cs show" 的输出）：

```
# pid    database name                       prepend path
0 27664  cscope.out                          /usr/local
```

启动方式	返回值
cscope_connection()	1
cscope_connection(1, "out")	1
cscope_connection(2, "out")	0
cscope_connection(3, "out")	0
cscope_connection(3, "out", "local")	1
cscope_connection(4, "out")	0
cscope_connection(4, "out", "local")	0
cscope_connection(4, "cscope.out", "/usr/local")	1

cursor({lnum}, {col} [, {off}]) cursor()
cursor({list})

把光标定位在第 {lnum} 行的第 {col} 列（字节计数）。第一列为 1。

如果只有一个参数 {list}，它被看作带两个、三个或四个参数的 List：

```
[{lnum}, {col}]
[{lnum}, {col}, {off}]
[{lnum}, {col}, {off}, {curswant}]
```

和 getpos() 和 getcurpos() 的返回值类似，但没有第一项。

不改变跳转表。
如果 {lnum} 超过缓冲区的行数，光标定位在缓冲区的末行。
如果 {lnum} 为零，光标留在当前行。
如果 {col} 超过该行的字节数，光标定位在该行的最后一个字符上。
如果 {col} 为零，光标留在当前列。
如果 {curswant} 给出，它被用于设置垂直移动的首选列，否则使用 {col}。

如果使用 'virtualedit'，{off} 指定从该字符开始以屏幕列计算的偏移。例如，在制表里或者在最后一个字符之后的某个位置。

如果定位成功，返回 0，否则返回 -1。

也可用作 method :
```
GetCursorPos()->cursor()
```

debugbreak({pid}) debugbreak()
专用于中断待调试的程序。给进程 {uid} 发送 SIGTRAP。其它进程的行为未指定。见 terminal-debugger 。
{仅适用于 MS-Windows}

也可用作 method :
```
GetPid()->debugbreak()
```

deepcopy({expr} [, {noref}]) deepcopy() E698
构造 {expr} 的备份。对数值和字符串而言，这和直接使用 {expr} 并无不同。
如果 {expr} 是 List ，建立一个完整的备份。这意味着，原来的 List 可以被改变，而不会影响新建的备份。反之亦然。如果某个项目是 List 或 Dictionary ，递归地建立它的备份。这样，备份里项目的修改也不会影响到原来 List 的内容。
Dictionary 的复制和 List 类似。
如果省略 {noref} 或为零，包含的 List 或 Dictionary 只复制一次。所有指向它的引用指向此一备份。如果 {noref} 设为 1，每次 List 或 Dictionary 的出现会导致建立一个新的备份。这意味着循环引用会使得 deepcopy() 失败。
E724
嵌套可达 100 层。如果有指向较高层的引用，{noref} 为 1 的深备份会失败。
另见 copy() 。

也可用作 method :
```
GetObject()->deepcopy()
```

delete({fname} [, {flags}]) delete()
无 {flags} 或 {flags} 为空：删除名为 {fname} 的文件。{fname} 是符号链接也可以。

如果 {flags} 是 "d"：删除名为 {fname} 的目录。如果目录 {fname} 非空会失败。

如果 {flags} 是 "rf"：递归删除名为 {fname} 的目录和其下的所有文件。 要小心 ！
备注：在 MS-Windows 上不能删除正在使用的目录。

符号链接本身被删除，而不是它链接到的文件。

返回类型为数值。如果成功删除文件，返回 0，如果删除失败或部分失败，返回 -1。

从 List 里删除项目请用 remove() 。
要删除缓冲区的行用 :delete 或 deletebufline() 。

也可用作 method :
```
GetName()->delete()
```

deletebufline({expr}, {first} [, {last}]) deletebufline()
删除缓冲区 {expr} 中 {first} 到 {last} (闭区间) 的文本行。
如果省略 {last}，只删除第 {first} 行。
成功时返回 0，失败时返回 1。

此函数只能用于已载入的缓冲区。有必要的话先调用 bufload() 。

{expr} 的用法见上述的 bufname() 。

{first} 和 {last} 的用法类似于 getline() 。注意 line() 指定当前缓冲区。"$" 指定缓冲区 {expr} 的末行。

也可用作 method :
```
GetBuffer()->deletebufline(1)
```

did_filetype() did_filetype()
如果执行自动命令时，激活 FileType 事件至少一次，则返回 TRUE 。可以用于防止在检测文件类型的脚本里再次激活 FileType 事件。 FileType
如果用过 ":setf FALLBACK"，返回 FALSE 。
如果编辑另外一个文件，该计数被复位，因而这只能检查 FileType 事件是否在当前缓冲区里激活过。它允许开始编辑另一个缓冲区的自动命令设置 'filetype' 并载入语法文件。

diff_filler({lnum}) diff_filler()
返回第 {lnum} 行之上的填充行的数目。
这些是在另一个进行比较的窗口里在此位置插入的行。这些填充行在屏幕上显示，但缓冲区里并不存在。
{lnum} 的用法类似于 getline() 。所以 "." 是当前行，"'m" 是位置标记 m，等等。
如果当前窗口不处于比较模式，返回 0。

也可用作 method :
```
GetLnum()->diff_filler()
```

diff_hlID({lnum}, {col}) diff_hlID()
返回比较模式下行 {lnum} 和列 {col} (字节位置) 所在的高亮 ID。
如果当前行没有差异，返回零。
{lnum} 的用法类似于 getline() 。所以 "." 是当前行，"'m" 是位置标记 m，等等。
最左列的 {col} 为 1，第一行的 {lnum} 为 1。
高亮 ID 可以用于 synIDattr() ，以得到高亮对应的语法信息。

也可用作 method :
```
GetLnum()->diff_hlID(col)
```

environ() environ()
返回所有的环境变量构成的字典。可以这样检查某个环境变量是否存在:
```
:echo has_key(environ(), 'HOME')
```
注意 变量名可以是驼峰式 (CamelCase) 的，要忽略大小写，可以这样:
```
:echo index(keys(environ()), 'HOME', 0, 1) != -1
```

empty({expr}) empty()
如果 {expr} 为空，返回数值 1，否则返回 0。
- List 或 Dictionary 没有项目时为空。
- String 的长度为零时为空。
- Number 或 Float 的值为零时为空。
- v:false 、 v:none 和 v:null 为空， v:true 不是。
- 启动失败的 Job 为空。
- 已关闭的 Channel 为空。
- Blob 的长度为零时为空。

如果 List 很大，这比把长度和零比较要快得多。

也可用作 method :
```
mylist->empty()
```

escape({string}, {chars}) escape()
在 {string} 里用反斜杠转义 {chars} 里的字符。例如:
```
:echo escape('c:\program files\vim', ' \')
```
返回:
```
c:\\program\ files\\vim
```
另见 shellescape() 和 fnameescape() 。

也可用作 method :
```
GetText()->escape(' \')
```

eval({string}) eval()
计算 {string} 并返回其结果。这对把 string() 的结果转换为原来的值尤其有用。适用于数值、浮点数、字符串、blob 和它们的复合类型。也可用于指向已有函数的 Funcref 。

也可用作 method :
```
argv->join()->eval()
```

eventhandler() eventhandler()
如果在事件处理中则返回 1。此时，Vim 在等待用户输入一个字符的时候被中断，比如，在 Vim 上拖放了一个文件。这也意味着此时不能使用交互的命令。如果不是，返回零。

executable({expr}) executable()
本函数检查名字由 {expr} 指定的可执行文件存在与否。{expr} 必须是程序不带任何参数的名字。
executable() 使用 $PATH 的值和/或程序的普通的搜索路径。
PATHEXT
MS-DOS 和 MS-Windows 上，可以可选地包含 ".exe"、".bat" 等。为此，$PATHEXT 里的扩展名会被尝试。这样，如果 "foo.exe" 不存在，可能会找到 "foo.exe.bat"。如果没有设置 $PATHEXT，使用的是 ".exe;.com;.bat;.cmd"。$PATHEXT 里可以使用单独的句号，以尝试没有扩展名的名字。如果 'shell' 看起来像 Unix 外壳，那么也尝试没有扩展名的名字。
MS-DOS 和 MS-Windows 上，只检查是否文件存在且不是目录，并不检查它是否真的可以执行。
MS-Windows 上，和 Vim 在同一目录的可执行文件总能找到。因为这个

目录加到了 $PATH 里，执行应该也没有问题 win32-PATH 。
返回数值：

```
1       存在
0       不存在
-1      此系统中没有实现
```

exepath() 可用于获取可执行文件的完整路径。

也可用作 method ：

```
GetCommand()->executable()
```

execute({command} [, {silent}]) execute()
执行 Ex 命令或命令序列，返回结果为字符串。
{command} 可以是字符串或列表。如果是列表，逐行执行命令。
等价于：

```
redir => var
{command}
redir END
```

可选的 {silent} 参数可取以下的值：

```
""              不用 :silent
"silent"        用 :silent
"silent!"       用 :silent!
```

缺省是 "silent"。注意 用 "silent!" 时，和 :redir 不同，错误被忽略。使用外部命令时，屏幕可能会搞乱，可用 system() 代替。
E930
{command} 里不可用 :redir 。

要得到行的列表，在结果上用 split() ：

```
split(execute('args'), "\n")
```

要在非当前窗口的其它窗口中执行命令，用 win_execute() 。

递归调用时，内层的输出结果不包含在外层的结果里。

也可用作 method ：

```
GetCommand()->execute()
```

exepath({expr}) exepath()
如果 {expr} 是可执行文件并且是绝对路径、相对路径或可在 $PATH 是找到，返回完整路径。
注意 如果 {expr} 以 "./" 开始，使用当前路径，这对 Vim 可能是个问题：

```
echo exepath(v:progpath)
```

如果 {expr} 在 $PATH 里找不到，或者不可执行，返回空字符串。

也可用作 method ：

```
GetCommand()->exepath()
```

exists({expr}) exists()
返回数值，如果 {expr} 被定义，返回 TRUE ，不然返回零。

要检查某特性是否支持，用 has() 。
要检查文件是否存在，用 filereadable() 。

{expr} 参数是字符串，可以使用以下选择之一：

&option-name	Vim 选项（只检查是否存在，而不是是否工作）
+option-name	能工作的 Vim 选项。
$ENVNAME	环境变量（也可以通过和空字符串比较完成）
*funcname	内建函数（见 functions ）或者用户定义的函数（见 user-functions ）。也可用于指向函数引用的变量。
varname	内部变量（见 internal-variables ）。也适用于 curly-braces-names 、 Dictionary 项目、 List 项目等等。 小心计算索引时可能会因为非法的表达式产生错误信息。例如： :let l = [1, 2, 3] :echo exists("l[5]") 0 :echo exists("l[xx]") E121: Undefined variable: xx 0
:cmdname	Ex 命令：内建命令、用户命令或者命令修饰符 :command 。 返回： 1 匹配命令的开始 2 完整匹配命令 3 匹配多个用户命令 要检查命令是否支持，检查返回值是否为 2。
:2match	:2match 命令。
:3match	:3match 命令。
#event	符合此事件的自动命令
#event#pattern	符合此事件和此模式的自动命令（模式按本义出现，和自动命令的模式逐字符比较）
#group	存在自动命令组
#group#event	在该组里定义并符合此事件的自动命令。
#group#event#pattern	为该组定义、符合事件和模式的自动命令。
##event	支持此事件的自动命令。

例如：

```
exists("&shortname")
exists("$HOSTNAME")
exists("*strftime")
exists("*s:MyFunc")
exists("bufcount")
exists(":Make")
exists("#CursorHold")
exists("#BufReadPre#*.gz")
exists("#filetypeindent")
exists("#filetypeindent#FileType")
exists("#filetypeindent#FileType#*")
exists("##ColorScheme")
```

符号 (&/$/*/#) 和名字之间不能有空格。
名字之后不能有附加字符，虽然现在有一些情况下会忽略这些字符，但将来的检查可能更严格。所以不要依赖此行为！
可以通过的例子：

```
exists(":make")
```

不 能通过的例子：

```
exists(":make install")
```

注意 参数必须是字符串，不是变量的名字本身。例如：

```
exists(bufcount)
```

不检查 "bufcount" 变量是否存在，而是提取 "bufcount" 的值，并检查其（根据此处的语法）是否存在。

也可用作 method ：

```
Varname()->exists()
```

exp({expr}) exp()
返回 {expr} 的指数函数值，返回值为 [0, inf] 区间内的浮点数。
{expr} 的计算结果必须是浮点数或数值。
示例：

```
:echo exp(2)
7.389056
:echo exp(-1)
0.367879
```

也可用作 method ：

```
Compute()->exp()
```

{仅当编译时加入 +float 特性才有效}

expand({expr} [, {nosuf} [, {list}]]) expand()
扩展 {expr} 里的通配符和下列特殊关键字。
'wildignorecase' 此处适用。

如果给出 {list} 且为 TRUE ，返回列表。否则返回的是字符串，且如果返回多个匹配，以 <NL> 字符分隔 [备注：5.0 版本使用空格。但是文件名如果也包含空格就会有问题]。

如果扩展失败，返回空字符串。如果 {expr} 以 '%'，'#' 或 '<' 开始，不返回不存在的文件名。详见下。

如果 {expr} 以 '%'、'#' 或 '<' 开始，以类似于 cmdline-special 变量的方式扩展，包括相关的修饰符。这里是一个简短的小结：

%	当前文件名
#	轮换文件名
#n	轮换文件名 n
<cfile>	光标所在的文件名
<afile>	自动命令文件名
<abuf>	自动命令缓冲区号（以字符串形式出现！）
<amatch>	自动命令匹配的名字
<sfile>	载入的脚本文件或函数名

<slnum>	载入脚本文件行号或函数行号
<sflnum>	载入脚本文件行号，即使在函数中亦然
<cword>	光标所在的单词
<cWORD>	光标所在的字串（WORD）
<client>	最近收到的消息的 {clientid} server2client()

修饰符：

:p	扩展为完整的路径
:h	头部（去掉最后一个部分）
:t	尾部（只保留最后一个部分）
:r	根部（去掉一个扩展名）
:e	只有扩展名

例如：

```
:let &tags = expand("%:p:h") . "/tags"
```

注意 扩展 '%'、'#' 或者 '<' 开头的字符串的时候，其后的文本被忽略。这样 不 行：

```
:let doesntwork = expand("%:h.bak")
```

应该这样：

```
:let doeswork = expand("%:h") . ".bak"
```

还要 注意 扩展 "<cfile>" 和其它形式只能返回被引用的文件名，而不会进一步扩展。如果 "<cfile>" 是 "~/.cshrc"，你需要执行另一个 expand() 把 "~/" 扩展为主目录的路径：

```
:echo expand(expand("<cfile>"))
```

变量名和其后的修饰符之间不能有空白。 fnamemodify() 函数可以用来修改普通的文件名。

使用 '%' 或 '#' 但当前或轮换文件名没有定义的时候，使用空字符串。在无名缓冲区使用 "%:p" 生成当前目录，后加一个 '/'。

如果 {expr} 不以 '%'、'#' 或 '<' 开始，它以命令行上的文件名那样被扩展。使用 'suffixes' 和 'wildignore'，除非给出可选的 {nosuf} 参数而且为 TRUE 。
这里可以有不存在的文件的名字。"**" 项目可以用来在目录树里查找。例如，要寻找当前目录及其下目录的所有的 "README"：

```
:echo expand("**/README")
```

expand() 也可用来扩展变量和只有外壳知道的环境变量。但这会很慢，因为需要使用外壳才能进行扩展。见 expr-env-expand 。
扩展后的变量还是被当作文件名的列表处理。如果不能扩展环境变量，保留其不变。这样， ":echo expand('$FOOBAR')" 返回的还是 "$FOOBAR"。

glob() 说明如何找到存在的文件。 system() 说明如何得到外部命令的原始输出。

也可用作 method ：

```
Getpattern()->expand()
```

expandcmd({expr}) expandcmd()

像 :edit 这样的 Ex 命令那样扩展 {expr} 里的特殊项目。在 {expr} 的所有位置像 expand() 那样扩展特殊关键字，也扩展环境

变量。"~user" 和 "~/path" 只在开始处被扩展。
返回扩展后的字符串。示例：

```
:echo expandcmd('make %<.o')
```

也可用作 method ：

```
GetCommand()->expandcmd()
```

extend({expr1}, {expr2} [, {expr3}]) extend()
{expr1} 和 {expr2} 必须都是 List 或者都是 Dictionary 。

如果都是 List ：把 {expr2} 附加到 {expr1} 之后。
如果给出 {expr3}，把 {expr2} 里的项目加到 {expr1} 的第 {expr3} 个项目之前。如果 {expr3} 为零，插在第一个项目之前。如果 {expr3} 等于 len({expr1})，那么 {expr2} 会附加在最后。
例如：

```
:echo sort(extend(mylist, [7, 5]))
:call extend(mylist, [2, 3], 1)
```

如果 {expr1} 和 {expr2} 是同一个列表，复制的项目数等于列表原来的长度。例如，如果 {expr} 为 1，则复制列表首值 N 次 (N 是列表原来的长度)。
用 add() 把一个项目加入列表。要连接两个列表成为一个新列表，用 + 操作符：

```
:let newlist = [1, 2, 3] + [4, 5]
```

如果都是 Dictionary ：把 {expr2} 里的所有项目加入 {expr1}。
如果 {expr1} 和 {expr2} 包含相同的键，那么 {expr3} 决定应该怎么做：
{expr3} = "keep"：保持 {expr1} 的值
{expr3} = "force"：使用 {expr2} 的值
{expr3} = "error"：给出错误信息 E737
如果省略 {expr3}，假设使用 "force"。

只要 {expr2} 非空，{expr1} 就被改变。如果需要，给 {expr1} 先做个备份。
{expr2} 保持不变。
如果 {expr1} 被锁住且 {expr2} 非空，操作失败。
返回 {expr1}。

也可用作 method ：

```
mylist->extend(otherlist)
```

feedkeys({string} [, {mode}]) feedkeys()
将 {string} 里的字符放在队列里等候处理，就像它们来自映射或者用户输入一样。

缺省它们加在预输入 (typeahead) 缓冲区的尾端，所以如果仍然在执行映射，这些字符会出现在映射内容之后。用 'i' 标志位则插入位置在其它字符之前，这样，它们在映射其它字符之前就会执行。

该函数不会等待 {string} 包含的键处理完毕。

{string} 如果要包含特殊键，可以使用双引号和 "\..." 记法 expr-quote 。例如，feedkeys("\<CR>") 会模拟 <Enter> 键击。但

feedkeys('\<CR>') 却实际压入五个字符。
一个可能有用的特殊代码是 <Ignore>，它会结束等待字符输入而不做任何事。 <Ignore>

{mode} 是字符串，包含以下字符标志位：
'm' 对键重映射。缺省。如果不给出 {mode}，键会经过重映射。
'n' 不对键重映射。
't' 像用户输入那样处理键；如果不包含，像来自映射一样处理。这会影响撤销、打开折叠等行为。
'L' 底层输入。只用于 Unix 或 GUI。键的使用就像来自终端那样。不使用其它标志位。 E980
'i' 插入字符串而不是附加（见上）。
'x' 执行命令，直至预输入为空为止。和 ":normal!" 类似。可以不带 'x' 执行 feedkeys() 数次，然后一次性执行带 'x' 的（可以只带空 {string}）来执行所有的输入。注意 Vim 退出插入模式时行为就如同按了 <Esc> 那样，以避免在脚本可以继续前等待字符键入的阻塞情况。
注意 你执行命令时调用了 feedkeys()，会导致递归调用，而所有的预输入会被最后的那个调用消耗。
'!' 使用 'x' 时不停止插入模式。可用于使定时器稍晚点才退出插入状态的测试环境。对测试 CursorHoldI 有用。

返回值总为 0。

也可用作 method :
```
GetInput()->feedkeys()
```

filereadable({file}) filereadable()
返回数值，如果名为 {file} 的文件存在且可读，则为 TRUE 。如果 {file} 不存在，或者是一个目录，返回 FALSE 。{file} 可以是任何返回字符串的表达式。
如果你不关心文件是否可读，可用 glob() 。
按本义使用 {file}，你可能需要先扩展通配符：
```
echo filereadable('~/.vimrc')
0
echo filereadable(expand('~/.vimrc'))
1
```

也可用作 method :
```
GetName()->filereadable()
```
file_readable()
已废弃的名字：file_readable()。

filewritable({file}) filewritable()
返回数值，如果名为 {file} 的文件存在且可写，则为 1。如果 {file} 不存在，或者不可写，返回 0。如果 {file} 是一个目录但是可写，返回 2。

也可用作 method :
```
GetName()->filewriteable()
```

filter({expr1}, {expr2}) filter()

{expr1} 必须是 List 或者 Dictionary 。
对 {expr1} 里的每个项目计算 {expr2}，如果结果为零，从该 List 或 Dictionary 里删除该项目。
{expr2} 必须是 string 或 Funcref 。

如果 {expr2} 是 string ，{expr2} 内的 v:val 包含当前项目的值。 Dictionary 中 v:key 包含当前项目的键。 List 中 v:key 包含当前项目的索引。
例如:

```
call filter(mylist, 'v:val !~ "OLD"')
```

删除所有出现 "OLD" 的项目。

```
call filter(mydict, 'v:key >= 8')
```

删除所有键小于 8 的值。

```
call filter(var, 0)
```

删除所有的值，从而清除该 List 或 Dictionary 。

如果 {expr2} 是 Funcref ，它必须接受两个参数:
1. 当前项目的键或索引。
2. 当前项目的值。
如果该项目应保留，此函数必须返回 TRUE 。
例如要保留列表中奇数项目:

```
func Odd(idx, val)
  return a:idx % 2 == 1
endfunc
call filter(mylist, function('Odd'))
```

使用 lambda 会更短:

```
call filter(myList, {idx, val -> idx * val <= 42})
```

如果不用 "val"，可以不写:

```
call filter(myList, {idx -> idx % 2 == 1})
```

该操作是原位操作 (直接在输入上修改)。要想不更动 List 或 Dictionary ，先建立备份:

```
:let l = filter(copy(mylist), 'v:val =~ "KEEP"')
```

返回 {expr1}，经过过滤的 List 或 Dictionary 。如果执行 {expr2} 有错，不再处理 {expr1} 的其余项目。{expr2} 是函数引用时，忽略函数里的错误，除非该函数用 "abort" 标志位定义。

也可用作 method :

```
mylist->filter(expr2)
```

finddir({name} [, {path} [, {count}]]) finddir()

在 {path} 里查找目录 {name}。支持向下和向上的递归目录搜索。
{path} 的语法参见 file-searching 。
返回第一个找到的路径。如果找到的
路径在当前目录之下，返回相对路径。否则，返回完整路径。
如果省略 {path}，使用 'path'。
如果给出可选的 {count}，寻找 {path} 里 {name} 第 {count} 次出现，而不是第一次。
如果 {count} 为负，返回所有的匹配的列表。
这和 ex 命令 :find 非常类似。
{仅当编译时加入 +file_in_path 特性才有效}

也可用作 method :
```
GetName()->finddir()
```

findfile({name} [, {path} [, {count}]]) findfile()
类似于 finddir() ，不过寻找文件而不是目录。
使用 'suffixesadd'。
例如:
```
:echo findfile("tags.vim", ".;")
```
从当前文件所在的目录开始往上搜索，直到找到文件 "tags.vim" 为止。

也可用作 method :
```
GetName()->findfile()
```

float2nr({expr}) float2nr()
返回数值，即 {expr} 省略小数点部分的结果。
{expr} 的计算结果必须是浮点数或数值。
如果 {expr} 的值超出 Number 的范围，结果为 0x7fffffff 或 -0x7fffffff (有 64-位 数值支持时，则为 0x7fffffffffffffff 或 -0x7fffffffffffffff)。而 NaN 转换为 -0x80000000 (有64-位数值支持时，则为 -0x8000000000000000)。
示例:
```
echo float2nr(3.95)
3
echo float2nr(-23.45)
-23
echo float2nr(1.0e100)
2147483647 (或 9223372036854775807)
echo float2nr(-1.0e150)
-2147483647 (或 -9223372036854775807)
echo float2nr(1.0e-100)
0
```

也可用作 method :
```
Compute()->float2nr()
```

{仅当编译时加入 +float 特性才有效}

floor({expr}) floor()
返回浮点数，即小于等于 {expr} 的最大整数 (向下取整)。
{expr} 的计算结果必须是浮点数或数值。
示例:
```
echo floor(1.856)
1.0
echo floor(-5.456)
-6.0
echo floor(4.0)
4.0
```

也可用作 method :
```
Compute()->floor()
```

{仅当编译时加入 +float 特性才有效}

fmod({expr1}, {expr2}) fmod()

返回 {expr1} / {expr2} 的余数，该除法可能实际无法表达出来。选择某整数 i，返回 {expr1} - i * {expr2}，使得如果 {expr2} 非零，结果和 {expr1} 同号而绝对值小于 {expr2} 的绝对值。如果 {expr2} 为零，返回零。返回值为浮点数。
{expr1} 和 {expr2} 的计算结果必须是浮点数或数值。
示例:

```
:echo fmod(12.33, 1.22)
0.13
:echo fmod(-12.33, 1.22)
-0.13
```

也可用作 method :

```
Compute()->fmod(1.22)
```

{仅当编译时加入 +float 特性才有效}

fnameescape({string}) fnameescape()

转义 {string} 以便用作命令的文件名参数。有特殊意义的字符，如 '%' 和 '|'，会用反斜杠转义。
多数系统上，会转义的字符是 " \t\n*?[{`$\\%#'\"|!<"。在反斜杠可以出现在文件名中的系统上，此字符集取决于 'isfname'。
也转义出现在开头的 '+' 和 '>' 字符 (:edit 和 :write 之后有特殊意义)，还有单个出现的 "-" (:cd 之后有特殊意义)。
示例:

```
:let fname = '+some str%nge|name'
:exe "edit " . fnameescape(fname)
```

则会执行:

```
edit \+some\ str\%nge\|name
```

也可用作 method :

```
GetName()->fnameescape()
```

fnamemodify({fname}, {mods}) fnamemodify()

根据 {mods} 修改文件名 {fname}。{mods} 是一个字符序列组成的字符串，就像命令行上使用的文件名那样。见 filename-modifiers 。
例如:

```
:echo fnamemodify("main.c", ":p:h")
```

返回:

```
/home/mool/vim/vim/src
```

注意: 环境变量不能用于 {fname}，需要先用 expand() 扩展。

也可用作 method :

```
GetName()->fnamemodify(':p:h')
```

foldclosed({lnum}) foldclosed()

返回数值，如果行 {lnum} 在关闭的折叠中，返回该折叠开始的行号。如果行 {lnum} 不在关闭的折叠中，返回 -1。

也可用作 method :

```
GetLnum()->foldclosed()
```

foldclosedend({lnum}) foldclosedend()
返回数值，如果行 {lnum} 在关闭的折叠中，返回该折叠结束的行号。如果行 {lnum} 不在关闭的折叠中，返回 -1。

也可用作 method :
```
GetLnum()->foldclosedend()
```

foldlevel({lnum}) foldlevel()
返回数值，当前缓冲区第 {lnum} 行的折叠级别。如果在嵌套的折叠里，返回最深的那层。如果行 {lnum} 没有折叠，返回零。这和折叠是打开还是关闭无关。在更新折叠时（在 'foldexpr' 里），如果折叠还在更新而相应的折叠级别未知，返回 -1。一个特例是前一行的级别通常总是知道的。

也可用作 method :
```
GetLnum()->foldlevel()
```

foldtext() foldtext()
返回关闭的折叠所显示的行。这是 'foldtext' 选项使用的缺省函数，而且也只应该在计算 'foldtext' 时使用。它使用 v:foldstart 、v:foldend 和 v:folddashes 变量。
返回的字符串看起来像：
```
+-- 45 lines: abcdef
```
开头的连字符的数目取决于折叠级别。"45" 是折叠的行数。"abcdef" 是折叠第一个非空白行的文本。开头的空白、"//" 和 "/*" 还有 'foldmarker' 和 'commentstring' 选项的文本都被去除。
{仅当编译时加入 +folding 特性才有效}

foldtextresult({lnum}) foldtextresult()
返回行 {lnum} 所在的关闭的折叠显示的文本。在合适的上下文里计算 'foldtext'。
如果 {lnum} 没有关闭的折叠，返回空字符串。
{lnum} 的用法类似于 getline() 。所以 "." 是当前行，"'m" 是位置标记 m，等等。
可用于输出折叠文本，例如，到 HTML 格式。
{仅当编译时加入 +folding 特性才有效}

也可用作 method :
```
GetLnum()->foldtextresult()
```

foreground() foreground()
把 Vim 窗口带到前台。用于从客户发送到 Vim 服务器的时候。 remote_send()
在 Win32 系统上，可能不行，操作系统并不总能允许窗口把自己带到前台。这时应使用 remote_foreground() 。
{仅当使用 Win32、Athena、Motif 和 GTK GUI 版本和 Win32 控制台版本时才有效}

funcref({name} [, {arglist}] [, {dict}]) funcref()
类似于 function() ，但返回的函数引用通过引用来查找函数，而不是名字。如果函数 {name} 之后被重定义，这很有意义。

和 function() 不同，{name} 必须是已经定义的用户函数。自动载入函数也可以。{name} 不能是内建函数。

也可用作 method :

```
GetFuncname()->funcref([arg])
```

function() E700 E922 E923

function({name} [, {arglist}] [, {dict}])

返回指向函数 {name} 的 Funcref 变量。{name} 可以是用户定义的函数或者内部函数的名字。

{name} 可以是函数引用或偏函数。如果是偏函数，会使用其保存的字典，而不接受 {dict} 参数。例如:

```
let FuncWithArg = function(dict.Func, [arg])
let Broken = function(dict.Func, [arg], dict)
```

使用函数引用时，通过 {name} 查找函数，即使以后被重定义了亦然。要保留相同的函数，用 funcref() 。

提供 {arglist} 或 {dict} 则会建立偏函数。这意味着参数列表和/或字典会存放在函数引用里，并在调用函数引用时使用。

参数被传递到函数里，在其它参数之前，但在来自 method 的参数之后。例如:

```
func Callback(arg1, arg2, name)
...
let Partial = function('Callback', ['one', 'two'])
...
call Partial('name')
```

函数的调用就类似于:

```
call Callback('one', 'two', 'name')
```

用 method 的话:

```
func Callback(one, two, three)
...
let Partial = function('Callback', ['two'])
...
eval 'one'->Partial('three')
```

函数的调用就类似于:

```
call Callback('one', 'two', 'three')
```

function() 调用可以嵌套，来给函数引用加入更多的参数。额外的参数附加于参数列表之后。如:

```
func Callback(arg1, arg2, name)
...
let Func = function('Callback', ['one'])
let Func2 = function(Func, ['two'])
...
call Func2('name')
```

函数的调用就类似于:

```
call Callback('one', 'two', 'name')
```

字典只对 "dict" 函数的调用有用。该情况下，{dict} 作为 "self"

传入。例如：

```
function Callback() dict
   echo "called for " . self.name
endfunction
...
let context = {"name": "example"}
let Func = function('Callback', context)
...
call Func()     " 会回显: called for example
```

如果不需额外参数，function() 是没有必要的。这两者等价：

```
let Func = function('Callback', context)
let Func = context.Callback
```

可以合并参数列表和字典：

```
function Callback(arg1, count) dict
...
let context = {"name": "example"}
let Func = function('Callback', ['one'], context)
...
call Func(500)
```

函数的调用就类似于：

```
call context.Callback('one', 500)
```

也可用作 method ：

```
GetFuncname()->function([arg])
```

garbagecollect([{atexit}]) garbagecollect()

清理不再使用但有循环引用的 List 、 Dictionary 、 Channel 和 Job 。

几乎没有需要调用这个函数，因为 Vim 内存不足或者 'updatetime' 之后等待用户按键时会自动执行此功能。没有循环引用的项目总是在不再使用的时候就被立即释放了。
可用于删除很大的 List 和/或 Dictionary 而且有循环引用的时候，尤其是在要运行很长时间的脚本里。

如果可选的 {atexit} 参数为一，并且之前还没做过的话，Vim 即使在退出时也会执行垃圾回收。可用于检查内存泄漏。

垃圾清理不是立即进行的，它会等待安全的时机，就是等待用户输入字符的时候。要强制立即进行垃圾清理，可用 test_garbagecollect_now() 。

get({list}, {idx} [, {default}]) get()

获取 List {list} 的第 {idx} 个项目。如果不存在此项目，返回 {default}。如果省略 {default}，返回零。
也可用作 method ：

```
mylist->get(idx)
```

get({blob}, {idx} [, {default}])

获取 Blob {blob} 的第 {idx} 个字节。如果不存在此字节，返回 {default}。如果省略 {default}，返回 -1。

get({dict}, {key} [, {default}])

获取 Dictionary {dict} 键为 {key} 的项目。如果不存在此项目，

返回 {default}。如果省略 {default}，返回零。有用的例子：

```
let val = get(g:, 'var_name', 'default')
```

如果 g:var_name 存在，返回它的值，如果不存在返回 'default'。

get({func}, {what})

获取 函数引用 {func} 的项目。{what} 的可能值是：

"name"	函数名
"func"	函数
"dict"	字典
"args"	参数列表

getbufinfo()

getbufinfo([{expr}])
getbufinfo([{dict}])

获取字典列表形式的缓冲区信息。

不带参数则返回关于所有缓冲区的信息。

只给出字典参数时，返回匹配相关条件的缓冲区。{dict} 中可指定以下键值：

buflisted	只包含在列表内的缓冲区。
bufloaded	只包含已载入的缓冲区。
bufmodified	只包含修改过的缓冲区。

否则，{expr} 指定特定的单个缓冲区并返回其信息。{expr} 的用法见上述 bufname() 。如果找到缓冲区，返回的列表有一个项目。不然返回空列表。

每个返回的列表项目是带有以下项目的字典：

bufnr	缓冲区号。
changed	若缓冲区已修改则为 TRUE。
changedtick	缓冲区作过的改动次数。
hidden	若缓冲区隐藏则为 TRUE。
lastused	缓冲区最近修改时间， localtime() 那样的以秒计的时间戳。 {仅当编译时加入 +viminfo 特性才有效}
listed	若缓冲区在列表内，则为 TRUE。
lnum	缓冲区的当前行号。
linecount	缓冲区行数（仅当缓冲区载入后才有效）
loaded	若缓冲区已载入则为 TRUE。
name	缓冲区文件的完整列表。
signs	缓冲区内放置的标号的列表。 每个列表项目是以下项目的字典： id 标号的识别符 lnum 行号 name 标号名
variables	缓冲区局部变量的字典的引用。
windows	显示此缓冲区的所有 window-ID 的列表
popups	显示此缓冲区的弹出窗口 window-ID 的列表

示例：

```
for buf in getbufinfo()
    echo buf.name
endfor
```

```
for buf in getbufinfo({'buflisted':1})
    if buf.changed
        ....
    endif
endfor
```

要获取缓冲区局部选项，可用：

```
getbufvar({bufnr}, '&option_name')
```

getbufline()

getbufline({expr}, {lnum} [, {end}])

返回 {expr} 缓冲区的第 {lnum} 到 {end} (包含) 行的 List 。如果省略 {end}，返回只有一行 {lnum} 的 List 。

{expr} 的用法见上述 bufname() 。

{lnum} 和 {end} 可以使用 "$" 来表示缓冲区的最后一行。除此以外，必须用数值。

如果 {lnum} 小于 1 或大于缓冲区的行数，返回空 List 。

如果 {end} 大于缓冲区的行数，就把它当成缓冲区的行数。如果 {end} 在 {lnum} 之前，返回空 List 。

此函数只能用于已经载入的缓冲区。未载入或不存在的缓冲区总是返回空 List 。

例如：

```
:let lines = getbufline(bufnr("myfile"), 1, "$")
```

也可用作 method :

```
GetBufnr()->getbufline(lnum)
```

getbufvar({expr}, {varname} [, {def}]) getbufvar()

返回缓冲区 {expr} 里的选项或者局部变量 {varname} 的值。注意 必须使用不带 "b:" 的名字。
如果 {varname} 为空，返回包含所有缓冲区局部变量的字典。
如果 {varname} 为 "&"，返回包含所有缓冲区局部选项的字典。
否则，如果 {varname} 以 "&" 开始，返回单个缓冲区局部选项值。
也可用于全局或者局部于缓冲区的选项，但不能用于全局或者局部于窗口的变量，还有局部于窗口的选项。
{expr} 的用法见上述 bufname() 。
如果缓冲区或者变量不存在，返回 {def} 或空字符串。不会有错误消息。
示例：

```
:let bufmodified = getbufvar(1, "&mod")
:echo "todo myvar = " . getbufvar("todo", "myvar")
```

也可用作 method :

```
GetBufnr()->getbufvar(varname)
```

getchangelist([{expr}]) getchangelist()

返回缓冲区 {expr} 里的 changelist 。{expr} 的用法见上述

bufname() 。缓冲区 {expr} 不存在时返回空列表。

返回列表包含两个项目：包含改变位置的列表以及在该列表中的当前位置。改变位置列表的每个项目是包含以下内容的字典：

col	列号
coladd	用于 'virtualedit' 的列偏移
lnum	行号

如果缓冲区 {expr} 是当前缓冲区，当前位置指的是在列表中的位置。如果是其它缓冲区，它设为列表的长度。

也可用作 method ：

```
GetBufnr()->getchangelist()
```

getchar([expr]) getchar()

从用户或输入流中提取单个字符。
如果忽略 [expr]，等待直到有字符输入为止。
如果 [expr] 为 0，只有在有字符可用时才取得字符，否则返回零。
如果 [expr] 为 1，只检查是否有字符可用，并不消耗该字符。如果没有字符，返回零。

如果省略 [expr] 或者 [expr] 为零，返回整个字符或者特殊键。如果是单个字符，以数值形式返回。用 nr2char() 把它转化成字符串。否则返回经过编码的字符构成的字符串。如果是特殊键，返回以 0x80 (十进制：128) 开始的一串字节构成的字符串。它和字符串 "\<Key>" 等值，例如 "\<Left>"。如果用带修饰符 (Shift，Control， Alt) 的字符而字符本身不包含该修饰符时，返回值也用字符串类型。

如果 [expr] 为 0 并已键入 Esc，会有短暂的延迟，以便 Vim 有机会判断是否是转义序序列的开始。

如果 [expr] 为 1，只返回第一个字节。如果是单字节字符，返回的就是该字符自身的数值形式。用 nr2char() 把它转化为字符串。

getcharmod() 可用于得到附加的修饰符。

用户点击鼠标时，返回鼠标事件。所在的位置可以在 v:mouse_col 、 v:mouse_lnum 、 v:mosue_winid 和 v:mouse_win 里找到。
也可用 getmousepos() 。下例用通常的处理方法定位鼠标：

```
let c = getchar()
if c == "\<LeftMouse>" && v:mouse_win > 0
  exe v:mouse_win . "wincmd w"
  exe v:mouse_lnum
  exe "normal " . v:mouse_col . "|"
endif
```

使用括号内粘贴模式时，只返回首个字符。粘贴文本其余部分被丢弃。 xterm-bracketed-paste 。

这里没有提示，你需要想办法告诉用户，需要输入一个字符。
字符不通过映射。
键码被替换。因而，用户输入 <Del> 键时，你得到 <Del> 的键码，而不是原始的字符序列。比如：

```
getchar() == "\<Del>"
getchar() == "\<S-Left>"
```

下例重新定义 "f"，使它忽略大小写：

```
:nmap f :call FindChar()<CR>
:function FindChar()
:  let c = nr2char(getchar())
:  while col('.') < col('$') - 1
:    normal l
:    if getline('.')[col('.') - 1] ==? c
:      break
:    endif
:  endwhile
:endfunction
```

也可能收到模拟字符，例如 <CursorHold> 。通常你想忽略之跳到下个字符：

```
:function GetKey()
:  let c = getchar()
:  while c == "\<CursorHold>"
:    let c = getchar()
:  endwhile
:  return c
:endfunction
```

getcharmod() getcharmod()

返回数值，反映最近用 getchar() 或其它方式输入字符的修饰符状态。这些值可以相加：

```
2       Shift
4       Control
8       Alt (Meta)
16      Meta (当和 ALT 不同时)
32      鼠标双击
64      鼠标三击
96      鼠标四击 (== 32 + 64)
128     Command (仅限于 Macintosh)
```

只有没有包含字符本身的修饰符被返回。因而，Shift-a 产生没有修饰符的 "A"。

getcharsearch() getcharsearch()

以 {dict} 形式返回当前字符搜索信息，带有以下项目：

char	上次字符搜索（ t 、 f 、 T 或 F ）使用的字符；空字符串代表没有字符搜索进行过
forward	字符搜索的方向；1 为正向，0 为反向
until	字符搜索的类型；1 为 t 为 T 字符搜索，0 为 f 或 F 字符搜索

可用于使 ; 和 , 永远进行正向/反向搜索而不管上次字符搜索的方向：

```
:nnoremap <expr> ; getcharsearch().forward ? ';' : ','
:nnoremap <expr> , getcharsearch().forward ? ',' : ';'
```

另见 setcharsearch() 。

getcmdline() getcmdline()

返回当前命令行。只有在编辑命令行时有效，所以必须在 c_CTRL-_e 或 c_CTRL-R_= 里使用。

例如：

```
:cmap <F7> <C-\>eescape(getcmdline(), ' \')<CR>
```

另见 getcmdtype() 、 getcmdpos() 和 setcmdpos() 。
输入密码或用 inputsecret() 时返回空串。

getcmdpos() getcmdpos()

返回命令行的字节计算的光标位置。第一列为 1。
只有在编辑命令行时有效，所以必须在 c_CTRL-_e 或 c_CTRL-R_= 或表达式映射里使用。
不然，返回 0。
另见 getcmdtype() 、 setcmdpos() 和 getcmdline() 。

getcmdtype() getcmdtype()

返回当前命令行类型。可能的返回值是：

```
:   普通 Ex 命令
>   调试模式命令 debug-mode
/   正向搜索命令
?   反向搜索命令
@   input() 命令
-   :insert 或 :append 命令
=   i_CTRL-R_=
```

只能在编辑命令行时调用，因而必须在 c_CTRL-_e 或 c_CTRL-R_= 或表达式映射里使用。
否则，返回空字符串。
另见 getcmdpos() 、 setcmdpos() 和 getcmdline() 。

getcmdwintype() getcmdwintype()

返回当前 command-line-window 类型。可能的返回值和 getcmdtype() 相同。如果不在命令行窗口内，返回空字符串。

getcompletion({pat}, {type} [, {filtered}]) getcompletion()

返回命令行补全匹配列表。{type} 指定类型。支持以下的匹配类型：

```
arglist         参数列表中的文件名
augroup         自动命令组
buffer          缓冲区名
behave          :behave 子选项
color           色彩方案
command         Ex 命令 (及参数)
compiler        编译器
cscope          :cscope 子选项
dir             目录名
environment     环境变量名
event           自动命令事件
expression      Vim 表达式
file            文件和目录名
file_in_path    'path' 中的文件和目录名
filetype        文件类型名 'filetype'
function        函数名
help            帮助主题
highlight       高亮组
history         :history 子选项
locale          locale 名 (可见 locale -a 的输出)
mapclear        缓冲区参数
```

mapping	映射名
menu	菜单
messages	:messages 子选项
option	选项
packadd	可选包 pack-add 名
shellcmd	外壳命令
sign	:sign 子选项
syntax	语法文件名 'syntax'
syntime	:syntime 子选项
tag	标签
tag_listfiles	标签、文件名
user	用户名
var	用户变量

如果 {pat} 为空串，返回所有匹配。否则只返回匹配 {pat} 的项目。
关于 {pat} 中特殊字符的使用，见 wildcards 。

如果可选的 {filtered} 标志位设为 1，应用 'wildignore' 来过滤结果。否则返回所有匹配。'wildignorecase' 选项则总是有效。

如果没有匹配，返回空列表。如果 {type} 为非法值，报错。

也可用作 method :

```
GetPattern()->getcompletion('color')
```

getcurpos()

getcurpos() 返回光标位置。类似于 getpos('.')，但包含在列表中一个额外的项目：

```
[bufnum, lnum, col, off, curswant]
```

"curswant" 数值是垂直移动光标时的首选列。另见 getpos() 。

可用于保存和恢复光标位置：

```
let save_cursor = getcurpos()
移动光标
call setpos('.', save_cursor)
```

注意 这只适用于同一窗口内的移动。要恢复更多状态，见 winrestview() 。

getcwd()

getcwd([{winnr} [, {tabnr}]])
返回字符串，当前工作目录的名字。

有 {winr} 则返回当前标签页的该窗口的本地当前目录。{winr} 可以是窗口号或 window-ID 。
{winr} 为 -1 时返回全局工作目录名。另见 haslocaldir() 。

有 {winr} 和 {tabnr} 则返回指定标签页和窗口的本地当前目录。如果 {winnr} 为 -1，返回指定标签页的当前目录。
如果 {winr} 为零使用当前窗口。如果 {tabnr} 为零使用当前标签页。
无参数时，返回当前窗口的工作目录。
如果参数非法，返回空串。

示例：

```
" 返回当前窗口的工作目录
```

```
:echo getcwd()
:echo getcwd(0)
:echo getcwd(0, 0)
" 返回标签页 2 中的窗口 3 的工作目录
:echo getcwd(3, 2)
" 返回全局工作目录
:echo getcwd(-1)
" 返回标签页 3 的工作目录
:echo getcwd(-1, 3)
" 返回当前标签页的工作目录
:echo getcwd(-1, 0)
```

也可用作 method :

```
GetWinnr()->getcwd()
```

getenv({name}) getenv()

返回环境变量 {name} 的值。
如果该变量不存在返回 v:null 。这和变量设为空串时不同，不过有的系统把空值解读为删除了的变量。另见 expr-env 。

也可用作 method :

```
GetVarname()->getenv()
```

getfontname([{name}]) getfontname()

如果没有参数，返回使用的正常字体的名字，也就是 Normal 高亮组 hl-Normal 使用的。
如果带了参数，检查 {name} 是否合法的字体名。如果不是，返回空字符串。否则，返回实际的字体名，或者如果 GUI 不支持取得真正的名字，返回 {name}。
只有在 GUI 运行的时候才能用，所以不能用于你的 vimrc 和 gvimrc 文件。用 GUIEnter 自动命令可以在 GUI 刚开始之后使用此函数。
注意 GTK GUI 接受任何字体名，所以不会检查名字是否合法。

getfperm({fname}) getfperm()

返回字符串，给定文件 {fname} 的读、写、执行权限。
如果 {fname} 不存在或者它所在的目录无法读取，返回空字符串。
返回值的形式是 "rwxrwxrwx"，其中每组 "rwx" 标志位分别代表文件所有者、文件所属组和其它用户的权限。如果用户没有某权限，相应的标志位被字符串 "-" 代替。例如:

```
:echo getfperm("/etc/passwd")
:echo getfperm(expand("~/.vimrc"))
```

希望它会（从安全角度而言）显示字符串 "rw-r--r--" 或者甚至 "rw-------"。

也可用作 method :

```
GetFilename()->getfperm()
```

要设置权限，可用 setfperm() 。

getfsize({fname}) getfsize()

返回数值，文件 {fname} 以字节数计算的大小。
如果 {fname} 是目录，返回 0。
如果找不到文件 {fname}，返回 -1。
如果 {fname} 文件过大，超出了 Vim 的数值的范围，返回 -2。

也可用作 method :
GetFilename()->getfsize()

getftime({fname}) getftime()
返回数值，给定文件 {fname} 的最新修改时间。该时间为 1970 年 1 月 1 日开始计算的秒数，可以传给 strftime()。
另见 localtime() 和 strftime() 。
如果找不到文件 {fname}，返回 -1。

也可用作 method :
GetFilename()->getftime()

getftype({fname}) getftype()
返回字符串，给定文件 {fname} 的文件类型的描述。
如果 {fname} 不存在，返回空字符串。
下表列出各种不同文件类型的返回值：

普通文件	"file"
目录	"dir"
符号链接	"link"
块设备	"bdev"
字符设备	"cdev"
套接字	"socket"
FIFO	"fifo"
其它	"other"

例如：
getftype("/home")
注意 只有在能支持的系统上才会返回 "link" 这样的类型。有的系统只支持 "dir" 和 "file"。MS-Windows 上目录的符号链接返回 "dir" 而不是 "link"。

也可用作 method :
GetFilename()->getftype()

getimstatus() getimstatus()
返回数值，IME 状态激活时为 TRUE 。
见 'imstatusfunc'。

getjumplist([{winnr} [, {tabnr}]]) getjumplist()
返回指定窗口的 jumplist 。

无参数时使用当前窗口。
如果只带 {winnr}，使用当前标签页的指定窗口。
{winnr} 也可以是 window-ID 。
有 {winr} 和 {tabnr} 则返回指定标签页的指定窗口。

返回列表包含两个项目：包含跳转位置的列表以及在该列表中的最近使用的跳转位置号。跳转位置列表的每个项目是包含以下内容的字典：

bufnr	缓冲区号
col	列号
coladd	用于 'virtualedit' 的列偏移
filename	文件名，如有的话
lnum	行号

也可用作 method :
```
GetWinnr()->getjumplist()
```

getline()

getline({lnum} [, {end}])
如果没有 {end}，返回字符串，即当前缓冲区第 {lnum} 行文本。
例如:
```
getline(1)
```
如果 {lnum} 是不以数字开始的字符串，调用 line() 来把该字符串转化成数值。要得到光标所在的行:
```
getline(".")
```
如果 {lnum} 小于 1 或者大于缓冲区的总行数，返回空字符串。

如果给出 {end}，返回 List ，其中每个项目是当前缓冲区从 {lnum} 到 {end} (包含) 范围的一行。
{end} 的用法同 {lnum}。
安静地忽略不存在的行。
如果 {end} 在 {lnum} 之前，返回空 List 。
例如:
```
:let start = line('.')
:let end = search("^$") - 1
:let lines = getline(start, end)
```

也可用作 method :
```
ComputeLnum()->getline()
```

要获取其它缓冲区的文本行，见 getbufline() 。

getloclist({nr} [, {what}]) getloclist()
返回列表，包含窗口 {nr} 的位置列表的所有项目。{nr} 可以是窗口号或 window-ID 。如果 {nr} 为 0，使用当前窗口。

如果是位置列表窗口，返回其显示的位置列表。如果窗口号 {nr} 非法，返回空列表。其它的情况和 getqflist() 相同。

如果提供了可选的 {what} 字典参数，以字典形式返回 {what} 列出的项目。关于 {what} 支持的项目，可参考 getqflist() 。

除了 getqflist() 的 {what} 支持的项目以外， getloclist() 还支持以下项目:

filewinid	用于显示来自位置列表的文件的窗口的 id。只有当调用来自位置列表窗口时此域才可用。详见 location-list-file-window 。

getmatches([{win}]) getmatches()
返回之前 matchadd() 和 :match 命令为当前窗口定义的所有匹配组成的列表 List 。 getmatches() 常和 setmatches() 组合使用，因为 setmatches() 可以恢复 getmatches() 保存的匹配列表。
示例:
```
:echo getmatches()
[{'group': 'MyGroup1', 'pattern': 'TODO',
```

```
'priority': 10, 'id': 1}, {'group': 'MyGroup2',
'pattern': 'FIXME', 'priority': 10, 'id': 2}]
:let m = getmatches()
:call clearmatches()
:echo getmatches()
[]
:call setmatches(m)
:echo getmatches()
[{'group': 'MyGroup1', 'pattern': 'TODO',
'priority': 10, 'id': 1}, {'group': 'MyGroup2',
'pattern': 'FIXME', 'priority': 10, 'id': 2}]
:unlet m
```

getmousepos()

getmousepos()

返回鼠标最近已知位置的字典。可用于鼠标点击的映射或弹出窗口的过滤。项目是：

screenrow	屏幕行
screencol	屏幕列
winid	点击所在的窗口 ID
winrow	"winid" 里的行
wincol	"winid" 里的列
line	"winid" 里的文本内容行
column	"winid" 里的文本列

所有的数值从 1 开始。

如果鼠标不在窗口上，比如在命令行里，则只有 "screenrow" 和 "screencol" 是合法的，其它的值均为零。

如果鼠标在窗口底下的状态行上或窗口右侧的垂直分割线上，"line" 和 "column" 的值为零。

如果位置在文本之后，那么 "column" 是文本以字节计的长度。

如果鼠标在弹出窗口上，使用该窗口。

使用 getchar() 时，Vim 变量 v:mouse_lnum 、 v:mouse_col 和 v:mouse_winid 也能提供相应的这些值。

getpid()

getpid()

返回数值，即 Vim 进程的进程号。Unix 和 MS-Windows 上这是个唯一的数值，直到 Vim 退出为止。MS-DOS 上该值总为零。

getpos()

getpos({expr})

得到 {expr} 的位置。可用的 {expr} 的值见 line() 。要得到光标位置，见 getcurpos() 。
返回 List ，包含四个数值：

```
[bufnum, lnum, col, off]
```

"bufnum" 为零，除非使用了 '0 或 'A 这样的位置标记，这时它是此位置标记所在的缓冲区号。
"lnum" 和 "col" 是缓冲区里的位置。第一列为 1。
除非使用了 'virtualedit'。"off" 值为零，这是从对应字符开始位置的以屏幕列计的位移。例如，在制表之中或最后一个字符之后的某个位置。

注意 '< 和 '> 和可视模式有关："V"（可视行模式）时 '< 的列为零，'> 的列为一大数。
可以用来保存和恢复光标位置：

```
let save_a_mark = getpos("'a")
...
call setpos("'a", save_a_mark)
```

另见 getcurpos() 和 setpos() 。

也可用作 method ：

```
GetMark()->getpos()
```

getqflist([{what}]) getqflist()
返回列表，包含所有当前 quickfix 错误。列表的每个项目是包含以下项目的字典：

bufnr	有此文件名的缓冲区号，bufname() 得到缓冲区名
module	模块名
lnum	缓冲区里的行号（第一行是 1）
col	列号（第一列是 1）
vcol	TRUE : "col" 是可视列 FALSE : "col" 是字节位置
nr	错误号
pattern	用于定位错误的搜索模式
text	错误描述
type	错误类型，'E'、'1' 等。
valid	TRUE : 能识别该错误信息

如果没有错误列表或者它为空，返回空列表。quickfix 列表项目如果包含不存在的缓冲区号，返回的 "bufnr" 置为零。

应用：在多个文件里寻找模式的匹配，并对之进行处理：

```
:vimgrep /theword/jg *.c
:for d in getqflist()
:   echo bufname(d.bufnr) ':' d.lnum '=' d.text
:endfor
```

如果提供了可选的 {what} 字典参数，以字典形式返回 {what} 列出的项目。{what} 支持以下字符串项目：

changedtick	quickfix-changedtick 列表改变的总数
context	获取 quickfix-context
efm	解析 "lines" 时使用的 errorformat。如果没有给出，使用 'errorformat' 选项值。
id	获取 quickfix 列表 quickfix-ID 说明的信息。零代表当前列表或 "nr" 指定的列表
idx	由 'id' 或 'nr' 指定的 quickfix 列表中当前项目的索引。 见 quickfix-index
items	获取 quickfix 列表的项目
lines	用 'efm' 解析文本行列表并返回其结果项目。只支持 List 类型。不修改当前 quickfix 列表。见 quickfix-parse 。
nr	获取关于指定 quickfix 列表的信息；零代表当前 quickfix 列表，"$" 代表最后的 quickfix 列表
qfbufnr	quickfix 窗口显示的缓冲区号。quickfix 缓冲区不

存在时返回 0。见 quickfix-buffer 。
size quickfix 列表的项目总数
title 获取列表标题 quickfix-title
winid 获取 quickfix window-ID
all 上述所有的 quickfix 属性
忽略 {what} 中非字符串项。要获取某个特定项之值，先设为零。
如果不给出 "nr"，使用当前 quickfix 列表。
同时给出 "nr" 和非零 "id" 时，使用 "id" 指定的列表。
要取得 quickfix 栈的列表数目，在 {what} 中设置 'nr' 为 "$"。返回字典中的 "nr" 值会告诉你 quickfix 的栈大小。
如果给出 "lines"，忽略除 "efm" 外的所有其它项。返回的字典包含 "items" 项，给出所有项目的列表。

返回字典包含以下项目:
changedtick quickfix-changedtick 列表改变的总数
context quickfix 列表的上下文，见 quickfix-context 如果不存在，设为 ""。
id quickfix 列表 ID quickfix-ID 。如果不存在，设为 0。
idx 列表中当前项目的索引。如果不存在，设为 0。
items quickfix 列表的项目。如果不存在，设为空列表。
nr quickfix 列表号。如果不存在，设为 0
qfbufnr quickfix 窗口显示的缓冲区号。如果不存在，设为 0。
size quickfix 列表的项数。如果不存在，设为 0。
title quickfix 列表标题文本。如果不存在，设为 ""。
winid quickfix window-ID 。如果不存在，设为 0

示例 (另见 getqflist-examples):
```
:echo getqflist({'all': 1})
:echo getqflist({'nr': 2, 'title': 1})
:echo getqflist({'lines' : ["F1:10:L10"]})
```

getreg([{regname} [, 1 [, {list}]]]) getreg()
返回字符串，寄存器 {regname} 的内容。例如:
```
:let cliptext = getreg('*')
```
如果 {regname} 对应的寄存器没有设过，返回空串。

getreg('=') 返回最近一次表达式寄存器计算的返回值 (用于映射)。
getreg('=', 1) 返回表达式自身，以便用 setreg() 恢复。对于其它寄存器，这个额外的参数被忽略，所以给出此参数总是无妨。

如果 {list} 给出且为 TRUE ，改变返回值为列表。每个列表项是一个文本行。如果你在意寄存器内可能存在的零字节，可以用这种形式:
不用第三个参数时 NL 和零字节都用 NL 表示 (见 NL-used-for-Nul) 。
如果 {regname} 对应的寄存器没有设过，返回空列表。

如果没有指定 {regname}，使用 v:register 。

也可用作 method :
```
GetRegname()->getreg()
```

getregtype([{regname}]) getregtype()

返回字符串，寄存器 {regname} 的类型。
该值会是以下可能之一：

"v"	characterwise （面向字符）的文本
"V"	linewise （面向行）的文本
"<CTRL-V>{width}"	blockwise-visual （面向列块）的文本
""	空或者未知的寄存器

<CTRL-V> 是一个字符，其值为 0x16。
如果没有指定 {regname}，使用 v:register 。

也可用作 method ：

```
GetRegname()->getregtype()
```

gettabinfo([{arg}]) gettabinfo()

如果没指定 {arg}，返回列表，包含所有标签页的信息。每个列表项是字典。
否则，{arg} 指定标签页号，返回关于该标签页的信息。如果该标志页不存在，返回空列表。

每个列表项是包含以下项目的字典：

tabnr	标签页号
variables	包含标签页局部变量的字典的引用
windows	标签页中的 window-ID 列表。

也可用作 method ：

```
GetTabnr()->gettabinfo()
```

gettabvar({tabnr}, {varname} [, {def}]) gettabvar()

得到标签页 {tabnr} 的标签页局部变量 {varname} 的值。 t:var
标签页的编号从一开始。
如果 {varname} 为空，返回包含所有标签页局部变量的字典。
注意 必须使用不带 "t:" 的名字。
如果标签页或者变量不存在，返回 {def} 或空字符串。不会有错误消息。

也可用作 method ：

```
GetTabnr()->gettabvar(varname)
```

gettabwinvar({tabnr}, {winnr}, {varname} [, {def}]) gettabwinvar()

得到标签页 {tabnr} 的窗口 {winnr} 的窗口局部变量 {varname} 的值。
如果 {varname} 为 "&"，返回包含所有窗口局部选项的字典。
否则，如果 {varname} 以 "&" 打头，得到窗口局部选项的值。
如果 {varname} 为空，返回一个包含所有窗口局部变量的字典。
注意 {varname} 必须为不带 "w:" 的名字。
标签页的编号从一开始。当前标签页可用 getwinvar() 。
{winnr} 可以是窗口号或 window-ID 。
如果 {winnr} 为零，使用当前窗口。
也可用于全局或者局部于缓冲区或局部于窗口的选项，但不能用于全局或者局部于缓冲区的变量。
如果标签页、窗口或者变量不存在，返回 {def} 或空字符串。不会有错误消息。
例如：

```
:let list_is_on = gettabwinvar(1, 2, '&list')
:echo "myvar = " . gettabwinvar(3, 1, 'myvar')
```

要得到所有窗口局部变量，可用：

```
gettabwinvar({tabnr}, {winnr}, '&')
```

也可用作 method ：

```
GetTabnr()->gettabwinvar(winnr, varname)
```

gettagstack([{nr}]) gettagstack()

返回字典，窗口 {nr} 的标签栈。
{nr} 可以是窗口号或 window-ID 。
如果没有给出 {nr}，使用当前窗口。
如果窗口 {nr} 不存在，返回空字典。

返回字典包含如下项目：

curidx	栈的当前索引。如果在栈顶，设为 (length + 1)。栈底的索引为 1。
items	栈中的项目列表。每个项目是包含下述项的字典。
length	栈中的项目数。

栈中的每个项目是以下项的字典：

bufnr	当前跳转的缓冲区号
from	标签跳转之前的光标位置。 getpos() 说明返回列表的格式。
matchnr	当前的匹配标签号。用于同一名字有多个匹配标签的时候。
tagname	标签名

tagstack 说明标签栈的更多信息。

也可用作 method ：

```
GetWinnr()->gettagstack()
```

getwininfo([{winid}]) getwininfo()

返回窗口信息，返回值是字典的列表。

如果给出 {winid}，返回给定 ID 的窗口的信息。如果窗口不存在，返回空列表。

如果没有给出 {winid}，返回所有标签页的所有窗口的信息。

每个列表项是包含以下项目的字典：

botline	最近显示的缓冲区行
bufnr	窗口中的缓冲区号
height	窗口高度（不计窗口工具条 winbar）
loclist	1 如果显示了位置列表 {仅当加入 +quickfix 特性才有效}
quickfix	1 如果是 quickfix 或位置列表窗口 {仅当加入 +quickfix 特性才有效}
terminal	1 如果是终端窗口 {仅当加入 +terminal 特性才有效}
tabnr	标签页号

topline	首个显示的缓冲区行
variables	包含窗口局部变量的字典的引用
width	窗口宽度
winbar	1 如果窗口有工具条，0 反之
wincol	窗口最左侧屏幕列，win_screenpos() 中的 col
winid	window-ID
winnr	窗口号
winrow	窗口最顶侧屏幕行，win_screenpos() 中的 row‘

也可用作 method：

```
GetWinnr()->getwininfo()
```

getwinpos([{timeout}]) getwinpos()

返回两个数值组成的列表，即 getwinposx() 和 getwinposy() 结果的混合：

```
[x-pos, y-pos]
```

{timeout} 可用于以毫秒计的等待终端反馈的超时。省略时缺省为 100 毫秒。远程终端可用更长的超时。

如果取值小于 10 且在指定时限内没有收到反馈，返回如果有的上次报告的位置。可用于轮询位置并在同时并发做其它的事：

```
while 1
  let res = getwinpos(1)
  if res[0] >= 0
    break
  endif
  " 干些话
endwhile
```

也可用作 method：

```
GetTimeout()->getwinpos()
```

getwinposx() getwinposx()

返回数值，即 GUI Vim 窗口以像素计从左起算的 X 坐标。也适用于 xterm（100 毫秒超时）。

如果该信息得不到，返回 -1。

返回值可用于 ":winpos"。

getwinposy() getwinposy()

返回数值，即 GUI Vim 窗口以像素计从顶部起算的 Y 坐标。也适用于 xterm（100 毫秒超时）。

如果该信息得不到，返回 -1。

返回值可用于 ":winpos"。

getwinvar({winnr}, {varname} [, {def}]) getwinvar()

类似于 gettabwinvar()，只用当前标签页。

例如：

```
:let list_is_on = getwinvar(2, '&list')
:echo "myvar = " . getwinvar(1, 'myvar')
```

也可用作 method：

```
GetWinnr()->getwinvar(varname)
```

glob({expr} [, {nosuf} [, {list} [, {alllinks}]]]) glob()

扩展 {expr} 里的文件通配符。 wildcards 说明其中特殊字符的使用方法。

除非给出可选的 {nosuf} 参数且为 TRUE ，应用 'suffixes' 和 'wildignore' 选项：跳过匹配任何 'wildignore' 模式的名字，而 'suffixes' 影响匹配结果的排序。
'wildignorecase' 则总是适用。

如果给出 {list} 且为 TRUE ，返回列表，包含所有匹配的文件。使用列表的优点是可以正确得到包含换行符的文件名。
否则返回的是字符串，且如果返回多个匹配，以 <NL> 字符分隔。

如果扩展失败，返回空字符串或空列表。

扩展结果不包含不存在文件的名字。仅当符号链接指向已存在文件时才会包含在扩展结果内。但若给出了 {alllinks} 参数且为 TRUE ，则包含所有符号链接。

多数系统上，可以用反引号从外部命令得到文件名。例如：

```
:let tagfiles = glob("`find . -name tags -print`")
:let &tags = substitute(tagfiles, "\n", ",", "g")
```

反引号包围的程序的输出结果必须每个项目一行。项目内部可以使用空格。

特殊 Vim 变量的扩展见 expand() 。 system() 说明如何得到外部命令的原始输出。

也可用作 method ：

```
GetExpr()->glob()
```

glob2regpat({expr}) glob2regpat()

转换 glob() 所用有文件模式为搜索模式。结果可用来匹配包含文件名的字符串。例如

```
if filename =~ glob2regpat('Make*.mak')
```

等价于：

```
if filename =~ '^Make.*\.mak$'
```

如果 {expr} 为空串，返回值是 "^$"，匹配空串。

备注 结果与所用系统有关。MS-Windows 上反斜杠通常用作路径分隔符。

也可用作 method ：

```
GetExpr()->glob2regpat()
```

globpath({path}, {expr} [, {nosuf} [, {list} [, {alllinks}]]]) globpath()

在 {path} 的所有目录下的 {expr} 执行 glob() 并连接所有的返回结果。例如：

```
:echo globpath(&rtp, "syntax/c.vim")
```

{path} 是逗号分隔的目录名的列表。每个目录名都附加在 {expr} 之前，然后如同 glob() 那样被扩展。必要的话，插入路径分隔符。
要在目录名字里加上逗号，可以使用反斜杠转义。注意 在 MS-Windows

上目录的最后可能有一个反斜杠。如果你要在后面加上逗号进行分隔，先把反斜杠去掉。
如果某个目录下的扩展失败，不会有错误信息。

除非给出可选的 {nosuf} 参数且为 TRUE ，应用 'suffixes' 和 'wildignore' 选项：跳过匹配任何 'wildignore' 模式的名字，而 'suffixes' 影响匹配结果的排序。

如果给出 {list} 且为 TRUE ，返回匹配文件的列表。列表的好处是会正确处理包含换行符的文件名，否则返回字符串，有多个匹配时，以 <NL> 字符分隔。例如：

```
:echo globpath(&rtp, "syntax/c.vim", 0, 1)
```

{alllinks} 的用法和 glob() 相同。

可以用 "**" 项目来搜索目录树。例如，寻找在 'runtimepath' 和它之下所有目录里的 "README.txt" 文件：

```
:echo globpath(&rtp, "**/README.txt")
```

不支持向上搜索和 "**" 的深度限制，所以 'path' 的使用不一定总能正确工作。

也可用作 method ，基是作为第二个参数传递的：

```
GetExpr()->globpath(&rtp)
```

has()

has({feature}) 返回数值，如果支持特性 {feature} 则为 1，不然为零。
{feature} 参数是字符串。见下面的 feature-list 。
另见 exists() 。

has_key({dict}, {key}) has_key()

返回数值，如果 Dictionary {dict} 有键为 {key} 的项目则为 1，不然为零。

也可用作 method :

```
mydict->has_key(key)
```

haslocaldir([{winnr} [, {tabnr}]]) haslocaldir()

返回数值：

1	如果窗口用 :lcd 设置过局部目录
2	如果标签页用 :tcd 设置过局部目录
0	其它。

无参数时使用当前窗口。
有 {winr} 则使用当前标签页的该窗口。
有 {winr} 和 {tabnr} 则使用指定标签页的该窗口。
{winr} 可以是窗口号或 window-ID 。
如果 {winnr} 为 -1，它被忽略，只使用标签页。
如果参数非法，返回 0。
Examples:

```
if haslocaldir() == 1
  " 窗口的局部目录情况
elseif haslocaldir() == 2
  " 标签页的局部目录情况
```

```
else
  " 全局目录情况
endif

" 当前窗口
:echo haslocaldir()
:echo haslocaldir(0)
:echo haslocaldir(0, 0)
" 当前标签页的窗口 n
:echo haslocaldir(n)
:echo haslocaldir(n, 0)
" 标签页 m 的窗口 n
:echo haslocaldir(n, m)
" 标签页 m
:echo haslocaldir(-1, m)
```

也可用作 method :

```
GetWinnr()->haslocaldir()
```

hasmapto({what} [, {mode} [, {abbr}]]) hasmapto()

返回数值，如果存在某映射，其右边的表达式（被映射到的部分）的某处包含 {what}，并且该映射在 {mode} 指定的模式下存在，返回 1。如果提供 {abbr} 且为 TRUE ，使用缩写而不是映射。不要忘记指定插入和/或命令行模式。
同时检查全局映射和局部于当前缓冲区的映射以寻找匹配。
如果没有匹配的映射，返回 0。
{mode} 识别下列字符：

n	普通模式
v	可视和选择模式
x	可视模式
s	选择模式
o	操作符等待模式
i	插入模式
l	Language-Argument ("r"、 "f"、"t" 等等) 模式
c	命令行模式

如果没有提供 {mode}，使用 "nvo"。

该函数可用于检查是否存在映射到 Vim 脚本的某个函数的映射。例如：

```
:if !hasmapto('\ABCdoit')
:   map <Leader>d \ABCdoit
:endif
```

这样，到 "\ABCdoit" 的映射只有在到 "\ABCdoit" 的映射还不存在的时候才会进行。

也可用作 method :

```
GetRHS()->hasmapto()
```

histadd({history}, {item}) histadd()

把字符串 {item} 加到历史 {history} 里。后者可以是：

hist-names

"cmd"	或 ":"	命令行历史
"search"	或 "/"	搜索模式历史
"expr"	或 "="	输入表达式历史

"input" 或 "@" 输入行历史
"debug" 或 ">" 调试命令历史
空 当前或最后使用的历史
{history} 字符串无须是完整名字，一个字符就够了。
如果 {item} 已经在历史里存在，它会被调整位置，从而成为最新的一项。
返回结果为数值：如果操作成功则为 1，不然返回 0。

例如：
```
:call histadd("input", strftime("%Y %b %d"))
:let date=input("Enter date: ")
```
该函数在沙盘里不可用 sandbox 。

也可用作 method ，基是作为第二个参数传递的：
```
GetHistory()->histadd('search')
```

histdel({history} [, {item}]) histdel()
清除 {history}，换而言之，删除它所有的项目。 hist-names 解释 {history} 的所有可能值。

如果 {item} 计算结果为字符串，它被看作正规表达式。从历史里删除所有匹配该模式的项目（如果有的话）。
必须匹配大小写，除非使用 "\c" /\c 。
如果 {item} 的计算结果为数值，它被解释为索引值，见 :history-indexing 。如果该索引存在，删除相应的项目。

返回结果为数值：如果操作成功则为 1，不然返回 0。

例如：
清除表达式寄存器历史：
```
:call histdel("expr")
```

删除所有 "*" 开始的搜索历史：
```
:call histdel("/", '^\*')
```

下面三者是等价的：
```
:call histdel("search", histnr("search"))
:call histdel("search", -1)
:call histdel("search", '^'.histget("search", -1).'$')
```

要删除最后的搜索模式，并在 "n" 命令和 'hlsearch' 里使用倒数第二个模式：
```
:call histdel("search", -1)
:let @/ = histget("search", -1)
```

也可用作 method ：
```
GetHistory()->histdel()
```

histget({history} [, {index}]) histget()
返回字符串，即 {history} 历史的第 {index} 项。 hist-names 解释 {history} 的所有可能值，而 :history-indexing 解释 {index}。如果没有这个项目，返回空字符串。如果忽略 {index}，返回历史里最近使用的项目。

例如：
重做历史里的倒数第二个搜索

```
:execute '/' . histget("search", -2)
```

定义 Ex 命令 ":H {num}"，以重新执行 :history 输出的第 {num} 项。

```
:command -nargs=1 H execute histget("cmd", 0+<args>)
```

也可用作 method：

```
GetHistory()->histget()
```

histnr({history}) histnr()
返回当前项目在 {history} 里的编号。hist-names 解释 {history} 的所有可能值。
如果有错，返回 -1。

例如：

```
:let inp_index = histnr("expr")
```

也可用作 method：

```
GetHistory()->histnr()
```

hlexists({name}) hlexists()
返回数值。只要名为 {name} 的高亮组用某种方法定义过，返回非零。不一定要为该组定义过高亮属性。一些语法项目可能已经使用该组。
highlight_exists()
已废弃的名字：highlight_exists()。

也可用作 method：

```
GetName()->hlexists()
```

hlID({name}) hlID()
返回数值，即名为 {name} 的高亮组的 ID。如果该高亮组不存在，返回零。
可用于提取高亮组的信息。比如，要得到 "Comment" 组的背景颜色：

```
:echo synIDattr(synIDtrans(hlID("Comment")), "bg")
```

highlightID()
已废弃的名字：highlightID()。

也可用作 method：

```
GetName()->hlID()
```

hostname() hostname()
返回字符串，即 Vim 运行的机器名字。超过 256 字符串长度的机器名被截短。

iconv({expr}, {from}, {to}) iconv()
返回字符串，即文本 {expr} 从 {from} 编码转到 {to} 编码以后的文本。
如果转换完全失败，返回空字符串。如果部分字符无法转换，以 "?" 代替之。
编码名字可以是任何 iconv() 库函数接受的名字，见 ":!man 3 iconv"。
大多数转换需要 Vim 编译时加入 +iconv 特性。不然，只支持

UTF-8 和 latin1 的相互转换。
这可以用来显示包含特殊字符的消息。不管 'encoding' 设为何值，总可以用 UTF-8 书写消息，然后使用：

```
echo iconv(utf8_str, "utf-8", &enc)
```

注意 Vim 使用 UTF-8 进行所有的 Unicode 编码，从/到 UCS-2 的转换都自动转为 UTF-8。你不能在字符串里使用 UCS-2，因为那里有 NUL 字节。

也可用作 method：

```
GetText()->iconv('latin1', 'utf-8')
```

indent()

indent({lnum}) 返回数值，第 {lnum} 行的缩进距离。缩进的计算以空格计，因而它和 'tabstop' 的值是有关系的。{lnum} 的使用方式和 getline() 相同。
如果 {lnum} 非法，返回 -1。

也可用作 method：

```
GetLnum()->indent()
```

index({object}, {expr} [, {start} [, {ic}]]) index()
如果 {object} 为 List，返回值等于 {expr} 的最小项目索引。这里不进行自动转换，字符串 "4" 不同于数值 4，数值 4 也不等同于浮点数 4.0。'ignorecase' 的值此处也不适用，大小写不忽略。

如果 {object} 为 Blob，返回字节值等于 {expr} 的最小索引。

如果给出 {start}，从索引为 {start} 的项目开始寻找（可以为负，指定相对于尾部的项目）。
如果给出 {ic} 且为 TRUE，忽略大小写。否则，必须匹配大小写。
如果在 {object} 里找不到 {expr}，返回 -1。
示例：

```
:let idx = index(words, "the")
:if index(numbers, 123) >= 0
```

也可用作 method：

```
GetObject()->index(what)
```

input({prompt} [, {text} [, {completion}]]) input()
返回字符串，即用户在命令行上的输入内容，可以为任何值。参数 {prompt} 或者是一个提示字符串，或者是一个空白字符串（没有提示）。'\n' 可以在提示里使用，以开始新行。
该提示使用 :echohl 设置的高亮。
输入方法和命令行相似，也使用相同的编辑命令和映射。但 input() 输入的行使用另外的历史。
示例：

```
:if input("咖啡还是啤酒? ") == "啤酒"
:  echo "干杯！"
:endif
```

如果给出可选的 {text} 参数，它被用作缺省的回答，就像是用户输入的那样。例如：

```
:let color = input("Color? ", "white")
```

可选的 {completion} 参数指定输入支持的补全类型。如果不给出，不使用补全。支持的补全类型和用户定义命令用 "-complete=" 参数能给出的类型相同。详情见 :command-completion 。例如:

```
let fname = input("File: ", "", "file")
```

注意: 在只能运行于 GUI 模式的版本里 (比如 Win32 GUI)，此函数不能在启动文件里使用。

注意: input() 在映射里调用时，它会消耗该映射余下的字符，因为映射的处理就像那些字符被键盘输入一样。在 input() 前使用 inputsave() 然后在 input() 输入之后 inputrestore() 可以避免这一点。另一个方法是避免在映射的后面提供任何字符，比如，使用 :execute 或 :normal 。

使用映射的例子:

```
:nmap \x :call GetFoo()<CR>:exe "/" . Foo<CR>
:function GetFoo()
:  call inputsave()
:  let g:Foo = input("enter search pattern: ")
:  call inputrestore()
:endfunction
```

也可用作 method :

```
GetPrompt()->input()
```

inputdialog({prompt} [, {text} [, {cancelreturn}]]) inputdialog()

类似于 input() ，但如果运行 GUI 且支持文本对话框，弹出一个对话框窗口来输入文本。

例如:

```
:let n = inputdialog("value for shiftwidth", shiftwidth())
:if n != ""
:  let &sw = n
:endif
```

如果对话框被取消，返回 {cancelreturn}。如果忽略，返回空字符串。

输入 <Enter> 和按 OK 按钮相同。按 <Esc> 和按 Cancel 按钮相同。

备注: 不支持命令行补全。

也可用作 method :

```
GetPrompt()->inputdialog()
```

inputlist({textlist}) inputlist()

{textlist} 必须是字符串的 List 。显示此 List ，每个字符串一行。用户得到提示要输入一个数值，返回此值。

用户也可以用鼠标点击项目来进行选择。第一个字符串返回 0。在第一个项目之上点击返回负数。在提示行上点击返回 {textlist} 的长度加一。

确保 {textlist} 不超过 'lines' 个项目，否则无法使用。建议把项目编号放在每个字符串的开始处，并在第一项上加上提示。例如:

```
let color = inputlist(['Select color:', '1. red',
      \ '2. green', '3. blue'])
```

也可用作 method :

```
GetChoices()->inputlist()
```

inputrestore() inputrestore()

恢复前一个 inputsave() 保存的预输入。应该和 inputsave() 调用的次数相同，不过调用更多次也无妨。
如果没有可以恢复的，返回 1，不然返回 0。

inputsave() inputsave()

保存预输入（也包括映射的）并清除之，使得下一个提示能从用户得到输入。在提示之后应该跟上配套的 inputrestore()。可以多次使用，此时应该有同样多次的 inputrestore() 调用。
如果内存不足，返回 1，不然返回 0。

inputsecret({prompt} [, {text}]) inputsecret()

该函数和 input() 函数类似，但有两个例外：
a) 用户的应答显示为一串星号 ("*")，从而输入可以保密，还有
b) 用户的应答不会记录在输入 history 栈中。
返回字符串，即用户在命令行上根据提示输入的应答。
备注：不支持命令行补全。

也可用作 method：

```
GetPrompt()->inputsecret()
```

insert({object}, {item} [, {idx}]) insert()

如果 {object} 为 List 或 Blob，在开始处插入 {item}。

如果指定 {idx}，{item} 的插入位置在索引 {idx} 之前。如果 {idx} 为零，插入在第一个项目之前，和省略 {idx} 效果相同。也可用负的 {idx}，见 list-index。-1 插入在最后一个项目之前。
返回新产生的 List 或 Blob。例如：

```
:let mylist = insert([2, 3, 5], 1)
:call insert(mylist, 4, -1)
:call insert(mylist, 6, len(mylist))
```

用 add() 可以更简单的完成最后一个例子。
注意 如 {item} 是 List，它被作为单个项目来插入。 extend() 用来连接多个 List。

也可用作 method：

```
mylist->insert(item)
```

interrupt() interrupt()

中断脚本的执行。它或多或少像用户按了 CTRL-C 一样。多数命令不再执行，控制权还给用户。可用于中止较底层如自动命令等的执行。
如：

```
:function s:check_typoname(file)
:   if fnamemodify(a:file, ':t') == '['
:       echomsg 'Maybe typo'
:       call interrupt()
:   endif
:endfunction
:au BufWritePre * call s:check_typoname(expand('<amatch>'))
```

invert({expr}) invert()

按位取反。参数须转换为数值。列表、字典或浮点数参数会报错。
示例：

```
:let bits = invert(bits)
```

也可用作 method :

```
:let bits = bits->invert()
```

isdirectory({directory}) isdirectory()

返回数值，如果名为 {directory} 的目录存在，返回 TRUE 。如果 {directory} 不存在或者不是目录，返回 FALSE 。{directory} 可以是任何表达式，最终用作字符串。

也可用作 method :

```
GetName()->isdirectory()
```

isinf({expr}) isinf()

如果 {expr} 是正无穷大返回 1，负无穷大返回 -1，否则返回 0。

```
:echo isinf(1.0 / 0.0)
1
:echo isinf(-1.0 / 0.0)
-1
```

也可用作 method :

```
Compute()->isinf()
```

{仅当编译时加入 +float 特性才有效}

islocked({expr}) islocked() E786

返回数值，如果 {expr} 是某个加锁的变量名，返回 TRUE 。{expr} 必须是变量名、 List 项目，或 Dictionary 项目，不是变量本身！例如:

```
:let alist = [0, ['a', 'b'], 2, 3]
:lockvar 1 alist
:echo islocked('alist')         " 1
:echo islocked('alist[1]')      " 0
```

如果 {expr} 是不存在的变量，得到错误信息。用 exists() 可以检查它是否存在。

isnan({expr}) isnan()

如果 {expr} 是值为 NaN 的浮点数，返回 TRUE 。

```
echo isnan(0.0 / 0.0)
1
```

也可用作 method :

```
Compute()->isnan()
```

{仅当编译时加入 +float 特性才有效}

items({dict}) items()

返回 List ，{dict} 的所有键-值组对。每个 List 项目是两个项目的列表: {dict} 项目的键和此项目的值。 List 项目的顺序不定。另见 keys() 和 values() 。
示例:

```
for [key, value] in items(mydict)
   echo key . ': ' . value
```

```
                endfor

            也可用作 method :
                mydict->items()

job_ 函数文档在这里: job-functions-details


join({list} [, {sep}])                                          join()
            连接所有 {list} 项目成为字符串。
            如果指定 {sep}，该分隔符出现在项目之间。如果省略 {sep}，用单个
            空格。
            注意 尾部不加 {sep}。如果你坚持要加入:
                let lines = join(mylist, "\n") . "\n"
            字符串项目照原样使用。用类似 string() 的方式把 List 和
             Dictionary 转化为字符串。
            逆函数是 split() 。

            也可用作 method :
                mylist->join()

js_decode({string})                                             js_decode()
            和 json_decode() 类似，但有以下区别:
            - 对象键名不需用引号括起。
            - 字符串可以用单引号括起。
            - 接受数组的空项目 (两个逗句之间)，返回 v:none 项目。

            也可用作 method :
                ReadObject()->js_decode()

js_encode({expr})                                               js_encode()
            和 json_encode() 类似，但有以下区别:
            - 对象键名不用引号括起。
            - 数组里的 v:none 项目生成逗号之间的空项目。
            例如，Vim 对象:
                [1,v:none,{"one":1},v:none]
            会被编码为:
                [1,,{one:1},,]
            而 json_encode() 会生成:
                [1,null,{"one":1},null]
            对 Javscript，这种编码也是合法的。但比 JSON 更高效，尤其是使用
            有可选项目的数组时。

            也可用作 method :
                GetObject()->js_encode()

json_decode({string})                                           json_decode()
            解析 JSON 格式的字符串，返回等价的 Vim 值。见 json_encode()
            了解 JSON 和 Vim 值的关系。
            解码是容错的:
            - 忽略数组和对象拖尾的逗号，例如 "[1, 2, ]" 等同 "[1, 2]"。
            - 对象中接受整数键，如 {1:2} 和 {"1":2} 相同。
            - 识别更多浮点数，例如 "1." 相当于 "1.0"，而 "001.2" 相当于
              "1.2"。接受特殊浮点值 "Infinity"、"-Infinity" 和 "NaN" (大小
```

写无关)。
- 忽略整数值前导零，例如 "012" 相当于 "12"，而 "-012" 相当于 "-12"。
- 按本义名 null、true 或 false 的大小写无关，例如 "NULL" 相当于 "null"，"True" 相当于 "true"。
- 接受字符串中不转义的控制字符 U+0000 到 U+001F，例如 " " (字符串中的制表符) 相当于 "\t"。
- 接受空或只有空白组成的 JSON 表达式，生成 v:none。
- 忽略非法的两字符转义序列中的反斜杠，例如 "\a" 解码为 "a"。
- JSON 字符串中正确的代理对正常应是 12 字符的序列，如 "\uD834\uDD1E"，但 json_decode() 安静地接受截断的代理对，例如 "\uD834" 或 "\uD834\u"。

E938

对象里的重复键值，虽然在 rfc7159 中合法，便不被 json_decode() 接受，因为转换结果必须是合法的 Vim 类型，例如，这样不行: {"a":"b", "a":"c"}

也可用作 method :
```
ReadObject()->json_decode()
```

json_encode({expr}) json_encode()

对 {expr} 进行 JSON 编码，返回字符串。
编码格式在此指定:
https://tools.ietf.org/html/rfc7159.html
Vim 值的转换如下:

Number	十进制数
Float	浮点数
Float nan	"NaN"
Float inf	"Infinity"
Float -inf	"-Infinity"
String	双引号括起 (可为 null)
Funcref	不接受，报错
List	作为数组 (可为 null); 若递归使用: []
Dict	作为对象 (可为 null); 若递归使用: {}
Blob	作为一个个字节组成的数组
v:false	"false"
v:true	"true"
v:none	"null"
v:null	"null"

备注 NaN 和 Infinity 作为值传递。这在 JSON 标准里没有提到，但若干实现支持。如果不支持，可能会报错。

也可用作 method :
```
GetObject()->json_encode()
```

keys({dict}) keys()

返回 List ，{dict} 的所有键。 List 项目的顺序不定。另见 items() 和 values() 。

也可用作 method :
```
mydict->keys()
```

len() E701

len({expr}) 返回数值，参数的长度。

如果 {expr} 为字符串或数值，返回它使用的字节数，和 strlen() 相同。
如果 {expr} 为 List ，返回 List 的项目数量。
如果 {expr} 为 Blob ，返回字节数。
如果 {expr} 为 Dictionary ，返回 Dictionary 的项目数量。
否则给出错误。

也可用作 method :
```
mylist->len()
```

libcall() E364 E368

libcall({libname}, {funcname}, {argument})
在运行库 {libname} 里调用函数 {funcname} 并给出单个参数 {argument}。
这可以用于调用库里的函数，尤其是 Vim 里用到的那些。因为只能使用单个参数，所以可以调用的标准库函数相当有限。
结果是函数返回的字符串。如果函数返回 NULL，在 Vim 里会以空字符串 "" 出现。
如果函数返回数值，请使用 libcallnr() ！
如果 {argument} 是数值，它以 int 类型传给函数；如果 {argument} 是字符串，它以 null 结尾的字符串类型传入。
在 restricted-mode 里，该函数不能运行。

libcall() 允许你写自己的 Vim ’插件’ 扩展，而无须重新编译程序。它并 不 是用来调用系统函数的一个方法！如果你试图这么做，Vim 很有可能会崩溃。

Win32 上，你写的函数必须在 DLL 里提供，而且必须使用普通的 C 调用惯例 (不是 Windows 系统 DLL 使用的 Pascal 惯例)。函数必须只能接受单个参数，或者是字符指针，或者是长整数，而且必须返回字符指针或者 NULL。返回的字符指针必须指向在函数返回之后仍然指向合法的内存 (比如 DLL 的静态区域)。如果指向分配的区域，那么内存会发生泄漏。在函数里使用静态缓冲区应该可以，在 DLL 卸载时会被释放。

警 告: 如果函数返回不合法的指针，Vim 会崩溃！如果函数返回数值也会发生同样的问题，因为 Vim 把它当作指针看待。
Win32 系统上，{libname} 必须是不带 ".DLL" 后缀的 DLL 文件名。只有 DLL 不在常见的位置的时候，才需要指定完整的路径名。
Unix 上: 如果编译你自己的插件，记住目标代码必须生成位置无关代码 (’PIC’)。
{仅当使用 Win32 和一些 Unix 版本且带有 +libcall 特性时才有效}
例如:
```
:echo libcall("libc.so", "getenv", "HOME")
```

也可用作 method ，基是作为第三个参数传递的:
```
GetValue()->libcall("libc.so", "getenv")
```

libcallnr()

libcallnr({libname}, {funcname}, {argument})
和 libcall() 类似，但函数返回 int，而不是字符串。
{仅当使用 Win32 和一些 Unix 版本且带有 +libcall 特性时才有效}

例如：

```
:echo libcallnr("/usr/lib/libc.so", "getpid", "")
:call libcallnr("libc.so", "printf", "Hello World!\n")
:call libcallnr("libc.so", "sleep", 10)
```

也可用作 method ，基是作为第三个参数传递的：

```
GetValue()->libcallnr("libc.so", "printf")
```

line({expr} [, {winid}]) line()

返回数值，即 {expr} 给定的文件位置的行号。可接受的位置是：

.	光标位置
$	缓冲区的最后一行
'x	位置标记 x 的位置（如果该位置标记没有设置，返回 0）
w0	当前窗口可见部分的首行（如果显示不刷新，如安静 Ex 模式下，则为一）
w$	当前窗口可见部分的末行（如果无行可见，返回比 w0 小一的值）
v	可视模式下：可视区域的开始行（光标是结束位置）。如果不在可视模式下，返回当前光标位置。和 '< 不同的是，会被立即更新。

注意 可以使用其它文件的位置标记。此时行号应用于那个缓冲区。
要得到列号用 col() 。两者都要可用 getpos() 。
给出可选的 {winid} 参数时，从该窗口取值而不是当前窗口。
例如：

```
line(".")               光标所在的行号
line(".", winid)        同上，但取自窗口 "winid"
line("'t")              位置标记 t 的行号
line("'" . marker)      名为 marker 的位置标记的行号
```

要在打开文件后跳转到最近已知的位置，见 last-position-jump 。

也可用作 method ：

```
GetValue()->line()
```

line2byte({lnum}) line2byte()

返回当前缓冲区第 {lnum} 行从缓冲区开始计算的字节数。这里包括换行符，但它具体的值取决于当前缓冲区的 'fileformat' 选项，第一行返回 1。这和 'encoding' 有关但忽略 'fileencoding'。
这也可以用来得到最后一行之后的 "那行" 的字节计数：

```
line2byte(line("$") + 1)
```

这就等于缓冲区大小加 1。如果 'fileencoding' 为空则等于文件大小加 1。
如果 {lnum} 非法或者编译时关闭了 +byte_offset 特性，返回 -1。另见 byte2line() 、 go 和 :goto 。

也可用作 method ：

```
GetLnum()->line2byte()
```

lispindent({lnum}) lispindent()

得到第 {lnum} 行根据 lisp 缩进规则应有的缩进距离，见 'lisp'。
缩进的计算以空格计，因而和 'tabstop' 的值是有关系的。
{lnum} 的使用方式和 getline() 相同。
如果 {lnum} 非法或者 Vim 编译时不带 +lispindent 特性，返回

-1。

也可用作 method :
GetLnum()->lispindent()

list2str({list} [, {utf8}]) list2str()
把 {list} 中的每个数值转换为字符，并连接成字符串。例如:
list2str([32]) 返回 " "
list2str([65, 66, 67]) 返回 "ABC"
也可以这样实现 (慢):
join(map(list, {nr, val -> nr2char(val)}), '')
str2list() 是逆操作。

{utf8} 省略或为零时，使用当前 'encoding'。
{utf8} 为 1 时，总是返回 utf-8 字符。
使用 utf-8 时，组合字符正常工作:
list2str([97, 769]) 返回 "a"

也可用作 method :
GetList()->list2str()

listener_add({callback} [, {buf}]) listener_add()
添加缓冲区 {buf} 有改动时会被执行的回调函数。
{buf} 可以是缓冲区名或号。它可接受的值见 bufname() 。省略
{buf} 时使用当前缓冲区。
返回唯一的 ID，可传递给 listener_remove() 。

{callback} 调用时带五个参数:
a:bufnr 发生改动的缓冲区
a:start 改动发生的首行行号
a:end 改动区域之下的首行行号
a:added 增加的行数，如果删除行则为负数
a:changes 关于改动细节的项目列表

示例:

```
func Listener(bufnr, start, end, added, changes)
  echo 'lines ' .. a:start .. ' until ' .. a:end .. ' changed'
endfunc
call listener_add('Listener', bufnr)
```

此列表不能修改。a:changes 的每个项目是带以下项的字典:
lnum 改动的首行行号
end 改动区域之下的首行行号
added 增加的行数，如果删除行则为负数
col "lnum" 中受改动影响的首列；如果未知或整行都受
影响则为一；这是字节计数，首个字符的值为一。
插入新行时，值为:
lnum 在该行之上加入了新行
end 等于 "lnum"
added 插入的行数
col 1
删除行时，值为:
lnum 首个删除行
end 删除发生之前首 (译者注: 应为末?) 个删除行之下

的那行
added 负数，删除的行数
col 1
行发生改动时：
lnum 首个改动行
end 最后改动行之下的那行
added 0
col 改动发生的首列，或 1

依改动发生的次序列出项目，这样最近发生的改动在最后。行号在回调发生时是合法的，但之后的改动可能使之非法，所以保存它们以备后用是不可行的。

在屏幕刚刚刷新前调用 {callback}，此后调用 listener_flush() 或进行影响行的数目的改动会使得改动列表里的行号变成非法。

{callback} 调用时文本被锁定，见 textlock 。如果需要改动缓冲区，使用定时器可以在之后某时进行需要的改动 timer_start() 。

缓冲区初次载入时不调用 {callback}。 BufReadPost 自动命令事件可用来处理缓冲区的初始文本。
缓冲区被卸载时也不调用 {callback}， BufUnload 自动事件可用于此。

也可用作 method ，基是作为第二个参数传递的：

```
GetBuffer()->listener_add(callback)
```

listener_flush([{buf}]) listener_flush()
调用缓冲区 {buf} 的监听器回调。如果没有待处理的改动则不调用回调。

{buf} 可以是缓冲区名或号。它可接受的值见 bufname() 。省略 {buf} 时使用当前缓冲区。

也可用作 method :

```
GetBuffer()->listener_flush()
```

listener_remove({id}) listener_remove()
删除之前用 listener_add() 加的监听器。
如果 {id} 找不到返回零，{id} 已删除时返回一。

也可用作 method :

```
GetListenerId()->listener_remove()
```

localtime() localtime()
返回当前时间，以 1970 年 1 月 1 日开始的秒数计算。另见 strftime() 、 strptime() 和 getftime() 。

log({expr}) log()
返回浮点数，即浮点数 {expr} 的自然对数 (即以 e 为底)。
{expr} 计算结果必须为 (0, inf] 区间内的浮点数或数值。
示例：

```
:echo log(10)
2.302585
```

```
:echo log(exp(5))
5.0
```

也可用作 method :

```
Compute()->log()
```

{仅当编译时加入 +float 特性才有效}

log10({expr}) log10()

返回浮点数，即浮点数 {expr} 以 10 为底的对数。
{expr} 计算结果必须为浮点数或数值。
示例:

```
:echo log10(1000)
3.0
:echo log10(0.01)
-2.0
```

也可用作 method :

```
Compute()->log10()
```

{仅当编译时加入 +float 特性才有效}

luaeval({expr} [, {expr}]) luaeval()

执行 Lua 表达式 {expr} 并把结果转换为 Vim 数据结构。第二个 {expr} 指定在第一个 {expr} 里可以使用的额外参数，可以 _A 形式访问。
字符串原样返回。
布尔值对象转换为数值。
vim 编译时带 +float 特性时，数值转换为 Float ，否则作为数值型返回。
vim.eval() 得到的字典和列表以原样返回。
lua-luaeval 有更多详情。

也可用作 method :

```
GetExpr()->luaeval()
```

{仅当编译时加入 +lua 特性才有效}

map({expr1}, {expr2}) map()

{expr1} 必须是 List 或 Dictionary 。
{expr1} 里的每个项目被 {expr2} 的计算结果替代。{expr2} 必须是 string 或 Funcref 。

如果 {expr2} 是 string ，{expr2} 内的 v:val 包含当前项目的值。 Dictionary 中 v:key 包含当前项目的键。 List 中 v:key 包含当前项目的索引。
例如:

```
:call map(mylist, '"> " . v:val . " <"')
```

"mylist" 里的每个项目之前放上 "> "，而之后放上 " <"。

注意 {string} 是表达式的计算结果，而它本身又用作表达式。通常，最好用 literal-string 来避免反斜杠加倍。当然，你仍然需要加倍 ' 引号。

如果 {expr2} 是 Funcref，它必须接受两个参数：
1. 当前项目的键或索引。
2. 当前项目的值。
函数必须返回项目的新值。例如要改变每个值为 "键-值"：
```
func KeyValue(key, val)
  return a:key . '-' . a:val
endfunc
call map(myDict, function('KeyValue'))
```
使用 lambda 会更短：
```
call map(myDict, {key, val -> key . '-' . val})
```
如果不用 "val"，可以不写：
```
call map(myDict, {key -> 'item: ' . key})
```
如果不用 "key"，可以使用短名：
```
call map(myDict, {_, val -> 'item: ' . val})
```

本操作是原位操作（直接在输入上修改）。要想不更动 List 或 Dictionary，先建立备份：
```
:let tlist = map(copy(mylist), ' v:val . "\t"')
```

返回 {expr1}，经过过滤的 List 或 Dictionary。如果执行 {expr2} 有错，不再处理 {expr1} 的其余项目。{expr2} 是函数引用时，忽略函数里的错误，除非该函数用 "abort" 标志位定义。

也可用作 method：
```
mylist->map(expr2)
```

maparg({name} [, {mode} [, {abbr} [, {dict}]]]) maparg()
当 {dict} 省略或为零，返回模式 {mode} 名为 {name} 的映射的右手边（rhs）。返回字符串中的特殊字符用和 ":map" 命令输出的同样方式进行翻译。

如果没有名为 {name} 的映射，返回 "<Nop>"。

{name} 可包含特殊键名，一如 ":map" 命令。

{mode} 可以使用下列字符串之一：

"n"	普通模式
"v"	可视模式（包括选择）
"o"	操作符等待模式
"i"	插入模式
"c"	命令行模式
"s"	选择模式
"x"	可视模式
"l"	语言映射 language-mapping
"t"	终端-作业
""	普通、可视和操作符等待模式。

如果没有提供 {mode}，使用 "" 指定的模式。

如果提供 {abbr} 且为 TRUE，使用缩写而不是映射。

如果提供 {dict} 且为 TRUE，返回字典，包含关于映射的所有信息。有如下项目：

"lhs"	映射的 {lhs}。
"rhs"	映射的 {rhs}，保持输入原样。

"silent" 1 若是 :map-silent 映射，否则为 0。
"noremap" 1 若是映射的 {rhs} 不能再映射。
"expr" 1 若是表达式映射 (:map-<expr>)。
"buffer" 1 若是缓冲区本地映射 (:map-local)。
"mode" 映射定义使用的模式。除了以上提到的模式以外，还会使用：
" " 普通、可视和操作符等待模式
"!" 插入和命令行模式
(mapmode-ic)
"sid" 脚本局部 ID，用于 <sid> 映射 (<SID>)。
"lnum" "sid" 中的行号，如果未知则为零。
"nowait" 不等待其它更长的映射。(:map-<nowait>)。

先检查局部于当前缓冲区的映射，然后再检查全局映射。
此函数可以用来给键映射。如果已经映射过，还可以使用原来映射的内容。大意：

```
exe 'nnoremap <Tab> ==' . maparg('<Tab>', 'n')
```

也可用作 method：

```
GetKey()->maparg('n')
```

mapcheck({name} [, {mode} [, {abbr}]]) mapcheck()
检查是否有模式 {mode} 下匹配 {name} 的映射。maparg() 说明 {mode} 和 {name} 里的特殊键名。
如果提供 {abbr} 且为 TRUE，使用缩写而不是映射。
匹配在映射名以 {name} 开始或者映射名等于 {name} 的开始部分时候发生。

匹配映射	"a"	"ab"	"abc"
mapcheck("a")	是	是	是
mapcheck("abc")	是	是	是
mapcheck("ax")	是	否	否
mapcheck("b")	否	否	否

和 maparg() 的差别是，mapcheck() 查找匹配 {name} 的映射，而 maparg() 只查找名字完全符合 {name} 的映射。
如果没有 {name} 开始的映射，返回空字符串。如果有一个，返回该映射的右手边。如果有多个，返回其中某一个的右手边。如果该右手边为空，返回 "<Nop>"。
先检查局部于当前缓冲区的映射，然后再检查全局映射。
该函数用于检查是否可以无二义性地添加映射。例如：

```
:if mapcheck("_vv") == ""
:   map _vv :set guifont=7x13<CR>
:endif
```

就避免了在已有 "_v" 或者 "_vvv" 映射的时候添加 "_vv" 映射。

也可用作 method：

```
GetKey()->mapcheck('n')
```

match({expr}, {pat} [, {start} [, {count}]]) match()
如果 {expr} 是 List，返回匹配 {pat} 的第一个项目的索引。每个项目用作字符串，List 和 Dictionary 使用回显的形式。

否则，{expr} 用作字符串。返回数值，给出 {expr} 里 {pat} 匹配的

(字节计算的偏移量) 位置。

在第一个字符或 List 项目上的匹配返回零。若无匹配，返回 -1。
要得到子匹配，见 matchlist() 。

例如:
```
:echo match("testing", "ing")    " 返回 4
:echo match([1, 'x'], '\a')      " 返回 1
```
string-match 说明如何使用 {pat}。

strpbrk()

Vim 没有 strpbrk() 函数。但你可以这么做:
```
:let sepidx = match(line, '[.,;: \t]')
```
strcasestr()

Vim 没有 strcasestr() 函数。但你可以在模式里加入 "\c" 以忽略大小写:
```
:let idx = match(haystack, '\cneedle')
```

如果给出 {start}，搜索从字符串的字节位置 {start} 或 List 索引为 {start} 的项目开始。
不过，结果仍然从第一个字符/项目开始算起。比如:
```
:echo match("testing", "ing", 2)
```
返回结果是 "4"。
```
:echo match("testing", "ing", 4)
```
返回结果还是 "4"。
```
:echo match("testing", "t", 2)
```
返回 "3"。
对字符串而言，如果 {start} > 0，其行为就像该字符串在 {start} 个字节后开始，因而 "^" 会从 {start} 开始匹配。如果给出 {count} 时则不是如此，此时忽略 {start} 字节前的匹配 (有一点复杂，这是为了后向兼容)。
对字符串而言，如果 {start} < 0，它被置为 0。对列表而言，此索引从尾部起算。
如果 {start} 越界 (字符串 {start} > strlen({expr})，而 List {start} > len({expr}))，返回 -1。

如果给出 {count}，使用第 {count} 个匹配。如果字符串里找到一个匹配，下一匹配从此匹配之后一个字符开始寻找。所以下例返回 1:
```
echo match("testing", "..", 0, 2)
```
List 里，搜索从下一个项目开始。
注意 如果加入 {count}，{start} 使用的方式有所改变。见上。

pattern 说明可以接受的模式。
'ignorecase' 选项用来设定模式是否忽略大小写。 不 使用 'smartcase'。匹配总是假定置位了 'magic' 而 'cpoptions' 为空。

也可用作 method :
```
GetList()->match('word')
```

matchadd() E798 E799 E801 E957

matchadd({group}, {pattern} [, {priority} [, {id} [, {dict}]]])
定义模式 (一个 "匹配")， 在当前窗口用高亮组 {group} 高亮。返回标识号 (ID)， matchdelete() 可用该 ID 来删除匹配。该 ID 是和窗口绑定的。
匹配是大小写敏感和带魔术的，但大小写敏感性和魔术性在 {pattern}

里可被显式关闭。不使用 'magic'、'smartcase' 和 'ignorecase' 选项。
"Conceal" 值是特别的，它使匹配被隐藏。

可选的 {priority} 参数指定匹配的优先级。高优先级的匹配的高亮会否决低优先级匹配的高亮。优先级用整数指定（负整数也无不可）。如果未指定 {priority} 参数，缺省优先级为 10。'hlsearch' 的优先级为零，这样所有正优先级的匹配都可以否决它。语法高亮（见 'syntax'）采用不同的机制，无论选择的优先级如何，匹配总会否决语法的高亮。

可选的 {id} 参数请求特定的匹配 ID。如果指定的 ID 已用，报错，并不加入该匹配。ID 用正整数指定（不含零）。ID 1、2 和 3 分别为 :match、:2match 和 :3match 命令保留。如果 {id} 未指定或为 -1，matchadd() 自动选择一个可用的 ID。

可选的 {dict} 参数允许更多定制。目前，可用来指定特定于匹配的隐藏字符，使用 hl-Conceal 高亮匹配来显示。该字典有以下成员：

conceal	显示的特殊字符，而非匹配（只用于 hl-Conceal 高亮匹配，见 :syn-cchar ）
window	不使用当前窗口，而使用指定窗口号或窗口 ID 的其它窗口。

匹配的数目不限，:match 诸命令则有此局限。

示例：

```
:highlight MyGroup ctermbg=green guibg=green
:let m = matchadd("MyGroup", "TODO")
```

要删除该模式：

```
:call matchdelete(m)
```

用 getmatches() 可以得到 matchadd() 和 :match 定义的匹配列表。clearmatches() 可一次删除所有的匹配。

也可用作 method：

```
GetGroup()->matchadd('TODO')
```

matchaddpos()

matchaddpos({group}, {pos} [, {priority} [, {id} [, {dict}]]])

和 matchadd() 相同，但需要位置列表 {pos} 而非匹配模式。比 matchadd() 更快，因为无需处理正规表达式并设置缓冲区行边界以重画屏幕。主要用于需要快速增加或删除匹配的场合，如匹配括号的高亮。

{pos} 列表包含以下项目之一：

- 数值。该行整行被高亮。首行的行号为 1。
- 单个数值的列表。如 [23]。该行整行被高亮。
- 包含两个数值的列表，如 [23, 11]。前面的数是行号，后面的是列号（首列为 1，必须对应 col() 返回的字节索引）。此位置的字符被高亮。
- 包含三个数值的列表，如 [23, 11, 3]。同上，但第三个数给出高亮字节计的长度。

最多可有 8 个位置。

示例:

```
:highlight MyGroup ctermbg=green guibg=green
:let m = matchaddpos("MyGroup", [[23, 24], 34])
```

模式的删除:

```
:call matchdelete(m)
```

matchaddpos() 加入的匹配在 getmatches() 返回时带有项目 "pos1"、"pos2"，等等，其值为 {pos} 项目提供的列表。

也可用作 method :

```
GetGroup()->matchaddpos([23, 11])
```

matcharg({nr}) matcharg()

选择 {nr} 号匹配的项目，它们分别用 :match 、 :2match 或 :3match 命令设置。
返回两个项目的 List :
使用的高亮组名
使用的模式。
如果 {nr} 不是 1、2 或 3，返回空 List 。
如果没有匹配的项目，返回 ['', ''] 。
这用来保存和恢复 :match 。
用 :match 命令高亮的匹配限于三个。 matchadd() 无此限制。

也可用作 method :

```
GetMatch()->matcharg()
```

matchdelete({id} [, {win}]) matchdelete() E802 E803

删除之前用 matchadd() 或 :match 诸命令定义的 ID 为 {id} 的匹配。如果成功，返回 0，不然返回 -1。示例见 matchadd() 。 clearmatches() 可一次删除所有的匹配。
如果指定 {win}，使用指定窗口号或窗口 ID 的窗口而不是当前窗口。

也可用作 method :

```
GetMatch()->matchdelete()
```

matchend({expr}, {pat} [, {start} [, {count}]]) matchend()

和 match() 相同，但返回匹配之后的第一个字符的位置。比如:

```
:echo matchend("testing", "ing")
```

返回 "7"。

strspn() strcspn()

Vim 没有 strspn() 或 strcspn() 函数，但可用 matchend() 实现:

```
:let span = matchend(line, '[a-zA-Z]')
:let span = matchend(line, '[^a-zA-Z]')
```

不过没有匹配时，它返回 -1。

如果给出 {start}，和 match() 里的用法相同。

```
:echo matchend("testing", "ing", 2)
```

返回 "7"。

```
:echo matchend("testing", "ing", 5)
```

返回 "-1"。
如果 {expr} 是 List ，结果和 match() 相同。

也可用作 method :
```
GetText()->matchend('word')
```

matchlist({expr}, {pat} [, {start} [, {count}]]) matchlist()
和 match() 相同，但返回 List 。列表第一项是匹配的字符串，和 matchstr() 返回值相同。其后的项目是子匹配，类似 :substitute 的 "\1"、"\2" 等。如果某个可选的子匹配不匹配，用空字符串代替。例如:
```
echo matchlist('acd', '\(a\)\?\(b\)\?\(c\)\?\(.*\)')
```
返回: ['acd', 'a', '', 'c', 'd', '', '', '', '', '']
如果没有匹配，返回空列表。

也可用作 method :
```
GetList()->matchlist('word')
```

matchstr({expr}, {pat} [, {start} [, {count}]]) matchstr()
和 match() 相同，但返回匹配的字符串。例如:
```
:echo matchstr("testing", "ing")
```
返回 "ing"。
如果没有匹配，返回 ""。
如果给出 {start}，它和 match() 里的用法相同。
```
:echo matchstr("testing", "ing", 2)
```
返回 "ing"。
```
:echo matchstr("testing", "ing", 5)
```
返回 ""。
如果 {expr} 是 List ，返回匹配的项目。其类型不改变，因而不一定是字符串。

也可用作 method :
```
GetText()->matchstr('word')
```

matchstrpos({expr}, {pat} [, {start} [, {count}]]) matchstrpos()
和 matchstr() 相同，但返回匹配的字符串和匹配的始末位置。例如:
```
:echo matchstrpos("testing", "ing")
```
返回 ["ing", 4, 7]。
如果没有匹配，返回 ["", -1, -1]。
如果给出 {start}，它和 match() 里的用法相同。
```
:echo matchstrpos("testing", "ing", 2)
```
返回 ["ing", 4, 7]。
```
:echo matchstrpos("testing", "ing", 5)
```
返回 ["", -1, -1]。
如果 {expr} 是 List ，返回匹配项目，{pat} 匹配的首个项目的索引，匹配的开始位置和结束位置。
```
:echo matchstrpos([1, '__x'], '\a')
```
返回 ["x", 1, 2, 3]。
不改变类型，不必然是字符串。

也可用作 method :
```
GetText()->matchstrpos('word')
```

max()
max({expr}) 返回 {expr} 所有项目的最大值。
{expr} 可以是列表或字典。如果是字典，返回字典中所有值的最大

值。
如果 {expr} 不是列表或字典，或者其中某个项目不能用作数值，出错。空 List 或 Dictionary 返回零。

也可用作 method :
```
mylist->max()
```

min()

min({expr}) 返回 {expr} 所有项目的最小值。
{expr} 可以是列表或字典。如果是字典，返回字典中所有值的最小值。
如果 {expr} 不是列表或字典，或者其中某个项目不能用作数值，出错。空 List 或 Dictionary 返回零。

也可用作 method :
```
mylist->min()
```

mkdir() E739

mkdir({name} [, {path} [, {prot}]])
建立目录 {name}。

如果 {path} 为 "p"，必要时建立中间的目录。否则它必须是 ""。

如果给出 {prot}，它用于设置新目录的权限。缺省为 0755 (rwxr-xr-x: 用户自己可读写，其它人可读)。用 0700 使其它人不可读。这只用于 {name} 的最后部分。所以，如果建立 /tmp/foo/bar，/tmp/foo 创建时的权限是 0755。
示例:
```
:call mkdir($HOME . "/tmp/foo/bar", "p", 0700)
```

该函数在沙盘里不可用 sandbox 。

如果目录已存在且传递了 "p" 标志位，不报错 (从补丁 8.0.1708 起)。如果没有 "p" 位就会失败。

此函数返回数值，如果调用成功则为 1，如果目录创建失败或部分失败则为 0。

不一定在所有系统上都可用。要检查这一点，使用:
```
:if exists("*mkdir")
```

也可用作 method :
```
GetName()->mkdir()
```

mode()

mode([expr]) 返回指示当前模式的字符串。
如果指定 [expr] 并且其值为非零的数值或非空的字符串 (non-zero-arg)，返回完整模式，不然，返回第一个字母。
另见 state() 。

n	普通模式，终端-普通
no	操作符等待模式
nov	操作符等待模式 (强制面向字符 o_v)
noV	操作符等待模式 (强制面向行 o_V)

noCTRL-v 操作符等待模式（强制面向列块 o_CTRL-V ）；
CTRL-V 是单个字符
niI Insert-mode 用 i_CTRL-O 进入的普通模式
niR Replace-mode 用 i_CTRL-O 进入的普通模式
niV Virtual-Replace-mode 用 i_CTRL-O 进入的普通模式
v 面向字符的可视模式
V 面向行的可视模式
CTRL-V 面向列块的可视模式
s 面向字符的选择模式
S 面向行的选择模式
CTRL-S 面向列块的选择模式
i 插入模式
ic 插入模式补全 compl-generic
ix 插入模式 i_CTRL-X 补全
R 替换模式 R
Rc 替换模式补全 compl-generic
Rv 虚拟替换模式 gR
Rx 替换模式 i_CTRL-X 补全
c 命令行编辑模式
cv Vim Ex 模式 gQ
ce 普通 Ex 模式 Q
r 输入回车的提示
rm -- more -- 提示
r? :confirm 等等的询问
! 执行外壳或外部命令时
t 终端-作业模式：键入传给作业

可用于 'statusline' 选项或 remote_expr() 。在其它的多数地方，它总是返回 "c" 或 "n"。
注意 将来可能会加入更多模式和更多特定模式。最好不要比较整个字符串而只比较开头的字符（一或多个）。
另见 visualmode() 。

也可用作 method :

```
DoFull()->mode()
```

mzeval({expr}) mzeval()

计算 MzScheme 表达式 {expr} 并返回计算结果，转换为 Vim 本身的数据结构。
数值和字符串返回本身。
组对（pair）（包含列表（list）和非常规列表（improper list））和向量（vector）以 Vim List 形式返回。
哈希表（hash table）以 Vim Dictionary 形式返回，其键转换成字符串。
所有其它类型依 display 函数调用的结果转换为字符串。
示例：

```
:mz (define l (list 1 2 3))
:mz (define h (make-hash)) (hash-set! h "list" l)
:echo mzeval("l")
:echo mzeval("h")
```

也可用作 method :

```
GetExpr()->mzeval()
```

{仅当编译时带 +mzscheme 特性才有效}

nextnonblank({lnum}) nextnonblank()
返回第一个从 {lnum} 开始的非空白行的行号。例如：
```
if getline(nextnonblank(1)) =~ "Java"
```
如果 {lnum} 非法或者在从该行开始都没有非空白行，返回零。
另见 prevnonblank() 。

也可用作 method :
```
GetLnum()->nextnonblank()
```

nr2char({expr} [, {utf8}]) nr2char()
返回单个字符组成的字符串，该字符的数值为 {expr}。例如：
```
nr2char(64)             返回 "@"
nr2char(32)             返回 " "
```
如果 {utf8} 省略或为零，使用当前的 'encoding'。比如对 "utf-8" 来说：
```
nr2char(300)            返回带有弓形的 I
```
如果 {utf8} 为 1，则总返回 utf-8 字符。
注意 文件里的 NUL 字符须用 nr2char(10) 指定。因为 Vim 用换行符来表示 NUL。真正的 NUL 是 nr2char(0)，而它会终结字符串，因而返回空串。
要把字符值的列表转换为字符串：
```
let list = [65, 66, 67]
let str = join(map(list, {_, val -> nr2char(val)}), '')
```
结果是："ABC"

也可用作 method :
```
GetNumber()->nr2char()
```

or({expr}, {expr}) or()
对两个参数进行按位或。参数须转换为数值。列表、字典或浮点数参数会报错。
示例：
```
:let bits = or(bits, 0x80)
```
也可用作 method :
```
:let bits = bits->or(0x80)
```

pathshorten({expr}) pathshorten()
缩短路径 {expr} 里的目录名，返回其结果。路径的尾部，即文件名，保持不变。路径的其余部分被缩短为单个字符。保持每个部分引导的 '~' 和 '.' 字符不变。例如：
```
:echo pathshorten('~/.vim/autoload/myfile.vim')
~/.v/a/myfile.vim
```
该路径实际存在与否并不相干。

也可用作 method :
```
GetDirectories()->pathshorten()
```

perleval({expr}) perleval()
在标量上下文计算 Perl 表达式 {expr}，返回结果转换为 Vim 数据结构。如果其值不能转换，返回 Perl 表示的字符串形式。

备注：如果要数组或哈希表，{expr} 必须返回它们的引用。
例如：

```
:echo perleval('[1 .. 4]')
[1, 2, 3, 4]
```

也可用作 method ：

```
GetExpr()->perleval()
```

{仅当编译时加入 +perl 特性才有效}

popup_ 函数文档在这里： popup-functions 。

pow({x}, {y}) pow()
返回浮点数，即 {x} 的 {y} 次方。
{x} 和 {y} 的计算结果必须是浮点数或数值。
示例：

```
:echo pow(3, 3)
27.0
:echo pow(2, 16)
65536.0
:echo pow(32, 0.20)
2.0
```

也可用作 method ：

```
Compute()->pow(3)
```

{仅当编译时加入 +float 特性才有效}

prevnonblank({lnum}) prevnonblank()
返回第一个 {lnum} 所在或之上的非空白行的行号。例如：

```
let ind = indent(prevnonblank(v:lnum - 1))
```

如果 {lnum} 非法或者在该行和它之前都没有非空白行，返回零。
另见 nextnonblank() 。

也可用作 method ：

```
GetLnum()->prevnonblank()
```

printf({fmt}, {expr1} ...) printf()
返回 {fmt} 指定的字符串，其中每个 "%" 项目被它们对应的参数排版后的形式取代。例如：

```
printf("%4d: E%d %.30s", lnum, errno, msg)
```

可能的返回结果：

```
"  99: E42 asdfasdfasdfasdfasdfasdfas"
```

也可用作 method ，基是作为第二个参数传递的：

```
Compute()->printf("result: %d")
```

常用的项目有：

%s	字符串
%6S	右对齐到 6 个显示单元的字符串
%6s	右对齐到 6 个字节的字符串
%.9s	截短到 9 个字节的字符串

```
%c     单个字节
%d     十进制数
%5d    十进制数，用空格补足到 5 个字符
%x     十六进制数
%04x   十六进制数，用 0 补足到 4 个字符
%X     十六进制数，用大写字母的十六进制字母
%o     八进制数
%08b   二进制数，用 0 补足到 8 个字符
%f     浮点数，形如 12.23、inf、-inf 或 nan
%F     浮点数，形如 12.23、INF、-INF 或 NAN
%e     浮点数，形如 1.23e3、inf、-inf 或 nan
%E     浮点数，形如 1.23E3、INF、-INF 或 NAN
%g     浮点数，根据不同的值，使用合适的 %f 或 %e
%G     浮点数，根据不同的值，使用合适的 %F 或 %E
%%     % 字符本身
```

转换规格说明以 '%' 开始，以转换类型结束。所有其它的字符按原样复制到结果中。

"%" 开始转换规格说明。其后的参数依序如下：

```
% [flags] [field-width] [.precision] type
```

flags
: 零或多个下面的标志位：

  `#` 转换值为 "替换形式"。对 c、d 和 s 转换，此选项无效。对 o 转换，增加数值的精度，使得输出字符串的第一个字符总是 0（除非打印零值，且显式地使用精度 0）。
  对 b 和 B 转换，非零值在前面加上字符串 "0b"（B 转换用 "0B"）。
  对 x 和 X 转换，非零值在前面加上字符串 "0x"（X 转换用 "0X"）。

  `0`（零） 以 0 填充。对所有的转换，左侧用 0 而非空格填充。如果对数值的转换给出精度（d、b、B、o、x 和 X），忽略 0 标志位。

  `-` 负域宽度标志位；转换后的值被左对齐到域边界上。该值右侧用空格填充，而不是在左侧用空格或 0 填充。如果两个标志位同时给出，- 否决 0。

  ' '（空格） 带符号转换（d）产生的正数左侧加上空格。

  `+` 带符号转换产生的数值之前总加上符号。如果两个标志位同时给出，+ 否决空格。

field-width
: 可选的十进制数位字符串，指定最小的字段宽度。如果转换后的值的字节数小于字段宽度，在左侧（或右侧，如果给定左对齐标志位的话）用空格填充到字段宽度。

.precision

可选的精度，形式为句号 '.' 后跟一个可选的数位字符串。
如果省略了数位字符串，假设精度为零。
它给出 d、o、x 和 X 转换显示的最小数位数量，或 s 转换显示的字符串的字节的最大数量。
对浮点数而言，指定小数点后的数位个数。

type
指定要进行的转换类型的单个字符，见下。

字段宽度、精度（两者都有亦可）可以用星号 '*' 代替数位字符串。此情形下，一个数值参数指定字段宽度或精度。负的字段宽度被理解为带左对齐的标志位后跟一个正数字段宽度；负的精度被理解为就像不存在一样。例如：

```
:echo printf("%d: %.*s", nr, width, line)
```

限制 "line" 文本的长度为 "width" 个字节。

转换标识符和它们的含义如下：

printf-d printf-b printf-B printf-o printf-x printf-X

dbBoxX 数值参数被转换为带符号十进制（d），无符号二进制（b 和 B）、无符号八进制（o）或无符号十六进制（x 和 X）记法。x 转换用字母 "abcdef"；X 转换用 "ABCDEF" 字母。
如果提供了精度，它给出必须出现的数位的最少数目；如果转换后的值需要更少的数位，左侧用 0 填充。
任何情况下数值字段都不会被不存在或者更小的字段宽度所截短；如果转换的结果宽于字段宽度，字段被扩展，以包含转换后的结果。
'h' 修饰符指示参数为 16 位。
'l' 修饰符指示参数为 32 位。
'L' 修饰符指示参数为 64 位。
通常这些修饰符不怎么有用。如果类型可从参数推知，修饰符被忽略。

i d 的别名
D ld 的别名
U lu 的别名
O lo 的别名

printf-c

c 数值参数被转换为字节，写入产生的字符。

printf-s

s 使用字符串参数的文本。如果指定精度，使用不多于给定数目的字节数。
如果参数不是字符串型，使用 ":echo" 相同的格式自动转换到文本。

printf-S

S 使用字符串参数的文本。如果指定精度，使用不多于给定数目的显示单元数。

printf-f E807

f F 浮点数参数被转换为形如 123.456 的字符串。精度指定小数点后面的位数。如果精度为零，则省略小数点本身。如果未指

定精度，缺省为 6。那个很大很大的数（超出返回或除以零的结果）用 %f 显示 "inf" 或 "-inf"（%F 则显示 INF 或 -INF）。
"0.0 / 0.0" 用 %f 显示 "nan"（%F 显示 NAN）。
示例：

```
echo printf("%.2f", 12.115)
12.12
```

注意 截断方式取决于系统库。如不确定，使用 round() 。

printf-e printf-E

e E 浮点数参数被转换为形如 1.234e+03 或用 'E' 的话 1.234E+03 的字符串。精度指定小数点后面的位数，和 'f' 一样。

printf-g printf-G

g G 如果浮点数参数在 0.001（含）和 10000000.0（不含）之间，则其转换同 'f'，不然，'g' 同 'e' 而 'G' 同 'E'。如果未指定精度，除了小数点之后的那个零以外，不显示多余的零和 '+' 负号。因而，10000000.0 显示为 1.0e7。

printf-%

% 写入 '%'。不转换参数。这里完整的转换规格说明是 "%%"。

如果期待数值参数，字符串参数也被接受并自动转换。
如果期待浮点数或字符串参数，数值参数也被接受并自动转换。
其它参数类型产生错误信息。

E766 E767

{exprN} 参数的数量必须和 "%" 项目的数量完全匹配。不论参数不足还是过多，都会给出错误。至多可用 18 个参数。

prompt_setcallback({buf}, {expr}) prompt_setcallback()

设置缓冲区 {buf} 的提示回调为 {expr}。{expr} 为空串时删除回调。只对 'buftype' 为 "prompt" 的 {buf} 有效。

按下回车时触发本回调。此时当前缓冲区总是提示缓冲区。在回调触发前，会加入一行新的提示行，因此，触发该回调的提示行会是倒数第二行。
如果回调要为缓冲区增加新文本，必须在最后一行之前插入，因为那是当前提示所在的位置，此操作可以异步进行。
回调调用时所带的一个参数是提示行输入的文本。如果用户直接按了回车，这会是一个空串。
示例：

```
call prompt_setcallback(bufnr(), function('s:TextEntered'))
func s:TextEntered(text)
  if a:text == 'exit' || a:text == 'quit'
    stopinsert
    close
  else
    call append(line('$') - 1, 'Entered: "' . a:text . '"')
    " 复位 'modified'，这样缓冲区才可以被关闭。
    set nomodified
  endif
```

```
endfunc
```

也可用作 method :
```
GetBuffer()->prompt_setcallback(callback)
```

prompt_setinterrupt({buf}, {expr}) prompt_setinterrupt()
设置缓冲区 {buf} 的回调为 {expr}。{expr} 为空串时删除回调。只对 'buftype' 为 "prompt" 的 {buf} 有效。

插入模式按 CTRL-C 时触发本回调。不设置回调会使 Vim 退出插入模式，就和其它缓冲区一样。

也可用作 method :
```
GetBuffer()->prompt_setinterrupt(callback)
```

prompt_setprompt({buf}, {text}) prompt_setprompt()
设置缓冲区 {buf} 的提示文本为 {text}。{text} 多数会以空格结尾。
结果只对 'buftype' 为 "prompt" 的 {buf} 可见。示例:
```
call prompt_setprompt(bufnr(), 'command: ')
```

也可用作 method :
```
GetBuffer()->prompt_setprompt('command: ')
```

prop_ 函数文档在这里: text-prop-functions 。

pum_getpos() pum_getpos()
如果弹出菜单（见 ins-completion-menu ）不可见，返回空 Dictionary ，不然，返回带以下键的 Dictionary :

height	可见项目数
width	屏幕单元格数
row	顶部屏幕行号 (0 为初行)
col	最左侧屏幕列号 (0 为初列)
size	项目总数
scrollbar	如果滚动条为可见则为 TRUE

这里的值和 CompleteChanged 发生时 v:event 的一样。

pumvisible() pumvisible()
如果弹出菜单可见，返回非零，不然返回零。见 ins-completion-menu 。
可以用来避免一些会删除弹出菜单的动作。

py3eval({expr}) py3eval()
计算 Python 表达式 {expr} 并返回计算结果，转换为 Vim 本身的数据结构。
数值和字符串返回本身 (字符串经过复制，Unicode 字符串还须额外用 'encoding' 转换)。
列表返回 Vim List 类型。
字典返回 Vim Dictionary 类型，键值转换为字符串。

也可用作 method :
```
GetExpr()->py3eval()
```

{仅当编译时加入 +python3 特性才有效}

pyeval({expr}) E858 E859 pyeval()
计算 Python 表达式 {expr} 并返回计算结果，转换为 Vim 本身的数据结构。
数值和字符串返回本身（字符串经过复制）。
列表返回 Vim List 类型。
字典返回 Vim Dictionary 类型，出现非字符串的键值报错。

也可用作 method :
```
GetExpr()->pyeval()
```

{仅当编译时加入 +python 特性才有效}

pyxeval({expr}) pyxeval()
计算 Python 表达式 {expr} 并返回计算结果，转换为 Vim 本身的数据结构。
使用 Python 2 或 3，见 python_x 和 'pyxversion'。
另见： pyeval() 、 py3eval()

也可用作 method :
```
GetExpr()->pyxeval()
```

{仅当编译时加入 +python 或 +python3 特性才有效}

range({expr} [, {max} [, {stride}]]) E726 E727 range()
返回数值的 List :
- 如果只有指定 {expr}: [0, 1, ..., {expr} - 1]
- 如果指定了 {max}: [{expr}, {expr} + 1, ..., {max}]
- 如果指定了 {stride}: [{expr}, {expr} + {stride}, ..., {max}] (每次给 {expr} 递增 {stride}，但不会产生超过 {max} 的值)。

如果最大值比开始值小一，返回空列表。如果更小，报错。
例如:
```
range(4)                " [0, 1, 2, 3]
range(2, 4)             " [2, 3, 4]
range(2, 9, 3)          " [2, 5, 8]
range(2, -2, -1)        " [2, 1, 0, -1, -2]
range(0)                " []
range(2, 0)             " 出错!
```

也可用作 method :
```
GetExpr()->range()
```

rand([{expr}]) rand()
用 {expr} 作为种子，返回 xoshiro128** 算法产生的伪随机数。返回数值是 32 位的，为了统一起见，即使是 64 位系统上也是。
可由 srand() 初始化 {expr} ，rand() 会进行更新。如果省略 {expr}，使用并更新一个内部的种子值。

示例：

```
:echo rand()
:let seed = srand()
:echo rand(seed)
:echo rand(seed) % 16  " 随机数 0 - 15
```

readdir()

readdir({directory} [, {expr}])

返回 {directory} 中的文件和目录名的列表。如果不需要复杂的处理，如限制匹配的数目等等，也可用 glob() 。

{expr} 省略时包含所有的项目。
如果给出 {expr}，计算其值决定如何处理：
如果 {expr} 结果为 -1， 不处理后续的项目。
如果 {expr} 结果为 0，不把本项加入列表。
如果 {expr} 结果为 1，把本项加入列表。
每次计算 {expr} 时 v:val 设为项目名。
{expr} 如果是函数，把名字作为参数传递。
例如，要得到 ".txt" 结尾的文件列表：

```
readdir(dirname, {n -> n =~ '.txt$'})
```

要跳隐藏和备份文件：

```
readdir(dirname, {n -> n !~ '^\.\|\~$'})
```

要想得到目录树：

```
function! s:tree(dir)
    return {a:dir : map(readdir(a:dir),
    \ {_, x -> isdirectory(x) ?
    \          {x : s:tree(a:dir . '/' . x)} : x})}
endfunction
echo s:tree(".")
```

也可用作 method ：

```
GetDirName()->readdir()
```

readfile()

readfile({fname} [, {type} [, {max}]])

读入文件 {fname} 并返回 List 。，文件每行一项。在 NL 字符处断开行。以 CR 分隔的 Macintosh 文件会返回单个长行 (除非某处出现了 NL)。
所有的 NUL 字符被 NL 字符替代。
如果 {type} 包含 "b"，使用二进制模式：
- 如果末行以 NL 结尾，附加额外的一个空列表项。
- 不删除 CR 字符。
如果 {type} 包含 "B"，返回 Blob ，包含未经修改的文件二进制数据。
否则：
- NL 之前的 CR 字符被删除。
- 末行是否以 NL 结尾没有影响。
- 'encoding' 如是 Unicode 编码，删除文本可能有的 UTF-8 字节顺序标识。

如果给出 {max}，指定读入的最大行数。可用于只想检查文件开始十行这样的场合：

```
:for line in readfile(fname, '', 10)
:  if line =~ 'Date' | echo line | endif
```

```
        :endfor
```
如果 {max} 为负，返回从文件尾部起算 -{max} 行，有多少算多少。
如果 {max} 为零，返回空列表。
注意 如果没有 {max}，把整个文件读到内存。
也要 注意 这里不识别编码。如果需要，把文件读到缓冲区里。
如果文件不能打开，给出错误信息，并返回空列表。
另见 writefile() 。

也可用作 method :
```
        GetFileName()->readfile()
```

reg_executing() reg_executing()
返回在执行中的单个字母寄存器名。如果没有在执行的寄存器，返回空串。见 @ 。

reg_recording() reg_recording()
返回在记录中的单个字母寄存器名。如果没有在记录的寄存器，返回空串。见 q 。

reltime([{start} [, {end}]]) reltime()
返回代表时间值的项目。项目的格式取决于不同的系统。可以把它传递给 reltimestr() 来转换为字符串，或 reltimefloat() 来转换为浮点数。
没有参数，返回当前时间。
带一个参数，返回参数指定的时间以来的时间。
带两个参数，返回 {start} 和 {end} 之间跨越的时间。
{start} 和 {end} 参数必须是 reltime() 返回的值。

也可用作 method :
```
        GetStart()->reltime()
```

{仅当编译时加入 +reltime 特性才有效}

reltimefloat({time}) reltimefloat()
返回代表 {time} 的时间值的浮点数。
示例:
```
        let start = reltime()
        call MyFunction()
        let seconds = reltimefloat(reltime(start))
```
参见 reltimestr() 关于开销的注释。
另见 profiling 。

也可用作 method :
```
        reltime(start)->reltimefloat()
```

{仅当编译时加入 +reltime 特性才有效}

reltimestr({time}) reltimestr()
返回字符串，代表 {time} 的时间值。
形式是秒数、句号和毫秒数。例如:
```
        let start = reltime()
        call MyFunction()
        echo reltimestr(reltime(start))
```

注意 命令本身额外的开销也计算在时间里。时间的准确度取决于系统。
返回结果包含引导的空格，使字符串能很好地对齐。如果你不需要，用 split() 可以删掉。

```
echo split(reltimestr(reltime(start)))[0]
```

另见 profiling 。

也可用作 method :

```
reltime(start)->reltimestr()
```

{仅当编译时加入 +reltime 特性才有效}

remote_expr() E449

remote_expr({server}, {string} [, {idvar} [, {timeout}]])
发送 {string} 到 {server}。该发送的字符串是一个表达式，而返回的是远端执行的结果。这个结果必然是字符串或 List 。List 被转换成字符串，转换方法是把项目用换行符连接起来（末项之后没有），就像用 join(expr, "\n") 那样。
如果给出 {idvar} 且非空，将 {serverid} 保存在以它命令的变量里，此后的 remote_read() 需要使用此值。
如果给出 {timeout}，在给定的秒数后读取超时。否则，使用 600 秒超时。
另见 clientserver RemoteReply 。
该函数在沙盘里不可用 sandbox 。
{仅当编译时加入 +clientserver 特性才有效}
注意：任何错误会在本地产生错误信息，但返回的结果只是一个空字符串。

在全局命名空间中计算变量，而和当前激活的函数无关。调试模式例外，此时会计算局部于函数的变量和参数。

例如:

```
:echo remote_expr("gvim", "2+2")
:echo remote_expr("gvim1", "b:current_syntax")
```

也可用作 method :

```
ServerName()->remote_expr(expr)
```

remote_foreground({server}) remote_foreground()
把名为 {server} 的 Vim 服务器带到前台。
这类似于:

```
remote_expr({server}, "foreground()")
```

Win32 系统除外。那里，客户端完成实际的工作。因为操作系统不总能允许服务器把自己带到前台。
注意：如果窗口最小化，并不恢复之，foreground() 会这么做。
该函数在沙盘里不可用 sandbox 。

也可用作 method :

```
ServerName()->remote_foreground()
```

{仅可用在 Win32、Athena、Motif 和 GTK 的 GUI 版本和 Win32 的控制台版本}

remote_peek({serverid} [, {retvar}]) remote_peek()

如果 {serverid} 有可用的字符串，返回正数。如果指定了 {retvar}，复制任何应答字符串到 {retvar} 指定的变量。{retvar} 必须是一个用来指定变量名的字符串。
如果没有可用的应答，返回 0。
如果出错，返回 -1。
另见 clientserver 。
该函数在沙盘里不可用 sandbox 。
{仅当编译时加入 +clientserver 特性才有效}
示例：

```
:let repl = ""
:echo "PEEK: ".remote_peek(id, "repl").": ".repl
```

也可用作 method ：

```
ServerId()->remote_peek()
```

remote_read({serverid}, [{timeout}]) remote_read()

返回从 {serverid} 发送的存在时间最长的应答，并删除之。除非给出以秒计的 {timeout}，该调用会等待直到有应答为止。
另见 clientserver 。
该函数在沙盘里不可用 sandbox 。
{仅当编译时加入 +clientserver 特性才有效}
例如：

```
:echo remote_read(id)
```

也可用作 method ：

```
ServerId()->remote_read()
```

remote_send() E241

remote_send({server}, {string} [, {idvar}])

发送 {string} 到 {server}。发送的字符串是输入键的序列。函数立即返回。Vim 的服务器端不对键进行映射 :map 。
如果给出 {idvar}，将 {serverid} 保存在以它命令的变量里，此后的 remote_read() 需要使用此值。
另见 clientserver RemoteReply 。
该函数在沙盘里不可用 sandbox 。
{仅当编译时加入 +clientserver 特性才有效}

注意：任何错误会在服务器端报告，从而影响那里的显示。
例如：

```
:echo remote_send("gvim", ":DropAndReply ".file, "serverid").
 \ remote_read(serverid)

:autocmd NONE RemoteReply *
 \ echo remote_read(expand("<amatch>"))
:echo remote_send("gvim", ":sleep 10 | echo ".
 \ 'server2client(expand("<client>"), "HELLO")<CR>')
```

也可用作 method ：

```
ServerName()->remote_send(keys)
```

remote_startserver() E941 E942

remote_startserver({name})

成为服务器 {name}。如果已作为服务器运行，也即 v:servername 不为空时此操作失败。

也可用作 method :

```
ServerName()->remote_startserver()
```

{仅当编译时加入 +clientserver 特性才有效}

remove({list}, {idx} [, {end}]) remove()

没有 {end}：删除 List {list} 里索引为 {idx} 的项目并返回之。有 {end}：删除从 {idx} 到 {end} (闭区间) 的项目，并返回这些项目的列表。如果 {idx} 指向和 {end} 相同的项目，返回单个项目的列表。如果 {end} 指向 {idx} 之前的项目，报错。

list-index 说明 {idx} 和 {end} 可能的取值。

例如：

```
:echo "last item: " . remove(mylist, -1)
:call remove(mylist, 0, 9)
```

用 delete() 来删除文件。

也可用作 method :

```
mylist->remove(idx)
```

remove({blob}, {idx} [, {end}])

没有 {end}：删除 Blob {blob} 里索引为 {idx} 的字节并返回之。有 {end}：删除从 {idx} 到 {end} (闭区间) 的字节，并返回这些字节构成的的 Blob 。如果 {idx} 指向和 {end} 相同的字节，返回单个字节构成的 Blob 。如果 {end} 指向 {idx} 之前的字节，报错。

示例：

```
:echo "last byte: " . remove(myblob, -1)
:call remove(mylist, 0, 9)
```

remove({dict}, {key})

删除 {dict} 里键为 {key} 的项目并返回其值。例如：

```
:echo "removed " . remove(dict, "one")
```

如果 {dict} 里没有键 {key}，报错。

rename({from}, {to}) rename()

把文件名 {from} 换成 {to}。这也可用来在文件系统间移动文件。返回数值，如果文件成功换名，返回零，如果换名失败，返回非零。

注意 如果 {to} 已存在，它被覆盖且没有提示。

该函数在沙盘里不可用 sandbox 。

也可用作 method :

```
GetOldName()->rename(newname)
```

repeat({expr}, {count}) repeat()

重复 {expr} {count} 次，并返回连接后的结果。例如：

```
:let separator = repeat('-', 80)
```

如果 {count} 为零或负，返回空。

如果 {expr} 是 List 类型，返回连接 {expr} {count} 次的结果。例如：

```
:let longlist = repeat(['a', 'b'], 3)
```

返回 ['a', 'b', 'a', 'b', 'a', 'b']。

也可用作 method :
```
mylist->repeat(count)
```

resolve({filename}) resolve() E655

在 MS-Windows 上，如果 {filename} 是一个快捷方式 (.lnk 文件)，返回简化的快捷方式指向的路径。当 {filename} 为符号链接或连接点 (junction point) 时，返回目的地的完整路径。如果该连接 (junction) 的目的地已被删除，返回 {filename}。
在 Unix 上，反复分析 {filename} 的所有路径部分的符号链接的真正路径，直到返回最简化的结果为止。为了处理循环链接的问题，符号链接的分析在 100 次叠代之后停止。
在其它系统上，返回简化了的 {filename}。
简化的工作通过 simplify() 完成。
resolve() 保留指向当前目录的首个路径部分 (保证结果仍然是相对路径名)，也保留出现在尾部的路径分隔符。

也可用作 method :
```
GetName()->resolve()
```

reverse({object}) reverse()

反转 {object} 项目的顺序，直接对 {object} 进行原位修改。
{object} 可以是 List 或 Blob 。
返回 {object}。
如果你不想修改，先构建一个备份:
```
:let revlist = reverse(copy(mylist))
```
也可用作 method :
```
mylist->reverse()
```

round({expr}) round()

返回浮点数，即最接近于 {expr} 的整数。如果 {expr} 在两个整数的正中间，使用 (译者注: 绝对值) 较大 (远离零的) 那个。
{expr} 的计算结果必须是浮点数或数值。
示例:
```
echo round(0.456)
0.0
echo round(4.5)
5.0
echo round(-4.5)
-5.0
```

也可用作 method :
```
Compute()->round()
```

{仅当编译时加入 +float 特性才有效}

rubyeval({expr}) rubyeval()

计算 Ruby 表达式 {expr} 并返回计算结果，转换为 Vim 本身的数据结构。
数值、浮点数和字符串返回本身 (字符串经过复制)。
数值返回 Vim List 类型。
哈希表返回 Vim Dictionary 类型。
其它对象返回它们 "Object#to_s" 方法返回的字符串。

也可用作 method :
```
GetRubyExpr()->rubyeval()
```

{仅当编译时加入 +ruby 特性才有效}

screenattr({row}, {col}) screenattr()

类似于 screenchar() ，但返回属性。是个相当任意的值，只可用于和其他位置的属性进行比较。

也可用作 method :
```
GetRow()->screenattr(col)
```

screenchar({row}, {col}) screenchar()

返回数值，即屏幕 [row, col] 位置的字符。可用于任何可能的屏幕位置，包括状态行，窗口分隔符和命令行。左上位置的行列号都为一。
返回字符不包括组合字符。双字节编码可能只返回第一个字节。
主要用于调试。
行号或列号越界时返回 -1。

也可用作 method :
```
GetRow()->screenchar(col)
```

screenchars({row}, {col}) screenchars()

返回数值列表。第一个数值和 screenchar() 返回的相同。其后的数值是附加于基础字符之上的组合字符。
主要用于测试。
如果 row 或 col 超出范围返回空列表。

也可用作 method :
```
GetRow()->screenchars(col)
```

screencol() screencol()

返回数值，即当前光标的屏幕列号。最左列的列号为 1。
主要用于调试。

注意：总是返回当前屏幕列，所以直接用于命令（例如 ":echo screencol()"）时会返回命令行内的列号，其在命令执行时永远为 1。
要得到文件中的光标位置，可用下面这些映射：
```
nnoremap <expr> GG ":echom ".screencol()."\n"
nnoremap <silent> GG :echom screencol()<CR>
```

screenpos({winid}, {lnum}, {col}) screenpos()

返回字典，窗口 {winid} 在缓冲区行 {lnum} 和列 {col} 上的文本字符的屏幕位置。{col} 是从一开始的字节索引。
字典有以下成员：

row	屏幕行
col	首个屏幕列
endcol	末尾屏幕列
curscol	光标所在屏幕列

如果指定位置不可见，所有的值均为零。
当字符占据多于一个屏幕单元格时，"endcol" 和 "col" 的值会不同。例如制表符的 "col" 可能是 1 而 "eolcol" 会是 8。
"curscol" 值是光标会放置的地方。对于制表符而言会和 "endcol" 相同。而对双宽字符来说，就会为 "col" 相同。

也可用作 method :
```
GetWinid()->screenpos(lnum, col)
```

screenrow() screenrow()

返回数值，即当前光标的屏幕行号。最顶行的行号为 1。
主要用于调试。
代为替代，也可用 winline() 。

注意：和 screencol() 的限制相同。

screenstring({row}, {col}) screenstring()

返回字符串，包含在屏幕位置 [row, col] 上的基本字符和任何组合字符。类似于 screenchars() ，但返回字符组成的字符串。
主要用于测试。
如果行或列超出范围，返回空串。

也可用作 method :
```
GetRow()->screenstring(col)
```

search({pattern} [, {flags} [, {stopline} [, {timeout}]]]) search()

搜索正规表达式模式 {pattern}。搜索从光标位置（用 cursor() 可以得到）开始。

如果找到了匹配，返回其所在的行号。
如果找不到匹配，返回 0 并且光标位置不改变。不会给出错误信息。

{flags} 是字符串，可以包含以下字符标志位：
'b' 反向（Backward）搜索，而不是正向搜索
'c' 接受光标（Cursor）位置上的匹配
'e' 移到匹配的尾部（End）
'n' 不（Not）移动光标
'p' 返回匹配的子模式（Pattern）号（见下）
's' 在光标上次的位置上设置（Set）' 位置标记
'w' 在文件尾部处回绕（Wrap）到文件开始处
'W' 不在文件尾部处回绕（Wrap）
'z' 从光标列开始搜索而不是第零（Zero）列
如果 'w' 和 'W' 都没有给出，根据 'wrapscan' 选项决定。

如果提供 's' 标志位，只有在光标移动的时候才设置 ' 位置标记。
's' 标志位不能和 'n' 标志位一起使用。

适用 'ignorecase'、'smartcase' 和 'magic' 标志位。

如果没给出 'z' 标志位，搜索总是从第零列开始，然后跳过光标之前的匹配。如果 'cpo' 中有 'c' 标志位，下次搜索从匹配之后的位置开始。没有 'c' 标志位则下次搜索从光标之后一列开始。

如果给出 {stopline} 参数，搜索在搜索完该行后结束。可用于限制搜索在给出的行范围内。例如：
```
let match = search('(', 'b', line("w0"))
let end = search('END', '', line("w$"))
```
如果使用了 {stopline} 且非零，隐含意味着搜索不会在文件尾回绕。
零就相当于没给出该参数。

如果给出 {timeout} 参数，搜索在超过给出的毫秒数后中止。这样，如果 {timeout} 为 500，搜索在半秒钟后中止。该值不能为负。零就相当于没给出该参数。
{仅当在编译时加入 +reltime 特性才有效}

search()-sub-match
如果有 'p' 标志位，返回值比第一个匹配的 \(\) 里的子模式的编号多一。如果所有子模式都不匹配但整个模式匹配，返回一。要得到列号，使用 searchpos() 。

光标定位在匹配的文本上，除非使用了 'n' 标志位。

示例 (遍历参数列表里的所有文件):

```
:let n = 1
:while n <= argc()      " 循环遍历参数列表的每个文件
:  exe "argument " . n
:  " 从文件最后一个字符开始并回绕，这样第一个搜索可以找到
:  " 文件开始的匹配
:  normal G$
:  let flags = "w"
:  while search("foo", flags) > 0
:    s/foo/bar/g
:    let flags = "W"
:  endwhile
:  update               " 如果修改过，写入文件
:  let n = n + 1
:endwhile
```

使用一些标志位的示例:

```
:echo search('\<if\|\(else\)\|\(endif\)', 'ncpe')
```

这会在光标之下或之后寻找关键字 "if"、"else" 和 "endif"。因为有 'p' 标志位，会根据找到的是哪 个关键字返回 1、2 或 3，如果找不到会返回 0。如果光标在行的第一个单词上:

```
if (foo == 0) | let foo = foo + 1 | endif
```

此函数返回 1。没有 'c' 标志位的话，函数会找到 "endif" 并返回 3。如果没有 'e' 标志位且光标在 "if" 的 "f" 上也会同样如此。'n' 标志位告诉函数不移动光标。

也可用作 method :

```
GetPattern()->search()
```

searchdecl({name} [, {global} [, {thisblock}]]) searchdecl()
搜索 {name} 的声明。

如果 {global} 参数非零，使用 gD 的工作方式，寻找文件的第一个匹配。否则使用 gd 的工作方式，寻找函数里的第一个匹配。

如果 {thisblock} 参数非零，忽略光标位置前结束的 {} 块里的匹配。可以避免只有在别的作用域里才有效的变量声明。

移动光标到找到的匹配上。
返回零代表成功，非零代表失败。
例如:

```
if searchdecl('myvar') == 0
   echo getline('.')
endif
```

也可用作 method :

```
GetName()->searchdecl()
```

searchpair()

searchpair({start}, {middle}, {end} [, {flags} [, {skip}
[, {stopline} [, {timeout}]]]])

搜索嵌套的 start-end 组对的匹配。这可以用来查找匹配 "if" 的 "endif"。在这里面的其它的 if/endif 组对被忽略。搜索从光标开始。缺省正向搜索，在 {flags} 里包含 'b' 的时候反向搜索。
如果找到一个匹配，光标移动到那里并返回行号。如果没有匹配，返回 0 或者 -1，光标不移动。不会给出错误信息。

{start}、{middle} 和 {end} 都是模式，见 pattern 。它们不能包含 \(\) 对，但可以使用 \%(\)。如果 {middle} 非空，在相应的方向试图寻找它 (如果找到，停留在哪里)，但在嵌套的 start-end 组对里面的不算。一个典型的应用是:

```
searchpair('\<if\>', '\<else\>', '\<endif\>')
```

如果 {middle} 为空，跳过 "else"。

{flags} 'b'、'c'、'n'、's'、'w' 和 'W' 的使用方式和 search() 类似。此外，还可用:

'r'　重复 (Repeat) 直到没有更多匹配位置；会找到最外层的组对。隐含 'W' 标志位。

'm'　返回匹配 (Match) 的数目而不是匹配的行号；使用 'r' 时会 > 1。

备注: 最好使用 'W' 标志位，避免在文件尾回绕。

如果找到 {start}、{middle} 或 {end} 的匹配，计算 {skip} 表达式，此时假定光标定位在匹配的开始处。如果返回零，该匹配被跳过。比如，可能是出现在注释里的匹配。
如果 {skip} 不提供或者为空，接受每一个匹配。如果计算 {skip} 时出现错误，搜索被中止，并返回 -1。
{skip} 可以是字符串、匿名函数、函数引用或偏函数。其它类型会报错。

{stopline} 和 {timeout} 见 search() 。

使用 'ignorecase' 的值。忽略 'magic'，使用模式时假设它总是置位的。

搜索从准确的光标处开始。根据搜索方向，寻找从下一个字符开始的 {start}、{middle} 或 {end}。比如:

```
if 1
  if 2
  endif 2
endif 1
```

如果从 "if 2" 开始且光标在 "i" 上并正向搜索，找到的是 "endif 2"。如果刚好在 "if 2" 之前开始，找到的是 "endif 1"。因为先找到的了 "if 2"，而它被认为是嵌套的 if/endif，以 "if 2" 开始，以 "endif 2" 结束。

如果反向搜索且 {end} 多于一个字符，在模式的最后加上 "\zs" 可能有用，这样光标在 end 匹配的中间某位置的时候，仍然可以找到匹配的 start 匹配。

例如，要找到 Vim 脚本里的 "endif" 命令：

```
:echo searchpair('\<if\>', '\<el\%[seif]\>', '\<en\%[dif]\>', 'W',
        \ 'getline(".") =~ "^\\s*\""')
```

光标必须在要寻找匹配的 "if" 之上或之后。注意 单引号字符串的使用，它避免了反斜杠的麻烦。skip 表达式只用来发现行首的注释，命令之后的不行。另外，一行中间的单词 "en" 或 "if" 也被认为是匹配。
另一个例子，搜索匹配 "}" 的 "{":

```
:echo searchpair('{', '', '}', 'bW')
```

只需要光标在需要匹配的 "}" 之上或之前就可以了。要拒绝语法高亮识别为字符串的匹配：

```
:echo searchpair('{', '', '}', 'bW',
     \ 'synIDattr(synID(line("."), col("."), 0), "name") =~? "string"')
```

searchpairpos()

searchpairpos({start}, {middle}, {end} [, {flags} [, {skip}
[, {stopline} [, {timeout}]]]])

和 searchpair() 相同，但返回 List ，包含匹配的行号和列号。 List 的第一个元素是行号，而第二个元素是匹配所在的列位置的字节位置。如果没有匹配，返回 [0, 0]。

```
:let [lnum,col] = searchpairpos('{', '', '}', 'n')
```

match-parens 提供一个更复杂更有用的例子。

searchpos({pattern} [, {flags} [, {stopline} [, {timeout}]]]) searchpos()

和 search() 相同，但返回 List ，包含匹配的行号和列号。 List 的第一个元素是行号，而第二个元素是匹配所在的列位置的字节位置。如果没有匹配，返回 [0, 0]。
例如：

```
:let [lnum, col] = searchpos('mypattern', 'n')
```

如果给出 'p' 标志位，返回值里有一个附加项目，包含匹配的子模式号 search()-sub-match 。例如：

```
:let [lnum, col, submatch] = searchpos('\(\l\)\|\(\u\)', 'np')
```

此例中如果找到小写字母 /\l "submatch" 为 2，如果找到大写字母 /\u 则为 3。

也可用作 method ：

```
GetPattern()->searchpos()
```

server2client({clientid}, {string}) server2client()

发送应答字符串到 {clientid}。最近刚发送过字符串的 {clientid} 可以通过 expand("<client>") 得到。
{仅当在编译时加入 +clientserver 特性才有效}

备注：
该 id 应在接受下一个命令前保存。也就是，在接收命令返回之前（译者注：似应为 "之后"）和任何等待输入的命令之前。
另见 clientserver 。
示例：

```
:echo server2client(expand("<client>"), "HELLO")
```

也可用作 method ：

```
GetClientId()->server2client(string)
```

serverlist() serverlist()
返回可用的服务器名字列表，每行一个。如果没有服务器或者该信息无法得到，返回空字符串。另见 clientserver 。
{仅当编译时加入 +clientserver 特性才有效}
示例：

```
:echo serverlist()
```

setbufline({expr}, {lnum}, {text}) setbufline()
设置缓冲区 {expr} 的第 {lnum} 行为 {text}。相当于为指定缓冲区调用 setline() 。

此函数只能用于已载入的缓冲区。有必要的话先调用 bufload() 。

要插入行可用 appendbufline() 。
清除 {lnum} 中的任何文本属性。

{text} 可以是字符串，用来设置单行文本，也可以是字符串列表，用来设置多行文本。如果列表范围超出末行之下，则添加额外的行。

{expr} 的用法可见上述 bufname() 。

{lnum} 的用法类似于 setline() 。
如果 {lnum} 正好在末行之下，在末行之下添加 {text}。

如果 {expr} 不是合法缓冲区、缓冲区未载入、或 {lnum} 不合法，返回 1。成功时返回 0。

也可用作 method ，基是作为第三个参数传递的：

```
GetText()->setbufline(buf, lnum)
```

setbufvar({expr}, {varname}, {val}) setbufvar()
设置缓冲区 {expr} 的选项或局部变量 {varname} 的值为 {val}。
也可用于全局或者局部于窗口的选项，但不能用于全局或者局部于窗口的变量。
如果设置局部于窗口的选项，全局值不会改变。
{expr} 的使用方式见上 bufname() 。
注意 必须使用不带 "b:" 的变量名。
示例：

```
:call setbufvar(1, "&mod", 1)
:call setbufvar("todo", "myvar", "foobar")
```

该命令在沙盘里不可用 sandbox 。

也可用作 method ，基是作为第三个参数传递的：

```
                    GetValue()->setbufvar(buf, varname)
```

setcharsearch({dict}) setcharsearch()

用 {dict} 设置当前字符搜索信息，带有以下项目：

char	下次 , 或 ; 命令使用的字符；空字符串清除字符搜索
forward	字符搜索的方向；1 为正向，0 为反向
until	字符搜索的类型；1 为 t 为 T 字符搜索，0 为 f 或 F 字符搜索

可用于在脚本中保存/恢复用户的字符搜索：

```
:let prevsearch = getcharsearch()
:" 执行改写用户搜索的命令
:call setcharsearch(prevsearch)
```

另见 getcharsearch() 。

也可用作 method ：

```
SavedSearch()->setcharsearch()
```

setcmdpos({pos}) setcmdpos()

设置命令行的光标位置到字节位置 {pos}。第一个位置为 1。
用 getcmdpos() 得到当前的位置。
只有在编辑命令行时有效，所以必须在 c_CTRL-_e 、 c_CTRL-R_= 或带 '=' 的 c_CTRL-R_CTRL-R 里使用。对于 c_CTRL-_e 和带 '=' 的 c_CTRL-R_CTRL-R ，在命令行设为表达式的内容之后才设置位置。对于 c_CTRL-R_= ，在计算表达式之后但在插入返回的文本之前设置位置。
如果数值太大，光标放在行尾。如果小于 1，结果没有定义。
如果成功，返回 0，如果不在编辑命令行，返回 1。

也可用作 method ：

```
GetPos()->setcmdpos()
```

setenv({name}, {val}) setenv()

设置环境变量 {name} 值为 {val}。
如果 {val} 为 v:null ，删除该环境变量。
另见 expr-env 。

也可用作 method ，基是作为第二个参数传递的：

```
GetPath()->setenv('PATH')
```

setfperm({fname}, {mode}) setfperm() chmod

设置 {fname} 的文件权限为 {mode}。
{mode} 必须是 9 个字符的字符串。形如 "rwxrwxrwx"，每组 "rwx" 标志位代表着，按次序，文件拥有者、文件所属组和其他用户的权限。
'-' 字符代表关闭权限，其他字符代表打开。不支持多字节字符。

例如 "rw-r-----" 意味着用户可读写，组只读，其他不可访问。
"xx-x-----" 作用相同。

返回非零代表成功，零代表失败。

也可用作 method ：

```
GetFilename()->setfperm(mode)
```

要读取权限可见 getfperm() 。

setline({lnum}, {text}) setline()
设置当前缓冲区第 {lnum} 行的内容为 {text}。要插入新行，用 append() 。要设置其它缓冲区的行，可用 setbufline() 。清除 {lnum} 中的任何文本属性。

{lnum} 的用法同 getline() 。
如果 {lnum} 正好在末行之下，在末行之下添加 {text}。

如果成功，返回 0。如果失败（多数是因为 {lnum} 不合法）返回 1。
例如：

```
:call setline(5, strftime("%c"))
```

如果 {text} 为 List ，那么第 {lnum} 行和其后的行被设为列表里的项目。例如：

```
:call setline(5, ['aaa', 'bbb', 'ccc'])
```

等价于：

```
:for [n, l] in [[5, 'aaa'], [6, 'bbb'], [7, 'ccc']]
:  call setline(n, l)
:endfor
```

注意：这里不会设置 '[和 '] 位置标记。

也可用作 method ，基是作为第二个参数传递的：

```
GetText()->setline(lnum)
```

setloclist({nr}, {list} [, {action} [, {what}]]) setloclist()
创建或替代或加入到窗口 {nr} 的位置列表。
{nr} 可以是窗口号或 window-ID 。
如果 {nr} 为零，使用当前窗口。

如果是位置列表窗口，修改所显示的位置列表。如果窗口号 {nr} 非法，返回 -1。
其它同 setqflist() 。
另见 location-list 。

如果给出可选的 {what} 字典参数，只设置 {what} 中列出的项目。
关于 {what} 支持的键值的列表，参见 setqflist() 。

setmatches({list} [, {win}]) setmatches()
恢复 getmatches() 为当前窗口保存的匹配列表。如果成功，返回 0，否则返回 -1。原有的所有匹配都被清除。示例见 getmatches() 。
如果给出 {win}，使用带此窗口号或窗口 ID 的窗口而不是当前窗口。

也可用作 method ：

```
GetMatches()->setmatches()
```

setpos({expr}, {list}) setpos()

设置 {expr} 的位置。可能的值:

.	光标
'x	位置标记 x

{list} 必须是带四个或五个数值的 List :

```
[bufnum, lnum, col, off]
[bufnum, lnum, col, off, curswant]
```

"bufnum" 是缓冲区号。零代表当前缓冲区。如果设置大写位置标记，"bufnum" 是位置标记位置的一部分。对其它的位置标记，它用于指定设置位置标记的缓冲区。可以用 bufnr() 函数把文件名转化为缓冲区号。
设置光标和 ' 位置标记时忽略 "bufnum"，因为它们和窗口相关，而不是缓冲区相关。
不修改跳转表。

"lnum" 和 "col" 是缓冲区里的位置。第一列为 1。"lnum" 为零则删除位置标记。"col" 如小于 1，则以 1 代替。

除非使用了 'virtualedit'，不用 "off" 值。这是从对应字符开始位置以屏幕列计的位移。例如，在制表之中或最后一个字符之后的某个位置。

"curswant" 值只用于设置光标位置。它指定垂直移动光标时的首选列。如果没有 "curswants" 值，不设置首选列。如果有但用于设置位置标记时，也没有效果。

注意 对 '< 和 '> 而言，改变行号可能会使两者在实际中被调换，以保证 '< 总是在 '> 之前。

如果位置可以设置，返回 0，否则返回 -1。如果 {expr} 不合法，报错。

另见 getpos() 和 getcurpos() 。

这并不能恢复垂直移动使用的列；如果用它设置了光标位置， j 和 k 动作会跳转到上次的列上！ cursor() 也可设置首选列。另见 winrestview() 的 "curswant" 键。

也可用作 method :

```
GetPosition()->setpos('.')
```

setqflist({list} [, {action} [, {what}]]) setqflist()
创建或替代或加入到 quickfix 表。

如果不给出 {what}，使用 {list} 里的项目。每个 {list} 项目是一个字典。{list} 里非字典的项目被忽略。每个字典项目可以包含以下的项目:

bufnr	缓冲区号；必须为某个合法缓冲区的编号
filename	文件名；仅当 "bufnr" 不存在或者不合法时才使用
module	模块名；如果给出，在 quickfix 错误窗口用它而不是文件名
lnum	缓冲区里的行号

pattern	用于定位错误的模式
col	列号
vcol	非零: "col" 是可视列 零: "col" 是字节位置
nr	错误号
text	错误描述
type	错误类型，'E'、'W' 等。
valid	经过识别的错误信息

"col"、"vcol"、"nr"、"type" 和 "text" 项目是可选的。"lnum" 或 "pattern" 项目用来定位匹配的错误行。
如果 "filename" 和 "bufnr" 项目都不存在或者 "lnum" 和 "pattern" 项目都不存在，那么此项目不被当作错误行处理。
如果 "pattern" 和 "lnum" 都存在，使用 "pattern"。
如果不提供 "valid" 项目，则在 "bufnr" 为合法的缓冲区或 "filename" 存在时，置位 valid 标志位。
如果 {list} 为空，quickfix 列表被清除。
注意 此列表和 getqflist() 返回之值不尽相同。

{action} 值: E927

'a' 把 {list} 项目加入已有的 quickfix 列表。如果该列表尚不存在，建立新表。

'r' {list} 项目替换当前 quickfix 列表项目。也可用于清除列表:

```
:call setqflist([], 'r')
```

'f' 释放 quickfix 堆栈上的所有 quickfix 列表。

如果 {action} 不存在或者设为 ' '，那么建立新表。栈中在当前 quickfix 列表之后加入新 quickfix 列表并释放其后的所有列表。要在栈的尾部加入新 quickfix 列表，把 {what} 中的 "nr" 设为 "$"。

如果提供了可选的 {what} 字典参数，只设置 {what} 列出的项目，忽略首个参数 {list}。{what} 支持以下项目:

context	quickfix 列表上下文。见 quickfix-context
efm	解析 "lines" 时使用的 errorformat。如果没有给出，使用 'errorformat' 选项值。 见 quickfix-parse
id	quickfix 列表标识号 quickfix-ID
idx	'id' 或 'nr' 指定的 quickfix 列表的当前项的索引。如果设为 '$'，列表的末项设为当前项。见 quickfix-index
items	quickfix 项目的列表，同 {list} 参数。
lines	用 'efm' (译者注: 原文作 'errorformat'，似不妥) 解析文本行列表并其结果项目加入 quickfix 列表 {nr} 或 {id} 中。只支持 List 类型。 见 quickfix-parse
nr	quickfix 堆栈中的列表号；零代表当前 quickfix 列表，"$" 代表最后的 quickfix 列表。
title	quickfix 列表标题。见 quickfix-title

忽略 {what} 中不支持的项目。
如果不给出 "nr"，修改当前 quickfix 列表。要创建新的 quickfix 列表，"nr" 可设为比 quickfix 栈大小大一的值。

在修改 quickfix 列表时，要保证修改的是正确的列表，应用 "id" 而不是 "nr" 来指定列表。

示例（另见 setqflist-examples ）：
```
:call setqflist([], 'r', {'title': 'My search'})
:call setqflist([], 'r', {'nr': 2, 'title': 'Errors'})
:call setqflist([], 'a', {'id':qfid, 'lines':["F1:10:L10"]})
```

返回零代表成功，-1 代表失败。

该函数用来独立于 'errorformat' 的设置建立 quickfix 列表。
‘:cc 1‘ 这样的命令可以跳转到第一个位置上。

也可用作 method ，基是作为第二个参数传递的：
```
GetErrorlist()->setqflist()
```

setreg()

setreg({regname}, {value} [, {options}])
设置寄存器 {regname} 的值为 {value}。
{value} 可以是 getreg() 返回的任何类型，包括 List 。
如果 {options} 包含 "a" 或者 {regname} 为大写，该值被附加于现有值之后。
{options} 还可以指定寄存器新类型的规格：
```
"c" 或 "v"          characterwise （面向字符）模式
"l" 或 "V"          linewise （面向行）模式
"b" 或 "<CTRL-V>"   blockwise-visual （面向列块）模式
```
如果 "b" 或 "<CTRL-V>" 之后紧跟数值，那么该数值用作选择的宽度 - 如果没有指定，那么列块的宽度设为最长的行字符数（把 <Tab> 看作一个字符）。

如果 {options} 没有寄存器设置，那么字符串值 {value} 缺省使用面向字符模式，除非 {value} 以 <NL> 结尾，列表值的 {value} 缺省使用面向行模式。不会自动选择面向列块模式。
返回零代表成功，非零代表失败。

E883
备注：设置搜索和表达式寄存器时，不可使用多于一个项目的 List 。无项目的列表相当于空串。

示例：
```
:call setreg(v:register, @*)
:call setreg('*', @%, 'ac')
:call setreg('a', "1\n2\n3", 'b5')
```

本例说明如何使用函数来保存和恢复寄存器：
```
:let var_a = getreg('a', 1)
:let var_amode = getregtype('a')
    ....
:call setreg('a', var_a, var_amode)
```

备注：如果不提供 getreg() 的第三个参数，就不能可靠地恢复寄存器值，因为此时，换行符和 Nul 字节都用换行符表示，见 NL-used-for-Nul ）。

你可以通过附加空串来改变寄存器的类型：

```
:call setreg('a', '', 'al')
```

也可用作 method ，基是作为第二个参数传递的：

```
GetText()->setreg('a')
```

settabvar({tabnr}, {varname}, {val}) settabvar()

设置标签页 {tabnr} 的标签页局部变量 {varname} 的值为 {val}。

注意 自动命令是阻塞的，可能不会触发副作用，比如设置 'filetype' 的时候。

注意 必须使用不带 "t:" 的名字。

标签页的编号从一开始。

该命令在沙盘里不可用 sandbox 。

也可用作 method ，基是作为第三个参数传递的：

```
GetValue()->settabvar(tab, name)
```

settabwinvar({tabnr}, {winnr}, {varname}, {val}) settabwinvar()

设置窗口 {nr} 的选项或局部变量 {varname} 的值为 {val}。

标签页的编号从一开始。 setwinvar() 总是使用当前标签页。

{winnr} 可以是窗口号或 window-ID 。

如果 {winnr} 为零，使用当前窗口。

注意 自动命令是阻塞的，可能不会触发副作用，比如设置 'filetype' 或 'syntax' 的时候。

也可用于全局或者局部于缓冲区的选项，但不能用于全局或者局部于缓冲区的变量。

如果设置局部于缓冲区的选项，全局值不会改变。

注意 必须使用不带 "w:" 的变量名。

示例：

```
:call settabwinvar(1, 1, "&list", 0)
:call settabwinvar(3, 2, "myvar", "foobar")
```

该命令在沙盘里不可用 sandbox 。

也可用作 method ，基是作为第四个参数传递的：

```
GetValue()->settabvar(tab, winnr, name)
```

(译者注：原文如此，应为 settabwinvar)

settagstack({nr}, {dict} [, {action}]) settagstack()

用 {dict} 修改窗口 {nr} 标签栈。

{nr} 可以是窗口号或 window-ID 。

要得到 {dict} 支持的项目的列表，可参考 gettagstack() 。 E962

如果 {action} 没给出或是 'r'，标签栈被替换。如果 {action} 是 'a'，则 {dict} 中的新项被压进标签栈里。

成功时返回零，失败则返回 -1。

示例：

设置标签栈的当前索引为 4：

```
call settagstack(1005, {'curidx' : 4})
```

清空窗口 3 的标签栈：

```
call settagstack(3, {'items' : []})
```

给标签栈压入新项目：

```
let pos = [bufnr('myfile.txt'), 10, 1, 0]
let newtag = [{'tagname' : 'mytag', 'from' : pos}]
call settagstack(2, {'items' : newtag}, 'a')
```

保存并恢复标签栈：

```
let stack = gettagstack(1003)
" do something else
call settagstack(1003, stack)
unlet stack
```

也可用作 method，基是作为第二个参数传递的：

```
GetStack()->settagstack(winnr)
```

setwinvar({winnr}, {varname}, {val}) setwinvar()

类似于 settabwinvar()，只用当前标签页。
示例：

```
:call setwinvar(1, "&list", 0)
:call setwinvar(2, "myvar", "foobar")
```

也可用作 method，基是作为第三个参数传递的：

```
GetValue()->setwinvar(winnr, name)
```

sha256({string}) sha256()

返回 64 位十六进制字符串，即 {string} 的 SHA256 校验码。

也可用作 method：

```
GetText()->sha256()
```

{仅当编译时加入 +cryptv 特性才有效}

shellescape({string} [, {special}]) shellescape()

转义 {string} 以便用作外壳命令的参数。
在 MS-Windows 上，如果未设定 'shellslash'，用双引号包围 {string}，并给 {string} 内的双引号加倍。
否则，用单引号包围，并把所有的 "'" 替换为 "'\''"。

如果给出 {special} 参数且它是非零的数值或非空的字符串 (non-zero-arg)，则特殊项目如 "!"、"%"、"#" 和 "<cword>" 等会在前面加上反斜杠。 :! 命令会再把反斜杠删除。

如果 'shell' 以 "csh" 结尾，"!" 字符会被转义（仍是当 {special} 为 non-zero-arg 时）。这是因为 csh 和 tcsh 即使在单引号内仍然使用 "!" 用于历史替换。

如果 {special} 为 non-zero-arg，<NL> 也被转义。'shell' 以 "csh" 结尾时，转义两次。

:! 命令的示例：

```
:exe '!dir ' . shellescape(expand('<cfile>'), 1)
```

返回光标所在文件给出的目录列表。 system() 的示例：

```
:call system("chmod +w -- " . shellescape(expand("%")))
```

另见 ::S 。

也可用作 method :

```
GetCommand()->shellescape()
```

shiftwidth([{col}]) shiftwidth()

返回 'shiftwidth' 的有效值。即 'shiftwidth' 值，除非其为零时则返回 'tabstop' 的值。此函数是 2012 年 7.3.694 补丁版本引入的，现在应该大家都有了 (不过可选的 {col} 参数是 8.1.542 以后才有的)。

给出一个参数 {col} 时，指定列号，返回它所用的 'shiftwidth' 值。这和 'vartabstop' 特性有关。如果打开 'vartabstop' 设置没有给出 {col}，假定为列 1。

也可用作 method :

```
GetColumn()->shiftwidth()
```

sign_ 函数文档在这里: sign-functions-details 。

simplify({filename}) simplify()

在不改变含义的前提下，尽可能简化文件名。快捷方式 (MS-Windows 上) 或者符号链接 (Unix 上) 不会被解析。如果 {filename} 第一个路径部分指定了当前目录，结果也会是如此。而结尾的路径分隔符也不会被删除。
示例:

```
simplify("./dir/.././/file/") == "./file/"
```

注意: 组合 "dir/.." 只有在 "dir" 是可以遍历的或者不存在的目录才会被删掉。Unix 上，如果 "dir" 是同一目录下的符号链接，也会删除该组合。为了在简化路径名之前解析所有牵涉到的符号链接，使用 resolve() 。

sin({expr}) sin()

返回浮点数，即以弧度测量的 {expr} 的正弦值。
{expr} 的计算结果必须是浮点数或数值。
示例:

```
:echo sin(100)
-0.506366
:echo sin(-4.01)
0.763301
```

也可用作 method :

```
Compute()->sin()
```

{仅当编译时加入 +float 特性才有效}

sinh({expr}) sinh()

返回 {expr} 的双曲正弦值，返回值为 [-inf, inf] 区间内的浮点数。

{expr} 的计算结果必须是浮点数或数值。
示例:

```
:echo sinh(0.5)
0.521095
:echo sinh(-0.9)
-1.026517
```

也可用作 method :

```
Compute()->sinh()
```

{仅当编译时加入 +float 特性才有效}

sort({list} [, {func} [, {dict}]]) sort() E702

给 {list} 项目排序，原地直接修改列表。返回 {list}。

如果你不想修改列表，先构建一个备份:

```
:let sortedlist = sort(copy(mylist))
```

如果 {func} 省略、为空或为零，则 sort() 对每个项目使用字符串表示形式进行排序。数值排在字符串之后， List 排在数值之后。要给当前缓冲区的文本排序，用 :sort 。

如果给出 {func} 且为 '1' 或 'i'，忽略大小写。

如果给出 {func} 且为 'n'，按数值顺序排序 (实现细节: 用 strtod() 函数来解析数值，字符串、列表、字典和函数引用均视作 0)。

如果给出 {func} 且为 'N'，按数值顺序排序。和 'n' 类似，但包含数位的字符串会被当作数值。

如果给出 {func} 且为 'f'，按数值顺序排序。所有值的类型必须是数值或浮点数。

如果 {func} 为 Funcref 或函数名，调用该函数来比较项目。函数调用时使用两个项目作为参数，函数返回时，0 代表相等，1 或更高代表第一个排在第二个之后，-1 或更小代表第一个排在第二个之前。

{dict} 用于带 "dict" 属性的函数，并设为局部变量 "self"。
Dictionary-function

排序是稳定的，相同值 (数值或字符串) 的项目保持原有的顺序不变。例如，按数值顺序排序时，文本字符串会紧挨在一起，按原先的顺序出现。

也可用作 method :

```
mylist->sort()
```

另见 uniq() 。

例如:

```
func MyCompare(i1, i2)
   return a:i1 == a:i2 ? 0 : a:i1 > a:i2 ? 1 : -1
```

```
        endfunc
        let sortedlist = sort(mylist, "MyCompare")
```

对这个简单的例子，有更简短的一个比较版本，不考虑溢出情况：

```
        func MyCompare(i1, i2)
           return a:i1 - a:i2
        endfunc
```

sound_clear() sound_clear()

停止播放所有声音。
{仅当编译时加入 +sound 特性才有效}

sound_playevent()

sound_playevent({name} [, {callback}])

播放 {name} 定义的声音。支持的事件名取决于系统。通常使用的是 XDG 声音名。Ubuntu 上可以在 /usr/share/sounds/freedesktop/stereo 中找到。例如：

```
        call sound_playevent('bell')
```

MS-Windows 上，{name} 可以是 SystemAsterisk、SystemDefault、SystemExclamation、SystemExit、SystemHand、SystemQuestion、SystemStart、SystemWelcome 等等。

{callback} 给出时，在声音结束后调用。首个参数是声音 ID，第二个参数是状态：

```
        0       声音播放到结束
        1       声音被中断
        2       声音启动后出错
```

示例：

```
   func Callback(id, status)
     echomsg "sound " .. a:id .. " finished with " .. a:status
   endfunc
   call sound_playevent('bell', 'Callback')
```

MS-Windows: 此函数不支持 {callback}。

返回声音 ID，可传递给 sound_stop() 。
如果声音不能播放，返回零。

也可用作 method :

```
        GetSoundName()->sound_playevent()
```

{仅当编译时加入 +sound 特性才有效}

sound_playfile()

sound_playfile({path} [, {callback}])

类似于 sound_playevent() 但播放声音文件 {path}。{path} 必须是完整路径。Ubuntu 上可以用此命令找可播放的文件：

```
    :!find /usr/share/sounds -type f | grep -v index.theme
```

也可用作 method :

```
        GetSoundPath()->sound_playfile()
```

{仅当编译时加入 +sound 特性才有效}

sound_stop({id}) sound_stop()

停止播放声音 {id}。{id} 必须是之前 sound_playevent() 或 sound_playfile() 的返回值。

MS-Windows 上，本函数不能用于 sound_playevent() 启动的事件声音。要停止事件声音，用 sound_clear() 。

也可用作 method :

```
soundid->sound_stop()
```

{仅当编译时加入 +sound 特性才有效}

soundfold()

soundfold({word})

返回 {word} 按发音折叠的等价形式。使用当前窗口的 'spelllang' 中第一个支持按发音折叠的语言。 'spell' 必须置位。如果不能按发音折叠，按原样返回 {word}。
可用来提供拼写建议。注意 此方法可能很慢。

也可用作 method :

```
GetWord()->soundfold()
```

spellbadword()

spellbadword([{sentence}])

没有参数：返回光标所在或之后的拼写错误的单词。光标移动到这个坏词的开始处。如果光标行上没有坏词，返回空字符串，不移动光标。

有参数：返回 {sentence} 里第一个拼写错误的单词。如果没有拼写错误，返回空字符串。

返回值是两个项目的列表：
- 错误拼写的单词，或空字符串。
- 拼写错误的类型：

```
"bad"           拼写错误
"rare"          偏僻词
"local"         只在其它区域里合法的单词
"caps"          单词应该大写开头
```

例如：

```
echo spellbadword("the quik brown fox")
['quik', 'bad']
```

使用当前窗口的拼写信息。'spell' 选项必须置位，也用到 'spelllang' 的值。

也可用作 method :

```
GetText()->spellbadword()
```

spellsuggest()

spellsuggest({word} [, {max} [, {capital}]])

返回 List ，包含替代 {word} 的拼写建议。
如果给出 {max}，返回的建议不超过此数目。否则，返回不超过 25 个建议。

如果给出 {capital} 参数且非零，只给出大写开头的拼写建议。

'spellcapcheck' 匹配后再使用此功能。

{word} 可以是后跟其它文本的错误拼写单词。这样可以对两个被分开的单词进行连接。建议里也包含附加文本，以便你替换整行。

{word} 也可以是个好词。返回和它类似的单词。建议里不包含 {word} 自身，但可能会出现其大写开头的形式。

使用当前窗口的拼写信息。'spell' 选项必须置位，也用到 'spelllang' 和 'spellsuggest' 的值。

也可用作 method :

```
GetWord()->spellsuggest()
```

split({expr} [, {pattern} [, {keepempty}]]) split()

从 {expr} 构造 List 。
如果 {pattern} 省略或为空，用每个空白分隔的字符序列构造一个项目。否则，在匹配 {pattern} 的地方分割字符串，删除匹配部分的字符。'ignorecase' 此处不适用，要忽略大小写，加上 \c。 /\c
如果列表的首末项目为空，省略它们，除非 {keepempty} 参数给出且非零。其它空项目在 {pattern} 匹配至少一个字符或者 {keepempty} 非零的时候被保留。
例如:

```
:let words = split(getline('.'), '\W\+')
```

要把字符串分割到每个字符:

```
:for c in split(mystring, '\zs')
```

如果你想保留分隔符，可以在模式尾部用 '\zs':

```
:echo split('abc:def:ghi', ':\zs')
['abc:', 'def:', 'ghi']
```

分割首项可能为空的表格:

```
:let items = split(line, ':', 1)
```

逆函数是 join() 。

也可用作 method :

```
GetString()->split()
```

sqrt({expr}) sqrt()

返回浮点数，即 {expr} 的非负平方根。
{expr} 的计算结果必须是浮点数或数值。如果 {expr} 为负，返回 NaN (Not a Number，非数)。
示例:

```
:echo sqrt(100)
10.0
:echo sqrt(-4.01)
nan
```

"nan" 可能不同，取决于系统库。

也可用作 method :

```
Compute()->sqrt()
```

{仅当编译时加入 +float 特性才有效}

srand([{expr}]) srand()

初始化 rand() 用的种子:
- 如果没给出 {expr}，如果可以，读取 /dev/urandom 并用作种子，否则，用 time(NULL) 也就是 epoch 时间；后者只有秒级精确度。
- {expr} 给出时必须是数值。用于初始种子值。可用于测试或需要可预测序列的场合。

示例:

```
:let seed = srand()
:let seed = srand(userinput)
:echo rand(seed)
```

state([{what}]) state()

返回字符串，包含指示当前状态的字符。主要用于要做一些可能不总是安全的操作的回调。大致上，做的事是:
- 回调使用 state() 检查工作是否安全。
  是: 马上开始做。
  否: 加入工作队列，新增 SafeState 和/或 SafeStateAgain 自动命令 (SafeState 在顶层触发， SafeStateAgain 在处理完信息和回调后触发)。
- SafeState 或 SafeStateAgain 触发时执行你的自动命令，检查 state() 看看工作现在是否可以了，如果是的话从队列中删除之并执行，如果队列为空也删除自动命令。

另见 mode() 。

{what} 给出时只加入此字符串中的字符。例如，下例检查屏幕是否有滚动:

```
if state('s') == ''
   " 屏幕还未滚动
```

这些字符指示状态，一般而言指示有东西在忙:

m	映射、:normal 命令、feedkeys() 或庞大的命令进行到中途时
o	操作符等待状态或命令行等待参数输入时，如 f 之后
a	插入模式自动补全激活时
x	执行自动命令时
w	处于阻塞等待时，如 ch_evalexpr()、ch_read() 或 ch_readraw() 读入 json 时。
S	不触发 SafeState 或 SafeStateAgain
c	调用了包括定时器在内的回调 (递归时会重复，多至 "ccc")
s	屏幕已经为消息滚动过

str2float({expr}) str2float()

把字符串 {expr} 转换为浮点数。这和使用浮点数的工作方式一样，见 floating-point-format ，但稍稍宽松一点。例如，接受 "1e40"，而表达式中你必须书写 "1.0e40"。也接受十六进制形式 "0x123"，但其它的如二进制或八进制不行。
安静地忽略数值之后的文本。
小数点必须是 '.'，和当前的 locale 无关。逗号会使数值转换结束: "12,345.67" 转换为 12.0。用 substitute() 可以拿掉千分位分隔符:

```
let f = str2float(substitute(text, ',', '', 'g'))
```

也可用作 method :

```
let f = text->substitute(',', '', 'g')->str2float()
```

{仅当编译时加入 +float 特性才有效}

str2list({expr} [, {utf8}]) str2list()
返回包含代表字符串 {expr} 中的每个字符数值的列表。例如:
```
str2list(" ")           返回 [32]
str2list("ABC")         返回 [65, 66, 67]
```
list2str() 是逆操作。

{utf8} 省略或为零时，使用当前 'encoding'。
{utf8} 为 1 时，总是把字符串当作 utf-8 字符。
使用 utf-8 时，组合字符正常工作:
```
str2list("a")         返回 [97, 769]
```

也可用作 method :
```
GetString()->str2list()
```

str2nr({expr} [, {base} [, {quoted}]]) str2nr()
把字符串 {expr} 转化为数值。
{base} 是转换的基底，可以为 2、8、10 或 16。
给出 {quoted} 且非零时，忽略内嵌的单引号，所以 "1'000'000" 是一百万。

如果省略 {base}，使用基底 10。这也意味着开头的零不会导致八进制的转换，缺省的字符串到数值的转化并非如此。例如:
```
let nr = str2nr('0123')
```

如果 {base} 为 16，忽略开头的 "0x" 或 "0X"。如果使用别的基底，返回零。类似的，如果 {base} 为 8，忽略开头的 "0"，如果 {base} 为 2，忽略开头的 "0b" 或 "0B"。
安静地忽略数值之后的文本。

也可用作 method :
```
GetText()->str2nr()
```

strcharpart({src}, {start} [, {len}]) strcharpart()
和 strpart() 类似，但用字符索引和长度而不是字节索引和长度。
使用字符索引时如果字符不存在，假定有一个字符存在。例如:
```
strcharpart('abc', -1, 2)
```
返回 'a'。

也可用作 method :
```
GetText()->strcharpart(5)
```

strchars({expr} [, {skipcc}]) strchars()
返回数值，给出字符串 {expr} 中的字符数。
{skipcc} 省略或为零时，组合字符也算一个字符。
{skipcc} 设为 1 时，计算时忽略组合字符。
另见 strlen() 、 strdisplaywidth() 和 strwidth() 。

{skipcc} 只在 7.4.755 之后才出现。为了后向兼容，可以这样定义包装函数:
```
if has("patch-7.4.755")
  function s:strchars(str, skipcc)
```

```
        return strchars(a:str, a:skipcc)
      endfunction
    else
      function s:strchars(str, skipcc)
        if a:skipcc
          return strlen(substitute(a:str, ".", "x", "g"))
        else
          return strchars(a:str)
        endif
      endfunction
    endif
```

也可用作 method：

```
GetText()->strchars()
```

strdisplaywidth({expr} [, {col}]) strdisplaywidth()

返回数值，给出字符串 {expr} 在 {col} 开始时在屏幕上占据的显示单元的数目（首列为零）。{col} 省略时假定为零。否则给出开始计算的屏幕列号。该值对制表符的计算有影响。
使用当前窗口的选项设置。其中影响显示的选项也对返回值有影响，如 'tabstop' 和 'display'。
{expr} 如包含东亚二义性宽度字符类，'ambiwidth' 也会影响返回结果。
另见 strlen()、strwidth() 和 strchars()。

也可用作 method：

```
GetText()->strdisplaywidth()
```

strftime({format} [, {time}]) strftime()

返回字符串，即经过 {format} 字符串的格式转换的日期和时间。使用给定的 {time}，如果没有给出时间，使用当前时间。可以接受的 {format} 取决于你的系统。这意味着该函数不是可移植的！
可用的格式参见 C 函数 strftime() 的参考手册。返回结果的最大长度是 80 个字符。另见 localtime()、getftime() 和 strptime()。
可以用 :language 命令改变语言。
示例：

```
:echo strftime("%c")              Sun Apr 27 11:49:23 1997
:echo strftime("%Y %b %d %X")     1997 Apr 27 11:53:25
:echo strftime("%y%m%d %T")       970427 11:53:55
:echo strftime("%H:%M")           11:55
:echo strftime("%c", getftime("file.c"))
                                  显示 file.c 的修改时间。
```

并非所有系统都可以用。要检查这一点，用：

```
:if exists("*strftime")
```

也可用作 method：

```
GetFormat()->strftime()
```

strgetchar({str}, {index}) strgetchar()

获取 {str} 中的第 {index} 个字符。使用字符索引而不是字节索引。这里，组合字符当作单独的字符。
另见 strcharpart() 和 strchars()。

也可用作 method :
```
GetText()->strgetchar(5)
```

stridx({haystack}, {needle} [, {start}]) stridx()

返回数值，给出字符串 {haystack} 里第一个字符串 {needle} 出现的字节位置。
如果给出 {start}，搜索从 {start} 位置开始。可用来寻找第二个匹配：
```
:let colon1 = stridx(line, ":")
:let colon2 = stridx(line, ":", colon1 + 1)
```
搜索对大小写敏感。
模式搜索可用 match() 。
如果 {needle} 不出现在 {haystack} 里，返回 -1。
另见 strridx() 。示例：
```
:echo stridx("An Example", "Example")      3
:echo stridx("Starting point", "Start")    0
:echo stridx("Starting point", "start")   -1
```
strstr() strchr()
stridx() 和 C 函数 strstr() 类似。如果使用单个字符，和 strchr() 类似。

也可用作 method :
```
GetHaystack()->stridx(needle)
```

string()

string({expr}) 返回 {expr} 转换后的字符串。如果 {expr} 为数值、浮点数、字符串、blob 或它们的复合形式，那么用 eval() 可以把结果转回去。

{expr} 类型	返回值
字符串	'string' (单引号加倍)
数值	123
浮点数	123.123456 或 1.23456e8
函数引用	function('name')
blob	0z00112233.44556677.8899
列表	[item, item]
字典	{key: value, key: value}

列表或字典中如有递归引用，被替换为 "[...]" 或 "{...}"。在此结果上运行 eval() 会出错。

也可用作 method :
```
mylist->string()
```

另见 strtrans() 。

strlen()

strlen({expr}) 返回数值，即字符串 {expr} 的字节长度。
如果参数为数值，先把它转化为字符串。其它类型报错。
要计算多字节字符的数目，可用 strchars() 。
另见 len() 、 strdisplaywidth() 和 strwidth() 。

也可用作 method :
```
GetString()->strlen()
```

strpart({src}, {start} [, {len}]) strpart()

返回字符串，{src} 从第 {start} 个字节开始字节长度为 {len} 的子串。
要以字符而不是字节计算，用 strcharpart() 。

如果选择不存在的字节，不会产生错误。只是那些字节被忽略而已。
如果没有提供 {len}，子串从 {start} 开始直到 {src} 的结尾。

```
strpart("abcdefg", 3, 2)    == "de"
strpart("abcdefg", -2, 4)   == "ab"
strpart("abcdefg", 5, 4)    == "fg"
strpart("abcdefg", 3)       == "defg"
```

注意：要得到第一个字符，{start} 必须是零。比如，要得到光标开始的三个字节：

```
strpart(getline("."), col(".") - 1, 3)
```

也可用作 method ：

```
GetText()->strpart(5)
```

strptime({format}, {timestring}) strptime()
返回数值，代表期待能匹配 {format} 指定格式的 {timestring} 中的日期和时间的 unix 时间戳。

可接受的 {format} 取决于系统，所以不可移植！具体格式见 C 函数 strptime() 的手册页。特别要注意避免 "%c"。$TZ 的值也相关。

如果 {timestring} 不能用 {format} 格式解析，返回零。如果不知道 {timestring} 的格式，可以尝试不同的 {format} 值直到返回非零值为止。

另见 strftime() 。
示例：

```
:echo strptime("%Y %b %d %X", "1997 Apr 27 11:49:23")
862156163
:echo strftime("%c", strptime("%y%m%d %T", "970427 11:53:55"))
Sun Apr 27 11:53:55 1997
:echo strftime("%c", strptime("%Y%m%d%H%M%S", "19970427115355") + 3600)
Sun Apr 27 12:53:55 1997
```

不是在所有系统都可用。要确认：

```
:if exists("*strptime")
```

strridx({haystack}, {needle} [, {start}]) strridx()
返回数值，给出字符串 {haystack} 里最后一个字符串 {needle} 出现的字节位置。
如果给出 {start}，此位置之外的匹配被忽略。可用来寻找上次匹配之前的匹配：

```
:let lastcomma = strridx(line, ",")
:let comma2 = strridx(line, ",", lastcomma - 1)
```

搜索对大小写敏感。
模式搜索可用 match() 。
如果 {needle} 不出现在 {haystack} 里，返回 -1。
如果 {needle} 为空，返回 {haystack} 的长度。
另见 stridx() 。示例：

```
:echo strridx("an angry armadillo", "an")          3
```
strrchr()
如果使用单个字符，和 C 函数 strrchr() 类似。

也可用作 method :
```
GetHaystack()->strridx(needle)
```

strtrans({expr}) strtrans()
返回等于 {expr} 的字符串，但所有的不可显示字符被翻译成可显示的字符序列 'isprint' ，类似于窗口里显示的形式。例如:
```
echo strtrans(@a)
```
会显示寄存器里的换行符为 "^@" 而不是开启新行。

也可用作 method :
```
GetString()->strtrans()
```

strwidth({expr}) strwidth()
返回数值，给出字符串 {expr} 在屏幕上占据的显示单元的数目。制表符算作一个单元。如果不想这样，可用 strdisplaywidth() 。
{expr} 如包含东亚二义性宽度字符类，'ambiwidth' 也会影响返回结果。
另见 strlen() 、 strdisplaywidth() 和 strchars() 。

也可用作 method :
```
GetString()->strwidth()
```

submatch({nr} [, {list}]) submatch() E935
只用于 :substitute 命令或 substitute() 函数中的表达式内。
返回匹配文本的第 {nr} 个子匹配。{nr} 为 0 则返回整个匹配的文本。
注意 字符串中的 NL 既可以代表多行匹配的行尾，也可以是文件中的 NUL 字符。
另见 sub-replace-expression 。

如果给出 {list} 且非零，submatch() 返回字符串的列表，和带两个参数的 getline() 类似。文本中的 NL 代表文件中的 NUL 字符。
只在 :substitute 中可能返回多于一个项目， substitute() 中此列表总是包含一个或零个项目，因为没有真正的行尾。

递归使用 substitute() 时，只能得到当前 (最深的) 调用的子匹配。

例如:
```
:s/\d\+/\=submatch(0) + 1/
:echo substitute(text, '\d\+', '\=submatch(0) + 1', '')
```
找到行内第一个数值并加 1。
使用 <NL> 可以包含换行符。

也可用作 method :
```
GetNr()->submatch()
```

substitute({expr}, {pat}, {sub}, {flags}) substitute()
返回相当于 {expr} 的字符串，但其中 {pat} 的首个匹配被替代成 {sub}。
如果 {flags} 为 "g"，替换 {expr} 里的所有 {pat} 匹配。否则，

{flags} 应该为 ""。

和 ":substitute" 命令（不带任何标志位）类似。但此处 {pat} 的匹配总假定 'magic' 选项已置位且 'cpoptions' 为空（为了脚本的可移植性）。
'ignorecase' 仍然适用， /\c 或 /\C 可用来直接指定是否忽略或匹配大小写并忽略 'ignorecase' 的设置。
'smartcase' 此处不适用。
string-match 说明如何使用 {pat}。

{sub} 里的 '~' 不会被换成前一个 {sub}。
注意 {sub} 里的一些代码有特殊含义 sub-replace-special 。比如，要替换若干文本为 "\n"（两个字符），要用 "\\\\n" 或 '\\n'。

如果 {pat} 在 {expr} 里不能匹配，返回没有修改的 {expr}。

示例：
```
:let &path = substitute(&path, ",\\=[^,]*$", "", "")
```
删除 'path' 选项的最后一部分。
```
:echo substitute("testing", ".*", "\\U\\0", "")
```
返回 "TESTING"。

{sub} 参数以 \= 开始时，其余部分视为一个表达式，见 sub-replace-expression 。示例：
```
:echo substitute(s, '%\(\x\x\)',
        \ '\=nr2char("0x" . submatch(1))', 'g')
```

{sub} 若是函数引用，调用该函数，带一个可选参数。示例：
```
:echo substitute(s, '%\(\x\x\)', SubNr, 'g')
```
可选参数是包含完整匹配及多达九个子匹配的列表，就像 submatch() 的返回值那样。示例：
```
:echo substitute(s, '%\(\x\x\)', {m -> '0x' . m[1]}, 'g')
```

也可用作 method ：
```
GetString()->substitute(pat, sub, flags)
```

swapinfo({fname}) swapinfo()
返回字曲，保存关于交换文件 {fname} 的信息。可用的域是：

version	Vim 版本
user	用户名
host	主机名
fname	原始文件名
pid	创建交换文件的 Vim 进程的 PID
mtime	以秒计的最近修改时间
inode	可选：文件的 INODE 值
dirty	如果文件修改过为 1，否则为 0

注意 "user" 和 "host" 被缩短为最多 39 个字节。如果失败，加入 "error" 项目说明相关原因：

Cannot open file:	文件找不到或不能访问
Cannot read file:	不能读入首块
Not a swap file:	没有包含正确的块 ID
Magic number mismatch:	首块的信息不正确

也可用作 method :
```
GetFilename()->swapinfo()
```

swapname({expr}) swapname()
返回缓冲区 {expr} 所用的交换文件路径。
{expr} 的使用方式见上 bufname() 。
如果缓冲区 {expr} 是当前缓冲区，结果相当于 :swapname （除非没有交换文件）。
如果缓冲区 {expr} 无交换文件，返回空串。

也可用作 method :
```
GetBufname()->swapname()
```

synID({lnum}, {col}, {trans}) synID()
返回数值，即当前窗口 {lnum} 行 {col} 列所在的语法 ID。
语法 ID 可以用在 synIDattr() 和 synIDtrans() ，以得到文本的语法信息。

最左列的 {col} 为 1。第一行的 {lnum} 为 1。适用 'synmaxcol' 的值，如果行比它更长，就返回零。
注意 位置在最后一个字符之后是插入模式下可能的光标位置。此时，synID() 返回零。

如果 {trans} 为 TRUE ，透明的项目被简约为它们实际显露的项目。这可以用于你想知道实际使用的颜色的情形。如果 {trans} 为 FALSE ，返回透明的项目本身。这可用于想知道实际有效的语法项目的情形（比如，在括号内部）。
警告：本函数可能很慢。最佳速度可以通过正向遍历文件获得。

例如（回显光标所在的语法项目的名字）：
```
:echo synIDattr(synID(line("."), col("."), 1), "name")
```

synIDattr({synID}, {what} [, {mode}]) synIDattr()
返回字符串，syntax ID {synID} 的 {what} 属性。可用于得到语法项目的相关信息。
{mode} 可以是 "gui"、"cterm" 或 "term"，从而得到的是该模式下的属性。如果忽略 {mode} 或者指定了非法的值，使用当前激活的高亮方式的属性（GUI、cterm 或 term）。
使用 synIDtrans() 来跟随链接的高亮组。

{what}	结果
"name"	语法项目的名字
"fg"	前景色（GUI：用于设置颜色的色彩名，cterm：色彩号，以字符串形式出现，term：空字符串）
"bg"	背景色（细节同 "fg"）
"font"	字体名（只适用于 GUI） highlight-font
"sp"	特殊颜色（细节同 "fg"） highlight-guisp
"fg#"	类似于 "fg"，但只适用于 GUI，而且 GUI 使用的名字形如 "#RRGGBB"。
"bg#"	"bg"，细节同 "fg#"
"sp#"	"sp"，细节同 "fg#"
"bold"	"1" 如果粗体
"italic"	"1" 如果斜体
"reverse"	"1" 如果反显

"inverse" "1" 如果反显 (= reverse)
"standout" "1" 如果突出
"underline" "1" 如果下划线
"undercurl" "1" 如果下曲线
"strike" "1" 如果删除线

示例 (回显光标所在的语法项目的颜色):

```
:echo synIDattr(synIDtrans(synID(line("."), col("."), 1)), "fg")
```

也可用作 method :

```
:echo synID(line("."), col("."), 1)->synIDtrans()->synIDattr("fg")
```

synIDtrans({synID}) synIDtrans()

返回数值，即 {synID} 经过翻译的语法 ID。这是用于高亮字符的语法组的 ID。":highlight link" 给出的高亮组被跟随，以找到实际使用的组。

也可用作 method :

```
:echo synID(line("."), col("."), 1)->synIDtrans()->synIDattr("fg")
```

synconcealed({lnum}, {col}) synconcealed()

返回列表，目前包含三个值:

1. 如果 {lnum} 和 {col} 所在位置的字符不在可隐藏区域，列表的第一个项目为 0，否则为 1。
2. 列表的第二个项目为字符串。如果第一个值为 1，第二个值包含代替被隐藏文本实际显示的文本，视乎 'conceallevel' 和 'listchars' 的当前值而定。
3. 列表的第三个，也即最后一个项目是代表匹配的特定语法区域的唯一的数值。这用于在有两个连续的使用相同替代字符的区域时，检测此处是否是一个新的可隐藏区域的开始。例如，如果文本是 "123456"，"23" 和 "45" 都被隐藏，而替代字符是 "X"，则:

```
call                    returns
synconcealed(lnum, 1)   [0, '', 0]
synconcealed(lnum, 2)   [1, 'X', 1]
synconcealed(lnum, 3)   [1, 'X', 1]
synconcealed(lnum, 4)   [1, 'X', 2]
synconcealed(lnum, 5)   [1, 'X', 2]
synconcealed(lnum, 6)   [0, '', 0]
```

synstack({lnum}, {col}) synstack()

返回 List，即当前窗口在 {lnum} 行 {col} 列语法项目的堆栈。列表的每个项目是像 synID() 返回那样的 ID。
列表的第一个项目是最外层区域，其后依次是包含在内的项目。末项即 synID() 返回的项目，除非不是整个项目都被高亮，或者它是一个透明项目。
此函数可用于调试语法文件。
显示光标所在的语法项目栈的示例:

```
for id in synstack(line("."), col("."))
   echo synIDattr(id, "name")
endfor
```

如果 {lnum} 和 {col} 指定的位置非法，不返回任何值。行末字符之后的位置以及空行的第一个位置是合法的位置。

system({expr} [, {input}]) system() E677

得到外壳命令 {expr} 字符串形式的输出结果。要得到列表形式的输出结果，见 systemlist() 。

如果给出 {input} 且为字符串，该字符串被写到文件里，并传给外壳命令作为标准输入。字符串照原样写入，你需要自己 注意 使用合适的换行符。
如果给出 {input} 且为 List ，它被写到文件里，就像 writefile() 带 {binary} 设为 "b" 的工作方式那样 (即，列表项目间写入换行符，列表项目内部的换行符改写为 NUL)。
如果给出 {input} 且为数值，代表已存在的缓冲区的合法缓冲区号，逐行将该缓冲区的内容写入文件，每行以 NL 结尾，文本内的 NL 以 NUL 字符代替。

不使用管道，也不使用 'shelltemp' 选项。

如果加上前缀 :silent ，终端不设为加工 (cooked) 模式。这用于不需要用户输入的命令，以避免屏幕上显示多余的字符而需要用 CTRL-L 来清除。

```
:silent let f = system('ls *.vim')
```

注意: shellescape() 或 expand() 的 ::S 或 fnamemodify() 可以转义命令参数里的特殊字符。{expr} 里的换行可能会使命令失败。'shellquote' 和 'shellxquote' 里的字符也可能会引起麻烦。
这不是用来执行交互命令的。

返回字符串。示例:

```
:let files = system("ls " .  shellescape(expand('%:h')))
:let files = system('ls ' . expand('%:h:S'))
```

要使结果更独立于所用的系统，外壳输出的结果被过滤，Macintosh 的 <CR> 被换成 <NL>，而 DOS 系列的系统上 <CR><NL> 也被换成 <NL>。为了避免字符串在 NUL 处被截断，所有的 NUL 的字符被替换为 SOH (0x01)。

使用若干选项，以下面的方法构造要执行的命令:
'shell' 'shellcmdflag' 'shellxquote' {expr} 'shellredir' {tmp} 'shellxquote'
({tmp} 是自动生成的一个文件名)。
Unix 上，{expr} 用大括号包围，以便支持连接的多条命令。

以加工 ("cooked") 模式执行命令，这样 CTRL-C 可以用来中止命令 (至少在 Unix 上是如此)。

返回的错误代码可以在 v:shell_error 里找到。
该函数不能运行于 restricted-mode 。

注意 上面提到的选项值如有错误，该函数就会失败。使用若干安全代理应用时也有报告说它会失败。
不同于 ":!cmd"，没有自动对改变过的文件的检查。使用 :checktime 来强制这种检查。

也可用作 method :
```
:echo GetCmd()->system()
```

systemlist({expr} [, {input}]) systemlist()
和 system() 相同，但返回由行组成的 List (以 NL 分隔的输出各部分)，NUL 转换为 NL。输出的工作方式和 readfile() 带 {binary} 参数设为 "b" 相同，除了结果以 NL 结尾时不会有额外的空项目以外。
注意 MS-Windows 上可能会有拖尾的 CR 字符。

要看到 "echo hello" 和 "echo -n hello" 的区别，参见 system() 和 split() :
```
echo system('echo hello')->split('\n', 1)
```

有错时，返回空串。

也可用作 method :
```
:echo GetCmd()->systemlist()
```

tabpagebuflist([{arg}]) tabpagebuflist()
返回 List ，每个项目是当前标签页里每个窗口相关联的缓冲区的编号。
{arg} 指定使用的标签页的编号。如果省略，使用当前标签页。
如果 {arg} 非法，返回数值零。
要得到所有标签页里的所有缓冲区的列表，这样用:
```
let buflist = []
for i in range(tabpagenr('$'))
   call extend(buflist, tabpagebuflist(i + 1))
endfor
```
注意 缓冲区可能出现于多于一个窗口里。

也可用作 method :
```
GetTabpage()->tabpagebuflist()
```

tabpagenr([{arg}]) tabpagenr()
返回数值，当前标签页号。第一个标签页的编号为 1。
如果可选参数为 "$"，返回最后一个标签页的编号 (即标签页总数)。
该数值可用于 :tab 命令。

tabpagewinnr({tabarg} [, {arg}]) tabpagewinnr()
类似于 winnr() ，但使用标签页 {tabarg}。
{tabarg} 指定要使用的标签页号。
{arg} 的用法类似于 winnr() :
- 如果省略，返回当前窗口号，也就是转到该标签页时会使用的窗口。
- 如果是 "$"，返回窗口的总数。
- 如果是 "#"，返回上次的窗口编号。

用于的例子:
```
    tabpagewinnr(1)          " 标签页 1 的当前窗口
    tabpagewinnr(4, '$')     " 标签页 4 的窗口总数
```
如果 {tabarg} 非法，返回零。

也可用作 method :
```
GetTabpage()->tabpagewinnr()
```

tagfiles()

tagfiles()　返回 List ，当前缓冲区用于搜索的标签文件名。这是 'tags' 选项扩展后的内容。

taglist({expr} [, {filename}])　taglist()

返回匹配正规表达式 {expr} 的标签列表。

如果传入 {filename}，用于像 :tselect 那样给结果排序。见 tag-priority 。{filename} 必须是文件的完整路径。

每个列表项目是一个至少包含以下项目的字典：

name	标签名。
filename	标签定义的文件名。它或者相对于当前目录，或者包含完整路径。
cmd	用于在文件里定位标签的 Ex 命令。
kind	标签类型。该项目的值取决于特定于语言的类型值。只在 Exuberant ctags 或 hdrtag 生成的标签文件里存在。
static	特定于文件的标签。详见 static-tag 。

可能还有一些其它项目，取决于标签文件的内容：access、implementation、inherits 和 signature。这些字段的信息参见 ctags 文档。C 代码里可能出现字段 "struct"、"class" 和 "enum"，它们给出标签所在的实体的名字。

ex 命令 "cmd" 可以是 ex 搜索模式、行号或者行号后跟字节位置。

如果没有匹配的标签，返回空列表。

要得到标签的准确匹配，{expr} 里必须使用 '^' 和 '$'。这也可加快函数的工作速度。
关于标签搜索正规表达式模式的详情见 tag-regexp 。

'tags' 提供 Vim 如何定位标签文件的信息。 tags-file-format 说明不同的 ctags 工具生成的标签文件的格式。

也可用作 method :
```
GetTagpattern()->taglist()
```

tan({expr})　tan()

返回以弧度测量的 {expr} 的正切值。返回值是 [-inf, inf] 区间内的浮点数。
{expr} 的计算结果必须是浮点数或数值。
示例：
```
:echo tan(10)
0.648361
:echo tan(-4.01)
-1.181502
```

也可用作 method :

```
Compute()->tan()
```

{仅当编译时加入 +float 特性才有效}

tanh({expr}) tanh()
返回 {expr} 的双曲正切值，返回值为 [-1, 1] 区间内的浮点数。
{expr} 的计算结果必须是浮点数或数值。
示例：

```
:echo tanh(0.5)
0.462117
:echo tanh(-1)
-0.761594
```

也可用作 method ：

```
Compute()->tanh()
```

{仅当编译时加入 +float 特性才有效}

tempname() tempname() temp-file-name
返回字符串，它是一个不存在的文件名。可以用作临时文件。该文件在至少 26 个接连的调用内不会重复。例如：

```
:let tmpfile = tempname()
:exe "redir > " . tmpfile
```

Unix 上，文件会在用户个人的目录中 tempfile 。
MS-Windows 上，如果置位了 'shellslash' 选项或者 'shellcmdflag' 以 '-' 开始的时候，使用正斜杠。

term_ 函数文档在这里： terminal-functions-details

test_ 函数文档在这里： test-functions-details

timer_info() timer_info([{id}])
返回定时器信息的列表。
给出 {id} 时只返回此定时器的信息。如果定时器 {id} 不存在，返回空列表。
省略 {id} 时，返回所有定时器的信息。

每个定时器的信息以字典形式保存，有以下项目：

"id"	定时器号
"time"	定时器开始时间
"remaining"	定时器剩余时间
"repeat"	定时器激活的次数；-1 代表永远
"callback"	回调
"paused"	如果定时器暂停为 1，否则为 0

也可用作 method ：

```
GetTimer()->timer_info()
```

{仅当编译时加入 +timers 特性才有效}

timer_pause({timer}, {paused}) timer_pause()
暂停或恢复定时器。暂停的定时器在到期是不调用回调。如果经过足够

时间，恢复定时器会使回调几乎被立即调用。

暂停定时器可用于短期停止回调的调用。

如果 {paused} 计算为非零的数值或非空的字符串，暂停定时器，否则恢复定时器。
见 non-zero-arg 。

也可用作 method :
```
GetTimer()->timer_pause(1)
```

{仅当编译时加入 +timers 特性才有效}

timer_start() timer timers

timer_start({time}, {callback} [, {options}])
新建定时器并返回定时器号。

{time} 是毫秒计的等待时间。只是调用回回调的最短时间。系统如果繁忙或 Vim 不在等待输入时，时间会更长。

{callback} 是要调用的函数。可以是函数名或 Funcref 。调用时接受一个参数，即定时器号。只有在 Vim 等待输入时才会调用回调。

{options} 是字典。支持的项目：
"repeat" 重复调用回调的次数。-1 代表永远。无定义时，调用回调一次。
如果定时器连续出错三次，取消重复。这避免了 Vim 因为过多的错误信息导致的不可用的情形。

示例：
```
func MyHandler(timer)
  echo 'Handler called'
endfunc
let timer = timer_start(500, 'MyHandler',
        \ {'repeat': 3})
```
这会以 500 毫秒间隔调用 MyHandle() 三次。

也可用作 method :
```
GetMsec()->timer_start(callback)
```

该命令在沙盘里不可用 sandbox 。
{仅当编译时加入 +timers 特性才有效}

timer_stop({timer}) timer_stop()
停止定时器。不再调用定时器回调。
{timer} 是 timer_start() 返回的定时器号，所以必须是数值。如果 {timer} 不存在，不报错。

也可用作 method :
```
GetTimer()->timer_stop()
```

{仅当编译时加入 +timers 特性才有效}

timer_stopall() timer_stopall()

停止所有定时器。不再调用定时器回调。用于有定时器工作不正常的场合。如果没有任何定时器，不报错。

{仅当编译时加入 +timers 特性才有效}

tolower({expr}) tolower()
返回给出字符串的备份，但所有的大写字符变为小写（就如同在字符串上应用了 gu 一样）。

也可用作 method :
```
GetText()->tolower()
```

toupper({expr}) toupper()
返回给出字符串的备份，但所有的小写字符变为大写（就如同在字符串上应用了 gU 一样）。

也可用作 method :
```
GetText()->toupper()
```

tr({src}, {fromstr}, {tostr}) tr()
返回 {src} 字符串的备份，其中 {fromstr} 里的每个字符被 {tostr} 字符串里同样的位置的字符替代。也就是，{fromstr} 的第一个字符被翻译成 {tostr} 的第一个字符，依此类推。和 unix 命令 "tr" 完全相同。
能正确处理多字节字符。

例如:
```
echo tr("hello there", "ht", "HT")
```
返回 "Hello THere"
```
echo tr("<blob>", "<>", "{}")
```
返回 "{blob}"

也可用作 method :
```
GetText()->tr(from, to)
```

trim({text} [, {mask}]) trim()
返回 {text} 从头部和尾部删除 {mask} 里出现的字符后的字符串。
如果没给出 {mask}，{mask} 是所有直到 0x20 的字符，包括 Tab、空格、NL 和 CR，加上不换行空格 0xa0。
此代码能正确处理多字节字符。

示例:
```
echo trim("   some text ")
```
返回 "some text"
```
echo trim("  \r\t\t\r RESERVE \t\n\x0B\xA0") . "_TAIL"
```
返回 "RESERVE_TAIL"
```
echo trim("rm<Xrm<>X>rrm", "rm<>")
```
返回 "Xrm<>X"（不删除中间部分的字符）

也可用作 method :
```
GetText()->trim()
```

trunc({expr}) trunc()
返回浮点数，即绝对值小于等于 {expr} 的最大整数（向零取整）。

{expr} 的计算结果必须是浮点数或数值。
示例：

```
echo trunc(1.456)
1.0
echo trunc(-5.456)
-5.0
echo trunc(4.0)
4.0
```

也可用作 method ：

```
Compute()->trunc()
```

{仅当编译时加入 +float 特性才有效}

type()

type({expr}) 返回数值，代表 {expr} 的类型。
不要直接使用此值，最好用含有此值的 v:t_ 变量：

数值：	0	v:t_number
字符串：	1	v:t_string
函数引用：	2	v:t_func
列表：	3	v:t_list
字典：	4	v:t_dict
浮点数：	5	v:t_float
布尔值：	6	v:t_bool (v:false 和 v:true)
None:	7	v:t_none (v:null 和 v:none)
作业：	8	v:t_job
通道：	9	v:t_channel
blob:	10	v:t_blob

为了后向兼容，可以这样用此函数：

```
:if type(myvar) == type(0)
:if type(myvar) == type("")
:if type(myvar) == type(function("tr"))
:if type(myvar) == type([])
:if type(myvar) == type({})
:if type(myvar) == type(0.0)
:if type(myvar) == type(v:false)
:if type(myvar) == type(v:none)
```

要检查 v:t_ 这些变量是否存在，可用：

```
:if exists('v:t_number')
```

也可用作 method ：

```
mylist->type()
```

undofile({name}) undofile()
返回用于名为 {name} 的文件的撤销文件名。使用 'undodir' 选项并寻找实际存在的目录。并不检查该撤销文件是否存在。
{name} 总是扩展为完整路径，因为内部是这么使用的。
{name} 为空时 undofile() 也返回空字符串，因为无名缓冲区不写入任何撤销文件。
可用于 :wundo 和 :rundo 。
如果编译时没有 +persistent_undo 选项，总是返回空字符串。

也可用作 method ：

```
GetFilename()->undofile()
```

undotree() undotree()

返回撤销树的当前状态。返回值是包含以下项目的字典：

"seq_last"	最大使用的撤销序列号。
"seq_cur"	撤销树中当前位置的序列号。如果有撤销过的改变，和 "seq_last" 会有不同。
"time_cur"	最近用于 :earlier 和相关命令的时间。可用 strftime() 转换成可读的格式。
"save_last"	最后的文件写入编号。如果没有写入，返回零。
"save_cur"	撤销树当前位置的编号。
"synced"	如果最后的撤销块已经同步，返回非零值。等待用户输入时会发生。见 undo-blocks 。
"entries"	关于撤销块的信息的字典的列表。

"entries" 列表的第一个值是最老的撤销项目。每个列表项目是一个包含以下项目的字典：

"seq"	撤销序列号。和 :undolist 显示的相同。
"time"	改变发生的时间。可用 strftime() 转换成可读的格式。
"newhead"	只出现在最后加入的项目。标识最后的改变，并指示将来的改变加入所在的位置。
"curhead"	只出现在最后撤销的项目。表示撤销树当前的位置，该块可用于 redo 命令。如果最后改变之后没有撤销动作，此项目不出现。
"save"	只出现在文件写入前最后的块。该值为写入计数。首次写入的编号为 1，最后一次是上面提及的 "save_last"。
"alt"	替代项。这又是一个撤销块的列表。每个项目又可以有 "alt" 项目。

uniq({list} [, {func} [, {dict}]]) uniq() E882

原地删除重复相邻的 {list} 项目的第二个及之后的版本。返回 {list} 。如果不想对列表进行修改，先建立一个备份：

```
:let newlist = uniq(copy(mylist))
```

缺省比较函数使用每个项目的字符串表示形式。关于 {func} 和 {dict} 的使用可见 sort() 。

也可用作 method ：

```
mylist->uniq()
```

values({dict}) values()

返回 List ，{dict} 的所有值。 List 项目的顺序不定。另见 items() 和 keys() 。

也可用作 method ：

```
mydict->values()
```

virtcol({expr}) virtcol()

要得到屏幕列的位置，用 virtcol() 。

注意 只能使用当前文件的位置标记。

返回数值，即 {expr} 给定的文件位置的屏幕列号。也就是，该位置的字符占据的最后一个屏幕位置，这里假设屏幕有无限的宽度。如果该位置是一个 <Tab>，返回的数值是 <Tab> 占据的最后一列。比如，如果

<Tab> 在第 1 列，而 'ts' 设为 8 的话，返回 8。忽略 conceal() 因素。
关于字节位置，见 col() 。
{expr} 用法见 col() 。
如果使用 'virtualedit'，{expr} 可以用 [lnum, col, off]，其中 "off" 是字符位置开始计算的屏幕列。例如，制表中或最后一个字符之后的某个位置。"off" 若省略，则假定为零。
如果在当前模式下使用了虚拟编辑，也可能返回行尾之后的位置。
'virtualedit'
可接受的位置是:

.	光标位置
$	光标行的行尾 (返回光标行显示的字符数加 1)
'x	位置标记 x 的位置 (如果该位置标记没有设置，返回 0)
v	可视模式下: 可视区域的开始位置 (光标是结束位置)。如果不在可视模式下，返回当前光标位置。和 '< 不同的是，会被立即更新。

注意 只能使用当前文件的位置标记。
示例:

```
virtcol(".")    文本 "foo^Lbar"，光标在 "^L" 上，返回 5
virtcol("$")    文本 "foo^Lbar"，返回 9
virtcol("'t")   文本 "    there"，'t 在 'h' 上，返回 6
```

第一列为 1。返回 0 代表错误。
一个更高级的示例，显示所有行的最大长度:

```
echo max(map(range(1, line('$')), "virtcol([v:val, '$'])"))
```

也可用作 method :

```
GetPos()->virtcol()
```

visualmode([{expr}]) visualmode()
返回字符串，它描述当前缓冲区最近使用的可视模式。一开始，它返回空字符串，一旦使用了可视模式，返回 "v"、"V" 或 "<CTRL-V>" (单个 CTRL-V 字符)，分别代表面向字符、面向行、和面向列块的可视模式。
例如:

```
:exe "normal " . visualmode()
```

进入和上次相同的可视模式。也可以用于在脚本里根据最近的可视模式采取不同的行动。
如果当前正处于可视模式中，mode() 可得到具体的可视模式 (例如在 :vmap 中可用)。
如果提供 {expr} 并且计算结果是非零数值或者是非空字符串，那么将清除可视模式，并返回旧的值。见 non-zero-arg 。

wildmenumode() wildmenumode()
如果 wildmenu 打开，返回 TRUE ，否则返回 FALSE 。参见 'wildmenu' 和 'wildmode'。
可用于在映射中方便地处理 'wildcharm' 选项。(只对 mapmode-c 映射有意义)。
例如要使 <c-j> 在 wildmode 中等价于 <down>，可用:

```
:cnoremap <expr> <C-j> wildmenumode() ? "\<Down>\<Tab>" : "\<c-j>"
```

(备注，这需要 'wildcharm' 合适的设置配合)。

win_execute({id}, {command} [, {silent}]) win_execute()
类似于 execute() 但在窗口 {id} 的上下文中执行。该窗口会被临时设为当前窗口，不触发自动命令。执行 {command} 时会触发自动命令，这可能有意想不到的副作用。如有必要可用 :noautocmd 。
示例：

```
call win_execute(winid, 'set syntax=python')
```

和 setwinvar() 相同但不触发自动命令也不实际显示语法高亮。
E994
不是所有的命令都允许在弹出窗口中使用。
窗口 {id} 不存在时不报错。

也可用作 method ，基是作为第二个参数传递的：

```
GetCommand()->win_execute(winid)
```

win_findbuf({bufnr}) win_findbuf()
返回包含缓冲区 {bufnr} 的窗口的所有 window-ID 的列表。如果没有这样的窗口，返回空列表。

也可用作 method ：

```
GetBufnr()->win_findbuf()
```

win_getid([{win} [, {tab}]]) win_getid()
返回指定窗口的 window-ID 。
忽略 {win} 则使用当前窗口。
给定 {win} 时，这是窗口号。顶部窗口的窗口号为 1。
忽略 {tab} 则使用当前标签页，否则使用标签页 {tab}。首个标签页的标签页号为一。
如果找不到这样的窗口，返回零。

也可用作 method ：

```
GetWinnr()->win_getid()
```

win_gotoid({expr}) win_gotoid()
转到 ID 为 {expr} 的窗口。可能会改变当前标签页。
如果成功返回 1，如果找不到这样的窗口，返回零。

也可用作 method ：

```
GetWinid()->win_gotoid()
```

win_id2tabwin({expr}) win_id2tabwin()
返回包含 ID 为 {expr} 的窗口的标签页号和窗口号的列表：[tabnr, winnr]。
如果找不到这样的窗口，返回 [0, 0]。

也可用作 method ：

```
GetWinid()->win_id2tabwin()
```

win_id2win({expr}) win_id2win()
返回 ID 为 {expr} 的窗口号。
如果在当前标签页下找不到这样的窗口，返回零。

也可用作 method ：

```
GetWinid()->win_id2win()
```

win_screenpos({nr}) win_screenpos()

返回两个数值的列表，即窗口 {nr} 的屏幕位置：[row, col]。首个窗口的位置总是 [1, 1]，除非它有标签页行，此时为 [2, 1]。
{nr} 可以是窗口号或 window-ID 。
如果在当前标签页找不到窗口，返回 [0, 0]。

也可用作 method :

```
GetWinid()->win_screenpos()
```

win_splitmove({nr}, {target} [, {options}]) win_splitmove()

移动窗口 {nr} 成为窗口 {target} 的新分割窗口。
类似于先移动到 {target}，用 :split 创建新窗口但使用窗口 {nr} 相同的内容，然后关闭 {nr}。

{nr} 和 {target} 都可以是窗口号或 window-ID 。

如果成功返回零，失败返回非零。

{options} 是包含以下可选项的字典：

"vertical"	为 TRUE 时垂直分割，类似于 :vsplit 。
"rightbelow"	为 TRUE 时分割在下方或右方（垂直时）。为 FALSE 时分割在上方或左方（垂直时）。如果未给出，使用 'splitbelow' 和 'splitright'。

也可用作 method :

```
GetWinid()->win_splitmove(target)
```

winbufnr()

winbufnr({nr}) 返回数值，即窗口 {nr} 相关联的缓冲区号。{nr} 可以是窗口号或 window-ID 。
如果 {nr} 为零，返回当前窗口的缓冲区号。
如果窗口 {nr} 不存在，返回 -1。
示例：

```
:echo "当前窗口的文件是 " . bufname(winbufnr(0))
```

也可用作 method :

```
FindWindow()->winbufnr()->bufname()
```

wincol()

wincol() 返回数值，窗口光标的虚拟列。亦即从窗口左侧起算的屏幕列数。最左列为第一列。

winheight({nr}) winheight()

返回数值，窗口 {nr} 的高度。
{nr} 可以是窗口号或 window-ID 。
如果 {nr} 为零，返回当前窗口的高度。如果窗口 {nr} 不存在，返回 -1。
存在的窗口的高度至少为零。
不计入任何窗口工具条行。
示例：

```
:echo "当前窗口有 " . winheight(0) . " 行。"
```

也可用作 method :

```
GetWinid()->winheight()
```

winlayout([{tabnr}]) winlayout()

返回嵌套列表，包含标签页中的窗口布局。

未给出 {tabnr} 则使用当前标签页，否则使用号码为 {tabnr} 的标签页。如果标签页 {tabnr} 找不到，返回空列表。

叶窗口返回：

```
['leaf', {winid}]
```

水平分割的窗口构成一列，它们返回：

```
['col', [{nested list of windows}]]
```

垂直分割的窗口构成一行，它们返回：

```
['row', [{nested list of windows}]]
```

示例：

```
" 标签页中只有一个窗口
:echo winlayout()
['leaf', 1000]
" 两个水平分割的窗口
:echo winlayout()
['col', [['leaf', 1000], ['leaf', 1001]]]
" 三个水平分割的窗口，中间窗口有两个垂直分割的窗口
:echo winlayout(2)
['col', [['leaf', 1002], ['row', ['leaf', 1003],
                    ['leaf', 1001]]], ['leaf', 1000]]
```

也可用作 method ：

```
GetTabnr()->winlayout()
```

winline() winline()

winline() 返回数值，窗口光标所在的屏幕行，亦即，从窗口顶部起算的屏幕行数。第一行返回 1。
如果光标移动，文件的视图会先更新，这可能会导致滚动。

winnr([{arg}]) winnr()

winnr([{arg}]) 返回数值，当前窗口的编号。最上面的窗口的编号为 1。

可选参数 {arg} 支持以下值：

$	返回最后一个窗口的编号（即窗口的总数）。
#	返回最近访问的窗口号（ CTRL-W_p 到的地方）。如果没有上次窗口或它在另一个标签页中，返回 0。
{N}j	当前窗口之下第 N 个窗口的窗口号（ CTRL-W_j 到的地方）。
{N}k	当前窗口之上第 N 个窗口的窗口号（ CTRL-W_k 到的地方）。
{N}h	当前窗口之左第 N 个窗口的窗口号（ CTRL-W_h 到的地方）。
{N}l	当前窗口之右第 N 个窗口的窗口号（ CTRL-W_l 到的地方）。

此编号可用于 CTRL-W_w 和 ":wincmd w" :wincmd 。
另见 tabpagewinnr() 和 win_getid() 。
示例：

```
let window_count = winnr('$')
let prev_window = winnr('#')
let wnum = winnr('3k')
```

也可用作 method :

```
GetWinval()->winnr()
```

winrestcmd()

winrestcmd() 返回 :resize 命令序列，该序列应该能够恢复当前窗口的大小。只有在没有窗口被打开或关闭且当前窗口和标签页都没有改变的时候才能正确工作。
示例:

```
:let cmd = winrestcmd()
:call MessWithWindowSizes()
:exe cmd
```

winrestview()

winrestview({dict})
使用 winsaveview() 返回的 Dictionary 来恢复当前窗口的视图。
注意 {dict} 不需要包含所有的 winsaveview() 返回的值。如果不提供某值，则不恢复相应的设置。所以可以用:

```
:call winrestview({'curswant': 4})
```

它只会设置光标的 curswant 值（垂直移动的光标想移动到的列）到第 5 列（是的，第 5 列），而其它设置保持不变。这可用于手动设置光标位置。

如果你改变了其中的值，结果无法预测。如果窗口大小改变了，结果不会完全一样。

也可用作 method :

```
GetView()->winrestview()
```

winsaveview()

winsaveview() 返回 Dictionary ，包含当前窗口的信息，这些信息可用来恢复视图。
winrestview() 进行视图的恢复。
可用于定义在缓冲区里跳转后想恢复的原来视图的映射。
这里不保存折叠的信息。用 'foldenable' 选项来暂时关闭折叠功能，这样在移动时折叠就不会打开。这可能有副作用。
返回值包括:

lnum	光标行号
col	光标列号（注意：首列为零，和 getpos() 不同）
coladd	'virtualedit' 使用的光标列偏移
curswant	垂直移动使用的列
topline	窗口的第一行
topfill	填充行，只用于比较模式
leftcol	显示的第一列
skipcol	跳过的列数

注意 这里不保存任何选项值。

winwidth({nr}) winwidth()

返回数值，窗口 {nr} 的宽度。
{nr} 可以是窗口号或 window-ID 。
如果 {nr} 为零，返回当前窗口的宽度。如果窗口 {nr} 不存在，返回 -1。
存在的窗口的宽度至少为零。
示例：

```
:echo "当前窗口有 " . winwidth(0) . " 列。"
:if winwidth(0) <= 50
:  50 wincmd |
:endif
```

要得到终端或屏幕大小，可见 'columns' 选项。

也可用作 method ：

```
GetWinid()->winwidth()
```

wordcount() wordcount()

返回字典，包含当前缓冲区的字节/字符/单词的统计。和 g_CTRL-G 包含的信息相同。
返回值包括：

bytes	缓冲区的字节数
chars	缓冲区的字符数
words	缓冲区的单词数
cursor_bytes	光标前的字节数（不在可视模式下）
cursor_chars	光标前的字符数（不在可视模式下）
cursor_words	光标前的单词数（不在可视模式下）
visual_bytes	可视选择的字节数（只在可视模式下）
visual_chars	可视选择的字符数（只在可视模式下）
visual_words	可视选择的单词数（只在可视模式下）

writefile() writefile()

writefile({object}, {fname} [, {flags}])

{object} 为 List 时，把列表写到文件 {fname} 里。列表的项目间以 NL 分隔。每个列表项必须是字符串或数值。
如果 {flags} 包含 "b"，使用二进制模式：最后一个列表项目之后没有 NL，最后的空项目使得文件的末行以 NL 结尾。

{object} 为 Blob 时，把这些字节不加修改地写到文件 {fname} 里。

如果 {flags} 包含 "a"，使用附加模式，写入行附加到文件之后：

```
:call writefile(["foo"], "event.log", "a")
:call writefile(["bar"], "event.log", "a")
```

如果 {flags} 包含 "s"，写入文件后调用 fsync()。在可能的情况下它把文件刷新到盘上。这会多花些时间，但避免系统崩溃时丢失文件的风险。
如果 {flags} 不包含 "S" 或 "s"，'fsync' 选项置位时调用 fsync()。
如果 {flags} 包含 "S"，无论 'fsync' 选项是否置位都不调用 fsync()。

所有的 NL 字符被 NUL 字符代替。
CR 字符的插入需要在把 {list} 传递给 writefile() 之前先做好。
如果可能，覆盖已有的文件。
如果写入失败，返回 -1，否则返回 0。如果文件不能建立或者写入失败，会有错误信息。
另见 readfile() 。
要按字节复制文件：

```
:let fl = readfile("foo", "b")
:call writefile(fl, "foocopy", "b")
```

也可用作 method ：

```
GetText()->writefile("thefile")
```

xor({expr}, {expr}) xor()
对两个参数进行按位异或。参数须转换为数值。列表、字典或浮点数参数会报错。
示例：

```
:let bits = xor(bits, 0x80)
```

也可用作 method ：

```
:let bits = bits->xor(0x80)
```

feature-list
有四种类型的特性：
1. 只有在 Vim 编译时加入才会支持的特性 +feature-list 。例如：

```
:if has("cindent")
```

2. 只有特定条件满足才会支持的特性。例如：

```
:if has("gui_running")
```

has-patch
3. 特定版本之后或在特定版本上并包含了特定补丁。"patch-7.4.248" 特性意味着 Vim 版本是 7.5 或之后，或 7.4 并包含了补丁 248。例如：

```
:if has("patch-7.4.248")
```

注意 包含了补丁 249 但不包含补丁 248 是可能的，但这只适用于 cherrypick 补丁的情况。
注意 此形式只可用于 7.4.237 补丁或之后版本，在这之前，你需要检查补丁号和 v:version。例如 (确定是 version 6.2.148 或更新的版本)：

```
:if v:version > 602 || v:version == 602 && has("patch148")
```

揭示：要知道 Vim 是否支持文件名中的反斜杠 (MS-Windows)，
用：‘if exists(’+shellslash’)‘

acl	编译时加入了 ACL 支持。
all_builtin_terms	编译时打开了所有的内建终端。
amiga	Vim 的 Amiga 版本。
arabic	编译时加入了阿拉伯语的支持。 Arabic 。
arp	编译时加入了 ARP 的支持。(Amiga)。
autocmd	编译时加入了自动命令的支持。(总为真)
autochdir	编译时加入了 ’autochdir’ 的支持
autoservername	自动打开 clientserver

balloon_eval	编译时加入了 balloon-eval 的支持。
balloon_multiline	GUI 支持多行气泡。
beos	Vim 的 BeOS 版本。
browse	编译时加入了 :browse 的支持。使 browse() 可以工作。
browsefilter	编译时加入了 browsefilter 的支持。
bsd	编译时使用了 BSD 家族的 OS (除了 macOS)。
builtin_terms	编译时打开了一些内建终端。
byte_offset	编译时加入了 'statusline' 中对 'o' 的支持。
cindent	编译时加入了 'cindent' 的支持。
clientserver	编译时加入了远程调用的支持。 clientserver 。
clipboard	编译时加入了 'clipboard' 的支持。
clipboard_working	编译时加入了 'clipboard' 的支持且可用。
cmdline_compl	编译时加入了 cmdline-completion 的支持。
cmdline_hist	编译时加入了 cmdline-history 的支持。
cmdline_info	编译时加入了 'showcmd' 和 'ruler' 的支持。
comments	编译时加入了 'comments' 的支持。
compatible	编译时确保和 Vi 非常兼容。
conpty	ConPTY 可用的平台。
cryptv	编译时加入了加密的支持。 encryption 。
cscope	编译时加入了 cscope 的支持。
cursorbind	编译时加入了 'cursorbind' 的支持 (总为真)
debug	编译时定义了 "DEBUG"。
dialog_con	编译时加入了控制台对话框的支持。
dialog_gui	编译时加入了 GUI 对话框的支持。
diff	编译时加入了 vimdiff 和 'diff' 的支持。
digraphs	编译时加入了二合字母的支持。
directx	编译时加入了 DirectX 和 'renderoptions' 的支持。
dnd	编译时加入了 "~ 寄存器的支持 quote_~ 。
ebcdic	在使用 ebcdic 字符集的机器上编译。
emacs_tags	编译时加入了 Emcac 标签的支持。
eval	编译时加入了表达式计算的支持。当然总要打开啦！
ex_extra	+ex_extra (总为真)
extra_search	编译时加入了 'incsearch' 和 'hlsearch' 的支持。
farsi	删除了波斯语的支持 farsi 。
file_in_path	编译时加入了 gf 和 <cfile> 的支持。
filterpipe	'shelltemp' 关闭时，外壳读/写/过滤命令使用管道
find_in_path	编译时加入了头文件搜索 +find_in_path 的支持。
float	编译时加入了 Float 的支持。
fname_case	文件名大小写敏感 (在 Amiga、MS-DOS 和 Windows 本特性不存在)。
folding	编译时加入了 folding 的支持。
footer	编译时加入了 GUI 信息页脚的支持。 gui-footer
fork	编译时决定使用 fork()/exec() 而不是 system()。
gettext	编译时加入了信息翻译 multi-lang 。
gui	编译时加入了 GUI 的支持。
gui_athena	编译时加入了 Athena GUI。
gui_gnome	编译时加入了 Gnome 支持 (同时也定义了 gui_gtk)。
gui_gtk	编译时加入了 GTK+ GUI (任何版本)。
gui_gtk2	编译时加入了 GTK+ 2 GUI (同时也定义了 gui_gtk)。
gui_gtk3	编译时加入了 GTK+ 3 GUI (同时也定义了 gui_gtk)。
gui_mac	编译时加入了 Macintosh GUI。
gui_motif	编译时加入了 Motif GUI。
gui_photon	编译时加入了 Photon GUI。
gui_running	Vim 在 GUI 上运行，或者 GUI 将很快启动。

gui_win32	编译时加入了 MS Windows Win32 GUI。
gui_win32s	同上，使用了 Win32s 系统 (Windows 3.1)
hangul_input	编译时加入了韩语 (Hangul) 输入的支持。 hangul
hpux	Vim 的 HP-UX 版本。
iconv	可以使用 iconv() 进行转换。
insert_expand	编译时加入了插入模式中 CTRL-X 扩展命令的支持。(总为真)
jumplist	编译时加入了 jumplist 的支持。
keymap	编译时加入了 'keymap' 的支持。
lambda	编译时加入了 lambda 的支持。
langmap	编译时加入了 'langmap' 的支持。
libcall	编译时加入了 libcall() 的支持。
linebreak	编译时加入了 'linebreak'、'breakat'、'showbreak' 和 'breakindent' 的支持。
linux	Vim 的 Linux 版本。
lispindent	编译时加入了 lisp 缩进的支持。
listcmds	编译时加入了缓冲区列表 :files 和参数列表 arglist 的命令。
localmap	编译时加入了局部映射和缩写。 :map-local
lua	编译时加入了 Lua 接口 Lua 。
mac	Vim 的 Macintosh 版本，参照 osx
macunix	等同于 osxdarwin
menu	编译时加入了 :menu 的支持。
mksession	编译时加入了 :mksession 的支持。
modify_fname	编译时加入了文件名的修饰符支持。 filename-modifiers (总为真)
mouse	编译时加入了鼠标的支持。
mouse_dec	编译时加入了 Dec 终端的鼠标支持。
mouse_gpm	编译时加入了 gpm (Linux 控制台鼠标) 的支持。
mouse_gpm_enabled	GPM 鼠标可用
mouse_netterm	编译时加入了 netterm 的鼠标支持。
mouse_pterm	编译时加入了 qnx 的鼠标支持。
mouse_sysmouse	编译时加入了 sysmouse 支持 (*BSD 控制台鼠标)
mouse_sgr	编译时加入了 sgr 的鼠标支持。
mouse_urxvt	编译时加入了 urxvt 的鼠标支持。
mouse_xterm	编译时加入了 xterm 的鼠标支持。
mouseshape	编译时加入了 'mouseshape' 的支持。
multi_byte	编译时加入了 'encoding' 的支持 (总为真)
multi_byte_encoding	'encoding' 设为某个多字节的编码。
multi_byte_ime	编译时加入了 IME 输入方法的支持。
multi_lang	编译时加入了多语言的支持。
mzscheme	编译时加入了 MzScheme 接口支持 mzscheme 。
netbeans_enabled	编译时加入了 netbeans 的支持并且已连接上。
netbeans_intg	编译时加入了 netbeans 的支持。
num64	编译时加入了64 位 Number 的支持。
ole	编译时加入了 Win32 OLE automation 的支持。
osx	为 macOS 编译，参照 mac
osxdarwin	为 macOS 编译，带有 mac-darwin-feature
packages	编译时加入了 packages 的支持。
path_extra	编译时加入了 'path' 和 'tags' 上下搜索的支持。
perl	编译时加入了 Perl 接口。
persistent_undo	编译时加入了持久化撤销历史的支持。
postscript	编译时加入了 PostScript 文件打印的支持。
printer	编译时加入了 :hardcopy 的支持。
profile	编译时加入了 :profile 的支持。

python	Python 2.x 接口可用。 has-python
python_compiled	编译时加入了 Python 2.x 接口。 has-python
python_dynamic	Python 2.x 接口已动态载入。 has-python
python3	Python 3.x 接口可用。 has-python
python3_compiled	编译时加入了 Python 3.x 接口。 has-python
python3_dynamic	Python 3.x 接口已动态载入。 has-python
pythonx	编译时加入了 python_x 接口。 has-pythonx
qnx	Vim 的 QNX 版本。
quickfix	编译时加入了 quickfix 的支持。
reltime	编译时加入了 reltime() 的支持。
rightleft	编译时加入了 'rightleft' 的支持。
ruby	编译时加入了 Ruby 接口 ruby 。
scrollbind	编译时加入了 'scrollbind' 的支持。(总为真)
showcmd	编译时加入了 'showcmd' 的支持。
signs	编译时加入了 :sign 的支持。
smartindent	编译时加入了 'smartindent' 的支持。
sound	编译时加入了声音的支持，例如 sound_playevent()
spell	编译时加入了拼写检查的支持 spell 。
startuptime	编译时加入了 --startuptime 支持。
statusline	编译时加入了 'statusline' 和 'rulerformat' 还有 'titlestring' 和 'iconstring' 的特殊格式的支持。
sun	Vim 的 SunOS 版本。
sun_workshop	删除了 Sun workshop 的支持。
syntax	编译时加入了语法高亮的支持 syntax 。
syntax_items	当前缓冲区有激活的语法高亮项目。
system	编译时决定使用 system() 而不是 fork()/exec()。
tag_binary	编译时加入了标签文件的二分搜索 tag-binary-search 。
tag_old_static	删除了老的静态标签的支持，见 tag-old-static 。
tcl	编译时加入了 Tcl 接口。
termguicolors	编译时加入了终端的真彩支持。
terminal	编译时加入 terminal 的支持。
terminfo	编译时决定使用 terminfo 而不是 termcap。
termresponse	编译时加入了 t_RV 和 v:termresponse 的支持。
textobjects	编译时加入了 text-objects 的支持。
textprop	编译时加入了 text-properties 的支持。
tgetent	编译时加入了 tgetent 的支持，可以使用外部 termcap 或 terminfo 文件。
timers	编译时加入了 timer_start() 支持。
title	编译时加入了窗口标题的支持。 'title' 。
toolbar	编译时加入了 gui-toolbar 的支持。
ttyin	输入是终端 (tty)
ttyout	输出是终端 (tty)
unix	Vim 的 Unix 版本。 +unix
unnamedplus	编译时加入了 'clipboard' 对 "unnamedplus" 的支持。
user_commands	用户定义命令支持。(总为真)
vartabs	编译时加入了可变制表位的支持 'vartabstop' 。
vcon	Win32: 有虚拟终端支持，可用 'termguicolors'。另见 +vtp 。
vertsplit	编译时加入了垂直分割窗口的支持 :vsplit 。(总为真)
vim_starting	如果在启动载入脚本的阶段则为真。 startup vim_starting
viminfo	编译时加入了 viminfo 的支持。
vimscript-1	编译时加入了 Vim 脚本版本 1 的支持
vimscript-2	编译时加入了 Vim 脚本版本 2 的支持

vimscript-3	编译时加入了 Vim 脚本版本 3 的支持
virtualedit	编译时加入了 'virtualedit' 选项支持。(总为真)
visual	编译时加入了可视模式的支持。(总为真)
visualextra	编译时加入了附加的可视模式命令支持。(总为真) blockwise-operators 。
vms	Vim 的 VMS 版本。
vreplace	编译时加入了 gR 和 gr 命令支持。(总为真)
vtp	编译时加入了 vcon 支持 +vtp (检查 vcon 可知当前控制台是否支持)。
wildignore	编译时加入了 'wildignore' 选项支持。
wildmenu	编译时加入了 'wildmenu' 选项支持。
win16	旧版本 MS-Windows 3.1 (总为假)
win32	Vim 的 Win32 版本。(MS-Windows 95 及其后的 32 或 64 位版本)。
win32unix	Vim 的 Win32 版本。使用 Unix 文件命名 (Cygwin)
win64	Vim 的 Win64 版本。(MS-Windows 64 位)。
win95	支持 MS-Windows 95/98/ME 的 Win32 版本 (总为假)
winaltkeys	编译时加入了 'winaltkeys' 选项。
windows	编译时加入了多窗口的支持。(总为真)
writebackup	编译时决定缺省打开 'writebackup'。
xfontset	编译时加入了 X 字体集 的支持。 xfontset 。
xim	编译时加入了 X 输入法 的支持。 xim 。
xpm	编译时加入了 pixmap 的支持。
xpm_w32	编译时加入了 Win32 的 pixmap 的支持。(只为后向兼容而保留，用 "xpm" 代替)
xsmp	编译时加入了 X 会话管理 的支持。
xsmp_interact	编译时加入了交互的 X 会话管理 的支持。
xterm_clipboard	编译时加入了 xterm 剪贴板的支持。
xterm_save	编译时加入了保存和恢复 xterm 屏幕的支持。
x11	编译时加入了 X11 的支持。

string-match

字符串里的模式匹配

pattern 说明的正规表达式通常用于寻找缓冲区行的匹配。如果匹配用来在字符串里寻找匹配，几乎所有的功能都相同。唯一的区别是，字符串是作为单行处理的。如果字符串里包含了 "\n" 字符，它并不看作是模式里的换行。它可以匹配模式里的 "\n"，甚至于 "."。示例:

```
:let a = "aaaa\nxxxx"
:echo matchstr(a, "..\n..")
aa
xx
:echo matchstr(a, "a.x")
a
x
```

不要忘记 "^" 只会在字符串的第一个字符匹配，而 "$" 在字符串的最后一个字符匹配。它们不会匹配 "\n" 之后和之前的位置。

---

5. 定义函数 user-functions

可以定义新的函数。调用的方式就像内建函数一样。函数执行一系列 Ex 命令。普通模式下的命令可以用 :normal 命令执行。

函数名须以大写字母开始，以免和内建函数引起混淆。要避免在不同脚本使用相同的名字，避免显见的或者过短的名字。一个好习惯是使用脚本名字作为函数名字的开头，比如 "HTMLcolor()"。

也可以使用花括号，见 curly-braces-names 。 autoload 机制可用于在调用时才提供函数的定义。

local-function
局部于脚本的函数必须以 "s:" 开始。局部于脚本的函数只能在同一脚本和脚本中定义的函数、用户命令和自动命令里调用。也可以在脚本定义的映射里调用该函数，但必须使用 <SID> 而不是 "s:"，如果映射会在脚本之外被扩展的话。
只有局部于脚本的函数，没有局部于缓冲区或局部于窗口的函数。

:fu :function E128 E129 E123

:fu[nction] 列出所有函数和它们的参数。

:fu[nction] {name} 列出 {name} 命名的函数。
{name} 也可以是 Funcref 类型的 Dictionary 项目：
```
:function dict.init
```

:fu[nction] /{pattern} 列出名字匹配 {pattern} 的函数。
列出所有以 "File" 结束的函数的例子：
```
:function /File$
```

:function-verbose
如果 'verbose' 非零，列出函数的同时也显示它上次定义的位置。例如：

```
:verbose function SetFileTypeSH
    function SetFileTypeSH(name)
        Last set from /usr/share/vim/vim-7.0/filetype.vim
```

:verbose-cmd 有更多信息。

E124 E125 E853 E884

:fu[nction][!] {name}([arguments]) [range] [abort] [dict] [closure]
定义 {name} 命名的新函数。函数体在之后的行给出，直到匹配的 :endfunction 为止。

名字必须由字母数字和 '_' 字符组成，而且必须以大写字母或者 "s:" 开头（见上）。注意 "b:" 或 "g:" 是不允许的。（从补丁 7.4.260 开始，如果函数名中有冒号，给出 E884，如 "foo:bar()"。此补丁之前不报错）。

{name} 也可以是 Funcref 类型的 Dictionary 项目：
```
:function dict.init(arg)
```
"dict" 必须是一个已经存在的字典。如果还不存在，项目 "init" 被加入此字典。否则必须提供 [!] 以覆盖已经存在的函数。返回指向一个编号函数的 Funcref 。该函数只能通过 Funcref 引用，没有引用指向它时，该函数会被删除。

E127 E122
如果同名的函数已经存在而且没有使用 [!]，给出错误信息。有一个例外：再次执行脚本时，该脚本中之前定义过的函数会被悄然替代。

如果给出 [!]，已有的函数被悄然替代。如果该函数正在执行期间除外。此时，这是一个错误。
备注：小心使用 !。如果不小心，可能会意外地替代已有的函数。这很难调试。

{arguments} 参见 function-argument 。

:func-range a:firstline a:lastline
如果给出 [range] 参数，则该函数自己能理解并处理行范围。该范围通过 "a:firstline" 和 "a:lastline" 定义。如果没有 [range]，":{range}call" 会在该范围的每一行分别执行该函数，每次光标都定位在处理行的行首。见 function-range-example 。
就像所有的 Ex 命令一样，光标仍然会被移动到范围的首行。

:func-abort
如果给出 [abort] 参数，该函数在遇到错误时立即中止。

:func-dict
如果给出 [dict] 参数，该函数必须通过 Dictionary 的项目才能调用。局部变量 "self" 这时设为该字典。见 Dictionary-function 。

:func-closure E932
加入 [closure] 参数时，函数可以访问外部作用域的变量和参数。通常这被称为闭包。此例中 Bar() 使用 Foo() 作用域的 "x"。即使 Foo() 返回后仍被引用：

```
:function! Foo()
:  let x = 0
:  function! Bar() closure
:    let x += 1
:    return x
:  endfunction
:  return funcref('Bar')
:endfunction

:let F = Foo()
:echo F()
1
:echo F()
2
:echo F()
3
```

function-search-undo
最近使用的搜索模式和重做命令 "." 不会受到函数的影响。这也意味着 :nohlsearch 的效果在函数返回时会被撤销。

:endf :endfunction E126 E193 W22
:endf[unction]
结束函数定义。最好单起一行，没有 [argument]。

[argument] 可以是：

| 命令	下面执行的命令
\n 命令	下面执行的命令
" 注释	总是忽略
其它	忽略，如果 'verbose' 非零给出

警告
对后续命令的支持是 Vim 8.0.0654 加入的，之前任何参数都被悄悄地忽略。

要在 :execute 命令里定义函数，用换行符而不是 :bar ：

```
:exe "func Foo()\necho 'foo'\nendfunc"
```

:delf :delfunction E130 E131 E933

:delf[unction][!] {name}
删除 {name} 命名的函数。
{name} 也可以是 Funcref 类型的 Dictionary 项目：

```
:delfunc dict.init
```

会删除 "dict" 的 "init" 项目。如果没有更多指向它的引用，该函数被删除。
用了 ! 后，即使函数不存在也不报错。

:retu :return E133

:retu[rn] [expr]
从函数返回。如果给出 "[expr]"，计算该表达式的结果成为函数的返回值。如果没有给出 "[expr]"，返回 0。
如果函数退出时没有显式的调用 ":return"，返回 0。
注意 没有不可到达行的检查，因而，如果有命令在 ":return" 之后，不会给出警告。

如果 ":return" 在 :try 之后使用但在匹配的 :finally (如果有的话) 之前的话，":finally" 之后直到匹配的 :endtry 的命令会先执行。该过程反复应用于所有函数内的嵌套 ":try" 块。在最外层 ":endtry" 结束之后才真正返回。

function-argument a:var

参数的定义只要给出它的名字。在函数里，可以使用 "a:name" 来访问 ("a:" 代表参数 (argument))。

a:0 a:1 a:000 E740 ...

可以给出不超过 20 个参数，以逗号分隔。最后，可以给出参数 "..."，意味着可以有更多的参数。在函数里，可以通过 "a:1"、"a:2" 等等访问它们。"a:0" 设为这些附加参数的数目 (可以为 0)。"a:000" 设为包含这些参数的 List 。注意 "a:1" 等同于 "a:000[0]"。

E742

a: 作用域和其中的变量不能修改，它们是固定的。不过，如果使用了复合类型，例如 List 或 Dictionary ，可以改变它们的内容。这样就可以传递给函数一个 List ，让该函数在里面增加项目。如果要确保函数不能修改 List 或 Dictionary ，用 :lockvar 。

可以定义没有参数的函数。但你这时仍然需要提供 ()。

可以在函数体里定义别的函数。

optional-function-argument

可以提供位置命名参数的缺省值。这使得函数调用时它们成为可选参数。位置参数如果在调用时不指定，则使用缺省表达式来初始化。
只用于 :function 声明的函数，而不适用于匿名表达式 expr-lambda 。

示例：

```
function Something(key, value = 10)
   echo a:key .. ": " .. a:value
endfunction
call Something('empty')       "empty: 10"
call Something('key', 20)     "key: 20"
```

参数的缺省表达式在函数调用时而非定义时进行计算。这样就可用函数定义时非法的表达式。该表达式也只在调用中未指定参数时才进行计算。

可传递 v:none 来使用缺省表达式。注意 这意味着参数如果有缺省表达式，你不能传递 v:none 作为正常值。

示例：

```
function Something(a = 10, b = 20, c = 30)
endfunction
call Something(1, v:none, 3)      " b = 20
```

E989

带缺省表达式的可选参数必须在任何必选参数之后出现。"..." 可在所有可选命名参数之后使用。

出现在后面的参数的缺省可引用在它之前的参数，反之不可以。就像所有参数一样，必须使用 "a:" 前缀。

合法的例子：

```
:function Okay(mandatory, optional = a:mandatory)
:endfunction
```

不 合法的例子：

```
:function NoGood(first = a:second, second = 10)
:endfunction
```

如果不使用 "..."，实际给出的参数数目必须等于必选参数的数目。如果使用 "..."，参数的数目可以更多。

local-variables

在函数里，可以使用局部变量。它们在函数返回时就会消失。全局变量的访问需要通过 "g:"。

例如：

```
:function Table(title, ...)
:  echohl Title
:  echo a:title
:  echohl None
:  echo a:0 . " items:"
:  for s in a:000
:    echon ' ' . s
:  endfor
:endfunction
```

该函数这时可以这样调用：

```
call Table("Table", "line1", "line2")
call Table("Empty Table")
```

要返回多于一个值，返回一个 List ：

```
:function Compute(n1, n2)
:  if a:n2 == 0
:    return ["fail", 0]
:  endif
:  return ["ok", a:n1 / a:n2]
:endfunction
```

该函数这时可以这样调用：

```
:let [success, div] = Compute(102, 6)
:if success == "ok"
:  echo div
:endif
```

:cal :call E107 E117

:[range]cal[l] {name}([arguments])

调用函数。函数名和参数通过 :function 指定。可以使用不超过 20 个参数。忽略返回值。

如果没有给出范围而函数又接受范围，该函数被调用一次。如果给出范围，光标在执行函数前定位在该范围的第一行的开始。

如果给出范围但函数自己不能处理之，该函数在范围里的每一行分别执行。光标定位在每个处理行的第一列。光标留在最后一行（但可能被最后一个函数调用移动）。每一行上，参数被重新计算。所以这是可以的：

function-range-example

```
:function Mynumber(arg)
:  echo line(".") . " " . a:arg
:endfunction
:1,5call Mynumber(getline("."))
```

"a:firstline" 和 "a:lastline" 总是有定义的。它们可以用来在范围的开始或结束处进行一些不同的处理。

能处理范围本身的函数示例：

```
:function Cont() range
:  execute (a:firstline + 1) . "," . a:lastline . 's/^/\t\\ '
:endfunction
:4,8call Cont()
```

该函数在范围里的每行开头插入续行符 "\"，除了第一行以外。

如果函数返回复合值，该值可被进一步解除参照（译者注：调用其上的方法），但该范围不能被继续使用。例如：

```
:4,8call GetDict().method()
```

这里 GetDict() 得到范围值，method() 不会。

E132

用户函数的递归调用受到 'maxfuncdepth' 选项的限制。

也可以使用 :eval 。不支持范围，但支持方法的链式调用，例如：

```
eval GetList()->Filter()->append('$')
```

函数也可以在表达式计算的一部分或作为方法调用：

```
let x = GetList()
```

```
let y = GetList()->Filter()
```

自 动 载 入 函 数

autoload-functions

如果使用很多或者很大的函数，可以在需要使用它们的时候才自动提供其定义。有两个方法：用自动命令，还有用 'runtimepath' 里的 "autoload" 目录。

使用自动命令

用户手册 41.14 一节有介绍。

自动命令可用于很长的 Vim 脚本的插件。你可以定义自动命令然后用 :finish 快速退出脚本。这使得 Vim 启动快得多。这时，自动命令应该再次载入相同的文件，并设置变量使得 :finish 命令被跳过。

使用 FuncUndefined 自动命令事件，它需要一个能匹配等待定义的函数的模式。例如：

```
:au FuncUndefined BufNet* source ~/vim/bufnetfuncs.vim
```

文件 "~/vim/bufnetfuncs.vim" 这时应该定义 "BufNet" 开始的函数。另见 FuncUndefined 。

使用 autoload 脚本

autoload E746

用户手册 41.15 一节有介绍。

在 "autoload" 目录里定义脚本更简单，但需要使用准确的文件名。能够自动载入的函数的名字形如：

```
:call filename#funcname()
```

这样的函数如果调用时还没有定义，Vim 在 'runtimepath' 里的 "autoload" 目录搜索脚本文件 "filename.vim"。例如 "~/.vim/autoload/filename.vim"。该文件这时应该这样定义函数：

```
function filename#funcname()
   echo "Done!"
endfunction
```

文件名和函数的 # 之前的名字必须完全匹配，而定义的函数名也必须和调用时使用的形式完全一致。

可以使用子目录。函数名每个 # 相当于路径分隔符。这样，调用函数：

```
:call foo#bar#func()
```

的时候，Vim 寻找 'runtimepath' 里的文件 "autoload/foo/bar.vim"。

也适用于读取还没有设置的变量：

```
:let l = foo#bar#lvar
```

不过，如果 autoload 脚本已经载入，不会为未知的变量再次载入该脚本。

给这样的变量赋值并没有什么特别。这可以用于在载入 autoload 脚本之前给它传递一些设置：

```
:let foo#bar#toggle = 1
:call foo#bar#func()
```

注意 如果你不小心调用了应该在 autoload 脚本里定义，但该脚本实际没有定义的函数时，每次试图对该函数的调用都会重新载入一次脚本。从而每次都会得到错误信息。

还有，注意 如果你有两个脚本文件，不能在使用的函数定义之前同时从一个文件里调用另一个文件里的函数并且从那个文件里调用这个文件的函数。
避免在顶层使用自动载入功能。

提示：如果你发布很多脚本，可以用 vimball 工具把它们捆绑在一起。另请阅读用户手册 distribute-script 。

---

6. 花括号名字 curly-braces-names

多数使用变量的地方可以改用 "花括号名字" 变量。和常规的变量名类似，但可以包含一到多个花括号 {} 包围的表达式，形如：

```
my_{adjective}_variable
```

如果 Vim 遇到这种情形，它会计算花括号内的表达式，把结果放在表达式所在的位置，然后重新解释整个字符串为完整的变量名。所以在上例中，如果变量 "adjective" 设为 "noisy"，那么引用的将是 "my_noisy_variable"。如果 "adjective" 设为 "quiet"，那么引用的将是 "my_quiet_variable"。

一个这种形式的应用是建立一系列变量，由一个选项管理。比如，语句

```
echo my_{&background}_message
```

会显示 "my_dark_message" 或者 "my_light_message" 的内容，取决于 'background' 的当前值。

你可以使用多个花括号对：

```
echo my_{adverb}_{adjective}_message
```

..甚至嵌套使用：

```
echo my_{ad{end_of_word}}_message
```

其中 "end_of_word" 可以是 "verb" 或者 "jective"。

不过，花括号里的表达式必须计算出合法的单个变量名，比如，这不行：

```
:let foo='a + b'
:echo c{foo}d
```

.. 因为扩展的结果是 "ca + bd"，这不是合法的变量名。

curly-braces-function-names

类似的，你可以调用和定义计算的出的函数名。比如：

```
:let func_end='whizz'
:call my_func_{func_end}(parameter)
```

会调用函数 "my_func_whizz(parameter)"。

这样 不 行:

```
:let i = 3
:let @{i} = ''  " 报错
:echo @{i}      " 报错
```

---

7. 命令 expression-commands

:let {var-name} = {expr1} :let E18

设置内部变量 {var-name} 为表达式 {expr1} 的计算结果。该变量也会得到 {expr} 的类型。如果 {var-name} 不存在，它会被创立。

:let {var-name}[{idx}] = {expr1} E689

设置列表项目为表达式 {expr1} 的返回值。{var-name} 必须引用列表而 {idx} 必须是该列表里合法的索引值。嵌套的列表可以重复使用索引。
不能用于给列表 List 增加项目。
不能用来给字符串改变个别字节。为此你可以这么做:

```
:let var = var[0:2] . 'X' . var[4:]
```

如果 {var-name} 是 Blob，{idx} 可以等于 blob 的长度，此时附加一个字节。

E711 E719
:let {var-name}[{idx1}:{idx2}] = {expr1} E708 E709 E710

设置 List 的一系列项目为表达式 {expr1} 的返回值，后者必须是正确数量项目的列表。
{idx1} 可以省略，这时以零代替。
{idx2} 可以省略，这时意味着到列表尾部。
如果选择的项目范围部分越过列表的尾部，会加入新的项目。

:let+= :let-= :letstar=
:let/= :let%= :let.= :let..= E734 E985

:let {var} += {expr1} 类似于 ":let {var} = {var} + {expr1}"。
:let {var} -= {expr1} 类似于 ":let {var} = {var} - {expr1}"。
:let {var} *= {expr1} 类似于 ":let {var} = {var} * {expr1}"。
:let {var} /= {expr1} 类似于 ":let {var} = {var} / {expr1}"。
:let {var} %= {expr1} 类似于 ":let {var} = {var} % {expr1}"。
:let {var} .= {expr1} 类似于 ":let {var} = {var} . {expr1}"。
:let {var} ..= {expr1} 类似于 ":let {var} = {var} .. {expr1}"。
如果 {var} 还没有设置或者 {var} 和 {expr1} 的类型不符合操作符的要求，失败。
Vim 脚本版本 2 或之后不支持 .= ，见 vimscript-version 。

:let ${env-name} = {expr1} :let-environment :let-$

设置环境变量 {env-name} 为表达式 {expr1} 的计算结果。它总是字符串型。

在有的系统中使环境变量为空会把它删除。很多系统不区别不置位的环境变量和空的环境变量。

:let ${env-name} .= {expr1}
把 {expr1} 附加到环境变量 {env-name} 之后。如果该环境变量还不存在，相当于 "="。

:let @{reg-name} = {expr1} :let-register :let-@
把表达式 {expr1} 的计算结果写到寄存器 {reg-name} 里。{reg-name} 必须是单个字符，而且是一个可以写入的寄存器（见 registers ）。"@@" 可以用来访问无名寄存器，而 "@/" 设置搜索模式。
如果 {expr1} 的结果以 <CR> 或 <NL> 结束，该寄存器会成为面向行类型，不然，它会成为面向字符类型。
这可以用来清除最近的搜索模式：

```
:let @/ = ""
```

这和搜索空字符串不同，后者会在任何地方得到匹配。

:let @{reg-name} .= {expr1}
把 {expr1} 附加到寄存器 {reg-name} 之后。如果寄存器为空，相当于把它设为 {expr1} 的值。

:let &{option-name} = {expr1} :let-option :let-&
设置选项 {option-name} 为表达式 {expr1} 的计算结果。字符串或数值类型的值总会被转化为选项需要的类型。
对于局部于窗口或者缓冲区的选项而言，这和 :set 命令的效果相同：局部值和全局值都被改变。
例如：

```
:let &path = &path . ',/usr/local/include'
```

也可用于形如 t_xx 的终端代码。但只可用字母数字形式的名字。例如：

```
:let &t_k1 = "\<Esc>[234;"
```

如果代码还不存在，会新建一个终端键值，因此不会报错。

:let &{option-name} .= {expr1}
对字符串选项：附加 {expr1} 到选项值之后。和 :set+= 不同，不会插入逗号。

:let &{option-name} += {expr1}
:let &{option-name} -= {expr1}
对数值或布尔选项：加减 {expr1}。

:let &l:{option-name} = {expr1}
:let &l:{option-name} .= {expr1}
:let &l:{option-name} += {expr1}
:let &l:{option-name} -= {expr1}
同上，但只设置选项的局部值（如果有的话）。和 :setlocal 类似。

:let &g:{option-name} = {expr1}
:let &g:{option-name} .= {expr1}
:let &g:{option-name} += {expr1}
:let &g:{option-name} -= {expr1}
同上，但只设置选项的全局值（如果有的话）。和 :setglobal 类似。

:let [{name1}, {name2}, ...] = {expr1} :let-unpack E687 E688

{expr1} 计算结果必须是 List 。该列表的第一项赋给 {name1}，第二项给 {name2}，依此类推。
命名的数量必须匹配 List 项目的数量。
每个名字必须是上面提到的 ":let" 命令的项目之一。
例如：

```
:let [s, item] = GetItem(s)
```

细节：先计算 {expr1}，然后按顺序依次进行赋值。如果 {name2} 依赖于 {name1}，该细节就有关系。例如：

```
:let x = [0, 1]
:let i = 0
:let [i, x[i]] = [1, 2]
:echo x
```

结果是 [0, 2]。

```
:let [{name1}, {name2}, ...] .= {expr1}
:let [{name1}, {name2}, ...] += {expr1}
:let [{name1}, {name2}, ...] -= {expr1}
```

同上，但附加/加/减值到每个 List 项目。

```
:let [{name}, ..., ; {lastname}] = {expr1}
```

类似于上面的 :let-unpack ，但 List 可以包含比给出名字的数量更多的项目。列表其余项目赋给 {lastname}。
如果没有余下的项目，{lastname} 设为空列表。
例如：

```
:let [a, b; rest] = ["aval", "bval", 3, 4]
```

```
:let [{name}, ..., ; {lastname}] .= {expr1}
:let [{name}, ..., ; {lastname}] += {expr1}
:let [{name}, ..., ; {lastname}] -= {expr1}
```

同上，但附加/加/减值到每个 List 项目。

:let=<< :let-heredoc
E990 E991 E172 E221

```
:let {var-name} =<< [trim] {endmarker}
text...
text...
{endmarker}
```

设置内部变量 {var-name} 为由 {endmarker} 字符串定界的文本行的列表。
{endmarker} 不能包含空白。
{endmarker} 不能以小写字母开头。
末行必须以 {endmarker} 字符串结束，而不能有其它字符。
小心 {endmarker} 之后的空白！

如果不给出 "trim" 保留文本行中的空白字符。如果 {endmarker} 之前给出 "trim"，则删除缩进，所以可以这样：

```
let text =<< trim END
   if ok
     echo 'done'
   endif
END
```

结果是：["if ok", " echo 'done'", "endif"]
marker 必须和 "let" 对齐，此时从所有文本行中删除首行的

缩进。
具体地说：和首个非空文本行引导缩进完全一致的所有输入行的缩引导进会被删除。
和 let 之前的引导缩进完全一致的包含 {endmarker} 的行的所有的引导缩进也会被删除。
注意 这里空格和制表是有区别的。

如果 {var-name} 还不存在，会被创建。
不能跟在其它命令后面，但后面可以跟注释。

要避免续行符的使用，考虑在 'cpoptions' 中加入 'C'：

```
set cpo+=C
let var =<< END
   \ leading backslash
END
set cpo-=C
```

示例：

```
let var1 =<< END
Sample text 1
    Sample text 2
Sample text 3
END

let data =<< trim DATA
        1 2 3 4
        5 6 7 8
DATA
```

E121

:let {var-name} .. 列出变量 {var-name} 的值。可以给出多个变量的名字。这里识别特殊的名字包括：E738

g:	全局变量
b:	缓冲区的局部变量
w:	窗口的局部变量
t:	标签页的局部变量
s:	脚本的局部变量
l:	函数的局部变量
v:	Vim 变量。

:let 列出所有变量的值。变量的类型在值之前给出：

<空>	字符串
#	数值
*	函数引用

:unl[et][!] {name} ... :unlet :unl E108 E795
删除内部变量 {var-name}。可以给出多个变量的名字。它们都被删除。该名字也可以是 List 或 Dictionary 项目。
如果使用 [!]，即使变量不存在也不会给出错误。
List 里可以删除一到多个项目：

```
:unlet list[3]    " remove fourth item
:unlet list[3:]   " remove fourth item to last
```

Dictionary 里一次只能删除一个项目：

```
:unlet dict['two']
:unlet dict.two
```

这对于清除全局和脚本局部变量很有用（脚本结束时并不自动删除这些变量）。函数局部变量在函数结束时是自动清除的。

:unl[et] ${env-name} ... :unlet-environment :unlet-$

删除环境变量 {env-name}。
在一个 :unlet 命令里可以混合 {name} 和 ${env-name}。
不对不存在的变量报错，即使没有 ! 也是如此。
如果系统不支持删除环境变量，则将它清空。

:cons :const

```
:cons[t] {var-name} = {expr1}
:cons[t] [{name1}, {name2}, ...] = {expr1}
:cons[t] [{name}, ..., ; {lastname}] = {expr1}
:cons[t] {var-name} =<< [trim] {marker}
text...
text...
{marker}
```

和 :let 类似，但设置值后额外地给变量加锁。等价于 :let 之后立即用 :lockvar 给变量加锁，所以：

```
:const x = 1
```

等价于：

```
:let x = 1
:lockvar 1 x
```

可用于保证变量之后不会被修改。

E995

:const 不能用来修改变量：

```
:let x = 1
:const x = 2  " Error!
```

E996

注意 这里不能使用环境变量、选项值和寄存器值，因为它们不能被锁定。

```
:cons[t]
:cons[t] {var-name}
```

如果不给出参数或只给出 {var-name}，等价于 :let 。

:lockv[ar][!] [depth] {name} ... :lockvar :lockv

给内部变量 {name} 加锁。加锁意味着不能再修改该变量（直到它被解锁为止）。
加锁的变量可以删除：

```
:lockvar v
:let v = 'asdf'       " fails!
:unlet v
```

E741 E940

如果试图修改加锁的变量，你会得到错误信息："E741: Value is locked: {name}"。
如果试图加锁或解锁内建变量，会报错："E940: Cannot lock or unlock variable {name}"。

给 List 或 Dictionary 加锁时用到 [depth]。它决定加锁到达的深度：

1 给 List 或 Dictionary 自身加锁。不能增加或者删除项目，但你可以修改它们的值。

2 给这些值加也锁，不能修改项目。如果项目是 List 或 Dictionary，不能增加或删除其中项目，但仍然可以修改项目值。

3 同 2，但又适用于 List / Dictionary 中的 List / Dictionary 项目，更深一层。

缺省的 [depth] 为 2，{name} 是 List 或 Dictionary 时，不能修改项目值。

E743

要使用没有限制的深度，用 [!] 并省略 [depth]。不过，为了捕获循环，设定最大深度为 100。

注意 如果两个变量引用同一个 List 而你锁住其中一个，通过另一个变量来访问 List 也同时被锁住。
例如：

```
:let l = [0, 1, 2, 3]
:let cl = l
:lockvar l
:let cl[1] = 99         " won't work!
```

为了避免这一点，可以给列表建立备份。见 deepcopy()。

:unlo[ckvar][!] [depth] {name} ... :unlockvar :unlo

给内部变量 {name} 解锁。和 :lockvar 刚好相反。

:if {expr1} :if :end :endif :en E171 E579 E580

:en[dif]

如果 {expr} 计算为非零，执行命令直到其后匹配的 ":else" 或者 ":endif" 为止。

从 Vim 版本 4.5 到 5.0，":if" 和 ":endif" 之间的 Ex 命令被忽略。提供这两个命令只是为了后向兼容（译者注，原文如此），以方便未来的扩展。可以嵌套。注意 任何的 ":else" 或 ":elseif" 也被忽略，"else" 部分也一样不会执行。

利用这一点，你可以保持和旧版本的兼容：

```
:if version >= 500
:  版本 5 专用的命令
:endif
```

为了找到 "endif"，仍然需要分析命令。有时，旧版本的 Vim 不能识别新的命令。比如，":silent" 被识别为 ":substitute" 命令。这种情形可以用 ":execute" 来避免：

```
:if version >= 600
:  execute "silent 1,$delete"
:endif
```

注意：":append" 和 ":insert" 命令在 ":if" 和 ":endif" 之间不能正常工作。

:el[se]　　:else　:el　E581　E583
如果这之前的命令没有被执行，执行命令直到其后匹配的 ":else" 或 ":endif"。

:elsei[f] {expr1}　　:elseif　:elsei　E582　E584
":else" ":if" 的缩写，而且无需另一个 ":endif"。

:wh[ile] {expr1}　　:while　:endwhile　:wh　:endw　E170　E585　E588　E733
:endw[hile]
只要 {expr1} 计算的结果非零，重复 ":while" 和 ":endwhile" 之间的命令。
如果发现循环里有命令出错，从 "endwhile" 之后继续执行。
例如：

```
:let lnum = 1
:while lnum <= line("$")
   :call FixLine(lnum)
   :let lnum = lnum + 1
:endwhile
```

注意：":append" 和 ":insert" 命令在 ":while" 和 ":for" 循环里不能正常工作。

:for {var} in {object}　　:for　E690　E732
:endfo[r]　　:endfo　:endfor
为每个 {object} 项目重复执行 ":for" 和 ":endfor" 之间的命令。{object} 可以是 List 或 Blob。变量 {var} 设为每个项目的值。如果循环里某个命令出错，从 "endfor" 之后继续执行。
在循环里修改 {object} 影响使用的项目。如果不希望如此，构建一个备份：

```
:for item in copy(mylist)
```

如果 {object} 为 List 且没有备份，Vim 在为当前项目执行命令前保存列表里下一个项目的引用。这样，删除当前项目不会影响循环的继续。而删除任何后来的项目也会使循环跳过它。这意味着下例可以工作（一个效率低下的清空列表的方法）：

```
for item in mylist
   call remove(mylist, 0)
endfor
```

注意 给 List 调整顺序（例如用 sort() 或 reverse()）可能会有意想不到的效果。

如果 {object} 为 Blob，Vim 总是先建立备份再循环。和 List 不同，对 Blob 的修改不影响循环。

:for [{var1}, {var2}, ...] in {listlist}
:endfo[r]
和上面 ":for" 类似，但每个 {listlist} 项目必须是列表，其中每个项目被依次赋予 {var1}、{var2} 等。例如：

```
:for [lnum, col] in [[1, 3], [2, 5], [3, 8]]
   :echo getline(lnum)[col]
:endfor
```

:continue :con E586

:con[tinue] 在 ":while" 或 ":for" 循环的内部，跳回循环开始的地方。如果在循环内部的 :try 之后但在匹配的 :finally （如果有的话）之前，":finally" 之后，匹配的 :endtry 之前的命令会被先执行。该过程反复应用于所有函数内的嵌套 ":try" 块。在最外层 ":endtry" 结束之后才跳回循环的开始处。

:break :brea E587

:brea[k] 在 ":while" 或 ":for" 循环的内部，跳到相匹配的 ":endwhile" 或 ":endfor" 之后的命令。
如果在循环内部的 :try 之后但在匹配的 :finally （如果有的话）之前，":finally" 之后，匹配的 :endtry 之前的命令会被先执行。该过程反复应用于所有函数内的嵌套 ":try" 块。在最外层 ":endtry" 结束之后才跳到循环之后的命令。

:try :try :endt :endtry E600 E601 E602

:endt[ry] 改变 ":try" 和 ":endtry" 之间命令的错误处理，包括所有执行的内容，":source" 里的命令，函数调用，或者自动命令的激活等。

如果检测到错误或者中断，而其后又跟随了 :finally 命令，执行从 ":finally" 之后继续。否则，或者在那以后遇到了 ":endtry"，则检查是否存在（动态的）往外一层的 ":try" 以及其相应的 ":finally" 等等。然后，脚本的处理被终止。（函数定义里是否有 "abort" 参数都不相干。）
示例：

```
:try | edit too much | finally | echo "cleanup" | endtry
:echo "impossible"      " 到不了这里，脚本在上面已经终止
```

另外，":try" 和 ":endtry" 之间的错误或者中断（动态地）被转换成一个例外。它的捕获过程如同它被 :throw 命令抛出那样（见 :catch ）。这种情况下，脚本的处理不会被终止。

"Vim:Interrupt" 的值用于中断例外。Vim 命令的错误被转换成形如 "Vim({command}):{errmsg}" 的值，其它错误被转换成形如 "Vim:{errmsg}"。这里，{command} 是完整的命令名，而 {errmsg} 是错误例外如果没有被捕获的时候会显示的消息，它总以错误号开始。
示例：

```
:try | sleep 100 | catch /^Vim:Interrupt$/ | endtry
:try | edit | catch /^Vim(edit):E\d\+/ | echo "error" | endtry
```

:cat :catch E603 E604 E605

:cat[ch] /{pattern}/ 匹配 {pattern} 的例外抛出时，如果它没有被前一个 ":catch" 捕获，则执行本语句之后的命令，直到遇到和本 ":catch" 处于同一 :try 块的下一个 :catch 、:finally 或者 :endtry 为止。否则，这些命令被跳过。如果没有提供 {pattern}，所有的错误都会被捕获。
示例：

```
:catch /^Vim:Interrupt$/          " 捕获中断（CTRL-C）
:catch /^Vim\%((\a\+)\)\=:E/      " 捕获所有的 Vim 错误
```

```
:catch /^Vim\%((\a\+)\)\=:/       " 捕获错误和中断
:catch /^Vim(write):/             " 捕获所有 :write 的错误
:catch /^Vim\%((\a\+)\)\=:E123:/ " 捕获错误 E123
:catch /my-exception/             " 捕获用户例外
:catch /.*/                       " 捕获一切
:catch                            " 等同于 /.*/
```

除了 / 以外，也可以用别的字符包围 {pattern}，只要它没有特殊含义（比如 '|' 或 '"'）而且不出现在 {pattern} 里。
关于例外的信息可见 v:exception 。另见 throw-variables 。
注意：依赖 ":catch" 去捕获错误信息的 文本 是不可靠的，因为不同的 locale 的信息可以不同。

:fina[lly] :fina :finally E606 E607

任何匹配的 :try 和本 ":finally" 之间的部分要离开的时候都执行本语句之后的命令，直到遇到匹配的 :endtry 为止。包括这些情形：正常完成且要执行到 ":finally"，通过 :continue 、 :break 、 :finish 或 :return ，或者由于错误或者中断或者例外（见 :throw ）。

:th[row] {expr1} :th :throw E608

计算 {expr1} 然后抛出例外。如果 ":throw" 在 :try 之后但在第一个对应的 :catch 之前使用，它之后的命令被跳过，直到遇到第一个匹配 {expr1} 为止。如果没有这样的 ":catch"，或者如果 ":throw" 在 ":catch" 之后 :finally 之前使用，执行 ":finally"（如果有的话）之后直到匹配的 :endtry 为止的命令。如果本 ":throw" 在 ":finally" 之后之后出现，直到 ":endtry" 为止的命令都被跳过。到达 ":endtry" 的时候，在动态计算的往外一层的 ":try" 块上再次重复本过程（这可能出现在外层调用的函数或者执行的脚本上），直到找到一个匹配的 ":catch"。如果最终该例外没有被捕获，命令处理被终止。
示例：

```
:try | throw "oops" | catch /^oo/ | echo "caught" | endtry
```

注意 ":catch" 可能需要放在单独一行上，因为某些错误可以导致整行在解析时被跳过而无法看到分隔命令的 "|"。

:ec[ho] {expr1} .. :ec :echo

回显每个 {expr1}，以空格分隔。第一个 {expr1} 开启一个新行。另见 :comment 。
使用 "\n" 来开启新行。使用 "\r" 把光标移到第一列。
使用 :echohl 命令的高亮设置。
后面不能跟注释。
示例：

```
:echo "'shell' 的值是 " &shell
```

:echo-redraw

后来的重画可能使消息再次消失。因为 Vim 常常会推迟重画直到整个命令序列执行完为止，这个问题会频繁出现。要避免 ":echo" 之前的命令引起它之后的重画（通常，重画被延迟到有输入的时候才进行），使用 :redraw 命令强制重画。例如：

```
:new | redraw | echo "这里有一个新窗口"
```

:echon

:echon {expr1} .. 回显每个 {expr1}，不附加其它字符。另见 :comment 。
使用 :echohl 命令的高亮设置。
后面不能跟注释。
例如：

```
:echon "'shell' 的值是 " &shell
```

注意 两者的区别："":echo"" 是一个 Vim 命令，而 "":!echo"" 是一个外部的外壳命令：

```
:!echo %                --> filename
```

":!" 的参数被扩展，见 :_% 。

```
:!echo "%"              --> filename or "filename"
```

和前例类似，你是否会看到双引号取决于你的 'shell'。

```
:echo %                 --> nothing
```

'%' 不是一个表达式合法的字符。

```
:echo "%"               --> %
```

只会回显 '%' 字符。

```
:echo expand("%")       --> filename
```

调用 expand() 函数来扩展 '%'。

:echoh :echohl

:echoh[l] {name} 让其后的 :echo 、:echon 和 :echomsg 命令使用高亮组 {name}。也可用于 input() 的提示。示例：

```
:echohl WarningMsg | echo "Don't panic!" | echohl None
```

不要忘记把组设回 "None"。不然其后的 echo 都会被高亮。

:echom :echomsg

:echom[sg] {expr1} .. 回显表达式的结果，将其作为一个真正的消息，并把该消息保存在 message-history 里。
参数之间加入空格，和 :echo 类似。但不可显示的字符只是回显而不会被解释。
这里的分析过程和 :echo 略有不同，而更像 :execute 。
所有的表达式都先经计算后进行连接，然后再进行回显。
如果表达式返回的不是数值或字符串，使用 string() 把它转化为字符串。
应用 :echohl 命令的高亮设置。
示例：

```
:echomsg "It's a Zizzer Zazzer Zuzz, as you can plainly see."
```

:echo-redraw 说明如何避免屏幕重画时消息的消失问题。

:echoe :echoerr

:echoe[rr] {expr1} .. 回显表达式的结果，将其作为一个错误消息，并把该消息保存在 message-history 里。如果用在脚本或函数里，会加入行号。
参数之间加入空格的方法和 :echomsg 类似。如果在 try 条件句里使用，该消息会抛出一个错误例外（见 try-echoerr ）。
示例：

```
:echoerr "This script just failed!"
```

如果你只想要使用 :echohl 高亮的消息。
要得到铃声：

```
:exe "normal \<Esc>"
```

:eval

:eval {expr} 计算 {expr} 并忽略其返回值。例如:

```
:eval Getlist()->Filter()->append('$')
```

因为不使用返回值，表达式假定有副作用。此例中 append() 调用会在缓冲区后附加列表中的文本。它类似于 :call ，但可用于所有表达式。

此命令本可缩短为 :ev 或 :eva ，但不易记，所以不提供。

:exe :execute

:exe[cute] {expr1} .. 计算 {expr1}，返回的字符串作为 Ex 命令执行。
多个参数用空格连接。如果不想有额外的空格，使用 "." 操作符来连接字符串使之成为一个参数。
{expr1} 用作被处理的命令，命令行编辑的键不会被识别。
后面不能跟注释。
示例:

```
:execute "buffer" nextbuf
:execute "normal" count . "w"
```

":execute" 可以用来把命令附加到不能接受 '|' 的命令后面。比如:

```
:execute '!ls' | echo "theend"
```

":execute" 也是一个避免在 Vim 脚本里为 ":normal" 命令输入控制字符的好方法:

```
:execute "normal ixxx\<Esc>"
```

这里给出一个 <Esc> 字符，见 expr-string 。

要谨慎对待文件名中特殊字符的正确转义。 fnameescape() 可用于 Vim 命令， shellescape() 可用于 :! 命令。示例:

```
:execute "e " . fnameescape(filename)
:execute "!ls " . shellescape(filename, 1)
```

注意: 执行的字符串可以是任何命令行，但开始或结束一个 "if"、"while" 和 "for" 命令不能保证没有问题，因为跳过命令时，不执行 ":execute"，Vim 就不能准确找到块开始和结束的地方。另外，"break" 和 "continue" 不应出现在 ":execute" 内部。
下例不能工作，因为 ":execute" 不被执行，Vim 看不见 ":while"，找到 ":endwhile" 时就报错:

```
:if 0
: execute 'while i > 5'
:  echo "test"
: endwhile
:endif
```

但如果执行的字符串里有完整的 "while" 和 "if" 命令就没有问题:

```
:execute 'while i < 5 | echo i | let i = i + 1 | endwhile'
```

:exe-comment

":execute"、":echo" 和 ":echon" 后面不能直接跟注释。因它们把 '"' 看成字符串的开始。但你可以把注释加到 '|' 后面。例如:

```
:echo "foo" | "这是一个注释
```

---

8. 例外处理 exception-handling

Vim 脚本语言包含了例外处理特性。本节解释如何在 Vim 脚本里应用该机制。

Vim 在出错或者中断的时候可以抛出例外。见 catch-errors 和 catch-interrupt 。你也可以显式地使用 ":throw" 命令抛出例外。见 throw-catch 。

TRY 条件句 try-conditionals

例外可以被捕获或者用来激发清理代码的运行。你可以使用 try 条件句来指定 catch 子句 (捕获例外) 和/或 finally 子句 (执行清理)。

try 条件句以 :try 命令开始，以匹配的 :endtry 命令结束。两者之间，你可以使用 :catch 命令开始 catch 子句，或者用 :finally 命令开始 finally 子句。catch 子句可有零到多个，但 finally 子句至多只有一个，且它之后不能再有 catch 子句。catch 子句和 finally 子句之前的行称为 try 块。

```
:try
:  ...
:  ...                              TRY 块
:  ...
:catch /{pattern}/
:  ...
:  ...                              CATCH 子句
:  ...
:catch /{pattern}/
:  ...
:  ...                              CATCH 子句
:  ...
:finally
:  ...
:  ...                              FINALLY 子句
:  ...
:endtry
```

try 子句允许观察代码里是否有例外，并采取合适的行动。try 块里的例外可能被捕获。try 块和 catch 子句里的例外可能引起清理动作。

如果 try 块的执行过程中没有抛出例外，控制转移到 finally 子句。在它执行后，脚本从 ":endtry" 之后的行继续。

如果 try 块的执行过程中抛出了例外，该 try 块其余的行被跳过。例外和 ":catch" 命令的模式参数一一比较。第一个匹配的 ":catch" 之后的 catch 子句被采用，其余的 catch 子句则不会执行。catch 子句在下一个最早遇到的 ":catch"、":finally" 或 ":endtry" 命令结束。这时，finally 子句 (如果有的话) 被执行。当遇到 ":endtry" 的时候，脚本从后面的行继续，一如往常。

如果 try 块抛出的例外不能匹配任何 ":catch" 命令的模式，该例外不能由本 try 条件句捕获，因而不会执行任何的 catch 子句。只有 finally 子句，如果有的话，被采

用。该例外在 finally 子句的执行时被暂时搁置。在 ":endtry" 之后才继续。这样，":endtry" 之后的命令不会被执行，而该例外可以在别的地方捕获，见 try-nesting 。

如果在 catch 子句的执行过程中抛出了另一个错误，catch 子句的其余部分不再执行。新的例外不会和试图和同一个 try 条件句的任何 ":catch" 命令的模式匹配，因而也不会执行任何它的 catch 子句。不过，如果有 finally 子句，它还是会被执行，而在它的执行过程中暂时搁置新的例外。":endtry" 之后的命令也不会执行。而新的例外仍可能在别的地方捕获，见 try-nesting 。

如果在 finally 子句（如果有的话）的执行过程中抛出了另一个错误，finally 子句的其余部分不再执行。如果 finally 子句是因为 try 块或者某个 catch 子句里产生的例外引起的，原先的（被暂时搁置的）例外被放弃。":endtry" 之后的命令也不会执行。而 finally 子句的这个例外被传播，而可以在别的地方捕获，见 try-nesting 。

在 ":while" 循环包含的完整的 try 条件句里的 try 块或者某个 catch 子句里遇到 ":break" 或 ":continue" 时，或者在函数或者被执行的脚本里的 try 条件句里的 try 块或者某个 catch 子句里执行 ":return"（函数）或者 ":finish"（脚本）的时候，也会执行 finally 子句。":break"、":continue"、":return" 或者 ":finish" 在 finally 子句的执行时被暂停，而在遇到 ":endtry" 时继续。不过，如果在执行 finally 子句时抛出例外，它们都被抛弃。

在 ":while" 循环包含的完整的 try 条件句里的 finally 子句里遇到 ":break" 或 ":continue" 时，或者在函数或者被执行的脚本里的 finally 子句里执行 ":return"（函数）或者 ":finish"（脚本）的时候，finally 子句的其余部分被跳过，而 ":break"、":continue"、":return" 或 ":finish" 会如常继续执行。如果 finally 的执行是因为例外或者早先的 try 块或者 catch 子句的 ":break"、":continue"、":return" 或者 ":finish" 引起的，暂停的例外或者命令被放弃。

例子可见 throw-catch 和 try-finally 。

TRY 条件句的嵌套 try-nesting

try 条件句可以任意嵌套。也就是说，完整的 try 条件句可以在另一个 try 条件句的 try 块、某个 catch 子句或者 finally 子句里出现。如果内层的 try 条件句不能捕获它的 try 块抛出的例外，或者在它的某个 catch 子句后者 finally 子句里抛出新的例外的话，那么根据上述规则由外层的 try 条件句继续检查是否能捕获该例外。如果内层 try 条件句在外层 try 条件句的 try 块里，检查外层的 catch 子句，不然只有 finally 子句会被执行。对嵌套的处理而言，内层 try 条件句是直接包含在外层里面，还是外层执行了脚本或者调用了函数，而后者又包含了内层 try 条件句，无关紧要。

如果没有活动的 try 条件句能捕获某个例外，只有它们的 finally 子句会执行。最后，脚本结束它的处理。如果是 ":throw" 命令显式地抛出的未捕获的例外，显示错误信息。对于 Vim 隐含抛出的未捕获的错误或者中断例外，错误信息或者中断信息也会像平常一样显示。

例子可见 throw-catch 。

检查例外处理代码 except-examine

例外处理的代码的编写可能很麻烦。如果你不知道发生了什么，把 'verbose' 设为 13，或者在执行脚本文件时使用 ":13verbose" 命令修饰符。这样，你能看到什么时候例外被抛出、放弃、捕获、或者最终处理。如果详细程度大于等于 14，finally 子句暂停什么也会显示。这些信息在调试模式里也会给出（见 debug-scripts ）。

抛 出 和 捕 获 例 外 throw-catch

你可以抛出任何数值或者字符串作为例外。使用 :throw 命令然后把要抛出的值作为参数传入:

```
:throw 4711
:throw "string"
```

throw-expression

你可以指定表达式参数。该表达式先进行计算，然后抛出其结果:

```
:throw 4705 + strlen("string")
:throw strpart("strings", 0, 6)
```

在计算 ":throw" 命令的参数的时候，也可能会抛出例外。除非它被捕获，不然表达式的计算会被放弃。":throw" 命令这时不会抛出新的例外。

例如:

```
:function! Foo(arg)
:  try
:    throw a:arg
:  catch /foo/
:  endtry
:  return 1
:endfunction
:
:function! Bar()
:  echo "in Bar"
:  return 4710
:endfunction
:
:throw Foo("arrgh") + Bar()
```

这里抛出了 "arrgh"，而不会显示 "in Bar"，因为 Bar() 没有执行。

```
:throw Foo("foo") + Bar()
```

却显示 "in Bar" 并且抛出 4711。

别的接受表达式作为参数的命令也可能因为表达式计算过程的 (未捕获的) 例外而被放弃。例外这时被传播给该命令的调用者。

例如:

```
:if Foo("arrgh")
:  echo "then"
:else
:  echo "else"
:endif
```

这里 "then" 和 "else" 都不会显示。

catch-order

try 条件句里的例外可以用一个或多个 :catch 命令捕获，见 try-conditionals 。每个 ":catch" 命令可以捕获的值通过模式参数指定。捕获匹配的例外时，执行其后的 catch 子句。

例如:

```
:function! Foo(value)
:  try
```

```
:    throw a:value
:  catch /^\d\+$/
:    echo "Number thrown"
:  catch /.*/
:    echo "String thrown"
:  endtry
:endfunction
:
:call Foo(0x1267)
:call Foo('string')
```

第一个 `Foo()` 的调用显示 `"Number thrown"`，第二个 `"String thrown"`。
按照 `":catch"` 命令本身的顺序，依次匹配例外。只用第一个成功匹配。所以，你应该把更专门的 `":catch"` 放在前面。下面的顺序并不合理：

```
:  catch /.*/
:    echo "String thrown"
:  catch /^\d\+$/
:    echo "Number thrown"
```

这里，第一个 `":catch"` 总是会被匹配，所以第二个子句永远不可能被采用。

throw-variables
如果你使用通用的模式捕获到例外，可以通过变量 v:exception 得到准确的例外值：

```
:  catch /^\d\+$/
:    echo "Number thrown.  Value is" v:exception
```

你也许会对在什么地方抛出例外也感兴趣。它被保存在 v:throwpoint 里。注意 `"v:exception"` 和 `"v:throwpoint"` 可用于最近捕获的例外，只要该例外还没有完成处理。
例如：

```
:function! Caught()
:  if v:exception != ""
:    echo 'Caught "' . v:exception . '" in ' . v:throwpoint
:  else
:    echo 'Nothing caught'
:  endif
:endfunction
:
:function! Foo()
:  try
:    try
:      try
:        throw 4711
:      finally
:        call Caught()
:      endtry
:    catch /.*/
:      call Caught()
:      throw "oops"
:    endtry
:  catch /.*/
```

```
:    call Caught()
:  finally
:    call Caught()
:  endtry
:endfunction
:
:call Foo()
```

会显示

```
Nothing caught
Caught "4711" in function Foo, line 4
Caught "oops" in function Foo, line 10
Nothing caught
```

更实际的例子： 下面的命令 ":LineNumber" 显示调用它时，脚本或者函数里的行号：

```
:function! LineNumber()
:    return substitute(v:throwpoint, '.*\D\(\d\+\).*', '\1', "")
:endfunction
:command! LineNumber try | throw "" | catch | echo LineNumber() | endtry
```

try-nested

try 条件句没有捕获的例外可以在包围它的 try 条件句中捕获：

```
:try
:  try
:    throw "foo"
:  catch /foobar/
:    echo "foobar"
:  finally
:    echo "inner finally"
:  endtry
:catch /foo/
:  echo "foo"
:endtry
```

内层的 try 条件句没有捕获例外，只执行了 finally 子句。例外在外层得到捕获。本例显示 "inner finally" 然后是 "foo"。

throw-from-catch

你可以捕获某例外，然后抛出另一个。它在该 catch 子句之外捕获：

```
:function! Foo()
:  throw "foo"
:endfunction
:
:function! Bar()
:  try
:    call Foo()
:  catch /foo/
:    echo "Caught foo, throw bar"
:    throw "bar"
:  endtry
```

```
:endfunction
:
:try
:  call Bar()
:catch /.*/
:  echo "Caught" v:exception
:endtry
```

显示 "Caught foo, throw bar" 然后是 "Caught bar"。

rethrow

Vim 脚本语言没有真正的 rethrow。但可以抛出 "v:exception" 来代替:

```
:function! Bar()
:  try
:    call Foo()
:  catch /.*/
:    echo "Rethrow" v:exception
:    throw v:exception
:  endtry
:endfunction
```

try-echoerr

注意 这个方法不能用来 "rethrow" Vim 错误或者中断例外，因为不能伪造 Vim 的内部例外。试图这么做会产生一个错误例外。你应该抛出自己的例外来说明这种情形。如果你想产生 Vim 的错误例外并包含原来的错误例外的值，可以使用 :echoerr 命令:

```
:try
:  try
:    asdf
:  catch /.*/
:    echoerr v:exception
:  endtry
:catch /.*/
:  echo v:exception
:endtry
```

本代码会显示

```
Vim(echoerr):Vim:E492: Not an editor command:   asdf
```

清 理 代 码 try-finally

脚本经常需要改变全局设定然后结束时恢复之。不过，如果用户按了 CTRL-C 中止脚本，这些设定会处于不一致的状态。如果你处于某脚本的开发阶段而发生了错误或者你显式地抛出例外而没有试图捕获之，也会有相同的情况。用带有 finally 子句的 try 条件句，可以恢复设置，从而解决这个问题。可以保证无论是正常的控制流、出错或者显式的例外 ":throw"、还是被中断，都会执行 finally 子句（注意 try 条件句的错误和中断被转换成例外。如果没有捕获，它们在 finally 子句执行完之后会终止脚本。）
例如:

```
:try
:  let s:saved_ts = &ts
```

```
:  set ts=17
:
:  " 这里执行重要的任务。
:
:finally
:  let &ts = s:saved_ts
:  unlet s:saved_ts
:endtry
```

无论任何函数还是脚本的一部分，只要它需要修改全局设置，而在失败或者成功退出该函数或者脚本部分时需要恢复这些设置，就应该在本地应用本方法。

break-finally

清理代码也适用于 ":continue"、":break"、":return" 或 ":finish" 退出的 try 块或 catch 子句。

例如：

```
:let first = 1
:while 1
:  try
:    if first
:      echo "first"
:      let first = 0
:      continue
:    else
:      throw "second"
:    endif
:  catch /.*/
:    echo v:exception
:    break
:  finally
:    echo "cleanup"
:  endtry
:  echo "still in while"
:endwhile
:echo "end"
```

会显示 "first"、"cleanup"、"second"、"cleanup" 和 "end"。

```
:function! Foo()
:  try
:    return 4711
:  finally
:    echo "cleanup\n"
:  endtry
:  echo "Foo still active"
:endfunction
:
:echo Foo() "returned by Foo"
```

会显示 "cleanup" 和 "4711 returned by Foo"。你不需要在 finally 子句里加上附加的 ":return"。(最终，它会覆盖原来的返回值。)

except-from-finally

finally 子句里可以使用 ":continue"、":break"、":return"、":finish" 或 ":throw"，但不推荐，因为它放弃了 try 条件句的清理工作。不过当然了，finally 子句里仍然可能有中断或者错误例外。

finally 子句的错误引起中断不能正常工作的例子：

```
:try
:  try
:    echo "Press CTRL-C for interrupt"
:    while 1
:    endwhile
:  finally
:    unlet novar
:  endtry
:catch /novar/
:endtry
:echo "Script still running"
:sleep 1
```

如果你需要在 finally 里放入可能出错的命令，考虑捕获或者忽略这些命令的错误，见 catch-errors 和 ignore-errors 。

捕 获 错 误 catch-errors

如果你想捕获特定的错误，你需要把要关注的代码放到 try 块里，然后为该错误消息加入 catch 子句。try 条件句的存在使得所有的错误被转换为例外。不会显示消息，而 v:errmsg 也不会设置。要找到 ":catch" 命令右边的模式，你需要知道错误例外的格式。

错误例外使用如下的格式：

```
Vim({cmdname}):{errmsg}
```

或

```
Vim:{errmsg}
```

{cmdname} 是失败的命令名；第二种形式用于命令名未知的场合。{errmsg} 是错误在 try 条件句发生时，本应产生的错误消息。它总是以大写的 "E" 开始，后面跟两或者三位的错误号，一个冒号和一个空格。

例如：

命令

```
:unlet novar
```

通常产生错误信息

```
E108: No such variable: "novar"
```

它在 try 条件句里被转换为例外

```
Vim(unlet):E108: No such variable: "novar"
```

命令

```
:dwim
```

通常产生错误信息

```
E492: Not an editor command: dwim
```

它在 try 条件句里被转换为例外

```
Vim:E492: Not an editor command: dwim
```

你可以这样捕获所有的 ":unlet" 错误

```
:catch /^Vim(unlet):/
```

或者这样捕获所有拼错命令名字的错误

```
:catch /^Vim:E492:/
```

有的错误信息可能由不同的命令产生：

```
:function nofunc
```

和

```
:delfunction nofunc
```

都会产生错误信息

```
E128: Function name must start with a capital: nofunc
```

它在 try 条件句里被分别转换为例外

```
Vim(function):E128: Function name must start with a capital: nofunc
```

或

```
Vim(delfunction):E128: Function name must start with a capital: nofunc
```

使用下面的模式，你可以根据其号码捕获错误，而不管产生的命令是什么：

```
:catch /^Vim(\a\+):E128:/
```

有些命令，比如

```
:let x = novar
```

产生多个错误信息，这里：

```
E121: Undefined variable: novar
E15: Invalid expression:  novar
```

只有第一个会用做例外的值，因为它是最专门的那个（见 except-several-errors ）。
所以你应该这样捕获它

```
:catch /^Vim(\a\+):E121:/
```

你可以这样捕获所有和名字 "nofunc" 相关的错误

```
:catch /\<nofunc\>/
```

你可以这样捕获 ":write" 和 ":read" 命令产生的所有 Vim 的错误

```
:catch /^Vim(\(write\|read\)):E\d\+:/
```

你可以这样捕获所有的 Vim 错误

```
:catch /^Vim\((\a\+)\)\=:E\d\+:/
```

catch-text

注意：永远不要根据错误信息文本本身捕获错误：

```
:catch /No such variable/
```

只适用于英语的 locale，如果用户用 :language 命令使用别的语言就不行了。不过，
在注释里引用该消息或许有帮助：

```
:catch /^Vim(\a\+):E108:/   " No such variable
```

忽略错误

ignore-errors

你可以通过在本地捕获来忽略某个 Vim 命令的错误：

```
:try
:  write
:catch
:endtry
```

但强烈建议， 不要 使用这种简单的形式，因为它捕获的东西超过你的想象。":write"

命令里，会执行一些自动命令，它们可能引起与写入无关的错误。例如：

```
:au BufWritePre * unlet novar
```

作为脚本的作者，你不应该负责处理这些错误：使用你书写的脚本的用户可能定义了这些自动命令。而你这么做只会屏蔽用户自己的错误。

更好的方法是用

```
:try
:  write
:catch /^Vim(write):/
:endtry
```

这样，只捕获真正的 `write` 错误。总之，只应该捕获你有意忽略的错误。

对于单个不会执行自动命令的命令，你可以用 `":silent!"` 命令来关闭错误到例外的转换：

```
:silent! nunmap k
```

即使在活动的 `try` 条件句里也能这么用。

捕 获 中 断 catch-interrupt

如果有活动的 `try` 条件句，中断 (CTRL-C) 被转换为例外 `"Vim:Interrupt"`。你可以和其他例外一样捕获它。那样，脚本就不会中止。

例如：

```
:function! TASK1()
:  sleep 10
:endfunction

:function! TASK2()
:  sleep 20
:endfunction

:while 1
:  let command = input("Type a command: ")
:  try
:    if command == ""
:      continue
:    elseif command == "END"
:      break
:    elseif command == "TASK1"
:      call TASK1()
:    elseif command == "TASK2"
:      call TASK2()
:    else
:      echo "\nIllegal command:" command
:      continue
:    endif
:  catch /^Vim:Interrupt$/
:    echo "\nCommand interrupted"
:    " Caught the interrupt.  Continue with next prompt.
:  endtry
```

```
:endwhile
```

这里，你可以用 CTRL-C 中止任务；脚本会询问新的命令。如果你在提示上按 CTRL-C，脚本就会中止。

要测试在你脚本的某一行上如果按了 CTRL-C 会发生什么，使用调试模式，然后在那行上执行 >quit 或 >interrupt 。见 debug-scripts 。

捕获一切 catch-all

命令

```
:catch /.*/
:catch //
:catch
```

会捕获一切：错误例外，中断例外和 :throw 命令显式抛出的例外。脚本的顶层可用此捕获所有意料不到的问题。

示例：

```
:try
:
:  " 这里做重要的工作
:
:catch /MyException/
:
:  " 处理未知的问题
:
:catch /^Vim:Interrupt$/
:    echo "脚本被中断"
:catch /.*/
:  echo "内部错误 (" . v:exception . ")"
:  echo " - 发生在 " . v:throwpoint
:endtry
:" 脚本结束
```

注意：捕获一切可能会捕获到比你想得到的更多的错误。所以，强烈建议你只用指定模式参数的 ":catch" 来捕获你真正处理的错误。

例如：捕获一切会使得按 CTRL-C 来中断脚本几乎没有办法：

```
:while 1
:  try
:    sleep 1
:  catch
:  endtry
:endwhile
```

例外和自动命令 except-autocmd

执行自动命令的过程中可以使用例外。例如：

```
:autocmd User x try
```

```
:autocmd User x   throw "Oops!"
:autocmd User x catch
:autocmd User x   echo v:exception
:autocmd User x endtry
:autocmd User x throw "Arrgh!"
:autocmd User x echo "Should not be displayed"
:
:try
:  doautocmd User x
:catch
:  echo v:exception
:endtry
```

会显示 "Oops!" 和 "Arrgh!"。

except-autocmd-Pre

有些命令里，自动命令在命令执行的主要动作之前执行。如果在自动命令的序列中抛出没有捕获的例外，该序列和导致其执行的命令本身被放弃，而例外被传播到命令的调用者那里。

例如：

```
:autocmd BufWritePre * throw "FAIL"
:autocmd BufWritePre * echo "应该不会显示"
:
:try
:  write
:catch
:  echo "Caught:" v:exception "from" v:throwpoint
:endtry
```

这里，":write" 命令不会写入当前编辑的文件（你可以通过查看 'modified' 发现）。因为例外来自 BufWritePre 自动命令，它放弃了 ":write"。然后，该例外被捕获而脚本会显示：

```
Caught: FAIL from BufWrite Auto commands for "*"
```

except-autocmd-Post

有些命令里，自动命令在命令执行的主要动作之后执行。如果主要动作失败，而命令包含在活动的 try 条件句里，将跳过这些自动命令并抛出错误例外，该命令的调用者可以捕获这些例外。

例如：

```
:autocmd BufWritePost * echo "文件被成功写入！"
:
:try
:  write /i/m/p/o/s/s/i/b/l/e
:catch
:  echo v:exception
:endtry
```

只会显示：

```
Vim(write):E212: Can't open file for writing (/i/m/p/o/s/s/i/b/l/e)
```

如果你真想在主要动作失败的时候也执行自动命令的话，在 `catch` 子句里激活自动命令事件。
例如：

```
:autocmd BufWritePre  * set noreadonly
:autocmd BufWritePost * set readonly
:
:try
:  write /i/m/p/o/s/s/i/b/l/e
:catch
:  doautocmd BufWritePost /i/m/p/o/s/s/i/b/l/e
:endtry
```

你也可以用 `":silent!"`：

```
:let x = "ok"
:let v:errmsg = ""
:autocmd BufWritePost * if v:errmsg != ""
:autocmd BufWritePost *   let x = "after fail"
:autocmd BufWritePost * endif
:try
:  silent! write /i/m/p/o/s/s/i/b/l/e
:catch
:endtry
:echo x
```

会显示 `"after fail"`。

如果命令的主要动作没有失败，可以在命令的调用者那里捕获自动命令产生的例外：

```
:autocmd BufWritePost * throw ":-("
:autocmd BufWritePost * echo "这里不应该被显示"
:
:try
:  write
:catch
:  echo v:exception
:endtry
```

except-autocmd-Cmd

有的命令的正常动作可以被自动命令的序列代替。可以在命令的调用者那里捕获该序列产生的例外。
例如：对于 `":write"` 命令，调用者并不知道发生例外时，文件是不是已经被写入。你需要想办法告知调用者。

```
:if !exists("cnt")
:  let cnt = 0
:
:  autocmd BufWriteCmd * if &modified
:  autocmd BufWriteCmd *   let cnt = cnt + 1
:  autocmd BufWriteCmd *   if cnt % 3 == 2
:  autocmd BufWriteCmd *     throw "BufWriteCmdError"
:  autocmd BufWriteCmd *   endif
:  autocmd BufWriteCmd *   write | set nomodified
```

```
:  autocmd BufWriteCmd *   if cnt % 3 == 0
:  autocmd BufWriteCmd *     throw "BufWriteCmdError"
:  autocmd BufWriteCmd *   endif
:  autocmd BufWriteCmd *   echo "File successfully written!"
:  autocmd BufWriteCmd * endif
:endif
:
:try
:       write
:catch /^BufWriteCmdError$/
:  if &modified
:    echo "Error on writing (file contents not changed)"
:  else
:    echo "Error after writing"
:  endif
:catch /^Vim(write):/
:    echo "Error on writing"
:endtry
```

如果脚本在修改后执行了多次，它先显示

```
File successfully written!
```

然后

```
Error on writing (file contents not changed)
```

然后

```
Error after writing
```

等等。

except-autocmd-ill

你不能把一个 `try` 条件句分散到不同事件的自动命令。
下面的代码是非法的构造：

```
:autocmd BufWritePre  * try
:
:autocmd BufWritePost * catch
:autocmd BufWritePost *   echo v:exception
:autocmd BufWritePost * endtry
:
:write
```

例外层次和参数化的例外 except-hier-param

有些编程语言支持使用例外类的层次结构，或者在例外类的对象里传入附加的信息。你可以在 `Vim` 里完成类似的工作。

为了抛出属于某层次的例外，只要抛出完整的类名，部件之间用冒号分隔。比如，在某个数学库里的溢出错误可以抛出字符串 `"EXCEPT:MATHERR:OVERFLOW"`。

如果你想给例外类传递附加的信息，把它加到括号里。比如写入文件 `"myfile"` 时的错误，可以抛出字符串 `"EXCEPT:IO:WRITEERR(myfile)"`。

在 `":catch"` 命令里使用合适的模式，可以捕获你的层次中的基本类或者派生类。括号里的附加信息也可以运用 `":substitute"` 命令从 v:exception 里切出。

例如：

```
:function! CheckRange(a, func)
:  if a:a < 0
```

```
:    throw "EXCEPT:MATHERR:RANGE(" . a:func . ")"
:  endif
:endfunction
:
:function! Add(a, b)
:  call CheckRange(a:a, "Add")
:  call CheckRange(a:b, "Add")
:  let c = a:a + a:b
:  if c < 0
:    throw "EXCEPT:MATHERR:OVERFLOW"
:  endif
:  return c
:endfunction
:
:function! Div(a, b)
:  call CheckRange(a:a, "Div")
:  call CheckRange(a:b, "Div")
:  if (a:b == 0)
:    throw "EXCEPT:MATHERR:ZERODIV"
:  endif
:  return a:a / a:b
:endfunction
:
:function! Write(file)
:  try
:    execute "write" fnameescape(a:file)
:  catch /^Vim(write):/
:    throw "EXCEPT:IO(" . getcwd() . ", " . a:file . "):WRITEERR"
:  endtry
:endfunction
:
:try
:
:  " 一些算术和 I/O
:
:catch /^EXCEPT:MATHERR:RANGE/
:  let function = substitute(v:exception, '.*(\(\a\+\)).*', '\1', "")
:  echo "Range error in" function
:
:catch /^EXCEPT:MATHERR/        " 捕获 OVERFLOW 和 ZERODIV
:  echo "Math error"
:
:catch /^EXCEPT:IO/
:  let dir = substitute(v:exception, '.*(\(.\+\),\s*.\+).*', '\1', "")
:  let file = substitute(v:exception, '.*(.\+,\s*\(.\+\)).*', '\1', "")
:  if file !~ '^/'
:    let file = dir . "/" . file
:  endif
:  echo 'I/O error for "' . file . '"'
:
:catch /^EXCEPT/
:  echo "Unspecified error"
:
:endtry
```

Vim 自己抛出的例外（错误或者按了 CTRL-C）使用扁平的层次：它们都在 "Vim" 类里。你自己不能抛出带有 "Vim" 前缀的例外；它们是 Vim 保留的。

如果已知失败的命令名，Vim 错误例外使用该命令名作为参数。见 catch-errors 。

特 别 之 处

except-compat

例外处理的概念需要产生例外的命令序列被立即中止，而控制转移到 finally 子句和/或 catch 子句。

在 Vim 脚本语言里，有一些情况下脚本和函数在错误后还会继续：在没有 "abort" 标志位的函数或者 ":silent!" 之后的命令里，控制流转到下一行。而在函数外，控制流转到最外层 ":endwhile" 或者 ":endif" 之后的行。另一方面，错误应该可以作为例外被捕获（因而，需要立即被中止）。

这个问题的解决方法是把仅在有活动 try 条件句的时候，把错误转化为例外，并立即中止（如果没有用 ":silent!" 抑制的话）。这不是一个限制，因为（错误）例外只能在活动的 try 条件句里被捕获。如果你需要立即终止而不需要捕获错误的话，只要用一个没有 catch 子句的 try 子句就可以了（你可以用 finally 子句指定终止前执行的清理代码。）

如果没有活动的 try 条件句，使用通常的中止和继续行为，而不是立即中止。这样，保证了与 Vim 6.1 和之前版本编写的脚本的兼容性。

不过，如果在活动的 try 条件句里执行已有的不使用例外处理命令的脚本（或者调用它的一个函数），你也许会改变已有脚本发生错误时的控制流。你会在错误时立即中止并且在新的脚本里捕获错误。如果被执行的脚本通过 ":silent!" 命令抑制了错误（在合适的时候测试 v:errmsg 来检查错误），它的执行路径没有改变。错误也不会转换为例外。（见 :silent 。）所以唯一留下的可能是不关心错误并产生错误信息的脚本。可能，你也不希望在新的脚本里使用这样的代码吧。

except-syntax-err

例外处理命令的语法错误永远不会被它所属的 try 条件句的任何 ":catch" 命令所捕获。不过，还是会执行它的 finally 子句。

例如：

```
:try
:  try
:    throw 4711
:  catch /\(/ " 有语法错误
:    echo "in catch with syntax error"
:  catch
:    echo "inner catch-all"
:  finally
:    echo "inner finally"
:  endtry
:catch
:  echo 'outer catch-all caught "' . v:exception . '"'
:  finally
:    echo "outer finally"
:endtry
```

会显示：

```
    inner finally
    outer catch-all caught "Vim(catch):E54: Unmatched \("
    outer finally
```
原来的例外被丢弃了，抛出的是取而代之的语法错误的错误例外。

except-single-line

":try"、":catch"、":finally" 和 ":endtry" 命令可以放在一行里，但这样如果有语法错误，可能使得 "catch" 行无法被识别。所以，最好不要这么做。

例如：

```
    :try | unlet! foo # | catch | endtry
```
":unlet!" 参数之后的拖尾字符抛出了错误例外，但因此无法看到 ":catch" 和 ":endtry" 命令，从而只能丢弃该错误例外并且显示消息 "E488: Trailing characters"。

except-several-errors

如果多个错误在一个命令里出现，第一个错误信息通常是最专门的，因而它被转换为错误例外。

例如：

```
    echo novar
```
产生

```
    E121: Undefined variable: novar
    E15: Invalid expression: novar
```
try 条件句里错误例外的值是：

```
    Vim(echo):E121: Undefined variable: novar
```
except-syntax-error

不过，如果同一命令在普通错误之后发现了语法错误，语法错误被用作抛出的例外。

例如：

```
    unlet novar #
```
产生

```
    E108: No such variable: "novar"
    E488: Trailing characters
```
try 条件句里错误例外的值是：

```
    Vim(unlet):E488: Trailing characters
```
这么做是因为语法错误可能会以用户意想不到的方式改变执行的路径。例如：

```
    try
        try | unlet novar # | catch | echo v:exception | endtry
    catch /.*/
        echo "outer catch:" v:exception
    endtry
```
显示 "outer catch: Vim(unlet):E488: Trailing characters"，然后给出错误信息 "E600: Missing :endtry"，见 except-single-line 。

---

9. 示例 eval-examples

用二进制显示

```
:" 函数 Nr2Bin() 返回数值的二进制字符串。
:func Nr2Bin(nr)
:  let n = a:nr
:  let r = ""
:  while n
:    let r = '01'[n % 2] . r
:    let n = n / 2
```

```
  :  endwhile
  :  return r
  :endfunc

  :" 函数 String2Hex() 把字符串里的每个字符转换成二进制字符串，用连字符分隔字
  :" 符。
  :func String2Bin(str)
  :  let out = ''
  :  for ix in range(strlen(a:str))
  :    let out = out . '-' . Nr2Bin(char2nr(a:str[ix]))
  :  endfor
  :  return out[1:]
  :endfunc
```

使用示例：

```
  :echo Nr2Bin(32)
```

返回：`"100000"`

```
  :echo String2Bin("32")
```

返回：`"110011-110010"`

给行排序

下例用特定比较函数给行排序。

```
  :func SortBuffer()
  :  let lines = getline(1, '$')
  :  call sort(lines, function("Strcmp"))
  :  call setline(1, lines)
  :endfunction
```

可写为一行程序：

```
  :call setline(1, sort(getline(1, '$'), function("Strcmp")))
```

scanf() 的替代

sscanf

Vim 里没有 `sscanf()` 函数。如果你需要提取一行的部分内容，可以使用 `matchstr()` 和 `substitute()` 完成。本例子说明如何得到从类似 `"foobar.txt, 123, 45"` 的行里提取文件名，行号和列号。

```
   :" 设置匹配模式
   :let mx='\(\f\+\),\s*\(\d\+\),\s*\(\d\+\)'
   :"取得匹配整个表达式的文本部分
   :let l = matchstr(line, mx)
   :"从匹配中提取每个项目
   :let file = substitute(l, mx, '\1', '')
   :let lnum = substitute(l, mx, '\2', '')
   :let col = substitute(l, mx, '\3', '')
```

这里，输入是变量 `"line"`，返回值放在变量 `"file"`、`"lnum"` 和 `"col"` 里。(Michael Geddes 提供的方法)

输出 scriptnames 到字典

scriptnames-dictionary

:scriptnames 命令可用于得到执行过的所有脚本文件的列表。没有等价的函数或变量(因为很少用到)。如果需要操作此列表，可以使用下面的代码：

```
" 把 ":scriptnames" 的输出存放到 scriptnames_output 变量中。
let scriptnames_output = ''
redir => scriptnames_output
silent scriptnames
redir END

" 把输出分拆为行，并对每行进行分析。在 "script" 字典中加入项目。
let scripts = {}
for line in split(scriptnames_output, "\n")
  " 只处理非空白行。
  if line =~ '\S'
    " 获取每行的第一个数字。
    let nr = matchstr(line, '\d\+')
    " 获取文件名，删除脚本号 " 123: "。
    let name = substitute(line, '.\+:\s*', '', '')
    " 在字典中加入项目
    let scripts[nr] = name
  endif
endfor
unlet scriptnames_output
```

---

10. Vim script versions

vimscript-version vimscript-versions scriptversion

随着时间推移，Vim 脚本加入很多新特性。这包括了 Ex 命令、函数、变量类型等等。每个单独的特性都可以用 has() 和 exists() 函数检查。

有时旧功能的语法阻碍了 Vim 的改进。而撤销相关支持又会破坏旧的 Vim 脚本。为此，可用显式的 :scriptversion 命令。当 Vim 脚本和旧版本的 Vim 不兼容时，会显式地报错，而不会出现各种奇怪的失败。

scriptversion-1

:scriptversion 1
: 这是原本的 Vim 脚本，和不使用 :scriptversion 命令一样。可用于把一段代码回到旧语法。这样测试是否支持：

```
has('vimscript-1')
```

scriptversion-2

:scriptversion 2
: 不支持用 "." 进行字符串连接，而用 ".." 代替。
这避免了字典成员访问和浮点数的二义性。现在 ".5" 意味着数值 0.5。

scriptversion-3

:scriptversion 3
: 所有的 vim-variable 必须使用 "v:" 前缀。例如 "version" 不再用作 v:version 的简称，而可以用作正常变量。
同样适用于一些显而易见的名字，如 "count" 等等。

这样测试是否支持：

```
has('vimscript-3')
```

scriptversion-4

:scriptversion 4

不把零开始的数值识别为八进制。在之前的版本中你会看到：

```
echo 017   " 显示 15
echo 018   " 显示 18
```

使用脚本版本 4 后：

```
echo 017   " 显示 17
echo 018   " 显示 18
```

同时，可以在数值内部使用单引号来提高可读性：

```
echo 1'000'000
```

引号必须在数位之间。

这样测试是否支持：

```
has('vimscript-4')
```

---

11. 不包含 +eval 特性

no-eval-feature

如果编译时关闭了 +eval 特性，以上的表达式计算命令都不可用。为了避免因此导致你的 Vim 脚本产生各种错误，":if" 和 ":endif" 命令仍然得到识别。不过 ":if" 的参数和一切 ":if" 和匹配的 ":endif" 之间的内容都被忽略。可以嵌套 ":if" 块，但只允许出现在行首。不识别 ":else" 命令。

下例演示如何在不存在 +eval 特性时不执行命令：

```
:if 1
:  echo "编译加入了表达式求值"
:else
:  echo "你_永远_看不到这条消息"
:endif
```

要在 +eval 特性关闭时才执行命令有两种方法。最简单的是提前退出脚本（或 Vim）：

```
if 1
   echo "带 +eval 执行的命令"
   finish
endif
args  " 不带 +eval 执行的命令
```

如果不想中止脚本的载入可以用一些小技巧，如下例所示：

```
silent! while 0
  set history=111
silent! endwhile
```

+eval 特性可用时，因为 "while 0" 跳过此命令。没有 +eval 特性时，"while 0" 是错误但被安静地忽略，从而执行此命令。

---

12. 沙盘（sandbox）

eval-sandbox sandbox E48

'foldexpr'、'formatexpr'、'includeexpr'、'indentexpr'、'statusline' 和 'foldtext' 选项在沙盘（sandbox）里进行计算。这意味着这些表达式不会产生可怕的副

作用。在模式行上设置这些选项时，以及在标签文件里和命令行上的 CTRL-R = 执行命令时，这项措施提供了一定的安全性。
沙盘也用于 :sandbox 命令。

沙盘里，不允许以下操作：
- 修改缓冲区文本
- 定义或者改变映射、自动命令和用户命令
- 设置若干选项（见 option-summary ）
- 设置若干 v: 变量（见 v:var ） E794
- 执行外壳命令
- 读入或者写到文件
- 跳转到另一缓冲区或者去编辑文件
- 执行 Python、Perl 等命令

这并不能保证 100% 安全，但应该可以挡住大多数攻击。

:san :sandbox
:san[dbox] {cmd} 在沙盘里执行 {cmd}。用于计算可能在模式行里设置的选项，比如 'foldexpr'。

sandbox-option
一些选项包含表达式。对这些表达式进行计算时可能要使用沙盘才能避免安全性的威胁。但沙盘限制较多，所以只有在从不安全的位置设置选项时才会如此。在此上下文中，不安全的位置指：
- 执行当前目录的 .vimrc 或 .exrc 时
- 在沙盘里执行时
- 来自模式行的值
- 执行沙盘里定义的函数时

注意 如果在沙盘里保存选项值然后恢复之，该选项仍然标记为在沙盘里设置。

---

13. 文本锁 textlock

在一些情况下，不允许修改缓冲区里的文本、跳转到其它窗口和一些其它会引起混淆或打断 Vim 正在进行的操作的动作。这主要适用于和 Vim 实际正在进行其它操作的同时发生的事情。例如，'balloonexpr' 的计算可能发生在鼠标指针定位在若干位置的任何时候。

文本锁激活时，不允许：
- 修改缓冲区文本
- 跳转到其它缓冲区或窗口
- 编辑其它文件
- 关闭窗口或者退出 Vim
- 其它

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

进 程 间 通 信 channel

Vim 使用通道（channel）和其他进程通信。
通道使用套接字（socket）或管道 pipe 机制。 socket-interface
作业（job）用来启动进程并与之通信。
Netbeans 接口也使用通道。 netbeans

1. 总览 job-channel-overview
2. 通道演示 channel-demo
3. 打开通道 channel-open
4. 使用 JSON 或 JS 通道 channel-use
5. 通道命令 channel-commands
6. 使用 RAW 或 NL 通道 channel-raw
7. 更多通道函数 channel-more
8. 通道函数详情 channel-functions-details
9. 带通道启动作业 job-start
10. 不带通道启动作业 job-start-nochannel
11. 作业函数 job-functions-details
12. 作业选项 job-options
13. 控制作业 job-control
14. 使用提示缓冲区 prompt-buffer

{仅当编译时带有 +channel 特性时才有通道功能}
可以这样查看：‘has('channel')‘
{仅当编译时带有 +job 特性时才有作业功能}
可以这样查看：‘has('job')‘

---

1. 总览 job-channel-overview

有四种主要的作业类型：
1. 守护进程（daemon），服务于多个 Vim 实例。
   Vim 使用套接字连接。
2. 用于单个 Vim 实例的单个作业，异步操作。
   使用套接字或管道。
3. 用于短期若干工作的作业，异步操作。
   使用套接字或管道。
4. 作为过滤器，同步操作。
   使用管道。

使用套接字请见 job-start 、 job-start-nochannel 和 channel-open 。2 和 3 类中一个或多个作业若使用管道，可见 job-start 。4 类即 ":{range}!cmd" 命令，参见 filter 。

套接字和管道可用以下协议：
RAW 无规范，Vim 不知信息如何终止

NL　　每条信息由 NL（换行）字符终止
JSON　　JSON 编码　json_encode()
JS　　JavaScript 风格类 JSON 的编码　js_encode()

常用组合有：
- 使用 NL 模式的管道连接的作业。例如，运行风格检查器并接收错误与警告。
- 使用 JSON 模式的套接字连接的守护进程。例如，从数据库中查找交叉索引。

---

2. 通道演示　　channel-demo　demoserver.py

需要 Python 支持。演示程序可见
$VIMRUNTIME/tools/demoserver.py
在终端上运行上。让我们称之为 T1。

在另一个终端上运行 Vim。连接到演示服务器：

```
let channel = ch_open('localhost:8765')
```

T1 中可见：

```
=== socket opened ===
```

现在可以给服务器发消息：

```
echo ch_evalexpr(channel, 'hello!')
```

T1 接收到消息，并发送响应给 Vim。
T1 中可见原始的消息流。Vim 发送的是：

```
[1,"hello!"]
```

响应则是：

```
[1,"got it"]
```

每次发送一条新消息时，该数值会递增。

服务器可以给 Vim 发送命令。在 T1 上输入（按文本，包括引号）：

```
["ex","echo 'hi there'"]
```

此时 Vim 中可以看到该消息。要移动光标前进一个单词：

```
["normal","w"]
```

要处理异步通信，需要提供回调函数：

```
func MyHandler(channel, msg)
  echo "from the handler: " . a:msg
endfunc
call ch_sendexpr(channel, 'hello!', {'callback': "MyHandler"})
```

Vim 不会等待响应。这样，服务器就可以稍后发送响应，响应时会调用 MyHandler。

回调除了每次在发送时以外，也可以在打开通道时提供：

```
call ch_close(channel)
let channel = ch_open('localhost:8765', {'callback': "MyHandler"})
call ch_sendexpr(channel, 'hello!')
```

测试通道时可能希望能知道发生了什么。可以告诉 Vim 在日志文件里记录事件：

```
call ch_logfile('channellog', 'w')
```

见　ch_logfile()　。

---

3. 打开通道 channel-open

要打开通道:

```
let channel = ch_open({address} [, {options}])
if ch_status(channel) == "open"
  " use the channel
```

可用 ch_status() 看到通道是否已打开。

{address} 的格式为 "hostname:port"。例如 "localhost:8765"。

{options} 是可选项目的字典: channel-open-options

"mode" 可以是: channel-mode

```
"json" - JSON，见下；最方便的方式。缺省。
"js"   - JS (JavaScript) 编码，比 JSON 经济。
"nl"   - Nl 字符结尾的消息
"raw"  - 原始消息
```

channel-callback E921

"callback" 消息收到时，如果还未被处理，调用本函数。接受两个参数: 通道和收到的消息。例如:

```
func Handle(channel, msg)
  echo 'Received: ' . a:msg
endfunc
let channel = ch_open("localhost:8765", {"callback": "Handle"})
```

"mode" 如果是 "json" 或 "js"，"msg" 参数是转换到 Vim 类型的收到消息的正文。
"mode" 如果是 "nl"，"msg" 参数是单个消息，不包含 NL。
"mode" 如果是 "raw"，"msg" 参数是作为字符串的整个消息。

对所有的回调: 可用 function() 绑定它到参数和/或字典。也可用 "dict.function" 形式来绑定到字典。

只有在 "安全" 时才会调用回调函数，通常是 Vim 等待用户输入字符的时候。Vim 不支持多线程。

close_cb

"close_cb" 通道关闭时，除非调用了 ch_close()，调用本函数。定义如下:

```
func MyCloseHandler(channel)
```

Vim 会调用回调函数先处理完所有数据，然后再调用 close_cb。当此函数调用时，不会再有更多数据传到回调函数。不过，如果回调在让 Vim 检查消息，close_cb 调用时可能还在回调处理中。插件必须以某种方式能处理这种情况，不过知道没有更多数据会传来还是有用的。

channel-drop

"drop" 指定如何丢弃消息:

```
"auto"    没有回调来处理消息时。"close_cb" 也考虑在内。
"never"   所有消息都会保存。
```

channel-noblock

"noblock" 和 job-noblock 效果相同。只适用于写操作。

waittime

"waittime" 等待连接完成的时间，以毫秒计。负数代表永久等待。

缺省为零，不等待。用于本地服务器已经运行的情形。Unix 上 Vim 实际使用 1 毫秒等待时间，在很多系统上这是必要的。对于远程服务服务器，使定更高的值，例如，至少 10 毫秒。

channel-timeout

"timeout" 等待请求处理完成的阻塞时间，例如 ch_evalexpr()。以毫秒计。缺省为 2000 (2 秒)。

"mode" 为 "json" 或 "js" 时，"callback" 是可选的。省略时只有在发送一条消息后才能收到消息。

打开通道后要改变通道选项，用 ch_setoptions() 。参数类似于 ch_open() 所用的参数，但不能指定 "waittime"，因为它只在通道打开时有意义。

例如，可以新增或改变处理函数：

```
call ch_setoptions(channel, {'callback': callback})
```

"callback" 为空 (零或空字符串) 时删除该处理函数。

回调执行后 Vim 会刷新屏幕，并复原光标之前所在的位置。因而，回调无须执行 :redraw 。

可以改变 timeout：

```
call ch_setoptions(channel, {'timeout': msec})
```

channel-close E906

用完通道后，这样可以关闭之：

```
call ch_close(channel)
```

使用套接字时，关闭双向的套接字。使用管道时 (stdin/stdout/stderr)，关闭所有的管道。这可能不是你想要的！用 job_stop() 停止作业可能更好。
放弃所有预读取，不再调用回调。

注意 通道关闭分三个阶段：

- I/O 结束，记录日志："Closing channel"。队列中可能还有消息等待读取或回调。
- 清理预读取，记录日志："Clearing channel"。有些变量还可能引用该通道。
- 释放通道，记录日志："Freeing channel"。

通道如果不能打开，会报错。MS-Windows 和 Unix 有一点区别。Unix 上如果打不开端口，会迅速失败。MS-Windows 上会等上 "waittime"。
E898 E901 E902

读写通道时如果有错，会关闭通道。
E630 E631

---

4. 使用 JSON 或 JS 通道 channel-use

JSON 模式可以以同步方式发送消息：

```
let response = ch_evalexpr(channel, {expr})
```

此时，等待对方回复响应。

JS 模式也是如此，但消息使用 JavasScript 编码。其中的区别，见 js_encode() 。

要发送消息，但不立即处理响应或让通道通过回调来处理响应：

```
call ch_sendexpr(channel, {expr})
```

要发送消息并异步通过指定函数来处理响应：

```
call ch_sendexpr(channel, {expr}, {'callback': Handler})
```

Vim 会通过消息 ID 来匹配请求和响应。一旦收到消息，会调用回调。相同 ID 的后来的响应会被忽略。如果你的服务器返回多个响应，必须用 ID 为零进行发送，它们都会被通道回调收到。

{expr} 转换为 JSON 格式，以数据形式出现。例如，{expr} 如果是字符串 "hello"，接收方可能会收到消息：

```
[12,"hello"]
```

JSON 发送的格式是：

```
[{number},{expr}]
```

{number} 每次都不同。对应的响应（如果有的话）必须使用同样的数值：

```
[{number},{response}]
```

这样 Vim 就会知道哪个发送的消息对应哪个收到的消息，并调用正确的处理函数。即使消息接收的顺序打乱也不要紧。

JSON 文本以换行符结尾。可用于分隔读取的文本。例如，Python 中：

```
splitidx = read_text.find('\n')
message = read_text[:splitidx]
rest = read_text[splitidx + 1:]
```

发送者必须发送合法的 JSON 给 Vim。Vim 会分析 JSON 来检查消息是否完整结尾。只有完整结尾的消息才会被接受。消息结尾的换行符是可选的。

一个进程如果没有先收到消息，而要主动发送消息给 Vim 时，应使用数值零：

```
[0,{response}]
```

此时通道处理函数会把 {response} 转换为 Vim 类型。如果通道没有处理函数，该消息被丢弃。

JSON 或 JS 通道也可以用 ch_sendraw() 和 ch_evalraw()。调用者必须自行负责正确的编码和解码。

---

5. 通道命令 channel-commands

通过 JSON 通道，进程可以向 Vim 发送一些 Vim 内部可以处理，无需通道处理函数的命令。

可能命令包括： E903 E904 E905

```
["redraw", {forced}]
["ex",     {Ex 命令}]
["normal", {普通模式命令}]
["expr",   {expression}, {number}]
["expr",   {expression}]
["call",   {函数名}, {参数列表}, {number}]
["call",   {函数名}, {参数列表}]
```

适用于所有以上命令：小心命令的后果！很容易干扰用户正在进行的操作。要避免麻烦，通过 mode() 检查编辑器是否在你期待的状态下。例如，要发送键序列作为文本插入，而不是作为命令执行：

```
["ex","if mode() == 'i' | call feedkeys('ClassName') | endif"]
```

这些命令产生的错误通常不向用户报告，以免弄乱用户的显示。如果你确实想看到，把 'verbose' 选项设为 3 或更高。

"redraw" 命令

其它命令特意不刷新屏幕，以便你发送若干命令而不会移动光标。有些命令的副作用仍可能导致重画。要显示改变的文本并移动光标到其恰当的位置，必须用 'redraw" 命令结束。

参数通常是空字符串：

```
["redraw", ""]
```

要先清屏，传入 "force"：

```
["redraw", "force"]
```

"ex" 命令

"ex" 命令可用以执行任意 Ex 命令。没有完成或报错的任何响应。可以调用 autoload 脚本内的函数：

```
["ex","call myscript#MyFunc(arg)"]
```

也可以用 "call feedkeys() " 插入任何键序列。

如果有错，通道日志（如果设置的话）会有记录。v:errmsg 设为错误信息。

"normal" 命令

"normal" 命令的执行如同 ":normal!"，命令不被映射。例如，要打开光标下的折叠：

```
["normal" "zO"]
```

带响应的 "expr" 命令

"expr" 命令可用于计算表达式求值。例如，要得到当然缓冲区的行数：

```
["expr","line('$')", -2]
```

它会把表达式计算的结果送回：

```
[-2, "last line"]
```

格式是：

```
[{number}, {result}]
```

{number} 和请求中的数值对应。使用负数以避免和 Vim 发送的信息有冲突。每个请求使用不同的数值有助于把响应的请求对应起来。

{result} 是计算的结果，以 JSON 编码。如果计算失败，或者结果不能以 JSON 编码，则返回字符串 "ERROR"。

不带响应的 "expr" 命令

和上面的 "expr" 命令类似，但不需要返回任何响应。例如：
```
["expr","setline('$', ['one', 'two', 'three'])"]
```
请求中没有第三个参数。

"call" 命令

和 "expr" 类似，但不是把整个表达式作为一个字符串传递，而是分别传递函数名和一个参数列表。这样有助于避免把参数经过转义、连接等步骤转换为字符串的过程。例如：
```
["call", "line", ["$"], -2]
```

如果不需要响应，不用提供第四个参数：
```
["call", "setline", ["$", ["one", "two", "three"]]]
```

---

6. 使用 RAW 或 NL 通道 channel-raw

使用 RAW 或 NL 模式时，可以这样发送信息：
```
let response = ch_evalraw(channel, {string})
```

{string} 直接传送。响应也是通道读取的直接结果。既然 Vim 不知道如何识别信息怎样结尾，你需要自己处理。超时只及于首个字节的读取，之后将不再等待。

模式为 "nl" 时，发送信息的方式也类似。每个消息之后你应该加上 NL。这样你可以一次发送多个 NL 结尾的信息。响应为到首个 NL（包含）为止的文本。也包括只有 NL 的空响应。如果通道超时还没有收到 NL，返回空字符串。

要发送信息并不等待响应：
```
call ch_sendraw(channel, {string})
```
进程会发返响应，会调用通道处理函数。

可发送信息并使响应异步地被指定函数处理：
```
call ch_sendraw(channel, {string}, {'callback': 'MyHandler'})
```

{string} 也可以是 JSON，用 json_encode() 可创建，json_decode() 可用来处理收到的 JSON 信息。

原始通道不可使用 ch_evalexpr() 或 ch_sendexpr() 。

Vim 的字符串不可含有 NUL 字符。要收发 NUL 字符，请见 in_io-buffer 和 out_io-buffer 。

---

7. 更多通道函数 channel-more

要获知通道的状态：ch_status(channel)。可能的值为：

"fail"	通道打开失败。
"open"	通道可用。
"buffered"	通道已关闭，但还有待读的数据。
"closed"	通道已关闭。

要获知通道相关的作业：ch_getjob(channel)

要从通道读取一条信息：

```
let output = ch_read(channel)
```

这里使用通道的 timeout。要读取不带超时以获得所有所用的信息：

```
let output = ch_read(channel, {'timeout': 0})
```

如果无信息可读，结果在 JSON 或 JS 模式通道下为 v:none，RAW 或 NL 通道下为空字符串。可以用 ch_canread() 检查是否有信息可读。

注意 如果没有回调，消息会被丢弃。要避免之，给通道加入一个关闭回调函数。

要读取 RAW 通道所有可用的输出：

```
let output = ch_readraw(channel)
```

可读取错误输出：

```
let output = ch_readraw(channel, {"part": "err"})
```

ch_read() 和 ch_readraw() 都适用通道 timeout。如果在该时间内没有任何消息可读，返回空字符串。要指定其他的超时，使用以毫秒计的 "timeout" 选项：

```
{"timeout": 123}
```

要读取错误输出，使用 "part" 选项：

```
{"part": "err"}
```

要在 JS 或 JSON 通道上读取指定 ID 的信息：

```
{"id": 99}
```

如果没有给出 ID 或 ID 为 -1，返回首个信息。这里的优先级高于等待该信息的回调。

RAW 通道这里返回任何可用的信息，因为 Vim 不知道信息如何终止。
NL 通道这里返回一条信息。
JS 或 JSON 通道这里返回一条解码的信息。
包含任何序列号。

---

8. 通道函数详情 channel-functions-details

ch_canread({handle}) ch_canread()

如果从 {handle} 有内容可读，返回非零。
{handle} 可以是通道或带有通道的作业。

用于在合适的时间才从通道读取的场合，例如在定时器中。

注意 如果通道没有回调，这些消息被丢弃。要避免这一点，加上关闭回调函数。

也可用作 method :

```
GetChannel()->ch_canread()
```

ch_close({handle}) ch_close()

关闭 {handle}。见 channel-close 。
{handle} 可以是通道或带有通道的作业。
不调用关闭回调函数。

也可用作 method :

```
GetChannel()->ch_close()
```

ch_close_in({handle}) ch_close_in()
只关闭 {handle} 的 "in" 部分。见 channel-close-in 。
{handle} 可以是通道或带有通道的作业。
不调用关闭回调函数。

也可用作 method :
```
GetChannel()->ch_close_in()
```

ch_evalexpr({handle}, {expr} [, {options}]) ch_evalexpr()
发送 {expr} 到 {handle} 上。{expr} 用通道类型进行编码。不能于原始通道。见 channel-use 。
{handle} 可以是通道或带有通道的作业。
E917
{options} 必须是字典。不能有 "callback" 项目。可以有 "timeout" 项目来指明此特定请求的超时。

ch_evalexpr() 等待响应并返回经解码的表达式。如果有错或超时，返回空字符串。

注意 在等待响应时，Vim 在处理其它的信息。要确保这不会有问题。

也可用作 method :
```
GetChannel()->ch_evalexpr(expr)
```

ch_evalraw({handle}, {string} [, {options}]) ch_evalraw()
发送 {expr} 到 {handle} 上。
{handle} 可以是通道或带有通道的作业。

类似于 ch_evalexpr() ，但不对请求编码也不对响应结果解码。调用者负责内容的正确性。也不对 NL 模式的通道附加换行符，调用者须自行负责。响应中的 NL 则被清除。
注意 Vim 不能判定原始通道上的文本何时被完整地收到。它只返回第一部分，你需用 ch_readraw() 来读取其余的部分。
见 channel-use 。

也可用作 method :
```
GetChannel()->ch_evalraw(rawstring)
```

ch_getbufnr({handle}, {what}) ch_getbufnr()
返回 {handle} 用于 {what} 的缓冲区号。
{handle} 可以是通道或带有通道的作业。
{what} 可以是 "err"，代表 stderr，"out" 代表 stdout，或空，代表套接字输出。
如果没有这样的缓冲区，返回 -1。

也可用作 method :
```
GetChannel()->ch_getbufnr(what)
```

ch_getjob({channel}) ch_getjob()
取得 {channel} 相关的作业。
如果没有这样的作业，在返回的作业上调用 job_status() 会返回 "fail"。

也可用作 method :
GetChannel()->ch_getjob()

ch_info({handle}) ch_info()
返回 {handle} 相关信息的一个字典。其项目为:
"id" 通道号
"status" "open"、"buffered" 或 "closed"，可见 ch_status()
用于 ch_open() 打开的通道时:
"hostname" 地址的机器名
"port" 地址的端口号
"sock_status" "open" 或 "closed"
"sock_mode" "NL"、"RAW"、"JSON" 或 "JS"
"sock_io" "socket"
"sock_timeout" 毫秒计的超时
用于 job_start() 打开的通道时:
"out_status" "open"、"buffered" 或 "closed"
"out_mode" "NL"、"RAW"、"JSON" 或 "JS"
"out_io" "null"、"pipe"、"file" 或 "buffer"
"out_timeout" 毫秒计的超时
"err_status" "open"、"buffered" 或 "closed"
"err_mode" "NL"、"RAW"、"JSON" 或 "JS"
"err_io" "out"、"null"、"pipe"、"file" 或 "buffer"
"err_timeout" 毫秒计的超时
"in_status" "open" 或 "closed"
"in_mode" "NL"、"RAW"、"JSON" 或 "JS"
"in_io" "null"、"pipe"、"file" 或 "buffer"
"in_timeout" 毫秒计的超时

也可用作 method :
GetChannel()->ch_info()

ch_log({msg} [, {handle}]) ch_log()
如果用 ch_logfile() 打开日志，把 {msg} 写入通道日志文件中。
如果传入 {handle}，使用该通道号写入信息。
{handle} 可以是通道或带有通道的作业。所用的通道号对应的通道必须已打开。

也可用作 method :
'did something'->ch_log()

ch_logfile({fname} [, {mode}]) ch_logfile()
启动通道活动记录，写入 {fname}。
如果 {fname} 为空字符串，停止记录。

如果 {mode} 省略或为 "a"，附加到文件。
如果 {mode} 为 "w"，用新文件开始。

用 ch_log() 来写入日志信息。每条消息后都会刷新文件，Unix 上可用 "tail -f" 实时监控。

此函数在 sandbox 中不可用。
注意：通道在文件中保存通信见容，小心其中可能包含机密和敏感的隐私信息，如你在终端窗口中输入的密码等。

也可用作 method :
```
'logfile'->ch_logfile('w')
```

ch_open({address} [, {options}]) ch_open()
打开通道到 {address}。见 channel 。
返回通道。错误检查可用 ch_status() 。

{address} 形如 "hostname:port"，例如 "localhost:8765"。

{options} 如果给出，必须是 Dictionary 。
见 channel-open-options 。

也可用作 method :
```
GetAddress()->ch_open()
```

ch_read({handle} [, {options}]) ch_read()
从 {handle} 读取，返回收到的消息。
{handle} 可以是通道或带有通道的作业。
NL 通道会等待 NL 的到来，除非没有内容可读了 (通道已被关闭)。
见 channel-more 。

也可用作 method :
```
GetChannel()->ch_read()
```

ch_readblob({handle} [, {options}]) ch_readblob()
和 ch_read() 类似，但读取二进制数据并返回 Blob 。
见 channel-more 。

也可用作 method :
```
GetChannel()->ch_readblob()
```

ch_readraw({handle} [, {options}]) ch_readraw()
类似于 ch_read()，但 JS 和 JSON 通道不对消息解码。NL 通道不阻塞等待 NL 的到来，其余和 ch_read() 类似。
见 channel-more 。

也可用作 method :
```
GetChannel()->ch_readraw()
```

ch_sendexpr({handle}, {expr} [, {options}]) ch_sendexpr()
发送 {expr} 到 {handle} 上。{expr} 用通道类型进行编码。不能于原始通道。见 channel-use 。 E912
{handle} 可以是通道或带有通道的作业。

也可用作 method :
```
GetChannel()->ch_sendexpr(expr)
```

ch_sendraw({handle}, {expr} [, {options}]) ch_sendraw()

发送 String 或 Blob {expr} 到 {handle} 上。
类似于 ch_sendexpr() ，但不对请求编码也不对响应结果解码。调用者负责内容的正确性。也不对 NL 模式的通道附加换行符，调用者须自行负责。响应中的 NL 则被清除。
见 channel-use 。

也可用作 method :
```
GetChannel()->ch_sendraw(rawexpr)
```

ch_setoptions({handle}, {options}) ch_setoptions()

设置 {handle} 的选项:

"callback"	通道回调函数
"timeout"	缺省读操作以毫秒计的超时
"mode"	整个通道所用的模式

更多的解释可见 ch_open() 。
{handle} 可以是通道或带有通道的作业。

注意 改变模式可能会丢失正在排队的消息。

以下选项不可改变:

"waittime"	只适用于 ch_open()

也可用作 method :
```
GetChannel()->ch_setoptions(options)
```

ch_status({handle} [, {options}]) ch_status()

返回 {handle} 的状态:

"fail"	打开通道失败
"open"	通道可用
"buffered"	通道可读，不可写
"closed"	通道不可用

{handle} 可以是通道或带有通道的作业。
"buffered" 用于通道已关闭但还有数据用 ch_read() 可读的情况。

{options} 给出时，可以包含 "part" 项目，指定通道要返回状态的部分: "out" 还是 "err"。例如，要得到错误输出的状态:
```
ch_status(job, {"part": "err"})
```

也可用作 method :
```
GetChannel()->ch_status()
```

---

9. 带通道启动作业 job-start job

要启动一个作业并打开对应 stdin/stdout/stderr 的通道:
```
let job = job_start(command, {options})
```

可以这样取得对应的通道：

```
let channel = job_getchannel(job)
```

此处，通道使用 NL 模式。如果想用其它模式，最好在 {options} 里指定。如果之后再改变模式，部分文本可能已经被接收而没有得到正确地处理。

如果你要处理命令生成的输出行，给出 stdout 的处理函数：

```
let job = job_start(command, {"out_cb": "MyHandler"})
```

该函数调用时会被提供通道和一条信息。可以这样定义该函数：

```
func MyHandler(channel, msg)
```

如果没有处理函数，你需要通过 ch_read() 或 ch_readraw() 读取输出。这一步可以在关闭回调函数进行。见 read-in-close-cb 。

注意 如果作业在你读取输出结果之前结束，输出可能会丢失。这取决于系统 (Unix 上是如此，因为管道的写入端的关闭使得读取端读到 EOF)。要避免之，作业退出之前可进行短暂地休眠。

"out_cb" 定义的处理函数不会读 stderr。如果另外要处理错误，加入 "err_cb" 处理函数：

```
let job = job_start(command, {"out_cb": "MyHandler",
      \                       "err_cb": "ErrHandler"})
```

如果要用相同的处理函数同时处理 stderr 和 stdout，使用 "callback" 选项：

```
let job = job_start(command, {"callback": "MyHandler"})
```

取决于系统，启动新的作业可能把 Vim 放到后台，使启动的作业获得焦点。要避免之，可使用 foreground() 函数。但如果调用太早，可能无效，可在回调处理函数中使用，或使用定时器使之推迟到作业启动后。

可以用 ch_evalraw() 给命令发送信息。如果通道使用 JSON 或 JS 模式，可用 ch_evalexpr()。

可用选项见 job-options 。
例如，要启动作业并把其输出写入缓冲区 "dummy"：

```
let logjob = job_start("tail -f /tmp/log",
                      \ {'out_io': 'buffer', 'out_name': 'dummy'})
sbuf dummy
```

作业从缓冲区获得输入

in_io-buffer

要运行的作业从缓冲区中读取内容：

```
let job = job_start({command},
    \ {'in_io': 'buffer', 'in_name': 'mybuffer'})
```

E915 E918

缓冲区是通过名字来查找的，类似于 bufnr() 。该缓冲区必须存在并在 job_start() 调用时已载入。

缺省读入整个缓冲区。可通过 "in_top" 和 "in_bot" 选项改变。

一个特殊的模式是 "in_top" 设为零而 "in_bot" 不设置：每次缓冲区加入一行新行时，倒数第二行被发送到作业的 stdin。这样你可以编辑末行并在回车时将它发送。

channel-close-in

如果不使用这种特殊模式，管道或套接字会在最后一行写入后关闭。这样就告诉了读入端输入已经完成。也可用 ch_close_in() 在更早之前关闭。

文本中的 NUL 字节会被传给作业 (Vim 内部用 NL 字节保存)。

在关闭回调中读取作业输出

read-in-close-cb

如果作业运行需时而你不需要中间结果，可以加入关闭回调函数，在那里读取输出：

```
func! CloseHandler(channel)
  while ch_status(a:channel, {'part': 'out'}) == 'buffered'
    echomsg ch_read(a:channel)
  endwhile
endfunc
let job = job_start(command, {'close_cb': 'CloseHandler'})
```

你要做的事情大概会比 "echomsg" 有用些吧。

---

10. 不带通道启动作业

job-start-nochannel

要启动别处的进程而不建立通道：

```
let job = job_start(command,
    \ {"in_io": "null", "out_io": "null", "err_io": "null"})
```

{command} 会在后台启动，Vim 不等待其运行结束。

如果 Vim 看到 stdin、stdout 或 stderr 没有一个被连接，则不会建立通道。一般，你需要进行重定向以保证命令不会被卡住。

可用选项见 job-options 。

job-start-if-needed

要想在不能连接某个地址的时候才启动某个作业，可以这么做：

```
let channel = ch_open(address, {"waittime": 0})
if ch_status(channel) == "fail"
  let job = job_start(command)
  let channel = ch_open(address, {"waittime": 1000})
endif
```

注意 这里 ch_open() 的 waittime 给作业一秒钟时间使端口可用。

---

11. 作业函数

job-functions-details

job_getchannel({job}) job_getchannel()

获取 {job} 使用的通道句柄。

要确认作业没有通道：

```
if string(job_getchannel()) == 'channel fail'
```

也可用作 method ：

GetJob()->job_getchannel()

job_info([{job}]) job_info()

返回关于 {job} 信息的字典：

"status"	job_status() 返回值
"channel"	job_getchannel() 返回值
"cmd"	启动作业的命令行参数列表
"process"	进程 ID
"tty_in"	终端输入名，如果没有则为空
"tty_out"	终端输出名，如果没有则为空
"exitval"	"status" 为 "dead" 时才有效
"exit_cb"	退出时调用的函数
"stoponexit"	job-stoponexit

仅用于 Unix：

"termsig"	终止程序的信号（相关值见 job_stop() ） 仅当 "status" 为 "dead" 时才有意义

仅用于 MS-Windows：

"tty_type"	使用的虚拟控制台类型。 可选值是 "winpty" 或 "conpty"。 见 'termwintype'。

无参数时返回所有作业对象的列表。

也可用作 method ：

```
GetJob()->job_info()
```

job_setoptions({job}, {options}) job_setoptions()

改变 {job} 的选项。目前支持：

"stoponexit"	job-stoponexit
"exit_cb"	job-exit_cb

也可用作 method ：

```
GetJob()->job_setoptions(options)
```

job_start({command} [, {options}]) job_start()

启动作业，返回 Job 对象。不同于 system() 和 :!cmd ，不等待作业完成。
要在终端窗口中启动作业，可见 term_start() 。

如果作业启动失败，在返回的 Job 对象上调用 job_status() 会报告 "fail"，而且不会调用任何回调。

{command} 可以是字符串。对 MS-Windows 最佳。Unix 上先切为空白分隔的部分，然后传给 execvp()。而双引号内的参数可能包含空格。

{command} 可以是列表，首项是可执行文件，后面诸项为参数。所有项目转换为字符串。这对 Unix 最佳。

MS-Windows 上，job_start() 启动隐藏的 GUI 应用。要使之可见，用 :!start 代替。

命令直接执行，不通过外壳，不使用 'shell' 选项。要使用外壳：

```
let job = job_start(["/bin/sh", "-c", "echo hello"])
```

或：

```
let job = job_start('/bin/sh -c "echo hello"')
```

注意 这样会启动两个过程，外壳及其启动的命令。如果不希望如此，可用 "exec" 外壳命令。

Unix 上仅当命令本身不包含斜杠时，才用 $PATH 来搜索可执行文件。

作业使用和 Vim 相同的终端。如果从 stdin 读入，该作业和 Vim 会竞争输入，这样不行。要避免此问题，重定向 stdin 和 stdout：

```
let job = job_start(['sh', '-c', "myserver </dev/null >/dev/null"])
```

返回的 Job 对象可用以通过 job_status() 获取状态，和通过 job_stop() 停止作业。

注意 作业对象如果没有被引用会被删除。此时 stdin 和 stderr 会被关闭，这可能会使作业失败报错。要避免之，总是保留对作业的引用。所以，不要这样：

```
call job_start('my-command')
```

而要：

```
let myjob = job_start('my-command')
```

不再需要作业，或者已经不介意作业是否会失败之后（如启动时已显示了信息）才可以 unlet "myjob"。记住函数返回后局部于函数的变量不复存在。如有需要，可用局部于脚本的变量：

```
let s:myjob = job_start('my-command')
```

{options} 必须是字典。可包含许多可选项目，见 job-options 。

也可用作 method ：

```
BuildCommand()->job_start()
```

job_status({job}) job_status() E916

返回字符串，返回 {job} 的状态：

"run"	作业运行中
"fail"	作业无法启动
"dead"	作业启动后结束或被终止

Unix 上不存在的命令会报告 "dead" 而不是 "fail"，因为在检测到问题之前，fork 操作已经发生。

如果用 "exit_cb" 选项设置了退出回调，而作业已检测到处于 "dead" 状态，调用该回调。

更多详情可见 job_info() 。

也可用作 method ：

```
GetJob()->job_status()
```

job_stop({job} [, {how}]) job_stop()

停止 {job}。也用于给作业民发送信号。

如果 {how} 省略或是 "term"，作业被终止。Unix 上发送 SIGTERM。
MS-Windows 上强制终止该作业（没有所谓 "温柔的" 方式）。
应用在整个进程组上，因而子进程也受影响。

Unix 上的效果：

"term"	SIGTERM（缺省）
"hup"	SIGHUP
"quit"	SIGQUIT
"int"	SIGINT
"kill"	SIGKILL（最强的停止方式）
数值	该数值对应的信号

MS-Windows 上的效果：

"term"	强制终止进程（缺省）
"hup"	CTRL_BREAK
"quit"	CTRL_BREAK
"int"	CTRL_C
"kill"	强制终止进程
Others	CTRL_BREAK

Unix 上发送信号到进程组。这意味着如果作业是 "sh -c command"，
外壳和命令同时受影响。

返回数值：如果操作可执行返回 1，如果系统不支持 "how"，返回 0。
注意 即使已执行了操作，作业停止与否需用 job_status() 检查。

如果作业的状态是 "dead"，不发送信号。这是为了避免给错误的作业
发送信号（尤其在 Unix 上，进程号是循环利用的）。

使用 "kill" 后，Vim 假定作业会终止，因而将通道关闭。

也可用作 method：

```
GetJob()->job_stop()
```

---

12. 作业选项 job-options

job_start() 的 {options} 参数是一个字典。所有的项目都是可选的。有些选项可以在
作业启动后通过 job_setoptions(job, {options}) 给出。许多选项可通过
ch_setoptions(channel, {options}) 对作业相关的通道给出。
见 job_setoptions() 和 ch_setoptions()。

in_mode out_mode err_mode

"in_mode"	stdin 特定的模式，仅适用于管道情形
"out_mode"	stdout 特定的模式，仅适用于管道情形
"err_mode"	stderr 特定的模式，仅适用于管道情形 可用值参见 channel-mode。 注意：设置 "mode" 时，覆盖部分特定的模式。因而，先设置 "mode"，再设置部分特定的模式。

备注：写入文件或缓冲区和从缓冲区读取时，缺省使用 NL 模式。

job-noblock

"noblock": 1
写入时使用非阻塞式的写调用。避免 Vim 在处理其它信息之间，比如作业把数据发还给 Vim 的时候，长时间等待，这意味着 ch_sendraw() 返回时未必所有的数据已经写回。
8.1.0350 补丁加入此选项，可如此测试之：

```
if has("patch-8.1.350")
  let options['noblock'] = 1
endif
```

job-callback

"callback": handler
通道任何部分有读取内容时的回调。

job-out_cb out_cb

"out_cb": handler
stdout 有读取内容时的回调。仅适用于通道使用管道时。如果不设 "out_cb"，使用通道本身的回调。
两个参数是通道和信息。

job-err_cb err_cb

"err_cb": handler
stderr 有读取内容时的回调。仅适用于通道使用管道时。如果不设 "err_cb"，使用通道本身的回调。
两个参数是通道和信息。

job-close_cb

"close_cb": handler
通道关闭时的回调。与 ch_open() 的 "close_cb" 相同，见 close_cb 。

job-drop

"drop": when
指定何时丢弃消息。与 ch_open() 的 "drop" 相同，见 channel-drop 。如用 "auto"，不考虑 exit_cb。

job-exit_cb

"exit_cb": handler
作业结束时的回调。参数是作业和退出状态。
对退出的作业，Vim 作最多每秒十次的检查。也可以通过 job_status() 调用触发检查，这样也会调用 exit_cb 处理函数。
备注 数据可能会缓冲，进程结束后还可能调用回调。

job-timeout

"timeout": time
等待阻塞请求，如 ch_evalexpr() 等的时间，以毫秒计。缺省是 2000 (2 秒)。

out_timeout err_timeout

"out_timeout": time
stdout 所用的超时。仅适用于管道情形。
"err_timeout": time
stderr 所用的超时。仅适用于管道情形。
注意：设置 "timeout" 时，覆盖部分特定的模式。因而，先设置 "timeout"，再设置部分特定的模式。

job-stoponexit

"stoponexit": {signal}
Vim 结束时给作业发信号 {signal}。可取值参见 job_stop() 。
"stoponexit": ""
Vim 结束时不结束作业。
缺省是 "term"。

job-term

"term": "open"
在新窗口中启动终端并重定向作业的 stdin/stdout/stderr 到终端。和 :terminal 类似。

注意：尚未实现！

`"channel": {channel}`	使用已有的通道，而不重新创建。 新作业所用的通道部分会从之前的使用者处断开。如果通道还在被其它的作业使用，这可能会导致 `I/O` 错误。 已有的回调和其它设置不变。
`"pty": 1`	如有可能，使用 `pty`（伪终端）而非管道。最常和终端窗口组合使用，见 `terminal` 。 {仅在 `Unix` 和类似系统中才有效}

job-in_io in_top in_bot in_name in_buf

`"in_io": "null"`	不连接 `stdin`（从 `/dev/null` 读取）
`"in_io": "pipe"`	连接 `stdin` 到通道（缺省）
`"in_io": "file"`	`stdin` 从文件读取
`"in_io": "buffer"`	`stdin` 从缓冲区读取
`"in_top": number`	用 `"buffer"` 时：发送的首行（缺省：1）
`"in_bot": number`	用 `"buffer"` 时：发送的末行（缺省：最后一行）
`"in_name": "/path/file"`	读取的文件或缓冲区名
`"in_buf": number`	读取的缓冲区号

job-out_io out_name out_buf

`"out_io": "null"`	不连接 `stdout`（写到 `/dev/null`）
`"out_io": "pipe"`	连接 `stdout` 到通道（缺省）
`"out_io": "file"`	`stdout` 写入文件
`"out_io": "buffer"`	`stdout` 附加到缓冲区（见下）
`"out_name": "/path/file"`	写入的文件或缓冲区名
`"out_buf": number`	写入的缓冲区号
`"out_modifiable": 0`	写入到缓冲区时，关闭 `'modifiable'`（见下）
`"out_msg": 0`	写入新缓冲区时，将首行置为 `"Reading from channel error..."`

job-err_io err_name err_buf

`"err_io": "out"`	`stderr` 信息写到 `stdout`
`"err_io": "null"`	不连接 `stderr`（写到 `/dev/null`）
`"err_io": "pipe"`	连接 `stderr` 到通道（缺省）
`"err_io": "file"`	`stderr` 写入文件
`"err_io": "buffer"`	`stderr` 附加到缓冲区（见下）
`"err_name": "/path/file"`	写入的文件或缓冲区名
`"err_buf": number`	写入的缓冲区号
`"err_modifiable": 0`	写入到缓冲区时，关闭 `'modifiable'`（见下）
`"err_msg": 0`	写入新缓冲区时，将首行置为 `"Reading from channel error..."`

`"block_write": number`	只用于测试：模拟每隔一个的 `stdin` 写入是阻塞写入
`"env": dict`	新进程使用的环境变量
`"cwd": "/path/to/dir"`	新进程的当前工作目录；如该目录不存在则报错

写入缓冲区

out_io-buffer

如果 `out_io` 或 `err_io` 模式是 `"buffer"`，而且回调存在，先把文本附加至缓冲区尾，然后再调用回调。

如果缓冲区既用于输入也用于输出，输出行放置在尾行之前，因为尾行是用于写到通道输入的。否则，输出行放置在尾行之后。

如果 "buffer" 使用 JS 或 JSON 模式，只有 ID 为零或负数的消息被经过解码和编码后加入缓冲区。带正数的消息则被回调处理。命令的处理则如常。

"out_name" 或 "err_name" 用的缓冲区名字和已有的缓冲区的完整名字相比较，当前目录也被扩展。例如，如果以 ":edit somename" 创建了缓冲区，而缓冲区名是 "somename"，则使用该缓冲区。

如果没有匹配的缓冲区，则创建新的缓冲区。如果使用空名，则一定会创建新缓冲区。
ch_getbufnr() 可用来得到缓冲区编号。

新缓冲区的 'buftype' 设为 "nofile"，而 'bufhidden' 为 "hide"。要想用别的设置，先建立好缓冲区，然后传入其缓冲区号。

out_modifiable err_modifiable
"out_modifiable" 和 "err_modifiable" 选项可用来关闭 'modifiable' 选项，或写入关闭了 'modifiable' 的缓冲区。这意味着新行被附加到缓冲区后，但用户却不能轻易地修改此缓冲区。

out_msg err_msg
"out_msg" 选项可用来指定新缓冲区是否将首行设为 "Reading from channel output..."。缺省是加入该信息。"err_msg" 则对错误通道同样办理。

如果要写入已有的缓冲区，但其 'modifiable' 已关闭并且 "out_modifiable" 或 "err_modifiable" 选项非零，报错并拒绝写入缓冲区。

如果写入的缓冲区在窗口显示并且光标在末行的首列，光标会被移动到新行上，有必要的话滚动窗口以显示光标所在的位置。

所有新加的行都同步了撤销历史。接受 NUL 字节 (Vim 内部存为 NL 字节)。

写入文件

E920
文件建立的缺省权限为 600 (用户可读写，其它人不能访问)。可用 setfperm() 改变。

如果文件已存在，文件内容被清空。

---

13. 控制作业 job-control

要获知作业的状态:

```
echo job_status(job)
```

要停止作业的运行:

```
job_stop(job)
```

这是停止作业的正常方式，Unix 上会给作业发出 SIGTERM。也可以用其它方式来停止作业，甚至可以发送任意的信号。例如，要强制终止作业，"杀了它":

```
job_stop(job, "kill")
```

更多选项，可见 job_stop() 。

14. 使用提示缓冲区 prompt-buffer

如果你希望在 Vim 窗口中为作业提供输入，有几种选项：
- 使用普通缓冲区，自行处理所有可能的命令。这很复杂，因为可能的命令有很多。
- 使用终端窗口。此方式直接在窗口中键入作业输入并显示作业的输出。
  见 terminal-window 。
- 使用提示窗口。此方式下在 Vim 中输入一行作业命令，而在窗口中显示作业的（可能经过过滤的）输出。

通过设置 'buftype' 为 "prompt" 创立提示缓冲区。通常只在缓冲区新建时使用。

用户可以在缓冲区的末行上编辑和输入一行文本行。在该提示行上输入 Enter 后，激活 prompt_setcallback() 设置的回调。通常，它会把该行发送给作业。另一个回调会从作业接收输出，并在缓冲区中显示结果。显示位置在提示文本之下（但在下一个提示文本的上方）。

只有末行提示文本之后的文本可进行编辑。普通模式命令不可修改缓冲区其余部分。但通过调用如 append() 等的函数，还是可以的。使用其它命令可能搞乱缓冲区。

设置 'buftype' 为 "prompt" 时，Vim 并不自动启动插入模式。如想开始插入模式，用 :startinsert ，这样用户可以行的输入。

通过 prompt_setprompt() 函数可设置提示文本。

用户可用普通模式浏览缓冲区，以便查看旧作业的输出或进行文本复制。

CTRL-W 键可用于开始窗口命令，例如，CTRL-W w 可切换到下一个窗口。这也适用于插入模式（用 Shift-CTRL-W 删除单词）。离开提示窗口时会停止插入模式。而回到提示窗口时会复原窗口模式。

任何启动插入模式的命令，如 "a"、"i"、"A" 和 "I" 等，会自动移动光标到末行。"A" 会移到行尾，而 "I" 会移到行首。

vim:tw=78:ts=8:noet:ft=help:norl:

fold.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar
译者：Zimin<cranecai@users.sf.net>，tocer

折叠　Folding　folding　folds

你可以在用户手册的第 28 章找到有关折叠的序论。 usr_28.txt

1. 折叠方法　fold-methods
2. 折叠命令　fold-commands
3. 折叠选项　fold-options
4. 折叠行为　fold-behavior

{仅当编译时加入 +folding 特性才有效}

---

1. 折叠方法　fold-methods

可用选项 'foldmethod' 来设定折叠方法。

设置选项 'foldmethod' 为非 "manual" 的其它值时，所有的折叠都会被删除并且创建新的。如果设置成 "manual"，将不去除已有的折叠。可以利用这一点来先自动定义折叠，然后手工调整。

有 6 种方法来选定折叠：

manual	手工定义折叠
indent	更多的缩进表示更高级别的折叠
expr	用表达式来定义折叠
syntax	用语法高亮来定义折叠
diff	对没有更改的文本进行折叠
marker	对文中的标志折叠

手 工　fold-manual

使用命令来手工定义要折叠的范围。分析文本并发现折叠的脚本也可用这种方法。

折叠的级别仅由嵌套次数来定义。要对一定行数的折叠增加级别，可以对相同的行反复定义折叠。

当你退出文件编辑时，手工折叠会被遗弃。要保存折叠，使用 :mkview 命令。之后要恢复可以使用 :loadview 。

缩 进　fold-indent

由缩进行自动定义折叠。

折叠级别由行的缩进除以 'shiftwidth' (向下取整) 计算而得。连续的，有同样或更高的折叠级别的行，形成一个折叠。在其中，有更高折叠级别的行形成嵌套的折叠。

嵌套的级别数受 'foldnestmax' 选项限制。

某些行会被忽略并得到上一行或下一行的折叠级别（取较小值）。符合条件的这样的行要么是空行，要么以 'foldignore' 选项里包含的字符开始。在查找 'foldignore' 里包含的字符时，空白字符会被忽略。对于 C，该选项使用 "#" 来略过要预处理的那些行。

当你想用另一种方式来略过行时，使用 "expr" 方法。在 'foldexpr' 中 indent() 函数被用来取得一行的缩进。

表 达 式 fold-expr

就像用 "indent" 方式一样，表达式方式的折叠也是由折叠级别自动定义的。对每行，通过计算选项 'foldexpr' 的值来并得到它的折叠级别。如：
对所有以制表符开始的连续的几行，创建折叠：

```
:set foldexpr=getline(v:lnum)[0]==\"\\t\"
```

调用一函数来计算折叠级别：

```
:set foldexpr=MyFoldLevel(v:lnum)
```

用空白行分开的段落构成折叠：

```
:set foldexpr=getline(v:lnum)=~'^\\s*$'&&getline(v:lnum+1)=~'\\S'?'<1':1
```

同上：

```
:set foldexpr=getline(v:lnum-1)=~'^\\s*$'&&getline(v:lnum)=~'\\S'?'>1':1
```

备注：":set" 要特殊处理的字符必须用反斜杠转义。(空格，反斜杠，双引号等等，参考 option-backslash)

这些是表达式的计算条件：
- 当前缓冲区和窗口值依所在行而定
- 变量 "v:lnum" 被定为该行行号
- 计算结果将用以下方式解释：

值	代表
0	这行不折叠
1, 2, ..	这行的折叠级别 1，2 等
-1	折叠级别没有定义，使用这行之前或之后一行的级别值，取其中较小的一个。
"="	使用上一行的折叠级别。
"a1", "a2", ..	上一行的折叠级别加 1，2，..，结果适用于当前行
"s1", "s2", ..	上一行的折叠级别减 1，2，..，结果适用于当前行
"<1", "<2", ..	此折叠级别在本行结束
">1", ">2", ..	此折叠级别在本行开始

不需要用 ">1" ("<1") 标志折叠的开始（结束）。当这行折叠级别高于（低于）上一行的级别时，折叠将开始（结束）。

表达式必须没有副作用。在缓冲区里的文字，光标位置，查找模式，选项等等，不能被改动。如果你非常小心，改动并恢复这些设置还是可以的。

表达式中有错误或者计算结果不能识别时，Vim 不会产生错误消息，而是将折叠级别设为 0。所以当需要调试时，可将 'debug' 选项设为 "msg"，错误消息就可以被见到了。

备注：由于每一行关于表达式的值都要被计算，这一折叠方式可能会很慢！

最好避免使用 "=" ， "a" 和 "s" 作为返回值，因为 Vim 不得不经常向后回溯以得到折叠级别。这会降低执行速度。

使用 "a1" 和 "s1" 的一例：用于多行 C 注释，以 "/*" 开始的行返回 "a1" 以开始折叠，包含 "*/" 的一行返回 "s1" 使该行之后结束折叠：

```
if match(thisline, '/\*') >= 0
  return 'a1'
elseif match(thisline, '\*/') >= 0
  return 's1'
else
  return '='
endif
```

不过，此代码不适用于单行注释，字符串等。

foldlevel() 计算相对于上一折叠级别的折叠级别。但要注意，如果该级别未知，foidlevel() 返回 -1。它返回的级别对应于该行开始的位置，尽管折叠本身可能在该行结束。

折叠可能会没有及时更新。用 zx 或者 zX 可以强制折叠更新。

语 法 fold-syntax

由带有 "fold" 参数的语法项来定义折叠。 :syn-fold

折叠级别由嵌套的折叠层数来定义。嵌套数由 'foldnestmax' 限定。

要小心指定合适的语法同步方式。如果这里有问题，折叠和显示高亮可能会不一致。尤其在使用可能跨越多行的模式时更要小心。如果有疑问，不妨尝试如下的强制同步：

```
:syn sync fromstart
```

比 较 fold-diff

对没有改动的文本或靠近改动的文本自动定义折叠。

这个方法仅适用于当前窗口设定 'diff' 选项来显示不同之处时才有效。不然，整个缓冲区就是一个大的折叠。

选项 'diffopt' 可以指定上下文。即折叠和不被折叠包括的改变之间相距的行数。比如，设定上下文为 8：

```
:set diffopt=filler,context:8
```

默认值是 6。

当设定了 'scrollbind' 选项时，Vim 会试图在其它比较窗口中打开相同的折叠，这样这些窗口就会显示同一处文本。

标 志 fold-marker

在文本中可以加入标志来指明折叠开始和结束的地方。这可以让你精确的定义折叠。这也让你可以放心地删除和复制折叠而不用担心错误包括了某些行。选项 'foldtext' 通常设置为使折叠行显示折叠标志之前的文本。这样做可以为折叠命名。

标志可以包含级别数，也可以使用匹配对。包含级别数较简单，你无须添加结束标志，并可以避免标志不配对的问题。如：

```
/* global variables {{{1 */
```

```
int varA, varB;

/* functions {{{1 */
/* funcA() {{{2 */
void funcA() {}

/* funcB() {{{2 */
void funcB() {}
```

折叠开始于 "{{{" 标志。接下去的数字决定了折叠级别。产生的效果依当前折叠级别和该折叠标志给定的级别的大小关系而定：
1. 如果遇到级别相同的标志，上一个折叠结束，另一个有同样级别的折叠开始。
2. 如果遇到级别更高的标志，开始一个嵌套的折叠。
3. 如果遇到级别更低的标志，所有大于或等于当前级别的折叠结束，且指定级别的折叠开始。

数字指定了折叠级别。不能使用 0 (忽略级别为 0 的标志)。你可以使用 "}}}" 带上一个数字来表示这个级别的折叠结束。此行以下的折叠级别将比当前级别小一。 备注 : Vim 不会回溯匹配的标志 (这将耗费太多的时间)。如：

```
{{{1
fold level here is 1
{{{3
fold level here is 3
}}}3
fold level here is 2
```

你也可以用一对匹配标志 "{{{" 和 "}}}" 来定义折叠。每个 "{{{" 使折叠级别加 1，"}}}" 使折叠级别减 1。 注意 一定要使标志匹配！如：

```
{{{
fold level here is 1
{{{
fold level here is 2
}}}
fold level here is 1
```

带数字的标志和没有带数字的标志可以混合使用。对于一个大折叠我们可以使用带数字的标志，在函数的内部我们可以使用不带数字的。如对文件中的 "结构定义"，"局部变量" 和 "函数原型" 各个部分，使用级别 1 的折叠。对定义和函数体使用级别 2 的标志。而在函数内部可以使用不带数字的标志。当你在一函数中作更改以分割折叠时，你无须记着标志在哪里。

选项 'foldmarker' 可设定所用的标志。建议保留默认值 "{{{,}}}"。这样在 Vim 用户间交换文件时就不会出问题。有时为了要编辑的文件必须改变该选项 (如，文件包含了来自另一个编辑器的折叠标志，或对于当前文件的语法而言，折叠标志会产生歧义等)。

fold-create-marker

"zf" 可通过标志来创建一个折叠。 Vim 将为你插入开始和结束标志。这些标志由 'foldmarker' 指定。标志会被添加在行末。如果 'commentstring' 不为空，使用之。以下情况下可能无法正常工作：
- 此行已有一个带级别的标志了。Vim 无法判断该如何做。
- 在附近的折叠里使用了带级别的标志，使添加无法进行。

- 这行包含在一注释中，且 'commentstring' 不为空，且嵌套的注释是非法的。如，在 C 的注释中加入 /* {{{ */ ，这将使现有的注释中断。你可以在注释之前或之后添加标志，或手工添加。

总的来说，当你已有带级别的标志时，再让 Vim 为你创建标志不是个好主意。

fold-delete-marker

"zd" 可用来删除一个用标志定义的折叠。 Vim 将为你删除标志。Vim 将在折叠的开始和结束处根据 'foldmarker' 查找开始和结束标志。如果标志周围文字和 'commentstring' 匹配，这些文字也会被删除。

但在以下情况下无法正确执行：

- 当一行包含有多于一个的标志且其中一个指定了级别。Vim 不考虑结果是不是真能删除折叠，而仅删除第一个标志。
- 当这个标志带有级别且同时被用来开始或结束几个折叠时。

---

2. 折叠命令 fold-commands E490

所有的折叠命令用 "z" 开头。提示：如果你从侧面看 "z" 像一张叠起来的纸。

创建和删除折叠

zf E350

zf{motion} 或
{Visual}zf
创建折叠操作符。
仅当 'foldmethod' 设为 "manual" 或 "marker" 时有效。
用 "manual" 方式，新建的折叠会被关闭。同时 'foldenable' 会被设定。
参考 fold-create-marker 。

zF
zF
对 [count] 行创建折叠。其余同 "zf" 。

:fold :fo
:{range}fo[ld]
对 {range} 内的行创建折叠。其余同 "zf" 。

zd E351
zd
删除 (delete) 在光标下的折叠。当光标在被折叠的行上，该折叠被删除。嵌套的折叠上移一级。在可视模式下所选区域 (部分) 涵盖的一层折叠被删除。
小心：这很容易比你想象地要删得多，手动折叠不能撤销。
仅当 'foldmethod' 设为 "manual" 或 "marker" 时有效。
参考 fold-delete-marker 。

zD
zD
循环删除 (Delete) 光标下的折叠。在可视模式下所选区域 (部分) 涵盖的折叠和嵌套的折叠都被删除。
仅当 'foldmethod' 设为 "manual" 或 "marker" 时有效。
参考 fold-delete-marker 。

zE E352
zE
除去 (Eliminate) 窗口里所有的折叠。
仅当 'foldmethod' 设为 "manual" 或 "marker" 时有效。
参考 fold-delete-marker 。

打 开 和 关 闭 折 叠

一个小于 'foldminlines' 的折叠的显示就像它被打开时一样。所以，以下的命令对于小折叠所起的作用与描述不同。

zo

打开 (open) 在光标下的折叠。当给定计数时，相应深度的折叠被打开。在可视模式下，所选区域的所有行的折叠被打开一级。

zO

循环打开 (Open) 光标下的折叠。不在光标下的折叠不改变。
在可视模式下，打开所有的在选中区域里的折叠，包括被部分选中的。

zc

关闭 (close) 在光标下的折叠。当给定计数时，相应深度的折叠被关闭。在可视模式下，所选区域里的所有行的折叠被关闭一级。
'foldenable' 被设定。

zC

循环关闭 (Close) 在光标下的所有折叠。不在光标下的折叠不改变。
在可视模式下，关闭所有的在选中区域里的折叠，包括被部分选中的。
'foldenable' 被设定。

za

当光标位于一关闭的折叠上时，打开之。当折叠嵌套时，你可能需要用 "za" 数次。当给定计数时，打开相应数量的被关闭的折叠。
当光标位于一打开的折叠上时，关闭之且设定 'foldenable' 。这仅关闭一级折叠，因为再次使用 "za" 将再次打开折叠。当给定计数时，关闭相应数量的折叠 (这不同于重复 "za" 许多次)。

zA

当处在一关闭的折叠上时，循环地打开折叠。
当处在一打开的折叠上时，循环地关闭折叠且设定 'foldenable'。

zv

查看 (view) 光标所在的行：仅打开足够的折叠使光标所在的行不被折叠。

zx

更新折叠：撤销被手工打开和关闭的折叠：再次应用 'foldlevel'。
然后使用 "zv" ：查看光标所在行。
同时强制重新计算折叠。使用 "foldexpr" 并且缓冲区发生改变但折叠不能正确地更新时，这会有用。

zX

手工恢复被打开和关闭的折叠：再次应用 'foldlevel'。
也同时强制重新计算折叠，同 zx 。

zm

折起更多 (more)：'foldlevel' 减 v:count1。如果 'foldlevel' 已经为 0，则不会被减小。
'foldenable' 被设定。

zM

zM 关闭所有折叠：'foldlevel' 设为 0。
'foldenable' 被设定。

zr

zr 减少 (reduce) 折叠：'foldlevel' 加 v:count1。

zR

zR 打开所有的折叠。'foldlevel' 设为最高级别。

:foldo :foldopen

:{range}foldo[pen][!]
在 {range} 内打开折叠。当加上 [!] 时，所有的折叠都被打开。对查看在 {range} 内的所有文本很有用。没有 [!] 时，打开一级折叠。

:foldc :foldclose

:{range}foldc[lose][!]
在 {range} 内关闭折叠。当加上 [!] 时，所有的折叠都被关闭。对隐藏在 {range} 内的所有文本很有用。没有 [!] 时，关闭一级折叠。

zn

zn 不折叠 (none)：复位 'foldenable'。所有的折叠被打开。

zN

zN 正常折叠 (normal)：设定 'foldenable'。所有的折叠都展现它们之前的样子。

zi

zi 翻转 'foldenable' 的值。

在 折 叠 间 移 动

[z

[z 到当前打开的折叠的开始。如果已在开始处，移到包含这个折叠的折叠开始处。如果没有包含它的折叠，命令执行失败。
当给定计数，重复此命令 [count] 次。

]z

]z 到当前打开的折叠的结束。如果已在结束处，移到包含这个折叠的折叠结束处。如果没有包含它的折叠，命令执行失败。
当给定计数，重复此命令 [count] 次。

zj

zj 向下移动。到达下一个折叠的开始处。关闭的折叠也被计入。
当给定计数，重复此命令 [count] 次。
此命令可在 operator 后使用。

zk

zk 向上移动到前一折叠的结束处。关闭的折叠也被计入。
当给定计数，重复此命令 [count] 次。
此命令可在 operator 后使用。

对 折 叠 执 行 命 令

:[range]foldd[oopen] {cmd}　　:foldd　:folddo　:folddoopen

对所有不在关闭的折叠中的行执行 {cmd}。
给定 [range] 时，仅对范围内那些行起作用。
每次命令被执行时，光标会被定位在要被操作的行上。
就如 ":global" 命令：首先标记出所有不在关闭的折叠中的行。然后对所有标记过的行，执行 {cmd}。所以当 {cmd} 改变了文本的折叠时，对命令执行的位置没有影响（当然，删除行例外）。
如：

```
:folddoopen s/end/loop_end/ge
```

使用标志位 "e" 避免了当 "end" 不匹配时得到错误消息。

:[range]folddoc[losed] {cmd}　　:folddoc　:folddoclosed

对所有在关闭的折叠里的行，执行 {cmd}。
其它同 ":folddoopen" 命令。

---

3. 折叠选项　　fold-options

颜 色　　fold-colors

对关闭的折叠的颜色的设定由 Folded 高亮组 hl-Folded 决定。对折叠栏的颜色的设定由 FolderColumn 高亮组 hl-FoldColumn 决定。
下面是设定颜色的例子：

```
:highlight Folded guibg=grey guifg=blue
:highlight FoldColumn guibg=darkgrey guifg=white
```

折 叠 级 别　　fold-foldlevel

'foldlevel' 是个数值选项：数字越大则打开的折叠更多。
当 'foldlevel' 为 0 时，所有的折叠关闭。
当 'foldlevel' 为正数时，一些折叠关闭。
当 'foldlevel' 很大时，所有的折叠打开。
'foldlevel' 的改变后立即生效。之后，折叠可以被手动地打开和关闭。
当其值增大，在新级别（译者注：旧级别？）之上的折叠被打开。手工打开的折叠不会被关闭。
当其值减小，在新级别之上的折叠被关闭。手工关闭的折叠不会被打开。

折 叠 文 本　　fold-foldtext

'foldtext' 是个字符串选项，定义了一个表达式。这个表达式被用来求得关闭折叠所显示的文字。如：

```
:set foldtext=v:folddashes.substitute(getline(v:foldstart),'/\\*\\|\\*/\\|{{{\\d\\=','','g')
```

显示了折叠的第一行，除去其中的 "/*"、"*/" 和 "{{{" 。
备注：使用反斜杠是为了避免一些字符被 ":set" 命令解释。使用一个函数更简单：

```
:set foldtext=MyFoldText()
:function MyFoldText()
:  let line = getline(v:foldstart)
```

```
:  let sub = substitute(line, '/\*\|\*/\|{{{\d\=', '', 'g')
:  return v:folddashes . sub
:endfunction
```

'foldtext' 的计算是在沙盘 sandbox 里完成的。其中的当前窗口设为要显示该行的窗口。错误被忽略。

默认值是 foldtext() 。对于大多数类型的折叠它可以返回适合的文字。如果你不喜欢它，你可以自己指定 'foldtext' 表达式。可以使用以下这些 Vim 变量:

v:foldstart	折叠首行的行号
v:foldend	折叠末行的行号
v:folddashes	一个含有连字符的字符串，用来表示折叠级别
v:foldlevel	折叠级别

在结果中，制表符被替换为空格，而不可显示的字符被替换为可显示的字符。

结果行被截短以适合窗口的宽度，永远不会回绕。
当在文字后有空余时，用 'fillchars' 来填充。

备注: 对那些 ":set" 命令作特殊处理的字符在其前面须加上反斜杠。如: 空格，反斜杠和双引号。 option-backslash

折 叠 栏 fold-foldcolumn

'foldcolumn' 是个数字，它设定了在窗口的边上表示折叠的栏的宽度。当为 0 时，没有折叠栏。一个普通值是 4 或 5。最小可用的值是 2，不过 1 仍然可以提供一些信息。最大是 12。

一个打开的折叠由一栏来表示，顶端是 '-'，其下方是 '|'。这栏在折叠结束的地方结束。当折叠嵌套时，嵌套的折叠出现在被包含的折叠右方一个字符位置。

一个关闭的折叠由 '+' 表示。

当折叠栏太窄而不能显示所有折叠时，显示一数字来表示嵌套的级别。

在折叠栏点击鼠标，可以打开和关闭折叠:
- 点击 '+' 打开在这行的关闭折叠
- 在任何其他非空字符上点击，关闭这行上的打开折叠

其 他 选 项

'foldenable' 'fen':	复位时打开所有折叠。
'foldexpr' 'fde':	用于 "expr" 折叠的表达式。
'foldignore' 'fdi':	用于 "indent" 折叠的字符。
'foldmarker' 'fmr':	用于 "marker" 折叠的标志。
'foldmethod' 'fdm':	当前折叠方法。
'foldminlines' 'fml':	关闭折叠的最小显示行数。
'foldnestmax' 'fdn':	用于 "indent" 和 "syntax" 折叠的最大嵌套层数。
'foldopen' 'fdo':	哪一种命令可以打开关闭的折叠。
'foldclose' 'fcl':	当光标不在折叠上时关闭折叠。

4. 折叠行为 fold-behavior

当上下移动和滚动时，光标将移至连续折叠的行的第一行。当光标已在被折叠的行上时，它移动到下一个没有被折叠的行上，或下一个关闭的折叠。

当光标在被折叠的行上时，光标总是显示在第一列。标尺显示确切的光标位置。光标本应也在实际位置显示的，但因为被折叠而无法做到。

许多移动命令处理连续被折叠的行就像处理空行一样。如，"w" 命令只在首列停一次。

在插入模式下，光标所在的行不会被折叠。这让你看得见你键入的内容。

当使用操作符时，一个关闭的行被作为整体处理。所以 "dl" 在光标下删除了整个关闭的折叠。

作用于缓冲区行的 Ex 命令的执行范围会被调整，使得它总是从关闭折叠的第一行开始，在关闭折叠的最后一行结束。所以，命令：

```
:s/foo/bar/g
```

当光标在关闭的折叠上时，它将在整个折叠里用 "bar" 替换 "foo"。
不过 :folddoopen 和 :folddoclosed 的操作并非如此。

编辑一个已经被编辑过的缓冲区时，上次被使用的折叠设置会被再次使用。对手工定义折叠方式，已经被定义的折叠会被恢复。对所有的折叠方式，手工打开和关闭的折叠会被恢复。如果这个缓冲区曾经在这个窗口里被编辑，过去用的值被恢复。否则，来自该缓冲区最近一次编辑的窗口值被使用。

---

vim:tw=78:ts=8:noet:ft=help:norl:

testing.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Willis
https://github.com/yianwillis/vimcdoc

测试 Vim 和 Vim 脚本 testing-support

eval.txt 说明表达式的计算。此文件描述 Vim 脚本中编写测试的细节。这可用于 Vim 自身和插件的测试。

1. 测试 Vim testing
2. 测试函数 test-functions-details
3. 断言函数 assert-functions-details

---

1. 测试 Vim testing

Vim 在编译后可以进行测试，通常用 "make test" 进行。
这些测试位于目录 "src/testdir"。

在不同的时间点，加入了几种不同类型的测试：

test33.in	最老的，不要再加了
test_something.in	旧风格测试
test_something.vim	新风格测试

new-style-testing
新写的测试需用新风格的测试。使用 assert_equal() 之类的函数，以便在同一处地方维护测试命令和期待的结果。

old-style-testing
有些场合仍然需要旧风格的测试。例如在没有 +eval 特性时测试 Vim。

更多信息可见文件 src/testdir/README.txt。

---

2. 测试函数 test-functions-details

test_alloc_fail({id}, {countdown}, {repeat}) test_alloc_fail()
用于测试：如果调用了 {id} 指定的内存分配，{countdown} 减一，到零时，让内存分配失败 {repeat} 次。如果 {repeat} 小于一，失败一次。

也可用作 method：
```
GetAllocId()->test_alloc_fail()
```

test_autochdir() test_autochdir()
设置标志位，在 Vim 启动过程结束之前，打开 'autochdir' 的效果。

test_feedinput({string}) test_feedinput()
{string} 中的字符会如果是用户键入的那样，排队依次等待处理。使

用一个低层输入缓冲区。此函数只适用于 +unix 或 GUI 运行时。

也可用作 method :
```
GetText()->test_feedinput()
```

test_garbagecollect_now() test_garbagecollect_now()
类似于 garbagecollect()，但立即执行。只能通过直接调用，以避免内部保存任何结构，而调用任何函数前， v:testing 必须置位。

test_garbagecollect_soon() test_garbagecollect_soon()
就像在 main 循环中那样，设置标志位来调用垃圾清理器。仅用于测试。

test_getvalue({name}) test_getvalue()
取得内部变量的值。支持以下的 {name} 值:
```
need_fileinfo
```

也可用作 method :
```
GetName()->test_getvalue()
```

test_ignore_error({expr}) test_ignore_error()
忽略包含 {expr} 的任何错误。显示正常的消息来替代。
只用于测试，因为那里 try/catch 不能用于捕捉错误 (因为跳过了下面的代码)。
{expr} 按本义使用，不用作模式。
如果 {expr} 为字符串 "RESET"，清空忽略错误的列表。

也可用作 method :
```
GetErrorText()->test_ignore_error()
```

test_null_blob() test_null_blob()
返回 null Blob 。仅用于测试。

test_null_channel() test_null_channel()
返回 null Channel 。仅用于测试。
{仅当编译时加入 +channel 特性才有效}

test_null_dict() test_null_dict()
返回 null Dict 。仅用于测试。

test_null_job() test_null_job()
返回 null Job 。仅用于测试。
{仅当编译时加入 +job 特性才有效}

test_null_list() test_null_list()
返回 null List 。仅用于测试。

test_null_partial() test_null_partial()
返回 null Partial 。仅用于测试。

test_null_string() test_null_string()
返回 null String 。仅用于测试。

test_option_not_set({name}) test_option_not_set()

复位指示选项 {name} 已设置过的标志位。就像看起来此选项还使用缺省值那样。这样来用：

```
set ambiwidth=double
call test_option_not_set('ambiwidth')
```

现在 'ambiwidth' 选项的行为就像它从未被改变过一样，即使它的值是 "double"。
仅用于测试！

也可用作 method：

```
GetOptionName()->test_option_not_set()
```

test_override({name}, {val}) test_override()

覆盖 Vim 内部处理的部分行为，用于运行测试。只能用于 Vim 测试！
如果 {val} 非零，打开覆盖行为，{val} 为零时，关闭覆盖行为。
目前支持的名字是：

名字	{val} 非零的效果
redraw	屏蔽 redrawing() 函数
redraw_flag	忽略 RedrawingDisabled 标志位
char_avail	屏蔽 char_avail() 函数
starting	复位 "starting" 变量，见下
nfa_fail	使 NFA 正规表达式引擎失败以强制回归到旧引擎
no_query_mouse	"dec" 终端不查询鼠标位置
no_wait_return	置位 "no_wait_return" 标志位。不被 "ALL" 复位。
ALL	复位所有覆盖（不使用 {val}）

"starting" 用于假定初始化已经完成的测试。因为测试的运行是通过运行一个 "starting" 变量为非零的脚本进行的。这通常有好处（测试更快），但有时会改变行为，使测试不能正常工作。
以下操作：

```
call test_override('starting', 1)
```

保存 "starting" 的值。以下操作恢复之：

```
call test_override('starting', 0)
```

也可用作 method：

```
GetOverrideVal()-> test_override('starting')
```

test_refcount({expr}) test_refcount()

返回 {expr} 的引用计数。如果 {expr} 是没有引用计数的类型，返回 -1。仅用于测试。

也可用作 method：

```
GetVarname()->test_refcount()
```

test_scrollbar({which}, {value}, {dragging}) test_scrollbar()

模拟使用滚动条 {which} 移动到位置 {value}。{which} 可以是：

```
left    当前窗口的左侧滚动条
right   当前窗口的右侧滚动条
hor     水平滚动条
```

对垂直滚动条而言，{value} 可取值 1 到缓冲区行数。对水平滚动条而言，{value} 可取值 1 到最大的行长，假定 'wrap' 未置位。

{dragging} 如果非零，行为就像拖动滚动条，否则像点击滚动条。只能用于确实存在的 {which} 滚动条，显然只能用于 GUI。

也可用作 method :

```
GetValue()->test_scrollbar('right', 0)
```

test_setmouse({row}, {col}) test_setmouse()

设置下个鼠标活动所用的鼠标位置。{row} 和 {col} 从 1 开始。

示例:

```
call test_setmouse(4, 20)
call feedkeys("\<LeftMouse>", "xt")
```

test_settime({expr}) test_settime()

设置 Vim 内部使用的时间。目前只用于历史中的时间戳，用于 viminfo 和撤销。

值为 1 使 Vim 在警告或信息后不睡眠。

{expr} 计算结果必须为数值。值为零时恢复正常行为。

也可用作 method :

```
GetTime()->test_settime()
```

---

3. 断言函数 assert-functions-details

assert_beeps({cmd}) assert_beeps()

执行 {cmd}，如果不响铃或可视响铃，加入错误信息到 v:errors 。

另见 assert_fails() 和 assert-return 。

也可用作 method :

```
GetCmd()->assert_beeps()
```

assert_equal() assert_equal()

assert_equal({expected}, {actual} [, {msg}])

{expected} 不等于 {actual} 时，加入错误信息到 v:errors ，并反回 1。否则返回零 assert-return 。

没有自动转换，字符串 "4" 不同于数值 4。数值 4 又不同于浮点数 4.0。不适用 'ignorecase'，大小写永远敏感。

省略 {msg} 则生成形如 "Expected {expected} but got {actual}" 的错误。

示例:

```
assert_equal('foo', 'bar')
```

会在 v:errors 加入字符串:

```
test.vim line 12: Expected 'foo' but got 'bar'
```

也可用作 method :

```
mylist->assert_equal([1, 2, 3])
```

assert_equalfile()

assert_equalfile({fname-one}, {fname-two})
文件 {fname-one} 和 {fname-two} 不包含相同文本时，加入错误信息到 v:errors 。
另见 assert-return 。
文件 {fname-one} 或 {fname-two} 不存在时，错误信息说明这一点。
最常用于 terminal-diff 。

也可用作 method :
```
GetLog()->assert_equalfile('expected.log')
```

assert_exception({error} [, {msg}]) assert_exception()
v:exception 不包含 {error} 时，加入错误信息到 v:errors 。另见 assert-return 。
可用于断言命令抛出例外。使用错误号后加冒号，就不用担心翻译的问题:
```
try
  commandthatfails
  call assert_false(1, 'command should have failed')
catch
  call assert_exception('E492:')
endtry
```

assert_fails({cmd} [, {error} [, {msg}]]) assert_fails()
执行 {cmd}，如果不出错，加入错误信息到 v:errors 。另见 assert-return 。
给出 {error} 时，必须匹配 v:errmsg 。
注意 响铃不作为错误处理，而有些出错的命令只会响铃。为此，可用 assert_beeps() 断言。

也可用作 method :
```
GetCmd()->assert_fails('E99:')
```

assert_false({actual} [, {msg}]) assert_false()
{actual} 不为假时，加入错误信息到 v:errors ，其余类同 assert_equal() 。
另见 assert-return 。
零值为假值。{actual} 不是数值时，断言失败。
省略 {msg} 则生成形如 "Expected False but got {actual}" 的错误。

也可用作 method :
```
GetResult()->assert_false()
```

assert_inrange({lower}, {upper}, {actual} [, {msg}]) assert_inrange()
对数值和 Float 值进行断言。{actual} 小于 {lower} 会大于 {upper} 时，加入错误信息到 v:errors 。另见 assert-return 。
省略 {msg} 则生成形如 "Expected range {lower} - {upper}, but got {actual}" 的错误。

assert_match()

assert_match({pattern}, {actual} [, {msg}])

{pattern} 不匹配 {actual} 时，加入错误信息到 v:errors 。另见 assert-return 。

{pattern} 的用法类同于 =~ ：匹配总是假定 'magic' 置位而 'cpoptions' 为空，而忽略 'magic' 或 'cpoptions' 的实际值。

{actual} 作字符串使用，适用自动转换。
"^" 和 "$" 可用来匹配文本的开始和结尾。两者皆用则可匹配整个文本。

省略 {msg} 则生成形如 "Pattern {pattern} does not match {actual}" 的错误。
示例：

```
assert_match('^f.*o$', 'foobar')
```

会在 v:errors 加入字符串：

```
test.vim line 12: Pattern '^f.*o$' does not match 'foobar'
```

也可用作 method ：

```
getFile()->assert_match('foo.*')
```

assert_notequal()

assert_notequal({expected}, {actual} [, {msg}])
和 assert_equal() 相反：{expected} 等于 {actual} 时，加入错误信息到 v:errors 。
另见 assert-return 。

也可用作 method ：

```
mylist->assert_notequal([1, 2, 3])
```

assert_notmatch()

assert_notmatch({pattern}, {actual} [, {msg}])
和 assert_match() 相反：{pattern} 匹配 {actual} 时，加入错误信息到 v:errors 。
另见 assert-return 。

也可用作 method ：

```
getFile()->assert_notmatch('bar.*')
```

assert_report({msg}) assert_report()
直接报告测试失败，用 {msg}。总返回一。

也可用作 method ：

```
GetMessage()->assert_report()
```

assert_true({actual} [, {msg}]) assert_true()
{actual} 不为真时，加入错误信息到 v:errors ，余类同 assert_equal() 。
另见 assert-return 。
非零值为真值。{actual} 不是数值时，断言失败。
省略 {msg} 则生成形如 "Expected True but got {actual}" 的错误。

也可用作 method :
GetResult()->assert_true()

vim:tw=78:ts=8:noet:ft=help:norl:

print.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

打印 printing

1. 简介 print-intro
2. 打印选项 print-options
3. PostScript 打印 postscript-printing
4. PostScript 打印编码 postscript-print-encoding
5. PostScript CJK 打印 postscript-cjk-printing
6. PostScript 打印疑难解答 postscript-print-trouble
7. PostScript 工具 postscript-print-util
8. 换页字符 printing-formfeed

{仅当编译时加入 +printer 特性才有效}

---

1. 简介 print-intro

在 MS-Windows 上 Vim 可以在任何打印机上打印你的文本。在别的系统上，它产生一个 PostScript 文件。该文件能直接发送到 PostScript 打印机上，也能通过类似 ghostscript 的程序用在别的打印机上。

备注：如果用 :hardcopy 打印有问题，还有一个办法是用 :TOhtml 并在浏览器上打印生成的 html。

:ha :hardcopy E237 E238 E324

:[range]ha[rdcopy][!] [arguments]
打印 [range] 指定的行（缺省是整个文件）。

在 MS-Windows 上，会显示打印对话框，用来选择打印机和纸张尺寸等。[!] 版本跳过对话框。这里，打印机由 'printdevice' 指定。如果 'printdevice' 为空，就使用系统缺省打印机。

对于非 MS-Windows 的系统，先在一个临时文件写入 Postscript，然后用 'printexpr' 打印它。这时，[arguments] 可以在 'printexpr' 里通过 v:cmdarg 得到。
否则的话，[arguments] 被忽略。'printoptions' 可以用来指定纸张尺寸，是否双面打印，等等。
备注：如果要 pdf，有 "ps2pdf" 这样的工具可把 PostScript 转换为 PDF。

:[range]ha[rdcopy][!] >{filename}
同上，但把 Postscript 打印到文件 {filename}。
其中，"%" 等结构通过 cmdline-special 得到扩展。
需要小心的是，现存的同名文件将被不加提示的覆盖。

{仅当编译时加入 +postscript 特性才能有此功能}
在 MS-Windows 上应使用打印机驱动程序的 "输出到文件" 功能。

打印过程中显示完成的页号和百分比。要中断打印，按中断键 (CTRL-C 或者在 MS-系统里。CTRL-Break)。

'printfont' 和 'printoptions' 选项控制打印输出。'printheader' 指定页眉的格式。

打印文件根据指定的边界 (margin) 排版，这不受当前窗口的 'wrap' 或 'linebreak' 设置影响。'printoptions' 里的 "wrap" 项可以用来关闭回绕。

打印的文本使用当前的高亮色彩，但有以下考虑：
1) 正常的背景色总是白色 (白纸的颜色)。
2) 白色文本和缺省前景总是黑色，这样你才能看得出来！
3) 如果 'background' 是 "dark"，则颜色会加深，否则在白纸上它们因太亮而无法清晰辨别。

---

2. 打印选项 print-options

这里提供改变打印方式的所有选项的细节。设置选项的一般信息见 options.txt 。

pdev-option
'printdevice' 'pdev' 字符串 (缺省为空)
全局
:hardcopy 命令的感叹号 (!) 版本跳过打印机选择对话框，本选项指定此时所用的打印机的名字。在 Win32 上，它必须和标准打印对话框里出现的打印机名字完全一致。如果该选项为空，vim 会为 ":hardcopy!" 使用系统的缺省打印机。

penc-option E620
'printencoding' 'penc' 字符串 (缺省为空，除了：
MS-Windows: cp1252,
Macintosh: mac-roman,
VMS: dec-mcs,
HPUX: hp-roman8,
EBCDIC: ebcdic-uk)
全局
设置打印时使用的字符编码。本选项告诉 Vim 使用 'runtimepath' 的 "print" 目录里哪个打印字符编码文件。

本选项接受任何 encoding-names 里的值。任何能识别的名字被转换为 Vim 的标准名字 - 详情见 'encoding'。Vim 不能识别的名字会被转换为小写形式，而且下划线被替换成 '-' 符号。

如果 'printencoding' 为空或者 Vim 找不到该文件，那么它会用 'encoding' (如果此选项设为 8 位编码的话) 来找到打印字符编码文件。如果 Vim 找不到字符编码文件，那么使用的是 "latin1" 打印字符编码文件。

如果 'encoding' 是多字节编码，Vim 会试图把字符转换为打印编码再进行打印 (如果 'printencoding' 为空，那么转换到 latin1)。转换到非 latin1 的打印编码需要 Vim 编译时加入 +iconv 特性。如果无法转换，打印会失败。任何不能转换的字符会被倒过来的问号代替。

提供了四个打印字符编码文件，分别支持缺省的 Mac、VMS、HPUX 和 EBCDIC 字符编码，而且也是这些平台上缺省使用的。MS-Windows 平台缺省使用代码页 1252 打印字符编码。

pexpr-option

'printexpr' 'pexpr' 字符串 (缺省：见下)
全局
{仅当编译时加入 +printer 和 +postscript 特性时才有效}
用于打印 :hardcopy 产生的 PostScript 文件的表达式。
要打印的文件名在 v:fname_in 。
":hardcopy" 命令的参数在 v:cmdarg 。
该表达式必须在打印后负责删除文件。如果有错误，表达式必须返回非零值。如果没有错，返回零或空字符串。
非 MS-Windows 或 VMS 系统的缺省就是简单地使用 "lpr" 来打印文件：

```
system('lpr' . (&printdevice == '' ? '' : ' -P' . &printdevice)
    . ' ' . v:fname_in) . delete(v:fname_in) + v:shell_error
```

在 MS-Windows 机器上，缺省是把文件复制到当前指定的 printdevice 上：

```
system('copy' . ' ' . v:fname_in . (&printdevice == ''
            ? ' LPT1:' : (' \"' . &printdevice . '\"')))
            . delete(v:fname_in)
```

在 VMS 机器上，缺省是把文件发送到缺省或者当前指定的 printdevice 上：

```
system('print' . (&printdevice == '' ? '' : ' /queue=' .
            &printdevice) . ' ' . v:fname_in) . delete(v:fname_in)
```

如果你改变此选项，使用函数是避免转义所有的空格的简便方法。例如：

```
:set printexpr=PrintFile(v:fname_in)
:function PrintFile(fname)
:  call system("ghostview " . a:fname)
:  call delete(a:fname)
:  return v:shell_error
:endfunc
```

要记住有些打印程序在读入文件之前就返回控制权。如果你过早地删除文件，它不会被打印。这些程序通常提供选项，在打印完成时让它们删除文件。

E365

如果表达式计算失败，或者产生非零值，你会得到错误信息。那种情况下，Vim 会删除文件。非 MS-Windows 的缺省值里使用一个技巧：加上 "v:shell_error" 会在 system() 调用失败时返回非零值。

为了安全原因，本选项不能在 modeline 或 sandbox 里设置。

pfn-option E613

'printfont' 'pfn' 字符串 (缺省为 "courier")
全局
:hardcopy 命令输出使用的字体名。它和 'guifont' 选项的格式一样，除了只能给出一个字体的名字，而且不能使用特殊的 "guifont=*" 语法以外。

在 Win32 GUI 版本上，它指定字体名还有附加属性，就如同 'guifont' 选项那样。

其他系统上，只能识别 ":h11"，其中 "11" 是字体的点数 (point) 大小。如果忽略，使用 10 点字。

pheader-option

'printheader' 'pheader' 字符串 (缺省为 "%<%f%h%m%=Page %N")
全局

定义 :hardcopy 输出的页眉格式。该选项和 'statusline' 选项以同样的方式定义。如果 Vim 编译时没有加入 +statusline 特性，本选项没有效果，而使用简单的缺省页眉，其中只包含页号。同样的简单页眉也用在此选项为空的时候。

pmbcs-option

'printmbcharset' 'pmbcs' 字符串 (缺省为 "")
全局

设置 :hardcopy 产生 CJK 输出时使用的 CJK 字符集。目前，Vim 识别以下这些预先定义的值：

	值	描述
中文 (简体)	GB_2312-80	
	GBT_12345-90	
	MAC	Apple Mac Simplified Chinese
	GBT-90_MAC	GB/T 12345-90 Apple Mac Simplified Chinese
	GBK	GBK (GB 13000.1-93)
	ISO10646	ISO 10646-1:1993
中文 (繁体)	CNS_1993	CNS 11643-1993, Planes 1 & 2
	BIG5	
	ETEN	Big5, 带 ETen 扩展
	ISO10646	ISO 10646-1:1993
日文	JIS_C_1978	
	JIS_X_1983	
	JIS_X_1990	
	MSWINDOWS	Win3.1/95J (JIS X 1997 + NEC + IBM 扩展)
	KANJITALK6	Apple Mac KanjiTalk V6.x
	KANJITALK7	Apple Mac KanjiTalk V7.x
韩文	KS_X_1992	
	MAC	Apple Macintosh Korean
	MSWINDOWS	KS X 1992, 带 MS 扩展
	ISO10646	ISO 10646-1:1993

上面的值和 'printencoding' 只有特定的一些组合。下表显示合法的组合可能：

		euc-cn	gbk	ucs-2	utf-8
中文 (简体)	GB_2312-80	x			
	GBT_12345-90	x			
	MAC	x			
	GBT-90_MAC	x			
	GBK		x		
	ISO10646			x	x

		euc-tw	big5	ucs-2	utf-8

中文	CNS_1993	x				
(繁体)	BIG5		x			
	ETEN		x			
	ISO10646				x	x
		euc-jp	sjis	ucs-2	utf-8	
日文	JIS_C_1978	x	x			
	JIS_X_1983	x	x			
	JIS_X_1990	x			x	x
	MSWINDOWS		x			
	KANJITALK6		x			
	KANJITALK7		x			
		euc-kr	cp949	ucs-2	utf-8	
韩文	KS_X_1992	x				
	MAC	x				
	MSWINDOWS		x			
	ISO10646				x	x

要设置打印日文文本所用的合适的编码和字符集，你需要做：

```
:set printencoding=euc-jp
:set printmbcharset=JIS_X_1983
```

如果 'printmbcharset' 不是以上给出的值，就假定这是个自定义的多字节字符集，因而没有它是否与 'printencoding' 的值兼容的检查。Vim 会在 'runtimepath' 里的 "print" 目录下寻找定义该字符集的文件。

pmbfn-option

'printmbfont' 'pmbfn'　字符串 (缺省为 "")
全局

这是逗号分隔的字段列表，用于 :hardcopy 生成的 CJK 输出使用的字体名。每个字体名必须有一个字符前导，指示要使用的字体的风格，如下：

r:{font-name}	用于普通字符的字体
b:{font-name}	用于粗体字符的字体
i:{font-name}	用于斜体字符的字体
o:{font-name}	用于粗体斜体字符的字体

执行 CJK 打印时，带有 r: 前缀的字体是必需的。其它字体名的描述符可选。如果某个描述符不存在，使用另一个字体，如下：

如果 b: 不存在，使用 r:
如果 i: 不存在，使用 r:
如果 o: 不存在，使用 b:

有些 CJK 字体不包含 ASCII 代码范围内的字符编码。还有，在少量代码点上，有些 CJK ASCII 代码范围里的字符和传统的 ASCII 字符有差异。有两个附加的域可以用来控制 ASCII 代码范围内的字符打印。

c:yes	使用 Courier 字体打印 ASCII 代码范围内的字符。
c:no (缺省)	
a:yes	在 ASCII 代码范围内的代码使用 ASCII 字符集。
a:no (缺省)	

下面的例子指定两个多字节字体，一个用于普通和斜体打印，一个用于粗体和粗体斜体打印，它使用 Courier 打印 ASCII 代码范围内的代码，但使用国家字符集：

```
:set printmbfont=r:WadaMin-Regular,b:WadaMin-Bold,c:yes
```

popt-option

'printoptions' 'popt'　字符串　(缺省为 "")
全局
{仅当编译时加入　+printer　特性才有效}

这是逗号分隔的项目列表，控制　:hardcopy　的输出格式：

left:{spec}	左边界 (缺省: 10pc)
right:{spec}	右边界 (缺省: 5pc)
top:{spec}	顶部边界 (缺省: 5pc)
bottom:{spec}	底部边界 (缺省: 5pc) {spec} 是一个数字，后面跟 "in" 代表英寸，"pt" 代表点数 (1 点等于 1/72 英寸)，"mm" 代表毫米，或者 "pc" 代表媒介大小的百分比。 古怪的例子： left:2in,top:30pt,right:16mm,bottom:3pc 如果单位不能识别，不会出错。只是使用缺省值。
header:{nr}	保留作页眉的行数。 只填入第一行，所以如果 {nr} 为 2，有一个空行。页眉的格式由 'printheader' 控制。
header:0	不打印页眉。
header:2　(缺省)	页眉使用两行
syntax:n	不使用语法高亮。这样更快，因而打印大文件也更实用。
syntax:y	使用语法高亮。
syntax:a　(缺省)	如果打印机可以打印色彩或者灰色，使用语法高亮。
number:y	打印输出里包含行号。
number:n　(缺省)	没有行号。
wrap:y　(缺省)	回绕长行。
wrap:n	截断长行。
duplex:off	单面打印。
duplex:long (缺省)	双面打印 (如果可以)，长边装订。
duplex:short	双面打印 (如果可以)，短边装订。
collate:y　(缺省)	逐份打印：1 2 3，1 2 3，1 2 3
collate:n	非逐份打印：1 1 1，2 2 2，3 3 3
jobsplit:n (缺省)	单个打印任务完成所有份数
jobsplit:y	每份使用不同的打印任务。用于 N-路后期处理。
portrait:y (缺省)	打印方向为竖向。
portrait:n	打印方向为横向。
	a4　letter
paper:A4　(缺省)	纸张大小：A4
paper:{name}	纸张大小可见下表： {name}　厘米计的尺寸　英寸计的尺寸

```
10x14          25.4  x 35.57    10    x 14
A3             29.7  x 42       11.69 x 16.54
A4             21    x 29.7      8.27 x 11.69
A5             14.8  x 21        5.83 x  8.27
B4             25    x 35.3     10.12 x 14.33
B5             17.6  x 25        7.17 x 10.12
executive      18.42 x 26.67     7.25 x 10.5
folio          21    x 33        8.27 x 13
ledger         43.13 x 27.96    17    x 11
legal          21.59 x 35.57     8.5  x 14
letter         21.59 x 27.96     8.5  x 11
quarto         21.59 x 27.5      8.5  x 10.83
statement      13.97 x 21.59     5.5  x  8.5
tabloid        27.96 x 43.13    11    x 17
```

`formfeed:n` (缺省)　把换页字符 (0x0c) 看成普通的打印字符。
`formfeed:y`　遇到换页字符时，当前行从新页的第一行开始处继续打印。

没有给出某项目时，使用标明为 (缺省) 的项目。不需要总是给出这些值，尤其是可以使用对话框来选择打印机和选项。
例如:

```
:set printoptions=paper:letter,duplex:off
```

---

3 PostScript 打印　postscript-printing

E455　E456　E457　E624

只要你有足够的磁盘空间，产生一个 PostScript 文件当无问题。你需要正确安装运行时文件 (如果你能找到帮助文件，它们大概都已经安装好了)。

PostScript 打印有若干限制:

- 'printfont' - 字体名被忽略 (总是用 Courier 家族字体 - 它应在所有 PostScript 打印机上可用) 但还能指定字体大小。

- 'printoptions' - 产生 PostScript 输出时使用了双面打印设置，但是否如此打印决定于打印机本身。如果打印机不支持双面打印，则该设置应简单给予忽略。但是有些打印机彻底不能打印。

- 8 位支持 - 尽管很多 8 位打印字符编码得到支持，还是有可能部分字符无法打印。这决定于打印机所带的字体是否能识别该字符。丢失的字符会以一个倒转的问号代替，或者如果该字体不能识别该字符的话，则留一个空格。安装新版的 Courier 字体家族可能会使某一个编码里的字符完全得到支持。

- 多字节支持 - 目前，Vim 会试图把多字节字符转换为 'printencoding' (如果为空，latin1) 所指定的 8 位编码。不能成功转换的字符会显示为未知字符。如果 Vim 不能完成该转换，打印将会失败。

---

4. 自定义 8 位打印字符编码　postscript-print-encoding

E618　E619

打印 8 位字符数据，如果要使用你自己的打印字符编码，你需要定义自己的 PostScript 字体编码。关于如果定义字体编码向量的细节超出了本帮助文件的范围。为此，你可查阅 Addison-Wesley 出版的 PostScript 语言参考手册第三版，或从 http://www.adobe.com

得到 PDF 版本。以下，说明一下如果使 Vim 找到并使用你自己的打印字符编码。

i. 为你的编码向量定义一个唯一的名字，以不与 Vim 使用的已识别的或标准的编码名字冲突为原则（这些编码可以参见 encoding-names ）。最好是一个别人也不太可能使用的名字。
ii. 把 $VIMRUNTIME/print/latin1.ps 复制到你的 'runtimepath' 里的 print 子目录并换成你选好的唯一的名字。
iii. 编辑你换名了的 latin1.ps。把所有出现的 latin1 替代成你的编码名（不要忘了 %%Title: 开头的那行）。同时，把字型名字（glyph names）数组改成你新定义的编码向量。该数组必须恰好有 256 项，不然你无法打印！
iv. 在 Vim 里，设置 'printencoding' 为你的唯一的编码名，然后打印你的文件。Vim 会使用你自定义的打印字符编码。

如果除了 %%Title: 开头一行里的编码名字和 %%Version:: 开头的一行里的版本以外，你修改了前三行的顺序或者任何别的内容，Vim 会报告此资源文件有错误。

[对通晓 PostScript 者的技术解释 - Vim 打印时，查找并使用和编码同名的文件。该文件定义一个新的名为 /VIM-name 的 PostScript Encoding 资源，其中 name 即 Vim 会使用的打印字符编码。]

---

5. PostScript CJK 打印 postscript-cjk-printing

E673 E674 E675

Vim 支持打印中文、日文和韩文文件。要能正确地打印 CJK 文件，Vim 需要设置更多的选项。

这些国家有许多标准字符集和编码，打印时两者都要指定。另外，CJK 字体通常没有斜体字形的概念，为了在打印时达到强调的效果，需要使用不同的重量（weight）和风格（stroke style）。因此，要指定打印时使用的字体，需要不同的方法。

'printencoding' 和 'printmbcharset' 选项分别指定编码和字符集。如果没有设置 'printencoding'，就像一般使用那样，使用 'encoding'。如果设定 'printencoding'，打印时字符会被翻译成此编码。你要确保该编码和文件内容需要的字符集兼容，否则有些字符可能打印不出来。

'printmbfont' 指定 CJK 打印要用的字体。该选项允许你指定在语法高亮时使用不同字体风格：普通、斜体、粗体和粗体斜体的字符使用不同的打印字体，

Vim 不提供 CJK 字体。一些免费的中（繁体）日韩字体可以在这里得到：

```
http://examples.oreilly.com/cjkvinfo/adobe/samples/
```

在 readme 文件里可以找到这些字体的说明，它在

```
http://examples.oreilly.de/english_examples/cjkvinfo/adobe/00README
```

请阅读你的打印机文档，以了解如何安装新的字体。

CJK 字体可以很大，因为它们包含了数以千计的字形，即使如此，某个字体只包含了国家标准的一个子集也是很常见的。如果你发现打印输出没有半宽的罗马字符，你需要在 'printmbfont' 里配置 Vim 使用 Courier 字体来打印半宽 ASCII 字符。如果你的字体不包括其它字符，也许你需要找一个能够包含所有需要的字形的字体。

另一个 ASCII 字符的问题是若干国家字符集指定一些 ASCII 代码范围的代码使用不同的字形。如果你用国家字符集打印 ASCII 文本，你可能会看到意想不到的字符。如果你需要真正的 ASCII 代码打印，你需要用 'printmbfont' 来配置 Vim 使 ASCII 代码范围内的代码输出 ASCII 字符。

可以定义你自己的多字节字符集，虽然这个过程不应轻易尝试。关于此过程的讨论超出了这些帮助文件的范围。具体细节可见 http://www.adobe.com 提供的 PDF 文件格式的文档 'Adobe CMap and CIDFont Files Specification, Version 1.0' 里的 CMap (字符映射表) 文件。

---

6. PostScript 打印疑难解答 postscript-print-trouble

E621

通常唯一说明打印 PostScript 出现问题的症状是你的打印结果没有出现。如果你运气好，你也许会得到一页打印说明，告诉你哪个 PostScript 操作符产生了使打印无法继续下去的错误。

有若干使打印失败的可能的原因：

- prolog 资源文件版本错误。prolog 资源文件包含一些 Vim 用来打印的必需的 PostScript。每个 Vim 版本需要它一个不同的版本。确信你安装了正确的运行时的文件，而且没有在你的 'runtimepath' 里的 print 目录里保留了一个旧版本的 prolog 文件。

- 纸张大小。有些 PostScript 打印机会因为不支持请求的纸张大小而中止打印。缺省，Vim 用 A4 纸。用 'printoptions' 找到和设置你打印机合用的纸张大小。如果你找不到所用纸张的名字，度量打印纸的尺寸，然后和 'printoptions' 里支持的纸张大小列表逐一比较，然后用宽 和 高都最接近的纸张。 注意 : 实际的纸张尺寸和列出的大小可能稍有差异。如果没有相近的纸张被列出，那你可以试试以下讨论的 PSUtils 里的 psresize 工具。

- 双面打印 (duplex)。通常，不支持双面打印的 PostScript 打印机只是忽略该请求。但有的打印机就彻底放弃该打印任务。试试关掉 duplex 再打印。 注意 : 用 PSUtils 里的工具可以手动设置双面打印 - 见下。

- 逐份打印 (collated)。和双面打印一样，大多数不支持逐份打印的打印机只是忽略该请求。有的或许不是这样。试试关掉逐份打印的选项。

- 语法高亮。有些打印管理代码不让含有语法高亮的 PostScript 文件在黑白打印机上打印，即使唯一用到的颜色是黑色。试试关掉语法高亮。

一个安全的 printoptions 设置是：

```
:set printoptions=paper:A4,duplex:off,collate:n,syntax:n
```

把 "A4" 换成你的打印机纸最适合的纸张大小。

---

7. PostScript 工具 postscript-print-util

7.1 Ghostscript

Ghostscript 是一个 PostScript 和 PDF 的解释器，从而可以显示和在非 PostScript 的打印机上打印 PostScript 和 PDF 文件。它还可以把 PostScript 文件转换成 PDF 文件。

Ghostscript 可在多种平台上运行。

有三个可用的版本：

- AFPL Ghostscript（以前的 Aladdin Ghostscript）是对非商业用途免费的。它可如下得到：

  http://www.cs.wisc.edu/~ghost/

- GNU Ghostscript 可在 GNU General Public License 下，从以下地址得到：

  ftp://mirror.cs.wisc.edu/pub/mirrors/ghost/gnu/

- 用于商业产品的一个商业版本。

更多关于 Ghostscript 的信息可以在这里找到：

http://www.ghostscript.com/

该发布包含对许多非 PostScript 打印机的标准支持。如果你没找到对你的打印机的支持，查一下 Ghostscript 网站了解如何支持其他缺省没有支持的打印机。

7.2 Ghostscript 预览器

Ghostscript 界面十分原始，所以有不少图形化的前端程序。它们提供了方便的 PostScript 文件选择，不同放大倍数的预览，还有打印。具体细节可查看它们提供的文档。

X11

- Ghostview。可如下取得：

  http://www.cs.wisc.edu/~ghost/gv/

- gv。基于 Ghostview。可如下取得：

  http://wwwthep.physik.uni-mainz.de/~plass/gv/

  这里也能找到（也许不是最新的）：

  http://www.cs.wisc.edu/~ghost/gv/

OpenVMS

- 看起来，在主代码已经支持（未经测试）。见：

  http://wwwthep.physik.uni-mainz.de/~plass/gv/

MS-Windows

- GSview。可如下取得:

    http://www.cs.wisc.edu/~ghost/gsview/

DOS

- ps_view。可如下取得:

    ftp://ftp.pg.gda.pl/pub/TeX/support/ps_view/
    ftp://ftp.dante.de/tex-archive/support/ps_view/

Linux

- GSview。流行的 MS-Windows 的 Linux 版本。可如下取得:

    http://www.cs.wisc.edu/~ghost/gsview/

- BMV。和 Ghostview 与 gv 不同，它不使用 X 而使用 svgalib。可如下取得:

    ftp://sunsite.unc.edu/pub/Linux/apps/graphics/viewers/svga/bmv-1.2.tgz

7.3 PSUtils

PSUtils 是一组操作 Postscript 文档的工具程序。可以得到许多平台上的二进制发布和完全的源代码。PSUtils 可如下得到:

  http://knackered.org/angus/psutils

我们感兴趣的工具包括:

- psnup. 把 PS 文件转化为 N-路打印版本。
- psselect. 选择页范围和打印顺序。
- psresize. 改变页面大小。
- psbook. 重新排列顺序和页面排版，从而打印成册。

一个程序的输出可以作为另一个的输入，从而可以创建复杂的打印文档。

N-路 打 印

psnup 工具把一个 Vim 生成的 PostScript 文件转化为一个 n-路版本 (即在一物理页上打印 n 个逻辑页)。最简单的生成 2-路输出的方法是先如下一个创建 PostScript 文件:

```
:hardcopy > test.ps
```

然后在你的命令行上执行:

```
psnup -n 2 test.ps final.ps
```

注意: 有些 Ghostscript 预览器可能会对 psnup 产生的文件给出警告 - 可以安全地忽略。

最后，用你平台上的打印命令把 final.ps 输出到 PostScript 打印机上。(这以后，你需要自己删除中间文件：两个 PostScript 文件)。可以修改 'printexpr' 来执行这项打印前的额外转换。

双 面 打 印 的 替 代 方 法

用 PS 工具 psselect 可以完成一个原始的双面打印任务。该工具有 -e 和 -o 选项，可以分别打印一个 PS 文件的偶数页和奇数页。

先用 'hardcopy' 命令生成 PS 文件，然后如下生成只包含奇数和偶数页的新文件：

```
psselect -o test.ps odd.ps
psselect -e test.ps even.ps
```

然后用你平台上普通的打印命令打印 odd.ps。然后取出打印输出，翻过来，重新放到取纸器里。然后用你平台上的打印命令打印 even.ps。现在，所有的偶数页面应该出现在奇数页的反面。

要记住以下几点：

1. 第一页的位置。如果打印奇数页时第一页出现在最上方，那你需要倒转奇数页打印的顺序。这可以通过 psselect 的 -r 选项完成。这样可以保证第 2 页打印在第 1 页的反面。
   注意：倒转奇数页的打印顺序比倒转偶数页更合适。因为原始的 PS 文件里可能有奇数页。

2. 纸张方位。取出奇数页的打印输出时，你或者需要横向翻转 (沿着长边)，或者需要纵向翻转 (沿着短边)，甚至有可能要旋转 180 度。这决定于打印机 - 桌上型的喷墨打印机可能比看不见纸张打印的路径的办公室激光打印机更明显一点。

---

8. 换页字符 printing-formfeed

缺省，Vim 不对换页 formfeed 控制字符作任何特殊处理。设置 'printoptions' 的 formfeed 项，使得 Vim 可以识别换页字符，从而在新的一页里打出当前行。换页字符提供了较为原始的打印控制。不过，还是有若干事情需要注意。

Vim 总是从包含换页字符的那行开始打印 (如果有相应设置，还包括行号)，即使它是这一行的第一个字符。这意味着，如果以换页字符开头的一行刚好在一页的第一行，Vim 就会打印完全空白的一页。

因为行号在包含换页字符的一行的开始打印，在新页打印的该行的其余部分就不会有行号 (和 'printoptions' 里置位了 wrap 项时长行的回绕行的打印结果相同)。

如果换页字符是一行的最后一个字符，那么打印会从新页的第二行而不是第一行开始。这是因为 Vim 在换页字符后处理行尾，从而在继续打印之前先往下移动了一行。

根据以上几点，我们推荐如果使用换页字符处理，关闭行号的打印，而且不要在行尾用换页字符。即使如此，你还是需要调整换页符之前的行的数目，以防止偶尔出现的空白页。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者: Willis, tocer
http://vimcdoc.sf.net

Vim 客户-服务器 通信 client-server

1. 通用功能 clientserver
2. X11 专用条目 x11-clientserver
3. MS-Windows 专用条目 w32-clientserver

---

1. 通用功能 clientserver

如果编译时加入 +clientserver 特性，Vim 可以作为命令服务器，接受客户的消息并执行。同时，Vim 也可以作为客户端，发送消息给 Vim 服务器。

可以使用以下的命令行参数:

参数	意义
--remote [+{cmd}] {file} ... --remote	在远端打开文件列表。如果没有 Vim 服务器，就在本地执行。 有一个可选的初始命令: #{cmd}，它必须是后面可以跟 "|" 的 Ex 命令。 命令行的其余部分将作为文件列表。所以，非文件的参数一定要在此之前出现。 你不能用这个方法编辑标准输入 -- 。 远端的 Vim 将被推向前台。如果不想如此，用 vim --remote-send "<C-\><C-N>:n filename<CR>"
--remote-silent [+{cmd}] {file} ... --remote-silent	同上，但如果服务器没打开而需要本地编辑，不产生警告。
--remote-wait [+{cmd}] {file} ... --remote-wait	同 --remote，但直到所有的文件在远端被完成（卸载）为止才返回。
--remote-wait-silent [+{cmd}] {file} ... --remote-wait-silent	同 --remote-wait，但如果服务器没打开，不产生警告。
--remote-tab --remote-tab	同 --remote 但用新标签页打开每个文件。
--remote-tab-silent --remote-tab-silent	同 --remote-silent 但用新标签页打开每个文件。
--remote-tab-wait --remote-tab-wait	同 --remote-wait 但用新标签页打开每个文件。
--remote-tab-wait-silent --remote-tab-wait-silent	同 --remote-wait-silent 但用新标签页打开每个文件。

--servername

--servername {name}　成为用 {name} 作为名字的服务器。如果同时也用了某个 --remote 命令，连接到服务器 {name}，而不使用缺省的名字。（见下）。用大写形式的名字。

--remote-send

--remote-send {keys}　发送键序列 {keys} 到服务器，然后退出。{keys} 不通过映射。能识别特殊的键名，如 "<CR>" 会生成 CR 字符。

--remote-expr

--remote-expr {expr}　在服务器上执行 {expr}。然后在标准输出上打出结果。

--serverlist

--serverlist　输出可用的服务器名字的列表。

示例

在一个已经运行的 GVIM 服务器上编辑 "file.txt"。

```
gvim --remote file.txt
```

在一个已经运行的名为 FOOBAR 的服务器上编辑 "file.txt"。

```
gvim --servername FOOBAR --remote file.txt
```

在名为 "FILES" 的服务器上编辑 "file.txt"，如果不存在这个服务器，自己成为服务器 "FILES"。

```
gvim --servername FILES --remote-silent file.txt
```

下面的例子不能工作。--remote 之后所有的参数都被视作文件名：

```
gvim --remote --servername FOOBAR file.txt
```

在远端服务器上编辑文件 "+foo"（注意 "./" 的使用避免了打头的加号的特别含义）。

```
vim --remote ./+foo
```

告诉远端服务器 "BLA" 写入所有文件，并且退出。

```
vim --servername BLA --remote-send '<C-\><C-N>:wqa<CR>'
```

服 务 器 名　client-server-name

缺省地，Vim 试图用启动时的名字注册服务器名（gvim、egvim ...），你可以用 --servername 参数覆盖之。如果不能用指定的名字，尝试附加一个后缀，直到找到一个空闲的名字为止。比如，在同一个 X 服务器上，第二次启动 gvim 的服务器名会是 "gvim1"。可以通过 v:servername 得到最终的服务器名。这里忽略大小写的区别，所以 "gvim" 和 "GVIM" 重复。

当 Vim 启动时带有 --remote、--remote-wait 或者 --remote-send 参数时，会根据上述的方法由启动名和指定的 --servername 参数决定要访问的服务器名。如果准确的匹配找不到，寻找第一个可以带数字后缀的服务器。不过，如果 --servername 给出的名字已经带有数字后缀，必须准确匹配。

如果使用 --remote 和 --remote-wait 但没有可用的服务器，Vim 会根据剩余的参数启动，自己编辑这些文件。这样，发送命令给 gvim 的时候，就不需要关心它是否已经启动。

--serverlist 参数让 Vim 在标准输出 (stdout) 上打出所有已经注册的命令服务器，然后退出。

Win32 注意事项：不一定总能把 Vim 服务器切换到在前台，因为 MS-Windows 不允许这么做。客户通过 --remote 和 --remote-wait 参数调用而服务器名以 "g" 开始的时候，服务器将被推向前台。

远 程 编 辑

用上面的结果，--remote 参数随后基于其后的参数构造一个 :drop 命令并发送之。--remote-wait 也会这么做，但同时等待所有文件编辑任务的完成。这是通过 BufUnload 事件完成的。文件卸载时，Vim 假设你完成了它的编辑任务。
注意：--remote 和 --remote-list 参数会吃掉命令行余下的部分。也就是说，余下的参数都被认为是文件名，不要把参数放在那里！

函 数

E240 E573

若干 Vim 的函数可以用来书写命令服务器的脚本。参见 eval.txt 的描述，也可以在下面的函数名字上按 CTRL-] 以得到它们完整的描述。

摘要	解释
remote_startserver(name)	运行服务器
remote_expr(server, string, idvar)	发送表达式
remote_send(server, string, idvar)	发送键序列
serverlist()	得到可用服务器的列表
remote_peek(serverid, retvar)	检查返回的字符串
remote_read(serverid)	读取返回的字符串
server2client(serverid, string)	发送返回字符串
remote_foreground(server)	把服务器推向前台

也可参考 CTRL-_CTRL-N ，它可用来引导键序列。
server2client() 的 {serverid} 可以通过 expand("<client>") 得到。

---

2. X11 专用条目

x11-clientserver
E247 E248 E251 E258 E277

客户和服务器的通信是通过 X 服务器进行的。必须指明 Vim 服务器所处的 DISPLAY 位置。通常的 X 服务器的保护措施这里也适用，你必须有权限在 X 服务器上打开一个窗口，这样不同系统间的通信才有可能。

缺省地，GUI 的 Vim 会在 X 服务器上注册一个名字，使得后续的执行能通过这个名字寻址并向它发送字符串。Vim 也可以作为一个客户端，给同一个 X11 显示上的其它 Vim 实例发送字符串。

X11 GUI 的 Vim (gvim) 启动时，会试图在根窗口的 'VimRegistry' 属性上注册发送服务器的名字。

非 GUI 但可以访问到 X11 显示的 Vim (打开了 xterm-clipboard) 也可以作为命令服务器，只要服务器名通过 --servername 参数明确给出，或者 Vim 编译时带 +autoservername 特性就行了。

空的 --servername 参数关闭命令服务器。

要从别的应用程序向 Vim 服务器发送命令，可以阅读源代码 src/if_xcmdsrv.c。那里包含了 Vim 使用的协议的一些情况。

---

3. Win32 专用条目 w32-clientserver

每个 Win32 的 Vim 都可以作为服务器，包括控制台版本。你不需要编译时带 OLE 的版本。我们使用 Windows 消息，它适用于任何版本的 MS-Windows。不过，它只允许在同一个机器上进行通信。

因为我们使用 MS-Windows 的消息，别的应用程序也可以和 Vim 服务器通信。一个替代方案是用 OLE 功能 ole-interface 。

如果用 gvim，--remote-wait 正确的使用方法只能是:

```
start /w gvim --remote-wait file.txt
```

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

term.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　by Bram Moolenaar
译者：Willis，tocer
http://vimcdoc.sf.net

终端信息　terminal-info

Vim 使用你使用的终端的信息来填充屏幕和识别你输入的键。如果信息不正确，屏幕可能会混乱，而一些键也可能识别不了。屏幕上必须执行的动作是通过输出字符串完成的。特殊键会产生一系列字符的序列。这些字符串存在终端选项里，见 terminal-options 。

注意：运行 GUI 的时候这里多数都用不到。

1. 启动　startup-terminal
2. 终端选项　terminal-options
3. 窗口大小　window-size
4. 慢速和快速终端　slow-fast-terminal
5. 使用鼠标　mouse-using

---

1. 启动　startup-terminal

Vim 启动时，假定一个缺省终端类型。Amiga 上，这是一个标准的 CLI 窗口，MS-DOS 是 pc 终端，而 Unix 则是 ansi 终端。还有一些终端类型，它们都总是可用的，见下 builtin-terms 。

使用 '-T' Vim 参数，你可以给出终端名。如果没有给出，Vim 会试图从 TERM 环境变量里读取名字。

termcap　terminfo　E557　E558　E559
Unix 系统使用 terminfo 数据库或者 termcap 文件。在所有的文档里，这被统称为 "termcap"。编译时，configure 的运行会自动决定是用 terminfo 还是 termcap。如果运行 Vim 时，":version" 的输出显示 +terminfo 的话，就意味着使用的是 terminfo。另见 xterm-screens 。

在非 Unix 系统中，termcap 只有在 Vim 编译时定义 TERMCAP 才有效。

builtin-terms　builtin_terms
内建的终端哪些可用，决定于 feature.h 的一些定义。这些需要在编译时确定：

定义	":version" 的输出	内建的终端
NO_BUILTIN_TCAPS	-builtin_terms	无
SOME_BUILTIN_TCAPS	+builtin_terms	大多数常用终端 (缺省)
ALL_BUILTIN_TCAPS	++builtin_terms	全部

用 ":set term=xxx" (在不运行 GUI 的时候)。你可以看到可用的内建终端类型列表。另见 +builtin_terms 。

如果包含了 termcap 的代码，Vim 会试图从 termcap 文件和内建的 termcap 里得到你使用的终端的字符串。如果你使用的终端项目确实存在，两者都会使用。哪个在前面决定于 'ttybuiltin' 选项的设置：

'ttybuiltin' 打开　　1: 内建 termcap　　2: 外部 termcap
'ttybuiltin' 关闭　　1: 外部 termcap　　2: 内建 termcap

如果其中某个选项不存在，就从另外一个那里读取。如果两个都存在，就从排名前面的那个读取。

使用哪个外部 termcap 文件和不同的系统有关，也可能依赖于环境变量 "TERMCAP" 和 "TERMPATH"。见 "man tgetent"。

依赖于终端的设置　　term-dependent-settings

如果你想根据终端名字设置选项或者映射，最好在 .vimrc 里设置。示例：

```
if &term == "xterm"
  ... xterm 映射和设置 ...
elseif &term =~ "vt10."
  ... vt100、vt102 映射和设置 ...
endif
```

raw-terminal-mode

普通的编辑会把终端设为原始 ("raw") 模式。't_ti'、't_TI' 和 't_ks' 定义的字符串会送到终端上。正常情况下，这把终端设为一个状态，该状态下能合法给出 termcap 代码，并激活光标和功能键。
Vim 退出终端时，它会回到 Vim 启动前的模式。't_te'、't_TE' 和 't_ke' 定义的字符串会发送给终端。在 Amiga 上，执行外部命令的命令 (例如 "!!") 会把终端暂时置回普通模式。这意味着你可以按打印键停止到屏幕的输出。输出在按 <BS> 时继续。

注意：如果 't_ti' 非空，Vim 假定它会切换到轮换屏幕上。执行外壳命令或退出 Vim 时产生的结果可能会因此稍有不同。如果不想如此，用 't_TI' 和 't_TE'。

xterm-bracketed-paste

如果设置了 't_BE' 选项，进入 "raw" 模式时会发送 't_BE' 到终端，退出 "raw" 模式时则发送 't_BD'。终端应在粘贴文本之前发送 't_PS'，而粘贴文本之后发送 't_PE'。这样 Vim 就能区隔粘贴的文本和键入的字符。粘贴文本的处理就像使用了鼠标中键，按本义插入而不作命令解释。

光标在首列时，在光标前插入粘贴文本。否则在光标位置之后附加粘贴文本。这意味着你不能在首列之后粘贴。不幸的是 Vim 没有办法知道鼠标指针在哪里。

注意 有些情况下 Vim 不能识别这种括号内的粘贴，而只能得到原始的文本。在其它一些情况下，Vim 只能得到首个粘贴的字符而丢失其余的部分，例如在使用 "r" 命令时。如果你对此不满意，在 .vimrc 这样关闭带括号的粘贴：

```
set t_BE=
```

如果这样做，Vim 运行时发送 't_BD' 给终端以关闭括号内的粘贴。

如果你的终端支持括号内的粘贴但没有自动设置相关选项，可以试试下面：

```
if &term =~ "screen"
  let &t_BE = "\e[?2004h"
  let &t_BD = "\e[?2004l"
  exec "set t_PS=\e[200~"
  exec "set t_PE=\e[201~"
endif
```

cs7-problem

注意：如果终端设置在运行 Vim 以后改变，你可能会碰到不合法的设置组合。有报告说 Solaris 2.5 上 "stty cs8 parenb"，使用 "stty cs7 parenb" 恢复就会有这个问题。如果你使用 "stty cs8 -parenb -istrip"，就能正确恢复了。

有些 termcap 项目在某种意义上是错误的，因为发送 't_ks' 后光标键发送的代码和 termcap 定义的不同。要避免这一点，你需要设置 't_ks' (和 't_ke') 为空值。这必须在初始化时进行 (见 initialization)，不然就太晚了。

有些终端项目假设最高位总是被复位的。例如：Amiga 上光标向上项目可以是 ":ku=\E[A:"。但是 Amiga 实际发送 "\233A"。如果最高位被复位，这没有问题，比如在串行线使用 Amiga 的时候。如果光标键不工作，尝试一下项目 ":ku=\233A:"。

有些 termcap 项目使用 ":ku=\E[A:"。但 Amiga 实际发送 "\233A"。输出结果 "\E[" 和 "\233" 经常是等价的，但输入时则不然。要修正这一问题，你需要修改 termcap 项目，或者用 :set 命令改变键码 (译者注：这段内容有重复，但原文如此)。

许多光标键以单个 <Esc> 开始。Vim 必须找出是否这确实是 <Esc> 键单击，还是光标键序列的开始。它等待下一个字符到达。如果一秒钟不出现，就假设是单个 <Esc>。在很慢的系统里，这可能失败，使得有时候光标键不能工作。如果你发现这一问题，复位 'timeout' 选项。Vim 会在 <Esc> 之后等待下一个字符的来临。如果你需要输入单个 <Esc>，必须输入两次。复位 'esckeys' 选项在插入模式下避免这个问题，但是这样你不能在插入模式下使用光标和功能键。

Amiga 上窗口调整大小的识别只有在终端名为 "amiga" 或 "builtin_amiga" 才有可能。

有些终端的光标键的代码不明确。televideo 925 就是这样的一个终端。光标左键发送的是 CTRL-H。这使得退格和光标左键无法区别。要避免这一问题，CTRL-H 永远不会识别为光标左键。

vt100-cursor-keys xterm-cursor-keys

其它终端 (例如，vt100 和 xterm) 的光标键发送 <Esc>OA、<Esc>OB，等等。不幸的是，这些是插入模式下合法的命令：结束插入，在上面打开新行，开始插入 'A'、'B'，等等。Vim 不会执行这些命令，相反，它会把该输入键序列识别为光标键的移动。要避免这一点而使 Vim 在两种情况下都能工作，你需要以下设置：

```
:set notimeout          " 映射时不检查超时
:set ttimeout           " 终端键码检查超时
:set timeoutlen=100     " 超时为 100 毫秒
```

这要求键码在 100 毫秒的间隔内发送，从而能识别其为光标键。正常情况下，你的输入不会那么快，所以它们能被识别为单独输入的命令，即使 Vim 收到了相同的字节序列。

vt100-function-keys xterm-function-keys

xterm 可以用两种模式发送功能键 F1 到 F4：vt100 兼容模式或不兼容模式。因为 Vim 可能不知道 xterm 发送的是哪种，两种都被识别。<Home> 和 <End> 键情况相同。

		普通	vt100		
<F1>	t_k1	<Esc>[11~	<xF1>	<Esc>OP	<xF1>-xterm
<F2>	t_k2	<Esc>[12~	<xF2>	<Esc>OQ	<xF2>-xterm
<F3>	t_k3	<Esc>[13~	<xF3>	<Esc>OR	<xF3>-xterm
<F4>	t_k4	<Esc>[14~	<xF4>	<Esc>OS	<xF4>-xterm
<Home>	t_kh	<Esc>[7~	<xHome>	<Esc>OH	<xHome>-xterm
<End>	t_@7	<Esc>[4~	<xEnd>	<Esc>OF	<xEnd>-xterm

Vim 启动时，<xF1> 被映射到 <F1>，<xF2> 到 <F2> 等等。这意味着缺省两种代码做相

同的事情。如果你为 <xF2> 定义了映射而你的终端支持两种键，那么就覆盖了缺省的映射，从而可以用 <F2> 和 <xF2> 键做不同的事情。

xterm-shifted-keys

更新的 xterm 版本支持 Shift + 功能键和特殊键。Vim 识别其中的大多数。使用 ":set termcap" 来检查哪些是支持的，相应的代码为何。多数情况下这些不在 termcap 里，只有 builtin_xterm termcap 才支持。

xterm-modifier-keys

新版 xterm 对于大多数功能键支持 Alt 和 Ctrl。为避免增加 Alt、Ctrl 和 Shift 对每个键的捆绑，在 termcap 项目末尾识别一个特殊序列：";*X"。"X" 可以是任何字符，通常是 '~'。";*" 支持一个可选的修饰符参数。";2" 代表 Shift，";3" 是 Alt，";5" 是 Ctrl，";9" 是 Meta（当它与 Alt 不同时）。它们可以组合使用。例如：

```
:set <F8>=^[[19;*~
:set <Home>=^[[1;*H
```

关于这些代码的另一个特性是它们不能被另外的代码覆盖。这就避免了被来自 xterm t_RV 所包含的代码直接覆盖掉。

xterm-scroll-region

Sun 和其它的平台上的 xterm 缺省的 termcap 项目不包含区域滚动的项目。在 /etc/termpcap 的 xterm 项目里增加 ":cs=\E[%i%d;%dr:"，那么一切都没有问题了。

xterm-end-home-keys

在一些系统上（至少使用 XFree86 3.1.2 的 FreeBSD），<End> 和 <Home> 键的代码里包含 <Nul> 字符。要使得这些键发送正确的键码，在你的 ~/.Xdefaults 文件里增加：

```
*VT100.Translations:            #override \n\
                <Key>Home: string("0x1b") string("[7~") \n\
                <Key>End: string("0x1b") string("[8~")
```

xterm-8bit xterm-8-bit

Xterm 可以在使用 8 位转义序列的模式里运行。这里，CSI 代码代替了 <Esc>[。它的优点是，<Esc> 键可以在插入模式下更快的识别，因为不会和特殊键的开始相混淆。
对于内建的 termcap 项目而言，Vim 检查 'term' 选项是否包含 "8bit"。如果是，就在 termcap 项目、鼠标和一些其它的项目中使用 8 位字符，通常，你在外壳里设置 $TERM 为 "xterm-8bit"。Vim 会发现这一点，并自动调整 8 位设置。
如果 Vim 接收到对 t_RV 序列的应答（请求版本）并且它以 CSI 开始，就假设终端处于 8 位模式，并把所有的健序列转换为它们的 8 位版本。

---

2. 终端选项 terminal-options termcap-options E436

终端选项可以和普通选项一样设置。但不会在 ":set all" 命令里显示。相反，应该用 ":set termcap"。

通过设置合适的选项，你总是可以调整个别字符串的值。例如：

```
:set t_ce=^V^[[K        (CTRL-V, <Esc>, [, K)
```

这些选项列在下面。相关的 termcap 代码总是等价于选项名字的最后两个字符。只有一个 termcap 代码是必须的：光标移动，'t_cm'。

选项 't_da'、't_db'、't_ms'、't_xs'、't_xn' 代表 termcap 里的标志位。termcap 标志位存在时，选项总是设为 "y"。但是任何非空的字符串意味着标志位已经置位。而空字符串代表该标志位没有置位。't_CS' 也是这样的，虽然它不是 termcap 的标志位。

输 出 代 码 terminal-output-codes

选项	含义			
t_AB	设置背景色 (ANSI)		t_AB	't_AB'
t_AF	设置前景色 (ANSI)		t_AF	't_AF'
t_AL	增加若干空行		t_AL	't_AL'
t_al	增加一行空行		t_al	't_al'
t_bc	退格字符		t_bc	't_bc'
t_cd	清除到屏幕尾		t_cd	't_cd'
t_ce	清除到行尾		t_ce	't_ce'
t_cl	清除屏幕		t_cl	't_cl'
t_cm	光标移动 (必需！)	E437	t_cm	't_cm'
t_Co	颜色数目		t_Co	't_Co'
t_CS	如果非空，光标相对于滚动区域		t_CS	't_CS'
t_cs	定义滚动区域		t_cs	't_cs'
t_CV	定义垂直滚动区域		t_CV	't_CV'
t_da	如果非空，上方的行可以向下滚动		t_da	't_da'
t_db	如果非空，下方的行可以向上滚动		t_db	't_db'
t_DL	删除若干行		t_DL	't_DL'
t_dl	删除一行		t_dl	't_dl'
t_fs	结束设置窗口标题 (退出 termcap 状态行)		t_fs	't_fs'
t_ke	结束 "keypad 传输" 模式		t_ke	't_ke'
t_ks	进入 "keypad 传输" 模式		t_ks	't_ks'
t_le	左移光标一个字符		t_le	't_le'
t_mb	闪烁模式		t_mb	't_mb'
t_md	加粗模式		t_md	't_md'
t_me	普通模式 (撤销 t_mr、t_mb、t_md 和颜色)		t_me	't_me'
t_mr	反转模式		t_mr	't_mr'
t_ms	如果非空，光标可以在突出/反转模式下移动		t_ms	't_ms'
t_nd	非破坏性的空格字符		t_nd	't_nd'
t_op	复位到原先的颜色对		t_op	't_op'
t_RI	光标右移若干字符		t_RI	't_RI'
t_Sb	设置背景色		t_Sb	't_Sb'
t_Sf	设置前景色		t_Sf	't_Sf'
t_se	结束突出		t_se	't_se'
t_so	退出模式		t_so	't_so'
t_sr	反向滚动 (往下)		t_sr	't_sr'
t_te	退出 "termcap" 模式		t_te	't_te'
t_ti	让终端进入 "termcap" 模式		t_ti	't_ti'
t_ts	开始设置窗口标题 (进入 termcap 状态行)		t_ts	't_ts'
t_ue	下划结束		t_ue	't_ue'
t_us	下划模式		t_us	't_us'
t_ut	使用当前背景色清除		t_ut	't_ut'
t_vb	可视铃声		t_vb	't_vb'
t_ve	光标可见		t_ve	't_ve'
t_vi	光标不可见		t_vi	't_vi'
t_vs	光标 "非常" 可见 (闪烁)		t_vs	't_vs'
t_xs	如果非空，突出部分不能通过覆盖删除 (hpterm)		t_xs	't_xs'
t_xn	如果非空，在屏幕最后单元里写入字符不导致滚动		t_xn	't_xn'
t_ZH	斜体模式		t_ZH	't_ZH'
t_ZR	斜体结束		t_ZR	't_ZR'

Vim 增加的 (它们没有标准代码):

t_Ce	下曲 (undercurl) 结束		t_Ce	't_Ce'

t_Cs	下曲模式	t_Cs 't_Cs'
t_Te	删除线结束	t_Te 't_Te'
t_Ts	删除线模式	t_Ts 't_Ts'
t_IS	开始设置图标文本	t_IS 't_IS'
t_IE	结束设置图标文本	t_IE 't_IE'
t_WP	设置窗口位置 (Y, X)，以像素计算	t_WP 't_WP'
t_GP	获取窗口位置 (Y, X)，以像素计算	t_GP 't_GP'
t_WS	设置窗口大小 (高度, 宽度) 以单元格数计算	t_WS 't_WS'
t_VS	光标正常可见 (无闪烁)	t_VS 't_VS'
t_SI	开始插入模式 (条形光标)	t_SI 't_SI'
t_SR	开始替换模式 (下划线光标)	t_SR 't_SR'
t_EI	结束插入或替换模式 (块形光标) termcap-cursor-shape	t_EI 't_EI'
t_RV	请求终端版本字符串 (适用于 xterm) 其响应存于 v:termresponse xterm-8bit 'ttymouse' xterm-codes	t_RV 't_RV'
t_u7	请求光标位置 (适用于 xterm) 见 'ambiwidth' 其响应存于 v:term7resp	t_u7 't_u7'
t_RF	请求终端前景色 其响应存于 v:termfgresp	t_RF 't_RF'
t_RB	请求终端背景色 其响应存于 v:termbgresp	t_RB 't_RB'
t_8f	设置终端前景色 (R, G, B) xterm-true-color	t_8f 't_8f'
t_8b	设置终端背景色 (R, G, B) xterm-true-color	t_8b 't_8b'
t_BE	打开括号内粘贴模式 xterm-bracketed-paste	t_BE 't_BE'
t_BD	关闭括号内粘贴模式 xterm-bracketed-paste	t_BD 't_BD'
t_SC	开始设置光标颜色	t_SC 't_SC'
t_EC	结束设置光标颜色	t_EC 't_EC'
t_SH	设置光标形状	t_SH 't_SH'
t_RC	请求终端光标闪烁 其响应存于 v:termblinkresp	t_RC 't_RC'
t_RS	请求终端光标风格 其响应存于 v:termstyleresp	t_RS 't_RS'
t_ST	在栈中保存窗口标题	t_ST 't_ST'
t_RT	从栈中恢复窗口标题	t_RT 't_RT'
t_Si	在栈中保存图标文本	t_Si 't_Si'
t_Ri	从栈中恢复图标文本	t_Ri 't_Ri'
t_TE	结束 "raw" 模式	t_TE 't_TE'
t_TI	让终端进入 "raw" 模式	t_TI 't_TI'

有些代码有开始、中间和结束部分。开始和结束由 termcap 选项定义，中间部分是文本。

```
设置标题文本:     t_ts {title text} t_fs
设置图标文本:     t_IS {icon text} t_IE
设置光标颜色:     t_SC  {color name}  t_EC
```

t_SH 必须提供一个参数:

```
0、1 或无        闪烁块状光标
2                块状光标
```

3	闪烁下划光标
4	下划光标
5	产供垂直条光标
6	垂直条光标

仅当接收到 t_RV 的反馈时才会发送 t_RS。Mac OS 上如果 termresponse 不能识别到 Terminal.app 也不会用它。

键 代 码 terminal-key-codes

备注：尽量使用 <> 形式

选项	名字	含义		
t_ku	<Up>	光标上移		t_ku 't_ku'
t_kd	<Down>	光标下移		t_kd 't_kd'
t_kr	<Right>	光标右移		t_kr 't_kr'
t_kl	<Left>	光标左移		t_kl 't_kl'
	<xUp>	替代的光标上移		<xUp>
	<xDown>	替代的光标下移		<xDown>
	<xRight>	替代的光标右移		<xRight>
	<xLeft>	替代的光标左移		<xLeft>
	<S-Up>	shift 光标上移		
	<S-Down>	shift 光标下移		
t_%i	<S-Right>	shift 光标右移		t_%i 't_%i'
t_#4	<S-Left>	shift 光标左移		t_#4 't_#4'
t_k1	<F1>	功能键 1		t_k1 't_k1'
	<xF1>	替代的 F1		<xF1>
t_k2	<F2>	功能键 2	<F2>	t_k2 't_k2'
	<xF2>	替代的 F2		<xF2>
t_k3	<F3>	功能键 3	<F3>	t_k3 't_k3'
	<xF3>	替代的 F3		<xF3>
t_k4	<F4>	功能键 4	<F4>	t_k4 't_k4'
	<xF4>	替代的 F4		<xF4>
t_k5	<F5>	功能键 5	<F5>	t_k5 't_k5'
t_k6	<F6>	功能键 6	<F6>	t_k6 't_k6'
t_k7	<F7>	功能键 7	<F7>	t_k7 't_k7'
t_k8	<F8>	功能键 8	<F8>	t_k8 't_k8'
t_k9	<F9>	功能键 9	<F9>	t_k9 't_k9'
t_k;	<F10>	功能键 10	<F10>	t_k; 't_k;'
t_F1	<F11>	功能键 11	<F11>	t_F1 't_F1'
t_F2	<F12>	功能键 12	<F12>	t_F2 't_F2'
t_F3	<F13>	功能键 13	<F13>	t_F3 't_F3'
t_F4	<F14>	功能键 14	<F14>	t_F4 't_F4'
t_F5	<F15>	功能键 15	<F15>	t_F5 't_F5'
t_F6	<F16>	功能键 16	<F16>	t_F6 't_F6'
t_F7	<F17>	功能键 17	<F17>	t_F7 't_F7'
t_F8	<F18>	功能键 18	<F18>	t_F8 't_F8'
t_F9	<F19>	功能键 19	<F19>	t_F9 't_F9'
	<S-F1>	shift 功能键 1		
	<S-xF1>	替代的 <S-F1>		<S-xF1>
	<S-F2>	shift 功能键 2		<S-F2>
	<S-xF2>	替代的 <S-F2>		<S-xF2>
	<S-F3>	shift 功能键 3		<S-F3>
	<S-xF3>	替代的 <S-F3>		<S-xF3>

	<S-F4>	shift 功能键 4		<S-F4>	
	<S-xF4>	替代的 <S-F4>		<S-xF4>	
	<S-F5>	shift 功能键 5		<S-F5>	
	<S-F6>	shift 功能键 6		<S-F6>	
	<S-F7>	shift 功能键 7		<S-F7>	
	<S-F8>	shift 功能键 8		<S-F8>	
	<S-F9>	shift 功能键 9		<S-F9>	
	<S-F10>	shift 功能键 10		<S-F10>	
	<S-F11>	shift 功能键 11		<S-F11>	
	<S-F12>	shift 功能键 12		<S-F12>	
t_%1	<Help>	帮助键		t_%1	't_%1'
t_&8	<Undo>	撤销键		t_&8	't_&8'
t_kI	<Insert>	插入键		t_kI	't_kI'
t_kD	<Del>	删除键		t_kD	't_kD'
t_kb	<BS>	退格键		t_kb	't_kb'
t_kB	<S-Tab>	反制表 (shift-tab)	<S-Tab>	t_kB	't_kB'
t_kh	<Home>	home 键		t_kh	't_kh'
t_#2	<S-Home>	shift home 键	<S-Home>	t_#2	't_#2'
	<xHome>	替代的 home 键		<xHome>	
t_@7	<End>	end 键		t_@7	't_@7'
t_*7	<S-End>	shift end 键	<S-End>	t_star7	't_star7'
	<xEnd>	替代的 end 键		<xEnd>	
t_kP	<PageUp>	page-up 键		t_kP	't_kP'
t_kN	<PageDown>	page-down 键		t_kN	't_kN'
t_K1	<kHome>	keypad home 键		t_K1	't_K1'
t_K4	<kEnd>	keypad end 键		t_K4	't_K4'
t_K3	<kPageUp>	keypad page-up 键		t_K3	't_K3'
t_K5	<kPageDown>	keypad page-down 键		t_K5	't_K5'
t_K6	<kPlus>	keypad 加号键	<kPlus>	t_K6	't_K6'
t_K7	<kMinus>	keypad 减号键	<kMinus>	t_K7	't_K7'
t_K8	<kDivide>	keypad 除号	<kDivide>	t_K8	't_K8'
t_K9	<kMultiply>	keypad 乘号	<kMultiply>	t_K9	't_K9'
t_KA	<kEnter>	keypad 回车键	<kEnter>	t_KA	't_KA'
t_KB	<kPoint>	keypad 小数点	<kPoint>	t_KB	't_KB'
t_KC	<k0>	keypad 0	<k0>	t_KC	't_KC'
t_KD	<k1>	keypad 1	<k1>	t_KD	't_KD'
t_KE	<k2>	keypad 2	<k2>	t_KE	't_KE'
t_KF	<k3>	keypad 3	<k3>	t_KF	't_KF'
t_KG	<k4>	keypad 4	<k4>	t_KG	't_KG'
t_KH	<k5>	keypad 5	<k5>	t_KH	't_KH'
t_KI	<k6>	keypad 6	<k6>	t_KI	't_KI'
t_KJ	<k7>	keypad 7	<k7>	t_KJ	't_KJ'
t_KK	<k8>	keypad 8	<k8>	t_KK	't_KK'
t_KL	<k9>	keypad 9	<k9>	t_KL	't_KL'
	<Mouse>	鼠标代码的引导键		<Mouse>	
t_PS	括号内粘贴的开始	xterm-bracketed-paste		t_PS	't_PS'
t_PE	括号内粘贴的结束	xterm-bracketed-paste		t_PE	't_PE'

关于 t_so 和 t_mr 的 注意 事项：如果 termcap 项目 "so" 不存在，则使用 "mr" 的项目。反之亦然。"se" 和 "me" 也是一样。如果你的终端同时支持反转和突出模式，你可以看到两种不同的模式。如果只支持其中一种，则两者看起来一样。

keypad-comma

keypad 键在没有映射时和等价的普通键等价。但有一个特例：如果你的 keypad 有逗号

而不是小数点，Vim 仍然认为那是小数点。下面的映射可以修正这个问题：

```
:noremap <kPoint> ,
:noremap! <kPoint> ,
```

xterm-codes

得到键码有一个特殊的技巧，现在这只适用于 xterm。如果定义了 t_RV ，而且得到的响应指示使用的版本是补丁级别不低于 141 的 xterm，Vim 就使用特殊的转义序列来从 xterm 里直接取得键码。响应的内容用于调整各种 t_ 代码。这样，xterm 可能根据所处的模式（8-位、VT102、VT220，等等）产生不同代码的问题就避免了。结果是，<xF1> 这样的代码就不需要了。

注意：这只能在启动时完成。如果 xterm 选项在 Vim 启动以后被改变，这些转义序列就不再能识别了。

xterm-true-color

只要终端支持，Vim 支持终端真彩（使用 highlight-guifg 和 highlight-guibg ）。为此，置位 'termguicolors' 选项。

https://gist.github.com/XVilka/8346728 有支持真彩的终端列表。

有时置位 'termguicolors' 还不够，还需要显式设置 t_8f 和 t_8b 选项。这些选项的缺省值分别是 "^[[38;2;%lu;%lu;%lum" 和 "^[[48;2;%lu;%lu;%lum"，但只当 $TERM 为 xterm 时才设置。有些终端也接受相同的序列，但把所有的分号替换为冒号（这样实际更兼容，但支持不是那么广泛）：

```
let &t_8f = "\<Esc>[38:2:%lu:%lu:%lum"
let &t_8b = "\<Esc>[48:2:%lu:%lu:%lum"
```

这些选项包含 printf 字符串，调用 printf() （实际上，它的 C 等价函数，因此带有 l 修饰符）时带上 t_ 选项值和三个无符号长整数，分别代表红、绿和蓝色的 0 到 255（包含）之间的任意值。

xterm-resize

xterm 改变窗口大小的功能只用在 allowWindowOps 资源打开时才可用。有些系统或有些版本的 xterm 缺省该值是关闭的，因为有安全上的考虑。我们不肯定是否真有此必要。

要覆盖缺省值，在 ~/.Xdefaults 或 ~/.Xresources 中放入：

```
XTerm*allowWindowOps:           true
```

然后运行 "xrdb -merge .Xresources" 以激活之。用上下文菜单可以检查该值（按住 CTRL 键的同时按鼠标右键），在 allow-window-ops 边上应该打了勾。

termcap-colors

关于颜色的 注意 事项：'t_Co' 选项告诉 Vim 可用的颜色数。如果非零，'t_AB' 和 't_AF' 选项可以用来设置颜色。如果这些都不可用，就使用 't_Sb' 和 't_Sf'。't_me' 用来复位到缺省的颜色设置。另见 'termguicolors'。

termcap-cursor-shape termcap-cursor-color

Vim 进入插入模式时，发送 't_SI' 转义序列。Vim 进入替换模式而 't_SR' 已设置时，发送 't_SR' 转义序列，否则发送 't_SI'。当离开插入或替换模式时，发送 't_EI'。这可以用来改变插入模式下光标的形状或颜色。它们不是标准的 termcap/terminfo 项目，你需要自己设定。

一个 xterm 的例子，改变光标颜色：

```
if &term =~ "xterm"
    let &t_SI = "\<Esc>]12;purple\x7"
    let &t_SR = "\<Esc>]12;red\x7"
```

```
        let &t_EI = "\<Esc>]12;blue\x7"
    endif
```

注意：当 Vim 退出时，将保留普通模式的光标形状，而不会恢复 Vim 启动之前的形状。
{仅当编译时加入 +cursorshape 特性才支持}

termcap-title
如果终端允许通过发送字符串设置标题的话，'t_ts' 和 't_fs' 选项用来设置窗口的标题。在发送标题字符串之前和之后分别发送两者。类似的，'t_IS' 和 't_IE' 用来设置图标文本。这些是 Vim 内部对 Unix termcap 的扩展，所以它们的值不能从外部 termcap 获取。不过，内建的 termcap 包含 xterm 和 iris-ansi 的合适的项目，所以你不需要在这里设置。

hpterm
如果反转和其他高亮方式不工作，尝试设置 't_xs' 选项为非空字符串。这使得 't_ce' 代码可以用于从行里撤销高亮。对 "hpterm" 这是必需的。置位 'weiredinvert' 选项和设置 't_xs' 非空的相同的效果，反之亦然。

scroll-region
有些 termcap 不包括 'cs'（滚动区域）的项目，尽管有些终端支持这一特性。比如，Sun 上的 xterm。你可以使用内建的 builtin_xterm 或者自己定义 t_cs。例如：

```
:set t_cs=^V^[[%i%d;%dr
```

其中 ^V 是 CTRL-V，而 ^[是 <Esc>。

垂直滚动区域 t_CV 不是标准的 termcap 代码。Vim 在 GUI 里内部使用之。但是如果你能找到支持这一特性的终端，也可以在终端里定义，两个参数分别是限制滚动的区域的左边列和右边列。就像 t_cs 定义顶部行和底部行那样。定义 t_CV 会使得在垂直分割的窗口里滚动窗口快很多。如果 t_da 或者 t_db 设置了，不要设置 t_CV（文本在滚动时不会清除）。

不幸的是无法从 termcap 里推断光标定位应不应该使用滚动区域：是相对于屏幕的开始处，还是相对于滚动区域的开始处。大多数终端使用第一种方法。已知的一个例外是 MS-DOS 的终端（pcterm）。在光标定位相对于滚动区域的情况下，'t_CS' 选项应该设置为任何非空字符串。反之，则设为空字符串。在 'term' 为 "pcterm" 时，它缺省为 "yes"。

xterm 用户 注意：shift + 光标键一般不工作。要使用 xmodmap 命令和 Vim 里的一些设置才可以。

在 xterm 里给出如下命令：

```
xmodmap -e "keysym Up = Up F13"
xmodmap -e "keysym Down = Down F16"
xmodmap -e "keysym Left = Left F18"
xmodmap -e "keysym Right = Right F19"
```

然后在 Vim 里使用这些映射：

```
:map <t_F3> <S-Up>
:map! <t_F3> <S-Up>
:map <t_F6> <S-Down>
:map! <t_F6> <S-Down>
:map <t_F8> <S-Left>
:map! <t_F8> <S-Left>
:map <t_F9> <S-Right>
:map! <t_F9> <S-Right>
```

与其使用，比如说， <S-Up>，你可以使用任何其它的命令来做你希望 shift + 光标上移做的事情。(注意 : 为了帮助使用有左侧键的 Sun 键盘的人，不要使用 F14，因为它和撤销键有冲突；不要用 F15，因为它执行把窗口转到前台的操作；也不要用 F17，因为它关闭窗口。其他系统上，你也许可以用这些键。)

---

3. 窗口大小 window-size

[这是关于 Vim 使用的整个窗口，不是用 ":split" 命令创建的那种窗口。(译者注: Vim 里的术语或者用屏幕更合适些。)]

如果你在 Amiga 上运行 Vim，而终端名是 "amiga" 或 "builtin_amiga"，打开 amiga 专用的窗口大小调整技术。Unix 系统上，有三个方法可以得到窗口的大小:

- ioctl 调用 (TIOCGSIZE 或者 TIOCGWINSZ，视乎你的系统而定)
- 环境变量 "LINES" 和 "COLUMNS"
- termcap 项目 "li" 和 "co"

如果所有这些都不行，假设缺省大小为 24 行和 80 列。如果收到改变窗口大小的信号，大小会重新设置。如果窗口大小不正确，你可以用 'lines' 和 'columns' 选项来设置正确的值。

这个命令可以设置屏幕大小:

:mod :mode E359

:mod[e] [mode]

如果不带参数，该命令检测屏幕大小并重画屏幕。在 MS-DOS 上，可以切换屏幕模式。[mode] 可以是取以下的值:

"bw40"	40 列黑白
"c40"	40 列彩色
"bw80"	80 列黑白
"c80"	80 列彩色 (大多数人用这个)
"mono"	80 列单色
"c4350"	43 或者 50 行 EGA/VGA 模式
number	使用模式号，取决于你的显示卡

---

4. 慢速和快速终端 slow-fast-terminal slow-terminal

如果你有一个快速终端，你可能愿意置位 'ruler' 选项。光标位置将显示在状态行上。如果你使用横向滚动 (关闭 'wrap' 选项)，考虑把 'sidescroll' 设小一点。

如果你有一个慢速终端，你也许希望复位 'showcmd' 选项。这样，命令字符就不会显示在状态行上。如果终端滚动很慢，把 'scrolljump' 设为 5 左右。如果光标离开屏幕 (例如，用 "j")，Vim 会一次滚动 5 行。另外一个可能性是减少 Vim 使用的行数，通过 "z{height}<CR>" 命令。

如果终端的字符到达时间相距超过 1 秒，你也许希望设置 'timeout' 和/或 'ttimeout' 选项。见 "Options" 一章 options 。

如果你的终端不支持滚动区域，但是支持插入/删除行命令，多个窗口的滚动也许会使得

行跳上跳下。如果你不希望这样，置位 'ttyfast' 选项，它使得窗口重画，而不是行滚动。

如果你的终端滚动很慢，但是重画却不慢，把 'ttyscroll' 设小一点，比如 3。这使得 Vim 在超过 3 行需要滚动时，重画屏幕而不是滚动。

如果你使用一个慢速的彩色终端，使用命令：

```
hi NonText cterm=NONE ctermfg=NONE
```

这避免了空格在改变属性时被重复发送。大多数终端上，你反正也不会看到任何区别。

如果你在慢速的串行线上使用 Vim，你也许想要尝试在 "screen" 程序里运行 Vim。screen 优化终端 I/O 挺不错的。

如果你在测试 termcap 选项，但是你看不到发生了什么，你也许希望设置 'writedelay' 选项。如果非零，每次给终端只发一个字符 (MS-DOS 上不行)。这使得屏幕刷新变慢很多，从而能看清楚发生了什么。

---

5. 使用鼠标 mouse-using

本节说明在终端或者终端窗口里如何使用鼠标。GUI 窗口里如何使用鼠标在 gui-mouse 里解释。关于使用鼠标滚轮进行滚动的部分见 scroll-mouse-wheel 。

不要忘记用以下命令打开鼠标：

```
:set mouse=a
```

否则 Vim 不会在所有模式下识别鼠标 (见 'mouse')。

目前，鼠标只有在 Unix 的 xterm 窗口、带 sysmouse 的 *BSD 控制台、Linux 终端 (带 GPM gpm-mouse)、MS-DOS 和 Windows 终端上才支持。鼠标键击可以用来定位光标，设置区域并粘贴。

'mouse' 选项的字符决定 Vim 在什么场合下会使用鼠标：

n	普通模式
v	可视模式
i	插入模式
c	命令行模式
h	在帮助文件里，以上所有的模式
a	以上所有的模式
r	跳过 hit-enter 提示

'mouse' 的缺省值为空，即不使用鼠标。通常你会用：

```
:set mouse=a
```

来开始使用鼠标 (这等价于设置 'mouse' 为 "nvich")。如果你只想在若干模式下或者在某两个任务上使用鼠标，你需要专注于那些模式对应的字母。例如：

```
:set mouse=nv
```

会使得鼠标在普通模式和可视模式下工作。

```
:set mouse=h
```

会使得鼠标只有在帮助文件里工作 (这样，可以用 "g<LeftMouse>" 来进行标签跳转)。

鼠标能否在可视模式或者选择模式下开始选择，决定于 "selectmode" 选项包不包括 "mouse"。

terminal-mouse

在 xterm 终端里，在 'mouse' 选项包含当前激活的模式的前提下，普通的鼠标键击归

Vim 使用，而带 shift 或者 ctrl 键的鼠标键击由 xterm 控制。如果 'mouse' 不包括当前激活的模式，则所有的鼠标键击由 xterm 控制。

对于自身不能通过修饰符来使用鼠标事件的终端，一个权宜之计是在 Vim 的命令模式中不使用鼠标事件：

```
:set mouse=nvi
```

然后在终端中选择文本，用 ":" 进入命令行模式，选择并复制文本到系统中，然后按 Esc。

另一个办法是临时地用 ":sh" 来运行外壳，复制文本，然后退出外壳。此时 'mouse' 仍应设为 "a"。

xterm-clipboard

Athena 和 Motif GUI 版本上，如果在终端运行但是可以访问 X-server（设置了 DISPLAY），那么复制和粘贴和 GUI 上的行为相似。如果不是，那么鼠标中键会插入无名寄存器。这种情况下，这里是如何复制和粘贴一段文本的方法：

用鼠标和可视模式复制/粘贴（'mouse' 选项必须设置，见上）：
1. 在文本的第一个字符上按鼠标左键，移动鼠标到文本的最后一个字母，然后释放左键。这会启动可视模式并高亮选择区域。
2. 按 "y" 抽出可视文本到无名寄存器里。
3. 在要插入的位置上按鼠标左键。
4. 按鼠标中键。

快捷方式：如果插入位置和可视文本同时出现在屏幕上，你可以一次做 2、3 和 4：在要插入的位置直接按鼠标中键。

注意：如果使用 -X 命令行参数，Vim 不会连接到 X 服务器，所以复制/粘贴到 X 剪贴板（选择区）不会工作。使用带 shift 键的鼠标键来让 xterm 完成选择。

xterm-command-server

如果 X-服务器的剪贴板可用， x11-clientserver 的命令服务器可以在命令行用 --servername 打开。

xterm-copy-paste

注意：在有些（较老的）xterm 版本里，光标不能移动超过第 95 或 223 列。这是 xterm 的问题，不是 Vim 的。用新一点的 xterm 吧 color-xterm 。另见 'ttymouse'。

使用 xterm 粘贴/复制（当前模式 不 包括在 'mouse' 里）：
1. 在文本的第一个字符上按鼠标左键，移动鼠标到文本的最后一个字母，然后释放。
2. 使用普通的 Vim 命令把光标移动到待插入的位置。
3. 按 "a" 开始插入模式。
4. 按鼠标中键。
5. 按 ESC 退出插入模式。

(对包含在 'mouse' 里的模式，这里的操作同样可以完成，只要你在使用鼠标的时候按住 shift 键就行了。)

注意：如果你在粘贴的时候丢失了第 8 位（特殊字符被翻译成其他字符），你需要在启动 Vim 之前在外壳上执行 "stty cs8 -istrip -parenb"。

这样，在 xterm 上 shift 和 ctrl 键不能和鼠标一起使用。需要 CTRL 修饰符的鼠标命令可以在使用鼠标前按 "g" 键模拟：

"g<LeftMouse>" 是 "<C-LeftMouse>" (跳转到鼠标点击的标签上)

"g<RightMouse>" 是 "<C-RightMouse>" ("CTRL-T")

mouse-mode-table mouse-overview

如果 'mousemodel' 是 "extend"，鼠标按钮的功能的概述在此：

普通模式：

事件	移动光标	选择	改变窗口	行动
<LeftMouse>	是	结束	是	
<C-LeftMouse>	是	结束	是	"CTRL-]" (2)
<S-LeftMouse>	是	不改变	是	"*" (2) <S-LeftMouse>
<LeftDrag>	是	开始或者扩展 (1)	否	<LeftDrag>
<LeftRelease>	是	开始或者扩展 (1)	否	
<MiddleMouse>	是	如果没有激活	否	放置
<MiddleMouse>	是	如果激活	否	抽出和放置
<RightMouse>	是	开始或者扩展	是	
<A-RightMouse>	是	开始或者扩展列块	是	<A-RightMouse>
<S-RightMouse>	是	不改变	是	"#" (2) <S-RightMouse>
<C-RightMouse>	否	不改变	否	"CTRL-T"
<RightDrag>	是	扩展	否	<RightDrag>
<RightRelease>	是	扩展	否	<RightRelease>

插入或替换模式：

事件	移动光标	选择	改变窗口	行动
<LeftMouse>	是	(不能激活)	是	
<C-LeftMouse>	是	(不能激活)	是	"CTRL-O^]" (2)
<S-LeftMouse>	是	(不能激活)	是	"CTRL-O*" (2)
<LeftDrag>	是	开始或者扩展 (1)	否	类似于 CTRL-O (1)
<LeftRelease>	是	开始或者扩展 (1)	否	类似于 CTRL-O (1)
<MiddleMouse>	否	(不能激活)	否	放置寄存器内容
<RightMouse>	是	开始或者扩展	是	类似于 CTRL-O
<A-RightMouse>	是	开始或者扩展列块	是	
<S-RightMouse>	是	(不能激活)	是	"CTRL-O#" (2)
<C-RightMouse>	否	(不能激活)	否	"CTRL-O CTRL-T"

在帮助窗口里：

事件	移动光标	选择	改变窗口	行动
<2-LeftMouse>	是	(不能激活)	否	"^]" (帮助标签跳转)

当 'mousemodel' 为 "popup" 时，下面这些有所差异：

普通模式：

事件	移动光标	选择	改变窗口	行动
<S-LeftMouse>	是	开始或者扩展 (1)	否	
<A-LeftMouse>	是	开始或者扩展列块	否	<A-LeftMouse>
<RightMouse>	否	弹出菜单	否	

插入或替换模式：

事件	移动光标	选择	改变窗口	行动
<S-LeftMouse>	是	开始或者扩展 (1)	否	类似于 CTRL-O (1)
<A-LeftMouse>	是	开始或者扩展列块	否	
<RightMouse>	否	弹出菜单	否	

(1) 只有当鼠标在按键后发生移动
(2) 只有当点击发生在同一缓冲区时

点击鼠标左键改变光标的位置。如果点击发生在另一个窗口，那么那个窗口成为活动窗口。在编辑命令行时，光标只能定位在命令行上。在插入模式下，Vim 仍然保持在插入模式。如果设置了 'scrolloff'，而光标定位在窗口边界 'scrolloff' 行范围内，则文本被滚动。

在第一个字符上按鼠标左键，移动鼠标到最后一个字符，然后释放，可以进行选择。直到你释放鼠标键，你不一定总能看到选择的区域。只有一些版本能够 (GUI、MS-DOS、WIN32) 使得拖动立即显示。注意 如果 'scrolloff' 非零，你可以在窗口的首/ 末行移动鼠标至少一个字符位置使文本滚动。

在普通、可视和选择模式下，单击鼠标右键使得可视区域被扩展。如果 'mousemodel' 为 'popup'，必须使用按住 shift 键的鼠标左键。如果在编辑另外一个缓冲区时点击一个窗口，可视和选择模式被终止。

在普通、可视和选择模式下，按下 Alt 键同时单击鼠标右键，可使选择区域成为面向列块。当 'mousemodel' 被设置为 "popup"，鼠标左键必须和 Alt 键一起使用。注意 在某些系统中失效，那些系统中的窗口管理器在按下 Alt 键时会吃掉鼠标事件 (可能会移动窗口)。

double-click

双击、三击和四击在 GUI 激活时、MS-DOS 和 Win32，以及 xterm (如果有 gettimeofday() 函数) 上得到支持。对于文本选择，附加的点击会扩展选择区：

点击	选择	
两次	单词或者 % 匹配	<2-LeftMouse>
三次	行	<3-LeftMouse>
四次	矩形区域	<4-LeftMouse>

特例：在帮助窗口上，双击跳到点击的单词对应的帮助。
双击单词选择该单词。'iskeyword' 用来指定单词可以包括哪些字符。双击一个有匹配的字符选择直到该匹配的区域 (类似于 "v%")。如果匹配是 #if/#else/#endif 块，选择变为面向行。
MS-DOS 和 xterm 上，双击相隔的时间可以通过 'mousetime' 选项设置。其他系统上，该值只能在 Vim 之外定义。
一个使用双击跳转到光标所在的标签的例子：

```
:map <2-LeftMouse> :exe "tag ". expand("<cword>")<CR>
```

使用双击拖动鼠标 (按下按钮、抬起按钮、按下按钮、然后拖动) 会导致完整的多个单词被选择，直到按钮被释放为止。这时选择重新以字符为单位。

gpm-mouse

GPM 鼠标只有在编译时打开 +mouse_gpm 特性才有效。GPM 鼠标驱动 (Linux 终端) 不支持四击。

插入模式下，但选择开始时，Vim 暂时回到普通模式。在可视或者选择模式结束时，又回到插入模式。这类似于在插入模式下使用 CTRL-O 的情形。选择模式在 'selectmode' 选项包含 "mouse" 时被使用。

sysmouse

只有编译时加入 +mouse_sysmouse 特性才会支持 sysmouse。sysmouse 驱动 (*BSD 控制台) 不支持键盘修饰符。

drag-status-line

如果有多个窗口一起工作，窗口的大小可以用通过鼠标拖动状态行来改变。把鼠标点在状态行上，按左键，移动鼠标把状态行带到新的位置，再释放左键。只是用鼠标点击状态行

而不移动鼠标使得那个窗口成为当前窗口。如果选择了一个窗口会改变状态行的位置或大小，状态行的拖动看起来很奇怪，但还是会工作的（试试吧）。

<MiddleRelease> <MiddleDrag>

鼠标点击可以映射。鼠标点击的代码是：

代码	鼠标按钮	一般的行动	
<LeftMouse>	按住左键	设置光标位置	
<LeftDrag>	按住并移动左键	扩展选择区	
<LeftRelease>	释放左键	结束选择区	
<MiddleMouse>	按住中键	在光标所在的位置上粘贴	
<MiddleDrag>	按住并移动中键	-	
<MiddleRelease>	释放中键	-	
<RightMouse>	按住右键	扩展选择区	
<RightDrag>	按住并移动右键	扩展选择区	
<RightRelease>	释放右键	结束选择区	
<X1Mouse>	按住 X1 键	-	X1Mouse
<X1Drag>	按住并移动 X1	-	X1Drag
<X1Release>	释放 X1 键	-	X1Release
<X2Mouse>	按住 X2 键	-	X2Mouse
<X2Drag>	按住并移动 X2	-	X2Drag
<X2Release>	释放 X2 键	-	X2Release

X1 和 X2 按钮指一些鼠标上的附加按钮。'Microsoft Explorer' 鼠标在右拇指位置有这些键。目前 X1 和 X2 只能用于 Win32 和 X11 环境。

示例：

```
:noremap <MiddleMouse> <LeftMouse><MiddleMouse>
```

在鼠标中键点击的位置粘贴（不然，粘贴在光标所在的位置进行）。

```
:noremap <LeftRelease> <LeftRelease>y
```

在可视模式下立即抽出选择区。

注意 使用了 ":noremap" 而不是 "map"，以免发生递归映射。

```
:map <X1Mouse> <C-O>
:map <X2Mouse> <C-I>
```

把 X1 和 X2 键定义为跳转表的前进和后退，见 CTRL-O 和 CTRL-I 。

mouse-swap-buttons

要交换鼠标左键和右键的含义：

```
:noremap  <LeftMouse>   <RightMouse>
:noremap  <LeftDrag>    <RightDrag>
:noremap  <LeftRelease> <RightRelease>
:noremap  <RightMouse>  <LeftMouse>
:noremap  <RightDrag>   <LeftDrag>
:noremap  <RightRelease> <LeftRelease>
:noremap  g<LeftMouse>  <C-RightMouse>
:noremap  g<RightMouse> <C-LeftMouse>
:noremap! <LeftMouse>   <RightMouse>
:noremap! <LeftDrag>    <RightDrag>
:noremap! <LeftRelease> <RightRelease>
:noremap! <RightMouse>  <LeftMouse>
:noremap! <RightDrag>   <LeftDrag>
:noremap! <RightRelease> <LeftRelease>
```

vim:tw=78:ts=8:noet:ft=help:norl:

terminal.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
https://github.com/yianwillis/vimcdoc

终端窗口支持 terminal terminal-window

终端特性是可选的，要检查你的 Vim 是否带有此特性：
```
echo has('terminal')
```
如果返回 "1" 就代表拥有此特性。

1. 基本用法 terminal-use
  键入 terminal-typing
  大小与色彩 terminal-size-color
  命令语法 :terminal
  调整大小 terminal-resizing
  终端模式 Terminal-mode
  光标风格 terminal-cursor-style
  会话 terminal-session
  特殊键 terminal-special-keys
  Unix terminal-unix
  MS-Windows terminal-ms-windows
2. 终端函数 terminal-function-details
3. 终端通信 terminal-communication
  Vim 到作业：term_sendkeys() terminal-to-job
  作业到 Vim：JSON API terminal-api
  使用客户-服务器特性 terminal-client-server
4. 远程测试 terminal-testing
5. 屏幕截图比较 terminal-diff
  编写 Vim 屏幕截图测试 terminal-dumptest
  创建屏幕截图 terminal-screendump
  比较屏幕截图 terminal-diffscreendump
6. 调试 terminal-debug
  启动 termdebug-starting
  示例会话 termdebug-example
  代码中步进 termdebug-stepping
  检视变量 termdebug-variables
  其它命令 termdebug-commands
  提示模式 termdebug-prompt
  通信 termdebug-communication
  定制 termdebug-customizing

{仅当编译时加入 +terminal 特性才有效}
终端特性需要 +job 和 +channel 特性的支持。

---

1. 基本用法 terminal-use

本特性用于在 Vim 窗口中运行终端模拟器。终端模拟器启动时可以连接作业。例如，运

行外壳程序：

```
:term bash
```

或运行编译命令：

```
:term make myprogram
```

作业和 Vim 异步运行，窗口会刷新以显示作业的输出，同时在其它窗口上进行编辑。

键入

terminal-typing

键盘的焦点在终端窗口时，输入的键会发送给其中的作业。可能的话使用 pty。在终端窗口之外点击把键盘焦点移到别处。

t_CTRL-W_CTRL-W t_CTRL-W_:

CTRL-W 可用来在窗口间切换和执行其它 CTRL-W 命令，如：

CTRL-W CTRL-W	移动焦点到下个窗口
CTRL-W :	输入 Ex 命令

更多命令见 CTRL-W 。

终端窗口特有的映射：

t_CTRL-W_. t_CTRL-W_N

CTRL-W .	给终端中的作业发送 CTRL-W
CTRL-W CTRL-\	给终端中的作业发送 CTRL-\
CTRL-W N	切换到终端普通模式，见 Terminal-mode
CTRL-\ CTRL-N	切换到终端普通模式，见 Terminal-mode
CTRL-W " {reg}	粘贴寄存器 {reg} t_CTRL-W_quote 也适用于 = 寄存器，插入表达式计算的结果。
CTRL-W CTRL-C	结束作业，见下 t_CTRL-W_CTRL-C
CTRL-W gt	转到下个标签页，相当于 gt t_CTRL-W_gt
CTRL-W gT	转到上个标签页，相当于 gT t_CTRL-W_gT

另见选项 'termwinkey'，指定 CTRL-W 的替代键，和 CTRL-W 的用法相同。不过，输入 'termwinkey' 两次会给作业发送 'termwinkey'。例如：

'termwinkey' CTRL-W	移动焦点到下个窗口
'termwinkey' :	输入 Ex 命令
'termwinkey' 'termwinkey'	给终端中的作业发送 'termwinkey'
'termwinkey' .	给终端中的作业发送 'termwinkey'
'termwinkey' CTRL-\	给终端中的作业发送 CTRL-\
'termwinkey' N	切换到终端普通模式，见下
'termwinkey' CTRL-N	等价于 CTRL-W N t_CTRL-W_N
'termwinkey' CTRL-C	等价于 CTRL-W CTRL-C t_CTRL-W_CTRL-C

t_CTRL-_CTRL-N

特殊键组合 CTRL-\ CTRL-N 可用于切换到普通模式，其它模式下也都如此。

t_CTRL-W_CTRL-C

CTRL-W CTRL-C 可可用强制结束作业。MS-Windows 上 CTRL-BREAK 也会杀掉作业。

直接按 CTRL-C 的效果取决于 pty 的相应配置。简单命令会导致 SIGINT 发送给作业，结果是中止作业。其它命令可能会忽略 SIGINT 或自行处理 CTRL-C (Vim 就是如此)。

要改变终端模式的映射所用的键，请见 :tmap 。和其它映射的定义类似，但适用于给终端中运行的作业发送的键。例如，要使 F1 切换到终端普通模式：

```
tnoremap <F1> <C-W>N
```

可以用 Esc，但务须保证不影响其它的键 (光标键以 Esc 开始，所以可能会受影响)：

```
tnoremap <Esc> <C-W>N
```

```
set notimeout ttimeout timeoutlen=100
```

类似于终端模式映射，也可建立菜单，但只能用 :tlmenu 而不是 :tmenu 。

options-in-terminal

打开终端窗口并设置其 'buftype' 为 "terminal" 后，激活 TerminalOpen 自动命令。这样便可以设置为其窗口和缓冲区特定的选项。例如:

```
au TerminalOpen * if &buftype == 'terminal' | setlocal bufhidden=hide | endif
```

这里的 <abuf> 设为终端缓冲区，但如果没有窗口（隐藏终端），那么选项会在错误的缓冲区上设置，为此上例中还须检查 &buftype。

鼠标事件（点击和拖曳）会传给终端。而鼠标移动事件只有 Vim 自身接收到时才会传递。对终端而言，这意味着 'balloonevalterm' 打开。

大小与色彩

terminal-size-color

关于控制终端窗口的大小，见 'termwinsize' 选项。
(TODO: 终端超出窗口大小时的滚动)

终端中运行的作业可以改变色彩。缺省前景和背景色来自 Vim 的 Normal 高亮组。

对色彩终端而言，'background' 选项用于决定终端窗口以白色或是黑色背景开始。

要使用其它颜色，使用 Terminal 高亮组，例如:

```
hi Terminal ctermbg=lightgrey ctermfg=blue guibg=lightgrey guifg=blue
```

g:terminal_ansi_colors

GUI 模式或使用 'termguicolors' 时，新的终端窗口缺省使用的 16 种 ANSI 颜色可通过 g:terminal_ansi_colors 设置，它应是 16 种色彩名或十六制色彩代码的列表，类似于 highlight-guifg 接受的格式。如果不使用 GUI 颜色，终端窗口总是使用下层终端的 16 种 ANSI 色。
term_setansicolors() 函数可用来改变颜色，而 term_getansicolors() 可获取当前使用的颜色。

命令语法

:[range]ter[minal] [options] [command] :ter :terminal

打开新终端窗口。

给出 [command] 时，把它作为作业运行，并连接其输入和输出到终端上。
如果没有给出 [command]，使用 'shell' 选项值。
如果 [command] 为 NONE，不启动作业，但终端的 pty 可被 gdb 这样的命令使用。

如果没有给出 [command]，缺省行为是外壳退出时关闭终端。可以用 ++noclose 参数改变此行为。
给出 [command] 时，缺省行为是保持终端打开在终端普通模式下。可以用 ++close 参数改变此行为。

建立新缓冲区，名字使用 [command] 或 'shell' 再加上前缀 "!"。如果带此名的缓冲区已存在，加一个括起的数值。比

如，如果 "gdb" 存在（译者注：似应为 "!gdb"），第二个终端缓冲区会用名字 "!gdb (1)" 。

如果给出 [range]，使用给定行的范围作为作业的输入。此时终端窗口不接受键盘输入。MS-Windows 请见下面的 ++eof 参数。

term++close term++open

支持的 [options] 有：

++close	作业终止时自动关闭终端窗口。 terminal-close
++noclose	作业终止时 不 自动关闭终端窗口。
++open	作业终止时如果没有窗口显示它，会打开窗口。 注意 这会打断正常的操作。

最后出现的 ++close、 ++noclose 和 ++open 才起作用并否决之前的参数。

++curwin	在当前窗口打开终端，不分割当前窗口。如果当前缓冲区不能 abandon 则失败。
++hidden	在隐藏缓冲区打开终端，不使用窗口。
++norestore	不把此终端窗口加到会话文件中。
++shell	不直接执行 {command}，而使用外壳，就像 :!command 那样 E279 {仅能用于 Unix，就目前来说}
++kill={how}	关闭终端窗口的尝试使用 {how} 信号杀死作业。相关值见 term_setkill() 。
++rows={height}	使用 {height} 作为终端窗口高度。如果终端使用整个 Vim 的高度（终端窗口上下没有其它窗口），有必要时会减少命令行高度。
++cols={width}	使用 {width} 作为终端窗口宽度。如果 终端使用整个 Vim 的宽度（终端窗口左右没有其它窗口），忽略此值。
++eof={text}	使用 [range] 时：写完最后一行后发送的文本。之后附加 CR。MS-Windows 缺省发送的是 CTRL-D。 例如，外壳用 "++eof=exit" 而 Python 用 "++eof=exit()"。特殊代码可以用 :map 类似的用法，如 "<C-Z>" 代表 CTRL-Z。
++type={pty}	(仅 MS-Windows)：使用 {pty} 作为虚拟控制台。相关值见 'termwintype'。
++api={expr}	允许以 {expr} 开始的函数名作为 terminal-api 函数调用。如果 {expr} 为空不能调用函数。

想要更多选项，可用 term_start() 函数。
要垂直分割窗口，用：

```
:vertical terminal
```

或更简短：

```
:vert ter
```

和终端相关联的缓冲区被强制卸载或真正删除时，杀死作业，相当于调用 ‘job_stop(job, "kill")‘。

正常关闭窗口会给出 E947 。如果用 "++kill={how}" 或 term_setkill() 给出杀死进程的方法，则窗口关闭时使用该方法杀掉或中断作业。例如：

```
:term ++kill=term tail -f /tmp/log
```

只要作业还在运行，窗口把它当做包含修改过的缓冲区。用 'CTRL-W :quit' 试图关闭窗口会失败。如果用 'CTRL-W :quit!'，作业会结束。但窗口中的文本会丢失。此时缓冲区还存在，但用 :buffer 在别的窗口打开它时会显示空缓冲区。

用 'CTRL-W :close' 试图关闭窗口也会失败。用 'CTRL-W :close!' 会关闭窗口并隐藏缓冲区。

用 'CTRL-W :hide' 关闭终端窗口并隐藏缓冲区，但作业继续执行。 :buffer 命令可用来把当前窗口变成终端窗口。如果有未保存的改变，这会失败，但和平常一样用 ! 可以强制进行。

terminal-close

如果关闭了终端窗口，比如外壳退出且使用了 "++close" 参数时，且此窗口是 Vim 最后一个正常的窗口，Vim 会退出。这类似于普通窗口中用了 :quit 。帮助和预览窗口不计在内。

要不打开窗口在后台运行作业，完成后再打开窗口，可以这样使用选项：

```
:term ++hidden ++open make
```

注意 窗口会在意想不到的时候打开，可能会中断你正在做的事情。

E947 E948

只要作业还在运行，缓冲区被认为是已修改，Vim 不能随便退出，见 abandon 。

作业结束且缓冲区没有发生改动时：关闭窗口会真正删除缓冲区。

在终端缓冲区可以改动之前，'modifiable' 选项必须置位。这只有在作业结束时才有可能。此时，第一个改动会把缓冲区变成正常的缓冲区并删除高亮。你也许要用 :file 来改变缓冲区名以便写入文件，不然缓冲区名仍然对应着命令。

调整大小

terminal-resizing

终端大小处于以下三种模式之一：

1. 'termwinsize' 选项为空：终端大小跟随窗口大小。最小尺寸是 2 行屏幕行，每行 10 个单元格。

2. 'termwinsize' 选项形如 "rows*cols"，其中 "rows" 为屏幕行的最少数目，而 "cols" 是单元格的最少数目。

3. 'termwinsize' 选项形如 "rowsXcols" (其中 x 可以大小写)。终端大小固定为指定的屏幕行和单元格数。如果窗口较之为大，会有多出来的不可用的空白空间。

如果窗口小于终端大小，终端只有部分可见 (左下部分)。

term_getsize() 函数可用于获取终端的当前大小。 term_setsize() 只可用于前两种模式，不能用于 'termwinsize' 为 "rowsXcols" 的情况。

终端作业和终端普通模式

Terminal-mode Terminal-Job
作业运行时终端的内容受作业控制。这也包括光标的位置。输入的键发送给作业。终端的内容可随时改变。这称为终端作业模式。

CTRL-W N（或 'termwinkey' N）切换到终端普通模式。此时，终端窗口内容由 Vim 控制，作业的输出被暂停。CTRL-\ CTRL-N 的效果相同。

终端作业模式适用 :tmap 映射。tmap 不用于 term_sendkeys() 发送的键，但会用于来自 feedkeys() 的键。

从终端作业模式不能进入插入模式。

Terminal-Normal E946
终端普通模式里可用通常的 Vim 命令来移动光标、可视标记文本、抽出文本等等。但不能修改缓冲区的内容。启动插入模式的命令如 'i' 和 'a' 等会回到终端作业模式。窗口会刷新以显示终端的内容。 :startinsert 无效。

终端普通模式下状态行和窗口标题会显示 "(Terminal)"。如果作业已结束且在终端普通模式下，则为 "(Terminal-finished)"。

终端中作业输出的行，包括滚过了屏幕顶部的内容，会被记住且在终端普通模式中可见。行数受 'termwinscroll' 选项限制。超过此限制时，滚动行的前 10% 被删除且丢失。

光标风格

terminal-cursor-style
缺省终端窗口使用不闪烁的块状光标。正常的 xterm 转义序列可用于改变闪烁状态和形状。一但焦点离开终端窗口，Vim 会恢复原有的光标。

一个例外是当 xterm 启动时带 "-bc" 参数或通过别的方式使光标闪烁的时候。这实际意味着反转闪烁标志位。因为 Vim 没法检测这一点，终端窗口的光标闪烁也被反转。

会话

terminal-session
如有可能且用户需要的话，使用会话文件可以恢复终端窗口。

如果 'sessionoptions' 删除了 "terminal"，不恢复终端窗口。

如果终端中运行的作业已经结束，不恢复其窗口。

如果终端可以恢复，会再次使用曾经用来打开它的命令。要改变之，使用 term_setrestore() 函数。把相应的命令设为 "NONE"，也可用这个函数来选择性不恢复特定的终端。

特殊键

terminal-special-keys
因为终端模拟器模拟的是 xterm 终端，终端窗口只能接收 Vim 和 xterm 都能识别的转义序列。要想给终端中运行的作业传递其它转义序列，需要设置转发。示例：

```
tmap <expr> <Esc>]b SendToTerm("\<Esc>]b")
func SendToTerm(what)
  call term_sendkeys('', a:what)
  return ''
```

```
endfunc
```

Unix

terminal-unix

Unix 上使用 pty 来运行所有类型的命令。包含在终端上运行 Vim 自身！这可用调试，见下。

可用环境变量为运行的作业传递信息：

TERM	终端名，来自 'term' 选项或 GUI 里的 $TERM；如果不以 "xterm" 开头的话，回退到 "xterm"
ROWS	终端初始行数
LINES	同 ROWS
COLUMNS	终端初始列数
COLORS	颜色数，'t_Co' (GUI 里是 256*256*256)
VIM_SERVERNAME	v:servername
VIM_TERMINAL	v:version

MS-Windows

terminal-ms-windows

MS-Windows 上使用 winpty 来运行所有类型的命令。显然，必须是终端上运行的命令，不能打开它们自己的窗口。

需要来自 winpty 的两个文件：

```
winpty.dll
winpty-agent.exe
```

可以从下面页面下载它们：

```
https://github.com/rprichard/winpty
```

只有把它们放在你的 PATH 路径某处就行了。有必要的话，设置 'winptydll' 选项指向正确的文件。如果同时有 32 位和 64 位版本，把它们分别换名为 winpty32.dll 和 winpty64.dll 来对应 Vim 编译时的用法。

ConPTY *E982*

在更新的 MS-Widnows 10 版本 (从 "October 2018 Update" 开始)，不再需要 winpty。在这些版本中，:terminal 会使用 Windows 的内建终端应用寄主支持，"ConPTY"。使用 ConPTY 时，可能有二义性宽度字符渲染的痕迹。如果这方面遇到问题，安装 "winpty" 。除非 ConPTY 的问题修正为止，还是建议 "winpty"。

可用环境变量为运行的作业传递信息：

VIM_SERVERNAME	v:servername

---

2. 终端函数

terminal-function-details

term_dumpdiff()

term_dumpdiff({filename}, {filename} [, {options}])
打开新窗口，显示两个文件的差异。必须是 term_dumpwrite() 创建的文件。

返回缓冲区号，如果比较失败返回零。
另见 terminal-diff 。
备注：还未支持双宽度字符。

缓冲区顶部包含第一个文件的内容，底部则包含第二个文件的内容。中间部分显示差异。部分之间以等号线行分隔。

如果给出 {options} 参数，必须是包含以下元素的字典：

"term_name"	用作缓冲区名，缺省是首个文件名。
"term_rows"	终端使用的垂直空间，缺省是 'termwinsize'
"term_cols"	终端使用的水平空间，缺省是 'termwinsize'
"vertical"	垂直分割窗口
"curwin"	使用当前窗口，不分割窗口；如果不能 abandon 当前缓冲区，则报错
"bufnr"	不创建新缓冲区，使用已有缓冲区 "bufnr"。此缓冲区必须是之前用 term_dumpdiff() 或 term_dumpload() 建立的，且在窗口中可见。
"norestore"	不把终端窗口加入会话文件

中间部分的每个字符指示某个差异。如果有多个差异只使用下面列表的第一个选择：

X	不同字符
w	不同宽度
f	不同前景色
b	不同后景色
a	不同属性
+	第一个文件缺失的位置
-	第二个文件缺失的位置

"s" 键交换顶部和底部，以方便发现不同之处。

也可用作 method ：
```
GetFilename()->term_dumpdiff(otherfile)
```

term_dumpload()

term_dumpload({filename} [, {options}])
打开新窗口显示 {filename} 内容。必须是 term_dumpwrite() 创建的文件。
返回缓冲区号，如果失败返回零。
另见 terminal-diff 。

{options} 可见 term_dumpdiff() 。

也可用作 method ：
```
GetFilename()->term_dumpload()
```

term_dumpwrite()

term_dumpwrite({buf}, {filename} [, {options}])
把 {buf} 的终端屏幕的内容保存到 {filename} 文件里。使用的格式可用于 term_dumpload() 和 term_dumpdiff() 。
如果终端中的作业已经结束，报错。 E958
如果 {filename} 已存在，报错。 E953
另见 terminal-diff 。

{options} 是包含以下可选项目的字典：
"rows" 保存的最大行数
"columns" 保存的最大列数

也可用作 method ，基用于文件名：
GetFilename()->term_dumpwrite(bufnr)

term_getaltscreen({buf}) term_getaltscreen()
如果 {buf} 的终端使用轮换屏幕，则返回 1。
{buf} 的用法同 term_getsize() 。

也可用作 method ：
GetBufnr()->term_getaltscreen()

term_getansicolors({buf}) term_getansicolors()
获得终端 {buf} 的 ANSI 色彩调色板。
返回 16 个元素的列表，每个元素是代表颜色的字符串，使用十六进制 "#rrggbb" 格式。
另见 term_setansicolors() 和 g:terminal_ansi_colors 。
如果两者都不用，返回缺省颜色。

{buf} 的用法同 term_getsize() 。如果缓冲区不存在或不是终端窗口，返回空列表。

也可用作 method ：
GetBufnr()->term_getansicolors()

{仅当编译时打开 GUI 和/或 +termguicolors 特性时才有效}

term_getattr({attr}, {what}) term_getattr()
给出一个由 term_scrape() 的 "attr" 项目返回的 {attr}，返回它是否打开了 {what}。{what} 是以下之一：
bold (粗体)
italic (斜体)
underline (带下划线)
strike (带删除线)
reverse (反相)

也可用作 method ：
GetAttr()->term_getattr()

term_getcursor({buf}) term_getcursor()
获取 {buf} 终端所在的光标位置。返回两个数值和一个字典组成的列表：[row, col, dict]。

"row" 和 "col" 从 1 开始，比如，首个屏幕单元格在第一列第一行。这是终端本身所在的光标位置，不是 Vim 窗口的光标位置。

"dict" 可有以下成员：
"visible" 一如果光标可见，零如果光标隐藏。
"blink" 一如果光标闪烁，零如果光标不闪烁。
"shape" 块状光标为 1，下划线为 2，垂直线为 3。

"color"　　光标的颜色，例如 "green"

{buf} 必须是对应终端窗口的缓冲区号。如果该缓冲区不存在或不是终端窗口，返回空列表。

也可用作 method :
```
GetBufnr()->term_getcursor()
```

term_getjob({buf})　　term_getjob()

获取终端窗口 {buf} 相关联的作业。
{buf} 的用法见 term_getsize() 。
没有相关作业时返回 v:null 。

也可用作 method :
```
GetBufnr()->term_getjob()
```

term_getline({buf}, {row})　　term_getline()

获取终端窗口 {buf} 的文本行。
{buf} 的用法见 term_getsize() 。

首行的 {row} 数为一。{row} 为 "." 时使用光标所在行。{row} 非法时返回空串。

要获取每个字符的属性可用 term_scrape() 。

也可用作 method :
```
GetBufnr()->term_getline(row)
```

term_getscrolled({buf})　　term_getscrolled()

返回终端 {buf} 滚动过顶部的行数。也就是 term_getline() 和 getline() 所用行号的相对偏移，亦即：
```
term_getline(buf, N)
```
等价于：
```
getline(N + term_getscrolled(buf))
```
(如果该行存在)。

{buf} 的用法见 term_getsize() 。

也可用作 method :
```
GetBufnr()->term_getscrolled()
```

term_getsize({buf})　　term_getsize()

返回终端 {buf} 的大小，即两个数值组成的列表：[rows, cols]。
这是终端的大小，不是包含终端窗口的大小。

{buf} 必须是对应终端窗口的缓冲区号。空串代表当前缓冲区。如果该缓冲区不存在或不是终端窗口，返回空列表。

也可用作 method :
```
GetBufnr()->term_getsize()
```

term_getstatus({buf}) term_getstatus()

返回终端 {buf} 的状态，即逗号分隔的项目列表：

running	作业在运行中
finished	作业已完成
normal	在终端普通模式中

"running" 或 "finished" 两者必有其一。

{buf} 必须是对应终端窗口的缓冲区号。如果该缓冲区不存在或不是终端窗口，返回空串。

也可用作 method :

```
GetBufnr()->term_getstatus()
```

term_gettitle({buf}) term_gettitle()

获取终端 {buf} 的标题。这是终端中的作业设置的标题。

{buf} 必须是对应终端窗口的缓冲区号。如果该缓冲区不存在或不是终端窗口，返回空串。

也可用作 method :

```
GetBufnr()->term_gettitle()
```

term_gettty({buf} [, {input}]) term_gettty()

获取终端窗口 {buf} 相关联的控制终端的名字。{buf} 的用法见 term_getsize() 。

{input} 省略或为 0 时，返回用于写入的终端名 (stdout)。{input} 为 1 时，返回用于读取的终端名 (stdin)。Unix 上，两者有相同的名字。

也可用作 method :

```
GetBufnr()->term_gettty()
```

term_list() term_list()

返回所有终端窗口的缓冲区号的列表。

term_scrape({buf}, {row}) term_scrape()

获取终端 {buf} 屏幕第 {row} 行的内容。{buf} 的用法见 term_getsize() 。

首行的 {row} 行为一。{row} 为 "." 时使用光标所在行。{row} 非法时返回空串 (译者注：应为空列表)。

返回对应每个屏幕单元格的字典的列表：

"chars"	该单元格上的字符 (可有多个)
"fg"	前景色，形如 #rrggbb
"bg"	背景色，形如 #rrggbb
"attr"	单元格的属性，可用 term_getattr() 获取个别标志位
"width"	单元格宽度：1 或 2

也可用作 method :
```
GetBufnr()->term_scrape(row)
```

term_sendkeys({buf}, {keys}) term_sendkeys()
发送键击序列 {keys} 给终端 {buf}。
{buf} 的用法见 term_getsize() 。

{keys} 作为键击序列进行翻译。例如，"\<c-x>" 代表 CTRL-X 字符。

也可用作 method :
```
GetBufnr()->term_sendkeys(keys)
```

term_setapi({buf}, {expr}) term_setapi()
设置终端 {buf} 中的 terminal-api 函数使用的函数名前缀。
例如:
```
:call term_setapi(buf, "Myapi_")
:call term_setapi(buf, "")
```

缺省是 "Tapi_"。{expr} 为空串时 {buf} 不能使用 terminal-api 函数。

term_setansicolors({buf}, {colors}) term_setansicolors()
设置终端 {buf} 使用的 ANSI 颜色调色板。
{colors} 必须是 16 种合法的色彩名或十六进制色彩代码的列表，如同 highlight-guifg 接受的格式一样。
另见 term_getansicolors() 和 g:terminal_ansi_colors 。

这些颜色通常是:
```
0    black
1    dark red
2    dark green
3    brown
4    dark blue
5    dark magenta
6    dark cyan
7    light grey
8    dark grey
9    red
10   green
11   yellow
12   blue
13   magenta
14   cyan
15   white
```

在 GUI 和 'termguicolors' 置位时在终端上使用这些颜色。如果不使用 GUI 颜色 (GUI 模式或 'termguicolors')，终端窗口总是使用下层终端的 16 种 ANSI 颜色。

也可用作 method :
```
GetBufnr()->term_setansicolors(colors)
```

{仅当编译时打开 GUI 和/或 +termguicolors 特性时才有效}

term_setkill({buf}, {how}) term_setkill()
退出 Vim 或用其它方式试图关闭终端窗口时，{how} 定义终端中的作业是否被中止。
若 {how} 为空（缺省），不中止作业。退出尝试会生成 E947 。
否则，{how} 指定发送给作业的信号。 job_stop() 说明可选的值。

发送信号后 Vim 等待不多于一秒的时间来检查作业是否确实中止。

也可用作 method :
```
GetBufnr()->term_setkill(how)
```

term_setrestore({buf}, {command}) term_setrestore()
设置写在会活文件中以恢复终端中的作业的命令。会话文件中写入的行是:
```
terminal ++curwin ++cols=%d ++rows=%d {command}
```
要确保对命令适当地转义。

使用空 {command} 来执行 'shell'。
使用 "NONE" 来不从此窗口恢复。

也可用作 method :
```
GetBufnr()->term_setrestore(command)
```

term_setsize({buf}, {rows}, {cols}) term_setsize() E955
设置终端 {buf} 的大小。如果可能，包含终端的窗口的尺寸也会跟着调整。如果 {rows} 或 {cols} 为零或负，大小不改变。

{buf} 必须是对应终端窗口的缓冲区号。空串代表当前缓冲区。如果该缓冲区不存在或不是终端窗口，报错。

也可用作 method :
```
GetBufnr()->term_setsize(rows, cols)
```

term_start({cmd} [, {options}]) term_start()
打开终端窗口并在其中运行 {cmd}。

类似于 job_start() ，{cmd} 可以是字符串或者列表。字符串 "NONE" 可用来打开终端窗口而不启动作业，终端的 pty 可用于 gdb 这样的命令。

返回终端窗口的缓冲区号。如果不能执行 {cmd}，仍打开窗口并显示错误信息。
如果打开窗口失败，返回零。

{options} 和用于 job_start() 的选项类似，见 job-options 。不过，不是所有的选项都可用。以下是支持的选项:
所有超时选项
"stoponexit"、"cwd"、"env"

"callback"、"out_cb"、"err_cb"、"exit_cb"、"close_cb"
"in_io"、"in_top"、"in_bot"、"in_name"、"in_buf"
"out_io"、"out_name"、"out_buf"、"out_modifiable"、"out_msg"
"err_io"、"err_name"、"err_buf"、"err_modifiable"、"err_msg"

不过，至少 stdin、stdout 或 stderr 其中之一必须连接到终端上。
I/O 和终端连接时，不使用该部分的回调函数。

以下是额外的选项：

"term_name"	缓冲区名，缺省是命令名。
"term_rows"	终端使用的垂直尺寸，缺省是 'termwinsize'
"term_cols"	终端使用的水平尺寸，缺省是 'termwinsize'
"vertical"	垂直分割窗口；备注 另一窗口位置可用命令行修饰符来定义，如 :belowright 。
"curwin"	使用当前窗口，不分割窗口；如果不能 abandon 当前缓冲区，则报错
"hidden"	不打开窗口
"norestore"	不把终端窗口加入到会话文件中
"term_kill"	试图关闭终端应如何操作，见 term_setkill()
"term_finish"	作业结束时的行为： "close": 关闭任何窗口 "open": 有必要时打开窗口 注意 "open" 可能会打断你的操作。 见 term++close 和 term++open 。
"term_opencmd"	"term_finish" 用 "open" 时打开窗口所用的命令；必须包含 "%d"，放在需要缓冲区号的位置上，如 "10split|buffer %d"；缺省是 "botright sbuf %d"
"eof_chars"	发送完缓冲区的所有行后写入终端的文本。缺省在 MS-Windows 上是 CTRL-D，Python 使用 CTRL-Z 或 "exit()"。外壳使用 "exit"。总会加上 CR。
"ansi_colors"	定义用于 GUI 色彩模式的 ANSI 调色板的十六种色彩名或十六进代码的列表。见 g:terminal_ansi_colors 。
"tty_type"	(仅用于 MS-Windows): 指定使用的 pty。'termwintype' 说明可用的值。
"term_api"	terminal-api 函数的函数名前缀。见 term_setapi() 。

也可用作 method :

```
GetCommand()->term_start()
```

term_wait({buf} [, {time}]) term_wait()

等待 {buf} 未处理的更新。
{buf} 的用法见 term_getsize() 。
{time} 是等待更新到达以毫秒计的时间。未指定则使用 10 毫秒。
也可用作 method :

```
GetBufnr()->term_wait()
```

---

3. 终端通信 terminal-communication

有几种和终端中运行的作业通信的办法：
- 使用 term_sendkeys() 从 Vim 发送文本和转义序列到作业。
- 使用 JSON API 从作业发送经编码的命令到 Vim。
- 使用 client-server 机制。适用于有 X server 和 MS-Windows 的机器。

Vim 到 job: term_sendkeys()

terminal-to-job

这允许运程控制在终端中运行的作业。这是单向的机制。但作业可以刷新显示给 Vim 传递信号。例如，如果终端中运行的是外壳，可以这样：

```
call term_sendkeys(buf, "ls *.java\<CR>")
```

这需要作业处于正确的状态，以便接收键击时，可以作出正确的反应。在上例中，外壳必须正在等待命令行的键入。

如果作业支持，可以反方向发送 JSON API 转义序列，如：

```
call term_sendkeys(buf, "\<Esc>]51;["response"]\x07")
```

Job 到 Vim: JSON API

terminal-api

作业可以使用特殊的转义序列发送 JSON 给 Vi。JSON 编码一个 Vim 能理解的命令。一个这样信息的示例：

```
<Esc>]51;["drop", "README.md"]<07>
```

本体总是列表，以便寻找结尾处：]<07>。
<Esc>]51;msg<07> 序列是 xterm 保留给 "Emacs shell" 的，和我们要做的差不多吧。

目前支持的命令：

call {funcname} {argument}

调用用户定义函数并带参数 {argument}。
函数调用时有两个参数：终端缓冲区号和 {argument}，一个解码的 JSON 参数。
缺省，函数名必须以 "Tapi_" 开始，以免不小心调用了为非终端 API 设计的函数。可用 term_setapi() 改变。
用户函数须检查参数的合法性。
函数可以用 term_sendkeys() 来发送反馈。
JSON 示例：

```
["call", "Tapi_Impression", ["play", 14]]
```

会调用以下定义的函数：

```
function Tapi_Impression(bufnum, arglist)
  if len(a:arglist) == 2
    echomsg "impression " . a:arglist[0]
    echomsg "count " . a:arglist[1]
  endif
endfunc
```

如果用 :echo ，重画可能被删除其输出，用 :echomsg 就会保证在 :messages 中看到这些消息。

drop {filename} [options]

让 Vim 打开文件，就像 :drop 命令那样。如果 {filename} 已经在窗口中打开，切换到该窗口。否则打开新窗口，并编辑 {filename}。
注意 作业和 Vim 都可能改变当前目录，所以最好使用完整路径。

[options] 只在打开新窗口里有用。如果给出，必须是字典。和 ++opt 类似，识别以下项目：

"ff"	文件格式："dos"、"mac" 或 "unix"
"fileformat"	同上
"enc"	覆盖 'fileencoding'
"encoding"	同上
"bin"	置位 'binary'
"binary"	同上
"nobin"	复位 'binary'
"nobinary"	同上
"bad"	指定坏字符的行为，见 ++bad

JSON 示例：

```
["drop", "path/file.txt", {"ff": "dos"}]
```

让 vim 发送此转义序列的一个小技巧：

```
exe "set t_ts=\<Esc>]51; t_fs=\x07"
let &titlestring = '["call","Tapi_TryThis",["hello",123]]'
redraw
set t_ts& t_fs&
```

理据：为什么不允许任何命令或表达式呢？因为可能会造成安全问题。

使用客户-服务器特性

terminal-client-server

只在 v:servername 非空时有用。如果需要可以这样在打开终端前设置之：

```
call remote_startserver('vim-server')
```

在终端里，$VIM_SERVERNAME 会设为传递进来的服务器名。

在作业中可以这样做：

```
vim --servername $VIM_SERVERNAME --remote +123 some_file.c
```

这会打开文件 "some_file.c" 并把光标放在第 123 行上。

---

4. 远程测试

terminal-testing

多数 Vim 测试在 Vim 内部执行脚本。有些测试这样不行，因为测试的执行和要测试的代码有干扰。为了避免这种干扰，可在终端窗口中执行 Vim。测试给它发送键击并检视结果的屏幕状态。

函数

term_sendkeys()	给终端发送键击（不应用映射）
term_wait()	等待屏幕的刷新
term_scrape()	检视终端屏幕

---

5. 屏幕截图比较 terminal-diff

有些情况下要测试 Vim 在屏幕上是否显示正确的字符很麻烦，例如带语法高亮的时候。现在有一个略简便的方法，为终端进行屏幕截图，并和期待的屏幕截图相比较。

Vim 使用实际显示窗口的大小、文本、颜色和其它属性。Vim 的屏幕大小、字体和其它属性是无关的。这样此机制在系统间可移植。而常规的屏幕截图会反映所有的差异，包括字体大小和家族等。

编写 Vim 屏幕截图测试

terminal-dumptest

一个实际的例子可见 src/testdir/test_syntax.vim 中的 Test_syntax_c() 函数。其主要步骤是：

- 写一个待测试的文件。用于测试语法高亮。也可以从空缓冲区启动 Vim。
- 在指定大小的终端中运行 Vim。缺省是 20 行每行 75 字符。这保证截图有恒定的大小。 函数 RunVimInTerminal() 会做到这一点。把 Vim 命令的参数传递给它就可以了。
- 用 term_sendkeys() 给 Vim 发送任何命令。例如：

```
call term_sendkeys(buf, ":echo &lines &columns\<CR>")
```

- 用 VerifyScreenDump() 检查屏幕现在匹配期待的状态。它假定参考屏幕截图在 src/testdir/dumps/ 目录下，名字不需要 ".dump" 部分。建议使用测试函数的名字加一个序列号，以便我们知道什么测试使用此文件。
- 重复发送命令和检查状态的步骤。
- 最后调用 StopVimInTerminal() 来停止 Vim。

第一次你还没有屏幕截图。先建立空文件，如：

```
touch src/testdir/dumps/Test_function_name_01.dump
```

此时测试会失败，并给出比较参考截图和失败截图的命令，如：

```
call term_dumpdiff("failed/Test_func.dump", "dumps/Test_func.dump")
```

把当前目录设为 src/testdir，在 Vim 中使用此命令。一旦你对测试满意了，把失败截图移动到参考截图的位置：

```
:!mv failed/Test_func.dump dumps/Test_func.dump
```

创建屏幕截图

terminal-screendump

要创建屏幕截图，在终端中运行 Vim (或其它程序)，并使它显示想要的状态。然后用 term_dumpwrite() 函数来创建屏幕截图文件。例如：

```
:call term_dumpwrite(77, "mysyntax.dump")
```

这里 "77" 是终端缓冲区号。 :ls! 可以看到。

要浏览屏幕截图，可用 term_dumpload() ：

```
:call term_dumpload("mysyntax.dump")
```

要验证 Vim 还显示完全相同的屏幕，一模一样地再运行一次 Vim，显示想要的状态。然后用另一个文件名再创建一份屏幕截图：

```
:call term_dumpwrite(88, "test.dump")
```

要断言两份文件完全相同，可用 assert_equalfile() ：

```
call assert_equalfile("mysyntax.dump", "test.dump")
```

如果有不同，v:errors 会包含错误信息。

比较屏幕截图

terminal-diffscreendump

assert_equalfile() 不能方便地看到差异之处。要发现问题，可用 term_dumpdiff()：

```
call term_dumpdiff("mysyntax.dump", "test.dump")
```

这会打开一个窗口，包含三个部分：
1. 第一份截图的内容
2. 两份截图的差异
3. 第二份截图的内容

你通常在第二部分看到差异之处。'ruler' 可用来把它和第一份或第二份截图的位置联系起来。以下字母指示区别的类型：

X	不同字符
>	光标在第一份但不在第二份
<	光标在第二份但不在第一份
w	不同字符宽度（单宽度对双宽度）
f	不同前景色
b	不同背景色
a	不同属性（粗体、下划线、反显等等）
?	两者都缺少的字符
+	第一份缺少的字符
-	第二份缺少的字符

另外，按 "s" 会交换两份截图。多做几次，在文本的上下文中找到差异会容易一些。

---

6. 调试

terminal-debug terminal-debugger

终端调试插件可用来使用 gdb 来调试程序而同时在 Vim 窗口中查看源代码。因为这完全包含在 Vim 内完成，我们通过 ssh 连接也可以远程进行工作。

如果没有 +terminal 特性但可能的话，此插件会使用 "prompt" 缓冲区类型。详见下述的 termdebug-prompt 。

启动

termdebug-starting

此命令载入插件：

```
packadd termdebug
```

:Termdebug

要启动调试，用 :Termdebug 或 :TermdebugCommand 后跟命令名，如：

```
:Termdebug vim
```

这会打开两个窗口：

gdb 窗口	终端窗口，其中执行 "gdb vim"。这里你可以直接和 gdb 交互。缓冲区名是 "!gdb"。

program 窗口　　用于执行程序的终端窗口，。gdb 里 "run" 时，程序 I/O 会在此窗口中发生，这样不会和控制的 gdb 有干扰。
缓冲区名是 "gdb program"。

当前窗口用于显示源代码。gdb 暂停时，如有可能会显示源代码的位置。标号用来高亮当前位置，使用高亮组 debugPC。

如果当前窗口缓冲区被修改，会打开另一个窗口来显示 gdb 的当前位置。用 :Winbar 可以在那里加上窗口工具条。

聚焦执行程序的终端可以和它互动。这和终端窗口运行任何命令的用法一样。

通常在 gdb 窗口中输入 "quit" 可以结束调试器。此时，这两个打开的窗口会被关闭。

一次只能激活一个调试器。

:TermdebugCommand

要指定要调试的命令，可用 :TermdebugCommand 命令后跟命令名和其它参数。

```
:TermdebugCommand vim --clean -c ':set nu'
```

:Termdebug 和 :TermdebugCommand 两者都支持可选的 "!" 感叹号参数，它会立即启动命令而不暂停在 gdb 窗口中（光标会留在待调试的窗口）。示例：

```
:TermdebugCommand! vim --clean
```

要把 gdb 和已经在运行的可执行程序或 core 文件绑定，传递额外的参数。如：

```
:Termdebug vim core
:Termdebug vim 98343
```

如果不给出参数，会打开 gdb 窗口，那里要通过如 gdb file 命令等方式去执行想要的命令。

示例会话

termdebug-example

从 Vim "src" 目录开始并编译 Vim：

```
% make
```

确保有调试符号，通常这意味着 $CFLAGS 应包含 "-g"。

启动 Vim：

```
% ./vim
```

载入 termdebug 插件并开始调试 Vim：

```
:packadd termdebug
:Termdebug vim
```

你现在应有三个窗口：

源代码 - 你开始的地方，有带按钮的窗口工具条
gdb - 这里你开始输入 gdb 命令
program - 执行中程序会使用本窗口

可用 CTRL-W CTRL-W 或鼠标在窗口间转换焦点。把焦点放在 gdb 窗口上并输入：

```
break ex_help
run
```

Vim 会在 program 窗口上开始运行。把焦点放在那里并输入：

```
:help gui
```

Gdb 会在 ex_help 断点处运行。源代码窗口此时会显示 ex_cmds.c 文件。一个红色的

"1 " 标记会出现在标号列里，指示断点所在的位置。调试器所在的行被高亮。现在你可以在程序中步进了。让我们用下鼠标：在窗口工具条的 "Next" 按钮上点击。你会看到随着调试器执行一行源代码，高亮也跟着移动。

点击 "Next" 几次直到 for 循环被高亮。把光标放在 "eap->arg" 的尾部，然后按工具条上的 "Eval"。你会看到：

```
"eap->arg": 0x555555e68855 "gui"
```

这样你可以检视局部变量的值。也可以聚焦在 gdb 窗口上并使用 "print" 命令，如：

```
print *eap
```

如果支持鼠标指针移动，Vim 也会在鼠标停留的文本上显示通过 gdb 计算的气泡。

现在回到源代码窗口，把光标放在 for 循环后的第一行，然后输入：

```
:Break
```

你会看到 ">>" 标记出现，指示一个新的断点。现在在工具条上点击 "Cont"，这会继续执行代码直到断点为止。

gdb 窗口上可以输入更高级的命令。例如，输入：

```
watch curbuf
```

现在在工具条点击 "Cont" (或在 gdb 窗口中输入 "cont")。现在会继续执行直到 "curbuf" 的值有改变，这应在 do_ecmd() 里。
要删除此观察点，在 gdb 窗口中再输入：

```
delete 3
```

要看到堆栈，在gdb 窗口中输入：

```
where
```

在堆栈框架间移动，比如可用：

```
frame 3
```

源代码窗口会显示在更深层的调用所在的代码。

代码中步进

termdebug-stepping

把焦点移到 gdb 窗口，在那里输入命令。常用的有：

- CTRL-C　　中断程序
- next　　执行当前行并停在下一行
- step　　执行当前行并停在下个语句上，进入函数
- finish　　执行直到退出当前函数为止
- where　　显示堆栈
- frame N　　转到第 N 堆栈框架
- continue　　继续执行

:Run　:Arguments

在显示源代码的窗口中以下命令可用于控制 gdb：

:Run [args]	执行程序，使用 [args] 或之前的参数
:Arguments {args}	设置下次 :Run 使用的参数
:Break	在光标位置设置断点
:Break {position}	在指定位置设置断点
:Clear	删除光标位置所在的断点
:Step	执行 gdb "step" 命令
:Over	执行 gdb "next" 命令 (:Next 是 Vim 的命令)
:Finish	执行 gdb "finish" 命令

```
:Continue       执行 gdb "continue" 命令
:Stop           中断程序
```

如果设置了 'mouse'，插件加入窗口工具条，带以下项目：

```
Step            :Step
Next            :Over
Finish          :Finish
Cont            :Continue
Stop            :Stop
Eval            :Evaluate
```

这样你就可用鼠标完成大部分的常用操作。需要设置 'mouse' 选项来支持鼠标点击。

:Winbar

你可以这样在其它打开的窗口加入窗口工具条：

```
:Winbar
```

如果 gdb 停在某个源代码行上而当前没有窗口显示该源代码，会创建新窗口来显示该源代码。如果源代码窗口的缓冲区被修改而不能放弃，这也会发生。

gdb 为每个断点编号。Vim 里在标号列中以红色背景显示该编号。可以使用这些 gdb 命令：

- info break   列出断点
- delete N   删除断点 N

如果光标在断点行上，也可用 :Clear 命令，或右键菜单项目中的 "Clear breakpoint"。

检视变量

termdebug-variables :Evaluate

```
:Evaluate           检视光标下的表达式
K                   同上
:Evaluate {expr}    计算 {expr}
:'<,'>Evaluate      计算可视选择的文本
```

类似于在 gdb 窗口用 "print"。
通常可缩短 :Evaluate 为 :Ev 。

其它命令

termdebug-commands

```
:Gdb        跳转到 gdb 窗口
:Program    跳转到运行程序的窗口
:Source     跳转到源代码窗口，如果还没有则创建之
```

提示模式

termdebug-prompt

如果不支持 +terminal 特性且在 MS-Windows 上，gdb 会在 'buftype' 设为 "prompt" 的缓冲区中运行。工作方式略有不同：

- 键入命令时，gdb 窗口会进入插入模式。<Esc> 会进入普通模式，然后你在缓冲区内可以移动，复制/粘贴，等等。任何开始插入模式的命令，如 a 或 i ，会回去进行 gdb 命令的编辑。
- 待调试的程序在单独的窗口中运行。MS-Windows 上这是个新的控制台窗口。Unix 上，如果 +terminal 特性可用，打开终端窗口以运行待调试的程序。

termdebug_use_prompt

即使 +terminal 特性可用，也可使用提示模式，用：

```
let g:termdebug_use_prompt = 1
```

通信

termdebug-communication

还有一个缓冲区，它是隐藏的，用于 Vim 和 gdb 的通信。缓冲区名是 "gdb communication"。不要删除此缓冲区，否则会破坏调试器。

gdb 有一些奇怪的行为，此插件会尽力而为使其工作。例如，在 gdb 窗口中输入 "continue" 后 CTRL-C 可用于中断运行中的程序。但用 MI 命令 "-exec-continue" 后 CTRL-C 并不中断。因此 :Continue 命令用 "continue" 而不是通信通道。

定制

GDB 命令

termdebug-customizing

要改变 gdb 命令的名字，在调用 :Termdebug 前设置 "termdebugger" 变量：

```
let termdebugger = "mygdb"
```

gdb-version

只能用和 gdb 完全兼容的调试器。Vim 使用 GDB/MI 接口。"new-ui" 命令需要 gdb 7.12 或更新的版本。如见此错误：

```
Undefined command: "new-ui". Try "help".
```

你的 gdb 版本太老了。

颜色

hl-debugPC hl-debugBreakpoint

标号的颜色可用以下高亮组调整：

- debugPC 当前位置
- debugBreakpoint 断点

'background' 为 "light" 的缺省：

```
hi debugPC term=reverse ctermbg=lightblue guibg=lightblue
hi debugBreakpoint term=reverse ctermbg=red guibg=red
```

'background' 为 "dark" 的缺省：

```
hi debugPC term=reverse ctermbg=darkblue guibg=darkblue
hi debugBreakpoint term=reverse ctermbg=red guibg=red
```

快捷键

termdebug_shortcuts

可以定义自己的快捷键（映射）用 TermDebugSendCommand() 函数来控制 gdb，可在任何窗口中使用。例如：

```
map ,w :call TermDebugSendCommand('where')<CR>
```

参数是 gdb 命令。

弹出式菜单

termdebug_popup

缺省 Termdebug 插件设置 'mousemodel' 为 "popup_setpos"，并在弹出式菜单里加入以

下项目：

```
        Set breakpoint          :Break
        Clear breakpoint        :Clear
        Evaluate                :Evaluate
```

如果不想要，可以这样关闭之：

```
        let g:termdebug_popup = 0
```

Vim 窗口宽度 termdebug_wide

要改变调试开始时的 Vim 窗口宽度并用垂直分割：

```
  let g:termdebug_wide = 163
```

这会在使用 :Termdebug 时设置 &columns 为 163。退出调试器后此值复原。
如果设置了 g:termdebug_wide 而 &columns 已经大于 g:termdebug_wide，则使用垂直分割且不改变 &columns。
把它设为 1 会保证垂直分割且不会改变 &columns (用于 Vim 不能改变终端大小的场合)。

vim:tw=78:ts=8:noet:ft=help:norl:

popup.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　by Bram Moolenaar
译者：Willis
https://github.com/yianwillis/vimcdoc

在浮动窗口中显示文本。　popup　popup-window　popupwin

1. 介绍　popup-intro
   窗口位置和大小　popup-position
   关闭弹出窗口　popup-close
   弹出缓冲区和窗口　popup-buffer
2. 函数　popup-functions
   细节　popup-function-details
3. 用法　popup-usage
   popup_create() 参数　popup_create-arguments
   弹出文本属性　popup-props
   用文本属性定位弹出　popup-textprop-pos
   弹出过滤　popup-filter
   弹出回调　popup-callback
   弹出滚动条　popup-scrollbar
   弹出掩码　popup-mask
4. 示例　popup-examples

{仅当编译时加入　popupwin　特性才有效}

---

1. 介绍　popup-intro

我们这里讨论弹出窗口，文本在正常窗口之上显示，可由插件控制。弹出窗口不能像正常窗口一样地编辑文本。

弹出窗口可用于：
- 简短地显示信息，而无须覆盖命令行
- 用对话框提示用户
- 键入时显示上下文信息
- 给出自动补全的额外信息

弹出窗口的文本可用　text-properties　加色。也可以使用语法高亮。

缺省使用的颜色是 "Pmenu"。如果喜欢其它颜色，在 'wincolor' 选项使用 "highlight" 参数，如：

```
hi MyPopupColor ctermbg=lightblue guibg=lightblue
call setwinvar(winid, '&wincolor', 'MyPopupColor')
```

弹出窗口中不显示 'hlsearch' 高亮。

弹出窗口和其它窗口一样，有自己的窗口 ID，但行为不同。它可以和整个 Vim 窗口一样大且和其它窗口重叠。弹出窗口间也可以互相重叠。"zindex" 属性指定谁在谁之上。

弹出窗口包含缓冲区，此缓冲区总是和弹出窗口相捆绑。此窗口不能在普通、可视或插入模式，也得不到键盘焦点。可用 setbufline() 函数来改变缓冲区的文本。此窗口和缓冲区和正常的窗口和缓冲区还有更多的差别，见 popup-buffer 。

如果这不是你想要的，查查其它的弹出功能：
- 弹出菜单，见 popup-menu
- 气泡，见 balloon-eval

窗口位置和大小 popup-position

窗口的高度通常等于缓冲区可能经回绕的行数。可由 "maxheight" 属性限制。可用空行或 "minheight" 属性来增加高度。

缺省置位 'wrap' 选项，不会有文本消失。否则，如果没有足够空间，窗口往左移以显示更多的文本。向右对齐时窗口向右移以显示更多文本。可用 "fixed" 属性关闭左移或右移。

Vim 试图在你指定的位置显示弹出。有些情况下，弹出会超出 Vim 窗口的范围，这时会在附近某处显示。例如，如果用 popup_atcursor() ，弹出通常在当前光标位置之上显示，但如果光标靠近 Vim 窗口的顶部，这里会在光标位置下方放置。

屏幕因为 Ex 命令的输出滚动时，弹出也跟着移动，这样不会遮挡输出。

即使在弹出窗口底下，仍然显示当前光标位置。这样你仍然看到它在哪里，虽然你看不到它所在的文本。

关闭弹出窗口 popup-close

通常创建弹出窗口的插件也负责它的关闭。如果因某些原因弹出依然残留，可以关闭所有的弹出：

```
call popup_clear()
```

有些如通知这样的弹出在指定时间后关闭。时间可用 popup_create() 的 "time" 属性设置。
其它的弹出可由点击右上角的 X 或在弹出内部任何位置点击关闭弹出。此行为必须通过 "close" 属性打开。通知类型缺省打开。

弹出缓冲区和窗口 popup-buffer

如果调用了弹出函数来从文本中建立弹出，建立新的缓冲区来保存文本和弹出窗口的文本属性。此缓冲区总是和弹出窗口捆绑，且有诸多操作上的限制：
- 缓冲区无名
- 'buftype' 为 "popup"
- 'swapfile' 关闭
- 'bufhidden' 为 "hide"
- 'buflisted' 关闭
- 'undolevels' 为 -1: 完全不能撤销
- 所有其它缓冲区局部和窗口局部选项设为其 Vim 缺省值。

可以修改特别列出的选项，但不知道会出什么问题，最好不要动吧。

窗口有它的光标位置，但光标不显示。事实上，显示的是底下窗口的光标，这就像从弹出

中窥视一样，这样你才能看到现在在哪里。

要在弹出窗口和缓冲区的上下文中执行命令，使用 win_execute() 。例如:

```
call win_execute(winid, 'syntax enable')
```

可用 setwinvar() 设置窗口上的选项，如:

```
call setwinvar(winid, '&wrap', 0)
```

可用 setbufvar() 设置缓冲区上的选项，如.:

```
call setbufvar(winbufnr(winid), '&filetype', 'java')
```

也可用 win_execute() 来执行 ":setlocal" 命令。

---

2. 函数 popup-functions

建立弹出窗口:

popup_create()	在屏幕中央
popup_atcursor()	在光标位置正上方，光标移开时关闭
popup_beval()	在 v:beval_ 变量指定的位置，光标移开时关闭
popup_notification()	用三秒钟显示通知
popup_dialog()	带填充和边框的中间对齐
popup_menu()	提示从列表中选择一个项目

操作弹出窗口:

popup_hide()	临时隐藏弹出
popup_show()	显示之前隐藏的弹出
popup_move()	改变弹出的位置和大小
popup_setoptions()	覆盖弹出的选项
popup_settext()	替换弹出缓冲区的内容

关闭弹出窗口:

popup_close()	关闭一个弹出
popup_clear()	关闭所有弹出

过滤函数:

popup_filter_menu()	从一列项目中选择
popup_filter_yesno()	等待直到按了 'y' 或 'n' 为止

其它:

popup_getoptions()	取得弹出的当前选项
popup_getpos()	取得弹出的实际位置和大小
popup_locate()	找到位于指定屏幕位置的弹出窗口

细 节 popup-function-details

popup_atcursor({what}, {options}) popup_atcursor()

在光标上方显示 {what}，光标移开时关闭。类似于:

```
call popup_create({what}, #{
      \ pos: 'botleft',
      \ line: 'cursor-1',
      \ col: 'cursor',
      \ moved: 'WORD',
      \ })
```

{options} 可用来改变属性。
如果 "pos" 为 "topleft"，那么 "line" 的缺省会成为 "cursor+1"。

也可用作 method :
```
GetText()->popup_atcursor({})
```

popup_beval({what}, {options}) popup_beval()
在来自 'balooneval' 的位置上方显示 {what}，光标移开时关闭。类似于:
```
let pos = screenpos(v:beval_winnr, v:beval_lnum, v:beval_col)
call popup_create({what}, #{
      \ pos: 'botleft',
      \ line: pos.row - 1,
      \ col: pos.col,
      \ mousemoved: 'WORD',
      \ })
```
{options} 可用来改变属性。
相关例子可见 popup_beval_example 。

也可用作 method :
```
GetText()->popup_beval({})
```

popup_clear() popup_clear()
处理表现不良插件的紧急方案：关闭当前标签页的所有弹出窗口和全局弹出。

popup_close({id} [, {result}]) popup_close()
关闭弹出 {id}。窗口和相关缓冲区被删除。

如果弹出有回调，会在弹出窗口刚刚删除之前调用。如果可选的 {result} 给出，作为回调的第二个参数被传递。否则传递零给回调。

也可用作 method :
```
GetPopup()->popup_close()
```

popup_create({what}, {options}) popup_create()
打开弹出窗口显示 {what}，后者可以是以下之一:
- 缓冲区号
- 字符串
- 字符串列表
- 带文本属性的文本行的列表

如果 {what} 不是缓冲区号，建立缓冲区并设 'buftype' 为 "popup"。一旦弹出关闭，此缓冲区会被真正删除。

{options} 为有许多可能项目的字典。详见 popup_create-arguments 。

返回窗口 ID，可用于其它的弹出函数。用 winbufnr() 可得到窗口的缓冲区号:
```
let winid = popup_create('hello', {})
let bufnr = winbufnr(winid)
call setbufline(bufnr, 2, 'second line')
```

如果有错，返回零。

也可用作 method :
```
GetText()->popup_create({})
```

popup_dialog({what}, {options}) popup_dialog()

就像 popup_create() 但带以下缺省选项:
```
call popup_create({what}, #{
        \ pos: 'center',
        \ zindex: 200,
        \ drag: 1,
        \ border: [],
        \ padding: [],
        \ mapping: 0,
        \})
```
{options} 可用来改变属性，例如，加入 'filter' 选项，其值为 'popup_filter_yesno'。示例:
```
call popup_create('do you want to quit (Yes/no)?', #{
        \ filter: 'popup_filter_yesno',
        \ callback: 'QuitCallback',
        \ })
```

缺省对话框可以拖动，这样如果需要的话可以读到下面的文本。

也可用作 method :
```
GetText()->popup_dialog({})
```

popup_filter_menu({id}, {key}) popup_filter_menu()

弹出可用的过滤。可用以下键值:

j <Down>	选择下面的项目
k <Up>	选择上面的项目
<Space> <Enter>	接受当前选择
x Esc CTRL-C	取消菜单

忽略其它键。

在选中行上设置匹配以高亮之，见 popup_menu() 。

如果当前选择被接受，调用弹出菜单的 "callback"，并把选中行的索引作为第二个参数传递。首个项目的索引为一。取消菜单后会带参数 -1 调用回调。

要加入快捷键，参见这里的例子: popup_menu-shortcut-example

popup_filter_yesno({id}, {key}) popup_filter_yesno()

弹出可用的过滤。它只处理以下键 'y'、'Y'、'n' 或 'N'。调用弹出菜单的 "callback"，它所带的第二个参数当 'y' 或 'Y' 时为 1，'n' 或 'N' 时为零。按 Esc 和 'x' 类似于按了 'n'。CTRL-C 则带参数 -1 调用回调。忽略其它键。
例子可见: popup_dialog-example

popup_findinfo() popup_findinfo()

寻找弹出菜单使用的弹出 info 窗口的 window-ID ，见

complete-popup 。info 弹出不用时会隐藏，可用 popup_clear() 和 popup_close() 删除。用 popup_show() 来根据弹出菜单中的项目重新定位。
如果找不到返回零。

popup_findpreview() popup_findpreview()
寻找弹出预览窗口的 window-ID ，
如果找不到返回零。

popup_getoptions({id}) popup_getoptions()
返回字典，弹出 {id} 的 {options}。
零值代表选项未设置。对 "zindex" 来说返回缺省值而不是零。

"moved" 项目是带行号、最小列和最大列的列表，没有设置的话则是 [0, 0, 0]。

"mousemoved" 项目是带屏幕行，最小屏幕列，最大屏幕列的列表。没有设置的话则是 [0, 0, 0]。

"firstline" 是弹出上设置的属性，和 popup_getpos() 取得的 "firstline" 不同，后者是实际在弹出窗口顶部的缓冲区行。

"border" 和 "padding" 的值全为零时不包含在返回值内。值全是一时包含一个空列表。

"borderhighlight" 所有值为空时不包含在返回值内。
"scrollbarhighlight" 和 "thumbhighlight" 仅在设置时被包含。

"tabpage" 对全局弹出会是 -1，当前标签页上的弹出会是零，其它标签页上的弹出会是个正数。

"textprop"、"textpropid" 和 "textpropwin" 仅在 "textprop" 设置时才存在。

如果找不到弹出窗口 {id}，返回空字典。

也可用作 method :
```
GetPopup()->popup_getoptions()
```

popup_getpos({id}) popup_getpos()
返回弹出 {id} 的位置和大小。返回以下项目的字典:

col	弹出的屏幕列，从一开始
line	弹出的屏幕行，从一开始
width	整个弹出以屏幕单元格计的宽度
height	整个弹出以屏幕单元格计的高度
core_col	文本框的屏幕列
core_line	文本框的屏幕行
core_width	文本框以屏幕单元格计的宽度
core_height	文本框以屏幕单元格计的高度
firstline	缓冲区的顶部行 (1 除非滚动过) (不是 "firstline" 属性的值)
lastline	缓冲区的底部行 (弹出重画时会刷新)

scrollbar　如果显示滚动条则非零
visible　如果弹出可见则为一，隐藏则为零

注意 这些是实际的屏幕位置。由于采用的大小和定位的机制，和 popup_getoptions() 的值不同。

"core_" 系列的值不计算填充和边框。

如果找不到弹出窗口 {id}，返回空字典。

也可用作 method :
```
GetPopup()->popup_getpos()
```

popup_hide({id}) popup_hide()
如果 {id} 是显示中的弹出，现在隐藏之。如果弹出有过滤，在弹出隐藏期间不调用过滤。
如果窗口 {id} 不存在，什么都不发生。如果窗口 {id} 存在但不是弹出窗口，报错。 E993

也可用作 method :
```
GetPopup()->popup_hide()
```

popup_locate({row}, {col}) popup_locate()
返回在屏幕位置 {row} 和 {col} 上的弹出 window-ID 。如果有多个弹出，返回最高 zindex 的弹出。如果在该位置上没有弹出，返回零。

popup_menu({what}, {options}) popup_menu()
在光标附近显示 {what}，用光标键处理项目的选择，用 Space 或 Enter 选中项目后关闭窗口。{what} 应有多行才能实际有用。
类似于:
```
call popup_create({what}, #{
        \ pos: 'center',
        \ zindex: 200,
        \ drag: 1,
        \ wrap: 0,
        \ border: [],
        \ cursorline: 1,
        \ padding: [0,1,0,1],
        \ filter: 'popup_filter_menu',
        \ mapping: 0,
        \ })
```
当前行使用 "PopupSelected" 的匹配高亮，如果无定义则使用 "PmenuSel"。

{options} 可用来改变属性。至少要设置 "callback" 为处理选中项目的函数。示例:
```
func ColorSelected(id, result)
   " use a:result
endfunc
call popup_menu(['red', 'green', 'blue'], #{
        \ callback: 'ColorSelected',
        \ })
```

也可用作 method :

```
GetChoices()->popup_menu({})
```

popup_move({id}, {options}) popup_move()

移动弹出 {id} 到 {options} 指定的位置。
{options} 可以包含 popup_create() 里指定弹出位置的选项:

```
line
col
pos
maxheight
minheight
maxwidth
minwidth
fixed
```

{id} 可见 popup_hide() 。
其它选项见 popup_setoptions() 。

也可用作 method :

```
GetPopup()->popup_move(options)
```

popup_notification({what}, {options}) popup_notification()

用三秒钟在 Vim 窗口顶部显示 {what}。
类似于:

```
call popup_create({what}, #{
        \ line: 1,
        \ col: 10,
        \ minwidth: 20,
        \ time: 3000,
        \ tabpage: -1,
        \ zindex: 300,
        \ drag: 1,
        \ highlight: 'WarningMsg',
        \ border: [],
        \ close: 'click',
        \ padding: [0,1,0,1],
        \ })
```

如果有定义，使用 PopupNotification 高亮组而不是 WarningMsg。

如果没有 +timers 特性，弹出不会自动消失。用户要点击它才行。

位置会经过调整以避免和其它通知重叠。
{options} 可用来改变属性。

也可用作 method :

```
GetText()->popup_notification({})
```

popup_show({id}) popup_show()

如果 {id} 是隐藏的弹出，现在显示之。
{id} 可见 popup_hide() 。
如果 {id} 是 info 弹出，会定位在当前弹出菜单项边上。

popup_setoptions({id}, {options}) popup_setoptions()

用 {options} 里的项目来覆盖弹出 {id} 的选项。
可以设置以下选项:

border
borderchars
borderhighlight
callback
close
cursorline
drag
filter
firstline
flip
highlight
mapping
mask
moved
padding
resize
scrollbar
scrollbarhighlight
thumbhighlight
time
title
wrap
zindex

也可用 popup_move() 里的选项。
对 "hidden"，用 popup_hide() 和 popup_show() 。
不能改变 "tabpage"。

也可用作 method :
```
GetPopup()->popup_setoptions(options)
```

popup_settext({id}, {text}) popup_settext()
设置弹出窗口 {id} 缓冲区的文本。{text} 和 popup_create() 提供的相同，除了不接受缓冲区号以外。
除了文本的改变引起的以外，不改变窗口大小或位置。

也可用作 method :
```
GetPopup()->popup_settext('hello')
```

---

3. 用法 popup-usage

POPUP_CREATE() 参 数 popup_create-arguments

popup_create() 的首个参数（以及 popup_settext() 的第二个参数）指定要显示的文本还有可选的文本属性。可以是以下四种形式之一：
- 缓冲区号
- 字符串
- 字符串列表
- 字典列表，其中每个字典有以下项目：

text 要显示的文本字符串
props 文本属性的列表。可选。
每项是一个字典，类似于 prop_add() 第三个参数，但字典中用 "col" 项指定列，见下： popup-props 。

如果你要自己创建新的缓冲区，用 bufadd() 并传递缓冲区号给 popup_create()。
不用使用终端窗口的缓冲区。 E278

popup_create() 的第二个参数是以下选项的字典：

line	定位弹出的屏幕行。可用数值、"cursor"、"cursor+1" 或 "cursor-1" 来使用光标所在行并加或减若干行。如果省略，弹出垂直中间对齐。首行为 1。 使用 "textprop" 时此值相对于文本属性，可为负。
col	定位弹出的屏幕列。可用数值、"cursor" 来指定光标列、"cursor+9" 或 "cursor-9" 来加或减若干列。如果省略，弹出水平中间对齐。首列为 1。 使用 "textprop" 时此值相对于文本属性，可为负。
pos	"topleft"、"topright"、"botleft" 或 "botright"：定义 "line" 和 "col" 用于指定弹出的哪个角。不指定时使用 "topleft"。 此外 "center" 可用来定位弹出在 Vim 窗口的中央，此时忽略 "line" 和 "col"。
posinvert	为 FALSE 时总是使用 "pos" 的值。为 TRUE (缺省) 时如果弹出在垂直方向放不进而在另一边有更多的空间，会把弹出放置在 "line" 指示的位置的另一边。
textprop	给出时定位弹出紧贴着使用此名的文本属性，文本属性移动时会跟着移动。空串会删除之。见 popup-textprop-pos 。
textpropwin	文本属性搜索的窗口。省略或值非法时使用当前窗口。
textpropid	给出 "textprop" 时用于识别文本属性。零会复位。
fixed	为 FALSE (缺省) 时，且： - "pos" 为 "botleft" 或 "topleft"，且 - "wrap" 关闭，且 - 弹出会在屏幕的右侧边缘被截断，那么 弹出会移动到左侧使得内容可放得进屏幕。设为 TRUE 关闭此行为。
flip	为 TRUE (缺省) 时，位置相对于光标，翻转到光标之下或之上以避免和 popupmenu-completion 或其它更多 "zindex" 的弹出重叠。如果在光标上/下没有空间就在弹出或弹出菜单边上显示弹出。 {尚未实现}
maxheight	内容的最大高度，边框和填充不计在内。
minheight	内容的最小高度，边框和填充不计在内。
maxwidth	内容的最大宽度，边框、填充和滚动条不计在内。
minwidth	内容的最小宽度，边框、填充和滚动条不计在内。
firstline	显示的首个缓冲区行。大于一时，看起来文本向上滚动了。如果超出范围，缓冲区末行出现在窗口顶部。设为零则保留命令设置的位置。另见 "scrollbar"。
hidden	为 TRUE 时弹出存在但不显示；用 popup_show() 来撤销隐藏。
tabpage	为 -1 时：显示所有标签页上的弹出。 为 0 (缺省) 时：显示当前标签页上的弹出。 否则，显示弹出的标签页号；非法时不创建弹出且报错。 E997
title	弹出的首个项目上方显示的文本，在边框之上。如果没有顶部边框，加上一行填充行来放置标题。你可能希望在开始和结束处加上一或更多的空格作为填充。
wrap	为 TRUE 时使行回绕 (缺省为 TRUE)。
drag	为 TRUE 时弹出可用鼠标在边框上拖曳。弹出无边框时无效。一旦开始拖曳，"pos" 如果为 "center" 就会被改为

"topleft"。

resize　为 TRUE 时弹出可用鼠标抓住右下角来调整大小。弹出无边框时无效。

close　为 "button" 时，在右上角显示 X，出现在任何边框、填充或文本之上。点击 X 时关闭弹出。调用回调，带参数 -2。
为 "click" 时，在弹出上任何鼠标点击会关闭之。
为 "none"（缺省）时，鼠标点击不关闭弹出窗口。

highlight　文本使用的高亮组名，在 'wincolor' 选项里保存。

padding　数值列表，定义弹出上/右/下/左的填充（和 CSS 类似）。空列表使用四周填充均为 1。填充包围文本，在任何边框之内。
填充使用 'wincolor' 高亮。
示例：[1, 2, 1, 3] 在上方有 1 行填充，右侧 2 列，下方 1 行和左侧 3 列。

border　数值列表，定义弹出上/右/下/左的边框宽度（和 CSS 类似）。仅识别零值和非零值。空列表代表四周均有边框。

borderhighlight　边框使用的高亮组名的列表。如果有一个项目用于所有边框，否则分别是上/右/下/左边的高亮。
示例：['TopColor', 'RightColor', 'BottomColor, 'LeftColor']

borderchars　字符列表，定义用于上/右/下/左边使用的字符。可选地可后跟用于左上/右上/右下/左下角的字符。
示例：['-', '|', '-', '|', '┌', '┐', '┘', '└']
如果列表有一个字符，用于所有边框。
如果列表有两个字符，第一个用于边框线，第二个用于角落。
缺省 'encoding' 为 "utf-8" 且 'ambiwidth' 为 "single" 时使用双线，否则使用 ASCII 字符。

scrollbar　为非零时：如果文本放不下，显示滚动条。
为零时：不显示滚动条。缺省为非零。
另见 popup-scrollbar 。

scrollbarhighlight　滚动条使用的高亮组名。重要的是背景色。如果不给出使用 PmenuSbar。

thumbhighlight　滚动条拇指使用的高亮组名。重要的是背景色。如果不给出使用 PmenuThumb。

zindex　弹出的优先级，缺省为 50。最小值是 1，最大值是 32000。

mask　坐标列表的列表，定义弹出的透明部分。见 popup-mask 。

time　以毫秒计的时间，在此之后关闭弹出。
省略时必须使用 popup_close() 。

moved　指定光标如何移开时关闭弹出：
- "any"：如果光标有任何移动
- "word"：如果光标移开了 <cword>
- "WORD"：如果光标移开了 <cWORD>
- "expr"：如果光标移开了 <cexpr>
- [{start}, {end}]：如果光标移动到 {start} 列之前或 {end} 列之后
- [{lnum}, {start}, {end}]：如果光标离开了 {lnum} 行，移动到 {start} 列之前或 {end} 列之后
- [0, 0, 0] 光标移动时不关闭弹出

光标移动到另一行或另一窗口时也关闭弹出。

mousemoved　类似于 "moved"，但指的是鼠标指针位置

cursorline　为非零时：高亮光标行。也滚动文本以显示此行（仅当 'wrap' 关闭时才能正确工作）。
为零时：　不高亮光标行。
缺省为零，但 popup_menu() 除外。

filter　过滤输入字符的回调，见 popup-filter 。

mapping	支持键映射。为 FALSE、弹出可见且有 filter 回调时关闭键映射。缺省值为 TRUE。
filtermode	过滤应用的模式（标志位同 hasmapto() ，加上 "a"）： n 普通模式 v 可视和选择模式 x 可视模式 s 选择模式 o 操作符等待模式 i 插入模式 l Language-Argument ("r"、"f"、"t" 等等) c 命令行模式 a 所有模式 缺省模式是 "a"。
callback	弹出关闭时调用的回调，例如在使用 popup_filter_menu() 时，见 popup-callback 。

取决于 "zindex"，弹出可以在其它弹出之上或之下显示。补全菜单（ popup-menu ）的 zindex 是 100。短暂出现的消息建议用 zindex 值 1000。

缺省文本回绕，它会使得 {lines} 的一行占据多于一个屏幕行。"wrap" 为 FALSE 时弹出之外或 Vim 窗口之外的文本不会显示，也就是被截短了。

弹出文本属性 popup-props

类似于 prop_add() 的第三个参数，除了：
- "lnum" 总是列表的当前行
- "bufnr" 总是弹出的缓冲区
- "col" 在字典只字典而不是单独的参数

这样我们有：

col	以字节计的起始列，首列为一。
length	以字节计的文本长度；可为零
end_lnum	文本结尾的行号
end_col	文本刚刚结束之后的列；如果给出 "length" 则不用；如果 {col} 和 "end_col" 相等，为零宽度的文本属性
id	用户定义的属性 ID；如果省略则用零
type	文本属性类型名，由 prop_type_add() 加入

用文本属性定位弹出 popup-textprop-pos

紧贴着文本属性定位弹出会在插入或删除文本时移动弹出。弹出起工具提示的作用。

需要三步才能完成此功能：

- 定义文本属性类型，定义名字。

```
call prop_type_add('popupMarker', {})
```

- 在想要的文本上放置文本属性：

```
let lnum = {文本行}
let col = {文本起始列}
let len = {文本长度}
let propId = {任意但唯一的数值}
call prop_add(lnum, col, #{
```

```
\ length: len,
\ type: 'popupMarker',
\ id: propId,
\ })
```

- 创建弹出:

```
let winid = popup_create('the text', #{
\ pos: 'botleft',
\ textprop: 'popupMarker',
\ textpropid: propId,
\ border: [],
\ padding: [0,1,0,1],
\ close: 'click',
\ })
```

缺省弹出定位在文本的角落上，和为弹出指定的 "pos" 相反。因而如果弹出使用 "botleft"，弹出的左下角定位在文本属性的右上角:

```
                    +----------+
                    | the text |
                    +----------+
just some PROPERTY as an example
```

这里文本属性在 "PROPERTY" 上。给 popup_create() 传递负的 "col" 值可以向左平移弹出。用 "col: -5" 你会得到:

```
               +----------+
               | the text |
               +----------+
just some PROPERTY as an example
```

如果文本属性移出了视窗，弹出会被隐藏。如果定义弹出的窗口被关闭，弹出也会被关闭。

如果弹出在指定的位置放不下，会在附近的位置上显示。

一些提示:
- 要避免和其它插件的冲突，文本属性类型名必须唯一。可用 "bufnr" 项目使之局部于缓冲区。
- 如果只有一个文本属性可见，可以略过文本属性 ID。
- 弹出可能挡住用户要做的事，如上例所示，让点击可以关闭它，会有帮助。
- 如果删除了文本属性，弹出会关闭。可以这么用:

```
call prop_remove(#{type: 'popupMarker', id: propId})
```

弹 出 过 滤 popup-filter

弹出显示时接收任何输入键的回调。弹出隐藏时过滤回调不被调用。

过滤可以返回 TRUE，指示键已处理完所以可以放弃，或者 FALSE 来让 Vim 按照当前状态的通常做法处理键。如果返回 FALSE 而且有另一个弹出窗口可见，也调用那个的过滤。先调用带最高 zindex 的弹出窗口的过滤。

过滤函数调用时有两个参数: 弹出的 ID 的字符串形式的键，例如:

```
func MyFilter(winid, key)
```

```
    if a:key == "\<F2>"
      " 做点什么
      return 1
    endif
    if a:key == 'x'
      call popup_close(a:winid)
      return 1
    endif
    return 0
  endfunc
```

popup-filter-mode

"filtermode" 属性可用来指定调用过滤的模式。缺省是 "a": 所有模式。如果使用 "nvi"，不包含命令行模式，这样任何命令行上的命令不接受过滤。不如，要能到命令行模式，过滤不能消耗掉 ":"。就像不能消耗 "v" 才能进入可视模式一样。

popup-mapping

键通常是映射后的结果，因为如果过滤不用，键作为正常输入传递。如果过滤消耗了键，设置 "mapping" 属性为零可以保证映射不会产生问题。这是 popup_menu() 和 popup_dialog() 的缺省。

一些建议的键动作：

x	关闭弹出（见下面 备注）
cursor keys	选择另一项
Tab	接受当前建议值

鼠标点击产生 <LeftMouse>。坐标可由 getmousepos() 得到。

Vim 提供标准过滤 popup_filter_menu() 和 popup_filter_yesno() 。

备注 "x" 是关闭弹出的正常方法。你可能想用 Esc，但许多键以 Esc 字符开始，Vim 识别 Esc 键时可能有延迟。如果确实要用 Esc，建议设置 'ttimeoutlen' 选项为 100 并置位 'timeout' 和/或 'ttimeout'。

弹出回调

popup-callback

弹出关闭时调用的回调。

调用回调时有两个参数：弹出窗口 ID 和结果，后者可以是弹出行的索引，或者是任意传递给 popup_close() 的第二个参数。

如果弹出被强制关闭，如光标移动或按了 CTRL-C，传递数值 -1 给回调。

示例：

```
func SelectedColor(id, result)
   echo 'choice made: ' .. a:result
endfunc
```

弹出滚动条

popup-scrollbar

如果弹出中的文本装不下，窗口右侧显示滚动条。可通过设置 "scrollbar" 选项为零来关闭之。如果显示滚动条，当鼠标位于弹出之上时，鼠标滚动事件像你预期地使文本往上或下滚动。点击滚动条上半部会下滚文本一行。点击滚动条下半部会上滚文本一行。不

过，滚动有限制，以确保弹出不会变小。

弹出掩码 popup-mask

要使弹出遮掩的文本最少，弹出的部分可以变透明。此行为由 "mask" 定义，这是列表的列表，其中每个列表有四个数值：

col	起始列，从左开始计数为正，1 是最左边，从右开始计数为负，-1 是最右边
endcol	结束列，类似于 "col"
line	起始行，从顶开始计数为正，1 是最顶行，从底开始计数为负，-1 是最底行
endline	结束行，类似于 "line"

例如，要使最后一行的最后 10 列透明：

```
[[-10, -1, -1, -1]]
```

要使四个角透明：

```
[[1, 1, 1, 1], [-1, -1, 1, 1], [1, 1, -1, -1], [-1, -1, -1, -1]]
```

---

4. 示例 popup-examples

TODO: 更多有意思的例子

popup_dialog-example

提示用户按 y/Y 或 n/N：

```
func MyDialogHandler(id, result)
   if a:result
      " ... 按了 'y' 或 'Y'
   endif
endfunc

call popup_dialog('Continue? y/n', #{
        \ filter: 'popup_filter_yesno',
        \ callback: 'MyDialogHandler',
        \ })
```

popup_menu-shortcut-example

用快捷键扩展 popup_filter_menu()：

```
call popup_menu(['Save', 'Cancel', 'Discard'], #{
        \ filter: 'MyMenuFilter',
        \ callback: 'MyMenuHandler',
        \ })

func MyMenuFilter(id, key)
  " 处理快捷键
  if a:key == 'S'
     call popup_close(a:id, 1)
     return 1
  endif
  if a:key == 'C'
     call popup_close(a:id, 2)
```

```
    return 1
  endif
  if a:key == 'D'
    call popup_close(a:id, 3)
    return 1
  endif

  " 没有快捷键，传递给通用过滤
  return popup_filter_menu(a:id, a:key)
endfunc
```

popup_beval_example

使用弹出窗口用作 'balooneval' 的例子：

```
set balooneval balloonevalterm
set balloonexpr=BalloonExpr()
let s:winid = 0
let s:last_text = ''

func BalloonExpr()
  if s:winid && popup_getpos(s:winid) != {}
    " 上次的弹出窗口还在显示
    if v:beval_text == s:last_text
      " 还是相同的文本，保留原有的弹出
      return ''
    endif
    call popup_close(s:winid)
  endif
  let s:winid = popup_beval(v:beval_text, #{mousemoved: 'word'})
  let s:last_text = v:beval_text
  return ''
endfunc
```

如果要异步获取文本，表达式函数返回空串，而一旦文本可用，调用 popup_beval()。下例用定时器调用来模拟：

```
set balooneval balloonevalterm
set balloonexpr=BalloonExpr()
let s:winid = 0
let s:balloonText = ''

func BalloonExpr()
  if s:winid && popup_getpos(s:winid) != {}
    " 上次的弹出窗口还在显示
    if  v:beval_text == s:balloonText
      " 还是相同的文本，保留原有的弹出
      return ''
    endif
    call popup_close(s:winid)
    let s:winid = 0
  endif
  " 模拟异步查询要显示的文本
  let s:balloonText = v:beval_text
  call timer_start(100, 'ShowPopup')
```

```
  return ''
endfunc

func ShowPopup(id)
  let s:winid = popup_beval(s:balloonText, #{mousemoved: 'word'})
endfunc
```

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者：wandys http://vimcdoc.sf.net

本文讨论 C 程序以及其它文件类型的缩进。

1. 缩进 C 风格的程序 C-indenting
2. 按表达式缩进 indent-expression

---

1. 缩进 C 风格的程序 C-indenting

有关 C 风格的缩进的基本操作请参用户手册的 30.2 。

Vim 有很多自动缩进 C 风格程序的选项。不少编程语言，如 Java 和 C++，都和 C 的排版习惯非常相近。这些选项只对缩进有影响而不处理其他形式的排版。另有一些选项影响其它类型的排版和缩进，大体可参阅 format-comments 、 fo-table 、 gq 和 formatting 。

备注 如果编译时没有指定 +smartindent 或者 +cindent 特性，这些选项将不起作用。

实际上主要有四种可用缩进的方式，如果同时指定或对 'indentexpr' 而言非空，列于后者优先于列于前者：

'autoindent' 沿用上一行的缩进。
'smartindent' 类似 'autoindent'，但是可以识别一些 C 语法以能在合适的地方增加 / 减少缩进。
'cindent' 比上面两个更聪明；可以设置不同的缩进风格。
'indentexpr' 最灵活的一个：根据表达式来计算缩进。若此选项非空，则优先于其它选项覆盖。参见 indent-expression 。

本节下面的内容介绍 'cindent' 选项。

注意 'cindent' 缩进不一定对所有的代码都起作用。Vim 不是 C 编译器：它不能识别出所有的语法。一个要求是顶层函数必须在第一列中含有 '{'。否则它们容易与声明混淆。

这四个选项控制 C 程序缩进：

'cindent' 使 Vim 对 C 程序自动缩进。
'cinkeys' 指定在插入模式下按哪个键可以再次缩进。
'cinoptions' 设定你喜好的缩进模式。
'cinwords' 定义在下一行中开始一个额外缩进的关键字。

如果 'lisp' 选项没被打开并且 'equalprg' 为空，"=" 操作符将使用 Vim 的内建算法而不调用外部程序。

若要对 C 文件自动设定 'cindent' 选项而在其它文件里复位，请参考 autocommand 。

cinkeys-format indentkeys-format

'cinkeys' 选项是这样的一个字符串：控制 Vim 在输入特定字符或是在特定上下文下的命令后引起的缩进。备注 这不仅仅触发 C-缩进。 若 'indentexpr' 非空，将会使用 'indentkeys' 而不是 'cinkeys'。'cinkeys' 和 'indentkeys' 的格式是一样的。

其缺省值是 "0{,0},0),0],:,0#,!^F,o,O,e"。解释如下:

```
"0{"    如果在该行输入的第一个字符是 '{'
"0}"    如果在该行输入的第一个字符是 '}'
"0)"    如果在该行输入的第一个字符是 ')'
"0]"    如果在该行输入的第一个字符是 ']'
":"     如果在标号或是 case 语句后输入 ':'
"0#"    如果在该行输入的第一个字符是 '#'
"!^F"   如果输入 CTRL-F (它不会被插入)
"o"     如果输入 <CR> 或是使用 "o" 命令
"O"     如果使用 "O" 命令
"e"     如果输入处于行首的 "else" 的第二个 'e'
```

可以加在键值前面的字符: i_CTRL-F

`!` 当 '!' 在一个键值前面, Vim 将不会插入那个键而是将当前行再次缩进。这可以用来定义一个再次缩进当前行的键。CTRL-F 是缺省的键。将 CTRL-I 设定为这个键时要小心, 因为 CTRL-I 用来表示 <Tab> 的 ASCII 码。

`*` 当 '*' 在一个键值前面, 会在插入此键之前再次缩进当前行。如果 'cinkeys' 包含 "*<Return>", Vim 在开启一个新行之前再次缩进当前行。

`0` 当 '0' 在一个键值前面 (但可以在 '!' 或 '*' 后面), 只有当此键是该行你输入的第一个字符时, Vim 才再次缩进当前行。如果在 "=" 之前使用, 只有当此单词前面全是空白字符时 Vim 才再次缩进当前行。

如果 '!' 和 '*' 都不在键值前, Vim 将在你输入那个键后再次缩进当前行。 因此 ';' 触发一个包含 ';' 本身在内的行的缩进。

特殊键:

`<>` 尖括号表示一个键名本身。例如: "<Up>", "<Ins>" (参见 key-notation)。

`^` 以 '^' 开头的字符串表示一个控制字符。比如, "^F" 是 CTRL-F。

`o` 当使用 "o" 命令或是在当前行后开启一个新行时 (例如, 在插入模式输入 <Enter> 时) 再次缩进当前行。

`O` 当使用 "O" 命令时再次缩进当前行。

`e` 当输入第二个 'e' 时, 再次缩进由 "else" 开头的行。

`:` 在一个标号或是 case 语句后输入 ':' 时, 再次缩进当前行; 不对 C++ 中的 "class::method" 这种形式缩进。若要对所有的 ":" 都缩进, 使用 "<:>"。

`=word` 当输入 "word" 的最后一个字母时再次缩进。"word" 可以是一个单词的一部分。比如当输入 "endif" 或是 "endwhile" 中的 "d" 时, "=end" 都能引起再次缩进。但当输入 "bend" 时就没有反应。另外, 当补全功能产生了一个以 "word" 开头的单词时也会再次缩进。"0=word" 是当单词前有空白符时才缩进。

`=~word` 类似 "=word", 但忽略大小写。

如果你想在输入 'o'、'O'、'e'、'0'、'<'、'>'、'*'、':'、 '!' 时产生再次缩进, 请分别使用 "<o>"、"<O>"、"<e>"、"<0>"、"<<>"、"<>>"、"<*>"、"<:>"、"<!>"。

emacs 风格的缩进模式并不是每次输入 <Enter> 都缩进, 而是只在输入 <Tab> 时才缩进。对此, 我建议使用:

```
:set cinkeys=0{,0},:,0#,!<Tab>,!^F
```

你也许还需要关掉 'autoindent' 选项。

备注: 如果你手动地改变了当前行的缩进, Vim 会忽略对此行的 cindent (译者注: 也包含 'indentexpr') 设定。因此如果你输入 <BS>、<Tab>、<Space>、CTRL-T、或是 CTRL-D 来改变缩进, Vim 将不会再次缩进。

cinoptions-values

'cinoptions' 选项决定 Vim 来如何进行缩进。选项之后的值采用以下形式之一 (N 为任何数字):

N	缩进 N 个空格
-N	向左缩进 N 个空格
Ns	N 倍 'shiftwidth' 个空格
-Ns	向左 N 倍 'shiftwidth' 个空格

在下面的列表中, "N" 表示一个你选择的数字 (可以为负数)。如果数字后有一个 's', Vim 将那个数字乘以 'shiftwidth': "1s" 是 'shiftwidth', "2s" 是 'shiftwidth' 的两倍, 以此类推。你也可以使用小数点: "-0.5s" 是 'shiftwidth' 一半的负值。
下面的例子假定 'shiftwidth' 为 4。

cino->

>N "一般" 缩进的增加值。在需要增加缩进的行 (比如, 以 "if" 或是 "{" 等开头的行) 之后使用。(缺省为 'shiftwidth')。

```
cino=                cino=>2              cino=>2s
  if (cond)            if (cond)            if (cond)
  {                    {                    {
      foo;                 foo;                     foo;
  }                    }                    }
```

cino-e

eN 当 '{' 在行尾 (End, 更准确地, 不在行首) 时, 相对 '一般' 缩进增加 N 个字符的缩进。如果你想对 '{' 在行首时和 '{' 在行尾时使用不同的缩进, 这将很有用。(缺省为 0)。

```
cino=                cino=e2              cino=e-2
  if (cond) {          if (cond) {          if (cond) {
      foo;                 foo;                 foo;
  }                    }                    }
  else                 else                 else
  {                    {                    {
      bar;                 bar;                 bar;
  }                    }                    }
```

cino-n

nN 如果一个在 "if", "while" 等等后面的语句不在 (Not) 大括号内, 则相对 '一般' 缩进增加 N 个字符的缩进。如果你要对语句前有 '{' 和没有 '{' 使用不同的缩进, 这将很有用。(缺省为 0)。

```
cino=                cino=n2              cino=n-2
  if (cond)            if (cond)            if (cond)
      foo;                 foo;                 foo;
  else                 else                 else
  {                    {                    {
      bar;                 bar;                 bar;
  }                    }                    }
```

cino-f

fN 将函数或是其他代码块开头的 '{' 放在第 N 列。这只对不包含于其他大括号内并且处于行首的 '{' 起作用。'{' 之后的代码对它的相对位置不变。 (缺省为 0)。

```
cino=                cino=f.5s            cino=f1s
  func()               func()               func()
  {                    {                    {
      int foo;             int foo;             int foo;
```

cino-{

{N　将 '{' 置于 '一般' 缩进后 N 个字符的位置。这只对包括在其他大括号内的 '{' 起作用。(缺省为 0)。

```
cino=                cino={.5s            cino={1s
  if (cond)            if (cond)            if (cond)
  {                    {                    {
      foo;                 foo;                 foo;
```

cino-}

}N　将 '}' 放在与之匹配的 '{' 之后 N 个字符处。(缺省为 0)。

```
cino=                cino={2,}-0.5s       cino=}2
  if (cond)            if (cond)            if (cond)
  {                    {                    {
      foo;                 foo;                 foo;
  }                    }                    }
```

cino-^

^N　如果 '{' 在第 0 列，则对其包含的语句增加相对 '一般' 进 N 个字符的缩进。这可以对整个函数设定一个不同的缩进。(有人喜欢将其设成负数)。(缺省为 0)。

```
cino=                cino=^-2             cino=^-s
  func()               func()               func()
  {                    {                    {
      if (cond)            if (cond)            if (cond)
      {                    {                    {
          a = b;               a = b;               a = b;
      }                    }                    }
  }                    }                    }
```

cino-L

LN　控制跳转标签的放置。如果 N 为负，标签放在第一列上。如果 N 非负，标签的缩进位置为 '一般' 缩进位置减去 N。(缺省为 -1)。

```
cino=                cino=L2              cino=Ls
  func()               func()               func()
  {                    {                    {
      {                    {                    {
          stmt;                stmt;                stmt;
  LABEL:               LABEL:               LABEL:
      }                    }                    }
  }                    }                    }
```

cino-:

:N　将 case 标号放在 switch() 缩进位置之后的 N 个字符处。(缺省为 'shiftwidth')。

```
cino=                cino=:0
  switch (x)           switch(x)
  {                    {
      case 1:          case 1:
          a = b;           a = b;
      default:         default:
  }                    }
```

cino-=

=N　将 case 标号之后的语句放在标号缩进位置之后的 N 个字符处。(缺省为 'shiftwidth')。

```
cino=                cino==10
   case 11:              case 11:  a = a + 1;
       a = a + 1;                  b = b + 1;
```

cino-l

lN　如果 N 不为 0，Vim 将会和 case 标号对齐，而不是和同一行其后的语句对齐。

```
cino=                        cino=l1
    switch (a) {                 switch (a) {
        case 1: {                    case 1: {
                    break;                       break;
                }                            }
```

cino-b

bN　如果 N 不为 0，Vim 会将最后的 "break" 和 case 标号对齐，这样 case..break 看起来有点像一个代码块。(缺省为 0)。
使用 1 时，考虑在 'cinkeys' 中加入 "0=break"。

```
cino=                cino=b1
  switch (x)           switch(x)
  {                    {
      case 1:              case 1:
          a = b;               a = b;
          break;           break;

      default:             default:
          a = 0;               a = 0;
          break;           break;
  }                    }
```

cino-g

gN　将 C++ 作用域声明置于其所在代码块的 N 个字符后。(缺省为 'shiftwidth')。作用域声明可以是 "public:"， "protected:" 或者 "private:"。

```
cino=                cino=g0
  {                    {
      public:              public:
          a = b;               a = b;
      private:             private:
```

```
    }                   }
```

cino-h

hN　将 C++ 作用域声明后面的语句置于对应标号的 N 个字符后。(缺省为 'shiftwidth')。

```
cino=               cino=h10
   public:              public:   a = a + 1;
       a = a + 1;                b = b + 1;
```

cino-N

NN　C++ namespace 之内的相对普通块的额外缩进 N 个字符。(缺省为 0)。

```
cino=                       cino=N-s
  namespace {                 namespace {
      void function();        void function();
  }                           }

  namespace my                namespace my
  {                           {
      void function();        void function();
  }                           }
```

cino-E

EN　C++ 链接性声明 (extern "C" 或 externa "C++") 之内的相对普通块的额外缩进 N 个字符。(缺省为 0)。

```
cino=                       cino=E-s
  extern "C" {                extern "C" {
      void function();        void function();
  }                           }

  extern "C"                  extern "C"
  {                           {
      void function();        void function();
  }                           }
```

cino-p

pN　在 K&R 形式的函数声明中，将参数的声明放在 N 个字符的相对位置。(缺省为 'shiftwidth')。

```
cino=               cino=p0             cino=p2s
  func(a, b)          func(a, b)          func(a, b)
      int a;          int a;                    int a;
      char b;         char b;                   char b;
```

cino-t

tN　将函数返回值类型声明放在 N 个字符的相对位置。(缺省为 'shiftwidth')。

```
cino=               cino=t0             cino=t7
      int             int                       int
  func()              func()              func()
```

cino-i

iN　缩进 C++ 的基类声明和构造函数初始化部分，如果它们处于行首（否则，它们对齐在 ':' 的右侧）。(缺省为 'shiftwidth')。

```
cino=                          cino=i0
  class MyClass :                class MyClass :
      public BaseClass           public BaseClass
  {}                             {}
  MyClass::MyClass() :           MyClass::MyClass() :
      BaseClass(3)               BaseClass(3)
  {}                             {}
```

cino-+

+N　函数之内的续行（在下一行继续的行）缩进增加额外的 N 个字符。(缺省为 'shiftwidth')。
函数之外，如果前行以反斜杠结尾，使用 2 * N。

```
cino=                          cino=+10
  a = b + 9 *                    a = b + 9 *
      c;                                 c;
```

cino-c

cN　如果没有其它文本可供对齐的话，注释头部（译者注: /*）之后的注释行相对注释头部 N 个字符的缩进。(缺省为 3)。
参见 format-comments 。

```
cino=                          cino=c5
  /*                             /*
     text.                            text.
   */                             */
```

cino-C

CN　如果 N 非零，像上面 c 选项一样对注释缩进，即使 /* 后面还有其他文字也是如此。

```
cino=c0                        cino=c0,C1
  /********                      /********
    text.                         text.
  ********/                       ********/
```

(示例使用了 ":set comments& comments-=s1:/* comments^=s0:/*")

cino-/

/N　将注释行增加额外的缩进 N 个字符。(缺省为 0)。

```
cino=                          cino=/4
  a = b;                         a = b;
  /* comment */                      /* comment */
  c = d;                         c = d;
```

cino-(

(N　在没有结束的括号内，相对没有结束的括号所在行进行 N 个字符的缩进。对每一个额外没有结束的括号增加 'shiftwidth' 的缩进。如果 N 为 0，或没有结束的括号是其所在行的首个非空白字符，和括号之后下一个非空白字符对齐。(缺省为 'shiftwidth' * 2)。

```
cino=                          cino=(0
  if (c1 && (c2 ||               if (c1 && (c2 ||
              c3))                           c3))
      foo;                           foo;
  if (c1 &&                      if (c1 &&
         (c2 || c3))                 (c2 || c3))
     {                              {
```

cino-u

uN　和 (N 一样，但更深一级嵌套层。(缺省为 'shiftwidth')。

```
cino=                          cino=u2
  if (c123456789                 if (c123456789
          && (c22345                     && (c22345
              || c3))                        || c3))
```

cino-U

UN　如果 N 非零，即使没结束的括号是所在行的首个非空白字符，也不忽略 (或是 u 指定的缩进。(缺省为 0)。

```
cino= 或 cino=(s               cino=(s,U1
  c = c1 &&                      c = c1 &&
      (                              (
       c2 ||                          c2 ||
       c3                             c3
      ) && c4;                       ) && c4;
```

cino-w

wN　如果 N 非零，在没有结束的括号内如果使用 "(0" 或是 "u0"，或当没有结束的括号是所在行的首个非空白字符但使用 "U0" 时，对齐使用的是紧随在没有结束括号之后的字符，而不一定是其后首个非空白字符。(缺省为 0)。

```
cino=(0                        cino=(0,w1
  if (   c1                      if (   c1
         && (   c2                      && (   c2
                || c3))                        || c3))
      foo;                           foo;
```

cino-W

WN　如果 N 非零，在没有结束的括号内如果使用 "(0" 或是 "u0" 且当没有结束的括号是所在行的最后一个非空白字符且不是闭括号时，下一行相对外部代码块 (比如，行首或是下一层没结束的括号) 缩进 N 个字符。(缺省为 0)。

```
cino=(0                         cino=(0,W4
  a_long_line(                    a_long_line(
              argument,                       argument,
              argument);                      argument);
  a_short_line(argument,          a_short_line(argument,
               argument);                     argument);
```

cino-k

kN　在 "if"、"for" 或 "while" 之后没有结束的括号内且 N 非零时，覆盖

"(N" 定义的行为：相对外层上下文（即 "if"、"for" 或 "while" 所在行）缩进 N 个字符。对更深层的嵌套没有影响。影响 "wN" 等标志位的行为，但仅限于 "if"、"for" 和 "while" 条件。0 值对应的缺省是 "(N" 标志位定义的行为。(缺省为 0)。

```
cino=(0                    cino=(0,ks
  if (condition1             if (condition1
      && condition2)                 && condition2)
      action();                  action();
  function(argument1         function(argument1
           && argument2);             && argument2);
```

cino-m

mN 如果 N 非零，则将 ')' 开始的行和与其匹配的 '(' 所在行的第一个字符对齐。(缺省为 0)。

```
cino=(s                    cino=(s,m1
  c = c1 && (                c = c1 && (
      c2 ||                      c2 ||
      c3                         c3
      ) && c4;                   ) && c4;
  if (                       if (
      c1 && c2                   c1 && c2
     )                          )
      foo;                       foo;
```

cino-M

MN 如果 N 非零，则将 ')' 开始的行与前一行的第一个字符对齐。(缺省为 0)。

```
cino=                      cino=M1
  if (cond1 &&               if (cond1 &&
        cond2                      cond2
     )                          )
```

java-cinoptions java-indenting cino-j

jN 正确地缩进 Java 匿名类，也适用于 Javascript。目前，'N' 的值并没有用到但其不能为 0 (比如 'j1')。'j1' 将对下面的代码段这样正确的缩进：

```
object.add(new ChangeListener() {
    public void stateChanged(ChangeEvent e) {
        do_something();
    }
});
```

javascript-cinoptions javascript-indenting cino-J

JN 正确缩进 JavaScript 对象声明，不把它错误地当成标签。目前 'N' 的值并没有用到但不能为 0 (比如 'J1'). 要打开之，也应考虑同时打开 cino-j 。

```
var bar = {
    foo: {
        that: this,
```

```
            some: ok,
        },
        "bar":{
            a : 2,
            b: "123abc",
            x: 4,
            "y": 5
        }
    }
```

cino-)

)N　Vim 最多在 N 行范围内查找没有结束的括号。这将缩小 Vim 用来查找括号的时间。(缺省 20 行)。

cino-star

*N　Vim 最多在 N 行范围内查找没有结束的注释。这将缩小 Vim 用来查找注释头部的时间。
如果你的 /* * / 注释在 N 行后停止缩进，这里就是你要改的地方。
(缺省 70 行)。

cino-#

#N　如果 N 非零，识别 shell/Perl 风格 # 开头的注释，不识别预处理命令行；允许以 "#" 开头的行的右移。
如果 N 为零 (缺省)：不识别 '#' 注释，但识别预处理命令行；不允许以 "#" 开头的行右移。

全部的缺省值是：

```
cinoptions=>s,e0,n0,f0,{0,}0,^0,L-1,:s,=s,l0,b0,gs,hs,N0,E0,ps,ts,is,+s,
        c3,C0,/0,(2s,us,U0,w0,W0,k0,m0,j0,J0,)20,*70,#0
```

Vim 将一行从第一列开始，如果：
- 它以 '#' 开头 (预处理指令)，并且 'cinkeys' 包含 '#0'。
- 它以一个标号 (后面跟随一个 ':' 的关键字，除了 "case" 或 "default" 以外) 开头，并且 'cinoptions' 不包含使用正数值的 'L' 项目。
- 任何缩进的组合使这一行产生小于 0 的缩进。

---

2. 按表达式缩进　indent-expression

关于灵活缩进的一些基本知识在用户手册的 30.3 有解释。

如果你要写一个自己的缩进文件，它必须设定 'indentexpr' 选项。设定 'indentkeys' 也经常有用。
$VIMRUNTIME/indent/README.txt 文件有提示。
$VIMRUNTIME/indent 目录下有例子可以参考。

特定缩进文件的若干评注

CLOJURE　ft-clojure-indent　clojure-indent

Clojure 的缩进和传统的 Lisp 有所不同，部分由于方括号和尖括号的使用，还有部分是

社群习惯的差异。这些惯例又不是普遍遵循的。因为这样，Closure 缩进脚本提供了若干可配置的选项，如下所述。

如果当前的 vim 不支持 searchpairpos()，缩进脚本回到普通 'lisp' 缩进方式并忽略以下所述的选项。

g:clojure_maxlines

设置 searchpairpos() 的最大检索距离。较大的值牺牲性能以换取处理更长表达式 (form) 的正确性。0 会使检索没有限制。

```
" 缺省
let g:clojure_maxlines = 100
```

g:clojure_fuzzy_indent
g:clojure_fuzzy_indent_patterns
g:clojure_fuzzy_indent_blacklist

'lispwords' 选项是一个逗号分隔的单词列表，指定那些需要以两个空格缩进子表达式的特殊表达式。

例如：

```
(defn bad []
      "错误的缩进")

(defn good []
  "正确的缩进")
```

如果想用 pattern 来代替 'lispwords'，可用模糊缩进功能：

```
" 缩进
let g:clojure_fuzzy_indent = 1
let g:clojure_fuzzy_indent_patterns = ['^with', '^def', '^let']
let g:clojure_fuzzy_indent_blacklist =
        \ ['-fn$', '\v^with-%(meta|out-str|loading-context)$']

" 旧式的逗号分隔字符串版本；建议采用上面的列表格式。注意这里的模式总是
" 隐含假定以 ^ 开始，以 $ 结束
let g:clojure_fuzzy_indent_patterns = 'with.*,def.*,let.*'
```

g:clojure_fuzzy_indent_patterns 和 g:clojure_fuzzy_indent_blacklist 是模式的列表 Lists，用于匹配 Lisp 列表 (list) 的头部去掉引号和限定词部分的符号。这意味着形如 "^foo" 的模式会匹配以下的候选："foobar"、"my.ns/foobar" 和 "#'foobar"。

每个候选词会以下列顺序进行是否需要特殊处理的测试：

1. 若单词按本义出现在 'lispwords' 中，返回真
2. 若单词匹配 g:clojure_fuzzy_indent_blacklist 中的某个模式，返回假
3. 若单词匹配 g:clojure_fuzzy_indent_patterns 中的某个模式，返回真
4. 否则返回假，按正常方式缩进

g:clojure_special_indent_words

有些 Closure 的表达式的缩进方式要求每个子表达式只缩进两格，不管 'lispwords' 如何设置。如果你有要求这种独特的缩进方式的结构，可以把相应的符号加入下面的缺省列表。

```
" 缺省
let g:clojure_special_indent_words =
   \ 'deftype,defrecord,reify,proxy,extend-type,extend-protocol,letfn'
```

g:clojure_align_multiline_strings

多行字符串的后续行和引导引号之后的列对齐，而不是对齐与同一列。

示例：

```
(def default
  "Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do
  eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut
  enim ad minim veniam, quis nostrud exercitation ullamco laboris
  nisi ut aliquip ex ea commodo consequat.")

(def aligned
  "Lorem ipsum dolor sit amet, consectetur adipisicing elit, sed do
   eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut
   enim ad minim veniam, quis nostrud exercitation ullamco laboris
   nisi ut aliquip ex ea commodo consequat.")
```

缺省关闭此选项。

```
" 缺省
let g:clojure_align_multiline_strings = 0
```

g:clojure_align_subforms

缺省括号内的复合表达式以函数调用形式出现，头部子表达式在单独一行上，后续的子表达式则相对开括号缩进 2 个空格：

```
(foo
  bar
  baz)
```

置位此选项改变此行为，使得所有的子表达式在同一列对齐，这模拟了 clojure-mode.el 的缺省行为：

```
(foo
 bar
 baz)
```

缺省关闭此选项。

```
" Default
let g:clojure_align_subforms = 0
```

FORTRAN ft-fortran-indent

Block if、select case、where 和 forall 代码块被缩进。type、interface、associate、block 和 enum 结构也是。subrountine、function、module 和 program block 的缩进可选。如果 Fortran 在自由格式下，注释，标号语句和续行被缩进；如果在固定宽度格式下，这些就不会被缩进，因为左边必须要求留出空白。因此，在固定宽度格式下，需要手动对标号语句和续行缩进。关于检测源码格式的更深入讨论，参见 ft-fortran-syntax 。

do 循环
缺省情况下，所有的 do 循环保持原样，不缩进。在 Fortran 里，循环甚至是多重循环可以结束于一个任意类型的带标号可执行语句，使得 do 循环非结构化。对其正确的缩进需要编译器级别的语法分析。对于这样 do 循环结束于任意类型的带标号语句的老式代码可以使用 Tidy (http://www.unb.ca/chem/ajit/f_tidy.htm) 等专门的程序缩进。结构化的 do/continue 循环也保持不缩进，因为 continue 除了结束 do 循环以外还有其它的用途。Tidy 可以将结构化的 do/continue 循环转换成 do/enddo 的形式。do/enddo 类型的循环能被缩进。如果你只使用结构化的 do/enddo 形式的循环，可以在 .vimrc 中作如下声明：

```
let fortran_do_enddo=1
```

这样 do 循环就被缩进了。如果只是在比如 .f90 文件里，你的所有循环都是 do/enddo 形式，那么可以像下面一样用自动命令设置一个缓冲区标志位

```
au! BufRead,BufNewFile *.f90 let b:fortran_do_enddo=1
```

使 .f90 文件里的 do 循环被缩进而在其它扩展名里，比如 .for 文件中不被缩进。

程序单元
缺省打开程序单元 (subrountine、function、module 和 program block) 的缩进，如果需要轻量级的节省屏幕宽度的缩进风格，可以关闭之。要关闭所有 fortran 文件的程序单元的缩进，在 .vimrc 里设置全局变量 fortran_indent_less

```
let fortran_indent_less=1
```

要更精细的控制，可以设置对应的缓冲区局部变量

```
let b:fortran_indent_less=1
```

HTML ft-html-indent html-indent html-indenting

这里说明定制 HTML 缩进的相关变量，可以把它们放到你的 vimrc 里。

可以设置 <script> 和 <style> 这些 "区块标签" 之后的首行缩进 (缺省为 "zero")：

```
:let g:html_indent_script1 = "inc"
:let g:html_indent_style1 = "inc"
```

值	含义
"zero"	零缩进
"auto"	自动缩进 (和匹块标签相同的缩进)
"inc"	自动缩进 + 一个缩进位置

缺省，给许多标签之后的内容增加缩进（见脚本里的 "Add Indent Tags"）。可以加入更多类似的标签：

```
:let g:html_indent_inctags = "html,body,head,tbody"
```

还可以删除已有的标签：

```
:let g:html_indent_autotags = "th,td,tr,tfoot,thead"
```

这两个变量的缺省值为空。注意：初始的 "inctags" 值只在每个 Vim 会话中定义一次。

只在脚本执行时读入这些用户变量。要在会话中间执行更改，除了重载 HTML 文件外，可以手动调用：

```
:call HtmlIndent_CheckUserSettings()
```

细节：
"区块标签" 内的缩进的计算，有些 "外来的" 内容：

区域标签	缩进表达式	适用情况
<script>	: {可定制}	如果是区块的首行
	: cindent(v:lnum)	如果没有属性或其中包含 "java"
	: -1	否则（vbscript、tcl、...）
<style>	: {可定制}	如果是区块的首行
	: GetCSSIndent()	否则
<!-- -->	: -1	

MATLAB　　ft-matlab-indent　matlab-indent　matlab-indenting

MATLAB 编辑器/调试器语言首选项中的设置函数缩进格式相当于：

```
:let g:MATLAB_function_indent = {0、1 或 2（缺省）}
```

其中 0 用于经典模式，1 用于缩进嵌套函数 而 2 用于缩进全部函数。

PHP　　ft-php-indent　php-indent　php-indenting

注意：　只有激活 PHP 语法 syntax 才能正确缩进 PHP 文件。

如果在 Unix 'fileformat' 下编辑文件而换行符之前有 '\r' 字符，缩进的处理会不正确。需要先去除这些没有的字符。例如可用：

```
:%s /\r$//g
```

或者，你可以简单地 :let 变量 PHP_removeCRwhenUnix 为 1，脚本会在 Vim 载入 PHP 文件（每次 BufRead ）时安静地删除它们。

选项：

PHP 缩进可以通过修改若干全局变量的值来对几个方面进行改变：

php-comment　PHP_autoformatcomment

要缺省不打开注释的自动排版（如果要使用自己的 'formatoptions' 的话）：

```
:let g:PHP_autoformatcomment = 0
```

否则，'t' 会从 'formatoptions' 字符串中去除，并加上 "qrowcb"，详见
fo-table 。

PHP_outdentSLComments

要给单行注释增加额外的缩进：

```
:let g:PHP_outdentSLComments = N
```

其中的 N 是要增加的 'shiftwidth' 的倍数。

只影响如下所示的单行注释：

```
# Comment
// Comment
/* Comment */
```

PHP_default_indenting

要给所有 PHP 行增加额外的缩进 N 个 'shiftwidth'：

```
:let g:PHP_default_indenting = N
```

例如，如果 N = 1：

```
<?php
    if (!isset($History_lst_sel))
        if (!isset($History_lst_sel))
            if (!isset($History_lst_sel)) {
                $History_lst_sel=0;
            } else
                $foo="bar";

    $command_hist = TRUE;
?>
```

(注意 PHP 标识符和它包含的代码之间额外的缩进)

PHP_outdentphpescape

要使 PHP 标签和和包围它的非 PHP 代码使用相同的缩进 (只影响 PHP 转义标签)：

```
:let g:PHP_outdentphpescape = 0
```

PHP_removeCRwhenUnix

要在 'fileformat' 设为 Unix 时自动删除 '\r' 字符：

```
:let g:PHP_removeCRwhenUnix = 1
```

PHP_BracesAtCodeLevel

要使花括号的缩进和其包含的代码相同：

```
:let g:PHP_BracesAtCodeLevel = 1
```

给出的结果会变成：

```
if ($foo)
    {
    foo();
```

```
        }
```
而不是：
```
    if ($foo)
    {
        foo();
    }
```

注意：　如果使用此选项，缩进可能会稍慢一些，因为无法使用若干优化。

PHP_vintage_case_default_indent

要缩进 switch() 块的 'case:' 和 'default:' 语句：
```
    :let g:PHP_vintage_case_default_indent = 1
```

PHP 中 'case/default' 块并非必须使用花括号，因此，'case:' 和 'default:' 和 'switch()' 的缩进层次相同，以避免不必要的缩进。可用上述的选项来恢复传统的缩进方式。

PHP_noArrowMatching

缺省缩进脚本会缩进多行的链式调用，使 '->' 位置匹配：

```
    $user_name_very_long->name()
                        ->age()
                        ->info();
```

把此选项设为 1 可以回到经典缩进方式：
```
    :let g:PHP_noArrowMatching = 1
```

会得到以下结果：

```
    $user_name_very_long->name()
        ->age()
        ->info();
```

PHP_IndentFunctionCallParameters

为多行函数调用的参数额外缩进。
```
    let g:PHP_IndentFunctionCallParameters = 1
```

函数调用参数会额外多缩进一层。如果是两个空格的缩进：

```
    function call_the_thing(
      $with_this,
      $and_that
    ) {
      $this->do_the_thing(
          $with_this,
          $and_that
      );
    }
```

PHP_IndentFunctionDeclarationParameters

为多行函数定义的参数额外缩进。

```
let g:PHP_IndentFunctionDeclarationParameters = 1
```

函数声明参数会额外多缩进一层。如果是两个空格的缩进：

```
function call_the_thing(
    $with_this,
    $and_that
) {
  $this->do_the_thing(
    $with_this,
    $and_that
  );
}
```

PYTHON ft-python-indent

可以为以下情形设置不同的缩进值。下面所举的例子给出其缺省值。注意 这些变量用表达式赋值，所以你以后可以改变 'shiftwidth' 的值来改变此变量。

在 "(" 后的缩进：

```
let g:pyindent_open_paren = 'shiftwidth() * 2'
```

在嵌套的括号后的缩进：

```
let g:pyindent_nested_paren = 'shiftwidth()'
```

续行的缩进：

```
let g:pyindent_continue = 'shiftwidth() * 2'
```

此方法使用 searchpair() 往回找未闭合的括号。这有时会较慢，因此有 150 毫秒的超时。如果你注意到缩进不正确，可以设置长一些的超时，以毫秒计：

```
let g:pyindent_searchpair_timeout = 500
```

如果往回找未闭合的括号还是太慢，特别是在复制粘贴操作的时候，或者你不需要在多行括号内进行缩进，可以完全关闭此功能：

```
let g:pyindent_disable_parentheses_indenting = 1
```

R ft-r-indent

函数参数如果跨越多行，会进行对齐。如果不想函数参数进行对齐，在 vimrc 里放入：

```
let r_indent_align_args = 0
```

所有注释字符 # 开始的行和正常 R 代码采用相同的缩进级别。Emacs/ESS 的用户可能习惯单个 # 的缩进于第 40 列，而 ## 以正常 R 代码方式缩进，### 不缩进。 如果你习惯 Emacs/ESS 的注释对齐方式，在 vimrc 里放入：

```
let r_indent_ess_comments = 1
```

要单个 # 开始行的对齐不在第 40 列，给 r_indent_common_column 设置一个新值，如下例所示：

```
let r_indent_comment_column = 30
```

缩进所有以 "<-" 结束的行之后的代码。Emacs/ESS 则不缩进顶层函数的代码。如果你希望 Vim-R-plugin 这方面采用 Emacs/ESS 风格，在 .vimrc 里放入：

```
let r_indent_ess_compatible = 1
```

下面举一个打开和关闭此选项的缩进例子：

```
### r_indent_ess_compatible = 1
foo <-
    function(x)
{
    paste(x)
}
```

```
### r_indent_ess_compatible = 0
foo <-
    function(x)
    {
        paste(x)
    }
```

此代码会在匹配模式 ‘’\(&\| \ +\ -\ *\ /\ =\ \~\ %\|->\)\s*$’‘ 的行后缩进。如果你希望在匹配其它模式的行后缩进，可以在 vimrc 中设置 r_indent_op_pattern 合适的值。

SHELL ft-sh-indent

通过 b:sh_indent_defaults 字典 Dictionary 的以下键，可以配置外壳文件在各种情况下使用的缩进距离。它们可以是某个特定的值或者是返回所需距离的函数引用 Funcref ：

b:sh_indent_options['default'] 缺省缩进距离。

b:sh_indent_options['continuation-line']
加到续行上的缩进距离。

b:sh_indent_options['case-labels']
加到 case 标签上的缩进距离。
(没有实际实现)

b:sh_indent_options['case-statements']
加到 case 语句上的缩进距离。

b:sh_indent_options['case-breaks']
加到 (更有可能，减少) case break 语句上的缩进距离。

VERILOG ft-verilog-indent

一般的代码块，像 if、for、case、always、initial、function、specify、begin 等等都被缩进。module 代码块 (处于第一级) 缺省不被缩进。你可以在 .vimrc 里打开这个缩进：

```
let b:verilog_indent_modules = 1
```

这样 module 代码块就被缩进。要关闭缩进，删除这个变量：

```
:unlet b:verilog_indent_modules
```

要只对 Verilog 文件设定变量，可以这样做：

```
au BufReadPost * if exists("b:current_syntax")
au BufReadPost *   if b:current_syntax == "verilog"
au BufReadPost *     let b:verilog_indent_modules = 1
au BufReadPost *   endif
au BufReadPost * endif
```

另外，可以设定 b:verilog_indent_width 来改变缩进值（缺省为 'shiftwidth'）：

```
let b:verilog_indent_width = 4
let b:verilog_indent_width = shiftwidth() * 2
```

你还可以打开详细模式来帮助调试：

```
let b:verilog_indent_verbose = 1
```

不要忘了要先设定 ":set cmdheight=2" 才能显示信息。

VHDL ft-vhdl-indent

缺省执行 类属（generic）/端口（port）映射语句的对齐。下例说明这种对齐的应用：

```
ENTITY sync IS
PORT (
       clk        : IN  STD_LOGIC;
       reset_n    : IN  STD_LOGIC;
       data_input : IN  STD_LOGIC;
       data_out   : OUT STD_LOGIC
     );
END ENTITY sync;
```

要关闭之，在 .vimrc 文件中加上

```
let g:vhdl_indent_genportmap = 0
```

而上例也随之改变了对齐：

```
ENTITY sync IS
PORT (
  clk        : IN  STD_LOGIC;
  reset_n    : IN  STD_LOGIC;
  data_input : IN  STD_LOGIC;
  data_out   : OUT STD_LOGIC
);
END ENTITY sync;
```

缺省执行右手边赋值 "<=" 语句的对齐。下例说明这种对齐的应用：

```
sig_out <= (bus_a(1) AND
           (sig_b OR sig_c)) OR
```

```
            (bus_a(0) AND sig_d);
```

要关闭之，在 .vimrc 文件中加上

```
let g:vhdl_indent_rhsassign = 0
```

而上例也随之改变了对齐：

```
sig_out <= (bus_a(1) AND
  (sig_b OR sig_c)) OR
  (bus_a(0) AND sig_d);
```

整行注释（"--" 打头的行）的缩进和上一行注释对齐，如果 "--" 之后有一个空格的话。

示例：

```
sig_a <= sig_b; -- 开始注释
                -- 注释继续
                -- 同一注释的更多内容
```

在插入模式下，输入 "-- " 后（注意空格 " "），按 CTRL-F 会使当前的 "-- " 和上一行的 "--" 对齐。

如果紧邻的上一行不包含 "--"，那么 整行注释会和下一个非空白且 不是 整行注释的行的起始位置对齐。

下列代码的缩进：

```
sig_c <= sig_d; -- 注释 0
       -- 注释 1
             -- 注释 2
  --debug_code:
  --PROCESS(debug_in)
       --BEGIN
          --  FOR i IN 15 DOWNTO 0 LOOP
           --    debug_out(8*i+7 DOWNTO 8*i) <= debug_in(15-i);
          --  END LOOP;
   --END PROCESS debug_code;

    -- 注释 3
sig_e <= sig_f; -- 注释 4
        -- 注释 5
```

结果是：

```
sig_c <= sig_d; -- 注释 0
                -- 注释 1
                -- 注释 2
--debug_code:
--PROCESS(debug_in)
--BEGIN
```

```
--  FOR i IN 15 DOWNTO 0 LOOP
--    debug_out(8*i+7 DOWNTO 8*i) <= debug_in(15-i);
--  END LOOP;
--END PROCESS debug_code;

-- 注释 3
sig_e <= sig_f; -- 注释 4
                -- 注释 5
```

注意到 "--debug_code:" 不和 "-- comment 2" 对齐，因为 "--debug_code:" 的 "--" 之后没有空白。

基于注释缩进的动态性，缩进应该执行 两次 。第一遍缩进代码，第二遍依据已正确缩进的代码来缩进整行注释。

VIM ft-vim-indent

对于缩进 Vim 脚本，有个变量可以指定续行的缩进距离，续行就是以反斜杠开始的行：

```
:let g:vim_indent_cont = shiftwidth() * 3
```

缺省值是三倍的 shiftwidth。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册

by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

语法高亮　　syntax　syntax-highlighting　coloring

语法高亮使得 Vim 可以用不同的字体或颜色显示文本的不同部分。这些部分可以是特定的关键字，或者匹配某模式的文本。Vim 不会分析整个文件（为了保持快速），所以高亮功能有其限制。词法高亮或者是一个更合适的名字，但既然大家都使用语法高亮，我们也就如此称呼。

Vim 支持所有终端上的语法高亮。但因为多数普通的终端只有有限的高亮的选择，GUI 版本 gvim 的语法高亮的效果最好。

用户手册里：
usr_06.txt　介绍语法高亮。
usr_44.txt　介绍如何写语法文件。

1. 快速入门　　:syn-qstart
2. 语法文件　　:syn-files
3. 语法载入过程　　syntax-loading
4. 转换为 HTML　　2html.vim
5. 语法文件附注　　:syn-file-remarks
6. 定义语法　　:syn-define
7. :syntax 参数　　:syn-arguments
8. 语法模式　　:syn-pattern
9. 语法簇　　:syn-cluster
10. 包含语法文件　　:syn-include
11. 同步　　:syn-sync
12. 列出语法项目　　:syntax
13. Highlight 命令　　:highlight
14. 链接组　　:highlight-link
15. 清理　　:syn-clear
16. 高亮标签　　tag-highlight
17. 窗口局部语法　　:ownsyntax
18. 彩色 xterm　　xterm-color
19. 语法太慢了　　:syntime

{Vi 无此功能}

如果编译时关闭了 +syntax 特性，就不会有语法高亮的功能。

---

1. 快速入门　　:syn-qstart

:syn-enable　:syntax-enable

这个命令打开语法高亮：

```
:syntax enable
```

实际上，它只是执行如下命令

```
:source $VIMRUNTIME/syntax/syntax.vim
```

如果没有设置 VIM 环境变量，Vim 会试图用其它方法找到该路径（见 $VIMRUNTIME ）。通常总能找到，如果不行，你可以自己设置 VIM 环境变量为存放 Vim 相关文件所在的目录。例如，如果你的语法文件放在 "/usr/vim/vim81/syntax" 目录，设置 $VIMRUNTIME 为 "/usr/vim/vim81"。启动 Vim 前，你必须在外壳上进行此设置。当 GUI 已经在运行或即将运行时，此命令也执行 menu.vim 脚本。要避免可见 'go-M' 。

:syn-on :syntax-on

‘:syntax enable‘ 命令会保持你当前的色彩设置。这样，不管在使用此命令的前后，你都可以用 :highlight 命令设置你喜欢的颜色。如果你希望 Vim 用缺省值覆盖你自己的，只要用:

```
:syntax on
```

:hi-normal :highlight-normal

如果你运行在 GUI 环境里，你可以这样得到黑底白字:

```
:highlight Normal guibg=Black guifg=White
```

关于色彩终端，见 :hi-normal-cterm 。
关于设置你自己的语法高亮色彩，见 syncolor 。

注意: MS-DOS 和 Windows 上的语法文件以 <CR><NL> 结束每一行。Unix 上则以 <NL> 结尾。这意味着你需要为自己的系统选择合适的文件。不过，在 MS-DOS 和 Windows 上，如果 'fileformats' 选项非空，会自动选择正确的格式。

注意: 使用反转视频时 ("gvim -fg white -bg black")，'background' 的缺省值直到 GUI 窗口打开时才会设置。这发生在 gvimrc 的读入之后，从而会使用错误的缺省高亮。要在打开高亮前设置 'background' 的缺省值，在 gvimrc 文件里包含 ":gui" 命令:

```
:gui          " 打开窗口并设置 'background' 的缺省值
:syntax on    " 启动语法高亮，使用 'background' 来设置颜色
```

注意: 在 gvimrc 里使用 ":gui" 意味着 "gvim -f" 不会在前台打开！这时，要使用 ":gui -f"。

g:syntax_on

你可以用这个命令切换语法的打开/关闭

```
:if exists("g:syntax_on") | syntax off | else | syntax enable | endif
```

要把它放到映射里，你可以用:

```
:map <F7> :if exists("g:syntax_on") <Bar>
    \   syntax off <Bar>
    \ else <Bar>
    \   syntax enable <Bar>
    \ endif <CR>
```

[使用 <> 记法，按本义输入]

细节:
":syntax" 命令是用执行文件脚本实现的。要了解它到底干了什么，察看以下文件:

命令	文件
:syntax enable	$VIMRUNTIME/syntax/syntax.vim
:syntax on	$VIMRUNTIME/syntax/syntax.vim

```
:syntax manual      $VIMRUNTIME/syntax/manual.vim
:syntax off         $VIMRUNTIME/syntax/nosyntax.vim
```
另见 syntax-loading 。

注意：如果显示长行很慢而关闭语法高亮就能加快，考虑设置 'synmaxcol' 选项为一个较小的值。

---

2. 语法文件 :syn-files

某个语言的语法和高亮命令通常存在一个语法文件里。命名惯例是："{name}.vim"。其中，{name} 是语言的名字或者缩写（一旦在 DOS 文件系统上，要适用 8.3 字符的名字限制）。
例如：

```
c.vim           perl.vim        java.vim        html.vim
cpp.vim         sh.vim          csh.vim
```

语法文件可以包含任何 Ex 命令，就像 vimrc 文件那样。但应该只有适用于特定语言的命令才放在里面。如果该语言是另外一个语言的超集，它可以包含那个语言对应的文件。例如，cpp.vim 可以包含 c.vim 文件：

```
:so $VIMRUNTIME/syntax/c.vim
```

.vim 文件通常使用自动命令载入。例如：

```
:au Syntax c     runtime! syntax/c.vim
:au Syntax cpp   runtime! syntax/cpp.vim
```

这些命令通常出现在 $VIMRUNTIME/syntax/synload.vim 文件里。

如果你有多个文件，可以用文件类型作为目录名。该目录下的所有 "*.vim" 文件都会被使用。例如：

```
~/.vim/after/syntax/c/one.vim
~/.vim/after/syntax/c/two.vim
```

创立你自己的语法文件 mysyntaxfile

一旦你创立自己的语法文件，并且需要 Vim 在使用 ":syntax enable" 的时候自动调用它们的时候，按如下操作进行：

1. 建立自己的用户运行时目录。通常，把它放在 'runtimepath' 选项的第一项。Unix 的例子：

```
mkdir ~/.vim
```

2. 在其中建立 "syntax" 目录。Unix 上：

```
mkdir ~/.vim/syntax
```

3. 编写 Vim 语法文件。或者从 internet 下载。然后写入你的 syntax 目录。例如，对于 "mine" 语言的语法：

```
:w ~/.vim/syntax/mine.vim
```

现在你可以手动启动语法文件：

```
:set syntax=mine
```

这时，你并不需要退出 Vim。

如果你想 Vim 也实现文件类型的检测，见 new-filetype 。

如果你想为多个用户设立系统，而不希望每个用户增加相同的语法文件的话，可以使用 'runtimepath' 里的其它目录。

添加到已存在的语法文件 mysyntaxfile-add

如果你对已有的语法文件大体满意，只需要增加或者修正一些高亮项目，执行以下步骤即可：

1. 建立你的 'runtimepath' 里的用户目录，见上。

2. 在那里建立目录 "after/syntax"。Unix 上：
```
mkdir ~/.vim/after
mkdir ~/.vim/after/syntax
```

3. 编写 Vim 脚本，包含你希望使用的命令。例如，要在 C 语法里改变颜色：
```
highlight cComment ctermfg=Green guifg=Green
```

4. 把该文件写道 "after/syntax" 目录。使用该语法的名字，另加上 ".vim"。对于我们的 C 语法：
```
:w ~/.vim/after/syntax/c.vim
```

这样就行了。下一次你编辑 C 文件时，Comment 色彩就不会一样了。你甚至不需要重启 Vim。

替换已存在的语法文件 mysyntaxfile-replace

如果你不喜欢发布的语法文件版本，或者你下载了新版本。可以采用上述的 mysyntaxfile 相同的步骤。只要确保你写的语法文件在 'runtimepath' 较早的位置就可以了。Vim 只会载入第一个找到的语法文件，假定它是用来设置 b:current_syntax 的那个文件。

命名惯例 group-name {group-name} E669 W18

高亮组名用于匹配相同类型事物的高亮项目。它们被链接到用于指定颜色的高亮组。语法组名本身不指定任何颜色或属性。

高亮或者语法组的名字必须由 ASCII 字母，数字和下划线组成。如使用正规表达式表示："[a-zA-Z0-9_]*"。不过，使用其它字符时 Vim 不报错。

要允许每个用户选择他自己喜欢的色彩系列，许多语言应该使用相同的高亮组名。以下是建议的组名（如果语法高亮能够正确工作，你应该能看到实际的颜色，除了 "Ignore" 以外）：

```
*Comment        v 任何注释

*Constant       v 任何常数
 String         v 字符串常数: "这是字符串"
 Character      v 字符常数: 'c'、'\n'
 Number         v 数值常数: 234、0xff
 Boolean        v 布尔型常数: TRUE、false
```

```
 Float          v 浮点常数: 2.3e10

*Identifier     v 任何变量名
 Function       v 函数名 (也包括: 类的方法名)

*Statement      v 任何语句
 Conditional    v if、then、else、endif、switch 等
 Repeat         v for、do、while 等
 Label          v case、default 等
 Operator       v "sizeof"、"+"、"*" 等
 Keyword        v 其它关键字
 Exception      v try、catch、throw

*PreProc        v 通用预处理命令
 Include        v 预处理命令 #include
 Define         v 预处理命令 #define
 Macro          v 同 Define
 PreCondit      v 预处理命令 #if、#else、#endif 等

*Type           v int、long、char 等
 StorageClass   v static、register、volatile 等
 Structure      v struct、union、enum 等
 Typedef        v typedef 定义

*Special        v 任何特殊符号
 SpecialChar    v 常数中的特殊字符
 Tag            v 可以使用 CTRL-] 的项目
 Delimiter      v 需要注意的字符
 SpecialComment v 注释里的特殊部分
 Debug          v 调试语句

*Underlined     v 需要突出的文本，HTML 链接

*Ignore         v 留空，被隐藏    hl-Ignore

*Error          v 有错的构造

*Todo           v 需要特殊注意的部分；主要是关键字 TODO FIXME 和 XXX
```

* 标记的名字是主要组，其它是次要组。对主要组而言，"syntax.vim" 文件包含缺省的高亮设置。次要组则链接到主要组。所以它们拥有相同的高亮设置。不过，你可以通过 ":highlight" 命令在 "syntax.vim" 文件之后改变所有组的缺省值。

注意 高亮组的名字对大小写不敏感。"String" 和 "string" 可用来表示相同的组。

以下名字是保留字，它们不应用作组名：

```
NONE    ALL    ALLBUT    contains    contained
```

如果你使用 Ignore 组，也可以考虑使用隐藏机制。见 conceal 。 hl-Ignore

---

3. 语法载入过程 syntax-loading

这里解释命令 ":syntax enable" 执行的细节。Vim 初始化自身时，它寻找运行时文件的位置。其值用来设置这里使用的变量 $VIMRUNTIME 。

":syntax enable" 和 ":syntax on" 完成以下步骤：

```
执行 $VIMRUNTIME/syntax/syntax.vim
|
+-  清除旧的语法：执行 $VIMRUNTIME/syntax/nosyntax.vim
|
+-  先执行 'runtimepath' 里的 syntax/synload.vim
|   |
|   +-  为语法高亮设置色彩。如果已经定义了色彩方案，使用 ":colors {name}"
|   |   再次载入。不然，执行 ":runtime! syntax/syncolor.vim"。":syntax on"
|   |   会覆盖已有的颜色，而 ":syntax enable" 只会设置没有设置过的组。
|   |
|   +-  设置 syntax 自动命令，以便在设置 'syntax' 选项时，自动载入合适的语
|   |   法文件。  synload-1
|   |
|   +-  通过  mysyntaxfile  变量，载入用户可选的文件。这只是为了和 Vim 5.x
|   |   兼容。  synload-2
|
+-  执行 ":filetype on"，后者又执行 ":runtime! filetype.vim"。它载入任何找
|   到的 filetype.vim。其中，总会执行 $VIMRUNTIME/filetype.vim。后者执行：
|   |
|   +-  根据后缀名，安装自动命令来设置 'filetype' 选项。这为已知的文件类型
|   |   建立了文件名和文件类型的联系。  synload-3
|   |
|   +-  通过  myfiletypefile  变量，载入用户可选的文件。这只是为了和 Vim
|   |   5.x 兼容。  synload-4
|   |
|   +-  安装自动命令，在没有检测到文件类型时执行 scripts.vim 。
|   |    synload-5
|   |
|   +-  执行 $VIMRUNTIME/menu.vim，设置 Syntax 菜单。  menu.vim
|
+-  安装 FileType 自动命令，在检测到文件类型时，设置 'syntax' 选项。
|    synload-6
|
+-  执行 syntax 自动命令，为每个已经载入的缓冲区启动语法高亮。
```

载入文件时，Vim 这样查找相关的语法文件：

```
文件的载入激活 BufReadPost 自动命令。
|
+-  如果和  synload-3  中的一个自动命令匹配（已知的文件类型）或者
|    synload-4  （用户的文件类型），'filetype' 选项设为文件的所属类型。
|
+-  如果文件类型没有找到，激活  synload-5  中的自动命令，并在
|   'runtimepath' 里找寻 scripts.vim。其中，总是会执行
|   $VIMRUNTIME/scripts.vim。后者执行以下步骤。
|   |
|   +-  通过  myscriptsfile  变量，载入用户可选的文件。这只是为了和 Vim
```

| | 5.x 兼容。
| |
| +- 如果文件类型仍然未知，检查文件的内容，并使用类似于
| "getline(1) =~ pattern" 的检查确定文件类型能否识别，并设置
| 'filetype'。
|
+- 如果文件类型被确定并且设置了 'filetype'，激活 FileType 自动命令，见上
| 面的 synload-6 。根据确定的文件类型设置 'syntax'。
|
+- 上面设置 'syntax' 选项的步骤激活 synload-1 (和 synload-2) 设置的自
| 动命令。它会使用以下命令找到 'runtimepath' 里的主语法文件：
| runtime! syntax/<name>.vim
|
+- 激活任何用户安装的 FileType 或 Syntax 自动命令。这可以用来为某项语法修
   正高亮方式。

---

4. 转换为 HTML 2html.vim convert-to-HTML

2html 本身并不是语法文件，只是一个把当前窗口转换成 HTML 的脚本。Vim 打开一个新窗口，在那里它构造 HTML 文件。

保存结果文件以后，你可以使用任何浏览器浏览。那里的颜色应该和你在 Vim 里看到的一模一样。 g:html_line_ids 可使你通过在浏览器的地址栏中 URL 尾部加入 (例如) #L123 或 #123 来跳到指定行。 g:html_dynamic_folds 则使你可以打开或关闭 Vim 中的折叠。

你不应该设置 'filetype' 或 'syntax' 选项为 "2html"！执行此脚本，以转换当前文件：

```
:runtime! syntax/2html.vim
```

许多变量可用来调整 2html.vim 的结果；见下。列出的开关选项可以通过显式设置其值来达到打开和关闭的目的，也可以通过用 :unlet 删除变量的方式来恢复其缺省值。

评注：
- 一些真的很老的浏览器可能不会显示背景色。
- 你可以在大多数浏览器里打印文件 (而且有颜色)！
- 本版本的 TOhtml 可以和旧版本的 Vim 共用，但某些功能，例如隐藏 (conceal) 支持不会工作，而对编译时没有加入 GUI 支持的旧版本，色彩也可能不对。

此处是一个如何在 Unix 外壳上给所有的 .c 和 .h 文件运行此脚本的例子：

```
for f in *.[ch]; do gvim -f +"syn on" +"run! syntax/2html.vim" +"wq" +"q" $f; done
```

g:html_start_line g:html_end_line

要限制转换的行范围，给下面介绍的 :TOhtml 命令加上行范围，或者设置 "g:html_start_line" 和 "g:html_end_line" 分别为要转换的第一行和最后一行。例如，要指定最近设置的可视区域：

```
:let g:html_start_line = line("'<")
:let g:html_end_line = line("'>")
:runtime! syntax/2html.vim
```

:TOhtml

:[range]TOhtml　　":TOhtml" 命令在一个标准插件里定义。该命令为你执行 2html.vim 。给出范围时，此命令设置 g:html_start_line 和 g:html_end_line 分别为范围的开始和结束行。缺省范围是整个缓冲区。

如果窗口处于比较模式，除非置位了 g:html_diff_one_file ，:TOhtml 会在生成的 HTML 里把当前标签页里所有参与比较的窗口并排放在一个 <table> 元素里。 g:html_line_ids 还可以允许你跳到指定窗口的指定行，(例如) #W1L42 跳到首个参加比较的窗口的第 42 行，#W3L87 则是第三个窗口的第 87 行。

示例:

```
:10,40TOhtml " 把第 10-40 行转换为 html
:'<,'>TOhtml " 转换当前/最近的可视选择
:TOhtml      " 转换整个缓冲区
```

g:html_diff_one_file

缺省: 0。
为 0 时， :TOhtml 会在生成的 HTML 里把当前标签页里所有参与比较的窗口并排放在一个 <table> 元素里。
为 1 时，只转换当前的缓冲区。
示例:

```
let g:html_diff_one_file = 1
```

g:html_whole_filler

缺省: 0。
为 0 时，如果 g:html_diff_one_file 为 1，连续多于三行的填充行显示为三行，中间行提示插入的总行数。
为 1 里，显示所有的插入行，这也是 g:html_diff_one_file 没有置位时的行为。

```
:let g:html_whole_filler = 1
```

TOhtml-performance　g:html_no_progress

缺省: 0。
为 0 时，2html.vim 转换的每个主要步骤都会在状态行上显示进度条。
为 1 时，不显示进度条。会有很不显著的速度提高，但你无法估计转换过程要花多长时间；对大文件而言可能会挺久的!
示例:

```
let g:html_no_progress = 1
```

通过指示 Vim 以非交互方式运行会得到更好的效率提高，因为不用在脚本切换缓冲区和窗口时花时间重画屏幕，例如:

```
vim -E -s -c "let g:html_no_progress=1" -c "syntax on" -c "set ft=c" -c "runtime syntax/2html.vi
```

注意 -s 标志位停止 .vimrc 和任何插件的载入，因此必须显式执行/打开影响 HTML 转换的设置。详见 -E 和 -s-ex 。最好单独建一个脚本来代替这里的 -c 命令，然后用 -u 标志位载入之，这样就不用一个个列出要执行的命令了。

hl-TOhtmlProgress　TOhtml-progress-color

如果显示，随着 HTML 转换程度的推进，进度条会沿着状态行显示带色彩的方块。缺省，背景色使用当前的 "DiffDelete" 高亮组。如果 "DiffDelete" 和 "StatusLine" 有相同的背景色，TOhtml 会自动调整颜色以区别。如果你不喜欢自动选择的颜色，可自定义进度条的高亮色。示例：

```
hi TOhtmlProgress guifg=#c0ffee ctermbg=7
```

g:html_number_lines

缺省：当前的 'number' 设置。
为 0 时，生成的 HTML 里显示缓冲区文本时不带行号。
为 1 时，生成的 HTML 里加入行号列，和 Vim 的行号列相同高亮 (hl-LineNr) 。
要强制给行编号，即使没有置位 'number'：

```
:let g:html_number_lines = 1
```

要强制关闭行号：

```
:let g:html_number_lines = 0
```

要回到缺省情况，即使用 'number' 的设置，可删除此变量：

```
:unlet g:html_number_lines
```

g:html_line_ids

缺省：1 如果 g:html_number_lines 置位，否则为 0。
为 1 时，为每个行号加上 HTML 的 id 属性，或者如果行号不显示，为同样的目的加入一个空的 。单个缓冲区的 HTML 页面的 ID 属性形如 L123，比较视图的页面则形如 W2L123，可以用 ID 来跳转到该行 (或比较视图中某个窗口)。同时，加入 Javascript 代码，在跳到指定行之前打开其所在的已关闭的动态折叠 (g:html_dynamic_folds)。
Javascript 还可以让你在 url 中省略窗口号或开头的那个 L。
示例：

```
page.html#L123  跳转到单个缓冲区文件的第 123 行
page.html#123   同上

diff.html#W1L42 跳转到比较的首个窗口的第 42 行
diff.html#42    同上
```

g:html_use_css

缺省：1。
为 1 时，生成使用层叠样式表 (CSS) 的合法 HTML 5 文件，所有的现代浏览器和许多旧式的浏览器都支持。
为 0 时，生成 <font> 标签和类似的过时标记。不推荐，但对真的很老的浏览器、电邮客户端、论坛帖子和类似不支持基本 CSS 的情形，可能会更好些。
例如：

```
:let g:html_use_css = 0
```

g:html_ignore_conceal

缺省：0。
为 0 时，可隐藏文本被从 HTML 中去除，并取决于 'conceallevel' 的当前值，用 :syn-cchar 或 'listchars' 代替。
为 1 时，在生成的 HTML 中包括所有的缓冲区文本，即使是经过隐藏 (conceal)。

以下两个命令的任何一个都可以确保缓冲区的所有文本都包含在生成的 HTML 里 (除非有折叠的情况)：

```
:let g:html_ignore_conceal = 1
:setl conceallevel=0
```

g:html_ignore_folding

缺省：0。
为 0 时，关闭折叠里的实际文本被 Vim 显示折叠所用的文本（ fold-foldtext ）代替。如果你还想允许在 HTML 中用户可以扩展折叠看到实际文本，就像 Vim 可以做的那样的话，另见 g:html_dynamic_folds 。
为 1 时，在生成的 HTML 中包括所有的缓冲区文本；不管文本是否在折叠中都一样。此时， g:html_dynamic_folds 无效。

以下两个命令的任何一个都可以确保缓冲区的所有文本都包含在生成的 HTML 里（除非有隐藏（conceal）的情况）：

```
zR
:let g:html_ignore_folding = 1
```

g:html_dynamic_folds

缺省：0。
为 0 时，生成的 HTML 不包含关闭折叠里的文本。
为 1 时，生成 Javascript 以动态打开折叠并显示其中的文本，如同 Vim 那样。

设置此变量为 1 时，2html.vim 总是使用 CSS 生成样式，不管 g:html_use_css 的设置。

g:html_ignore_folding 置位时，忽略此变量。

```
:let g:html_dynamic_folds = 1
```

g:html_no_foldcolumn

缺省：0。
为 0 时，如果 g:html_dynamic_folds 为 1，生成折叠列，如同 vim 的 foldcolumn（ fold-foldcolumn ）那样，用户点击该列时切换折叠的开与关。
为 1 时，不生成折叠列；如果置位了 g:html_hover_unfold ，可以用鼠标移到折叠文本来打开折叠。

```
:let g:html_no_foldcolumn = 1
```

TOhtml-uncopyable-text g:html_prevent_copy

缺省：空字符串。
如果在浏览器中选中生成的 HTML 文档的全部文本并复制，此选项防止其中部分区域被复制。此项功能用于保证用户只复制-粘贴原始的文本，而不包括生成内容显示的折叠列或行号。可以这样来指定不被复制的区域：

```
f:      折叠列
n:      行号（也包括折叠文本中的）
t:      折叠文本
d:      比较填充行
```

例如，要使折叠列和行号不被复制：

```
:let g:html_prevent_copy = "fn"
```

此功能目前的实现方法是插入只读的 <input> 元素来包围不想复制的区域。这不总是适用。若干理解 HTML 的应用还是会粘贴 <input> 元素。但只接受平凡文本的粘贴目标应该都行。
在生成页中用于防止复制的方法取决于 g:html_use_input_for_pc 的值。

g:html_use_input_for_pc

缺省："fallback"

如果 g:html_prevent_copy 非空，则：

为 "all" 时，使用只读的 <input> 元素来代替不可复制区域的普通文本。在有些浏览器，特别是旧一些的浏览器上，选择整页并复制选择内容时，不随页面文本一起粘贴 <input> 标签。如果 g:html_no_invalid 为 0，<input> 标签会带有非法类型，这会在更多的浏览器上工作，但页面不能通过合法验证。
注意：此方法 不 能用于最近版本的 Chrome 和等价的浏览器上；<input> 标签会随着文本一起粘贴。

为 "fallback"（缺省值）时，为旧浏览器生成相同的 <input> 元素，但新浏览器（通过 CSS 特性查询来检测）会隐藏 <input> 元素，取而代之的是把生成的内容放在 ::before 伪元素里面来显示不可复制的文本。此方法适用于最多数量的浏览器，既包括旧也包括和新的。

为 "none" 时，完全不生成 <input> 元素。只使用生成内容方法，这意味着旧浏览器，尤其是 Interenet Explorer，会使本不应复制的文本要么能复制，要么会完全消失不见。不过，这是最合标准的方法，使用的标注也会少得多。

g:html_no_invalid

缺省：0。
为 0 时，如 g:html_prevent_copy 非空且 g:html_use_input_for_pc 不是 "none"，有意地给不能复制的区域里的 <input> 元素插入一个非法的属性。这使一些应用不会粘贴这些 <input> 元素。特别地，有些版本的 Microsoft Word 不会粘贴有非法属性的 <input> 元素。
为 1 时，不会有意插入非法的标记，生成的页面总是合法的。不过，有些应用里一旦粘贴了 <input> 元素，很难在之后去掉。

g:html_hover_unfold

缺省：0。
为 0 时，打开 2html.vim 用 g:html_dynmaic_folds 设置产生折叠的唯一方法是点击生成的折叠列。
为 1 时，用 CSS 2.0 机制，使得用户把鼠标移到显示的折叠文本上时可以打开折叠。对不想打开 Javascript 但想看到折叠文本的用户会有用。

注意 旧版本的浏览器（尤其是 Internet Explorer 6）不支持该功能。我们生成 IE6 浏览器特定的 HTML 标记，使之回归到普通的 CSS 1 代码，这样折叠还能在此浏览器下正常显示，但没有折叠列它们无法被打开。

```
:let g:html_hover_unfold = 1
```

g:html_id_expr

缺省："
动态折叠和行号的跳转依赖于文档中生成的 ID 的唯一性。如果生成的 HTML 被复制到更大的文档时，ID 的唯一性就很难保证了。设置 g:html_id_expr 可以指定一个表达式，让 Vim 用来生成唯一的字符串，附加到给定文档中的每个 ID 之后，以保证生成的完整 ID 即使和更大的 HTML 文档的其他内容合并时也保持唯一。例如，要给每个 ID 后附加 _ 和缓冲区号：

```
:let g:html_id_expr = '"_".bufnr("%")'
```

要给每个 ID 号附加字符串 "_mystring"：

```
:let g:html_id_expr = '"_mystring"'
```

注意 转换比较视图到 HTML 时，只对比较中的首个窗口计算此表达式，计算结果会用于所有的窗口。

TOhtml-wrap-text g:html_pre_wrap

缺省：当前 'wrap' 的设置。
为 0 时，如果 g:html_no_pre 为 0 或未设，生成的 HTML 中的文本在浏览器窗口边缘处不回绕。
为 1 时，如果 g:html_use_css 为 1，使用 CSS 2.0 的 "white-space:pre-wrap" 值，使文本在浏览器窗口边缘处回绕。
要显式打开回绕：

```
:let g:html_pre_wrap = 1
```

要关闭回绕：

```
:let g:html_pre_wrap = 0
```

回归到缺省值，由 'wrap' 设置值来决定回绕与否：

```
:unlet g:html_pre_wrap
```

g:html_no_pre

缺省：0。
为 0 时，生成的 HTML 用 <pre>...</pre> 标签来包围缓冲区文本。连续空格的显示如同 Vim 那样，无需特别标记，制表符也可以按本义使用（见 g:html_expand_tabs ）。
为 1 时（不推荐），使用普通的 代替 <pre> 标签，用一串 " " 来显示连续的空格，
 用于结束每行。这是支持生成的 HTML 中文本回绕的另一个方法，此方法和旧式的浏览器更兼容，但 2html.vim 生成的页面显示效果与 Vim 的差别显著。

```
:let g:html_no_pre = 1
```

g:html_expand_tabs

缺省：0 如果 'tabstop' 为 8，'expandtab' 为 0，不使用 'vartabstop'，且生成的 HTML 不用折叠列或行号；
1 不然。
为 1 时，缓冲区文本中的 <Tab> 被合适数目的空格，或当 g:html_no_pre 为 1 时， ，所替代。
为 0 时，如果 g:html_no_pre 为 0 或未设，在生成的 HTML 中保留缓冲区文本中的 <Tab> 字符。这使得浏览器中复制粘贴不会丢失原始文档中的实际空白。注意 除非按照缺省设置的那些条件，这会很容易打乱 HTML 中文本的对齐和缩进。

强制 2html.vim 保留 <Tab> 字符：

```
:let g:html_expand_tabs = 0
```

强制扩展制表符：

```
:let g:html_expand_tabs = 1
```

TOhtml-encoding-detect TOhtml-encoding

强烈建议用 g:html_use_encoding 来为任何要放到网页服务器上的内容指定编码。

如果没有指定编码， 2html.vim 使用 'fileencoding' 的当前值（如设置）或者 'encoding' 值的 IANA 的推荐名字来设置。
对某些 'buftype' 类型，总使用 'encoding'。会根据选中的文档编码相应的设置 'fileencoding' 的值。

自动检测适用于所有 encoding-names 指定的编码，但 TOhtml 目前只自动使用那些广为浏览器支持的编码，对缺省不自动检测的特定编码，你可以强制其检测（见下面的选项）。IANA 名字可见 http://www.iana.org/assignments/character-sets。

备注，缺省所有 Unicode 编码所生成的 HTML 会转化为不带 BOM 的 UTF-8，这是 W3C 建议的：

```
http://www.w3.org/International/questions/qa-choosing-encodings
http://www.w3.org/International/questions/qa-byte-order-mark
```

g:html_use_encoding

缺省：无，使用 'fileencoding' 的当前值的 IANA 名字，见上述。
要完全跳过自动字符集检测，设置 g:html_use_encoding 为你要使用的字符集。强烈建议对所有要放上网页服务器的内容设置此值为广为支持的编码，如 UTF-8：

```
:let g:html_use_encoding = "UTF-8"
```

如果不想产生指定字符集的 HTML 行，设置 g:html_use_encoding 为空字符串（不推荐）：

```
:let g:html_use_encoding = ""
```

要回到自动的机制，删除 g:html_use_encoding 变量：

```
:unlet g:html_use_encoding
```

g:html_encoding_override

缺省：无，autoload/tohtml.vim 包含 encoding-names 中提到名字的编码的转换。
如果用 g:html_use_encoding 指定的编码不在缺省的转换列表中，本选项可以使 2html.vim 检测到正确的 'fileencoding'。

这是个由字符集到编码的组对所构成的字典，它会替代 TOhtml 自动检测所生成的相应组对，或扩充新组对以支持新的字符集。

要检测 HTML 字符集 "windows-1252" 对应的编码为 "8bit-cp1252"，用：

```
:let g:html_encoding_override = {'windows-1252': '8bit-cp1252'}
```

g:html_charset_override

缺省：无，autoload/tohtml.vim 包含 encoding-names 中提到名字的和有广泛浏览器支持的转换。
本选项指定 2html.vim 不能自动检测的 'fileencoding' 或 'encoding' 所对应的 HTML 字符集。也可用来覆盖现有的编码到字符集的组对。例如，
TOhtml 缺省对所有 Unicode/UCS 编码使用 UTF-8。要用 UTF-16 和 UTF-32 代替之，用：

```
:let g:html_charset_override = {'ucs-4': 'UTF-32', 'utf-16': 'UTF-16'}
```

注意 UTF-32 或 UTF-16 编码的文档和已知至少一个主要浏览器有兼容问题。

g:html_font

缺省："monospace"
用 g:html_font 可以指定转换后的文档使用的字体或字体集。如果本选项设为字符串，以单引号括起该值。如果本选项设为列表，以单引号括起每个列表项目，然后以逗号连接列表。不管哪种方式，加入 "monospace" 作为后备的通用字体族名，然后把整个结果用作 font family（用 CSS 时）或 font face（不用 CSS 时）。例如：

```
" font-family: 'Consolas', monospace;
:let g:html_font = "Consolas"

" font-family: 'DejaVu Sans Mono', 'Consolas', monospace;
:let g:html_font = ["DejaVu Sans Mono", "Consolas"]
```

convert-to-XML convert-to-XHTML g:html_use_xhtml

缺省：0。
为 0 时，生成标准 HTML 4.01（如有可能，strict）。
为 1 时，生成 XHTML 1.0（XML 兼容的 HTML）。

```
:let g:html_use_xhtml = 1
```

5．语法文件附注 :syn-file-remarks

b:current_syntax-variable
Vim 把载入的语法的名字保存在 "b:current_syntax" 变量里。你可用来根据当前激活的是哪部语法来载入其它设定。例如：

```
:au BufReadPost * if b:current_syntax == "csh"
:au BufReadPost *   做一些事
:au BufReadPost * endif
```

ABEL abel.vim ft-abel-syntax

ABEL 高亮设置提供一些用户定义的选项。要打开这些选项，给相应的选项设置任意值。例如：

```
:let abel_obsolete_ok=1
```

可用 ":unlet" 关闭。例如：

```
:unlet abel_obsolete_ok
```

变量	高亮
abel_obsolete_ok	废弃的关键字接受为 statement，而非 error
abel_cpp_comments_illegal	不把 '//' 解释为行内注释的引导符

ADA

见 ft-ada-syntax

ANT ant.vim ft-ant-syntax

ant 语法文件缺省提供了 javascript 和 python 的语法高亮。其他脚本语言的语法高亮可以通过 AntSyntaxScript() 函数安装。它接受的第一个参数是标签名，而第二个是脚本语法文件名。例如：

```
:call AntSyntaxScript('perl', 'perl.vim')
```

会为下面的 ant 代码提供 Perl 的语法高亮

```
<script language = 'perl'><![CDATA[
    # 这里的一切都作为 perl 脚本高亮
]]></script>
```

mysyntaxfile-add 说明如何永久的安装脚本语言。

APACHE apache.vim ft-apache-syntax

Apache 语法文件为 Apache HTTP 服务器 2.2.3 版本提供语法高亮。

ASSEMBLY	asm.vim asmh8300.vim nasm.vim masm.vim asm68k ft-asm-syntax ft-asmh8300-syntax ft-nasm-syntax ft-masm-syntax ft-asm68k-syntax fasm.vim

匹配 "*.i" 的可以是 Progress 或者汇编文件。如果自动检测对你不工作，或者你从来不编辑 Progress，在你启动的 vimrc 里这么用：

```
:let filetype_i = "asm"
```

把 "asm" 替换成你实际使用的汇编语言。

许多类型的汇编语言都使用相同的文件扩展名。因而，你或者需要自己选择类型，或者在汇编文件里增加使 Vim 能识别的行。目前，包含了以下的语法文件：

asm	GNU 汇编 (缺省)
asm68k	Motorola 680x0 汇编
asmh8300	Hitachi H-8300 版本的 GNU 汇编
ia64	Intel Itanium 64
fasm	Flat 汇编 (http://flatassembler.net)
masm	Microsoft 汇编 (可能适用于任何 80x86)
nasm	Netwide 汇编
tasm	Turbo 汇编 (提供到 Penitum 和 MMX 的 80x86 操作码)
pic	PIC 汇编 (目前支持 PIC16F84)

最灵活的方式是在你的汇编文件里加上如下一行：

```
asmsyntax=nasm
```

把 "nasm" 换成实际的汇编语法的名字。该行必须是文件头五行中的一行。此文本前后不能紧接着非空白的其它文本。注意 指定 asmsyntax=foo 等价于在 modeline 中设置 ft=foo，如果两者有冲突，模式行的设置优先 (尤其，如果模式行上有 ft=asm，总是使用 GNU 的语法高亮，而不管 asmsyntax 如何设置)。

语法类型可以为某个缓冲区定制。方法是设置 b:asmsyntax 变量：

```
:let b:asmsyntax = "nasm"
```

如果没有设置 b:asmsyntax，不管是自动还是手动，就会使用全局变量 asmsyntax 的值。它可以看作是缺省的汇编语言：

```
:let asmsyntax = "nasm"
```

如果什么都没有定义，最后就假定是 "asm" 语法。

Netwide 汇编器 (nasm.vim) 可选高亮特性

要打开特性：

```
:let   {variable}=1|set syntax=nasm
```

要关闭特性：

```
:unlet {variable}  |set syntax=nasm
```

变量	高亮
nasm_loose_syntax	非正式的分析器允许的语法不视为错误 (取决于分析器；不推荐)
nasm_ctx_outside_macro	宏之外的上下文不视为错误
nasm_no_warn	潜在有危险的语法不视为 Todo 那样的警告

ASPPERL 和 ASPVBS ft-aspperl-syntax ft-aspvbs-syntax

*.asp 和 *.asa 文件可以是 Perl 或者 Visual Basic 脚本。因为很难检测，你可以设置两个全局变量，来告诉 Vim 你要用哪一个。Perl 脚本可用：

```
:let g:filetype_asa = "aspperl"
:let g:filetype_asp = "aspperl"
```

Visual Basic 可用：

```
:let g:filetype_asa = "aspvbs"
:let g:filetype_asp = "aspvbs"
```

BAAN baan.vim baan-syntax

baan.vim 提供 BaanIV 到 SSA ERP LN 发行版的 BaanC 的语法支持，用于 3 GL 和 4 GL 编程。支持很多的标准宏定义/常数。

如果你在某个 .vimrc 里指定下面这行，不合编码标准的一些特殊错误会被报告：

```
let baan_code_stds=1
```

baan-folding

使用下面提到的变量，可以在不同层次上打开语法折叠（在 .vimrc 里设置）。源代码块和 SQL 上越复杂的折叠越需要 CPU。

要允许折叠并在函数级别上打开折叠：

```
let baan_fold=1
```

可以在源代码块级别上打开折叠，如 if、while、for，... 这里开始/结束关键字之前的缩进必须一致（空格不等于制表）。

```
let baan_fold_block=1
```

可以为内嵌的 SQL 块打开折叠，如 SELECT、SELECTDO、SELECTEMPTY，... 这里开始/结束关键字之前的缩进必须一致（空格不等于制表）。

```
let baan_fold_sql=1
```

注意：代码块级别的折叠可能产生许多小的折叠。建议在 .vimrc 里用 :set 设置选项 'foldminlines' 和 'foldnestmax'，在 .../after/syntax/baan.vim 里用 :setlocal 设置也可以（见 after-directory）。例如：

```
set foldminlines=5
set foldnestmax=6
```

BASIC basic.vim vb.vim ft-basic-syntax ft-vb-syntax

Visual Basic 和 "普通的" Basic 都使用扩展名 ".bas"。要检测使用的是哪一个，Vim 在文件的头五行检查字符串 "VB_Name"。如果没找到，文件类型将是 "basic"，不然就是 "vb"。带 ".frm" 扩展名的文件总被视为 Visual Basic 类型。

C c.vim ft-c-syntax

C 高亮的一些设置是可选的。要打开，给相应的变量赋任何值（包括零）。例如：

```
:let c_comment_strings = 1
:let c_no_bracket_error = 0
```

可用 :unlet 关闭。例如:

```
:unlet c_comment_strings
```

这里设置值为零是不行的!

一个替代办法是切换到 C++ 高亮:

```
:set filetype=cpp
```

变量	高亮
c_gnu	GNU gcc 专用的项目
c_comment_strings	注释里的字符串和数字
c_space_errors	行尾的空格和 <Tab> 之前的空格
c_no_trail_space_error	... 不包括行尾的空格
c_no_tab_space_error	... 不包括 <Tab> 之前的空格
c_no_bracket_error	不高亮 {}; 如果在 [] 里面视为错误
c_no_curly_error	不高亮 {}; 如果在 [] 和 () 里面视为错误; 除了出现在首列的 { 和 } 以外 缺省高亮它们, 否则找不到缺失的 ")"。
c_curly_error	通过寻找所有的对来高亮丢失的 }; 强制从文件首开始同步, 因而可能会较慢
c_no_ansi	不使用 ANSI 标准类型和常数
c_ansi_typedefs	... 但使用 ANSI 标准类型
c_ansi_constants	... 但使用 ANSI 标准常数
c_no_utf	不高亮字符串里的 \u 和 \U
c_syntax_for_h	*.h 文件使用 C 语法, 而不是 C++ 的, 用 objc 语法, 而不是 objcpp 的
c_no_if0	不把 "#if 0" 块高亮为注释
c_no_cformat	不高亮字符串里的 %-格式
c_no_c99	不高亮 C99 标准的项目
c_no_c11	不高亮 C11 标准的项目
c_no_bsd	不高亮 BSD 特定类型

如果 'foldmethod' 设为 "syntax", 那么 /* */ 注释和 { } 块成为折叠。如果你不想让注释成为折叠:

```
:let c_no_comment_fold = 1
```

"#if 0" 块也被折叠, 除非:

```
:let c_no_if0_fold = 1
```

如果你注意到往回滚动时, 高亮出现问题, 但 CTRL-L 重画又可以修正的话, 尝试设置 "c_minlines" 内部变量为较大的值:

```
:let c_minlines = 100
```

这使得语法同步在第一个显示行之前的 100 行开始。缺省值为 50 (如果设置了 c_no_if0, 则为 15)。使用较大的值的缺点是重画会变慢。

如果使用 "#if 0" / "#endif" 风格的注释高亮, 注意它只适用于 "#if 0" 出现在窗口顶部往上不超过 "c_minlines" 行的场合。如果你有很长的 "#if 0" 构造, 它不会被正确地高亮。

要匹配注释里的附加项目, 使用 cCommentGroup 簇。例如:

```
:au Syntax c call MyCadd()
:function MyCadd()
:  syn keyword cMyItem contained Ni
:  syn cluster cCommentGroup add=cMyItem
:  hi link cMyItem Title
:endfun
```

ANSI 常数使用 "cConstant" 组高亮。这包括 "NULL"、"SIG_IGN" 还有其它。但不包括，比如说，"TRUE"。因为它不在 ANSI 标准里。如果你觉得弄不清楚，删除 cConstant 高亮：

```
:hi link cConstant NONE
```

如果你看到 '{' 和 '}' 被高亮为错误，而实际并非如此的时候，复位 cErrInParen 和 cErrInBracket 的高亮。

如果想在你的 C 文件里使用折叠，可以在 'runtimpath' 的 "after" 目录下的一个文件里加上这些行。Unix 上，这会是 ~/.vim/after/syntax/c.vim.

```
syn sync fromstart
set foldmethod=syntax
```

CH ch.vim ft-ch-syntax

C/C++ 解释器。Ch 的语法高亮和 C 相似，它基于 C 语法文件。 c.vim 说明所有 C 可用的设置。

设置此变量，告诉 Vim 用 Ch 语法来处理 *.h 文件，而不是 C 或 C++ 语法：

```
:let ch_syntax_for_h = 1
```

CHILL chill.vim ft-chill-syntax

Chill 语法高亮和 C 类似。可用的设置见 c.vim 。此外，还有：

chill_space_errors	类似于 c_space_errors
chill_comment_string	类似于 c_comment_strings
chill_minlines	类似于 c_minlines

CHANGELOG changelog.vim ft-changelog-syntax

ChangeLog 支持行首的空格高亮。如果你不喜欢，在你的 .vimrc 里加入下行：

```
let g:changelog_spacing_errors = 0
```

下次你编辑 changelog 文件时，就会应用。你也可以使用 "b:changelog_spacing_errors" 来为每个缓冲区分别设置（在载入语法文件以前）。

你可以改变现在使用的高亮，例如把空格标记为错误：

```
:hi link ChangelogError Error
```

或者避免高亮：

```
:hi link ChangelogError NONE
```

这些会立即生效。

CLOJURE ft-clojure-syntax

缺省的语法组可以用 g:clojure_syntax_keywords 和 b:clojure_syntax_keywords 变量进行扩展。其值应为语法组名映射到定制标识符的 List 的 Dictionary ：

```
let g:clojure_syntax_keywords = {
    \ 'clojureMacro': ["defproject", "defcustom"],
    \ 'clojureFunc': ["string/join", "string/replace"]
```

```
\ }
```

合法的语法组名可参见 Closure 语法脚本。

如果置位 buffer-variable b:clojure_syntax_without_core_keywords ，只匹配语言常数和特殊表达式。

置位 g:clojure_fold 允许通过语法引擎来折叠 Clojure 代码。任何超过一行的 list、 vector 或 map 会使用标准 Vim fold-commands 进行折叠。

请 注意 此选项和重定义括号语法区域的脚本不兼容，如 rainbox-parentheses 插件。

缺省关闭此选项。

```
" 缺省
let g:clojure_fold = 0
```

COBOL cobol.vim ft-cobol-syntax

COBOL 高亮对传统的代码和新开发的代码有不同的需要。这来自于需求的差异 (维护还是开发)，以及一些其它因素。要使用传统代码的高亮，在 .vimrc 里加上此行:

```
:let cobol_legacy_code = 1
```

要再次关闭，可用:

```
:unlet cobol_legacy_code
```

COLD FUSION coldfusion.vim ft-coldfusion-syntax

ColdFusion 有 HTML 注释的自己的版本。要打开 ColdFusion 风格的注释高亮，在你的启动文件里加上此行:

```
:let html_wrong_comments = 1
```

ColdFusion 语法文件是基于 HTML 的语法文件的。

CPP cpp.vim ft-cpp-syntax

多数设置同 ft-c-syntax 。

变量	特色
cpp_no_cpp11	不高亮 C++11 标准项目
cpp_no_cpp14	不高亮 C++14 标准项目

CSH csh.vim ft-csh-syntax

这里讨论名为 "csh" 的外壳。 注意 有的系统实际使用的是 tcsh。

要检测一个文件是 csh 还是 tcsh 异常困难。有的系统把 /bin/csh 符号链接到 /bin/tcsh，使得区别 csh 和 tcsh 几乎不可能。如果 VIM 猜错，你可以自己设置 "filetype_csh" 变量。要使用 csh: g:filetype_csh

```
:let g:filetype_csh = "csh"
```

要使用 tcsh:

```
:let g:filetype_csh = "tcsh"
```

带 tcsh 扩展名的脚本或者 tcsh 的标准文件名 (.tcshrc、tcsh.tcshrc、tcsh.login) 会使用文件类型 tcsh。其它的 tcsh/csh 脚本也会被识别为 tcsh， 除非 存在 "filetype_csh" 变量。如果存在 "filetype_csh" 变量，文件类型会设为该变量的值。

CYNLIB cynlib.vim ft-cynlib-syntax

Cynlib 文件是使用 Cynlib 类库的 C++ 文件。它们使用 C++ 来为硬件建模和模拟。通常 Cynlib 文件使用 .cc 或 .cpp 扩展名，这使得它们和普通的 C++ 文件难以区别。因而，要使用 Cynlib 为 .cc 文件高亮，在你的 .vimrc 文件里加上此行:

```
:let cynlib_cyntax_for_cc=1
```

cpp 文件也类似 (该扩展名通常为 Windows 使用)

```
:let cynlib_cyntax_for_cpp=1
```

要再次关闭，可用:

```
:unlet cynlib_cyntax_for_cc
:unlet cynlib_cyntax_for_cpp
```

CWEB cweb.vim ft-cweb-syntax

匹配 "*.w" 的文件可为 Progress 文件或者 cweb 文件。如果自动检测不行，或者你从来不编辑 Progress，可以在启动的 vimrc 文件里用:

```
:let filetype_w = "cweb"
```

DART dart.vim ft-dart-syntax

Dart 是面向对象，强类型的，有类定义的，垃圾回收的语言，用于开发移动、台式、网站和后台应用。Dart 使用来源于 C、Java 和 JavaScript 的 C 风格的语法，提供来自 Smalltalk、Python、Ruby 和其它语言的特性。

关于语言和其开发环境的详情，可见官方 Dart 语言网站 https://dart.dev

dart.vim 语法检测并高亮 Dart 语句、保留字、类型声明、存储类、条件句、循环、插值和注释。没有 Flutter 或其它 Dart 框架的习惯用法的支持。

改动，修正? 请这样提交议题或拉取请求:

https://github.com/pr3d4t0r/dart-vim-syntax/

DESKTOP desktop.vim ft-desktop-syntax

该语法文件的主要目的是根据 freedesktop.org 标准:
http://standards.freedesktop.org/desktop-entry-spec/latest/
来高亮 .desktop 和 .directory 文件。但实际上，几乎没人完全实现了标准。所以，它实际被用来高亮所有的 Unix ini 文件。如果你希望强制使用符合标准的严格的高亮方式，在 vimrc 文件放上:

```
:let enforce_freedesktop_standard = 1
```

DIFF diff.vim

比较高亮通常会寻找翻译的头部。如果文件里有很长的行，这会变慢。这样关闭翻译:

```
:let diff_translations = 0
```

另见 diff-slow 。

DIRCOLORS dircolors.vim ft-dircolors-syntax

dircolors 工具的高亮定义有一个选项。它的存在是为了和 Slackware GNU/Linux 发布版本里的该命令兼容。它增加了一些多数版本忽略的关键字。在 Slackware 系统中，该工具接受这些关键字并在处理中应用。要允许使用 Slackware 关键字，在你的启动文件里加入如下一行:

```
let dircolors_is_slackware = 1
```

DOCBOOK docbk.vim ft-docbk-syntax docbook
DOCBOOK XML docbkxml.vim ft-docbkxml-syntax
DOCBOOK SGML docbksgml.vim ft-docbksgml-syntax

有两种类型的 DocBook 文件: SGML 和 XML。要指定使用那种类型，需要设置 "b:docbk_type" 变量。如果 Vim 能够识别，它会自动设置该变量。如果 Vim 猜不到，缺省的类型是 XML。
你可以手动设置:

```
:let docbk_type = "sgml"
```

或者:

```
:let docbk_type = "xml"
```

你需要在载入语法文件前作如此设置，这有点复杂。
更简单的方法是设置文件类型为 "docbkxml" 或 "docbksgml":

```
:set filetype=docbksgml
```

或:

```
:set filetype=docbkxml
```

可以指定 DocBook 版本:

```
:let docbk_ver = 3
```

如无指定假定为 4。

DOSBATCH dosbatch.vim ft-dosbatch-syntax

DOS 批处理文件的高亮有一个选项。它和 Windows 2000 的命令解释器的新扩展有关，可以通过 dosbatch_cmdextversion 变量控制。Windows NT 上，它应该取值为 1，Windows 2000 应该为 2。下面这行可以选择你想要的版本:

```
:let dosbatch_cmdextversion = 1
```

如果没有定义该变量，缺省设为 2，以支持 Windows 2000 版本。

第二个选项控制 *.btm 文件是否被识别为 "dosbatch" 类型 (MS-DOS 批处理文件) 或 "btm" 类型 (4DOS 批处理文件)。缺省使用后者。下行可以让你选择前者:

```
:let g:dosbatch_syntax_for_btm = 1
```

如果此变量无定义或为零，选择 btm 语法。

DOXYGEN

doxygen.vim doxygen-syntax

Doxygen 使用特殊的文档格式生成代码文档 (和 Javadoc 类似)。本语法脚本为 c、cpp、idl 和 php 文件加入 Doxygen 的高亮，Java 应该也能用。

有几个办法可以打开 Doxygen 格式。首先，在文件的模式行上可以给 syntax 加入 ".doxygen"，进行显式设置。示例:

```
:set syntax=c.doxygen
```

或

```
// vim:syntax=c.doxygen
```

对于 C、C++、C#、IDL 和 PHP 文件，也可以设置全局或局部于缓冲区变量 load_doxygen_syntax 来自动完成这个操作。在 .vimrc 里加上

```
:let g:load_doxygen_syntax=1
```

有一些变量影响语法高亮，它们大多和非标准的高亮选项有关。

变量	缺省	效果
g:doxygen_enhanced_color g:doxygen_enhanced_colour	0	Doxygen 注释使用非标准的高亮。
doxygen_my_rendering	0	关闭 HTML 粗体、斜体和 html_my_rendering 下划线的高亮。
doxygen_javadoc_autobrief	1	如果为 0，关闭 Javadoc autobrief 色彩高亮。
doxygen_end_punctuation	'[.]'	匹配 brief 结束标点的正规表达式。

还有一些高亮组值得提提，你可以对它们进行配置。

高亮	效果
doxygenErrorComment	如果 code、verbatim 或 dot 段里丢失标点，注释尾部的颜色。
doxygenLinkError	如果丢失 \link 段的 \endlink，注释尾部的颜色。

DTD

dtd.vim ft-dtd-syntax

DTD 语法高亮缺省是大小写敏感的。如果不想如此，在你的启动文件里加入以下一行:

```
:let dtd_ignore_case=1
```

DTD 语法会把未知的标签高亮为错误。如果这很讨厌，在执行 dtd.vim 语法文件之前，可以用以下设置来关闭：

```
:let dtd_no_tag_errors=1
```

参数实体 (parameter entity) 的名字使用 'Type' 高亮组高亮，标点和 '%' 使用 'Comment'。参数实体的实例使用 'Constant' 高亮组高亮，定界符 % 和 ; 则使用 'Type' 高亮组。以下设置可以关闭这些设定：

```
:let dtd_no_param_entities=1
```

xml.vim 也包含了 DTD 语法文件，用以高亮内嵌的 dtd。

EIFFEL eiffel.vim ft-eiffel-syntax

尽管 Eiffel 不是大小写敏感的，它的风格指南鼓励如此，而且语法高亮文件也鼓励这样使用。这使得同一类名的高亮可以不同。如果你想关闭大小写敏感的语法高亮，在启动文件里加上这行：

```
:let eiffel_ignore_case=1
```

大小写对类名和注释里的 TODO 标记仍然有影响。

相反，要更严格的检查，加入以下诸行之一：

```
:let eiffel_strict=1
:let eiffel_pedantic=1
```

设置 eiffel_strict 只会捕获以下五个预定义的单词错误的大小写："Current"、"Void"、"Result"、"Precursor" 和 "NONE"。用于警告它们可能被不小心用作特性或类名。

设置 eiffel_pedantic 会非常严格地遵循 Eiffel 风格指南 (例如，大小写字母的任意混合和其它过时的大写关键字的方法都会被捕获)。

如果希望使用小写版本的 "Current"、"Void"、"Result" 和 "Precursor"，可以用

```
:let eiffel_lower_case_predef=1
```

而不用完全关闭大小写敏感的高亮方式。

一些编译器里已经能够实验性地处理 ISE 推荐的新的创建语法，要打开此语法：

```
:let eiffel_ise=1
```

最后，有的供应商支持十六进制的常数。在你的启动文件里加上这行就可以处理它们

```
:let eiffel_hex_constants=1
```

EUPHORIA euphoria3.vim euphoria4.vim ft-euphoria-syntax

Euphoria 有两种语法高亮文件。一种用于 Euphoria 3.1.1 版本，这是缺省的语法高亮

文件，另一种用于 Euphoria 4.0.5 或更新版本。

Euphoria 3.1.1 版本 (http://www.rapideuphoria.com/) 对开发 DOS 平台上的应用还是需要的，Euphoria 4 (http://www.openeuphoria.org/) 不再支持。

以下文件扩展名被自动识别为 Euphoria 文件类型:

```
*.e, *.eu, *.ew, *.ex, *.exu, *.exw
*.E, *.EU, *.EW, *.EX, *.EXU, *.EXW
```

要选择 Euphoria 的语法文件，同时也自动识别 *.e 和 *.E 文件扩展名为 Euphoria 文件类型，在启动文件里加入以下一行:

```
:let filetype_euphoria="euphoria3"
```

或

```
:let filetype_euphoria="euphoria4"
```

ERLANG erlang.vim ft-erlang-syntax

Erlang 是 Ericsson 开发的函数编程语言。下列文件后缀被识别为 Erlang 文件: erl、hrl、yaws。

缺省打开 BIF (内建函数) 的高亮。要关闭之，在 .vimrc 里加上:

```
:let g:erlang_highlight_bifs = 0
```

要打开某些特殊原子 (atom) 的高亮，在 .vimrc 里加上:

```
:let g:erlang_highlight_special_atoms = 1
```

FLEXWIKI flexwiki.vim ft-flexwiki-syntax

FlexWiki 是基于 ASP.NET 的 wiki 包，从 http://www.flexwiki.com 可以获得。
备注: 此网站已不可用，Wikipedia 指出开发已在 2009 年停止。

多数常用的 FlexWiki 语法元素都提供了语法高亮。本文件类型插件脚本设置了若干局部于缓冲区的选项，使 FlexWiki 页面的编辑更加便捷。因为 FlexWiki 把换行符看成新段落的开始，所以本插件设置 'tw'=0 (无限行长)，置位 'wrap' (回绕长行而不使用水平滚动)，也置位 'linebreak' (在 'breakat' 包含的字符而不是在屏幕最后一个字符上回绕)，等等。本插件也包含了一些键盘映射，但它们缺省是关闭的。

如果你打开键盘映射，"j" 和 "k" 和光标键就会在显示行间上下移动。为此，在 .vimrc 里加入:

```
:let flexwiki_maps = 1
```

FORM form.vim ft-form-syntax

FORM 文件里的语法元素的色彩方案使用缺省模式: Conditional、Number、Statement、Comment、PreProc、Type 和 String。它遵循 1991 年版荷兰 CAN 组织

J.A.M. Vermaseren 编著的语言说明书 'Symbolic Manipulation with FORM'。

如果你想自己增加对缺省颜色的修改，需要重新定义以下语法组：

- formConditional
- formNumber
- formStatement
- formHeaderStatement
- formComment
- formPreProc
- formDirective
- formType
- formString

注意 缺省，form.vim 语法文件使用相同的语法组里实现 FORM 预处理器命令和指令。

预定义的 FORM 增强色彩模式可以用来区别头部语句和 FORM 程序本体的语句。要激活此模式，在你的 vimrc 文件里定义以下变量

```
:let form_enhanced_color=1
```

在深色 gvim 显示中，增强模式也利用了附加的色彩特性。这里，语句 (formStatement) 使用 LightYellow 而不是 Yellow。而条件句 (formConditional) 使用 LightBlue，以达到更好的区分度。

FORTRAN fortran.vim ft-fortran-syntax

缺省高亮方式和方言

缺省使用适合 Fortran 2008 的高亮方式。这样的选择应该适用于绝大多数的用户，因为 Fortran 2008 几乎是以前所有版本的超集 (Fortran 2003、95、90 和 77)。

Fortran 源代码形式

Fortran 代码可以使用固定宽度的源代码，也可以使用自由形式。注意 如果形式设置错误，语法高亮也不会正确。

创建新的 Fortran 文件时，语法脚本假定使用的是固定宽度的源代码。如果你总是使用自由形式，那么在你的 .vimrc 里，把

```
:let fortran_free_source=1
```

放在 :syntax 命令前。如果你总是使用固定宽度格式，那么在你的 .vimrc 里，把

```
:let fortran_fixed_source=1
```

放在 .syntax 命令前。

如果以非标准的形式让源代码的格式取决于文件扩展名，那么最方便的方法是在 ftplugin 文件里设置 fortran_free_source。关于 ftplugin 文件的更多信息，见 ftplugin 。注意，要使之工作，你需要在 .vimrc 文件的 "syntax on" 命令之前加上 "filetype plugin indent on" 命令。

编辑已有的 Fortran 文件时，如果定义了 fortran_free_source 变量，语法脚本假定使用自由形式，如果定义了 fortran_fixed_source 变量，则假定使用固定宽度形式。如果两者都没有，语法脚本检验通用于 ifort、gfortran、Cray、NAG 和 PathScale 编译器惯例的文件扩展名 (.f、.for、.f77 为固定宽度，.f90、.f95、.f03、.f08 为自由形式)。如果都不行，脚本再检查文件的头 500 行的头 5 列。如果没有发现自由形式的迹

象，那么假定文件为固定宽度形式的源文件。这个算法应该在绝大多数情况下适用。有的情况下，比如文件的开始有 500 行或更多完整行的注释，该脚本也许会错误地认为该 Fortran 代码使用的是固定宽度形式。如果此种情况真的发生，只要在前 500 行的头 5 列里的任何地方加上一个非注释的语句，然后保存 (:w) 并重新读入 (:e!) 文件就可以了。

Fortran 文件里的制表键

Fortran 标准不识别制表键。在固定宽度格式的 Fortran 源代码里，制表不是一个好主意，因为需要固定的列边界。因此，制表被识别为错误。不过，有的程序员喜欢使用制表。如果你的 Fortran 文件里包含制表，那么需要在 .vimrc 里使用这样的命令来设置变量 fortran_have_tabs

```
:let fortran_have_tabs=1
```

并把它放在 :syntax 命令之前。不幸的是，制表的使用意味着语法文件无法识别不正确的边界。

Fortran 文件的语法折叠

如果你希望使用 foldmethod=syntax，那么你需要先使用命令设置变量 fortran_fold

```
:let fortran_fold=1
```

来指示语法脚本为程序单元定义折叠区域。程序单元 (program unit) 包括程序语句开始的主程序、子例程、函数子程序、块数据子程序、接口块和模块。如果你也使用如下命令设置变量 fortran_fold_conditionals

```
:let fortran_fold_conditionals=1
```

那么也会为 do 循环、if 块和 select case 构造定义折叠区域。如果你同时使用如下命令设置变量 fortran_fold_multilinecomments

```
:let fortran_fold_multilinecomments=1
```

那么还会为三行或更多连续的注释定义折叠区域。注意 定义折叠区域会使大文件变慢。

如果设置了 fortran_fold 和可能的 fortran_fold_conditionals 与/或 fortran_fold_multilinecomments，vim 会在你使用 foldmethod=syntax 时折叠文件。两个程序单元之间的注释或空行不会被折叠，因为它们不被视作任何一个程序单元的一部分。

更精确的 Fortran 语法

如果使用如下命令设置变量 fortran_more_precise

```
:let fortran_more_precise=1
```

那么语法高亮会更精确，但也更慢。特别是，能够识别 do、goto 和算术 if 语句使用的语句标签，还有 do、if、select、或 forall 构造结尾的构造名。

非缺省的 Fortran 方言

本语法脚本支持两种 Fortran 方言: f08 和 F。缺省高亮方式 (f08) 基本令人满意。一些 2008 标准宣告过时或删除的传统结构被高亮为 todo 项目。

如果你使用 F，设置合适的方言的优点包括，F 排除的传统特性会被高亮为 todo 项目；还有，总是假定使用自由形式的源程序。

有几个方式选择方言。如果你所有的 Fortran 文件使用相同的方言，在你的 .vimrc 文件里 syntax 命令之前设置 fortran_dialect。可以接受的 fortran_dialect 的值是大小写敏感的，而且必须是 "f08" 或 "F"。不合法的 fortran_dialect 的值被忽略。

如果源代码的格式取决于文件扩展名，那么最方便的方法是在 ftplugin 文件里设置一个缓冲区局部变量。关于 ftplugin 文件的详情，见 ftplugin 。例如，如果你使用 .f90 扩展名的所有 Fortran 文件都使用 F 子集，那么在你的 ftplugin 文件里应该包含代码

```
let s:extfname = expand("%:e")
```

```
if s:extfname ==? "f90"
    let b:fortran_dialect="F"
else
    unlet! b:fortran_dialect
endif
```

注意 只有在你的 .vimrc 文件的 "syntax on" 命令前加上了 "filetype plugin indent on" 命令，才能使这段代码工作。

如果文件扩展名还不能唯一确定方言，那么你需要更精细的控制。可以为每个文件定制方言。方法是在文件的头三行内加上指令 "fortran_dialect=xx" (其中 xx=F 或 f08)。例如，你较老的 .f 文件可能用传统代码编写，但较新的那些可能使用 F，那么你需要在后者每个文件的头三行内加上以下形式的 Fortran 注释，以便标识

```
! fortran_dialect=F
```

使用较早的语法版本的用户，可能会设置 fortran_dialet 为现已废弃的值 "f77"、"f90"、"f95" 或 "elf"。这些设置都会被悄悄地当作 "f08" 处理。"elf" 用户可能想试试 "F" 也许会更好。

syntax/fortran.vim 脚本包含内嵌注释，说明如何为某些行加上注释和/或去掉注释来 (a) 识别一些非标准厂商自定的内在过程 (intrinsic)，(b) 使某些 2008 标准删除或废止的功能不采用 todo 项目高亮。

限制
括号检查不能发现过少的闭括号。也不能识别 Hollerith 字符串。有些关键字可能高亮不正确，因为 Fortran90 没有保留字。

更多关于 Fortran 的信息可见 ft-fortran-indent 和 ft-fortran-plugin 。

FVWM 配置文件 fvwm.vim ft-fvwm-syntax

要使得 Vim 识别不符合模式 fvwmrc 或 fvwm2rc 的 Fvwm 配置文件，你需要在你 myfiletypes.vim 文件里放入附加适合你系统的模式。对于这些模式，你需要设置变量 "b:fvwm_version" 为 Fvwm 的主版本号，同时设置 'filetype' 选项为 fvwm。

例如，要使 Vim 识别 /etc/X11/fvwm2/ 里的所有文件为 Fvwm2 的配置文件，可以这样设置:

```
:au! BufNewFile,BufRead /etc/X11/fvwm2/*  let b:fvwm_version = 2 |
                                        \ set filetype=fvwm
```

如果你想让 Vim 高亮所有的合法颜色名，告诉它如何找到在你的系统上颜色数据库 (rgb.txt)。设置 "rgb_file" 指向该文件就可以了。假如你的颜色数据库在 /usr/X11/lib/X11/，加入下行

```
:let rgb_file = "/usr/X11/lib/X11/rgb.txt"
```

到你的 .vimrc 文件里。

GSP gsp.vim ft-gsp-syntax

GSP 页面的缺省色彩风格由 html.vim 定义，而 Java 代码 (在 Java 标签里或反引号

之间的内含代码）的颜色由 java.vim 定义。 html.vim 里定义的以下 HTML 组在这里被重新定义，以配合高亮的内含 (inline) Java 代码：

```
htmlString
htmlValue
htmlEndTag
htmlTag
htmlTagN
```

多数你看到内含 Java 代码的地方，高亮应该没问题，但在一些特殊情况下可能有问题。要加入其它可包含内含 Java 代码而高亮不正确的 HTML 组，只要从 html.vim 里把你想要的行复制出来，并在 contains 子句里加上 gspJava 就可以了。

反引号里的内含 Java 使用 htmlError 组高亮，使之更易看清。

GROFF groff.vim ft-groff-syntax

groff 语法文件是 nroff.vim 的包装，见该标题下的说明参阅使用和配置的示例。提供这个包装的目的是通过 modeline 或个人的文件类型文件（见 filetype.txt ）来设置文件类型，从而设置 groff 专用的语法扩展。

HASKELL haskell.vim lhaskell.vim ft-haskell-syntax

Haskell 语法文件支持普通的 Haskell 代码和文学的 (literate) Haskell 代码，后者包括 Bird 风格和 Tex 风格。Haskell 语法高亮也能高亮 C 预处理指令。

如果你想高亮定界符（适用于浅色背景），在 .vimrc 里加上：

```
:let hs_highlight_delimiters = 1
```

要把 True 和 False 识别为关键字而不是普通的标识符，加上：

```
:let hs_highlight_boolean = 1
```

要把基本类型的名字识别为关键字：

```
:let hs_highlight_types = 1
```

要把更多相对常用的类型识别为关键字：

```
:let hs_highlight_more_types = 1
```

如果想高亮调试函数的名字，在你的 .vimrc 里加上：

```
:let hs_highlight_debug = 1
```

Haskell 语法高亮也高亮 C 预处理指令，但非法的 # 开始的指令被标记为错误。这和 Haskell 的操作符语法有冲突，因为它们可能会用 # 开始。如果你想高亮这些为操作符而不是错误，在 .vimrc 里加入：

```
:let hs_allow_hash_operator = 1
```

文学的 Haskell 代码的语法高亮会试图自动猜测你的文学 Haskell 代码是否包含 Tex 标记，并相应地高亮 Tex 构造或什么也不做。要在全局改变此行为，在你的 .vimrc 文件里放上

```
:let lhs_markup = none
```

就可完全关闭高亮。或者

```
:let lhs_markup = tex
```

强制使用 Tex 标记风格的高亮。更灵活的方法是使用该变量的局部于缓冲区的版本，例如

```
:let b:lhs_markup = tex
```

会强制为特定的缓冲区使用 TeX 高亮。必须在该缓冲区打开语法高亮或者载入文件前设

置。

HTML html.vim ft-html-syntax

HTML 文件里，标签的色彩方案工作方式如下。

开放标签的 <> 和关闭标签的 </> 的颜色不同。这是有意的！开放标签使用 'Function' 色，而关闭标签使用 'Type' 色（察看 syntax.vim 了解它们是如何为你定义的）。

已知的标签名使用 C 语句（Statement）的色彩。未知的标签名分别和相应的 <> 或 </> 颜色相同，以便纠错。

注意 这同样适用于参数（或属性）名。已知的属性名和未知的颜色不同。

一些 HTML 标签用于改变文本的显示。html.vim 语法色彩文件识别以下的标签，并相应地改变普通文本的显示方式：<B> <I> <U> <EM> <STRONG>（<EM> 是 <I> 的别名，而
<STRONG> 是 <B> 的别名），<H1> - <H6>，<HEAD>，<TITLE> 和 <A>（<A> 必须在作为链
接，即包含了 href，才会如此。如 <A href="somefile.html">）。

如果你想改变文本显示的方式，必须重定义以下的语法组：

- htmlBold
- htmlBoldUnderline
- htmlBoldUnderlineItalic
- htmlUnderline
- htmlUnderlineItalic
- htmlItalic
- htmlTitle 设定标题
- htmlH1 - htmlH6 设定标题头部（header）的文本

要使得重定义能够工作，你必须重定义所有的组，最后两组（htmlTitle 和 htmlH[1-6] 可选）可以除外。在你的 vimrc（这是根据初始化时读入文件的顺序）里定义下面的变量

```
:let html_my_rendering=1
```

要想看一个例子，下载 http://www.fleiner.com/vim/download.html 的 mysyntax.vim 文件。

在你的 vimrc 文件里加上这行，可以屏蔽这种显示方式：

```
:let html_no_rendering=1
```

HTML 注释相当特别（详情见 HTML 的参考文档），此语法高亮方案会高亮所有的错误。不过，如果你喜欢不太正确（以 <!-- 开始并以 --> 结束）的风格，可以定义

```
:let html_wrong_comments=1
```

HTML 文档的内嵌（embedded）JavaScript 和 Visual Basic 使用 'Special' 高亮，而其中的语句、注释、字符串等使用标准的编程语言的颜色。注意 现在只支持 JavaScript 和 Visual Basic，还没有加入其它的脚本语言。

内嵌和内含（inline）的层叠样式表（CSS）也被高亮。

有多种 html 预处理器语言，html.vim 的编写方式使得包含它非常容易。要想这么做，只要在相应语言的语法高亮文件里加入如下两行（该例子来自 asp.vim 文件）就可以了：

```
runtime! syntax/html.vim
syn cluster htmlPreproc add=asp
```

现在你只需要把所有包含预处理语言的区域项目加到 htmlPreproc 簇里就可以了。

HTML/OS（Aestiva 提供）　　htmlos.vim　ft-htmlos-syntax

HTML/OS 的色彩高亮方案工作方式如下：

缺省为函数和变量名使用相同的颜色，因为 VIM 并不为 Function 和 Identifier 指定不同的颜色。要改变这一点（如果你希望函数名能用不同的颜色加以识别的话，建议使用），你需要在 ~/.vimrc 里加入下行：

```
:hi Function term=underline cterm=bold ctermfg=LightGray
```

当然如果你愿意，ctermfg 可以使用别的颜色。

HTML/OS 碰到的另一个问题是没有特殊的指示 HTML/OS 编码的文件类型。打开文件并用下面的方法打开 HTML/OS 语法，你就可以改变这一点：

```
:set syntax=htmlos
```

最后要提醒一下，启动 HTML/OS 代码块的打开和关闭字符序列分别是 << 或 [[和 >> 或]]。

IA64　　ia64.vim　intel-itanium　ft-ia64-syntax

为 Intel Itanium 64 汇编语言提供高亮。 asm.vim 说明如何识别该文件类型。

要识别 *.inc 文件为 IA64，在你的 .vimrc 文件里加入这行：

```
:let g:filetype_inc = "ia64"
```

INFORM　　inform.vim　ft-inform-syntax

Inform 高亮包含 Inform 库提供的符号，因为多数程序大量使用它们。如果不希望高亮这些库里的符号，在你的 vim 启动文件里加入：

```
:let inform_highlight_simple=1
```

缺省假定 Inform 程序面向 Z 机器。并适当地高亮 Z 机器的汇编语言符号。如果期望程序面向 Glulx/Glk 环境，你需要在启动文件的序列中加入：

```
:let inform_highlight_glulx=1
```

这样就只高亮 Glulx 操作码，并把 glk() 加到高亮的系统函数集合里。

Inform 编译器遇到特定已废弃的关键字时，会标记它们为错误。通常，Vim 也把这些关键字标为错误。如果不想高亮这些错误，你必须在启动文件的序列中加入：

```
:let inform_suppress_obsolete=1
```

缺省的高亮设置符合编译器版本 6.30 和库版本 6.11 的语言特性。如果你使用较老的 Inform 开发环境，可能需要在启动文件的序列中加入：

```
:let inform_highlight_old=1
```

IDL idl.vim idl-syntax

IDL (Interface Definition Language，接口定义语言) 文件用于定义 RFC 调用。Microsoft 的领地里也用来定义 COM 接口和调用。

IDL 的结构足够简单，所以可以分析完整语法，而不需要一些启发式的猜测。结果很大，甚至可能有些重复工作，但看来能用了。

这里有一些 Microsft 的 idl 文件扩展。有些可以通过 idl_no_ms_extensions 关闭。

更复杂的扩展可以通过定义 idl_no_extensions 来关闭。

变量	效果
idl_no_ms_extensions	关闭一些 Microsoft 专用的扩展
idl_no_extensions	关闭复杂扩展
idlsyntax_showerror	显示 IDL 错误 (可能过于侵略性，但很有用)
idlsyntax_showerror_soft	使缺省的错误颜色柔和一些

JAVA java.vim ft-java-syntax

java.vim 语法高亮文件提供若干选项：

在 Java 1.0.2 里，小括号里不可能有大括号，所以这被标识为错误。但从 Java 1.1 开始，这是合法的 (无名类的使用)，因而它不再标为错误。如果你喜欢旧的方式，在 vim 启动文件里加入下行：

```
:let java_mark_braces_in_parens_as_errors=1
```

所有 java.lang.* 的标识符在所有的类里都是可见的。要高亮它们，可用：

```
:let java_highlight_java_lang_ids=1
```

如果你从 http://www.fleiner.com/vim/download.html 下载 javaid.vim 脚本，你也可以高亮大多数标准 java 包里的标识符。
如果你只想高亮特定包里的标识符，比如说 java.io，可以用：

```
:let java_highlight_java_io=1
```

察看 javaid.vim 文件，可以得到它支持的所有包的列表。

函数名不会高亮，因为找到函数的方法取决于你如何写 Java 代码。语法文件知道两种可以高亮函数的方法：

如果你的函数定义总是使用一个制表、8 个空格或者 2 个空格的缩进，可以设置

```
:let java_highlight_functions="indent"
```

不过，如果你遵循 Java 指南里函数和类的命名规则 (关于大小写)，就可以用

```
:let java_highlight_functions="style"
```

如果两个选项都不合适，但你仍然期望高亮函数声明，修改 java.vim 里的定义或者创建你自己的 java.vim。你自己的版本应该包含原来的版本，并增加高亮函数的代码。

Java 1.1 里，只应该用函数 System.out.println() 和 System.err.println() 来进行调试。所以你可以用不同的方式高亮调试语句。要这么做，你必须在启动文件里加上以下定义：

```
:let java_highlight_debug=1
```

结果那些语句会被高亮为 'Special' 字符序列。如果你喜欢用不同的方式分别高亮，必须为以下各组定义新的高亮：

Debug、DebugSpecial、DebugString、DebugBoolean、DebugType

它们分别用来高亮语句本身，调试字符串里的特殊字符、字符串、布尔常量和类型 (this, super)。我本人喜欢给语句设置别的背景。

Javadoc 是一个程序，它接受特殊的 Java 程序文件里的注释，并创建 HTML 页面。标准的配置会以类似于 HTML 文件 (见 html.vim) 方式高亮该 HTML 代码，你甚至可以在代码里加入 Javascript 和 CSS (见下)。但有四处不同：

1. 标题 (第一个后面有若干空白跟随的 '.' 或第一个 '@' 之前的所有字符) 使用不同的颜色 (要改变其颜色，修改 CommentTitle 组)。
2. 文本使用 'Comment' 高亮。
3. HTML 注释使用 'Special' 高亮。
4. 特殊的 Javadoc 标签 (@see、@param、...) 用 Special 高亮。 而 (@see、@param、@exception 的)参数则使用 Function 高亮。

要关闭该特性，在你的启动文件里加入该行：

```
:let java_ignore_javadoc=1
```

如果你使用上述的特殊 Javadoc 注释高亮方式，你也可以打开 Javascript、Visual Basic 脚本和内嵌 CSS (样式表) 的特殊高亮。只有在你实际有包含 Javascript 或内嵌 CSS 的 Javadoc 注释时，这才有意义。要使用的选项分别是

```
:let java_javascript=1
:let java_css=1
:let java_vb=1
```

要以不同的颜色高亮嵌套的括号，分别定义 javaParen、javaParen1 和 javaParen2 的颜色。比如用

```
:hi link javaParen Comment
```

或

```
:hi javaParen ctermfg=blue guifg=#0000ff
```

如果你注意到往回滚动时，高亮出现问题，但 CTRL-L 重画又可以修正的话，尝试设置 "java_minlines" 内部变量为较大的值：

```
:let java_minlines = 50
```

这使得语法同步在第一个显示行之前的 50 行开始。缺省值为 10。使用较大的值的缺点是重画会变慢。

JSON

json.vim ft-json-syntax

json 语法文件缺省提供带隐藏支持的语法高亮。要关闭隐藏：

```
let g:vim_json_conceal = 0
```

要关闭错误的语法高亮：

```
let g:vim_json_warnings = 0
```

LACE

lace.vim ft-lace-syntax

Lace (Language for Assembly of Classes in Eiffel, Eiffel 类整合语言) 对大小写不敏感，但风格指南不是这么建议的。如果你喜欢对大小写不敏感的高亮，在启动文件里定义 vim 变量 'lace_case_insensitive'：

```
:let lace_case_insensitive=1
```

LEX lex.vim ft-lex-syntax

Lex 使用强力攻击 (brute-force) 的方式进行同步，因为 "^%%$" 段定界符没有提供任何关于后续段的提示。因而，如果用户有同步问题的话 (比如使用很大的 lex 文件)，他/她可以尝试改变

```
:syn sync minlines=300
```

的值。

LIFELINES lifelines.vim ft-lifelines-syntax

要把废弃函数高亮为错误，在 .vimrc 中加入：

```
:let g:lifelines_deprecated = 1
```

LISP lisp.vim ft-lisp-syntax

Lisp 语法高亮提供两个选项：

```
g:lisp_instring : 如果存在，那么 "(...)" 字符串会被高亮，就像字符串里
                  的内容是 Lisp 代码一样。对 AutoLisp 有用。
g:lisp_rainbow  : 如果存在且非零，那么不同的括号层次产生不同的高亮。
```

g:lisp_rainbow 选项为小括号和反引号提供 10 层不同的色彩。因为色彩层次的数量关系，不同于非 rainbow (彩虹) 方式，rainbow 模式直接使用 ctermfg 和 guifg 指定高亮色彩，而回避了标准的使用高亮组的色彩方案控制。实际使用的高亮值仍然取决于深/浅设置 (见 'bg')。

LITE lite.vim ft-lite-syntax

lite 语法高亮有两个选项。

如果你喜欢字符串里的 SQL 语法高亮，使用：

```
:let lite_sql_query = 1
```

同步的缺省 minlines 为 100。如果你喜欢别的值，可以把 "lite_minlines" 设为你想要的值。例如：

```
:let lite_minlines = 200
```

LPC lpc.vim ft-lpc-syntax

LPC 代表一种简单又节省内存的语言：Lars Pensj C。LPC 的文件名通常是 *.c。把这些文件识别为 LPC 会惹恼那些只用 C 程序的用户。如果你想使用 Vim 的 LPC 语法，在你的 .vimrc 文件里设置变量：

```
:let lpc_syntax_for_c = 1
```

如果这对某些特殊的 C 或 LPC 文件不能工作，用模式行。在 LPC 文件里：

```
// vim:set ft=lpc:
```

对于被识别为 LPC 的 C 文件:

```
// vim:set ft=c:
```

如果你不想设置此变量，在 每个 LPC 文件里使用模式行。

LPC 有若干实现，我们打算支持最常用的实现。这里缺省的 LPC 语法基于 MudOS 系列。对于 MudOS v22 和以前的版本。你应该关闭合适的修饰符，它也会把 v22 之后的新的 efuns 认定为非法。如果你使用最新的 MudOS 版本，不要设置该变量:

```
:let lpc_pre_v22 = 1
```

对于 LpMud 3.2 系列的 LPC:

```
:let lpc_compat_32 = 1
```

对于 LPC4 系列的 LPC:

```
:let lpc_use_lpc4_syntax = 1
```

对于 uLPC 系列的 LPC:
uLPC 是为 Pike 开发的，所以你应该使用 Pike 的语法，而且源文件应该是 *.pike。

LUA lua.vim ft-lua-syntax

Lua 语法文件可用于 Lua 4.0、5.0、5.1 或 5.2 (5.2 是缺省)。用全局变量 lua_version 和 lua_subversion 可以选择其中一个版本。例如，如果要激活 Lua 5.1 高亮，设置变量如下:

```
:let lua_version = 5
:let lua_subversion = 1
```

MAIL mail.vim ft-mail.vim

Vim 高亮 email 的所有标准元素 (信头、签名、引用文本和 URL / email 地址)。要符合标准的习惯，签名应该以 "--" 开头，跟随可选的若干空格并以回车结束的一行开始。

Vim 把 ']'、'}'、'|'、'>' 或者有 '>' 跟随的单词开始的行高亮为引用文本。不过，只有在引用文本用 '>' 的方式引用 (后面可跟一个可选的空格)，Vim 才把把该文本里的信头和签名高亮为引用文本。

mail.vim 缺省从第一个显示行之前的 100 行开始同步语法。如果你的机器很慢，而且通常处理的 email 的信头不长，你可以把它设为较小的值:

```
:let mail_minlines = 30
```

MAKE make.vim ft-make-syntax

Makefile 里，命令通常被高亮以便你发现错误。不过，如果你觉得颜色太多了，可以这样关闭此特性：

```
:let make_no_commands = 1
```

MAPLE maple.vim ft-maple-syntax

Waterloo Maple Inc 的 Maple V 支持符号代数。该语言支持很多函数包，用户可以选择性地装载。如果用户愿意，可以高亮 Maple V release 4 提供的标准包函数。用户可以在 .vimrc 文件里加入：

```
:let mvpkg_all= 1
```

来高亮所有的包里的函数。用户也可以通过从下表选择变量/包来挑选一个子集，并在 .vimrc 文件里（在执行 $VIMRUNTIME/syntax/syntax.vim 之前）设置挑选的变量为 1 就可以了。

Maple V 包函数选择器表

mv_DEtools	mv_genfunc	mv_networks	mv_process
mv_Galois	mv_geometry	mv_numapprox	mv_simplex
mv_GaussInt	mv_grobner	mv_numtheory	mv_stats
mv_LREtools	mv_group	mv_orthopoly	mv_student
mv_combinat	mv_inttrans	mv_padic	mv_sumtools
mv_combstruct	mv_liesymm	mv_plots	mv_tensor
mv_difforms	mv_linalg	mv_plottools	mv_totorder
mv_finance	mv_logic	mv_powseries	

MATHEMATICA mma.vim ft-mma-syntax ft-mathematica-syntax

自动假设空白的 *.m 文件为 Matlab 文件，除非你在 .vimrc 里指定了：

```
let filetype_m = "mma"
```

MOO moo.vim ft-moo-syntax

如果你在表达式里使用 C 风格的注释但发现它影响了高亮，可以尝试使用扩展的（会变慢！）C 风格注释的匹配：

```
:let moo_extended_cstyle_comments = 1
```

要关闭字符串里的代词替换（pronoun substitution）模式高亮：

```
:let moo_no_pronoun_sub = 1
```

要关闭正规表达式 '%|' 操作符和字符串里匹配的 '%(' 和 '%)' 所用的高亮：

```
:let moo_no_regexp = 1
```

可以识别不匹配的双引号并高亮为错误：

```
:let moo_unmatched_quotes = 1
```

要高亮内建的属性 (.name、.location、.programmer 等):

```
:let moo_builtin_properties = 1
```

可以识别未知的内建函数并高亮为错误。如果你使用该选项，应该把自己的扩展加到 mooKnownBuiltinFunction 组里。要打开该选项:

```
:let moo_unknown_builtin_functions = 1
```

把 sprintf() 加到已知内建函数列表的例子:

```
:syn keyword mooKnownBuiltinFunction sprintf contained
```

MSQL msql.vim ft-msql-syntax

msql 语法高亮有两个选项。

如过你希望高亮字符串里的 SQL 语法，使用:

```
:let msql_sql_query = 1
```

同步的 minlines 缺省为 100。如果你喜欢别的值，可以设置 "msql_minlines" 为你所希望的值。例如:

```
:let msql_minlines = 200
```

N1QL n1ql.vim ft-n1ql-syntax

N1QL 是类 SQL 的声明式语言，用于操作 Couchbase Server 数据库的JSON 文档。

Vim 语法高亮 N1QL 语句、关键字、操作符、类型、注释和特殊值。Vim 忽略 SQL 或它的许多方言中特定的如 COLUMN 或 CHAR 等 N1QL 不存在的语法成分。

NCF ncf.vim ft-ncf-syntax

NCF 语法高亮有一个选项。

如果你想把不能识别的 (依据 ncf.vim) 语句高亮为错误，使用:

```
:let ncf_highlight_unknowns = 1
```

如果你不想高亮它们为错误，留着该变量不设置就可以了。

NROFF nroff.vim ft-nroff-syntax

nroff 语法文件可直接用于 AT&T n/troff 而无需修改。如果要使用 GNU groff，你需要在使用之前激活语法文件里的一些附加特性。

例如，Linux 和 BSD 的发布版本使用 groff 作为缺省的文本处理包。要激活 groff

附加的语法高亮特性，在你的启动文件里加入以下选项：

```
:let b:nroff_is_groff = 1
```

Groff 和老的 AT&T n/troff 不同，后者还可以在 Solaris 找到。Groff 宏和请求名可以超过 2 个字符，而且有语言基本命令之外的扩展。例如，AT&T troff 里你可以用请求 \(yr 得到 2 位数的年份。groff 里为了照顾兼容性，可以使用相同的请求，你也可以直接使用 groff 本身的宏：\[year]。宏请求可以超过 2 个字符，比如，GNU mm 接受 ".VERBON" 和 ".VERBOFF" 请求，以创建 verbatim（不作转换的）环境。

要得到 g/troff 能给出的最好的输出，需要遵循一些关于空格和标点的简单的规则。

1. 不要在行尾留空白。

2. 在句尾的句号、感叹号等之后留且只留一个空格。

3. 由于下面的原因，最好在所有的句号之后立即回车。

这些不寻常的提示的背后原因是，如果你不遵循上面的这些规则，g/n/troff 使用的换行算法很容易弄错。

和 TeX 不同，troff 逐行而不是逐段填充文本。此外，它没有 glue（可伸缩的距离）或 stretch 的概念，所有的水平和垂直空白输入都直接成为输出。

因此你必须小心，不要在句子之间留下比你在最终文档想要的更多的空白。因此，通常在每个标点符号之后都立即插入一个回车。如果你想要最终处理过的文本 "对齐"，需要在输入文本里维持常规的空间。要把行尾的空格和标点之后两个或更多的空格标为错误，可用：

```
:let nroff_space_errors = 1
```

另一个检测额外的空格和其它错误的技术会影响你文件的正确排版。这个方法是在你的配置文件里定义语法组 "nroffDefinition" 和 "nroffDefSpecial" 显眼的高亮定义。例如：

```
hi def nroffDefinition term=italic cterm=italic gui=reverse
hi def nroffDefSpecial term=italic,bold cterm=italic,bold
                       \ gui=reverse,bold
```

如果你想像段标记符那样方便地浏览源文件里的预处理项目，可以在 .vimrc 文件里激活以下选项：

```
let b:preprocs_as_sections = 1
```

还有，语法文件为 ms 包里设置带缩进的（exdented）段落宏（.XP）增加了一个附加的段标记符。

最后，有一个 groff.vim 语法文件，可以基于每个文件或，缺省情况下，在全局打开 groff 的语法高亮。

OCAML　　ocaml.vim　ft-ocaml-syntax

OCaml 语法文件处理带以下后缀的文件：.ml、.mli、.mll 和 .mly。设置以下变量

```
:let ocaml_revised = 1
```

你就可以切换标准的 OCaml 语法为 camlp4 预处理器支持的改进的语法。设置变量

```
:let ocaml_noend_error = 1
```

防止把 "end" 高亮为错误，这可用于源程序包含很长的结构而 Vim 不再能保持同步的场合。

PAPP papp.vim ft-papp-syntax

PApp 语法文件处理 .papp 文件和，在一定程度上，.pxml 和 .pxsl 文件。它们都是 perl / xml / html / 其它格式 的混合，并使用 xml 作为顶层的文件格式。缺省，所有 phtml 和 pxml 段里的内容都被处理为包含内嵌预处理器命令的字符串。如果你在启动文件里设置变量:

```
:let papp_include_html=1
```

它就会试图语法高亮 pthml 段里的 html 代码，但这相对较慢，而且对于有效的编辑未免色彩太鲜艳了些 ;)

可以在 http://papp.plan9.de 找到最新的 papp.vim 语法文件的版本。

PASCAL pascal.vim ft-pascal-syntax

匹配 "*.p" 的文件可以是 Progress 或者 Pascal 的。如果自动检测对你不适用，或者你从来不编辑 Progress，在启动 vimrc 里加入:

```
:let filetype_p = "pascal"
```

Pascal 语法文件被扩展，以支持 Turbo Pascal、Free Pascal 编译器和 GNU Pascal 编译器的一些扩展。也支持 Delphi 的关键字。缺省打开 Turbo Pascal 7.0 特性。如果你只想使用标准的 Pascal 关键字，在你的启动文件里加入下行:

```
:let pascal_traditional=1
```

要打开 Delphi 专用的构造 (比如单行注释、关键字、等等):

```
:let pascal_delphi=1
```

pascal_symbol_operator 选项控制符号 (symbol) 操作符，如 +、* 等，是否使用 Operator 的色彩高亮。要给符号的操作符加上颜色，在你的启动文件里加入下行:

```
:let pascal_symbol_operator=1
```

有些函数缺省是高亮的。要关闭:

```
:let pascal_no_functions=1
```

另外，一些编译器有专门的变量。除了 pascal_delphi 以外，还有 pascal_gpc 和

pascal_fpc。缺省试图匹配 Turbo Pascal 的扩展。

```
:let pascal_gpc=1
```

或

```
:let pascal_fpc=1
```

要确保字符串在一行内定义，你可以定义 pascal_one_line_string 变量。

```
:let pascal_one_line_string=1
```

如果你不喜欢 <Tab> 字符，你可以设置 pascal_no_tabs 变量。制表会被高亮为 Error。

```
:let pascal_no_tabs=1
```

PERL perl.vim ft-perl-syntax

perl 的语法高亮有一些可用的选项。

现在缺省打开内建 POD 的高亮。如果不希望因 Perl 文件内嵌的 POD 高亮增加复杂度，可以把 'perl_include_pod' 选项设为 0:

```
:let perl_include_pod = 0
```

要减低分析的复杂度（同时提高了效率），你可以关闭变量名和内容的分析过程的两个元素。

要使变量和函数名里对包的引用与名字的其它部分不区别显示（如 '$PkgName::VarName' 里的 'PkgName::'）:

```
:let perl_no_scope_in_variables = 1
```

(Vim 6.x 里相反，用 "perl_want_scope_in_variables" 打开区别显示。)

如果你不想分析复杂的结构，比如 '@{${"foo"}}':

```
:let perl_no_extended_vars = 1
```

(Vim 6.x 里相反，用 "perl_extended_vars" 打开此项分析。)

你可以改变颜色字符串。缺省，字符串和 qq 等变形会像下面第一行那样高亮。如果你设置了变量 perl_string_as_statement，那么就像下面第二行那样高亮。

```
"hello world!"; qq|hello world|;
^^^^^^^^^^^^^^NN^^^^^^^^^^^^^^^N       (unlet perl_string_as_statement)
S^^^^^^^^^^^^^SNNSSS^^^^^^^^^^^SN      (let perl_string_as_statement)
```

(^ = perlString、S = perlStatement、N = 什么都没有)

同步有三个选项。前两个关掉一些激活同步的方法，而只有在无法正确工作的时候你才需要它们。比如，如果滚动时突然全屏的颜色发生改变，那么你应该尝试改变并关闭其中的

某一个。如果你可以发现哪一行导致这种错误，请告诉我。

大致上，其中一个在 "^\s*sub\s*" 上激活，另一个则在 "^[$@%]" 上。

```
:let perl_no_sync_on_sub
:let perl_no_sync_on_global_var
```

下面，你还可以设置 VIM 往前找语法高亮的起始点的最大距离。

```
:let perl_sync_dist = 100
```

如果你想要在 perl 里使用折叠，设置 perl_fold:

```
:let perl_fold = 1
```

如果你想折叠 if 等语句块，设置如下:

```
:let perl_fold_blocks = 1
```

'perl_fold' 置位时，缺省折叠例程。如果不想要，可以设置 'perl_nofold_subs':

```
:let perl_nofold_subs = 1
```

缺省不折叠无名例程；可以用 'perl_fold_anonymous_subs' 打开之:

```
:let perl_fold_anonymous_subs = 1
```

'perl_fold' 置位时，缺省对包折叠。如果不想要，可以设置 'perl_nofold_packages':

```
:let perl_nofold_packages = 1
```

PHP3 和 PHP4 php.vim php3.vim ft-php-syntax ft-php3-syntax

[注意 : 以前这被称为 "php3"，但因为现在这也支持 php4，它被改名为 "php"]

php 的语法高亮支持以下选项。

如果你喜欢字符串里的 SQL 语法高亮:

```
let php_sql_query = 1
```

要高亮 Baselib 方法:

```
let php_baselib = 1
```

打开字符串里的 HTML 语法高亮:

```
let php_htmlInStrings = 1
```

使用旧的色彩风格:

```
let php_oldStyle = 1
```

打开 ASP 风格的短标签的高亮：

```
let php_asp_tags = 1
```

关闭短标签：

```
let php_noShortTags = 1
```

要高亮外层] 或) 的错误：

```
let php_parent_error_close = 1
```

要在有打开的 (和 [但没有相应的结束符号的情况下跳过 php 结束标签：

```
let php_parent_error_open = 1
```

打开类和函数的折叠：

```
let php_folding = 1
```

选择同步方法：

```
let php_sync_method = x
```

x = -1 使得同步以搜索方法进行 (缺省)，
x > 0 使得同步至少往回 x 行，
x = 0 使得同步从头开始。

PLAINTEX plaintex.vim ft-plaintex-syntax

TeX 是排版语言，而 plaintex 是代表 Tex 的 "平凡" 变种的文件类型。如果你想 *.tex 文件被识别为平凡 TeX，见 ft-tex-plugin 。

此语法文件有以下选项

```
let g:plaintex_delimiters = 1
```

如果你想高亮方括号 "[]" 和大括号 "{}" 的话。

PPWIZARD ppwiz.vim ft-ppwiz-syntax

PPWizard 是 HTML 和 OS/2 INF 文件的预处理器。

该语法文件有如下选项：

- ppwiz_highlight_defs : 决定 PPWizard 定义的高亮模式。可能值是

  ppwiz_highlight_defs = 1 : PPWizard #define 语句保留其内容的色彩 (比如，PPWizard 的宏和变量)

  ppwiz_highlight_defs = 2 : 预处理器 #define 和 #evaluate 语句使用单色显示，

除了续行符以外

缺省 ppwiz_highlight_defs 的设置为 1。

- ppwiz_with_html : 如果该值为 1 (缺省)，高亮按本义出现的 HTML 代码；如果为 0，把 HTML 代码当成普通的文本。

PHTML phtml.vim ft-phtml-syntax

phtml 语法高亮有两个选项。

如果你喜欢字符串里的 SQL 语法高亮，使用：

```
:let phtml_sql_query = 1
```

同步的 minlines 缺省为 100。如果你喜欢别的值，可以设置 "phtml_minlines" 为你所希望的值。例如：

```
:let phtml_minlines = 200
```

POSTSCRIPT postscr.vim ft-postscr-syntax

PostScript 的高亮有若干选项。

首先决定是 PostScript 语言的哪个版本要高亮。目前定义了三个语言版本。Level 1 是原始和基础的版本，包括所有的 Level 2 发布之前的扩展。Level 2 是最常用的版本，包括 Level 3 发布之前它自身的所有扩展。Level 3 是目前支持的最高版本。你可以这样定义 postscr_level 变量，以选择需要高亮的 PostScript 的语言级别：

```
:let postscr_level=2
```

如果该变量没有定义，缺省值为 2 (Level 2)，因为这是目前最常用的版本。

注意，不是所有的 PS 解释器都支持某一特定语言级别的所有语言特性。特别是，PS 文件开头的 %!PS-Adobe-3.0 并 不 意味着使用的 PostScript 是 Level 3 的 PostScript！

如果你使用 Display PostScript，可以这样定义 postscr_display 变量来包含 Display PS 语言特性的高亮：

```
:let postscr_display=1
```

如果你使用 Ghostscript，可以这样定义 postscr_ghostscript 变量来包含 Ghostscript 特有的语言特性的高亮：

```
:let postscr_ghostscript=1
```

PostScript 是一个很大的语言，有许多预定义的元素。尽管包含所有这些元素的高亮很有用，在较慢的机器上这会使得 Vim 变慢。为了使得对机器更友善，缺省不给字体名和字符编码高亮。如果你不是显式地打开它们，应该没有问题。如果你确实想看到它们的高亮，可以设置下面之中的一个或两个变量：

```
:let postscr_fonts=1
:let postscr_encodings=1
```

关于 and、or 和 not 的高亮有一个风格的选项。PostScript 里，这些操作符的函数取决于它们操作数的类型 - 如果操作数都是布尔型，它们是逻辑操作符。如果是整数，它们是二进制操作符。如果二进制和布尔型操作符高亮方式不同，它们可以用任何一种方式高亮。缺省它们被作为逻辑操作符。如果这样定义 postscr_andornot_binary 变量，它们可以用二进制操作符方式进行高亮：

```
:let postscr_andornot_binary=1
```

PRINTCAP + TERMCAP　　ptcap.vim　ft-printcap-syntax　ft-ptcap-syntax　ft-termcap-syntax

该语法文件适用于 printcap 和 termcap 数据库。

要使得 Vim 识别不匹配模式 "printcap" 或 "termcap" 的 printcap/termcap 文件，你需要在 myfiletypefile 文件里定义合适你的系统的附加的模式。对这些模式，你必须设置变量 "b:ptcap_type" 为 "print" 或 "term"，然后设置 'filetype' 选项为 ptcap。

比如，要使得 Vim 识别 /etc/termcaps/ 里的所有文件为 termcap 文件，加入下行：

```
:au BufNewFile,BufRead /etc/termcaps/* let b:ptcap_type = "term" |
                                      \ set filetype=ptcap
```

如果你注意到往回滚动时高亮有问题，但 CTRL-L 又可以修正的时候，尝试设置 "ptcap_minlines" 内部变量为一个大的数字：

```
:let ptcap_minlines = 50
```

(缺省为 20 行。)

PROGRESS　　progress.vim　ft-progress-syntax

匹配 "*.w" 的文件可以是 Progress 或者 cweb 的。如果自动识别对你无效，或者你从来不编辑 cweb，在你的启动 vimrc 里加入：

```
:let filetype_w = "progress"
```

这同样适用于可为汇编文件的 "*.i" 和可为 Pascal 文件的 "*.p"。如果你不使用汇编和 Pascal，你可以这么用：

```
:let filetype_i = "progress"
:let filetype_p = "progress"
```

PYTHON　　python.vim　ft-python-syntax

有六个选项可以控制 Python 的语法高亮。

关于高亮数值：

```
:let python_no_number_highlight = 1
```

关于高亮内建函数：

```
:let python_no_builtin_highlight = 1
```

关于高亮标准例外：

```
:let python_no_exception_highlight = 1
```

关于高亮 doctest 和其中的代码：

```
:let python_no_doctest_highlight = 1
```

或

```
:let python_no_doctest_code_highlight = 1
```

(第一个选项隐含第二个)。

要高亮行尾的空白还有空格和制表的混合：

```
:let python_space_error_highlight = 1
```

如果你想要所有可能的 Python 高亮 (等同于置位上面最后的选项及复位其余选项)：

```
:let python_highlight_all = 1
```

注意：对以上选项，只关心其存在于否，值并不重要，可以把上面的 1 换成任何值。

QUAKE quake.vim ft-quake-syntax

Quake 语法定义应可用于多数基于某个 Quake 引擎的 FPS (First Person Shooter)。不过，在相关的三个游戏 (Quake、Quake 2 和 Quake 3 Arena) 中，命令的名字略有不同。所以，语法定义检查三个全局变量是否存在，从而使用户可以指定他们的文件里哪些命令是合法的。这三个变量的设置有如下效果。

设置使得高亮命令只适用于 Quake：

```
:let quake_is_quake1 = 1
```

设置使得高亮命令只适用于 Quake 2：

```
:let quake_is_quake2 = 1
```

设置使得高亮命令只适用于 Quake 3 Arena：

```
:let quake_is_quake3 = 1
```

组合这三个变量的使用也是可以的，不过高亮的命令也许比你的游戏里实际可用的命令要多。

R r.vim ft-r-syntax

R 代码的语法高亮的解析从反向 40 行前开始，但在 vimrc 里可设置不同的值。如：

```
let r_syntax_minlines = 60
```

也可以关闭 ROxygen 的语法高亮：

```
let r_syntax_hl_roxygen = 0
```

打开小括号、方括号和花括号括起的代码的折叠：

```
let r_syntax_folding = 1
```

把所有后跟开括号的关键字作为函数来高亮：

```
let r_syntax_fun_pattern = 1
```

R MARKDOWN rmd.vim ft-rmd-syntax

要关闭 YAML 头部的语法高亮，在 vimrc 加入：

```
let rmd_syn_hl_yaml = 0
```

要关闭引用键的语法高亮：

```
let rmd_syn_hl_citations = 0
```

要高亮 knitr 块头部的 R 代码：

```
let rmd_syn_hl_chunk = 1
```

缺省，R 代码块使用 R 语言的规则进行高亮。如果要其它语言块正确的语法高亮，应把它们加入 markdown_fenced_languages 或 rmd_fenced_languages 中。例如要正确地同时高亮 R 和 Python，在 vimrc 加入：

```
let rmd_fenced_languages = ['r', 'python']
```

R RESTRUCTURED TEXT rrst.vim ft-rrst-syntax

要高亮 knitr 块头部的 R 代码，在 vimrc 加入：

```
let rrst_syn_hl_chunk = 1
```

READLINE readline.vim ft-readline-syntax

readline 库主要由 BASH 外壳使用，在已有的命令和选项的基础上，它又增加了不少。要高亮这些附加的命令和选项，可以把这行加到你的 vimrc 里，或者在载入使用 readline 语法的文件前，在命令行输入：

```
let readline_has_bash = 1
```

这使得 BASH (2.05a 和其后的版本，也包括部分以前的) 增加的命令被高亮。

REGO rego.vim ft-rego-syntax

Rego 是 Styra 开发的查询语言。主要用作 kubernetes 的政策语言，但可用于几乎任何功能。带以下扩展名的文件被认为是 rego 文件：.rego。

RESTRUCTURED TEXT rst.vim ft-rst-syntax

对一些特定的文件类型，打开文档内代码块的语法高亮。缺省的语法列表见 $VIMRUNTIME/syntax/rst.vim。

要设置用户定义的代码块语法高亮的列表：

```
let rst_syntax_code_list = ['vim', 'lisp', ...]
```

要把多个代码块类型赋给同一个语法，定义 rst_syntax_code_list 为如下映射：

```
let rst_syntax_code_list = {
        \ 'cpp': ['cpp', 'c++'],
        \ 'bash': ['bash', 'sh'],
        ...
\ }
```

要使用色彩高亮来强调文本：

```
let rst_use_emphasis_colors = 1
```

要打开段的折叠：

```
let rst_fold_enabled = 1
```

备注 某些平台上折叠可能造成性能问题。

REXX rexx.vim ft-rexx-syntax

如果你注意到往回滚动时，高亮出现问题，但 CTRL-L 重画又可以修正的话，尝试设置 "rexx_minlines" 内部变量为较大的值：

```
:let rexx_minlines = 50
```

这使得语法同步在第一个显示行之前的 50 行开始。缺省值为 10。使用较大的值的缺点是重画会变慢。

Vim 试图自己猜测 ".r" 文件的真实类型。如果无法检测（根据注释行内容），假定为 "r"。要使缺省为 rexx，给 .vimrc 文件加上： g:filetype_r

```
:let g:filetype_r = "r"
```

RUBY ruby.vim ft-ruby-syntax

Ruby: 操作符高亮	ruby_operators
Ruby: 空白错误	ruby_space_errors
Ruby: 折叠	ruby_fold ruby_foldable_groups
Ruby: 减少昂贵操作	ruby_no_expensive ruby_minlines
Ruby: 拼写检查字符串	ruby_spellcheck_strings

ruby_operators

Ruby: 操作符高亮

可通过定义 "ruby_operators" 来高亮操作符：

```
:let ruby_operators = 1
```

ruby_space_errors

Ruby: 空白错误

可以通过定义 "ruby_space_errors" 打开空白错误的高亮：

```
:let ruby_space_errors = 1
```

会高亮行尾的空白，而空格后的制表也被认为是错误。通过定义 "ruby_no_trail_space_error" 和 "ruby_no_tab_space_error"，可以进一步限定。这两个变量分别忽略行尾空白和空格之后的制表。

ruby_fold ruby_foldable_groups

Ruby: 折叠

定义 "ruby_fold" 可以打开折叠：

```
:let ruby_fold = 1
```

会把局部于当前缓冲区或窗口的 'foldmethod' 选项设为 "syntax"，它打开 Ruby 文件类型编辑时的基于语法的折叠。

缺省折叠相当详细，例如，"if"、"do"、"%w[]" 那样的小的语法单元会创建对应的折叠层次。

可以设置 "ruby_foldable_groups" 来限制可折叠的组：

```
:let ruby_foldable_groups = 'if case %'
```

可选值为空格分隔的关键字列表：

关键字	含义
ALL	绝大多数的块语法（缺省）
NONE	无
if	"if" 或 "unless" 块
def	"def" 块
class	"class" 块
module	"module" 块
do	"do" 块
begin	"begin" 块
case	"case" 块
for	"for"、"while"、"until" 循环
{	花括号块或哈希字面值
[	数组字面值
%	"%" 记号的字面值，如：%w(STRING)、%!STRING!
/	正规表达式
string	字符串和外壳命令输出（被 '、"、` 包围）
:	符号
#	多行注释
<<	Here 文档
__END__	"__END__" 指令之后的源代码

ruby_no_expensive

Ruby：减少昂贵操作

缺省，"end" 关键字根据它关闭的块对应的打开语句设定颜色。尽管很有用，该特性很消耗资源：如果你发现重画变慢（或者你所在的终端色彩支持不好），你可能想关闭该特性，只要定义 "ruby_no_expensive" 变量即可：

```
:let ruby_no_expensive = 1
```

此时，所有的控制关键字使用相同的颜色。

ruby_minlines

如果你想使用该特性，但注意到往回滚动时，高亮出现问题，但 CTRL-L 重画又可以修正的话，尝试设置 "ruby_minlines" 变量超过 50：

```
:let ruby_minlines = 100
```

理想的话，该值应该足够大，使得最大的类或模块能够得到处理。

ruby_spellcheck_strings

Ruby：拼写检查字符串

定义 "ruby_spellcheck_strings" 可以用 Ruby 语法来执行字符串的拼写检查：

```
:let ruby_spellcheck_strings = 1
```

SCHEME scheme.vim ft-scheme-syntax

缺省只高亮 R7RS 关键字并进行适当的缩进。

scheme.vim 也支持 Chicken Scheme->C compiler 的扩展。如果需要，定义 b:is_chicken 或 g:is_chicken。

SDL sdl.vim ft-sdl-syntax

SDL 的高亮可能会缺少一些关键字，但 SDL 的关键字太多了，完全照顾过来是不太可能的。

新的标准 SDL-2000 指定所有的标识符都是大小写敏感的（以前并非如此），而所有使用的关键字必须或者是完全小写，或者完全大写。要使得高亮能够反映这些特性，你可以设置如下的变量：

```
:let sdl_2000=1
```

这也会设置很多新的关键字。如果你想屏蔽旧的关键字（其实，这是个好主意），可以用：

```
:let SDL_no_96=1
```

缩进可能还没完全处理好，不过我在自己的项目目前的应用里已经相当满意了。

SED sed.vim ft-sed-syntax

要使得制表在普通的空白里突出显示（方法是在制表上使用 Todo 高亮），在 vimrc 文件里如此定义 "highlight_sedtabs"

```
:let highlight_sedtabs = 1
```

（这种特殊高亮只适用于搜索模式、替换文本、地址或者 Append/Change/Insert 命令里包含的文本中的制表。）如果你打开该选项，那么最好把制表宽度设为一个字符；这么做，你很容易计算字符串里的制表数量。

漏洞：

transform 命令（y）和 substitute 命令的处理相同。也就是说，就语法文件而言，transform 和 substitute 接受相同的标志。这不正确（Transform 不接受标志）。但我容忍这个问题，因为牵涉的命令需要很复杂的处理（95 个模式，每个可能的模式定界符就需要一个模式）。

SGML sgml.vim ft-sgml-syntax

SGML 文件里，标签的色彩方案工作方式如下。

开放标签的 <> 和关闭标签的 </> 的色彩不同。这是有意的。开放标签使用 'Function' 色彩，而关闭标签使用 'Type' 色彩（见 syntax.vim 察看它们是怎么定义的）。

已知的标签名和 C 语句的色彩相同。未知的标签名和相应的 <> 或 </> 颜色相同，以便纠错。

注意 这也适用于参数（或属性）的名字。已知的属性名和未知的标色不同。

一些 SGML 标签用于改变文本的显示。sgml.vim 语法色彩文件识别以下的标签，并相应地改变普通文本的显示方式：<varname> <emphasis> <command> <function> <literal> <replaceable> <ulink> 和 <link>。

如果你想改变文本显示的方式，必须重定义以下的语法组：

- sgmlBold
- sgmlBoldItalic
- sgmlUnderline
- sgmlItalic
- sgmlLink 设定链接

要使得重定义能够工作，你必须重定义所有的组。在你的 vimrc（这是根据初始化时读入文件的顺序）里定义下面的变量

```
:let sgml_my_rendering=1
```

在你的 vimrc 文件里加上这行，可以屏蔽这种显示方式：

```
:let sgml_no_rendering=1
```

(从 Claudio Fleiner <claudio@fleiner.com> 的 html.vim 的帮助文本转来)

ft-posix-synax ft-dash-syntax
SH sh.vim ft-sh-syntax ft-bash-syntax ft-ksh-syntax

这里讨论较古老的 Unix (Bourne) sh 和较新的外壳如 bash、dash、posix 和 Korn shell 的语法高亮。

Vim 试图根据文件名决定使用的外壳类型，例如：

```
ksh : .kshrc* *.ksh
bash: .bashrc* bashrc bash.bashrc .bash_profile* *.bash
```

关于模式的完整列表，见 $VIMRUNTIME/filetype.vim。如果这些都不符，那么就检查文件的第一行（比如寻找 /bin/sh /bin/ksh /bin/bash）。如果第一行指定了外壳类型，那么就使用该类型。不过有的文件（比如 .profile）肯定是外壳文件，但其类型并不容易推出。另外，有的系统里 sh 被符号链接到 "bash"（linux、Windows+cygwin）或 "ksh"（posix）。

你可以在 <.vimrc> 里设置下列变量中的一个，以指定全局的缺省值：

```
ksh:
```

```
    let g:is_kornshell = 1
posix: (和设置 g:is_kornshell 为 1 效果几乎相同)
    let g:is_posix     = 1
bash:
    let g:is_bash      = 1
sh: (缺省) Bourne shell
    let g:is_sh        = 1

(dash 用户应使用 posix)
```

如果没有 "#! ..." 一行，而用户也没有用上述方法设定缺省的 sh.vim 语法设置，那么 syntax/sh.vim 假定使用 Bourne shell 语法。请不要在错误报告里引用 RFC 或者市场占有率的统计数据（译者注：此处大概指希望使用其它缺省值的用户）-- 自己选择系统使用的缺省 sh 版本并在 <.vimrc> 文件里安装相应的 "let..." 就可以了。

syntax/sh.vim 文件提供若干级别的基于语法的折叠：

```
let g:sh_fold_enabled= 0     (缺省，无语法高亮)
let g:sh_fold_enabled= 1     (打开函数折叠)
let g:sh_fold_enabled= 2     (打开 here 文档折叠)
let g:sh_fold_enabled= 4     (打开 if/do/for 折叠)
```

那么若干语法项目（即 Here 文档和函数体）就可以进行语法折叠（见 :syn-fold ）。把这些值加在一起可以得到多种项目的折叠：

```
let g:sh_fold_enabled= 3     (打开函数和 here 文档的折叠)
```

如果你注意到往回滚动时，高亮出现问题，但 CTRL-L 重画又可以修正的话，尝试设置 "sh_minlines" 内部变量为较大的值：

```
:let sh_minlines = 500
```

这使得语法同步在第一个显示行之前的 500 行开始。缺省值为 200。使用较大的值的缺点是重画会变慢。

如果你没有要同步的东西，但显示又很慢，可以设置 "sh_maxlines" 内部变量来加速。比如：

```
let sh_maxlines = 100
```

缺省值是 sh_minlines 的两倍。设置为较小的值可以提高显示的速度。缺点是高亮错误出现的可能性也较大。

syntax/sh.vim 试图标记若干问题为错误；包括常见的多余的 ']'、'done'、'fi' 等情况。如果发现错误处理对你有问题，可以在 .vimrc 里这样屏蔽错误高亮：

```
let g:sh_no_error= 1
```

sh-embed sh-awk

Sh: 内 嵌 语 言

你也许想要在 sh 里内嵌其他语言。多谢 Lorance Stinson 提供了一个 awk 的例子。把下面内容写入 $HOME/.vim/after/syntax/sh/awkembed.vim：

```
" AWK Embedding:
```

```
" ==============
" Shamelessly ripped from aspperl.vim by Aaron Hope.
if exists("b:current_syntax")
  unlet b:current_syntax
endif
syn include @AWKScript syntax/awk.vim
syn region AWKScriptCode matchgroup=AWKCommand start=+[=\\]\@<!'+ skip=+\\'+ end=+'+ contains=
syn region AWKScriptEmbedded matchgroup=AWKCommand start=+\<awk\>+ skip=+\\$+ end=+[=\\]\@<!'
syn cluster shCommandSubList add=AWKScriptEmbedded
hi def link AWKCommand Type
```

此代码会接受如下单引号括起的 awk 代码:

```
awk '...awk code here...'
```

使之采用 awk 高亮语法。显然此方法可以类似地扩展到其他语言上。

SPEEDUP spup.vim ft-spup-syntax
(AspenTech plant simulator)

Speedup 语法文件有如下选项:

- strict_subsections : 如果定义该变量，只有段 (section) 和子段 (subsection) 里的关键字会作为 Statement 高亮，而其它关键字不会 (比如 OPERATION 段里的 WITHIN)。

- highlight_types : 该变量的定义使得流类型 (stream type)，比如 temperature 或 pressure，用 Type 高亮，而不是普通的 Identifier。这里包括 DECLARE 段常见的类型；如果定义了自己的类型，要在语法文件里自己加入。

- oneline_comments : 该值可选 1 到 3，它决定了 # 风格的注释的高亮方式。

  oneline_comments = 1 : 允许偶数个 # 之后的正常 Speedup 代码。

  oneline_comments = 2 : 第二个 # 开始的代码显示为出错。这是缺省设置。

  oneline_comments = 3 : 如果某行包含超过一个 #，把整行显示为出错。

特别因为 OPERATION 段因为 PRESET (预设) 的变量而可能会很大，同步的正确设置很重要。如果你的机器足够快，你可以在语法文件的末尾增加 minlines 和/或 maxlines 的值。

SQL sql.vim ft-sql-syntax
sqlinformix.vim ft-sqlinformix-syntax
sqlanywhere.vim ft-sqlanywhere-syntax

尽管有 ANSI 的 SQL 标准，多数数据库引擎都增加了自己的扩展。Vim 目前支持 Oracle 和 Informix 的 SQL 方言。Vim 缺省假设 "*.sql" 文件使用 Oracle SQL。

Vim 目前通过不同语法脚本提供不同供应商的 SQL 支持。你可以把 Vim 的缺省设置从 Oracle 改为任何目前支持的 SQL 类型。你也可以方便地为每个缓冲区设置不同的 SQL 方言。

详细的操作可见 ft_sql.txt 。

TCSH tcsh.vim ft-tcsh-syntax

这里讨论名为 "tcsh" 的外壳。这是 csh 的超集。关于如何检测文件类型，见 csh.vim 。

Tcsh 不允许字符串里的 \"，除非设置了 "backslash_quote" 外壳变量。如果你希望 VIM 认定不应该存在反斜杠 + 引号的构造，在 .vimrc 里加入这行：

```
:let tcsh_backslash_quote = 0
```

如果你注意到往回滚动时，高亮出现问题，但 CTRL-L 重画又可以修正的话，尝试设置 "tcsh_minlines" 内部变量为较大的值：

```
:let tcsh_minlines = 1000
```

这使得语法同步在第一个显示行之前的 1000 行开始。如果设置 "tcsh_minlines" 为 "fromstart"，同步从文件开始处进行。tcsh_minlines 的缺省值为 100。使用较大的值的缺点是重画会变慢。

TEX tex.vim ft-tex-syntax latex-syntax

Tex 内容

Tex: 要语法折叠么? tex-folding
Tex: 不想拼写检查 g:tex_nospell
Tex: 不想检查注释里的拼写? tex-nospell
Tex: 需要在 Verbatim 区中使用拼写检查? tex-verb
Tex: 在注释还是数学模式里 tex-runon
Tex: 语法高亮很慢? tex-slow
Tex: 想高亮更多的命令? tex-morecommands
Tex: 过多的 Error 高亮? tex-error
Tex: 需要新的数学模式的组? tex-math
Tex: 开始新的风格? tex-style
Tex: 利用隐藏模式 tex-conceal
Tex: 选择性的隐藏模式 g:tex_conceal
Tex: 控制 iskeyword g:tex_isk
Tex: 下标和上标的精细控制 tex-supersub

tex-folding g:tex_fold_enabled

Tex: 要语法折叠么?

<syntax/tex.vim> 的版本 28 支持基于语法的 part、chapter、section、subsection 等等的折叠。把

```
let g:tex_fold_enabled=1
```

放到你的 <.vimrc> 里，并 :set fdm=syntax。我建议把后者放到你的 LaTeX 文件末尾的模式行里来执行：

```
% vim: fdm=syntax
```

如果系统过慢，可以看看

```
https://vimhelp.org/vim_faq.txt.html#faq-29.7
```

g:tex_nospell

Tex: 不想拼写检查

如果不想 LaTeX 文档在任何地方检查拼写，在 .vimrc 里加入

```
let g:tex_nospell=1
```
如果只想关闭注释中的拼写检查，见 g:tex_comment_nospell 。

tex-nospell g:tex_comment_nospell

Tex: 不想检查注释里的拼写?

有些家伙喜欢在注释里写源代码，所以希望在 LaTeX 文件的注释里关闭拼写检查。为此，在 <.vimrc> 里放上:
```
let g:tex_comment_nospell= 1
```
如果你要关闭 LaTeX 文档中所有地方的拼写检查，见 g:tex_nospell 。

tex-verb g:tex_verbspell

Tex: 需要在 Verbatim 区中使用拼写检查?

通常 verbatim 区域用于源代码这类的东西；很少有人想对源代码进行拼写检查。如果你真的想要对 verbatim 区进行拼写检查，在 <.vimrc> 里放入:
```
let g:tex_verbspell= 1
```

tex-runon tex-stopzone

Tex: 在注释还是数学模式里?

<syntax/tex.vim> 高亮支持 TeX、LaTeX 和部分的 AmsTeX。高亮支持包括三个主要的区域: normal、texZone 和 texMathZone。尽管付出了相当的努力使得这些区域能正确地终止，$..$ 和$$..$$ 定界的区域无法同步，因为开始和结束模式无法区别。因而，提供了一个特殊的 "TeX 注释"
```
%stopzone
```
它会使得 texZone 或 texMathZone 强迫终止。

tex-slow tex-sync

Tex: 语法高亮很慢?

如果你使用机器速度很慢，可能会想减小这些变量的值
```
:syn sync maxlines=200
:syn sync minlines=50
```
(特别是后者)。如果你的机器很快，可以考虑增大它们的值。它们主要影响同步 (确切地说也就是: 如果有的话，哪个语法组包含屏幕顶部的文本?)。

另外一个高亮缓慢的原因是基于语法的折叠；解决办法见 tex-folding 。

g:tex_fast

最后，如果语法高亮还是太慢，可以在 .vimrc 里设置
```
:let g:tex_fast= ""
```
g:tex_fast 变量使得语法高亮脚本避免定义任何语法区域和相关的同步。这使得语法高亮速度大大加快；作为代价：高亮和基于语法的折叠会少很多，也不能进行基于语法的错误检查。

可以选择接受若干而非全部的语法项目；用下表可以选择性地打开部分语法高亮:

```
b : 接受粗体和斜体语法
c : 接受 texComment 语法
m : 接受 texMatcher 语法 (即 {...} 和 [...])
```

M : 接受 texMath 语法
p : 接受 部分、章节、小节等的语法
r : 接受 texRefZone 语法 (nocite、bibliography、label、pageref、eqref)
s : 接受 上标/下标区域
S : 接受 texStyle 语法
v : 接受 verbatim 语法
V : 接受 texNewEnv 和 texNewCmd 语法

例如，g:tex_fast= "M" 会打开数学相关的高亮但关闭其他基于区域的语法高亮。
(另见： g:tex_conceal 和 tex-supersub)

tex-morecommands tex-package

Tex: 想高亮更多的命令?

LaTeX 是可编程的语言，因而有数以千计的包，包含各种专门的 LaTeX 命令、语法和字体。如果你用了某个包，当然希望发布的 syntax/tex.vim 能支持它。但这显然是不实际的。请考虑使用 mysyntaxfile-add 介绍的技术来扩展或者修改 syntax/tex.vim 提供的高亮处理。任何你编写的扩展一般应放在 $HOME/after/syntax/tex/[pkgname].vim，并请考虑上传到 http://vim.sf.net/。

在我的网站上有若干流行包的支持:

http://www.drchip.org/astronaut/vim/index.html#LATEXPKGS

那里的语法文件可以放到你的 .../after/syntax/tex/ 目录。

tex-error g:tex_no_error

Tex: 过多的 Error 高亮?

<tex.vim> 支持各种的词法检查。尽管错误检查经常很有用，它指示的地方实际上可能没有错误。如果你有这个问题，可以在 <.vimrc> 里放上如下的语句:

```
let g:tex_no_error=1
```

从而 <tex.vim> 提供的所有错误检查都会被抑制。

tex-math

Tex: 需要新的数学模式的组?

如果你需要在 LaTeX 里包含新的数学组，下面的代码给出一个告诉你可以如何操作的例子:

```
call TexNewMathZone(sfx,mathzone,starform)
```

你需要为新数学组起一个独一无二的后缀 (目前，A-L 和 V-Z 被 <syntax/tex.vim> 自己占用)。比如，看看 <syntax/tex.vm> 是怎么设置 eqnarray 的:

```
call TexNewMathZone("D","eqnarray",1)
```

需要把 "mathzone" 换成新数学组的组名，然后在 .vim/after/syntax/tex.vim 里调用。如果 "starform" 变量为真，意味着新数学组有星号的形式 (比如， eqnarray*)。

tex-style b:tex_stylish

Tex: 开始新的风格?

你可以在 *.tex 文件里使用 "\makeatletter"，从而在命令里可用 "@"。不过，因为 *.tex 文件没有如下的后缀: sty cls clo dtx ltx，语法高亮会把这里使用的 @ 标为错误。要解决这个问题:

```
:let b:tex_stylish = 1
```

```
:set ft=tex
```

把 "let g:tex_stylish=1" 放到你的 <.vimrc> 里，这会使得 <syntax/tex.vim> 总能接受 @ 的这种使用方式。

tex-cchar tex-cole tex-conceal

Tex: 利用隐藏模式

如果你设置 'conceallevel' 为 2 而编码是 utf-8，若干字符序列会翻译为合适的 utf-8 字形，包括各种重音字符、数学模式的希腊字母。数学模式的上标和下标。不是所有的字符都可以转为上标和下标；这是由于 utf-8 支持的限制。事实上，只有很少的字符支持下标。

一个用法是垂直分割窗口（见 CTRL-W_v ）；其中一个把 'conceallevel' 设为 0，另一个设为 2；两者都用 'scrollbind' 。

g:tex_conceal

Tex: 选择性的隐藏模式

通过在 <.vimrc> 中设置 g:tex_conceal，可以有选择性地使用隐藏模式。缺省的设置为 "admgs"，隐藏以下对应的字符集合：

```
a = 重音/连写体 (accents/ligatures)
b = 粗体和斜体
d = 定界符
m = 数学符号
g = 希腊字母
s = 上标/下标
```

省略其中一个或几个字符关闭对应的使用隐藏字符进行替代的操作。

g:tex_isk g:tex_stylish

Tex: 控制 iskeyword

通常，LaTeX 关键字只支持 0-9, a-z, A-Z, 192-255。Latex 关键字不支持下划线，除了 *.sty 文件以外。语法高亮脚本使用以下的逻辑：

```
* 如果 g:tex_stylish 存在且为 1
        文件被作为 "sty" 文件对待，允许 "_" 成为关键字一的部分
        (独立于 g:tex_isk)
* 否则如果文件名后缀是 sty、cls、clo、dtx 或 ltx
        文件被作为 "sty" 文件对待，允许 "_" 成为关键字一的部分
        (独立于 g:tex_isk)

* 如果 g:tex_isk 存在，它被用作局部的 'iskeyword'
* 否则局部的 'iskeyword' 设为 48-57,a-z,A-Z,192-255
```

tex-supersub g:tex_superscripts g:tex_subscripts

Tex: 下标和上标的精细控制

关于如何打开隐藏字符的替换，见 tex-conceal 。

关于如何选择性的隐藏重音、粗体/斜体、数学、希腊文、和上标/下标，见 g:tex_conceal 。

可以更加精细地控制基于语法地隐藏哪些上标和下标（见 :syn-cchar ）。因为不是所有的字体都支持所有字符，可以覆盖隐藏替换列表；缺省这些列表是这样给出的：

```
let g:tex_superscripts= "[0-9a-zA-W.,:;+-<>/()=]"
let g:tex_subscripts= "[0-9aehijklmnoprstuvx,+-/().]"
```

例如，我用 Luxi Mono Bold；它不支持下标字符 "hklmnpst"，所以我在 ~/.vim/ftplugin/tex/tex.vim 里放上

```
let g:tex_subscripts= "[0-9aeijoruvx,+-/().]"
```

以防止出现不可理解的 utf8 字形。

TF tf.vim ft-tf-syntax

tf 语法高亮有一个选项。

同步的 minlines 缺省为 100。如果你希望设为别的值，可以把 "tf_minlines" 设为你希望的值。例如：

```
:let tf_minlines = 你的选择
```

VIM vim.vim ft-vim-syntax
g:vimsyn_minlines g:vimsyn_maxlines

准确的语法高亮和屏幕刷新速度需要一定的折衷。要提高准确性，你可能想增加 g:vimsyn_minlines 变量的值。而 g:vimsyn_maxlines 变量可以用来增加屏幕的刷新速度（详情可见 :syn-sync ）。

```
g:vimsyn_minlines : 用于设置同步的 minlines
g:vimsyn_maxlines : 用于设置同步的 maxlines

(g:vim_minlines 和 g:vim_maxlines 是这些选项过时的名字)
```

g:vimsyn_embed

g:vimsyn_embed 选项允许用户选择是否使用及使用何种类型的嵌入脚本高亮。

```
g:vimsyn_embed == 0   : 不支持任何内嵌脚本
g:vimsyn_embed =~ 'l' : 支持内嵌 lua
g:vimsyn_embed =~ 'm' : 支持内嵌 mzscheme
g:vimsyn_embed =~ 'p' : 支持内嵌 perl
g:vimsyn_embed =~ 'P' : 支持内嵌 python
g:vimsyn_embed =~ 'r' : 支持内嵌 ruby
g:vimsyn_embed =~ 't' : 支持内嵌 tcl
```

g:vimsyn_embed 缺省是代表 vim 自身支持的解释器的字符串。连接多个字符以支持多个内嵌解释器的类型；例如 g:vimsyn_embed= "mp" 支持内嵌的 mzscheme 和内嵌 perl。

g:vimsyn_folding

syntax/vim.vim 现在支持一些折叠：

```
g:vimsyn_folding == 0 或不存在: 没有基于语法的折叠
g:vimsyn_folding =~ 'a' : 自动命令组
```

```
g:vimsyn_folding =~ 'f' : 折叠函数
g:vimsyn_folding =~ 'l' : 折叠 lua       脚本
g:vimsyn_folding =~ 'm' : 折叠 mzscheme 脚本
g:vimsyn_folding =~ 'p' : 折叠 perl      脚本
g:vimsyn_folding =~ 'P' : 折叠 python    脚本
g:vimsyn_folding =~ 'r' : 折叠 ruby      脚本
g:vimsyn_folding =~ 't' : 折叠 tcl       脚本
```

g:vimsyn_noerror

syntax/vim.vim 给出的错误高亮未必都正确；Vim 脚本是一个要正确高亮难度很高的语言。如果不想出现错误高亮，在你的 vimrc 里放上：

```
let g:vimsyn_noerror = 1
```

XF86CONFIG

xf86conf.vim ft-xf86conf-syntax

XFree86 v3.x 和 v4.x 版本里，XF86Config 文件的语法有所不同。两者都支持且有自动检测，但离完善还很远。你可能仍然需要手动指定版本。根据你的 XFree86 的版本，在 .vimrc 里把 xf86conf_xfree86_version 变量设为 3 或 4。例如：

```
:let xf86conf_xfree86_version=3
```

如果混合使用多种版本，设置 b:xf86conf_xfree86_version 变量。

注意 不支持选项名的空格和下划线。如果你想高亮选项名，使用 "SyncOnGreen" 而不是 "__s yn con gr_e_e_n"。

XML

xml.vim ft-xml-syntax

缺省高亮 Xml 的命名空间。设置了下面的全局变量以后可以继承此设置：

```
:let g:xml_namespace_transparent=1
```

xml-folding

xml 语法文件提供打开和关闭标签间的语法折叠 folding （见 :syn-fold ）。这可以用下面的代码打开

```
:let g:xml_syntax_folding = 1
:set foldmethod=syntax
```

注意：语法折叠会显著地减慢语法高亮。大文件尤其如此。

X Pixmaps (XPM)

xpm.vim ft-xpm-syntax

xpm.vim 根据 XPM 文件的内容动态地建立语法项目。这样你就可以修改色彩规格字符串。修改后，可用 ":set syn=xpm" 再次执行。

要复制带某颜色的像素，使用 "yl" 命令抽出 "像素" 然后在别的地方使用 "P" 插入。

你想用鼠标画图么？试试这些代码：

```
:function! GetPixel()
:   let c = getline(".")[col(".") - 1]
:   echo c
```

```
:    exe "noremap <LeftMouse> <LeftMouse>r".c
:    exe "noremap <LeftDrag>  <LeftMouse>r".c
:endfunction
:noremap <RightMouse> <LeftMouse>:call GetPixel()<CR>
:set guicursor=n:hor20          " 可以看到光标下的颜色
```

这使得右键变成像素提取工具，而左键成为一支笔。该代码只可用于每个像素一个字符的 XPM 文件，而且你不能在像素字符串之外点击。不过，你自己可以尝试改进之。

如果使用大小减半的字体，看其来会舒服得多。比如，在 X 上：

```
:set guifont=-*-clean-medium-r-*-*-8-*-*-*-*-80-*
```

YAML yaml.vim ft-yaml-syntax

g:yaml_schema b:yaml_schema

YAML 模型是一个标签集合和解析非特定标签的机制的组合。对用户来说，这意味着取决于简单向量内容，YAML 分析器可把简单向量（实际可以只是字符串而没有其它）当作其它类型的值：null、布尔型、浮点数、整数。 g:yaml_schema 选择决定根据什么模型来对值进行特殊高亮。支持的模型为

模型	描述
failsafe	没有额外高亮。
json	支持 JSON 风格的数值、布尔型和 null。
core	支持更多风格的数值、布尔型和 null。
pyyaml	除了 core 模型以外，还支持高亮时间戳，但和 core 模型识别的数值有些区别，也有很多 core 模型没有的附加的布尔值。

缺省的模型是 core 。

注意 模型实际不局限于简单的向量，但这是 YAML 规格定义的模型的唯一区别，也是语法文件定义的唯一区别。

ZSH zsh.vim ft-zsh-syntax

zsh 的语法高亮脚本打开基于语法的折叠：

```
:let g:zsh_fold_enable = 1
```

---

6. 定义语法 :syn-define E410

Vim 理解三种语法项目的类型：

1. 关键字
   它只能包含由 'iskeyword' 选项定义的关键字字符，而且不能包含其它语法项目。
   它必须匹配完整的单词（在匹配的前后不能有其它的关键字字符）。
   关键词 "if" 只在 "if(a=b)" 里匹配，而不在 "ifdef x" 里匹配。因为 "(" 不是关键字字符，但 "d" 是。

2. 匹配
   它匹配单个正规表达式模式。

3. 区域

它始于 "start" 正规表达式模式的匹配，结束于 "end" 正规表达式模式的匹配。两者之间可以包含任何文本。其中，"skip" 正规表达式模式可以用来避免 "end" 模式的匹配。

若干语法*项目*可以放在一个语法*组*里。你可以为一个语法组设置高亮属性。例如，你可以定义 "/* .. */" 注释为一个项目，"// .." 注释为另一个，并把两者都放在 "Comment" 组里。这时，你就可以设置 "Comment" 以粗体字体和蓝色出现。你可以自由选择各种组合，比如为每个语法项目设置一个高亮组，乃至把所有项目都放到一个组。这取决于你如何指定你的高亮属性。把每个项目放到单独的组里的后果是你需要为很多组指定高亮属性。

注意 语法组和高亮组类似。你为高亮组指定高亮属性，而这些属性会被用于同名的语法组。

如果有多个项目在相同位置匹配，*最后*定义的那个胜出。这样，你可以覆盖较早定义的匹配相同文本的语法项目。不过，关键字总是优先于匹配和区域，而匹配大小写的关键字又优先于忽略大小写的关键字。

优 先 级 :syn-priority

如果多个语法项目可以匹配，使用如下规则：

1. 如果多个匹配或区域项目在相同的位置开始，最后定义者优先。
2. 关键字比匹配和区域项目优先。
3. 从较早位置开始的项目优先于从较后位置开始的项目。

定 义 大 小 写 敏 感 :syn-case E390

:sy[ntax] case [match | ignore]
: 要求其后的 ":syntax" 命令在本设定为 "match" 时必须匹配大小写，本设定为 "ignore" 时则可以忽略大小写。注意，它不影响之前的项目，而只影响其后直到下一个 ":syntax case" 命令为止的所有项目。

:sy[ntax] case
: 显示 "syntax case match" 或 "syntax case ignore" 之一 (经过翻译)。

拼 写 检 查 :syn-spell

:sy[ntax] spell [toplevel | notoplevel | default]
: 定义不在任何语法项目里的文本在何处进行拼写检查：

  toplevel: 文本进行拼写检查。
  notoplevel: 文本不进行拼写检查。
  default: 如果有 @Spell 簇，不进行拼写检查。

  语法项目里的文本用 @Spell 和 @NoSpell 簇 spell-syntax 。如果没有 @Spell 和 @NoSpell 簇，那么 "default" 和 "toplevel" 进行拼写检查。

  要激活拼写检查，必须置位 'spell' 选项。

:sy[ntax] spell
显示 "syntax spell toplevel"、"syntax spell notoplevel" 或 "syntax spell default" 之一（经过翻译）。

语法 ISKEYWORD 设置 :syn-iskeyword

:sy[ntax] iskeyword [clear | {option}]
定义关键字字符。类似于 'iskeyword' 选项，但只适用于语法高亮。

clear: 关闭语法特定的 iskeyword 设置，使用局部于缓冲区的 'iskeyword' 设置。
{option} 设置语法的 'iskeyword' 选项为新值。

示例：

```
:syntax iskeyword @,48-57,192-255,$,_
```

这使得语法特定的 iskeyword 选项包含所有的字母、数位，所有的重音字符，还包含 "_" 和 "$"。

如果不给出参数，输出当前值。

此选项的设置影响语法模式中的 /\k 匹配，也决定新匹配会如何检查 :syn-keyword 。

建议写语法文件时，使用此命令也为特定的语法语言设置正确的值，而不改变 'iskeyword' 选项。

定义关键字 :syn-keyword

:sy[ntax] keyword {group-name} [{options}] {keyword} .. [{options}]

定义一系列关键字。

{group-name} 是语法组名，比如 "Comment"。
[{options}] 见下 :syn-arguments 。
{keyword} .. 是关键字列表，这些关键字成为该组的成员。

示例：

```
:syntax keyword   Type   int long char
```

{options} 可以在该行的任何位置给出。它们应用于所有的关键字，包括选项之前的关键字。以下例子完全相同：

```
:syntax keyword   Type   contained int long char
:syntax keyword   Type   int long contained char
:syntax keyword   Type   int long char contained
```

E789 E890

如果像 Vim 里的 Ex 命令那样，使用有可选尾部的关键字并把可选字符放在 [] 里，你可以一次定义各种变化形式：

```
:syntax keyword   vimCommand   ab[breviate] n[ext]
```

不要忘记只有所有字符都包含在 'iskeyword' 选项里才能作为关键字识别。如果有一个字符不是，该关键字永远不会被识别。不过，可以使用多字节字符，它们不需要出现在 'iskeyword' 里。

:syn-iskeyword 说明如何定义语法特定的 iskeyword 设置。

关键字比匹配和区域有更高的优先级。如果有多个项目匹配，会优先使用关键字。关键字不会嵌套，也不能包含其它项目。

注意 你不能使用和选项同名的关键字（即使这里不允许的选项也不行）。这时，应该使用匹配。

关键字的最大长度为 80 个字符。

根据被包含与否的不同，可以多次定义同一关键字，例如，你可以定义一次不被包含的关键字并使用一个高亮组。而为被包含的同一关键字使用不同的高亮组。例如：

```
:syn keyword vimCommand tag
:syn keyword vimSetting contained tag
```

如果发现独立于别的语法项目之外的 "tag"，使用 "vimCommand" 高亮组。如果发现 "tag" 出现在能包含 "vimSetting" 的项目里，则使用 "vimSetting"。

定义匹配 :syn-match

```
:sy[ntax] match {group-name} [{options}]
                [excludenl]
                [keepend]
                {pattern}
                [{options}]
```

定义一个匹配。

{group-name}	语法组名，比如 "Comment"。
[{options}]	见下 :syn-arguments 。
[excludenl]	使得包含行尾匹配 "$" 的模式不扩展包含本项目的匹配或者区域项目。必须在模式之前给出。 :syn-excludenl
keepend	不允许被包含的匹配项目超越结束模式的匹配文本。见 :syn-keepend 。
{pattern}	定义匹配的搜索模式。见下面的 :syn-pattern 。 注意 模式可以匹配多于一行的模式，这使得匹配依赖于 Vim 从哪里开始搜索模式。你需要确信同步机制能正确处理这个问题。

例如（匹配字符常数）：

```
:syntax match Character /'.'/hs=s+1,he=e-1
```

定义区域 :syn-region :syn-start :syn-skip :syn-end E398 E399

```
:sy[ntax] region {group-name} [{options}]
                 [matchgroup={group-name}]
                 [keepend]
                 [extend]
                 [excludenl]
                 start={start_pattern} ..
```

[skip={skip_pattern}]
end={end_pattern} ..
[{options}]

定义一个区域。区域可以覆盖多行。

{group-name}	语法组名，比如 "Comment"。
[{options}]	见下 :syn-arguments 。
[matchgroup={group-name}]	其后定义的开始或者结束模式的匹配文本专用的语法组。该语法组并不用于两者之间的文本。如果不想开始或结束匹配使用另外的语法组，可以用 NONE 作为组名来复位。见 :syn-matchgroup 。
keepend	不允许被包含的匹配项目越过结束模式的匹配文本。见 :syn-keepend 。
extend	否决包含本区域的项目的 "keepend" 设置。见 :syn-extend 。
excludenl	使得包含行尾匹配 "$" 的模式不扩展包含本项目的匹配或者区域。只适用于结束模式。必须在该模式之前给出。 :syn-excludenl
start={start_pattern}	定义区域开始的搜索模式。见下 :syn-pattern 。
skip={skip_pattern}	定义不需要查找结束模式的区域内部文本的搜索模式。见下 :syn-pattern 。
end={end_pattern}	定义区域结束的搜索模式。见下 :syn-pattern 。

示例：

```
:syntax region String  start=+"+  skip=+\\"+  end=+"+
```

开始 / 跳过 / 结束模式和其它选项可用任何顺序给出。跳过模式可有零到一个。开始和结束模式必须有一个或更多。这意味着，你可以忽略跳过模式，但你必须给出至少一个的开始和结束模式。在等号前后，可以使用空白字符（不过多数情况下，其实没有空白字符更清楚一些）。

如果给出多于一个的开始模式，只须匹配其中任何一个。这意味着这些开始模式之间是*或*的关系。如有多个匹配，使用最后一个。结束模式也是如此。

结束模式从开始模式之后立即开始搜索，这里不考虑位移。这意味着，结束模式的匹配文本永远不会与开始模式的重叠。

跳过和结束模式可以跨行匹配，但因为模式的搜索可以从任何一行开始，这经常不能如你所愿。跳过模式也不能避免下一行中结束模式的匹配。要避免麻烦，最好使用单行的模式。

注意：一个区域的开始完全取决于开始模式的匹配。不检查是否存在结束模式的匹配。下面 不能 工作：

```
:syn region First  start="("  end=":"
:syn region Second start="("  end=";"
```

Second 总是在 First 之前得到匹配（最后定义的模式享有更高的优先权）。Second 区域会继续到后面出现的 ';' 为止。这之前是否出现 ':' 无关紧要。要解决这个问题，可以使用匹配：

```
:syn match First  "(\_.\{-}:"
:syn match Second "(\_.\{-};"
```

该模式使用 "_." 匹配任何字符或换行符，并以 "\{-}" 重复（重复尽量少的次数）。

:syn-keepend

缺省，被包含的项目可以隐藏结束模式的匹配。这对嵌套有用。比如，"{" 开始 "}" 结束的区域可以包含另一个这样的区域。第一个遇到的 "}" 会结束被包含的区域，不是外面的那个：

```
{           开始外层 "{}" 区域
    {       开始被包含的 "{}" 区域
    }       结束被包含的 "{}" 区域
}           结束外层 "{} 区域
```

如果你不希望如此，"keepend" 参数可以使得外层区域结束模式的匹配同时结束任何包含在内的项目。这使得相同区域的嵌套不再可能，但可以允许被包含项目高亮结束模式的部分内容，而不会因此跳过结束模式的匹配。例如：

```
:syn match  vimComment +"[^"]\+$+
:syn region vimCommand start="set" end="$" contains=vimComment keepend
```

"keepend" 使得 vimCommand 总是在行尾结束，即使被包含的 vimComment 包括了 <EOL> 的匹配也不例外。

如果不使用 "keepend"，在每个被包含项目的匹配文本之后，才会重新尝试结束模式的匹配。如果使用 "keepend"，寻找结束模式第一次出现的匹配，并截断任何被包含的项目。

:syn-extend

"keepend" 的行为可以被 "extend" 参数改变。当一个项目使用 "extend" 的时候，包含它的外层项目所用的 "keepend" 被忽略，从而使得那个项目可以得到扩展。
这可以使一些被包含的项目能扩展某区域，而另一些则不能。例如：

```
:syn region htmlRef start=+<a>+ end=+</a>+ keepend contains=htmlItem,htmlScript
:syn match htmlItem +<[^>]*>+ contained
:syn region htmlScript start=+<script+ end=+</script[^>]*>+ contained extend
```

这里，htmlItem 项目不扩展 htmlRef 项目，它只是用来高亮 <> 条目。而 htmlScript 项目则扩展 htmlRef 项目。

另一个例子：

```
:syn region xmlFold start="<a>" end="</a>" fold transparent keepend extend
```

定义使用 "keepend" 的区域，使得它的结尾不会被包含在内的项目所改变，比如匹配 "" 以赋予不同高亮属性的那些项目。但如果 xmlFold 区域本身嵌套（自己包含自己），则应用 "extend"，使得嵌套在内的 "" 只会结束里面的区域，而不是包含该区域的外层区域。

:syn-excludenl

当一个匹配项目的模式或者区域项目的结束模式包含 '$' 以匹配行尾的时候，包含该项目的外部区域项目会在下一行继续。比如，使用 "\\$"（行尾出现的反斜杠）的匹配项目可以使得一个通常在行尾结束的区域继续下去。这是缺省的行为。如果你不希望如此，有两个解决方法：

1. 外部项目使用 "keepend"。这会使得所有被包含的项目不能扩展外部的匹配或区域项目。这可用于所有被包含的项目都不能扩展外部项目的场合。
2. 在被包含的项目中使用 "excludenl"。这使得该匹配不能扩展外部的匹配或区域项目。这可用于只有一些被包含的项目不需要扩展外部项目的场合。"excludenl" 必须在它适用的模式之前给出。

:syn-matchgroup

"matchgroup" 可以用来高亮开始和/或结束模式，使之和区域本体不同。例如：

```
:syntax region String matchgroup=Quote start=+"+  skip=+\\"+  end=+"+
```
会使得引号本身使用 "Quote" 组高亮。而其中的文本使用 "String" 高亮组。"matchgroup" 用于其后所有的开始和结束模式，直到下一个 "matchgroup" 为止。使用 "matchgroup=NONE" 回到不使用 matchgroup 的情况。

用 "matchgroup" 高亮的开始或结束模式的匹配文本不会用于包含在该区域内的项目。这样可以避免被包含项目也能在开始或结束模式的匹配文本中匹配。"transparent" 选项不适用于使用 "matchgroup" 高亮的开始或结束模式的匹配文本。

这里是一个例子，它以不同的颜色高亮三层括号：

```
:sy region par1 matchgroup=par1 start=/(/ end=/)/ contains=par2
:sy region par2 matchgroup=par2 start=/(/ end=/)/ contains=par3 contained
:sy region par3 matchgroup=par3 start=/(/ end=/)/ contains=par1 contained
:hi par1 ctermfg=red guifg=red
:hi par2 ctermfg=blue guifg=blue
:hi par3 ctermfg=darkgreen guifg=darkgreen
```

E849

语法组的最大数目为 19999。

---

7. :syntax 参数 :syn-arguments

定义语法项目的 :syntax 命令接受多个参数。其中，通用的部分在这里解释。这些参数可以用任何顺序给出，也可和模式相互间杂。

不是所有的命令都接受每个参数。下表显示什么参数不能在所有命令里使用：

E395

	contains	oneline	fold	display	extend	concealends
:syntax keyword	-	-	-	-	-	-
:syntax match	是	-	是	是	是	-
:syntax region	是	是	是	是	是	是

以下参数可以在所有三个命令里使用：

```
conceal
cchar
contained
containedin
nextgroup
transparent
skipwhite
skipnl
skipempty
```

conceal conceal :syn-conceal

如果给出 "conceal" 参数，本项目被标为可隐藏。实际隐藏与否取决于 'conceallevel' 选项的值。'concealcursor' 选项决定当前行的可隐藏项目是否会以正常方式显示，以便对本行进行编辑。
另一个隐藏文本的办法是用 matchadd() 。

concealends :syn-concealends

如果给出 "concealends" 参数，区域的开始和结束匹配（不包括区域内容本身）被标为可隐藏，实际隐藏与否取决于 'conceallevel' 选项的设置。只有这种方式，区域结束处才可以用 "matchgroup" 来定义自己单独的高亮隐藏方式。

cchar :syn-cchar
E844

"cchar" 参数定义代替可隐藏项目的字符（只有给出 conceal 参数时设置 "cchar" 才有意义）。如果没有给出 "cchar"，缺省的隐藏代替字符是 'listchars' 选项定义的字符。该字符不能是控制字符，如制表符。示例：

```
:syntax match Entity "&amp;" conceal cchar=&
```

关于高亮，见 hl-Conceal 。

contained :syn-contained

如果给出 "contained" 参数，本项目在顶层不会被识别。只有包含在另外一个项目里才可以，而且那个项目必须给出 "contains" 参数且其中包含本项目。例如：

```
:syntax keyword Todo    TODO    contained
:syntax match   Comment "//.*"  contains=Todo
```

display :syn-display

如果给出 "display" 参数，本项目会在检测到高亮不会显示的时候被跳过。这样，高亮速度可以加快，因为只要发现要显示的文本的语法状态就可以跳过本项目。

通常，你使用 "display" 来匹配满足以下条件的匹配和区域项目：
- 该项目不会跨过行尾。C 的例子：包含 "/*" 的注释不能使用 "display"，因为它会在下一行继续。
- 该项目不包含会跨过行尾或使得本项目在下一行继续的项目。
- 该项目不改变任何包含它在内的项目的大小。C 的例子：预处理指令里的匹配 "\\$" 不能使用 "display"，因为它可以使得预处理指令的匹配变短。
- 该项目不允许其它项目匹配本来不能匹配的内容，而所扩展的匹配文本会走的太远。C 的例子：定义 "//" 注释的匹配不能使用 "display"，因为在该注释里的 "/*" 这时会匹配并开始一个跨越行尾的注释。

例如，在 C 程序里，"display" 可以用在：
- 数值的匹配
- 标签的匹配

transparent :syn-transparent

如果给出 "transparent"（透明）参数，本项目自身不会被高亮，但会使用包含它的外层项目的高亮属性。这对本身不需要特殊高亮但要用来跳过一段文本的语法项目有用。

除非包含 transparent 的这个项目本身包含了 "contains" 参数，"contains=" 参数会从外层的项目继承，要避免包含不需要的项目，可使用 "contains=NONE"。例如，高亮字符串里的单词，但不包括 "vim"：

```
:syn match myString /'[^']*'/ contains=myWord,myVim
:syn match myWord   /\<[a-z]*\>/ contained
:syn match myVim    /\<vim\>/ transparent contained contains=NONE
:hi link myString String
:hi link myWord   Comment
```

"myVim" 匹配出现在 "myWord" 之后，因而它是更优先的匹配（在相同的位置上，后出

现的匹配优先于先出现的匹配)。"transparent" 参数使得 "myVim" 的匹配使用和 "myString" 相同的高亮，但它本身不再包含其它项目。如果没有指定 "contains=NONE" 参数，那么 "myVim" 会使用 myString 的 contains 参数，从而包含了 "myWord"，因而被高亮为 Constant (译者注: 应为 Comment)。这之所以会发生，是因为被包含的项目不会在同样的位置上匹配外层同一个语法项目，所以这里，在最内层的 "myVim" 匹配不能否决 "myWord" 的匹配。

如果你看有色彩的文本，它实际上是由一层层被包含的项目组成的。被包含的项目在包含它的项目之上，因而你能看到被包含的项目。如果一个被包含的项目是透明的，你会看透它，从而看到包含它的项目。以图示之:

```
                从这里看

            |   |   |   |   |   |
            V   V   V   V   V   V

               xxxx        yyy          被包含更深的项目
              .....................      被包含项目 (透明)
          ============================   第一个项目
```

'x'、'y' 和 '=' 分别表示一种高亮的语法项目。'.' 代表透明层。

你实际看到的是:

```
        =======xxxx=======yyy========
```

这里，你 "看穿" 了透明的 "...."。

oneline :syn-oneline

"oneline" 参数指示本区域不会跨过行边界。它必须在当前行内完整匹配。不过，如果本区域包含跨行的项目，那么本区域还是会从下一行继续。被包含的项目可以用来识别续行模式。不过，结束模式必须仍然在第一行上匹配，不然本区域根本不会开始。

如果开始模式包含 "\n" 从而匹配换行符，结束模式必须在开始模式结束处所在的同一行上找到。该结束模式也可以包含换行符。因而，"oneline" 参数只是意味着开始模式的结束处和结束模式的开始处必须在同一行上。这一点，即使跳过模式包含换行符也不能改变。

fold :syn-fold

"fold" 参数使得本项目的折叠级别加 1。示例:

```
:syn region myFold start="{" end="}" transparent fold
:syn sync fromstart
:set foldmethod=syntax
```

这使得每个 {} 块形成一个折叠。

该折叠从项目开始的行开始，而在项目结束之行上结束。如果开始和结束处在同一行，则不形成折叠。'foldnestmax' 选项限制语法折叠的嵌套级别。
{仅当 Vim 编译时带 +folding 特性才有效}

:syn-contains E405 E406 E407 E408 E409

contains={group-name},..

"contains" 参数跟随语法组名的列表。这些组因而被允许包含在本项目里（它们可能扩展包含它们的组的结束位置）。这使得匹配和区域的递归嵌套成为可能。如果没有 "contains" 参数，本项目不能包含任何组。组名不需要在这里的使用前定义。

contains=ALL
如果唯一在包含列表里出现的名字是 "ALL"，那么本项目里可以包含所有的组。

contains=ALLBUT,{group-name}...
如果包含列表的第一个名字是 "ALLBUT"，那么除了列出的那些以外，所有的组都可以出现在本项目里。例如：

```
:syntax region Block start="{" end="}" ... contains=ALLBUT,Function
```

contains=TOP
如果包含列表的第一个名字是 "TOP"，那么所有不包含 "contained" 参数的组都可以接受。

contains=TOP，{group-name},..
类似于 "TOP"，但除了列出的组以外。

contains=CONTAINED
如果包含列表的第一个名字是 "CONTAINED"，那么所有包含 "contained" 参数的组都可以接受。

contains=CONTAINED，{group-name},..
类似于 "CONTAINED"，但列出的组除外。

"contains" 列表里的 {group-name} 可以是模式。所有匹配该模式的组名都会包含进来（或者排除出去，如果使用 "ALLBUT" 的话）。该模式不能包含空白或者 ’,’。例如：

```
... contains=Comment.*,Keyw[0-3]
```

在执行 syntax 命令时完成该匹配。后来定义的组不会再参与匹配。另外，如果本 syntax 命令自己定义一个新组，它也不会参与匹配。小心：在文件里放入 syntax 命令的时候，你不能指望某些组 没有 定义，因为该文件以前可能已经执行过。而 ":syn clear" 并不会删除组名。

被包含的组也会匹配一个区域项目的开始和结束模式。如果不想如此，可以用 "matchgroup" 参数 :syn-matchgroup 。"ms=" 和 "me=" 位移可以用来调整被包含项目能够匹配的区域。注意 这同时可能限制了高亮的区域。

containedin={group-name}... :syn-containedin

"containedin" 参数跟随语法组名的列表。然后，本项目就可以包含在那些组里，就像那些项目使用了包含本项目的 "contains=" 参数一样。

{group-name}... 的使用方式和 "contains" 相同，见上面的解释。

这可以用于后加的语法项目。一个项目可以告知要包含在已经存在的项目里面，而无须修改后者的定义。例如，要在载入 C 语法以后高亮 C 注释里的一个单词：

```
:syn keyword myword HELP containedin=cComment contained
```

注意 同时使用了 "contained" 参数是为了避免本项目在顶层得到匹配。

"containedin" 的匹配被加到该项目可以出现的其它地方。像平常一样，本项目也可以使用 "contains" 参数。不要忘记关键字项目不可能包含其它项目。所以把它们加在 "containedin" 里是徒劳的。

nextgroup={group-name},.. :syn-nextgroup

"nextgroup" (下一个组) 参数跟随语法组名的列表，以逗号分隔 (和 "contains" 类似，你也可以使用模式)。

如果给出 "nextgroup" 参数，提到的语法组会在本匹配或区域结束之后尝试匹配。如果没有一个组能够匹配，高亮和平常一样继续。如果有匹配，则使用匹配的组高亮，即使该组没有在当前组的 "contains" 字段里提到，就像这里的组给赋予了比其它的组更高的优先级一样。例如:

```
:syntax match  ccFoobar  "Foo.\{-}Bar"  contains=ccFoo
:syntax match  ccFoo     "Foo"           contained nextgroup=ccFiller
:syntax region ccFiller  start="."  matchgroup=ccBar  end="Bar"  contained
```

会分别高亮 "Foo" 和 "Bar"，但只有在 "Bar" 跟在 "Foo" 之后才行。在下面的文本行里，"f" 显示使用 ccFoo 的高亮，而 "bbb" 则是使用 ccBar 的地方。

```
Foo asdfasd Bar asdf Foo asdf Bar asdf
fff         bbb      fff      bbb
```

注意 ".\{-}" 的使用跳过尽可能少的内容以到达下一个 Bar。如果使用了 ".*"，"Bar" 和 "Foo" 之间的 "asdf" 会以 "ccFoobar" 组高亮，因为 ccBar 匹配行内第一个 "Foo" 和最后一个 "Bar" 的全部内容 (见 pattern)。

skipwhite :syn-skipwhite
skipnl :syn-skipnl
skipempty :syn-skipempty

这些参数只能和 "nextgroup" 组合使用。它们可以用来指定如何跳过一些文本到达下一个组:

skipwhite	跳过空格和制表字符
skipnl	跳过换行符
skipempty	跳过空行 (意味着 "skipnl")

如果 "skipwhite" 存在，空白字符只有在没有下一个组会匹配空白时才会跳过。

如果 "skipnl" 存在，下一个组的匹配会在第二行内寻找。这只有在当前项目在行尾结束时才会发生！如果 "skipnl" 不存在，下一个组只会在当前项目相同的行上寻找。

如果在寻找下一个组时跳过文本，其它组的匹配就会忽略。只有在没有下一个组匹配时，其它的项目才会重新尝试被匹配。这意味着下一个组，包括跳过的空白和 <EOL>，的匹配比其它项目的优先级要高。

示例:

```
:syn match ifstart "\<if.*"   nextgroup=ifline skipwhite skipempty
:syn match ifline  "[^ \t].*" nextgroup=ifline skipwhite skipempty contained
:syn match ifline  "endif"    contained
```

注意 "[^ \t].*" 匹配所有的非空白文本。这样，它也能匹配 "endif"。所以 "endif"

匹配项目必须放在最后，以取得优先权。
注意 本例子不适用于嵌套的 "if"。你需要加上 "contains" 参数才行（本例之所以省略，是为了简洁起见）。

隐含隐藏 :syn-conceal-implicit

:sy[ntax] conceal [on|off]
        指令后续的 ":syntax" 命令所定义的关键字、匹配或区域是否会自动带上 "conceal" 标志位。":syn conceal on" 之后的所有 ":syn keyword"、":syn match" 或 ":syn region" 会隐含地带上 "conceal" 标志位。而 ":syn conceal off" 回复到正常状态，必须显式给出 "conceal" 标志位。

:sy[ntax] conceal
        显示 "syntax conceal on" 或 "syntax conceal off" 之一（经过翻译）。

---

8. 语法模式 :syn-pattern E401 E402

在 syntax 命令里，模式必须被两个相同的字符包围。这和 ":s" 命令类似。最常用的是双引号。但如果模式本身包含双引号，你可以使用别的不在模式里出现的字符。例如：

```
:syntax region Comment  start="/\*"  end="\*/"
:syntax region String   start=+"+    end=+"+   skip=+\\"+
```

关于模式的解释，见 pattern 。对语法模式的解释总是假设打开了 'magic' 选项，而与实际的 'magic' 值无关。而且也假设 'cpoptions' 里没有 'l' 标志位。这些设置使得语法文件易移植，而独立于 'compatible' 和 'magic' 的设置。

要避免能够匹配空字符串的模式，比如 "[a-z]*"。这会显著减慢高亮，因为这样的模式会在任何地方得到匹配。

:syn-pattern-offset
模式可以后跟一个字符位移。它可以用来修改高亮的部分，也可以修改匹配或区域项目的文本区域（只有其它项目的匹配会受影响）。两者都是相对于已经匹配的模式的。跳过模式的字符位移可以用来指示从哪里开始继续寻找结束模式。

位移的形式是 "{what}={offset}"
{what} 可以是七种字符串之一：

ms	匹配开始	匹配文本开始的偏移
me	匹配结束	匹配文本结束的偏移
hs	高亮开始	高亮开始的偏移
he	高亮结束	高亮开始的偏移
rs	区域开始	区域本体开始的偏移
re	区域结束	区域本体结束的偏移
lc	引导上下文	超越模式的 "引导上下文" 的偏移

{offset} 可以是：

s	模式匹配部分的开始位置
s+{nr}	模式匹配部分的开始位置向右 {nr} 个字符
s-{nr}	模式匹配部分的开始位置向左 {nr} 个字符
e	模式匹配部分的结束位置
e+{nr}	模式匹配部分的结束位置向右 {nr} 个字符
e-{nr}	模式匹配部分的结束位置向左 {nr} 个字符

{nr}　(只用于 "lc"): 从开始处向右 {nr} 个字符开始匹配

例如: "ms=s+1", "hs=e-2", "lc=3"。

尽管所有的位移形式在任何模式之后都能接受，它们不都有意义。下表说明什么样的位移在实际中会应用到:

	ms	me	hs	he	rs	re	lc
匹配项目	是	是	是	是	-	-	是
区域项目开始模式	是	-	是	-	是	-	是
区域项目跳过模式	-	是	-	-	-	-	是
区域项目结束模式	-	是	-	是	-	是	是

位移可用 ',' 连接。例如:

```
:syn match String  /"[^"]*"/hs=s+1,he=e-1
```

```
  一些 "字符串" 文本
       ^^^^^^               高亮部分
```

注意:
- 模式和位移字符之间不能有空白。
- 高亮区域永远不会超出匹配文本的范围。
- 结束模式上的负偏移不一定总能用，因为结束模式可能在高亮本应结束的地方才检测到。
- Vim 7.2 之前，位移用字节数而不是字符数计算。这对多字节字符不适用，所以 Vim 7.2 版本发行时改了。
- 匹配不能从实际匹配的模式所在的行之外开始。这样不行: "a\nb"ms=e。高亮可以从另一个行开始，这样没有问题: "a\nb"hs=e。

示例 (匹配注释，但不高亮 /* 和 */):

```
:syntax region Comment start="/\*"hs=e+1 end="\*/"he=s-1
```

```
      /* 这是一个注释 */
        ^^^^^^^^^^^^^^          高亮部分
```

一个更复杂的例子:

```
:syn region Exa matchgroup=Foo start="foo"hs=s+2,rs=e+2 matchgroup=Bar end="bar"me=e-1,he=e-1,re
```

```
      abcfoostringbarabc
         mmmmmmmmmmm      匹配
           ssrrrreee      高亮 开始 (s)/区域 (r)/结束 (e)
                              ("Foo"、"Exa" 和 "Bar")
```

引导上下文　　:syn-lc　:syn-leading　:syn-context

注意: 这是一个已经废弃的特性，包含它只是为了和以前的 Vim 版本后向兼容。现在，我们推荐使用在模式里使用 /\@<= 构造。

"lc" 位移指定引导上下文 -- 模式的一部分: 必须存在，但不包含在实际的匹配中。形如 "lc=n" 的位移会使得 Vim 在试图匹配模式前先后退 n 列，从而可以使得已在较早的模式中匹配的字符仍然可以做为本匹配的引导上下文。这可用于，比如说，要求本匹配不出现在某个前导的 "转义" 字符之后:

```
:syn match ZNoBackslash "[^\\]z"ms=s+1
```

```
:syn match WNoBackslash "[^\\]w"lc=1
:syn match Underline "_\+"

        ___zzzz ___wwww
        ^^^     ^^^     匹配 Underline
            ^ ^         匹配 ZNoBackslash
                  ^^^^  匹配 WNoBackslash
```

"ms" 位移自动设为与 "lc" 位移相同的值，除非你显式地设置 "ms"。

多行模式 :syn-multi-line

模式里可以包含 "\n" 匹配换行符。多数情况下，它能正常工作，但有以下一些例外。

使用带位移的开始模式时，匹配不允许从真实匹配之后的行开始。不过，高亮不存在这样的问题。"\zs" 项目也同样需要匹配的开始不能移动到另一行。

跳过模式可以包含 "\n"，但结束模式的搜索会从下一行的第一个字符开始，即使跳过模式匹配该字符也是如此。这是因为重画可以从区域中间的任何一行启动，而不会检查跳过模式是否从那一行之前就已经开始的缘故。例如，如果跳过模式是 "a\nb" 而结束模式为 "b"，结束模式的确会匹配下面情况的第二行：

```
        x x a
        b x x
```

通常，这意味着跳过模式不应匹配 "\n" 之后的任何字符。

外部匹配 :syn-ext-match

以下附加的正规表达式项目可以用在区域项目的模式中：

/\z(/\z(\) E50 E52 E879

\z(\) 标记该子表达式为 "外部的"，这意味着它可以在别的模式匹配里访问。目前，只能在语法区域的开始模式中应用。

/\z1 /\z2 /\z3 /\z4 /\z5
/\z6 /\z7 /\z8 /\z9 E66 E67

\z1 ... \z9 匹配和前面的开始模式匹配里相应子表达式的匹配的相同的字符串。

有时区域项目的开始和结束模式需要共享一个相同的子表达式。常见的例子是 Perl 和许多 Unix 外壳里的 "here" 文档。这种效果可以通过特殊的 "\z" 正规表达式项目完成。它把子表达式标为 "外部的"，也就是说可以从定义所在的模式的外部引用。例如，here 文档的例子可以如此完成：

```
:syn region hereDoc start="<<\z(\I\i*\)" end="^\z1$"
```

由此可见，\z 实际上有双重任务。在开始模式里，它标记 "\(\I\i*\)" 子表达式为外部的；在结束模式里，它把 \z1 反向引用重新定义为指向开始模式里的第一个外部子表达式的外部引用。跳过模式里也可以使用外部引用：

```
:syn region foo start="start \(\I\i*\)" skip="not end \z1" end="end \z1"
```

注意 普通和外部子表达式是完全不相关联，它们分别索引。如果模式 "\z(..\)\(..\)" 应用于字符串 "aabb"，\1 会指向 "bb" 而 \z1 会指向 "aa"。也要 注意，外部子表达式不能和普通的子表达式那样，在同一个模式里作为反向引用来访问。如果你需要把一个

子表达式同时作为普通和外部子表达式来使用，可以嵌套使用这两者，形如 "\(\z(...\)\)"。

注意 这里只能使用行内的匹配，不能从外部引用多行匹配。

---

9. 语法簇 :syn-cluster E400

:sy[ntax] cluster {cluster-name} [contains={group-name}..]
[add={group-name}..]
[remove={group-name}..]

本命令允许你把若干语法组捆绑在一起，以便使用单个名字访问。

contains={group-name}..
本簇使用此处指定的组名列表。
add={group-name}..
将指定的组加入本簇。
remove={group-name}..
将指定的组从本簇里删除。

用这种方式定义的簇可以在 contains=..、containedin=..、nextgroup=..、add=.. 或者 remove=.. 的列表里使用，只要加上 "@" 前缀即可。用这种表示方式，你可以隐含地在指定一个簇的内容之前先声明之。

示例：

```
:syntax match Thing "# [^#]\+ #" contains=@ThingMembers
:syntax cluster ThingMembers contains=ThingMember1,ThingMember2
```

如同前例所暗示的，对簇的修改追溯既往；可以说，在最后一刻才进行簇成员身份的检查：

```
:syntax keyword A aaa
:syntax keyword B bbb
:syntax cluster AandB contains=A
:syntax match Stuff "( aaa bbb )" contains=@AandB
:syntax cluster AandB add=B    " 现在两个关键字都在 Stuff 里匹配
```

对嵌套的簇而言，这也有若干意含：

```
:syntax keyword A aaa
:syntax keyword B bbb
:syntax cluster SmallGroup contains=B
:syntax cluster BigGroup contains=A,@SmallGroup
:syntax match Stuff "( aaa bbb )" contains=@BigGroup
:syntax cluster BigGroup remove=B    " 没有效果，因为 B 不在 BigGroup 里
:syntax cluster SmallGroup remove=B  " 现在 Stuff 不再匹配 bbb
```

E848

簇的最大数目是 9767。

---

10. 包含语法文件 :syn-include E397

一个语言的语法文件经常需要包含相关语言的语法文件。取决于它们实际的关系，可以用

两种不同的方式完成：

- 如果允许被包含的语法文件里的顶层的语法项目也出现在包含它的语法的顶层中，可以简单的使用 :runtime 命令：

```
" 在 cpp.vim 里:
:runtime! syntax/c.vim
:unlet b:current_syntax
```

- 如果被包含的语法文件里的顶层语法项目应在包含它的语法的某区域中应用，可以使用 ":syntax include" 命令：

```
:sy[ntax] include [@{grouplist-name}] {file-name}
```

被包含文件里定义所有的语法项目会自动加上 "contained" 标志。同时，如果指定了组群（簇），被包含文件所有的顶层语法项目会加到该组群里。

```
" 在 perl.vim 里:
:syntax include @Pod <sfile>:p:h/pod.vim
:syntax region perlPOD start="^=head" end="^=cut" contains=@Pod
```

如果 {file-name} 是绝对路径（以 "/"、"c:"、"$VAR" 或者 "<sfile>" 开始），则载入该文件。如果它是是相对路径（例如，"syntax/pod.vim"），则先在 'runtimepath' 里搜索该文件，然后载入所有匹配的文件。建议使用相对路径，因为它允许用户用自己的版本替代被包含的文件，而不用修改使用 ":syn include" 命令的文件。

E847

包含的最大数目是 999。

---

11. 同步 :syn-sync E403 E404

Vim 期待在文档的任何位置都能开始重画过程。为此目的，它需要知道重画开始所在的位置相应的语法状态。

```
:sy[ntax] sync [ccomment [group-name] | minlines={N} | ...]
```

有四个同步方法：

1. 总是从文件头开始分析。
   :syn-sync-first
2. 基于 C 风格的注释。Vim 理解 C 注释的工作方式，因而可以发现当前行是在注释里面还是外面。
   :syn-sync-second
3. 回跳若干行，从那里开始分析。
   :syn-sync-third
4. 反向搜索某模式的匹配文本，从那里开始同步。
   :syn-sync-fourth

:syn-sync-maxlines :syn-sync-minlines

对于后三个方法而言，开始分析所在的行范围受到 "minlines" 和 "maxlines" 的限制。

如果给出 "minlines={N}" 参数，分析总是至少倒退给出的行数，然后才开始。这适用于分析过程在能判断正确之前至少需要若干行的情况、或者完全无法使用同步机制的场合。

如果给出 "maxlines={N}" 参数，反向搜索注释或者同步用搜索模式的行数限于 N 行之内（包括 "minlines" 指定的行数）。可用于同步的项目较少且机器较慢的场合。例如：

```
:syntax sync maxlines=500 ccomment
```

:syn-sync-linebreaks

使用可以匹配多行的模式时，某行的改变可以使得匹配不再能从上一行开始匹配。这意味着同步必须从改变发生所在位置之前就进行。具体需要多少行可由 "linebreaks" 参数指定。例如，如果模式包含一个换行符，可以这样：

```
:syntax sync linebreaks=1
```

结果是，重画至少从改变所在之前的一行开始。缺省的 "linebreaks" 值为零。通常，"minlines" 的值比 "linebreaks" 要大。

第一种同步方法： :syn-sync-first

```
:syntax sync fromstart
```

文件从头开始分析。这使得语法高亮完全准确，但对很长的文件需时也久。Vim 预存以前分析过的文本，所以只有第一次分析文本的时候才会缓慢。不过，如果修改了文本，其后的部分需要重新分析（最差情况下：一直到文件尾）。

使用 "fromstart" 等价于把 "minlines" 指定为很大的数字。

第二种同步方法： :syn-sync-second :syn-sync-ccomment

第二种方法只需指定 "ccomment" 参数。示例：

```
:syntax sync ccomment
```

Vim 如果发现显示开始的行出现在 C 风格注释的内部，就使用组名为 "Comment" 的最后一个区域语法项目。这需要组名为 "Comment" 的区域项目存在！也可指定替代的组名，例如：

```
:syntax sync ccomment javaComment
```

这意味着最后出现的 "syn region javaComment" 会用来检测 C 注释的区域。只有该区域确实包含了开始模式 "\/*" 和结束模式 "*\/" 的时候才能有效。

"maxlines" 参数可以用来限制搜索的行数。"minlines" 参数用来指定至少回退若干行才开始（例如一些只占用若干行，但很难同步的构造）。

注意：如果使用跨行且包含 "*/" 的字符串，C 注释的同步方法不一定能正确工作。让字符串跨行是一个不好的编程习惯（许多编译器会给出警告），而 "*/" 出现在注释（译者注：应为字符串）的机率又相当小，一般很难注意到这个限制。

第三种同步方法： :syn-sync-third

第三种同步方法只要指定 "minlines={N}" 参数。Vim 会从行号上减去 {N} 然后从那里开始分析。这意味着需要额外分析 {N} 行，该方法也因此较慢。例如：

```
:syntax sync minlines=50
```

"lines" 等价于 "minlines"（用于较老的版本）。

第四种同步方法：　　:syn-sync-fourth

本方法试图在若干指定区域的一端同步，称为同步模式。因为只有区域可以跨行。所以如果我们能找到某区域的一端，或许就可以知道现在在哪个语法项目之中。该方法中，搜索从重画开始所在的上一行开始，然后在文件中反向进行。

和非同步的语法项目类似，同步项目也可以使用 contained、matches、nextgroup 等。但有如下区别：
- 不能使用关键字项目。
- 使用 "sync" 关键字的语法项目构成完全独立的语法项目组。你不能混合同步组和非同步组。
- 匹配在缓冲区里（逐行）反向进行，而不是正向。
- 可以给出续行模式，用于决定哪些行的组合可以在搜索时当成单行。这意味着搜索指定项目的匹配从包含续行模式的连续多行的第一行开始。
- "nextgroup" 或 "contains" 只适用于一行（或者连续行构成的组合）之内。
- 使用区域项目时，必须在同一行上（或者连续行构成的组合）开始和结束。否则，假定行尾（或者连续行构成的组合的尾部）会结束此项目。
- 如果找到同步模式的匹配，该行（或者连续行构成的组合）的其余部分会搜索其它匹配。最后发现的匹配被使用。这可以用于同时包括区域开始和结束的行（例如，C 注释 /* this */，使用最后找到的 "*/"）。

有两个使用同步模式的匹配方法：
1. 高亮分析从重画开始处开始（也就是同步模式的搜索起点）。必须指定在那里出现的合法语法组。如果跨行的区域不会包含别的区域时，这很有效。
2. 高亮分析从匹配之后立即开始。必须指定匹配之后立即出现的合法语法组。可用于上述方法不适用的情形。它慢得多，因为需要分析更多的文本。

可以同时使用两种类型的同步模式。

除了同步模式以外，还可以指定另外的匹配和区域项目，以跳过不需要的匹配。

[之所以单独给出同步模式，是因为多数情况下，同步点的搜索比高亮部分的确定要简单得多。而模式的减少意味着速度的（大大）加快。]

　　syn-sync-grouphere　E393　E394
```
:syntax sync match {sync-group-name} grouphere {group-name} "pattern" ..
```

"这里的组"，定义用于同步的匹配。{group-name} 为匹配之后立刻出现的语法组名（译者注：但不一定从那里开始，尤其是同步模式本身就可以属于该组）。文本的高亮分析在匹配之后立刻进行。{group-name} 至少应包含一个区域项目，并使用其中的第一个这样的定义。"NONE" 用来指示匹配之后不存在语法组。

　　syn-sync-groupthere
```
:syntax sync match {sync-group-name} groupthere {group-name} "pattern" ..
```

"那里的组"，类似于 "grouphere"，但 {group-name} 是在同步点的搜索起点所在行的行首使用的语法组名。匹配和同步点的搜索起点之间的文本假定不会改变语法的高亮。例如，在 C 里，你可以反向搜索 "/*" 和 "*/"。如果先找到 "/*"，你知道在注释内部，所以 "那里的组" 是 "cComment"。如果先找到 "*/"，你知道不在注释里，所以 "那里的组" 是 "NONE"。(实际应用中更复杂，因为 "/*" 和 "*/" 可以出现在字符串中。留给读者作为练习吧……)。

```
:syntax sync match ..
```

:syntax sync region ..

没有 "groupthere" 参数。定义区域或匹配项目，在同步点搜索过程中跳过这些项目（译者注：注意 同步模式只能使用匹配项目，但在这里定义的同步过程的 "跳过" 模式可以指定区域或匹配项目。区域项目的限制上面已有叙述）。

syn-sync-linecont

:syntax sync linecont {pattern}

如果 {pattern} 在行内匹配，本行被认为会在下一行继续。这意味着同步点的搜索会把这些行当作连接在一起的一行处理。

如果同时给出 "maxlines={N}" 参数，寻找匹配的搜索行数限于 N 行之内。可以用于同步项目很少且机器速度较慢的场合。例如：

```
:syntax sync maxlines=100
```

你可以这样清除所有的同步设置：

```
:syntax sync clear
```

你也可以清除特定的同步模式：

```
:syntax sync clear {sync-group-name} ..
```

---

12. 列出语法项目 :syntax :sy :syn :syn-list

本命令列出所有的语法项目：

```
:sy[ntax] [list]
```

要显示单个语法组的所有语法项目：

```
:sy[ntax] list {group-name}
```

要列出单个簇的所有语法组： E392

```
:sy[ntax] list @{cluster-name}
```

":syntax" 命令的其它参数见上。

注意 ":syntax" 命令可以简化成 ":sy"，不过 ":syn" 更常用，因为看起来更舒服点。

---

13. Highlight 命令 :highlight :hi E28 E411 E415

有三种类型的高亮组：
- 用于特定语言的。这些组的名字以该语言的名字开始。它们中很多没有属性，而是链接到第二种类型的组。
- 用于所有语法语言的。
- 用于 'highlight' 选项的。

hitest.vim

用这个命令，你可以看到当前激活的所有组：

```
:so $VIMRUNTIME/syntax/hitest.vim
```

它会打开一个新窗口，其中包含所有的高亮组名，以它们本身的颜色显示。

:colo :colorscheme E185

:colo[rscheme] 输出当前激活的色彩方案名。基本上等同

```
:echo g:colors_name
```

如果 g:colors_name 没有定义 :colo 会输出 "default"。如果编译时没有带 +eval 特性，输出 "unknown"。

:colo[rscheme] {name} 载入色彩方案 {name}。它会在 'runtimepath' 里搜索 "colors/{name}.vim"，载入第一个找到的文件。
也在 'packpath' 中的所有插件里寻找，先在 "start" 下，然后在 "opt" 下查找。

它不能递归调用，所以你不能在色彩方案脚本里使用 ":colorscheme"。

要定制色彩方案，用新名，如 "~/.vim/colors/mine.vim"，并用 :runtime 载入原来的色彩方案：

```
runtime colors/evening.vim
hi Statement ctermfg=Blue guifg=Blue
```

色彩方案载入前，激活 ColorSchemePre 自动命令事件。色彩方案载入后，激活 ColorScheme 自动命令事件。关于如何编写色彩方案文件的信息：

```
:edit $VIMRUNTIME/colors/README.txt
```

:hi[ghlight] 列出当前所有的有属性设置的高亮组。

:hi[ghlight] {group-name}
列出一个高亮组。

:hi[ghlight] clear 复位高亮设置为缺省值。删除所有用户增加的组的高亮属性。用当前的 'background' 的值来决定所使用的缺省颜色。

:hi[ghlight] clear {group-name}
:hi[ghlight] {group-name} NONE
屏蔽一个高亮组的所有高亮设置。并 不 复原缺省的颜色。

:hi[ghlight] [default] {group-name} {key}={arg} ..
增加高亮组，或者更改已有的组高亮设置。
highlight-args 说明 {key}={arg} 的参数。
:highlight-default 说明可选的 [default] 参数。

通常，在启动时加入高亮组。它设置高亮的缺省值。在这之后，你可以使用附加的 highlight 命令来修改你希望设置为非缺省值的参数。也可以用 "NONE" 来撤销某个值并恢复缺省的值。

修改颜色的简单方式是 :colorscheme 命令。它载入一个文件，里面包含了这样的 ":highlight" 命令：

```
:hi Comment  gui=bold
```

注意 所有没有包含在内的设置都保持原样，只使用指定的字段，从而和以前的设置进行了合并。所以实际结果就像用了下面这样的一条命令：

```
:hi Comment  term=bold ctermfg=Cyan guifg=#80a0ff gui=bold
```

:highlight-verbose

如果列出高亮组时 'verbose' 非零，同时列出高亮组最近在哪里设置。例如：

```
:verbose hi Comment
Comment        xxx term=bold ctermfg=4 guifg=Blue
   Last set from /home/mool/vim/vim7/runtime/syntax/syncolor.vim
```

如果使用了 ":hi clear"，那么列出缺省值的同时提到使用该命令的脚本。详见 :verbose-cmd 。

highlight-args E416 E417 E423

用于高亮，有三种类型的终端：

term 普通的终端 (vt100、xterm)
cterm 色彩终端 (MS-DOS 控制台、color-xterm，带有 "Co" termcap 项目的终端)
gui GUI

每种类型可以分别设置高亮属性。这样，单个语法文件就可用于所有的终端，并使用每个终端最优的高亮设置。

1. 普通终端的高亮参数

bold underline undercurl
inverse italic standout
nocombine strikethrough
term={attr-list} attr-list highlight-term E418

attr-list 是逗号分隔的下述项目 (不能有空格) 的列表 (任何顺序都可以)：

bold	
underline	
undercurl	不一定总是可用
strikethrough	不一定总是可用
reverse	
inverse	等同于 reverse
italic	
standout	
nocombine	覆盖属性而不是混合属性
NONE	不使用属性 (用于复位)

注意 这里使用 "bold" 和使用粗体字体都可以，效果相同。

"undercurl" 是波浪状的下划线。如果不能用 "undercurl"，使用 "underline" 代替。一般，只有 GUI 才能使用 "undercurl" 和 "strikethrough"。颜色可用 highlight-guisp 设置。

start={term-list} highlight-start E422
stop={term-list} term-list highlight-stop

可以得到终端上的非标准的属性的终端代码的列表。

"start" 参数指定的转义码序列在高亮区域的字符之前被写入，它可以是你希望给终端发送的用于高亮区域的任何内容。"stop" 参数指定的转义码序列在高亮区域之后被写入，它可以撤销 "start" 参数的效果。否则屏幕会被弄乱。

{term-list} 可有两种形式：

1. 转义序列的字符串。
   可以是任何字符的序列，除了不能以 "t_" 开始而且不能有空格之外。这里

识别 <> 记法。所以你可使用 "<Esc>" 和 "<Space>"。例如:

```
start=<Esc>[27h;<Esc>[<Space>r;
```

2. 终端代码的列表。
终端代码的形式是 "t_xx" ，其中 "xx" 是 termcap 项目的名字。这些代码必须以逗号分隔，而且不允许有空格。例如:

```
start=t_C1,t_BL
```

要使之工作，这些终端代码必须存在。

2. 色彩终端的高亮参数

cterm={attr-list} highlight-cterm

{attr-list} attr-list 的描述见上。"cterm" 参数可以和 "term" 不同，比如可以使用颜色。例如，在普通的终端上注释可以用下划线表示，在色彩终端上可以用蓝色显示。

注意：有些终端（例如，DOS 控制台）不能混合使用颜色和这些属性。为可移植性，只用 "cterm=" 或者 "ctermfg=" 与 "ctermbg=" 两者之一。

ctermfg={color-nr} highlight-ctermfg E421
ctermbg={color-nr} highlight-ctermbg

{color-nr} 参数指定颜色号。其范围从零到 termcap 项目 "Co" 给出的数字(不含)。实际的颜色取决于终端的类型和设置。有时，颜色也取决于 "cterm" 的值。例如，有的系统上 "cterm=bold ctermfg=3" 给出另外一个颜色，别的系统上你只会得到颜色 3。

xterm 上，它取决于你的资源，这并不容易预测。你的 xterm 文档会说明缺省值。color-xterm 的颜色可以通过 .Xdefaults 文件修改。不幸的是，这意味着每个用户未必会得到完全相同的颜色。 xterm-color 提供带色彩的 xterm 的信息。

MSDOS 的标准颜色是固定的（在控制窗口上），所以我们用这些名字。在 X11 上，颜色名字的含义是固定的，所以我们用这些颜色设置，以使高亮设置可移植(是不是很复杂？)。下面列出这些被识别的名字和所使用的颜色号:

cterm-colors

NR-16	NR-8	颜色名
0	0	Black (黑)
1	4	DarkBlue (深蓝)
2	2	DarkGreen (深绿)
3	6	DarkCyan (深青)
4	1	DarkRed (深红)
5	5	DarkMagenta (深品红)
6	3	Brown, DarkYellow (褐、深黄)
7	7	LightGray、LightGrey, Gray, Grey (浅灰)
8	0*	DarkGray, DarkGrey (深灰)
9	4*	Blue, LightBlue (浅蓝)
10	2*	Green, LightGreen (浅绿)
11	6*	Cyan, LightCyan (浅青)
12	1*	Red, LightRed (浅品红)
13	5*	Magenta, LightMagenta (浅红)
14	3*	Yellow, LightYellow (浅黄)
15	7*	White (白)

"NR-16" 下的数字用于 16 色的终端 ('t_Co' 大于等于 16)。"NR-8" 下的数字用于 8 色终端 ('t_Co' 小于 16)。'*' 表明 ctermfg 使用的时候设置 bold 属性。在许多 8 色终端上 (比如, "linux"), 它会显示增亮的颜色。但这不适用于背景色。如果没有 '*', 则不包含 bold 属性。如果你想用不同的方式设置 bold 属性, 在 "ctermfg=" 或 "ctermbg=" 参数 之后 设置 "cterm=" 参数。或者使用数字, 而不是颜色名。

忽略颜色名的大小写。

注意 16 色 ansi 风格的终端 (包括 xterm) 使用 NR-8 一列的数字。这里, '*' 意味着 '加 8', 所以 Blue 成了 12, DarkGray 是 8, 依此类推。

注意 有些色彩终端下, 这些名字可能会产生错误的颜色!

也可用 "NONE" 来删除颜色。

:hi-normal-cterm

Normal 组设置的 "ctermfg" 或 "ctermbg" 颜色成为非高亮文本使用的颜色。例如:

```
:highlight Normal ctermfg=grey ctermbg=darkblue
```

设置 Normal 组的 "ctermbg" 颜色时, 假定此颜色能识别而 'background' 还未显式设置过, 会自动设置 'background' 选项。这使得依赖于 'background' 的高亮组会发生改变! 这意味着, 你应该先设置 Normal 的颜色, 然后再设置其它的。
已经使用色彩方案的时候, 修改 'background' 使之被重新载入, 这会复位所有颜色 (包括 Normal)。如果你不希望如此, 先删除 "g:colors_name" 变量。

如果你给 Normal 组设置了 "ctermfg" 或 "ctermbg", Vim 退出时需要复位颜色, 这通过 "op" termcap 项目 t_op 完成。如果不能正确工作, 尝试在你的 .vimrc 里设置 't_op' 选项。

E419 E420

如果 Vim 知道 Normal 的前景和背景色, "fg" 和 "bg" 可以用做颜色名字。这只有在设置了 Normal 组的颜色以后和 (译者注: 或?) 在 MS-DOS 控制台上才可以。比如, 设置反显视频:

```
:highlight Visual ctermfg=bg ctermbg=fg
```

注意 使用的颜色必须在给出命令的时候已经合法。如果之后 Normal 组的颜色发生改变, "fg" 和 "bg" 颜色不会被调整。

3. GUI 的高亮参数

gui={attr-list} highlight-gui

给出 GUI 模式适用的属性。具体描述见 attr-list 。

注意 这里可以使用 "bold", 也可以使用粗体字体。效果相同。

注意 "Normal" 组的属性被忽略。

font={font-name} highlight-font

font-name 是字体的名字, 根据 Vim 运行所在的系统而定。X11 而言, 它是一个复杂的名字。比如:

```
font=-misc-fixed-bold-r-normal--14-130-75-75-c-70-iso8859-1
```

字体名 "NONE" 可以用来恢复到缺省字体。如果为 "Normal" 组设置字体, 该字体成为缺省字体 (直到改变了 'guifont' 选项为止; 使用最后设置的那个)。
以下说明只适用于 Motif 和 Athena, 不包括其它 GUI:

设置 "Menu" 组的字体会改变菜单。设置 "Tooltip" 组的字体会改变工具提示。除了 Menu 和 Tooltip 的所有使用的字体必须和缺省字体的字符大小相同。否则，重画时会有问题。
要用含有内嵌空格或其它特殊字符的字体名，把名字放在单引号内。因而，不能使用单引号本身。
例如:

```
:hi comment font='Monospace 10'
```

guifg={color-name} highlight-guifg
guibg={color-name} highlight-guibg
guisp={color-name} highlight-guisp
给出 GUI 使用的前景色 (guifg)、背景色 (guibg) 和特殊颜色 (guisp)。"guisp" 用于 undercurl 和 strikethrough。可用一些特殊的名字:

NONE	没有颜色 (透明)
bg	使用 Normal 的背景色
background	使用 Normal 的背景色
fg	使用 Normal 的前景色
foreground	使用 Normal 的前景色

要使用有内嵌空格或者其它特殊字符的颜色名，把它放在单引号里面。这时，不能使用单引号。例如:

```
:hi comment guifg='salmon pink'
```

gui-colors
建议的颜色名 (可用于多数系统):

Red	LightRed	DarkRed	
Green	LightGreen	DarkGreen	SeaGreen
Blue	LightBlue	DarkBlue	SlateBlue
Cyan	LightCyan	DarkCyan	
Magenta	LightMagenta	DarkMagenta	
Yellow	LightYellow	Brown	DarkYellow
Gray	LightGray	DarkGray	
Black	White		
Orange	Purple	Violet	

Win32 GUI 版本可以使用更多的系统颜色。见 win32-colors 。

你可以用红、绿、蓝的色值指定颜色。格式为 "#rrggbb"，其中

"rr"	为红色值
"bb"	为蓝色值
"gg"	为绿色值

所有的值都以十六进制表示，范围是 "00" 到 "ff"。例如:

```
:highlight Comment guifg=#11f0c3 guibg=#ff00ff
```

highlight-groups highlight-default
有一些缺省的高亮组。'highlight' 选项缺省使用这些组。注意 高亮取决于 'background' 的值。你可以用 ":highlight" 命令看到当前的设置。
hl-ColorColumn
ColorColumn 用于 'colorcolumn' 设置的列
hl-Conceal
Conceal 代替隐藏文本的填充字符 (见 'conceallevel')
hl-Cursor
Cursor 光标所在的字符
lCursor 使用 language-mapping 时光标所在的字符 (见 'guicursor')
hl-CursorIM

CursorIM 类似于 Cursor，但用于 IME 模式 CursorIM

hl-CursorColumn

CursorColumn 置位 'cursorcolumn' 时，光标所在的屏幕列

hl-CursorLine

CursorLine 置位 'cursorline' 时，光标所在的屏幕行

hl-Directory

Directory 目录名（还有列表里的其它特殊名字）

hl-DiffAdd

DiffAdd 比较模式：增加的行 diff.txt

hl-DiffChange

DiffChange 比较模式：改变的行 diff.txt

hl-DiffDelete

DiffDelete 比较模式：删除的行 diff.txt

hl-DiffText

DiffText 比较模式：改变行里的改动文本 diff.txt

hl-EndOfBuffer

EndOfBuffer 缓冲区的末行之后的填充行（~）。缺省用 hl-NonText 类似的高亮。

hl-ErrorMsg

ErrorMsg 命令行上的错误信息

hl-VertSplit

VertSplit 分离垂直分割窗口的列

hl-Folded

Folded 用于关闭的折叠的行

hl-FoldColumn

FoldColumn 'foldcolumn'

hl-SignColumn

SignColumn 显示 signs 的列

hl-IncSearch

IncSearch 'incsearch' 高亮；也用于被 ":s///c" 替换的文本

hl-LineNr

LineNr ":number" 和 ":#" 命令与置位 'number' 或 'relativenumber' 选项时的行号。

hl-LineNrAbove

LineNrAbove 置位 'relativenumber' 选项时，光标上方的行号。

hl-LineNrBelow

LineNrBelow 置位 'relativenumber' 选项时，光标下方的行号。

hl-CursorLineNr

CursorLineNr 和 LineNr 类似，置位 'cursorline' 且 'cursorlineopt' 设为 "number" 或 "both"，或置位 'relativenumber' 时用于光标行。

hl-MatchParen

MatchParen 如果光标所在或刚刚在它之前的字符是配对的括号一部分的话，它和它的配对。 pi_paren.txt

hl-ModeMsg

ModeMsg 'showmode' 消息（例如，"-- INSERT --"）

hl-MoreMsg

MoreMsg more-prompt

hl-NonText

NonText 窗口尾部的 '@'，'showbreak' 的字符和其它在文本里实际不存在的字符（例如，代替行尾放不下的双宽字符而显示的 ">"）。

hl-Normal

Normal 普通文本

hl-Pmenu

Pmenu 弹出菜单：普通项目。

hl-PmenuSel

PmenuSel 弹出菜单：选中项目。

hl-PmenuSbar

PmenuSbar 弹出菜单：滚动条。

hl-PmenuThumb

PmenuThumb 弹出菜单：滚动条的拇指（thumb）。

hl-Question

Question hit-enter 提示和 yes/no 问题

hl-QuickFixLine

QuickFixLine quickfix 窗口的当前 quickfix 项目。

hl-Search

Search 最近搜索模式的高亮（见 'hlsearch'）。
也用于类似的要突出显示的项目。

hl-SpecialKey

SpecialKey ":map" 列出的 Meta 和特殊键，也包括文本里不可显示字符的显示和 'listchars'。
一般的：和实际内容显示上有差异的文本。

hl-SpellBad

SpellBad 拼写检查器不能识别的单词。 spell
它会和本来应该使用的高亮进行组合。

hl-SpellCap

SpellCap 应该大写字母开头的单词。 spell
它会和本来应该使用的高亮进行组合。

hl-SpellLocal

SpellLocal 拼写检查器能识别但只在其它区域使用的单词。 spell
它会和本来应该使用的高亮进行组合。

hl-SpellRare

SpellRare 拼写检查器能识别但很少使用的单词。 spell
它会和本来应该使用的高亮进行组合。

hl-StatusLine

StatusLine 当前窗口的状态行

hl-StatusLineNC

StatusLineNC 非当前窗口的状态行
注意：如果等于 "StatusLine"，Vim 会使用 "^^^" 指示当前窗口的状态行。

hl-StatusLineTerm

StatusLineTerm terminal 窗口为当前窗口时的状态行。

hl-StatusLineTermNC

StatusLineTermNC terminal 窗口不为当前窗口时的状态行。

hl-TabLine

TabLine 标签页行，非活动标签页标签

hl-TabLineFill

TabLineFill 标签页行，没有标签的地方

hl-TabLineSel

TabLineSel 标签页行，活动标签页标签

hl-Terminal

Terminal terminal 窗口（见 terminal-size-color ）

hl-Title

Title ":set all"、":autocmd" 等输出的标题

hl-Visual

Visual 可视模式的选择区

hl-VisualNOS

VisualNOS Vim 是 "选择区的非拥有者" 时，可视模式的选择区。只有 X11 GUI 的 gui-x11 和 xterm-clipboard 才提供此支持。

hl-WarningMsg

WarningMsg 警告消息

hl-WildMenu

WildMenu 'wildmenu' 补全的当前匹配

hl-User1 hl-User1..9 hl-User9

'statusline' 语法允许在状态行和标尺 (通过 'rulerformat') 上使用 9 种不同的高亮。这些高亮组的名字是 User1 到 User9。

GUI 里，你可以使用以下这些组来设置菜单、滚动条和工具提示的色彩。它们没有缺省值。这不适用于 Win32 GUI。这里，只有三种高亮参数有效：font、guibg 和 guifg。

hl-Menu

Menu 当前菜单的字体、背景色和前景色。也包括工具栏。
可用的高亮参数：font、guibg、guifg。

注意：Motif 和 Athena 里，font 参数在所有情况下实际上指定字体集 (fontset)，不管 'guifontset' 是否为空。所以，它和当前的 :language 有关系。

hl-Scrollbar

Scrollbar 主窗口滚动条的当前背景和前景色。
可用的高亮参数：guibg、guifg。

hl-Tooltip

Tooltip 当前工具提示的字体、背景色和前景色。也包括工具栏。
可用的高亮参数：font、guibg、guifg。

注意：Motif 和 Athena 里，font 参数在所有情况下都指定字体集 (fontset)，不管 'guifontset' 是否为空，从而在设置的时候和当前 :language 相联系。

---

14. 链接组 :hi-link :highlight-link E412 E413

如果如果你想为几个语法组使用相同的高亮设置，更简单的方法是把这些组链接到一个共同的高亮组，并且只为该组指定颜色属性。

要设置链接：

```
:hi[ghlight][!] [default] link {from-group} {to-group}
```

要删除链接：

```
:hi[ghlight][!] [default] link {from-group} NONE
```

注意： E414

- 如果 {from-group} 和/或 {to-group} 不存在，先建立之。不存在的组不会给你错误信息。
- 一旦你为被链接的组使用 ":highlight" 命令，该链接关系就不存在了。
- 如果已经有 {from-group} 的高亮设置，链接不会建立，除非你给出了 '!'。如果在载入的文件里执行 ":highlight link" 命令，你不会得到错误信息。这可以用来跳过已有设置的组的链接。

:hi-default :highlight-default
用 [default] 参数可以设置组的缺省高亮。如果该组已经设置了高亮，或者已经存在链接，忽略该命令。

这里， [default] 对否决特定语法文件的高亮特别有用。例如，C 语法文件包含：

```
:highlight default link cComment Comment
```

如果你喜欢 C 注释像 Question 一样高亮，在你的 vimrc 文件里放入此行：

```
:highlight link cComment Question
```

如果没有 "default"，执行 C 语法文件后会覆盖你的高亮设置。

---

15. 清理 :syn-clear E391

如果你想为当前缓冲区清除所有的语法设置，可以使用如下命令：

```
:syntax clear
```

该命令应该在你想关闭语法高亮或者切换到别的语法时使用。通常，语法文件本身不需要它。因为载入语法文件的自动命令会清理语法。
该命令也删除 "b:current_syntax" 变量，因为该命令之后没有载入的语法了。

要为当前缓冲区清理特定的语法组：

```
:syntax clear {group-name} ..
```

它删除了 {group-name} 里所有的模式和关键字。

要为当前缓冲区清理特定的语法组群（簇）：

```
:syntax clear @{grouplist-name} ..
```

它把 {grouplist-name} 的内容设为空表。

:syntax-off :syn-off
如果想为所有的缓冲区关闭语法高亮，你需要删除载入语法文件的自动命令：

```
:syntax off
```

该命令实际执行的是命令

```
:source $VIMRUNTIME/syntax/nosyntax.vim
```

详情见 "nosyntax.vim" 文件。 注意 要使它工作，$VIMRUNTIME 必须合法。见 $VIMRUNTIME 。

:syntax-reset :syn-reset
如果你修改了颜色并且把颜色弄乱了，使用这个命令取回缺省值：

```
:syntax reset
```

这个名字起的不太好，因为它并不复位任何语法项目，而只对高亮有影响。

它不会改变 'highlight' 选项的颜色值。

注意 你在 vimrc 文件里设定的语法颜色也会复位成其 Vim 的缺省值。
注意 如果你使用了色彩方案，色彩方案定义的语法高亮的色彩会丢失。

实际上，它做的是：

```
let g:syntax_cmd = "reset"
```

```
runtime! syntax/syncolor.vim
```

注意 这里使用了 'runtimepath' 选项。

syncolor
如果想为语法高亮使用不同的色彩，你可以增加一个 Vim 脚本来设置这些颜色。把该文件放在 'runtimepath' 里 $VIMRUNTIME 之后的一个目录里，以使你的设置覆盖缺省颜色值。这样，这些颜色会在 ":syntax reset" 命令之后被使用。

Unix 上你可以使用文件 ~/.vim/after/syntax/syncolor.vim。例如:

```
if &background == "light"
  highlight comment ctermfg=darkgreen guifg=darkgreen
else
  highlight comment ctermfg=green guifg=green
endif
```

E679
要确信这个 syncolor.vim 脚本不使用 "syntax on" 命令，设置 'background' 选项或者使用 "colorscheme" 命令，不然会导致死循环。

注意 如果使用了色彩方案，可能会比较混淆：到底使用的是你自己定义的颜色还是方案里的颜色。这取决于色彩方案文件，见 :colorscheme 。

syntax_cmd
载入 syntax/syncolor.vim 文件时，"syntax_cmd" 变量设为以下这些值之一:

"on"	":syntax on" 命令。高亮颜色被覆盖，但是链接被保持
"enable"	":syntax enable" 命令。只为没有设置过高亮的组定义颜色。使用 ":syntax default"。
"reset"	":syntax reset" 命令或者载入色彩方案。定义所有的颜色。
"skip"	不定义颜色。用来跳过 'runtimepath' 里较早出现的 syncolor.vim 已经设置过的缺省设置。

---

16. 高亮标签

tag-highlight

如果你想高亮文件里的所有标签，可以使用如下映射。

```
<F11>   -- 生成 tags.vim 文件，并高亮标签。
<F12>   -- 只根据已有的 tags.vim 文件高亮标签。

:map <F11>  :sp tags<CR>:%s/^\([^     :]*:\)\=\([^    ]*\).*/syntax keyword Tag \2/<CR>:wq! t
:map <F12>  :so tags.vim<CR>
```

警 告: 标签文件越长，这会越慢，而 Vim 消耗的内存也越多。

这里只高亮 typedef，也可以针对 union 和 struct 进行设置。为此，你需要 Exuberant ctags (可在 http://ctags.sf.net 找到)。

在你的 Makefile 里放入以下的行:

```
# 建立 types 的高亮文件。需要 Exuberant ctags 和 awk
types: types.vim
types.vim: *.[ch]
```

```
ctags --c-kinds=gstu -o- *.[ch] |\
        awk 'BEGIN{printf("syntax keyword Type\t")}\
                {printf("%s ", $$1)}END{print ""}' > $@
```

在你的 .vimrc 里放入以下的行：

```
" 载入 types.vim 高亮文件，如果存在的话
autocmd BufRead,BufNewFile *.[ch] let fname = expand('<afile>:p:h') . '/types.vim'
autocmd BufRead,BufNewFile *.[ch] if filereadable(fname)
autocmd BufRead,BufNewFile *.[ch]   exe 'so ' . fname
autocmd BufRead,BufNewFile *.[ch] endif
```

---

17. 窗口局部语法 :ownsyntax

通常，同一个缓冲区对应的所有窗口共用相同的语法设置。不过，可以为某个文件的某个窗口设置单独的语法设置。一个可能的例子，在一个窗口中使用普通的高亮方式编辑 LaTeX 源代码，另一个窗口用不同的高亮方式访问相同的代码（例如隐藏控制序列，实际对文本加粗，加斜体等等）。'scrollbind' 选项可用于此。

要使当前窗口使用语法 "foo"，而不影响缓冲区的其它窗口：

```
:ownsyntax foo
```

w:current_syntax

这样做，"w:current_syntax" 变量会设为 "foo"。"b:current_syntax" 的值不变。实际上，这是通过保存和恢复 "b:current_syntax" 来实现的，因为语法文件会设置 "b:current_syntax"。但语法文件设置的值会被转赋到 "w:current_syntax" 上。

备注：这会复位 'spell'、'spellcapcheck' 和 'spellfile' 选项。

一旦窗口有自己的语法，同一缓冲区其它窗口执行的语法命令（包括 :syntax clear）就不会有效果。相反地，此窗口执行的语法命令对同一缓冲区其它窗口也没有效果。

带有自己语法的窗口当载入其它缓冲区或重载文件时，会恢复到正常行为。
分割窗口时，新窗口使用原有的语法。

---

18. 彩色 xterm xterm-color color-xterm

多数彩色 xterm 只有八色。如果你得不到缺省设置的颜色，在 .vimrc 里使用这些行应该就可以了：

```
:if &term =~ "xterm"
:  if has("terminfo")
:    set t_Co=8
:    set t_Sf=<Esc>[3%p1%dm
:    set t_Sb=<Esc>[4%p1%dm
:  else
:    set t_Co=8
:    set t_Sf=<Esc>[3%dm
:    set t_Sb=<Esc>[4%dm
:  endif
:endif
```

[<Esc> 是真正的 escape，输入 CTRL-V <Esc>]

你可能需要修改第一个 "if" 来匹配你的终端的名字。比如，"dtterm" 取代 "xterm"。

注意：在 ":syntax on" 执行之前这些设置。否则颜色可能不正确。

xiterm rxvt

上面提到的设置也适用于 xiterm 和 rxvt。但要在 rxvt 里使用 16 色，可以使用 terminfo 并执行如下命令：

```
:set t_AB=<Esc>[%?%p1%{8}%<%t25;%p1%{40}%+%e5;%p1%{32}%+%;%dm
:set t_AF=<Esc>[%?%p1%{8}%<%t22;%p1%{30}%+%e1;%p1%{22}%+%;%dm
```

colortest.vim

要测试你的色彩设置，Vim 发布版本里包含了一个文件。要使用它，执行如下命令：

```
:runtime syntax/colortest.vim
```

即使颜色数定义为 8，一些版本的 xterm（还有其他终端，比如 linux 控制台）可以输出更亮的前景色。因此如果 't_Co' 为 8，Vim 为浅色的前景色设置 "cterm=bold" 属性。

xfree-xterm

要得到 16 色或更多，需要最新的 xterm 版本（应该包含在 Xfree86 3.3 或更新的版本里）。你可以在这里得到最新的版本：

```
http://invisible-island.net/xterm/xterm.html
```

下面是 configure 的好方法。它使用 88 色，并打开 termcap-query 特性。这使得 Vim 能够询问 xterm 可以支持多少颜色。

```
./configure --disable-bold-color --enable-88-color --enable-tcap-query
```

如果你只有 8 色，检查 xterm 的编译设置。
(另见 UTF8-xterm ，了解如何在该 xterm 里使用 UTF-8 字符编码)。

在你的 .vimrc 放入以下几行，该 xterm 应该工作（16 色）：

```
:if has("terminfo")
:  set t_Co=16
:  set t_AB=<Esc>[%?%p1%{8}%<%t%p1%{40}%+%e%p1%{92}%+%;%dm
:  set t_AF=<Esc>[%?%p1%{8}%<%t%p1%{30}%+%e%p1%{82}%+%;%dm
:else
:  set t_Co=16
:  set t_Sf=<Esc>[3%dm
:  set t_Sb=<Esc>[4%dm
:endif
```

[<Esc> 是真正的 escape，输入 CTRL-V <Esc>]

没有 +terminfo 的话，Vim 会识别这些设置，并自动把 cterm 的第 8 色或更高的颜色翻译成 "<Esc>[9%dm" 和 "<Esc>[10%dm"。第 16 色以上也自动进行翻译。

有报告说这样可以用于 256 色：

```
:set t_AB=<Esc>[48;5;%dm
:set t_AF=<Esc>[38;5;%dm
```

或者只设置 TERM 环境变量为 "xterm-color" 或 "xterm-16color"，然后试试行不行。

你也许想使用如下 X 资源（在你的 ~/.Xdefaults 文件里）：

```
XTerm*color0:                   #000000
XTerm*color1:                   #c00000
XTerm*color2:                   #008000
XTerm*color3:                   #808000
```

```
XTerm*color4:                   #0000c0
XTerm*color5:                   #c000c0
XTerm*color6:                   #008080
XTerm*color7:                   #c0c0c0
XTerm*color8:                   #808080
XTerm*color9:                   #ff6060
XTerm*color10:                  #00ff00
XTerm*color11:                  #ffff00
XTerm*color12:                  #8080ff
XTerm*color13:                  #ff40ff
XTerm*color14:                  #00ffff
XTerm*color15:                  #ffffff
Xterm*cursorColor:              Black
```

[注意 : 需要 cursorColor 来绕过一个漏洞。该漏洞把光标颜色设为最后显示的文本的颜色。在较新的版本里该问题被修正，但还不是所有人都使用新版本。]

要立刻使用，在 X 选项数据库管理器里重新载入 .Xdefaults 文件 (你只须在改变 .Xdefaults 文件的时候这么做就可以了):

```
xrdb -merge ~/.Xdefaults
```

xterm-blink xterm-blinking-cursor

要使光标在 xterm 上闪烁，见 tools/blink.c。或者使用 Thomas Dickey 的 xterm 补丁号 107 以上的版本 (取得的方法见上)，使用这些资源:

```
XTerm*cursorBlink:      on
XTerm*cursorOnTime:     400
XTerm*cursorOffTime:    250
XTerm*cursorColor:      White
```

hpterm-color

下面的设置 (或多或少) 在 hpterm 里工作，它只支持 8 种前景色:

```
:if has("terminfo")
:  set t_Co=8
:  set t_Sf=<Esc>[&v%p1%dS
:  set t_Sb=<Esc>[&v7S
:else
:  set t_Co=8
:  set t_Sf=<Esc>[&v%dS
:  set t_Sb=<Esc>[&v7S
:endif
```

[<Esc> 是真正的 escape，输入 CTRL-V <Esc>]

Eterm enlightened-terminal

有报告称下面的设置可以使得 Enlightened terminal emulator，或 Eterm，工作。它们可能适用于所有的和 xterm 类似并使用 bold 属性来取得亮色的终端。有必要的话，增加类似于上面那样的 ":if"。

```
:set t_Co=16
:set t_AF=^[[%?%p1%{8}%<%t3%p1%d%e%p1%{22}%+%d;1%;m
:set t_AB=^[[%?%p1%{8}%<%t4%p1%d%e%p1%{32}%+%d;1%;m
```

TTpro-telnet

下面的设置应该适用于 TTpro telnet。Tera Term Pro 是 MS-Windows 上一个自由软件 / 开源程序。

```
set t_Co=16
```

```
        set t_AB=^[[%?%p1%{8}%<%t%p1%{40}%+%e%p1%{32}%+5;%;%dm
        set t_AF=^[[%?%p1%{8}%<%t%p1%{30}%+%e%p1%{22}%+1;%;%dm
```

要确信打开了 TTpro 的 Setup / Window / Full Color，并确信 没有 打开 Setup / Font / Enable Bold。
(由 John Love-Jensen <eljay@Adobe.COM> 提供信息)

---

19. 语 法 太 慢 了 :syntime

本节主要针对语法文件的作者。

如果语法导致重画变慢，有一些小窍门。打开一些通常会有干扰的特性，比如 'relativenumber' 和 folding ，会看到是否变慢。

注意：仅当编译时带 +profile 特性才可用。可能需要编译 Vim 时带 "huge" 特性包。

要知道哪些模式花了最多时间，下面的命令序列会给出概要:

```
        :syntime on
        [ 至少用 CTRL-L 重画文本一次 ]
        :syntime report
```

此时会显示使用到的语法模式的列表，按匹配文本使用的时间排序。

:syntime on	开始测量语法时间。会增加若干开销，用来计算语法模式匹配的时间。
:syntime off	停止测量语法时间。
:syntime clear	把所有的计数清零，重新开始测量。
:syntime report	在当前窗口显示 ":syntime on" 开始使用的语法项目。使用更宽的显示可以看到更多输出。 列表按总时间排序。显示以下诸列: TOTAL 用于匹配本模式的总时间，按秒计。 COUNT 本模式使用的次数。 MATCH 本模式实际匹配的次数。 SLOWEST 单次匹配最长的时间。 AVERAGE 单次匹配平均时间。 NAME 语法项目名。注意 这未必唯一。 PATTERN 使用的模式。

模式的匹配如果要尝试不同的分支时会变慢。尽量包括更多的直义文本，以减少模式 不匹配的可能性。

如果用 "\@<=" 和 "\@<!" 项目，加入最大范围来避免匹配本行和前行的所有可能位置。例如，如要项目是直义文本，指明文本的长度 (以字节计):

"<\@<=span"	匹配 "<span" 中的 "span"。会试图在很多位置匹配 "<"。
"<\@1<=span"	相同匹配，但仅尝试 "span" 之前的一个字节。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

textprop.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
https://github.com/yianwillis/vimcdoc

显示带附属属性的文本。 textprop text-properties

1. 介绍 text-prop-intro
2. 函数 text-prop-functions
3. 文本有改动时 text-prop-changes

{仅当编译时加入 textprop 特性才可以使用文本属性}

---

1. 介绍 text-prop-intro

缓冲区中的文本可附属文本属性。可随着文本移动：如果删除或插入了行，属性会随着附属的文本移动。在文本属性之前的行上插入/删除文本也是如此。如果插入/删除文本属性内的文本，文本属性会增加/减少长度。

文本属性的主要用途是文本的高亮。可以看作是语法高亮的替代。它不定义匹配文本的模式，而是用脚本来设置高亮，可能使用了外部解析器的输出。这只需要做一次，而不是每次重画屏幕的时候，所以会快许多，而付出的是附属文本属性的最初代价。

文本属性也可用于其它识别文本的目的。例如，给函数名加上文本属性，这样就可以定义跳转到下个/上个函数的搜索了。

文本属性附属在指定的行和列上，有指定的长度。属性可以跨多行。

文本属性有以下字段：

"id"	按你喜欢所使用的数值
"type"	属性类型名

属性类型

E971

文本属性通常有属性类型名，定义如何高亮文本。属性类型可有以下项目：

"highlight"	使用的高亮组名
"combine"	省略或 TRUE 时，文本属性高亮和任何语法高亮合并；FALSE 时文本属性高亮替代语法高亮
"priority"	属性有重叠昌，使用最高优先级的那个。
"start_incl"	TRUE 时在起始位置的插入会包含在文本属性内
"end_incl"	TRUE 时在结束位置的插入会包含在文本属性内

示例

假定缓冲区第 11 行有以下文本（除了缩进外）：

```
The number 123 is smaller than 4567.
```

要高亮此文本的数值：

```
call prop_type_add('number', {'highlight': 'Constant'})
call prop_add(11, 12, {'length': 3, 'type': 'number'})
call prop_add(11, 32, {'length': 4, 'type': 'number'})
```

试试在文本上方插入或删除行，你会看到文本属性和文本捆绑，所以会按需调整行号。

设置 "start_incl" 和 "end_incl" 可用于空白围绕文本，比如函数名的场合。为假时可用于文本以特定字符开始和/或结束，比如引号包围字符串的场合。

```
func FuncName(arg)
     ^^^^^^^^        置位 start_incl 和 end_incl 的属性

var = "text";
      ^^^^^^        复位 start_incl 和 end_incl 的属性
```

不过，插入或删除文本时，可能需要解析文本并更新文本属性。但这会异步进行。

内部错误 E967

如果你看到 E967，请报告此漏洞。可以通过 Github:
https://github.com/vim/vim/issues/new

---

2. 函数 text-prop-functions

操作文本属性类型：

```
prop_type_add({name}, {props})          定义新的属性类型
prop_type_change({name}, {props})       改变已有的属性类型
prop_type_delete({name} [, {props}])    删除属性类型
prop_type_get([{name} [, {props}])      获取属性类型值
prop_type_list([{props}])               获取属性类型的列表
```

操作文本属性：

```
prop_add({lnum}, {col}, {props})        新增文本属性
prop_clear({lnum} [, {lnum-end} [, {bufnr}]])
                                        删除所有文本属性
prop_find({props} [, {direction}])      搜索文本属性
prop_list({lnum} [, {props})            {lnum} 中的文本属性
prop_remove({props} [, {lnum} [, {lnum-end}]])
                                        删除一个文本属性
```

prop_add() E965

```
prop_add({lnum}, {col}, {props})
```

在位置 {lnum}, {col} 上附属文本属性。{col} 以字节计算，首列为一。
{lnum} 如果非法，报错。 E966
{col} 如果非法，报错。 E964

{props} 是带以下域的字典：
   length   以字节计的文本长度，只能用于不在另一行继续的属性；可为零
   end_lnum   文本结尾的行号
   end_col   文本刚刚结束之后的列；如果 "length" 存在则不用；如果 {col} 和 "end_col" 相等，且 "end_lnum" 省略或等于 {lnum}，这是个零宽度的文本属性
   bufnr   加入属性的缓冲区；省略是使用当前缓冲区
   id   用户定义的属性 ID；省略时使用零
   type   文本属性类型名
除 "type" 外，所有域均是可选的。

同时提供 "length" 和 "end_lnum" 或 "end_col" 是错误。要么用 "length" 或 "end_col" 用于单行的属性，或者用 "end_lnum" 和 "end_col" 用于跨多行的属性。
如果不给 "length" 也不给 "end_col"，属性为零宽度。这意味着它不会高亮，但会随着文本移动，就像位置标记一样。
属性可以正好在文本的最后字符上结束，也可以在它之后结束。后者如果在行上附加文本，文本属性会跟着变长，属性类型不置位 "end_incl" 时也是如此。

先在加入属性的缓冲区中寻找 "type"。如果找不到，再使用全局属性类型。还找不到的话会报错。

文本属性的相关信息可见 text-properties 。

也可用作 method :
```
GetLnum()->prop_add(col, props)
```

prop_clear({lnum} [, {lnum-end} [, {props}]]) prop_clear()
删除所有行 {lnum} 上的文本属性。
如果给出 {lnum-end}，删除从行 {lnum} 到 {lnum-end} (闭区间) 所有的文本属性。

如果 {props} 包含 "bufnr" 项目，使用此缓冲区，否则使用当前缓冲区。

文本属性的相关信息可见 text-properties 。

也可用作 method :
```
GetLnum()->prop_clear()
```

prop_find({props} [, {direction}]) prop_find()
{尚未实现}
搜索 {props} 指定的文本属性：
   id   带此 ID 的属性
   type   带此类型名的属性
   bufnr   在其中搜索的缓冲区；如果给出必须用 "lnum" 和 "col" 指定开始位置；如果省略使用当前缓冲区
   lnum   从此行开始 (省略时从光标所在行开始)

col 从此列开始（省略且给出 "lnum" 时：使用列 1，否则从光标开始）
skipstart 不寻找开始位置所在的匹配

{direction} 可以是 "f" 用于正向，"b" 用于反向。如果省略执行正向搜索。

如果找到匹配，返回字典，其项和 prop_list() 相同，另有 "lnum" 项。如果没找到匹配，返回空字典。

文本属性的相关信息可见 text-properties 。

prop_list({lnum} [, {props}]) prop_list()
返回行 {lnum} 上的所有文本属性的列表。

如果 {props} 包含 "bufnr" 项目，使用此缓冲区，否则使用当前缓冲区。

属性以开始列和优先级排序。每个属性是带下列项的字典：
col 开始列
length 以字节计的长度，如果包含换行符加一
id 属性 ID
type 属性类型名，如果类型已删除则省略
start 为 TRUE 时属性从这行开始
end 为 TRUE 时属性在这行结束

如果 "start" 为零，属性从前面行开始，本行是继续行。
如果 "end" 为零，属性从下面行继续。包含此行之后的换行符。

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetLnum()->prop_list()

prop_remove() E968
prop_remove({props} [, {lnum} [, {lnum-end}]])
删除 {lnum} 上的匹配文本属性。如果给出 {lnum-end}，删除从行 {lnum} 到 {lnum-end}（闭区间）所有匹配的文本属性。

{props} 是包含下列域的字典：
id 删除带此 ID 的文本属性
type 删除带此类型名的文本属性
bufnr 使用此缓冲区而非当前缓冲区
all 为 TRUE 时删除所有的匹配文本属性，而不仅是头一个
"id" 或 "type" 之一匹配则属性匹配。
如果缓冲区 "bufnr" 不存在会报错。
如果缓冲区 "bufnr" 未载入，什么都不会发生。

返回删除的属性个数。

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetProps()->prop_remove()

prop_type_add({name}, {props}) prop_type_add() E969 E970
新增文本属性类型 {name}。如果带此名的属性类型已存在则报错。不返回任何值。
{props} 是包含下面可选域的字典:

bufnr	只为此缓冲区定义属性；它避免了名字冲突，而且缓冲区删除时自动清除所有的属性类型。
highlight	使用的高亮组名
priority	当一个字符有多个文本属性时使用最高优先级的属性；可用负值，缺省优先级为零
combine	为 TRUE 时和任何语法高亮合并；省略或为 FALSE 时，不使用语法高亮
start_incl	TRUE 时在起始位置的插入会包含在文本属性内
end_incl	TRUE 时在结束位置的插入会包含在文本属性内

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetPropName()->prop_type_add(props)

prop_type_change({name}, {props}) prop_type_change()
修改已有的文本属性类型的属性。如果带此名的属性不存在则报错。
{props} 参数可见 prop_type_add() 。

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetPropName()->prop_type_change(props)

prop_type_delete({name} [, {props}]) prop_type_delete()
删除文本属性类型 {name}。如果带类型 {name} 的文本属性还在使用中，它们不再有效，而且也不能依名字删除。

{props} 可以包含 "bufnr" 项目。如果给出，删除来自此缓冲区的属性类型而不是全局属性类型。

如果找不到文本属性类型 {name}，不报错。

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetPropName()->prop_type_delete()

prop_type_get([{name} [, {props}]) prop_type_get()
返回属性类型 {name} 的属性。这是带和 prop_type_add() 给出的相同域的字典。
如果属性类型 {name} 不存在，返回空字典。

{props} 可以包含 "bufnr" 项目。如果给出，使用来自此缓冲区的属性类型而不是全局属性类型。

文本属性的相关信息可见 text-properties 。

也可用作 method :
GetPropName()->prop_type_get()

prop_type_list([{props}]) prop_type_list()
返回掺属性类型名的列表。

{props} 可以包含 "bufnr" 项目。如果给出，使用来自此缓冲区的属性类型而不是全局属性类型。

文本属性的相关信息可见 text-properties 。

---

3. 文本有改动时 text-prop-changes

Vim 会尽力保持文本属性在原先附属的文本上。插入或删除文本后，属性会相应地移动。

当文本删除后导致文本属性不再包含任何文本，会删除文本属性。不过，定义时就是零宽度的文本属性会保留，除非整行都被删除。

E275
卸载缓冲区时，所有的文本属性都消失了。没有办法把属性保存在文件里。你可以重建它们。缓冲区隐藏时，保留文本所以也保留文本属性。不可以给卸载的缓冲区增加文本属性。

使用替换模式时，在相同的字符位置上保留文本属性，虽然字符本身发生了改变。

要在文本有改动后更新文本属性，用 listener_add() 安装回调。例如，如果你的插件进行拼写检查，可以用回调为改动过的文本更新拼写错误。Vim 会移动改动文本下方的属性，这样它们还能高亮相同的文本，你不需要自己去更新。

不更新文本属性列：

- 通过 setline() 或 Lua、Tcl 或 Python 等界面设置行时，Vim 不知道插入或删除的是什么文本。

vim:tw=78:ts=8:noet:ft=help:norl:

filetype.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Willis，tocer
http://vimcdoc.sf.net

文件类型 filetype file-type

1. 文件类型 filetypes
2. 文件类型插件 filetype-plugins
3. 缺省文件类型插件的文档 ftplugin-docs

另见 autocmd.txt 。

---

1. 文件类型 filetypes file-types

Vim 可以检测要编辑的文件类型。这是通过检查文件名完成的，有的时候也检查文本里是否包含特定的文本。

:filetype :filet

要打开文件类型的检测，在你的 vimrc 里加入以下命令：

```
:filetype on
```

每次一个新的或者已经存在的文件被编辑时，Vim 会试图识别文件的类型，并设置 'filetype' 选项。同时，也触发 FileType 事件。该事件可以设置语法高亮，特定选项，等等。

注意：文件类型和 'compatibile' 不能很好的一起工作。因为 Vi 兼容意味着所有的选项都是全局的。如果你还那么做，建议复位 'compatible' 选项。

详情："filetype on" 命令会加载以下文件中的一个：

```
Amiga       $VIMRUNTIME/filetype.vim
Mac         $VIMRUNTIME:filetype.vim
MS-DOS      $VIMRUNTIME\filetype.vim
RiscOS      Vim:Filetype
Unix        $VIMRUNTIME/filetype.vim
VMS         $VIMRUNTIME/filetype.vim
```

该文件是一个 Vim 脚本，其中定义了 BufNewFile 和 BufRead 事件的自动命令。如果根据文件名不能检测到文件类型，则使用 $VIMRUNTIME/scripts.vim 来根据文件内容检测之。
正在或即将启动 GUI 时，同时也会执行 menu.vim 脚本。如果不想要，参见 'go-M' 。

要加入你自己的文件类型，见下面的 new-filetype 。要查找一种文件类型的帮助，加上 "ft-" 前缀和可选的 "-syntax"、"-indent" 或者 "-plugin" 后缀。例如：

```
:help ft-vim-indent
:help ft-vim-syntax
:help ft-man-plugin
```

如果文件类型没有被自动识别，或者找到的是错误的类型，你可以手动设置 'filetype' 选项，或者在文件里加入模式行。例如，对于 IDL 文件，可以使用

```
:set filetype=idl
```

或者在文件中加入模式行 modeline :

```
/* vim: set filetype=idl : */
```

:filetype-plugin-on

你可以为特定的文件类型允许插件文件的载入:

```
:filetype plugin on
```

如果文件类型检测以前尚未打开，载入该文件后就被打开了。该命令实际上载入了 'runtimepath' 里的 "ftplugin.vim" 文件。其结果是，在文件编辑时，相应的插件文件也会载入 (如果文件类型能检测到)。 filetype-plugin

:filetype-plugin-off

你可以如此关闭之:

```
:filetype plugin off
```

文件类型检测此时没有关闭。但如果你关闭了，相应的插件就不会被自动载入。该命令实际上载入了 'runtimepath' 里的 "ftplugof.vim" 文件。

:filetype-indent-on

你可以为特定的文件类型载入缩进文件:

```
:filetype indent on
```

如果文件类型检测以前尚未打开，载入该文件后就被打开了。该命令实际上载入了 'runtimepath' 里的 "indent.vim" 文件。其结果是，在文件编辑时，相应的缩进文件也会载入 (如果文件类型能检测到)。 indent-expression

:filetype-indent-off

你可以如此关闭之:

```
:filetype indent off
```

文件类型检测此时没有关闭。但如果你关闭了，相应的缩进文件就不会被自动载入。该命令实际上载入了 'runtimepath' 里的 "indoff.vim" 文件。
这会关闭你后面所打开的文件中的自动缩进。在已经打开的文件中，自动缩进还依然有效。要在已打开的文件中关闭缩进，复位 'autoindent'、'cindent'、'smartindent' 和/或 'indentexpr'。

:filetype-off

要关闭文件类型检测，使用如下命令:

```
:filetype off
```

"plugin" 和 "indent" 的标志位还会保持，但是因为没有文件类型被检测到，直到下次 ":filetype on" 为止，它们不会真的工作。

总览: :filetype-overview

命令	检测	plugin	indent
:filetype on	打开	未改变	未改变
:filetype off	关闭	未改变	未改变
:filetype plugin on	打开	打开	未改变
:filetype plugin off	未改变	关闭	未改变
:filetype indent on	打开	未改变	打开
:filetype indent off	未改变	未改变	关闭
:filetype plugin indent on	打开	打开	打开
:filetype plugin indent off	未改变	关闭	关闭

要查看当前的状态，只需输入:

```
:filetype
```

输出结果看起来大概如此：

```
filetype detection:ON  plugin:ON  indent:OFF
```

文件类型也用于语法高亮。如果使用 ":syntax on" 命令，文件类型检测也被安装。所以，没有必要在 ":syntax on" 之后再执行 ":filetype on"。

要关闭某种文件类型，在你的文件类型文件里加入一行。见 remove-filetype 。

filetype-detect

要再次检测文件类型：

```
:filetype detect
```

使用这个命令可以使你先编辑一个空白文件，任何输入足够的文本，使得文件类型能够被识别。例如，你以此开始输入一个新的外壳脚本："#!/bin/csh"。

如果文件类型检测被关闭，它会先被打开。就像用了 "on" 参数那样。

filetype-overrule

如果两个不同的文件类型使用了相同的扩展名，Vim 试图猜测到底该文件属于哪个类型。这并不总行。一些全局变量可以用来为某些特定的扩展名决定具体的文件类型：

文件名	变量	
*.asa	g:filetype_asa	ft-aspvbs-syntax ft-aspperl-syntax
*.asp	g:filetype_asp	ft-aspvbs-syntax ft-aspperl-syntax
*.asm	g:asmsyntax	ft-asm-syntax
*.prg	g:filetype_prg	
*.pl	g:filetype_pl	
*.inc	g:filetype_inc	
*.w	g:filetype_w	ft-cweb-syntax
*.i	g:filetype_i	ft-progress-syntax
*.p	g:filetype_p	ft-pascal-syntax
*.sh	g:bash_is_sh	ft-sh-syntax
*.tex	g:tex_flavor	ft-tex-plugin

filetype-ignore

要避免检测某些文件，用 g:ft_ignore_pat 变量。缺省值是：

```
:let g:ft_ignore_pat = '\.\(Z\|gz\|bz2\|zip\|tgz\)$'
```

这意味着压缩文件的内容不会被检测。

new-filetype

如果你想使用的文件没检测到文件类型，有四种方法。不管如何，最好不要修改 $VIMRUNTIME/filetype.vim 文件。再安装新的 Vim 版本的时候它会被覆盖。

A. 如果你想否决所有的缺省文件类型检查。
这里，每种文件类型需要写一个文件。缺点就是，你需要很多文件。优点就是，你只需要把文件放到合适的目录就可以了。

ftdetect

1. 建立你自己的运行时目录。通常，你会用 'runtimepath' 选项的第一项。然后，在其中建立 "ftdetect" 目录。Unix 上的例子：

```
:!mkdir ~/.vim
:!mkdir ~/.vim/ftdetect
```

2. 建立包含检测文件类型的自动命令的文件。例如：

```
au BufRead,BufNewFile *.mine            set filetype=mine
```

注意 这里没有 "augroup" 命令。在执行你的文件的时候这点已经做好了。你可以使用 "*" 模式，然后分析文件内容来检测其类型。

把该文件叫做 "mine.vim"，并把它放在用户运行时目录的 "ftdetect" 目录里。例如，在 Unix 上：

```
:w ~/.vim/ftdetect/mine.vim
```

3. 要使用新的文件类型检测，你需要重新启动 Vim。

"ftdetect" 目录的文件在所有的缺省检查之后进行，这样它们就能否决之前进行的文件类型检测。但是你还可以用 :setfiletype 保持前一个检测到的文件类型。

B. 如果你想在缺省文件类型检查之后检测自己的文件。

和上述的 A 类似。但不是无条件的设置 'filetype'，而是使用 ":setfiletype"。该命令只有在没有文件类型被监测到的前提下才会设置 'filetype'。例如：

```
au BufRead,BufNewFile *.txt             setfiletype text
```

在你的命令里，你可以使用已经检测到的文件类型。例如，在 "pascal" 类型已经被检测到的前提下，要使用文件类型 "mypascal"：

```
au BufRead,BufNewFile *         if &ft == 'pascal' | set ft=mypascal
                                                                | endif
```

C. 如果你的文件类型可以通过文件名检测。

1. 建立你自己的运行时目录。通常，你会用 'runtimepath' 选项的第一项。Unix 上的例子：

```
:!mkdir ~/.vim
```

2. 创建包含自动命令的文件用于检测文件类型。示例：

```
" my filetype file
if exists("did_load_filetypes")
  finish
endif
augroup filetypedetect
  au! BufRead,BufNewFile *.mine         setfiletype mine
  au! BufRead,BufNewFile *.xyz          setfiletype drawing
augroup END
```

在用户运行时目录下，把该文件存为 "filetype.vim"。例如，在 Unix 上：

```
:w ~/.vim/filetype.vim
```

3. 要使用新的文件类型检测，你需要重新启动 Vim。

这里，你的 filetype.vim 会在缺省的 FileType 自动命令安装前被执行。所以，你的自动命令会先被匹配，而 ":setfiletype" 命令的时候保证了其后没有其他的自动命令会改变 'filetype'。

new-filetype-scripts

D. 如果你的文件类型只能通过分析文件内容来检测。

1. 建立你的用户运行时目录。通常，你会用 'runtimepath' 选项的第一项。Unix 上的例子：

```
:!mkdir ~/.vim
```

2. 创立一个 vim 脚本，来实现需要的功能。示例：

```
if did_filetype()       " filetype 已经设立..
  finish                " ..不需要这些检测
endif
if getline(1) =~ '^#!.*\<mine\>'
```

```
  setfiletype mine
elseif getline(1) =~? '\<drawing\>'
  setfiletype drawing
endif
```

更多的例子可以参见 $VIMRUNTIME/scripts.vim。
把该文件命名为 "scripts.vim"，并保存在你的用户运行时目录。例如，在 Unix 上：

```
:w ~/.vim/scripts.vim
```

3. 检测会立即生效，无须重新启动 Vim。

你的 scripts.vim 在文件类型的缺省之前被载入，这意味着你的规则会覆盖 $VIMRUNTIME/scripts.vim 的缺省规则。

remove-filetype

如果某个文件类型检测有问题，安装一个 filetype.vim 或者 scripts.vim 去捕获这个问题（见上）。你可以把 'filetype' 设成一个不存在的名字，使得以后不会被设置：

```
:set filetype=ignored
```

如果你为许多用户设置系统，而不想为每个用户分别增加/删除相同的为件类型，考虑把 filetype.vim 和 scripts.vim 写到一个大家都能用的运行时目录里。查看 'runtimepath' 里有没有能用的目录。如果没有，在 system-vimrc 里设置 'runtimepath'。切记要保存缺省的目录。

autocmd-osfiletypes

备注：现在此段代码已不再用，因为我们已移除了 RISC OS 实现。将来就用 'filetype' 选项便可。

如果操作系统支持在文件里存储文件类型，你可以指定某一文件类型专用的自动命令。

实际的类型检查决定于你运行 Vim 所处的平台；详情参见你的系统文档。

要在自动命令里使用 osfiletype 检查，你应该把待匹配类型的模式串用尖括号括起来，就像这样：

```
:au BufRead *.html,<&faf;HTML>  runtime! syntax/html.vim
```

这会匹配：

- 任何名字以 ".html" 结尾的文件
- 任何类型为 "&faf" 或者 "HTML" 的文件，这些类型的具体含义取决于你使用的 Vim 的版本。
  未知的类型被认为是 不 匹配。

你可以同时自动指定类型和模式（这时两者必须都匹配）：

```
:au BufRead <&fff>diff*
```

这个例子会匹配类型为 "&fff" 而且文件名以 "diff" 开始的文件。

plugin-details

"plugin" 目录可以在 'runtimepath' 选项里的任何一个目录里。所有这些目录都会被搜

索，查找到的插件会被全部调入。例如，如果命令：

```
set runtimepath
```

产生这样的输出：

```
runtimepath=/etc/vim,~/.vim,/usr/local/share/vim/vim60
```

那么 Vim 会调入以下目录及其下的所有插件：

```
/etc/vim/plugin/
~/.vim/plugin/
/usr/local/share/vim/vim60/plugin/
```

注意 最后一项是 $VIMRUNTIME 被扩展的值。

备注 使用插件管理器或 packages 时，会在 'runtimepath' 中加入很多目录。这些插件每个需要自己的目录，请不要把它们直接加进 ~/.vim/plugin。

有没有可能你的插件好像没有被调入呢？通过 -V 参数，你可以看看 Vim 启动的时候发生了什么：

```
vim -V2
```

你会看到很多信息。其中有关于调入插件的一行注释。它这样开始：

```
Searching for "plugin/**/*.vim" in
```

这样，你就能看到 Vim 在哪里查找你的插件脚本了。

---

2. 文件类型插件 filetype-plugins

如果允许载入文件类型插件 :filetype-plugin-on ，相应的选项会被设置，映射会被定义。它们都局部于缓冲区，不会影响其他的文件。

定义文件类型的映射可能会影响你自己定义的映射。有一些方法可以避免这一点：

1. 设置 "maplocalleader" 变量为你想映射开始的键序列。例如：
   ```
   :let maplocalleader = ","
   ```
   所有的映射现在都以逗号开始，而不是缺省的反斜杠。另见 <LocalLeader> 。

2. 定义你自己的映射。例如：
   ```
   :map ,p <Plug>MailQuote
   ```
   你需要检查下面给出的插件文件的描述，以了解它们的功能和映射到的字符串。
   你需要在插件载入之前定义自己的映射。（在编辑该类型的文件之前）。该插件就可以跳过缺省映射的安装了。

3. 屏蔽特定文件类型的映射可以通过设置一个变量进行。该变量包含文件类型的名字。 no_mail_maps
   对于 "mail" 文件类型它可能是：
   ```
   :let no_mail_maps = 1
   ```

4. 通过设置变量屏蔽所有文件类型的映射： no_plugin_maps
   ```
   :let no_plugin_maps = 1
   ```

ftplugin-overrule

如果某个全局文件类型插件不完全符合你的期待，有三个办法可以改变：

1. 增加一些设置。
   你必须在 'runtimepath' 靠前的一个目录里建立一个新的文件类型插件。Unix 上，你可以使用这个文件：
   ```
   vim ~/.vim/ftplugin/fortran.vim
   ```
   你可以设置你希望有的选项和映射。注意，全局插件会在其后进行。它可能会否决你做的那些设置。如果发生了，你有以下两种方法。

2. 建立一个插件的备份，并改变之。
   你需要把该备份放在 'runtimepath' 靠前的位置。Unix 上，你可以这么做：
   ```
   cp $VIMRUNTIME/ftplugin/fortran.vim ~/.vim/ftplugin/fortran.vim
   ```
   这样你可以自由的编辑你的备份文件。因为 b:did_ftplugin 会被设置，全局的插件就不会载入自身。这样做的一个缺点是，如果发行的插件版本更新，你需要重新复制并再次进行修改。

3. 在载入全局插件以后否决一些设置。
   你需要在 'runtimepath' 的尾部建立一个新的文件类型插件。Unix 上，你可以使用这个文件：
   ```
   vim ~/.vim/after/ftplugin/fortran.vim
   ```
   在这个文件里，你可以对需要的设置进行改变。

---

3. 缺省文件类型插件的文档

ftplugin-docs

CHANGELOG

ft-changelog-plugin

允许方便的输入 Changlog 文件的 Changelog 段 (entry)。有些命令，映射和变量值得探索一下：

选项：

'comments'	置为空，以免影响排版。
'textwidth'	设为 78，这是标准。
'formatoptions'	增加 't' 标志位，使得插入文本时能进行回绕。

命令：

NewChangelogEntry	智能地增加新的 Changelog 段 (见下)。

局部映射：

<Leader>o	同样智能地开始一个新的 Changlog 段 (见下)。

全局映射：

注意：全局设置是通过先执行 ftplugin/changelog.vim 文件设置的。例如在你的 .vimrc 里加上：
```
runtime ftplugin/changelog.vim
```

<Leader>o	切换到为当前目录打开的 ChangeLog 缓冲区。如果当前目录已存在该文件，在新的缓冲区打开之。然后执行上述局部 <Leader>o 的功能。

变量:

g:changelog_timeformat 已废弃；请用 g:changelog_dateformat 代替。

g:changelog_dateformat Changelog 段使用的日期（和时间）格式。该格式和 strftime() 函数使用的语法相同。
缺省值是 "%Y-%m-%d"，这是许多 ChangeLog 排版所使用的标准格式。

g:changelog_username 用户的名字和 email 地址。缺省值从环境变量和系统文件里推得。它先搜索当前用户 /etc/passwd 的注释部分。非正式地，该项在第一个分隔的逗号之前的部分包含了用户的实际名字。然后它再检查 $NAME 环境变量，最后运行 'whoami' 和 'hostname' 来构造一个 email 地址。最终形式是

```
Full Name  <user@host>
```

g:changelog_new_date_format
建立新的日期项目时所用的格式。下表描述字符串中的特殊记号:

```
%%      插入单个 '%' 号
%d      插入上述的日期
%u      插入上述的用户
%p      插入 b:changelog_entry_prefix 的值
%c      完成时光标的位置
```

缺省值是 "%d %u\n\n\t* %p%c\n\n"，它会产生如下的结果（| 是光标键要到的位置，除非出现在行首。那里它就是意味着一行的开始）

```
|2003-01-14  Full Name  <user@host>
|
|       * prefix|
```

g:changelog_new_entry_format
建立新段所用的格式。下表描述字符串里的特殊记号:

```
%p      插入 b:changelog_entry_prefix 的值
%c      完成时光标的位置
```

缺省值是 "\t*%c"，产生的结果如下

```
|       * prefix|
```

g:changelog_date_entry_search
搜索日期项所用的搜索模式。
g:changelog_new_date_format 所使用的特殊记号这里也可以使用。
缺省值是 '^\s*%d_s*%u'，它能找到匹配如下形式的行

```
|2003-01-14  Full Name  <user@host>
```

和一些类似的格式。

g:changelog_date_end_entry_search
搜索日期项结尾所用的搜索模式。
g:changelog_new_date_format 所使用的特殊记号这里也可以使用。
缺省值是 '^\s*$'，它匹配只包含空白或全空的行。

b:changelog_name b:changelog_name
寻找的 ChangeLog 文件名。缺省是 'ChangeLog'。

b:changelog_path
当前缓冲区使用的 ChangeLog 路径。缺省为空，也就是寻找

和当前缓冲区同一目录的名为 b:changelog_name 的文件。如果没找到，搜索当前缓冲区的父目录，如此递归直到找到文件或者没有更多的父目录可供搜索。

b:changelog_entry_prefix
函数名，用于生成新段的前缀。该函数没有参数，而应返回包含前缀的字符串。
可以返回空前缀。
缺省生成 ChangeLog 的路径名和当前缓冲区路径名两者的较短者。此变量将来应该可以使用其它变量上下文，例如 g:。

Changelog 段插入时，只需要输入最少量的文本。在确定当前日期和用户后，在该文件里会搜索以当前日期和用户开头的一段，如果找到了，则在其后添加一个新项。如果没有，则从 Changelog 文件开始处添加新的段和项目 (item)。

FORTRAN ft-fortran-plugin

选项:
'expandtab' 被打开，按照 Fortran 标准的要求，不应出现制表字符。用户在 .vimrc 文件里设置 fortran_have_tabs 除外。
'textwidth' 按照 Fortrean 标准要求， 固定宽度源文件格式设为 72，自由格式的源代码设为 80。
'formatoptions' 设为能断开代码和注释行，并保留长行。你可以用 gq 排版注释。
更多的关于 fortran_have_tabs 和检测源代码格式的方法的讨论，可见 ft-fortran-syntax 。

GIT COMMIT ft-gitcommit-plugin

提供了一个命令 :DiffGitCached 来在预览窗口中显示当前提交的差异部分。等价于 "git diff --cached" 加上此命令后面的参数。

MAIL ft-mail-plugin

选项:
'modeline' 被关闭，以防特洛伊木马。也防止含有 "Vim:" 字样的标题会引起错误信息。
'textwidth' 设为 72。这是 e-mail 推荐的格式。
'formatoptions' 设为能断开文本行，并且在新行里重复注释的前导符，这样引用开头的 ">" 就可以重复了。你也可以用 gq 排版引用文本。

局部映射:
<LocalLeader>q 或者 \\MailQuote
引用可视模式下的选择文本，或者在普通模式下从光标位置到文件末的文本。这意味着 "> " 会插到每行的开始。

MAN ft-man-plugin :Man man.vim

较好地显示手册页。另见用户手册中的 find-manpage 。

要在调入任何手册页之前就能使用 ":Man" 命令，你需要在启动的 vimrc 文件里先执行以下脚本:

```
runtime ftplugin/man.vim
```

选项:
'iskeyword'　　加入 '.' 字符，以便能在手册页名字上使用 CTRL-]。

命令:
Man {name}　　在窗口里显示 {name} 的手册页
Man {number} {name}
　　在第 {number} 节里显示 {name} 的手册页。

全局映射:
<Leader>K　　显示当前光标下的单词的相应手册页。
<Plug>ManPreGetPage　同上，可用于映射:

```
nmap <F1> <Plug>ManPreGetPage<CR>
```

局部映射:
CTRL-]　　跳到当前光标下的单词的相应手册页。
CTRL-T　　跳回前一次的手册页。
q　　同 ":quit"

要使用垂直映射而不是水平映射:

```
let g:ft_man_open_mode = 'vert'
```

要用新标签页:

```
let g:ft_man_open_mode = 'tab'
```

要打开折叠:

```
let g:ft_man_folding_enable = 1
```

如果不喜欢缺省的折叠方式，使用自动命令加入你喜欢和折叠风格。例如:

```
autocmd FileType man setlocal foldmethod=indent foldenable
```

如果你希望 :Man {number} {name} 像 man {number} {name} 那样，如果找不到页面就不运行 man {name}，可以这样用:

```
let g:ft_man_no_sect_fallback = 1
```

你可能还想设置 'keywordprg'，使 K 命令在 Vim 窗口打开手册页:

```
set keywordprg=:Man
```

MANPAGER　　manpager.vim

:Man 命令把 Vim 变成手册浏览器 (语法高亮手册页，按 CTRL-] 时点击相连的手册页)。

对 bash、zsh、ksh 或 dash，加入对应的配置文件 (.bashrc、.zshrc、...)

```
export MANPAGER="vim -M +MANPAGER -"
```

对 (t)csh，加入配置文件

```
setenv MANPAGER "vim -M +MANPAGER -"
```

对 fish，加入配置文件

```
set -x MANPAGER "vim -M +MANPAGER -"
```

PDF ft-pdf-plugin

提供两个映射， <C-]> 和 <C-T>，来模拟用于 PDF 浏览的标签栈。以下内容视为标签：

- "startxref" 后指向 xref 表的字节位移
- 文件尾部中 /Prev 键后指向以前 xref 表的字节位移
- xref 表中形如 "0123456789 00000 n" 的行
- PDF 任意位置上形如 "1 0 R" 的对象引用

可以关闭这些映射

```
:let g:no_pdf_maps = 1
```

PYTHON ft-python-plugin PEP8

缺省设置以下选项，根据 PEPB：

```
setlocal expandtab shiftwidth=4 softtabstop=4 tabstop=8
```

要关闭此行为，在 vimrc 里设置变量：

```
let g:python_recommended_style = 0
```

QF QUICKFIX qf.vim ft-qf-plugin

"qf" 文件类型用于 quickfix 窗口，见 quickfix-window 。

quickfix 文件类型插件包括在 status-line 中显示生成 quickfix 列表命令的配置。
要关闭此设置，可以这样配置：

```
:let g:qf_disable_statusline = 1
```

R MARKDOWN ft-rmd-plugin

缺省不执行 ftplugin/html.vim。要执行之，在 vimrc 中加入：

```
let rmd_include_html = 1
```

动态地设置 R 代码和 markdown 代码不同的 'formatexpr' 值。如果你宁可
'formatexpr' 不置位，在 vimrc 中加入：

```
let rmd_dynamic_comments = 0
```

R RESTRUCTURED TEXT ft-rrst-plugin

动态地设置 R 代码和 ReStructured 代码不同的 'formatexpr' 值。如果你宁可
'formatexpr' 不置位，在 vimrc 中加入：

```
let rrst_dynamic_comments = 0
```

RESTRUCTUREDTEXT ft-rst-plugin

以下排版设置可选：

```
setlocal expandtab shiftwidth=3 softtabstop=3 tabstop=8
```

要打开此排版，在 vimrc 中设置以下变量：

```
let g:rst_style = 1
```

RPM SPEC ft-spec-plugin

因为这个插件的文本相当长，它被单独列出： pi_spec.txt 。

RUST ft-rust

因为这个插件的文本相当长，它被单独列出： ft_rust.txt 。

SQL ft-sql

因为这个插件的文本相当长，它被单独列出： ft_sql.txt 。

TEX ft-tex-plugin g:tex_flavor

如果 *.tex 文件的首行是如下的形式

```
%&<format>
```

那么它决定了文件类型： plaintex (普通的 TeX)、context (ConTeXt)、或者 tex (LaTeX)。否则在文件里搜索关键字以判断是 context 还是 tex。如果找不到关键字，缺省是 plaintex。可以通过定义变量 g:tex_flavor 改变缺省值为最常用的格式 (不是文件类型)。使用下列选择之一：

```
let g:tex_flavor = "plain"
let g:tex_flavor = "context"
let g:tex_flavor = "latex"
```

目前还不能识别其它格式。

VIM ft-vim-plugin

Vim 文件类型插件定义 [[和]] 用来移动到函数开头和结尾处的映射。用]" 和 [" 在注释间移动。

可以这样关闭这些映射：

```
let g:no_vim_maps = 1
```

ZIMBU ft-zimbu-plugin

Zimbu 文件类型插件定义 [[和]] 用来移动到函数开头和结尾处的映射。

可以这样关闭这些映射：

```
let g:no_zimbu_maps = 1
```

vim:tw=78:ts=8:noet:ft=help:norl:

quickfix.txt For Vim version 8.2. 最近更新: 2020年1月

VIM 参考手册 by Bram Moolenaar

译者: wandys、Willis http://vimcdoc.sf.net

本主题曾在用户手册的 30.1 节有所介绍。

1. 使用 QuickFix 命令 quickfix
2. 出错窗口 quickfix-window
3. 使用多个错误列表 quickfix-error-lists
4. 使用 :make :make_makeprg
5. 使用 :grep grep
6. 选择编译器 compiler-select
7. 错误格式 error-file-format
8. 目录栈 quickfix-directory-stack
9. 特定错误格式 errorformats

如编译时没有指定 +quickfix 则无法使用 quickfix 相关命令。

---

1. 使用 QuickFix 命令 quickfix Quickfix E42

Vim 有一个特殊的模式用来加快 编辑-编译-编辑 循环。这是受到 Amiga 系统上 Manx 的 Aztec C 编译器的 quickfix 选项的启发。它会将编译器产生的出错信息保存到一个文件中并用 Vim 跳转到各个错误。无需记住所有的出错信息，你可以检查每一个错误并对其修正。

Vim 里，quickfix 命令的使用更广泛，它可以找到文件各种位置的列表。例如，:vimgrep 寻找模式的匹配。在脚本里，用 getqflist() 函数可以得到这些位置。这样你可以比编辑/编译/修正循环做更多的事情！

如果有保存在文件里的出错信息，这样启动 Vim:

```
vim -q filename
```

从 Vim 里，一个简便的运行命令并处理输出的方法是使用 :make 命令（见下）。

你需要设定 'errorformat'，使其和编译器产生的出错信息格式一致（见 errorformat ）。

quickfix-ID

每个 quickfix 列表有唯一的标识符，叫做 quickfix ID，此数值在同一个 Vim 会话中保持不变。 getqflist() 函数可用于获取赋给该列表的标识符。还有一个 quickfix 列表号，该值在 quickfix 栈加入超过十个列表的时候可能会改变。

location-list E776

位置列表是一个窗口局部的 quickfix 列表。由 :lvimgrep 、 :lgrep 、 :lhelpgrep 、 :lmake 等命令产生，它们生成位置列表而不是对应的 :vimgrep 、 :grep 、 :helpgrep 、 :make 生成的 quickfix 列表。

location-list-file-window

位置列表和窗口相关联，而每个窗口都要单独的位置列表。一个位置列表只能和一个窗口相关联。位置列表和 quickfix 列表相互独立。

如果包含位置列表的窗口被分割，新窗口得到该位置列表的一个备份。如果一个位置列表不再被引用，该位置列表被删除。

quickfix-changedtick
每个 quickfix 和位置列表都有一个只读的 changedtick 变量，记录该列表发生过的总改动数。每次 quickfix 列表有改动时，此计数增一。此计数可用于在任何列表发生改动时执行些操作。 getqflist() 和 getloclist() 函数可用于查询 changedtick 的当前值。不可直接修改 changedtick 变量。

以下 quickfix 命令可供使用。位置列表命令和 quickfix 命令类似，把 quickfix 命令里的 'c' 前缀换成 'l' 就行了。

E924
如果 autocommand 关闭当前窗口的同时正在处理位置列表命令，中止该命令。

E925 E926
如果 autocommand 改变当前 quickfix 或位置列表的同时正在处理 quickfix 或位置列表命令，中止该命令。

:cc
:cc[!] [nr]
:[nr]cc[!]
显示错误 [nr]。如果不指定 [nr]，将再次显示相同的错误。
如果不指定 [!]，则以下情况发生时无法跳转到另一个缓冲区：若 'hidden' 和 'autowrite' 选项没被设定并且当前缓冲区只有一个窗口且被修改。
当指定 [!] 跳转到另一个缓冲区时，当前缓冲区的修改会丢失，除非设定了 'hidden' 或该缓冲区还有其它窗口。
当跳转到另一个缓冲区时，会受到 'switchbuf' 有关设定的影响。

:ll
:ll[!] [nr]
:[nr]ll[!]
和 ":cc" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cn :cne :cnext E553
:[count]cn[ext][!]d
显示含有文件名的列表中的往后第 [count] 个错误。如果没有文件名，则跳转到往后第 [count] 个错误。有关 [!] 和 'switchbuf'，参见 :cc 。

:lne :lnext
:[count]lne[xt][!]
和 ":cnext" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cp :cprevious :cprev :cN :cNext
:[count]cN[ext][!]
:[count]cp[revious][!]
显示含有文件名的列表中的往前第 [count] 个错误。如果没有文件名，则跳转到往前第 [count] 个错误。有关 [!] 和 'switchbuf'，参见 :cc 。

:lp :lprevious :lprev :lN :lNext
:[count]lN[ext][!]
:[count]lp[revious][!]
和 ":cNext" 与 ":cprevious" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cabo :cabove

:[count]cabo[ve]

转到当前缓冲区当前行上方第 [count] 个错误。[count] 省略时为 1。如果没有错误，报错。假定 quickfix 列表的项目按照它们的缓冲区号和行号排序。如果同一行上有多个错误，使用第一项。如果 [count] 超过当前行上方的项目数，选择文件的第一个错误。

:lab :labove

:[count]lab[ove]

同 ":cabove"，但使用当前窗口的位置列表而不是 quickfix 列表。

:cbel :cbelow

:[count]cbel[ow]

转到当前缓冲区当前行下方第 [count] 个错误。[count] 省略时为 1。如果没有错误，报错。假定 quickfix 列表的项目按照它们的缓冲区号和行号排序。如果同一行上有多个错误，使用第一项。如果 [count] 超过当前行下方的项目数，选择文件的最后一个错误。

:lbel :lbelow

:[count]lbel[ow]

同 ":cbelow"，但使用当前窗口的位置列表而不是 quickfix 列表。

:cbe :cbefore

:[count]cbe[fore]

转到当前缓冲区当前光标之前第 [count] 个错误。[count] 省略时为 1。如果没有错误，报错。假定 quickfix 列表的项目按照它们的缓冲区号、行号和列号排序。如果同一行上有多个错误，使用第一项。如果 [count] 超过当前位置之前的项目数，选择文件的第一个错误。

:lbe :lbefore

:[count]lbe[fore]

同 ":cbefore"，但使用当前窗口的位置列表而不是 quickfix 列表。

:caf :cafter

:[count]caf[ter]

转到当前缓冲区当前光标之后第 [count] 个错误。[count] 省略时为 1。如果没有错误，报错。假定 quickfix 列表的项目按照它们的缓冲区号、行号和列号排序。如果同一行上有多个错误，使用第一项。如果 [count] 超过当前位置之后的项目数，选择文件的最后一个错误。

:laf :lafter

:[count]laf[ter]

同 ":cafter"，但使用当前窗口的位置列表而不是 quickfix 列表。

:cnf :cnfile

:[count]cnf[ile][!]

显示含有文件名的列表中往后第 [count] 个文件中的首个错误。如果没有文件名或是列表中没有下一个文件，则跳转到往后第 [count] 个错误。有关 [!] 和 'switchbuf'，参见 :cc 。

:lnf :lnfile

:[count]lnf[ile][!]

和 ":cnfile" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:[count]cNf[ile][!]
:[count]cpf[ile][!]

:cpf :cpfile :cNf :cNfile

显示含有文件名的列表中往前第 [count] 个文件中的最后一个错误。如果没有文件名或是列表中没有下一个文件，则跳转到往前第 [count] 个错误。有关 [!] 和 'switchbuf'，参见 :cc 。

:[count]lNf[ile][!]
:[count]lpf[ile][!]

:lpf :lpfile :lNf :lNfile

和 ":cNfile" 与 ":cpfile" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cr[ewind][!] [nr]

:crewind :cr

显示错误 [nr]。如果没指定 [nr]，则显示 首 个错误。参见 :cc 。

:lr[ewind][!] [nr]

:lrewind :lr

和 ":crewind" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cfir[st][!] [nr]

:cfirst :cfir

和 ":crewind" 相同。

:lfir[st][!] [nr]

:lfirst :lfir

和 ":lrewind" 相同。

:cla[st][!] [nr]

:clast :cla

显示错误 [nr]。如果没指定 [nr]，则显示最后一个错误。参见 :cc 。

:lla[st][!] [nr]

:llast :lla

和 ":clast" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cq[uit][!]

:cq :cquit

以错误码退出 Vim。这样编译器就不会再次编译相同文件。
警告：对文件的所有改动都会丢失！即使没有 [!] 也是如此。类似于 ":qall!" :qall ，但 Vim 返回非零退出码。

:cf[ile][!] [errorfile]

:cf :cfile

读入错误文件并跳转到首个错误。以 -q 选项启动 Vim 时会自动这样处理。用这个命令，你就可以在编译的同时继续用 Vim 编辑。如果你指定错误文件名，那么 'errorfile' 选择将被设定为 [errorfile]。有关 [!]，参见 :cc 。
如果错误文件的编码和 'encoding' 选项不同，可用 'makeencoding' 选项来指定编码。

:lf[ile][!] [errorfile]

:lf :lfi :lfile

和 ":cfile" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。
你不能用 -q 命令行选项设置位置列表。

:cg[etfile] [errorfile]

:cg :cgetfile

读入错误文件。和 ":cfile" 类似，只是不跳转到首个错误。

如果错误文件的编码和 'encoding' 选项不同，可用
'makeencoding' 选项来指定编码。

:lg[etfile] [errorfile] :lg :lge :lgetfile
和 ":cgetfile" 类似，但使用当前窗口的位置列表而不是
quickfix 列表。

:caddf[ile] [errorfile] :caddf :caddfile
读入错误文件，把错误文件里的错误加到当前 quickfix 列表
中。如果 quickfix 列表不存在，建立新的列表。
如果错误文件的编码和 'encoding' 选项不同，可用
'makeencoding' 选项来指定编码。

:laddf[ile] [errorfile] :laddf :laddfile
和 ":caddfile" 类似，但使用当前窗口的位置列表而不是
quickfix 列表。

:cb[uffer][!] [bufnr] :cb :cbuffer E681
从当前缓冲区读入错误列表。如果给出 {bufnr}，它必须是已
经载入的缓冲区号。这时使用该缓冲区，而不是当前缓冲区。
可以指定使用的行范围。否则使用缓冲区的所有行。
[!] 部分见 :cc 。

:lb[uffer][!] [bufnr] :lb :lbuffer
和 ":cbuffer" 类似，但使用当前窗口的位置列表而不是
quickfix 列表。

:cgetb[uffer] [bufnr] :cgetb :cgetbuffer
从当前缓冲区读入错误列表。类似于 ":cbuffer"，只是不跳
转到首个错误。

:lgetb[uffer] [bufnr] :lgetb :lgetbuffer
和 ":cgetbuffer" 类似，但使用当前窗口的位置列表而不是
quickfix 列表。

:cad[dbuffer] [bufnr] :cad :cadd :caddbuffer
读入当前缓冲区的错误列表，把这些错误加到当前 quickfix
列表中。如果 quickfix 列表不存在，建立新的列表。其它部
分和 ":cbuffer" 相同。

:laddb[uffer] [bufnr] :laddb :laddbuffer
和 ":caddbuffer" 类似，但使用当前窗口的位置列表而不是
quickfix 列表。

:cex[pr][!] {expr} :cex :cexpr E777
用 {expr} 的计算结果建立 quickfix 列表并跳转到首个错
误。
如果 {expr} 是字符串，那么字符串里每个换行符结尾的
行用全局值 'errorformat' 进行处理，其结果用来产生
quickfix 列表。
如果 {expr} 是列表类型，那么列表里的每个字符串项目被处
理并加到 quickfix 列表里。列表里的非字符串项目被忽略。
[!] 部分见 :cc 。

示例：

```
:cexpr system('grep -n xyz *')
:cexpr getline(1, '$')
```

:lex[pr][!] {expr}　　:lex　:lexpr

和 :cexpr 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cgete[xpr] {expr}　　:cgete　:cgetexpr

用 {expr} 的计算结果建立 quickfix 列表。和 :cexpr 类似，只是不跳转到首个错误。

:lgete[xpr] {expr}　　:lgete　:lgetexpr

和 :cgetexpr 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cadde[xpr] {expr}　　:cadde　:caddexpr

把 {expr} 的计算结果行加到当前 quickfix 列表中。如果 quickfix 列表不存在，建立新的列表。不改变当前的光标位置。详情见 :cexpr 。
示例：

```
:g/mypattern/caddexpr expand("%") . ":" . line(".") . ":" . getline(".")
```

:lad[dexpr] {expr}　　:lad　:addd　:laddexpr

和 ":caddexpr" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cl[ist] [from] [, [to]]　　:cl　:clist

显示所有有效的错误（参见 quickfix-valid ）。
如果 [from] 和/或 [to] 被指定，则显示指定范围内的错误。负数从最后一个错误向前算起，-1 为最后一个错误。当跳转到另一个缓冲区时，会受到 'switchbuf' 有关设定的影响。
:filter 命令可用来只显示匹配指定模式的 quickfix 项目。该模式匹配文件名、模块名、项目的模式和文本。

:cl[ist] +{count}　　列出当前和下面 {count} 个有效的错误。类似于 ":clist from from+count"，这里 "from" 是当前的错误位置。

:cl[ist]! [from] [, [to]]
显示所有的错误。

:cl[ist]! +{count}　　列出当前和下面 {count} 个错误行。可用于看见当前行后未识别的行。例如，如果 ":clist" 给出：

```
8384 testje.java:252: error: cannot find symbol
```

然后，用 ":cl! +3" 显示原因：

```
8384 testje.java:252: error: cannot find symbol
8385:   ZexitCode = Fmainx();
8386:               ^
8387:   symbol:   method Fmainx()
```

:lli[st] [from] [, [to]]　　:lli　:llist

和 ":clist" 类似，但使用当前窗口的位置列表而不是

quickfix 列表。

:lli[st]! [from] [, [to]]
列出当前窗口的位置列表的所有项目。

如果你插入或删除行，大部分错误仍能被正确的找到，因为隐藏标记会被使用。有时，当标记由于某种原因被删除，"line changed" 信息会显示以警告你出错位置可能有误。如果你退出并重新运行 Vim，标记会丢失，而出错位置也可能不再正确。

可以在 quickfix 命令 (':make'、':grep' 等) 执行前与后使用两个自动命令。详见 QuickFixCmdPre 和 QuickFixCmdPost 。

QuickFixCmdPost-example
如果 'encoding' 不同于 locale，错误信息可能和 Vim 使用的编码不同。以下代码可以进行消息的转换:

```
function QfMakeConv()
   let qflist = getqflist()
   for i in qflist
      let i.text = iconv(i.text, "cp936", "utf-8")
   endfor
   call setqflist(qflist)
endfunction

au QuickfixCmdPost make call QfMakeConv()
```

另一个选项是用 'makeencoding'。

quickfix-title
每个 quickfix 和位置列表都有标题。缺省标题设为建立此列表的命令。 getqflist() 和 getloclist() 函数可分别用于获取 quickfix 和位置列表的标题。 setqflist() 和 setloclist() 函数可分别用于修改 quickfix 和位置列表的标题。示例:

```
call setqflist([], 'a', {'title' : 'Cmd output'})
echo getqflist({'title' : 1})
call setloclist(3, [], 'a', {'title' : 'Cmd output'})
echo getloclist(3, {'title' : 1})
```

quickfix-index
使用任何一个 quickfix 命令 (例如 :cc 、 :cnext 、 :cprev 等) 来跳转到某 quickfix/位置列表项目时，该项成为当前选择项。quickfix/位置列表的当前选择项的索引可由 getqflist()/getloclist() 函数获得。例如:

```
echo getqflist({'idx' : 0}).idx
echo getqflist({'id' : qfid, 'idx' : 0}).idx
echo getloclist(2, {'idx' : 0}).idx
```

对于新 quickfix 列表而言，选择的是首项，其索引为 1。quickfix/位置列表的任何项目都可以用 setqflist() 函数设为当前选择项。例如:

```
call setqflist([], 'a', {'idx' : 12})
call setqflist([], 'a', {'id' : qfid, 'idx' : 7})
call setloclist(1, [], 'a', {'idx' : 7})
```

quickfix-size
getqflist() 和 getloclist() 函数可分别用于获取 quickfix 和位置列表的项目数 (size)。示例:

```
echo getqflist({'size' : 1})
echo getloclist(5, {'size' : 1})
```

quickfix-context

quickfix 和位置列表可关联任何 Vim 类型作为上下文。 setqflist() 和 setloclist() 函数可分别用于建立 quickfix 和位置列表上下文的关联。 getqflist() 和 getloclist() 函数可分别用于获取 quickfix 和位置列表的上下文。这可用于处理多个 quickfix/位置列表的 Vim 插件。
示例:

```
let somectx = {'name' : 'Vim', 'type' : 'Editor'}
call setqflist([], 'a', {'context' : somectx})
echo getqflist({'context' : 1})

let newctx = ['red', 'green', 'blue']
call setloclist(2, [], 'a', {'id' : qfid, 'context' : newctx})
echo getloclist(2, {'id' : qfid, 'context' : 1})
```

quickfix-parse

使用 getqflist() 函数，不用建立或修改 quickfix 列表你就可以用 'errorformat' 来解析若干行。例如:

```
echo getqflist({'lines' : ["F1:10:Line10", "F2:20:Line20"]})
echo getqflist({'lines' : systemlist('grep -Hn quickfix *')})
```

返回字典，其中的 'items' 键包含从 lines 解析出来的 quickfix 项目。下面展示了如何使用定制的 'errorformat' 来解析 lines 而无须修改 'errorformat' 选项:

```
echo getqflist({'efm' : '%f#%l#%m', 'lines' : ['F1#10#Line']})
```

在所有 QUICKFIX 或位置列表的缓冲区中执行一个命令:

:cdo

:cdo[!] {cmd}

在 quickfix 列表的每个有效的项目上执行 {cmd}。
工作方式相当于:

```
:cfirst
:{cmd}
:cnext
:{cmd}
等等
```

如果当前文件不能 abandon 且没给出 [!] 时，命令失败。
转到下个项目失败时，执行中止。
最后一个缓冲区 (或错误发生所在的那个) 成为当前缓冲区。
{cmd} 可包含 '|' 以连接几个命令。

只使用 quickfix 列表里的有效的项目。可使用范围来选择项目，例如:

```
:10,$cdo cmd
```

来跳过 1-9 个项目。

备注: 此命令执行时，把它加到 'eventignore' 以关闭语法自动命令事件。这样显著地提高了缓冲区编辑的速度。
另见 :bufdo 、 :tabdo 、 :argdo 、 :windo 、 :ldo 、 :cfdo 和 :lfdo 。

:cfdo

:cfdo[!] {cmd}

在 quickfix 列表的每个文件上执行 {cmd}。
工作方式相当于:

```
:cfirst
```

```
:{cmd}
:cnfile
:{cmd}
等等
```

此外，和 :cdo 相同。

:ldo

:ld[o][!] {cmd}　在当前窗口位置列表的每个有效的项目上执行 {cmd}。
工作方式相当于：

```
:lfirst
:{cmd}
:lnext
:{cmd}
etc.
```

只使用位置列表里的有效的项目，此外，和 :cdo 相同。

:lfdo

:lfdo[!] {cmd}　在当前窗口位置列表的每个文件上执行 {cmd}。
工作方式相当于：

```
:lfirst
:{cmd}
:lnfile
:{cmd}
etc.
```

此外，和 :ldo 相同。

过滤 QUICKFIX 或位置列表：

cfilter-plugin :Cfilter :Lfilter

如果 quickfix 列表有太多项目，可用 cfilter 插件来减少项目数。这样载入插件：

```
packadd cfilter
```

然后可用下列命令来过滤 quickfix/位置列表：

```
:Cfilter[!] /{pat}/
:Lfilter[!] /{pat}/
```

:Cfilter 命令从当前 quickfix 列表中匹配 {pat} 的项目来创建新的 quickfix 列表。{pat} 是 Vim regular-expression 模式。{pat} 既匹配文件名，也匹配项目文本。如果提供了可选的 !，则使用不匹配 {pat} 的项目。可选地，可用 '、"、/ 字符之一来括起模式。如果模式为空，使用前次使用的搜索模式。

:Lfilter 和 :Cfilter 相同，但用于当前位置列表。

---

2. 出错窗口　quickfix-window

:cope :copen w:quickfix_title

:cope[n] [height]　打开一个窗口显示当前的错误列表。

如果指定 [height]，窗口设为指定高度（如果有空间的话），
如果省略 [height]，窗口为 10 行高。

如果已有了一个 quickfix 窗口，它将成为当前窗口。不可能

再打开第二个 quickfix 窗口。如果指定 [height]，用来调整已有的窗口的大小。

quickfix-buffer

该窗口含有一个特殊的缓冲区，'buftype' 为 "quickfix"。不要修改此选项！该窗口会设置 w:quickfix_title 变量为生成 quickfix 列表的命令。可用于通过适当地调整 'statusline' 来定制状态行。任何时候 quickfix 命令或函数要是修改了此缓冲区， b:changedtick 变量增一。可通过 getqflist() 和 getloclist() 函数传递 'qfbufnr' 项目来获取此缓冲区号。对位置列表而言，当位置列表删除时也真正删除此缓冲区。

:lop :lopen

:lop[en] [height] 打开一个窗口显示当前窗口的位置列表。只有当前窗口有位置列表的情况才能用。你可以同时打开多于一个的位置列表窗口。除此以外，和 ":copen" 相同。

:ccl :cclose

:ccl[ose] 关闭 quickfix 窗口。

:lcl :lclose

:lcl[ose] 关闭显示当前窗口的位置列表的窗口。

:cw :cwindow

:cw[indow] [height] 当存在可以识别的错误时，打开窗口。如果该窗口已经打开且没有可识别的错误，则将此窗口关闭。

:lw :lwindow

:lw[indow] [height] 和 ":cwindow" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cbo :cbottom

:cbo[ttom] 光标放在 quickfix 窗口的末行并滚动使其可见。可用于异步地加入错误。如果有很多更新，不要太频繁调用此命令，以避免过多的重画。

:lbo :lbottom

:lbo[ttom] 同 ":cbottom"，但使用当前窗口的位置列表。

quickfix 窗口一般会在屏幕的底端。如果有垂直分割，它会在最右边窗口的底端。要使其获得最大宽度：

```
:botright cwindow
```

你可以用 window-moving 命令来移动窗口。
比如，将它移动到顶端：CTRL-W K
'winfixheight' 选项会被设定，也就是说窗口会尽可能的保持它原来的高度，而忽略 'winheight' 和 'equalalways' 选项。你可以手动改变其高度（例如，用鼠标拖动它上方的状态行）。

在 quickfix 窗口中，每一行是一个错误。行号和错误号相等。用 QuickFixLine 高亮来高亮当前项目。可以按你喜欢的改变，例如：

```
:hi QuickFixLine ctermbg=Yellow guibg=Yellow
```

你可以用 ":.cc" 跳转到当前光标处的错误。

按 <Enter> 键或是双击鼠标有同样的效果。在 quickfix 窗口上方会开一窗口显示含有错误的文件。如果已有为该文件打开的窗口，则使用该窗口。如果已打开窗口的缓冲区已被修改，而错误出在另一个文件中，则不能完成向出错处的跳转。你需要首先确定窗口含有一个可被放弃的缓冲区。

CTRL-W_<Enter> CTRL-W_<CR>

你可以用 CTRL-W <Enter> 来打开新窗口并在那里跳转到错误。

当 quickfix 窗口被填充后，将会触发两个自动命令事件。首先，'filetype' 选项会被设为 "qf"，这会触发 FileType 事件（亦见 qf.vim ）。然后 BufReadPost 事件会被触发，缓冲区名为 "quickfix"。这可以用来完成一些在错误列表上的操作。比如：

```
au BufReadPost quickfix  setlocal modifiable
      \ | silent exe 'g/^/s//\=line(".")." "/'
      \ | setlocal nomodifiable
```

这会在每行开始添加行号。注意 ":s" 命令中 "\=" 的使用，它是为了计算表达式。同时激活 BufWinEnter 事件，还是用 "quickfix" 作为缓冲区名。

注意：加入到已有的 quickfix 列表时，不激活自动命令。

备注：在 quickfix 窗口中作更改对出错列表不起作用。'modifiable' 选项关闭以免作出更改。如果你坚持还是要删除或是插入行，文本和错误号的关系就会混乱。如果真的想这么做，你可以将 quickfix 窗口的内容保存到一个文件中然后用 ":cfile" 命令将其解析并指定其作为新的错误列表。

location-list-window

位置列表窗口显示位置列表里的项目。如果打开位置列表窗口，它建立在当前窗口之下，并显示当前窗口的位置列表。位置列表窗口和 quickfix 窗口类似，除了你可以同时打开多于一个位置列表窗口以外。如果你在这个窗口里使用位置列表命令，使用的是其中显示的位置列表。

当你从位置列表窗口里选择文件时，采用下面的步骤以寻找编辑该文件的窗口：

1. 如果该位置列表窗口显示的位置列表所属的窗口存在，用该窗口打开文件。
2. 如果上面的步骤失败并且文件已经在另外的窗口里打开，使用那个窗口。
3. 如果上面的步骤失败，使用已经存在且显示的缓冲区没有置位 'buftype' 的窗口。
4. 如果上面的步骤失败，用新窗口编辑文件。

在所有这些情形下，如果选择窗口的位置列表还没有设置，那么它被设置为位置列表窗口里显示的位置列表。

quickfix-window-ID

getqflist() 和 getloclist() 函数可分别用来获取 quickfix 窗口和位置列表的窗口 ID（如果存在）。例如：

```
echo getqflist({'winid' : 1}).winid
echo getloclist(2, {'winid' : 1}).winid
```

getqflist-examples

getqflist() 和 getloclist() 函数可分别用来获取 quickfix 和位置列表的若干属性。以下提供了使用这些函数的一些例子：

```
" 获取当前 quickfix 列表的标题
:echo getqflist({'title' : 0}).title

" 获取当前 quickfix 列表的标识符
```

```
:let qfid = getqflist({'id' : 0}).id

" 获取栈中第四个 quickfix 列表的标识符
:let qfid = getqflist({'nr' : 4, 'id' : 0}).id

" 检查是否存在指定标识符的 quickfix 列表
:if getqflist({'id' : qfid}).id == qfid

" 获取当前 quickfix 列表在栈中的索引
:let qfnum = getqflist({'nr' : 0}).nr

" 获取指定标识符的 quickfix 列表的项目
:echo getqflist({'id' : qfid, 'items' : 0}).items

" 获取指定标识符的 quickfix 列表的项目数
:echo getqflist({'id' : qfid, 'size' : 0}).size

" 获取栈中第三个 quickfix 列表的上下文
:echo getqflist({'nr' : 3, 'context' : 0}).context

" 获取栈中 quickfix 列表的数目
:echo getqflist({'nr' : '$'}).nr

" 获取当前 quickfix 列表改动的次数
:echo getqflist({'changedtick' : 0}).changedtick

" 获取指定标识符的 quickfix 列表的当前项目
:echo getqflist({'id' : qfid, 'idx' : 0}).idx

" 获取指定标识符的 quickfix 列表的所有属性
:echo getqflist({'id' : qfid, 'all' : 0})

" 解析文本行的列表，并返回 quickfix 列表
:let myList = ["a.java:10:L10", "b.java:20:L20"]
:echo getqflist({'lines' : myList}).items

" 使用定制 'efm' 解析文本并返回 quickfix 列表
:echo getqflist({'lines' : ['a.c#10#Line 10'], 'efm':'%f#%l#%m'}).items

" 获取 quickfix 列表的窗口 id
:echo getqflist({'winid' : 0}).winid

" 获取 quickfix 列表窗口缓冲区号
:echo getqflist({'qfbufnr' : 0}).qfbufnr

" 获取当前位置列表的上下文
:echo getloclist(0, {'context' : 0}).context

" 获取第三个窗口的位置列表的窗口 id
:echo getloclist(3, {'winid' : 0}).winid

" 获取第三个窗口的位置窗口缓冲区号
:echo getloclist(3, {'qfbufnr' : 0}).qfbufnr
```

```
" 获取位置列表窗口 (窗口号: 4) 的文件窗口 id
:echo getloclist(4, {'filewinid' : 0}).filewinid
```

setqflist-examples

setqflist() 和 setloclist() 函数可分别用来设置 quickfix 和位置列表的若干属性。以下提供了使用这些函数的一些例子:

```
" 建立空 quickfix 列表, 指定标题和上下文
:let t = 'Search results'
:let c = {'cmd' : 'grep'}
:call setqflist([], ' ', {'title' : t, 'context' : c})

" 设置当前 quickfix 列表的标题
:call setqflist([], 'a', {'title' : 'Mytitle'})

" 设置指定标识符的列表的当前项目
:call setqflist([], 'a', {'id' : qfid, 'idx' : 10})

" 设置指定标识符的 quickfix 列表的上下文
:call setqflist([], 'a', {'id' : qfid, 'context' : {'val' : 100}})

" 从命令行输出生成新的 quickfix 列表
:call setqflist([], ' ', {'lines' : systemlist('grep -Hn main *.c')})

" 使用定制 efm 来解析文本, 并加入指定的 quickfix 列表
:call setqflist([], 'a', {'id' : qfid,
            \ 'lines' : ["a.c#10#L10", "b.c#20#L20"], 'efm':'%f#%l#%m'})

" 为指定标识符的 quickfix 列表加入项目
:let newItems = [{'filename' : 'a.txt', 'lnum' : 10, 'text' : "Apple"},
                \ {'filename' : 'b.txt', 'lnum' : 20, 'text' : "Orange"}]
:call setqflist([], 'a', {'id' : qfid, 'items' : newItems})

" 清空指定标识符的 quickfix 列表
:call setqflist([], 'r', {'id' : qfid, 'items' : []})

" 清空栈中所有的 quickfix 列表
:call setqflist([], 'f')

" 设置第四个 quickfix 列表的标题
:call setqflist([], 'a', {'nr' : 4, 'title' : 'SomeTitle'})

" 在栈尾创建新的 quickfix 列表
:call setqflist([], ' ', {'nr' : '$',
                    \ 'lines' : systemlist('grep -Hn class *.java')})

" 从命令行输出生成新的位置列表
:call setloclist(0, [], ' ', {'lines' : systemlist('grep -Hn main *.c')})

" 替换第三个窗口位置列表的项目
:call setloclist(3, [], 'r', {'items' : newItems})
```

3. 使用多个错误列表 quickfix-error-lists

到目前为止，我们都假定只有一个错误列表。实际上最近使用的 10 个错误列表都会被记住。当开始一个新的列表时，前面的列表会被自动保存。有两个命令可以用来访问旧的错误列表。它们会将其中一个已经使用的错误列表指定为当前列表。

:colder :col E380

:col[der] [count] 到前一个旧的错误列表。如果指定 [count]，则执行 [count] 次。如果已经到达最旧的列表，则报错。

:lolder :lol

:lol[der] [count] 和 :colder 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:cnewer :cnew E381

:cnew[er] [count] 到下一个新的错误列表。如果指定 [count]，则执行 [count] 次。如果已经到达最新的列表，则报错。

:lnewer :lnew

:lnew[er] [count] 和 :cnewer 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:chistory :chi

:[count]chi[story] 显示错误列表的列表。当前列表以 ">" 标记。结果类似于:

```
error list 1 of 3; 43 errors
> error list 2 of 3; 0 errors
error list 3 of 3; 15 errors
```

给出 [count] 时，第 count 个 quickfix 列表成为当前列表。例如:

```
" 使第 4 个 quickfix 列表成为当前列表
:4chistory
```

:lhistory :lhi

:[count]lhi[story] 显示位置列表的列表，此外类似于 :chistory 。

当增加一个新的错误列表时，它会成为当前列表。

如果使用 ":colder" 后又使用 ":make" 或 ":grep" 来增加一个错误列表，那么原来的一个较新列表会被覆盖。这在使用 ":grep" (见 grep) 时相当有用。如果你想保留最近的错误列表，先要使用 ":cnewer 99" 命令。

getqflist() 和 getloclist() 函数可分别用来获取 quickfix 和位置列表的栈的列表数目，只要把列表号设为特殊值 '$'。例如:

```
echo getqflist({'nr' : '$'}).nr
echo getloclist(3, {'nr' : '$'}).nr
```

要获取栈中当前列表的编号:

```
echo getqflist({'nr' : 0}).nr
```

---

4. 使用 :make :make_makeprg

:mak :make

:mak[e][!] [arguments] 1. 执行所有相关的 QuickFixCmdPre 自动命令。

2. 如果设定 'autowrite'，则写入所有修改过的缓冲区。
3. 根据 'makeef' 的值产生错误文件。如果 'makeef' 值不包含 "##" 且此文件已存在，则该文件会被先删除。
4. 'makeprg' 选项指定的程序（缺省为 "make"）被启动，[arguments] 为其参数。其输出被储存到错误文件名（如是 Unix，也会在屏幕上显示）。
5. 错误文件以 'errorformat' 格式读入。
6. 执行所有相关的 QuickFixCmdPost 自动命令。见下例。
7. 如果没指定 [!]，则跳转到首个错误。
8. 错误文件被删除。
9. 你现在可以用 :cnext，:cprevious 等命令在错误间移动。见上。

本命令不接受注释，所有的 " 将被视为参数的一部分。
如果错误文件的编码和 'encoding' 选项不同，可用 'makeencoding' 选项来指定编码。

:lmak :lmake

:lmak[e][!] [arguments]

和 ":make" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

":make" 命令执行 'makeprg' 选项指定的程序。这会通过将命令传递给 'shell' 指定的外壳完成。基本就像输入下面的命令一样：

":!{makeprg} [arguments] {shellpipe} {errorfile}"。

{makeprg} 是 'makeprg' 选项指定的一个字符串。并非只是 "make"，所有的命令都可被使用。'%' 和 '#' 会像在命令行中一样被展开。你可以用 "%<" 表示不含扩展名的当前文件名，用 "#<" 表示不含扩展名的轮换文件名。例如：

```
:set makeprg=make\ #<.o
```

[arguments] 为 ":make" 之后输入的所有字符。
{shellpipe} 是 'shellpipe' 选项值。
{errorfile} 是 'makeef' 选项值（## 会被一个唯一字符串取代）。

在设定 {makeprg} 时，占位符 "$*" 可用来指代 [arguments]，这样就可以在这些参数之后加上其它字符。这时，$* 被所有参数取代。比如：

```
:set makeprg=latex\ \\\\nonstopmode\ \\\\input\\{$*}
```

或更简单些：

```
:let &mp = 'latex \\nonstopmode \\input\{$*}'
```

"$*" 可被使用多次：

```
:set makeprg=gcc\ -o\ $*\ $*
```

'shellpipe' 在 Amiga 以及 Win32 系统上缺省为 ">"。这就是说编译器的输出会保存在一个文件中而不在屏幕中直接显示。在 Unix 系统上，会用到 "| tee"。输出在被保存在文件中的同时也会显示在屏幕上。取决于所使用的外壳，"|& tee" 或是 "2>&1| tee" 为缺省值，这样 stderr 输出也被包括。

如果 'shellpipe' 为空，{errorfile} 将被忽略。这对那些自身将输出写入文件的编译器很有用（比如，Manx 的 Aztec C）。

用 QuickFixCmdPost 来修正编码

'encoding' 可能不同于编译程序产生的信息的编码，下例说明 Vim 读入错误信息后如何修正之：

```
function QfMakeConv()
   let qflist = getqflist()
   for i in qflist
      let i.text = iconv(i.text, "cp936", "utf-8")
   endfor
   call setqflist(qflist)
endfunction

au QuickfixCmdPost make call QfMakeConv()
```

(Faque Cheng 提供示例)
另一个选项是用 'makeencoding'。

---

5. 使用 :vimgrep 和 :grep grep lid

Vim 有两种寻找模式匹配的方式：内部方式和外部方式。内部 grep 的优点是能适用于所有系统并且使用强大的 Vim 搜索模式。外部 grep 程序可用于 Vim grep 实现不了你需要的功能的场合。

内部方法会慢一些，因为文件要先读入内存。优点是：
- 就像编辑文件一样，自动识别换行符和编码。
- 使用 Vim 搜索模式。可以使用多行模式。
- 如果打开了插件：可以搜索压缩和远程文件。
  gzip netrw

要能够这么做，Vim 需要载入每个文件，就像编辑它们那样。如果文件里没有匹配，相应的缓冲区又会被彻底删除。这里忽略 'hidden' 选项，以避免搜索许多文件时内存或文件描述符的溢出。不过如果使用 :hide 命令修饰符，缓冲区保持载入状态。这使得其后在相同文件里的搜索快得多。

备注 :copen (或对应 :lgrep 的 :lopen) 可用来打开包含链接形式出现的搜索结果的缓冲区。可用 :silent 命令来抑制缺省的全屏 grep 输出结果。":grep!" 形式的 :grep 命令不自动跳转到首个匹配。这些命令可以组合成一个新的命令：

```
command! -nargs=+ NewGrep execute 'silent grep! <args>' | copen 42
```

5.1 使用 Vim 的内部 grep

:vim :vimgrep E682 E683

:vim[grep][!] /{pattern}/[g][j] {file} ...
在文件 {file} ... 里搜索模式 {pattern}，并用匹配结果设置错误列表。忽略匹配 'wildignore' 的文件；最后搜索匹配 'suffixes' 的文件。
如果没有 'g' 标志位，每行只加一次。
如果有 'g'，每个匹配都被加入。

{pattern} 是 Vim 的搜索模式。除了用 / 之外，任何非 ID 字符 (见 |'isident') 都可以用来包围该模式，只要在 {pattern} 里不出现就行了。

适用 'ignorecase'。要否决该设置，在模式里用 /\c 来忽略大小写或 /\C 来匹配大小写。
不使用 'smartcase'。
如果 {pattern} 为空（例如，指定 //），使用前次搜索模式。 last-pattern

:{count}vim[grep] ...
如果命令前面带数字，它定义要寻找的最大匹配数目。这样，":1vimgrep pattern file" 只会找到第一个。这可以用于只想检查是否有匹配而在找到后能快速退出的场合。

如果没有 'j' 标志位，Vim 跳到第一个匹配上。
如果有 'j'，只更新 quickfix 列表。
如果有 [!]，放弃当前缓冲区的任何修改。

大约每过一秒，显示正在搜索的文件名，让你了解当前的进展情况。
例如:

```
:vimgrep /an error/ *.c
:vimgrep /\<FileName\>/ *.h include/*
:vimgrep /myfunc/ **/*.c
```

"**" 的使用可见 starstar-wildcard 。

:vim[grep][!] {pattern} {file} ...
同上，但不用非 ID 的字符包围模式，而只用空白分隔。该模式必须以 ID 字符开头。
例如:

```
:vimgrep Error *.c
```

:lv :lvimgrep
:lv[imgrep][!] /{pattern}/[g][j] {file} ...
:lv[imgrep][!] {pattern} {file} ...
和 ":vimgrep" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:vimgrepa :vimgrepadd
:vimgrepa[dd][!] /{pattern}/[g][j] {file} ...
:vimgrepa[dd][!] {pattern} {file} ...
和 ":vimgrep" 类似，但不是建立新的错误列表，而是附加到现有的列表之后。

:lvimgrepa :lvimgrepadd
:lvimgrepa[dd][!] /{pattern}/[g][j] {file} ...
:lvimgrepa[dd][!] {pattern} {file} ...
和 ":vimgrepadd" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

5.2 外部 grep

Vim 可以和 "grep" 或类似 grep 的程序（比如 GNU id-utils）交互，其方式和与编译器交互方式类似。

[Unix 小常识：Unix 上 "grep" 命令的名字来源于 ":g/re/p"，其中 "re" 是正则表达式（Regular Expression）的意思。]

:gr[ep][!] [arguments]

:gr :grep

和 ":make" 类似，但以 'grepprg' 替代 'makeprg'，以 'grepformat' 替代 'errorformat'。
如果 'grepprg' 是 "internal"，这等价于 :vimgrep 。
注意 这时模式必须用分隔字符包围。
如果错误文件的编码和 'encoding' 选项不同，可用 'makeencoding' 选项来指定编码。

:lgr[ep][!] [arguments]

:lgr :lgrep

和 ":grep" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

:grepa[dd][!] [arguments]

:grepa :grepadd

和 ":grep" 类似，但不另建一个新的错误列表，只是将匹配附加到当前列表中。
比如:

```
:call setqflist([])
:bufdo grepadd! something %
```

第一个命令新建一个空的错误列表。第二个命令对每个列出的缓冲区执行 "grepadd"。注意 ! 的使用是避免 ":grepadd" 跳转到首个错误， :bufdo 不允许这样。
一个使用参数列表并在没有匹配文件时不报错的例子:

```
:silent argdo try
  \ | grepadd! something %
  \ | catch /E480:/
  \ | endtry"
```

如果错误文件的编码和 'encoding' 选项不同，可用 'makeencoding' 选项来指定编码。

:lgrepa[dd][!] [arguments]

:lgrepa :lgrepadd

和 ":grepadd" 类似，但使用当前窗口的位置列表而不是 quickfix 列表。

5.3 设定外部 grep

如果你已安装了一个标准的 "grep" 程序，:grep 命令会以缺省方式运行。格式和标准命令很类似:

```
:grep foo *.c
```

会对每个 .c 文件搜索字符串 "foo"。:grep 的参数直接传递给 "grep" 程序，这样你就可以使用 "grep" 支持的任何选项。

缺省状态下，:grep 会以 -n 参数启动 grep (显示文件和行号)。你可以通过设定 'grepprg' 对此改变。在以下这些情况下你需要设定 'grepprg':

a) 你在使用一个不叫 "grep" 的程序。
b) 你需要以绝对路径启动 grep。

c)　　你想自动传递某些参数（比如，忽略大小写）。

"grep" 执行后，Vim 会用 'grepformat' 选项解析其结果。该选项和 'errorformat' 的工作方式相同。如果你的 grep 输出不是标准格式或是你使用具有特定格式的程序，你可以通过设定 'errorformat' 来改变缺省值。

解析完结果，Vim 会载入第一个含有匹配的文件并跳转到相应行，与在 quickfix 模式中跳转到一个编译错误的方式相同。你可以使用 :cnext ， :clist 等命令来查看其它的匹配。

5.4 配合 id-utils 使用 :grep

你可以设定 :grep 来使用 GNU id-utils:

```
:set grepprg=lid\ -Rgrep\ -s
:set grepformat=%f:%l:%m
```

然后:

```
:grep (regexp)
```

结果就像你所期望的那样。
(只要你记得首先 mkid :)

5.5 用 :vimgrep 或 :grep 浏览源代码

使用 Vim 保存的错误列表，你可以查找某些函数或是其调用的其它函数。举个例子，如果你要对 read_file() 函数添加一个参数，可以输入下面的命令:

```
:vimgrep /\<read_file\>/ *.c
```

你可以用 ":cn" 在一系列匹配中跳转，然后加上所需要的参数。在某个地方，你需要从上一级函数 msg() 得到这个新的参数，因此也要对它做出修改:

```
:vimgrep /\<msg\>/ *.c
```

在修改 msg() 函数的同时，你又找到另一个需要从上级函数中获得该参数的函数。你可以用 ":vimgrep" 命令找到这些函数。当完成一个函数后，你可以用:

```
:colder
```

来跳转到上一个函数。

这就像浏览一棵树一样，":vimgrep" 每深入一层，就创建新的分支。":colder" 跳转到上一层。你可以混合使用 ":vimgrep" 和 ":colder"，以树型方式来浏览所有的位置。如果经常这么做，你不需写一个 "todo list" 就能找到所有的位置。

---

6. 选择编译器　　compiler-select

:comp　:compiler　E666

:comp[iler][!] {name}　　为 {name} 编译器设定相关选项。
当没有 "!" 时设定只作用于当前缓冲区。如果使用

了 "!" 则修改全局选项的值。
如果你对于 "file.foo" 使用 ":compiler foo" 然后对另外一个缓冲区使用了 ":compiler! bar"，Vim 会继续对 "file.foo" 文件使用 foo。
{仅当编译时指定 +eval 特性才有效}

在 "compiler" 目录中的 Vim 插件被用来为选择的编译器设定各种选项。对 :compiler 设定局部选项，对 :compiler! 设定全局选项。

current_compiler
为了支持老版本的 Vim，这些插件总使用 "current_compiler" 而不是 "b:current_compiler"。这个命令实际上做的是：

- 删除 "current_compiler" 和 "b:current_compiler" 变量。
- 定义 "CompilerSet" 用户命令。对 "!" 使用 ":set"，没有 "!" 使用 ":setlocal"。
- 执行 ":runtime! compiler/{name}.vim"。插件应该使用 "CompilerSet" 来做各种设定并将编译器的名字赋给 "current_compiler"。
- 删除 "CompilerSet" 用户命令。
- 将 "b:current_compiler" 设为 "current_compiler" 的值。
- 没有 "!" 的话 "current_compiler" 的旧值会被恢复。

如要编写编译器插件，请参考 write-compiler-plugin 。

GCC quickfix-gcc compiler-gcc

GCC 编译器有一个变量可以设置：

g:compiler_gcc_ignore_unmatched_lines
忽略不匹配 GCC 定义的任何模式的行。如果 make 运行的命令的输出产生误判 (false positive)。

MANX AZTEC C quickfix-manx compiler-manx

要使 Vim 和 Amiga 上的 Manx Aztec C 编译器工作，需要以下步骤：
- 设定 CCEDIT 环境变量：
```
mset "CCEDIT=vim -q"
```
- 用 -qf 选项编译。如果编译器发现任何错误，Vim 被启动，光标定位在首个错误处。错误信息会在最后一行显示。你可以用上面提到的命令跳转到其它的错误，对其修定并存盘。
- 如果正常退出 Vim，编译器会再次编译相同文件。如果用 :cq 命令退出，编译器就会终止。如果不能修定错误或是需要先编译其它的文件，你可以这样做。

在 Amiga 系统上，QuickFix 模式会有某些限制。编译器只能在错误文件中写入前 25 个错误 (Manx 的文档没有说明怎样才能写入更多)。如果想查找其它的错误，你需要首先修定一些错误并退出编译器。重新编译后，剩下的错误 (最多 25 个) 就会找到。

如果 Vim 是在编译器中启动的，:sh 和一些 :! 命令不能工作，因为 Vim 是和编译器在同一个进程中运行而 stdin (标准输入) 此时不在交互状态。

PERL quickfix-perl compiler-perl

Perl 编译器插件并不实际编译，而是调用 Perl 的内部语法检查功能并分析结果来得到可能的错误，以便你能在 quick-fix 模式中修正。

不管检查的文件里是否有 "no warnings" 或 "$^W = 0"，警告还会产生。要关闭之，设置 g:perl_compiler_force_warnings 为零值。例如:

```
let g:perl_compiler_force_warnings = 0
```

PYUNIT 编 译 器 compiler-pyunit

这其实不是一个编译器，而是一个为 Python 语言而设的单元测试工具。从版本 2.0 起，它就在标准的 Python 发布中。对老版本，你可以在这里获得: http://pyunit.sourceforge.net。

当你运行测试时，Vim 会解析错误并进入到 quick-fix 模式。

不幸的是，没有一个运行测试的标准方法。
一般都会用使用 alltest.py。我们就知道这么多。
有关 'makeprg' 选项就可以这样设定:

```
setlocal makeprg=./alltests.py " 执行全套测试
setlocal makeprg=python\ %:S   " 执行单个测试用例
```

另外，参考 http://vim.sourceforge.net/tip_view.php?tip_id=280。

TEX 编 译 器 compiler-tex

Vim 发布版本中的 Tex 编译器 ($VIMRUNTIME/compiler/tex.vim) 可以处理各种风格的 TeX 格式。如果编译器在当前目录找到文件 "Makefile" 或 "makefile"，它将假定你希望用 make 来处理 *TeX 文件，而 makefile 已经做好了各种设定。这种情况下编译器会将 'errorformat' 设定为 *TeX 的输出格式并将 'makeprg' 保留原样。如果 "Makefile" 和 "makefile" 都找不到，编译器就不会使用 make。你可以通过定义 b:tex_ignore_makefile 或 g:tex_ignore_makefile 来强制编译器忽略 makefile (Vim 只检查这两个变量存在与否)。

如果编译器选择了不使用 make，它必须选择一个正确的程序来处理你的输入。如果变量 b:tex_flavor 或 g:tex_flavor (依此优先顺序) 存在，它将为 :make (即为所要运行的命令) 定义 TeX 风格；如果这两个变量都不存在，将会使用缺省的 "latex"。比如，当编辑以 AMS-TeX 格式编写的 chapter2.tex (被 \input 进入 mypaper.tex 中) 时:

```
:let b:tex_flavor = 'amstex'
:compiler tex
[editing...]
:make mypaper
```

注意，你必须将所处理文件的文件名作为参数 (为了在编辑被 \input 或 \include 的文件时能处理正确的文件；我们期待任何可移植的能不用参数就可以替代 % 的解决方案)。这与 make 指定目标而非源的语法是相悖的，但你可以指定不含扩展名 ".tex" 的文件名 filename，来表示 "make filename.dvi 或 filename.pdf 或 filename.其它 目标 扩展名"，具体名字视编译器而定。

备注: tex 命令行格式对 MikTeX (Srinath Avadhanula 报告) 和 teTeX (Artem

Chuprina 测试）都适用。要使 errorformat-LaTeX 对不同外壳不同 OS 都适用太过复杂，因此不允许使用其它的 TeX 的选项。如果你的 TeX 不支持 "-interaction=nonstopmode"，请在命令行中用其它的方式表示 \nonstopmode。

---

7. 错误格式 error-file-format

errorformat E372 E373 E374 E375 E376 E377 E378

'errorformat' 选项指定可识别出的一系列格式。首个匹配的格式会被使用。你可以为编译器产生的不同信息添加不同格式，或是为多个编译器添加项目。参见 efm-entries 。

'errorformat' 里的每一项是一个类似 scanf 的格式字符串。首先，你需要了解 scanf 是如何工作的。请查看你的 C 编译器文档。下面你可以看到 Vim 能理解 % 的条目。其它的无效。

'errorformat' 中的特殊字符是逗号和反斜杠，参见 efm-entries 。注意按本义出现的字符 "%" 用 "%%" 来匹配，而不用反斜杠来转义。
记住 :make 和 :grep 输出中的所有 NUL 字符会被 SOH (0x01) 替换。

备注：缺省忽略大小写差别。如果要匹配大小写，在模式中加入 "\C"，参见 /\C 。

Vim 会读入任何长度的行，但只使用前 4095 个字符，忽略其余。项目只能有 1023 个字节长。

基 本 条 目

%f	文件名（期待字符串（译者注：以下 "期待" 二字从略））
%o	模块名（字符串）
%l	行号（数字）
%c	列号（代表错误中字符列的数字，（<tab> 为一个字符列））
%v	虚拟列号（代表错误中屏幕列的数字，（<tab> 等于 8 个屏幕列））
%t	错误类型（单个字符）
%n	错误号（数字）
%m	错误信息（字符串）
%r	匹配 "余下的" 的单行文件信息 %O/P/Q
%p	指针行（'-'、'.'、' ' 或是制表符的序列，用其长度指定列号）
%	
%*{conv}	任何 scanf 免于赋值的转换
%%	单个 '%' 字符
%s	搜索文本（字符串）

"%f" 转换可能依赖于当前 'isfname' 的设定。"~/" 被扩展为主目录，环境变量也被扩展。

"%f" 和 "%m" 转换需要识别字符串结尾。通常这是通过匹配后面的字符和项目完成的。如果后面没有别的项目，该行的其余部分都被匹配。如果 "%f" 后面跟着 '%' 或者反斜杠，它会寻找 'isfname' 字符的序列。

在 MS-Windows 上，"C:" 会加到 "%f" 开头中，即使使用 "%f:"。这就是说单字母的文件名将不被识别。

"%p" 转换一般后跟 "^"。这是为了配合编译器产生的这种输出：

```
          ^
```

或

```
   ---------^
```

以指定错误所在列。这被用在一个多行错误信息中。 errorformat-javac 有一个有用的例子。

"%s" 转换指定用于定位错误行的搜索文本。该文本用作按本义出现的字符串。文本会加入 "^" 和 "%" 使要定位的错误行完全匹配搜索文本。文本也加入 "\V" 匹配原作为前缀，使得它 "very nomagic"。"%s" 转换可用于定位错误输出里没有行号的行。"grep" 外壳命令的输出就是如此。
如果此模式存在，不使用行号。

"%o" 转换指定 quickfix 项目的模块名。如果存在，用于 quickfix 错误窗口，而不是缺省的文件名。模块名只用于显示目的，总是用文件名来跳转到相应文件。

改 变 目 录

下面的大写转换指定特殊的格式字符串的类型。至多只能用其中的一个作为以逗号分隔的单个格式的前缀。
一些编译器产生含有目录名的信息，这些目录名需要加在 %f 读入文件名的前面（比如 GNU make）。下面的代码可用于扫描这些目录名；它们将被保存在内部的目录栈中。

E379

%D	"进入目录" 格式字符串；期待后跟 %f 以找到目录名
%X	"退出目录" 格式字符串；期待跟在 %f 之后

当定义 "进入目录" 或是 "退出目录" 格式时，"%D" 或是 "%X" 必须在子字符串的开头。Vim 会跟踪目录的变化并把当前目录添加在带有相对路径的错误文件名前。有关细节，参见 quickfix-directory-stack 。

多 行 信 息 errorformat-multi-line

Vim 可以读入程序输出的多行信息，也就是超过一行的错误字串。有关前缀有：

%E	多行错误信息开始
%W	多行警告信息开始
%I	多行一般信息开始
%A	多行信息开始（未指定类型）
%>	下行再次从当前模式开始寻找 efm-%>
%C	多行信息继续
%Z	多行信息结束

这些可同 '+' 和 '-' 一起使用，参见下面的 efm-ignore 。

模式里的 "\n" 不能用于匹配多行消息。

例子：你的编译器以下面格式输出（开头的行号并不是实际输出）：

```
1  Error 275
2  line 42
3  column 3
4  ' ' expected after '--'
```

正确的错误格式串应如下：

```
:set efm=%EError\ %n,%Cline\ %l,%Ccolumn\ %c,%Z%m
```

这样，:clist 产生的出错信息为：

```
1:42 col 3 error 275:  ' ' expected after '--'
```

另一个例子：Python 解释器产生下面的错误信息（行号不是实际输出）：

```
 1  ======================================================================
 2  FAIL: testGetTypeIdCachesResult (dbfacadeTest.DjsDBFacadeTest)
 3  ----------------------------------------------------------------------
 4  Traceback (most recent call last):
 5    File "unittests/dbfacadeTest.py", line 89, in testFoo
 6      self.assertEquals(34, dtid)
 7    File "/usr/lib/python2.2/unittest.py", line 286, in
 8   failUnlessEqual
 9      raise self.failureException, \
10  AssertionError: 34 != 33
11
12  ----------------------------------------------------------------------
13  Ran 27 tests in 0.063s
```

假使你想让 :clist 输出这样的相关信息：

```
5 unittests/dbfacadeTest.py:89:  AssertionError: 34 != 33
```

错误格式字符串可这样定义：

```
:set efm=%C\ %.%#,%A\ \ File\ \"%f\"\\,\ line\ %l%.%#,%Z%[%^\ ]%\\@=%m
```

注意 %C 字符串在 %A 前指定：因为 '%.%#' 表达式（表示正则表达式 '.*'）匹配每一个以空格开头后接任何字符的行，这样就包括了本来会单独产生错误信息的第 7 行。错误格式字符串总是一个模式接一个模式的分析，直到找到首个匹配。

efm-%>

%> 项目可用来避免在 'errorformat' 较早的位置出现的模式。尤其是那些会匹配所有内容的模式。例如，如果错误看起来像：

```
Error in line 123 of foo.c:
unknown variable "i"
```

可以这样找到它：

```
:set efm=xxx,%E%>Error in line %l of %f:,%Z%m
```

其中 "xxx" 包含也可以匹配第二行的模式。

重要：没有关于上次在 errorformat 哪个部分匹配的记忆；错误文件的每行都重新开始遍历错误格式行。例如，如果：

```
setlocal efm=aa,bb,cc,dd,ee
```

其中 aa、bb 等是错误格式字符串。错误文件的每行都会依次匹配 aa 模式，然后 bb，然后 cc 等等。只因为 cc 匹配了上一个错误行并 不 意味着当前行会优先尝试 dd，即使 cc 和 dd 都是多行错误格式字符串也不例外。

单 独 文 件 名 errorformat-separate-filename

如果编译器输出只给定一次文件名而其后有多个信息指向这个文件，下面的前缀就很有

用：

%O	单行文件信息：过读（overread，见下）匹配部分
%P	单行文件信息：将文件 %f 入栈
%Q	单行文件信息：将最后一个文件名出栈

例子：编译器产生以下出错日志（不含开头的行号）：

```
 1  [a1.tt]
 2  (1,17)  error: ';' missing
 3  (21,2)  warning: variable 'z' not defined
 4  (67,3)  error: end of file found before string ended
 5
 6  [a2.tt]
 7
 8  [a3.tt]
 9  NEW compiler v1.1
10  (2,2)   warning: variable 'x' not defined
11  (67,3)  warning: 's' already defined
```

这个文件对每个在 [...] 中包括的文件列出了多个信息。我们可以这样设定格式：

```
:set efm=%+P[%f],(%l\\,%c)%*[\ ]%t%*[^:]:\ %m,%-Q
```

:clist 会正确的显示它们的文件名：

```
2 a1.tt:1 col 17 error: ';' missing
3 a1.tt:21 col 2 warning: variable 'z' not defined
4 a1.tt:67 col 3 error: end of file found before string ended
8 a3.tt:2 col 2 warning: variable 'x' not defined
9 a3.tt:67 col 3 warning: 's' already defined
```

和其它匹配整行的前缀不同，%P、%Q 和 %O 可以匹配同一行中的多个模式。这样就可以对像下面这样嵌套的文件解析：

```
{"file1" {"file2" error1} error2 {"file3" error3 {"file4" error4 error5}}}
```

%O 解析不包含需要出入栈的文件名的信息的字符串。更深入的例子，参见 errorformat-LaTeX 。

(译者注：overread 有读过度的意思，这里的意思是说对读到的文件名不用太过认真)

忽略和使用整个信息 efm-ignore

'+' 和 '-' 可与上面的大写代码结合；这种情况下，它们要在字母的前面，比如 '%+A' 或 '%-G'。

%-	不在任何输出中包含相匹配的（多）行
%+	在 %m 错误串中包含整个匹配行

%G 只在和 '+' 和 '-' 结合使用时才有用。它解析编译器版本等可被忽略的字符串。

%-G	忽略此信息
%+G	通用信息

模式匹配

scanf() 风格的 "%*[]" 记法仍被支持，以和老版本兼容。但你可以在格式化字符串中指定 Vim 支持的（几乎）任何正则表达式。因为元字符本身可能为匹配字符串或文件名的

一部分（因此内部需要被转义），要起到元字符的作用，它们的开头要加上一个 '%'：

%\	单个 '\' 字符。注意 在 ":set errorformat=" 定义中需要用 "%\\" 来转义。
%.	单个 '.' 字符。
%#	单个 '*'（注意！）字符。
%^	单个 '^' 字符。注意 这并不有用，模式总是匹配行首
%$	单个 '$' 字符。注意 这并不有用，模式总是匹配行尾
%[	单个 '[' 字符（指定一个 [] 字符范围）。
%~	单个 '~' 字符。

当在表达式中使用字符类（参见 /\i ）时，含有 "\+" 数量符的串可以使用 scanf() 的 "%*" 形式代替。比如："%\\d%\\+" ("\d\+" 或 "任意数字") 和 "%*\\d" 相同。

重要 备注：子匹配 \(...\) 不能用于格式规格中，因为它被留作内部变换使用。

'errorformat' 中的多个选项 efm-entries

为能识别从不同编译器产生的输出，可在 'errorformat' 中指定多个格式模式，用逗号将它们分隔（备注：逗号后的空格被忽略）。使用第一个完全匹配的模式。如果没有匹配，则使用最近一次匹配行的匹配部分，然而文件名被删除并且错误信息被设定为整个信息。如果有模式可以匹配多个编译器产生的输出（但不是正确的方式），将它放到一个更严格的模式的后面。

要在模式中包括逗号，在它前面加一个反斜杠（如果是在 ":set" 命令中，要加两个）。要包括反斜杠本身，则使用两个反斜杠（在 ":set" 命令中使用四个）。":set" 命令中，在空格前也要加一个反斜杠。

有效匹配 quickfix-valid

如果某行不能完全匹配 'errorformat' 中的项目，则此行被放入到错误信息中并标记为 "无效"。这些行会被 ":cn" 和 ":cp" 命令忽略（除非没有任何有效行存在）。你可以用 ":cl!" 来显示所有的出错信息。

如果出错格式不含文件名，Vim 不能跳转到正确的文件。你需要手动跳转。

例子

Amiga 上的 Aztec 编译器产生的错误文件格式是：

```
filename>linenumber:columnnumber:errortype:errornumber:errormessage
```

filename	错误所在文件的文件名
linenumber	错误所在行号
columnnumber	错误所在列号
errortype	错误类型，一般为单个 'E' 或 'W'
errornumber	错误号（用于查找手册）
errormessage	错误描述

可以用这个 'errorformat' 项目来匹配：

```
%f>%l:%c:%t:%n:%m"
```

一些产生单行错误输出的 C 编译器的例子：

```
%f:%l:\ %t%*[^0123456789]%n:\ %m        Manx/Aztec C 出错信息
                                        (scanf() 不能理解 [0-9])
%f\ %l\ %t%*[^0-9]%n:\ %m               SAS C
\"%f\"\\,%*[^0-9]%l:\ %m                通用 C 编译器
%f:%l:\ %m                              GCC
%f:%l:\ %m,%Dgmake[%*\\d]:\ Entering\ directory\ ‘%f’,
%Dgmake[%*\\d]:\ Leaving\ directory\ ‘%f’
                                        与 gmake 配合使用的 GCC（只有一行！）
%f(%l)\ :\ %*[^:]:\ %m                  老式 SCO C 编译器（OS5 之前）
%f(%l)\ :\ %t%*[^0-9]%n:\ %m            同上，增加错误类型和错误号
%f:%l:\ %m,In\ file\ included\ from\ %f:%l:,\^I\^Ifrom\ %f:%l%m
                                        GCC，一些附加信息
```

对多行信息处理更深入的例子，参见 errorformat-Jikes 和 errorformat-LaTeX 。

注意在空格和双引号前的反斜杠。这对 :set 命令是必需的。在逗号前有两个反斜杠，一个是为 :set 命令需要，一个是为避免将逗号视为出错格式的分隔符。

过 滤 信 息

如果编译器产生的错误信息和格式串不适合，你可以写一个程序将出错信息转化为适合格式。通过设定 'makeprg' 选项，你可以用 ":make" 命令运行它。比如：

```
:set mp=make\ \\\|&\ error_filter
```

管道符前的那些反斜杠是为了避免将其视作命令分隔符。每个空格前的反斜杠是 :set 命令所必需的。

---

8. 目录栈 quickfix-directory-stack

Quickfix 根据 make 输出，维护一个保存所有使用过的目录的栈。对 GNU-make 来讲，这非常简单，因为它总是输出绝对路径，不管是通过 makefile 中的一个 'cd' 命令还是以 "-C dir"（在读入 makefile 之前进入到指定目录）参数运行。'-w' 选项可用来强制 GNU-make 在处理前后输出当前目录。

如果你使用的不是 GNU-make，那目录栈的维护就会复杂得多。比如 AIX-make 从不输出有关当前目录的任何信息。这样你就需要在 makefile 上多花些功夫。Lesstif 的 makefile 中有一个命令可以显示 "Making {target} in {dir}"。这里的问题是它并不在离开目录时输出信息，而且它不输出绝对路径。

为解决相对路径和缺少 "退出目录" 信息问题，Vim 使用下面的算法：

1) 检查给定目录是否为当前目录的子目录。如是，则将其存为当前目录。
2) 如果不是，则检查它是否为上级目录之一的某个子目录。
3) 如果目录仍没找到，则假定为它为 Vim 当前目录的子目录。

此外，每一个文件都被检查，看其是否存在于识别出的目录中。如果不存在，则在目录栈（不是 子目录树！）的其它目录中查找。如果仍没找到，则假定它在 Vim 当前目录中。

此算法仍有限制。本例假定 make 只在进入到一个目录时输出类似 "Making all in dir" 的信息。

1) 假定你有以下目录和文件：

```
./dir1
./dir1/file1.c
./file1.c
```

如果 make 在处理当前目录前处理 "./dir1" 目录，而在文件 "./file1.c" 中存在一处错误，Vim 会载入文件 "./dir1/file.c"。

这只能靠 "退出目录" 信息解决。

2) 假定你有以下目录和文件:

```
./dir1
./dir1/dir2
./dir2
```

你得到如下结果:

```
Make 输出                    Vim 理解的目录
-----------------------      ----------------------------
Making all in dir1           ./dir1
Making all in dir2           ./dir1/dir2
Making all in dir2           ./dir1/dir2
```

这可以通过在 "进入目录" 信息中输出绝对路径或是输出 "退出目录" 信息的方法解决。

为避免这些问题，要确保输出绝对路径和 "退出目录" 信息。

Makefiles 例子:

```
Unix:
    libs:
            for dn in $(LIBDIRS); do                              \
                (cd $$dn; echo "Entering dir '$$(pwd)'"; make); \
                echo "Leaving dir";                               \
            done
```

并将

```
    %DEntering\ dir\ '%f',%XLeaving\ dir
```

加到 'errorformat' 选项中以处理上面的输出。

注意 Vim 并不检查 "退出目录" 信息中的目录是否为当前目录。因此只要使用 "Leaving dir" 这样的信息就可以了。

---

9. 特定错误格式 errorformats

errorformat-Jikes

Jikes(TM) 是 IBM Research 发布的一个源码到字节码的 Java 编译器。它提供简单的多行错误信息。

下面是一个与其所产生的信息相匹配的 'errorformat' 格式字符串。这几行可加到用户的 vimrc 以覆盖 Vim 识别的缺省模式。或者，你可以参考 :set+= 将这个模式安装到缺省格式之中。

```
:set efm=%A%f:%l:%c:%*\\d:%*\\d:,
      \%C%*\\s%trror:%m,
      \%+C%*[^:]%trror:%m,
      \%C%*\\s%tarning:%m,
      \%C%m
```

以 "+E" 选项启动时，Jikes(TM) 产生一个单行信息。对此，可以这样匹配：

```
:setl efm=%f:%l:%v:%*\\d:%*\\d:%*\\s%m
```

errorformat-javac

'errorformat' 可以很好的和 javac 工作，javac 的输出中用 "^" 表示错误所在列：

```
:setl efm=%A%f:%l:\ %m,%-Z%p^,%-C%.%#
```

或者：

```
:setl efm=%A%f:%l:\ %m,%+Z%p^,%+C%.%#,%-G%.%#
```

这里还有一个用于 Unix 的替代方法，由 Michael F. Lamb 提供。首先过滤错误：

```
:setl errorformat=%Z%f:%l:\ %m,%A%p^,%-G%*[^sl]%.%#
:setl makeprg=javac\ %:S\ 2>&1\ \\\|\ vim-javac-filter
```

然后，你需要在路径的某处（如 ~/bin）加入 "vim-javac-filter" 并使之可执行。它包含如下内容：

```
#!/bin/sed -f
/\^$/s/\t/\ /g;/:[0-9]\+:/{h;d};/^[ \t]*\^/G;
```

用中文来说，该 sed 脚本：
- 把单个制表符换成单个空格；并
- 把包含文件名、行号及错误信息的行移到指针行下方。这样，两者之间无用的错误文本就不会打破 vim "多行消息" 的概念，我们也不用硬要把它当作 "多行信息的继续"。

errorformat-ant

ant (http://jakarta.apache.org/) 输出的每个行首都为 [javac]，这样我们可将上面的格式修改如下：

```
:set efm=%A\ %#[javac]\ %f:%l:\ %m,%-Z\ %#[javac]\ %p^,%-C%.%#
```

'errorformat' 还可以处理与 javac 或 jikes 之一配合使用的 ant。如使用 jikes，你需要告诉 ant 使用 jikes 的 +E 选项以使 jikes 产生单行错误信息。这就是下面（一个 build.xml 文件的）第二行所做的：

```
<property name = "build.compiler"       value = "jikes"/>
<property name = "build.compiler.emacs" value = "true"/>
```

下面的 'errorformat' 能同时处理与 javac 和 jikes 配合使用的 ant：

```
:set efm=\ %#[javac]\ %#%f:%l:%c:%*\\d:%*\\d:\ %t%[%^:]%#:%m,
        \%A\ %#[javac]\ %f:%l:\ %m,%-Z\ %#[javac]\ %p^,%-C%.%#
```

errorformat-jade

解析 jade（见 http://www.jclark.com/）错误十分简单：

```
:set efm=jade:%f:%l:%c:%t:%m
```

errorformat-LaTeX

下面是 'errorformat' 处理 (La)TeX 产生的多行错误输出的例子。":clist" 和 ":cc" 等命令将多行错误信息显示在一行，开头的空格被删除。
将 LaTeX 的错误格式运用于其它产生多行错误信息的编译器就很简单了。

这些命令可被加到 vimrc 或是其它 Vim 脚本 －－ 比如一个仅在编辑 LaTeX 源文件时才载入的含有 LaTeX 相关内容的脚本。
确保将例子中的所有行都拷贝 (按照给定顺序)，然后可删除注释行。行首的 '\' 可以参考 line-continuation 。

首先设定 'makeprg' 使 LaTeX 可以报告多个错误；不在首个错误发生时就停止：

```
:set makeprg=latex\ \\\\nonstopmode\ \\\\input\\{$*}
```

多行错误信息开始：

```
:set efm=%E!\ LaTeX\ %trror:\ %m,
      \%E!\ %m,
```

多行警告信息开始；开始的两个包含行号。若干正则表达式的含义：

- "%.%#" (".*") 匹配一个字符串 (可为空)
- "%*\\d" ("\d\+") 匹配一个数字

```
      \%+WLaTeX\ %.%#Warning:\ %.%#line\ %l%.%#,
      \%+W%.%#\ at\ lines\ %l--%*\\d,
      \%WLaTeX\ %.%#Warning:\ %m,
```

错误/警告信息可能的续行；其中的首行也含有行号：

```
      \%Cl.%l\ %m,
      \%+C\ \ %m.,
      \%+C%.%#-%.%#,
      \%+C%.%#[]%.%#,
      \%+C[]%.%#,
      \%+C%.%#%[{}\\]%.%#,
      \%+C<%.%#>%.%#,
      \%C\ \ %m,
```

匹配下面模式的行不含有任何重要的信息；将其忽略：

```
      \%-GSee\ the\ LaTeX%m,
      \%-GType\ \ H\ <return>%m,
      \%-G\ ...%.%#,
      \%-G%.%#\ (C)\ %.%#,
      \%-G(see\ the\ transcript%.%#),
```

删除所有空白行：

```
      \%-G\\s%#,
```

LaTeX 输出并不在每一行中都指定出错的文件名；而只在括号中指定一次。
下面的模式试图匹配这些文件名并将它们保存到内部栈中。这些模式可能扫描相同行 (一个接一个的)，结尾的 "%r" 表示行中 "余下" 的部分，这些部分在下轮被重新解析，直到遇到行尾为止。

过读 '(...)' 包含的文件名；不将其入栈，因为此文件不含有任何错误：

```
      \%+O(%f)%r,
```

将 '(' 后的文件名入栈：

```
      \%+P(%f%r,
      \%+P\ %\\=(%f%r,
      \%+P%*[^()](%f%r,
      \%+P[%\\d%[^()]%#(%f%r,
```

当扫描到 ')'，将最后一个文件名出栈：

```
      \%+Q)%r,
      \%+Q%*[^()])%r,
      \%+Q[%\\d%*[^()])%r
```

注意 某些情况下，LaTeX 输出中的文件名不能正确解析。解析会被不成对的括号打乱。上面的例子只试图捕获最普通的情况。你可以根据你的需要而修改有关设定，比如，所有烦人的 "Overfull ..." 警告可不被视为出错。
除了过滤 LaTeX 的输出信息外你还可以直接读入 [La]TeX 编译器生成的 *.log 文件。这包括其它的一些由错误引起的有用信息。但要解析这么复杂的文件，你需要使用外部过滤器。参考上面关于 Vim 过滤器的描述。

errorformat-Perl

在 $VIMRUNTIME/tools 中，你可以找到 efm_perl.pl 脚本。它将 Perl 出错信息过滤为 quickfix 模式可以理解的格式。使用方法可以查看该文件的开头部分。(此脚本已过时，见 compiler-perl 。)

vim:tw=78:ts=8:noet:ft=help:norl:

ft_ada.txt　　For Vim version 8.2.　最近更新：2019年12月

ADA 文 件 类 型 参 考 手 册

译者：Willis
http://vimcdoc.sf.net

ADA　　ada.vim

1. 语法高亮　　ft-ada-syntax
2. 文件类型插件　　ft-ada-plugin
3. 全能补全　　ft-ada-omni
    3.1 使用 "gnat xref" 的全能补全　　gnat-xref
    3.2 使用 "ctags" 的全能补全　　ada-ctags
4. 编译器支持　　ada-compiler
    4.1 GNAT　　compiler-gnat
    4.2 Dec Ada　　compiler-decada
5. 参考　　ada-reference
    5.1 选项　　ft-ada-options
    5.2 命令　　ft-ada-commands
    5.3 变量　　ft-ada-variables
    5.4 常数　　ft-ada-constants
    5.5 函数　　ft-ada-functions
6. 附加插件　　ada-extra-plugins

---

1. 语法高亮

ft-ada-syntax

本模式为 2005 年版的 Ada ("Ada 2005") 设计，它支持面向对象编程，受保护的类型，等等。它也能处理为原始 Ada 语言的代码 ("Ada83"、"Ada87"、"Ada95")，不过代码里如果使用 Ada 2005 才有的关键字的话，颜色显示会不正确 (不过这样的代码本来也应该修正)。更多关于 Ada 的信息，见 http://www.adapower.com。

Ada 模式正确地处理一些情况。

例如，它知道 "-5" 里的 "-" 是数字的一部分，但 "A-5" 里的该字符则是操作符。通常，引用别的编译单元 (unit) 的 "with" 或 "use" 语句使用 C 里面的 "#include" 相同的色彩。如果你为 "Conditional" 或 "Repeat" 组指定了不同的色彩，那么 "end if" 和 "end loop" 会根据它们所在的组被分别标色。

你可以用 vim 的 "highlight" 命令来给出不同的颜色 (例如，要改变循环的显示方式，输入命令 ":hi Repeat"，后面跟色彩的规格说明；在普通终端上，色彩规格 ctermfg=White 通常就有不错的显示效果)。

本 Ada 模式可以设置若干选项。完整的列表可以参见 ft-ada-options 。

给选项赋值就可以打开。例如，要打开一个选项：

```
> let g:ada_standard_types = 1
```

可用 ":unlet" 关闭。例如：

```
> unlet g:ada_standard_types
```

使用 ":" 并在命令行输入这些命令就可以在载入 Ada 文件时临时设置选项。要永久设置，把 "let" 命令加到你的 "~/.vimrc" 文件里，无需冒号。

即使在慢速 (90Mhz) PC 上，该模式也很快。但如果你还是觉得速度无法忍受，可以打开 g:ada_withuse_ordinary 。

置位 g:ada_folding 时，加入语法折叠的指令 (fold-syntax)。

---

2. 文件类型插件

ft-ada-indent ft-ada-plugin

Ada 插件提供以下支持:

- 自动缩进 (indent.txt)
- 插入补全 (i_CTRL-N)
- 用户定义补全 (i_CTRL-X_CTRL-U)
- 标签搜索 (tagsrch.txt)
- Quick Fix (quickfix.txt)
- 退格处理 ('backspace')
- 注释处理 ('comments' 、 'commentstring')

当且仅当打开 Ada 文件时，本插件才激活 Ada 模式的功能，并把 Ada 相关的条目加到主菜单和弹出菜单中。

---

3. 全能补全

ft-ada-omni

Ada 全能补全 (i_CTRL-X_CTRL-O) 使用 "gnat xref -v" 或 "exuberant Ctags (http://ctags.sourceforge.net) 创建的标签数据库。补全函数会自动地识别创建标签文件使用的工具。

3.1 使用 "gnat xref" 的全能补全

gnat-xref

GNAT XREF 使用编译器的内部信息 (ali-文件) 来产生标签文件。它的优点是 100% 正确，也能进行深层的嵌套分析。不过代码必须能通过编译，生成器相当慢，而且产生的标签文件只为每个条目包含了 Ctags 提供的基本信息 - 不足以应付一些高级的 Vim 代码浏览器插件的需要。

备注: "gnat xref -v" 非常难用，因为它几乎没有任何诊断输出 - 如果没有输出，通常是因为输入的参数有问题。这里是一些重要的提示:

1) 你需要先编译代码，并使用 "-aO" 选项指向你的 .ali 文件。
2) "gnat xref -v ../Include/adacl.ads" 不行 - 应该用 "gnat xref -v -aI../Include adacl.ads" 代替。
3) "gnat xref -v -aI../Include *.ad?" 不行 - 用 "cd ../Include" 然后 "gnat xref -v *.ad?"
4) 项目管理支持彻底不能用 - 试都不用试 "gnat xref -Padacl.gpr"。

5) 使用排过序的标签文件时 Vim 速度更快 - 用 "sort --unique --ignore-case --output=tags tags"。
6) 记住在第一行插入 "!_TAG_FILE_SORTED 2 %sort ui" 来标记该文件为已排序。

3.2 使用 "ctags" 的全能补全

ada-ctags

Exuberant Ctags 使用自己的多语言代码分析器。分析速度快，产生很多附加的信息（因此得名 "Exuberant（丰富的）Ctags"），并且能处理尚不能编译的文件。

有大量其它的 Vim 工具使用 exuberant Ctags。

You 需要安装带有 Ada 支持补丁的 Exuberant Ctags 版本。GNU Ada 项目提供这样的版本（http://gnuada.sourceforge.net）。

Exuberant Ctags 的 Ada 分析器还很新 - 不要过多期望完整的支持。

---

4. 编译器支持

ada-compiler

Ada 模式支持多个 Ada 编译器，并会自动在打开 Ada 源代码时载入 g:ada_default_compiler 设置的编译器。提供的编译器插件分为两部分：实际的编译器插件和支持函数及变量的集合。这样方便了为你的开发环境专门定制的编译器插件的开发。

4.1 GNAT

compiler-gnat

GNAT 是仅有的 free（(免费的）啤酒和（自由的）言论）Ada 编译器。它有若干在许可证条款上有出入的不同版本。

按 <F7> 时，GNAT 编译器插件执行编译并立即显示结果。你可以设置项目文件如下：

```
> call g:gnat.Set_Project_File ('my_project.gpr')
```

设置项目的同时，也建立 Vim 会话（ views-sessions ），这样 - 就像 GPS 那样 - 为所有项目分别记住已打开的文件、窗口位置、等等。

gnat_members

GNAT 对 象

g:gnat.Make()

g:gnat.Make()
调用 g:gnat.Make_Command 并在 quickfix 窗口里显示结果。

g:gnat.Pretty()

g:gnat.Pretty()
调用 g:gnat.Pretty_Program

g:gnat.Find()

g:gnat.Find()
调用 g:gnat.Find_Program

g:gnat.Tags()

g:gnat.Tags()
调用 g:gnat.Tags_Command

g:gnat.Set_Project_File()

g:gnat.Set_Project_File([{file}])
设置 gnat 项目文件并载入相应的会话。已经打开的项目会被关闭，其会话也被写回。如果调用时没有提供文件名，打开文件选择对话框来选择项目文件。如果调用时提供空字符串，关闭项目和相关的会话。

g:gnat.Project_File

g:gnat.Project_File 字符串
当前项目文件。

g:gnat.Make_Command

g:gnat.Make_Command 字符串
用于 g:gnat.Make() 的外部命令 ('makeprg')。

g:gnat.Pretty_Program

g:gnat.Pretty_Program 字符串
用于 g:gnat.Pretty() 的外部命令

g:gnat.Find_Program

g:gnat.Find_Program 字符串
用于 g:gnat.Find() 的外部命令

g:gnat.Tags_Command

g:gnat.Tags_Command 字符串
用于 g:gnat.Tags() 的外部命令

g:gnat.Error_Format

g:gnat.Error_Format 字符串
错误格式 ('errorformat')

4.2 Dec Ada

compiler-hpada compiler-decada
compiler-vaxada compiler-compaqada

Dec Ada (也叫做 - 按照编年序 - VAX Ada、Dec Ada、Compaq Ada 和 HP Ada) 是一个相当古老的 Ada 83 编译器。提供的支持也很基本: <F7> 编译当前单元。

Dec Ada 编译器期待传入的参数是包 (package) 名而不是文件名。本编译器插件支持一般的文件名格式，并把文件转换为单元名。可以用 '-' 和 '__' 进行分隔。

decada_members

DEC ADA 对象

g:decada.Make()

g:decada.Make() 函数
调用 g:decada.Make_Command 并在 quickfix 窗口里显示结果。

g:decada.Unit_Name()

g:decada.Unit_Name() 函数
得到当前文件的单元名。

g:decada.Make_Command

g:decada.Make_Command 字符串
用于 g:decada.Make() 的外部命令 ('makeprg')。

g:decada.Error_Format

g:decada.Error_Format 字符串
错误格式 ('errorformat')。

---

5. 参考

ada-reference

5.1 选项

ft-ada-options

g:ada_standard_types

g:ada_standard_types 布尔型 (如果存在即为真)
高亮 Standard 包里的类型 (例如, "Float")。

g:ada_space_errors
g:ada_no_trail_space_error
g:ada_no_tab_space_error
g:ada_all_tab_usage

g:ada_space_errors 布尔型 (如果存在即为真)
高亮多余空格的错误...
g:ada_no_trail_space_error
- 但忽略行尾的空格
g:ada_no_tab_space_error
- 但忽略空格之后的制表
g:ada_all_tab_usage
- 高亮所有制表的使用

g:ada_line_errors

g:ada_line_errors 布尔型 (如果存在即为真)
高亮过长的行。注意: 本高亮选项相当占用 CPU。

g:ada_rainbow_color

g:ada_rainbow_color 布尔型 (如果存在即为真)
'(' 和 ')' 使用彩虹色。为此你需要 rainbow_parenthesis。

g:ada_folding

g:ada_folding 集合型 ('sigpft')
为 Ada 源代码提供折叠。
's': 载入时激活语法折叠
'p': 折叠包
'f': 折叠函数和过程
't': 折叠类型

'c': 折叠条件句
'g': 载入时激活美化显示折叠
'i': 单个 'is' 和上行一起折叠
'b': 单个 'begin' 和上行一起折叠
'p': 单个 'private' 和上行一起折叠
'x': 单个 'exception' 和上行一起折叠
'i': 载入时激活缩进折叠

备注：语法折叠尚处于早期（不可用）阶段，建议使用缩进或 gnat 美化折叠。

要使 gnat 美化折叠可用，建议使用下列设置：-cl3 -M79 -c2 -c3 -c4 -A1 -A2 -A3 -A4 -A5

要使缩进折叠可用，建议使用下列设置：shiftwidth=3 softtabstop=3

g:ada_abbrev

g:ada_abbrev 布尔型（如果存在即为真）
增加若干缩写。此特性或多或少为各种补全的方法取代。

g:ada_withuse_ordinary

g:ada_withuse_ordinary 布尔型（如果存在即为真）
把 "with" 和 "use" 显示为普通的关键字（用于引用其它编译单元时，它们通常被特殊高亮）。

g:ada_begin_preproc

g:ada_begin_preproc 布尔型（如果存在即为真）
使用 C 预处理命令的色彩来显示所有 begin 风格的关键字。

g:ada_omni_with_keywords

g:ada_omni_with_keywords
给全能补全加入 Keywords、Pragmas、Attributes（compl-omni）。
备注：你总可以用用户定义补全方式对它们进行补全（i_CTRL-X_CTRL-U）。

g:ada_extended_tagging

g:ada_extended_tagging 枚举型（'jump'、'list'）
使用扩展标签，可用如下两个选项
'jump': 使用 tjump 进行跳转
'list': 把标签加到 quick fix 列表。
普通的标签不支持函数或操作符重载，因为 C 语言没有这些特性，而标签原本是为 C 开发的。

g:ada_extended_completion

g:ada_extended_completion
为 <C-N> 和 <C-R> 补全（i_CTRL-N）提供扩展的补全。在这个模式中，'.' 用作标识符的一部分，这样 'Object.Method' 或 'Package.Procedure' 等就能作为一个整体进行补全。

g:ada_gnat_extensions

g:ada_gnat_extensions 布尔型（如果存在即为真）
支持 GNAT 扩充。

g:ada_with_gnat_project_files

g:ada_with_gnat_project_files　　布尔型（如果存在即为真）
加入 gnat 项目文件关键字和属性。

g:ada_default_compiler

g:ada_default_compiler　　字符串
设置缺省编译器。目前支持 'gnat' 和 'decada'。

"存在" 类型视作布尔型，变量定义时取值为真，变量未定义时为假。设置的变量值本身无关紧要。

5.2 命令

ft-ada-commands

:AdaRainbow　　:AdaRainbow
切换用于 '(' 和 ')' 的彩虹颜色（ g:ada_rainbow_color ）模式。

:AdaLines　　:AdaLines
切换行错误（ g:ada_line_errors ）显示。

:AdaSpaces　　:AdaSpaces
切换空格错误（ g:ada_space_errors ）显示。

:AdaTagDir　　:AdaTagDir
为当前文件所在目录建立标签文件。

:AdaTagFile　　:AdaTagFile
为当前文件建立标签文件。

:AdaTypes　　:AdaTypes
切换标准类型（ g:ada_standard_types ）颜色。

:GnatFind　　:GnatFind
调用 g:gnat.Find()

:GnatPretty　　:GnatPretty
调用 g:gnat.Pretty()

:GnatTags　　:GnatTags
调用 g:gnat.Tags()

5.3 变量

ft-ada-variables

g:gnat

g:gnat　　对象
管理 GNAT 编译的控制对象。只要 g:ada_default_compiler 设为 'gnat'，该对象会在载入首个 Ada 源代码时建立。详见 gnat_members 。

g:decada

g:decada 对象
管理 Dec Ada 编译的控制对象。只要 g:ada_default_compiler 设为 'decada'，该对象会在载入首个 Ada 源代码时建立。详见 decada_members 。

5.4 常数

ft-ada-constants

所有常数都是加锁的。详见 :lockvar 。

g:ada#WordRegex

g:ada#WordRegex 字符串
搜索 Ada 单词的正规表达式。

g:ada#DotWordRegex

g:ada#DotWordRegex 字符串
搜索句号分隔的 Ada 单词的正规表达式。

g:ada#Comment

g:ada#Comment 字符串
搜索 Ada 注释的正规表达式。

g:ada#Keywords

g:ada#Keywords 字典的列表
关键字、属性等的列表，使用全能补全需要的格式。详见 complete-items 。

g:ada#Ctags_Kinds

g:ada#Ctags_Kinds 列表的字典
由 Ctags 省城的 Ada 支持的各种项目的类型 (kind) 的字典。

5.5 函数

ft-ada-functions

ada#Word([{line}, {col}]) ada#Word()
返回光标所在 (或在任何给定的行/列上) 的 Ada 实体的全名，必要时删除空白/换行符。

ada#List_Tag([{line}, {col}]) ada#Listtags()
在 quick-fix 窗口列出光标所在 (或在任何给定的行/列上) 的 Ada 实体的所有出现。

ada#Jump_Tag ({ident}, {mode}) ada#Jump_Tag()
在标签跳转列表中列出光标所在 (或在任何给定的行/列上) 的 Ada 实体的所有出现。模式 (mode) 可以是 'tjump' 或 'stjump'。

ada#Create_Tags ({option}) ada#Create_Tags()
用 Ctags 建立标签文件。选项 (option) 可以设为 'file' 指定当前文件，或 'dir' 指定当前文件所在的目录，或文件名。

gnat#Insert_Tags_Header() gnat#Insert_Tags_Header()

把标签文件头部 (!_TAG_) 信息加入当前文件，GNAT XREF 的输出没有该项信息。

ada#Switch_Syntax_Option ({option}) ada#Switch_Syntax_Option()
切换高亮选项的开关。用于 Ada 菜单。

gnat#New()
gnat#New ()
建立一个新 gnat 对象。详见 g:gnat 。

---

6. 附加插件

ada-extra-plugins

可选地，你可以安装以下附加的插件。它们能和 Ada 模式一起有效工作，以增强 Ada 模式的能力。:

backup.vim
http://www.vim.org/scripts/script.php?script_id=1537
保留任意多个备份，以备不时之需。

rainbow_parenthsis.vim
http://www.vim.org/scripts/script.php?script_id=1561
非常有用，因为 Ada 只使用 '(' 和 ')'。

nerd_comments.vim
http://www.vim.org/scripts/script.php?script_id=1218
出色的注释和撤销注释支持，支持几乎所有编程语言。

matchit.vim
http://www.vim.org/scripts/script.php?script_id=39
支持任何语言的 '%' 跳转。正常的 '%' 跳转只支持 '{}' 风格的语言。Ada 模式会设置需要的搜索模式。

taglist.vim
http://www.vim.org/scripts/script.php?script_id=273
源代码浏览器侧栏。有 Ada 补丁。

Vim 的 GNU Ada 项目发布 (http://gnuada.sourceforge.net) 包含以上所有的插件。

---

vim: textwidth=78 nowrap tabstop=8 shiftwidth=4 softtabstop=4 noexpandtab
vim: filetype=help

ft_rust.txt For Vim version 8.2. 最近更新: 2019年12月

译者: Willis
http://vimcdoc.sf.net

本文档是关于 Rust 文件类型插件的。

内容 rust

1. 简介 rust-intro
2. 设置 rust-settings
3. 命令 rust-commands
4. 映射 rust-mappings

简介 rust-intro

本插件提供 Rust 文件类型的语法和支持功能。

设置 rust-settings

本插件有若干可在 vimrc 中定义的变量，用来改变插件的行为。

g:rustc_path
g:rustc_path
此选项设置 rustc 的路径，用于 :RustRun 和 :RustExpand 命令。如果不设，"rustc" 会从 $PATH 中寻找:

```
let g:rustc_path = $HOME."/bin/rustc"
```

g:rustc_makeprg_no_percent
g:rustc_makeprg_no_percent
此选项如设为 1，'makeprg' 缺省为 "rustc" 而不是 "rustc %":

```
let g:rustc_makeprg_no_percent = 1
```

g:rust_conceal
g:rust_conceal
置位此选项打开基本 conceal 支持:

```
let g:rust_conceal = 1
```

g:rust_conceal_mod_path
g:rust_conceal_mod_path
置位此选项隐藏 conceal 路径连接符 "::":

```
let g:rust_conceal_mod_path = 1
```

g:rust_conceal_pub
g:rust_conceal_pub
置位此选项隐藏 conceal "pub" 符:

```
let g:rust_conceal_pub = 1
```

g:rust_recommended_style

g:rust_recommended_style

置位此选项设置 vim 缩进和 textwidth 以遵循 rust 标准库的风格惯例 (即, 缩进为 4 个空格, 'textwidth' 设为 99)。缺省打开此选项。要关闭之:

```
let g:rust_recommended_style = 0
```

g:rust_fold

g:rust_fold

置位此选项打开 folding :

```
let g:rust_fold = 1
```

值	效果
0	无折叠
1	折叠花括号块。缺省打开全部折叠。
2	折叠花括号块。'foldlevel' 仍用其全局值 (缺省关闭全部折叠)。

g:rust_bang_comment_leader

g:rust_bang_comment_leader

设置此选项为 1 以保留使用 /*! 语法的多行文档注释的中间注释字符串:

```
let g:rust_bang_comment_leader = 1
```

g:ftplugin_rust_source_path

g:ftplugin_rust_source_path

此选项设置加到 'path' 前部的路径, 用于 Rust 源文件:

```
let g:ftplugin_rust_source_path = $HOME.'/dev/rust'
```

g:rustfmt_command

g:rustfmt_command

此选项设置 $PATH 里 'rustfmt' 可执行文件的名字。如果不设, 缺省为 'rustfmt' :

```
let g:rustfmt_command = 'rustfmt'
```

g:rustfmt_autosave

g:rustfmt_autosave

设置此选项为 1 使缓冲区保存时自动执行 :RustFmt 。如果不设, 缺省为 0:

```
let g:rustfmt_autosave = 0
```

g:rustfmt_fail_silently

g:rustfmt_fail_silently

设置此选项为 1 则 'rustfmt' 不在 location-list 中保存错误。如果不设, 缺省为 0:

```
let g:rustfmt_fail_silently = 0
```

g:rustfmt_options

g:rustfmt_options

此选项设置传递给 'rustfmt' 的选项字符串。write-mode 已经设为

'overwrite'。如果不设，缺省为 '' :

```
let g:rustfmt_options = ''
```

g:rust_playpen_url

g:rust_playpen_url

此选项覆盖 playpen 使用的 URL:

```
let g:rust_playpen_url = 'https://play.rust-lang.org/'
```

g:rust_shortener_url

g:rust_shortener_url

此选项覆盖短 URL 生成器使用的 URL:

```
let g:rust_shortener_url = 'https://is.gd/'
```

命令 rust-commands

:RustRun [args] :RustRun

:RustRun! [rustc-args] [--] [args]

编译并运行当前文件。如果有未保存的改动，先用 :update 保存。如果当前文件为无名缓冲区，先写入一个临时文件。编译后的程序总是放在一个临时目录里，但从当前目录处执行。

:RustRun 接受的参数传递给编译后的程序。

如果指定 !，取而代之的是参数会传递给 rustc。"--" 参数可用来分隔 rustc 的参数和传递给执行程序的参数。

g:rustc_path 如有定义，则用作 rustc 的路径。否则假定在 $PATH 中找到 rustc。

:RustExpand [args] :RustExpand

:RustExpand! [TYPE] [args]

使用 --pretty 扩展当前文件，并在新分割中显示结果。如果当前文件有未保存的改动，先用 :update 保存。如果当前文件为无名缓冲区，先写入一个临时文件。

:RustExpand 接受的参数传递给 rustc。主要用来指定各种 --cfg 配置。

如果指定 !，首个参数是传给 rustc --pretty 的扩展类型。否则缺省为 "expanded"。

g:rustc_path 如有定义，则用作 rustc 的路径。否则假定在 $PATH 中找到 rustc。

:RustEmitIr [args] :RustEmitIr

编译当前文件到 LLVM IR 并在新分割中显示结果。如果当前文件有未保存的改动，先用 :update 保存。如果当前文件为无名缓冲区，先写入一个临时文件。

:RustEmitIr 接受的参数传递给 rustc。

g:rustc_path 如有定义，则用作 rustc 的路径。否则假定在 $PATH 中找到 rustc。

:RustEmitAsm [args] :RustEmitAsm

编译当前文件到汇编程序，并在新分割中显示结果。如果当前文件有未保存的改动，先用 :update 保存。如果当前文件为无名缓冲区，先写入一个临时文件。

:RustEmitAsm 接受的参数传递给 rustc。

g:rustc_path 如有定义，则用作 rustc 的路径。否则假定在 $PATH 中找到 rustc。

:RustPlay :RustPlay

此命令只有在安装了 web-api.vim 后才能工作（可从这里找到 https://github.com/mattn/webapi-vim）。发送当前选择区，如果没有选择过的话，则发送整个缓冲区到 Rust playpen，并发送短 URL 到 playpen。

g:rust_playpen_url 是 playpen 的基准 URL，缺省是 "https://play.rust-lang.org/"。

g:rust_shortener_url 是短 URL 生成器的基准 URL，缺省是 "https://is.gd/"

:RustFmt :RustFmt

在当前缓冲区上执行 g:rustfmt_command 。如果设置了 g:rustfmt_options ，传递给该程序。

如果 g:rustfmt_fail_silently 为 0（缺省），则 g:rustfmt_command 的错误会添加到 location-list 里。如果 g:rustfmt_fail_silently 设为 1，则不会写入错误到 location-list 。

:RustFmtRange :RustFmtRange

在指定范围上执行 g:rustfmt_command 。其它信息见 :RustFmt

映射 rust-mappings

此插件定义了 [[和]] 的映射以支持悬挂缩进。

还定义了一些其它映射：

<D-r> rust_<D-r>

执行 :RustRun ，不带参数。
备注：此绑定只适用于 MacVim。

<D-R> rust_<D-R>

重用上次调用使用的参数来生成 :RustRun ! 命令行，但先不执行。
备注：此绑定只适用于 MacVim。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

ft_sql.txt　　For Vim version 8.2.　最近更新：2013年8月

作者：David Fishburn
译者：Willis
http://vimcdoc.sf.net

这里介绍用于 SQL 文件的文件类型插件。

结构化查询语言（Structured Query Language，SQL）是规范支持用户和关系型数据库进行交互的语句的标准。Vim 包含了 SQL 导航、缩进和语法高亮的功能。

1. 导航　sql-navigation
    1.1 matchit　sql-matchit
    1.2 文本对象动作　sql-object-motions
    1.3 预定义对象动作　sql-predefined-objects
    1.4 宏　sql-macros
2. SQL 方言　sql-dialects
    2.1 SQLSetType　SQLSetType
    2.2 SQLGetType　SQLGetType
    2.3 SQL 方言缺省值　sql-type-default
3. 增加新的 SQL 方言　sql-adding-dialects
4. 全能 SQL 补全　sql-completion
    4.1 静态模式　sql-completion-static
    4.2 动态模式　sql-completion-dynamic
    4.3 教程　sql-completion-tutorial
        4.3.1 补全表　sql-completion-tables
        4.3.2 补全列　sql-completion-columns
        4.3.3 补全过程　sql-completion-procedures
        4.3.4 补全视图　sql-completion-views
    4.4 定制补全　sql-completion-customization
    4.5 SQL 映射　sql-completion-maps
    4.6 使用其它文件类型　sql-completion-filetypes

---

1. 导航　sql-navigation

SQL 文件类型为文件导航提供了一些选项。

1.1 matchit　sql-matchit

---

matchit 插件（http://www.vim.org/scripts/script.php?script_id=39）提供许多附加功能，并且可以为不同语言定制。matchit 插件通过定义缓冲区局部变量 b:match_words 来进行配置。在若干关键字上按 % 键会把光标移动到匹配文本处。例如，如果光标在 "if" 上，按 % 会在 "else"、"elseif" 和 "end if" 关键字之间循环。

支持以下关键字：

```
if
elseif | elsif
else [if]
end if

[while 条件] loop
```

```
    leave
    break
    continue
    exit
end loop

for
    leave
    break
    continue
    exit
end loop

do
    语句
doend

case
when
when
default
end case

merge
when not matched
when matched

create[ 或 replace] procedure|function|event
returns
```

1.2 文本对象动作 sql-object-motions

Vim 预定义了若干处理文本对象动作 object-motions 的键。本文件插件试图把这些键翻译成对 SQL 语言有意义的映射。

存在以下的普通模式 Normal 和可视模式 Visual 映射 (编辑 SQL 文件时):

```
]]              正向移动到下一个 'begin'
[[              反向移动到前一个 'begin'
][              正向移动到下一个 'end'
[]              反向移动到前一个 'end'
```

1.3 预定义对象动作 sql-predefined-objects

多数关系型数据库支持若干标准特性、表、索引、触发器和存储过程。每个供应商也有一些私有的对象。以下一组映射的建立是为了方便在这些对象间移动。取决你使用的数据库供应商，应该可以调整可用的对象。本文件插件试图定义许多标准和附加的对象。为了尽可能灵活， vimrc 里可以这样改写可用的对象列表:

```
let g:ftplugin_sql_objects = 'function,procedure,event,table,trigger' .
            \ ',schema,service,publication,database,datatype,domain' .
            \ ',index,subscription,synchronization,view,variable'
```

存在使用上述列表的下述普通模式 Normal 和可视模式 Visual 映射:

```
]}              正向移动到下一个 'create <对象名>'
[{              反向移动到前一个 'create <对象名>'
```

重复按]} 会在每个 create 语句之间循环:

```
create table t1 (
    ...
);

create procedure p1
begin
    ...
end;

create index i1 on t1 (c1);
```

缺省的 g:ftplugin_sql_objects 设置是:

```
let g:ftplugin_sql_objects = 'function,procedure,event,' .
            \ '\\(existing\\|global\\s\\+temporary\\s\\+\\)\\{,1}' .
            \ 'table,trigger' .
            \ ',schema,service,publication,database,datatype,domain' .
            \ ',index,subscription,synchronization,view,variable'
```

该设置同时能处理以下情形:

```
create table t1 (
    ...
);
create existing table t2 (
    ...
);
create global temporary table t3 (
    ...
);
```

缺省，文件类型插件只寻找 CREATE 语句。在 vimrc 里你可以改写下行为:

```
let g:ftplugin_sql_statements = 'create,alter'
```

文件类型插件定义了三种注释类型:

```
1.  --
2.  //
3.  /*
     *
     */
```

存在用于注释的下述普通模式 Normal 和可视模式 Visual 映射:

```
]"              正向移动到注释开始处
["              反向移动到注释结束处
```

1.4 宏 sql-macros

支持 Vim 寻找宏定义的特性 'define' 。使用以下正规表达式:

```
\c\<\(VARIABLE\|DECLARE\|IN\|OUT\|INOUT\)\>
```

该设置可以处理下面的代码：

```
CREATE VARIABLE myVar1 INTEGER;

CREATE PROCEDURE sp_test(
    IN myVar2 INTEGER,
    OUT myVar3 CHAR(30),
    INOUT myVar4 NUMERIC(20,0)
)
BEGIN
    DECLARE myVar5 INTEGER;

    SELECT c1, c2, c3
      INTO myVar2, myVar3, myVar4
      FROM T1
     WHERE c4 = myVar1;
END;
```

把光标放在下面这行的 "myVar1" 上：

```
     WHERE c4 = myVar1;
                ^
```

按下面的任何一个键：

```
[d
[D
[CTRL-D
```

---

2. SQL 方言 sql-dialects sql-types

sybase TSQL Transact-SQL
sqlanywhere
oracle plsql sqlj
sqlserver
mysql postgresql psql
informix

所有关系型数据库都支持 SQL。各供应商之间通用 SQL 的一个语言子集（例如，CREATE TABLE、CREATE INDEX），但供应商自己对 SQL 有相当多的扩展。Oracle 支持 "CREATE OR REPLACE" 语法，CREATE TABLE 语句也可以指定列缺省值，还有过程语言（用于存储过程和激发器）。

Vim 发布提供的语法高亮缺省基于 Oracle 的 PL/SQL。SQL 缩进脚本缺省支持 Oracle 和 SQL Anywhere。文件类型插件缺省支持所有供应商并保持供应商中立，但可以进行扩展。

Vim 支持许多不同的供应商，目前这是通过语法脚本完成的。不幸的是，如果你要切换语法规则，必须建立以下之一：

1. 新文件类型
2. 定制的自动命令
3. 手动步骤 / 命令

大多数用户只使用一个供应商的数据库产品，因此最好能在 vimrc 里指定缺省值。

2.1 SQLSetType　　sqlsettype　SQLSetType

对于使用多个不同的数据库的用户，最好随时为每个缓冲区都可以切换不同的供应商规则(缩进、语法)。ftplugin/sql.vim 文件定义以下函数:

```
SQLSetType
```

如果不带任何参数，此函数设置缩进和语法脚本回缺省值，见 sql-type-default 。
如果关闭 Vi 兼容模式 'compatible' ，<Tab> 键可以补全可选参数。

输入函数名和空格，然后用补全就可以提供参数。该函数接受的参数是待执行的 Vim 脚本名。因此，利用 cmdline-completion 特性，它在 'runtimepath' 里查找所有名字包含 'sql' 的脚本。
此过程需要对名字的拼写进行猜测。下面是一些例子:

```
:SQLSetType
:SQLSetType sqloracle
:SQLSetType sqlanywhere
:SQLSetType sqlinformix
:SQLSetType mysql
```

最容易的方法是用 <Tab> 字符，先用第一个 <Tab> 补全命令名 (SQLSetType)，在空格和另一个 <Tab> 之后，显示可用的 Vim 脚本名的列表:

```
:SQL<Tab><space><Tab>
```

2.2 SQLGetType　　sqlgettype　SQLGetType

你随时可以用 SQLGetType 命令得到正在使用的 SQL 方言。ftplugin/sql.vim 定义此函数:

```
SQLGetType
```

会显示:

```
Current SQL dialect in use:sqlanywhere
```

2.3 SQL 方言缺省值　　sql-type-default

前面已经提到，Vim 语法规则缺省基于 Oracle (PL/SQL)。你可以在 vimrc 里加上以下各行之一，对此进行修改:

```
let g:sql_type_default = 'sqlanywhere'
let g:sql_type_default = 'sqlinformix'
let g:sql_type_default = 'mysql'
```

如果你在 vimrc 里加入:

```
let g:sql_type_default = 'sqlinformix'
```

下次编辑 SQL 文件时，Vim 会自动载入以下的脚本:

```
ftplugin/sql.vim
syntax/sqlinformix.vim
indent/sql.vim
```

注意 没有载入 indent/sqlinformix.sql，这时因为没有 Informix 专用的缩进文件。如

果指定文件不存在，Vim 会载入缺省文件。

---

3. 增加新的 SQL 方言 sql-adding-dialects

如果使用 Vim 缺省发布还没有提供定制版本的 SQL 方言，查看 http://www.vim.org，了解现在是否已经有你需要的定制版本。如果还没有，你可以从复制一个现有的脚本开始做起。阅读 filetype-plugins 了解详情。

为了帮助识别这些脚本，建立文件时应使用 "sql" 前缀。例如，如果你决定建立 SQLite 数据库的定制版本，可以从下表中挑选要建立的文件：

```
Unix
    ~/.vim/syntax/sqlite.vim
    ~/.vim/indent/sqlite.vim
Windows
    $VIM/vimfiles/syntax/sqlite.vim
    $VIM/vimfiles/indent/sqlite.vim
```

不需要修改 SQLSetType 函数。在你使用 SQLSetType 命令时，它会自动挑选新的 SQL 文件并载入它们。

---

4. 全能 SQL 补全 sql-completion omni-sql-completion

Vim 7 包含了代码补全的接口和函数以允许插件开发者为任何语言提供代码补全。Vim 7 已经包含了 SQL 语言代码补全。

SQL 补全插件有两种方式，静态和动态。静态模式从当前语法高亮规则里提取数据来构造弹出菜单，而动态模式直接从数据库提取数据来构造弹出菜单。这包括所有的表和列、过程名和更多对象。

4.1 静态模式 sql-completion-static

编辑文件类型为 SQL 的文件时，静态弹出菜单包含激活的语法规则里定义的项目。插件缺省定义了一些映射以帮助用户重新定义要显示的项目清单。
缺省的静态映射是：

```
imap <buffer> <C-C>a <C-\><C-O>:call sqlcomplete#Map('syntax')<CR><C-X><C-O>
imap <buffer> <C-C>k <C-\><C-O>:call sqlcomplete#Map('sqlKeyword\w*')<CR><C-X><C-O>
imap <buffer> <C-C>f <C-\><C-O>:call sqlcomplete#Map('sqlFunction')<CR><C-X><C-O>
imap <buffer> <C-C>o <C-\><C-O>:call sqlcomplete#Map('sqlOption')<CR><C-X><C-O>
imap <buffer> <C-C>T <C-\><C-O>:call sqlcomplete#Map('sqlType')<CR><C-X><C-O>
imap <buffer> <C-C>s <C-\><C-O>:call sqlcomplete#Map('sqlStatement')<CR><C-X><C-O>
```

"<C-C>" 也可以由用户在 .vimrc 里自定义，因为不是在所有平台下它都可用：

```
let g:ftplugin_sql_omni_key = '<C-C>'
```

静态映射（基于语法高亮组）都符合以下格式：

```
imap <buffer> <C-C>k <C-\><C-O>:call sqlcomplete#Map('sqlKeyword')<CR><C-X><C-O>
```

让我们把这行命令拆开来解释一下：

```
imap                    - 建立插入模式映射
<buffer>                 - 只用于此缓冲区
<C-C>k                   - 你选择的要映射的键
<C-\><C-O>                - 执行单个命令，然后回到插入模式
:call sqlcomplete#Map( - 允许 SQL 补全插件执行一些例行函数，使之可以和其他
                         补全插件一起使用。
                         指示你想要 SQL 补全插件补全哪个项目。
                         该例中我们要求插件显示语法高亮组 'sqlKeyword' 的
                         项目。
                         编辑 SQL 文件时，执行下面命令可以查看高亮组名清单
                         以供选择
                             :syntax list
'sqlKeyword'            - 我们要求显示 sqlKeyword 高亮组的项目
'sqlKeyword\w*'         - Vim 7.4 开始可用的另一个选项，使用正规表达式来决
                         定使用的语法高亮组
)<CR>                    - 执行 :let 命令
<C-X><C-O>                - 激活全能补全的标准键击。传递 'sqlKeyword' 参数指
                         定 SQL 补全插件用 sqlKeyword 高亮组的项目来构造弹
                         出菜单。该插件同时缓存结果直到 Vim 重启为止。它用
                         syntaxcomplete 插件来提取 syntax 列表。
```

使用 'syntax' 关键字是个特例。它使用 syntaxcomplete 插件来提取所有的语法项目。这样可以有效使用所有 Vim SQL 语法文件。此文档编写时，这里包含了不同 SQL 方言的 10 种语法文件（见上第 3 节， sql-dialects ）。

这里是从语法文件提取的可用项目的例子：

```
All
    - 包含所有语法高亮组的内容
Statement
    - Select、Insert、Update、Delete、Create、Alter、...
Function
    - Min、Max、Trim、Round、Date、...
Keyword
    - Index、Database、Having、Group、With
Option
    - Isolation_level、On_error、Qualify_owners、Fire_triggers、...
Type
    - Integer、Char、Varchar、Date、DateTime、Timestamp、...
```

4.2 动态模式 sql-completion-dynamic

动态模式直接从数据库提取数据来构造弹出菜单。为了打开动态功能，你需要安装 dbext.vim 插件。(http://vim.sourceforge.net/script.php?script_id=356)。

SQL 补全插件的若干特性使用动态模式。安装完 dbext 插件后，参阅 dbext-tutorial 教程来了解附加的配置和使用说明。dbext 插件允许 SQL 补全插件显示表、过程、视图和列的清单。

```
表清单
    - 所有模式 (schema) 拥有者的所有表
过程清单
    - 所有模式拥有者的所有存储过程
视图清单
```

- 所有模式拥有者的所有视图

列清单
- 选择的表包含的所有列

要在插入模式下打开弹出菜单，可以分别为每组使用下面的键组合（其中 <C-C> 意味着按住 CTRL 键的同时按住 C）：

表清单
- <C-C>t
- <C-X><C-O>（缺省映射假定使用表）

存储过程清单 - <C-C>p

视图清单 - <C-C>v

列清单 - <C-C>c

钻进 / 钻出
- 查看显示表清单的弹出窗口时，<Right> 可将当前高亮的表替代为该表的列清单。
- 查看显示列清单的弹出窗口时，<Left> 可将列清单替代为表清单。
- 这样你可以快速钻进表里查看包含的列然后快速回来。
- <Right> 和 <Left> 可在 .vimrc 中指定：

```
let g:ftplugin_sql_omni_key_right = '<Right>'
let g:ftplugin_sql_omni_key_left  = '<Left>'
```

SQL 补全插件缓存若干弹出窗口的显示清单。这样，重新显示这些清单的速度就会大大加快。如果数据库加入了新表或新列，插件缓存需要清空。此功能的缺省映射是：

```
imap <buffer> <C-C>R <C-\><C-O>:call sqlcomplete#Map('ResetCache')<CR><C-X><C-O>
```

4.3 SQL 教程 sql-completion-tutorial

本教程的目的是为了带你了解 SQL 补全插件的常用功能，以便：

a) 你能熟悉插件的使用
b) 你能得到一些常用功能的介绍
c) 说明如何定制你的特定偏好
d) 展示插件 "最好用" 的方法（最容易的配置方法）。

首先，建立新缓冲区：

```
:e tutorial.sql
```

静态特性

要得到各种清单，只要进入插入模式，输入：

<C-C>s （显示 SQL 语句）

这时你可以往下翻页查看清单，直到看到 "select" 为止。如果你知道要找什么项目，比如你知道该语句以字母 "s" 开头，可以先预输入（不输入引号）"se" 然后按：

<C-Space>t

假定弹出清单里高亮了 "select"，按 <Enter> 选择该项目。现在输入：

* fr<C-C>a（显示所有语法项目）

然后选择弹出清单里的 "from"。

编写存储过程时可用 "type" 清单。它包含数据库支持类型的清单。这取决于你使用的语法文件，因此可能是也可能不是完全准确。SQL Anywhere 语法文件（sqlanywhere.vim）包含此支持：

```
BEGIN
```

```
DECLARE customer_id <C-C>T <-- 从清单里选择一个类型
```

动态特性

要利用动态特性，先安装 (http://vim.sourceforge.net/script.php?script_id=356) dbext.vim 插件。它也带有自己的教程。从 SQL 补全插件的角度而言，dbext 提供的主要特性是连接到数据库。dbext 的连接设置文件是定义连接信息的最有效的方法。一旦建立好连接，SQL 补全插件在后台使用 dbext 的功能来为弹出菜单填充数据。

下面假设 dbext.vim 已经经过正确配置。简单的测试方法是运行命令 :DBListTable。如果显示了表清单，那么 dbext.vim 工作正常。如果不行，查询 dbext.txt 文档。

假定你照着 dbext-tutorial 教程来做，可以按 <C-C>t 来显示表清单。这时会有一些延迟，因为 dbext 正在建立表清单。显示清单后按 <C-W> 会同时删除弹出菜单和清单激活时已经选中的表名。

4.3.1 补全表: sql-completion-tables

按 <C-C>t 显示通过 dbext 插件连接的数据库里的表清单。
注意：所有 SQL 补全弹出菜单都支持在按键映射之前输入前缀。这时，弹出菜单只包含该前缀开始的项目。

4.3.2 补全列: sql-completion-columns

SQL 补全插件也能显示特定表的列清单。<C-C>c 激活列补全。

注意：下面的例子在弹出菜单激活时用 <Right> 来激活列清单。

使用列补全的示例:

- 再按一次 <C-C>t 显示表清单。
- 补全窗口显示清单时，按 <Right>，表清单会被替换为高亮表包含的列清单 (在同样短暂的延迟后)。
- 按下 <Left>，列清单又被替换为表清单。这样，你就可以来回快速进出各种表和列清单。
- 高亮同一个表时，如果又按 <Right>，你会注意到不再有延迟，这是因为列清单已经进行了缓存。如果改变了已经缓存的表的模式 (schema)， <C-C>R 命令可以清除 SQL 补全缓存。
- 注意：<Right> 和 <Left> 设计在补全窗口激活时工作。如果没有激活补全窗口，执行普通的 <Right> 或 <Left>。

让我们看看如何动态构造 SQL 语句。select 语句需要一个列清单。SQL 补全插件有两个方法构造列清单。

一次一列:

1. 输入 SELECT 后，按 <C-C>t 显示表清单。
2. 从清单里选择一个表。
3. 按 <Right> 显示列清单。
4. 从清单里选择列并按回车。
5. 输入 "," 并按 <C-C>c。一般，生成列清单需要光标在表名上。插件使用该名字来决定从哪个表提取列清单。在这一步，因为我们按 <C-C>c 时光标不在表名上，显示前一次使用的表的列清单。选择另一列，然后继续。
6. 根据需要，反复进行第 5 步。

特定表的所有列:

1. 输入 SELECT 后，按 <C-C>t 显示表清单。
2. 高亮你需要所有列清单的表。
3. 按 <Enter> 从清单里选择该表。
4. 按 <C-C>l 来请求该表以逗号分隔的所有列的清单。
5. 基于第三步选择的表名，插件试图决定合适的表别名。你会得到提示，接受或者修改该别名。然后按 OK。
6. 表名被该表用逗号分隔的列清单替换，每列前面都会加上表的别名。
7. 步骤 3 和 4 可以被 <C-C>L 替换，它在映射里内嵌了 <C-Y> 以选择清单里当前正在高亮的表。

编写 select 语句时进行了一项特殊的准备工作。考虑下列语句：

```
select *
  from customer c,
       contact cn,
       department as dp,
       employee e,
       site_options so
 where c.
```

"c" 是 "customer" 表的别名。在插入模式下输入最后一个 "c." 之后，按 <C-C>c 或 <C-X><C-O> 会因此弹出 customer 表的列清单。因为插件会往回查找到 select 语句开始处并从那里开始寻找 FROM 字句定义的表清单。该例中它发现了字符串 "customer c" 并因此知道 "c" 是 customer 表的别名。这里也支持可选的 "AS" 关键字， "customer AS c"。

4.3.3 补全过程： sql-completion-procedures

和表清单类似，<C-C>p 显示数据库里保存的存储过程的清单。

4.3.4 补全视图： sql-completion-views

和表清单类似，<C-C>v 显示数据库里视图的清单。

4.4 定制补全 sql-completion-customization

在 vimrc 里，可以用若干选项定制 SQL 补全插件：

omni_sql_no_default_maps (无缺省映射)
- 缺省：没有定义
- 如果定义此变量，不建立全能补全的映射。详见 sql-completion-maps 。

omni_sql_use_tbl_alias (使用表别名)
- 缺省：a
- 只有生成逗号分隔的列清单时才使用此设置。缺省映射为 <C-C>l。生成列清单时，每列前面都加上一个别名，例如： e.emp_id, c.emp_name。此选项有三种设置：

  n - 不使用别名
  d - 使用缺省的 (经过计算的) 别名
  a - 要求确认别名

  以下一些规则决定别名：

  1. 如果表名包含 '_'，用它作为分隔符：

MY_TABLE_NAME --> MTN
my_table_name --> mtn
My_table_NAME --> MtN

2. 如果表名 不 包含 '_'，但 包含 混合大小写，使用大小写作为分隔符：
   MyTableName --> MTN
3. 如果表名 不 包含 '_' 而且也 不 包含混合大小写，使用表的第一个字母：
   mytablename --> m
   MYTABLENAME --> M

omni_sql_ignorecase (忽略大小写)
- 缺省：'ignorecase' 的当前设置
- 合法的设置是 0 或 1。
- 启动补全之前如果输入一些字符，产生的列表会被过滤，只显示这些字符开始的项目。如果此选项为 0，列表过滤时考虑大小写的敏感性。

omni_sql_include_owner (包含拥有者)
- 缺省：0，除非安装了 dbext.vim 3.00
- 合法的设置是 0 或 1。
- 如果用 dbext.vim 3.00 或更高版本补全表、过程或视图，对象清单里同时包含拥有者的名字。补全这些对象时如果打开了 omni_sql_include_owner，拥有者的名字会被替代。

omni_sql_precache_syntax_groups (预缓存语法组)
- 缺省：
  ['syntax','sqlKeyword','sqlFunction','sqlOption','sqlType','sqlStatement']
- sqlcomplete 可以和其它补全插件一起使用。 sql-completion-filetypes 提供简要说明。当文件类型暂时改为 SQL 时， sqlcompletion 插件缓存此选项列表列出的语法组。

4.5 SQL 映射 sql-completion-maps

缺省 SQL 映射在本文档的其它章节里已经有更详细的描述。这里列出所有映射，并带有每个映射的简短描述。

静态映射

这些映射使用 Vim 的语法高亮规则来给补全清单填充数据。

<C-C>a
- 显示所有的 SQL 语法项目。

<C-C>k
- 显示所有 'sqlKeyword' 定义的 SQL 语法项目。

<C-C>f
- 显示所有 'sqlFunction' 定义的 SQL 语法项目。

<C-C>o
- 显示所有 'sqlOption' 定义的 SQL 语法项目。

<C-C>T
- 显示所有 'sqlType' 定义的 SQL 语法项目。

<C-C>s
- 显示所有 'sqlStatement' 定义的 SQL 语法项目。

动态映射

这些映射使用 dbext.vim 插件来给补全清单填充数据。

<C-C>t
- 显示表清单。

<C-C>p
- 显示过程清单。

<C-C>v
- 显示视图清单。

<C-C>c
- 显示特定表的列清单。

<C-C>l
- 显示特定表用逗号分隔的列清单。

<C-C>L
- 显示特定表用逗号分隔的列清单。
  只有补全窗口激活时才能使用此映射。

<Right>
- 显示补全窗口当前高亮表的列清单。

<Left>
- 显示表清单。

<C-C>R
- 本映射删除所有缓存项目并强制 SQL 补全重新生成所有的项目清单。

定制映射

根据需要，你可以建立许多附加的键映射。一般来说，映射指定不同的语法高亮组。

如果你不希望建立缺省的映射或者选择的键不能用于你的平台 (通常在 *nix 上)，在 vimrc 里定义下面的变量:

```
let g:omni_sql_no_default_maps = 1
```

不要直接编辑 ftplugin/sql.vim！如果修改了此文件，将来的更新会覆盖你的改动。Vim 有一个特殊的目录结构，你可以进行定制而无须修改 Vim 发布里包含的文件。如果你希望定制映射，建立 after/ftplugin/sql.vim (见 after-directory)，并在那里放上和 ftplugin/sql.vim 相同的映射并使用你自己的键击。之所以选择 <C-C>，是因为它在 Windows 和 *nix 平台上都能工作。Windows 平台上也可用 <C-Space> 或 ALT 键。

4.6 使用其它文件类型 sql-completion-filetypes

很多时候，SQL 可以用于不同的文件类型。例如，Perl、Java、PHP 和 Javascript 都可以和数据库进行交互。通常你既需要 SQL 补全，也需要当前编辑的语言的补全功能。

使用下面的步骤，这很容易做到 (假设 Perl 文件):

1. :e test.pl
2. :set filetype=sql
3. :set ft=perl

步骤 1

开始编辑 Perl 文件。Vim 自动设置文件类型为 "perl"。缺省，Vim 执行合适的文件类型文件 ftplugin/perl.vim。如果你依照 ft-syntax-omni 的指示使用语法补全插件，那么 'omnifunc' 选项已经被设置为 "syntax#Complete"。按 <C-X><C-O> 会显示全能补全的弹出菜单，里面包含 Perl 的语法项目。

步骤 2

手动设置文件类型为 'sql' 会激活相应的文件类型文件 ftplugin/sql.vim。该文件定义很多特定于缓冲区的 SQL 补全的映射，见 sql-completion-maps 。现在这些映射已经建立，而 SQL 补全插件也已经初始化。所有 SQL 语法项目也在缓存里准备完毕。SQL 文件类型脚本会检测到我们在试图使用两个不同的补全插件。因为 SQL 映射都以 <C-C> 开始，而这些映射只在使用时才会激活 'omnifunc' 。所以用 <C-X><C-O> 仍然可以继续使用 Perl 补全 (使用语法补全插件)，而 <C-C> 则使用 SQL 补全的功能。

步骤 3

把文件类型设回 Perl，那么所有与 "perl" 有关的项目都回到原来的设置。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册

by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

二合字母

digraph digraphs Digraphs

二合字母用来输入不能用普通键盘输入的字符。它们通常是可显示的非 ASCII 字符。二合字母比用 CTRL-V 输入十进制编码更方便记忆（见 i_CTRL-V ）。

用户手册 24.9 有二合字母的简短介绍。一个替代方案是用 'keymap' 选项。

1. 定义二合字母 digraphs-define
2. 使用二合字母 digraphs-use
3. 缺省的二合字母 digraphs-default

---

1. 定义二合字母

digraphs-define

:dig :digraphs

:dig[raphs][!] 显示当前定义的二合字母。
带 [!] 使用表头，方便查找特定的字符。

E104 E39

:dig[raphs] {char1}{char2} {number} ...
把二合字母 {char1}{char2} 加入列表。{number} 是字符的十进制表示形式。例如：

```
:digr e: 235 a: 228
```

避免定义 '_'（下划线）开头的二合字母，将来它会有特殊的含义。

Vim 通常编译时带有 +digraphs 特性。如果没有，":digraph" 命令会显示错误信息。

":digraphs" 输出的例子：

```
TH  222  ss  223  a! à  224  a' á  225  a>  226  a?  227  a:  228
```

每列（译者注："TH 222" 称为一列）前两个字符是你用来输入二合字母的两个字符。

每列的中间是产生的字符。如果你用不支持二合字母的系统察看或者打印，它们可能是乱码。

digraph-encoding

通常，十进制表示形式是字符的 Unicode 数值。注意 'encoding' 改变的时候该数值并不改变。如果需要，字符从 Unicode 转换为 'encoding'。这需要该转换确实能实现，有时并非如此。NUL 字符会显示为 "10"，这是因为 NUL 字符内部用 NL 字符表示。写回文件时它会还原为 NUL 字符的。

示例：

```
digraph oe 339
```

这样就定义了 "oe" 二合字母为 Unicode 数值 339 的字符。

---

2. 使用二合字母 digraphs-use

有两个方法输入二合字母: i_digraph

```
CTRL-K {char1} {char2}          或
{char1} <BS> {char2}
```

第一个方法总是可以用的; 第二个方法只有在 'digraph' 选项置位的时候才可用。

如果二合字母 {char1}{char2} 不存在, Vim 会搜索二合字母 {char2}{char1}。这样, 即使你不记得何者在前, 也能输入。

注意 如果输入 CTRL-K {char1} 时, {char1} 是特殊键, Vim 只会输入该键的编码。这就不是输入二合字母了。

一旦你输入了二合字母, Vim 把该字符当成占据文件和屏幕的一个正常字符。示例:

```
'B' <BS> 'B'     会输入中间断开的 '|' 字符 (166)
'a' <BS> '>'     会输入带 circumflex 重音的 'a' (226)
CTRL-K '-' '-'   会输入软的破折号 (173)
```

":digraphs" 列出当前的二合字母。一些缺省的二合字母列在下面的 digraph-table 里。

对 CTRL-K 而言, 有一个通用的二合字母: CTRL-K <Space> {char} 会输入 {char}, 并设置最高位。你可以用这个方法来输入带 meta 的字符。

二合字母里不能有 <Esc> 字符。按 <Esc> 的时候, Vim 停止二合字母的输入并回到插入或命令行模式, 这也可以理解为 <Esc> 退出了二合字母的上下文。用 CTRL-V 155 输入 meta-ESC (CSI)。

如果你本来想输入 'e' 但不小心输入了 'a', 你也许会键入 'a' <BS> 'e'。但是这个序列成了二合字母。这也许不是你想要的。要纠正这个错误, 你需要再按一次 <BS> e。要避免这一点, 不要置位 'digraph' 选项, 只用 CTRL-K 输入二合字母。

Vim 里使用值高于 128 的字符可能会有问题。例如: 你输入 ue (u-umlaut) 而编辑器在插入模式回显 \334。离开插入模式一切就正常了。有的 Unix 系统上, 解决方法是定义 LC_CTYPE。如果你使用 csh, 把这行放入 .cshrc:

```
setenv LC_CTYPE en_US.utf8
```

(或用于不同语言或国家的类似值)。该值必须是你的系统支持的 locale, 亦即, 在 Unix 系列的系统上必须在下面的输出中存在

```
locale -a
```

---

3. 缺省的二合字母 digraphs-default

Vim 缺省提供了一些二合字母。":digraphs" 的输出列出这些二合字母。

Vim 在大多数系统中使用相同二合字母。它们适用于 Unicode 和 ISO-8859-1 字符集。缺省的二合字母是 RFC1345 提供的助记符。为了方便记住这些助记符, 第二个字符有一个标准的含义:

字符名	字符	含义
Exclamation mark	!	Grave (钝音符号)
Apostrophe	'	Acute (锐音符号)
Greater-Than sign	>	Circumflex (折音/抑扬符号)
Question Mark	?	Tilde (腭化/鼻音化符号)

Hyphen-Minus	-	Macron (长音符号)
Left parenthesis	(	Breve (短音符号)
Full Stop	.	上带点 (Dot)
Colon	:	Diaeresis (分音符号)
Comma	,	Cedilla (软音符号)
Underline	_	带下划线 (Underline)
Solidus	/	带短划线 (Stroke)
Quotation mark	"	Double acute (双锐音符号)
Semicolon	;	Ogonek (鼻音化符号)
Less-Than sign	<	Caron (楔形/倒折音符号)
Zero	0	上带圆圈 (Ring)
Two	2	上带钩 (Hook)
Nine	9	带角 (Horn)
Equals	=	西里尔字母 (Cyrillic) (= 用作第二字符)
Asterisk	*	希腊语
Percent sign	%	特殊的希腊/西里尔字母
Plus	+	小写：阿拉伯语，大写：希伯来语
Three	3	一些拉丁/希腊/西里尔字母
Four	4	注音符号
Five	5	平假名
Six	6	片假名

示例：a: 是　而 o: 是

下面列出 RFC1345 定义的单字节二合字母。其他的二合字母见 ":digraphs"。

EURO

特例：RFC1345 没有指定欧元 (euro) 符号。Vim 里为此目的加入了二合字母 =e。注意以下区别：latin1 里，二合字母 Cu 用于货币符号，而 latin9 (iso-8859-15) 里二合字母 =e 用于欧元符号，尽管两者都代表字符 164，0xa4。为了和 zsh 兼容，Eu 也用于欧元符号。

ROUBLE

卢布 (rouble) 符号在 2014 年加入，其代码为 0x20bd。Vim 里为此支持二合字母 =R 和 =P。注意 R= 和 P= 是其它字符。

digraph-table

字符	二合字母	十六进	十进制	正式名字
^@	NU	0x00	0	NULL (NUL)
^A	SH	0x01	1	START OF HEADING (SOH)
^B	SX	0x02	2	START OF TEXT (STX)
^C	EX	0x03	3	END OF TEXT (ETX)
^D	ET	0x04	4	END OF TRANSMISSION (EOT)
^E	EQ	0x05	5	ENQUIRY (ENQ)
^F	AK	0x06	6	ACKNOWLEDGE (ACK)
^G	BL	0x07	7	BELL (BEL)
^H	BS	0x08	8	BACKSPACE (BS)
^I	HT	0x09	9	CHARACTER TABULATION (HT)
^@	LF	0x0a	10	LINE FEED (LF)
^K	VT	0x0b	11	LINE TABULATION (VT)
^L	FF	0x0c	12	FORM FEED (FF)

^M	CR	0x0d	13	CARRIAGE RETURN (CR)
^N	SO	0x0e	14	SHIFT OUT (SO)
^O	SI	0x0f	15	SHIFT IN (SI)
^P	DL	0x10	16	DATALINK ESCAPE (DLE)
^Q	D1	0x11	17	DEVICE CONTROL ONE (DC1)
^R	D2	0x12	18	DEVICE CONTROL TWO (DC2)
^S	D3	0x13	19	DEVICE CONTROL THREE (DC3)
^T	D4	0x14	20	DEVICE CONTROL FOUR (DC4)
^U	NK	0x15	21	NEGATIVE ACKNOWLEDGE (NAK)
^V	SY	0x16	22	SYNCHRONOUS IDLE (SYN)
^W	EB	0x17	23	END OF TRANSMISSION BLOCK (ETB)
^X	CN	0x18	24	CANCEL (CAN)
^Y	EM	0x19	25	END OF MEDIUM (EM)
^Z	SB	0x1a	26	SUBSTITUTE (SUB)
^[	EC	0x1b	27	ESCAPE (ESC)
^\	FS	0x1c	28	FILE SEPARATOR (IS4)
^]	GS	0x1d	29	GROUP SEPARATOR (IS3)
^^	RS	0x1e	30	RECORD SEPARATOR (IS2)
^_	US	0x1f	31	UNIT SEPARATOR (IS1)
	SP	0x20	32	SPACE
#	Nb	0x23	35	NUMBER SIGN
$	DO	0x24	36	DOLLAR SIGN
@	At	0x40	64	COMMERCIAL AT
[	<(	0x5b	91	LEFT SQUARE BRACKET
\	//	0x5c	92	REVERSE SOLIDUS
]	)>	0x5d	93	RIGHT SQUARE BRACKET
^	'>	0x5e	94	CIRCUMFLEX ACCENT
‘	'!	0x60	96	GRAVE ACCENT
{	(!	0x7b	123	LEFT CURLY BRACKET
|	!!	0x7c	124	VERTICAL LINE
}	!)	0x7d	125	RIGHT CURLY BRACKET
~	'?	0x7e	126	TILDE
^?	DT	0x7f	127	DELETE (DEL)
~@	PA	0x80	128	PADDING CHARACTER (PAD)
~A	HO	0x81	129	HIGH OCTET PRESET (HOP)
~B	BH	0x82	130	BREAK PERMITTED HERE (BPH)
~C	NH	0x83	131	NO BREAK HERE (NBH)
~D	IN	0x84	132	INDEX (IND)
~E	NL	0x85	133	NEXT LINE (NEL)
~F	SA	0x86	134	START OF SELECTED AREA (SSA)
~G	ES	0x87	135	END OF SELECTED AREA (ESA)
~H	HS	0x88	136	CHARACTER TABULATION SET (HTS)
~I	HJ	0x89	137	CHARACTER TABULATION WITH JUSTIFICATION (HTJ)
~J	VS	0x8a	138	LINE TABULATION SET (VTS)
~K	PD	0x8b	139	PARTIAL LINE FORWARD (PLD)
~L	PU	0x8c	140	PARTIAL LINE BACKWARD (PLU)
~M	RI	0x8d	141	REVERSE LINE FEED (RI)
~N	S2	0x8e	142	SINGLE-SHIFT TWO (SS2)
~O	S3	0x8f	143	SINGLE-SHIFT THREE (SS3)
~P	DC	0x90	144	DEVICE CONTROL STRING (DCS)
~Q	P1	0x91	145	PRIVATE USE ONE (PU1)
~R	P2	0x92	146	PRIVATE USE TWO (PU2)
~S	TS	0x93	147	SET TRANSMIT STATE (STS)
~T	CC	0x94	148	CANCEL CHARACTER (CCH)

~U	MW	0x95	149	MESSAGE WAITING (MW)
~V	SG	0x96	150	START OF GUARDED AREA (SPA)
~W	EG	0x97	151	END OF GUARDED AREA (EPA)
~X	SS	0x98	152	START OF STRING (SOS)
~Y	GC	0x99	153	SINGLE GRAPHIC CHARACTER INTRODUCER (SGCI)
~Z	SC	0x9a	154	SINGLE CHARACTER INTRODUCER (SCI)
~[	CI	0x9b	155	CONTROL SEQUENCE INTRODUCER (CSI)
~\	ST	0x9c	156	STRING TERMINATOR (ST)
~]	OC	0x9d	157	OPERATING SYSTEM COMMAND (OSC)
~^	PM	0x9e	158	PRIVACY MESSAGE (PM)
~_	AC	0x9f	159	APPLICATION PROGRAM COMMAND (APC)
|	NS	0xa0	160	NO-BREAK SPACE
	!I	0xa1	161	INVERTED EXCLAMATION MARK
	Ct	0xa2	162	CENT SIGN
	Pd	0xa3	163	POUND SIGN
¤	Cu	0xa4	164	CURRENCY SIGN
	Ye	0xa5	165	YEN SIGN
	BB	0xa6	166	BROKEN BAR
§	SE	0xa7	167	SECTION SIGN
¨	':	0xa8	168	DIAERESIS
	Co	0xa9	169	COPYRIGHT SIGN
	-a	0xaa	170	FEMININE ORDINAL INDICATOR
	<<	0xab	171	LEFT-POINTING DOUBLE ANGLE QUOTATION MARK
	NO	0xac	172	NOT SIGN
	--	0xad	173	SOFT HYPHEN
	Rg	0xae	174	REGISTERED SIGN
	'm	0xaf	175	MACRON
°	DG	0xb0	176	DEGREE SIGN
±	+-	0xb1	177	PLUS-MINUS SIGN
	2S	0xb2	178	SUPERSCRIPT TWO
	3S	0xb3	179	SUPERSCRIPT THREE
	''	0xb4	180	ACUTE ACCENT
	My	0xb5	181	MICRO SIGN
	PI	0xb6	182	PILCROW SIGN
·	.M	0xb7	183	MIDDLE DOT
	',	0xb8	184	CEDILLA
	1S	0xb9	185	SUPERSCRIPT ONE
	-o	0xba	186	MASCULINE ORDINAL INDICATOR
	>>	0xbb	187	RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
	14	0xbc	188	VULGAR FRACTION ONE QUARTER
	12	0xbd	189	VULGAR FRACTION ONE HALF
	34	0xbe	190	VULGAR FRACTION THREE QUARTERS
	?I	0xbf	191	INVERTED QUESTION MARK
	A!	0xc0	192	LATIN CAPITAL LETTER A WITH GRAVE
	A'	0xc1	193	LATIN CAPITAL LETTER A WITH ACUTE
	A>	0xc2	194	LATIN CAPITAL LETTER A WITH CIRCUMFLEX
	A?	0xc3	195	LATIN CAPITAL LETTER A WITH TILDE
	A:	0xc4	196	LATIN CAPITAL LETTER A WITH DIAERESIS
	AA	0xc5	197	LATIN CAPITAL LETTER A WITH RING ABOVE
	AE	0xc6	198	LATIN CAPITAL LETTER AE
	C,	0xc7	199	LATIN CAPITAL LETTER C WITH CEDILLA
	E!	0xc8	200	LATIN CAPITAL LETTER E WITH GRAVE
	E'	0xc9	201	LATIN CAPITAL LETTER E WITH ACUTE
	E>	0xca	202	LATIN CAPITAL LETTER E WITH CIRCUMFLEX

```
      E:      0xcb    203     LATIN CAPITAL LETTER E WITH DIAERESIS
      I!      0xcc    204     LATIN CAPITAL LETTER I WITH GRAVE
      I'      0xcd    205     LATIN CAPITAL LETTER I WITH ACUTE
      I>      0xce    206     LATIN CAPITAL LETTER I WITH CIRCUMFLEX
      I:      0xcf    207     LATIN CAPITAL LETTER I WITH DIAERESIS
      D-      0xd0    208     LATIN CAPITAL LETTER ETH (Icelandic)
      N?      0xd1    209     LATIN CAPITAL LETTER N WITH TILDE
      O!      0xd2    210     LATIN CAPITAL LETTER O WITH GRAVE
      O'      0xd3    211     LATIN CAPITAL LETTER O WITH ACUTE
      O>      0xd4    212     LATIN CAPITAL LETTER O WITH CIRCUMFLEX
      O?      0xd5    213     LATIN CAPITAL LETTER O WITH TILDE
      O:      0xd6    214     LATIN CAPITAL LETTER O WITH DIAERESIS
×       *X      0xd7    215     MULTIPLICATION SIGN
      O/      0xd8    216     LATIN CAPITAL LETTER O WITH STROKE
      U!      0xd9    217     LATIN CAPITAL LETTER U WITH GRAVE
      U'      0xda    218     LATIN CAPITAL LETTER U WITH ACUTE
      U>      0xdb    219     LATIN CAPITAL LETTER U WITH CIRCUMFLEX
      U:      0xdc    220     LATIN CAPITAL LETTER U WITH DIAERESIS
      Y'      0xdd    221     LATIN CAPITAL LETTER Y WITH ACUTE
      TH      0xde    222     LATIN CAPITAL LETTER THORN (Icelandic)
      ss      0xdf    223     LATIN SMALL LETTER SHARP S (German)
à       a!      0xe0    224     LATIN SMALL LETTER A WITH GRAVE
á       a'      0xe1    225     LATIN SMALL LETTER A WITH ACUTE
      a>      0xe2    226     LATIN SMALL LETTER A WITH CIRCUMFLEX
      a?      0xe3    227     LATIN SMALL LETTER A WITH TILDE
      a:      0xe4    228     LATIN SMALL LETTER A WITH DIAERESIS
      aa      0xe5    229     LATIN SMALL LETTER A WITH RING ABOVE
      ae      0xe6    230     LATIN SMALL LETTER AE
      c,      0xe7    231     LATIN SMALL LETTER C WITH CEDILLA
è       e!      0xe8    232     LATIN SMALL LETTER E WITH GRAVE
é       e'      0xe9    233     LATIN SMALL LETTER E WITH ACUTE
ê       e>      0xea    234     LATIN SMALL LETTER E WITH CIRCUMFLEX
      e:      0xeb    235     LATIN SMALL LETTER E WITH DIAERESIS
ì       i!      0xec    236     LATIN SMALL LETTER I WITH GRAVE
í       i'      0xed    237     LATIN SMALL LETTER I WITH ACUTE
      i>      0xee    238     LATIN SMALL LETTER I WITH CIRCUMFLEX
      i:      0xef    239     LATIN SMALL LETTER I WITH DIAERESIS
      d-      0xf0    240     LATIN SMALL LETTER ETH (Icelandic)
      n?      0xf1    241     LATIN SMALL LETTER N WITH TILDE
ò       o!      0xf2    242     LATIN SMALL LETTER O WITH GRAVE
ó       o'      0xf3    243     LATIN SMALL LETTER O WITH ACUTE
      o>      0xf4    244     LATIN SMALL LETTER O WITH CIRCUMFLEX
      o?      0xf5    245     LATIN SMALL LETTER O WITH TILDE
      o:      0xf6    246     LATIN SMALL LETTER O WITH DIAERESIS
÷       -:      0xf7    247     DIVISION SIGN
      o/      0xf8    248     LATIN SMALL LETTER O WITH STROKE
ù       u!      0xf9    249     LATIN SMALL LETTER U WITH GRAVE
ú       u'      0xfa    250     LATIN SMALL LETTER U WITH ACUTE
      u>      0xfb    251     LATIN SMALL LETTER U WITH CIRCUMFLEX
ü       u:      0xfc    252     LATIN SMALL LETTER U WITH DIAERESIS
      y'      0xfd    253     LATIN SMALL LETTER Y WITH ACUTE
      th      0xfe    254     LATIN SMALL LETTER THORN (Icelandic)
      y:      0xff    255     LATIN SMALL LETTER Y WITH DIAERESIS
```

如果 Vim 编译时带有 multibyte 支持而你使用多字节 'encoding'，Vim 提供带有额外二合字母的增强版：

digraph-table-mbyte

字符	二合字母	十六进	十进制	正式名字
	A-	0100	0256	LATIN CAPITAL LETTER A WITH MACRON
ā	a-	0101	0257	LATIN SMALL LETTER A WITH MACRON
	A(	0102	0258	LATIN CAPITAL LETTER A WITH BREVE
	a(	0103	0259	LATIN SMALL LETTER A WITH BREVE
	A;	0104	0260	LATIN CAPITAL LETTER A WITH OGONEK
	a;	0105	0261	LATIN SMALL LETTER A WITH OGONEK
	C'	0106	0262	LATIN CAPITAL LETTER C WITH ACUTE
	c'	0107	0263	LATIN SMALL LETTER C WITH ACUTE
	C>	0108	0264	LATIN CAPITAL LETTER C WITH CIRCUMFLEX
	c>	0109	0265	LATIN SMALL LETTER C WITH CIRCUMFLEX
	C.	010A	0266	LATIN CAPITAL LETTER C WITH DOT ABOVE
	c.	010B	0267	LATIN SMALL LETTER C WITH DOT ABOVE
	C<	010C	0268	LATIN CAPITAL LETTER C WITH CARON
	c<	010D	0269	LATIN SMALL LETTER C WITH CARON
	D<	010E	0270	LATIN CAPITAL LETTER D WITH CARON
	d<	010F	0271	LATIN SMALL LETTER D WITH CARON
	D/	0110	0272	LATIN CAPITAL LETTER D WITH STROKE
	d/	0111	0273	LATIN SMALL LETTER D WITH STROKE
	E-	0112	0274	LATIN CAPITAL LETTER E WITH MACRON
ē	e-	0113	0275	LATIN SMALL LETTER E WITH MACRON
	E(	0114	0276	LATIN CAPITAL LETTER E WITH BREVE
	e(	0115	0277	LATIN SMALL LETTER E WITH BREVE
	E.	0116	0278	LATIN CAPITAL LETTER E WITH DOT ABOVE
	e.	0117	0279	LATIN SMALL LETTER E WITH DOT ABOVE
	E;	0118	0280	LATIN CAPITAL LETTER E WITH OGONEK
	e;	0119	0281	LATIN SMALL LETTER E WITH OGONEK
	E<	011A	0282	LATIN CAPITAL LETTER E WITH CARON
ě	e<	011B	0283	LATIN SMALL LETTER E WITH CARON
	G>	011C	0284	LATIN CAPITAL LETTER G WITH CIRCUMFLEX
	g>	011D	0285	LATIN SMALL LETTER G WITH CIRCUMFLEX
	G(	011E	0286	LATIN CAPITAL LETTER G WITH BREVE
	g(	011F	0287	LATIN SMALL LETTER G WITH BREVE
	G.	0120	0288	LATIN CAPITAL LETTER G WITH DOT ABOVE
	g.	0121	0289	LATIN SMALL LETTER G WITH DOT ABOVE
	G,	0122	0290	LATIN CAPITAL LETTER G WITH CEDILLA
	g,	0123	0291	LATIN SMALL LETTER G WITH CEDILLA
	H>	0124	0292	LATIN CAPITAL LETTER H WITH CIRCUMFLEX
	h>	0125	0293	LATIN SMALL LETTER H WITH CIRCUMFLEX
	H/	0126	0294	LATIN CAPITAL LETTER H WITH STROKE
	h/	0127	0295	LATIN SMALL LETTER H WITH STROKE
	I?	0128	0296	LATIN CAPITAL LETTER I WITH TILDE
	i?	0129	0297	LATIN SMALL LETTER I WITH TILDE
	I-	012A	0298	LATIN CAPITAL LETTER I WITH MACRON
ī	i-	012B	0299	LATIN SMALL LETTER I WITH MACRON
	I(	012C	0300	LATIN CAPITAL LETTER I WITH BREVE
	i(	012D	0301	LATIN SMALL LETTER I WITH BREVE
	I;	012E	0302	LATIN CAPITAL LETTER I WITH OGONEK
	i;	012F	0303	LATIN SMALL LETTER I WITH OGONEK
	I.	0130	0304	LATIN CAPITAL LETTER I WITH DOT ABOVE

```
i.      0131    0305    LATIN SMALL LETTER DOTLESS I
IJ      0132    0306    LATIN CAPITAL LIGATURE IJ
ij      0133    0307    LATIN SMALL LIGATURE IJ
J>      0134    0308    LATIN CAPITAL LETTER J WITH CIRCUMFLEX
j>      0135    0309    LATIN SMALL LETTER J WITH CIRCUMFLEX
K,      0136    0310    LATIN CAPITAL LETTER K WITH CEDILLA
k,      0137    0311    LATIN SMALL LETTER K WITH CEDILLA
kk      0138    0312    LATIN SMALL LETTER KRA
L'      0139    0313    LATIN CAPITAL LETTER L WITH ACUTE
l'      013A    0314    LATIN SMALL LETTER L WITH ACUTE
L,      013B    0315    LATIN CAPITAL LETTER L WITH CEDILLA
l,      013C    0316    LATIN SMALL LETTER L WITH CEDILLA
L<      013D    0317    LATIN CAPITAL LETTER L WITH CARON
l<      013E    0318    LATIN SMALL LETTER L WITH CARON
L.      013F    0319    LATIN CAPITAL LETTER L WITH MIDDLE DOT
l.      0140    0320    LATIN SMALL LETTER L WITH MIDDLE DOT
L/      0141    0321    LATIN CAPITAL LETTER L WITH STROKE
l/      0142    0322    LATIN SMALL LETTER L WITH STROKE
N'      0143    0323    LATIN CAPITAL LETTER N WITH ACUTE ‘
  n'      0144    0324    LATIN SMALL LETTER N WITH ACUTE ‘
N,      0145    0325    LATIN CAPITAL LETTER N WITH CEDILLA ‘
n,      0146    0326    LATIN SMALL LETTER N WITH CEDILLA ‘
N<      0147    0327    LATIN CAPITAL LETTER N WITH CARON ‘
  n<      0148    0328    LATIN SMALL LETTER N WITH CARON ‘
'n      0149    0329    LATIN SMALL LETTER N PRECEDED BY APOSTROPHE ‘
NG      014A    0330    LATIN CAPITAL LETTER ENG
ng      014B    0331    LATIN SMALL LETTER ENG
O-      014C    0332    LATIN CAPITAL LETTER O WITH MACRON
ō       o-      014D    0333    LATIN SMALL LETTER O WITH MACRON
O(      014E    0334    LATIN CAPITAL LETTER O WITH BREVE
o(      014F    0335    LATIN SMALL LETTER O WITH BREVE
O"      0150    0336    LATIN CAPITAL LETTER O WITH DOUBLE ACUTE
o"      0151    0337    LATIN SMALL LETTER O WITH DOUBLE ACUTE
OE      0152    0338    LATIN CAPITAL LIGATURE OE
oe      0153    0339    LATIN SMALL LIGATURE OE
R'      0154    0340    LATIN CAPITAL LETTER R WITH ACUTE
r'      0155    0341    LATIN SMALL LETTER R WITH ACUTE
R,      0156    0342    LATIN CAPITAL LETTER R WITH CEDILLA
r,      0157    0343    LATIN SMALL LETTER R WITH CEDILLA
R<      0158    0344    LATIN CAPITAL LETTER R WITH CARON
r<      0159    0345    LATIN SMALL LETTER R WITH CARON
S'      015A    0346    LATIN CAPITAL LETTER S WITH ACUTE
s'      015B    0347    LATIN SMALL LETTER S WITH ACUTE
S>      015C    0348    LATIN CAPITAL LETTER S WITH CIRCUMFLEX
s>      015D    0349    LATIN SMALL LETTER S WITH CIRCUMFLEX
S,      015E    0350    LATIN CAPITAL LETTER S WITH CEDILLA
s,      015F    0351    LATIN SMALL LETTER S WITH CEDILLA
S<      0160    0352    LATIN CAPITAL LETTER S WITH CARON
s<      0161    0353    LATIN SMALL LETTER S WITH CARON
T,      0162    0354    LATIN CAPITAL LETTER T WITH CEDILLA
t,      0163    0355    LATIN SMALL LETTER T WITH CEDILLA
T<      0164    0356    LATIN CAPITAL LETTER T WITH CARON
t<      0165    0357    LATIN SMALL LETTER T WITH CARON
T/      0166    0358    LATIN CAPITAL LETTER T WITH STROKE
```

t/ 0167 0359 LATIN SMALL LETTER T WITH STROKE
U? 0168 0360 LATIN CAPITAL LETTER U WITH TILDE
u? 0169 0361 LATIN SMALL LETTER U WITH TILDE
U- 016A 0362 LATIN CAPITAL LETTER U WITH MACRON
ū u- 016B 0363 LATIN SMALL LETTER U WITH MACRON
U(016C 0364 LATIN CAPITAL LETTER U WITH BREVE
u(016D 0365 LATIN SMALL LETTER U WITH BREVE
U0 016E 0366 LATIN CAPITAL LETTER U WITH RING ABOVE
u0 016F 0367 LATIN SMALL LETTER U WITH RING ABOVE
U" 0170 0368 LATIN CAPITAL LETTER U WITH DOUBLE ACUTE
u" 0171 0369 LATIN SMALL LETTER U WITH DOUBLE ACUTE
U; 0172 0370 LATIN CAPITAL LETTER U WITH OGONEK
u; 0173 0371 LATIN SMALL LETTER U WITH OGONEK
W> 0174 0372 LATIN CAPITAL LETTER W WITH CIRCUMFLEX
w> 0175 0373 LATIN SMALL LETTER W WITH CIRCUMFLEX
Y> 0176 0374 LATIN CAPITAL LETTER Y WITH CIRCUMFLEX
y> 0177 0375 LATIN SMALL LETTER Y WITH CIRCUMFLEX
Y: 0178 0376 LATIN CAPITAL LETTER Y WITH DIAERESIS
Z' 0179 0377 LATIN CAPITAL LETTER Z WITH ACUTE
z' 017A 0378 LATIN SMALL LETTER Z WITH ACUTE
Z. 017B 0379 LATIN CAPITAL LETTER Z WITH DOT ABOVE
z. 017C 0380 LATIN SMALL LETTER Z WITH DOT ABOVE
Z< 017D 0381 LATIN CAPITAL LETTER Z WITH CARON
z< 017E 0382 LATIN SMALL LETTER Z WITH CARON
O9 01A0 0416 LATIN CAPITAL LETTER O WITH HORN
o9 01A1 0417 LATIN SMALL LETTER O WITH HORN
OI 01A2 0418 LATIN CAPITAL LETTER OI
oi 01A3 0419 LATIN SMALL LETTER OI
yr 01A6 0422 LATIN LETTER YR
U9 01AF 0431 LATIN CAPITAL LETTER U WITH HORN
u9 01B0 0432 LATIN SMALL LETTER U WITH HORN
Z/ 01B5 0437 LATIN CAPITAL LETTER Z WITH STROKE
z/ 01B6 0438 LATIN SMALL LETTER Z WITH STROKE
ED 01B7 0439 LATIN CAPITAL LETTER EZH
A< 01CD 0461 LATIN CAPITAL LETTER A WITH CARON
ǎ a< 01CE 0462 LATIN SMALL LETTER A WITH CARON
I< 01CF 0463 LATIN CAPITAL LETTER I WITH CARON
ǐ i< 01D0 0464 LATIN SMALL LETTER I WITH CARON
O< 01D1 0465 LATIN CAPITAL LETTER O WITH CARON
ǒ o< 01D2 0466 LATIN SMALL LETTER O WITH CARON
U< 01D3 0467 LATIN CAPITAL LETTER U WITH CARON
ǔ u< 01D4 0468 LATIN SMALL LETTER U WITH CARON
A1 01DE 0478 LATIN CAPITAL LETTER A WITH DIAERESIS AND MACRON
a1 01DF 0479 LATIN SMALL LETTER A WITH DIAERESIS AND MACRON
A7 01E0 0480 LATIN CAPITAL LETTER A WITH DOT ABOVE AND MACRON
a7 01E1 0481 LATIN SMALL LETTER A WITH DOT ABOVE AND MACRON
A3 01E2 0482 LATIN CAPITAL LETTER AE WITH MACRON
a3 01E3 0483 LATIN SMALL LETTER AE WITH MACRON
G/ 01E4 0484 LATIN CAPITAL LETTER G WITH STROKE
g/ 01E5 0485 LATIN SMALL LETTER G WITH STROKE
G< 01E6 0486 LATIN CAPITAL LETTER G WITH CARON
g< 01E7 0487 LATIN SMALL LETTER G WITH CARON
K< 01E8 0488 LATIN CAPITAL LETTER K WITH CARON
k< 01E9 0489 LATIN SMALL LETTER K WITH CARON

	O;	01EA	0490	LATIN CAPITAL LETTER O WITH OGONEK
	o;	01EB	0491	LATIN SMALL LETTER O WITH OGONEK
	O1	01EC	0492	LATIN CAPITAL LETTER O WITH OGONEK AND MACRON
	o1	01ED	0493	LATIN SMALL LETTER O WITH OGONEK AND MACRON
	EZ	01EE	0494	LATIN CAPITAL LETTER EZH WITH CARON
	ez	01EF	0495	LATIN SMALL LETTER EZH WITH CARON
	j<	01F0	0496	LATIN SMALL LETTER J WITH CARON
	G’	01F4	0500	LATIN CAPITAL LETTER G WITH ACUTE
	g’	01F5	0501	LATIN SMALL LETTER G WITH ACUTE
	;S	02BF	0703	MODIFIER LETTER LEFT HALF RING
ˇ	’<	02C7	0711	CARON
	’(	02D8	0728	BREVE
	’.	02D9	0729	DOT ABOVE
	’0	02DA	0730	RING ABOVE
	’;	02DB	0731	OGONEK
	’"	02DD	0733	DOUBLE ACUTE ACCENT
	A%	0386	0902	GREEK CAPITAL LETTER ALPHA WITH TONOS
	E%	0388	0904	GREEK CAPITAL LETTER EPSILON WITH TONOS
	Y%	0389	0905	GREEK CAPITAL LETTER ETA WITH TONOS
	I%	038A	0906	GREEK CAPITAL LETTER IOTA WITH TONOS
	O%	038C	0908	GREEK CAPITAL LETTER OMICRON WITH TONOS
	U%	038E	0910	GREEK CAPITAL LETTER UPSILON WITH TONOS
	W%	038F	0911	GREEK CAPITAL LETTER OMEGA WITH TONOS
	i3	0390	0912	GREEK SMALL LETTER IOTA WITH DIALYTIKA AND TONOS
A	A*	0391	0913	GREEK CAPITAL LETTER ALPHA
B	B*	0392	0914	GREEK CAPITAL LETTER BETA
Γ	G*	0393	0915	GREEK CAPITAL LETTER GAMMA
Δ	D*	0394	0916	GREEK CAPITAL LETTER DELTA
E	E*	0395	0917	GREEK CAPITAL LETTER EPSILON
Z	Z*	0396	0918	GREEK CAPITAL LETTER ZETA
H	Y*	0397	0919	GREEK CAPITAL LETTER ETA
Θ	H*	0398	0920	GREEK CAPITAL LETTER THETA
I	I*	0399	0921	GREEK CAPITAL LETTER IOTA
K	K*	039A	0922	GREEK CAPITAL LETTER KAPPA
Λ	L*	039B	0923	GREEK CAPITAL LETTER LAMDA
M	M*	039C	0924	GREEK CAPITAL LETTER MU
N	N*	039D	0925	GREEK CAPITAL LETTER NU
Ξ	C*	039E	0926	GREEK CAPITAL LETTER XI
O	O*	039F	0927	GREEK CAPITAL LETTER OMICRON
Π	P*	03A0	0928	GREEK CAPITAL LETTER PI
P	R*	03A1	0929	GREEK CAPITAL LETTER RHO
Σ	S*	03A3	0931	GREEK CAPITAL LETTER SIGMA
T	T*	03A4	0932	GREEK CAPITAL LETTER TAU
ϒ	U*	03A5	0933	GREEK CAPITAL LETTER UPSILON
Φ	F*	03A6	0934	GREEK CAPITAL LETTER PHI
X	X*	03A7	0935	GREEK CAPITAL LETTER CHI
Ψ	Q*	03A8	0936	GREEK CAPITAL LETTER PSI
Ω	W*	03A9	0937	GREEK CAPITAL LETTER OMEGA
	J*	03AA	0938	GREEK CAPITAL LETTER IOTA WITH DIALYTIKA
	V*	03AB	0939	GREEK CAPITAL LETTER UPSILON WITH DIALYTIKA
	a%	03AC	0940	GREEK SMALL LETTER ALPHA WITH TONOS
	e%	03AD	0941	GREEK SMALL LETTER EPSILON WITH TONOS
	y%	03AE	0942	GREEK SMALL LETTER ETA WITH TONOS
	i%	03AF	0943	GREEK SMALL LETTER IOTA WITH TONOS

	u3	03B0	0944	GREEK SMALL LETTER UPSILON WITH DIALYTIKA AND TONOS
α	a*	03B1	0945	GREEK SMALL LETTER ALPHA
β	b*	03B2	0946	GREEK SMALL LETTER BETA
γ	g*	03B3	0947	GREEK SMALL LETTER GAMMA
δ	d*	03B4	0948	GREEK SMALL LETTER DELTA
ε	e*	03B5	0949	GREEK SMALL LETTER EPSILON
ζ	z*	03B6	0950	GREEK SMALL LETTER ZETA
η	y*	03B7	0951	GREEK SMALL LETTER ETA
θ	h*	03B8	0952	GREEK SMALL LETTER THETA
ι	i*	03B9	0953	GREEK SMALL LETTER IOTA
$\varkappa$	k*	03BA	0954	GREEK SMALL LETTER KAPPA
λ	l*	03BB	0955	GREEK SMALL LETTER LAMDA
μ	m*	03BC	0956	GREEK SMALL LETTER MU
ν	n*	03BD	0957	GREEK SMALL LETTER NU
ξ	c*	03BE	0958	GREEK SMALL LETTER XI
o	o*	03BF	0959	GREEK SMALL LETTER OMICRON
π	p*	03C0	0960	GREEK SMALL LETTER PI
ρ	r*	03C1	0961	GREEK SMALL LETTER RHO
	*s	03C2	0962	GREEK SMALL LETTER FINAL SIGMA
σ	s*	03C3	0963	GREEK SMALL LETTER SIGMA
τ	t*	03C4	0964	GREEK SMALL LETTER TAU
υ	u*	03C5	0965	GREEK SMALL LETTER UPSILON
ϕ	f*	03C6	0966	GREEK SMALL LETTER PHI
$\varkappa$	x*	03C7	0967	GREEK SMALL LETTER CHI
ψ	q*	03C8	0968	GREEK SMALL LETTER PSI
ω	w*	03C9	0969	GREEK SMALL LETTER OMEGA
	j*	03CA	0970	GREEK SMALL LETTER IOTA WITH DIALYTIKA
	v*	03CB	0971	GREEK SMALL LETTER UPSILON WITH DIALYTIKA
	o%	03CC	0972	GREEK SMALL LETTER OMICRON WITH TONOS
	u%	03CD	0973	GREEK SMALL LETTER UPSILON WITH TONOS
	w%	03CE	0974	GREEK SMALL LETTER OMEGA WITH TONOS
	'G	03D8	0984	GREEK LETTER ARCHAIC KOPPA
	,G	03D9	0985	GREEK SMALL LETTER ARCHAIC KOPPA
	T3	03DA	0986	GREEK LETTER STIGMA
	t3	03DB	0987	GREEK SMALL LETTER STIGMA
	M3	03DC	0988	GREEK LETTER DIGAMMA
	m3	03DD	0989	GREEK SMALL LETTER DIGAMMA
	K3	03DE	0990	GREEK LETTER KOPPA
	k3	03DF	0991	GREEK SMALL LETTER KOPPA
	P3	03E0	0992	GREEK LETTER SAMPI
	p3	03E1	0993	GREEK SMALL LETTER SAMPI
	'%	03F4	1012	GREEK CAPITAL THETA SYMBOL
	j3	03F5	1013	GREEK LUNATE EPSILON SYMBOL
Ë	IO	0401	1025	CYRILLIC CAPITAL LETTER IO
	D%	0402	1026	CYRILLIC CAPITAL LETTER DJE
	G%	0403	1027	CYRILLIC CAPITAL LETTER GJE
	IE	0404	1028	CYRILLIC CAPITAL LETTER UKRAINIAN IE
	DS	0405	1029	CYRILLIC CAPITAL LETTER DZE
	II	0406	1030	CYRILLIC CAPITAL LETTER BYELORUSSIAN-UKRAINIAN I
	YI	0407	1031	CYRILLIC CAPITAL LETTER YI
	J%	0408	1032	CYRILLIC CAPITAL LETTER JE
	LJ	0409	1033	CYRILLIC CAPITAL LETTER LJE
	NJ	040A	1034	CYRILLIC CAPITAL LETTER NJE
	Ts	040B	1035	CYRILLIC CAPITAL LETTER TSHE

	KJ	040C	1036	CYRILLIC CAPITAL LETTER KJE
	V%	040E	1038	CYRILLIC CAPITAL LETTER SHORT U
	DZ	040F	1039	CYRILLIC CAPITAL LETTER DZHE
А	A=	0410	1040	CYRILLIC CAPITAL LETTER A
Б	B=	0411	1041	CYRILLIC CAPITAL LETTER BE
В	V=	0412	1042	CYRILLIC CAPITAL LETTER VE
Г	G=	0413	1043	CYRILLIC CAPITAL LETTER GHE
Д	D=	0414	1044	CYRILLIC CAPITAL LETTER DE
Е	E=	0415	1045	CYRILLIC CAPITAL LETTER IE
Ж	Z%	0416	1046	CYRILLIC CAPITAL LETTER ZHE
З	Z=	0417	1047	CYRILLIC CAPITAL LETTER ZE
И	I=	0418	1048	CYRILLIC CAPITAL LETTER I
Й	J=	0419	1049	CYRILLIC CAPITAL LETTER SHORT I
К	K=	041A	1050	CYRILLIC CAPITAL LETTER KA
Л	L=	041B	1051	CYRILLIC CAPITAL LETTER EL
М	M=	041C	1052	CYRILLIC CAPITAL LETTER EM
Н	N=	041D	1053	CYRILLIC CAPITAL LETTER EN
О	O=	041E	1054	CYRILLIC CAPITAL LETTER O
П	P=	041F	1055	CYRILLIC CAPITAL LETTER PE
Р	R=	0420	1056	CYRILLIC CAPITAL LETTER ER
С	S=	0421	1057	CYRILLIC CAPITAL LETTER ES
Т	T=	0422	1058	CYRILLIC CAPITAL LETTER TE
У	U=	0423	1059	CYRILLIC CAPITAL LETTER U
Ф	F=	0424	1060	CYRILLIC CAPITAL LETTER EF
Х	H=	0425	1061	CYRILLIC CAPITAL LETTER HA
Ц	C=	0426	1062	CYRILLIC CAPITAL LETTER TSE
Ч	C%	0427	1063	CYRILLIC CAPITAL LETTER CHE
Ш	S%	0428	1064	CYRILLIC CAPITAL LETTER SHA
Щ	Sc	0429	1065	CYRILLIC CAPITAL LETTER SHCHA
Ъ	="	042A	1066	CYRILLIC CAPITAL LETTER HARD SIGN
Ы	Y=	042B	1067	CYRILLIC CAPITAL LETTER YERU
Ь	%"	042C	1068	CYRILLIC CAPITAL LETTER SOFT SIGN
Э	JE	042D	1069	CYRILLIC CAPITAL LETTER E
Ю	JU	042E	1070	CYRILLIC CAPITAL LETTER YU
Я	JA	042F	1071	CYRILLIC CAPITAL LETTER YA
а	a=	0430	1072	CYRILLIC SMALL LETTER A
б	b=	0431	1073	CYRILLIC SMALL LETTER BE
в	v=	0432	1074	CYRILLIC SMALL LETTER VE
г	g=	0433	1075	CYRILLIC SMALL LETTER GHE
д	d=	0434	1076	CYRILLIC SMALL LETTER DE
е	e=	0435	1077	CYRILLIC SMALL LETTER IE
ж	z%	0436	1078	CYRILLIC SMALL LETTER ZHE
з	z=	0437	1079	CYRILLIC SMALL LETTER ZE
и	i=	0438	1080	CYRILLIC SMALL LETTER I
й	j=	0439	1081	CYRILLIC SMALL LETTER SHORT I
к	k=	043A	1082	CYRILLIC SMALL LETTER KA
л	l=	043B	1083	CYRILLIC SMALL LETTER EL
м	m=	043C	1084	CYRILLIC SMALL LETTER EM
н	n=	043D	1085	CYRILLIC SMALL LETTER EN
о	o=	043E	1086	CYRILLIC SMALL LETTER O
п	p=	043F	1087	CYRILLIC SMALL LETTER PE
р	r=	0440	1088	CYRILLIC SMALL LETTER ER
с	s=	0441	1089	CYRILLIC SMALL LETTER ES
т	t=	0442	1090	CYRILLIC SMALL LETTER TE

у	u=	0443	1091	CYRILLIC SMALL LETTER U
ф	f=	0444	1092	CYRILLIC SMALL LETTER EF
х	h=	0445	1093	CYRILLIC SMALL LETTER HA
ц	c=	0446	1094	CYRILLIC SMALL LETTER TSE
ч	c%	0447	1095	CYRILLIC SMALL LETTER CHE
ш	s%	0448	1096	CYRILLIC SMALL LETTER SHA
щ	sc	0449	1097	CYRILLIC SMALL LETTER SHCHA
ъ	=’	044A	1098	CYRILLIC SMALL LETTER HARD SIGN
ы	y=	044B	1099	CYRILLIC SMALL LETTER YERU
ь	%’	044C	1100	CYRILLIC SMALL LETTER SOFT SIGN
э	je	044D	1101	CYRILLIC SMALL LETTER E
ю	ju	044E	1102	CYRILLIC SMALL LETTER YU
я	ja	044F	1103	CYRILLIC SMALL LETTER YA
ё	io	0451	1105	CYRILLIC SMALL LETTER IO
	d%	0452	1106	CYRILLIC SMALL LETTER DJE
	g%	0453	1107	CYRILLIC SMALL LETTER GJE
	ie	0454	1108	CYRILLIC SMALL LETTER UKRAINIAN IE
	ds	0455	1109	CYRILLIC SMALL LETTER DZE
	ii	0456	1110	CYRILLIC SMALL LETTER BYELORUSSIAN-UKRAINIAN I
	yi	0457	1111	CYRILLIC SMALL LETTER YI
	j%	0458	1112	CYRILLIC SMALL LETTER JE
	lj	0459	1113	CYRILLIC SMALL LETTER LJE
	nj	045A	1114	CYRILLIC SMALL LETTER NJE
	ts	045B	1115	CYRILLIC SMALL LETTER TSHE
	kj	045C	1116	CYRILLIC SMALL LETTER KJE
	v%	045E	1118	CYRILLIC SMALL LETTER SHORT U
	dz	045F	1119	CYRILLIC SMALL LETTER DZHE
	Y3	0462	1122	CYRILLIC CAPITAL LETTER YAT
	y3	0463	1123	CYRILLIC SMALL LETTER YAT
	O3	046A	1130	CYRILLIC CAPITAL LETTER BIG YUS
	o3	046B	1131	CYRILLIC SMALL LETTER BIG YUS
	F3	0472	1138	CYRILLIC CAPITAL LETTER FITA
	f3	0473	1139	CYRILLIC SMALL LETTER FITA
	V3	0474	1140	CYRILLIC CAPITAL LETTER IZHITSA
	v3	0475	1141	CYRILLIC SMALL LETTER IZHITSA
	C3	0480	1152	CYRILLIC CAPITAL LETTER KOPPA
	c3	0481	1153	CYRILLIC SMALL LETTER KOPPA
	G3	0490	1168	CYRILLIC CAPITAL LETTER GHE WITH UPTURN
	g3	0491	1169	CYRILLIC SMALL LETTER GHE WITH UPTURN
	A+	05D0	1488	HEBREW LETTER ALEF
	B+	05D1	1489	HEBREW LETTER BET
	G+	05D2	1490	HEBREW LETTER GIMEL
	D+	05D3	1491	HEBREW LETTER DALET
	H+	05D4	1492	HEBREW LETTER HE
	W+	05D5	1493	HEBREW LETTER VAV
	Z+	05D6	1494	HEBREW LETTER ZAYIN
	X+	05D7	1495	HEBREW LETTER HET
	Tj	05D8	1496	HEBREW LETTER TET
	J+	05D9	1497	HEBREW LETTER YOD
	K%	05DA	1498	HEBREW LETTER FINAL KAF
	K+	05DB	1499	HEBREW LETTER KAF
	L+	05DC	1500	HEBREW LETTER LAMED
	M%	05DD	1501	HEBREW LETTER FINAL MEM
	M+	05DE	1502	HEBREW LETTER MEM

```
N%      05DF    1503    HEBREW LETTER FINAL NUN ‘
N+      05E0    1504    HEBREW LETTER NUN ‘
S+      05E1    1505    HEBREW LETTER SAMEKH
E+      05E2    1506    HEBREW LETTER AYIN
P%      05E3    1507    HEBREW LETTER FINAL PE
P+      05E4    1508    HEBREW LETTER PE
Zj      05E5    1509    HEBREW LETTER FINAL TSADI
ZJ      05E6    1510    HEBREW LETTER TSADI
Q+      05E7    1511    HEBREW LETTER QOF
R+      05E8    1512    HEBREW LETTER RESH
Sh      05E9    1513    HEBREW LETTER SHIN
T+      05EA    1514    HEBREW LETTER TAV
,+      060C    1548    ARABIC COMMA
;+      061B    1563    ARABIC SEMICOLON
?+      061F    1567    ARABIC QUESTION MARK
H’      0621    1569    ARABIC LETTER HAMZA
aM      0622    1570    ARABIC LETTER ALEF WITH MADDA ABOVE
aH      0623    1571    ARABIC LETTER ALEF WITH HAMZA ABOVE
wH      0624    1572    ARABIC LETTER WAW WITH HAMZA ABOVE
ah      0625    1573    ARABIC LETTER ALEF WITH HAMZA BELOW
yH      0626    1574    ARABIC LETTER YEH WITH HAMZA ABOVE
a+      0627    1575    ARABIC LETTER ALEF
b+      0628    1576    ARABIC LETTER BEH
tm      0629    1577    ARABIC LETTER TEH MARBUTA
t+      062A    1578    ARABIC LETTER TEH
tk      062B    1579    ARABIC LETTER THEH
g+      062C    1580    ARABIC LETTER JEEM
hk      062D    1581    ARABIC LETTER HAH
x+      062E    1582    ARABIC LETTER KHAH
d+      062F    1583    ARABIC LETTER DAL
dk      0630    1584    ARABIC LETTER THAL
r+      0631    1585    ARABIC LETTER REH
z+      0632    1586    ARABIC LETTER ZAIN
s+      0633    1587    ARABIC LETTER SEEN
sn      0634    1588    ARABIC LETTER SHEEN
c+      0635    1589    ARABIC LETTER SAD
dd      0636    1590    ARABIC LETTER DAD
tj      0637    1591    ARABIC LETTER TAH
zH      0638    1592    ARABIC LETTER ZAH
e+      0639    1593    ARABIC LETTER AIN
i+      063A    1594    ARABIC LETTER GHAIN
++      0640    1600    ARABIC TATWEEL
f+      0641    1601    ARABIC LETTER FEH
q+      0642    1602    ARABIC LETTER QAF
k+      0643    1603    ARABIC LETTER KAF
l+      0644    1604    ARABIC LETTER LAM
m+      0645    1605    ARABIC LETTER MEEM
n+      0646    1606    ARABIC LETTER NOON
h+      0647    1607    ARABIC LETTER HEH
w+      0648    1608    ARABIC LETTER WAW
j+      0649    1609    ARABIC LETTER ALEF MAKSURA
y+      064A    1610    ARABIC LETTER YEH
:+      064B    1611    ARABIC FATHATAN
"+      064C    1612    ARABIC DAMMATAN
```

```
=+      064D    1613    ARABIC KASRATAN
/+      064E    1614    ARABIC FATHA
’+      064F    1615    ARABIC DAMMA
1+      0650    1616    ARABIC KASRA
3+      0651    1617    ARABIC SHADDA
0+      0652    1618    ARABIC SUKUN
aS      0670    1648    ARABIC LETTER SUPERSCRIPT ALEF
p+      067E    1662    ARABIC LETTER PEH
v+      06A4    1700    ARABIC LETTER VEH
gf      06AF    1711    ARABIC LETTER GAF
0a      06F0    1776    EXTENDED ARABIC-INDIC DIGIT ZERO
1a      06F1    1777    EXTENDED ARABIC-INDIC DIGIT ONE
2a      06F2    1778    EXTENDED ARABIC-INDIC DIGIT TWO
3a      06F3    1779    EXTENDED ARABIC-INDIC DIGIT THREE
4a      06F4    1780    EXTENDED ARABIC-INDIC DIGIT FOUR
5a      06F5    1781    EXTENDED ARABIC-INDIC DIGIT FIVE
6a      06F6    1782    EXTENDED ARABIC-INDIC DIGIT SIX
7a      06F7    1783    EXTENDED ARABIC-INDIC DIGIT SEVEN
8a      06F8    1784    EXTENDED ARABIC-INDIC DIGIT EIGHT
9a      06F9    1785    EXTENDED ARABIC-INDIC DIGIT NINE
B.      1E02    7682    LATIN CAPITAL LETTER B WITH DOT ABOVE
b.      1E03    7683    LATIN SMALL LETTER B WITH DOT ABOVE
B_      1E06    7686    LATIN CAPITAL LETTER B WITH LINE BELOW
b_      1E07    7687    LATIN SMALL LETTER B WITH LINE BELOW
D.      1E0A    7690    LATIN CAPITAL LETTER D WITH DOT ABOVE
d.      1E0B    7691    LATIN SMALL LETTER D WITH DOT ABOVE
D_      1E0E    7694    LATIN CAPITAL LETTER D WITH LINE BELOW
d_      1E0F    7695    LATIN SMALL LETTER D WITH LINE BELOW
D,      1E10    7696    LATIN CAPITAL LETTER D WITH CEDILLA
d,      1E11    7697    LATIN SMALL LETTER D WITH CEDILLA
F.      1E1E    7710    LATIN CAPITAL LETTER F WITH DOT ABOVE
f.      1E1F    7711    LATIN SMALL LETTER F WITH DOT ABOVE
G-      1E20    7712    LATIN CAPITAL LETTER G WITH MACRON
g-      1E21    7713    LATIN SMALL LETTER G WITH MACRON
H.      1E22    7714    LATIN CAPITAL LETTER H WITH DOT ABOVE
h.      1E23    7715    LATIN SMALL LETTER H WITH DOT ABOVE
H:      1E26    7718    LATIN CAPITAL LETTER H WITH DIAERESIS
h:      1E27    7719    LATIN SMALL LETTER H WITH DIAERESIS
H,      1E28    7720    LATIN CAPITAL LETTER H WITH CEDILLA
h,      1E29    7721    LATIN SMALL LETTER H WITH CEDILLA
K’      1E30    7728    LATIN CAPITAL LETTER K WITH ACUTE
k’      1E31    7729    LATIN SMALL LETTER K WITH ACUTE
K_      1E34    7732    LATIN CAPITAL LETTER K WITH LINE BELOW
k_      1E35    7733    LATIN SMALL LETTER K WITH LINE BELOW
L_      1E3A    7738    LATIN CAPITAL LETTER L WITH LINE BELOW
l_      1E3B    7739    LATIN SMALL LETTER L WITH LINE BELOW
M’      1E3E    7742    LATIN CAPITAL LETTER M WITH ACUTE
m’      1E3F    7743    LATIN SMALL LETTER M WITH ACUTE
M.      1E40    7744    LATIN CAPITAL LETTER M WITH DOT ABOVE
m.      1E41    7745    LATIN SMALL LETTER M WITH DOT ABOVE
N.      1E44    7748    LATIN CAPITAL LETTER N WITH DOT ABOVE ‘
n.      1E45    7749    LATIN SMALL LETTER N WITH DOT ABOVE ‘
N_      1E48    7752    LATIN CAPITAL LETTER N WITH LINE BELOW ‘
n_      1E49    7753    LATIN SMALL LETTER N WITH LINE BELOW ‘
```

```
P'      1E54    7764    LATIN CAPITAL LETTER P WITH ACUTE
p'      1E55    7765    LATIN SMALL LETTER P WITH ACUTE
P.      1E56    7766    LATIN CAPITAL LETTER P WITH DOT ABOVE
p.      1E57    7767    LATIN SMALL LETTER P WITH DOT ABOVE
R.      1E58    7768    LATIN CAPITAL LETTER R WITH DOT ABOVE
r.      1E59    7769    LATIN SMALL LETTER R WITH DOT ABOVE
R_      1E5E    7774    LATIN CAPITAL LETTER R WITH LINE BELOW
r_      1E5F    7775    LATIN SMALL LETTER R WITH LINE BELOW
S.      1E60    7776    LATIN CAPITAL LETTER S WITH DOT ABOVE
s.      1E61    7777    LATIN SMALL LETTER S WITH DOT ABOVE
T.      1E6A    7786    LATIN CAPITAL LETTER T WITH DOT ABOVE
t.      1E6B    7787    LATIN SMALL LETTER T WITH DOT ABOVE
T_      1E6E    7790    LATIN CAPITAL LETTER T WITH LINE BELOW
t_      1E6F    7791    LATIN SMALL LETTER T WITH LINE BELOW
V?      1E7C    7804    LATIN CAPITAL LETTER V WITH TILDE
v?      1E7D    7805    LATIN SMALL LETTER V WITH TILDE
W!      1E80    7808    LATIN CAPITAL LETTER W WITH GRAVE
w!      1E81    7809    LATIN SMALL LETTER W WITH GRAVE
W'      1E82    7810    LATIN CAPITAL LETTER W WITH ACUTE
w'      1E83    7811    LATIN SMALL LETTER W WITH ACUTE
W:      1E84    7812    LATIN CAPITAL LETTER W WITH DIAERESIS
w:      1E85    7813    LATIN SMALL LETTER W WITH DIAERESIS
W.      1E86    7814    LATIN CAPITAL LETTER W WITH DOT ABOVE
w.      1E87    7815    LATIN SMALL LETTER W WITH DOT ABOVE
X.      1E8A    7818    LATIN CAPITAL LETTER X WITH DOT ABOVE
x.      1E8B    7819    LATIN SMALL LETTER X WITH DOT ABOVE
X:      1E8C    7820    LATIN CAPITAL LETTER X WITH DIAERESIS
x:      1E8D    7821    LATIN SMALL LETTER X WITH DIAERESIS
Y.      1E8E    7822    LATIN CAPITAL LETTER Y WITH DOT ABOVE
y.      1E8F    7823    LATIN SMALL LETTER Y WITH DOT ABOVE
Z>      1E90    7824    LATIN CAPITAL LETTER Z WITH CIRCUMFLEX
z>      1E91    7825    LATIN SMALL LETTER Z WITH CIRCUMFLEX
Z_      1E94    7828    LATIN CAPITAL LETTER Z WITH LINE BELOW
z_      1E95    7829    LATIN SMALL LETTER Z WITH LINE BELOW
h_      1E96    7830    LATIN SMALL LETTER H WITH LINE BELOW
t:      1E97    7831    LATIN SMALL LETTER T WITH DIAERESIS
w0      1E98    7832    LATIN SMALL LETTER W WITH RING ABOVE
y0      1E99    7833    LATIN SMALL LETTER Y WITH RING ABOVE
A2      1EA2    7842    LATIN CAPITAL LETTER A WITH HOOK ABOVE
a2      1EA3    7843    LATIN SMALL LETTER A WITH HOOK ABOVE
E2      1EBA    7866    LATIN CAPITAL LETTER E WITH HOOK ABOVE
e2      1EBB    7867    LATIN SMALL LETTER E WITH HOOK ABOVE
E?      1EBC    7868    LATIN CAPITAL LETTER E WITH TILDE
e?      1EBD    7869    LATIN SMALL LETTER E WITH TILDE
I2      1EC8    7880    LATIN CAPITAL LETTER I WITH HOOK ABOVE
i2      1EC9    7881    LATIN SMALL LETTER I WITH HOOK ABOVE
O2      1ECE    7886    LATIN CAPITAL LETTER O WITH HOOK ABOVE
o2      1ECF    7887    LATIN SMALL LETTER O WITH HOOK ABOVE
U2      1EE6    7910    LATIN CAPITAL LETTER U WITH HOOK ABOVE
u2      1EE7    7911    LATIN SMALL LETTER U WITH HOOK ABOVE
Y!      1EF2    7922    LATIN CAPITAL LETTER Y WITH GRAVE
y!      1EF3    7923    LATIN SMALL LETTER Y WITH GRAVE
Y2      1EF6    7926    LATIN CAPITAL LETTER Y WITH HOOK ABOVE
y2      1EF7    7927    LATIN SMALL LETTER Y WITH HOOK ABOVE
```

	Y?	1EF8	7928	LATIN CAPITAL LETTER Y WITH TILDE
	y?	1EF9	7929	LATIN SMALL LETTER Y WITH TILDE
	;’	1F00	7936	GREEK SMALL LETTER ALPHA WITH PSILI
	,’	1F01	7937	GREEK SMALL LETTER ALPHA WITH DASIA
	;!	1F02	7938	GREEK SMALL LETTER ALPHA WITH PSILI AND VARIA
	,!	1F03	7939	GREEK SMALL LETTER ALPHA WITH DASIA AND VARIA
	?;	1F04	7940	GREEK SMALL LETTER ALPHA WITH PSILI AND OXIA
	?,	1F05	7941	GREEK SMALL LETTER ALPHA WITH DASIA AND OXIA
	!:	1F06	7942	GREEK SMALL LETTER ALPHA WITH PSILI AND PERISPOMENI
	?:	1F07	7943	GREEK SMALL LETTER ALPHA WITH DASIA AND PERISPOMENI
	1N	2002	8194	EN SPACE
	1M	2003	8195	EM SPACE
	3M	2004	8196	THREE-PER-EM SPACE
	4M	2005	8197	FOUR-PER-EM SPACE
	6M	2006	8198	SIX-PER-EM SPACE
	1T	2009	8201	THIN SPACE
	1H	200A	8202	HAIR SPACE
	-1	2010	8208	HYPHEN
	-N	2013	8211	EN DASH ‘
—	-M	2014	8212	EM DASH
	-3	2015	8213	HORIZONTAL BAR
‖	!2	2016	8214	DOUBLE VERTICAL LINE
	=2	2017	8215	DOUBLE LOW LINE
‘	’6	2018	8216	LEFT SINGLE QUOTATION MARK
’	’9	2019	8217	RIGHT SINGLE QUOTATION MARK
	.9	201A	8218	SINGLE LOW-9 QUOTATION MARK
	9’	201B	8219	SINGLE HIGH-REVERSED-9 QUOTATION MARK
“	"6	201C	8220	LEFT DOUBLE QUOTATION MARK
”	"9	201D	8221	RIGHT DOUBLE QUOTATION MARK
	:9	201E	8222	DOUBLE LOW-9 QUOTATION MARK
	9"	201F	8223	DOUBLE HIGH-REVERSED-9 QUOTATION MARK
	/-	2020	8224	DAGGER
	/=	2021	8225	DOUBLE DAGGER
	..	2025	8229	TWO DOT LEADER
…	,.	2026	8230	HORIZONTAL ELLIPSIS
‰	%0	2030	8240	PER MILLE SIGN
′	1’	2032	8242	PRIME
″	2’	2033	8243	DOUBLE PRIME
	3’	2034	8244	TRIPLE PRIME
	1"	2035	8245	REVERSED PRIME
	2"	2036	8246	REVERSED DOUBLE PRIME
	3"	2037	8247	REVERSED TRIPLE PRIME
	Ca	2038	8248	CARET
	<1	2039	8249	SINGLE LEFT-POINTING ANGLE QUOTATION MARK
	>1	203A	8250	SINGLE RIGHT-POINTING ANGLE QUOTATION MARK
※	:X	203B	8251	REFERENCE MARK
	’-	203E	8254	OVERLINE
	/f	2044	8260	FRACTION SLASH
	0S	2070	8304	SUPERSCRIPT ZERO
	4S	2074	8308	SUPERSCRIPT FOUR
	5S	2075	8309	SUPERSCRIPT FIVE
	6S	2076	8310	SUPERSCRIPT SIX
	7S	2077	8311	SUPERSCRIPT SEVEN
	8S	2078	8312	SUPERSCRIPT EIGHT

	9S	2079	8313	SUPERSCRIPT NINE
	+S	207A	8314	SUPERSCRIPT PLUS SIGN
	-S	207B	8315	SUPERSCRIPT MINUS
	=S	207C	8316	SUPERSCRIPT EQUALS SIGN
	(S	207D	8317	SUPERSCRIPT LEFT PARENTHESIS
	)S	207E	8318	SUPERSCRIPT RIGHT PARENTHESIS
	nS	207F	8319	SUPERSCRIPT LATIN SMALL LETTER N ‘
	0s	2080	8320	SUBSCRIPT ZERO
	1s	2081	8321	SUBSCRIPT ONE
	2s	2082	8322	SUBSCRIPT TWO
	3s	2083	8323	SUBSCRIPT THREE
	4s	2084	8324	SUBSCRIPT FOUR
	5s	2085	8325	SUBSCRIPT FIVE
	6s	2086	8326	SUBSCRIPT SIX
	7s	2087	8327	SUBSCRIPT SEVEN
	8s	2088	8328	SUBSCRIPT EIGHT
	9s	2089	8329	SUBSCRIPT NINE
	+s	208A	8330	SUBSCRIPT PLUS SIGN
	-s	208B	8331	SUBSCRIPT MINUS
	=s	208C	8332	SUBSCRIPT EQUALS SIGN
	(s	208D	8333	SUBSCRIPT LEFT PARENTHESIS
	)s	208E	8334	SUBSCRIPT RIGHT PARENTHESIS
	Li	20A4	8356	LIRA SIGN
	Pt	20A7	8359	PESETA SIGN
	W=	20A9	8361	WON SIGN
	Eu	20AC	8364	EURO SIGN
	=R	20BD	8381	ROUBLE SIGN
	=P	20BD	8381	ROUBLE SIGN
℃	oC	2103	8451	DEGREE CELSIUS
	co	2105	8453	CARE OF
	oF	2109	8457	DEGREE FAHRENHEIT
№	NO	2116	8470	NUMERO SIGN
	PO	2117	8471	SOUND RECORDING COPYRIGHT
	Rx	211E	8478	PRESCRIPTION TAKE
	SM	2120	8480	SERVICE MARK
	TM	2122	8482	TRADE MARK SIGN
	Om	2126	8486	OHM SIGN
	AO	212B	8491	ANGSTROM SIGN
	13	2153	8531	VULGAR FRACTION ONE THIRD
	23	2154	8532	VULGAR FRACTION TWO THIRDS
	15	2155	8533	VULGAR FRACTION ONE FIFTH
	25	2156	8534	VULGAR FRACTION TWO FIFTHS
	35	2157	8535	VULGAR FRACTION THREE FIFTHS
	45	2158	8536	VULGAR FRACTION FOUR FIFTHS
	16	2159	8537	VULGAR FRACTION ONE SIXTH
	56	215A	8538	VULGAR FRACTION FIVE SIXTHS
	18	215B	8539	VULGAR FRACTION ONE EIGHTH
	38	215C	8540	VULGAR FRACTION THREE EIGHTHS
	58	215D	8541	VULGAR FRACTION FIVE EIGHTHS
	78	215E	8542	VULGAR FRACTION SEVEN EIGHTHS
Ⅰ	1R	2160	8544	ROMAN NUMERAL ONE
Ⅱ	2R	2161	8545	ROMAN NUMERAL TWO
Ⅲ	3R	2162	8546	ROMAN NUMERAL THREE
Ⅳ	4R	2163	8547	ROMAN NUMERAL FOUR

Ⅴ	5R	2164	8548	ROMAN NUMERAL FIVE
Ⅵ	6R	2165	8549	ROMAN NUMERAL SIX
Ⅶ	7R	2166	8550	ROMAN NUMERAL SEVEN
Ⅷ	8R	2167	8551	ROMAN NUMERAL EIGHT
Ⅸ	9R	2168	8552	ROMAN NUMERAL NINE
Ⅹ	aR	2169	8553	ROMAN NUMERAL TEN
Ⅺ	bR	216A	8554	ROMAN NUMERAL ELEVEN
Ⅻ	cR	216B	8555	ROMAN NUMERAL TWELVE
	1r	2170	8560	SMALL ROMAN NUMERAL ONE
	2r	2171	8561	SMALL ROMAN NUMERAL TWO
	3r	2172	8562	SMALL ROMAN NUMERAL THREE
	4r	2173	8563	SMALL ROMAN NUMERAL FOUR
	5r	2174	8564	SMALL ROMAN NUMERAL FIVE
	6r	2175	8565	SMALL ROMAN NUMERAL SIX
	7r	2176	8566	SMALL ROMAN NUMERAL SEVEN
	8r	2177	8567	SMALL ROMAN NUMERAL EIGHT
	9r	2178	8568	SMALL ROMAN NUMERAL NINE
	ar	2179	8569	SMALL ROMAN NUMERAL TEN
	br	217A	8570	SMALL ROMAN NUMERAL ELEVEN
	cr	217B	8571	SMALL ROMAN NUMERAL TWELVE
←	<-	2190	8592	LEFTWARDS ARROW
↑	-!	2191	8593	UPWARDS ARROW
→	->	2192	8594	RIGHTWARDS ARROW
↓	-v	2193	8595	DOWNWARDS ARROW
	<>	2194	8596	LEFT RIGHT ARROW
	UD	2195	8597	UP DOWN ARROW
	<=	21D0	8656	LEFTWARDS DOUBLE ARROW
	=>	21D2	8658	RIGHTWARDS DOUBLE ARROW
	==	21D4	8660	LEFT RIGHT DOUBLE ARROW
	FA	2200	8704	FOR ALL
	dP	2202	8706	PARTIAL DIFFERENTIAL
	TE	2203	8707	THERE EXISTS
	/0	2205	8709	EMPTY SET
	DE	2206	8710	INCREMENT
	NB	2207	8711	NABLA
∈	(-	2208	8712	ELEMENT OF
	-)	220B	8715	CONTAINS AS MEMBER
∏	*P	220F	8719	N-ARY PRODUCT ‘
∑	+Z	2211	8721	N-ARY SUMMATION ‘
	-2	2212	8722	MINUS SIGN
	-+	2213	8723	MINUS-OR-PLUS SIGN
	*-	2217	8727	ASTERISK OPERATOR
	Ob	2218	8728	RING OPERATOR
	Sb	2219	8729	BULLET OPERATOR
√	RT	221A	8730	SQUARE ROOT
∝	0(	221D	8733	PROPORTIONAL TO
∞	00	221E	8734	INFINITY
	-L	221F	8735	RIGHT ANGLE
∠	-V	2220	8736	ANGLE
//	PP	2225	8741	PARALLEL TO
∧	AN	2227	8743	LOGICAL AND
∨	OR	2228	8744	LOGICAL OR
∩	(U	2229	8745	INTERSECTION
∪	)U	222A	8746	UNION

∫	In	222B	8747	INTEGRAL
	DI	222C	8748	DOUBLE INTEGRAL
∮	Io	222E	8750	CONTOUR INTEGRAL
∴	.:	2234	8756	THEREFORE
∵	:.	2235	8757	BECAUSE
∶	:R	2236	8758	RATIO
∷	::	2237	8759	PROPORTION
	?1	223C	8764	TILDE OPERATOR
	CG	223E	8766	INVERTED LAZY S
	?-	2243	8771	ASYMPTOTICALLY EQUAL TO
	?=	2245	8773	APPROXIMATELY EQUAL TO
≈	?2	2248	8776	ALMOST EQUAL TO
≌	=?	224C	8780	ALL EQUAL TO
	HI	2253	8787	IMAGE OF OR APPROXIMATELY EQUAL TO
≠	!=	2260	8800	NOT EQUAL TO
≡	=3	2261	8801	IDENTICAL TO
≤	=<	2264	8804	LESS-THAN OR EQUAL TO
≥	>=	2265	8805	GREATER-THAN OR EQUAL TO
	<*	226A	8810	MUCH LESS-THAN
	*>	226B	8811	MUCH GREATER-THAN
≮	!<	226E	8814	NOT LESS-THAN
≯	!>	226F	8815	NOT GREATER-THAN
	(C	2282	8834	SUBSET OF
	)C	2283	8835	SUPERSET OF
	(_	2286	8838	SUBSET OF OR EQUAL TO
	)_	2287	8839	SUPERSET OF OR EQUAL TO
⊙	0.	2299	8857	CIRCLED DOT OPERATOR
	02	229A	8858	CIRCLED RING OPERATOR
⊥	-T	22A5	8869	UP TACK
	.P	22C5	8901	DOT OPERATOR
	:3	22EE	8942	VERTICAL ELLIPSIS
	.3	22EF	8943	MIDLINE HORIZONTAL ELLIPSIS
	Eh	2302	8962	HOUSE
	<7	2308	8968	LEFT CEILING
	>7	2309	8969	RIGHT CEILING
	7<	230A	8970	LEFT FLOOR
	7>	230B	8971	RIGHT FLOOR
	NI	2310	8976	REVERSED NOT SIGN
⌒	(A	2312	8978	ARC
	TR	2315	8981	TELEPHONE RECORDER
	Iu	2320	8992	TOP HALF INTEGRAL
	Il	2321	8993	BOTTOM HALF INTEGRAL
	</	2329	9001	LEFT-POINTING ANGLE BRACKET
	/>	232A	9002	RIGHT-POINTING ANGLE BRACKET
	Vs	2423	9251	OPEN BOX
	1h	2440	9280	OCR HOOK
	3h	2441	9281	OCR CHAIR
	2h	2442	9282	OCR FORK
	4h	2443	9283	OCR INVERTED FORK
	1j	2446	9286	OCR BRANCH BANK IDENTIFICATION
	2j	2447	9287	OCR AMOUNT OF CHECK
	3j	2448	9288	OCR DASH
	4j	2449	9289	OCR CUSTOMER ACCOUNT NUMBER
⒈	1.	2488	9352	DIGIT ONE FULL STOP

2.	2.	2489	9353	DIGIT TWO FULL STOP
3.	3.	248A	9354	DIGIT THREE FULL STOP
4.	4.	248B	9355	DIGIT FOUR FULL STOP
5.	5.	248C	9356	DIGIT FIVE FULL STOP
6.	6.	248D	9357	DIGIT SIX FULL STOP
7.	7.	248E	9358	DIGIT SEVEN FULL STOP
8.	8.	248F	9359	DIGIT EIGHT FULL STOP
9.	9.	2490	9360	DIGIT NINE FULL STOP
─	hh	2500	9472	BOX DRAWINGS LIGHT HORIZONTAL
━	HH	2501	9473	BOX DRAWINGS HEAVY HORIZONTAL
│	vv	2502	9474	BOX DRAWINGS LIGHT VERTICAL
┃	VV	2503	9475	BOX DRAWINGS HEAVY VERTICAL
┄	3-	2504	9476	BOX DRAWINGS LIGHT TRIPLE DASH HORIZONTAL
┅	3_	2505	9477	BOX DRAWINGS HEAVY TRIPLE DASH HORIZONTAL
┆	3!	2506	9478	BOX DRAWINGS LIGHT TRIPLE DASH VERTICAL
┇	3/	2507	9479	BOX DRAWINGS HEAVY TRIPLE DASH VERTICAL
┈	4-	2508	9480	BOX DRAWINGS LIGHT QUADRUPLE DASH HORIZONTAL
┉	4_	2509	9481	BOX DRAWINGS HEAVY QUADRUPLE DASH HORIZONTAL
┊	4!	250A	9482	BOX DRAWINGS LIGHT QUADRUPLE DASH VERTICAL
┋	4/	250B	9483	BOX DRAWINGS HEAVY QUADRUPLE DASH VERTICAL
┌	dr	250C	9484	BOX DRAWINGS LIGHT DOWN AND RIGHT
┍	dR	250D	9485	BOX DRAWINGS DOWN LIGHT AND RIGHT HEAVY
┎	Dr	250E	9486	BOX DRAWINGS DOWN HEAVY AND RIGHT LIGHT
┏	DR	250F	9487	BOX DRAWINGS HEAVY DOWN AND RIGHT
┐	dl	2510	9488	BOX DRAWINGS LIGHT DOWN AND LEFT
┑	dL	2511	9489	BOX DRAWINGS DOWN LIGHT AND LEFT HEAVY
┒	Dl	2512	9490	BOX DRAWINGS DOWN HEAVY AND LEFT LIGHT
┓	LD	2513	9491	BOX DRAWINGS HEAVY DOWN AND LEFT
└	ur	2514	9492	BOX DRAWINGS LIGHT UP AND RIGHT
┕	uR	2515	9493	BOX DRAWINGS UP LIGHT AND RIGHT HEAVY
┖	Ur	2516	9494	BOX DRAWINGS UP HEAVY AND RIGHT LIGHT
┗	UR	2517	9495	BOX DRAWINGS HEAVY UP AND RIGHT
┘	ul	2518	9496	BOX DRAWINGS LIGHT UP AND LEFT
┙	uL	2519	9497	BOX DRAWINGS UP LIGHT AND LEFT HEAVY
┚	Ul	251A	9498	BOX DRAWINGS UP HEAVY AND LEFT LIGHT
┛	UL	251B	9499	BOX DRAWINGS HEAVY UP AND LEFT
├	vr	251C	9500	BOX DRAWINGS LIGHT VERTICAL AND RIGHT
┝	vR	251D	9501	BOX DRAWINGS VERTICAL LIGHT AND RIGHT HEAVY
┞	Vr	2520	9504	BOX DRAWINGS VERTICAL HEAVY AND RIGHT LIGHT
┣	VR	2523	9507	BOX DRAWINGS HEAVY VERTICAL AND RIGHT
┤	vl	2524	9508	BOX DRAWINGS LIGHT VERTICAL AND LEFT
┥	vL	2525	9509	BOX DRAWINGS VERTICAL LIGHT AND LEFT HEAVY
┦	Vl	2528	9512	BOX DRAWINGS VERTICAL HEAVY AND LEFT LIGHT
┫	VL	252B	9515	BOX DRAWINGS HEAVY VERTICAL AND LEFT
┬	dh	252C	9516	BOX DRAWINGS LIGHT DOWN AND HORIZONTAL
┯	dH	252F	9519	BOX DRAWINGS DOWN LIGHT AND HORIZONTAL HEAVY
┰	Dh	2530	9520	BOX DRAWINGS DOWN HEAVY AND HORIZONTAL LIGHT
┳	DH	2533	9523	BOX DRAWINGS HEAVY DOWN AND HORIZONTAL
┴	uh	2534	9524	BOX DRAWINGS LIGHT UP AND HORIZONTAL
┷	uH	2537	9527	BOX DRAWINGS UP LIGHT AND HORIZONTAL HEAVY
┸	Uh	2538	9528	BOX DRAWINGS UP HEAVY AND HORIZONTAL LIGHT
┻	UH	253B	9531	BOX DRAWINGS HEAVY UP AND HORIZONTAL
┼	vh	253C	9532	BOX DRAWINGS LIGHT VERTICAL AND HORIZONTAL
┿	vH	253F	9535	BOX DRAWINGS VERTICAL LIGHT AND HORIZONTAL HEAVY

┼	Vh	2542	9538	BOX DRAWINGS VERTICAL HEAVY AND HORIZONTAL LIGHT
╋	VH	254B	9547	BOX DRAWINGS HEAVY VERTICAL AND HORIZONTAL
	FD	2571	9585	BOX DRAWINGS LIGHT DIAGONAL UPPER RIGHT TO LOWER LEFT
	BD	2572	9586	BOX DRAWINGS LIGHT DIAGONAL UPPER LEFT TO LOWER RIGHT
	TB	2580	9600	UPPER HALF BLOCK
	LB	2584	9604	LOWER HALF BLOCK
	FB	2588	9608	FULL BLOCK
	lB	258C	9612	LEFT HALF BLOCK
	RB	2590	9616	RIGHT HALF BLOCK
	.S	2591	9617	LIGHT SHADE
	:S	2592	9618	MEDIUM SHADE
	?S	2593	9619	DARK SHADE
■	fS	25A0	9632	BLACK SQUARE
□	OS	25A1	9633	WHITE SQUARE
	RO	25A2	9634	WHITE SQUARE WITH ROUNDED CORNERS
	Rr	25A3	9635	WHITE SQUARE CONTAINING BLACK SMALL SQUARE
	RF	25A4	9636	SQUARE WITH HORIZONTAL FILL
	RY	25A5	9637	SQUARE WITH VERTICAL FILL
	RH	25A6	9638	SQUARE WITH ORTHOGONAL CROSSHATCH FILL
	RZ	25A7	9639	SQUARE WITH UPPER LEFT TO LOWER RIGHT FILL
	RK	25A8	9640	SQUARE WITH UPPER RIGHT TO LOWER LEFT FILL
	RX	25A9	9641	SQUARE WITH DIAGONAL CROSSHATCH FILL
	sB	25AA	9642	BLACK SMALL SQUARE
	SR	25AC	9644	BLACK RECTANGLE
	Or	25AD	9645	WHITE RECTANGLE
▲	UT	25B2	9650	BLACK UP-POINTING TRIANGLE
△	uT	25B3	9651	WHITE UP-POINTING TRIANGLE
	PR	25B6	9654	BLACK RIGHT-POINTING TRIANGLE
	Tr	25B7	9655	WHITE RIGHT-POINTING TRIANGLE
	Dt	25BC	9660	BLACK DOWN-POINTING TRIANGLE
	dT	25BD	9661	WHITE DOWN-POINTING TRIANGLE
	PL	25C0	9664	BLACK LEFT-POINTING TRIANGLE
	Tl	25C1	9665	WHITE LEFT-POINTING TRIANGLE
◆	Db	25C6	9670	BLACK DIAMOND
◇	Dw	25C7	9671	WHITE DIAMOND
	LZ	25CA	9674	LOZENGE
○	0m	25CB	9675	WHITE CIRCLE
◎	0o	25CE	9678	BULLSEYE
●	0M	25CF	9679	BLACK CIRCLE
	0L	25D0	9680	CIRCLE WITH LEFT HALF BLACK
	0R	25D1	9681	CIRCLE WITH RIGHT HALF BLACK
	Sn	25D8	9688	INVERSE BULLET
	Ic	25D9	9689	INVERSE WHITE CIRCLE
	Fd	25E2	9698	BLACK LOWER RIGHT TRIANGLE
	Bd	25E3	9699	BLACK LOWER LEFT TRIANGLE
★	*2	2605	9733	BLACK STAR
☆	*1	2606	9734	WHITE STAR
	<H	261C	9756	WHITE LEFT POINTING INDEX
	>H	261E	9758	WHITE RIGHT POINTING INDEX
	0u	263A	9786	WHITE SMILING FACE
	0U	263B	9787	BLACK SMILING FACE
	SU	263C	9788	WHITE SUN WITH RAYS
♀	Fm	2640	9792	FEMALE SIGN
♂	Ml	2642	9794	MALE SIGN

```
cS      2660    9824    BLACK SPADE SUIT
cH      2661    9825    WHITE HEART SUIT
cD      2662    9826    WHITE DIAMOND SUIT
cC      2663    9827    BLACK CLUB SUIT
Md      2669    9833    QUARTER NOTE ‘
M8      266A    9834    EIGHTH NOTE ‘
M2      266B    9835    BEAMED EIGHTH NOTES
Mb      266D    9837    MUSIC FLAT SIGN
Mx      266E    9838    MUSIC NATURAL SIGN
MX      266F    9839    MUSIC SHARP SIGN
OK      2713    10003   CHECK MARK
XX      2717    10007   BALLOT X
-X      2720    10016   MALTESE CROSS
  IS      3000    12288   IDEOGRAPHIC SPACE
、  ,_      3001    12289   IDEOGRAPHIC COMMA
。  ._      3002    12290   IDEOGRAPHIC FULL STOP
〃  +"      3003    12291   DITTO MARK
+_      3004    12292   JAPANESE INDUSTRIAL STANDARD SYMBOL
々  *_      3005    12293   IDEOGRAPHIC ITERATION MARK
;_      3006    12294   IDEOGRAPHIC CLOSING MARK
〇  0_      3007    12295   IDEOGRAPHIC NUMBER ZERO
《  <+      300A    12298   LEFT DOUBLE ANGLE BRACKET
》  >+      300B    12299   RIGHT DOUBLE ANGLE BRACKET
「  <’      300C    12300   LEFT CORNER BRACKET
」  >’      300D    12301   RIGHT CORNER BRACKET
『  <"      300E    12302   LEFT WHITE CORNER BRACKET
』  >"      300F    12303   RIGHT WHITE CORNER BRACKET
【  ("      3010    12304   LEFT BLACK LENTICULAR BRACKET
】  )"      3011    12305   RIGHT BLACK LENTICULAR BRACKET
=T      3012    12306   POSTAL MARK
〒  =_      3013    12307   GETA MARK
〔  (’      3014    12308   LEFT TORTOISE SHELL BRACKET
〕  )’      3015    12309   RIGHT TORTOISE SHELL BRACKET
〖  (I      3016    12310   LEFT WHITE LENTICULAR BRACKET
〗  )I      3017    12311   RIGHT WHITE LENTICULAR BRACKET
-?      301C    12316   WAVE DASH
ぁ  A5      3041    12353   HIRAGANA LETTER SMALL A
あ  a5      3042    12354   HIRAGANA LETTER A
ぃ  I5      3043    12355   HIRAGANA LETTER SMALL I
い  i5      3044    12356   HIRAGANA LETTER I
ぅ  U5      3045    12357   HIRAGANA LETTER SMALL U
う  u5      3046    12358   HIRAGANA LETTER U
ぇ  E5      3047    12359   HIRAGANA LETTER SMALL E
え  e5      3048    12360   HIRAGANA LETTER E
ぉ  O5      3049    12361   HIRAGANA LETTER SMALL O
お  o5      304A    12362   HIRAGANA LETTER O
か  ka      304B    12363   HIRAGANA LETTER KA
が  ga      304C    12364   HIRAGANA LETTER GA
き  ki      304D    12365   HIRAGANA LETTER KI
ぎ  gi      304E    12366   HIRAGANA LETTER GI
く  ku      304F    12367   HIRAGANA LETTER KU
ぐ  gu      3050    12368   HIRAGANA LETTER GU
け  ke      3051    12369   HIRAGANA LETTER KE
げ  ge      3052    12370   HIRAGANA LETTER GE
```

こ ko 3053 12371 HIRAGANA LETTER KO
ごご go 3054 12372 HIRAGANA LETTER GO
さ sa 3055 12373 HIRAGANA LETTER SA
ざざ za 3056 12374 HIRAGANA LETTER ZA
し si 3057 12375 HIRAGANA LETTER SI
じじ zi 3058 12376 HIRAGANA LETTER ZI
すず su 3059 12377 HIRAGANA LETTER SU
ずず zu 305A 12378 HIRAGANA LETTER ZU
せ se 305B 12379 HIRAGANA LETTER SE
ぜぜ ze 305C 12380 HIRAGANA LETTER ZE
そ so 305D 12381 HIRAGANA LETTER SO
ぞぞ zo 305E 12382 HIRAGANA LETTER ZO
た ta 305F 12383 HIRAGANA LETTER TA
だだ da 3060 12384 HIRAGANA LETTER DA
ち ti 3061 12385 HIRAGANA LETTER TI
ぢぢ di 3062 12386 HIRAGANA LETTER DI
っ tU 3063 12387 HIRAGANA LETTER SMALL TU
つつ tu 3064 12388 HIRAGANA LETTER TU
づづ du 3065 12389 HIRAGANA LETTER DU
て te 3066 12390 HIRAGANA LETTER TE
でで de 3067 12391 HIRAGANA LETTER DE
と to 3068 12392 HIRAGANA LETTER TO
どど do 3069 12393 HIRAGANA LETTER DO
なな na 306A 12394 HIRAGANA LETTER NA
にに ni 306B 12395 HIRAGANA LETTER NI
ぬぬ nu 306C 12396 HIRAGANA LETTER NU
ねね ne 306D 12397 HIRAGANA LETTER NE
のの no 306E 12398 HIRAGANA LETTER NO
はは ha 306F 12399 HIRAGANA LETTER HA
ばば ba 3070 12400 HIRAGANA LETTER BA
ぱぱ pa 3071 12401 HIRAGANA LETTER PA
ひひ hi 3072 12402 HIRAGANA LETTER HI
びび bi 3073 12403 HIRAGANA LETTER BI
ぴぴ pi 3074 12404 HIRAGANA LETTER PI
ふふ hu 3075 12405 HIRAGANA LETTER HU
ぶぶ bu 3076 12406 HIRAGANA LETTER BU
ぷぷ pu 3077 12407 HIRAGANA LETTER PU
へへ he 3078 12408 HIRAGANA LETTER HE
べべ be 3079 12409 HIRAGANA LETTER BE
ぺぺ pe 307A 12410 HIRAGANA LETTER PE
ほほ ho 307B 12411 HIRAGANA LETTER HO
ぼぼ bo 307C 12412 HIRAGANA LETTER BO
ぽぽ po 307D 12413 HIRAGANA LETTER PO
ま ma 307E 12414 HIRAGANA LETTER MA
みみ mi 307F 12415 HIRAGANA LETTER MI
むむ mu 3080 12416 HIRAGANA LETTER MU
めめ me 3081 12417 HIRAGANA LETTER ME
もも mo 3082 12418 HIRAGANA LETTER MO
ゃ yA 3083 12419 HIRAGANA LETTER SMALL YA
やや ya 3084 12420 HIRAGANA LETTER YA
ゅ yU 3085 12421 HIRAGANA LETTER SMALL YU
ゆゆ yu 3086 12422 HIRAGANA LETTER YU
ょ yO 3087 12423 HIRAGANA LETTER SMALL YO
よよ yo 3088 12424 HIRAGANA LETTER YO

ら	ra	3089	12425	HIRAGANA LETTER RA
り	ri	308A	12426	HIRAGANA LETTER RI
る	ru	308B	12427	HIRAGANA LETTER RU
れ	re	308C	12428	HIRAGANA LETTER RE
ろ	ro	308D	12429	HIRAGANA LETTER RO
ゎ	wA	308E	12430	HIRAGANA LETTER SMALL WA
わ	wa	308F	12431	HIRAGANA LETTER WA
ゐ	wi	3090	12432	HIRAGANA LETTER WI
ゑ	we	3091	12433	HIRAGANA LETTER WE
を	wo	3092	12434	HIRAGANA LETTER WO
ん	n5	3093	12435	HIRAGANA LETTER N ‘
	vu	3094	12436	HIRAGANA LETTER VU
	"5	309B	12443	KATAKANA-HIRAGANA VOICED SOUND MARK
	05	309C	12444	KATAKANA-HIRAGANA SEMI-VOICED SOUND MARK
	*5	309D	12445	HIRAGANA ITERATION MARK
	+5	309E	12446	HIRAGANA VOICED ITERATION MARK
ァ	a6	30A1	12449	KATAKANA LETTER SMALL A
ア	A6	30A2	12450	KATAKANA LETTER A
ィ	i6	30A3	12451	KATAKANA LETTER SMALL I
イ	I6	30A4	12452	KATAKANA LETTER I
ゥ	u6	30A5	12453	KATAKANA LETTER SMALL U
ウ	U6	30A6	12454	KATAKANA LETTER U
ェ	e6	30A7	12455	KATAKANA LETTER SMALL E
エ	E6	30A8	12456	KATAKANA LETTER E
ォ	o6	30A9	12457	KATAKANA LETTER SMALL O
オ	O6	30AA	12458	KATAKANA LETTER O
カ	Ka	30AB	12459	KATAKANA LETTER KA
ガ	Ga	30AC	12460	KATAKANA LETTER GA
キ	Ki	30AD	12461	KATAKANA LETTER KI
ギ	Gi	30AE	12462	KATAKANA LETTER GI
ク	Ku	30AF	12463	KATAKANA LETTER KU
グ	Gu	30B0	12464	KATAKANA LETTER GU
ケ	Ke	30B1	12465	KATAKANA LETTER KE
ゲ	Ge	30B2	12466	KATAKANA LETTER GE
コ	Ko	30B3	12467	KATAKANA LETTER KO
ゴ	Go	30B4	12468	KATAKANA LETTER GO
サ	Sa	30B5	12469	KATAKANA LETTER SA
ザ	Za	30B6	12470	KATAKANA LETTER ZA
シ	Si	30B7	12471	KATAKANA LETTER SI
ジ	Zi	30B8	12472	KATAKANA LETTER ZI
ス	Su	30B9	12473	KATAKANA LETTER SU
ズ	Zu	30BA	12474	KATAKANA LETTER ZU
セ	Se	30BB	12475	KATAKANA LETTER SE
ゼ	Ze	30BC	12476	KATAKANA LETTER ZE
ソ	So	30BD	12477	KATAKANA LETTER SO
ゾ	Zo	30BE	12478	KATAKANA LETTER ZO
タ	Ta	30BF	12479	KATAKANA LETTER TA
ダ	Da	30C0	12480	KATAKANA LETTER DA
チ	Ti	30C1	12481	KATAKANA LETTER TI
ヂ	Di	30C2	12482	KATAKANA LETTER DI
ッ	TU	30C3	12483	KATAKANA LETTER SMALL TU
ツ	Tu	30C4	12484	KATAKANA LETTER TU
ヅ	Du	30C5	12485	KATAKANA LETTER DU
テ	Te	30C6	12486	KATAKANA LETTER TE

デ De 30C7 12487 KATAKANA LETTER DE
ト To 30C8 12488 KATAKANA LETTER TO
ド Do 30C9 12489 KATAKANA LETTER DO
ナ Na 30CA 12490 KATAKANA LETTER NA
ニ Ni 30CB 12491 KATAKANA LETTER NI
ヌ Nu 30CC 12492 KATAKANA LETTER NU
ネ Ne 30CD 12493 KATAKANA LETTER NE
ノ No 30CE 12494 KATAKANA LETTER NO
ハ Ha 30CF 12495 KATAKANA LETTER HA
バ Ba 30D0 12496 KATAKANA LETTER BA
パ Pa 30D1 12497 KATAKANA LETTER PA
ヒ Hi 30D2 12498 KATAKANA LETTER HI
ビ Bi 30D3 12499 KATAKANA LETTER BI
ピ Pi 30D4 12500 KATAKANA LETTER PI
フ Hu 30D5 12501 KATAKANA LETTER HU
ブ Bu 30D6 12502 KATAKANA LETTER BU
プ Pu 30D7 12503 KATAKANA LETTER PU
ヘ He 30D8 12504 KATAKANA LETTER HE
ベ Be 30D9 12505 KATAKANA LETTER BE
ペ Pe 30DA 12506 KATAKANA LETTER PE
ホ Ho 30DB 12507 KATAKANA LETTER HO
ボ Bo 30DC 12508 KATAKANA LETTER BO
ポ Po 30DD 12509 KATAKANA LETTER PO
マ Ma 30DE 12510 KATAKANA LETTER MA
ミ Mi 30DF 12511 KATAKANA LETTER MI
ム Mu 30E0 12512 KATAKANA LETTER MU
メ Me 30E1 12513 KATAKANA LETTER ME
モ Mo 30E2 12514 KATAKANA LETTER MO
ャ YA 30E3 12515 KATAKANA LETTER SMALL YA
ヤ Ya 30E4 12516 KATAKANA LETTER YA
ュ YU 30E5 12517 KATAKANA LETTER SMALL YU
ユ Yu 30E6 12518 KATAKANA LETTER YU
ョ YO 30E7 12519 KATAKANA LETTER SMALL YO
ヨ Yo 30E8 12520 KATAKANA LETTER YO
ラ Ra 30E9 12521 KATAKANA LETTER RA
リ Ri 30EA 12522 KATAKANA LETTER RI
ル Ru 30EB 12523 KATAKANA LETTER RU
レ Re 30EC 12524 KATAKANA LETTER RE
ロ Ro 30ED 12525 KATAKANA LETTER RO
ヮ WA 30EE 12526 KATAKANA LETTER SMALL WA
ワ Wa 30EF 12527 KATAKANA LETTER WA
ヰ Wi 30F0 12528 KATAKANA LETTER WI
ヱ We 30F1 12529 KATAKANA LETTER WE
ヲ Wo 30F2 12530 KATAKANA LETTER WO
ン N6 30F3 12531 KATAKANA LETTER N ‘
ヴ Vu 30F4 12532 KATAKANA LETTER VU
ヵ KA 30F5 12533 KATAKANA LETTER SMALL KA
ヶ KE 30F6 12534 KATAKANA LETTER SMALL KE
Va 30F7 12535 KATAKANA LETTER VA
Vi 30F8 12536 KATAKANA LETTER VI
Ve 30F9 12537 KATAKANA LETTER VE
Vo 30FA 12538 KATAKANA LETTER VO
.6 30FB 12539 KATAKANA MIDDLE DOT
-6 30FC 12540 KATAKANA-HIRAGANA PROLONGED SOUND MARK

	*6	30FD	12541	KATAKANA ITERATION MARK
	+6	30FE	12542	KATAKANA VOICED ITERATION MARK
ㄅ	b4	3105	12549	BOPOMOFO LETTER B
ㄆ	p4	3106	12550	BOPOMOFO LETTER P
ㄇ	m4	3107	12551	BOPOMOFO LETTER M
ㄈ	f4	3108	12552	BOPOMOFO LETTER F
ㄉ	d4	3109	12553	BOPOMOFO LETTER D
ㄊ	t4	310A	12554	BOPOMOFO LETTER T
ㄋ	n4	310B	12555	BOPOMOFO LETTER N ‘
ㄌ	l4	310C	12556	BOPOMOFO LETTER L
ㄍ	g4	310D	12557	BOPOMOFO LETTER G
ㄎ	k4	310E	12558	BOPOMOFO LETTER K
ㄏ	h4	310F	12559	BOPOMOFO LETTER H
ㄐ	j4	3110	12560	BOPOMOFO LETTER J
ㄑ	q4	3111	12561	BOPOMOFO LETTER Q
ㄒ	x4	3112	12562	BOPOMOFO LETTER X
ㄓ	zh	3113	12563	BOPOMOFO LETTER ZH
ㄔ	ch	3114	12564	BOPOMOFO LETTER CH
ㄕ	sh	3115	12565	BOPOMOFO LETTER SH
ㄖ	r4	3116	12566	BOPOMOFO LETTER R
ㄗ	z4	3117	12567	BOPOMOFO LETTER Z
ㄘ	c4	3118	12568	BOPOMOFO LETTER C
ㄙ	s4	3119	12569	BOPOMOFO LETTER S
ㄚ	a4	311A	12570	BOPOMOFO LETTER A
ㄛ	o4	311B	12571	BOPOMOFO LETTER O
ㄜ	e4	311C	12572	BOPOMOFO LETTER E
ㄞ	ai	311E	12574	BOPOMOFO LETTER AI
ㄟ	ei	311F	12575	BOPOMOFO LETTER EI
ㄠ	au	3120	12576	BOPOMOFO LETTER AU
ㄡ	ou	3121	12577	BOPOMOFO LETTER OU
ㄢ	an	3122	12578	BOPOMOFO LETTER AN
ㄣ	en	3123	12579	BOPOMOFO LETTER EN
ㄤ	aN	3124	12580	BOPOMOFO LETTER ANG
ㄥ	eN	3125	12581	BOPOMOFO LETTER ENG
ㄦ	er	3126	12582	BOPOMOFO LETTER ER
ㄧ	i4	3127	12583	BOPOMOFO LETTER I
ㄨ	u4	3128	12584	BOPOMOFO LETTER U
ㄩ	iu	3129	12585	BOPOMOFO LETTER IU
	v4	312A	12586	BOPOMOFO LETTER V
	nG	312B	12587	BOPOMOFO LETTER NG
	gn	312C	12588	BOPOMOFO LETTER GN
㈠	1c	3220	12832	PARENTHESIZED IDEOGRAPH ONE
㈡	2c	3221	12833	PARENTHESIZED IDEOGRAPH TWO
㈢	3c	3222	12834	PARENTHESIZED IDEOGRAPH THREE
㈣	4c	3223	12835	PARENTHESIZED IDEOGRAPH FOUR
㈤	5c	3224	12836	PARENTHESIZED IDEOGRAPH FIVE
㈥	6c	3225	12837	PARENTHESIZED IDEOGRAPH SIX
㈦	7c	3226	12838	PARENTHESIZED IDEOGRAPH SEVEN
㈧	8c	3227	12839	PARENTHESIZED IDEOGRAPH EIGHT
㈨	9c	3228	12840	PARENTHESIZED IDEOGRAPH NINE
	ff	FB00	64256	LATIN SMALL LIGATURE FF
	fi	FB01	64257	LATIN SMALL LIGATURE FI
	fl	FB02	64258	LATIN SMALL LIGATURE FL
	ft	FB05	64261	LATIN SMALL LIGATURE LONG S T

```
     st      FB06    64262   LATIN SMALL LIGATURE ST

vim:tw=78:ts=8:noet:ft=help:norl:
```

VIM 参考手册 by Bram Moolenaar et al.
译者:yemao & lang2, Willis http://vimcdoc.sf.net

多字节支持 multibyte multi-byte Chinese Japanese Korean

本章讲述了关于使用多字节语言相关的内容。这样的语言包括汉语，日语，韩语。同时也讲述了 Unicode。

一般性的常识，请参考用户手册 usr_45.txt 。
如何更改菜单语言和消息语言，请参考 mlang.txt 。

1. 初步 mbyte-first
2. 本地化 mbyte-locale
3. 编码 mbyte-encoding
4. 使用终端 mbyte-terminal
5. X11 的字体 mbyte-fonts-X11
6. MS-Windows 的字体 mbyte-fonts-MSwin
7. 在 X11 上输入 mbyte-XIM
8. 在 MS-Windows 上输入 mbyte-IME
9. 使用键盘映射表输入 mbyte-keymap
10. 用 imactivatefunc() 输入 mbyte-func
11. 使用 UTF-8 mbyte-utf8
12. 选项总述 mbyte-options

注意：本文件包含 UTF-8 编码字符。如果使用其它编码，这些字符可能显示为奇怪的符号或者方框。

1.初步 mbyte-first

这里对 Vim 的多字节特点作总体介绍。幸运的话，你的 vim 一切都运行正常，如果不正常，请阅读以下内容。你可能需要花一些时间，经过多次试验才能使 vim 支持多字节。因为不幸的是，每个系统都有一套自己处理多字节语言的方法，而且非常复杂。

本 地 化

首先，确认当前的 locale 设置正确。如果系统已经设定了对该语言的支持，一切将正常工作。否则，你需要在外壳里面设置 $LANG 变量:

```
setenv LANG ja_JP.EUC
```

很不幸，locale 的名字取决于你的系统。中文可能被叫做 "zh_CN.gbk"，也可能只是 "zh"。这样可以查看当前的语言:

```
:language
```

修改 Vim 所用的 locale:

```
:language zh_CN.gbk
```

如果设置的 locale 不能工作，Vim 将会给出错误信息。要找出需要使用的 locale，这是一个好办法。不过最好是在外壳里面设置 locale，那样，启动的时候就可以正常使用。具体请看 mbyte-locale 。

编 码

如果 locale 正常工作，Vim 随后将相应设置 'encoding'，如果不能使用，你可以重新设置来取代之：

```
:set encoding=utf-8
```

encoding-values 列出了可以使用的值。

设置之后，正在用 Vim 编辑的文件将使用这种编码。不仅仅是缓冲区的文件，寄存器，变量等都会使用这种编码。这也意味着 'encoding' 的改变使已有的文本变得无效，不过不是内容的变化，而是不能正常显示。

你可以编辑另一种编码下的文件，Vim 在读文件的时候把从该编码转化过来，而保存时又按原来的编码写回。具体请看 'fileencoding'，'fileencodings' 和 ++enc 。

字 体 和 显 示

如果使用终端（模拟），你必须确定终端可以使用 Vim 所使用的编码。否则你必须修改 'termencoding' 使 Vim 自动转化文件编码。

在 GUI 上，你必须选择在当前编码下可以使用的字体。这并不容易。这和终端有所不同。它和你用的系统，locale 和一些其它的东西有关。具体请看有关字体的章节：mbyte-fonts-X11 讨论 X-Windows，而 mbyte-fonts-MSwin 讨论 MS-Windows。

GTK+ 2 上，你可以跳过本节的大部分讨论。'guifontset' 不再存在。你只需要设置 'guifont'，其余的一切就 "自然而然地" 解决了。如果你的系统带有 Xft2 和 fontconfig，而当前字体缺少若干字形的话，系统会自动使用其它能找到的字体。'guifontwide' 选项仍然支持，但一般没有必要设置它。只有自动字体选择机制不合你心意时才需要如此。

在 X11 上，你可以设置 'guifontset'，把要使用的字体写入列表。以韩语为例：

```
:set guifontset=k12,r12
```

同时，你可以设置 'guifont' 和 'guifontwide'。'guifont' 设置单宽字体，'guifontwide' 设置双宽字体。因此，'guifontwide' 字体的宽度必须是 'guifont' 字体的两倍。
以 UTF-8 为例：

```
:set guifont=-misc-fixed-medium-r-normal-*-18-120-100-100-c-90-iso10646-1
:set guifontwide=-misc-fixed-medium-r-normal-*-18-120-100-100-c-90-iso10646-1
```

你也可以只设置 'guifont'，Vim 将尝试自动设置 'guifontwide'。

输 入

有多种方法输入多字节字符：

-在 X11 上，XIM 可以输入多字节字符，具体请看 XIM 。
-在 MS-Windows 上，IME 可以输入多字节，具体请看 IME 。
-在所有的系统上都可以使用键盘映射表。具体请看 mbyte-keymap 。

设置 'iminsert'，'imsearch' 和 'imcmdline' 可以选择不同的输入方法或临时禁用它们。

---

2. Locale mbyte-locale

最简单的设置就是整个系统使用你想用的 locale，不过你也可以设置外壳的 locale，或者只在 Vim 内部使用特定的 locale。

什么是 LOCALE? locale

世界上有很多种语言，而不同的文化和环境的数量至少有语言的种类那么多。一个地区对应的语言环境就叫 "locale"，它包括所使用的语言，文字，排序规则，时间格式，货币格式等信息，Vim 只和语言和文字有关。

你只能使用系统支持的 locale，有的系统的只有很少的 locale，在美国特别如此。也许你想使用的语言系统没有，这样你就需要把它当一个额外的包来安装，具体请按系统文档操作。

不同的系统安装 locale 的位置也不同，例如，"/usr/share/locale" 或者 "/usr/lib/locale"，具体请看 setlocale() 的手册页。

在这些目录里你可以看到每个 locale 的全称，它们也区别大小写，因此 "ja_JP.EUC" 和 "ja_jp.euc" 不同。有的系统有一个 locale.alias 文件，它允许一个简称如 "nl" 和它的全称 "nl_NL.ISO_8859-1" 转化。

备注：X-windows 有自己的 locale 设置，不幸的是它使用的 locale 名也和其它地方不同。这非常混乱！Vim 使用 setlocale() 的设置，而它一般并 不 同于 X-windows 的方法，也许你需要经过多次测试来怎么才能工作。

locale-name

locale 名的（简单的）格式是

```
        language
或者    language_territory
或者    language_territory.codeset
```

territory 表示国家（或者它的一部份），codeset 表示文字（字符集） charset 。例如，"ja_JP.eucJP" 表示

```
        ja      语言是 Japanese
        JP      国家是 Japan
        ecuJP   文字是 EUC-JP
```

不过它也可以是 "ja"，"ja_JP.EUC"，"ja_JP.ujis" 等等，糟糕的是，即使对一个特定的语言，国家和文字，locale 的名字也是不统一的，它取决于你的系统。

例子如下：

字符集	语言	locale 名
GB2312	中文（简体）	zh_CN.EUC、zh_CN.GB2312
Big5	中文（繁体）	zh_TW.BIG5、zh_TW.Big5
CNS-11643	中文（繁体）	zh_TW

EUC-JP	日语	ja、ja_JP.EUC、ja_JP.ujis、ja_JP.eucJP
Shift_JIS	日语	ja_JP.SJIS、ja_JP.Shift_JIS
EUC-KR	韩语	ko、ko_KR.EUC

使 用 一 种 LOCALE

使整个系统使用一个 locale，请看系统文档。大部分情况你需要在 "/etc" 下的配置文件中设置它。

要在外壳里使用 locale，设置 $LANG 环境变量。如果你想使用韩语， locale 名为 "ko"，那么：

```
sh:    export LANG=ko
csh:   setenv LANG ko
```

要经常使用它，把它们写进 ~/.profile 或者 ~/.cshrc 中。

要在 Vim 中使用某个 locale，用 :language 命令：

```
:language ko
```

把它写进 ~/.vimrc 就可以经常使用。

或者在启动 vim 的时候设置 $LANG：

```
sh:    LANG=ko vim {vim-arguments}
csh:   env LANG=ko vim {vim-arguments}
```

为此，你可以编写若干短小的外壳脚本。

---

3. 编码 mbyte-encoding

Vim 用 'encoding' 来设置 vim 识别字符和编码的方法。这个设置对文本使用的地方都有效，包括缓冲区（读入内存的文件），寄存器和变量。

charset codeset

Charset（字符集）是 encoding（编码）的另一种叫法，它们有一点很小的区别。但是对 vim 来说无关紧要。"codeset" 是它的另一个别名。

每个字符都被编码成为一个或者两个字节。如果所有的字符都被编码成一个字节，我们就称之为单字节编码。最常用的是 "latin1"。它把可用的字符数限制为 256 个。其中还有一部分控制字符，这使得可用于文本的字符数更少。

如果某些字符使用两个或更多字节，我们称其为多字节编码。从而能够使用多得多的字符，这样就能符合大多数东亚语种的要求。

多数多字节编码方式使用开始的 127 个字符作为一个字节。这和 ASCII 码相同，也使得与纯 ASCII 码之间的转换十分容易。不管你使用哪种语言，所以就算你的 encoding 设置错了，你也可以看到正确的文本。

encoding-names

Vim 可以使用多种不同的字符编码，主要有以下三大类：

1 8bit 单字节编码，256 个不同的字符。主要用于美国和欧洲。例如：ISO-8859-1 (Latin1)，所有的字符占一个屏幕单元。

2 2byte 双字节编码，超过 10000 个字符。主要在亚洲各国使用。例如：euc-cn (中文)。所占的屏幕单元数和字节数相同。(euc-jp 第一个字节是 0x8e 的编码除外)。

u Unicode 通用编码，可以取代其它所有的编码格式。ISO 10646。有几百万个字符。例如：UTF-8。字节数和屏幕单元的关系很复杂。

其它的编码不能在 Vim 内部使用。但是以这些编码的文件在转换之后可以被 Vim 编辑，具体请看 'fileencoding'。注意 所有的编码都必须对 128 之内的字符使用 ASCII 编码 (编译时加入 EBCDIC 除外)。

Vim 支持的 'encoding' 值有: encoding-values

1	latin1	8 位字符 (ISO 8859-1，也用于 cp1252)
1	iso-8859-n	ISO_8859 变体 (n = 2 to 15)
1	koi8-r	俄语
1	koi8-u	乌克兰语
1	macroman	MacRoman (Macintosh 编码)
1	8bit-{name}	任何 8 位编码 (Vim 特定名称)
1	cp437	类似于 iso-8859-1
1	cp737	类似于 iso-8859-7
1	cp775	波罗的语
1	cp850	类似于 iso-8859-4
1	cp852	类似于 iso-8859-1
1	cp855	类似于 iso-8859-2
1	cp857	类似于 iso-8859-5
1	cp860	类似于 iso-8859-9
1	cp861	类似于 iso-8859-1
1	cp862	类似于 iso-8859-1
1	cp863	类似于 iso-8859-8
1	cp865	类似于 iso-8859-1
1	cp866	类似于 iso-8859-5
1	cp869	类似于 iso-8859-7
1	cp874	泰语
1	cp1250	捷克语、波兰语等
1	cp1251	西里尔字母
1	cp1253	希腊语
1	cp1254	土耳其语
1	cp1255	希伯来语
1	cp1256	阿拉伯语
1	cp1257	波罗的语
1	cp1258	越南语
1	cp{number}	MS-Windows: 任何已安装的单字节代码页 (codepage)
2	cp932	日语 (仅限 Windows)
2	euc-jp	日语 (仅限 Unix)
2	sjis	日语 (仅限 Unix)
2	cp949	韩语 (Unix 和 Windows)
2	euc-kr	韩语 (仅限 Unix)
2	cp936	简体中文 (仅限 Windows)
2	euc-cn	简体中文 (仅限 Unix)
2	cp950	繁体中文 (Unix 上，big5 的别名)
2	big5	繁体中文 (Windows 上，cp950 的别名)

2	euc-tw	繁体中文（仅限 Unix）
2	2byte-{name}	Unix：任何双字节编码（Vim 专用名称）
2	cp{number}	MS-Windows：任何已安装的双字节代码页（codepage）
u	utf-8	32 位 UTF-8 编码的 Unicode（ISO/IEC 10646-1）
u	ucs-2	16 位 UCS-2 编码的 Unicode（ISO/IEC 10646-1）
u	ucs-2le	类似于 ucs-2，little endian（高位字节在后）
u	utf-16	ucs-2 并带有更多字符的双词扩展
u	utf-16le	类似于 utf-16，little endian
u	ucs-4	32 位 UCS-4 编码的 Unicode（ISO/IEC 10646-1）
u	ucs-4le	类似于 ucs-4，little endian

{name} 可以是任何系统支持的编码名。Vim 会调用 iconv() 在该编码名及当前 locale 之间转换。对 MS-Windows 来说，"cp{number}" 意味着使用 {number} 代码页（codepage）。
例如：

```
:set encoding=8bit-cp1252
:set encoding=2byte-cp932
```

MS-Windows 代码页 1252 和 latin1 相近。从实际角度考虑，使用相同的编码，也叫 latin1。'isprint' 可用来判断 0x80 - 0xA0 间的字符是否可显示。

也可以使用别名，它们被翻译为上表中的某一个。以下是一个不完全列表：

1	ansi	同 latin1（已废弃，保留是为了后向兼容）
2	japan	日语：Unix 上的 "euc-jp"，MS-Windows 上的 cp932
2	korea	韩语：Unix 上的 "euc-kr"，MS-Windows 上的 cp949
2	prc	简体中文：Unix 上的 "euc-cn"，MS-Windows 上的 cp936
2	chinese	等同于 "prc"
2	taiwan	繁体中文：Unix 上的 "euc-tw"，MS-Windows 上的 cp950
u	utf8	同 utf-8
u	unicode	同 ucs-2
u	ucs2be	同 ucs-2（big endian）
u	ucs-2be	同 ucs-2（big endian）
u	ucs-4be	同 ucs-4（big endian）
u	utf-32	同 ucs-4
u	utf-32le	同 ucs-4le
	default	代表 'encoding' 的默认值，取决于环境变量

对于 UCS 编码，字节次序会影响编码结果。这比较麻烦。因此尽可能使用 UTF-8。默认使用 big-endian（高位字节在前）：

name	bytes	char
ucs-2	11 22	1122
ucs-2le	22 11	1122
ucs-4	11 22 33 44	11223344
ucs-4le	44 33 22 11	11223344

在 MS-Windows 系统上你经常需要使用 "ucs-2le"，因为它使用了 little-endian UCS-2 编码。

有一些编码看起来相似，实际上是不完全相同的。Vim 把它们当不同的编码来处理，必要时做转换。当转换不必要或者需要避免时，你可以使用相近的编码名。

cp932、shift-jis、sjis
cp936、euc-cn

encoding-table

一般情况下，'encoding' 和当前的 locale 相同，'termencoding' 为空，这意味着键盘和显示方式以当前 locale 编码字符，Vim 内部也使用相同的字符。

你可以通过设置 'encoding' 为不同的值来使 Vim 使用另一种编码方式。但是由於键盘和显示仍用当前的 locale，这就需要编码之间的转换。这时 'termencoding' 替代了当前 locale 值，而 Vim 负责在 'encoding' 和 'termencoding' 之间转换。例如:

```
:let &termencoding = &encoding
:set encoding=utf-8
```

尽管如此，并不是所有组合都可以转换。下面这个表列出 9 种组合如何转换。这还受到 iconv() 的功能影响。因为这取决于你所用的系统，这里无法给出具体的信息。

('tenc' 是 'termencoding' 的缩写，'enc' 是 'encoding' 的缩写)

'tenc'	'enc'	解释
8bit	8bit	可以转换，当 'termencoding' 和 'encoding' 不同时，有些字符的输入和显示会有问题，Vim 不做转换 (把 'encoding' 设成 "utf-8" 可以解决)。
8bit	2byte	MS-Windows: 系统上安装的所有代码页都可以转换，同时，你只能输入 8bit 字符，在其它的系统上都不行。
8bit	Unicode	可以转换，但是你只能直接输入 8bit 字符 (其它字符可以通过二合字母、键盘映射表等方式输入)，在终端下你只能看到 8bit 字符，GUI 上可以看到 'guifont' 支持的所有字符。
2byte	8bit	可以转换，但是输入非 ASCII 字符可能会有问题。
2byte	2byte	MS-Windows: 支持系统安装的所有的代码页之间的转换，当 locale 和 'encoding' 不同时输入可能会有问题。 在其它系统上，只有当 'termencoding' 和 'encoding' 相同或者为空时可以转换。
2byte	Unicode	可以转换。Vim 会转换输入的字符。
Unicode	8bit	可以转换 (很少见)
Unicode	2byte	不能转换
Unicode	Unicode	转换非常好 ('termencoding' 为空也可，因为所有的 Unicode 内部使用 UTF-8)

转换 charset-conversion

以下情况，Vim 会自动把一种编码转换成另一种编码:
- 读文件时 'fileencoding' 和 'encoding' 不同
- 写文件时 'fileencoding' 和 'encoding' 不同
- 显示字符时 'termencoding' 和 'encoding' 不同
- 读取输入时 'termencoding' 和 'encoding' 不同
- 显示信息时 LC_MESSAGE 使用的编码和 'encoding' 不同 (需要支持此功能的 gettext)
- Vim 脚本 :scriptencoding 和 'encoding' 不同
- 读写一个 viminfo 文件

以上很多都需要 +iconv 特性，读写文件的转换也可以通过指定 'chaconvert' 来完成。

转换字符集一些有用的工具：

所有语言：　　iconv

GNU iconv 可以转换很多编码，Unicode 作为中间编码，它可以和其它所有的编码相互转换。具体请看 http://www.gnu.org/directory/libiconv.html。

日文：　　nkf

Nkf 是 "Network Kanji code conversion Filter" 的缩写，nkf 最特别的地方是它可以猜测输入的 Kanji 的编码。所以，你不必知道输入文件的 charset 是什么。要从 ISO-2202-JP 或 Shift_JIS 转换成 EUC-JP，在 Vim 里输入以下命令：

```
:%!nkf -e
```

Nkf 可以在下面地址找到：

http://www.sfc.wide.ad.jp/~max/FreeBSD/ports/distfiles/nkf-1.62.tar.gz

中文：　　hc

Hc 是 "Hanzi Converter" 的简写，Hc 把一个 GB 文件转换成 Big5 文件，或者把一个 Big5 文件转换成 GB 文件。Hc 可以在以下地址找到：

ftp://ftp.cuhk.hk/pub/chinese/ifcss/software/unix/convert/hc-30.tar.gz

韩文：　　hmconv

Hmconv 是一套 E-mail 文字转换的工具，它可以在 EUC-KR 和 ISO-2202-KR 之间转换。Hmconv 可以在以下地址找到：

ftp://ftp.kaist.ac.kr/pub/hangul/code/hmconv/

多语言：　　lv

Lv 是一个强大的多语言文件查看器，它还可以作为 charset 转换器，支持的 charset 有：ISO-2202-CN，ISO-2202-JP，ISO-2202-KR，EUC-CN，EUC-JP，EUC-KR，EUC-TW，UTF-7，UTF-8，ISO-8859 系列，Shift_JIS，Big5 和 HZ。Lv 可以在以下地址找到：

http://www.ff.iij4u.or.jp/~nrt/lv/index.html

mbyte-conversion

如果读入和写回文件所用的编码和 'encoding' 不同，需要进行转换。支持下面的转换：

- 所有 Latin-1 (ISO-8859-1)，UTF-8，UCS-2 和 UCS-4 之间的转换都是内部完成的。
- MS-Windows 上，如果 'encoding' 是 Unicode 的编码，从/到任何代码页的转换都应该没有问题。
- 'charconvert' 指定的转换。
- 如果可用的话，使用 iconv 库进行的转换。

旧版本的 GNU iconv() 可能会使得转换失败 (它们需要一个很大的缓冲区，比 Vim 能够提供的还有大)。试试获得别的 iconv() 的实现。

iconv-dynamic

MS-Windows 上，Vim 可以带 +iconv/dyn 特性编译。这意味着 Vim 会搜索 "iconv.dll" 和 "libiconv.dll" 库。如果两者都找不到，Vim 依然可以运行，但是某些编码转换就无法完成了。

---

4. 使用终端　　mbyte-terminal

GUI 版本的 Vim 全面支持多字节字符。在终端内使用多字节编码需要终端本身的支持。因此灵活性不高。

举个例子，你可以在支持多字节及/或 XIM 的 xterm 里使用 Vim。这样的终端有：

kterm (Kanji term), hanterm (Korean), Eterm (Enlightened terminal) 和 rxvt。

如果你的终端不支持正确的编码，可以通过设置 'termencoding' 来解决。Vim 会将输入的字符从 'termencoding' 转化成 'encoding'，而把显示的字符从 'encoding' 转化成 'termencoding'。如果终端所支持的编码不包含 Vim 所使用的字符，会导致字符丢失，也会引起显示混乱。如果你的终端支持 Unicode，如以下提到的 xterm，应该就可以正常工作，因为几乎所有的编码都可以无损地转换为 Unicode。

在 XFREE86 XTERM 中 使 用 UTF-8 UTF8-xterm

下面是一个简要的关于如何在 XFree86 所带 xterm 中使用 UTF-8 的说明。XFree86 的作者是 Thomas Dickey (本段来自 Markus Kuhn)。

从以下地址获得最新的支持 UTF-8 的 xterm:

http://invisible-island.net/xterm/xterm.html

带以下参数编译 "./configure --enable-wide-chars;make"

从以下地址获得 ISO 10646-1 所支持的各种字体:

http://www.cl.cam.ac.uk/~mgk25/download/ucs-fonts.tar.gz

并按照 README 文件安装那些字体。

现在，用以下命令启动 xterm

```
xterm -u8 -fn -misc-fixed-medium-r-semicondensed--13-120-75-75-c-60-iso10646-1
```

要使用大一点的字体，用以下命令:

```
xterm -u8 -fn -misc-fixed-medium-r-normal--15-140-75-75-c-90-iso10646-1
```

现在，你就拥有一个支持 UTF-8 的模拟终端了。用以下两种方法测试一下

```
cat utf-8-demo.txt
vim utf-8-demo.txt
```

示例文件来自于 ucs-fonts.tar.gz，它的目的是测试你的 xterm 使用 UTF-8 是否有问题。

对於 Vim 你可能还需设置 'encoding' 为 "utf-8"。

---

5. X11 中的字体 mbyte-fonts-X11

不幸的是，在 X11 下使用字体是非常复杂的。单字节字体的名称是一个长字符串，而多字节字体还有需要多个...

备注: 这里的说明多数和 GTK+ 2 已经无关。那里不支持通过 XLFD 选择字体; 'guifont' 提供了此时如何设置字体的例子。为了你好，忽略下面的 XLFD 和 xfontset 小节。

首先，Vim 显示文本只能使用等宽的字体。你不能使用按比例缩放 (proportionally spaced) 字体，包括许多可用的字体 (也许还很好看)。尽管如此，菜单和工具栏可以使用任何字体。

备注：显示和输入是独立的，就算没有输入你所使用的语言的输入法，你也可能看到你所使用的语言。

你可以使用默认的字体来设置菜单和工具栏，但可能非常丑陋，阅读以下内容，你可以学到怎样选择一个较好的字体。

X LOGICAL FONT DESCRIPTION (XLFD)

XLFD

XLFD 是 X 中使用的包括字体大小，字符集等信息的字体名。格式如下：

```
FOUNDRY-FAMILY-WEIGHT-SLANT-WIDTH-STYLE-PIXEL-POINT-X-Y-SPACE-AVE-CR-CE
```

每个字段的意思是：

- FOUNDRY: FOUNDRY 字段，编写字体的公司名称。
- FAMILY: FAMILY_NAME 字段，基本的字体族名称 (helvetica、gothic、times 等等)
- WEIGHT: WEIGHT_NAME 字段，字母的粗细。(浅 light、中等 medium、粗 bold 等)
- SLANT: SLANT 字段。
    - r: 罗马体 Roman (没有倾斜度)
    - i: 斜体 Italic
    - o: 倾斜体 Oblique
    - ri: 反向斜体 Reverse Italic
    - ro: 反向倾斜体 Reverse Oblique
    - ot: 其他
    - number: 可缩放字体
- WIDTH: SETWIDTH_NAME 字段，字符宽度 (正常 normal、压缩 condensed、窄 narrow、双宽度 double wide 等)
- STYLE: ADD_STYLE_NAME 字段，字体的额外信息 (Serif、Sans-Serif、Informal、Decorated 等等)。
- PIXEL: PIXEL_SIZE 字段，字体以像素计的高度。
- POINT: POINT_SIZE 字段，10 倍字体以 point 数计的高度。
- X: RESOLUTION_X 字段，X 的位数 (每英寸多少点)
- Y: RESOLUTION_Y 字段，Y 的位数 (每英寸多少点)
- SPACE: SPACING 字段
    - p: 比例 (Proportional)
    - m: 等宽 (Monospaced)
    - c: 等宽等高 (CharCell)
- AVE: AVERAGE_WIDTH 字段，10 倍字体的以像素计的宽度。
- CR: CHARSET_REGISTRY 字段。字符集的组名。
- CE: CHARSET_ENCODING 字段。字符集名的其余部分。对有些字符集，比如 JIS X 0208，来说，如果该字段为 0，编码点 (code point) 取 GL 的值， 若为 1，则取 GR 的值。

举个例子，设置大小为 16，对应 JIS X 0208 编码，可以写成以下格式

```
-misc-fixed-medium-r-normal--16-110-100-100-c-160-jisx0208.1990-0
```

X FONTSET

fontset xfontset

单字节字符通常只用一个字体，而对多字节来讲，经常用到字体的组合，这意味着一组字符使用了一种字体，而另一组字符使用别的字体 (也许是双宽度)，字体的集合就叫做字体集 (fontset)。

字体集里的字体依赖于你系统的 locale，X windows 维护着这个 locale 所需要的字符组信息的表，你需要在 'guifontset' 里指定 locale 所需要的全部字体。

设置 'guifontset' 选项意味着所有字体名作为字体集名处理。这也包括 :highlight 命令的 "font" 参数。

注意 'guifont' 和 'guifontset' 的区别：'guifont' 里，逗号分隔的名字是相互替代的名字，只有一个会使用。'guifontset' 里整个字符串是一个字体集名，包括逗号在内。不可以指定相互替代的多个字体集名。
下例在许多 X11 系统上工作：

```
:set guifontset=-*-*-medium-r-normal--16-*-*-*-c-*-*-*
```

这些字体必须和当前 locale 匹配。如果没有包含当前 locale 使用的字符集的字体，设置 'guifontset' 会失败。

备注：字体集总是使用当前的 locale，即使 'encoding' 已设置成另一种编码。在那种情况下，你要设置 'guifont' 和 'guifontwide' 而不是 'guifontset'。

例如：

charset	语言	"字符组"
GB2312	中文（简体）	ISO-8859-1 及 GB 2312
Big5	中文（繁体）	ISO-8859-1 及 Big5
CNS-11643	中文（繁体）	ISO-8859-1、CNS 11643-1 及 CNS 11643-2
EUC-JP	日语	JIS X 0201 及 JIS X 0208
EUC-KR	韩语	ISO-8859-1 及 KS C 5601 (KS X 1001)

你可以用 xlsfonts 命令来查找字体，例如，你需要找 KS C 5601 的一种字体：

```
xlsfonts | grep ksc5601
```

这个比较复杂，也令人迷惑，也许你需要阅读 X-Windows 的文档，它可以帮助你理解你不懂的地方。

base_font_name_list

当你找到你所需要的字体以后，你就可以通过设置 'guifontset' 来设置字体。要使用一系列的字体，你可以用逗号把它们隔开。

例如，如果你要使用 ja_JP.eucJP locale，你需要 JIS X 0201 和 JIS X 0208。你可以提供显式指定字符集的字体列表，就像这样：

```
:set guifontset=-misc-fixed-medium-r-normal--14-130-75-75-c-140-jisx0208.1983-0,
      \-misc-fixed-medium-r-normal--14-130-75-75-c-70-jisx0201.1976-0
```

另外，你也可以设置一个省略编码名的基本字体名列表，让 X-Windows 选择 locale 所需要的字体字符。例如：

```
:set guifontset=-misc-fixed-medium-r-normal--14-130-75-75-c-140,
      \-misc-fixed-medium-r-normal--14-130-75-75-c-70
```

另外，你也可以提供单个基本字体名，让 X-Windows 选择系统可用的所有字体，例如：

```
:set guifontset=-misc-fixed-medium-r-normal--14-*
```

另外，你也可以指定字体别名，请参考 fonts 目录下的 fonts.alias 文件（例如，/usr/X11R6/lib/X11/fonts）。例如：

```
:set guifontset=k14,r14
```

E253
在东亚字体中，正规的字符单元是正方形，当你混合 Latin 字体和东亚字体的时候，东亚字体的宽度应该是 Latin 字体的两倍。

如果 'guifontset' 不为空，:highlight 命令的 "font" 参数也被解释为字体集。例如，你可以设置高亮显示：

```
:hi Comment font=英文_字体,你的_字体
```

如果你使用了一个错误的 "font" 参数，你会收到错误信息。
也要确定你在设置高亮组的字体之前设置 'guifontset'。

使 用 资 源 文 件

除了设置 'guifontset' 以外，你也可以设置 X11 资源，Vim 会从它们中取值。这仅仅对了解 X 资源的人有用。

如果你使用 Motif 和 Athena，把以下三行插入 $HOME/.Xdefaults 文件：

```
Vim.font:  base_font_name_list
Vim*fontSet:  base_font_name_list
Vim*fontList: your_language_font
```

备注：Vim.font 设置文本。
Vim*fontSet 设置菜单。
Vim*fontList 设置 Moitf GUI 的菜单。

举个例子，你使用日语，14 号字体

```
Vim.font: -misc-fixed-medium-r-normal--14-*
Vim*fontSet: -misc-fixed-medium-r-normal--14-*
Vim*fontList: -misc-fixed-medium-r-normal--14-*
```

或者：

```
Vim*font: k14,r14
Vim*fontSet: k14,r14
Vim*fontList: k14,r14
```

要使它们立即生效，你可以这样做

```
xrdb -merge ~/.Xdefaults
```

不然，你要关闭 X 之后重新启动 X 来使它们生效。

GTK+ 版的 GUI Vim 不使用 .Xdefaults，它使用 ~/.gtkrc，大多数都可以正常工作，但菜单可能需要修改。例如：

```
style "default"
```

```
{
        fontset="-*-*-medium-r-normal--14-*-*-*-c-*-*-*"
}
widget_class "*" style "default"
```

---

6. MS-Windows 上的字体 mbyte-fonts-MSwin

最简单的方法就是用对话框来选择字体。你可以在菜单 "Edit/Select Font..." 中找到该对话框。一旦你找到一个好用的字体，你可以使用以下命令来查看它的名字：

```
:set guifont
```

然后你可以在 gvimrc 中添加一个命令来设置 'guifont'：

```
:set guifont=courier_new:h12
```

---

7. X11 上的输入 mbyte-XIM

X INPUT METHOD (XIM) 背景 XIM xim x-input-method

XIM 是 X 的多语种输入模块。有两种构架，Xlib unit 类型和 IM-server (Input-Method server 输入法服务器) 类型。 IM-server 类型适用于复杂的输入，比如 CJK。

- IM-server

  IM-server

  在 IM-server 类型的输入结构中，输入事件的处理有两种方式：FrontEnd 系统和 BackEnd 系统。在 FrontEnd 系统中，输入事件先被 IM-server 捕获，待处理之后， IM-server 为应用程序提供输入结果。BackEnd 系统处理的顺序则完全相反。MS Windows 选择了 BackEnd 系统。在 X 中，大多数的 IM-server 选择 FrontEnd 系统。BackEnd 系统的缺点在于通讯的支出较大，但是它提供比较安全的同步，并且对应用程序没有什么特殊的限制。

  例如，有两种 FrontEnd 系统的日语输入法，xwnmo 和 kinput2 IM-server 。Xwnmo 是和 Wnn 一起发布的 (见下)，kinput2 可以在这里找到：
  ftp://ftp.sra.co.jp/pub/x11/kinput2/

  还有另外一个 XIM 服务器名叫 "xcin"。你可以输入简体和繁体的中文。如果你有合适的输入表，也可以接受其他 locale。Xcin 可以在这里找到：
  http://cle.linux.org.tw/xcin/
  其他的有 scim: http://scim.freedesktop.org/ 和 fcitx: http://www.fcitx.org/

- 转换服务器

  conversion-server

  有的系统需要额外的服务器：转换服务器。大多数日文 IM-server 需要它，即 Kana-Kanji (假名->汉字) 转换器。对于中文输入来说，这取决于输入的方法。有些方法需要拼音或注音到汉字的转换服务器。对于韩语输入来说，如果你想输入汉字 (Hanja)，就需要 Hangul-Hanja (谚文->汉字) 转换器。

  举例来说，日文输入分为两部分。首先，我们预先输入平假名 (Hira-gana)，然后进行 Kana-Kanji (假名->汉字) 的转换。日本汉字很多 (JIS X 0208 标准定义了 6349 个

汉字)，而假名字符的数目只有 76 个。所以，首先我们预输入平假名发音的文本。然后，我们把平假名转换成汉字或者需要的话，片假名 (Kata-Kana)。有很多 Kana-Kanji 的服务器: jserver (随 Wnn 发布，见下) 和 canna。Canna 可以在这里找到:
http://canna.sourceforge.jp/

Wnn4.2 是一个好输入系统。Wnn 4.2 包括，
xwnmo (IM-server)
jserver (日语 Kana-Kanji 转换器)
cserver (中文拼音或者注音到简体汉字转换器)
tserver (中文拼音或者注音到繁体汉字转换器)
kserver (Hangul-Hanja 转换器)
不同系统上的 Wnn 4.2 可以在网上不同地方找到。使用合适你的系统的 RPM 或其它包。

- 输入风格

xim-input-style

在输入 CJK 字符时，有四个屏幕区域:
1. 显示键盘输入的区域
2. 显示当前输入模式的区域
3. 显示备选项的选择区域
4. 显示其它工具的区域

第三个区域在转换时会被用到。例如，在中文输入时，多个中文字符可能拥有同样的发音。这样一个序列的拼音字母就可以唯一的对应一个中文字符。

在 X 的国际化输入中，第一和第二个区被分别称为 "预编辑区" 和 "状态区"。第三和第四个区未被定义，而是留给输入法服务器 IM-server 来控制。在国际化输入中，根据预编辑区与状态区的组合，划分出了四种输入风格: OnTheSpot 、 OffTheSpot 、 OverTheSpot 和 Root 。

目前 GUI Vim 支持三种输入方式: OverTheSpot 、 OffTheSpot 和 Root 。编译时带 +GUI_GTK 特性时，GUI Vim 支持两种输入方式: OnTheSpot 和 OverTheSpot 。可用 'imstyle' 选项选择。

*. on-the-spot OnTheSpot
客户软件在其所用显示区内执行预编辑区和状态区。客户软件由 IM-server 控制在文本插入位置显示所有预编辑所用数据。客户软件注册的回调函数在预编辑时被调用。
*. over-the-spot OverTheSpot
状态区是客户程序显示区内的一个固定的位置。对于 Vim 而言，这个位置是一个另加的状态行。预编辑区出现在程序的当前插入位置。输入法程序在插入位置上方的一个窗口显示预编辑数据。
*. off-the-spot OffTheSpot
预编辑区和状态区都在客户程序的显示区内。对于 Vim 而言，这个区域是一个另加的状态行。客户程序提供输入法程序预编辑所需的显示窗口，而预编辑程序直接控制这些窗口的显示。
*. root-window Root
预编辑区和状态区位于程序之外。输入法程序在一个专有的窗口中显示所有预编辑数据。

使 用 XIM multibyte-input E284 E286 E287 E288 E285 E289

注意 显示和输入是独立的。很可能你能读用你的语言写的文字但无法输入。但是，当输入法和你显示方法不匹配时，文本就不会被正常地显示。

> 备注：你只有在指定了 'guifontset' 之后才能使用 IM。所以，Latin 用户，如果你想用 IM，你也要使用 'guifontset'（GTK2 版本不在限制之内：译者）。

要输入你的语言你应该运行支持你所用语言的输入法服务器 IM-server ，必要时还要运行转换服务器 conversion-server 。

你应该将下面三行放到你的 ~/.Xdefaults 文件中。所有用到 XIM 的 X 应用程序都会用到这些设定。如果你已经用了 XIM ，你可以跳过这一步。

```
*international: True
*.inputMethod: your_input_server_name
*.preeditType: your_input_style
```

input_server_name	是你的 IM-server 名。
your_input_style	是三种输入风格 OverTheSpot ， OffTheSpot ， Root 之一。另见 xim-input-style 。

*international 对于 X11R6 的用户来说是不必要的。
*.inputMethod 和 *.preeditType 对于 X11R6 来说是可选的。

例如，当你用 kinput2 作为 IM-server 时，

```
*international: True
*.inputMethod: kinput2
*.preeditType: OverTheSpot
```

当使用 OverTheSpot 风格时，GUI Vim 即使在普通模式也会连接到 IM 服务器，所以你可以将类似 "f" 和 "r" 等命令与你的语言联用。但当使用另外两种方法时，GUI Vim 只在不处于普通模式时才会连接到 IM 服务器。

如果你的 IM 服务器不支持 OverTheSpot ，而你又想在 Vim 中对你的语言使用普通模式命令 "f" 或 "r"，那么你需要一个本地化的 xterm 或一个支持 XIM 的 xterm。

如果需要，你可以设定 XMODIFIERS 环境变量：

```
sh:  export XMODIFIERS="@im=input_server_name"
csh: setenv XMODIFIERS "@im=input_server_name"
```

例如，当你使用 kinput2 作为 IM-server 而用 sh 作为外壳时，

```
export XMODIFIERS="@im=kinput2"
```

完 全 控 制 XIM

你可以像使用 MS-Windows IME（见 multibyte-ime ）那样完全的控制 XIM。目前此特性仅对 GTK GUI 版本有效。

在使用完全控制之下的 XIM 前，需要设定一个选项：'imactivatekey'。该选项所指定的

热键会被用来激活输入法。例如，当你使用 SCIM 作为 IM Server，激活键很可能是 Ctrl+Space:

```
:set imactivatekey=C-space
```

具体的格式参阅 'imactivatekey'。

---

8. MS-Windows 上的输入 mbyte-IME

(Windows IME 支持) multibyte-ime IME

{仅对 Windows GUI 有效，而且编译时需加入 +multi_byte_ime 特性}

要在 Windows 上输入多字节字符，你可以使用输入法编辑器 Input Method Editor (IME)。在输入的过程中，你必须多次打开／关闭 IME。因为在打开状态的 IME 对任何的输入动作都起作用，你不能在 Vim 中直接使用 'j'，'k' 等几乎所有的键。

+multi_byte_ime 特性在这个问题上很有用。它减少了手动切换 IME 状态的次数。普通状态下几乎不需要 IME，即使是在编辑多字节文本也是如此。因此当你离开插入模式时，Vim 会记住 IME 上次的状态并关闭 IME。当再次进入插入模式时，Vim 会自动的恢复上次的 IME 状态。

这不仅对插入-普通模式有效，同时也对查找命令的输入和替换模式有效。
设置 'iminsert'，'imsearch' 和 'imcmdline' 可以选择不同的输入方法，也可以临时禁用它们。

什 么 是 IME

IME 是东亚版 Windows 的一个部件。用来帮助输入多字节字符。英语和其它语言版本的 Windows 不包括 IME (通常也没有需要)。但是有一个版本叫做 Microsoft Global IME。Global IME 是 Internet Exproler 4.0 或以上的一个组件。你可以在以下的地址找到有关 Global IME 更多的信息。

什 么 是 GLOBAL IME global-ime

Global IME 允许用户在任何版本的 Windows 98，Windows 95，和 Windows NT 4.0 上在 Vim 中输入中文，日文，及韩文文字。
Windows 2000 和 XP 也应该可以用 (无需下载)。在 Windows 2000 Professional 上，Global IME 是内建的，而输入法可以在控制面板/区域选项/输入法里设置。

更详细的请参考下面的地址。 你也可以在这些地方找到不同版本的 Global IME。

- Global IME 详细信息。
  http://search.microsoft.com/results.aspx?q=global+ime

- Active Input Method Manager (Global IME)
  http://msdn.microsoft.com/en-us/library/aa741221(v=VS.85).aspx

对 Global IME 的支持是实验性的。

备注：要使 IME 工作，你需要将输入语言的 locale 加到你的系统里。具体如何操作根据 Windows 的版本不同而不同。比如，在我的 Windows 2000 机器里:
1. 控制面板
2. 区域设置
3. 语言与输入法 (译者注：此处似乎不同版本不同。但大致可以参考)

4. 加入安装的输入语言 -> Chinese(PRC) (中文/中国)
缺省仍然 (可以) 是 English (United Stated) (英语/美国)

当 IME 或 XIM 打开时光标的色彩 CursorIM

这是一个很 cute 的特性。光标可以改变色彩来指示 IME 的状态。通常 IME 的状态是被桌面或任务栏上的一个小图标来指示的。这不是很方便。有了这个特性就容易多了。
XIM 也是如此。

你可以使用高亮组 CursorIM 来选择光标的色彩。例如，在 gvimrc 中加入下面几行:

```
if has('multi_byte_ime')
    highlight Cursor guibg=Green guifg=NONE
    highlight CursorIM guibg=Purple guifg=NONE
endif
```

光标会在 IME 关闭时显示绿色，而在开启时显示紫色。

---

9. 使用键盘映射表输入 mbyte-keymap

如果键盘不能提供你需要输入的字符时，你可以使用 'keymap' (键盘映射表) 选项。它会把一个或多个 (英语) 字符翻译成另外一个 (非英语) 字符。这只会在输入文本，而不是输入 Vim 命令时发生。这样可以避免在两个键盘设置之间来回切换。
{仅当编译时加入 +keymap 特性才有效}

'keymap' 选项的值指定所使用的键盘映射表的文件名。该文件名可以是两者之一:

```
keymap/{keymap}_{encoding}.vim
keymap/{keymap}.vim
```

这里，{keymap} 是 'keymap' 选项的值，而 {encoding} 是 'encoding' 选项的值。带有 {encoding} 的文件名先被尝试。

查找这些文件时，要用到 'runtimepath'。要查看可用的键盘映射表文件，可以:

```
:echo globpath(&rtp, "keymap/*.vim")
```

在插入和命令行模式下，你可以用 CTRL-^ 切换是否使用键盘映射表。
i_CTRL-^ c_CTRL-^
插入模式下的这个标志位记录在 'iminsert' 选项中。每次退出和进入插入模式的时候，该值被记住和再次使用。相同的值也用于需要单个字符输入的命令，如 f 和 r 。
命令行模式的标志位 不 会被记住。因为你需要先输入一个 ASCII 的 Ex 命令。
搜索模式的输入则使用 'imsearch' 选项。它可以设置为 'iminsert' 相同的值。

lCursor

在使用语言映射的时候，GUI 光标可以指定另外的颜色。缺省不使用这个功能，以防你使用非标准的背景颜色时光标不可见。这里是一个使用更亮颜色的例子:

```
:highlight Cursor guifg=NONE guibg=Green
:highlight lCursor guifg=NONE guibg=Cyan
```

keymap-file-format :loadk :loadkeymap E105 E791
键盘映射表的格式如下:

```
" Maintainer:   name <email@address>
" Last Changed: 2001 Jan 1

let b:keymap_name = "short"

loadkeymap
a       A
b       B       注释
```

" 开始的行是注释行，会被忽略掉。空行也被忽略。定义映射的行在有用的文本之后也可以有注释。

"b:keymap_name" 可以是个短的名字，它会在提示行上显示。这样可以减少用于直接显示 'keymap' 的值所需的空间，为了区别不同的语言，键盘和编码，该值可能很长。

实际的映射是 "loadkeymap" 之后的行。在这个例子里，"a" 被映射成 "A" 而 "b" 被映射成 "B"。亦即，第一项被映射成第二项。每一行都如此处理，直到文件末。
这些项和 :lnoremap 命令所用的参数完全相同，可以使用 "<buffer>" 是的映射局部于缓冲区。
你可以用这个命令查看映射的结果：

```
:lmap
```

两项之间必须用空白分隔。你不能在某项内部插入空白，而应该用特殊的名字 "<Tab>" 和 "<Space>"。两项的长度之和不能超过 200 个字节。

第一列可以超过一个字符。这类似于一个死键 (dead key)。例如：

```
'a      *
```

因为 Vim 不知道引号之后的字符是不是真的是 "a"，它会等待下一个字符。要插入单个引号，同时加入以下一行：

```
''      '
```

既然映射是用 :lnoremap 定义的，这里产生的引号不能用来引导另外一个字符。
"accents" 键盘映射表用到这一点。 keymap-accents

第一列也可使用 <> 形式：

```
<C-c>           Ctrl-C
<A-c>           Alt-c
<A-C>           Alt-C
```

注意 取决于键盘和终端的设置，Alt 映射未必可用。

在第二列虽然可以有多个字符，但是并不常见。你可以用多种方式指定所产生的字符：

```
A       a               按本义出现的字符
A       <char-97>       十进制编码
A       <char-0x61>     十六进制编码
A       <char-0141>     八进制编码
x       <Space>         特殊键名
```

字符假定是用 'encoding' 的当前值编码的。如果所有的字符都按本义出现，可以用 ":scriptencoding" 命令。这不适用于 <char-> 的构造，因为转换针对的是键盘映射表文件本身，而不是产生的字符。

"loadkeymap" 之后的行的解释时，'cpotions' 假设为 "C"。这意味着不能用继续行，而反斜杠在映射中有特殊含义。例如：

```
" 注释行
```

```
\"      x       把 " 映射为 x
\\      y       把 \ 映射为 y
```

如果你写了一个键盘映射表文件，而且对别人也有帮助，希望你能把它发给 Vim 的维护者，以便在其后的版本中能够发行：<maintainer@vim.org>

希 伯 来 键 盘 映 射 表 keymap-hebrew

这个文件说明在 UTF-8 和 CP1255 编码中可用的字符，也说明如何使用键盘映射得到这些字符：

字型	编码方式		键盘映射		
字符	utf-8	cp1255	hebrew	hebrewp	名字
	0x5d0	0xe0	t	a	'alef
	0x5d1	0xe1	c	b	bet
	0x5d2	0xe2	d	g	gimel
	0x5d3	0xe3	s	d	dalet
	0x5d4	0xe4	v	h	he
	0x5d5	0xe5	u	v	vav
	0x5d6	0xe6	z	z	zayin
	0x5d7	0xe7	j	j	het
	0x5d8	0xe8	y	T	tet
	0x5d9	0xe9	h	y	yod
	0x5da	0xea	l	K	kaf sofit
	0x5db	0xeb	f	k	kaf
	0x5dc	0xec	k	l	lamed
	0x5dd	0xed	o	M	mem sofit
	0x5de	0xee	n	m	mem
	0x5df	0xef	i	N	nun sofit
	0x5e0	0xf0	b	n	nun
	0x5e1	0xf1	x	s	samech
	0x5e2	0xf2	g	u	'ayin
	0x5e3	0xf3	;	P	pe sofit
	0x5e4	0xf4	p	p	pe
	0x5e5	0xf5	.	X	tsadi sofit
	0x5e6	0xf6	m	x	tsadi
	0x5e7	0xf7	e	q	qof
	0x5e8	0xf8	r	r	resh
	0x5e9	0xf9	a	w	shin
	0x5ea	0xfa	,	t	tav

元音符号和特殊标点：

0x5b0	0xc0	A:	A:	sheva
0x5b1	0xc1	HE	HE	hataf segol
0x5b2	0xc2	HA	HA	hataf patah
0x5b3	0xc3	HO	HO	hataf qamats
0x5b4	0xc4	I	I	hiriq
0x5b5	0xc5	AY	AY	tsere
0x5b6	0xc6	E	E	segol
0x5b7	0xc7	AA	AA	patah
0x5b8	0xc8	AO	AO	qamats
0x5b9	0xc9	O	O	holam
0x5bb	0xcb	U	U	qubuts

```
    0x5bc  0xcc     D       D    dagesh
    0x5bd  0xcd     ]T      ]T   meteg
   0x5be  0xce     ]Q      ]Q   maqaf
    0x5bf  0xcf     ]R      ]R   rafe
   0x5c0  0xd0     ]p      ]p   paseq
    0x5c1  0xd1     SR      SR   shin-dot
    0x5c2  0xd2     SL      SL   sin-dot
    0x5c3  0xd3     ]P      ]P   sof-pasuq
    0x5f0  0xd4     VV      VV   double-vav
    0x5f1  0xd5     VY      VY   vav-yod
    0x5f2  0xd6     YY      YY   yod-yod
```

以下符号只有在 utf-8 下才有

吟咏符号:
字型
字符 utf-8 hebrew 名字

```
    0x591   C:   etnahta
    0x592   Cs   segol
    0x593   CS   shalshelet
    0x594   Cz   zaqef qatan
    0x595   CZ   zaqef gadol
    0x596   Ct   tipeha
    0x597   Cr   revia
    0x598   Cq   zarqa
    0x599   Cp   pashta
    0x59a   C!   yetiv
    0x59b   Cv   tevir
    0x59c   Cg   geresh
    0x59d   C*   geresh qadim
    0x59e   CG   gershayim
    0x59f   CP   qarnei-parah
    0x5aa   Cy   yerach-ben-yomo
    0x5ab   Co   ole
    0x5ac   Ci   iluy
    0x5ad   Cd   dehi
    0x5ae   Cn   zinor
    0x5af   CC   masora circle
```

混合形式:

```
    0xfb20  X‘   Alternative ‘ayin
    0xfb21  X’   Alternative ’alef
    0xfb22  X-d  Alternative dalet
    0xfb23  X-h  Alternative he
    0xfb24  X-k  Alternative kaf
    0xfb25  X-l  Alternative lamed
    0xfb26  X-m  Alternative mem-sofit
    0xfb27  X-r  Alternative resh
    0xfb28  X-t  Alternative tav
    0xfb29  X-+  Alternative plus
    0xfb2a  XW   shin+shin-dot
    0xfb2b  Xw   shin+sin-dot
    0xfb2c  X..W  shin+shin-dot+dagesh
    0xfb2d  X..w  shin+sin-dot+dagesh
```

```
0xfb2e  XA   alef+patah
0xfb2f  XO   alef+qamats
0xfb30  XI   alef+hiriq (mapiq)
0xfb31  X.b  bet+dagesh
0xfb32  X.g  gimel+dagesh
0xfb33  X.d  dalet+dagesh
0xfb34  X.h  he+dagesh
0xfb35  Xu  vav+dagesh
0xfb36  X.z  zayin+dagesh
0xfb38  X.T  tet+dagesh
0xfb39  X.y  yud+dagesh
0xfb3a  X.K  kaf sofit+dagesh
0xfb3b  X.k  kaf+dagesh
0xfb3c  X.l  lamed+dagesh
0xfb3e  X.m  mem+dagesh
0xfb40  X.n  nun+dagesh
0xfb41  X.s  samech+dagesh
0xfb43  X.P  pe sofit+dagesh
0xfb44  X.p  pe+dagesh
0xfb46  X.x  tsadi+dagesh
0xfb47  X.q  qof+dagesh
0xfb48  X.r  resh+dagesh
0xfb49  X.w  shin+dagesh
0xfb4a  X.t  tav+dagesh
0xfb4b  Xo   vav+holam
0xfb4c  XRb  bet+rafe
0xfb4d  XRk  kaf+rafe
0xfb4e  XRp  pe+rafe
0xfb4f  Xal  alef-lamed
```

---

10. 用 imactivatefunc() 输入 mbyte-func

Vim 有 'imactivatefunc' 和 'imstatusfunc' 选项。用于以任意方式来激活/关闭输入法，包含使用外部命令。例如，fcitx 提供了 fcitx-remote 命令：

```
set iminsert=2
set imsearch=2
set imcmdline

set imactivatefunc=ImActivate
function! ImActivate(active)
  if a:active
    call system('fcitx-remote -o')
  else
    call system('fcitx-remote -c')
  endif
endfunction

set imstatusfunc=ImStatus
function! ImStatus()
  return system('fcitx-remote')[0] is# '2'
endfunction
```

使用此脚本，即使编译时没有带 +xim 也能通过 Vim 激活/关闭 XIM。

---

11. 使用 UTF-8

mbyte-utf8 UTF-8 utf-8 utf8 Unicode unicode

Unicode 字符集的设计就是要包含所有其他的字符集。这样你就可以用 Unicode 写任何文字（除了少数偏僻的语种以外）。它也最大限度的提供了在一个文件里混合各种语言的能力，这用其他编码是不可能的。

Unicode 可以有几种编码方式。最流行的是 UTF-8 编码，它使用一到多字节编码单个字符，和 ASCII 后向兼容。MS-Windows 上也使用 UTF-16（以前的 UCS-2）编码，它使用 16-位字。Vim 支持所有的编码，但内部使用的总是 UTF-8。

Vim 有完整的 UTF-8 支持。它在以下环境下工作得很好：
- 带有 utf-8 支持的 xterm
- Athena、Motif 和 GTK GUI
- MS-Windows GUI
- 若干其它的平台

支持双宽度的字符。如能配合 'guifontwide' 和 'guifontset' 最好。如果只使用 'guifont'，宽字符使用正常宽度显示，而空格用来填补之间的空隙。注意 'guifontset' 选项不再用于 GTK+ 2 GUI。

bom-bytes

读文件时 BOM（字节顺序标识）可以用来识别 Unicode 编码方式：

```
EF BB BF      utf-8
FE FF         utf-16 big endian (高位字节在前)
FF FE         utf-16 little endian
00 00 FE FF   utf-32 big endian
FF FE 00 00   utf-32 little endian
```

utf-8 是推荐的编码方式。注意 很难把 utf-16 和 utf-32 区别开。utf-16 常用于 MS-Windows，utf-32 并不是常用的文件格式。

mbyte-combining mbyte-composing

组合字符用来改变前面字符的含义。组合字符显示在前导字符之上。
缺省至多可以使用两个组合字符。但可以用 'maxcombine' 选项改变其最大值。
编辑文本时，组合字符多数被认为是前导字符的一部分。例如，"x" 缺省删除一个字符和它后面的组合字符。
如果 'delcombine' 选项打开，那么 'x' 第一次删除一个组合字符，下一次再删除基本字符。但在插入时，你先输入第一个字符，然后再输入后面的组合字符，最后它们被合并在一起。不能用 "r" 命令输入组合字符，因为它不知道这些字符会不会出现。应该用 "R"。

不属于合法 UTF-8 编码的字符序列并作为单个字符处理，并显示为 <xx>，其中的 "xx" 是该字节的十六进制值。

过长的序列不会被特殊处理，显示的和合法的字符差不多。不过，模式搜索不会匹配过长的序列（过长序列指包含多过字符的编码需要的字节）。一个例外是 NUL（零）被显示为 "<00>"。

在文件和缓冲区里，可以使用 Unicode 字符的整个范围（31 位）。不过，在显示的时候只限于当前所选字体支持的字符。

有用的命令:
- "ga" 显示光标下字符的十、十六、和八进制值。如果有组合字符，也一并显示。(如果消息被截断，用 ":messages" 命令)。
- "g8" 显示 UTF-8 字符的相应字节，包括组合字符，以十六进制表示。
- ":set encoding=utf-8 fileencodings=" 强制为所有文件使用 UTF-8。缺省使用 'encoding' 的当前 locale，并自动设置 'fileencodings' 为文件使用的编码。

启 动 VIM

如果你当前的 locale 就是 utf-8，Vim 会自动以 utf-8 模式启动。

如果你使用别的 locale:

```
set encoding=utf-8
```

你也需要设置菜单所需的字体。不幸的是这不总是能行。参见以下特定系统的参考，和 'langmenu' 选项。

在 X-Windows 上 使 用 UTF-8 utf-8-in-xwindows

注意: 本节不适用于 GTK+ 2 GUI。

你需要指定要用的字体。如果需要显示双宽度的字符，还要指定两倍宽度的另一个字体。有三种指定的方法:

1. 设置 'guifont' 并让 Vim 找到合适的 'guifontwide'
2. 设置 'guifont' 和 'guifontwide'
3. 设置 'guifontset'

参见每个选项的文档以了解详情。示例:

```
:set guifont=-misc-fixed-medium-r-normal--15-140-75-75-c-90-iso10646-1
```

你可能需要设置菜单所需的字体。目前，这只适用于 Motif。你可以用 ":hi Menu font={fontname}" 命令。 :highlight

输 入 UTF-8 utf-8-typing

如果你使用 X-Windows，你应该找一下能支持 utf-8 的输入法。

如果你的系统不支持 utf-8 的输入，你可以用 'keymap' 特性。通过写一个键盘映射表文件，你可以用 ASCII 字符的序列来定义 utf-8 的字符。见 mbyte-keymap 。

另外一个方法是设置当前的 locale 为你需要的语言，并且提供该语言相应的 XIM。然后设置 'termencoding' 为该语言，Vim 会把输入的语言转换成 'encoding' 指定的编码。

如果这些都不行，你可以用四位十六进制输入任何字符:

```
CTRL-V u 1234
```

"1234" 被解释成十六进制数。你必须要输入四位字符，有必要的话前面加零。

命 令 行 参 数 utf-8-char-arg

f 、 F 、 t 和 r 这样的命令接受单个字符作为参数。UTF-8 的字符可能意味着该参数需要包括一到两个组合字符。它们需要和基本字符组合。但是，Vim 不会等待下个字符的输入然后再判断它是不是组合字符。'keymap' 或者 :lmap 是一个输入这些字符的好办法。

在行中搜索字符的命令这样处理组合字符。当搜索不带组合字符的字符时，带或不带组合字符的文本都可能被搜索到。当搜索带组合字符的字符时，只会搜索到带组合字符的文本。之所以这样实现，是因为不是每个人都能输入组合字符。

---

12. 选项总述 mbyte-options

以下的各选项会影响对多字节文件的编辑。更详细的信息请参考 options.txt。

'encoding' 键盘和显示使用的编码。同时也是文件的缺省编码。

'fileencoding' 某一文件的编码。在该选项的值与 'encoding' 不同的情况下，读写文件时会有编码转换。

'fileencodings' 文件可能编码的列表。当打开一个文件时，Vim 会尝试该列表并使用第一个找到的无错的编码。'fileencoding' 会被设为该编码。

'charconvert' 用于将文件转码的表达式。

'formatoptions' 'm' 标志位可以用来改变文本断行的方式：在一个多字节字符处可以断行。这对与那些可以在任意字符处断行的语言很有用。

'guifontset' 一个用于多字节编码的字体的列表。当该选项值不为空时，'guifont' 被它取代。

'keymap' 给定所用键盘映射表名字。

对多字节功能作出特别贡献的有：

Chi-Deok Hwang <hwang@mizi.co.kr>
SungHyun Nam <goweol@gmail.com>
K.Nagano <nagano@atese.advantest.co.jp>
Taro Muraoka <koron@tka.att.ne.jp>
Yasuhiro Matsumoto <mattn@mail.goo.ne.jp>

vim:tw=78:ts=8:noet:ft=help:norl:

mlang.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　by Bram Moolenaar
译者：jwdu <junweidu@ustc.edu>，iCrazy <icrazy@ustc.edu>
http://vimcdoc.sf.net

多语言特性　multilang　multi-lang

这里讲的是使用各种不同语言的信息和菜单。如果希望编辑多字节文本请参看 multibyte 。

基本的使用请参看用户手册 usr_45.txt 。

1. 信息　multilang-messages
2. 菜单　multilang-menus
3. 脚本　multilang-scripts

同时请参看 help-translated 阅读有关多语言帮助使用。

{仅当编译时加入 +multi_lang 特性才有效}

---

1. 信息　multilang-messages

Vim 从环境变量中获取当前 locale 信息。这意味着在大部分的情形下，Vim 将使用你喜欢的语言，除非这种语言不存在。

为了知道你的系统所支持的语言列表，可以查看以下的路径（针对 Unix）：
　　/usr/lib/locale
　　/usr/share/locale
不幸的是，大小写是区分的。同时还要注意 "-" 和 "_" 不要用混了。

（译者注：Windows 上，简体/繁体中文对应语言是 chinese_china 和 chinese_taiwan，它们在内部被翻译为 zh_CN 和 zh_TW。换而言之，可以用
　　:language chinese_china " 不能直接用 zh_CN
但
　　:echo v:lang
　　zh_CN
这一翻译机制目前来自硬编码，因而看来没有 UTF-8 对应形式的支持。）

:lan　:lang　:language　E197

:lan[guage]
:lan[guage] mes[sages]
:lan[guage] cty[pe]
:lan[guage] tim[e]
显示当前的语言（即 locale）。
使用 "messages" 参数会显示出用作消息的语言。术语：LC_MESSAGES。
使用 "ctype" 参数，会显示出用作字符编码的语言。术语：LC_CTYPE。
使用 "time" 参数会显示出用作 strftime() 的语言。术语：LC_TIME。

不加参数时显示当地语言的全称（这个功能依赖于系统）。
当前的语言同样也可以通过变量 v:lang 、 v:ctype 和 v:lc_time 来获得。

:lan[guage] {name}
:lan[guage] mes[sages] {name}
:lan[guage] cty[pe] {name}
:lan[guage] tim[e] {name}
把当前的语言（即 locale）设置成 {name}。
此处的 {name} 必须是你系统上的一个有效的语言。一些系统接受像 "en" 或 "en_US" 这样的别名，但是，一些系统只接受像 "en_US.ISO_8859-1" 这样完整的名称。Unix 系统上可以用以下命令查看支持的 locale:

```
:!locale -a
```

借助于参数 "messages"，你就可以设置消息使用的语言。如果你希望的话，这两者可以不同，例如：当你编辑日文文本时使用英文消息。它设置 $LC_MESSAGES。
借助于参数 "ctype"，你就可以设置用作字符编码的语言。这将会影响到 Vim 所连接的库。一般不能将它和 'encoding' 设成不同的值，除非是 "C"。它设置 $LC_CTYPE。
借助于参数 "time" 参数，你就可以设置用作时间和日期消息的语言。它影响到 strftime()。它设置 $LC_TIME。
没有参数时，以上两者都会被设置，另外还会设置 $LANG。
编译时加上 +float 特性时，LC_NUMERIC 值总会被设为 "C"，以便浮点数使用 '.' 作为十进制小数点。
这将会使那些依赖语言的项目（信息，时间和日期的格式）看起来有一些不同。这不是所有系统都完全支持的。
如果设置失败，将会返回一个出错信息。如果设置成功，将不会有任何信息。例如：

```
:language
Current language: C
:language de_DE.ISO_8859-1
:language mes
Current messages language: de_DE.ISO_8859-1
:lang mes en
```

MS-WINDOWS 信息的翻译 win32-gettext

如果你用了自安装的 .exe 文件，应该不需要什么设置。否则你可能找不到 libintl.dll 文件。可以从下面的网站得到它：

```
http://sourceforge.net/projects/gettext
```

或：

```
https://mlocati.github.io/gettext-iconv-windows/
```

这里还包含了 xgettext，msgformat 以及其它工具。

libintl.dll 应该和 (g)vim.exe 放在同一个目录内，或者放在环境变量 PATH 指定的那些目录之一。Vim 也会查找备用名 "libintl-8.dll" 和 "intl.dll"。

消息数据文件（vim.mo）应放在 "$VIMRUNTIME/lang/xx/LC_MESSAGES"，在这里 "xx" 是语言的缩写（多数是两个字母）。

如果你制作了自己的翻译，那么，你需要生成 .po 文件并把它转化为 .mo 文件。同时，找到 Vim 的源代码并阅读 "src/po/README.txt" 文件。

如果你不想使用自动选择的语言，那么就把 $LANG 变量设为你希望的语言。如果你不希望使用语言翻译，请设置成 "en"。

```
:let $LANG = 'ja'
```

(Windows 的文本由 Muraoka Taro 完成)

---

2. 菜单 multilang-menus

基本的使用请参看 45.2 ，尤其是 'langmenu' 的使用部分。

注意 如果在翻译后对菜单作了更改，菜单的一些选项可能会显示成英文。请联系翻译的维护者并请求他更新菜单的翻译。你可以在 "$VIMRUNTIME/lang/menu_<lang>.vim" 中找到翻译者的名字和 e-mail 地址。

为菜单设定字体（或字体集），请用 :highlight 命令。例如：

```
:highlight Menu font=k12,r12
```

LOCALE 的别名

不幸的是，即使使用相同的语言和编码，不同系统上的当地语言名称也可能是不同的。如果你没有得到你需要的菜单翻译，请检查下面命令的输出：

```
echo v:lang
```

现在，检查 "$VIMRUNTIME/lang" 目录中有没有使用相似语言的翻译文件。注意 符号 "-" 和 "_" 的区别，它可能是找不到文件的罪魁祸首！另外一个通常要注意的差别是 "iso8859-1" 和 "iso_8859-1"。所幸的是，Vim 把所有的名字都转换成小写了。因此，你没有必要担心大小写的差别。为了避免空格被忘掉，它们被转化成下划线。

如果你发现了一个不同文件名的菜单翻译，你可以在自己的 runtime 目录下建立一个文件并装载之。这个文件的名字可以是：

```
~/.vim/lang/menu_<v:lang>.vim
```

检查 'runtimepath' 选项以确定会搜索哪些目录。在此文件中写一个命令来装载那个使用不同名字的菜单文件：

```
runtime lang/menu_<other_lang>.vim
```

翻译菜单

如果你想自己翻译菜单，你可以使用 :menutrans 命令，下面有一些说明。推荐把每种语言的翻译放到单独的 Vim 脚本文件中。如果一种语言还没有被翻译，你可以成为它的维护者，并把你的翻译贡献出来，以便能让其它 Vim 用户使用。如果你要这样的意愿的话，请发 e-mail 给 Vim 的维护者：<maintainer@vim.org>。

:menut :menutrans :menutranslate

:menut[ranslate] clear
清除所有的菜单翻译。

:menut[ranslate] {english} {mylang}
将菜单名称 {english} 翻译成 {mylang}。像 "&" 和 "<Tab>" 这样的特殊字符也必须包括进去。如同其它 :menu 命令中的那样，空格和句点需要用反斜杠来转义。
忽略 {english} 中的大小写。

你可以在 $VIMRUNTIME/lang 目录中找到例子。

为了检验你的翻译，首先你需要清除所有的菜单。用下面的方法你可以不重新启动 Vim 就达到这样的效果：

```
:source $VIMRUNTIME/delmenu.vim
:source <your-new-menu-file>
:source $VIMRUNTIME/menu.vim
```

菜单路径的每一个部分是被独立的翻译的。这就意味着：如果 "Help" 被翻译成 "帮助"，"Overview" 被翻译成 "总览"，那么 "Help.Overview" 就会被翻译成 "帮助.总览"。

---

3. 脚本 multilang-scripts

在 Vim 脚本中，你可以用 v:lang 变量获得当前的 locale。缺省值是 "C" 或者来自于环境变量 $LANG。

以下这个例子展示了这个变量的一个简单的应用，它可以显示出针对用户喜好语言的信息：

```
:if v:lang =~ "zh_CN"
:  echo "早上好"
:else
:  echo "Good morning"
:endif
```

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

rileft.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Avner Lottem

更新：Nadim Shaikli
译者：Willis
http://vimcdoc.sf.net

Vim 的从右到左显示模式 rileft

这些功能是由 Avner Lottem 最早创建的：

E-mail: alottem@iil.intel.com
Phone: +972-4-8307322

E26

{仅当编译时加入 +rightleft 特性才可用}

简介

有些语言，例如阿拉伯语、波斯语、希伯来语（还有其他），要求从右到左显示他们的文字的能力。这些语言的文件以常规的方式储存，从右到左只是一个对显示功能的要求（根据 Unicode 规格的定义）：设定为从右到左的文件里字符在屏幕的显示应是从右到左。

双向显示（英语缩写是 bidi）是 Unicode 提供的对这些语言的一个完整解决方案。bidi 提供用户在同一个窗口同时显示从右到左和从左到右文本的能力。目前，为了简单起见，Vim 并没有提供 bidi，而只是提供一个方法来显示/输入/使用从右到左的语言。一个较旧的混合解决方案是为每个字符（或一组字符）编码一个不同的方向。我们不支持这个方案，因为这样的支持对于一个已有的编辑器来说不是简单的增加功能而已（而且 Unicode 也不认可这种方案）。

特色

- 就像原有的 Vim 一样，编辑从左到右的文件。
- 在从右到左的窗口里察看和编辑文件。文件朝向是窗口的设置，所以可以在从左到右和从右到左两种模式下同时察看同一个文件。（这有助于编辑混合两种文字的文件）。
- 和原有的 Vim 兼容。几乎所有的功能都支持从右到左的模式（参看下面的漏洞）。
- 从反向的插入模式退回到文件里正确的位置（如果可能的话）。
- 不需要终端对从右到左能力的特别支持。这项改动完全独立于硬件。
- 许多语言使用和需要从右到左的支持。这些语言可以相当容易地通过加入他们需要的键盘映射和少量的代码改进得到支持。其中已经支持的语言包括 - arabic.txt 、farsi.txt 和 hebrew.txt 。

如果你有兴趣 ...

---

o 启动

+ 'rightleft' ('rl') 把窗口朝向设为从右到左。
+ 'delcombine' ('deco')，布尔值，如果编辑 UTF-8 编码的语言，我们可以单独删除组合字符，它被附加 (superimpose) 于在它之前的字符之上 (一些语言要求如此)。
+ 'rightleftcmd' ('rlc') 在某些模式 (比如：搜索) 下，把命令行也设成从右到左的朝向。

o 反向输入 ins-reverse

为便于最大限度地利用 'rightleft' 选项，我们可以允许反向插入。如果 'revins' (反向输入) 选项被打开，插入是反向的。这可以用来输入从右到左的文字。这时，输入文字时，光标不动，文本向右移动。<BS> 删除光标下的字符，CTRL-W 和 CTRL-U 工作的方向也经过反转。无论 'backspace' 选项如何设置，<BS>、CTRL-W 和 CTRL-U 并不会在插入点开始处和行尾停住。

现在 (还) 没有反向替换模式。

如果设置了 "showmode" 选项，反向插入模式激活时，状态行会显示 "-- REVERSE INSERT --"。

o 在从右到左的窗口粘贴

如果用鼠标剪切文本，然后在从右到左的窗口中粘贴，文本会被反向，这时因为从剪切缓冲区里得到的字符是从左到右的，而文件中的插入操作是从右到左的。要避免这个问题，在粘贴前切换 'revins' 选项。

漏洞

---

o 在从右到左的窗口里不能正确处理 CTRL-A 和 CTRL-X 命令 (加和减)。

o 在命令行中不支持反向插入和从右到左的模式。不过，编辑器功能并未减少，因为可以在命令行上从左到右地输入键盘映射、缩写和搜索命令。

o 从右到左的模式较慢，因为从右到左是 Vim 自己模拟而不是控制终端处理的。

o 使用 Athena GUI 时，底部的滚动条工作方向是错的。这很难修正。

o 如果 'rightleft' 和 'revins' 都打开，'textwidth' 不能工作。行不会自动回绕。你只会得到一个很长的行。

o 没有完整的双向显示 (bidi) 支持。

vim:tw=78:ts=8:noet:ft=help:norl:

arabic.txt For Vim version 8.2. 最近更新: 2019年12月

VIM 参考手册 by Nadim Shaikli
译者: Willis
http://vimcdoc.sf.net

Vim 的阿拉伯语支持 (选项和映射) Arabic

为了使用从右到左和阿拉伯语的映射支持，需要编译 Vim 时加入 +arabic 特性。 E800

这些功能由 Nadim Shaikli <nadim-at-arabeyes.org> 编写。

本文件最好在 Vim 的 GUI 版本里用以下设置阅读:

```
:set encoding=utf-8
:set arabicshape
```

简介

阿拉伯语是一个需要多项特殊特性的要求很高的语言。字符是从右到左导向的，应该在屏幕上从右到左的显示。阿拉伯语也要求字型重整: 相同的字符出现在词的不同相对位置 (开头，中间，结尾，或者单独出现) 时有不同的字型。阿拉伯语也要求两种不同的组合形式和在一些情况下，把多达两个的字符附加在 (superimpose) 别的字符上的能力 (合成) 或者实际把两个字符替代成一个 (组合) 的能力。最后，要正确地显示阿拉伯语，需要 ISO-8859-6 (U+0600-U+06FF) 字体和 B-表现型 (U+FE70-U+FEFF) 字体，两者都是所谓的 ISO-10646-1 字体的一部分。

命令、提示和帮助文件不是用阿拉伯语书写的，以使用户界面保持和标准 Vi 的界面一致。

特色

- 就像原有的 Vim 一样，编辑从左到右的文件。
- 和从右到左两种模式下同时察看同一个文件。(这有助于编辑混合两种文字的文件)。
- 不需要终端对从右到左能力的特别支持。这项改动完全独立于硬件。只需要阿拉伯语的字体。
- 和原有的 Vim 兼容。几乎所有的功能都支持从右到左的模式 (除了一些漏洞以外)。
- 单个命令就可以改变键盘映射和反向插入模式。
- 从反向的插入模式退回到文件里正确的位置 (如果可能的话)。
- 单个命令可以切换完全的阿拉伯语支持。

- 在阿拉伯语模式，数字的输入是从左到右的。在输入一个非数字的字符时，它会插在最后一个数字的左面。

- 在命令行的反向插入模式里，提供阿拉伯语的映射。

- 如果 Vim 在一个有双向显示 (Bidi) 能力的终端模拟器中启动，那么可能有一定的双向显示能力。

阿拉伯语字体 arabicfonts

Vim 要求等宽字体，这已经有很多了。阿拉伯语要求有 ISO-8859－6 和 B-表现型 (Presentation Form-B) 字体 (没有 B-型，阿拉伯语支持是 不能 用的)。强烈推荐用户寻找所谓的 'ISO-10646-1' 字体。在因特网上找找看，或者查看 www.arabeyes.org，了解更多关于哪里可以得到所需的阿拉伯字体的信息。

字体安装

- 在 X Window 系统下 (Unix/Linux) 安装字体

  根据你的系统，把你的 阿拉伯 字体文件复制到你选择的一个目录下。转到这个目录，然后执行如下命令:

```
% mkfontdir
% xset +fp path_name_of_arabic_fonts_directory
```

使用方法

在实际在 Vim 使用阿拉伯语之前，要考虑到并完成一些设置。

- 设置阿拉伯字体

  + 对于 Vim GUI 而言，把 'guifont' 设成你的 阿拉伯 字体。这是通过在 Vim 窗口输入以下命令完成的。

```
:set guifont=你的_阿拉伯_字体
```

    注意: 字符串 '你的 阿拉伯 字体' 应该用一个完整的 linux/unix 系统下的字体名字来代替 (比如，
    -misc-fixed-medium-r-normal--20-200-75-75-c-100-iso10646-1)。

    你可以在 .vimrc 文件里加上 'guifont' 的设置命令以得到相同的结果。换句话说，在你的 .vimrc 文件里加上 ':set guifont=你的 阿拉伯 字体'。

  + 在 X Window 环境上，你也可以在启动 Vim 时加上 '-fn 你的 阿拉伯 字体' 选项。

- 设置合适的编码
  要打开正确的阿拉伯语设置，下面的命令:

```
:set encoding=utf-8
```

应该加在你的 .vimrc 文件里（最好不要在 Vim 窗口的命令行里手工输入）。简单的说，把 ':set encoding=utf-8' 加到你的 '.vimrc' 文件里。

试图在没有 UTF-8 的时候使用阿拉伯语会得到以下的警告消息，

```
                                                              W17
Arabic requires UTF-8, do ':set encoding=utf-8'
```

o 进入阿拉伯语的设置 [快捷方式]

为了把事情简化和流水线化，你可以有两种方案：或者如下启动 Vim：

```
% vim -A my_utf8_arabic_file ...
```

或者在 Vim 里执行如下的命令：

```
:set arabic
```

建议用户采取这两种可能的激活方式。在没有打开 'termbidi' 设置的情况下，两者都：

1. 设置相应的键盘映射
2. 允许删除一个单独的组合字符对
3. 打开从右到左模式
4. 打开从右到左的命令行模式（影响命令行）
5. 打开阿拉伯字型重整模式（进行字型的替换）

你也可以在 .vimrc 文件里这么做，只要增加这一行 ':set arabic' 就行了。

你也可以用如下方式关闭阿拉伯语的支持：

```
:set noarabic
```

它会关闭上条打开命令所做的一切，而不影响全局设置，从而也就不影响别的打开的缓冲区。简单地说，'noarabic' 命令：

1. 关闭替换了的键盘映射
2. 关闭删除单独的组合字符对的功能
3. 关闭从右到左模式

注意：为了支持外部的双向显示 (bidi) ("mlterm" 就是一个例子)，'arabic' 命令考虑到 'termbidi' 设置。因为双向显示比从右到左更好，提供的功能更多，如果这个设置存在，我们优先使用它。这时，'arabic' 只修改键盘映射。

从另一方面说， 如果你喜欢详细而具体的操作，你可以不用 'arabic' 的快捷方式命令，这里是你需要做的（换而言之，如果你用 ':set arabic' 就可以不用看了）-

+ 阿拉伯语的键盘映射激活

  要激活阿拉伯语的键盘映射（也即，重新映射你的英语/拉丁语系键盘到一个类似于标准阿拉伯的键盘），把 'keymap' 命令设置成 "arabic"。这可以由在你的 Vim 窗口中输入以下命令完成。

```
:set keymap=arabic
```

你可以把 'keymap' 设置命令加到你的 .vimrc 文件里。也就是说，把 ':set keymap=arabic' 加到你的 .vimrc 文件里。

要在你的缺省映射（英语）和阿拉伯语的键盘映射间切换，用户最好在插入（或者加入/替换）模式下按 'CTRL-^'。命令行下会在插入模式后面提示当前映射为阿拉伯状态（例如，-- INSERT Arabic --）。

+ 阿拉伯的组合字符对的删除

缺省时，Vim 关闭 'delcombine' 选项。这个选项允许删除 LAM_ALEF（LAA）组合字符里的 ALEF，但还保留那个 LAM（也就是说，把组合字符恢复到原来的两个字符的形式 — 这也适用于 harakat 和他们的组合形式）。你可以在你的 Vim 窗口下打开这个选项

```
:set delcombine
```

你可以把 'delcombine' 设置命令加到你的 .vimrc 文件里。也就是说，把 ':set delcombine' 加到你的 .vimrc 文件里。

+ 阿拉伯的从右到左模式

缺省时，Vim 以从左到右模式启动。'rightleft' 是改变窗口朝向的命令 — 有如下的设置方式，

- 在从左到右和从右到左模式间切换可以用 ':set rightleft' 和 ':set norightleft'。
- 在从左到右模式时，在命令行里输入 ':set rl'（'rl' 是 rightleft 的缩写）。
- 把 ':set rl' 一行放到你的 '.vimrc' 文件里，就可以一劳永逸的以从右到左模式启动 Vim。

+ 阿拉伯的从右到左的命令行模式

有些命令的编辑可以在从右到左的模式下完成。目前，这只对搜索命令适用。

控制这点的是 'rightleftcmd' 选项。缺省的是 "search"，也就是说在置位了 'rightleft' 的窗口里会以从右到左的模式编辑搜索命令。要关闭它，

```
:set rightleftcmd=
```

要重新打开从右到左方式对搜索命令进行编辑，

```
:set rightleftcmd&
```

+ 阿拉伯的字型重整模式

要激活阿拉伯语所需的字型替换（重整、合成、组合），打开 'arabicshape' 命令。这可以通过在我们的 Vim 窗口下执行

```
:set arabicshape
```

你可以把 'arabicshape' 设置命令加到你的 .vimrc 文件里。也就是说，把 ':set arabicshape' 加到你的 .vimrc 文件里。

键盘映射/键盘 arabickeymap

Vim 的字符/字母编码是标准的 UTF-8。我不希望你用任何别的编码形式。

注意: UTF-8 是一个包罗万象的编码，因此也是阿拉伯语唯一支持（和鼓励使用）的编码（不鼓励并反对用任何别的私有编码）。

o 键盘

+ 插入/替换模式下的 CTRL-^ 切换阿拉伯/拉丁语系模式。

+ 键盘映射基于 Microsoft 的键盘映射表（阿拉伯世界的实际标准）：

```
+-----------------------------------------------------------------------+
|!   |@   |#   |$   |%   |^   |&   |*   |(   |)   |_   |+   ||   |~   |
|1   |2   |3   |4   |5   |6   |7   |8   |9   |0   |-   |=   |\   |‘   |
+-----------------------------------------------------------------------+
     |Q   |W   |E   |R   |T   |Y   |U ‘ |I ÷ |O x |P   |{ < |} > |
     |q   |w   |e   |r   |t   |y   |u   |i   |o   |p   |[   |]   |
     +-------------------------------------------------------------+
       |A   |S   |D [ |F ] |G   |H   |J   |K   |L / |:   |"   |
       |a   |s   |d   |f   |g   |h   |j   |k   |l   |;   |’   |
       +-------------------------------------------------------+
         |Z ~ |X   |C { |V } |B   |N   |M ’ |< , |> . |?   |
         |z   |x   |c   |v   |b   |n   |m   |,   |.   |/   |
         +-----------------------------------------------+
```

限制

---

o Vim 的 GUI 形式目前不支持双向显示（也就是，能在一行里同时看到阿拉伯语和拉丁语系的混合文字）。

已知漏洞

有一个已知的小漏洞：

1. 如果你在一个 LAM (U+0644) 之后插入一个 haraka（例如，Fatha (U+064E)），然后再插入一个 ALEF (U+0627)，期望的组合不会发生，因为夹在中间的 haraka 产生了一些 不 会正确显示的东西。

   临时解决方法：不要在 LAM 和 ALEF 组合之间插入 harakat。基本上，不要期望看到 harakat 和 LAM+ALEF 组合的字符的正确显示（即使是它们之后的字符也有问题）。这个问题纯粹是显示上的，在文件里存储的内容包含所有的信息/编码，无一遗漏。

现在还不知道别的漏洞。

vim:tw=78:ts=8:noet:ft=help:norl:

farsi.txt For Vim version 8.2. 最近更新: 2019年12月

VIM 参考手册 by Mortaza Ghassab Shiran
译者: Willis
http://vimcdoc.sf.net

从右到左的显示和 Vim 的波斯语映射 farsi Farsi

E27
8.1.0932 补丁删除了波斯语支持。那时，此功能已过时且无人使用。

如果你想把波斯语支持加回来，请看一下 8.1 发布版本中的旧的波斯语代码。它应和带 Unicode 字符集的阿拉伯语支持合并。

vim:tw=78:ts=8:noet:ft=help:norl:

hebrew.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　　by Ron Aaron (and Avner Lottem)
译者：Willis
http://vimcdoc.sf.net

Vim 的希伯来语支持（选项和映射）　　hebrew

'rightleft' 的功能是由 Avner Lottem <alottem at gmail dot com> 最早创建的。
Ron Aaron <ron at ronware dot org> 现在为支持这些功能提供帮助。

{仅当编译时加入 +rightleft 特性才有效}

简介

希伯来语专用的选项包括 'hkmap'、'hkmapp'、'keymap'=hebrew 和 'aleph'。
对希伯来语有用的选项包括 'delcombine'、'allowrevins'、'revins'、'rightleft'
和 'rightleftcmd'。

'rightleft' 模式反转显示的顺序，所以字符不是通常的从左到右，而是从右到左的显示。这主要是对希伯来语和其它中东的语言有用。参阅 rileft.txt 去了解更多。

细节

+ 选项：
  + 'rightleft' ('rl') 把窗口朝向设为从右到左。这意味着逻辑上的文字 'ABC' 会被显示成 'CBA'。文字将从窗口右边界开始输出，而不是左边。
  + 'hkmap' ('hk') 在插入/替换模式中把键盘映射设成希伯来方式。
  + 'aleph' ('al')，数值，保存 Aleph 的十进制代码，键盘映射用。
  + 'hkmapp' ('hkp') 把键盘映射设成 '根据读音的希伯来语'。

  注意：这三个选项 ('hkmap'、'hkmapp' 和 'aleph') 现在已经作废。你应该用 ":set keymap=hebrewp" 来代替。

  + 'delcombine' ('deco')，布尔值，如果编辑 UTF-8 编码的希伯来文字，可以在一个字符上（带有 niqud ）上按 'x' 来去掉 niqud 或者 te'amim。

  + 'rightleftcmd' ('rlc') 从右边开始显示用于搜索的命令行提示，这只适用于 'rightleft' 的窗口。

+ 编码：
  + Unix 上，ISO 8859-8 编码（希伯来字母编码：224-250）。
  + MS DOS 上，PC 编码（希伯来字母编码：128-154）。
    这些都是缺省值，可以用 'aleph' 选项来覆盖。
  + 你应该选用 UTF8，因为它支持组合字符（如果不是 UTF8 编码，'deco' 什么都不做）。

+ Vim 参数：
  + 'vim -H file' 启动希伯来文件 file 的编辑，也就是，打开 'rightleft' 和

'hkmap'。

+ 按键：
  + 'allowrevins' 选项使得 CTRL-_ 命令在插入和命令行模式下可用。

  + 插入/替换模式下的 CTRL-_ 切换 'revins' 和 'hkmap'：

    在从右到左的窗口里，'revins' 和 'nohkmap' 被切换，因为可能输入英语。

    在从左到右的窗口里，'revins' 和 'hkmap' 被切换，因为可能输入希伯来语。

    CTRL-_ 把光标移到输入文本的结尾。

  + CTRL-_ 在命令行模式下切换键盘映射（参见下面的漏洞）。
    这个设置和 'hkmap' 无关，后者只适用于插入/替换模式。

    注意：在有些键盘上，CTRL-_ 被映射成 CTRL-?。

  + 'hkmap' 设置时，键盘的映射如下（标准以色列键盘）：

```
q w e r t y u i o p
/ '

 a s d f g h j k l ; '
           ,

  z x c v b n m , . /
           .
```

    这也是当 'keymap=hebrew' 设置时的映射方式。使用 'keymap' 的好处是它在 UTF8 下正常工作，例如，输入的字符是正确的；'hkmap' 则不然。'keymap' 键盘也可以插入 niqud 和 te'amim。要了解具体这些映射是什么，看看 'hebrew' 等键盘映射文件。

反向输入

如果 'revins'（反向输入）选项被打开，插入是反向的。这可以用来输入希伯来语。这时，输入文字时，光标不动，文本向右移动。<BS> 删除光标下的字符，CTRL-W 和 CTRL-U 工作的方向也经过反转。无论 'backspace' 选项如何设置，<BS>、CTRL-W 和 CTRL-U 并不会在插入点开始处和行尾停住。

现在（还）没有反向替换模式。

如果设置了 "showmode" 选项，反向插入模式激活时，状态行会显示 "-- REVERSE INSERT --"。

如果打开了 'allowrevins' 选项，CTRL-_ 也可以进入反向插入模式，而且还有一些附带的功能：首先，键盘映射根据窗口的朝向自动切换 - 如果在从左到右的窗口，'revins' 被用来输入希伯来文，所以键盘改变为希伯来方式（设置 'hkmap'）；如果在从右到左的窗口，'revins' 被用来输入英语，所以键盘切换成英语方式（关闭 'hkmap'）。其次，通过 CTRL-_ 来退出 'revins' 时的光标移动到输入文字的末尾（如果可能的话）。

在从右到左的窗口粘贴

如果用鼠标剪切文本，然后在从右到左的窗口中粘贴，文本会被反向，这时因为从剪切缓冲区里得到的字符是从左到右的，而文件中的插入操作是从右到左的。要避免这个问题，在粘贴前切换 'revins' 选项（通过键入 CTRL-? 或者 CTRL-_ ）。

希伯来字母和 'isprint' 变量

有时希伯来字母代码在 'isprint' 变量定义的非可显示字符的范围内。例如，在 Linux 控制台上，希伯来字体的编码从 128 开始，而缺省的 'isprint' 变量是 @,161-255。结果就是所有的希伯来字母都被显示成 ~x。要解决这个问题，执行

```
set isprint=@,128-255
```

vim:tw=78:ts=8:noet:ft=help:norl:

russian.txt For Vim version 8.2. 最近更新：2006年7月

VIM 参考手册 by Vassily Ragosin
译者：Willis
http://vimcdoc.sf.net

Vim 中的俄罗斯语的本地化和支持 russian Russian

1. 简介 russian-intro
2. 俄语键盘映射 russian-keymap
3. 本地化 russian-l18n
4. 已知漏洞 russian-issues

---

1. 简介 russian-intro

Vim 的俄罗斯语支持已经相当完善。你可以像其他语言一样输入和查看俄语文字，无须特别改变设置。

---

2. 俄语键盘映射 russian-keymap

要切换语言，你可以使用系统自带的键盘切换工具。你也可以使用 Vim 发布中的某一个俄语的键盘映射。例如，

```
:set keymap=russian-jcukenwin
```

如果是后者，即使你没有系统俄语键盘或者一个独立的系统范围内的键盘设置，你还是可以在语言间切换，参看 'keymap'。你也可以专门映射一个键来进行键盘切换。如果你选择这么做，参看 :map 。

为了你的方便起见，为了避免直接键盘切换，当你需要键入普通模式的命令时，你可以使用 'langmap' 选项：

```
:set langmap=Ф И С В У А П Р Ш О Л Д Ь Т Щ З Й К Ы Е Г М Ц Ч Н Я;ABCDEFGHIJKLMNOPQRSTUVWXYZ,
ф и с в у а п р ш о л д ь т щ з й к ы е г м ц ч н я;abcdefghijklmnopqrstuvwxyz
```

这是 utf-8 编码的，如果你的 'encoding' 不是，你无法正常阅读。你也必须在一行内输入整个命令，上面的文字换了行，但只是为了方便阅读起见。

---

3. 本地化 russian-l18n

如果你希望把消息，帮助文件，菜单，和其他项目翻译成俄语，你需要安装 RuVim 语言包。多种代码页的该软件可以从这里得到：

```
http://www.sourceforge.net/projects/ruvim/
```

请你确定你的 Vim 至少是 6.2.506 或以上的版本，并且如果要自动安装，请使用 ruvim 0.5 或以上的版本。Vim 必须在编译时加入 +gettext 特性，才能使用户界面的项目被翻译。

下载了 RuVim 项目的软件包以后，把它解压到 $VIMRUNTIME 目录。我们建议使用 UTF-8 包。

要看到俄语的文档，请将 'helplang' 选项设为 "ru"。

---

4. 已知漏洞 russian-issues

-- 如果你在 Win32 控制台使用俄语的消息翻译，那你也许会看 "vim --help"、"vim --version" 命令和 Win32 控制窗口的标题使用了错误的代码页。这是由 GNU gettext 库的一个问题造成的，将来的 gettext 的发布也许能解决这个问题。

---

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 By Chi-Deok Hwang and Sung-Hyun Nam
译者: Willis
http://vimcdoc.sf.net

hangul

对没有 XIM (X 输入法) 的用户而言，Vim 曾经有内建的 hangul，韩国语的一种书写方式的支持。因为工作得不好且没人维护，在 Vim 8.1.2327 中删除此支持。

如果你仍要此 hangul 输入方法，可以回到 Vim 8.1.2326 或更早版本。如果你认为此代码仍然有用且想维护它，不妨提供一个补丁把它加回来。不过，最好让它能支持 UTF-8 编码。

vim:tw=78:ts=8:noet:ft=help:norl:

gui.txt　　For Vim version 8.2.　最近更新：2019年12月

VIM 参考手册　by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

Vim 的图形用户界面　　gui　GUI

1. 启动 GUI　　gui-start
2. 滚动条　　gui-scrollbars
3. 鼠标控制　　gui-mouse
4. 使用 GUI 进行选择　　gui-selections
5. 菜单　　menus
6. 字体　　gui-font
7. 附加　　gui-extras
8. 外壳命令　　gui-shell

其它的 GUI 文档：
gui_x11.txt　关于 X11 GUI 的特定内容。
gui_w32.txt　关于 Win32 GUI 的特定内容。

---

1. 启动 GUI　　gui-start　E229　E233

首先，需要确定你确实有包含 GUI 代码的 Vim 版本。你可以用 ":version" 命令检查这一点。它应该输出 "with xxx GUI"，而 "xxx" 是 X11-Motif、X11-Athena、Photon、GTK、GTK2、GTK3 等，或者 "MS-Windows 32 bit GUI version"。

如何启动 GUI 决定于你的系统。多数情况下，你可以这样启动 Vim 的 GUI 版本：

```
gvim [options] [files...]
```

GUI 和非 GUI 版本在 Vim 的 X11 版本上都可以运行。见 gui-x11-start 。

gui-init　gvimrc　.gvimrc　_gvimrc　$MYGVIMRC
gvimrc 文件是 GUI 专用的启动命令应该放置的地方。它总在 vimrc 文件之后执行。如果你有这样的文件，$MYGVIMRC 环境变量保存它的名字。

当 GUI 开始时，开展以下初始化操作（按照给出的顺序）：
- 'term' 选项设为 "builtin_gui"，复位终端选项到其 GUI 的缺省值 terminal-options 。
- 如果存在系统菜单文件，执行之。该文件名通常是 "$VIMRUNTIME/menu.vim"。你需要用 ":version" 检查这一点。另见 $VIMRUNTIME 。要跳过菜单载入，在 'guioptions' 里加入 'M'。　buffers-menu　no_buffers_menu
  系统菜单文件包含 "Buffers" 菜单。如果你不需要，在你的 .vimrc（不是 .gvimrc！）里，设置 "no_buffers_menu" 变量：
  ```
  :let no_buffers_menu = 1
  ```
  注意：切换语法高亮也会载入菜单文件，因此屏蔽 Buffers 菜单必须在 ":syntax on" 之前。
  路径名被截短到 35 个字符。你可以截短到不同的长度，例如 50：
  ```
  :let bmenu_max_pathlen = 50
  ```
- 如果启动 Vim 时 使用 "-U {gvimrc}" 命令行选项，初始化过程会读入 {gvimrc} 文

件。以下的步骤被跳过。如果 {gvimrc} 为 "NONE"，初始化过程不读入任何文件。
- 对 Unix 和 MS-Windows 而言，如果系统 gvimrc 存在，执行之。该文件的名字通常是 "$VIM/gvimrc"。你可以用 ":version" 检查这一点。另见 $VIM 。
- 尝试以下步骤，只使用第一个可用的：
  - 如果 GVIMINIT 环境变量存在而且非空，把它作为 Ex 命令执行之。
  - 如果用户 gvimrc 文件存在，执行之。该文件的名字通常是 "$HOME/.gvimrc"。用 ":version" 检查之。
  - 对 Win32 而言，需要时 Vim 会设置 $HOME，见 $HOME-windows 。
  - 如果 "_gvimrc" 文件没找到，尝试 ".gvimrc"。反之亦然。

  第一个找到的文件名保存在 $MYGVIMRC 里，除非后者已经被设置。
- 如果置位 'exrc' 选项（缺省 并非 如此），执行文件 ./.gvimrc。如果该文件不属于你，有一些安全的限制。如果 ".gvimrc" 找不到，尝试 "_gvimrc"。Macintosh 和 DOS/Win32 环境下先尝试 "_gvimrc"。

注意：如果 Vim 启动时使用 "-u NONE" 或 "-u DEFAULTS" 而且没有给出 "-U" 参数，或者使用 "-U NONE"，则除了第一步以外，都不执行。

这些步骤都在普通 Vim 初始化之 后 执行，比如读取 .vimrc 文件。见 initialization 。
GUI 窗口只在全部的初始化执行完毕后才会打开。如果你希望在打开 GUI 窗口后执行某些命令，使用 GUIEnter 自动事件。例如：

```
:autocmd GUIEnter * winpos 100 50
```

你可以使用 gvimrc 文件设置你自己定制的菜单（见 :menu ），并初始化其它你希望和终端版本不同的设置。

建议你把自己的 GUI 初始化部分放在：

Unix	$HOME/.gvimrc 或 $HOME/.vim/gvimrc
Win32	$HOME/_gvimrc、$HOME/vimfiles/gvimrc 或 $VIM/_gvimrc
Amiga	s:.gvimrc、home:.gvimrc、home:vimfiles:gvimrc 或 $VIM/.gvimrc

按上述的顺序依次查找个人化的初始文件，只读入第一个找到的文件。

若干选项只有在 GUI 版本的 Vim 里才有意义。它们是 'guicursor'、'guifont'、'guipty' 和 'guioptions'。和其它选项一样，他们的文档在 options.txt 。

如果使用 Motif 或者 Athena 版本的 GUI（不适用于 GTK+ 或 Win32 版本），可以使用一些 X 资源。见 gui-resources 。

另外一个在不同场合下设置颜色的方法是使用高亮组。"Normal" 组用来设置背景和前景色。示例（看上去很悦目）：

```
:highlight Normal guibg=grey90
```

"guibg" 和 "guifg" 设置覆盖正常的背景和前景设置。其它 Normal 高亮组的设置都没有用到。使用 "guifont" 选项设置字体。

要设置光标在不同模式下的颜色，也请查看 'guicursor' 选项。

Vim 试图在启动时使窗口在屏幕上的大小合适，以免部分内容不可见。在 X 窗口系统下，这需要一些猜测。使用 'guiheadroom' 选项，你可以改变窗口标题和任务栏的高度。

:winp :winpos E188

:winp[os]
显示 GUI vim 窗口以像素计的当前左上角位置。并非在所有系统上都可以。
另见 getwinpos() 、 getwinposx() 和 getwinposy() 。

:winp[os] {X} {Y} E466
把 GUI vim 窗口放在给定 {X} 和 {Y} 坐标。坐标应指定以像素计的窗口左上角的位置。并非在所有系统上都可以。
可以在（新版的）xterm 里使用 xterm-color 。
如果 GUI 窗口还没有打开，坐标值被记住，直到窗口打开为止。该位置会被调整，使得窗口在屏幕上大小合适（如果可能）。

:win :winsize E465

:win[size] {width} {height}
设置窗口高度为 {width} 宽 {height} 高，以字符计。
已废弃，应使用 ":set lines=11 columns=22"。
如果你得到比期待中少的行数，检查 'guiheadroom' 选项。

如果你运行 X 窗口系统，你可以这样得到关于 Vim 运行所在的窗口的信息：

```
:!xwininfo -id $WINDOWID
:!xprop -id $WINDOWID
:execute '!xwininfo -id ' . v:windowid
:execute '!xprop -id ' . v:windowid
```

gui-IME iBus

在 X 里依赖于 XIM 框架输入国际字符的输入法，尤其是 iBus，已知会给 gvim 带来一些不良效果。包括不能输入空格，输入字符间有长时间的延迟，或者输入的字符不能为应用所识别等。

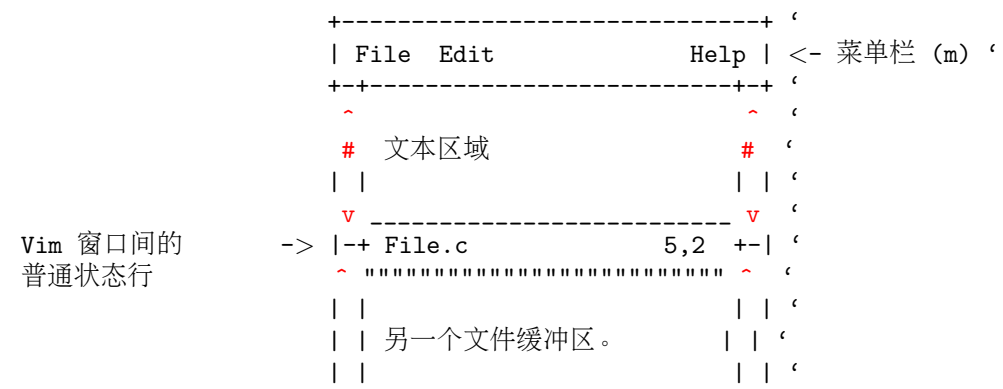
一个可行的解决方法，虽然不知什么原因，是启动时用 -f 参数，不让 gvim 进入后台执行。

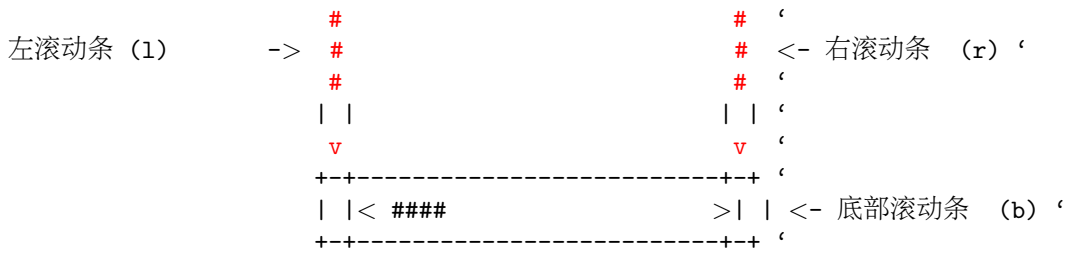
---

2. 滚动条 gui-scrollbars

系统有若干垂直滚动条和一个水平滚动条。你可以用 'guioptions' 选项设置会出现哪一个。

界面看起来是这样（使用 ":set guioptions=mlrb" 的情形下）：





任何滚动条或者菜单都可以关闭，只要在 'guioptions' 字符串里没有相应的字母就行了。底部滚动条只有在置位 'nowrap' 时才有用。

垂 直 滚 动 条 gui-vert-scroll

每个 Vim 窗口都有紧挨着的滚动条，可以上下滚动来实现缓冲区内文本的位置移动。滚动条拇指 (thumb) 指示缓冲区可见的部分。如果滚动条被一直往下拖动，最后文件的末行会出现在窗口的顶部。

如果窗口被缩小到零高度 (因为另一个窗口的增大)，其滚动条消失。在窗口恢复时才重新出现。

如果窗口被垂直分割，该窗口会在成为当前窗口时得到滚动条。想象当前窗口的正中间有一条贯穿窗口的垂直线。如果两边都有滚动条，而当前窗口的中间线属于左半边，那么右面的滚动条列包含当前窗口右边各窗口的滚动条。另外一侧也是如此。

水 平 滚 动 条 gui-horiz-scroll

水平滚动条 (出现在 Vim GUI 的底部) 可以在 'wrap' 选项关闭时用来水平滚动文本。滚动条拇指的大小设置会使得最长可见行的文本可以最大限度的左右滚动。光标在有必要时才会移动，使得它总保持在一个可视的字符上 (除非置位了 'virtualedit')。

计算最长可见行的长度需要若干计算，而且每次有改变的时候对需要进行。如果这需要太多时间，或者你不希望光标跳到别的行上，在 'guioptions' 里包含 'h' 标志位。这样，滚动限于当前光标所在文本行上。

athena-intellimouse

如果你有 Intellimouse 鼠标并且 X 服务器支持滚轮，那你就可以用滚轮在 gvim 里上下滚动文本。XFree86 4.0 和以后的版本都没有问题，一些老的版本里你需要加上补丁。见 scroll-mouse-wheel 。

对较老的 XFree86 版本你需要给 X 服务器打上补丁。以下页面有关于 Linux 上使用 Intellimouse 的一点信息，还有补丁和 X 服务器可执行文件的链接 (后者也许不一定有你需要的):

http://www.inria.fr/koala/colas/mouse-wheel-scroll/

---

3. 鼠标控制 gui-mouse

鼠标只有在 'mouse' 选项设置合适的标志位时才会工作。如果打开了 GUI 的时候还没有设置 'mouse'，'mouse' 选项会自动设为 "a"，使得它除了在 hit-enter 提示以外的所有的模式下都能使用。如果你不希望这样，修改 'mouse' 选项的一个好地方是在 "gvimrc" 文件里。

其它相关的选项：
'mousefocus'　　窗口焦点随鼠标指针改变　gui-mouse-focus
'mousemodel'　　哪个鼠标按钮执行什么动作
'mousehide'　　输入文本时隐藏鼠标指针
'selectmode'　　开始选择模式还是可视模式

一个设置这些选项的快速方法是使用 ":behave" 命令。

:behave　:be

:be[have] {model}　　设置鼠标和选择的行为。可接受的参数是：
　　mswin　　MS-Windows 行为
　　xterm　　Xterm 行为

":behave" 的使用改变了以下选项：

选项	mswin	xterm
'selectmode'	"mouse,key"	""
'mousemodel'	"popup"	"extend"
'keymodel'	"startsel,stopsel"	""
'selection'	"exclusive"	"inclusive"

在 $VIMRUNTIME 目录下，有一个脚本叫 mswin.vim 。它会定义一些 MS-Windows 的剪切/复制/粘贴的键盘映射，但这是 不 兼容的，因为它使用了 CTRL-V、 CTRL-X 和 CTRL-C 键。不过如果你不在意，使用这个命令：

```
:so $VIMRUNTIME/mswin.vim
```

关于使用鼠标滚轮进行滚动的部分，见 scroll-mouse-wheel 。

3.1 使用鼠标移动光标　gui-mouse-move

在文本缓冲区某处点击鼠标左键，就会移动光标到那里！

在此模式下工作	当 'mouse' 包含...时
普通模式	'n' 或 'a'
可视模式	'v' 或 'a'
插入模式	'i' 或 'a'

选择模式和可视模式一样处理。

你可以和操作符一起使用这个功能，比如 'd' 删除从当前光标位置到你用鼠标指向的位置。即，按 'd'，然后在某处点击鼠标。

gui-mouse-focus

可以置位 'mousefocus' 选项使得键盘焦点随着鼠标指针转移。这意味着，鼠标在哪个窗口，哪个窗口就被激活。警告：使用菜单的时候，这并不完善，因为菜单命令总是针对最顶上的窗口。

如果你在 ':' 行上 (或者 '/' 或 '?')，那么点击鼠标左键或右键只会把光标定位在 ':' 行上 (如果 'mouse' 包含 'c' 或 'a' 的话)。

任何情况下，点击鼠标中键可以用来粘贴当前的选择内容。

3.2 使用鼠标进行选择　gui-mouse-select

鼠标可以用来开始选择。怎样开始决定于 'mousemodel' 选项：

'mousemodel' 是 "extend": 使用鼠标右键
'mousemodel' 是 "popup": 使用鼠标左键，按住 Shift 键。

如果还没有开始选择，那么选择范围从以前的光标位置开始，到鼠标所点击的位置。如果已经有选择，那么离鼠标最近的那端会被扩展。

如果 'selectmode' 包含 "mouse"，那么选择区处于选择模式。这意味着输入正常的文本会替代选择内容。见 Select-mode 。不然，选择区处于可视模式。

双击可以选择单词，三击可以选择行，四击可以选择矩形块。

关于如何使用选择区，见 gui-selections 。

3.3 另一种鼠标的文本选择

gui-mouse-modeless
modeless-selection

以下情形使用另一种选择:
- 在命令行模式下
- 在命令行窗口中，指针在另外一个窗口
- 在 hit-enter 提示时
- 当前模式不在 'mouse' 选项里
- 在 GUI 模式里，按住 CTRL 和 SHIFT 键

因为 Vim 就像没有选择区那样继续，而且没有和选择区相关联的模式，这种选择称为无模式的选择。可以选择任何在 Vim 窗口里的文本。在文本开始的地方鼠标左键，拖到尾部再释放。要扩展选择，在 'mousemodel' 为 "extend" 的时候，使用鼠标右键；在 'mousemodel' 为 "popup" 的时候，使用鼠标左键并按住 Shift 键。

选择区在选择文本滚动或者改变时被撤销。

在命令行上 CTRL-Y 可以用来复制选择区到剪贴板里。在插入模式下，可以使用 CTRL-O : CTRL-Y <CR>。如果 'guioptions' 包含 a 或 A (X11 上的缺省值)，选择被自动复制到 "* 寄存器里。

这时用鼠标中键就可以粘贴文本。非 X11 系统上，可以使用 CTRL-R +。

3.4 在状态行上使用鼠标

gui-mouse-status

在 Vim 窗口下方的状态行上点击鼠标左键或者右键使得该窗口成为当前窗口。实际上，这发生在鼠标释放的时候 (以区别拖动时的点击操作)。

使用鼠标左键，状态行可以上下拖动，从而调整上下窗口的大小。这样并不改变窗口焦点。

这同样适用于垂直分割线：点击把焦点给左侧的窗口，左右拖动使得两侧窗口更宽和更窄。

3.5 各种鼠标点击

gui-mouse-various

<S-LeftMouse> 正向搜索鼠标点击所在的单词。
当 'mousemodel' 为 "popup" 时，该操作开始或者扩展选择区。

<S-RightMouse> 反向搜索鼠标点击所在的单词。

<C-LeftMouse>　　跳转到鼠标点击所在的标签上。
<C-RightMouse>　　跳回上次标签跳转之前的位置（等同于 "CTRL-T"）

3.6 鼠标映射　　gui-mouse-mapping

鼠标事件，包括各种修饰符，可以被映射。例如：

```
:map <S-LeftMouse>     <RightMouse>
:map <S-LeftDrag>      <RightDrag>
:map <S-LeftRelease>   <RightRelease>
:map <2-S-LeftMouse>   <2-RightMouse>
:map <2-S-LeftDrag>    <2-RightDrag>
:map <2-S-LeftRelease> <2-RightRelease>
:map <3-S-LeftMouse>   <3-RightMouse>
:map <3-S-LeftDrag>    <3-RightDrag>
:map <3-S-LeftRelease> <3-RightRelease>
:map <4-S-LeftMouse>   <4-RightMouse>
:map <4-S-LeftDrag>    <4-RightDrag>
:map <4-S-LeftRelease> <4-RightRelease>
```

这些映射使得选择的使用方式符合和 Motif 应用方式，Shift + 鼠标左键允许扩展可视区域，而不是鼠标右键。

带修饰符的鼠标映射不适用于无模式的选择。

3.7 拖放　　drag-n-drop

你可以拖放一个或多个文件到 Vim 窗口里，这样，它们就会被打开，如果执行了 :drop 命令那样。

如果这么做的时候你按住 Shift，Vim 改变到第一个放下的文件所在的目录。如果你按住 Ctrl，Vim 总会为文件分割出一个新的窗口。不然，只有当前缓冲区被改变的时候才会这么做。

你可以在 Vim 里放下目录。这会为该目录启动探索器插件（假设该插件被打开，不然你会得到错误信息）。按住 Shift 这改变到该目录。

如果 Vim 碰巧在编辑命令行，放下文件的名字和目录会插入在光标位置上。这使得你在任何 Ex 命令上都可以使用这些名字。特殊字符（空格、制表、双引号、'|'、非 MS-Windows 系统上的反斜杠）会被转义。

---

4. 使用 GUI 进行选择　　gui-selections

quotestar

你可以用鼠标进行选择（见 gui-mouse-select），或者使用 Vim 的可视模式（见 v）。如果 'guioptions' 里包含 'a'，那么无论什么时候开始选择（可视或选择模式），或者选择内容发生改变的时候，Vim 成为窗口系统的主选择区的拥有者（MS-Windows 上使用 gui-clipboard；X11 上使用 x11-selection - 不管具体哪个更合适，你应该据此理解所谓的主选择区）。

clipboard

有一个特殊的寄存器可以存放选择内容，这就是 "* 寄存器。除非选择文本的相关信息发生改变（比如你在某处按了鼠标左键），或者另一个应用想要粘贴选择文本。这时，文本

被放到 "* 寄存器里。例如，剪切一行并使之当前的选择区并放到剪贴板上：

```
"*dd
```

类似的，你想粘贴别的应用的选择内容，比如通过点击鼠标中键，的时候，该选择内容被先放到 "* 寄存器上，然后再像其它寄存器那样被 'put'（放置）。例如，放置选择内容（剪贴板的内容）可以这样：

```
"*p
```

如果在 X11 中使用该寄存器，另见 x11-selection 。那里也解释了相关的 "+ 寄存器。

注意 在从一个 Vim 粘贴文本到另一个单独的 Vim 的时候，选择类型（字符、行、或块）也会被复制。对于其它应用，类型总是面向字符的。不过，如果文本通过 x11-cut-buffer 传输，选择类型 总 会丢失。

当 'clipboard' 选项包含了 "unnamed" 字符串的时候，无名寄存器和 "* 相同。这样，你可以抽出和粘贴选择内容，而无须在这些命令前面附加 "*。

---

5. 菜单 menus

用户手册 usr_42.txt 有本节内容的简单介绍。

5.1 使用菜单 using-menus

基本上，菜单和映射类似。你可以定义自己的菜单，多少都可以。
Vim 的长期用户不太用菜单。但是它的优势在你可以增加自己的菜单和菜单项。它们在你记不住键序列的功能的时候很有用。

关于如何创建不同语言的菜单，见 :menutrans 。
如果完全不想使用菜单，见 'go-M' 。

menu.vim

缺省菜单从 "$VIMRUNTIME/menu.vim" 文件里读取。关于该路径从何而来，见 $VIMRUNTIME 。 你可以设置你自己的菜单。从缺省菜单开始是一个好主意。你可以增加更多项目，或者如果完全不喜欢缺省菜单，从删除所有菜单开始 :unmenu-all 。你可以把下行加到 .vimrc（不是 .gvimrc）文件以禁止缺省的菜单的调入：

```
:let did_install_default_menus = 1
```

如果你也不想载入 Syntax 菜单：

```
:let did_install_syntax_menu = 1
```

Syntax 菜单第一项可以用来显示包含所有可用的文件类型的菜单（载入需要不少时间）。如果你确实想要启动时就同时得到所有文件类型，加入：

```
:let do_syntax_sel_menu = 1
```

以下菜单项显示所有的可用配色方案，键盘映射和编译器设定：

```
Edit > Color Scheme
Edit > Keymap
Tools > Set Compiler
```

不过，载入这些项目很花时间，因为它们要从 'runtimepath' 的众目录中搜索所有的相关文件。因此，它们被延后载入（通过 CursorHold 事件），你也可以手动载入之。
如果你希望系统启动时载入所有这些项目，加入：

```
:let do_no_lazyload_menus = 1
```

注意 ‘:syntax on‘ 或 ‘:filetype on‘ 执行时或 .vimrc 文件载入时，会载入 menu.vim。这意味着 ’encoding’ 选项和消息语言 (‘:language messages‘) 须在那之前设置好 (如果你想有所改变的话)。

console-menus

尽管本文档讨论 GUI，你其实也可以在控制台模式使用菜单。你需要自行载入 menu.vim ，因为在那里，这一步不是自动进行的。你可以使用 :emenu 命令和带 ’wildmenu’ 的命令行补全，从而能像真实的菜单系统那样对菜单项进行访问。要做到这一点，在你的 .vimrc 文件里放入如下命令:

```
:source $VIMRUNTIME/menu.vim
:set wildmenu
:set cpo-=<
:set wcm=<C-Z>
:map <F4> :emenu <C-Z>
```

按 <F4> 会启动菜单。你可以用光标移动键来选择菜单项。按 <Enter> 执行。如果你想取消，按 <Esc>。
本功能需要在编译时加入 +menu 特性。

tear-off-menus

GTK+ 2 和 Motif 支持可撕下的菜单。它们是一种有粘性的菜单，亦即一直在屏幕上存在的弹出式菜单。如果它们不能调整大小，可能你在 defaults 文件里使用了类似于 "Vim*geometry" 的设置。你应该使用 "Vim.geometry"。

至于 GTK+ 3，从 GTK+ 3.4 开始，可撕下的菜单被废弃了。因而如果 gvim 链接了 GTK+ 3.4 或更新的版本，关闭其支持。

Win32 GUI 版本模拟 Motif 的可撕下菜单。实际上，Motif 用户很容易说出两者的区别，但我们只希望能一样有用就够了。你也可以同时使用 :tearoff 命令和 hidden-menus ，来创建不在主菜单栏出现的浮动菜单。

5.2 创建新的菜单

creating-menus

```
:me    :menu     :noreme    :noremenu
:am    :amenu    :an        :anoremenu
:nme   :nmenu    :nnoreme   :nnoremenu
:ome   :omenu    :onoreme   :onoremenu
:vme   :vmenu    :vnoreme   :vnoremenu
:xme   :xmenu    :xnoreme   :xnoremenu
:sme   :smenu    :snoreme   :snoremenu
:ime   :imenu    :inoreme   :inoremenu
:cme   :cmenu    :cnoreme   :cnoremenu
:tlm   :tlmenu   :tln       :tlnoremenu
E330   E327   E331   E336   E333
E328   E329   E337   E792
```

要创建新的菜单项，使用 ":menu" 命令群。它们和 ":map" 命令群非常类似 (见 map-modes)，但是第一个参数是菜单项名。形式为 ’.’ 分隔的菜单或子菜单的路径，例如:

```
:menu File.Save  :w<CR>
:inoremenu File.Save  <C-O>:w<CR>
:menu Edit.Big\ Changes.Delete\ All\ Spaces  :%s/[ ^I]//g<CR>
```

最后一项会在菜单栏里创建一个新的项目 "Edit"，在它上面按下鼠标键，会弹出菜单，里面包含 "Big Changes" 一项，这又是包含 "Delete All Spaces" 项的一个子菜单。如果选择了该项，执行操作。

要创建终端模式的菜单，用 :tlmenu 而不是 :tmenu ，这和键盘映射不一致 (:tmap)。这是因为 :tmenu 已经用于定义菜单的工具提示了。见 terminal-typing 。

菜单名里可用的特殊字符：

&	下一个字符是快捷键。确信该快捷键在（子）菜单里只使用一次。如果你想插入按本义出现的 "&"，使用 "&&"。
<Tab>	分隔菜单名和右对齐的文本。这被用来显示等价的命令输入。为了方便起见，这里可以使用文本 "<Tab>"。如果你使用真正的制表键，别忘了在前面加上反斜杠！

示例：

```
:amenu &File.&Open<Tab>:e  :browse e<CR>
```

[按文本直接输入]
通过快捷键 "F"（按着 <Alt> 键），然后 "O"，就可以使用这个菜单。第二部分显示为 "Open :e"。这里 ":e" 右对齐，而 "O" 有下划线，指示这是那个快捷键。

":amenu" 命令可以用来一次定义除终端模式以外所有模式下的菜单项。要使得该命令正确工作，在有些模式下需要自动插入一个字符：

模式	前面插入	后面附加
普通	不需要	不需要
可视	<C-C>	<C-\><C-G>
插入	<C-\><C-O>	
命令行	<C-C>	<C-\><C-G>
等待操作符	<C-C>	<C-\><C-G>

附加 CTRL-\ CTRL-G 是为了能在置位 'insertmode' 的时候回到插入模式。 CTRL-_CTRL-G

示例：

```
:amenu File.Next     :next^M
```

等价于：

```
:nmenu File.Next     :next^M
:vmenu File.Next     ^C:next^M^\^G
:imenu File.Next     ^\^O:next^M
:cmenu File.Next     ^C:next^M^\^G
:omenu File.Next     ^C:next^M^\^G
```

小心：插入模式下，因为使用 CTRL-O，只对单个普通模式的命令有效。如果你有两个或多个命令，必须使用 ":imenu" 命令。要在任何模式下插入文本，可以使用表达式寄存器 :

```
:amenu Insert.foobar   "='foobar'<CR>P
```

注意 这里使用 'cpoptions' 里的 '<' 和 'k' 标志位（如果包含了它们，<> 形式和原始的键码都不会被识别）。

注意 命令行模式下的 <Esc> 执行该命令，和映射的情况相同。这是 Vi 兼容的行为。使用 CTRL-C 退出命令行模式。

:menu-<silent> :menu-silent

要定义在命令行上不会回显的菜单，增加 "<silent>" 作为第一个参数。例如：

```
:menu <silent> Settings.Ignore\ case :set ic<CR>
```

":set ic" 在使用该菜单时不会回显。但是执行命令时给出的消息仍然会。要把那些也去掉，在执行命令时增加 ":silent"：

```
:menu <silent> Search.Header :exe ":silent normal /Header\r"<CR>
```

"<silent>" 也可以出现（但只能）在 "<special>" 或 "<script>" 之后。

:menu-<special> :menu-special

定义菜单时，特殊键可用 <> 记法，即使 'cpoptions' 包含了 "<" 标志位也没问题。可用于不希望看到设置 'cpoptions' 时出现副作用的场合。例如：

```
:menu <special> Search.Header /Header<CR>
```

"<special>" 必须是第一个参数或出现（但只能）在 "<silent>" 或 "<script>" 之后。

:menu-<script> :menu-script

菜单的 "to" 部分会检查映射。如果你不想这样。使用 ":noremenu" 命令（或者特定模式下的类似命令）。
如果你需要局部于脚本的映射，增加 "<script>" 作为 ":menu" 的第一个或紧跟在 "<silent>" 或 "<special>" 之后的参数。

menu-priority

你可以给菜单指定优先级。高优先级的菜单出现在右边。优先级在 ":menu" 命令之前以数字形式给出。例如：

```
:80menu Buffer.next :bn<CR>
```

缺省菜单有如下的优先级：

```
File            10
Edit            20
Tools           40
Syntax          50
Buffers         60
Window          70
Help            9999
```

如果没有或者给出优先级为零，则使用 500。
弹出菜单不使用优先级。

如果系统支持（Motif 和 GTK+），Help 菜单会出现在菜单栏的最右侧。GTK+ 2 和 3 版本已经不再继续这么做，因为该环境现在不再鼓励右对齐帮助菜单的界面设计风格。

你可以使用超过 9999 的优先级，使得菜单能出现在 Help 菜单之后。但这是不合标准的做法，并不鼓励。最高的可能优先级大约是 32000。最低为 1。

sub-menu-priority

相同的机制可以用来定位子菜单。这里，优先级用出现在菜单名前且用句号分隔的优先级列表给出：

```
:menu 80.500 Buffer.next :bn<CR>
```

子菜单优先级只有在不想该项目在正常的位置出现的时候才需要。例如，把某个子菜单放在其它项目之前：

```
:menu 80.100 Buffer.first :brew<CR>
```

也可以把子菜单放在其它项目之后，并期望使用缺省优先级的未来项目会加在它之前：

```
:menu 80.900 Buffer.last :blast<CR>
```

如果不给出某个数字，使用缺省值 500：

```
:menu .900 myMenu.test :echo "text"<CR>
```

菜单优先级只有在建立新菜单时才会用到。一旦已经存在（比如，在别的模式下），优先级不会改变。因此，优先级只有在菜单第一次用到的时候才需要给出。弹出菜单是一个例外。每个模式都有单独的菜单（普通、操作符等待、可视、插入、命令行）。每个菜单里的顺序可以不同。这和菜单栏的菜单不同，那里所有模式下都使用相同的顺序。

注意：现在，子菜单优先级不是在所有版本的 GUI 上都能工作。

menu-separator E332

菜单项可以用一个特殊项目分隔，它在项目间插入一些空白。根据不同系统情况，有的显示为一条线，有的显示为虚线。这些项目必须以 '-' 开始和结尾。两者之间的部分用来给出一个独一无二的名字。和正常的项目一样，也可以使用优先级。例如：

```
:menu Example.item1     :做些事
:menu Example.-Sep-     :
:menu Example.item2     :做些别的事
```

注意 分隔符也需要一个右手边的表达式。是什么没有关系，因为该项目永远不会选到。简单点，用单个冒号就可以了。

gui-toolbar

目前，只有 Win32、Athena、Motif、GTK+ (X11) 和 Photon GUI 才有工具栏。适当的时候，在其它 GUI 上也会开放这一功能。缺省的工具栏在 menu.vim 里定义。
工具栏显示与否有 'guioptions' 里的 'T' 字母控制。你可以同时有菜单和工具栏，或者只用其中一项，甚至全不要。其外观通过 'toolbar' 选项控制。你可以在图像、文本或者两者兼有之间选择。

toolbar-icon

工具栏定义为一个特殊的菜单，名为 ToolBar。它只有一层。Vim 如此解释该菜单里面的项目：

1) 如果给出 "icon=" 参数，则使用该名字指定的文件。这个文件可以给出完整路径，或者只是基础名。如果是后者，在 'runtimepath' 的 "bitmaps" 目录里搜索该文件，就像第三点那样。例如：

```
:amenu icon=/usr/local/pixmaps/foo_icon.xpm ToolBar.Foo :echo "Foo"<CR>
:amenu icon=FooIcon ToolBar.Foo :echo "Foo"<CR>
```

注意 在第一个例子里，需要包含扩展名，而第二个例子就不需要。
如果该文件不能打开，则尝试下面的步骤。
文件名里的空格必须用反斜杠转义。
菜单优先级必须在 icon 参数 之后 给出：

```
:amenu icon=foo 1.42 ToolBar.Foo :echo "42!"<CR>
```

2) 名为 'BuiltIn##' 的项目，其中 ## 是一个数字，用来指定 Vim 第 ## 个内建的位图。目前，有 31 个编号的位图，从 0 到 30。它们包含了多数常见的编辑操作 builtin-tools 。

```
:amenu ToolBar.BuiltIn22 :call SearchNext("back")<CR>
```

3) 首先在 'runtimpath' 里的 "bitmaps" 里搜索某个其它名字的项目。如果找到，该位图文件被用作工具按钮的图像。注意 准确的文件名和具体的操作系统有关：例如，Win32 上命令

```
:amenu ToolBar.Hello :echo "hello"<CR>
```

会找到文件 'hello.bmp'。在 GTK+/X11 上则是 'Hello.xpm'。在 GTK+ 2 上则按序查找文件 'Hello.png'、'Hello.xpm' 和 'Hello.bmp'，最先找到的被使用。

MS-Windows 和 GTK+ 2 上位图被缩放到合适的按钮大小。在 MS-Windows 上，18 乘 18 像素最好。
在 MS-Windows 上，位图使用标准的调色板，使用 16 色。浅灰色的像素会被替换成窗口框架的颜色，而深灰色像素被换成窗口阴影的颜色。根据你的系统，也许可以使用比这个更多的颜色。

4) 如果该位图找不到，Vim 在内建名字的列表里查找该名字。因为每个内建按钮图像都有名字。
因此，如果没有 open.bmp 的话，命令

```
:amenu ToolBar.Open :e
```

会显示内建的 "open a file" 按钮图标。
在 menu.vim 里可以看到所有内建的名字。

5) 如果以上都不行，就显示一个可用的空白按钮。

builtin-tools

号	名字	正常定义的操作
00	New	打开新窗口
01	Open	浏览要在当前窗口打开的文件
02	Save	把缓冲区写回文件
03	Undo	撤销上次改变
04	Redo	重做上次撤销的改变
05	Cut	删除选择的文本到剪贴板
06	Copy	复制选择的文本到剪贴板
07	Paste	从剪贴板粘贴文本
08	Print	打印当前缓冲区
09	Help	打开缓冲区，显示 Vim 的内建帮助
10	Find	开始搜索命令
11	SaveAll	把所有修改过的缓冲区写回文件
12	SaveSesn	把当前的情况写到会话文件
13	NewSesn	创建一个新的会话文件
14	LoadSesn	载入会话文件
15	RunScript	浏览文件，并作为 Vim 脚本执行
16	Replace	执行 substitute 命令，给出提示
17	WinClose	关闭当前窗口
18	WinMax	使得当前窗口使用尽量多行
19	WinMin	使得当前窗口使用尽量少行
20	WinSplit	分割当前窗口
21	Shell	启动外壳
22	FindPrev	反向再次搜索
23	FindNext	正向再次搜索
24	FindHelp	提示关键字，搜索相关帮助
25	Make	运行 make，跳到第一个错误
26	TagJump	跳转至光标所在的标签
27	RunCtags	运行 Ctags，为当前目录的文件构造标签
28	WinVSplit	垂直分割当前窗口
29	WinMaxWidth	使得当前窗口使用尽量多列
30	WinMinWidth	使得当前窗口使用尽量少列

hidden-menus win32-hidden-menus

在 Win32 和 GTK+ GUI 上，以 ’]’ 开始的菜单名使得该菜单不会在主菜单栏里出现。你可以用 :popup 或者 :tearoff 命令来显示之。

window-toolbar WinBar

每个窗口都可有本地工具条。使用窗口的首行，这会减少一行文本可用的空间。工具条项目须以 "WinBar" 开始。

只能用文本。使用 Unicode 可以用特殊字符来使项目看上去像个图标。

如果不能将全部项目安放于一行，则后面几个不能使用。工具条不会回绕。

备注 Vim 执行这些命令时可能处于任何模式。菜单应为普通模式定义，执行时也不应改变当前模式。这样，如果当然窗口处于可视模式而菜单命令没有主动改变模式的话，Vim 会继续留在可视模式。最好用 :nnoremenu 来避免副作用。

调试工具示例：

```
nnoremenu 1.10 WinBar.Step :Step<CR>
nnoremenu 1.20 WinBar.Next :Next<CR>
nnoremenu 1.30 WinBar.Finish :Finish<CR>
nnoremenu 1.40 WinBar.Cont :Continue<CR>
```

窗口工具条使用 ToolbarLine 和 ToolbarButton 高亮组。

分割窗口时，并不在新窗口中复制窗口工具条。

popup-menu

在 Win32、GTK+、Motif、Athena 和 Photon GUI 上，你可以定义特殊的 "PopUp" 菜单。如果 'mousemodel' 设为 popup 或者 popup_setpos，这将是按下鼠标右键时显示的菜单。
示例：

```
nnoremenu 1.40 PopUp.&Paste "+gP
menu PopUp
```

5.3 显示菜单如何被映射 showing-menus

要看看现有的菜单是如何被映射的，在 menu 命令之后只使用一个参数（就像你使用 ":map" 命令那样）。如果指定的菜单是一个子菜单，那么所有该子树下面的层次结构都会显示。如果 :menu 后面不给出任何参数，显示所在模式下的 所有 菜单项（例如，命令行模式下显示 :cmenu）。

该列表里，在右手边表达式之前显示的特殊字符的含义是：

*	菜单是通过 "nore" 形式定义的，以禁止重映射。
&	菜单定义时包含了 "<script>"，使得重映射只限于局部于脚本的映射。
-	菜单被屏蔽。

注意 在 menu 命令里输入菜单名时，按 <Tab> 可以用来补全菜单项目名。

5.4 执行菜单 execute-menus

:em :emenu E334 E335

:[range]em[enu] {menu} 从命令行上执行 {menu} 菜单。缺省执行普通模式的命令。如果给出行范围 range，执行可视模式的菜单。
如果从 <c-o> 那里调用，执行插入模式命令。例如：

```
:emenu File.Exit
```

:[range]em[enu] {mode} {menu} 同上，但执行 {mode} 下的菜单：

'n':	:nmenu	普通模式
'v':	:vmenu	可视模式
's':	:smenu	选择模式
'o':	:omenu	操作符等待模式
't':	:tlmenu	终端模式
'i':	:imenu	插入模式
'c':	:cmenu	命令行模式

如果控制台模式的 vim 编译时加入 WANT_MENU，你可以用 :emenu 来访问你已经习惯的 GUI 模式下有用的菜单项目。关于一个能用于此目的的选项，见 'wildmenu'。例子又可以见 console-menus 。

如果使用行范围 range，并且选择的行和 '<、'> 指定的范围相同，则使用最近一次可视选择的范围执行菜单命令。

5.5 删除菜单 delete-menus

:unme :unmenu
:aun :aunmenu
:nunme :nunmenu
:ounme :ounmenu
:vunme :vunmenu
:xunme :xunmenu
:sunme :sunmenu
:iunme :iunmenu
:cunme :cunmenu
:tlu :tlunmenu

要删除一个菜单项或者整个子菜单，使用 umenu 命令，类似于 unmap 命令。例如：

```
:unmenu! Edit.Paste
```

会删除插入和命令行模式下的 Edit 菜单的 Paste 项。

注意 在 umenu 命令里输入菜单名时，按 <Tab> 可以用来补全菜单项目名。

要删除所有的菜单，可用： :unmenu-all

```
:unmenu *       " 删除普通和可视模式下的所有菜单
:unmenu! *      " 删除插入和命令行模式下的所有菜单
:aunmenu *      " 删除终端模式除外的所有模式下的所有菜单
:tlunmenu *     " 删除终端模式下的所有菜单
```

如果你想去掉菜单栏：

```
:set guioptions-=m
```

5.6 屏蔽菜单 disable-menus

:menu-disable :menu-enable

如果你不想删除某个菜单，但想暂时屏蔽之，可以在 ":menu" 命令里使用 "enable" 或 "disable" 关键字。例如：

```
:menu disable &File.&Open\.\.\.
:amenu enable *
:amenu disable &Tools.*
```

和其它 menu 命令一样，该命令作用于指定的模式上。注意 需要包含进来 "&" 这样的字符，以便找到经过翻译的名字。如果参数是 "*"，所有的菜单都受到影响。否则，给定的菜单名和下面所有的子菜单（如果存在的话）都会受影响。

5.7 菜单示例 menu-examples

这里是一个如何用 menu 增加菜单项的例子。你可以为光标下的关键字增加菜单项。使用寄存器 "z"。

```
:nmenu Words.Add\ Var          wb"zye:menu! Words.<C-R>z <C-R>z<CR>
:nmenu Words.Remove\ Var       wb"zye:unmenu! Words.<C-R>z<CR>
:vmenu Words.Add\ Var          "zy:menu! Words.<C-R>z <C-R>z <CR>
:vmenu Words.Remove\ Var       "zy:unmenu! Words.<C-R>z<CR>
:imenu Words.Add\ Var          <Esc>wb"zye:menu! Words.<C-R>z <C-R>z<CR>a
:imenu Words.Remove\ Var       <Esc>wb"zye:unmenu! Words.<C-R>z<CR>a
```

(这里的右手边表达式使用 <> 记法，你可以复制/粘贴这里的文本来测试这些映射，或者把这些行加到你的 gvimrc 文件里；"<C-R>" 是 CTRL-R，"<CR>" 是 <CR> 键。 <>)

5.8 工具提示 (Tooltips) 和菜单提示

见用户手册的 42.4 一节。

:tmenu :tm

:tm[enu] {menupath} {rhs} 为菜单或者工具定义提示。{仅存在于 X11 和 Win32 GUI}

:tm[enu] [menupath] 列出菜单提示。{仅存在于 X11 和 Win32 GUI}

:tunmenu :tu

:tu[nmenu] {menupath} 删除菜单或者工具的提示。{仅存在于 X11 和 Win32 GUI}

备注：要创建终端模式的菜单，使用 :tlmenu 代替。

如果为某菜单项定义了提示，当鼠标移过该项目时，命令行区域会出现该提示，就像标准 Windows 菜单提示出现在状态栏那样。(除非 Vim 在命令行模式的时候，这时当然什么都不会显示。)
如果为某工具栏项目定义了提示，当鼠标移过该按钮时，提示以通常的方式出现。要改变它们的颜色，使用 hl-Tooltip 高亮组。

"提示" 可以为每个菜单项目定义。例如，在这样定义菜单项时：

```
:amenu MyMenu.Hello :echo "Hello"<CR>
```

提示可以这样定义：

```
:tmenu MyMenu.Hello Displays a greeting.
```

然后这样删除：

```
:tunmenu MyMenu.Hello
```

现在，工具提示只有在 X11 和 Win32 GUI 上才支持。不过，在不远的将来，应该在其它 gui 平台上也会出现。

":tmenu" 菜单和其它菜单命令使用相同的参数。":tunmenu" 删除已存在的菜单提示，使用方式和其它的 umenu 命令一样。

如果菜单项目不再合法，(即，在所有模式下，它的动作都被删除) Vim 为你删除菜单的提示 (和菜单项)。这意味着 :aunmenu 彻底删除了菜单项目 - 你不需要再运行 :tunmenu 了。

5.9 弹出式菜单

在 Win32 和 GTK+ GUI 上，你可以在光标位置下生成一个菜单。这和 PopUp 菜单类似，除了你可以弹出任何菜单树以外。

该命令只是为了后向兼容才存在，不鼓励使用。因为它的行为很奇特。

:popup :popu

:popu[p] {name} 弹出菜单 {name}。菜单名必须有至少一项子项，但不一定需要出现在菜单栏上 (见 hidden-menus)。{仅在 Win32 和 GTK GUI 或终端上有效}

:popu[p]! {name} 和上面类似，但使用鼠标指针所在的位置，而不是光标位置。
在终端中，这是最后已知的位置，通常这是最后点击或释放所在的位置 (和鼠标的移动不相干)。

示例:

```
:popup File
```

会使得 "File" 菜单 (如果有的话) 出现在文本光标 (如果有 ! 则是鼠标指针) 的位置上。

```
:amenu ]Toolbar.Make    :make<CR>
:popup ]Toolbar
```

会创建一个弹出式菜单，该菜单并不存在于主菜单栏中。

注意 在 GUI 上 :popup 命令立即返回，不等用户作出选择。终端中此命令则等待用户的选择完成。

注意 ']' 开始的菜单不会显示。

---

6. 字体

本小节描述字体相关的选项。

GUIFONT gui-font

'guifont' 是告诉 Vim 要使用什么字体的选项。它最简单的形式就是单个字体名。也可以是逗号分隔的字体名列表。使用第一个合法的字体。如果没有合法的字体则报错。

支持 'guifontset' 的系统上 (X11)，如果 'guifontset' 不为空，不使用 'guifont'。见 xfontset 。

注意: 对 GTK GUI 而言，非法的名字不会报错，总是选择并使用列表的首个单元。这是因为 GTK GUI 并不去识别给出名字的字体，而是把它作为一个模式去在可选的字体中寻

找最佳的匹配字体，从这个意义上，匹配永远不会失败。非法的名字不要紧，因为除了名字以外，还有一些其它的字体属性会帮助匹配的完成。

忽略逗号之后的空格。要在字体名里包含逗号，在它之前加上反斜杠。选项的设置需要在空格和反斜杠之前加上额外的反斜杠。另见 option-backslash 。比如:

```
:set guifont=Screen15,\ 7x13,font\\,with\\,commas
```

会使 Vim 先寻找字体 "Screen15"，如果失败，再找 "7x13"，最后 "font,with,commas" 。

如果没有载入任何字体，Vim 保持原先的设置。如果给出空的字体列表，Vim 会试图使用别的资源设置 (对 X 而言，会寻找 Vim.font 资源)，最后，选择总是可用的内建缺省值 (X 使用的是 "7x13")。给出的字体必须是 "正常" 字体。 Vim 会试图找到相关的粗体和斜体字体。

对于 Win32、GTK、Motif、Mac OS 和 Photon:

```
:set guifont=*
```

给出字体请求对话框，从中可以选择你需要的字体。

字体名取决于不同的 GUI。 setting-guifont 提供一个为不同系统设置 'guifont' 的方法。

GTK+ 2 和 3 GUI 的字体名看起来如此:

```
:set guifont=Andale\ Mono\ 11
```

就这样多。不使用 XLFD (X 逻辑字体描述)。有报告说对中文而言，可以这样设:

```
if has("gui_gtk2")
  set guifont=Bitstream\ Vera\ Sans\ Mono\ 12,Fixed\ 12
  set guifontwide=Microsoft\ Yahei\ 12,WenQuanYi\ Zen\ Hei\ 12
endif
```

(GTK+ 3 GUI 上把 gui_gtk2 换成 gui_gtk3)

Mac OSX 上，你可以这么用:

```
:set guifont=Monaco:h10
```

另见 'macatsui'。可以修正显示问题。

E236

注意 字体必须等宽 (所有字符相同宽度)。GTK 是例外: 可以接受所有字体，不过等宽字体的视觉效果最好。

要在 X11 上预览字体，可以使用 "xfontsel" 程序。"xlsfonts" 程序给出所有可用的字体的列表。

对 Win32 GUI 而言 E244 E245

- 字体名里接受如下选项 (选项间用 ':' 分隔):

```
        hXX - 高度为 XX (点 (point)，可以是浮点数)
        wXX - 宽度为 XX (点 (point)，可以是浮点数)
        WXX - 权重为 XX (见下面关于权重的 备注 )
        b   - 粗体。等价于权重设为 700。
        i   - 斜体
        u   - 下划线
        s   - 突出
        cXX - 字符集 XX。合法的字符集是: ANSI、ARABIC、BALTIC、
              CHINESEBIG5、DEFAULT、EASTEUROPE、GB2312、GREEK、HANGEUL、
              HEBREW、JOHAB、MAC、OEM、RUSSIAN、SHIFTJIS、SYMBOL、THAI、
```

TURKISH 和 VIETNAMESE。通常你会用 "cDEFAULT"。
qXX - 质量值 XX。合法的质量名为：PROOF、DRAFT、ANTIALIASED、NONANTIALIASED、CLEARTYPE 和 DEFAULT。通常你会用 "qDEFAULT"。若干质量值在旧式的操作系统上不支持。

- '_' 可以代替空格，这样就无须用反斜杠转义空格。

例如：

```
:set guifont=courier_new:h12:w5:b:cRUSSIAN
:set guifont=Andale_Mono:h7.5:w4.5
```

另见 font-sizes 。

关于权重的 备注：字体通常提供不同的权重。Windows 上的 "正常" 权重值为 400，如果未指定，这是寻找字体所使用的值。和名字比较，Windows 对字体的匹配更看重权重，这意味着可能会用某个 Book 或 Medium 的字体变体而不是 Light 或 ExtraLight 的变体。如果你觉得替代的字体权重太重了，可以显式设置低一点的权重值来抵消不合适替代的影响。

GUIFONTWIDE gui-fontwide

如果非空，'guifontwide' 指定逗号分隔的用于双宽字符的字体列表。使用第一个能载入的字体。

注意：这些字体必须使用 'guifont' 指定的两倍宽度和相同的高度。如果不符合要求，文本的绘制会不正确。

除了 GTK+ 之外的所有 GUI 版本：

'guifontwide' 只有在 'encoding' 设为 "utf-8" 并且 'guifontset' 为空或非法时才使用。
如果设置 'guifont' 并且从中找到合法的字体，但 'guifontwide' 为空的时候，Vim 会试图找到匹配的双宽的字体并设置 'guifontwide' 为此值。

仅限于 GTK+ GUI: guifontwide_gtk

如果设置且合法，即使 'encoding' 没有设为 "utf-8"，总为双宽字符使用 'guifontwide' 。
Vim 不会试图自动寻找 'guifontwide' 的合适的值。如果 'guifontwide' 为空，Pango/Xft 会选择 'guifont' 里没有的字符的字体。所以，你完全可以不必设置 'guifontwide'，除非你想改变 Pango/Xft 的选择。

仅限于 Windows +multibyte: guifontwide_win_mbyte

如果设置且合法，IME 使用 'guifontwide' 代替 'guifont'。

---

7. 附加 gui-extras

这一节说明其它和 GUI 相关的特性。

- 在 GUI 上，按 escape 之后不需要等一秒，因为没有键码会以 <Esc> 开始。

- 在 GUI 上，输入 ^V 紧跟一个特殊键会插入 "<Key>"，因为所用到的内部字符串没有意义。也可以按下修饰符，得到 "<Modifiers-Key>"。

- 在 GUI 上，修饰符 SHIFT、CTRL 和 ALT（或 META）可以在映射里和特殊键或鼠标一起使用。
  例如 :map <M-LeftDrag> <LeftDrag>

- 在 GUI 上，若干普通键在映射等命令里可以带修饰符，它们是 <Space>、<Tab>、<NL>、<CR> 和 <Esc>。

- 要在 Vim 脚本里检查是不是使用 GUI，你可以这么使用：

```
if has("gui_running")
   echo "是的，我们有 GUI"
else
   echo "无聊的老终端"
endif
```

setting-guifont

- 如果你在多个系统上使用同一个 vimrc 文件，你可以这样设置不同类型的 GUI 专用的选项：

```
if has("gui_running")
    if has("gui_gtk2")
        :set guifont=Luxi\ Mono\ 12
    elseif has("x11")
        " Also for GTK 1
        :set guifont=*-lucidatypewriter-medium-r-normal-*-*-180-*-*-m-*-*
    elseif has("gui_win32")
        :set guifont=Luxi_Mono:h12:cANSI
    endif
endif
```

MS Mincho 是一个推荐的日文字体。相关信息见：http://www.lexikan.com/mincho.htm

---

8. 外壳命令 gui-shell

X11 GUI 上，外部命令在 gvim 窗口里执行。见 gui-pty 。

警 告：在 X11 GUI 上执行外部命令不总是可以的。"普通" 的命令，像 "ls"、"grep"、"make" 大多数没问题。需要一个智能终端的命令，比如 "less" 和 "ispell" 就不行。有的可能挂起，需要从另外一个终端杀死进程才能终止。所以，要小心一点。

Win32 GUI 上，外部命令在一个单独的窗口上执行。见 gui-shell-win32 。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

gui_w32.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Dasn、Willis
http://vimcdoc.sf.net

Vim 的 Win32 图形用户界面 gui-w32 win32-gui

1. 启动图形界面 gui-w32-start
2. 设置 Vim 为默认的编辑器 vim-default-editor
3. 使用剪贴板 gui-clipboard
4. 命令行命令 gui-shell-win32
5. 特殊颜色 win32-colors
6. Windows 对话框和浏览器 gui-w32-dialogs
7. 命令行参数 gui-w32-cmdargs
8. 杂项 gui-w32-various

其它相关文章：
gui.txt 所有的 GUI 版本的通用功能
os_win32.txt Win32 版本特有的功能

---

1. 启动图形界面 gui-w32-start

Vim 的 Win32 GUI 版本 (gvim) 呈现给你的是一个真正 Windows 图形界面程序。无论你用什么方式启动它，也不管你是否修改了它的文件名，gvim 总是启动一个图形界面。

图形界面程序一直在 Windows 子系统里运行。大多数命令行会在启动 gvim 之后立即返回提示符。如果没有返回，那你应该使用 "start" 命令：

```
start gvim [option] file ..
```

E988

控制台版本用 -g 选项也可通过执行 gvim.exe 来启动 GUI：

```
vim -g [options] file ..
```

要可以这样，gvim.exe 必须和 vim.exe 在同一个目录，而且编译时要打开此特性。

也可从控制台版本使用 :gui 。不过，这是个实验特性且编译时必须打开此特性。它使用会话文件来在 GUI Vim 中重建控制台 Vim 的当前状态。

注意：所有的字体（无论粗体还是斜体）必须大小相同！！！如果不是，那你可能什么字都看不到或者只能看到一些杂乱无章的文字。Vim 不会检查文字的大小。需要强调一点，这里的字体大小是以 "像素" 为单位计算的。有些字体的大小以 "点数" 为单位计算，这时即便你选的两种字体的 "点" 数相同，但它们的像素数却可能不同。另外，与字体位置相关的属性（比如：ascent 和 descent）也必须相同。(ascent 和 descent 是用来说明字体高度的两种属性，具体可以参考相关资料 译者)

Win32 GUI 版本里有一个额外的菜单选项 "编辑/选择字体"，它能调用标准的 Windows 字体选择对话框。

设定菜单的高度对 Win32 GUI 不起作用。

gui-win32-maximized
如果你想让 Vim 在启动时最大化窗口，可以把下面的命令加到你的 vimrc 或 gvimrc 文

件中:

```
au GUIEnter * simalt ~x
```

插件方式使用 Vim gui-w32-windowid

gvim 正常启动时，建立它自己的顶层窗口。如果传给 Vim 命令行选项 --windowid 和一个十进制或十六进制的参数，Vim 会建立窗口号为给定 ID 的窗口的子窗口。这样，Vim 就可以作为另一个应用的插件。这实际上是一个程序员接口，必须要有配套的应用来正确启动 Vim 才行。

---

2. 设定 Vim 为默认的文本编辑器 vim-default-editor

设定 Vim 为某一类文件的默认编辑器:

1. 打开 Windows 资源管理器
2. 选择 查看/选项 -> 文件类型
3. 选定想让 gvim 处理的文件类型，然后选择 gvim 的路径。(在文件类型选项卡中，你也可以用 3 个空格给没有扩展名的文件指定关联程序)。在 "操作" 列表框的 open 一栏中输入:

   ```
   gvim "%1"
   ```

   这里加上引号可以让程序处理带空格的文件名。
   你也可以这样用:

   ```
   gvim "%L"
   ```

   这样可以在某些场合避免处理短文件名 (8.3 格式)，但我不敢保证在每个地方都好用。

现在你已经做好了 Vim 与文件的关联了，当你双击鼠标打开文件时，Vim 会切换到文件所在目录。

如果你想让 Vim 打开文件时最大化，可以在 Open 一栏中输入:

```
gvim -c "simalt ~x" "%1"
```

如果你把 Vim 放到了其它目录 (比如，你刚刚在别的目录里安装了一个新版本的 Vim，并且想马上用它)，用下面的办法也可以:

1. 选定想用 Vim 打开的文件
2. <Shift-F10>
3. 选 "打开方式……" 菜单项
4. 点击 "其它……"
5. 找到 Vim 的 (新) 位置，点击 "打开"
6. 在 "始终使用该程序打开这些文件" 划勾
7. <OK>

send-to-menu sendto

你也可以把 Vim 放到 "发送到" 菜单里去:

1. 打开资源管理器
2. 找到你的 sendto 目录:
   Windows NT: %windir%\profiles\%user%\sendto (例如:
   "c:\winnt\profiles\mattha\sendto")
   Windows XP: C:\Documents and Settings\%user%\SendTo
   Windows Vista: C:\Users\%user%\AppData\Roaming\Microsoft\Windows\SendTo 。
3. 点右键，选 "新建->快捷方式"
4. 在快捷方式向导程序里点击 "浏览"，找到 VIM/GVIM，程序会得到其完整路径。

当你使用 "发送到 Vim" 时，Vim 会切换到那个文件的目录。注意，任何长的目录名都会以短的 (MS-DOS) 的形式出现。这是 Windows "发送到" 功能的一个限制。

notepad
你可以用 gvim.exe 来替换 notepad.exe，但是这样会带来许多副作用。有些程序依赖于 notepad 参数。比如，有的程序调用 "notepad -p" 来打印文件，但是这些参数放到 gvim 上就不好用了。所以最好还是别改动 notepad，用别的方法启动 gvim。

win32-popup-menu
有一种更直接的方法，就是安装 "Edit with Vim" 选项到右键弹出菜单里，这样你就可以用 vim 编辑任何文件了。

这种方法可以和上面提到的默认打开方式 (文件关联) 共存。不同之处在于：如果把 Vim 设定为默认打开方式，当你用鼠标双击该类型的文件时，系统会自动调用 Vim 打开该文件。而如果你把 "用 vim 编辑" 加到弹出菜单里，就可以保留原来的文件关联，当你想用 Vim 处理该文件时，只要在文件上单击鼠标右键，选择 "Edit with Vim" 即可。例如：你可以把 "*.mak" 文件与 make 程序关联，这样你就可以通过双击文件来调用 make 程序执行它，通过右键 "Edit with Vim" 对 Makefile 进行编辑。

你可以用鼠标右键 "Edit with Vim" 打开你选择的文件。如果你选择了多个文件，你会发现弹出菜单里有两个相关选项：
"Edit with multiple gvims"   -- 为所选的每一个文件打开一个 gvim
"Edit with single gvim"   -- 用一个 gvim 打开所选的全部文件
如果已经有一个 gvim 正在运行，弹出菜单里还会多一个选项：
"Edit with existing gvim"   -- 用正在运行的 gvim 编辑所选文件

"edit with existing gvim" 菜单选项可以通过注册表关闭，在 HKLM\Software\Vim\Gvim 里加入名为 DisableEditWithExisting 的项目，可以是任何值。

install-registry
你可以通过安装程序 "install.exe" 十分容易地把 "Edit with Vim" 加到右键菜单中，安装程序会帮你在注册表中添加几个键值。

当然，你也可以自己修改注册表。这比较麻烦！尽量用 install.exe 程序。

1. 打开注册表编辑器 "regedit.exe"
2. 加入这些键值：

```
   key             value name                  value
   HKEY_CLASSES_ROOT\CLSID\{51EEE242-AD87-11d3-9C1E-0090278BBD99}
                   {default}                   Vim Shell Extension

HKEY_CLASSES_ROOT\CLSID\{51EEE242-AD87-11d3-9C1E-0090278BBD99}\InProcServer32
                   {default}                   {path}\gvimext.dll
                   ThreadingModel              Apartment
   HKEY_CLASSES_ROOT\*\shellex\ContextMenuHandlers\gvim
                   {default}                   {51EEE242-AD87-11d3-9C1E-0090278BBD99}
   HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Shell Extensions\Approved
                   {51EEE242-AD87-11d3-9C1E-0090278BBD99}
                                               Vim Shell Extension
   HKEY_LOCAL_MACHINE\Software\Vim\Gvim
                   path                        {path}\gvim.exe
   HKEY_LOCAL_MACHINE\Software\Microsoft\Windows\CurrentVersion\Uninstall\vim 5.6
                   DisplayName                 Vim 5.6: Edit with Vim popup menu entry
                   UninstallString             {path}\uninstal\.exe
```

用实际的路径替换 {path}。
不要输入 {default} (默认)，这是注册表键自身的值。

要从弹出菜单里删除 "Edit with Vim"，只要在注册表里删除上面提到的各项即可。卸载程序 "uninstall.exe" 可以帮你完成这项工作，用 "控制面板" 的 "添加/删除程序" 也可以。

如果你发现用 "添加/删除程序" 卸载 gvim 之后，其它类型文件的关联出错了，你可以自己动手修改过来 (用 "资源管理器"，前面已经提过)。这种情况好像只出现在 NT 系列的 Windows 上 (Windows 的漏洞?)。解决办法:
1. 找到文件的类型名。打开注册表编辑器在 \\HKEY_CLASSES_ROOT 下搜索就能找到。
2. 在资源管理器里，"查看" > "选项" > "文件类型"。找到该类型文件，单击 "编辑"。在 "操作" 列表中选择应该被设置为默认的选项 (正常情况下是 open)，单击 "设置为默认值"。

把 Vim 加入到 "Open With..." 关联菜单 win32-open-with-menu

如果你在 vim 安装程序里选择了 Add Vim to the "Open With..." menu (把 Vim 加入 "Open With..." 菜单)，这样你可以通过右键菜单调用 Vim 来编辑大多数文件，虽然不是所有的文件 (不清楚具体原因)。因而 "Edit with Vim" 菜单选项还是有用的。

还有一个原因，就是加入这个选项之后可以在 Internet Explorer 里直接调用 Vim 编辑当前的 HTML 文档。要使用这种功能，在 "工具" 菜单 "Internet 选项" 里，使用 "程序" 选项卡，"HTML 编辑器" 里选择 Vim。如果没有 Vim 选项，说明 Vim 安装的不正确。

手工安装可以使用下面的脚本:

```
REGEDIT4

[HKEY_CLASSES_ROOT\Applications\gvim.exe]

[HKEY_CLASSES_ROOT\Applications\gvim.exe\shell]

[HKEY_CLASSES_ROOT\Applications\gvim.exe\shell\edit]

[HKEY_CLASSES_ROOT\Applications\gvim.exe\shell\edit\command]
@="c:\\vim\\vim63\\gvim.exe \"%1\""

[HKEY_CLASSES_ROOT\.htm\OpenWithList\gvim.exe]

[HKEY_CLASSES_ROOT\*\OpenWithList\gvim.exe]
```

"c:\\vim\\vim63\\gvim.exe" 是 Vim 程序的实际位置，请根据具体情况修改。

删除 Vim 可以用 Vim 的卸载程序，或者用 "regedit" 手动删除注册表中的这些键值。

---

3. 使用剪贴板

gui-clipboard

Windows 有一个剪贴板，你可以把文本从别的程序 "剪" 下来，放到剪贴板里，也可以把剪贴板里的文本 "粘贴" 到别的程序里。在 Vim 里有多种办法完成这些操作。对于其它操作系统请参阅 gui-selections

寄存器 "* 反应剪贴板里的内容。 quotestar

当 'clipboard' 选项的值包括 "unnamed" 时，无名寄存器同样如此。这样，你不用在命令前加 "* 就可以从剪贴板做剪切/粘贴操作。

默认情况下，'guioptions' 选项里不包含标志位 'a'。这意味着如果你通过可视模式选定的文本不进行操作，那么它们不会被复制到剪贴板。当 'guioptions' 包含 'a' 时，在你选定文本的同时，这些被选定的文本已经被复制到剪贴板上。

mswin.vim

如果想用 CTRL-X，CTRL-C 和 CTRL-V 这些标准的 MS-Windows 操作方式怎么办？有办法！用 $VIMRUNTIME\mswin.vim 这个脚本解决。你可以把下面这条命令放到你的 _vimrc 文件里：

```
source $VIMRUNTIME/mswin.vim
```

这样一来 CTRL-C 就变成 "复制" 命令了，无法用作中止操作命令。改用 CTRL-Break。

CTRL-Z 用于撤销。这意味着你无法用此快捷键挂起 Vim，可用 :suspend 代替（如果支持的话）。

CTRL-V-alternative CTRL-Q

因为 CTRL-V 被用做 "粘贴"，所以你不能用它来启动 "可视列块" 的选择模式，不过可以用 CTRL-Q 代替。你也可以在插入模式和命令行模式中用 CTRL-Q 代替 CTRL-V。但是 CTRL-Q 常常被用作控制流，这时候它在某些终端下就不好用了。

备注：gvim 对剪贴板的支持仍有一些漏洞。见 todo 。

---

4. 命令行命令

gui-shell-win32

Vim 用另一个窗口执行外部命令，这使得 Vim 可以运行任何命令。外部命令运行时使用单独的环境，这和你在 DOS 提示符下运行命令是一样的。

win32-vimrun

Vim 通过调用 "vimrun" 间接执行外部命令。所以 vimrun.exe 必须包含在搜索路径内才能工作，或者把它和 gvim.exe 放到同一个目录下。如果找不到 "vimrun"，命令会直接执行，但 DOS 窗口会在程序运行完毕后立即关闭。
警 告：如果你这时用 "X" 按钮关闭命令提示符窗口，并且在弹出确认终止程序对话框时选择了 "是"，Vim 也会同时被关闭。（用 ":!start" 这种异步方式运行外部命令时，不会出现这种情况。）

Vim 使用的命令提示符窗口使用在 "控制面板" 里给 "Console" 预先定义的设置。

win32-!start

通常 Vim 在一个命令结束之前不会继续进行下面的工作（这对 Vim 从命令行中捕获输出是很有用的）。如果你想让 Vim 启动一个命令后立即返回，可以这样做：

```
:!start [/min] {command}
```

可选的 "/min" 使窗口最小化。

5. 特殊颜色 win32-colors

在 Win32 上，可以使用通常的 DOS 颜色。参见 dos-colors 。

另外，系统颜色也一样可以使用。它们以 Sys_XXX 的形式出现，这里 XXX 是相应的颜色名称（对大小写不敏感），如下表所示：（更翔实的描述可以参看 Win32 手册）。

```
Sys_3DDKShadow          Sys_3DFace              Sys_BTNFace
Sys_3DHilight           Sys_3DHighlight         Sys_BTNHilight
Sys_BTNHighlight        Sys_3DLight             Sys_3DShadow
Sys_BTNShadow           Sys_ActiveBorder        Sys_ActiveCaption
Sys_AppWorkspace        Sys_Background          Sys_Desktop
Sys_BTNText             Sys_CaptionText         Sys_GrayText
Sys_Highlight           Sys_HighlightText       Sys_InactiveBorder
Sys_InactiveCaption     Sys_InactiveCaptionText Sys_InfoBK
Sys_InfoText            Sys_Menu                Sys_MenuText
Sys_ScrollBar           Sys_Window              Sys_WindowFrame
Sys_WindowText
```

可能最有用的值是下面这些：

Sys_Window	正常的窗口背景颜色
Sys_WindowText	正常的窗口文字颜色
Sys_Highlight	加亮的背景颜色
Sys_HighlightText	加亮的文字颜色

还有一些额外可供使用的颜色：
Gray、Grey、LightYellow、SeaGreen、Orange、Purple、SlateBlue、Violet。

rgb.txt

此外，还可以用在文件 "rgb.txt" 中定义的颜色。这个文件对于 X11 的用户来说是再熟悉不过了。下面是从中抽取的几行：

```
255 218 185             peach puff
205 133  63             peru
255 181 197             pink
```

该文件看起来就是这个样子：3 个十进制数分别代表颜色的 R（红）、G（绿）、B（蓝），然后是颜色的名字。四个字段用空格分开。

你可以从任何的 X11 发行中得到 "rgb.txt"。它位于 "/usr/X11R6/lib/X11/" 之类的目录中。对于 Vim ，"rgb.txt" 必须放在 $VIMRUNTIME 目录里，也就是说，该文件可以在 "$VIMRUNTIME/rgb.txt" 找到。

---

gui-w32-dialogs dialog

6. Windows 对话框和浏览器

Vim 的控制台版本在各个平台上提供的用户界面是类似的，同样 Vim 的 Win32 GUI 版本也会用与其它平台相似的视窗组件进行操作。

6.1 对话框

用 "confirm" 命令族（例如：'confirm' 选项，:confirm 命令和 confirm() 函数）调用的对话框是基于 GUI 而不是像其它版本那样是基于控制台的。选项 'guioptions' 中的标志位 'c' 可以改变这种情况。

6.2 文件浏览器

如果你在文件编辑命令前加上 ":browse"，Vim 就会弹出一个文件选择窗口，选择现存文件。参见 :browse 。

6.3 撕下菜单（Tearoff Menus）

Vim 的 Win32 GUI 模拟 Motif 的 "撕下菜单"。在每项菜单的顶部有一个 "剪刀" 的标志。选择后会出现一个浮动窗口，窗口上的按钮与菜单上的选项一一对应。直接点击这些按钮与点击菜单里的选项是一样的（包括子菜单），只是不用每次到菜单里去找了。

当你要反复使用埋藏在子菜单里很深位置的命令时，这个功能就很有用了。

这些撕下菜单分离后，可以放在任何你喜欢的地方，并且总在 gvim 主窗口上方。你可以像一般的窗口一样关掉它们；如果你退出 gvim，它们也会自动关闭。

:tearoff :te

:te[aroff] {name} 撕下菜单 {name}。菜单中必须至少有一个选项，但是不必出现在主菜单栏里（参见 win32-hidden-menus ）。

例如：

```
:tearoff File
```

这条命令把 "File" 菜单（如果存在的话）撕下来。

```
:amenu ]Toolbar.Make    :make<CR>
:tearoff ]Toolbar
```

这两条命令创建了一个浮动菜单，这个菜单不在主菜单栏里。

注意：以 ']' 开头的菜单名不会在菜单栏里显示。

---

7. 命令行参数 gui-w32-cmdargs

命令行参数的行为和控制台应用的相同，见 win32-cmdargs 。

---

8. 杂项 gui-w32-various

gui-w32-printing

菜单 "文件/打印"，会打印出经过语法高亮的文本，参见 :hardcopy 。如果你只想打印原始文本，并且已经安装好了默认的打印机，可以用下面这条命令：

```
:w >>prn
```

Vim 支持许多 MS-Windows 的标准特性。一些相关的介绍可以在别的地方找到：参见 'mouse' ， win32-hidden-menus 。

drag-n-drop-win32

你可以把一个或多个文件拖到 Vim 窗口里，Vim 会用一般的方式打开它们。
参见 drag-n-drop 。

:simalt :sim

:sim[alt] {key} 模拟 Alt-{key} 组合键。
{仅适用 Win32 版本}
注意：":si" 意味着 ":s" 带 "i" 标志位。

正常情况下，为了增加键映射 (map) 的数量，Vim 控制所有 Alt-{key} 组合键。但是这样做可能与用 Alt-{key} 访问菜单的标准方法冲突。
一个快速的解决办法是：设置 'winaltkeys' 选项的值为 "yes"。但是这样阻止了所有与 Alt 键有关的映射。另一个办法是：设置 'winaltkeys' 选项为 "menu"。这样与菜单有关的快捷键有 Windows 管理，其余的与 Alt 相关的映射仍然好用。但这样又产生了对当前状态的依赖性。
要解决这个问题，就要用 :simalt 命令告诉 Vim ('winaltkeys' 不要设置为 "yes") 虚拟一个 Windows 风格的 Alt 按键行为。你可以用它来映射 Alt 组合键 (或者任何其它键) 来产生标准的 Windows 操作。下面是一些例子：

```
:map <M-f> :simalt f<CR>
```

这个命令通过把 Alt-F 映射为模拟按键的 Alt 和 F，使你按下 Alt-F 时弹出 "文件" 菜单 (对于缺省的 Menu.vim 而言)

```
:map <M-Space> :simalt ~<CR>
```

这个命令通过映射 Alt-Space 弹出 Vim 窗口的系统菜单。注意：~ 在 simalt 命令里代表 <Space> (空格)。

```
:map <C-n> :simalt ~n<CR>
```

把 CTRL-N 映射成 Alt-Space + N。也就是打开窗口的系统菜单，然后按 N，最小化 Vim 窗口。

备注 键映射取决于你使用的语言。

intellimouse-wheel-problems

用智能鼠 (Intellimouse) 的滑轮会引起 Vim 不能接收输入。到：

```
ControlPanel - Mouse - Wheel - UniversalScrolling - Exceptions
```

把 gvim 加到列表里就可以了。这个问题仅在使用智能鼠驱动程序 2.2 版本，并且打开 "Universal Scrolling" 时才会出现。

XPM 支持

w32-xpm-support

GVim 可以在 MS-Window 上编译时加入 XPM 文件的支持。 +xpm_w32
具体步骤见 Make_mvc.mak 文件，查找 XPM。

要测试 XPM 支持是否可用：

```
:help
:exe 'sign define vimxpm icon=' . $VIMRUNTIME . '\\vim16x16.xpm'
:exe 'sign place 1 line=1 name=vimxpm file=' . expand('%:p')
```

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

gui_x11.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Zimin<cranecai@users.sf.net>，tocer
http://vimcdoc.sf.net

Vim 的图形用户界面 gui-x11 GUI-X11
Athena Motif

1. 启动 X11 GUI gui-x11-start
2. GUI 资源 gui-resources
3. Shell 命令行 gui-pty
4. 杂项 gui-x11-various
5. GTK 版本 gui-gtk
6. GNOME 版本 gui-gnome
7. KDE 版本 gui-kde
8. 编译 gui-x11-compiling
9. X11 选择机制 x11-selection

其他相关文章：
gui.txt 所有的 GUI 版本的通用功能

---

1. 启动 X11 GUI gui-x11-start E665

你可以通过以下两种方式之一来启动 GUI 版本的 Vim：

```
gvim [options] [files...]
vim -g [options] [files...]
```

所以，如果你将可执行文件取名为 "gvim"，或者做一个叫 "gvim" 的链接，GUI 版本的 Vim 就会被自动运行。在 "gvim" 后也可以添加字符，如 "gvim-5"。

通过以下命令之一，你也可以从终端版本进入 GUI：

```
:gui [++opt] [+cmd] [-f|-b] [files...]  :gu  :gui
:gvim [++opt] [+cmd] [-f|-b] [files...]  :gv  :gvim
```

"-f" 参数在前台运行 Vim。
"-b" 参数在后台运行 Vim（默认）。
参考 ++opt 和 +cmd 。

gui-fork

当 GUI 启动时，它调用 fork() 并退出当前进程。当 gvim 从外壳启动时，这将允许外壳接受其后更多的命令。如果你不想这样（例如，当一邮件程序要等待运行的 gvim 退出时），可以用 "gvim -f"，"vim -gf" 或 ":gui -f" 来启动。不要用 "vim -fg"，因为 "-fg" 表示前台颜色。

当用 "vim -f" 启动，然后使用 ":gui"，Vim 将在前台运行。参数 "-f" 将被记住。要强迫 Vim 在后台运行，使用 ":gui -b"。

"gvim --nofork" 和 "gvim -f" 的效果一样。

有还在运行的作业时 Vim 不会 fork，因为进程不再是子进程。

E851 E852

GUI 方式启动失败时，Vim 仍会继续尝试以终端方式运行。

如果你想一直在前台运行 GUI，给 'guioptions' 加入 'f' 标志位。 -f 。

---

2. GUI 资源 gui-resources .Xdefaults

如果使用 Motif 或者 Athena 版本的 GUI（不是 KDE、GTK+ 或 Win32 版本），有很多 X 资源可以用。你应该使用类名 Vim 来设置。它们被列如下：

资源名称	意义
reverseVideo	布尔型：是不是翻转视频?
background	背景色。
foreground	普通文字颜色。
scrollBackground	滚动条槽的颜色。
scrollForeground	滚动条滑块和箭头颜色。
menuBackground	菜单背景色。
menuForeground	菜单前景色。
tooltipForeground	工具和气泡提示前景色。
tooltipBackground	工具和气泡提示背景色。
font	普通文字的字体名称。
boldFont	加粗文字的字体名称。
italicFont	斜体文字的字体名称。
boldItalicFont	加粗、斜体文字的字体名称。
menuFont	菜单字体名称。编译时没有 +xfontset 特性时使用
menuFontSet	菜单字体集名称。编译时有 +xfontset 特性时使用
tooltipFont	工具和气泡提示的字体名称。 当编译时带有 +xfontset 特性时，使用字体集名称。
geometry	启动时的 gvim 窗口大小（默认和启动它的终端窗口一样）。
scrollbarWidth	滚动条宽度。
borderWidth	文本区域周围边框的宽度。
menuHeight	菜单栏高度（仅对 Athena 有效）。

只有用户通过上述资源指定了粗体、斜体、斜粗体文本的特定字体时，Vim 才会使用它。Vim 不会根据正常字体来推测应该使用的字体。

备注：颜色也可用 ":highlight" 命令设置。可以使用这几个组："Normal"，"Menu"，"Tooltip"，"Scrollbar"。例如：

```
:highlight Menu guibg=lightblue
:highlight Tooltip guibg=yellow
:highlight Scrollbar guibg=lightblue guifg=blue
:highlight Normal guibg=grey90
```

font-sizes

备注：所有的字体（除了菜单和工具按钮提示）必须是同样大小！！！如果你不这样做，文字将无法显示或混成一团。Vim 不检查字体大小。必须一样的是以屏幕像素计算的尺寸。注意有些字体有同样的 "点" 数但是像素尺寸却不同！另外，与字体位置相关的属性摆放（ascent 和 descent）也必须相同。你可以用 "xlsfonts -l {fontname}" 核实。

如果上述任何一个资源其后用 Vim 命令设置，例如用 ":set guifont=Screen15"，X 资源的设置将被覆盖（目前唯一支持的选项是 'guifont'）。

这里有个例子，你可以考虑加入到 ~/.Xdefaults 文件中：

```
Vim*useSchemes:                 all
Vim*sgiMode:                    true
Vim*useEnhancedFSB:             true
Vim.foreground:                 Black
Vim.background:                 Wheat
Vim*fontList:                   7x13
```

前三个是 Silicon Graphics 机器的标准资源，这将使 Motif 应用效果更好，强烈推荐！

"Vim*fontList" 设置 Motif 的菜单字体。例如：

```
Vim*menuBar*fontList:         -*-courier-medium-r-*-*-10-*-*-*-*-*-*-*
```

在 Athena 上：

```
Vim*menuBar*SmeBSB*font:      -*-courier-medium-r-*-*-10-*-*-*-*-*-*-*
Vim*menuBar*MenuButton*font: -*-courier-medium-r-*-*-10-*-*-*-*-*-*-*
```

备注：一个更通用，更完善，可同时在 Motif 和 Athena 使用的设置字体的方法：

```
Vim.menuFont:         -*-courier-medium-r-*-*-10-*-*-*-*-*-*-*
```

或者，当编译进 +xfontset 特性时：

```
Vim.menuFontSet:      -*-courier-medium-r-*-*-10-*-*-*-*-*-*-*
```

用 "Vim.geometry" 替代 "Vim*geometry"。不然，菜单将被破坏。

如果出现 "Cannot allocate colormap entry for "gray60" 的错误信息，试试在你的 Vim 资源中加入这些（改变成你喜欢的颜色）：

```
Vim*scrollBackground:           Black
Vim*scrollForeground:           Blue
```

资源也可用 Vim 参数来设定：

参数	含义	
		-gui
-display {display}	在 {display} 上运行 Vim	-display
-iconic	图标化运行 Vim	-iconic
-background {color}	背景使用 {color}	-background
-bg {color}	同上	-bg
-foreground {color}	普通文字使用 {color}	-foreground
-fg {color}	同上	-fg
-ul {color}	同上	-ul
-font {font}	普通文字使用 {font}	-font
-fn {font}	同上	-fn
-boldfont {font}	粗体字使用 {font}	-boldfont
-italicfont {font}	斜体字使用 {font}	-italicfont
-menufont {font}	菜单项使用 {font}	-menufont
-menufontset {fontset}	菜单项使用 {fontset}	-menufontset
-mf {font}	同上	-mf
-geometry {geom}	初始大小设为 {geom}	-geometry
-geom {geom}	同上，参见 -geometry-example	-geom

-borderwidth {width} 使用宽度 {width} -borderwidth
-bw {width} 同上 -bw
-scrollbarwidth
-scrollbarwidth {width} 滚动条宽度使用 {width}
-sw {width} 同上 -sw
-menuheight {height} 菜单栏高度使用 {height} -menuheight
-mh {height} 同上 -mh
备注 :Motif 忽略此值，那里自动计算高度来使菜单合身
-reverse 使用反转视频 -reverse
-rv 同上 -rv
+reverse 不使用反转视频 -+reverse
+rv 同上 -+rv
-xrm {resource} 设定指定资源 -xrm

注意 有关反转视频的注意事项：Vim 检查最终效果是在暗的背景配亮文字。原因是有些版本的 X11 交换颜色，有些不这么做。以下两个例子都是设定了黄色文字、蓝色背景：

```
gvim -fg Yellow -bg Blue -reverse
gvim -bg Yellow -fg Blue -reverse
```

-geometry-example

一个尺寸的例子：

```
gvim -geometry 80x63+8+100
```

这个命令创建一个自左 8 像素，自顶 100 像素的一个 80 列宽，63 行高的窗口。

---

3. Shell 命令行 gui-pty

警 告：从 GUI 中执行外部命令并不总是可行的。像 "ls"、"grep"、 "make" 这样的普通命令一般都是有效的。那些需要智能终端的命令如，"less"、"ispell" 就未必了。有些甚至可能使 GUI 挂起，你从而必须从另外个终端里 kill 它。所以要小心。

有两种方式可以用外壳命令做 I/O 操作：管道和伪终端。默认是伪终端，它可以很好地工作在许多系统上。

不幸的是，伪终端的实现在每个 Unix 系统上是不同的。并且，一些系统上需要 root 权限。为了避免这类问题，可以在编辑文件之前先试一下。做好 kill 那个命令或 Vim 的准备。如 ":r !cat" 这样的命令会导致挂起的。

如果你的无法使用伪终端，可以复位 'guipty' 选项：

```
:set noguipty
```

管道可用在任一 Unix 系统上，但也有缺点：
- 一些外壳命令会注意到管道的存在，而作出不同的操作。如 ":!ls"，将在一栏里列文件。
- ":sh" 命令将不会显示提示符，但可以部分工作。
- 当使用 ":make" 时，很可能无法用 CTRL-C 中断。

当外部命令运行时，事先键入的内容经常丢失。这在管道和伪终端中，都有发生。这是个已知的问题，但看上去无法解决（至少很困难）。

gui-pty-erase

当你的删除键工作不正常时，你应该修改 "~/.cshrc" 文件，或者你所用的外壳的等同文

件。例如，当你使用退格键来删除字符时，反而产生了 "^H"，你可以添加以下内容到 "~/.cshrc" 中：

```
stty erase ^H
```

^H 是 CTRL-H，要键入它，用 CTRL-V CTRL-H。

---

4. 杂项 gui-x11-various

gui-x11-printing

"File/Print" 仅仅发送当前的缓冲区内容到 "lpr"。没有选项或其他东西。如果你需要其它效果，可以定义你自己的打印命令。如：

```
:10amenu File.Print :w !lpr -Php3
:10vmenu File.Print :w !lpr -Php3
```

X11-icon

默认的 Motif 或 Athena 版本的 Vim 使用黑白按钮。在 $VIMRUNTIME/vim32x32.xpm 包含一个彩色的 Vim 图标。在 GTK+，这是内建的图标。不幸的是，你该怎样安装它取决于你的窗口管理器。你可以从 'guioptions' 除去 'i' 标志，来除去黑白按钮：

```
:set guioptions-=i
```

如果你使用 fvwm* 系列窗口管理器，你只需要在 .fvwm2rc 配置文件中加入：

```
Style "vim"           Icon vim32x32.xpm
```

要保证图标文件的位置和窗口管理器 ImagePath 语句声明中所指定的一致。要么在 .fvwm2rc 修改图标路径，要么将图标放入以下预定义的任一目录中：

```
ImagePath /usr/X11R6/include/X11/pixmaps:/usr/X11R6/include/X11/bitmaps
```

备注：旧版本的 fvwm 使用 "IconPath" 而不是 "ImagePath"。

对于 CDE "dtwm" (Motif 的一个变体)，在 .Xdefaults 中加入这一行：

```
Dtwm*Vim*iconImage: /usr/local/share/vim/vim32x32.xpm
```

在 "mwm" (Motif 窗口管理器) 该行应是：

```
Mwm*Vim*iconImage: /usr/local/share/vim/vim32x32.xpm
```

X11 中可用的鼠标指针

X11_mouse_shapes

当使用 'mouseshape' 选项时，vim 进入各种模式时鼠标指针都会自动改变 (例如，在插入或命令模式中)。目前可用的指针如下：

arrow	一个指向西北的箭头
beam	像 I 的竖直条
size	一个指着上下的箭头
busy	一个时间漏斗
blank	一个不可见的指针
crosshair	一个瘦 "+" 符号
hand1	一个指向西北的暗手
hand2	一个指向西北的亮手
pencil	一支指向东南的笔
question	带问号的箭头

```
right_arrow          一个指向东南的箭头
up_arrow             一个指向上面的箭头
```

另外，任何 X11 内建的鼠标指针都可以通过指定一个在 X11/cursorfont.h 中定义的整数来使用。

如果使用一个在其它系统有效，而在 X11 中无效的名字，使用默认的 "arrow"。

---

5. GTK 版本 gui-gtk GTK+ GTK GTK3

GUI 的 GTK 版本的使用略有不同。

GTK__不__使用传统的 X 资源设置。所以，你的 ~/.Xdefaults 或 app-defaults 文件不会被使用。
不支持许多传统的 X 命令行参数。(例如，-bg，-fg 等)。支持的有：

命令行参数	资源名称	含义
-fn 或 -font	.font	文字字体名称
-geom 或 -geometry	.geometry	gvim 窗口尺寸
-rv 或 -reverse	*reverseVideo	白字黑底
-display		使用显示
-fg -foreground {color}		前景色
-bg -background {color}		背景色

要设置字体，参阅 'guifont' 。对 GTK，也可以使用相应的菜单选项设置。

另外，也有 GTK 内部处理的命令行参数。要知道如何使用以下的参数，请查看 GTK 文档：

```
--sync
--gdk-debug
--gdk-no-debug
--no-xshm        (GTK+ 2 版本不使用)
--xim-preedit    (GTK+ 2 版本不使用)
--xim-status     (GTK+ 2 版本不使用)
--gtk-debug
--gtk-no-debug
--g-fatal-warnings
--gtk-module
--display        (GTK+ 对应 -display 的选项；用法相同)
--screen         (屏幕号；用于对 GTK+ 2.2 多屏幕支持。)
```

以下的参数在用到 +netbeans_intg 特性时被忽略：

```
-xrm
-mf
```

关于颜色，vim 的颜色设置 (用于语法高亮) 仍然和传统的 Vim 方式一致。参考 :highlight 。

如果你想设置其它的 gui 组件 (如，菜单栏，滚动条，等等) 的颜色，那些是 GTK 的特别设置，你需要改动 gtkrc 之类的文件。至于如何改动，可参考 GTK 文档。更多的信息请参考 http://developer.gnome.org/doc/API/2.0/gtk/gtk-Resource-Files.html。

工具提示条色彩

gtk-tooltip-colors

如，设定工具提示条为黑字亮黄色背景：

```
style "tooltips"
{
        bg[NORMAL] = "#ffffcc"
        fg[NORMAL] = "#000000"
}

widget "gtk-tooltips*"          style "tooltips"
```

写入文件 ~/.gtkrc，这些设置就会被 GTK+ 使用。对于 GTK+ 2 而言，可能要修改的文件是 ~/.gtkrc-2.0。但各个发行版本不一样。

GTK+ 3 上，要达成类似的效果，可在 $XDG_HOME_DIR/gtk-3.0/gtk.css（见下个小节）中加入以下的 CSS 代码：

对 GTK+ 3 < 3.20 而言：

```
.tooltip {
        background-color: #ffffcc;
        color: #000000;
}
```

对 GTK+ 3 >= 3.20 而言：

```
tooltip {
    background-color: #ffffcc;
    text-shadow: none;
}

tooltip label {
    color: #2e3436;
}
```

GTK+ CSS 的快速一览

gtk-css

这一小段的内容只适用于 GTK+ 3.2 或以后版本，因为它们有对 GTK+ CSS 的稳定支持：

```
https://developer.gnome.org/gtk3/stable/theming.html
```

GTK+ 使用 CSS 来提供 小组件的风格和排版。本小段会通过简单和说明性的示例来提供 GTK+ CSS 的快速一览。

通常可以这样编辑此配置：

```
vim $HOME/.config/gtk-3.0/gtk.css
```

示例 1。空白空间调整

缺省 GTK+ 3 GUI 的工具条和标签条行比 GTK+ 2 GUI 稍大。有些人可能希望和 GTK+ 2 GUI 在大小上更加接近一些。

为此，我们要减少图标和标签周围看来过多的空白。

在 $XDG_HOME_DIR/gtk-3.0/gtk.css 中加入以下内容（通常是 $HOME/.config/gtk-3.0/gtk.css）：

```
toolbar button {
    margin-top: -2px;
    margin-right: 0px;
    margin-bottom: -2px;
    margin-left: 0px;

    padding-top: 0px;
    padding-right: 0px;
    padding-bottom: 0px;
    padding-left: 0px
}

notebook tab {
    margin-top: -1px;
    margin-right: 3px;
    margin-bottom: -1px;
    margin-left: 3px;

    padding-top: 0px;
    padding-right: 0px;
    padding-bottom: 0px;
    padding-left: 0px
}
```

因为这是 CSS，可用以下的缩写重写：

```
toolbar button {
    margin: -2px 0px;
    padding: 0px;
}

notebook tab {
    margin: -1px 3px;
    padding: 0px
}
```

备注：也可用 'toolbariconsize' 来调整图标大小。

备注：取决于图标方案和/或使用的字体，要达到满意的结果可能需要更多额外微调。

备注：除了 margin 和 padding 以外，还可用 border。具体细节可见 CSS 的盒模式，例如，

```
https://www.w3schools.com/css/css_boxmodel.asp
```

示例 2。色彩之外

GTK+ CSS 也支持渐变色：

```
tooltip {
    background-image: -gtk-gradient(linear,
                                    0 0, 0 1,
                                    color-stop(0, #344752),
                                    color-stop(0.5, #546772),
                                    color-stop(1, #243742));
}

tooltip label {
    color: #f3f3f3;
}
```

渐变色用于使 GUI 元素视觉上和其它元素视觉上有区隔。相应地，有效地使用渐变色是使主题方案的色彩和光线有统一的感觉一项有用的技术。

备注：制定主题方案是困难的，因为要让所有应用都同样悦目；使单个应用更吸引往往使其它的应用意料之外地不好看或破坏其可用性。要改进主题方案时请记住这一点。

示例 3。边框颜色

要最大化时去掉边框：

```
@define-color bg_color #1B2B34;
#vim-main-window {
    background-color: @bg_color;
}
```

使用 Vim 作为 GTK+ 的插件

gui-gtk-socketid

当 GTK+ 版本的 Vim 启动时，它会创建自己的顶层窗口（技术上，'GtkWindow'）。GTK+ 提供了用 GtkSocket 和 GtkPlug 组件（widget）嵌入的便利手段。如果一 GTK+ 应用程序在它的一个窗口里创建了一个 GtkSocket 组件，另外一个完全不同的 GTK+ 程序就可以使用该 socket 的 ID 来建立一个在顶层的 GtkPlug 组件，从而将自己完全嵌入前一个应用程序中。

如果你通过命令行参数 '--socketid' 传给 Vim 一个十进制或十六进制的值，Vim 将使用该值来创建一个 GtkPlug 组件，而不是普通的 GtkWindow。这将使 Vim 作为 GTK 的插件来运行。

这其实是一个编程的接口，必须有专门的应用程序支持（该程序必须正确地启动 Vim）。有关更多的 GTK+ socket 信息，参考 http://www.gtk.org/api/

备注：这个功能需要最新的 GTK 版本。GTK 1.2.10 仍然有小问题。socket 特性还没有在 GTK+ 2 下测试过，欢迎毛遂自荐。

---

6. GNOME 版本

gui-gnome Gnome GNOME

Gnome GUI 就像 GTK 一样工作。有关细节可以参考 GTK 。但是程序的外观有些不同，而且 Gnome GUI 有一个很重要的特性是 GTK+ 没有的：和会话管理器的交互。

gui-gnome-session

以下是外观上的不同之处：
- 使用 GNOME 对话框（仅对 GNOME 1 而言）。GNOME 2 GUI 使用和 GTK+ 2 一样的对话框。
- 使用 GNOME 锚接机制（dock），这使得工具栏、菜单栏可以移至不同的地方，而不是必须在顶部（如，工具栏可以放在上下左右任何一处）。只有在 GNOME 2 中，保存它们的位置。
- 那意味着菜单栏和工具栏的句柄回来了！Yeah！并且重设网格尺寸又可以工作了。

在 configure（配置）阶段如果发现 GNOME 或者使用 --enable-gnome-check 参数，GNOME 将被编译进来。

注意：GTK+ 3 GUI 版本不要用 --enable-gnome-check。上述的功能已经整合进了 GTK+ 3。

GNOME 会话支持

gui-gnome-session gnome-session

如果在退出登录时有任何改动过的缓冲区，Vim 会显示一个确认窗口。单击 [Cancel] 会终止退出的过程。否则当前的会话会被用 :mksession 命令存入硬盘，并在下次登陆时自动恢复。

GNOME 会话支持应该也对 KDE 会话管理器有效。如果你有在使用时遇到什么问题的话，请提交漏洞报告。

备注：会话的自动存储完全是透明的。这样避免了与你自己的会话文件，脚本和自动命令的冲突。具体的细节如下：
- 会话文件被存储于一个独立的路径（通常是 $HOME/.gnome2）。
- 'sessionoptions' 被忽略，Vim 会使用一些合适的，硬性规定的选项：
  blank,curdir,folds,globals,help,options,tabpages,winsize
- 在存储时，内部变量 v:this_session 的值不会被改变。同时，当再次登录时，该值会被恢复。

Vim 不会存储 GUI 窗口的大小和位置，因为这是窗口管理器的职责。但是如果编译了 GTK+ 2 支持，Vim 会通过恢复窗口角色的方法（用命令行参数 --role ），帮助窗口管理器识别窗口。

---

7. KDE 版本

gui-kde kde KDE KVim
gui-x11-kde

现在没有 Vim 的 KDE 版本。曾有过基于 Qt 工具包的一些移植工作，但是一直不能正常运行，而该项工作也已放弃。在 Yzis 上，还在开发。
https://github.com/chrizel/Yzis

---

8. 编译

gui-x11-compiling

如果使用 X11，Vim 的 configure 将首先试着在你的系统里找到需要的 GTK+ 的文件。如果同时有 GTK+ 2 和 GTK+ 3 可用，优先选择 GTK+ 2，除非给 configure 显式传入 --enable-gui=gtk3。

如果找不到 GTK+ 文件，就试着找 Motif 文件。如果还是找不到，就找 Athena 文件。如果全失败， GUI 会被禁用。

对于 GTK+，Vim 的配置过程使用 pkg-config(1) 来确保特定版本的 GTK+ 被正确安装并可用。因此，最好在运行 configure 前确认系统的 pkg-config 可用，且有所需的 GTK+ 版本的 .pc 文件。为此，运行以下命令来检查 pkg-config 是否可用于 GTK+ 2:

```
$ pkg-config --modversion gtk+-2.0
```

对 GTK+ 3，把 gtk+-2.0 替换为 gtk+-3.0。如果你有正确的 GTK+ 版本号，可以继续；不然，你可能要做些系统维护的例行工作来正确设置 pkg-config 和 GTK+。

缺省编译已经带了 GTK+ 2 GUI。所以，通常不需要传入 --enable-gui=gtk2 之类的选项来 configure 和编译。

可选地，GTK+ 2 GUI 可以整合进 GNOME 2 的支持。给 configure 传入 --enable-gnome-check 可以打开此项支持。

如要你想编译 GTK+ 3 GUI，需要给 configure 显式传入 --enable-gui=gtk3，并避免传入 --enable-gnome-check，因为 GNOME 2 支持所需的功能已经整合进了 GTK+ 3 里。

否则，如果你安装了 Motif 或 Athena，但配置程序无法找到它们所在的目录时，更改 Makefile 使之含有它们所在的目录名称。如，查找 "GUI_INC_LOC" 来设置 Motif 的目录，或 "GUI_OPT_X" 来设置 Athena 的目录。

gui-x11-gtk

目前，Vim 支持 GTK+ 2 和 GTK+ 3。

GTK+ 2 GUI 需要 GTK+ 2.2 或更新版本。

尽管 GTK+ 3 GUI 源代码的编写方式使得 3.x 系列的所有版本都可以编译通过，我们建议 GTK+ 3.10 或更新版本，因为此版本在重画上有较大的实现改动。

gui-x11-motif

对于 Motif，你需要至少 1.2 和/或 X11R5 版本。Motif 2.0 和 X11R6 也可以。Motif 1.1 和 X11R4 可能可以工作，但不保证 (可能有些问题，但也许稍作改动就可以编译、运行成功。如果你这样做了，请将补丁发给我)。最新版的 LessTif 据报告是可以的。

gui-x11-athena

Athena 版本默认使用 Xaw widget。如果你有 3D 版本的，有可能要用 Xaw3d 来连接。这样菜单看上去好看些。编辑 Makefile 文件并查找 "XAW_LIB"。滚动条仍然一样，因为 Vim 使用它自己的。它已经有 3D 效果了 (事实上，它更像 Motif 里的滚动条)。

gui-x11-neXtaw

neXtaw 版本和 Athena 版本非常像，但使用不同的窗口组件 (widget)。

gui-x11-misc

总的来说，不要混合不同版本的 GTK+，Motif，Athena，和 X11 的文件。这会引起问题。如，使用 X11R5 的头文件和 X11R6 的库，大半不能工作 (尽管连接不报错，Vim 之后的运行将崩溃)。

---

9. X11 选择机制 x11-selection

如果使用 X11，无论是 GUI 的还是 xterm 中运行的，Vim 都提供了多种使用 X11 的选择机制和剪贴板的方法。它们是通过两个选择寄存器 "* 和 "+ 来实现的。

X11 提供了两种基本类型的全局存储方式，选择区和剪切缓冲区。在一个重要的方面它们是有区别的：选择区是程序拥有的，并在程序退出时（如 Vim）消失，因而数据也消失了。而剪切缓冲区存储在 X 服务器的内部，会保持到被重写或 X 服务器退出（例如登出时）为止。

发起程序拥有选择区的内容（如，通过复制），仅当其它程序请求时才执行传送操作（如，通过粘贴）。

剪切缓冲区立即被写入，之后便可直接通过 X 服务器进行存取，无需与发起程序交互。

quoteplus quote+

文档上说，有三种 X 选择区：PRIMARY（用于代表当前的可视选择内容，就像 Vim 的可视模式的操作对象），SECONDARY（定义不明）和 CLIPBOARD（用于剪切、复制和粘贴操作）。

在三种选择区里，Vim 使用 PRIMARY 用于读写 "* 寄存器（因而，如果 X11 选择区可用，Vim 为 'clipboard' 的缺省值加上 "autoselect"）。CLIPBOARD 用于读写 "+ 寄存器。Vim 不使用 SECONDARY 选择区。

例如：（假定使用默认选项）
- 在 Vim 的可视模式里选中一个 URL。转到你的浏览器的 URL 文本框里并单击鼠标中键。所选择的文字将被插入（希望如此！）。备注：在 Firefox 中，可以在 about:config 里设置 middlemouse.contentLoadURL 属性为 true，这样在窗口的大多数位置单击鼠标中键都可以打开选中的 URL。
- 通过在你的浏览器中拖动鼠标选择一段文字。再到 Vim 中并按鼠标中键：选中的文字被插入。
- 在 Vim 里选中一段文字并执行 "+y。转到你的浏览器，拖动鼠标选中另一段文字。现在使用鼠标右键并在弹出菜单里选择 "Paste"。所选中的文字被 Vim 中选中的文字所替换。

备注：使用可视模式进行选择时，"+ 寄存器中的文字仍然存在。而选中的文字存于 "* 寄存器中。这样就可以选择被覆盖的文字了。

x11-cut-buffer

默认有 8 个剪切缓冲区：CUT_BUFFER0 到 CUT_BUFFER7。Vim 仅使用 CUT_BUFFER0，就是 xterm 默认使用的那一个。

只有当 Vim 不可用（或退出或挂起），并且因此不能响应另一个程序的选择区请求时，Vim 才将它自己的选择区内容写入 CUT_BUFFER0 中。如果是 Vim 拥有 "+ CLIPBOARD 选择区，那么优先写入它的值；否则，如果 Vim 拥有 "* PRIMARY 选择区，写入该值。

类似的，当 Vim 试图从 "* 或 "+ 来粘贴时（或显式或通过点击鼠标中键隐式地对 "* 寄存器操作），如果被请求的 X 选择区为空或不可用，Vim 退而求取 CUT_BUFFER0 的值。

备注：用这种方式复制文字到 CUT_BUFFER0 时，选择区的类型（字符，行或列块）总是被丢失，即使之后要粘贴的还是 Vim。

默认情况下 Xterm 总是将选择区内容同时写入 PRIMARY 和 CUT_BUFFER0 中。当它粘贴时，xterm 会优先选用 PRIMARY。如果失败才会选用 CUT_BUFFER0。因此，在 Vim 和 Xterm 之间剪切和粘贴时，你应该使用 "* 寄存器。Xterm 不用 CLIPBOARD，所以 "+ 在 xterm 中无效。

多数较新的应用程序会通过 PRIMARY ("*") 来提供它们当前的选择区内容，并使用 CLIPBOARD ("+") 用作剪切、拷贝、粘贴操作。你可以通过选择 "* 或 "+ 寄存器来访问两者。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

VIM 参考手册 by Andy Kahn

译者：lang2，tocer http://vimcdoc.sf.net

cscope Cscope

本文档阐述如何使用 Vim 的 cscope 接口。

Cscope 是一个类似 ctags 的工具。 你可以把它想作是超过频的 ctags，因为它功能比 ctags 强大很多。 在 Vim 里，通过 cscope 查询结果来跳转就像跳转到其他的标签完全一样；它被保存在标签栈里。这样你就可以像使用 tags 一样在函数等等之间便捷的跳转。

1. Cscope 简介 cscope-intro
2. Cscope 相关命令 cscope-commands
3. Cscope 选项 cscope-options
4. 如何在 Vim 中使用 cscope cscope-howtouse
5. 缺陷 cscope-limitations
6. 建议的使用方法 cscope-suggestions
7. 如何获取 cscope 等信息 cscope-info

目前为止 cscope 接口仅在 Unix 及 Win32 平台上有效。

---

1. Cscope 简介 cscope-intro

下面文本摘自 cscope 的手册页：

Cscope 是一个交互式的屏幕下使用的工具，用来帮助你：

无须在厚厚的程序清单中翻来翻去就可以认识一个 C 程序的工作原理。

无须熟悉整个程序就可以知道清楚程序 bug 所要修改的代码位置。

检查提议的改动（如添加一个枚举值）可能会产生的效果。

验证所有的源文件都已经作了需要的修改；例如给某一个现存的函数添加一个参数。

在所有相关的源文件中对一个全局变量改名。

在所有相关的位置将一个常数改为一个预处理符号。

它被设计用来回答以下的问题：

什么地方用到了这个符号?
这是在什么地方定义的?
这个变量在哪里被赋值?
这个全局符号的定义在哪里?
这个函数在源文件中的哪个地方?
哪些函数调用了这个函数?
这个函数调用了哪些函数?

信息 "out of space" 从哪来?
这个源文件在整个目录结构中处于什么位置?
哪些文件包含这个头文件?

Cscope 在第一次被使用在指定的源文件时会建立一个符号的数据库。接下来调用时，cscope 仅仅重建那些被改动或者和新文件相关的数据库。那些没有被改动的文件相关的数据库会被直接复制使用。这使得重建数据库要比第一次运行快许多。

当 cscope 正常调用时，你会得到一个全屏幕的选择窗口。你可以针对以上问题输入一个查询。然而，一旦找到一个匹配而你进入了一个文本编辑器来查看和编辑包含匹配的源文件时，你无法像在 vi 中使用 Ctrl-] 或 :tag 命令那样方便地跳转。

Vim 的 cscope 接口的工作原理是：先调用 cscope 的命令行接口，然后分析其输出结果从而找到匹配处。最终结果是：cscope 查询结果就像一般的标签一样。你可以使用一般的标签命令 (Ctrl-] 或 :tag) 然后再用 Ctrl-T 回跳。 (注意 不过，其实你不能直接使用 Ctrl-] 或 :tag 跳转到标签。在那之前你必须重新定义映射或设定一些选项值。下面的几节会告诉你 cscope 接口如何工作，以及建议的使用方式。)

---

2. Cscope 相关命令 cscope-commands

:cscope :cs :scs :scscope E259 E262 E561 E560

所有 cscope 命令都是通过 ":cscope" 命令的子项完成的。
:cscope 或 :cs 是主命令
:scscope 或 :scs 做同样的事并分割窗口
:lcscope 或 :lcs 使用位置列表，见 :lcscope

可用的子命令有：

E563 E564 E566 E568 E622 E623 E625
E626 E609

add : 添加一个新的 cscope 数据库／连接。

用法 :cs add {file|dir} [pre-path] [flags]

[pre-path] 用来通知 cscope 使用 -P [pre-path] 选项。

[flags] 可以用来给 cscope 传递额外的选项。

例子

```
:cscope add /usr/local/cdb/cscope.out
:cscope add /projects/vim/cscope.out /usr/local/vim
:cscope add cscope.out /usr/local/vim -C
```

cscope-find cs-find E567

find : 查询 cscope。除了第 5 个之外的所有 cscope 查询功能都可以使用。
第 5 个是 "Change this grep pattern"。

用法 :cs find {querytype} {name}

{querytype} 对应 cscope 控制台接口编号和 nvi 的缺省命令：

0 或 s: 查找本 C 符号
1 或 g: 查找本定义
2 或 d: 查找本函数调用的函数
3 或 c: 查找调用本函数的函数
4 或 t: 查找本字符串
6 或 e: 查找本 egrep 模式
7 或 f: 查找本文件
8 或 i: 查找包含本文件的文件
9 或 a: 查找此符号被赋值的位置

除了 4 和 6 之外的类型忽略 {name} 之前的空格。4 和 6 在 {querytype} 和 {name} 之间只能有一个空格分隔。多余的空格是 {name} 的一部分。

例如

```
:cscope find c vim_free
:cscope find 3  vim_free
```

这两个例子执行同样的查询：名为 "vim_free" 的函数。

```
:cscope find t initOnce
:cscope find t  initOnce
```

前者查找文本 "initOnce"，后者查找 " initOnce"。

```
:cscope find 0 DEFAULT_TERM
```

在 Vim 5.1 的源代码内执行以上的命令得到以下的结果：

```
Cscope tag: DEFAULT_TERM
   #   line  filename / context / line
   1   1009  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"amiga"
   2   1013  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"win32"
   3   1017  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"pcterm"
   4   1021  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"ansi"
   5   1025  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"vt52"
   6   1029  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"os2ansi"
   7   1033  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"ansi"
   8   1037  vim-5.1-gtk/src/term.c <<GLOBAL>>
             # undef DEFAULT_TERM
   9   1038  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"beos-ansi"
  10   1042  vim-5.1-gtk/src/term.c <<GLOBAL>>
             #define DEFAULT_TERM (char_u *)"mac-ansi"
  11   1335  vim-5.1-gtk/src/term.c <<set_termname>>
             term = DEFAULT_TERM;
  12   1459  vim-5.1-gtk/src/term.c <<set_termname>>
```

```
                if (STRCMP(term, DEFAULT_TERM))
  13    1826    vim-5.1-gtk/src/term.c <<termcapinit>>
                term = DEFAULT_TERM;
  14    1833    vim-5.1-gtk/src/term.c <<termcapinit>>
                term = DEFAULT_TERM;
  15    3635    vim-5.1-gtk/src/term.c <<update_tcap>>
                p = find_builtin_term(DEFAULT_TERM);
Enter nr of choice (<CR> to abort):
```

这样的输出显示了几类信息：
1. 标签号码（例子中有 15 个）。
2. 标签所在的行号。
3. 标签所在文件的文件名。
4. 标签的上下文（如：全局或函数名）。
5. 标签所在行的文本。

`help` ：显示一个简单的帮助。

用法　`:cs help`

E261

`kill` ：终止一个 cscope 连接（或终止所有 cscope 连接）。

用法　`:cs kill {num|partial_name}`

要终止一个连接，需要给出连接号或者部分的连接名称。部分名称可以是数据库文件路径的任何一部分。所以要当心使用这个功能！

如果给定的连接号是 -1，那么 所 有 的 cscope 连接都会被终止。

`reset` ：重新初始化所有连接。

用法　`:cs reset`

`show` ：显示当前的连接。

用法　`:cs show`

:lcscope　:lcs

此命令和 ":cscope" 命令相同，除了在置位选项 'cscopequickfix' 的时候，使用当前窗口的位置列表而不是 quickfix 列表来显示 cscope 的结果。

:cstag　E257　E562

如果你同时使用 cscope 和 ctags，:cstag 允许你在跳转前对两者都进行查询。例如，你可以选择先查询 cscope 数据库，如果无法找到匹配，再查询标签文件。这种做法的次序可以通过 csto 的值来调整。参见 cscope-options 。

:cstag 在搜索 cscope 数据库时执行和 ":cs find g" 加上标识符一样的操作。

:cstag 在搜索标签文件时执行和 :tjump 加上标识符一样的操作。

---

3. Cscope 选项　cscope-options

所有 cscope 有关的选项都可以用 :set 命令来设定。比较理想的做法是将这些设定添加到你的启动文件中（如：.vimrc）。某些 cscope 选项仅仅在 .vimrc 文件内才有效。在 vim 启动之后再设定它们的值没有任何效果！

cscopeprg csprg
'cscopeprg' 指定用来执行 cscope 的命令。缺省值是 "cscope"。例如：

```
:set csprg=/usr/local/bin/cscope
```

cscopequickfix csqf E469
{仅当编译时加入 +quickfix 特性才有效}
'cscopequickfix' 设定是否使用 quickfix 窗口来显示 cscope 结果。这是一组用逗号分隔的值。每个值可以代表一个 cscope-find 命令（s、g、d、c、t、e、f、i 或 a）加一个标志位（+、- 或 0）。
'+' 表示结果必须被添加到 quickfix 窗口中。'-' 表示清除上次的结果，'0' 或命令本身不存在表示不使用 quickfix。搜索从字符串首开始直到命令第一次出现时结束。缺省值是 ""（不使用 quickfix 窗口）。下面是个可能有用的值：

```
:set cscopequickfix=s-,c-,d-,i-,t-,e-,a-
```

cscopetag cst
若置位了 'cscopetag' 选项，":tag" 命令，CTRL-] 以及 "vim -t" 都会使用 :cstag 而不是缺省的 :tag。也就是说，设定 'cst' 选项意味着你总同时搜索 cscope 数据库和标签文件。缺省值为关。例如：

```
:set cst
:set nocst
```

cscoperelative csre
若置位了 'cscoperelative'，而没有给 cscope 提供前缀（即 cscope 的 -P 选项参数），cscope.out 所在位置的目录名（通常是项目的根目录）用作前缀来构造绝对路径。缺省关闭此功能。注意：此选项仅当 cscope（cscopeprg）启动时没有前缀路径（-P）时才适用。示例：

```
:set csre
:set nocsre
```

cscopetagorder csto
'csto' 的值决定 :cstag 命令查找的次序。如果 'csto' 被设为 0，cscope 数据库先被搜索，搜索失败的情况下在搜索标签文件。如果 'csto' 被设为 1，标签文件会在 cscope 数据库之前被搜索。缺省值为 0。
例：

```
:set csto=0
:set csto=1
```

cscopeverbose csverb
如果 'cscopeverbose' 没有被设定（缺省情况），当添加一个数据库时，成功与否不会被显示。理想的情况是，你先在 .vimrc 文件中复位该选项，再添加 cscope 数据库，然后再置位。这样，如果你在使用 vim 当中动态添加数据库，你就会被 vim 告知结果。例如：

```
:set csverb
:set nocsverb
```

cscopepathcomp cspc
'cspc' 的值决定显示文件路径的多少部分。在缺省值 0 的情况下整个路径都会被显示。1 只显示文件名，而不显示路径名。其它值会显示该值表示数目的部分。例如：

```
:set cspc=3
```

会显示文件路径的最后 3 个部分，包括文件名本身。

---

4. 如何在 Vim 中使用 cscope cscope-howtouse

首先你要做的是为你的源文件建立一个 cscope 数据库。最简单的做法是使用 "cscope -b" 命令。更详细的说明请查阅 cscope 的手册页。

假定你已经建立了一个 cscope 数据库，你需要将其添加 "add" 到 Vim 中来。这会建立一个 cscope "连接" 给 Vim 来使用。你可以把这些加入到你的 .vimrc 文件里。也可以在 vim 启动之后手动完成。例如，要添加一个 cscope 数据库 "cscope.out"，你可以这样做:

```
:cs add cscope.out
```

你可以用 ":cs show" 命令来检查一下结果。该命令会产生类似下面的结果:

```
# pid    database name                       prepend path
0 28806  cscope.out                          <none>
```

备注:
因为 Microsoft RTL 的缺陷，Win32 版本只能显示 0 而不是真正的进程 ID。

一旦建立了 cscope 连接，你就可以做各种查询并得到结果。查询所用的命令是 ":cs find"。例如:

```
:cs find g ALIGN_SIZE
```

这样要做很多的打字工作，所以有些麻烦。幸运的是，我们可以通过定义快捷键来避免过多的输入。参考 cscope-suggestions 可以得到一些使用的建议。

如果匹配结果只有一个，你会被自动带到该处。如果匹配结果多于一个，Vim 会显示一个选择屏幕让你来选择一处匹配。在你跳转到新的位置后，简单地用 Ctrl-T 就可以跳回原处。

---

5. 缺陷 cscope-limitations

Vim 对 cscope 的支持仅当所在系统支持以下四个系统调用时才可用: fork(), pipe(), execl(), waitpid()。这意味着它基本上仅限于 Unix 系统。

另外 cscope 支持在 Win32 也可以使用。更多的信息以及一个 Win32 版本的 cscope 可以在这里找到:

```
http://iamphet.nm.ru/cscope/index.html
```

http://cscope.sourceforge.net 的 DJGPP 编译版本已知不能和 Vim 一起工作。

硬性的限制: 在用 :cstag 搜索标签文件时要执行 :tjump，这一点是无法配置的 (如: 你不能选择使用 tselect)。

6. 建议的使用方法 cscope-suggestions

将下面的设定加入到你的 .vimrc 里（根据你的情况调整文件路径）：

```
if has("cscope")
        set csprg=/usr/local/bin/cscope
        set csto=0
        set cst
        set nocsverb
        " add any database in current directory
        if filereadable("cscope.out")
            cs add cscope.out
        " else add database pointed to by environment
        elseif $CSCOPE_DB != ""
            cs add $CSCOPE_DB
        endif
        set csverb
endif
```

我们设定了 'cscopetag'，这样所有的 :tag 命令就会实际上调用 :cstag。这包括 :tag、Ctrl-] 及 "vim -t"。结果是一般的 tag 命令不仅搜索由 ctags 产生的标签文件，同时也搜索 cscope 数据库。

有些用户也许向保留原来的标签命令，而用另外一个快捷键来执行 :cstag 命令。例如，你可以用下面的命令将 Ctrl-_（下划线）映射到 :cstag：

```
map <C-_> :cstag <C-R>=expand("<cword>")<CR><CR>
```

常用的 cscope 查询（用 ":cs find"）包括寻找所有调用指定函数的函数以及寻找所有出现某个指定的 C 符号的地方。你可以参照以下的映射：

```
map g<C-]> :cs find 3 <C-R>=expand("<cword>")<CR><CR>
map g<C-\> :cs find 0 <C-R>=expand("<cword>")<CR><CR>
```

这些对 Ctrl-]（右方括号）和 Ctrl-\（反斜杠）的映射使你可以将光标移动到你想作查询的函数或 C 符号，然后快速地查询 cscope 数据库。

或者，你也可以使用下面的方式。这些映射借鉴了 Cscope 主页 (http://cscope.sourceforge.net/) 上的 Vim/Cscope 教程：

```
nmap <C-_>s :cs find s <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>g :cs find g <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>c :cs find c <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>t :cs find t <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>e :cs find e <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>f :cs find f <C-R>=expand("<cfile>")<CR><CR>
nmap <C-_>i :cs find i ^<C-R>=expand("<cfile>")<CR>$<CR>
nmap <C-_>d :cs find d <C-R>=expand("<cword>")<CR><CR>
nmap <C-_>a :cs find a <C-R>=expand("<cword>")<CR><CR>

" Using 'CTRL-spacebar' then a search type makes the vim window
" split horizontally, with search result displayed in
" the new window.
```

```
nmap <C-Space>s :scs find s <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>g :scs find g <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>c :scs find c <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>t :scs find t <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>e :scs find e <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>f :scs find f <C-R>=expand("<cfile>")<CR><CR>
nmap <C-Space>i :scs find i ^<C-R>=expand("<cfile>")<CR>$<CR>
nmap <C-Space>d :scs find d <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space>a :scs find a <C-R>=expand("<cword>")<CR><CR>

" Hitting CTRL-space *twice* before the search type does a vertical
" split instead of a horizontal one

nmap <C-Space><C-Space>s
        \:vert scs find s <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>g
        \:vert scs find g <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>c
        \:vert scs find c <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>t
        \:vert scs find t <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>e
        \:vert scs find e <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>i
        \:vert scs find i ^<C-R>=expand("<cfile>")<CR>$<CR>
nmap <C-Space><C-Space>d
        \:vert scs find d <C-R>=expand("<cword>")<CR><CR>
nmap <C-Space><C-Space>a
        \:vert scs find a <C-R>=expand("<cword>")<CR><CR>
```

---

7. 如何获取 Cscope 等信息 cscope-info

如果你还没有 cscope (你的编译器或 OS 发布不包括 cscope)，你可以从下面的地址免费下载:

http://cscope.sourceforge.net/

这是 SCO 以 BSD 许可发布的。

在 Solaris 2.x 上，如果你有 C 编译器的许可，你也就有 cscope。两者通常都位于 /opt/SUNWspro/bin。

网上还有一个旧版本的 cscope 的克隆 (叫 "cs") 。出于各种原因，Vim 不支持该程序。

Vim cscope 接口/支持的原作者是 Andy Kahn <ackahn@netapp.com>。原始的结构 (包括很少一部分代码) 来自 nvi 的 cscope 接口。

cscope-win32

这里可以找到一个 Win32 版本的 cscope (看来已放弃):

https://code.google.com/archive/p/cscope-win32/

Win32 支持是 Sergey Khorev <sergey.khorev@gmail.com> 加入的。如果你有 Win32 相关的问题，请联系他。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Luis Carvalho
译者: Willis
http://vimcdoc.sf.net

Vim 的 Lua 接口 lua Lua

1. 命令 lua-commands
2. vim 模块 lua-vim
3. 列表用户数据 lua-list
4. 字典用户数据 lua-dict
5. blob 用户数据 lua-blob
6. 函数引用用户数据 lua-funcref
7. 缓冲区用户数据 lua-buffer
8. 窗口用户数据 lua-window
9. luaeval 函数 lua-luaeval
10. 动态调入 lua-dynamic

{仅当 Vim 编译时加入 +lua 特性时才有效}

---

1. 命令 lua-commands

:lua

:[range]lua {chunk}
执行 Lua 语言块 {chunk}。

示例:

```
:lua print("Hello, Vim!")
:lua local curbuf = vim.buffer() curbuf[7] = "line #7"
```

:[range]lua << [endmarker]
{script}
{endmarker}
执行 Lua 脚本 {script}。
注意: 如果编译时没有加入 Lua 特性，此命令不能工作。要避免错误，见 script-here 。

{endmarker} 之前 不能 有任何空白。

如果 "<<" 之后的 {endmarker} 省略，{script} 之后必须加上句号 '.'，就像 :append 和 :insert 命令那样。

这种形式的 :lua 命令主要用于在 Vim 脚本中嵌入 Lua 代码。

示例:

```
function! CurrentLineInfo()
```

```
lua << EOF
local linenr = vim.window().line
local curline = vim.buffer()[linenr]
print(string.format("Current line [%d] has %d chars",
        linenr, #curline))
EOF
endfunction
```

要看运行的 Lua 版本:

```
:lua print(_VERSION)
```

如果用 LuaJIT，也可以用:

```
:lua print(jit.version)
```

:luado

:[range]luado {body}　在 [range] 行范围的每行执行 Lua 函数 "function (line,linenr) {body} end"，其中，函数参数是每行的文本，结尾的 <EOL> 不计，和当前的行号。函数返回值为字符串时用来替代当前行的文本。缺省的 [range] 是整个文件: "1,$"。

示例:

```
:luado return string.format("%s\t%d", line:reverse(), #line)

:lua require"lpeg"
:lua -- balanced parenthesis grammar:
:lua bp = lpeg.P{ "(" * ((1 - lpeg.S"()") + lpeg.V(1))^0 * ")" }
:luado if bp:match(line) then return "-->\t" .. line end
```

:luafile

:[range]luafile {file}
　执行 Lua {file} 文件中的脚本。整个参数用作单个文件名。

示例:

```
:luafile script.lua
:luafile %
```

以上的命令都可执行一段 Lua 代码块 (chunk)，或从命令行 (:lua 和 :luado)，或从文件 (:luafile)，并可给出行范围 [range]。和 Lua 解释器类似，每个代码块都有自己的作用域，所以命令之间只有全局变量可以共享。所有的 Lua 缺省库都可用。此外，Lua 的 "print" 函数的输出重定向到 Vim 消息区，参数以空格而不是制表符分隔。

Lua 使用 "vim" 模块 (见 lua-vim) 来对 Vim 发出指令以及对缓冲区 (lua-buffer) 和窗口 (lua-window) 进行管理。不过在 sandbox 中执行命令时，修改缓冲区内容、打开新缓冲区和改变光标位置的过程受到限制。

2. vim 模块 lua-vim

Lua 通过 "vim" 模块和 Vim 进行接口。输入行范围的首末行分别存入 "vim.firstline" 和 "vim.lastline"。该模块也包含一些对缓冲区、窗口以及当前行查询的例程、Vim 调用和命令执行，以及其它各种操作。

vim.list([arg])

返回空列表或者，如果给出 "arg" 为带有数值键值 1, ..., n 的Lua 表 (即 "sequence")，返回列表 l，l[i] = arg[i] 对 i = 1, .., n (见 List)。非数值的键值不用于列表的初始化。转换规则参见 lua-eval 。示例:

```
:lua t = {math.pi, false, say = 'hi'}
:echo luaeval('vim.list(t)')
:" [3.141593, v:false], 'say' 被忽略
```

vim.dict([arg])

返回空字典或者，如果给出 "arg" 为 Lua 表，返回字典 d，d[k] = arg[k] 对所有 "arg" 中的字符串键值 k (见 Dictionary)。数值键值转化为字符串。非字符串的键值不用于列表的初始化。转换规则参见 lua-eval 。示例:

```
:lua t = {math.pi, false, say = 'hi'}
:echo luaeval('vim.dict(t)')
:" {'1': 3.141593, '2': v:false,
:" 'say': 'hi'}
```

vim.blob([arg])

返回空 blob 或者，如果给出 "arg" 为 Lua 字符串，返回 blob b，b 等价于作为字节字符串的 "arg"。
例如:

```
:lua s = "12ab\x00\x80\xfe\xff"
:echo luaeval('vim.blob(s)')
:" 0z31326162.0080FEFF
```

vim.funcref({name})

返回对应函数名 {name} 的函数引用 (参见 Funcref)。等价于 Vim 的 function()。

vim.buffer([arg])

如果 "arg" 是数值，返回缓冲区列表中编号为 "arg" 的缓冲区。如果 "arg" 为字符串，返回完整明或简短名为 "arg" 的缓冲区。这两种情况下，如果找不到缓冲区，返回 'nil'。此外，如果 "toboolean(arg)" 为 'true'，返回缓冲区列表的首个缓冲区，否则返回当前缓冲区。

vim.window([arg])

如果 "arg" 是数值，返回编号为 "arg" 的窗口，如果找不到，返回 'nil'。此外，如果 "toboolean(arg)" 为 'true'，返回首个窗口，否则返回当前窗口。

vim.type({arg})

返回 {arg} 的类型。等价于 Lua 的 "type" 函数，但若 {arg} 是列表、字典、缓冲区、窗口则分别返回 "list"、"dict"、"funcref"、"buffer" 或 "window"。示例:

```
:lua l = vim.list()
```

```
:lua print(type(l), vim.type(l))
:" list
```

vim.command({cmd}) 执行 vim (ex 模式) 命令 {cmd}。
示例:
```
:lua vim.command"set tw=60"
:lua vim.command"normal ddp"
```

vim.eval({expr}) 计算表达式 {expr} (见 expression)，把结果转化为 Lua 格式并返回。Vim 字符串和数值被直接转为响应的 Lua 字符串和数值类型。Vim 列表和字典被转化为 Lua 的用户数据 (userdata) (见 lua-list 和 lua-dict)。
示例:
```
:lua tw = vim.eval"&tw"
:lua print(vim.eval"{'a': 'one'}".a)
```

vim.line() 返回当前行 (没有结尾的 <EOL>)，Lua 字符串。

vim.beep() 鸣笛。

vim.open({fname}) 为文件 {fname} 打开新缓冲区并返回之。注意 并不把该缓冲区设为当前缓冲区。

---

3. 列表用户数据 lua-list

列表用户数据代表 vim 的列表，其接口尽量和 Vim 的列表语法相近。因为列表为对象，Lua 中列表引用的改变会反映在 Vim 中，反之亦然。列表 "l" 有以下属性和方法:

属性

- "#l" 是列表 "l" 的项目数，相当于 Vim 中的 "len(l)"。
- "l[k]" 返回 "l" 的第 k 项; "l" 索引从零开始，和 Vim 一样。要修改第 k 项，可用 "l[k] = newitem"; 特别地，"l[k] = nil" 删除 "l" 的第 k 项。
- "l()" 返回遍历 "l" 的 iterator。

方法

- "l:add(item)" 在 "l" 尾部附加 "item" 项目。
- "l:insert(item[, pos])" 在可选的 "pos" 指定的位置上插入 "item" 项目。"pos" 缺省值为 "0"。

示例:

```
:let l = [1, 'item']
:lua l = vim.eval('l') -- 相同的 'l'
:lua l:add(vim.list())
:lua l[0] = math.pi
:echo l[0] " 3.141593
```

```
:lua l[0] = nil -- 删除第一项
:lua l:insert(true, 1)
:lua print(l, #l, l[0], l[1], l[-1])
:lua for item in l() do print(item) end
```

---

4. 字典用户数据 lua-dict

和列表用户数据类似，字典用户数据代表 vim 的字典；既然字典也是对象，Lua 和 Vim 共享对象的引用。字典 "d" 有以下属性：

属性

- "#d" 是字典 "d" 的项目数，相当于 Vim 中的 "len(d)"。
- "d.key" 或 "d['key']" 返回 "d" "key" 对应项目的值。要修改其值，可用 "d.key = newvalue"；特别地，"d[k] = nil" 删除 "d" 中的对应项目。
- "d()" 返回遍历 "d" 的 iterator，等价于 Vim 的 "items(d)"。

示例：

```
:let d = {'n':10}
:lua d = vim.eval('d') -- 相同的 'd'
:lua print(d, d.n, #d)
:let d.self = d
:lua for k, v in d() do print(d, k, v) end
:lua d.x = math.pi
:lua d.self = nil -- 删除项目
:echo d
```

---

5. blob 用户数据 lua-blob

blob 用户数据代表 vim blob。blob "b" 有以下属性：

属性

- "#b" 是 blob "b" 的长度，等价于 Vim 中的 "len(b)"。
- "b[k]" 返回 "b" 的第 k 项；就像 Vim 那样，"b" 的索引从零开始。要修改第 k 项，简单地用 "b[k] = number"；特别地，"b[#b] = number" 可以在尾处附加一个字节。

方法

- "b:add(bytes)" 在 "b" 的尾处附加 "bytes"。

示例：

```
:let b = 0z001122
:lua b = vim.eval('b') -- 相同的 'b'
```

```
:lua print(b, b[0], #b)
:lua b[1] = 32
:lua b[#b] = 0x33 -- 在尾处附加一个字节
:lua b:add("\x80\x81\xfe\xff")
:echo b
```

+==========

6. 函数引用用户数据 lua-funcref

函数引用用户数据代表 Vim 的函数引用变量。定义时带有 "dict" 属性的函数引用需通过赋给某字典的一个键值来使其 "self" 对应该字典 (见下例)。函数引用 "f" 有以下属性:

属性

- "#f" 是 "f" 引用的函数名
- "f(...)" 调用 "f" 引用的函数 (带参数)

示例:

```
:function I(x)
:  return a:x
:  endfunction
:let R = function('I')
:lua i1 = vim.funcref('I')
:lua i2 = vim.eval('R')
:lua print(#i1, #i2) -- 都应返回 'I'
:lua print(i1, i2, #i2(i1) == #i1(i2))
:function Mylen() dict
:  return len(self.data)
:  endfunction
:let mydict = {'data': [0, 1, 2, 3]}
:lua d = vim.eval('mydict'); d.len = vim.funcref('Mylen')
:echo mydict.len()
:lua l = d.len -- 把 d 视为 'self'
:lua print(l())
```

---

7. 缓冲区用户数据 lua-buffer

缓冲区用户数据代表 vim 的缓冲区。缓冲区用户数据 "b" 包含以下属性和方法:

属性

- "b()" 设置 "b" 为当前缓冲区。
- "#b" 是缓冲区 "b" 的行数。
- "b[k]" 代表行号 k: "b[k] = newline" 把第 "k" 行替换为字符串 "newline", 还有 "b[k] = nil" 删除第 "k" 行。
- "b.name" 包含缓冲区 "b" 的简短名 (只读)。
- "b.fname" 包含缓冲区 "b" 的完整名 (只读)。

- "b.number" 包含缓冲区 "b" 在缓冲区列表的位置（只读）。

方法

- "b:insert(newline"[, pos]")" 在缓冲区 "pos"（可选）位置插入 "newline" 字符串。"pos" 缺省值为 "#b + 1"。如果 "pos == 0"，"newline" 将成为缓冲区的首行。
- "b:next()" 返回缓冲区列表中 "b" 的下一个缓冲区。
- "b:previous()" 返回缓冲区列表 "b" 的前一个缓冲区。
- "b:isvalid()" 如果缓冲区 "b" 对应 "真正的"（内存没有释放的）Vim 缓冲区时，返回 'true'（布尔值）。

示例：

```
:lua b = vim.buffer() -- 当前缓冲区
:lua print(b.name, b.number)
:lua b[1] = "first line"
:lua b:insert("FIRST!", 0)
:lua b[1] = nil -- 删除首行
:lua for i=1,3 do b:insert(math.random()) end
:3,4lua for i=vim.lastline,vim.firstline,-1 do b[i] = nil end
:lua vim.open"myfile"() -- 打开缓冲区，设为当前缓冲区

function! ListBuffers()
lua << EOF
local b = vim.buffer(true) -- 列表中的首个缓冲区
while b ~= nil do
        print(b.number, b.name, #b)
        b = b:next()
end
vim.beep()
EOF
endfunction
```

---

8. 窗口用户数据 lua-window

窗口对象代表 vim 窗口。窗口用户数据 "w" 有以下属性和方法：

属性

- "w()" 设置 "w" 为当前窗口。
- "w.buffer" 返回窗口 "w" 对应的缓冲区（只读）。
- "w.line" 返回窗口 "w" 的光标行位置。
- "w.col" 返回窗口 "w" 的光标列位置。
- "w.width" 代表窗口 "w" 的宽度。
- "w.height" 代表窗口 "w" 的高度。

方法

- "w:next()" 返回 "w" 的下一个窗口。
- "w:previous()" 返回 "w" 的前一个窗口。
- "w:isvalid()" 如果窗口 "w" 对应 "真正的" (内存没有释放的) Vim 窗口，返回 'true' (布尔值)。

示例:

```
:lua w = vim.window() -- 当前窗口
:lua print(w.buffer.name, w.line, w.col)
:lua w.width = w.width + math.random(10)
:lua w.height = 2 * math.random() * w.height
:lua n,w = 0,vim.window(true) while w~=nil do n,w = n + 1,w:next() end
:lua print("有 " .. n .. " 个窗口")
```

---

9. luaeval 函数 lua-luaeval lua-eval

和 "vim.eval" 对应，"luaeval" 在 Vim 中接受 Lua 值的传入。"luaeval" 接受一个表达式字符串以及一个可选的参数，并返回表达式计算的结果。语义上，等价于在 Lua 里进行如下的操作:

```
local chunkheader = "local _A = select(1, ...) return "
function luaeval (expstr, arg)
    local chunk = assert(loadstring(chunkheader .. expstr, "luaeval"))
    return chunk(arg) -- return typval
end
```

注意 "_A" 接收给 "luaeval" 的参数。Lua 数值，字符串，列表，字典、blob 和函数引用用户数据转化为 Vim 对应的类型，Lua 布尔型转化为数值。试图转换其他 Lua 类型的企图，包括非列表、字典、blob、函数引用的用户数据，会抛出错误。

示例:

```
:echo luaeval('math.pi')
:lua a = vim.list():add('newlist')
:let a = luaeval('a')
:echo a[0] " 'newlist'
:function Rand(x,y) " random uniform between x and y
:  return luaeval('(_A.y-_A.x)*math.random()+_A.x', {'x':a:x,'y':a:y})
:  endfunction
:echo Rand(1,10)
```

---

10. 动态调入 lua-dynamic

MS-Windows 和 Unix 上，可以动态调入 Lua 库。此时 :version 输出包含 +lua/dyn 。

这意味着 Vim 只有在必要时才寻找 Lua DLL 或共享文件。如果不使用 Lua 接口，你就不需要它们。这样，即使没有这些 DLL 文件，你也可使用 Vim。

MS-Windows

要使用 Lua 接口，Lua DLL 必须在搜索路径上。控制台窗口里输入 "path" 可以看到（搜索路径）当前使用的目录。也可用 'luadll' 选项指定 Lua DLL。DLL 的版本必须和 Vim 编译使用的 Lua 版本保持一致。

Unix

'luadll' 选项可用来指定 Lua 共享库文件，而不用编译时指定的 DYNAMIC_LUA_DLL 文件。共享库的版本必须和 Vim 编译使用的 Lua 版本保持一致。

---

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

Vim 的 MzScheme 接口 mzscheme MzScheme

1. 命令 mzscheme-commands
2. 示例 mzscheme-examples
3. 线程 mzscheme-threads
4. MzScheme 对 Vim 的访问 mzscheme-vim
5. mzeval() Vim 函数 mzscheme-mzeval
6. 使用函数引用 mzscheme-funcref
7. 动态调入 mzscheme-dynamic
8. MzScheme 设置 mzscheme-setup

{仅当 Vim 编译时加入 +mzscheme 特性才有效}

来自于 Brent Fulgham 的工作。
动态调入部分则由 Sergey Khorev 加入。

MzScheme 和 PLT Scheme 已经重新命名为 Racket。详情可查阅
http://racket-lang.org。

Racket 5.x 版本直至 5.3.1（包含）的 futures 和 places 和 Vim 创建的进程不能一同工作。
最简单的解决方法是自己编译 Racket，关闭以下这些特性：

```
./configure --disable-futures --disable-places --prefix=your-install-prefix
```

要加快进程的速度，也可考虑使用 --disable-gracket 和 --disable-docs。

---

1. 命令 mzscheme-commands

:mzscheme :mz

:[range]mz[scheme] {stmt}
执行 MzScheme 语句 {stmt}。

:[range]mz[scheme] << [endmarker]
{script}
{endmarker}
执行内含 MzScheme 脚本 {script}。
注意：如果编译时不带 MzScheme 特性，此命令会失败。要避免错误，见 script-here 。

{script} 之下的 {endmarker} 必须 没有 前导的空白。

如果省略 "<<" 之后的 [endmarker]，在 {script} 后一定要有一个点 '.'，就像 :append 和 :insert 命令那样。

:mzfile :mzf

:[range]mzf[ile] {file} 执行 {file} 文件里的 MzScheme 脚本。

这些命令基本上做同样的一件事 - 执行一段 MzScheme 代码。执行期间，"当前行范围"设为给定的行范围。

:mzscheme 执行的代码来自命令行。
:mzfile 执行的代码来自给定文件的内容。

MzScheme 接口定义从 exn 派生的 exn:vim 例外。若干 Vim 错误抛出此例外。

编译时，MzScheme 接口记住当前的 MzScheme collection 路径。如果你想指定额外的路径，使用 'current-library-collection-paths' 参数，例如，要附加 (cons) 用户的局部 MzScheme collection 路径:

```
:mz << EOF
(current-library-collection-paths
    (cons
        (build-path (find-system-path 'addon-dir) (version) "collects")
        (current-library-collection-paths)))
EOF
```

vimext 模块提供所有的功能。

exn:vim 无需显式导入 (import)。

为了避免和 MzScheme 冲突，require 模块时考虑使用 prefix。
例如:

```
:mzscheme (require (prefix vim- vimext))
```

下面的例子都使用此命名方案。

mzscheme-sandbox

在沙盘 sandbox 里执行时，对一些文件系统和 Vim 接口的过程的访问受到限制。

---

2. 示例 mzscheme-examples

```
:mzscheme (display "Hello")
:mz (display (string-append "Using MzScheme version " (version)))
:mzscheme (require (prefix vim- vimext)) ; 用于 MzScheme < 4.x
:mzscheme (require (prefix-in vim- 'vimext)) ; MzScheme 4.x
:mzscheme (vim-set-buff-line 10 "This is line #10")
```

要看运行的 MzScheme 版本:

```
:mzscheme (display (version))
```

内含脚本的使用:

```
function! <SID>SetFirstLine()
    :mz << EOF
    (display "!!!")
    (require (prefix vim- vimext))
    ; 更新的版本 (require (prefix-in vim- 'vimext))
    (vim-set-buff-line 1 "This is line #1")
```

```
            (vim-beep)
            EOF
        endfunction

        nmap <F9> :call <SID>SetFirstLine() <CR>
```

文件执行:

```
        :mzfile supascript.scm
```

Vim 例外处理:

```
        :mz << EOF
        (require (prefix vim- vimext))
        ; 更新的版本 (require (prefix-in vim- 'vimext))
        (with-handlers
          ([exn:vim? (lambda (e) (display (exn-message e)))])
          (vim-eval "nonsense-string"))
        EOF
```

vimext 模块自动实例化 (可以放在 vimrc 里):

```
    function! MzRequire()
        :redir => l:mzversion
        :mz (version)
        :redir END
        if strpart(l:mzversion, 1, 1) < "4"
            " MzScheme 版本号 < 4.x:
            :mz (require (prefix vim- vimext))
        else
            " 更新的版本:
            :mz (require (prefix-in vim- 'vimext))
        endif
    endfunction

    if has("mzscheme")
        silent call MzRequire()
    endif
```

---

3. 线程 mzscheme-threads

MzScheme 接口支持线程。它们和操作系统的线程相互独立，因此调度是必要的。
'mzquantum' 选项决定 Vim 应每隔多久轮询可用的 MzScheme 线程。

备注

Vim 控制台版本的线程调度不如 GUI 版本的可靠。

---

4. MzScheme 对 Vim 的访问 mzscheme-vim

mzscheme-vimext

'vimext' 模块提供对 MzScheme 接口定义的例程的访问。

通用

(command {command-string}) 执行 Vim ":Ex" 风格的命令。
(eval {expr-string}) 计算 vim 表达式，返回相应的 MzScheme 对象: List 返回 Scheme 列表， Dictionary 返回 哈希表， Funcref 返回函数 (另见 mzscheme-funcref)
注意 这和 MzScheme 的 eval 有冲突，可用模块限定符避免之。
(range-start) Scheme 命令传递的行范围的开始行
(range-end) 和结束行。
(beep) 响铃
(get-option {option-name} [buffer-or-window]) 得到 Vim 选项值 (局部或全局值，见 set-option)。
(set-option {string} [buffer-or-window])
设置 Vim 选项。字符串 {string} 使用设置选项形式 (optname=optval 或 optname+=optval 等)。如果带 {buffer} 或 {window}，设置局部选项。{buffer-or-window} 也可以使用符号 'global，这时就会使用 :setglobal 。

缓冲区 mzscheme-buffer

(buff? {object}) 对象是缓冲区么?
(buff-valid? {object}) 对象是合法的缓冲区 (也就是，对应真正 Vim 缓冲区) 么?
(get-buff-line {linenr} [buffer])
得到缓冲区的行。
(set-buff-line {linenr} {string} [buffer])
设置缓冲区的行。如果 {string} 是 #f，删除得到的行。[buffer] 参数可选。如果省略，使用当前缓冲区。
(get-buff-line-list {start} {end} [buffer])
得到缓冲区的行列表。{Start} 和 {end} 从 1 开始。包含第 {Start} 和 {end} 行。
(set-buff-line-list {start} {end} {string-list} [buffer])
设置缓冲区的行列表。如果字符串列表 {string-list} 是 #f 或 null，删除所有行。如果列表短于 {end}-{start}，删除其余行。
(get-buff-name [buffer]) 得到缓冲区的文本名。
(get-buff-num [buffer]) 得到缓冲区的编号。
(get-buff-size [buffer]) 得到缓冲区的行数。
(insert-buff-line-list {linenr} {string/string-list} [buffer])
在缓冲区第 {linenr} 行之后插入行列表。如果 {linenr} 为 0，在缓冲区开头插入。
(curr-buff) 得到当前缓冲区。使用其它的 MzScheme 接口历程来修改当前缓冲区。
(buff-count) 得到编辑器的缓冲区总数。
(get-next-buff [buffer]) 得到下一个缓冲区。
(get-prev-buff [buffer]) 得到前一个缓冲区。如果没有更多缓冲区，返回 #f。
(open-buff {filename}) 打开新缓冲区 (编辑文件 "name")。
(get-buff-by-name {buffername}) 根据文件名得到缓冲区，如果没有此缓冲区，返回 #f。
(get-buff-by-num {buffernum}) 根据缓冲区号得到缓冲区 (如果没有对应此编号

的缓冲区，返回 #f)。

窗口 mzscheme-window

(win? {object})	对象是窗口么?
(win-valid? {object})	对象是合法的窗口（也就是，对应真正 Vim 窗口）么?
(curr-win)	得到当前窗口。
(win-count)	得到窗口总数。
(get-win-num [window])	得到窗口的编号。
(get-win-by-num {windownum})	根据窗口号得到窗口。
(get-win-buffer [window])	得到给定窗口的缓冲区。
(get-win-height [window])	
(set-win-height {height} [window])	取得/设置 窗口高度。
(get-win-width [window])	
(set-win-width {width} [window])	取得/设置 窗口宽度。
(get-win-list [buffer])	得到对应特定缓冲区的窗口列表。
(get-cursor [window])	得到窗口的光标位置，以组对（行号 . 列号）形式出现。
(set-cursor (line . col) [window])	设置光标位置。

---

5. mzeval() Vim 函数 mzscheme-mzeval

使用 mzeval() 函数可以提供另一方向的接口，它计算 MzScheme 表达式并把结果赋给 Vim 脚本。

---

6. 使用函数引用 mzscheme-funcref

MzScheme 接口允许使用 Funcref 来从 Scheme 里直接调用 Vim 的函数。例如:

```
function! MyAdd2(arg)
    return a:arg + 2
endfunction
mz (define f2 (vim-eval "function(\"MyAdd2\")"))
mz (f2 7)
```

或 :

```
:mz (define indent (vim-eval "function('indent')"))
" return Vim indent for line 12
:mz (indent 12)
```

---

7. 动态调入 mzscheme-dynamic E815

MS-Windows 上，可以动态调入 MzScheme 库。 :version 输出这时应包括 +mzscheme/dyn 。

这意味着 Vim 只有在必要时才寻找 MzScheme DLL 文件。如果不使用 MzScheme 接口，你就不需要它们。这样，即使没有这些 DLL 文件，你也可使用 Vim。
注意 新版的 MzScheme (Racket) 需要通过 scheme_main_setup 进行早期（蹦床式的）的初始化。这样如果可能的话 Vim 总能在启动时载入 MzScheme DLL。但会减慢 Vim 的

启动。

要使用 MzScheme 接口，MzScheme DLL 必须在搜索路径上。控制台窗口里输入 "path" 可以看到（搜索路径）当前使用的目录。

MS-Windows 上，'mzschemadll' 和 'mzschemagcdll' 选项用于载入的库名。初始值在编译时决定。

DLL 的名字必须匹配 Vim 编译时所使用的 MzScheme 版本。对于 MzScheme 209 版本，它们的名字分别为 "libmzsch209_000.dll" 和 "libmzgc209_000.dll"。要确信这一点，察看 ":version" 命令的输出，注意 "Compilation" 信息中的 -DDYNAMIC_MZSCH_DLL="内容" 和 -DDYNAMIC_MZGC_DLL="内容"。

例如，如果 MzScheme (Racket) 安装在 C:\Racket63，可能需要以下的环境变量设置：

```
PATH=%PATH%;C:\Racket63\lib
PLTCOLLECTS=C:\Racket63\collects
PLTCONFIGDIR=C:\Racket63\etc
```

---

8. MzScheme 设置 mzscheme-setup E895

Vim 的 if_mzsch 核心部分需要 "racket/base" 模块（如果不存在，"scheme/base" 作为后备），测试需要 "r5rs" 模块，Vim 编译时需要 "raco ctool" 命令。如果 MzScheme 没有这些，可以通过 MzScheme 的 raco 命令安装：

```
raco pkg install scheme-lib       # scheme/base 模块
raco pkg install r5rs-lib         # r5rs 模块
raco pkg install cext-lib         # raco ctool 命令
```

---

vim:tw=78:ts=8:noet:sts=4:ft=help:norl:

if_perl.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Sven Verdoolaege
and Matt Gerassimof
译者：foolstone@ustc.edu，Willis，tocer
http://vimcdoc.sf.net

Perl 与 Vim perl Perl

1. 编辑 Perl 文档 perl-editing
2. 编译带 Perl 接口的 Vim perl-compiling
3. Perl 接口的使用 perl-using
4. 动态调入 perl-dynamic

{仅当编译 Vim 时加入 +perl 特性才有效}

---

1. 编辑 Perl 文件 perl-editing

Vim 语法高亮显示支持 Perl 和 POD 文件。Vim 假设后缀为 .pl 和 .pm 的文件是 Perl 文件。Vim 同时不管文件后缀而测试文件的首行来检查一个文件是否为 Perl 脚本（参见 Vim 的 syntax 目录下的 scripts.vim）。Vim 假设带后缀 .POD 的文件是 POD 文本。

要使用 Perl 标签，你需要 Exuberant ctags 的最新版本。在这里可以找到它：
http://ctags.sourceforge.net

当然，作为选择你还可以用一个叫 pltags.pl 的 Perl 脚本，它和 Vim 一起发行的，位于 $VIMRUNTIME/tools 目录下。这个脚本比 Exuberant ctags 的 Perl 支持具备更多的功能。

---

2. 编译带 Perl 接口的 Vim perl-compiling

为了编译带 Perl 接口的 Vim，需要 Perl 5.004（或者更新）的版本。Perl 必须在你编译 Vim 前安装。 Vim 的 Perl 接口 不能 和官方发行的 Perl 5.003 一起正常工作！它也许能和 Perl 5.003_05 或者更新版本一起正常工作。
[译者注]到翻译的时候，这个问题已经不会是大问题了

Vim 的 Perl 补丁由
Sven Verdoolaege<skimo@breughel.ufsia.ac.be> 和
Matt Gerassimof
制作。

Perl 的 MS-Windows（和其它平台）版本可以在下面这个网址找到：
http://www.perl.org/
ActiveState 的版本应该可用，Strawberry Perl 是另一个好的版本。

---

3. Perl 接口的使用 perl-using

:perl :pe

:pe[rl] {cmd}　　执行 Perl 命令 {cmd}。当前包是 "main"。测试 :perl 是否可用的简单例子：

```
:perl VIM::Msg("Hello")
```

:pe[rl] << [endmarker]
{script}
{endmarker}

执行 Perl 脚本 {script}。
{script} 之下的 {endmarker} 不能 有任何前导空白。

如果省略了 [endmarker]，就会像 :append 和 :insert 命令那样，默认为点 '.'。在函数内使用时，最好指明 '.'，因为 "$i;" 看起来像 Vim 的 :insert 命令的开始。

:perl 命令的这种形式主要是用来在 vim 脚本里面包含 perl 代码。
备注：当 Perl 特性没有被编译进 vim 时这条命令不能工作。为了避免出错，请参考 script-here 。

vim 脚本示例：

```
function! WhitePearl()
perl << EOF
        VIM::Msg("pearls are nice for necklaces");
        VIM::Msg("rubys for rings");
        VIM::Msg("pythons for bags");
        VIM::Msg("tcls????");
EOF
endfunction
```

要看运行的 Perl 的版本：

```
:perl print $^V
```

:perldo :perld

:[range]perld[o] {cmd}　　对 [range] 中的每行执行 Perl 命令 {cmd}，同时 $_ 依次被赋值为每行的文本，但不包括行尾的 <EOL>。改变 $_ 会改变每行的内容，但 注意 不能用这个命令添加或者删除行。
[range] 的缺省值是整个文件："1，$"。

可以试一下下面的代码：

```
:perl $a=1
:perldo $_ = reverse($_);1
:perl VIM::Msg("hello")
:perl $line = $curbuf->Get(42)
```

E299

在沙盘 sandbox 里执行 Perl 命令是受限制的。":perldo" 根本就不能执行。如果可能，":perl" 会在安全环境下被执行。

perl-overview

下面是在 Perl 下可用的函数概要：

```
:perl VIM::Msg("Text")                  # 显示信息
:perl VIM::Msg("Wrong!", "ErrorMsg")    # 显示错误信息
:perl VIM::Msg("remark", "Comment")     # 显示高亮的信息
:perl VIM::SetOption("ai")              # 设定一个 vim 选项
:perl $nbuf = VIM::Buffers()            # 返回缓冲区的个数
:perl @buflist = VIM::Buffers()         # 返回所有缓冲区的数组
:perl $mybuf = (VIM::Buffers('qq.c'))[0] # 返回一个 'qq.c' 的缓冲区对象
:perl @winlist = VIM::Windows()         # 返回所有窗口的数组
:perl $nwin = VIM::Windows()            # 返回窗口的个数
:perl ($success, $v) = VIM::Eval('&path') # $v: 'path' 选项值，$success: 1
:perl ($success, $v) = VIM::Eval('&xyz')  # $v: '' 而 $success: 0
:perl $v = VIM::Eval('expand("<cfile>")') # 展开文件 <cfile>
:perl $curwin->SetHeight(10)            # 设定窗口高度
:perl @pos = $curwin->Cursor()          # 返回坐标 (row, col) 数组
:perl @pos = (10, 10)
:perl $curwin->Cursor(@pos)             # 把光标位置设为 @pos
:perl $curwin->Cursor(10,10)            # 把光标位置设为第 10 行第 10 列
:perl $mybuf = $curwin->Buffer()        # 返回窗口的缓冲区对象
:perl $curbuf->Name()                   # 返回缓冲区名字
:perl $curbuf->Number()                 # 返回缓冲区编号
:perl $curbuf->Count()                  # 返回行数
:perl $l = $curbuf->Get(10)             # 返回第 10 行
:perl @l = $curbuf->Get(1 .. 5)         # 返回第 1 到第 5 行
:perl $curbuf->Delete(10)               # 删除第 10 行
:perl $curbuf->Delete(10, 20)           # 删除第 10－20 行
:perl $curbuf->Append(10, "Line")       # 添加一行
:perl $curbuf->Append(10, "Line1", "Line2", "Line3") # 添加三行
:perl @l = ("L1", "L2", "L3")
:perl $curbuf->Append(10, @l)           # 添加 L1, L2 和 L3
:perl $curbuf->Set(10, "Line")          # 替换第 10 行
:perl $curbuf->Set(10, "Line1", "Line2")      # 替换第 10 行和第 11 行
:perl $curbuf->Set(10, @l)              # 分别用 L1, L2 和 L3 替换 10－12 行
```

perl-Msg

VIM::Msg({msg}, {group}?)
: 显示消息 {msg}。可选 {group} 参数指定用来显示这个消息的高亮方式。

perl-SetOption

VIM::SetOption({arg})
: 设置一个 vim 选项。{arg} 可以是任何 ":set" 命令能接受的参数。这就意味着参数里面不允许有空格！见 :set 。

perl-Buffers

VIM::Buffers([{bn}...])
: 没有参数的时候，在数组上下文里面返回所有缓冲区的列表，在标量上下文里返回缓冲区的个数。对于一个缓冲区名字或者编号的列表 {bn}，返回匹配 {bn} 的缓冲区列表，与 Vim 内部函数 bufname() 使用同样的规则。
警告：使用 :bwipe 后，该列表就不合法了。如果要强行使用会使 Vim 崩溃。

perl-Windows

VIM::Windows([{wn}...])
: 没有参数的时候，在数组上下文里面返回所有窗口的列表，在

标量上下文里返回窗口的个数。对于一个窗口编号的列表 {wn}，返回匹配 {wn} 的窗口列表。
警告：窗口关闭后，该列表就不合法了。如果要强行使用会使 Vim 崩溃。

perl-DoCommand

VIM::DoCommand({cmd}) 执行 Ex 命令 {cmd}。

perl-Eval

VIM::Eval({expr}) 对 {expr} 求值并在列表上下文返回 (success, value)，在标量上下文则只返回 value。
success=1 表示 val 包含 {expr} 的值。success=0 表示表达式求值失败。 '@x' 返回寄存器 x 的内容， '&x' 返回选项 x 的值，'x' 返回内部变量 x 的值，'$x' 等价于 perl 中的 $ENV{x}。所有在命令行可以访问的函数都可用于 {expr}。
列表 List 通过连接项目并在项目间插入换行符来转化为字符串。

perl-Blob

VIM::Blob({expr}) 返回来自标量值的 Blob 字面常量字符串 0zXXXX。

perl-SetHeight

Window->SetHeight({height})
把窗口的高度设定为 {height}，不超过屏幕范围。

perl-GetCursor

Window->Cursor({row}?, {col}?)
在没有参数的情况下，返回窗口光标所处位置的坐标 (row, col) 数组。当有参数 {row} 和 {col} 的情况下，把窗口光标的位置设为 {row} 和 {col}。注意 {col} 是按 Perl 风格从0 数起的，比 Vim 的尺度小 1。

Window->Buffer() perl-Buffer
返回给定窗口的缓冲区对象。

perl-Name

Buffer->Name() 返回缓冲区的文件名。

perl-Number

Buffer->Number() 返回缓冲区的编号。

perl-Count

Buffer->Count() 返回缓冲区中的行数。

perl-Get

Buffer->Get({lnum}, {lnum}?, ...)
对每个指定的 {lnum}，返回缓冲区中第 {lnum} 行的文本字符串。通过指定一个 {lnum} 列表可以传递一个数组。

perl-Delete

Buffer->Delete({lnum}, {lnum}?)
删除缓冲区的第 {lnum} 行。当有第二个参数 {lnum} 时，删除由第一个 {lnum} 到第二个 {lnum} 指定的范围。

perl-Append

Buffer->Append({lnum}, {line}, {line}?, ...)
把每个 {line} 字符串添加到缓冲区的第 {lnum} 行后面。{line} 列表可以是一个数组。

perl-Set

Buffer->Set({lnum}, {line}, {line}?, ...)
替代缓冲区从第 {lnum} 行开始，由 {line} 指定的一行或者多行。{line} 列表可以是一个数组。如果参数无效，就不作替代。

$main::curwin
当前窗口对象。

$main::curbuf
当前缓冲区对象。

script-here

当使用嵌入式脚本语言时，你可能想在语言不支持时跳过这些语言的代码。但是以下机制不能工作:

```
if has('perl')
  perl << EOF
    this will NOT work!
EOF
endif
```

相反，应把 Perl / Python / Ruby 等的命令放在一个函数里面并调用它:

```
if has('perl')
  function DefPerl()
    perl << EOF
      this works
EOF
  endfunction
  call DefPerl()
endif
```

备注: "EOF" 必须在行首（前面不能有前导空格）。

---

4. 动态调入 perl-dynamic

MS-Windows 和 Unix 上，可以动态调入 Perl 库。 :version 输出这时应包括 +perl/dyn 。

这意味着 Vim 只有在必要时才寻找 Perl DLL 或共享库文件。如果不使用 Perl 接口，你就不需要它。这样，即使没有该文件，你也可使用 Vim。

MS-Windows

你可以从 http://www.perl.org 下载 Perl。用于编译 Vim 的是 ActiveState 提供的版本。

要使用 Perl 接口，Perl DLL 必须在搜索路径上。
如果 Vim 报告找不到 perl512.dll，确保 $PATH 包含该文件所在的路径。Perl 安装程序通常会做好这一点。
控制台窗口里输入 "path" 可以看到（搜索路径）当前使用的目录。也可以用 'perldll'

选项指定 Perl DLL。

DLL 名必须匹配 Vim 编译时所使用的 Perl 版本。目前，该名字为 "perl512.dll"，也就是 Perl 5.12。要确信这一点，编辑 "gvim.exe" 文件并查找 "perl\d*.dll\c"。

Unix

'perldll' 选项可用来指定 Perl 共享库文件，而不用编译时指定的 DYNAMIC_PERL_DLL 文件。共享库的版本必须和 Vim 编译使用的 Perl 版本保持一致。

---

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Paul Moore

译者：lang2，Willis http://vimcdoc.sf.net

Vim 的 Python 编程接口 python Python

1. 命令 python-commands
2. vim 模块 python-vim
3. 缓冲区对象 python-buffer
4. 范围对象 python-range
5. 窗口对象 python-window
6. 标签页对象 python-tabpage
7. vim.bindeval 对象 python-bindeval-objects
8. pyeval()、py3eval() Vim 函数 python-pyeval
9. 动态调入 python-dynamic
10. Python 3 python3
11. Python X python_x
12. 编译时带 Python 支持 python-building

Vim 的 Python 2.x 接口仅当 +python 特性被编译进 Vim 时才有效。
Vim 的 Python 3 接口仅当 +python3 特性被编译进 Vim 时才有效。
可以同时使用两者，但请阅读 python-2-and-3 。

---

1. 命令 python-commands

:python :py E263 E264 E887

:[range]py[thon] {stmt}
执行 Python 语句 {stmt}。一个判断 ":python" 命令是否可用的简单检查：

```
:python print "Hello"
```

:[range]py[thon] << [endmarker]
{script}
{endmarker}
执行 Python 脚本 {script}。
备注：此命令在没有编译进 Python 特性时无效。为了避免这样的错误，参阅 script-here 。

{script} 之下的 {endmarker} 前面 不能 有任何空白字符。

如果在 "<<" 之后省略了 [endmarker]，就像 :append 及 :insert 命令那样，在 {script} 之后一定要有一个点 '.'。

这种形式的 :python 命令主要用于在 Vim 脚本中嵌入 Python 代码。

例子：

```
function! IcecreamInitialize()
python << EOF
class StrawberryIcecream:
```

```
        def __call__(self):
                print 'EAT ME'
EOF
endfunction
```

要看运行的 Python 版本：

```
:python print(sys.version)
```

没必要 import sys，缺省就会。

备注：Python 对于缩进是非常敏感的。要确保 "class" 所在行及 "EOF" 没有任何缩进。

:pydo

:[range]pydo {body}　　对 [range] 里的每一行执行 Python 函数 "def _vim_pydo(line, linenr): {body}"，其中的函数参数设为每行不带尾部 <EOL> 的文本，以及行号。该函数应该返回字符串或者 None。如果返回字符串，它会用来替代当前执行的行。缺省 [range] 为整个文件："1,$"。

示例：

```
:pydo return "%s\t%d" % (line[::-1], len(line))
:pydo if line: return "%4d: %s" % (linenr, line)
```

可以和 :py 配合，用 :pydo 来利用 python 过滤行范围。例如：

```
:py3 << EOF
needle = vim.eval('@a')
replacement = vim.eval('@b')

def py_vim_string_replace(str):
        return str.replace(needle, replacement)
EOF
:'<,'>py3do return py_vim_string_replace(line)
```

:pyfile :pyf

:[range]pyf[ile] {file}
执行 {file} 文件中包含的 Python 脚本。整个参数被用作一个文件名。

这些命令根本上都差不多 — 它们都将当前范围 python-range 设定为指定的行范围，并对其执行 Python 代码。

:python 的情况所执行的代码来自命令行。
:pyfile 的情况所执行的代码来自一个指定的文件。

Python 命令不能在 sandbox 里使用。

需要传递参数的话，你得使用 sys.argv[]。例如：

```
:python sys.argv = ["foo", "bar"]
:pyfile myscript.py
```

下面是一些例子 python-examples

```
:python from vim import *
:python from string import upper
:python current.line = upper(current.line)
:python print "Hello"
:python str = current.buffer[42]
```

(注意 如同导入 (import) 模块一样，变动对后续命令持续有效。这和 Python 的解释程序是一样的。)

---

2. vim 模块 python-vim

Python 的代码所有对 Vim 的操作 (只有一个例外 — 看下面的 python-output) 都是通过 "vim" 模块来进行的。该模块包括两个方法，三个常量，以及一个异常对象。在使用它们之前你得先导入 (import) vim 模块。

总览

```
:py print "Hello"                # 显示信息
:py vim.command(cmd)             # 执行 Ex 命令
:py w = vim.windows[n]           # 获取窗口 "n"
:py cw = vim.current.window      # 获取当前窗口
:py b = vim.buffers[n]           # 获取缓冲区 "n"
:py cb = vim.current.buffer      # 获取当前缓冲区
:py w.height = lines             # 设定窗口高度
:py w.cursor = (row, col)        # 设定光标位置
:py pos = w.cursor               # 获取 tuple (行, 列)
:py name = b.name                # 获取缓冲区的文件名
:py line = b[n]                  # 获取缓冲区内的一行
:py lines = b[n:m]               # 获取数行
:py num = len(b)                 # 获取统计的行数
:py b[n] = str                   # 在缓冲区里写入一行文本
:py b[n:m] = [str1, str2, str3] # 一次性写入多行文本
:py del b[n]                     # 删除一行
:py del b[n:m]                   # 删除数行
```

"vim" 模块中的方法

vim.command(str) python-command

执行 vim (ex-模式) 命令 str。无返回值。
例如:

```
:py vim.command("set tw=72")
:py vim.command("%s/aaa/bbb/g")
```

下面的定义可以执行普通模式命令:

```
def normal(str):
        vim.command("normal "+str)
# 注意使用单引号标识出含有双引号的字符串
normal('"a2dd"aP')
```

E659

":python" 在 Python 2.2 及之前版本上不能嵌套使用。下面命令只对 Python 2.3 之后版本适用:

```
:py vim.command("python print 'Hello again Python'")
```

vim.eval(str) python-eval

使用 vim 内部的表达式处理器来对表达式 str 求值。(参阅 expression)。
返回表达式的结果:
-如果 Vim 表达式计算结果是字符串或者数值，那么返回一个字符串。
-如果 Vim 表达式计算结果是 Vim 列表，那么返回一个列表
-如果 Vim 表达式计算结果是 Vim 字典，那么返回一个字典
字典和列表被递归扩展。
例:

```
:" 'textwidth' 选项值
:py text_width = vim.eval("&tw")
:
:" 'a' 寄存器的内容
:py a_reg = vim.eval("@a")
:
:" 结果是个字符串！使用 string.atoi() 把它转换成数值。
:py str = vim.eval("12+12")
:
:py tagList = vim.eval('taglist("eval_expr")')
```

后一个将返回一个 python 类型的字典列表，例如:
[{'cmd': '/^eval_expr(arg, nextcmd)$/', 'static': 0, 'name':
'eval_expr', 'kind': 'f', 'filename': './src/eval.c'}]

vim.bindeval(str) python-bindeval

类似于 python-eval ，但返回 python-bindeval-objects 中描述的特殊对象。这些 python 对象可用于修改 (List 或 Dictionary) 或调用 (Funcref) vim 对象。

vim.strwidth(str) python-strwidth

类似于 strwidth() : 返回 str 占据的显示单元数，制表符算作一个单元。

vim.foreach_rtp(callable) python-foreach_rtp

对 'runtimepath' 的每个路径调用给定的 callable，直到某次 callable 返回非 None 值、抛出例外、或者没有更多的路径为止。如果因为 callable 返回非 None 而停止，vim.foreach_rtp 函数返回 callable 返回的值。

vim.chdir(*args, **kwargs) python-chdir
vim.fchdir(*args, **kwargs) python-fchdir

运行 os.chdir 或 os.fchdir，然后做合适的 vim 的事情。
备注: 你不应直接使用这些函数，用 os.chdir 或 os.fchdir 代替。如果 os.fchdir 不存在，则 vim.fchdir 的行为未定义。

"vim" 模块中的异常对象

vim.error python-error

当遇到 Vim 的错误时，Python 引起一个 vim.error 类的异常。
例:

```
try:
        vim.command("put a")
except vim.error:
        # nothing in register a
```

"vim" 模块中的常量

注意 这些其实并非真正的常量 — 你还是可以对它们赋值。但这毫无意义，因为你会丢失该 vim 对象本来代表的值。

vim.buffers python-buffers

一个映射对象，用来提供对 vim 缓冲区的操作。该对象支持以下操作：

```
:py b = vim.buffers[i]      # 索引（只读）
:py b in vim.buffers        # 成员测试
:py n = len(vim.buffers)    # 成员个数
:py for b in vim.buffers:   # 遍历缓冲区列表
```

vim.windows python-windows

一个序列对象，用来提供对 vim 窗口的操作。该对象支持以下操作：

```
:py w = vim.windows[i]      # 索引（只读）
:py w in vim.windows        # 成员测试
:py n = len(vim.windows)    # 成员个数
:py for w in vim.windows:   # 顺序存取
```

注意：vim.windows 对象总是访问当前标签页。 python-tabpage .windows 对象和父 python-tabpage 对象绑定，总是使用该标签页的窗口（如果该标签页该删除，抛出 vim.error）。你可以得到两者的引用而无需保存 vim 模块对象或 python-tabpage 的引用。此时它们不会失去自己的属性。

vim.tabpages python-tabpages

一个序列对象，用来提供 vim 标签页的操作。该对象支持以下操作：

```
:py t = vim.tabpages[i]     # 索引（只读）
:py t in vim.tabpages       # 成员测试
:py n = len(vim.tabpages)   # 成员个数
:py for t in vim.tabpages:  # 顺序存取
```

vim.current python-current

一个用来提供对各种各样当前 "current" 对象进行操作的对象。它包括一些特定的属性：

vim.current.line	当前行（读写）	字符串
vim.current.buffer	当前缓冲区（读写）	缓冲区
vim.current.window	当前窗口（读写）	窗口
vim.current.tabpage	当前标签页（读写）	标签页
vim.current.range	当前行范围（只读）	范围

最后一种情况需要一些额外的解释。当用 :python 或 :pyfile 命令指定一个范围之后，该范围将成为 "当前范围"。一个范围就如同一个缓冲区，只不过所操作的对象界限于所有行的一个子集。请参阅 python-range 。

注意：赋值给 vim.current.{buffer,window,tagpage} 时，分别期待合法的 python-buffer 、 python-window 或 python-tabpage 对象。赋值触发到给定缓冲区、窗口、标签页的正常切换（带有 autocommand 自动命令）。这是在 python 里切换 UI 对象的唯一方法：不能给 python-tabpage .window 属性赋值。要想进行切换但不执行自动命令，可用

```
py << EOF
saved_eventignore = vim.options['eventignore']
vim.options['eventignore'] = 'all'
try:
    vim.current.buffer = vim.buffers[2] # 切换到缓冲区 2
finally:
    vim.options['eventignore'] = saved_eventignore
```

EOF

vim.vars python-vars
vim.vvars python-vvars
类似字典的对象，分别对应全局（g:）和 vim（v:）变量。等价于 vim.bindeval("g:")，但更快。

vim.options python-options
部分支持映射协议（即读写子项目）的对象，用于对全局选项进行读写访问。
注意：和 :set 不同，此对象只能访问全局选项，不能用于读写局部变量的值或以任何方式去访问只限局部的变量。如果没有指定名字的全局变量，抛出 KeyError（也即，对 global-local 或只限全局的选项，不会抛出此例外，但对局部于窗口或局部于缓冲区的则会）。python-buffer 对象可用于访问局部于缓冲区的选项，python-window 选项则可用于访问局部于窗口的选项。

可通过 vim 模块的 "Options" 属性取得此对象的类型。

Python 的输出 python-output
Vim 将所有 Python 代码的输出都显示在信息区。普通的输出会以一般信息出现，错误会以出错信息出现。

用具体实现的术语来讲，这表示所有 sys.stdout（包括 print 语句的输出）以一般信息形式出现，而所有 sys.stderr（包括 error tracebacks）都会被显示成出错信息。

python-input
Vim 并不支持用 Python 来输入（通过 sys.stdin，包括 input() 和 raw_input()）。这些调用可能会导致崩溃。这个问题可能以后会修正。

python2-directory python3-directory pythonx-directory
Python 'runtimepath' 处理 python-special-path

在 python 中，vim.VIM_SPECIAL_PATH 特殊目录用于代表 'runtimepath' 的路径列表：如果 sys.path 包含此目录而且 sys.path_hooks 包含 vim.path_hooks，python 会试图载入 'runtimepath' 的每个 {rtp} 对应的 {rtp}/python2（或 python3）和 {rtp}/pythonx（同时适用于两个 python 版本）模块。

实现大致如下，但以 C 编写：

```
from imp import find_module, load_module
import vim
import sys

class VimModuleLoader(object):
    def __init__(self, module):
        self.module = module

    def load_module(self, fullname, path=None):
        return self.module

def _find_module(fullname, oldtail, path):
    idx = oldtail.find('.')
    if idx > 0:
        name = oldtail[:idx]
```

```
            tail = oldtail[idx+1:]
            fmr = find_module(name, path)
            module = load_module(fullname[:-len(oldtail)] + name, *fmr)
            return _find_module(fullname, tail, module.__path__)
        else:
            fmr = find_module(fullname, path)
            return load_module(fullname, *fmr)

    # It uses vim module itself in place of VimPathFinder class: it does not
    # matter for python which object has find_module function attached to as
    # an attribute.
    class VimPathFinder(object):
        @classmethod
        def find_module(cls, fullname, path=None):
            try:
                return VimModuleLoader(_find_module(fullname, fullname, path or vim._get_paths()))
            except ImportError:
                return None

        @classmethod
        def load_module(cls, fullname, path=None):
            return _find_module(fullname, fullname, path or vim._get_paths())

    def hook(path):
        if path == vim.VIM_SPECIAL_PATH:
            return VimPathFinder
        else:
            raise ImportError

    sys.path_hooks.append(hook)
```

vim.VIM_SPECIAL_PATH python-VIM_SPECIAL_PATH

字符串常数，和 vim 路径钩联合使用。如果 vim 安装的路径钩被要求处理任何非 vim.VIM_SPECIAL_PATH 常数的路径，抛出 ImportError。在唯一可能的另一种情况下，使用特殊载入程序。

注意：不能直接使用该常数的值，必须使用 vim.VIM_SPECIAL_PATH 对象。

vim.find_module(...) python-find_module

vim.path_hook(path) python-path_hook

用于实现上述路径载入的方法或对象。除非你想对 sys.meta_path 进行处理而可以看看 vim.path_hook，请不要直接使用它们。将来的 vim 版本不一定保证任何这里描述的对象会继续存在。

vim._get_paths python-_get_paths

用于产生给路径钩搜索的路径列表的方法。可以用于调试，但不要假定将来的 vim 版本中此方法会继续存在。

返回 'runtimepath' 中每个 {rtp} 对应的 {rtp}/python2 (或 {rtp}/python3) 和 {rtp}/pythonx 的目录列表。

---

3. 缓冲区对象 python-buffer

缓冲区对象代表 vim 缓冲区。你可以用以下这几种办法来获取缓冲区的列表:

- 通过 vim.current.buffer (python-current)
- 通过 vim.buffers 的索引访问 (python-buffers)
- 通过一个窗口的 "buffer" 属性 (python-window)

缓冲区对象有两个只读属性 - name - 缓冲区的文件全名，还有 number - 缓冲区号。该对象还包括三个方法 (append, mark 及 range; 见下)。

你也可以将缓冲区对象视为序列对象。这样的话，它们就可以被看作字符串的列表进行存取 (它们是可变的)。每一个元素是缓冲区中的一行。所有通常的序列存取操作，包括索引，索引赋值，切片 (slice) 及切片赋值，等等，都可以被使用。注意 索引 (切片) 操作的结果是一个字符串 (字符串列表)。这产生了一个意想不到的结果 - b[:] 和 b 是不同的。确切一些，"b[:] = None" 会清空整个缓冲区，而 "b = None" 仅仅更新变量 b 的值，完全不会影响到缓冲区。

缓冲区索引从 0 开始算起，这与通常的 Python 语法一致。但这和 Vim 的行号从 1 算起有分歧。这一点在处理标记 (见下) 是要特别留意，因为标记是以行号区分的。

缓冲区对象的属性有:

b.vars	类似字典的对象，可用于访问 buffer-variable 。
b.options	映射对象 (支持子项目的读写和删除)，用于访问局部于缓冲区的选项和 global-local 选项的缓冲区局部值。如果选项局部于窗口，请使用 python-window .options，使用此对象会抛出 KeyError。如果选项是 global-local 而没有局部的部分，返回 None。
b.name	字符串，可读写。包含缓冲区名 (完整路径)。 备注: 给 b.name 赋值时激活 BufFilePre 和 BufFilePost 自动事件。
b.number	缓冲区编号。可用作 python-buffers 的键。只读。
b.valid	True 或 False。如果对应的缓冲区被删除 (wipe)，缓冲区对象就成为非法。

缓冲区对象的方法有:

b.append(str)	对缓冲区附加一行
b.append(str, nr)	同上，在第 "nr" 行之后
b.append(list)	对缓冲区附加一系列行 备注: append 方法可以带一个字符串列表作为参数，这和 Python 中内建的列表对象的对应方法是不同的。
b.append(list, nr)	同上，在第 "nr" 行之后
b.mark(name)	返回一个 tuple (行，列) 用来代表该位置上的一个命名标记 (也可以用于 []"<> 等标记)
b.range(s,e)	返回一个范围对象 (参见 python-range) 用来代表指定缓冲区中行 s 与 行 e (包含 s 和 e inclusive) 之间的部分。

注意: 当增加一行的时候，这一行里一定不要含有换行符 '\n'。行尾的 '\n' 可以，但会被忽略，所以下面的操作是可以的:

```
:py b.append(f.readlines())
```

可通过 vim 模块的 "Buffer" 属性取得缓冲区对象的类型。

例如 (假定 b 是当前缓冲区)

```
:py print b.name                # 输出缓冲区的名字
```

```
:py b[0] = "hello!!!"           # 替换最顶上的一行
:py b[:] = None                 # 删除整个缓冲区
:py del b[:]                    # 删除整个缓冲区
:py b[0:0] = [ "a line" ]       # 在第一行前添加一行
:py del b[2]                    # 删除一行 (第三行)
:py b.append("bottom")          # 在缓冲区结尾添加一行
:py n = len(b)                  # 总行数
:py (row,col) = b.mark('a')     # 命名标记
:py r = b.range(1,5)            # 缓冲区内的一个范围
:py b.vars["foo"] = "bar"       # 给 b:foo 变量赋值
:py b.options["ff"] = "dos"     # 设置 fileformat
:py del b.options["ar"]         # 等同于 :set autoread<
```

---

4. 范围对象 python-range

范围对象代表一个 vim 缓冲区内的一个部分。你可以用以下的方法之一来获取一个缓冲区对象:

- 通过 vim.current.range (python-current)
- 通过一个缓冲区的 range() 方法 (python-buffer)

一个范围对象在操作上几乎和一个缓冲区对象完全一样。不过，其操作的目标仅显于范围指定的行 (当然，这个行范围会随着切片赋值，行删除，或者 range.append() 等等操作而改变)。

范围对象的属性有:

```
r.start         首行在缓冲区内的索引
r.end           尾行在缓冲区内的索引
```

范围对象的方法有:

```
r.append(str)   给范围附加一行
r.append(str, nr)  同上，在第 "nr" 行之后
r.append(list)  给范围附加一系列行
                备注: append 方法可以带一个字符串列表作为参数，这和
                Python 中内建的列表对象的对应方法是不同的。
r.append(list, nr)  同上，在第 "nr" 行之后
```

可通过 vim 模块的 "Range" 属性取得范围对象的类型。

例如 (假设 r 是当前范围):

```
# 发送范围内的所有行给缺省打印机
vim.command("%d,%dhardcopy!" % (r.start+1,r.end+1))
```

---

5. 窗口对象 python-window

窗口对象代表一个 vim 窗口。你可以用以下几种方法来获取一个窗口对象:

- 通过 vim.current.window (python-current)
- 通过对 vim.windows 的索引操作 (python-windows)
- 通过对标签页的 "windows" 属性的索引操作 (python-tabpage)
- 通过标签页的 "window" 属性 (python-tabpage)

你只能通过窗口对象的属性来控制它。这些对象没有方法，也没有序列等其它接口。

窗口的属性包括:

buffer (只读)	窗口中显示的缓冲区
cursor (读写)	窗口中的当前光标位置 这是一个 tuple (行, 列)。
height (读写)	行数表示的窗口高度
width (读写)	列数表示的窗口宽度
vars (只读)	窗口 w: 变量。不能给本属性赋值, 但可以用此属性修改窗口变量
options (只读)	局部于窗口的选项。不能给本属性赋值, 但可以用此属性修改窗口选项。只能用于访问局部于窗口的选项, 对局部于缓冲区的选项要用 python-buffer , 对全局选项用 python-options 。如果选项是 global-local 的而没有局部值, 返回 None。
number (只读)	窗口编号。首个窗口的编号为 1。如果不能决定才会返回 0 (例如当窗口对象属于其他标签页时)。
row、col (只读)	窗口在屏幕上的位置, 以显示单元计。首个位置为零。
tabpage (只读)	窗口的标签页。
valid (读写)	True 或 False。当对应窗口关闭时窗口对象成为非法。

height 属性只有当屏幕被水平分割时才可写。
width 属性只有当屏幕被垂直分割时才可写。

可通过 vim 模块的 "Window" 属性取得窗口对象的类型。

---

6. 标签页对象 python-tabpage

标签页对象代表一个 vim 标签页。你可以用以下几种方法来获取之:
- 通过 vim.current.tabpage (python-current)
- 通过对 vim.tabpages 的索引操作 (python-tabpages)

你可以通过该对象来访问标签页里的窗口。这些对象没有方法, 也没有序列等其它接口。

标签页的属性包括:

number	标签页编号, 和 tabpagenr() 相同。
windows	类似于 python-windows , 但对应当前标签页
vars	标签页 t: 变量。
valid	True 或 False。当对应标签页关闭时标签页对象成为非法。

可通过 vim 模块的 "TabPage" 属性取得标签页对象的类型。

---

7. vim.bindeval 对象 python-bindeval-objects

vim.Dictionary 对象 python-Dictionary
类字典对象, 用于访问 vim Dictionary 类型。
属性:

属性	描述	
locked	以下值之一	python-.locked

值	描述

zero	变量没有上锁
vim.VAR_LOCKED	变量已上锁，但可以解锁
vim.VAR_FIXED	变量已上锁，且不可以解锁

读写。可以通过给本属性赋值 True 或 False 来给变量解锁。不支持递归上锁。

scope 以下值之一

值	描述
zero	字典无作用域
vim.VAR_DEF_SCOPE	g: 或 l: 字典
vim.VAR_SCOPE	其他作用域的字典，见 internal-variables

方法（备注：方法不支持关键字参数）：

方法	描述
keys()	返回字典所有键的列表。
values()	返回字典所有值的列表。
items()	返回字典所有内容的二元组的列表。

update(iterable)、update(dictionary)、update(**kwargs)
把新的键值加入字典。

get(key[, default=None])
从字典读入 key 对应值，如果不存在返回 default。

pop(key[, default])
从字典中删除给定 key 并返回对应值。如果找不到 key 且给出 default 返回 default，不然抛出 KeyErrror。

popitem()
从字典中删除随机键，并返回（键，值）对。

has_key(key)
检查字典中是否包含给定的键 key，类似于 'key in dict'。

__new__(), __new__(iterable), __new__(dictionary), __new__(update)
用 vim.Dictionary() 可以建立新的 vim 字典。d=vim.Dictionary(arg) 等价于 d=vim.bindeval('{}');d.update(arg) 。无参数的版本构建空字典。

示例：

```
d = vim.Dictionary(food="bar")          # 构造函数
d['a'] = 'b'                            # 子项目赋值
print d['a']                            # 读取子项目
d.update({'c': 'd'})                    # .update(dictionary)
d.update(e='f')                         # .update(**kwargs)
d.update((('g', 'h'), ('i', 'j')))      # .update(iterable)
for key in d.keys():                    # .keys()
for val in d.values():                  # .values()
for key, val in d.items():              # .items()
print isinstance(d, vim.Dictionary)     # True
for key in d:                           # 遍历所有键
class Dict(vim.Dictionary):             # 子类
```

注意：遍历键时不应对字典进行修改。

vim.List 对象 python-List

类序列对象，用于访问 vim List 类型。

支持 .locked 属性，见 python-.locked 。还支持以下方法：

方法	描述

extend(item)　给列表加入项目 item。

__new__(), __new__(iterable)
用 vim.List() 可以建立新的 vim 列表。l=vim.List(iterable) 等价于 l=vim.bindeval('[]');l.extend(iterable) 。无参数的版本方法构建空列表。

示例:

```
l = vim.List("abc")             # 构造函数，返回: ['a', 'b', 'c']
l.extend(['abc', 'def'])        # .extend() 方法
print l[1:]                     # 分片 (slicing)
l[:0] = ['ghi', 'jkl']          # 分片赋值
print l[0]                      # 读取项目
l[0] = 'mno'                    # 赋值
for i in l:                     # 遍历
print isinstance(l, vim.List)   # True
class List(vim.List):           # 子类
```

vim.Function 对象　python-Function

类函数对象，相当于 vim Funcref 对象。接受特殊关键字参数 self ，见 Dictionary-function 。也可用 vim.Function(name) 构造函数，等价于 vim.bindeval('function(%s)'%json.dumps(name)) 。

属性 (只读):

属性	描述
name	函数名。
args	None 或用作参数的 python-List 对象。注意 这是参数列表的备份，每次请求参属性时都会重新构建。列表的修改被忽略 (但不适用于参数列表里的容器: 类似于 copy() 而不是 deepcopy())。
self	None 或用作 self 字典的 python-Dictionary 对象。注意 调用函数对象时显式的 self 参数覆盖此属性。
auto_rebind	布尔值。如果为真，此 Python 对象建立和在 Vim 脚本里存贮的字典里的的偏函数会在此字典被访问时自动重新绑定此字典。这体现了 Vim 内部 dict.func (auto_rebind=True) 和 function(dict.func,dict) (auto_rebind=False) 的差异。如果 self 属性为 None ，此属性无意义。

构造函数额外接受 args 、 self 和 auto_rebind 关键字。如果给出 args 和/或 self 参数，构造偏函数，见 function() 。只有给定 self 时才使用 auto_rebind ，否则无论是否给出，总是假定为 True 。如果给出 self ，缺省为 False 。

示例:

```
f = vim.Function('tr')                  # 构造函数
print f('abc', 'a', 'b')                 # 调用 tr('abc', 'a', 'b')
vim.command('''
    function DictFun() dict
        return self
    endfunction
''')
f = vim.bindeval('function("DictFun")')
print f(self={})                        # 相当于 call('DictFun', [], {})
```

```
print isinstance(f, vim.Function)       # True

p = vim.Function('DictFun', self={})
print f()
p = vim.Function('tr', args=['abc', 'a'])
print f('b')
```

---

8. pyeval() 和 py3eval() Vim 函数 python-pyeval

为了访问双向接口的方便，可用 pyeval() 和 py3eval() 函数来计算 Python 表达式然后返回计算结果给 Vim 脚本。也可用 pyxeval() 。

Python "None" 值转化为 v:none。

---

9. 动态调入 python-dynamic

MS-Windows 和 Unix 上，可以动态调入 Python 库。 :version 输出这时应包括 +python/dyn 或 +python3/dyn 。

这意味着 Vim 只有在必要时才寻找 Python DLL 文件或共享库。如果不使用 Python 接口，你就不需要它。这样，即使没有该文件，你也可使用 Vim。

MS-Windows

要使用 Python 接口，Python DLL 必须在搜索路径上。控制台窗口里输入 "path" 可以看到（搜索路径）当前使用的目录。也可用 'pythondll' 或 'pythonthreedll' 选项来指定 Python DLL。

DLL 的名字应该匹配 Vim 编译时所使用的 Python 版本。目前，用于 Python 2 的名字为 "python27.dll"，也就是 Python 2.7。用于 Python 3 的是 python36.dll (Python 3.6)。要确信这一点，编辑 "gvim.exe" 文件并查找 "python\d*.dll\c"。

Unix

'pythondll' 或 'pythonthreedll' 选项可用来指定 Python 共享库文件，而不用编译时指定的 DYNAMIC_PYTHON_DLL 或 DYNAMIC_PYTHON3_DLL 文件。共享库的版本必须和 Vim 编译使用的 Python 2.x 或 Python 3 版本保持一致。

---

10. Python 3 python3

:py3 :python3

```
:[range]py3 {stmt}
:[range]py3 << [endmarker]
{script}
{endmarker}

:[range]python3 {stmt}
:[range]python3 << [endmarker]
```

{script}
{endmarker}
:py3 和 :python3 命令和 :python 类似。判断 :py3 命令是否可用的一个简单检查:

```
:py3 print("Hello")
```

要看运行的 Python 版本:

```
:py3 import sys
:py3 print(sys.version)
```

:py3file

:[range]py3f[ile] {file}
:py3file 命令和 :pyfile 类似。

:py3do

:[range]py3do {body}
:py3do 命令和 :pydo 类似。

Vim 可以用四种方式编译 (:version 输出结果):
1. 无 Python 支持 (-python、-python3)
2. 只有 Python 2 支持 (+python 或 +python/dyn、-python3)
3. 只有 Python 3 支持 (-python、+python3 或 +python3/dyn)
4. Python 2 和 3 支持 (+python/dyn、+python3/dyn)

关于第四种特殊情况的更多细节: python-2-and-3

要同时支持 Python 2 和 Python 3，两者必须都是动态载入。

在 Linux/Unix 系统上用这种方式并导入全局符号的时候，使用第二个 Python 版本会导致系统崩溃。所以，要么载入全局符号但只激活一个 Python 版本，要么考虑不载入全局符号。后者使 Python 导入 ("import") 某些期待 Vim 提供某些符号的库的时候会失败。

E836 E837

Vim 的配置脚本根据一个特定的 Python 标准库 (termios) 对所有的库进行猜测。如果对两个 Python 版本导入该库均成功，那么可以在 Vim 中同时使用两者。否则在一个会话中，只允许使用先用到的版本。使用第二个版本会得到 E836 或 E837 的错误信息。

这里 Vim 的行为取决于配置所在的系统。如果两个 Python 版本都用了 --enable-shared 进行配置，两者会同时激活。但没有链接进 libPython 的第三方库仍然会有问题。

要回避这些问题，有以下几个方案:
1. 重新编译有问题的库，把它链接进相应的 libpython.so 里。
2. 重新编译 Vim，只用一个 Python 版本。
3. 配置完后，撤销 auto/config.h 中 PY_NO_RTLD_GLOBAL 的定义。这可能会使 Vim 崩溃。

E880

从 python 抛出 SystemExit 例外不是退出 vim 的好方法，要用:

```
:py vim.command("qall!")
```

has-python

以下方法可用来测试哪个 Python 版本可用:

```
if has('python')
```

```
  echo '有 Python 2.x'
endif
if has('python3')
  echo '有 Python 3.x'
endif
```

注意 不过，当 Python 2 和 3 同时存在且都是动态载入，这些 has() 调用会试图载入它们。如果两个版本不能同时载入，单单对 Python 2 或 3 是否可用的检查就会导致另一个版本不能调入。

要避免载入动态库，只需要检查 Vim 编译时是否有 python 支持：

```
if has('python_compiled')
  echo '编译时带 Python 2.x 支持'
  if has('python_dynamic')
    echo 'Python 2.x 已动态载入'
  endif
endif
if has('python3_compiled')
  echo '编译时带 Python 3.x 支持'
  if has('python3_dynamic')
    echo 'Python 3.x 已动态载入'
  endif
endif
```

这段代码也会告诉你 Python 是否已动态载入，如果找不到运行库时会报错。

---

11. Python X python_x pythonx

大多数 python 代码可以用 python 2.6+ 和 python 3 兼容的方式编写，因而我们提供了 pyx+ 函数和命令。其工作方式和 Python 2 和 3 变种完全相同，但根据 'pyxversion' 设置选择 Python 版本。

应在 .vimrc 中设置 'pyxversion'，可为 Python 命令选择 Python 2 或 Python 3。如果在运行时改变此设置，可能会冒丢失插件状态（如初始化）的风险。

要用的模块可放在 {rtp}/pythonx 目录里。见 pythonx-directory 。

:pyx :pythonx

:pyx 和 :pythonx 命令的工作方式类似于 :python 。要看 :pyx 命令是否可用的简单检查：

```
:pyx print("Hello")
```

要看运行的 Python 版本：

```
:pyx import sys
:pyx print(sys.version)
```

:pyxfile python_x-special-comments

:pyxfile 命令的工作方式类似于 :pyfile 。不过你可以加上以下注释之一强制 Vim 使用 :pyfile 或 :py3file ：

```
#!/任何字符串/python2          " Shebang。必须是文件的首行。
#!/任何字符串/python3          " Shebang。必须是文件的首行。
# requires python 2.x          " 最大行数取决于 'modelines'。
# requires python 3.x          " 最大行数取决于 'modelines'。
```

和普通模式行不同，不检查文件尾部。如果找不到这样注释，使用 'pyxversion' 设置。
W20 W21
如果 Vim 不支持所选的 Python 版本，安静地打印一行消息。用 :messages 可读该消息。

:pyxdo
:pyxdo 命令的工作方式类似于 :pydo 。

has-pythonx
可以这样测试 pyx* 系列命令是否可用：

```
if has('pythonx')
  echo 'pyx* 命令可用. (Python ' . &pyx . ')'
endif
```

如果编译时只有 +python 或 +python3 之一，has() 返回 1。
如果编译时 +python 和 +python3 两者兼有，测试结果取决于 'pyxversion' 设置。如果 'pyxversion' 为 0，先测试 Python 3，如果不可用再测试 Python 2。如果 'pyxversion' 为 2 或 3，只测试对应的 Python 2 或 3。

注意 为了使 'has('pythonx')' 能工作，可能会试图动态载入 Python 3 或 2。这可能有副作用，尤其是 Vim 只能载入两者之一的时候。

如果用户希望首选 Python 2 而使用 Python 3 作后备，需要在 .vimrc 里显式设置 'pyxversion'。例如：

```
if has('python')
  set pyx=2
elseif has('python3')
  set pyx=3
endif
```

---

12. 编译时带 Python 支持 python-building

编译时带 Python 2 或 3 支持的若干提示。

UNIX

关于如何打开 Python 接口的支持，见 src/Makefile。

Ubuntu 上需要为 Python 2 安装以下包：

```
        python
        python-dev
Python 3:
        python3
        python3-dev
Python 3.6:
        python3.6
        python3.6-dev
```

如果有多于一个 Python 3 版本，在运行 configure 前链接 python3 到你选择的那个版本。

---

vim:tw=78:ts=8:noet:ft=help:norl:

if_tcl.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Ingo Wilken
译者：Willis
http://vimcdoc.sf.net

Vim 的 Tcl 接口 tcl Tcl TCL

1. 命令 tcl-ex-commands
2. Tcl 命令 tcl-commands
3. Tcl 变量 tcl-variables
4. Tcl 窗口命令 tcl-window-cmds
5. Tcl 缓冲区命令 tcl-buffer-cmds
6. 杂项；Tcl 的输出 tcl-misc tcl-output
7. 已知漏洞和问题 tcl-bugs
8. 示例 tcl-examples
9. 动态调入 tcl-dynamic

E280
{仅当 Vim 编译时加入 +tcl 特性时才有效}

警告：可能还有没发现的漏洞。请把漏洞报告、意见、建议等等发送到
<Ingo.Wilken@informatik.uni-oldenburg.de>

---

1. 命令 tcl-ex-commands E571 E572

:tcl :tc

:tc[l] {cmd} 执行 Tcl 命令 {cmd}。测试 :tcl 是否可用的简单检查：
```
:tcl puts "Hello"
```

:[range]tc[l] << [endmarker]
{script}
{endmarker}

执行 Tcl 脚本 {script}。
注意：如果编译时没有加入 Tcl 特性，该命令不会工作。要避免错误，参见 script-here 。

{script} 之后的 {endmarker} 不能 有前导空白。

若 "<<" 之后省略 [endmarker]，{script} 之后必须由点 '.' 结束，类似于 :append 和 :insert 命令。

这种形式的 :tcl 命令主要用于在 Vim 脚本里包含 tcl 代码。

示例：
```
function! DefineDate()
    tcl << EOF
    proc date {} {
        return [clock format [clock seconds]]
    }
```

```
EOF
endfunction
```

要看运行的 Tcl 版本:

```
:tcl puts [info patchlevel]
```

:[range]tcld[o] {cmd}　对 [range] 范围内的每行执行 Tcl 命令 {cmd}。执行过程中，变量 "line" 被赋值为每行的文本，"lnum" 为相应的行号。改变 "line" 会改变文本，但是你不能用此命令增加或者删除行。如果 {cmd} 有错，整个命令被中断。缺省的范围 [range] 是整个文件，"1,$"。参见 tcl-var-line 和 tcl-var-lnum 。　:tcldo :tcld

:tclf[ile] {file}　执行 Tcl 脚本文件 {file}。它和 ":tcl source {file}" 相当，但可用文件名自动补全功能。　:tclfile :tclf

注意 Tcl 对象（类似于变量）在命令之间保持值不变。就像在 Tcl 外壳里使用那样。

sandbox 里不能执行 Tcl 命令。

---

2. Tcl 命令　tcl-commands

Tcl 完全通过 "::vim" 命名空间的命令访问 vim 的功能。已实现以下功能:

```
::vim::beep                     # 猜猜看。
::vim::buffer {n}               # 为单个缓冲区建立 Tcl 命令。
::vim::buffer list              # 为所有缓冲区建立 Tcl 命令。
::vim::command [-quiet] {cmd}   # 执行 Ex 命令。
::vim::expr {expr}              # 使用 Vim 的表达式计算器。
::vim::option {opt}             # 得到 vim 选项值。
::vim::option {opt} {val}       # 设置 vim 选项值。
::vim::window list              # 为所有窗口建立 Tcl 命令。
```

命令:

::vim::beep　tcl-beep
鸣笛。没有返回值。

::vim::buffer {n}　tcl-buffer
::vim::buffer exists {n}
::vim::buffer list
提供对 vim 缓冲区的访问。如果给出整数参数，为相应的缓冲区建立缓冲区命令（参见 tcl-buffer-cmds ），并返回其名称作为结果。非法的缓冲区号会产生标准 Tcl 错误。要测试哪些缓冲区号合法，可用 Vim 的内部函数:

```
set nbufs [::vim::expr bufnr("$")]
set isvalid [::vim::expr "bufexists($n)"]
```

"list" 选项则为每个合法的缓冲区建立缓冲区命令，并返回所有命令名称的列表作为结果。例如:

```
set bufs [::vim::buffer list]
foreach b $bufs { $b append end "The End!" }
```

"exists" 选项检查给定序号的缓冲区是否存在。例如：

```
if { [::vim::buffer exists $n] } { ::vim::command ":e #$n" }
```

该命令将来也许会被某个变量代替。要得到当前缓冲区，另见 tcl-var-current 。

::vim::command {cmd} tcl-command
::vim::command -quiet {cmd}
执行 vim (ex 模式) 的 {cmd} 命令。任何涉及缓冲区或者窗口的 Ex 命令均使用当前缓冲区/窗口。除了标准的 Tcl 错误代码外，不会返回任何其他的结果。在命令执行完毕后，"::vim::current" 变量被更新。
"-quiet" 标志位屏蔽 vim 产生的任何错误信息。
例如：

```
::vim::command "set ts=8"
::vim::command "%s/foo/bar/g"
```

要执行普通模式下的命令，使用 "normal" (参见 :normal)：

```
set cmd "jj"
::vim::command "normal $cmd"
```

另见 tcl-window-command 和 tcl-buffer-command 。

::vim::expr {expr} tcl-expr
使用 vim 的内部表达式计算器计算表达式 {expr} (参见 expression)。任何查询缓冲区或窗口属性的表达式均使用当前缓冲区/窗口。结果以字符串形式返回。列表 List 通过连接项目并在项目间插入换行符来转化为字符串。
例如：

```
set perl_available [::vim::expr has("perl")]
```

另见 tcl-window-expr 和 tcl-buffer-expr 。

::vim::option {opt} tcl-option
::vim::option {opt} {value}
如果没有第二个参数，查询 vim 选项之值。不然，设置 vim 选项值为 {value}，并返回原先的值作为结果。任何标为 "局部于缓冲区" 或 "局部于窗口" 的选项只影响当前的缓冲区/窗口。该命令不会改变全局值，而应该用 ":set" 命令。对于布尔值，{value} 必须是 "0" 或者 "1"，或者以下的关键字 "on"、"off" 或者 "toggle"。选项列表请参见 option-summary 。
示例：

```
::vim::option ts 8
```

另见 tcl-window-option 和 tcl-buffer-option 。

::vim::window {option} tcl-window
提供对 vim 窗口的访问。目前，只实现了 "list" 选项，即为每个窗口建立窗口命令 (参见 tcl-window-cmds)，并返回所有命令名称的列表作为结果。
示例：

```
set wins [::vim::window list]
foreach w $wins { $w height 4 }
```

该命令将来也许会被某个变量代替。要得到当前窗口，另见 tcl-var-current 。

---

3. Tcl 变量 tcl-variables

::vim 命名空间包含若干变量。Tcl 解释器调用时，它们被建立并赋予它们的当前值。

```
::vim::current          # "当前" 对象的数组
::vim::lbase            # 第一行的行号
```

```
::vim::range            # 当前范围行号的数组
line                    # 当前行号的字符串表示（只限于 :tcldo）
lnum                    # 当前行号（只限于 :tcldo）
```

命令：

::vim::current　　tcl-var-current

这是一个提供对各种 vim "当前" 对象访问的数组。每次 "::vim::command" 执行后都会更新该数组的内容，因为命令的执行可能改变 vim 的当前设定（例如，删除当前缓冲区）。

其中，"buffer" 元素包含当前缓冲区的缓冲区命令名，可用来直接调用缓冲区命令（参见 tcl-buffer-cmds ）。该元素只读。

示例：

```
$::vim::current(buffer) insert begin "Hello world"
```

"window" 元素则包含当前窗口的窗口命令名。可用来直接调用窗口命令（参见 tcl-window-cmds ）。该元素只读。

示例：

```
$::vim::current(window) height 10
```

::vim::lbase　　tcl-var-lbase

该变量控制 Tcl 如何看待行号。如果设为 '1'，行号与列号从 1 开始。这样，Tcl 命令和 vim 表达式行号的使用方式一致。如果设为 '0'，则行号与列号从 0 开始。如果你倾向于把缓冲区看成 Tcl 的列表或者把行看成 Tcl 字符串，那么 Tcl 中标准的返回索引的命令（例如 "lsort" 或 "string first"）就适用于这种设置。缺省值为 '1'。目前，任何非零的值都被看成 '1'，但你的脚本不应依赖于此假定。另见 tcl-linenumbers 。

::vim::range　　tcl-var-range

这是一个由三个元素组成的数组："start"、"begin" 和 "end"。它包含当前行范围的起始和结尾行号。"begin" 等价于 "start"。该变量只读。参见 tcl-examples 。

line　　tcl-var-line

lnum　　tcl-var-lnum

这些全局变量只在 ":tcldo" 这个 Ex 命令执行时可用。它们分别包含文本和数字形式的当前行号。当 ":tcldo" 激活的 Tcl 命令执行完毕时，当前行被设为 "line" 变量的内容，除非该变量被 Tcl 命令删除（unset）。"lnum" 变量是只读的。这些变量不在 "::vim" 命名空间里，这是为了 ":tcldo" 里使用时能减少点输入（将来的版本里可能会改变）。另见 tcl-linenumbers 。

---

4. Tcl 窗口命令　　tcl-window-cmds

窗口命令代表 vim 的窗口。以下若干命令可以创建之：

::vim::window list　　tcl-window

缓冲区命令的 "windows" 选项　　tcl-buffer-windows

::vim::current(window) 变量包含当前窗口的窗口命令名。当 vim 窗口关闭时，对应的窗口命令自动被删除。

让我们假设窗口命令名保存在 Tcl 变量 "win" 中，亦即，$win 调用该命令。那么可以使用以下的选项：

```
$win buffer             # 创建窗口对应缓冲区的 Tcl 命令。
$win command {cmd}      # 使用窗口的上下文，执行 Ex 命令。
$win cursor             # 得到当前的光标位置。
```

```
$win cursor {var}       # 把光标位置保存在数组变量里。
$win cursor {row} {col} # 设置光标位置。
$win delcmd {cmd}       # 在窗口被关闭前，执行 Tcl 命令。
$win expr {expr}        # 使用窗口的上下文，计算 vim 表达式。
$win height             # 报告窗口的高度。
$win height {n}         # 设置窗口的高度。
$win option {opt} [val] # 使用窗口的上下文，得到/设置 vim 选项。
```

选项：

$win buffer　　tcl-window-buffer

创建窗口对应缓冲区的 Tcl 命令，并返回其名字作为结果。该名字应该被存在变量里：

```
set buf [$win buffer]
```

$buf 现在成为合法的 Tcl 命令。参见 tcl-buffer-cmds 了解其可用选项。

$win cursor　　tcl-window-cursor

$win cursor {var}

$win cursor {row} {col}

在没有参数的情况下，报告（字符串形式的）当前的光标位置。该形式可以被转换成 Tcl 的数组变量：

```
array set here [$win cursor]
```

"here(row)" 和 "here(column)" 现在包含了光标位置。

在有一个参数的情况下，该参数被解释为 Tcl 数组变量名，该数组变量应该有两个元素："row" 和 "column"。它们用来设置光标的新位置：

```
$win cursor here        ;# 不是 $here !
```

在有两个参数的情况下，设置光标到对应的行和列：

```
$win cursor $here(row) $here(column)
```

非法的位置产生标准 Tcl 错误，但可用 "catch" 捕获。行号和列号的值和 "::vim::lbase" 有关。参见 tcl-var-lbase 。

$win delcmd {cmd}　　tcl-window-delcmd

注册窗口的关闭回调函数 {cmd}。该命令（在全局范围下）在窗口被关闭前调用。复杂的命令应该用 "list" 构造：

```
$win delcmd [list puts vimerr "window deleted"]
```

另见 tcl-buffer-delcmd 。

$win height　　tcl-window-height

$win height {n}

在没有参数的情况下，报告当前的窗口高度。在有参数的情况下，设置窗口高度为 {n}，并报告新的高度（有可能和 {n} 不同）。

$win command [-quiet] {cmd}　　tcl-window-command

$win expr {expr}　　tcl-window-expr

$win option {opt} [val]　　tcl-window-option

它们和 "::vim::command" 类似，只不过执行在 $win 代表的窗口的上下文下，而不是当前窗口。例如，设置 "局部于窗口" 的选项涉及的是 $win 窗口。任何涉及或查询缓冲区的命令使用的该窗口所显示的缓冲区（亦即，"$win buffer" 所指定的缓冲区）。参见 tcl-command 、 tcl-expr 和 tcl-option 。

示例：

```
$win option number on
```

---

5. Tcl 缓冲区命令　　tcl-buffer-cmds

缓冲区命令代表 vim 的缓冲区。以下若干命令可以创建之：

```
::vim::buffer {N}                          tcl-buffer
::vim::buffer list                         tcl-buffer
"buffer" option of a window command        tcl-window-buffer
```

::vim::current(buffer) 变量包含当前缓冲区的缓冲区命令名。vim 缓冲区被删除时，对应的缓冲区命令自动被删除。缓冲区一旦改变，缓冲区里的所有的位置标记会进行自动调整。Tcl 命令对缓冲区内容所做的任何改变都可以通过 vim 的 "undo" 命令撤销（参见 undo ）。

让我们假设缓冲区命令名保存在 Tcl 变量 "buf" 中，亦即，$buf 调用该命令。那么可以使用以下的选项：

```
$buf append {n} {str}   # 在缓冲区的第 {n} 行之后添加一行内容。
$buf command {cmd}      # 使用缓冲区上下文。执行 Ex 命令。
$buf count              # 报告缓冲区的行数。
$buf delcmd {cmd}       # 在缓冲区被删除前，执行 Tcl 命令。
$buf delete {n}         # 删除一行。
$buf delete {n} {m}     # 删除多行。
$buf expr {expr}        # 使用窗口的上下文，计算 vim 表达式。
$buf get {n}            # 得到字符串形式的一行内容。
$buf get {n} {m}        # 得到列表形式的多行内容。
$buf insert {n} {str}   # 在缓冲区里插入一行，使之成为第 {n} 行。
$buf last               # 报告缓冲区末行的行号。
$buf mark {mark}        # 报告缓冲区位置标记的位置。
$buf name               # 报告缓冲区使用的文件名。
$buf number             # 报告缓冲区号。
$buf option {opt} [val] # 使用缓冲区的上下文，得到/设置 vim 选项。
$buf set {n} {text}     # 替换一行。
$buf set {n} {m} {list} # 替换多行。
$buf windows            # 创建缓冲区对应窗口的 Tcl 命令。
```

tcl-linenumbers

多数缓冲区命令需要行号作为参数。Tcl 如何处理这些数字取决于 "::vim::lbase" 变量（参见 tcl-var-lbase ）。除了数字形式的行号以外，还可以使用如下的关键字："top"、"start"、"begin"、"first"（以上均代表首行），"bottom"、"end" 和 "last"（以上均代表末行）。

选项：

$buf append {n} {str} tcl-buffer-append
$buf insert {n} {str} tcl-buffer-insert

往缓冲区中加入一行。如果使用 "insert" 选项，字符串成为新的第 {n} 行。而如果使用 "append"，它被插入在第 {n} 行之后。

例如：

```
$buf insert top "This is the beginning."
$buf append end "This is the end."
```

要往缓冲区里加入多行，可以使用循环：

```
foreach line $list { $buf append $num $line ; incr num }
```

$buf count tcl-buffer-count

报告缓冲区的行数。

$buf delcmd {cmd} tcl-buffer-delcmd

注册缓冲区的删除回调函数 {cmd}。该命令（在全局范围下）在缓冲区被删除前调用。复杂的命令应该用 "list" 构造：

```
        $buf delcmd [list puts vimerr "buffer [$buf number] gone"]
```
另见 tcl-window-delcmd 。

$buf delete {n} tcl-buffer-delete
$buf delete {n} {m}
删除缓冲区的第 {n} 行或者第 {n} 到 {m} 行。以下例子删除除了末行以外的所有内容:
```
        $buf delete first [expr [$buf last] - 1]
```

$buf get {n} tcl-buffer-get
$buf get {n} {m}
从缓冲区里取得一行或多行。如果是前者，结果是字符串。如果是后者，结果是字符串列表。例如:
```
        set topline [$buf get top]
```

$buf last tcl-buffer-last
报告末行的行号。行号和 "::vim::lbase" 有关。参见 tcl-var-lbase 。

$buf mark {mark} tcl-buffer-mark
报告命名位置标记的位置。结果以字符串形式出现，类似于窗口命令的 "cursor" 选项的光标位置 (参见 tcl-window-cursor)。它可以被转换成 Tcl 数组变量:
```
        array set mpos [$buf mark "a"]
```
"mpos(column)" 和 "mpos(row)" 现在包含了标记的位置。如果标记没有设置，产生标准 Tcl 错误。

$buf name
报告缓冲区使用的文件名。如果是无名缓冲区，返回空字符串。

$buf number
报告缓冲区号。参见 :buffers 。
以下示例从 vim 里删除一个缓冲区:
```
        ::vim::command "bdelete [$buf number]"
```

$buf set {n} {string} tcl-buffer-set
$buf set {n} {m} {list}
替换缓冲区里的一行或多行。如果列表 {list} 里包含超过被替换的行数，多余的部分被插入。如果不足，则未替换的部分从缓冲区里删除。

$buf windows tcl-buffer-windows
创建缓冲区对应的所有的窗口命令，返回所有命令名字的列表。
示例:
```
        set winlist [$buf windows]
        foreach win $winlist { $win height 4 }
```
参见 tcl-window-cmds 了解窗口命令可用的选项。

$buf command [-quiet] {cmd} tcl-buffer-command
$buf expr {expr} tcl-buffer-expr
$buf option {opt} [val] tcl-buffer-option
它们和 "::vim::command" 类似，只不过执行在 $buf 代表的缓冲区的上下文下， 而不是当前缓冲区。例如，设置 "局部于缓冲区" 的选项涉及的是 $buf 缓冲区。任何涉及或查询窗口的命令使用的该缓冲区所处的窗口列表的第一个窗口 (亦即，"$buf windows" 所指定的首个窗口)。参见 tcl-command 、 tcl-expr 和 tcl-option 。

示例：

```
if { [$buf option modified] } { $buf command "w" }
```

---

6. 杂项；Tcl 的输出 tcl-misc tcl-output

标准 Tcl 命令 "exit" 和 "catch" 被定制的版本所取代。 "exit" 终止当前 Tcl 脚本并返回 vim，并关闭 Tcl 解释器。 下一个 ":tcl" 的调用相应的创建一个新的 Tcl 解释器。"exit" 并 不 终止 vim！"catch" 和原先的工作方式相同，只不过，它不能防止脚本从 "exit" 退出。退出时，非零的结束代码使得执行 Tcl 脚本的 Ex 命令生成一个错误。

在 Tcl 里，有两个新的 I/O 流："vimout" 和 "vimerr"。所有其上的输出都在 vim 的消息区域分别作为消息和错误显示。标准的 Tcl 输出流 stdout 和 stderr 被映射为 vimout 和 vimerr。从而，正常的 "puts" 命令可以用来在 vim 里显示消息。

---

7. 已知的漏洞和问题 tcl-bugs

在 Tcl 里调用另一个 Tcl 的 Ex 命令（通过 "::vim::command"）可能有意想不到的副作用。该命令创建的新的解释器和标准的解释器的能力相同。因而，在安全的子解释器里使得 "::vim::command" 可用使得该子解释器不再安全。（为了防止这一点而防止嵌套的 :tcl* 调用很容易，但不切实际。因为根据具体的 vim 的配置，"::vim::command" 可以在任何其他的脚本语言里执行任何代码。）在这个新的解释器里 "exit" 不会影响旧的解释器；它只会终止新的解释器，而旧解释器的脚本处理继续进行。

现在还不支持从标准输入读入。

---

8. 示例： tcl-examples

这里提供若干简短（但可能有用）的 Tcl 脚本。

此脚本对整个缓冲区进行排序（不妨假设缓冲区内容包含名字或类似事物的列表）：

```
set buf $::vim::current(buffer)
set lines [$buf get top bottom]
set lines [lsort -dictionary $lines]
$buf set top bottom $lines
```

此脚本对缓冲区的行进行倒排。注意 "::vim::lbase" 和 "$buf last" 的正确使用，以适用行号的所有可能情况。

```
set buf $::vim::current(buffer)
set t $::vim::lbase
set b [$buf last]
while { $t < $b } {
        set tl [$buf get $t]
        set bl [$buf get $b]
        $buf set $t $bl
        $buf set $b $tl
        incr t
        incr b -1
}
```

此脚本为当前范围的每行加上连续的行号：

```
set buf $::vim::current(buffer)
set i $::vim::range(start)
set n 1
while { $i <= $::vim::range(end) } {
        set line [$buf get $i]
        $buf set $i "$n\t$line"
        incr i ; incr n
}
```

用 ":tcldo" 可以更快地以两个 Ex 命令完成同样的工作：

```
:tcl set n 1
:[range]tcldo set line "$n\t$line" ; incr n
```

此过程为每个缓冲区执行相同的 Ex 命令（从 Ron Aaron 偷来的主意）：

```
proc eachbuf { cmd } {
        foreach b [::vim::buffer list] {
                $b command $cmd
        }
}
```

使用方法为：

```
:tcl eachbuf %s/foo/bar/g
```

小心使用 Tcl 的字符串和反斜杠替换，很麻烦。如果不确定，不妨在 Ex 命令前后括上花括号。

如果你想为 vim 永久地加入若干 Tcl 过程，只要把它们放在一个文件里（例如在 Unix 机器上，"~/.vimrc.tcl"），并在启动文件里（在 Unix 上通常是 "~/.vimrc"）加入：

```
if has("tcl")
        tclfile ~/.vimrc.tcl
endif
```

---

9. 动态调入 tcl-dynamic

MS-Windows 和 Unix 上，可以动态调入 Tcl 库。 :version 输出这时应包括 +tcl/dyn 。

这意味着 Vim 只有在必要时才寻找 Tcl DLL 或共享库文件。如果不使用 Tcl 接口，你就不需要它。这样，即使没有该文件，你也可使用 Vim。

MS-Windows

要使用 Tcl 接口，Tcl DLL 必须在搜索路径上。控制台窗口里输入 "path" 可以看到（搜索路径）当前使用的目录。也可用 'tcldll' 选项指定 Tcl DLL。

DLL 的名字必须匹配 Vim 编译时所使用的 Tcl 版本。目前，该名字为 "tcl86.dll"，也就是 Tcl 8.6。要确信这一点，编辑 "gvim.exe" 文件并查找 "tcl\d*.dll\c"。

Unix

'tcldll' 选项可用来指定 Tcl 共享库文件，而不用编译时指定的 DYNAMIC_TCL_DLL 文件。共享库的版本必须和 Vim 编译使用的 Tcl 版本保持一致。

---

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Paul Moore
译者: Dasn
http://vimcdoc.sf.net

Vim 的 OLE 接口 ole-interface

1. 激活 OLE 功能 ole-activation
2. 方法 ole-methods
3. "normal" 命令 ole-normal
4. 注册 ole-registration
5. 与微软的 Visual Studio 集成 MSVisualStudio

{仅当编译时加入 +ole 特性才有效。参见 src/if_ole.INSTALL}
一个替代方案是使用 客户机服务器 模式进行通信 clientserver 。

---

1. 激活 OLE 功能 ole-activation

Vim 可以作为 OLE 服务器运行，能被任意的 OLE 客户端访问，例如：我们可以用 Visual Basic，Python，或者 Perl 与 Vim 通信。Vim 的程序名（在 OLE 中称为 "ProgID"）是 "Vim.Application"。

因此，要启动一个 Vim 实例（或者要与一个已经启动的实例通信），你应该使用类似于下面的代码：

```
[Visual Basic]
        Dim Vim As Object
        Set Vim = CreateObject("Vim.Application")

[Python]
        from win32com.client.dynamic import Dispatch
        vim = Dispatch('Vim.Application')

[Perl]
        use Win32::OLE;
        $vim = new Win32::OLE 'Vim.Application';

[C#]
        // 在项目中加入 Vim 的引用。
        // 选择 COM 标签页。
        // 选择 "Vim Ole Interface 1.1 Type Library"
        Vim.Vim vimobj = new Vim.Vim();
```

Vim 不能作为 "隐藏" 的 OLE 服务器来运行，这一点 Vim 不像其它一些 OLE 服务器。Vim 的 OLE 客户端启动时，一个 Vim 程序的实例会立刻显现出来。简单地断开 OLE 连接不能关掉 Vim 实例，需要显式地执行退出命令才可以（例如，:qa!，:wqa）。

---

2. 方法 ole-methods

Vim 为客户端提供了四种方法。

ole-sendkeys

SendKeys(keys)　　执行一系列键击。

这个函数只需要一个参数，参数 keys 代表要敲击的字符串。通过此函数发送这些字符串和你直接在 Vim 里敲击键盘的效果是一样的。特殊键可以用它们的 <..> 名字，就像映射的右手边里的格式一样（参考 :map 特殊键的表达方式译者）。备注：不支持 Ex 模式下的 "normal" 命令 - 见下面 ole-normal 。

例子：(Visual Basic 语法)

```
Vim.SendKeys "ihello<Esc>"
Vim.SendKeys "ma1GV4jy`a"
```

这个例子假设 Vim 处于普通模式，要强制进入普通模式，可以在字符序列前加入 CTRL-\ CTRL-N，就像这样：

```
Vim.SendKeys "<C-\><C-N>ihello<Esc>"
```

(译者注：下面是 Perl 语法：

```
$vim->SendKeys("<C-\\><C-N>ihello<Esc>");
```

备注：在 Perl 语法里，特殊字符前需要加 "\"。)

CTRL-\ CTRL-N 使得 Vim 从插入模式或命令行模式返回普通模式。
备注：不能用这个命令发送 Vim 命令的一部分。

ole-eval

Eval(expr)　　计算表达式的值。

函数只有一个参数 expr，Vim 正常格式的表达式（参考 expression ）。返回值是一个包含表达式结果的字符串。 List 通过连接项目并插入换行符以转化为字符串。

例子：(Visual Basic 语法)

```
Line20 = Vim.Eval("getline(20)")
Twelve = Vim.Eval("6 + 6")              ' 注意，这是一个字符串。
Font = Vim.Eval("&guifont")
```

ole-setforeground

SetForeground()　　让 Vim 窗口转入前台工作。

这个函数没有参数，没有返回值。

例子：(Visual Basic 语法)

```
Vim.SetForeground
```

ole-gethwnd

GetHwnd()　　返回 Vim 窗口的句柄。

这个函数没有参数，它返回 Vim 主窗口的句柄。你可以用它来控制 Vim 窗口，或者跟踪

窗口的 z-深度 等位置。

例子: (Visual Basic 语法)

```
Vim_Hwnd = Vim.GetHwnd
```

---

3. "normal" 命令 ole-normal

因为 Vim 是通过 Ex 模式下的 :normal 命令来实现对 OLE 命令的处理，所以 Vim 不能通过 OLE 执行 :normal 命令。如果你这样做，肯定没用。至于产生的结果，也许没什么害处，也许不能预料。

现在没有办法能捕获这种错误，所以用户在使用的时候要注意有这么一个限制。

---

4. 注册 ole-registration E243

在 Vim 作为 OLE 服务器运行之前，必须向系统注册。如果你想注册，可以在运行 Vim 的时候加一个参数 "-register"。

-register

```
gvim -register
```

如果运行 OLE 版本的 gvim 时，Vim 的 OLE 服务没有注册，gvim 会弹出一个对话框，让你选择是否注册，选择 "Yes" 注册。

有时候可能注册不会成功，这可能是因为注册表不可写。如果是这个原因，你需要以管理员的身份来运行 gvim。

一旦注册，Vim 的路径就会被存储到注册表里。在你移动、删除或升级 Vim 之前，应该使用 "-unregister" 清除这些注册信息。

-unregister

```
gvim -unregister
```

系统会使用最先注册的 Vim 执行 OLE 命令。如果 Vim 已经运行，那么就使用当前运行的 Vim。如果你不想让 (多个) Vim 响应 OLE 命令，请使用非 OLE 版本的 Vim，并且把它放到别的目录里。这时，OLE 版本的 Vim 路径不应包含在你的正常路径里，这样当你键入 "gvim" 时启动的才是非 OLE 版本。

-silent

如果不想让 Vim 在注册时弹出对话框，可以在前面加 "-silent" 参数:

```
gvim -silent -register
gvim -silent -unregister
```

---

5. 与微软的 Visual Studio 集成 MSVisualStudio VisVim

Vim 的 OLE 版本可以用做 Visual Studio 的文本编辑器。这被称作 "VisVim"。OLE 版本的软件包里有一个 runtime 目录，详细的说明请参阅该目录下的 README_VisVim.txt 文件。

在 Visual Studio .Net 里使用 Vim

.Net 里不再需要 VisVim 了，因为 .Net Studio 已经提供了外部编辑器的支持。按以下步骤进行:

在 .Net Studio 里，选择菜单 Tools->External Tools...
加入

```
Title     - Vim
Command   - c:\vim\vim63\gvim.exe
Arguments - --servername VS_NET --remote-silent "+call cursor($(CurLine), $(CurCol))" $(ItemP
Init Dir  - 空
```

现在在 .Net 里打开文件时，可以在 .Net 菜单里选择: Tools->Vim

这样就能在 Vim 里打开文件。
可以把该外部命令作为图标加入，把它放在任何你喜欢的位置上。也可以把它设为你的缺省编辑器。

如果你有进一步的改进，请到 Vim 邮件列表上发帖，以告知我们。

--servername VS_NET
建立 vim 的新实例，名为 VS_NET。这样如果你从 VS 里打开多个文件，它们会使用相同的 Vim 实例。这样，你可以运行多个 Vim 的备份，但可以控制在哪个里编辑 VS 文件。

--remote-silent "+call cursor(10, 27)"
- 把光标放在第 10 行第 27 列
Vim 里
:h --remote-silent 可以了解详情

[.Net 的评注由 Dave Fishburn 和 Brian Sturk 提供]

---

if_ruby.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Shugo Maeda
译者：Willis，tocer
http://vimcdoc.sf.net

Vim 的 Ruby 接口 ruby Ruby

1. 命令 ruby-commands
2. Vim 模块 ruby-vim
3. Vim::Buffer 对象 ruby-buffer
4. Vim::Window 对象 ruby-window
5. 全局变量 ruby-globals
6. rubyeval() Vim 函数 ruby-rubyeval
7. 动态调入 ruby-dynamic

E266 E267 E268 E269 E270 E271 E272 E273

{仅当编译 Vim 时加入 +ruby 特性才有效}

ruby 的主页是 http://www.ruby-lang.org/。你可以在那里找到 Ruby 下载的链接。

---

1. 命令 ruby-commands

:ruby :rub

:rub[y] {cmd} 执行 Ruby 命令 {cmd}。要试一试：
```
:ruby print "Hello"
```

:rub[y] << [endmarker]
{script}
{endmarker}

执行 Ruby 脚本 {script}。
{script} 之后的 {endmarker} 不能 有前导空白。

如果省略 [endmarker]，缺省为单个句号 '.'，类似于 :append 和 :insert 命令。

这种形式的 :ruby 命令主要用于在 vim 脚本里包含 ruby 代码。

注意：如果编译时没有加入 Ruby 特性，这个命令不会工作。为了避免错误，参看 script-here 。

示例 Vim 脚本：

```
function! RedGem()
ruby << EOF
class Garnet
        def initialize(s)
                @buffer = Vim::Buffer.current
                vimputs(s)
```

```
        end
        def vimputs(s)
                @buffer.append(@buffer.count,s)
        end
end
gem = Garnet.new("pretty")
EOF
endfunction
```

要看运行的 Ruby 版本：

```
:ruby print RUBY_VERSION
```

:rubydo :rubyd E265

:[range]rubyd[o] {cmd} 在 [range] 界定的每行上分别执行 Ruby 命令 {cmd}，执行时，$_ 设为相应的行文本，不含结尾的 <EOL>。修改 $_ 就会修改相应的行，不过要 注意：此命令不能用来增加或者删除行。
缺省的 [range] 是整个文件："1,$"。

:rubyfile :rubyf

:rubyf[ile] {file} 执行 Ruby 脚本文件 {file}。这和 ':ruby load 'file'' 相当，但可用文件名自动补全功能。

sandbox 里不能执行 Ruby 命令。

---

2. Vim 模块 ruby-vim

Ruby 代码通过 "Vim" 模块调用所有的 vim 功能。

概述：

```
print "Hello"                          # 显示消息
Vim.command(cmd)                       # 执行 ex 命令
num = Vim::Window.count                # 得到窗口行数
w = Vim::Window[n]                     # 得到第 "n" 个窗口
cw = Vim::Window.current               # 得到当前窗口
num = Vim::Buffer.count                # 得到缓冲区的个数
b = Vim::Buffer[n]                     # 得到第 "n" 个缓冲区
cb = Vim::Buffer.current               # 得到当前缓冲区
w.height = lines                       # 设置窗口高度
w.cursor = [row, col]                  # 设置窗口里的光标位置
pos = w.cursor                         # 得到数组 [行, 列]
name = b.name                          # 得到缓冲区的文件名
line = b[n]                            # 得到缓冲区的某一行
num = b.count                          # 得到总的行数
b[n] = str                             # 设置缓冲区的某一行
b.delete(n)                            # 删除某一行
b.append(n, str)                       # 在第 n 行后增加一行
line = Vim::Buffer.current.line        # 得到当前行
num = Vim::Buffer.current.line_number  # 得到当前行号
Vim::Buffer.current.line = "test"      # 设置当前行
```

模块函数：

ruby-message

Vim::message({msg})
显示消息 {msg}。

ruby-blob

Vim::blob({arg})
返回来自 {arg} 的 Blob 字面常量字符串。

ruby-set_option

Vim::set_option({arg})
设置 vim 选项。{arg} 可以是任何一个 ":set" 命令接受的命令。 注意 这意味着参数里不能有空格！参见 :set 。

ruby-command

Vim::command({cmd})
执行 Ex 命令 {cmd}。

ruby-evaluate

Vim::evaluate({expr})
用 vim 内部的表达式分析器执行 {expr} (参看 expression)。执行结果返回的类型如下：
- Integer, 如果 Vim 表达式返回数值
- Float, 如果 Vim 表达式返回浮点数
- String, 如果 Vim 表达式返回字符串
- Array, 如果 Vim 表达式返回 Vim 列表
- Hash, 如果 Vim 表达式返回 Vim 字典
递归扩展字典和列表。

---

3. Vim::Buffer 对象 ruby-buffer

Vim::Buffer 对象代表 Vim 缓冲区。

类方法：

current 返回当前缓冲区对象。
count 返回缓冲区的总数。
self[{n}] 返回第 {n} 个缓冲区。首个缓冲区的序号是 0。

方法：

name 返回缓冲区的全名。
number 返回缓冲区的序号。
count 返回总行数。
length 返回总行数。
self[{n}] 从缓冲区中返回一行。{n} 是行号。
self[{n}] = {str}
设置缓冲区的一行。{n} 是行号。
delete({n}) 删除缓冲区的一行。{n} 是行号。
append({n}, {str})

	在第 {n} 行后增加一行。
line	如果缓冲区激活，返回缓冲区的当前行。
line = {str}	如果缓冲区激活，设置缓冲区的当前行。
line_number	如果缓冲区激活，返回当前行的行号。

---

4. Vim::Window 对象 ruby-window

Vim::Window 对象代表 Vim 窗口。

类方法:

current	返回当前窗口对象。
count	返回窗口的总数。
self[{n}]	返回第 {n} 个窗口。首个窗口的序号是 0。

方法:

buffer	返回显示的缓冲区对象。
height	返回窗口的高度。
height = {n}	设置窗口的高度为 {n}。
width	返回窗口的宽度。
width = {n}	设置窗口的宽度为 {n}。
cursor	返回光标位置数组: [行、列]。 首行号为 1 而首列号为 0。
cursor = [{row}, {col}]	设置光标位置为行 {row} 和列 {col}。

---

5. 全局变量 ruby-globals

有两个全局变量。

$curwin	当前窗口对象。
$curbuf	当前缓冲区对象。

---

6. rubyeval() Vim 函数 ruby-rubyeval

为提供双向接口的方便，可用 rubyeval() 函数来计算 Ruby 表达式并传递其值给 Vim 脚本。

Ruby 值 "true"、"false" 和 "nil" 分别转换为 v:true、v:false 和 v:null。

---

7. 动态调入 ruby-dynamic

MS-Windows 和 Unix 上，可以动态调入 Ruby 库。 :version 输出这时应包括 +ruby/dyn 。

这意味着 Vim 只有在必要时才寻找 Ruby DLL 或动态库文件。如果不使用 Ruby 接口，你就不需要它。这样，即使没有该 DLL 文件，你也可使用 Vim。

MS-Windows

必须安装正确的 Ruby 版本才能工作。该包可以从这里下载:
http://rubyinstaller.org/downloads/
目前的版本是 rubyinstaller-2.2.5.exe

要使用 Ruby 接口，Ruby DLL 必须在搜索路径上。控制台窗口里输入 "path" 可以看到(搜索路径) 当前使用的目录。也可用 'rubydll' 选项设置 Ruby DLL。

DLL 名必须匹配 Vim 编译时所使用的 Ruby 版本。目前，该名字为 "msvcrt-ruby220.dll"，也就是 Ruby 2.2.X。要确信这一点，编辑 "gvim.exe" 文件并查找 "ruby\d*.dll\c"。

如果编译 Vim 时要使用带 MSVC 的 RubyInstaller 1.9 或 2.X，需要一点技巧。详见 src/INSTALLpc.txt。

如果 Vim 使用 RubyInstaller 2.4 或以后版本编译，可能还需要在 PATH 环境变量里加入 "C:\Ruby<version>\bin\ruby_builtin_dlls"。

Unix

'rubydll' 选项可用来指定 Ruby 共享库文件，而不用编译时指定的 DYNAMIC_RUBY_DLL 文件。共享库的版本必须和 Vim 编译使用的 Ruby 版本保持一致。

VIM 参考手册 by Gordon Prieur
译者：iCrazy <icrazy@ustc.edu> http://vimcdoc.sf.net

支持调试器的特性 debugger-support

这里的特性是为了和调试器、集成编程环境（IPE）或集成开发环境（IDE）集成使用的。要在 Vim 终端窗口中运行调试器，可见 terminal-debugger 。

1. 调试器的功能 debugger-features
2. Vim 编译选项 debugger-compilation
3. 整合的调试器 debugger-integration

---

1. 调试器的功能 debugger-features

有以下功能可用：

替代命令输入	alt-input
调试标号	debug-signs
调试源代码高亮	debug-highlight
信息页脚	gui-footer
气泡表达式求值	balloon-eval

这些特性主要是针对图形界面的 gvim 而新增的。然而， alt-input 和 debug-highlight 是同时针对 vim 和 gvim 进行开发的。一些其他的功能略做修改，即可在非 GUI 的 vim 上使用。但是，我没有这样做，我也没有测试过针对非图形 GUI 界面编译出的 vim 的可靠性。

1.1 替代命令输入 alt-input

要使 Vim 和一个调试器一起工作，至少必须有一种和调试器进行输入连接的方式，或者有一个外部工具。尽管很多情况下，也同时会有一个输出连接，但这并不是必要的。

输入连接的目的是让外部调试器把命令传送给 Vim。调试器送来的命令必须足以使它自己显示当前的调试环境和状态。

目前的实现是基于 X 工具包中的 dispatch 循环和对 XtAddInput() 函数的调用的。

1.2 调试标号 debug-signs

很多调试器用在一些行前面加一个小记号或使这些行颜色高亮的方式来给它们做标记。命令 :sign 可以使得调试器设定这种图形标号。这项特性有什么作用呢？举例来说，它可以用来使调试器显示一个箭头，从而指示正在被调试的程序的程序计数器（PC）。另一个用处是在有断点的行后面显示一个小的停止标号。这些可见的标号可以使用户得知一些调试器的状态信息。

这些特性不仅仅可以和调试器一起使用。它们可以使一个 IPE 高亮显示编译错误、查找到的文本等等。标号功能还可以和 debug-highlight 一起工作，从而保证标号可以被

显眼的显示出来。

调试标号可以用命令 :sign 来定义和设置。

1.3 调试源代码高亮 debug-highlight

这项特性可以使得您把一行文字突出地高亮。这种高亮是用来使某一行突出显示的。高亮功能在 vim 和 gvim 中都可以工作，而调试标号一般仅在 gvim 下工作。Sun Microsystem 的 dtterm 是一个例外，它有一个 "sign gutter" 可以用来显示标号。

1.4 信息页脚 gui-footer

信息页脚可以用来显示调试器或 IPE 中传来的信息。它还可以用来显示菜单和工具栏提示。页脚区域位于 GUI 窗口的底部，在用来显示冒号 (:) 命令的那一行的下方。

页脚的显示是用 'guioptions' 中的字母 'F' 来控制的。

1.5 气泡表达式求值 balloon-eval

这项特性可以使一个调试器或其他的外部工具根据鼠标所指的位置显示动态的信息。它的目的在于使 Sun 的 Visual WorkShop 调试器能够显示表达式表达式求值。实际上，这个特性是用通用的方式实现的，所以它还可以用来显示其他的信息。不过此功能很有限，更高级的弹出界面可见 popup-window 。

另一种使用气泡的方法是使用 'balloonexpr' 选项。完全由用户定义。

气泡表达式求值还有一些可以设置的参数。对于 Motif，字体列表和颜色可以通过 X 资源 (XmNballoonEvalFontList、XmNballoonEvalBackground 和 XmNballoonEvalForeground) 来设定。

选项 'balloondelay' 用来设定显示气泡之前的延迟时间。
选项 'balooneval' 和/或 'balloonevalterm' 选项必须打开。

气泡表达式求值仅当用 +balloon_eval 特性编译时有效。对终端而言，适用的是 +balloon_eval_term 特性。

气泡表达式求值功能还被用来为工具栏显示提示。对于这个用途，无须设置选项 'balooneval'，但是其他选项的值将被应用。

---

2. Vim 编译选项 debugger-compilation

调试器特性是为了和 Sun 的 Visual WorkShop 集成编程环境 (ipe) 一起使用而开发的。事实上，它们使用了一种通用的方式，所以其他的调试器也可以使用一部分，甚至是所有的那些原本被 Sun 的 ipe 所使用的工具。

以下的编译时的预处理程序变量控制着这些特性：

交替命令输入	ALT_X_INPUT
调试符号	FEAT_SIGNS
调试高亮	FEAT_SIGNS

信息页脚	FEAT_FOOTER
气泡表达式求值	FEAT_BEVAL

最先提供且带有完整 IPE/IDE 的整合来自 Sun Visual WorkShop。如果需要编译一个带有 VWS 接口的 gvim，请设置下面这个标志位，它会设置以上所有的标志位：

Sun Visual WorkShop	FEAT_SUN_WORKSHOP

---

3. 整合的调试器 debugger-integration

一个完全整合的 调试器/IPE/IDE 是 Sun 的 Visual WorkShop 集成编程环境。

对 Sun NetBeans 的支持，可见 netbeans 。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

netbeans.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Gordon Prieur et al.
译者：Willis
http://vimcdoc.sf.net

netbeans NetBeans netbeans-support

Vim NetBeans 协议：把 Vim 集成到 IDE 中的套接字接口。

1. 简介 netbeans-intro
2. 集成特性 netbeans-integration
3. 支持 NetBeans 的 Vim 配置 netbeans-configure
4. 错误消息 netbeans-messages
5. 从 NetBeans 模式里运行 Vim netbeans-run
6. NetBeans 协议 netbeans-protocol
7. NetBeans 命令 netbeans-commands
8. 已知问题 netbeans-problems
9. 调试 NetBeans 协议 netbeans-debugging
10. NetBeans 外部编辑器
    10.1. 下载 NetBeans netbeans-download
    10.2. NetBeans 键盘映射 netbeans-keybindings
    10.3. 为 Vim 准备 NetBeans netbeans-preparation
    10.4. 取得外部编辑器模块 obtaining-exted
    10.5. 设置 NetBeans 以运行 Vim netbeans-setup

{仅当编译时加入 +netbeans_intg 特性才有效}

---

1. 简介 netbeans-intro

NetBeans 接口原本是为了集成 Vim 到 NetBeans Java IDE 里而使用外部编辑器插件开发的。现在已经没有支持 NetBeans 最近版本的该 NetBeans 插件，但该协议的开发方式使得任何 IDE 都可以用它来集成 Vim。

Vim 的 NetBeans 协议是一个建立在传统的 TCP 套接字接口之上的基于文本的通信协议。它不依赖于 Java 或 NetBeans。任何语言或环境只要提供了套接字接口，就可以使用该协议控制 Vim。该协议已经有 C、C++、Python 和 Java 的实现。今天我们之所以还保持 NetBeans 这个名字，只是基于历史上的渊源。

目前使用 Vim 的 NetBeans 协议的项目有：
- VimIntegration，有关于若干 Vim 集成项目的描述：
  http://www.freehackers.org/VimIntegration
- Agide，Python 编写的用于 AAP 项目的 IDE：
  http://www.a-a-p.org
- Clewn，C 编写的 Vim 上的 gdb 集成：
  http://clewn.sourceforge.net/
- Pyclewn，Python 编写的 Vim 的 gdb 集成：
  http://pyclewn.sourceforge.net/
- VimPlugin，Eclipse 上的 Vim 集成：
  http://vimplugin.sourceforge.net/wiki/pmwiki.php
- PIDA，Python 编写的集成 Vim 的 IDE：

        http://pida.co.uk/
- VimWrapper，方便把 Vim 集成到 IDE 的库:
        http://www.freehackers.org/VimWrapper

查阅具体的项目页面来了解如何在这些项目中使用 Vim。

一个替代的方法是使用通道，见 channel 。

本帮助页面的其余部分使用术语 "Vim 控制器" 来描述通过 NetBeans 套接字接口来控制 Vim 的程序。

关于 NetBeans IDE

NetBeans 是一个开源的集成开发环境，由 Sun Microsystems, Inc 和 netbeans.org 开发团体共同开发。刚开始只是一个 Java IDE，在最近的发行版本中， NetBeans 已经有 C、C++、和 Fortran 的支持。

要知道更多的信息，访问 NetBeans 的主网站 http://www.netbeans.org。
不幸地，外部编辑器现已宣布被废弃了。见 http://externaleditor.netbeans.org。

Sun Microsystems, Inc. 也以 Sun ONE Studio 的名字发行 NetBeans。访问 http://www.sun.com 可以了解更多关于 Sun ONE Studio 产品线的信息。

目前的 NetBeans 发行版本提供 Java 的完全支持和 C、C++ 以及 Fortran 的有限支持。目前的 Sun ONE Studio 发行版本提供 Java、C、C++ 和 Fortran 的完全支持。

---

2. 集成特性 netbeans-integration

Vim 的 NetBeans 套接字接口可以从 Vim 那里得到信息或者告诉 Vim 执行特定任务:
- 得到缓冲区的信息: 缓冲区名、光标位置、缓冲区内容，等等
- 缓冲区被打开或关闭时被告知
- 缓冲区内容修改时被告知
- 读入或保存文件
- 修改缓冲区内容
- 安装特别的键绑定
- 抬起 (raise) 窗口，控制窗口的尺寸

要发送键击到 Vim 以执行 Vim 的功能，必须使用 clientserver 接口。

---

3. 支持 NetBeans 的 Vim 配置 netbeans-configure

要得到安装 Vim 的更多帮助，请阅读 Vim 用户手册的 usr_90.txt 。

在 Unix 上:

不带参数的 configure 应该已经包含 NetBeans 接口，如果 configure 检查你的系统支持所需的功能的话。

如果你不想要 NetBeans 接口，你可以在 Makefile 里去掉包含 "--disable-netbeans" 的一行的注释。

目前，支持 Netbeans 接口的有控制台运行的 Vim 和运行以下 GUI 的 gvim: GTK、GNOME、Windows、Athena 和 Motif。

netbeans-xpm

如果需要 Motif 支持，用户应该提供 XPM 库。
支持 Motif 或 Athena 的 Vim 要显示图像，XPM 库是必需的。没有它，工具栏和标号 (sign) 特性会被关闭。

XPM 库由法国国立计算机科学与控制研究学院的 Arnaud Le Hors 提供，它可在 http://cgit.freedesktop.org/xorg/lib/libXpm 下载。撰写此文时，它的最新版本是 xpm-3.4k-solaris.tgz，这是一个 gzip 压缩的 tar 文件。如果你建立目录 /usr/local/xpm 然后在那里解包，就可以直接使用 Makefile 里去掉注释的行而无须修改。如果安装在别的目录，请修改 src/Makefile 里的 XPM_DIR。

在 MS-Windows 上:

Win32 的支持现在还处于实验阶段。

要使用 Win32 上的 XPM 标号，(例如使用 NetBeans 时)，你需要自己编译 XPM，或者使用预编译的库: http://iamphet.nm.ru/misc/ (MS Visual C++ 版本) 或 http://gnuwin32.sourceforge.net (MinGW 版本)。

允许调试:

要允许调试 Vim 和 NetBeans 协议，需要定义 "NBDEBUG" 宏。在相应平台的 Makefile 里查找 "NBDEBUG" 就可以知道需要去掉哪些行的注释。基本上就是给编译的命令加上 "-DNBDEBUG"。另见 netbeans-debugging

---

4. 错误消息 netbeans-messages

以下这些错误消息是 NetBeans 套接字协议特有的:

E463

Region is guarded, cannot modify
Vim 控制器定义了文本的保护区域，你不能编辑这个区域。
如果需要，设置当前缓冲区。

E532

The defineAnnoType highlighting color name is too long
defineAnnoType 命令的 "fg" 或 "bg" 颜色参数最大长度是 32 个字符。
2.5 版本新出现。

E656

Writes of unmodified buffers forbidden
不能写入 Vim 控制器打开的未修改的缓冲区。

E657

Partial writes disallowed
不允许部分写入 Vim 控制器打开的缓冲区。

E658

Connection lost for this buffer
Vim 控制器不清楚这个文件的目前状态。与其冒着破坏文件的风险，NetBeans 关闭了这个文件的连接。Vim 自己要负责修改和保存文件的改变，而 Vim 控制器将不会知道这些改变。

E744

Read-only file
Vim 通常允许修改只读文件，只有在写文件的时候坚持执行只读规则。不过，NetBeans 不允许你修改只读的文件，如果 Vim 这么做，会引起混淆。为此，Vim 不允许执行 NetBeans 模式时修改文件。

---

5. 从 NetBeans 模式里运行 Vim netbeans-run

有两种方式可以启动 Vim 的 NetBeans 模式:

+ IDE 可用 -nb 命令行参数启动 Vim
+ 在 Vim 内部，可用 :nbstart 命令启动 NetBeans

Vim 试图建立连接时使用 3 秒超时。

netbeans-parameters

有三种形式可以用来设置 NetBeans 连接参数。
在命令行上启动时， -nb 命令行参数可以是:

-nb={fname}	从文件读取
-nb:{hostname}:{addr}:{password}	直接
-nb	从文件或者环境变量里读取

在 Vim 内部启动时， :nbstart 可选参数可以是:

={fname}	从文件读取
:{hostname}:{addr}:{password}	直接
<无参数>	从文件或者环境变量里读取

E660 E668

在命令行上启动时，为了安全的原因，最好的方法是把信息写到只有用户能读得文件里。文件名字可以通过 "-nb={fname}" 的参数传递。或者，在不带的参数的 "-nb" 形式里，由环境变量 "__NETBEANS_CONINFO" 指定。文件里必须包含以下三行内容，顺序随意:

```
host={hostname}
port={addr}
auth={password}
```

其他的行被忽略。Vim 控制器须自己负责在不用的时候删除这个文件。

{hostname} 指定 Vim 控制器运行的机器名。如果不提供，则取环境变量 "__NETBEANS_HOST" 的值，如果还没有，缺省值是 "localhost"。

{addr} 指定 NetBeans 协议所用的端口号。如果不提供，则取环境变量 "__NETBEANS_SOCKET" 的值，如果还没有，缺省值是 3219。

{password} 指定连接到 NetBeans 所用的密码。如果不提供，则取环境变量 "__NETBEANS_VIM_PASSWORD" 的值，如果还没有，缺省值是 "changeme"。

启动时，Vim 会建立套接字连接（客户端）到指定的终端和端口。密码会在连接建立后使用 AUTH 事件发送。

---

6. NetBeans 协议 netbeans-protocol

Vim 控制器和 Vim 的通信使用简单文本消息。该协议原先是为 NetBeans 的外部编辑器所设计的。后来它被扩展，使得 Agide (A-A-P 的 GUI IDE，见 http://www.a-a-p.org) 也能使用，然后又扩展到其它 IDE。该扩展被标为 "2.1 版本"。

该协议的 2.2 版本有若干小的改动，这些改动应该只影响 NetBeans 用户（而不是 Agide 用户）。不过，一个修正的漏洞可能会引起混淆：netbeans_saved() 函数以前发送一个 "save" 协议命令。在 2.1 版本和以前，它被不正确地解读为写操作已发生的通知。实际上，它告诉 NetBeans 保存文件，所以多个写操作可能已经发生了。这个错误引起了若干问题，而 2.2 已修正了这个错误。为了减少这个混淆再次发生的可能性，netbeans_saved() 已经被改名为 netbeans_save_buffer()。

我们现在已经使用 2.5 版本。2.4 和 2.5 版本间的差异可以在下面搜索 "2.5"。

目前，消息通过套接字传送。因为消息是简单的 UTF-8 文本，该协议也许可以用其它通信机制完成。

在 Vim 空闲等待用户输入时处理 Netbeans 消息。Vim 运行在非交互模式时，如执行 Vim 脚本的自动测试中，调用空闲循环的次数可能过少。此时，可以调用 :sleep 命令。:sleep 命令会启动 Netbeans 消息处理。

6.1 消息类型 nb-messages
6.2 术语 nb-terms
6.3 命令 nb-commands
6.4 函数和应答 nb-functions
6.5 事件 nb-events
6.6 特殊消息 nb-special
6.7 协议错误信息 nb-protocol_errors

6.1 消息类型 nb-messages

有四类消息：

类型	方向	注释
命令	IDE -> 编辑器	不需要应答
函数	IDE -> 编辑器	编辑器必须应答
应答	编辑器 -> IDE	只用于函数的应答
事件	编辑器 -> IDE	不需要应答

每条消息是换行符终止的一行文本。参数以单个空格分隔。每个消息的第一项由消息的类

型决定：

类型	第一项	示例
命令	bufID:name!seqno	11:showBalloon!123 "text"
函数	bufID:name/seqno	11:getLength/123
应答	seqno	123 5000
事件	bufID:name=seqno	11:keyCommand=123 "S-F2"

6.2 术语 nb-terms

bufID
: 缓冲区号。每条消息可以指定一个特定的缓冲区，也可以不指定任何缓冲区（泛指）。泛指的消息使用为零的 bufID。注意：缓冲区号是 IDE 分配的。这不是 Vim 的缓冲区号。bufID 应该是从一开始顺序递增的数字。'switchbuf' 设为 "usetab" 而当前标签页找不到 "bufID" 缓冲区时，netbeans 设置当缓冲区为当前缓冲区的命令和函数会跳转到首个其它标签页中包含此缓冲区的打开的窗口，而不是替代当前窗口所在的缓冲区。

seqno
: IDE 用序列号标示命令和函数。应答需要指明相关联的函数的序列号。事件可以使用为零的序列号（也可以用最近一次收到的命令和函数的序列号）。

string
: 双引号括起的参数。文本使用 UTF-8 编码。这也意味着 ASCII 字符可以原封不动的使用。一些特殊字符用反斜杠的形式表示：

\"	双引号
\n	换行
\r	回车
\t	制表（可选，也可以直接表示）
\\	反斜杠

  NUL 字节是不允许的！

boolean
: 可取两个可能值的参数：

T	真
F	假

number
: 十进制的参数。

color
: 参数，或为十进制数字，或为 "none"（没有引号），或为在 highlight-ctermfg 和 gui-colors 中定义的色彩名（没有引号），
  2.5 版本新出现。

offset
: 数值参数，表示缓冲区的一个字节位置（偏移量）。第一个字节的偏移量为 0。行尾的字节计数决定于在文件里如何表示（CR/LF 是两个字节）。注意一个多字节的字符的字节计数可能多于一。

lnum/col
: 参数，表示行号和列号。行号从一开始，列号以字节位置表示，从零开始。注意一个多字节的字符可能跨越多列。

pathname
: 字符串参数：带完整路径的文件名。

actionMenuItem 没有实现。

actionSensitivity
没有实现。

addAnno serNum typeNum off len
在缓冲区里加上评注 (annotation)。
参数:

serNum	number	该评注的序列号，删除该评注时可以用到
typeNum	number	用 defineAnnoType 定义的该缓冲区的评注类型序列号
off	number	评注所加的位置
len	number	没有用到

在 2.1 版本中，"lnum/col" 可以用来代替 "off"。

balloonResult text
没有实现。

close 关闭缓冲区。这会使得我们失去当前的缓冲区。使用时要小心!

create 建立一个没有名字的缓冲区。替换当前的缓冲区 (改变的时候，它是隐藏的)。
这是 Vim 控制器打开文件时使用的第一个命令。命令序列应该是:

create	
setCaretListener	(忽略)
setModified	(没有影响)
setContentType	(忽略)
startDocumentListen	
setTitle	
setFullName	

defineAnnoType typeNum typeName tooltip glyphFile fg bg
定义该缓冲区的一种评注类型。
参数:

typeNum	number	序列号 (这里没有用到)
typeName	string	标识这个评注的名字
tooltip	string	没有用到
glyphFile	string	图标文件名
fg	color	高亮行前景色
bg	color	高亮行背景色

Vim 会定义评注所用的标号。
如果 color 为数值，解释为 "#rrggbb"，即红绿蓝色彩值 (见 gui-colors)，高亮只在 gVim 有定义。
如果 color 为名字，该颜色在带色彩的终端和 gVim 都有定义。
当 "fg" 和 "bg" 都是 "none" 的时候，不使用高亮行 (2.1 版本新出现)。
当 "glyphFile" 为空时，不使用文本标号 (2.1 版本新出现)。
但 "glyphFile" 为一到两个字符长时，使用文本标号 (2.1 版本新出现)。
注意: 评注会按顺序定义，而其序列号以后可以在 addAnno 里用到。

editFile pathname

设置缓冲区的名字为字符串参数 pathname，并编辑该参数指定的文件。
这是通常 IDE 告诉编辑器编辑一个文件的方法。

使用该命令，你必须设置 bufId 为非零以把该 bufId 赋给缓冲区。它会触发 bufId 为零的 fileOpened 事件，但缓冲区已经分配好了。

如果 IDE 要把文件内容直接传递给编辑器，那么应该用这些命令：

```
setFullName
insert
initDone
```

2.1 版本新出现。

enableBalloonEval
没有实现。

endAtomic 结束一个原子操作。"startAtomic" 和 "endAtomic" 之间的所有操作可以一次全部撤销。不过，这还没有实现。如果需要，重画屏幕。

guard off len
标识缓冲区的一片区域为保护区域。这意味着那里不能被编辑。"off" 和 "len" 是两个数值，它们界定保护文本的范围。

initDone 标识这个缓冲区可用。这也隐含地意味着它成为当前缓冲区。激活 BufReadPost 自动事件。

insertDone
Vim 控制器发送，告知 Vim 完成了初始的文件插入。这使得读入信息被显示。2.3 版本之前，打开文件之后不会显示读入消息。
2.3 版本新出现。

moveAnnoToFront serNum
没有实现。

netbeansBuffer isNetbeansBuffer
如果 "isNetbeansBuffer" 为 "T"，那么该缓冲区为 NetBeans 所 "拥有"。2.2 版本新出现。

putBufferNumber pathname
把一个缓冲区号和 Vim 和以字符串参数 "pathname" 命名的 Vim 缓冲区联系起来。它用于编辑器向 IDE 报告它要编辑另外一个文件，而 IDE 需要告诉编辑器它为这个文件分配的缓冲区号。同时，这个命令也把缓冲区标为已初始化。
2.1 版本新出现。

raise 把编辑器带到前台。
仅用于 GUI 版本的 Vim。
2.1 版本新出现。

removeAnno serNum
删除以前这个缓冲区里放置的评注。"serNum" 是 addAnno 使用的相同的数字。

save 如果缓冲区已修改，就保存之。接口的另一侧应该写入缓冲区，把缓冲

区标为已修改 ('setModified')，并复位缓冲区的 "changed" 标志位。
如果以下条件有一为真，则跳过写入的步骤：
- 'write' 没有置位
- 缓冲区只读
- 缓冲区没有文件名
- 'buftype' 不允许写入
2.2 版本新出现。

saveDone
Vim 控制器发送，告知 Vim 保存已经完成。这使得保存消息被显示。
2.3 版本之前，保存后不显示保存消息。
2.3 版本新出现。

setAsUser 没有实现。

setBufferNumber pathname
把一个缓冲区号和以字符串参数 "pathname" 命名的 Vim 缓冲区联系起来。它用于编辑器向 IDE 报告它要编辑另外一个文件，而 IDE 需要告诉编辑器它为这个文件分配的缓冲区号。它还有把这个缓冲区设为当前缓冲区的副作用。
参见 "putBufferedBumber"，它也许更有用。

setContentType
没有实现。

setDot off 使得缓冲区成为当前缓冲区，并设置光标到相应的位置。如果缓冲区在别的窗口里打开，使得该窗口成为当前窗口。
折叠会相应打开，使得光标行可见。
2.1 版本里 "lnum/col" 可以用来替代 "off"。

setExitDelay seconds
设置退出延迟为 seconds 秒，second 为数值参数。
该延迟用于给 IDE 在真正退出之前处理某些事情的时间。缺省的延迟是 2 秒。
2.1 版本新出现。
2.3 版本已废弃。

setFullName pathname
为缓冲区的文件名设为字符串参数 "pathname"。
在 IDE 想编辑 IDE 控制的文件时使用该命令。这使得该缓冲区成为当前缓冲区，但不从该文件读。"insert" 命令可以用来随后设置它的内容。

setLocAndSize 没有实现。

setMark 没有实现。

setModified modified
如果布尔参数 "modified" 为 "T"，把缓冲区标为已修改，如果为 "F"，则标为未修改。

setModtime time
在 Vim 控制器自己保存文件后，更新缓冲区的修改时间。

2.3 版本新出现。

setReadOnly
设置文件为只读
2.3 版本已经实现。

setStyle 没有实现。

setTitle name
把缓冲区的标题改为字符串参数 "name"。标题用于 Vim 控制器，而不是 Vim。

setVisible visible
如果布尔参数 "visible" 为 "T"，转到这个缓冲区。
"F" 参数不做任何事。

showBalloon text
在鼠标当前位置显示一个气泡（弹出窗口），显示文本 "text"，一个字符串表达式。气泡在鼠标移动若干像素点后自动消失。
仅用于 GUI 版本的 Vim。
2.1 版本新出现。

specialKeys
映射一组键（最常见的是功能键），使之可以被传递回 Vim 控制器进行处理。这使得 IDE 常规的热键可以从 Vim 处调用。
2.3 版本已经实现。

startAtomic 开始一个原子操作。屏幕不更新，直到遇到 "endAtomic" 为止。

startCaretListen
没有实现。

startDocumentListen
标识缓冲区，使之向 IDE 报告 "insert" 和 "remove" 事件。缺省就是报告这些改变。

stopCaretListen
没有实现。

stopDocumentListen
标识缓冲区，使之停止向 IDE 报告改变。
和 startDocumentListen 相反。
注意：如果用过 "netbeansBuffer" 把本缓冲区标注为 NetBeans 缓冲区，那么 Vim 就删除该缓冲区。这是为了和 Sun Stdio 10 兼容。

unguard off len
和 "guard" 相反，撤销文本区域的保护。
如有必要，同时设置当前缓冲区。

version 没有实现。

6.4 函数和应答 nb-functions

getDot　　没有实现。

getCursor　　返回当前缓冲区和光标位置。
应答是：

```
        seqno bufID lnum col off
```

seqno = 该函数的序列号
bufID = 当前缓冲区的号码（如果未知，返回 -1）
lnum = 当前光标的行号（第一行为 1）
col = 当前光标的列号（字节数，从 0 开始）
off = 缓冲区当前光标的位移（以字节数计算）
2.1 版本新出现。

getLength　　返回缓冲区的字节计算的长度。
如果缓冲区有 5000 个字节，那么返回的可能是：

```
        123 5000
```

TODO：解释部分行的使用情况。

getMark　　没有实现。

getAnno serNum
返回缓冲区中评注所在的行号。
参数：

serNum	该放置评注的序列号

应答是：

123 lnum	评注所在的行号
123 0	非法的评注序列号

2.4 版本新出现。

getModified　　如果指定一个缓冲区：返回 0 如果缓冲区没有改动，反之为 1。
如果没有指定缓冲区（缓冲区号为 0）：返回有改动的缓冲区的数目。
如果结果为 0，那么 Vim 就可以安全退出了。
2.1 版本新出现。

getText　　返回缓冲区的内容，以字符串形式表示。
考虑一个只有两行的缓冲区的例子，应答可能是：

```
        123 "first line\nsecond line\n"
```

注意：文档指出应该有位移和长度两个参数。但是这没有实现。

insert off text
在位置 "off" 之前插入文本 "text"。"text" 是字符串参数，"off" 是个数值。
"text" 必须在每行结尾处包括 "\n"（新行），或 'fileformat' 为 "dos" 时 "\r\n"。如果在空缓冲区上使用 "insert"，Vim 会自动相应地设置 'fileformat'。
如果 "off" 指向行首，文本插在此行之上。这样，如果 "off" 为零，文本行被插到第一行之前。
如果 "off" 指向行的开始位置之后，包括行尾的 NUL 处，那么文本的第一行附加于此行之后。其余的行出现于其后。
可能的应答：

123	没有问题
123 !message	失败

注意　回答里的消息不用引号括起。

如有必要，同时设置当前缓冲区。
不把光标移动到修改过的文本上。
复位撤销相关信息。

remove off length
从位置 "off" 开始删除 "length" 长度字节的文本。两个参数都是数值。
可能的应答:

123	没有问题
123 !message	失败

注意 回答里的消息不用引号括起。
如有必要，同时设置当前缓冲区。

saveAndExit 完成关闭 Vim 等价的操作: ":confirm qall"。如果文件内容没有改动，或者用户没有取消 Vim 退出的操作，不返回没有任何结果。IDE 认为成功的连接关闭代表成功的结果。如果用户取消该操作，尚存的修改过的缓冲区被作为结果返回，Vim 也不会退出。
2.1 版本新出现。

6.5 事件 nb-events

balloonEval off len type
鼠标在文本上短暂停留。如果 "len" 为零，没有选择区域，光标在 "off" 位置上。如果 "len" 非零，从 "off" 到 "off" + "len" 的区域被选择。
只有在该缓冲区设置了 "enableBalloonEval" 以后才会发送。"type" 尚未有定义。
尚未实现。

balloonText text
如果设置了 'balooneval'，鼠标在某段文字上出现。"text" 是字符串，鼠标下的文本。
仅用于 GUI 版本的 Vim。
2.1 版本新出现。

buttonRelease button lnum col
报告按了那个鼠标键，和释放时光标的位置。只有对 Vim 控制器拥有的缓冲区有效。如果鼠标键在释放时在状态行或者分隔行时，该事件不会被发送。如果 col 小于 1，鼠标键释放的位置在前导的标号栏。
2.2 版本新出现。

disconnect
报告 Vim 控制器 Vim 在退出，不会尝试读写更多的命令。
2.3 版本新出现。

fileClosed 没有实现。

fileModified 没有实现。

fileOpened pathname open modified
用户打开某个文件。
参数:

pathname	字符串	文件名

open	布尔	总是 "T"
modified	布尔	总是 "F"

geometry cols rows x y
报告编辑器窗口的大小和位置。
参数:

cols	数值	文本列数
rows	数值	文本行数
x	数值	光标像素点横坐标
y	数值	光标像素点纵坐标

只适用于 Motif。

insert off text
Vim 里在 "off" 位置插入了文本 "text"。只有在允许的时候才发送。见 "startDocumentListen"。

invokeAction 没有实现。

keyCommand keyName
报告按了某个特殊键，键名是 "keyName"，是个字符串。
支持的键名:

F1	功能键 1
F2	功能键 2
...	
F12	功能键 12
' '	空格 (没有引号)
!	感叹号
...	任何其他 ASCII 可显示字符
~	波浪号
X	不能识别的键

键名前面可能有 "C"、"S" 和/或 "M"，分别代表 Control、Shift 和 Meta (Alt) 修饰符。如果有的话，修饰符和键名以连字符分隔。例如: "C-F2"。
ASCII 字符是 2.1 版本新出现的。

keyAtPos keyName lnum/col
类似于 "keyCommand"，同时报告光标所在的行号和列号。
2.1 版本新出现。

killed 用户删除或者清除了一个文件，而缓冲区的评注也已经被删除。此缓冲区的 bufID 已经无效。只有对 IDE 已经给出 bufID 的文件才有效。

newDotAndMark off off
报告光标在缓冲区的 "off" 字节位置。只有在 "keyCommand" 事件之前发送。

quit 没有实现。

remove off len
从 "off" 位置开始长度为 "len" 的文本在 Vim 里被删除。只有在允许的时候才发送。见 "startDocumentListen"。

revert　　没有实现。

save　　缓冲区被保存，现在已处于没有修改的状态。只有在允许的时候才发送。见 "startDocumentListen"。

startupDone　　编辑器完成启动工作，可以开始编辑文件。
2.1 版本新出现。

unmodified　　缓冲区现在处于未修改状态。只有在允许的时候才发送。见 "startDocumentListen"。

version vers　　报告接口实现的版本号。Vim 报告 "2.4" (包括引号)。

6.6 特殊消息　　nb-special

这些消息不遵循以上说明的消息格式。它们仍由换行符终止。

ACCEPT　　不用。

AUTH password　　编辑器 -> IDE: 编辑器发送给 IDE 的第一个消息。它提供套接字服务器所需的密码。该密码由 -nb 参数提供。不需要引号！

DISCONNECT　　IDE -> 编辑器: 终止连接。编辑器将会退出。IDE 应该只有在没有未保存的改变的时候才发出此消息。

DETACH　　IDE -> 编辑器: 终止连接，不退出编辑器。用于当 IDE 终止但不想关闭编辑器的场合。2.1 版本新的特性。

REJECT　　不用。

6.7 协议错误信息　　nb-protocol_errors

这些错误发生意味着某个消息违反了协议。
E627 E628 E629 E632 E633 E634 E635 E636
E637 E638 E639 E640 E641 E642 E643 E644 E645 E646
E647 E648 E649 E650 E651 E652

---

7. NetBeans 命令　　netbeans-commands

:nbstart E511 E838

:nbs[tart] {connection}　使用 {connection} 作为套接字连接参数，启动新的 NetBeans 会话。{connection} 的格式见 netbeans-parameters 。在任何时候，用户可以用以下命令检查 netbeans 套接字是否已连接:
':echo has("netbeans_enabled")'

:nbclose

:nbc[lose]　关闭当前 NetBeans 会话。删除所有放置的标号。

:nbkey

:nb[key] {key}　　把 {key} 传递给 Vim 控制器进行处理。用 specialKeys 命令安装了热键后，此命令用来生成热键的相应信息发给 Vim 控制器。
此命令也可用来给 Vim 控制器传送任何文本。例如，Pyclewn 用它来生成以 Vim 用户命令形式出现的完整的 gdb 命令集。生成的依次是 newDotAndMark、keyCommand 和 keyAtPos 事件。

---

8. 已知问题　　netbeans-problems

不可能有 NUL 字节。编辑器到 IDE 的转换使得它们成为 NL 字符。反之，IDE -> 编辑器却不能插入它们。

NetBeans 会话可以在终端运行的 Vim 中启动，然后在运行了 :gui 命令后切换到 GUI 环境中继续使用。此时 NetBeans 评注定义的高亮可能在 ":gui" 命令执行的 .gvimrc 中被清除，因为该文件会载入 colorscheme，后者会执行命令 ":highlight clear"。2.5 版本新出现。

---

9. 调试 NetBeans 协议　　netbeans-debugging

要调试 Vim 协议，必须在编译 Vim 时加入调试支持和 NetBeans 调试支持。关于 Vim 编译和如果打开调试支持的操作，请见 netbeans-configure 。

运行 Vim 时，设置以下环境变量:

```
export SPRO_GVIM_DEBUG=netbeans.log
export SPRO_GVIM_DLEVEL=0xffffffff
```

Vim 就会把所有收到和发送的 NetBeans 协议信息记录到 netbeans.log 文件中。

一个会话后，netbeans.log 的内容大致如下:

```
Tue May 20 17:19:27 2008
EVT: 0:startupDone=0
CMD 1: (1) create
CMD 2: (1) setTitle "testfile1.txt"
CMD 3: (1) setFullName "testfile1.txt"
EVT(suppressed): 1:remove=3 0 -1
EVT: 1:fileOpened=0 "d:\\work\\vimWrapper\\vimWrapper2\\pyvimwrapper\\tests\\testfile1.txt" T F
CMD 4: (1) initDone
FUN 5: (0) getCursor
REP 5: 1 1 0 0
CMD 6: (2) create
CMD 7: (2) setTitle "testfile2.txt"
CMD 8: (2) setFullName "testfile2.txt"
EVT(suppressed): 2:remove=8 0 -1
EVT: 2:fileOpened=0 "d:\\work\\vimWrapper\\vimWrapper2\\pyvimwrapper\\tests\\testfile2.txt" T F
CMD 9: (2) initDone
```

10. NetBeans 外部编辑器

注 意：以下信息已经过时！只对还在使用旧版的 NetBeans 的人有用。

10.1. 下载 NetBeans netbeans-download

NetBeans IDE 可以从 netbeans.org 下载。你可以下载发行版，源代码，或者用 CVS 得到当前的源代码树。如果你选择下载源代码，遵照 netbeans.org 的指示来编译 NetBeans。

视乎你下载的 NetBeans 版本，你可能需要更多的工作：下载所需的外部编辑器模块。这是使 NetBeans 支持 gvim (或者 xemacs :-)) 的模块。如果你的 NetBeans 不带之，参见 http://externaleditor.netbeans.org 了解如何下载该模块的详情。

关于 C、C++ 和 Fortran 的支持，你需要得到 cpp 模块。关于这个模块的信息，参见 http://cpp.netbeans.org。

你已可以从 Sun Microsystems, Inc 下载 30 天免费测试的 Sun ONE Studio。详情见 http://www.sun.com。

10.2. NetBeans 键盘映射 netbeans-keybindings

Vim 理解执行 NetBeans 命令的键盘映射。它们通常是功能键的组合。要执行某个 NetBeans 命令，用户必须按 Pause 键，再按 NetBeans 的键盘映射。比如，要编译 Java 文件，NetBeans 的键盘映射是 "F9"。所以，在 vim 里，按 "Pause F9" 起到编译 java 文件的功能。要切换当前行的断点状态，按 "Pause Shift F8"。

Pause 键是功能键 21。如果你没有可用的 Pause 键，而想用 F8 代替，用：

```
:map <F8> <F21>
```

外部编辑器模块动态地读取 NetBeans 的键盘映射，所以 vim 应该总能得到最新的键盘映射，即使在 NetBeans 改变它们以后。

10.3. 为 Vim 准备 NetBeans netbeans-preparation

要使得 NetBeans 支持 vim，NetBeans 外部编辑器模块必须已经调入并且能使用。如果你使用 Sun ONE Studio Enterprise Edition，那么这一模块应该已经能用了。如果你用的是 NetBeans 发行版，你需要自己去找这个开源的模块。

你可以检查该模块是不是已经存在。打开 Tools->Options 对话框，找到 "Modules" 列表 (IDE Configuration->System->Modules)。如果该列表包含 "External Editor"，检查一下它是不是被许可使用了 ("Enabled" 属性应该为 "True")。如果你的列表里没有这一项，见下一节 obtaining-exted 。

10.4. 取得外部编辑器模块 obtaining-exted

有两个方法可以取得外部编辑器模块。最简单的方式是使用 NetBeans 更新中心，下载并安装该模块。不幸的是，有的版本的更新中心找不到这个模块。如果这样不行，你需要自

己下载源代码并编译该模块。我会试图使这个模块出现在 NetBeans 更新中心的，这样自己编译就不需要了。也可查看 http://externaleditor.netbeans.org，看看有无取得的方法。

要通过 CVS 下载外部编辑器的源码并自己编译，见 http://externaleditor.netbeans.org 和 http://www.netbeans.org。不幸的是，这个方法并不是很简单。

10.5. 设置 NetBeans 以运行 Vim netbeans-setup

假设你已经根据 netbeans-preparation 调入和允许使用 NetBeans 外部编辑器模块，你只需要检查一下 gvim 命令行已经为你的环境正确配置好了。

打开 Tools->Options 对话框，打开 Editing 种类，选择 External Editor。右边的窗格应该包含 Properties 和 Expert 页。在 Properties 页里，确保 "Editor Type" 设为 "Vim"。在 Expert 页里确保 "Vim Command" 是正确的。

如果你要改变 "Vim Command"，要小心。必须提供有些命令选项，以便正确的建立连接。你可以改变命令名字本身，但也就那么多了。如果你的 gvim 可以在 $PATH 里找到，那么 Vim Command 就可以用 "gvim" 开始。如果你不想从 $PATH 里搜索 gvim，那么就提供完整的路径名。这样，你就可以对 NetBeans 打开的任何源代码使用 gvim 编辑了。

如果有些文件在 gvim 里打开，有些 (有着不同后缀) 则使用缺省的 NetBeans 编辑器，那你就需要检查 Expert 页的 MIME Type 属性。NetBeans 面向 MIME 类型，而外部编辑器只会使用这个属性所指定的 MIME 类型。

vim:tw=78:ts=8:noet:ft=help:norl:

sign.txt　　For Vim version 8.2.　最近更新：2020年1月

VIM 参考手册　by Gordon Prieur
and Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

特性：标号支持　　sign-support

1. 简介　　sign-intro
2. 命令　　sign-commands
3. 函数　　sign-functions-details

{仅当编译时加入 +signs 特性才可用}

---

1. 简介　　sign-intro　signs

当调试器或者其他集成开发环境的工具控制编辑器时，它需要通过给出某些特定的高亮以给用户一些关于文件有用信息的反馈。一个例子是调试器在左边的列中给出图标来表示这里有一个断点。另一个可能的例子是用一个箭头代表当前程序计数器 PC 的位置。标号特性既包括在窗口的左边放置标号或者图标的支持，也包括该行使用的高亮的定义。显示图像化的标号可能只适用于 gvim（虽然 Sun Microsystem 的 dtterm 也支持，它是我所知的唯一支持的终端模拟器）。但文字化的标号和高亮应该可用于任何彩色终端模拟器。

标号和相应的高亮并不仅对调试器有用。Sun 的 Visual WorkShop 用标号和高亮来标识错误和源程序浏览器的命中位置。它的调试器也支持 8 到 10 种不同的标号和高亮色。Netbeans 也是如此。

应用标号的过程分两步进行：

1. 定义标号。这里指定图像，文字和高亮设定。例如，你可以定义 "break" 标号为路边的停止符号和文字 "!!"。

2. 放置标号。这里指定标号显示的文件和行号。每个定义好的标号可以在不同的文件和行号上多次放置。

sign-column
如果文件里定义了标号，Vim 自动增加两个字符的纵列来显示标号。撤销最后一个标号之后，该列又会自动消失。'signcolumn' 选项可改变此行为。

该列的颜色由 SignColumn 高亮组定义 hl-SignColumn 。一个设定该颜色的示例：

```
:highlight SignColumn guibg=darkgrey
```

sign-identifier
已放置的标号以一个数值识别，称为标号标识符。此标识符用于跳转到标号或删除标号。标识符在用 :sign-place 命令或 sign_place() 函数放置标号时分配。每个标号标识符应是唯一的数值。如果多个已放置的标号使用相同的标识符，标号的跳转或删除将不可预测。为了避免标识符重复，可用标号组。 sign_place() 函数调用时可提供零标号标识符来分配下个可用的标识符。

sign-group

每个已放置的标号可在全局组或某命名组中分配。放置标号时，如果不提供组名或组名为空串时，标号放置在全局组。否则标号放置在命名组。标号标识符在每个组里是唯一的。标号组允许 Vim 插件使用唯一的标号，而不会为其它使用标号的插件相互干扰。

要在弹出窗口中放置标号，组名必须以 "PopUp" 开始。其它标号不能在弹出窗口中显示。组名 "PopUpMenu" 用于 'cursorline' 置位的弹出窗口。

sign-priority

每个已放置的标号会分配一个优先值。同一行上如有多个标号放置，使用最高优先级的标号的属性，这与标号组无关。标号的缺省优先级是 10。优先级在标号放置时分配。

标号放置的行如被删除，移动标号到下一行（如果没有下一行则为缓冲区的末行）。删除如被撤销，标号并不移回。

---

2. 命令

sign-commands :sig :sign

这里提供了一例，在当前文件的第 23 行显示 "piet" 标号，以文字 ">>" 标明。

```
:sign define piet text=>> texthl=Search
:exe ":sign place 2 line=23 name=piet file=" . expand("%:p")
```

删除标号的命令：

```
:sign unplace 2
```

注意 ":sign" 命令后面不能跟其它命令或者注释。如果需要，使用 :execute 命令。

定义标号

:sign-define E255 E160 E612

见 sign_define() 给出等价的 Vim 脚本函数。

:sign define {name} {argument}...
定义新标号或为已存在的标号设置属性。{name} 可以是完全由数位组成的数值，也可以是由非数位开头的名字。忽略开头的零，所以 "0012"、"012" 和 "12" 认为是相同的名字。
可以定义大约 120 个不同的标号。

允许的参数是：

icon={bitmap}
位图文件名。必须是完整的路径名。位图应该能够在两个字符的宽度内显示。但是不会实际检查这一点，如果位图太大，刷新时会有问题。只有 GTK 2 可以按比例缩放位图，使之刚好能占据可用的空间。

工具包	支持
GTK 1	pixmap (.xpm)
GTK 2	许多
Motif	pixmap (.xpm)
Win32	.bmp, .ico, .cur
	pixmap (.xpm) +xpm_w32

linehl={group}
用以显示整个标号行的高亮组。多用于定义背景色。

text={text} E239
没有图标或者不用 GUI 时显示的文本。只允许可显示字符，而且必须占据一到两个显示单元。

texthl={group}
显示文本项目的高亮组。

删除标号 :sign-undefine E155

见 sign_undefine() 给出等价的 Vim 脚本函数。

:sign undefine {name}
删除标号的定义。如果 {name} 标号已经放置，会有问题。

列出标号 :sign-list E156

见 sign_getdefined() 给出等价的 Vim 脚本函数。

:sign list 列出所有的标号和它们的属性。

:sign list {name}
列出单个标号和他的属性。

放置标号 :sign-place E158

见 sign_place() 给出等价的 Vim 脚本函数。

:sign place {id} line={lnum} name={name} file={fname}
把 {name} 指定的标号放在文件 {fname} 的第 {lnum} 行。 :sign-fname
文件 {fname} 必须已经在某个缓冲区调入，而且必须使用准确的文件名。通配符，$ENV 和 ~ 不会被扩展，空白不会被转义。结尾的空白则会忽略。

用 {id} 记住这个标号，将来可作进一步应用。{id} 必须是数值。用户可以自行决定 {id} 在每个文件内是否唯一。(如果多次使用，撤销放置就必须做多次，而且进行修改时未必尽如预期)。

以下可选的标号属性可在 "file=" 前指定:
group={group} 把标号放置在标号组 {group}
priority={prio} 给标号分配优先级 {prio}

缺省，标号放置在全局标号组。

缺省，标号分配的缺省优先级为 10。要分配不同的优先值，用 "priority={prio}" 来指定。优先级用于在同一行有多个标号放置时决定显示的标号。

示例:
```
:sign place 5 line=3 name=sign1 file=a.py
:sign place 6 group=g2 line=2 name=sign2 file=x.py
```

```
:sign place 9 group=g2 priority=50 line=5
                                        \ name=sign1 file=a.py
```

:sign place {id} line={lnum} name={name} [buffer={nr}]
同上，但指定缓冲区 {nr}。如果不给出缓冲区参数，标号放置在当前缓冲区。

E885

:sign place {id} name={name} file={fname}
把文件 {fname} 里用 {id} 记住的标号换成 {name} 指定的新标号。参看关于 {fname} 的说明 :sign-fname 。可用于不移动某个标号的同时，更改它的显示方式（例如，当调试器停在某个断点时）。

可选的 "group={group}" 属性可在 "file=" 前使用来选择特定组的标号。可选的 "priority={prio}" 属性可用来发来已有标号的优先级。

:sign place {id} name={name} [buffer={nr}]
同上，但指定缓冲区 {nr}。如果不给出缓冲区参数，使用当前缓冲区。

撤销放置标号

:sign-unplace E159

见 sign_unplace() 给出等价的 Vim 脚本函数。

:sign unplace {id} file={fname}
从文件 {fname} 里撤销已放置的标号 {id} 的放置。参看关于 {fname} 的说明 :sign-fname 。

:sign unplace {id} group={group} file={fname}
同上，但撤销标号组 {group} 里的标号 {id}。

:sign unplace {id} group=* file={fname}
同上，但摊销所有标号组里的标号 {id}。

:sign unplace * file={fname}
从文件 {fname} 里撤销所有标号的放置。

:sign unplace * group={group} file={fname}
从文件 {fname} 里撤销组 {group} 里所有标号的放置。

:sign unplace * group=* file={fname}
从文件 {fname} 里撤销所有组里所有标号的放置。

:sign unplace {id} buffer={nr}
从缓冲区 {nr} 里撤销已放置的标号 {id} 的放置。

:sign unplace {id} group={group} buffer={nr}
从缓冲区 {nr} 里撤销组 {group} 里已放置的标号 {id} 的放置。

:sign unplace {id} group=* buffer={nr}
从缓冲区 {nr} 里撤销所有组里已放置的标号 {id} 的放置。

:sign unplace * buffer={nr}

从缓冲区 {nr} 里撤销所有标号的放置。

:sign unplace * group={group} buffer={nr}
从缓冲区 {nr} 里撤销组 {group} 里所有标号的放置。

:sign unplace * group=* buffer={nr}
从缓冲区 {nr} 里撤销所有组里所有标号的放置。

:sign unplace {id}
撤销所有文件里出现的 {id} 标号的放置。

:sign unplace {id} group={group}
撤销所有文件里出现的组 {group} 里 {id} 标号的放置。

:sign unplace {id} group=*
撤销所有文件里出现的所有组里 {id} 标号的放置。

:sign unplace *
从所有文件撤销全局组里所有标号的放置。

:sign unplace * group={group}
从所有文件撤销组 {group} 里所有标号的放置。

:sign unplace * group=*
从所有文件撤销所有组里所有标号的放置。

:sign unplace
撤销光标所在标号的放置。如果同一行有多个标号，只删除一个。

:sign unplace group={group}
撤销组 {group} 里光标所在标号的放置。

:sign unplace group=*
撤销所有组里光标所在标号的放置。

列出放置标号 :sign-place-list

见 sign_getplaced() 给出等价的 Vim 脚本函数。

:sign place file={fname}
列出文件 {fname} 里所有放置的标号。
参看关于 {fname} 的说明 :sign-fname 。

:sign place group={group} file={fname}
列出文件 {fname} 里组 {group} 里所有放置的标号。

:sign place group=* file={fname}
列出文件 {fname} 里所有组里所有放置的标号。

:sign place buffer={nr}
列出缓冲区 {nr} 里所有放置的标号。

:sign place group={group} buffer={nr}

列出缓冲区 {nr} 里组 {group} 里所有放置的标号。

:sign place group=* buffer={nr}
列出缓冲区 {nr} 里所有组里所有放置的标号。

:sign place 列出所有文件里全局组里放置的标号。

:sign place group={group}
列出所有文件里组 {group} 里放置的标号。

:sign place group=*
列出所有文件里所有组里放置的标号。

跳 到 标 号 :sign-jump E157

见 sign_jump() 给出等价的 Vim 脚本函数。

:sign jump {id} file={fname}
打开文件 {fname} 或者跳转到包含 {fname} 的窗口并把光标设在标号 {id} 所在的位置。
参看关于 {fname} 的说明 :sign-fname 。
如果此文件不在窗口显示但当前文件又不能被放弃 abandon ，此动作失败。

:sign jump {id} group={group} file={fname}
同上，但跳转到组 {group} 里的标号

:sign jump {id} [buffer={nr}] E934
同上，但指定缓冲区 {nr}。如果缓冲区 {nr} 没有名字，报错。如果未给出缓冲区参数，使用当前缓冲区。

:sign jump {id} group={group} [buffer={nr}]
同上，但跳转到组 {group} 里的标号

---

3. Functions sign-functions-details

sign_define({name} [, {dict}]) sign_define()
sign_define({list})
定义名为 {name} 的新标号或为已存在的标号设置属性。类似于 :sign-define 命令。

给 {name} 加上独有的前缀以避免名字冲突。不像放置标号那样有 {group} 的区隔。

{name} 可以是字符串或数值。可选的 {dict} 参数指定标号的属性。支持以下的值:

icon	用于标号的位图文件完整路径。
linehl	用于标号放置所在的整行的高亮组。
text	没有图标或不使用 GUI 时显示的文本。
texthl	文本项目使用的高亮组

如果名为 {name} 的标号已存在，更新标号的属性。

单个参数 {list} 可用来定义一列标号。每个列表项是 {dict} 中上述项目的字典，此外还有 "name" 项目指定标号名。

成功时返回 0，失败时返回 -1。如果使用单个参数 {list}，返回值的列表，其中每个值对应一个定义的标号。

示例：

```
call sign_define("mySign", {
        \ "text" : "=>",
        \ "texthl" : "Error",
        \ "linehl" : "Search"})
call sign_define([
        \ {'name' : 'sign1',
        \  'text' : '=>'},
        \ {'name' : 'sign2',
        \  'text' : '!!'}
        \ ])
```

也可用作 method :

```
GetSignList()->sign_define()
```

sign_getdefined([{name}]) sign_getdefined()

获取已定义标号及其属性的列表。类似于 :sign-list 命令。

如果不给出 {name}，返回所有已定义标号的列表，否则返回指定标号的属性。

返回值的每个列表项是包含以下项目的字典：

icon	标号的位图文件完整路径
linehl	标号放置所在的整行的高亮组。
name	标号名
text	没有图标或不使用 GUI 时显示的文本。
texthl	文本项目使用的高亮组

如果没有标号或 {name} 找不到，返回空列表。

示例：

```
" 获取所有定义的标号的列表
echo sign_getdefined()

" 获取名为 mySign 的标号的属性
echo sign_getdefined("mySign")
```

也可用作 method :

```
GetSignList()->sign_getdefined()
```

sign_getplaced([{expr} [, {dict}]]) sign_getplaced()

返回一个或所有缓冲区里放置的标号的列表。类似于 :sign-place-list 命令。

如果给出可选的缓冲区名 {expr}，只返回该缓冲区里放置的标号列表。{expr} 的用法见 bufname() 。可选的 {dict} 可包含以下项目：

group	只选择此组内的标号

id 选择带此标识符的标号
lnum 选择放置在此行的标号。{lnum} 的用法见 line() 。

如果 {group} 为 '*'，返回包括全局组的所有组的标号。如果 {group} 未给出或为空串，只返回全局组里的标号。如果无参数，返回所有缓冲区里放置的全局组里的所有标号。见 sign-group 。

返回值的每个列表项目是带有以下项目的字典：

bufnr 带有此标号的缓冲区号
signs 放置在 {bufnr} 的标号列表。每个列表项目是带有下列项的字典

每个标号的字典包含以下项：

group 标号组。全局组设为 ''。
id 标号标识符
lnum 标号放置的行号
name 已定义的标号名
priority 标号优先级

缓冲区里返回的标号以行号和优先级排序。

失败或没有已放置的标号时返回空列表。

示例：

```
" 获取在 eval.c 里放置的全局组里的标号列表
echo sign_getplaced("eval.c")

" 获取在 eval.c 里放置的组 'g1' 里的标号列表
echo sign_getplaced("eval.c", {'group' : 'g1'})

" 获取在 eval.c 里放置在第 10 行的标号列表
echo sign_getplaced("eval.c", {'lnum' : 10})

" 获取在 a.py 里放置的标识符为 10 的标号
echo sign_getplaced("a.py", {'id' : 10})

" 获取在 a.py 里放置的组 'g1' 里带 id 20 的标号
echo sign_getplaced("a.py", {'group' : 'g1',
                        \  'id' : 20})

" 获取所有放置的标号的列表
echo sign_getplaced()
```

也可用作 method :

```
GetBufname()->sign_getplaced()
```

sign_jump()

sign_jump({id}, {group}, {expr})

打开缓冲区 {expr} 或跳转到包含 {expr} 的窗口，并定位光标到组 {group} 里的标号 {id}。类似于 :sign-jump 命令。

{expr} 的用法见 bufname() 。

返回标号的行号。如果参数非法返回 -1。

示例：

```
" 跳转到当前缓冲区的标号 10
call sign_jump(10, '', '')
```

也可用作 method ：

```
GetSignid()->sign_jump()
```

sign_place()

sign_place({id}, {group}, {name}, {expr} [, {dict}])

把 {name} 指定的标号放在文件 {fname} 或缓冲区 {expr} 的第 {lnum} 行，并给标号指定 {id} 和 {group}。类似于 :sign-place 命令。

如果标号标识符 {id} 为零，分配新的标识符。否则使用指定的数值。{group} 为标号组名。要指定全局标号组，使用空串。{group} 相当于 {id} 的命名空间，这样两个组就可有相同的 ID。详情参见 sign-identifier 和 sign-group 。

{name} 指定已定义的标号。
{expr} 指定缓冲区名或号。可接受的值见 bufname() 。

可选的 {dict} 参数支持以下项目：

lnum	放置标号的文件或缓冲区 {expr} 的行号。可接受的值见 line() 。
priority	标号优先级。详见 sign-priority 。

如果不给出可选的 {dict}，那么修改已放置的在组 {group} 里的标号 {id} 使用已定义的标号 {name}。

成功时返回标号标识符，失败时返回 -1。

示例：

```
" 放置名为 sign1 的标号在缓冲区 json.c 第 20 行带 id
" 为 5
call sign_place(5, '', 'sign1', 'json.c',
                                  \ {'lnum' : 20})

" 更新缓冲区 json.c 的标号 5 使用 sign2
call sign_place(5, '', 'sign2', 'json.c')

" 放置名为 sign3 的标号在缓冲区 json.c 第 30 行带新分
" 配的标识符
let id = sign_place(0, '', 'sign3', 'json.c',
                                  \ {'lnum' : 30})

" 放置名为 sign4 的标号在缓冲区 json.c 第 40 行带 id
" 为 10 且优先级为 90
call sign_place(10, 'g3', 'sign4', 'json.c',
                \ {'lnum' : 40, 'priority' : 90})
```

也可用作 method ：

```
GetSignid()->sign_place(group, name, expr)
```

sign_placelist()

sign_placelist({list})

放置一或多个标号。类似于 sign_place() 函数。{list} 参数指定要放置的一列标号。每个列表项目是带以下标号属性的字典：

buffer	缓冲区名或号。可接受的值见 bufname() 。
group	标号组名。{group} 相当于 {id} 的命名空间，这样两个组就可有相同的 ID。不指定或设为空串时则指定全局标号组，详见 sign-group 。
id	标号标识符。如果未指定或为零，分配新的唯一的标识符。否则使用指定的数值。详见 sign-identifier 。
lnum	放置标号的缓冲区 {expr} 中的行号。可接受值见 line() 。
name	放置的标号名。详见 sign_define() 。
priority	标号优先级。多个标号放置在同一行时，使用最高优先级的标号。如果不指定，缺省优先级是 10。详见 sign-priority 。

如果 {id} 指定已存在的桔，则修改已有的标号以使用指定的 {name} 和/或 {priority}。

返回标号标识符列表。如果放置某标号失败，对应的列表项设为 -1。

示例：

```
" 放置标号 s1 在缓冲区 a.c 的第 20 行带 id 5，还有第
" 30 行带 id 10
let [n1, n2] = sign_placelist([
        \ {'id' : 5,
        \  'name' : 's1',
        \  'buffer' : 'a.c',
        \  'lnum' : 20},
        \ {'id' : 10,
        \  'name' : 's1',
        \  'buffer' : 'a.c',
        \  'lnum' : 30}
        \ ])

" 放置标号 s1 在缓冲区 a.c 的第 40 和 50 行带自动生成
" 的标识符
let [n1, n2] = sign_placelist([
        \ {'name' : 's1',
        \  'buffer' : 'a.c',
        \  'lnum' : 40},
        \ {'name' : 's1',
        \  'buffer' : 'a.c',
        \  'lnum' : 50}
        \ ])
```

也可用作 method ：

```
GetSignlist()->sign_placelist()
```

sign_undefine([{name}])
sign_undefine({list})
sign_undefine()

删除之前定义的标号 {name}。类似于 :sign-undefine 命令。如果

不给出 {name}，删除所有定义的标号。

单个参数 {list} 可用来删除一列标号的定义。每个列表项是标号名。

成功时返回 0，失败时返回 -1。如果使用单个参数 {list}，返回值的列表，其中每个值对应一个删除定义的标号。

示例：

```
" 删除名为 mySign 的标号
call sign_undefine("mySign")

" 删除标号 'sign1' 和 'sign2'
call sign_undefine(["sign1", "sign2"])

" 删除所有标号
call sign_undefine()
```

也可用作 method：

```
GetSignlist()->sign_undefine()
```

sign_unplace({group} [, {dict}]) sign_unplace()

撤销在一或多个缓冲区里已放置的标号的放置。类似于 :sign-unplace 命令。

{group} 是标号组名。要使用全局标号组，使用空串。如果 {group} 设为 '*'，使用所有的组，包括全局组。
{group} 里标号的选择基于 {dict} 的项目。支持以下的 {dict} 可选项目：

buffer	缓冲区名或号。见 bufname()。
id	标号标识符

如果不给出 {dict}，删除 {group} 里的所有标号。

成功时返回 0，失败时返回 -1。

示例：

```
" 从缓冲区 a.vim 撤销标号 10
call sign_unplace('', {'buffer' : "a.vim", 'id' : 10})

" 从缓冲区 3 撤销 'g1' 组里的标号 20
call sign_unplace('g1', {'buffer' : 3, 'id' : 20})

" 从缓冲区 10 撤销 'g2' 组里的所有标号
call sign_unplace('g2', {'buffer' : 10})

" 从所有缓冲区撤销 'g3' 组里的标号 30
call sign_unplace('g3', {'id' : 30})

" 从缓冲区 5 撤销所有已放置的标号
call sign_unplace('*', {'buffer' : 5})

" 从所有缓冲区撤销 'g4' 组里的所有标号
call sign_unplace('g4')

" 从所有缓冲区撤销标号 40
```

```
call sign_unplace('*', {'id' : 40})

" 从所有缓冲区撤销所有已放置的标号
call sign_unplace('*')
```

也可用作 method :

```
GetSigngroup()->sign_unplace()
```

sign_unplacelist({list}) sign_unplacelist()

撤销在一或多个缓冲区里已放置的标号的放置。类似于 sign_unplace() 函数。

{list} 参数指定要撤销的标号的列表。每个列表项目是带有以下标号属性的字典:

buffer	缓冲区名或号。可接受的值见 bufname() 。如无指定，从所有缓冲区里撤销指定的标号。
group	标号组名。如无指定或设为空串，使用全局标号组，如果设为 '*'，使用所有的组，包括全局组。
id	标号标识符。如无指定，撤销指定组的所有标号。

返回列表，如果标号被成功撤销，对应项设为 0，如果失败设为 -1。

示例:

```
" 从缓冲区 a.vim 撤销带 id 10 的标号，从缓冲区 b.vim
" 撤销带 id 20 的标号
call sign_unplacelist([
        \ {'id' : 10, 'buffer' : "a.vim"},
        \ {'id' : 20, 'buffer' : 'b.vim'},
        \ ])
```

也可用作 method :

```
GetSignlist()->sign_unplacelist()
```

vim:tw=78:ts=8:noet:ft=help:norl:

vi_diff.txt For Vim version 8.2. 最近更新: 2020年1月

VIM 参考手册 by Bram Moolenaar

译者: Willis
http://vimcdoc.sf.net

Vim 和 Vi 的差异 vi-differences

此文件列出 Vim 和 Vi/Ex 的差异，并给出 Vim 有而 Vi 没有的功能的总览。

Vim 基本上遵守 POSIX 1003.2-1。唯一只知缺少的命令是 ":open"。还有众多小的差异(或者因为 Vim 的缺失，或者因为此处并不涉及 Posix)。

1. 模拟的命令 simulated-command
2. 缺失的选项 missing-options
3. 限制 limits
4. 最有意思的新增功能 vim-additions
5. 其它 Vim 特色 other-features
6. 支持的 Vi 特性 vi-features
7. 命令行参数 cmdline-arguments
8. POSIX 相容性 posix-compliance

---

1. 模拟的命令 simulated-command

该命令在 Vi 存在，但 Vim 只是对它进行模拟:

:o :op :open

:[range]o[pen] 类似于 :visual : 结束 Ex 模式。
{Vi: 在打开模式开始编辑}

:[range]o[pen] /pattern/ 同上，另外移动光标到光标行上匹配 "pattern" 的列。

Vim 不支持 open 模式，因为它其实没什么用。对于那些 ":open" 会启动 open 模式的情形，Vim 会离开 Ex 模式，你可以执行相同的命令，但刷新的是整个屏幕而不是只有一行。

---

2. 缺失的选项 missing-options

这些选项只有在 Unix Vi 里存在，但 Vim 没有。如果你试图设置这些选项，你不会得到错误消息。但你给出的值不会使用和显示。

autoprint (ap)	布尔型	(缺省打开)	'autoprint' 'ap'
beautify (bf)	布尔型	(缺省关闭)	'beautify' 'bf'
flash (fl)	布尔型	(缺省未知)	'flash' 'fl'
graphic (gr)	布尔型	(缺省关闭)	'graphic' 'gr'
hardtabs (ht)	数值型	(缺省 8)	'hardtabs' 'ht'
	<Tab> 显示时移动的空白的数目		
mesg	布尔型	(缺省打开)	'mesg'

novice	布尔型 (缺省关闭)	'novice'
open	布尔型 (缺省打开)	'open'
optimize (op)	布尔型 (缺省关闭)	'optimize' 'op'
redraw	布尔型 (缺省关闭)	'redraw'
slowopen (slow)	布尔型 (缺省关闭)	'slowopen' 'slow'
sourceany	布尔型 (缺省关闭)	'sourceany'
w300	数值型 (缺省 23)	'w300'
w1200	数值型 (缺省 23)	'w1200'
w9600	数值型 (缺省 23)	'w9600'

Vi 不支持改变 termcap 项目，需要退出 Vi，编辑 termcap 项目然后再试。Vim 有 terminal-options 。

---

3. 限制 limits

Vim 有一些关于能够编辑的文件的限制
{Vi: 不能处理 <Nul> 和超过 128 的字符，行长有限制，还有其它的限制}。

E340

最大行长	在 16 位整数的机器上，(Amiga 和 MS-DOS 实模式): 32767，否则 2147483647 个字符。更长的行被分割。
最大行数	2147483647 行。
最大文件大小	2147483647 字节 (2 千兆) 如果长整数是 32 位。对于 64 位的长整型，则更多。另外对于 swap-file 而言，也受限于磁盘可用空间。

E75

文件路径长度	Unix 和 Win32: 1024 字符，否则 256 字符 (或系统能支持的最大值)。
扩展后字符选项的长度	Unix 和 Win32: 1024 字符，否则 256 字符
最大显示宽度	Unix 和 Win32: 1024 字符，否则 255 字符
最大映射左手边序列	50 个字符。
不同高亮类型数目:	超过 30000
数值类型的范围:	-2147483648 到 2147483647 (64 位系统上可能更大)
标签文件行的最大长度:	512 字节。

撤销信息和寄存器内的文本保存在内存里，因为做 (大的) 改变时，可用的 (虚拟) 内存限制了撤销的层数和能保存在寄存器里的文本。其它在内存里的信息包括: 命令行历史、QuickFix 模式的错误信息，等等。

内存使用限制

'maxmem' ('mm') 选项用来设置单个缓冲区可用的最大内存 (以千字节计)。'maxmemtot' 用来设置所有缓冲区可用的最大内存 (以千字节计)。它们的缺省值视系统而定。在 Amiga 和 MS-DOS 上，'maxmemtot' 的设置根据可用的内存决定。
这不是硬性限制，只是用来告诉 Vim 什么时候把文本移到交换文件。
如果你不喜欢 Vim 交换到文件上，设置 'maxmem' 和 'maxmemtot' 为非常大的值。交换文件这时只会作恢复用。如果你完全不想要交换文件，设置 'updatecount' 为 0，或者在启动 Vim 时加上 "-n" 参数。

---

4. 最有意思的新增功能 vim-additions

Vi 兼容性。 'compatible'

尽管 Vim 99% 和 Vi 兼容，有些 Vi 的特性被认为是漏洞，或至少需要改进。不过，Vim 启动时尽量和 "真正的" Vi 行为相似。要使得 Vim 行为有所改进，试试复位 'compatible' 选项:

```
:set nocompatible
```

或者启动 Vim 时带上 "-N" 参数:

```
vim -N
```

如果你有 .vimrc 文件，Vim 启动时自动置位 'nocompatible'。见 startup 。
'cpoptions' 选项可以设置若干专门项目的 Vi 兼容性。

支持不同的系统。

Vim 可以用在:
- 所有的 Unix 系统 (所有测试过的系统都没有问题。不过，GUI 和 Perl 接口不一定哪里都可以用)。
- Amiga (500、1000、1200、2000、3000、4000、...)。
- Windows NT，支持长文件名。
- VMS
- BeOS
- Macintosh
- Risc OS
- IBM OS/390

注意 在有的系统上需要关闭一些特性来节省资源的使用，尤其是 MS-DOS。有的老旧的系统上你需要使用旧一点的 Vim 版本。

多层持久撤销。 undo

'u' 在时间点上往后退。'CTRL-R' 则往前走。'undolevels' 选项可以设置能记住的改变数目 (缺省为 1000)。如果 'undolevels' 设为 0，就能得到 Vi-兼容的单次撤销行为。设为 -1 则完全关闭撤销功能。
如果缓冲区所有的改变都被撤销，缓冲区不再被认为改变过。这时，你可以用 :q 退出，不用 <!>。
撤销一些改变然后又做了新的改编后，Vim 在撤销树上建立一个分支。这意味着你可以回到文本过去的任何状态，任何改变都不会有丢失文本的风险。 undo-tree
如果置位 'undofile' 选项，在文件里保存撤销信息。这就是说，你可以退出 Vim，启动 Vim 打开以前编辑过的文件，然后撤销在退出 Vim 之前做出的改动。

图形用户界面 (GUI)。 gui

包括对 GUI 的支持: 菜单、鼠标、滚动条，等等。你可以定义自己的菜单。更好支持供对 CTRL/SHIFT/ALT 键加特殊键/鼠标键的组合。支持多种平台，比如 X11 (Motif 和 Athena 界面)、GTK、Win32 (Windows 95 和其后版本)、BeOS、Amiga 和 Macintosh。

多重窗口和缓冲区。 windows.txt

Vim 可以把屏幕分割为多个窗口，每个编辑一个不同的缓冲区，或者在不同位置的同一缓冲区。缓冲区可以被载入 (和改变)，但是不一定在窗口里显示。这被称为隐藏缓冲区。为了支持这个机制，增加了许多命令和选项。
Vim 也可以使用多重标签页。每个标签页由一个或多个窗口。使用包含所有标签页标签的一行可以快速在这些页面之间切换。 tab-page

终端窗口。 :terminal

Vim 可以创建窗口，其中运行终端模拟器。可用于执行任意命令、外壳或调试

器。

语法高亮。 :syntax

Vim 可以高亮关键字、模式和其它。这由若干 :syntax 命令定义，而能实现大多数语言和文件类型的高亮。Vim 包含了诸多文件提供大多数语言的高亮，比如 C、C++、Java、Pascal、Makefile、外壳脚本等等。高亮所用的颜色可以为普通终端、色彩终端和 GUI 用 :highlight 分别定义。一个方便的方式是使用 :colorscheme 命令。

高亮的文本可以汇出为 HTML 格式。 convert-to-HTML

其它可以高亮的项目包括搜索字符串的匹配 'hlsearch'，匹配的括号 matchparen 和光标所在的行和列 'cursorline' 'cursorcolumn'。

文本属性 textprop.txt

Vim 支持插件进行文本高亮。可用 prop_type_add() 指定属性类型，而用 prop_add() 放置属性。

拼写检查。 spell

如果置位 'spell' 选项，Vim 会高亮拼写错误。现在支持大约 50 种语言，用 'spelllang' 选项选择。源代码中，只检查注释和字符串的拼写。

折叠。 folding

一个范围内的行可以显示为单个 "折叠" 行。这样允许概览文件并快速移过大段文字。折叠可以手动创建，也可以基于语法、缩进等等创立。

比较模式。 diff

Vim 可以显示文件的两个版本并高亮两者的差异。相同的文本部分用折叠关掉。相关命令可以用来从一个版本把文本移到另一个版本。

插件。 add-plugin

把插件文件放在合适的目录下就可以对基本功能扩展。这是一个启动 Vim 时执行别人写的 Vim 脚本的简便方法。插件可以应用在所有文件上，或者为某个文件类型特别编写。

软件包使这项功能更加简便。 packages

异步通信和定时器。 channel job timer

Vim 可以在后台和其它进程交换信息。这样可以让服务器完成工作，并发送结果给 Vim。 channel

Vim 可以启动作业，与之通信，并结束作业。 job

可以一次或重复激活定时器，并调用函数完成任何功能。 timer

重复命令序列。 q

"q{c}" 开始记录输入的字符，并保存在命名寄存器 {c} 里。另一个 "q" 停止记录。该寄存器可以通过 "@{c}" 命令执行。这对重复一个复杂的序列十分有用。

灵活的插入模式。 ins-special-special

插入模式下，光标键可以用来在文件移动。就撤销/重做而言，这使得插入内容分成不同的部分。

CTRL-O 可以用来执行单个普通模式的命令。这和按 <Esc> 输入命令，再按 a 几乎相同。

可视模式。 Visual-mode

可视模式可以用来先高亮选择一段文本并执行一个命令在其上。这是一个代替先

给出操作符，然后移动到待操作文本的尾部的（容易使用的）替代方法。
v 和 V 用来开始可视模式，v 作用于字符上，V 作用于行上。移动光标可以扩展可视的部分。屏幕上以高亮显示该部分。输入 "o"，可视文本的另一端也可移动。可视文本可以被一个操作符影响：

d	删除
c	修改
y	抽出
> 或 <	插入和删除缩进
!	通过外部程序过滤
=	使用排版机制过滤
:	对可视行开始 : 命令。
gq	'textwidth' 列内的文本进行排版
J	连接行
~	交换大小写
u	改变为小写
U	改变为大写

{Vi 不包含可视模式。"可视" 这个名称专指普通模式，以区别于 Ex 模式}

列块操作符。 visual-block
可以用可视模式选择一个文本列块。用 CTRL-V 开始可视模式。该列块可以被删除（"d"），抽出（"y"）和改变大小写（"~"、"u" 和 "U"）。被删除和抽出的列块可以用 "p" 和 "P" 命令放回文本中。

帮助系统。 :help
帮助显示在一个窗口中。可以使用常见的命令移动，搜索，等等。标签可以用来在帮助文件间跳转，就像超链接一样。 :help 命令需要一个参数，可以快速跳到某主题的信息。<F1> 能快速访问到帮助系统。帮助索引文件名可以用 "helpfile" 选项设置。

命令行编辑和历史。 cmdline-editing
使用光标，你可以在命令行的任何位置插入或者删除。右/左光标键可以用来前移/后移一个字符。Shift + 右/左光标键可以用来前移/后移一个单词。
CTRL-B/CTRL-E 可以用来跳转到命令行的开始和结束位置。
{Vi: 只能改变每一行的最后一个字符}
{Vi: 当敲 <Esc> 的时候命令行会被执行。大多数人对此感觉不爽；因此 Vim 进行了改动。但是如果 <Esc> 是映射的一部分，命令行仍然会执行。如果你想使用 Vi 的特性包括输入的 <Esc> 的这种反应，那么可以使用 ":cmap ^V<Esc> ^V^M"}

cmdline-history
会记住命令行。上/下光标键可以用来记起以前的命令行。'history' 选项可以设置能记住的行数。命令行和搜索模式的历史分别记忆。

命令行补全。 cmdline-completion
输入一个命令行（在屏幕的最底行）时，<Tab> 可以用来补全

什么	例子
- 命令	:e<Tab>
- 标签	:ta scr<Tab>
- 选项	:set sc<Tab>
- 选项值	:set hf=<Tab>
- 文件名	:e ve<Tab>
- 等等	

如果有多个匹配，CTRL-N（下一个）和 CTRL-P（前一个）会遍历各个匹配。<Tab> 和 CTRL-N 类似，但会回绕到第一个匹配。

'wildchar' 选项可以设置进行命令行补全的字符。<Tab> 是缺省值。CTRL-D 可以在（不完全）通配符后输入；所有的匹配都会被列出。CTRL-A 会插入所有的匹配，CTRL-L 会插入所有匹配的最长的共同部分。

插入模式补全。 ins-completion

在插入模式下，CTRL-N 和 CTRL-P 键可以用来补全别的地方出现过的单词。

i_CTRL-N

CTRL-X 进入另外一个模式，完成一些补全：

i_CTRL-X_CTRL-F	文件名
i_CTRL-X_CTRL-K	'dictionary' 文件的单词
i_CTRL-X_CTRL-T	'thesaurus' 文件的单词
i_CTRL-X_CTRL-I	头文件的单词
i_CTRL-X_CTRL-L	整行
i_CTRL-X_CTRL-]	标签文件的单词
i_CTRL-X_CTRL-D	定义或者宏
i_CTRL-X_CTRL-O	全能补全：为不同文件类型分别定制的智能补全

等等

长行支持。 'wrap' 'linebreak'

如果 'wrap' 选项关闭，长行不会回绕，只有部分的内容会被显示。如果光标移动到未显示的部分，屏幕会水平滚动。最少滚动的列数由 'sidescroll' 选项决定。 zh 和 zl 命令可用来向边框滚动。

另一种方法是，长行可以在 'linebreak' 选项置位时，在单词间断开。这使得单行段落的编辑较为便利（例如，后来把文本读到 DTP 程序时）。用 gk 和 gj 命令上下移动光标。

文本排版。 formatting

'textwidth' 选项可以用来自动限制行的长度。这可以补充 Vi 的 'wrapmargin' 选项，后者并不十分有用。 gq 操作符可以用来排版一段文本（例如， gqap 排版整个段落）。文本对齐的命令： :center 、 :left 和 :right 。

扩展的搜索模式。 pattern

为了匹配各种文本项目，提供了很多附加的模式。例如："\n" 可以用来在模式中匹配换行符。

"x\{2,4}" 匹配 "x" 2 到 4 次。

"\s" 匹配单个空白字符。

目录、远程和归档浏览。 netrw

Vim 可以浏览文件系统。简单编辑一个目录。在列表里用通常的命令移动，然后按 <Enter> 进入光标所在的目录或文件。

也能用于通过 ftp、http、ssh 等连接的远程文件。

也能浏览 zip 和 tar 归档。 tar zip

加快编辑-编译-编辑循环。 quickfix

:make 命令可以用来执行编译，并跳转到第一个错误。然后对编译器错误信息构成的文件进行解释。这时，Vim 开始编辑第一个错误。

错误文件的每一行被检索一个文件名，行号，和错误信息。'errorformat' 选项可以设置类似于 scanf 格式的字符串列表，从可以处理很多编译器的输出。

:cn 命令可以用来跳转到下一个错误。

:cl 列出所有的错误信息。还有其他的命令。

'makeef' 选项指出错误信息的文件名。
'makeprg' 选项包含 :make 命令要执行的程序名。
'shellpipe' 选项包含用来把编译器的结果输出到错误信息的命令字符串。

在文件中寻找匹配。 :vimgrep
Vim 可以在多个文件中搜索模式。可以使用先进的 Vim 正规表达式模式，适用于所有系统，同时也能在压缩文件里搜索。

改进的程序缩进。 'cindent'
'cindent' 选项打开时，每行的缩进被自动调整。大部分的 C 语法能够识别。各种风格的缩进可以用 'cinoptions' 设置。切换缩进的键可以用 'cinkeys' 来设置。

注释可以自动排版。'comments' 选项设置开始和结束注释的字符。这对 C 代码最有效，但是对 e-mail 也适用 (行首的 ">") 和其他类型的文本。 = 操作符可以用来重新排版行。

许多其他的语言包含了缩进插件，支持自动缩进。 30.3

搜索头文件里的单词。 include-search
[i 命令可以用来搜索光标下的单词在当前和包含的头文件里的匹配。
'include' 选项可以设置描述包含文件的命令的模式 (缺省是为 C 程序设计)。
[I 命令列出所有的匹配。 [_CTRL-I 命令跳转到一个匹配。
[d 、 [D 和 [_CTRL-D 命令做相同的事情，不过只查找用 'define' 选项给出的模式匹配的行。

自动命令。 autocommand
在读写文件，跳转到别的缓冲区等等时候，可以根据文件名，自动执行命令。这对设置 C 程序、文档、普通文本、e-mail 等等的选项和映射有用。这也使得编辑压缩文件成为可能。

脚本和表达式。 expression
增加了命令，可以组成强大的脚本语言。
:if 条件执行，比如可以用来根据 $TERM 的值设置不同选项。
:while 重复一系列命令。
:for 遍历列表。
:echo 显示表达式的结果。
:let 给一个内部变量、选项等赋值。
变量类型包括数值、字符串、列表和字典。
:execute 执行表达式构造的命令。
:try 捕获例外。
等等，等等。见 eval 。
支持调试和刨视。 debug-scripts profile
如果还不够，提供了到 Python 、 Ruby 、 Tcl 、 Lua 、 Perl 和 MzScheme 的接口。

Viminfo。 viminfo-file
命令行历史，位置标记，和寄存器可以存在一个在启动时读入的文件里。这可以用来在退出和重启 Vim 时重复上次的搜索或命令行命令。也可以用 '0 跳转回最近一次的编辑。'viminfo' 选项可以设置选项什么内容存储在 .viminfo 文件里。缺省该功能是关闭的。

打印。 printing
:hardcopy 命令发送文本给打印机。可以包含语法高亮。

鼠标支持。 mouse-using

支持鼠标的环境包括 GUI 版本、Unix 的 xterm、支持 sysmouse 的 BSD、支持 gpm 的 Linux、MS-DOS 和 Win32。它也可用于定位光标，选择可视区域、粘贴寄存器，等等。

键名的使用。 <> key-notation

特殊键现在都有名字，类似于 <Up>、<End>，等等。名字可以用来映射，使编辑变得方便。

编辑二进制文件。 edit-binary

Vim 可以编辑二进制文件。你可以修改可执行文件的若干字符，而不会破坏其它内容。 Vim 不会删除 NUL 字符（内部，它们被表示成 <NL>）。

-b 开始编辑二进制文件的命令行参数

'binary' -b 设置的选项。防止在文件最后一行增加 <EOL>。

多语言支持。 multi-lang

可以编辑双字节或者多字节编码的文件。使用 UTF-8 支持可以同时编辑各种语言，而无须切换字体。 UTF-8

消息和菜单可以用多种语言提供。

移到光标到行外。

如果设置 'virtualedit' 选项，光标可以移动到屏幕的任何位置，包括没有文本的地方。可以用于很方便地编辑表格和图表。

---

5. 其它 Vim 特色 other-features

随机选取的一些其他的优美特色的列表。

Vim 以 "-s scriptfile" 启动时，"scriptfile" 读取的字符就像你输入的那样。如果在编辑器还没退出时文件结束，多余的字符从控制台读入。

"-w" 选项用来把所有的输入字符记录在脚本文件里。该文件可以用来重做编辑的操作，甚至可以作用在通过修改脚本文件里的一些命令后作用在别的文件上。

"-o" 选项为每个参数打开一个窗口。"-o4" 则打开四个窗口。

要能在全屏工作，Vi 需要几项 termcap 项目。Vim 只要 "cm" 项目（光标移动）。

命令模式下：

'showcmd' 选项置位时，命令字符在屏幕末行上显示。命令结束时它们被删除。

如果置位了 'ruler' 选项，当前的光标位置显示在屏幕末行。

"U" 在离开最近修改的行和 "u" 之后都还能工作。

能显示设置了第 8 位的字符。'~' 和 0xa0 之间的字符显示为 "~?"、 "~@"、"~A"，等等。除非它们包含在 'isprint' 选项里。

"][" 跳到 C 函数的下一个结束处（第一列的 '}'）。

"[]" 转到 C 函数的前一个结束处（第一列的 '}'）。

"]f"、"[f" 和 "gf" 开始编辑光标所在的文件名。
CTRL-W f 分割窗口，并开始编辑光标所在的文件名。

"*" 正向搜索光标所在的标识符，"#" 反向搜索之。
"K" 运行 'keywordprg' 选项定义的程序，使用光标所在的标识符作为参数。

"%" 可以在前面加一个计数。光标跳转到文件该百分比所处的行。普通的 "%" 功能则跳转到匹配的括号上，引号里的括号跳过。

CTRL-] 命令可以在光标位于标识符中间时使用。

记住用过的标签。使用标签栈的命令包括 CTRL-T、":pop" 和 ":tag"。":tags" 列出标签栈。

Vi 使用 'wrapscan' 来搜索标签。跳转到标签时 Vi 从别的文件的第二行开始搜索。如果没有置位 'wrapscan'，它不会去找别的文件的第一行的标签。

'tags' 选项可以设置标签文件名字的列表。可以指定多个标签文件。"./" 开始的文件名里，"./" 被当前文件所在的路径所代替。这样可以指定编辑中的文件的相同目录下的标签文件。
{Vi: 有的版本总是使用二分法查找}
{Vi 没有针对标签文件命令的安全保护措施}

以前用过的文件名被记在轮换文件名列表里。
CTRL-^ 接受一个计数，指定列表的一个索引。
":files" 命令显示轮换文件名列表。
"#<N>" 被替换成列表中第 <N> 个轮换文件名。
"#<" 被替换成不带扩展名的当前文件名。

搜索模式可以使用更多特性。<NL> 字符被看成搜索模式和 ":s" 的替换字符串的一部分。Vi 把它看成命令的结束。

搜索可以把光标放在匹配的结束处，还可以包括一个字符位移。

计数可以应用于 "~"、":next"、":Next"、"n" 和 "N"。

":next!" 命令在置位 'autowrite' 时，不写入文件。过去，vi 写入文件，但这被认为是一个漏洞，因为用户不期待该行为，而且 ":rewind!" 也不写入文件。

Vi 在替换模式下输入 <CR> 时，只有置位 'ai' 以后才删除一个字符（但直到你按 <Esc> 时才看到）。Vim 在任何情况下都删除一个字符（并立即显示）。

增加了 :wnext 命令。和 ":write" 接着 ":next" 相同。

":w!" 命令总是写入，即使文件写保护时也是如此。Vi 里你需要先执行 ":!chmod +w %:S" 和 ":set noro"。

置位 'tildeop' 以后，"~" 是操作符（一定要跟在移动命令之后使用）。

"J"（连接）命令里，你可以复位 'joinspaces' 选项，使得句号之后只有一个空格（Vi 插入两个空格）。

"cw" 可以用来改变多个字符组成的空白 (Vi 则不明确: "cw" 只改变一个空格，而 "dw" 删除所有的空白)。
{Vi: 在多个空白的某一个空白之上做 "cw" 只会改动单个空白字符；这可能不太对，因为 "dw" 是删除所有后面的空白的}

"o" 和 "O" 接受计数，以便重复插入的部分 (Vi 清除部分的显示)。

不支持 Ex 命令之后的标志位 (也无实现它的计划)。

非 UNIX 系统上，":cd" 命令显示当前的目录，而不是转到主目录 (因为没有)。":pwd" 在所有的系统上显示当前目录。

":cd" 命令之后，文件名 (参数列表和打开的文件) 仍然指向原来的文件。Vi 里，改动过的文件里不允许执行 ":cd"，否则文件名的含义就改变了。

":source!" 命令从文件里读取 Vi 命令。

":mkexrc" 命令写入当前修改过的选项和映射到 ".exrc" 文件里。":mkvimrc" 写入到 ".vimrc" 文件。

映射不检查所谓的 "尾递归"。这样可以允许 ":map! foo ^]foo" 类似的形式。

映射以数字开始时，vi 丢失了之前输入的计数 (例如，如果使用映射 ":map g 4G"，命令 "7g" 转到第 4 行)。这被认为是一个 vi 的漏洞。Vim 连接计数 (这个例子里，它成为了 "74G")，符合多数人的期待。

:put! 命令把寄存器的内容插到当前行之前。

如果放置的文本少于一行，vi 的 "p" 和 "P" 命令不能用 "." 重复。Vim 它们总是可以重复的。

":noremap" 命令可以用来输入不会被重映射的映射。这可以用来交换两个键的含义。
":cmap"、":cunmap" 和 ":cnoremap" 可以用来只在命令行编辑时映射。":imap"、":iunmap" 和 ":inoremap" 则可用作插入模式的映射。类似的命令对缩写也存在:
":noreabbrev"、":iabbrev"、":cabbrev"、":iunabbrev"、":cunabbrev"、":inoreabbrev"、":cnoreabbrev"。

Vi 里命令 ":map foo bar" 会删除以前定义过的映射 ":map bug foo"。这被认为是一个漏洞。所以 Vim 没有这一特性。
":unmap! foo" 倒是会删除 ":map! bug foo"。如果不这样，撤销映射很难实现 (这和 vi 兼容)。

':' 寄存器包含最近的命令行。
'%' 寄存器包含当前的文件名。
'.' 寄存器包含最近插入的文本。

":dis" 命令显示抽出的寄存器的内容。

CTRL-O/CTRL-I 可用作跳转到较旧的/较新的位置。这和 '' 命令所用到的位置相同，但是可以到别的文件里。":jumps" 命令列出较老的所有位置。

如果置位了 'shiftround' 选项，">" 和 "<" 命令的缩进被取整到 'shiftwidth' 的倍数。

'scrolljump' 选项可以设置为光标离开屏幕时最少滚动的行数。可用在滚动缓慢的场合。

'scrolloff' 选项可以设置光标上面和光标下面最少保留的行数。这样，你在编辑时，总能看到一些上下文。如果设为了一个大值，光标总保持在窗口的中间。

大写位置标记用来在文件间跳转。":marks" 命令列出当前设置的所有位置标记。"']" 和 "`]" 命令跳转到前一个操作符的最后，或者放置 (put) 命令插入文本的最后。"'[" 和 "`[" 这跳转到相应的开始处。{Vi: 没有大写位置标记}

'shelltype' 选项的设置反映了 Amiga 使用的外壳类型。

'highlight' 选项可以设置若干命令使用的高亮模式。

CTRL-A (加) 和 CTRL-X (减) 命令是新增的。命令的计数 (缺省为 1) 被从光标所在或其后的数字加上/减去。该数字可以是十进制、八进制 ('0' 开始)，或者十六进制 ('0x' 开始)。对宏很有用。

:set 命令可以使用 "inv" 前缀来反转布尔型选项的值。

Vi 和 Vim 上你都可以用 ":substitute" 命令的 CTRL-M 增加换行符。Vi 里，这意味着你不能在文本里插入一个真正的 CTRL-M。Vim 里，你可以用前导 CTRL-V 的方式在文本里输入一个真正的 CTRL-M。

插入模式下:

如果打开 'revins' 选项，则进行反方向插入。这可以用来输入希伯来文。插入普通字符时，光标不会移动，而文字会往右移动。退格、CTRL-W 和 CTRL-U 同样反向工作。CTRL-B 切换 'revins' 选项。在替换模式下，'revins' 没有效果。必须在编译时打开这一功能。

退格键和 CTRL-D 一样可以用来删除自动的缩进。

如果 'backspace' (bs) 选项包含 "eol"，你可以在换行符上退格、CTRL-U 和 CTRL-W。如果 'backspace' 选项包含了 "start"，你可以在插入的开始处退格。

如果置位了 'paste' 选项，一些选项被复位，而且插入模式和缩写的映射被禁止。这允许在窗口系统里粘贴文本不会产生以外的效果。如果复位 'paste' 选项，旧的选项值重新恢复。

CTRL-T/CTRL-D 总是插入/删除当前行的一个缩进，不管光标处于何列。

CTRL-@ (插入上次输入过的文本) 总能工作 (Vi: 只有在作为第一个字符输入时)。

CTRL-A 和 CTRL-@ 类似，但不离开插入模式。

CTRL-R {register} 可以用来插入寄存器的内容。

如果置位了 'smartindent' 选项，C 程序会更好的自动缩进，'cindent' 则做得更多。

CTRL-Y 和 CTRL-E 可以用来从当前光标位置的上/下复制字符。

CTRL-V 之后，你可以输入三个十进制数字。该字节作为单个字符插入在文本里。这可以

用来输入你的键盘上没有的国际字符。

打开 'expandtab' (et) 选项，使得 <Tab> 被扩展为合适数量的空格。

窗口总是反映缓冲区的内容 (Vi 在改变文本和其他一些情况下并不如此)。

如果 Vim 编译时定义了 DIGRAPHS，就能支持二合字母。已经包含了一组常用的二合字母。它们可以用 ":digraph" 显示。用 ":digraph {char1}{char2} {number}" 可以定义更多。二合字母本身以 "CTRL-K {char1} {char2}" 或 "{char1} BS {char2}" 输入 (后者只有在打开 'digraph' 选项时)。

在重复插入时，例如 "10atest <Esc>"，vi 只会处理第一个插入的回绕边界 ('wrapmargin')。Vim 则对整体处理。

"i" 或 "a" 命令的计数用于所有文本。Vi 只针对第一行。在 Vi 里，"3iabc<NL>def<Esc>" 会插入 "abcabcabc<NL>def"，但 Vim 会给出 "abc<NL>defabc<NL>defabc<NL>def"。

命令行模式下:

<Esc> 退出命令行但不执行之。vi 里，命令行还是会执行，这和大多数人的期待不符 (按 <Esc> 应该总是让你退回到普通模式)。要避免一些古老的宏的问题，宏中的 <Esc> 仍然执行命令行命令。如果期待 <Esc> 的输入像 vi 那样执行命令行命令，你可以如此修正 ":cmap ^V<Esc> ^V<CR>"

通用:

'ttimeout' 选项和 'timeout' 类似，但只对光标和功能键有效，而不是普通的经过映射的字符。'timeoutlen' 选项给出等待的毫秒数。如果 'esckeys' 选项没有置位，<Esc> 开始的光标和功能键在插入模式下不会识别。

每个终端字符串有一个选项。这可以用在 termcap 不支持或者要改变单独的字符串的场合。

'fileformat' 选项可以设置 <EOL> 的选择: "dos" <CR><NL>、"unix" <NL> 或 "mac" <CR>。如果 'fileformats' 选项非空，Vim 试图自动检测 <EOL> 的类型，并据此设置 'fileformat' 选项。

有的系统没有任务 (job) 控制 (旧 Unix 系统和非 Unix 系统)。CTRL-Z、":stop" 或 ":suspend" 命令启动一个新的外壳。

如果 Vim 在 Amiga 上启动时没有交互窗口作为输出，则打开一个窗口 (并且 :sh 仍然工作)。你可以用 -d 参数给出编辑的设备，例如 "-d con:20/20/600/150"。

'columns' 和 'lines' 选项可以用来设置或者取得显示的宽度和高度。

从文件的第一行或者最后数行里读取选项的设置。'modelines' 选项决定尝试的行数 (缺省为 5)。注意: 这和 Vi 版本不同，后者会执行模式行里的任何 Ex 命令 (严重的安全问题)。 trojan-horse

如果置位了 'insertmode' 选项 (例如在 .exec 里)，Vim 以插入模式启动。并且在按了 <Esc> 以后回到这一模式。

撤销的信息保存在内存里。可用的内存限制了可以撤销的改变的数目和大小。这可能对 MS-DOS 是一个问题，但在 Amiga 上很少是，而 Unix 和 Win32 上几乎不会。

如果置位了 'backup' 或 'writebackup' 选项：在文件覆盖前，备份文件 (.bak) 被创建。如果使用 "backup" 选项，覆盖后该文件得以保留。

Vim 创建 ".swp" 结尾的文件，用以存储文件的改动部分和内存放不下的内容。该文件用于 "vim -r file"，即从强行终止的编辑会话恢复。交换文件的使用可以用把 updatecount' 选项设为 0，或者以 "-n" 选项启动 Vim 来关闭。使用 'directory' 选项把 .swp 文件放到其它地方。

Vim 在 8.3 文件名的文件系统下能正常工作。这也包括 Amiga 的 messydos 或者 crossdos 文件系统，或者 Unix 上装载 (mount) 的任何 8.3 文件系统。见 'shortname' 。

错误信息显示至少一秒钟 (Vi 的错误信息会相互覆盖)。

如果 Vim 给出 hit-enter 提示，你可以输入任何键。除了 <CR>、<NL> 和 <Space> 的字符都被认为是命令 (的开始)。
{Vi 只执行 ":" 开始的命令}

计数和无名寄存器的内容在改变文件后仍然记住。

"No lines in buffer" 消息是一个普通的消息，而不是错误信息。因为后者会使得映射被非正常终止。
{Vi: 在你有机会读到之前，错误信息可能会相互覆盖}

Amiga 上支持 AUX: 设备。

---

6. 支持的 Vi 特性 vi-features

Vim 支持几乎所有的 Vi 命令且方法基本一致。也就是置位 'compatible' 选项且 'cpoptions' 包含所有标志位。复位 'compatible' 和从 'cpoptions' 中删除标志位的效果可在特定命令的帮助上找到。

帮助文件过去用 {Vi 无此功能} 标出 Vim 有而 Vi 没有的特性。不过，这些说明使得帮助文件显得杂乱，我们现在反其道而行之：以下列出 Vi 已经支持的功能。其余的均是 Vim 新加的。

Vi 支持以下的 Ex 命令:

:abbreviate	输入缩写
:append	附加文本
:args	显示参数列表
:cd	改变目录；Vi: 没有 "cd -"
:change	替换一行或若干行
:chdir	改变当前目录
:copy	复制行
:delete	删除行
:edit	编辑文件
:exit	等同于 ":xit"
:file	显示或者设置当前文件名；Vi: 没有列号

:global	对匹配的行执行命令
:insert	插入文本
:join	连接行；Vi: 没有 :join，
:k	设置位置标记
:list	显示行
:map	显示或者输入映射
:mark	设置位置标记
:move	移动行
:Next	转到参数列表的上一个文件 {Vi: 没有计数}
:next	跳转到参数列表的下一个文件 {Vi: 没有计数}
:number	显示文本行时给出行号
:open	启动打开模式 (Vim 没有实现)
:pop	跳转到标签栈较旧的项目上 (仅在某些版本有)
:preserve	写入所有文本到交换文件 {Vi: 也可能退出}
:previous	跳转到参数列表里的上一个文件 {Vi: 仅在某此版本有}
:print	显示行
:put	插入寄存器的内容到文本
:quit	退出 Vi
:read	读入文件到文本
:recover	从交换文件里恢复文件 {Vi: 在有可以恢复的内容时，用另一种方式恢复并且发送邮件}
:rewind	转到参数列表的第一个文件；没有 ++opt
:set	显示或设置选项；但没有 ‘:set inv{option} 、 :set option&‘、‘:set all& 、 :set option+=value 、 :set option^=value‘ ‘:set option-=value‘ ‘:set option<‘
:shell	退到外壳程序
:source	从文件里读取 Vi 或 Ex 命令
:stop	暂停编辑器并退回外壳
:substitute	查找和替代文本；Vi: 没有 ’&’、’i’、’s’、’r’ 或 ’I’ 标志位，确认提示只支持 ’y’ 和 ’n’，没有高亮
:suspend	等同于 ":stop"
:t	等同于 ":copy"
:tag	跳转到标签上
:unabbreviate	删除缩写
:undo	撤销最近的改变 {Vi: 只有一层}
:unmap	删除映射
:vglobal	对不匹配的行，执行命令
:version	显示版本号和其它信息
:visual	等同于 ":edit"，但关掉 "Ex" 模式
:wq	写回文件，然后退出 Vi
:write	写回文件
:xit	如果缓冲区被改动，写入之。然后退出 Vi
:yank	抽出行到寄存器
:z	显示一些行 {Vi: 不是所有版本都有}
:!	过滤行，或可通过外部命令执行之
:"	注释
:#	等同于 ":number"
:*	执行寄存器的内容
:&	重复上次的 ":substitute"
:<	将行左移一个 ’shiftwidth’
:=	显示光标所在的行号
:>	将行右移一个 ’shiftwidth’
:@	执行寄存器的内容；但没有 :@ (译者注：原文如此，或指 :@:)；:@@ 只有某些版本有

这些命令的共同点是 Vi 不支持 :edit 和其它打开文件的命令的 ++opt 参数。

Vi 支持以下的普通模式命令:

备注: 参见 normal-index 的开始处说明描述文本中的 WORD, N, Nmove 等的含义。

CTRL-B	反向 (Back) 滚动 N 屏
CTRL-C	终止当前的 (搜索) 命令
CTRL-D	向下 (Down) 滚动 N 行 (缺省: 半屏); Vim 滚动 'scroll' 屏幕行, Vi 滚动文件行; 有行回绕时两者会有差异
CTRL-E	向上滚动 N 行 (附加 (Extra) N 行)
CTRL-F	向前 (Forward) 滚动 N 屏
CTRL-G	显示当前文件名和位置
<BS>	等同于 "h"
CTRL-H	等同于 "h"
<NL>	等同于 "j"
CTRL-J	等同于 "j"
CTRL-L	重画屏幕
<CR>	光标移到 N 行下面的第一个 CHAR
CTRL-M	等同于 <CR>
CTRL-N	等同于 "j"
CTRL-P	等同于 "k"
CTRL-R	在有些 Vi 版本中: 等同于 CTRL-L
CTRL-T	跳转到标签列表的第 N 个较老的标签 (Tag)
CTRL-U	上滚 (Upwards) N 行 (缺省: 半屏) {Vi 使用文件行而 Vim 滚动 'scroll' 屏幕行; 有行回绕时两者会有差异}
CTRL-Y	下滚 N 行
CTRL-Z	暂停程序 (或者启动新的外壳)
CTRL-]	:ta 到光标所在的标识符 {Vi: 光标之后的标识符}
CTRL-^	编辑轮换文件 {Vi: 没有计数}
<Space>	等同于 "l"
!	通过 {filter} 命令过滤 Nmove 文本
!!	通过 {filter} 命令过滤 N 行
"	指定下次删除、抽出或放置使用的寄存器 {a-zA-Z0-9.%#:-"} (大写用于添加) ({.%#:} 只能用于放置)
$	光标移动到往下第 N 行的行尾之后
%	查找本行下一个 (花/方) 括号, 并转到其匹配的括号上, 或者转到匹配的注释配对上, 或者转到匹配的预处理命令上 (Vi: 不支持计数)
&	重复上次 :s
'	跳转到位置标记 (Vi: 只有小写位置标记)
(	光标回退 N 个句子
)	光标前进 N 个句子
+	等同于 <CR>
,	反方向重复最近的 f、t、F 或 T N 次
-	光标移动到上方 N 行的第一个 CHAR 上
.	重复最近的改变, 其计数被 N 取代
/	正向搜索第 N 次出现的 {pattern}
0	光标移动到本行的第一个字符上
:	开始进入 Ex 命令
;	重复最近的 f、t、F 或 T N 次
<	左移 Nmove 文本行一个 'shiftwidth'
<<	左移 N 行一个 'shiftwidth'

=	通过 "indent" 过滤 Nmove 文本行
==	通过 "indent" 过滤 N 行
>	右移 Nmove 文本行一个 'shiftwidth'
>>	右移 N 行一个 'shiftwidth'
?	反向搜索第 N 次出现的 {pattern}
@	执行 {a-z} 寄存器的内容 N 次 {Vi: 只有命名寄存器}
@@	重复上次的 @{a-z} N 次
A	在行尾附加文本 N 次
B	光标反向移动 N 个 WORD (字串)
C	从光标位置到行尾部分进行修改
D	删除位置到行尾部分，再加上 N-1 行的字符 [到寄存器 x]；等价于 "d$"
E	光标正向移动到第 N 个 WORD 的结束处
F	光标左移到第 N 次出现的 {char} 上
G	光标移动到第 N 行，缺省是最后一行
H	光标移动到屏幕顶部开始算的第 N 行
I	在本行第一个 CHAR 之前插入文本 N 次
J	连接 (Join) N 行；缺省为 2
L	光标移动到屏幕底部开始算的第 N 行
M	光标移动到屏幕的中间一行
N	反方向重复最近的 '/' 或 '?' N 次
O	在光标上方开始一个新行，并插入文本，重复 N 次 {Vi: 清空 [count] 个屏幕行}
P	在光标放置文本 [到寄存器 x] N 次 {Vi: 没有计数}
Q	切换到 "Ex" 模式
R	进入替换模式：覆盖存在的字符，重复输入文本 N-1 次
S	删除 N 行 [到寄存器 x] 并开始输入；等价于 "cc"
T	光标移动到左侧第 N 次出现的 {char} 之前
U	撤销一行内所有最近的改变 {Vi: 仅当这些修改不超出最近作改变那行的范围}
W	光标前移 N 个 WORD
X	删除光标前的 N 个字符 [到寄存器 x]
Y	抽出 N 行 [到寄存器 x]；等价于 "yy"
ZZ	如果修改过，保存当前文件，然后退出
[[	光标后退 N 个小节
[]	光标后退 N 个 SECTION
^	光标移动到本行的第一个 CHAR 上
_	光标移动到下方第 N - 1 行的第一个 CHAR 上
`	光标移动到位置标记 {a-zA-Z0-9}
a	在光标处附加文本 N 次
b	光标反向移动 N 个单词
c	删除 Nmove 文本 [到寄存器 x] 并开始插入
cc	删除 N 行 [到寄存器 x] 并开始插入
d	删除 Nmove 文本 [到寄存器 x]
dd	删除 N 行 [到寄存器 x]
e	光标正向移动到第 N 个单词的结束处
f	光标右移到第 N 次出现的 {char} 上
h	光标左移 N 个字符
i	在光标前插入文本 N 次
j	光标下移 N 行
k	光标上移 N 行
l	光标右移 N 个字符
m	在光标位置处设置位置标记 {A-Za-z}
n	重复最近的 '/' 或 '?' N 次

o 在光标下方开始一个新行，并插入文本
{Vi: 清空 [count] 个屏幕行}
p 在光标后放置文本 [从寄存器 x] N 次
{Vi: 没有计数}
r 用 {char} 替换 N 个字符 {Vi: CTRL-V <CR> 还是换成换行符。没有办法换成 <CR>}
s (替换) 删除 N 个字符 [到寄存器 x] 并开始插入
t 光标移动到右侧第 N 次出现的 {char} 之前
u 撤销改变 {Vi: 只有一层}
w 光标前移 N 个单词
x 删除光标开始的 N 个字符 [到寄存器 x]
y 抽出 Nmove 文本 [到寄存器 x]
yy 抽出 N 行 [到寄存器 x]
z<CR> 光标移动到窗口的顶行
z- 光标移动到窗口末行
z+ 光标移动到第 N 行
z^ 光标移动到第 N 行
{ 光标反向移动 N 个段落
| 光标移到第 N 列
} 光标正向移动 N 个段落
~ 切换光标所在处开始的 N 个字符的大小写；Vim: 取决于 'tildeop'
{Vi: 没有计数，没有 'tildeop'}
<Del> 等同于 "x"

Vi 的插入模式支持以下的命令：

CTRL-@ 插入最近插入的文本并停止插入
{Vi: 仅当敲入第一个字符时，而且只限于前 128 个字符}
CTRL-C 退出插入模式，不检查缩写，除非置位了 'insertmode'。
CTRL-D 在当前行删除一个 shiftwidth 的缩进
{Vi: CTRL-D 只有在使用自动缩进之后才有效}
<BS> 删除光标前的字符 {Vi: 不删除自动的缩进，不会跨行，不会删除插入开始位置之前的内容}
CTRL-H 等同于 <BS>
<Tab> 插入 <Tab> 字符
CTRL-I 等同于 <Tab>
<NL> 等同于 <CR>
CTRL-J 等同于 <CR>
<CR> 开始新行
CTRL-M 等同于 <CR>
CTRL-T 在当前行插入一个 shiftwidth 的缩进 {Vi: 只有在缩进内部才能用}
CTRL-V {char} 按本义插入下一个非数字字符 {Vi: 没有十进制字节输入}
CTRL-W 删除光标前的单词
CTRL-Z 'insertmode' 打开时：暂停 Vi
<Esc> 结束插入模式 (除非打开了 'insertmode')
CTRL-[等同于 <Esc>
0 CTRL-D 删除当前行的所有缩进
^ CTRL-D 删除当前行的所有缩进，在下一行恢复删除的缩进
<Del> 删除光标所在的字符

Vi 支持以下的选项：

'autoindent'	'ai'	根据上一行决定新行的缩进 {Vi 稍有不同：输入 <Esc> 或 <CR> 删除缩进后，上下移动把光标放在删除的缩进之后；Vi 则把光标放在已删除的缩进的某处}
'autowrite'	'aw'	有改动时自动写回文件
'directory'	'dir'	交换文件所用的目录名列表 {Vi：放置临时文件的目录，缺省为 "/tmp"}
'edcompatible'	'ed'	切换 ":substitute" 命令的标志位
'errorbells'	'eb'	有错误信息时响铃
'ignorecase'	'ic'	搜索模式时忽略大小写
'lines'		显示屏幕的行数
'lisp'		自动 lisp 缩进 {Vi：行为略有不同}
'list'		显示 <Tab> 和 <EOL>
'magic'		改变搜索模式所用的特殊字符
'modeline'	'ml'	在文件开头或结尾识别模式行 {Vi：部分版本称为 modelines}
'number'	'nu'	行前显示行号
'paragraphs'	'para'	分隔段落的 nroff 宏
'prompt'	'prompt'	Ex 模式打开提示
'readonly'	'ro'	禁止写入缓冲区 {Vim 用 :view 编辑文件时置位 'readonly'}
'remap'		允许映射可以递归调用
'report'		报告行改变的行数下限
'scroll'	'scr'	用 CTRL-U 和 CTRL-O 滚动的行数
'sections'	'sect'	分隔小节的 nroff 宏
'shell'	'sh'	使用外部命令的外壳程序名字
'shiftwidth'	'sw'	(自动) 缩进使用的步进单位，以空白数目计
'showmatch'	'sm'	插入括号时短暂跳转到匹配的括号
'showmode'	'smd'	在状态行上显示当前模式的消息
'tabstop'	'ts'	<Tab> 在文件里使用的空格数
'taglength'	'tl'	标签里的有效字符数
'tags'	'tag'	标签命令使用的文件名列表 {Vi：缺省是 "tags /usr/lib/tags"}
'tagstack'	'tgst'	把标签推入标签栈 {仅有部分 Vi 版本支持}
'term'		终端名
'terse'		简化部分消息
'timeout'	'to'	映射和键盘代码等待超时
'timeoutlen'	'tm'	超时时间 (以毫秒计) {Vi：仅在某些版本上有}
'ttytype'	'tty'	'term' 的别名
'verbose'	'vbs'	给出详细信息 {Vi 仅在有些版本会有布尔型的选项}
'warn'		当缓冲区改变时，对外壳命令给出警告
'window'	'wi'	CTRL-F 和 CTRL-B 滚动的行数 {Vi 也用此选项指定显示的行数}
'wrapmargin'	'wm'	使 (硬) 回绕开始的从右侧起算的字符数 {Vi：行为不同，可用性也差}
'wrapscan'	'ws'	搜索在文件尾折回文件头
'writeany'	'wa'	写入文件不需 "!" 强制

另见 missing-options 。

---

7. 命令行参数 cmdline-arguments

不同版本的 Vi 有不同的命令行参数。这会引起混乱。为了帮助你，本节给出这些差别的

概述。

这里考虑五个 Vi 的不同版本:

Elvis	Elvis 2.1b 版本
Nvi	Nvi 1.79 版本
Posix	Posix 1003.2
Vi	Vi 3.7 版本 (Sun 4.1.x 版本)
Vile	Vile 7.4 版本 (不完整)
Vim	Vim 5.2 版本

只有 Vim 支持在文件名之间和之后接受选项。

+{command}	Elvis、Nvi、Posix、Vi、Vim: 和 "-c {command}" 相同。
-	Nvi、Posix、Vi: 在批处理模式下运行 Ex。 Vim: 从标准输入读取文件 (批处理模式用 -s)。
--	Vim: 选项结束，其后的全被解释为文件名。
--cmd {command}	Vim: 在执行 vimrc 文件前执行 {command} 命令。
--echo-wid	Vim: GTK+ 在标准输出回显窗口 ID。
--help	Vim: 显示帮助信息并退出。
--literal	Vim: 按本义接受文件名，不扩展通配符。
--nofork	Vim: 和 -f 相同。
--noplugin[s]	Vim: 跳过插件的载入。
--remote	Vim: 在另外一个 Vim 服务器编辑文件
--remote-expr {expr}	Vim: 在另外一个 Vim 服务器执行 {expr}
--remote-send {keys}	Vim: 发送键序列 {keys} 到 Vim 服务器并退出
--remote-silent {file}	Vim: 如有可能，在另外一个 Vim 服务器编辑文件
--remote-wait	Vim: 在另外一个 Vim 服务器编辑文件，并等待之
--remote-wait-silent	Vim: 和 --remote-wait 类似，但如果不可能不会抱怨
--role {role}	Vim: GTK+ 2: 设置主窗口的角色
--serverlist	Vim: 输出 Vim 服务器的列表并退出
--servername {name}	Vim: 指定 Vim 服务器名
--socketid {id}	Vim: 运行 Vim 的 GTK 窗口套接字
--windowid {id}	Vim: 运行 Vim 的 Win32 窗口号
--version	Vim: 显示版本信息并退出。

-? Vile: 显示使用小结并退出。

-a Elvis: 同一个窗口载入所有指定的文件 (Vim 使用 -o)。

-A Vim: 以阿拉伯模式启动 (如果编译时加入阿拉伯语功能)。

-b {blksize} Elvis: 会话文件使用 {blksize} 块大小。
-b Vim: 置位 'binary' 模式。

-C Vim: 兼容模式。

-c {command} Elvis、Nvi、Posix、Vim: 在载入编辑缓冲区后，运行 Ex 命令 {command}。
Vim: 允许至多 10 个 "-c" 参数

-d {device} Vim: 使用 {device} 作为 I/O (仅限于 Amiga)
{仅当编译时加入 +diff 特性才有效}
-d Vim: 开始时置位 'diff'。 vimdiff

-dev {device} Vim: 使用 {device} 作为 I/O (仅限于 Amiga)。

-D Vim: 调试模式。

-e Elvis、Nvi、Vim: 以 Ex 模式启动，就像可执行程序叫 "ex" 那样。

-E Vim: 以改进的 Ex 模式启动 gQ ，就像 "exim" 那样。

-f Vim: 在前台运行 GUI (Amiga: 不打开新窗口)。
-f {session} Elvis: 使用 {session} 作为会话文件。

-F Vim: 以波斯语模式启动 (如果编译时加入波斯语功能)。
Nvi: 快速启动，编辑开始时不读入整个文件。

-G {gui} Elvis: 使用 {gui} 作为用户界面。

-g Vim: 启动 GUI。
-g N Vile: 开始编辑第 N 行

-h Vim: 给出帮助消息。
Vile: 编辑帮助文件

-H Vim: 以希伯来语模式启动 (如果编译时加入该功能的话)。

-i Elvis: 以插入模式开始编辑每个窗口。
-i {viminfo} Vim: 使用 {viminfo} 作为 viminfo 文件。

-L Vim: 和 "-r" 相同 {Vi: 只在某些版本里存在: "List recoverable edit sessions"}。

-l Nvi、Vi、Vim: 置位 'lisp' 和 'showmatch' 选项。

-m Vim: 修改不允许被写入，复位 'write' 选项。

-M Vim: 不允许修改，复位 'modifiable' 和 'write' 选项。

-N Vim: 非兼容模式。

-n Vim: 不用交换文件。

-nb[args] Vim: 打开一个 NetBeans 接口连接

-O[N] Vim: 和 -o 类似，但使用垂直分割的窗口。

-o[N] Vim: 打开 [N] 个窗口，或者每个文件一个。

-p[N] Vim: 打开 [N] 个标签页，或者每个文件一个。

-P {parent-title} Win32 Vim: 在父应用窗口内部打开 Vim

-q {name} Vim: 使用 quickfix 错误文件 {name}。
-q{name} Vim: 同上。

-R Elvis、Nvi、Posix、Vile、Vim: 置位 'readonly' 选项。

-r Elvis、Nvi、Posix、Vi、Vim: 恢复模式。

-S Nvi: 置位 'secure' 选项。
-S {script} Vim: 在启动后执行 script 脚本。

-s Nvi、Posix、Vim: 在 Ex 模式时，和 "-" 相同 (安静模式)。
Elvis: 置位 'safer' 选项。
-s {scriptin} Vim: 读入 {scriptin} 脚本文件；只有在非 Ex 模式下才有效。
-s {pattern} Vile: 搜索 {pattern}

-t {tag} Elvis、Nvi、Posix、Vi、Vim: 编辑包含 {tag} 的文件。
-t{tag} Vim: 同上。

-T {term} Vim: 设置终端名为 {term}。

-u {vimrc} Vim: 从 {vimrc} 文件读入初始化设置。

-U {gvimrc} Vim: 从 {gvimrc} 文件读入 GUI 初始化设置。

-v Nvi、Posix、Vi、Vim: 以普通模式启动 (Vi 的术语是可视模式 (Visual))。
Vile: 阅读模式，不允许修改。

-V Elvis、Vim: 详细模式。
-V{nr} Vim: 指定特定级别的详细模式。

-w {size} Elvis、Posix、Nvi、Vi、Vim: 设置 'window' 的值为 {size}。
-w{size} Nvi、Vi: 和 "-w {size}" 相同。
-w {name} Vim: 写入脚本文件 {name} (必须以非数值字符启动)。

-W {name} Vim: 附加到脚本文件 {name} 里。

-x Vi、Vim: 要求输入加密密钥。见 encryption 。

-X　　Vim: 不连接到 X 服务器。

-y　　Vim: 以简易模式启动，类似于 evim 。

-Z　　Vim: 受限模式

@{cmdfile}　　Vile: 使用 {cmdfile} 作为启动文件。

---

8. POSIX 相容性　　posix　posix-compliance

2005 年，运行了 POSIX 测试包以对 Vim 的相容性进行了检查。大多数测试都能正确执行。有一些行为 Vim 和 POSIX 不相容，即使运行于 Vi 兼容模式也是如此。

$VIM_POSIX

Vim 启动时设置 $VIM_POSIX 环境变量会使得 'cpoptions' 包含 POSIX 的标志位。这使 Vim 尽可能地照 POSIX 方式运行。这和 Vi 兼容模式稍有不同。

这里可以找到 Posix 的 Vi 规格:
https://pubs.opengroup.org/onlinepubs/9699919799/utilities/vi.html
还有相关的 Ex 规格:
https://pubs.opengroup.org/onlinepubs/9699919799/utilities/ex.html

这里是 Vim 和 POSIX 指定行为不同之处以及原因:

posix-screen-size

如果终端获得的屏幕大小信息更加可靠，Vim 忽略 $COLUMNS 和 $LINES 环境变量。在 'cpoptions' 里加入 'I' 标志位使 $COLUMNS 和 $LINES 可以否决另一种方式获得的大小。

在原来的 Vi 上，"{" 和 "}" 命令不停在 "{" 上，但 POSIX 指定应该如此。
如果你希望 POSIX 方式，在 'cpoptions' 里加入 '{' 标志位。

"D"、"o" 和 "O" 命令接受计数，它们被重复时也是如此。
如果你希望忽略计数，在 'cpoptions' 里加入 '#' 标志位。

如果 'cpoptions' 里加入 '.' 标志位，那么当前缓冲区被修改时 ":cd" 命令失败。

如果 'shortmess' 里加入 "A" 标志位，没有 ATTENTION 消息。

以下是运行 POSIX 测试包相关的评注:
- vi 测试 33 有时因为不明原因失败
- vi 测试 250 失败；新版本的行为会有改变
  http://www.opengroup.org/austin/mailarchives/ag-review/msg01710.html
  (链接不再有效，或许现在应是:
  https://www.opengroup.org/sophocles/show_mail.tpl?CALLER=show_archive.tpl&source=L&listname=au
- vi 测试 310 失败；任何错误出现时返回代码必须非零?
- ex 测试 24 失败，因为测试有问题。SUSv2 和 SUSv3 之间已修改。
- ex 测试 47、48、49、72、73 失败，因为安静模式下不读入 .exrc 文件也不使用 $EXINIT。
- ex 测试 76、78 失败，因为使用 echo 代替了 printf。(已修正)
  另: \s 没有被换成空格的问题。
- ex 测试 355 失败，因为 'window' 不在 "30z" 里应用。

- ex 测试 368 失败，因为外壳命令在安静模式下不回显。
- ex 测试 394 失败，因为 "=" 命令输出在安静模式下不可见。
- ex 测试 411 失败，因为测试文件不正确，包含了多余的 ':'。
- ex 测试 475 和 476 失败，因为 reprint 输出在安静模式下不可见。
- ex 测试 480 和 481 失败，因为标签文件使用空格而不是制表。
- ex 测试 502 失败，因为安静模式下不读入 .exrc。
- ex 测试 509 失败，因为安静模式下不读入 .exrc。而且退出代码是 1 而不是 2。
- ex 测试 534 失败，因为安静模式下不读入 .exrc。

vim:tw=78:ts=8:noet:ft=help:norl:

version4.txt For Vim version 8.2. 最近更新：2006年2月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

本文档列出 Vim 3.0 和 Vim 4.0 不兼容的不同之处。尽管这里提到 4.0，但也适用于 4.1，4.2 版本等。

本文件对所有从 Vim 3.0 升级的用户很重要。仔细阅读以避免意想不到的问题。

'backup' 选项改变缺省值 backup-changed
备份文件扩展名改变 backup-extension
交换文件结构改变 swapfile-changed
"-w scriptout" 参数改变 scriptout-changed
退格和删除键 backspace-delete
| 转义键改变 escape-bar
键码改变 key-codes-changed
终端选项改变 termcap-changed
'errorformat' 选项改变 errorformat-changed
不再有 'graphic' 选项 graphic-option-gone
不再有 'yankendofline' 选项 ye-option-gone
'icon' 和 'title' 缺省值改变 icon-changed
'highlight' 选项改变 highlight-changed
'tildeop' 和 'weirdinvert' 缩写改变 short-name-changed
"v"、"V" 和 "CTRL-V" 在可视模式的改变 use-visual-cmds
删除插入模式的 CTRL-B 功能 toggle-revins

'backup' 选项改变缺省值 backup-changed

过去 'backup' 缺省值是打开的。结果是在原来文件被覆盖的时候备份文件建立起来。

现在 'backup' 的缺省值是关闭。一旦文件成功的写入，备份文件就删除了。如果你想保留备份文件，在你的 vimrc 文件里置位 'backup'。这个改变的原因是许多人抱怨留下备份文件和 Vi 不兼容。 'backup'

备份文件扩展名改变 backup-extension

过去备份文件的扩展名是 '.bak'。因为别的程序也用这个扩展名，而且有的用户也自己建立用这个扩展名的备份，现在换成了不太明显的 "~"。另外一个好处是这样使用较少的空间，尤其是对短文件名的系统很有效。比如，在 MS-DOS 上 "longfile.c" 和 "longfile.h" 用的备份文件名本来都是 "longfile.bak"；现在他们分别成为 "longfile.c~" 和 "longfile.h~"。

如果你宁可还是使用 ".bak"，你可以设置 'backupext' 选项：

```
:set bex=.bak
```

交换文件结构改变 swapfile-changed

交换文件的内容增加了若干参数。Vim 保存用户名字和被编辑文件的其它信息，使得恢复更加容易，而且能知道交换文件是从哪里来的。交换文件的第一部分可以识别字节顺序或者 sizeof(int) 不同的机器，当你在这样的机器上试图恢复时，你会得到错误信息，提示这不可能。

因为这个改变，3.0 和 4.0 的交换文件不能互换。如果你有一个 3.0 的崩溃对话的交换文件，用 Vim 3.0 恢复之---不要用 4.0。 swap-file

"-w scriptout" 参数改变 scriptout-changed

"vim -w scriptout" 过去用来附加到 scriptout 文件。因为这不合逻辑，现在它用来建立新文件。已有的文件不会被覆盖（为了避免破坏期待这个附加功能的人不小心删除已有的文件）。
[后来这个也删除了] -w

退格和删除键 backspace-delete

3.0 里，删除和退格键在插入模式里都起到退格的功能；它们删除光标左面的字符。在 4.0 里，删除键有了新功能：他删除光标所在的字符，就像在命令行上那样。如果光标在行尾之后并且置位了 'bs'，两行会连接起来。 <Del> i_<Del>

3.0 里退格键总是定义为 CTRL-H，而删除键定义为 CTRL-?。4.0 里退格和删除键的代码通过 termcap 和 termlib 定义，而且可以通过 Unix 的 "stty erase" 调整。这使得用户可以根据他们的键盘重新定义这些键。 <BS> i_<BS>

如果你喜欢插入模式下的退格和删除键的旧行为，在你的 vimrc 里加入一行：

```
inoremap ^? ^H
```

你也许想加上这些行来调整 <BS> 和 <Del> 的代码：

```
set t_kb=^H
set t_kD=^?
```

(用 CTRL-V CTRL-H 输入 ^H，用 CTRL-V CTRL-? 或者 <Del> 输入 ^?。)

如果 t_kb 的值正确，但是 t_kD 的值不对，使用 ":fixdel" 命令。它会根据 t_kb 的值设置 t_kD 的值。这可用于使用不同终端的场合。 :fixdel

如果 ^H 不是识别成 <BS> 或者 <Del>，它的用法类似于退格键。

| 转义键改变 escape-bar

如果 'cpoptions' 里有 'b' 标志位，反斜杠不能用于在映射和缩写命令里转义 '|'。只有 CTRL-V 可以。这是 Vi 兼容的。如果工作在 Vi 兼容模式下，而在映射里为了加入竖杠而包含 "\|"，必须把它改成 "^V|"。参见 :bar 。

键码的内部表示发生了很大的改变。3.0 里每个键用单字节代码表示。这导致了不同字符集使用相同的代码时的很多问题。4.0 里，用三字节的代码来表示。这样不会和一个字符发生混淆。 key-notation

如果你在 vimrc 里用单字节键码定义映射，你需要把他们换成 4.0 的代码。你不需要把它们直接写成三字节的代码，而应该用 <> 括起来的符号表示形式。见下表。该表也列出来 3.0 文档使用的旧名字。

<> 内的键码可以用来直接定义映射。这样，你可以直接复制/粘贴例子或者输入。<> 的表示方法在这里有介绍 <> 。要使之工作，'cpoptions' 里必须不能用 'B' 和 '<' 标志位 'cpoptions' 。

旧名字	新名字	旧代码		旧的 MS-DOS 代码	
		十六进	十进制	十六进	十进制
<ESC>	<Esc>				
<TAB>	<Tab>				
<LF>	<NL> <NewLine> <LineFeed>				
<SPACE>	<Space>				
<NUL>	<Nul>				
<BELL>	<Bell>				
<BS>	<BS> <BackSpace>				
<INSERT>	<Insert>				
<DEL>	<Del> <Delete>				
<HOME>	<Home>				
<END>	<End>				
<PAGE_UP>	<PageUp>				
<PAGE_DOWN>	<PageDown>				
<C_UP>	<Up>	0x80	128	0xb0	176
<C_DOWN>	<Down>	0x81	129	0xb1	177
<C_LEFT>	<Left>	0x82	130	0xb2	178
<C_RIGHT>	<Right>	0x83	131	0xb3	179
<SC_UP>	<S-Up>	0x84	132	0xb4	180
<SC_DOWN>	<S-Down>	0x85	133	0xb5	181
<SC_LEFT>	<S-Left>	0x86	134	0xb6	182
<SC_RIGHT>	<S-Right>	0x87	135	0xb7	183
<F1>	<F1>	0x88	136	0xb8	184
<F2>	<F2>	0x89	137	0xb9	185
<F3>	<F3>	0x8a	138	0xba	186
<F4>	<F4>	0x8b	139	0xbb	187
<F5>	<F5>	0x8c	140	0xbc	188
<F6>	<F6>	0x8d	141	0xbd	189
<F7>	<F7>	0x8e	142	0xbe	190
<F8>	<F8>	0x8f	143	0xbf	191
<F9>	<F9>	0x90	144	0xc0	192
<F10>	<F10>	0x91	145	0xc1	193
<SF1>	<S-F1>	0x92	146	0xc2	194

<SF2>	<S-F2>	0x93	147	0xc3	195
<SF3>	<S-F3>	0x94	148	0xc4	196
<SF4>	<S-F4>	0x95	149	0xc5	197
<SF5>	<S-F5>	0x96	150	0xc6	198
<SF6>	<S-F6>	0x97	151	0xc7	199
<SF7>	<S-F7>	0x98	152	0xc8	200
<SF8>	<S-F8>	0x99	153	0xc9	201
<SF9>	<S-F9>	0x9a	154	0xca	202
<SF10>	<S-F10>	0x9b	155	0xcb	203
<HELP>	<Help>	0x9c	156	0xcc	204
<UNDO>	<Undo>	0x9d	157	0xcd	205
	(没用到)	0x9e	158	0xce	206
	(没用到)	0x9f	159	0xcf	207

终端选项改变 termcap-changed

终端选项的名字改成了 termcap 定义的这些选项的名字。所有的终端选项现在都有 t_xx 这样的名字，其中 xx 是 termcap 的名字。通常这些选项都用不到。除非你有一个错误或者不完整的 termcap 名字，或者你设置不同的光亮选项的值。 terminal-options

注意 有些键没有 termcap 的名字。应该用 <> 形式的名字，不管这么说，这总是一个好主意。

注意 "t_ti" 变成了 "t_mr"（反转输出）而 "t_ts" 变成了 "t_ti"（初始终端模式）。用 "t_ti" 的时候要小心！

旧名字	新名字	含义	
t_cdl	t_DL	删除行数	t_cdl
t_ci	t_vi	光标不可见	t_ci
t_cil	t_AL	插入行数	t_cil
t_cm	t_cm	移动光标	
t_cri	t_RI	光标右侧字符数	t_cri
t_cv	t_ve	光标可见	t_cv
t_cvv	t_vs	光标非常可见	t_cvv
t_dl	t_dl	删除行	
t_cs	t_cs	滚动区域	
t_ed	t_cl	清除显示	t_ed
t_el	t_ce	清除行	t_el
t_il	t_al	插入行	t_il
	t_da	屏幕上方保留显示	
	t_db	屏幕下方保留显示	
t_ke	t_ke	设置终端离开 keypad 传输模式	
t_ks	t_ks	设置终端进入 keypad 传输模式	
t_ms	t_ms	保存以便在高亮模式里移动光标	
t_se	t_se	正常模式（撤销 t_so）	
t_so	t_so	突出（standout）模式	
t_ti	t_mr	反转高亮	
t_tb	t_md	粗体模式	t_tb
t_tp	t_me	结束高亮	t_tp
t_sr	t_sr	反转滚动	

t_te		t_te	结束 termcap 模式	
t_ts		t_ti	进入 termcap 模式	t_ts_old
t_vb		t_vb	可视响铃	
t_csc		t_CS	光标相对滚动区域	t_csc
t_ku	t_ku	<Up>	向上光标键	
t_kd	t_kd	<Down>	向下光标键	
t_kr	t_kr	<Right>	向右光标键	
t_kl	t_kl	<Left>	向左光标键	
t_sku		<S-Up>	shifted 向上光标键	t_sku
t_skd		<S-Down>	shifted 向下光标键	t_skd
t_skr	t_%i	<S-Right>	shifted 向右光标键	t_skr
t_skl	t_#4	<S-Left>	shifted 向左光标键	t_skl
t_f1	t_k1	<F1>	功能键 1	t_f1
t_f2	t_k2	<F2>	功能键 2	t_f2
t_f3	t_k3	<F3>	功能键 3	t_f3
t_f4	t_k4	<F4>	功能键 4	t_f4
t_f5	t_k5	<F5>	功能键 5	t_f5
t_f6	t_k6	<F6>	功能键 6	t_f6
t_f7	t_k7	<F7>	功能键 7	t_f7
t_f8	t_k8	<F8>	功能键 8	t_f8
t_f9	t_k9	<F9>	功能键 9	t_f9
t_f10	t_k;	<F10>	功能键 10	t_f10
t_sf1		<S-F1>	shifted 功能键 1	t_sf1
t_sf2		<S-F2>	shifted 功能键 2	t_sf2
t_sf3		<S-F3>	shifted 功能键 3	t_sf3
t_sf4		<S-F4>	shifted 功能键 4	t_sf4
t_sf5		<S-F5>	shifted 功能键 5	t_sf5
t_sf6		<S-F6>	shifted 功能键 6	t_sf6
t_sf7		<S-F7>	shifted 功能键 7	t_sf7
t_sf8		<S-F8>	shifted 功能键 8	t_sf8
t_sf9		<S-F9>	shifted 功能键 9	t_sf9
t_sf10		<S-F10>	shifted 功能键 10	t_sf10
t_help	t_%1	<Help>	help 键	t_help
t_undo	t_&8	<Undo>	undo 键	t_undo

'errorformat' 选项改变 errorformat-changed

'errorformat' 现在包含多种格式，以逗号分隔。第一个匹配的格式被使用。缺省值现在能捕获许多常见格式。 errorformat

如果你有一种格式包含逗号，前面需要加上反斜杠转义。输入两个反斜杠，因为 ":set" 会吃掉一个。

不再有 'graphic' 选项 graphic-option-gone

'graphic' 选项过去使得 <~> 和 0xa0 之间的字符在屏幕上直接显示。现在 'isprint' 选项替代了它，而且增加了很多可能性。缺省的设置一样；如果你以前在 vimrc 置位了 'graphic' 选项，你只需要看到这一点就行了。

'isprint'

不再有 'yankendofline' 选项 ye-option-gone

'yankendofline' 选项被删除了。你可以用
```
:map Y y$
```

'icon' 和 'title' 缺省值改变 icon-changed

'title' 选项现在缺省只有在原来的标题恢复的时候才设置。不再有 "Thanks for flying Vim" 标题。如果你还是想要，在你的 vimrc 文件加上 ":set title"。 'title'

'icon' 的缺省值现在决定于原来的值是否恢复，和 'title' 类似。如果你不喜欢你的 icon 标题被改变，在 vim 里设置这一行: 'icon'
```
:set noicon
```

'highlight' 选项改变 highlight-changed

'i' 标志位现在意味着斜体高亮，而不是反转。'r' 标志位现在用来代表反转高亮，也就是 'i' 过去的含义。通常你看不到区别，因为多数终端不支持斜体模式，而用反转模式作为替代。 'highlight'

如果 'highlight' 的一种场合不存在，使用 'highlight' 的缺省值，而不是反转模式。

'tildeop' 和 'weirdinvert' 缩写改变 short-name-changed

'to' 换名成 ('tildeop' 的缩写) 'top'。 'tildeop'
'wi' 换名成 ('weirdinvert' 的缩写) 'wiv'。 'weirdinvert'

之所以这么做，是因为 Vi 使用 "wi" 作为 'window' 的缩写，而 'to' 作为 'timeout' 的缩写。这意味着你如果试图设置这些选项，你不会得到错误信息，但是效果不同。

"v"、"V" 和 "CTRL-V" 在可视模式的改变 use-visual-cmds

在可视模式下，"v"、"V" 和 "CTRL-V" 用来终止可视模式。现在这只有在可视模式处于相应的类型时才会，不然可视模式的类型发生相应的改变。现在只有 ESC 用来在所有情况下不做别的，只终止可视模式。 v_V

删除插入模式的 CTRL-B 功能 toggle-revins

插入模式下的 CTRL-B 原来用来切换 'revins' 选项。如果你不知道这一点而不小心按了 CTRL-B，要发现怎么撤销这一点很困难。因为几乎没人用到这一功能，缺省这个功能被关闭了。如果需要，编译前在 feature.h 里定义 RIGHTLEFT。 'revins'

vim:tw=78:ts=8:noet:ft=help:norl:

version5.txt For Vim version 8.2. 最近更新：2017年8月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

欢迎来到 Vim 5.0 版本！

本文档列出 Vim 4.x 和 Vim 5.0 的不同之处。尽管这里提到 5.0，但也适用于 5.1，5.2 版本等。
vi_diff.txt 提供 Vi 和 Vim 5.0 的区别的一个概述。
version4.txt 说明 Vim 3.0 和 Vim 4.0 的区别。

不 兼 容 点： incompatible-5

改变 'compatible' 缺省值 cp-default
改变文本排版命令 "Q" Q-command-changed
改变命令行参数 cmdline-changed
保留自动命令 autocmds-kept
改变 'hidden' 的使用 hidden-changed
改变文本对象命令 text-objects-changed
删除 X-Windows 资源 x-resources
$VIM 的使用 $VIM-use
MS-DOS 和 Win32 的 $HOME 的使用 $HOME-use
改变标签文件格式 tags-file-changed
改变的选项 options-changed
取消插入命令的 CTRL-B i_CTRL-B-gone

新 特 性： new-5

语法高亮 new-highlighting
内建的脚本语言 new-script
Perl 和 Python 支持 new-perl-python
Win32 GUI 版本 added-win32-GUI
VMS 版本 added-VMS
BeOS 版本 added-BeOS
Macintosh GUI 版本 added-Mac
和 Vi 更兼容 more-compatible
从标准输入读取输入 read-stdin
正规表达式模式 added-regexp
重载的标签 tag-overloaded
新命令 new-commands
新选项 added-options
新命令行参数 added-cmdline-args
其它新增 added-various

改 进 improvements-5

编 译 时 的 改 变 compile-changes-5

漏 洞 修 正 bug-fixes-5

5.1 版 本 version-5.1

改动 changed-5.1
新增 added-5.1
修正 fixed-5.1

5.2 版 本 version-5.2
长行可编辑 long-lines
加入文件浏览器 file-browser-5.2
加入对话框 dialogs-added
加入弹出式菜单 popup-menu-added
加入选择模式 new-Select-mode
加入会话文件 new-session-files
用户定义的函数和命令 new-user-defined
新接口 interfaces-5.2
新移植版本 ports-5.2
多字节支持 new-multi-byte
新函数 new-functions-5.2
新选项 new-options-5.2
新 Ex 命令 new-ex-commands-5.2
改动 changed-5.2
新增 added-5.2
修正 fixed-5.2

5.3 版 本 version-5.3
改动 changed-5.3
新增 added-5.3
修正 fixed-5.3

5.4 版 本 version-5.4
引入运行时目录 new-runtime-dir
引入文件类型 new-filetype-5.4
Vim 脚本续行符 new-line-continuation
改进会话文件 improved-sessions
改进自动命令 improved-autocmds-5.4
加密 new-encryption
GTK GUI 移植版本 new-GTK-GUI
菜单改变 menu-changes-5.4
改进 Viminfo improved-viminfo
若干新命令 new-commands-5.4
若干新选项 new-options-5.4
Vim 脚本 new-script-5.4
避免 hit-enter 提示 avoid-hit-enter
改进 quickfix improved-quickfix
正规表达式 regexp-changes-5.4
改动 changed-5.4
新增 added-5.4
修正 fixed-5.4

5.5 版 本 version-5.5
改动 changed-5.5
新增 added-5.5
修正 fixed-5.5

5.6 版 本 version-5.6
改动 changed-5.6

新增 added-5.6
修正 fixed-5.6

5.7 版本 version-5.7
改动 changed-5.7
新增 added-5.7
修正 fixed-5.7

5.8 版本 version-5.8
改动 changed-5.8
新增 added-5.8
修正 fixed-5.8

(译者注：各版本的修正一节，因内容过于琐碎，保留原文未翻译)

不兼容点 incompatible-5

改变 'compatible' 缺省值 cp-default

Vim 5.0 版本试图变得和 Vi 更兼容。这有助于那些使用 Vim 作为 Vi 短期代替品的人，但导致了和 4.x 版本的部分不兼容。

4.x 版本里，'compatible' 选项的缺省值是关闭的。现在缺省打开了。第一件事你就会注意到，"u" 命令撤销了它自己。其它的副作用包括映射可能工作方式不同，甚至完全不工作。

因为许多从 4.x 切换到 5.0 的人觉得这很讨厌，如果 Vim 找到了 vimrc 文件，'compatible' 选项被关闭。这有点像魔术，但它确保 90% 的 Vim 用户不会因为这个改变而受影响。

这意味什么呢?
- 如果你偏好运行在 'compatible' 模式里且没有 vimrc 文件，你不需要做任何事。
- 如果你偏好运行在 'nocompatible' 模式里且有 vimrc 文件，你不需要做任何事。
- 如果你偏好运行在 'compatible' 模式里，但你有 vimrc 文件，你应该在 vimrc 文件先放上这行:
  ```
  :set compatible
  ```
- 如果你偏好运行在 'nocompatible' 模式里，但你没有 vimrc 文件，你可以选择以下方法之一:
  - 建立空的 vimrc 文件 (比如: Unix 的 "~/.vimrc")。
  - 在你的 .exrc 文件或 $EXINIT 里放上这个命令:
    ```
    :set nocompatible
    ```
  - 以 "-N" 参数启动 Vim。

如果你是 Vi 和 Vim 的新用户，强烈建议使用 'nocompatible'。因为 Vi 有很多意想不到的副作用，这个设置可以避免。见 'compatible'。

如果你喜欢 'compatible' 的一些设置，不喜欢另一些，你可以用 'cpoptions' 兼容性选项微调。

如果用 "ex" 或 "gex" 启动 Vim，Vim 总是以兼容模式启动。

改变文本排版命令 "Q" Q-command-changed

"Q" 命令以前使用 'textwidth' 选项指定的宽度对行进行排版。现在这个命令是 "gq" (详情见 gq)。这个改变的原因是 "Q" 是个标准 Vi 的命令，用以进入 "Ex" 模式，而事实上，Vim 现在有了 "Ex" 模式 (详情见 Q)。

如果你仍然想用 "Q" 进行排版，使用此映射:

```
:noremap Q gq
```

如果你同时想用 "Q" 的功能:

```
:noremap gQ Q
```

改变命令行参数 cmdline-changed

现在可以混合命令行文件参数和选项参数。你可以在文件名之后给出选项。例如:

```
vim main.c -g
```

这不能用来编辑 '-' 开始的文件。使用 "--" 参数 --- ，然后:

```
vim -g -- -main.c
```

"-v" 现在意味着启动 Ex 进入 Vi 模式，使用 "-R" 进入只读模式。
旧版: "vim -v file" -v
新版: "vim -R file" -R

"-e" 现在意味着启动 Vi 进入 Ex 模式，使用 "-q" 进入 quickfix。
旧版: "vim -e errorfile" -e
新版: "vim -q errorfile" -q

Ex 模式里的 "-s" 现在意味着进入安静 (批处理) 模式。 -s-ex

"-x" 为加密选项保留，使用 "-f" 来避免启动时进入新 CLI (Amiga)。
旧版: "vim -x file" -x
新版: "vim -f file" -f

Vim 允许有不超过十个的 "+cmd" 和 "-c cmd" 参数。过去，Vim 只执行最后一个。

"-n" 现在覆盖 vimrc 文件里 'updatecount' 的任何设置，但不会覆盖 gvimrc 文件的设置。

保留自动命令 autocmds-kept

5.0 版本以前，有相同的事件、文件名模式和命令的自动命令只能出现一次。对简单的自动命令这没有问题 (比如设置选项值)，但对更复杂的自动命令，相同的命令可能会出现两次。这个限制导致了一些问题。因此，Vim 记住所有的自动命令，并以它们定义的顺序保存。

- 在定义自动命令之前，删除任何可能已经定义过的自动命令。例如:

```
:au! * *.ext
:au BufEnter *.ext ...
```

- 把自动命令放到 ":if" 命令里面。例如:

```
if !exists("did_ext_autocmds")
  let did_ext_autocmds = 1
  autocmd BufEnter *.ext ...
endif
```

- 把自动命令放到不同的自动命令组里，这样你可以在定义之前删除它们 :augroup :

```
augroup uncompress
  au!
  au BufReadPost *.gz ...
augroup END
```

改变 'hidden' 的使用 hidden-changed

在 4.x 版本里，只有一些命令使用 'hidden' 选项。现在任何缓冲区从窗口里消失的命令都使用之。

以前，你可以只阿改变过的缓冲区里使用 ":buf xxx"，那个缓冲区就会隐藏。限制阿，你必须置位 'hidden' 选项来做此工作。

新的行为更简单: Vim 是否隐藏缓冲区不再取决于你使用的特定命令。
- 没有置位 'hidden'，你永远不会得到隐藏的缓冲区。特例包括 ":hide" 和 ":close!" 命令，还有，在罕见的情况下，当你不这么做就会丢失缓冲区里的改动的情况。
- 置位了 'hidden'，你几乎不会卸载缓冲区。例外是 ":bunload" 或 ":bdel" 命令。

":buffer" 现在支持 "!": 放弃当前缓冲区的改动，":bnext"、":brewind" 等亦然。

改变文本对象命令 text-objects-changed

文本对象命令现在有了新的名字。这允许更多的文本对象，并且使这些字符可用于其它的可视模式命令。因为不再提供单字符的名字，文本对象名字现在需要两个字符。第一个字符总是 'i' 或 'a'。

旧版	新版		
a	aw	一个单词	v_aw
A	aW	一个字串	v_aW
s	as	一个句子	v_as
p	ap	一个段落	v_ap
S	ab	一个 () 块	v_ab
P	aB	一个 {} 块	v_aB

有另外一组文本对象，它们以 "i" 开始，代表 "inner" (内部)。它们选择相同的对象，但排除空白。

删除 X-Windows 资源 x-resources

Vim 不再支持下面的 X 资源:
- boldColor

- italicColor
- underlineColor
- cursorColor

Vim 现在使用高亮组设定颜色。这避免和使用粗体字体的混淆，其中隐含着使用特定颜色。见 :highlight 和 gui-resources 。

$VIM 的使用 $VIM-use

Vim 现在使用 VIM 环境变量来找到所有的 Vim 系统文件。其中包括全局 vimrc、gvimrc 和 menu.vim 文件以及所有的在线帮助和语法文件。见 $VIM 。从 5.4 版本开始，也可以使用 $VIMRUNTIME 。
Unix 上，Vim 在 "make install" 的时候设置 $VIM 的缺省值。
如果没有设置 $VIM，它的缺省值取自 'helpfile' 所在的目录，减去后面的 "/doc/help.txt"。

MS-DOS 和 Win32 的 $HOME 的使用 $HOME-use

MS-DOS 和 Win32 版本的 Vim 现在在搜索 vimrc 或 exrc 文件和读写 viminfo 文件的时候，检查 $HOME。以前，Vim 在这些系统上使用 $VIM，但在有多个用户的系统上这会有问题。现在 Vim 只有在 $HOME 没有设置或者在 $HOME 里没找到文件的时候才使用 $VIM。见 _vimrc 。

改变标签文件格式 tags-file-changed

标签文件里，只有制表可以用来分隔域。这样，文件名里就可以允许有空格，而且仍然和 Vi 兼容。以前的 Vim 版本里，任何的空白都可以用来分隔域。如果你的文件不使用单个制表分隔域，编辑 tags 文件，并执行下面的命令:

```
:%s/\(\S*\)\s\+\(\S*\)\s\+\(.*\)/\1\t\2\t\3/
```

改变的选项 options-changed

'errorfile' 的缺省值从 "errors.vim" 改为 "errors.err"。原因是只有 Vim 脚本才应该有 ".vim" 扩展名。

":make" 命令不在使用 'errorfile' 选项。这样避免了 ":make" 命令的输出覆盖手动保存的错误文件。":make" 使用 'makeef' 选项代替。它也可以产生唯一的名字，防止同时运行的 ":make" 命令覆盖对方的文件。

'insertmode' 置位以后，更多的东西改变了:
- 普通模式里的 <Esc> 进入插入模式。
- 插入模式里的 <Esc> 不退出插入模式。
- 执行 ":set im" 时，立即进入插入模式。

'fileformat' (以前是 'textmode' 选项) 和缓冲区的初始格式不同时，Vim 把缓冲区当作已改变。

取消插入命令的 CTRL-B i_CTRL-B-gone

Vim 编译时加入 +rightleft 特性时，你可以使用 CTRL-B 切换 'revins' 选项。不幸的是，有的人想按 CTRL-V 或者 CTRL-N 的时候不小心按了 'B' 键，然后不知道怎么撤销此功能。因为切换 'revins' 选项可以很容易用下面的映射执行，CTRL-B 键的该功能被取消了。为此目的，你仍然可用 CTRL-_ 键 i_CTRL-_ 。

```
:imap <C-B> <C-O>:set revins!<CR>
```

新 特 性 new-5

语法高亮 new-highlighting

Vim 现在有了非常灵活的为所有文件件类型设定高亮的方式。
见 syntax 。一个小结:

```
:syntax on
```

语法高亮可以设置色彩和属性，也可以用 'highlight' 的 ':' 标志位为其它高亮项目设置色彩和属性。所有的高亮项目被分配了一个高亮组，它指定这些项目的高亮方式。见 :highlight 。缺省颜色也有改进。

你可以用 "Normal" 组来设定色彩终端的缺省前/背景色。对于 GUI，你同样可以使用该铸来指定字体。

"2html.vim" 脚本可以用来把任何有语法高亮的文件转化为 HTML。其中的色彩和你在 Vim 看到的一模一样。使用 HTML 浏览器，你可以打印彩色的文件。

内建的脚本语言 new-script

一些附加的命令和表达式计算器是你可以编写简单但强大的脚本。命令包括 ":if" 和 ":while"。表达式可以处理数值和字符串。你可以用 '=' 寄存器来直接插入表达式的结果。见 expression 。

Perl 和 Python 支持 new-perl-python

Vim 可以用 ":perldo"、":perl" 等调用 Perl 命令。见 perl 。
补丁由 Sven Verdoolaege 和 Matt Gerassimoff 提供。

Vim 可以用 ":python" 和 ":pyfile" 调用 Python 命令。见 python 。

两者都必须在编译时打开才能得到。

Win32 GUI 版本 added-win32-GUI

GUI 被移植到 MS Windows 95 和 NT 上。Windows 用户现在可以使用 X11 GUI 的所有特性了。 gui-w32
这也修正了在 Windows 95 里运行 Win32 控制台的问题，那里，控制台的支持总是很

差。
还有一个支持 OLE automation 接口的版本。 if_ole.txt
使用 VisVim DLL，Vim 可以和 Microsoft Developer Studio 集成。
使用 Borland C++，可以产生 gvim 的 DLL 版本 (Aaron)。

VMS 版本 added-VMS

Vim 现在可以用于 VMS 系统。移植由 Henk Elbers 提供。
还没有充分测试。但应该可以用了。
对不起，没有文档。

BeOS 版本 added-BeOS

Vim 可以用于 BeOS 系统上 (包括 BeBox)。(Olaf Seibert)
见 os_beos.txt 。

Macintosh GUI 版本 added-Mac

Vim 现在可以用于 Macintosh。(Dany St-Amant)
还没有充分测试，要小心！
见 os_mac.txt 。

和 Vi 更兼容 more-compatible

现在，有一个真正的 Ex 模式。以 "Q" 命令执行，或者启用可执行程序 "ex" 或 "gex"。 Ex-mode

如果 'cpoptions' 里包含 'u' 标志位，即使在 Vi 兼容模式里，也允许多层撤销。
CTRL-R 用于重复撤销或重做 (像 Nvi 里的 ".")。

从标准输入读取输入 read-stdin

使用 "-" 命令行参数时，Vim 从标准输入读入文本输入。
这可以用来在管道的末端使用 Vim:

```
grep "^a.*" *.c | vim -
```

见 -- 。

正规表达式模式 added-regexp

增加了指定匹配原匹配次数的范围: "\{a,b}"。 /\{
增加了 "最短匹配" 正规表达式 "\{-}" (Webb)。
增加了 "\s"，匹配空白字符。可以替换 "[\t]"。 /\s
增加了 "\S"，匹配非空白字符，可以替换 "[^ \t]"。 /\S

重载的标签 tag-overloaded

使用 C++ 这样的语言的时候，可能有多个标签对应相同的标签名。增加了命令，可以跳转到任何一个重载的标签：

:tselect	列出匹配的标签，并跳转到其中一个。
:stselect	同上，并分割窗口。
g_CTRL-]	使用光标所在的单词执行 ":tselect"。

":ta {tagname}" 如有多个匹配，之后：

:tnext	转到下一个匹配的标签。
:tprevious	转到前一个匹配的标签。
:trewind	转到第一个匹配的标签。
:tlast	转到最后一个匹配的标签。

":tag" 命令现在也接受通配符。执行命令行标签补全时，也可以（在最后）使用大小写不敏感的匹配。

新命令 new-commands

:amenu	定义所有模式下的菜单，在插入模式里插入 CTRL-O，可视模式下 ESC，命令行模式下 CTRL-C。"amenu" 用于定义缺省菜单和 Syntax 菜单。
:augroup	设置后面的自动命令使用的组。自动命令的编组允许了指定组内部的删除。
:crewind :clast	转到第一个错误。 转到最后一个错误，
:doautoall	在所有载入的缓冲区里执行自动命令。
:echo	回显表达式参数。可以用来显示包含变量的消息。
:execute	执行表达式参数。可以用来构造任意的 Ex 命令。
:hide	和 ":close" 类似。
:if	条件执行，用于内建脚本语言。
:intro	显示介绍文字。Vim 启动式，如果没有文件参数，总是执行它。
:let	给内部变量赋值。
:omap	只给操作符等待模式定义映射。从而可以映射文本命令。
:redir	把消息的输出重定向到文件里。
:update	缓冲区改变时写回。
:while	While 循环，用于内建脚本语言。

可视模式：

v_O	可视列块模式里的 "O" 水平移动光标到另一个角。

v_D 可视列块模式里的 "D" 删到行尾。

插入模式:

i_CTRL-] 切换缩写、无须插入任何字符。

新选项 added-options

'background' 用于选择缺省的高亮色。"syntax.vim" 也用于选择语法颜色。通常自动根据使用的终端自动设置。

'complete' 指定插入模式补全如何完成。

'eventignore' 可以暂时屏蔽自动命令。

'fileformat' 目前的文件格式。替换了 'textmode'。

'fileformats' 目前的文件格式列表。替换了 'textauto'。
新的特点还包括对 Macintosh 格式的支持: 单个 <CR> 分隔行。
'fileformats' 在 MS-DOS、Win32 和 OS/2 的缺省是 "dos,unix", 即使 'compatible' 置位也是如此。Unix 类型的文件在 'fileformats' 为空的时候无论如何都无法工作。

'guicursor' 设置光标在不同模式里的形状和闪烁。缺省为插入和替换模式和等待操作符时调整光标。缺省也打开闪烁。

'fkmap' 波斯语键盘映射。

'hlsearch' 高亮所有的最近使用的搜索模式的匹配。

'hkmapp' '根据读音的希伯来语' 映射 (Ilya Dogolazky)

'iconstring' 如果非空, 定义图标名字。(5.2 版本: 按本义使用该字符串, 可是用换行建立两行。)

'lazyredraw' 执行宏、寄存器或其它不靠输入的命令时, 不重画屏幕。

'makeef' ":make" 使用的错误文件。"##" 被唯一的数字替换。可以避免两个 Vim 会话覆盖彼此的错误文件的情况。Unix 的缺省是 "/tmp/vim##.err"; Amiga 是 "t:vim##.Err", 其它 "vim##.err"。

'matchtime' 如果打开了 'showmatch', 1/10 秒计算的括号匹配显示的时间。类似于 Nvi。

'mousehide' 输入文本时隐藏鼠标指针。

'nrformats' 定义使用 CTRL-A 和 CTRL-X 命令时 Vim 考虑的数值的基底。缺省为: "hex,octal"。

'shellxquote' 围绕整个外壳命令的附加引号, 包括重定向。

'softtabstop' 使得输入的感觉就像 tabstop 设为此值一样, 而不用实际改变 'tabstop' 的值。使得保持 'ts' 为 8 更简单, 而实际上输入 <Tab> 仍然可以得到四个空格。

'titlestring'　如果非空，窗口标题字符串。(5.2 版本：按本义使用该字符串，可是用换行建立两行。)

'verbose'　详细程度。可以用来显示初始化时使用了哪个 .vimrc、.exrc、.viminfo 文件等等。也可以显示执行中的自动命令。也可以用 "-V" 命令行参数设置。

新命令行参数　added-cmdline-args

-U　指定要用的 gvimrc 文件。类似于指定 vimrc 的 "-u"。

-V　设置 'verbose' 选项。比如 "vim -V10"。

-N　以非兼容模式启动。

-C　以兼容模式启动。

-Z　以受限模式启动，禁止外壳命令。也可以用执行可执行程序 "rvim" 完成。

-h　显示使用信息并退出。

其它新增　added-various

增加了和 SNiFF+ 连接的功能 (Toni Leherbauer 提交)。Vim 可以用做 SNiFF 的编辑器。没有文档...

要产生漏洞报告，可用包含的 bugreport.vim 脚本。
可以这样使用：":so $VIMRUNTIME/bugreport.vim"，它在当前目录建立文件 "bugreport.txt"。 bugs

":normal" 命令增加了范围。现在你可以为某范围内的每行重复相同的命令。
:normal-range

加入了波斯语的支持 (Shiran)。只有在编译时打开才能使用。见 farsi 。

改 进　improvements-5

性能：
- 如果设置了 'showcmd'，映射的执行过去会慢的多，因为需要频繁刷新输出。这对你执行 "至关重要的" 宏且需要置位 'showcmd' 时太有帮助了。
- 包含了关于标签文件进行二分法查找的补丁。(David O'Neill)
  可以通过复位 'tagbsearch' 选项关闭。
- 重复插入时，不更新标尺 (会显著减慢)。
- Unix 上，文件名扩展现在在内部完成了，不用启动外壳来处理。
- 用 expand_env() 扩展环境变量，而不用调用外壳。使得 ":so $VIMRUNTIME/syntax/syntax.vim" 快的 多 了。
- 减少了光标定位的输出：用 CR-LF 移动到下面数行的前面数列。使用 termios 时，不

输出 CR 两次。
- 优化了光标定位。用 CR、BS 和 NL，如果产生的代码比光标绝对定位要短的话。
- 重复插入 "1000ii<Esc>" 时，关闭重画。
- 使长行的 "d$" 或 "D" 快的多（一次删除所有的字符，而不是一个一个删）。
- 用地一个字母访问选项表，而不是从头开始搜索。
- 使用 highlight_attr[]，而不是从 'highlight' 字符串里搜索，这使设置特殊高亮属性快的多。
- 重画关闭时，不显示模式。
- 设置选项时，只有在需要的时候才重画屏幕。
- 通过第一个字母的查找表，提高 Ex 命令的性能。

选项：

'cinoptions'	加入 'g' 标志位，用于 C++ 作用域定义。
'cpoptions'	加入 'E' 标志位：禁止抽出、删除等作用于空文本区域。缺省允许抽出空区域。如果包含 'E'，在空行上 "y$" 现在被处理为错误（Vi 兼容）。 加入 'j' 标志位：连接只在 '.' 之后，而不是 '?' 或 '!' 之后加入两个空格。 加入 'A' 标志位：不给出 ATTENTION 消息。 加入 'L' 标志位：如果不给出并且置位了 'list'，'textwidth' 排版如同没有置位 'list' 那样。 加入 'W' 标志位： 使 ":w!" 的行为类似 Vi：不覆盖只读文件或者别人拥有的文件。
'highlight'	加入 '@' 标志位，用于置位 'list' 时，屏幕最后一行之后的 '@' 字符和行尾的 '$'。 加入 'i' 标志位：设置 'incsearch' 的高亮。缺省使用 "IncSearch" 高亮组，它是 "Visual" 的链接。 不再允许 'highlight' 的 'h' 标志位（3.0 之后就不再用了）。
'guifont'	仅限于 Win32 GUI：设置为 "*" 时弹出字体请求器。
'guipty'	缺省多，因为太多人需要它了。
'path'	可以包含通配符，"**" 可用于搜索整个树。
'shortmess'	加入 'I' 标志位已避免介绍文字。
'viminfo'	加入 '%' 标志位：在 viminfo 文件里保存缓冲区列表。

- 提高了 'maxmem' 和 'maxmemtot' 在 Unix 和 Win32 上的缺省值。许多机器现在有更多的 RAM，因为价钱便宜多了。
- 实现了 ":set all&"，设置所有选项为其缺省值。 :set

交换文件：
- 不为只读文件建立交换文件。只有在第一次修改时，或在需要的内存太高的时候建立交换文件。 swap-file
- 如有可能，使得交换文件 "隐藏"。Unix 上是通过在交换文件前附加句号完成的。如果使用长文件名，DJGPP 和 Win32 版本也在前面附加句号，这样在编辑 Unix 文件系统上装载的文件时就有效果了。 :swapname MSDOS 上， 不 隐藏文件属性，因为这会导致 share.exe 的问题。
- 'updatecount' 缺省总是非零，即使在 Vi 兼容模式也是如此。
  这意味着总会有交换文件，它可以用来恢复。

标签：
- 包含了 ctags 2.0 (Darren Hiebert)。静态标签的语法从
  {tag}:{fname} {fname} {command}
  改为
  {tag} {fname} {command};" file:
  这样，分析更快，更短，而且和 Vi 兼容。仍然接受旧的格式，除非在 src/feature.h

里关闭（见 OLD_STATIC_TAGS）。 tags-file-format
- 标签的补全现在也包含了其它文件的静态标签，放在最后。
- 包含了 Stephen Riehm 的 "shtags"。
- 寻找标签匹配但文件不存在时，继续寻找下一个匹配。有助于为不同版本的源代码使用相同的标签文件（通过链接）。
- 和其它文件的全局匹配相比，给当前文件的全局标签匹配更高优先级。

包含 xxd V1.8 版本（Juergen Weigert）。

自动命令：
- VimLeave 自动命令再写入 viminfo 文件之后执行，而不是在它之前。 VimLeave
- 允许在执行自动命令时修改之。可用于自修改的自动命令。（Goldberg 的想法）
- 使用两个或更多模式的自动命令时，不能把 ":if/:endif" 分到两行上。现在所有匹配的自动命令在一个 do_cmdline() 里执行。
- 自动命令不再改变 "." 重复的命令。
- 执行自动命令之后恢复搜索模式。则避免了自动命令弄乱了 'hlsearch' 高亮。
- 试图执行自动命令时，同时尝试用短文件名匹配模式。有助于短文件名和长文件名不同的场合（扩展后的符号链接）。 autocmd-patterns
- 使得 ":autocmd" 的输出更短且更好。
- ":autocmd" 在定义时扩展 <sfile>。 <sfile>
- ":autocmd" 加入 "nested" 标志位，允许嵌套。 autocmd-nested
- ":autocmd" 加入 [group] 参数。覆盖目前设置的组。 autocmd-groups
- 新事件：

BufUnload	缓冲区卸载前
BufDelete	缓冲区从缓冲区列表删除前
FileChangedShell	执行外壳命令后，文件修改时间被改变
User	用户定义的自动命令

- 如果 BufRead* 自动命令置位了 'modified'，过去在它之后又会复位。现在会记住 ":set modified"。

GUI：
- 改进 GUI 滚动条在重画比产生滚动事件时的处理。
- "vim -u NONE" 现在也不再载入 .gvimrc 和其它 GUI 初始化设置。 -u
  用 "-U" 来使用别的 gvimrc 文件。 -U
- 处理外部命令的 CTRL-C，也包括支持 "setsid()" 的系统。
- 启动 GUI 时，限制窗口大小为屏幕大小。
- 缺省菜单从 $VIMRUNTIME/menu.vim 里读入。这使得缺省菜单可以定制。 menu.vim
- 改进缺省菜单。加入 File/Print、Window 菜单、Syntax 菜单等等。
- 加入 ":menu" 命令的优先级。现在每个菜单可以放在你想要的地方，和菜单定义的顺序相互独立。 menu-priority

在 Windows 95 上运行 Win32 控制台版本时，介绍文字屏幕给出警告，因为在 Windows 95 上运行此版本会有问题。 win32-problems

":substitute" 加入 'e' 标志位：找不到匹配不会抱怨（Campbell）。 :s

映射中使用搜索命令时，只把最后一个保存进历史。避免过长的映射滥用历史。

检查启动模式时，忽略 "ex"、"view" 和 "gvim" 之后的所有字符。可用来使用 "gvim5" 等名字。 gvim "gview" 启动只读模式的 GUI。 gview

调整窗口大小时，尽可能使光标的相对位置保持不变。（Webb）

":all" 和 ":ball" 不再为同一缓冲区关闭然后然后打开窗口。避免选项、跳转表和其它

信息的丢失。

现在如果 Vim 编译时不带 GUI，忽略 "-f" 命令行参数。 -f

可视列块模式里，鼠标右键选择最近的一角。

修改了 DOS 等的缺省映射。删除 DOS 专用的映射，用 Windows 的那些就可以了。加入 Shift-Insert、Ctrl-Insert、Ctrl-Del 和 Shift-Del。

修改 ":jumps" 输出的编号，使你能够看见 {count} CTRL-O 到哪里。 :jumps

用 "~" 代替 $HOME 现在适用于所有系统。 $HOME

Unix: 除了 CTRL-C 以外，也可用 tty 设置的 INTR 字符。有人把 INTR 设为 DEL。

允许 ":help" 命令参数里的 <LF> 结束帮助命令，从而可以跟随下一个命令。

" #if" 开头的行上按 "%" 过去不跳转到匹配的 "#else"。
'cpo' 包含 '%' 时，'%' 不识别 "#if"、"#else" 等。 %

改进了用 "CTRL-N"、"CTRL-P" 和 "CTRL-X" 的插入模式扩展 ins-completion :
- 加入 'complete' 选项。
- 置位 'nowrapscan' 而没有匹配时，错误信息里报告搜索的方向。
- 重复执行 CTRL-X 命令加入匹配之后的单词/行。
- 加入扩展时，接受单个字符的匹配。
- 重复的 CTRL-X CTRL-N 不把撤销断开，这样 "." 会重复整个插入过程。也修正了不能在用 CTRL-N 插入的单词上退格的问题。

插入模式用 CTRL-E 或 CTRL-Y 从上/下行复制字符时，不再使用 'textwidth'。
i_CTRL-E

在参数表内移动的命令，如 ":n" 和 ":rew"，保持文件原有的光标位置。(主要为了和 Vi 兼容)。

Vim 现在记住每个缓冲区的 '< 和 '> 位置标记。修正了在一个缓冲区删除行后使另一缓冲区的 '< 和 '> 位置标记非法的问题。 '<

MSDOS、Unix 和 OS/2: 如果没有设置 $VIM，使用可执行文件的路径。用可执行文件路径代替 $VIM 时如果存在 "src/"，删除之。编译后直接运行 Vim 时，这个方法使得 Vim 可以找到文档和语法文件。 $VIM

用 <Esc> 退出可视模式时，光标位于可视区域开始处 (就像执行操作符之后的情况)。

Win32 和 Unix 版本: 删除外部命令的 1100 字符限制。

加入 ":edit +command" 参数包含空格的可能性。在它之前加上反斜杠。 +cmd

恢复后，提供 BufReadPost 自动命令。 :recover

加入 "os2ansi"，OS/2 控制台，的色彩支持。(Slootman)

% 为空也能接受 "%:p:h"。 :_%

包含 "<sfile>": ":source" 命令的文件名。 <sfile>

加入 "<Bslash>" 特殊字符。有助于避免映射和菜单里的多重反斜杠。

帮助窗口里，双击跳转到光标所在的标签（类似 CTRL-]）。

<C-Left> 和 <C-Right> 现在和 <S-Left> 和 <S-Right> 类似，正向/反向移动一个单词
(和 Windows 兼容)， <C-Left>

删除 .vimrc 文件对 ":version" 命令的需要。没有任何用处。用 ":if" 可以处理版本间的差异。 :version

MS-DOS、Win32 和 OS/2: 自动命令比较文件时，不区别路径分隔符 '/' 和 '\'。

新的 termcap 选项:
"mb": 闪烁。只能赋予其它高亮选项之一。 t_mb
"bc": 退格字符。 t_bc
"nd": GUI 上用于向右移动光标，避免删除从最后一个粗体字符开始的整行像素行。
t_nd
"xs": 对 hpterm 而言，高亮不用覆盖来删除。和 'weirdinvert' 组合使用。hpterm 现在可以用可视模式了。 t_xs

Unix: 补丁和备份文件的时间设置为和原来文件相同的时间。(Hiebert)。

Amiga: QuickFix 模式不再打开另一个窗口。现在可以用外壳命令。

加入 David Binette 的 decmouse 补丁。只要编译时打开。现在可以使用 Dec 和 Netterm 鼠标。

加入 '#' 寄存器: 轮换文件名 quote# 。":dis" 命令显示 '#' 寄存器。 :display

Unix 上从 'isfname' 的缺省值里删除 ':'。总是检查文件名里的 "://"。MS-DOS 也检查 ":\\"。

"K"eyword 命令加入计数，如果 'keywordprg' 为 "man"，计数插入 man 命令。"2K" 产生 "!man 2 <cword>"。 K

相对路径名上用 "gf" 删除文件名里的 "../"，和标签文件里对文件名的处理相同。 gf

结束纪录时，不把记录的寄存器设置为放置 (put) 用的缺省寄存器。

使用 "!!" 时，命令行上不放 ":5,5!"，而是 ":.!"。还有一些改进，可能的情况下把具体的行号换成 "." 或 "$"。

MSDOS 等: $VIM/viminfo 换名为 $VIM/_viminfo。和下面这些更一致: .vimrc/_vimrc 和 .viminfo/_viminfo

文件名大小写不敏感的系统 (MSDOS、Amiga) 上，给文件名排序时忽略大小写。也包括缓冲区的名字。

从标准输入读取如果失败，改为从标准错误读取 (可用于 "foo | xargs vim")。

32 位 MS-DOS 版本: 把 csdpmi3 改为 csdpmi4。

<C-Left> 和 <C-Right> 改为跳过字串 (WORD) 而不是单词 (word)。

Unix 之外的系统现在也支持覆盖文件时关于修改时间被改变的警告。

Unix: 修改 configure 的缺省，使之和 Makefile 的缺省一致: 这包括 GUI、Perl 和 Python 等方面。

一些 Motif 的版本需要 "-lXpm"。configure 时加入这方面的检查。

link 行不加入 "-L/usr/lib"，一些系统可能有问题。

编 译 时 的 改 变 compile-changes-5

编译时，允许通过修改 src/feature.h 的一行，简单的选择 minimal、normal 还是 maximal 特性包。
DOS16 版本使用 minimal 特性包编译，以免内存太快溢出。
Win32、DJGPP 和 OS/2 版本使用 maximal 特性包，因为它们有足够的内存。
Amiga 可以有 normal 和 maximal 特性包的不同版本。

在 Unix 版本的 Makefile 里加上了 "make test"。允许快速检查是否多数 "普通" 命令可以正确工作。也测试一些特定的命令。

DJGPP 版本里，加入 setlocale() 支持代码页。

autoconf:
- autoconf 增加 -lXdmcp 检查。
- 包含 -lXmu 的检查，为此不再需要编辑 Makefile。
- 切换到 autoconf 2.12。
- configure 增加对 <poll.h> 的检查。Linux 上包含 Perl 的时候看来需要?
- 现在，在 termcap 之前检查 termlib。
- configure 增加对 strncasecmp()、stricmp() 和 strnicmp() 检查。加入 vim_stricmp()，如果没有 stricmp() 库函数的话。
- 在 configure 里使用 "datadir"，而不是我们自己对 HELPDIR 的检查。

从 Makefile.manx 里删除 "make proto"。没有很多的 #ifdef，无法使它工作。

从 proto.h 的 paths 里删除 "proto/"。Mac 移植版本需要如此。

Makefile.mint 进行了大改动。现在它包含 Unix Makefile。

Makefile.b32 加了 Dos16 的支持 (Makefile.b32 换名为 Makefile.bor)。

所有的源程序现在编辑时使用 tabstop 为 8 而不是 4，对于调试和其它工具的使用，这样更合适。为了方便编辑。'softtabstop' 设为 4。

Unix: 加入 "link.sh" 脚本，它从 link 命令里删除一些不必要的库。

缺省，HPUX 上不使用二合字母。除非定义了 HPUX_DIGRAPHS 才如此。
digraphs-default

漏 洞 修 正 bug-fixes-5

注意： 这些修正有的只适用于 4.6 之后但在 5.0 之前的测试版。

When doing ":bdel", try going to the next loaded buffer. Don't rewind to the start of the buffer list.

mch_isdir() for Unix returned TRUE for "" on some systems.

Win32: 'shell' set to "mksnt/sh.exe" breaks ":!" commands. Don't use backslashes in the temp file names.

On linux, with a FAT file system, could get spurious "file xxx changed since editing started" messages, because the time is rounded off to two seconds unexpectedly.

Crash in GUI, when selecting a word (double click) and then extend until an empty line.

For systems where isdigit() can't handle characters > 255, get_number() caused a crash when moving the mouse during the prompt for recovery.

In Insert mode, "CTRL-O P" left the cursor on the last inserted character. Now the cursor is left after the last putted character.

When quickfix found an error type other than 'e' or 'w', it was never printed.

A setting for 'errorfile' in a .vimrc overruled the "-q errorfile" argument.

Some systems create a file when generating a temp file name. Filtering would then create a backup file for this, which was never deleted. Now no backup file is made when filtering.

simplify_filename() could remove a ".." after a link, resulting in the wrong file name. Made simplify_filename also work for MSDOS. Don't use it for Amiga, since it doesn't have "../".

otherfile() was unreliable when using links. Could think that reading/writing was for a different file, when it was the same.

Pasting with mouse in Replace mode didn't replace anything.

Window height computed wrong when resizing a window with an autocommand (could cause a crash).

":s!foo!bar!" wasn't possible (Vi compatible).

do_bang() freed memory twice when called recursively, because of autocommands (test11). Thanks to Electric Fence!

"v$d" on an empty line didn't remove the "-- VISUAL --" mode message from the command-line, and inverted the cursor.

":mkexrc" didn't check for failure to open the file, causing a crash. (Felderhoff).

Win32 mch_write() wrote past fixed buffer, causing terminal keys no longer to be recognized. Both console and GUI version.

Athena GUI: Crash when removing a menu item. Now Vim doesn't crash, but the reversing of the menu item is still wrong.

Always reset 'list' option for the help window.

When 'scrolloff' is non-zero, a 'showmatch' could cause the shown match to be in the wrong line and the window to be scrolled (Acevedo).

After ":set all&", 'lines' and 'ttytype' were still non-default, because the defaults never got set. Now the defaults for 'lines' and 'columns' are set after detecting the window size. 'term' and 'ttytype' defaults are set when detecting the terminal type.

For (most) non-Unix systems, don't add file names with illegal characters when expanding. Fixes "cannot open swapfile" error when doing ":e *.burp", when there is no match.

In X11 GUI, drawing part of the cursor obscured the text. Now the text is drawn over the cursor, like when it fills the block. (Seibert)

when started with "-c cmd -q errfile", the cursor would be left in line 1. Now a ":cc" is done after executing "cmd".

":ilist" never ignored case, even when 'ignorecase' set.

"vim -r file" for a readonly file, then making a change, got ATTENTION message in insert mode, display mixed up until <Esc> typed. Also don't give ATTENTION message after recovering a file.

The abbreviation ":ab #i #include" could not be removed.

CTRL-L completion (longest common match) on command-line didn't work properly for case-insensitive systems (MS-DOS, Windows, etc.). (suggested by Richard Kilgore).

For terminals that can hide the cursor ("vi" termcap entry), resizing the window caused the cursor to disappear.

Using an invalid mark in an Ex address didn't abort the command.

When 'smarttab' set, would use 'shiftround' when inserting a TAB after a space. Now it always rounds to a tabstop.

Set '[and '] marks for ":copy", ":move", ":append", ":insert", ":substitute" and ":change". (Acevedo).

"d$" in an empty line still caused an error, even when 'E' is not in 'cpoptions'.

Help files were stored in the viminfo buffer list without a path.

GUI: Displaying cursor was not synchronized with other displaying. Caused several display errors. For example, when the last two lines in the file start with spaces, "dd" on the last line copied text to the (then) last line.

Win32: Needed to type CTRL-SHIFT-- to get CTRL-_.

GUI: Moving the cursor forwards over bold text would remove one column of bold pixels.

X11 GUI: When a bold character in the last column was scrolled up or down, one column of pixels would not be copied.

Using <BS> to move the cursor left can sometimes erase a character. Now use "le" termcap entry for this.

Keyword completion with regexp didn't work. e.g., for "b.*crat".

Fixed: With CTRL-O that jumps to another file, cursor could end up just after the line.

Amiga: '$' was missing from character recognized as wildcards, causing $VIM sometimes not to be expanded.

":change" didn't adjust marks for deleted lines.

":help [range]" didn't work. Also for [pattern], [count] and [quotex].

For 'cindent'ing, typing "class::method" doesn't align like a label when the second ':' is typed.
When inserting a CR with 'cindent' set (and a bunch of other conditions) the cursor went to a wrong location.
'cindent' was wrong for a line that ends in '}'.
'cindent' was wrong after "else {".

While editing the cmdline in the GUI, could not use the mouse to select text from the command-line itself.

When deleting lines, marks in tag stack were only adjusted for the current window, not for other windows on the same buffer.

Tag guessing could find a function "some_func" instead of the "func" we were looking for.

Tags file name relative to the current file didn't work.

":g/pat2/s//pat2/g", causing the number of subs to be reported, used to cause a scroll up. Now you no longer have to hit <CR>.

X11 GUI: Selecting text could cause a crash.

32 bit DOS version: CTRL-C in external command killed Vim. When SHELL is set to "sh.exe", external commands didn't work. Removed using of command.com, no longer need to set 'shellquote'.

Fixed crash when using ":g/pat/i".

Fixed (potential) crash for X11 GUI, when using an X selection. Was giving a pointer on the stack to a callback function, now it's static.

Using "#" and "*" with an operator didn't work. E.g. "c#".

Command-line expansion didn't work properly after ":*". (Acevedo)

Setting 'weirdinvert' caused highlighting to be wrong in the GUI.

":e +4 #" didn't work, because the "4" was in unallocated memory (could cause a crash).

Cursor position was wrong for ":e #", after ":e #" failed, because of changes to the buffer.

When doing ":buf N", going to a buffer that was edited with ":view", the readonly flag was reset. Now make a difference between ":e file" and ":buf file": Only set/reset 'ro' for the first one.

Avoid hit-enter prompt when not able to write viminfo on exit.

When giving error messages in the terminal where the GUI was started, GUI escape codes would be written to the terminal. In an xterm this could be seen as a '$' after the message.

Mouse would not work directly after ":gui", because full_screen isn't set, which causes starttermcap() not to do its work.

'incsearch' did not scroll the window in the same way as the actual search. When 'nowrap' set, incsearch didn't show a match when it was off the side of the screen. Now it also shows the whole match, instead of just the cursor position (if possible).

":unmap", ":unab" and ":unmenu" did not accept a double quote, it was seen as the start of a comment. Now it's Vi compatible.

Using <Up><Left><Left><Up> in the command-line, when there is no previous cmdline in the history, inserted a NUL on the command-line.

"i<Esc>" when on a <Tab> in column 0 left the cursor in the wrong place.

GUI Motif: When adding a lot of menu items, the menu bar goes into two rows. Deleting menu items, reducing the number of rows, now also works.

With ":g/pat/s//foo/c", a match in the first line was scrolled off of the screen, so you could not see it.
When using ":s//c", with 'nowrap' set, a match could be off the side of the screen, so you could not see it.

When 'helpfile' was set to a fixed, non-absolute path in feature.h, Vim would crash. mch_Fullname can now handle file names in read-only memory. (Lottem)

When using CTRL-A or CTRL-@ in Insert mode, there could be strange effects when using CTRL-D next. Also, when repeating inserted text that included "0 CTRL-D" or "^ CTRL-D" this didn't work. (Acevedo)
Using CTRL-D after using CTRL-E or CTRL-Y in Insert mode that inserted a '0' or '^', removed the '0' or '^' and more indent.

The command "2".p" caused the last inserted text to be executed as commands. (Acevedo)

Repeating the insert of "CTRL-V 048" resulted in "^@" to be inserted.

Repeating Insert completion could fail if there are special characters in the text. (Acevedo)

":normal /string<CR>" caused the window to scroll. Now all ":normal" commands are executed without scrolling messages.

Redo of CTRL-E or CTRL-Y in Insert mode interpreted special characters as commands.

Line wrapping for 'tw' was done one character off for insert expansion inserts.

buffer_exists() function didn't work properly for buffer names with a symbolic link in them (e.g. when using buffer_exists(#)).

Removed the "MOTIF_COMMENT" construction from Makefile. It now works with FreeBSD make, and probably with NeXT make too.

Matching the 'define' and 'include' arguments now honor the settings for 'ignorecase'. (Acevedo)

When one file shown in two windows, Visual selection mixed up cursor position in current window and other window.

When doing ":e file" from a help file, the 'isk' option wasn't reset properly, because of a modeline in the help file.

When doing ":e!", a cursor in another window on the same buffer could become invalid, leading to "ml_get: invalid lnum" errors.

Matching buffer name for when expanded name has a different path from not expanded name (Brugnara).

Normal mappings didn't work after an operator. For example, with ":map Q gq", "QQ" didn't work.

When ":make" resulted in zero errors, a "No Errors" error message was given (which breaks mappings).

When ":sourcing" a file, line length was limited to 1024 characters. CTRL-V before <EOL> was not handled Vi compatible. (Acevedo)

Unexpected exit for X11 GUI, caused by SAVE_YOURSELF event. (Heimann)

CTRL-X CTRL-I only found one match per line. (Acevedo)
When using an illegal CTRL-X key in Insert mode, the CTRL-X mode message
was stuck.

Finally managed to ignore the "Quit" menu entry of the Window manager! Now
Vim only exists when there are no changed buffers.

Trying to start the GUI when $DISPLAY is not set resulted in a crash.
When $DISPLAY is not set and gvim starts vim, title was restored to "Thanks
for flying Vim".
When $DISPLAY not set, starting "gvim" (dropping back to vim) and then
selecting text with the mouse caused a crash.

"J", with 'joinspaces' set, on a line ending in ". ", caused one space too
many to be added. (Acevedo)

In insert mode, a CTRL-R {regname} which didn't insert anything left the '"'
on the screen.

":z10" didn't work. (Clapp)

"Help "*" didn't work.

Renamed a lot of functions, to avoid clashes with POSIX name space.

When adding characters to a line, making it wrap, the following lines were
sometimes not shifted down (e.g. after a tag jump).

CTRL-E, with 'so' set and cursor on last line, now does not move cursor as
long as the last line is on the screen.

When there are two windows, doing "^W+^W-" in the bottom window could cause
the status line to be doubled (not redrawn correctly).

This command would hang: ":n cat ". Now connect stdin of the external
command to /dev/null, when expanding.

Fixed lalloc(0,) error for ":echo %:e:r". (Acevedo)

The "+command" argument to ":split" didn't work when there was no file name.

When selecting text in the GUI, which is the output of a command-line command
or an external command, the inversion would sometimes remain.

GUI: "-mh 70" argument was broken. Now, when menuheight is specified, it is
not changed anymore.

GUI: When using the scrollbar or mouse while executing an external command,
this caused garbage characters.

Showmatch sometimes jumped to the wrong position. Was caused by a call to
findmatch() when redrawing the display (when syntax highlighting is on).

Search pattern "\(a *\)\{3} did not work correctly, also matched "a a".
Problem with brace_count not being decremented.

Wildcard expansion added too many non-matching file names.

When 'iskeyword' contains characters like '~', "*" and "#" didn't work
properly. (Acevedo)

On Linux, on a FAT file system, modification time can change by one second.
Avoid a "file has changed" warning for a one second difference.

When using the page-switching in an xterm, Vim would position the cursor on
the last line of the window on exit. Also removed the cursor positioning for
":!" commands.

":g/pat/p" command (partly) overwrote the command. Now the output is on a
separate line.

With 'ic' and 'scs' set, a search for "Keyword", ignore-case matches were
highlighted too.

"^" on a line with only white space, put cursor beyond the end of the line.

When deleting characters before where insertion started ('bs' == 2), could not
use abbreviations.

CTRL-E at end of file puts cursor below the file, in Visual mode, when 'so' is
non-zero. CTRL-E didn't work when 'so' is big and the line below the window
wraps. CTRL-E, when 'so' is non-zero, at end of the file, caused jumping
up-down.

":retab" didn't work well when 'list' is set.

Amiga: When inserting characters at the last line on the screen, causing it
to wrap, messed up the display. It appears that a '\n' on the last line
doesn't always cause a scroll up.

In Insert mode "0<C-D><C-D>" deleted an extra character, because Vim thought
that the "0" was still there. (Acevedo)

"z{count}l" ignored the count. Also for "zh" et. al. (Acevedo)

"S" when 'autoindent' is off didn't delete leading white space.

"/<Tab>" landed on the wrong character when 'incsearch' is set.

Asking a yes/no question could cause a hit-enter prompt.

When the file consists of one long line (>4100 characters), making changes
caused various errors and a crash.

DJGPP version could not save long lines (>64000) for undo.

"yw" on the last char in the file didn't work. Also fixed "6x" at the end of the line. "6X" at the start of a line fails, but does not break a mapping. In general, a movement for an operator doesn't beep or flush a mapping, but when there is nothing to operate on it beeps (this is Vi compatible).

"m'" and "m`" now set the '' mark at the cursor position.

Unix: Resetting of signals for external program didn't work, because SIG_DFL and NULL are the same! For "!!yes|dd count=1|, the yes command kept on running.

Partly fixed: Unix GUI: Typeahead while executing an external command was lost. Now it's not lost while the command is producing output.

Typing <S-Tab> in Insert mode, when it isn't mapped, inserted "<S-Tab>". Now it works like a normal <Tab>, just like <C-Tab> and <M-Tab>.

Redrawing ruler didn't check for old value correctly (caused UMR warnings in Purify).

Negative array index in finish_viminfo_history().

":g/^/d|mo $" deleted all the lines. The ":move" command now removes the :global mark from the moved lines.

Using "vG" while the last line in the window is a "@" line, didn't update correctly. Just the "v" showed "~" lines.

"daw" on the last char of the file, when it's a space, moved the cursor beyond the end of the line.

When 'hlsearch' was set or reset, only the current buffer was redrawn, while this affects all windows.

CTRL-^, positioning the cursor somewhere from 1/2 to 1 1/2 screen down the file, put the cursor at the bottom of the window, instead of halfway.

When scrolling up for ":append" command, not all windows were updated correctly.

When 'hlsearch' is set, and an auto-indent is highlighted, pressing <Esc> didn't remove the highlighting, although the indent was deleted.

When 'ru' set and 'nosc', using "$j" showed a wrong ruler.

Under Xfree 3.2, Shift-Tab didn't work (wrong keysym is used).

Mapping <S-Tab> didn't work. Changed the key translations to use the shortest key code possible. This makes the termcode translations and mappings more consistent. Now all modifiers work in all combinations, not only with <Tab>, but also with <Space>, <CR>, etc.

For Unix, restore three more signals. And Vim catches SIGINT now, so CTRL-C in Ex mode doesn't make Vim exit.

""a5Y" yanked 25 lines instead of 5.

"vrxxx<Esc>" in an empty line could not be undone.

A CTRL-C that breaks ":make" caused the errorfile not to be read (annoying when you want to handle what ":make" produced so far).

":0;/pat" didn't find "pat" in line 1.

Search for "/test/s+1" at first char of file gave bottom-top message, or didn't work at all with 'nowrapscan'.

Bug in viminfo history. Could cause a crash on exit.

":print" didn't put cursor on first non-blank in line.

":0r !cat </dev/null" left cursor in line zero, with very strange effects.

With 'showcmd' set and 'timeoutlen' set to a few seconds, trick to position the cursor leftwards didn't work.

AIX stty settings were restored to cs5 instead of cs8 (Winn).

File name completion didn't work for "zsh" versions that put spaces between file names, instead of NULs.

Changed "XawChain*" to "XtChain*", should work for more systems.

Included quite a few fixes for rightleft mode (Lottem).

Didn't ask to hit-enter when GUI is started and error messages are printed.

When trying to edit a file in a non-existent directory, ended up with editing "No file".

"gqap" to format a paragraph did too much redrawing.

When 'hlsearch' set, only the current window was updated for a new search pattern.

Sometimes error messages on startup didn't cause a hit-enter prompt, because of autocommands containing an empty line.

Was possible to select part of the window in the border, below the command line.

'< and '> marks were not at the correct position after linewise Visual selection.

When translating a help argument to "CTRL-x", prepend or append a '_', when applicable.

Blockwise visual mode wasn't correct when moving vertically over a special

character (displayed as two screen characters).

Renamed "struct option" to "struct vimoption" to avoid name clash with GNU getopt().

":abclear" didn't work (but ":iabclear" and ":cabclear" did work).

When 'nowrap' used, screen wasn't always updated correctly.

"vim -c split file" displayed extra lines.

After starting the GUI, searched the termcap for a "gui" term.

When 'hls' used, search for "^$" caused a hang.
When 'hls' was set, an error in the last regexp caused trouble.

Unix: Only output an extra <EOL> on exit when outputted something in the alternate screen, or when there is a message that needs to be cleared.

"/a\{" did strange things, depending on previous search.

"c}" only redrew one line (with -u NONE).

For mappings, CTRL-META-A was shown as <M-^A> instead of <MC-A>, while :map only accepts <MC-A>. Now <M-C-A> is shown.

Unix: When using full path name in a tags file, which contains a link, and 'hidden' set and jumping to a tag in the current file, would get bogus ATTENTION message. Solved by always expanding file names, even when starting with '/'.

'hlsearch' highlighting of special characters (e.g., a TAB) didn't highlight the whole thing.

"r<CR>" didn't work correctly on the last char of a line.

Sometimes a window resize or other signal caused an endless loop, involving set_winsize().

"vim -r" didn't work, it would just hang (using tgetent() while 'term' is empty).

"gk" while 'nowrap' set moved two lines up.

When windows are split, a message that causes a scroll-up messed up one of the windows, which required a CTRL-L to be typed.

Possible endless loop when using shell command in the GUI.

Menus defined in the .vimrc were removed when GUI started.

Crash when pasting with the mouse in insert mode.

Crash with ":unmenu *" in .gvimrc for Athena.

"5>>" shifted 5 lines 5 times, instead of 1 time.

CTRL-C when getting a prompt in ":global" didn't interrupt.

When 'so' is non-zero, and moving the scrollbar completely to the bottom, there was a lot of flashing.

GUI: Scrollbar ident must be long for DEC Alpha.

Some functions called vim_regcomp() without setting reg_magic, which could lead to unpredictable magicness.

Crash when clicking around the status line, could get a selection with a backwards range.

When deleting more than one line characterwise, the last character wasn't deleted.

GUI: Status line could be overwritten when moving the scrollbar quickly (or when 'wd' is non-zero).

An ESC at the end of a ":normal" command caused a wait for a terminal code to finish. Now, a terminal code is not recognized when its start comes from a mapping or ":normal" command.

Included patches from Robert Webb for GUI. Layout of the windows is now done inside Vim, instead of letting the layout manager do this. Makes Vim work with Lesstif!

UMR warning in set_expand_context().

Memory leak: b_winlnum list was never freed.

Removed TIOCLSET/TIOCLGET code from os_unix.c. Was changing some of the terminal settings, and looked like it wasn't doing anything good. (suggested by Juergen Weigert).

Ruler overwrote "is a directory" message. When starting up, and 'cmdheight' set to > 1, first message could still be in the last line.

Removed prototype for putenv() from proto.h, it's already in osdef2.h.in.

In replace mode, when moving the cursor and then backspacing, wrong characters were inserted.

Win32 GUI was checking for a CTRL-C too often, making it slow.

Removed mappings for MS-DOS that were already covered by commands.

When visually selecting all lines in a file, cursor at last line, then "J". Gave ml_get errors. Was a problem with scrolling down during redrawing.

When doing a linewise operator, and then an operator with a mouse click, it

was also linewise, instead of characterwise.

When 'list' is set, the column of the ruler was wrong.

Spurious error message for "/\(b\+\)*".

When visually selected many lines, message from ":w file" disappeared when redrawing the screen.

":set <M-b>=^[b", then insert "^[b", waited for another character. And then inserted "<M-b>" instead of the real <M-b> character. Was trying to insert K_SPECIAL x NUL.

CTRL-W] didn't use count to set window height.

GUI: "-font" command-line argument didn't override 'guifont' setting from .gvimrc. (Acevedo)

GUI: clipboard wasn't used for "*y". And some more Win32/X11 differences fixed for the clipboard (Webb).

Jumping from one help file to another help file, with 'compatible' set, removed the 'help' flag from the buffer.

File-writable bit could be reset when using ":w!" for a readonly file.

There was a wait for CTRL-O n in Insert mode, because the search pattern was shown.
Reduced wait, to allow reading a message, from 10 to 3 seconds. It seemed nothing was happening.

":recover" found same swap file twice.

GUI: "*yy only worked the second time (when pasting to an xterm)."

DJGPP version (dos32): The system flags were cleared.

Dos32 version: Underscores were sometimes replaced with y-umlaut (Levin).

Version 4.1 of ncurses can't handle tputs("", ..). Avoid calling tputs() with an empty string.

<S-Tab> in the command-line worked like CTRL-P when no completion started yet. Now it does completion, last match first.

Unix: Could get annoying "can't write viminfo" message after doing "su". Now the viminfo file is overwritten, and the user set back to the original one.

":set term=builtin_gui" started the GUI in a wrong way. Now it's not allowed anymore. But "vim -T gui" does start the GUI correctly now.

GUI: Triple click after a line only put last char in selection, when it is a single character word.

When the window is bigger than the screen, the scrolling up of messages was wrong (e.g. ":vers", ":hi"). Also when the bottom part of the window was obscured by another window.

When using a wrong option only an error message is printed, to avoid that the usage information makes it scroll off the screen.

When exiting because of not being able to read from stdin, didn't preserve the swap files properly.

Visual selecting all chars in more than one line, then hit "x" didn't leave an empty line. For one line it did leave an empty line.

Message for which autocommand is executing messed up file write message (for FileWritePost event).

"vim -h" included "-U" even when GUI is not available, and "-l" when lisp is not available.

Crash for ":he <C-A>" (command-line longer than screen).

":s/this/that/gc", type "y" two times, then undo, did reset the modified option, even though the file is still modified.

Empty lines in a tags file caused a ":tag" to be aborted.

When hitting 'q' at the more prompt for ":menu", still scrolled a few lines.

In an xterm that uses the bold trick a single row of characters could remain after an erased bold character. Now erase one extra char after the bold char, like for the GUI.

":pop!" didn't work.

When the reading a buffer was interrupted, ":w" should not be able to overwrite the file, ":w!" is required.

":cf%" caused a crash.

":gui longfilename", when forking is enabled, could leave part of the longfilename at the shell prompt.

5.1 版 本 version-5.1

5.0 和 5.1 版本之间的改进。

主要是一个漏洞修正的发布版，没有很多新特性。

改动 changed-5.1

---

expand() 函数现在用 <NL> 分隔文件名而不是空格。避免内嵌空格文件名出现的问题。

要得到旧的结果，用 substitute(expand(foo), "\n", " ", "g")。

插入扩展用的字典可用反斜杠代表通配符。如果 'isk' 包含反斜杠的话，允许扩展 "ze\kra"。

Win32 GUI 新图标。

":tag"、":tselect" 等只当参数以 '/' 开始时把它当作正规表达式。避免 ":tag xx~" 给出错误信息: "No previous sub. regexp"。:tag 参数包含通配字符时亦然，过去和 Vi 不兼容。
用 '/' 时，按本义接受参数有更高的优先级，在通配匹配之前先使用。
只有使用 '/' 时才寻找不同大小写的匹配，即使没有置位 'ignorecase' 也是如此。
"g^]" 被改为只在匹配的标签多于一个时才做 ":tselect"。

修改部分缺省颜色，因为在深色背景下它们不方便阅读。

搜索模式的字符偏移可以移动光标到下行或上行。同时修正 "/pattern/e+2" 在行尾的 "pattern" 停滞的问题。

状态行上双击不再启动可视模式。拖动状态行不再停止可视模式。

Perl 接口: Buffers() 和 Windows() 现在使用更合乎逻辑的参数，就像 Vim 其余部分使用的那样 (Moore)。

'" 位置标记的初始值设为首行的第一个字符。自动命令使用 '" 不会出错。

新增 added-5.1

"shell_error" 内部变量: 最近外壳命令的输出。

":echohl" 命令: 设置 ":echo" 的高亮。

'highlight' 的 'S' 标志位和 StatusLineNC 高亮组: 非当前窗口的状态行高亮。当前窗口的缺省设置使用粗体。

加入 buffer_name() 和 buffer_number() 函数 (Aaron)。
加入 substitute() 函数的 "g" 标志位参数 (Aaron)。
加入 winheight() 函数。

Win32: 如果外部命令用 "start " 开始，不为它打开控制台 (Aaron)。

Win32 控制台: 使用 termcap 代码，根据当前控制台属性处理粗体/反转属性。

configure 检查 "strip"。(Napier)

插入模式的 CTRL-R CTRL-R x: 按本义插入寄存器内容，而不是像键盘输入那样插入。

使一些 "No match" 错误消息提供更多信息: 不匹配的模式。

"make install" 现在也复制宏文件。

tools/tcltags，从 TCL 文件生成标签文件的外壳脚本。

configure 的 "--with-tlib" 设置。使用 termlib 的简单办法: "./configure --with-tlib=termlib"。

'cino' 的 'u' 标志位可以设置被包含的 () 部分的缩进。

如果 Win32 OLE 版本不能载入注册的类型库，询问用户是否现在要注册 Vim。(Erhardt)
带 OLE 的 Win32: 自动注册时退出 Vim。
加入 VisVim 1.1b，若干改进和新图标 (Heiko Erhardt)。

加入 Vince Negri 关于 Win32s 支持的补丁。需要用 VC 4.1 编译!

Perl 接口: 加入 $curbuf。使 Buffers() 和 Windows() 更合理。
(Moore) 加入 Msg() 的 "group" 参数。

DOS 源代码归档加入 Perl 文件。修改 Makefile.bor 和 Makefile.w32 以支持编译带 Perl 的 Win32 版本。

加入 Ken Scott 提供的新 Makefile.w32。现在可以 make 所有的 Win32 版本，包括 OLE、Perl 和 Python。

加入 CTRL-W g] 和 CTRL-W g ^]: 分割窗口并执行 g] 或 g^]。

加入 "g]"，总是为光标所在的标识符执行 ":tselect"。
加入 ":tjump" 和 ":stjump" 命令。
改进 ":tselect" 的列表标签名过长的问题。

加入 Macintosh 版本和 Python 接口的一些补丁。(St-Amant)

如果以前用过某缓冲区，":buf foo" 现在也恢复光标列。

为不同的最终目标调整了语法文件和脚本的 Makefile (用于 Debian Linux)。

Amiga: $VIM 可以在任何地方使用。如果没定义 $VIM，使用 "VIM:"。修正帮助和语法文件需要 "VIM:" 赋值的问题。现在两者都可以用了。

一些 xterms 发送和 vt100 兼容的功能键 F1-F4。因为无法检测这一点，同时识别两种类型的键并把它们翻译成 <F1> - <F4>。

加入 "VimEnter" 自动命令。在所有的启动任务之后执行。

BeOS 版本现在可以在 Intel CPU 上运行。(Seibert)。

修正 fixed-5.1

":ts" changed position in the tag stack when cancelled with <CR>.
":ts" changed the cursor position for CTRL-T when cancelled with <CR>.
":tn" would always jump to the second match. Was using the wrong entry in the tag stack.
Doing "tag foo", then ":tselect", overwrote the original cursor position in the tag stack.

"make install" changed the vim.1 manpage in a wrong way, causing "doc/doc"
to appear for the documentation files.

When compiled with MAX_FEAT, xterm mouse handling failed. Was caused by DEC
mouse handling interfering.

Was leaking memory when using selection in X11.

CTRL-D halfway a command-line left some characters behind the first line(s)
of the listing.

When expanding directories for ":set path=", put two extra backslashes
before a space in a directory name.

When 'lisp' set, first line of a function would be indented. Now its indent
is set to zero. And use the indent of the first previous line that is at
the same () level. Added test33.

"so<Esc>u" in an empty file didn't work.

DOS: "seek error in swap file write" errors, when using DOS 6.2 share.exe,
because the swap file was made hidden. It's no longer hidden.

":global" command would sometimes not execute on a matching line. Happened
when a data block is full in ml_replace().

For AIX use a tgetent buffer of 2048 bytes, instead of 1024.

Win32 gvim now only sets the console size for external commands to 25x80
on Windows 95, not on NT.

Win32 console: Dead key could cause a crash, because of a missing "WINAPI"
(Deshpande).

The right mouse button started Visual mode, even when 'mouse' is empty, and
in the command-line, a left click moved the cursor when 'mouse' is empty.
In Visual mode, 'n' in 'mouse' would be used instead of 'v'.

A blinking cursor or focus change cleared a non-Visual selection.

CTRL-Home and CTRL-End didn't work for MS-DOS versions.

Could include NUL in 'iskeyword', causing a crash when doing insert mode
completion.

Use _dos_commit() to flush the swap file to disk for MSDOS 16 bit version.

In mappings, CTRL-H was replaced by the backspace key code. This caused
problems when it was used as text, e.g. ":map _U :%s/.^H//g<CR>".

":set t_Co=0" was not handled like a normal term. Now it's translated into
":set t_Co=", which works.

For ":syntax keyword" the "transparent" option did work, although not

mentioned in the help. But synID() returned wrong name.

"gqG" in a file with one-word-per-line (e.g. a dictionary) was very slow and not interruptible.

"gq" operator inserted screen lines in the wrong situation. Now screen lines are inserted or deleted when this speeds up displaying.

cindent was wrong when an "if" contained "((".

'r' flag in 'viminfo' was not used for '%'. Could get files in the buffer list from removable media.

Win32 GUI with OLE: if_ole_vc.mak could not be converted into a project. Hand-edited to fix this...

With 'nosol' set, doing "$kdw" below an empty line positioned the cursor at the end of the line.

Dos32 version changed "\dir\file" into "/dir/file", to work around a DJGPP bug. That bug appears to have been fixed, therefore this translation has been removed.

"/^*" didn't work (find '*' in first column).

"<afile>" was not always set for autocommands. E.g., for ":au BufEnter * let &tags = expand("<afile>:p:h") . "/tags".

In an xterm, the window may be a child of the outer xterm window. Use the parent window when getting the title and icon names. (Smith)

When starting with "gvim -bg black -fg white", the value of 'background' is only set after reading the .gvimrc file. This causes a ":syntax on" to use the wrong colors. Now allow using ":gui" to open the GUI window and set the colors. Previously ":gui" in a gvimrc crashed Vim.

tempname() returned the same name all the time, unless the file was actually created. Now there are at least 26 different names.

File name used for <afile> was sometimes full path, sometimes file name relative to current directory.

When 'background' was set after the GUI window was opened, it could change colors that were set by the user in the .gvimrc file. Now it only changes colors that have not been set by the user.

Ignore special characters after a CSI in the GUI version. These could be interpreted as special characters in a wrong way. (St-Amant)

Memory leak in farsi code, when using search or ":s" command.
Farsi string reversing for a mapping was only done for new mappings. Now it also works for replacing a mapping.

Crash in Win32 when using a file name longer than _MAX_PATH. (Aaron)

When BufDelete autocommands were executed, some things for the buffer were already deleted (esp. Perl stuff).

Perl interface: Buffer specific items were deleted too soon; fixes "screen no longer exists" messages. (Moore)

The Perl functions didn't set the 'modified' flag.

link.sh did not return an error on exit, which may cause Vim to start installing, even though there is no executable to install. (Riehm)

Vi incompatibility: In Vi "." redoes the "y" command. Added the 'y' flag to 'cpoptions'. Only for 'compatible' mode.

":echohl" defined a new group, when the argument was not an existing group.

"syn on" and ":syn off" could move the cursor, if there is a hidden buffer that is shorter that the current cursor position.

The " mark was not set when doing ":b file".

When a "nextgroup" is used with "skipwhite" in syntax highlighting, space at the end of the line made the nextgroup also be found in the next line.

":he g<CTRL-D>", then ":" and backspace to the start didn't redraw.

X11 GUI: "gvim -rv" reversed the colors twice on Sun. Now Vim checks if the result is really reverse video (background darker than foreground).

"cat link.sh | vim -" didn't set syntax highlighting.

Win32: Expanding "file.sw?" matched ".file.swp". This is an error of FindnextFile() that we need to work around. (Kilgore)

"gqgq" gave an "Invalid lnum" error on the last line.
Formatting with "gq" didn't format the first line after a change of comment leader.

There was no check for out-of-memory in win_alloc().

"vim -h" didn't mention "-register" and "-unregister" for the OLE version.

Could not increase 'cmdheight' when the last window is only one line. Now other windows are also made smaller, when necessary.

Added a few {} to avoid "suggest braces around" warnings from gcc 2.8.x.
Changed return type of main() from void to int. (Nam)

Using '~' twice in a substitute pattern caused a crash.

"syn on" and ":syn off" could scroll the window, if there is a hidden buffer that is shorter that the current cursor position.

":if 0 | if 1 | endif | endif" didn't work. Same for ":while" and "elseif".

With two windows on modified files, with 'autowrite' set, cursor in second window, ":qa" gave a warning for the file in the first window, but then auto-wrote the file in the second window. (Webb)

Win32 GUI scrollbar could only handle 32767 lines. Also makes the intellimouse wheel use the configurable number of scrolls. (Robinson)

When using 'patchmode', and the backup file is on another partition, the file copying messed up the write-file message.

GUI X11: Alt-Backspace and Alt-Delete didn't work.

"`0" could put the cursor after the last character in the line, causing trouble for other commands, like "i".

When completing tags in insert mode with ^X^], some matches were skipped, because the compare with other tags was wrong. E.g., when "mnuFileSave" was already there, "mnuFile" would be skipped. (Negri)

When scrolling up/down, a syntax item with "keepend" didn't work properly. Now the flags are also stored for the syntax state at the start of each line.

When 'ic' was changed while 'hlsearch' is on, there was no redraw to show the effect.

Win32 GUI: Don't display "No write since last chance" in a message box, but in the Vim window.

5.2 版 本 version-5.2

5.1 和 5.2 版本之间的改进。

长行可编辑 long-lines

窗口里放不下的长行不再显示 @@@ 了。重画从行内的某个字符开始继续，使得光标周围的文本可见。这使得打开回绕时，可以编辑长行。

加入文件浏览器 file-browser-5.2

Win32、Ahena 和 Motif GUI 上，如果用户请求 ":browse" 并用于 ":e"、":w"、":r"、":so"、":redirect" 和 ":mkexrc/vimrc/vsess" 命令时，弹出文件请求器。
":browse e /foo/bar" 在 /foo/bar 目录打开请求器，这样，你可以得到很好的右手边的映射，比如 ":browse so $vim/macros"。如购没有为 ":browse e" 指定初始目录，编译时可以选择从当前目录还是当前缓冲区的目录开始。(Negri 和 Kahn)
增加了 'browsedir' 选项，可取值 "current"、"last" 或 "buffer"。它说明浏览对话框从最后使用的目录、当前缓冲区的目录，还是当前目录开始。":browse w" 不受影响。
修改了缺省菜单，现在使用 ":browse" 命令。

加入对话框 dialogs-added

增加了 ":confirm" 命令。可用于 ":e"、":q"、":w"、":cl"。Win32、Athena 和 Motif GUI 使用窗口对话框。所有其它的平台使用命令行上的提示。":confirm qa" 提供了保存所有修改过的文件的选择。

confirm() 函数：允许用户访问 confirm 机制。

增加了 'guioptions' 的 'v' 标志位。如果包含，总是在 Win32 GUI 对话框里使用垂直的按钮布局。否则，偏向使用水平布局。

Win32 GUI: ":promptfind" 和 ":promptrepl" 弹出对话框进行搜索/替换。
可用于菜单项目。(Negri)

加入弹出式菜单 popup-menu-added

如果 'mousemodel' 选项设为 "popup"，鼠标右键显示 "PopUp" 顶层菜单，作为弹出的关联菜单。"PopUp" 菜单不再普通的菜单栏上提示。目前，只支持 Win32 和 Athena GUI。

加入选择模式 new-Select-mode

加入了新模式："选择模式"。类似于可视模式，但输入可显示的字符将替换选择区。
- CTRL-G 可以用来切换可视模式和选择模式。
- CTRL-O 可以用来为单个命令从选择模式切换到选择模式。
- 加入 'selectmode' 选项：说明何时启动选择模式，而不是可视模式。
- 加入 'mousemodel' 选项：改变鼠标键的应用。
- 加入 'keymodel' 选项：指示使用 Shift + 特殊键可以启动可视或选择模式的选择。
- 加入 ":behave"。可以用来快速设置 'selectmode'、'mousemodel' 和 'keymodel' 的 MS-Windows 和 xterm 行为。
- xterm 风格的选择现在成为无模式的选择。
- 选择模式使用可视模式的映射和菜单。它们自动先切换到可视模式。之后，重新选择区域，除非该区域已被删除。"gV" 命令用来在映射里跳过重新选择这一步。
- 增加 "gh"、"gH" 和 "g^H" 命令：开始选择（高亮）模式。
- 选择模式的退格键删除选择区。

"mswin.vim" 脚本。设置非常类似于 MS-Windows 的行为。

加入会话文件 new-session-files

":mks[ession]" 类似于 "mkvimrc"，但写入当前载入缓冲区的完整文件名和当前目录，使得执行该文件可以重新载入这些文件并切换到该目录。也保存和恢复窗口。使文件名相对于会话文件。
'sessionoptions' 选项设置 ":mksession" 的行为。(Negri)

用户定义的函数和命令 new-user-defined

加入用户定义函数。用 ":function" 开始定义，到 ":endfunction" 位置。
调用方式为 "Func()"。允许使用可变数量的参数。
包含局部变量 "l:name" 的支持。用 ":return" 返回值。
见 :function 。
用 ":call" 调用函数。如果提供了范围，该范围的每行执行一次该函数。 :call
"macros/justify.vim" 是使用用户定义函数的例子。
用户函数不改变最近使用的搜索模式，或 "." 重复的命令。
'maxfuncdepth' 函数。限制函数调用的深度。避免函数使用无限递归的麻烦（因为内存溢出而崩溃）。

用户定义的 Ex 命令: ":command"、":delcommand" 和 ":comclear"。(Moore) 见
user-commands 。

新接口 interfaces-5.2

Tcl 接口。(Wilken) 见 tcl 。
使用 ":tcl"、":tcldo" 和 "tclfile" 命令。

Cscope 支持。(Kahn) (Sekera) 见 cscope 。
使用 ":cscope" 和 ":cstag" 命令。使用选项 'cscopeprg'、'cscopetag'、
'cscopetagorder' 和 'cscopeverbose'。

新移植版本 ports-5.2

Amiga GUI 移植版本。(Nielsen) 还没有充分测试！

RISC OS 版本。(Thomas Leonard) 见 riscos 。
该版本可以在 GUI 或文本模式下运行，取决于在哪里调用。
删除了 "os_archie" 文件，反正它们也不再工作了。

多字节支持 new-multi-byte

Win32 GUI 的多字节支持。(Baek)
'fileencoding' 选项决定文件里的文本是如何编码的。
多字节字符可以用 ":ascii"。多字节字符可以在 Windows 95 工作，即使使用北美版本也是如此。(Aaron)
需要在 feature.h 里打开。
还没有充分测试！

新函数 new-functions-5.2

---

browse()	如果可用，给出文件请求器。(Negri)
escape()	字符串里用反斜杠给字符转义。
fnamemodify()	修改文件名。

input()	请求用户输入一行。(Aaron) input() 函数输入的行有单独的历史。
argc()	
argv()	可以用来访问参数列表。
winbufnr()	窗口的缓冲区号。(Aaron)
winnr()	窗口号。(Aaron)
matchstr()	返回匹配的字符串。
setline()	设置一行为某字符串值。

新选项 new-options-5.2

'allowrevins'	允许在插入和命令行模式里使用 CTRL-_ 命令。
'browsedir'	指示浏览对话框从在哪个目录开始。
'confirm'	如果置位，:q :w 和 :e 命令总是和使用了 ":confirm" 一样。(Negri)
'cscopeprg'	
'cscopetag'	
'cscopetagorder'	
'cscopeverbose'	设置 cscope 行为。
'filetype'	RISC-OS 专用的文件类型。
'grepformat'	
'grepprg'	为 :grep 命令使用。
'keymodel'	指示使用 Shift + 特殊键可以启动可视或选择模式的选择。
'listchars'	设置 'list' 模式里行尾、制表和拖尾空格所显示的字符。(部分来自 Smith) 也设置如果置位了 'nowrap' 时，如果一行放不下时要显示的字符。
'matchpairs'	允许 '<' 和 '>' 的匹配，还有其它的单字符组对。
'mousefocus'	窗口焦点追随鼠标 (部分来自 Terhaar)。用键盘命令改变焦点移动指针到该窗口。同时在改变窗口布局时 (分割窗口、改变窗口高度等) 移动指针。
'mousemodel'	改变鼠标按钮的使用方式。
'selection'	如果设为 "inclusive" 或 "exclusive"，光标可以在可视或选择模式里到行尾之后一个字符的位置。如果设为 "old"，则使用旧行为。如果是 "inclusive"，光标所在的字符包含在操作里。如果使用 "exclusive"，使用 'guicursor' 新的 "ve" 入口。缺省为垂直线。
'selectmode'	说明何时开始选择模式，而不是可视模式。
'sessionoptions'	设置 ":mksession" 的行为。(Negri)
'showfulltag'	在插入模式补全标签时，显示标签搜索模式 (整齐格式) 作为可能的一个选择 (如果有的话)。
'swapfile'	缓冲区是否使用交换文件。
'syntax'	如果设置，载入该名字的语法。允许从模式行里设置特定的语法。
'ttymouse'	允许名字不以 "xterm" 开始的终端使用 xterm 的鼠标代码。
'wildignore'	不应该参与补全的文件名模式的列表。
'wildmode'	可以用于设置 'wildchar' 的扩展类型。 替换用于命令行补全的 CTRL-T 命令。 列出所有匹配时，不响铃。
'winaltkeys'	Win32 和 Motif GUI。如果为 "yes"，ALT 键完全由窗口系统处理。如果为 "no"，窗口系统完全不使用 ALT 键。如果为 "menu"，这取决于该键是否是菜单快捷键。
'winminheight'	每个窗口的最小高度。缺省为 1。如果你想要零行的窗口，设为 0。如果窗口高度为零，不显示滚动条。(Negri)

新 Ex 命令 new-ex-commands-5.2

:badd	把文件名加到缓冲区列表，没有副作用。(Negri)
:behave	快速设置 MS-Windows 或 xterm 行为。
:browse	使用文件选择对话框。
:call	调用函数，可选使用行范围。
:cnewer	
:colder	访问 quickfix 错误列表的堆栈。
:comclear	清除所有的用户定义命令。
:command	定义用户命令。
:continue	回到 ":while"。
:confirm	如果有意想不到的事发生，请求确认。
:cscope	执行 cscope 命令。
:cstag	用 cscope 来跳转到标签。
:delcommand	删除用户定义命令。
:delfunction	删除用户定义函数。
:endfunction	结束用户定义函数。
:function	定义用户定义函数。
:grep	类似于 ":make"。(Negri)
:mksession	建立会话文件。
:nohlsearch	暂时停止 'hlsearch' 高亮。
:Print	和 Vi 兼容。和 ":print" 相同。
:promptfind	搜索对话框 (Win32 GUI)。
:promptrepl	搜索/替代对话框 (Win32 GUI)。
:return	从用户定义函数里返回。
:simalt	Win32 GUI: 模拟 alt 键的键击。(Negri)
:smagic	类似于 ":substitute"，但总是使用 'magic'。
:snomagic	类似于 ":substitute"，但总是使用 'nomagic'。
:tcl	执行 TCL 命令。
:tcldo	在行范围上执行 TCL 命令。
:tclfile	执行 TCL 脚本文件。
:tearoff	可撕下的菜单 (Win32 GUI)。
:tmenu	
:tunmenu	Win32 GUI: 菜单工具提示。(Negri)
:star :*	执行寄存器。

改动 changed-5.2

换名的函数:

```
buffer_exists()     -> bufexists()
buffer_name()       -> bufname()
buffer_number()     -> bufnr()
file_readable()     -> filereadable()
highlight_exists()  -> hlexists()
highlightID()       -> hlID()
last_buffer_nr()    -> bufnr("$")
```

为了后向兼容，仍然可以使用旧名。

只有新的 'allowrevins' 选项置位时才可以用插入模式和命令行模式的 CTRL-_ 命令。
这是为了避免按 SHIFT-_ 的人不小心进入反向插入模式而不知所措。

如果 ":tselect" 列表的文件名过长，删除中间部分并放上 "..."。

Win32 GUI: 使字体选择器出现在 Vim 窗口中间，不是随便的某个位置。(Negri)

":bn" 跳过帮助缓冲区，除非现在就在帮助缓冲区。(Negri)

如果有状态行并且只有一个窗口，不显示当前窗口的状态行中的 '^'。

过去 ":*" 相当于 "'<,'>"，即可视区域。但在 Vi 里它用作 ":@"。如果 'cpoptions' 包含 '*'，使用 Vi 兼容行为。

如果置位 'insertmode'，用 CTRL-O 执行映射如同没有置位 'insertmode' 。这样，即使置位 'insertmode'，也可以使用 "正常的" 映射。

如果已经设置 'mouse' (比如通过 .vimrc 文件)，GUI 启动时不自动设置 'mouse'。

从 'mouse' 选项删除 'N'、'I' 和 'A' 标志位。

"切换选项" 更名为 "布尔选项"。有人会认为 ":set xyz" 每次都会切换 'xyz' 开/关。

内部变量 "shell_error" 包含外壳的错误代码，而不只是 0 或 1。

插入或替换时，输入 CTRL-V CTRL-<CR> 过去插入 "<C-CR>"。这不是很有用。现在忽略 CTRL 键而插入 <CR>，和其他带修饰符的 "普通" 键相同。这些带修饰符的键组合仍然可以映射。
插入模式下，可以用 CTRL-K 加 <C-CR> 或 <S-Space> 插入这些特殊字符。

"quotes" 文件移到 doc/quotes.txt，而 "todo" 文件移到 doc/todo.txt。它们现在和其它文档文件一样安装。

winheight() 函数对不存在的窗口返回 -1。过去返回 0，但现在这是一个合法的高度。

'selection' 的缺省值现在是 "inclusive"。使用 "$" 或在可视模式下用鼠标移动光标时会看到区别。

存在修改过的隐藏缓冲区时 ":q!" 不退出。":qa!" 则总能退出。

缺省关闭 Perl/Python/Tcl 接口。用的人不是很多，但它们的存在使得可执行文件大很多。现在对于多数任务而言，内部脚本语言已经足够强大。

'titlestring' 和 'iconstring' 选项的字符串不经翻译而直接用于窗口标题和图标。这样就可以在里面包含 <CR>。以前 <CR> 会显示为 "^M" (两个字符)。

从插入模式启动可视或选择模式下的映射时 (模式显示 "(insert) Visual")，直到映射完成为止才返回插入模式。这样，可视模式的映射也能用于从选择模式开始的可视模式。

$VIMRUNTIME/menu.vim 的菜单不再否决已有的菜单。这样可以在 .vimrc 文件或者执行 mswin.vim 时定义菜单。

Unix: 如果存在，使用 /var/tmp 来保存 .swp 文件。那里的文件在系统重启时仍然存在 (至少 Linux 如此)。

configure 参数 --with-motif-lib，使用静态 Motif 库。

支持数字键盘 +、-、*、/ 键的映射。(Negri)
如果不映射，和普通字符相同。

Win32 GUI: 如果放置目录到 gVim 上，cd 到那里并编辑新缓冲区。(Negri)

Win32 GUI: CTRL-Break 用于中断，这样 CTRL-C 就可用作映射。

":map" 的输出高亮 "*"，明白说明它不是 rhs 的一部分。(Roemer)

显示可视区域时，现在不关闭光标，方便定位。 GUI 现在使用灰色背景高亮可视区域，
这样即使光标反转也依然保持可见。

Win32: 启动时如果使用单个完整路径名（例如双击文件），cd 到文件的目录。(Negri)

Win32 GUI: 可撕下菜单，用 ":tearoff <menu-name>" 命令实现。(Negri)
'guioptions' 的 't' 选项：为 Win32 GUI 和 Motif 加入可撕下菜单项目。缺省包含。
Win32 GUI: 用 ">>" 指示带子菜单的可撕下菜单。(Negri)

加入 ^Kaa 和 ^KAA 二合字母。
"euro" 符号加到 digraph.c。(Corry)

支持 Motif 菜单快捷键。'&' 的使用方式类似于 MS-Windows (Ollis)。
其它的 GUI 忽略菜单名里面的 '&'。

DJGPP: 屏幕更新更快 (John Lange)。

对语法组进行簇合 ":syntax cluster" (Bigham)。
语法文件的包含：":syntax include" (Bigham)。

如果 'nosol' 置位的话，切换缓冲区时保持列不变 Radics)。

Perl 接口的 Number 函数。

Athena GUI 的 Intellimouse（智能鼠标）支持。(Jensen)

":sleep" 也接受 "m" 表示的毫秒参数。

'guioptions' 加入 'p' 标志位：安装进入/退出窗口事件的回调。Terhaar 的光标闪烁
现在能用了，但我的不行了。

"--help" 和 "--version" 命令行参数。

":list" 输出的非文本使用 NonText 高亮。

加入文本对象："i(" 和 "i)" 同义于 "ib"，"i{" 和 "i}" 同义于 "iB"。新出现："i<"
和 "i>"，选择 <thing> (<> 块)。另外，这些都支持 "a" 对象。

'shortmess' 的 'O' 标志位：读入文件的消息覆盖以前的消息。(Negri)

Win32 GUI: 'guioptions' 的 'T' 标志位：切换工具栏的开/关。包含自己制作的工具栏位图的列表。(Negri)

加入子菜单的菜单优先级。Win32 和 Motif GUI 有实现。
":menu" 命令显示菜单优先级。
Default 和 Syntax 菜单现在包含项目的优先级。可以在缺省项目之间插入菜单项目。

打开 'number' 选项时，用 LineNr 组高亮行号。

"Ignore" 高亮组：用它高亮的文本被置为空白。用于隐藏帮助文本的特殊字符。

加入 Exuberant Ctags 2.3 版本。它包含 C++ 支持、Java 支持和递归访问目录。(Hiebert)

如果标签文件没有排序并且检测到如此（用一个简单的方法），报错。

":unlet" 接受 "!"，忽略不存在的变量并接受多于一个参数。(Roemer)
补全 ":unlet" 的变量名。(Roemer)

如果被其它函数调用的函数里有错误，错误信息里显示调用栈。

新文件名修饰符：
":.": 简约相对于当前目录的文件名。
":~": 简约相对于主目录的文件名。
":s?pat?sub?": "pat" 替代为 "sub" 一次。
":gs?pat?sub?": 所有的 "pat" 替代为 "sub"。

新 configure 参数：--enable-min-features 和 --enable-max-features。切换到最小和最大特性包的简单方法。

新的编译时特性：modify_fname。用于文件名修饰符，例如 "%:p:h"。可关闭以减少一些代码（16 位 DOS）。

插入模式下使用整行补全时如果置位 'cindent'，合适地缩进该行。

MSDOS 和 Win32 控制台：'guicursor' 设置光标的厚度。(Negri)

包含新的 Farsi 字体集合。(Shiran)

加速键文本现在也能用于 Motif。所有菜单可以用 & 定义助记，TAB 定义加速键文本。
不支持的系统忽略之。
删除或替换菜单时，菜单名的比较到助记符之前的 <Tab> 为止。

":substitute" 之后的 'i' 和 'I' 标志位：忽略大小写与否。

如果丢失运行时文件，"make install" 会抱怨。

Unix：找到已有但不能打开的交换文件时，在 ATTENTION 消息里提及文件的拥有者。

'complete' 的 'i'、't' 和 'k' 选项现在显示正在寻找的匹配所在的位置。(Acevedo)

"gJ" 命令：连接行且不插入空格。

设置 'keywordprg' 为 "man -s" 会进行特殊处理。如果没有计数，删除 "-s"，否则加

入计数。configure 会检查 "man -s 2 read" 是否能用，并据此设置 'keywordprg' 的缺省值。

如果执行 ":bd" 时只打开一个窗口，VIm 试图移动到相同类型的缓冲区（也就是说，非帮助到非帮助，帮助到帮助），这样和 :bnext/:bprev 的行为一致。(Negri)

允许映射的右手边使用 "<Nop>"。":map xx <Nop>"，定义 "xx" 为空。

":menu" 命令可用 "<Tab>" 代替菜单路径里真正的制表。这使输入更容易，因为不需要反斜杠。

POSIX 兼容的正规表达式字符类：[:alnum:]、[:alpha:]、[:blank:]、[:cntrl:]、[:digit:]、[:graph:]、[:lower:]、[:print:]、[:punct:]，[:space:]、[:upper:] 和 [:xdigit:]。(Briscoe)

正规表达式字符类（为了快速语法高亮匹配）：

数字：	\d [0-9]	\D	非数字 (Roemer)
十六进制：	\x [0-9a-fA-F]	\X	非十六进制
八进制：	\o [0-7]	\O	非八进制
单词：	\w [a-zA-Z0-9_]	\W	非单词
单词首字符：	\h [a-zA-Z_]	\H	非单词首字符
字母：	\a [a-zA-Z]	\A	非字母
小写：	\l [a-z]	\L	非小写
大写：	\u [A-Z]	\U	非大写

":set" 现在接受 "+="、"^=" 和 "-=": 从字符串选项加入或删除部分内容，从数值选项加入或减去数值。逗号分隔的列表选项自动（神奇）地插入或删除逗号。

文件类型特性，用于自动命令。使用文件类型代替模式来匹配文件。目前只用于 RISC OS。(Leonard)

自动命令的模式里可用环境变量。它们在定义自动命令时扩展。

"BufFilePre" 和 "BufFilePost" 自动命令事件：":file" 命令改变缓冲区名的前与后。"VimLeavePre" 自动命令事件：写入 .viminfo 文件前。

自动命令参数：<abuf> 代表缓冲区号，和 <afile> 类似。

保持更多的同步上下文，往回滚动时语法高亮的速度稍快。

Win32 GUI：删除或插入屏幕行时避免重画以加快滚动。

GUI：拇指移动小于一个像素时不重画滚动条以加快滚动。

bugreport.vim 里加入 ":highlight"。

建立 install.exe 程序，简化 DOS 和 MS-Windows 的安装。

新寄存器：'_'，黑洞。写入时什么都不做。读取则总是为空。可用于避免删除或修改命令对寄存器的改变，或用于减少大改变的内存开销。

CTRL-V xff 用十六进制值输入字符。CTRL-V o123 用八进制输入字符。(Aaron)

改进语法高亮的性能，如果没有 "keepend"，跳过相应的检查。

模式消息（"-- INSERT --"）移到屏幕的末行。如果 'cmdheight' 大于一，消息仍然可以阅读。

列出匹配文件时，用 'suffixes' 进行排序，这和 CTRL-N 提取它们时列出的顺序相同。

synIDattr() 接受（可选的）第三个参数，属性所属的终端类型。可用来在 gvim 之外运行 2html.vim（要用颜色名代替 #RRGGBB）。

对内存进行刨视（profile），用于调试。退出或 "g^A" 命令会显示结果。(Kahn)

DOS: 使用当前驱动器的文件时删除驱动器名: "A:\dir\file" -> "\dir\file"。有助于移动软盘上的会话文件: "A:\dir" 到 "B:\dir"。

能记住的跳转的数目从每窗口 30 个增加到 50 个。

暂时关闭 'hls' 高亮直到下次搜索: ":nohlsearch"。

"gp" 和 "gP" 命令: 类似于 "p" 和 "P"，但把光标刚好放在插入文本之后。MS-Windows 模式用于 CTRL-V 命令。

修正 fixed-5.2

Win32 GUI: Could draw text twice in one place, for fake-bold text. Removed this, Windows will handle the bold text anyway. (Negri)

patch 5.1.1: Win32s GUI: pasting caused a crash (Negri)

patch 5.1.2: When entering another window, where characters before the cursor have been deleted, could have a cursor beyond the end of the line.

patch 5.1.3: Win32s GUI: Didn't wait for external command to finish. (Negri)

patch 5.1.4: Makefile.w32 can now also be used to generate the OLE version (Scott).

patch 5.1.5: Crashed when using syntax highlighting: cursor on a line that doesn't fit in the window, and splitting that line in two.

patch 5.1.6: Visual highlighting bug: After ":set nowrap", go to end of line (so that the window scrolls horizontally), ":set wrap". Following Visual selection was wrong.

patch 5.1.7: When 'tagbsearch' off, and 'ignorecase' off, still could do binary searching.

patch 5.1.8: Win32 GUI: dragging the scrollbar didn't update the ruler.

patch 5.1.9: Using ":gui" in .vimrc, caused xterm cursor to disappear.

patch 5.1.10: A CTRL-N in Insert mode could cause a crash, when a buffer without a name exists.

patch 5.1.11: "make test" didn't work in the shadow directory. Also adjusted "make shadow" for the links in the ctags directory.

patch 5.1.12: "buf 123foo" used "123" as a count, instead as the start of a buffer name.

patch 5.1.13: When completing file names on the command-line, reallocating the command-line may go wrong.

patch 5.1.14: ":[nvci]unmenu" removed menu for all modes, when full menu patch specified.

Graceful handling of NULLs in drag-dropped file list. Handle passing NULL to Fullname_save(). (Negri)

Win32: ":!start" to invoke a program without opening a console, swapping screens, or waiting for completion in either console or gui version. e.g. you can type ":!start winfile". ALSO fixes "can't delete swapfile after spawning a shell" bug. (enhancement of Aaron patch) (Negri)

Win32 GUI: Fix CTRL-X default keymapping to be more Windows-like. (Negri)

Shorten filenames on startup. If in /foo/bar, entering "vim ../bar/bang.c" displays "bang.c" in status bar, not "/foo/bar/bang.c" (Negri)

Win32 GUI: No copy to Windows clipboard when it's not desired.

Win32s: Fix pasting from clipboard - made an assumption not valid under Win32s. (Negri)

Win32 GUI: Speed up calls to gui_mch_draw_string() and cursor drawing functions. (Negri)

Win32 GUI: Middle mouse button emulation now works in GUI! (Negri)

Could skip messages when combining commands in one line, e.g.:
":echo "hello" | write".

Perl interpreter was disabled before executing VimLeave autocommands. Could not use ":perl" in them. (Aaron)

Included patch for the Intellimouse (Aaron/Robinson).

Could not set 'ls' to one, when last window has only one line. (Mitterand)

Fixed a memory leak when removing menus.

After ":only" the ruler could overwrite a message.

Dos32: removed changing of __system_flags. It appears to work better when it's left at the default value.

p_aleph was an int instead of along, caused trouble on systems where

sizeof(int) != sizeof(long). (Schmidt)

Fixed enum problems for Ultrix. (Seibert)

Small redraw problem: "dd" on last line in file cleared wrong line.

Didn't interpret "cmd | endif" when "cmd" starts with a range. E.g. "if 0 |
.d | endif".

Command "+|" on the last line of the file caused ml_get errors.

Memory underrun in eval_vars(). (Aaron)

Don't rename files in a difficult way, except on Windows 95 (was also done on
Windows NT).

Win32 GUI: An external command that produces an error code put the error
message in a dialog box. had to close the window and close the dialog. Now
the error code is displayed in the console. (Negri)

"comctl32.lib" was missing from the GUI libraries in Makefile.w32. (Battle)

In Insert mode, when entering a window in Insert mode, allow the cursor to be
one char beyond the text.

Renamed machine dependent rename() to mch_rename(). Define mch_rename() to
rename() when it works properly.

Rename vim_chdir() to mch_chdir(), because it's machine dependent.

When using an arglist, and editing file 5 of 4, ":q" could cause "-1 more
files to edit" error.

In if_python.c, VimCommand() caused an assertion when a do_cmdline() failed.
Moved the Python_Release_Vim() to before the VimErrorCheck(). (Harkins)

Give an error message for an unknown argument after "--". E.g. for "vim
--xyz".

The FileChangedShell autocommand didn't set <afile> to the name of the changed
file.

When doing ":e file", causing the attention message, there sometimes was no
hit-enter prompt. Caused by empty line or "endif" at end of sourced file.

A large number of patches for the VMS version. (Hunsaker)

When CTRL-L completion (find longest match) results in a shorter string, no
completion is done (happens with ":help").

Crash in Win32 GUI version, when using an Ex "@" command, because
LinePointers[] was used while not initialized.

Win32 GUI: allow mapping of Alt-Space.

Output from "vim -h" was sent to stderr. Sending it to stdout is better, so one can use "vim -h | more".

In command-line mode, ":vi[!]" should reload the file, just like ":e[!]".
In Ex mode, ":vi" stops Ex mode, but doesn't reload the file. This is Vi compatible.

When using a ":set ls=1" in the .gvimrc file, would get a status line for a single window. (Robinson)

Didn't give an error message for ":set ai,xx". (Roemer)
Didn't give an error message for ":set ai?xx", ":set ai&xx", ":set ai!xx".

Non-Unix systems: That a file exists but is unreadable is recognized as "new file". Now check for existence when file can't be opened (like Unix).

Unix: osdef.sh didn't handle declarations where the function name is at the first column of the line.

DJGPP: Shortening of file names didn't work properly, because get_cwd() returned a path with backslashes. (Negri)

When using a 'comments' part where a space is required after the middle part, always insert a space when starting a new line. Helps for C comments, below a line with "/****".

Replacing path of home directory with "~/" could be wrong for file names with embedded spaces or commas.

A few fixes for the Sniff interface. (Leherbauer)

When asking to hit 'y' or 'n' (e.g. for ":3,1d"), using the mouse caused trouble. Same for ":s/x/y/c" prompt.

With 'nowrap' and 'list', a Tab halfway on the screen was displayed as blanks, instead of the characters specified with 'listchars'. Also for other characters that take more than one screen character.

When setting 'guifont' to an unknown font name, the previous font was lost and a default font would be used. (Steed)

DOS: Filenames in the root directory didn't get shortened properly. (Negri)

DJGPP: making a full path name out of a file name didn't work properly when there is no _fullpath() function. (Negri)

Win32 console: ":sh" caused a crash. (Negri)

Win32 console: Setting 'lines' and/or 'columns' in the _vimrc failed miserably (could hang Windows 95). (Negri)

Win32: The change-drive function was not correct, went to the wrong drive. (Tsindlekht)

GUI: When editing a command line in Ex mode, Tabs were sometimes not
backspaced properly, and unprintable characters were displayed directly.
non-GUI can still be wrong, because a system function is called for this.

":set" didn't stop after an error. For example ":set no ai" gave an error for
"no", but still set "ai". Now ":set" stops after the first error.

When running configure for ctags, $LDFLAGS wasn't passed to it, causing
trouble for IRIX.

"@%" and "@#" when file name not set gave an error message. Now they just
return an empty string. (Steed)

CTRL-X and CTRL-A didn't work correctly with negative hex and octal numbers.
(Steed)

":echo" always started with a blank.

Updating GUI cursor shape didn't always work (e.g., when blinking is off).

In silent Ex mode ("ex -s" or "ex <file") ":s///p" didn't print a line. Also
a few other commands that explicitly print a text line didn't work. Made this
Vi compatible.

Win32 version of _chdrive() didn't return correct value. (Tsindlekht)

When using 't' in 'complete' option, no longer give an error message for a
missing tags file.

Unix: tgoto() can return NULL, which was not handled correctly in configure.

When doing ":help" from a buffer where 'binary' is set, also edited the help
file in binary mode. Caused extra ^Ms for DOS systems.

Cursor position in a file was reset to 1 when closing a window.

":!ls" in Ex mode switched off echo.

When doing a double click in window A, while currently in window B, first
click would reset double click time, had to click three times to select a
word.

When using <F11> in mappings, ":mkexrc" produced an exrc file that can't be
used in Vi compatible mode. Added setting of 'cpo' to avoid this. Also, add
a CTRL-V in front of a '<', to avoid a normal string to be interpreted as a
special key name.

Gave confusing error message for ":set guifont=-*-lucida-*": first "font is
not fixed width", then "Unknown font".

Some options were still completely left out, instead of included as hidden
options.

While running the X11 GUI, ignore SIGHUP signals. Avoids a crash after executing an external command (in rare cases).

In os_unixx.h, signal() was defined to sigset(), while it already was.

Memory leak when executing autocommands (was reported as a memory leak in syntax highlighting).

Didn't print source of error sometimes, because pointers were the same, although names were different.

Avoid a number of UMR errors from Purify (third argument to open()).

A swap file could still be created just after setting 'updatecount' to zero, when there is an empty buffer and doing ":e file". (Kutschera)

Test 35 failed on 64 bit machines. (Schild)

With "p" and "P" commands, redrawing was slow.

Awk script for html documentation didn't work correctly with AIX awk.
Replaced "[,.);\]   ]" with "[] ,.);   ]". (Briscoe)
The makehtml.awk script had a small problem, causing extra lines to be inserted. (Briscoe)

"gqgq" could not be repeated. Repeating for "gugu" and "gUgU" worked in a wrong way. Also made "gqq" work to be consistent with "guu".

C indent was wrong after "case ':':".

":au BufReadPre *.c put": Line from put text was deleted, because the buffer was still assumed to be empty.

Text pasted with the Edit/Paste menu was subject to 'textwidth' and 'autoindent'. That was inconsistent with using the mouse to paste. Now "*p is used.

When using CTRL-W CTRL-] on a word that's not a tag, and then CTRL-] on a tag, window was split.

":ts" got stuck on a tags line that has two extra fields.

In Insert mode, with 'showmode' on, <C-O><C-G> message was directly overwritten by mode message, if preceded with search command warning message.

When putting the result of an expression with "=<expr>p, newlines were inserted like ^@ (NUL in the file). Now the string is split up in lines at the newline.

putenv() was declared with "const char *" in pty.c, but with "char *" in osdef2.h.in. Made the last one also "const char *".

":help {word}", where +{word} is a feature, jumped to the feature list instead of where the command was explained. E.g., ":help browse", ":help autocmd".

Using the "\<xx>" form in an expression only got one byte, even when using a
special character that uses several bytes (e.g., "\<F9>").
Changed "\<BS>" to produce CTRL-H instead of the special key code for the
backspace key. "\<Del>" produces 0x7f.

":mkvimrc" didn't write a command to set 'compatible' or 'nocompatible'.

The shell syntax didn't contain a "syn sync maxlines" setting. In a long file
without recognizable items, syncing took so long it looked like Vim hangs.
Added a maxlines setting, and made syncing interruptible.

The "gs" command didn't flush output before waiting.

Memory leaks for:
    ":if 0 | let a = b . c | endif"
    "let a = b[c]"
    ":so {file}" where {file} contains a ":while"

GUI: allocated fonts were never released. (Leonard)

Makefile.bor:
- Changed $(DEFINES) into a list of "-D" options, so that it can also be used
  for the resource compiler. (not tested!)
- "bcc.cfg" was used for all configurations. When building for another
  configuration, the settings for the previous one would be used. Moved
  "bcc.cfg" to the object directory. (Geddes)
- Included targets for vimrun, install, ctags and xxd. Changed the default to
  use the Borland DLL Runtime Library, makes Vim.exe a log smaller. (Aaron)

"2*" search for the word under the cursor with "2" prepended. (Leonard)

When deleting into a specific register, would still overwrite the non-Win32
GUI selection. Now ""x"*P works.

When deleting into the "" register, would write to the last used register.
Now ""x always writes to the unnamed register.

GUI Athena: A submenu with a '.' in it didn't work. E.g.,
":amenu Syntax.XY\.Z.foo lll".

When first doing ":tag foo" and then ":tnext" and/or ":tselect" the order of
matching tags could change, because the current file is different. Now the
existing matches are kept in the same order, newly found matches are added
after them, not matter what the current file is.

":ta" didn't find the second entry in a tags file, if the second entry was
longer than the first one.

When using ":set si tw=7" inserting "foo {^P}" made the "}" inserted at the
wrong position. can_si was still TRUE when the cursor is not in the indent of
the line.

Running an external command in Win32 version had the problem that Vim exits

when the X on the console is hit (and confirmed). Now use the "vimrun" command to start the external command indirectly. (Negri)

Win32 GUI: When running an external filter, do it in a minimized DOS box. (Negri)

":let" listed variables without translation into printable characters.

Win32 console: When resizing the window, switching back to the old size (when exiting or executing an external command) sometimes failed. (Negri)
This appears to also fix a "non fixable" problem:
Win32 console in NT 4.0: When running Vim in a cmd window with a scrollbar, the scrollbar disappeared and was not restored when Vim exits. This does work under NT 3.51, it appears not to be a Vim problem.

When executing BufDelete and BufUnload autocommands for a buffer without a name, the name of the current buffer was used for <afile>.

When jumping to a tag it reported "tag 1 of >2", while in fact there could be only two matches. Changed to "tag 1 of 2 or more".

":tjump tag" did a linear search in the tags file, which can be slow.

Configure didn't find "LibXm.so.2.0", a Xm library with a version number.

Win32 GUI: When using a shifted key with ALT, the shift modifier would remain set, even when it was already used by changing the used key. E.g., "<M-S-9>" resulted in "<M-S-(>", but it should be "<M-(>". (Negri)

A call to ga_init() was often followed by setting growsize and itemsize. Created ga_init2() for this, which looks better. (Aaron)

Function filereadable() could call fopen() with an empty string, which might be illegal.

X Windows GUI: When executing an external command that outputs text, could write one character beyond the end of a buffer, which caused a crash. (Kohan)

When using "*" or "#" on a string that includes '/' or '?' (when these are included in 'isk'), they were not escaped. (Parmelan)

When adding a ToolBar menu in the Motif GUI, the submenu_id field was not cleared, causing random problems.

When adding a menu, the check if this menu (or submenu) name already exists didn't compare with the simplified version (no mnemonic or accelerator) of the new menu. Could get two menus with the same name, e.g., "File" and "&File".

Breaking a line because of 'textwidth' at the last line in the window caused a redraw of the whole window instead of a scroll. Speeds up normal typing with 'textwidth' a lot for slow terminals.

An invalid line number produced an "invalid range" error, even when it wasn't to be executed (inside "if 0").

When the unnamed, first buffer is re-used, the "BufDelete" autocommand was not called. It would stick in a buffer list menu.

When doing "%" on the NUL after the line, a "{" or "}" in the last character of the line was not found.

The Insert mode menu was not used for the "s" command, the Operator-pending menu was used instead.

With 'compatible' set, some syntax highlighting was not correct, because of using "[\t]" for a search pattern. Now use the regexps for syntax highlighting like the 'cpoptions' option is empty (as was documented already).

When using "map <M-Space> ms" or "map <Space> sss" the output of ":map" didn't show any lhs for the mapping (if 'isprint' includes 160). Now always use <Space> and <M-Space>, even when they are printable.

Adjusted the Syntax menu, so that the lowest entry fits on a small screen (for Athena, where menus don't wrap).

When using CTRL-E or CTRL-Y in Insert mode for characters like 'o', 'x' and digits, repeating the insert didn't work.

The file "tools/ccfilter.README.txt" could not be unpacked when using short file names, because of the two dots. Renamed it to "tools/ccfilter_README.txt".

For a dark 'background', using Blue for Directory and SpecialKey highlight groups is not very readable. Use Cyan instead.

In the function uc_scan_attr() in ex_docmd.c there was a goto that jumped into a block with a local variable. That's illegal for some compilers.

Win32 GUI: There was a row of pixels at the bottom of the window which was not drawn. (Aaron)

Under DOS, editing "filename/" created a swap file of "filename/.swp". Should be "filename/_swp".

Win32 GUI: pointer was hidden when executing an external command.

When 'so' is 999, "J" near the end of the file didn't redisplay correctly.

":0a" inserted after the first line, instead of before the first line.

Unix: Wildcard expansion didn't handle single quotes and {} patterns. Now ":file 'window.c'" removes the quotes and ":e 'main*.c'" works (literal '*'). ":file {o}{n}{e}" now results in file name "one".

Memory leak when setting a string option back to its default value.

5.3 版本是 5.2 的漏洞修正版。改动不是很多。
5.2 和 5.3 之间的改进:

改动 changed-5.3

"IDE" 菜单更名为 "Tools" 菜单。

新增 added-5.3

Win32 GUI: 激活 Vim 时如果发现已经开始的编辑文件被改变，给出警告。(Negri)

修正 fixed-5.3

5.2.1: Win32 GUI: space for external command was not properly allocated, could cause a crash. (Aaron) This was the reason to bring out 5.3 quickly after 5.2.

5.2.2: Some commands didn't complain when used without an argument, although they need one: ":badd", ":browse", ":call", ":confirm", ":behave", ":delfunction", ":delcommand" and ":tearoff".
":endfunction" outside of a function gave wrong error message: "Command not implemented". Should be ":endfunction not inside a function".

5.2.3: Win32 GUI: When gvim was installed in "Program files", or another path with a space in it, executing external commands with vimrun didn't work.

5.2.4: Pasting with the mouse in Insert mode left the cursor on the last pasted character, instead of behind it.

5.2.5: In Insert mode, cursor after the end of the line, a shift-cursor-left didn't include the last character in the selection.

5.2.6: When deleting text from Insert mode (with "<C-O>D" or the mouse), which includes the last character in the line, the cursor could be left on the last character in the line, instead of just after it.

5.2.7: Win32 GUI: scrollbar was one pixel too big.

5.2.8: Completion of "PopUp" menu showed the derivatives "PopUpc", "PopUPi", etc. ":menu" also showed these.

5.2.9: When using two input() functions on a row, the prompt would not be drawn in column 0.

5.2.10: A loop with input() could not be broken with CTRL-C.

5.2.11: ":call asdf" and ":call asdf(" didn't give an error message.

5.2.12: Recursively using ":normal" crashes Vim after a while. E.g.:
":map gq :normal gq<CR>"

5.2.13: Syntax highlighting used 'iskeyword' from wrong buffer. When using
":help", then "/\k*" in another window with 'hlsearch' set.

5.2.14: When using ":source" from a function, global variables would not be
available unless "g:" was used.

5.2.15: XPM files can have the extension ".pm", which is the same as for Perl
modules. Added "syntax/pmfile.vim" to handle this.

5.2.16: On Win32 and Amiga, "echo expand("%:p:h")" removed one dirname in an
empty buffer. mch_Fullname() didn't append a slash at the end of a directory
name.

Should include the character under the cursor in the Visual area when using
'selection' "exclusive". This wasn't done for "%", "e", "E", "t" and "f".

""p would always put register 0, instead of the unnamed (last used) register.
Reverse the change that ""x doesn't write in the unnamed (last used) register.
It would always write in register 0, which isn't very useful. Use "-x for the
paste mappings in Visual mode.

When there is one long line on the screen, and 'showcmd' is off, "0$" didn't
redraw the screen.

Win32 GUI: When using 'mousehide', the pointer would flicker when the cursor
shape is changed. (Negri)

When cancelling Visual mode, and the cursor moves to the start, the wanted
column wasn't set, "k" or "j" moved to the wrong column.

When using ":browse" or ":confirm", was checking for a comment and separating
bar, which can break some commands.

Included fixes for Macintosh. (Kielhorn)

5.4 版 本 version-5.4

5.4 版本引入了新特性、有用的改变、以及很多的漏洞修正。

引入运行时目录 new-runtime-dir

现在，发布的运行时目录文件在 $VIMRUNTIME，即 $VIM 里的用户文件。
你通常不需要设置 $VIMRUNTIME，而应该让 Vim 去找。它使用 $VIM/vim{version}，或者如果那不存在， $VIM。这允许把用户文件和发布文件分离，使得版本更新更容易。这也使保持两个不同版本的 Vim 成为可能，每个带有它自己的运行时文件。

在 Unix 发布里，运行时文件移到了 "runtime" 目录。这可以用来一次复制所有的运行时文件，而不需了解需要复制什么。

DOS、Windows、Amiga 或 OS/2 的归档现在增加了额外的顶层 "vim" 目录。目的是清楚的说明用户定义文件应该放在这里。包含可执行文件的目录不能有 '-' 或 '.' 字符，以避免奇怪的扩展。

$VIM 和 $VIMRUNTIME 变量在第一次使用时定义。这使得它们，比如说，可以为 Perl 使用。

运行时文件也在 "$VIM/runtime" 里寻找。这有助于在只有解压运行时文件归档之后就可运行 Vim 的场合。如果用 "src" 目录里的可执行文件，Vim 检查该目录去掉 "src" 的部分之后是否可以加上 "vim54" 或 "runtime"。这使得编译后立即找到运行时文件。

可以在 Unix Makefile 里指定 $VIMRUNTIME 的缺省值。这可以使得 $VIM 不指向上述的运行时目录，而是，比如，"/etc/"。

引入文件类型 new-filetype-5.4

现在，语法文件使用新的 FileType 自动命令载入。旧的 "mysyntaxfile" 文件不再可用。 filetypes

载入语法高亮的脚本被修改，以使用新的 Syntax 自动命令事件。

Filetype 和 Syntax 事件的组合，包括 Syntax 菜单的选择，提供更好的语法高亮的微调。FileType 事件也可以用来设置该文件类型特定的选项和映射。

"$VIMRUNTIME/filetype.vim" 文件不是自动载入的。为此目的，增加了 ":filetype on" 命令。":syntax on" 也载入此脚本。

加入了 'filetype' 选项。它被用来激活 FileType 自动命令事件，就像 'syntax' 选项之于 Syntax 事件。

":set syntax=OFF" 和 ":set syntax=ON" 可以（在模式行里）开/关当前文件语法高亮。

Syntax 菜单命令被移到了 $VIMRUNTIME/menu.vim。":filetype on" 和 ":syntax manual" 都会打开 Syntax 菜单。

旧的 'filetype' 选项改名为 'osfiletype'。只有 RISCOS 使用它。'filetype' 现在用作通用的文件类型。

加入 ":syntax manual" 命令。允许手动选择，比如从模式行上，要加入的语法。

Vim 脚本续行符 new-line-continuation

如果 Ex 行以反斜杠开始，它被连接到上一行。这样，避免了长行的需要。
line-continuation (Roemer)
例如:

```
        if   has("dialog_con") ||
           \ has("dialog_gui")
            :let result = confirm("Enter your choice",
```

```
                                \ "&Yes\n&No\n&Maybe",
                                \ 2)
        endif
```

改进会话文件 improved-sessions

'sessionoptions' 的新单词:
- "help" 恢复帮助窗口。
- "blank" 恢复空窗口。
- "winpos" 恢复 Vim 窗口位置。使用新的 ":winpos" 命令。
- "buffers" 恢复隐藏和卸载的缓冲区。不使用它，只有窗口里的缓冲区会恢复。
- "slash" 把文件名里的反斜杠换成正斜杠。
- "globals" 保存全局变量。
- "unix" 使用 unix 文件格式 (<NL> 代替 <CR><NL>)

":mksession" 和 'sessionoptions' 现在出现在 +mksession 特性里。

使用会话文件时，同时恢复窗口的顶行位置。

":mksession" 和 ":mkvimrc" 不保存 'fileformat'，应该在载入文件时会被检测到。

(多数工作有 Vince Negri 和 Robert Webb 完成)

改进自动命令 improved-autocmds-5.4

新事件:

FileType	检测到文件类型时。
FocusGained	Vim 得到输入焦点时。(Negri)
FocusLost	Vim 失去输入焦点时。(Negri)
BufCreate	新缓冲区刚建立或换名时调用。(Madsen)
CursorHold	如果在 'updatetime' 时间内没有输入过键则激活。可以用来对光标所在的单词进行一些操作。(Negri) Unix 上实现了 CursorHold 自动命令事件。(Zellner) 也适用于 Amiga 和 MS-DOS。
GUIEnter	可以用在刚建立 GUI 窗口时完成一些事情。(比如，":winpos 100 50")。
BufHidden	缓冲区被隐藏时。用于当选项窗口隐藏时，删除之。

同时，在缓冲区将要换名前，激活 BufDelete 。(Madsen)

自动命令里，"<amatch>" 模式使用方法类似于 "<afile>"，只不过它是 FileType 和 Syntax 事件的匹配值。

如果自动命令里使用了 ":let @/ = <string>"，自动命令结束时会使用该最近搜索模式。

使自动命令的载入速度稍快了一些。通过记住新模式的长度，避免在每个退出模式上都执行 strlen()。

加密 new-encryption

写入文件时可以加密，读入文件时可以解密。加入 'key' 选项，"-x" 命令行参数和 ":X" 命令。 encryption （基于 Mohsin Ahmed 的补丁）

读入文件时，进行该文件是否加密的自动检测。Vim 这时会提示输入秘钥。

注意 加密的方法和 Vi 不兼容。该加密方法不是不可破解的。这是为了满足能从美国出口的条件。

GTK GUI 移植版本 new-GTK-GUI

新的 GTK+ 的 GUI 移植版本。包括工具栏、可撕下的菜单，等等。 gui-gtk
加入 :helpfind 命令。(Kahn 和 Dalecki)

菜单改变 menu-changes-5.4

菜单现在可以控制台上使用。用新的 'wildmenu' 选项可以打开。这会显示命令行补全的匹配，就像菜单一样。它的工作方式如同一个小型的文件浏览器。

新的 :emenu 命令可以用来执行菜单项。

使用最后的状态行列出项目，或者在命令行上方插入一行。(Negri)

'wildcharx' 选项可以用在映射打开 'wildmenu' 补全。

可以用 has("menu") 检测编译时是否不带菜单。也可以从 ":version" 的输出看到。可以编译没有菜单支持的 GUI 版本。只有在有菜单支持时，才能包含工具栏支持。

把 "Window" 菜单一路移到右边（优先级 70)。用 MS-Windows 工作的人会觉得更熟悉，而其它的人应该也不会受到影响。

包含 "Buffers" 菜单。它使用现有的自动命令和函数。可以通过设置 "no_buffers_menu" 变量关闭。(Aaron 和 Madsen)

Win32 支持菜单里的分隔符: "-.*-"。(Geddes)
Motif 里现在也应该支持菜单分隔符。

使得 Motif GUI 的弹出式菜单能够工作。(Madsen)

'guioptions' 的 'M' 标志位: 不载入系统菜单。

所有的菜单代码从 gui.c 移到了 menu.c。

改进 Viminfo improved-viminfo

'viminfo' 的新标志位:
'!' 如果全局变量以大写字母出现，保存在 viminfo 文件里。(Negri)

'h'　　载入 viminfo 文件时，执行 ":nohlsearch"。

viminfo 文件保存搜索模式的位移、magic 与否，等等。同时保存 'hlsearch' 高亮是否打开的标志位（如果 'viminfo' 有 'h' 标志位，不使用该记录）。

如果设置 'viminfo' 的时候没有逗号，给出错误信息。

若干新命令　　new-commands-5.4

操作符 g? : rot13 编码。(Negri)

zH 和 zL 命令：水平滚动半页。
gm 移动光标到屏幕行的中间。(Ideas by Campbell)

可视列块上的操作：v_b_I 、v_b_A 、v_b_c 、v_b_C 、v_b_r 、v_b_< 和 v_b_> 。(Kelly)

新命令：CTRL-\ CTRL-N，在普通模式里不作任何事，在插入或命令行模式里进入普通模式。可以在 VisVim 或其它 OLE 程序里使用，确保 Vim 在普通模式里，而且不会响铃。
CTRL-_CTRL-N

":cscope kill" 命令使用连接文件名。:cscope (Kahn)

:startinsert 命令：下一步启动插入模式。

:history 命令，显示所有四种类型的历史。(Roemer)

[m 、[M 、]m 和]M 命令，反向/正向跳转到 (Java) 类的方法的开始/结束处。

":@*" 执行 * 寄存器。:@ (Acevedo)

go 和 :goto 命令：根据文件的字节偏移跳转。

gR and gr 命令：虚拟替换模式。替换字符，而不改变布局。(Webb)

":cd -" 回到上一次 ":cd" 命令之前的目录。:cd- (Webb)

标签预览命令 :ptag 。在单独的窗口里显示 ":tag" 的结果。可以用来显示标签的上下文（比如，函数参数）。(Negri)
:pclose 命令和 CTRL-W CTRL-Z：关闭预览窗口。(Moore)
'previewheight' 选项，预览窗口的高度。
还有 :ppop 、:ptnext 、:ptprevious 、:ptNext 、:ptrewind 、:ptlast 。

:find 和 :sfind 命令：找到 'path' 里的文件，(分割窗口) 并编辑之。

:options 命令打开选项窗口，显示当前的选项值。或者使用 ":browse set" 来打开。选项按功能分组。提供每个选项简短的帮助。按 <CR> 可以得到更多的帮助。编辑选项值，然后在 "set" 行按 <CR> 可以设置新值。

若干新选项　　new-options-5.4

滚动绑定：'scrollbind' 和 'scrollopt' 选项。加入 :syncbind 命令。使得窗口滚动相同的量（水平和/或垂直）。(Ralston)

MS-DOS 的 'conskey' 选项。使用直接的控制台输入/输出。应该能用于 telnet（没有测试过！）。

'statusline' 选项：可以配置的状态行内容。也可以显示文件的字节偏移。可以使用 %1* 到 %9* 高亮，它们使用新的高亮组 User1 到 User9。(Madsen)

'rulerformat' 选项：可以配置的标尺内容，类似于 'statusline'。(Madsen)

'write' 选项：如果关闭，不允许写入文件。避免覆盖文件，即使是 ":w!" 也适用。-m 命令行选项复位 'write'。

'clipboard' 选项：如果使用剪贴板。值 "unnamed"：就像 "* 那样使用无名寄存器。(Cortopassi) 值 "autoselect"：就像 'guioptions' 里的 'a' 一样，但也能在终端上工作。

'guifontset' 选项：指定 +fontset 特性的字体，用于 X11 GUI 版本。在用这个特性编译 vim 时，可以指定普通所用的字体。(Nam)

'guiheadroom' 选项：允许 GUI 窗口之上/下保留多少空间。用于 Motif、Athena 和 GTK。

实现了 'tagstack' 选项：如果关闭，不允许把标签放到堆栈上（Vi 兼容）。对映射有用。

'shellslash' 选项。只用于使用反斜杠作为文件分隔符的系统。该选项会在扩展文件名时在文件名里使用正斜杠。可在 'shell' 为 sh 或 csh 时使用。

'pastetoggle' 选项：切换 'paste' 的键序列。解决 'paste' 置位时，映射在插入模式下不工作的问题。

'display' 选项：如果设为 "lastline"，末行填充窗口，而不是用 "@" 行代替。只有最后三个字符被 "@@@" 代替，指示该行尚未完成。

'switchbuf' 选项：允许重用已有的被跳转到的缓冲区所在窗口，否则分割窗口，以打开新的缓冲区。(Roemer)

'titleold' 选项。替换固定的字符串 "Thanks for flying Vim"，用来在退出时设置标题。

---

exists() 函数也可以检查函数是否存在。(Roemer)
现在用二分法查找内部函数，应该会快一些。(Roemer)

新函数：
- getwinposx() 和 getwinposy()：得到 Vim 窗口位置。(Webb)
- histnr()、histadd()、histget() 和 histdel()：提供历史。(Roemer)
- maparg()：返回映射的右边。基于 Vikas 的补丁。

- mapcheck() : 检查映射是否匹配已有的名字。
- visualmode() : 返回最近的可适模式的类型。(Webb)
- libcall() : 调用库里的函数，目前只适用于 Win32。(Negri)
- bufwinnr() : 寻找包含指定缓冲区的窗口。(Roemer)
- bufloaded() : 缓冲区是否存在且载入。
- localtime() 和 getftime() : 包装时钟时间和文件最近的修改时间 (Webb)
- glob() : 只扩展文件名通配符。
- system() : 得到外部命令的原始输出。(基于 Aaron 的补丁)。
- strtrans() : 翻译字符串成为可显示的字符。用于 2html.vim 脚本。
- append() : 缓冲区附加文本行的简易方法。

函数改动:
- strftime() 可选参数，给出以秒数计的时间。(Webb)
- expand() 现在也返回不存在文件的文件名。

允许用户命令名有数字。(Webb)

Vim 的内部变量使用 "v:": "v:errmsg"、"v:shell_error" 等。为了后相兼容，来自 5.3 版本的那些可以不用 "v:"。

新变量:
"v:warningmsg" 和 "v:statusmsg" 内部变量。包含最近给出的警告和状态信息。
v:warningmsg v:statusmsg (Madsen)
"v:count1" 变量: 类似于 "v:count"，但如果没有给出计数时，缺省为一。 v:count1

如果编译时不带表达式计算，"if 1" 可以用来包围不支持的命令，以免它们被执行。和 Vim 4.x 类似。
在使用以最小特性包编译的 Vim 时，有些运行时脚本会出错。现在，"if 1" 可以用来包围不总被支持的代码。

如果计算带 && 和 || 表达式，忽略不会影响结果的部分。这样速度更快，而且避免错误信息。(Webb)
同时对跳过 "if 0" 里的表达式进行优化。

避免 hit-enter 提示 avoid-hit-enter

加入 'shortmess' 的 'T' 标志位: 截短所有会产生 hit-enter 提示的消息 (除了总会发生的那些)。
'shortmess' 的 'O' 标志位现在也适用于 quickfix 消息，比如，从 ":cn" 命令那里。

'shortmess' 的缺省值现在是 "filnxtToO"，使得多数消息能放在命令行上，而不需要 hit-enter 提示。

以前的消息可以用新的 :messages 命令看到。

即使 'shortmess' 指示要缩短消息，有些消息仍然被完整显示，因为用户期待完整地看到它们: CTRL-G 和一些 quickfix 命令。

改进 quickfix improved-quickfix

分析 gmake 的改变目录行: "make[1]: Entering directory 'name'"。
使用 'errorformat' 的 "%D" 和 "%X"。
同时分析 make 的 "Making {target} in {dir}" 消息。有助于不使用 GNU make 的场合。(Schandl)

'errorformat' 的 "%f" 使用 'isfname'。

分析多行消息。 errorformat-multi-line

允许 :clist 命令指定行范围。(Roemer)

支持 "全局" 文件名，用于为多个错误输出一次文件名的错误格式。(Roemer)

:cnfile 跳转到下一个文件的第一个错误。

'makeprg' 里的 "$*" 被 ":make" 的参数替换。(Roemer)

正规表达式 regexp-changes-5.4

正规表达式里，"\)" 之前的 '$' 被认为是一行的结束。 /$
模式里，"\|" 或 "\(" 之后的 "^" 被认为是一行的开始。 /^ (Robinson)

正规表达式里，在 "\)" 和 "\|" 的前面，"$" 和 "\$" 过去都被认为是一行的结束。现在，"$" 用作行尾而 "\$" 用作按本义出现的美元符号。这同样适用于 "\(" 和 "\|" 之后的 '^'。 /\$ /\^

有些搜索模式即使它们实际上并不是不合法，但它们可能非常慢。比如:
"\([^a-z]\+\)\+Q"。用 CTRL-C 可以中止任何正规表达式的搜索。

寄存器 "/: 最近的搜索字符串 (只读)，(Kohan) 被改变为最近使用的搜索模式 (就像 'hlsearch' 用的一样)。可以这样设置搜索模式 ":let @/ = {expr}"。

搜索模式里，加入字符类，以避免从 'cpoptions' 里去除 'l' 标志位的需要:
[:tab:] 、 [:return:] 、 [:backspace:] 和 [:escape:] 。

在表达式的比较操作符之后加入 '?' 可以使该比较忽略大小写。 expr-==?

其它 5.3 和 5.4 版本之间的改进

改动 changed-5.4

Unix: 如果 $TMPDIR 设置且存在，用来存放临时文件。

删除 "Empty buffer" 信息。它没有用而且会产生 hit-enter 提示。(Negri)

"ex -" 现在从标准输入读入命令并以安静模式工作。和原来用于脚本的的 "ex" 命令兼容。

":tcldo" 的缺省范围是整个文件。

用 ESC 撤销可视模式过去移动光标。看来没有理由这么做。现在光标留在原来的位置。

":grep" 和 ":make" 命令把 " 看成参数的一部分，而不是注释的开始。

表达式里 "=~" 和 "!~" 操作符不再受 'ignorecase' 影响。

vimrc_example 换名为 vimrc_example.vim 而 gvimrc_example 换名为 gvimrc_example.vim。使它们能识别为 vim 脚本。

"gd" 不再从前一个函数的尾部开始搜索，而是从当前函数的开始处的之前的第一个空白行开始。避免在第一个函数上使用 "gd" 找到的是全局变量。

'complete' 的缺省值从 ".,b" 改为 ".,w,b,u,t,i"。能找到许多其它匹配，代价是时间长一点（但搜索可以中断）。

模式行上不再能设置 'shell*' 选项。以前只会给出警告。这样做减少了安全风险。

文档文件的索引顺序进行调整，以便更容易寻找某一主题。

MS-DOS 和 win32 上如果没有设置 $VIM，过去使用 $HOME。如果 $HOME 因为其它工具的缘故而设为，比如说，"C:\"，这会引起问题，因为会找不到运行时文件。现在 $HOME 只用来寻找 _vimrc、_gvimrc 等，而不用来寻找运行时文件。

如果 'tags' 为 "./{fname}" 而当前缓冲区没有文件名，就这么用。过去它被跳过，使得 "vim -t {tag}" 找不到许多标签。

在 'scrolloff' 区域如果用拖动试图选择文本，产生的滚动使之十分困难。现在暂时把 'scrolloff' 设为 0 或 1，以避免这一问题。但在顶行上还是允许滚动，以扩展到显示的文本范围之上。

'comments' 的缺省值现在包含 "sl:/*,mb: *,ex:*/"，使 Javadoc 注释可用。也有助于 "/*******" 开始的 C 注释。

CTRL-X CTRL-] 插入模式标签扩展在非 ID 字符之后试图扩展所有的标签，这样要花很长时间。现在限制为 200 个匹配。也用于命令行的标签补全。

OS/2 发布被分为两个文件。不然太大以致于在一张软盘上放不下。现在使用和 PC 相同的运行时归档。

文档里，非可选参数的 <a-z> 项目被 {a-z} 替换。这样避免了和键名 <C-Z> 的冲突，后者代表 CTRL-Z 而不是 C 到 Z 之间的字符，那应该用 {C-Z}。

新增 added-5.4

---

iris-ansi 内建 termcap 项目的色彩支持。(Tubman)

加入 VisVim 1.3a 版本。(Erhardt)

SNiFF+ 接口的 Win32 移植版本。(Leherbauer)
SniFF 接口文档: if_sniff.txt。(Leherbauer)

在 OleVim 目录里包含 "SendToVim" 和 "OpenWithVim" 程序。用于 MS-Windows 上的 OLE 版本的 gvim。(Schaller)

包含 Exuberant Ctags 3.2.4 版本，提供 Eiffel 支持。(Hiebert)

如果删除了正在编辑中文件，给出警告 (就像改变修改时间一样)。

包含新版本生成 HTML 的 Awk 和 Perl 脚本。(Colombo)

基于 "gpm" 的 Linux 控制台鼠标支持。(Tsindlekht)

安全修正：不允许从模式行改变 'secure' 和 'exrc'。如果置位 'secure'，修改包含程序名的选项时给出警告。

Perl 接口支持 Perl 5.005 和线程。(Verdoolaege)

给出关于二义性映射的错误信息时，同时包含引起问题的映射。(Roemer)

命令行编辑：
- 命令行的映射补全。(Roemer)
- 命令行 ":function"、":delfunction"、":let"、":call"、":if" 等的补全。(Roemer)
- 使用 CTRL-D 对有 "-complete=tag_listfiles" 的用户命令进行补全时同时列出文件名。(Madsen)
- 补全 ":command" 命令的参数。(Webb)
- 命令行的 CTRL-R . 插入最近插入的文本。CTRL-R 之后的 CTRL-F、CTRL-P、CTRL-W 和 CTRL-A 用于插入光标所在的对象。(Madsen)

uganda.txt 的文本关于复制 Vi 的部分更加明确。

更新 Vim 教程。增加 "vimtutor" 命令，它复制了教程并启动 Vim 显示该教程。"make install" 现在也复制教程。

":clist" 的输出使用 'i' 高亮方式高亮当前项目 (和 'incsearch' 所用的相同)。

":clist" 命令可以用 "b" 往回滚动 (一整屏)、"u" (半屏) 和 "k" (一行)。

多字节支持：
- 多字节字符的 X 输入法。多字节支持的其它一些修正。(Nam)
- Hangul 输入法特性：hangul。(Nam)
- 清理了多字节支持、XIM、字体集和 Hangul 输入的配置。现在每项可以单独配置。
- 修改 GTK_KEYBOARD 的检查为对 HANGUL_KEYBOARD_TYPE 的检查。(Nam)
- 加入 doc/hangulin.txt: Hangul 输入代码的文档。(Nam)
- GTK+ 的 XIM 支持。(Nam)
- 包含 SJTS 编码支持的第一次尝试。(Nagano)
- 如果多字节字符在行尾放不下，在那里放上 "~" 然后在下一行显示。
- 优化了多字节文本的输出。(Park)
- Win32 IME: 预编辑风格相当于 over-the-spot。(Nagano)
- Win32 IME: IME 模式的改变现在使用 ImmSetOpenStatus。(Nagano)
- GUI Athena: 文件选择对话框现在可以显示多字节字符。(Nagano)
- XA_TEXT 的选择区应答是 XA_STRING。(Nagano)

"runtime/macros/diffwin.vim"。建立比较窗口的映射。(Campbell)

'suffixes' 选项加入 ".obj"。

用 ":SynAu" 用户命令，减小了 syntax/synload.vim 的长度。
menu.vim 自动给 Syntax 菜单项目编号。
Syntax 菜单中为开头字母不同的语法名之间插入分隔符。(Geddes)

xterm:
- 在 xterm 里使用鼠标时提供剪贴板支持。(Madsen)
- 使用 xterm 鼠标时，跟踪鼠标的拖动。如有可能，使用 xterm 转义序列，比其它方法要精确，但需要相当新的 xterm 版本。'ttymouse' 设为 "xterm2" 打开这一功能。(Madsen)
- 检查 xterm 的补丁号以设置 'ttymouse'。只用于最新的 xterm (补丁号 > 95)。使用新的 't_RV' termcap 选项。如果能够识别应答为正确，设置 'ttymouse' 为 "xterm2"。这样 xterm 鼠标的拖动工作得更好。
- 支持 xterm 上 shift + 功能键。修改 shift + 光标键为 xterm 实际产生的值。加入 shift + <End> 和 <Home> 的编码。
- 加入 't_WP'，像素计的窗口位置，和 't_WS'，字符计的窗口大小。xterm 现在可以移动 (用于 ":winpos") 和调整大小 (用于 ":set lines=" 和 ":set columns=")。

X11:
- 可视模式下如果没有拥有选择区，使用 VisualNOS 组显示可视区域。(Madsen)
- 支持剪贴板支持类型的请求。用于 AIX 和 dtterm。(Wittig)
- 支持 compound_text 选择 (即使编译时没有多字节支持)。

交换文件:
- 交换文件命名的新变化：把路径分隔符替换为 %，把所有的交换文件都放在一个目录。如果 'dir' 的某个名字以两个路径分隔符结尾时有用。(Madsen)
- 如果找到交换文件，相关信息里显示它是否包含了修改。(Madsen)
- 如果支持对话框，交换文件已存在时使用对话框询问用户应该如何做。

'mousemodel' 选项的 "popup_setpos"。允许使用鼠标右键时移动光标。

如果删除缓冲区，要显示的缓冲区现在变成了跳转表的最近的项目。(Madsen)

用 CTRL-O/CTRL-I 跳过删除的缓冲区。

标尺里如有空间，显示百分比。

用 autoconf 1.13 生成 configure。

包含 Dirk van Deun 的 get_lisp_indent()。如果 'cpoptions' 不包含 'p' 标志位，提供更好的 Lisp 缩进。

2html.vim 脚本加快不少。(基于 Geddes 的建议)

Unix:
- version 消息里包含了编译 Vim 的用户名和使用的系统名。
- "make install" 现在也安装 "tools" 目录。使得所有人都能用它们。
- "make check" 现在和 "make test" 相同。"make test" 检查可视列块模式的左右移、插入、替换和修改。
- 加快文件名和已有缓冲区的比较速度。方法是保存缓冲区的 device/inode 号。
- configure 参数 "--disable-gtk"、"--disable-motif" 和 "--disable-athena"，关闭特定的 GUI (如果不能用)。
- 换名 configure 关闭检查特定 GUI 的参数。现在更清楚。(Kahn)

- Digital Unix 系统上 ("OSF1")，在 termlib 和 termcap 之前检查 curses 库。(Schild)
- "make uninstall_runtime" 只会删除特定版本的文件。可以用于删除以前版本的运行时文件。

Macintosh: (St-Amant)
- 拖动滚动条的行为类似于 Win32 GUI。把 gui_w32.c 的公共代码移到 gui.c。
- 加入对话框和文件浏览。
- 保留资源分叉 (resource fork)，如果丢失给出警告。
- 复制原始文件的属性到新写入的文件。
- 已修正设置 title/notitle 的漏洞。
- 改进了文件名补全。
- Grow box 的限制调整为基于字符的大小。
- 用 rgb.txt 提供更多颜色 (但不能用的颜色又回来了)。
- Apple 菜单能用了 (除了 about...)。
- 内部边界现在和 vim 相容。
- 删除菜单不再崩溃。
- Python 1.5.1 的弱连接 (只用于 PPC)。需要库来提供 Python 的支持。
- 执行用户 .vimrc 文件时如果出错，现在立即显示警告窗口，缺省按钮为 OK，每行的开头不再有 "Delete" 键的符号。
- 更好地管理环境变量。现在 $VIM 只计算一次，而不是每次使用时都重新生成。
- 后台运行时不再大量占用 CPU。
- 用于执行的 Vim 脚本可以识别 Mac 的文件格式，就像 DOS 文件格式那样。

如果 'fileformats' 里同时有 "unix" 和 "mac" 并且 CR 比 NL 字符多，优先选择 "mac" 格式。
用 "mac" 文件格式时用 CR 代替 NL，因为 NL 用作 NUL。文件的所有字符必须都保留下来。(Madsen)

DOS install.exe 现在对已存在的安装进行检查。避免重复设置 $VIM 和 $PATH。
Dos/Windows 的安装程序现在可以通过加入两个注册表键来安装 Vim 到弹出菜单。

EGCS/mingw32 的移植。新的 Makefile.ming。(Aaron)

DOS 16 位：不包含光标形状的代码。节省一些字节。

TCL 支持加入 Makefile.w32。(Duperval)

OS/2: 用 argv[0] 来寻找运行时文件。

用 "gf" 转到已经使用的缓冲区时，跳转到上次光标所在的行。

为 ":tselect" 的输出提供更多的颜色。用不同的高亮区别标签名和文件名。
("struct:") 字段名的高亮和参数也进行区隔。

非 Unix 系统的反引号扩展。基于 Aaron 的补丁。允许用 ":n `grep -l test *.c`" 和 "echo expand(``ls m*``)"。

设置时检查 'complete' 选项。(Acevedo)
'complete' 的 'd' 标志位检查宏定义的名字。
插入模式搜索头文件和标签文件的补全时，检查预输入，从而可以早匹配。(Webb)
'complete' 的 '.' 标志位现在完全扫描当前缓冲区，不依赖 'nowrapscan'。(Webb)

'whichwrap' 加入 '~' 标志位。(Acevedo)

结束可视模式时（例如用 ESC），不夺取选择区的所有权。

彩色终端里，可用 "fg" 和 "bg" 作颜色名。它们分别代表 "Normal" 的两种颜色。

一些 cscope 的清理。(Kahn)

包含 Schemenauer 提供的 vimspell.sh 修改。

表达式里用 "." 进行字符串连接的速度稍稍加快。(Roemer)

":redir" 命令现在可以重定向到寄存器: ":redir @r". (Roemer)

":marks" 现在和 ":jumps" 的输出格式类似，如果位置标记位于当前文件，显示它之后的文本。":tags" 亦然。

如果 configure 能找到 ftello() 和 fseeko()，在 tag.c 里使用（如果你有非常庞大的标签文件时有用）。

configure 检查编译器的 "-FOlimit,2000" 参数。(Borsenkow)

GUI:
- 在非 GUI 的 Vim 上使用 ":gui" 给出清楚的错误信息。
- "gvim -v" 不启动 GUI（如果支持终端的话）。
- Ex 模式里，使用整屏的非可视选择。
- 以 "gvim -f" 启动并在 .gvimrc 文件里使用 ":gui" 时，Vim 过去仍然进行分叉 (fork)。现在记住了 "-f" 标志位并用于 ":gui"。加入 "gui -b" 使得 gvim 总在后台运行。

Motif GUI:
- configure 检查 "-lXp" 库（但还不能用 ...）。
- configure 在 "/usr/local/Lesstif/Motif*" 检查 Lesstif。修改顺序使 local 的 Motif 版本能覆盖系统标准版本。

Win32 GUI:
- 非 OLE 版本使用 "-register" 或 "-unregister" 报错。
- 使用 GTK 工具栏图标。使窗口边界看起来舒服点。窗口左下/右下角使用可改变大小的指针。(Negri)
- 用 ":!start" 启动外部命令但不能执行该命令时报错。(Webb)
- 滚动条下方的灰色矩形上使用可改变大小的指针。用平坦模式来画工具栏。这样看起来舒服点。(Negri)
- 为 MS-Windows 3.1 的加入做好准备。主要把 WIN32 改为 MSWIN，USE_GUI_WIN32 改为 USE_GUI_MSWIN。(Negri)

buflist_findpat() 里不要分配相同的字符串四次。(Williams)

用 termcap 选项 't_ts'、't_fs'、't_IS' 和 't_IE' 设置标题和图标文本。支持任何支持设置标题和/或图标文本的终端。(Schild)

'comments' 的新 'x' 标志位: 输入结尾字符串的最后一个字符时，自动插入结尾部分。有助于结束 C 的 /* */ 注释。(Webb)

expand() 有第二个参数且非零时，不用 'suffixes' 和 'wildignore'，返回所有匹配。

'cpoptions' 的 'O' 标志位：如果不包含，那么当文件在编辑开始时不存在但缓冲区写回时却存在的时候，Vim 不会覆盖文件。文件一定在 Vim 之外被建立，用户自己都不一定意识到这一点。如果在外壳命令之后发现这一点，给出警告。

编辑新文件时，CTRL-G 显示 [New file]。读入文件如果有错，CTRL-G 显示 [Read errors]。

":wall" 现在可以在需要时使用对话框和文件浏览。

组合若干功能成为新的特性，这样做的主要目的是为了能控制最小版本的大小：

+linebreak:	'showbreak'、'breakat' 和 'linebreak'
+visualextra:	可视列块模式的插入 ("I") 和附加 ("A")、列块中 修改 ("c") 所有行、 ">" 和 "<": 左/右移列块、"r": 用单个字符替换可视区域。
+comments:	'comments'
+cmdline_info:	'ruler' 和 'showcmd'。替换过去的 +showcmd。
"+title"	MSDOS 不加入设置标题和图标的代码，反正这也不可能做到。
+cmdline_compl	编译时关闭除了文件、目录和帮助项目之外的命令行补全。

特性从函数调用列表移到一个数组。这样可以节省一点空间。

输入函数体时，根据 "if" 和 "while" 命令调整缩进。

VMS: 根据 Arpadffy 的建议稍稍调整 os_vms.mms。

'comments' 选项的标志位现在可以包含位移。这样就可以用相同的 'comments' 设置对齐 "/*****"、"/* xxx" 和 "/*" 注释。'comments' 的缺省值用到这一特性。
加入 'O' 标志位：这部分不用于 "O" 命令。用于 "set com=sO:*\ -,mO:*\ \ ,exO:*/"

FileType 自动命令识别 ".bak"、".orig" 和 "~" 扩展名，删除它们以寻找相关的扩展名。

编写 Vim 脚本部分的教程得到扩充。

帮助文件提供更多高亮，用于不能按本义输入的项目。

提供了 "CTRL-W CTRL-G"，相当于 "CTRL-W g"。

OS/2 的 "make test"。

调整 configure，BeOS 自动使用 GUI。

修正 fixed-5.4

5.3.1: When using an autocommand for BufWritePre that changes the name of the buffer, freed memory would be used. (Geddes)

Mac: Compiler didn't understand start of skip_class_name().

Win32 GUI:
- When cancelling the font requester, don't give an error message.
- When a tearoff-menu is open and its menu is deleted, Vim could crash. (Negri)

- There was a problem on Windows 95 with (un)maximizing the window.
  (Williams)
- when 'mousehide' is set, the mouse would stay hidden when a menu is dropped
  with the keyboard. (Ralston)
- The tempname() function already created the file. Caused problems when
  using ":w". Now the file is deleted.
- Cursor disappeared when ending up in the top-left character on the screen
  after scrolling. (Webb)
- When adding a submenu for a torn-off menu, it was not updated.
- Menu tooltip was using the toolbar tooltip. (Negri)
- Setting 'notitle' didn't remove the title. (Steed)
- Using ":!start cmd" scrolled the screen one line up, and didn't wait for
  return when the command wasn't found.

Cscope interface: Sorting of matches was wrong. Starting the interface could
fail. (Kahn)

Motif GUI: Could not compile with Motif 1.1, because some tear-off
functionality was not in #ifdefs.

Configure could sometimes not compile or link the test program for sizeof(int)
properly. This caused alignment problems for the undo structure allocations.
Added a safety check that SIZEOF_INT is not zero.

Added configure check to test if strings.h can be included after string.h.
Some systems can't handle it.
Some systems need both string.h and strings.h included. Adjusted vim.h for
that. Removed including string.h from os_unixx.h, since it's already in
vim.h. (Savage)
AIX: defining _NO_PROTO in os_unix.h causes a conflict between string.h and
strings.h, but after the configure check said it was OK. Also define
_NO_PROTO for AIX in the configure check. (Winn)

When closing a window with CTRL-W c, the value of 'hidden' was not taken into
account, the buffer was always unloaded. (Negri)

Unix Makefile: "make install" always tried to rename an older executable and
remove it. This caused an error message when it didn't exit. Added a check
for the existence of an old executable.
The command line for "make install" could get too long, because of the many
syntax files. Now first do a "cd" to reduce the length.

On RISCOS and MSDOS, reading a file could fail, because the short filename was
used, which can be wrong after a ":!cd".

In the DOS versions, the wrong install.exe was included (required Windows).
Now the install.exe version is included that is the same as the Vim version.
This also supports long file names where possible.

When recording, and stopping while in Insert mode with CTRL-O q, the CTRL-O
would also be recorded.

32bit DOS version: "vim \file", while in a subdirectory, resulted in "new
file" for "file" in the local directory, while "\file" did exist. When

"file" in the current directory existed, this didn't happen.

MSDOS: Mouse could not go beyond 80 columns in 132 columns mode. (Young)

"make test" failed in the RedHat RPM, because compatible is off by default.

In Insert mode <C-O><C-W><C-W> changes to other window, but the status bars were not updated until another character was typed.

MSDOS: environment options in lowercase didn't work, although they did in the Win32 versions. (Negri)

After ":nohlsearch", a tag command switched highlighting back on.

When using "append" command as the last line in an autocommand, Vim would crash.

RISCOS: The scroll bumpers (?) were not working properly. (Leonard)

"zl" and "zh" could move the cursor, but this didn't set the column in which e.g., "k" would move the cursor.

When doing ":set all&" the value of 'scroll' was not set correctly. This caused an error message when later setting any other number option.

When 'hlsearch' highlighting has been disabled with ":nohlsearch", incremental searching would switch it back on too early.

When listing tags for ":tselect", and using a non-search command, and the last character was equal to the first (e.g., "99"), the last char would not be shown.

When searching for tags with ":tag" Vim would assume that all matches had been found when there were still more (e.g. from another tags file).

Win32: Didn't recognize "c:\" (e.g., in tags file) as absolute path when upper/lowercase was different.

Some xterms (Debian) send <Esc>OH for HOME and <Esc>OF for END. Added these to the builtin-xterm.

In ex mode, any CR was seen as the end of the line. Only a NL should be handled that way. broke ":s/foo/some^Mtext/".

In menu.vim, a vmenu was used to override an amenu. That didn't work, because the system menu file doesn't overwrite existing menus. Added explicit vunmenu to solve this.

Configure check for terminal library could find a library that doesn't work at runtime (Solaris: shared library not found). Added a check that a program with tgoto() can run correctly.

Unix: "echo -n" in the Makefile doesn't work on all systems, causing errors compiling pathdef.c. Replaced it with "tr".

Perl: DO_JOIN was redefined by Perl. Undefined it in the perl files.

Various XIM and multi-byte fixes:
- Fix user cannot see his language while he is typing his language with
  off-the-spot method. (Nagano)
- Fix preedit position using text/edit area (using gui.wid). (Nagano)
- remove 'fix dead key' codes. It was needed since XNFocusWindow was
  "x11_window", XNFocusWindow is now gui.wid. (Nagano)
- Remove some compile warnings and fix typos. (Namsh)
- For status area, check the gtk+ version while Vim runs. I believe it is
  better than compile time check. (Namsh)
- Remove one FIXME for gtk+-xim. (Namsh)
- XIM: Dead keys didn't work for Czech. (Vyskovsky)
- Multibyte: If user input only 3byte such as mb1_mb2_eng or eng_mb1_mb2 VIM
  could convert it to special character. (Nam)
- Athena/Motif with XIM: fix preedit area. (Nam)
- XIM: Composed strings were sometimes ignored. Vim crashed when compose
  string was longer than 256 bytes. IM's geometry control is fixed. (Nam,
  Nagano)
- Win32 multi-byte: hollowed cursor width on a double byte char was wrong.
  (Nagano)
- When there is no GUI, selecting XIM caused compilation problems.
  Automatically disable XIM when there is no GUI in configure.
- Motif and Athena: When compiled with XIM, but the input method was not
  enabled, there would still be a status line. Now the status line is gone if
  the input method doesn't work. (Nam)

Win32: tooltip was not removed when selecting a parent menu (it was when
selecting a menu entry). (Negri)

Unix with X: Some systems crash on exit, because of the XtCloseDisplay() call.
Removed it, it should not be necessary when exiting.

Win32: Crash on keypress when compiled with Borland C++. (Aaron)

When checking for Motif library files, prefer the same location as the include
files (with "include" replaced with "lib") above another entry.

Athena GUI: Changed "XtOffset()" in gui_at_fs.c to "XtOffsetOf()", like it's
used in gui_x11.c.

Win32: When testing for a timestamp of a file on floppy, would get a dialog
box when the floppy has been removed. Now return with an error. (Negri)

Win32 OLE: When forced to come to the foreground, a minimized window was still
minimized, now it's restored. (Zivkov)

There was no check for a positive 'shiftwidth'. A negative value could cause
a hangup, a zero value a crash.

Athena GUI: horizontal scrollbar wasn't updated correctly when clicking right
or left of the thumb.

When making a Visual-block selection in one window, and trying to scroll another, could cause errors for accessing non-existent line numbers.

When 'matchpairs' contains "‘:’", jumping from the ‘ to the ’ didn't work properly.

Changed '\"' to '"' to make it compatible with old C compilers.

The command line expansion for mappings caused a script with a TAB between lhs and rhs of a map command to fail. Assume the TAB is to separate lhs and rhs when there are no mappings to expand.

When editing a file with very long lines with 'scrolloff' set, "j" would sometimes end up in a line which wasn't displayed.

When editing a read-only file, it was completely read into memory, even when it would not fit. Now create a swap file for a read-only file when running out of memory while reading the file.

When using ":set cino={s,e-s", a line after "} else {" was not indented properly. Also added a check for this in test3.in.

The Hebrew mapping for the command line was remembered for the next command line. That isn't very useful, a command is not Hebrew. (Kol)

When completing file names with embedded spaces, like "Program\ files", this didn't work. Also for user commands. Moved backslash_halve() down to mch_expandpath().

When using "set mouse=a" in Ex mode, mouse events were handled like typed text. Then typing "quit" screwed up the mouse behavior of the xterm.

When repeating an insert with "." that contains a CTRL-Y, a number 5 was inserted as "053".

Yanking a Visual area, with the cursor past the line, didn't move the cursor back onto the line. Same for "~", "u", "U" and "g?"

Win32: Default for 'grepprg' could be "findstr /n" even though there is no findstr.exe (Windows 95). Check if it exists, and fall back to "grep -n" if it doesn't.

Because gui_mouse_moved() inserted a leftmouse click in the input buffer, remapping a leftmouse click caused strange effects. Now Insert another code in the input buffer. Also insert a leftmouse release, to avoid the problem with ":map <LeftMouse> l" that the next release is seen as the release for the focus click.

With 'wrap' on, when using a line that doesn't fit on the screen, if the start of the Visual area is before the start of the screen, there was no highlighting. Also, 'showbreak' doesn't work properly.

DOS, Win32: A pattern "[0-9]\+" didn't work in autocommands.

When creating a swap file for a buffer which isn't the current buffer, could get a mixup of short file name, resulting in a long file name when a short file name was required. makeswapname() was calling modname() instead of buf_modname().

When a function caused an error, and the error message was very long because of recursiveness, this would cause a crash.

'suffixes' were always compared with matching case. For MS-DOS, Win32 and OS/2 case is now ignored.

The use of CHARBITS in regexp.c didn't work on some Linux. Don't use it.

When generating a script file, 'cpo' was made empty. This caused backslashes to disappear from mappings. Set it to "B" to avoid that.

Lots of typos in the documentation. (Campbell)

When editing an existing (hidden) buffer, jump to the last used cursor position. (Madsen)

On a Sun the xterm screen was not restored properly when suspending. (Madsen)

When $VIMINIT is processed, 'nocompatible' was only set after processing it.

Unix: Polling for a character wasn't done for GPM, Sniff and Xterm clipboard all together. Cleaned up the code for using select() too.

When executing external commands from the GUI, some typeahead was lost. Added some code to regain as much typeahead as possible.

When the window height is 5 lines or fewer, <PageDown> didn't use a one-line overlap, while <PageUp> does. Made sure that <PageUp> uses the same overlap as <PageDown>, so that using them both always displays the same lines.

Removed a few unused functions and variables (found with lint).

Dictionary completion didn't use 'infercase'. (Raul)

Configure tests failed when the Perl library was not in LD_LIBRARY_PATH. Don't use the Perl library for configure tests, add it to the linker line only when linking Vim.

When using ncurses/terminfo, could get a 't_Sf' and 't_Sb' termcap entry that has "%d" instead of "%p1%d". The light background colors didn't work then.

GTK GUI with ncurses: Crashed when starting up in tputs(). Don't use tputs() when the GUI is active.

Could use the ":let" command to set the "count", "shell_error" and "version" variables, but that didn't work. Give an error message when trying to set them.

On FreeBSD 3.0, tclsh is called tclsh8.0. Adjusted configure.in to find it.

When Vim is linked with -lncurses, but python uses -ltermcap, this causes trouble: "OOPS". Configure now removes the -ltermcap.

:@" and :*" didn't work properly, because the " was recognized as the start of a comment.

Win32s GUI: Minimizing the console where a filter command runs in caused trouble for detecting that the filter command has finished. (Negri)

After executing a filter command from an xterm, the mouse would be disabled. It would work again after changing the mode.

Mac GUI: Crashed in newenv(). (St-Amant)

The menus and mappings in mswin.vim didn't handle text ending in a NL correctly. (Acevedo)

The ":k" command didn't check if it had a valid argument or extra characters. Now give a meaningful error message. (Webb)

On SGI, the signal function doesn't always have three arguments. Check for struct sigcontext to find out. Might still be wrong...

Could crash when using 'hlsearch' and search pattern is "^".

When search patterns were saved and restored, status of no_hlsearch was not also saved and restored (from ":nohlsearch" command).

When using setline() to make a line shorter, the cursor position was not adjusted.

MS-DOS and Win95: When trying to edit a file and accidentally adding a slash or backslash at the end, the file was deleted. Probably when trying to create the swap file. Explicitly check for a trailing slash or backslash before trying to read a file.

X11 GUI: When starting the GUI failed and received a deadly signal while setting the title, would lock up when trying to exit, because the title is reset again. Avoid using mch_settitle() recursively.

X11 GUI: When starting the GUI fails, and then trying it again, would crash, because argv[] has been freed and x11_display was reset to NULL.

Win32: When $HOME was set, would put "~user" in the swap file, which would never compare with a file name, and never cause the attention message. Put the full path in the swap file instead.

Win32 console: There were funny characters at the end of the "vim -r" swap files message (direct output of CR CR LF).

DOS 32 bit: "vim -r" put the text at the top of the window.

GUI: With 'mousefocus' set, got mouse codes as text with "!sleep 100" or "Q".

Motif and Win32 GUI: When changing 'guifont' to a font of the same size the screen wasn't redrawn.

Unix: When using ":make", jumping to a file b.c, which is already open as a symbolic link a.c, opened a new buffer instead of using the existing one.

Inserting text in the current buffer while sourcing the .vimrc file would cause a crash or hang. The memfile for the current buffer was never allocated. Now it's allocated as soon as something is written in the buffer.

DOS 32 bit: "lightblue" background worked for text, but not drawn parts were black.

DOS: Colors of console were not restored upon exiting.

When recording, with 'cmdheight' set to 2 and typing Esc> in Insert mode caused the "recording" message to be doubled.

Spurious "file changed" messages could happen on Windows. Now tolerate a one second difference, like for Linux.

GUI: When returning from Ex mode, scrollbars were not updated.

Win32: Copying text to the clipboard containing a <CR>, pasting it would replace it with a <NL> and drop the next character.

Entering a double byte character didn't work if the second byte is in [xXoO]. (Eric Lee)

vim_realloc was both defined and had a prototype in proto/misc2.pro. Caused conflicts on Solaris.

A pattern in an autocommand was treated differently on DOS et al. than on Unix. Now it's the same, also when using backslashes.

When using <Tab> twice for command line completion, without a match, the <Tab> would be inserted. (Negri)

Bug in MS-Visual C++ 6.0 when compiling ex_docmd.c with optimization. (Negri)

Testing the result of mktemp() for failure was wrong. Could cause a crash. (Peters)

GUI: When checking for a ".gvimrc" file in the current directory, didn't check for a "_gvimrc" file too.

Motif GUI: When using the popup menu and then adding an item to the menu bar, the menu bar would get very high.

Mouse clicks and special keys (e.g. cursor keys) quit the more prompt and dialogs. Now they are ignored.

When at the more-prompt, xterm selection didn't work. Now use the 'r' flag in

'mouse' also for the more-prompt.

When selecting a Visual area of more than 1023 lines, with 'guioptions' set to "a", could mess up the display because of a message in free_yank(). Removed that message, except for the Amiga.

Moved auto-selection from ui_write() to the screen update functions. Avoids unexpected behavior from a low-level function. Also makes the different feedback of owning the selection possible.

Vi incompatibility: Using "i<CR>" in an indent, with 'ai' set, used the original indent instead of truncating it at the cursor. (Webb)

":echo x" didn't stop at "q" for the more prompt.

Various fixes for Macintosh. (St-Amant)

When using 'selectmode' set to "exclusive", selecting a word and then using CTRL-] included the character under the cursor.

Using ":let a:name" in a function caused a crash. (Webb)

When using ":append", an empty line didn't scroll up.

DOS etc.: A file name starting with '!' didn't work. Added '!' to default for 'isfname'.

BeOS: Compilation problem with prototype of skip_class_name(). (Price)

When deleting more than one line, e.g., with "de", could still use "U" command, which didn't work properly then.

Amiga: Could not compile ex_docmd.c, it was getting too big. Moved some functions to ex_cmds.c.

The expand() function would add a trailing slash for directories.

Didn't give an error message when trying to assign a value to an argument of a function. (Webb)

Moved including sys/ptem.h to after termios.h. Needed for Sinix.

OLE interface: Don't delete the object in CVimCF::Release() when the reference count becomes zero. (Cordell)
VisVim could still crash on exit. (Erhardt)

"case a: case b:" (two case statements in one line) aligned with the second case. Now it uses one 'sw' for indent. (Webb)

Font initialisation wasn't right for Athena/Motif GUI. Moved the call to highlight_gui_started() gui_mch_init() to gui_mch_open(). (Nam)

In Replace mode, backspacing over a TAB before where the replace mode started while 'sts' is different from 'ts', would delete the TAB.

Win32 console: When executing external commands and switching between the two console screens, Vim would copy the text between the buffers. That caused the screen to be messed up for backtick expansion.

":winpos -1" then ":winpos" gave wrong error message.

Windows commander creates files called c:\tmp\$wc\abc.txt. Don't remove the backslash before the $. Environment variables were not expanded anyway, because of the backslash before the dollar.

Using "-=" with ":set" could remove half a part when it contains a "\,".
E.g., ":set path+=a\\,b" and then "set path-=b" removed ",b".

When Visually selecting lines, with 'selection' set to "inclusive", including the last char of the line, "<<" moved an extra line. Also for other operators that always work on lines.

link.sh changed "-lnsl_s" to "_s" when looking for "nsl" to be removed.
Now it only remove whole words.

When jumped to a mark or using "fz", and there is an error, the current column was lost. E.g. when using "$fzj".

The "g CTRL-G" command could not be interrupted, even though it can take a long time.

Some terminals do have <F4> and <xF4>. <xF4> was always interpreted as <F4>.
Now map <xF4> to <F4>, so that the user can override this.

When compiling os_win32.c with MIN_FEAT the apply_autocmds() should not be used. (Aaron)

This autocommand looped forever: ":au FileChangedShell * nested e <afile>"
Now FileChangeShell never nests. (Roemer)

When evaluating an ":elseif" that was not going to matter anyway, ignore errors. (Roemer)

GUI Lesstif: Tearoff bar was the last item, instead of the first.

GUI Motif: Colors of tear-off widgets was wrong when 't' flag added to 'guioptions' afterwards. When 't' flag in 'guioptions' is excluded, would still get a tearoff item in a new menu.

An inode number can be "long long". Use ino_t instead of long. Added configure check for ino_t.

Binary search for tags was using a file offset "long" instead of "off_t".

Insert mode completion of tags was not using 'ignorecase' properly.

In Insert mode, the <xFn> keys were not properly mapped to <Fn> for the default mappings. Also caused errors for ":mkvimrc" and ":mksession".

When jumping to another window while in Insert mode, would get the "warning: changing readonly file" even when not making a change.

A '(' or '{' inside a trailing "//" comment would disturb C-indenting.
When using two labels below each other, the second one was not indented properly. Comments could mess up C-indenting in many places. (Roemer)

Could delete or redefine a function while it was being used. Could cause a crash.
In a function it's logical to prepend "g:" to a system variable, but this didn't work. (Roemer)

Hangul input: Buffer would overflow when user inputs invalid key sequence. (Nam)

When BufLoad or BufEnter autocommands change the topline of the buffer in the window, it was overruled and the cursor put halfway the window. Now only put the cursor halfway if the autocommands didn't change the topline.

Calling exists("&option") always returned 1. (Roemer)

Win32: Didn't take actually available memory into account. (Williams)

White space after an automatically inserted comment leader was not removed when 'ai' is not set and <CR> hit just after inserting it. (Webb)

A few menus had duplicated accelerators. (Roemer)

Spelling errors in documentation, quite a few "the the". (Roemer)

Missing prototypes for Macintosh. (Kielhorn)

Win32: When using 'shellquote' or 'shellxquote', the "!start cmd" wasn't executed in a disconnected process.

When resizing the window, causing a line before the cursor to wrap or unwrap, the cursor was displayed in the wrong position.

There was quite a bit of dead code when compiling with minimal features.

When doing a ":%s///" command that makes lines shorter, such that lines above the final cursor position no longer wrap, the cursor position was not updated.

get_id_list() could allocate an array one too small, when a "contains=" item has a wildcard that matches a group name that is added just after it. E.g.: "contains=a.*b,axb". Give an error message for it.

When yanking a Visual area and using the middle mouse button -> crash. When clipboard doesn't work, now make "* always use "".

Win32: Using ":buf a\ b\file" didn't work, it was interpreted as "ab\file".

Using ":ts ident", then hit <CR>, with 'cmdheight' set to 2: command line was

not cleared, the tselect prompt was on the last but one line.

mksession didn't restore the cursor column properly when it was after a tab.
Could not get all windows back when using a smaller terminal screen. Didn't
restore all windows when "winsize" was not in 'sessionoptions'. (Webb)

Command line completion for ":buffer" depended on 'ignorecase' for Unix, but
not for DOS et al. Now don't use 'ignorecase', but let it depend on whether
file names are case sensitive or not (like when expanding file names).

Win32 GUI: (Negri)
- Redrawing the background caused flicker when resizing the window. Removed
  _OnEraseBG(). Removed CS_HREDRAW and CS_VREDRAW flags from the
  sndclass.style.
- Some parts of the window were drawn in grey, instead of using the color from
  the user color scheme.
- Dropping a file on gvim didn't activate the window.
- When there is no menu ('guioptions' excludes 'm'), never use the ALT key for
  it.

GUI: When resizing the window, would make the window height a bit smaller.
Now round off to the nearest char cell size. (Negri)

In Vi the ")" and "(" commands don't stop at a single space after a dot.
Added 'J' flag in 'cpoptions' to make this behave Vi compatible. (Roemer)

When saving a session without any buffers loaded, there would be a ":normal"
command without arguments in it. (Webb)

Memory leaks fixed: (Madsen)
- eval.c: forgot to release func structure when func deleted
- ex_docmd.c: forgot to release string after "<sfile>"
- misc1.c: leak when completion pattern had no matches.
- os_unix.c: forgot to release regexp after file completions

Could crash when using a buffer without a name. (Madsen)
Could crash when doing file name completion, because of backslash_halve().
(Madsen)

":@a" would do mappings on register a, which is not Vi compatible. (Roemer)

":g/foo.*()/s/foobar/_&/gc" worked fine, but then "n" searched for "foobar"
and displayed "/foo.*()". (Roemer)

OS/2: get_cmd_output() was not included. Didn't check for $VIM/.vimrc file.

Command line completion of options didn't work after "+=" and "-=".

Unix configure: Test for memmove()/bcopy()/memcpy() tried redefining these
functions, which could fail if they are defined already. Use mch_memmove() to
redefine.

Unix: ":let a = expand(" xterm &")" started an xterm asynchronously, but
":let a = expand(" xterm& ")" generated an error message, because the

redirection was put after the '&'.

Win32 GUI: Dialog buttons could not be selected properly with cursor keys,
when the default is not the first button. (Webb)

The "File has changed since editing started" (when regaining focus) could not
always be seen. (Webb)

When starting with "ex filename", the file message was overwritten with
the "entering Ex mode" message.

Output of ":tselect" listed name of file directly from the tags file. Now it
is corrected for the position of the tags file.

When 'backspace' is 0, could backspace over autoindent. Now it is no longer
allowed (Vi compatible).

In Replace mode, when 'noexpandtab' and 'smarttab' were set, and inserting
Tabs, backspacing didn't work correctly for Tabs inserted at the start of the
line (unless 'sts' was set too). Also, when replacing the first non-blank
after which is a space, rounding the indent was done on the first non-blank
instead of on the character under the cursor.

When 'sw' at 4, 'ts' at 8 and 'smarttab' set: When a tab was appended after
four spaces (they are replaced with a tab) couldn't backspace over the tab.

In Insert mode, with 'bs' set to 0, couldn't backspace to before autoindent,
even when it was removed with CTRL-D.

When repeating an insert command where a <BS>, <Left> or other key causes an
error, would flush buffers and remain in Insert mode. No longer flush
buffers, only beep and continue with the insert command.

Dos and Win32 console: Setting t_me didn't work to get another color. Made
this works backwards compatible.

For Turkish (LANG = "tr") uppercase 'i' is not an 'I'. Use ASCII uppercase
translation in vim_strup() to avoid language problems. (Komur)

Unix: Use usleep() or nanosleep() for mch_delay() when available. Hopefully
this avoids a hangup in select(0, ..) for Solaris 2.6.

Vim would crash when using a script file with 'let &sp = "| tee"', starting
vim with "vim -u test", then doing ":set sp=". The P_WAS_SET flag wasn't set
for a string option, could cause problems with any string option.

When using "cmd | vim -", stdin is not a terminal. This gave problems with
GPM (Linux console mouse) and when executing external commands. Now close
stdin and re-open it as a copy of stderr.

Syntax highlighting: A "nextgroup" item was not properly stored in the state
list. This caused missing of next groups when not redrawing from start to
end, but starting halfway.

Didn't check for valid values of 'ttymouse'.

When executing an external command from the GUI, waiting for the child to terminate might not work, causing a hang. (Parmelan)

"make uninstall" didn't delete the vimrc_example.vim and gvimrc_example.vim files and the vimtutor.

Win32: "expand("%:p:h")" with no buffer name removed the directory name.
"fnamemodify("", ":p")" did not add a trailing slash, fname_case() removed it.

Fixed: When 'hlsearch' was set and the 'c' flag was not in 'cpoptions': highlighting was not correct. Now overlapping matches are handled correctly.

Athena, Motif and GTK GUI: When started without focus, cursor was shown as if with focus.

Don't include 'shellpipe' when compiled without quickfix, it's not used.
Don't include 'dictionary' option when compiled without the +insert_expand feature.
Only include the 'shelltype' option for the Amiga.

When making a change to a line, with 'hlsearch' on, causing it to wrap, while executing a register, the screen would not be updated correctly. This was a generic problem in update_screenline() being called while must_redraw is VALID.

Using ":bdelete" in a BufUnload autocommand could cause a crash. The window height was added to another window twice in close_window().

Win32 GUI: When removing a menu item, the tearoff wasn't updated. (Negri)

Some performance bottlenecks removed. Allocating memory was not efficient. For Win32 checking for available memory was slow, don't check it every time now. On NT obtaining the user name takes a long time, cache the result (for all systems).

fnamemodify() with an argument ":~:." or ":.:~" didn't work properly.

When editing a new file and exiting, the marks for the buffer were not saved in the viminfo file.

":confirm only" didn't put up a dialog.

These text objects didn't work when 'selection' was "exclusive": va(vi(va{ vi{ va< vi< vi[va[.

The dialog for writing a readonly file didn't have a valid default. (Negri)

The line number used for error messages when sourcing a file was reset when modelines were inspected. It was wrong when executing a function.

The file name and line number for an error message wasn't displayed when it was the same as for the last error, even when this was long ago. Now reset

the name/lnum after a hit-enter prompt.

In a session file, a "%" in a file name caused trouble, because fprintf() was used to write it to the file.

When skipping statements, a mark in an address wasn't skipped correctly: "ka|if 0 'ad else echo endif". (Roemer)

":wall" could overwrite a not-edited file without asking.

GUI: When $DISPLAY was not set or starting the GUI failed in another way, the console mode then started with wrong colors and skipped initializations. Now do an early check if the GUI can be started. Don't source the menu.vim or gvimrc when it will not. Also do normal terminal initializations if the GUI might not start.

When using a BufEnter autocommand to position the cursor and scroll the window, the cursor was always put at the last used line and halfway the window anyhow.

When 'wildmode' was set to "longest,list:full", ":e *.c<Tab><Tab>" didn't list the matches. Also avoid that listing after a "longest" lists the wrong matches when the first expansion changed the string in front of the cursor.

When using ":insert", ":append" or ":change" inside a while loop, was not able to break out of it with a CTRL-C.

Win32: ":e ." took an awful long time before an error message when used in "C:\". Was caused by adding another backslash and then trying to get the full name for "C:\\".

":winpos -10 100" was working like ":winpos -10 -10", because a pointer was not advanced past the '-' sign.

When obtaining the value of a hidden option, would give an error message. Now just use a zero value.

OS/2: Was using argv[0], even though it was not a useful name. It could be just "vim", found in the search path.

Xterm: ":set columns=78" didn't redraw properly (when lines wrap/unwrap) until after a delay of 'updatetime'. Didn't check for the size-changed signal.

'scrollbind' didn't work in Insert mode.
Horizontal scrollbinding didn't always work for "0" and "$" commands (e.g., when 'showcmd' was off).

When compiled with minimal features but with GUI, switching on the mouse in an xterm caused garbage, because the mouse codes were not recognized. Don't enable the mouse when it can't be recognized. In the GUI it also didn't work, the arguments to the mouse code were not interpreted.

When 'showbreak' used, in Insert mode, when the cursor is just after the last character in the line, which is also the in the rightmost column, the cursor

position would be like the 'showbreak' string is shown, but it wasn't.

Autocommands could move the cursor in a new file, so that CTRL-W i didn't show the right line. Same for when using a filemark to jump to another file.

When redefining the argument list, the title used for other windows could be showing the wrong info about the position in the argument list. Also update this for a ":split" command without arguments.

When editing file 97 of 13, ":Next" didn't work. Now it goes to the last file in the argument list.

Insert mode completion (for dictionaries or included files) could not be interrupted by typing an <Esc>. Could get hit-enter prompt after line completion, or whenever the informative message would get too long.

When using the ":edit" command to re-edit the same file, an autocommand to jump to the last cursor position caused the cursor to move. Now set the last used cursor position to avoid this.

When 'comments' has a part that starts with white space, formatting the comment didn't work.

At the ":tselect" prompt Normal mode mappings were used. That has been disabled.

When 'selection' is not "old", some commands still didn't allow the cursor past the end-of-line in Visual mode.

Athena: When a menu was deleted, it would appear again (but not functional) when adding another menu. Now they don't reappear anymore (although they are not really deleted either).

Borland C++ 4.x had an optimizer problem in fill_breakat_flags(). (Negri)

"ze" didn't work when 'number' was on. (Davis)

Win32 GUI: Intellimouse code didn't work properly on Windows 98. (Robinson)

A few files were including proto.h a second time, after vim.h had already done that, which could cause problems with the vim_realloc() macro.

Win32 console: <M-x> or ALT-x was not recognized. Also keypad '+', '-' and '*'. (Negri)
MS-DOS: <M-x> didn't work, produced a two-byte code. Now the alphabetic and number keys work. (Negri)

When finding a lot of matches for a tag completion, the check for avoiding double matches could take a lot of time. Add a line_breakcheck() to be able to interrupt this. (Deshpande)

When the command line was getting longer than the screen, the more-prompt would be given regularly, and the cursor position would be wrong. Now only show the part of the command line that fits on the screen and force the cursor

to be positioned on the visible part. There can be text after the cursor which isn't editable.

At the more prompt and with the console dialog, a cursor key was interpreted as <Esc> and OA. Now recognize special keys in get_keystroke(). Ignore mouse and scrollbar events.

When typing a BS after inserting a middle comment leader, typing the last char of the end comment leader still changed it into the end comment leader. (Webb)

When a file system is full, writing to a swap file failed. Now first try to write one block to the file. Try next entry in 'dir' if it fails.

When "~" is in 'whichwrap', doing "~" on last char of a line didn't update the display.

Unix: Expanding wildcards for ":file {\\}" didn't work, because "\}" was translated to "}" before the shell got it. Now don't remove backslashes when wildcards are going to be expanded.

Unix: ":e /tmp/$uid" didn't work. When expanding environment variables in a file name doesn't work, use the shell to expand the file name. ":e /tmp/$tty" still doesn't work though.

"make test" didn't always work on DOS/Windows for test30, because it depended on the external "echo" command.

The link.sh script used "make" instead of $MAKE from the Makefile. Caused problems for generating pathdef.c when "make" doesn't work properly.

On versions that can do console and GUI: In the console a typed CSI code could cause trouble.

The patterns in expression evaluation didn't ignore the 'l' flag in 'cpoptions'. This broke the working of <CR> in the options window.

When 'hls' off and 'ai' on, "O<Esc>" did remove the indent, but it was still highlighted red for trailing space.

Win32 GUI: Dropping an encrypted file on a running gvim didn't work right. Vim would loop while outputting "*" characters. vgetc() was called recursively, thus it returns NUL. Added safe_vgetc(), which reads input directly from the user in this situation.

While reading text from stdin, only an empty screen was shown. Now show that Vim is reading from stdin.

The cursor shape wasn't set properly when returning to Insert mode, after using a CTRL-O /asdf command which fails. It would be OK after a few seconds. Now it's OK right away.

The 'isfname' default for DOS/Windows didn't include the '@' character. File names that contained "dir\@file" could not be edited.

Win32 console: <C-S-Left> could cause a crash when compiled with Borland or egcs. (Aaron)

Unix and VMS: "#if HAVE_DIRENT_H" caused problems for some compilers. Use "#ifdef HAVE_DIRENT_H" instead. (Jones)

When a matching tag is in the current file but has a search pattern that doesn't match, the cursor would jump to the first line.

Unix: Dependencies for pty.c were not included in Makefile. Dependency of ctags/config.h was not included (only matters for parallel make).

Removed a few Uninitialized Memory Reads (potential crashes). In do_call() calling clear_var() when not evaluating. In win32_expandpath() and dos_expandpath() calling backslash_halve() past the end of a file name.

Removed memory leaks: Set_vim_var_string() never freed the value. The next_list for a syntax keyword was never freed.

On non-Unix systems, using a file name with wildcards without a match would silently fail. E.g., ":e *.sh". Now give a "No match" error message.

The life/life.mac, urm/urm.mac and hanoi/hanoi.mac files were not recognized as Vim scripts. Renamed them to *.vim.

[备注：有些编号的补丁在从 5.3 版本升级时没有意义，它们被删除了]

Patch 5.4m.1
问题: When editing a file with a long name, would get the hit-enter prompt, even though all settings are such that the name should be truncated to avoid that. filemess() was printing the file name without truncating it.
解决方案: Truncate the message in filemess(). Use the same code as for msg_trunc_attr(), which is moved to the new function msg_may_trunc().
相关文件: src/message.c, src/proto/message.pro, src/fileio.c

Patch 5.4m.3
问题: The Motif libraries were not found by configure for Digital Unix.
解决方案: Add "/usr/shlib" to the search path. (Andy Kahn)
相关文件: src/configure.in, src/configure

Patch 5.4m.5
问题: Win32 GUI: When using the Save-As menu entry and selecting an existing file in the file browser, would get a dialog to confirm overwriting twice. (Ed Krall)
解决方案: Removed the dialog from the file browser. It would be nicer to set the "forceit" flag and skip Vim's ":confirm" dialog, but it requires quite a few changes to do that.
相关文件: src/gui_w32.c

Patch 5.4m.6
问题: Win32 GUI: When reading text from stdin, e.g., "cat foo | gvim -", a message box would pop up with "-stdin-" (when exiting). (Michael

Schaap)
解决方案: Don't switch off termcap mode for versions that are GUI-only.
They use another terminal to read from stdin.
相关文件: src/main.c, src/fileio.c

Patch 5.4m.7
问题: Unix: running configure with --enable-gtk-check,
--enable-motif-check, --enable-athena-check or --enable-gtktest
had the reverse effect. (Thomas Koehler)
解决方案: Use $enable_gtk_check variable correctly in AC_ARG_ENABLE().
相关文件: src/configure.in, src/configure

Patch 5.4m.9
问题: Multi-byte: With wrapping lines, the cursor was sometimes 2
characters to the left. Syntax highlighting was wrong when a
double-byte character was split for a wrapping line. When
'showbreak' was on the splitting also didn't work.
解决方案: Adjust getvcol() and win_line(). (Chong-Dae Park)
相关文件: src/charset.c, src/screen.c

Patch 5.4m.11
问题: The ":call" command didn't check for illegal trailing characters.
(Stefan Roemer)
解决方案: Add the check in do_call().
相关文件: src/eval.c

Patch 5.4m.13
问题: With the ":s" command:
1. When performing a substitute command, the mouse would be
disabled and enabled for every substitution.
2. The cursor position could be beyond the end of the line.
Calling line_breakcheck() could try to position the cursor,
which causes a crash in the Win32 GUI.
3. When using ":s" in a ":g" command, the cursor was not put on
the first non-white in the line.
4. There was a hit-enter prompt when confirming the substitution
and the replacement was a bit longer.
解决方案: 1. Only disable/enable the mouse when asking for confirmation.
2. Always put the cursor on the first character, it is going to be
moved to the first non-blank anyway.
Don't use the cursor position in gui_mch_draw_hollow_cursor(),
get the character from the screen buffer.
3. Added global_need_beginline flag to call beginline() after ":g"
has finished all substitutions.
4. Clear the need_wait_return flag after prompting the user.
相关文件: src/ex_cmds.c, src/gui_w32.c

Patch 5.4m.14
问题: When doing "vim xxx", ":opt", ":only" and then ":e xxx" we end
up with two swapfiles for "xxx". That is caused by the ":bdel"
command which is executed when unloading the option-window.
Also, there was no check if closing a buffer made the new one
invalid, this could cause a crash.
解决方案: When closing a buffer causes the current buffer to be deleted,

use the new buffer to replace it. Also detect that the new buffer has become invalid as a side effect of closing the current one. Make autocommand that calls ":bdel" in optwin.vim nested, so that the buffer loading it triggers also executes autocommands. Also added a test for this in test13.

相关文件: runtime/optwin.vim, src/buffer.c, src/ex_cmds.c, src/globals.h src/testdir/test13.in, src/testdir/test13.ok

Patch 5.4m.15

问题: When using a BufEnter autocommand to reload the syntax file, conversion to HTML caused a crash. (Sung-Hyun Nam)

解决方案: When using ":syntax clear" the current stack of syntax items was not cleared. This will cause memory to be used that has already been freed. Added call to invalidate_current_state() in syntax_clear().

相关文件: src/syntax.c

Patch 5.4m.17

问题: When omitting a ')' in an expression it would not be seen as a failure.
When detecting an error inside (), there would be an error message for a missing ')' too.
When using ":echo 1+|echo 2" there was no error message. (Roemer)
When using ":exe 1+" there was no error message.
When using ":return 1+" there was no error message.

解决方案: Fix do_echo(), do_execute() and do_return() to give an error message when eval1() returns FAIL.
Fix eval6() to handle trailing ')' correctly and return FAIL when it's missing.

相关文件: src/eval.c

Patch 5.4m.18

问题: When using input() from inside an expression entered with "CTRL-R =" on the command line, there could be a crash. And the resulting command line was wrong.

解决方案: Added getcmdline_prompt(), which handles recursive use of getcmdline() correctly. It also sets the command line prompt. Removed cmdline_prompt(). Also use getcmdline_prompt() for getting the crypt key in get_crypt_key().

相关文件: src/proto/ex_getln.pro, src/ex_getln.c, src/eval.c, src/misc2.c

Patch 5.4m.21

问题: When starting up, the screen structures were first allocated at the minimal size, then initializations were done with Rows possibly different from screen_Rows. Caused a crash in rare situations (GTK with XIM and fontset).

解决方案: Call screenalloc() in main() only after calling ui_get_winsize(). Also avoids a potential delay because of calling screenclear() while "starting" is non-zero.

相关文件: src/main.c

Patch 5.4m.22

问题: In the GUI it was possible that the screen was resized and the screen structures re-allocated while redrawing the screen. This

could cause a crash (hard to reproduce). The call sequence goes
through update_screen() .. syntax_start() .. ui_breakcheck() ..
gui_resize_window() .. screenalloc().
解决方案: Set updating_screen while redrawing. If the window is resized
remember the new size and handle it only after redrawing is
finished.
This also fixes that resizing the screen while still redrawing
(slow syntax highlighting) would not work properly.
Also disable display_hint, it was never used.
相关文件: src/globals.h, src/gui.c, src/screen.c, src/proto/gui.pro

Patch 5.4m.23
问题: When using expand("<cword>") when there was no word under the
cursor, would get an error message. Same for <cWORD> and <cfile>.
解决方案: Don't give an error message, return an empty string.
相关文件: src/eval.c

Patch 5.4m.24
问题: ":help \|" didn't find anything. It was translated to "/\\|".
解决方案: Translate "\|" into "\\bar". First check the table for specific
translations before checking for "\x".
相关文件: src/ex_cmds.c

Patch 5.4m.25
问题: Unix: When using command line completion on files that contain
''', '"' or '|' the file name could not be used.
Adding this file name to the Buffers menu caused an error message.
解决方案: Insert a backslash before these three characters.
Adjust Mungename() function to insert a backslash before '|'.
相关文件: src/ex_getln.c, runtime/menu.vim

Patch 5.4m.26
问题: When using a mapping of two function keys, e.g., <F1><F1>, and
only the first char of the second key has been read, the mapping
would not be recognized. Noticed on some Unix systems with xterm.
解决方案: Add 'K' flag to 'cpoptions' to wait for the whole key code, even
when halfway a mapping.
相关文件: src/option.h, src/term.c

Patch 5.4m.27
问题: When making test33 without the lisp feature it hangs. Interrupting
the execution of the script then might cause a crash.
解决方案: In inchar(), after closing a script, don't use buf[] anymore.
closescript() has freed typebuf[] and buf[] might be pointing
inside typebuf[].
Avoid that test33 hangs when the lisp feature is missing.
相关文件: src/getchar.c src/testdir/test33.in

"os2" was missing from the feature list. Useful for has("os2").

BeOS:
- Included patches from Richard Offer for BeOS R4.5.
- menu code didn't work right. Crashed in the Buffers menu. The window title
wasn't set. (Offer)

Patch 5.4n.3
问题: C-indenting was wrong after " } else". The white space was not skipped. Visible when 'cino' has "+10".
解决方案: Skip white space before calling cin_iselse(). (Norbert Zeh)
相关文件: src/misc1.c

Patch 5.4n.4
问题: When the 't' flag in 'cpoptions' is included, after a ":nohlsearch" the search highlighting would not be enabled again after a tag search. (Norbert Zeh)
解决方案: When setting the new search pattern in jumpto_tag(), don't restore no_hlsearch.
相关文件: src/tag.c

Patch 5.4n.5
问题: When using ":normal" from a CursorHold autocommand Vim hangs. The autocommand is executed down from vgetc(). Calling vgetc() recursively to execute the command doesn't work then.
解决方案: Forbid the use of ":normal" when vgetc_busy is set. Give an error message when this happens.
相关文件: src/ex_docmd.c, runtime/doc/autocmd.txt

Patch 5.4n.6
问题: "gv" could reselect a Visual that starts and/or ends past the end of a line. (Robert Webb)
解决方案: Check that the start and end of the Visual area are on a valid character by calling adjust_cursor().
相关文件: src/normal.c

Patch 5.4n.8
问题: When a mark was on a non existing line (e.g., when the .viminfo was edited), jumping to it caused ml_get errors. (Alexey Marinichev).
解决方案: Added check_cursor_lnum() in nv_gomark().
相关文件: src/normal.c

Patch 5.4n.9
问题: ":-2" moved the cursor to a negative line number. (Ralf Schandl)
解决方案: Give an error message for a negative line number.
相关文件: src/ex_docmd.c

Patch 5.4n.10
问题: Win32 GUI: At the hit-enter prompt, it was possible to scroll the text. This erased the prompt and made Vim look like it is in Normal mode, while it is actually still waiting for a <CR>.
解决方案: Disallow scrolling at the hit-enter prompt for systems that use on the fly scrolling.
相关文件: src/message.c

Patch 5.4n.14
问题: Win32 GUI: When using ":winsize 80 46" and the height is more than what fits on the screen, the window size was made smaller than asked for (that's OK) and Vim crashed (that's not OK)>

解决方案: Call check_winsize() from gui_set_winsize() to resize the windows.
相关文件: src/gui.c

Patch 5.4n.16
问题: Win32 GUI: The <F10> key both selected the menu and was handled as
a key hit.
解决方案: Apply 'winaltkeys' to <F10>, like it is used for Alt keys.
相关文件: src/gui_w32.c

Patch 5.4n.17
问题: Local buffer variables were freed when the buffer is unloaded.
That's not logical, since options are not freed. (Ron Aaron)
解决方案: Free local buffer variables only when deleting the buffer.
相关文件: src/buffer.c

Patch 5.4n.19
问题: Doing ":e" (without argument) in an option-window causes trouble.
The mappings for <CR> and <Space> are not removed. When there is
another buffer loaded, the swap file for it gets mixed up.
(Steve Mueller)
解决方案: Also remove the mappings at the BufUnload event, if they are still
present.
When re-editing the same file causes the current buffer to be
deleted, don't try editing it.
Also added a test for this situation.
相关文件: runtime/optwin.vim, src/ex_cmds.c, src/testdir/test13.in,
src/testdir/test13.ok

Patch 5.4n.24
问题: BeOS: configure never enabled the GUI, because $with_x was "no".
Unix prototypes caused problems, because Display and Widget are
undefined.
Freeing fonts on exit caused a crash.
解决方案: Only disable the GUI when $with_x is "no" and $BEOS is not "yes".
Add dummy defines for Display and Widget in proto.h.
Don't free the fonts in gui_exit() for BeOS.
相关文件: src/configure.in, src/configure, src/proto.h, src/gui.c.

The runtime/vim48x48.xpm icon didn't have a transparent background. (Schild)

Some versions of the mingw32/egcs compiler didn't have WINBASEAPI defined.
(Aaron)

VMS:
- mch_setenv() had two arguments instead of three.
- The system vimrc and gvimrc files were called ".vimrc" and ".gvimrc".
  Removed the dot.
- call to RealWaitForChar() had one argument too many. (Campbell)
- WaitForChar() is static, removed the prototype from proto/os_vms.pro.
- Many file accesses failed, because Unix style file names were used.
  Translate file names to VMS style by using vim_fopen().
- Filtering didn't work, because the temporary file name was generated wrong.
- There was an extra newline every 9192 characters when writing a file. Work

  around it by writing line by line. (Campbell)
- os_vms.c contained "# typedef int DESC". Should be "typedef int DESC;".
  Only mattered for generating prototypes.
- Added file name translation to many places. Made easy by defining macros
  mch_access(), mch_fopen(), mch_fstat(), mch_lstat() and mch_stat().
- Set default for 'tagbsearch' to off, because binary tag searching apparently
  doesn't work for VMS.
- make mch_get_host_name() work with /dec and /standard=vaxc. (Campbell)

Patch 5.4o.2
问题: Crash when using "gf" on "file.c://comment here". (Scott Graham)
解决方案: Fix wrong use of pointers in get_file_name_in_path().
相关文件: src/window.c

Patch 5.4o.3
问题: The horizontal scrollbar was not sized correctly when 'number' is
            set and 'wrap' not set.
            Athena: Horizontal scrollbar wasn't updated when the cursor was
            positioned with a mouse click just after dragging.
解决方案: Subtract 8 from the size when 'number' set and 'wrap' not set.
            Reset gui.dragged_sb when a mouse click is received.
相关文件: src/gui.c

Patch 5.4o.4
问题: When running in an xterm and $WINDOWID is set to an illegal value,
            Vim would exit with "Vim: Got X error".
解决方案: When using the display which was opened for the xterm clipboard,
            check if x11_window is valid by trying to obtain the window title.
            Also add a check in setup_xterm_clip(), for when using X calls to
            get the pointer position in an xterm.
相关文件: src/os_unix.c

Patch 5.4o.5
问题: Motif version with Lesstif: When removing the menubar and then
            using a menu shortcut key, Vim would crash. (raf)
解决方案: Disable the menu mnemonics when the menu bar is removed.
相关文件: src/gui_motif.c

Patch 5.4o.9
问题: The DOS install.exe program used the "move" program. That doesn't
            work on Windows NT, where "move" is internal to cmd.exe.
解决方案: Don't use an external program for moving the executables. Use C
            functions to copy the file and delete the original.
相关文件: src/dosinst.c

Motif and Athena obtained the status area height differently from GTK. Moved
status_area_enabled from global.h to gui_x11.c and call
xim_get_status_area_height() to get the status area height.

Patch 5.4p.1
问题: When using auto-select, and the "gv" command is used, would not
            always obtain ownership of the selection. Caused by the Visual
            area still being the same, but ownership taken away by another

program.
解决方案: Reset the clipboard Visual mode to force updating the selection.
相关文件: src/normal.c

Patch 5.4p.2
问题: Motif and Athena with XIM: Typing 3-byte
<multibyte><multibyte><space> doesn't work correctly with Ami XIM.
解决方案: Avoid using key_sym XK_VoidSymbol. (Nam)
相关文件: src/multbyte.c, src/gui_x11.c

Patch 5.4p.4
问题: Win32 GUI: The scrollbar values were reduced for a file with more
than 32767 lines. But this info was kept global for all
scrollbars, causing a mixup between the windows.
Using the down arrow of a scrollbar in a large file didn't work.
Because of round-off errors there is no scroll at all.
解决方案: Give each scrollbar its own scroll_shift field. When the down
arrow is used, scroll several lines.
相关文件: src/gui.h, src/gui_w32.c

Patch 5.4p.5
问题: When changing buffers in a BufDelete autocommand, there could be
ml_line errors and/or a crash. (Schandl) Was caused by deleting
the current buffer.
解决方案: When the buffer to be deleted unexpectedly becomes the current
buffer, don't delete it.
Also added a check for this in test13.
相关文件: src/buffer.c, src/testdir/test13.in, src/testdir/test13.ok

Patch 5.4p.7
问题: Win32 GUI: When using 'mousemodel' set to "popup_setpos" and
clicking the right mouse button outside of the selected area, the
selected area wasn't removed until the popup menu has gone.
(Aaron)
解决方案: Set the cursor and update the display before showing the popup
menu.
相关文件: src/normal.c

Patch 5.4p.8
问题: The generated bugreport didn't contain information about
$VIMRUNTIME and whether runtime files actually exist.
解决方案: Added a few checks to the bugreport script.
相关文件: runtime/bugreport.vim

Patch 5.4p.9
问题: The windows install.exe created a wrong entry in the popup menu.
The "%1" was "". The full directory was included, even when the
executable had been moved elsewhere. (Ott)
解决方案: Double the '%' to get one from printf. Only include the path to
gvim.exe when it wasn't moved and it's not in $PATH.
相关文件: src/dosinst.c

Patch 5.4p.10
问题: Win32: On top of 5.4p.9: The "Edit with Vim" entry sometimes used

a short file name for a directory.
解决方案: Change the "%1" to "%L" in the registry entry.
相关文件: src/dosinst.c

Patch 5.4p.11
问题: Motif, Athena and GTK: When closing the GUI window when there is a changed buffer, there was only an error message and Vim would not exit.
解决方案: Put up a dialog, like for ":confirm qa". Uses the code that was already used for MS-Windows.
相关文件: src/gui.c, src/gui_w32.c

Patch 5.4p.12
问题: Win32: Trying to expand a string that is longer than 256 characters could cause a crash. (Steed)
解决方案: For the buffer in win32_expandpath() don't use a fixed size array, allocate it.
相关文件: src/os_win32.c

MSDOS: Added "-Wall" to Makefile.djg compile flags. Function prototypes for fname_case() and mch_update_cursor() were missing. "fd" was unused in mf_sync(). "puiLocation" was unused in myputch(). "newcmd" unused in mch_call_shell() for DJGPP version.

5.5 版 本 version-5.5

5.5 版本是 5.4 的漏洞修正版。

改动 changed-5.5

DJGPP 版本现在用 "-O2" 编译而不是 "-O4"，以减少可执行文件的大小。

src/STYLE 文件移到 runtime/doc/develop.txt。加入设计目标。

'backspace' 现在成为字符串选项。见 patch 5.4.15。

新增 added-5.5

加入 Exuberant Ctags 3.3 版本。(Darren Hiebert)

runtime/mswin.vim 里，把 CTRL-Q 映射为 CTRL-V，这样任何 CTRL-V 能做的事情 CTRL-Q 都可以做。

vimrc_example.vim 里支持 bzip2 文件的解压。

如果加入补丁，补丁号在 version.c 表格里输入。这样就可以跳过一个补丁而不影响后面。

支持 X11 的鼠标滚轮。见 patch 5.5a.14。

line2byte() 可以用来得到缓冲区的大小。见 patch 5.4.35。

用插入模式下的 CTRL-R CTRL-O r 和 CTRL-R CTRL-P r 命令按本义插入寄存器。见 patch 5.4.48。

MS-Windows 的卸载 (uninstall) 程序。可以删除 "Edit with Vim" 的注册表项目。也进行注册使 "Add/Remove programs" 应用可以找到它。见 patch 5.4.x7。

修正 fixed-5.5

When using vimrc_example.vim: An error message when the cursor is on a line higher than the number of lines in the compressed file. Move the autocommand for jumping to the last known cursor position to after the decompressing autocommands.

":mkexrc" and ":mksession" wrote the current value of 'textmode'. That may mark a file as modified, which causes problems. This is a buffer-specific setting, it should not affect all files.

"vim --version" wrote two empty lines.

Unix: The alarm signal could kill Vim. It is generated by the Perl alarm() function. Ignore SIGALRM.

Win32 GUI: Toolbar still had the yellow bitmap for running a Vim script.

BeOS: "tmo" must be bigtime_t, instead of double. (Seibert)

Patch 5.4.1
问题: Test11 fails when $GZIP is set to "-v". (Matthew Jackson)
解决方案: Set $GZIP to an empty string.
相关文件: src/testdir/test11.in

Patch 5.4.2
问题: Typing <Esc> at the crypt key prompt caused a crash. (Kallingal)
解决方案: Check for a NULL pointer returned from get_crypt_key().
相关文件: src/fileio.c

Patch 5.4.3
问题: Python: Trying to use the name of an unnamed buffer caused a crash. (Daniel Burrows)
解决方案: Check for b_fname being a NULL pointer.
相关文件: src/if_python.c

Patch 5.4.4
问题: Win32: When compiled without toolbar, but the 'T' flag is in 'guioptions', there would be an empty space for the toolbar.
解决方案: Add two #ifdefs where checking for the 'T' flag. (Vince Negri)
相关文件: src/gui.c

Patch 5.4.5

问题:　　Athena GUI: Using the Buffers.Refresh menu entry caused a crash.
Looks like any ":unmenu" command may cause trouble.
解决方案:　Disallow ":unmenu" in the Athena version. Disable the Buffers
menu, because the Refresh item would not work.
相关文件:　src/menu.c, runtime/menu.vim

Patch 5.4.6
问题:　　GTK GUI: Using ":gui" in the .gvimrc file caused an error. Only
happens when the GUI forks.
解决方案:　Don't fork in a recursive call of gui_start().
相关文件:　src/gui.c

Patch 5.4.7
问题:　　Typing 'q' at the more prompt for the ATTENTION message causes the
file loading to be interrupted. (Will Day)
解决方案:　Reset got_int after showing the ATTENTION message.
相关文件:　src/memline.c

Patch 5.4.8
问题:　　Edit some file, ":he", ":opt": options from help window are shown,
but pressing space updates from the other window. (Phillipps)
Also: When there are changes in the option-window, ":q!" gives an
error message.
解决方案:　Before creating the option-window, go to a non-help window.
Use ":bdel!" to delete the buffer.
相关文件:　runtime/optwin.vim

Patch 5.4.9
Just updates version.h. The real patch has been moved to 5.4.x1.
This patch is just to keep the version number correct.

Patch 5.4.10
问题:　　GTK GUI: When $DISPLAY is invalid, "gvim -f" just exits. It
should run in the terminal.
解决方案:　Use gtk_init_check() instead of gtk_init().
相关文件:　src/gui_gtk_x11.c

Patch 5.4.11
问题:　　When using the 'S' flag in 'cpoptions', 'tabstop' is not copied to
the next buffer for some commands, e.g., ":buffer".
解决方案:　When the BCO_NOHELP flag is given to buf_copy_options(), still
copy the options used by do_help() when neither the "from" or "to"
buffer is a help buffer.
相关文件:　src/option.c

Patch 5.4.12
问题:　　When using 'smartindent', there would be no extra indent if the
current line did not have any indent already. (Hanus Adler)
解决方案:　There was a wrongly placed "else", that previously matched with
the "if" that set trunc_line. Removed the "else" and added a
check for trunc_line to be false.
相关文件:　src/misc1.c

Patch 5.4.13

问题:　　New SGI C compilers need another option for optimisation.
解决方案:　Add a check in configure for "-OPT:Olimit". (Chin A Young)
相关文件:　src/configure.in, src/configure

Patch 5.4.14
问题:　　Motif GUI: When the popup menu is present, a tiny window appears
on the desktop for some users.
解决方案:　Set the menu widget ID for a popup menu to 0. (Thomas Koehler)
相关文件:　src/gui_motif.c

Patch 5.4.15
问题:　　Since 'backspace' set to 0 has been made Vi compatible, it is no
longer possible to only allow deleting autoindent.
解决方案:　Make 'backspace' a list of parts, to allow each kind of
backspacing separately.
相关文件:　src/edit.c, src/option.c, src/option.h, src/proto/option.pro,
runtime/doc/option.txt, runtime/doc/insert.txt

Patch 5.4.16
问题:　　Multibyte: Locale zh_TW.Big5 was not checked for in configure.
解决方案:　Add zh_TW.Big5 to configure check. (Chih-Tsun Huang)
相关文件:　src/configure.in, src/configure

Patch 5.4.17
问题:　　GUI: When started from inside gvim with ":!gvim", Vim would not
start. ":!gvim -f" works fine.
解决方案:　After forking, wait a moment in the parent process, to give the
child a chance to set its process group.
相关文件:　src/gui.c

Patch 5.4.18
问题:　　Python: The clear_history() function also exists in a library.
解决方案:　Rename clear_history() to clear_hist().
相关文件:　src/ex_getln.c, src/eval.c, src/proto/ex_getln.pro

Patch 5.4.19
问题:　　In a terminal with 25 lines, there is a more prompt after the
ATTENTION message. When hitting 'q' here the dialog prompt
doesn't appear and file loading is interrupted. (Will Day)
解决方案:　Don't allow quitting the printing of a message for the dialog
prompt. Added the msg_noquit_more flag for this.
相关文件:　src/message.c

Patch 5.4.20
问题:　　GTK: When starting gvim, would send escape sequences to the
terminal to switch the cursor off and on.
解决方案:　Don't call msg_start() if the GUI is expected to start.
相关文件:　src/main.c

Patch 5.4.21
问题:　　Motif: Toplevel menu ordering was wrong when using tear-off items.
解决方案:　Don't add one to the index for a toplevel menu.
相关文件:　src/gui_motif.c

Patch 5.4.22
问题: In Insert mode, <C-Left>, <S-Left>, <C-Right> and <S-Right> didn't update the column used for vertical movement.
解决方案: Set curwin->w_set_curswant for those commands.
相关文件: src/edit.c

Patch 5.4.23
问题: When a Visual selection is lost to another program, and then the same text is Visually selected again, the clipboard ownership wasn't regained.
解决方案: Set clipboard.vmode to NUL to force regaining the clipboard.
相关文件: src/normal.c

Patch 5.4.24
问题: Encryption: When using ":r file" while 'key' has already entered, the 'key' option would be messed up. When writing the file it would be encrypted with an unknown key and lost! (Brad Despres)
解决方案: Don't free cryptkey when it is equal to the 'key' option.
相关文件: src/fileio.c

Patch 5.4.25
问题: When 'cindent' is set, but 'autoindent' isn't, comments are not properly indented when starting a new line. (Mitterand)
解决方案: When there is a comment leader for the new line, but 'autoindent' isn't set, do C-indenting.
相关文件: src/misc1.c

Patch 5.4.26
问题: Multi-byte: a multi-byte character is never recognized in a file name, causing a backslash before it to be removed on Windows.
解决方案: Assume that a leading-byte character is a file name character in vim_isfilec().
相关文件: src/charset.c

Patch 5.4.27
问题: Entries in the PopUp[nvic] menus were added for several modes, but only deleted for the mode they were used for. This resulted in the entry remaining in the PopUp menu.
When removing a PopUp[nvic] menu, the name had been truncated, could result in greying-out the whole PopUp menu.
解决方案: Remove entries for all modes from the PopUp[nvic] menus. Remove the PopUp[nvic] menu entries first, before the name is changed.
相关文件: src/menu.c

Patch 5.4.28
问题: When using a BufWritePre autocommand to change 'fileformat', the new value would not be used for writing the file.
解决方案: Check 'fileformat' after executing the autocommands instead of before.
相关文件: src/fileio.c

Patch 5.4.29
问题: Athena GUI: When removing the 'g' flag from 'guioptions', using a menu can result in a crash.

解决方案: Always grey-out menus for Athena, don't hide them.
相关文件: src/menu.c

Patch 5.4.30
问题: BeOS: Suspending Vim with CTRL-Z didn't work (killed Vim). The first character typed after ":sh" goes to Vim, instead of the started shell.
解决方案: Don't suspend Vim, start a new shell. Kill the async read thread when starting a new shell. It will be restarted later. (Will Day)
相关文件: src/os_unix.c, src/ui.c

Patch 5.4.31
问题: GUI: When 'mousefocus' is set, moving the mouse over where a window boundary was, causes a hit-enter prompt to be finished. (Jeff Walker)
解决方案: Don't use 'mousefocus' at the hit-enter prompt. Also ignore it for the more prompt and a few other situations. When an operator is pending, abort it first.
相关文件: src/gui.c

Patch 5.4.32
问题: Unix: $LDFLAGS was not passed to configure.
解决方案: Pass $LDFLAGS to configure just like $CFLAGS. (Jon Miner)
相关文件: src/Makefile

Patch 5.4.33
问题: Unix: After expanding an environment variable with the shell, the next expansion would also use the shell, even though it is not needed.
解决方案: Reset "recursive" before returning from gen_expand_wildcards().
相关文件: src/misc1.c

Patch 5.4.34 (also see 5.4.x5)
问题: When editing a file, and the file name is relative to a directory above the current directory, the file name was made absolute. (Gregory Margo)
解决方案: Add an argument to shorten_fnames() which indicates if all file names should be shortened, or only absolute names. In main() only use shorten_fnames() to shorten absolute names.
相关文件: src/ex_docmd.c, src/fileio.c, src/main.c, src/proto/fileio.pro

Patch 5.4.35
问题: There is no function to get the current file size.
解决方案: Allow using line2byte() with the number of lines in the file plus one. This returns the offset of the line past the end of the file, which is the file size plus one.
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 5.4.36
问题: Comparing strings while ignoring case didn't work correctly for some machines. (Mide Steed)
解决方案: vim_stricmp() and vim_strnicmp() only returned 0 or 1. Changed them to return -1 when the first argument is smaller.
相关文件: src/misc2.c

Patch 5.4.37 (also see 5.4.40 and 5.4.43)
问题: Long strings from the viminfo file are truncated.
解决方案: When writing a long string to the viminfo file, first write a line with the length, then the string itself in a second line.
相关文件: src/eval.c, src/ex_cmds.c, src/ex_getln.c, src/mark.c, src/ops.c, src/search.c, src/proto/ex_cmds.pro, runtime/syntax/viminfo.vim

Patch 5.4.38
问题: In the option-window, ":set go&" resulted in 'go' being handled like a boolean option.
Mappings for <Space> and <CR> were overruled by the option-window.
解决方案: When the value of an option isn't 0 or 1, don't handle it like a boolean option.
Save and restore mappings for <Space> and <CR> when entering and leaving the option-window.
相关文件: runtime/optwin.vim

Patch 5.4.39
问题: When setting a hidden option, spaces before the equal sign were not skipped and cause an error message. E.g., ":set csprg =cmd".
解决方案: When skipping over a hidden option, check for a following "=val" and skip it too.
相关文件: src/option.c

Patch 5.4.40 (depends on 5.4.37)
问题: Compiler error for "atol(p + 1)". (Axel Kielhorn)
解决方案: Add a typecast: "atol((char *)p + 1)".
相关文件: src/ex_cmds.c

Patch 5.4.41
问题: Some commands that were not included would give an error message, even when after "if 0".
解决方案: Don't give an error message for an unsupported command when not executing the command.
相关文件: src/ex_docmd.c

Patch 5.4.42
问题: ":w" would also cause a truncated message to appear in the message history.
解决方案: Don't put a kept message in the message history when it starts with "<".
相关文件: src/message.c

Patch 5.4.43 (depends on 5.4.37)
问题: Mixing long lines with multiple lines in a register causes errors when writing the viminfo file. (Robinson)
解决方案: When reading the viminfo file to skip register contents, skip lines that start with "<".
相关文件: src/ops.c

Patch 5.4.44
问题: When 'whichwrap' includes '~', a "~" command that goes on to the next line cannot be properly undone. (Zellner)

解决方案: Save each line for undo in n_swapchar().
相关文件: src/normal.c

Patch 5.4.45 (also see 5.4.x8)
问题: When expand("$ASDF") fails, there is an error message.
解决方案: Remove the global expand_interactively. Pass a flag down to skip the error message.
Also: expand("$ASDF") returns an empty string if $ASDF isn't set. Previously it returned "$ASDF" when 'shell' is "sh".
Also: system() doesn't print an error when the command returns an error code.
相关文件: many

Patch 5.4.46
问题: Backspacing did not always use 'softtabstop' after hitting <CR>, inserting a register, moving the cursor, etc.
解决方案: Reset inserted_space much more often in edit().
相关文件: src/edit.c

Patch 5.4.47
问题: When executing BufWritePre or BufWritePost autocommands for a hidden buffer, the cursor could be moved to a non-existing position. (Vince Negri)
解决方案: Save and restore the cursor and topline for the current window when it is going to be used to execute autocommands for a hidden buffer. Use an existing window for the buffer when it's not hidden.
相关文件: src/fileio.c

Patch 5.4.48
问题: A paste with the mouse in Insert mode was not repeated exactly the same with ".". For example, when 'autoindent' is set and pasting text with leading indent. (Perry)
解决方案: Add the CTRL-R CTRL-O r and CTRL-R CTRL-P r commands in Insert mode, which insert the contents of a register literally.
相关文件: src/edit.c, src/normal.c, runtime/doc/insert.txt

Patch 5.4.49
问题: When pasting text with [<MiddleMouse>, the cursor could end up after the last character of the line.
解决方案: Correct the cursor position for the change in indent.
相关文件: src/ops.c

Patch 5.4.x1 (note: Replaces patch 5.4.9)
问题: Win32 GUI: menu hints were never used, because WANT_MENU is not defined until vim.h is included.
解决方案: Move the #ifdef WANT_MENU from where MENUHINTS is defined to where it is used.
相关文件: src/gui_w32.c

Patch 5.4.x2
问题: BeOS: When pasting text, one character was moved to the end.
解决方案: Re-enable the BeOS code in fill_input_buf(), and fix timing out with acquire_sem_etc(). (Will Day)

相关文件: src/os_beos.c, src/ui.c

Patch 5.4.x3
问题: Win32 GUI: When dropping a directory on a running gvim it crashes.
解决方案: Avoid using a NULL file name. Also display a message to indicate
that the current directory was changed.
相关文件: src/gui_w32.c

Patch 5.4.x4
问题: Win32 GUI: Removing an item from the popup menu doesn't work.
解决方案: Don't remove the item from the menubar, but from the parent popup
menu.
相关文件: src/gui_w32.c

Patch 5.4.x5 (addition to 5.4.34)
相关文件: src/gui_w32.c

Patch 5.4.x6
问题: Win32: Expanding (dir)name starting with a dot doesn't work.
(McCormack) Only when there is a path before it.
解决方案: Fix the check, done before expansion, if the file name pattern
starts with a dot.
相关文件: src/os_win32.c

Patch 5.4.x7
问题: Win32 GUI: Removing "Edit with Vim" from registry is difficult.
解决方案: Add uninstall program to remove the registry keys. It is installed
in the "Add/Remove programs" list for ease of use.
Also: don't set $VIM when the executable is with the runtime files.
Also: Add a text file with a step-by-step description of how to
uninstall Vim for DOS and Windows.
相关文件: src/uninstal.c, src/dosinst.c, src/Makefile.w32, uninstal.txt

Patch 5.4.x8 (addition to 5.4.45)
相关文件: many

Patch 5.4.x9
问题: Win32 GUI: After executing an external command, focus is not
always regained (when using focus-follows-mouse).
解决方案: Add SetFocus() in mch_system(). (Mike Steed)
相关文件: src/os_win32.c

Patch 5.5a.1
问题: ":let @* = @:" did not work. The text was not put on the
I clipboard. (Fisher)
解决方案: Own the clipboard and put the text on it.
相关文件: src/ops.c

Patch 5.5a.2
问题: append() did not mark the buffer modified. Marks below the
new line were not adjusted.
解决方案: Fix the f_append() function.
相关文件: src/eval.c

Patch 5.5a.3
问题: Editing compressed ".gz" files doesn't work on non-Unix systems, because there is no "mv" command.
解决方案: Add the rename() function and use it instead of ":!mv". Also: Disable the automatic jump to the last position, because it changes the jumplist.
相关文件: src/eval.c, runtime/doc/eval.txt, runtime/vimrc_example.vim

Patch 5.5a.4
问题: When using whole-line completion in insert mode while the cursor is in the indent, get "out of memory" error. (Stekrt)
解决方案: Don't allocate a negative amount of memory in ins_complete().
相关文件: src/edit.c

Patch 5.5a.5
问题: Win32: The 'path' option can hold only up to 256 characters, because _MAX_PATH is 256. (Robert Webb)
解决方案: Use a fixed path length of 1024.
相关文件: src/os_win32.h

Patch 5.5a.6
问题: Compiling with gcc on Win32, using the Unix Makefile, didn't work.
解决方案: Add $(SUFFIX) to all places where an executable is used. Also pass it to ctags. (Reynolds)
相关文件: src/Makefile

Patch 5.5a.7
问题: When using "cat | vim -" in an xterm, the xterm version reply would end up in the file.
解决方案: Read the file from stdin before switching the terminal to RAW mode. Should also avoid problems with programs that use a specific terminal setting.
Also: when using the GUI, print "Reading from stdin..." in the GUI window, to give a hint why it doesn't do anything.
相关文件: src/main.c, src/fileio.c

Patch 5.5a.8
问题: On multi-threaded Solaris, suspending doesn't work.
解决方案: Call pause() when the SIGCONT signal was not received after sending the SIGTSTP signal. (Nagano)
相关文件: src/os_unix.c

Patch 5.5a.9
问题: 'winaltkeys' could be set to an empty argument, which is illegal.
解决方案: Give an error message when doing ":set winaltkeys=".
相关文件: src/option.c

Patch 5.5a.10
问题: Win32 console: Using ALTGR on a German keyboard to produce "}" doesn't work, because the 8th bit is set when ALT is pressed.
解决方案: Don't set the 8th bit when ALT and CTRL are used. (Leipert)
相关文件: src/os_win32.c

Patch 5.5a.11
问题:　　Tcl: Configure always uses tclsh8.0.
　　Also: Loading a library doesn't work.
解决方案:　Add "--with-tclsh" configure argument to allow specifying another
　　name for the tcl shell.
　　Call Tcl_Init() in tclinit() to make loading libraries work.
　　(Johannes Zellner)
相关文件:　src/configure.in, src/configure, src/if_tcl.c

Patch 5.5a.12
问题:　　The "user_commands" feature is called "user-commands".
解决方案:　Replace "user-commands" with "user_commands". (Kim Sung-bom)
　　Keep "user-commands" for the has() function, to remain backwards
　　compatible with 5.4.
相关文件:　src/eval.c, src/version.c

Patch 5.5a.13
问题:　　OS/2: When $HOME is not defined, "C:/" is used for the viminfo
　　file. That is very wrong when OS/2 is on another partition.
解决方案:　Use $VIM for the viminfo file when it is defined, like for MSDOS.
　　Also: Makefile.os2 didn't depend on os_unix.h.
相关文件:　src/os_unix.h, src/Makefile.os2

Patch 5.5a.14
问题:　　Athena, Motif and GTK: The Mouse scroll wheel doesn't work.
解决方案:　Interpret a click of the wheel as a key press of the <MouseDown>
　　or <MouseUp> keys. Default behavior is to scroll three lines, or
　　a full page when Shift is used.
相关文件:　src/edit.c, src/ex_getln.c, src/gui.c, src/gui_gtk_x11.c,
　　src/gui_x11.c, src/keymap.h, src/message.c, src/misc1.c,
　　src/misc2.c, src/normal.c, src/proto/normal.pro, src/vim.h,
　　runtime/doc/scroll.txt

Patch 5.5a.15
问题:　　Using CTRL-A in Insert mode doesn't work correctly when the insert
　　started with the <Insert> key. (Andreas Rohrschneider)
解决方案:　Replace <Insert> with "i" before setting up the redo buffer.
相关文件:　src/normal.c

Patch 5.5a.16
问题:　　VMS: GUI does not compile and run.
解决方案:　Various fixes. (Zoltan Arpadffy)
　　Moved functions from os_unix.c to ui.c, so that VMS can use them
　　too: open_app_context(), x11_setup_atoms() and clip_x11* functions.
　　Made xterm_dpy global, it's now used by ui.c and os_unix.c.
　　Use gethostname() always, sys_hostname doesn't exist.
相关文件:　src/globals.h, src/gui_x11.c, src/os_vms.mms, src/os_unix.c,
　　src/os_vms.c, src/ui.c, src/proto/os_unix.pro, src/proto/ui.pro

Renamed AdjustCursorForMultiByteCharacter() to AdjustCursorForMultiByteChar()
to avoid symbol length limit of 31 characters. (Steve P. Wall)

Patch 5.5b.1
问题:　　SASC complains about dead assignments and implicit type casts.

解决方案: Removed the dead assignments. Added explicit type casts.
相关文件: src/buffer.c, src/edit.c, src/eval.c, src/ex_cmds.c,
src/ex_getln.c, src/fileio.c, src/getchar.c, src/memline.c,
src/menu.c, src/misc1.c, src/normal.c, src/ops.c, src/quickfix.c,
src/screen.c

Patch 5.5b.2
问题: When using "CTRL-O O" in Insert mode, hit <Esc> and then "o" in
another line truncates that line. (Devin Weaver)
解决方案: When using a command that starts Insert mode from CTRL-O, reset
"restart_edit" first. This avoids that edit() is called with a
mix of starting a new edit command and restarting a previous one.
相关文件: src/normal.c

5.6 版 本 version-5.6

5.6 版本是 5.5 的漏洞修正版。

改动 changed-5.6

OleVim 文件的小改动。(Christian Schaller)

src/version.c 的补丁号之间加入 "/**/"。提供一行上下文，有些版本的 patch 需要如此。

重新排列 Syntax 菜单，避免子菜单过长。删除字母表项目的键盘快捷键，避免和固定项目发生冲突。

新增 added-5.6

加入 Exuberant Ctags 3.4 版本。(Darren Hiebert)

Python 的 OpenWithVim。(Christian Schaller)

Win32 GUI: gvimext.dll，用于关联菜单 "Edit with Vim" 项目。避免了 MS Office 任务栏上报告的问题。现在用作外壳扩展。(Tianmiao Hu)

新语法文件:

abel	Abel (John Cook)
aml	Arc Macro Language (Nikki Knuit)
apachestyle	Apache-style config file (Christian Hammers)
cf	Cold Fusion (Jeff Lanzarotta)
ctrlh	带有 CTRL-H 序列的文件 (Bram Moolenaar)
cupl	CUPL (John Cook)
cuplsim	CUPL simulation (John Cook)
erlang	Erlang (Kresimir Marzic)
gedcom	Gedcom (Paul Johnson)
icon	Icon (Wendell Turner)
ist	MakeIndex style (Peter Meszaros)

```
jsp             Java Server Pages (Rafael Garcia-Suarez)
rcslog          Rcslog (Joe Karthauser)
remind          Remind (Davide Alberani)
sqr             Structured Query Report Writer (Paul Moore)
tads            TADS (Amir Karger)
texinfo         Texinfo (Sandor Kopanyi)
xpm2            X Pixmap v2 (Steve Wall)
```

用 'cpoptions' 的 'C' 标志位可以关闭源代码行的连接功能。见下 patch 5.5.013。
line-continuation

":syntax" 命令的 "excludenl" 参数。见下 patch 5.5.032。 :syn-excludenl

实现 z+ 和 z^ 命令。见下 patch 5.5.050。

Corel Draw 格式的 Vim 标志 (logo)。可以使用任何精度的比例。

修正 fixed-5.6

```
Using this mapping in Select mode, terminated completion:
":vnoremap <C-N> <Esc>a<C-N>" (Benji Fisher)
Ignore K_SELECT in ins_compl_prep().

VMS (Zoltan Arpadffy, David Elins):
- ioctl() in pty.c caused trouble, #ifndef VMS added.
- Cut & paste mismatch corrected.
- Popup menu line crash corrected.  (Patch 5.5.047)
- Motif directories during open and save as corrected.
- Handle full file names with version numbers. (Patch 5.5.046)
- Directory handling (CD command etc.)
- Corrected file name conversion VMS to Unix and v.v.
- Recovery was not working.
- Terminal and signal handling was outdated compared to os_unix.c.
- Improved os_vms.txt.

Configure used fprintf() instead of printf() to check for __DATE__ and
__TIME__. (John Card II)

BeOS: Adjust computing the char_height and char_ascent.  Round them up
separately, avoids redrawing artifacts. (Mike Steed)

Fix a few multi-byte problems in menu_name_skip(), set_reg_ic(), searchc() and
findmatchlimit(). (Taro Muraoka)

GTK GUI:
- With GTK 1.2.5 and later the scrollbars were not redrawn correctly.
- Adjusted the gtk_form_draw() function.
- SNiFF connection didn't work.
- 'mousefocus' was not working. (Dalecki)
- Some keys were not working with modifiers: Shift-Tab, Ctrl-Space and CTRL-@.
```

Patch 5.5.001
问题: Configure in the top directory did not pass on an argument with a space correctly. For example "./configure --previs="/My home". (Stephane Chazelas)
解决方案: Use '"$@"' instead of '$*' to pass on the arguments.
相关文件: configure

Patch 5.5.002
问题: Compilation error for using "fds[] & POLLIN". (Jeff Walker)
解决方案: Use "fds[].revents & POLLIN".
相关文件: src/os_unix.c

Patch 5.5.003
问题: The autoconf check for sizeof(int) is wrong on machines where sizeof(size_t) != sizeof(int).
解决方案: Use our own configure check. Also fixes the warning for cross-compiling.
相关文件: src/configure.in, src/configure

Patch 5.5.004
问题: On Unix it's not possible to interrupt ":sleep 100".
解决方案: Switch terminal to cooked mode while asleep, to allow a SIGINT to wake us up. But switch off echo, added TMODE_SLEEP.
相关文件: src/term.h, src/os_unix.c

Patch 5.5.005
问题: When using <f-args> with a user command, an empty argument to the command resulted in one empty string, while no string was expected.
解决方案: Catch an empty argument and pass no argument to the function. (Paul Moore)
相关文件: src/ex_docmd.c

Patch 5.5.006
问题: Python: When platform-dependent files are in another directory than the platform-independent files it doesn't work.
解决方案: Also check the executable directory, and add it to CFLAGS. (Tessa Lau)
相关文件: src/configure.in, src/configure

Patch 5.5.007 (extra)
问题: Win32 OLE: Occasional crash when exiting while still being used via OLE.
解决方案: Move OleUninitialize() to before deleting the application object. (Vince Negri)
相关文件: src/if_ole.cpp

Patch 5.5.008
问题: 10000@@ takes a long time and cannot be interrupted.
解决方案: Check for CTRL-C typed while in the loop to push the register.
相关文件: src/normal.c

Patch 5.5.009
问题: Recent Sequent machines don't link with "-linet". (Kurtis Rader)

解决方案: Remove configure check for Sequent.
相关文件: src/configure.in, src/configure

Patch 5.5.010
问题: Ctags freed a memory block twice when exiting. When out of memory, a misleading error message was given.
解决方案: Update to ctags 3.3.2. Also fixes a few other problems. (Darren Hiebert)
相关文件: src/ctags/*

Patch 5.5.011
问题: After "CTRL-V s", the cursor jumps back to the start, while all other operators leave the cursor on the last changed character. (Xiangjiang Ma)
解决方案: Position cursor on last changed character, if possible.
相关文件: src/ops.c

Patch 5.5.012
问题: Using CTRL-] in Visual mode doesn't work when the text includes a space (just where it's useful). (Stefan Bittner)
解决方案: Don't escape special characters in a tag name with a backslash.
相关文件: src/normal.c

Patch 5.5.013
问题: The ":append" and ":insert" commands allow using a leading backslash in a line. The ":source" command concatenates those lines. (Heinlein)
解决方案: Add the 'C' flag in 'cpoptions' to switch off concatenation.
相关文件: src/ex_docmd.c, src/option.h, runtime/doc/options.txt, runtime/filetype.vim, runtime/scripts.vim

Patch 5.5.014
问题: When executing a register with ":@", the ":append" command would get text lines with a ':' prepended. (Heinlein)
解决方案: Remove the ':' characters.
相关文件: src/ex_docmd.c, src/ex_getln.c, src/globals.h

Patch 5.5.015
问题: When using ":g/pat/p", it's hard to see where the output starts, the ":g" command is overwritten. Vi keeps the ":g" command.
解决方案: Keep the ":g" command, but allow overwriting it with the report for the number of changes.
相关文件: src/ex_cmds.c

Patch 5.5.016 (extra)
问题: Win32: Using regedit to install Vim in the popup menu requires the user to confirm this in a dialog.
解决方案: Use "regedit /s" to avoid the dialog
相关文件: src/dosinst.c

Patch 5.5.017
问题: If an error occurs when closing the current window, Vim could get stuck in the error handling.
解决方案: Don't set curwin to NULL when closing the current window.

相关文件: src/window.c

Patch 5.5.018
问题: Absolute paths in shell scripts do not always work.
解决方案: Use /usr/bin/env to find out the path.
相关文件: runtime/doc/vim2html.pl, runtime/tools/efm_filter.pl,
runtime/tools/shtags.pl

Patch 5.5.019
问题: A function call in 'statusline' stops using ":q" twice from
exiting, when the last argument hasn't been edited.
解决方案: Don't decrement quitmore when executing a function. (Madsen)
相关文件: src/ex_docmd.c

Patch 5.5.020
问题: When the output of CTRL-D completion in the commandline goes all
the way to the last column, there is an empty line.
解决方案: Don't add a newline when the cursor wrapped already. (Madsen)
相关文件: src/ex_getln.c

Patch 5.5.021
问题: When checking if a file name in the tags file is relative,
environment variables were not expanded.
解决方案: Expand the file name before checking if it is relative. (Madsen)
相关文件: src/tag.c

Patch 5.5.022
问题: When setting or resetting 'paste' the ruler wasn't updated.
解决方案: Update the status lines when 'ruler' changes because of 'paste'.
相关文件: src/option.c

Patch 5.5.023
问题: When editing a new file and autocommands change the cursor
position, the cursor was moved back to the first non-white, unless
'startofline' was reset.
解决方案: Keep the new column, just like the line number.
相关文件: src/ex_cmds.c

Patch 5.5.024 (extra)
问题: Win32 GUI: When using confirm() to put up a dialog without a
default button, the dialog would not have keyboard focus.
(Krishna)
解决方案: Always set focus to the dialog window. Only set focus to a button
when a default one is specified.
相关文件: src/gui_w32.c

Patch 5.5.025
问题: When using "keepend" in a syntax region, a contained match that
includes the end-of-line could still force that region to
continue, if there is another contained match in between.
解决方案: Check the keepend_level in check_state_ends().
相关文件: src/syntax.c

Patch 5.5.026

问题:　　When starting Vim in a white-on-black xterm, with 'bg' set to "dark", and then starting the GUI with ":gui", setting 'bg' to "light" in the gvimrc, the highlighting isn't set. (Tsjokwing)
解决方案:　Set the highlighting when 'bg' is changed in the gvimrc, even though full_screen isn't set.
相关文件:　src/option.c

Patch 5.5.027
问题:　　Unix: os_unix.c doesn't compile when XTERM_CLIP is used but WANT_TITLE isn't. (Barnum)
解决方案:　Move a few functions that are used by the X11 title and clipboard and put another "#if" around it.
相关文件:　src/os_unix.c

Patch 5.5.028 (extra)
问题:　　Win32 GUI: When a file is dropped on Win32 gvim while at the ":" prompt, the file is edited but the command line is actually still there, the cursor goes back to command line on the next command. (Krishna)
解决方案:　When dropping a file or directory on gvim while at the ":" prompt, insert the name of the file/directory. Allows using the file/directory name for any Ex command.
相关文件:　src/gui_w32.c

Patch 5.5.029
问题:　　"das" at the end of the file didn't delete the last character of the sentence.
解决方案:　When there is no character after the sentence, make the operation inclusive in current_sent().
相关文件:　src/search.c

Patch 5.5.030
问题:　　Unix: in os_unix.c, "term_str" is used, which is also defined in vim.h as a macro. (wuxin)
解决方案:　Renamed "term_str" to "buf" in do_xterm_trace().
相关文件:　src/os_unix.c

Patch 5.5.031 (extra)
问题:　　Win32 GUI: When exiting Windows, gvim will leave swap files behind and will be killed ungracefully. (Krishna)
解决方案:　Catch the WM_QUERYENDSESSION and WM_ENDSESSION messages and try to exit gracefully. Allow the user to cancel the shutdown if there is a changed buffer.
相关文件:　src/gui_w32.c

Patch 5.5.032
问题:　　Patch 5.5.025 wasn't right. And C highlighting was still not working correctly for a #define.
解决方案:　Added "excludenl" argument to ":syntax", to be able not to extend a containing item when there is a match with the end-of-line.
相关文件:　src/syntax.c, runtime/doc/syntax.txt, runtime/syntax/c.vim

Patch 5.5.033
问题:　　When reading from stdin, a long line in viminfo would mess up the

file message. readfile() uses IObuff for keep_msg, which could be overwritten by anyone.
解决方案: Copy the message from IObuff to msg_buf and set keep_msg to that. Also change vim_fgets() to not use IObuff any longer.
相关文件: src/fileio.c

Patch 5.5.034
问题: "gvim -rv" caused a crash. Using 't_Co' before it's set.
解决方案: Don't try to initialize the highlighting before it has been initialized from main().
相关文件: src/syntax.c

Patch 5.5.035
问题: GTK with XIM: Resizing with status area was messy, and ":set guioptions+=b" didn't work.
解决方案: Make status area a separate widget, but not a separate window. (Chi-Deok Hwang)
相关文件: src/gui_gtk_f.c, src/gui_gtk_x11.c, src/multbyte.c

Patch 5.5.036
问题: The GZIP_read() function in $VIMRUNTIME/vimrc_example.vim to uncompress a file did not do detection for 'fileformat'. This is because the filtering is done with 'binary' set.
解决方案: Split the filtering into separate write, filter and read commands.
相关文件: runtime/vimrc_example.vim

Patch 5.5.037
问题: The "U" command didn't mark the buffer as changed. (McCormack)
解决方案: Set the 'modified' flag when using "U".
相关文件: src/undo.c

Patch 5.5.038
问题: When typing a long ":" command, so that the screen scrolls up, causes the hit-enter prompt, even though the user just typed return to execute the command.
解决方案: Reset need_wait_return if (part of) the command was typed in getcmdline().
相关文件: src/ex_getln.c

Patch 5.5.039
问题: When using a custom status line, "%a" (file # of #) reports the index of the current window for all windows.
解决方案: Pass a window pointer to append_arg_number(), and pass the window being updated from build_stl_str_hl(). (Stephen P. Wall)
相关文件: src/buffer.c, src/screen.c, src/proto/buffer.pro

Patch 5.5.040
问题: Multi-byte: When there is some error in xim_real_init(), it can close XIM and return. After this there can be a segv.
解决方案: Test "xic" for being non-NULL, don't set "xim" to NULL. Also try to find more matches for supported styles. (Sung-Hyun Nam)
相关文件: src/multbyte.c

Patch 5.5.041

问题:　　X11 GUI: CTRL-_ requires the SHIFT key only on some machines.
解决方案:　Translate CTRL-- to CTRL-_. (Robert Webb)
相关文件:　src/gui_x11.c

Patch 5.5.042
问题:　　X11 GUI: keys with ALT were assumed to be used for the menu, even
when the menu has been disabled by removing 'm' from 'guioptions'.
解决方案:　Ignore keys with ALT only when gui.menu_is_active is set. (Raf)
相关文件:　src/gui_x11.c

Patch 5.5.043
问题:　　GTK: Handling of fontset fonts was not right when 'guifontset'
contains exactly 14 times '-'.
解决方案:　Avoid setting fonts when working with a fontset. (Sung-Hyun Nam)
相关文件:　src/gui_gtk_x11.c

Patch 5.5.044
问题:　　pltags.pl contains an absolute path "/usr/local/bin/perl". That
might not work everywhere.
解决方案:　Use "/usr/bin/env perl" instead.
相关文件:　runtime/tools/pltags.pl

Patch 5.5.045
问题:　　Using "this_session" variable does not work, requires preceding it
with "v:". Default filename for ":mksession" isn't mentioned
in the docs. (Fisher)
解决方案:　Support using "this_session" to be backwards compatible.
相关文件:　src/eval.c, runtime/doc/options.txt

Patch 5.5.046 (extra)
问题:　　VMS: problems with path and filename.
解决方案:　Truncate file name at last ';', etc. (Zoltan Arpadffy)
相关文件:　src/buffer.c, src/fileio.c, src/gui_motif.c, src/os_vms.c,
src/proto/os_vms.pro

Patch 5.5.047
问题:　　VMS: Crash when using the popup menu
解决方案:　Turn the #define MENU_MODE_CHARS into an array. (Arpadffy)
相关文件:　src/structs.h, src/menu.c

Patch 5.5.048
问题:　　HP-UX 11: Compiling doesn't work, because both string.h and
strings.h are included. (Squassabia)
解决方案:　The configure test for including both string.h and strings.h
must include <Xm/Xm.h> first, because it causes problems.
相关文件:　src/configure.in, src/configure, src/config.h.in

Patch 5.5.049
问题:　　Unix: When installing Vim, the protection bits of files might be
influenced by the umask.
解决方案:　Add $(FILEMOD) to Makefile. (Shetye)
相关文件:　src/Makefile

Patch 5.5.050

问题:　　"z+" and "z^" commands are missing.
解决方案:　Implemented "z+" and "z^".
相关文件:　src/normal.c, runtime/doc/scroll.txt, runtime/doc/index.txt

Patch 5.5.051
问题:　　Several Unix systems have a problem with the optimization limits
　　check in configure.
解决方案:　Removed the configure check, let the user add it manually in
　　Makefile or the environment.
相关文件:　src/configure.in, src/configure, src/Makefile

Patch 5.5.052
问题:　　Crash when using a cursor key at the ATTENTION prompt. (Alberani)
解决方案:　Ignore special keys at the console dialog. Also ignore characters
　　> 255 for other uses of tolower() and toupper().
相关文件:　src/menu.c, src/message.c, src/misc2.c

Patch 5.5.053
问题:　　Indenting is wrong after a function when 'cino' has "fs". Another
　　problem when 'cino' has "{s".
解决方案:　Put line after closing "}" of a function at the left margin.
　　Apply ind_open_extra in the right way after a '{'.
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 5.5.054
问题:　　Unix: ":e #" doesn't work if the alternate file name contains a
　　space or backslash. (Hudacek)
解决方案:　When replacing "#", "%" or other items that stand for a file name,
　　prepend a backslash before special characters.
相关文件:　src/ex_docmd.c

Patch 5.5.055
问题:　　Using "<C-V>$r-" in blockwise Visual mode replaces one character
　　beyond the end of the line. (Zivkov)
解决方案:　Only replace existing characters.
相关文件:　src/ops.c

Patch 5.5.056
问题:　　After "z20<CR>" messages were printed at the old command line
　　position once. (Veselinovic)
解决方案:　Set msg_row and msg_col when changing cmdline_row in
　　win_setheight().
相关文件:　src/window.c

Patch 5.5.057
问题:　　After "S<Esc>" it should be possible to restore the line with "U".
　　(Veselinovic)
解决方案:　Don't call u_clearline() in op_delete() when changing only one
　　line.
相关文件:　src/ops.c

Patch 5.5.058
问题:　　Using a long search pattern and then "n" causes the hit-enter
　　prompt. (Krishna)

解决方案: Truncate the echoed pattern, like other messages. Moved code for
           truncating from msg_attr() to msg_strtrunc().
相关文件: src/message.c, src/proto/message.pro, src/search.c

Patch 5.5.059
问题: GTK GUI: When $term is invalid, using "gvim" gives an error
           message, even though $term isn't really used. (Robbins)
解决方案: When the GUI is about to start, skip the error messages for a
           wrong $term.
相关文件: src/term.c

Patch 5.5.060 (extra)
问题: Dos 32 bit: When a directory in 'backupdir' doesn't exist, ":w"
           causes the file to be renamed to "axlqwqhy.ba~". (Matzdorf)
解决方案: The code to work around a LFN bug in Windows 95 doesn't handle a
           non-existing target name correctly. When renaming fails, make
           sure the file has its original name. Also do this for the Win32
           version, although it's unlikely that it runs into this problem.
相关文件: src/os_msdos.c, src/os_win32.c

Patch 5.5.061
问题: When using "\:" in a modeline, the backslash is included in the
           option value. (Mohsin)
解决方案: Remove one backslash before the ':' in a modeline.
相关文件: src/buffer.c, runtime/doc/options.txt

Patch 5.5.062 (extra)
问题: Win32 console: Temp files are created in the root of the current
           drive, which may be read-only. (Peterson)
解决方案: Use the same mechanism of the GUI version: Use $TMP, $TEMP or the
           current directory. Cleaned up vim_tempname() a bit.
相关文件: src/fileio.c, src/os_win32.h, runtime/doc/os_dos.txt

Patch 5.5.063
问题: When using whole-line completion in Insert mode, 'cindent' is
           applied, even after changing the indent of the line.
解决方案: Don't reindent the completed line after inserting/removing indent.
           (Robert Webb)
相关文件: src/edit.c

Patch 5.5.064
问题: has("sniff") doesn't work correctly.
解决方案: Return 1 when Vim was compiled with the +sniff feature. (Pruemmer)
相关文件: src/eval.c

Patch 5.5.065
问题: When dropping a file on Vim, the 'shellslash' option is not
           effective. (Krishna)
解决方案: Fix the slashes in the dropped file names according to
           'shellslash'.
相关文件: src/ex_docmd.c, runtime/doc/options.txt

Patch 5.5.066
问题: For systems with backslash in file name: Setting a file name

option to a value starting with "\\machine" removed a backslash.
解决方案: Keep the double backslash for "\\machine", but do change "\\\\machine" to "\\machine" for backwards compatibility.
相关文件: src/option.c, runtime/doc/options.txt

Patch 5.5.067
问题: With 'hlsearch' set, the pattern "\>" doesn't highlight the first match in a line. (Benji Fisher)
解决方案: Fix highlighting an empty match. Also highlight the first character in an empty line for "$".
相关文件: src/screen.c

Patch 5.5.068
问题: Crash when a ":while" is used with an argument that has an error. (Sylvain Viart)
解决方案: Was using an uninitialized index in the cs_line[] array. The crash only happened when the index was far off. Made sure the uninitialized index isn't used.
相关文件: src/ex_docmd.c

Patch 5.5.069
问题: Shifting lines in blockwise Visual mode didn't set the 'modified' flag.
解决方案: Do set the 'modified' flag.
相关文件: src/ops.c

Patch 5.5.070
问题: When editing a new file, creating that file outside of Vim, then editing it again, ":w" still warns for overwriting an existing file. (Nam)
解决方案: The BF_NEW flag in the "b_flags" field wasn't cleared properly.
相关文件: src/buffer.c, src/fileio.c

Patch 5.5.071
问题: Using a matchgroup in a ":syn region", which is the same syntax group as the region, didn't stop a contained item from matching in the start pattern.
解决方案: Also push an item on the stack when the syntax ID of the matchgroup is the same as the syntax ID of the region.
相关文件: src/syntax.c

Patch 5.5.072 (extra)
问题: Dos 32 bit: When setting 'columns' to a too large value, Vim may crash, and the DOS console too.
解决方案: Check that the value of 'columns' isn't larger than the number of columns that the BIOS reports.
相关文件: src/os_msdos.c, src/proto/os_msdos.pro, src/option.c

Patch 5.5.073 (extra)
问题: Win 32 GUI: The Find and Find/Replace dialogs didn't show the "match case" checkbox. The Find/Replace dialog didn't handle the "match whole word" checkbox.
解决方案: Support the "match case" and "match whole word" checkboxes.
相关文件: src/gui_w32.c

Patch 5.6a.001
问题: Using <C-End> with a count doesn't work like it does with "G".
(Benji Fisher)
解决方案: Accept a count for <C-End> and <C-Home>.
相关文件: src/normal.c

Patch 5.6a.002
问题: The script for conversion to HTML was an older version.
解决方案: Add support for running 2html.vim on a color terminal.
相关文件: runtime/syntax/2html.vim

Patch 5.6a.003
问题: Defining a function inside a function didn't give an error
message. A missing ":endfunction" doesn't give an error message.
解决方案: Allow defining a function inside a function.
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 5.6a.004
问题: A missing ":endwhile" or ":endif" doesn't give an error message.
(Johannes Zellner)
解决方案: Check for missing ":endwhile" and ":endif" in sourced files.
Add missing ":endif" in file selection macros.
相关文件: src/ex_docmd.c, runtime/macros/file_select.vim

Patch 5.6a.005
问题: 'hlsearch' was not listed alphabetically. The value of 'toolbar'
was changed when 'compatible' is set.
解决方案: Moved entry of 'hlsearch' in options[] table down.
Don't reset 'toolbar' option to the default value when
'compatible' is set.
相关文件: src/option.c

Patch 5.6a.006
问题: Using a backwards range inside ":if 0" gave an error message.
解决方案: Don't complain about a range when it is not going to be used.
(Stefan Roemer)
相关文件: src/ex_docmd.c

Patch 5.6a.007
问题: ":let" didn't show internal Vim variables. (Ron Aaron)
解决方案: Do show ":v" variables for ":let" and ":let v:name".
相关文件: src/eval.c

Patch 5.6a.008
问题: Selecting a syntax from the Syntax menu gives an error message.
解决方案: Replace "else if" in SetSyn() with "elseif". (Ronald Schild)
相关文件: runtime/menu.vim

Patch 5.6a.009
问题: When compiling with +extra_search but without +syntax, there is a
compilation error in screen.c. (Axel Kielhorn)
解决方案: Adjust the #ifdef for declaring and initializing "line" in
win_line(). Also solve compilation problem when +statusline is

used without +eval. Another one when +cmdline_compl is used
without +eval.
相关文件: src/screen.c, src/misc2.c

Patch 5.6a.010
问题: In a function, ":startinsert!" does not append to the end of the
line if a ":normal" command was used to move the cursor. (Fisher)
解决方案: Reset "w_set_curswant" to avoid that w_curswant is changed again.
相关文件: src/ex_docmd.c

Patch 5.6a.011 (depends on 5.6a.004)
问题: A missing ":endif" or ":endwhile" in a function doesn't give an
error message.
解决方案: Give that error message.
相关文件: src/ex_docmd.c

Patch 5.6a.012 (depends on 5.6a.008)
问题: Some Syntax menu entries caused a hit-enter prompt.
解决方案: Call a function to make the command shorter. Also rename a few
functions to avoid name clashes.
相关文件: runtime/menu.vim

Patch 5.6a.013
问题: Command line completion works different when another completion
was done earlier. (Johannes Zellner)
解决方案: Reset wim_index when starting a new completion.
相关文件: src/ex_getln.c

Patch 5.6a.014
问题: Various warning messages when compiling and running lint with
different combinations of features.
解决方案: Fix the warning messages.
相关文件: src/eval.c, src/ex_cmds.c, src/ex_docmd.c, src/gui_gtk_x11.c,
src/option.c, src/screen.c, src/search.c, src/syntax.c,
src/feature.h, src/globals.h

Patch 5.6a.015
问题: The vimtutor command doesn't always know the value of $VIMRUNTIME.
解决方案: Let Vim expand $VIMRUNTIME, instead of the shell.
相关文件: src/vimtutor

Patch 5.6a.016 (extra)
问题: Mac: Window size is restricted when starting. Cannot drag the
window all over the desktop.
解决方案: Get real screen size instead of assuming 640x400. Do not use a
fixed number for the drag limits. (Axel Kielhorn)
相关文件: src/gui_mac.c

Patch 5.6a.017
问题: The "Paste" entry in popup menu for Visual, Insert and Cmdline
mode is in the wrong position. (Stol)
解决方案: Add priority numbers for all Paste menu entries.
相关文件: runtime/menu.vim

Patch 5.6a.018
问题: GTK GUI: submenu priority doesn't work.
Help dialog could be destroyed too soon.
When closing a dialog window (e.g. the "ATTENTION" one), Vim would just hang.
When GTK theme is changed, Vim doesn't adjust to the new colors.
Argument for ":promptfind" isn't used.
解决方案: Fixed the mentioned problems.
Made the dialogs look&feel nicer.
Moved functions to avoid the need for a forward declaration.
Fixed reentrancy of the file browser dialog.
Added drag&drop support for GNOME.
Init the text for the Find/replace dialog from the last used search string. Set "match whole word" toggle button correctly.
Made repeat rate for drag outside of window depend on the distance from the window. (Marcin Dalecki)
Made the drag in Visual mode actually work.
Removed recursiveness protection from gui_mch_get_rgb(), it might cause more trouble than it solves.
相关文件: src/ex_docmd.c, src/gui_gtk.c, src/gui_gtk_x11.c, src/ui.c, src/proto/ui.pro, src/misc2.c

Patch 5.6a.019
问题: When trying to recover through NFS, which uses a large block size, Vim might think the swap file is empty, because mf_blocknr_max is zero. (Scott McDermott)
解决方案: When computing the number of blocks of the file in mf_open(), round up instead of down.
相关文件: src/memfile.c

Patch 5.6a.020
问题: GUI GTK: Could not set display for gvim.
解决方案: Add "-display" and "--display" arguments. (Marcin Dalecki)
相关文件: src/gui_gtk_x11.c

Patch 5.6a.021
问题: Recovering still may not work when the block size of the device where the swap file is located is larger than 4096.
解决方案: Read block 0 with the minimal block size.
相关文件: src/memline.c, src/memfile.c, src/vim.h

Patch 5.6a.022 (extra)
问题: Win32 GUI: When an error in the vimrc causes a dialog to pop up (e.g., for an existing swap file), Vim crashes. (David Elins)
解决方案: Before showing a dialog, open the main window.
相关文件: src/gui_w32.c

Patch 5.6a.023
问题: Using expand("%:gs??/?") causes a crash. (Ron Aaron)
解决方案: Check for running into the end of the string in do_string_sub().
相关文件: src/eval.c

Patch 5.6a.024
问题: Using an autocommand to delete a buffer when leaving it can cause

a crash when jumping to a tag. (Franz Gorkotte)
解决方案: In do_tag(), store tagstacklen before jumping to another buffer.
Check tagstackidx after jumping to another buffer.
Add extra check in win_split() if tagname isn't NULL.
相关文件: src/tag.c, src/window.c

Patch 5.6a.025 (extra)
问题: Win32 GUI: The tables for toupper() and tolower() are initialized too late. (Mike Steed)
解决方案: Move the initialization to win32_init() and call it from main().
相关文件: src/main.c, src/os_w32.c, src/proto/os_w32.pro

Patch 5.6a.026
问题: When the SNiFF connection is open, shell commands hang. (Pruemmer)
解决方案: Skip a second wait() call if waitpid() already detected that the child has exited.
相关文件: src/os_unix.c

Patch 5.6a.027 (extra)
问题: Win32 GUI: The "Edit with Vim" popup menu entry causes problems for the Office toolbar.
解决方案: Use a shell extension dll. (Tianmiao Hu)
Added it to the install and uninstal programs, replaces the old "Edit with Vim" menu registry entries.
相关文件: src/dosinst.c, src/uninstal.c, gvimext/*, runtime/doc/gui_w32.txt

Patch 5.6a.028 (extra)
问题: Win32 GUI: Dialogs and tear-off menus can't handle multi-byte characters.
解决方案: Adjust nCopyAnsiToWideChar() to handle multi-byte characters correctly.
相关文件: src/gui_w32.c

5.7 版 本 version-5.7

5.7 版本是 5.6 的漏洞修正版。

改动 changed-5.7

---

src/INSTALL.mac 更名为 INSTALL_mac.txt，以避免错认文件类型。同时更名 src/INSTALL.amiga 为 INSTALL_ami.txt。

新增 added-5.7

新增语法文件:
stp Stored Procedures (Jeff Lanzarotta)
snnsnet、snnspat、snnsres SNNS (Davide Alberani)
mel MEL (Robert Minsk)
ruby Ruby (Mirko Nasato)
tli TealInfo (Kurt W. Andrews)

```
ora             Oracle config file (Sandor Kopanyi)
abaqus          Abaqus (Carl Osterwisch)
jproperties     Java Properties (Simon Baldwin)
apache          Apache config (Allan Kelly)
csp             CSP (Jan Bredereke)
samba           Samba config (Rafael Garcia-Suarez)
kscript         KDE script (Thomas Capricelli)
hb              Hyper Builder (Alejandro Forero Cuervo)
fortran         Fortran (rewritten) (Ajit J. Thakkar)
sml             SML (Fabrizio Zeno Cornelli)
cvs             CVS commit (Matt Dunford)
aspperl         ASP Perl (Aaron Hope)
bc              BC calculator (Vladimir Scholtz)
latte           Latte (Nick Moffitt)
wml             WML (Gerfried Fuchs)
```

加入 Exuberant ctags 3.5.1。(Darren Hiebert)

语法项目的 "display" 和 "fold" 参数，为了将来扩展用。现在忽略它们。

Macintosh 的 strftime() 函数。

macros/explorer.vim: 文件浏览器脚本 (M A Aziz Ahmed)

修正 fixed-5.7

The 16 bit MS-DOS version is now compiled with Bcc 3.1 instead of 4.0. The executable is smaller.

When a "make test" failed, the output file was lost. Rename it to test99.failed to be able to see what went wrong.

After sourcing bugreport.vim, it's not clear that bugreport.txt has been written in the current directory. Edit bugreport.txt to avoid that.

Adding IME support when using Makefile.w32 didn't work. (Taro Muraoka)

Win32 console: Mouse drags were passed on even when the mouse didn't move.

Perl interface: In Buffers(), type of argument to SvPV() was int, should be STRLEN. (Tony Leneis)

Problem with prototype for index() on AIX 4.3.0. Added check for _AIX43 in os_unix.h. (Jake Hamby)

Mappings in mswin.vim could break when some commands are mapped. Add "nore" to most mappings to avoid re-mapping.

modify_fname() made a copy of a file name for ":p" when it already was a full path name, which is a bit slow.

Win32 with Borland C++ 5.5: Pass the path to the compiler on to xxd and ctags,

to avoid depending on $PATH. Fixed "make clean".

Many fixes to Macintosh specific parts: (mostly by Dany StAmant)
- Only one Help menu.
- No more crash when removing a menu item.
- Support as External Editor for Codewarrior (still some little glitches).
- Popup menu support.
- Fixed crash when pasting after application switch.
- Color from rgb.txt properly displayed.
- 'isprint' default includes all chars above '~'. (Axel Kielhorn)
- mac_expandpath() was leaking memory.
- Add digraphs table. (Axel Kielhorn)
- Multi-byte support: (Kenichi Asai)
  Switch keyscript when going in/out of Insert mode.
  Draw multi-byte character correctly.
  Don't use mblen() but highest bit of char to detect multi-byte char.
  Display value of multi-byte in statusline (also for other systems).
- mouse button was not initialized properly to MOUSE_LEFT when
  USE_CTRLCLICKMENU not defined.
- With Japanese SJIS characters: Make "w", "b", and "e" work
  properly. (Kenichi Asai)
- Replaced old CodeWarrior file os_mac.CW9.hqx with os_mac.cw5.sit.hqx.

Fixes for VMS: (Zoltan Arpadffy) (also see patch 5.6.045 below)
- Added Makefile_vms.mms and vimrc.vms to src/testdir to be able to run the
  tests.
- Various fixes.
- Set 'undolevels' to 1000 by default.
- Made mch_settitle() equivalent to the one in os_unix.c.

RiscOS: A few prototypes for os_riscos.c were outdated. Generate prototypes
automatically.

预发布补丁:

Patch 5.6.001
问题: When using "set bs=0 si cin", Inserting "#<BS>" or "}<BS>" which
reduces the indent doesn't delete the "#" or "}". (Lorton)
解决方案: Adjust ai_col in ins_try_si().
相关文件: src/edit.c

Patch 5.6.002
问题: When using the vim.vim syntax file, a comment with all uppercase
characters causes a hang.
解决方案: Adjust pattern for vimCommentTitle (Charles Campbell)
相关文件: runtime/syntax/vim.vim

Patch 5.6.003
问题: GTK GUI: Loading a user defined toolbar bitmap gives a warning
about the colormap. Probably because the window has not been
opened yet.
解决方案: Use gdk_pixmap_colormap_create_from_xpm() to convert the xpm file.
(Keith Radebaugh)

相关文件: src/gui_gtk.c

Patch 5.6.004 (extra)
问题: Win32 GUI with IME: When setting 'guifont' to "*", the font requester appears twice.
解决方案: In gui_mch_init_font() don't call get_logfont() but copy norm_logfont from fh. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 5.6.005
问题: When 'winminheight' is zero, CTRL-W - with a big number causes a crash. (David Kotchan)
解决方案: Check for negative window height in win_setheight().
相关文件: src/window.c

Patch 5.6.006
问题: GTK GUI: Bold font cannot always be used. Memory is freed too early in gui_mch_init_font().
解决方案: Move call to g_free() to after where sdup is used. (Artem Hodyush)
相关文件: src/gui_gtk_x11.c

Patch 5.6.007 (extra)
问题: Win32 IME: Font is not changed when screen font is changed. And IME composition window does not trace the cursor.
解决方案: Initialize IME font. When cursor is moved, set IME composition window with ImeSetCompositionWindow(). Add call to ImmReleaseContext() in several places. (Taro Muraoka)
相关文件: src/gui.c, src/gui_w32.c, src/proto/gui_w32.pro

Patch 5.6.008 (extra)
问题: Win32: When two files exist with the same name but different case (through NFS or Samba), fixing the file name case could cause the wrong one to be edited.
解决方案: Prefer a perfect match above a match while ignoring case in fname_case(). (Flemming Madsen)
相关文件: src/os_win32.c

Patch 5.6.009 (extra)
问题: Win32 GUI: Garbage in Windows Explorer help line when selecting "Edit with Vim" popup menu entry.
解决方案: Only return the help line when called with the GCS_HELPTEXT flag. (Tianmiao Hu)
相关文件: GvimExt/gvimext.cpp

Patch 5.6.010
问题: A file name which contains a TAB was not read correctly from the viminfo file and the ":ls" listing was not aligned properly.
解决方案: Parse the buffer list lines in the viminfo file from the end backwards. Count a Tab for two characters to align the ":ls" list.
相关文件: src/buffer.c

Patch 5.6.011
问题: When 'columns' is huge (using a tiny font) and 'statusline' is used, Vim can crash.

解决方案: Limit maxlen to MAXPATHL in win_redr_custom(). (John Mullin)
相关文件: src/screen.c

Patch 5.6.012
问题: When using "zsh" for /bin/sh, toolcheck may hang until "exit" is
           typed. (Kuratczyk)
解决方案: Add "-c exit" when checking for the shell version.
相关文件: src/toolcheck

Patch 5.6.013
问题: Multibyte char in tooltip is broken.
解决方案: Consider multibyte char in replace_termcodes(). (Taro Muraoka)
Files: src/term.c

Patch 5.6.014
问题: When cursor is at the end of line and the character under cursor
           is a multibyte character, "yl" doesn't yank 1 multibyte-char.
           (Takuhiro Nishioka)
解决方案: Recognize a multibyte-char at end-of-line correctly in oneright().
           (Taro Muraoka)
           Also: make "+quickfix" in ":version" output appear alphabetically.
相关文件: src/edit.c

Patch 5.6.015
问题: New xterm delete key sends <Esc>[3~ by default.
解决方案: Added <kDel> and <kIns> to make the set of keypad keys complete.
相关文件: src/edit.c, src/ex_getln.c, src/keymap.h, src/misc1.c,
           src/misc2.c, src/normal.c, src/os_unix.c, src/term.c

Patch 5.6.016
问题: When deleting a search string from history from inside a mapping,
           another entry is deleted too. (Benji Fisher)
解决方案: Reset last_maptick when deleting the last entry of the search
           history. Also: Increment maptick when starting a mapping from
           typed characters to avoid a just added search string being
           overwritten or removed from history.
相关文件: src/ex_getln.c, src/getchar.c

Patch 5.6.017
问题: ":s/e/\^M/" should replace an "e" with a CTRL-M, not split the
           line. (Calder)
解决方案: Replace the backslash with a CTRL-V internally. (Stephen P. Wall)
相关文件: src/ex_cmds.c

Patch 5.6.018
问题: ":help [:digit:]" takes a long time to jump to the wrong place.
解决方案: Insert a backslash to avoid the special meaning of '[]'.
相关文件: src/ex_cmds.c

Patch 5.6.019
问题: "snd.c", "snd.java", etc. were recognized as "mail" filetype.
解决方案: Make pattern for mail filetype more strict.
相关文件: runtime/filetype.vim

Patch 5.6.020 (extra)
问题:　　The DJGPP version eats processor time (Walter Briscoe).
解决方案:　Call __dpmi_yield() in the busy-wait loop.
相关文件:　src/os_msdos.c

Patch 5.6.021
问题:　　When 'selection' is "exclusive", a double mouse click in Insert mode doesn't select last char in line. (Lutz)
解决方案:　Allow leaving the cursor on the NUL past the line in this case.
相关文件:　src/edit.c

Patch 5.6.022
问题:　　":e \~<Tab>" expands to ":e ~\$ceelen", which doesn't work.
解决方案:　Re-insert the backslash before the '~'.
相关文件:　src/ex_getln.c

Patch 5.6.023 (extra)
问题:　　Various warnings for the Ming compiler.
解决方案:　Changes to avoid the warnings. (Bill McCarthy)
相关文件:　src/ex_cmds.c, src/gui_w32.c, src/os_w32exe.c, src/os_win32.c, src/syntax.c, src/vim.rc

Patch 5.6.024 (extra)
问题:　　Win32 console: Entering CTRL-_ requires the shift key. (Kotchan)
解决方案:　Specifically catch keycode 0xBD, like the GUI.
相关文件:　src/os_win32.c

Patch 5.6.025
问题:　　GTK GUI: Starting the GUI could be interrupted by a SIGWINCH. (Nils Lohner)
解决方案:　Repeat the read() call to get the gui_in_use value when interrupted by a signal.
相关文件:　src/gui.c

Patch 5.6.026 (extra)
问题:　　Win32 GUI: Toolbar bitmaps are searched for in $VIMRUNTIME/bitmaps, while GTK looks in $VIM/bitmaps. (Keith Radebaugh)
解决方案:　Use $VIM/bitmaps for both, because these are not part of the distribution but defined by the user.
相关文件:　src/gui_w32.c, runtime/doc/gui.txt

Patch 5.6.027
问题:　　TCL: Crash when using a Tcl script (reported for Win32).
解决方案:　Call Tcl_FindExecutable() in main(). (Brent Fulgham)
相关文件:　src/main.c

Patch 5.6.028
问题:　　Xterm patch level 126 sends codes for mouse scroll wheel. Fully works with xterm patch level 131.
解决方案:　Recognize the codes for button 4 (0x60) and button 5 (0x61).
相关文件:　src/term.c

Patch 5.6.029

问题:　　GTK GUI: Shortcut keys cannot be used for a dialog. (Johannes Zellner)
解决方案:　Add support for shortcut keys. (Marcin Dalecki)
相关文件:　src/gui_gtk.c

Patch 5.6.030
问题:　　When closing a window and 'ea' is set, Vim can crash. (Yasuhiro Matsumoto)
解决方案:　Set "curbuf" to a valid value in win_close().
相关文件:　src/window.c

Patch 5.6.031
问题:　　Multi-byte: When a double-byte character ends in CSI, Vim waits for another character to be typed.
解决方案:　Recognize the CSI as the second byte of a character and don't wait for another one. (Yasuhiro Matsumoto)
相关文件:　src/getchar.c

Patch 5.6.032
问题:　　Functions with an argument that is a line number don't all accept ".", "$", etc. (Ralf Arens)
解决方案:　Add get_art_lnum() and use it for setline(), line2byte() and synID().
相关文件:　src/eval.c

Patch 5.6.033
问题:　　Multi-byte: "f " sometimes skips to the second space. (Sung-Hyun Nam)
解决方案:　Change logic in searchc() to skip trailing byte of a double-byte character.
Also: Ask for second byte when searching for double-byte character. (Park Chong-Dae)
相关文件:　src/search.c

Patch 5.6.034 (extra)
问题:　　Compiling with Borland C++ 5.5 fails on tolower() and toupper().
解决方案:　Use TO_LOWER() and TO_UPPER() instead. Also adjust the Makefile to make using bcc 5.5 easier.
相关文件:　src/edit.c, src/ex_docmd.c, src/misc1.c, src/Makefile.bor

Patch 5.6.035
问题:　　Listing the"+comments" feature in the ":version" output depended on the wrong ID. (Stephen P. Wall)
解决方案:　Change "CRYPTV" to "COMMENTS".
相关文件:　src/version.c

Patch 5.6.036
问题:　　GTK GUI: Copy/paste text doesn't work between gvim and Eterm.
解决方案:　Support TEXT and COMPOUND_TEXT selection targets. (ChiDeok Hwang)
相关文件:　src/gui_gtk_x11.c

Patch 5.6.037
问题:　　Multi-byte: Can't use "f" command with multi-byte character in GUI.
解决方案:　Enable XIM in Normal mode for the GUI. (Sung-Hyun Nam)

相关文件: src/gui_gtk_x11.c, src/multbyte.c

Patch 5.6.038
问题: Multi-clicks in GUI are interpreted as a mouse wheel click. When 'ttymouse' is "xterm" a mouse click is interpreted as a mouse wheel click.
解决方案: Don't recognize the mouse wheel in check_termcode() in the GUI. Use 0x43 for a mouse drag in do_xterm_trace(), not 0x63.
相关文件: src/term.c, src/os_unix.c

Patch 5.6.039
问题: Motif GUI under KDE: When trying to logout, Vim hangs up the system. (Hermann Rochholz)
解决方案: When handling the WM_SAVE_YOURSELF event, set the WM_COMMAND property of the window to let the session manager know we finished saving ourselves.
相关文件: src/gui_x11.c

Patch 5.6.040
问题: When using ":s" command, matching the regexp is done twice.
解决方案: After copying the matched line, adjust the pointers instead of finding the match again. (Loic Grenie) Added vim_regnewptr().
相关文件: src/ex_cmds.c, src/regexp.c, src/proto/regexp.pro

Patch 5.6.041
问题: GUI: Athena, Motif and GTK don't give more than 10 dialog buttons.
解决方案: Remove the limit on the number of buttons. Also support the 'v' flag in 'guioptions'. For GTK: Center the buttons.
相关文件: src/gui_athena.c, src/gui_gtk.c, src/gui_motif.c

Patch 5.6.042
问题: When doing "vim -u vimrc" and vimrc contains ":q", the cursor in the terminal can remain off.
解决方案: Call cursor_on() in mch_windexit().
相关文件: src/os_unix.c

Patch 5.6.043 (extra)
问题: Win32 GUI: When selecting guifont with the dialog, 'guifont' doesn't include the bold or italic attributes.
解决方案: Append ":i" and/or ":b" to 'guifont' in gui_mch_init_font().
相关文件: src/gui_w32.c

Patch 5.6.044 (extra)
问题: MS-DOS and Windows: The line that dosinst.exe appends to autoexec.bat to set PATH is wrong when Vim is in a directory with an embedded space.
解决方案: Use double quotes for the value when there is an embedded space.
相关文件: src/dosinst.c

Patch 5.6.045 (extra) (fixed version)
问题: VMS: Various small problems.
解决方案: Many small changes. (Zoltan Arpadffy) File name modifier ":h" keeps the path separator.

File name modifier ":e" also removes version.
Compile with MAX_FEAT by default.
When checking for autocommands ignore version in file name.
Be aware of file names being case insensitive.
Added vt320 builtin termcap.
Be prepared for an empty default_vim_dir.
相关文件: runtime/gvimrc_example.vim, runtime/vimrc_example.vim, runtime/doc/os_vms.txt, src/eval.c, src/feature.h, src/fileio.c, src/gui_motif.c, src/gui_vms_conf.h, src/main.c, src/memline.c, src/misc1.c, src/option.c, src/os_vms_conf.h, src/os_vms.c, src/os_vms.h, src/os_vms.mms, src/tag.c, src/term.c, src/version.c

Patch 5.6.046
问题: Systems with backslash in file name: With 'shellslash' set, "vim */*.c" only uses a slash for the first file name. (Har'El)
解决方案: Fix slashes in file name arguments after reading the vimrc file.
相关文件: src/option.c

Patch 5.6.047
问题: $CPPFLAGS is not passed on to ctags configure.
解决方案: Add it. (Walter Briscoe)
相关文件: src/config.mk.in, src/Makefile

Patch 5.6.048
问题: CTRL-R in Command-line mode is documented to insert text as typed, but inserts text literally.
解决方案: Make CTRL-R insert text as typed, use CTRL-R CTRL-R to insert literally. This is consistent with Insert mode. But characters that end Command-line mode are inserted literally.
相关文件: runtime/doc/index.txt, runtime/doc/cmdline.txt, src/ex_getln.c, src/ops.c, src/proto/ops.pro

Patch 5.6.049
问题: Documentation for [!] after ":ijump" is wrong way around. (Benji Fisher)
解决方案: Fix the documentation. Also improve the code to check for a match after a /* */ comment.
相关文件: runtime/doc/tagsearch.txt, src/search.c

Patch 5.6.050
问题: Replacing is wrong when replacing a single-byte char with double-byte char or the other way around.
解决方案: Shift the text after the character when it is replaced. (Yasuhiro Matsumoto)
相关文件: src/normal.c, src/misc1.c

Patch 5.6.051
问题: ":tprev" and ":tnext" don't give an error message when trying to go before the first or beyond the last tag. (Robert Webb)
解决方案: Added error messages. Also: Delay a second when a file-read message is going to overwrite an error message, otherwise it won't be seen.
相关文件: src/fileio.c, src/tag.c

Patch 5.6.052
问题: Multi-byte: When an Ex command has a '|' or '"' as a second byte, it terminates the command.
解决方案: Skip second byte of multi-byte char when checking for '|' and '"'. (Asai Kenichi)
相关文件: src/ex_docmd.c

Patch 5.6.053
问题: CTRL-] doesn't work on a tag that contains a '|'. (Cesar Crusius)
解决方案: Escape '|', '"' and '\' in tag names when using CTRL-] and also for command-line completion.
相关文件: src/ex_getln.c, src/normal.c

Patch 5.6.054
问题: When using ":e" and ":e #" the cursor is put in the first column when 'startofline' is set. (Cordell)
解决方案: Use the last known column when 'startofline' is set.
Also, use ECMD_LAST more often to simplify the code.
相关文件: src/buffer.c, src/ex_cmds.c, src/ex_docmd.c, src/proto/buffer.pro

Patch 5.6.055
问题: When 'statusline' only contains a text without "%" and doesn't fit in the window, Vim crashes. (Ron Aaron)
解决方案: Don't use the pointer for the first item if there is no item.
相关文件: src/screen.c

Patch 5.6.056 (extra)
问题: MS-DOS: F11 and F12 don't work when 'bioskey' is set.
解决方案: Use enhanced keyboard functions. (Vince Negri)
Detect presence of enhanced keyboard and set bioskey_read and bioskey_ready.
相关文件: src/os_msdos.c

Patch 5.6.057 (extra)
问题: Win32 GUI: Multi-byte characters are wrong in dialogs and tear-off menus.
解决方案: Use system font instead of a fixed font. (Matsumoto, Muraoka)
相关文件: src/gui_w32.c

Patch 5.6.058
问题: When the 'a' flag is not in 'guioptions', non-Windows systems copy Visually selected text to the clipboard/selection on a yank or delete command anyway. On Windows it isn't done even when the 'a' flag is included.
解决方案: Respect the 'a' flag in 'guioptions' on all systems.
相关文件: src/normal.c

Patch 5.6.059 (extra)
问题: When moving the cursor over italic text and the characters spill over to the cell on the right, that spill-over is deleted.
Noticed in the Win32 GUI, can happen on other systems too.
解决方案: Redraw italic text starting from a blank, like this is already done for bold text. (Vince Negri)
相关文件: src/gui.c, src/gui.h, src/gui_w32.c

Patch 5.6.060
问题: Some bold characters spill over to the cell on the left, that spill-over can remain sometimes.
解决方案: Redraw a character when the next character was bold and needs redrawing. (Robert Webb)
相关文件: src/screen.c

Patch 5.6.061
问题: When xterm sends 8-bit controls, recognizing the version response doesn't work.
When using CSI instead of <Esc>[for the termcap color codes, using 16 colors doesn't work. (Neil Bird)
解决方案: Also accept CSI in place of <Esc>[for the version string.
Also check for CSI when handling colors 8-15 in term_color().
Use CSI for builtin xterm termcap entries when 'term' contains "8bit".
相关文件: runtime/doc/term.txt, src/ex_cmds.c, src/option.c, src/term.c, src/os_unix.c, src/proto/option.pro, src/proto/term.pro

Patch 5.6.062
问题: The documentation says that setting 'smartindent' doesn't have an effect when 'cindent' is set, but it does make a difference for lines starting with "#". (Neil Bird)
解决方案: Really ignore 'smartindent' when 'cindent' is set.
相关文件: src/misc1.c, src/ops.c

Patch 5.6.063
问题: Using "I" in Visual-block mode doesn't accept a count. (Johannes Zellner)
解决方案: Pass the count on to do_insert() and edit(). (Allan Kelly)
相关文件: src/normal.c, src/ops.c, src/proto/ops.pro

Patch 5.6.064
问题: MS-DOS and Win32 console: Mouse doesn't work correctly after including patch 5.6.28. (Vince Negri)
解决方案: Don't check for mouse scroll wheel when the mouse code contains the number of clicks.
相关文件: src/term.c

Patch 5.6.065
问题: After moving the cursor around in Insert mode, typing a space can still trigger an abbreviation. (Benji Fisher)
解决方案: Don't check for an abbreviation after moving around in Insert mode.
相关文件: src/edit.c

Patch 5.6.066
问题: Still a few bold character spill-over remains after patch 60.
解决方案: Clear character just in front of blanking out rest of the line. (Robert Webb)
相关文件: src/screen.c

Patch 5.6.067
问题: When a file name contains a NL, the viminfo file is corrupted.

解决方案: Use viminfo_writestring() to convert the NL to CTRL-V n.
           Also fix the Buffers menu and listing a menu name with a newline.
相关文件: runtime/menu.vim, src/buffer.c, src/mark.c, src/menu.c

Patch 5.6.068
问题: Compiling the Perl interface doesn't work with Perl 5.6.0.
           (Bernhard Rosenkraenzer)
解决方案: Also check xs_apiversion for the version number when prepending
           defines for PL_*.
相关文件: src/Makefile

Patch 5.6.069
问题: "go" doesn't always end up at the right character when
           'fileformat' is "dos". (Bruce DeVisser)
解决方案: Correct computations in ml_find_line_or_offset().
相关文件: src/memline.

Patch 5.6.070 (depends on 5.6.068)
问题: Compiling the Perl interface doesn't work with Perl 5.6.0.
           (Bernhard Rosenkraenzer)
解决方案: Simpler check instead of the one from patch 68.
相关文件: src/Makefile

Patch 5.6.071
问题: "A" in Visual block mode on a Tab positions the cursor one char to
           the right. (Michael Haumann)
解决方案: Correct the column computation in op_insert().
相关文件: src/ops.c

Patch 5.6.072
问题: When starting Vim with "vim +startinsert", it enters Insert mode
           only after typing the first command. (Andrew Pimlott)
解决方案: Insert a dummy command in the stuff buffer.
相关文件: src/main.c

Patch 5.6.073 (extra) (depends on 5.6.034)
问题: Win32 GUI: When compiled with Bcc 5.5 menus don't work.
           In dosinst.c toupper() and tolower() give an "internal compiler
           error" for Bcc 5.5.
解决方案: Define WINVER to 4 to avoid compiling for Windows 2000. (Dan
           Sharp) Also cleaned up compilation arguments.
           Use our own implementation of toupper() in dosinst.c. Use
           mytoupper() instead of tolower().
相关文件: src/Makefile.bor, src/dosinst.c

Patch 5.6.074 (extra)
问题: Entering CSI directly doesn't always work, because it's recognized
           as the start of a special key. Mostly a problem with multi-byte
           in the GUI.
解决方案: Use K_CSI for a typed CSI character. Use <CSI> for a normal CSI,
           <xCSI> for a CSI typed in the GUI.
相关文件: runtime/doc/intro.txt, src/getchar.c, src/gui_amiga.c,
           src/gui_gtk_x11.c, src/gui_mac.c, src/gui_riscos.c, src/gui_w32.c,
           src/keymap.h, src/misc2.c

Patch 5.6.075
问题: When using "I" or "A" in Visual block mode while 'sts' is set may change spaces to a Tab the inserted text is not correct. (Mike Steed) And some other problems when using "A" to append after the end of the line.
解决方案: Check for change in spaces/tabs after inserting the text. Append spaces to fill the gap between the end-of-line and the right edge of the block.
相关文件: src/ops.c

Patch 5.6.076
问题: GTK GUI: Mapping <M-Space> doesn't work.
解决方案: Don't use the "Alt" modifier twice in key_press_event().
相关文件: src/gui_gtk_x11.c

Patch 5.6.077
问题: GUI: When interrupting an external program with CTRL-C, gvim might crash. (Benjamin Korvemaker)
解决方案: Avoid using a NULL pointer in ui_inchar_undo().
相关文件: src/ui.c

Patch 5.6.078
问题: Locale doesn't always work on FreeBSD. (David O'Brien)
解决方案: Link with the "xpg4" library when available.
相关文件: src/configure.in, src/configure

Patch 5.6.079
问题: Vim could crash when several Tcl interpreters are created and destroyed.
解决方案: handle the "exit" command and nested ":tcl" commands better. (Ingo Wilken)
相关文件: runtime/doc/if_tcl.txt, src/if_tcl.c

Patch 5.6.080
问题: When jumping to a tag, generating the tags file and jumping to the same tag again uses the old search pattern. (Sung-Hyun Nam)
解决方案: Flush cached tag matches when executing an external command.
相关文件: src/misc2.c, src/proto/tag.pro, src/tag.c

Patch 5.6.081
问题: ":syn include" uses a level for the included file, this confuses contained items included at the same level.
解决方案: Use a unique tag for each included file. Changed sp_syn_inc_lvl to sp_syn_inc_tag. (Scott Bigham)
相关文件: src/syntax.c, src/structs.h

Patch 5.6.082
问题: When using cscope, Vim can crash.
解决方案: Initialize tag_fname in find_tags(). (Anton Blanchard)
相关文件: src/tag.c

Patch 5.6.083 (extra)
问题: Win32: The visual beep can't be seen. (Eric Roesinger)

解决方案: Flush the output before waiting with GdiFlush(). (Maurice S. Barnum)
Also: Allow specifying the delay in t_vb for the GUI.
相关文件: src/gui.c, src/gui_amiga.c, src/gui_gtk_x11.c, src/gui_mac.c,
src/gui_riscos.c, src/gui_w32.c, src/gui_x11.c, src/gui_beos.cc,
src/proto/gui_amiga.pro, src/proto/gui_gtk_x11.pro,
src/proto/gui_mac.pro, src/proto/gui_riscos.pro,
src/proto/gui_w32.pro, src/proto/gui_x11.pro,
src/proto/gui_beos.pro

Patch 5.6.084 (depends on 5.6.074)
问题: GUI: Entering CSI doesn't always work for Athena and Motif.
解决方案: Handle typed CSI as <xCSI> (forgot this bit in 5.6.074).
相关文件: src/gui_x11.c

Patch 5.6.085
问题: Multi-byte: Using "r" to replace a double-byte char with a
single-byte char moved the cursor one character. (Matsumoto)
Also, using a count when replacing a single-byte char with a
double-byte char didn't work.
解决方案: Don't use del_char() to delete the second byte.
Get "ptr" again after calling ins_char().
相关文件: src/normal.c

Patch 5.6.086 (extra)
问题: Win32: When using libcall() and the returned value is not a valid
pointer, Vim crashes.
解决方案: Use IsBadStringPtr() to check if the pointer is valid.
相关文件: src/os_win32.c

Patch 5.6.087
问题: Multi-byte: Commands and messages with multi-byte characters are
displayed wrong.
解决方案: Detect double-byte characters. (Yasuhiro Matsumoto)
相关文件: src/ex_getln.c, src/message.c, src/misc2.c, src/screen.c

Patch 5.6.088
问题: Multi-byte with Motif or Athena: The message "XIM requires
fontset" is annoying when Vim was compiled with XIM support but it
is not being used.
解决方案: Remove that message.
相关文件: src/multbyte.c

Patch 5.6.089
问题: On non-Unix systems it's possible to overwrite a read-only file
without using "!".
解决方案: Check if the file permissions allow overwriting before moving the
file to become the backup file.
相关文件: src/fileio.c

Patch 5.6.090
问题: When editing a file in "/home/dir/home/dir" this was replaced with
"~~". (Andreas Jellinghaus)
解决方案: Replace the home directory only once in home_replace().
相关文件: src/misc1.c

Patch 5.6.091
问题: When editing many "no file" files, can't create swap file, because .sw[a-p] have all been used. (Neil Bird)
解决方案: Also use ".sv[a-z]", ".su[a-z]", etc.
相关文件: src/memline.c

Patch 5.6.092
问题: FreeBSD: When setting $TERM to a non-valid terminal name, Vim hangs in tputs().
解决方案: After tgetent() returns an error code, call it again with the terminal name "dumb". This apparently creates an environment in which tputs() doesn't fail.
相关文件: src/term.c

Patch 5.6.093 (extra)
问题: Win32 GUI: "ls | gvim -" will show a message box about reading stdin when Vim exits. (Donohue)
解决方案: Don't write a message about the file read from stdin until the GUI has started.
相关文件: src/fileio.c

Patch 5.6.094
问题: Problem with multi-byte string for ":echo var".
解决方案: Check for length in msg_outtrans_len_attr(). (Sung-Hyun Nam)
Also make do_echo() aware of multi-byte characters.
相关文件: src/eval.c, src/message.c

Patch 5.6.095
问题: With an Emacs TAGS file that include another a relative path doesn't always work.
解决方案: Use expand_tag_fname() on the name of the included file. (Utz-Uwe Haus)
相关文件: src/tag.c

Patch 5.6.096
问题: Unix: When editing many files, startup can be slow. (Paul Ackersviller)
解决方案: Halve the number of stat() calls used to add a file to the buffer list.
相关文件: src/buffer.c

Patch 5.7a.001
问题: GTK doesn't respond on drag&drop from ROX-Filer.
解决方案: Add "text/uri-list" target. (Thomas Leonard)
Also: fix problem with checking for trash arguments.
相关文件: src/gui_gtk_x11.c

Patch 5.7a.002
问题: Multi-byte: 'showmatch' is performed when second byte of an inserted double-byte char is a paren or brace.
解决方案: Check IsTrailByte() before calling showmatch(). (Taro Muraoka)
相关文件: src/misc1.c

Patch 5.7a.003
问题: Multi-byte: After using CTRL-O in Insert mode with the cursor at the end of the line on a multi-byte character the cursor moves to the left.
解决方案: Check for multi-byte character at end-of-line. (Taro Muraoka)
Also: fix cls() to detect a double-byte character. (Chong-Dae Park)
相关文件: src/edit.c, src/search.c

Patch 5.7a.004
问题: When reporting the search pattern offset, the string could be unterminated, which may cause a crash.
解决方案: Terminate the string for the search offset. (Stephen P. Wall)
相关文件: src/search.c

Patch 5.7a.005
问题: When ":s//~/" doesn't find a match it reports "[NULL]" for the pattern.
解决方案: Use get_search_pat() to obtain the actually used pattern.
相关文件: src/ex_cmds.c, src/proto/search.pro, src/search.c

Patch 5.7a.006 (extra)
问题: VMS: Various problems, also with the VAXC compiler.
解决方案: In many places use the Unix code for VMS too.
Added time, date and compiler version to version message.
(Zoltan Arpadffy)
相关文件: src/ex_cmds.c, src/ex_docmd.c, src/globals.h, src/gui_vms_conf.h, src/main.c, src/message.c, src/misc1.c, src/os_vms.c, src/os_vms.h, src/os_vms.mms, src/os_vms_conf.h, src/proto/os_vms.pro, src/proto/version.pro, src/term.c, src/version.c, src/xxd/os_vms.mms, src/xxd/xxd.c

Patch 5.7a.007
问题: Motif and Athena GUI: CTRL-@ is interpreted as CTRL-C.
解决方案: Only use "intr_char" when it has been set.
相关文件: src/gui_x11.c

Patch 5.7a.008
问题: GTK GUI: When using CTRL-L the screen is redrawn twice, causing trouble for bold characters. Also happens when moving with the scrollbar. Best seen when 'writedelay' is non-zero.
When starting the GUI with ":gui" the screen is redrawn once with the wrong colors.
解决方案: Only set the geometry hints when the window size really changed. This avoids setting it each time the scrollbar is forcefully redrawn.
Don't redraw in expose_event() when gui.starting is still set.
相关文件: src/gui_gtk_x11.c

5.8 版 本 version-5.8

5.8 版本是 5.7 的漏洞修正版。

改动 changed-5.8

Vim 不再包含 ctags。它已经成为独立的项目。这里可以找到：http://ctags.sf.net。
编写程序时，高度建议和 Vim 一起使用。

新增 added-5.8

新语法文件：

acedb	AceDB (Stewart Morris)
aflex	Aflex (Mathieu Clabaut)
antlr	Antlr (Mathieu Clabaut)
asm68k	68000 Assembly (Steve Wall)
automake	Automake (John Williams)
ayacc	Ayacc (Mathieu Clabaut)
b	B (Mathieu Clabaut)
bindzone	BIND zone (glory hump)
blank	Blank (Rafal Sulejman)
cfg	Configure 文件 (Igor Prischepoff)
changelog	ChangeLog (Gediminas Paulauskas)
cl	Clever (Phil Uren)
crontab	Crontab (John Hoelzel)
csc	Essbase 脚本 (Raul Segura Acevedo)
cynlib	Cynlib(C++) (Phil Derrick)
cynpp	Cyn++ (Phil Derrick)
debchangelog	Debian Changelog (Wichert Akkerman)
debcontrol	Debian Control (Wichert Akkerman)
dns	DNS 区域文件 (Jehsom)
dtml	Zope 的 DTML (Jean Jordaan)
dylan	Dylan、Dylan-intr 和 Dylan-lid (Brent Fulgham)
ecd	Embedix Component Description (John Beppu)
fgl	Informix 4GL (Rafal Sulejman)
foxpro	FoxPro (Powing Tse)
gsp	GNU Server Pages (Nathaniel Harward)
gtkrc	GTK rc (David Necas)
hercules	Hercules (Avant! Corporation) (Dana Edwards)
htmlos	Aestiva 的 HTML/OS (Jason Rust)
inittab	SysV process control (David Necas)
iss	Inno Setup (Dominique Stephan)
jam	Jam (Ralf Lemke)
jess	Jess (Paul Baleme)
lprolog	LambdaProlog (Markus Mottl)
ia64	Intel Itanium (parth malwankar)
kix	Kixtart (Nigel Gibbs)
mgp	MaGic Point (Gerfried Fuchs)
mason	Mason (带 Perl 的 HTML) (Andrew Smith)
mma	Mathematica (Wolfgang Waltenberger)
nqc	Not Quite C (Stefan Scherer)
omnimark	Omnimark (Paul Terray)
openroad	OpenROAD (Luis Moreno Serrano)
named	BIND configuration (glory hump)

```
papp            PApp (Marc Lehmann)
pfmain          Postfix main config (Peter Kelemen)
pic             PIC assembly (Aleksandar Veselinovic)
ppwiz           PPWizard (Stefan Schwarzer)
progress        Progress (Phil Uren)
psf             Product Specification File (Rex Barzee)
r               R (Tom Payne)
registry        MS-Windows 注册表 (Dominique Stephan)
robots          Robots.txt (Dominique Stephan)
rtf             Rich Text Format (Dominique Stephan)
setl            SETL (Alex Poylisher)
sgmldecl        SGML Declarations (Daniel A. Molina W.)
sinda           Sinda input (Adrian Nagle)
sindacmp        Sinda compare (Adrian Nagle)
sindaout        Sinda output (Adrian Nagle)
smith           SMITH (Rafal Sulejman)
snobol4         Snobol 4 (Rafal Sulejman)
strace          Strace (David Necas)
tak             TAK input (Adrian Nagle)
takcmp          TAK compare (Adrian Nagle)
takout          TAK output (Adrian Nagle)
tasm            Turbo assembly (FooLman)
texmf           TeX configuration (David Necas)
trasys          Trasys input (Adrian Nagle)
tssgm           TSS Geometry (Adrian Nagle)
tssop           TSS Optics (Adrian Nagle)
tsscl           TSS Command line (Adrian Nagle)
virata          Virata Configuration Script (Manuel M.H. Stol)
vsejcl          VSE JCL (David Ondrejko)
wdiff           Wordwise diff (Gerfried Fuchs)
wsh             Windows Scripting Host (Paul Moore)
xkb             X Keyboard Extension (David Necas)
```

php3 更名为 php，现在同时支持 php4 (Lutz Eymers)

Patch 5.7.015
问题:　　Vim 6.0 的语法文件不能用于 5.x。
解决方案:　":highlight" 命令加入 "default" 参数: 如果已经指定高亮，忽略命令。
相关文件:　src/syntax.c

用 makemenu.vim 生成 Syntax 菜单，从而不需要在 Vim 启动时完成。减少了 GUI 启动时间。

修正　　fixed-5.8

Conversion of docs to HTML didn't convert " tag s" to a hyperlink.

Fixed compiling under NeXT. (Jeroen C.M. Goudswaard)

optwin.vim gave an error when used in Vi compatible mode ('cpo' contains 'C').

Tcl interpreter: "buffer" command didn't check for presence of an argument.

(Dave Bodenstab)

dosinst.c: Added checks for too long file name.

Amiga: a file name starting with a colon was considered absolute but it isn't.
Amiga: ":pwd" added a slash when in the root of a drive.

Macintosh: Warnings for unused variables. (Bernhard Pruemmer)

Unix: When catching a deadly signal, handle it in such a way that it's
unlikely that Vim will hang. Call _exit() instead of exit() in case of a
severe problem.

Setting the window title from nothing to something didn't work after patch 29.

Check for ownership of .exrc and .vimrc was done with stat(). Use lstat() as
well for extra security.

Win32 GUI: Printing a file with 'fileformat' "unix" didn't work. Set
'fileformat' to "dos" before writing the temp file.

Unix: Could start waiting for a character when checking for a CTRL-C typed
when an X event is received.

Could not use Perl and Python at the same time on FreeBSD, because Perl used
"-lc" and Python used the threaded C library.

Win32: The Mingw compiler gave a few warning messages.

When using "ZZ" and an autocommand for writing uses an abbreviation it didn't
work. Don't stuff the ":x" command but execute it directly. (Mikael Berthe)

VMS doesn't always have lstat(), added an #ifdef around it.

Added a few corrections for the Macintosh. (Axel Kielhorn)

Win32: GvimExt could not edit more than a few files at once, the length of the
argument was fixed.

Vim 5.7 预发布补丁:

Patch 5.7.001
问题: When the current buffer is crypted, and another modified buffer
isn't, ":wall" will encrypt the other buffer.
解决方案: In buf_write() use "buf" instead of "curbuf" to check for the
crypt key.
相关文件: src/fileio.c

Patch 5.7.002
问题: When 'showmode' is set, using "CTRL-O :r file" waits three seconds
before displaying the read text. (Wichert Akkerman)
解决方案: Set "keep_msg" to the file message so that the screen is redrawn
before the three seconds wait for displaying the mode message.

相关文件: src/fileio.c

Patch 5.7.003
问题: Searching for "[[:cntrl:]]" doesn't work.
解决方案: Exclude NUL from the matching characters, it terminates the list.
相关文件: src/regexp.c

Patch 5.7.004
问题: GTK: When selecting a new font, Vim can crash.
解决方案: In gui_mch_init_font() unreference the old font, not the new one.
相关文件: src/gui_gtk_x11.c

Patch 5.7.005
问题: Multibyte: Inserting a wrapped line corrupts kterm screen.
Pasting TEXT/COMPOUND_TEXT into Vim does not work.
On Motif no XIM status line is displayed even though it is available.
解决方案: Don't use xterm trick for wrapping lines for multibyte mode.
Correct a missing "break", added TEXT/COMPOUND_TEXT selection request.
Add XIMStatusArea fallback code.
(Katsuhito Nagano)
相关文件: src/gui_gtk_x11.c, src/multbyte.c, src/screen.c, src/ui.c

Patch 5.7.006
问题: GUI: redrawing the non-Visual selection is wrong when the window is unobscured. (Jean-Pierre Etienne)
解决方案: Redraw the selection properly and don't clear it. Added "len" argument to clip_may_redraw_selection().
相关文件: src/gui.c, src/ui.c, src/proto/ui.pro

Patch 5.7.007
问题: Python: Crash when using the current buffer twice.
解决方案: Increase the reference count for buffer and window objects.
(Johannes Zellner)
相关文件: src/if_python.c

Patch 5.7.008
问题: In Ex mode, backspacing over the first TAB doesn't work properly.
(Wichert Akkerman)
解决方案: Switch the cursor on before printing the newline.
相关文件: src/ex_getln.c

Patch 5.7.009 (extra)
问题: Mac: Crash when using a long file.
解决方案: Don't redefine malloc() and free(), because it will break using realloc().
相关文件: src/os_mac.h

Patch 5.7.010
问题: When using CTRL-A on a very long number Vim can crash. (Michael Naumann)
解决方案: Truncate the length of the new number to avoid a buffer overflow.
相关文件: src/ops.c

Patch 5.7.011 (extra)
问题: Win32 GUI on NT 5 and Win98: Displaying Hebrew is reversed.
解决方案: Output each character separately, to avoid that Windows reverses the text for some fonts. (Ron Aaron)
相关文件: src/gui_w32.c

Patch 5.7.012
问题: When using "-complete=buffer" for ":command" the user command fails.
解决方案: In a user command don't replace the buffer name with a count for the buffer number.
相关文件: src/ex_docmd.c

Patch 5.7.013
问题: "gD" didn't always find a match in the first line, depending on the column the search started at.
解决方案: Reset the column to zero before starting to search.
相关文件: src/normal.c

Patch 5.7.014
问题: Rot13 encoding was done on characters with accents, which is wrong. (Sven Gottwald)
解决方案: Only do rot13 encoding on ASCII characters.
相关文件: src/ops.c

Patch 5.7.016
问题: When hitting 'n' for a ":s///c" command, the ignore-case flag was not restored, some matches were skipped. (Daniel Blaustein)
解决方案: Restore the reg_ic variable when 'n' was hit.
相关文件: src/ex_cmds.c

Patch 5.7.017
问题: When using a Vim script for Vim 6.0 with <SID> before a function name, it produces an error message even when inside an "if version >= 600". (Charles Campbell)
解决方案: Ignore errors in the function name when the function is not going to be defined.
相关文件: src/eval.c

Patch 5.7.018
问题: When running "rvim" or "vim -Z" it was still possible to execute a shell command with system() and backtick-expansion. (Antonios A. Kavarnos)
解决方案: Disallow executing a shell command in get_cmd_output() and mch_expand_wildcards().
相关文件: src/misc1.c, src/os_unix.c

Patch 5.7.019
问题: Multibyte: In a substitute string, a multi-byte character isn't skipped properly, can be a problem when the second byte is a backslash.
解决方案: Skip an extra byte for a double-byte character. (Muraoka Taro)
相关文件: src/ex_cmds.c

Patch 5.7.020
问题:　　Compilation doesn't work on MacOS-X.
解决方案:　Add a couple of #ifdefs. (Jamie Curmi)
相关文件:　src/regexp.c, src/ctags/general.h

Patch 5.7.021
问题:　　Vim sometimes produces a beep when started in an xterm. Only
　　happens when compiled without mouse support.
解决方案:　Requesting the xterm version results in a K_IGNORE. This wasn't
　　handled when mouse support is disabled. Accept K_IGNORE always.
相关文件:　src/normal.c

Patch 5.7.022
问题:　　%v in 'statusline' is not displayed when it's equal to %c.
解决方案:　Check if %V or %v is used and handle them differently.
相关文件:　src/screen.c

Patch 5.7.023
问题:　　Crash when a WinLeave autocommand deletes the buffer in the other
　　window.
解决方案:　Check that after executing the WinLeave autocommands there still
　　is a window to be closed. Also update the test that was supposed
　　to check for this problem.
相关文件:　src/window.c, testdir/test13.in, testdir/test13.ok

Patch 5.7.024
问题:　　Evaluating an expression for 'statusline' can have side effects.
解决方案:　Evaluate the expression in a sandbox.
相关文件:　src/edit.c, src/eval.c, src/proto/eval.pro, src/ex_cmds.c,
　　src/ex_cmds.h, src/ex_docmd.c, src/globals.h, src/option.c,
　　src/screen.c, src/undo.c

Patch 5.7.025 (fixed)
问题:　　Creating a temp file has a race condition.
解决方案:　Create a private directory to write the temp files in.
相关文件:　src/fileio.c, src/misc1.c, src/proto/misc1.pro,
　　src/proto/fileio.pro, src/memline.c, src/os_unix.h

Patch 5.7.026 (extra)
问题:　　Creating a temp file has a race condition.
解决方案:　Create a private directory to write the temp files in.
　　This is the extra part of patch 5.7.025.
相关文件:　src/os_msdos.h

Patch 5.7.027
问题:　　Starting to edit a file can cause a crash. For example when in
　　Insert mode, using CTRL-O :help abbr<Tab> to scroll the screen and
　　then <CR>, which edits a help file. (Robert Bogomip)
解决方案:　Check if keep_msg is NULL before copying it.
相关文件:　src/fileio.c

Patch 5.7.028
问题:　　Creating a backup or swap file could fail in rare situations.

解决方案: Use O_EXCL for open().
相关文件: src/fileio.c, src/memfile.c

Patch 5.7.029
问题: Editing a file with an extremely long name crashed Vim.
解决方案: Check for length of the name when setting the window title.
相关文件: src/buffer.c

Patch 5.7.030
问题: A ":make" or ":grep" command with a very long argument could cause
           a crash.
解决方案: Allocate the buffer for the shell command.
相关文件: src/ex_docmd.c

vim:tw=78:ts=8:noet:ft=help:norl:

version6.txt For Vim version 8.2. 最近更新: 2017年8月

VIM 参考手册

by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

欢迎来到 Vim 6.0 版本! 新版本增加了很多新特性。本文档列出所有加入的新项目、已有特性的改动以及 Vim 5.x 的漏洞修正。

vi_diff.txt 提供 Vi 和 Vim 6.0 的区别的一个概述。
version4.txt 说明 Vim 3.0 和 Vim 4.0 的区别。
version5.txt 说明 Vim 4.0 和 Vim 5.0 的区别。

不兼容的改动 incompatible-6

可视模式的光标位置 curpos-visual
替代命令与 Vi 兼容 substitute-CR
引入全局选项值 new-global-values
改变 'fileencoding' fileencoding-changed
改变二合字母 digraphs-changed
改变文件类型检测 filetypedetect-changed
引入列表外缓冲区 new-unlisted-buffers
改变命令行模式的 CTRL-U CTRL-U-changed
去掉 Ctags ctags-gone
重新调整文档 documentation-6
无模式选择和剪贴板 modeless-and-clipboard
细小的不兼容之处 incomp-small-6

新特性 new-6

折叠 new-folding
垂直分割窗口 new-vertsplit
比较模式 new-diff-mode
简易 Vim: 点击-输入 new-evim
用户手册 new-user-manual
灵活的缩进方式 new-indent-flex
扩展的搜索模式 new-searchpat
UTF-8 支持 new-utf-8
多语言支持 new-multi-lang
插件支持 new-plugins
文件类型插件 new-filetype-plugins
文件浏览器 new-file-browser
在网络上编辑文件 new-network-files
命令行编辑的窗口 new-cmdwin
调试模式 new-debug-mode
虚拟位置的光标 new-virtedit
调试器接口 new-debug-itf
Vim 间的通信 new-vim-server
缓冲区类型选项 new-buftype
打印 new-printing
移植版本 ports-6
扩展的 Quickfix quickfix-6
操作符修饰符 new-operator-mod

搜索路径 new-search-path
改进文件写入 new-file-writing
参数列表 new-argument-list
恢复视图 new-View
色彩方案 new-color-schemes
若干新项目 new-items-6

改 进 improvements-6

编 译 时 的 改 变 compile-changes-6

漏 洞 修 正 bug-fixes-6

6.1 版 本 version-6.1
改动 changed-6.1
新增 added-6.1
修正 fixed-6.1

6.2 版 本 version-6.2
改动 changed-6.2
新增 added-6.2
修正 fixed-6.2

6.3 版 本 version-6.3
改动 changed-6.3
新增 added-6.3
修正 fixed-6.3

6.4 版 本 version-6.4
改动 changed-6.4
新增 added-6.4
修正 fixed-6.4

(译者注：各版本的修正一节，因内容过于琐碎，保留原文未翻译)

不 兼 容 的 改 动 incompatible-6

这些改动和以前的发布版本不兼容。如果你从 Vim 5.x 升级到 6.0 时遇到问题，检查此列表。

可视模式的光标位置 curpos-visual

在可视模式里，如果从同一缓冲区的一个窗口跳到另一个窗口，另一个窗口的光标位置被调整，以维持相同的可视区域。这可以用来在一个窗口设置可视区域的开始，而在另一个窗口里设置可视区域的结束。vim 5.x 里会使用另一个窗口的光标位置，它可以在任何地方，从而不是十分有用。

替代命令与 Vi 兼容 substitute-CR

替代字符串 (substitute 命令的 "to" 部分) 现在和 Vi 兼容。以前，CTRL-V 有特殊含义，可以用来防止 <CR> 插入换行符。这使得在换行符之前无法插入 CTRL-V。现在使用反斜杠来防止 <CR> 产生换行符。因为通过反斜杠数量减半，仍然可能在行尾插入换行符 (译者注：似应为反斜杠)。这和 Vi 类似，但和 6.0 之前的 Vim 版本不兼容。

如果 ":s" 命令不进行任何替代，它不再设置 '[和 '] 位置标记。这和 Vi 无关，因为它没有这些位置标记。

引入全局选项值 new-global-values

现在，局部于缓冲区或者窗口的选项有了全局值。以前，局部选项从一个缓冲区复制到另一个。如果编辑另一个文件，可能导致模式行的选项值用在错误的文件上。现在进入以前没有使用过的缓冲区会使用全局值。另外，如果在一个窗口里编辑另一个缓冲区，窗口的局部变量被复位到它们的全局值上。":set" 命令同时设置局部和全局值，这仍然是兼容的行为。但模式行只会设置局部值，这与后向不兼容。

现在，":let &opt = val" 设置局部和全局值，类似于 ":set"。加入新的命令，以分别设置全局或局部值:

```
:let &opt = val          类似于 ":set"
:let &g:opt = val        类似于 ":setglobal"
:let &l:opt = val        类似于 ":setlocal"
```

改变 'fileencoding' fileencoding-changed

在 Vim 5.x 里，'fileencoding' 用来设置 Vim 里所有的编码。这有点奇怪，因为它是局部于缓冲区的，但它影响所有的缓冲区。不同缓冲区不可能有不一样，因为它改变了所有缓冲区的文本的解释方式。
现在，它被用作缓冲区相关文件的编码。如果你仍然设置 'fileencoding'，它可能被 'fileencodings' 检测到的编码替代，因而是 "基本上无害的"。
旧的 FileEncoding 自动命令现在和新的 EncodingChanged 事件相同。

改变二合字母 digraphs-changed

缺省的二合字母现在和 RFC1345 对应。这和 Vim 5.x 使用的非常不同。 digraphs

改变文件类型检测 filetypedetect-changed

过去，文件类型检测使用 "filetype" 自动命令组。这和 FileType 事件名混淆 (大小写是忽略的)。现在，该组叫做 "filetypedetect"。旧的名字仍然可以用，但如果使用了 "filetype" 组，":filetype off" 不会删除里面的自动命令。
'runtimepath' 的支持使得 "myfiletypefile" 和 "mysyntaxfile" 机制被废弃。为了后向兼容，它们仍然保留。

FileType 事件和 'syntax' 选项的设置之间的联系，以前在 "syntax" 自动命令组里设置。这和 Syntax 事件名有混淆。该组现在叫 "syntaxset"。

发布的语法文件不再包含 "syntax clear"。这使得在一个文件包含另一个文件不再需要什么技巧。现在，语法的清理在设置 'syntax' 选项时进行 (通过 synload.vim 里加入的自动命令)。这使得 'syntax' 的值不对应语法文件时进行语法清理。以前则保留已有的高亮。

引入列表外缓冲区 new-unlisted-buffers

现在缓冲区有不出现在缓冲区列表和实际不在缓冲区列表的区别。":ls"、 ":bnext"、":blast" 和 Buffers 菜单的命令会跳过不出现在缓冲区列表的缓冲区。
unlisted-buffer
'buflisted' 选项可以用来决定缓冲区是否出现在缓冲区列表里。

以前把缓冲区加到缓冲区列表的一些命令现在会建立列表外缓冲区。这意味着 ":bnext" 和 ":ball" 不会找到这些文件，除非实际开始编辑。比如，":write file" 和 ":read file" 的轮换文件使用的缓冲区。
　别的命令以前完全删除缓冲区，而现在只从缓冲区列表里删除缓冲区。依赖于缓冲区不存在的命令可能会失败。例如，自动命令里用了 ":bdelete" 命令，该自动命令可能会依赖于其后发生的事情会失败 (过去用于自动测试里)。
:bwipeout 可以用来实现 ":bdelete" 的旧含义。

BufDelete 自动命令事件现在当从缓冲区列表里删除缓冲区时激活。BufCreate 事件则在缓冲区列表里加入了新建立或者已有的缓冲区时激活。BufAdd 是 BufCreate 的新名字。新的 BufNew 事件用于任何缓冲区的建立，而 BufWipeout 用于真正地删除缓冲区。

如果进行插入模式的补全，只有缓冲区列表里的缓冲区被扫描。在 'complete' 里加入 'U' 标志位可以从列表外缓冲区中寻找补全。

列表外缓冲区不会在 viminfo 文件里保存。

改变命令行模式的 CTRL-U CTRL-U-changed

编辑命令行时，CTRL-U 过去清除整行。多数外壳里，它只清除光标之前的字符。现在使其行为类似于此。(Steve Wall)

你可以用 CTRL-E CTRL-U 得到旧的行为:

```
:cnoremap <C-U> <C-E><C-U>
```

去掉 Ctags ctags-gone

Ctags 不再是 Vim 发布的一部分。现在它自己独立门户，值得自己单独进行发布。
Ctags 可以在这里找到: http://ctags.sf.net/。

重新调整文档 documentation-6

文档现在被重新调整，不一定能在 Vim 5.x 相同的位置里找到每个项目。

- 加入了用户手册，有些项目被移到了参考手册。
- 快速参考指南现在放到了单独文件里（以便打印）。

文档里的示例以前用第一列的 ">" 标出。它增加了复制/粘贴这些示例的困难。现在示例以单个 ">" 开始，而以第一列出现的 "<" 或非空白结束。这使得没有高亮的时候看起来好一些。

不再用 'helpfile' 找到帮助标签文件。这使得用户可以加入自己的帮助文件（比如，插件的文档）。

无模式选择和剪贴板 modeless-and-clipboard

无模式的选择用来在无法使用可视模式时，例如在编辑命令行时或者在 more 提示时。选择文本。
Vim 5.x 里，总是使用无模式的选择。MS-Windows 上，这导致剪贴板被覆盖，而且没有办法避免。现在，无模式的选择由 'guioptions' 里的 'a' 和 'A' 标志位以及 'clipboard' 里的 "autoselect" 和 "autoselectml" 控制。缺省，在 MS-Windows 上没有自动复制。用 c_CTRL-Y 命令手动复制选择区。

要得到旧的行为，可以这么做：

```
:set clipboard^=autoselectml guioptions+=A
```

细小的不兼容之处 incomp-small-6

为了更好的安全性，模式行上不再能设置 'backupdir'、'cdpath'、'directory'、'equalprg'、'errorfile'、'formatprg'、 'grepprg'、'helpfile'、'makeef'、'makeprg'、'keywordprg'、'cscopeprg'、 'viminfo' 和 'runtimepath'。

'breakat' 的缺省删除了 '_': 关键字里经常用到。

'mousehide' 的缺省是打开的，因为这适用于多数人。

Amiga 二进制程序现在总是用 "big" 特性编译。"big" 二进制归档不再存在。

'statusline' 里的 "[RO]"、"[+]"、"[help]"、"[Preview]" 和 "[filetype]" 项目不再有引导空格。

非 Unix 系统：Vim 参数里扩展通配符时，不使用 'suffixes'。现在它的工作方式就像外壳扩展了这些参数一样。

'paste' 置位时，'lisp'、'smartindent' 和 'cindent' 选项不关闭。'paste' 置位时，自动缩进被关闭，但用 "=" 的手动缩进仍然可以工作。

用 "=" 排版时，使用 'cindent' 或 'indentexpr' 缩进，如果没有缩进的变动，不作为改变处理（不置位 'modified'，而且没有什么可撤销的）。

如果设置了 'fileencoding' 或 'fileformat'，报告 'modified' 为已修改。这样它忠实地反映了放弃缓冲区是否会丢失改变的可能性。

"Save As" 菜单项目现在编辑要保存的文件。多数人期望它就是如此。

目录的缓冲区不再加到 Buffers 菜单里。

<Return> 被换名为 <Enter>，因为这是多数键盘的称呼方式。这样，现在我们说 hit-enter 提示，而不是 hit-return 提示。
可以就像 <CR> 或 <Return> 那样映射 <Enter>。

如果没有置位 'compatible'，'viminfo' 选项的缺省现在是 '20,"50,h。多数用户希望使用该功能，包括初学者，但这需要设置该选项，这并不容易。

用了 ":colder" 以后，新的错误列表被覆盖。可用于使用 ":grep" 进行树状方式浏览。必须使用 ":cnewer 99" 才能得到原来的行为。

'errorformat' 的模式有时会忽略大小写 (MS-Windows)，有时又不会 (Unix)。现在，总是忽略大小写。在模式里加上 "\C" 以匹配大小写。

现在，16 位 MS-DOS 版本编译时不加入 +listcmds 特性 (操作缓冲区列表的命令)。并不经常需要它们，而这使需要的可执行文件小一些。

现在，'sessionoptions' 缺省包括 "curdir"。这意味着恢复会话会使得当前目录也被恢复，而不是转到会话文件所在的目录上。

过去，会话删除所有的缓冲区，删除所有的位置标记。现在保持了缓冲区列表，不应该影响已有的缓冲区的继续存在。
如果参数列表为空，":argdel *" 过去产生错误信息。

不再把标签跳转的搜索模式放到历史里。

不可显示字符的高亮使用 "SpecialKey" 高亮，而不再是 "NonText"。这里的思路是，不可显示字符或者任何和文件里的字符的显示不同的文本使用 "SpecialKey"，而 "NonText" 用于文件里实际不存在的文本。

Motif 的菜单和滚动条现在使用系统的缺省颜色。过去是灰色的。你仍然可以用 ":highlight" 命令和资源设置颜色。

"gq" 的文本排版在非空的空白行上断开段落。过去该行被删除，看来那种方法不怎么有用。

如果参数在命令未完成时结束，":normal" 不再挂起。
过去 Vim 会等到输入更多的字符，但不刷新屏幕。现在，它假设输入了一个 <Esc>。

工具栏的位图不再从 "$VIM/bitmaps" 里寻找，而是从 'runtimepath' 的 "bitmaps" 目录下寻找。

现在，hit-enter 提示使用命令行模式的菜单，而不是普通模式的菜单。通常，这样效果更好，而且允许使用 "Copy" 菜单，以产生 CTRL-Y 菜复制无模式选择区。

字体选择从 Windows 移到了 Edit 菜单，包括其它的设置。

缺省的 'isfname' 包含了更多字符，使得 "gf" 效果更好。

文档的许可证改为 Open Publication License。看起来很公平，因为它包含了 "the Vim book" 的部分内容，而后者也在 OPL 下发行。不利之处是，我们不能再强制销售手册的

人为乌干达作贡献了。

"ayy 之后，不要让 ""yy 或 :let @" = val 覆盖 "a 寄存器。使用无名寄存器代替。

MSDOS: 模式 "*.*" 过去也匹配没有句号的文件名。这和其它版本不一致。

插入模式里，CTRL-O CTRL-\ CTRL-N {cmd} 留在普通模式。过去它会回到插入模式。这使得 CTRL-\ CTRL-N 的含义不清，因为它应该把我们带到普通模式（尤其是在 ":amenu" 里）。

允许操作符后面使用 ":" 命令。可以用来实现新的移动命令。它不再中止等待的操作符。

Amiga 上可以使用 "-d {device}" 参数。如果编译时加入 diff 特性，就不行了。可用 "-dev {device}" 代替。 -dev

插入模式下的 <S-Insert> 缺省映射按本义插入文本，这样避免了 BS 这样的特殊字符产生副作用。

":confirm" 过去适用于一行的其余部分。现在只适用于其后的命令。因而，
":confirm if x | edit | endif" 就不行了。
要用 ":if x | confirm edit | endif"。这才是原来的想法，过去的工作方式不同是个漏洞。

新 特 性 new-6

折叠 new-folding

Vim 现在可以显示文本折叠的缓冲区。能用于快速浏览文件结构。也可以抽出、删除和放置折叠的文本，比如把函数移动到另一个位置。

有一系列相关的新的命令和选项。见 folding 。

垂直分割窗口 new-vertsplit

窗口也可以垂直分割。这使得窗口可以肩并肩的显示。它很好的一个使用方式是比较两个相似的文件（见 new-diff-mode ）。'scrollbind' 选项可以用来同步滚动。

垂直分割可以用下面的命令建立：

:vsplit 或 CTRL-W v 或 CTRL-W CTRL-V	:vsplit
:vnew	:vnew
:vertical {cmd}	:vertical

最后一个是修饰符，它对任何分割窗口的命令有意义。比如：

```
:vertical stag main
```

会垂直分割窗口，然后在新窗口里跳转到标签 "main"。

窗口到窗口的水平切换可以用 CTRL-W_h 和 CTRL-W_l 命令完成。 CTRL-W_k 和 CTRL-W_j 命令被改为跳转到光标位置之上和之下的窗口。

垂直和水平分割可以用你愿意的任何方式组合。如果使用鼠标，改变窗口大小很容易，只

要把指针定位在状态行上或者垂直分割线上然后拖动就可以了。GUI 上，特殊的鼠标指针形状指示你在哪里可以拖动状态行或分割线。

要改变垂直分割的窗口的大小，用 CTRL-W_< 和 CTRL-W_> 命令。
要使得窗口使用最大宽度，用 CTRL-W | 命令 CTRL-W_bar 。

要强制新窗口使用 Vim 窗口的全部宽度或高度，可以使用这两个修饰符：

:topleft {cmd}	新窗口出现在顶部并使用全部宽度，或在左侧并使用全部高度。
:botright {cmd}	新窗口出现在底部并使用全部宽度，或在右侧并使用全部高度。

这可以和 ":vertical" 一起使用，来强制垂直分割：

```
:vert bot dsplit DEBUG
```

会在最右侧打开窗口，占据 Vim 窗口的全部高度，光标在第一个 "DEBUG" 的定义上。
如果当前窗口的宽度不到 80 个字符，帮助窗口在上方打开，就像使用了 ":topleft" 一样。

若干选项可以用来设置垂直分割窗口的属性。这和已经有的水平的等价命令相类似：

水平	垂直
'splitbelow'	'splitright'
'winheight'	'winwidth'
'winminheight'	'winminwidth'

可以设置 'winminwidth' 为零，使得不用的窗口暂时几乎不占据任何空间，而不用关闭它们。

新的 'eadirection' 选项指示哪里可以应用 'equalalways'：

:set eadirection=both	两个方向
:set eadirection=ver	使窗口高度相同
:set eadirection=hor	使窗口宽度相同

可以用来避免在你需要保持窗口大小的时候改变之。

因为垂直分割可以使窗口变得很窄，文本行经常可能放不下。加入了 'sidescrolloff' 选项，使得光标左右可以保留一些上下文。'listchars' 选项被扩展，加入了 "precedes" 项目，它在文本左侧离开屏幕时显示记号，比如说，"<"。(Utz-Uwe Haus)

"-O" 命令行参数：类似于 "-o"，但垂直分割窗口。(Scott Urban)

加入命令，使得当前窗口移到最顶部 (CTRL-W K)，底部 (CTRL-W J)，左侧 (CTRL-W H) 和右侧 (CTRL-W L)。在新的位置上，窗口使用屏幕的完整的宽度/高度。

如果状态行上没有足够的空间同时显示文件名和标尺，标尺使用一半的宽度。用于狭窄的窗口。

比较模式 new-diff-mode

在比较模式里，Vim 显示两个、三个或四个文件的差异。折叠用来隐藏文件的相同部分。高亮用来显示删除和改变的行。见 diff-mode 。

启动比较模式的简单方法是用 "vimdiff file1 file2" 启动 Vim。
加入了 vimdiff 手册页面。

在运行中的 Vim 里， :diffsplit 命令用当前文件和另一个文件开始比较模式。

:diffpatch 命令使用当前文件和补丁文件开始比较模式。 :diffthis 命令开始当前窗口的比较模式。

可以用 :diffget 和 :diffput 命令消除不同之处。

- 'diff' 选项打开窗口的比较模式。
- :diffupdate 命令刷新比较结果。
- 'diffopt' 选项改变差异文的显示方法。
- 'diffexpr' 选项可以设置如何得到比较结果。
- 'patchexpr' 选项可以设置如何把补丁应用到文件上。
- 加入 "diff" 折叠方法。如果打开窗口进入比较模式，设置 'foldlevel' 为零并打开 'foldenable' 可以关闭折叠。
- 加入 DiffAdd、DiffChange、DiffDelete 和 DiffText 高亮组以指定差异文的高亮方式。缺省设置挺难看的...
- Unix: "make install" 里使 vimdiff 成为一个符号链接。
- 从发布里删除现在废弃的 "vimdiff.vim" 脚本。
- 加入 "[c" 和 "]c" 命令，在比较模式里移动到下一个/前一个改变。

简易 Vim: 点击-输入 new-evim

---

eVim 代表 "Easy Vim" (简易 Vim)。这是一个单独的程序，但也可用 "vim -y" 启动。

Vim 这样启动时，置位 'insertmode'，以打开点击-输入的编辑方式。使用 $VIMRUNTIME/evim.vim 脚本加入一些映射和设置一些选项，使得多数行为类似于 Notepad。这是为那些不能忍受两种模式的人专门设计的。

eView 和它一样，但使用只读模式。

GUI 上，CTRL-C 现在只在忙于做事情的时候中断，等待输入的时候不算。可以使用 CTRL-C 来复制文本到剪贴板。

用户手册 new-user-manual

加入了用户手册。它的组织围绕着编辑任务进行。从头到底读起来，就像一本书。它方便初学者开始学习 Vim，也帮助所有人学习使用最有用的 Vim 特性。它比阅读参考手册容易得多，但没有提供其中一些细节。见 user-manual 。

用户手册包含了 Steve Oualline 的 "the Vim book" 的部分内容 frombook 。
它在 OPL 下发行 manual-copyright 。

如果没有打开语法高亮，帮助文件里标注示例 ('>' 和 '<') 和头部行 ('~') 的字符被空格代替。

关闭帮助窗口时，如果窗口布局从那时起没有改变的话，窗口布局恢复为打开帮助之前的状况。
打开帮助窗口时，如果当前窗口的宽度小于 80 个字符并且不是屏幕的全部宽度，把它放在 Vim 窗口的顶部。

灵活的缩进方式 new-indent-flex

自动缩进现在可以用于任何语言。它用 Vim 脚本实现，使得缩进的计算非常灵活。

":filetype indent on" 命令允许使用提供的缩进脚本。用户手册有解释： 30.3 。

计算 'indentexpr' 选项可以得到行的缩进。'indentkeys' 选项指示什么时候重新启动缩进。通常，这些选项在缩进脚本里设置。就像语法文件一样，缩进脚本也会由很多人提供和维护。

扩展的搜索模式 new-searchpat

增加了模式匹配多行的可能性。 (部分由 Loic Grenie 提供)
搜索模式里增加的用于多行匹配的新项目:

\n	匹配行尾，在 [] 里也是
_[]	匹配范围里的字符还有行尾
_x	匹配字符类还有行尾
_.	匹配任何字符还有行尾
_^	匹配行首，可以用在正规表达式的任何位置
_$	匹配行尾，可以用在正规表达式的任何位置

若干其它搜索模式的新项目:

\c	忽略整个模式的大小写
\C	匹配整个模式的大小写
\m	为后面打开 magic
\M	为后面关闭 magic
\v	使得后面的字符 "非常 magic"
\V	使得后面的字符 "非常不 magic"
\@!	不匹配前导的匹配原。 例如: "foo\(bar\)\@!" 匹配 "foo " 但不是 "foobar"。
\@=	匹配匹配原，产生零宽度的匹配 例如: "foo\(bar\)\@=" 匹配 "foobar" 里的 "foo"。
\@<!	光标之前不匹配前导的匹配原
\@<=	光标之前匹配前导的匹配原
\@>	作为子表达式匹配前导的匹配原 (不能回溯)
\&	只有在前面和后面的分支都匹配的时候才匹配
\%[]	可选的匹配匹配原的列表; "end\%[if]" 匹配 "end"、"endi" 和 "endif"
\%(\)	类似于 \(\)，但不建立反向引用; 可以由任何数量，突破了 \(\) 组对只能有九个的限制
\%^	匹配文件首 (Chase Tingley)
\%$	匹配文件尾 (Chase Tingley)
\%#	匹配光标位置。(Chase Tingley)
\?	类似于 "\="，但不能用于 "?" 命令。
\%23l	在第 23 行匹配
\%<23l	在第 23 行前匹配
\%>23l	在第 23 行后匹配
\%23c、\%<23c、\%>23c	在/不到/超过第 23 列处匹配

\%23v、\%<23v、\%>23v　在/不到/超过虚拟第 23 列处匹配

用于语法项目:
\z(...\)　　设置外部引用匹配 (在区域项目的开始模式里)
\z1 - \z9　　使用外部引用匹配 (在区域项目的跳过或结束模式里)
　　(Scott Bigham)

\zs　　使用该位置作为匹配的开始
\ze　　使用该位置作为匹配的结束

取消了匹配只能有不超过 32767 次的 *, \+ 等等的限制。

增加了匹配多字节字符的支持。(部分有 Muraoka Taro 提供)
使得 "\<" 和 "\>" 支持 UTF-8。(Muraoka Taro)

UTF-8 支持　　new-utf-8

Vim 现在可以编辑 UTF-8 编码的文件。可以使用至多 31 位的字符，但只能显示 16 位字符。支持不超过两个的组合字符，它们叠加在前导字符之上。还支持双宽字符。见 UTF-8 。

也支持 UCS-2、UCS-4 和 UTF-16 编码，它们在内部转换为 UTF-8。还提供在 Latin1 环境下编辑 Unicode 文件的支持。其它的编码使用 iconv() 转换，或者使用 'charconvert' 指定的外部转换器。

有许多多字节支持的新项目:
- 加入 'encoding' 选项: 指定 Vim 内使用的字符编码。可以是任何 8 位编码，一些双字节编码或者是 Unicode。它从环境中初始化，如果找到能支持的值的话。
- 加入 'fileencoding' 和 'fileencodings': 指定文件的字符编码，类似于 'fileformat' 和 'fileformats'。
  如果 'encoding' 为 "utf-8" 而 'fileencodings' 为 "utf-8,latin1"，那么如果文件不是合法的 UTF-8，它被自动切换为 latin1。
- 加入 'bomb' 选项以检测文件开头的 BOM。可以在 'fileencodings' 里指定 "ucs-bom" 以自动判断文件是否以 BOM 开头，从而检测出 Unicode 文件。在 MS-Windows (NT 和 2000) 里尤其有用，因为它使用带 BOM 的 ucs-2le 文件 (比如，注册表的导出文件)。
- 加入 'termencoding' 选项: 指定终端使用的编码。可用于在非 Unicode 的 locale 里把 Vim 置于 utf-8 模式:
```
:let &termencoding = &encoding
:set encoding=utf-8
```
- 如果 'viminfo' 包含 'c' 标志位，viminfo 文件从写入时的 'encoding' 转换为当前的 'encoding'。
- 加入 ":scriptencoding" 命令: 转换要执行的脚本的文本行到 'encoding' 编码。可用于菜单文件。
- 加入 'guifontwide' 已指定双宽字符的字体。
- 加入韩语的字符类检测支持。同时修正了 search.c 的 cls()。(Chong-Dae Park)
- Win32: 无需 IME 输入多字节字符。(Alexander Smishlajev)
- Win32 使用 Mingw: 编译时加入 iconv 库。(Ron Aaron)
- Win32 使用 MSVC: 动态载入 iconv.dll 库。(Muraoka Taro)
- 可以用 Borland 5.5 编译多字节版本和 iconv 支持。(Yasuhiro Matsumoto)
- 加入 'delcombine' 选项: 单独删除组合字符。(Ron Aaron)

- "xim" 不需要 "xfontset" 特性。现在，它们是两个独立的特性。
- XIM: 输入语言字符时打开 XIM (插入模式、搜索模式、"f" 或 "r" 命令)。输入普通模式命令时关闭 XIM。
- XIM 激活时，'showmode' 消息显示 "XIM"。(Nam SungHyun)
- 支持 XIM 的 "CursorIM"。(Nam SungHyun)
- 'formatoptions' 加入 'm' 标志位：回绕单词时，允许在每个多字节上分割，而不单是空格上。
- 使 ":syntax keyword" 能用于多字节字符。
- 加入 Unicode 大写/小写反转和比较的支持。(基于 Raphael Finkel 的补丁)
  有第三种写法 ("title case"，词首字母大写) 的多字节字符上用 "~" 可以在三种写法间切换。 (Raphael Finkel)

允许为多字节字符定义二合字母。
加入 Unicode 的 RFC1345 二合字母。
多数接受字符参数的普通模式命令，比如 "r"、"t" 和 "f" 现在可以接受二合字母。
'cpoptions' 的 'D' 标志位关闭这一功能，以和 Vi 兼容。

加入语言映射和 'keymap' 以便输入多字节字符：
- 加入 ":lmap" 命令和相关设置：定义输入该文本语言的字符使用的映射，也适用于 "r"、"t" 等。插入和命令行模式下，CTRL-^ 切换映射的开关。
  如果没有语言映射的话。CTRL-^ 也切换输入方法 (Input Method) 的使用。可以在输入一半的时候切换出 IM。
- ":map" 的 "<char-123>" 参数可以指定字符的十进制、八进制和十六进制的值。
- 实现了 'keymap' 选项：载入键盘映射表文件。用 ":lnoremap" 来定义该键盘映射表的映射。新的 ":loadkeymap" 命令可以在键盘映射表文件里使用。
- 加入 'statusline' 的 'k' 标志位：使用键盘映射表时，"b:keymap_name" 或者说 'keymap' 的值。如果没有载入键盘映射表而激活了 ":lmap" 的话，"<lang>"。在缺省的状态行里也显示此文本。
- 加入 'iminsert' 和 'imsearch' 选项：用选项指定使用的是 langmap 映射和输入方法。(Muraoka Taro)
  加入 'imcmdline' 选项：如果置位，开始编辑命令行时总是打开输入方法。可用于使用死键 (dead key) 输入带重音的字符的 XIM。
  加入 'imactivatekey' 选项以更好的控制 XIM。(Muraoka Taro)
- 输入还没有结束的映射时，在插入模式和命令行模式里，显示光标下的最后一个字符。对死键字符看起来不错。
- 使得 'langmap' 选项能识别多字节字符。不过映射只能用于 8-位字符。有助于使用 UTF-8。
- ":lmap" 映射激活时，使用不同的光标。可以在 'guicursor' 的每个项目里指定两种高亮组 (用和不用语言映射)。缺省 "lCursor" 和 "Cursor" 是一样的，用户必须自己设定他喜欢的颜色。
  韩语输入页使用光标颜色。(Sung-Hyun Nam)
- 如果语言映射打开，'showmode' 里显示 "(lang)"。
- UTF-8: "r" 可用包含组合字符的 ":lmap"。也适用于 "f"，现在可以寻找包含组合字符的字符。

其它多字节字符的新增特点：
- euc-jp 支持双字节单宽度字符：0x8E 开始的字符。增加 ScreenLines2[] 来保存第二个字节。

多语言支持 new-multi-lang

Vim 使用的消息可以翻译。已经有几个翻译版本。这里，使用了 gettext 机制。它可以

加入翻译版本而无需重新编译 Vim。 multi-lang （部分由 Marcin Dalecki 提供）

翻译文件在 src/po 目录里。src/po/README.txt 文件解释进行翻译的一些事项。

也可以用翻译的菜单。用新的 :menutranslate 命令。翻译可以在运行时目录 "lang" 里找到。用户因而可以增加翻译。

加入 :language 命令来设置消息、时间和字符类型的语言 (locale)。可以用来在 Vim 里切换语言，而无需离开 Vim 去更改 locale。

可以使用不同语言的 vimtutor。(Eduardo Fernandez) 现在包含了西班牙语 (Eduardo Fernandez)、意大利语 (Antonio Colombo)、日语 (Yasuhiro Matsumoto) 和法语 (Adrien Beau) 的翻译。
加入 "vimtutor.bat": MS-Windows 上，在教学文件上启动 Vim 的脚本。(Dan Sharp)

- 加入 v:lang 变量已得到当前的语言设置。(Marcin Dalecki) 还有 v:lc_time 和 v:ctype。
- 可以翻译菜单使用的对话框。使用全局的 "menutrans_" 变量。":menutrans clear" 删除它们。
- 删除了 "有问题的 locale" (Marcin Dalecki)。
- 图标里不使用色彩名，使用 RGB 值。名字可能被翻译。
- Win32: 加入 global IME 支持 (Muraoka)
- Win32: 加入动态载入 IME 的支持。
- ":messages" 显示关于谁维护消息或翻译的信息。可用于找到哪里可以给出错误翻译的意见的相关信息。
- configure 的 --disable-nls 参数: 关闭 gettext() 的使用。(Sung-Hyun Nam)
- 加入使用 MingW 编译器的 Win32 版本的 NLS 支持。(Eduardo Fernandez)
- 如果可用，调用 bind_textdomain_codeset() 来使得 gettext() 把消息翻译成 'encoding'。这需要 GNU gettext 0.10.36 或其后的版本。
- 加入 Win32 的 gettext 支持。这意味着消息会在设置了 locale 和找到 libintl.dll 的前提下被翻译。(Muraoka Taro)
  也可以用于 MingW 编译器的版本。(Eduardo Fernandez)
  检测语言并设置 $LANG 已得到合适的翻译过的消息 (如果支持的话)。也用 $LANG 来选择语言，:lang 是不同类型的名字。
- 使得 gvimext.dll 使用翻译过的消息，如果可能的话。(Yasuhiro Matsumoto)

插件支持 new-plugins

要在启动 Vim 时非常方便地载入 Vim 脚本，可以用 "plugin" 运行时目录。所以这里面的 "*.vim" 文件被自动载入。Unix 上，缺省使用 "~/.vim/plugin"。 'runtimepath' 选项可以设置在别的目录下查找插件。 load-plugins add-plugin

加入 :runtime 命令以载入一个或多个 'runtimepath' 的文件。

标准插件:
netrw.vim - 在网络上编辑文件 new-network-files
gzip.vim - 编辑压缩文件
explorer.vim - 浏览目录 new-file-browser

加入本地帮助文件的支持。 add-local-help 。
搜索帮助标签时，使用 'runtimepath' 的 "doc/tags" 的所有文件。
加入 ":helptags" 命令: 为一个帮助目录生成标签文件。

每个帮助文件的第一行被自动（神奇地）加到 doc/help.txt 的 "本地附加文档" 一节。

":map" 加入 <unique> 参数：只有在没有定义过的时候才加入映射。

设置 'verbose' 以后显示选项值会给出关于该选项最后在哪里设置的消息。对发现什么脚本设置了选项非常有用。

新的 :scriptnames 命令显示所有执行过的脚本的列表。

GUI：Athena、Motif 和 GTK 版本在 'runtimepath' 的 "bitmaps" 目录里寻找工具栏位图。可以加入你自己的位图。

文件类型插件 new-filetype-plugins

加入一组新的文件，提供特定文件类型的设置。它们是专用于 'filetype' 某个值的选项和映射。

这些文件在 "$VIMRUNTIME/ftplugin" 里。'runtimepath' 选项可以使你使用几组不同的插件：你自己的、系统范围的、Vim 发布里包含的，等等。

要使之工作，加入了若干特性：
- 加入 "s:" 变量，局部于脚本。避免和全局变量的命令冲突。可用于脚本和脚本里定义的函数、自动命令和用户命令。在同一脚本的多次调用间保持它们的值。 s:var
- 加入局部选项的全局值。该值在打开新缓冲区或者编辑另一个文件的时候使用。模式行或文件类型设置不会带到另一个缓冲区里去。
  ":set" 同时设置局部和全局值。
  ":setlocal" 只设置局部选项值。
  ":setglobal" 设置和显示局部选项的全局值。
  ":setlocal name<" 设置局部选项为其全局值。
- 加入一些全局选项的局部于缓冲区的值：'equalprg'、'makeprg'、'errorformat'、'grepprg'、'path'、'dictionary'、'thesaurus'、'tags'、'include' 和 'define'。可以设置这些全局选项的局部值，而不会使它不兼容。
- 加入局部于缓冲区的映射和缩写：":map <buffer>"。
- 映射里可以用 "<Leader>" 得到 "mapleader" 变量的值。这简化了使用 "mapleader" 的映射。"<Leader>" 缺省为 "\"。"<LocalLeader>" 和 "maplocalleader" 相同。它用在局部于缓冲区的映射里。
- 加入脚本 ID <SID>，定义局部于脚本的函数和映射。
- ":noremap" 和 ":noremenu" 加入 <script> 参数：只重映射局部于脚本的映射。避免其它脚本的映射的妨碍，但允许使用脚本里定义的映射。
- 用户命令可以局部于缓冲区：":command -buffer"。

文件类型检测的自动命令使用新的 ":setfiletype" 命令，以避免设置 'filetype' 两次。

文件浏览器 new-file-browser

编辑目录时，explorer 插件会列出该目录的文件。在文件名上按 <Enter> 编辑该文件。在目录上按 <Enter> 切换浏览器到该目录里。

有若干其它的可能，比如在预览窗口里打开文件，给文件换名或者删除文件。

在网络上编辑文件 new-network-files

scp://、rcp://、ftp:// 和 http:// 开始的文件被识别为远程文件。使用指定的方法，作出相应的尝试以取得该文件。http:// 只能读取，其他的方法也支持写入。这里，使用了标准 "插件" netrw.vim 脚本。 netrw

使得 "gf" 可用于 URL 上。它不再假设该文件是在本地计算机上的（多数也不会工作，因为需要完整路径）。为此，调整了 test2。

允许在 'path' 里使用 URL。使得 ":find index.html" 可以工作。

GTK: 允许在 Vim 上放置（drop）http:// 和 ftp:// URL。netrw 插件会处理文件的下载。（Mikael Berthe）

命令行编辑的窗口 new-cmdwin

命令行窗口可以使用普通模式和插入模式的命令来编辑命令行。打开的时候，它包含历史记录。从而可以从以前的命令行上复制部分内容。 cmdwin

命令行窗口可以从命令行上用 'cedit' 选项指定的键打开（类似于 Nvi）。也可以直接在普通模式里用 "q:"、"q/" 和 "q?" 打开。

'cmdwinheight' 指定该窗口的初始高度。

插入模式里，CTRL-X CTRL-V 补全 Ex 命令行，就像命令行所做的那样。可以用来编写 Vim 脚本！

另外，还有 "增强的 Ex 模式"。Vim 以 "exim" 或 "vim -E" 启动，或者使用 "gQ" 命令都可以进入该模式。就像重复使用 ":" 命令一样，提供完整的命令行编辑和补全功能。（Ulf Carlsson）

调试模式 new-debug-mode

在调试模式里，脚本和用户函数的执行可以逐行进行。提供跳过（step over）或跳入（step into）命令的命令。 debug-mode

可以设置断点，使得继续运行，直到在脚本或用户函数的特定行上停止。 :breakadd

调试可以用 ":debug {cmd}" 开始，它调试执行该命令时发生了什么事。启动时，可以用 -D 参数来调试。

虚拟位置的光标 new-virtedit

加入 'virtualedit' 选项：插入模式、可视模式或任何情况下，可以定位光标到没有实际字符存在的位置。（Matthias Kramm）

对可视列块模式特别有用。它能定位到区域的没有文本字符的一角。(许多改进由 Chase Tingley 提供)

调试器接口 new-debug-itf

原先，这是为 Sun Visual Workshop 提供的。(Gordon Prieur)
见 debugger.txt 、 sign.txt 和 workshop.txt 。

加入 ":sign" 命令，定义和放置标号。它们可以显示为两个 ASCII 字符或一个图标。标号之后的行被高亮显示。可以用来显示断点和当前的 PC 位置。

加入 :wsverb 命令来执行调试器命令。

加入气泡的设置: 'balloondelay' 和 'balooneval' 选项。

加入 ":menu" 的 "icon=" 参数。可以为某个 ToolBar 项目定义特定的图标。

Vim 间的通信 new-vim-server

加入两个 Vim 之间的通信机制。可以从一个 Vim 发送命令到另一个 Vim。支持 X-Windows 和 MS-Windows clientserver 。

用 "--remote" 来用已经运行的 Vim 里编辑文件。
用 "--remote-wait" 来做相同的事，并等待编辑结束。
用 "--remote-send" 从一个 Vim 发送命令给另一个。
用 "--remote-expr" 来在另一个 Vim 里计算表达式。
用 "--serverlist" 列出当前可用的 Vim 服务器。(只限于 X)
还有服务器和客户之间通信的函数。 remote_send() remote_expr()

(X-windows 版本由 Flemming Madsen 实现，MS-Windows 版本由 Paul Moore 提供)

在窗口标题里加入命令服务器的名字，使得你可以看到哪个服务器名属于哪个 Vim。

从发布里删除了 OleVim 目录、SendToVim.exe 和 EditWithVim.exe。现在可以用可以移植的 "gvim --remote" 和 "gvim --remote-send"。

GTK+: 支持在别的窗口里运行 Vim。用 --socketid 参数 (Neil Bird)

缓冲区类型选项 new-buftype

加入了 'buftype' 和 'bufhidden' 选项。它们可以设置不同类型的缓冲区。例如:
- 'buftype' = "quickfix": 错误列表缓冲区
- 'buftype' = "nofile" 而 'bufhidden' = "delete": 草稿缓冲区，一旦没有窗口显示，会尽快删除。

'bufhidden' 可以用来否决一个缓冲区的 'hidden' 选项。

这些选项与 'buflisted' 和 'swapfile' 的组合提供了使用不同类型的特殊缓冲区的可

能性。见 special-buffers 。

打印 new-printing

包含了第一个 ":hardcopy" 命令的实现，用以打印到纸张上。MS-Windows 上，可以使用任何安装的打印机。其他系统上，生成 PostScript 文件，可以用 'printexpr' 选项进行打印。
(MS-Windows 部分由 Vince Negri、Vipin Aravind 提供，PostScript 由 Vince Negri 和 Mike Williams 提供)

使得 ":hardcopy" 可用于多字节字符。(Muraoka Taro，Yasuhiro Matsumoto)

加入选项，微调打印的工作方式：(Vince Negri)
- 'printoptions' 定义一些设置。
- 'printheader' 指定页眉格式。'statusline' 加入了 "N" 域，显示页号。
- 'printfont' 指定字体名和属性。
- 'printdevice' 定义 ":hardcopy!" 的缺省打印机。

移植版本 ports-6

OS/390 Unix 的移植版本 (Ralf Schandl)
- 很多改动，为了处理 EBCDIC 编码。
- 改变 Ctrl('x') 为 Ctrl_x 定义。

包含 jsbmouse 支持。(Darren Garth)
Unix 上支持 dec 鼠标。(Steve Wall)

移植到 16 位 MS Windows (Windows 3.1x) (Vince Negri)

移植到 QNX。支持 Photon GUI、鼠标等。(Julian Kinraid)

用 Make_ming.mak 可以进行 Win32 版本的跨平台编译。(Ron Aaron)
Mingw 的编译加入 Python 支持。(Ron Aaron)

32 位 Dos: 加入 Windows 剪贴板的支持。(David Kotchan)

Win32: 动态载入 Perl 和 Python。可以编译 Vim 来使用这些接口，并在运行时寻找这些 DLL。(Muraoka Taro)

用 Cygwin 编译 Win32 GUI。同时编译 vimrun、dosinst 和 uninstall。(Gerfried)

Mac: 使得 Vim 能用 Apple 提供的免费的 MPW 编译器编译。CodeWarrior 的更新。
(Axel Kielhorn)

加入一些类型转换和 ifdef，作为 Vim 在 Win64 上工作的开始 (George Reilly)

扩展的 Quickfix quickfix-6

加入 "错误窗口"。它包含了所有的当前错误列表的错误。在一行上按 <Enter> 使得 Vim 跳转到该行上（在别的窗口里）。这使得错误列表的导航容易得多。 quickfix-window 。

- :copen 打开 quickfix 窗口。
- :cclose 关闭 quickfix 窗口。
- :cwindow 只在有识别出来的错误时，才打开 quickfix 窗口。(Dan Sharp)

- 除了 "warning" 和 "error" 类型以外，Quickfix 也知道 "info"。"%I" 可用来定义多行信息消息的开始。(Tony Leneis)
- 'errorformat' 里的 "%p" 参数可用来得到指针行的列号，其中 "^" 指向该列。(Stefan Roemer)
- DOS/Windows 系统上，'errorformat' 的 "%f" 会在文件名里加入 "c:"，即使使用的是 "%f:"。

操作符修饰符 new-operator-mod

操作符和动作命令之间插入 "v"、"V" 或 CTRL-V 强制该操作符以面向字符、面向行和面向列块的方式工作。 o_v

搜索路径 new-search-path

Vim 可以在目录树里搜索，不只是往下，也可以往上。
可用于 'path'、'cdpath' 和 'tags' 选项。(Ralf Schandl)

也用于 'tags' 选项的 "**"。(Ralf Schandl)

加入 'includeexpr'，用来修改 'include' 选项找到的文件名。
'includeexpr' 也在 "gf" 和 "<cfile>" 里使用，如果找不到未经修改的文件的话。用于在 include 或 import 语句之后的名字上执行 "gf"。

加入 'cdpath' 选项：找到 ":cd" 参数的位置。(Raf)

加入 'suffixesadd' 选项：用 "gf"、"[I" 等命令搜索文件时给文件名附加的后缀。

改进文件写入 new-file-writing

加入 'backupcopy' 选项：选择文件要通过复制还是换名来建立备份文件。用于在 Unix 上加快写入普通文件的速度。其它系统上，也有助于保留文件属性和编辑 Unix 文件系统的文件。

加入 'autowriteall' 选项。和 'autowrite' 类似，但用于更多命令。

加入 'backupskip' 选项：文件名模式的列表，如果匹配，不建立备份。Unix 的缺省值包含 "/tmp/*"，这使得 "crontab -e" 可以工作。

加入 FreeBSD 和 Win32 上的访问控制表（ACL）的支持。ACL 从原来的文件复制到新文件里（或者备份文件，如果选择它是复制建立的话）。

AIX、Solaris 和通用的 POSIX 也支持 ACL。(Tomas Ogren)
SGI 亦然。

参数列表 new-argument-list

参数列表的支持也有了扩展。现在，可以操作参数列表以包含你要包含的文件。

参数列表现在可以局部于窗口。用 :arglocal 命令建立这样的参数列表。
:argglobal 命令可以用来回到全局的参数列表。

:argdo 命令在参数列表的所有文件上执行命令。

用 :argadd 可以把文件名加进参数列表。 :argdelete 可以删除文件名。

"##" 类似于 "#"，它被参数列表所有名字的连接替换。用于 ":grep foo ##" 这样的场合。

:argedit 把文件名加入参数列表，并编辑之。类似于 ":argadd" 然后 ":edit"。

恢复视图 new-View

":mkview" 命令写入 Vim 脚本，包含单个窗口的设置和映射。执行建立的文件可以恢复该窗口的视图。类似于单个窗口的 ":mksession"。
视图也包含局部参数列表和手动建立的打开或关闭的折叠。

加入 ":loadview" 命令和 'viewdir' 选项：可以用简单的命令保存和恢复文件的视图。":mkview 1" 保存当前文件的视图 1，而 ":loadview 1" 又可以载入。另外，可以在同一文件的两个视图间快速切换。还可以保存和恢复手动折叠以及折叠的状态。

加入 'viewoptions' 以指定 ":mkview" 的工作方式。

现在，":mksession" 也支持垂直分割。它被进一步改进，以恢复每个窗口的视图。它对预览和 quickfix 窗口同样能正确工作。

'sessionoptions' 也用于 ":mkview"。
'sessionoptions' 加入 "curdir" 和 "sesdir"。它们可以选择如何恢复当前目录。

会话文件现在也包含了参数列表。

色彩方案 new-color-schemes

支持载入色彩方案。加入了 ":colorscheme" 命令。
自动为可用的方案加入菜单项。
现在，应该会在 'background' 或 't_Co' 改变时，正确地自动复位色彩。
":highlight clear" 可以再次设置缺省的颜色。
":syntax reset" 设置语法高亮颜色为其缺省值。
":set bg&" 猜测其值。这使得色彩方案可以切换回缺省的颜色。
如果打开了语法高亮时已经定义了色彩方案，该色彩方案被重新载入以（再次）定义颜

色。

若干新项目 new-items-6

普通模式命令：

"gi" 跳转到 ^ 位置标记并开始插入模式。即使位置标记在行尾之后也可以。 gi

"g'm" 和 "g`m" 跳转到位置标记，而不修改跳转表。现在，你可以用 g`" 来跳回最近所知的文件位置，而不会有副作用。对映射也有用。

['、[`、]' 和 ]` 移动光标到下一个/前一个小写的位置标记。

g_ 转到行内的最后的一个非空白。(Steve Wall)

选项：

'autoread' 如果检测到文件在 Vim 之外改变而缓冲区没有改动的话，自动再次读入缓冲区。它有全局和局部值。用 ":setlocal autoread<" 来回到直接使用 'autoread' 的全局值的设定。

'debug' 如果设为 "msg"，会显示平时忽略的错误消息。用于调试 'indentexpr' 和 'foldexpr'。

'lispwords' Lisp 缩进用的单词列表。以前是直接在源代码里编码的。缺省加入了一些 Lisp 名字。

'fold...' 折叠的许多新选项。

'modifiable' 如果关闭，不能修改缓冲区。'statusline' 的 %m 和 %M 项目显示 '-'。

'previewwindow' 在预览窗口里置位。在会话文件里使用，以标记一窗口为预览窗口。

'printfont'
'printexpr'
'printheader'
'printdevice'
'printoptions' 用于 ":hardcopy"。

'buflisted' 使得缓冲区出现或不出现在缓冲区列表里。

模式行里使用 "vim{version}:" 可以使它只在版本 >= {version} 时才执行。还有 "vim>{version}"、"vim<{version}" 和 "vim={version}"。

Ex 命令：

:sav[eas][!] {file}
类似于 ":w file" 和 ":e #"，但不会再次载入，也避免了其它的副作用。 :saveas

:silent[!] {cmd}
安静的执行命令。也不使用消息之后可能的延迟，也不使用 'showmatch'。
RISCOS: 删除了 "!~cmd" 不产生输出也不等待之后的 <Enter> 的效果。现在可以用 ":silent !cmd"。
:menu <silent> 加入不回显 Ex 命令的菜单。
:map <silent> 加入不回显 Ex 命令的映射。

:checktime 检查缓冲区的改变。

:verbose {cmd} 为单个命令设置 'verbose'。

:echomsg {expr}
:echoerr {expr} 类似于 ":echo"，但在历史里保存消息。(Mark Waggoner)

:grepadd 类似于 ":grep"，但加到当前的错误列表，而不是定义新的列表。 :grepadd

:finish 结束文件的执行。可以用来跳过 Vim 脚本的其余部分。 :finish

:leftabove
:aboveleft 在当前窗口的左侧/上方分割窗口。

:rightbelow
:belowright 在当前窗口的右侧/下方分割窗口。

:first、:bfirst、:ptfirst 等
":rewind" 的别名。和 ":last" 相较，更加合乎逻辑。

:enew 编辑新的无名缓冲区。这有必要，因为 ":edit" 重新编辑同一文件。(Wall)

:quitall 等同于 ":qall"。

:match 定义局部于窗口的匹配高亮。可以为当前窗口高亮项目，而不会影响语法高亮。

:menu enable
:menu disable 打开/关闭菜单项的命令，而不用删除它们。(Monish Shah)

:windo 在所有窗口里执行命令。
:bufdo 在所有缓冲区里执行命令。

:wincmd 窗口 (CTRL-W) 命令。可用于不能使用普通模式命令的场合 (比如，CursorHold 自动命令里)。 CursorHold-example 给出一个很好的应用实例。

:lcd 和 :lchdir
设置窗口的本地目录。(Benjie Chen)

:hide {command}
以置位的 'hidden' 执行 {command}。

:emenu 在可视模式里，执行 ":vmenu" 项目。

:popup 弹出弹出式菜单。

:redraw 即使忙于执行脚本或函数时，也刷新屏幕。

:hardcopy 打印到纸张。

:compiler 载入 Vim 脚本，以处理特定编译器的设定。

:z# 列出编号的行。(Bohdan Vlasyuk)

新的位置标记：

'(和 ') 当前句子的开始或结束。用于 Ex 命令。
'{ 和 '} 当前段落的开始或结束。用于 Ex 命令。
'. 当前缓冲区的最近改变位置。
'^ 插入模式停止的位置。

viminfo 文件保存 ^ 和 . 位置标记。可以用来跳转到最近的插入位置或改变文本上。

新函数：

argidx() 参数列表的当前索引。
buflisted() 检查缓冲区是否存在并置位了 'buflisted'。
cindent() 得到 'cindent' 的缩进。
eventhandler() 如果在事件处理程序而不能使用交互命令的时候返回 1。
executable() 检查是否程序或批处理脚本可执行。
filewritable() 检查文件是否可写。(Ron Aaron)
foldclosed() 检查是否有关闭折叠。(Johannes Zellner)。
foldcloseend() 发现关闭折叠的结尾。
foldlevel() 发现折叠级别。(Johannes Zellner)
foreground() 把 GUI 窗口移到前台。
getchar() 让用户输入一个字符。可以用来定义接受一个参数的映射。
getcharmod() 得到最近使用的键修饰符。
getbufvar() 得到缓冲区里选项或局部变量的值 (Ron Aaron)
getfsize() 返回文件大小。
getwinvar() 得到窗口里选项或局部变量的值 (Ron Aaron)
globpath() 发现目录列表里匹配的文件
hasmapto() 检测是否到字符串的映射已经存在。
iconv() 把字符串从一个编码转换为另一个编码。
indent() 得到行的缩进 (Ron Aaron)
inputdialog() 类似于 input()，但在可能的情况下，使用 GUI 对话框。目前支持 Win32、Motif、Athena 和 GTK。
Edit/Settings/Text Width 菜单使用 inputdialog()。
Help/Find.. 和 Toolbar FindHelp 项目也是。
(Win32 支持由 Thore B. Karlsen 提供)
(Win16 支持由 Vince Negri 提供)
inputsecret() 要求用户输入字符串，但不回显输入的键。(Charles Campbell)
libcall() 用于 Unix (Neil Bird, Johannes Zellner, Stephen Wall)

libcallnr()	用于 Win32 和 Unix
lispindent()	得到 'lisp' 的缩进。
mode()	返回指示当前模式的字符串。
nextnonblank()	跳过正向的空行。
prevnonblank()	跳过反向的空行。可用于缩进脚本。
resolve()	MS-Windows: 解析快捷方式为它指向的文件。 Unix: 解析符号链接。
search()	搜索模式。
searchpair()	搜索匹配的组对。可用于缩进文件，以找到匹配 endif 的 "if"。
setbufvar()	设置缓冲区的选项或局部变量。(Ron Aaron)
setwinvar()	设置窗口的选项或局部变量。(Ron Aaron)
stridx()	搜索一个字符串在另一个字符串里的第一次出现。
strridx()	搜索一个字符串在另一个字符串里的最后一次出现。
tolower()	把字符串转换为全部小写。
toupper()	把字符串转换为全部大写。
type()	检查表达式的类型。
wincol()	光标的窗口列
winwidth()	窗口宽度。(Johannes Zellner)
winline()	光标的窗口行

加入了变量和函数命令的花括号扩展。可以用于包含选项值的变量名。也可以用来提供原始的数组。(Vince Negri)

新的自动命令事件：

BufWinEnter	缓冲区在窗口显示时激活。在使用模式行之后。可用于载入视图。
BufWinLeave	缓冲区不再出现于窗口时激活。也在退出 Vim 时激活。可用于保存视图。
FileChangedRO	在第一次修改只读文件之前激活。可用于文件的刷新。(Scott Graham)
TermResponse	在终端应答版本请求时激活。v:termresponse 内部变量保存该值。可用于对终端版本进行处理。(Ronald Schild)
FileReadCmd	在读入文件前激活。
BufReadCmd	在读入文件到缓冲区前激活。
FileWriteCmd	在写回文件前激活。
BufWriteCmd	在写回缓冲区到文件前激活。
FileAppendCmd	在附加到文件前激活。
FuncUndefined	在用户函数没有定义时激活。(Ron Aaron)

*Cmd 事件的自动命令进行文件读写，以取代普通的文件读写。netrw.vim 利用这一点来编辑远程系统上的文件。(Charles Campbell)

新的语法文件：

bdf	BDF 字体定义 (Nikolai Weibull)
catalog	SGML 目录 (Johannes Zellner)
debchangelog	Debian Changelog (Wichert Akkerman)
debcontrol	Debian 控制 (Wichert Akkerman)
dot	dot (Markus Mottl)
dsl	DSSSL 语法 (Johannes Zellner)
eterm	Eterm 配置 (Nikolai Weibull)
indent	缩进概要文件 (profile) (Nikolai Weibull)
lftp	LFTP (Nikolai Weibull)

lynx Lynx 配置 (Doug Kearns)
mush mush 源代码 (Bek Oberin)
natural Natural (Marko Leipert)
pilrc Pal 资源编译器 (Brian Schau)
plm PL/M (Philippe Coulonges)
povini Povray 配置 (David Necas)
ratpoison Ratpoison 配置/命令 (Doug Kearns)
readline readline 配置 (Nikolai Weibull)
screen Screen RC (Nikolai Weibull)
specman Specman (Or Freund)
sqlforms SQL*Forms (Austin Ziegler)
terminfo terminfo (Nikolai Weibull)
tidy Tidy 配置 (Doug Kearns)
wget Wget 配置 (Doug Kearns)

更新了许多语法文件，使得在 Vim 5.7 和 6.0 里都能工作。

Ruby 接口。(Shugo Maeda)
支持 MS-Windows 上的 Ruby 接口的动态载入。(Muraoka Taro)
Mingw 也支持。(Benoit Cerrina)

Win32: 加入动态载入 TCL 的可能性。(Muraoka Taro)
也适用于 Borland 5.5。(Dan Sharp)

Win32: 编辑快捷方式的文件 (*.lnk 文件) 时，编辑它指向的文件。除非置位了 'binary'，此时编辑快捷方式文件自身。(Yasuhiro Matsumoto)

":command" 命令现在接受 "-bar" 参数。使得用户命令可以被 "| command" 跟随。

现在，预览窗口也被以下命令使用:
- :pedit 在预览窗口编辑指定文件
- :psearch 类似于 :ijump ，在头文件里搜索单词，并在预览窗口显示找到的文本。
加入 CTRL-W P 命令: 转到预览窗口。

MS-DOS 和 MS-Windows 也读入系统范围的 vimrc 文件 $VIM/vimrc。多用于多用户的 NT 系统。

鼠标在字符上双击时，如果该字符有 "%" 匹配，选择从该字符到对应匹配的文本。类似于 "v%"。

"-S session.vim" 参数: 启动时执行脚本。便于 Vim 启动时载入会话文件。

加入 "--cmd {command}" Vim 参数，在 vimrc 文件载入之前执行命令。(Vince Negri)

加入 "-M" Vim 参数: 复位 'modifiable' 和 'write'，从而禁止对文件修改和写回。

加入 runtime/delmenu.vim。执行之可以删除所有菜单并为载入新菜单做好准备。可用于修改 'langmenu'。

Perl 脚本，用来过滤 Perl 错误信息并转化为 quickfix 可用的格式。(Joerg Ziefle)

加入 runtime/macros/less.vim: 模拟 less 的 Vim 脚本，但提供语法高亮。

MS-Windows 安装程序：(Jon Merz)
- Win32 程序现在可以在桌面上建立快捷方式，并在开始菜单安装 Vim。
- 可以删除旧的 "Edit with Vim" 项目。
- Vim 可执行文件无须移动或者改变 $PATH。在 $PATH 里建立了一个小的批处理文件。减少了要作出的选择。
- 检测已经安装的 Vim 的版本，并提供先卸载它们的选择。

改进的 MS-Windows 卸载程序。现在，它同时删除开始菜单的项目，桌面的图标和建立的批处理文件。(Jon Merz)
也可以只删除其中的一部分。同时撤销 gvim 的 OLE 注册。

生成 MS-Windows 上自安装的 Vim 包。用的是 NSIS。(Jon Merz 等人)

加入 ":filetype detect"。尝试再次检测文件类型。有助于（比如说）加入 "#!/bin/csh" 以后，编写的新外壳脚本。

加入 ":augroup! name" 命令，删除自动命令组。客户-服务器 "--remote-wait" 需要。

viminfo 文件里加入 Vim 的版本号，有助于调试。

改 进 improvements-6

'cpoptions' 加入 'n' 标志位：如果省略，回绕行的文本不放在 'number' 选项显示的行号中间。这样阅读回绕行容易多了。

如果标签文件有格式错误，报告其字节位置以定位错误。

"gf" 可用于可视模式：使用选择文本作为文件名。(Chase Tingley)

允许有二义性的映射。这样可以同时映射 "aa" 和 "aaa"，使用最长的匹配。这对 ":lmap" 和 'keymap' 特别有用。

加密：第一次加密时要求加密键输入两次。否则一旦输错，文本就永久丢失了。(Chase Tingley)

现在窗口标题在最后加上了 "VIM"。文件名在前面，这样任务栏上能够看清楚。如果文件修改过，加 "+"，如果文件只读，加 "="。如果文件 'modifiable' 关闭，加 "-"。

可视模式里，在 'showcmd' 的位置提及选择区域的大小。

加入 "b:changedtick" 变量。每次改变都递增，包括撤销。用于在缓冲区有改变时采取行动。

":s" 命令的替代字符串中，用 "\=" 可以替代为表达式的运算结果。该表达式内部可用 submatch() 函数访问子匹配。

如果执行 ":qall" 且其它窗口里正在编辑的缓冲区有修改，跳转到那个窗口，而不是编辑当前窗口的缓冲区。

文件读/写命令加入 "++enc=" 和 "++ff=" 参数，强制使用给定的 'encoding' 或 'fileformat'。同时加入 "v:cmdarg" 变量，用于 FileReadCmd 这些自己读写文件的自动命令。

读入标准输入时，先用二进制模式读入文本，然后再用基于 'fileformat' 和 'fileencoding' 的自动选择重新读入。这样避免了重新开始文件（例如，如果在靠近文件末的行以 LF 而不是 CR-LF 结束的话）。
如果从标准输入读取文本而缓冲区为空，不标记它为已改变。退出时省得麻烦。

给许多错误信息加入 ID，方便寻找消息的帮助。

插入模式：
- 用 "CTRL-G j" 和 "CTRL-G k" 可以在另一行的相同列插入。可用于编辑表格。
- 加入同义词补全，CTRL-X CTRL-T。(Vince Negri)
- 加入 'thesaurus' 选项，在同义词补全里代替 'dictionary'。'complete' 加入 's' 标志位。
- 插入模式的 CTRL-X CTRL-L 使用 'complete' 选项。也在其它载入的缓冲区里扫描匹配行。
- 插入模式用 CTRL-X 或 CTRL-N 补全时也可用 CTRL-R。(Neil Bird)
- 插入模式补全时，如果补全结束，检查来自 'cinkeys' 或 'indentkeys' 的单词匹配。

性能：
- 使得显示更新更有效率。所有改变都可能会使用插入/删除行，包括撤销/重做。
- 如果插入模式下有预输入，不刷新显示。大大加快了 CTRL-R。
- 为 32 位 DOS 版本改进了屏幕的输出速度。(Vince Negri)
- 鼠标拖动时，往前看鼠标代码，如果下面还有一个，跳过之前的鼠标键。这使得鼠标拖动快很多。
- 另一个内存用量的改进：如果调用 u_save 时只有一行且该行最近已经为相同的撤销保存过，不再保存。
- 使用一次附加一个字符的脚本时，过去分配的内存慢慢增加。'undolevels' 为 -1 也是如此。因为 "U" 保存的行从来不释放。现在撤销块为空时就释放内存。
- GUI 和 Dos32：使用垂直滚动区域，使得垂直分割的窗口滚动更快。不需要重画整个窗口。
- 如果终端代码不支持滚动（例如，用于垂直分割的窗口），从 ScreenLines[] 重画。应该比通过 win_line() 的行更快，使用语法高亮时尤其如此。
- 用单独的脚本预先生成 Syntax 菜单。使得菜单的载入加快了 70%，也使 gvim 的启动时间减半。
- 执行 ":help tag" 时，不先打开 help.txt，直接跳转到要帮助的标签上。更快也避免额外的信息。
- Win32：如果文件名不以 ".lnk" 结尾，不试图解析快捷方式。那需要不少时间。
- 如果有预输入的字符，不更新鼠标指针形状。
- META[] 从字符串改为数组，避免使用 strchr()。
- 加入字符时不清除命令行，避免调用 screen_fill 而不做任何事。

健壮性：
- Unix：执行正规表达式时检查堆栈是否溢出。避免很麻烦的溢出。只用于支持能在另一个堆栈上运行 signal 函数的系统。
- 不允许 ":source <dirname>"。Unix 上可以读取目录，但用它作为 Vim 命令没有任何意义。

安全：
- 从临时文件读或写时，检查它不是符号链接。提供一些针对符号链接攻击的保护。
- 建立备份文件或交换文件时，检查是否已经存在以避免符号链接攻击。(Colin Phipps)
- 在沙盘 sandbox 里计算表达式选项。如果用模式行设置选项，它不能造成破坏。
- 使用安全的方式生成临时文件名：建立一个私人目录用于临时文件。可用于 Unix、MS-DOS 和 OS/2。
- 'makeef' 可为空，意味着使用内部生成的文件名。旧的缺省值是 "/tmp/file"，造成

了安全风险。
在当前目录写入 'makeef' 会在只读目录失败，因而导致在所有文件上 ":grep" 会有问题。所有系统上的缺省都为空，以便使用临时文件。
- 标签文件的命令在沙盘里执行，以提供更好的安全性。
- 不能在沙盘里使用 Ruby、Tcl 和 Python 接口。它们可能执行危险的任务。Perl 可以，但只限于安全环境。(Donnie Smith)

语法高亮:
- 通过缓存整个文件的状态栈而不只是显示的部分优化速度。折叠需要如此。
- 加入 ":syntax sync fromstart": 总是从文件开始处进行分析。
- 为语法项目加入 "display" 参数: 只在显示结果时使用该项目。使得不用显示的文本的分析加快。
- 用 CTRL-L 时，删除缓存的状态以强制再次分析文本。
- 关键字表的存储使用 elfhash 算法。应该会产生更好的分布并加快关键字的查找。(Campbell)
- "lc" 引导上下文也可用于跳过和结束模式。(Scott Bigham)
- 语法项目可用 "extend" 参数来撤销它所在的项目的 "keepend" 参数的效果。它可以用来使一些包含的项目扩展一个区域，而另一些不能。
- ":syntax clear" 现在删除 b:current_syntax 变量。这合乎逻辑，因为此命令之后就没有定义任何语法了。
- 加入 ":syntax enable": 打开语法高亮而不改变颜色。可以在 .vimrc 文件里指定颜色，而不需要 mysyntaxfile。
- 加入 ":syntax reset": 复位色彩为其缺省值。
- 加入 "contains=TOP" 和 "contains=CONTAINED" 参数，可以定义不包含自身的透明项目。
- 给语法项目加入 "containedin" 参数。允许加入 contained 项目到已有的项目里 (例如，要高亮注释里的一个名字)。

无模式选择:
- 在命令行窗口里，为其它窗口使用无模式选择。这样可以复制可视文本到命令行窗口。
- 支持终端的命令行的无模式选择，以前只能用于 GUI。
- 无模式选择里双击右键选择整个单词。单击右键不使用双击左键开始的单词选择。使之和可视模式。
- 无模式选择不再隐含自动复制到剪贴板。现在它取决于 'guioptions' 的 'a' 和 'A' 标志位或者 'clipboard' 的 "autoselect" 和 "autoselectml"。
- 命令行模式加入 CTRL-Y 命令，复制无模式选择内容到剪贴板。也可用于 hit-enter 提示和 more 提示。
  删除 runtime/mswin.vim 中命令行模式里 CTRL-Y 和 CTRL-Z 的映射，以便使用新的 CTRL-Y 方式。

减少了 regmatch() 使用的堆栈空间的用量，可以在更长的文本上处理复杂的模式。

'isfname' 现在包含 '%' 和 '#'。使得 "vim dir\#file" 可用于 MS-DOS。

加入数字键盘的特殊键 <kEnter>、<k0> - <k9>，如果没有映射，和 ASCII 等价字符完全相同。(Ivan Wellesz 和 Vince Negri)
识别更多的一些 xterm 键: <C-Right>、<C-Left>、<C-End>、<C-Home>

Vim 将要退出时同时激活 BufUnload 事件。可能脚本需要进行一些处理。

扩展反引号里的表达式: =\{expr\} 。任何进行反引号扩展的地方都可用。(Vince Negri)

GUI:
- 'guioptions' 加入 'L' 和 'R' 标志位: 只有在垂直分割窗口存在时才加入左或右滚

动条。
- X11: 如果一个颜色不能分配，使用最接近的 colormap 的匹配。否则太多东西会使用黑色。(Monish Shah)
  也用于菜单和滚动条，否则它们都变成黑色了。
- Win32 和 X11: 加入 'mouseshape' 选项：根据当前模式调整鼠标指针形状。(Vince Negri)
- 加入 'linespace' 选项：行间插入像素行。(Nam)
- 允许按住 CTRL 和 SHIFT 来进入无模式选择（不用移动光标）。(Ivan Wellesz)
- Motif: 加入工具栏。(Gordon Prieur) 也加入工具提示。
- Athena: 加入工具栏和工具提示。(David Harrison -- 基于 Gordon Prieur 的工作)
- 'toolbar' 选项可用于 Athena 和 Motif。现在可以在线切换文本和图标。(David Harrison)
- 支持 Athena 的菜单分隔行。(David Harrison)
- Athena: 调整下拉菜单里的箭头位图，使之和字体的大小相同。(David Harrison)
- Win32: 'guifont' 加入 "c" 标志位以指定字符集。(Artem Khodush)
- 如果没有给出 --enable-xim 参数，使用 X GUI 时自动打开它。需要它来提供死键支持（和多字节支持）。
- 文件选择对话框之后，检查编辑文件没有被改变或删除。Win32 对话框允许删除和换名文件。
- Motif 和 Athena: 加入 "editres" 的支持。(Marcin Dalecki)
- Motif 和 Athena: 加入 "menuFont"，指定菜单的字体或字体集。也可以用 "Menu" 高亮组设置。用于 locale 和 'encoding' 不同的场合。(David Harrison)
  如果定义了 FONTSET_ALWAYS，菜单总是使用字体集。避免从字体切换到字体集时出现的问题。(David Harrison)
- 工具提示的高亮和字体可以用 "Tooltip" 高亮组指定。(David Harrison)
- more 提示里可用命令行模式菜单。这基本上都能工作，因为它们以 CTRL-C 启动。"Copy" 菜单可用于复制无模式的选择。允许不用自动选择而复制 ":set all" 或 ":intro" 的输出。
- 启动 GUI 时如果标准输出和标准错误没有连接到终端，在对话框里显示错误信息。过去彻底不显示。
- 允许设置 'browsedir' 为目录名，用于文件对话框。(Dan Sharp)
- 可以设置 b:browsefilter 和 g:browsefilter 为文件对话框使用的过滤。Win32 和 Motif GUI 支持。(Dan Sharp)

X11:
- 支持剪贴板的选择区作为寄存器 "+。退出或者暂停时，复制选择区到剪切缓冲区 0。这样可以用 X11 的标准方式复制/粘贴到许多应用程序。(Neil Bird)
- 在任何终端上使用 X 剪贴板，不只是 xterm。
  'clipboard' 加入 "exclude:": 指定模式，匹配的终端名不建立到 X 服务器的连接。缺省值目前可用于 FreeBSD 和 Linux 控制台。
- 如果 'verbose' 非零，加入一些消息显示连接到 X 服务器时发生了什么。如果要研究为什么启动很慢，应该有帮助。

GTK GUI: (部分由 Marcin Dalecki 提供)
- 一些字体的字符可能比 ascent + descent 还要高，例如 "-misc-fixed-*-*-*-*-18-*-*-*-*-*-iso10646-1"。给字符单元高度加一。
- 实现 'winaltkeys' 的 "no" 值：如果建立菜单后改变了 'wak'，不使用 Alt 键作为菜单快捷键。
- GUI 启动后可以设置 'wak'。
- 回收文本 GC 以减少通信。
- 调整窗口管理器的图标大小。
- 字体处理进行了清理。
- XQueryColor 被 GDK 调用替换。

- Gnome 支持。configure 里会检测 Gnome 并为此使用不同的窗口组件 (widget)。否则和 GTK 基本类似。(Andy Kahn)
  缺省关闭，因为会造成一些问题。
- 删除先分叉 (fork) 然后启动 GUI 用的特殊代码。现在用 _exit() 代替 exit()，这不需要任何特殊技巧。
- 对话框有时出现的位置远了些。现在把对话框定位在 gvim 窗口内部。
  (Brent Verner)
- 放置文件到 Vim 时，删除路径开始处额外的斜杠。如果可能，也缩短文件名。

Motif: (Marcin Dalecki)
- 使对话框的布局更好一点。
- 加入搜索和搜索/替代对话框。
- 菜单里把 "iso-8859" 换成 "iso_8859"。Linux 看来需要如此。
- 对话框里加入图标，就像 GTK 那样。
- 如有可能，图标使用 XPM 位图。如果存在，使用 Solaris 的 XpmP.h 头文件。
- 改变工具栏项目的阴影，使得按下时它的可视反馈不那么 Lesstif。
- 为了提高速度，一些项目使用 gadget 而不是 window。

命令行补全:
- 补全环境变量名。(Mike Steed)
- ":command" 加入一些补全方法: "mapping"、"function"、"expression" 和 "environment"。
- 如果函数不接受参数，补全加上 () 而不是 (。

MS-DOS、MS-Windows 和 OS/2 上: 扩展 %VAR% 环境变量，就像 $VAR 一样。
(Walter Briscoe)

重定向消息到剪贴板 ":redir @*" 和到无名寄存器 ":redir @"". (Wall)

":let @/ = ''" 清除搜索模式，而不是把它设为空字符串。

表达式计算:
- 可用 "? :"，就像 C 那样。
- col("$") 返回光标行的长度加一。(Stephen P. Wall)
- match()、matchend() 和 matchstr() 可选的附加参数: 开始寻找匹配的位移。
- 使 strpart() 第三个参数可选。(Paul Moore，Zdenek Sekera)
- exists() 也可用于检查 Ex 命令和自动命令的定义与否。
- input() 加入额外参数: 缺省文本。
- ":silinet! cmd" 同时设置 "v:errmsg"。
- 加入 v:prevcount 变量: 前一个命令的 v:count。
- 加入 "v:progname"，Vim 启动时所用的名字。(Vince Negri)
- 关于函数返回的 verbose 消息也显示返回值。

Cscope:
- 加入 cscope_connection() 函数。(Andy Kahn)
- ":cscope kill -1" 杀死所有的 cscope 连接。(Andy Kahn)
- 加入 'cscopepathcomp' 选项。(Scott Hauck)
- 加入 ":scscope" 命令，分割窗口并执行 Cscope 命令。(Jason Duell)

VMS:
- 命令行参数总是大写的。"-X" 参数被理解为 "-x" 而 "-/X" 则成为 "-X"。
- 使得 'makeprg' 和 'grepprg' 的缺省值有意义。(Zoltan Arpadffy)
- 使用 X 剪贴板特性和 X 命令服务器。(Zoltan Arpadffy)

Macintosh: (Dany St-Amant)
- 允许标签文件使用 CR、CR-LF 或 LF 作为换行符。(Axel Kielhorn)
- 代码改用 Carbon 环境 (同时保留非 Carbon 的代码)
  (一些工作是从 Ammon Skidmore 那里 "偷来的")
- 改进了菜单项目索引的处理 (应该更快了)
- 运行时命令现在能处理文件名里的 / (MacOS 9 版本)
- 加入 ":winpos" 支持。
- 支持在文件里用 "~" 代表主目录。

选项:
- 用 set += 或 ^= 时，检查使用两次的项目。删除重复项目。(Vince Negri)
- 设置标志位列表的选项时，删除重复的标志位。
- 如果可能，使用 getrlimit() 来设置 'maxmemtot' 和 'maxmem'。(Pina)
- 'nrformats' 加入 "alpha": 用 CTRL-A 和 CTRL-X 可以递增或递减字母表字符。
- ":set opt&vi" 设置选项为 Vi 缺省值。":set opt&vim" 则设为 Vim 缺省值。可用于只想设置 'cpo' 为 Vim 缺省值而不想知道其中包含什么标志位的场合。
- 'scrolloff' 现在也适用于长的窗口放不下的回绕行。
- 缺省菜单加入更多选项设置。
- 更新选项窗口以包含新的选项。使它更易阅读。

内部改变:
- 把行指针分成文本部分和属性部分。这样将来可以方便修改，属性可以多于一个字节。
- 如果系统没有的话，提供 qsort() 函数。
- 把一个用于普通模式的大的 switch 换成一个表。这使代码清晰很多，也避免了一些编译器的优化出现的麻烦。
- 赋给特殊键一个负值，避免它们和 Unicode 字符混淆起来。
- 全局变量 expand_context 和 expand_pattern 本来不打算是全局的。把它们传给 ExpandOne() 和所有它调用的函数。
- 不再使用全局的 reg_ic 标志位。它造成了麻烦，而且一些地方它也没有设置。
- 删除用于 "*"、"K"、CTRL-] 等的 stuff 缓冲区的使用。避免自动命令的问题。
- 一些代码从 ex_docmd.c 移到了 ex_cmds2.c。该文件太大了。一些代码也从 screen.c 移到了 move.c。
- 不直接包含加密用的 CRC 表而是生成它。这样减少了不少源代码的空间。
  (Matthias Kramm)
- multibyte.c 换名为 mbyte.c，避免 8.3 文件系统的问题。
- 删除 ":findhelp" 的 GTK 实现，现在使用 ToolBar.FindHelp 菜单项。
- 换名 mch_windexit() 为 mch_exit()，mch_init() 为 mch_early_init()，还有 mch_shellinit() 为 mch_init()。

高亮:
- ":highlight" 列表中，用高亮自己的颜色显示 "xxx"。
- 加入 xterm 88 色或 256 色的支持。正确的高亮数目被用作 ":highlight" 命令的名字。 (Steve Wall)
- ":highlight" 加入 "default" 参数。如果包含且该高亮组已经定义，忽略此命令。所有的语法文件现在使用 ":hi default ...", 这样用户可以在自己的 vimrc 文件里指定颜色。另外，不再需要 "if did_xxx_syntax_inits"。这大大简化了为特定语言使用非缺省颜色的工作。
- 调整了 colortest.vim: 包含正常背景的颜色，还有用 while 循环减少了代码。
  (Rafael Garcia-Suarez)
- 加入 "DarkYellow" 颜色名，使标准颜色的清单更一致。虽然它本身并不那么好看。

如果 xterm 处于 8 位模式，用 t_RV 返回的编码来检测这种情况。所有的键码都被自动转换为它们的 8 位版本。

用 xterm patch level 141 和之后版本的 OPT_TCAP_QUERY 获得实际使用的键码和 t_Co 中颜色的编码。只有同时使用 t_RV 时才会如此。

":browse set" 现在也可用于控制台模式。":browse edit" 则会给出错误信息。

":bdelete" 和 ":bunload" 只报告多于 'report' 的删除/卸载缓冲区数目。如果在脚本里删除缓冲区，这个消息的出现会相当讨厌。

跳转表:
- 跳转表保存的位置标记的数目从 50 个增加到 100 个。
- viminfo 文件现在保存跳转表。CTRL-O 可以用来跳转到前一个编辑会话的位置。
- 执行 ":split" 时把跳转表复制到新窗口里。

"~" 和 "r" 命令同时设置 '[和 '] 位置标记。现在用普通模式命令进行改变时，总是设置这些位置标记。

Python 接口: 允许设置垂直分割窗口的宽度。(John Cook)

'cinkeys' 加入 "=word" 和 "=~word" (也用于 'indentkeys')。

'cinoptions' 加入 "j1" 参数: 缩进 Java () 里的 {}。(Johannes Zellner)
'cinoptions' 加入 "l" 标志位。(Anduin Withers)
'cinoptions' 加入 'C'、'U'、'w' 和 'm' 标志位。(Servatius Brandt)

执行 ":wall" 或 ":wqall" 而修改过的缓冲区没有名字时，在错误信息里提到缓冲区的编号。

":function Name" 列出函数和行号。容易发现错误在何处。

非面向列块的可视模式里，"r" 替换所有选择的字符为输入的单个字符，和面向列块的可视模式相同。

用任何方式编辑参数列表中的最后一个文件就允许退出。以前用 ":next" 或 ":last" 访问该文件才可以。

'formatoptions' 加入 '1' 标志位。(Vit Stradal)
'formatoptions' 加入 'n' 标志位: 排版编号列表。

交换文件:
- 如果交换文件已经存在并且用户在 ATTENTION 提示时选择了 "Delete"，使用相同的 ".swp" 交换文件以避免建立 ".swo" 文件，后者不一定总能找到。
- 如果给出 ATTENTION 消息并且文件的日期比交换文件的日期更新，给出警告。
- 简化关于已有的交换文件的消息，使之能在 24 行的屏幕上显示。
- 原来可以建立和交换文件同名且到不存在的文件的符号链接。Vim 这时会安静的使用另一个文件 (如果用 O_EXCL 打开文件时拒绝符号链接的话) 现在检查符号链接是否指向存在的文件。也在建立之前检查交换文件是否已经存在以发现符号链接攻击。

g CTRL-G 命令也可用于可视模式并计算单词的个数。 (Chase Tingley)

如果使用 'shell' 但为空，报错。

加入 'shellpipe' 包含 "%s" 的可能性。

'display' 加入 "uhex" 值: 用 <xx> 形式显示不可显示字符。不可显示字符使用

NonText 高亮，命令行上也是如此。

要求显示隐藏选项的值时，说明不支持这么做。

Win32:
- 把快捷方式放置在 gvim (.lnk file) 之上会编辑目标文件，而不是快捷方式本身。(Yasuhiro Matsumoto)
- 加入 C 版本的 OpenWithVim 和 SendToVim 程序。(Walter Briscoe)
- 如果 'shell' 是 "cmd" 或 "cmd.exe"，设置 'shellredir' 使之同时重定向标准错误 (stderr)。也检查 Unix 的外壳名。
- 如果定义 $HOMEDRIVE 和 $HOMEPATH，用它们来定义 $HOME。(Craig Barkhouse)

Win32 控制台版本:
- ":version" 消息，如有可能，包含用户名和系统名。为此，生成 pathdef.c 文件。(Jon Miner)
- 设置窗口图标为 Vim 的图标 (只用于 Windows 2000)。执行外壳命令时，修改窗口标题以显示这一点。退出时也恢复光标位置。(Craig Barkhouse)
- Win32 控制台版本可以在编译时加入 OLE 支持，但只会作为客户端而非 OLE 服务器。

错误格式:
- 'errorformat' 的 "%p" (一列字符指示的错误列) 也能接受一行句号。
- 'errorformat' 加入 "%v" 项目: 虚拟列号。(Dan Sharp)
- 'errorformat' 加入 VMS 的缺省值。(Jim Bush)

"p" 命令现在可用于可视模式。它用寄存器的内容覆盖选择文本。

介绍消息里高亮 <> 项目，明确它们是特殊的。

使用 ":substitute" 的 "c" 标志位时，允许输入 "l" 来替换项目并停止: l 代表 "last (最后)"。

执行其它文件时显示 verbose 消息时，显示行号。

改变 Vim 窗口大小时，不使用 'equalalways'。避免 Vim 窗口变小时分割窗口变大。文档是这么说的。

插入模式输入 CTRL-D 时，如果刚在自动缩进之后，输入 CR 过去保持余下的空白。现在和 BS 类似: 删除自动缩进，避免产生空白 (但非空) 的行。

OLE 接口加入 GetHwnd() 调用。(Vince Negri)

使得 ":normal" 可用于事件处理程序。用于把文件放置到 Vim 中和 CursorHold 自动命令。

MS-Windows 版本上，如果使用斜杠而不是反斜杠，不改变到文件的目录。探索器总是用反斜杠，而用户可以在输入命令时使用斜杠。

修改时间 (时间戳):
- 如果缓冲区在 Vim 的外部被改变并重新获得焦点，给出对话框，用于用户重新载入文件。现在也适用于 MS-Windows 之外的其它 GUI，还有编译时加入对话框支持的控制台。
- 检查文件内容以发现它是否真的改变了，忽略只有时间戳本身被修改的情况 (例如，从 CVS check out 文件)。
- 检查时间戳时，先检查文件大小是否改变，如果是就不用进行文件比较了。不断附加的

大（日志）文件会快很多。
- 如果设置 'buftype'，不给出改变或删除文件的警告。
- 不再警告目录的改变，否则文件探索器会产生警告。
- 只对不隐藏的缓冲区检查时间戳。否则，等它们不隐藏时才检查。
- 检查文件是否在 Vim 之外改变时，同时检查文件权限是否改变。如果文件内容不改变但权限改变了，给出警告。
- 避免过于频繁的检查，不然对于不断增长的文件，对话框会不断弹出来。

如果 'encoding' 为 "latin1" 映射 <M-A> 然后把 'encoding' 改为 "utf-8" 会使得多字节的第一个字节被映射。这个问题很难发现。现在禁止映射多字节字符的一部分。

":python" 和 ":tcl" 接受内含脚本。(Johannes Zellner)
也适用于 ":ruby" 和 ":perl"。(Benoit Cerrina)

如果文件名不是完整路径，":syn include" 使用 'runtimepath'。

如果 'switchbuf' 包含 "split" 而当前窗口为空，不分割窗口。

Unix: 捕获 SIGPWR，电源将要关闭时保存文件。

Sniff 接口: (Anton Leherbauer)
- 修正窗口代码，尤其是事件处理部分
- 更新到 sniff 4.x ($SNIFF_DIR4)
- 支持运行时加入 sniff 请求

支持 <A-x> 记法，作为 <M-x> 的别名。这合乎逻辑，因为使用了 Alt 键。

":find" 接受计数，意味着使用 'path' 的第 count 个匹配。

":ls" 和 ":buffers" 输出显示 modified/readonly/modifiable 标志位。如果缓冲区激活，显示 "a" 而不是什么都不显示。如果缓冲区未载入，什么都不显示而不是 "-"。

Unix 安装:
- 安装 tools 时，使用 tools 脚本 efm_perl.pl 和 mve.awk 的绝对路径。这样用户就不用编辑这些文件。
- KDE 在目录存在但图标还不存在时安装图标。

加入 has("win95")，在 Vim 脚本里用于区别 MS-Windows 95/98/ME 和 NT/2000/XP。

输入 ":cd" 命令时，回显新的当前目录。(Dan Sharp)

如果在 GUI 窗口打开前使用 ":winpos"，先记住其值，打开后才使用。

":version" 的输出里，加入 "/dyn" 代表特性能动态载入。这也同时意味着该特性不一定总能用。

Windows NT 上目录可以只读但允许删除文件。用换名文件建立备份而 'backupdir' 不用当前目录时，原先的文件被被删除，但新文件却不能建立。现在给出额外的错误信息，警告用户这一情况。

命令行上的 CTRL-R CTRL-O 和 CTRL-R CTRL-R 类似，和插入模式的行为一致。

所有生成的文件现在从 "src" 目录移出。这使得观察哪些文件是或不是手动编辑更为容易。configure 生成的文件现在放在 "src/auto" 目录。Unix 上，经过编译的目标文件放在 objects 目录里。

源代码的归档已经超过了 1.4M 软盘的大小。现在把归档分割为两个运行时归档和两个源代码归档。另外提供 bzip2 压缩归档，它包含所有的源程序和运行时文件。

make 增加了 "reconfig" 目标。在修改某些需要刷新缓存的参数时有用。比如从 GTK 到 Motif 的切换。调整 GUI_INC_LOC 和 GUI_LIB_LOC 的含义，使得不同 GUI 之间能够一致。

加入 src/README.txt，提供源代码主要部分的总览。

Unix 的 Makefile 现在完全支持使用 $(DESTDIR) 来安装到特定的位置。替换手动的 *ENDLOC 变量设置。

configure 加入 --with-compiledby 参数，提供二进制版本的维护者加入 e-mail 地址的可能性。

包含的特性现在使用如下的特性包 "tiny"、"small"、"normal"、"big" 和 "huge"。这替代了以前的 "min-features" 和 "max-features"。要得到一个真的很小的 Vim，可以使用 "tiny"，它关闭了多重窗口。

使用 tiny 版本或者如果没有定义 FEAT_WINDOWS: Firstwin 和 lastwin 等于 curwin，而且不使用 w_next 和 w_prev。

加入 +listcmds 特性。可以用来使得编译时没有操作缓冲区列表和参数列表的 Vim 命令(缓冲区列表自己还是在的，不能没有它)。

加入 +vreplace 特性。在 "small" 版本里，它被关闭，以免 16 位的 DOS 版本内存占用过多。

删除了 1.6.16 版本之前的 GTK+ 支持。

configure 对是否可以使用 PTY 的检查有了改进。代码来自 screen 的新近版本。

configure 加入安装 Vim、Ex 和 View 为其它名字的选项 (比如，vim6、ex6 和 view6)。

configure 加入 "--with-global-runtime" 参数。可以指定 'runtimepath' 使用的缺省全局目录。

可以用 configure 参数打开 SNiFF+ 接口。

configure 现在总是在 /usr/local/lib 里检查库，还有在 /usr/local/include 里检查头文件。这有助于找到 iconv() 和 gettext()。

把命令行历史的代码移到 +cmdline_hist 特性里，以从 tiny 版本里去掉命令行历史。

MS-Windows: Win16 和 Win32 公用的函数移到了 os_mswin.c。避免为同一问题修改两个文件。(Vince Negri)

把 gui_w16.c 和 gui_w32.c 的公共代码移到 gui_w48.c (Vince Negri)

现在，跳转表是一个单独的特性。"small" 版本关闭此特性 (16 位 MS-DOS)。

所有 _t 结尾的类型换名为 _T 结尾。避免潜在的和系统类型冲突的问题。

configure 加入隐含定义返回类型为 int 的 X11 头文件的检查。(Steve Wall)

顶层目录的 "make doslang" 建立 Windows 的菜单和 .mo 文件的归档。它使用 Unix 上产生的文件，这些文件应该在 MS-Windows 上也能用。

os_vms.c 和 os_unix.c 有很大一部分进行了合并。过去，重复的代码使得维护需要更多工作。(Zoltan Arpadffy)

Borland C 版本 5 Makefile 的改进: (Dan Sharp)
- 修正 Perl 支持
- 加入 python 和 tcl 支持
- 加入动态的 perl 和动态的 python 支持
- 加入 uninstal.exe 支持
- 选项使用 "yes" 和 "no"，类似于 Make_mvc.mak。

Win32: 合并 Make_gvc.mak 和 Make_ovc.mak 为一个文件: Make_ivc.mak。它更小，许多不必要的文本被删除了。(Walter Briscoe)
增加 Make_dvc.mak，允许调试 MS-Devstudio 的 Make_mvc.mak 生成的 exe。
(Walter Briscoe)

MS-Windows: big 版本的 gvim.exe，包括 OLE，现在也包含了动态载入的 Tcl、Perl 和 Python。它使用 ActivePerl 5.6.1、ActivePython 2.1.1 和 ActiveTCL 8.3.3

configure.in 里加入 AC_EXEEXT，在 Cygwin 或 MingW 下检查可执行文件是否需要 ".exe"，以便在 Makefile 里把 SUFFIX 换名为 EXEEXT。

Win32: 延迟载入 comdlg32.dll 以加快启动。只用于 VC 6。(Vipin Aravind)

Win32: 如果用 Borland 编译，允许使用 IME。(Yasuhiro Matsumoto)

Win32: 加入 Borland 5 编译 gvimext.dll 的 Makefile。(Yasuhiro Matsumoto)

漏 洞 修 正 bug-fixes-6

When checking the command name for "gvim", "ex", etc. ignore case. Required for systems where case is ignored in command names.

Search pattern "[a-c-e]" also matched a 'd' and didn't match a '-'.

When double-clicking in another window, wasn't recognized as double click, because topline is different. Added set_mouse_topline().

The BROKEN_LOCALE check was broken. (Marcin Dalecki)

When "t_Co" is set, the default colors remain the same, thus wrong. Reset the colors after changing "t_Co". (Steve Wall)

When exiting with ":wqall" the messages about writing files could overwrite

each other and be lost forever.

When starting Vim with an extremely long file name (around 1024 characters) it would crash. Added a few checks to avoid buffer overflows.

CTRL-E could get stuck in a file with very long lines.

":au syntax<Tab>" expanded event names while it should expand groups starting with "syntax".

When expanding a file name caused an error (e.g., for <amatch>) it was produced even when inside an "if 0".

'cindent' formatted C comments differently from what the 'comments' option specified. (Steve Wall)

Default for 'grepprg' didn't include the file name when only grepping in one file. Now /dev/null has been added for Unix.

Opening the option window twice caused trouble. Now the cursor goes to the existing option window.

":sview" and ":view" didn't set 'readonly' for an existing buffer. Now do set 'readonly', unless the buffer is also edited in another window.

GTK GUI: When 'guioptions' excluded 'g', the more prompt caused the toolbar and menubar to disappear and resize the window (which clears the text).
Now always grey-out the toplevel menus to avoid that the menubar changes size or disappears.

When re-using the current buffer for a new buffer, buffer-local variables were not deleted.

GUI: when 'scrolloff' is 0 dragging the mouse above the window didn't cause a down scroll. Now pass on a mouse event with mouse_row set to -1.

Win32: Console version didn't work on telnet, because of switching between two console screens. Now use one console screen and save/restore the contents when needed. (Craig Barkhouse)

When reading a file the magic number for encryption was included in the file length. (Antonio Colombo)

The quickfix window contained leading whitespace and NULs for multi-line messages. (David Harrison)

When using cscope, redundant tags were removed. This caused a numbering problem, because they were all listed. Don't remove redundant cscope tags. (David Bustos).

Cscope: Test for which matches are in the current buffer sometimes failed, causing a jump to another match than selected. (David Bustos)

Win32: Buffer overflow when adding a charset name in a font.

'titlestring' and 'iconstring' were evaluating an expression in the current context, which could be a user function, which is a problem for local variables vs global variables.

Win32 GUI: Mapping <M-F> didn't work. Now handle SHIFT and CTRL in _OnSysChar().

Win32 GUI: (on no file), :vs<CR>:q<CR> left a trail of pixels down the middle. Could also happen for the ruler. screen_puts() didn't clear the right char in ScreenLines[] for the bold trick.

Win32: ":%!sort|uniq" didn't work, because the input file name touches the "|". Insert a space before the "|".

OS/2: Expanding wildcards included non-existing files. Caused ":runtime" to fail, which caused syntax highlighting to fail.

Pasting a register containing CTRL-R on the command line could cause an endless loop that can't be interrupted. Now it can be stopped with CTRL-C.

When 'verbose' is set, a message for file read/write could overwrite the previous message.
When 'verbose' is set, the header from ":select" was put after the last message. Now start a new line.

The hit-enter prompt reacted to the response of the t_RV string, causing messages at startup to disappear.

When t_Co was set to 1, colors were still used. Now only use color when t_Co > 1.

Listing functions with ":function" didn't quit when 'q' or ':' was typed at the more prompt.

Use mkstemp() instead of mktemp() when it's available, avoids a warning for linking on FreeBSD.

When doing Insert mode completion it's possible that b_sfname is NULL. Don't give it to printf() for the "Scanning" message.

":set runtimepath-=$VIMRUNTIME" didn't work, because expansion of wildcards was done after trying to remove the string. Now for ":set opt+=val" and ":set opt-=val" the expansion of wildcards is done before adding or removing "val".

Using CTRL-V with the "r" command with a blockwise Visual selection inserted a CTRL-V instead of getting a special character.

Unix: Changed the order of libraries: Put -lXdmcp after -lX11 and -lSM -lICE after -lXdmcp. Should fix link problem on HP-UX 10.20.

Don't remove the last "-lm" from the link line. Vim may link but fail later when the GUI starts.

When the shell returns with an error when trying to expand wildcards, do include the pattern when the "EW_NOTFOUND" flag was set.
When expanding wildcards with the shell fails, give a clear error message instead of just "1 returned".

Selecting a Visual block, with the start partly on a Tab, deleting it leaves the cursor too far to the left. Causes "s" to work in the wrong position.

Pound sign in normal.c caused trouble on some compilers. Use 0xA3 instead.

Warning for changing a read-only file wasn't given when 'insertmode' was set.

Win32: When 'shellxquote' is set to a double quote (e.g., using csh), ":!start notepad file" doesn't work. Remove the double quotes added by 'shellxquote' when using ":!start". (Pavol Juhas)

The "<f-args>" argument of ":command" didn't accept Tabs for white space. Also, don't add an empty argument when there are trailing blanks.

":e test\\je" edited "test\je", but ":next test\\je" edited "testje". Backslashes were removed one time too many for ":next".

VMS: "gf" didn't work properly. Use vms_fixfilename() to translate the file name. (Zoltan Arpadffy)

After ":hi Normal ctermbg=black ctermfg=white" and suspending Vim not all characters are redrawn with the right background.

When doing "make test" without +eval or +windows feature, many tests failed. Now have test1 generate a script to copy the correct output, so that a test that doesn't work is skipped.

On FreeBSD the Perl interface added "-lc" to the link command and Python added "-pthread". These two don't work together, because the libc_r library should be used. Removed "-lc" from Perl, it should not be needed.
Also: Add "-pthread" to $LIBS, so that the checks for functions is done with libc_r. Sigaltstack() appears to be missing from libc_r.

The Syntax sub-menus were getting too long, reorganized them and added another level for some languages.

Visual block "r"eplace didn't work well when a Tab is partly included. (Matthias Kramm)

When yanking a Visual block, where some lines end halfway the block, putting the text somewhere else doesn't insert a block. Padd with spaces for missing characters. Added "y_width" to struct yankreg. (Matthias Kramm)

If a substitute string has a multibyte character after a backslash only the first byte of it was skipped. (Muraoka Taro)

Win32: Numeric keypad keys were missing from the builtin termcap entry.

When a file was read-only ":wa!" didn't force it to be written. (Vince Negri)

Amiga: A file name starting with a colon was considered absolute but it isn't.
Amiga: ":pwd" added a slash when in the root of a drive.

Don't let 'ttymouse' default to "dec" when compiled with dec mouse support.
It breaks the gpm mouse (Linux console).

The prototypes for the Perl interface didn't work for threaded Perl. Added a sed command to remove the prototypes from proto/if_perl.pro and added them manually to if_perl.xs.

When ":w!" resets the 'readonly' option the title and status lines were not updated.

":args" showed the current file when the argument list was empty. Made this work like Vi: display nothing.

"99:<C-U>echo v:count" echoed "99" in Normal mode, but 0 in Visual mode.
Don't set v:count when executing a stuffed command.

Amiga: Got a requester for "home:" because it's in the default runtime path.
Don't bring up a requester when searching for a file in 'path', sourcing the .vimrc file or using ":runtime".

Win16 and Win32: Considered a file "\path\file" absolute. Can cause the same file to appear as two different buffers.

Win32: Renaming a file to an empty string crashed Vim. Happened when using explorer.vim and hitting ESC at the rename prompt.

Win32: strftime() crashed when called with a "-1" value for the time.

Win32 with Borland compiler: mch_FullName() didn't work, caused tag file not to be found.

Cscope sometimes jumped to the wrong tag. (David Bustos)

OS/2: Could not find the tags file. mch_expand_wildcards() added another slash to a directory name.

When using ">>" the `] mark was not in the last column.

When Vim was compiled without menu support, filetype.vim was still trying to source the menu.vim script. (Rafael Garcia-Suarez)

":ptag" added an item to the tag stack.

Win32 IME: "gr" didn't use IME mode.

In the "vim --help" message the term "options" was used for arguments. That's confusing, call them "arguments".

When there are two windows, and a BufUnload autocommand for closing window #1 closed window #2, Vim would crash.

When there is a preview window and only one other window, ":q" wouldn't exit.

In Insert mode, when cancelling a digraph with ESC, the '?' wasn't removed.

On Unix glob(".*") returned "." and "..", on Windows it didn't. On Windows glob("*") also returned files starting with a dot. Made this work like Unix on all systems.

Win32: Removed old code to open a console. Vimrun is now used and works fine.

Compute the room needed by the intro message accurately, so that it also fits on a 25 line console. (Craig Barkhouse)

":ptnext" was broken. Now remember the last tag used in the preview window separately from the tagstack.

Didn't check for "-display" being the last argument. (Wichert Akkerman)

GTK GUI: When starting "gvim" under some conditions there would be an X error. Don't replace the error handler when creating the xterm clipboard. (Wichert Akkerman)

Adding a space after a help tag caused the tag not to be found. E.g., ":he autoindent ".

Was trying to expand a URL into a full path name. On Windows this resulted in the current directory to be prepended to the URL. Added vim_isAbsName() and vim_FullName() to avoid that various machine specific functions do it differently.

":n *.c" ":cd .." ":n" didn't use the original directory of the file. Vi only does it for the current file (looks like a bug). Now remember the buffer used for the entry in the argument list and use its name (adjusted when doing ":cd"), unless it's deleted.

When inserting a special key as its name ("<F8>" as four characters) after moving around in Insert mode, undo didn't work properly.

Motif GUI: When using the right mouse button, for some people gvim froze for a couple of seconds (Motif 1.2?). This doesn't happen when there is no Popup menu. Solved by only creating a popup menu when 'mousemodel' is "popup" or "popup_setpos". (David Harrison)

Motif: When adding many menu items, the "Help" menu disappeared but the menubar didn't wrap. Now manually set the menubar height.

When using <BS> in Insert mode to remove a line break, or using "J" to join lines, the cursor could end up halfway a multi-byte character. (Muraoka Taro)

Removed defining SVR4 in configure. It causes problems for some X header files and doesn't appear to be used anywhere.

When 'wildignore' is used, 'ignorecase' for a tag match was not working.

When 'wildignore' contains "*~" it was impossible to edit a file ending in a "~". Now don't recognize a file ending in "~" as containing wildcards.

Disabled the mouse code for OS/2. It was not really used.

":mksession" always used the full path name for a buffer, also when the short name could be used.
":mkvimrc" and ":mksession" didn't save 'wildchar' and 'pastetoggle' in such a way that they would be restored. Now use the key name if possible, this is portable.

After recovering a file and abandoning it, an ":edit" command didn't give the ATTENTION prompt again. Would be useful to be able to delete the file in an easy way. Reset the BF_RECOVERED flag when unloading the buffer.

histdel() could match or ignore case, depending on what happened before it. Now always match case.

When a window size was specified when splitting a window, it would still get the size from 'winheight' or 'winwidth' if it's larger.

When using "append" or "insert" inside a function definition, a line starting with "function" or "endfunction" caused confusion. Now recognize the commands and skip lines until a ".".

At the end of any function or sourced file need_wait_return could be reset, causing messages to disappear when redrawing.

When in a while loop the line number for error messages stayed fixed. Now the line number is remembered in the while loop.

"cd c:/" didn't work on MS-DOS. mch_isdir() removed a trailing slash.

MS-Windows: getftime() didn't work when a directory had a trailing slash or backslash. Didn't show the time in the explorer because of this.

When doing wildcard completion, a directory "a/" sorted after "a-b". Now recognize path separators when sorting files.

Non-Unix systems: When editing "c:/dir/../file" and "c:/file" they were created as different buffers, although it's the same file. Expand to a full file name also when an absolute name contains "..".

"g&" didn't repeat the last substitute properly.

When 'clipboard' was set to "unnamed", a "Y" command would not write to "0. Now make a copy of register 0 to the clipboard register.

When the search pattern matches in many ways, it could not always be interrupted with a CTRL-C. And CTRL-C would have to be hit once for every line when 'hlsearch' is on.
When 'incsearch' is on and interrupting the search for a match, don't abandon the command line.

When turning a directory name into a full path, e.g., with fnamemodify(), sometimes a slash was added. Make this consistent: Don't add a slash.

When a file name contains a "!", using it in a shell command will cause trouble: ":!cat %". Escape the "!" to avoid that. Escape it another time when 'shell' contains "sh".

Completing a file name that has a tail that starts with a "~" didn't work: ":e view/~<Tab>".

Using a ":command" argument that contains < and > but not for a special argument was not skipped properly.

The DOS install program: On Win2000 the check for a vim.exe or gvim.exe in $PATH didn't work, it always found it in the current directory.
Rename the vim.exe in the current dir to avoid this. (Walter Briscoe)

In the MS-DOS/Windows install program, use %VIM% instead of an absolute path, so that moving Vim requires only one change in the batch file.

Mac: mch_FullName() changed the "fname" argument and didn't always initialize the buffer.

MS-DOS: mch_FullName() didn't fix forward/backward slashes in an absolute file name.

"echo expand("%:p:h")" with an empty file name removed one directory name on MS-DOS. For Unix, when the file name is a directory, the directory name was removed. Now make it consistent: "%:p" adds a path separator for all systems, but no path separator is added in other situations.

Unix: When checking for a CTRL-C (could happen any time) and there is an X event (e.g., clipboard updated) and there is typeahead, Vim would hang until a character was typed.

MS-DOS, MS-Windows and Amiga: expanding "$ENV/foo" when $ENV ends in a colon, had the slash removed.

":he \^=" gave an error for using _. ":he ^=" didn't find tag :set^=. Even "he :set^=" didn't find it.

A tags file name "D:/tags" was used as file "tags" in "D:". That doesn't work when the current path for D: isn't the root of the drive.

Removed calls to XtInitializeWidgetClass(), they shouldn't be necessary.

When using a dtterm or various other color terminals, and the Normal group has been set to use a different background color, the background wouldn't always be displayed with that color. Added check for "ut" termcap entry: If it's missing, clearing the screen won't give us the current background color. Need to draw each character instead. Vim now also works when the "cl" (clear screen) termcap entry is missing.

When repeating a "/" search command with a line offset, the "n" did use the offset but didn't make the motion linewise. Made "d/pat/+2" and "dn" do the same.

Win32: Trying to use ":tearoff" for a menu that doesn't exist caused a crash.

OpenPTY() didn't work on Sequent. Add a configure check for getpseudotty().

C-indenting: Indented a line starting with ")" with the matching "(", but not a line starting with "x)" looks strange. Also compute the indent for aligning with items inside the () and use the lowest indent.

MS-DOS and Windows: ":n *.vim" also matched files ending in "~".
Moved mch_expandpath() from os_win16.c and os_msdos.c to misc1.c, they are equal.

Macintosh: (Dany St-Amant)
- In Vi-compatible mode didn't read files with CR line separators.
- Fixed a bug in the handling of Activate/Deactivate Event
- Fixed a bug in gui_mch_dialog (using wrong pointer)

Multibyte GDK XIM: While composing a multibyte-word, if user presses a mouse button, then the word is removed. It should remain and composing end. (Sung-Hyun Nam)

MS-DOS, MS-Windows and OS/2: When reading from stdin, automatic CR-LF conversion by the C library got in the way of detecting a "dos" 'fileformat'.

When 'smartcase' is set, patterns with "\S" would also make 'ignorecase' reset.

When clicking the mouse in a column larger than 222, it moved to the first column. Can't encode a larger number in a character. Now limit the number to 222, don't jump back to the first column.

GUI: In some versions CSI would cause trouble, either when typed directly or when part of a multi-byte sequence.

When using multibyte characters in a ":normal" command, a trailing byte that is CSI or K_SPECIAL caused problems.

Wildmenu didn't handle multi-byte characters.

":sleep 10" could not be interrupted on Windows, while "gs" could. Made them both work the same.

Unix: When waiting for a character is interrupted by an X-windows event (e.g., to obtain the contents of the selection), the wait time would not be honored. A message could be overwritten quickly. Now compute the remaining waiting time.

Windows: Completing "\\share\c$\S" inserted a backslash before the $ and then the name is invalid. Don't insert the backslash.

When doing an auto-write before ":make", IObuff was overwritten and the wrong text displayed later.

On the Mac the directories "c:/tmp" and "c:/temp" were used in the defaults for 'backupdir' and 'directory', they don't exist.

The check for a new file not to be on an MS-DOS filesystem created the file temporarily, which can be slow. Don't do this if there is another check for the swap file being on an MS-DOS filesystem.

Don't give the "Changing a readonly file" warning when reading from stdin.

When using the "Save As" menu entry and not entering a file name, would get an error message for the trailing ":edit #". Now only do that when the alternate file name was changed.

When Vim owns the X11 selection and is being suspended, an application that tries to use the selection hangs. When Vim continues it could no longer obtain the selection. Now give up the selection when suspending.

option.h and globals.h were included in some files, while they were already included in vim.h. Moved the definition of EXTERN to vim.h to avoid doing it twice.

When repeating an operator that used a search pattern and the search pattern contained characters that have a special meaning on the cmdline (e.g., CTRL-U) it didn't work.

Fixed various problems with using K_SPECIAL (0x80) and CSI (0x9b) as a byte in a (multibyte) character. For example, the "r" command could not be repeated.

The DOS/Windows install program didn't always work from a directory with a long filename, because $VIM and the executable name would not have the same path.

Multi-byte:
- Using an any-but character range [^x] in a regexp didn't work for UTF-8. (Muraoka Taro)
- When backspacing over inserted characters in Replace mode multi-byte characters were not handled correctly. (Muraoka Taro)
- Search commands "#" and "*" didn't work with multibyte characters. (Muraoka Taro)
- Word completion in Insert mode didn't work with multibyte characters. (Muraoka Taro)
- Athena/Motif GUI: when 'linespace' is non-zero the cursor would be drawn too wide (number of bytes instead of cell width).
- When changing 'encoding' to "euc-jp" and inserting a character Vim would crash.
- For euc-jp characters positioning the cursor would sometimes be wrong. Also, with two characters with 0x8e leading byte only the first one would be displayed.
- When using DYNAMIC_ICONV on Win32 conversion might fail because of using the wrong error number. (Muraoka Taro)
- Using Alt-x in the GUI while 'encoding' was set to "utf-8" didn't produce

  the right character.
- When using Visual block selection and only the left halve of a double-wide
  character is selected, the highlighting continued to the end of the line.
- Visual-block delete didn't work properly when deleting the right halve of a
  double-wide character.
- Overstrike mode for the cmdline replaced only the first byte of a multibyte
  character.
- The cursor in Replace mode (also in the cmdline) was to small on a
  double-wide character.
- When a multibyte character contained a 0x80 byte, it didn't work (was using
  a CSI byte instead). (Muraoka Taro)
- Wordwise selection with the mouse didn't work.
- Yanking a modeless selection of multi-byte characters didn't work.
- When 'selection' is "exclusive", selecting a word that ends in a multi-byte
  character used wrong highlighting for the following character.

Win32 with Make_mvc.mak: Didn't compile for debugging. (Craig Barkhouse)

Win32 GUI: When "vimrun.exe" is used to execute an external command, don't
give a message box with the return value, it was already printed by vimrun.
Also avoid printing the return value of the shell when ":silent!" is used.

Win32: selecting a lot of text and using the "find/replace" dialog caused a
crash.

X11 GUI: When typing a character with the 8th bit set and the Meta/Alt
modifier, the modifier was removed without changing the character.

Truncating a message to make it fit on the command line, using "..." for the
middle, didn't always compute the space correctly.

Could not imap <C-@>. Now it works like <Nul>.

VMS:
- Fixed a few things for VAXC. os_vms_fix.com had some strange CTRL-M
  characters. (Zoltan Arpadffy and John W. Hamill)
- Added VMS-specific defaults for the 'isfname' and 'isprint' options.
  (Zoltan Arpadffy)
- Removed os_vms_osdef.h, it's no longer used.

The gzip plugin used a ":normal" command, this doesn't work when dropping a
compressed file on Vim.

In very rare situations a binary search for a tag would fail, because an
uninitialized value happens to be half the size of the tag file. (Narendran)

When using BufEnter and BufLeave autocommands to enable/disable a menu, it
wasn't updated right away.

When doing a replace with the "c"onfirm flag, the cursor was positioned after
the ruler, instead of after the question. With a long replacement string the
screen could scroll up and cause a "more" prompt. Now the message is
truncated to make it fit.

Motif: The autoconf check for the Xp library didn't work.

When 'verbose' is set to list lines of a sourced file, defining a function would reset the counter used for the "more" prompt.

In the Win32 find/replace dialog, a '/' character caused problems. Escape it with a backslash.

Starting a shell with ":sh" was different from starting a shell for CTRL-Z when suspending doesn't work. They now work the same way.

Jumping to a file mark while in a changed buffer gave a "mark not set" error.

":execute histget("cmd")" causes an endless loop and crashed Vim. Now catch all commands that cause too much recursiveness.

Removed "Failed to open input method" error message, too many people got this when they didn't want to use a XIM.

GUI: When compiled without the +windows feature, the scrollbar would start below line one.

Removed the trick with redefining character class functions from regexp.c.

Win32 GUI: Find dialog gives focus back to main window, when typing a character mouse pointer is blanked, it didn't reappear when moving it in the dialog window. (Vince Negri)

When recording and typing a CTRL-C, no character was recorded. When in Insert mode or cancelling half a command, playing back the recorded sequence wouldn't work. Now record the CTRL-C.

When the GUI was started, mouse codes for DEC and netterm were still checked for.

GUI: When scrolling and 'writedelay' is non-zero, the character under the cursor was displayed in the wrong position (one line above/below with CTRL-E/CTRL-Y).

A ":normal" command would reset the 'scrollbind' info. Causes problems when using a ":normal" command in an autocommand for opening a file.

Windows GUI: a point size with a dot, like "7.5", wasn't recognized. (Muraoka Taro)

When 'scrollbind' wasn't set would still remember the current position, wasting time.

GTK: Crash when 'shell' doesn't exist and doing":!ls". Use _exit() instead of exit() when the child couldn't execute the shell.

Multi-byte:
- GUI with double-byte encoding: a mouse click in left halve of double-wide character put the cursor in previous char.

- Using double-byte encoding and 'selection' is "exclusive": "vey" and "^Vey" included the character after the word.
- When using a double-byte encoding and there is a lead byte at the end of the line, the preceding line would be displayed. "ga" also showed wrong info.
- "gf" didn't include multi-byte characters before the cursor properly. (Muraoka Taro)

GUI: The cursor was sometimes not removed when scrolling. Changed the policy from redrawing the cursor after each call to gui_write() to only update it at the end of update_screen() or when setting the cursor position. Also only update the scrollbars at the end of update_screen(), that's the only place where the window text may have been scrolled.

Formatting "/*<Tab>long text", produced "* <Tab>" in the next line. Now remove the space before the Tab.
Formatting "/*<Tab> long text", produced "* <Tab> long text" in the next line. Now keep the space after the Tab.

In some places non-ASCII alphabetical characters were accepted, which could cause problems. For example, ":X" (X being such a character).

When a pattern matches the end of the line, the last character in the line was highlighted for 'hlsearch'. That looks wrong for "/\%3c". Now highlight the character just after the line.

Motif: If a dialog was closed by clicking on the "X" of the window frame Vim would no longer respond.

When using CTRL-X or CTRL-A on a number with many leading zeros, Vim would crash. (Matsumoto)

When 'insertmode' is set, the mapping in mswin.vim for CTRL-V didn't work in Select mode. Insert mode wasn't restarted after overwriting the text.
Now allow nesting Insert mode with insert and change commands. CTRL-O cwfoo<Esc> now also works.

Clicking with the right mouse button in another window started Visual mode, but used the start position of the current window. Caused ml_get errors when the line number was invalid. Now stay in the same window.

When 'selection' is "exclusive", "gv" sometimes selected one character fewer.

When 'comments' contains more than one start/middle/end triplet, the optional flags could be mixed up. Also didn't align the end with the middle part.

Double-right-click in Visual mode didn't update the shown mode.

When the Normal group has a font name, it was never used when starting up.
Now use it when 'guifont' and 'guifontset' are empty.
Setting a font name to a highlight group before the GUI was started didn't work.

"make test" didn't use the name of the generated Vim executable.

'cindent' problems:
- Aligned with an "else" inside a do-while loop for a line below that loop.
  (Meikel Brandmeyer)
- A line before a function would be indented even when terminated with a
  semicolon. (Meikel Brandmeyer)
- 'cindent' gave too much indent to a line after a "};" that ends an array
  init.
- Support declaration lines ending in "," and "\". (Meikel Brandmeyer)
- A case statement inside a do-while loop was used for indenting a line after
  the do-while loop. (Meikel Brandmeyer)
- When skipping a string in a line with one double quote it could continue in
  the previous line. (Meikel Brandmeyer)

When 'list' is set, 'hlsearch' didn't highlight a match at the end of the
line. Now highlight the '$'.

The Paste menu item in the menu bar, the popup menu and the toolbar were all
different. Now made them all equal to how it was done in mswin.vim.

st_dev can be smaller than "unsigned". The compiler may give an overflow
warning. Added a configure check for dev_t.

Athena: closing a confirm() dialog killed Vim.

Various typos in the documentation. (Matt Dunford)

Python interface: The definition of _DEBUG could cause trouble, undefine it.
The error message for not being able to load the shared library wasn't
translated. (Muraoka Taro)

Mac: (Dany St-Amant and Axel Kielhorn)
- Several fixes.
- Vim was eating 80% of the CPU time.
- The project os_mac.pbxproj didn't work, Moved it to a subdirectory.
- Made the menu priority work for the menubar.
- Fixed a problem with dragging the scrollbar.
- Cleaned up the various #ifdefs.

Unix: When catching a deadly signal and we keep getting one use _exit() to
exit in a quick and dirty way.

Athena menu ordering didn't work correctly. (David Harrison)

A ":make" or ":grep" command with a long argument could cause a crash.

Doing ":new file" and using "Quit" for the ATTENTION dialog still opened a new
window.

GTK: When starting the GUI and there is an error in the .vimrc file, don't
present the wait-return prompt, since the message was given in the terminal.

When there was an error in a .vimrc file the terminal where gvim was started
could be cleared. Set msg_row in main.c before writing any messages.

GTK and X11 GUI: When trying to read characters from the user (e.g. with input()) before the Vim window was opened caused Vim to hang when it was started from the desktop.

OS/390 uses 31 bit pointers. That broke some computations with MAX_COL. Reduce MAX_COL by one bit for OS/390. (Ralf Schandl)

When defining a function and it already exists, Vim didn't say it existed until after typing it. Now do this right away when typing it.

The message remembered for displaying later (keep_msg) was sometimes pointing into a generic buffer, which might be changed by the time the message is displayed. Now make a copy of the message.

When using multi-byte characters in a menu and a trailing byte is a backslash, the menu would not be created correctly. (Muraoka Taro)
Using a multibyte character in the substitute string where a trail byte is a backslash didn't work. (Muraoka Taro)

When setting "t_Co" in a vimrc file, then setting it automatically from an xterm termresponse and then setting it again manually caused a crash.

When getting the value of a string option that is not supported, the number zero was returned. This breaks a check like "&enc == "asdf". Now an empty string is returned for string options.

Crashed when starting the GTK GUI while using 'notitle' in the vimrc, setting 'title' in the gvimrc and starting the GUI with ":gui". Closed the connection to the X server accidentally.

Had to hit return after selecting an entry for ":ts".

The message from ":cn" message was sometimes cleared. Now display it after redrawing if it doesn't cause a scroll (truncated when necessary).

hangulin.c didn't compile when the GUI was disabled. Disable it when it won't work.

When setting a termcap option like "t_CO", the value could be displayed as being for a normal key with a modifier, like "<M-=>".

When expanding the argument list, entries which are a directory name did not get included. This stopped "vim c:/" from opening the file explorer.

":syn match sd "^" nextgroup=asdf" skipped the first column and matched the nextgroup in the second column.

GUI: When 'lazyredraw' is set, 'showmatch' didn't work. Required flushing the output.

Don't define the <NetMouse> termcode in an xterm, reduces the problem when someone types <Esc> } in Insert mode.

Made slash_adjust() work correctly for multi-byte characters. (Yasuhiro

Matsumoto)
Using a filename in Big5 encoding for autocommands didn't work (backslash in trailbyte). (Yasuhiro Matsumoto)

DOS and Windows: Expanding *.vim also matched file.vimfoo. Expand path like Unix to avoid problems with Windows dir functions. Merged the DOS and Win32 functions.

Win32: GvimExt could not edit more than a few files at once, the length of the argument was fixed.

"ls -1 * | xargs vim" worked, but the input was in cooked mode. Now switch to raw mode when needed. Use dup() to copy the stderr file descriptor to stdin to make shell commands work. No longer requires an external program to do this.

When using ":filetype off", ftplugin and indent usage would be switched off at the same time. Don't do this, setting 'filetype' manually can still use them.

GUI: When writing a double-byte character, it could be split up in two calls to gui_write(), which doesn't work. Now flush before the output buffer becomes full.

When 'laststatus' is set and 'cmdheight' is two or bigger, the intro message would be written over the status line.
The ":intro" command didn't work when there wasn't enough room.

Configuring for Ruby failed with a recent version of Ruby. (Akinori Musha)

Athena: When deleting the directory in which Vim was started, using the file browser made Vim exit. Removed the use of XtAppError().

When using autoconf 2.50, UNIX was not defined. Moved the comment for "#undef UNIX" to a separate line.

Win32: Disabled _OnWindowPosChanging() to make maximize work better.

Win32: Compiling with VC 4.0 didn't work. (Walter Briscoe)

Athena:
- Finally fixed the problems with deleting a menu. (David Harrison)
- Athena: When closing the confirm() dialog, worked like OK was pressed, instead of Cancel.

The file explorer didn't work in compatible mode, because of line continuation.

Didn't give an error message for ":digraph a".

When using Ex mode in the GUI and typing a special key, <BS> didn't delete it correctly. Now display '?' for a special key.

When an operator is pending, clicking in another window made it apply to that window, even though the line numbers could be beyond the end of the buffer.

When a function call doesn't have a terminating ")" Vim could crash.

Perl interface: could crash on exit with perl 5.6.1. (Anduin Withers)

Using %P in 'errorformat' wasn't handled correctly. (Tomas Zellerin)

Using a syntax cluster that includes itself made Vim crash.

GUI: With 'ls' set to 2, dragging the status line all the way up, then making the Vim window smaller: Could not the drag status line anymore.

"vim -c startinsert! file" placed cursor on last char of a line, instead of after it. A ":set" command in the buffer menu set w_set_curswant. Now don't do this when w_curswant is MAXCOL.

Win32: When the gvim window was maximized and selecting another font, the window would no longer fill the screen.

The line with 'pastetoggle' in ":options" didn't show the right value when it is a special key. Hitting <CR> didn't work either.

Formatting text, resulting in a % landing in the first line, repeated the % in the following lines, like it's the start of a comment.

GTK: When adding a toolbar item while gvim is already running, it wasn't possible to use the tooltip. Now it works by adding the tooltip first.

The output of "g CTRL-G" mentioned "Char" but it's actually bytes.

Searching for the end of a oneline region didn't work correctly when there is an offset for the highlighting.

Syntax highlighting: When synchronizing on C-comments, //*/ was seen as the start of a comment.

Win32: Without scrollbars present, the MS mouse scroll wheel didn't work. Also handle the scrollbars when they are not visible.

Motif: When there is no right scrollbar, the bottom scrollbar would still leave room for it. (Marcin Dalecki)

When changing 'guicursor' and the value is invalid, some of the effects would still take place. Now first check for errors and only make the new value effective when it's OK.

Using "A" In Visual block mode, appending to lines that don't extend into the block, padding was wrong.

When pasting a block of text, a character that occupies more than one screen column could be deleted and spaces inserted instead. Now only do that with a tab.

Fixed conversion of documentation to HTML using Perl. (Dan Sharp)

Give an error message when a menu name starts with a dot.

Avoid a hang when executing a shell from the GUI on HP-UX by pushing "ptem" even when sys/ptem.h isn't present.

When creating the temp directory, make sure umask is 077, otherwise the directory is not accessible when it was set to 0177.

Unix: When resizing the window and a redraw is a bit slow, could get a window resize event while redrawing, resulting in a messed up window. Any input (e.g., a mouse click) would redraw.

The "%B" item in the status line became zero in Insert mode (that's normal) for another than the current window.

The menu entries to convert to xxd and back didn't work in Insert mode.

When ":vglobal" didn't find a line where the pattern doesn't match, the error message would be the wrong way around.

When ignoring a multi-line error message with "%-A", the continuation lines would be used anyway. (Servatius Brandt)

"grx" on a double-wide character inserted "x", instead of replacing the character with "x ". "gR" on <xx> ('display' set the "uhex") didn't replace at all. When doing "gRxx" on a control character the first "x" would be inserted, breaking the alignment.

Added "0)" to 'cinkeys', so that when typing a) it is put in the same place as where "==" would put it.

Win32: When maximized, adding/removing toolbar didn't resize the text area.

When using <C-RightMouse> a count was discarded.

When typing CTRL-V and <RightMouse> in the command line, would insert <LeftMouse>.

Using "vis" or "vas" when 'selection' is exclusive didn't include the last character.

When adding to an option like 'grepprg', leading space would be lost. Don't expand environment variables when there is no comma separating the items.

GUI: When using a bold-italic font, would still use the bold trick and underlining.

Motif: The default button didn't work in dialogs, the first one was always used. Had to give input focus to the default button.

When using CTRL-T to jump within the same file, the '' mark wasn't set.

Undo wasn't Vi compatible when using the 'c' flag for ":s". Now it undoes the

whole ":s" command instead of each confirmed replacement.

The Buffers menu, when torn-off, disappeared when being refreshed. Add a dummy item to avoid this.

Removed calling msg_start() in main(), it should not be needed.

vim_strpbrk() did not support multibyte characters. (Muraoka Taro)

The Amiga version didn't compile, the code was too big for relative jumps. Moved a few files from ex_docmd.c to ex_cmds2.c

When evaluating the "= register resulted in the "= register being changed, Vim would crash.

When doing ":view file" and it fails, the current buffer was made read-only.

Motif: For some people the separators in the toolbar disappeared when resizing the Vim window. (Marcin Dalecki)

Win32 GUI: when setting 'lines' to a huge number, would not compute the available space correctly. Was counting the menu height twice.

Conversion of the docs to HTML didn't handle the line with the +quickfix tag correctly. (Antonio Colombo)

Win32: fname_case() didn't handle multi-byte characters correctly. (Yasuhiro Matsumoto)

The Cygwin version had trouble with fchdir(). Don't use that function for Cygwin.

The generic check in scripts.vim for "conf" syntax was done before some checks in filetype.vim, resulting in "conf" syntax too often.

Dos32: Typing lagged behind. Would wait for one biostick when checking if a character is available.

GTK: When setting 'columns' while starting up "gvim", would set the width of the terminal it was started in.

When using ESC in Insert mode, an autoindent that wraps to the next line caused the cursor to move to the end of the line temporarily. When the character before the cursor was a double-wide multi-byte character the cursor would be on the right halve, which causes problems with some terminals.

Didn't handle multi-byte characters correctly when expanding a file name. (Yasuhiro Matsumoto)

Win32 GUI: Errors generated before the GUI is decided to start were not reported.

globpath() didn't reserve enough room for concatenated results. (Anduin Withers)

When expanding an option that is very long already, don't do the expansion, it would be truncated to MAXPATHL. (Anduin Withers)

When 'selection' is "exclusive", using "Fx" in Visual mode only moved until just after the character.

When using IME on the console to enter a file name, the screen may scroll up. Redraw the screen then. (Yasuhiro Matsumoto)

Motif: In the find/replace dialog the "Replace" button didn't work first time, second time it replaced all matches. Removed the use of ":s///c".
GTK: Similar problems with the find/replace dialog, moved the code to a common function.

X11: Use shared GC's for text. (Marcin Dalecki)

"]i" found the match under the cursor, instead of the first one below it. Same for "]I", "] CTRL-I", "]d", "]D" and "] CTRL-D".

Win16: When maximized and the font is changed, don't change the window size. (Vince Negri)

When 'lbr' is set, deleting a block of text could leave the cursor in the wrong position.

Win32: When opening a file with the "Edit with Vim" popup menu entry, wildcards would cause trouble. Added the "--literal" argument to avoid expanding file names.

When using "gv", it didn't restore that "$" was used in Visual block mode.

Win32 GUI: While waiting for a shell command to finish, the window wasn't redrawn at all. (Yasuhiro Matsumoto)

Syntax highlighting: A match that continues on a next line because of a contained region didn't end when that region ended.

The ":s" command didn't allow flags like 'e' and 'i' right after it.

When using ":s" to split a line, marks were moved to the next line. Vi keeps them in the first line.

When using ":n" ":rew", the previous context mark was at the top of the file, while Vi puts it in the same place as the cursor. Made it Vi compatible.

Fixed Vi incompatibility: Text was not put in register 1 when using "c" and "d" with a motion character, when deleting within one line with one of the commands: % ( ) `<character> / ? N n { }

Win32 GUI: The tooltip for tear-off items remained when the tear-off item was no longer selected.

GUI: When typing ":" at the more prompt, would return to Normal mode and not

redraw the screen.

When starting Vim with an argument "-c g/at/p" the printed lines would
overwrite each other.

BeOS: Didn't compile. Configure didn't add the os_beos files, the QNX check
removed them. Various changes to os_beos.cc. (Joshua Haberman)
Removed the check for the hardware platform, the BeBox has not been produced
for a long time now.

Win32 GUI: don't use a message box when the shell returns an error code,
display the message in the Vim window.

Make_mvc.mak always included "/debug" for linking. "GUI=no" argument didn't
work. Use "DEBUG=yes" instead of "DEBUG=1" to make it consistent. (Dan Sharp)

When a line in the tags file ended in ;" (no TAB following) the command would
not be recognized as a search command.

X11: The inputMethod resource never worked. Don't use the "none" input method
for SGI, it apparently makes the first character in Input method dropped.

Fixed incorrect tests in os_mac.h. (Axel Kielhorn)

Win32 console: When the console where Vim runs in is closed, Vim could hang in
trying to restore the window icon. (Yasuhiro Matsumoto)

When using ":3call func()" or ":3,3call func() the line number was ignored.

When 'showbreak' and 'linebreak' were both set, Visual highlighting sometimes
continued until the end of the line.

GTK GUI: Tearoff items were added even when 'guioptions' didn't contain 't'
when starting up.

MS-Windows: When the current directory includes a "~", searching files with
"gf" or ":find" didn't work. A "$" in the directory had the same problem.
Added mch_has_exp_wildcard() functions.

When reducing the Vim window height while starting up, would get an
out-of-memory error message.

When editing a very long search pattern, 'incsearch' caused the redraw of the
command line to fail.

Motif GUI: On some systems the "Help" menu would not be on the far right, as
it should be. On some other systems (esp. IRIX) the command line would not
completely show. Solution is to only resize the menubar for Lesstif.

Using "%" in a line that contains "\\" twice didn't take care of the quotes
properly. Now make a difference between \" and \\".

For non-Unix systems a dummy file is created when finding a swap name to
detect a 8.3 filesystem. When there is an existing swap file, would get a

warning for the file being created outside of Vim. Also, when closing the Vim window the file would remain.

Motif: The menu height was always computed, using a "-menuheight" argument was setting the room for the command line. Now make clear the argument is not supported.

For some (EBCDIC) systems, POUND was equal to '#'. Added an #if for that to avoid a duplicate case in a switch.

The GUI may have problems when forking. Always call _exit() instead of exit() in the parent, the child will call exit().

Win32 GUI: Accented characters were often wrong in dialogs and tearoff menus. Now use CP_ACP instead of CP_OEMCP. (Vince Negri)

When displaying text with syntax highlighting causes an error (e.g., running out of stack) the syntax highlighting is disabled to avoid further messages.

When a command in a .vimrc or .gvimrc causes an ATTENTION prompt, and Vim was started from the desktop (no place to display messages) it would hang. Now open the GUI window early to be able to display the messages and pop up the dialog.

"r<CR>" on a multi-byte character deleted only the first byte of the character. "3r<CR>" deleted three bytes instead of three characters.

When interrupting reading a file, Vi considers the buffer modified. Added the 'i' flag in 'cpoptions' flag for this (we don't want it modified to be able to do ":q").

When using an item in 'guicursor' that starts with a colon, Vim would get stuck or crash.

When putting a file mark in a help file and later jumping back to it, the options would not be set. Extended the modeline in all help files to make this work better.

When a modeline contained "::" the local option values would be printed. Now ignore it.

Some help files did not use a 8.3 names, which causes problems when using MS-DOS unzip. Renamed "multibyte.txt" to "mbyte.txt", "rightleft.txt" to "rileft.txt", "tagsearch.txt" to "tagsrch.txt", "os_riscos.txt" to "os_risc.txt".

When Visual mode is blockwise, using "iw" or "aw" made it characterwise. That doesn't seem right, only do this when in linewise mode. But then do it always, not only when start and end of Visual mode are equal.

When using "viw" on a single-letter word and 'selection' is exclusive, would not include the word.

When formatting text from Insert mode, using CTRL-O, could mess up undo

information.

While writing a file (also for the backup file) there was no check for an interrupt (hitting CTRL-C). Vim could hang when writing a large file over a slow network, and moving the mouse didn't make it appear (when 'mousehide' is set) and the screen wasn't updated in the GUI. Also allow interrupting when syncing the swap file, it can take a long time.

When using ":mksession" while there is help window, it would later be restored to the right file but not marked as a help buffer. ":help" would then open another window. Now use the value "help" for 'buftype' to mark a help buffer.

The session file contained absolute path names in option values, that doesn't work when the home directory depends on the situation. Replace the home directory with ~/ when possible.

When using 'showbreak' a TAB just after the shown break would not be counted correctly, the cursor would be positioned wrong.

With 'showbreak' set to "--->" or "------->" and 'sts' set to 4, inserting tabs did not work right. Could cause a crash. Backspacing was also wrong, could get stuck at a line break.

Win32: crashed when tearing off a menu with over 300 items.

GUI: A menu or toolbar item would appear when only a tooltip was defined for it.

When 'scrolloff' is non-zero and "$" is in 'cpoptions', using "s" while the last line of the file is the first line on screen, the text wasn't displayed.

When running "autoconf", delete the configure cache to force starting cleanly when configure is run again.

When changing the Normal colors for cterm, the value of 'background' was changed even when the GUI was used.

The warning for a missing vimrun.exe was always given on startup, but some people just editing a file don't need to be bothered by it. Only show it when vimrun would be used.

When using "%" in a multibyte text it could get confused by trailbytes that match. (Muraoka Taro)

Termcap entry for RiscOS was wrong, using 7 and 8 in octal codes.

Athena: The title of a dialog window and the file selector window were not set. (David Harrison)

The "htmlLink" highlight group specified colors, which gives problems when using a color scheme. Added the "Underlined" highlight group for this.

After using ":insert" or ":change" the '[mark would be one line too low.

When looking for the file name after a match with 'include' one character was skipped. Same for 'define'.

Win32 and DJGPP: When editing a file with a short name in a directory, and editing the same file but using the long name, would end up with two buffers on the same file.

"gf" on a filename that starts with "../" only worked when the file being edited is in the current directory. An include file search didn't work properly for files starting with "../" or "./". Now search both relative to the file and to the current directory.

When 'printheader', 'titlestring', 'iconstring', 'rulerformat' or 'statusline' contained "%{" but no following "}" memory was corrupted and a crash could happen.

":0append" and then inserting two lines did not redraw the blank lines that were scrolled back down.

When using insert mode completion in a narrow window, the message caused a scroll up. Now shorten the message if it doesn't fit and avoid writing the ruler over the message.

XIM still didn't work correctly on some systems, especially SGI/IRIX. Added the 'imdisable' option, which is set by default for that system.

Patch 6.0aw.008
问题: When the first character of a file name is over 127, the Buffers menu entry would get a negative priority and cause problems.
解决方案: Reduce the multiplier for the first character when computing the hash value for a Buffers menu entry.
相关文件: runtime/menu.vim

Patch 6.0aw.010
问题: Win32: ":browse edit dir/dir" didn't work. (Vikas)
解决方案: Change slashes to backslashes in the directory passed to the file browser.
相关文件: src/gui_w48.c

Athena file browser: On some systems wcstombs() can't be used to get the length of a multi-byte string. Use the maximum length then. (Yasuhiro Matsumoto)

Patch 6.0ax.001
问题: When 'patchmode' is set, appending to a file gives an empty original file. (Ed Ralston)
解决方案: Also make a backup copy when appending and 'patchmode' is set.
相关文件: src/fileio.c

Patch 6.0ax.002
问题: When 'patchmode' is set, appending to a compressed file gives an uncompressed original file. (Ed Ralston)
解决方案: Create the original file before decompressing.
相关文件: runtime/plugin/gzip.vim

Patch 6.0ax.005
问题:　　Athena file selector keeps the title of the first invocation.
解决方案:　Set the title each time the file selector is opened. (David Harrison)
相关文件:　src/gui_at_fs.c

Patch 6.0ax.007
问题:　　When using GPM (mouse driver in a Linux console) a double click is interpreted as a scroll wheel click.
解决方案:　Check if GPM is being used when deciding if a mouse event is for the scroll wheel.
相关文件:　src/term.c

Patch 6.0ax.010
问题:　　The Edit.Save menu and the Save toolbar button didn't work when the buffer has no file name.
解决方案:　Use a file browser to ask for a file name. Also fix the toolbar Find item in Visual mode.
相关文件:　runtime/menu.vim

Patch 6.0ax.012
问题:　　When 'cpoptions' contains "$", breaking a line for 'textwidth' doesn't redraw properly. (Stefan Schulze)
解决方案:　Remove the dollar before breaking the line.
相关文件:　src/edit.c

Patch 6.0ax.014
问题:　　Win32: On Windows 98 ":make -f file" doesn't work when 'shell' is "command.com" and 'makeprg' is "nmake". The environment isn't passed on to "nmake".
解决方案:　Also use vimrun.exe when redirecting the output of a command.
相关文件:　src/os_win32.c

Patch 6.0ax.016
问题:　　The version number was reported wrong in the intro screen.
解决方案:　Check for a version number with two additional letters.
相关文件:　src/version.c

Patch 6.0ax.019
问题:　　When scrolling a window with folds upwards, switching to another vertically split window and back may not update the scrollbar.
解决方案:　Limit w_botline to the number of lines in the buffer plus one.
相关文件:　src/move.c

6.1 版 本　　version-6.1

本小节讨论 6.0 和 6.1 版本之间的改进。

这是一个漏洞修正版。其实没有什么新特性。

改动 changed-6.1

用 ":lmap" 定义语言映射时，不再有设置 'iminsert' 和 'imsearch' 的副作用。

新增 added-6.1

语法文件:

ampl	AMPL (David Krief)
ant	Ant (Johannes Zellner)
baan	Baan (Her van de Vliert)
cs	C# (Johannes Zellner)
lifelines	Lifelines (Patrick Texier)
lscript	LotusScript (Taryn East)
moo	MOO (Timo Frenay)
nsis	NSIS (Alex Jakushev)
ppd	Postscript Printer Description (Bjoern Jacke)
rpl	RPL/2 (Joel Bertrand)
scilab	Scilab (Benoit Hamelin)
splint	Splint (Ralf Wildenhues)
sqlj	SQLJ (Andreas Fischbach)
wvdial	WvDial (Prahlad Vaidyanathan)
xf86conf	XFree86 config (Nikolai Weibull)
xmodmap	Xmodmap (Nikolai Weibull)
xslt	Xslt (Johannes Zellner)
monk	Monk (Mike Litherland)
xsd	Xsd (Johannes Zellner)
cdl	CDL (Raul Segura Acevedo)
sendpr	Send-pr (Hendrik Scholz)

加入 Scheme 缩进文件。(Dorai Sitaram)
加入 Prolog 缩进文件。(Kontra Gergely)
加入 Povray 缩进文件。(David Necas)
加入 IDL 缩进文件 (Aleksandar Jelenak)
加入 C# 缩进和文件类型插件脚本。

加入乌克兰语菜单翻译。(Bohdan Vlasyuk)
加入捷克菜单的 ASCII 版本。(Jiri Brezina)

加入教程的简体中文翻译。(Mendel L Chan)

加入 yawerty 键盘的俄罗斯键盘映射表。

教程里加上使用 vimrc 文件的解释。
修改 tutor.vim，台湾版教程能使用合适的编码。

加入俄语教程。(Andrey Kiselev)
加入波兰语教程。(Mikolaj Machowski)

加入 darkblue 色彩方案。(Bohdan Vlasyuk)

打包时，dos 语言归档自动生成需要的 .mo 文件。

改进了 NSIS 脚本以支持 NSIS 180。加入打开/关闭状态的图标。(Mirek Pruchnik)

cp1250 版本的斯洛伐克消息翻译版本。

IRIX 编译器的编译器插件。(David Harrison)

修正 fixed-6.1

The license text was updated to make the meaning clearer and make it
compatible with the GNU GPL. Otherwise distributors have a problem when
linking Vim with a GPL'ed library.

When installing the "less.sh" script it was not made executable. (Chuck Berg)

Win32: The "9" key on the numpad wasn't working. (Julian Kinraid)

The NSIS install script didn't work with NSIS 1.80 or later. Also add
Vim-specific icons. (Pruchnik)

The script for conversion to HTML contained an "if" in the wrong place.
(Michael Geddes)

Allow using ":ascii" in the sandbox, it's harmless.

Removed creat() from osdef2.h.in, it wasn't used and may cause a problem when
it's redefined to creat64().

The text files in the VisVim directory were in "dos" format. This caused
problems when applying a patch. Now keep them in "unix" format and convert
them to "dos" format only for the PC archives.

Add ruby files to the dos source archive, they can be used by Make_mvc.mak.
(Mirek Pruchnik)

"cp -f" doesn't work on all systems. Change "cp -f" in the Makefile to "rm
-f" and "cp".

Didn't compile on a Compaq Tandem Himalaya OSS. (Michael A. Benzinger)

The GTK file selection dialog didn't include the "Create Dir", "Delete File"
and "Rename File" buttons.

When doing ":browse source" the dialog has the title "Run Macro". Better
would be "Source Vim script". (Yegappan Lakshmanan)

Win32: Don't use the printer font as default for the font dialog.

"make doslang" didn't work when configure didn't run (yet). Set $MAKEMO to
"yes". (Mirek Pruchnik)

The ToolBar TagJump item used "g]", which prompts for a selection even when

there is only one matching tag. Use "g<C-]>" instead.

The ming makefile for message translations didn't have the right list of files.

The MS-Windows 3.1 version complains about LIBINTL.DLL not found. Compile this version without message translations.

The Borland 5 makefile contained a check for Ruby which is no longer needed. The URLs for the TCL library was outdated. (Dan Sharp)

The eviso.ps file was missing from the DOS runtime archive, it's needed for printing PostScript in the 32bit DOS version.

In menu files ":scriptencoding" was used in a wrong way after patch 6.1a.032 Now use ":scriptencoding" in the file where the translations are given. Do the same for all menus in latin1 encoding.

Included a lot of fixes for the Macintosh, mostly to make it work with Carbon. (Dany StAmant, Axel Kielhorn, Benji Fisher)

Improved the vimtutor shell script to use $TMPDIR when it exists, and delete the copied file when exiting in an abnormal way. (Max Ischenko)

When "iconv.dll" can't be found, try using "libiconv.dll".

When encryption is used, filtering with a shell command wasn't possible.

DJGPP: ":cd c:" always failed, can't get permissions for "c:".
Win32: ":cd c:/" failed if the previous current directory on c: had become invalid.

DJGPP: Shift-Del and Del both produce \316\123. Default mapping for Del is wrong. Disabled it.

Dependencies on header files in MingW makefile was wrong.

Win32: Don't use ACL stuff for MSVC 4.2, it's not supported. (Walter Briscoe)

Win32 with Borland: bcc.cfg was caching the value for $(BOR), but providing a different argument to make didn't regenerate it.

Win32 with MSVC: Make_ivc.mak generates a new if_ole.h in a different directory, the if_ole.h in the src directory may be used instead. Delete the distributed file.

When a window is vertically split and then ":ball" is used, the window layout is messed up, can cause a crash. (Muraoka Taro)

When 'insertmode' is set, using File/New menu and then double clicking, "i" is soon inserted. (Merlin Hansen)

When Select mode is active and using the Buffers menu to switch to another buffer, an old selection comes back. Reset VIsual_reselect for a ":buffer"

command.

When Select mode is active and 'insertmode' is set, using the Buffers menu to switch to another buffer, did not return to Insert mode. Make sure "restart_edit" is set.

When double clicking on the first character of a word while 'selection' is "exclusive" didn't select that word.

Patch 6.0.001
问题: Loading the sh.vim syntax file causes error messages. (Corinna Vinschen)
解决方案: Add an "if". (Charles Campbell)
相关文件: runtime/syntax/sh.vim

Patch 6.0.002
问题: Using a '@' item in 'viminfo' doesn't work. (Marko Leipert)
解决方案: Add '@' to the list of accepted items.
相关文件: src/option.c

Patch 6.0.003
问题: The configure check for ACLs on AIX doesn't work.
解决方案: Fix the test program so that it compiles. (Tomas Ogren)
相关文件: src/configure.in, src/auto/configure

Patch 6.0.004
问题: The find/replace dialog doesn't reuse a previous argument properly.
解决方案: After removing a "\V" terminate the string. (Zwane Mwaikambo)
相关文件: src/gui.c

Patch 6.0.005
问题: In Insert mode, "CTRL-O :ls" has a delay before redrawing.
解决方案: Don't delay just after wait_return() was called. Added the did_wait_return flag.
相关文件: src/globals.h, src/message.c, src/normal.c, src/screen.c

Patch 6.0.006
问题: With a vertical split, 'number' set and 'scrolloff' non-zero, making the window width very small causes a crash. (Niklas Lindstrom)
解决方案: Check for a zero width.
相关文件: src/move.c

Patch 6.0.007
问题: When setting 'filetype' while there is no FileType autocommand, a following ":setfiletype" would set 'filetype' again. (Kobus Retief)
解决方案: Set did_filetype always when 'filetype' has been set.
相关文件: src/option.c

Patch 6.0.008
问题: 'imdisable' is missing from the options window. (Michael Naumann)

解决方案: Add an entry for it.
相关文件: runtime/optwin.vim

Patch 6.0.009
问题: Nextstep doesn't have S_ISBLK. (John Beppu)
解决方案: Define S_ISBLK using S_IFBLK.
相关文件: src/os_unix.h

Patch 6.0.010
问题: Using "gf" on a file name starting with "./" or "../" in a buffer
without a name causes a crash. (Roy Lewis)
解决方案: Check for a NULL file name.
相关文件: src/misc2.c

Patch 6.0.011
问题: Python: After replacing or deleting lines get an ml_get error.
(Leo Lipelis)
解决方案: Adjust the cursor position for deleted or added lines.
相关文件: src/if_python.c

Patch 6.0.012
问题: Polish translations contain printf format errors, this can result
in a crash when using one of them.
解决方案: Fix for translated messages. (Michal Politowski)
相关文件: src/po/pl.po

Patch 6.0.013
问题: Using ":silent! cmd" still gives some error messages, like for an
invalid range. (Salman Halim)
解决方案: Reset emsg_silent after calling emsg() in do_one_cmd().
相关文件: src/ex_docmd.c

Patch 6.0.014
问题: When 'modifiable' is off and 'virtualedit' is "all", "rx" on a TAB
still changes the buffer. (Muraoka Taro)
解决方案: Check if saving the line for undo fails.
相关文件: src/normal.c

Patch 6.0.015
问题: When 'cpoptions' includes "S" and "filetype plugin on" has been
used, can get an error for deleting the b:did_ftplugin variable.
(Ralph Henderson)
解决方案: Only delete the variable when it exists.
相关文件: runtime/ftplugin.vim

Patch 6.0.016
问题: bufnr(), bufname() and bufwinnr() don't find unlisted buffers when
the argument is a string. (Hari Krishna Dara)
Also for setbufvar() and getbufvar().
解决方案: Also find unlisted buffers.
相关文件: src/eval.c

Patch 6.0.017
问题: When 'ttybuiltin' is set and a builtin termcap entry defines t_Co

and the external one doesn't, it gets reset to empty. (David Harrison)
解决方案: Only set t_Co when it wasn't set yet.
相关文件: src/term.c

Patch 6.0.018
问题: Initializing 'encoding' may cause a crash when setlocale() is not used. (Dany St-Amant)
解决方案: Check for a NULL pointer.
相关文件: src/mbyte.c

Patch 6.0.019
问题: Converting a string with multi-byte characters to a printable string, e.g., with strtrans(), may cause a crash. (Tomas Zellerin)
解决方案: Correctly compute the length of the result in transstr().
相关文件: src/charset.c

Patch 6.0.020
问题: When obtaining the value of a global variable internally, could get the function-local value instead. Applies to using <Leader> and <LocalLeader> and resetting highlighting in a function.
解决方案: Prepend "g:" to the variable name. (Aric Blumer)
相关文件: src/syntax.c, src/term.c

Patch 6.0.021
问题: The 'cscopepathcomp' option didn't work.
解决方案: Change USE_CSCOPE to FEAT_CSCOPE. (Mark Feng)
相关文件: src/option.c

Patch 6.0.022
问题: When using the 'langmap' option, the second character of a command starting with "g" isn't adjusted.
解决方案: Apply 'langmap' to the second character. (Alex Kapranoff)
相关文件: src/normal.c

Patch 6.0.023
问题: Loading the lhaskell syntax doesn't work. (Thore B. Karlsen)
解决方案: Use ":runtime" instead of "source" to load haskell.vim.
相关文件: runtime/syntax/lhaskell.vim

Patch 6.0.024
问题: Using "CTRL-V u 9900" in Insert mode may cause a crash. (Noah Levitt)
解决方案: Don't insert a NUL byte in the text, use a newline.
相关文件: src/misc1.c

Patch 6.0.025
问题: The pattern "\vx(.|$)" doesn't match "x" at the end of a line. (Preben Peppe Guldberg)
解决方案: Always see a "$" as end-of-line after "\v". Do the same for "^".
相关文件: src/regexp.c

Patch 6.0.026
问题: GTK: When using arrow keys to navigate through the menus, the

separators are selected.
解决方案: Set the separators "insensitive". (Pavel Kankovsky)
相关文件: src/gui_gtk.c, src/gui_gtk_x11.c

Patch 6.0.027
问题: VMS: Printing doesn't work, the file is deleted too quickly.
No longer need the VMS specific printing menu.
gethostname() is not available with VAXC.
The makefile was lacking selection of the tiny-huge feature set.
解决方案: Adjust the 'printexpr' option default. Fix the other problems and update the documentation. (Zoltan Arpadffy)
相关文件: runtime/doc/os_vms.txt, runtime/menu.vim, src/INSTALLvms.txt, src/Make_vms.mms, src/option.c, src/os_unix.c, src/os_vms_conf.h

Patch 6.0.028
问题: Can't compile without +virtualedit and with +visualextra. (Geza Lakner)
解决方案: Add an #ifdef for +virtualedit.
相关文件: src/ops.c

Patch 6.0.029
问题: When making a change in line 1, then in line 2 and then deleting line 1, undo info could be wrong. Only when the changes are undone at once. (Gerhard Hochholzer)
解决方案: When not saving a line for undo because it was already done before, remember for which entry the last line must be computed. Added ue_getbot_entry pointer for this. When the number of lines changes, adjust the position of newer undo entries.
相关文件: src/structs.h, src/undo.c

Patch 6.0.030
问题: Using ":source! file" doesn't work inside a loop or after ":argdo". (Pavol Juhas)
解决方案: Execute the commands in the file right away, do not let the main loop do it.
相关文件: src/ex_cmds2.c, src/ex_docmd.c, src/getchar.c, src/globals.h, src/proto/ex_docmd.pro, src/proto/getchar.pro

Patch 6.0.031
问题: Nextstep doesn't have setenv() or putenv(). (John Beppu)
解决方案: Move putenv() from pty.c to misc2.c
相关文件: src/misc2.c, src/pty.c

Patch 6.0.032
问题: When changing a setting that affects all folds, they are not displayed immediately.
解决方案: Set the redraw flag in foldUpdateAll().
相关文件: src/fold.c

Patch 6.0.033
问题: Using 'wildmenu' on MS-Windows, file names that include a space are only displayed starting with that space. (Xie Yuheng)
解决方案: Don't recognize a backslash before a space as a path separator.
相关文件: src/screen.c

Patch 6.0.034
问题: Calling searchpair() with three arguments could result in a crash or strange error message. (Kalle Bjorklid)
解决方案: Don't use the fifth argument when there is no fourth argument.
相关文件: src/eval.c

Patch 6.0.035
问题: The menu item Edit/Global_Settings/Toggle_Toolbar doesn't work when 'ignorecase' is set. (Allen Castaban)
解决方案: Always match case when checking if a flag is already present in 'guioptions'.
相关文件: runtime/menu.vim

Patch 6.0.036
问题: OS/2, MS-DOS and MS-Windows: Using a path that starts with a slash in 'tags' doesn't work as expected. (Mathias Koehrer)
解决方案: Only use the drive, not the whole path to the current directory. Also make it work for "c:dir/file".
相关文件: src/misc2.c

Patch 6.0.037
问题: When the user has set "did_install_syntax_menu" to avoid the default Syntax menu it still appears. (Virgilio)
解决方案: Don't add the three default items when "did_install_syntax_menu" is set.
相关文件: runtime/menu.vim

Patch 6.0.038
问题: When 'selection' is "exclusive", deleting a block of text at the end of a line can leave the cursor beyond the end of the line.
解决方案: Correct the cursor position.
相关文件: src/ops.c

Patch 6.0.039
问题: "gP" leaves the cursor in the wrong position when 'virtualedit' is used. Using "c" in blockwise Visual mode leaves the cursor in a strange position.
解决方案: For "gP" reset the "coladd" field for the '] mark. For "c" leave the cursor on the last inserted character.
相关文件: src/ops.c

Patch 6.0.040
问题: When 'fileencoding' is invalid and writing fails because of this, the original file is gone. (Eric Carlier)
解决方案: Restore the original file from the backup.
相关文件: src/fileio.c

Patch 6.0.041
问题: Using ":language messages en" when LC_MESSAGES is undefined results in setting LC_CTYPE. (Eric Carlier)
解决方案: Set $LC_MESSAGES instead.
相关文件: src/ex_cmds2.c

Patch 6.0.042
问题:　　":mksession" can't handle file names with a space.
解决方案:　Escape special characters in file names with a backslash.
相关文件:　src/ex_docmd.c

Patch 6.0.043
问题:　　Patch 6.0.041 was wrong.
解决方案:　Use mch_getenv() instead of vim_getenv().
相关文件:　src/ex_cmds2.c

Patch 6.0.044
问题:　　Using a "containedin" list for a syntax item doesn't work for an
　　item that doesn't have a "contains" argument. Also, "containedin"
　　doesn't ignore a transparent item. (Timo Frenay)
解决方案:　When there is a "containedin" argument somewhere, always check for
　　contained items. Don't check for the transparent item but the
　　item it's contained in.
相关文件:　src/structs.h, src/syntax.c

Patch 6.0.045
问题:　　After creating a fold with a Visual selection, another window
　　with the same buffer still has inverted text. (Sami Salonen)
解决方案:　Redraw the inverted text.
相关文件:　src/normal.c

Patch 6.0.046
问题:　　When getrlimit() returns an 8 byte number the check for running
　　out of stack may fail. (Anthony Meijer)
解决方案:　Skip the stack check if the limit doesn't fit in a long.
相关文件:　src/auto/configure, src/config.h.in, src/configure.in,
　　src/os_unix.c

Patch 6.0.047
问题:　　Using a regexp with "\(\)" inside a "\%[]" item causes a crash.
　　(Samuel Lacas)
解决方案:　Don't allow nested atoms inside "\%[]".
相关文件:　src/regexp.c

Patch 6.0.048
问题:　　Win32: In the console the mouse doesn't always work correctly.
　　Sometimes after getting focus a mouse movement is interpreted like
　　a button click.
解决方案:　Use a different function to obtain the number of mouse buttons.
　　Avoid recognizing a button press from undefined bits. (Vince Negri)
相关文件:　src/os_win32.c

Patch 6.0.049
问题:　　When using evim the intro screen is misleading. (Adrian Nagle)
解决方案:　Mention whether 'insertmode' is set and the menus to be used.
相关文件:　runtime/menu.vim, src/version.c

Patch 6.0.050
问题:　　UTF-8: "viw" doesn't include non-ASCII characters before the
　　cursor. (Bertilo Wennergren)

解决方案: Use dec_cursor() instead of decrementing the column number.
相关文件: src/search.c

Patch 6.0.051
问题: UTF-8: Using CTRL-R on the command line doesn't insert composing
characters. (Ron Aaron)
解决方案: Also include the composing characters and fix redrawing them.
相关文件: src/ex_getln.c, src/ops.c

Patch 6.0.052
问题: The check for rlim_t in patch 6.0.046 does not work on some
systems. (Zdenek Sekera)
解决方案: Also look in sys/resource.h for rlim_t.
相关文件: src/auto/configure, src/configure.in

Patch 6.0.053 (extra)
问题: Various problems with QNX.
解决方案: Minor fix for configure. Switch on terminal clipboard support in
main.c. Fix "pterm" mouse support. os_qnx.c didn't build without
photon. (Julian Kinraid)
相关文件: src/auto/configure, src/configure.in, src/gui_photon.c,
src/main.c, src/misc2.c, src/option.h, src/os_qnx.c, src/os_qnx.h,
src/syntax.c

Patch 6.0.054
问题: When using mswin.vim, CTRL-V pastes a block of text like it is
normal text. Using CTRL-V in blockwise Visual mode leaves "x"
characters behind.
解决方案: Make CTRL-V work as it should. Do the same for the Paste menu
entries.
相关文件: runtime/menu.vim, runtime/mswin.vim

Patch 6.0.055
问题: GTK: The selection isn't copied the first time.
解决方案: Own the selection at the right moment.
相关文件: src/gui_gtk_x11.c

Patch 6.0.056
问题: Using "CTRL-O cw" in Insert mode results in a nested Insert mode.
<Esc> doesn't leave Insert mode then.
解决方案: Only use nested Insert mode when 'insertmode' is set or when a
mapping is used.
相关文件: src/normal.c

Patch 6.0.057
问题: Using ":wincmd g}" in a function doesn't work. (Gary Holloway)
解决方案: Execute the command directly, instead of putting it in the
typeahead buffer.
相关文件: src/normal.c, src/proto/normal.pro, src/window.c

Patch 6.0.058
问题: When a Cursorhold autocommand moved the cursor, the ruler wasn't
updated. (Bohdan Vlasyuk)
解决方案: Update the ruler after executing the autocommands.

相关文件: src/gui.c

Patch 6.0.059
问题: Highlighting for 'hlsearch' isn't visible in lines that are highlighted for diff highlighting. (Gary Holloway)
解决方案: Let 'hlsearch' highlighting overrule diff highlighting.
相关文件: src/screen.c

Patch 6.0.060
问题: Motif: When the tooltip is to be popped up, Vim crashes. (Gary Holloway)
解决方案: Check for a NULL return value from gui_motif_fontset2fontlist().
相关文件: src/gui_beval.c

Patch 6.0.061
问题: The toolbar buttons to load and save a session do not correctly use v:this_session.
解决方案: Check for v:this_session to be empty instead of existing.
相关文件: runtime/menu.vim

Patch 6.0.062
问题: Crash when 'verbose' is > 3 and using ":shell". (Yegappan Lakshmanan)
解决方案: Avoid giving a NULL pointer to printf(). Also output a newline and switch the cursor on.
相关文件: src/misc2.c

Patch 6.0.063
问题: When 'cpoptions' includes "$", using "cw" to type a ')' on top of the "$" doesn't update syntax highlighting after it.
解决方案: Stop displaying the "$" when typing a ')' in its position.
相关文件: src/search.c

Patch 6.0.064 (extra)
问题: The NSIS install script doesn't work with newer versions of NSIS. The diff feature doesn't work when there isn't a good diff.exe on the system.
解决方案: Replace the GetParentDir instruction by a user function.
Fix a few cosmetic problems. Use defined constants for the version number, so that it's defined in one place only.
Only accept the install directory when it ends in "vim". (Eduardo Fernandez)
Add a diff.exe and use it from the default _vimrc.
相关文件: nsis/gvim.nsi, nsis/README.txt, src/dosinst.c

Patch 6.0.065
问题: When using ":normal" in 'indentexpr' it may use redo characters before its argument. (Neil Bird)
解决方案: Save and restore the stuff buffer in ex_normal().
相关文件: src/ex_docmd.c, src/getchar.c, src/globals.h, src/structs.h

Patch 6.0.066
问题: Sometimes undo for one command is split into two undo actions. (Halim Salman)

解决方案: Don't set the undo-synced flag when reusing a line that was already saved for undo.
相关文件: src/undo.c

Patch 6.0.067
问题: if_xcmdsrv.c doesn't compile on systems where fd_set isn't defined in the usual header file (e.g., AIX). (Mark Waggoner)
解决方案: Include sys/select.h in if_xcmdsrv.c for systems that have it.
相关文件: src/if_xcmdsrv.c

Patch 6.0.068
问题: When formatting a Visually selected area with "gq" and the number of lines increases the last line may not be redrawn correctly. (Yegappan Lakshmanan)
解决方案: Correct the area to be redrawn for inserted/deleted lines.
相关文件: src/ops.c

Patch 6.0.069
问题: Using "K" on a word that includes a "!" causes a "No previous command" error, because the "!" is expanded. (Craig Jeffries)
解决方案: Put a backslash before the "!".
相关文件: src/normal.c

Patch 6.0.070
问题: Win32: The error message for a failed dynamic linking of a Perl, Ruby, Tcl and Python library is unclear about what went wrong.
解决方案: Give the name of the library or function that could not be loaded. Also for the iconv and gettext libraries when 'verbose' is set.
相关文件: src/eval.c, src/if_perl.xs, src/if_python.c, src/if_ruby.c, src/if_tcl.c, src/mbyte.c, src/os_win32.c, src/proto/if_perl.pro, src/proto/if_python.pro, src/proto/if_ruby.pro, src/proto/if_tcl.pro, src/proto/mbyte.pro

Patch 6.0.071
问题: The "iris-ansi" builtin termcap isn't very good.
解决方案: Fix the wrong entries. (David Harrison)
相关文件: src/term.c

Patch 6.0.072
问题: When 'lazyredraw' is set, a mapping that stops Visual mode, moves the cursor and starts Visual mode again causes a redraw problem. (Brian Silverman)
解决方案: Redraw both the old and the new Visual area when necessary.
相关文件: src/normal.c, src/screen.c

Patch 6.0.073 (extra)
问题: DJGPP: When using CTRL-Z to start a shell, the prompt is halfway the text. (Volker Kiefel)
解决方案: Position the system cursor before starting the shell.
相关文件: src/os_msdos.c

Patch 6.0.074
问题: When using "&" in a substitute string a multi-byte character with a trailbyte 0x5c is not handled correctly.

解决方案: Recognize multi-byte characters inside the "&" part. (Muraoka Taro)
相关文件: src/regexp.c

Patch 6.0.075
问题: When closing a horizontally split window while 'eadirection' is "hor" another horizontally split window is still resized. (Aron Griffis)
解决方案: Only resize windows in the same top frame as the window that is split or closed.
相关文件: src/main.c, src/proto/window.pro, src/window.c

Patch 6.0.076
问题: Warning for wrong pointer type when compiling.
解决方案: Use char instead of char_u pointer.
相关文件: src/version.c

Patch 6.0.077
问题: Patch 6.0.075 was incomplete.
解决方案: Fix another call to win_equal().
相关文件: src/option.c

Patch 6.0.078
问题: Using "daw" at the end of a line on a single-character word didn't include the white space before it. At the end of the file it didn't work at all. (Gavin Sinclair)
解决方案: Include the white space before the word.
相关文件: src/search.c

Patch 6.0.079
问题: When "W" is in 'cpoptions' and 'backupcopy' is "no" or "auto", can still overwrite a read-only file, because it's renamed. (Gary Holloway)
解决方案: Add a check for a read-only file before renaming the file to become the backup.
相关文件: src/fileio.c

Patch 6.0.080
问题: When using a session file that has the same file in two windows, the fileinfo() call in do_ecmd() causes a scroll and a hit-enter prompt. (Robert Webb)
解决方案: Don't scroll this message when 'shortmess' contains 'O'.
相关文件: src/ex_cmds.c

Patch 6.0.081
问题: After using ":saveas" the new buffer name is added to the Buffers menu with a wrong number. (Chauk-Mean Proum)
解决方案: Trigger BufFilePre and BufFilePost events for the renamed buffer and BufAdd for the old name (which is with a new buffer).
相关文件: src/ex_cmds.c

Patch 6.0.082
问题: When swapping screens in an xterm and there is an (error) message from the vimrc script, the shell prompt is after the message.
解决方案: Output a newline when there was output on the alternate screen.

Also when starting the GUI.
相关文件: src/main.c

Patch 6.0.083
问题: GTK: When compiled without menu support the buttons in a dialog don't have any text. (Erik Edelmann)
解决方案: Add the text also when GTK_USE_ACCEL isn't defined. And define GTK_USE_ACCEL also when not using menus.
相关文件: src/gui_gtk.c

Patch 6.0.084
问题: UTF-8: a "r" command with an argument that is a keymap for a character with a composing character can't be repeated with ".". (Raphael Finkel)
解决方案: Add the composing characters to the redo buffer.
相关文件: src/normal.c

Patch 6.0.085
问题: When 'mousefocus' is set, using "s" to go to Insert mode and then moving the mouse pointer to another window stops Insert mode, while this doesn't happen with "a" or "i". (Robert Webb)
解决方案: Reset finish_op before calling edit().
相关文件: src/normal.c

Patch 6.0.086
问题: When using "gu" the message says "~ed".
解决方案: Make the message say "changed".
相关文件: src/ops.c

Patch 6.0.087 (lang)
问题: Message translations are incorrect, which may cause a crash. (Peter Figura)
The Turkish translations needed more work and the maintainer didn't have time.
解决方案: Fix order of printf arguments. Remove %2$d constructs.
Add "-v" to msgfmt to get a warning for wrong translations.
Don't install the Turkish translations for now.
Update a few more translations.
相关文件: src/po/Makefile, src/po/af.po, src/po/cs.po, src/po/cs.cp1250.po, src/po/de.po, src/po/es.po, src/po/fr.po, src/po/it.po, src/po/ja.po, src/po/ja.sjis.po, src/po/ko.po, src/po/pl.po, src/po/sk.po, src/po/uk.po, src/po/zh_CN.UTF-8.po, src/po/zh_CN.cp936.po, src/po/zh_CN.po, src/po/zh_TW.po

Patch 6.0.088
问题: "." doesn't work after using "rx" in Visual mode. (Charles Campbell)
解决方案: Also store the replacement character in the redo buffer.
相关文件: src/normal.c

Patch 6.0.089
问题: In a C file, using "==" to align a line starting with "* " after a line with "* -" indents one space too few. (Piet Delport)
解决方案: Align with the previous line if the comment-start-string matches

there.
相关文件: src/misc1.c

Patch 6.0.090
问题: When a wrapping line does not fit in a window and 'scrolloff' is bigger than half the window height, moving the cursor left or right causes the screen to flash badly. (Lubomir Host)
解决方案: When there is not enough room to show 'scrolloff' screen lines and near the end of the line, show the end of the line.
相关文件: src/move.c

Patch 6.0.091
问题: Using CTRL-O in Insert mode, while 'virtualedit' is "all" and the cursor is after the end-of-line, moves the cursor left. (Yegappan Lakshmanan)
解决方案: Keep the cursor in the same position.
相关文件: src/edit.c

Patch 6.0.092
问题: The explorer plugin doesn't ignore case of 'suffixes' on MS-Windows. (Mike Williams)
解决方案: Match or ignore case as appropriate for the OS.
相关文件: runtime/plugin/explorer.vim

Patch 6.0.093
问题: When the Tcl library couldn't be loaded dynamically, get an error message when closing a buffer or window. (Muraoka Taro)
解决方案: Only free structures if already using the Tcl interpreter.
相关文件: src/if_tcl.c

Patch 6.0.094
问题: Athena: When clicking in the horizontal scrollbar Vim crashes. (Paul Ackersviller)
解决方案: Use the thumb size instead of the window pointer of the scrollbar (which is NULL). (David Harrison)
Also avoid that scrolling goes the wrong way in a narrow window.
相关文件: src/gui_athena.c

Patch 6.0.095
问题: Perl: Deleting lines may leave the cursor beyond the end of the file.
解决方案: Check the cursor position after deleting a line. (Serguei)
相关文件: src/if_perl.xs

Patch 6.0.096
问题: When ":saveas fname" fails because the file already exists, the file name is changed anyway and a following ":w" will overwrite the file. (Eric Carlier)
解决方案: Don't change the file name if the file already exists.
相关文件: src/ex_cmds.c

Patch 6.0.097
问题: Re-indenting in Insert mode with CTRL-F may cause a crash with a multi-byte encoding.

解决方案: Avoid using a character before the start of a line. (Sergey Vlasov)
相关文件: src/edit.c

Patch 6.0.098
问题: GTK: When using Gnome the "Search" and "Search and Replace" dialog boxes are not translated.
解决方案: Define ENABLE_NLS before including gnome.h. (Eduardo Fernandez)
相关文件: src/gui_gtk.c, src/gui_gtk_x11.c

Patch 6.0.099
问题: Cygwin: When running Vi compatible MS-DOS line endings cause trouble.
解决方案: Make the default for 'fileformats' "unix,dos" in Vi compatible mode. (Michael Schaap)
相关文件: src/option.h

Patch 6.0.100
问题: ":badd +0 test%file" causes a crash.
解决方案: Take into account that the "+0" is NUL terminated when allocating room for replacing the "%".
相关文件: src/ex_docmd.c

Patch 6.0.101
问题: ":mksession" doesn't restore editing a file that has a '#' or '%' in its name. (Wolfgang Blankenburg)
解决方案: Put a backslash before the '#' and '%'.
相关文件: src/ex_docmd.c

Patch 6.0.102
问题: When changing folds the cursor may appear halfway a closed fold. (Nam SungHyun)
解决方案: Set w_cline_folded correctly. (Yasuhiro Matsumoto)
相关文件: src/move.c

Patch 6.0.103
问题: When using 'scrollbind' a large value of 'scrolloff' will make the scroll binding stop near the end of the file. (Coen Engelbarts)
解决方案: Don't use 'scrolloff' when limiting the topline for scroll binding. (Dany StAmant)
相关文件: src/normal.c

Patch 6.0.104
问题: Multi-byte: When '$' is in 'cpoptions', typing a double-wide character that overwrites the left halve of an old double-wide character causes a redraw problem and the cursor stops blinking.
解决方案: Clear the right half of the old character. (Yasuhiro Matsumoto)
相关文件: src/edit.c, src/screen.c

Patch 6.0.105
问题: Multi-byte: In a window of one column wide, with syntax highlighting enabled a crash might happen.
解决方案: Skip getting the syntax attribute when the character doesn't fit anyway. (Yasuhiro Matsumoto)

相关文件: src/screen.c

Patch 6.0.106 (extra)
问题: Win32: When the printer font is wrong, there is no error message.
解决方案: Give an appropriate error message. (Yasuhiro Matsumoto)
相关文件: src/os_mswin.c

Patch 6.0.107 (extra)
问题: VisVim: When editing another file, a modified file may be written unexpectedly and without warning.
解决方案: Split the window if a file was modified.
相关文件: VisVim/Commands.cpp

Patch 6.0.108
问题: When using folding could try displaying line zero, resulting in an error for a NULL pointer.
解决方案: Stop decrementing w_topline when the first line of a window is in a closed fold.
相关文件: src/window.c

Patch 6.0.109
问题: XIM: When the input method is enabled, repeating an insertion with "." disables it. (Marcel Svitalsky)
解决方案: Don't store the input method status when a command comes from the stuff buffer.
相关文件: src/ui.c

Patch 6.0.110
问题: Using undo after executing "OxjAxkdd" from a register in an empty buffer gives an error message. (Gerhard Hochholzer)
解决方案: Don't adjust the bottom line number of an undo block when it's zero. Add a test for this problem.
相关文件: src/undo.c, src/testdir/test20.in, src/testdir/test20.ok

Patch 6.0.111
问题: The virtcol() function doesn't take care of 'virtualedit'.
解决方案: Add the column offset when needed. (Yegappan Lakshmanan)
相关文件: src/eval.c

Patch 6.0.112
问题: The explorer plugin doesn't sort directories with a space or special character after a directory with a shorter name.
解决方案: Ignore the trailing slash when comparing directory names. (Mike Williams)
相关文件: runtime/plugin/explorer.vim

Patch 6.0.113
问题: ":edit ~/fname" doesn't work if $HOME includes a space. Also, expanding wildcards with the shell may fail. (John Daniel)
解决方案: Escape spaces with a backslash when needed.
相关文件: src/ex_docmd.c, src/misc1.c, src/proto/misc1.pro, src/os_unix.c

Patch 6.0.114
问题: Using ":p" with fnamemodify() didn't expand "~/" or "~user/" to a

full path. For Win32 the current directory was prepended.
(Michael Geddes)
解决方案: Expand the home directory.
相关文件: src/eval.c

Patch 6.0.115 (extra)
问题: Win32: When using a dialog with a textfield it cannot scroll the
text.
解决方案: Add ES_AUTOHSCROLL to the textfield style. (Pedro Gomes)
相关文件: src/gui_w32.c

Patch 6.0.116 (extra)
问题: MS-Windows NT/2000/XP: filewritable() doesn't work correctly for
filesystems that use ACLs.
解决方案: Use ACL functions to check if a file is writable. (Mike Williams)
相关文件: src/eval.c, src/macros.h, src/os_win32.c, src/proto/os_win32.pro

Patch 6.0.117 (extra)
问题: Win32: when disabling the menu, "set lines=999" doesn't use all
the available screen space.
解决方案: Don't subtract the fixed caption height but the real menu height
from the available screen space. Also: Avoid recursion in
gui_mswin_get_menu_height().
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 6.0.118
问题: When $TMPDIR is a relative path, the temp directory is missing a
trailing slash and isn't deleted when Vim exits. (Peter Holm)
解决方案: Add the slash after expanding the directory to an absolute path.
相关文件: src/fileio.c

Patch 6.0.119 (depends on patch 6.0.116)
问题: VMS: filewritable() doesn't work properly.
解决方案: Use the same method as for Unix. (Zoltan Arpadffy)
相关文件: src/eval.c

Patch 6.0.120
问题: The conversion to html isn't compatible with XHTML.
解决方案: Quote the values. (Jess Thrysoee)
相关文件: runtime/syntax/2html.vim

Patch 6.0.121 (extra) (depends on patch 6.0.116)
问题: Win32: After patch 6.0.116 Vim doesn't compile with mingw32.
解决方案: Add an #ifdef HAVE_ACL.
相关文件: src/os_win32.c

Patch 6.0.122 (extra)
问题: Win16: Same resize problems as patch 6.0.117 fixed for Win32. And
dialog textfield problem from patch 6.0.115.
解决方案: Set old_menu_height only when used. Add ES_AUTOHSCROLL flag.
(Vince Negri)
相关文件: src/gui_w16.c

Patch 6.0.123 (depends on patch 6.0.119)

问题:　　Win16: Compilation problems.
解决方案:　Move "&&" to other lines. (Vince Negri)
相关文件:　src/eval.c

Patch 6.0.124
问题:　　When using a ":substitute" command that starts with "\="
(evaluated as an expression), "~" was still replaced with the
previous substitute string.
解决方案:　Skip the replacement when the substitute string starts with "\=".
Also adjust the documentation about doubling backslashes.
相关文件:　src/ex_cmds.c, runtime/doc/change.txt

Patch 6.0.125 (extra)
问题:　　Win32: When using the multi_byte_ime feature pressing the shift
key would be handled as if a character was entered, thus mappings
with a shifted key didn't work. (Charles Campbell)
解决方案:　Ignore pressing the shift, control and alt keys.
相关文件:　src/os_win32.c

Patch 6.0.126
问题:　　The python library was always statically linked.
解决方案:　Link the python library dynamically. (Matthias Klose)
相关文件:　src/auto/configure, src/configure.in

Patch 6.0.127
问题:　　When using a terminal that swaps screens and the Normal background
color has a different background, using an external command may
cause the color of the wrong screen to be changed. (Mark Waggoner)
解决方案:　Don't call screen_stop_highlight() in stoptermcap().
相关文件:　src/term.c

Patch 6.0.128
问题:　　When moving a vertically split window to the far left or right,
the scrollbars are not adjusted. (Scott E Lee) When 'mousefocus'
is set the mouse pointer wasn't adjusted.
解决方案:　Adjust the scrollbars and the mouse pointer.
相关文件:　src/window.c

Patch 6.0.129
问题:　　When using a very long file name, ":ls" (repeated a few times)
causes a crash. Test with "vim `perl -e 'print "A"x1000'`".
(Tejeda)
解决方案:　Terminate a string before getting its length in buflist_list().
相关文件:　src/buffer.c

Patch 6.0.130
问题:　　When using ":cprev" while the error window is open, and the new
line at the top wraps, the window isn't correctly drawn.
(Yegappan Lakshmanan)
解决方案:　When redrawing the topline don't scroll twice.
相关文件:　src/screen.c

Patch 6.0.131
问题:　　When using bufname() and there are two matches for listed buffers

and one match for an unlisted buffer, the unlisted buffer is used.
(Aric Blumer)
解决方案: When there is a match with a listed buffer, don't check for unlisted buffers.
相关文件: src/buffer.c

Patch 6.0.132
问题: When setting 'iminsert' in the vimrc and using an xterm with two screens the ruler is drawn in the wrong screen. (Igor Goldenberg)
解决方案: Only draw the ruler when using the right screen.
相关文件: src/option.c

Patch 6.0.133
问题: When opening another buffer while 'keymap' is set and 'iminsert' is zero, 'iminsert' is set to one unexpectedly. (Igor Goldenberg)
解决方案: Don't set 'iminsert' as a side effect of defining a ":lmap" mapping. Only do that when 'keymap' is set.
相关文件: src/getchar.c, src/option.c

Patch 6.0.134
问题: When completing ":set tags=" a path with an embedded space causes the completion to stop. (Sektor van Skijlen)
解决方案: Escape spaces with backslashes, like for ":set path=". Also take backslashes into account when searching for the start of the path to complete (e.g., for 'backupdir' and 'cscopeprg').
相关文件: src/ex_docmd.c, src/ex_getln.c, src/option.c, src/structs.h

Patch 6.0.135
问题: Menus that are not supposed to do anything used "<Nul>", which still produced an error beep.
When CTRL-O is mapped for Insert mode, ":amenu" commands didn't work in Insert mode.
Menu language falls back to English when $LANG ends in "@euro".
解决方案: Use "<Nop>" for a menu item that doesn't do anything, just like mappings.
Use ":anoremenu" instead of ":amenu".
Ignore "@euro" in the locale name.
相关文件: runtime/makemenu.vim, runtime/menu.vim, src/menu.c

Patch 6.0.136
问题: When completing in Insert mode, a mapping could be unexpectedly applied.
解决方案: Don't use mappings when checking for a typed character.
相关文件: src/edit.c

Patch 6.0.137
问题: GUI: When using the find or find/replace dialog from Insert mode, the input mode is stopped.
解决方案: Don't use the input method status when the main window doesn't have focus.
相关文件: src/ui.c

Patch 6.0.138
问题: GUI: When using the find or find/replace dialog from Insert mode,

the text is inserted when CTRL-O is mapped. (Andre Pang)
When opening the dialog again, a whole word search isn't recognized.
When doing "replace all" a whole word search was never done.

解决方案: Don't put a search or replace command in the input buffer, execute it directly.
Recognize "\<" and "\>" after removing "\V".
Add "\<" and "\>" also for "replace all".

相关文件: src/gui.c

Patch 6.0.139

问题: When stopping 'wildmenu' completion, the statusline of the bottom-left vertically split window isn't redrawn. (Yegappan Lakshmanan)

解决方案: Redraw all the bottom statuslines.

相关文件: src/ex_getln.c, src/proto/screen.pro, src/screen.c

Patch 6.0.140

问题: Memory allocated for local mappings and abbreviations is leaked when the buffer is wiped out.

解决方案: Clear the local mappings when deleting a buffer.

相关文件: src/buffer.c, src/getchar.c, src/proto/getchar.pro, src/vim.h

Patch 6.0.141

问题: When using ":enew" in an empty buffer, some buffer-local things are not cleared. b:keymap_name is not set.

解决方案: Clear user commands and mappings local to the buffer when re-using the current buffer. Reload the keymap.

相关文件: src/buffer.c

Patch 6.0.142

问题: When Python is linked statically, loading dynamic extensions might fail.

解决方案: Add an extra linking flag when needed. (Andrew Rodionoff)

相关文件: src/configure.in, src/auto/configure

Patch 6.0.143

问题: When a syntax item includes a line break in a pattern, the syntax may not be updated properly when making a change.

解决方案: Add the "linebreaks" argument to ":syn sync".

相关文件: runtime/doc/syntax.txt, src/screen.c, src/structs.h, src/syntax.c

Patch 6.0.144

问题: After patch 6.0.088 redoing "veU" doesn't work.

解决方案: Don't add the "U" to the redo buffer, it will be used as an undo command.

相关文件: src/normal.c

Patch 6.0.145

问题: When Vim can't read any input it might get stuck. When redirecting stdin and stderr Vim would not read commands from a file. (Servatius Brandt)

解决方案: When repeatedly trying to read a character when it's not possible, exit Vim. When stdin and stderr are not a tty, still try reading

from them, but don't do a blocking wait.
相关文件: src/ui.c

Patch 6.0.146
问题: When 'statusline' contains "%{'-'}" this results in a zero.
(Milan Vancura)
解决方案: Don't handle numbers with a minus as a number, they were not
displayed anyway.
相关文件: src/buffer.c

Patch 6.0.147
问题: It's not easy to mark a Vim version as being modified. The new
license requires this.
解决方案: Add the --modified-by argument to configure and the MODIFIED_BY
define. It's used in the intro screen and the ":version" output.
相关文件: src/auto/configure, src/configure.in, src/config.h.in,
src/feature.h, src/version.c

Patch 6.0.148
问题: After "p" in an empty line, '[goes to the second character.
(Kontra Gergely)
解决方案: Don't increment the column number in an empty line.
相关文件: src/ops.c

Patch 6.0.149
问题: The pattern "\(.\{-}\)*" causes a hang. When using a search
pattern that causes a stack overflow to be detected Vim could
still hang.
解决方案: Correctly report "operand could be empty" when using "\{-}".
Check for "out_of_stack" inside loops to avoid a hang.
相关文件: src/regexp.c

Patch 6.0.150
问题: When using a multi-byte encoding, patch 6.0.148 causes "p" to work
like "P". (Sung-Hyun Nam)
解决方案: Compute the byte length of a multi-byte character.
相关文件: src/ops.c

Patch 6.0.151
问题: Redrawing the status line and ruler can be wrong when it contains
multi-byte characters.
解决方案: Use character width and byte length correctly. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 6.0.152
问题: strtrans() could hang on an illegal UTF-8 byte sequence.
解决方案: Skip over illegal bytes. (Yasuhiro Matsumoto)
相关文件: src/charset.c

Patch 6.0.153
问题: When using (illegal) double-byte characters and Vim syntax
highlighting Vim can crash. (Yasuhiro Matsumoto)
解决方案: Increase a pointer over a character instead of a byte.
相关文件: src/regexp.c

Patch 6.0.154
问题: MS-DOS and MS-Windows: The menu entries for xxd don't work when there is no xxd in the path.
When converting back from Hex the filetype may remain "xxd" if it is not detected.
解决方案: When xxd is not in the path use the one in the runtime directory, where the install program has put it.
Clear the 'filetype' option before detecting the new value.
相关文件: runtime/menu.vim

Patch 6.0.155
问题: Mac: compilation problems in ui.c after patch 6.0.145. (Axel Kielhorn)
解决方案: Don't call mch_inchar() when NO_CONSOLE is defined.
相关文件: src/ui.c

Patch 6.0.156
问题: Starting Vim with the -b argument and two files, ":next" doesn't set 'binary' in the second file, like Vim 5.7. (Norman Diamond)
解决方案: Set the global value for 'binary'.
相关文件: src/option.c

Patch 6.0.157
问题: When defining a user command with "-complete=dir" files will also be expanded. Also, "-complete=mapping" doesn't appear to work. (Michael Naumann)
解决方案: Use the expansion flags defined with the user command.
Handle expanding mappings specifically.
相关文件: src/ex_docmd.c

Patch 6.0.158
问题: When getting the warning for a file being changed outside of Vim and reloading the file, the 'readonly' option is reset, even when the permissions didn't change. (Marcel Svitalsky)
解决方案: Keep 'readonly' set when reloading a file and the permissions didn't change.
相关文件: src/fileio.c

Patch 6.0.159
问题: Wildcard expansion for ":emenu" also shows separators.
解决方案: Skip menu separators for ":emenu", ":popup" and ":tearoff".
Also, don't handle ":tmenu" as if it was ":tearoff". And leave out the alternatives with "&" included.
相关文件: src/menu.c

Patch 6.0.160
问题: When compiling with GCC 3.0.2 and using the "-O2" argument, the optimizer causes a problem that makes Vim crash.
解决方案: Add a configure check to avoid "-O2" for this version of gcc.
相关文件: src/configure.in, src/auto/configure

Patch 6.0.161 (extra)
问题: Win32: Bitmaps don't work with signs.

解决方案: Make it possible to use bitmaps with signs. (Muraoka Taro)
相关文件: src/ex_cmds.c, src/feature.h, src/gui_w32.c, src/gui_x11.c,
src/proto/gui_w32.pro, src/proto/gui_x11.pro

Patch 6.0.162
问题: Client-server: An error message for a wrong expression appears in
the server instead of the client.
解决方案: Pass the error message from the server to the client. Also
adjust the example code. (Flemming Madsen)
相关文件: src/globals.h, src/if_xcmdsrv.c, src/main.c, src/os_mswin.c,
src/proto/if_xcmdsrv.pro, src/proto/os_mswin.pro,
runtime/doc/eval.txt, runtime/tools/xcmdsrv_client.c

Patch 6.0.163
问题: When using a GUI dialog, a file name is sometimes used like it was
a directory.
解决方案: Separate path and file name properly.
For GTK, Motif and Athena concatenate directory and file name for
the default selection.
相关文件: src/diff.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c,
src/gui_athena.c, src/gui_gtk.c, src/gui_motif.c, src/message.c

Patch 6.0.164
问题: After patch 6.0.135 the menu entries for pasting don't work in
Insert and Visual mode. (Muraoka Taro)
解决方案: Add <script> to allow script-local mappings.
相关文件: runtime/menu.vim

Patch 6.0.165
问题: Using --remote and executing locally gives unavoidable error
messages.
解决方案: Add --remote-silent and --remote-wait-silent to silently execute
locally.
For Win32 there was no error message when a server didn't exist.
相关文件: src/eval.c, src/if_xcmdsrv.c, src/main.c, src/os_mswin.c,
src/proto/if_xcmdsrv.pro, src/proto/os_mswin.pro

Patch 6.0.166
问题: GUI: There is no way to avoid dialogs to pop up.
解决方案: Add the 'c' flag to 'guioptions': Use console dialogs. (Yegappan
Lakshmanan)
相关文件: runtime/doc/options.txt, src/option.h, src/message.c

Patch 6.0.167
问题: When 'fileencodings' is "latin2" some characters in the help files
are displayed wrong.
解决方案: Force the 'fileencoding' for the help files to be "latin1".
相关文件: src/fileio.c

Patch 6.0.168
问题: ":%s/\n/#/" doesn't replace at an empty line. (Bruce DeVisser)
解决方案: Don't skip matches after joining two lines.
相关文件: src/ex_cmds.c

Patch 6.0.169
问题: When run as evim and the GUI can't be started we get stuck in a terminal without menus in Insert mode.
解决方案: Exit when using "evim" and "gvim -y" when the GUI can't be started.
相关文件: src/main.c

Patch 6.0.170
问题: When printing double-width characters the size of tabs after them is wrong. (Muraoka Taro)
解决方案: Correctly compute the column after a double-width character.
相关文件: src/ex_cmds2.c

Patch 6.0.171
问题: With 'keymodel' including "startsel", in Insert mode after the end of a line, shift-Left does not move the cursor. (Steve Hall)
解决方案: CTRL-O doesn't move the cursor left, need to do that explicitly.
相关文件: src/edit.c

Patch 6.0.172
问题: CTRL-Q doesn't replace CTRL-V after CTRL-X in Insert mode while it does in most other situations.
解决方案: Make CTRL-X CTRL-Q work like CTRL-X CTRL-V in Insert mode.
相关文件: src/edit.c

Patch 6.0.173
问题: When using "P" to insert a line break the cursor remains past the end of the line.
解决方案: Check for the cursor being beyond the end of the line.
相关文件: src/ops.c

Patch 6.0.174
问题: After using "gd" or "gD" the search direction for "n" may still be backwards. (Servatius Brandt)
解决方案: Reset the search direction to forward.
相关文件: src/normal.c, src/search.c, src/proto/search.pro

Patch 6.0.175
问题: ":help /\z(\)" doesn't work. (Thomas Koehler)
解决方案: Double the backslashes.
相关文件: src/ex_cmds.c

Patch 6.0.176
问题: When killed by a signal autocommands are still triggered as if nothing happened.
解决方案: Add the v:dying variable to allow autocommands to work differently when a deadly signal has been trapped.
相关文件: src/eval.c, src/os_unix.c, src/vim.h

Patch 6.0.177
问题: When 'commentstring' is empty and 'foldmethod' is "marker", "zf" doesn't work. (Thomas S. Urban)
解决方案: Add the marker even when 'commentstring' is empty.
相关文件: src/fold.c, src/normal.c

Patch 6.0.178
问题: Uninitialized memory read from xp_backslash field.
解决方案: Initialize xp_backslash field properly.
相关文件: src/eval.c, src/ex_docmd.c, src/ex_getln.c, src/misc1.c, src/tag.c

Patch 6.0.179
问题: Win32: When displaying UTF-8 characters may read uninitialized memory.
解决方案: Add utfc_ptr2len_check_len() to avoid reading past the end of a string.
相关文件: src/mbyte.c, src/proto/mbyte.pro, src/gui_w32.c

Patch 6.0.180
问题: Expanding environment variables in a string that ends in a backslash could go past the end of the string.
解决方案: Detect the trailing backslash.
相关文件: src/misc1.c

Patch 6.0.181
问题: When using ":cd dir" memory was leaked.
解决方案: Free the allocated memory. Also avoid an uninitialized memory read.
相关文件: src/misc2.c

Patch 6.0.182
问题: When using a regexp on multi-byte characters, could try to read a character before the start of the line.
解决方案: Don't decrement a pointer to before the start of the line.
相关文件: src/regexp.c

Patch 6.0.183
问题: Leaking memory when ":func!" redefines a function.
解决方案: Free the function name when it's not used.
相关文件: src/eval.c

Patch 6.0.184
问题: Leaking memory when expanding option values.
解决方案: Don't always copy the expanded option into allocated memory.
相关文件: src/option.c

Patch 6.0.185
问题: Crash in Vim when pasting a selection in another application, on a 64 bit machine.
解决方案: Fix the format for an Atom to 32 bits. (Peter Derr)
相关文件: src/ui.c

Patch 6.0.186
问题: X11: Three warnings when compiling the client-server code.
解决方案: Add a typecast to unsigned char.
相关文件: src/if_xcmdsrv.c

Patch 6.0.187
问题: "I" in Visual mode and then "u" reports too many changes. (Andrew

Stryker)
"I" in Visual linewise mode adjusts the indent for no apparent reason.
解决方案: Only save those lines for undo that are changed.
Don't change the indent after inserting in Visual linewise mode.
相关文件: src/ops.c

Patch 6.0.188
问题: Win32: After patch 6.0.161 signs defined in the vimrc file don't work.
解决方案: Initialize the sign icons after initializing the GUI. (Vince Negri)
相关文件: src/gui.c, src/gui_x11.c

Patch 6.0.189
问题: The size of the Visual area isn't always displayed when scrolling ('ruler' off, 'showcmd' on). Also not when using a search command. (Sylvain Hitier)
解决方案: Redisplay the size of the selection after showing the mode.
相关文件: src/screen.c

Patch 6.0.190
问题: GUI: when 'mouse' is empty a click with the middle button still moves the cursor.
解决方案: Paste at the cursor position instead of the mouse position.
相关文件: src/normal.c

Patch 6.0.191
问题: When no servers are available serverlist() gives an error instead of returning an empty string. (Hari Krishna)
解决方案: Don't give an error message.
相关文件: src/eval.c

Patch 6.0.192
问题: When 'virtualedit' is set, "ylj" goes to the wrong column. (Andrew Nikitin)
解决方案: Reset the flag that w_virtcol is valid when moving the cursor back to the start of the operated area.
相关文件: src/normal.c

Patch 6.0.193
问题: When 'virtualedit' is set, col(".") after the end of the line should return one extra.
解决方案: Add one to the column.
相关文件: src/eval.c

Patch 6.0.194
问题: "--remote-silent" tries to send a reply to the client, like it was "--remote-wait".
解决方案: Properly check for the argument.
相关文件: src/main.c

Patch 6.0.195
问题: When 'virtualedit' is set and a search starts in virtual space

":call search('x')" goes to the wrong position. (Eric Long)
解决方案: Reset coladd when finding a match.
相关文件: src/search.c

Patch 6.0.196
问题: When 'virtualedit' is set, 'selection' is "exclusive" and visually selecting part of a tab at the start of a line, "x" joins it with the previous line. Also, when the selection spans more than one line the whole tab is deleted.
解决方案: Take coladd into account when adjusting for 'selection' being "exclusive". Also expand a tab into spaces when deleting more than one line.
相关文件: src/normal.c, src/ops.c

Patch 6.0.197
问题: When 'virtualedit' is set and 'selection' is "exclusive", "v$x" doesn't delete the last character in the line. (Eric Long)
解决方案: Don't reset the inclusive flag. (Helmut Stiegler)
相关文件: src/normal.c

Patch 6.0.198
问题: When 'virtualedit' is set and 'showbreak' is not empty, moving the cursor over the line break doesn't work properly. (Eric Long)
解决方案: Make getviscol() and getviscol2() use getvvcol() to obtain the virtual cursor position. Adjust coladvance() and oneleft() to skip over the 'showbreak' characters.
相关文件: src/edit.c, src/misc2.c

Patch 6.0.199
问题: Multi-byte: could use iconv() after calling iconv_end(). (Yasuhiro Matsumoto)
解决方案: Stop converting input and output stream after calling iconv_end().
相关文件: src/mbyte.c

Patch 6.0.200
问题: A script that starts with "#!perl" isn't recognized as a Perl filetype.
解决方案: Ignore a missing path in a script header. Also, speed up recognizing scripts by simplifying the patterns used.
相关文件: runtime/scripts.vim

Patch 6.0.201
问题: When scrollbinding and doing a long jump, switching windows jumps to another position in the file. Scrolling a few lines at a time is OK. (Johannes Zellner)
解决方案: When setting w_topline reset the flag that indicates w_botline is valid.
相关文件: src/diff.c

Patch 6.0.202
问题: The "icon=" argument for the menu command to define a toolbar icon with a file didn't work for GTK. (Christian J. Robinson)
For Motif and Athena a full path was required.
解决方案: Search the icon file using the specified path. Expand environment

variables in the file name.
相关文件: src/gui_gtk.c, src/gui_x11.c

Patch 6.0.203
问题: Can change 'fileformat' even though 'modifiable' is off.
(Servatius Brandt)
解决方案: Correct check for kind of set command.
相关文件: src/option.c

Patch 6.0.204
问题: ":unlet" doesn't work for variables with curly braces. (Thomas
Scott Urban)
解决方案: Handle variable names with curly braces properly. (Vince Negri)
相关文件: src/eval.c

Patch 6.0.205 (extra)
问题: "gvim -f" still forks when using the batch script to start Vim.
解决方案: Add an argument to "start" to use a foreground session (Michael
Geddes)
相关文件: src/dosinst.c

Patch 6.0.206
问题: Unix: if expanding a wildcard in a file name results in a
wildcard character and there are more parts in the path with a
wildcard, it is expanded again.
Windows: ":edit \[abc]" could never edit the file "[abc]".
解决方案: Don't expand wildcards in already expanded parts.
Don't remove backslashes used to escape the special meaning of a
wildcard; can edit "[abc]" if '[' is removed from 'isfname'.
相关文件: src/misc1.c, src/os_unix.c

Patch 6.0.207 (extra)
问题: Win32: The shortcuts and start menu entries let Vim startup in the
desktop directory, which is not very useful.
解决方案: Let shortcuts start Vim in $HOME or $HOMEDIR$HOMEPATH.
相关文件: src/dosinst.c

Patch 6.0.208
问题: GUI: When using a keymap and the cursor is not blinking, CTRL-^ in
Insert mode doesn't directly change the cursor color. (Alex
Solow)
解决方案: Force a redraw of the cursor after CTRL-^.
相关文件: src/edit.c

Patch 6.0.209
问题: GUI GTK: After selecting a 'guifont' with the font dialog there
are redraw problems for multi-byte characters.
解决方案: Separate the font dialog from setting the new font name to avoid
that "*" is used to find wide and bold fonts.
When redrawing extra characters for the bold trick, take care of
UTF-8 characters.
相关文件: src/gui.c, src/gui_gtk_x11.c, src/option.c, src/proto/gui.pro,
src/proto/gui_gtk_x11.pro

Patch 6.0.210
问题: After patch 6.0.167 it's no longer possible to edit a help file in another encoding than latin1.
解决方案: Let the "++enc=" argument overrule the encoding.
相关文件: src/fileio.c

Patch 6.0.211
问题: When reading a file fails, the buffer is empty, but it might still be possible to write it with ":w" later. The original file is lost then. (Steve Amerige)
解决方案: Set the 'readonly' option for the buffer.
相关文件: src/fileio.c

Patch 6.0.212
问题: GUI GTK: confirm("foo", "") causes a crash.
解决方案: Don't make a non-existing button the default. Add a default "OK" button if none is specified.
相关文件: src/eval.c, src/gui_gtk.c

Patch 6.0.213
问题: When a file name contains unprintable characters, CTRL-G and other commands don't work well.
解决方案: Turn unprintable into printable characters. (Yasuhiro Matsumoto)
相关文件: src/buffer.c, src/charset.c

Patch 6.0.214
问题: When there is a buffer without a name, empty entries appear in the jumplist saved in the viminfo file.
解决方案: Don't write jumplist entries without a file name.
相关文件: src/mark.c

Patch 6.0.215
问题: After using "/" from Visual mode the Paste menu and Toolbar entries don't work. Pasting with the middle mouse doesn't work and modeless selection doesn't work.
解决方案: Use the command line mode menus and use the mouse like in the command line.
相关文件: src/gui.c, src/menu.c, src/ui.c

Patch 6.0.216
问题: After reloading a file, displayed in another window than the current one, which was changed outside of Vim the part of the file around the cursor set by autocommands may be displayed, but jumping back to the original cursor position when entering the window again.
解决方案: Restore the topline of the window.
相关文件: src/fileio.c

Patch 6.0.217
问题: When getting help from a help file that was used before, an empty unlisted buffer remains in the buffer list. (Eric Long)
解决方案: Wipe out the buffer used to do the tag jump from.
相关文件: src/buffer.c, src/ex_cmds.c, src/proto/buffer.pro

Patch 6.0.218
问题: With explorer plugin: "vim -o filename dirname" doesn't load the explorer window until entering the window.
解决方案: Call s:EditDir() for each window after starting up.
相关文件: runtime/plugin/explorer.vim

Patch 6.0.219
问题: ":setlocal" and ":setglobal", without arguments, display terminal options. (Zdenek Sekera)
解决方案: Skip terminal options for these two commands.
相关文件: src/option.c

Patch 6.0.220
问题: After patch 6.0.218 get a beep on startup. (Muraoka Taro)
解决方案: Don't try going to another window when there isn't one.
相关文件: runtime/plugin/explorer.vim

Patch 6.0.221
问题: When using ":bdel" and all other buffers are unloaded the lowest numbered buffer is jumped to instead of the most recent one. (Dave Cecil)
解决方案: Prefer an unloaded buffer from the jumplist.
相关文件: src/buffer.c

Patch 6.0.222
问题: When 'virtualedit' is set and using autoindent, pressing Esc after starting a new line leaves behind part of the autoindent. (Helmut Stiegler)
解决方案: After deleting the last char in the line adjust the cursor position in del_bytes().
相关文件: src/misc1.c, src/ops.c

Patch 6.0.223
问题: When splitting a window that contains the explorer, hitting CR on a file name gives error messages.
解决方案: Set the window variables after splitting the window.
相关文件: runtime/plugin/explorer.vim

Patch 6.0.224
问题: When 'sidescroll' and 'sidescrolloff' are set in a narrow window the text may jump left-right and the cursor is displayed in the wrong position. (Aric Blumer)
解决方案: When there is not enough room, compute the left column for the window to put the cursor in the middle.
相关文件: src/move.c

Patch 6.0.225
问题: In Visual mode "gk" gets stuck in a closed fold. (Srinath Avadhanula)
解决方案: Behave differently in a closed fold.
相关文件: src/normal.c

Patch 6.0.226
问题: When doing ":recover file" get the ATTENTION prompt.

After recovering the same file five times get a read error or a crash. (Alex Davis)
解决方案: Set the recoverymode flag before setting the file name.
Correct the amount of used memory for the size of block zero.
相关文件: src/ex_docmd.c

Patch 6.0.227 (extra)
问题: The RISC OS port has several problems.
解决方案: Update the makefile and fix some of the problems. (Andy Wingate)
相关文件: src/Make_ro.mak, src/os_riscos.c, src/os_riscos.h, src/proto/os_riscos.pro, src/search.c

Patch 6.0.228
问题: After putting text in Visual mode the '] mark is not at the end of the put text.
Undo doesn't work properly when putting a word into a Visual selection that spans more than one line.
解决方案: Correct the '] mark for the deleting the Visually selected text.
#ifdef code that depends on FEAT_VISUAL properly.
Also fix that "d" crossing line boundary puts '[just before deleted text.
Fix undo by saving all deleted lines at once.
相关文件: src/ex_docmd.c, src/globals.h, src/normal.c, src/ops.c, src/structs.h, src/vim.h

Patch 6.0.229
问题: Multi-byte: With 'm' in 'formatoptions', formatting doesn't break at a multi-byte char followed by an ASCII char, and the other way around. (Muraoka Taro)
When joining lines a space is inserted between multi-byte characters, which is not always wanted.
解决方案: Check for multi-byte character before and after the breakpoint.
Don't insert a space before or after a multi-byte character when joining lines and the 'M' flag is in 'formatoptions'. Don't insert a space between multi-byte characters when the 'B' flag is in 'formatoptions'.
相关文件: src/edit.c, src/ops.c, src/option.h

Patch 6.0.230
问题: The ":" used as a motion after an operator is exclusive, but sometimes it should be inclusive.
解决方案: Make the "v" in between an operator and motion toggle inclusive/exclusive. (Servatius Brandt)
相关文件: runtime/doc/motion.txt, src/normal.c

Patch 6.0.231
问题: "gd" and "gD" don't work when the variable matches in a comment just above the match to be found. (Servatius Brandt)
解决方案: Continue searching in the first column below the comment.
相关文件: src/normal.c

Patch 6.0.232
问题: "vim --version" prints on stderr while "vim --help" prints on stdout.

解决方案: Make "vim --version" use stdout.
相关文件: runtime/doc/starting.txt, src/globals.h, src/main.c, src/message.c

Patch 6.0.233
问题: "\1\{,8}" in a regexp is not allowed, but it should work, because
there is an upper limit. (Jim Battle)
解决方案: Allow using "\{min,max}" after an atom that can be empty if there
is an upper limit.
相关文件: src/regexp.c

Patch 6.0.234
问题: It's not easy to set the cursor position without modifying marks.
解决方案: Add the cursor() function. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 6.0.235
问题: When writing a file and renaming the original file to make the
backup, permissions could change when setting the owner.
解决方案: Only set the owner when it's needed and set the permissions again
afterwards.
When 'backupcopy' is "auto" check that the owner and permissions
of a newly created file can be set properly.
相关文件: src/fileio.c

Patch 6.0.236
问题: ":edit" without argument should move cursor to line 1 in Vi
compatible mode.
解决方案: Add 'g' flag to 'cpoptions'.
相关文件: runtime/doc/options.txt, src/ex_docmd.c, src/option.h

Patch 6.0.237
问题: In a C file, using the filetype plugin, re-indenting a comment
with two spaces after the middle "*" doesn't align properly.
解决方案: Don't use a middle entry from a start/middle/end to line up with
the start of the comment when the start part doesn't match with
the actual comment start.
相关文件: src/misc1.c

Patch 6.0.238
问题: Using a ":substitute" command with a substitute() call in the
substitution expression causes errors. (Srinath Avadhanula)
解决方案: Save and restore pointers when doing substitution recursively.
相关文件: src/regexp.c

Patch 6.0.239
问题: Using "A" to append after a Visually selected block which is after
the end of the line, spaces are inserted in the wrong line and
other unexpected effects. (Michael Naumann)
解决方案: Don't advance the cursor to the next line.
相关文件: src/ops.c

Patch 6.0.240
问题: Win32: building with Python 2.2 doesn't work.
解决方案: Add support for Python 2.2 with dynamic linking. (Paul Moore)

相关文件: src/if_python.c

Patch 6.0.241
问题: Win32: Expanding the old value of an option that is a path that starts with a backslash, an extra backslash is inserted.
解决方案: Only insert backslashes where needed.
Also handle multi-byte characters properly when removing backslashes.
相关文件: src/option.c

Patch 6.0.242
问题: GUI: On a system with an Exceed X server sometimes get a "Bad Window" error. (Tommi Maekitalo)
解决方案: When forking, use a pipe to wait in the parent for the child to have done the setsid() call.
相关文件: src/gui.c

Patch 6.0.243
问题: Unix: "vim --version" outputs a NL before the last line instead of after it. (Charles Campbell)
解决方案: Send the NL to the same output stream as the text.
相关文件: src/message.c, src/os_unix.c, src/proto/message.pro

Patch 6.0.244
问题: Multi-byte: Problems with (illegal) UTF-8 characters in menu and file name (e.g., icon text, status line).
解决方案: Correctly handle unprintable characters. Catch illegal UTF-8 characters and replace them with <xx>. Truncating the status line wasn't done correctly at a multi-byte character. (Yasuhiro Matsumoto)
Added correct_cmdspos() and transchar_byte().
相关文件: src/buffer.c, src/charset.c, src/ex_getln.c, src/gui.c, src/message.c, src/screen.c, src/vim.h

Patch 6.0.245
问题: After using a color scheme, setting the 'background' option might not work. (Peter Horst)
解决方案: Disable the color scheme if it switches 'background' back to the wrong value.
相关文件: src/option.c

Patch 6.0.246
问题: ":echomsg" didn't use the highlighting set by ":echohl". (Gary Holloway)
解决方案: Use the specified attributes for the message. (Yegappan Lakshmanan)
相关文件: src/eval.c

Patch 6.0.247
问题: GTK GUI: Can't use gvim in a kpart widget.
解决方案: Add the "--echo-wid" argument to let Vim echo the window ID on stdout. (Philippe Fremy)
相关文件: runtime/doc/starting.txt, src/globals.h, src/gui_gtk_x11.c, src/main.c

Patch 6.0.248
问题:　　When using compressed help files and 'encoding' isn't "latin1",
　　Vim converts the help file before decompressing. (David Reviejo)
解决方案:　Don't convert a help file when 'binary' is set.
相关文件:　src/fileio.c

Patch 6.0.249
问题:　　"vim -t edit -c 'sta ex_help'" doesn't move cursor to edit().
解决方案:　Don't set the cursor on the first line for "-c" arguments when
　　there also is a "-t" argument.
相关文件:　src/main.c

Patch 6.0.250 (extra)
问题:　　Macintosh: Various problems when compiling.
解决方案:　Various fixes, mostly #ifdefs. (Dany St. Amant)
相关文件:　src/gui_mac.c, src/main.c, src/misc2.c, src/os_mac.h,
　　src/os_mac.pbproj/project.pbxproj, src/os_unix.c

Patch 6.0.251 (extra)
问题:　　Macintosh: menu shortcuts are not very clear.
解决方案:　Show the shortcut with the Mac clover symbol. (raindog)
相关文件:　src/gui_mac.c

Patch 6.0.252
问题:　　When a user function was defined with "abort", an error that is
　　not inside if/endif or while/endwhile doesn't abort the function.
　　(Servatius Brandt)
解决方案:　Don't reset did_emsg when the function is to be aborted.
相关文件:　src/ex_docmd.c

Patch 6.0.253
问题:　　When 'insertmode' is set, after "<C-O>:edit file" the next <C-O>
　　doesn't work. (Benji Fisher)　<C-L> has the same problem.
解决方案:　Reset need_start_insertmode once in edit().
相关文件:　src/edit.c

Patch 6.0.254 (extra)
问题:　　Borland C++ 5.5: Checking for stack overflow doesn't work
　　correctly.　Matters when using a complicated regexp.
解决方案:　Remove -N- from Make_bc5.mak. (Yasuhiro Matsumoto)
相关文件:　src/Make_bc5.mak

Patch 6.0.255 (extra) (depends on patch 6.0.116 and 6.0.121)
问题:　　Win32: ACL support doesn't work well on Samba drives.
解决方案:　Add a check for working ACL support. (Mike Williams)
相关文件:　src/os_win32.c

Patch 6.0.256 (extra)
问题:　　Win32: ":highlight Comment guifg=asdf" does not give an error
　　message. (Randall W. Morris)　Also for other systems.
解决方案:　Add gui_get_color() to give one error message for all systems.
相关文件:　src/gui.c, src/gui_amiga.c, src/gui_athena.c, src/gui_motif.c,
　　src/gui_riscos.c, src/gui_x11.c, src/gui_gtk_x11.c,

src/proto/gui.pro, src/syntax.c

Patch 6.0.257
问题: Win32: When 'mousefocus' is set and there is a BufRead autocommand, after the dialog for permissions changed outside of Vim: 'mousefocus' stops working. (Robert Webb)
解决方案: Reset need_mouse_correct after checking timestamps.
相关文件: src/fileio.c

Patch 6.0.258
问题: When 'scrolloff' is 999 and there are folds, the text can jump up and down when moving the cursor down near the end of the file. (Lubomir Host)
解决方案: When putting the cursor halfway the window start counting lines at the end of a fold.
相关文件: src/move.c

Patch 6.0.259
问题: MS-DOS: after editing the command line the cursor shape may remain like in Insert mode. (Volker Kiefel)
解决方案: Reset the cursor shape after editing the command line.
相关文件: src/ex_getln.c

Patch 6.0.260
问题: GUI: May crash while starting up when giving an error message for missing color. (Servatius Brandt)
解决方案: Don't call gui_write() when still starting up. Don't give error message for empty color name. Don't use 't_vb' while the GUI is still starting up.
相关文件: src/fileio.c, src/gui.c, src/misc1.c, src/ui.c

Patch 6.0.261
问题: nr2char() and char2nr() don't work with multi-byte characters.
解决方案: Use 'encoding' for these functions. (Yasuhiro Matsumoto)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 6.0.262 (extra)
问题: Win32: IME doesn't work properly. OnImeComposition() isn't used at all.
解决方案: Adjust various things for IME.
相关文件: src/globals.h, src/gui_w32.c, src/mbyte.c, src/proto/ui.pro, src/structs.h, src/ui.c

Patch 6.0.263
问题: GTK: When a dialog is closed by the window manager, Vim hangs. (Christian J. Robinson)
解决方案: Use GTK_WIDGET_DRAWABLE() instead of GTK_WIDGET_VISIBLE().
相关文件: src/gui_gtk.c, src/gui_gtk_x11.c

Patch 6.0.264
问题: The amount of virtual memory is used to initialize 'maxmemtot', which may be much more than the amount of physical memory, resulting in a lot of swapping.
解决方案: Get the amount of physical memory with sysctl(), sysconf() or

sysinfo() when possible.
相关文件: src/auto/configure, src/configure.in, src/config.h.in, src/os_unix.c, src/os_unix.h

Patch 6.0.265
问题: Win32: Using backspace while 'fkmap' is set causes a crash. (Jamshid Oasjmoha)
解决方案: Don't try mapping special keys.
相关文件: src/farsi.c

Patch 6.0.266
问题: The rename() function deletes the file if the old and the new name are the same. (Volker Kiefel)
解决方案: Don't do anything if the names are equal.
相关文件: src/fileio.c

Patch 6.0.267
问题: UTF-8: Although 'isprint' says a character is printable, utf_char2cells() still considers it unprintable.
解决方案: Use vim_isprintc() for characters upto 0x100. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 6.0.268 (extra) (depends on patch 6.0.255)
问题: Win32: ACL check crashes when using forward slash in file name.
解决方案: Improve the check for the path in the file name.
相关文件: src/os_win32.c

Patch 6.0.269
问题: Unprintable characters in a file name may cause problems when using the 'statusline' option or when 'buftype' is "nofile".
解决方案: call trans_characters() for the resulting statusline. (Yasuhiro Matsumoto)
相关文件: src/buffer.c, src/screen.c, src/charset.c

Patch 6.0.270 (depends on patch 6.0.267)
问题: A tab causes UTF-8 text to be displayed in the wrong position. (Ron Aaron)
解决方案: Correct utf_char2cells() again.
相关文件: src/mbyte.c

Patch 6.1a.001 (extra)
问题: 32bit DOS: copying text to the clipboard may cause a crash. (Jonathan D Johnston)
解决方案: Don't copy one byte too much in SetClipboardData().
相关文件: src/os_msdos.c

Patch 6.1a.002
问题: GTK: On some configurations, when closing a dialog from the window manager, Vim hangs.
解决方案: Catch the "destroy" signal. (Aric Blumer)
相关文件: src/gui_gtk.c

Patch 6.1a.003
问题: Multi-byte: With UTF-8 double-wide char and 'virtualedit' set:

yanking in Visual mode doesn't include the last byte. (Eric Long)
解决方案: Don't add a space for a double-wide character.
相关文件: src/ops.c

Patch 6.1a.004 (extra)
问题: MINGW: undefined type. (Ron Aaron)
解决方案: Make GetCompositionString_inUCS2() static.
相关文件: src/gui_w32.c, src/gui_w48.c, src/proto/gui_w32.pro

Patch 6.1a.005 (extra)
问题: Win32: ":hardcopy" doesn't work after ":hardcopy!". (Jonathan Johnston)
解决方案: Don't keep the driver context when using ":hardcopy!". (Vince Negri)
相关文件: src/os_mswin.c

Patch 6.1a.006
问题: multi-byte: after setting 'encoding' the window title might be wrong.
解决方案: Force resetting the title. (Yasuhiro Matsumoto)
相关文件: src/option.c

Patch 6.1a.007
问题: Filetype detection for "*.inc" doesn't work.
解决方案: Use a ":let" command. (David Schweikert)
相关文件: runtime/filetype.vim

Patch 6.1a.008 (extra)
问题: Win32: ACL detection for network shares doesn't work.
解决方案: Include the trailing (back)slash in the root path. (Mike Williams)
相关文件: src/os_win32.c

Patch 6.1a.009
问题: When using "\@<=" or "\@<!" in a pattern, a "\1" may refer to a () part that follows, but it generates an error message.
解决方案: Allow a forward reference when there is a following "\@<=" or "\@<!".
相关文件: runtime/doc/pattern.txt, src/regexp.c

Patch 6.1a.010
问题: When using ":help" and opening a new window, the alternate file isn't set.
解决方案: Set the alternate file to the previously edited file.
相关文件: src/ex_cmds.c

Patch 6.1a.011
问题: GTK: ":set co=77", change width with the mouse, ":set co=77" doesn't resize the window. (Darren Hiebert)
解决方案: Set the form size after handling a resize event.
相关文件: src/gui_gtk_x11.c

Patch 6.1a.012
问题: GTK: The file browser always returns a full path. (Lohner)
解决方案: Shorten the file name if possible.

相关文件: src/gui_gtk.c

Patch 6.1a.013
问题: When using "=~word" in 'cinkeys' or 'indentkeys', the case of the last character of the word isn't ignored. (Raul Segura Acevedo)
解决方案: Ignore case when checking the last typed character.
相关文件: src/edit.c

Patch 6.1a.014
问题: After patch 6.1a.006 can't compile without the title feature.
解决方案: Add an #ifdef.
相关文件: src/option.c

Patch 6.1a.015
问题: MS-Windows: When expanding a file name that contains a '[' or '{' an extra backslash is inserted. (Raul Segura Acevedo)
解决方案: Avoid adding the backslash.
相关文件: src/ex_getln.c

Patch 6.1a.016
问题: Completion after ":language" doesn't include "time". (Raul Segura Acevedo)
解决方案: Add the alternative to the completions.
相关文件: src/ex_cmds2.c

Patch 6.1a.017
问题: Clicking the mouse in the top row of a window where the first line doesn't fit moves the cursor to the wrong column.
解决方案: Add the skipcol also for the top row of a window.
相关文件: src/ui.c

Patch 6.1a.018
问题: When 'scrolloff' is one and the window height is one, "gj" can put the cursor above the window. (Raul Segura Acevedo)
解决方案: Don't let skipcol become bigger than the cursor column.
相关文件: src/move.c

Patch 6.1a.019
问题: When using a composing character on top of an ASCII character, the "l" command clears the composing character. Only when 'ruler' and 'showcmd' are off. (Raphael Finkel)
解决方案: Don't move the cursor by displaying characters when there are composing characters.
相关文件: src/screen.c

Patch 6.1a.020
问题: GTK: after patch 6.1a.011 resizing with the mouse doesn't always work well for small sizes. (Adrien Beau)
解决方案: Use another way to avoid the problem with ":set co=77".
相关文件: src/gui_gtk_x11.c

Patch 6.1a.021
问题: Several Syntax menu entries are wrong or confusing.
解决方案: Rephrase and correct the menu entries. (Adrien Beau)

相关文件: runtime/makemenu.vim, runtime/menu.vim

Patch 6.1a.022
问题: A tags file might be used twice on case insensitive systems.
(Rick Swanton)
解决方案: Don't use the same file name twice in the default for the 'tags'
option. Ignore case when comparing names of already visited
files.
相关文件: src/misc2.c, src/option.c

Patch 6.1a.023
问题: When starting the GUI get "C" characters echoed in the terminal.
解决方案: Don't try sending a clear-screen command while the GUI is starting
up.
相关文件: src/screen.c

Patch 6.1a.024
问题: In other editors CTRL-F is often used for a find dialog.
解决方案: In evim use CTRL-F for the find dialog.
相关文件: runtime/evim.vim

Patch 6.1a.025
问题: The choices for the fileformat dialog can't be translated.
解决方案: Add g:menutrans_fileformat_choices. (Adrien Beau)
相关文件: runtime/menu.vim

Patch 6.1a.026
问题: Indenting Java files is wrong with "throws", "extends" and
"implements" clauses.
解决方案: Update the Java indent script.
相关文件: runtime/indent/java.vim

Patch 6.1a.027
问题: A few Syntax menu entries missing or incorrect.
解决方案: Add and correct the menu entries. (Adrien Beau)
Shorten a few menus to avoid they become too long.
相关文件: runtime/makemenu.vim, runtime/menu.vim

Patch 6.1a.028
问题: XIM: problems with feedback and some input methods.
解决方案: Use iconv for calculating the cells. Remove the queue for
key_press_event only when text was changed. (Yasuhiro Matsumoto)
相关文件: src/globals.h, src/mbyte.c, src/screen.c

Patch 6.1a.029
问题: After patch 6.1a.028 can't compile GTK version with XIM but
without multi-byte chars.
解决方案: Add an #ifdef. (Aschwin Marsman)
相关文件: src/mbyte.c

Patch 6.1a.030
问题: With double-byte encodings toupper() and tolower() may have wrong
results.
解决方案: Skip double-byte characters. (Eric Long)

相关文件: src/eval.c

Patch 6.1a.031
问题: Accessing the 'balloondelay' variable may cause a crash.
解决方案: Make the variable for 'balloondelay' a long. (Olaf Seibert)
相关文件: src/option.h

Patch 6.1a.032 (extra)
问题: Some menu files used a wrong encoding name for "scriptencoding".
解决方案: Move the translations to a separate file, which is sourced after setting "scriptencoding".
Also add Czech menu translations in ASCII and update the other encodings.
相关文件: runtime/lang/menu_cs_cz.iso_8859-1.vim,
runtime/lang/menu_cs_cz.iso_8859-2.vim,
runtime/lang/menu_czech_czech_republic.1250.vim,
runtime/lang/menu_czech_czech_republic.1252.vim,
runtime/lang/menu_czech_czech_republic.ascii.vim,
runtime/lang/menu_de_de.iso_8859-1.vim,
runtime/lang/menu_de_de.latin1.vim,
runtime/lang/menu_fr_fr.iso_8859-1.vim,
runtime/lang/menu_fr_fr.latin1.vim,
runtime/lang/menu_french_france.1252.vim,
runtime/lang/menu_german_germany.1252.vim,
runtime/lang/menu_ja_jp.euc-jp.vim,
runtime/lang/menu_ja_jp.utf-8.vim,
runtime/lang/menu_japanese_japan.932.vim

Patch 6.1a.033
问题: XIM: doesn't reset input context.
解决方案: call xim_reset() with im_set_active(FALSE). (Takuhiro Nishioka)
相关文件: src/mbyte.c

Patch 6.1a.034 (extra)
问题: Win32: The ACL checks for a readonly file still don't work well.
解决方案: Remove the ACL checks, go back to how it worked in Vim 6.0.
相关文件: src/os_win32.c

Patch 6.1a.035
问题: multi-byte: When using ":sh" in the GUI, typed and displayed multi-byte characters are not handled correctly.
解决方案: Deal with multi-byte characters to and from the shell. (Yasuhiro Matsumoto) Also handle UTF-8 composing characters.
相关文件: src/os_unix.c

Patch 6.1a.036
问题: GTK: the save-yourself event was not handled.
解决方案: Catch the save-yourself event and preserve swap files. (Neil Bird)
相关文件: src/gui_gtk_x11.c

Patch 6.1a.037
问题: The MS-Windows key mapping doesn't include CTRL-S for saving. (Vlad Sandrini)
解决方案: Map CTRL-S to ":update".

相关文件: runtime/mswin.vim

Patch 6.1a.038
问题: Solaris: Including both sys/sysctl.h and sys/sysinfo.h doesn't work. (Antonio Colombo)
解决方案: Don't include sys/sysinfo.h when not calling sysinfo().
相关文件: src/os_unix.c

Patch 6.1a.039
问题: Not all visual basic files are recognized.
解决方案: Add checks to catch *.ctl files. (Raul Segura Acevedo)
相关文件: runtime/filetype.vim

Patch 6.1a.040
问题: A *.pl file is recognized as Perl, but it could be a prolog file.
解决方案: Check the first non-empty line. (Kontra Gergely)
相关文件: runtime/filetype.vim

Patch 6.1a.041
问题: When pressing the left mouse button in the command line and them moving the mouse upwards, nearly all the text is selected.
解决方案: Don't try extending a modeless selection when there isn't one.
相关文件: src/ui.c

Patch 6.1a.042
问题: When merging files, ":diffput" and ":diffget" are used a lot, but they require a lot of typing.
解决方案: Add "dp" for ":diffput" and "do" for ":diffget".
相关文件: runtime/doc/diff.txt, src/diff.c, src/normal.c, src/proto/diff.pro

Patch 6.1b.001 (extra)
问题: Checking for wildcards in a path does not handle multi-byte characters with a trail byte which is a wildcard.
解决方案: Handle multi-byte characters correctly. (Muraoka Taro)
相关文件: src/os_amiga.c, src/os_mac.c, src/os_msdos.c, src/os_mswin.c, src/os_unix.c

Patch 6.1b.002
问题: A regexp that ends in "\{" is not flagged as an error. May cause a stack overflow when 'incsearch' is set. (Gerhard Hochholzer)
解决方案: Handle a missing "}" as an error.
相关文件: src/regexp.c

Patch 6.1b.003 (extra)
问题: The RISC OS GUI doesn't compile.
解决方案: Include changes since Vim 5.7. (Andy Wingate)
相关文件: src/Make_ro.mak, src/gui_riscos.c, src/os_riscos.c, src/os_riscos.h, src/proto/gui_riscos.pro

Patch 6.1b.004
问题: col("'>") returns a negative number for linewise selection. (Neil Bird)
解决方案: Don't add one to MAXCOL.

相关文件: src/eval.c

Patch 6.1b.005
问题: Using a search pattern that causes an out-of-stack error while 'hlsearch' is set keeps giving the hit-Enter prompt. A search pattern that takes a long time delays typing when 'incsearch' is set.
解决方案: Stop 'hlsearch' highlighting when the regexp causes an error. Stop searching for 'incsearch' when a character is typed.
相关文件: src/globals.h, src/message.c, src/screen.c, src/search.c, src/vim.h

Patch 6.1b.006
问题: When entering a composing character on the command line with CTRL-V, the text isn't redrawn correctly.
解决方案: Redraw the text under and after the cursor.
相关文件: src/ex_getln.c

Patch 6.1b.007
问题: When the cursor is in the white space between two sentences, "dis" deletes the first character of the following sentence, "das" deletes a space after the sentence.
解决方案: Backup the cursor one character in these situations.
相关文件: src/search.c

Patch 6.1b.008
问题: *.xsl files are not recognized as xslt but xml. Monk files are not recognized.
解决方案: Delete the duplicate line for *.xsl. (Johannes Zellner) Recognize monk files.
相关文件: runtime/filetype.vim

Patch 6.1b.009
问题: Can't always compile small features and then adding eval feature, "sandbox" is undefined. (Axel Kielhorn)
解决方案: Always define "sandbox" when the eval feature is used.
相关文件: src/globals.h

Patch 6.1b.010 (extra)
问题: When compiling gvimext.cpp with MSVC 4.2 get a number of warnings.
解决方案: Change "true" to "TRUE". (Walter Briscoe)
相关文件: GvimExt/gvimext.cpp

Patch 6.1b.011
问题: When using a very long string for confirm(), can't quit the displaying at the more prompt. (Hari Krishna Dara)
解决方案: Jump to the end of the message to show the choices.
相关文件: src/message.c

Patch 6.1b.012
问题: Multi-byte: When 'showbreak' is set and a double-wide character doesn't fit at the right window edge the cursor gets stuck there. Using cursor-left gets stuck when 'virtualedit' is set. (Eric Long)

解决方案: Fix the way the extra ">" character is counted when 'showbreak' is
           set. Don't correct cursor for virtual editing on a double-wide
           character.
相关文件: src/charset.c, src/edit.c

Patch 6.1b.013
问题: A user command that partly matches with a buffer-local user
           command and matches full with a global user command unnecessarily
           gives an 'ambiguous command' error.
解决方案: Find the full global match even after a partly local match.
相关文件: src/ex_docmd.c

Patch 6.1b.014
问题: EBCDIC: switching mouse events off causes garbage on screen.
           Positioning the cursor in the GUI causes garbage.
解决方案: Insert an ESC in the terminal code. (Ralf Schandl)
           Use "\b" instead of "\010" for KS_LE.
相关文件: src/os_unix.c, src/term.c

Patch 6.1b.015
问题: Vimtutor has a typo. Get a warning for "tempfile" if it
           doesn't exist.
解决方案: Move a quote to the end of a line. (Max Ischenko)
           Use "mktemp" first, more systems have it.
相关文件: src/vimtutor

Patch 6.1b.016
问题: GTK: loading a fontset that works partly, Vim might hang or crash.
解决方案: Avoid that char_width becomes zero. (Yasuhiro Matsumoto)
相关文件: src/gui_gtk_x11.c

Patch 6.1b.017
问题: GUI: When using ":shell" and there is a beep, nothing happens.
解决方案: Call vim_beep() to produce the beep from the shell. (Yasuhiro
           Matsumoto)
相关文件: src/message.c

Patch 6.1b.018 (depends on 6.1b.006)
问题: When entering the encryption key, special keys may still reveal
           the typed characters.
解决方案: Make sure stars are used or nothing is shown in all cases.
相关文件: src/digraph.c, src/getchar.c, src/ex_getln.c

Patch 6.1b.019 (depends on 6.1b.005)
问题: A search pattern that takes a long time slows down typing when
           'incsearch' is set.
解决方案: Pass SEARCH_PEEK to dosearch().
相关文件: src/ex_getln.c

Patch 6.1b.020
问题: When using the matchit plugin, "%" finds a match on the "end" of a
           ":syntax region" command in Vim scripts.
解决方案: Skip over ":syntax region" commands by setting b:match_skip.
相关文件: runtime/ftplugin/vim.vim

Patch 6.1b.021
问题: when 'mousefocus' is set, CTRL-W CTRL-] sometimes doesn't warp the pointer to the new window. (Robert Webb)
解决方案: Don't reset need_mouse_correct when checking the timestamp of a file.
相关文件: src/fileio.c

Patch 6.1b.022
问题: With lots of folds "j" does not obey 'scrolloff' properly. (Srinath Avadhanula)
解决方案: Go to end of the fold before counting context lines.
相关文件: src/move.c

Patch 6.1b.023
问题: On MS-Windows system() may cause checking timestamps, because Vim loses and gains input focus, while this doesn't happen on Unix.
解决方案: Don't check timestamps while system() is busy.
相关文件: src/ex_cmds2.c, src/fileio.c, src/globals.h, src/misc1.c

Patch 6.1b.024 (extra)
问题: Gettext 0.11 complains that "sjis" is not a standard name.
解决方案: Use "cp932" instead.
相关文件: src/po/sjiscorr.c

Patch 6.1b.025 (extra)
问题: Win32: When closing gvim while it is minimized and has a changed file, the file-changed dialog pops up in a corner of the screen.
解决方案: Put the dialog in the middle of the screen.
相关文件: src/gui_w48.c

Patch 6.1b.026
问题: When 'diffopt' contains 'iwhite' but not 'icase': differences in case are not highlighted properly. (Gerhard Hochholzer)
解决方案: Don't ignore case when ignoring white space differences.
相关文件: src/diff.c

Patch 6.1b.027
问题: "vim --remote +" may cause a crash.
解决方案: Check for missing file name argument. (Martin Kahlert)
相关文件: src/main.c

Patch 6.1b.028 (extra)
问题: Win16: Can't compile after patch 6.1b.025.
解决方案: Add code specifically for Win16. (Vince Negri)
相关文件: src/gui_w48.c

Patch 6.1b.029
问题: Win32: When a directory on an NTFS partition is read/execute (no delete,modify,write) and the file has modify rights, trying to write the file deletes it. Making the file read/write/execute (not delete) solves it. (Mark Canup)
解决方案: Use the Unix code to check for a writable directory. If not, then make a backup copy and overwrite the file.

相关文件: src/fileio.c

Patch 6.1b.030 (extra)
问题: Mac: small mistake in the build script and prototypes.
解决方案: Fix the build script and add the prototypes. (Axel Kielhorn)
相关文件: src/os_mac.build, src/gui_mac.c

Patch 6.1b.031 (extra)
问题: Win32 GUI: ":set guifont=*" doesn't set 'guifont' to the resulting font name. (Vlad Sandrini)
解决方案: Put the code back in gui_mch_init_font() to form the font name out of the logfont.
相关文件: src/gui_w48.c

Patch 6.1b.032
问题: Athena: Setting a color scheme before the GUI has started causes a crash. (Todd Blumer)
解决方案: Don't try using color names that haven't been set yet.
相关文件: src/gui_athena.c

Patch 6.1b.033
问题: When using a count after a ":s" command may get ml_get errors. (Dietmar Lang)
解决方案: Check that the resulting range does not go past the end of the buffer.
相关文件: src/ex_cmds.c

Patch 6.1b.034
问题: After sourcing mswin.vim, when using <C-S-Right> after auto-indenting and then <Del>, get warning for allocating ridiculous amount of memory. (Dave Delgreco)
解决方案: Adjust the start of the Visual area when deleting the auto-indent.
相关文件: src/edit.c

Patch 6.1b.035
问题: When using evim, dropping a file on Vim and then double clicking on a word, it is changed to "i". (Merlin Hansen)
解决方案: Reset need_start_insertmode after editing the file.
相关文件: src/ex_docmd.c

6.2 版 本 version-6.2

本小节讨论 6.2 和 6.3 版本之间的改进。

这是一个漏洞修正版。也有一些新特性。

主要的新特性:
- 支持 GTK 2。(Daniel Elstner)
- 支持编辑阿拉伯文本。(Nadim Shaikli 和 Isam Bayazidi)
- ":try" 命令和例外处理。(Servatius Brandt)
- 支持 neXtaw GUI 工具集 (非常像 Athena)。(Alexey Froloff)
- Win32 的 Cscope 支持。(Khorev Sergey)

- 支持各种 8 位编码的 PostScript 打印。(Mike Williams)

改动 changed-6.2

删除 scheme 缩进文件，内部的 Lisp 缩进现在已经很好了。

GvimEXt、OleVim 和 VisVim 目录移到 "src" 目录。和 xxd 的处理方式更加一致。

VisVim.dll 文件安装在顶层目录，紧靠着 gvimext.dll，而不是在 "VisVim" 子目录。修正 NSIS 卸载它时找到错误目录的问题。

删除 art 缩进文件。它什么都不做。

submatch() 返回的换行符过去使用 CR 而不是 LF。

Win32 的 makefile 更加统一，也能编译 gvimext.dll。(Dan Sharp)

'cindent': "//" 注释和前一行的 "//" 注释对齐。(Helmut Stiegler)

以前只有 xterm 和相近的终端上跟踪父窗口组件 (widget) 以寻找标题和图标文字。现在对所有的终端模拟器都这么做。

"%" 匹配时可以识别反斜杠。'cpoptions' 的 'M' 标志位禁止如此。(Haakon Riiser)

删除 Turbo C 的 Make_tcc.mak makefile。已经不能用而我们可能也没有办法让它好起来 (编译器内存溢出)。

即使文档说明的是关键字，"[CTRL-D" 过去使用 'isident' 来寻找匹配。现在改为 'iskeyword'。也适用于其它搜索头文件里宏定义单词的命令，包括 ":dsearch"、"[D" 和 "[d"。

使得 'keywordprg' 为全局-局部变量。(Christian Robinson)

缺省打开 Netbeans 接口。反转 configure 的参数，从 "--enable-netbeans" 到 "--disable-netbeans"。

新增 added-6.2

新选项:

```
'arabic'
'arabicshape'
'ambiwidth'
'autochdir'
'casemap'
'copyindent'
'cscopequickfix'
'preserveindent'
'printencoding'
'rightleftcmd'
'termbidi'
```

'toolbariconsize'
'winfixheight'

新键盘映射表:
塞尔维亚语 (Aleksandar Veselinovic)
中文拼音 (Fredrik Roubert)
世界语 (Antoine J. Mechelynck)

新语法文件:
Valgrind (Roger Luethi)
Smarty template (Manfred Stienstra)
MySQL (Kenneth Pronovici)
RockLinux package description (Piotr Esden-Tempski)
MMIX (Dirk Huesken)
gkrellmrc (David Necas)
Tilde (Tobias Rundtrom)
Logtalk (Paulo Moura)
PLP (Juerd Waalboer)
fvwm2m4 (David Necas)
IPfilter (Hendrik Scholz)
fstab (Radu Dineiu)
Quake (Nikolai Weibull)
Occam (Mario Schweigler)
lpc (Shizhu Pan)
Exim conf (David Necas)
EDIF (Artem Zankovich)
.cvsrc (Nikolai Weibull)
.fetchmailrc (Nikolai Weibull)
GNU gpg (Nikolai Weibull)
Grub (Nikolai Weibull)
Modconf (Nikolai Weibull)
RCS (Dmitry Vasiliev)
Art (Dorai Sitaram)
Renderman Interface Bytestream (Andrew J Bromage)
Mailcap (Doug Kearns)
Subversion commit file (Dmitry Vasiliev)
Microsoft IDL (Vadim Zeitlin)
WildPackets EtherPeek Decoder (Christopher Shinn)
Spyce (Rimon Barr)
Resolv.conf (Radu Dineiu)
A65 (Clemens Kirchgatterer)
sshconfig 和 sshdconfig (David Necas)
Cheetah 和 HTMLCheetah (Max Ischenko)
Packet filter (Camiel Dobbelaar)

新缩进文件:
Eiffel (David Clarke)
Tilde (Tobias Rundtrom)
Occam (Mario Schweigler)
Art (Dorai Sitaram)
PHP (Miles Lott)
Dylan (Brent Fulgham)

新教程翻译:

斯洛伐克语 (Lubos Celko)
希腊语 (Christos Kontas)
德语 (Joachim Hofmann)
挪威语 (yvind Holm)

新文件类型插件:
Occam (Mario Schweigler)
Art (Dorai Sitaram)
ant.vim、aspvbs.vim、config.vim、csc.vim、csh.vim、dtd.vim、html.vim,
jsp.vim、pascal.vim、php.vim、sgml.vim、sh.vim、svg.vim、tcsh.vim,
xhtml.vim、xml.vim、xsd.vim。(Dan Sharp)

新编译器插件:
Checkstyle (Doug Kearns)
g77 (Ralf Wildenhues)
fortran (Johann-Guenter Simon)
Xmllint (Doug Kearns)
Ruby (Tim Hammerquist)
Modelsim vcom (Paul Baleme)

新菜单翻译:
巴西 (葡萄牙) 语 (José de Paula)
英国英语 (Mike Williams)
UTF-8 韩语 (Nam SungHyun)
挪威语 (yvind Holm)
塞尔维亚语 (Aleksandar Jelenak)

挪威语的新消息翻译。(yvind Holm)

新色彩方案:
desert (Hans Fugal)

阿拉伯专用特性。'arabicshape'、'termbidi'、'arabic' 和 'rightleftcmd' 选项。
(Nadim Shaikli 和 Isam Bayazidi)

支持 neXtaw GUI toolkit，基本类似于 Athena。(Alexey Froloff)

Win32: cscope 支持。(Khorev Sergey)

VMS: 若干文档和 makefile 的改进。(Zoltan Arpadffy)

探索器插件加入 "x" 键: 执行缺省动作。(Yasuhiro Matsumoto)

用 MingW 编译 gvimext.dll。(Rene de Zwart)

加入 "tohtml.vim" 插件。它定义了 ":TOhtml" 用户命令，提供把文本转换为 HTML 的
简易方法。

加入 ":try" / ":catch" / ":finally" / ":endtry" 命令。加入 E999 编号到所有的错
误信息，以便能用编号捕获它们。(Servatius Brandt)
ex_docmd.c 部分内容移到新的 ex_eval.c 源文件。

加入 GTK+ 2.2.x 的支持 (Daniel Elstner)
加入 "~" 寄存器: 拖放文本。

加入 'toolbariconsize' 选项。
运行 lint 时加入 -Dalloca，处理 alloca() 原型的问题。

在错误窗口里选择项目用于跳转时，努力寻找一个普通窗口（不是预览窗口）来显示该文件。

支持若干 8 位编码的 PostScript 打印。(Mike Williams)

inputdialog() 接受第三个参数，取消对话框时使用。可以用来区别取消和什么都不输入的情况。

加入 Aap 构造文件 (recipe)。用于更新 Vim 到最新的版本、编译和安装。

'cinoptions' 的 "/" 选项：注释行的额外缩进。(Helmut Stiegler)

Vim 变量 "v:register" 和 setreg()、getreg() 和 getregtype() 函数。(Michael Geddes)

'cpoptions' 的 "v" 标志位：插入模式使用退格保留屏幕上的文本。(Phillip Vandry)

Dosinst.exe 也会在 "GvimExt" 目录里寻找 gvimext.dll。用于在 "src" 目录安装时进行测试。

支持忽略大小写的排序方式的标签文件。(Flemming Madsen)

补全开头的路径部分里的通配符，如 "../*/Makefile" 时，只列出最后一个部分 ("Makefile")。支持自定义的命令行补全。(Flemming Madsen)

"rxvt" 也被识别为和 xterm 类似的终端。(Tomas Styblo)

适当的 X11 会话处理。修正流行的桌面系统不使用 WM_SAVE_YOURSELF 事件的问题。(Neil Bird)
不用于 Gnome 2，它有自己的处理方式。

支持 Borland 3 makefile 的 BOR、DEBUG 和 SPAWNO 参数。(Walter Briscoe)

支持打印时的换页。'printoptions' 加入 "formfeed" 字段。(Mike Williams)

Mac OSX：多语言支持：iconv 和 gettext。(Muraoka Taro，Axel Kielhorn)

模式里的 "\Z" 标志位：忽略组合字符的差异。(Ron Aaron)

加入 'preserveindent' 和 'copyindent' 选项。它们使用已存在的空白字符，而不是尽可能多地使用制表。(Chris Leishman)

更新 Unicode 表到 Unicode 4.0。(Raphael Finkel)

支持 rxvt 里的鼠标滚轮。(AIDA Shinra)

Win32：加入 ":8" 文件名修饰符以得到短文件名。Test50 测试 Win32 系统的 ":8" 扩展。(Michael Geddes)

'cscopequickfix' 选项：Cscope 命令的 quickfix 窗口。同时清理代码，给出消息。(Khorev Sergey)

GUI: 鼠标位置支持多于 222 列。

":stopinsert" 命令: 不回到插入模式。

调试模式的 "interrupt" 命令。用于模拟 CTRL-C。(Servatius Brandt)

修正 fixed-6.2

Removed a few unused #defines from config.h.in, os_os2_cfg.h and os_vms_conf.h.

The Vim icons in PNG format didn't have a transparent background. (Greg Roelofs)

Fixed a large number of spelling mistakes in the docs. (Adri Verhoef)

The #defines for prototype generation were causing trouble. Changed them to typedefs.

A new version of libintl.h uses __asm__, which confuses cproto. Define a dummy __asm__ macro.

When 'virtualedit' is set can't move to halfway an unprintable character. Cripples CTRL-V selection. (Taro Muraoka)
Allow moving to halfway an unprintable character. Don't let getvvcol() change the pos->coladd argument.

When a tab wraps to the next line, 'listchars' is set and 'foldcolumn' is non-zero, only one character of the foldcolumn is highlighted. (Muraoka Taro)

When using ":catch" without an argument Vim crashes. (Yasuhiro Matsumoto)
When no argument given use the ".*" pattern.

Win32: When gvim.exe is started from a shortcut with the window style property set to maximize Vim doesn't start with a maximized window. (Yasuhiro Matsumoto) Open the window with the default size and don't call ShowWindow() again when it's already visible. (Helmut Stiegler)

gui_gtk.c used MAX, but it's undefined to avoid a conflict with system header files.

Win32: When closing a window from a mapping some pixels remain on the statusline. (Yasuhiro Matsumoto)

A column number in an errorformat that goes beyond the end of the line may cause a crash.

":throw 'test'" crashes Vim. (Yasuhiro Matsumoto)

The file selector's scrollbar colors are not set after doing a ":hi Scrollbar guifg=color". And the file selector's colors are not changed by the colorscheme command. (David Harrison)

Motif: When compiling with FEAT_FOOTER defined, the text area gets a few pixels extra space on the right. Remove the special case in gui_get_base_width(). (David Harrison)

Using CTRL-R CTRL-P in Insert mode puts the ’] mark in the wrong position. (Helmut Stiegler)

When ’formatoptions’ includes "awct" a non-comment wasn’t auto-formatted.

Using a "--cmd" argument more than 10 times caused a crash.

DEC style mouse support didn’t work if the page field is not empty. (Uribarri)

"vim -l one two" did only set ’lisp’ in the first file. Vi does it for every file.

":set tw<" didn’t work. Was checking for ’^’ instead of ’<’.

In ":hardcopy > %.ps" the "%" was not expanded to the current filename.

Made ":redraw" also update the Visual area.

When a not implemented command, such as ":perl", has wrong arguments the less important error was reported, giving the user the idea the command could work.

On non-Unix systems autocommands for writing did not attempt a match with the short file name, causing a pattern like "a/b" to fail.

VMS: e_screenmode was not defined and a few other fixes for VMS. (Zoltan Arpadffy)

redraw_msg() depended on FEAT_ARABIC instead of FEAT_RIGHTLEFT. (Walter Briscoe)

Various changes for the PC Makefiles. (Walter Briscoe)

Use _truename() instead of our own code to expand a file name into a full path. (Walter Briscoe)

Error in filetype check for /etc/modutils. (Lubomir Host)

Cscope interface: allocated a buffer too small.

Win16: remove a trailing backslash from a path when obtaining the permission flags. (Vince Negri)

When searching for tags with case ignored Vim could hang.

When searching directories with a stopdir could get a crash. Did not re-allocate enough memory. (Vince Negri)

A user command may cause a crash. Don’t use the command index when it’s

negative. (Vince Negri)

putenv() didn't work for MingW and Cygwin. (Dan Sharp)

Many functions were common between os_msdos.c and os_win16.c. Use os_msdos.c for compiling the Win16 version and remove the functions from os_win16.c. (Vince Negri)

For terminals that behave like an xterm but didn't have a name that is recognized, the window title would not always be set.

When syntax highlighting is off ":hardcopy" could still attempt printing colors.

Crash when using ":catch" without an argument. (Servatius Brandt)

Win32: ":n #" doubled the backslashes.

Fixed Arabic shaping for the command line. (Nadim Shaikli)

Avoid splitting up a string displayed on the command line into individual characters, it breaks Arabic shaping.

Updated Cygwin and MingW makefiles to use more dependencies. (Dan Sharp)

2html.vim didn't work with 'nomagic' set.

When a local argument list is used and doing ":only" Vim could crash later. (Muraoka Taro)

When using "%P" in 'statusline' and the fillchar is "-", a percentage of 3% could result in "-3%". Also avoid changing a space inside a filename to the fill character.

MSwin: Handling of backslashes and double quotes for command line arguments was not like what other applications do. (Walter Briscoe)

Test32 sometimes didn't work, because test11.out was written as TEST11.OUT.

Avoid pointer conversions warnings for Borland C 5.5 in dosinst.c and uninstal.c.

More improvements for Make_bc3.mak file. (Walter Briscoe)

When ":syn sync linebreaks=1" is used, editing the first line caused a redraw of the whole screen.

Making translated messages didn't work, if_perl.xs wasn't found. (Vlad Sandrini)

Motif and Athena: moving Vim to the foreground didn't uniconify it. Use XMapRaised() instead of XRaiseWindow(). (Srikanth Sankaran)

When using ":ptag" in a window where 'scrollbind' is set the preview window

would also have 'scrollbind' set. Also reset 'foldcolumn' and 'diff'.

Various commands that split a window took over 'scrollbind', which is hardly
ever desired. Esp. for "q:" and ":copen". Mostly reset 'scrollbind' when
splitting a window.

When 'shellslash' is set in the vimrc file the first entry of ":scriptnames"
would still have backslashes. Entries in the quickfix list could also have
wrong (back)slashes.

Win32: printer dialog texts were not translated. (Yasuhiro Matsumoto)

When using a multi-byte character with a K_SPECIAL byte or a special key code
with "--remote-send" the received byte sequence was mangled. Put it in the
typeahead buffer instead of the input buffer.

Win32: The cursor position was incorrect after changing cursor shape.
(Yasuhiro Matsumoto).

Win32: When 'encoding' is not the current codepage the title could not be set
to non-ascii characters.

"vim -d scp://machine/file1 scp://machine/file2" did not work, there was only
one window. Fixed the netrw plugin not to wipe out the buffer if it is
displayed in other windows.

"/$" caused "e" in last column of screen to disappear, a highlighted blank was
displayed instead.

":s/ *\ze\n//e" removed the line break and introduced arbitrary text. Was
using the line count including what matched after the "\ze".

Using the "c" flag with ":s" changed the behavior when a line break is
replaced and "\@<=" is used. Without "c" a following match was not found.

":%s/\vA@<=\nB@=//gce" got stuck on "A\nB" when entering "n".

VMS: add HAVE_STRFTIME in the config file. (Zoltan Arpadffy)

When a delete prompts if a delete should continue when yanking is not
possible, restore msg_silent afterwards.

":sign" did not complain about a missing argument.

When adding or deleting a sign 'hlsearch' highlighting could disappear.
Use the generic functions for updating signs.

On MS-Windows NT, 2K and XP don't use command.com but cmd.exe for testing.
Makes the tests work on more systems.

In the DOS tests don't create "/tmp" to avoid an error.

Mac classic: Problems with reading files with CR vs CR/LF. Rely on the
library version of fgets() to work correctly for Metrowerks 2.2. (Axel

Kielhorn)

When typing a password a "*" was shown for each byte instead of for each character. Added multi-byte handling to displaying the stars. (Yasuhiro Matsumoto)

When using Perl 5.6 accessing $curbuf doesn't work. Add an #ifdef to use different code for 5.6 and 5.8. (Dan Sharp)

MingW and Cygwin: Don't strip the debug executable. (Dan Sharp)

An assignment to a variable with curlies that includes "==" doesn't work. Skip over the curlies before searching for an "=". (Vince Negri)

When cancelling the selection of alternate matching tags the tag stack index could be advanced too far, resulting in an error message when using CTRL-T.

Patch 6.1.001
问题: When formatting UTF-8 text it might be wrapped at a space that is followed by a composing character. (Raphael Finkel)
Also correct a display error for removing a composing char on top of a space.
解决方案: Check for a composing character on a space.
相关文件: src/edit.c, src/misc1.c, src/screen.c

Patch 6.1.002 (extra)
问题: Win32: after a ":popup" command the mouse pointer stays hidden.
解决方案: Unhide the mouse pointer before showing the menu.
相关文件: src/gui_w48.c

Patch 6.1.003
问题: When 'laststatus' is zero and there is a vertical split, the vertical separator is drawn in the command line. (Srikant Sankaran)
解决方案: Don't draw the vertical separator where there is no statusline.
相关文件: src/screen.c

Patch 6.1.004
问题: Unicode 3.2 changes width and composing of a few characters. (Markus Kuhn)
解决方案: Adjust the Unicode functions for the character width and composing characters.
相关文件: src/mbyte.c

Patch 6.1.005
问题: When using more than 50 items in 'statusline' Vim might crash. (Steve Hall)
解决方案: Increment itemcnt in check_stl_option(). (Flemming Madsen)
相关文件: src/option.c

Patch 6.1.006
问题: When using "P" in Visual mode to put linewise selected text, the wrong text is deleted. (Jakub Turski)

解决方案: Put the text before the Visual area and correct the text to be
           deleted for the inserted lines.
           Also fix that "p" of linewise text in Visual block mode doesn't
           work correctly.
相关文件: src/normal.c, src/ops.c

Patch 6.1.007
问题: Using ":filetype plugin off" when filetype plugins were never
           enabled causes an error message. (Yiu Wing)
解决方案: Use ":silent!" to avoid the error message.
相关文件: runtime/ftplugof.vim

Patch 6.1.008
问题: The "%" command doesn't ignore \" inside a string, it's seen as
           the end of the string. (Ken Clark)
解决方案: Skip a double quote preceded by an odd number of backslashes.
相关文件: src/search.c

Patch 6.1.009
问题: Vim crashes when using a huge number for the maxwid value in a
           statusline. (Robert M. Nowotniak)
解决方案: Check for an overflow that makes maxwid negative.
相关文件: src/buffer.c

Patch 6.1.010
问题: Searching backwards for a question mark with "?\?" doesn't work.
           (Alan Isaac) Same problem in ":s?\??" and ":g?\??".
解决方案: Change the "\?" in a pattern to "?" when using "?" as delimiter.
相关文件: src/ex_cmds.c, src/ex_docmd.c, src/proto/regexp.pro, src/regexp.c,
           src/search.c, src/syntax.c, src/tag.c

Patch 6.1.011
问题: XIM: doesn't work correctly when 'number' is set. Also, a focus
           problem when selecting candidates.
解决方案: Fix the XIM problems. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c, src/screen.c

Patch 6.1.012
问题: A system() call might fail if fread() does CR-LF to LF
           translation.
解决方案: Open the output file in binary mode. (Pavol Huhas)
相关文件: src/misc1.c

Patch 6.1.013
问题: Win32: The default for 'printexpr' doesn't work when there are
           special characters in 'printdevice'.
解决方案: Add double quotes around the device name. (Mike Williams)
相关文件: runtime/doc/option.txt, src/option.c

Patch 6.1.014
问题: An operator like "r" used in Visual block mode doesn't use
           'virtualedit' when it's set to "block".
解决方案: Check for 'virtualedit' being active in Visual block mode when the
           operator was started.

相关文件: src/ex_docmd.c, src/globals.h, src/misc2.c, src/normal.c,
           src/ops.c, src/undo.c

Patch 6.1.015
问题: After patch 6.1.014 can't compile with tiny features. (Christian
           J. Robinson)
解决方案: Add the missing define of virtual_op.
相关文件: src/vim.h

Patch 6.1.016 (extra)
问题: Win32: Outputting Hebrew or Arabic text might have a problem with
           reversing.
解决方案: Replace the RevOut() function with ETO_IGNORELANGUAGE. (Ron Aaron)
相关文件: src/gui_w32.c

Patch 6.1.017
问题: Cygwin: After patch 6.1.012 Still doesn't do binary file I/O.
           (Pavol Juhas)
解决方案: Define BINARY_FILE_IO for Cygwin.
相关文件: src/os_unix.h

Patch 6.1.018
问题: Error message when using cterm highlighting. (Leonardo Di Lella)
解决方案: Remove a backslash before a question mark.
相关文件: runtime/syntax/cterm.vim

Patch 6.1.019 (extra)
问题: Win32: File name is messed up when editing just a drive name.
           (Walter Briscoe)
解决方案: Append a NUL after the drive name. (Vince Negri)
相关文件: src/os_win32.c

Patch 6.1.020
问题: col("'>") returns a huge number after using Visual line mode.
解决方案: Return the length of the line instead.
相关文件: src/eval.c

Patch 6.1.021 (depends on patch 6.1.009)
问题: Vim crashes when using a huge number for the minwid value in a
           statusline. (Robert M. Nowotniak)
解决方案: Check for an overflow that makes minwid negative.
相关文件: src/buffer.c

Patch 6.1.022
问题: Grabbing the status line above the command-line window works like
           the bottom status line was grabbed. (Jim Battle)
解决方案: Make it possible to grab the status line above the command-line
           window, so that it can be resized.
相关文件: src/ui.c

Patch 6.1.023 (extra)
问题: VMS: running tests doesn't work properly.
解决方案: Adjust the makefile. (Zoltan Arpadffy)
相关文件: src/testdir/Make_vms.mms

Patch 6.1.024
问题: When header files use a new syntax for declaring functions, Vim can't figure out missing prototypes properly.
解决方案: Accept braces around a function name. (M. Warner Losh)
相关文件: src/osdef.sh

Patch 6.1.025
问题: Five messages for "vim --help" don't start with a capital. (Vlad Sandrini)
解决方案: Make the messages consistent.
相关文件: src/main.c

Patch 6.1.026
问题: *.patch files are not recognized as diff files. In a script a "VAR=val" argument after "env" isn't ignored. PHP scripts are not recognized.
解决方案: Add *.patch for diff filetypes. Ignore "VAR=val". Recognize PHP scripts. (Roman Neuhauser)
相关文件: runtime/filetype.vim, runtime/scripts.vim

Patch 6.1.027
问题: When 'foldcolumn' is non-zero, a special character that wraps to the next line disturbs the foldcolumn highlighting. (Yasuhiro Matsumoto)
解决方案: Only use the special highlighting when drawing text characters.
相关文件: src/screen.c

Patch 6.1.028
问题: Client-server: When a --remote-expr fails, Vim still exits with status zero.
解决方案: Exit Vim with a non-zero status to indicate the --remote-expr failed. (Thomas Scott Urban)
相关文件: src/main.c

Patch 6.1.029
问题: When 'encoding' is an 8-bit encoding other than "latin1", editing a utf-8 or other Unicode file uses the wrong conversion. (Jan Fedak)
解决方案: Don't use Unicode to latin1 conversion for 8-bit encodings other than "latin1".
相关文件: src/fileio.c

Patch 6.1.030
问题: When CTRL-N is mapped in Insert mode, it is also mapped after CTRL-X CTRL-N, while it is not mapped after CTRL-X CTRL-F. (Kontra Gergely)
解决方案: Don't map CTRL-N after CTRL-X CTRL-N. Same for CTRL-P.
相关文件: src/getchar.c

Patch 6.1.031
问题: Cygwin: Xxd could read a file in text mode instead of binary mode.
解决方案: Use "rb" or "rt" when needed. (Pavol Juhas)
相关文件: src/xxd/xxd.c

Patch 6.1.032
问题: Can't specify a quickfix file without jumping to the first error.
解决方案: Add the ":cgetfile" command. (Yegappan Lakshmanan)
相关文件: runtime/doc/index.txt, runtime/doc/quickfix.txt, src/ex_cmds.h, src/quickfix.c

Patch 6.1.033
问题: GUI: When the selection is lost and the Visual highlighting is changed to underlining, the cursor is left in a different position. (Christian Michon)
解决方案: Update the cursor position after redrawing the selection.
相关文件: src/ui.c

Patch 6.1.034
问题: A CVS diff file isn't recognized as diff filetype.
解决方案: Skip lines starting with "? " before checking for an "Index:" line.
相关文件: runtime/scripts.vim

Patch 6.1.035 (extra, depends on 6.1.016)
问题: Win32: Outputting Hebrew or Arabic text might have a problem with reversing on MS-Windows 95/98/ME.
解决方案: Restore the RevOut() function and use it in specific situations only. (Ron Aaron)
相关文件: src/gui_w32.c

Patch 6.1.036
问题: This command may cause a crash: ":v/./,//-j". (Ralf Arens)
解决方案: Compute the right length of the regexp when it's empty.
相关文件: src/search.c

Patch 6.1.037
问题: When 'lazyredraw' is set, pressing "q" at the hit-enter prompt causes an incomplete redraw and the cursor isn't positioned. (Lubomir Host)
解决方案: Overrule 'lazyredraw' when do_redraw is set.
相关文件: src/main.c, src/screen.c

Patch 6.1.038
问题: Multi-byte: When a ":s" command contains a multi-byte character where the trail byte is '~' the text is messed up.
解决方案: Properly skip multi-byte characters in regtilde() (Muraoka Taro)
相关文件: src/regexp.c

Patch 6.1.039
问题: When folds are defined and the file is changed outside of Vim, reloading the file doesn't update the folds. (Anders Schack-Nielsen)
解决方案: Recompute the folds after reloading the file.
相关文件: src/fileio.c

Patch 6.1.040
问题: When changing directory for expanding a file name fails there is no error message.

解决方案: Give an error message for this situation. Don't change directory if we can't return to the original directory.
相关文件: src/diff.c, src/ex_docmd.c, src/globals.h, src/misc1.c, src/os_unix.c

Patch 6.1.041
问题: ":mkvimrc" doesn't handle a mapping that has a leading space in the rhs. (Davyd Ondrejko)
解决方案: Insert a CTRL-V before the leading space. Also display leading and trailing white space in <> form.
相关文件: src/getchar.c, src/message.c

Patch 6.1.042
问题: "vim -r" doesn't show all matches when 'wildignore' removes swap files. (Steve Talley)
解决方案: Keep all matching swap file names.
相关文件: src/memline.c

Patch 6.1.043
问题: After patch 6.1.040 a few warnings are produced.
解决方案: Add a type cast to "char *" for mch_chdir(). (Axel Kielhorn)
相关文件: src/diff.c, src/ex_docmd.c.c, src/misc1.c, src/os_unix.c

Patch 6.1.044 (extra)
问题: GUI: When using the find/replace dialog with text that contains a slash, an invalid substitute command is generated.
On Win32 a find doesn't work when 'insertmode' is set.
解决方案: Escape slashes with a backslash.
Make the Win32, Motif and GTK gui use common code for the find/replace dialog.
Add the "match case" option for Motif and GTK.
相关文件: src/feature.h, src/proto/gui.pro, src/gui.c, src/gui.h, src/gui_motif.c, src/gui_gtk.c, src/gui_w48.c

Patch 6.1.045
问题: In Visual mode, with lots of folds and 'scrolloff' set to 999, moving the cursor down near the end of the file causes the text to jump up and down. (Lubomir Host)
解决方案: Take into account that the cursor may be on the last line of a closed fold.
相关文件: src/move.c

Patch 6.1.046
问题: X11 GUI: ":set lsp=2 gcr=n-v-i:hor1-blinkon0" draws a black rectangle. ":set lsp=2 gcr=n-v-i:hor10-blinkon0" makes the cursor disappear. (Nam SungHyun)
解决方案: Correctly compute the height of the horizontal cursor.
相关文件: src/gui_gtk_x11.c, src/gui_x11.c

Patch 6.1.047
问题: When skipping commands after an error was encountered, expressions for ":if", ";elseif" and ":while" are still evaluated.
解决方案: Skip the expression after an error. (Servatius Brandt)
相关文件: src/ex_docmd.c

Patch 6.1.048
问题: Unicode 3.2 changes were missing a few Hangul Jamo characters.
解决方案: Recognize more characters as composing characters. (Jungshik Shin)
相关文件: src/mbyte.c

Patch 6.1.049 (extra)
问题: On a 32 bit display a valid color may cause an error message,
because its pixel value is negative. (Chris Paulson-Ellis)
解决方案: Check for -11111 instead of the color being negative.
Don't add one to the pixel value, -1 may be used for white.
相关文件: src/globals.h, src/gui.c, src/gui.h, src/gui_amiga.c,
src/gui_athena.c, src/gui_beos.cc, src/gui_gtk_x11.c,
src/gui_mac.c, src/gui_motif.c, src/gui_photon.c,
src/gui_riscos.c, src/gui_w16.c, src/gui_w32.c, src/gui_w48.c,
src/gui_x11.c, src/mbyte.c, src/syntax.c

Patch 6.1.050 (depends on 6.1.049)
问题: After patch 6.1.049 the non-GUI version doesn't compile.
解决方案: Add an #ifdef FEAT_GUI. (Robert Stanton)
相关文件: src/syntax.c

Patch 6.1.051 (depends on 6.1.044)
问题: Doesn't compile with GUI and small features.
解决方案: Adjust the #if for ga_append().
相关文件: src/misc2.c

Patch 6.1.052
问题: Unix: The executable() function doesn't work when the "which"
command isn't available.
解决方案: Go through $PATH manually. Also makes it work for VMS.
相关文件: src/os_unix.c

Patch 6.1.053
问题: When 'sessionoptions' contains "globals", or "localoptions" and an
option value contains a line break, the resulting script is wrong.
解决方案: Use "\n" and "\r" for a line break. (Srinath Avadhanula)
相关文件: src/eval.c

Patch 6.1.054
问题: GUI: A mouse click is not recognized at the more prompt, even when
'mouse' includes 'r'.
解决方案: Recognize a mouse click at the more prompt.
Also accept a mouse click in the last line in the GUI.
Add "ml" entry in 'mouseshape'.
相关文件: src/gui.c, src/message.c, src/misc1.c, src/misc2.c, src/option.c,
src/structs.h

Patch 6.1.055
问题: When editing a compressed file, Vim will inspect the contents to
guess the filetype.
解决方案: Don't source scripts.vim for .Z, .gz, .bz2, .zip and .tgz files.
相关文件: runtime/filetype.vim, runtime/plugin/gzip.vim

Patch 6.1.056
问题: Loading the Syntax menu can take quite a bit of time.
解决方案: Add the "skip_syntax_sel_menu" variable. When it's defined the available syntax files are not in the Syntax menu.
相关文件: runtime/doc/gui.txt, runtime/menu.vim

Patch 6.1.057
问题: An ESC inside a mapping doesn't work as documented when 'insertmode' is set, it does go from Visual or Normal mode to Insert mode. (Benji Fisher)
解决方案: Make it work as documented.
相关文件: src/normal.c

Patch 6.1.058
问题: When there is a closed fold just above the first line in the window, using CTRL-X CTRL-Y in Insert mode will show only one line of the fold. (Alexey Marinichev)
解决方案: Correct the topline by putting it at the start of the fold.
相关文件: src/move.c

Patch 6.1.059
问题: ":redir > ~/file" doesn't work. (Stephen Rasku)
解决方案: Expand environment variables in the ":redir >" argument.
相关文件: src/ex_docmd.c

Patch 6.1.060
问题: When 'virtualedit' is set and 'selection' is "exclusive", deleting a character just before a tab changes the tab into spaces. Undo doesn't restore the tab. (Helmut Stiegler)
解决方案: Don't replace the tab by spaces when it's not needed. Correctly save the line before it's changed.
相关文件: src/ops.c

Patch 6.1.061
问题: When 'virtualedit' is set and 'selection' is "exclusive", a Visual selection that ends just after a tab doesn't include that tab in the highlighting. (Helmut Stiegler)
解决方案: Use a different way to exclude the character under the cursor.
相关文件: src/screen.c

Patch 6.1.062
问题: The "man" filetype plugin doesn't work properly on Solaris 5.
解决方案: Use a different way to detect that "man -s" should be used. (Hugh Sasse)
相关文件: runtime/ftplugin/man.vim

Patch 6.1.063
问题: Java indenting doesn't work properly.
解决方案: Ignore comments when checking if the indent doesn't increase after a "}".
相关文件: runtime/indent/java.vim

Patch 6.1.064
问题: The URLs that the netrw plugin recognized for ftp and rcp did not

conform to the standard method://[user@]host[:port]/path.
解决方案: Use ftp://[user@]host[[:#]port]/path, which supports both the new and the previous style. Also added a bit of dav/cadaver support. (Charles Campbell)
相关文件: runtime/plugin/netrw.vim

Patch 6.1.065
问题: VMS: The colorscheme, keymap and compiler menus are not filled in.
解决方案: Ignore case when looking for ".vim" files. (Coen Engelbarts)
相关文件: runtime/menu.vim

Patch 6.1.066 (extra)
问题: When calling system() in a plugin reading stdin hangs.
解决方案: Don't set the terminal to RAW mode when it wasn't in RAW mode before the system() call.
相关文件: src/os_amiga.c, src/os_msdos.c, src/os_riscos.c, src/os_unix.c, src/os_win16.c, src/os_win32.c

Patch 6.1.067
问题: ":set viminfo+=f0" is not working. (Benji Fisher)
解决方案: Check the "f" flag instead of "'" in 'viminfo'.
相关文件: src/mark.c

Patch 6.1.068
问题: When a file is reloaded after it was changed outside of Vim, diff mode isn't updated. (Michael Naumann)
解决方案: Invalidate the diff info so that it's updated when needed.
相关文件: src/fileio.c

Patch 6.1.069
问题: When 'showmatch' is set and "$" is in 'cpoptions', using "C}<Esc>" may forget to remove the "$". (Preben Guldberg)
解决方案: Restore dollar_vcol after displaying the matching cursor position.
相关文件: src/search.c

Patch 6.1.070 (depends on 6.1.060)
问题: Compiler warning for signed/unsigned mismatch. (Mike Williams)
解决方案: Add a typecast to int.
相关文件: src/ops.c

Patch 6.1.071
问题: When 'selection' is exclusive, g CTRL-G in Visual mode counts one character too much. (David Necas)
解决方案: Subtract one from the end position.
相关文件: src/ops.c

Patch 6.1.072
问题: When a file name in a tags file starts with http:// or something else for which there is a BufReadCmd autocommand, the file isn't opened anyway.
解决方案: Check if there is a matching BufReadCmd autocommand and try to open the file.
相关文件: src/fileio.c, src/proto/fileio.pro, src/tag.c

Patch 6.1.073 (extra)
问题: BC5: Can't easily specify a tiny, small, normal, big or huge version.
解决方案: Allow selecting the version with the FEATURES variable. (Ajit Thakkar)
相关文件: src/Make_bc5.mak

Patch 6.1.074
问题: When 'cdpath' includes "../..", changing to a directory in which we currently already are doesn't work. ff_check_visited() adds the directory both when using it as the root for searching and for the actual matches. (Stephen Rasku)
解决方案: Use a separate list for the already searched directories.
相关文件: src/misc2.c

Patch 6.1.075 (depends on 6.1.072)
问题: Can't compile fileio.c on MS-Windows.
解决方案: Add a declaration for the "p" pointer. (Madoka Machitani)
相关文件: src/fileio.c

Patch 6.1.076 (extra)
问题: Macintosh: explorer plugin doesn't work on Mac Classic. IME doesn't work. Dialog boxes don't work on Mac OS X
解决方案: Fix explorer plugin and key modifiers. (Axel Kielhorn)
Fix IME support. (Muraoka Taro)
Disable dialog boxes. (Benji Fisher)
相关文件: src/edit.c, src/feature.h, src/gui_mac.c, src/os_mac.c

Patch 6.1.077
问题: On a Debian system with ACL linking fails. (Lubomir Host)
解决方案: When the "acl" library is used, check if the "attr" library is present and use it.
相关文件: src/auto/configure, src/configure.in, src/link.sh

Patch 6.1.078
问题: When using 'foldmethod' "marker" and the end marker appears before the start marker in the file, no fold is found. (Nazri Ramliy)
解决方案: Don't let the fold depth go negative.
相关文件: src/fold.c

Patch 6.1.079
问题: When using "s" in Visual block mode with 'virtualedit' set, when the selected block is after the end of some lines the wrong text is inserted and some lines are skipped. (Servatius Brandt)
解决方案: Insert the right text and extend short lines.
相关文件: src/ops.c

Patch 6.1.080
问题: When using gcc with /usr/local already in the search path, adding it again causes problems.
解决方案: Adjust configure.in to avoid adding /usr/local/include and /usr/local/lib when using GCC and they are already used. (Johannes Zellner)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.081
问题: ":help CTRL-_CTRL-N" doesn't work. (Christian J. Robinson)
解决方案: Double the backslash to avoid the special meaning of "_".
相关文件: src/ex_cmds.c

Patch 6.1.082
问题: On MS-Windows the vimrc_example.vim script is sourced and then mswin.vim. This enables using select mode, but since "p" is mapped it doesn't replace the selection.
解决方案: Remove the mapping of "p" from vimrc_example.vim, it's obsolete. (Vlad Sandrini)
相关文件: runtime/vimrc_example.vim

Patch 6.1.083
问题: When $LANG is "sk" or "sk_sk", the Slovak menu file isn't found. (Martin Lacko)
解决方案: Guess the right menu file based on the system.
相关文件: runtime/lang/menu_sk_sk.vim

Patch 6.1.084 (depends on 6.1.080)
问题: "include" and "lib" are mixed up when checking the directories gcc already searches.
解决方案: Swap the variable names. (SunHo Kim)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.085
问题: When using CTRL-O CTRL-\ CTRL-N from Insert mode, the displayed mode "(insert)" isn't removed. (Benji Fisher)
解决方案: Clear the command line.
相关文件: src/normal.c

Patch 6.1.086 (depends on 6.1.049)
问题: The guifg color for CursorIM doesn't take effect.
解决方案: Use the foreground color when it's defined. (Muraoka Taro)
相关文件: src/gui.c

Patch 6.1.087
问题: A thesaurus with Japanese characters has problems with characters in different word classes.
解决方案: Only separate words with single-byte non-word characters. (Muraoka Taro)
相关文件: src/edit.c

Patch 6.1.088 (extra)
问题: Win32: no debugging info is generated. Tags file excludes .cpp files.
解决方案: Add "/map" to compiler flags. Add "*.cpp" to ctags command. (Muraoka Taro)
相关文件: src/Make_mvc.mak

Patch 6.1.089
问题: On BSDI systems there is no ss_sp field in stack_t. (Robert Jan)
解决方案: Use ss_base instead.

相关文件: src/auto/configure, src/configure.in, src/config.h.in,
           src/os_unix.c

Patch 6.1.090
问题: CTRL-F gets stuck when 'scrolloff' is non-zero and there is a mix
           of long wrapping lines and a non-wrapping line.
解决方案: Check that CTRL-F scrolls at least one line.
相关文件: src/move.c

Patch 6.1.091
问题: GTK: Can't change preeditstate without setting 'imactivatekey'.
解决方案: Add some code to change preeditstate for OnTheSpot. (Yasuhiro
           Matsumoto)
相关文件: src/mbyte.c

Patch 6.1.092
问题: ":mapclear <buffer>" doesn't work. (Srikanth Adayapalam)
解决方案: Allow an argument for ":mapclear".
相关文件: src/ex_cmds.h

Patch 6.1.093 (extra)
问题: Mac and MS-Windows GUI: when scrolling while ":s" is working the
           results can be messed up, because the cursor is moved.
解决方案: Disallow direct scrolling when not waiting for a character.
相关文件: src/gui_mac.c, src/gui_w16.c, src/gui_w32.c, src/gui_w48.c

Patch 6.1.094
问题: Cygwin: Passing a file name that has backslashes isn't handled
           very well.
解决方案: Convert file name arguments to Posix. (Chris Metcalf)
相关文件: src/main.c

Patch 6.1.095
问题: When using signs can free an item on the stack.
           Overruling sign colors doesn't work. (Srikanth Sankaran)
解决方案: Don't free the item on the stack. Use NULL instead of "none" for
           the value of the color.
相关文件: src/gui_x11.c

Patch 6.1.096
问题: When erasing the right halve of a double-byte character, it may
           cause further characters to be erased. (Yasuhiro Matsumoto)
解决方案: Make sure only one character is erased.
相关文件: src/screen.c

Patch 6.1.097 (depends on 6.1.090)
问题: When 'scrolloff' is set to a huge value, CTRL-F at the end of the
           file scrolls one line. (Lubomir Host)
解决方案: Don't scroll when CTRL-F detects the end-of-file.
相关文件: src/move.c

Patch 6.1.098
问题: MS-Windows: When the xxd program is under "c:\program files" the
           "Convert to Hex" menu doesn't work. (Brian Mathis)

解决方案: Put the path to xxd in double quotes.
相关文件: runtime/menu.vim

Patch 6.1.099
问题: Memory corrupted when closing a fold with more than 99999 lines.
解决方案: Allocate more space for the fold text. (Walter Briscoe)
相关文件: src/eval.c

Patch 6.1.100 (extra, depends on 6.1.088)
问题: Win32: VC5 and earlier don't support the /mapinfo option.
解决方案: Add "/mapinfo" only when "MAP=lines" is specified. (Muraoka Taro)
相关文件: src/Make_mvc.mak

Patch 6.1.101
问题: After using ":options" the tabstop of a new window is 15. Entry in ":options" window for 'autowriteall' is wrong. (Antoine J Mechelynck) Can't insert a space in an option value.
解决方案: Use ":setlocal" instead of ":set". Change "aw" to "awa". Don't map space in Insert mode.
相关文件: runtime/optwin.vim

Patch 6.1.102
问题: Unprintable and multi-byte characters in a statusline item are not truncated correctly. (Yasuhiro Matsumoto)
解决方案: Count the width of characters instead of the number of bytes.
相关文件: src/buffer.c

Patch 6.1.103
问题: A function returning from a while loop, with 'verbose' set to 12 or higher, doesn't mention the return value. A function with the 'abort' attribute may return -1 while the verbose message says something else.
解决方案: Move the verbose message about returning from a function to call_func(). (Servatius Brandt)
相关文件: src/eval.c

Patch 6.1.104
问题: GCC 3.1 appears to have an optimizer problem that makes test 3 crash.
解决方案: For GCC 3.1 add -fno-strength-reduce to avoid the optimizer bug. Filter out extra info from "gcc --version".
相关文件: src/auto/configure, src/configure.in

Patch 6.1.105
问题: Win32: The default for 'shellpipe' doesn't redirect stderr. (Dion Nicolaas)
解决方案: Redirect stderr, depending on the shell (like for 'shellredir').
相关文件: src/option.c

Patch 6.1.106
问题: The maze program crashes.
解决方案: Change "11" to "27" and it works. (Greg Roelofs)
相关文件: runtime/macros/maze/mazeansi.c

Patch 6.1.107
问题: When 'list' is set the current line in the error window may be displayed wrong. (Muraoka Taro)
解决方案: Don't continue the line after the $ has been displayed and the rightmost column is reached.
相关文件: src/screen.c

Patch 6.1.108
问题: When interrupting a filter command such as "!!sleep 20" the file becomes read-only. (Mark Brader)
解决方案: Only set the read-only flag when opening a buffer is interrupted. When the shell command was interrupted, read the output that was produced so far.
相关文件: src/ex_cmds.c, src/fileio.c

Patch 6.1.109
问题: When 'eadirection' is "hor", using CTRL-W = doesn't equalize the window heights. (Roman Neuhauser)
解决方案: Ignore 'eadirection' for CTRL-W =
相关文件: src/window.c

Patch 6.1.110
问题: When using ":badd file" when "file" is already present but not listed, it stays unlisted. (David Frey)
解决方案: Set 'buflisted'.
相关文件: src/buffer.c

Patch 6.1.111
问题: It's not possible to detect using the Unix sources on Win32 or Mac.
解决方案: Add has("macunix") and has("win32unix").
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 6.1.112
问题: When using ":argdo", ":bufdo" or ":windo", CTRL-O doesn't go to the cursor position from before this command but every position where the argument was executed.
解决方案: Only remember the cursor position from before the ":argdo", ":bufdo" and ":windo".
相关文件: src/ex_cmds2.c, src/mark.c

Patch 6.1.113
问题: ":bufdo bwipe" only wipes out half the buffers. (Roman Neuhauser)
解决方案: Decide what buffer to go to next before executing the command.
相关文件: src/ex_cmds2.c

Patch 6.1.114
问题: ":python import vim", ":python vim.current.buffer[0:0] = []" gives a lalloc(0) error. (Chris Southern)
解决方案: Don't allocate an array when it's size is zero.
相关文件: src/if_python.c

Patch 6.1.115
问题: "das" on the white space at the end of a paragraph does not delete the "." the sentence ends with.

解决方案: Don't exclude the last character when it is not white space.
相关文件: src/search.c

Patch 6.1.116
问题: When 'endofline' is changed while 'binary' is set a file should be considered modified. (Olaf Buddenhagen)
解决方案: Remember the 'eol' value when editing started and consider the file changed when the current value is different and 'binary' is set. Also fix that the window title isn't updated when 'ff' or 'bin' changes.
相关文件: src/option.c, src/structs.h

Patch 6.1.117
问题: Small problem with editing a file over ftp: and with Cygwin.
解决方案: Remove a dot from a ":normal" command. Use "cygdrive" where appropriate. (Charles Campbell)
相关文件: runtime/plugin/netrw.vim

Patch 6.1.118
问题: When a file in diff mode is reloaded because it changed outside of Vim, other windows in diff mode are not always updated. (Michael Naumann)
解决方案: After reloading a file in diff mode mark all windows in diff mode for redraw.
相关文件: src/diff.c

Patch 6.1.119 (extra)
问题: With the Sniff interface, using Sniff 4.0.X on HP-UX, there may be a crash when connecting to Sniff.
解决方案: Initialize sniff_rq_sep such that its value can be changed. (Martin Egloff)
相关文件: src/if_sniff.c

Patch 6.1.120 (depends on 6.1.097)
问题: When 'scrolloff' is non-zero and there are folds, CTRL-F at the end of the file scrolls part of a closed fold. (Lubomir Host)
解决方案: Adjust the first line to the start of a fold.
相关文件: src/move.c

Patch 6.1.121 (depends on 6.1.098)
问题: When starting Select mode from Insert mode, then using the Paste menu entry, the cursor is left before the last pasted character. (Mario Schweigler)
解决方案: Set the cursor for Insert mode one character to the right.
相关文件: runtime/menu.vim

Patch 6.1.122
问题: ":file name" creates a new buffer to hold the old buffer name, which becomes the alternate file. This buffer is unexpectedly listed.
解决方案: Create the buffer for the alternate name unlisted.
相关文件: src/ex_cmds.c

Patch 6.1.123

问题:　　A ":match" command with more than one argument doesn't report an error.
解决方案:　Check for extra characters. (Servatius Brandt)
相关文件:　src/ex_docmd.c

Patch 6.1.124
问题:　　When trying to exit and there is a hidden buffer that had 'eol' off and 'bin' set exiting isn't possible. (John McGowan)
解决方案:　Set b_start_eol when clearing the buffer.
相关文件:　src/buffer.c

Patch 6.1.125
问题:　　Explorer plugin asks for saving a modified buffer even when it's open in another window as well.
解决方案:　Count the number of windows using the buffer.
相关文件:　runtime/plugin/explorer.vim

Patch 6.1.126
问题:　　Adding the choices in the syntax menu is consuming much of the startup time of the GUI while it's not often used.
解决方案:　Only add the choices when the user wants to use them.
相关文件:　Makefile, runtime/makemenu.vim, runtime/menu.vim, runtime/synmenu.vim, src/Makefile

Patch 6.1.127
问题:　　When using "--remote file" and the server has 'insertmode' set, commands are inserted instead of being executed. (Niklas Volbers)
解决方案:　Go to Normal mode again after the ":drop" command.
相关文件:　src/main.c

Patch 6.1.128
问题:　　The expression "input('very long prompt')" puts the cursor in the wrong line (column is OK).
解决方案:　Add the wrapped lines to the indent. (Yasuhiro Matsumoto)
相关文件:　src/ex_getln.c

Patch 6.1.129
问题:　　On Solaris editing "file/" and then "file" results in using the same buffer. (Jim Battle)
解决方案:　Before using stat(), check that there is no illegal trailing slash.
相关文件:　src/auto/configure, src/config.h.in, src/configure.in, src/macros.h src/misc2.c, src/proto/misc2.pro

Patch 6.1.130
问题:　　The documentation for some of the 'errorformat' items is unclear.
解决方案:　Add more examples and explain hard to understand items. (Stefan Roemer)
相关文件:　runtime/doc/quickfix.txt

Patch 6.1.131
问题:　　X11 GUI: when expanding a CSI byte in the input stream to K_CSI, the CSI byte itself isn't copied.
解决方案:　Copy the CSI byte.

相关文件: src/gui_x11.c

Patch 6.1.132
问题: Executing a register in Ex mode may cause commands to be skipped.
(John McGowan)
解决方案: In Ex mode use an extra check if the register contents was
consumed, to avoid input goes into the typeahead buffer.
相关文件: src/ex_docmd.c

Patch 6.1.133
问题: When drawing double-wide characters in the statusline, may clear
half of a character. (Yasuhiro Matsumoto)
解决方案: Force redraw of the next character by setting the attributes
instead of putting a NUL in ScreenLines[]. Do put a NUL in
ScreenLines[] when overwriting half of a double-wide character.
相关文件: src/screen.c

Patch 6.1.134
问题: An error for a trailing argument of ":match" should not be given
after ":if 0". (Servatius Brandt)
解决方案: Only do the check when executing commands.
相关文件: src/ex_docmd.c

Patch 6.1.135
问题: Passing a command to the shell that includes a newline always has
a backslash before the newline.
解决方案: Remove one backslash before the newline. (Servatius Brandt)
相关文件: src/ex_docmd.c

Patch 6.1.136
问题: When $TERM is "linux" the default for 'background' is "dark", even
though the GUI uses a light background. (Hugh Allen)
解决方案: Don't mark the option as set when defaulting to "dark" for the
linux console. Also reset 'background' to "light" when the GUI
has a light background.
相关文件: src/option.c

Patch 6.1.137
问题: Converting to HTML has a clumsy way of dealing with tabs which may
change the highlighting.
解决方案: Replace tabs with spaces after converting a line to HTML. (Preben
Guldberg)
相关文件: runtime/syntax/2html.vim

Patch 6.1.138 (depends on 6.1.126)
问题: Adding extra items to the Syntax menu can't be done when the "Show
individual choices" menu is used.
解决方案: Use ":runtime!" instead of ":source", so that all synmenu.vim
files in the runtime path are loaded. (Servatius Brandt)
Also fix that a translated menu can't be removed.
相关文件: runtime/menu.vim

Patch 6.1.139
问题: Cygwin: PATH_MAX is not defined.

解决方案: Include limits.h. (Dan Sharp)
相关文件: src/main.c

Patch 6.1.140
问题: Cygwin: ":args `ls *.c`" does not work if the shell command produces CR NL line separators.
解决方案: Remove the CR characters ourselves. (Pavol Juhas)
相关文件: src/os_unix.c

Patch 6.1.141
问题: ":wincmd gx" may cause problems when mixed with other commands. ":wincmd c" doesn't close the window immediately. (Benji Fisher)
解决方案: Pass the extra command character directly instead of using the stuff buffer and call ex_close() directly.
相关文件: src/ex_docmd.c, src/normal.c, src/proto/normal.pro, src/proto/window.pro, src/window.c

Patch 6.1.142
问题: Defining paragraphs without a separating blank line isn't possible. Paragraphs can't be formatted automatically.
解决方案: Allow defining paragraphs with lines that end in white space. Added the 'w' and 'a' flags in 'formatoptions'.
相关文件: runtime/doc/change.txt, src/edit.c, src/misc1.c, src/normal.c, src/option.h, src/ops.c, src/proto/edit.pro, src/proto/ops.pro, src/vim.h

Patch 6.1.143 (depends on 6.1.142)
问题: Auto formatting near the end of the file moves the cursor to a wrong position. In Insert mode some lines are made one char too narrow. When deleting a line undo might not always work properly.
解决方案: Don't always move to the end of the line in the last line. Don't position the cursor past the end of the line in Insert mode. After deleting a line save the cursor line for undo.
相关文件: src/edit.c, src/ops.c, src/normal.c

Patch 6.1.144
问题: Obtaining the size of a line in screen characters can be wrong. A pointer may wrap around zero.
解决方案: In win_linetabsize() check for a MAXCOL length argument. (Jim Dunleavy)
相关文件: src/charset.c

Patch 6.1.145
问题: GTK: Drag&drop with more than 3 files may cause a crash. (Mickael Marchand)
解决方案: Rewrite the code that parses the received list of files to be more robust.
相关文件: src/charset.c, src/gui_gtk_x11.c

Patch 6.1.146
问题: MS-Windows: When $HOME is constructed from $HOMEDRIVE and $HOMEPATH, it is not used for storing the _viminfo file. (Normal Diamond)
解决方案: Set $HOME with the value obtained from $HOMEDRIVE and $HOMEPATH.

相关文件: src/misc1.c

Patch 6.1.147 (extra)
问题: MS-Windows: When a dialog has no default button, pressing Enter ends it anyway and all buttons are selected.
解决方案: Don't end a dialog when there is no default button. Don't select all button when there is no default. (Vince Negri)
相关文件: src/gui_w32.c

Patch 6.1.148 (extra)
问题: MS-Windows: ACL is not properly supported.
解决方案: Add an access() replacement that also works for ACL. (Mike Williams)
相关文件: runtime/doc/editing.txt, src/os_win32.c

Patch 6.1.149 (extra)
问题: MS-Windows: Can't use diff mode from the file explorer.
解决方案: Add a "diff with Vim" context menu entry. (Dan Sharp)
相关文件: GvimExt/gvimext.cpp, GvimExt/gvimext.h

Patch 6.1.150
问题: OS/2, MS-Windows and MS-DOS: When 'shellslash' is set getcwd() still uses backslash. (Yegappan Lakshmanan)
解决方案: Adjust slashes in getcwd().
相关文件: src/eval.c

Patch 6.1.151 (extra)
问题: Win32: The NTFS substream isn't copied.
解决方案: Copy the substream when making a backup copy. (Muraoka Taro)
相关文件: src/fileio.c, src/os_win32.c, src/proto/os_win32.pro

Patch 6.1.152
问题: When $LANG is iso8859-1 translated menus are not used.
解决方案: Change iso8859 to iso_8859.
相关文件: runtime/menu.vim

Patch 6.1.153
问题: Searching in included files may search recursively when the path starts with "../". (Sven Berkvens-Matthijsse)
解决方案: Compare full file names, use inode/device when possible.
相关文件: src/search.c

Patch 6.1.154 (extra)
问题: DJGPP: "vim -h" leaves the cursor in a wrong position.
解决方案: Don't position the cursor using uninitialized variables. (Jim Dunleavy)
相关文件: src/os_msdos.c

Patch 6.1.155
问题: Win32: Cursor may sometimes disappear in Insert mode.
解决方案: Change "hor10" in 'guicursor' to "hor15". (Walter Briscoe)
相关文件: src/option.c

Patch 6.1.156

问题:　　Conversion between DBCS and UCS-2 isn't implemented cleanly.
解决方案:　Clean up a few things.
相关文件:　src/mbyte.c, src/structs.h

Patch 6.1.157
问题:　　'hlsearch' highlights only the second comma in ",,,,," with "/,\@<=[^,]*". (Preben Guldberg)
解决方案:　Also check for an empty match to start just after a previous match.
相关文件:　src/screen.c

Patch 6.1.158
问题:　　"zs" and "ze" don't work correctly with ":set nowrap siso=1". (Preben Guldberg)
解决方案:　Take 'siso' into account when computing the horizontal scroll position for "zs" and "ze".
相关文件:　src/normal.c

Patch 6.1.159
问题:　　When expanding an abbreviation that includes a multi-byte character too many characters are deleted. (Andrey Urazov)
解决方案:　Delete the abbreviation counting characters instead of bytes.
相关文件:　src/getchar.c

Patch 6.1.160
问题:　　":$read file.gz" doesn't work. (Preben Guldberg)
解决方案:　Don't use the '[mark after it has become invalid.
相关文件:　runtime/plugin/gzip.vim

Patch 6.1.161 (depends on 6.1.158)
问题:　　Warning for signed/unsigned compare. Can set 'siso' to a negative value. (Mike Williams)
解决方案:　Add a typecast. Add a check for 'siso' being negative.
相关文件:　src/normal.c, src/option.c

Patch 6.1.162
问题:　　Python interface: Didn't initialize threads properly.
解决方案:　Call PyEval_InitThreads() when starting up.
相关文件:　src/if_python.c

Patch 6.1.163
问题:　　Win32: Can't compile with Python after 6.1.162.
解决方案:　Dynamically load PyEval_InitThreads(). (Dan Sharp)
相关文件:　src/if_python.c

Patch 6.1.164
问题:　　If 'modifiable' is off, converting to xxd fails and 'filetype' is changed to "xxd" anyway.
解决方案:　Don't change 'filetype' when conversion failed.
相关文件:　runtime/menu.vim

Patch 6.1.165
问题:　　Making changes in several lines and then a change in one of these lines that splits it in two or more lines, undo information was

corrupted. May cause a crash. (Dave Fishburn)
解决方案: When skipping to save a line for undo because it was already saved, move it to become the last saved line, so that when the command changes the line count other saved lines are not involved.
相关文件: src/undo.c

Patch 6.1.166
问题: When 'autoindent' is set and mswin.vim has been sourced, pasting with CTRL-V just after auto-indenting removes the indent. (Shlomi Fish)
解决方案: First insert an "x" and delete it again, so that the auto-indent remains.
相关文件: runtime/mswin.vim

Patch 6.1.167
问题: When giving a negative argument to ":retab" strange things start happening. (Hans Ginzel)
解决方案: Check for a negative value.
相关文件: src/ex_cmds.c

Patch 6.1.168
问题: Pressing CTRL-C at the hit-enter prompt doesn't end the prompt.
解决方案: Make CTRL-C stop the hit-enter prompt.
相关文件: src/message.c

Patch 6.1.169
问题: bufexists() finds a buffer by using the name of a symbolic link to it, but bufnr() doesn't. (Yegappan Lakshmanan)
解决方案: When bufnr() can't find a buffer, try using the same method as bufexists().
相关文件: src/eval.c

Patch 6.1.170
问题: Using ":mksession" uses the default session file name, but "vim -S" doesn't. (Hans Ginzel)
解决方案: Use the default session file name if "-S" is the last command line argument or another option follows.
相关文件: runtime/doc/starting.txt, src/main.c

Patch 6.1.171
问题: When opening a line just above a closed fold with "O" and the comment leader is automatically inserted, the cursor is displayed in the first column. (Sung-Hyun Nam)
解决方案: Update the flag that indicates the cursor is in a closed fold.
相关文件: src/misc1.c

Patch 6.1.172
问题: Command line completion of ":tag /pat" does not show the same results as the tags the command actually finds. (Gilles Roy)
解决方案: Don't modify the pattern to make it a regexp.
相关文件: src/ex_getln.c, src/tag.c

Patch 6.1.173
问题: When using remote control to edit a position in a file and this

file is the current buffer and it's modified, the window is split and the ":drop" command fails.
解决方案: Don't split the window, keep editing the same buffer. Use the ":drop" command in VisVim to avoid the problem there.
相关文件: src/ex_cmds.c, src/ex_cmds2.c, src/proto/ex_cmds2.pro, VisVim/Commands.cpp

Patch 6.1.174
问题: It is difficult to know in a script whether an option not only exists but really works.
解决方案: Add "exists('+option')".
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 6.1.175
问题: When reading commands from a pipe and a CTRL-C is pressed, Vim will hang. (Piet Delport)
解决方案: Don't keep reading characters to clear typeahead when an interrupt was detected, stop when a single CTRL-C is read.
相关文件: src/getchar.c, src/ui.c

Patch 6.1.176
问题: When the stack limit is very big a false out-of-stack error may be detected.
解决方案: Add a check for overflow of the stack limit computation. (Jim Dunleavy)
相关文件: src/os_unix.c

Patch 6.1.177 (depends on 6.1.141)
问题: ":wincmd" does not allow a following command. (Gary Johnson)
解决方案: Check for a following " | cmd". Also give an error for trailing characters.
相关文件: src/ex_docmd.c

Patch 6.1.178
问题: When 'expandtab' is set "r<C-V><Tab>" still expands the Tab. (Bruce deVisser)
解决方案: Replace with a literal Tab.
相关文件: src/normal.c

Patch 6.1.179 (depends on 6.1.091)
问题: When using X11R5 XIMPreserveState is undefined. (Albert Chin)
解决方案: Include the missing definitions.
相关文件: src/mbyte.c

Patch 6.1.180
问题: Use of the GUI code for forking is inconsistent.
解决方案: Define MAY_FORK and use it for later #ifdefs. (Ben Fowlwer)
相关文件: src/gui.c

Patch 6.1.181
问题: If the terminal doesn't wrap from the last char in a line to the next line, the last column is blanked out. (Peter Karp)
解决方案: Don't output a space to mark the wrap, but the same character again.

相关文件: src/screen.c

Patch 6.1.182 (depends on 6.1.142)
问题: It is not possible to auto-format comments only. (Moshe Kaminsky)
解决方案: When the 'a' and 'c' flags are in 'formatoptions' only auto-format comments.
相关文件: runtime/doc/change.txt, src/edit.c

Patch 6.1.183
问题: When 'fencs' is empty and 'enc' is utf-8, reading a file with illegal bytes gives "CONVERSION ERROR" even though no conversion is done. 'readonly' is set, even though writing the file results in an unmodified file.
解决方案: For this specific error use "ILLEGAL BYTE" and don't set 'readonly'.
相关文件: src/fileio.c

Patch 6.1.184 (extra)
问题: The extra mouse buttons found on some mice don't work.
解决方案: Support two extra buttons for MS-Windows. (Michael Geddes)
相关文件: runtime/doc/term.txt, src/edit.c, src/ex_getln.c, src/gui.c, src/gui_w32.c, src/gui_w48.c, src/keymap.h, src/message.c, src/misc1.c, src/misc2.c, src/normal.c, src/vim.h

Patch 6.1.185 (depends on 6.1.182)
问题: Can't compile without +comments feature.
解决方案: Add #ifdef FEAT_COMMENTS. (Christian J. Robinson)
相关文件: src/edit.c

Patch 6.1.186 (depends on 6.1.177)
问题: ":wincmd" does not allow a following comment. (Aric Blumer)
解决方案: Check for a following double quote.
相关文件: src/ex_docmd.c

Patch 6.1.187
问题: Using ":doarg" with 'hidden' set and the current file is the only argument and was modified gives an error message. (Preben Guldberg)
解决方案: Don't try re-editing the same file.
相关文件: src/ex_cmds2.c

Patch 6.1.188 (depends on 6.1.173)
问题: Unused variable in the small version.
解决方案: Move the declaration for "p" inside #ifdef FEAT_LISTCMDS.
相关文件: src/ex_cmds2.c

Patch 6.1.189
问题: inputdialog() doesn't work when 'c' is in 'guioptions'. (Aric Blumer)
解决方案: Fall back to the input() function in this situation.
相关文件: src/eval.c

Patch 6.1.190 (extra)
问题: VMS: doesn't build with GTK GUI. Various other problems.

解决方案: Fix building for GTK. Improved Perl, Python and TCL support.
Improved VMS documentation. (Zoltan Arpadffy)
Added Vimtutor for VMS (T. R. Wyant)
相关文件: runtime/doc/os_vms.txt, src/INSTALLvms.txt, src/gui_gtk_f.h,
src/if_tcl.c, src/main.c, src/gui_gtk_vms.h, src/Make_vms.mms,
src/os_vms.opt, src/proto/if_tcl.pro, vimtutor.com,
src/testdir/Make_vms.mms

Patch 6.1.191
问题: When using "vim -s script" and redirecting the output, the delay
for the "Output is not to a terminal" warning slows Vim down too
much.
解决方案: Don't delay when reading commands from a script.
相关文件: src/main.c

Patch 6.1.192
问题: ":diffsplit" doesn't add "hor" to 'scrollopt'. (Gary Johnson)
解决方案: Add "hor" to 'scrollopt' each time ":diffsplit" is used.
相关文件: src/diff.c, src/main.c

Patch 6.1.193
问题: Crash in in_id_list() for an item with a "containedin" list. (Dave
Fishburn)
解决方案: Check for a negative syntax id, used for keywords.
相关文件: src/syntax.c

Patch 6.1.194
问题: When "t_ti" is set but it doesn't cause swapping terminal pages,
"ZZ" may cause the shell prompt to appear on top of the file-write
message.
解决方案: Scroll the text up in the Vim page before swapping to the terminal
page. (Michael Schroeder)
相关文件: src/os_unix.c

Patch 6.1.195
问题: The quickfix and preview windows always keep their height, while
other windows can't fix their height.
解决方案: Add the 'winfixheight' option, so that a fixed height can be
specified for any window. Also fix that the wildmenu may resize a
one-line window to a two-line window if 'ls' is zero.
相关文件: runtime/doc/options.txt, runtime/optwin.vim, src/ex_cmds.c,
src/ex_getln.c, src/globals.h, src/option.c, src/quickfix.c,
src/screen.c, src/structs.h, src/window.c

Patch 6.1.196 (depends on 6.1.084)
问题: On Mac OS X 10.2 generating osdef.h fails.
解决方案: Add -no-cpp-precomp to avoid using precompiled header files, which
disables printing the search path. (Ben Fowler)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.197
问题: ":help <C-V><C-\><C-V><C-N>" (resulting in <1c><0e>) gives an
error message. (Servatius Brandt)
解决方案: Double the backslash in "CTRL-\".

相关文件: src/ex_cmds.c

Patch 6.1.198 (extra) (depends on 6.1.076)
问题: Mac OS X: Dialogues don't work.
解决方案: Fix a crashing problem for some GUI dialogues. Fix a problem when saving to a new file from the GUI. (Peter Cucka)
相关文件: src/feature.h, src/gui_mac.c

Patch 6.1.199
问题: 'guifontwide' doesn't work on Win32.
解决方案: Output each wide character separately. (Michael Geddes)
相关文件: src/gui.c

Patch 6.1.200
问题: ":syn sync fromstart" is not skipped after ":if 0". This can make syntax highlighting very slow.
解决方案: Check "eap->skip" appropriately. (Rob West)
相关文件: src/syntax.c

Patch 6.1.201 (depends on 6.1.192)
问题: Warning for illegal pointer combination. (Zoltan Arpadffy)
解决方案: Add a typecast.
相关文件: src/diff.c

Patch 6.1.202 (extra)(depends on 6.1.148)
问题: Win32: filewritable() doesn't work properly on directories.
解决方案: fix filewritable(). (Mike Williams)
相关文件: src/os_win32.c

Patch 6.1.203
问题: ":%s/~//" causes a crash after ":%s/x//". (Gary Holloway)
解决方案: Avoid reading past the end of a line when "~" is empty.
相关文件: src/regexp.c

Patch 6.1.204 (depends on 6.1.129)
问题: Warning for an illegal pointer on Solaris.
解决方案: Add a typecast. (Derek Wyatt)
相关文件: src/misc2.c

Patch 6.1.205
问题: The gzip plugin changes the alternate file when editing a compressed file. (Oliver Fuchs)
解决方案: Temporarily remove the 'a' and 'A' flags from 'cpo'.
相关文件: runtime/plugin/gzip.vim

Patch 6.1.206
问题: The script generated with ":mksession" doesn't work properly when some commands are mapped.
解决方案: Use ":normal!" instead of ":normal". And use ":wincmd" where possible. (Muraoka Taro)
相关文件: src/ex_docmd.c, src/fold.c

Patch 6.1.207
问题: Indenting a Java file hangs below a line with a comment after a

command.
解决方案: Break out of a loop. (Andre Pang)
Also line up } with matching {.
相关文件: runtime/indent/java.vim

Patch 6.1.208
问题: Can't use the buffer number from the Python interface.
解决方案: Add buffer.number. (Michal Vitecek)
相关文件: src/if_python.c

Patch 6.1.209
问题: Printing doesn't work on Mac OS classic.
解决方案: Use a ":" for path separator when opening the resource file. (Axel Kielhorn)
相关文件: src/ex_cmds2.c

Patch 6.1.210
问题: When there is an iconv() conversion error when reading a file there can be an error the next time iconv() is used.
解决方案: Reset the state of the iconv() descriptor. (Yasuhiro Matsumoto)
相关文件: src/fileio.c

Patch 6.1.211
问题: The message "use ! to override" is confusing.
解决方案: Make it "add ! to override".
相关文件: src/buffer.c, src/eval.c, src/ex_docmd.c, src/fileio.c, src/globals.h

Patch 6.1.212
问题: When Vim was started with "-R" ":new" creates a buffer 'noreadonly' while ":enew" has 'readonly' set. (Preben Guldberg)
解决方案: Don't set 'readonly' in a new empty buffer for ":enew".
相关文件: src/ex_docmd.c

Patch 6.1.213
问题: Using CTRL-W H may cause a big gap to appear below the last window. (Aric Blumer)
解决方案: Don't set the window height when there is a vertical split. (Yasuhiro Matsumoto)
相关文件: src/window.c

Patch 6.1.214
问题: When installing Vim and the runtime files were checked out from CVS the CVS directories will also be installed.
解决方案: Avoid installing the CVS dirs and their contents.
相关文件: src/Makefile

Patch 6.1.215
问题: Win32: ":pwd" uses backslashes even when 'shellslash' is set. (Xiangjiang Ma)
解决方案: Adjust backslashes before printing the message.
相关文件: src/ex_docmd.c

Patch 6.1.216

问题:　　When dynamically loading the iconv library, the error codes may be confused.
解决方案:　Use specific error codes for iconv and redefine them for dynamic loading. (Yasuhiro Matsumoto)
相关文件:　src/fileio.c, src/mbyte.c, src/vim.h

Patch 6.1.217
问题:　　When sourcing the same Vim script using a different name (symbolic link or MS-Windows 8.3 name) it is listed twice with ":scriptnames". (Tony Mechelynck)
解决方案:　Turn the script name into a full path before using it. On Unix compare inode/device numbers.
相关文件:　src/ex_cmds2.c

Patch 6.1.218
问题:　　No error message for using the function argument "5+". (Servatius Brandt)
解决方案:　Give an error message if a function or variable is expected but is not found.
相关文件:　src/eval.c

Patch 6.1.219
问题:　　When using ":amenu :b 1<CR>" with a Visual selection and 'insertmode' is set, Vim does not return to Insert mode. (Mickael Marchand)
解决方案:　Add the command CTRL-\ CTRL-G that goes to Insert mode if 'insertmode' is set and to Normal mode otherwise. Append this to menus defined with ":amenu".
相关文件:　src/edit.c, src/ex_getln.c, src/normal.c

Patch 6.1.220
问题:　　When using a BufReadPost autocommand that changes the line count, e.g., "$-1join", reloading a file that was changed outside Vim does not work properly. (Alan G Isaac)
解决方案:　Make the buffer empty before reading the new version of the file. Save the lines in a dummy buffer, so that they can be put back when reading the file fails.
相关文件:　src/buffer.c, src/ex_cmds.c, src/fileio.c, src/globals.h, src/proto/buffer.pro

Patch 6.1.221
问题:　　Changing case may not work properly, depending on the current locale.
解决方案:　Add the 'casemap' option to let the user chose how changing case is to be done.
Also fix lowering case when an UTF-8 character doesn't keep the same byte length.
相关文件:　runtime/doc/options.txt, src/ascii.h, src/auto/configure, src/buffer.c, src/charset.c, src/config.h.in, src/configure.in, src/diff.c, src/edit.c, src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/gui_amiga.c src/gui_mac.c, src/gui_photon.c, src/gui_w48.c, src/gui_beos.cc, src/macros.h, src/main.c, src/mbyte.c, src/menu.c, src/message.c, src/misc1.c, src/misc2.c, src/option.c, src/os_msdos.c,

src/os_mswin.c, src/proto/charset.pro, src/regexp.c, src/option.h, src/syntax.c

Patch 6.1.222 (depends on 6.1.219)
问题: Patch 6.1.219 was incomplete.
解决方案: Add the changes for ":amenu".
相关文件: src/menu.c

Patch 6.1.223 (extra)
问题: Win32: When IME is activated 'iminsert' is set, but it might never be reset when IME is disabled. (Muraoka Taro)
All systems: 'iminsert' is set to 2 when leaving Insert mode, even when langmap is being used. (Peter Valach)
解决方案: Don't set "b_p_iminsert" in _OnImeNotify(). (Muraoka Taro)
Don't store the status of the input method in 'iminsert' when 'iminsert' is one. Also for editing the command line and for arguments to Normal mode commands.
相关文件: src/edit.c, src/ex_getln.c, src/gui_w32.c, src/normal.c

Patch 6.1.224
问题: "expand('$VAR')" returns an empty string when the expanded $VAR is not an existing file. (Aric Blumer)
解决方案: Included non-existing files, as documented.
相关文件: src/eval.c

Patch 6.1.225
问题: Using <C-O><C-^> in Insert mode has a delay when starting "vim -u NONE" and ":set nocp hidden". (Emmanuel) do_ecmd() uses fileinfo(), the redraw is done after a delay to give the user time to read the message.
解决方案: Put the message from fileio() in "keep_msg", so that the redraw is done before the delay (still needed to avoid the mode message overwrites the fileinfo() message).
相关文件: src/buffer.c

Patch 6.1.226
问题: Using ":debug" with a ":normal" command may cause a hang. (Colin Keith)
解决方案: Save the typeahead buffer when obtaining a debug command.
相关文件: src/ex_cmds2.c, src/getchar.c, src/proto/getchar.pro

Patch 6.1.227
问题: It is possible to use a variable name "asdf:asdf" and ":let j:asdf = 5" does not give an error message. (Mikolaj Machowski)
解决方案: Check for a ":" inside the variable name.
相关文件: src/eval.c

Patch 6.1.228 (extra)
问题: Win32: The special output function for Hangul is used too often, causing special handling for other situations to be skipped. bInComposition is always FALSE, causing ImeGetTempComposition() always to return NULL.
解决方案: Remove HanExtTextOut(). Delete the dead code around bInComposition and ImeGetTempComposition().

相关文件: src/gui_w16.c, src/gui_w32.c, src/gui_w48.c

Patch 6.1.229
问题: Win32: Conversion to/from often used codepages requires the iconv library, which is not always available.
解决方案: Use standard MS-Windows functions for the conversion when possible. (mostly by Glenn Maynard)
Also fixes missing declaration for patch 6.1.220.
相关文件: src/fileio.c

Patch 6.1.230 (extra)
问题: Win16: building doesn't work.
解决方案: Exclude the XBUTTON handling. (Vince Negri)
相关文件: src/gui_w48.c

Patch 6.1.231
问题: Double clicking with the mouse to select a word does not work for multi-byte characters.
解决方案: Use vim_iswordc() instead of vim_isIDc(). This means 'iskeyword' is used instead of 'isident'. Also fix that mixing ASCII with multi-byte word characters doesn't work, the mouse class for punctuation and word characters was mixed up.
相关文件: src/normal.c

Patch 6.1.232 (depends on 6.1.226)
问题: Using ex_normal_busy while it might not be available. (Axel Kielhorn)
解决方案: Only use ex_normal_busy when FEAT_EX_EXTRA is defined.
相关文件: src/ex_cmds2.c

Patch 6.1.233
问题: ":help expr-||" does not work.
解决方案: Don't use the '|' as a command separator
相关文件: src/ex_cmds.c

Patch 6.1.234 (depends on 6.1.217)
问题: Get a warning for using a negative value for st_dev.
解决方案: Don't assign a negative value to st_dev.
相关文件: src/ex_cmds2.c

Patch 6.1.235 (depends on 6.1.223)
问题: 'iminsert' is changed from 1 to 2 when leaving Insert mode. (Peter Valach)
解决方案: Check "State" before resetting it to NORMAL.
相关文件: src/edit.c

Patch 6.1.236
问题: Memory leaks when appending lines for ":diffget" or ":diffput" and when reloading a changed buffer.
解决方案: Free a line after calling ml_append().
相关文件: src/diff.c, src/fileio.c

Patch 6.1.237
问题: Putting in Visual block mode does not work correctly when "$" was

used or when the first line is short. (Christian Michon)
解决方案: First delete the selected text and then put the new text. Save and restore registers as necessary.
相关文件: src/globals.h, src/normal.c, src/ops.c, src/proto/ops.pro, src/vim.h

Patch 6.1.238 (extra)
问题: Win32: The "icon=" argument for the ":menu" command does not search for the bitmap file.
解决方案: Expand environment variables and search for the bitmap file. (Vince Negri)
Make it consistent, use the same mechanism for X11 and GTK.
相关文件: src/gui.c src/gui_gtk.c, src/gui_w32.c, src/gui_x11.c, src/proto/gui.pro

Patch 6.1.239
问题: Giving an error for missing :endif or :endwhile when being interrupted.
解决方案: Don't give these messages when interrupted.
相关文件: src/ex_docmd.c, src/os_unix.c

Patch 6.1.240 (extra)
问题: Win32 with BCC 5: CPU may be defined in the environment, which causes a wrong argument for the compiler. (Walter Briscoe)
解决方案: Use CPUNR instead of CPU.
相关文件: src/Make_bc5.mak

Patch 6.1.241
问题: Something goes wrong when drawing or undrawing the cursor.
解决方案: Remember when the cursor invalid in a better way.
相关文件: src/gui.c

Patch 6.1.242
问题: When pasting a large number of lines on the command line it is not possible to interrupt. (Jean Jordaan)
解决方案: Check for an interrupt after each pasted line.
相关文件: src/ops.c

Patch 6.1.243 (extra)
问题: Win32: When the OLE version is started and wasn't registered, a message pops up to suggest registering, even when this isn't possible (when the registry is not writable).
解决方案: Check if registering is possible before asking whether it should be done. (Walter Briscoe)
Also avoid restarting Vim after registering.
相关文件: src/if_ole.cpp

Patch 6.1.244
问题: Patch 6.1.237 was missing the diff for vim.h. (Igor Goldenberg)
解决方案: Include it here.
相关文件: src/vim.h

Patch 6.1.245
问题: Comparing with ignored case does not work properly for Unicode

with a locale where case folding an ASCII character results in a multi-byte character. (Glenn Maynard)
解决方案: Handle ignore-case compare for Unicode differently.
相关文件: src/mbyte.c

Patch 6.1.246
问题: ":blast" goes to the first buffer if the last one is unlisted. (Andrew Stryker)
解决方案: From the last buffer search backwards for the first listed buffer instead of forwards.
相关文件: src/ex_docmd.c

Patch 6.1.247
问题: ACL support doesn't always work properly.
解决方案: Add a configure argument to disable ACL "--disable-acl". (Thierry Vignaud)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.248
问题: Typing 'q' at the more-prompt for ":let" does not quit the listing. (Hari Krishna Dara)
解决方案: Quit the listing when got_int is set.
相关文件: src/eval.c

Patch 6.1.249
问题: Can't expand a path on the command line if it includes a "|" as a trail byte of a multi-byte character.
解决方案: Check for multi-byte characters. (Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c

Patch 6.1.250
问题: When changing the value of 'lines' inside the expression set with 'diffexpr' Vim might crash. (Dave Fishburn)
解决方案: Don't allow changing the screen size while updating the screen.
相关文件: src/globals.h, src/option.c, src/screen.c

Patch 6.1.251
问题: Can't use completion for ":lcd" and ":lchdir" like ":cd".
解决方案: Expand directory names for these commands. (Servatius Brandt)
相关文件: src/ex_docmd.c

Patch 6.1.252
问题: "vi}" does not include a line break when the "}" is at the start of a following line. (Kamil Burzynski)
解决方案: Include the line break.
相关文件: src/search.c

Patch 6.1.253 (extra)
问题: Win32 with Cygwin: Changes the path of arguments in a wrong way. (Xiangjiang Ma)
解决方案: Don't use cygwin_conv_to_posix_path() for the Win32 version. Update the Cygwin makefile to support more features. (Dan Sharp)
相关文件: src/Make_cyg.mak, src/if_ole.cpp, src/main.c

Patch 6.1.254
问题: exists("foo{bar}") does not work. ':unlet v{"a"}r' does not work. ":let v{a}r1 v{a}r2" does not work. ":func F{(1)}" does not work. ":delfunc F{" does not give an error message. ':delfunc F{"F"}' does not work.
解决方案: Support magic braces for the exists() argument. (Vince Negri) Check for trailing comments explicitly for ":unlet". Add support for magic braces in further arguments of ":let". Look for a parenthesis only after the function name. (Servatius Brandt) Also expand magic braces for "exists('*expr')". Give an error message for an invalid ":delfunc" argument. Allow quotes in the ":delfunc" argument.
相关文件: src/eval.c, src/ex_cmds.h, src/ex_docmd.c

Patch 6.1.255 (depends on 6.1.254)
问题: Crash when loading menu.vim a second time. (Christian Robinson) ":unlet garbage foo" tries unletting "foo" after an error message. (Servatius Brandt) Very long function arguments cause very long messages when 'verbose' is 14 or higher.
解决方案: Avoid reading from uninitialized memory. Break out of a loop after an invalid argument for ":unlet". Truncate long function arguments to 80 characters.
相关文件: src/eval.c

Patch 6.1.256 (depends on 6.1.255)
问题: Defining a function after ":if 0" could still cause an error message for an existing function. Leaking memory when there are trailing characters for ":delfunc".
解决方案: Check the "skip" flag. Free the memory. (Servatius Brandt)
相关文件: src/eval.c

Patch 6.1.257
问题: ":cwindow" always sets the previous window to the last but one window. (Benji Fisher)
解决方案: Set the previous window properly.
相关文件: src/globals.c, src/quickfix.c, src/window.c

Patch 6.1.258
问题: Buffers menu doesn't work properly for multibyte buffer names.
解决方案: Use a pattern to get the left and right part of the name. (Yasuhiro Matsumoto)
相关文件: runtime/menu.vim

Patch 6.1.259 (extra)
问题: Mac: with 'patchmode' is used filenames are truncated.
解决方案: Increase the BASENAMELEN for Mac OS X. (Ed Ralston)
相关文件: src/os_mac.h

Patch 6.1.260 (depends on 6.1.104)
问题: GCC 3.2 still seems to have an optimizer problem. (Zvi Har'El)
解决方案: Use the same configure check as used for GCC 3.1.
相关文件: src/auto/configure, src/configure.in

Patch 6.1.261
问题: When deleting a line in a buffer which is not the current buffer, using the Perl interface Delete(), the cursor in the current window may move. (Chris Houser)
解决方案: Don't adjust the cursor position when changing another buffer.
相关文件: src/if_perl.xs

Patch 6.1.262
问题: When jumping over folds with "z[", "zj" and "zk" the previous position is not remembered. (Hari Krishna Dara)
解决方案: Set the previous context mark before jumping.
相关文件: src/fold.c

Patch 6.1.263
问题: When typing a multi-byte character that triggers an abbreviation it is not inserted properly.
解决方案: Handle adding the typed multi-byte character. (Yasuhiro Matsumoto)
相关文件: src/getchar.c

Patch 6.1.264 (depends on patch 6.1.254)
问题: exists() does not work for built-in functions. (Steve Wall)
解决方案: Don't check for the function name to start with a capital.
相关文件: src/eval.c

Patch 6.1.265
问题: libcall() can be used in 'foldexpr' to call any system function. rename(), delete() and remote_send() can also be used in 'foldexpr'. These are security problems. (Georgi Guninski)
解决方案: Don't allow using libcall(), rename(), delete(), remote_send() and similar functions in the sandbox.
相关文件: src/eval.c

Patch 6.1.266 (depends on 6.1.265)
问题: Win32: compile error in eval.c. (Bill McCarthy)
解决方案: Move a variable declaration.
相关文件: src/eval.c

Patch 6.1.267
问题: Using "p" to paste into a Visual selected area may cause a crash.
解决方案: Allocate enough memory for saving the register contents. (Muraoka Taro)
相关文件: src/ops.c

Patch 6.1.268
问题: When triggering an abbreviation with a multi-byte character, this character is not correctly inserted after expanding the abbreviation. (Taro Muraoka)
解决方案: Add ABBR_OFF to all characters above 0xff.
相关文件: src/edit.c, src/ex_getln.c, src/getchar.c

Patch 6.1.269
问题: After using input() text written with ":redir" gets extra indent. (David Fishburn)
解决方案: Restore msg_col after using input().

相关文件: src/ex_getln.c

Patch 6.1.270 (depends on 6.1.260)
问题: GCC 3.2.1 still seems to have an optimizer problem.
解决方案: Use the same configure check as used for GCC 3.1.
相关文件: src/auto/configure, src/configure.in

Patch 6.1.271
问题: When compiling without the +syntax feature there are errors.
解决方案: Don't use some code for syntax highlighting. (Roger Cornelius)
Make test 45 work without syntax highlighting.
Also fix an error in a pattern matching: "\%(" was not supported.
相关文件: src/ex_cmds2.c, src/regexp.c, src/testdir/test45.in

Patch 6.1.272
问题: After using ":set define<" a crash may happen. (Christian Robinson)
解决方案: Make a copy of the option value in allocated memory.
相关文件: src/option.c

Patch 6.1.273
问题: When the cursor doesn't blink, redrawing an exposed area may hide
the cursor.
解决方案: Always draw the cursor, also when it didn't move. (Muraoka Taro)
相关文件: src/gui.c

Patch 6.1.274 (depends on 6.1.210)
问题: Resetting the iconv() state after each error is wrong for an
incomplete sequence.
解决方案: Don't reset the iconv() state.
相关文件: src/fileio.c

Patch 6.1.275
问题: When using "v" in a startup script, get warning message that
terminal cannot highlight. (Charles Campbell)
解决方案: Only give the message after the terminal has been initialized.
相关文件: src/normal.c

Patch 6.1.276
问题: "gvim --remote file" doesn't prompt for an encryption key.
解决方案: The further characters the client sends to the server are used.
Added inputsave() and inputrestore() to allow prompting the
user directly and not using typeahead.
Also fix possible memory leak for ":normal".
相关文件: src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/getchar.c,
src/main.c, src/proto/getchar.pro, src/proto/ui.pro,
src/runtime/doc/eval.txt, src/structs.h, src/ui.c, src/vim.h

Patch 6.1.277 (depends on 6.1.276)
问题: Compilation error when building with small features.
解决方案: Define trash_input_buf() when needed. (Kelvin Lee)
相关文件: src/ui.c

Patch 6.1.278
问题: When using signs the line number of a closed fold doesn't line up

with the other line numbers. (Kamil Burzynski)
解决方案: Insert two spaces for the sign column.
相关文件: src/screen.c

Patch 6.1.279
问题: The prototype for smsg() and smsg_attr() do not match the function definition. This may cause trouble for some compilers. (Nix)
解决方案: Use va_list for systems that have stdarg.h. Use "int" instead of "void" for the return type.
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/proto.h, src/message.c

Patch 6.1.280
问题: It's possible to use an argument "firstline" or "lastline" for a function but using "a:firstline" or "a:lastline" in the function won't work. (Benji Fisher)
解决方案: Give an error message for these arguments.
Also avoid that the following function body causes a whole row of errors, skip over it after an error in the first line.
相关文件: src/eval.c

Patch 6.1.281
问题: In Insert mode CTRL-X CTRL-G leaves the cursor after the ruler.
解决方案: Set the cursor position before waiting for the argument of CTRL-G. (Yasuhiro Matsumoto)
相关文件: src/edit.c

Patch 6.1.282
问题: Elvis uses "se" in a modeline, Vim doesn't recognize this.
解决方案: Also accept "se " where "set " is accepted in a modeline. (Yasuhiro Matsumoto)
相关文件: src/buffer.c

Patch 6.1.283
问题: For ":sign" the icon file name cannot contain a space.
解决方案: Handle backslashes in the file name. (Yasuhiro Matsumoto)
相关文件: src/ex_cmds.c

Patch 6.1.284
问题: On Solaris there is a warning for "struct utimbuf".
解决方案: Move including "utime.h" to outside the function. (Derek Wyatt)
相关文件: src/fileio.c

Patch 6.1.285
问题: Can't wipe out a buffer with 'bufhide' option.
解决方案: Add "wipe" value to 'bufhide'. (Yegappan Lakshmanan)
相关文件: runtime/doc/options.txt, src/buffer.c, src/option.c, src/quickfix.c

Patch 6.1.286
问题: 'showbreak' cannot contain multi-byte characters.
解决方案: Allow using all printable characters for 'showbreak'.
相关文件: src/charset.c, src/move.c, src/option.c

Patch 6.1.287 (depends on 6.1.285)
问题:　　Effect of "delete" and "wipe" in 'bufhide' were mixed up.
解决方案:　Wipe out when wiping out is asked for.
相关文件:　src/buffer.c

Patch 6.1.288
问题:　　":silent function F" hangs. (Hari Krishna Dara)
解决方案:　Don't use msg_col, it is not incremented when using ":silent".
　　Also made the function output look a bit better. Don't translate
　　"function".
相关文件:　src/eval.c

Patch 6.1.289 (depends on 6.1.278)
问题:　　Compiler warning for pointer. (Axel Kielhorn)
解决方案:　Add a typecast for " ".
相关文件:　src/screen.c

Patch 6.1.290 (extra)
问题:　　Truncating long text for message box may break multi-byte
　　character.
解决方案:　Adjust to start of multi-byte character. (Yasuhiro Matsumoto)
相关文件:　src/os_mswin.c

Patch 6.1.291 (extra)
问题:　　Win32: CTRL-@ doesn't work. Don't even get a message for it.
解决方案:　Recognize the keycode for CTRL-@. (Yasuhiro Matsumoto)
相关文件:　src/gui_w48.c

Patch 6.1.292 (extra, depends on 6.1.253)
问题:　　Win32: Can't compile with new MingW compiler.
　　Borland 5 makefile doesn't generate pathdef.c.
解决方案:　Remove -wwide-multiply argument. (Rene de Zwart)
　　Various fixes for other problems in Win32 makefiles. (Dan Sharp)
相关文件:　src/Make_bc5.mak, src/Make_cyg.mak, src/Make_ming.mak,
　　src/Make_mvc.mak

Patch 6.1.293
问题:　　byte2line() returns a wrong result for some values.
解决方案:　Change ">=" to ">" in ml_find_line_or_offset(). (Bradford C Smith)
　　Add one to the line number when at the end of a block.
相关文件:　src/memline.c

Patch 6.1.294
问题:　　Can't include a multi-byte character in a string by its hex value.
　　(Benji Fisher)
解决方案:　Add "\u....": a character specified with up to four hex numbers
　　and stored according to the value of 'encoding'.
相关文件:　src/eval.c

Patch 6.1.295 (extra)
问题:　　Processing the cs.po file generates an error. (Rahul Agrawal)
解决方案:　Fix the printf format characters in the translation.
相关文件:　src/po/cs.po

Patch 6.1.296
问题: Win32: When cancelling the font dialog 'guifont' remains set to "*".
解决方案: Restore the old value of 'guifont' (Yasuhiro Matsumoto)
相关文件: src/option.c

Patch 6.1.297
问题: "make test" fails in test6 in an UTF-8 environment. (Benji Fisher)
解决方案: Before executing the BufReadPost autocommands save the current fileencoding, so that the file isn't marked changed.
相关文件: src/fileio.c

Patch 6.1.298
问题: When using signs and the first line of a closed fold has a sign it can be redrawn as if the fold was open. (Kamil Burzynski)
解决方案: Don't redraw a sign inside a closed fold.
相关文件: src/screen.c

Patch 6.1.299
问题: ":edit +set\ ro file" doesn't work.
解决方案: Halve the number of backslashes in the "+cmd" argument.
相关文件: src/ex_docmd.c

Patch 6.1.300 (extra)
问题: Handling of ETO_IGNORELANGUAGE is confusing.
解决方案: Clean up the handling of ETO_IGNORELANGUAGE. (Glenn Maynard)
相关文件: src/gui_w32.c

Patch 6.1.301 (extra)
问题: French translation of file-save dialog doesn't show file name.
解决方案: Insert a star in the printf string. (Francois Terrot)
相关文件: src/po/fr.po

Patch 6.1.302
问题: Counting lines of the Visual area is incorrect for closed folds. (Mikolaj Machowski)
解决方案: Correct the start and end for the closed fold.
相关文件: src/normal.c

Patch 6.1.303 (extra)
问题: The Top/Bottom/All text does not always fit in the ruler when translated to Japanese. Problem with a character being wider when in a bold font.
解决方案: Use ETO_PDY to specify the width of each character. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 6.1.304 (extra, depends on 6.1.292)
问题: Win32: Postscript is always enabled in the MingW Makefile. Pathdef.c isn't generated properly with Make_bc5.mak. (Yasuhiro Matsumoto)
解决方案: Change an ifdef to an ifeq. (Madoka Machitani)
Use the Borland make redirection to generate pathdef.c. (Maurice Barnum)

相关文件: src/Make_bc5.mak, src/Make_ming.mak

Patch 6.1.305
问题: When 'verbose' is 14 or higher, a function call may cause reading
uninitialized data. (Walter Briscoe)
解决方案: Check for end-of-string in trunc_string().
相关文件: src/message.c

Patch 6.1.306
问题: The AIX VisualAge cc compiler doesn't define __STDC__.
解决方案: Use __EXTENDED__ like __STDC__. (Jess Thrysoee)
相关文件: src/os_unix.h

Patch 6.1.307
问题: When a double-byte character has an illegal tail byte the display
is messed up. (Yasuhiro Matsumoto)
解决方案: Draw "XX" instead of the wrong character.
相关文件: src/screen.c

Patch 6.1.308
问题: Can't reset the Visual mode returned by visualmode().
解决方案: Use an optional argument to visualmode(). (Charles Campbell)
相关文件: runtime/doc/eval.txt, src/eval.c, src/normal.c,
src/structs.h

Patch 6.1.309
问题: The tutor doesn't select German if the locale name is
"German_Germany.1252". (Joachim Hofmann)
解决方案: Check for "German" in the locale name. Also check for
".ge". And include the German and Greek tutors.
相关文件: runtime/tutor/tutor.de, runtime/tutor/tutor.vim,
runtime/tutor/tutor.gr, runtime/tutor/tutor.gr.cp737

Patch 6.1.310 (depends on 6.1.307)
问题: All double-byte characters are displayed as "XX".
解决方案: Use ">= 32" instead of "< 32". (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 6.1.311 (extra)
问题: VMS: path in window title doesn't include necessary separator.
file version doesn't always work properly with Unix.
Crashes because of memory overwrite in GUI.
Didn't always handle files with lowercase and correct path.
解决方案: Fix the problems. Remove unnecessary file name translations.
(Zoltan Arpadffy)
相关文件: src/buffer.c, src/ex_cmds2.c, src/fileio.c, src/memline.c,
src/misc1.c, src/misc2.c, src/os_unix.c, src/os_vms.c, src/tag.c

Patch 6.1.312
问题: When using ":silent" debugging is also done silently.
解决方案: Disable silence while at the debug prompt.
相关文件: src/ex_cmds2.c

Patch 6.1.313

问题:　　When a ":drop fname" command is used and "fname" is open in another window, it is also opened in the current window.
解决方案:　Change to the window with "fname" instead. Don't redefine the argument list when dropping only one file.
相关文件:　runtime/doc/windows.txt, src/ex_cmds2.c, src/ex_cmds.c, src/ex_docmd.c, src/proto/ex_cmds2.pro, src/proto/ex_docmd.pro

Patch 6.1.314 (depends on 6.1.126)
问题:　　Missing backslash in "Generic Config file" syntax menu.
解决方案:　Insert the backslash. (Zak Beck)
相关文件:　runtime/makemenu.vim, runtime/synmenu.vim

Patch 6.1.315 (extra)
问题:　　A very long hostname may lead to an unterminated string. Failing to obtain a hostname may result in garbage. (Walter Briscoe)
解决方案:　Add a NUL at the end of the hostname buffer.
相关文件:　src/os_mac.c, src/os_msdos.c, src/os_unix.c, src/os_win16.c, src/os_win32.c

Patch 6.1.316
问题:　　When exiting with "wq" and there is a hidden buffer, after the "file changed" dialog there is a warning for a changed buffer. (Ajit Thakkar)
解决方案:　Do update the buffer timestamps when exiting.
相关文件:　src/fileio.c

Patch 6.1.317
问题:　　Closing a window may cause some of the remaining windows to be positioned wrong if there is a mix of horizontal and vertical splits. (Stefan Ingi Valdimarsson)
解决方案:　Update the frame sizes before updating the window positions.
相关文件:　src/window.c

Patch 6.1.318
问题:　　auto/pathdef.c can include wrong quotes when a compiler flag includes quotes.
解决方案:　Put a backslash before the quotes in compiler flags. (Shinra Aida)
相关文件:　src/Makefile

Patch 6.1.319 (depends on 6.1.276)
问题:　　Using "--remote +cmd file" does not execute "cmd".
解决方案:　Call inputrestore() in the same command line as inputsave(), otherwise it will never get executed.
相关文件:　src/main.c

Patch 6.1.320 (depends on 6.1.313)
问题:　　When a ":drop one\ file" command is used the file "one\ file" is opened, the backslash is not removed. (Taro Muraoka)
解决方案:　Handle backslashes correctly. Always set the argument list to keep it simple.
相关文件:　runtime/doc/windows.txt, src/ex_cmds.c

Patch 6.1.321
问题:　　When 'mouse' includes 'n' but not 'v', don't allow starting Visual

mode with the mouse.
解决方案: Don't use MOUSE_MAY_VIS when there is no 'v' in 'mouse'. (Flemming Madsen)
相关文件: src/normal.c

Patch 6.1.322 (extra, depends on 6.1.315)
问题: Win32: The host name is always "PC " plus the real host name.
解决方案: Don't insert "PC " before the host name.
相关文件: src/os_win32.c

Patch 6.1.323
问题: ":registers" doesn't stop listing for a "q" at the more prompt. (Hari Krishna Dara)
解决方案: Check for interrupt and got_int.
相关文件: src/ops.c, src/proto/ops.pro

Patch 6.1.324
问题: Crash when dragging a vertical separator when <LeftMouse> is remapped to jump to another window.
解决方案: Pass the window pointer to the function doing the dragging instead of always using the current window. (Daniel Elstner)
Also fix that starting a drag changes window focus.
相关文件: src/normal.c, src/proto/window.pro, src/ui.c, src/vim.h, src/window.c

Patch 6.1.325
问题: Shift-Tab is not automatically recognized in an xterm.
解决方案: Add <Esc>[Z as the termcap code. (Andrew Pimlott)
相关文件: src/term.c

Patch 6.1.326
问题: Using a search pattern may read from uninitialized data (Yasuhiro Matsumoto)
解决方案: Initialize pointers to NULL.
相关文件: src/regexp.c

Patch 6.1.327
问题: When opening the "mbyte.txt" help file the utf-8 characters are unreadable, because the fileencoding is forced to be latin1.
解决方案: Check for utf-8 encoding first in help files. (Daniel Elstner)
相关文件: runtime/doc/mbyte.txt, src/fileio.c

Patch 6.1.328
问题: Prototype for enc_canon_search() is missing.
解决方案: Add the prototype. (Walter Briscoe)
相关文件: src/mbyte.c

Patch 6.1.329
问题: When editing a file "a b c" replacing "%" in ":Cmd %" or ":next %" does not work properly. (Hari Krishna Dara)
解决方案: Always escape spaces when expanding "%". Don't split argument for <f-args> in a user command when only one argument is used.
相关文件: src/ex_docmd.c

Patch 6.1.330
问题: GTK, Motif and Athena: Keypad keys produce the same code as non-keypad keys, making it impossible to map them separately.
解决方案: Use different termcap codes for the keypad keys. (Neil Bird)
相关文件: src/gui_gtk_x11.c, src/gui_x11.c

Patch 6.1.331
问题: When translating the help files, "LOCAL ADDITIONS" no longer marks the spot where help files from plugins are to be listed.
解决方案: Add a "local-additions" tag and use that to find the right spot.
相关文件: runtime/doc/help.txt, src/ex_cmds.c

Patch 6.1.332 (extra)
问题: Win32: Loading Perl dynamically doesn't work with Perl 5.8. Perl 5.8 also does not work with Cygwin and Ming.
解决方案: Adjust the function calls. (Taro Muraoka) Adjust the cyg and ming makefiles. (Dan Sharp)
相关文件: src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak, src/if_perl.xs

Patch 6.1.333 (extra)
问题: Win32: Can't handle Unicode text on the clipboard. Can't pass NUL byte, it becomes a line break. (Bruce DeVisser)
解决方案: Support Unicode for the clipboard (Ron Aaron and Glenn Maynard) Also support copy/paste of NUL bytes.
相关文件: src/os_mswin.c, src/os_win16.c src/os_win32.c

Patch 6.1.334 (extra, depends on 6.1.303)
问题: Problem with drawing Hebrew characters.
解决方案: Only use ETO_PDY for Windows NT and the like. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 6.1.335 (extra)
问题: Failure of obtaining the cursor position and window size is ignored.
解决方案: Remove a semicolon after an "if". (Walter Briscoe)
相关文件: src/gui_w32.c

Patch 6.1.336 (extra)
问题: Warning for use of function prototypes of smsg().
解决方案: Define HAVE_STDARG_H. (Walter Briscoe)
相关文件: src/os_win32.h

Patch 6.1.337
问题: When using "finish" in debug mode in function B() for ":call A(B())" does not stop after B() is finished.
解决方案: Increase debug_level while evaluating a function.
相关文件: src/ex_docmd.c

Patch 6.1.338
问题: When using a menu that checks out the current file from Insert mode, there is no warning for the changed file until exiting Insert mode. (Srikanth Sankaran)
解决方案: Add a check for need_check_timestamps in the Insert mode loop.

相关文件: src/edit.c

Patch 6.1.339
问题: Completion doesn't allow "g:" in ":let g:did_<Tab>". (Benji Fisher)
解决方案: Return "g:var" for global variables when that is what is being expanded. (Flemming Madsen)
相关文件: src/eval.c

Patch 6.1.340 (extra, depends on 6.1.332)
问题: Win32: Can't compile the Perl interface with nmake.
解决方案: Don't compare the version number as a string but as a number. (Juergen Kraemer)
相关文件: src/Make_mvc.mak

Patch 6.1.341
问题: In Insert mode with 'rightleft' set the cursor is drawn halfway a double-wide character. For CTRL-R and CTRL-K in Insert mode the " or ? is not displayed.
解决方案: Draw the cursor in the next character cell. Display the " or ? over the right half of the double-wide character. (Yasuhiro Matsumoto) Also fix that cancelling a digraph doesn't redraw a double-byte character correctly.
相关文件: src/edit.c, src/gui.c, src/mbyte.c

Patch 6.1.342 (depends on 6.1.341)
问题: With 'rightleft' set typing "c" on a double-wide character causes the cursor to be displayed one cell to the left.
解决方案: Draw the cursor in the next character cell. (Yasuhiro Matsumoto)
相关文件: src/gui.c

Patch 6.1.343 (depends on 6.1.342)
问题: Cannot compile with the +multi_byte feature but without +rightleft. Cannot compile without the GUI.
解决方案: Fix the #ifdefs. (partly by Nam SungHyun)
相关文件: src/gui.c, src/mbyte.c, src/ui.c

Patch 6.1.344
问题: When using ":silent filetype" the output is still put in the message history. (Hari Krishna Dara)
解决方案: Don't add messages in the history when ":silent" is used.
相关文件: src/message.c

Patch 6.1.345 (extra)
问题: Win32: 'imdisable' doesn't work.
解决方案: Make 'imdisable' work. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 6.1.346
问题: The scroll wheel can only scroll the current window.
解决方案: Make the scroll wheel scroll the window that the mouse points to. (Daniel Elstner)
相关文件: src/edit.c, src/gui.c, src/normal.c, src/term.c

Patch 6.1.347
问题: When using cscope to list matching tags, the listed number is sometimes not equal to what cscope uses. (Vihren Milev)
解决方案: For cscope tags use only one table, don't give tags in the current file a higher priority.
相关文件: src/tag.c

Patch 6.1.348
问题: Wildmode with wildmenu: ":set wildmode=list,full", ":colorscheme <tab>" results in "zellner" instead of the first entry. (Anand Hariharan)
解决方案: Don't call ExpandOne() from globpath(). (Flemming Madsen)
相关文件: src/ex_getln.c

Patch 6.1.349
问题: "vim --serverlist" when no server was ever started gives an error message without "\n".
"vim --serverlist" doesn't exit when the X server can't be contacted, it starts Vim unexpectedly. (Ricardo Signes)
解决方案: Don't give an error when no Vim server was ever started.
Treat failing of opening the display equal to errors inside the remote*() functions. (Flemming Madsen)
相关文件: src/if_xcmdsrv.c, src/main.c

Patch 6.1.350
问题: When entering a buffer with ":bnext" for the first time, using an autocommand to restore the last used cursor position doesn't work. (Paolo Giarusso)
解决方案: Don't use the last known cursor position of the current Vim invocation if an autocommand changed the position.
相关文件: src/buffer.c

Patch 6.1.351 (depends on 6.1.349)
问题: Crash when starting Vim the first time in an X server. (John McGowan)
解决方案: Don't call xFree() with a fixed string.
相关文件: src/if_xcmdsrv.c

Patch 6.1.352 (extra, depends on 6.1.345)
问题: Win32: Crash when setting "imdisable" in _vimrc.
解决方案: Don't call IME functions when imm32.dll was not loaded (yet).
Also add typecasts to avoid Compiler warnings for ImmAssociateContext() argument.
相关文件: src/gui_w32.c

Patch 6.1.353 (extra, depends on 6.1.334)
问题: Problem with drawing Arabic characters.
解决方案: Don't use ETO_PDY, do use padding.
相关文件: src/gui_w32.c

Patch 6.1.354 (extra, depends on 6.1.333)
问题: MS-Windows 98: Notepad can't paste text copied from Vim when 'encoding' is "utf-8".
解决方案: Also make CF_TEXT available on the clipboard. (Ron Aaron)

相关文件: src/os_mswin.c

Patch 6.1.355
问题: In a regexp '\n' will never match anything in a string.
解决方案: Make '\n' match a newline character.
相关文件: src/buffer.c, src/edit.c, src/eval.c, src/ex_cmds2.c,
src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/misc1.c,
src/option.c, src/os_mac.c, src/os_unix.c, src/quickfix.c,
src/regexp.c, src/search.c, src/syntax.c, src/tag.c, src/vim.h

Patch 6.1.356 (extra, depends on, well, eh, several others)
问题: Compiler warnings for using convert_setup() and a few other
things.
解决方案: Add typecasts.
相关文件: src/mbyte.c, src/os_mswin.c, src/proto/os_win32.pro, src/os_win32.c

Patch 6.1.357
问题: CR in the quickfix window jumps to the error under the cursor, but
this doesn't work in Insert mode. (Srikanth Sankaran)
解决方案: Handle CR in Insert mode in the quickfix window.
相关文件: src/edit.c

Patch 6.1.358
问题: The tutor doesn't select another locale version properly.
解决方案: Insert the "let" command. (Yasuhiro Matsumoto)
相关文件: runtime/tutor/tutor.vim

Patch 6.1.359 (extra)
问题: Mac Carbon: Vim doesn't get focus when started from the command
line. Crash when using horizontal scroll bar.
解决方案: Set Vim as the frontprocess. Fix scrolling. (Peter Cucka)
相关文件: src/gui_mac.c

Patch 6.1.360 (depends on 6.1.341)
问题: In Insert mode CTRL-K ESC messes up a multi-byte character.
(Anders Helmersson)
解决方案: Save all bytes of a character when displaying a character
temporarily.
相关文件: src/edit.c, src/proto/screen.pro, src/screen.c

Patch 6.1.361
问题: Cannot jump to a file mark with ":'M".
解决方案: Allow jumping to another file for a mark in an Ex address when it
is the only thing in the command line.
相关文件: src/ex_docmd.c

Patch 6.1.362
问题: tgetent() may return zero for success. tgetflag() may return -1
for an error.
解决方案: Check tgetflag() for returning a positive value. Add an autoconf
check for the value that tgetent() returns.
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/term.c

Patch 6.1.363

问题:　　byte2line() can return one more than the number of lines.
解决方案:　Return -1 if the offset is one byte past the end.
相关文件:　src/memline.c

Patch 6.1.364
问题:　　That the FileChangedShell autocommand event never nests makes it difficult to reload a file in a normal way.
解决方案:　Allow nesting for the FileChangedShell event but do not allow triggering itself again.
Also avoid autocommands for the cmdline window in rare cases.
相关文件:　src/ex_getln.c, src/fileio.c, src/window.c

Patch 6.1.365 (depends on 6.1.217)
问题:　　Setting a breakpoint in a sourced file with a relative path name doesn't work. (Servatius Brandt)
解决方案:　Expand the file name to a full path.
相关文件:　src/ex_cmds2.c

Patch 6.1.366
问题:　　Can't use Vim with Netbeans.
解决方案:　Add the Netbeans interface. Includes support for sign icons and "-fg" and "-bg" arguments for GTK. Add the 'autochdir' option. (Gordon Prieur, George Hernandez, Dave Weatherford)
Make it possible to display both a sign with a text and one with line highlighting in the same line.
Add support for Agide, interface version 2.1.
Also fix that when 'iskeyword' includes '?' the "*" command doesn't work properly on a word that includes "?" (Bill McCarthy): Don't escape "?" to "\?" when searching forward.
相关文件:　runtime/doc/Makefile, runtime/doc/netbeans.txt, runtime/doc/options.txt, runtime/doc/various.txt, src/Makefile, src/auto/configure, src/buffer.c, src/config.h.in, src/config.mk.in, src/configure.in, src/edit.c, src/ex_cmds.c, src/ex_docmd.c, src/feature.h, src/fileio.c, src/globals.h, src/gui.c, src/gui_beval.c, src/gui_gtk_x11.c, src/gui_x11.c, src/main.c, src/memline.c, src/misc1.c, src/misc2.c, src/move.c, src/nbdebug.c, src/nbdebug.h, src/netbeans.c, src/normal.c, src/ops.c, src/option.c, src/option.h, src/proto/buffer.pro, src/proto/gui_beval.pro, src/proto/gui_gtk_x11.pro, src/proto/gui_x11.pro, src/proto/misc2.pro, src/proto/netbeans.pro, src/proto/normal.pro, src/proto/ui.pro, src/proto.h, src/screen.c, src/structs.h, src/ui.c, src/undo.c, src/vim.h, src/window.c, src/workshop.c

Patch 6.1.367 (depends on 6.1.365)
问题:　　Setting a breakpoint in a function doesn't work. For a sourced file it doesn't work when symbolic links are involved. (Servatius Brandt)
解决方案:　Expand the file name in the same way as do_source() does. Don't prepend the path to a function name.
相关文件:　src/ex_cmds2.c

Patch 6.1.368
问题:　　Completion for ":map" does not include <silent> and <script>.

":mkexrc" do not save the <silent> attribute of mappings.
解决方案: Add "<silent>" to the generated map commands when appropriate. (David Elstner)
Add <silent> and <script> to command line completion.
相关文件: src/getchar.c

Patch 6.1.369 (extra)
问题: VMS: Vim hangs when attempting to edit a read-only file in the terminal. Problem with VMS filenames for quickfix.
解决方案: Rewrite low level input. Remove version number from file name in a couple more places. Fix crash after patch 6.1.362. Correct return code for system(). (Zoltan Arpadffy, Tomas Stehlik)
相关文件: src/misc1.c, src/os_unix.c, src/os_vms.c, src/proto/os_vms.pro, src/os_vms_conf.h, src/quickfix.c, src/ui.c

Patch 6.1.370
问题: #ifdef nesting is unclear.
解决方案: Insert spaces to indicate the nesting.
相关文件: src/os_unix.c

Patch 6.1.371
问题: "%V" in 'statusline' doesn't show "0-1" in an empty line.
解决方案: Add one to the column when comparing with virtual column (Andrew Pimlott)
相关文件: src/buffer.c

Patch 6.1.372
问题: With 16 bit ints there are compiler warnings. (Walter Briscoe)
解决方案: Change int into long.
相关文件: src/structs.h, src/syntax.c

Patch 6.1.373
问题: The default page header for printing is not translated.
解决方案: Add _() around the two places where "Page" is used. (Mike Williams) Translate the default value of the 'titleold' and 'printheader' options.
相关文件: src/ex_cmds2.c, src/option.c

Patch 6.1.374 (extra)
问题: MS-Windows: Cannot build GvimExt with MingW or Cygwin.
解决方案: Add makefile and modified resource files. (Rene de Zwart)
Also support Cygwin. (Alejandro Lopez_Valencia)
相关文件: GvimExt/Make_cyg.mak, GvimExt/Make_ming.mak, GvimExt/Makefile, GvimExt/gvimext_ming.def, GvimExt/gvimext_ming.rc

Patch 6.1.375
问题: MS-Windows: ':!dir "%"' does not work for a file name with spaces. (Xiangjiang Ma)
解决方案: Don't insert backslashes for spaces in a shell command.
相关文件: src/ex_docmd.c

Patch 6.1.376
问题: "vim --version" and "vim --help" have a non-zero exit code. That is unusual. (Petesea)

解决方案: Use a zero exit code.
相关文件: src/main.c

Patch 6.1.377
问题: Can't add words to 'lispwords' option.
解决方案: Add P_COMMA and P_NODUP flags. (Haakon Riiser)
相关文件: src/option.c

Patch 6.1.378
问题: When two buffer-local user commands are ambiguous, a full match
with a global user command isn't found. (Hari Krishna Dara)
解决方案: Detect this situation and accept the global command.
相关文件: src/ex_docmd.c

Patch 6.1.379
问题: Linux with kernel 2.2 can't use the alternate stack in combination
with threading, causes an infinite loop.
解决方案: Don't use the alternate stack in this situation.
相关文件: src/os_unix.c

Patch 6.1.380
问题: When 'winminheight' is zero and the quickfix window is zero lines,
entering the window doesn't make it higher. (Christian J.
Robinson)
解决方案: Make sure the current window is at least one line high.
相关文件: src/window.c

Patch 6.1.381
问题: When a BufWriteCmd is used and it leaves the buffer modified, the
window may still be closed. (Hari Krishna Dara)
解决方案: Return FAIL from buf_write() when the buffer is still modified
after a BufWriteCmd autocommand was used.
相关文件: src/fileio.c

Patch 6.1.382 (extra)
问题: Win32 GUI: When using two monitors, the code that checks/fixes the
window size and position (e.g. when a font changes) doesn't work
properly. (George Reilly)
解决方案: Handle a double monitor situation. (Helmut Stiegler)
相关文件: src/gui_w32.c

Patch 6.1.383
问题: The filling of the status line doesn't work properly for
multi-byte characters. (Nam SungHyun)
There is no check for going past the end of the buffer.
解决方案: Properly distinguish characters and bytes. Properly check for
running out of buffer space.
相关文件: src/buffer.c, src/ex_cmds2.c, src/proto/buffer.pro, src/screen.c

Patch 6.1.384
问题: It is not possible to find if a certain patch has been included.
(Lubomir Host)
解决方案: Support using has() to check if a patch was included.
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/version.pro,

src/version.c

Patch 6.1.385 (depends on 6.1.383)
问题: Can't compile without the multi-byte feature.
解决方案: Move an #ifdef. (Christian J. Robinson)
相关文件: src/buffer.c

Patch 6.1.386
问题: Get duplicate tags when running ":helptags".
解决方案: Do the other halve of moving a section to another help file.
相关文件: runtime/tagsrch.txt

Patch 6.1.387 (depends on 6.1.373)
问题: Compiler warning for pointer cast.
解决方案: Add (char_u *).
相关文件: src/option.c

Patch 6.1.388 (depends on 6.1.384)
问题: Compiler warning for pointer cast.
解决方案: Add (char *). Only include has_patch() when used.
相关文件: src/eval.c, src/version.c

Patch 6.1.389 (depends on 6.1.366)
问题: Balloon evaluation doesn't work for GTK.
has("balloon_eval") doesn't work.
解决方案: Add balloon evaluation for GTK. Also improve displaying of signs.
(Daniel Elstner)
Also make ":gui" start the netbeans connection and avoid using
netbeans functions when the connection is not open.
相关文件: src/Makefile, src/feature.h, src/gui.c, src/gui.h,
src/gui_beval.c, src/gui_beval.h, src/gui_gtk.c,
src/gui_gtk_x11.c, src/eval.c, src/memline.c, src/menu.c,
src/netbeans.c, src/proto/gui_beval.pro, src/proto/gui_gtk.pro,
src/structs.h, src/syntax.c, src/ui.c, src/workshop.c

Patch 6.1.390 (depends on 6.1.389)
问题: It's not possible to tell Vim to save and exit through the
Netbeans interface. Would still try to send balloon eval text
after the connection is closed.
Can't use Unicode characters for sign text.
解决方案: Add functions "saveAndExit" and "getModified". Check for a
working connection before sending a balloonText event.
various other cleanups.
Support any character for sign text. (Daniel Elstner)
相关文件: runtime/doc/netbeans.txt, runtime/doc/sign.txt, src/ex_cmds.c,
src/netbeans.c, src/screen.c

Patch 6.1.391
问题: ml_get() error when using virtualedit. (Charles Campbell)
解决方案: Get a line from a specific window, not the current one.
相关文件: src/charset.c

Patch 6.1.392 (depends on 6.1.383)
问题: Highlighting in the 'statusline' is in the wrong position when an

item is truncated. (Zak Beck)
解决方案: Correct the start of 'statusline' items properly for a truncated item.
相关文件: src/buffer.c

Patch 6.1.393
问题: When compiled with Python and threads, detaching the terminal may cause Vim to loop forever.
解决方案: Add -pthread to $CFLAGS when using Python and gcc. (Daniel Elstner)
相关文件: src/auto/configure,, src/configure.in

Patch 6.1.394 (depends on 6.1.390)
问题: The netbeans interface doesn't recognize multibyte glyph names.
解决方案: Check the number of cells rather than bytes to decide whether a glyph name is not a filename. (Daniel Elstner)
相关文件: src/netbeans.c

Patch 6.1.395 (extra, depends on 6.1.369)
问题: VMS: OLD_VMS is never defined. Missing function prototype.
解决方案: Define OLD_VMS in Make_vms.mms. Add vms_sys_status() to os_vms.pro. (Zoltan Arpadffy)
相关文件: src/Make_vms.mms, src/proto/os_vms.pro

Patch 6.1.396 (depends on 6.1.330)
问题: Compiler warnings for using enum.
解决方案: Add typecast to char_u.
相关文件: src/gui_gtk_x11.c, src/gui_x11.c

Patch 6.1.397 (extra)
问题: The install program may use a wrong path for the diff command if there is a space in the install directory path.
解决方案: Use double quotes around the path if necessary. (Alejandro Lopez-Valencia) Also use double quotes around the file name arguments.
相关文件: src/dosinst.c

Patch 6.1.398
问题: Saving the typeahead for debug mode causes trouble for a test script. (Servatius Brandt)
解决方案: Add the ":debuggreedy" command to avoid saving the typeahead.
相关文件: runtime/doc/repeat.txt, src/ex_cmds.h, src/ex_cmds2.c, src/ex_docmd.c, src/proto/ex_cmds2.pro

Patch 6.1.399
问题: Warning for unused variable.
解决方案: Remove the variable two_or_more.
相关文件: src/ex_cmds.c

Patch 6.1.400 (depends on 6.1.381)
问题: When a BufWriteCmd wipes out the buffer it may still be accessed.
解决方案: Don't try accessing a buffer that has been wiped out.
相关文件: src/fileio.c

Patch 6.1.401 (extra)
问题: Building the Win16 version with Borland 5.01 doesn't work.
"make test" doesn't work with Make_dos.mak. (Walter Briscoe)
解决方案: Various fixes to the w16 makefile. (Walter Briscoe)
Don't use deltree. Use "mkdir \tmp" instead of "mkdir /tmp".
相关文件: src/Make_w16.mak, src/testdir/Make_dos.mak

Patch 6.1.402
问题: When evaluating a function name with curly braces, an error
is not handled consistently.
解决方案: Accept the result of a curly braces expression when an
error was encountered. Skip evaluating an expression in curly
braces when skipping. (Servatius Brandt)
相关文件: src/eval.c

Patch 6.1.403 (extra)
问题: MS-Windows 16 bit: compiler warnings.
解决方案: Add typecasts. (Walter Briscoe)
相关文件: src/ex_cmds2.c, src/gui_w48.c, src/os_mswin.c, src/os_win16.c,
src/syntax.c

Patch 6.1.404 (extra)
问题: Various small problems.
解决方案: Fix comments. Various small additions, changes in indent, removal
of unused items and fixes.
相关文件: Makefile, README.txt, runtime/menu.vim, runtime/vimrc_example.vim,
src/INSTALL, src/INSTALLole.txt, src/Make_bc5.mak,
src/Make_cyg.mak, src/Make_ming.mak, src/Makefile,
src/config.h.in, src/edit.c, src/eval.c, src/ex_cmds2.c,
src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/getchar.c,
src/gui.c, src/gui_gtk.c, src/gui_photon.c, src/if_cscope.c,
src/if_python.c, src/keymap.h, src/mark.c, src/mbyte.c,
src/message.c, src/misc1.c, src/misc2.c, src/normal.c,
src/option.c, src/os_os2_cfg.h, src/os_win32.c,
src/proto/getchar.pro, src/proto/message.pro,
src/proto/regexp.pro, src/screen.c, src/structs.h, src/syntax.c,
src/term.c, src/testdir/test15.in, src/testdir/test15.ok,
src/vim.rc, src/xxd/Make_cyg.mak, src/xxd/Makefile

Patch 6.1.405
问题: A few files are missing from the toplevel Makefile.
解决方案: Add the missing files.
相关文件: Makefile

Patch 6.1.406 (depends on 6.1.392)
问题: When a statusline item doesn't fit arbitrary text appears.
(Christian J. Robinson)
解决方案: When there is just enough room but not for the "<" truncate the
statusline item like there is no room.
相关文件: src/buffer.c

Patch 6.1.407
问题: ":set scrollbind | help" scrollbinds the help window. (Andrew
Pimlott)

解决方案: Reset 'scrollbind' when opening a help window.
相关文件: src/ex_cmds.c

Patch 6.1.408
问题: When 'rightleft' is set unprintable character 0x0c is displayed as ">c0<".
解决方案: Reverse the text of the hex character.
相关文件: src/screen.c

Patch 6.1.409
问题: Generating tags for the help doesn't work for some locales.
解决方案: Set LANG=C LC_ALL=C in the environment for "sort". (Daniel Elstner)
相关文件: runtime/doc/Makefile

Patch 6.1.410 (depends on 6.1.390)
问题: Linking error when compiling with Netbeans but without sign icons. (Malte Neumann)
解决方案: Don't define buf_signcount() when sign icons are unavailable.
相关文件: src/buffer.c

Patch 6.1.411
问题: When 'virtualedit' is set, highlighting a Visual block beyond the end of a line may be wrong.
解决方案: Correct the virtual column when the end of the line is before the displayed part of the line. (Muraoka Taro)
相关文件: src/screen.c

Patch 6.1.412
问题: When swapping terminal screens and using ":gui" to start the GUI, the shell prompt may be after a hit-enter prompt.
解决方案: Output a newline in the terminal when starting the GUI and there was a hit-enter prompt..
相关文件: src/gui.c

Patch 6.1.413
问题: When 'clipboard' contains "unnamed", "p" in Visual mode doesn't work correctly.
解决方案: Save the register before overwriting it and put the resulting text on the clipboard afterwards. (Muraoka Taro)
相关文件: src/normal.c, src/ops.c

Patch 6.1.414 (extra, depends on 6.1.369)
问题: VMS: Vim busy waits when waiting for input.
解决方案: Delay for a short while before getting another character. (Zoltan Arpadffy)
相关文件: src/os_vms.c

Patch 6.1.415
问题: When there is a vertical split and a quickfix window, reducing the size of the Vim window may result in a wrong window layout and a crash.
解决方案: When reducing the window size and there is not enough space for 'winfixheight' set the frame height to the larger height, so that

there is a retry while ignoring 'winfixheight'. (Yasuhiro Matsumoto)
相关文件: src/window.c

Patch 6.1.416 (depends on 6.1.366)
问题: When using the Netbeans interface, a line with a sign cannot be changed.
解决方案: Respect the GUARDEDOFFSET for sign IDs when checking for a guarded area.
相关文件: src/netbeans.c

Patch 6.1.417
问题: Unprintable multi-byte characters are not handled correctly. Multi-byte characters above 0xffff are displayed as another character.
解决方案: Handle unprintable multi-byte characters. Display multi-byte characters above 0xffff with a marker. Recognize UTF-16 words and BOM words as unprintable. (Daniel Elstner)
相关文件: src/charset.c, src/mbyte.c, src/screen.c

Patch 6.1.418
问题: The result of strftime() is in the current locals. Need to convert it to 'encoding'.
解决方案: Obtain the current locale and convert the argument for strftime() to it and the result back to 'encoding'. (Daniel Elstner)
相关文件: src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/mbyte.c, src/proto/mbyte.pro, src/option.c, src/os_mswin.c

Patch 6.1.419
问题: Vim doesn't compile on AIX 5.1.
解决方案: Don't define _NO_PROTO on this system. (Uribarri)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.420 (extra)
问题: convert_input() has an unnecessary STRLEN(). Conversion from UCS-2 to a codepage uses word count instead of byte count.
解决方案: Remove the STRLEN() call. (Daniel Elstner) Always use byte count for string_convert().
相关文件: src/gui_w32.c, src/mbyte.c

Patch 6.1.421 (extra, depends on 6.1.354)
问题: MS-Windows 9x: When putting text on the clipboard it can be in the wrong encoding.
解决方案: Convert text to the active codepage for CF_TEXT. (Glenn Maynard)
相关文件: src/os_mswin.c

Patch 6.1.422
问题: Error in .vimrc doesn't cause hit-enter prompt when swapping screens. (Neil Bird)
解决方案: Set msg_didany also when sending a message to the terminal directly.
相关文件: src/message.c

Patch 6.1.423
问题:　　Can't find arbitrary text in help files.
解决方案:　Added the ":helpgrep" command.
相关文件:　runtime/doc/various.txt, src/ex_cmds.h, src/ex_docmd.c,
　　src/proto/quickfix.pro, src/quickfix.c

Patch 6.1.424 (extra)
问题:　　Win32: gvim compiled with VC++ 7.0 run on Windows 95 does not show
　　menu items.
解决方案:　Define $WINVER to avoid an extra item is added to MENUITEMINFO.
　　(Muraoka Taro)
相关文件:　src/Make_mvc.mak

Patch 6.1.425
问题:　　":helptags $VIMRUNTIME/doc" does not add the "help-tags" tag.
解决方案:　Do add the "help-tags" tag for that specific directory.
相关文件:　src/ex_cmds.c

Patch 6.1.426
问题:　　"--remote-wait +cmd file" waits forever. (Valery Kondakoff)
解决方案:　Don't wait for the "+cmd" argument to have been edited.
相关文件:　src/main.c

Patch 6.1.427
问题:　　Several error messages for regexp patterns are not translated.
解决方案:　Use _() properly. (Muraoka Taro)
相关文件:　src/regexp.c

Patch 6.1.428
问题:　　FreeBSD: wait() may hang when compiled with Python support and
　　doing a system() call in a startup script.
解决方案:　Use waitpid() instead of wait() and poll every 10 msec, just like
　　what is done in the GUI.
相关文件:　src/os_unix.c

Patch 6.1.429 (depends on 6.1.390)
问题:　　Crash when using showmarks.vim plugin. (Charles Campbell)
解决方案:　Check for sign_get_text() returning a NULL pointer.
相关文件:　src/screen.c

Patch 6.1.430
问题:　　In Lisp code backslashed parens should be ignored for "%". (Dorai)
解决方案:　Skip over backslashed parens.
相关文件:　src/search.c

Patch 6.1.431
问题:　　Debug commands end up in redirected text.
解决方案:　Disable redirection while handling debug commands.
相关文件:　src/ex_cmds2.c

Patch 6.1.432 (depends on 6.1.375)
问题:　　MS-Windows: ":make %:p" inserts extra backslashes. (David Rennalls)
解决方案:　Don't add backslashes, handle it like ":!cmd".
相关文件:　src/ex_docmd.c

Patch 6.1.433
问题: ":popup" only works for Win32.
解决方案: Add ":popup" support for GTK. (Daniel Elstner)
相关文件: runtime/doc/gui.txt, src/ex_docmd.c, src/gui_gtk.c, src/menu.c, src/proto/gui_gtk.pro

Patch 6.1.434 (extra)
问题: Win32: When there are more than 32767 lines, the scrollbar has a roundoff error.
解决方案: Make a click on an arrow move one line. Also move the code to gui_w48.c, there is hardly any difference between the 16 bit and 32 bit versions. (Walter Briscoe)
相关文件: src/gui_w16.c, src/gui_w32.c, src/gui_w48.c

Patch 6.1.435
问题: ":winsize x" resizes the Vim window to the minimal size. (Andrew Pimlott)
解决方案: Give an error message for wrong arguments of ":winsize" and ":winpos".
相关文件: src/ex_docmd.c

Patch 6.1.436
问题: When a long UTF-8 file contains an illegal byte it's hard to find out where it is. (Ron Aaron)
解决方案: Add the line number to the error message.
相关文件: src/fileio.c

Patch 6.1.437 (extra, depends on 6.1.421)
问题: Using multi-byte functions when they are not available.
解决方案: Put the clipboard conversion inside an #ifdef. (Vince Negri) Also fix a pointer type mistake. (Walter Briscoe)
相关文件: src/os_mswin.c

Patch 6.1.438
问题: When Perl has thread support Vim cannot use the Perl interface.
解决方案: Add a configure check and disable Perl when it will not work. (Aron Griffis)
相关文件: src/auto/configure, src/configure.in

Patch 6.1.439
问题: Netbeans: A "create" function doesn't actually create a buffer, following functions may fail.
解决方案: Create a Vim buffer without a name when "create" is called. (Gordon Prieur)
相关文件: runtime/doc/netbeans.txt, src/netbeans.c

Patch 6.1.440
问题: The "@*" command doesn't obtain the actual contents of the clipboard. (Hari Krishna Dara)
解决方案: Obtain the clipboard text before executing the command.
相关文件: src/ops.c

Patch 6.1.441

问题:　　"zj" and "zk" cannot be used as a motion command after an operator. (Ralf Hetzel)
解决方案:　Accept these commands as motion commands.
相关文件:　src/normal.c

Patch 6.1.442
问题:　　Unicode 3.2 defines more space and punctuation characters.
解决方案:　Add the new characters to the Unicode tables. (Raphael Finkel)
相关文件:　src/mbyte.c

Patch 6.1.443 (extra)
问题:　　Win32: The gvimext.dll build with Borland 5.5 requires another DLL.
解决方案:　Build a statically linked version by default. (Dan Sharp)
相关文件:　GvimExt/Make_bc5.mak

Patch 6.1.444 (extra)
问题:　　Win32: Enabling a build with gettext support is not consistent.
解决方案:　Use "GETTEXT" for Borland and msvc makefiles. (Dan Sharp)
相关文件:　src/Make_bc5.mak, src/Make_mvc.mak

Patch 6.1.445 (extra)
问题:　　DJGPP: get warning for argument of putenv()
解决方案:　Define HAVE_PUTENV to use DJGPP's putenv(). (Walter Briscoe)
相关文件:　src/os_msdos.h

Patch 6.1.446 (extra)
问题:　　Win32: The MingW makefile uses a different style of arguments than other makefiles.
Dynamic IME is not supported for Cygwin.
解决方案:　Use "no" and "yes" style arguments. Remove the use of the dyn-ming.h include file. (Dan Sharp)
Do not include the ime.h file and adjust the makefile. (Alejandro Lopez-Valencia)
相关文件:　src/Make_cyg.mak, src/Make_ming.mak, src/gui_w32.c, src/if_perl.xs, src/if_python.c, src/if_ruby.c, src/os_win32.c

Patch 6.1.447
问题:　　"make install" uses "make" directly for generating help tags.
解决方案:　Use $(MAKE) instead of "make". (Tim Mooney)
相关文件:　src/Makefile

Patch 6.1.448
问题:　　'titlestring' has a default maximum width of 50 chars per item.
解决方案:　Remove the default maximum (also for 'statusline').
相关文件:　src/buffer.c

Patch 6.1.449
问题:　　When "1" and "a" are in 'formatoptions', auto-formatting always moves a newly added character to the next line. (Servatius Brandt)
解决方案:　Don't move a single character to the next line when it was just typed.
相关文件:　src/edit.c

Patch 6.1.450
问题: Termcap entry "kB" for back-tab is not recognized.
解决方案: Use back-tab as the shift-tab code.
相关文件: src/keymap.h, src/misc2.c, src/term.c

Patch 6.1.451
问题: GUI: When text in the find dialog contains a slash, a backslash is inserted the next time it is opened. (Mezz)
解决方案: Remove escaped backslashes and question marks. (Daniel Elstner)
相关文件: src/gui.c

Patch 6.1.452 (extra, after 6.1.446)
问题: Win32: IME support doesn't work for MSVC.
解决方案: Use _MSC_VER instead of __MSVC. (Alejandro Lopez-Valencia)
相关文件: src/gui_w32.c

Patch 6.1.453 (after 6.1.429)
问题: When compiled without sign icons but with sign support, adding a sign may cause a crash.
解决方案: Check for the text sign to exist before using it. (Kamil Burzynski)
相关文件: src/screen.c

Patch 6.1.454 (extra)
问题: Win32: pasting Russian text in Vim with 'enc' set to cp1251 results in utf-8 bytes. (Perelyubskiy)
Conversion from DBCS to UCS2 does not work when 'encoding' is not the active codepage.
解决方案: Introduce enc_codepage and use it for conversion to 'encoding' (Glenn Maynard)
Use MultiByteToWideChar() and WideCharToMultiByte() instead of iconv(). Should do most needed conversions without iconv.dll.
相关文件: src/globals.h, src/gui_w32.c, src/mbyte.c, src/os_mswin.c, src/proto/mbyte.pro, src/proto/os_mswin.pro, src/structs.h

Patch 6.1.455
问题: Some Unicode characters can be one or two character cells wide.
解决方案: Add the 'ambiwidth' option to tell Vim how to display these characters. (Jungshik Shin)
Also reset the script ID when setting an option to its default value, so that ":verbose set" won't give wrong info.
相关文件: runtime/doc/options.txt, src/mbyte.c, src/option.c, src/option.h

Patch 6.1.456 (extra, after 6.1.454)
问题: Win32: IME doesn't work.
解决方案: ImmGetCompositionStringW() returns the size in bytes, not words. (Yasuhiro Matsumoto) Also fix typecast problem.
相关文件: src/gui_w32.c, src/os_mswin.c

Patch 6.1.457
问题: An empty register in viminfo causes conversion to fail.
解决方案: Don't convert an empty string. (Yasuhiro Matsumoto)
相关文件: src/ex_cmds.c, src/mbyte.c

Patch 6.1.458
问题: Compiler warning for pointer.
解决方案: Add a typecast.
相关文件: src/ex_cmds.c

Patch 6.1.459 (extra)
问题: Win32: libcall() may return an invalid pointer and cause Vim to crash.
解决方案: Add a strict check for the returned pointer. (Bruce Mellows)
相关文件: src/os_mswin.c

Patch 6.1.460
问题: GTK: after scrolling the text one line with a key, clicking the arrow of the scrollbar does not always work. (Nam SungHyun)
解决方案: Always update the scrollbar thumb when the value changed, even when it would not move, like for RISCOS. (Daniel Elstner)
相关文件: src/gui.c, src/gui.h

Patch 6.1.461
问题: When a keymap is active, typing a character in Select mode does not use it. (Benji Fisher)
解决方案: Apply Insert mode mapping to the character typed in Select mode.
相关文件: src/normal.c

Patch 6.1.462
问题: When autocommands wipe out a buffer, a crash may happen. (Hari Krishna Dara)
解决方案: Don't decrement the window count of a buffer before calling the autocommands for it. When re-using the current buffer, watch out for autocommands changing the current buffer.
相关文件: src/buffer.c, src/ex_cmds.c, src/proto/buffer.pro

Patch 6.1.463
问题: When writing a compressed file, the file name that gzip stores in the file is the weird temporary file name. (David Rennalls)
解决方案: Use the real file name when possible.
相关文件: runtime/plugin/gzip.vim

Patch 6.1.464
问题: Crash when using C++ syntax highlighting. (Gerhard Hochholzer)
解决方案: Check for a negative index.
相关文件: src/syntax.c

Patch 6.1.465 (after 6.1.454)
问题: Compile error when using cygwin.
解决方案: Change #ifdef WIN32 to #ifdef WIN3264. (Alejandro Lopez-Valencia)
Undefine WIN32 after including windows.h
相关文件: src/mbyte.c

Patch 6.1.466
问题: The "-f" argument is a bit obscure.
解决方案: Add the "--nofork" argument. Improve the help text a bit.
相关文件: runtime/doc/starting.txt, src/main.c

Patch 6.1.467
问题:　　Setting the window title doesn't work for Chinese.
解决方案:　Use an X11 function to convert text to a text property. (Kentaro Nakazawa)
相关文件:　src/os_unix.c

Patch 6.1.468
问题:　　":mksession" also stores folds for buffers which will not be restored.
解决方案:　Only store folds for a buffer with 'buftype' empty and help files.
相关文件:　src/ex_docmd.c

Patch 6.1.469
问题:　　'listchars' cannot contain multi-byte characters.
解决方案:　Handle multi-byte UTF-8 list characters. (Matthew Samsonoff)
相关文件:　src/message.c, src/option.c, src/screen.c

Patch 6.1.470 (lang)
问题:　　Polish messages don't show up correctly on MS-Windows.
解决方案:　Convert messages to cp1250. (Mikolaj Machowski)
Also add English message translations, because it got in the way of the patch.
相关文件:　Makefile, src/po/Makefile, src/po/en_gb.po, src/po/pl.po

Patch 6.1.471
问题:　　":jumps" output continues after pressing "q" at the more-prompt. (Hari Krishna Dara)
解决方案:　Check for "got_int" being set.
相关文件:　src/mark.c

Patch 6.1.472
问题:　　When there is an authentication error when connecting to the X server Vim exits.
解决方案:　Use XSetIOErrorHandler() to catch the error and longjmp() to avoid the exit. Also do this in the main loop, so that when the X server exits a Vim running in a console isn't killed.
相关文件:　src/globals.h, src/main.c, src/os_unix.c

Patch 6.1.473
问题:　　Referring to $curwin or $curbuf in Perl 5.6 causes a crash.
解决方案:　Add "pTHX_" to cur_val(). (Yasuhiro Matsumoto)
相关文件:　src/if_perl.xs

Patch 6.1.474
问题:　　When opening the command-line window in Ex mode it's impossible to go back. (Pavol Juhas)
解决方案:　Reset "exmode_active" and restore it when the command-line window is closed.
相关文件:　src/ex_getln.c

Patch 6.2f.001
问题:　　The configure check for Ruby didn't work properly for Ruby 1.8.0.
解决方案:　Change the way the Ruby check is done. (Aron Griffis)

相关文件: src/auto/configure, src/configure.in

Patch 6.2f.002
问题: The output of ":ls" doesn't show whether a buffer had read errors.
解决方案: Add the "x" flag in the ":ls" output.
相关文件: runtime/doc/windows.txt, src/buffer.c

Patch 6.2f.003
问题: Test49 doesn't properly test the behavior of ":catch" without an
argument.
解决方案: Update test49. (Servatius Brandt)
相关文件: src/testdir/test49.ok, src/testdir/test49.vim

Patch 6.2f.004
问题: "vim --version" always uses CR/LF in the output.
解决方案: Omit the CR.
相关文件: src/message.c, src/os_unix.c

Patch 6.2f.005
问题: Two error messages without a colon after the number.
解决方案: Add the colon. (Taro Muraoka)
相关文件: src/if_cscope.c

Patch 6.2f.006
问题: When saving a file takes a while and Vim regains focus this can
result in a "file changed outside of Vim" warning and ml_get()
errors. (Mike Williams)
解决方案: Add the "b_saving" flag to avoid checking the timestamp while the
buffer is being saved. (Michael Schaap)
相关文件: src/fileio.c, src/structs.h

Patch 6.2f.007
问题: Irix compiler complains about multiple defined symbols.
vsnprintf() is not available. (Charles Campbell)
解决方案: Insert EXTERN for variables in globals.h. Change the configure
check for vsnprintf() from compiling to linking.
相关文件: src/auto/configure, src/configure.in, src/globals.h

Patch 6.2f.008
问题: The Aap recipe doesn't work with Aap 0.149.
解决方案: Change targetarg to TARGETARG. Update the mysign file.
相关文件: src/main.aap, src/mysign

Patch 6.2f.009 (extra)
问题: Small problem when building with Borland 5.01.
解决方案: Use mkdir() instead of _mkdir(). (Walter Briscoe)
相关文件: src/dosinst.h

Patch 6.2f.010
问题: Warning for missing prototypes.
解决方案: Add missing prototypes. (Walter Briscoe)
相关文件: src/if_cscope.c

Patch 6.2f.011

问题:　　The configure script doesn't work with autoconf 2.5x.
解决方案:　Add square brackets around a header check. (Aron Griffis)
　　Note: touch src/auto/configure after applying this patch.
相关文件:　src/configure.in

Patch 6.2f.012
问题:　　":echoerr" doesn't work correctly inside try/endtry.
解决方案:　Don't reset did_emsg inside a try/endtry. (Servatius Brandt)
相关文件:　src/eval.c

Patch 6.2f.013 (extra)
问题:　　Macintosh: Compiler warning for a trigraph.
解决方案:　Insert a backslash before each question mark. (Peter Cucka)
相关文件:　src/os_mac.h

Patch 6.2f.014 (extra)
问题:　　Macintosh: ex_eval is not included in the project file.
解决方案:　Add ex_eval. (Dany St-Amant)
相关文件:　src/os_mac.pbproj/project.pbxproj

Patch 6.2f.015 (extra)
问题:　　Win32: When changing header files not all source files involved
　　are recompiled.
解决方案:　Improve the dependency rules. (Dan Sharp)
相关文件:　src/Make_cyg.mak, src/Make_ming.mak

Patch 6.2f.016
问题:　　"vim --version > ff" on non-Unix systems results in a file with a
　　missing line break at the end. (Bill McCArthy)
解决方案:　Add a line break.
相关文件:　src/main.c

Patch 6.2f.017
问题:　　Unix: starting Vim in the background and then bringing it to the
　　foreground may cause the terminal settings to be wrong.
解决方案:　Check for tcsetattr() to return an error, retry when it does.
　　(Paul Tapper)
相关文件:　src/os_unix.c

Patch 6.2f.018
问题:　　Mac OS X 10.2: OK is defined to zero in cursus.h while Vim uses
　　one. Redefining it causes a warning message.
解决方案:　Undefine OK before defining it to one. (Taro Muraoka)
相关文件:　src/vim.h

Patch 6.2f.019
问题:　　Mac OS X 10.2: COLOR_BLACK and COLOR_WHITE are defined in
　　curses.h.
解决方案:　Rename them to PRCOLOR_BLACK and PRCOLOR_WHITE.
相关文件:　src/ex_cmds2.c

Patch 6.2f.020
问题:　　Win32: test50 produces beeps and fails with some versions of diff.
解决方案:　Remove empty lines and convert the output to dos fileformat.

相关文件: src/testdir/test50.in

Patch 6.2f.021
问题: Running configure with "--enable-netbeans" disables Netbeans.
(Gordon Prieur)
解决方案: Fix the tests in configure.in where the default is to enable a
feature. Fix that "--enable-acl" reported "yes" confusingly.
相关文件: src/auto/configure, src/configure.in, src/mysign

Patch 6.2f.022
问题: A bogus value for 'foldmarker' is not rejected, possibly causing a
hang. (Derek Wyatt)
解决方案: Check for a non-empty string before and after the comma.
相关文件: src/option.c

Patch 6.2f.023
问题: When the help files are not in $VIMRUNTIME but 'helpfile' is
correct Vim still can't find the help files.
解决方案: Also look for a tags file in the directory of 'helpfile'.
相关文件: src/tag.c

Patch 6.2f.024
问题: When 'delcombine' is set and a character has more than two
composing characters "x" deletes them all.
解决方案: Always delete only the last composing character.
相关文件: src/misc1.c

Patch 6.2f.025
问题: When reading a file from stdin that has DOS line endings but a
missing end-of-line for the last line 'fileformat' becomes "unix".
(Bill McCarthy)
解决方案: Don't add the missing line break when re-reading the text from the
buffer.
相关文件: src/fileio.c

Patch 6.2f.026
问题: When typing new text at the command line, old composing characters
may be displayed.
解决方案: Don't read composing characters from after the end of the
text to be displayed.
相关文件: src/ex_getln.c, src/mbyte.c, src/message.c, src/proto/mbyte.pro,
src/screen.c

Patch 6.2f.027
问题: Compiler warnings for unsigned char pointers. (Tony Leneis)
解决方案: Add typecasts to char pointer.
相关文件: src/quickfix.c

Patch 6.2f.028
问题: GTK: When 'imactivatekey' is empty and XIM is inactive it can't be
made active again. Cursor isn't updated immediately when changing
XIM activation. Japanese XIM may hang when using 'imactivatekey'.
Can't activate XIM after typing fFtT command or ":sh".
解决方案: Properly set the flag that indicates the IM is active. Update the

cursor right away. Do not send a key-release event. Handle
Normal mode and running an external command differently.
(Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 6.2f.029
问题: Mixing use of int and enum.
解决方案: Adjust argument type of cs_usage_msg(). Fix wrong typedef.
相关文件: src/if_cscope.c, src/if_cscope.h

Patch 6.2f.030 (after 6.2f.028)
问题: Cursor moves up when using XIM.
解决方案: Reset im_preedit_cursor. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 6.2f.031
问题: Crash when listing a function argument in the debugger. (Ron Aaron)
解决方案: Init the name field of an argument to NULL.
相关文件: src/eval.c

Patch 6.2f.032
问题: When a write fails for a ":silent!" while inside try/endtry the
BufWritePost autocommands are not triggered.
解决方案: Check the emsg_silent flag in should_abort(). (Servatius Brandt)
相关文件: src/ex_eval.c, src/testdir/test49.ok, src/testdir/test49.vim

Patch 6.2f.033
问题: Cscope: re-entrance problem for ":cscope" command. Checking for
duplicate database didn't work well for Win95. Didn't check for
duplicate databases after an empty entry.
解决方案: Don't set postponed_split too early. Remember first empty
database entry. (Sergey Khorev)
相关文件: src/if_cscope.c

Patch 6.2f.034
问题: The netbeans interface cannot be used on systems without
vsnprintf(). (Tony Leneis)
解决方案: Use EMSG(), EMSGN() and EMSG2() instead.
相关文件: src/auto/configure, src/configure.in, src/netbeans.c

Patch 6.2f.035
问题: The configure check for the netbeans interface doesn't work if the
socket and nsl libraries are required.
解决方案: Check for the socket and nsl libraries before the netbeans check.
相关文件: src/auto/configure, src/configure.in

Patch 6.2f.036
问题: Moving leftwards over text with an illegal UTF-8 byte moves one
byte instead of one character.
解决方案: Ignore an illegal byte after the cursor position.
相关文件: src/mbyte.c

Patch 6.2f.037
问题: When receiving a Netbeans command at the hit-enter or more prompt

the screen is redrawn but Vim is still waiting at the prompt.
解决方案: Quit the prompt like a CTRL-C was typed.
相关文件: src/netbeans.c

Patch 6.2f.038
问题: The dependency to run autoconf causes a patch for configure.in to run autoconf, even though the configure script was updated as well.
解决方案: Only run autoconf with "make autoconf".
相关文件: src/Makefile

Patch 6.2f.039
问题: CTRL-W K makes the new top window very high.
解决方案: When 'equalalways' is set equalize the window heights.
相关文件: src/window.c

6.3 版 本 version-6.3

本小节讨论 6.2 和 6.3 版本之间的改进。

这是一个漏洞修正版。也有一些新特性。主要的新项目来自运行时文件和翻译。

改动 changed-6.3

介绍文字也显示赞助 Vim 的说明，和帮助乌干达儿童的消息混合在一起。

正常运行时文件里包含了包含翻译的菜单、键盘映射表和教程。单独的 "lang" 归档现在只包含翻译的消息。

翻译菜单的文件名更加一致。使用 "latin1" 代替 "iso_8859-1" 和 "iso_8859-15"。

发布里删除了 "file_select.vim" 脚本。和 www.vim.org 可以下载的其它脚本比起来，并不更有用。

"runtime/doc/tags" 文件现在总是 unix 文件格式。MS-Windows 上过去使用 dos 文件格式，但 ":helptags" 产生 unix 格式的文件。

新增 added-6.3

新增命令:

:cNfile	跳转到上个文件的最后的一个错误
:cpfile	同上
:changes	显示改变表
:keepmarks	后面跟随的命令保持位置标记不变
:keepjumps	后面跟随的命令保持跳转表和位置标记不变
:lockmarks	后面跟随的命令保持位置标记不变
:redrawstatus	强制刷新状态行

新选项：

'antialias'　　Mac OS X：使用平滑反锯齿字体
'helplang'　　偏好的帮助语言

语法文件：

Arch inventory (Nikolai Weibull)
Calendar (Nikolai Weibull)
Ch (Wayne Cheng)
Controllable Regex Mutilator (Nikolai Weibull)
D (Jason Mills)
Desktop (Mikolaj Machowski)
Dircolors (Nikolai Weibull)
Elinks configuration (Nikolai Weibull)
FASM (Ron Aaron)
GrADS 脚本 (Stefan Fronzek)
Icewm 菜单 (James Mahler)
LDIF (Zak Johnson)
Locale input，fdcc。(Dwayne Bailey)
Pinfo config (Nikolai Weibull)
Pyrex (Marco Barisione)
Relax NG Compact (Nikolai Weibull)
Slice (Morel Bodin)
VAX Macro Assembly (Tom Uijldert)
grads (Stefan Fronzek)
libao (Nikolai Weibull)
mplayer (Nikolai Weibull)
rst (Nikolai Weibull)
tcsh (Gautam Iyer)
yaml (Nikolai Weibull)

编译器插件：

ATT dot (Marcos Macedo)
Apple Project Builder (Alexander von Below)
Intel (David Harrison)
bdf (Nikolai Weibull)
icc (Peter Puck)
javac (Doug Kearns)
neato (Marcos Macedo)
onsgmls (Robert B. Rowsome)
perl (Christian J. Robinson)
rst (Nikolai Weibull)
se (SmartEiffel) (Doug Kearns)
tcl (Doug Kearns)
xmlwf (Robert B. Rowsome)

文件类型插件：

Aap (Bram Moolenaar)
Ch (Wayne Cheng)
Css (Nikolai Weibull)
Pyrex (Marco Barisione)
Rst (Nikolai Weibull)

缩进脚本：

Aap (Bram Moolenaar)

Ch (Wayne Cheng)
DocBook (Nikolai Weibull)
MetaPost (Eugene Minkovskii)
Objective-C (Kazunobu Kuriyama)
Pyrex (Marco Barisione)
Rst (Nikolai Weibull)
Tcsh (Gautam Iyer)
XFree86 配置文件 (Nikolai Weibull)
Zsh (Nikolai Weibull)

键盘映射表:
cp1253 的希腊语 (Panagiotis Louridas)
匈牙利语 (Magyar) (Laszlo Zavaleta)
波斯-伊朗语 (Behnam Esfahbod)

消息翻译:
加泰隆语 (Ernest Adrogue)
俄语 (Vassily Ragosin)
瑞典语 (Johan Svedberg)

菜单翻译:
加泰隆语 (Ernest Adrogue)
俄语 (Tim Alexeevsky)
瑞典语 (Johan Svedberg)

教程翻译:
加泰隆语 (Ernest Adrogue)
cp1251 的俄语 (Alexey Froloff)
cp1250 和 iso8859-2 的斯洛伐克语 (Lubos Celko)
瑞典语 (Johan Svedberg)
韩语 (Kee-Won Seo)
UTF-8 版本的日语教程 (Yasuhiro Matsumoto) 这是原版，其它日语教程都是由它转换而成。

包含了 "russian.txt" 帮助文件。(Vassily Ragosin)

extra 归档里包含了 Encapsulated PostScript 和 PDF 版本的 Vim 记号 (logo)。

帮助高亮寻找高亮组，是用它们实际的颜色来显示它们自身。(Yakov Lerner 的意见)

big 的 Win32 版本现在编译时加入 Ruby 接口，1.8 版本。Python 使用 2.3 版本。Perl 使用 5.8 版本。

文档里提到了 "ftdetect" 目录。DOS 安装程序会建立它。

修正 fixed-6.3

Test 42 failed on MS-Windows. Set and reset 'fileformat' and 'binary' options here and there. (Walter Briscoe)

The explorer plugin didn't work for double-byte 'encoding's.

Use "copy /y" in Make_bc5.mak to avoid a prompt for overwriting.

Patch 6.2.001
问题: The ":stopinsert" command doesn't have a help tag.
解决方案: Add the tag. (Antoine J. Mechelynck)
相关文件: runtime/doc/insert.txt, runtime/doc/tags

Patch 6.2.002
问题: When compiled with the +multi_byte feature but without +eval, displaying UTF-8 characters may cause a crash. (Karsten Hopp)
解决方案: Also set the default for 'ambiwidth' when compiled without the +eval feature.
相关文件: src/option.c

Patch 6.2.003
问题: GTK 2: double-wide characters below 256 are not displayed correctly.
解决方案: Check the cell width for characters above 127. (Yasuhiro Matsumoto)
相关文件: src/gui_gtk_x11.c

Patch 6.2.004
问题: With a line-Visual selection at the end of the file a "p" command puts the text one line upwards.
解决方案: Detect that the last line was deleted and put forward. (Taro Muraoka)
相关文件: src/normal.c

Patch 6.2.005
问题: GTK: the "Find" and "Find and Replace" tools don't work. (Aschwin Marsman)
解决方案: Show the dialog after creating it. (David Necas)
相关文件: src/gui_gtk.c

Patch 6.2.006
问题: The Netbeans code contains an obsolete function that uses "vim61" and sets the fall-back value for $VIMRUNTIME.
解决方案: Delete the obsolete function.
相关文件: src/main.c, src/netbeans.c, src/proto/netbeans.pro

Patch 6.2.007
问题: Listing tags for Cscope doesn't always work.
解决方案: Avoid using smgs_attr(). (Sergey Khorev)
相关文件: src/if_cscope.c

Patch 6.2.008
问题: XIM with GTK 2: After backspacing preedit characters are wrong.
解决方案: Reset the cursor position. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 6.2.009
问题: Win32: The self-installing executable "Full" selection only selects some of the items to install. (Salman Mohsin)
解决方案: Change commas to spaces in between section numbers.

相关文件: nsis/gvim.nsi

Patch 6.2.010
问题: When 'virtualedit' is effective and a line starts with a
multi-byte character, moving the cursor right doesn't work.
解决方案: Obtain the right character to compute the column offset. (Taro
Muraoka)
相关文件: src/charset.c

Patch 6.2.011
问题: Alpha OSF1: stat() is a macro and doesn't allow an #ifdef halfway.
(Moshe Kaminsky)
解决方案: Move the #ifdef outside of stat().
相关文件: src/os_unix.c

Patch 6.2.012
问题: May hang when polling for a character.
解决方案: Break the wait loop when not waiting for a character.
相关文件: src/os_unix.c

Patch 6.2.013 (extra)
问题: Win32: The registry key for uninstalling GvimExt still uses "6.1".
解决方案: Change the version number to "6.2". (Ajit Thakkar)
相关文件: src/GvimExt/GvimExt.reg

Patch 6.2.014 (after 6.2.012)
问题: XSMP doesn't work when using poll().
解决方案: Use xsmp_idx instead of gpm_idx. (Neil Bird)
相关文件: src/os_unix.c

Patch 6.2.015
问题: The +xsmp feature is never enabled.
解决方案: Move the #define for USE_XSMP to below where WANT_X11 is defined.
(Alexey Froloff)
相关文件: src/feature.h

Patch 6.2.016
问题: Using ":scscope find" with 'cscopequickfix' does not always split
the window. (Gary Johnson)
Win32: ":cscope add" could make the script that contains it
read-only until the corresponding ":cscope kill".
Errors during ":cscope add" may not be handled properly.
解决方案: When using the quickfix window may need to split the window.
Avoid file handle inheritance for the script.
Check for a failed connection and/or process. (Sergey Khorev)
相关文件: src/ex_cmds2.c, src/if_cscope.c

Patch 6.2.017
问题: Test11 sometimes prompts the user, because a file would have been
changed outside of Vim. (Antonio Colombo)
解决方案: Add a FileChangedShell autocommand to avoid the prompt.
相关文件: src/testdir/test11.in

Patch 6.2.018

问题: When using the XSMP protocol and reading from stdin Vim may wait
            for a key to be pressed.
解决方案: Avoid that RealWaitForChar() is used recursively.
相关文件: src/os_unix.c

Patch 6.2.019 (lang)
问题: Loading the Portuguese menu causes an error message.
解决方案: Join two lines. (Jose Pedro Oliveira, José de Paula)
相关文件: runtime/lang/menu_pt_br.vim

Patch 6.2.020
问题: The "Syntax/Set syntax only" menu item causes an error message.
            (Oyvind Holm)
解决方案: Set the script-local variable in a function. (Benji Fisher)
相关文件: runtime/synmenu.vim

Patch 6.2.021
问题: The user manual section on exceptions contains small mistakes.
解决方案: Give a good example of an error that could be missed and other
            improvements. (Servatius Brandt)
相关文件: runtime/doc/usr_41.txt

Patch 6.2.022 (extra)
问题: Win32: After deleting a menu item it still appears in a tear-off
            window.
解决方案: Set the mode to zero for the deleted item. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 6.2.023 (extra)
问题: Win32: Make_ivc.mak does not clean everything.
解决方案: Delete more files in the clean rule. (Walter Briscoe)
相关文件: src/Make_ivc.mak

Patch 6.2.024 (extra)
问题: Win32: Compiler warnings for typecasts.
解决方案: Use DWORD instead of WORD. (Walter Briscoe)
相关文件: src/gui_w32.c

Patch 6.2.025
问题: Missing prototype for sigaltstack().
解决方案: Add the prototype when it is not found in a header file.
相关文件: src/os_unix.c

Patch 6.2.026
问题: Warning for utimes() argument.
解决方案: Add a typecast.
相关文件: src/fileio.c

Patch 6.2.027
问题: Warning for uninitialized variable.
解决方案: Set mb_l to one when not using multi-byte characters.
相关文件: src/message.c

Patch 6.2.028

问题:　　Cscope connection may kill Vim process and others.
解决方案:　Check for pid being larger than one. (Khorev Sergey)
相关文件:　src/if_cscope.c

Patch 6.2.029
问题:　　When using the remote server functionality Vim may leak memory.
(Srikanth Sankaran)
解决方案:　Free the result of XListProperties().
相关文件:　src/if_xcmdsrv.c

Patch 6.2.030
问题:　　Mac: Warning for not being able to use precompiled header files.
解决方案:　Don't redefine select. Use -no-cpp-precomp for compiling, so that function prototypes are still found.
相关文件:　src/os_unix.c, src/osdef.sh

Patch 6.2.031
问题:　　The langmenu entry in the options window doesn't work. (Rodolfo Lima)
With GTK 1 the ":options" command causes an error message. (Michael Naumann)
解决方案:　Change "lmenu" to "langmenu". Only display the 'tbis' option for GTK 2.
相关文件:　runtime/optwin.vim

Patch 6.2.032
问题:　　The lpc filetype is never recognized. (Shizhu Pan)
解决方案:　Check for g:lpc_syntax_for_c instead of the local variable lpc_syntax_for_c. (Benji Fisher)
相关文件:　runtime/filetype.vim

Patch 6.2.033 (extra)
问题:　　Mac: Various compiler warnings.
解决方案:　Don't include Classic-only headers in Unix version.
Remove references to several unused variables. (Ben Fowler)
Fix double definition of DEFAULT_TERM.
Use int instead of unsigned short for pixel values, so that the negative error values are recognized.
相关文件:　src/gui_mac.c, src/term.c

Patch 6.2.034
问题:　　Mac: Compiler warning for redefining DEFAULT_TERM.
解决方案:　Fix double definition of DEFAULT_TERM.
相关文件:　src/term.c

Patch 6.2.035
问题:　　Mac: Compiler warnings in Python interface.
解决方案:　Make a difference between pure Mac and Unix-Mac. (Peter Cucka)
相关文件:　src/if_python.c

Patch 6.2.036 (extra)
问题:　　Mac Unix version: If foo is a directory, then ":e f<Tab>" should expand to ":e foo/" instead of ":e foo" . (Vadim Zeitlin)
解决方案:　Define DONT_ADD_PATHSEP_TO_DIR only for pure Mac. (Benji Fisher)

相关文件: src/os_mac.h

Patch 6.2.037
问题: Win32: converting an encoding name to a codepage could result in
an arbitrary number.
解决方案: make encname2codepage() return zero if the encoding name doesn't
contain a codepage number.
相关文件: src/mbyte.c

Patch 6.2.038 (extra)
问题: Warning messages when using the MingW compiler. (Bill McCarthy)
Can't compile console version without +mouse feature.
解决方案: Initialize variables, add parenthesis.
Add an #ifdef around g_nMouseClick. (Ajit Thakkar)
相关文件: src/eval.c, src/os_win32.c, src/gui_w32.c, src/dosinst.c

Patch 6.2.039 (extra)
问题: More warning messages when using the MingW compiler.
解决方案: Initialize variables. (Bill McCarthy)
相关文件: src/os_mswin.c

Patch 6.2.040
问题: FreeBSD: Crash while starting up when compiled with +xsmp feature.
解决方案: Pass a non-NULL argument to IceAddConnectionWatch().
相关文件: src/os_unix.c

Patch 6.2.041 (extra, after 6.2.033)
问题: Mac: Compiler warnings for conversion types, missing prototype,
missing return type.
解决方案: Change sscanf "%hd" to "%d", the argument is an int now. Add
gui_mch_init_check() prototype. Add "int" to termlib functions.
相关文件: src/gui_mac.c, src/proto/gui_mac.pro, src/termlib.c.

Patch 6.2.042 (extra)
问题: Cygwin: gcc 3.2 has an optimizer problem, sometimes causing a
crash.
解决方案: Add -fno-strength-reduce to the compiler arguments. (Dan Sharp)
相关文件: src/Make_cyg.mak

Patch 6.2.043
问题: Compiling with both netbeans and workshop doesn't work.
解决方案: Move the shellRectangle() function to gui_x11.c. (Gordon Prieur)
相关文件: src/gui_x11.c, src/integration.c, src/netbeans.c,
src/proto/netbeans.pro

Patch 6.2.044
问题: ":au filetypedetect" gives an error for a non-existing event name,
but it's actually a non-existing group name. (Antoine Mechelynck)
解决方案: Make the error message clearer.
相关文件: src/fileio.c

Patch 6.2.045
问题: Obtaining the '(mark changes the '' mark. (Gary Holloway)
解决方案: Don't set the '' mark when searching for the start/end of the

current sentence/paragraph.
相关文件: src/mark.c

Patch 6.2.046
问题: When evaluating an argument of a function throws an exception the function is still called. (Hari Krishna Dara)
解决方案: Don't call the function when an exception was thrown.
相关文件: src/eval.c

Patch 6.2.047 (extra)
问题: Compiler warnings when using MingW. (Bill McCarthy)
解决方案: Give the s_dwLastClickTime variable a type. Initialize dwEndTime.
相关文件: src/os_win32.c

Patch 6.2.048
问题: The Python interface doesn't compile with Python 2.3 when dynamically loaded.
解决方案: Use dll_PyObject_Malloc and dll_PyObject_Free. (Paul Moore)
相关文件: src/if_python.c

Patch 6.2.049
问题: Using a "-range=" argument with ":command" doesn't work and doesn't generate an error message.
解决方案: Generate an error message.
相关文件: src/ex_docmd.c

Patch 6.2.050
问题: Test 32 didn't work on MS-Windows.
解决方案: Write the temp file in Unix fileformat. (Walter Briscoe)
相关文件: src/testdir/test32.in

Patch 6.2.051
问题: When using "\=submatch(0)" in a ":s" command, line breaks become NUL characters.
解决方案: Change NL to CR characters, so that they become line breaks.
相关文件: src/regexp.c

Patch 6.2.052
问题: A few messages are not translated.
解决方案: Add _() to the messages. (Muraoka Taro)
相关文件: src/ex_cmds.c

Patch 6.2.053
问题: Prototype for bzero() doesn't match most systems.
解决方案: Use "void *" instead of "char *" and "size_t" instead of "int".
相关文件: src/osdef1.h.in

Patch 6.2.054
问题: A double-byte character with a second byte that is a backslash causes problems inside a string.
解决方案: Skip over multi-byte characters in a string properly. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 6.2.055
问题: Using col('.') from CTRL-O in Insert mode does not return the correct value for multi-byte characters.
解决方案: Correct the cursor position when it is necessary, move to the first byte of a multi-byte character. (Yasuhiro Matsumoto)
相关文件: src/edit.c

Patch 6.2.056 (extra)
问题: Building with Sniff++ doesn't work.
解决方案: Use the multi-threaded libc when needed. (Holger Ditting)
相关文件: src/Make_mvc.mak

Patch 6.2.057 (extra)
问题: Mac: With -DMACOS_X putenv() is defined twice, it is in a system library. Get a warning for redefining OK. Unused variables in os_mac.c
解决方案: Define HAVE_PUTENV. Undefine OK after including curses.h. Remove declarations for unused variables.
相关文件: src/os_mac.c, src/os_mac.h, src/vim.h

Patch 6.2.058
问题: When 'autochdir' is set ":bnext" to a buffer without a name causes a crash.
解决方案: Don't call vim_chdirfile() when the file name is NULL. (Taro Muraoka)
相关文件: src/buffer.c

Patch 6.2.059
问题: When 'scrolloff' is a large number and listing completion results on the command line, then executing a command that jumps close to where the cursor was before, part of the screen is not updated. (Yakov Lerner)
解决方案: Don't skip redrawing part of the window when it was scrolled.
相关文件: src/screen.c

Patch 6.2.060 (extra)
问题: Win32: When 'encoding' is set to "iso-8859-7" copy/paste to/from the clipboard gives a lalloc(0) error. (Kriton Kyrimis)
解决方案: When the string length is zero allocate one byte. Also fix that when the length of the Unicode text is zero (conversion from 'encoding' to UCS-2 was not possible) the normal text is used.
相关文件: src/os_mswin.c

Patch 6.2.061
问题: GUI: Using the left mouse button with the shift key should work like "*" but it scrolls instead. (Martin Beller)
解决方案: Don't recognize an rxvt scroll wheel event when using the GUI.
相关文件: src/term.c

Patch 6.2.062
问题: When one buffer uses a syntax with "containedin" and another buffer does not, redrawing depends on what the current buffer is. (Brett Pershing Stahlman)
解决方案: Use "syn_buf" instead of "curbuf" to get the b_syn_containedin

flag.
相关文件: src/syntax.c

Patch 6.2.063
问题: When using custom completion end up with no matches.
解决方案: Make cmd_numfiles and cmd_files local to completion to avoid that they are overwritten when ExpandOne() is called recursively by f_glob().
相关文件: src/eval.c, src/ex_docmd.c, src/ex_getln.c, src/proto/ex_getln.pro, src/misc1.c, src/structs.h, src/tag.c

Patch 6.2.064
问题: resolve() only handles one symbolic link, need to repeat it to resolve all of them. Then need to simplify the file name.
解决方案: Make resolve() resolve all symbolic links and simplify the result. Add simplify() to just simplify a file name. Fix that test49 doesn't work if /tmp is a symbolic link. (Servatius Brandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/tag.c, src/testdir/test49.vim

Patch 6.2.065
问题: ":windo 123" only updates other windows when entering them. (Walter Briscoe)
解决方案: Update the topline before going to the next window.
相关文件: src/ex_cmds2.c

Patch 6.2.066 (extra)
问题: Ruby interface doesn't work with Ruby 1.8.0.
解决方案: Change "defout" to "stdout". (Aron Griffis)
Change dynamic loading. (Taro Muraoka)
相关文件: src/if_ruby.c, src/Make_mvc.mak

Patch 6.2.067
问题: When searching for a string that starts with a composing character the command line isn't drawn properly.
解决方案: Don't count the space to draw the composing character on and adjust the cursor column after drawing the string.
相关文件: src/message.c

Patch 6.2.068
问题: Events for the netbeans interface that include a file name with special characters don't work properly.
解决方案: Use nb_quote() on the file name. (Sergey Khorev)
相关文件: src/netbeans.c

Patch 6.2.069 (after 6.2.064)
问题: Unused variables "limit" and "new_st" and unused label "fail" in some situation. (Bill McCarthy)
解决方案: Put the declarations inside an #ifdef. (Servatius Brandt)
相关文件: src/eval.c, src/tag.c

Patch 6.2.070 (after 6.2.069)
问题: Still unused variable "new_st". (Bill McCarthy)
解决方案: Move the declaration to the right block this time.

相关文件: src/tag.c

Patch 6.2.071
问题: 'statusline' can only contain 50 % items. (Antony Scriven)
解决方案: Allow 80 items and mention it in the docs.
相关文件: runtime/doc/option.txt, src/vim.h

Patch 6.2.072
问题: When using expression folding, foldexpr() mostly returns -1 for the previous line, which makes it difficult to write a fold expression.
解决方案: Make the level of the previous line available while still looking for the end of a fold.
相关文件: src/fold.c

Patch 6.2.073
问题: When adding detection of a specific filetype for a plugin you need to edit "filetype.vim".
解决方案: Source files from the "ftdetect" directory, so that a filetype detection plugin only needs to be dropped in a directory.
相关文件: runtime/doc/filetype.txt, runtime/doc/usr_05.txt, runtime/doc/usr_41.txt, runtime/filetype.vim

Patch 6.2.074
问题: Warnings when compiling the Python interface. (Ajit Thakkar)
解决方案: Use ANSI function declarations.
相关文件: src/if_python.c

Patch 6.2.075
问题: When the temp file for writing viminfo can't be used "NULL" appears in the error message. (Ben Lavender)
解决方案: Print the original file name when there is no temp file name.
相关文件: src/ex_cmds.c

Patch 6.2.076
问题: The tags listed for cscope are in the wrong order. (Johannes Stezenbach)
解决方案: Remove the reordering of tags for the current file. (Sergey Khorev)
相关文件: src/if_cscope.c

Patch 6.2.077
问题: When a user function specifies custom completion, the function gets a zero argument instead of an empty string when there is no word before the cursor. (Preben Guldberg)
解决方案: Don't convert an empty string to a zero.
相关文件: src/eval.c

Patch 6.2.078
问题: "make test" doesn't work if Vim wasn't compiled yet. (Ed Avis)
解决方案: Build Vim before running the tests.
相关文件: src/Makefile

Patch 6.2.079

问题:　　":w ++enc=utf-8 !cmd" doesn't work.
解决方案:　Check for the "++" argument before the "!".
相关文件:　src/ex_docmd.c

Patch 6.2.080
问题:　　When 't_ti' is not empty but doesn't swap screens, using "ZZ" in an unmodified file doesn't clear the last line.
解决方案:　Call msg_clr_eos() when needed. (Michael Schroeder)
相关文件:　src/os_unix.c

Patch 6.2.081
问题:　　Problem when using a long multibyte string for the statusline.
解决方案:　Use the right pointer to get the cell size. (Taro Muraoka)
相关文件:　src/buffer.c

Patch 6.2.082
问题:　　Can't compile with Perl 5.8.1.
解决方案:　Rename "e_number" to "e_number_exp". (Sascha Blank)
相关文件:　src/digraph.c, src/globals.h

Patch 6.2.083
问题:　　When a compiler uses ^^^^ to mark a word the information is not visible in the quickfix window. (Srikanth Sankaran)
解决方案:　Don't remove the indent for a line that is not recognized as an error message.
相关文件:　src/quickfix.c

Patch 6.2.084
问题:　　"g_" in Visual mode always goes to the character after the line. (Jean-Rene David)
解决方案:　Ignore the NUL at the end of the line.
相关文件:　src/normal.c

Patch 6.2.085
问题:　　":verbose set ts" doesn't say an option was set with a "-c" or "--cmd" argument.
解决方案:　Remember the option was set from a Vim argument.
相关文件:　src/main.c, src/ex_cmds2.c, src/vim.h

Patch 6.2.086
问题:　　"{" and "}" stop inside a closed fold.
解决方案:　Only stop once inside a closed fold. (Stephen Riehm)
相关文件:　src/search.c

Patch 6.2.087
问题:　　CTRL-^ doesn't use the 'confirm' option. Same problem with ":bnext". (Yakov Lerner)
解决方案:　Put up a dialog for a changed file when 'confirm' is set in more situations.
相关文件:　src/buffer.c, src/ex_cmds.c

Patch 6.2.088
问题:　　When 'sidescrolloff' is set 'showmatch' doesn't work correctly if the match is less than 'sidescrolloff' off from the side of the

window. (Roland Stahn)
解决方案: Set 'sidescrolloff' to zero while displaying the match.
相关文件: src/search.c

Patch 6.2.089
问题: ":set isk+=" adds a comma. (Mark Waggoner)
解决方案: Don't add a comma when the added value is empty.
相关文件: src/option.c

Patch 6.2.090 (extra)
问题: Win32: MingW compiler complains about #pragmas. (Bill McCarthy)
解决方案: Put an #ifdef around the #pragmas.
相关文件: src/os_win32.c

Patch 6.2.091
问题: When an autocommand is triggered when a file is dropped on Vim and it produces output, messages from a following command may be scrolled unexpectedly. (David Rennalls)
解决方案: Save and restore msg_scroll in handle_drop().
相关文件: src/ex_docmd.c

Patch 6.2.092
问题: Invalid items appear in the help file tags. (Antonio Colombo)
解决方案: Only accept tags with white space before the first "*".
相关文件: runtime/doc/doctags.c, src/ex_cmds.c

Patch 6.2.093
问题: ":nnoremenu" also defines menu for Visual mode. (Klaus Bosau)
解决方案: Check the second command character for an "o", not the third.
相关文件: src/menu.c

Patch 6.2.094
问题: Can't compile with GTK and tiny features.
解决方案: Include handle_drop() and vim_chdirfile() when FEAT_DND is defined. Do not try to split the window.
相关文件: src/ex_docmd.c, src/misc2.c

Patch 6.2.095
问题: The message "Cannot go to buffer x" is confusing for ":buf 6". (Frans Englich)
解决方案: Make it "Buffer x does not exist".
相关文件: src/buffer.c

Patch 6.2.096
问题: Win32: ":let @* = ''" put a newline on the clipboard. (Klaus Bosau)
解决方案: Put zero bytes on the clipboard for an empty string.
相关文件: src/ops.c

Patch 6.2.097
问题: Setting or resetting 'insertmode' in a BufEnter autocommand doesn't always have immediate effect. (Nagger)
解决方案: When 'insertmode' is set, set need_start_insertmode, when it's reset set stop_insert_mode.

相关文件: src/option.c

Patch 6.2.098 (after 6.2.097)
问题: Can't build Vim with tiny features. (Christian J. Robinson)
解决方案: Declare stop_insert_mode always.
相关文件: src/edit.c, src/globals.h

Patch 6.2.099 (extra)
问题: Test 49 fails. (Mikolaj Machowski)
解决方案: The Polish translation must not change "E116" to "R116".
相关文件: src/po/pl.po

Patch 6.2.100
问题: "make proto" fails when compiled with the Perl interface.
解决方案: Remove "-fno.*" from PERL_CFLAGS, cproto sees it as its option.
相关文件: src/auto/configure, src/configure.in

Patch 6.2.101
问题: When using syntax folding, opening a file slows down a lot when
it's size increases by only 20%. (Gary Johnson)
解决方案: The array with cached syntax states is leaking entries. After
cleaning up the list obtain the current entry again.
相关文件: src/syntax.c

Patch 6.2.102
问题: The macros equal() and CR conflict with a Carbon header file.
解决方案: Rename equal() to equalpos(). Rename CR to CAR.
Do this in the non-extra files only.
相关文件: src/ascii.h, src/buffer.c, src/charset.c, src/edit.c, src/eval.c,
src/ex_cmds.c, src/ex_cmds2.c, src/ex_getln.c, src/fileio.c,
src/getchar.c, src/gui.c, src/gui_athena.c, src/gui_gtk_x11.c,
src/gui_motif.c, src/macros.h, src/mark.c, src/message.c,
src/misc1.c, src/misc2.c, src/normal.c, src/ops.c, src/os_unix.c,
src/regexp.c, src/search.c, src/ui.c, src/workshop.c

Patch 6.2.103 (extra)
问题: The macros equal() and CR conflict with a Carbon header file.
解决方案: Rename equal() to equalpos(). Rename CR to CAR.
Do this in the extra files only.
相关文件: src/gui_photon.c, src/gui_w48.c

Patch 6.2.104
问题: Unmatched braces in the table with options.
解决方案: Move the "}," outside of the #ifdef. (Yakov Lerner)
相关文件: src/option.c

Patch 6.2.105
问题: When the cursor is past the end of the line when calling
get_c_indent() a crash might occur.
解决方案: Don't look past the end of the line. (NJ Verenini)
相关文件: src/misc1.c

Patch 6.2.106
问题: Tag searching gets stuck on a very long line in the tags file.

解决方案: When skipping back to search the first matching tag remember the
           offset where searching started looking for a line break.
相关文件: src/tag.c

Patch 6.2.107 (extra)
问题: The NetBeans interface cannot be used on Win32.
解决方案: Add support for the NetBeans for Win32. Add support for reading
           XPM files on Win32. Also fixes that a sign icon with a space in
           the file name did not work through the NetBeans interface.
           (Sergey Khorev)
           Also: avoid repeating error messages when the connection is lost.
相关文件: Makefile, runtime/doc/netbeans.txt, src/Make_bc5.mak,
           src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak,
           src/bigvim.bat, src/feature.h, src/gui_beval.c, src/gui_beval.h,
           src/gui_w32.c, src/gui_w48.c, src/menu.c, src/nbdebug.c,
           src/nbdebug.h, src/netbeans.c, src/os_mswin.c, src/os_win32.h,
           src/proto/gui_beval.pro, src/proto/gui_w32.pro,
           src/proto/netbeans.pro, src/proto.h, src/version.c, src/vim.h,
           src/xpm_w32.c, src/xpm_w32.h

Patch 6.2.108
问题: Crash when giving a message about ignoring case in a tag. (Manfred
           Kuehn)
解决方案: Use a longer buffer for the message.
相关文件: src/tag.c

Patch 6.2.109
问题: Compiler warnings with various Amiga compilers.
解决方案: Add typecast, prototypes, et al. that are also useful for other
           systems. (Flavio Stanchina)
相关文件: src/eval.c, src/ops.c

Patch 6.2.110
问题: When $LANG includes the encoding, a menu without an encoding name
           is not found.
解决方案: Also look for a menu file without any encoding.
相关文件: runtime/menu.vim

Patch 6.2.111
问题: Encoding "cp1251" is not recognized.
解决方案: Add "cp1251" to the table of encodings. (Alexey Froloff)
相关文件: src/mbyte.c

Patch 6.2.112
问题: After applying patches test32 fails. (Antonio Colombo)
解决方案: Have "make clean" in the testdir delete *.rej and *.orig files.
           Use this when doing "make clean" in the src directory.
相关文件: src/Makefile, src/testdir/Makefile

Patch 6.2.113
问题: Using ":startinsert" after "$" works like "a" instead of "i".
           (Ajit Thakkar)
解决方案: Reset "w_curswant" for ":startinsert" and reset o_eol in edit().
相关文件: src/edit.c, src/ex_docmd.c

Patch 6.2.114
问题: When stdout is piped through "tee", the size of the screen may not be correct.
解决方案: Use stdin instead of stdout for ioctl() when stdin is a tty and stdout isn't.
相关文件: src/os_unix.c

Patch 6.2.115 (extra)
问题: Compiler warnings with various Amiga compilers.
解决方案: Add typecast, prototypes, et al. Those changes that are Amiga-specific. (Flavio Stanchina)
相关文件: src/fileio.c, src/memfile.c, src/os_amiga.c, src/os_amiga.h, src/vim.h

Patch 6.2.116 (extra)
问题: German keyboard with Numlock set different from system startup causes problems.
解决方案: Ignore keys with code 0xff. (Helmut Stiegler)
相关文件: src/gui_w48.c

Patch 6.2.117
问题: Breakpoints in loops of sourced files and functions are not detected. (Hari Krishna Dara)
解决方案: Check for breakpoints when using lines that were previously read. (Servatius Brandt)
相关文件: src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/proto/eval.pro, src/proto/ex_cmds2.pro

Patch 6.2.118 (extra)
问题: Mac: Compiling is done in a non-standard way.
解决方案: Use the Unix method for Mac OS X, with autoconf. Add "CARBONGUI" to Makefile and configure. (Eric Kow)
Move a few prototypes from os_mac.pro to gui_mac.pro.
相关文件: src/Makefile, src/auto/configure, src/configure.in, src/config.mk.in, src/gui_mac.c, src/os_mac.h, src/os_macosx.c, src/proto/gui_mac.pro, src/proto/os_mac.pro, src/infplist.xml, src/vim.h

Patch 6.2.119 (after 6.2.107)
问题: When packing the MS-Windows archives a few files are missing. (Guopeng Wen)
解决方案: Add gui_beval.* to the list of generic source files.
相关文件: Makefile

Patch 6.2.120
问题: Win32 GUI: The console dialogs are not supported on MS-Windows, disabling the 'c' flag of 'guioptions'. (Servatius Brandt)
解决方案: Define FEAT_CON_DIALOG also for GUI-only builds.
相关文件: src/feature.h

Patch 6.2.121 (after 6.2.118)
问题: Not all make programs support "+=". (Charles Campbell)
解决方案: Use a normal assignment.

相关文件: src/Makefile

Patch 6.2.122 (after 6.2.119)
问题: Not all shells can expand [^~]. File missing. (Guopeng Wen)
解决方案: Use a simpler pattern. Add the Aap recipe for the maze program and a clean version of the source code.
相关文件: Makefile, runtime/macros/maze/Makefile, runtime/macros/maze/README.txt, runtime/macros/maze/main.aap, runtime/macros/maze/mazeclean.c

Patch 6.2.123 (after 6.2.118)
问题: Running configure fails. (Tony Leneis)
解决方案: Change "==" to "=" for a test.
相关文件: src/auto/configure, src/configure.in

Patch 6.2.124 (after 6.2.121)(extra)
问题: Mac: Recursive use of M4FLAGS causes problems. When running Vim directly it can't find the runtime files. (Emily Jackson) Using GNU constructs causes warnings with other make programs. (Ronald Schild)
解决方案: Use another name for the M4FLAGS variable. Don't remove "Vim.app" from the path. Update the explanation for compiling on the Mac. (Eric Kow) Don't use $(shell) and $(addprefix).
相关文件: src/INSTALLmac.txt, src/Makefile, src/misc1.c

Patch 6.2.125 (after 6.2.107)
问题: The "winsock2.h" file isn't always available.
解决方案: Don't include this header file.
相关文件: src/netbeans.c

Patch 6.2.126
问题: Typing CTRL-C at a confirm() prompt doesn't throw an exception.
解决方案: Reset "mapped_ctrl_c" in get_keystroke(), so that "got_int" is set in _OnChar().
相关文件: src/misc1.c

Patch 6.2.127 (extra)
问题: Win32 console: Typing CTRL-C doesn't throw an exception.
解决方案: Set got_int immediately when CTRL-C is typed, don't wait for mch_breakcheck() being called.
相关文件: src/os_win32.c

Patch 6.2.128 (after 6.2.118)
问题: src/auto/configure is not consistent with src/configure.in.
解决方案: Use the newly generated configure script.
相关文件: src/auto/configure

Patch 6.2.129
问题: When 'number' is set 'wrapmargin' does not work Vi-compatible. (Yasuhiro Matsumoto)
解决方案: Reduce the textwidth when 'number' is set. Also for 'foldcolumn' and similar things.
相关文件: src/edit.c

Patch 6.2.130 (extra)
问题: Win32 console: When 'restorescreen' is not set exiting Vim causes the screen to be cleared. (Michael A. Mangino)
解决方案: Don't clear the screen when exiting and 'restorescreen' isn't set.
相关文件: src/os_win32.c

Patch 6.2.131 (extra)
问题: Win32: Font handles are leaked.
解决方案: Free italic, bold and bold-italic handles before overwriting them. (Michael Wookey)
相关文件: src/gui_w48.c

Patch 6.2.132 (extra)
问题: Win32: console version doesn't work on latest Windows Server 2003.
解决方案: Copy 12000 instead of 15000 cells at a time to avoid running out of memory.
相关文件: src/os_win32.c

Patch 6.2.133
问题: When starting the GUI a bogus error message about 'imactivatekey' may be given.
解决方案: Only check the value of 'imactivatekey' when the GUI is running.
相关文件: src/gui.c, src/option.c

Patch 6.2.134 (extra)
问题: Win32: When scrolling parts of the window are redrawn when this isn't necessary.
解决方案: Only invalidate parts of the window when they are obscured by other windows. (Michael Wookey)
相关文件: src/gui_w48.c

Patch 6.2.135
问题: An item <> in the ":command" argument is interpreted as <args>.
解决方案: Avoid that <> is recognized as <args>.
相关文件: src/ex_docmd.c

Patch 6.2.136
问题: ":e ++enc=latin1 newfile" doesn't set 'fenc' when the file doesn't exist. (Miroslaw Dobrzanski-Neumann)
解决方案: Set 'fileencoding' to the specified encoding when editing a file that does not exist.
相关文件: src/fileio.c

Patch 6.2.137
问题: "d:cmd<CR>" cannot be repeated with ".". Breaks repeating "d%" when using the matchit plugin.
解决方案: Store the command to be repeated. This is restricted to single-line commands.
相关文件: src/ex_docmd.c, src/globals.h, src/normal.c, src/vim.h

Patch 6.2.138 (extra)
问题: Compilation problem on VMS with dynamic buffer on the stack.
解决方案: Read one byte less than the size of the buffer, so that we can

check for the string length without an extra buffer.
相关文件: src/os_vms.c

Patch 6.2.139
问题: Code is repeated in the two Perl files.
解决方案: Move common code from if_perl.xs and if_perlsfio.c to vim.h.
Also fix a problem with generating prototypes.
相关文件: src/if_perl.xs, src/if_perlsfio.c, src/vim.h

Patch 6.2.140 (after 6.2.121)
问题: Mac: Compiling with Python and Perl doesn't work.
解决方案: Adjust the configure check for Python to use "-framework Python"
for Python 2.3 on Mac OS/X.
Move "-ldl" after "DynaLoader.a" in the link command.
Change "perllibs" to "PERL_LIBS".
相关文件: src/auto/configure, src/configure.in, src/config.mk.in

Patch 6.2.141 (extra)
问题: Mac: The b_FSSpec field is sometimes unused.
解决方案: Change the #ifdef to FEAT_CW_EDITOR and defined it in feature.h
相关文件: src/fileio.c, src/gui_mac.c, src/structs.h, src/feature.h

Patch 6.2.142 (after 6.2.124)
问题: Mac: building without GUI through configure doesn't work.
When the system is slow, unpacking the resource file takes too
long.
解决方案: Don't always define FEAT_GUI_MAC when MACOS is defined, define it
in the Makefile.
Add a configure option to skip Darwin detection.
Use a Python script to unpack the resources to avoid a race
condition. (Taro Muraoka)
相关文件: Makefile, src/Makefile, src/auto/configure, src/configure.in,
src/dehqx.py, src/vim.h

Patch 6.2.143
问题: Using "K" on Visually selected text doesn't work if it ends in
a multi-byte character.
解决方案: Include all the bytes of the last character. (Taro Muraoka)
相关文件: src/normal.c

Patch 6.2.144
问题: When "g:html_use_css" is set the HTML header generated by the
2html script is wrong.
解决方案: Add the header after adding HREF for links.
Also use ":normal!" instead of ":normal" to avoid mappings
getting in the way.
相关文件: runtime/syntax/2html.vim

Patch 6.2.145 (after 6.2.139)
问题: Undefining "bool" doesn't work for older systems. (Wojtek Pilorz)
解决方案: Only undefine "bool" on Mac OS.
相关文件: src/vim.h

Patch 6.2.146

问题:　　On some systems the prototype for iconv() is wrong, causing a warning message.
解决方案:　Use a cast (void *) to avoid the warning. (Charles Campbell)
相关文件:　src/fileio.c, src/mbyte.c

Patch 6.2.147
问题:　　":s/pat/\=col('.')" always replaces with "1".
解决方案:　Set the cursor to the start of the match before substituting. (Helmut Stiegler)
相关文件:　src/ex_cmds.c

Patch 6.2.148
问题:　　Can't break an Insert into several undoable parts.
解决方案:　Add the CTRL-G u command.
相关文件:　runtime/doc/insert.txt, src/edit.c

Patch 6.2.149
问题:　　When the cursor is on a line past 21,474,748 the indicated percentage of the position is invalid. With that many lines "100%" causes a negative cursor line number, resulting in a crash. (Daniel Goujot)
解决方案:　Divide by 100 instead of multiplying. Avoid overflow when computing the line number for "100%".
相关文件:　src/buffer.c, src/ex_cmds2.c, src/normal.c

Patch 6.2.150
问题:　　When doing "vim - < file" lines are broken at NUL chars. (Daniel Goujot)
解决方案:　Change NL characters back to NUL when reading from the temp buffer.
相关文件:　src/fileio.c

Patch 6.2.151
问题:　　When doing "vim --remote +startinsert file" some commands are inserted as text. (Klaus Bosau)
解决方案:　Put all the init commands in one Ex line, not using a <CR>, so that Insert mode isn't started too early.
相关文件:　src/main.c

Patch 6.2.152
问题:　　The cursor() function doesn't reset the column offset for 'virtualedit'.
解决方案:　Reset the offset to zero. (Helmut Stiegler)
相关文件:　src/eval.c

Patch 6.2.153
问题:　　Win32: ":lang german" doesn't use German messages.
解决方案:　Add a table to translate the Win32 language names to two-letter language codes.
相关文件:　src/ex_cmds2.c

Patch 6.2.154
问题:　　Python bails out when giving a warning message. (Eugene Minkovskii)

解决方案: Set sys.argv[] to an empty string.
相关文件: src/if_python.c

Patch 6.2.155
问题: Win32: Using ":tjump www" in a help file gives two results.
(Dave Roberts)
解决方案: Ignore differences between slashes and backslashes when checking
for identical tag matches.
相关文件: src/tag.c

Patch 6.2.156 (after 6.2.125)
问题: Win32: Netbeans fails to build, EINTR is not defined.
解决方案: Redefine EINTR to WSAEINTR. (Mike Williams)
相关文件: src/netbeans.c

Patch 6.2.157
问题: Using "%p" in 'errorformat' gives a column number that is too
high.
解决方案: Set the flag to use the number as a virtual column. (Lefteris
Koutsoloukas)
相关文件: src/quickfix.c

Patch 6.2.158
问题: The sed command on Solaris and HPUX doesn't work for a line that
doesn't end in a newline.
解决方案: Add a newline when feeding text to sed. (Mark Waggoner)
相关文件: src/configure.in, src/auto/configure

Patch 6.2.159
问题: When using expression folding and 'foldopen' is "undo" an undo
command doesn't always open the fold.
解决方案: Save and restore the KeyTyped variable when evaluating 'foldexpr'.
(Taro Muraoka)
相关文件: src/fold.c

Patch 6.2.160
问题: When 'virtualedit' is "all" and 'selection' is "exclusive",
selecting a double-width character below a single-width character
may cause a crash.
解决方案: Avoid overflow on unsigned integer decrement. (Taro Muraoka)
相关文件: src/normal.c

Patch 6.2.161 (extra)
问题: VMS: Missing header file. Reading input busy loops.
解决方案: Include termdef.h. Avoid the use of a wait function in
vms_read(). (Frank Ries)
相关文件: src/os_unix.h, src/os_vms.c

Patch 6.2.162
问题: ":redraw" doesn't always display the text that includes the cursor
position, e.g. after ":call cursor(1, 0)". (Eugene Minkovskii)
解决方案: Call update_topline() before redrawing.
相关文件: src/ex_docmd.c

Patch 6.2.163
问题:　　"make install" may also copy AAPDIR directories.
解决方案:　Delete AAPDIR directories, just like CVS directories.
相关文件:　src/Makefile

Patch 6.2.164 (after 6.2.144)
问题:　　When "g:html_use_css" is set the HTML header generated by the
　　2html script is still wrong.
解决方案:　Search for a string instead of jumping to a fixed line number.
　　Go to the start of the line before inserting the header.
　　(Jess Thrysoee)
相关文件:　runtime/syntax/2html.vim

Patch 6.2.165
问题:　　The configure checks hang when using autoconf 2.57.
解决方案:　Invoke AC_PROGRAM_EGREP to set $EGREP. (Aron Griffis)
相关文件:　src/auto/configure, src/configure.in

Patch 6.2.166
问题:　　When $GZIP contains "-N" editing compressed files doesn't work
　　properly.
解决方案:　Add "-n" to "gzip -d" to avoid restoring the file name. (Oyvind
　　Holm)
相关文件:　runtime/plugin/gzip.vim

Patch 6.2.167
问题:　　The Python interface leaks memory when assigning lines to a
　　buffer. (Sergey Khorev)
解决方案:　Do not copy the line when calling ml_replace().
相关文件:　src/if_python.c

Patch 6.2.168
问题:　　Python interface: There is no way to get the indices from a range
　　object.
解决方案:　Add the "start" and "end" attributes. (Maurice S. Barnum)
相关文件:　src/if_python.c, runtime/doc/if_pyth.txt

Patch 6.2.169
问题:　　The prototype for _Xmblen() appears in a recent XFree86 header
　　file, causing a warning for our prototype. (Hisashi T Fujinaka)
解决方案:　Move the prototype to an osdef file, so that it's filtered out.
相关文件:　src/mbyte.c, src/osdef2.h.in

Patch 6.2.170
问题:　　When using Sun WorkShop the current directory isn't changed to
　　where the file is.
解决方案:　Set the 'autochdir' option when using WorkShop. And avoid using
　　the basename when 'autochdir' is not set.
相关文件:　src/gui_x11.c, src/ex_cmds.c

Patch 6.2.171 (after 6.2.163)
问题:　　The "-or" argument of "find" doesn't work for SysV systems.
解决方案:　Use "-o" instead. (Gordon Prieur)
相关文件:　src/Makefile

Patch 6.2.172 (after 6.2.169)
问题: The prototype for _Xmblen() still causes trouble.
解决方案: Include the X11 header file that defines the prototype.
相关文件: src/osdef2.h.in, src/osdef.sh

Patch 6.2.173 (extra)
问题: Win32: Ruby interface doesn't work with Ruby 1.8.0 for other compilers than MSVC.
解决方案: Fix the BC5, Cygwin and Mingw makefiles. (Dan Sharp)
相关文件: src/Make_bc5.mak, src/Make_cyg.mak, src/Make_ming.mak

Patch 6.2.174
问题: After the ":intro" message only a mouse click in the last line gets past the hit-return prompt.
解决方案: Accept a click at or below the hit-return prompt.
相关文件: src/gui.c, src/message.c

Patch 6.2.175
问题: Changing 'backupext' in a *WritePre autocommand doesn't work. (William Natter)
解决方案: Move the use of p_bex to after executing the *WritePre autocommands. Also avoids reading allocated memory after freeing.
相关文件: src/fileio.c

Patch 6.2.176
问题: Accented characters in translated help files are not handled correctly. (Fabien Vayssiere)
解决方案: Include "192-255" in 'iskeyword' for the help window.
相关文件: src/ex_cmds.c

Patch 6.2.177 (extra)
问题: VisVim: Opening a file with a space in the name doesn't work. (Rob Retter) Arbitrary commands are being executed. (Neil Bird)
解决方案: Put a backslash in front of every space in the file name. (Gerard Blais) Terminate the CTRL-\ CTRL-N command with a NUL.
相关文件: src/VisVim/Commands.cpp, src/VisVim/VisVim.rc

Patch 6.2.178
问题: People who don't know how to exit Vim try pressing CTRL-C.
解决方案: Give a message how to exit Vim when CTRL-C is pressed and it doesn't cancel anything.
相关文件: src/normal.c

Patch 6.2.179 (extra)
问题: The en_gb messages file isn't found on case sensitive systems.
解决方案: Rename en_gb to en_GB. (Mike Williams)
相关文件: src/po/en_gb.po, src/po/en_GB.po, src/po/Make_ming.mak, src/po/Make_mvc.mak, src/po/Makefile, src/po/README_mvc.txt

Patch 6.2.180
问题: Compiling with GTK2 on Win32 doesn't work.
解决方案: Include gdkwin32.h instead of gdkx.h. (Srinath Avadhanula)
相关文件: src/gui_gtk.c, src/gui_gtk_f.c, src/gui_gtk_x11.c, src/mbyte.c

Patch 6.2.181 (after 6.2.171)
问题: The "-o" argument of "find" has lower priority than the implied "and" with "-print".
解决方案: Add parenthesis around the "-o" expression. (Gordon Prieur)
相关文件: src/Makefile

Patch 6.2.182 (after 6.2.094)
问题: Compilation with tiny features fails because of missing get_past_head() function.
解决方案: Adjust the #ifdef for get_past_head().
相关文件: src/misc1.c

Patch 6.2.183 (after 6.2.178)
问题: Warning for char/unsigned char mixup.
解决方案: Use MSG() instead of msg(). (Tony Leneis)
相关文件: src/normal.c

Patch 6.2.184
问题: With 'formatoptions' set to "1aw" inserting text may cause the paragraph to be ended. (Alan Schmitt)
解决方案: Temporarily add an extra space to make the paragraph continue after moving the word after the cursor to the next line. Also format when pressing Esc.
相关文件: src/edit.c, src/normal.c, src/proto/edit.pro

Patch 6.2.185
问题: Restoring a session with zero-height windows does not work properly. (Charles Campbell)
解决方案: Accept a zero argument to ":resize" as intended. Add a window number argument to ":resize" to be able to set the size of other windows, because the current window cannot be zero-height. Fix the explorer plugin to avoid changing the window sizes. Add the winrestcmd() function for this.
相关文件: runtime/doc/eval.txt, runtime/plugin/explorer.vim, src/eval.c, src/ex_cmds.h, src/ex_docmd.c, src/proto/window.pro, src/window.c

Patch 6.2.186 (after 6.2.185)
问题: Documentation file eval.txt contains examples without indent.
解决方案: Insert the indent. Also fix other mistakes.
相关文件: runtime/doc/eval.txt

Patch 6.2.187
问题: Using Insure++ reveals a number of bugs. (Dominique Pelle)
解决方案: Initialize variables where needed. Free allocated memory to avoid leaks. Fix comparing tags to avoid reading past allocated memory.
相关文件: src/buffer.c, src/diff.c, src/fileio.c, src/mark.c, src/misc1.c, src/misc2.c, src/ops.c, src/option.c, src/tag.c, src/ui.c

Patch 6.2.188 (extra)
问题: MS-Windows: Multi-byte characters in a filename cause trouble for the window title.
解决方案: Return when the wide function for setting the title did its work.
相关文件: src/gui_w48.c

Patch 6.2.189
问题: When setting 'viminfo' after editing a new buffer its marks are not stored. (Keith Roberts)
解决方案: Set the "b_marks_read" flag when skipping to read marks from the viminfo file.
相关文件: src/fileio.c

Patch 6.2.190
问题: When editing a compressed files, marks are lost.
解决方案: Add the ":lockmarks" modifier and use it in the gzip plugin. Make exists() also check for command modifiers, so that the existence of ":lockmarks" can be checked for.
Also add ":keepmarks" to avoid that marks are deleted when filtering text.
When deleting lines put marks 'A - 'Z and '0 - '9 at the first deleted line instead of clearing the mark. They were kept in the viminfo file anyway.
Avoid that the gzip plugin puts deleted text in registers.
相关文件: runtime/doc/motion.txt, runtime/plugin/gzip.vim, src/ex_cmds.c, src/ex_docmd.c, src/mark.c, src/structs.h

Patch 6.2.191
问题: The intro message is outdated. Information about sponsoring and registering is missing.
解决方案: Show info about sponsoring and registering Vim in the intro message now and then. Add help file about sponsoring.
相关文件: runtime/doc/help.txt, runtime/doc/sponsor.txt, runtime/doc/tags, runtime/menu.vim, src/version.c

Patch 6.2.192
问题: Using CTRL-T and CTRL-D with "gR" messes up the text. (Jonathan Hankins)
解决方案: Avoid calling change_indent() recursively.
相关文件: src/edit.c

Patch 6.2.193
问题: When recalling a search pattern from the history from a ":s,a/c," command the '/' ends the search string. (JC van Winkel)
解决方案: Store the separator character with the history entries. Escape characters when needed, replace the old separator with the new one.
Also fixes that recalling a "/" search for a "?" command messes up trailing flags.
相关文件: src/eval.c, src/ex_getln.c, src/normal.c, src/proto/ex_getln.pro, src/search.c, src/tag.c

Patch 6.2.194 (after 6.2.068)
问题: For NetBeans, instead of writing the file and sending an event about it, tell NetBeans to write the file.
解决方案: Add the "save" command, "netbeansBuffer" command and "buttonRelease" event to the netbeans protocol. Updated the interface to version 2.2. (Gordon Prieur)
Also: open a fold when the cursor has been positioned.
Also: fix memory leak, free result of nb_quote().

相关文件: runtime/doc/netbeans.txt, src/fileio.c, src/netbeans.c,
           src/normal.c, src/proto/netbeans.pro, src/structs.h

Patch 6.2.195 (after 6.2.190)
问题: Compiling fails for missing CPO_REMMARK symbol.
解决方案: Add the patch I forgot to include...
相关文件: src/option.h

Patch 6.2.196 (after 6.2.191)
问题: Rebuilding the documentation doesn't use the sponsor.txt file.
解决方案: Add sponsor.txt to the Makefile. (Christian J. Robinson)
相关文件: runtime/doc/Makefile

Patch 6.2.197
问题: It is not possible to force a redraw of status lines. (Gary
           Johnson)
解决方案: Add the ":redrawstatus" command.
相关文件: runtime/doc/various.txt, src/ex_cmds.h, src/ex_docmd.c,
           src/screen.c

Patch 6.2.198
问题: A few messages are not translated. (Ernest Adrogue)
解决方案: Mark the messages to be translated.
相关文件: src/ex_cmds.c

Patch 6.2.199 (after 6.2.194)
问题: Vim doesn't work perfectly well with NetBeans.
解决方案: When NetBeans saves the file, reset the timestamp to avoid "file
           changed" warnings. Close a buffer in a proper way. Don't try
           giving a debug message with an invalid pointer. Send a
           newDotAndMark message when needed. Report a change by the "r"
           command to NetBeans. (Gordon Prieur)
相关文件: src/netbeans.c, src/normal.c

Patch 6.2.200
问题: When recovering a file, 'fileformat' is always the default, thus
           writing the file may result in differences. (Penelope Fudd)
解决方案: Before recovering the file try reading the original file to obtain
           the values of 'fileformat', 'fileencoding', etc.
相关文件: src/memline.c

Patch 6.2.201
问题: When 'autowriteall' is set ":qall" still refuses to exit if there
           is a modified buffer. (Antoine Mechelynck)
解决方案: Attempt writing modified buffers as intended.
相关文件: src/ex_cmds2.c

Patch 6.2.202
问题: Filetype names of CHILL and ch script are confusing.
解决方案: Rename "ch" to "chill" and "chscript" to "ch".
相关文件: runtime/filetype.vim, runtime/makemenu.vim, runtime/synmenu.vim
           runtime/syntax/ch.vim, runtime/syntax/chill.vim

Patch 6.2.203

问题:　　With characterwise text that has more than one line, "3P" works wrong. "3p" has the same problem. There also is a display problem. (Daniel Goujot)
解决方案:　Perform characterwise puts with a count in the right position.
相关文件:　src/ops.c

Patch 6.2.204 (after 6.2.086)
问题:　　"]]" in a file with closed folds moves to the end of the file. (Nam SungHyun)
解决方案:　Find one position in each closed fold, then move to after the fold.
相关文件:　src/search.c

Patch 6.2.205 (extra)
问题:　　MS-Windows: When the taskbar is at the left or top of the screen, the Vim window placement is wrong.
解决方案:　Compute the size and position of the window correctly. (Taro Muraoka)
相关文件:　src/gui_w32.c, src/gui_w48.c

Patch 6.2.206
问题:　　Multi-byte characters cannot be used as hotkeys in a console dialog. (Mattias Erkisson)
解决方案:　Handle multi-byte characters properly. Also put () or [] around default hotkeys.
相关文件:　src/message.c, src/macros.h

Patch 6.2.207
问题:　　When 'encoding' is a multi-byte encoding, expanding an abbreviation that starts where insertion started results in characters before the insertion to be deleted. (Xiangjiang Ma)
解决方案:　Stop searching leftwards for the start of the word at the position where insertion started.
相关文件:　src/getchar.c

Patch 6.2.208
问题:　　When using fold markers, three lines in a row have the start marker and deleting the first one with "dd", a nested fold is not deleted. (Kamil Burzynski)
Using marker folding, a level 1 fold doesn't stop when it is followed by "{{{2", starting a level 2 fold.
解决方案:　Don't stop updating folds at the end of a change when the nesting level of folds is larger than the fold level.
Correctly compute the number of folds that start at "{{{2".
Also avoid a crash for a NULL pointer.
相关文件:　src/fold.c

Patch 6.2.209
问题:　　A bogus fold is created when using "P" while the cursor is in the middle of a closed fold. (Kamil Burzynski)
解决方案:　Correct the line number where marks are modified for closed folds.
相关文件:　src/ops.c

Patch 6.2.210 (extra)
问题:　　Mac OSX: antialiased fonts are not supported.

解决方案: Add the 'antialias' option to switch on antialiasing on Mac OSX
           10.2 and later. (Peter Cucka)
相关文件: runtime/doc/options.txt, src/gui_mac.c, src/option.h, src/option.c

Patch 6.2.211 (extra)
问题: Code for handling file dropped on Vim is duplicated.
解决方案: Move the common code to gui_handle_drop().
           Add code to drop the files in the window under the cursor.
           Support drag&drop on the Macintosh. (Taro Muraoka)
           When dropping a directory name edit that directory (using the
           explorer plugin)
           Fix that changing directory with Shift pressed didn't work for
           relative path names.
相关文件: src/fileio.c, src/gui.c, src/gui_gtk_x11.c, src/gui_mac.c,
           src/gui_w48.c, src/proto/fileio.pro, src/proto/gui.pro

Patch 6.2.212 (after 6.2.199)
问题: NetBeans: Replacing with a count is not handled correctly.
解决方案: Move reporting the change outside of the loop for the count.
           (Gordon Prieur)
相关文件: src/normal.c

Patch 6.2.213 (after 6.2.208)
问题: Using marker folding, "{{{1" doesn't start a new fold when already
           at fold level 1. (Servatius Brandt)
解决方案: Correctly compute the number of folds that start at "{{{1".
相关文件: src/fold.c

Patch 6.2.214 (after 6.2.211) (extra)
问题: Warning for an unused variable.
解决方案: Delete the declaration. (Bill McCarthy)
相关文件: src/gui_w48.c

Patch 6.2.215
问题: NetBeans: problems saving an unmodified file.
解决方案: Add isNetbeansModified() function. Disable netbeans_unmodified().
           (Gordon Prieur)
相关文件: src/fileio.c, src/netbeans.c, src/proto/netbeans.pro,
           runtime/doc/netbeans.txt, runtime/doc/tags

Patch 6.2.216 (after 6.2.206)
问题: Multi-byte characters still cannot be used as hotkeys in a console
           dialog. (Mattias Erkisson)
解决方案: Make get_keystroke() handle multi-byte characters.
相关文件: src/misc1.c

Patch 6.2.217
问题: GTK: setting the title doesn't always work correctly.
解决方案: Invoke gui_mch_settitle(). (Tomas Stehlik)
相关文件: src/os_unix.c

Patch 6.2.218
问题: Warning for function without prototype.
解决方案: Add argument types to the msgCB field of the BalloonEval struct.

相关文件: src/gui_beval.h

Patch 6.2.219
问题: Syntax highlighting hangs on an empty match of an item with a nextgroup. (Charles Campbell)
解决方案: Remember that the item has already matched and don't match it again at the same position.
相关文件: src/syntax.c

Patch 6.2.220
问题: When a Vim server runs in a console a remote command isn't handled before a key is typed. (Joshua Neuheisel)
解决方案: Don't try reading more input when a client-server command has been received.
相关文件: src/os_unix.c

Patch 6.2.221
问题: No file name completion for ":cscope add".
解决方案: Add the XFILE flag to ":cscope". (Gary Johnson)
相关文件: src/ex_cmds.h

Patch 6.2.222
问题: Using "--remote" several times on a row only opens some of the files. (Dany St-Amant)
解决方案: Don't delete all typeahead when the server receives a command from a client, only delete typed characters.
相关文件: src/main.c

Patch 6.2.223
问题: Cscope: Avoid a hang when cscope waits for a response while Vim waits for a prompt.
Error messages from Cscope mess up the display.
解决方案: Detect the hit-enter message and respond by sending a return character to cscope. (Gary Johnson)
Use EMSG() and strerror() when possible. Replace perror() with PERROR() everywhere, add emsg3().
相关文件: src/diff.c, src/if_cscope.c, src/integration.c, src/message.c, src/proto/message.pro, src/misc2.c, src/netbeans.c, src/vim.h

Patch 6.2.224
问题: Mac: Can't compile with small features. (Axel Kielhorn)
解决方案: Also include vim_chdirfile() when compiling for the Mac.
相关文件: src/misc2.c

Patch 6.2.225
问题: NetBeans: Reported modified state isn't exactly right.
解决方案: Report a file being modified in the NetBeans way.
相关文件: src/netbeans.c

Patch 6.2.226 (after 6.2.107) (extra)
问题: The "ws2-32.lib" file isn't always available.
解决方案: Use "WSock32.lib" instead. (Taro Muraoka, Dan Sharp)
相关文件: src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak

Patch 6.2.227 (extra)
问题: The "PC" symbol is defined but not used anywhere.
解决方案: Remove "-DPC" from the makefiles.
相关文件: src/Make_bc3.mak, src/Make_bc5.mak, src/Make_cyg.mak, src/Make_ming.mak

Patch 6.2.228
问题: Receiving CTRL-\ CTRL-N after typing "f" or "m" doesn't switch Vim back to Normal mode. Same for CTRL-\ CTRL-G.
解决方案: Check if the character typed after a command is CTRL-\ and obtain another character to check for CTRL-N or CTRL-G, waiting up to 'ttimeoutlen' msec.
相关文件: src/normal.c

Patch 6.2.229
问题: ":function" with a name that uses magic curlies does not work inside a function. (Servatius Brandt)
解决方案: Skip over the function name properly.
相关文件: src/eval.c

Patch 6.2.230 (extra)
问题: Win32: a complex pattern may cause a crash.
解决方案: Use __try and __except to catch the exception and handle it gracefully, when possible. Add myresetstkoflw() to reset the stack overflow. (Benjamin Peterson)
相关文件: src/Make_bc5.mak, src/os_mswin.c src/os_win32.c, src/os_win32.h, src/proto/os_win32.pro, src/regexp.c

Patch 6.2.231 (after 6.2.046)
问题: Various problems when an error exception is raised from within a builtin function. When it is invoked while evaluating arguments to a function following arguments are still evaluated. When invoked with a line range it will be called for remaining lines.
解决方案: Update "force_abort" also after calling a builtin function, so that aborting() always returns the correct value. (Servatius Brandt)
相关文件: src/eval.c, src/ex_eval.c, src/proto/ex_eval.pro, src/testdir/test49.ok, src/testdir/test49.vim

Patch 6.2.232
问题: ":python vim.command('python print 2*2')" crashes Vim. (Eugene Minkovskii)
解决方案: Disallow executing a Python command recursively and give an error message.
相关文件: src/if_python.c

Patch 6.2.233
问题: On Mac OSX adding -pthread for Python only generates a warning.
The test for Perl threads rejects Perl while it's OK.
Tcl doesn't work at all.
The test for Ruby fails if ruby exists but there are no header files. The Ruby library isn't detected properly
解决方案: Avoid adding -pthread on Mac OSX. Accept Perl threads when it's not the 5.5 threads.

Use the Tcl framework for header files. For Ruby rename cWindow
to cVimWindow to avoid a name clash. (Ken Scott)
Only enable Ruby when the header files can be found. Use "-lruby"
instead of "libruby.a" when it can't be found.
相关文件: src/auto/configure, src/configure.in, src/if_ruby.c

Patch 6.2.234
问题: GTK 2 GUI: ":sp" and the ":q" leaves the cursor on the command
line.
解决方案: Flush output before removing scrollbars. Also do this in other
places where gui_mch_*() functions are invoked.
相关文件: src/ex_cmds.c, src/option.c, src/window.c

Patch 6.2.235 (extra)
问题: Win32: Cursor isn't removed with a 25x80 window and doing:
"1830ia<Esc>400a-<Esc>0w0". (Yasuhiro Matsumoto)
解决方案: Remove the call to gui_undraw_cursor() from gui_mch_insert_lines().
相关文件: src/gui_w48.c

Patch 6.2.236
问题: Using gvim with Agide gives "connection lost" error messages.
解决方案: Only give the "connection lost" message when the buffer was once
owned by NetBeans.
相关文件: src/netbeans.c, src/structs.h

Patch 6.2.237
问题: GTK 2: Thai text is drawn wrong. It changes when moving the
cursor over it.
解决方案: Disable the shaping engine, it moves combining characters to a
wrong position and combines characters, while drawing the cursor
doesn't combine characters.
相关文件: src/gui_gtk_x11.c

Patch 6.2.238 (after 6.2.231)
问题: ":function" does not work inside a while loop. (Servatius Brandt)
解决方案: Add get_while_line() and pass it to do_one_cmd() when in a while
loop, so that all lines are stored and can be used again when
repeating the loop.
Adjust test 49 so that it checks for the fixed problems.
(Servatius Brandt)
相关文件: src/digraph.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c,
src/proto/ex_cmds2.pro, src/proto/ex_docmd.pro,
src/testdir/test49.in, src/testdir/test49.ok,
src/testdir/test49.vim

Patch 6.2.239
问题: GTK 2: With closed folds the arrow buttons of a vertical scrollbar
often doesn't scroll. (Moshe Kaminsky)
解决方案: Hackish solution: Detect that the button was pressed from the
mouse pointer position.
相关文件: src/gui_gtk.c, src/gui.c

Patch 6.2.240
问题: GTK 2: Searching for bitmaps for the toolbar doesn't work as with

other systems. Need to explicitly use "icon=name". (Ned Konz, Christian J. Robinson)
解决方案: Search for icons like done for Motif.
相关文件: src/gui_gtk.c

Patch 6.2.241
问题: GTK 2: Search and Search/Replace dialogs are synced, that makes no sense. Buttons are sometimes greyed-out. (Jeremy Messenger)
解决方案: Remove the code to sync the two dialogs. Adjust the code to react to an empty search string to also work for GTK2. (David Necas)
相关文件: src/gui_gtk.c

Patch 6.2.242
问题: Gnome: "vim --help" only shows the Gnome arguments, not the Vim arguments.
解决方案: Don't let the Gnome code remove the "--help" argument and don't exit at the end of usage().
相关文件: src/gui_gtk_x11.c, src/main.c

Patch 6.2.243 (extra)
问题: Mac: Dropping a file on a Vim icon causes a hit-enter prompt.
解决方案: Move the dropped files to the global argument list, instead of the usual drop handling. (Eckehard Berns)
相关文件: src/main.c, src/gui_mac.c

Patch 6.2.244
问题: ':echo "\xf7"' displays the illegal byte as if it was a character and leaves "cho" after it.
解决方案: When checking the length of a UTF-8 byte sequence and it's shorter than the number of bytes available, assume it's an illegal byte.
相关文件: src/mbyte.c

Patch 6.2.245
问题: Completion doesn't work for ":keepmarks" and ":lockmarks".
解决方案: Add the command modifiers to the table of commands. (Madoka Machitani)
相关文件: src/ex_cmds.h, src/ex_docmd.c

Patch 6.2.246
问题: Mac: Starting Vim from Finder doesn't show error messages.
解决方案: Recognize that output is being displayed by stderr being "/dev/console". (Eckehard Berns)
相关文件: src/main.c, src/message.c

Patch 6.2.247 (after 6.2.193)
问题: When using a search pattern from the viminfo file the last character is replaced with a '/'.
解决方案: Store the separator character in the right place. (Kelvin Lee)
相关文件: src/ex_getln.c

Patch 6.2.248
问题: GTK: When XIM is enabled normal "2" and keypad "2" cannot be distinguished.
解决方案: Detect that XIM changes the keypad key to the expected ASCII

character and fall back to the non-XIM code. (Neil Bird)
相关文件: src/gui_gtk_x11.c, src/mbyte.c, src/proto/mbyte.pro

Patch 6.2.249
问题: ":cnext" moves to the error in the next file, but there is no method to go back.
解决方案: Add ":cpfile" and ":cNfile".
相关文件: src/ex_cmds.h, src/quickfix.c, src/vim.h, runtime/doc/quickfix.txt

Patch 6.2.250
问题: Memory leaks when using signs. (Xavier de Gaye)
解决方案: Delete the list of signs when unloading a buffer.
相关文件: src/buffer.c

Patch 6.2.251
问题: GTK: The 'v' flag in 'guioptions' doesn't work. (Steve Hall) Order of buttons is reversed for GTK 2.2.4. Don't always get focus back after handling a dialog.
解决方案: Make buttons appear vertically when desired. Reverse the order in which buttons are added to a dialog. Move mouse pointer around when the dialog is done and we don't have focus.
相关文件: src/gui_gtk.c

Patch 6.2.252 (extra, after 6.2.243)
问题: Mac: Dropping a file on a Vim icon causes a hit-enter prompt for Mac OS classic.
解决方案: Remove the #ifdef from the code that fixes it for Mac OSX.
相关文件: src/gui_mac.c

Patch 6.2.253
问题: When 'tagstack' is not set a ":tag id" command does not work after a ":tjump" command.
解决方案: Set "new_tag" when 'tagstack' isn't set. (G. Narendran)
相关文件: src/tag.c

Patch 6.2.254
问题: May run out of space for error messages.
解决方案: Keep room for two more bytes.
相关文件: src/quickfix.c

Patch 6.2.255
问题: GTK: A new item in the popup menu is put just after instead of just before the right item. (Gabriel Zachmann)
解决方案: Don't increment the menu item index.
相关文件: src/gui_gtk.c

Patch 6.2.256
问题: Mac: "macroman" encoding isn't recognized, need to use "8bit-macroman".
解决方案: Recognize "macroman" with an alias "mac". (Eckehard Berns)
相关文件: src/mbyte.c

Patch 6.2.257 (after 6.2.250)
问题: Signs are deleted for ":bdel", but they could still be useful.

解决方案: Delete signs only for ":bwipe".
相关文件: src/buffer.c

Patch 6.2.258
问题: GUI: can't disable (grey-out) a popup menu item. (Ajit Thakkar)
解决方案: Loop over the popup menus for all modes.
相关文件: src/menu.c

Patch 6.2.259
问题: If there are messages when exiting, on the console there is a hit-enter prompt while the message can be read; in the GUI the message may not be visible.
解决方案: Use the hit-enter prompt when there is an error message from writing the viminfo file or autocommands, or when there is any output in the GUI and 'verbose' is set. Don't use a hit-enter prompt for the non-GUI version unless there is an error message.
相关文件: src/main.c

Patch 6.2.260
问题: GTK 2: Can't quit a dialog with <Esc>.
GTK 1 and 2: <Enter> always gives a result, even when the default button has been disabled.
解决方案: Handle these keys explicitly. When no default button is specified use the first one (works mostly like it was before).
相关文件: src/gui_gtk.c

Patch 6.2.261
问题: When 'autoindent' and 'cindent' are set and a line is recognized as a comment, starting a new line won't do 'cindent' formatting.
解决方案: Also use 'cindent' formatting for lines that are used as a comment. (Servatius Brandt)
相关文件: src/misc1.c

Patch 6.2.262
问题: 1 CTRL-W w beeps, even though going to the first window is possible. (Charles Campbell)
解决方案: Don't beep.
相关文件: src/window.c

Patch 6.2.263
问题: Lint warnings: Duplicate function prototypes, duplicate macros, use of a zero character instead of a zero pointer, unused variable. Clearing allocated memory in a complicated way.
解决方案: Remove the function prototypes from farsi.h. Remove the duplicated lines in keymap.h. Change getvcol() argument from NUL to NULL. Remove the "col" variable in regmatch(). Use lalloc_clear() instead of lalloc(). (Walter Briscoe)
相关文件: src/farsi.h, src/keymap.h, src/ops.c, src/regexp.c, src/search.c

Patch 6.2.264 (after 6.2.247)
问题: Writing past allocated memory when using a command line from the viminfo file.
解决方案: Store the NUL in the right place.
相关文件: src/ex_getln.c

Patch 6.2.265
问题: Although ":set" is not allowed in the sandbox, ":let &opt = val" works.
解决方案: Do allow changing options in the sandbox, but not the ones that can't be changed from a modeline.
相关文件: src/ex_cmds.h, src/options.c

Patch 6.2.266
问题: When redirecting output and using ":silent", line breaks are missing from output of ":map" and ":tselect". Alignment of columns is wrong.
解决方案: Insert a line break where "msg_didout" was tested. Update msg_col when redirecting and using ":silent".
相关文件: src/getchar.c, src/message.c

Patch 6.2.267 (extra)
问题: Win32: "&&" in a tearoff menu is not shown. (Luc Hermitte)
解决方案: Use the "name" item from the menu instead of the "dname" item.
相关文件: src/gui_w32.c, src/menu.c

Patch 6.2.268
问题: GUI: When changing 'guioptions' part of the window may be off screen. (Randall Morris)
解决方案: Adjust the size of the window when changing 'guioptions', but only when adding something.
相关文件: src/gui.c

Patch 6.2.269
问题: Diff mode does not highlight a change in a combining character. (Raphael Finkel)
解决方案: Make diff_find_change() multi-byte aware: find the start byte of a character that contains a change.
相关文件: src/diff.c

Patch 6.2.270
问题: Completion in Insert mode, then repeating with ".", doesn't handle composing characters in the completed text. (Raphael Finkel)
解决方案: Don't skip over composing chars when adding completed text to the redo buffer.
相关文件: src/getchar.c

Patch 6.2.271
问题: NetBeans: Can't do "tail -f" on the log. Passing socket info with an argument or environment variable is not secure.
解决方案: Wait after initializing the log. Allow passing the socket info through a file. (Gordon Prieur)
相关文件: runtime/doc/netbeans.txt, src/main.c, src/netbeans.c

Patch 6.2.272
问题: When the "po" directory exists, but "po/Makefile" doesn't, building fails. Make loops when the "po" directory has been deleted after running configure.
解决方案: Check for the "po/Makefile" instead of just the "po" directory.

Check this again before trying to run make with that Makefile.
相关文件: src/auto/configure, src/configure.in, src/Makefile

Patch 6.2.273
问题: Changing the sort order in an explorer window for an empty directory produces error messages. (Doug Kearns)
解决方案: When an invalid range is used for a function that is not going to be executed, skip over the arguments anyway.
相关文件: src/eval.c

Patch 6.2.274
问题: ":print" skips empty lines when 'list' is set and there is no "eol" in 'listchars'. (Yakov Lerner)
解决方案: Skip outputting a space for an empty line only when 'list' is set and the end-of-line character is not empty.
相关文件: src/message.c

Patch 6.2.275 (extra, after 6.2.267)
问题: Warning for uninitialized variable when using gcc.
解决方案: Initialize "acLen" to zero. (Bill McCarthy)
相关文件: src/gui_w32.c

Patch 6.2.276
问题: ":echo X()" does not put a line break between the message that X() displays and the text that X() returns. (Yakov Lerner)
解决方案: Invoke msg_start() after evaluating the argument.
相关文件: src/eval.c

Patch 6.2.277
问题: Vim crashes when a ":runtime ftplugin/ada.vim" causes a recursive loop. (Robert Nowotniak)
解决方案: Restore "msg_list" before returning from do_cmdline().
相关文件: src/ex_docmd.c

Patch 6.2.278
问题: Using "much" instead of "many".
解决方案: Correct the error message.
相关文件: src/eval.c

Patch 6.2.279
问题: There is no default choice for a confirm() dialog, now that it is possible not to have a default choice.
解决方案: Make the first choice the default choice.
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 6.2.280
问题: "do" and ":diffget" don't work in the first line and the last line of a buffer. (Aron Griffis)
解决方案: Find a difference above the first line and below the last line. Also fix a few display updating bugs.
相关文件: src/diff.c, src/fold.c, src/move.c

Patch 6.2.281
问题: PostScript printing doesn't work on Mac OS X 10.3.2.

解决方案: Adjust the header file. (Mike Williams)
相关文件: runtime/print/prolog.ps

Patch 6.2.282
问题: When using CTRL-O to go back to a help file, it becomes listed.
(Andrew Nesbit)
Using ":tag" or ":tjump" in a help file doesn't keep the help file
settings (e.g. for 'iskeyword').
解决方案: Don't mark a buffer as listed when its help flag is set. Put all
the option settings for a help buffer together in do_ecmd().
相关文件: src/ex_cmds.c

Patch 6.2.283
问题: The "local additions" in help.txt are used without conversion,
causing latin1 characters showing up wrong when 'enc' is utf-8.
(Antoine J. Mechelynck)
解决方案: Convert the text to 'encoding'.
相关文件: src/ex_cmds.c

Patch 6.2.284
问题: Listing a function puts "endfunction" in the message history.
Typing "q" at the more prompt isn't handled correctly when listing
variables and functions. (Hara Krishna Dara)
解决方案: Don't use msg() for "endfunction". Check "got_int" regularly.
相关文件: src/eval.c

Patch 6.2.285
问题: GUI: In a single wrapped line that fills the window, "gj" in the
last screen line leaves the cursor behind. (Ivan Tarasov)
解决方案: Undraw the cursor before scrolling the text up.
相关文件: src/gui.c

Patch 6.2.286
问题: When trying to rename a file and it doesn't exist, the destination
file is deleted anyway. (Luc Deux)
解决方案: Don't delete the destination when the source doesn't exist. (Taro
Muraoka)
相关文件: src/fileio.c

Patch 6.2.287 (after 6.2.264)
问题: Duplicate lines are added to the viminfo file.
解决方案: Compare with existing entries without an offset. Also fixes
reading very long history lines from viminfo.
相关文件: src/ex_getln.c

Patch 6.2.288 (extra)
问题: Mac: An external program can't be interrupted.
解决方案: Don't use the 'c' key for backspace. (Eckehard Berns)
相关文件: src/gui_mac.c

Patch 6.2.289
问题: Compiling the Tcl interface with thread support causes ":make" to
fail. (Juergen Salk)
解决方案: Use $TCL_DEFS from the Tcl config script to obtain the required

compile flags for using the thread library.
相关文件: src/auto/configure, src/configure.in

Patch 6.2.290 (extra)
问题: Mac: The mousewheel doesn't work.
解决方案: Add mousewheel support. Also fix updating the thumb after a drag and then using another way to scroll. (Eckehard Berns)
相关文件: src/gui_mac.c

Patch 6.2.291 (extra)
问题: Mac: the plus button and close button don't do anything.
解决方案: Make the plus button maximize the window and the close button close Vim. (Eckehard Berns)
相关文件: src/gui.c, src/gui_mac.c

Patch 6.2.292
问题: Motif: When removing GUI arguments from argv[] a "ps -ef" shows the last argument repeated.
解决方案: Set argv[argc] to NULL. (Michael Jarvis)
相关文件: src/gui_x11.c

Patch 6.2.293 (after 6.2.255)
问题: GTK: A new item in a menu is put before the tearoff item.
解决方案: Do increment the menu item index for non-popup menu items.
相关文件: src/gui_gtk.c

Patch 6.2.294 (extra)
问题: Mac: Cannot use modifiers with Space, Tab, Enter and Escape.
解决方案: Handle all modifiers for these keys. (Eckehard Berns)
相关文件: src/gui_mac.c

Patch 6.2.295
问题: When in debug mode, receiving a message from a remote client causes a crash. Evaluating an expression causes Vim to wait for "cont" to be typed, without a prompt. (Hari Krishna Dara)
解决方案: Disable debugging when evaluating an expression for a client. (Michael Geddes) Don't try reading into the typeahead buffer when it may have been filled in another way.
相关文件: src/ex_getln.c, src/getchar.c, src/if_xcmdsrv.c, src/main.c, src/misc1.c, src/proto/getchar.pro, src/proto/main.pro, src/proto/os_unix.pro, src/proto/ui.pro, src/structs.h, src/os_unix.c, src/ui.c

Patch 6.2.296 (extra)
问题: Same as 6.2.295.
解决方案: Extra files for patch 6.2.295.
相关文件: src/os_amiga.c, src/os_msdos.c, src/os_riscos.c, src/os_win32.c, src/proto/os_amiga.pro, src/proto/os_msdos.pro, src/proto/os_riscos.pro, src/proto/os_win32.pro

Patch 6.2.297 (after 6.2.232)
问题: Cannot invoke Python commands recursively.
解决方案: With Python 2.3 and later use the available mechanisms to invoke Python recursively. (Matthew Mueller)

相关文件: src/if_python.c

Patch 6.2.298
问题: A change always sets the '. mark and an insert always sets the '^
mark, even when this is not wanted.
Cannot go back to the position of older changes without undoing
those changes.
解决方案: Add the ":keepjumps" command modifier.
Add the "g," and "g;" commands.
相关文件: runtime/doc/motion.txt, src/ex_cmds.h, src/ex_docmd.c, src/edit.c,
src/mark.c, src/misc1.c, src/normal.c, src/proto/mark.pro,
src/structs.h, src/undo.c

Patch 6.2.299
问题: Can only use one language for help files.
解决方案: Add the 'helplang' option to select the preferred language(s).
Make ":helptags" generate tags files for all languages.
相关文件: runtime/doc/options.txt, runtime/doc/various.txt, src/Makefile,
src/ex_cmds.c, src/ex_cmds2.c, src/ex_cmds.h, src/ex_getln.c,
src/normal.c, src/option.c, src/option.h, src/proto/ex_cmds.pro,
src/proto/ex_cmds2.pro, src/proto/option.pro, src/structs.h,
src/tag.c, src/vim.h

Patch 6.2.300 (after 6.2.297)
问题: Cannot build Python interface with Python 2.2 or earlier.
解决方案: Add a semicolon.
相关文件: src/if_python.c

Patch 6.2.301
问题: The "select all" item from the popup menu doesn't work for Select
mode.
解决方案: Use the same commands as for the "Edit.select all" menu.
(Benji Fisher)
相关文件: runtime/menu.vim

Patch 6.2.302
问题: Using "CTRL-O ." in Insert mode doesn't work properly. (Benji
Fisher)
解决方案: Restore "restart_edit" after an insert command that was not typed.
Avoid waiting with displaying the mode when there is no text to be
overwritten.
Fix that "CTRL-O ." sometimes doesn't put the cursor back after
the end-of-line. Only reset the flag that CTRL-O was used past
the end of the line when restarting editing. Update "o_lnum"
number when inserting text and "o_eol" is set.
相关文件: src/edit.c, src/normal.c

Patch 6.2.303
问题: Cannot use Unicode digraphs while 'encoding' is not Unicode.
解决方案: Convert the character from Unicode to 'encoding' when needed.
Use the Unicode digraphs for the Macintosh. (Eckehard Berns)
相关文件: src/digraph.c

Patch 6.2.304 (extra, after 6.2.256)

问题:　　Mac: No proper support for 'encoding'. Conversion without iconv() is not possible.
解决方案:　Convert input from 'termencoding' to 'encoding'. Add mac_string_convert(). Convert text for the clipboard when needed. (Eckehard Berns)
相关文件:　src/gui_mac.c, src/mbyte.c, src/structs.h, src/vim.h

Patch 6.2.305 (after 6.2.300)
问题:　　Win32: Cannot build Python interface with Python 2.3. (Ajit Thakkar)
解决方案:　Add two functions to the dynamic loading feature.
相关文件:　src/if_python.c

Patch 6.2.306 (extra)
问题:　　Win32: Building console version with BCC 5.5 gives a warning for get_cmd_args() prototype missing. (Ajit Thakkar)
解决方案:　Don't build os_w32exe.c for the console version.
相关文件:　src/Make_bc5.mak

Patch 6.2.307 (after 6.2.299)
问题:　　Installing help files fails.
解决方案:　Expand wildcards for translated help files separately.
相关文件:　src/Makefile

Patch 6.2.308
问题:　　Not all systems have "whoami", resulting in an empty user name.
解决方案:　Use "logname" when possible, "whoami" otherwise. (David Boyce)
相关文件:　src/Makefile

Patch 6.2.309
问题:　　"3grx" waits for two ESC to be typed. (Jens Paulus)
解决方案:　Append the ESC to the stuff buffer when redoing the "gr" insert.
相关文件:　src/edit.c

Patch 6.2.310
问题:　　When setting 'undolevels' to -1, making a change and setting 'undolevels' to a positive value an "undo list corrupt" error occurs. (Madoka Machitani)
解决方案:　Sync undo before changing 'undolevels'.
相关文件:　src/option.c

Patch 6.2.311 (after 6.2.298)
问题:　　When making several changes in one line the changelist grows quickly. There is no error message for reaching the end of the changelist. Reading changelist marks from viminfo doesn't work properly.
解决方案:　Only make a new entry in the changelist when making a change in another line or 'textwidth' columns away. Add E662, E663 and E664 error messages. Put a changelist mark from viminfo one position before the end.
相关文件:　runtime/doc/motion.txt, src/mark.c, src/misc1.c, src/normal.c

Patch 6.2.312 (after 6.2.299)
问题:　　"make install" clears the screen when installing the docs.

解决方案: Execute ":helptags" in silent mode.
相关文件: runtime/doc/Makefile

Patch 6.2.313
问题: When opening folds in a diff window, other diff windows no longer show the same text.
解决方案: Sync the folds in diff windows.
相关文件: src/diff.c, src/fold.c, src/move.c, src/proto/diff.pro, src/proto/move.pro

Patch 6.2.314
问题: When 'virtualedit' is set "rx" may cause a crash with a blockwise selection and using "$". (Moritz Orbach)
解决方案: Don't try replacing chars in a line that has no characters in the block.
相关文件: src/ops.c

Patch 6.2.315
问题: Using CTRL-C in a Visual mode mapping while 'insertmode' is set stops Vim from returning to Insert mode.
解决方案: Don't reset "restart_edit" when a CTRL-C is found and 'insertmode' is set.
相关文件: src/normal.c

Patch 6.2.316 (after 6.2.312)
问题: "make install" tries connecting to the X server when installing the docs. (Stephen Thomas)
解决方案: Add the "-X" argument.
相关文件: runtime/doc/Makefile

Patch 6.2.317 (after 6.2.313)
问题: When using "zi" in a diff window, other diff windows are not adjusted. (Richard Curnow)
解决方案: Distribute a change in 'foldenable' to other diff windows.
相关文件: src/normal.c

Patch 6.2.318
问题: When compiling with _THREAD_SAFE external commands don't echo typed characters.
解决方案: Don't set the terminal mode to TMODE_SLEEP when it's already at TMODE_COOK.
相关文件: src/os_unix.c

Patch 6.2.319 (extra)
问题: Building gvimext.dll with Mingw doesn't work properly.
解决方案: Use gcc instead of dllwrap. Use long option names. (Alejandro Lopez-Valencia)
相关文件: src/GvimExt/Make_ming.mak

Patch 6.2.320
问题: Win32: Adding and removing the menubar resizes the Vim window. (Jonathon Merz)
解决方案: Don't let a resize event change 'lines' unexpectedly.
相关文件: src/gui.c

Patch 6.2.321
问题: When using modeless selection, wrapping lines are not recognized, a line break is always inserted.
解决方案: Add LineWraps[] to remember whether a line wrapped or not.
相关文件: src/globals.h, src/screen.c, src/ui.c

Patch 6.2.322
问题: With 'showcmd' set, after typing "dd" the next "d" may not be displayed. (Jens Paulus)
解决方案: Redraw the command line after updating the screen, scrolling may have set "clear_cmdline".
相关文件: src/screen.c

Patch 6.2.323
问题: Win32: expanding "~/file" in an autocommand pattern results in backslashes, while this pattern should only have forward slashes.
解决方案: Make expanding environment variables respect 'shellslash' and set p_ssl when expanding the autocommand pattern.
相关文件: src/fileio.c, src/misc1.c, src/proto/fileio.pro

Patch 6.2.324 (extra)
问题: Win32: when "vimrun.exe" has a path with white space, such as "Program Files", executing external commands may fail.
解决方案: Put double quotes around the path to "vimrun".
相关文件: src/os_win32.c

Patch 6.2.325
问题: When $HOME includes a space, doing ":set tags=~/tags" doesn't work, the space is used to separate file names. (Brett Stahlman)
解决方案: Escape the space with a backslash.
相关文件: src/option.c

Patch 6.2.326
问题: ":windo set syntax=foo" doesn't work. (Tim Chase)
解决方案: Don't change 'eventignore' for ":windo".
相关文件: src/ex_cmds2.c

Patch 6.2.327
问题: When formatting text all marks in the formatted lines are lost. A word is not joined to a previous line when this would be possible. (Mikolaj Machowski)
解决方案: Try to keep marks in the same position as much as possible. Also keep mark positions when joining lines. Start auto-formatting in the previous line when appropriate. Add the "gw" operator: Like "gq" but keep the cursor where it is.
相关文件: runtime/doc/change.txt, src/edit.c, src/globals.h, src/mark.c, src/misc1.c, src/normal.c, src/ops.c, src/proto/edit.pro, src/proto/mark.pro, src/proto/ops.pro, src/structs.h, src/vim.h

Patch 6.2.328
问题: XIM with GTK: It is hard to understand what XIM is doing.
解决方案: Add xim_log() to log XIM events and help with debugging.
相关文件: src/mbyte.c

Patch 6.2.329
问题: ":=" does not work Vi compatible. (Antony Scriven)
解决方案: Print the last line number instead of the current line. Don't print "line".
相关文件: src/ex_cmds.h, src/ex_docmd.c

Patch 6.2.330 (extra, after 6.2.267)
问题: Win32: Crash when tearing off a menu.
解决方案: Terminate a string with a NUL. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 6.2.331 (after 6.2.327)
问题: "gwap" leaves cursor in the wrong line.
解决方案: Remember the cursor position before finding the ends of the paragraph.
相关文件: src/normal.c, src/ops.c, src/structs.h

Patch 6.2.332 (extra)
问题: Amiga: Compile error for string array. Compiling the Amiga GUI doesn't work.
解决方案: Use a char pointer instead. Move including "gui_amiga.h" to after including "vim.h". Add a semicolon. (Ali Akcaagac)
相关文件: src/gui_amiga.c, src/os_amiga.c

Patch 6.2.333 (extra)
问题: Win32: printing doesn't work with specified font charset.
解决方案: Use the specified font charset. (Mike Williams)
相关文件: src/os_mswin.c

Patch 6.2.334 (extra, after 6.2.296)
问题: Win32: evaluating client expression in debug mode requires typing "cont".
解决方案: Use eval_client_expr_to_string().
相关文件: src/os_mswin.c

Patch 6.2.335
问题: The ":sign" command cannot be followed by another command.
解决方案: Add TRLBAR to the command flags.
相关文件: src/ex_cmds.h

Patch 6.2.336 (after 6.2.327)
问题: Mixup of items in an expression.
解决方案: Move "== NUL" to the right spot.
相关文件: src/edit.c

Patch 6.2.337 (extra, after 6.2.319)
问题: Building gvimext.dll with Mingw doesn't work properly.
解决方案: Fix white space and other details. (Alejandro Lopez-Valencia)
相关文件: src/GvimExt/Make_ming.mak

Patch 6.2.338 (after 6.2.331)
问题: When undoing "gwap" the cursor is always put at the start of the paragraph. When undoing auto-formatting the cursor may be above

the change.
解决方案: Try to move the cursor back to where it was or to the first line that actually changed.
相关文件: src/normal.c, src/ops.c, src/undo.c

Patch 6.2.339
问题: Crash when using many different highlight groups and a User highlight group. (Juergen Kraemer)
解决方案: Do not use the sg_name_u pointer when it is NULL. Also simplify use of the highlight group table.
相关文件: src/syntax.c

Patch 6.2.340
问题: ":reg" doesn't show the actual contents of the clipboard if it was filled outside of Vim. (Stuart MacDonald)
解决方案: Obtain the clipboard contents before displaying it.
相关文件: src/ops.c

Patch 6.2.341 (extra)
问题: Win32: When the path to diff.exe contains a space and using the vimrc generated by the install program, diff mode does not work.
解决方案: Put the first double quote just before the space instead of before the path.
相关文件: src/dosinst.c

Patch 6.2.342 (extra)
问题: Win32: macros are not always used as expected.
解决方案: Define WINVER to 0x0400 instead of 0x400. (Alejandro Lopez-Valencia)
相关文件: src/Make_bc5.mak, src/Make_cyg.mak, src/Make_mvc.mak

Patch 6.2.343
问题: Title doesn't work with some window managers. X11: Setting the text property for the window title is hard coded.
解决方案: Use STRING format when possible. Use the UTF-8 function when it's available and 'encoding' is utf-8. Use XStringListToTextProperty(). Do the same for the icon name. (David Harrison)
相关文件: src/os_unix.c

Patch 6.2.344 (extra, after 6.2.337)
问题: Cannot build gvimext.dll with MingW on Linux.
解决方案: Add support for cross compiling. (Ronald Hoellwarth)
相关文件: src/GvimExt/Make_ming.mak

Patch 6.2.345 (extra)
问题: Win32: Copy/paste between two Vims fails if 'encoding' is not set properly or there are illegal bytes.
解决方案: Use a raw byte format. Always set it when copying. When pasting use the raw format if 'encoding' is the same.
相关文件: src/os_mswin.c, src/os_win16.c, src/os_win32.c, src/vim.h

Patch 6.2.346
问题: Win32 console: After using "chcp" Vim does not detect the

different codepage.
解决方案: Use GetConsoleCP() and when it is different from GetACP() set 'termencoding'.
相关文件: src/option.c

Patch 6.2.347 (extra)
问题: Win32: XP theme support is missing.
解决方案: Add a manifest and refer to it from the resource file. (Michael Wookey)
相关文件: Makefile, src/gvim.exe.mnf, src/vim.rc

Patch 6.2.348
问题: Win32: "vim c:\dir\(test)" doesn't work, because the 'isfname' default value doesn't contain parenthesis.
解决方案: Temporarily add '(' and ')' to 'isfname' when expanding file name arguments.
相关文件: src/main.c

Patch 6.2.349
问题: Finding a match using 'matchpairs' may cause a crash. 'matchpairs' is not used for 'showmatch'.
解决方案: Don't look past the NUL in 'matchpairs'. Use 'matchpairs' for 'showmatch'. (Dave Olszewkski)
相关文件: src/misc1.c, src/normal.c, src/proto/search.pro, src/search.c

Patch 6.2.350
问题: Not enough info about startup timing.
解决方案: Add a few more TIME_MSG() calls.
相关文件: src/main.c

Patch 6.2.351
问题: Win32: $HOME may be set to %USERPROFILE%.
解决方案: Expand %VAR% at the start of $HOME.
相关文件: src/misc1.c

Patch 6.2.352 (after 6.2.335)
问题: ":sign texthl=||" does not work.
解决方案: Remove the check for a following command. Give an error for extra arguments after "buff=1".
相关文件: src/ex_cmds.c, src/ex_cmds.h

Patch 6.2.353 (extra)
问题: Win32: Supported server name length is limited. (Paul Bossi)
解决方案: Use MAX_PATH instead of 25.
相关文件: src/os_mswin.c

Patch 6.2.354 (extra)
问题: Win32: When the mouse pointer is on a tear-off menu it is hidden when typing but is not redisplayed when moved. (Markx Hackmann)
解决方案: Handle the pointer move event for the tear-off menu window.
相关文件: src/gui_w32.c

Patch 6.2.355 (after 6.2.303)
问题: When 'encoding' is a double-byte encoding different from the

current locale, the width of characters is not correct.
Possible failure and memory leak when using iconv, Unicode
digraphs and 'encoding' is not "utf-8".
解决方案: Use iconv() to discover the actual width of characters.
Add the "vc_fail" field to vimconv_T.
When converting a digraph, init the conversion type to NONE and
cleanup afterwards.
相关文件: src/digraph.c, src/mbyte.c, src/structs.h

Patch 6.2.356
问题: When using a double-byte 'encoding' and 'selection' is
"exclusive", "vy" only yanks the first byte of a double-byte
character. (Xiangjiang Ma)
解决方案: Correct the column in unadjust_for_sel() to position on the first
byte, always include the trailing byte of the selected text.
相关文件: src/normal.c

Patch 6.2.357 (after 6.2.321)
问题: Memory leak when resizing the Vim window.
解决方案: Free the LineWraps array.
相关文件: src/screen.c

Patch 6.2.358 (after 6.2.299)
问题: Memory leak when using ":help" and the language doesn't match.
解决方案: Free the array with matching tags.
相关文件: src/ex_cmds.c

Patch 6.2.359 (after 6.2.352)
问题: Compiler warning for long to int type cast.
解决方案: Add explicit type cast.
相关文件: src/ex_cmds.c

Patch 6.2.360
问题: "100|" in an empty line results in a ruler "1,0-100". (Pavol
Juhas)
解决方案: Recompute w_virtcol if the target column was not reached.
相关文件: src/misc2.c

Patch 6.2.361 (extra)
问题: Win32: Run gvim, ":set go-=m", use Alt-Tab, keep Alt pressed while
pressing Esc, then release Alt: Cursor disappears and typing a key
causes a beep. (Hari Krishna Dara)
解决方案: Don't ignore the WM_SYSKEYUP event when the menu is disabled.
相关文件: src/gui_w32.c

Patch 6.2.362 (extra, after 6.2.347)
问题: Win32: The manifest causes gvim not to work. (Dave Roberts)
解决方案: Change "x86" to "X86". (Serge Pirotte)
相关文件: src/gvim.exe.mnf

Patch 6.2.363
问题: In an empty file with 'showmode' off, "i" doesn't change the ruler
from "0-1" to "1". Typing "x<BS>" does show "1", but then <Esc>
doesn't make it "0-1" again. Same problem for ruler in

statusline. (Andrew Pimlott)
解决方案: Remember the "empty line" flag with Insert mode and'ed to it.
相关文件: src/screen.c

Patch 6.2.364
问题: HTML version of the documentation doesn't mention the encoding, which is a problem for mbyte.txt.
解决方案: Adjust the awk script. (Ilya Sher)
相关文件: runtime/doc/makehtml.awk

Patch 6.2.365
问题: The configure checks for Perl and Python may add compile and link arguments that break building Vim.
解决方案: Do a sanity check: try building with the arguments.
相关文件: src/auto/configure, src/configure.in

Patch 6.2.366
问题: When the GUI can't start because no valid font is found, there is no error message. (Ugen)
解决方案: Add an error message.
相关文件: src/gui.c

Patch 6.2.367
问题: Building the help tags file while installing may fail if there is another Vim in $PATH.
解决方案: Specify the just installed Vim executable. (Gordon Prieur)
相关文件: src/Makefile

Patch 6.2.368
问题: When 'autochdir' is set, closing a window doesn't change to the directory of the new current window. (Salman Halim)
解决方案: Handle 'autochdir' always when a window becomes the current one.
相关文件: src/window.c

Patch 6.2.369
问题: Various memory leaks: when using globpath(), when searching for help tags files, when defining a function inside a function, when giving an error message through an exception, for the final "." line in ":append", in expression "cond ? a : b" that fails and for missing ")" in an expression. Using NULL pointer when adding first user command and for pointer computations with regexp. (tests by Dominique Pelle)
解决方案: Fix the leaks by freeing the allocated memory. Don't use the array of user commands when there are no entries. Use a macro instead of a function call for saving and restoring regexp states.
相关文件: src/eval.c, src/ex_cmds.c, src/ex_docmd.c, src/ex_getln.c, src/misc2.c, src/regexp.c, src/screen.c, src/tag.c

Patch 6.2.370 (extra, after6.2.341)
问题: Win32: When the path to diff.exe contains a space and using the vimrc generated by the install program, diff mode may not work. (Alejandro Lopez-Valencia)
解决方案: Do not use double quotes for arguments that do not have a space.
相关文件: src/dosinst.c

Patch 6.2.371
问题: When 'virtualedit' is set and there is a Tab before the next "x", "dtx" does not delete the whole Tab. (Ken Hashishi)
解决方案: Move the cursor to the last position of the Tab. Also for "df<Tab>".
相关文件: src/normal.c

Patch 6.2.372
问题: When using balloon evaluation, no value is displayed for members of structures and items of an array.
解决方案: Include "->", "." and "[*]" in the expression.
相关文件: src/gui_beval.c, src/normal.c, src/vim.h

Patch 6.2.373
问题: When 'winminheight' is zero and a window is reduced to zero height, the ruler always says "Top" instead of the cursor position. (Antoine J. Mechelynck)
解决方案: Don't recompute w_topline for a zero-height window.
相关文件: src/window.c

Patch 6.2.374
问题: ":echo "hello" | silent normal n" removes the "hello" message. (Servatius Brandt)
解决方案: Don't echo the search string when ":silent" was used. Also don't show the mode. In general: don't clear to the end of the screen.
相关文件: src/gui.c, src/message.c, src/os_unix.c, src/proto/message.pro, src/screen.c, src/search.c, src/window.c

Patch 6.2.375
问题: When changing 'guioptions' the hit-enter prompt may be below the end of the Vim window.
解决方案: Call screen_alloc() before showing the prompt.
相关文件: src/message.c

Patch 6.2.376
问题: Win32: Ruby interface cannot be dynamically linked with Ruby 1.6.
解决方案: Add #ifdefs around use of rb_w32_snprintf(). (Benot Cerrina)
相关文件: src/if_ruby.c

Patch 6.2.377 (after 6.2.372)
问题: Compiler warnings for signed/unsigned compare. (Michael Wookey)
解决方案: Add type cast.
相关文件: src/normal.c

Patch 6.2.378 (extra, after 6.2.118)
问题: Mac: cannot build with Project Builder.
解决方案: Add remove_tail_with_ext() to locate and remove the "build" directory from the runtime path. Include os_unix.c when needed. (Dany St Amant)
相关文件: src/misc1.c, src/os_macosx.c, src/vim.h

Patch 6.2.379
问题: Using ":mkvimrc" in the ":options" window sets 'bufhidden' to

"delete". (Michael Naumann)
解决方案: Do not add buffer-specific option values to a global vimrc file.
相关文件: src/option.c

Patch 6.2.380 (extra)
问题: DOS: "make test" fails when running it again. Can't "make test" with Borland C.
解决方案: Make sure ".out" files are deleted when they get in the way. Add a "test" target to the Borland C Makefile.
相关文件: src/Make_bc5.mak, src/testdir/Make_dos.mak

Patch 6.2.381
问题: Setting 'fileencoding' to a comma separated list (confusing it with 'fileencodings') does not result in an error message. Setting 'fileencoding' in an empty file marks it as modified. There is no "+" in the title after setting 'fileencoding'.
解决方案: Check for a comma in 'fileencoding'. Only consider a non-empty file modified by changing 'fileencoding'. Update the title after changing 'fileencoding'.
相关文件: src/option.c

Patch 6.2.382
问题: Running "make test" puts marks from test files in viminfo.
解决方案: Specify a different viminfo file to use.
相关文件: src/testdir/test15.in, src/testdir/test49.in

Patch 6.2.383
问题: ":hi foo term='bla" crashes Vim. (Antony Scriven)
解决方案: Check that the closing ' is there.
相关文件: src/syntax.c

Patch 6.2.384
问题: ":menu a.&b" ":unmenu a.b" only works if "&b" isn't translated.
解决方案: Also compare the names without '&' characters.
相关文件: src/menu.c

Patch 6.2.385 (extra)
问题: Win32: forward_slash() and trash_input_buf() are undefined when compiling with small features. (Ajit Thakkar)
解决方案: Change the #ifdefs for forward_slash(). Don't call trash_input_buf() if the input buffer isn't used.
相关文件: src/fileio.c, src/os_win32.c

Patch 6.2.386
问题: Wasting time trying to read marks from the viminfo file for a buffer without a name.
解决方案: Skip reading marks when the buffer has no name.
相关文件: src/fileio.c

Patch 6.2.387
问题: There is no highlighting of translated items in help files.
解决方案: Search for a "help_ab.vim" syntax file when the help file is called "*.abx". Also improve the help highlighting a bit.
相关文件: runtime/syntax/help.vim

Patch 6.2.388
问题: GTK: When displaying some double-width characters they are drawn as single-width, because of conversion to UTF-8.
解决方案: Check the width that GTK uses and add a space if it's one instead of two.
相关文件: src/gui_gtk_x11.c

Patch 6.2.389
问题: When working over a slow connection, it's very annoying that the last line is partly drawn and then cleared for every change.
解决方案: Don't redraw the bottom line if no rows were inserted or deleted. Don't draw the line if we know "@" lines will be used.
相关文件: src/screen.c

Patch 6.2.390
问题: Using "r*" in Visual mode on multi-byte characters only replaces every other character. (Tyson Roberts)
解决方案: Correct the cursor position after replacing each character.
相关文件: src/ops.c

Patch 6.2.391 (extra)
问题: The ":highlight" command is not tested.
解决方案: Add a test script for ":highlight".
相关文件: src/testdir/Makefile, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/test51.in, src/testdir/test51.ok

Patch 6.2.392 (after 6.2.384)
问题: Unused variable.
解决方案: Remove "dlen".
相关文件: src/menu.c

Patch 6.2.393
问题: When using very long lines the viminfo file can become very big.
解决方案: Add the "s" flag to 'viminfo': skip registers with more than the specified Kbyte of text.
相关文件: runtime/doc/options.txt, src/ops.c, src/option.c

Patch 6.2.394 (after 6.2.391)
问题: Test 51 fails on a terminal with 8 colors. (Tony Leneis)
解决方案: Use "DarkBlue" instead of "Blue" to avoid the "bold" attribute.
相关文件: src/testdir/test51.in

Patch 6.2.395
问题: When using ":tag" or ":pop" the previous matching tag is used. But since the current file is different, the ordering of the tags may change.
解决方案: Remember what the current buffer was for when re-using cur_match.
相关文件: src/edit.c, src/ex_cmds.c, src/proto/tag.pro, src/structs.h, src/tag.c

Patch 6.2.396

问题:　　When CTRL-T jumps to another file and an autocommand moves the cursor to the '" mark, don't end up on the right line. (Michal Malecki)
解决方案:　Set the line number after loading the file.
相关文件:　src/tag.c

Patch 6.2.397
问题:　　When using a double-byte 'encoding' mapping <M-x> doesn't work. (Yasuhiro Matsumoto)
解决方案:　Do not set the 8th bit of the character but use a modifier.
相关文件:　src/gui_gtk_x11.c, src/gui_x11.c, src/misc2.c

Patch 6.2.398 (extra)
问题:　　Win32 console: no extra key modifiers are supported.
解决方案:　Encode the modifiers into the input stream. Also fix that special keys are converted and stop working when 'tenc' is set. Also fix that when 'tenc' is initialized the input and output conversion is not setup properly until 'enc' or 'tenc' is set.
相关文件:　src/getchar.c, src/option.c, src/os_win32.c

Patch 6.2.399
问题:　　A ":set" command that fails still writes a message when it is inside a try/catch block.
解决方案:　Include all the text of the message in the error message.
相关文件:　src/charset.c, src/option.c

Patch 6.2.400
问题:　　Can't compile if_xcmdsrv.c on HP-UX 11.0.
解决方案:　Include header file poll.h. (Malte Neumann)
相关文件:　src/if_xcmdsrv.c

Patch 6.2.401
问题:　　When opening a buffer that was previously opened, Vim does not restore the cursor position if the first line starts with white space. (Gregory Margo)
解决方案:　Don't skip restoring the cursor position if it is past the blanks in the first line.
相关文件:　src/buffer.c

Patch 6.2.402
问题:　　Mac: "make install" doesn't generate help tags. (Benji Fisher)
解决方案:　Generate help tags before copying the runtime files.
相关文件:　src/Makefile

Patch 6.2.403
问题:　　":@y" checks stdin if there are more commands to execute. This fails if stdin is not connected, e.g., when starting the GUI from KDE. (Ned Konz)
解决方案:　Only check for a next command if there still is typeahead.
相关文件:　src/ex_docmd.c

Patch 6.2.404
问题:　　Our own function to determine width of Unicode characters may get outdated. (Markus Kuhn)

解决方案: Use wcwidth() when it is available. Also use iswprint().
相关文件: src/auto/configure, src/configure.in, src/config.h.in, src/mbyte.c

Patch 6.2.405
问题: Cannot map zero without breaking the count before a command.
(Benji Fisher)
解决方案: Disable mapping zero when entering a count.
相关文件: src/getchar.c, src/globals.h, src/normal.c

Patch 6.2.406
问题: ":help \zs", ":help \@=" and similar don't find useful help.
解决方案: Prepend "/\" to the arguments to find the desired help tag.
相关文件: src/ex_cmds.c

Patch 6.2.407 (after 6.2.299)
问题: ":help \@<=" doesn't find help.
解决方案: Avoid that ":help \@<=" searches for the "<=" language.
相关文件: src/tag.c

Patch 6.2.408
问题: ":compiler" is not consistent: Sets local options and a global
variable. (Douglas Potts) There is no error message when a
compiler is not supported.
解决方案: Use ":compiler!" to set a compiler globally, otherwise it's local
to the buffer and "b:current_compiler" is used. Give an error
when no compiler script could be found.
Note: updated compiler plugins can be found at
ftp://ftp.vim.org/pub/vim/runtime/compiler/
相关文件: runtime/compiler/msvc.vim, runtime/doc/quickfix.txt, src/eval.c,
src/ex_cmds2.c

Patch 6.2.409
问题: The cursor ends up in the last column instead of after the line
when doing "i//<Esc>o" with 'indentexpr' set to "cindent(v:lnum)".
(Toby Allsopp)
解决方案: Adjust the cursor as if in Insert mode.
相关文件: src/misc1.c

Patch 6.2.410 (after 6.2.389)
问题: In diff mode, when there are more filler lines than fit in the
window, they are not drawn.
解决方案: Check for filler lines when skipping to draw a line that doesn't
fit.
相关文件: src/screen.c

Patch 6.2.411
问题: A "\n" inside a string is not seen as a line break by the regular
expression matching. (Hari Krishna Dara)
解决方案: Add the vim_regexec_nl() function for strings where "\n" is to be
matched with a line break.
相关文件: src/eval.c, src/ex_eval.c, src/proto/regexp.c, src/regexp.c

Patch 6.2.412
问题: Ruby: "ruby << EOF" inside a function doesn't always work. Also

for ":python", ":tcl" and ":perl".
解决方案: Check for "<< marker" and skip until "marker" before checking for "endfunction".
相关文件: src/eval.c

Patch 6.2.413 (after 6.2.411)
问题: Missing prototype for vim_regexec_nl(). (Marcel Svitalsky)
解决方案: Now really include the prototype.
相关文件: src/proto/regexp.pro

Patch 6.2.414
问题: The function used for custom completion of user commands cannot have <SID> to make it local. (Hari Krishna Dara)
解决方案: Pass the SID of the script where the user command was defined on to the completion. Also clean up #ifdefs.
相关文件: src/ex_docmd.c, src/eval.c, src/ex_getln.c, src/structs.h

Patch 6.2.415
问题: Vim may crash after a sequence of events that change the window size. The window layout assumes a larger window than is actually available. (Servatius Brandt)
解决方案: Invoke win_new_shellsize() from screenalloc() instead of from set_shellsize().
相关文件: src/screen.c, src/term.c

Patch 6.2.416
问题: Compiler warning for incompatible pointer.
解决方案: Remove the "&" in the call to poll(). (Xavier de Gaye)
相关文件: src/os_unix.c

Patch 6.2.417 (after 6.2.393)
问题: Many people forget that the '"' item in 'viminfo' needs to be preceded with a backslash,
解决方案: Add '<' as an alias for the '"' item.
相关文件: runtime/doc/options.txt, src/ops.c, src/option.c

Patch 6.2.418
问题: Using ":nnoremap <F12> :echo "cheese" and ":cabbr cheese xxx": when pressing <F12> still uses the abbreviation. (Hari Krishna)
解决方案: Also apply "noremap" to abbreviations.
相关文件: src/getchar.c

Patch 6.2.419 (extra)
问题: Win32: Cannot open the Vim window inside another application.
解决方案: Add the "-P" argument to specify the window title of the application to run inside. (Zibo Zhao)
相关文件: runtime/doc/starting.txt, src/main.c, src/gui_w32.c, src/gui_w48.c, src/if_ole.cpp, src/os_mswin.c, src/proto/gui_w32.pro

Patch 6.2.420
问题: Cannot specify a file to be edited in binary mode without setting the global value of the 'binary' option.
解决方案: Support ":edit ++bin file".

相关文件: runtime/doc/editing.txt, src/buffer.c, src/eval.c, src/ex_cmds.h, src/ex_docmd.c, src/fileio.c, src/misc2.c

Patch 6.2.421
问题: Cannot set the '[and '] mark, which may be necessary when an autocommand simulates reading a file.
解决方案: Allow using "m[" and "m]".
相关文件: runtime/doc/motion.txt, src/mark.c

Patch 6.2.422
问题: In CTRL-X completion messages the "/" makes them less readable.
解决方案: Remove the slashes. (Antony Scriven)
相关文件: src/edit.c

Patch 6.2.423
问题: ":vertical wincmd]" does not split vertically.
解决方案: Add "postponed_split_flags".
相关文件: src/ex_docmd.c, src/globals.h, src/if_cscope.c, src/tag.c

Patch 6.2.424
问题: A BufEnter autocommand that sets an option stops 'mousefocus' from working in Insert mode (Normal mode is OK). (Gregory Seidman)
解决方案: In the Insert mode loop invoke gui_mouse_correct() when needed.
相关文件: src/edit.c

Patch 6.2.425
问题: Vertical split and command line window: can only drag status line above the cmdline window on the righthand side, not lefthand side.
解决方案: Check the status line row instead of the window pointer.
相关文件: src/ui.c

Patch 6.2.426
问题: A syntax region end match with a matchgroup that includes a line break only highlights the last line with matchgroup. (Gary Holloway)
解决方案: Also use the line number of the position where the region highlighting ends.
相关文件: src/syntax.c

Patch 6.2.427 (extra)
问题: When pasting a lot of text in a multi-byte encoding, conversion from 'termencoding' to 'encoding' may fail for some characters. (Kuang-che Wu)
解决方案: When there is an incomplete byte sequence at the end of the read text keep it for the next time.
相关文件: src/mbyte.c, src/os_amiga.c, src/os_mswin.c, src/proto/mbyte.pro, src/proto/os_mswin.pro, src/ui.c

Patch 6.2.428
问题: The X11 clipboard supports the Vim selection for char/line/block mode, but since the encoding is not included can't copy/paste between two Vims with a different 'encoding'.
解决方案: Add a new selection format that includes the 'encoding'. Perform conversion when necessary.

相关文件: src/gui_gtk_x11.c, src/ui.c, src/vim.h

Patch 6.2.429
问题: Unix: glob() doesn't work for a directory with a single quote in the name. (Nazri Ramliy)
解决方案: When using the shell to expand, only put double quotes around spaces and single quotes, not the whole thing.
相关文件: src/os_unix.c

Patch 6.2.430
问题: BOM at start of a vim script file is not recognized and causes an error message.
解决方案: Detect the BOM and skip over it. Also fix that after using ":scriptencoding" the iconv() file descriptor was not closed (memory leak).
相关文件: src/ex_cmds2.c

Patch 6.2.431
问题: When using the horizontal scrollbar, the scrolling is limited to the length of the cursor line.
解决方案: Make the scroll limit depend on the longest visible line. The cursor is moved when necessary. Including the 'h' flag in 'guioptions' disables this.
相关文件: runtime/doc/gui.txt, runtime/doc/options.txt, src/gui.c, src/misc2.c, src/option.h

Patch 6.2.432 (after 6.2.430 and 6.2.431)
问题: Lint warnings.
解决方案: Add type casts.
相关文件: src/ex_cmds2.c, src/gui.c

Patch 6.2.433
问题: Translating "VISUAL" and "BLOCK" separately doesn't give a good result. (Alejandro Lopez Valencia)
解决方案: Use a string for each combination.
相关文件: src/screen.c

Patch 6.2.434 (after 6.2.431)
问题: Compiler warning. (Salman Halim)
解决方案: Add type casts.
相关文件: src/gui.c

Patch 6.2.435
问题: When there are vertically split windows the minimal Vim window height is computed wrong.
解决方案: Use frame_minheight() to correctly compute the minimal height.
相关文件: src/window.c

Patch 6.2.436
问题: Running the tests changes the user's viminfo file.
解决方案: In test 49 tell the extra Vim to use the test viminfo file.
相关文件: src/testdir/test49.vim

Patch 6.2.437

问题:　　":mksession" always puts "set nocompatible" in the session file. This changes option settings. (Ron Aaron)
解决方案:　Add an "if" to only change 'compatible' when needed.
相关文件:　src/ex_docmd.c

Patch 6.2.438
问题:　　When the 'v' flag is present in 'cpoptions', backspacing and then typing text again: one character too much is overtyped before inserting is done again.
解决方案:　Set "dollar_vcol" to the right column.
相关文件:　src/edit.c

Patch 6.2.439
问题:　　GTK 2: Changing 'lines' may cause a mismatch between the window layout and the size of the window.
解决方案:　Disable the hack with force_shell_resize_idle().
相关文件:　src/gui_gtk_x11.c

Patch 6.2.440
问题:　　When 'lazyredraw' is set the window title is still updated. The size of the Visual area and the ruler are displayed too often.
解决方案:　Postpone redrawing the window title. Only show the Visual area size when waiting for a character. Don't draw the ruler unnecessary.
相关文件:　src/buffer.c, src/normal.c, src/screen.c

Patch 6.2.441
问题:　　":unabbreviate foo " doesn't work, because of the trailing space, while an abbreviation with a trailing space is not possible. (Paul Jolly)
解决方案:　Accept a match with the lhs of an abbreviation without the trailing space.
相关文件:　src/getchar.c

Patch 6.2.442
问题:　　Cannot manipulate the command line from a function.
解决方案:　Add getcmdline(), getcmdpos() and setcmdpos() functions and the CTRL-\ e command.
相关文件:　runtime/doc/cmdline.txt, runtime/doc/eval.txt, src/eval.c src/ex_getln.c, src/ops.c, src/proto/ex_getln.pro, src/proto/ops.pro

Patch 6.2.443
问题:　　With ":silent! echoerr something" you don't get the position of the error. emsg() only writes the message itself and returns.
解决方案:　Also redirect the position of the error.
相关文件:　src/message.c

Patch 6.2.444
问题:　　When adding the 'c' flag to a ":substitute" command it may replace more times than without the 'c' flag. Happens for a match that starts with "\ze" (Marcel Svitalsk) and when using "\@<=" (Klaus Bosau).
解决方案:　Correct "prev_matchcol" when replacing the line. Don't replace

the line when the pattern uses look-behind matching.
相关文件: src/ex_cmds.c, src/proto/regexp.pro, src/regexp.c

Patch 6.2.445
问题: Copying vimtutor to /tmp/something is not secure, a symlink may cause trouble.
解决方案: Create a directory and create the file in it. Use "umask" to create the directory with mode 700. (Stefan Nordhausen)
相关文件: src/vimtutor

Patch 6.2.446 (after 6.2.404)
问题: Using library functions wcwidth() and iswprint() results in display problems for Hebrew characters. (Ron Aaron)
解决方案: Disable the code to use the library functions, use our own.
相关文件: src/mbyte.c

Patch 6.2.447 (after 6.2.440)
问题: Now that the title is only updated when redrawing, it is no longer possible to show it while executing a function. (Madoka Machitani)
解决方案: Make ":redraw" also update the title.
相关文件: src/ex_docmd.c

Patch 6.2.448 (after 6.2.427)
问题: Mac: conversion done when 'termencoding' differs from 'encoding' fails when pasting a longer text.
解决方案: Check for an incomplete sequence at the end of the chunk to be converted. (Eckehard Berns)
相关文件: src/mbyte.c

Patch 6.2.449 (after 6.2.431)
问题: Get error messages when switching files.
解决方案: Check for a valid line number when calculating the width of the horizontal scrollbar. (Helmut Stiegler)
相关文件: src/gui.c

Patch 6.2.450
问题: " #include" and " #define" are not recognized with the default option values for 'include' and 'defined'. (RG Kiran)
解决方案: Adjust the default values to allow white space before the #.
相关文件: runtime/doc/options.txt, src/option.c

Patch 6.2.451
问题: GTK: when using XIM there are various problems, including setting 'modified' and breaking undo at the wrong moment.
解决方案: Add "xim_changed_while_preediting", "preedit_end_col" and im_is_preediting(). (Yasuhiro Matsumoto)
相关文件: src/ex_getln.c, src/globals.h, src/gui_gtk.c, src/gui_gtk_x11.c, src/mbyte.c, src/misc1.c, src/proto/mbyte.pro, src/screen.c, src/undo.c

Patch 6.2.452
问题: In diff mode, when DiffAdd and DiffText highlight settings are equal, an added line is highlighted with DiffChange. (Tom Schumm)
解决方案: Remember the diff highlight type instead of the attributes.

相关文件: src/screen.c

Patch 6.2.453
问题: ":s/foo\|\nbar/x/g" does not replace two times in "foo\nbar". (Pavel Papushev)
解决方案: When the pattern can match a line break also try matching at the NUL at the end of a line.
相关文件: src/ex_cmds.c, src/regexp.c

Patch 6.2.454
问题: ":let b:changedtick" doesn't work. (Alan Schmitt) ":let b:changedtick = 99" does not give an error message.
解决方案: Add code to recognize ":let b:changedtick".
相关文件: src/eval.c

Patch 6.2.455 (after 6.2.297)
问题: In Python commands the current locale changes how certain Python functions work. (Eugene M. Minkovskii)
解决方案: Set the LC_NUMERIC locale to "C" while executing a Python command.
相关文件: src/if_python.c

Patch 6.2.456 (extra)
问题: Win32: Editing a file by its Unicode name (dropping it on Vim or using the file selection dialog) doesn't work. (Yakov Lerner, Alex Jakushev)
解决方案: Use wide character functions when file names are involved and convert from/to 'encoding' where needed.
相关文件: src/gui_w48.c, src/macros.h, src/memfile.c, src/memline.c, src/os_mswin.c, src/os_win32.c

Patch 6.2.457 (after 6.2.244)
问题: When 'encoding' is "utf-8" and writing text with chars above 0x80 in latin1, conversion is wrong every 8200 bytes. (Oyvind Holm)
解决方案: Correct the utf_ptr2len_check_len() function and fix the problem of displaying 0xf7 in utfc_ptr2len_check_len().
相关文件: src/mbyte.c

Patch 6.2.458
问题: When 'virtualedit' is set "$" doesn't move to the end of an unprintable character, causing "y$" not to include that character. (Fred Ma)
解决方案: Set "coladd" to move the cursor to the end of the character.
相关文件: src/misc2.c

Patch 6.2.459 (after 6.2.454)
问题: Variable "b" cannot be written. (Salman Halim)
解决方案: Compare strings properly.
相关文件: src/eval.c

Patch 6.2.460 (extra, after 6.2.456)
问题: Compiler warnings for missing prototypes.
解决方案: Include the missing prototypes.
相关文件: src/proto/os_win32.pro

Patch 6.2.461
问题: After using a search command "x" starts putting single characters in the numbered registers.
解决方案: Reset "use_reg_one" at the right moment.
相关文件: src/normal.c

Patch 6.2.462
问题: Finding a matching parenthesis does not correctly handle a backslash in a trailing byte.
解决方案: Handle multi-byte characters correctly. (Taro Muraoka)
相关文件: src/search.c

Patch 6.2.463 (extra)
问题: Win32: An NTFS file system may contain files with extra info streams. The current method to copy them creates one and then deletes it again. (Peter Toennies) Also, only three streams with hard coded names are copied.
解决方案: Use BackupRead() to check which info streams the original file contains and only copy these streams.
相关文件: src/os_win32.c

Patch 6.2.464 (extra, after 6.2.427)
问题: Amiga: Compilation error with gcc. (Ali Akcaagac)
解决方案: Move the #ifdef outside of Read().
相关文件: src/os_amiga.c

Patch 6.2.465
问题: When resizing the GUI window the window manager sometimes moves it left of or above the screen. (Michael McCarty)
解决方案: Check the window position after resizing it and move it onto the screen when it isn't.
相关文件: src/gui.c

Patch 6.2.466 (extra, after 6.2.456)
问题: Win32: Compiling with Borland C fails, and an un/signed warning.
解决方案: Redefine wcsicmp() to wcscmpi() and add type casts. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 6.2.467 (extra, after 6.2.463)
问题: Win32: can't compile without multi-byte feature. (Ajit Thakkar)
解决方案: Add #ifdefs around the info stream code.
相关文件: src/os_win32.c

Patch 6.2.468
问题: Compiler warnings for shadowed variables. (Matthias Mohr)
解决方案: Delete superfluous variables and rename others.
相关文件: src/eval.c, src/ex_docmd.c, src/ex_eval.c, src/if_cscope.c, src/fold.c, src/option.c, src/os_unix.c, src/quickfix.c, src/regexp.c

Patch 6.2.469 (extra, after 6.2.456)
问题: Win32: Can't create swap file when 'encoding' differs from the active code page. (Kriton Kyrimis)

解决方案: In enc_to_ucs2() terminate the converted string with a NUL
相关文件: src/os_mswin.c

Patch 6.2.470
问题: The name returned by tempname() may be equal to the file used for
shell output when ignoring case.
解决方案: Skip 'O' and 'I' in tempname().
相关文件: src/eval.c

Patch 6.2.471
问题: "-L/usr/lib" is used in the link command, even though it's
supposed to be filtered out. "-lw" and "-ldl" are not
automatically added when needed for "-lXmu". (Antonio Colombo)
解决方案: Check for a space after the argument instead of before. Also
remove "-R/usr/lib" if it's there. Check for "-lw" and "-ldl"
before trying "-lXmu".
相关文件: src/auto/configure, src/configure.in, src/link.sh

Patch 6.2.472
问题: When using a FileChangedShell autocommand that changes the current
buffer, a buffer exists that can't be wiped out.
Also, Vim sometimes crashes when executing an external command
that changes the buffer and a FileChangedShell autocommand is
used. (Hari Krishna Dara)
Users are confused by the warning for a file being changed outside
of Vim.
解决方案: Avoid that the window counter for a buffer is incremented twice.
Avoid that buf_check_timestamp() is used recursively.
Add a hint to look in the help for more info.
相关文件: src/ex_cmds.c, src/fileio.c

Patch 6.2.473
问题: Using CTRL-] in a help buffer without a name causes a crash.
解决方案: Check for name to be present before using it. (Taro Muraoka)
相关文件: src/tag.c

Patch 6.2.474 (extra, after 6.2.456)
问题: When Vim is starting up conversion is done unnecessarily. Failure
to find the runtime files on Windows 98. (Randall W. Morris)
解决方案: Init enc_codepage negative, only use it when not negative.
Don't use GetFileAttributesW() on Windows 98 or earlier.
相关文件: src/globals.h, src/gui_w32.c, src/gui_w48.c, src/os_mswin.c,
src/os_win32.c

Patch 6.2.475
问题: Commands after "perl <<EOF" are parsed as Vim commands when they
are not executed.
解决方案: Properly skip over the perl commands.
相关文件: src/ex_docmd.c, src/ex_getln.c, src/if_perl.xs, src/if_python.c,
src/if_ruby.c, src/if_tcl.c, src/misc2.c

Patch 6.2.476
问题: When reloading a hidden buffer changed outside of Vim and the
current buffer is read-only, the reloaded buffer becomes

read-only. (Hari Krishna Dara)
解决方案: Save the 'readonly' flag of the reloaded buffer instead of the current buffer.
相关文件: src/fileio.c

Patch 6.2.477
问题: Using remote_send(v:servername, "\<C-V>") causes Vim to hang. (Yakov Lerner)
解决方案: When the resulting string is empty don't set received_from_client.
相关文件: src/main.c

Patch 6.2.478
问题: Win32: "--remote file" fails changing directory if the current directory name starts with a single quote. (Iestyn Walters)
解决方案: Add a backslash where it will be removed later.
相关文件: src/main.c, src/misc2.c, src/proto/misc2.pro

Patch 6.2.479
问题: The error message for errors during recovery goes unnoticed.
解决方案: Avoid that the hit-enter prompt overwrites the message. Add a few lines to make the error stand out.
相关文件: src/main.c, src/message.c, src/memline.c

Patch 6.2.480
问题: NetBeans: Using negative index in array. backslash at end of message may cause Vim to crash. (Xavier de Gaye)
解决方案: Initialize buf_list_used to zero. Check for trailing backslash.
相关文件: src/netbeans.c

Patch 6.2.481
问题: When writing a file it is not possible to specify that hard and/or symlinks are to be broken instead of preserved.
解决方案: Add the "breaksymlink" and "breakhardlink" values to 'backupcopy'. (Simon Ekstrand)
相关文件: runtime/doc/options.txt, src/fileio.c, src/option.c, src/option.h

Patch 6.2.482
问题: Repeating insert of CTRL-K 1 S doesn't work. The superscript 1 is considered to be a digit. (Juergen Kraemer)
解决方案: In vim_isdigit() only accept '0' to '9'. Use VIM_ISDIGIT() for speed where possible. Also add vim_isxdigit().
相关文件: src/buffer.c, src/charset.c, src/diff.c, src/digraph.c, src/edit.c, src/eval.c,, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c, src/if_xcmdsrv.c, src/farsi.c, src/fileio.c, src/fold.c, src/getchar.c, src/gui.c, src/if_cscope.c, src/macros.h, src/main.c, src/mark.c, src/mbyte.c, src/menu.c, src/misc1.c, src/misc2.c, src/normal.c, src/ops.c, src/option.c, src/proto/charset.pro, src/regexp.c, src/screen.c, src/search.c, src/syntax.c, src/tag.c, src/term.c, src/termlib.c

Patch 6.2.483 (extra, after 6.2.482)
问题: See 6.2.482.
解决方案: Extra part of patch 6.2.482.

相关文件: src/gui_photon.c, src/gui_w48.c, src/os_msdos.c, src/os_mswin.c

Patch 6.2.484
问题: MS-Windows: With the included diff.exe, differences after a CTRL-Z are not recognized. (Peter Keresztes)
解决方案: Write the files with unix fileformat and invoke diff with --binary if possible.
相关文件: src/diff.c

Patch 6.2.485
问题: A BufWriteCmd autocommand cannot know if "!" was used or not. (Hari Krishna Dara)
解决方案: Add the v:cmdbang variable.
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/eval.pro, src/fileio.c, src/vim.h

Patch 6.2.486 (6.2.482)
问题: Diff for eval.c is missing.
解决方案: Addition to patch 6.2.482.
相关文件: src/eval.c

Patch 6.2.487 (extra, after 6.2.456)
问题: Compiler warnings for wrong prototype. (Alejandro Lopez Valencia)
解决方案: Delete the prototype for Handle_WM_Notify().
相关文件: src/proto/gui_w32.pro

Patch 6.2.488
问题: Missing ")" in *.ch filetype detection.
解决方案: Add the ")". (Ciaran McCreesh)
相关文件: runtime/filetype.vim

Patch 6.2.489
问题: When accidentally opening a session in Vim which has already been opened in another Vim there is a long row of ATTENTION prompts. Need to quit each of them to get out. (Robert Webb)
解决方案: Add the "Abort" alternative to the dialog.
相关文件: src/memline.c

Patch 6.2.490
问题: With 'paragraph' it is not possible to use a single dot as a paragraph boundary. (Dorai Sitaram)
解决方案: Allow using " " (two spaces) in 'paragraph' to match ".$" or ". $"
相关文件: src/search.c

Patch 6.2.491
问题: Decrementing a position doesn't take care of multi-byte chars.
解决方案: Adjust the column for multi-byte characters. Remove mb_dec(). (Yasuhiro Matsumoto)
相关文件: src/mbyte.c, src/misc2.c, src/proto/mbyte.pro

Patch 6.2.492
问题: When using ":redraw" while there is a message, the next ":echo" still causes text to scroll. (Yasuhiro Matsumoto)

解决方案: Reset msg_didout and msg_col, so that after ":redraw" the next message overwrites an existing one.
相关文件: src/ex_docmd.c

Patch 6.2.493
问题: "@x" doesn't work when 'insertmode' is set. (Benji Fisher)
解决方案: Put "restart_edit" in the typeahead buffer, so that it's used after executing the register contents.
相关文件: src/ops.c

Patch 6.2.494
问题: Using diff mode with two windows, when moving horizontally in inserted lines, a fold in the other window may open.
解决方案: Compute the line number in the other window correctly.
相关文件: src/diff.c

Patch 6.2.495 (extra, after 6.2.456)
问题: Win32: The file dialog doesn't work on Windows 95.
解决方案: Put the wide code of gui_mch_browse() in gui_mch_browseW() and use it only on Windows NT/2000/XP.
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 6.2.496
问题: FreeBSD 4.x: When compiled with the pthread library (Python) a complicated pattern may cause Vim to crash. Catching the signal doesn't work.
解决方案: When compiled with threads, instead of using the normal stacksize limit, use the size of the initial stack.
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/os_unix.c

Patch 6.2.497 (extra)
问题: Russian messages are only available in one encoding.
解决方案: Convert the messages to MS-Windows codepages. (Vassily Ragosin)
相关文件: src/po/Makefile

Patch 6.2.498
问题: Non-latin1 help files are not properly supported.
解决方案: Support utf-8 help files and convert them to 'encoding' when needed.
相关文件: src/fileio.c

Patch 6.2.499
问题: When writing a file and halting the system, the file might be lost when using a journaling file system.
解决方案: Use fsync() to flush the file data to disk after writing a file. (Radim Kolar)
相关文件: src/fileio.c

Patch 6.2.500 (extra)
问题: The DOS/MS-Windows the installer doesn't use the --binary flag for diff.
解决方案: Add --binary to the diff argument in MyDiff(). (Alejandro Lopez-Valencia)

相关文件: src/dosinst.c

Patch 6.2.501
问题: Vim does not compile with MorphOS.
解决方案: Add a Makefile and a few changes to make Vim work with MorphOS.
(Ali Akcaagac)
相关文件: runtime/doc/os_amiga.txt, src/INSTALLami.txt,
src/Make_morphos.mak, src/memfile.c, src/term.c

Patch 6.2.502
问题: Building fails for generating message files.
解决方案: Add dummy message files.
相关文件: src/po/ca.po, src/po/ru.po, src/po/sv.po

Patch 6.2.503
问题: Mac: Can't compile MacRoman conversions without the GUI.
解决方案: Also link with the Carbon framework for the terminal version, for
the MacRoman conversion functions. (Eckehard Berns)
Remove -ltermcap from the GUI link command, it is not needed.
相关文件: src/auto/configure, src/Makefile, src/configure.in

Patch 6.2.504
问题: Various problems with 'cindent', among which that a
list of variable declarations is not indented properly.
解决方案: Fix the wrong indenting. Improve indenting of C++ methods.
Add the 'i', 'b' and 'W' options to 'cinoptions'. (mostly by
Helmut Stiegler)
Improve indenting of preprocessor-continuation lines.
相关文件: runtime/doc/indent.txt, src/misc1.c, src/testdir/test3.in,
src/testdir/test3.ok

Patch 6.2.505
问题: Help for -P argument is missing. (Ronald Hoellwarth)
解决方案: Add the patch that was missing in 6.2.419.
相关文件: runtime/doc/starting.txt

Patch 6.2.506 (extra)
问题: Win32: When 'encoding' is a codepage then reading a utf-8 file
only works when iconv is available. Writing a file in another
codepage uses the wrong kind of conversion.
解决方案: Use internal conversion functions. Enable reading and writing
files with 'fileencoding' different from 'encoding' for all valid
codepages and utf-8 without the need for iconv.
相关文件: src/fileio.c, src/testdir/Make_dos.mak, src/testdir/test52.in,
src/testdir/test52.ok

Patch 6.2.507
问题: The ownership of the file with the password for the NetBeans
connection is not checked. "-nb={file}" doesn't work for GTK.
解决方案: Only accept the file when owned by the user and not accessible by
others. Detect "-nb=" for GTK.
相关文件: src/netbeans.c, src/gui_gtk_x11.c

Patch 6.2.508

问题:	Win32: "v:lang" does not show the current language for messages if it differs from the other locale settings.
解决方案:	Use the value of the $LC_MESSAGES environment variable.
相关文件:	src/ex_cmds2.c

Patch 6.2.509 (after 6.2.508)
问题:	Crash when $LANG is not set.
解决方案:	Add check for NULL pointer. (Ron Aaron)
相关文件:	src/ex_cmds2.c

Patch 6.2.510 (after 6.2.507)
问题:	Warning for pointer conversion.
解决方案:	Add a type cast.
相关文件:	src/gui_gtk_x11.c

Patch 6.2.511
问题:	Tags in Russian help files are in utf-8 encoding, which may be different from 'encoding'.
解决方案:	Use the "TAG_FILE_ENCODING" field in the tags file to specify the encoding of the tags. Convert help tags from 'encoding' to the tag file encoding when searching for matches, do the reverse when listing help tags.
相关文件:	runtime/doc/tagsrch.txt, src/ex_cmds.c, src/tag.c

Patch 6.2.512
问题:	Translating "\"\n" is useless. (Gerfried Fuchs)
解决方案:	Remove the _() around it.
相关文件:	src/main.c, src/memline.c

Patch 6.2.513 (after 6.2.507)
问题:	NetBeans: the check for owning the connection info file can be simplified. (Nikolay Molchanov)
解决方案:	Only check if the access mode is right.
相关文件:	src/netbeans.c

Patch 6.2.514
问题:	When a highlight/syntax group name contains invalid characters there is no warning.
解决方案:	Add an error for unprintable characters and a warning for other invalid characters.
相关文件:	src/syntax.c

Patch 6.2.515
问题:	When using the options window 'swapfile' is reset.
解决方案:	Use ":setlocal" instead of ":set".
相关文件:	runtime/optwin.vim

Patch 6.2.516
问题:	The sign column cannot be seen, looks like there are two spaces before the text. (Rob Retter)
解决方案:	Add the SignColumn highlight group.
相关文件:	runtime/doc/options.txt, runtime/doc/sign.txt, src/option.c, src/screen.c, src/syntax.c, src/vim.h

Patch 6.2.517
问题: Using "r*" in Visual mode on multi-byte characters replaces too many characters. In Visual Block mode replacing with a multi-byte character doesn't work.
解决方案: Adjust the operator end for the difference in byte length of the original and the replaced character. Insert all bytes of a multi-byte character, take care of double-wide characters.
相关文件: src/ops.c

Patch 6.2.518
问题: Last line of a window is not updated after using "J" and then "D". (Adri Verhoef)
解决方案: When no line is found below a change that doesn't need updating, update all lines below the change.
相关文件: src/screen.c

Patch 6.2.519
问题: Mac: cannot read/write files in MacRoman format.
解决方案: Do internal conversion from/to MacRoman to/from utf-8 and latin1. (Eckehard Berns)
相关文件: src/fileio.c

Patch 6.2.520 (extra)
问题: The NSIS installer is outdated.
解决方案: Make it work with NSIS 2.0. Also include console executables for Win 95/98/ME and Win NT/2000/XP. Use LZWA compression. Use "/oname" to avoid having to rename files before running NSIS.
相关文件: Makefile, nsis/gvim.nsi

Patch 6.2.521
问题: When using silent Ex mode the "changing a readonly file" warning is omitted but the one second wait isn't. (Yakov Lerner)
解决方案: Skip the delay when "silent_mode" is set.
相关文件: src/misc1.c

Patch 6.2.522
问题: GUI: when changing 'cmdheight' in the gvimrc file the window layout is messed up. (Keith Dart)
解决方案: Skip updating the window layout when changing 'cmdheight' while still starting up.
相关文件: src/option.c

Patch 6.2.523
问题: When loading a session and aborting when a swap file already exists, the user is left with useless windows. (Robert Webb)
解决方案: Load one file before creating the windows.
相关文件: src/ex_docmd.c

Patch 6.2.524 (extra, after 6.2.520)
问题: Win32: (un)installing gvimext.dll may fail if it was used.
The desktop and start menu links are created for the current user instead of all users.
Using the home directory as working directory for the links is a bad idea for multi-user systems.

Cannot use Vim from the "Open With..." menu.
解决方案: Force a reboot if necessary. (Alejandro Lopez-Valencia) Also use macros for the directory of the source and runtime files. Use "CSIDL_COMMON_*" instead of "CSIDL_*" when possible.
Do not specify a working directory in the links.
Add Vim to the "Open With..." menu. (Giuseppe Bilotta)
相关文件: nsis/gvim.nsi, src/dosinst.c, src/dosinst.h, src/uninstal.c

Patch 6.2.525
问题: When the history contains a very long line ":history" causes a crash. (Volker Kiefel)
解决方案: Shorten the history entry to fit it in one line.
相关文件: src/ex_getln.c

Patch 6.2.526
问题: When s:lang is "ja" the Japanese menus are not used.
解决方案: Add 'encoding' to the language when there is no charset.
相关文件: runtime/menu.vim

Patch 6.2.527
问题: The 2html script uses ":wincmd p", which breaks when using some autocommands.
解决方案: Remember the window numbers and jump to them with ":wincmd w". Also add XHTML support. (Panagiotis Issaris)
相关文件: runtime/syntax/2html.vim

Patch 6.2.528
问题: NetBeans: Changes of the "~" command are not reported.
解决方案: Call netbeans_inserted() after performing "~". (Gordon Prieur)
Also change NetBeans debugging to append to the log file.
Also fix that "~" in Visual block mode changes too much if there are multi-byte characters.
相关文件: src/nbdebug.c, src/normal.c, src/ops.c

Patch 6.2.529 (extra)
问题: VisVim only works for Admin. Doing it for one user doesn't work. (Alexandre Gouraud)
解决方案: When registering the module fails, simply continue.
相关文件: src/VisVim/VisVim.cpp

Patch 6.2.530
问题: Warning for missing prototype on the Amiga.
解决方案: Include time.h
相关文件: src/version.c

Patch 6.2.531
问题: In silent ex mode no messages are given, which makes debugging very difficult.
解决方案: Do output messages when 'verbose' is set.
相关文件: src/message.c, src/ui.c

Patch 6.2.532 (extra)
问题: Compiling for Win32s with VC 4.1 doesn't work.
解决方案: Don't use CP_UTF8 if it's not defined. Don't use CSIDL_COMMON*

when not defined.
相关文件: src/dosinst.h, src/fileio.c

Win32 console: After patch 6.2.398 Ex mode did not work. (Yasuhiro Matsumoto)

Patch 6.3a.001
问题: Win32: if testing for the "--binary" option fails, diff isn't used at all.
解决方案: Handle the "ok" flag properly. (Yasuhiro Matsumoto)
相关文件: src/diff.c

Patch 6.3a.002
问题: NetBeans: An insert command from NetBeans beyond the end of a buffer crashes Vim. (Xavier de Gaye)
解决方案: Use a local pos_T structure for the position.
相关文件: src/netbeans.c

Patch 6.3a.003
问题: E315 error with auto-formatting comments. (Henry Van Roessel)
解决方案: Pass the line number to same_leader().
相关文件: src/ops.c

Patch 6.3a.004
问题: Test32 fails on Windows XP for the DJGPP version. Renaming test11.out fails.
解决方案: Don't try renaming, create new files to use for the test.
相关文件: src/testdir/test32.in, src/testdir/test32.ok

Patch 6.3a.005
问题: ":checkpath!" does not use 'includeexpr'.
解决方案: Use a file name that was found directly. When a file was not found and the located name is empty, use the rest of the line.
相关文件: src/search.c

Patch 6.3a.006
问题: "yip" moves the cursor to the first yanked line, but not to the first column. Looks like not all text was yanked. (Jens Paulus)
解决方案: Move the cursor to the first column.
相关文件: src/search.c

Patch 6.3a.007
问题: 'cindent' recognizes "enum" but not "typedef enum".
解决方案: Skip over "typedef" before checking for "enum". (Helmut Stiegler) Also avoid that searching for this item goes too far back.
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 6.3a.008 (extra)
问题: Windows 98: Some of the wide functions are not implemented, resulting in file I/O to fail. This depends on what Unicode support is installed.
解决方案: Handle the failure and fall back to non-wide functions.
相关文件: src/os_win32.c

Patch 6.3a.009

问题:　　Win32: Completion of filenames does not work properly when 'encoding' differs from the active code page.
解决方案:　Use wide functions for expanding wildcards when appropriate.
相关文件:　src/misc1.c

Patch 6.3a.010 (extra)
问题:　　Win32: Characters in the window title that do not appear in the active codepage are replaced by a question mark.
解决方案:　Use DefWindowProcW() instead of DefWindowProc() when possible.
相关文件:　src/glbl_ime.cpp, src/globals.h, src/proto/gui_w16.pro, src/proto/gui_w32.pro, src/gui_w16.c, src/gui_w32.c, src/gui_w48.c

Patch 6.3a.011
问题:　　Using the explorer plugin changes a local directory to the global directory.
解决方案:　Don't use ":chdir" to restore the current directory. Make "expand('%:p')" remove "/../" and "/./" items from the path.
相关文件:　runtime/plugin/explorer.vim, src/eval.c, src/os_unix.c

Patch 6.3a.012 (extra)
问题:　　On Windows 98 the installer doesn't work, don't even get the "I agree" button. The check for the path ending in "vim" makes the browse dialog hard to use. The default path when no previous Vim is installed is "c:\vim" instead of "c:\Program Files\Vim".
解决方案:　Remove the background gradient command. Change the .onVerifyInstDir function to a leave function for the directory page. Don't let the install program default to c:\vim when no path could be found.
相关文件:　nsis/gvim.nsi, src/dosinst.c

Patch 6.3a.013 (extra)
问题:　　Win32: Characters in the menu that are not in the active codepage are garbled.
解决方案:　Convert menu strings from 'encoding' to the active codepage.
相关文件:　src/gui_w32.c, src/gui_w48.c

Patch 6.3a.014
问题:　　Using multi-byte text and highlighting in a statusline causes gaps to appear. (Helmut Stiegler)
解决方案:　Advance the column by text width instead of number of bytes. Add the vim_strnsize() function.
相关文件:　src/charset.c, src/proto/charset.pro, src/screen.c

Patch 6.3a.015
问题:　　Using the "select all" menu item when 'insertmode' is set and clicking the mouse button doesn't return to Insert mode. The Buffers/Delete menu doesn't offer a choice to abandon a changed buffer. (Jens Paulus)
解决方案:　Don't use CTRL-\ CTRL-N. Add ":confirm" for the Buffers menu items.
相关文件:　runtime/menu.vim

Patch 6.3a.016
问题:　　After cancelling the ":confirm" dialog the error message and

hit-enter prompt may not be displayed properly.
解决方案: Flush output after showing the dialog.
相关文件: src/message.c

Patch 6.3a.017
问题: servername() doesn't work when Vim was started with the "-X" argument or when the "exclude" in 'clipboard' matches the terminal name. (Robert Nowotniak)
解决方案: Force connecting to the X server when using client-server commands.
相关文件: src/eval.c, src/globals.h, src/os_unix.c

Patch 6.3a.018 (after 6.3a.017)
问题: Compiler warning for return value of make_connection().
解决方案: Use void return type.
相关文件: src/eval.c

Patch 6.3a.019 (extra)
问题: Win32: typing non-latin1 characters doesn't work.
解决方案: Invoke _OnChar() directly to avoid that the argument is truncated to a byte. Convert the UTF-16 character to bytes according to 'encoding' and ignore 'termencoding'. Same for _OnSysChar().
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 6.3a.020 (extra)
问题: Missing support for AROS (AmigaOS reimplementation). Amiga GUI doesn't work.
解决方案: Add AROS support. (Adam Chodorowski)
Fix Amiga GUI problems. (Georg Steger, Ali Akcaagac)
相关文件: Makefile, src/Make_aros.mak, src/gui_amiga.c, src/gui_amiga.h, src/memfile.c, src/os_amiga.c, src/term.c

Patch 6.3a.021 (after 6.3a.017)
问题: Can't compile with X11 but without GUI.
解决方案: Put use of "gui.in_use" inside an #ifdef.
相关文件: src/eval.c

Patch 6.3a.022
问题: When typing Tabs when 'softtabstop' is used and 'list' is set a tab is counted for two spaces.
解决方案: Use the "L" flag in 'cpoptions' to tell whether a tab is counted as two spaces or as 'tabstop'. (Antony Scriven)
相关文件: runtime/doc/options.txt, src/edit.c

Patch 6.3a.023
问题: Completion on the command line doesn't handle backslashes properly. Only the tail of matches is shown, even when not completing filenames.
解决方案: When turning the string into a pattern double backslashes. Don't omit the path when not expanding files or directories.
相关文件: src/ex_getln.c

Patch 6.3a.024
问题: The "save all" toolbar item fails for buffers that don't have a

name. When using ":wa" or closing the Vim window and there are nameless buffers, browsing for a name may cause the name being given to the wrong buffer or not stored properly. ":browse" only worked for one file.
解决方案: Use ":confirm browse" for "save all".
Pass buffer argument to setfname(). Restore "browse" flag and "forceit" after doing the work for one file.
相关文件: runtime/menu.vim, src/buffer.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/memline.c, src/message.c, src/window.c, src/proto/buffer.pro, src/proto/ex_cmds2.pro, src/proto/memline.pro

Patch 6.3a.025
问题: Setting 'virtualedit' moves the cursor. (Benji Fisher)
解决方案: Update the virtual column before using it.
相关文件: src/option.c

Patch 6.3a.026 (extra, after 6.3a.008)
问题: Editing files on Windows 98 doesn't work when 'encoding' is "utf-8" (Antoine Mechelynck)
Warning for missing function prototype.
解决方案: For all wide functions check if it failed because it is not implemented. Use ANSI function declaration for char_to_string().
相关文件: src/gui_w48.c, src/os_mswin.c, src/os_win32.c

Patch 6.3a.027 (extra, after 6.3a.026)
问题: Compiler warning for function argument.
解决方案: Declare both char and WCHAR arrays.
相关文件: src/gui_w48.c

Patch 6.3a.028
问题: ":normal ." doesn't work inside a function, because redo is saved and restored. (Benji Fisher)
解决方案: Make a copy of the redo buffer when executing a function.
相关文件: src/getchar.c

Patch 6.3b.001 (extra)
问题: Bcc 5: The generated auto/pathdef can't be compiled.
解决方案: Fix the way quotes and backslashes are escaped.
相关文件: src/Make_bc5.mak

Patch 6.3b.002
问题: Win32: conversion during file write fails when a double-byte character is split over two writes.
解决方案: Fix the conversion retry without a trailing byte. (Taro Muraoka)
相关文件: src/fileio.c

Patch 6.3b.003 (extra)
问题: Win32: When compiling with Borland C 5.5 and 'encoding' is "utf-8" then Vim can't open files under MS-Windows 98. (Antoine J. Mechelynck)
解决方案: Don't use _wstat(), _wopen() and _wfopen() in this situation.
相关文件: src/os_mswin.c, src/os_win32.c

Patch 6.3b.004
问题: ":helpgrep" includes a trailing CR in the text line.
解决方案: Remove the CR.
相关文件: src/quickfix.c

Patch 6.3b.005
问题: ":echo &g:ai" results in the local option value. (Salman Halim)
解决方案: Pass the flags from find_option_end() to get_option_value().
相关文件: src/eval.c

Patch 6.3b.006
问题: When using "mswin.vim", CTRL-V in Insert mode leaves cursor before last pasted character. (Mathew Davis)
解决方案: Use the same Paste() function as in menu.vim.
相关文件: runtime/mswin.vim

Patch 6.3b.007
问题: Session file doesn't restore view on windows properly. (Robert Webb)
解决方案: Restore window sizes both before and after restoring the view, so that the view, cursor position and size are restored properly.
相关文件: src/ex_docmd.c

Patch 6.3b.008
问题: Using ":finally" in a user command doesn't always work. (Hari Krishna Dara)
解决方案: Don't assume that using getexline() means the command was typed.
相关文件: src/ex_docmd.c

Patch 6.3b.009 (extra)
问题: Win32: When the -P argument is not found in a window title, there is no error message.
解决方案: When the window can't be found give an error message and exit. Also use try/except to catch failing to open the MDI window. (Michael Wookey)
相关文件: src/gui_w32.c

Patch 6.3b.010
问题: Win32: Using the "-D" argument and expanding arguments may cause a hang, because the terminal isn't initialized yet. (Vince Negri)
解决方案: Don't go into debug mode before the terminal is initialized.
相关文件: src/main.c

Patch 6.3b.011
问题: Using CTRL-\ e while obtaining an expression aborts the command line. (Hari Krishna Dara)
解决方案: Insert the CTRL-\ e as typed.
相关文件: src/ex_getln.c

Patch 6.3b.012 (after 6.3b.010)
问题: Can't compile with tiny features. (Norbert Tretkowski)
解决方案: Add #ifdefs.
相关文件: src/main.c

Patch 6.3b.013
问题: Loading a session file results in editing the wrong file in the first window when this is not the file at the current position in the argument list. (Robert Webb)
解决方案: Check w_arg_idx_invalid to decide whether to edit a file.
相关文件: src/ex_docmd.c

Patch 6.3b.014
问题: ":runtime! foo*.vim" may using freed memory when a sourced script changes the value of 'runtimepath'.
解决方案: Make a copy of 'runtimepath' when looping over the matches.
相关文件: src/ex_cmds2.c

Patch 6.3b.015
问题: Get lalloc(0) error when using "p" in Visual mode while 'clipboard' contains "autoselect,unnamed". (Mark Wagonner)
解决方案: Avoid allocating zero bytes. Obtain the clipboard when necessary.
相关文件: src/ops.c

Patch 6.3b.016
问题: When 'virtualedit' is used "x" doesn't delete the last character of a line that has as many characters as 'columns'. (Yakov Lerner)
解决方案: When the cursor isn't moved let oneright() return FAIL.
相关文件: src/edit.c

Patch 6.3b.017
问题: Win32: "vim --remote-wait" doesn't exit when the server finished editing the file. (David Fishburn)
解决方案: In the rrhelper plugin change backslashes to forward slashes and escape special characters.
相关文件: runtime/plugin/rrhelper.vim

Patch 6.3b.018
问题: The list of help files in the "local additions" table doesn't recognize utf-8 encoding. (Yasuhiro Matsumoto)
解决方案: Recognize utf-8 characters.
相关文件: src/ex_cmds.c

Patch 6.3b.019
问题: When $VIMRUNTIME is not a full path name the "local additions" table lists all the help files.
解决方案: Use fullpathcmp() instead of fnamecmp() to compare the directory names.
相关文件: src/ex_cmds.c

Patch 6.3b.020
问题: When using CTRL-^ when entering a search string, the item in the statusline that indicates the keymap is not updated. (Ilya Dogolazky)
解决方案: Mark the statuslines for updating.
相关文件: src/ex_getln.c

Patch 6.3b.021
问题: The swapfile is not readable for others, the ATTENTION prompt does

not show all info when someone else is editing the same file.
(Marcel Svitalsky)
解决方案: Use the protection of original file for the swapfile and set it after creating the swapfile.
相关文件: src/fileio.c

Patch 6.3b.022
问题: Using "4v" to select four times the old Visual area may put the cursor beyond the end of the line. (Jens Paulus)
解决方案: Correct the cursor column.
相关文件: src/normal.c

Patch 6.3b.023
问题: When "3dip" starts in an empty line, white lines after the non-white lines are not deleted. (Jens Paulus)
解决方案: Include the white lines.
相关文件: src/search.c

Patch 6.3b.024
问题: "2daw" does not delete leading white space like "daw" does. (Jens Paulus)
解决方案: Include the white space when a count is used.
相关文件: src/search.c

Patch 6.3b.025
问题: Percentage in ruler isn't updated when a line is deleted. (Jens Paulus)
解决方案: Check for a change in line count when deciding to update the ruler.
相关文件: src/screen.c, src/structs.h

Patch 6.3b.026
问题: When selecting "abort" at the ATTENTION prompt for a file that is already being edited Vim crashes.
解决方案: Don't abort creating a new buffer when we really need it.
相关文件: src/buffer.c, src/vim.h

Patch 6.3b.027
问题: Win32: When enabling the menu in a maximized window, Vim uses more lines than what is room for. (Shizhu Pan)
解决方案: When deciding to call shell_resized(), also compare the text area size with Rows and Columns, not just with screen_Rows and screen_Columns.
相关文件: src/gui.c

Patch 6.3b.028
问题: When in diff mode, setting 'rightleft' causes a crash. (Eddine)
解决方案: Check for last column differently when 'rightleft' is set.
相关文件: src/screen.c

Patch 6.3b.029
问题: Win32: warning for uninitialized variable.
解决方案: Initialize to zero.
相关文件: src/misc1.c

Patch 6.3b.030
问题: After Visually selecting four characters, changing it to other text, Visually selecting and yanking two characters: "." changes four characters, another "." changes two characters. (Robert Webb)
解决方案: Don't store the size of the Visual area when redo is active.
相关文件: src/normal.c

6.4 版 本 version-6.4

本小节讨论 6.3 和 6.4 版本之间的改进。

这是一个漏洞修正版。也有一些新特性。主要的新项目来自运行时文件和翻译。

MS-Windows 的 big 版本现在使用:
Ruby 1.8.3 版本
Perl 5.8.7 版本
Python 2.4.2 版本

改动 changed-6.4

删除了 runtime/tools/tcltags, Exuberant ctags 做得更好。

新增 added-6.4

Alsaconf 语法文件 (Nikolai Weibull)
Eruby 语法、缩进、编译器和文件类型插件文件 (Doug Kearns)
Esterel 语法文件 (Maurizio Tranchero)
Mathematica 缩进文件 (Steve Layland)
Netrc 语法文件 (Nikolai Weibull)
PHP 编译器文件 (Doug Kearns)
Pascal 缩进文件 (Neil Carter)
Prescribe 语法文件 (Klaus Muth)
Rubyunit 编译器文件 (Doug Kearns)
SMTPrc 语法文件 (Kornel Kielczewski)
Sudoers 语法文件 (Nikolai Weibull)
TPP 语法文件 (Gerfried Fuchs)
VHDL 文件类型插件文件 (R. Shankar)
Verilog-AMS 语法文件 (S. Myles Prather)

保加利亚键盘映射表 (Alberto Mardegan)
加拿大键盘映射表 (Eric Joanis)

匈牙利语菜单翻译, UTF-8 格式 (Kantra Gergely)
乌克兰语菜单翻译 (Bohdan Vlasyuk)

爱尔兰语消息翻译 (Kevin Patrick Scannell)

configure 也加入 tclsh8.4 的检查。

"dFxd;" deleted the character under the cursor, "d;" didn't remember the exclusiveness of the motion.

When using "set laststatus=2 cmdheight=2" in the .gvimrc you may only get one line for the cmdline. (Christian Robinson) Invoke command_height() after the GUI has started up.

Gcc would warn "dereferencing type-punned pointer will break strict -aliasing rules". Avoid using typecasts for variable pointers.

Gcc 3.x interprets the -MM argument differently. Change "-I /path" to "-isystem /path" for "make depend".

Patch 6.3.001
问题: ":browse split" gives the file selection dialog twice. (Gordon Bazeley) Same problem for ":browse diffpatch".
解决方案: Reset cmdmod.browse before calling do_ecmd().
相关文件: src/diff.c, src/ex_docmd.c

Patch 6.3.002
问题: When using translated help files with non-ASCII latin1 characters in the first line the utf-8 detection is wrong.
解决方案: Properly detect utf-8 characters. When a mix of encodings is detected continue with the next language and avoid a "no matches" error because of "got_int" being set. Add the directory name to the error message for a duplicate tag.
相关文件: src/ex_cmds.c

Patch 6.3.003
问题: Crash when using a console dialog and the first choice does not have a default button. (Darin Ohashi)
解决方案: Allocate two more characters for the [] around the character for the default choice.
相关文件: src/message.c

Patch 6.3.004
问题: When searching for a long string (140 chars in a 80 column terminal) get three hit-enter prompts. (Robert Webb)
解决方案: Avoid the hit-enter prompt when giving the message for wrapping around the end of the buffer. Don't give that message again when the string was not found.
相关文件: src/message.c, src/search.c

Patch 6.3.005
问题: Crash when searching for a pattern with a character offset and starting in a closed fold. (Frank Butler)
解决方案: Check for the column to be past the end of the line. Also fix that a pattern with a character offset relative to the end isn't read back from the viminfo properly.

相关文件: src/search.c

Patch 6.3.006
问题: ":breakadd file *foo" prepends the current directory to the file pattern. (Hari Krishna Dara)
解决方案: Keep the pattern as-is.
相关文件: src/ex_cmds2.c

Patch 6.3.007
问题: When there is a buffer with 'buftype' set to "nofile" and using a ":cd" command, the swap file is not deleted when exiting.
解决方案: Use the full path of the swap file also for "nofile" buffers.
相关文件: src/fileio.c

Patch 6.3.008
问题: Compiling fails under OS/2.
解决方案: Include "e_screenmode" also for OS/2. (David Sanders)
相关文件: src/globals.h

Patch 6.3.009 (after 6.3.006)
问题: ":breakadd file /path/foo.vim" does not match when a symbolic link is involved. (Servatius Brandt)
解决方案: Do expand the pattern when it does not start with "*".
相关文件: runtime/doc/repeat.txt, src/ex_cmds2.c

Patch 6.3.010
问题: When writing to a named pipe there is an error for fsync() failing.
解决方案: Ignore the fsync() error for devices.
相关文件: src/fileio.c

Patch 6.3.011
问题: Crash when the completion function of a user-command uses a "normal :cmd" command. (Hari Krishna Dara)
解决方案: Save the command line when invoking the completion function.
相关文件: src/ex_getln.c

Patch 6.3.012
问题: Internal lalloc(0) error when using a complicated multi-line pattern in a substitute command. (Luc Hermitte)
解决方案: Avoid going past the end of a line.
相关文件: src/ex_cmds.c

Patch 6.3.013
问题: Crash when editing a command line and typing CTRL-R = to evaluate a function that uses "normal :cmd". (Hari Krishna Dara)
解决方案: Save and restore the command line when evaluating an expression for CTRL-R =.
相关文件: src/ex_getln.c, src/ops.c, src/proto/ex_getln.pro, src/proto/ops.pro

Patch 6.3.014
问题: When using Chinese or Taiwanese the default for 'helplang' is wrong. (Simon Liang)

解决方案: Use the part of the locale name after "zh_".
相关文件: src/option.c

Patch 6.3.015
问题: The string that winrestcmd() returns may end in garbage.
解决方案: NUL-terminate the string. (Walter Briscoe)
相关文件: src/eval.c

Patch 6.3.016
问题: The default value for 'define' has "\s" before '#'.
解决方案: Add a star after "\s". (Herculano de Lima Einloft Neto)
相关文件: src/option.c

Patch 6.3.017
问题: "8zz" may leave the cursor beyond the end of the line. (Niko Maatjes)
解决方案: Correct the cursor column after moving to another line.
相关文件: src/normal.c

Patch 6.3.018
问题: ":0argadd zero" added the argument after the first one, instead of before it. (Adri Verhoef)
解决方案: Accept a zero range for ":argadd".
相关文件: src/ex_cmds.h

Patch 6.3.019
问题: Crash in startup for debug version. (David Rennals)
解决方案: Move the call to nbdebug_wait() to after allocating NameBuff.
相关文件: src/main.c

Patch 6.3.020
问题: When 'encoding' is "utf-8" and 'delcombine' is set, "dw" does not delete a word but only a combining character of the first character, if there is one. (Raphael Finkel)
解决方案: Correctly check that one character is being deleted.
相关文件: src/misc1.c

Patch 6.3.021
问题: When the last character of a file name is a multi-byte character and the last byte is a path separator, the file cannot be edited.
解决方案: Check for the last byte to be part of a multi-byte character. (Taro Muraoka)
相关文件: src/fileio.c

Patch 6.3.022 (extra)
问题: Win32: When the last character of a file name is a multi-byte character and the last byte is a path separator, the file cannot be written. A trail byte that is a space makes that a file cannot be opened from the command line.
解决方案: Recognize double-byte characters when parsing the command line. In mch_stat() check for the last byte to be part of a multi-byte character. (Taro Muraoka)
相关文件: src/gui_w48.c, src/os_mswin.c

Patch 6.3.023
问题: When the "to" part of a mapping starts with its "from" part, abbreviations for the same characters is not possible. For example, when <Space> is mapped to something that starts with a space, typing <Space> does not expand abbreviations.
解决方案: Only disable expanding abbreviations when a mapping is not remapped, don't disable it when the RHS of a mapping starts with the LHS.
相关文件: src/getchar.c, src/vim.h

Patch 6.3.024
问题: In a few places a string in allocated memory is not terminated with a NUL.
解决方案: Add ga_append(NUL) in script_get(), gui_do_findrepl() and serverGetVimNames().
相关文件: src/ex_getln.c, src/gui.c, src/if_xcmdsrv.c, src/os_mswin.c

Patch 6.3.025 (extra)
问题: Missing NUL for list of server names.
解决方案: Add ga_append(NUL) in serverGetVimNames().
相关文件: src/os_mswin.c

Patch 6.3.026
问题: When ~/.vim/after/syntax/syncolor.vim contains a command that reloads the colors an endless loop and/or a crash may occur.
解决方案: Only free the old value of an option when it was originally allocated. Limit recursiveness of init_highlight() to 5 levels.
相关文件: src/option.c, src/syntax.c

Patch 6.3.027
问题: VMS: Writing a file may insert extra CR characters. Not all terminals are recognized correctly. Vt320 doesn't support colors. Environment variables are not expanded correctly.
解决方案: Use another method to write files. Add vt320 termcap codes for colors. (Zoltan Arpadffy)
相关文件: src/fileio.c, src/misc1.c, src/os_unix.c, src/structs.h, src/term.c

Patch 6.3.028
问题: When appending to a file the BOM marker may be written. (Alex Jakushev)
解决方案: Do not write the BOM marker when appending.
相关文件: src/fileio.c

Patch 6.3.029
问题: Crash when inserting a line break. (Walter Briscoe)
解决方案: In the syntax highlighting code, don't use an old state after a change was made, current_col may be past the end of the line.
相关文件: src/syntax.c

Patch 6.3.030
问题: GTK 2: Crash when sourcing a script that deletes the menus, sets 'encoding' to "utf-8" and loads the menus again. GTK error message when tooltip text is in a wrong encoding.

解决方案: Don't copy characters from the old screen to the new screen when switching 'encoding' to utf-8, they may be invalid. Only set the tooltip when it is valid utf-8.
相关文件: src/gui_gtk.c, src/mbyte.c, src/proto/mbyte.pro, src/screen.c

Patch 6.3.031
问题: When entering a mapping and pressing Tab halfway the command line isn't redrawn properly. (Adri Verhoef)
解决方案: Reposition the cursor after drawing over the "..." of the completion attempt.
相关文件: src/ex_getln.c

Patch 6.3.032
问题: Using Python 2.3 with threads doesn't work properly.
解决方案: Release the lock after initialization.
相关文件: src/if_python.c

Patch 6.3.033
问题: When a mapping ends in a Normal mode command of more than one character Vim doesn't return to Insert mode.
解决方案: Check that the mapping has ended after obtaining all characters of the Normal mode command.
相关文件: src/normal.c

Patch 6.3.034
问题: VMS: crash when using ":help".
解决方案: Avoid using "tags-??", some Open VMS systems can't handle the "?" wildcard. (Zoltan Arpadffy)
相关文件: src/tag.c

Patch 6.3.035 (extra)
问题: RISC OS: Compile errors.
解决方案: Change e_screnmode to e_screenmode. Change the way __riscosify_control is set. Improve the makefile. (Andy Wingate)
相关文件: src/os_riscos.c, src/search.c, src/Make_ro.mak

Patch 6.3.036
问题: ml_get errors when the whole file is a fold, switching 'foldmethod' and doing "zj". (Christian J. Robinson) Was not deleting the fold but creating a fold with zero lines.
解决方案: Delete the fold properly.
相关文件: src/fold.c

Patch 6.3.037 (after 6.3.032)
问题: Warning for unused variable.
解决方案: Change the #ifdefs for the saved thread stuff.
相关文件: src/if_python.c

Patch 6.3.038 (extra)
问题: Win32: When the "file changed" dialog pops up after a click that gives gvim focus and not moving the mouse after that, the effect of the click may occur when moving the mouse later. (Ken Clark) Happened because the release event was missed.
解决方案: Clear the s_button_pending variable when any input is received.

相关文件: src/gui_w48.c

Patch 6.3.039
问题: When 'number' is set and inserting lines just above the first displayed line (in another window on the same buffer), the line numbers are not updated. (Hitier Sylvain)
解决方案: When 'number' is set and lines are inserted/deleted redraw all lines below the change.
相关文件: src/screen.c

Patch 6.3.040
问题: Error handling does not always work properly and may cause a buffer to be marked as if it's viewed in a window while it isn't. Also when selecting "Abort" at the attention prompt.
解决方案: Add enter_cleanup() and leave_cleanup() functions to move saving/restoring things for error handling to one place. Clear a buffer read error when it's unloaded.
相关文件: src/buffer.c, src/ex_docmd.c, src/ex_eval.c, src/proto/ex_eval.pro, src/structs.h, src/vim.h

Patch 6.3.041 (extra)
问题: Win32: When the path to a file has Russian characters, ":cd %:p:h" doesn't work. (Valery Kondakoff)
解决方案: Use a wide function to change directory.
相关文件: src/os_mswin.c

Patch 6.3.042
问题: When there is a closed fold at the top of the window, CTRL-X CTRL-E in Insert mode reduces the size of the fold instead of scrolling the text up. (Gautam)
解决方案: Scroll over the closed fold.
相关文件: src/move.c

Patch 6.3.043
问题: 'hlsearch' highlighting sometimes disappears when inserting text in PHP code with syntax highlighting. (Marcel Svitalsky)
解决方案: Don't use pointers to remember where a match was found, use an index. The pointers may become invalid when searching in other lines.
相关文件: src/screen.c

Patch 6.3.044 (extra)
问题: Mac: When 'linespace' is non-zero the Insert mode cursor leaves pixels behind. (Richard Sandilands)
解决方案: Erase the character cell before drawing the text when needed.
相关文件: src/gui_mac.c

Patch 6.3.045
问题: Unusual characters in an option value may cause unexpected behavior, especially for a modeline. (Ciaran McCreesh)
解决方案: Don't allow setting termcap options or 'printdevice' in a modeline. Don't list options for "termcap" and "all" in a modeline. Don't allow unusual characters in 'filetype', 'syntax',

'backupext', 'keymap', 'patchmode' and 'langmenu'.
相关文件: src/option.c, runtime/doc/options.txt

Patch 6.3.046
问题: ":registers" doesn't show multi-byte characters properly.
(Valery Kondakoff)
解决方案: Get the length of each character before displaying it.
相关文件: src/ops.c

Patch 6.3.047 (extra)
问题: Win32 with Borland C 5.5 on Windows XP: A new file is created with
read-only attributes. (Tony Mechelynck)
解决方案: Don't use the _wopen() function for Borland.
相关文件: src/os_win32.c

Patch 6.3.048 (extra)
问题: Build problems with VMS on IA64.
解决方案: Add dependencies to the build file. (Zoltan Arpadffy)
相关文件: src/Make_vms.mms

Patch 6.3.049 (after 6.3.045)
问题: Compiler warning for "char" vs "char_u" mixup. (Zoltan Arpadffy)
解决方案: Add a typecast.
相关文件: src/option.c

Patch 6.3.050
问题: When SIGHUP is received while busy exiting, non-reentrant
functions such as free() may cause a crash.
解决方案: Ignore SIGHUP when exiting because of an error. (Scott Anderson)
相关文件: src/misc1.c, src/main.c

Patch 6.3.051
问题: When 'wildmenu' is set and completed file names contain multi-byte
characters Vim may crash.
解决方案: Reserve room for multi-byte characters. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 6.3.052 (extra)
问题: Windows 98: typed keys that are not ASCII may not work properly.
For example with a Russian input method. (Jiri Jezdinsky)
解决方案: Assume that the characters arrive in the current codepage instead
of UCS-2. Perform conversion based on that.
相关文件: src/gui_w48.c

Patch 6.3.053
问题: Win32: ":loadview" cannot find a file with non-ASCII characters.
(Valerie Kondakoff)
解决方案: Use mch_open() instead of open() to open the file.
相关文件: src/ex_cmds2.c

Patch 6.3.054
问题: When 'insertmode' is set <C-L>4ixxx<C-L> hangs Vim. (Jens Paulus)
Vim is actually still working but redraw is disabled.
解决方案: When stopping Insert mode with CTRL-L don't put an Esc in the redo

buffer but a CTRL-L.
相关文件: src/edit.c

Patch 6.3.055 (after 6.3.013)
问题: Can't use getcmdline(), getcmdpos() or setcmdpos() with <C-R>= when editing a command line. Using <C-\>e may crash Vim. (Peter Winters)
解决方案: When moving ccline out of the way for recursive use, make it available to the functions that need it. Also save and restore ccline when calling get_expr_line(). Make ccline.cmdbuf NULL at the end of getcmdline().
相关文件: src/ex_getln.c

Patch 6.3.056
问题: The last characters of a multi-byte file name may not be displayed in the window title.
解决方案: Avoid to remove a multi-byte character where the last byte looks like a path separator character. (Yasuhiro Matsumoto)
相关文件: src/buffer.c, src/ex_getln.c

Patch 6.3.057
问题: When filtering lines folds are not updated. (Carl Osterwisch)
解决方案: Update folds for filtered lines.
相关文件: src/ex_cmds.c

Patch 6.3.058
问题: When 'foldcolumn' is equal to the window width and 'wrap' is on Vim may crash. Disabling the vertical split feature breaks compiling. (Peter Winters)
解决方案: Check for zero room for wrapped text. Make compiling without vertical splits possible.
相关文件: src/move.c, src/quickfix.c, src/screen.c, src/netbeans.c

Patch 6.3.059
问题: Crash when expanding an ":edit" command containing several spaces with the shell. (Brian Hirt)
解决方案: Allocate enough space for the quotes.
相关文件: src/os_unix.c

Patch 6.3.060
问题: Using CTRL-R CTRL-O in Insert mode with an invalid register name still causes something to be inserted.
解决方案: Check the register name for being valid.
相关文件: src/edit.c

Patch 6.3.061
问题: When editing a utf-8 file in an utf-8 xterm and there is a multi-byte character in the last column, displaying is messed up. (Jol Rio)
解决方案: Check for a multi-byte character, not a multi-column character.
相关文件: src/screen.c

Patch 6.3.062
问题: ":normal! gQ" hangs.

解决方案: Quit getcmdline() and do_exmode() when out of typeahead.
相关文件: src/ex_getln.c, src/ex_docmd.c

Patch 6.3.063
问题: When a CursorHold autocommand changes to another window
(temporarily) 'mousefocus' stops working.
解决方案: Call gui_mouse_correct() after triggering CursorHold.
相关文件: src/gui.c

Patch 6.3.064
问题: line2byte(line("$") + 1) sometimes returns the wrong number.
(Charles Campbell)
解决方案: Flush the cached line before counting the bytes.
相关文件: src/memline.c

Patch 6.3.065
问题: The euro digraph doesn't always work.
解决方案: Add an "e=" digraph for Unicode euro character and adjust the
help files.
相关文件: src/digraph.c, runtime/doc/digraph.txt

Patch 6.3.066
问题: Backup file may get wrong permissions.
解决方案: Use permissions of original file for backup file in more places.
相关文件: src/fileio.c

Patch 6.3.067 (after 6.3.066)
问题: Newly created file gets execute permission.
解决方案: Check for "perm" to be negative before using it.
相关文件: src/fileio.c

Patch 6.3.068
问题: When editing a compressed file xxx.gz which is a symbolic link to
the actual file a ":write" renames the link.
解决方案: Resolve the link, so that the actual file is renamed and
compressed.
相关文件: runtime/plugin/gzip.vim

Patch 6.3.069
问题: When converting text with illegal characters Vim may crash.
解决方案: Avoid that too much is subtracted from the length. (Da Woon Jung)
相关文件: src/mbyte.c

Patch 6.3.070
问题: After ":set number linebreak wrap" and a vertical split, moving
the vertical separator far left will crash Vim. (Georg Dahn)
解决方案: Avoid dividing by zero.
相关文件: src/charset.c

Patch 6.3.071
问题: The message for CTRL-X mode is still displayed after an error for
'thesaurus' or 'dictionary' being empty.
解决方案: Clear "edit_submode".
相关文件: src/edit.c

Patch 6.3.072
问题: Crash in giving substitute message when language is Chinese and encoding is utf-8. (Yongwei)
解决方案: Make the msg_buf size larger when using multi-byte.
相关文件: src/vim.h

Patch 6.3.073
问题: Win32 GUI: When the Vim window is partly above or below the screen, scrolling causes display errors when the taskbar is not on that side.
解决方案: Use the SW_INVALIDATE flag when the Vim window is partly below or above the screen.
相关文件: src/gui_w48.c

Patch 6.3.074
问题: When mswin.vim is used and 'insertmode' is set, typing text in Select mode and then using CTRL-V results in <SNR>99_Pastegi. (Georg Dahn)
解决方案: When restart_edit is set use "d" instead of "c" to remove the selected text to avoid calling edit() twice.
相关文件: src/normal.c

Patch 6.3.075
问题: After unloading another buffer, syntax highlighting in the current buffer may be wrong when it uses "containedin". (Eric Arnold)
解决方案: Use "buf" instead of "curbuf" in syntax_clear().
相关文件: src/syntax.c

Patch 6.3.076
问题: Crash when using cscope and there is a parse error (e.g., line too long). (Alexey I. Froloff)
解决方案: Pass the actual number of matches to cs_manage_matches() and correctly handle the error situation.
相关文件: src/if_cscope.c

Patch 6.3.077 (extra)
问题: VMS: First character input after ESC was not recognized.
解决方案: Added TRM$M_TM_TIMED in vms_read(). (Zoltan Arpadffy)
相关文件: src/os_vms.c

Patch 6.3.078 (extra, after 6.3.077)
问题: VMS: Performance issue after patch 6.3.077
解决方案: Add a timeout in the itemlist. (Zoltan Arpadffy)
相关文件: src/os_vms.c

Patch 6.3.079
问题: Crash when executing a command in the command line window while syntax highlighting is enabled. (Pero Brbora)
解决方案: Don't use a pointer to a buffer that has been deleted.
相关文件: src/syntax.c

Patch 6.3.080 (extra)
问题: Win32: With 'encoding' set to utf-8 while the current codepage is

Chinese editing a file with some specific characters in the name
fails.
解决方案: Use _wfullpath() instead of _fullpath() when necessary.
相关文件: src/os_mswin.c

Patch 6.3.081
问题: Unix: glob() may execute a shell command when it's not wanted.
(Georgi Guninski)
解决方案: Verify the sandbox flag is not set.
相关文件: src/os_unix.c

Patch 6.3.082 (after 6.3.081)
问题: Unix: expand() may execute a shell command when it's not wanted.
(Georgi Guninski)
解决方案: A more generic solution than 6.3.081.
相关文件: src/os_unix.c

Patch 6.3.083
问题: VMS: The vt320 termcap entry is incomplete.
解决方案: Add missing function keys. (Zoltan Arpadffy)
相关文件: src/term.c

Patch 6.3.084 (extra)
问题: Cygwin: compiling with DEBUG doesn't work. Perl path was ignored.
Failure when $(OUTDIR) already exists. "po" makefile is missing.
解决方案: Use changes tested in Vim 7. (Tony Mechelynck)
相关文件: src/Make_cyg.mak, src/po/Make_cyg.mak

Patch 6.3.085
问题: Crash in syntax highlighting code. (Marc Espie)
解决方案: Prevent current_col going past the end of the line.
相关文件: src/syntax.c

Patch 6.3.086 (extra)
问题: Can't produce message translation file with msgfmt that checks
printf strings.
解决方案: Fix the Russian translation.
相关文件: src/po/ru.po, src/po/ru.cp1251.po

Patch 6.3.087
问题: MS-DOS: Crash. (Jason Hood)
解决方案: Don't call fname_case() with a NULL pointer.
相关文件: src/ex_cmds.c

Patch 6.3.088
问题: Editing ".in" causes error E218. (Stefan Karlsson)
解决方案: Require some characters before ".in". Same for ".orig" and others.
相关文件: runtime/filetype.vim

Patch 6.3.089
问题: A session file doesn't work when created while the current
directory contains a space or the directory of the session files
contains a space. (Paolo Giarrusso)
解决方案: Escape spaces with a backslash.

相关文件: src/ex_docmd.c

Patch 6.3.090
问题: A very big value for 'columns' or 'lines' may cause a crash.
解决方案: Limit the values to 10000 and 1000.
相关文件: src/option.c

Patch 6.4a.001
问题: The Unix Makefile contained too many dependencies and a few
uncommented lines.
解决方案: Run "make depend" with manual changes to avoid a gcc
incompatibility. Comment a few lines.
相关文件: src/Makefile

Patch 6.4b.001
问题: Vim reports "Vim 6.4a" in the ":version" output.
解决方案: Change "a" to "b". (Tony Mechelynck)
相关文件: src/version.h

Patch 6.4b.002
问题: In Insert mode, pasting a multi-byte character after the end of
the line leaves the cursor just before that character.
解决方案: Make sure "gP" leaves the cursor in the right place when
'virtualedit' is set.
相关文件: src/ops.c

Patch 6.4b.003 (after 6.4b.002)
问题: The problem still exists when 'encoding' is set to "cp936".
解决方案: Fix the problem in getvvcol(), compute the coladd field correctly.
相关文件: src/charset.c, src/ops.c

Patch 6.4b.004
问题: Selecting a {} block with "viB" includes the '}' when there is an
empty line before it.
解决方案: Don't advance the cursor to include a line break when it's already
at the line break.
相关文件: src/search.c

vim:tw=78:ts=8:noet:ft=help:norl:

version7.txt For Vim version 8.2. 最近更新: 2017年8月

VIM 参考手册 by Bram Moolenaar

译者: Willis
http://vimcdoc.sf.net

vim7 version-7.0 version7.0

欢迎来到 Vim 版本 7! 新版本增加了很多新特性。本文档列出所有加入的新项目、已有特性的改动以及 Vim 6.x 以来的漏洞修正。使用这个命令可以看到你使用的版本:

```
:version
```

vi_diff.txt 提供 Vi 和 Vim 7.0 的区别的一个概述。
version4.txt 说明 Vim 3.x 和 Vim 4.x 的区别。
version5.txt 说明 Vim 4.x 和 Vim 5.x 的区别。
version6.txt 说明 Vim 5.x 和 Vim 6.x 的区别。

不兼容的改动 incompatible-7

新特性 new-7

Vim 脚本增强 new-vim-script
拼写检查 new-spell
全能补全 new-omni-completion
MzScheme 接口 new-MzScheme
打印多字节文本 new-print-multi-byte
标签页 new-tab-pages
撤销分支 new-undo-branches
扩展的 Unicode 支持 new-more-unicode
更多高亮 new-more-highlighting
翻译的手册页 new-manpage-trans
内部 grep new-vimgrep
消息往回滚动 new-scroll-back
光标超越行尾 new-onemore
POSIX 兼容性 new-posix
调试器支持 new-debug-support
远程文件探索器 new-netrw-explore
定义操作符 new-define-operator
映射到表达式 new-map-expression
可视和选择模式映射 new-map-select
位置列表 new-location-list
若干新项目 new-items-7

改进 improvements-7

编译时的改变 compile-changes-7

漏洞修正 bug-fixes-7

7.1 版本 version-7.1
改动 changed-7.1
新增 added-7.1
修正 fixed-7.1

7.2 版本 version-7.2
改动 changed-7.2
新增 added-7.2
修正 fixed-7.2

7.3 版本 version-7.3

持久化撤销 new-persistent-undo
更多加密 new-more-encryption
隐藏文本 new-conceal
Lua 接口 new-lua
Python3 接口 new-python3

改动 changed-7.3
新增 added-7.3
修正 fixed-7.3

7.4 版本 version-7.4
新正规表达式引擎 new-regexp-engine
更好的 Python 接口 better-python-interface
改动 changed-7.4
新增 added-7.4
修正 fixed-7.4

(译者注：各版本的修正一节，因内容过于琐碎，保留原文未翻译)

不兼容的改动 incompatible-7

这些改动和以前版本不兼容。如果从 Vim 6.x 升级到 7.0 时出现问题，检查这个列表。

":write file" 命令不再复位缓冲区的 'modified' 标志位，除非 'cpoptions' 里包含 '+' 标志位 cpo-+ 。本来的行为不合逻辑，因为缓冲区和原来的文件相比，仍然有修改的内容。另外，撤销所有改变时，文件仍然会被标记为修改状态。现在，执行此命令后用 ":quit" 会失败。

":helpgrep" 现在使用帮助窗口来显示匹配。

参数列表里本来可用双引号在文件名里包含空格。这使得 ":edit" 和 ":next" 转义双引号时的行为不同，而且和 Vi 有些版本也不兼容。

命令	Vim 6.x 文件名	Vim 7.x 文件名
:edit foo\"888	foo"888	foo"888
:next foo\"888	foo888	foo"888
:next a\"b c\"d	ab cd	a"b 和 c"d

在 literal-string 里，单引号加倍可以得到一个。":echo 'a''b'" 本来返回 "a b"，现在因为两个引号代表一个，它返回 "a'b"。

用 ":w! fname" 覆盖文件时，本来不会警告 "fname" 正在被另一个 Vim 编辑。Vim 现在给出错误信息 E768 。

删除了 Mac OS 9 的支持。

.tex 结尾的文件现在设置 'filetype' 为 "context"、"plaintex" 或 "tex"。
ft-tex-plugin

次要不兼容点：

文件类型检测：许多类型用 */.dir/filename 代替了 ~/.dir/filename，从而也能使用别的用户的文件。

不少文件类型的缩进设置从文件类型插件移到了缩进插件。如果你用：
```
:filetype plugin on
```
有些缩进设置可能会丢失。你需要用：
```
:filetype plugin indent on
```

":0verbose" 现在设置 'verbose' 为零而不是一。

删除了旧且不完整的 "VimBuddy" 代码。

无名缓冲区报告 "No Name" 而不是 "No File"。过去这和有名字但 'buftype' 设为 "nofile" 的缓冲区容易混淆。

如果 ":file xxx" 用于无名缓冲区，不设置轮换文件名。这避免了没有意义的另一个无名缓冲区的建立。

"2html.vim" 脚本现在把关闭折叠也转换到 HTML。因为折叠的打开关闭相同，HTML 看上去和显示接近。如果 HTML 里不要折叠，用 "zR" 或 "let html_ignore_folding=1"。(部分来自 Carl Osterwisch)
比较模式也被转换到 HTML，和显示一致。

Win32: <F10> 键的效果过去取决于 'winaltkeys'。现在它取决于 <F10> 是否被映射。这样就可以映射 <F10> 而不修改 'winaltkeys'。

如果 'nrformats' 包含 'octal'，"08" 上用 CTRL-A 过去得到 "018"，这不合逻辑。现在得到 "9"。同时也删除引导的一到多个零，以免 "009" 递增到 "010" 后该数变成八进制。

如果 'encoding' 设为一个 Unicode 编码，'fileencodings' 的值现在在 "latin1" 之前包含 "default"。这意味着如有可能，8 位编码的文件缺省使用环境指定的编码。以前总是使用 latin1，在非 latin1 的环境下这是错的，俄语就是如此。

如果有两个窗口存在且都显示帮助文件时，过去用 ":quit" 会退出。现在只关闭窗口。

"-w {scriptout}" 只有当 {scriptout} 不以数位开头时才可用。否则，用它设置 'window' 选项。

<Home> 和 <xHome> 过去可以分别映射。这样做的缺点是所有映射（加上修饰符）不得不都重复一遍，因为你不能确信键盘会产生哪个值。现在所有的 <xHome> 在内部都被翻译为 <Home>，键码和映射都是。同样的改动也适用于 <xEnd>、<xF1> 等键。

":put" 现在把光标放在插入的末行。

如果 .gvimrc 存在，关闭 'compatible'，就像 ".vimrc" 文件存在时一样。

用 "vlllU" 或类似的命令使字符串大写时，德语 sharp s 被替换为 "SS"。因为兼容性

问题，"~" 不这么做，这也是因为 "SS" 不能被改回 sharp s。

"gd" 过去在函数里找到变量的第一次出现，而这可以是没有类型的函数参数。现在只会去找包含类型的位置。

函数里的后继行过去不参加计算，错误信息的行号是逻辑行而不是源代码里的行。这使得定位错误十分困难。现在包含函数开始处起算的源代码行号。这也意味着 ":breakadd func" 里的行号含义有了变化。

用 :command 定义用户命令时，过去特殊项目可以是缩写。这导致不可预测的行为，如 <li> 会被识别为 <line1>。现在这些项目不再允许缩写。

执行 FileChangedRO 自动命令时，不再允许切换到另一个缓冲区或者编辑另一个文件。这是为了防止崩溃（激活事件所在的代码埋藏得很深，那里我们不期待改变缓冲区）。但重新载入缓冲区仍然允许。

more-prompt 和 hit-enter-prompt 提示上，如果置位 'more' 选项，现在可以使用 'k'、'u'、'g' 和 'b' 键来往回滚动到以前的消息。它们不再用作预输入。

新 特 性 new-7

Vim 脚本增强 new-vim-script

Vim 脚本加入以下类型：

List	项目的有序列表
Dictionary	项目的关联数组
Funcref	指向函数的引用

加入许多函数和命令以支持这些新类型。

用 string() 函数可以得到变量的字符串表示形式。这包括数值、字符串和它们的复合形式。用 eval() 又可把字符串转换变量值。

:let 命令现在可用 "+=", "-=" 和 ".="。

```
:let var += expr        " 相当于 :let var = var + expr
:let var -= expr        " 相当于 :let var = var - expr
:let var .= string      " 相当于 :let var = var . string
```

用 :profile 命令，你可以确定哪个函数或脚本最耗时。

Python 接口 vim.eval() 也能处理字典和列表。 python-eval (G. Sumner Hayes)

加入 getscript 插件，提供从 www.vim.org 自动更新脚本的便捷方法。(Charles Campbell)

加入 vimball 插件，提供发布包含一组文件（插件文件、autoload 脚本、文档）的插件的便捷方法。(Charles Campbell)

拼写检查 new-spell

Vim 已经集成了拼写检查。本来有一些脚本的实现，但都较慢和/或需要外部程序。

用 'spell'　　选项切换拼写检查的开关。
用 'spelllang'　选项指定能接受的语言。
用 'spellfile'　选项指定加入新词的文件名。
用 'spellsuggest' 选项指定提供建议的方法。

用]s 和 [s 命令移动到下一个或前一个错误。
用 zg 和 zw 命令加入好词和坏词。
用 z=　　命令改正单词。
用 :mkspell　命令从单词列表生成 Vim 的拼写文件。

加入 "undercurl" 高亮属性，GUI 里可以清楚地指出拼写错误（基于 Marcin Dalecki 的补丁）。
用 "guisp" 颜色指定和前景背景不同的第三种颜色。
可用高亮属性的数目从大约 220 个增加到超过 30000 个。这使拼写的属性和语法高亮的属性可以进行合并。也可用于语法高亮和标注可视区域。

详情可见： spell 。

全能补全　　new-omni-completion

本来可以叫 "intellisense" 的，但它已经是注册商标了。这是一种智能的补全形式，它检视光标前面的文本，然后寻找后面可以跟什么内容。它可以用来提供结构和类的成员，系统函数，等等。
(译者注：原文为 omni completion，但 omni 具体何义并无解释。暂按其字面意思直译为 "全能"。)

插入模式下用 CTRL-X CTRL-O 开始全能补全。 i_CTRL-X_CTRL-O

文件类型插件设置 'omnifunc' 选项来定义进行补全的函数。

目前支持的语言：

C	ft-c-omni
(X)HTML，带有 CSS	ft-html-omni
JavaScript	ft-javascript-omni
PHP	ft-php-omni
Python	
Ruby	ft-ruby-omni
SQL	ft-sql-omni
XML	ft-xml-omni
任何支持语法高亮的语言	ft-syntax-omni

你可以加入自己的全能补全脚本。

如果 'completeopt' 选项包含 "menu"，使用（相当原始的）弹出菜单显示插入模式的补全匹配。

MzScheme 接口　　new-MzScheme

支持 MzScheme 解释器。 MzScheme
用 :mzscheme 命令执行 MzScheme 命令。
用 :mzfile 命令执行 MzScheme 脚本文件。

这需要 Vim 编译时加入 +mzscheme 特性。

打印多字节文本 new-print-multi-byte

:hardcopy 命令现在支持多字节字符的 PostScript 打印。

为此可使用 'printmbcharset' 和 'printmbfont' 选项。
另见 postscript-cjk-printing 。(Mike Williams)

标签页 new-tab-pages

标签页 (tab page) 是包含一或多个窗口并在顶部贴有标签 (tab) 的页面 (page)。点击标签可以快速在标签页间切换。键盘则可用 gt (转到标签页, Goto Tab) 命令。这是一个需要使用很多窗口工作的便捷方式。

要在启动 Vim 时为每个文件参数打开一个单独的标签页, 使用 -p 参数。
用 'tabpagemax' 设置可打开的最大标签页数。

标签页标签所在的行可以使用高亮的普通文本, 也可以使用 GUI 的机制。GUI 标签看起来更舒服, 但只有少数系统支持。用 'tabline'、'guitablabel' 和 'guitabtooltip' 可以定制标签页行, 而它显示与否则由 'showtabline' 控制。至于是否使用 GUI 标签由 'guioptions' 的 "e" 标志位决定。

用 :tab 命令修饰符, 多数打开新窗口的命令会用新标签页代替。

--remote-tab 参数使已运行的 Vim 服务器用新标签页编辑文件。

"t:" 开始的变量局部于标签页。

详情见: tabpage
多数 GUI 部分由 Yegappan Lakshmanan 实现。

撤销分支 new-undo-branches

以前只有单线的撤销-重做。如果撤销一系列改变后进行了一项新改变, 所有撤销的改变就全丢失了。这可能会不小心导致大量工作的丢失。

为此情形, Vim 现在实现了撤销分支机制, 使你可以回到任何改变之后的文本, 即使曾经撤销过也没问题。唯一的例外是你到达了 'undolevels' 的限制, 这时需要释放撤销信息以减少内存用量。

要能在撤销分支间游历, 按顺序给每个改变编号。命令 g- 和 :earlier 在时间上往后退到旧的改变上。而命令 g+ 和 :later 在时间上往前进到新的改变上。

每个改变同时记录时间。":earlier 10m" 回到大约十分钟前的文本。

用 :undolist 命令了解存在的撤销分支的大概情况。 :undo 命令现在接受参数以直接跳转到该列表的任意位置。用 changenr() 函数得到改变号。

现在还没有图形化的改变树显示，导航机制可能不甚简明。

扩展的 Unicode 支持 new-more-unicode

以前只能显示两个组合字符。现在限制最高提高到 6 个。用 'maxcombine' 选项可以设置这个限制。缺省值仍然是 2 个。

ga 现在显示所有的组合字符，而不只是前面两个。

以前只支持 16 位 Unicode 字符的显示，现在可用完整的 32 位字符集。除非编译时手动关闭这项功能以减少一些内存使用。

模式匹配现在可以搜索单个组合字符。 patterns-composing

8g8 命令搜索非法的 UTF-8 字节序列。

更多高亮 new-more-highlighting

高亮匹配的括号：

在文本上移动时光标如果在括号上，同时高亮匹配的另一个括号。这使用了新的 CursorMoved 自动命令事件。

这意味着每次移动光标都会执行一些命令。如果对你来说速度减慢太多，可以关闭：

```
:NoMatchParen
```

matchparen 提供更多信息。

该插件使用 :match 命令。现在该命令同时支持三个匹配模式。插件使用的是第三个。第一个给用户使用，第二个可以用于其它插件。

高亮光标行和列：

加入 'cursorline' 和 'cursorcolumn' 选项。它们高亮光标所在的屏幕行和屏幕列。这阳方便定位光标位置。'cursorcolumn' 也可用于对齐文本。它们可能会使屏幕刷新慢很多。它们使用 CursorColumn 和 CursorLine 高亮，你可以修改使用的颜色。
hl-CursorColumn hl-CursorLine

可用高亮属性的数目从大约 220 个增加到超过 30000 个。这使拼写的属性和语法高亮的属性可以进行合并。也可用于语法高亮，标注可视区域，CursorColumn 等等。

翻译的手册页 new-manpage-trans

现在 Vim 有了若干其它语言的手册页和相关程序。

法语　　- David Blanchet 翻译
意大利语 - Antonio Colombo 翻译
俄语　　- Vassily Ragosin 翻译
波兰语　- Mikolaj Machowski 翻译

Unix 的 Makefile 在 .../man/it/man1/，.../man/it.ISO8859-1/man1/ 和 .../man/it.UTF-8/man1/ 安装意大利语的手册页。看来没有关于 "it" 目录使用什么编码的公认标准。作为我们的最佳猜测，目前使用的是 8 位编码文件。
其它语言也被安装在类似的位置。
Vim configure 时如果使用 "--disable-nls"，不自动安装翻译页。但仍然可用 "make install-languages install-tool-languages" 完成这项工作。

内部 grep　　new-vimgrep

用 ":vimgrep" 命令可以在一组文件里搜索模式。和 ":grep" 命令类似，但它不需要外部程序。除了移植性更好之外，它还能处理不同的文件编码并使用多行模式。它也能用来在压缩和远程文件上进行搜索。 :vimgrep 。

如果想在脚本里使用搜索的结果，用 getqflist() 函数。

要 grep 多个目录下的文件，可用 "**" 模式。它会扩展为任意深度的目录。任何需要扩展文件名的地方都可以用 "**"，包括 :next 和 :args 。

消息往回滚动　　new-scroll-back

显示消息并停在 more-prompt 和 hit-enter-prompt 时，用 'k'、'g'、'u' 和 'b' 键可以往回滚动到以前的消息。这对 ":syntax"、":autocommand" 和 ":highlight" 的输出阅读特别有用。使用通用的实现所以支持所有的命令，同时也保持高亮。只有 'more' 选项置位时才能使用。以前，只有 ":clist" 实现了部分这项功能。

如果你在 hit-enter-prompt 时按了 <Enter>，用 g< 命令可以查看最后一个消息页。这时你可以继续往回滚动。

光标超越行尾　　new-onemore

如果 'virtualedit' 选项包含 "onemore"，光标可以移动到刚刚跨越行尾的位置。就像它在换行符上一样。

这时有些命令的行为更加一致。以前，在空行上的光标总是跨越行尾。但这和 Vi 兼容差得很远，而且也可能会影响一些插件或 Vim 脚本。小心使用！

Mattias Flodin 提供此补丁。

POSIX 兼容性　　new-posix

使用 POSIX 测试包来验证 POSIX 的兼容性。修正了不少问题，使 Vim 和 POSIX 更兼容。一些改变和传统的 Vi 或期待的行为有冲突。设置 $VIM_POSIX 环境变量得到 POSIX 兼容行为。见 posix 。

为了 Vi 和 POSIX 的兼容性而修正的项目：
- 用计数重复 "R" 只覆盖文本一次；'cpoptions' 加入 'X' 标志位 cpo-X
- 移动到不存在行的垂直移动命令失败；'cpoptions' 加入 '-' 标志位 cpo--
- 用 preserve 保护文件时并执行 ":q!"，文件可以恢复；'cpoptions' 加入 '&' 标志位 cpo-&
- 部分实现了 'window' 选项。指定只有一个窗口时 CTRL-F 和 CTRL-B 滚动的幅度。现在接受 "-w {number}" 参数。"-w {scriptout}" 里的 {scriptout} 必须不以数位开始。
- 接受 "-c{command}" 参数，"-c" 和 {command} 之间没有空格。
- 用 ":w!" 写入文件时不复位 'readonly'，如果 'cpoptions' 里包含 'Z' 的话。
- 接受 ":list"、":print" 和 ":number" 的 'l' 和 '#' 标志位。
- 'cpoptions' 加入 '.' 标志位：如果缓冲区被修改，":cd" 失败。
- Ex 模式里，空缓冲区使用 ":read file" 不在新行上下保留空行。
- ":global" 命令删除换行符前面的反斜杠。
- ":append"、":insert" 或 ":change" 和 ":global" 一起使用时，从命令中获取插入的行。可以用反斜杠-换行符来分隔行。
- 可用 ":global /pat/ visual" 在每个匹配行上执行普通模式命令。"Q" 可以继续并转到下一行。
- 部分实现 :open 命令。它退出 Ex 模式，但重画整个屏幕，而不是像 open 模式应该做的那样，重画一行。
- 支持使用管道读入外部程序的输出和把输入写到外部程序。加入 'shelltemp' 选项和 has("filterpipe")。
- Ex 安静模式里显示 ":set" 的命令输出。
- 如果以前没用过寄存器，":@@" 和 ":**" 报错。
- 搜索模式 "[]-']" 匹配 ']'、'^'、'_' 和 '`'。
- ":insert" 的自动缩进使用插入位置下方的行。
- ":change" 的自动缩进使用第一个改变的行。
- Ex 命令行的编辑不用加工 (cooked) 模式，因为这样不能处理 CTRL-D 和 CTRL-T。
- Ex 模式里 "1,3" 显示三行，"%" 显示所有行。
- Ex 模式里 "undo" 会撤销所有从启动 Ex 模式时开始的改变。
- 实现 'prompt' 选项。

调试器支持 new-debug-support

加入 'balloonexpr' 选项。这是实现气泡功能的通用机制。用它显示鼠标指针所在单词的相关信息。

远程文件探索器 new-netrw-explore

---

netrw 插件现在支持使用 "scp://" 的目录浏览。也可以删除和换名文件。

为了避免大量重复代码，以前的文件探索器插件现在已经和 netrw 插件集成。这意味着本地和远程的文件浏览现在使用相同的方式。

":browse edit" 和 ":browse split" 在 GUI 对话框不可用而 netrw 插件可用时，使用该插件。

Charles Campbell 维护 netrw 插件。

定义操作符 new-define-operator

以前你不能定义自己的操作符：即 {motion} 动作后面可以跟随的命令。Vim 7 引入了 'operatorfunc' 选项和 g@ 操作符。这样就可以定义和操作符行为相同的映射。实际工作由函数完成，而 g@ 操作符调用该函数。

:map-operator 给出解释和一个示例。

映射到表达式 new-map-expression

映射的 {rhs} 参数可以是表达式。这意味着产生的字符可以取决于上下文。例如：

```
:inoremap <expr> . InsertDot()
```

这里句号并映射到任何 InsertDot() 返回的内容。

这也可用于缩写。详情见 :map-<expr> 。

可视和选择模式映射 new-map-select

以前的可视模式映射同时适用于可视和选择模式。使用一点技巧可以使选择模式的映射和可视模式下的行为相同。

现在加入了分别定义可视和选择模式映射的命令：:xmap 和 :smap ，以及配套的 "noremap" 和 "unmap" 命令。

这个改动同样适用于菜单：:xmenu 、 :smenu 等等。

位置列表 new-location-list

加入了每个窗口分别拥有的 quickfix 列表（称为位置列表）的支持。位置列表显示在位置窗口里（和 quickfix 窗口类似）。你可以打开多于一个位置列表窗口。也加入了一组和 quickfix 命令类似的命令，用来浏览位置列表。(Yegappan Lakshmanan)

若干新项目 new-items-7

普通模式命令：

a"、a' 和 a`	选择引号括起的字符串的新文本对象。 a'
i"、i' 和 i`	(Taro Muraoka)

CTRL-W <Enter>　　quickfix 窗口里：打开新窗口，显示光标所在错误的位置。

at 和 it 文本对象选择 HTML 或 XML 标签之间的文本块。

<A-LeftMouse> ('mousemodel' "popup" 或 "popup-setpos")
<A-RightMouse> ('mousemodel' "extend")
进行面向列块的选择。 <A-LeftMouse>

gF　　开始编辑光标所在的文件名并跳转到文件名后跟随的行号。(Yegappan Lakshmanan)

CTRL-W F　　在新窗口里开始编辑光标所在的文件名并跳转到文件名后跟随的行号。(Yegappan Lakshmanan)

插入模式命令：

CTRL-\ CTRL-O　　执行普通模式命令。类似于 CTRL-O，但不移动光标。 i_CTRL-_CTRL-O

选项：

'balloonexpr'　　用于计算气泡显示文本的表达式
'completefunc'　　用户指定的插入模式补全使用的函数名。插入模式下用 CTRL-X CTRL-U 可以进行任何形式的补全。(Taro Muraoka)
'completeopt'　　插入模式补全可以使用弹出菜单。
'cursorcolumn'　　高亮光标列
'cursorline'　　高亮光标行
'formatexpr'　　gq 或插入模式下文本超过 'textwidth' 以后的文本排版使用的表达式。
'formatlistpat'　　排版时识别编号列表的模式。(Hugo Haas 的建议)
'fsync'　　写入文件时是否调用 fsync()。(Ciaran McCreesh)
'guitablabel'　　GUI 标签页标签用于显示文本的表达式
'guitabtooltip'　　GUI 标签页标签用于显示工具提示文本的表达式
'macatsui'　　Mac: 用 ATSUI 文本显示函数
'maxcombine'　　显示的组合字符的最大数目
'maxmempattern'　　模式匹配使用的最大内存量
'mkspellmem'　　:mkspell 使用内存的相关参数
'mzquantum'　　毫秒计的调度 MzScheme 线程的时间间隔。
'numberwidth'　　'number' 和 'relativenumber' 选项使用空间的最小宽度。(Emmanuel Renieris)
'omnifunc'　　全能补全使用的函数名。
'operatorfunc'　　g@ 操作符使用的函数
'printmbcharset'　　:hardcopy 使用的 CJK 字符集
'printmbfont'　　:hardcopy 产生的 CJK 输出使用的字体名
'pumheight'　　弹出菜单显示的最大项目数
'quoteescape'　　字符串内转义引号的字符。用于 a"、a' 和 a` 文本对象。 a'
'shelltemp'　　外壳命令使用临时文件还是管道
'showtabline'　　是否显示标签页行
'spell'　　打开/关闭拼写检查
'spellcapcheck'　　定位句子尾部的模式
'spellfile'　　加入好词和坏词所用的文件
'spelllang'　　拼写检查所用的语言
'spellsuggest'　　拼写建议的方法

'synmaxcol'	检查语法项目的最大列数；避免长行的重画过慢
'tabline'	标签页行用于显示文本的表达式
'tabpagemax'	-p 打开的最大标签页数
'verbosefile'	记录消息到文件。
'wildoptions'	"tagfile" 值使 CTRL-D 命令行补全可以列出匹配标签所在的文件名。(基于 Yegappan Lakshmanan 的意见)
'winfixwidth'	固定宽度的窗口，类似于 'winfixheight'

Ex 命令:

Win32: ":winpos" 命令现在支持控制台版本。(Vipin Aravind)

:startreplace	开始替换模式。(Charles Campbell)
:startgreplace	开始虚拟替换模式。
:0file	删除缓冲区的名字。(Charles Campbell)
:diffoff	关闭当前窗口或所有窗口的比较模式。
:delmarks	删除位置标记。
:exusage	Ex 命令的帮助 (Nvi 命令)。
:viusage	Vi 命令的帮助 (Nvi 命令)。
:sort	给缓冲区的行排序，不依赖外部命令。(部分由 Bryce Wagner 提供)
:vimgrep	内部 grep 命令，在多个文件中搜索模式。
:vimgrepadd	类似于 :vimgrep ，但不建立新列表。
:caddfile	加入错误信息到已存在的 quickfix 列表 (Yegappan Lakshmanan)。
:cbuffer	读入缓冲区的错误行。(部分由 Yegappan Lakshmanan 提供)
:cgetbuffer	基于缓冲区创建 quickfix 列表，但不跳到第一个错误。
:caddbuffer	从当前缓冲区加入错误到 quickfix 列表。
:cexpr	从 Vim 表达式读入错误信息 (Yegappan Lakshmanan)。
:caddexpr	从 Vim 表达式加入错误信息到已存在的 quickfix 列表 (Yegappan Lakshmanan)。
:cgetexpr	基于 Vim 表达式创建 quickfix 列表，但不跳到第一个错误 (Yegappan Lakshmanan)。
:lfile	类似于 :cfile 但使用位置列表。
:lgetfile	类似于 :cgetfile 但使用位置列表。
:laddfile	类似于 :caddfile 但使用位置列表。
:lbuffer	类似于 :cbuffer 但使用位置列表。
:lgetbuffer	类似于 :cgetbuffer 但使用位置列表。
:laddbuffer	类似于 :caddbuffer 但使用位置列表。
:lexpr	类似于 :cexpr 但使用位置列表。
:lgetexpr	类似于 :cgetexpr 但使用位置列表。
:laddexpr	类似于 :caddexpr 但使用位置列表。
:ll	类似于 :cc 但使用位置列表。
:llist	类似于 :clist 但使用位置列表。

:lnext	类似于 :cnext 但使用位置列表。
:lprevious	类似于 :cprevious 但使用位置列表。
:lNext	类似于 :cNext 但使用位置列表。
:lfirst	类似于 :cfirst 但使用位置列表。
:lrewind	类似于 :crewind 但使用位置列表。
:llast	类似于 :clast 但使用位置列表。
:lnfile	类似于 :cnfile 但使用位置列表。
:lpfile	类似于 :cpfile 但使用位置列表。
:lNfile	类似于 :cNfile 但使用位置列表。
:lolder	类似于 :colder 但使用位置列表。
:lnewer	类似于 :cnewer 但使用位置列表。
:lwindow	类似于 :cwindow 但使用位置列表。
:lopen	类似于 :copen 但使用位置列表。
:lclose	类似于 :cclose 但使用位置列表。
:lmake	类似于 :make 但使用位置列表。
:lgrep	类似于 :grep 但使用位置列表。
:lgrepadd	类似于 :grepadd 但使用位置列表。
:lvimgrep	类似于 :vimgrep 但使用位置列表。
:lvimgrepadd	类似于 :vimgrepadd 但使用位置列表。
:lhelpgrep	类似于 :helpgrep 但使用位置列表。
:lcscope	类似于 :cscope 但使用位置列表。
:ltag	跳转到标签，并把匹配的标签加入位置列表。
:undojoin	将下一个改变和上一个撤销块合并。
:undolist	列出撤销树的叶结点。
:earlier	在时间上退到以前的文本改变。
:later	在时间上进到以后的文本改变。
:for	在 List 上循环。
:endfor	
:lockvar	给变量加锁，防止被改变。
:unlockvar	给加锁的变量解锁。
:mkspell	建立 Vim 拼写文件。
:spellgood	增加单词到好词列表。
:spellwrong	增加单词到坏词列表。
:spelldump	转储好词列表。
:spellinfo	显示关于使用的拼写文件的信息。
:spellrepall	为整个缓冲区重复某个拼写校正。
:spellundo	从好词和坏词列表里删除单词。
:mzscheme	执行 MzScheme 命令。
:mzfile	执行 MzScheme 脚本文件。
:nbkey	把键传递给 NetBeans 进行处理。
:profile	用于 Vim 脚本刨视的命令。
:profdel	为指定项目停止刨视。
:smap	选择模式映射。
:smapclear	
:snoremap	
:sunmap	

:xmap	可视模式映射，不用于选择模式。
:xmapclear	
:xnoremap	
:xunmap	
:smenu	选择模式菜单。
:snoremenu	
:sunmenu	
:xmenu	可视模式菜单，不用于选择模式。
:xnoremenu	
:xunmenu	
:tabclose	关闭当前标签页。
:tabdo	为每个标签页执行命令。
:tabedit	在新标签页里编辑文件。
:tabnew	打开新标签页。
:tabfind	搜索文件并在新标签页里打开。
:tabnext	转到下一个标签页。
:tabprevious	转到前一个标签页。
:tabNext	转到前一个标签页。
:tabfirst	转到第一个标签页。
:tabrewind	转到第一个标签页。
:tablast	转到最后一个标签页。
:tabmove	把当前标签页移到别处。
:tabonly	关闭所有其它标签页。
:tabs	列出标签页和它们包含的窗口。

Ex 命令修饰符：

:keepalt	不改变轮换文件。
:noautocmd	不激活自动命令。
:sandbox	在沙盘里执行命令。
:tab	打开新窗口时，同时建立新标签页。

Ex 命令参数：

++bad	指定出现不能转换的字符或者非法字节时的行为。(代码示例由 Yasuhiro Matsumoto 提供) 另外，如果出现转换错误或者发现非法字节，错误信息里包含行号。

新出现和经过扩充的函数：

add()	在列表后附加项目
append()	附加行列表到缓冲区
argv()	没有参数返回整个参数列表

browsedir()	选择目录的对话框
bufnr()	接受额外参数：建立缓冲区
byteidx()	字符的位置 (Ilya Sher)
call()	调用函数，参数以列表出现
changenr()	当前改变号
complete()	设定插入模式补全的匹配
complete_add()	加入用于 'completefunc' 的匹配
complete_check()	检察键击，用于 'completefunc'
copy()	建立列表或字典的浅备份
count()	计算列表或字典里某值的出现次数
cursor()	也接受 'virtualedit' 的偏移，而第一个参数可以是列表：[lnum, col, off]
deepcopy()	建立列表或字典的完整备份
diff_filler()	得到行 {lnum} 上方填充行的数目。
diff_hlID()	得到比较模式高亮标识号
empty()	检查列表或字典是否为空
eval()	计算 {string} 并返回其结果
extend()	为列表/字典附加另一个列表/字典
feedkeys()	把若干字符放入预输入缓冲区
filter()	删除列表/字典的选定项目
finddir()	寻找 'path' 里的目录
findfile()	寻找 'path' 里的文件 (Johannes Zellner)
foldtextresult()	行 "lnum" 所在关闭折叠的显示用文本
function()	得到函数名对应的函数引用
garbagecollect()	清理只含循环引用的 Lists 和 Dictionaries
get()	得到列表/字典的项目
getbufline()	得到指定缓冲区的行列表 (Yegappan Lakshmanan)
getcmdtype()	得到当前命令行的类型 (Yegappan Lakshmanan)
getfontname()	得到当前使用的字体名
getfperm()	得到文件权限字符串 (Nikolai Weibull)
getftype()	得到文件类型 (Nikolai Weibull)
getline()	带第二个参数：得到缓冲区行列表
getloclist()	位置列表项目的列表 (Yegappan Lakshmanan)
getpos()	得到光标、位置标记等的位置的列表
getqflist()	quickfix 错误的列表 (Yegappan Lakshmanan)
getreg()	取得寄存器内容
gettabwinvar()	取得指定标签页的窗口变量。
has_key()	检查某键是否出现在字典里
haslocaldir()	检查当前窗口是否使用过 :lcd
hasmapto()	检查到字符串的映射是否存在
index()	列表中某值的索引
inputlist()	提示用户从列表选择一项
insert()	在列表某处插入一个项目
islocked()	检查变量是否加锁
items()	得到字典的键-值组对的列表
join()	合并列表项目成为字符串
keys()	得到字典的键列表
len()	列表或字典项目的总数
map()	改变每个列表或字典项目
maparg()	额外参数：使用缩写
mapcheck()	额外参数：使用缩写
match()	额外参数：计数
matcharg()	返回 :match 命令的参数
matchend()	额外参数：计数

matchlist()　　字符串对模式的匹配和子匹配列表
matchstr()　　额外参数：计数
max()　　列表或字典项目的最大值
min()　　列表或字典项目的最小值
mkdir()　　建立目录
pathshorten()　　把目录名简化到一个字符
printf()　　格式化文本
pumvisible()　　检查弹出菜单是否显示
range()　　产生数字序列的列表
readfile()　　读入文件到一个行列表
reltime()　　得到时间值，可能是相对的
reltimestr()　　把时间值转换为字符串
remove()　　删除列表或字典里一或多个项目
repeat()　　重复 "expr" "count" 次 (Christophe Poucet)
reverse()　　反转列表项目的顺序
search()　　额外参数：
searchdecl()　　查找变量的声明
searchpair()　　额外参数：停止搜索的行
searchpairpos()　　返回匹配位置的列表
searchpos()　　返回匹配位置的列表
setloclist()　　修改位置列表 (Yegappan Lakshmanan)
setpos()　　设置光标或位置标记到某位置
setqflist()　　修改 quickfix 列表 (Yegappan Lakshmanan)
settabwinvar()　　设置指定标签页的窗口变量
sort()　　给列表排序
soundfold()　　返回 "发音相似" 的单词等价形式
spellbadword()　　返回错误拼写的单词
spellsuggest()　　返回建议的拼写校正列表
split()　　分割字符串成为列表
str2nr()　　把字符串转换为 2、8、10、16 进制的数值
stridx()　　额外参数：开始位置
strridx()　　额外参数：开始位置
string()　　列表或字典的字符串表示形式
system()　　额外参数：用外壳命令过滤 {input}
tabpagebuflist()　　得到标签页里的缓冲区列表
tabpagenr()　　得到当前或最后一个标签页号
tabpagewinnr()　　得到标签页里的窗口号
tagfiles()　　标签文件名的列表
taglist()　　得到匹配标签的列表 (Yegappan Lakshmanan)
tr()　　翻译字符 (Ron Aaron)
uniq()　　删除重复相邻列表项目的多余备份
values()　　得到列表或字典的值列表
winnr()　　接受参数：使用哪个窗口
winrestview()　　恢复当前窗口的视图
winsaveview()　　保存当前窗口的视图
writefile()　　把一个行列表写到文件里

用户定义的函数现在可以从 'runtimepath' 里的 "autoload" 目录自动载入。见 autoload-functions 。

新 Vim 变量：

v:insertmode　　用于 InsertEnter 和 InsertChange 自动命令

v:val	map() 或 filter() 函数的项目值
v:key	map() 或 filter() 函数的项目键
v:profiling	":profile start" 命令之后非零
v:fcs_reason	激活 FileChangedShell 的原因
v:fcs_choice	FileChangedShell 之后应该做什么
v:beval_bufnr	'balloonexpr' 的缓冲区号
v:beval_winnr	'balloonexpr' 的窗口号
v:beval_lnum	'balloonexpr' 的行号
v:beval_col	'balloonexpr' 的列号
v:beval_text	'balloonexpr' 鼠标指针所在的文本
v:scrollstart	屏幕上滚的原因
v:swapname	SwapExists 事件所用的交换文件名
v:swapchoice	交换文件存在时应该做什么
v:swapcommand	处理 SwapExists 后应该执行的命令
v:char	执行 'formatexpr' 时的参数

新自动命令事件：

ColorScheme	载入色彩方案后
CursorHoldI	在插入模式下，用户有一段时间没有按键
CursorMoved	普通模式下移动了光标
CursorMovedI	插入模式下移动了光标
FileChangedShellPost	对在 Vim 外部改变的文件的处理完成后
InsertEnter	开始插入或替换模式时
InsertChange	出入插入或替换模式时
InsertLeave	离开插入或替换模式时
MenuPopup	刚要显示弹出菜单前
QuickFixCmdPre	执行 :make、:grep 等命令前 (Ciaran McCreesh)
QuickFixCmdPost	执行 :make、:grep 等命令后 (Ciaran McCreesh)
SessionLoadPost	载入会话文件后。(Yegappan Lakshmanan)
ShellCmdPost	执行外壳命令后
ShellFilterPost	用外壳命令执行完过滤后
SourcePre	执行 Vim 脚本之前
SpellFileMissing	使用不存在的拼写文件
SwapExists	编辑文件时检测到交换文件已经存在
TabEnter	刚进入标签页后
TabLeave	刚离开标签页前
VimResized	Vim 窗口大小改变后 (Yakov Lerner)

新高亮组：

Pmenu	弹出菜单：普通项目 hl-Pmenu
PmenuSel	弹出菜单：选中项目 hl-PmenuSel
PmenuThumb	弹出菜单：滚动条 hl-PmenuThumb
PmenuSbar	弹出菜单：滚动条的拇指 hl-PmenuSbar
TabLine	标签页行，非活动标签页标签 hl-TabLine
TabLineSel	标签页行，活动标签页标签 hl-TabLineSel
TabLineFill	标签页行，没有标签的地方 hl-TabLineFill
SpellBad	拼写错误的单词 hl-SpellBad
SpellCap	大小写错误的单词 hl-SpellCap
SpellRare	偏僻词 hl-SpellRare
SpellLocal	只存在于别的区域的单词 hl-SpellLocal
CursorColumn	'cursorcolumn' hl-CursorColumn
CursorLine	'cursorline' hl-CursorLine
MatchParen	匹配的括号 pi_paren.txt hl-MatchParen

搜索模式里的新项目：

/\%d	\%d123	匹配十进制指定的字符
/\]	[\d123]	同上，用于字符类
/\%o	\%o103	匹配八进制指定的字符
/\]	[\o1o3]	同上，用于字符类
/\%x	\%x1a	匹配两位十六进制指定的字符
/\]	[\x1a]	同上，用于字符类
/\%u	\%u12ab	匹配四位十六进制指定的字符
/\]	[\u12ab]	同上，用于字符类
/\%U	\%U1234abcd	匹配八位十六进制指定的字符
/\]	[\U1234abcd]	同上，用于字符类
		(以上部分由 Ciaran McCreesh 提供)
/[[=	[[=a=]]	等价类 (只用于 latin1 字符)
/[[.	[[.a.]]	排序规则元素 (只能使用单个字符)
/\%'m	\%'m	在位置标记 m 处匹配
/\%<'m	\%<'m	在位置标记 m 之前匹配
/\%>'m	\%>'m	在位置标记 m 之后匹配
/\%V	\%V	在可视区域内匹配

如有空匹配，嵌套的 /multi 项目不再是错误。

现在可用 \{0}，匹配前导的匹配原零次。没有用处，只为兼容性而设置。

新语法/缩进/文件类型插件文件：

所有缩进设置都从文件类型插件移到了缩进文件。
实现 b:undo_indent 在 'filetype' 设为其它值时撤销缩进设置。

a2ps 语法和文件类型插件文件。(Nikolai Weibull)
ABAB/4 语法文件。(Marius van Wyk)

alsaconf 文件类型插件文件。(Nikolai Weibull)
AppendMatchGroup 文件类型插件文件。(Dave Silvia)
arch 文件类型插件文件。(Nikolai Weibull)
asterisk 和 asteriskvm 语法文件。(Tilghman Lesher)
BDF 文件类型插件文件。(Nikolai Weibull)
BibTeX 缩进文件。(Dorai Sitaram)
BibTeX Bibliography Style 语法文件。(Tim Pope)
BTM 文件类型插件文件。(Bram Moolenaar)
calendar 文件类型插件文件。(Nikolai Weibull)
Changelog 缩进文件。(Nikolai Weibull)
ChordPro 语法文件。(Niels Bo Andersen)
Cmake 缩进和语法文件。(Andy Cedilnik)
conf 文件类型插件文件。(Nikolai Weibull)
context 语法和文件类型插件文件。(Nikolai Weibull)
CRM114 文件类型插件文件。(Nikolai Weibull)
cvs RC 文件类型插件文件。(Nikolai Weibull)
D 缩进文件。(Jason Mills)
Debian Sources.list 语法文件。(Matthijs Mohlmann)
dictconf 和 dictdconf 语法、缩进和文件类型插件文件。(Nikolai Weibull)
diff 文件类型插件文件。(Bram Moolenaar)
dircolors 文件类型插件文件。(Nikolai Weibull)
django 和 htmldjango 语法文件。(Dave Hodder)
doxygen 语法文件。(Michael Geddes)
elinks 文件类型插件文件。(Nikolai Weibull)
eterm 文件类型插件文件。(Nikolai Weibull)
eviews 语法文件。(Vaidotas Zemlys)
fetchmail RC 文件类型插件文件。(Nikolai Weibull)
FlexWiki 语法和文件类型插件文件。(George Reilly)
Generic 缩进文件。(Dave Silvia)
gpg 文件类型插件文件。(Nikolai Weibull)
gretl 语法文件。(Vaidotas Zemlys)
groovy 语法文件。(Alessio Pace)
group 语法和文件类型插件文件。(Nikolai Weibull)
grub 文件类型插件文件。(Nikolai Weibull)
Haskell 文件类型插件文件。(Nikolai Weibull)
help 文件类型插件文件。(Nikolai Weibull)
indent 文件类型插件文件。(Nikolai Weibull)
Javascript 文件类型插件文件。(Bram Moolenaar)
Kconfig 文件类型插件和语法文件。(Nikolai Weibull)
ld 语法、缩进和文件类型插件文件。(Nikolai Weibull)
lftp 文件类型插件文件。(Nikolai Weibull)
libao config 文件类型插件文件。(Nikolai Weibull)
limits 语法和文件类型插件文件。(Nikolai Weibull)
Lisp 缩进文件。(Sergey Khorev)
loginaccess 和 logindefs 语法和文件类型插件文件。(Nikolai Weibull)
m4 文件类型插件文件。(Nikolai Weibull)
mailaliases 语法文件。(Nikolai Weibull)
mailcap 文件类型插件文件。(Nikolai Weibull)
manconf 语法和文件类型插件文件。(Nikolai Weibull)
matlab 文件类型插件文件。(Jake Wasserman)
Maxima 语法文件。(Robert Dodier)
MGL 语法文件。(Gero Kuhlmann)
modconf 文件类型插件文件。(Nikolai Weibull)
mplayer config 文件类型插件文件。(Nikolai Weibull)

Mrxvtrc 语法和文件类型插件文件。(Gautam Iyer)
MuPAD source 语法、缩进和文件类型插件。(Dave Silvia)
mutt RC 文件类型插件文件。(Nikolai Weibull)
nanorc 语法和文件类型插件文件。(Nikolai Weibull)
netrc 文件类型插件文件。(Nikolai Weibull)
pamconf 语法和文件类型插件文件。(Nikolai Weibull)
Pascal 缩进文件。(Neil Carter)
passwd 语法和文件类型插件文件。(Nikolai Weibull)
PHP 编译器插件。(Doug Kearns)
pinfo 文件类型插件文件。(Nikolai Weibull)
plaintex 语法和文件类型插件文件。(Nikolai Weibull, Benji Fisher)
procmail 文件类型插件文件。(Nikolai Weibull)
prolog 文件类型插件文件。(Nikolai Weibull)
protocols 语法和文件类型插件文件。(Nikolai Weibull)
quake 文件类型插件文件。(Nikolai Weibull)
racc 语法和文件类型插件文件。(Nikolai Weibull)
readline 文件类型插件文件。(Nikolai Weibull)
rhelp 语法文件。(Johannes Ranke)
rnoweb 语法文件。(Johannes Ranke)
Relax NG compact 文件类型插件文件。(Nikolai Weibull)
Scheme 缩进文件。(Sergey Khorev)
screen 文件类型插件文件。(Nikolai Weibull)
sensors 语法和文件类型插件文件。(Nikolai Weibull)
services 语法和文件类型插件文件。(Nikolai Weibull)
setserial 语法和文件类型插件文件。(Nikolai Weibull)
sieve 语法和文件类型插件文件。(Nikolai Weibull)
SiSU 语法文件。(Ralph Amissah)
Sive 语法文件。(Nikolai Weibull)
slp config、reg 和 spi 语法和文件类型插件文件。(Nikolai Weibull)
SML 缩进文件。(Saikat Guha)
SQL anywhere 语法和缩进文件。(David Fishburn)
SQL 缩进文件。
SQL-Informix 语法文件。(Dean L Hill)
SQL: 处理各种不同的变种。(David Fishburn)
sshconfig 文件类型插件文件。(Nikolai Weibull)
Stata 和 SMCL 语法文件。(Jeff Pitblado)
sudoers 文件类型插件文件。(Nikolai Weibull)
sysctl 语法和文件类型插件文件。(Nikolai Weibull)
terminfo 文件类型插件文件。(Nikolai Weibull)
trustees 语法文件。(Nima Talebi)
Vera 语法文件。(David Eggum)
udev config、permissions 和 rules 语法和文件类型插件文件。(Nikolai Weibull)
updatedb 语法和文件类型插件文件。(Nikolai Weibull)
VHDL 缩进文件。(Gerald Lai)
WSML 语法文件。(Thomas Haselwanter)
Xdefaults 文件类型插件文件。(Nikolai Weibull)
XFree86 config 文件类型插件文件。(Nikolai Weibull)
xinetd 语法、缩进和文件类型插件文件。(Nikolai Weibull)
xmodmap 文件类型插件文件。(Nikolai Weibull)
Xquery 语法文件。(Jean-Marc Vanel)
xsd (XML schema) 缩进文件。
YAML 文件类型插件文件。(Nikolai Weibull)
Zsh 文件类型插件文件。(Nikolai Weibull)

新键盘映射表：

僧伽罗语（斯里兰卡）(Harshula Jayasuriya)
TSCII 编码的泰米尔语 (Yegappan Lakshmanan)
cp737 的希腊语 (Panagiotis Louridas)
波兰语-slash (HS6_06)
乌克兰语-jcuken 键盘 (Anatoli Sakhnik)
日语假名 (Edward L. Fox)

新消息翻译：

现在也有了 cp1251 的乌克兰语消息。
越南语消息翻译和菜单。(Phan Vinh Thinh)

其它：

:read 参数现在有了 ++edit 参数。从而使用检测到的缓冲区的 'fileformat'、'fileencoding' 和其它选项。同时也修正了压缩文件编辑时不设置这些选项的问题。

为 Sun Studio 10 更新 Netbeans 接口。协议号从 2.2 上到 2.3。(Gordon Prieur)

Mac: 启动时 Vim 会载入 $VIMRUNTIME/macmap.vim 脚本来定义缺省的 command 键的映射。(多由 by Benji Fisher 提供)

Mac: 加入剪贴板选择区的类型，这样两个 Vim 之间可以共享面向列块、行和字符的选择区。(Eckehard Berns)
也修正设置 'clipboard' 为 "unnamed" 时 "yyp" 出现的问题。

Mac: GUI 字体选择器。(Peter Cucka)

Mac: 支持多字节字符。(Da Woon Jung)
并不总能工作。如果有文本绘制的问题，可以尝试关闭 'macatsui' 选项。

Mac: 支持非 GUI 版本的 xterm 鼠标。

Mac: 和 Xcode 更好的集成。在 odoc 事件和拖动接收处理程序之后发送 (post) 伪鼠标释放事件，以解决 Vim 载入文件后不响应的问题。修正一个 "一行之差" 的行号错误。(Da Woon Jung)

Mac: 从 Finder 启动时改变目录为编辑文件所在的目录或用户主目录。

加入 t_SI 和 t_EI 转义序列以启动和结束插入模式。
用于设置光标形状为条状或块状。没有缺省值，因为 termcap/terminfo 不支持它们。

Motif 上有了 GUI 字体选择器。(Marcin Dalecki)

Motif 有更好的工具栏按钮。(Marcin Dalecki)

Motif 有搜索/替代对话框助记。(Marcin Dalecki)

加入 Marcin Dalecki 提供的若干 Motif 改进。现在我们自己绘制标签内容，使得字体

的处理方式可以通过 Vim 定制，也相对减少了对 X11 字体管理的依赖。

可以定义局部于缓冲区的自动命令。这意味着即使缓冲区没有名字或者没有特定的名字，也能使用自动命令。见 autocmd-buflocal 。(Yakov Lerner)

xterm 现在能够识别多数功能键和修饰符的组合。 xterm-modifier-keys

如果设置 'verbose'，":highlight" 的输出显示高亮项目最近在哪里设置。
如果设置 'verbose'，":map"、":abbreviate"、":command"、":function" 和 ":autocmd" 的输出显示它最近在哪里定义。(Yegappan Lakshmanan)

":function /pattern" 列出匹配模式的函数。

"1gd" 和 "gd" 类似，但忽略光标位置之前的 {} 块里的匹配。"1gD" 和 "gD" 也是。

'scrolljump' 可以设为负数，滚动到窗口高度的一个百分比。

加入 v:scrollstart 变量，帮助寻找产生 hit-enter 提示的脚本位置。

要能处理待编辑的文件已经被别的 Vim 实例编辑的情形，加入 SwapExists 事件。可以使用 v:swapname 、 v:swapchoice 和 v:swapcommand 变量。用此事件可以调用 client-server 功能，把另一个 Vim 带到前台。
启动 Vim 时如有 "-t tag" 参数，已经有交换文件而用户选择了 "quit" 或 "abort"，退出 Vim。

撤销现在同时恢复 '< 和 '> 位置标记。"gv" 可以选择改变和撤销之前的相同区域。

编辑 "/" 或 "?" 命令的搜索模式并且置位 'incsearch' 时，可用 CTRL-L 从当前匹配上加一个字符。CTRL-R CTRL-W 会加入一个单词，但排除已经输入的单词部分。

Ruby 接口：加入行号方法。(Ryan Paul)

$MYVIMRC 环境变量设为第一个找到的 vimrc 文件。
$MYGVIMRC 环境变量设为第一个找到的 gvimrc 文件。

改 进 improvements-7

":helpgrep" 在模式之后接受语言描述符：":helpgrep pat@it"。

打印的帮助信息被移到独立的帮助文件。现在这部分篇幅已经不短。

执行 ":!cmd"、":r !cmd" 或 ":w !cmd" 的补全时，寻找 $PATH 里的可执行文件而不是当前目录的普通文件。

如果使用 ":silent" 并且 Ex 命令使用反向的行范围，自动交换范围的两端而不询问用户是否需要这么做。

模式匹配的代码从递归函数修改为迭代机制。这样避免了堆栈溢出错误。分配的内存保存了状态，所以总是可以检测到内存溢出。允许匹配更加复杂的模式，但过程可能很长，所以期间 Vim 看起来可能像挂起一样。

以前总在沙盘 sandbox 里计算一些选项，现在只有从模式行里设置选项或在安全模式里才会这么做。应用于 'balloonexpr'、'foldexpr'、'foldtext' 和 'includeexpr'。

(Sumner Hayes)

一些命令和表达式可能包含可怕的副作用，比如编辑搜索模式时使用 CTRL-R =，而该表达式又调用了跳到其它窗口的函数的情况。加入 textlock 防止这种情况的发生。

用 ":breakadd here" 和 ":breakdel here" 可以在光标位置设置或删除断点。

现在可以这样定义函数:
```
:exe "func Test()\n ...\n endfunc"
```

教程进行了更新，使它更容易使用，也增加了一些解释更多重要命令的文本。建议由 Gabriel Zachmann 提供。

Unix: libcall() 失败时，用 dlerror() 得到错误信息并显示。(Johannes Zellner)

Mac 和 Cygwin: 编辑已有的文件时，使文件名使用和编辑文件相同的大小写。因而输入 ":e os_UNIX.c" 后文件名会成为 "os_unix.c"。

'listchars' 里加入 "nbsp"。(David Blanchet)

'buftype' 选项加入 "acwrite" 值。适用于无名但引用某个文件的缓冲区，写回时会用到 BufWriteCmd 自动命令。

Lisp 缩进和括号匹配: (Sergey Khorev)
- 正确识别方括号
- 识别 #\(、#\)、#\[和 #\] 为字符常数
- 识别 Lisp 行注释 (分号定界)

match()、matchend() 和 matchstr() 加入 "count" 参数。(Ilya Sher)

winnr() 接受可选的 "$" 或 "#" 参数。(Nikolai Weibull, Yegappan Lakshmanan)

search() 加入 's' 标志位: 如果光标移动，设置 ' 位置标记。(Yegappan Lakshmanan)
search() 加入 'n' 标志位: 不移动光标。(Nikolai Weibull)
search() 加入 'c' 标志位: 接受光标所在的匹配。
search() 加入 'e' 标志位: 移到匹配的尾部。(Benji Fisher)
search() 加入 'p' 标志位: 返回子模式号。(Benji Fisher)
同样适用于 searchpos()、searchpair() 和 searchpairpos()。

search() 和 searchpair() 函数包含附加参数来指定在哪里停止搜索。如果搜索无须走得太远，这加快了搜索的速度。

gzip 插件解压失败时，给出错误消息但不删除原始文本。这有助于文件有 .gz 扩展名但实际没有压缩的情形。(Andrew Pimlott)

使用 C、C++ 或 IDL 语法时，可以附加载入 doxygen 语法。(Michael Geddes)

支持设置 'filetype' 和 'syntax' 为 "aaa.bbb"，代表 "aaa" 加上 "bbb" 的文件类型和语法。

":registers" 命令现在正确显示多字节字符。

VMS: 使用说明提到了可以用斜杠使标志位大写。加入内建的 vt320 终端代码色彩支持。(Zoltan Arpadffy)

'viminfo' 的 '%' 项目允许设置缓冲区数目的上限。

文件类型的识别：如果文件看起来像外壳脚本，检查启动 tcl 解释器的 "exec" 命令。(Alexios Zavras 建议)

支持 utf-8 和 latin9 (iso-8859-15) 的内部转换，这样就算没有 iconv 也可以使用二合字母。

编辑无名且空白的缓冲区时如果载入会话文件，彻底删除该缓冲区。避免在缓冲区列表里留下一个不用的缓冲区。

Win32: 如果 libintl.dll 支持 bind_textdomain_codeset()，使用此函数。(NAKADAIRA Yukihiro)

Win32: Vim 以前不识别 NTFS 文件系统的硬链接。现在当 'backupcopy' 为 "auto" 时会检测到。同时也修正关于链接的 "file has been changed since reading it" 虚假信息。

如果 foldtext() 发现删除注释引导符后没有文本，使用折叠的第二行。可用于 C 风格的 /* */ 注释，其中第一行只包含 "/*"。

两个系统上编辑相同文件时（例如 Unix 和 MS-Windows），以前大多不会给出交换文件已经存在的警告，因为使用的编辑文件名不同（如 y:\dir\file 和 /home/me/dir/file）。现在给交换文件加入标志位，指示和编辑文件放在相同的目录。因为不受使用路径的影响，现在关于是否编辑相同文件的检查可靠得多。

Unix: 通过符号链接编辑文件时，交换文件过去使用符号链接的名字。现在使用实际文件的名字，这样可以检测到对文件的重复编辑。(Stefano Zacchiroli 和 James Vega 的建议)

客户-服务器通信现在支持 'encoding'。如果在 Vim 服务器上设置 'encoding' 为 "utf-8" 而从控制台上调用 "vim --remote fname"，"fname" 被从控制台编码转换为 utf-8。这样也允许使用不同 'encoding' 设置的 Vim 之间交换消息。

内部改动：修改 ga_room 为 ga_maxlen，这样它不需要每次都递增/递减。

如果寄存器为空，不把它保存到 viminfo 文件。

删除 tcltags 脚本，它已经废弃了。

":redir @*>>" 和 ":redir @+>>" 附加到剪贴板。寄存器名后非法字符的检查更严格。
:redir

":redir => variable" 和 ":redir =>> variable" 写入或附加到变量。(Yegappan Lakshmanan) :redir

":redir @{a-z}>>" 附加到寄存器 a 到 z。(Yegappan Lakshmanan)

'verbosefile' 选项可以把消息记录到文件。此时不再显示 verbose 消息。也可用 "-V{filename}" 参数记录启动消息。

":let g:" 列出全局变量。
":let b:" 列出缓冲区局部变量。

":let w:" 列出窗口局部变量。
":let v:" 列出 Vim 变量。

stridx() 和 strridx() 函数接受第三个参数，指定从哪里开始搜索。(Yegappan Lakshmanan)

getreg() 函数接受额外的一个参数，从而得到 '=' 寄存器表达式本身而不是它计算之后的值。

setline() 函数接受列表参数以设置多行。如果行号刚刚在末行之后，附加行。

如果字符数和字节数不同的话，g CTRL-G 同时显示字符数。

":debug" 和 '=' 寄存器表达式的补全。跳过行范围和命令名之间的 ":"。
(Peter Winters)

插入模式的 CTRL-Q 现在缺省同 CTRL-V。过去它被忽略。

如果 'debug' 包含 "beep"，发出铃声的函数或脚本会给出消息并包括错误源。

补全缓冲区名时，用 "\(^\|[\/]\)" 而不是 "^" 进行匹配，这样 ":buf stor<Tab>" 能同时找到 "include/storage.h" 和 "storage/main.c"。

要计算（模式匹配）的项目数而不修改缓冲区，可用 :substitute 的 'n' 标志位。见 count-items 。

:substitute 命令的 \u、\U、\l 和 \L 项目现在支持多字节字符。

识别名为 "screen.linux" 的 $TERM 以设置 'background' 的缺省值为 "dark"。
(Ciaran McCreesh) "cygwin" 和 "putty" 也是如此。

FileChangedShell 自动命令事件现在使用 v:fcs_reason 变量来指定激活事件的原因。 v:fcs_choice 可以选择重新载入缓冲区或询问用户应该如何进行。

xterm 功能键过去不能识别所有的修饰符。现在终端代码可用 ";*X" 或 "O*X" 结尾，其中 X 是任何字符而 * 代表修饰符代码。
加入 <xUp>、<xDown>、<xLeft> 和 <xRight> 键以识别 xterm 发送这些代码的两种可能
形式，同时仍然可以处理所有的修饰符。

getwinvar() 现在也可以从指定窗口取得局部于缓冲区的选项。

'errorformat' 加入 "%s" 项目。(Yegappan Lakshmanan)
'errorformat' 加入 "%>" 项目。

'errorformat' 过去不能使用包含出现在 "%f" 之后的字符的文件名。例如，"%f:%l:%m" 里文件名不能包含 ":"。现在必须余下的模式匹配以后才会包含第一个 ":"。在这个例子里，文件名可以包含后面没有行号的 ":"。(Emanuele Giaquinta 建议)

GTK GUI: 如果可能，使用 GTK 文件对话框。Grahame Bowland 和 Evan Webb 补丁的混合。

bugreport.vim 加入 ":scriptnames"，这样可以看到使用了哪些插件。

Win32: 如果用户改变了滚轮点击滚动行数的设置，立即使用。过去 Vim 需要重启。

在表达式里使用 @= 时，值是 @= 包含的内容。用 ":let @= = value" 可以设置寄存器内容。

":popup" 可以加上 !，使弹出菜单出现在鼠标指针的位置而不是文本光标的位置。

扩充了编码表，支持许多 MS-Windows 代码页，如 cp1250 和 cp737。这样 Unix 就可以直接使用它们而无需在前面加上 "8bit-"。
如果编码名以 "microsoft-cp" 开始，忽略 "microsoft-" 部分。

用户定义的命令加入 "customlist" 补全参数。用户定义的补全参数应该返回 Vim 列表形式的补全的候选值，Vim 不过滤返回值。(Yegappan Lakshmanan)

Win32: 如果 common controls 支持，气泡可以多行。(Sergey Khorev)

命令行补全现在对各种类型参数的匹配结果进行排序：用户命令、变量、语法名等。

如果不设置 locale，使用 "C" locale。Vim 会处理 latin1 字符并使用自己的 isupper()/toupper()/等函数。

rxvt 终端模拟器用 COLORFGBG 环境变量来猜测 'background' 的值。
(Ciaran McCreesh)

同时支持 Unix 上 t_SI 和 t_EI 的正常功能。(Ciaran McCreesh)

如果 'foldcolumn' 为 1，放入尽量多的信息。鼠标点击 '-' 可以关闭折叠。

input() 接受可选的补全参数，指定输入能接受的补全类型。(Yegappan Lakshmanan)

比较模式下，"dp" 可接受多于两个缓冲区，如果只有一个置位 'modifiable' 的话。

'diffopt' 选项有三个新值："horizontal"、"vertical" 和 "foldcolumn"。

如果 'include' 选项包含 \zs，找到的文件名是 \zs 到模式结束处或 \ze。这样可以给 'includeexpr' 传递更多的信息。

现在，在启动过程的插件载入时支持 plugin 目录下的子目录。 load-plugins

foldcolumn 总是为关闭折叠显示 '+'，方便打开。这可能覆盖其它字符，'foldcolumn' 为 1 尤其如此。

现在置位 'readonly' 可以再次得到 W10 消息。如果 FileChangedRO 自动命令获得文件失败，可以利用这一点。

Unix: open() 返回 EFBIG 时会给出合适的信息。

":mksession" 设置 SessionLoad 变量以通知插件。会话文件也加入了模式行来设置 'filetype' 为 "vim"。

ATTENTION 提示把 "Delete it" 选择放在 "Quit" 之前，这样更合逻辑。(Robert Webb)

附加到文件而缓冲区没有名字时，过去使用附加文件的名字作为缓冲区名。但是缓冲区的内容实际和文件内容不同。现在不设置文件名，除非 'cpoptions' 包含 'P' 标志位。

开始编辑新文件而文件所在的目录不存在时，Vim 会报告 "[New DIRECTORY]" 而不是 "[New File]"，从而提示用户可能出现了错误。

Win32: 保持 viminfo 文件的隐藏属性。

过去用了 CTRL-O 和光标键后，插入模式下的 CTRL-A 会记不住最近插入的文本。现在 CTRL-O 之后的那个命令如果不插入文本，最近插入的文本会被记住。这样，CTRL-O 命令可以移动光标而不用担心会丢失最近插入的文本。

exists() 函数现在提供对自动命令组是否定义和自动命令事件是否支持的检查。(Yegappan Lakshmanan)

允许在沙盘里使用 ":global"，因为它本身不会造成任何伤害。

如果 'filetype' 为空，":saveas asdf.c" 会设置它为 c。":w asdf.c" 如果会设置缓冲区的文件名，也是如此。

插入模式的整行补全现在同时搜索未载入的缓冲区。

colortest.vim 脚本现在可以用 ":source" 或 ":runtime syntax/colortest.vim" 直接调用。

'statusline' 选项可以局部于窗口，这样每个窗口就可以有不同的值。(部分由 Yegappan Lakshmanan 提供)

现在，'statusline' 和其它支持相同格式的选项可以使用新特性:
- 如果以 "%!" 开始，先作为表达式计算该值，然后再对计算结果进行分析。
- 用 "%#HLname#" 开始名为 HLname 的高亮。

如果 'statusline' 设定的值产生了错误信息，它被清空以免出现无限重画循环。其它一些选项，如 'tabline' 和 'titlestring'，也是如此。":verbose set statusline" 会提到该选项的设置位置在错误处理程序里。

如果有若干匹配的标签，":tag <name>" 和 CTRL-] 命令跳转到第 [count] 个匹配的标签。(Yegappan Lakshmanan)

Win32: 如果设置的话，安装程序生成的批处理文件使用 $VIMRUNTIME 或 $VIM。Mathias Michaelis 提供了示例。
同时新建 vimtutor.bat 批处理文件。

'balloonexpr' 选项现在变成了 global-local 类型。

system() 函数现在运行于加工 (cooked) 模式，从而可以用 CTRL-C 中断。

编 译 时 的 改 变 compile-changes-7

放弃对 BeOS 和 Amiga GUI 的支持。没有人维护，可能也已经无法工作。如果你想做:
获取 Vim 6.x 版本并重新把你的工作合并进来。

运行测试时，如果其中一个失败而不能生成 "test.out"，其余的测试继续运行。如果有内存溢出的情况，这么做会有帮助。

编译时定义 EXITFREE 并且带上 ccmalloc 库可来检测内存泄漏。有些内存总是报告为泄漏，例如 X11 库函数分配的和 alloc_cmdbuff() 分配用于保存 ":quit" 命令的内存。

打印代码移到 src/hardcopy.c。

main() 的一些代码移到独立的若干函数，这使得操作更清晰。这些函数也使用结构来避免过多的参数。

unix_expandpath() 移到 misc1.c，这使得 os_mac.c 也可以使用它而无须复制代码。

--- Mac ---

"make" 现在建立 Vim.app 目录，而 "make install" 把它复制到最终的目标位置。(Raf)

运行时目录不直接放在 Vim.app，而放在 Vim.app/Contents/Resources/vim，这样做遵从了 Mac 的规范。

编译时可以不用任何修改就加入 Motif、Athena 或 GTK 支持，而且仍然可用 MacRoman 转换。加入 os_mac_conv.c 文件。

运行 "make install" 时，像 Unix 那样安装运行时文件，以免复制过多文件。
运行 "make" 时建立到运行时文件的链接，以免递归复制需要太多时间。

configure 尝试同时编译 Intel 和 PowerPC 的 Vim。configure 参数 --with-mac-arch 可以改变这一点。

--- Win32 ---

调整 Make_mvc.mak 文件，使它可用于最新的 MS 编译器，包括 Visual Studio 2005 的免费版本。(George Reilly)

INSTALLpc.txt 进行更新，以包含最近的改变。(George Reilly)

现在用免费的 Visual C++ Toolkit 2003 和其它免费 SDK 工具生成发布的可执行文件。加入 msvcsetup.bat，提供这方面的支持。

也生成 .pdb 文件，可用于生成 MS-Windows 上有用的崩溃报告。(George Reilly)

漏 洞 修 正 bug-fixes-7

When using PostScript printing on MS-DOS the default 'printexpr' used "lpr" instead of "copy". When 'printdevice' was empty the copy command did not work. Use "LPT1" then.

The GTK font dialog uses a font size zero when the font name doesn't include a size. Use a default size of 10.

This example in the documentation didn't work:
```
    :e `=foo . ".c"`
```
Skip over the expression in =expr when looking for comments, |, % and #.

When ":helpgrep" doesn't find anything there is no error message.

"L" and "H" did not take closed folds into account.

Win32: The "-P title" argument stopped at the first title that matched, even when it doesn't support MDI.

Mac GUI: CTRL-^ and CTRL-@ did not work.

"2daw" on "word." at the end of a line didn't include the preceding white space.

Win32: Using FindExecutable() doesn't work to find a program. Use SearchPath() instead. For executable() use $PATHEXT when the program searched for doesn't have an extension.

When 'virtualedit' is set, moving the cursor up after appending a character may move it to a different column. Was caused by auto-formatting moving the cursor and not putting it back where it was.

When indent was added automatically and then moving the cursor, the indent was not deleted (like when pressing ESC). The "I" flag in 'cpoptions' can be used to make it work the old way.

When opening a command-line window, 'textwidth' gets set to 78 by the Vim filetype plugin. Reset 'textwidth' to 0 to avoid lines are broken.

After using cursor(line, col) moving up/down doesn't keep the same column.

Win32: Borland C before 5.5 requires using ".u." for LowPart and HighPart fields. (Walter Briscoe)

On Sinix SYS_NMLN isn't always defined. Define it ourselves. (Cristiano De Michele)

Printing with PostScript may keep the printer waiting for more. Append a CTRL-D to the printer output. (Mike Williams)

When converting a string with a hex or octal number the leading '-' was ignored. ":echo '-05' + 0" resulted in 5 instead of -5.

Using "@:" to repeat a command line didn't work when it contains control characters. Also remove "'<,'>" when in Visual mode to avoid that it appears twice.

When using file completion for a user command, it would not expand environment variables like for a regular command with a file argument.

'cindent': When the argument of a #define looks like a C++ class the next line is indented too much.

When 'comments' includes multi-byte characters inserting the middle part and alignment may go wrong. 'cindent' also suffers from this for right-aligned items.

Win32: when 'encoding' is set to "utf-8" getenv() still returns strings in the active codepage. Convert to utf-8. Also for $HOME.

The default for 'helplang' was "zh" for both "zh_cn" and "zh_tw". Now use "cn" or "tw" as intended.

When 'bin' is set and 'eol' is not set then line2byte() added the line break after the last line while it's not there.

Using foldlevel() in a WinEnter autocommand may not work. Noticed when resizing the GUI shell upon startup.

Python: Using buffer.append(f.readlines()) didn't work. Allow appending a string with a trailing newline. The newline is ignored.

When using the ":saveas f2" command for buffer "f1", the Buffers menu would contain "f2" twice, one of them leading to "f1". Also trigger the BufFilePre and BufFilePost events for the alternate buffer that gets the old name.

strridx() did not work well when the needle is empty. (Ciaran McCreesh)

GTK: Avoid a potential hang in gui_mch_wait_for_chars() when input arrives just before it is invoked

VMS: Occasionally CR characters were inserted in the file. Expansion of environment variables was not correct. (Zoltan Arpadffy)

UTF-8: When 'delcombine' is set "dw" only deleted the last combining character from the first character of the word.

When using ":sball" in an autocommand only the filetype in one buffer was detected. Reset did_filetype in enter_buffer().

When using ":argdo" and the window already was at the first argument index, but not actually editing it, the current buffer would be used instead.

When ":next dir/*" includes many matches, adding the names to the argument list may take an awful lot of time and can't be interrupted. Allow interrupting this.

When editing a file that was already loaded in a buffer, modelines were not used. Now window-local options in the modeline are set. Buffer-local options and global options remain unmodified.

Win32: When 'encoding' is set to "utf-8" in the vimrc file, files from the command line with non-ASCII characters are not used correctly. Recode the file names when 'encoding' is set, using the Unicode command line.

Win32 console: When the default for 'encoding' ends up to be "latin1", the default value of 'isprint' was wrong.

When an error message is given while waiting for a character (e.g., when an xterm reports the number of colors), the hit-enter prompt overwrote the last line. Don't reset msg_didout in normal_cmd() for K_IGNORE.

Mac GUI: Shift-Tab didn't work.

When defining tooltip text, don't translate terminal codes, since it's not going to be used like a command.

GTK 2: Check the tooltip text for valid utf-8 characters to avoid getting a GTK error. Invalid characters may appear when 'encoding' is changed.

GTK 2: Add a safety check for invalid utf-8 sequences, they can crash pango.

Win32: When 'encoding' is changed while starting up, use the Unicode command line to convert the file arguments to 'encoding'. Both for the GUI and the console version.

Win32 GUI: latin9 text (iso-8859-15) was not displayed correctly, because there is no codepage for latin9. Do our own conversion from latin9 to UCS2.

When two versions of GTK+ 2 are installed it was possible to use the header files from one and the library from the other. Use GTK_LIBDIR to put the directory for the library early in the link flags.

With the GUI find/replace dialog a replace only worked if the pattern was literal text. Now it works for any pattern.

When 'equalalways' is set and 'eadirection' is "hor", ":quit" would still cause equalizing window heights in the vertical direction.

When ":emenu" is used in a startup script the command was put in the typeahead buffer, causing a prompt for the crypt key to be messed up.

Mac OS/X: The default for 'isprint' included characters 128-160, causes problems for Terminal.app.

When a syntax item with "containedin" is used, it may match in the start or end of a region with a matchgroup, while this doesn't happen for a "contains" argument.

When a transparent syntax items matches in another item where the highlighting has already stopped (because of a he= argument), the highlighting would come back.

When cscope is used to set the quickfix error list, it didn't get set if there was only one match. (Sergey Khorev)

When 'confirm' is set and using ":bdel" in a modified buffer, then selecting "cancel", would still give an error message.

The PopUp menu items that started Visual mode didn't work when not in Normal mode. Switching between selecting a word and a line was not possible.

Win32: The keypad decimal point always resulted in a '.', while on some keyboards it's a ','. Use MapVirtualKey(VK_DECIMAL, 2).

Removed unused function DisplayCompStringOpaque() from gui_w32.c

In Visual mode there is not always an indication whether the line break is selected or not. Highlight the character after the line when the line break is included, e.g., after "v$o".

GTK: The <F10> key can't be mapped, it selects the menu. Disable that with a GTK setting and do select the menu when <F10> isn't mapped. (David Necas)

After "Y" '[and '] were not at start/end of the yanked text.

When a telnet connection is dropped Vim preserves files and exits. While doing that a SIGHUP may arrive and disturbe us, thus ignore it. (Scott Anderson) Also postpone SIGHUP, SIGQUIT and SIGTERM until it's safe to handle. Added handle_signal().

When completing a file name on the command line backslashes are required for white space. Was only done for a space, not for a Tab.

When configure could not find a terminal library, compiling continued for a long time before reporting the problem. Added a configure check for tgetent() being found in a library.

When the cursor is on the first char of the last line a ":g/pat/s///" command may cause the cursor to be displayed below the text.

Win32: Editing a file with non-ASCII characters doesn't work when 'encoding' is "utf-8". use _wfullpath() instead of _fullpath(). (Yu-sung Moon)

When recovering the 'fileformat' and 'fileencoding' were taken from the original file instead of from the swapfile. When the file didn't exist, was empty or the option was changed (e.g., with ":e ++fenc=cp123 file") it could be wrong. Now store 'fileformat' and 'fileencoding' in the swapfile and use the values when recovering.

":bufdo g/something/p" overwrites each last printed text line with the file message for the next buffer. Temporarily clear 'shortmess' to avoid that.

Win32: Cannot edit a file starting with # with --remote. Do escape % and # when building the ":drop" command.

A comment or | just after an expression-backtick argument was not recognized. E.g. in :e ="foo" "comment.

"(" does not stop at an empty sentence (single dot and white space) while ")" does. Also breaks "das" on that dot.

When doing "yy" with the cursor on a TAB the ruler could be wrong and "k" moved the cursor to another column.

When 'commentstring' is '"%s' and there is a double quote in the line a double quote before the fold marker isn't removed in the text displayed for a closed fold.

In Visual mode, when 'bin' and 'eol' set, g CTRL-G counted the last line break, resulting in "selected 202 of 201 bytes".

Motif: fonts were not used for dialog components. (Marcin Dalecki)

Motif: After using a toolbar button the keyboard focus would be on the toolbar (Lesstif problem). (Marcin Dalecki)

When using "y<C-V>`x" where mark x is in the first column, the last line was not included.

Not all test scripts work properly on MS-Windows when checked out from CVS. Use a Vim command to fix all fileformats to dos before executing the tests.

When using ":new" and the file fits in the window, lines could still be above the window. Now remove empty lines instead of keeping the relative position.

Cmdline completion didn't work after ":let var1 var<Tab>".

When using ":startinsert" or ":startreplace" when already in Insert mode (possible when using CTRL-R =), pressing Esc would directly restart Insert mode. (Peter Winters)

"2daw" didn't work at end of file if the last word is a single character.

Completion for ":next a'<Tab>" put a backslash before single quote, but it was not removed when editing a file. Now halve backslashes in save_patterns(). Also fix expanding a file name with the shell that contains "\'".

When doing "1,6d|put" only "fewer lines" was reported. Now a following "more lines" overwrites the message.

Configure could not handle "-Dfoo=long\ long" in the TCL config output.

When searching backwards, using a pattern that matches a newline and uses \zs after that, didn't find a match. Could also get a hang or end up in the right column in the wrong line.

When $LANG is "sl" for slovenian, the slovak menu was used, since "slovak" starts with "sl".

When 'paste' is set in the GUI the Paste toolbar button doesn't work. Clear 'paste' when starting the GUI.

A message about a wrong viminfo line included the trailing NL.

When 'paste' is set in the GUI the toolbar button doesn't work in Insert mode. Use ":exe" in menu.vim to avoid duplicating the commands, instead of using a mapping.

Treat "mlterm" as an xterm-like terminal. (Seiichi Sato)

":z.4" and ":z=4" didn't work Vi compatible.

When sourcing a file, editing it and sourcing it again, it could appear twice in ":scriptnames" and get a new <SID>, because the inode has changed.

When $SHELL is set but empty the 'shell' option would be empty. Don't use an empty $SHELL value.

A command "w! file" in .vimrc or $EXINIT didn't work. Now it writes an empty file.

When a CTRL-F command at the end of the file failed, the cursor was still moved to the start of the line. Now it remains where it is.

When using ":s" or "&" to repeat the last substitute and "$" was used to put the cursor in the last column, put the cursor in the last column again. This is Vi compatible.

Vim is not fully POSIX compliant but sticks with traditional Vi behavior. Added a few flags in 'cpoptions' to behave the POSIX way when wanted. The $VIM_POSIX environment variable is checked to set the default.

Appending to a register didn't insert a line break like Vi. Added the '>' flag to 'cpoptions' for this.

Using "I" in a line with only blanks appended to the line. This is not Vi compatible. Added the 'H' flag in 'cpoptions' for this.

When joining multiple lines the cursor would be at the last joint, but Vi leaves it at the position where "J" would put it. Added the 'q' flag in 'cpoptions' for this.

Autoindent didn't work for ":insert" and ":append".

Using ":append" in an empty buffer kept the dummy line. Now it's deleted to be Vi compatible.

When reading commands from a file and stdout goes to a terminal, would still request the xterm version. Vim can't read it, thus the output went to the shell and caused trouble there.

When redirecting to a register with an invalid name the redirection would still be done (after an error message). Now reset "redir_reg". (Yegappan Lakshmanan)

It was not possible to use a NL after a backslash in Ex mode. This is sometimes used to feed multiple lines to a shell command.

When 'cmdheight' is set to 2 in .vimrc and the GUI uses the number of lines from the terminal we actually get 3 lines for the cmdline in gvim.

When setting $HOME allocated memory would leak.

Win32: bold characters may sometimes write in another character cell. Use unicodepdy[] as for UTF-8. (Taro Muraoka)

":w fname" didn't work for files with 'buftype' set to "nofile".

The method used to locate user commands for completion differed from when they are executed. Ambiguous command names were not completed properly.

Incremental search may cause a crash when there is a custom statusline that indirectly invokes ":normal".

Diff mode failed when $DIFF_OPTIONS was set in the environment. Unset it before invoking "diff".

Completion didn't work after ":argdo", ":windo" and ":bufdo". Also for ":set &l:opt" and ":set &g:opt". (Peter Winters)

When setting 'ttymouse' to "dec" in an xterm that supports the DEC mouse locator it doesn't work. Now switch off the mouse before selecting another mouse model.

When the CursorHold event is triggered and the commands peek for typed characters the typeahead buffer may be messed up, e.g., when a mouse-up event is received. Avoid invoking the autocommands from the function waiting for a character, let it put K_CURSORHOLD in the input buffer.

Removed the "COUNT" flag from ":argadd", to avoid ":argadd 1*" to be used like ":1argadd *". Same for ":argdelete" and ":argedit".

Avoid that $LANG is used for the menus when LC_MESSAGES is "en_US".

Added backslashes before dashes in the vim.1 manual page to make them appear as real dashes. (Pierre Habouzit)

Where "gq" left the cursor depended on the value of 'formatprg'. Now "gq" always leaves the cursor at the last line of the formatted text.

When editing a compressed file, such as "changelog.Debian.gz" file, filetype detection may try to check the contents of the file while it's still compressed. Skip setting 'filetype' for compressed files until they have been decompressed. Required for patterns that end in a "*".

Starting with an argument "+cmd" or "-S script" causes the cursor to be moved to the first line. That breaks a BufReadPost autocommand that uses g`". Don't move the cursor if it's somewhere past the first line.

"gg=G" while 'modifiable' is off was uninterruptible.

When 'encoding' is "sjis" inserting CTRL-V u d800 a few times causes a crash. Don't insert a DBCS character with a NUL second byte.

In Insert mode CTRL-O <Home> didn't move the cursor. Made "ins_at_eol" global and reset it in nv_home().

Wildcard expansion failed: ":w /tmp/$$.`echo test`". Don't put quotes around spaces inside backticks.

After this sequence of commands: Y V p gv: the wrong line is selected. Now let "gv" select the text that was put, since the original text is deleted. This should be the most useful thing to do.

":sleep 100u" sleeps for 100 seconds, not 100 usec as one might expect. Give an error message when the argument isn't recognized.

In gui_mch_draw_string() in gui_w32.c "unibuflen" wasn't static, resulting in reallocating the buffer every time. (Alexei Alexandrov)

When using a Python "atexit" function it was not invoked when Vim exits. Now call Py_Finalize() for that. (Ugo Di Girolamo)
This breaks the thread stuff though, fixed by Ugo.

GTK GUI: using a .vimrc with "set cmdheight=2 lines=43" and ":split" right after startup, the window layout is messed up. (Michael Schaap) Added win_new_shellsize() call in gui_init() to fix the topframe size.

Trick to get ...MOUSE_NM not used when there are vertical splits. Now pass column -1 for the left most window and add MOUSE_COLOFF for others. Limits mouse column to 10000.

searchpair() may hang when the end pattern has "\zs" at the end. Check that we find the same position again and advance one character.

When in diff mode and making a change that causes the "changed" highlighting to disappear or reappear, it was still highlighted in another window.

When a ":next" command fails because the user selects "Abort" at the ATTENTION prompt the argument index was advanced anyway.

When "~" is in 'iskeyword' the "gd" doesn't work, it's used for the previous substitute pattern. Put "\V" in the pattern to avoid that.

Use of sprintf() sometimes didn't check properly for buffer overflow. Also when using smsg(). Included code for snprintf() to avoid having to do size checks where invoking them

":help \=<Tab>" didn't find "sub-replace-\=". Wild menu for help tags didn't show backslashes. ":he :s\=" didn't work.

When reading an errorfile "~/" in a file name was not expanded.

GTK GUI: When adding a scrollbar (e.g. when using ":vsplit") in a script or removing it the window size may change. GTK sends us resize events when we change the window size ourselves, but they may come at an unexpected moment. Peek for a character to get any window resize events and fix 'columns' and 'lines' to undo this.

When using the GTK plug mechanism, resizing and focus was not working properly. (Neil Bird)

After deleting files from the argument list a session file generated with ":mksession" may contain invalid ":next" commands.

When 'shortmess' is empty and 'keymap' set to accents, in Insert mode CTRL-N may cause the hit-enter prompt. Typing 'a then didn't result in the accented character. Put the character typed at the prompt back in the typeahead buffer so that mapping is done in the right mode.

setbufvar() and setwinvar() did not give error messages.

It was possible to set a variable with an illegal name, e.g. with setbufvar(). It was possible to define a function with illegal name, e.t. ":func F{-1}()"

CTRL-W F and "gf" didn't use the same method to get the file name.

When reporting a conversion error the line number of the last error could be given. Now report the first encountered error.

When using ":e ++enc=name file" and iconv() was used for conversion an error caused a fall-back to no conversion. Now replace a character with '?' and continue.

When opening a new buffer the local value of 'bomb' was not initialized from the global value.

Win32: When using the "Edit with Vim" entry the file name was limited to about 200 characters.

When using command line completion for ":e *foo" and the file "+foo" exists the resulting command ":e +foo" doesn't work. Now insert a backslash: ":e \+foo".

When the translation of "-- More --" was not 10 characters long the following message would be in the wrong position.

At the more-prompt the last character in the last line wasn't drawn.

When deleting non-existing text while 'virtualedit' is set the '[and '] marks were not set.

Win32: Could not use "**/" in 'path', it had to be "**\".

The search pattern "\n" did not match at the end of the last line.

Searching for a pattern backwards, starting on the NUL at the end of the line and 'encoding' is "utf-8" would match the pattern just before it incorrectly. Affected searchpair('/*', '', '*/').

For the Find/Replace dialog it was possible that not finding the text resulted in an error message while redrawing, which cleared the syntax highlighting while it was being used, resulting in a crash. Now don't clear syntax highlighting, disable it with b_syn_error.

Win32: Combining UTF-8 characters were drawn on the previous character. Could be noticed with a Thai font.

Output of ":function" could leave some of the typed text behind. (Yegappan Lakshmanan)

When the command line history has only a few lines the command line window would be opened with these lines above the first window line.

When using a command line window for search strings ":qa" would result in searching for "qa" instead of quitting all windows.

GUI: When scrolling with the scrollbar and there is a line that doesn't fit redrawing may fail. Make sure w_skipcol is valid before redrawing.

Limit the values of 'columns' and 'lines' to avoid an overflow in Rows * Columns. Fixed bad effects when running out of memory (command line would be reversed, ":qa!" resulted in ":!aq").

Motif: "gvim -iconic" opened the window anyway. (David Harrison)

There is a tiny chance that a symlink gets created between checking for an existing file and creating a file. Use the O_NOFOLLOW for open() if it's available.

In an empty line "ix<CTRL-O>0" moved the cursor to after the line instead of sticking to the first column.

When using ":wq" and a BufWriteCmd autocmd uses inputsecret() the text was echoed anyway. Set terminal to raw mode in getcmdline().

Unix: ":w a;b~c" caused an error in expanding wildcards.

When appending to a file with ":w >>fname" in a buffer without a name, causing the buffer to use "fname", the modified flag was reset.

When appending to the current file the "not edited" flag would be reset. ":w" would overwrite the file accidentally.

Unix: When filtering text with an external command Vim would still read input, causing text typed for the command (e.g., a password) to be eaten and echoed. Don't read input when the terminal is in cooked mode.

The Cygwin version of xxd used CR/LF line separators. (Corinna Vinschen)

Unix: When filtering text through a shell command some resulting text may be dropped. Now after detecting that the child has exited try reading some more of its output.

When inside input(), using "CTRL-R =" and the expression throws an exception the command line was not abandoned but it wasn't used either. Now abandon typing the command line.

'delcombine' was also used in Visual and Select mode and for commands like "cl". That was illogical and has been disabled.

When recording while a CursorHold autocommand was defined special keys would

appear in the register. Now the CursorHold event is not triggered while recording.

Unix: the src/configure script used ${srcdir-.}, not all shells understand that. Use ${srcdir:-.} instead.

When editing file "a" which is a symlink to file "b" that doesn't exist, writing file "a" to create "b" and then ":split b" resulted in two buffers on the same file with two different swapfile names. Now set the inode in the buffer when creating a new file.

When 'esckeys' is not set don't send the xterm code to request the version string, because it may cause trouble in Insert mode.

When evaluating an expression for CTRL-R = on the command line it was possible to call a function that opens a new window, resulting in errors for incremental search, and many other nasty things were possible. Now use the textlock to disallow changing the buffer or jumping to another window to protect from unexpected behavior. Same for CTRL-\ e.

"d(" deleted the character under the cursor, while the documentation specified an exclusive motion. Vi also doesn't delete the character under the cursor.

Shift-Insert in Insert mode could put the cursor before the last character when it just fits in the window. In coladvance() don't stop at the window edge when filling with spaces and when in Insert mode. In mswin.vim avoid getting a beep from the "l" command.

Win32 GUI: When Alt-F4 is used to close the window and Cancel is selected in the dialog then Vim would insert <M-F4> in the text. Now it's ignored.

When ":silent! {cmd}" caused the swap file dialog, which isn't displayed, there would still be a hit-enter prompt.

Requesting the termresponse (t_RV) early may cause problems with "-c" arguments that invoke an external command or even "-c quit". Postpone it until after executing "-c" arguments.

When typing in Insert mode so that a new line is started, using CTRL-G u to break undo and start a new change, then joining the lines with <BS> caused undo info to be missing. Now reset the insertion start point.

Syntax HL: When a region start match has a matchgroup and an offset that happens to be after the end of the line then it continued in the next line and stopped at the region end match, making the region continue after that.
Now check for the column being past the end of the line in syn_add_end_off().

When changing a file, setting 'swapfile' off and then on again, making another change and killing Vim, then some blocks may be missing from the swapfile.
When 'swapfile' is switched back on mark all blocks in the swapfile as dirty.
Added mf_set_dirty().

Expanding wildcards in a command like ":e aap;<>!" didn't work. Put backslashes before characters that are special to the shell. (Adri Verhoef)

A CursorHold autocommand would cause a message to be cleared. Don't show the special key for the event for 'showcmd'.

When expanding a file name for a shell command, as in "!cmd foo<Tab>" or ":r !cmd foo<Tab>" also escape characters that are special for the shell: "!;&()<>".

When the name of the buffer was set by a ":r fname" command cpo-f no autocommands were triggered to notify about the change in the buffer list.

In the quickfix buffer 'bufhidden' was set to "delete", which caused closing the quickfix window to leave an unlisted "No Name" buffer behind every time.

Win32: when using two screens of different size, setting 'lines' to a large value didn't fill the whole screen. (SungHyun Nam)

Win32 installer: The generated _vimrc contained an absolute path to diff.exe. After upgrading it becomes invalid. Now use $VIMRUNTIME instead.

The command line was cleared to often when 'showmode' was set and ":silent normal vy" was used. Don't clear the command line unless the mode was actually displayed. Added the "mode_displayed" variable.

The "load session" toolbar item could not handle a space or other special characters in v:this_session.

":set sta ts=8 sw=4 sts=2" deleted 4 spaces halfway a line instead of 2.

In a multi-byte file the foldmarker could be recognized in the trail byte. (Taro Muraoka)

Pasting with CTRL-V and menu didn't work properly when some commands are mapped. Use ":normal!" instead of ":normal". (Tony Apuzzo)

Crashed when expanding a file name argument in backticks.

In some situations the menu and scrollbar didn't work, when the value contains a CSI byte. (Yukihiro Nakadaira)

GTK GUI: When drawing the balloon focus changes and we might get a key release event that removed the balloon again. Ignore the key release event.

'titleold' was included in ":mkexrc" and ":mksession" files.

":set background&" didn't use the same logic as was used when starting up.

When "umask" is set such that nothing is writable then the viminfo file would be written without write permission. (Julian Bridle)

Motif: In diff mode dragging one scrollbar didn't update the scrollbar of the other diff'ed window.

When editing in an xterm with a different number of colors than expected the

screen would be cleared and redrawn, causing the message about the edited file to be cleared. Now set "keep_msg" to redraw the last message.

For a color terminal: When the Normal HL uses bold, possibly to make the color lighter, and another HL group specifies a color it might become light as well. Now reset bold if a HL group doesn't specify bold itself.

When using 256 color xterm the color 255 would show up as color 0. Use a short instead of a char to store the color number.

ml_get errors when searching for "\n\zs" in an empty file.

When selecting a block and using "$" to select until the end of every line and not highlighting the character under the cursor the first character of the block could be unhighlighted.

When counting words for the Visual block area and using "$" to select until the end of every line only up to the length of the last line was counted.

"dip" in trailing empty lines left one empty line behind.

The script ID was only remembered globally for each option. When a buffer- or window-local option was set the same "last set" location was changed for all buffers and windows. Now remember the script ID for each local option separately.

GUI: The "Replace All" button didn't handle backslashes in the replacement in the same way as "Replace". Escape backslashes so that they are taken literally.

When using Select mode from Insert mode and typing a key, causing lines to be deleted and a message displayed, delayed the effect of inserting the key. Now overwrite the message without delay.

When 'whichwrap' includes "l" then "dl" and "yl" on a single letter line worked differently. Now recognize all operators when using "l" at the end of a line.

GTK GUI: when the font selector returned a font name with a comma in it then it would be handled like two font names. Now put a backslash before the comma.

MS-DOS, Win32: When 'encoding' defaults to "latin1" then the value for 'iskeyword' was still for CPxxx. And when 'nocompatible' was set 'isprint' would also be the wrong value.

When a command was defined not to take arguments and no '|' no warning message would be given for using a '|'. Also with ":loadkeymap".

Motif: When using a fontset and 'encoding' is "utf-8" and sizeof(wchar_t) != sizeof(XChar2b) then display was wrong. (Yukihiro Nakadaira)

":all" always set the current window to the first window, even when it contains a buffer that is not in the argument list (can't be closed because it

is modified). Now go to the window that has the first item of the argument list.

GUI: To avoid left-over pixels from bold text all characters after a character with special attributes were redrawn. Now only do this for characters that actually are bold. Speeds up displaying considerably.

When only highlighting changes and the text is scrolled at the same time everything is redrawn instead of using a scroll and updating the changed text. E.g., when using ":match" to highlight a paren that the cursor landed on. Added SOME_VALID: Redraw the whole window but also try to scroll to minimize redrawing.

Win32: When using Korean IME making it active didn't work properly. (Moon, Yu-sung, 2005 March 21)

Ruby interface: when inserting/deleting lines display wasn't updated. (Ryan Paul)

--- fixes since Vim 7.0b ---

Getting the GCC version in configure didn't work with Solaris sed. First strip any "darwin." and then get the version number.

The "autoload" directory was missing from the self-installing executable for MS-Windows.

The MS-Windows install program would find "vimtutor.bat" in the install directory. After changing to "c:" also change to "\" to avoid looking in the install directory.

To make the 16 bit DOS version compile exclude not used highlight initializations and build a tiny instead of small version.

finddir() and findfile() accept a negative count and return a List then.

The Python indent file contained a few debugging statements, removed.

Expanding {} for a function name, resulting in a name starting with "s:" was not handled correctly.

Spelling: renamed COMPOUNDMAX to COMPOUNDWORDMAX. Added several items to be able to handle the new Hungarian dictionary.

Mac: Default to building for the current platform only, that is much faster than building a universal binary. Also, using Perl/Python/etc. only works for the current platform.

The time on undo messages disappeared for someone. Using %T for strftime() apparently doesn't work everywhere. Use %H:%M:%S instead.

Typing BS at the "z=" prompt removed the prompt.

--- fixes and changes since Vim 7.0c ---

When jumping to another tab page the Vim window size was always set, even when nothing in the layout changed.

Win32 GUI tab pages line wasn't always enabled. Do a proper check for the compiler version.

Win32: When switching between tab pages the Vim window was moved when part of it was outside of the screen. Now only do that in the direction of a size change.

Win32: added menu to GUI tab pages line. (Yegappan Lakshmanan)

Mac: Added document icons. (Benji Fisher)

Insert mode completion: Using Enter to accept the current match causes confusion. Use CTRL-Y instead. Also, use CTRL-E to go back to the typed text.

GUI: When there are left and right scrollbars, ":tabedit" kept them instead of using the one that isn't needed.

Using "gP" to replace al the text could leave the cursor below the last line, causing ml_get errors.

When 'cursorline' is set don't use the highlighting when Visual mode is active, otherwise it's difficult to see the selected area.

The matchparen plugin restricts the search to 100 lines, to avoid a long delay when there are closed folds.

Sometimes using CTRL-X s to list spelling suggestions used text from another line.

Win32: Set the default for 'isprint' back to the wrong default "@,~-255", because many people use Windows-1252 while 'encoding' is "latin1".

GTK: Added a workaround for gvim crashing when used over an untrusted ssh link, caused by GTK doing something nasty. (Ed Catmur)

Win32: The font used for the tab page labels is too big. Use the system menu font. (George Reilly)

Win32: Adjusting the window position and size to keep it on the screen didn't work properly when the taskbar is on the left or top of the screen.

The installman.sh and installml.sh scripts use ${10}, that didn't work with old shells. And use "test -f" instead of "test -e".

Win32: When 'encoding' was set in the vimrc then a directory argument for diff mode didn't work.

GUI: at the inputlist() prompt the cursorshape was adjusted as if the windows were still at their old position.

The parenmatch plugin didn't remember the highlighting per window.

Using ":bd" for a buffer that's the current window in another tab page caused a crash.

For a new tab page the 'scroll' option wasn't set to a good default.

Using an end offset for a search "/pat/e" didn't work properly for multi-byte text. (Yukihiro Nakadaira)

":s/\n/,/" doubled the text when used on the last line.

When "search" is in 'foldopen' "[s" and "]s" now open folds.

When using a numbered function "dict" can be omitted, but "self" didn't work then. Always add FC_DICT to the function flags when it's part of a dictionary.

When "--remote-tab" executes locally it left an empty tab page.

"gvim -u NONE", ":set cursorcolumn", "C" in the second line didn't update text. Do update further lines even though the "$" is displayed.

VMS: Support GTK better, also enable +clientserver. (Zoltan Arpadffy)

When highlighting of statusline or tabline is changed there was no redraw to show the effect.

Mac: Added "CFBundleIdentifier" to infplist.xml.

Added tabpage-local variables t:var.

Win32: Added double-click in tab pages line creates new tab. (Yegappan Lakshmanan)

Motif: Added GUI tab pages line. (Yegappan Lakshmanan)

Fixed crash when 'lines' was set to 1000 in a modeline.

When init_spellfile() finds a writable directory in 'runtimepath' but it doesn't contain a "spell" directory, create one.

Win32: executable() also finds "xxd" in the directory where Vim was started, but "!xxd" doesn't work. Append the Vim starting directory to $PATH.

The tab page labels are shortened, directory names are reduced to a single letter by default. Added the pathshorten() function to allow a user to do the same.

":saveas" now resets 'readonly' if the file was successfully written.

Set $MYVIMRC file to the first found .vimrc file.
Set $MYGVIMRC file to the first found .gvimrc file.

Added menu item "Startup Settings" that edits the $MYVIMRC file

Added matcharg().

Error message E745 appeared twice. Renamed one to E786.

Fixed crash when using "au BufRead * Sexplore" and doing ":help". Was wiping out a buffer that's still in a window.

":hardcopy" resulted in an error message when 'encoding' is "utf-8" and 'printencoding' is empty. Now it assumes latin1. (Mike Williams)

The check for the toolbar feature for Motif, depending on certain included files, wasn't detailed enough, causing building to fail in gui_xmebw.c.

Using CTRL-E in Insert mode completion after CTRL-P inserted the first match instead of the original text.

When displaying a UTF-8 character with a zero lower byte Vim might think the previous character is double-wide.

The "nbsp" item of 'listchars' didn't work when 'encoding' was utf-8.

Motif: when Xm/xpm.h is missing gui_xmebw.c would not compile.
HAVE_XM_UNHIGHLIGHTT_H was missing a T.

Mac: Moved the .icns files into src/os_mac_rsrc, so that they can all be copied at once. Adjusted the Info.plist file for three icons.

When Visual mode is active while switching to another tabpage could get ml_get errors.

When 'list' is set, 'nowrap' the $ in the first column caused 'cursorcolumn' to move to the right.

When a line wraps, 'cursorcolumn' was never displayed past the end of the line.

'autochdir' was only available when compiled with NetBeans and GUI. Now it's a separate feature, also available in the "big" version.

Added CTRL-W gf: open file under cursor in new tab page.

When using the menu in the tab pages line, "New Tab" opens the new tab before where the click was. Beyond the labels the new tab appears at the end instead of after the current tab page.

Inside a mapping with an expression getchar() could not be used.

When vgetc is used recursively vgetc_busy protects it from being used recursively. But after a ":normal" command the protection was reset.

":s/a/b/n" didn't work when 'modifiable' was off.

When $VIMRUNTIME includes a multi-byte character then rgb.txt could not be found. (Yukihiro Nakadaira)

":mkspell" didn't work correctly for non-ASCII affix flags when conversion is needed on the spell file.

glob('/dir/\$ABC/*') didn't work.

When using several tab pages and changing 'cmdheight' the display could become messed up. Now store the value of 'cmdheight' separately for each tab page.

The user of the Enter key while the popup menu is visible was still confusing. Now use Enter to select the match after using a cursor key.

Added "usetab" to 'switchbuf'.

--- fixes and changes since Vim 7.0d ---

Added CTRL-W T: move a window to a new tab page.

Using CTRL-X s in Insert mode to complete spelling suggestions and using BS deleted characters before the bad word.

A few small fixes for the VMS makefile. (Zoltan Arpadffy)

With a window of 91 lines 45 cols, ":vsp" scrolled the window. Copy w_wrow when splitting a window and skip setting the height when it's already at the right value.

Using <silent> in a mapping with a shell command and the GUI caused redraw to use wrong attributes.

Win32: Using MSVC 4.1 for install.exe resulted in the start menu items to be created in the administrator directory instead of "All Users". Define the CSIDL_ items if they are missing.

Motif: The GUI tabline did not use the space above the right scrollbar. Work around a bug in the Motif library. (Yegappan Lakshmanan)

The extra files for XML Omni completion are now also installed.
xml-omni-datafile

GTK GUI: when 'm' is missing from 'guioptions' during startup and pressing <F10> GTK produced error messages. Now do create the menu but disable it just after the first gui_mch_update().

":mkspell" doesn't work well with the Hungarian dictionary from the Hunspell project. Back to the Myspell dictionary.

In help files hide the | used around tags.

Renamed pycomplete to pythoncomplete.

Added "tabpages" to 'sessionoptions'.

When 'guitablabel' is set the effect wasn't visible right away.

Fixed a few 'cindent' errors.

When completing menu names, e.g., after ":emenu", don't sort the entries but keep them in the original order.

Fixed a crash when editing a directory in diff mode. Don't trigger autocommands when executing the diff command.

Getting a keystroke could get stuck if 'encoding' is a multi-byte encoding and typing a special key.

When 'foldignore' is set the folds were not updated right away.

When a list is indexed with [a : b] and b was greater than the length an error message was given. Now silently truncate the result.

When using BS during Insert mode completion go back to the original text, so that CTRL-N selects the first matching entry.

Added the 'M' flag to 'cinoptions'.

Win32: Make the "gvim --help" window appear in the middle of the screen instead of at an arbitrary position. (Randall W. Morris)

Added gettabwinvar() and settabwinvar().

Command line completion: pressing <Tab> after ":e /usr/*" expands the whole tree, because it becomes ":e /usr/**". Don't add a star if there already is one.

Added grey10 to grey90 to all GUIs, so that they can all be used for initializing highlighting. Use grey40 for CursorColumn and CursorLine when 'background' is "dark".

When reading a file and using iconv for conversion, an incomplete byte sequence at the end caused problems. (Yukihiro Nakadaira)

--- fixes and changes since Vim 7.0e ---

Default color for MatchParen when 'background' is "dark" is now DarkCyan.

":syn off" had to be used twice in a file that sets 'syntax' in a modeline. (Michael Geddes)

When using ":vsp" or ":sp" the available space wasn't used equally between windows. (Servatius Brandt)

Expanding <cWORD> on a trailing blank resulted in the first word in the line if 'encoding' is a multi-byte encoding.

Spell checking: spellbadword() didn't see a missing capital in the first word of a line. Popup menu now only suggest the capitalized word when appropriate.

When using whole line completion CTRL-L moves through the matches but it didn't work when at the original text.

When completion finds the longest match, don't go to the first match but stick at the original text, so that CTRL-N selects the first one.

Recognize "zsh-beta" like "zsh" for setting the 'shellpipe' default. (James Vega)

When using ":map <expr>" and the expression results in something with a special byte (NUL or CSI) then it didn't work properly. Now escape special bytes.

The default Visual highlighting for a color xterm with 8 colors was a magenta background, which made magenta text disappear. Now use reverse in this specific situation.

After completing the longest match "." didn't insert the same text. Repeating also didn't work correctly for multi-byte text.

When using Insert mode completion and BS the whole word that was completed would result in all possible matches. Now stop completion. Also fixes that for spell completion the previous word was deleted.

GTK: When 'encoding' is "latin1" and using non-ASCII characters in a file name the tab page label was wrong and an error message would be given.

The taglist() function could hang on a tags line with a non-ASCII character.

Win32: When 'encoding' differs from the system encoding tab page labels with non-ASCII characters looked wrong. (Yegappan Lakshmanan)

Motif: building failed when Xm/Notebook.h doesn't exist. Added a configure check, disable GUI tabline when it's missing.

Mac: When compiled without multi-byte feature the clipboard didn't work.

It was possible to switch to another tab page when the cmdline window is open.

Completion could hang when 'lines' is 6 and a preview window was opened.

Added CTRL-W gF: open file under cursor in new tab page and jump to the line number following the file name.
Added 'guitabtooltip'. Implemented for Win32 (Yegappan Lakshmanan).

Added "throw" to 'debug' option: throw an exception for error messages even whey they would otherwise be ignored.

When 'keymap' is set and a line contains an invalid entry could get a "No mapping found" warning instead of a proper error message.

Motif: default to using XpmAttributes instead of XpmAttributes_21.

A few more changes for 64 bit MS-Windows. (George Reilly)

Got ml_get errors when doing "o" and selecting in other window where there are
less lines shorter than the cursor position in the other window. ins_mouse()
was using position in wrong window.

Win32 GUI: Crash when giving a lot of messages during startup. Allocate twice
as much memory for the dialog template.

Fixed a few leaks and wrong pointer use reported by coverity.

When showing menus the mode character was sometimes wrong.

Added feedkeys(). (Yakov Lerner)

Made matchlist() always return all submatches.

Moved triggering QuickFixCmdPost to before jumping to the first location.

Mac: Added the 'macatsui' option as a temporary work around for text drawing
problems.

Line completion on "/**" gave error messages when scanning an unloaded buffer.

--- fixes and changes since Vim 7.0f ---

Win32: The height of the tab page labels is now adjusted to the font height.
(Yegappan Lakshmanan)

Win32: selecting the tab label was off by one. (Yegappan Lakshmanan)

Added tooltips for Motif and GTK tab page labels. (Yegappan Lakshmanan)

When 'encoding' is "utf-8" then ":help spell" would report an illegal byte and
the file was not converted from latin1 to utf-8. Now retry with latin1 if
reading the file as utf-8 results in illegal bytes.

Escape the argument of feedkeys() before putting it in the typeahead buffer.
(Yukihiro Nakadaira)

Added the v:char variable for evaluating 'formatexpr'. (Yukihiro Nakadaira)

With 8 colors Search highlighting combined with Statement highlighted text
made the text disappear.

VMS: avoid warnings for redefining MAX and MIN. (Zoltan Arpadffy)

When 'virtualedit' includes "onemore", stopping Visual selection would still
move the cursor left.

Prevent that using CTRL-R = in Insert mode can start Visual mode.

Fixed a crash that occurred when in Insert mode with completion active and a mapping caused edit() to be called recursively.

When using CTRL-O in Insert mode just after the last character while 'virtualedit' is "all", then typing CR moved the last character to the next line. Call coladvance() before starting the new line.

When using :shell ignore clicks on the tab page labels. Also when using the command line window.

When 'eventignore' is "all" then adding more to ignoring some events, e.g., for ":vimgrep", would actually trigger more events.

Win32: When a running Vim uses server name GVIM1 then "gvim --remote fname" didn't find it. When looking for a server name that doesn't end in a digit and it is not found then use another server with that name and a number (just like on Unix).

When using "double" in 'spellsuggest' when the language doesn't support sound folding resulted in too many suggestions.

Win32: Dropping a shortcut on the Vim icon didn't edit the referred file like editing it in another way would. Use fname_expand() in buf_set_name() instead of simply make the file name a full path.

Using feedkeys() could cause Vim to hang.

When closing another tab page from the tabline menu in Insert mode the tabline was not updated right away.

The syntax menu didn't work in compatible mode.

After using ":tag id" twice with the same "id", ":ts" and then ":pop" a ":ts" reported no matching tag. Clear the cached tag name.

In Insert mode the matchparen plugin highlighted the wrong paren when there is a string just next to a paren.

GTK: After opening a new tab page the text was sometimes not drawn correctly. Flush output and catch up with events when updating the tab page labels.

In the GUI, using CTRL-W q to close the last window of a tab page could cause a crash.

GTK: The tab pages line menu was not converted from 'encoding' to utf-8.

Typing a multi-byte character or a special key at the hit-enter prompt did not work.

When 'virtualedit' contains "onemore" CTRL-O in Insert mode still moved the cursor left when it was after the end of the line, even though it's allowed to be there.

Added test for using tab pages.

towupper() and towlower() were not used, because of checking for
__STDC__ISO_10646__ instead of __STDC_ISO_10646__. (sertacyildiz)

For ":map <expr>" forbid changing the text, jumping to another buffer and
using ":normal" to avoid nasty side effects.

--- fixes and changes since Vim 7.0g ---

Compilation error on HP-UX, use of "dlerr" must be inside a #ifdef.
(Gary Johnson)

Report +reltime feature in ":version" output.

The tar and zip plugins detect failure to get the contents of the archive and
edit the file as-is.

When the result of 'guitablabel' is empty fall back to the default label.

Fixed crash when using ":insert" in a while loop and missing "endwhile".

"gt" and other commands could move to another window when textlock active
and when the command line window was open.

Spell checking a file with syntax highlighting and a bad word at the end of
the line is ignored could make "]s" hang.

Mac: inputdialog() didn't work when compiled with big features.

Interrupting ":vimgrep" while it is busy loading a file left a modified and
hidden buffer behind. Use enter_cleanup() and leave_cleanup() around
wipe_buffer().

When making 'keymap' empty the b:keymap_name variable wasn't deleted.

Using CTRL-N that searches a long time, pressing space to interrupt the
searching and accept the first match, the popup menu was still displayed
briefly.

When setting the Vim window height with -geometry the 'window' option could be
at a value that makes CTRL-F behave differently.

When opening a quickfix window in two tabs they used different buffers,
causing redrawing problems later. Now use the same buffer for all quickfix
windows. (Yegappan Lakshmanan)

When 'mousefocus' is set moving the mouse to the text tab pages line would
move focus to the first window. Also, the mouse pointer would jump to the
active window.

In a session file, when an empty buffer is wiped out, do this silently.

When one window has the cursor on the last line and another window is resized

to make that window smaller, the cursor line could go below the displayed
lines. In win_new_height() subtract one from the available space.
Also avoid that using "~" lines makes the window scroll down.

Mac: When sourcing the "macmap.vim" script and then finding a .vimrc file the
'cpo' option isn't set properly, because it was already set and restored.
Added the <special> argument to ":map", so that 'cpo' doesn't need to be
changed to be able to use <> notation. Also do this for ":menu" for
consistency.

When using "/encoding=abc" in a spell word list, only "bc" was used.

When 'encoding' and 'printencoding' were both "utf-8" then ":hardcopy" didn't
work. (Mike Williams)

Mac: When building with "--disable-gui" the install directory would still be
"/Applications" and Vim.app would be installed. Now install in /usr/local as
usual for a console application.

GUI: when doing completion and there is one match and still searching for
another, the cursor was displayed at the end of the line instead of after the
match. Now show the cursor after the match while still searching for matches.

GUI: The mouse shape changed on the statusline even when 'mouse' was empty and
they can't be dragged.

GTK2: Selecting a button in the confirm() dialog with Tab or cursor keys and
hitting Enter didn't select that button. Removed GTK 1 specific code. (Neil
Bird)

When evaluating 'balloonexpr' takes a long time it could be called
recursively, which could cause a crash.

exists() could not be used to detect whether ":2match" is supported. Added a
check for it specifically.

GTK1: Tab page labels didn't work. (Yegappan Lakshmanan)

Insert mode completion: When finding matches use 'ignorecase', but when adding
matches to the list don't use it, so that all words with different case are
added, "word", "Word" and "WORD".

When 'cursorline' and 'hlsearch' are set and the search pattern is "x\n"
the rest of the line was highlighted as a match.

Cursor moved while evaluating 'balloonexpr' that invokes ":isearch" and
redirects the output. Don't move the cursor to the command line if msg_silent
is set.

exists() ignored text after a function name and option name, which could
result in false positives.

exists() ignored characters after the recognized word, which can be wrong when
using a name with non-keyword characters. Specifically, these calls no longer

allow characters after the name: exists('*funcname') exists('*funcname(...')
exists('&option') exists(':cmd') exists('g:name') exists('g:name[n]')
exists('g:name.n')

Trigger the TabEnter autocommand only after entering the current window of the
tab page, otherwise the commands are executed with an invalid current window.

Win32: When using two monitors and Vim is on the second monitor, changing the
width of the Vim window could make it jump to the first monitor.

When scrolling back at the more prompt and the quitting a line of text would
be left behind when 'cmdheight' is 2 or more.

Fixed a few things for Insert mode completion, especially when typing BS,
CTRL-N or a printable character while still searching for matches.

7.1 版 本 version-7.1 version7.1

本小节讨论 7.0 和 7.1 版本之间的改进。

这是一个漏洞修正版。没有什么突出的新特性。

改动 changed-7.1

---

vimrc_example.vim 里，增加了 'mouse' 的设置。

带 MZscheme 编译时同时寻找 "plt" 子目录下的头文件。FreeBSD 用到那里的文件。

Ruby 界面的模块名现在叫做 "Vim" 而非 "VIM"。但为了反向兼容起见，保留 "VIM" 为
其别名。 (Tim Pope)

新增 added-7.1

新语法文件:
```
        /var/log/messages (Yakov Lerner)
        Autohotkey (Nikolai Weibull)
        AutoIt v3 (Jared Breland)
        Bazaar commit file "bzr" (Dmitry Vasiliev)
        Cdrdao TOC (Nikolai Weibull)
        Cmusrc (Nikolai Weibull)
        Conary recipe (rPath Inc)
        Framescript (Nikolai Weibull)
        FreeBasic (Mark Manning)
        Hamster (David Fishburn)
        IBasic (Mark Manning)
        Initng (Elan Ruusamae)
        Ldapconf (Nikolai Weibull)
        Litestep (Nikolai Weibull)
```

Privoxy actions file (Doug Kearns)
Streaming Descriptors "sd" (Puria Nafisi Azizi)

新教程文件:
捷克语 (Lubos Turek)
匈牙利语 (Arpad Horvath)
土耳其语 (Serkan kkk)
希腊语教程的 utf-8 版本。
俄语教程的 utf-8 版本。
斯洛伐克语教程的 utf-8 版本。

新文件类型插件:
Bst (Tim Pope)
Cobol (Tim Pope)
Fvwm (Gautam Iyer)
Hamster (David Fishburn)
Django HTML 模板 (Dave Hodder)

新缩进文件:
Bst (Tim Pope)
Cobol (Tim Pope)
Hamster (David Fishburn)
Django HTML template (Dave Hodder)
Javascript
JSP (David Fishburn)

新键盘映射表:
保加利亚语 (Boyko Bantchev)
蒙古语 (Natsagdorj Shagdar)
马尔代夫语 (Ibrahim Fayaz)
越南语 (Samuel Thibault)

其它新运行时文件:
Ada 支持文件。(Neil Bird, Martin Krischik)
斯洛文尼亚语菜单翻译 (Mojca Miklavec)
Mono C# 编译器插件 (Jarek Sobiecki)

修正 fixed-7.1

Could not build the Win32s version. Added a few structure definitions in
src/gui_w32.c

Patch 7.0.001
问题: ":set spellsuggest+=10" does not work. (Suresh Govindachar)
解决方案: Add P_COMMA to the 'spellsuggest' flags.
相关文件: src/option.c

Patch 7.0.002
问题: C omni completion has a problem with tags files with a path
containing "#" or "%".

解决方案: Escape these characters. (Sebastian Baberowski)
相关文件: runtime/autoload/ccomplete.vim

Patch 7.0.003
问题: GUI: clicking in the lower part of a label in the tab pages line while 'mousefocus' is set may warp the mouse pointer. (Robert Webb)
解决方案: Check for a negative mouse position.
相关文件: src/gui.c

Patch 7.0.004
问题: Compiler warning for debug_saved used before set. (Todd Blumer)
解决方案: Remove the "else" for calling save_dbg_stuff().
相关文件: src/ex_docmd.c

Patch 7.0.005 (extra)
问题: Win32: The installer doesn't remove the "autoload" and "spell" directories. (David Fishburn)
解决方案: Add the directories to the list to be removed.
相关文件: nsis/gvim.nsi

Patch 7.0.006
问题: Mac: "make shadow" doesn't make a link for infplist.xml. (Axel Kielhorn)
解决方案: Make the link.
相关文件: src/Makefile

Patch 7.0.007
问题: AIX: compiling fails for message.c. (Ruediger Hornig)
解决方案: Move the #if outside of memchr().
相关文件: src/message.c

Patch 7.0.008
问题: Can't call a function that uses both <SID> and {expr}. (Thomas)
解决方案: Check both the expanded and unexpanded name for <SID>.
相关文件: src/eval.c

Patch 7.0.009
问题: ml_get errors with both 'sidescroll' and 'spell' set.
解决方案: Use ml_get_buf() instead of ml_get(), get the line from the right buffer, not the current one.
相关文件: src/spell.c

Patch 7.0.010
问题: The spellfile plugin required typing login name and password.
解决方案: Use "anonymous" and "vim7user" by default. No need to setup a .netrc file.
相关文件: runtime/autoload/spellfile.vim

Patch 7.0.011
问题: Can't compile without the folding and with the eval feature.
解决方案: Add an #ifdef. (Vallimar)
相关文件: src/option.c

Patch 7.0.012
问题: Using the matchparen plugin, moving the cursor in Insert mode to a shorter line that ends in a brace, changes the preferred column
解决方案: Use winsaveview()/winrestview() instead of getpos()/setpos().
相关文件: runtime/plugin/matchparen.vim

Patch 7.0.013
问题: Insert mode completion: using CTRL-L to add an extra character also deselects the current match, making it impossible to use CTRL-L a second time.
解决方案: Keep the current match. Also make CTRL-L work at the original text, using the first displayed match.
相关文件: src/edit.c

Patch 7.0.014
问题: Compiling gui_xmebw.c fails on Dec Alpha Tru64. (Rolfe)
解决方案: Disable some code for Motif 1.2 and older.
相关文件: src/gui_xmebw.c

Patch 7.0.015
问题: Athena: compilation problems with modern compiler.
解决方案: Avoid type casts for lvalue. (Alexey Froloff)
相关文件: src/gui_at_fs.c

Patch 7.0.016
问题: Printing doesn't work for "dec-mcs" encoding.
解决方案: Add "dec-mcs", "mac-roman" and "hp-roman8" to the list of recognized 8-bit encodings. (Mike Williams)
相关文件: src/mbyte.c

Patch 7.0.017 (after 7.0.014)
问题: Linking gui_xmebw.c fails on Dec Alpha Tru64. (Rolfe)
解决方案: Adjust defines for Motif 1.2 and older.
相关文件: src/gui_xmebw.c

Patch 7.0.018
问题: VMS: plugins are not loaded on startup.
解决方案: Remove "**" from the path. (Zoltan Arpadffy)
相关文件: src/main.c

Patch 7.0.019
问题: Repeating "VjA789" may cause a crash. (James Vega)
解决方案: Check the cursor column after moving it to another line.
相关文件: src/ops.c

Patch 7.0.020
问题: Crash when using 'mousefocus'. (William Fulton)
解决方案: Make buffer for mouse coordinates 2 bytes longer. (Juergen Weigert)
相关文件: src/gui.c

Patch 7.0.021
问题: Crash when using "\\[" and "\\]" in 'errorformat'. (Marc Weber)
解决方案: Check for valid submatches after matching the pattern.
相关文件: src/quickfix.c

Patch 7.0.022
问题: Using buffer.append() in Ruby may append the line to the wrong buffer. (Alex Norman)
解决方案: Properly switch to the buffer to do the appending. Also for buffer.delete() and setting a buffer line.
相关文件: src/if_ruby.c

Patch 7.0.023
问题: Crash when doing spell completion in an empty line and pressing CTRL-E.
解决方案: Check for a zero pointer. (James Vega)
Also handle a situation without a matching pattern better, report "No matches" instead of remaining in undefined CTRL-X mode. And get out of CTRL-X mode when typing a letter.
相关文件: src/edit.c

Patch 7.0.024
问题: It is possible to set arbitrary "v:" variables.
解决方案: Disallow setting "v:" variables that are not predefined.
相关文件: src/eval.c

Patch 7.0.025
问题: Crash when removing an element of a:000. (Nikolai Weibull)
解决方案: Mark the a:000 list with VAR_FIXED.
相关文件: src/eval.c

Patch 7.0.026
问题: Using libcall() may show an old error.
解决方案: Invoke dlerror() to clear a previous error. (Yukihiro Nakadaira)
相关文件: src/os_unix.c

Patch 7.0.027 (extra)
问题: Win32: When compiled with SNIFF gvim may hang on exit.
解决方案: Translate and dispatch the WM_USER message. (Mathias Michaelis)
相关文件: src/gui_w48.c

Patch 7.0.028 (extra)
问题: OS/2: Vim doesn't compile with gcc 3.2.1.
解决方案: Add argument to after_pathsep(), don't define vim_handle_signal(), define HAVE_STDARG_H. (David Sanders)
相关文件: src/os_unix.c, src/vim.h, src/os_os2_cfg.h

Patch 7.0.029
问题: getchar() may not position the cursor after a space.
解决方案: Position the cursor explicitly.
相关文件: src/eval.c

Patch 7.0.030
问题: The ":compiler" command can't be used in a FileChangedRO event. (Hari Krishna Dara)
解决方案: Add the CMDWIN flag to the ":compiler" command.
相关文件: src/ex_cmds.h

Patch 7.0.031
问题: When deleting a buffer the buffer-local mappings for Select mode remain.
解决方案: Add the Select mode bit to MAP_ALL_MODES. (Edwin Steiner)
相关文件: src/vim.h

Patch 7.0.032 (extra, after 7.0.027)
问题: Missing semicolon.
解决方案: Add the semicolon.
相关文件: src/gui_w48.c

Patch 7.0.033
问题: When pasting text, with the menu or CTRL-V, autoindent is removed.
解决方案: Use "x<BS>" to avoid indent to be removed. (Benji Fisher)
相关文件: runtime/autoload/paste.vim

Patch 7.0.034
问题: After doing completion and typing more characters or using BS repeating with "." didn't work properly. (Martin Stubenschrott)
解决方案: Don't put BS and other characters in the redo buffer right away, do this when finishing completion.
相关文件: src/edit.c

Patch 7.0.035
问题: Insert mode completion works when typed but not when replayed from a register. (Hari Krishna Dara)
Also: Mappings for Insert mode completion don't always work.
解决方案: When finding a non-completion key in the input don't interrupt completion when it wasn't typed.
Do use mappings when checking for typeahead while still finding completions. Avoids that completion is interrupted too soon.
Use "compl_pending" in a different way.
相关文件: src/edit.c

Patch 7.0.036
问题: Can't compile with small features and syntax highlighting or the diff feature.
解决方案: Define LINE_ATTR whenever syntax highlighting or the diff feature is enabled.
相关文件: src/screen.c

Patch 7.0.037
问题: Crash when resizing the GUI window vertically when there is a line that doesn't fit.
解决方案: Don't redraw while the screen data is invalid.
相关文件: src/screen.c

Patch 7.0.038
问题: When calling complete() from an Insert mode expression mapping text could be inserted in an improper way.
解决方案: Make undo_allowed() global and use it in complete().
相关文件: src/undo.c, src/proto/undo.pro, src/eval.c

Patch 7.0.039

问题:　　Calling inputdialog() with a third argument in the console doesn't work.
解决方案:　Make a separate function for input() and inputdialog(). (Yegappan Lakshmanan)
相关文件:　src/eval.c

Patch 7.0.040
问题:　　When 'cmdheight' is larger than 1 using inputlist() or selecting a spell suggestion with the mouse gets the wrong entry.
解决方案:　Start listing the first alternative on the last line of the screen.
相关文件:　src/eval.c, src/spell.c

Patch 7.0.041
问题:　　cursor([1, 1]) doesn't work. (Peter Hodge)
解决方案:　Allow leaving out the third item of the list and use zero for the virtual column offset.
相关文件:　src/eval.c

Patch 7.0.042
问题:　　When pasting a block of text in Insert mode Vim hangs or crashes. (Noam Halevy)
解决方案:　Avoid that the cursor is positioned past the NUL of a line.
相关文件:　src/ops.c

Patch 7.0.043
问题:　　Using "%!" at the start of 'statusline' doesn't work.
解决方案:　Recognize the special item when the option is being set.
相关文件:　src/option.c

Patch 7.0.044
问题:　　Perl: setting a buffer line in another buffer may result in changing the current buffer.
解决方案:　Properly change to the buffer to be changed.
相关文件:　src/if_perl.xs

Patch 7.0.045 (extra)
问题:　　Win32: Warnings when compiling OLE version with MSVC 2005.
解决方案:　Move including vim.h to before windows.h. (Ilya Bobir)
相关文件:　src/if_ole.cpp

Patch 7.0.046
问题:　　The matchparen plugin ignores parens in strings, but not in single quotes, often marked with "character".
解决方案:　Also ignore parens in syntax items matching "character".
相关文件:　runtime/plugin/matchparen.vim

Patch 7.0.047
问题:　　When running configure the exit status is wrong.
解决方案:　Handle the exit status properly. (Matthew Woehlke)
相关文件:　configure, src/configure

Patch 7.0.048
问题:　　Writing a compressed file fails when there are parens in the name. (Wang Jian)

解决方案: Put quotes around the temp file name.
相关文件: runtime/autoload/gzip.vim

Patch 7.0.049
问题: Some TCL scripts are not recognized. (Steven Atkinson)
解决方案: Check for "exec wish" in the file.
相关文件: runtime/scripts.vim

Patch 7.0.050
问题: After using the netbeans interface close command a stale pointer may be used.
解决方案: Clear the pointer to the closed buffer. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.0.051 (after 7.0.44)
问题: The Perl interface doesn't compile or doesn't work properly.
解决方案: Remove the spaces before #ifdef and avoid an empty line above it.
相关文件: src/if_perl.xs

Patch 7.0.052
问题: The user may not be aware that the Vim server allows others more functionality than desired.
解决方案: When running Vim as root don't become a Vim server without an explicit --servername argument.
相关文件: src/main.c

Patch 7.0.053
问题: Shortening a directory name may fail when there are multi-byte characters.
解决方案: Copy the correct bytes. (Titov Anatoly)
相关文件: src/misc1.c

Patch 7.0.054
问题: Mac: Using a menu name that only has a mnemonic or accelerator causes a crash. (Elliot Shank)
解决方案: Check for an empty menu name. Also delete empty submenus that were created before detecting the error.
相关文件: src/menu.c

Patch 7.0.055
问题: ":startinsert" in a CmdwinEnter autocommand doesn't take immediate effect. (Bradley White)
解决方案: Put a NOP key in the typeahead buffer. Also avoid that using CTRL-C to go back to the command line moves the cursor left.
相关文件: src/edit.c, src/ex_getln.c

Patch 7.0.056
问题: "#!something" gives an error message.
解决方案: Ignore this line, so that it can be used in an executable Vim script.
相关文件: src/ex_docmd.c

Patch 7.0.057 (extra, after 7.0.45)
问题: Win32: Compilation problem with Borland C 5.5.

解决方案: Include vim.h as before. (Mark S. Williams)
相关文件: src/if_ole.cpp

Patch 7.0.058
问题: The gbk and gb18030 encodings are not recognized.
解决方案: Add aliases to cp936. (Edward L. Fox)
相关文件: src/mbyte.c

Patch 7.0.059
问题: The Perl interface doesn't compile with ActiveState Perl 5.8.8.
解决方案: Remove the __attribute__() items. (Liu Yubao)
相关文件: src/if_perl.xs

Patch 7.0.060 (after 7.0.51)
问题: Code for temporarily switching to another buffer is duplicated in
           quite a few places.
解决方案: Use aucmd_prepbuf() and aucmd_restbuf() also when FEAT_AUTOCMD is
           not defined.
相关文件: src/buffer.c, src/eval.c, src/fileio.c, src/if_ruby.c,
           src/if_perl.xs, src/quickfix.c, src/structs.h

Patch 7.0.061
问题: Insert mode completion for Vim commands may crash if there is
           nothing to complete.
解决方案: Instead of freeing the pattern make it empty, so that a "not
           found" error is given. (Yukihiro Nakadaira)
相关文件: src/edit.c

Patch 7.0.062
问题: Mac: Crash when using the popup menu for spell correction. The
           popup menu appears twice when letting go of the right mouse button
           early.
解决方案: Don't show the popup menu on the release of the right mouse
           button. Also check that a menu pointer is actually valid.
相关文件: src/proto/menu.pro, src/menu.c, src/normal.c, src/term.c

Patch 7.0.063
问题: Tiny chance for a memory leak. (coverity)
解决方案: Free pointer when next memory allocation fails.
相关文件: src/eval.c

Patch 7.0.064
问题: Using uninitialized variable. (Tony Mechelynck)
解决方案: When not used set "temp" to zero. Also avoid a warning for
           "files" in ins_compl_dictionaries().
相关文件: src/edit.c

Patch 7.0.065 (extra)
问题: Mac: left-right movement of the scrollwheel causes up-down
           scrolling.
解决方案: Ignore mouse wheel events that are not up-down. (Nicolas Weber)
相关文件: src/gui_mac.c

Patch 7.0.066

问题:　　After the popup menu for Insert mode completion overlaps the tab pages line it is not completely removed.
解决方案:　Redraw the tab pages line after removing the popup menu. (Ori Avtalion)
相关文件:　src/popupmnu.c

Patch 7.0.067
问题:　　Undo doesn't always work properly when using "scim" input method. Undo is split up when using preediting.
解决方案:　Reset xim_has_preediting also when preedit_start_col is not MAXCOL. Don't split undo when <Left> is used while preediting. (Yukihiro Nakadaira)
相关文件:　src/edit.c, src/mbyte.c

Patch 7.0.068
问题:　　When 'ignorecase' is set and using Insert mode completion, typing characters to change the list of matches, case is not ignored. (Hugo Ahlenius)
解决方案:　Store the 'ignorecase' flag with the matches where needed.
相关文件:　src/edit.c, src/search.c, src/spell.c

Patch 7.0.069
问题:　　Setting 'guitablabel' to %!expand(\%) causes Vim to free an invalid pointer. (Kim Schulz)
解决方案:　Don't try freeing a constant string pointer.
相关文件:　src/buffer.c

Patch 7.0.070
问题:　　Compiler warnings for shadowed variables and uninitialized variables.
解决方案:　Rename variables such as "index", "msg" and "dup". Initialize variables.
相关文件:　src/edit.c, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/gui_beval.c, src/gui_gtk.c, src/gui_gtk_x11.c, src/hardcopy.c, src/if_cscope.c, src/main.c, src/mbyte.c, src/memline.c, src/netbeans.c, src/normal.c, src/option.c, src/os_unix.c, src/quickfix.c, src/regexp.c, src/screen.c, src/search.c, src/spell.c, src/ui.c, src/undo.c, src/window.c, src/version.c

Patch 7.0.071
问题:　　Using an empty search pattern may cause a crash.
解决方案:　Avoid using a NULL pointer.
相关文件:　src/search.c

Patch 7.0.072
问题:　　When starting the GUI fails there is no way to adjust settings or do something else.
解决方案:　Add the GUIFailed autocommand event.
相关文件:　src/fileio.c, src/gui.c, src/vim.h

Patch 7.0.073
问题:　　Insert mode completion: Typing <CR> sometimes selects the original text instead of keeping what was typed. (Justin Constantino)

解决方案: Don't let <CR> select the original text if there is no popup menu.
相关文件: src/edit.c

Patch 7.0.074 (extra)
问题: Win32: tooltips were not converted from 'encoding' to Unicode.
解决方案: Set the tooltip to use Unicode and do the conversion. Also cleanup the code for the tab pages tooltips. (Yukihiro Nakadaira)
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 7.0.075
问题: winsaveview() did not store the actual value of the desired cursor column. This could move the cursor in the matchparen plugin.
解决方案: Call update_curswant() before using the value w_curswant.
相关文件: src/eval.c

Patch 7.0.076 (after 7.0.010)
问题: Automatic downloading of spell files only works for ftp.
解决方案: Don't add login and password for non-ftp URLs. (Alexander Patrakov)
相关文件: runtime/autoload/spellfile.vim

Patch 7.0.077
问题: ":unlet v:this_session" causes a crash. (Marius Roets)
解决方案: When trying to unlet a fixed variable give an error message.
相关文件: src/eval.c

Patch 7.0.078
问题: There are two error messages E46.
解决方案: Change the number for the sandbox message to E794.
相关文件: src/globals.h

Patch 7.0.079
问题: Russian tutor doesn't work when 'encoding' is "utf-8".
解决方案: Use tutor.ru.utf-8 as the master, and generate the other encodings from it. Select the right tutor depending on 'encoding'. (Alexey Froloff)
相关文件: runtime/tutor/Makefile, runtime/tutor/tutor.vim, runtime/tutor/tutor.ru.utf-8

Patch 7.0.080
问题: Generating auto/pathdef.c fails for CFLAGS with a backslash.
解决方案: Double backslashes in the string. (Alexey Froloff)
相关文件: src/Makefile

Patch 7.0.081
问题: Command line completion doesn't work for a shell command with an absolute path.
解决方案: Don't use $PATH when there is an absolute path.
相关文件: src/ex_getln.c

Patch 7.0.082
问题: Calling a function that waits for input may cause List and Dictionary arguments to be freed by the garbage collector.
解决方案: Keep a list of all arguments to internal functions.
相关文件: src/eval.c

Patch 7.0.083
问题: Clicking with the mouse on an item for inputlist() doesn't work when 'compatible' is set and/or when 'cmdheight' is more than one. (Christian J. Robinson)
解决方案: Also decrement "lines_left" when 'more' isn't set. Set "cmdline_row" to zero to get all mouse events.
相关文件: src/message.c, src/misc1.c

Patch 7.0.084
问题: The garbage collector may do its work while some Lists or Dictionaries are used internally, e.g., by ":echo" that runs into the more-prompt or ":echo [garbagecollect()]".
解决方案: Only do garbage collection when waiting for a character at the toplevel. Let garbagecollect() set a flag that is handled at the toplevel before waiting for a character.
相关文件: src/eval.c, src/getchar.c, src/globals.h, src/main.c

Patch 7.0.085
问题: When doing "make test" the viminfo file is modified.
解决方案: Use another viminfo file after setting 'compatible'.
相关文件: src/testdir/test56.in

Patch 7.0.086
问题: getqflist() returns entries for pattern and text with the number zero. Passing these to setqflist() results in the string "0".
解决方案: Use an empty string instead of the number zero.
相关文件: src/quickfix.c

Patch 7.0.087
问题: After ":file fname" and ":saveas fname" the 'autochdir' option does not take effect. (Yakov Lerner)
Commands for handling 'autochdir' are repeated many times.
解决方案: Add the DO_AUTOCHDIR macro and do_autochdir(). Use it for ":file fname" and ":saveas fname".
相关文件: src/proto/buffer.pro, src/buffer.c, src/ex_cmds.c, src/macros.h, src/netbeans.c, src/option.c, src/window.c

Patch 7.0.088
问题: When compiled with Perl the generated prototypes have "extern" unnecessarily added.
解决方案: Remove the "-pipe" argument from PERL_CFLAGS.
相关文件: src/auto/configure, src/configure.in

Patch 7.0.089
问题: "ga" does not work properly for a non-Unicode multi-byte encoding.
解决方案: Only check for composing chars for utf-8. (Taro Muraoka)
相关文件: src/ex_cmds.c

Patch 7.0.090
问题: Cancelling the conform() dialog on the console with Esc requires typing it twice. (Benji Fisher)
解决方案: When the start of an escape sequence is found use 'timeoutlen' or 'ttimeoutlen'.

相关文件: src/misc1.c

Patch 7.0.091
问题: Using winrestview() while 'showcmd' is set causes the cursor to be displayed in the wrong position. (Yakov Lerner)
解决方案: Set the window topline properly.
相关文件: src/eval.c

Patch 7.0.092 (after 7.0.082 and 7.0.084)
问题: The list of internal function arguments is obsolete now that garbage collection is only done at the toplevel.
解决方案: Remove the list of all arguments to internal functions.
相关文件: src/eval.c

Patch 7.0.093
问题: The matchparen plugin can't handle a 'matchpairs' value where a colon is matched.
解决方案: Change the split() that is used to change 'matchpairs' into a List.
相关文件: runtime/plugin/matchparen.vim

Patch 7.0.094
问题: When a hidden buffer is made the current buffer and another file edited later, the file message will still be given. Using ":silent" also doesn't prevent the file message. (Marvin Renich)
解决方案: Reset the need_fileinfo flag when reading a file. Don't set need_fileinfo when msg_silent is set.
相关文件: src/buffer.c, src/fileio.c

Patch 7.0.095
问题: The Greek tutor is not available in utf-8. "el" is used for the language, only "gr" for the country is recognized.
解决方案: Add the utf-8 Greek tutor. Use it for conversion to iso-8859-7 and cp737. (Lefteris Dimitroulakis)
相关文件: runtime/tutor/Makefile, runtime/tutor/tutor.gr.utf-8, runtime/tutor/tutor.vim

Patch 7.0.096
问题: taglist() returns the filename relative to the tags file, while the directory of the tags file is unknown. (Hari Krishna Dara)
解决方案: Expand the file name. (Yegappan Lakshmanan)
相关文件: src/tag.c

Patch 7.0.097
问题: ":tabclose N" that closes another tab page does not remove the tab pages line. Same problem when using the mouse.
解决方案: Adjust the tab pages line when needed in tabpage_close_other().
相关文件: src/ex_docmd.c

Patch 7.0.098
问题: Redirecting command output in a cmdline completion function doesn't work. (Hari Krishna Dara)
解决方案: Enable redirection when redirection is started.
相关文件: src/ex_docmd.c, src/ex_getln.c

Patch 7.0.099
问题: GUI: When the popup menu is visible using the scrollbar messes up the display.
解决方案: Disallow scrolling the current window. Redraw the popup menu after scrolling another window.
相关文件: src/gui.c

Patch 7.0.100
问题: "zug" may report the wrong filename. (Lawrence Kesteloot)
解决方案: Call home_replace() to fill NameBuff[].
相关文件: src/spell.c

Patch 7.0.101
问题: When the "~/.vim/spell" directory does not exist "zg" may create a wrong directory. "zw" doesn't work.
解决方案: Use the directory of the file name instead of NameBuff. For "zw" not only remove a good word but also add the word with "!".
相关文件: src/spell.c

Patch 7.0.102
问题: Redrawing cmdline is not correct when using SCIM.
解决方案: Don't call im_get_status(). (Yukihiro Nakadaira)
相关文件: src/ex_getln.c

Patch 7.0.103 (after 7.0.101)
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Init variable.
相关文件: src/spell.c

Patch 7.0.104
问题: The CursorHoldI event only triggers once in Insert mode. It also triggers after CTRL-V and other two-key commands.
解决方案: Set "did_cursorhold" before getting a second key. Reset "did_cursorhold" after handling a command.
相关文件: src/edit.c, src/fileio.c

Patch 7.0.105
问题: When using incremental search the statusline ruler isn't updated. (Christoph Koegl)
解决方案: Update the statusline when it contains the ruler.
相关文件: src/ex_getln.c

Patch 7.0.106
问题: The spell popup menu uses ":amenu", triggering mappings. Other PopupMenu autocommands are removed. (John Little)
解决方案: Use ":anoremenu" and use an autocmd group.
相关文件: runtime/menu.vim

Patch 7.0.107
问题: Incremental search doesn't redraw the text tabline. (Ilya Bobir) Also happens in other situations with one window in a tab page.
解决方案: Redraw the tabline after clearing the screen.
相关文件: src/screen.c

Patch 7.0.108 (extra)
问题: Amiga: Compilation problem.
解决方案: Have mch_mkdir() return a failure flag. (Willy Catteau)
相关文件: src/os_amiga.c, src/proto/os_amiga.pro

Patch 7.0.109
问题: Lisp indenting is confused by escaped quotes in strings. (Dorai Sitaram)
解决方案: Check for backslash inside strings. (Sergey Khorev)
相关文件: src/misc1.c

Patch 7.0.110
问题: Amiga: Compilation problems when not using libnix.
解决方案: Change a few #ifdefs. (Willy Catteau)
相关文件: src/memfile.c

Patch 7.0.111
问题: The gzip plugin can't handle filenames with single quotes.
解决方案: Add and use the shellescape() function. (partly by Alexey Froloff)
相关文件: runtime/autoload/gzip.vim, runtime/doc/eval.txt, src/eval.c, src/mbyte.c, src/misc2.c, src/proto/misc2.pro

Patch 7.0.112
问题: Python interface does not work with Python 2.5.
解决方案: Change PyMem_DEL() to Py_DECREF(). (Sumner Hayes)
相关文件: src/if_python.c

Patch 7.0.113
问题: Using CTRL-L in Insert completion when there is no current match may cause a crash. (Yukihiro Nakadaira)
解决方案: Check for compl_leader to be NULL
相关文件: src/edit.c

Patch 7.0.114
问题: When aborting an insert with CTRL-C an extra undo point is created in the GUI. (Yukihiro Nakadaira)
解决方案: Call gotchars() only when advancing.
相关文件: src/getchar.c

Patch 7.0.115
问题: When 'ignorecase' is set, Insert mode completion only adds "foo" and not "Foo" when both are found.
A found match isn't displayed right away when 'completeopt' does not have "menu" or "menuone".
解决方案: Do not ignore case when checking if a completion match already exists. call ins_compl_check_keys() also when not using a popup menu. (Yukihiro Nakadaira)
相关文件: src/edit.c

Patch 7.0.116
问题: 64 bit Windows version reports "32 bit" in the ":version" output. (M. Veerman)
解决方案: Change the text for Win64.

相关文件: src/version.c

Patch 7.0.117
问题: Using "extend" on a syntax item inside a region with "keepend", an
intermediate item may be truncated.
When applying the "keepend" and there is an offset to the end
pattern the highlighting of a contained item isn't adjusted.
解决方案: Use the seen_keepend flag to remember when to apply the "keepend"
flag. Adjust the keepend highlighting properly. (Ilya Bobir)
相关文件: src/syntax.c

Patch 7.0.118
问题: printf() does not do zero padding for strings.
解决方案: Do allow zero padding for strings.
相关文件: src/message.c

Patch 7.0.119
问题: When going back from Insert to Normal mode the CursorHold event
doesn't trigger. (Yakov Lerner)
解决方案: Reset "did_cursorhold" when leaving Insert mode.
相关文件: src/edit.c

Patch 7.0.120
问题: Crash when using CTRL-R = at the command line and entering
"getreg('=')". (James Vega)
解决方案: Avoid recursiveness of evaluating the = register.
相关文件: src/ops.c

Patch 7.0.121
问题: GUI: Dragging the last status line doesn't work when there is a
text tabline. (Markus Wolf)
解决方案: Take the text tabline into account when deciding to start modeless
selection.
相关文件: src/gui.c

Patch 7.0.122
问题: GUI: When clearing after a bold, double-wide character half a
character may be drawn.
解决方案: Check for double-wide character and redraw it. (Yukihiro Nakadaira)
相关文件: src/screen.c

Patch 7.0.123
问题: On SCO Openserver configure selects the wrong terminal library.
解决方案: Put terminfo before the other libraries. (Roger Cornelius)
Also fix a small problem compiling on Mac without Darwin.
相关文件: src/configure.in, src/auto/configure

Patch 7.0.124
问题: getwinvar() obtains a dictionary with window-local variables, but
it's always for the current window.
解决方案: Get the variables of the specified window. (Geoff Reedy)
相关文件: src/eval.c

Patch 7.0.125

问题:　　When "autoselect" is in the 'clipboard' option then the '< and '>
　　marks are set while Visual mode is still active.
解决方案:　Don't set the '< and '> marks when yanking the selected area for
　　the clipboard.
相关文件:　src/normal.c

Patch 7.0.126
问题:　　When 'formatexpr' uses setline() and later internal formatting is
　　used undo information is not correct. (Jiri Cerny, Benji Fisher)
解决方案:　Set ins_need_undo after using 'formatexpr'.
相关文件:　src/edit.c

Patch 7.0.127
问题:　　Crash when swap files has invalid timestamp.
解决方案:　Check return value of ctime() for being NULL.
相关文件:　src/memline.c

Patch 7.0.128
问题:　　GUI: when closing gvim is cancelled because there is a changed
　　buffer the screen isn't updated to show the changed buffer in the
　　current window. (Krzysztof Kacprzak)
解决方案:　Redraw when closing gvim is cancelled.
相关文件:　src/gui.c

Patch 7.0.129
问题:　　GTK GUI: the GTK file dialog can't handle a relative path.
解决方案:　Make the initial directory a full path before passing it to GTK.
　　(James Vega) Also postpone adding the default file name until
　　after setting the directory.
相关文件:　src/gui_gtk.c

Patch 7.0.130 (extra)
问题:　　Win32: Trying to edit or write devices may cause Vim to get stuck.
解决方案:　Add the 'opendevice' option, default off. Disallow
　　reading/writing from/to devices when it's off.
　　Also detect more devices by the full name starting with "\\.\".
相关文件:　runtime/doc/options.txt, src/fileio.c, src/option.c, src/option.h,
　　src/os_win32.c

Patch 7.0.131
问题:　　Win32: "vim -r" does not list all the swap files.
解决方案:　Also check for swap files starting with a dot.
相关文件:　src/memline.c

Patch 7.0.132 (after 7.0.130)
问题:　　Win32: Crash when Vim reads from stdin.
解决方案:　Only use mch_nodetype() when there is a file name.
相关文件:　src/fileio.c

Patch 7.0.133
问题:　　When searching included files messages are added to the history.
解决方案:　Set msg_hist_off for messages about scanning included files.
　　Set msg_silent to avoid message about wrapping around.
相关文件:　src/edit.c, src/globals.h, src/message.c, src/search.c

Patch 7.0.134
问题: Crash when comparing a recursively looped List or Dictionary.
解决方案: Limit recursiveness for comparing to 1000.
相关文件: src/eval.c

Patch 7.0.135
问题: Crash when garbage collecting list or dict with loop.
解决方案: Don't use DEL_REFCOUNT but don't recurse into Lists and Dictionaries when freeing them in the garbage collector. Also add allocated Dictionaries to the list of Dictionaries to avoid leaking memory.
相关文件: src/eval.c, src/proto/eval.pro, src/tag.c

Patch 7.0.136
问题: Using "O" while matching parens are highlighted may not remove the highlighting. (Ilya Bobir)
解决方案: Also trigger CursorMoved when a line is inserted under the cursor.
相关文件: src/misc1.c

Patch 7.0.137
问题: Configure check for big features is wrong.
解决方案: Change "==" to "=". (Martti Kuparinen)
相关文件: src/auto/configure, src/configure.in

Patch 7.0.138 (extra)
问题: Mac: modifiers don't work with function keys.
解决方案: Use GetEventParameter() to obtain modifiers. (Nicolas Weber)
相关文件: src/gui_mac.c

Patch 7.0.139
问题: Using CTRL-PageUp or CTRL-PageDown in Insert mode to go to another tab page does not prepare for undo properly. (Stefano Zacchiroli)
解决方案: Call start_arrow() before switching tab page.
相关文件: src/edit.c

Patch 7.0.140 (after 7.0.134)
问题: Comparing recursively looped List or Dictionary doesn't work well.
解决方案: Detect comparing a List or Dictionary with itself.
相关文件: src/eval.c

Patch 7.0.141
问题: When pasting a while line on the command line an extra CR is added literally.
解决方案: Don't add the trailing CR when pasting with the mouse.
相关文件: src/ex_getln.c, src/proto/ops.pro, src/ops.c

Patch 7.0.142
问题: Using the middle mouse button in Select mode to paste text results in an extra "y". (Kriton Kyrimis)
解决方案: Let the middle mouse button replace the selected text with the contents of the clipboard.
相关文件: src/normal.c

Patch 7.0.143
问题: Setting 'scroll' to its default value was not handled correctly.
解决方案: Compare the right field to PV_SCROLL.
相关文件: src/option.c

Patch 7.0.144
问题: May compare two unrelated pointers when matching a pattern against a string. (Dominique Pelle)
解决方案: Avoid calling reg_getline() when REG_MULTI is false.
相关文件: src/regexp.c

Patch 7.0.145 (after 7.0.142)
问题: Compiler warning.
解决方案: Add type cast.
相关文件: src/normal.c

Patch 7.0.146
问题: When 'switchbuf' is set to "usetab" and the current tab has only a quickfix window, jumping to an error always opens a new window. Also, when the buffer is open in another tab page it's not found.
解决方案: Check for the "split" value of 'switchbuf' properly. Search in other tab pages for the desired buffer. (Yegappan Lakshmanan)
相关文件: src/buffer.c, src/quickfix.c

Patch 7.0.147
问题: When creating a session file and there are several tab pages and some windows have a local directory a short file name may be used when it's not valid. (Marius Roets)
A session with multiple tab pages may result in "No Name" buffers. (Bill McCarthy)
解决方案: Don't enter tab pages when going through the list, only use a pointer to the first window in each tab page.
Use "tabedit" instead of "tabnew | edit" when possible.
相关文件: src/ex_docmd.c

Patch 7.0.148
问题: When doing "call a.xyz()" and "xyz" does not exist in dictionary "a" there is no error message. (Yegappan Lakshmanan)
解决方案: Add the error message.
相关文件: src/eval.c

Patch 7.0.149
问题: When resizing a window that shows "~" lines the text sometimes jumps down.
解决方案: Remove code that uses "~" lines in some situations. Fix the computation of the screen line of the cursor. Also set w_skipcol to handle very long lines.
相关文件: src/misc1.c, src/window.c

Patch 7.0.150
问题: When resizing the Vim window scrollbinding doesn't work. (Yakov Lerner)
解决方案: Do scrollbinding in set_shellsize().
相关文件: src/term.c

Patch 7.0.151
问题: Buttons in file dialog are not according to Gnome guidelines.
解决方案: Swap Cancel and Open buttons. (Stefano Zacchiroli)
相关文件: src/gui_gtk.c

Patch 7.0.152
问题: Crash when using lesstif 2.
解决方案: Fill in the extension field. (Ben Hutchings)
相关文件: src/gui_xmebw.c

Patch 7.0.153
问题: When using cscope and opening the temp file fails Vim crashes.
(Kaya Bekiroglu)
解决方案: Check for NULL pointer returned from mch_open().
相关文件: src/if_cscope.c

Patch 7.0.154
问题: When 'foldnestmax' is negative Vim can hang. (James Vega)
解决方案: Avoid the fold level becoming negative.
相关文件: src/fold.c, src/syntax.c

Patch 7.0.155
问题: When getchar() returns a mouse button click there is no way to get
the mouse coordinates.
解决方案: Add v:mouse_win, v:mouse_lnum and v:mouse_col.
相关文件: runtime/doc/eval.txt, src/eval.c, src/vim.h

Patch 7.0.156 (extra)
问题: Vim doesn't compile for Amiga OS 4.
解决方案: Various changes for Amiga OS4. (Peter Bengtsson)
相关文件: src/feature.h, src/mbyte.c, src/memfile.c, src/memline.c,
src/os_amiga.c, src/os_amiga.h, src/pty.c

Patch 7.0.157
问题: When a function is used recursively the profiling information is
invalid. (Mikolaj Machowski)
解决方案: Put the start time on the stack instead of in the function.
相关文件: src/eval.c

Patch 7.0.158
问题: In a C file with ":set foldmethod=syntax", typing {<CR> on the
last line results in the cursor being in a closed fold. (Gautam
Iyer)
解决方案: Open fold after inserting a new line.
相关文件: src/edit.c

Patch 7.0.159
问题: When there is an I/O error in the swap file the cause of the error
cannot be seen.
解决方案: Use PERROR() instead of EMSG() where possible.
相关文件: src/memfile.c

Patch 7.0.160

问题:　　":@a" echoes the command, Vi doesn't do that.
解决方案:　Set the silent flag in the typeahead buffer to avoid echoing the command.
相关文件:　src/ex_docmd.c, src/normal.c, src/ops.c, src/proto/ops.pro

Patch 7.0.161
问题:　　Win32: Tab pages line popup menu isn't using the right encoding. (Yongwei Wu)
解决方案:　Convert the text when necessary. Also fixes the Find/Replace dialog title. (Yegappan Lakshmanan)
相关文件:　src/gui_w48.c

Patch 7.0.162
问题:　　"vim -o a b" when file "a" triggers the ATTENTION dialog, selecting "Quit" exits Vim instead of editing "b" only.
When file "b" triggers the ATTENTION dialog selecting "Quit" or "Abort" results in editing file "a" in that window.
解决方案:　When selecting "Abort" exit Vim. When selecting "Quit" close the window. Also avoid hit-enter prompt when selecting Abort.
相关文件:　src/buffer.c, src/main.c

Patch 7.0.163
问题:　　Can't retrieve the position of a sign after it was set.
解决方案:　Add the netbeans interface getAnno command. (Xavier de Gaye)
相关文件:　runtime/doc/netbeans.txt, src/netbeans.c

Patch 7.0.164
问题:　　":redir @+" doesn't work.
解决方案:　Accept "@+" just like "@*". (Yegappan Lakshmanan)
相关文件:　src/ex_docmd.c

Patch 7.0.165
问题:　　Using CTRL-L at the search prompt adds a "/" and other characters without escaping, causing the pattern not to match.
解决方案:　Escape special characters with a backslash.
相关文件:　src/ex_getln.c

Patch 7.0.166
问题:　　Crash in cscope code when connection could not be opened. (Kaya Bekiroglu)
解决方案:　Check for the file descriptor to be NULL.
相关文件:　src/if_cscope.c

Patch 7.0.167
问题:　　":function" redefining a dict function doesn't work properly. (Richard Emberson)
解决方案:　Allow a function name to be a number when it's a function reference.
相关文件:　src/eval.c

Patch 7.0.168
问题:　　Using uninitialized memory and memory leak. (Dominique Pelle)
解决方案:　Use alloc_clear() instead of alloc() for w_lines. Free b_ml.ml_stack after recovery.

相关文件: src/memline.c, src/window.c

Patch 7.0.169
问题: With a Visual block selection, with the cursor in the left upper corner, pressing "I" doesn't remove the highlighting. (Guopeng Wen)
解决方案: When checking if redrawing is needed also check if Visual selection is still active.
相关文件: src/screen.c

Patch 7.0.170 (extra)
问题: Win32: Using "gvim --remote-tab foo" when gvim is minimized while it previously was maximized, un-maximizing doesn't work properly. And the labels are not displayed properly when 'encoding' is utf-8.
解决方案: When minimized check for SW_SHOWMINIMIZED. When updating the tab pages line use TCM_SETITEMW instead of TCM_INSERTITEMW. (Liu Yubao)
相关文件: src/gui_w48.c

Patch 7.0.171 (extra)
问题: VMS: A file name with multiple paths is written in the wrong file.
解决方案: Get the actually used file name. (Zoltan Arpadffy)
Also add info to the :version command about compilation.
相关文件: src/Make_vms.mms, src/buffer.c, src/os_unix.c, src/version.c

Patch 7.0.172
问题: Crash when recovering and quitting at the "press-enter" prompt.
解决方案: Check for "msg_list" to be NULL. (Liu Yubao)
相关文件: src/ex_eval.c

Patch 7.0.173
问题: ":call f().TT()" doesn't work. (Richard Emberson)
解决方案: When a function returns a Dictionary or another composite continue evaluating what follows.
相关文件: src/eval.c

Patch 7.0.174
问题: ":mksession" doesn't restore window layout correctly in tab pages other than the current one. (Zhibin He)
解决方案: Use the correct topframe for producing the window layout commands.
相关文件: src/ex_docmd.c

Patch 7.0.175
问题: The result of tr() is missing the terminating NUL. (Ingo Karkat)
解决方案: Add the NUL.
相关文件: src/eval.c

Patch 7.0.176
问题: ":emenu" isn't executed directly, causing the encryption key prompt to fail. (Life Jazzer)
解决方案: Fix wrong #ifdef.
相关文件: src/menu.c

Patch 7.0.177
问题: When the press-enter prompt gets a character from a non-remappable mapping, it's put back in the typeahead buffer as remappable, which may cause an endless loop.
解决方案: Restore the non-remappable flag and the silent flag when putting a char back in the typeahead buffer.
相关文件: src/getchar.c, src/message.c, src/normal.c

Patch 7.0.178
问题: When 'enc' is "utf-8" and 'ignorecase' is set the result of ":echo ("\xe4" == "\xe4")" varies.
解决方案: In mb_strnicmp() avoid looking past NUL bytes.
相关文件: src/mbyte.c

Patch 7.0.179
问题: Using ":recover" or "vim -r" without a swapfile crashes Vim.
解决方案: Check for "buf" to be unequal NULL. (Yukihiro Nakadaira)
相关文件: src/memline.c

Patch 7.0.180 (extra, after 7.0.171)
问题: VMS: build failed. Problem with swapfiles.
解决方案: Add "compiled_arch". Always expand path and pass it to buf_modname(). (Zoltan Arpadffy)
相关文件: src/globals.h, src/memline.c, src/os_unix.c, runtime/menu.vim

Patch 7.0.181
问题: When reloading a file that starts with an empty line, the reloaded buffer has an extra empty line at the end. (Motty Lentzitzky)
解决方案: Delete all lines, don't use bufempty().
相关文件: src/fileio.c

Patch 7.0.182
问题: When using a mix of undo and "g-" it may no longer be possible to go to every point in the undo tree. (Andy Wokula)
解决方案: Correctly update pointers in the undo tree.
相关文件: src/undo.c

Patch 7.0.183
问题: Crash in ":let" when redirecting to a variable that's being displayed. (Thomas Link)
解决方案: When redirecting to a variable only do the assignment when stopping redirection to avoid that setting the variable causes a freed string to be accessed.
相关文件: src/eval.c

Patch 7.0.184
问题: When the cscope program is called "mlcscope" the Cscope interface doesn't work.
解决方案: Accept "\S*cscope:" instead of "cscope:". (Frodak D. Baksik)
相关文件: src/if_cscope.c

Patch 7.0.185
问题: Multi-byte characters in a message are displayed with attributes from what comes before it.

解决方案: Don't use the attributes for a multi-byte character. Do use
           attributes for special characters. (Yukihiro Nakadaira)
相关文件: src/message.c

Patch 7.0.186
问题: Get an ml_get error when 'encoding' is "utf-8" and searching for
           "/_s*/e" in an empty buffer. (Andrew Maykov)
解决方案: Don't try getting the line just below the last line.
相关文件: src/search.c

Patch 7.0.187
问题: Can't source a remote script properly.
解决方案: Add the SourceCmd event. (Charles Campbell)
相关文件: runtime/doc/autocmd.txt, src/ex_cmds2.c, src/fileio.c, src/vim.h

Patch 7.0.188 (after 7.0.186)
问题: Warning for wrong pointer type.
解决方案: Add a type cast.
相关文件: src/search.c

Patch 7.0.189
问题: Translated message about finding matches is truncated. (Yukihiro
           Nakadaira)
解决方案: Enlarge the buffer. Also use vim_snprintf().
相关文件: src/edit.c

Patch 7.0.190
问题: "syntax spell default" results in an error message.
解决方案: Change 4 to 7 for STRNICMP(). (Raul Nunez de Arenas Coronado)
相关文件: src/syntax.c

Patch 7.0.191
问题: The items used by getqflist() and setqflist() don't match.
解决方案: Support the "bufnum" item for setqflist(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c

Patch 7.0.192
问题: When 'swapfile' is switched off in an empty file it is possible
           that not all blocks are loaded into memory, causing ml_get errors
           later.
解决方案: Rename "dont_release" to "mf_dont_release" and also use it to
           avoid using the cached line and locked block.
相关文件: src/globals.h, src/memfile.c, src/memline.c

Patch 7.0.193
问题: Using --remote or --remote-tab with an argument that matches
           'wildignore' causes a crash.
解决方案: Check the argument count before using ARGLIST[0].
相关文件: src/ex_cmds.c

Patch 7.0.194
问题: Once an ml_get error is given redrawing part of the screen may
           cause it again, resulting in an endless loop.
解决方案: Don't give the error message for a recursive call.

相关文件: src/memline.c

Patch 7.0.195
问题: When a buffer is modified and 'autowriteall' is set, ":quit"
results in an endless loop when there is a conversion error while
writing. (Nikolai Weibull)
解决方案: Make autowrite() return FAIL if the buffer is still changed after
writing it.
/* put the cursor on the last char, for 'tw' formatting */
相关文件: src/ex_cmds2.c

Patch 7.0.196
问题: When using ":vert ball" the computation of the mouse pointer
position may be off by one column. (Stefan Karlsson)
解决方案: Recompute the frame width when moving the vertical separator from
one window to another.
相关文件: src/window.c

Patch 7.0.197 (extra)
问题: Win32: Compiling with EXITFREE doesn't work.
解决方案: Adjust a few #ifdefs. (Alexei Alexandrof)
相关文件: src/misc2.c, src/os_mswin.c

Patch 7.0.198 (extra)
问题: Win32: Compiler warnings. No need to generate gvim.exe.mnf.
解决方案: Add type casts. Use "*" for processorArchitecture. (George Reilly)
相关文件: src/Make_mvc.mak, src/eval.c, src/gvim.exe.mnf, src/misc2.c

Patch 7.0.199
问题: When using multi-byte characters the combination of completion and
formatting may result in a wrong cursor position.
解决方案: Don't decrement the cursor column, use dec_cursor(). (Yukihiro
Nakadaira) Also check for the column to be zero.
相关文件: src/edit.c

Patch 7.0.200
问题: Memory leaks when out of memory.
解决方案: Free the memory.
相关文件: src/edit.c, src/diff.c

Patch 7.0.201
问题: Message for ":diffput" about buffer not being in diff mode may be
wrong.
解决方案: Check for buffer in diff mode but not modifiable.
相关文件: src/diff.c

Patch 7.0.202
问题: Problems on Tandem systems while compiling and at runtime.
解决方案: Recognize root uid is 65535. Check select() return value for it
not being supported. Avoid wrong function prototypes. Mention
use of -lfloss. (Matthew Woehlke)
相关文件: src/Makefile, src/ex_cmds.c, src/fileio.c, src/main.c,
src/osdef1.h.in, src/osdef2.h.in, src/os_unix.c, src/pty.c,
src/vim.h

Patch 7.0.203
问题: 0x80 characters in a register are not handled correctly for the "@" command.
解决方案: Escape CSI and 0x80 characters. (Yukihiro Nakadaira)
相关文件: src/ops.c

Patch 7.0.204
问题: Cscope: Parsing matches for listing isn't done properly.
解决方案: Check for line number being found. (Yu Zhao)
相关文件: src/if_cscope.c

Patch 7.0.205 (after 7.0.203)
问题: Can't compile.
解决方案: Always include the vim_strsave_escape_csi function.
相关文件: src/getchar.c

Patch 7.0.206 (after 7.0.058)
问题: Some characters of the "gb18030" encoding are not handled properly.
解决方案: Do not use "cp936" as an alias for "gb18030" encoding. Instead initialize 'encoding' to "cp936".
相关文件: src/mbyte.c, src/option.c

Patch 7.0.207
问题: After patch 2.0.203 CSI and K_SPECIAL characters are escaped when recorded and then again when the register is executed.
解决方案: Remove escaping before putting the recorded characters in a register. (Yukihiro Nakadaira)
相关文件: src/getchar.c, src/ops.c, src/proto/getchar.pro

Patch 7.0.208 (after 7.0.171 and 7.0.180)
问题: VMS: changes to path handling cause more trouble than they solve.
解决方案: Revert changes.
相关文件: src/buffer.c, src/memline.c, src/os_unix.c

Patch 7.0.209
问题: When replacing a line through Python the cursor may end up beyond the end of the line.
解决方案: Check the cursor column after replacing the line.
相关文件: src/if_python.c

Patch 7.0.210
问题: ":cbuffer" and ":lbuffer" always fail when the buffer is modified. (Gary Johnson)
解决方案: Support adding a !. (Yegappan Lakshmanan)
相关文件: runtime/doc/quickfix.txt, src/ex_cmds.h

Patch 7.0.211
问题: With ":set cindent noai bs=0" using CTRL-U in Insert mode will delete auto-indent. After ":set ai" it doesn't.
解决方案: Also check 'cindent' being set. (Ryan Lortie)
相关文件: src/edit.c

Patch 7.0.212
问题:　　The GUI can't be terminated with SIGTERM. (Mark Logan)
解决方案:　Use the signal protection in the GUI as in the console, allow
　　signals when waiting for 100 msec or longer.
相关文件:　src/ui.c

Patch 7.0.213
问题:　　When 'spellfile' has two regions that use the same sound folding
　　using "z=" will cause memory to be freed twice. (Mark Woodward)
解决方案:　Clear the hashtable properly so that the items are only freed once.
相关文件:　src/spell.c

Patch 7.0.214
问题:　　When using <f-args> in a user command it's not possible to have an
　　argument end in '\ '.
解决方案:　Change the handling of backslashes. (Yakov Lerner)
相关文件:　runtime/doc/map.txt, src/ex_docmd.c

Patch 7.0.215 (extra)
问题:　　Mac: Scrollbar size isn't set. Context menu has disabled useless
　　Help entry. Call to MoreMasterPointers() is ignored.
解决方案:　Call SetControlViewSize() in gui_mch_set_scrollbar_thumb(). Use
　　kCMHelpItemRemoveHelp for ContextualMenuSelect(). Remove call to
　　MoreMasterPointers(). (Nicolas Weber)
相关文件:　src/gui_mac.c

Patch 7.0.216
问题:　　":tab wincmd]" does not open a tab page. (Tony Mechelynck)
解决方案:　Copy the cmdmod.tab value to postponed_split_tab and use it.
相关文件:　src/globals.h, src/ex_docmd.c, src/if_cscope.c, src/window.c

Patch 7.0.217
问题:　　This hangs when pressing "n": ":%s/\n/,\r/gc". (Ori Avtalion)
解决方案:　Set "skip_match" to advance to the next line.
相关文件:　src/ex_cmds.c

Patch 7.0.218
问题:　　"%B" in 'statusline' always shows zero in Insert mode. (DervishD)
解决方案:　Remove the exception for Insert mode, check the column for being
　　valid instead.
相关文件:　src/buffer.c

Patch 7.0.219
问题:　　When using the 'editexisting.vim' script and a file is being
　　edited in another tab page the window is split. The "+123"
　　argument is not used.
解决方案:　Make the tab page with the file the current tab page. Set
　　v:swapcommand when starting up to the first "+123" or "-c" command
　　line argument.
相关文件:　runtime/macros/editexisting.vim, src/main.c

Patch 7.0.220
问题:　　Crash when using winnr('#') in a new tab page. (Andy Wokula)
解决方案:　Check for not finding the window.

相关文件: src/eval.c

Patch 7.0.221
问题: finddir() uses 'path' by default, where "." means relative to the current file. But it works relative to the current directory. (Tye Zdrojewski)
解决方案: Add the current buffer name to find_file_in_path_option() for the relative file name.
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.0.222
问题: Perl indenting using 'cindent' works almost right.
解决方案: Recognize '#' to start a comment. (Alex Manoussakis) Added '#' flag in 'cinoptions'.
相关文件: runtime/doc/indent.txt, src/misc1.c

Patch 7.0.223
问题: Unprintable characters in completion text mess up the popup menu. (Gombault Damien)
解决方案: Use strtrans() to make the text printable.
相关文件: src/charset.c, src/popupmnu.c

Patch 7.0.224
问题: When expanding "##" spaces are escaped twice. (Pavol Juhas)
解决方案: Don't escape the spaces that separate arguments.
相关文件: src/eval.c, src/ex_docmd.c, src/proto/ex_docmd.pro

Patch 7.0.225
问题: When using setline() in an InsertEnter autocommand and doing "A" the cursor ends up on the last byte in the line. (Yukihiro Nakadaira)
解决方案: Only adjust the column when using setline() for the cursor line. Move it back to the head byte if necessary.
相关文件: src/eval.c, src/misc2.c

Patch 7.0.226
问题: Display flickering when updating signs through the netbeans interface. (Xavier de Gaye)
解决方案: Remove the redraw_later(CLEAR) call.
相关文件: src/netbeans.c

Patch 7.0.227
问题: Crash when closing a window in the GUI. (Charles Campbell)
解决方案: Don't call out_flush() from win_free().
相关文件: src/window.c

Patch 7.0.228
问题: Cygwin: problem with symlink to DOS style path.
解决方案: Invoke cygwin_conv_to_posix_path(). (Luca Masini)
相关文件: src/os_unix.c

Patch 7.0.229
问题: When 'pastetoggle' starts with Esc then pressing Esc in Insert mode will not time out. (Jeffery Small)

解决方案: Use KL_PART_KEY instead of KL_PART_MAP, so that 'ttimeout' applies
           to the 'pastetoggle' key.
相关文件: src/getchar.c

Patch 7.0.230
问题: After using ":lcd" a script doesn't know how to restore the
           current directory.
解决方案: Add the haslocaldir() function. (Bob Hiestand)
相关文件: runtime/doc/usr_41.txt, runtime/doc/eval.txt, src/eval.c

Patch 7.0.231
问题: When recovering from a swap file the page size is likely to be
           different from the minimum. The block used for the first page
           then has a buffer of the wrong size, causing a crash when it's
           reused later. (Zephaniah Hull)
解决方案: Reallocate the buffer when the page size changes. Also check that
           the page size is at least the minimum value.
相关文件: src/memline.c

Patch 7.0.232 (extra)
问题: Mac: doesn't support GUI tab page labels.
解决方案: Add GUI tab page labels. (Nicolas Weber)
相关文件: src/feature.h, src/gui.c, src/gui.h, src/gui_mac.c,
           src/proto/gui_mac.pro

Patch 7.0.233 (extra)
问题: Mac: code formatted badly.
解决方案: Fix code formatting
相关文件: src/gui_mac.c

Patch 7.0.234
问题: It's possible to use feedkeys() from a modeline. That is a
           security issue, can be used for a trojan horse.
解决方案: Disallow using feedkeys() in the sandbox.
相关文件: src/eval.c

Patch 7.0.235
问题: It is possible to use writefile() in the sandbox.
解决方案: Add a few more checks for the sandbox.
相关文件: src/eval.c

Patch 7.0.236
问题: Linux 2.4 uses sysinfo() with a mem_unit field, which is not
           backwards compatible.
解决方案: Add an autoconf check for sysinfo.mem_unit. Let mch_total_mem()
           return Kbyte to avoid overflow.
相关文件: src/auto/configure, src/configure.in, src/config.h.in,
           src/option.c, src/os_unix.c

Patch 7.0.237
问题: For root it is recommended to not use 'modeline', but in
           not-compatible mode the default is on.
解决方案: Let 'modeline' default to off for root.
相关文件: runtime/doc/options.txt, src/option.c

Patch 7.0.238
问题: Crash when ":match" pattern runs into 'maxmempattern'. (Yakov Lerner)
解决方案: Don't free the regexp program of match_hl.
相关文件: src/screen.c

Patch 7.0.239
问题: When using local directories and tab pages ":mksession" uses a short file name when it shouldn't. Window-local options from a modeline may be applied to the wrong window. (Teemu Likonen)
解决方案: Add the did_lcd flag, use the full path when it's set. Don't use window-local options from the modeline when using the current window for another buffer in ":doautoall".
相关文件: src/fileio.c, src/ex_docmd.c

Patch 7.0.240
问题: Crash when splitting a window in the GUI. (opposite of 7.0.227)
解决方案: Don't call out_flush() from win_alloc(). Also avoid this for win_delete(). Also block autocommands while the window structure is invalid.
相关文件: src/window.c

Patch 7.0.241
问题: ":windo throw 'foo'" loops forever. (Andy Wokula)
解决方案: Detect that win_goto() doesn't work.
相关文件: src/ex_cmds2.c

Patch 7.0.242 (extra)
问题: Win32: Using "-register" in a Vim that does not support OLE causes a crash.
解决方案: Don't use EMSG() but mch_errmsg(). Check p_go for being NULL. (partly by Michael Wookey)
相关文件: src/gui_w32.c

Patch 7.0.243 (extra)
问题: Win32: When GvimExt is built with MSVC 2005 or later, the "Edit with vim" context menu doesn't appear in the Windows Explorer.
解决方案: Embed the linker manifest file into the resources of GvimExt.dll. (Mathias Michaelis)
相关文件: src/GvimExt/Makefile

Fixes after Vim 7.1a BETA:

The extra archive had CVS directories included below "farsi" and "runtime/icons". CVS was missing the farsi icon files.

Fix compiling with Gnome 2.18, undefine bind_textdomain_codeset. (Daniel Drake)

Mac: "make install" didn't copy rgb.txt.

When editing a compressed file while there are folds caused "ml_get" errors

and some lines could be missing. When decompressing failed option values were not restored.

Patch 7.1a.001
问题: Crash when downloading a spell file. (Szabolcs Horvat)
解决方案: Avoid that did_set_spelllang() is used recursively when a new window is opened for the download.
Also avoid wiping out the wrong buffer.
相关文件: runtime/autoload/spellfile.vim, src/buffer.c, src/ex_cmds.c, src/spell.c

Patch 7.1a.002 (extra)
问题: Compilation error with MingW.
解决方案: Check for LPTOOLTIPTEXT to be defined.
相关文件: src/gui_w32.c

Fixes after Vim 7.1b BETA:

Made the Mzscheme interface build both with old and new versions of Mzscheme, using an #ifdef. (Sergey Khorev)
Mzscheme interface didn't link, missing function. Changed order of libraries in the configure script.

Ruby interface didn't compile on Mac. Changed #ifdef. (Kevin Ballard)

Patch 7.1b.001 (extra)
问题: Random text in a source file. No idea how it got there.
解决方案: Delete the text.
相关文件: src/gui_w32.c

Patch 7.1b.002
问题: When 'maxmem' is large there can be an overflow in computations. (Thomas Wiegner)
解决方案: Use the same mechanism as in mch_total_mem(): first reduce the multiplier as much as possible.
相关文件: src/memfile.c

7.2 版 本 version-7.2 version7.2

本小节讨论 7.1 和 7.2 版本之间的改进。

这主要是一个漏洞修正版。主要的新特性是浮点数支持。 Float

改动 changed-7.2

把命令行缓冲区的名字从 "command-line" 改为 "[Command Line]"。

删除了 ":caddexpr"、":cgetexpr"、":cgetfile"、":laddexpr"、":lgetexpr" 和 ":lgetfile" 命令可选的 !。没有必要。(Yegappan Lakshmanan)

语法匹配的位移过去是字节数而不是字符数。这很不一致而且很容易弄错。现在使用字符位移。
(Yukihiro Nakadaira)

FileChangedShellPost 事件本来在文件没有改变的时候也会给出。
(John Little)

如果当前行过长（放不下），会看不见弹出菜单。现在它在屏幕行而不是文本行下方显示。
(Francois Ingelrest)

切换到 autoconf 2.62 版本。

把 fcntl.h 的包含移到了 vim.h，从所有的 .c 文件中删除其引用。

引入 STRMOVE(d, s) 宏，类似于 STRCPY()，用于有重叠的字符串。用此宏就不需要 mch_memmove(p, p + x, STRLEN(p + x) + 1)。

删除 bulgarian.vim 键盘映射文件，它被两个更标准的文件代替 (Boyko Bantchev)

命令行补全的标签最大匹配数目从 200 增加到 300 个。

帮助文件 sql.txt 被换名为 ft_sql.txt，而 ada.txt 也被更名为 ft_ada.txt。

新增 added-7.2

新语法文件:

- CUDA (Timothy B. Terriberry)
- Cdrdao config (Nikolai Weibull)
- Coco/R (Ashish Shukla)
- Denyhosts config (Nikolai Weibull)
- Dtrace script (Nicolas Weber)
- Git output、commit、config、rebase、send-email (Tim Pope)
- HASTE 和 HastePreProc (M. Tranchero)
- Haml (Tim Pope)
- Host conf (Nikolai Weibull)
- Linden script (Timo Frenay)
- MS messages (Kevin Locke)
- PDF (Tim Pope)
- ProMeLa (Maurizio Tranchero)
- Reva Foth (Ron Aaron)
- Sass (Tim Pope)
- Symbian meta-makefile、MMP (Ron Aaron)
- VOS CM macro (Andrew McGill)
- XBL (Doug Kearns)

新教程文件:

- 创建了所有教程文件的 UTF-8 版本。
- 希腊文把 ".gr" 换名为 ".el" (Greek vs Greece)。
- 世界语 (Dominique Pelle)
- 克罗地亚语 (Paul B. Mahol)

新文件类型插件:
	Cdrdao config (Nikolai Weibull)
	Debian control files (Debian Vim 维护者)
	Denyhosts (Nikolai Weibull)
	Dos .ini file (Nikolai Weibull)
	Dtrace script (Nicolas Weber)
	FnameScript (Nikolai Weibull)
	Git、Git config、Git commit、Git rebase、Git send-email (Tim Pope)
	Haml (Tim Pope)
	Host conf (Nikolai Weibull)
	Host access (Nikolai Weibull)
	Logtalk (Paulo Moura)
	MS messages (Kevin Locke)
	NSIS script (Nikolai Weibull)
	PDF (Tim Pope)
	Reva Forth (Ron Aaron)
	Sass (Tim Pope)

新缩进文件:
	DTD (Nikolai Weibull)
	Dtrace script (Nicolas Weber)
	Erlang (Csaba Hoch)
	FrameScript (Nikolai Weibull)
	Git config (Tim Pope)
	Haml (Tim Pope)
	Logtalk (Paulo Moura)
	Sass (Tim Pope)
	Tiny Fugue (Christian J. Robinson)

新编译器插件:
	RSpec (Tim Pope)

新键盘映射表:
	克罗地亚语 (Paul B. Mahol)
	俄罗斯语 Dvorak (Serhiy Boiko)
	乌克兰语 Dvorak (Serhiy Boiko)
	删除简单保加利亚语，"bds" 和 phonetic 足够了。

其它新运行时文件:
	世界语菜单和消息翻译。(Dominique Pelle)
	芬兰语菜单和消息翻译。(Flammie Pirinen)
	巴西葡萄牙语菜单和消息翻译。(Eduardo Dobay)

加入浮点数支持。 Float

mode() 接受参数以返回关于当前模式更详细的信息。(Ben Schmidt)

加入 BSD 控制台鼠标的支持： sysmouse 。(Paul B. Mahol)

'switchbuf' 选项新增 "newtab" 值。(部分由 Yegappan Lakshmanan 提供)

改进了 netbeans 接口的错误信息。(Philippe Fremy)

加入 screen 使用 xterm 鼠标代码的支持。(Micah Cowan)

加入跨平台编译的支持:
调整了 configure.in 并新增了 INSTALLcross.txt。(Marc Haisenko) 之后修正了 configure.in 的错误。
configure 中不使用 /usr/local/include 和 /usr/local/lib。(Philip Prindeville)
为了在 Unix 上跨平台编译 Cygwin 版本，在 src/vim.rc 中把 VIM.TLB 换名为 vim.tlb。(Tsuneo Nakagawa)

加入 v:searchforward 变量: 当前搜索的方向。(Yakov Lerner)

修正 fixed-7.2

Patch 7.1.001
问题: Still can't build with Gnome libraries.
解决方案: Fix typo in bind_textdomain_codeset. (Mike Kelly)
相关文件: src/gui_gtk.c, src/gui_gtk_x11.c

Patch 7.1.002
问题: Oracle Pro*C/C++ files are not detected.
解决方案: Add the missing star. (Micah J. Cowan)
相关文件: runtime/filetype.vim

Patch 7.1.003 (extra)
问题: The "Tear off this menu" message appears in the message history when using a menu. (Yongwei Wu)
解决方案: Disable message history when displaying the menu tip.
相关文件: src/gui_w32.c

Patch 7.1.004
问题: Crash when doing ":next directory". (Raphael Finkel)
解决方案: Do not use "buf", it may be invalid after autocommands.
相关文件: src/ex_cmds.c

Patch 7.1.005
问题: "cit" used on <foo></foo> deletes <foo>. Should not delete anything and start insertion, like "ci'" does on "". (Michal Bozon)
解决方案: Handle an empty object specifically. Made it work consistent for various text objects.
相关文件: src/search.c

Patch 7.1.006
问题: Resetting 'modified' in a StdinReadPost autocommand doesn't work.
解决方案: Set 'modified' before the autocommands instead of after it.
相关文件: src/buffer.c

Patch 7.1.007 (extra)
问题: Mac: Context menu doesn't work on Intel Macs.
Scrollbars are not dimmed when Vim is not the active application.
解决方案: Remove the test whether context menus are supported. They are always there in OS/X. Handle the dimming. (Nicolas Weber)

相关文件: src/gui_mac.c, src/gui.h

Patch 7.1.008
问题: getfsize() returns a negative number for very big files.
解决方案: Check for overflow and return -2.
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.1.009
问题: In diff mode, displaying the difference between a tab and spaces is not highlighted correctly.
解决方案: Only change highlighting at the end of displaying a tab.
相关文件: src/screen.c

Patch 7.1.010
问题: The Gnome session file doesn't restore tab pages.
解决方案: Add SSOP_TABPAGES to the session flags. (Matias D'Ambrosio)
相关文件: src/gui_gtk_x11.c

Patch 7.1.011
问题: Possible buffer overflow when $VIMRUNTIME is very long. (Victor Stinner)
解决方案: Use vim_snprintf().
相关文件: src/main.c

Patch 7.1.012
问题: ":let &shiftwidth = 'asdf'" doesn't produce an error message.
解决方案: Check for a string argument. (Chris Lubinski)
相关文件: src/option.c

Patch 7.1.013
问题: ":syn include" only loads the first file, while it is documented as doing the equivalent of ":runtime!".
解决方案: Change the argument to source_runtime(). (James Vega)
相关文件: src/syntax.c

Patch 7.1.014
问题: Crash when doing C indenting. (Chris Monson)
解决方案: Obtain the current line again after invoking cin_islabel().
相关文件: src/edit.c

Patch 7.1.015
问题: MzScheme interface: current-library-collection-paths produces no list. Interface doesn't build on a Mac.
解决方案: Use a list instead of a pair. (Bernhard Fisseni) Use "-framework" argument for MZSCHEME_LIBS in configure.
相关文件: src/configure.in, src/if_mzsch.c, src/auto/configure

Patch 7.1.016 (after patch 7.1.012)
问题: Error message about setting 'diff' to a string.
解决方案: Don't pass an empty string to set_option_value() when setting 'diff'.
相关文件: src/quickfix.c, src/popupmnu.c

Patch 7.1.017

问题:　　":confirm w" does give a prompt when 'readonly' is set, but not when the file permissions are read-only. (Michael Schaap)
解决方案:　Provide a dialog in both situations. (Chris Lubinski)
相关文件:　src/ex_cmds.c, src/fileio.c, src/proto/fileio.pro

Patch 7.1.018
问题:　　When 'virtualedit' is set a "p" of a block just past the end of the line inserts before the cursor. (Engelke)
解决方案:　Check for the cursor being just after the line (Chris Lubinski)
相关文件:　src/ops.c

Patch 7.1.019
问题:　　":py" asks for an argument, ":py asd" then gives the error that ":py" isn't implemented. Should already happen for ":py".
解决方案:　Compare with ex_script_ni. (Chris Lubinski)
相关文件:　src/ex_docmd.c

Patch 7.1.020
问题:　　Reading from uninitialized memory when using a dialog. (Dominique Pelle)
解决方案:　In msg_show_console_dialog() append a NUL after every appended character.
相关文件:　src/message.c

Patch 7.1.021 (after 7.1.015)
问题:　　Mzscheme interface doesn't compile on Win32.
解决方案:　Fix the problem that 7.1.015 fixed in a better way. (Sergey Khorev)
相关文件:　src/if_mzsch.c

Patch 7.1.022
问题:　　When setting 'keymap' twice the b:keymap_name variable isn't set. (Milan Berta)
解决方案:　Don't unlet b:keymap_name for ":loadkeymap". (Martin Toft)
相关文件:　src/digraph.c

Patch 7.1.023
问题:　　"dw" in a line with one character deletes the line. Vi and nvi don't do this. (Kjell Arne Rekaa)
解决方案:　Check for one-character words especially.
相关文件:　src/search.c

Patch 7.1.024
问题:　　Using a pointer that has become invalid. (Chris Monson)
解决方案:　Obtain the line pointer again after we looked at another line.
相关文件:　src/search.c

Patch 7.1.025
问题:　　search() and searchpos() don't use match under cursor at start of line when using 'bc' flags. (Viktor Kojouharov)
解决方案:　Don't go to the previous line when the 'c' flag is present. Also fix that "j" doesn't move the cursor to the right column.
相关文件:　src/eval.c, src/search.c

Patch 7.1.026

问题: "[p" doesn't work in Visual mode. (David Brown)
解决方案: Use checkclearop() instead of checkclearopq().
相关文件: src/normal.c

Patch 7.1.027
问题: On Sun systems opening /dev/fd/N doesn't work, and they are used by process substitutions.
解决方案: Allow opening specific character special files for Sun systems. (Gary Johnson)
相关文件: src/fileio.c, src/os_unix.h

Patch 7.1.028
问题: Can't use last search pattern for ":sort". (Brian McKee)
解决方案: When the pattern is empty use the last search pattern. (Martin Toft)
相关文件: runtime/doc/change.txt, src/ex_cmds.c

Patch 7.1.029 (after 7.1.019)
问题: Can't compile when all interfaces are used. (Taylor Venable)
解决方案: Only check for ex_script_ni when it's defined.
相关文件: src/ex_docmd.c

Patch 7.1.030
问题: The "vimtutor" shell script checks for "vim6" but not for "vim7". (Christian Robinson)
解决方案: Check for more versions, but prefer using "vim".
相关文件: src/vimtutor

Patch 7.1.031
问题: virtcol([123, '$']) doesn't work. (Michael Schaap)
解决方案: When '$' is used for the column number get the last column.
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.1.032
问题: Potential crash when editing a command line. (Chris Monson)
解决方案: Check the position to avoid access before the start of an array.
相关文件: src/ex_getln.c

Patch 7.1.033
问题: A buffer is marked modified when it was first deleted and then added again using a ":next" command. (John Mullin)
解决方案: When checking if a buffer is modified use the BF_NEVERLOADED flag.
相关文件: src/option.c

Patch 7.1.034
问题: Win64: A few compiler warnings. Problems with optimizer.
解决方案: Use int instead of size_t. Disable the optimizer in one function. (George V. Reilly)
相关文件: src/eval.c, src/spell.c

Patch 7.1.035
问题: After ":s/./&/#" all listed lines have a line number. (Yakov Lerner)
解决方案: Reset the line number flag when not using the "&" flag.

相关文件: src/ex_cmds.c

Patch 7.1.036
问题: Completing ":echohl" argument should include "None". (Ori Avtalion) ":match" should have "none" too.
解决方案: Add flags to use expand_highlight(). Also fix that when disabling FEAT_CMDL_COMPL compilation fails. (Chris Lubinski)
相关文件: src/eval.c, src/ex_docmd.c, src/ex_getln.c, src/proto/syntax.pro src/syntax.c

Patch 7.1.037
问题: strcpy() used for overlapping strings. (Chris Monson)
解决方案: Use mch_memmove() instead.
相关文件: src/option.c

Patch 7.1.038
问题: When 'expandtab' is set then a Tab copied for 'copyindent' is expanded to spaces, even when 'preserveindent' is set. (Alexei Alexandrov)
解决方案: Remove the check for 'expandtab'. Also fix that ">>" doesn't obey 'preserveindent'. (Chris Lubinski)
相关文件: src/misc1.c

Patch 7.1.039
问题: A tag in a help file that starts with "help-tags" and contains a percent sign may make Vim crash. (Ulf Harnhammar)
解决方案: Use puts() instead of fprintf().
相关文件: src/ex_cmds.c

Patch 7.1.040
问题: ":match" only supports three matches.
解决方案: Add functions clearmatches(), getmatches(), matchadd(), matchdelete() and setmatches(). Changed the data structures for this. A small bug in syntax.c is fixed, so newly created highlight groups can have their name resolved correctly from their ID. (Martin Toft)
相关文件: runtime/doc/eval.txt, runtime/doc/pattern.txt, runtime/doc/usr_41.txt, src/eval.c, src/ex_docmd.c, src/proto/window.pro, src/screen.c, src/structs.h, src/syntax.c, src/testdir/Makefile, src/testdir/test63.in, src/testdir/test63.ok, src/window.c

Patch 7.1.041 (extra, after 7.1.040)
问题: Some changes for patch 7.1.040 are in extra files.
解决方案: Update the extra files.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.1.042 (after 7.1.040)
问题: Internal error when using matchadd(). (David Larson)
解决方案: Check the third argument to be present before using the fourth argument. (Martin Toft)
相关文件: src/eval.c

Patch 7.1.043
问题: In Ex mode using CTRL-D twice may cause a crash. Cursor isn't positioned properly after CTRL-D.
解决方案: Set prev_char properly. Position the cursor correctly. (Antony Scriven)
相关文件: src/ex_getln.c

Patch 7.1.044
问题: In Insert mode 0 CTRL-T deletes all indent, it should add indent. (Gautam Iyer)
解决方案: Check for CTRL-D typed.
相关文件: src/edit.c

Patch 7.1.045
问题: Unnecessary screen redrawing. (Jjgod Jiang)
解决方案: Reset "must_redraw" after clearing the screen.
相关文件: src/screen.c

Patch 7.1.046
问题: ":s" command removes combining characters. (Ron Aaron)
解决方案: Copy composing characters individually. (Chris Lubinski)
相关文件: src/regexp.c

Patch 7.1.047
问题: vim_regcomp() called with invalid argument. (Xiaozhou Liu)
解决方案: Change TRUE to RE_MAGIC + RE_STRING.
相关文件: src/ex_eval.c

Patch 7.1.048
问题: The matchparen plugin doesn't update the match when scrolling with the mouse wheel. (Ilya Bobir)
解决方案: Set the match highlighting for text that can be scrolled into the viewable area without moving the cursor. (Chris Lubinski)
相关文件: runtime/plugin/matchparen.vim

Patch 7.1.049
问题: Cannot compile GTK2 version with Hangul input feature.
解决方案: Don't define FEAT_XFONTSET when using GTK2.
相关文件: src/feature.h

Patch 7.1.050
问题: Possible crash when using C++ indenting. (Chris Monson)
解决方案: Keep the line pointer to the line to compare with. Avoid going past the end of line.
相关文件: src/misc1.c

Patch 7.1.051
问题: Accessing uninitialized memory when finding spell suggestions.
解决方案: Don't try swapping characters at the end of a word.
相关文件: src/spell.c

Patch 7.1.052
问题: When creating a new match not all fields are initialized, which may lead to unpredictable results.

解决方案: Initialise rmm_ic and rmm_maxcol.
相关文件: src/window.c

Patch 7.1.053
问题: Accessing uninitialized memory when giving a message.
解决方案: Check going the length before checking for a NUL byte.
相关文件: src/message.c

Patch 7.1.054
问题: Accessing uninitialized memory when displaying the fold column.
解决方案: Add a NUL to the extra array. (Dominique Pelle). Also do this in a couple of other situations.
相关文件: src/screen.c

Patch 7.1.055
问题: Using strcpy() with arguments that overlap.
解决方案: Use mch_memmove() instead.
相关文件: src/buffer.c, src/charset.c, src/eval.c, src/ex_getln.c, src/misc1.c, src/regexp.c, src/termlib.c

Patch 7.1.056
问题: More prompt does not behave correctly after scrolling back. (Randall W. Morris)
解决方案: Avoid lines_left becomes negative. (Chris Lubinski) Don't check mp_last when deciding to show the more prompt. (Martin Toft)
相关文件: src/message.c

Patch 7.1.057
问题: Problem with CursorHoldI when using "r" in Visual mode (Max Dyckhoff)
解决方案: Ignore CursorHold(I) when getting a second character for a Normal mode command. Also abort the "r" command in Visual when a special key is typed.
相关文件: src/normal.c

Patch 7.1.058
问题: When 'rightleft' is set the completion menu is positioned wrong. (Baha-Eddine MOKADEM)
解决方案: Fix the completion menu. (Martin Toft)
相关文件: src/popupmnu.c, src/proto/search.pro, src/search.c

Patch 7.1.059
问题: When in Ex mode and doing "g/^/vi" and then pressing CTRL-C Vim hangs and beeps. (Antony Scriven)
解决方案: Clear "got_int" in the main loop to avoid the hang. When typing CTRL-C twice in a row abort the ":g" command. This is Vi compatible.
相关文件: src/main.c

Patch 7.1.060
问题: Splitting quickfix window messes up window layout. (Marius Gedminas)
解决方案: Compute the window size in a smarter way. (Martin Toft)
相关文件: src/window.c

Patch 7.1.061
问题: Win32: When 'encoding' is "latin1" 'ignorecase' doesn't work for characters with umlaut. (Joachim Hofmann)
解决方案: Do not use islower()/isupper()/tolower()/toupper() but our own functions. (Chris Lubinski)
相关文件: src/mbyte.c, src/regexp.c, src/vim.h

Patch 7.1.062 (after 7.1.038)
问题: Indents of C comments can be wrong. (John Mullin)
解决方案: Adjust ind_len. (Chris Lubinski)
相关文件: src/misc1.c

Patch 7.1.063 (after 7.1.040)
问题: Warning for uninitialized variable.
解决方案: Initialise it to NULL.
相关文件: src/ex_docmd.c

Patch 7.1.064
问题: On Interix some files appear not to exist.
解决方案: Remove the top bit from st_mode. (Ligesh)
相关文件: src/os_unix.c

Patch 7.1.065 (extra)
问题: Win32: Compilation problem for newer version of w32api.
解决方案: Only define __IID_DEFINED__ when needed. (Chris Sutcliffe)
相关文件: src/Make_ming.mak, src/iid_ole.c

Patch 7.1.066
问题: When 'bomb' is set or reset the file should be considered modified. (Tony Mechelynck)
解决方案: Handle like 'endofline'. (Martin Toft)
相关文件: src/buffer.c, src/fileio.c, src/option.c, src/structs.h

Patch 7.1.067
问题: 'thesaurus' doesn't work when 'infercase' is set. (Mohsin)
解决方案: Don't copy the characters being completed but check the case and apply it to the suggested word. Also fix that the first word in the thesaurus line is not used. (Martin Toft)
相关文件: src/edit.c

Patch 7.1.068
问题: When 'equalalways' is set and splitting a window, it's possible that another small window gets bigger.
解决方案: Only equalize window sizes when after a split the windows are smaller than another window. (Martin Toft)
相关文件: runtime/doc/options.txt, runtime/doc/windows.txt, src/window.c

Patch 7.1.069
问题: GTK GUI: When using confirm() without a default button there still is a default choice.
解决方案: Ignore Enter and Space when there is no default button. (Chris Lubinski)
相关文件: src/gui_gtk.c

Patch 7.1.070 (extra)
问题: Win32 GUI: When using confirm() without a default button there still is a default choice.
解决方案: Set focus on something else than a button. (Chris Lubinski)
相关文件: src/gui_w32.c

Patch 7.1.071 (after 7.1.040)
问题: Regexp patterns are not tested.
解决方案: Add a basic test, to be expanded later. Also add (commented-out) support for valgrind.
相关文件: src/testdir/Makefile, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.1.072 (extra, after 7.1.041 and 7.1.071)
问题: Some changes for patch 7.1.071 are in extra files.
解决方案: Update the extra files. Also fix a few warnings from the DOS test makefile.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.1.073 (after 7.1.062)
问题: Wrong cursor position and crash when 'preserveindent' is set. (Charles Campbell)
解决方案: Handle the situation that we start without indent. (Chris Lubinski)
相关文件: src/misc1.c

Patch 7.1.074
问题: Crash when calling string() on a recursively nested List.
解决方案: Check result value for being NULL. (Yukihiro Nakadaira)
相关文件: src/eval.c

Patch 7.1.075
问题: ":let v:statusmsg" reads memory already freed.
解决方案: Don't set v:statusmsg when listing it.
相关文件: src/eval.c

Patch 7.1.076
问题: Another strcpy() with overlapping arguments.
解决方案: Use mch_memmove(). (Dominique Pelle) And another one.
相关文件: src/ex_docmd.c, src/normal.c

Patch 7.1.077
问题: Using "can_spell" without initializing it. (Dominique Pelle)
解决方案: Set a default for get_syntax_attr().
相关文件: src/syntax.c

Patch 7.1.078
问题: Dropping a file name on gvim that contains a CSI byte doesn't work when editing the command line.
解决方案: Escape the CSI byte when inserting in the input buffer. (Yukihiro Nakadaira)
相关文件: src/gui.c, src/ui.c

Patch 7.1.079
问题: When the locale is "C" and 'encoding' is "latin1" then the "@" character in 'isfname', 'isprint', etc. doesn't pick up accented characters.
解决方案: Instead of isalpha() use MB_ISLOWER() and MB_ISUPPER().
相关文件: src/charset.c, src/macros.h

Patch 7.1.080 (extra)
问题: Compiler warnings for using "const char *" for "char *".
解决方案: Add type casts. (Chris Sutcliffe)
相关文件: src/GvimExt/gvimext.cpp

Patch 7.1.081
问题: Command line completion for a shell command: "cat </tmp/file<Tab>" doesn't work.
解决方案: Start the file name at any character that can't be in a file name. (Martin Toft)
相关文件: src/ex_docmd.c

Patch 7.1.082
问题: After a ":split" the matchparen highlighting isn't there.
解决方案: Install a WinEnter autocommand. Also fixes that after ":NoMatchParen" only the current window is updated. (Martin Toft)
相关文件: runtime/doc/pi_paren.txt, runtime/plugin/matchparen.vim

Patch 7.1.083 (after 7.1.081)
问题: Command line completion doesn't work with wildcards.
解决方案: Add vim_isfilec_or_wc() and use it. (Martin Toft)
相关文件: src/charset.c, src/proto/charset.pro, src/ex_docmd.c

Patch 7.1.084
问题: Using the "-nb" argument twice causes netbeans not to get fileOpened events.
解决方案: Change "&" to "&&". (Xavier de Gaye)
相关文件: src/ex_cmds.c

Patch 7.1.085
问题: ":e fold.c" then ":sp fold.c" results in folds of original window to disappear. (Akita Noek)
解决方案: Invoke foldUpdateAll() for all windows of the changed buffer. (Martin Toft)
相关文件: src/ex_cmds.c

Patch 7.1.086
问题: Crash when using specific Python syntax highlighting. (Quirk)
解决方案: Check for a negative index, coming from a keyword match at the start of a line from a saved state.
相关文件: src/syntax.c

Patch 7.1.087
问题: Reading past ":cscope find" command. Writing past end of a buffer.
解决方案: Check length of the argument before using the pattern. Use vim_strncpy(). (Dominique Pelle)
相关文件: if_cscope.c

Patch 7.1.088 (extra)
问题: The coordinates used by ":winpos" differ from what getwinposx() and getwinposy() return.
解决方案: Use MoveWindowStructure() instead of MoveWindow(). (Michael Henry)
相关文件: src/gui_mac.c

Patch 7.1.089
问题: ":let loaded_getscriptPlugin" doesn't clear to eol, result is "#1in".
解决方案: Clear to the end of the screen after displaying the first variable value.
相关文件: src/eval.c

Patch 7.1.090
问题: Compiler warning on Mac OS X 10.5.
解决方案: Don't redeclare sigaltstack(). (Hisashi T Fujinaka)
相关文件: src/os_unix.c

Patch 7.1.091 (extra)
问题: Win32: Can't embed Vim inside another application.
解决方案: Add the --windowid argument. (Nageshwar)
相关文件: runtime/doc/gui_w32.txt, runtime/doc/starting.txt, runtime/doc/vi_diff.txt, src/globals.h, src/gui_w32.c, src/main.c

Patch 7.1.092 (extra, after 7.1.088)
问题: Wrong arguments for MoveWindowStructure().
解决方案: Remove "TRUE". (Michael Henry)
相关文件: src/gui_mac.c

Patch 7.1.093
问题: Reading past end of a screen line when determining cell width. (Dominique Pelle)
解决方案: Add an argument to mb_off2cells() for the maximum offset.
相关文件: src/globals.h, src/gui.c, src/mbyte.c, src/proto/mbyte.pro, src/screen.c

Patch 7.1.094
问题: When checking if syntax highlighting is present, looking in the current buffer instead of the specified one.
解决方案: Use "buf" instead of "curbuf".
相关文件: src/syntax.c

Patch 7.1.095
问题: The FocusLost and FocusGained autocommands are triggered asynchronously in the GUI. This may cause arbitrary problems.
解决方案: Put the focus event in the input buffer and handle it when ready for it.
相关文件: src/eval.c, src/getchar.c, src/gui.c, src/gui_gtk_x11.c, src/keymap.h

Patch 7.1.096
问题: Reading past end of a string when resizing Vim. (Dominique Pelle)
解决方案: Check the string pointer before getting the char it points to.

相关文件: src/message.c

Patch 7.1.097
问题: ":setlocal stl=%!1+1" does not work.
解决方案: Adjust check for pointer. (Politz)
相关文件: src/option.c

Patch 7.1.098
问题: ":call s:var()" doesn't work if "s:var" is a Funcref. (Andy Wokula)
解决方案: Before converting "s:" into a script ID, check if it is a Funcref.
相关文件: src/eval.c

Patch 7.1.099
问题: When the 'keymap' and 'paste' options have a non-default value,
":mkexrc" and ":mksession" do not correctly set the options.
解决方案: Set the options with side effects before other options.
相关文件: src/option.c

Patch 7.1.100
问题: Win32: Executing cscope doesn't always work properly.
解决方案: Use another way to invoke cscope. (Mike Williams)
相关文件: src/if_cscope.c, src/if_cscope.h, src/main.c,
src/proto/if_cscope.pro

Patch 7.1.101
问题: Ruby: The Buffer.line= method does not work.
解决方案: Add the "self" argument to set_current_line(). (Jonathan Hankins)
相关文件: src/if_ruby.c

Patch 7.1.102
问题: Perl interface doesn't compile with new version of Perl.
解决方案: Add two variables to the dynamic library loading. (Suresh
Govindachar)
相关文件: src/if_perl.xs

Patch 7.1.103
问题: Using "dw" with the cursor past the end of the last line (using
CTRL-\ CTRL-O from Insert mode) deletes a character. (Tim Chase)
解决方案: Don't move the cursor back when the movement failed.
相关文件: src/normal.c

Patch 7.1.104 (after 7.1.095)
问题: When 'lazyredraw' is set a focus event causes redraw to be
postponed until a key is pressed.
解决方案: Instead of not returning from vgetc() when a focus event is
encountered return K_IGNORE. Add plain_vgetc() for when the
caller doesn't want to get K_IGNORE.
相关文件: src/digraph.c, src/edit.c, src/ex_cmds.c, src/ex_getln.c,
src/getchar.c, src/normal.c, src/proto/getchar.pro, src/window.c

Patch 7.1.105
问题: Internal error when using "0 ? {'a': 1} : {}". (A.Politz)
解决方案: When parsing a dictionary value without using the value, don't try
obtaining the key name.

相关文件: src/eval.c

Patch 7.1.106
问题: ":messages" doesn't quit listing on ":".
解决方案: Break the loop when "got_int" is set.
相关文件: src/message.c

Patch 7.1.107
问题: When doing a block selection and using "s" to change the text,
while triggering auto-indenting, causes the wrong text to be
repeated in other lines. (Adri Verhoef)
解决方案: Compute the change of indent and compensate for that.
相关文件: src/ops.c

Patch 7.1.108 (after 7.1.100)
问题: Win32: Compilation problems in Cscope code. (Jeff Lanzarotta)
解决方案: Use (long) instead of (intptr_t) when it's not defined.
相关文件: src/if_cscope.c

Patch 7.1.109
问题: GTK: when there are many tab pages, clicking on the arrow left of
the labels moves to the next tab page on the right. (Simeon Bird)
解决方案: Check the X coordinate of the click and pass -1 as value for the
left arrow.
相关文件: src/gui_gtk_x11.c, src/term.c

Patch 7.1.110 (after 7.1.102)
问题: Win32: Still compilation problems with Perl.
解决方案: Change the #ifdefs. (Suresh Govindachar)
相关文件: src/if_perl.xs

Patch 7.1.111
问题: When using ":vimgrep" with the "j" flag folds from another buffer
may be displayed. (A.Politz)
解决方案: When not jumping to another buffer update the folds.
相关文件: src/quickfix.c

Patch 7.1.112
问题: Using input() with a wrong argument may crash Vim. (A.Politz)
解决方案: Init the input() return value to NULL.
相关文件: src/eval.c

Patch 7.1.113
问题: Using map() to go over an empty list causes memory to be freed
twice. (A.Politz)
解决方案: Don't clear the typeval in restore_vimvar().
相关文件: src/eval.c

Patch 7.1.114
问题: Memory leak in getmatches().
解决方案: Don't increment the refcount twice.
相关文件: src/eval.c

Patch 7.1.115 (after 7.1.105)

问题:　　Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案:　Init variable to NULL.
相关文件:　src/eval.c

Patch 7.1.116
问题:　　Cannot display Unicode characters above 0x10000.
解决方案:　Remove the replacement with a question mark when UNICODE16 is not
defined. (partly by Nicolas Weber)
相关文件:　src/screen.c

Patch 7.1.117
问题:　　Can't check whether Vim was compiled with Gnome. (Tony Mechelynck)
解决方案:　Add gui_gnome to the has() list.
相关文件:　src/eval.c

Patch 7.1.118 (after 7.1.107)
问题:　　Compiler warning for Visual C compiler.
解决方案:　Add typecast. (Mike Williams)
相关文件:　src/ops.c

Patch 7.1.119
问题:　　Crash when 'cmdheight' set to very large value. (A.Politz)
解决方案:　Limit 'cmdheight' to 'lines' minus one. Store right value of
'cmdheight' when running out of room.
相关文件:　src/option.c, src/window.c

Patch 7.1.120
问题:　　Can't properly check memory leaks while running tests.
解决方案:　Add an argument to garbagecollect(). Delete functions and
variables in the test scripts.
相关文件:　runtime/doc/eval.txt src/eval.c, src/globals.h, src/main.c,
src/testdir/Makefile, src/testdir/test14.in,
src/testdir/test26.in, src/testdir/test34.in,
src/testdir/test45.in, src/testdir/test47.in,
src/testdir/test49.in, src/testdir/test55.in,
src/testdir/test56.in, src/testdir/test58.in,
src/testdir/test59.in, src/testdir/test60.in,
src/testdir/test60.vim, src/testdir/test62.in,
src/testdir/test63.in, src/testdir/test64.in,

Patch 7.1.121
问题:　　Using ":cd %:h" when editing a file in the current directory
results in an error message for using an empty string.
解决方案:　When "%:h" results in an empty string use ".".
相关文件:　src/eval.c

Patch 7.1.122
问题:　　Mac: building Vim.app fails. Using wrong architecture.
解决方案:　Use line continuation for the gui_bundle dependency. Detect the
system architecture with "uname -a".
相关文件:　src/main.aap

Patch 7.1.123
问题:　　Win32: ":edit foo ~ foo" expands "~".

解决方案: Change the call to expand_env().
相关文件: src/ex_docmd.c, src/misc1.c, src/proto/misc1.pro, src/option.c

Patch 7.1.124 (extra)
问题: Mac: When dropping a file on Vim.app that is already in the buffer list (from .viminfo) results in editing an empty, unnamed buffer. (Axel Kielhorn) Also: warning for unused variable.
解决方案: Move to the buffer of the first argument. Delete unused variable.
相关文件: src/gui_mac.c

Patch 7.1.125
问题: The TermResponse autocommand event is not always triggered. (Aron Griffis)
解决方案: When unblocking autocommands check if v:termresponse changed and trigger the event then.
相关文件: src/buffer.c, src/diff.c, src/ex_getln.c, src/fileio.c, src/globals.h, src/misc2.c, src/proto/fileio.pro, src/window.c

Patch 7.1.126 (extra)
问题: ":vimgrep */*" fails when a BufRead autocommand changes directory. (Bernhard Kuhn)
解决方案: Change back to the original directory after loading a file. Also: use shorten_fname1() to avoid duplicating code.
相关文件: src/buffer.c, src/ex_docmd.c, src/fileio.c, src/gui_gtk.c, src/gui_w48.c, src/proto/ex_docmd.pro, src/proto/fileio.pro, src/quickfix.c

Patch 7.1.127
问题: Memory leak when doing cmdline completion. (Dominique Pelle)
解决方案: Free "orig" argument of ExpandOne() when it's not used.
相关文件: src/ex_getln.c

Patch 7.1.128 (extra)
问题: Build problems with new version of Cygwin.
解决方案: Remove -D__IID_DEFINED__, like with MingW. (Guopeng Wen)
相关文件: src/Make_cyg.mak

Patch 7.1.129 (extra)
问题: Win32: Can't get the user name when it is longer than 15 characters.
解决方案: Use UNLEN instead of MAX_COMPUTERNAME_LENGTH. (Alexei Alexandrov)
相关文件: src/os_win32.c

Patch 7.1.130
问题: Crash with specific order of undo and redo. (A.Politz)
解决方案: Clear and adjust pointers properly. Add u_check() for debugging.
相关文件: src/undo.c, src/structs.h

Patch 7.1.131
问题: ":mksession" always adds ":setlocal autoread". (Christian J. Robinson)
解决方案: Skip boolean global/local option using global value.
相关文件: src/option.c

Patch 7.1.132
问题: getpos("'>") may return a negative column number for a Linewise selection. (A.Politz)
解决方案: Don't add one to MAXCOL.
相关文件: src/eval.c

Patch 7.1.133 (after 7.1.126)
问题: shorten_fname1() linked when it's not needed.
解决方案: Add #ifdef.
相关文件: src/fileio.c

Patch 7.1.134 (extra)
问题: Win32: Can't build with VC8
解决方案: Detect the MSVC version instead of using NMAKE_VER. (Mike Williams)
相关文件: src/Make_mvc.mak

Patch 7.1.135
问题: Win32: When editing a file c:\tmp\foo and c:\tmp\\foo we have two buffers for the same file. (Suresh Govindachar)
解决方案: Invoke FullName_save() when a path contains "//" or "\\".
相关文件: src/buffer.c

Patch 7.1.136
问题: Memory leak when using Ruby syntax highlighting. (Dominique Pelle)
解决方案: Free the contained-in list.
相关文件: src/syntax.c

Patch 7.1.137
问题: Build failure when using EXITFREE. (Dominique Pelle)
解决方案: Add an #ifdef around using clip_exclude_prog.
相关文件: src/misc2.c

Patch 7.1.138
问题: The Perl Msg() function doesn't stop when "q" is typed at the more prompt. (Hari Krishna Dara)
解决方案: Check got_int.
相关文件: src/if_perl.xs

Patch 7.1.139
问题: When using marker folding and ending Insert mode with CTRL-C the current fold is truncated. (Fred Kater)
解决方案: Ignore got_int while updating folds.
相关文件: src/fold.c

Patch 7.1.140
问题: v:count is set only after typing a non-digit, that makes it difficult to make a nice mapping.
解决方案: Set v:count while still typing the count.
相关文件: src/normal.c

Patch 7.1.141
问题: GTK: -geom argument doesn't support a negative offset.
解决方案: Compute position from the right/lower corner.

相关文件: src/gui_gtk_x11.c

Patch 7.1.142
问题: ":redir @A>" doesn't work.
解决方案: Ignore the extra ">" also when appending. (James Vega)
相关文件: src/ex_docmd.c

Patch 7.1.143
问题: Uninitialized memory read when diffing three files. (Dominique Pelle)
解决方案: Remove "+ !notset" so that we don't use fields that were not computed.
相关文件: src/diff.c

Patch 7.1.144
问题: After ":diffup" cursor can be in the wrong position.
解决方案: Force recomputing the cursor position.
相关文件: src/diff.c

Patch 7.1.145
问题: Insert mode completion: When using the popup menu, after completing a word and typing a non-word character Vim is still completing the same word, following CTRL-N doesn't work.
Insert mode Completion: When using CTRL-X O and there is only "struct." before the cursor, typing one char to reduce the matches, then BS completion stops.
解决方案: When typing a character that is not part of the item being completed, stop complete mode. For whole line completion also accept a space. For file name completion stop at a path separator.
For omni completion stay in completion mode even if completing with empty string.
相关文件: src/edit.c

Patch 7.1.146 (extra)
问题: VMS: Files with a very rare record organization (VFC) cannot be properly written by Vim.
On older VAX systems mms runs into a syntax error.
解决方案: Check for this special situation. Do not wrap a comment, make it one long line. (Zoltan Arpadffy)
相关文件: src/fileio.c, src/Make_vms.mms

Patch 7.1.147 (after 7.1.127)
问题: Freeing memory already freed when completing user name. (Meino Cramer)
解决方案: Use a flag to remember if "orig" needs to be freed.
相关文件: src/ex_getln.c

Patch 7.1.148
问题: Some types are not found by configure.
解决方案: Test for the sys/types.h header file. (Sean Boudreau)
相关文件: src/configure.in, src/auto/configure

Patch 7.1.149

问题:　　GTK GUI: When the completion popup menu is used scrolling another window by the scrollbar is OK, but using the scroll wheel it behaves line <Enter>.
解决方案:　Ignore K_MOUSEDOWN and K_MOUSEUP. Fix redrawing the popup menu.
相关文件:　src/edit.c, src/gui.c

Patch 7.1.150
问题:　　When 'clipboard' has "unnamed" using "p" in Visual mode doesn't work correctly. (Jianrong Yu)
解决方案:　When 'clipboard' has "unnamed" also obtain the selection when getting the default register.
相关文件:　src/ops.c

Patch 7.1.151
问题:　　Using whole line completion with 'ignorecase' and 'infercase' set and the line is empty get an lalloc(0) error.
解决方案:　Don't try changing case for an empty match. (Matthew Wozniski)
相关文件:　src/edit.c

Patch 7.1.152
问题:　　Display problem when 'hls' and 'cursorcolumn' are set and searching for "$". (John Mullin) Also when scrolling horizontally when 'wrap' is off.
解决方案:　Keep track of the column where highlighting was set. Check the column offset when skipping characters.
相关文件:　src/screen.c

Patch 7.1.153
问题:　　Compiler warnings on SGI. Undefined XpmAllocColor (Charles Campbell)
解决方案:　Add type casts. Init st_dev and st_ino separately. Don't use type casts for vim_snprintf() when HAVE_STDARG_H is defined. Define XpmAllocColor when needed.
相关文件:　src/eval.c, src/ex_cmds.c, src/fileio.c, src/misc2.c, src/gui_xmebw.c

Patch 7.1.154
问题:　　Compiler warning for signed/unsigned compare.
解决方案:　Add type cast.
相关文件:　src/screen.c

Patch 7.1.155
问题:　　Crash when 'undolevels' is 0 and repeating "udd". (James Vega)
解决方案:　When there is only one branch use u_freeheader() to delete it.
相关文件:　src/undo.c

Patch 7.1.156
问题:　　Overlapping arguments for strcpy() when expanding command line variables.
解决方案:　Use mch_memmove() instead of STRCPY(). Also fix a few typos. (Dominique Pelle)
相关文件:　src/ex_docmd.c

Patch 7.1.157

问题:　　In Ex mode, :" gives an error at end-of-file. (Michael Hordijk)
解决方案:　Only give an error for an empty line, not for a comment.
相关文件:　src/ex_docmd.c

Patch 7.1.158 (extra)
问题:　　Win32 console: When 'encoding' is "utf-8" and typing Alt-y the result is wrong. Win32 GUI: Alt-y results in "u" when 'encoding' is "cp1250" (Lukas Cerman)
解决方案:　For utf-8 don't set the 7th bit in a byte, convert to the correct byte sequence. For cp1250, when conversion to 'encoding' results in the 7th bit not set, set the 7th bit after conversion.
相关文件:　src/os_win32.c, src/gui_w48.c

Patch 7.1.159
问题:　　strcpy() has overlapping arguments.
解决方案:　Use mch_memmove() instead. (Dominique Pelle)
相关文件:　src/ex_cmds.c

Patch 7.1.160
问题:　　When a focus autocommand is defined, getting or losing focus causes the hit-enter prompt to be redrawn. (Bjorn Winckler)
解决方案:　Overwrite the last line.
相关文件:　src/message.c

Patch 7.1.161
问题:　　Compilation errors with tiny features and EXITFREE.
解决方案:　Add #ifdefs. (Dominique Pelle)
相关文件:　src/edit.c, src/misc2.c

Patch 7.1.162
问题:　　Crash when using a modifier before "while" or "for". (A.Politz)
解决方案:　Skip modifiers when checking for a loop command.
相关文件:　src/proto/ex_docmd.pro, src/ex_docmd.c, src/ex_eval.c

Patch 7.1.163
问题:　　Warning for the unknown option 'bufsecret'.
解决方案:　Remove the lines .vim that use this option. (Andy Wokula)
相关文件:　runtime/menu.vim

Patch 7.1.164
问题:　　Reading past end of regexp pattern. (Dominique Pelle)
解决方案:　Use utf_ptr2len().
相关文件:　src/regexp.c

Patch 7.1.165
问题:　　Crash related to getting X window ID. (Dominique Pelle)
解决方案:　Don't trust the window ID that we got in the past, check it every time.
相关文件:　src/os_unix.c

Patch 7.1.166
问题:　　Memory leak for using "gp" in Visual mode.
解决方案:　Free memory in put_register(). (Dominique Pelle)
相关文件:　src/ops.c

Patch 7.1.167
问题: Xxd crashes when using "xxd -b -c 110". (Debian bug 452789)
解决方案: Allocate more memory. Fix check for maximum number of columns.
相关文件: src/xxd/xxd.c

Patch 7.1.168 (extra)
问题: Win32 GUI: Since patch 7.1.095, when the Vim window does not have focus, clicking in it doesn't position the cursor. (Juergen Kraemer)
解决方案: Don't reset s_button_pending just after receiving focus.
相关文件: src/gui_w48.c

Patch 7.1.169
问题: Using uninitialized variable when system() fails. (Dominique Pelle)
解决方案: Let system() return an empty string when it fails.
相关文件: src/eval.c

Patch 7.1.170
问题: Valgrind warning for overlapping arguments for strcpy().
解决方案: Use mch_memmove() instead. (Dominique Pelle)
相关文件: src/getchar.c

Patch 7.1.171
问题: Reading one byte before allocated memory.
解决方案: Check index not to become negative. (Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.1.172
问题: When 'buftype' is "acwrite" Vim still checks if the file or directory exists before overwriting.
解决方案: Don't check for overwriting when the buffer name is not a file name.
相关文件: src/ex_cmds.c

Patch 7.1.173
问题: Accessing freed memory. (Dominique Pelle)
解决方案: Don't call reg_getline() to check if a line is the first in the file.
相关文件: src/regexp.c

Patch 7.1.174
问题: Writing NUL past end of a buffer.
解决方案: Copy one byte less when using strncat(). (Dominique Pelle)
相关文件: src/ex_cmds.c, src/ex_docmd.c,

Patch 7.1.175
问题: <BS> doesn't work with some combination of 'sts', 'linebreak' and 'backspace'. (Francois Ingelrest)
解决方案: When adding white space results in not moving back delete one character.
相关文件: src/edit.c

Patch 7.1.176
问题: Building with Aap fails when the "compiledby" argument contains '<' or '>' characters. (Alex Yeh)
解决方案: Change how quoting is done in the Aap recipe.
相关文件: src/main.aap

Patch 7.1.177
问题: Freeing memory twice when in debug mode while reading a script.
解决方案: Ignore script input while in debug mode.
相关文件: src/ex_cmds2.c, src/getchar.c, src/globals.h

Patch 7.1.178
问题: "%" doesn't work on "/* comment // comment */".
解决方案: Don't handle the "//" in "*//*" as a C++ comment. (Markus Heidelberg)
相关文件: src/search.c

Patch 7.1.179
问题: Need to check for TCL 8.5.
解决方案: Adjust configure script. (Alexey Froloff)
相关文件: src/configure.in, src/auto/configure

Patch 7.1.180
问题: Regexp patterns not tested sufficiently.
解决方案: Add more checks to the regexp test.
相关文件: src/testdir/test64.in, src/testdir/test64.ok

Patch 7.1.181
问题: Accessing uninitialized memory in Farsi mode. (Dominique Pelle)
解决方案: Only invoke lrF_sub() when there is something to do.
相关文件: src/ex_cmds.c

Patch 7.1.182
问题: When using tab pages and an argument list the session file may contain wrong "next" commands. (Alexander Bluem)
解决方案: Use "argu" commands and only when needed.
相关文件: src/ex_docmd.c

Patch 7.1.183
问题: "Internal error" for ":echo matchstr('a', 'a\%[\&]')" (Mitanu Paul)
解决方案: Inside "\%[]" detect \&, \| and \) as an error.
相关文件: src/regexp.c

Patch 7.1.184
问题: Crash when deleting backwards over a line break in Insert mode.
解决方案: Don't advance the cursor when it's already on the NUL after a line. (Matthew Wozniski)
相关文件: src/normal.c

Patch 7.1.185
问题: Using "gR" with a multi-byte encoding and typing a CR pushes characters onto the replace stack incorrectly, resulting in BS putting back the wrong characters. (Paul B. Mahol)

解决方案: Push multi-byte characters onto the replace stack in reverse byte
           order. Add replace_push_mb().
相关文件: src/edit.c, src/misc1.c, src/proto/edit.pro

Patch 7.1.186
问题: "expand('<afile>')" returns a bogus value after changing
           directory. (Dave Fishburn)
解决方案: Copy "autocmd_fname" to allocated memory and expand to full
           filename. Shorten the path when expanding <afile>.
相关文件: src/ex_docmd.c, src/fileio.c

Patch 7.1.187
问题: Win32 GUI: Custom completion using system() no longer works
           after patch 7.1.104. (Erik Falor)
解决方案: Loop when safe_vgetc() returns K_IGNORE.
相关文件: src/ex_getln.c

Patch 7.1.188
问题: When 'showmode' is off the message for changing a readonly file is
           given in the second column instead of the first. (Payl B. Mahol)
解决方案: Put the W10 message in the first column.
相关文件: src/edit.c

Patch 7.1.189 (after 7.1.104)
问题: Patch 7.1.104 was incomplete.
解决方案: Also call plain_vgetc() in ask_yesno().
相关文件: src/misc1.c

Patch 7.1.190
问题: Cursor after end-of-line: "iA sentence.<Esc>)"
解决方案: Move cursor back and make motion inclusive.
相关文件: src/normal.c

Patch 7.1.191
问题: Win32 GUI: after patch 7.1.168 there is still a problem when
           clicking in a scrollbar. (Juergen Jottkaerr)
解决方案: Don't check the input buffer when dragging the scrollbar.
相关文件: src/gui.c

Patch 7.1.192
问题: With Visual block selection, "s" and typing something, CTRL-C
           doesn't stop Vim from repeating the replacement in other lines,
           like happens for "I".
解决方案: Check for "got_int" to be set.
相关文件: src/ops.c

Patch 7.1.193
问题: Some Vim 5.x digraphs are missing in Vim 7, even though the
           character pairs are not used. (Philippe de Muyter)
解决方案: Add those Vim 5.x digraphs that don't conflict with others.
相关文件: src/digraph.c

Patch 7.1.194
问题: ":echo glob('~/{}')" results in /home/user//.

解决方案: Don't add a slash if there already is one.
相关文件: src/os_unix.c

Patch 7.1.195
问题: '0 mark doesn't work for "~/foo ~ foo".
解决方案: Don't expand the whole file name, only "~/".
相关文件: src/mark.c

Patch 7.1.196 (extra)
问题: Win32 GUI: "\n" in a tooltip doesn't cause a line break. (Erik Falor)
解决方案: Use the TTM_SETMAXTIPWIDTH message.
相关文件: src/gui_w32.c

Patch 7.1.197
问题: Mac: "make install" doesn't work when prefix defined.
解决方案: Pass different arguments to "make installruntime". (Jjgod Jiang)
相关文件: src/Makefile

Patch 7.1.198
问题: Hang when using ":s/\n//gn". (Burak Gorkemli)
解决方案: Set "skip_match".
相关文件: src/ex_cmds.c

Patch 7.1.199
问题: Can't do command line completion for a specific file name extension.
解决方案: When the pattern ends in "$" don't add a star for completion and remove the "$" before matching with file names.
相关文件: runtime/doc/cmdline.txt, src/ex_getln.c

Patch 7.1.200 (after 7.1.177 and 7.1.182)
问题: Compiler warnings for uninitialized variables.
解决方案: Init variables.
相关文件: src/ex_cmds2.c, src/ex_docmd.c

Patch 7.1.201
问题: When reading stdin 'fenc' and 'ff' are not set.
解决方案: Set the options after reading stdin. (Ben Schmidt)
相关文件: src/fileio.c

Patch 7.1.202
问题: Incomplete utf-8 byte sequence is not checked for validity.
解决方案: Check the bytes that are present for being valid. (Ben Schmidt)
相关文件: src/mbyte.c

Patch 7.1.203
问题: When 'virtualedit' is "onemore" then "99|" works but ":normal 99|" doesn't. (Andy Wokula)
解决方案: Check for "onemore" flag in check_cursor_col().
相关文件: src/misc2.c

Patch 7.1.204 (extra)
问题: Win32: Using the example at 'balloonexpr' the balloon disappears

after four seconds and then comes back again. Also moves the mouse pointer a little bit. (Yongwei Wu)
解决方案: Set the autopop time to 30 seconds (the max value). (Sergey Khorev) Move the mouse two pixels forward and one back to end up in the same position (really!).
相关文件: src/gui_w32.c

Patch 7.1.205
问题: Can't get the operator in an ":omap".
解决方案: Add the "v:operator" variable. (Ben Schmidt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/normal.c, src/vim.h

Patch 7.1.206
问题: Compiler warnings when using MODIFIED_BY.
解决方案: Add type casts. (Ben Schmidt)
相关文件: src/version.c

Patch 7.1.207
问题: Netbeans: "remove" cannot delete one line.
解决方案: Remove partial lines and whole lines properly. Avoid a memory leak. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.1.208
问题: On Alpha get an unaligned access error.
解决方案: Store the dictitem pointer before using it. (Matthew Luckie)
相关文件: src/eval.c

Patch 7.1.209
问题: GTK: When using the netrw plugin and doing ":gui" Vim hangs.
解决方案: Stop getting a selection after three seconds. This is a hack.
相关文件: src/gui_gtk_x11.c

Patch 7.1.210
问题: Listing mapping for 0xdb fails when 'encoding' is utf-8. (Tony Mechelynck)
解决方案: Recognize K_SPECIAL KS_EXTRA KE_CSI as a CSI byte.
相关文件: src/mbyte.c

Patch 7.1.211
问题: The matchparen plugin may take an unexpected amount of time, so that it looks like Vim hangs.
解决方案: Add a timeout to searchpair(), searchpairpos(), search() and searchpos(). Use half a second timeout in the plugin.
相关文件: runtime/doc/eval.txt, runtime/plugin/matchparen.vim, src/edit.c, src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/normal.c, src/proto/eval.pro, src/proto/ex_cmds2.pro, src/proto/search.pro, src/search.c

Patch 7.1.212
问题: Accessing a byte before a line.
解决方案: Check that the column is 1 or more. (Dominique Pelle)
相关文件: src/edit.c

Patch 7.1.213
问题: A ":tabedit" command that results in the "swap file exists" dialog and selecting "abort" doesn't close the new tab. (Al Budden)
解决方案: Pass "old_curwin" to do_exedit().
相关文件: src/ex_docmd.c

Patch 7.1.214
问题: ":1s/g\n\zs1//" deletes characters from the first line. (A Politz)
解决方案: Start replacing in the line where the match starts.
相关文件: src/ex_cmds.c

Patch 7.1.215
问题: It is difficult to figure out what syntax items are nested at a certain position.
解决方案: Add the synstack() function.
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/syntax.pro, src/syntax.c

Patch 7.1.216
问题: Variants of --remote-tab are not mentioned for "vim --help".
解决方案: Display optional -wait and -silent.
相关文件: src/main.c

Patch 7.1.217
问题: The "help-tags" tag may be missing from runtime/doc/tags when it was generated during "make install".
解决方案: Add the "++t" argument to ":helptags" to force adding the tag.
相关文件: runtime/doc/Makefile, runtime/doc/various.txt, src/ex_cmds.c, src/ex_cmds.h

Patch 7.1.218
问题: A syntax region without a "keepend", containing a region with "extend" could be truncated at the end of the containing region.
解决方案: Do not call syn_update_ends() when there are no keepend items.
相关文件: src/syntax.c

Patch 7.1.219 (after 7.1.215)
问题: synstack() returns situation after the current character, can't see the state for a one-character region.
解决方案: Don't update ending states in the requested column.
相关文件: runtime/doc/eval.txt, src/eval.c, src/hardcopy.c, src/proto/syntax.pro, src/screen.c, src/spell.c, src/syntax.c

Patch 7.1.220
问题: When a ")" or word movement command moves the cursor back from the end of the line it may end up on the trail byte of a multi-byte character. It's also moved back when it isn't needed.
解决方案: Add the adjust_cursor() function.
相关文件: src/normal.c

Patch 7.1.221
问题: When inserting a "(", triggering the matchparen plugin, the following highlighting may be messed up.
解决方案: Before triggering the CursorMovedI autocommands update the display

to update the stored syntax stacks for the change.
相关文件: src/edit.c

Patch 7.1.222 (after 7.1.217)
问题: Wildcards in argument of ":helptags" are not expanded. (Marcel Svitalsky)
解决方案: Expand wildcards in the directory name.
相关文件: src/ex_cmds.c

Patch 7.1.223
问题: glob() doesn't work properly when 'shell' is "sh" or "bash" and the expanded name contains spaces, '~', single quotes and other special characters. (Adri Verhoef, Charles Campbell)
解决方案: For Posix shells define a vimglob() function to list the matches instead of using "echo" directly.
相关文件: src/os_unix.c

Patch 7.1.224
问题: When using "vim -F -o file1 file2" only one window is right-to-left. Same for "-H". (Ben Schmidt)
解决方案: use set_option_value() to set 'rightleft'.
相关文件: src/main.c

Patch 7.1.225
问题: Using uninitialized value when XGetWMNormalHints() fails.
解决方案: Check the return value. (Dominique Pelle)
相关文件: src/os_unix.c

Patch 7.1.226
问题: Command line completion doesn't work when a file name contains a '&' character.
解决方案: Accept all characters in a file name, except ones that end a command or white space.
相关文件: src/ex_docmd.c

Patch 7.1.227
问题: Hang in syntax HL when moving over a ")". (Dominique Pelle)
解决方案: Avoid storing a syntax state in the wrong position in the list of remembered states.
相关文件: src/syntax.c

Patch 7.1.228
问题: When 'foldmethod' is "indent" and a fold is created with ">>" it can't be closed with "zc". (Daniel Shahaf)
解决方案: Reset the "small" flag of a fold when adding a line to it.
相关文件: src/fold.c

Patch 7.1.229
问题: A fold is closed when it shouldn't when 'foldmethod' is "indent" and backspacing a non-white character so that the indent increases.
解决方案: Keep the fold open after backspacing a character.
相关文件: src/edit.c

Patch 7.1.230

问题:　　Memory leak when executing SourceCmd autocommands.
解决方案:　Free the memory. (Dominique Pelle)
相关文件:　src/ex_cmds2.c

Patch 7.1.231
问题:　　When shifting lines the change is acted upon multiple times.
解决方案:　Don't have shift_line() call changed_bytes.
相关文件:　src/edit.c, src/ops.c, src/proto/edit.pro, src/proto/ops.pro

Patch 7.1.232 (after 7.1.207 and 7.1.211)
问题:　　Compiler warnings with MSVC.
解决方案:　Add type casts. (Mike Williams)
相关文件:　src/ex_cmds2.c, src/netbeans.c

Patch 7.1.233
问题:　　Crash when doing Insert mode completion for a user defined
　　command. (Yegappan Lakshmanan)
解决方案:　Don't use the non-existing command line.
相关文件:　src/ex_getln.c

Patch 7.1.234
问题:　　When diff'ing three files the third one isn't displayed correctly.
　　(Gary Johnson)
解决方案:　Compute the size of diff blocks correctly when merging blocks.
　　Compute filler lines correctly when scrolling.
相关文件:　src/diff.c

Patch 7.1.235
问题:　　Pattern matching is slow when using a lot of simple patterns.
解决方案:　Avoid allocating memory by not freeing it when it's not so much.
　　(Alexei Alexandrov)
相关文件:　src/regexp.c

Patch 7.1.236
问题:　　When using 'incsearch' and 'hlsearch' a complicated pattern may
　　make Vim hang until CTRL-C is pressed.
解决方案:　Add the 'redrawtime' option.
相关文件:　runtime/doc/options.txt, src/ex_cmds.c, src/ex_docmd.c,
　　src/ex_getln.c, src/gui.c, src/misc1.c, src/normal.c,
　　src/option.c, src/quickfix.c, src/regexp.c, src/proto/regexp.pro,
　　src/proto/search.pro, src/search.c, src/screen.c,
　　src/option.h, src/spell.c, src/structs.h, src/syntax.c, src/tag.c,
　　src/vim.h

Patch 7.1.237
问题:　　Compiler warning on an Alpha processor in Motif code.
解决方案:　Change a typecast. (Adri Verhoef)
相关文件:　src/gui_motif.c

Patch 7.1.238
问题:　　Using the 'c' flag with searchpair() may cause it to fail. Using
　　the 'r' flag doesn't work when 'wrapscan' is set. (A.Politz)
解决方案:　Only use the 'c' flag for the first search, not for repeating.
　　When using 'r' imply 'W'. (Antony Scriven)

相关文件: src/eval.c

Patch 7.1.239 (after 7.1.233)
问题: Compiler warning for sprintf() argument.
解决方案: Add a typecast. (Nico Weber)
相关文件: src/ex_getln.c

Patch 7.1.240
问题: When "gUe" turns a German sharp s into SS the operation stops before the end of the word. Latin2 has the same sharp s but it's not changed to SS there.
解决方案: Make sure all the characters are operated upon. Detect the sharp s in latin2. Also fixes that changing case of a multi-byte character that changes the byte count doesn't always work.
相关文件: src/ops.c

Patch 7.1.241
问题: Focus change events not always ignored. (Erik Falor)
解决方案: Ignore K_IGNORE in Insert mode in a few more places.
相关文件: src/edit.c

Patch 7.1.242 (after 7.1.005)
问题: "cib" doesn't work properly on "(x)". (Tim Pope)
解决方案: Use ltoreq() instead of lt(). Also fix "ciT" on "<a>x".
相关文件: src/search.c

Patch 7.1.243 (after 7.1.240)
问题: "U" doesn't work on all text in Visual mode. (Adri Verhoef)
解决方案: Loop over all the lines to be changed. Add tests for this.
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.1.244
问题: GUI may have part of the command line cut off.
解决方案: Don't round the number of lines up, always round down. (Tony Houghton, Scott Dillard)
相关文件: src/gui.c

Patch 7.1.245
问题: Pressing CTRL-\ three times causes Vim to quit. (Ranganath Rao). Also for f CTRL-\ CTRL-\.
解决方案: When going to cooked mode in mch_delay() set a flag to ignore SIGQUIT.
相关文件: src/os_unix.c

Patch 7.1.246
问题: Configure hangs when the man pager is something strange. (lorien)
解决方案: Set MANPAGER and PAGER to "cat". (Micah Cowan)
相关文件: src/auto/configure, src/configure.in

Patch 7.1.247
问题: When using Netbeans backspacing in Insert mode skips a character now and then. (Ankit Jain)
解决方案: Avoid calling netbeans_removed(), it frees the line pointer. (partly by Dominique Pelle).

相关文件: src/misc1.c

Patch 7.1.248
问题: Can't set the '" mark. Can't know if setpos() was successful.
解决方案: Allow setting the '" mark with setpos(). Have setpos() return a value indicating success/failure.
相关文件: runtime/doc/eval.txt, src/eval.c, src/mark.c

Patch 7.1.249
问题: After "U" the cursor can be past end of line. (Adri Verhoef)
解决方案: Adjust the cursor position in u_undoline().
相关文件: src/undo.c

Patch 7.1.250
问题: ":setglobal fenc=anything" gives an error message in a buffer where 'modifiable' is off. (Ben Schmidt)
解决方案: Don't give an error if 'modifiable' doesn't matter.
相关文件: src/option.c

Patch 7.1.251
问题: Using freed memory when spell checking enabled.
解决方案: Obtain the current line again after calling spell_move_to(). (Dominique Pelle)
相关文件: src/screen.c

Patch 7.1.252 (after 7.1.243)
问题: Test 39 fails when the environment has a utf-8 locale. (Dominique Pelle)
解决方案: Force 'encoding' to be latin1.
相关文件: src/testdir/test39.in

Patch 7.1.253
问题: ":sort" doesn't work in a one line file. (Patrick Texier)
解决方案: Don't sort if there is only one line. (Dominique Pelle)
相关文件: src/ex_cmds.c

Patch 7.1.254
问题: Tests 49 and 55 fail when the locale is French.
解决方案: Using C messages for test 49. Filter the error message in test 55 such that it works when the number is halfway the message.
相关文件: src/testdir/test49.in, src/testdir/test55.in

Patch 7.1.255
问题: Vim doesn't support utf-32. (Yongwei Wu)
解决方案: Add aliases for utf-32, it's the same as ucs-4.
相关文件: src/mbyte.c

Patch 7.1.256
问题: findfile() also returns directories.
解决方案: Cleanup the code for finding files and directories in a list of directories. Remove the ugly global ff_search_ctx.
相关文件: src/eval.c, src/misc2.c, src/vim.h, src/tag.c

Patch 7.1.257

问题:　　Configure can't always find the Tcl header files.
解决方案:　Also look in /usr/local/include/tcl$tclver and
　　/usr/include/tcl$tclver (James Vega)
相关文件:　src/auto/configure, src/configure.in

Patch 7.1.258
问题:　　Crash when doing "d/\n/e" and 'virtualedit' is "all". (Andy Wokula)
解决方案:　Avoid that the column becomes negative. Also fixes other problems
　　with the end of a pattern match is in column zero. (A.Politz)
相关文件:　src/search.c

Patch 7.1.259
问题:　　Cursor is in the wrong position when 'rightleft' is set,
　　'encoding' is "utf-8" and on an illegal byte. (Dominique Pelle)
解决方案:　Only put the cursor in the first column when actually on a
　　double-wide character. (Yukihiro Nakadaira)
相关文件:　src/screen.c

Patch 7.1.260
问题:　　Cursor positioning problem after ^@ wrapping halfway when
　　'encoding' is utf-8.
解决方案:　Only count a position for printable characters. (partly by
　　Yukihiro Nakadaira)
相关文件:　src/charset.c

Patch 7.1.261
问题:　　When a 2 byte BOM is detected Vim uses UCS-2, which doesn't work
　　for UTF-16 text. (Tony Mechelynck)
解决方案:　Default to UTF-16.
相关文件:　src/fileio.c, src/testdir/test42.ok

Patch 7.1.262
问题:　　Can't get the process ID of Vim.
解决方案:　Implement getpid().
相关文件:　src/eval.c, runtime/doc/eval.txt

Patch 7.1.263
问题:　　The filetype can consist of two dot separated names. This works
　　for syntax and ftplugin, but not for indent. (Brett Stahlman)
解决方案:　Use split() and loop over each dot separated name.
相关文件:　runtime/indent.vim

Patch 7.1.264
问题:　　Crash when indenting lines. (Dominique Pelle)
解决方案:　Set the cursor column when changing the cursor line.
相关文件:　src/ops.c, src/misc1.c

Patch 7.1.265
问题:　　When 'isfname' contains a space, cmdline completion can hang.
　　(James Vega)
解决方案:　Reset the "len" variable.
相关文件:　src/ex_docmd.c

Patch 7.1.266

问题:　　When the version string returned by the terminal contains unexpected characters, it is used as typed input. (James Vega)
解决方案:　Assume the escape sequence ends in a letter.
相关文件:　src/term.c

Patch 7.1.267
问题:　　When changing folds cursor may be positioned in the wrong place.
解决方案:　Call changed_window_setting_win() instead of changed_window_setting().
相关文件:　src/fold.c

Patch 7.1.268
问题:　　Always shows "+" at end of screen line with: ":set listchars=eol:$,extends:+ nowrap list cursorline" (Gary Johnson)
解决方案:　Check for lcs_eol_one instead of lcs_eol.
相关文件:　src/screen.c

Patch 7.1.269
问题:　　The matchparen plugin has an arbitrary limit for the number of lines to look for a match.
解决方案:　Rely on the searchpair() timeout.
相关文件:　runtime/plugin/matchparen.vim

Patch 7.1.270
问题:　　":?foo?" matches in current line since patch 7.1.025. (A.Politz)
解决方案:　Remove the SEARCH_START flag.
相关文件:　src/ex_docmd.c, src/search.c

Patch 7.1.271
问题:　　In a Vim build without autocommands, checking a file that was changed externally causes the current buffer to be changed unexpectedly. (Karsten Hopp)
解决方案:　Store "curbuf" instead of "buf".
相关文件:　src/fileio.c

Patch 7.1.272
问题:　　The special buffer name [Location List] is not used for a buffer displayed in another tab page.
解决方案:　Use FOR_ALL_TAB_WINDOWS instead of FOR_ALL_WINDOWS. (Hiroaki Nishihara)
相关文件:　src/buffer.c

Patch 7.1.273
问题:　　When profiling on Linux Vim exits early. (Liu Yubao)
解决方案:　When profiling don't exit on SIGPROF.
相关文件:　src/Makefile, src/os_unix.c

Patch 7.1.274 (after 7.1.272)
问题:　　Compiler warning for optimized build.
解决方案:　Init win to NULL.
相关文件:　src/buffer.c

Patch 7.1.275 (extra)
问题:　　Mac: ATSUI and 'antialias' don't work properly together.

解决方案: Fix this and the input method. (Jjgod Jiang)
相关文件: src/vim.h, src/gui_mac.c

Patch 7.1.276
问题: "gw" uses 'formatexpr', even though the docs say it doesn't.
解决方案: Don't use 'formatexpr' for "gw".
相关文件: src/vim.h, src/edit.c, src/ops.c, src/proto/ops.pro

Patch 7.1.277
问题: Default for 'paragraphs' misses some items (Colin Watson)
解决方案: Add TP, HP, Pp, Lp and It to 'paragraphs'. (James Vega)
相关文件: runtime/doc/options.txt, src/option.c

Patch 7.1.278 (extra, after 7.1.275)
问题: Build failure when USE_CARBONKEYHANDLER is not defined.
解决方案: Remove #ifdef.
相关文件: src/gui_mac.c

Patch 7.1.279
问题: When using cscope temporary files are left behind.
解决方案: Send the quit command to cscope and give it two seconds to exit nicely before killing it. (partly by Dominique Pelle)
相关文件: src/if_cscope.c

Patch 7.1.280 (after 7.1.275)
问题: Mac: build problems when not using multibyte feature. (Nicholas Stallard)
解决方案: Don't define USE_IM_CONTROL when not using multibyte.
相关文件: src/vim.h

Patch 7.1.281 (after 7.1.279)
问题: sa.sa_mask is not initialized. Cscope may not exit.
解决方案: Use sigemptyset(). Use SIGKILL instead of SIGTERM. (Dominique Pelle)
相关文件: src/if_cscope.c

Patch 7.1.282 (extra)
问题: Win64: Edit with Vim context menu isn't installed correctly. Compiler warnings and a few other things.
解决方案: Add [and] to entry of class name. Use UINT_PTR instead of UINT. And fixes for other things. (George V. Reilly)
相关文件: src/GvimExt/Makefile, src/dosinst.c, src/if_ole.cpp, src/if_ole.h, src/if_ole.idl, src/INSTALLpc.txt, src/Make_mvc.mak, src/os_win32.c,

Patch 7.1.283
问题: Non-extra part for 7.1.282.
解决方案: Various changes.
相关文件: src/ex_docmd.c, src/globals.h, src/if_cscope.c, src/main.c, src/mark.c, src/netbeans.c, src/popupmnu.c, src/vim.h, src/window.c

Patch 7.1.284
问题: Compiler warnings for functions without prototype.

解决方案: Add the function prototypes. (Patrick Texier)
相关文件: src/eval.c, src/quickfix.c

Patch 7.1.285 (extra)
问题: Mac: dialog hotkeys don't work.
解决方案: Add hotkey support. (Dan Sandler)
相关文件: src/gui_mac.c

Patch 7.1.286 (after 7.1.103)
问题: "w" at the end of the buffer moves the cursor past the end of the line. (Markus Heidelberg)
解决方案: Move the cursor back from the NUL when it was moved forward.
相关文件: src/normal.c

Patch 7.1.287
问题: Crash when reversing a list after using it. (Andy Wokula)
解决方案: Update the pointer to the last used element. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.1.288 (after 7.1.281)
问题: Cscope still leaves behind temp files when using gvim.
解决方案: When getting the ECHILD error loop for a while until cscope exits. (Dominique Pelle)
相关文件: if_cscope.c

Patch 7.1.289
问题: When EXITFREE is defined and 'acd' is set freed memory is used. (Dominique Pelle)
解决方案: Reset p_acd before freeing all buffers.
相关文件: src/misc2.c

Patch 7.1.290
问题: Reading bytes that were not written when spell checking and a line has a very large indent.
解决方案: Don't copy the start of the next line when it only contains spaces. (Dominique Pelle)
相关文件: src/spell.c

Patch 7.1.291 (after 7.1.288)
问题: Compiler warning.
解决方案: Change 50 to 50L.
相关文件: src/if_cscope.c

Patch 7.1.292
问题: When using a pattern with "\@<=" the submatches can be wrong. (Brett Stahlman)
解决方案: Save the submatches when attempting a look-behind match.
相关文件: src/regexp.c

Patch 7.1.293
问题: Spell checking considers super- and subscript characters as word characters.
解决方案: Recognize the Unicode super and subscript characters.
相关文件: src/spell.c

Patch 7.1.294
问题:　　Leaking memory when executing a shell command.
解决方案:　Free memory when not able to save for undo. (Dominique Pelle)
相关文件:　src/ex_cmds.c

Patch 7.1.295
问题:　　Vimtutor only works with vim, not gvim.
解决方案:　Add the -g flag to vimtutor. (Dominique Pelle) Add gvimtutor.
相关文件:　src/Makefile, src/gvimtutor, src/vimtutor, runtime/doc/vimtutor.1

Patch 7.1.296
问题:　　SELinux is not supported.
解决方案:　Detect the selinux library and use mch_copy_sec(). (James Vega)
相关文件:　src/auto/configure, src/config.h.in, src/configure.in,
　　src/fileio.c, src/memfile.c, src/os_unix.c, src/proto/os_unix.pro

Patch 7.1.297
问题:　　When using the search/replace dialog the parenmatch highlighting
　　can be wrong. (Tim Duncan)
解决方案:　In the GUI redraw function invoke the CursorMoved autocmd.
相关文件:　src/gui.c

Patch 7.1.298 (after 7.1.295)
问题:　　src/gvimtutor is not distributed.
解决方案:　Add it to the list of distributed files.
相关文件:　Filelist

Patch 7.1.299
问题:　　Filetype detection doesn't work properly for file names ending in
　　a part that is ignored and contain a space or other special
　　characters.
解决方案:　Escape the special characters using the new fnameescape function.
相关文件:　runtime/doc/eval.txt, runtime/filetype.vim, src/eval.c,
　　src/ex_getln.c, src/proto/ex_getln.pro, src/vim.h

Patch 7.1.300
问题:　　Value of asmsyntax argument isn't checked for valid characters.
解决方案:　Only accepts letters and digits.
相关文件:　runtime/filetype.vim

Patch 7.1.301
问题:　　When the "File/Save" menu is used in Insert mode, a tab page label
　　is not updated to remove the "+".
解决方案:　Call draw_tabline() from showruler(). (Bjorn Winckler)
相关文件:　src/screen.c

Patch 7.1.302 (after 7.1.299)
问题:　　Compilation error on MS-Windows.
解决方案:　Don't use xp_shell when it's not defined.
相关文件:　src/ex_getln.c

Patch 7.1.303 (after 7.1.302)
问题:　　Compilation error on MS-Windows, again.

解决方案: Declare p.
相关文件: src/ex_getln.c

Patch 7.1.304
问题: Shortpath_for_invalid_fname() does not work correctly and is unnecessary complex.
解决方案: Clean up shortpath_for_invalid_fname(). (mostly by Yegappan Lakshmanan)
相关文件: src/eval.c

Patch 7.1.305
问题: Editing a compressed file with special characters in the name doesn't work properly.
解决方案: Escape special characters.
相关文件: runtime/autoload/gzip.vim

Patch 7.1.306
问题: Some Unicode characters are handled like word characters while they are symbols.
解决方案: Adjust the table for Unicode classification.
相关文件: src/mbyte.c

Patch 7.1.307
问题: Many warnings when compiling with Python 2.5.
解决方案: Use ssize_t instead of int for some types. (James Vega)
相关文件: src/if_python.c

Patch 7.1.308
问题: When in readonly mode ":options" produces an error.
解决方案: Reset 'readonly'. (Gary Johnson)
相关文件: runtime/optwin.vim

Patch 7.1.309
问题: Installing and testing with a shadow directory doesn't work. (James Vega)
解决方案: Add "po" to the list of directories to link. Also link the Vim scripts in testdir. And a few more small fixes.
相关文件: src/Makefile

Patch 7.1.310
问题: Incomplete utf-8 byte sequence at end of the file is not detected. Accessing memory that wasn't written.
解决方案: Check the last bytes in the buffer for being a valid utf-8 character. (mostly by Ben Schmidt)
Also fix that the reported line number of the error was wrong.
相关文件: src/fileio.c

Patch 7.1.311
问题: Compiler warning for missing sentinel in X code.
解决方案: Change 0 to NULL. (Markus Heidelberg)
相关文件: src/mbyte.c

Patch 7.1.312
问题: The .po files have mistakes in error numbers.

解决方案: Search for these mistakes in the check script. (Dominique Pelle)
相关文件: src/po/check.vim

Patch 7.1.313
问题: When the netbeans interface setModified call is used the status lines and window title are not updated.
解决方案: Redraw the status lines and title. (Philippe Fremy)
相关文件: src/netbeans.c

Patch 7.1.314
问题: The value of 'pastetoggle' is written to the session file without any escaping. (Randall Hansen)
解决方案: Use put_escstr(). (Ben Schmidt)
相关文件: src/option.c

Patch 7.1.315
问题: Crash with specific search pattern using look-behind match. (Andreas Politz)
解决方案: Also save the value of "need_clear_subexpr".
相关文件: src/regexp.c

Patch 7.1.316
问题: When 'cscopetag' is set ":tag" gives an error message instead of going to the next tag in the tag stack.
解决方案: Don't call do_cstag() when there is no argument. (Mark Goldman)
相关文件: src/ex_docmd.c

Patch 7.1.317
问题: Compiler warnings in Motif calls.
解决方案: Change zero to NULL. (Dominique Pelle)
相关文件: src/gui_motif.c

Patch 7.1.318
问题: Memory leak when closing xsmp connection. Crash on exit when using Lesstif.
解决方案: Don't close the X display to work around a Lesstif bug. Free clientid. Also fix a leak for Motif and Athena. (Dominique Pelle)
相关文件: src/gui_x11.c, src/os_unix.c

Patch 7.1.319
问题: When a register has an illegal utf-8 sequence, pasting it on the command line causes an illegal memory access.
解决方案: Use mb_cptr2char_adv(). (Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.1.320 (extra)
问题: Win64: Warnings while compiling Python interface.
解决方案: Use PyInt in more places. Also update version message for the console. (George Reilly)
相关文件: src/if_python.c, src/version.c

Patch 7.1.321 (extra)
问题: Win32 / Win64: Install file is outdated.
解决方案: Update the text for recent compiler. (George Reilly)

相关文件: src/INSTALLpc.txt

Patch 7.1.322
问题: Can't get start of Visual area in an <expr> mapping.
解决方案: Add the 'v' argument to getpos().
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.1.323
问题: Test 19 fails with some termcaps. (Dominique Pelle)
解决方案: Set the t_kb and t_kD termcap values.
相关文件: src/testdir/test19.in, src/testdir/test38.in

Patch 7.1.324
问题: File name path length on Unix is limited to 1024.
解决方案: Use PATH_MAX when it's more than 1000.
相关文件: src/os_unix.h

Patch 7.1.325
问题: When editing a command line that's longer than available space in the window, the characters at the end are in reverse order.
解决方案: Increment the insert position even when the command line doesn't fit. (Ingo Karkat)
相关文件: src/ex_getln.c

Patch 7.1.326
问题: ":s!from!to!" works, but ":smagic!from!to!" doesn't. It sees the "!" as a flag to the command. Same for ":snomagic". (Johan Spetz)
解决方案: When checking for a forced command also ignore ":smagic" and ":snomagic". (Ian Kelling)
相关文件: src/ex_docmd.c

Patch 7.1.327
问题: The GUI tutor is installed when there is no GUI version.
解决方案: Only install gvimtutor when building a GUI version.
相关文件: src/Makefile

Patch 7.1.328
问题: Crash when using Cygwin and non-posix path name in tags file.
解决方案: Use separate buffer for posix path. (Ben Schmidt)
相关文件: src/os_unix.c

Patch 7.1.329
问题: When the popup menu is removed a column of cells, the right halve of double-wide characters, may not be redrawn.
解决方案: Check if the right halve of a character needs to be redrawn. (Yukihiro Nakadaira)
相关文件: src/screen.c

Patch 7.1.330
问题: Reading uninitialized memory when using Del in replace mode.
解决方案: Use utfc_ptr2len_len() instead of mb_ptr2len(). (Dominique Pelle)
相关文件: src/misc1.c

Warning for missing sentinel in gui_xmldlg.c. (Dominique Pelle)

A search offset from the end of a match didn't work properly for multi-byte characters. (Yukihiro Nakadaira)

When displaying the value of 'key' don't show "*****" when the value is empty. (Ben Schmidt)

Internal error when compiled with EXITFREE and using the nerd_tree plugin. Set last_msg_hist to NULL when history becomes empty. Call free_all_functions() after garbage collection. (Dominique Pelle)

GTK with XIM: <S-Space> does not work. (Yukihiro Nakadaira)

Some shells do not support "echo -n", which breaks glob(). Use "echo" instead of "echo -n $1; echo". (Gary Johnson)

"echo 22,44" printed "22" on top of the command, the error messages caused the rest not to be cleared. Added the need_clr_eos flag.

Netbeans events are handled while updating the screen, causing a crash. Change the moment when events are handled. Rename nb_parse_messages() to netbeans_parse_messages(). (Xavier de Gaye)

Test 11 was broken after patch 7.1.186 on Win32 console. (Daniel Shahaf)
Use shellescape() on the file name.

IM was turned off in im_preedit_end_cb() for no good reason. (Takuhiro Nishioka)

A corrupted spell file could cause Vim to use lots of memory. Better detection for running into the end of the file. (idea from James Vega)

Mac: Included a patch to make it build with GTK. Moved language init to mac_lang_init() function. (Ben Schmidt)

Problem with 'wildmenu' after ":lcd", up/down arrows don't work. (Erik Falor)

Fix configure.in to avoid "implicitly declared" warnings when running configure.

Fixed a memory leak when redefining a keymap. (Dominique Pelle)

Setting 'pastetoggle' to "jj" didn't work.

'ic' and 'smartcase' don't work properly when using \%V in a search pattern. (Kana Natsuno)

Patch 7.2a.001
问题: On some systems X11/Xlib.h exists (from X11-dev package) but X11/Intrinsic.h does not (in Xt-dev package). This breaks the build. Also, on Solaris 9 sys/ptem.h isn't found.
解决方案: Have configure only accept X11 when X11/Intrinsic.h exists. Check for sys/ptem.h while including sys/stream.h. (Vladimir

Marek)
相关文件: src/auto/configure, src/configure.in

Patch 7.2a.002
问题: getbufvar(N, "") gets the dictionary of the current buffer instead of buffer N.
解决方案: Set curbuf before calling find_var_in_ht(). (Kana Natsuno)
相关文件: src/eval.c

Patch 7.2a.003
问题: Leaking memory when using ":file name" and using access control lists.
解决方案: Invoke mch_free_acl() in vim_rename(). (Dominique Pelle)
相关文件: src/fileio.c

Patch 7.2a.004
问题: Some systems can't get spell files by ftp.
解决方案: Use http when it looks like it's possible. (James Vega)
相关文件: runtime/autoload/spellfile.vim

Patch 7.2a.005
问题: A few error messages use confusing names. Misspelling.
解决方案: Change "dissallows" to "disallows". (Dominique Pelle) Change "number" to "Number".
相关文件: src/eval.c, src/fileio.c

Patch 7.2a.006
问题: Reading past NUL in a string.
解决方案: Check for invalid utf-8 byte sequence. (Dominique Pelle)
相关文件: src/charset.c

Patch 7.2a.007
问题: ":let v = 1.2.3" was OK in Vim 7.1, now it gives an error.
解决方案: Don't look for a floating point number after the "." operator.
相关文件: src/eval.c

Patch 7.2a.008
问题: printf("%g", 1) doesn't work.
解决方案: Convert Number to Float when needed.
相关文件: src/message.c

Patch 7.2a.009
问题: cygwin_conv_to_posix_path() does not specify buffer size.
解决方案: Use new Cygwin function: cygwin_conv_path(). (Corinna Vinschen)
相关文件: src/main.c, src/os_unix.c

Patch 7.2a.010
问题: When a file name has an illegal byte sequence Vim may read uninitialised memory.
解决方案: Don't use UTF_COMPOSINGLIKE() on an illegal byte. In msg_outtrans_len_attr() use char2cells() instead of ptr2cells(). In utf_ptr2char() don't check second byte when first byte is illegal. (Dominique Pelle)
相关文件: src/mbyte.c, src/message.c

Patch 7.2a.011
问题:　　The Edit/Startup Settings menu doesn't work.
解决方案:　Expand environment variables. (Ben Schmidt)
相关文件:　runtime/menu.vim

Patch 7.2a.012
问题:　　Compiler warnings for casting int to pointer.
解决方案:　Add cast to long in between. (Martin Toft)
相关文件:　src/gui_gtk_x11.c

Patch 7.2a.013
问题:　　shellescape() does not escape "%" and "#" characters.
解决方案:　Add find_cmdline_var() and use it when the second argument to
　　shellescape() is non-zero.
相关文件:　runtime/doc/eval.txt, src/eval.c, src/ex_docmd.c,
　　src/proto/ex_docmd.pro, src/proto/misc2.pro, src/misc2.c

Patch 7.2a.014
问题:　　Problem with % in message.
解决方案:　Put % in single quotes.
相关文件:　src/eval.c

Patch 7.2a.015 (after 7.2a.010)
问题:　　Misaligned messages.
解决方案:　Compute length of unprintable chars correctly.
相关文件:　src/message.c

Patch 7.2a.016
问题:　　Using CTRL-W v in the quickfix window results in two quickfix
　　windows, which is not allowed. ":tab split" should be allowed to
　　open a new quickfix window in another tab.
解决方案:　For CTRL-W v instead of splitting the window open a new one.
　　When using ":tab" do allow splitting the quickfix window (was
　　already included in patch 7.2a.013).
相关文件:　src/window.c

Patch 7.2a.017
问题:　　":doautoall" executes autocommands for all buffers instead of just
　　for loaded buffers.
解决方案:　Change "curbuf" to "buf".
相关文件:　src/fileio.c

Patch 7.2a.018
问题:　　Compiler warnings when compiling with Gnome. (Tony Mechelynck)
解决方案:　Add type casts.
相关文件:　src/gui_gtk_x11.c

Patch 7.2a.019
问题:　　":let &g:tw = 44" sets the local option value. (Cyril Slobin)
解决方案:　Use get_varp_scope() instead of get_varp(). (Ian Kelling)
相关文件:　src/option.c

There is no way to avoid adding /usr/local/{include|lib} to the build

commands. Add the --with-local-dir argument to configure. (Michael Haubenwallner)

When using CTRL-D after ":help", the number of matches could be thousands. Restrict to TAG_MANY to avoid this taking too long. (Ian Kelling)

The popup menu could be placed at a weird location. Caused by w_wcol computed by curs_columns(). (Dominique Pelle)

Overlapping STRCPY() arguments when using %r item in 'errorformat'. Use STRMOVE() instead. (Ralf Wildenhues)

Mac: On Leopard gvim, when using the mouse wheel nothing would happen until another event occurs, such as moving the mouse. Then the recorded scrolling would take place all at once. (Eckehard Berns)

Solution for cursor color not reflecting IM status for GTK 2. Add preedit_is_active flag. (SungHyun Nam)

filereadable() can hang on a FIFO on Linux. Use open() instead of fopen(), with O_NONBLOCK. (suggested by Lars Kotthoff)

Included patch to support Perl 5.10. (Yasuhiro Matsumoto)

When files are dropped on gvim while the screen is being updated, ignore the drop command to avoid freeing memory that is being used.

In a terminal, when drawing the popup menu over double-wide characters, half characters may not be cleared properly. (Yukihiro Nakadaira)

The #ifdef for including "vimio.h" was inconsistent. In a few files it depended on MSWIN, which isn't defined until later.

Patch 7.2b.001
问题: Compilation problem: mb_fix_col() missing with multi-byte feature but without GUI or clipboard.
解决方案: Remove #ifdef.
相关文件: src/mbyte.c

Patch 7.2b.002
问题: Compiler warnings for signed/unsigned mismatch.
解决方案: Add type casts.
相关文件: src/screen.c

Patch 7.2b.003
问题: Still a compilation problem, check_col() and check_row() missing.
解决方案: Add FEAT_MBYTE to the #if.
相关文件: src/ui.c

Patch 7.2b.004
问题: Trying to free memory for a static string when using ":helpgrep". (George Reilly)
解决方案: Set 'cpo' to empty_option instead of an empty string. Also for searchpair() and substitute().

相关文件: src/quickfix.c, src/eval.c

Patch 7.2b.005
问题: The special character "!" isn't handled properly in shellescape().
(Jan Minar)
解决方案: Escape "!" when using a "csh" like shell and with
shellescape(s, 1). Twice for both. Also escape <NL>.
相关文件: src/misc2.c

Patch 7.2b.006
问题: Reading past end of string when reading info from tags line.
解决方案: Break the loop when encountering a NUL. (Dominique Pelle)
相关文件: src/tag.c

Patch 7.2b.007
问题: Part of a message cannot be translated.
解决方案: Put _() around the message.
相关文件: src/search.c

Patch 7.2b.008
问题: A few filetypes are not detected or not detected properly.
解决方案: Add filetype detection patterns. (Nikolai Weibull)
相关文件: runtime/filetype.vim

Patch 7.2b.009
问题: Reading past end of screen line. (Epicurus)
解决方案: Avoid going past the value of Columns.
相关文件: src/screen.c

Patch 7.2b.010
问题: ":mksession" doesn't work for ":map , foo", ":sunmap ,". (Ethan
Mallove)
解决方案: Check for "nxo", "nso" and other strange mapping combinations.
相关文件: src/getchar.c

Patch 7.2b.011
问题: Configure for TCL ends up with include file in compiler command.
(Richard Hogg)
解决方案: Delete items from $TCL_DEFS that do not start with a dash.
相关文件: src/auto/configure, src/configure.in

Patch 7.2b.012
问题: Build failure with +multi_byte but without +diff.
解决方案: Add #ifdef. (Patrick Texier)
相关文件: src/main.c

Patch 7.2b.013
问题: Build fails with tiny features and Perl. (Dominique Pelle)
解决方案: Define missing functions. Also when compiling Python.
相关文件: src/if_perl.xs, src/if_python.c

Patch 7.2b.014
问题: Configure uses an unsafe temp file to store commands.
解决方案: Create the temp file in local directory.

相关文件: src/auto/configure, src/configure.in

Patch 7.2b.015
问题: Build fails on Mac when using Aap.
解决方案: Fix typo in configure script.
相关文件: src/auto/configure, src/configure.in

Patch 7.2b.016
问题: Build fails with normal features but without +autocmd.
解决方案: Fix #ifdefs. (Ian Kelling)
相关文件: src/eval.c, src/ex_cmds.c, src/quickfix.c, src/option.c,
src/ex_docmd.c

Patch 7.2b.017
问题: "vim -O foo foo" results in only one window. (Zdenek Sekera)
解决方案: Handle result of ATTENTION prompt properly. (Ian Kelling)
相关文件: src/main.c

Patch 7.2b.018
问题: When doing command line completion on a file name for a csh-like
shell argument a '!' character isn't escaped properly.
解决方案: Add another backslash.
相关文件: src/ex_getln.c, src/misc2.c, src/proto/misc2.pro, src/screen.c

Patch 7.2b.019 (extra)
问题: Win32: Various compiler warnings.
解决方案: Use __w64 attribute. Comment-out unused parameters. Adjust a few
#ifdefs. (George Reilly)
相关文件: src/gui_w48.c, src/GvimExt/gvimext.cpp, src/Make_mvc.mak,
src/os_mswin.c, src/os_win32.c, src/vim.h

Patch 7.2b.020
问题: ":sort n" doesn't handle negative numbers. (James Vega)
解决方案: Include '-' in the number.
相关文件: src/charset.c, src/ex_cmds.c

Patch 7.2b.021
问题: Reloading doesn't read the BOM correctly. (Steve Gardner)
解决方案: Accept utf-8 BOM when specified file encoding is utf-8.
相关文件: src/fileio.c

Patch 7.2b.022
问题: When using ":normal" while updating the status line the count of
an operator is lost. (Dominique Pelle)
解决方案: Save and restore "opcount".
相关文件: src/ex_docmd.c, src/globals.h, src/normal.c

Patch 7.2b.023
问题: Crash when using the result of synstack(0,0). (Matt Wozniski)
解决方案: Check for v_list to be NULL in a few more places.
相关文件: src/eval.c

Patch 7.2b.024
问题: Using ":gui" while the netrw plugin is active causes a delay in

updating the display.
解决方案: Don't check for terminal codes when starting the GUI.
相关文件: src/term.c

Patch 7.2b.025
问题: When the CursorHold event triggers a pending count is lost.
(Juergen Kraemer)
解决方案: Save the counts and restore them.
相关文件: src/normal.c, src/structs.h

Patch 7.2b.026
问题: The GTK 2 file chooser causes the ~/.recently-used.xbel file to be
written over and over again. This may cause a significant
slowdown. (Guido Berhoerster)
解决方案: Don't use the GTK 2 file chooser.
相关文件: src/gui_gtk.c

Patch 7.2b.027
问题: Memory leak for Python, Perl, etc. script command with end marker.
解决方案: Free the memory of the end marker. (Andy Kittner)
相关文件: src/ex_getln.c

Patch 7.2b.028
问题: Reading uninitialized memory when doing ":gui -f". (Dominique
Pelle)
解决方案: Don't position the cursor when the screen size is invalid.
相关文件: src/gui.c

Patch 7.2b.029
问题: ":help a" doesn't jump to "a" tag in docs. (Tony Mechelynck)
解决方案: Get all tags and throw away more than TAG_MANY after sorting.
When there is no argument find matches for "help" to avoid a long
delay.
相关文件: src/ex_cmds.c, src/ex_getln.c

Patch 7.2b.030
问题: When changing the value of t_Co from 8 to 16 the Visual
highlighting keeps both reverse and a background color.
解决方案: Remove the attribute when setting the default highlight color.
(Markus Heidelberg)
相关文件: src/syntax.c

Error when cancelling completion menu and auto-formatting. (fixed by Ian
Kelling)

Patch 7.2c.001
问题: ":let x=[''] | let x += x" causes hang. (Matt Wozniski)
解决方案: Only insert elements up to the original length of the List.
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.2c.002
问题: fnameescape() doesn't handle a leading '+' or '>'. (Jan Minar)
解决方案: Escape a leading '+' and '>'. And a single '-'.
相关文件: runtime/doc/eval.txt, src/ex_getln.c

Patch 7.2c.003
问题: Searching for "foo\%[bar]\+" gives a "Corrupted regexp program" error. (Joachim Hofmann)
解决方案: Mark the \%[] item as not being simple.
相关文件: src/regexp.c

On Vista access to system directories is virtualized. (Michael Mutschler) Adjusted the manifest file to avoid this. (George Reilly)

Memory leak when using CTRL-C to cancel listing the jump list. (Dominique Pelle)

Mac: Could not build with Perl interface.

7.3 版本 version-7.3 version7.3

本小节讨论 7.2 和 7.3 版本之间的改进。

这个版本有几百个漏洞修正，和若干新特性。最主要的新特性有：

持久化撤销 new-persistent-undo

保存撤销信息到文件。可以撤销到文件读入之前的状态，即使是已卸载的缓冲区也可。见 undo-persistence (部分由 Jordan Lewis 提供)

新增 ":earlier 1f" 和 ":later 1f" 命令。
撤销信息中新增文件保存计数。
新增 undotree() 和 undofile() 函数。

新增 'undoreload' 选项。可以在缓冲区重载时保存当前文本，以便使重载可以撤销。

更多加密 new-more-encryption

支持 Blowfish 加密。新增 'cryptmethod' 选项。
主要由 Mohsin Ahmed 提供。

另外，也对交换文件和撤销文件的文本进行加密。

隐藏文本 new-conceal

新增 +conceal 功能。(Vince Negri)
这样可以基于语法高亮隐藏成串的文本。
也可以用单个字符替换成串的文本 :syn-cchar 。
'conceallevel' 选项指定文本匹配包含 conceal 属性的语法项目时如何操作。
'concealcursor' 选项指定光标行的行为。

帮助文件隐藏用于标记标签和示例的字符。

新增 synconcealed() 函数，用于 :TOhtml。(Benjamin Fritz)

新增 'cursorbind' 函数，使相同文本的两个窗口光标保持同步。

Lua 接口 new-lua

新增 Lua 接口。(Luis Carvalho)

Python3 接口 new-python3

新增 Python3 接口。 它和 Python 2.x 同时支持，也可以同时使用。见 python3
(Roland Puntaier)

改动 changed-7.3

MS-Windows 安装程序不再需要用户在控制台输入任何字符。安装程序现在也可用于 64
位系统，包括 "Edit with Vim" 上下文菜单。
gvim 可执行程序是 32 位的，安装的 gvimext.dll 可以是 32 或 64 位 (主要由
George Reilly 提供)
DOS 安装程序可用于更多编译器。
MS-Windows big gvim 现在使用 Python 2.7 和 3.1.2、Perl 5.12 和 Ruby 1.9.1。需
要配套的 .dll 文件才能使用。

附加和语言文件不再分别发布。
所有系统需要的源代码都包含在同一个发布里。

用 ":recover" 或其它方式来恢复文件后，":x" 和 "ZZ" 过去不保存你看到的内容。这
会使得工作丢失。现在恢复后的文本会和原来文本比较。如果不同，标注缓冲区为已修改
。

如果 Vim 因为收到致命信号而退出且 v:dying 为 2 或以上，不执行 VimLeavePre、
VimLeave、BufWinLeave 和 BufUnload 自动命令。

删除 GTK 1 的支持。没有人维护它，而且源代码需要很多的 #ifdef。GTK 2 应该在所有
的系统上都可用了。(James Vega)

不再允许在模式行上设置 'encoding' 选项。可能会破坏文本。(Patrick Texier)

把 runtime/spell/fixdup 换名为 runtime/spell/fixdup.vim。

删除废弃的 Mac 代码。

更新拼写文件使用的 Ubuntu locale 名。

切换 autoconf 2.63 到 2.65。

删除 Mupad 缩进和文件类型插件文件，没有用。

在历史中记忆的消息最大数目现在是 200（原来是 100）。

新增 added-7.3

加入 'relativenumber' 选项。(Markus Heidelberg)

加入 'colorcolumn' 选项：高亮窗口的一到多个列。例如高亮 'textwidth' 之后的列。(主要由 Gregor Uhlenheuer 提供)

加入 NetBeans 的终端支持。加入 :nbstart 和 :nbclose 。(Xavier de Gaye)

更多浮点函数：acos() 、asin() 、atan2() 、cosh() 、exp() 、fmod() 、log() 、sinh() 、tan() 、tanh() 。(Bill McCarthy)

加入 gettabvar() 和 settabvar() 函数。(Yegappan Lakshmanan)

加入 strchars() 、strwidth() 和 strdisplaywidth() 函数。

支持 Mac 上 GTK 的 GDK_SUPER_MASK。(Stephan Schulz)

使 CTRL 和 ALT 修饰符可用于鼠标滚轮。(Benjamin Haskell)

加入横向滚轮的支持。(Bjorn Winckler)

缓冲区处于比较模式时，:TOhtml 创建显示并排比较的 HTML。(Christian Brabandt)

":TOhtml" 和 2html.vim 脚本的若干改进。(Benjamin Fritz)

'cinoptions' 新增 'L' 项目。(Manuel Konig)

改进 Javascript 缩进。'cinoptions' 新增 "J" 标志位。(Hari Kumar G)

Mac: 支持关闭反锯齿。(LC Mi)

Mac: 新增 Mac 控制台的剪贴板支持。(Bjorn Winckler)

支持把标签页标签拖拽到其它位置。(Paul B. Mahol)

建立色彩方案菜单的更好的实现。(Juergen Kraemer)

可视模式下使用 'showcmd' 显示字节和字符数。

编译不带 GUI 时使 synIDattr() 可以得到 GUI 属性。(Matt Wozniski)

支持 ":find" 进行补全。加入测试 73。(Nazri Ramliy)

:ownsyntax 和 :setfiletype 的命令行补全。(Dominique Pelle)

:lmap 和 :lunmap 的命令行补全。

支持用户命令的语法和文件类型补全。(Christian Brabandt)

避免使用 GTK mail_loop()，以便使用 GtkFileChooser。(James Vega)

'formatexpr' 计算非零时回到内部排版机制，"gq" 亦同。(James Vega)

:browse 支持接受错误文件的参数的命令。(Lech Lorens)

gvimext 支持宽文件名。(Szabolcs Horvat)

改进合并行的测试。(Milan Vancura)
使行范围的合并加快很多。(Milan Vancura)

新增改进 z/OS (OS/390) 支持的补丁。(Ralf Schandl)

新增 helphelp.txt 文件。文本来自原来的 various.txt。

新增 'statusline' 的 "q" 项目。新增 w:quickfix_title 。(Lech Lorens)

VMS 的各种改进。(Zoltan Arpadffy)

新语法文件:
Haskell Cabal 构造文件 (Vincent Berthoux)
ChaiScript (Jason Turner)
Cucumber (Tim Pope)
Datascript (Dominique Pelle)
Fantom (Kamil Toman)
Liquid (Tim Pope)
Logcheck (Debian)
Markdown (Tim Pope)
Perl 6 (Andy Lester)
Quickfix window (Lech Lorens)
wavefront 的 obj 文件 (Vincent Berthoux)
Perl 6 (Andy Lester)
SDC - Synopsys Design Constraints (Maurizio Tranchero)
SVG - Scalable Vector Graphics (Vincent Berthoux)
task data (John Florian)
task 42 edit (John Florian)

新文件类型插件:
Cucumber (Tim Pope)
Liquid (Tim Pope)
Markdown (Tim Pope)
Tcl (Robert L Hicks)

新缩进插件:
CUDA (Bram Moolenaar)
ChaiScript (Jason Turner)
Cucumber (Tim Pope)
LifeLines (Patrick Texier)
Liquid (Tim Pope)
Mail (Bram Moolenaar)
Perl 6 (Andy Lester)

其它新运行时文件：
布列塔尼语拼写文件 (Dominique Pelle)
Dvorak 键盘映射表 (Ashish Shukla)
韩语翻译。(SungHyun Nam)
Python 3 补全 (Aaron Griffin)
塞尔维亚菜单翻译 (Aleksandar Jelenak)
德顿语拼写文件
巴伐利亚语教程 (Sepp Hell)
世界语教程。(Dominique Pellé)
葡萄牙语教程。
挪威教程现在可从 tutor.nb 获得

删除 Mupad 运行时文件，没有人维护。

修正 fixed-7.3

Patch 7.2.001
问题: Mac: pseudo-ttys don't work properly on Leopard, resulting in the shell not to have a prompt, CTRL-C not working, etc.
解决方案: Don't use SVR4 compatible ptys, even though they are detected. (Ben Schmidt)
相关文件: src/pty.c

Patch 7.2.002
问题: Leaking memory when displaying menus.
解决方案: Free allocated memory. (Dominique Pelle)
相关文件: src/menu.c

Patch 7.2.003
问题: Typo in translated message. Message not translated.
解决方案: Correct spelling. Add _(). (Dominique Pelle)
相关文件: src/spell.c, src/version.c

Patch 7.2.004
问题: Cscope help message is not translated.
解决方案: Put it in _(). (Dominique Pelle)
相关文件: src/if_cscope.c, src/if_cscope.h

Patch 7.2.005
问题: A few problems when profiling. Using flag pointer instead of flag value. Allocating zero bytes. Not freeing used memory.
解决方案: Remove wrong '&' characters. Skip dumping when there is nothing to dump. Free used memory. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.2.006
问题: HTML files are not recognized by contents.
解决方案: Add a rule to the scripts file. (Nico Weber)
相关文件: runtime/scripts.vim

Patch 7.2.007 (extra)

问题:	Minor issues for VMS.
解决方案:	Minor fixes for VMS. Add float support. (Zoltan Arpadffy)
相关文件:	runtime/doc/os_vms.txt, src/os_vms_conf.h, src/Make_vms.mms, src/testdir/Make_vms.mms, src/testdir/test30.in, src/testdir/test54.in

Patch 7.2.008
问题:	With a BufHidden autocommand that invokes ":bunload" the window count for a buffer can be wrong. (Bob Hiestand)
解决方案:	Don't call enter_buffer() when already in that buffer.
相关文件:	src/buffer.c

Patch 7.2.009
问题:	Can't compile with Perl 5.10 on MS-Windows. (Cesar Romani)
解决方案:	Add the Perl_sv_free2 function for dynamic loading. (Dan Sharp)
相关文件:	src/if_perl.xs

Patch 7.2.010
问题:	When using "K" in Visual mode not all characters are properly escaped. (Ben Schmidt)
解决方案:	Use a function with the functionality of shellescape(). (Jan Minar)
相关文件:	src/mbyte.c, src/misc2.c, src/normal.c

Patch 7.2.011
问题:	Get an error when inserting a float value from the expression register.
解决方案:	Convert the Float to a String automatically in the same place where a List would be converted to a String.
相关文件:	src/eval.c

Patch 7.2.012
问题:	Compiler warnings when building with startup timing.
解决方案:	Add type casts.
相关文件:	src/ex_cmds2.c

Patch 7.2.013
问题:	While waiting for the X selection Vim consumes a lot of CPU time and hangs until a response is received.
解决方案:	Sleep a bit when the selection event hasn't been received yet. Time out after a couple of seconds to avoid a hang when the selection owner isn't responding.
相关文件:	src/ui.c

Patch 7.2.014
问题:	synstack() doesn't work in an empty line.
解决方案:	Accept column zero as a valid position.
相关文件:	src/eval.c

Patch 7.2.015
问题:	"make all test install" doesn't stop when the test fails. (Daniel Shahaf)
解决方案:	When test.log contains failures exit with non-zero status.
相关文件:	src/testdir/Makefile

Patch 7.2.016
问题: The pattern being completed may be in freed memory when the command line is being reallocated. (Dominique Pelle)
解决方案: Keep a pointer to the expand_T in the command line structure. Don't use <S-Tab> as CTRL-P when there are no results. Clear the completion when using a command line from the history.
相关文件: src/ex_getln.c

Patch 7.2.017
问题: strlen() used on text that may not end in a NUL. (Dominique Pelle) Pasting a very big selection doesn't work.
解决方案: Use the length passed to the XtSelectionCallbackProc() function. After getting the SelectionNotify event continue dispatching events until the callback is actually called. Also dispatch the PropertyNotify event.
相关文件: src/ui.c

Patch 7.2.018
问题: Memory leak when substitute is aborted.
解决方案: Free the buffer allocated for the new text. (Dominique Pelle)
相关文件: src/ex_cmds.c

Patch 7.2.019
问题: Completion of ":noautocmd" doesn't work and exists(":noautocmd") returns zero. (Ben Fritz)
解决方案: Add "noautocmd" to the list of modifiers and commands.
相关文件: src/ex_cmds.h, src/ex_docmd.c

Patch 7.2.020
问题: Starting the GUI when the executable starts with 'k', but the KDE version no longer exists.
解决方案: Don't have "kvim" start the GUI.
相关文件: src/main.c

Patch 7.2.021
问题: When executing autocommands getting the full file name may be slow. (David Kotchan)
解决方案: Postpone calling FullName_save() until autocmd_fname is used.
相关文件: src/ex_docmd.c, src/fileio.c, src/globals.h

Patch 7.2.022 (extra)
问题: Testing is not possible when compiling with MingW.
解决方案: Add a MingW specific test Makefile. (Bill McCarthy)
相关文件: Filelist, src/testdir/Make_ming.mak

Patch 7.2.023
问题: 'cursorcolumn' is in the wrong place in a closed fold when the display is shifted left. (Gary Johnson)
解决方案: Subtract w_skipcol or w_leftcol when needed.
相关文件: src/screen.c

Patch 7.2.024
问题: It's possible to set 'history' to a negative value and that causes

an out-of-memory error.
解决方案: Check that 'history' has a positive value. (Doug Kearns)
相关文件: src/option.c

Patch 7.2.025
问题: When a CursorHold event invokes system() it is retriggered over and over again.
解决方案: Don't reset did_cursorhold when getting K_IGNORE.
相关文件: src/normal.c

Patch 7.2.026 (after 7.2.010)
问题: "K" doesn't use the length of the identifier but uses the rest of the line.
解决方案: Copy the desired number of characters first.
相关文件: src/normal.c

Patch 7.2.027
问题: Can use cscope commands in the sandbox.
解决方案: Disallow them, they might not be safe.
相关文件: src/ex_cmds.h

Patch 7.2.028
问题: Confusing error message for missing ().
解决方案: Change "braces" to "parentheses". (Gary Johnson)
相关文件: src/eval.c

Patch 7.2.029
问题: No completion for ":doautoall".
解决方案: Complete ":doautoall" like ":doautocmd". (Doug Kearns)
相关文件: src/ex_docmd.c

Patch 7.2.030 (after 7.2.027)
问题: Can't compile.
解决方案: Remove prematurely added ex_oldfiles.
相关文件: src/ex_cmds.h

Patch 7.2.031
问题: Information in the viminfo file about previously edited files is not available to the user. There is no way to get a complete list of files edited in previous Vim sessions.
解决方案: Add v:oldfiles and fill it with the list of old file names when first reading the viminfo file. Add the ":oldfiles" command, ":browse oldfiles" and the "#<123" special file name. Increase the default value for 'viminfo' from '20 to '100.
相关文件: runtime/doc/cmdline.txt, runtime/doc/eval.txt, runtime/doc/starting.txt, runtime/doc/usr_21.txt, src/eval.c, src/ex_cmds.c, src/ex_cmds.h, src/ex_docmd.c, src/feature.h, src/fileio.c, src/main.c, src/mark.c, src/misc1.c, src/proto/eval.pro, src/proto/ex_cmds.pro, src/proto/mark.pro, src/option.c, src/structs.h, src/vim.h

Patch 7.2.032 (after 7.2.031)
问题: Can't build with EXITFREE defined. (Dominique Pelle)
解决方案: Change vv_string to vv_str.

相关文件: src/eval.c

Patch 7.2.033
问题: When detecting a little endian BOM "ucs-2le" is used, but the text might be "utf-16le".
解决方案: Default to "utf-16le", it also works for "ucs-2le". (Jia Yanwei)
相关文件: src/fileio.c, src/testdir/test42.ok

Patch 7.2.034
问题: Memory leak in spell info when deleting buffer.
解决方案: Free the memory. (Dominique Pelle)
相关文件: src/buffer.c

Patch 7.2.035
问题: Mismatches between alloc/malloc, free/vim_free, realloc/vim_realloc.
解决方案: Use the right function. (Dominique Pelle)
相关文件: src/gui_x11.c, src/mbyte.c, src/misc2.c, src/os_unix.c

Patch 7.2.036 (extra)
问题: Mismatches between alloc/malloc, free/vim_free, realloc/vim_realloc.
解决方案: Use the right function. (Dominique Pelle)
相关文件: src/gui_riscos.c, src/gui_w48.c, src/mbyte.c, src/os_vms.c, src/os_w32exe.c, src/os_win16.c

Patch 7.2.037
问题: Double free with GTK 1 and compiled with EXITFREE.
解决方案: Don't close display. (Dominique Pelle)
相关文件: src/os_unix.c

Patch 7.2.038
问题: Overlapping arguments to memcpy().
解决方案: Use mch_memmove(). (Dominique Pelle)
相关文件: src/if_xcmdsrv.c

Patch 7.2.039
问题: Accessing freed memory on exit when EXITFREE is defined.
解决方案: Call hash_init() on the v: hash table.
相关文件: src/eval.c

Patch 7.2.040
问题: When using ":e ++ff=dos fname" and the file contains a NL without a CR before it and 'ffs' contains "unix" then the fileformat becomes unix.
解决方案: Ignore 'ffs' when using the ++ff argument. (Ben Schmidt)
Also remove unreachable code.
相关文件: src/fileio.c

Patch 7.2.041
问题: In diff mode, when using two tabs, each with two diffed buffers, editing a buffer of the other tab messes up the diff. (Matt Mzyzik)
解决方案: Only copy options from a window where the buffer was edited that

doesn't have 'diff' set or is for the current tab page.
Also fix that window options for a buffer are stored with the
wrong window.
相关文件: src/buffer.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c,
src/ex_getln.c, src/if_sniff.c, src/main.c, src/netbeans.c,
src/normal.c, src/popupmnu.c, src/proto/buffer.pro,
src/proto/ex_cmds.pro src/quickfix.c, src/window.c

Patch 7.2.042
问题: When using winrestview() in a BufWinEnter autocommand the window
is scrolled anyway. (Matt Zyzik)
解决方案: Don't recompute topline when above 'scrolloff' from the bottom.
Don't always put the cursor halfway when entering a buffer. Add
"w_topline_was_set".
相关文件: src/buffer.c, src/move.c, src/structs.h

Patch 7.2.043
问题: VMS: Too many characters are escaped in filename and shell
commands.
解决方案: Escape fewer characters. (Zoltan Arpadffy)
相关文件: src/vim.h

Patch 7.2.044
问题: Crash because of STRCPY() being over protective of the destination
size. (Dominique Pelle)
解决方案: Add -D_FORTIFY_SOURCE=1 to CFLAGS. Use an intermediate variable
for the pointer to avoid a warning.
相关文件: src/auto/configure, src/configure.in, src/eval.c

Patch 7.2.045
问题: The Python interface has an empty entry in sys.path.
解决方案: Filter out the empty entry. (idea from James Vega)
相关文件: src/if_python.c

Patch 7.2.046
问题: Wrong check for filling buffer with encoding. (Danek Duvall)
解决方案: Remove pointers. (Dominique Pelle)
相关文件: src/mbyte.c

Patch 7.2.047
问题: Starting Vim with the -nb argument while it's not supported causes
the other side to hang.
解决方案: When -nb is used while it's not supported exit Vim. (Xavier de
Gaye)
相关文件: src/main.c, src/vim.h

Patch 7.2.048
问题: v:prevcount is changed too often. Counts are not multiplied when
setting v:count.
解决方案: Set v:prevcount properly. Multiply counts. (idea by Ben Schmidt)
相关文件: src/eval.c, src/normal.c, src/proto/eval.pro

Patch 7.2.049 (extra)
问题: Win32: the clipboard doesn't support UTF-16.

解决方案: Change UCS-2 support to UTF-16 support. (Jia Yanwei)
相关文件: src/gui_w32.c, src/gui_w48.c, src/mbyte.c, src/misc1.c,
           src/os_mswin.c, src/os_win32.c, src/proto/os_mswin.pro

Patch 7.2.050
问题: Warnings for not checking return value of fwrite(). (Chip Campbell)
解决方案: Use the return value.
相关文件: src/spell.c

Patch 7.2.051
问题: Can't avoid 'wildignore' and 'suffixes' for glob() and globpath().
解决方案: Add an extra argument to these functions. (Ingo Karkat)
相关文件: src/eval.c, src/ex_getln.c, src/proto/ex_getln.pro,
           runtime/doc/eval.txt, runtime/doc/options.txt

Patch 7.2.052
问题: synIDattr() doesn't support "sp" for special color.
解决方案: Recognize "sp" and "sp#". (Matt Wozniski)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.2.053
问题: Crash when using WorkShop command ":ws foo". (Dominique Pelle)
解决方案: Avoid using a NULL pointer.
相关文件: src/workshop.c

Patch 7.2.054
问题: Compilation warnings for format in getchar.c.
解决方案: Use fputs() instead of fprintf(). (Dominique Pelle)
相关文件: src/getchar.c

Patch 7.2.055
问题: Various compiler warnings with strict checking.
解决方案: Avoid the warnings by using return values and renaming.
相关文件: src/diff.c, src/eval.c, src/ex_cmds.c, src/ex_docmd.c,
           src/fileio.c, src/fold.c, src/globals.h, src/gui.c,
           src/gui_at_sb.c, src/gui_gtk_x11.c, src/gui_xmdlg.c,
           src/gui_xmebw.c, src/main.c, src/mbyte.c, src/message.c,
           src/netbeans.c, src/option.c, src/os_unix.c, src/spell.c,
           src/ui.c, src/window.c

Patch 7.2.056 (after 7.2.050)
问题: Tests 58 and 59 fail.
解决方案: Don't invoke fwrite() with a zero length. (Dominique Pelle)
相关文件: src/spell.c

Patch 7.2.057 (after 7.2.056)
问题: Combination of int and size_t may not work.
解决方案: Use size_t for variable.
相关文件: src/spell.c

Patch 7.2.058
问题: Can't add a patch name to the ":version" output.
解决方案: Add the extra_patches array.
相关文件: src/version.c

Patch 7.2.059
问题: Diff display is not always updated.
解决方案: Update the display more often.
相关文件: src/diff.c

Patch 7.2.060
问题: When a spell files has many compound rules it may take a very long
time making the list of suggestions. Displaying also can be slow
when there are misspelled words.
Can't parse some Hunspell .aff files.
解决方案: Check if a compounding can possibly work before trying a
combination, if the compound rules don't contain wildcards.
Implement using CHECKCOMPOUNDPATTERN.
Ignore COMPOUNDRULES. Ignore a comment after most items.
Accept ONLYINCOMPOUND as an alias for NEEDCOMPOUND.
Accept FORBIDDENWORD as an alias for BAD.
相关文件: runtime/doc/spell.txt, src/spell.c

Patch 7.2.061
问题: Can't create a funcref for an autoload function without loading
the script first. (Marc Weber)
解决方案: Accept autoload functions that don't exist yet in function().
相关文件: src/eval.c

Patch 7.2.062
问题: "[Scratch]" is not translated.
解决方案: Mark the string for translation. (Dominique Pelle)
相关文件: src/buffer.c

Patch 7.2.063
问题: Warning for NULL argument of Perl_sys_init3().
解决方案: Use Perl_sys_init() instead. (partly by Dominique Pelle)
相关文件: src/if_perl.xs

Patch 7.2.064
问题: Screen update bug when repeating "~" on a Visual block and the
last line doesn't change.
解决方案: Keep track of changes for all lines. (Moritz Orbach)
相关文件: src/ops.c

Patch 7.2.065
问题: GTK GUI: the cursor disappears when doing ":vsp" and the Vim
window is maximized. (Dominique Pelle, Denis Smolyar)
解决方案: Don't change "Columns" back to an old value at a wrong moment.
Do change "Rows" when it should not be a problem.
相关文件: src/gui.c

Patch 7.2.066
问题: It's not easy to see whether 'encoding' is a multi-byte encoding.
解决方案: Add has('multi_byte_encoding').
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.2.067

问题:　　Session file can't load extra file when the path contains special characters.
解决方案:　Escape the file name. (Lech Lorens)
相关文件:　src/ex_docmd.c

Patch 7.2.068
问题:　　Emacs tags file lines can be too long, resulting in an error message. (James Vega)
解决方案:　Ignore lines with errors if they are too long.
相关文件:　src/tag.c

Patch 7.2.069 (after 7.2.060)
问题:　　Compiler warning for storing size_t in int.
解决方案:　Add type cast.
相关文件:　src/spell.c

Patch 7.2.070
问题:　　Crash when a function returns a:000. (Matt Wozniski)
解决方案:　Don't put the function struct on the stack, allocate it. Free it only when nothing in it is used.
相关文件:　src/eval.c

Patch 7.2.071 (extra)
问题:　　Win32: Handling netbeans events while Vim is busy updating the screen may cause a crash.
解决方案:　Like with GTK, only handle netbeans messages in the main loop. (Xavier de Gaye)
相关文件:　src/gui_w48.c, src/netbeans.c

Patch 7.2.072 (extra)
问题:　　Compiler warning in Sniff code.
解决方案:　Use return value of pipe(). (Dominique Pelle)
相关文件:　src/if_sniff.c

Patch 7.2.073
问题:　　":set <xHome>" has the same output as ":set <Home>". (Matt Wozniski)
解决方案:　Don't translate "x" keys to its alternative for ":set".
相关文件:　src/gui_mac.c, src/misc2.c, src/option.c, src/proto/misc2.pro

Patch 7.2.074 (extra, after 7.2.073)
问题:　　":set <xHome>" has the same output as ":set <Home>". (Matt Wozniski)
解决方案:　Don't translate "x" keys to its alternative for ":set".
相关文件:　src/gui_mac.c

Patch 7.2.075 (after 7.2.058)
问题:　　Explanation about making a diff for extra_patches is unclear.
解决方案:　Adjust comment.
相关文件:　src/version.c

Patch 7.2.076
问题:　　rename(from, to) deletes the file if "from" and "to" are not equal but still refer to the same file. E.g., on a FAT32 filesystem

under Unix.
解决方案: Go through another file name.
相关文件: src/fileio.c

Patch 7.2.077 (after 7.2.076)
问题: rename(from, to) doesn't work if "from" and "to" differ only in case on a system that ignores case in file names.
解决方案: Go through another file name.
相关文件: src/fileio.c

Patch 7.2.078
问题: When deleting a fold that is specified with markers the cursor position may be wrong. Folds may not be displayed properly after a delete. Wrong fold may be deleted.
解决方案: Fix the problems. (mostly by Lech Lorens)
相关文件: src/fold.c

Patch 7.2.079
问题: "killed" netbeans events are not handled correctly.
解决方案: A "killed" netbeans event is sent when the buffer is deleted or wiped out (in this case, the netbeans annotations in this buffer have been removed). A user can still remove a sign with the command ":sign unplace" and this does not trigger a "killed" event. (Xavier de Gaye)
相关文件: runtime/doc/netbeans.txt, src/buffer.c, src/globals.h, src/netbeans.c, src/proto/netbeans.pro

Patch 7.2.080
问题: When typing a composing character just after starting completion may access memory before its allocation point. (Dominique Pelle)
解决方案: Don't delete before the completion start column. Add extra checks for the offset not being negative.
相关文件: src/edit.c

Patch 7.2.081
问题: Compiler warning for floating point overflow on VAX.
解决方案: For VAX use a smaller number. (Zoltan Arpadffy)
相关文件: src/message.c

Patch 7.2.082
问题: When 'ff' is "mac" then "ga" on a ^J shows 0x0d instead of 0x0a. (Andy Wokula)
解决方案: Use NL for this situation. (Lech Lorens)
相关文件: src/ex_cmds.c

Patch 7.2.083
问题: ":tag" does not return to the right tag entry from the tag stack.
解决方案: Don't change the current match when there is no argument. (Erik Falor)
相关文件: src/tag.c

Patch 7.2.084
问题: Recursive structures are not handled properly in Python vim.eval().

解决方案: Keep track of references in a better way. (Yukihiro Nakadaira)
相关文件: src/if_python.c

Patch 7.2.085
问题: ":set <M-b>=<Esc>b" does not work when 'encoding' is utf-8.
解决方案: Put the <M-b> character in the input buffer as valid utf-8.
(partly by Matt Wozniski)
相关文件: src/term.c

Patch 7.2.086
问题: Using ":diffget 1" in buffer 1 corrupts the text.
解决方案: Don't do anything when source and destination of ":diffget" or
":diffput" is the same buffer. (Dominique Pelle)
相关文件: src/diff.c

Patch 7.2.087
问题: Adding URL to 'path' doesn't work to edit a file.
解决方案: Skip simplify_filename() for URLs. (Matt Wozniski)
相关文件: src/misc2.c

Patch 7.2.088 (extra)
问题: OpenClipboard() may fail when another application is using the
clipboard.
解决方案: Retry OpenClipboard() a few times. (Jianrong Yu)
相关文件: src/os_mswin.c

Patch 7.2.089 (extra)
问题: Win32: crash when using Ultramon buttons.
解决方案: Don't use a WM_OLE message of zero size. (Ray Megal)
相关文件: src/if_ole.cpp, src/gui_w48.c

Patch 7.2.090
问题: User command containing 0x80 in multi-byte character does not work
properly. (Yasuhiro Matsumoto)
解决方案: Undo replacement of K_SPECIAL and CSI characters when executing
the command.
相关文件: src/ex_docmd.c

Patch 7.2.091
问题: ":cs help" output is not aligned for some languages.
解决方案: Compute character size instead of byte size. (Dominique Pelle)
相关文件: src/if_cscope.c

Patch 7.2.092
问题: Some error messages are not translated.
解决方案: Add _() around the messages. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.2.093 (extra)
问题: Win32: inputdialog() and find/replace dialogs can't handle
multi-byte text.
解决方案: Use the wide version of dialog functions when available. (Yanwei
Jia)
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 7.2.094
问题:　　Compiler warning for signed/unsigned compare.
解决方案:　Add type cast. Also fix a few typos.
相关文件:　src/edit.c

Patch 7.2.095
问题:　　With Visual selection, "r" and then CTRL-C Visual mode is stopped
　　but the highlighting is not removed.
解决方案:　Call reset_VIsual().
相关文件:　src/normal.c

Patch 7.2.096
问题:　　After ":number" the "Press Enter" message may be on the wrong
　　screen, if switching screens for shell commands.
解决方案:　Reset info_message. (James Vega)
相关文件:　src/ex_cmds.c

Patch 7.2.097
问题:　　"!xterm&" doesn't work when 'shell' is "bash".
解决方案:　Ignore SIGHUP after calling setsid(). (Simon Schubert)
相关文件:　src/os_unix.c

Patch 7.2.098
问题:　　Warning for signed/unsigned pointer.
解决方案:　Add type cast.
相关文件:　src/eval.c

Patch 7.2.099
问题:　　Changing GUI options causes an unnecessary redraw when the GUI
　　isn't active.
解决方案:　Avoid the redraw. (Lech Lorens)
相关文件:　src/option.c

Patch 7.2.100
问题:　　When using ":source" on a FIFO or something else that can't rewind
　　the first three bytes are skipped.
解决方案:　Instead of rewinding read the first line and detect a BOM in that.
　　(mostly by James Vega)
相关文件:　src/ex_cmds2.c

Patch 7.2.101 (extra)
问题:　　MSVC version not recognized.
解决方案:　Add the version number to the list. (Zhong Zhang)
相关文件:　src/Make_mvc.mak

Patch 7.2.102 (after 7.2.100)
问题:　　When 'encoding' is "utf-8" a BOM at the start of a Vim script is
　　not removed. (Tony Mechelynck)
解决方案:　When no conversion is taking place make a copy of the line without
　　the BOM.
相关文件:　src/ex_cmds2.c

Patch 7.2.103

问题: When 'bomb' is changed the window title is updated to show/hide a "+", but the tab page label isn't. (Patrick Texier)
解决方案: Set "redraw_tabline" in most places where "need_maketitle" is set. (partly by Lech Lorens)
相关文件: src/option.c

Patch 7.2.104
问题: When using ":saveas bar.c" the tab label isn't updated right away.
解决方案: Set redraw_tabline. (Francois Ingelrest)
相关文件: src/ex_cmds.c

Patch 7.2.105
问题: Modeline setting for 'foldmethod' overrules diff options. (Ingo Karkat)
解决方案: Don't set 'foldmethod' and 'wrap' from a modeline when 'diff' is on.
相关文件: src/option.c

Patch 7.2.106
问题: Endless loop when using "]s" in HTML when there are no misspellings. (Ingo Karkat)
解决方案: Break the search loop. Also fix pointer alignment for systems with pointers larger than int.
相关文件: src/spell.c

Patch 7.2.107
问题: When using a GUI dialog and ":echo" commands the messages are deleted after the dialog. (Vincent Birebent)
解决方案: Don't call msg_end_prompt() since there was no prompt.
相关文件: src/message.c

Patch 7.2.108 (after 7.2.105)
问题: Can't build without the diff feature.
解决方案: Add #ifdef.
相关文件: src/option.c

Patch 7.2.109
问题: 'langmap' does not work for multi-byte characters.
解决方案: Add a list of mapped multi-byte characters. (based on work by Konstantin Korikov, Agathoklis Hatzimanikas)
相关文件: runtime/doc/options.txt, src/edit.c, src/getchar.c, src/macros.h, src/normal.c, src/option.c, src/proto/option.pro, src/window.c

Patch 7.2.110
问题: Compiler warning for unused variable.
解决方案: Init the variable.
相关文件: src/ex_docmd.c

Patch 7.2.111
问题: When using Visual block mode with 'cursorcolumn' it's unclear what is selected.
解决方案: Don't use 'cursorcolumn' highlighting inside the Visual selection. (idea by Dominique Pelle)
相关文件: src/screen.c

Patch 7.2.112
问题: Cursor invisible in Visual mode when 'number' is set and cursor in first column. (Matti Niemenmaa, Renato Alves)
解决方案: Check that vcol_prev is smaller than vcol.
相关文件: src/screen.c

Patch 7.2.113
问题: Crash for substitute() call using submatch(1) while there is no such submatch. (Yukihiro Nakadaira)
解决方案: Also check the start of the submatch is set, it can be NULL when an attempted match didn't work out.
相关文件: src/regexp.c

Patch 7.2.114
问题: Using wrong printf format.
解决方案: Use "%ld" instead of "%d". (Dominique Pelle)
相关文件: src/netbeans.c

Patch 7.2.115
问题: Some debugging code is never used.
解决方案: Remove nbtrace() and nbprt(). (Dominique Pelle)
相关文件: src/nbdebug.c, src/nbdebug.h

Patch 7.2.116
问题: Not all memory is freed when EXITFREE is defined.
解决方案: Free allocated memory on exit. (Dominique Pelle)
相关文件: src/ex_docmd.c, src/gui_gtk_x11.c, src/misc2.c, src/search.c, src/tag.c

Patch 7.2.117
问题: Location list incorrectly labelled "Quickfix List".
解决方案: Break out of both loops for finding window for location list buffer. (Lech Lorens)
相关文件: src/buffer.c, src/quickfix.c, src/screen.c

Patch 7.2.118
问题: <PageUp> at the more prompt only does half a page.
解决方案: Make <PageUp> go up a whole page. Also make 'f' go a page forward, but not quit the more prompt. (Markus Heidelberg)
相关文件: src/message.c

Patch 7.2.119
问题: Status line is redrawn too often.
解决方案: Check ScreeenLinesUC[] properly. (Yukihiro Nakadaira)
相关文件: src/screen.c

Patch 7.2.120
问题: When opening the quickfix window or splitting the window and setting the location list, the location list is copied and then deleted, which is inefficient.
解决方案: Don't copy the location list when not needed. (Lech Lorens)
相关文件: src/quickfix.c, src/vim.h, src/window.c

Patch 7.2.121
问题: In gvim "!grep a *.c" spews out a lot of text that can't be stopped with CTRL-C.
解决方案: When looping to read and show text, do check for typed characters every two seconds.
相关文件: src/os_unix.c

Patch 7.2.122
问题: Invalid memory access when the VimResized autocommand changes 'columns' and/or 'lines'.
解决方案: After VimResized check for changed values. (Dominique Pelle)
相关文件: src/screen.c

Patch 7.2.123
问题: Typing 'q' at more prompt for ":map" output still displays another line, causing another more prompt. (Markus Heidelberg)
解决方案: Quit listing maps when 'q' typed.
相关文件: src/getchar.c

Patch 7.2.124
问题: Typing 'q' at more prompt for ":tselect" output still displays more lines, causing another more prompt. (Markus Heidelberg)
解决方案: Quit listing tags when 'q' typed.
相关文件: src/tag.c

Patch 7.2.125
问题: Leaking memory when reading XPM bitmap for a sign.
解决方案: Don't allocate the memory twice. (Dominique Pelle)
相关文件: src/gui_x11.c

Patch 7.2.126
问题: When EXITFREE is defined signs are not freed.
解决方案: Free all signs on exit. Also free keymaps. (Dominique Pelle)
相关文件: src/misc2.c, src/ex_cmds.c, src/proto/ex_cmds.pro

Patch 7.2.127
问题: When listing mappings and a wrapping line causes the more prompt, after typing 'q' there can be another more prompt. (Markus Heidelberg)
解决方案: Set "lines_left" to allow more lines to be displayed.
相关文件: src/message.c

Patch 7.2.128 (after 7.2.055)
问题: Using ":lcd" makes session files not work.
解决方案: Compare return value of mch_chdir() properly. (Andreas Bernauer)
相关文件: src/ex_docmd.c

Patch 7.2.129
问题: When opening a command window from input() it uses the search history.
解决方案: Use get_cmdline_type(). (James Vega)
相关文件: src/ex_getln.c

Patch 7.2.130

问题:　　Vim may hang until CTRL-C is typed when using CTRL-Z.
解决方案:　Avoid using pause(). Also use "volatile" for variables used in signal functions. (Dominique Pelle)
相关文件:　src/auto/configure, src/configure.in, src/config.h.in, src/globals.h, src/os_unix.c

Patch 7.2.131
问题:　　When 'keymap' is cleared may still use the cursor highlighting for when it's enabled.
解决方案:　Reset 'iminsert' and 'imsearch'. (partly by Dominique Pelle) Also avoid ":setlocal" for these options have a global effect.
相关文件:　src/option.c

Patch 7.2.132
问题:　　When changing directory during a SwapExists autocmd freed memory may be accessed. (Dominique Pelle)
解决方案:　Add the allbuf_lock flag.
相关文件:　src/ex_getln.c, src/globals.h, src/fileio.c, src/proto/ex_getln.pro

Patch 7.2.133
问题:　　":diffoff!" changes settings in windows not in diff mode.
解决方案:　Only change settings in other windows when 'diff' is set, always do it for the current window. (Lech Lorens)
相关文件:　src/diff.c

Patch 7.2.134
问题:　　Warning for discarding "const" from pointer.
解决方案:　Don't pass const pointer to mch_memmove().
相关文件:　src/fileio.c

Patch 7.2.135
问题:　　Memory leak when redefining user command with complete argument.
解决方案:　Free the old complete argument. (Dominique Pelle)
相关文件:　src/ex_docmd.c

Patch 7.2.136 (after 7.2.132)
问题:　　":cd" is still possible in a SwapExists autocmd.
解决方案:　Check the allbuf_lock flag in ex_cd().
相关文件:　src/ex_docmd.c

Patch 7.2.137
问题:　　When 'virtualedit' is set, a left shift of a blockwise selection that starts and ends inside a tab shifts too much. (Helmut Stiegler)
解决方案:　Redo the block left shift code. (Lech Lorens)
相关文件:　src/ops.c, src/testdir/Makefile, src/testdir/test66.in, src/testdir/test66.ok

Patch 7.2.138 (extra part of 7.2.137)
问题:　　See 7.2.137.
解决方案:　See 7.2.137.
相关文件:　src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,

src/testdir/Make_vms.mms

Patch 7.2.139
问题: Crash when 'virtualedit' is "all". (James Vega)
解决方案: Avoid overflow when column is MAXCOL. (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.2.140
问题: Diff highlighting isn't displayed before the Visual area if it starts at the cursor position. (Markus Heidelberg)
解决方案: Also check fromcol_prev.
相关文件: src/screen.c

Patch 7.2.141
问题: When redrawing a character for bold spill this causes the next character to be redrawn as well.
解决方案: Only redraw one extra character. (Yukihiro Nakadaira)
相关文件: src/screen.c

Patch 7.2.142
问题: Motif and Athena balloons don't use tooltip colors.
解决方案: Set the colors. (Matt Wozniski)
相关文件: src/gui_beval.c

Patch 7.2.143
问题: No command line completion for ":cscope" command.
解决方案: Add the completion for ":cscope". (Dominique Pelle)
相关文件: src/ex_docmd.c, src/ex_getln.c, src/if_cscope.c, src/proto/if_cscope.pro, src/vim.h

Patch 7.2.144
问题: When 't_Co' is set to the value it already had the color scheme is reloaded anyway.
解决方案: Only load the colorscheme when the t_Co value changes. (Dominique Pelle)
相关文件: src/option.c

Patch 7.2.145
问题: White space in ":cscope find" is not ignored.
解决方案: Ignore the white space, but not when the leading white space is useful for the argument.
相关文件: runtime/doc/if_scop.txt, src/if_cscope.c

Patch 7.2.146
问题: v:warningmsg isn't used for all warnings.
解决方案: Set v:warningmsg for relevant warnings. (Ingo Karkat)
相关文件: src/fileio.c, src/misc1.c, src/option.c

Patch 7.2.147
问题: When compiled as small version and 'number' is on the cursor is displayed in the wrong position after a tab. (James Vega)
解决方案: Don't increment vcol when still displaying the line number.
相关文件: src/screen.c

Patch 7.2.148
问题: When searching for "$" while 'hlsearch' is set, highlighting the character after the line does not work in the cursor column. Also highlighting for Visual mode after the line end when this isn't needed. (Markus Heidelberg)
解决方案: Only compare the cursor column in the cursor line. Only highlight for Visual selection after the last character when it's needed to see where the Visual selection ends.
相关文件: src/screen.c

Patch 7.2.149
问题: Using return value of function that doesn't return a value results in reading uninitialized memory.
解决方案: Set the default to return zero. Make cursor() return -1 on failure. Let complete() return an empty string in case of an error. (partly by Dominique Pelle)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.2.150 (extra)
问题: Can't use tab pages from VisVim.
解决方案: Add tab page support to VisVim. (Adam Slater)
相关文件: src/VisVim/Commands.cpp, src/VisVim/Resource.h, src/VisVim/VisVim.rc

Patch 7.2.151
问题: ":hist a" doesn't work like ":hist all" as the docs suggest.
解决方案: Make ":hist a" and ":hist al" work. (Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.2.152
问题: When using "silent echo x" inside ":redir" a next echo may start halfway the line. (Tony Mechelynck, Dennis Benzinger)
解决方案: Reset msg_col after redirecting silently.
相关文件: src/ex_docmd.c, src/message.c, src/proto/message.pro

Patch 7.2.153
问题: Memory leak for ":recover empty_dir/".
解决方案: Free files[] when it becomes empty. (Dominique Pelle)
相关文件: src/memline.c

Patch 7.2.154 (after 7.2.132)
问题: ":cd" is still possible in a SwapExists autocmd.
解决方案: Set allbuf_lock in do_swapexists().
相关文件: src/memline.c

Patch 7.2.155
问题: Memory leak in ":function /pat".
解决方案: Free the memory. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.2.156 (after 7.2.143)
问题: No completion for :scscope and :lcscope commands.
解决方案: Implement the completion. (Dominique Pelle)
相关文件: src/if_cscope.c, src/ex_docmd.c, src/proto/if_cscope.pro

Patch 7.2.157
问题:　　Illegal memory access when searching in path.
解决方案:　Avoid looking at a byte after end of a string. (Dominique Pelle)
相关文件:　src/search.c

Patch 7.2.158
问题:　　Warnings from VisualC compiler.
解决方案:　Add type casts. (George Reilly)
相关文件:　src/ops.c

Patch 7.2.159
问题:　　When $x_includes ends up being "NONE" configure fails.
解决方案:　Check for $x_includes not to be "NONE" (Rainer)
相关文件:　src/auto/configure, src/configure.in

Patch 7.2.160
问题:　　Search pattern not freed on exit when 'rightleft' set.
解决方案:　Free mr_pattern_alloced.
相关文件:　src/search.c

Patch 7.2.161
问题:　　Folds messed up in other tab page. (Vlad Irnov)
解决方案:　Instead of going over all windows in current tab page go over all
windows in all tab pages. Also free memory for location lists in
other tab pages when exiting. (Lech Lorens)
相关文件:　src/fileio.c, src/mark.c, src/misc1.c, src/misc2.c

Patch 7.2.162
问题:　　The quickfix window may get wrong filetype.
解决方案:　Do not detect the filetype for the quickfix window. (Lech Lorens)
相关文件:　src/quickfix.c

Patch 7.2.163
问题:　　The command line window may get folding.
解决方案:　Default to no/manual folding. (Lech Lorens)
相关文件:　src/ex_getln.c

Patch 7.2.164
问题:　　When 'showbreak' is set the size of the Visual block may be
reported wrong. (Eduardo Daudt Flach)
解决方案:　Temporarily make 'sbr' empty.
相关文件:　src/normal.c, src/ops.c

Patch 7.2.165
问题:　　The argument for the FuncUndefined autocmd event is expanded like
a file name.
解决方案:　Don't try expanding it. (Wang Xu)
相关文件:　src/fileio.c

Patch 7.2.166
问题:　　No completion for ":sign" command.
解决方案:　Add ":sign" completion. (Dominique Pelle)
相关文件:　src/ex_cmds.c, src/ex_docmd.c, src/ex_getln.c, src/vim.h,

src/proto/ex_cmds.pro

Patch 7.2.167
问题: Splint doesn't work well for checking the code.
解决方案: Add splint arguments in the Makefile. Exclude some code from splint that it can't handle. Tune splint arguments to give reasonable errors. Add a filter for removing false warnings from splint output. Many small changes to avoid warnings. More to follow...
相关文件: Filelist, src/Makefile, src/buffer.c, src/charset.c, src/cleanlint.vim, src/digraph.c, src/edit.c, src/ex_cmds.c, src/globals.h, src/ops.c, src/os_unix.c, src/os_unix.h, src/proto/buffer.pro, src/proto/edit.pro, src/screen.c, src/structs.h

Patch 7.2.168
问题: When no ctags program can be found, "make tags" attempts to execute the first C file.
解决方案: Default to "ctags" when no ctags program can be found.
相关文件: src/configure.in, src/auto/configure

Patch 7.2.169
问题: Splint complains about a lot of things.
解决方案: Add type casts, #ifdefs and other changes to avoid warnings. Change colnr_T from unsigned to int. Avoids mistakes with subtracting columns.
相关文件: src/cleanlint.vim, src/diff.c, src/edit.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/proto/ex_cmds.pro, src/proto/spell.pro, src/quickfix.c, src/spell.c, src/structs.h, src/term.h, src/vim.h

Patch 7.2.170
问题: Using b_dev while it was not set. (Dominique Pelle)
解决方案: Add the b_dev_valid flag.
相关文件: src/buffer.c, src/fileio.c, src/structs.h

Patch 7.2.171 (after 7.2.169)
问题: Compiler warnings. (Tony Mechelynck)
解决方案: Add function prototype. (Patrick Texier) Init variable.
相关文件: src/ex_cmds.c

Patch 7.2.172 (extra)
问题: Compiler warning.
解决方案: Adjust function prototype. (Patrick Texier)
相关文件: src/os_mswin.c

Patch 7.2.173
问题: Without lint there is no check for unused function arguments.
解决方案: Use gcc -Wunused-parameter instead of lint. For a few files add attributes to arguments that are known not to be used.
相关文件: src/auto/configure, src/buffer.c, src/charset.c, src/diff.c, src/configure.in, src/config.h.in, src/edit.c, src/ex_cmds.c, src/ex_cmds2.c, src/version.c, src/vim.h

Patch 7.2.174
问题: Too many warnings from gcc -Wextra.
解决方案: Change initializer. Add UNUSED. Add type casts.
相关文件: src/edit.c, src/eval.c, src/ex_cmds.c, src/ex_docmd.c,
            src/ex_getln.c, src/fileio.c, getchar.c, globals.h, main.c,
            memline.c, message.c, src/misc1.c, src/move.c, src/normal.c,
            src/option.c, src/os_unix.c, src/os_unix.h, src/regexp.c,
            src/search.c, src/tag.c

Patch 7.2.175
问题: Compiler warning in OpenBSD.
解决方案: Add type cast for NULL. (Dasn)
相关文件: src/if_cscope.c

Patch 7.2.176
问题: Exceptions for splint are not useful.
解决方案: Remove the S_SPLINT_S ifdefs.
相关文件: src/edit.c, src/ex_cmds.c, src/ex_docmd.c, src/os_unix.c,
            src/os_unix.h, src/os_unixx.h, src/structs.h, src/term.h

Patch 7.2.177
问题: Compiler warnings when using -Wextra
解决方案: Add UNUSED and type casts.
相关文件: src/eval.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c,
            src/fileio.c, src/hardcopy.c, src/if_cscope.c, src/if_xcmdsrv.c,
            src/farsi.c, src/mark.c, src/menu.c

Patch 7.2.178
问题: Using negative value for device number might not work.
解决方案: Use a separate flag for whether ffv_dev was set.
相关文件: src/misc2.c

Patch 7.2.179
问题: Using negative value for device number might not work.
解决方案: Use a separate flag for whether sn_dev was set.
相关文件: src/ex_cmds2.c

Patch 7.2.180
问题: Some more compiler warnings when using gcc -Wextra.
解决方案: Add UNUSED and type casts.
相关文件: src/buffer.c, src/ex_cmds.c, src/macros.h, src/main.c,
            src/menu.c, src/message.c, src/misc1.c, src/mbyte.c,
            src/normal.c, src/option.c, src/os_unix.c, src/quickfix.c,
            src/screen.c, src/search.c, src/spell.c, src/syntax.c, src/tag.c,
            src/term.c, src/ui.c

Patch 7.2.181
问题: Some more compiler warnings when using gcc -Wextra.
解决方案: Add UNUSED and type casts.
相关文件: src/if_mzsch.c, src/gui.c, src/gui_gtk.c, src/gui_gtk_x11.c,
            src/gui_gtk_f.c, src/gui_beval.c, src/netbeans.c

Patch 7.2.182 (after 7.2.181)
问题: Compilation problems after previous patch for Motif. Gvim with

GTK crashes on startup.
解决方案: Add comma. Init form structure to zeroes.
相关文件: src/netbeans.c, src/gui_gtk_f.c

Patch 7.2.183
问题: Configure problem for sys/sysctl.h on OpenBSD. (Dasn)
解决方案: Add separate check for this header file. Also switch to newer version of autoconf.
相关文件: src/auto/configure, src/configure.in

Patch 7.2.184
问题: Some more compiler warnings when using gcc -Wextra.
解决方案: Add UNUSED and type casts. Autoconf check for wchar_t.
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/gui_athena.c, src/gui_x11.c, src/gui.c, src/gui_beval.c, src/gui_at_sb.c, src/gui_at_fs.c, src/gui_motif.c, src/gui_xmdlg.c, src/gui_xmebw.c, src/if_python.c, src/window.c, src/workshop.c

Patch 7.2.185
问题: Some more compiler warnings when using gcc -Wextra.
解决方案: Add UNUSED and type casts.
相关文件: src/Makefile, src/if_tlc.c, src/if_ruby.c

Patch 7.2.186
问题: Some more compiler warnings when using gcc -Wextra.
解决方案: Now with the intended if_tcl.c changes.
相关文件: src/if_tcl.c

Patch 7.2.187 (after 7.2.186)
问题: Doesn't build with older versions of TCL. (Yongwei Wu)
解决方案: Add #ifdefs. (Dominique Pelle)
相关文件: src/if_tcl.c

Patch 7.2.188
问题: Crash with specific use of function calls. (Meikel Brandmeyer)
解决方案: Make sure the items referenced by a function call are not freed twice. (based on patch from Nico Weber)
相关文件: src/eval.c

Patch 7.2.189
问题: Possible hang for deleting auto-indent. (Dominique Pelle)
解决方案: Make sure the position is not beyond the end of the line.
相关文件: src/edit.c

Patch 7.2.190
问题: The register executed by @@ isn't restored.
解决方案: Mark the executable register in the viminfo file.
相关文件: src/ops.c

Patch 7.2.191
问题: Mzscheme interface doesn't work on Ubuntu.
解决方案: Change autoconf rules. Define missing macro. Some changes to avoid gcc warnings. Remove per-buffer namespace. (Sergey Khorev)

相关文件: runtime/doc/if_mzsch.txt, src/Makefile, src/Make_ming.mak,
           src/Make_mvc.mak, src/auto/configure, src/configure.in,
           src/config.mk.in, src/eval.c, src/if_mzsch.c, src/if_mzsch.h,
           src/main.c, src/proto/if_mzsch.pro

Patch 7.2.192 (after 7.2.188)
问题: Still a crash in the garbage collector for a very rare situation.
解决方案: Make sure current_copyID is always incremented correctly. (Kent
           Sibilev)
相关文件: src/eval.c

Patch 7.2.193
问题: Warning for uninitialized values.
解决方案: Initialize all the struct items.
相关文件: src/eval.c

Patch 7.2.194 (extra)
问题: MSVC: rem commands are echoed.
解决方案: Add commands to switch off echo. (Wang Xu)
相关文件: src/msvc2008.bat

Patch 7.2.195
问题: Leaking memory for the command Vim was started with.
解决方案: Remember the pointer and free it.
相关文件: src/gui_gtk_x11.c

Patch 7.2.196 (after 7.2.167)
问题: Turns out splint doesn't work well enough to be usable.
解决方案: Remove splint support.
相关文件: Filelist, src/cleanlint.vim

Patch 7.2.197
问题: Warning for uninitialized values.
解决方案: Initialize all the struct items of typebuf.
相关文件: src/globals.h

Patch 7.2.198
问题: Size of buffer used for tgetent() may be too small.
解决方案: Use the largest known size everywhere.
相关文件: src/vim.h

Patch 7.2.199
问题: Strange character in comment.
解决方案: Change to "message". (Yongwei Wu)
相关文件: src/term.c

Patch 7.2.200
问题: Reading past end of string when navigating the menu bar or
           resizing the window.
解决方案: Add and use mb_ptr2len_len(). (partly by Dominique Pelle)
           Also add mb_ptr2cells_len() to prevent more trouble.
相关文件: src/gui_gtk_x11.c, src/os_unix.c, src/globals.h, src/mbyte.c,
           src/proto/mbyte.pro

Patch 7.2.201
问题: Cannot copy/paste HTML to/from Firefox via the clipboard.
解决方案: Implement this for GTK. Add the "html" value to 'clipboard'.
相关文件: runtime/doc/options.txt, src/globals.h, src/gui_gtk_x11.c, src/mbyte.c, src/proto/mbyte.pro, src/option.c

Patch 7.2.202
问题: BufWipeout autocommand that edits another buffer causes problems.
解决方案: Check for the situation, give an error and quit the operation.
相关文件: src/fileio.c

Patch 7.2.203
问题: When reloading a buffer or doing anything else with a buffer that is not displayed in a visible window, autocommands may be applied to the current window, folds messed up, etc.
解决方案: Instead of using the current window for the hidden buffer use a special window, splitting the current one temporarily.
相关文件: src/fileio.c, src/globals.h, src/gui.c, src/if_perl.xs, src/progo/gui.pro, src/proto/window.pro, src/screen.c, src/structs.h, src/window.c

Patch 7.2.204 (extra)
问题: Win32: Can't build with Visual Studio 2010 beta 1.
解决方案: Fix the makefile. (George Reilly)
相关文件: src/Make_mvc.mak

Patch 7.2.205 (extra)
问题: Win32: No support for High DPI awareness.
解决方案: Fix the manifest file. (George Reilly)
相关文件: src/Make_mvc.mak, src/gvim.exe.mnf

Patch 7.2.206
问题: Win32: Can't build netbeans interface with Visual Studio 2010.
解决方案: Undefine ECONNREFUSED. (George Reilly)
相关文件: src/netbeans.c

Patch 7.2.207
问题: Using freed memory with ":redrawstatus" when it works recursively.
解决方案: Prevent recursively updating the status line. (partly by Dominique Pelle)
相关文件: src/screen.c

Patch 7.2.208
问题: "set novice" gives an error message, it should be ignored.
解决方案: Don't see "no" in "novice" as unsetting an option. (Patrick Texier)
相关文件: src/option.c

Patch 7.2.209
问题: For xxd setmode() is undefined on Cygwin.
解决方案: Include io.h. (Dominique Pelle)
相关文件: src/xxd/xxd.c

Patch 7.2.210

问题:　　When a file that is being edited has its timestamp updated outside of Vim and ":checktime" is used still get a warning when writing the file. (Matt Mueller)
解决方案:　Store the timestamp in b_mtime_read when the timestamp is the only thing that changed.
相关文件:　src/fileio.c

Patch 7.2.211
问题:　　Memory leak when expanding a series of file names.
解决方案:　Use ga_clear_strings() instead of ga_clear().
相关文件:　src/misc1.c

Patch 7.2.212 (extra)
问题:　　Warnings for redefining SIG macros.
解决方案:　Don't define them if already defined. (Bjorn Winckler)
相关文件:　src/os_mac.h

Patch 7.2.213
问题:　　Warning for using vsprintf().
解决方案:　Use vim_vsnprintf().
相关文件:　src/netbeans.c

Patch 7.2.214
问题:　　Crash with complete function for user command. (Andy Wokula)
解决方案:　Avoid using a NULL pointer (Dominique Pelle)
相关文件:　src/ex_getln.c

Patch 7.2.215
问题:　　ml_get error when using ":vimgrep".
解决方案:　Load the memfile for the hidden buffer before putting it in a window. Correct the order of splitting the window and filling the window and buffer with data.
相关文件:　src/fileio.c, src/proto/window.pro, src/quickfix.c, src/window.c

Patch 7.2.216
问题:　　Two error messages have the same number E812.
解决方案:　Give one message a different number.
相关文件:　runtime/doc/autocmd.txt, runtime/doc/if_mzsch.txt, src/if_mzsch.c

Patch 7.2.217
问题:　　Running tests with valgrind doesn't work as advertised.
解决方案:　Fix the line in the Makefile.
相关文件:　src/testdir/Makefile

Patch 7.2.218
问题:　　Cannot build GTK with hangul_input feature. (Dominique Pelle)
解决方案:　Adjust #ifdef. (SungHyun Nam)
相关文件:　src/gui.c

Patch 7.2.219 (extra)
问题:　　Photon GUI is outdated.
解决方案:　Updates for QNX 6.4.0. (Sean Boudreau)
相关文件:　src/gui_photon.c

Patch 7.2.220 (after 7.2.215)
问题: a BufEnter autocommand that changes directory causes problems.
(Ajit Thakkar)
解决方案: Disable autocommands when opening a hidden buffer in a window.
相关文件: src/fileio.c

Patch 7.2.221
问题: X cut_buffer0 text is used as-is, it may be in the wrong encoding.
解决方案: Convert between 'enc' and latin1. (James Vega)
相关文件: src/gui_gtk_x11.c, src/message.c, src/ops.c, src/proto/ui.pro,
src/ui.c

Patch 7.2.222
问题: ":mksession" doesn't work properly with 'acd' set.
解决方案: Make it work. (Yakov Lerner)
相关文件: src/ex_docmd.c

Patch 7.2.223
问题: When a script is run with ":silent" it is not able to give warning
messages.
解决方案: Add the ":unsilent" command.
相关文件: runtime/doc/various.txt, src/ex_cmds.h, src/ex_docmd.c

Patch 7.2.224
问题: Crash when using 'completefunc'. (Ingo Karkat)
解决方案: Disallow entering edit() recursively when doing completion.
相关文件: src/edit.c

Patch 7.2.225
问题: When using ":normal" a saved character may be executed.
解决方案: Also store old_char when saving typeahead.
相关文件: src/getchar.c, src/structs.h

Patch 7.2.226
问题: ml_get error after deleting the last line. (Xavier de Gaye)
解决方案: When adjusting marks a callback may be invoked. Adjust the cursor
position before invoking deleted_lines_mark().
相关文件: src/ex_cmds.c, src/ex_docmd.c, src/if_mzsch.c, src/if_python.c,
src/if_perl.xs, src/misc1.c

Patch 7.2.227
问题: When using ":cd" in a script there is no way to track this.
解决方案: Display the directory when 'verbose' is 5 or higher.
相关文件: src/ex_docmd.c

Patch 7.2.228
问题: Cscope is limited to 8 connections.
解决方案: Allocated the connection array to handle any number of
connections. (Dominique Pelle)
相关文件: runtime/doc/if_cscop.txt, src/if_cscope.h, src/if_cscope.c

Patch 7.2.229
问题: Warning for shadowed variable.
解决方案: Rename "wait" to "wait_time".

相关文件: src/os_unix.c

Patch 7.2.230
问题: A few old lint-style ARGUSED comments.
解决方案: Change to the new UNUSED style.
相关文件: src/getchar.c

Patch 7.2.231
问题: Warning for unreachable code.
解决方案: Add #ifdef.
相关文件: src/if_perl.xs

Patch 7.2.232
问题: Cannot debug problems with being in a wrong directory.
解决方案: When 'verbose' is 5 or higher report directory changes.
相关文件: src/os_unix.c, src/os_unix.h, src/proto/os_unix.pro

Patch 7.2.233 (extra part of 7.2.232)
问题: Cannot debug problems with being in a wrong directory.
解决方案: When 'verbose' is 5 or higher report directory changes.
相关文件: src/os_msdos.c, src/os_mswin.c, src/os_riscos.c, src/os_mac.h

Patch 7.2.234
问题: It is not possible to ignore file names without a suffix.
解决方案: Use an empty entry in 'suffixes' for file names without a dot.
相关文件: runtime/doc/cmdline.txt, src/misc1.c

Patch 7.2.235
问题: Using CTRL-O z= in Insert mode has a delay before redrawing.
解决方案: Reset msg_didout and msg_scroll.
相关文件: src/misc1.c, src/spell.c

Patch 7.2.236
问题: Mac: Compiling with Ruby doesn't always work.
解决方案: In configure filter out the --arch argument (Bjorn Winckler)
相关文件: src/configure.in, src/auto/configure

Patch 7.2.237
问题: Crash on exit when window icon not set.
解决方案: Copy terminal name when using it for the icon name.
相关文件: src/os_unix.c

Patch 7.2.238
问题: Leaking memory when setting term to "builtin_dumb".
解决方案: Free memory when resetting term option t_Co.
相关文件: src/option.c, src/proto/option.pro, src/term.c

Patch 7.2.239
问题: Using :diffpatch twice or when patching fails causes memory
corruption and/or a crash. (Bryan Venteicher)
解决方案: Detect missing output file. Avoid using non-existing buffer.
相关文件: src/diff.c

Patch 7.2.240

问题:　　Crash when using find/replace dialog repeatedly. (Michiel Hartsuiker)
解决方案:　Avoid doing the operation while busy or recursively. Also refuse replace when text is locked.
相关文件:　src/gui.c

Patch 7.2.241
问题:　　When using a combination of ":bufdo" and "doautoall" we may end up in the wrong directory. (Ajit Thakkar)
Crash when triggering an autocommand in ":vimgrep". (Yukihiro Nakadaira)
解决方案:　Clear w_localdir and globaldir when using the aucmd_win.
Use a separate flag to decide aucmd_win needs to be restored.
相关文件:　src/fileio.c, src/globals.h, src/structs.h

Patch 7.2.242
问题:　　Setting 'lazyredraw' causes the cursor column to be recomputed. (Tom Link)
解决方案:　Only recompute the cursor column for a boolean option if changes the cursor position.
相关文件:　src/option.c

Patch 7.2.243
问题:　　Memory leak when using :vimgrep and resizing. (Dominique Pelle)
解决方案:　Free memory for aucmd_win when resizing and don't allocate it twice.
相关文件:　src/screen.c

Patch 7.2.244
问题:　　When 'enc' is utf-8 and 'fenc' is latin1, writing a non-latin1 character gives a conversion error without any hint what is wrong.
解决方案:　When known add the line number to the error message.
相关文件:　src/fileio.c

Patch 7.2.245
问题:　　When 'enc' is "utf-16" and 'fenc' is "utf-8" writing a file does conversion while none should be done. (Yukihiro Nakadaira) When 'fenc' is empty the file is written as utf-8 instead of utf-16.
解决方案:　Do proper comparison of encodings, taking into account that all Unicode values for 'enc' use utf-8 internally.
相关文件:　src/fileio.c

Patch 7.2.246
问题:　　Cscope home page link is wrong.
解决方案:　Update the URL. (Sergey Khorev)
相关文件:　runtime/doc/if_cscop.txt

Patch 7.2.247
问题:　　Mzscheme interface minor problem.
解决方案:　Better error message when build fails. (Sergey Khorev)
相关文件:　src/if_mzsch.c

Patch 7.2.248 (extra)
问题:　　Mzscheme interface building minor problems.

解决方案: Update Win32 makefiles. (Sergey Khorev)
相关文件: src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak

Patch 7.2.249
问题: The script to check .po files can't handle '%' in plural forms.
解决方案: Remove "Plural-Forms:" from the checked string.
相关文件: src/po/check.vim

Patch 7.2.250 (extra)
问题: Possible buffer overflow.
解决方案: Compute the remaining space. (Dominique Pelle)
相关文件: src/GvimExt/gvimext.cpp

Patch 7.2.251 (after 7.2.044)
问题: Compiler adds invalid memory bounds check.
解决方案: Remove _FORTIFY_SOURCE=2 from CFLAGS. (Dominique Pelle)
相关文件: src/auto/configure, src/configure.in

Patch 7.2.252
问题: When using a multi-byte 'enc' the 'iskeyword' option cannot
           contain characters above 128.
解决方案: Use mb_ptr2char_adv().
相关文件: src/charset.c

Patch 7.2.253
问题: Netbeans interface: getLength always uses current buffer.
解决方案: Use ml_get_buf() instead of ml_get(). (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.2.254
问题: Compiler warning for assigning size_t to int.
解决方案: Use size_t for the variable. (George Reilly)
相关文件: src/fileio.c

Patch 7.2.255 (after 7.2.242)
问题: Setting 'rightleft', 'linebreak' and 'wrap' may cause cursor to be
           in wrong place.
解决方案: Recompute the cursor column for these options.
相关文件: src/option.c

Patch 7.2.256
问题: When 'guifont' was not set GTK font dialog doesn't have a default.
           (Andreas Metzler)
解决方案: Set default to DEFAULT_FONT. (James Vega)
相关文件: src/gui_gtk_x11.c

Patch 7.2.257
问题: With GTK 2.17 lots of assertion error messages.
解决方案: Remove check for static gravity. (Sebastian Droege)
相关文件: src/gui_gtk_f.c

Patch 7.2.258
问题: v:beval_col and b:beval_text are wrong in UTF-8 text. (Tony
           Mechelynck)

解决方案: Use byte number instead of character number for the column.
相关文件: src/ui.c

Patch 7.2.259
问题: exists() doesn't work properly for an empty aucmd group.
解决方案: Change how au_exists() handles a missing pattern. Also add a test for this. (Bob Hiestand)
相关文件: src/fileio.c, src/testdir/Makefile, src/testdir/test67.in, src/testdir/test67.ok

Patch 7.2.260 (extra part of 7.2.259)
问题: exists() doesn't work properly for empty aucmd group.
解决方案: Change how au_exists() handles a missing pattern. Also add a test for this. (Bob Hiestand)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.2.261
问题: When deleting lines with a specific folding configuration E38 may appear. (Shahaf)
解决方案: When adjusting nested folds for deleted lines take into account that they don't start at the top of the enclosing fold.
相关文件: src/fold.c

Patch 7.2.262
问题: When using custom completion for a user command the pattern string goes beyond the cursor position. (Hari Krishna Dara)
解决方案: Truncate the string at the cursor position.
相关文件: src/ex_getln.c, src/structs.h

Patch 7.2.263
问题: GTK2: when using the -geom argument with an offset from the right edge and the size is smaller than the default, the Vim window is not positioned properly.
解决方案: Use another function to set the size. (Vitaly Minko)
相关文件: src/gui_gtk_x11.c

Patch 7.2.264
问题: GTK2: When the Vim window is maximized setting 'columns' or 'lines' doesn't work.
解决方案: Unmaximize the window before setting the size. (Vitaly Minko)
相关文件: src/gui.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro

Patch 7.2.265
问题: When using ":silent broken" inside try/catch silency may persist. (dr-dr xp)
解决方案: Set msg_silent when there is an error and it's bigger than the saved value.
相关文件: src/ex_docmd.c

Patch 7.2.266
问题: When an expression abbreviation is triggered, the typed character is unknown.

解决方案: Make the typed character available in v:char.
相关文件: runtime/doc/map.txt, src/eval.c, src/getchar.c, src/ops.c,
           src/proto/eval.pro

Patch 7.2.267
问题: Crash for narrow window and double-width character.
解决方案: Check for zero width. (Taro Muraoka)
相关文件: src/charset.c

Patch 7.2.268
问题: Crash when using Python to set cursor beyond end of line.
           (winterTTr)
解决方案: Check the column to be valid.
相关文件: src/if_python.c

Patch 7.2.269
问题: Many people struggle to find out why Vim startup is slow.
解决方案: Add the --startuptime command line flag.
相关文件: runtime/doc/starting.txt, src/globals.h, src/feature.h,
           src/main.c, src/macros.h

Patch 7.2.270
问题: Using ":@c" when the c register contains a CR causes the rest to
           be executed later. (Dexter Douglas)
解决方案: Don't check for typeahead to start with ':', keep executing
           commands until all added typeahead has been used.
相关文件: src/ex_docmd.c

Patch 7.2.271
问题: Using freed memory in Motif GUI version when making a choice.
解决方案: Free memory only after using it. (Dominique Pelle)
相关文件: src/gui_xmdlg.c

Patch 7.2.272
问题: "_.svz" is not recognized as a swap file. (David M. Besonen)
解决方案: Accept .s[uvw][a-z] as a swap file name extension.
相关文件: src/memline.c

Patch 7.2.273
问题: Crash with redir to unknown array. (Christian Brabandt)
解决方案: Don't assign the redir result when there was an error.
相关文件: src/eval.c

Patch 7.2.274
问题: Syntax folding doesn't work properly when adding a comment.
解决方案: Fix it and add a test. (Lech Lorens)
相关文件: src/fold.c, src/testdir/test45.in, src/testdir/test45.ok

Patch 7.2.275
问题: Warning for unused argument and comparing signed and unsigned.
解决方案: Add type cast.
相关文件: src/memline.c

Patch 7.2.276

问题:　　Crash when setting 'isprint' to a small bullet. (Raul Coronado)
解决方案:　Check for the character to be < 256. Also make it possible to specify a range of multi-byte characters. (Lech Lorens)
相关文件:　src/charset.c

Patch 7.2.277
问题:　　CTRL-Y in a diff'ed window may move the cursor outside of the window. (Lech Lorens)
解决方案:　Limit the number of filler lines to the height of the window. Don't reset filler lines to zero for an empty buffer.
相关文件:　src/move.c

Patch 7.2.278
问题:　　Using magic number in the folding code.
解决方案:　Use the defined MAX_LEVEL.
相关文件:　src/fold.c

Patch 7.2.279
问题:　　Invalid memory read with visual mode "r". (Dominique Pelle)
解决方案:　Make sure the cursor position is valid. Don't check the cursor position but the position being used. And make sure we get the right line.
相关文件:　src/misc2.c, src/ops.c

Patch 7.2.280
问题:　　A redraw in a custom statusline with %! may cause a crash. (Yukihiro Nakadaira)
解决方案:　Make a copy of 'statusline'. Also fix typo in function name redraw_custom_statusline. (partly by Dominique Pelle)
相关文件:　src/screen.c

Patch 7.2.281
问题:　　'cursorcolumn' highlighting is wrong in diff mode.
解决方案:　Adjust the column computation. (Lech Lorens)
相关文件:　src/screen.c

Patch 7.2.282
问题:　　A fold can't be closed.
解决方案:　Initialize fd_small to MAYBE. (Lech Lorens)
相关文件:　src/fold.c

Patch 7.2.283
问题:　　Changing font while the window is maximized doesn't keep the window maximized.
解决方案:　Recompute number of lines and columns after changing font. (James Vega)
相关文件:　src/gui_gtk_x11.c

Patch 7.2.284
问题:　　When editing the same buffer in two windows, one with folding, display may be wrong after changes.
解决方案:　Call set_topline() to take care of side effects. (Lech Lorens)
相关文件:　src/misc1.c

Patch 7.2.285 (after 7.2.169)
问题: CTRL-U in Insert mode also deletes indent. (Andrey Voropaev)
解决方案: Fix mistake made in patch 7.2.169.
相关文件: src/edit.c

Patch 7.2.286 (after 7.2.269)
问题: The "--startuptime=<file>" argument is not consistent with other arguments.
解决方案: Use "--startuptime <file>". Added the +startuptime feature.
相关文件: runtime/doc/eval.txt, runtime/doc/starting.txt, runtime/doc/various.txt, src/eval.c, src/main.c, src/version.c

Patch 7.2.287
问题: Warning from gcc 3.4 about uninitialized variable.
解决方案: Move assignment outside of #ifdef.
相关文件: src/if_perl.xs

Patch 7.2.288
问题: Python 2.6 pyconfig.h redefines macros.
解决方案: Undefine the macros before including pyconfig.h.
相关文件: src/if_python.c

Patch 7.2.289
问题: Checking wrong struct member.
解决方案: Change tb_buf to tb_noremap. (Dominique Pelle)
相关文件: src/getchar.c

Patch 7.2.290
问题: Not freeing memory from ":lmap", ":xmap" and ":menutranslate".
解决方案: Free the memory when exiting. (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.2.291
问题: Reading uninitialised memory in arabic mode.
解决方案: Use utfc_ptr2char_len() rather than utfc_ptr2char(). (Dominique Pelle)
相关文件: src/screen.c

Patch 7.2.292
问题: Block right-shift doesn't work properly with multi-byte encoding and 'list' set.
解决方案: Add the missing "else". (Lech Lorens)
相关文件: src/ops.c

Patch 7.2.293
问题: When setting 'comments' option it may be used in a wrong way.
解决方案: Don't increment after skipping over digits. (Yukihiro Nakadaira)
相关文件: src/misc1.c

Patch 7.2.294
问题: When using TEMPDIRS dir name could get too long.
解决方案: Overwrite tail instead of appending each time. Use mkdtemp() when available. (James Vega)
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/fileio.c

Patch 7.2.295
问题: When using map() on a List the index is not known.
解决方案: Set v:key to the index. (Hari Krishna Dara)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.2.296
问题: Help message about startuptime is wrong. (Dominique Pelle)
解决方案: Remove the equal sign.
相关文件: src/main.c

Patch 7.2.297
问题: Reading freed memory when writing ":reg" output to a register.
(Dominique Pelle)
解决方案: Skip the register being written to.
相关文件: src/ops.c

Patch 7.2.298
问题: ":vimgrep" crashes when there is an autocommand that sets a
window-local variable.
解决方案: Initialize the w: hashtab for re-use. (Yukihiro Nakadaira)
相关文件: src/fileio.c

Patch 7.2.299
问题: Crash when comment middle is longer than start.
解决方案: Fix size computation. (Lech Lorens)
相关文件: src/misc1.c

Patch 7.2.300
问题: Vim doesn't close file descriptors when forking and executing
another command, e.g., ":shell".
解决方案: Use FD_CLOEXEC when available. (James Vega)
相关文件: auto/configure, src/config.h.in, src/configure.in,
src/ex_cmdds2.c, src/fileio.c, src/memfile.c, src/memline.c

Patch 7.2.301
问题: Formatting is wrong when 'tw' is set to a small value.
解决方案: Fix it and add tests. Also fix behavior of "1" in 'fo'. (Yukihiro
Nakadaira)
相关文件: src/edit.c, src/testdir/Makefile, src/testdir/test68.in,
src/testdir/test68.ok, src/testdir/test69.in,
src/testdir/test69,ok

Patch 7.2.302 (extra part of 7.2.301)
问题: Formatting wrong with small 'tw' value.
解决方案: Add build rules for tests.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms

Patch 7.2.303 (after 7.2.294)
问题: Can't build on MS-Windows.
解决方案: Add #ifdef around vim_settempdir(). (James Vega)
相关文件: src/fileio.c

Patch 7.2.304
问题:　　Compiler warning for bad pointer cast.
解决方案:　Use another variable for int pointer.
相关文件:　src/ops.c

Patch 7.2.305
问题:　　Recursively redrawing causes a memory leak. (Dominique Pelle)
解决方案:　Disallow recursive screen updating.
相关文件:　src/screen.c

Patch 7.2.306
问题:　　shellescape("10%%", 1) only escapes first %. (Christian Brabandt)
解决方案:　Don't copy the character after the escaped one.
相关文件:　src/misc2.c

Patch 7.2.307
问题:　　Crash with a very long syntax match statement. (Guy Gur Ari)
解决方案:　When the offset does not fit in the two bytes available give an
　　error instead of continuing with invalid pointers.
相关文件:　src/regexp.c

Patch 7.2.308
问题:　　When using a regexp in the "\=" expression of a substitute
　　command, submatch() returns empty strings for further lines.
　　(Clockwork Jam)
解决方案:　Save and restore the line number and line count when calling
　　reg_getline().
相关文件:　src/regexp.c

Patch 7.2.309 (after 7.2.308)
问题:　　Warning for missing function prototype. (Patrick Texier)
解决方案:　Add the prototype.
相关文件:　src/regexp.c

Patch 7.2.310
问题:　　When a filetype plugin in ~/.vim/ftdetect uses ":setfiletype" and
　　the file starts with a "# comment" it gets "conf" filetype.
解决方案:　Check for "conf" filetype after using ftdetect plugins.
相关文件:　runtime/filetype.vim

Patch 7.2.311
问题:　　Can't compile with FreeMiNT.
解决方案:　Change #ifdef for limits.h. (Alan Hourihane)
相关文件:　src/fileio.c

Patch 7.2.312
问题:　　iconv() returns an invalid character sequence when conversion
　　fails. It should return an empty string. (Yongwei Wu)
解决方案:　Be more strict about invalid characters in the input.
相关文件:　src/mbyte.c

Patch 7.2.313
问题:　　Command line completion doesn't work after "%:h" and similar.

解决方案: Expand these items before doing the completion.
相关文件: src/ex_getln.c, src/misc1.c, src/proto/misc1.pro

Patch 7.2.314
问题: Missing function in small build.
解决方案: Always include concat_str.
相关文件: src/misc1.c

Patch 7.2.315
问题: Python libs can't be found on 64 bit system.
解决方案: Add lib64 to the list of directories. (Michael Henry)
相关文件: src/auto/configure, src/configure.in

Patch 7.2.316
问题: May get multiple _FORTIFY_SOURCE arguments. (Tony Mechelynck)
解决方案: First remove all these arguments and then add the one we want.
           (Dominique Pelle)
相关文件: src/auto/configure, src/configure.in

Patch 7.2.317
问题: Memory leak when adding a highlight group with unprintable
           characters, resulting in E669.
解决方案: Free the memory. And fix a few typos. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.2.318
问题: Wrong locale value breaks floating point numbers for gvim.
解决方案: Set the locale again after doing GUI inits. (Dominique Pelle)
相关文件: src/main.c

Patch 7.2.319
问题: Motif: accessing freed memory when cancelling font dialog.
解决方案: Destroy the widget only after accessing it. (Dominique Pelle)
相关文件: src/gui_xmdlg.c

Patch 7.2.320
问题: Unused function in Mzscheme interface.
解决方案: Remove the function and what depends on it. (Dominique Pelle)
相关文件: src/if_mzsch.c, src/proto/if_mzsch.pro

Patch 7.2.321
问题: histadd() and searching with "*" fails to add entry to history
           when it is empty.
解决方案: Initialize the history. (Lech Lorens)
相关文件: src/eval.c, src/normal.c

Patch 7.2.322
问题: Wrong indenting in virtual replace mode with CTRL-Y below a short
           line.
解决方案: Check for character to be NUL. (suggested by Lech Lorens)
相关文件: src/edit.c

Patch 7.2.323 (extra)
问题: Balloon evaluation crashes on Win64.

解决方案: Change pointer types. (Sergey Khorev)
相关文件: src/gui_w32.c

Patch 7.2.324
问题: A negative column argument in setpos() may cause a crash.
解决方案: Check for invalid column number. (James Vega)
相关文件: src/eval.c, src/misc2.c

Patch 7.2.325
问题: A stray "w" in the startup vimrc file causes the edited file to be
replaced with an empty file. (Stone Kang).
解决方案: Do not write a buffer when it has never been loaded.
相关文件: src/fileio.c

Patch 7.2.326
问题: Win32: $HOME doesn't work when %HOMEPATH% is not defined.
解决方案: Use "\" for %HOMEPATH% when it is not defined.
相关文件: src/misc1.c

Patch 7.2.327
问题: Unused functions in Workshop.
解决方案: Add "#if 0" and minor cleanup. (Dominique Pelle)
相关文件: src/workshop.c, src/integration.c, src/integration.h

Patch 7.2.328
问题: has("win64") does not return 1 on 64 bit MS-Windows version.
解决方案: Also check for _WIN64 besides WIN64.
相关文件: src/eval.c

Patch 7.2.329
问题: "g_" doesn't position cursor correctly when in Visual mode and
'selection' is "exclusive". (Ben Fritz)
解决方案: Call adjust_for_sel().
相关文件: src/normal.c

Patch 7.2.330
问题: Tables for Unicode case operators are outdated.
解决方案: Add a Vim script for generating the tables. Include tables for
Unicode 5.2.
相关文件: runtime/tools/README.txt, runtime/tools/unicode.vim, src/mbyte.c

Patch 7.2.331
问题: Can't interrupt "echo list" for a very long list.
解决方案: Call line_breakcheck() in list_join().
相关文件: src/eval.c

Patch 7.2.332
问题: Crash when spell correcting triggers an autocommand that reloads
the buffer.
解决方案: Make a copy of the line to be modified. (Dominique Pelle)
相关文件: src/spell.c

Patch 7.2.333
问题: Warnings from static code analysis.

解决方案: Small changes to various lines. (Dominique Pelle)
相关文件: src/buffer.c, src/edit.c, src/ex_getln.c, src/fileio.c,
           src/if_cscope.c, src/netbeans.c, src/ops.c, src/quickfix.c,
           src/syntax.c, src/ui.c

Patch 7.2.334
问题: Postponing keys in Netbeans interface does not work properly.
解决方案: Store the key string instead of the number. Avoid an infinite
           loop. (Mostly by Xavier de Gaye)
相关文件: src/netbeans.c, src/proto/netbeans.pro

Patch 7.2.335
问题: The CTRL-] command escapes too many characters.
解决方案: Use a different list of characters to be escaped. (Sergey Khorev)
相关文件: src/normal.c

Patch 7.2.336
问题: MzScheme interface can't evaluate an expression.
解决方案: Add mzeval(). (Sergey Khorev)
相关文件: runtime/doc/eval.txt, runtime/doc/if_mzsch.txt,
           runtime/doc/usr_41.txt, src/eval.c, src/if_mzsch.c,
           src/proto/eval.pro, src/proto/if_mzsch.pro,
           src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
           src/testdir/Makefile, src/testdir/main.aap, src/testdir/test1.in,
           src/testdir/test70.in, src/testdir/test70.ok

Patch 7.2.337
问题: The :compiler command doesn't function properly when invoked in a
           function.
解决方案: Add "g:" before "current_compiler". (Yukihiro Nakadaira)
相关文件: src/ex_cmds2.c

Patch 7.2.338 (after 7.2.300)
问题: Part of FD_CLOEXEC change is missing.
解决方案: Include source file skipped because of typo.
相关文件: src/ex_cmds2.c

Patch 7.2.339 (after 7.2.269)
问题: Part of --startuptime patch is missing.
解决方案: Add check for time_fd.
相关文件: src/ex_cmds2.c

Patch 7.2.340
问题: Gcc warning for condition that can never be true. (James Vega)
解决方案: Use start_lvl instead flp->lvl.
相关文件: src/fold.c

Patch 7.2.341
问题: Popup menu wraps to next line when double-wide character doesn't
           fit. (Jiang Ma)
解决方案: Display a ">" instead. (Dominique Pelle)
相关文件: src/screen.c

Patch 7.2.342

问题:　　Popup menu displayed wrong in 'rightleft' mode when there are multi-byte characters.
解决方案:　Adjust the column computations. (Dominique Pelle)
相关文件:　src/popupmnu.c

Patch 7.2.343 (after 7.2.338)
问题:　　Can't compile on Win32.
解决方案:　Insert the missing '|'.
相关文件:　src/ex_cmds2.c

Patch 7.2.344 (after 7.2.343)
问题:　　Can't compile on some systems
解决方案:　Move the #ifdef outside of the mch_open macro. (Patrick Texier)
相关文件:　src/ex_cmds2.c

Patch 7.2.345
问题:　　Tab line is not updated when the value of 'bt' is changed.
解决方案:　Call redraw_titles(). (Lech Lorens)
相关文件:　src/option.c

Patch 7.2.346
问题:　　Repeating a command with @: causes a mapping to be applied twice.
解决方案:　Do not remap characters inserted in the typeahead buffer. (Kana Natsuno)
相关文件:　src/ops.c

Patch 7.2.347
问题:　　Crash when executing <expr> mapping redefines that same mapping.
解决方案:　Save the values used before evaluating the expression.
相关文件:　src/getchar.c

Patch 7.2.348 (after 7.2.330)
问题:　　Unicode double-width characters are not up-to date.
解决方案:　Produce the double-width table like the others.
相关文件:　runtime/tools/unicode.vim, src/mbyte.c

Patch 7.2.349
问题:　　CTRL-W gf doesn't put the new tab in the same place as "tab split" and "gf". (Tony Mechelynck)
解决方案:　Store the tab number in cmdmod.tab.
相关文件:　src/window.c

Patch 7.2.350
问题:　　Win32: When changing font the window may jump from the secondary to the primary screen. (Michael Wookey)
解决方案:　When the screen position was negative don't correct it to zero.
相关文件:　src/gui.c

Patch 7.2.351 (after 7.2.347)
问题:　　Can't build with some compilers.
解决方案:　Move the #ifdef outside of a macro. Cleanup the code.
相关文件:　src/getchar.c

Patch 7.2.352 (extra)

问题: Win64: Vim doesn't work when cross-compiled with MingW libraries.
解决方案: Always return TRUE for the WM_NCCREATE message. (Andy Kittner)
相关文件: src/gui_w48.c

Patch 7.2.353
问题: No command line completion for ":profile".
解决方案: Complete the subcommand and file name.
相关文件: src/ex_docmd.c, src/ex_cmds2.c, src/ex_getln.c,
            src/proto/ex_cmds2.pro, src/vim.h

Patch 7.2.354
问题: Japanese single-width double-byte characters not handled correctly.
解决方案: Put 0x8e in ScreenLines[] and the second byte in ScreenLines2[].
            (partly by Kikuchan)
相关文件: src/screen.c

Patch 7.2.355
问题: Computing the cursor column in validate_cursor_col() is wrong when
            line numbers are used and 'n' is not in 'cpoptions', causing the
            popup menu to be positioned wrong.
解决方案: Correctly use the offset. (partly by Dominique Pelle)
相关文件: src/move.c

Patch 7.2.356
问题: When 'foldmethod' is changed not all folds are closed as expected.
解决方案: In foldUpdate() correct the start position and reset fd_flags when
            w_foldinvalid is set. (Lech Lorens)
相关文件: src/fold.c

Patch 7.2.357
问题: When changing 'fileformat' from/to "mac" and there is a CR in the
            text the display is wrong.
解决方案: Redraw the text when 'fileformat' is changed. (Ben Schmidt)
相关文件: src/option.c

Patch 7.2.358
问题: Compiler warnings on VMS. (Zoltan Arpadffy)
解决方案: Pass array itself instead its address. Return a value.
相关文件: src/gui_gtk_x11.c, src/os_unix.c

Patch 7.2.359
问题: Crash when using the Netbeans join command.
解决方案: Make sure the ml_flush_line() function is not used recursively.
            (Xavier de Gaye)
相关文件: src/memline.c

Patch 7.2.360
问题: Ruby on MS-Windows: can't use sockets.
解决方案: Call NtInitialize() during initialization. (Ariya Mizutani)
相关文件: src/if_ruby.c

Patch 7.2.361
问题: Ruby 1.9 is not supported.
解决方案: Add Ruby 1.9 support. (Masaki Suketa)

相关文件: src/Makefile, src/auto/configure, src/configure.in, src/if_ruby.c

Patch 7.2.362 (extra, after 7.2.352)
问题: Win64: Vim doesn't work when cross-compiled with MingW libraries.
解决方案: Instead of handling WM_NCCREATE, create wide text area window class if the parent window iw side. (Sergey Khorev)
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 7.2.363
问题: Can't dynamically load Perl 5.10.
解决方案: Add the function Perl_croak_xs_usage. (Sergey Khorev)
相关文件: src/if_perl.xs

Patch 7.2.364 (extra)
问题: Can't build gvimext.dll on Win 7 x64 using MinGW (John Marriott)
解决方案: Check if _MSC_VER is defined. (Andy Kittner)
相关文件: src/GvimExt/gvimext.h

Patch 7.2.365 (extra)
问题: MS-Windows with MingW: "File->Save As" does not work. (John Marriott)
解决方案: Correctly fill in structure size. (Andy Kittner)
相关文件: src/gui_w48.c

Patch 7.2.366
问题: CTRL-B doesn't go back to the first line of the buffer.
解决方案: Avoid an overflow when adding MAXCOL.
相关文件: src/move.c

Patch 7.2.367
问题: "xxd -r -p" doesn't work as documented.
解决方案: Skip white space. (James Vega)
相关文件: src/xxd/xxd.c

Patch 7.2.368 (after 7.2.361)
问题: Ruby interface: Appending line doesn't work. (Michael Henry)
解决方案: Reverse check for NULL line. (James Vega)
相关文件: src/if_ruby.c

Patch 7.2.369
问题: Error message is not easy to understand.
解决方案: Add quotes. (SungHyun Nam)
相关文件: src/ex_cmds2.c

Patch 7.2.370 (after 7.2.356)
问题: A redraw may cause folds to be closed.
解决方案: Revert part of the previous patch. Add a test. (Lech Lorens)
相关文件: src/diff.c, src/fold.c, src/option.c, src/testdir/test45.in, src/testdir/test45.ok

Patch 7.2.371
问题: Build problems on Tandem NonStop.
解决方案: A few changes to #ifdefs (Joachim Schmitz)
相关文件: src/auto/configure, src/configure.in, src/config.h.in, src/vim.h,

src/if_cscope.c, src/osdef1.h.in, src/tag.c

Patch 7.2.372 (extra)
问题: Cross-compiling GvimExt and xxd doesn't work.
解决方案: Change the build files. (Markus Heidelberg)
相关文件: src/INSTALLpc.txt, src/GvimExt/Make_ming.mak, src/Make_cyg.mak, src/Make_ming.mak, src/xxd/Make_cyg.mak

Patch 7.2.373
问题: Gcc 4.5 adds more error messages. (Chris Indy)
解决方案: Update default 'errorformat'.
相关文件: src/option.h

Patch 7.2.374
问题: Ruby eval() doesn't understand Vim types.
解决方案: Add the vim_to_ruby() function. (George Gensure)
相关文件: src/eval.c, src/if_ruby.c

Patch 7.2.375
问题: ml_get errors when using ":bprevious" in a BufEnter autocmd. (Dominique Pelle)
解决方案: Clear w_valid when entering another buffer.
相关文件: src/buffer.c

Patch 7.2.376
问题: ml_get error when using SiSU syntax. (Nathan Thomas)
解决方案: If the match ends below the last line move it to the end of the last line.
相关文件: src/syntax.c

Patch 7.2.377 (extra, after 7.2.372)
问题: Misplaced assignment. Duplicate build line for gvimext.dll.
解决方案: Move setting CROSS_COMPILE to before ifneq. Remove the wrong build line. (Markus Heidelberg)
相关文件: src/Make_ming.mak

Patch 7.2.378
问题: C function declaration indented too much. (Rui)
解决方案: Don't see a line containing { or } as a type. (Matt Wozniski)
相关文件: src/misc1.c

Patch 7.2.379
问题: 'eventignore' is set to an invalid value inside ":doau". (Antony Scriven)
解决方案: Don't include the leading comma when the option was empty.
相关文件: src/fileio.c

Patch 7.2.380 (after 7.2.363)
问题: Perl interface builds with 5.10.1 but not with 5.10.0.
解决方案: Change the #ifdefs. (Sergey Khorev)
相关文件: src/if_perl.xs

Patch 7.2.381
问题: No completion for :behave.

解决方案: Add :behave completion. Minor related fixes. (Dominique Pelle)
相关文件: src/ex_docmd.c, src/ex_getln.c, src/proto/ex_docmd.pro, src/vim.h

Patch 7.2.382
问题: Accessing freed memory when closing the cmdline window when
           'bufhide' is set to "wipe".
解决方案: Check if the buffer still exists before invoking close_buffer()
           (Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.2.383
问题: Vim doesn't build cleanly with MSVC 2010.
解决方案: Change a few types. (George Reilly)
相关文件: src/ex_cmds2.c, src/if_python.c, src/syntax.c

Patch 7.2.384 (extra)
问题: Vim doesn't build properly with MSVC 2010.
解决方案: Add the nmake version to the build file. (George Reilly)
相关文件: src/Make_mvc.mak, src/testdir/Make_dos.mak

Patch 7.2.385
问题: When in the command line window dragging status line only works
           for last-but-one window. (Jean Johner)
解决方案: Remove the code that disallows this.
相关文件: src/ui.c

Patch 7.2.386
问题: Focus hack for KDE 3.1 causes problems for other window managers.
解决方案: Remove the hack. (forwarded by Joel Bradshaw)
相关文件: src/gui_gtk.c

Patch 7.2.387
问题: Ruby with MingW still doesn't build all versions.
解决方案: More #ifdefs for the Ruby code. (Sergey Khorev)
相关文件: src/if_ruby.c

Patch 7.2.388 (extra part of 7.2.387)
问题: Ruby with MingW still doesn't build all versions.
解决方案: Different approach to build file. (Sergey Khorev)
相关文件: src/Make_ming.mak

Patch 7.2.389
问题: synIDattr() cannot return the font.
解决方案: Support the "font" argument. (Christian Brabandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/syntax.c

Patch 7.2.390
问题: In some situations the popup menu can be displayed wrong.
解决方案: Remove the popup menu if the cursor moved. (Lech Lorens)
相关文件: src/edit.c

Patch 7.2.391
问题: Internal alloc(0) error when doing "CTRL-V $ c". (Martti Kuparinen)
解决方案: Fix computations in getvcol(). (partly by Lech Lorens)

相关文件: src/charset.c, src/memline.c

Patch 7.2.392
问题: Netbeans hangs reading from a socket at the maximum block size.
解决方案: Use select() or poll(). (Xavier de Gaye)
相关文件: src/vim.h, src/os_unixx.h, src/if_xcmdsrv.c, src/netbeans.c

Patch 7.2.393
问题: Mac: Can't build with different Xcode developer tools directory.
解决方案: make "Developer" directory name configurable. (Rainer Muller)
相关文件: src/configure.in, src/auto/configure

Patch 7.2.394
问题: .lzma and .xz files are not supported.
解决方案: Recognize .lzma and .xz files so that they can be edited.
相关文件: runtime/plugin/gzip.vim

Patch 7.2.395
问题: In help CTRL=] on g?g? escapes the ?, causing it to fail. (Tony Mechelynck)
解决方案: Don't escape ? for a help command. (Sergey Khorev)
相关文件: src/normal.c

Patch 7.2.396
问题: Get E38 errors. (Dasn)
解决方案: Set cursor to line 1 instead of 0. (Dominique Pelle)
相关文件: src/popupmnu.c

Patch 7.2.397
问题: Redundant check for w_lines_valid.
解决方案: Remove the if. (Lech Lorens)
相关文件: src/fold.c

Patch 7.2.398
问题: When moving windows the cursor ends up in the wrong line.
解决方案: Set the window width and height properly. (Lech Lorens)
相关文件: src/window.c

Patch 7.2.399 (extra, after 7.2.388)
问题: Cannot compile on MingW.
解决方案: Move ifneq to separate line. (Vlad Sandrini, Dominique Pelle)
相关文件: src/Make_ming.mak

Patch 7.2.400 (after 7.2.387)
问题: Dynamic Ruby is not initialised properly for version 1.9.1. Ruby cannot create strings from NULL.
解决方案: Cleanup #ifdefs. Handle NULL like an empty string. Add ruby_init_stack. (Sergey Khorev)
相关文件: src/if_ruby.c

Patch 7.2.401
问题: ":e dir<Tab>" with 'wildmode' set to "list" doesn't highlight directory names with a space. (Alexandre Provencio)
解决方案: Remove the backslash before checking if the name is a directory.

(Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.2.402
问题: This gives a #705 error: let X = function('haslocaldir')
let X = function('getcwd')
解决方案: Don't give E705 when the name is found in the hashtab. (Sergey Khorev)
相关文件: src/eval.c

Patch 7.2.403 (after 7.2.400)
问题: Compiler warning for pointer type. (Tony Mechelynck)
解决方案: Move type cast to the right place.
相关文件: src/if_ruby.c

Patch 7.2.404
问题: Pointers for composing characters are not properly initialized.
解决方案: Compute the size of the pointer, not what it points to. (Yukihiro Nakadaira)
相关文件: src/screen.c

Patch 7.2.405
问题: When built with small features the matching text is not highlighted for ":s/pat/repl/c".
解决方案: Remove the #ifdef for IncSearch. (James Vega)
相关文件: src/syntax.c

Patch 7.2.406
问题: Patch 7.2.119 introduces uninit mem read. (Dominique Pelle)
解决方案: Only used ScreeenLinesC when ScreeenLinesUC is not zero. (Yukihiro Nakadaira) Also clear ScreeenLinesC when allocating.
相关文件: src/screen.c

Patch 7.2.407
问题: When using an expression in ":s" backslashes in the result are dropped. (Sergey Goldgaber, Christian Brabandt)
解决方案: Double backslashes.
相关文件: src/regexp.c

Patch 7.2.408
问题: With ":g/the/s/foo/bar/" the '[and '] marks can be set to a line that was not changed.
解决方案: Only set '[and '] marks when a substitution was done.
相关文件: src/ex_cmds.c

Patch 7.2.409
问题: Summary of number of substitutes is incorrect for ":folddo". (Jean Johner)
解决方案: Reset sub_nsubs and sub_nlines in global_exe().
相关文件: src/ex_cmds.c

Patch 7.2.410
问题: Highlighting directories for completion doesn't work properly.
解决方案: Don't halve backslashes when not needed, expanded "~/".

(Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.2.411
问题: When parsing 'cino' a comma isn't skipped properly.
解决方案: Skip the comma. (Lech Lorens)
相关文件: src/misc1.c

Patch 7.2.412
问题: [or] followed by mouse click doesn't work.
解决方案: Reverse check for key being a mouse event. (Dominique Pelle)
相关文件: src/normal.c

Patch 7.2.413
问题: Large file support is incorrect.
解决方案: Add AC_SYS_LARGEFILE to configure. (James Vega)
相关文件: src/configure.in, src/config.h.in, src/auto/configure

Patch 7.2.414
问题: CTRK-K <space> <space> does not produce 0xa0 as expected. (Tony Mechelynck)
解决方案: Remove the Unicode range 0xe000 - 0xefff from digraphs, these are not valid characters.
相关文件: src/digraph.c

Patch 7.2.415
问题: Win32: Can't open a remote file when starting Vim.
解决方案: Don't invoke cygwin_conv_path() for URLs. (Tomoya Adachi)
相关文件: src/main.c

Patch 7.2.416
问题: Logtalk.dict is not installed.
解决方案: Add it to the install target. (Markus Heidelberg)
相关文件: src/Makefile

Patch 7.2.417
问题: When 'shell' has an argument with a slash then 'shellpipe' is not set properly. (Britton Kerin)
解决方案: Assume there are no spaces in the path, arguments follow.
相关文件: src/option.c

Patch 7.2.418
问题: Vim tries to set the background or foreground color in a terminal to -1. (Graywh) Happens with ":hi Normal ctermbg=NONE".
解决方案: When resetting the foreground or background color don't set the color, let the clear screen code do that.
相关文件: src/syntax.c

Patch 7.2.419
问题: Memory leak in Motif when clicking on "Search Vim Help".
解决方案: Free string returned by XmTextGetString(). (Dominique Pelle)
相关文件: src/gui_motif.c

Patch 7.2.420

问题:　　":argedit" does not accept "++enc=utf8" as documented. (Dominique Pelle)
解决方案:　Add the ARGOPT flag to ":argedit".
相关文件:　src/ex_cmds.h

Patch 7.2.421
问题:　　Folds are sometimes not updated properly and there is no way to force an update.
解决方案:　Make "zx" and "zX" recompute folds (suggested by Christian Brabandt)
相关文件:　src/normal.c

Patch 7.2.422
问题:　　May get E763 when using spell dictionaries.
解决方案:　Avoid utf-8 case folded character to be truncated to 8 bits and differ from latin1. (Dominique Pelle)
相关文件:　src/spell.c

Patch 7.2.423
问题:　　Crash when assigning s: to variable. (Yukihiro Nakadaira)
解决方案:　Make ga_scripts contain pointer to scriptvar_T instead of scriptvar_T itself. (Dominique Pelle)
相关文件:　src/eval.c

Patch 7.2.424
问题:　　":colorscheme" without an argument doesn't do anything.
解决方案:　Make it echo the current color scheme name. (partly by Christian Brabandt)
相关文件:　runtime/doc/syntax.txt, src/ex_cmds.h, src/ex_docmd.c

Patch 7.2.425
问题:　　Some compilers complain about fourth EX() argument.
解决方案:　Add cast to long_u.
相关文件:　src/ex_cmds.h

Patch 7.2.426
问题:　　Commas in 'langmap' are not always handled correctly.
解决方案:　Require commas to be backslash escaped. (James Vega)
相关文件:　src/option.c

Patch 7.2.427
问题:　　The swapfile is created using the destination of a symlink, but recovery doesn't follow symlinks.
解决方案:　When recovering, resolve symlinks. (James Vega)
相关文件:　src/memline.c

Patch 7.2.428
问题:　　Using setqflist([]) to clear the error list doesn't work properly.
解决方案:　Set qf_nonevalid to TRUE when appropriate. (Christian Brabandt)
相关文件:　src/quickfix.c

Patch 7.2.429
问题:　　A file that exists but access is denied may result in a "new file" message. E.g. when its directory is unreadable.

解决方案: Specifically check for ENOENT to decide a file doesn't exist.
           (partly by James Vega)
相关文件: src/fileio.c

Patch 7.2.430
问题: The ++bad argument is handled wrong, resulting in an invalid
           memory access.
解决方案: Use the bad_char field only for the replacement character, add
           bad_char_idx to store the position. (Dominique Pelle)
相关文件: src/eval.c, src/ex_cmds.h, src/ex_docmd.c

Patch 7.2.431
问题: ":amenu" moves the cursor when in Insert mode.
解决方案: Use CTRL-\ CTRL-O instead of CTRL-O. (Christian Brabandt)
相关文件: src/menu.c

Patch 7.2.432
问题: When menus are translated they can only be found by the translated
           name. That makes ":emenu" difficult to use.
解决方案: Store the untranslated name and use it for completion and :emenu.
           (Liang Peng (Bezetek James), Edward L. Fox)
相关文件: src/menu.c, src/structs.h

Patch 7.2.433
问题: Can't use cscope with QuickFixCmdPre and QuickFixCmdPost.
解决方案: Add cscope support for these autocmd events. (Bryan Venteicher)
相关文件: runtime/doc/autocmd.txt, src/if_cscope.c

Patch 7.2.434 (after 7.2.432)
问题: Compilation fails without the multi-lang feature.
解决方案: Add #ifdefs. (John Marriott)
相关文件: src/menu.c

Patch 7.2.435 (after 7.2.430)
问题: Crash when using bad_char_idx uninitialized. (Patrick Texier)
解决方案: Don't use bad_char_idx, reproduce the ++bad argument from bad_char.
相关文件: src/eval.c, src/ex_cmds.h, src/ex_docmd.c

Patch 7.2.436
问题: Reproducible crash in syntax HL. (George Reilly, Dominique Pelle)
解决方案: Make sst_stacksize an int instead of short. (Dominique Pelle)
相关文件: src/structs.h

Patch 7.2.437 (after 7.2.407)
问题: When "\\\n" appears in the expression result the \n doesn't result
           in a line break. (Andy Wokula)
解决方案: Also replace a \n after a backslash into \r.
相关文件: src/regexp.c

Patch 7.2.438 (after 7.2.427)
问题: "vim -r" crashes.
解决方案: Don't use NULL pointer argument.
相关文件: src/memline.c

Patch 7.2.439
问题:　　Invalid memory access when doing thesaurus completion and 'infercase' is set.
解决方案:　Use the minimal length of completed word and replacement. (Dominique Pelle)
相关文件:　src/edit.c

Patch 7.2.440
问题:　　Calling a function through a funcref, where the function deletes the funcref, leads to an invalid memory access.
解决方案:　Make a copy of the function name. (Lech Lorens)
相关文件:　src/eval.c, src/testdir/test34.in, src/testdir/test34.ok

Patch 7.2.441
问题:　　When using ":earlier" undo information may be wrong.
解决方案:　When changing alternate branches also adjust b_u_oldhead.
相关文件:　src/undo.c

Patch 7.2.442 (after 7.2.201)
问题:　　Copy/paste with OpenOffice doesn't work.
解决方案:　Do not offer the HTML target when it is not supported. (James Vega)
相关文件:　src/gui_gtk_x11.c, src/option.c, src/proto/gui_gtk_x11.pro

Patch 7.2.443
问题:　　Using taglist() on a tag file with duplicate fields generates an internal error. (Peter Odding)
解决方案:　Check for duplicate field names.
相关文件:　src/eval.c, src/proto/eval.pro, src/tag.c

Patch 7.2.444 (after 7.2.442)
问题:　　Can't build with GTK 1, gtk_selection_clear_targets() is not available. (Patrick Texier)
解决方案:　Don't change the targets for GTK 1, set them once.
相关文件:　src/gui_gtk_x11.c, src/option.c

Patch 7.2.445
问题:　　Crash when using undo/redo and a FileChangedRO autocmd event that reloads the buffer. (Dominique Pelle)
解决方案:　Do not allow autocommands while performing and undo or redo.
相关文件:　src/misc1.c, src/undo.c

Patch 7.2.446
问题:　　Crash in GUI when closing the last window in a tabpage. (ryo7000)
解决方案:　Remove the tabpage from the list before freeing the window.
相关文件:　src/window.c

When writing a file, switching tab pages and selecting a word the file write message would be displayed again. This happened in Insert mode and with 'cmdheight' set to 2.

When using ":lang" to set a locale that uses a comma for decimal separator and using GTK floating point numbers stop working. Use gtk_disable_setlocale(). (James Vega)

"g8" didn't produce the right value on a NUL. (Dominique Pelle)

Use BASEMODLIBS instead of MODLIBS for Python configuration to pick up the right compiler flags. (Michael Bienia)

Window title was not updated after dropping a file on Vim. (Hari G)

synstack() did not return anything when just past the end of the line. Useful when using the cursor position in Insert mode.

When entering a digraph or special character after a line that fits the window the '?' or '^' on the next line is not redrawn. (Ian Kelling)

Composing characters in :s substitute text were dropped.

exists() was causing an autoload script to be loaded.

Filter out -pthread for cproto.

Make CTRL-L in command line mode respect 'ignorecase' and 'smartcase'. (Martin Toft)

Spell menu moved the cursor, causing Copy not to work. Spell replacement didn't work in 'compatible' mode.

Various small fixes from Dominique Pelle.

Fix that :mksession may generate "2argu" even though there is no such argument. (Peter Odding)

Fixes for time in clipboard request. Also fix ownership. (David Fries)

Fixed completion of file names with '%' and '*'.

Fixed MSVC makefile use of /Wp64 flag.

Correct use of long instead of off_t for file size. (James Vega)

Add a few #ifdefs to exclude functions that are not used. (Dominique Pelle)

Remove old and unused method to allocate memory for undo.

Fix definition of UINT_PTR for 64 bit systems.

Some versions of Ruby redefine rb_str_new2 to rb_str_new_cstr.

Window title not updated after file dropped.

Fixed crash for ":find" completion, might also happen in other path expansion usage.

When 'searchhl' causes a hang make CTRL-C disable 'searchhl'.

When resetting both 'title' and 'icon' the title would be set after a shell
command.

Reset 'title' and 'icon' in test47 to avoid the xterm title getting messed up.

Fix for compiler warning about function prototype in pty.c.

Added 'window' to the options window.

Fixed: errors for allocating zero bytes when profiling an empty function.

Remove -arch flag from build flags for Perl. (Bjorn Wickler)

Fix 'autochdir' not showing up in :options window. (Dominique Pelle)

Fix: test 69 didn't work on MS-Windows. Test 72 beeped too often.

Avoid illegal memory access in spell suggestion. (Dominique Pelle)
Fix: crash in spell checking with a 0x300 character.

Avoid that running tests changes viminfo.

Fix: changing case of a character removed combining characters.
Fixed: CTRL-R in Insert mode doesn't insert composing characters.

Added the WOW64 flag to OLE registration, for 64 bit Windows systems.

Various fixes for coverity warnings.

Fix compile warnings, esp. for 64-bit systems. (Mike Williams)

Fix: :redir to a dictionary that is changed before ":redir END" causes a
memory access error.

Fix: terminal title not properly restored when there are multi-byte
characters. (partly by James Vega)

Set 'wrapscan' when checking the .po files. (Mike Williams)

Win32: Put quotes around the gvim.exe path for the "Open with" menu entry.

On MS-Windows sometimes files with number 4913 or higher are left behind.

'suffixesadd' was used for finding tags file.

Removed unused code.

Improved positioning of combining characters in GTK.

Made test 11 pass when there is no gzip program. (John Beckett)

Changed readfile() to ignore byte order marks, unless in binary mode.

On MS-Windows completion of shell commands didn't work.

An unprintable multi-byte character at the start of the screen line caused the following text to be drawn at the wrong position.

Got ml_get errors when using undo with 'virtualedit'.

Call gui_mch_update() before triggering GuiEnter autocmd. (Ron Aaron)

Unix "make install" installed a few Amiga .info files.

Disallow setting 'ambiwidth' to "double" when 'listchars' or 'fillchars' contains a character that would become double width.

Set 'wrapscan' when checking the .po files. (Mike Williams)

Fixed: using expression in command line may cause a crash.

Avoid warnings from the clang compiler. (Dominique Pelle)

Fix: Include wchar.h in charset.c for towupper().

Fixed: Using ":read file" in an empty buffer when 'compatible' is set caused an error. Was caused by patch 7.2.132.

Make the references to features in the help more consistent. (Sylvain Hitier)

7.4 版 本 version-7.4 version7.4 vim-7.4

本小节讨论 7.3 和 7.4 版本之间的改进。

这个版本有几百个漏洞修正，和若干新特性。最主要的新特性有：

- 新正规表达式引擎 new-regexp-engine
- 更像 python 的 Python 接口 better-python-interface

新正规表达式引擎 new-regexp-engine

现在我们称为 "老式的" 正规表达式引擎使用一种回溯算法。它试图用一种方式匹配模式与文本，如果失败，回退再试一种方式。这对简单模式很好，但对复杂模式和很长的文本上会很慢。

新的引擎使用状态机。它会尝试当前字符所有的可能性，并保存模式的所有可能状态。对简单模式会稍慢些，但对复杂模式和长文本会快很多。

尤其显著地是，Javascript 和 XML 文件对长行的语法高亮现在好多了。过去 Vim 可能会停顿很久。

详情见： two-engines

更好的 Python 接口 better-python-interface

新增 python-bindeval 函数。和 python-eval 不同，这个函数分别为字典、列表和函数返回 python-Dictionary 、 python-List 和 python-Function 对象，而不是它们的 Python 内建对应对象（说起函数的话就是 None）。
对简单类型，此函数返回 Python 内建类型，而不是像 python-eval 那样返回 Python str() 。Python 3 则返回 bytes() 对象而不是 str() ，以避免出错 UnicodeDecodeError 的可能性。
新对象的接口一定程度上模仿标准 Python 的 dict() 和 list() 接口。将来会改进模仿的程度。

新增特殊的 python-vars 对象，也可用于 python-buffer 和 python-window 。它们方便了从 Python 访问 Vim 脚本变量。

现在你不再需要修改 sys.path 来导入自己的模块：特殊的钩子现在负责从 {rtp}/python2、{rtp}/python3 和 {rtp}/pythonx 目录（分别用于 Python 2、Python 3 和两者都可用的版本）导入模块。
见 python-special-path 。

现在可以通过 python-tabpage 对象来访问 tabpage 。

新增 Vim 错误和例外到 Python 例外的自动转换。

改变 python-buffers 对象的行为：现在使用缓冲区编号而不是内部缓冲区列表的缓冲区序列号作为键。这不应该影响任何现有的程序，因为过去得到该序列号的唯一方法是遍历 python-buffers 。

新增 :pydo 和 :py3do 命令。

新增 pyeval() 和 py3eval() 函数。

现在所有过去支持 str() 对象的地方，都能同时接受 str() 和 unicode() (Python 2) 或 bytes() 和 str() (Python 3)。

python-window 有了 .col 和 .row 属性，这是现在唯一可以得到内部窗口位置的方法。

新增或修正 Vim Python 对象对 dir() 的支持。

改动 changed-7.4

不再支持旧的 Python 版本（⩽2.2）。反正也不能正确地进行编译了。

选项：

非 GUI 版本的 Vim 现在可以自动保存选择区到系统剪贴板（'clipboard' 中的 autoselectplus）。也可以使用系统剪贴板为缺省寄存器（以前只能用主选择区）。(Ivan Krasilnikov, Christian Brabandt, Bram Moolenaar)

新增 'shiftwidth' 的特殊值，使 'sw' 可以依赖于 'tabstop'。因为 'indentexpr' 的缩进会因此变得麻烦，又加入 shiftwidth() 函数。还增加了等价的 'softtabstop' 的特殊值。(Christian Brabandt, so8res)

'relativenumber' 选项打开时在行号列显示绝对行号。现在 'number' 和 'relativenumber' 有四种不同的组合。(Christian Brabandt)

命令:

:diffoff 现在保存某些设置的局部值并恢复，而不是盲目地重设为它们的缺省值。(Christian Brabandt)

其他:

Lua 接口现在也使用用户数据来绑定到 Vim 数据结构。(Taro Muraoka, Luis Carvalho)

glob() 和自动命令的模式过去可以使用未公开的正规表达式学来的 "\{n,m\}" 项目。"\{" 现在是按本义出现的 "{"，因为这是外壳文件模式的正常行为。现在须用 "\\\{n,m\}" 来得到类似于正规表达式模式的 "\{n,m}"。

新增 added-7.4

新增若干语法、缩进和其他插件。

新增 viminfo 对 Lists 和 Dictionaries 的支持。(Christian Brabandt)

函数:

位操作函数: and() 、 or() 、 invert() 、 xor() .

新增 luaeval() 函数。(Taro Muraoka, Luis Carvalho)

新增 sha256() 函数。(Tyru, Hirohito Higashi)

新增 wildmenumode() 函数。(Christian Brabandt)

调试函数: screenattr() 、 screenchar() 、 screencol() 、 screenrow() 。(Simon Ruderich, Bram Moolenaar)

新增 sort() 通过可选的第三个参数使用 Dictionary-function 的功能。(Nikolay Pavlov)

新增特殊的 expand() 参数，用于扩展当前行号。

现在可以强制 char2nr() 给出 unicode 代码点，而不取决于当前编码。(Yasuhiro Matsumoto)

现在可以使返回文件列表的函数生成 List 而不是 NL 分隔的字符串。(例如 glob() 、 expand()) (Christian Brabandt)

从指定窗口、标签页或缓冲区读取变量的函数现在可以在找不到变量时返回指定的缺省值，而不是空字符串。(gettabvar() 、 getwinvar() 、 getbufvar()) (Shougo Matsushita, Hirohito Higashi)

自动命令:

新增 InsertCharPre 事件，在插入字符前发生。(Jakson A. Aquino)

新增 CompleteDone 事件，在插入模式补全结束后发生。(Florian Klein 的

主意)

新增 QuitPre 事件，在可以关闭 Vim 或只是关闭若干窗口的命令执行时发生。

新增 TextChanged 和 TextChangedI 事件，在文本修改时发生。

命令:

:syntime 命令，用于调试。

现在可以用 :sign-unplace 删除当前缓冲区的所有标号。(Christian Brabandt)

新增 :language 自动补全。(Dominique Pelle)

新增更多 :command-complete 补全类型: :behave 子选项、色彩方案、编译器、 :cscope 子选项、'path' 里的文件、 :history 子选项、locale 名、 :syntime 子选项、用户名。(Dominique Pelle)

加入 :map-nowait ，若创建映射的 lhs 是另一映射的 lhs 的前缀时，不允许 Vim 等待用户输入更多字符来消除二义性，而总是迫使 Vim 使用较短的选择: 带 <nowait> 的那个。

选项:

现在可以补全时忽略大小写: 'wildignorecase'。

可以用 'formatoptions' 的 j 标志位使得 J 删除注释前导符 (fo-table)。(Lech Lorens)

现在可以控制 namespace 里如何缩进: cino-N 。(Konstantin Lepa)

现在可以分别控制 if 条件内的对齐和函数参数内的对齐: cino-k 。(Lech Lorens)

其他:

改进 cmd.exe 的支持。(Ben Fritz, Bram Moolenaar)

加入 v:windowid 参数，包含 GUI Vim 的当前窗口号。(Christian J. Robinson, Lech Lorens)

加入 rxvt-unicode 和 SGR 鼠标支持。(Yiding Jia, Hayaki Saito)

7.4 版本的所有修正 fixed-7.4

Patch 7.3.001
问题: When editing "src/main.c" and 'path' set to "./proto", ":find e<C-D" shows ./proto/eval.pro instead of eval.pro.
解决方案: Check for path separator when comparing names. (Nazri Ramliy)
相关文件: src/misc1.c

Patch 7.3.002
问题: ":find" completion doesn't work when halfway an environment

variable. (Dominique Pelle)
解决方案: Only use in-path completion when expanding file names. (Nazri Ramliy)
相关文件: src/ex_docmd.c

Patch 7.3.003
问题: Crash with specific BufWritePost autocmd. (Peter Odding)
解决方案: Don't free the quickfix title twice. (Lech Lorens)
相关文件: src/quickfix.c

Patch 7.3.004
问题: Crash when using very long regexp. (Peter Odding)
解决方案: Reset reg_toolong. (Carlo Teubner)
相关文件: src/regexp.c

Patch 7.3.005
问题: Crash when using undotree(). (Christian Brabandt)
解决方案: Increase the list reference count. Add a test for undotree() (Lech Lorens)
相关文件: src/eval.c, src/testdir/Makefile, src/testdir/test61.in

Patch 7.3.006
问题: Can't build some multi-byte code with C89.
解决方案: Move code to after declarations. (Joachim Schmitz)
相关文件: src/mbyte.c, src/spell.c

Patch 7.3.007
问题: Python code defines global "buffer". Re-implements a grow-array.
解决方案: Use a grow-array instead of coding the same functionality. Handle out-of-memory situation properly.
相关文件: src/if_py_both.h

Patch 7.3.008
问题: 'cursorbind' is kept in places where 'scrollbind' is reset.
解决方案: Reset 'cursorbind'.
相关文件: src/buffer.c, src/diff.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/if_cscope.c, src/macros.h, src/quickfix.c, src/search.c, src/tag.c, src/window.c

Patch 7.3.009
问题: Win32: Crash on Windows when using a bad argument for strftime(). (Christian Brabandt)
解决方案: Use the bad_param_handler(). (Mike Williams)
相关文件: src/os_win32.c

Patch 7.3.010
问题: Mac GUI: Missing break statements.
解决方案: Add the break statements. (Dominique Pelle)
相关文件: src/gui_mac.c

Patch 7.3.011
问题: X11 clipboard doesn't work in Athena/Motif GUI. First selection after a shell command doesn't work.
解决方案: When using the GUI use XtLastTimestampProcessed() instead of

changing a property. (partly by Toni Ronkko)
When executing a shell command disown the selection.
相关文件: src/ui.c, src/os_unix.c

Patch 7.3.012
问题: Problems building with MingW.
解决方案: Adjust the MingW makefiles. (Jon Maken)
相关文件: src/Make_ming.mak, src/GvimExt/Make_ming.mak

Patch 7.3.013
问题: Dynamic loading with Ruby doesn't work for 1.9.2.
解决方案: Handle rb_str2cstr differently. Also support dynamic loading on Unix. (Jon Maken)
相关文件: src/if_ruby.c

Patch 7.3.014
问题: Ending a line in a backslash inside an ":append" or ":insert" command in Ex mode doesn't work properly. (Ray Frush)
解决方案: Halve the number of backslashes, only insert a NUL after an odd number of backslashes.
相关文件: src/ex_getln.c

Patch 7.3.015
问题: Test is using error message that no longer exists.
解决方案: Change E106 to E121. (Dominique Pelle)
相关文件: src/testdir/test49.vim

Patch 7.3.016
问题: Netbeans doesn't work under Athena.
解决方案: Support Athena, just like Motif. (Xavier de Gaye)
相关文件: runtime/doc/netbeans.txt, src/gui.c, src/main.c, src/netbeans.c

Patch 7.3.017
问题: smatch reports errors.
解决方案: Fix the reported errors. (Dominique Pelle)
相关文件: src/spell.c, src/syntax.c

Patch 7.3.018 (after 7.3.012)
问题: Missing argument to windres in MingW makefiles.
解决方案: Add the argument that was wrapped in the patch. (Jon Maken)
相关文件: src/Make_ming.mak, src/GvimExt/Make_ming.mak

Patch 7.3.019
问题: ":nbstart" can fail silently.
解决方案: Give an error when netbeans is not supported by the GUI. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.3.020
问题: Cursor position wrong when joining multiple lines and 'formatoptions' contains "a". (Moshe Kamensky)
解决方案: Adjust cursor position for skipped indent. (Carlo Teubner)
相关文件: src/ops.c, src/testdir/test68.in, src/testdir/test68.ok

Patch 7.3.021
问题:　　Conflict for defining Boolean in Mac header files.
解决方案:　Define NO_X11_INCLUDES. (Rainer Muller)
相关文件:　src/os_macosx.m, src/vim.h

Patch 7.3.022
问题:　　When opening a new window the 'spellcapcheck' option is cleared.
解决方案:　Copy the correct option value. (Christian Brabandt)
相关文件:　src/option.c

Patch 7.3.023
问题:　　External program may hang when it tries to write to the tty.
解决方案:　Don't close the slave tty until after the child exits. (Nikola
　　Knezevic)
相关文件:　src/os_unix.c

Patch 7.3.024
问题:　　Named signs do not use a negative number as intended.
解决方案:　Fix the numbering of named signs. (Xavier de Gaye)
相关文件:　src/ex_cmds.c

Patch 7.3.025
问题:　　":mksession" does not square brackets escape file name properly.
解决方案:　Improve escaping of file names. (partly by Peter Odding)
相关文件:　src/ex_docmd.c

Patch 7.3.026
问题:　　CTRL-] in a help file doesn't always work. (Tony Mechelynck)
解决方案:　Don't escape special characters. (Carlo Teubner)
相关文件:　src/normal.c

Patch 7.3.027
问题:　　Opening a file on a network share is very slow.
解决方案:　When fixing file name case append "*" to directory, server and
　　network share names. (David Anderson, John Beckett)
相关文件:　src/os_win32.c

Patch 7.3.028 (after 7.3.024)
问题:　　Signs don't show up. (Charles Campbell)
解决方案:　Don't use negative numbers. Also assign a number to signs that
　　have a name of all digits to avoid using a sign number twice.
相关文件:　src/ex_cmds.c

Patch 7.3.029
问题:　　":sort n" sorts lines without a number as number zero. (Beeyawned)
解决方案:　Make lines without a number sort before lines with a number. Also
　　fix sorting negative numbers.
相关文件:　src/ex_cmds.c, src/testdir/test57.in, src/testdir/test57.ok

Patch 7.3.030
问题:　　Cannot store Dict and List in viminfo file.
解决方案:　Add support for this. (Christian Brabandt)
相关文件:　runtime/doc/options.txt, src/eval.c, src/testdir/Make_amiga.mak,
　　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,

src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/main.aap, src/testdir/test74.in,
src/testdir/test74.ok

Patch 7.3.031
问题: Can't pass the X window ID to another application.
解决方案: Add v:windowid. (Christian J. Robinson, Lech Lorens)
相关文件: runtime/doc/eval.txt, src/eval.c, src/gui.c, src/vim.h,
src/os_unix.c

Patch 7.3.032
问题: maparg() doesn't return the flags, such as <buffer>, <script>,
<silent>. These are needed to save and restore a mapping.
解决方案: Improve maparg(). (also by Christian Brabandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/getchar.c, src/gui_w48.c,
src/message.c, src/proto/getchar.pro, src/proto/message.pro,
src/structs.h src/testdir/test75.in, src/testdir/test75.ok

Patch 7.3.033 (after 7.3.032)
问题: Can't build without FEAT_LOCALMAP.
解决方案: Add an #ifdef. (John Marriott)
相关文件: src/getchar.c

Patch 7.3.034
问题: Win32: may be loading .dll from the wrong directory.
解决方案: Go to the Vim executable directory when opening a library.
相关文件: src/gui_w32.c, src/if_lua.c, src/if_mzsch.c, src/if_perl.xs,
src/if_python.c, src/if_python3.c, src/if_ruby.c, src/mbyte.c,
src/os_mswin.c, src/os_win32.c, src/proto/os_win32.pro

Patch 7.3.035 (after 7.3.034)
问题: Stray semicolon after if statement. (Hari G)
解决方案: Remove the semicolon.
相关文件: src/os_win32.c

Patch 7.3.036
问题: Win32 GUI: When building without menus, the font for dialogs and
tab page headers also changes.
解决方案: Define USE_SYSMENU_FONT always. (Harig G.)
相关文件: src/gui_w32.c

Patch 7.3.037
问题: Compiler warnings for loss of data. (Mike Williams)
解决方案: Add type casts.
相关文件: src/if_py_both.h, src/getchar.c, src/os_win32.c

Patch 7.3.038
问题: v:windowid isn't set on MS-Windows.
解决方案: Set it to the window handle. (Chris Sutcliffe)
相关文件: runtime/doc/eval.txt, src/gui_w32.c

Patch 7.3.039
问题: Crash when using skk.vim plugin.
解决方案: Get length of expression evaluation result only after checking for

NULL. (Noriaki Yagi, Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.3.040
问题: Comparing strings while ignoring case goes beyond end of the string when there are illegal bytes. (Dominique Pelle)
解决方案: Explicitly check for illegal bytes.
相关文件: src/mbyte.c

Patch 7.3.041
问题: Compiler warning for accessing mediumVersion. (Tony Mechelynck)
解决方案: Use the pointer instead of the array itself. (Dominique Pelle)
相关文件: src/version.c

Patch 7.3.042
问题: No spell highlighting when re-using an empty buffer.
解决方案: Clear the spell checking info only when clearing the options for a buffer. (James Vega)
相关文件: src/buffer.c

Patch 7.3.043
问题: Can't load Ruby dynamically on Unix.
解决方案: Adjust the configure script. (James Vega)
相关文件: src/Makefile, src/config.h.in, src/configure.in, src/auto/configure, src/if_ruby.c

Patch 7.3.044
问题: The preview window opened by the popup menu is larger than specified with 'previewheight'. (Benjamin Haskell)
解决方案: Use 'previewheight' if it's set and smaller.
相关文件: src/popupmnu.c

Patch 7.3.045
问题: Compiler warning for uninitialized variable.
解决方案: Initialize the variable always.
相关文件: src/getchar.c

Patch 7.3.046 (after 7.3.043)
问题: Can't build Ruby on MS-Windows.
解决方案: Add #ifdef, don't use WIN3264 before including vim.h.
相关文件: src/if_ruby.c

Patch 7.3.047 (after 7.3.032)
问题: Missing makefile updates for test 75.
解决方案: Update the makefiles.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Makefile, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.048
问题: ":earlier 1f" doesn't work after loading undo file.
解决方案: Set b_u_save_nr_cur when loading an undo file. (Christian Brabandt)
Fix only showing time in ":undolist"

相关文件: src/undo.c

Patch 7.3.049
问题: PLT has rebranded their Scheme to Racket.
解决方案: Add support for Racket 5.x. (Sergey Khorev)
相关文件: src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak,
src/auto/configure, src/configure.in, src/if_mzsch.c

Patch 7.3.050
问题: The link script is clumsy.
解决方案: Use the --as-needed linker option if available. (Kirill A.
Shutemov)
相关文件: src/Makefile, src/auto/configure, src/config.mk.in,
src/configure.in, src/link.sh

Patch 7.3.051
问题: Crash when $PATH is empty.
解决方案: Check for vim_getenv() returning NULL. (Yasuhiro Matsumoto)
相关文件: src/ex_getln.c, src/os_win32.c

Patch 7.3.052
问题: When 'completefunc' opens a new window all kinds of errors follow.
(Xavier Deguillard)
解决方案: When 'completefunc' goes to another window or buffer and when it
deletes text abort completion. Add a test for 'completefunc'.
相关文件: src/edit.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test76.in, src/testdir/test76.ok

Patch 7.3.053
问题: complete() function doesn't reset complete direction. Can't use
an empty string in the list of matches.
解决方案: Set compl_direction to FORWARD. Add "empty" key to allow empty
words. (Kikuchan)
相关文件: src/edit.c

Patch 7.3.054
问题: Can define a user command for :Print, but it doesn't work. (Aaron
Thoma)
解决方案: Let user command :Print overrule the builtin command (Christian
Brabandt) Disallow :X and :Next as a user defined command.
相关文件: src/ex_docmd.c

Patch 7.3.055
问题: Recursively nested lists and dictionaries cause a near-endless
loop when comparing them with a copy. (ZyX)
解决方案: Limit recursiveness in a way that non-recursive structures can
still be nested very deep.
相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.3.056
问题: "getline" argument in do_cmdline() shadows global.
解决方案: Rename the argument.

相关文件: src/ex_docmd.c

Patch 7.3.057
问题: Segfault with command line abbreviation. (Randy Morris)
解决方案: Don't retrigger the abbreviation when abandoning the command line. Continue editing the command line after the error.
相关文件: src/ex_getln.c

Patch 7.3.058
问题: Error "code converter not found" when loading Ruby script.
解决方案: Load Gem module. (Yasuhiro Matsumoto)
相关文件: src/if_ruby.c

Patch 7.3.059
问题: Netbeans: Problem with recursively handling messages for Athena and Motif.
解决方案: Call netbeans_parse_messages() in the main loop, like it's done for GTK. (Xavier de Gaye)
相关文件: src/gui_x11.c, src/netbeans.c

Patch 7.3.060
问题: Netbeans: crash when socket is disconnected unexpectedly.
解决方案: Don't cleanup when a read fails, put a message in the queue and disconnect later. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.3.061
问题: Remote ":drop" does not respect 'autochdir'. (Peter Odding)
解决方案: Don't restore the directory when 'autochdir' is set. (Benjamin Fritz)
相关文件: src/main.c

Patch 7.3.062
问题: Python doesn't work properly when installed in another directory than expected.
解决方案: Figure out home directory in configure and use Py_SetPythonHome() at runtime. (Roland Puntaier)
相关文件: src/configure.in, src/auto/configure, src/if_python.c, src/if_python3.c

Patch 7.3.063
问题: Win32: Running a filter command makes Vim lose focus.
解决方案: Use SW_SHOWMINNOACTIVE instead of SW_SHOWMINIMIZED. (Hong Xu)
相关文件: src/os_win32.c

Patch 7.3.064
问题: Win32: ":dis +" shows nothing, but "+p does insert text.
解决方案: Display the * register, since that's what will be inserted. (Christian Brabandt)
相关文件: src/globals.h, src/ops.c

Patch 7.3.065
问题: Can't get current line number in a source file.
解决方案: Add the <slnum> item, similar to <sfile>.

相关文件: src/ex_docmd.c

Patch 7.3.066
问题: Crash when changing to another window while in a :vimgrep command.
(Christian Brabandt)
解决方案: When wiping out the dummy before, remove it from aucmd_win.
相关文件: src/quickfix.c

Patch 7.3.067 (after 7.3.058)
问题: Ruby: Init_prelude is not always available.
解决方案: Remove use of Init_prelude. (Yasuhiro Matsumoto)
相关文件: src/if_ruby.c

Patch 7.3.068
问题: Using freed memory when doing ":saveas" and an autocommand sets
'autochdir'. (Kevin Klement)
解决方案: Get the value of fname again after executing autocommands.
相关文件: src/ex_cmds.c

Patch 7.3.069
问题: GTK: pressing Enter in inputdialog() doesn't work like clicking OK
as documented.
解决方案: call gtk_entry_set_activates_default(). (Britton Kerin)
相关文件: src/gui_gtk.c

Patch 7.3.070
问题: Can set environment variables in the sandbox, could be abused.
解决方案: Disallow it.
相关文件: src/eval.c

Patch 7.3.071
问题: Editing a file in a window that's in diff mode resets 'diff'
but not cursor binding.
解决方案: Reset cursor binding in two more places.
相关文件: src/quickfix.c, src/option.c

Patch 7.3.072
问题: Can't complete file names while ignoring case.
解决方案: Add 'wildignorecase'.
相关文件: src/ex_docmd.c, src/ex_getln.c, src/misc1.c, src/option.c,
src/option.h, src/vim.h, src/runtime/options.txt

Patch 7.3.073
问题: Double free memory when netbeans command follows DETACH.
解决方案: Only free the node when owned. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.3.074
问题: Can't use the "+ register like "* for yank and put.
解决方案: Add "unnamedplus" to the 'clipboard' option. (Ivan Krasilnikov)
相关文件: runtime/doc/options.txt, src/eval.c, src/globals.h, src/ops.c,
src/option.c

Patch 7.3.075 (after 7.3.072)

问题: Missing part of 'wildignorecase'
解决方案: Also adjust expand()
相关文件: src/eval.c

Patch 7.3.076
问题: Clang warnings for dead code.
解决方案: Remove it. (Carlo Teubner)
相关文件: src/gui_gtk.c, src/if_ruby.c, src/misc2.c, src/netbeans.c,
            src/spell.c

Patch 7.3.077
问题: When updating crypt of swapfile fails there is no error message.
            (Carlo Teubner)
解决方案: Add the error message.
相关文件: src/memline.c

Patch 7.3.078
问题: Warning for unused variable.
解决方案: Adjust #ifdefs.
相关文件: src/ops.c

Patch 7.3.079
问题: Duplicate lines in makefile.
解决方案: Remove the lines. (Hong Xu)
相关文件: src/Make_mvc.mak

Patch 7.3.080
问题: Spell doesn't work on VMS.
解决方案: Use different file names. (Zoltan Bartos, Zoltan Arpadffy)
相关文件: src/spell.c

Patch 7.3.081
问题: Non-printable characters in 'statusline' cause trouble. (ZyX)
解决方案: Use transstr(). (partly by Caio Ariede)
相关文件: src/screen.c

Patch 7.3.082
问题: Leaking file descriptor when hostname doesn't exist.
解决方案: Remove old debugging lines.
相关文件: src/netbeans.c

Patch 7.3.083
问题: When a read() or write() is interrupted by a signal it fails.
解决方案: Add read_eintr() and write_eintr().
相关文件: src/fileio.c, src/proto/fileio.pro, src/memfile.c, src/memline.c,
            src/os_unix.c, src/undo.c, src/vim.h

Patch 7.3.084
问题: When splitting the window, the new one scrolls with the cursor at
            the top.
解决方案: Compute w_fraction before setting the new height.
相关文件: src/window.c

Patch 7.3.085 (after 7.3.083)

问题:　　Inconsistency with preproc symbols. void * computation.
解决方案:　Include vimio.h from vim.h. Add type cast.
相关文件:　src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/fileio.c,
　　src/if_cscope.c, src/if_sniff.c, src/main.c, src/memfile.c,
　　src/memline.c, src/netbeans.c, src/os_msdos.c, src/os_mswin.c,
　　src/os_win16.c, src/os_win32.c, src/spell.c, src/tag.c,
　　src/undo.c, src/vim.h

Patch 7.3.086
问题:　　When using a mapping with an expression and there was no count,
　　v:count has the value of the previous command. (ZyX)
解决方案:　Also set v:count and v:count1 before getting the character that
　　could be a command or a count.
相关文件:　src/normal.c

Patch 7.3.087
问题:　　EINTR is not always defined.
解决方案:　Include errno.h in vim.h.
相关文件:　src/if_cscope.c, src/if_tcl.c, src/integration.c, src/memline.c,
　　src/os_mswin.c, src/os_win16.c, src/os_win32.c, src/vim.h,
　　src/workshop.c

Patch 7.3.088
问题:　　Ruby can't load Gems sometimes, may cause a crash.
解决方案:　Undefine off_t. Use ruby_process_options(). (Yasuhiro Matsumoto)
相关文件:　src/if_ruby.c

Patch 7.3.089
问题:　　Compiler warning on 64 bit MS-Windows.
解决方案:　Add type cast. (Mike Williams)
相关文件:　src/netbeans.c

Patch 7.3.090
问题:　　Wrong help text for Cscope.
解决方案:　Adjust the help text for "t". (Dominique Pelle)
相关文件:　src/if_cscope.c

Patch 7.3.091
问题:　　"vim -w foo" writes special key codes for removed escape
　　sequences. (Josh Triplett)
解决方案:　Don't write K_IGNORE codes.
相关文件:　src/getchar.c, src/misc1.c, src/term.c, src/vim.h

Patch 7.3.092
问题:　　Resizing the window when exiting.
解决方案:　Don't resize when exiting.
相关文件:　src/term.c

Patch 7.3.093
问题:　　New DLL dependencies in MingW with gcc 4.5.0.
解决方案:　Add STATIC_STDCPLUS, LDFLAGS and split up WINDRES. (Guopeng Wen)
相关文件:　src/GvimExt/Make_ming.mak, src/Make_ming.mak

Patch 7.3.094

问题:　　Using abs() requires type cast to int.
解决方案:　Use labs() so that the value remains long. (Hong Xu)
相关文件:　src/screen.c

Patch 7.3.095
问题:　　Win32: In Chinese tear-off menu doesn't work. (Weasley)
解决方案:　Use menu_name_equal(). (Alex Jakushev)
相关文件:　src/menu.c

Patch 7.3.096
问题:　　"gvim -nb" is not interruptible. Leaking file descriptor on
netbeans connection error.
解决方案:　Check for CTRL-C typed. Free file descriptor. (Xavier de Gaye)
相关文件:　src/netbeans.c

Patch 7.3.097
问题:　　Using ":call" inside "if 0" does not see that a function returns a
Dict and gives error for "." as string concatenation.
解决方案:　Use eval0() to skip over the expression. (Yasuhiro Matsumoto)
相关文件:　src/eval.c

Patch 7.3.098
问题:　　Function that ignores error still causes called_emsg to be set.
E.g. when expand() fails the status line is disabled.
解决方案:　Move check for emsg_not_now() up. (James Vega)
相关文件:　src/message.c

Patch 7.3.099
问题:　　Crash when splitting a window with zero height. (Yukihiro
Nakadaira)
解决方案:　Don't set the fraction in a window with zero height.
相关文件:　src/window.c

Patch 7.3.100
问题:　　When using :normal v:count isn't set.
解决方案:　Call normal_cmd() with toplevel set to TRUE.
相关文件:　src/ex_docmd.c

Patch 7.3.101
问题:　　ino_t defined with wrong size.
解决方案:　Move including auto/config.h before other includes. (Marius
Geminas)
相关文件:　src/if_ruby.c, src/if_lua.c

Patch 7.3.102
问题:　　When using ":make", typing the next command and then getting the
"reload" prompt the next command is (partly) eaten by the reload
prompt.
解决方案:　Accept ':' as a special character at the reload prompt to accept
the default choice and execute the command.
相关文件:　src/eval.c, src/fileio.c, src/gui.c, src/gui_xmdlg.c,
src/memline.c, src/message.c, src/proto/message.pro,
src/gui_athena.c, src/gui_gtk.c, src/gui_mac.c, src/gui_motif.c,
src/gui_photon.c, src/gui_w16.c, src/gui_w32.c, src/os_mswin.c

src/proto/gui_athena.pro, src/proto/gui_gtk.pro,
src/proto/gui_mac.pro, src/proto/gui_motif.pro,
src/proto/gui_photon.pro, src/proto/gui_w16.pro,
src/proto/gui_w32.pro

Patch 7.3.103
问题: Changing 'fileformat' and then using ":w" in an empty file sets the 'modified' option.
解决方案: In unchanged() don't ignore 'ff' for an empty file.
相关文件: src/misc1.c, src/option.c, src/proto/option.pro, src/undo.c

Patch 7.3.104
问题: Conceal: using Tab for cchar causes problems. (ZyX)
解决方案: Do not accept a control character for cchar.
相关文件: src/syntax.c

Patch 7.3.105
问题: Can't get the value of "b:changedtick" with getbufvar().
解决方案: Make it work. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.3.106
问题: When 'cursorbind' is set another window may scroll unexpectedly when 'scrollbind' is also set. (Xavier Wang)
解决方案: Don't call update_topline() if 'scrollbind' is set.
相关文件: src/move.c

Patch 7.3.107
问题: Year number for :undolist can be confused with month or day.
解决方案: Change "%y" to "%Y".
相关文件: src/undo.c

Patch 7.3.108
问题: Useless check for NULL when calling vim_free().
解决方案: Remove the check. (Dominique Pelle)
相关文件: src/eval.c, src/ex_cmds.c, src/os_win32.c

Patch 7.3.109
问题: Processing new Esperanto spell file fails and crashes Vim. (Dominique Pelle)
解决方案: When running out of memory give an error. Handle '?' in COMPOUNDRULE properly.
相关文件: src/spell.c

Patch 7.3.110
问题: The "nbsp" item in 'listchars' isn't used for ":list".
解决方案: Make it work. (Christian Brabandt)
相关文件: src/message.c

Patch 7.3.111 (after 7.3.100)
问题: Executing a :normal command in 'statusline' evaluation causes the cursor to move. (Dominique Pelle)
解决方案: When updating the cursor for 'cursorbind' allow the cursor beyond the end of the line. When evaluating 'statusline' temporarily

reset 'cursorbind'.
相关文件: src/move.c, src/screen.c

Patch 7.3.112
问题: Setting 'statusline' to "%!'asdf%' reads uninitialized memory.
解决方案: Check for NUL after %.
相关文件: src/buffer.c

Patch 7.3.113
问题: Windows: Fall back directory for creating temp file is wrong.
解决方案: Use "." instead of empty string. (Hong Xu)
相关文件: src/fileio.c

Patch 7.3.114
问题: Potential problem in initialization when giving an error message early.
解决方案: Initialize 'verbosefile' empty. (Ben Schmidt)
相关文件: src/option.h

Patch 7.3.115
问题: Vim can crash when tmpnam() returns NULL.
解决方案: Check for NULL. (Hong Xu)
相关文件: src/fileio.c

Patch 7.3.116
问题: 'cursorline' is displayed too short when there are concealed characters and 'list' is set. (Dennis Preiser)
解决方案: Check for 'cursorline' when 'list' is set. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.3.117
问题: On some systems --as-needed does not work, because the "tinfo" library is included indirectly from "ncurses". (Charles Campbell)
解决方案: In configure prefer using "tinfo" instead of "ncurses".
相关文件: src/configure.in, src/auto/configure

Patch 7.3.118
问题: Ruby uses SIGVTALARM which makes Vim exit. (Alec Tica)
解决方案: Ignore SIGVTALARM. (Dominique Pelle)
相关文件: src/os_unix.c

Patch 7.3.119
问题: Build problem on Mac. (Nicholas Stallard)
解决方案: Use "extern" instead of "EXTERN" for p_vfile.
相关文件: src/option.h

Patch 7.3.120
问题: The message for an existing swap file is too long to fit in a 25 line terminal.
解决方案: Make the message shorter. (Chad Miller)
相关文件: src/memline.c

Patch 7.3.121
问题: Complicated 'statusline' causes a crash. (Christian Brabandt)

解决方案: Check that the number of items is not too big.
相关文件: src/buffer.c

Patch 7.3.122
问题: Having auto/config.mk in the repository causes problems.
解决方案: Remove auto/config.mk from the distribution. In the toplevel
Makefile copy it from the "dist" file.
相关文件: Makefile, src/Makefile, src/auto/config.mk

Patch 7.3.123
问题: ml_get error when executing register being recorded into, deleting
lines and 'conceallevel' is set. (ZyX)
解决方案: Don't redraw a line for concealing when it doesn't exist.
相关文件: src/main.c

Patch 7.3.124
问题: When writing a file in binary mode it may be missing the final EOL
if a file previously read was missing the EOL. (Kevin Goodsell)
解决方案: Move the write_no_eol_lnum into the buffer struct.
相关文件: src/structs.h, src/fileio.c, src/globals.h, src/os_unix.c

Patch 7.3.125
问题: MSVC: Problem with quotes in link argument.
解决方案: Escape backslashes and quotes. (Weasley)
相关文件: src/Make_mvc.mak

Patch 7.3.126
问题: Compiler warning for signed pointer.
解决方案: Use unsigned int argument for sscanf().
相关文件: src/blowfish.c

Patch 7.3.127
问题: Compiler complains about comma.
解决方案: Remove comma after last enum element.
相关文件: src/ex_cmds2.c

Patch 7.3.128
问题: Another compiler warning for signed pointer.
解决方案: Use unsigned int argument for sscanf().
相关文件: src/mark.c

Patch 7.3.129
问题: Using integer like a boolean.
解决方案: Nicer check for integer being non-zero.
相关文件: src/tag.c

Patch 7.3.130
问题: Variable misplaced in #ifdef.
解决方案: Move clipboard_event_time outside of #ifdef.
相关文件: src/gui_gtk_x11.c

Patch 7.3.131
问题: Including errno.h too often.
解决方案: Don't include errno.h in Unix header file.

相关文件: src/os_unix.h

Patch 7.3.132
问题: C++ style comments.
解决方案: Change to C comments.
相关文件: src/if_python3.c

Patch 7.3.133
问题: When using encryption it's not clear what method was used.
解决方案: In the file message show "blowfish" when using blowfish.
相关文件: src/fileio.c

Patch 7.3.134
问题: Drag-n-drop doesn't work in KDE Dolphin.
解决方案: Add GDK_ACTION_MOVE flag. (Florian Degner)
相关文件: src/gui_gtk_x11.c

Patch 7.3.135
问题: When there is no previous substitute pattern, the previous search pattern is used. The other way around doesn't work.
解决方案: When there is no previous search pattern, use the previous substitute pattern if possible. (Christian Brabandt)
相关文件: src/search.c

Patch 7.3.136
问题: Duplicate include of assert.h.
解决方案: Remove it.
相关文件: src/if_cscope.c

Patch 7.3.137 (after 7.3.091)
问题: When 'lazyredraw' is set the screen may not be updated. (Ivan Krasilnikov)
解决方案: Call update_screen() before waiting for input.
相关文件: src/misc1.c, src/getchar.c

Patch 7.3.138
问题: ":com" changes the multi-byte text of :echo. (Dimitar Dimitrov)
解决方案: Search for K_SPECIAL as a byte, not a character. (Ben Schmidt)
相关文件: src/ex_docmd.c

Patch 7.3.139 (after 7.3.137)
问题: When 'lazyredraw' is set ":ver" output can't be read.
解决方案: Don't redraw the screen when at a prompt or command line.
相关文件: src/getchar.c, src/message.c, src/misc1.c

Patch 7.3.140
问题: Crash when drawing the "$" at end-of-line for list mode just after the window border and 'cursorline' is set.
解决方案: Don't check for 'cursorline'. (Quentin Carbonneaux)
相关文件: src/screen.c

Patch 7.3.141
问题: When a key code is not set get a confusing error message.
解决方案: Change the error message to say the key code is not set.

相关文件: src/option.c, runtime/doc/options.txt

Patch 7.3.142
问题: Python stdout doesn't have a flush() method, causing an import to fail.
解决方案: Add a dummy flush() method. (Tobias Columbus)
相关文件: src/if_py_both.h

Patch 7.3.143
问题: Memfile is not tested sufficiently. Looking up blocks in a memfile is slow when there are many blocks.
解决方案: Add high level test and unittest. Adjust the number of hash buckets to the number of blocks. (Ivan Krasilnikov)
相关文件: Filelist, src/Makefile, src/main.c, src/memfile.c, src/memfile_test.c src/structs.h src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mak, src/testdir/Makefile, src/testdir/test77.in, src/testdir/test77.ok

Patch 7.3.144
问题: Crash with ":python help(dir)". (Kearn Holliday)
解决方案: Fix the way the type is set on objects. (Tobias Columbus)
相关文件: src/if_python.c

Patch 7.3.145 (after 7.3.144)
问题: Can't build with Python dynamically loading.
解决方案: Add dll_PyType_Ready.
相关文件: src/if_python.c

Patch 7.3.146
问题: It's possible to assign to a read-only member of a dict.
It's possible to create a global variable "0". (ZyX)
It's possible to add a v: variable with ":let v:.name = 1".
解决方案: Add check for dict item being read-only.
Check the name of g: variables.
Disallow adding v: variables.
相关文件: src/eval.c

Patch 7.3.147 (after 7.3.143)
问题: Can't build on HP-UX.
解决方案: Remove an unnecessary backslash. (John Marriott)
相关文件: src/Makefile

Patch 7.3.148
问题: A syntax file with a huge number of items or clusters causes weird behavior, a hang or a crash. (Yukihiro Nakadaira)
解决方案: Check running out of IDs. (partly by Ben Schmidt)
相关文件: src/syntax.c

Patch 7.3.149
问题: The cursor disappears after the processing of the 'setDot' netbeans command when vim runs in a terminal.
解决方案: Show the cursor after a screen update. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.3.150
问题: readline() does not return the last line when the NL is missing.
(Hong Xu)
解决方案: When at the end of the file Also check for a previous line.
相关文件: src/eval.c

Patch 7.3.151 (after 7.3.074)
问题: When "unnamedplus" is in 'clipboard' the selection is sometimes
also copied to the star register.
解决方案: Avoid copy to the star register when undesired. (James Vega)
相关文件: src/ops.c

Patch 7.3.152
问题: Xxd does not check for errors from library functions.
解决方案: Add error checks. (Florian Zumbiehl)
相关文件: src/xxd/xxd.c

Patch 7.3.153 (after 7.3.152)
问题: Compiler warning for ambiguous else, missing prototype.
解决方案: Add braces. (Dominique Pelle) Add prototype for die().
相关文件: src/xxd/xxd.c

Patch 7.3.154 (after 7.3.148)
问题: Can't compile with tiny features. (Tony Mechelynck)
解决方案: Move #define outside of #ifdef.
相关文件: src/syntax.c

Patch 7.3.155
问题: Crash when using map(), filter() and remove() on v:. (ZyX)
Also for extend(). (Yukihiro Nakadaira)
解决方案: Mark v: as locked. Also correct locking error messages.
相关文件: src/eval.c

Patch 7.3.156
问题: Tty names possibly left unterminated.
解决方案: Use vim_strncpy() instead of strncpy().
相关文件: src/pty.c

Patch 7.3.157
问题: Superfluous assignment.
解决方案: Remove assignment.
相关文件: src/misc1.c

Patch 7.3.158
问题: Might use uninitialized memory in C indenting.
解决方案: Init arrays to empty.
相关文件: src/misc1.c

Patch 7.3.159
问题: Using uninitialized pointer when out of memory.
解决方案: Check for NULL return value.
相关文件: src/mbyte.c

Patch 7.3.160
问题: Unsafe string copying.
解决方案: Use vim_strncpy() instead of strcpy(). Use vim_strcat() instead of strcat().
相关文件: src/buffer.c, src/ex_docmd.c, src/hardcopy.c, src/menu.c, src/misc1.c, src/misc2.c, src/proto/misc2.pro, src/netbeans.c, src/os_unix.c, src/spell.c, src/syntax.c, src/tag.c

Patch 7.3.161
问题: Items on the stack may be too big.
解决方案: Make items static or allocate them.
相关文件: src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/fileio.c, src/hardcopy.c, src/quickfix.c, src/main.c, src/netbeans.c, src/spell.c, src/tag.c, src/vim.h, src/xxd/xxd.c

Patch 7.3.162
问题: No error message when assigning to a list with an index out of range. (Yukihiro Nakadaira)
解决方案: Add the error message.
相关文件: src/eval.c

Patch 7.3.163
问题: For the default of 'shellpipe' "mksh" and "pdksh" are not recognized.
解决方案: Recognize these shell names.
相关文件: src/option.c

Patch 7.3.164
问题: C-indenting: a preprocessor statement confuses detection of a function declaration.
解决方案: Ignore preprocessor lines. (Lech Lorens) Also recognize the style to put a comma before the argument name.
相关文件: src/misc1.c, testdir/test3.in, testdir/test3.ok

Patch 7.3.165
问题: ":find" completion does not escape spaces in a directory name. (Isz)
解决方案: Add backslashes for EXPAND_FILES_IN_PATH. (Carlo Teubner)
相关文件: src/ex_getln.c

Patch 7.3.166
问题: Buffer on the stack may be too big
解决方案: Allocate the space.
相关文件: src/option.c

Patch 7.3.167
问题: When using the internal grep QuickFixCmdPost is not triggered. (Yukihiro Nakadaira)
解决方案: Change the place where autocommands are triggered.
相关文件: src/quickfix.c

Patch 7.3.168
问题: When the second argument of input() contains a CR the text up to that is used without asking the user. (Yasuhiro Matsumoto)

解决方案: Change CR, NL and ESC in the text to a space.
相关文件: src/getchar.c

Patch 7.3.169
问题: Freeing memory already freed, warning from static code analyzer.
解决方案: Initialize pointers to NULL, correct use of "mustfree". (partly by Dominique Pelle)
相关文件: src/mis1.c

Patch 7.3.170
问题: VMS Makefile for testing was not updated for test77.
解决方案: Add test77 to the Makefile.
相关文件: src/testdir/Make_vms.mms

Patch 7.3.171
问题: When the clipboard isn't supported: ":yank*" gives a confusing error message.
解决方案: Specifically mention that the register name is invalid. (Jean-Rene David)
相关文件: runtime/doc/change.txt, src/ex_docmd.c, src/globals.h

Patch 7.3.172
问题: MS-Windows: rename() might delete the file if the name differs but it's actually the same file.
解决方案: Use the file handle to check if it's the same file. (Yukihiro Nakadaira)
相关文件: src/if_cscope.c, src/fileio.c, src/os_win32.c, src/proto/os_win32.pro, src/vim.h

Patch 7.3.173
问题: After using setqflist() to make the quickfix list empty ":cwindow" may open the window anyway. Also after ":vimgrep".
解决方案: Correctly check whether the list is empty. (Ingo Karkat)
相关文件: src/quickfix.c

Patch 7.3.174
问题: When Exuberant ctags binary is exctags it's not found.
解决方案: Add configure check for exctags. (Hong Xu)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.175
问题: When 'colorcolumn' is set locally to a window, ":new" opens a window with the same highlighting but 'colorcolumn' is empty. (Tyru)
解决方案: Call check_colorcolumn() after clearing and copying options. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.3.176
问题: Ruby linking doesn't work properly on Mac OS X.
解决方案: Fix the configure check for Ruby. (Bjorn Winckler)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.177

问题:　　MS-Windows: mkdir() doesn't work properly when 'encoding' is "utf-8".
解决方案:　Convert to utf-16. (Yukihiro Nakadaira)
相关文件:　src/os_win32.c, src/os_win32.h, src/proto/os_win32.pro

Patch 7.3.178
问题:　　C-indent doesn't handle code right after { correctly.
解决方案:　Fix detecting unterminated line. (Lech Lorens)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.179
问题:　　C-indent doesn't handle colon in string correctly.
解决方案:　Skip the string. (Lech Lorens)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.180
问题:　　When both a middle part of 'comments' matches and an end part, the middle part was used erroneously.
解决方案:　After finding the middle part match continue looking for a better end part match. (partly by Lech Lorens)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.181
问题:　　When repeating the insert of CTRL-V or a digraph the display may not be updated correctly.
解决方案:　Only call edit_unputchar() after edit_putchar(). (Lech Lorens)
相关文件:　src/edit.c

Patch 7.3.182 (after 7.3.180)
问题:　　Compiler warning for uninitialized variable.
解决方案:　Add dummy initializer.
相关文件:　src/misc1.c

Patch 7.3.183 (after 7.3.174)
问题:　　When Exuberant ctags binary is exuberant-ctags it's not found.
解决方案:　Add configure check for exuberant-ctags.
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.184
问题:　　Static code analysis errors in riscOS.
解决方案:　Make buffer size bigger. (Dominique Pelle)
相关文件:　src/gui_riscos.c

Patch 7.3.185
问题:　　":windo g/pattern/q" closes windows and reports "N more lines". (Tim Chase)
解决方案:　Remember what buffer ":global" started in. (Jean-Rene David)
相关文件:　src/ex_cmds.c

Patch 7.3.186
问题:　　When 'clipboard' contains "unnamed" or "unnamedplus" the value of v:register is wrong for operators without a specific register.
解决方案:　Adjust the register according to 'clipboard'. (Ingo Karkat)
相关文件:　src/normal.c

Patch 7.3.187
问题: The RISC OS port has obvious errors and is not being maintained.
解决方案: Remove the RISC OS files and code.
相关文件: src/ascii.h, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c,
src/ex_docmd.c, src/fileio.c, src/globals.h, src/gui.c, src/gui.h,
src/main.c, src/memfile.c, src/memline.c, src/misc1.c,
src/proto.h, src/quickfix.c, src/search.c, src/structs.h,
src/term.c, src/termlib.c, src/version.c, src/vim.h,
src/gui_riscos.h, src/os_riscos.h, src/gui_riscos.c,
src/os_riscos.c, runtime/doc/os_risc.txt

Patch 7.3.188
问题: More RISC OS files to remove.
解决方案: Remove them. Update the file list.
相关文件: src/proto/gui_riscos.pro, src/proto/os_riscos.pro, Filelist

Patch 7.3.189 (after 7.3.186)
问题: Can't build without +clipboard feature. (Christian Ebert)
解决方案: Add the missing #ifdef.
相关文件: src/normal.c

Patch 7.3.190
问题: When there is a "containedin" syntax argument highlighting may be
wrong. (Radek)
解决方案: Reset current_next_list. (Ben Schmidt)
相关文件: src/syntax.c

Patch 7.3.191
问题: Still some RISC OS stuff to remove.
解决方案: Remove files and lines. (Hong Xu)
Remove the 'osfiletype' option code.
相关文件: README_extra.txt, src/Make_ro.mak, src/INSTALL, src/Makefile,
src/buffer.c, src/eval.c, src/feature.h, src/option.c,
src/option.h, src/structs.h, src/version.c, src/pty.c, Filelist

Patch 7.3.192
问题: Ex command ":s/ \?/ /g" splits multi-byte characters into bytes.
(Dominique Pelle)
解决方案: Advance over whole character instead of one byte.
相关文件: src/ex_cmds.c

Patch 7.3.193
问题: In the command line window ":close" doesn't work properly. (Tony
Mechelynck)
解决方案: Use Ctrl_C instead of K_IGNORE for cmdwin_result. (Jean-Rene
David)
相关文件: src/ex_docmd.c, src/ex_getln.c

Patch 7.3.194
问题: When "b" is a symlink to directory "a", resolve("b/") doesn't
result in "a/". (ZyX)
解决方案: Remove the trailing slash. (Jean-Rene David)
相关文件: src/eval.c

Patch 7.3.195
问题: "} else" causes following lines to be indented too much. (Rouben Rostamian)
解决方案: Better detection for the "else". (Lech Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.196
问题: Can't intercept a character that is going to be inserted.
解决方案: Add the InsertCharPre autocommand event. (Jakson A. Aquino)
相关文件: runtime/doc/autocmd.txt, runtime/doc/eval.txt, runtime/doc/map.txt, src/edit.c, src/eval.c, src/fileio.c, src/vim.h

Patch 7.3.197
问题: When a QuickfixCmdPost event removes all errors, Vim still tries to jump to the first error, resulting in E42.
解决方案: Get the number of error after the autocmd event. (Mike Lundy)
相关文件: src/quickfix.c

Patch 7.3.198
问题: No completion for ":lang".
解决方案: Get locales to complete from. (Dominique Pelle)
相关文件: src/eval.c, src/ex_cmds2.c, src/ex_getln.c, src/proto/ex_cmds2.pro, src/proto/ex_getln.pro, src/vim.h

Patch 7.3.199
问题: MS-Windows: Compilation problem of OLE with MingW compiler.
解决方案: Put #ifdef around declarations. (Guopeng Wen)
相关文件: src/if_ole.h

Patch 7.3.200 (after 7.3.198)
问题: CTRL-D doesn't complete :lang.
解决方案: Add the missing part of the change. (Dominique Pelle)
相关文件: src/ex_docmd.c

Patch 7.3.201 (after 7.3.195)
问题: "} else" still causes following lines to be indented too much.
解决方案: Better detection for the "else" block. (Lech Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.202
问题: Cannot influence the indent inside a namespace.
解决方案: Add the "N" 'cino' parameter. (Konstantin Lepa)
相关文件: runtime/doc/indent.txt, src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.203
问题: MS-Windows: Can't run an external command without a console window.
解决方案: Support ":!start /b cmd". (Xaizek)
相关文件: runtime/doc/os_win32.txt, src/os_win32.c

Patch 7.3.204 (after 7.3.201)
问题: Compiler warning.

解决方案: Add type cast. (Mike Williams)
相关文件: src/misc1.c

Patch 7.3.205
问题: Syntax "extend" doesn't work correctly.
解决方案: Avoid calling check_state_ends() recursively (Ben Schmidt)
相关文件: src/syntax.c

Patch 7.3.206
问题: 64bit MS-Windows compiler warning.
解决方案: Use HandleToLong() instead of type cast. (Mike Williams)
相关文件: src/gui_w32.c

Patch 7.3.207
问题: Can't compile with MSVC with pentium4 and 64 bit.
解决方案: Only use SSE2 for 32 bit. (Mike Williams)
相关文件: src/Make_mvc.mak

Patch 7.3.208
问题: Early terminated if statement.
解决方案: Remove the semicolon. (Lech Lorens)
相关文件: src/gui_mac.c

Patch 7.3.209
问题: MSVC Install instructions point to wrong batch file.
解决方案: Add a batch file for use with MSVC 10.
相关文件: src/msvc2010.bat, src/INSTALLpc.txt, Filelist

Patch 7.3.210
问题: Can't always find the file when using cscope.
解决方案: Add the 'cscoperelative' option. (Raghavendra D Prabhu)
相关文件: runtime/doc/if_cscop.txt, runtime/doc/options.txt,
src/if_cscope.c

Patch 7.3.211 (after 7.3.210)
问题: Compiler warning.
解决方案: Add type cast.
相关文件: src/if_cscope.c

Patch 7.3.212
问题: With Python 3.2 ":py3" fails.
解决方案: Move PyEval_InitThreads() to after Py_Initialize(). (Roland
Puntaier) Check abiflags in configure. (Andreas Behr)
相关文件: src/if_python3.c, src/auto/configure, src/configure.in

Patch 7.3.213
问题: Javascript object literal is not indented correctly.
解决方案: Make a special case for when "J1" is in 'cino'. (Luc Deschenaux)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.214
问题: The text displayed by ":z-" isn't exactly like old Vi.
解决方案: Add one to the start line number. (ChangZhuo Chen)
相关文件: src/ex_cmds.c

Patch 7.3.215 (after 7.3.210)
问题: Wrong file names in previous patch. (Toothpik)
解决方案: Include the option changes.
相关文件: src/option.c, src/option.h

Patch 7.3.216
问题: When recovering a file a range of lines is missing. (Charles Jie)
解决方案: Reset the index when advancing to the next pointer block. Add a test to verify recovery works.
相关文件: src/memline.c, src/testdir/test78.in, src/testdir/test78.ok, src/testdir/Makefile, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.217
问题: Inside an "if" a ":wincmd" causes problems.
解决方案: When skipping commands let ":wincmd" skip over its argument.
相关文件: src/ex_docmd.c

Patch 7.3.218 (after 7.3.212)
问题: Tiny configuration problem with Python 3.
解决方案: Add abiflags in one more place. (Andreas Behr)
相关文件: src/auto/configure, src/configure.in

Patch 7.3.219
问题: Can't compile with GTK on Mac.
解决方案: Add some #ifdef trickery. (Ben Schmidt)
相关文件: src/os_mac_conv.c, src/os_macosx.m, src/vim.h

Patch 7.3.220
问题: Python 3: vim.error is a 'str' instead of an 'Exception' object, so 'except' or 'raise' it causes a 'SystemError' exception.
Buffer objects do not support slice assignment.
When exchanging text between Vim and Python, multibyte texts become garbage or cause Unicode Exceptions, etc.
'py3file' tries to read in the file as Unicode, sometimes causes UnicodeDecodeException
解决方案: Fix the problems. (lilydjwg)
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.221
问题: Text from the clipboard is sometimes handled as linewise, but not consistently.
解决方案: Assume the text is linewise when it ends in a CR or NL.
相关文件: src/gui_gtk_x11.c, src/gui_mac.c, src/ops.c, src/os_msdos.c, src/os_mswin.c, src/os_qnx.c, src/ui.c

Patch 7.3.222
问题: Warning for building GvimExt.
解决方案: Comment-out the DESCRIPTION line. (Mike Williams)
相关文件: src/GvimExt/gvimext.def, src/GvimExt/gvimext_ming.def

Patch 7.3.223

问题:　　MingW cross compilation doesn't work with tiny features.
解决方案:　Move acp_to_enc(), enc_to_utf16() and utf16_to_enc() outside of "#ifdef CLIPBOARD". Fix typo in makefile.
相关文件:　src/Make_ming.mak, src/os_mswin.c

Patch 7.3.224
问题:　　Can't pass dict to sort function.
解决方案:　Add the optional {dict} argument to sort(). (ZyX)
相关文件:　runtime/doc/eval.txt, src/eval.c

Patch 7.3.225
问题:　　Using "\n" in a substitute inside ":s" does not result in a line break.
解决方案:　Change behavior inside vim_regexec_nl(). Add tests. (Motoya Kurotsu)
相关文件:　src/regexp.c, src/testdir/test79.in, src/testdir/test79.ok, src/testdir/test80.in, src/testdir/test80.ok, src/testdir/Makefile, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.226
问题:　　On a 64 bit system "syn sync fromstart" is very slow. (Bjorn Steinbrink)
解决方案:　Store the state when starting to parse from the first line.
相关文件:　src/syntax.c

Patch 7.3.227 (after 7.3.221)
问题:　　Mac OS doesn't have the linewise clipboard fix.
解决方案:　Also change the Mac OS file. (Bjorn Winckler)
相关文件:　src/os_macosx.m

Patch 7.3.228
问题:　　"2gj" does not always move to the correct position.
解决方案:　Get length of line after moving to a next line. (James Vega)
相关文件:　src/normal.c

Patch 7.3.229
问题:　　Using fork() makes gvim crash on Mac when build with CoreFoundation.
解决方案:　Disallow fork() when __APPLE__ is defined. (Hisashi T Fujinaka)
相关文件:　src/gui.c

Patch 7.3.230
问题:　　":wundo" and ":rundo" don't unescape their argument. (Aaron Thoma)
解决方案:　Use FILE1 instead of XFILE.
相关文件:　src/ex_cmds.h

Patch 7.3.231
问题:　　Runtime file patches failed.
解决方案:　Redo the patches made against the patched files instead of the files in the mercurial repository.
相关文件:　runtime/doc/indent.txt, runtime/doc/os_win32.txt

Patch 7.3.232
问题: Python doesn't compile without +multi_byte
解决方案: Use "latin1" when MULTI_BYTE is not defined.
相关文件: src/if_py_both.h

Patch 7.3.233
问题: ":scriptnames" and ":breaklist" show long file names.
解决方案: Shorten to use "~/" when possible. (Jean-Rene David)
相关文件: src/ex_cmds2.c

Patch 7.3.234
问题: With GTK menu may be popping down.
解决方案: Use event time instead of GDK_CURRENT_TIME. (Hong Xu)
相关文件: src/gui.c, src/gui.h, src/gui_gtk.c, src/gui_gtk_x11.c

Patch 7.3.235
问题: ";" gets stuck on a "t" command, it's not useful.
解决方案: Add the ';' flag in 'cpo'. (Christian Brabandt)
相关文件: runtime/doc/motion.txt, runtime/doc/options.txt, src/option.h,
src/search.c src/testdir/test81.in, src/testdir/test81.ok,
src/testdir/Makefile, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.236 (after 7.3.232)
问题: Python 3 doesn't compile without +multi_byte
解决方案: Use "latin1" when MULTI_BYTE is not defined. (lilydjwg)
相关文件: src/if_python3.c

Patch 7.3.237
问题: "filetype" completion doesn't work on Windows. (Yue Wu)
解决方案: Don't use a glob pattern for the directories, use a list of
directories. (Dominique Pelle)
相关文件: src/ex_getln.c

Patch 7.3.238
问题: Compiler warning for conversion.
解决方案: Add type cast. (Mike Williams)
相关文件: src/ex_getln.c

Patch 7.3.239
问题: Python corrects the cursor column without taking 'virtualedit'
into account. (lilydjwg)
解决方案: Call check_cursor_col_win().
相关文件: src/if_py_both.h, src/mbyte.c, src/misc2.c, src/normal.c,
src/proto/mbyte.pro, src/proto/misc2.pro

Patch 7.3.240
问题: External commands can't use pipes on MS-Windows.
解决方案: Implement pipes and use them when 'shelltemp' isn't set. (Vincent
Berthoux)
相关文件: src/eval.c, src/ex_cmds.c, src/misc2.c, src/os_unix.c,
src/os_win32.c, src/proto/misc2.pro, src/ui.c

Patch 7.3.241
问题:　　Using CTRL-R CTRL-W on the command line may insert only part of
　　the word.
解决方案:　Use the cursor position instead of assuming it is at the end of
　　the command. (Tyru)
相关文件:　src/ex_getln.c

Patch 7.3.242
问题:　　Illegal memory access in after_pathsep().
解决方案:　Check that the pointer is not at the start of the file name.
　　(Dominique Pelle)
相关文件:　src/misc2.c

Patch 7.3.243
问题:　　Illegal memory access in readline().
解决方案:　Swap the conditions. (Dominique Pelle)
相关文件:　src/eval.c

Patch 7.3.244
问题:　　MS-Windows: Build problem with old compiler. (John Beckett)
解决方案:　Only use HandleToLong() when available. (Mike Williams)
相关文件:　src/gui_w32.c

Patch 7.3.245
问题:　　Python 3.2 libraries not correctly detected.
解决方案:　Add the suffix to the library name. (Niclas Zeising)
相关文件:　src/auto/configure, src/configure.in

Patch 7.3.246 (after 7.3.235)
问题:　　Repeating "f4" in "4444" skips one 4.
解决方案:　Check the t_cmd flag. (Christian Brabandt)
相关文件:　src/search.c

Patch 7.3.247
问题:　　Running tests changes the users viminfo file. Test for patch
　　7.3.246 missing.
解决方案:　Add "nviminfo" to the 'viminfo' option. Include the test.
相关文件:　src/testdir/test78.in, src/testdir/test81.in

Patch 7.3.248
问题:　　PC Install instructions missing install instructions.
解决方案:　Step-by-step explanation. (Michael Soyka)
相关文件:　src/INSTALLpc.txt

Patch 7.3.249
问题:　　Wrong indenting for array initializer.
解决方案:　Detect '}' in a better way. (Lech Lorens)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.250
问题:　　Python: Errors in Unicode characters not handled nicely.
解决方案:　Add the surrogateescape error handler. (lilydjwg)
相关文件:　src/if_python3.c

Patch 7.3.251
问题: "gH<Del>" deletes the current line, except when it's the last line.
解决方案: Set the "include" flag to indicate the last line is to be deleted.
相关文件: src/normal.c, src/ops.c

Patch 7.3.252 (after 7.3.247)
问题: Tests fail. (David Northfield)
解决方案: Add missing update for .ok file.
相关文件: src/testdir/test81.ok

Patch 7.3.253
问题: "echo 'abc' > ''" returns 0 or 1, depending on 'ignorecase'. Checks in mb_strnicmp() for illegal and truncated bytes are wrong. Should not assume that byte length is equal before case folding.
解决方案: Add utf_safe_read_char_adv() and utf_strnicmp(). Add a test for this. (Ivan Krasilnikov)
相关文件: src/mbyte.c src/testdir/test82.in, src/testdir/test82.ok, src/testdir/Makefile, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.254
问题: The coladd field is not reset when setting the line number for a ":call" command.
解决方案: Reset it.
相关文件: src/eval.c

Patch 7.3.255
问题: When editing a file such as "File[2010-08-15].vim" an E16 error is given. (Manuel Stol)
解决方案: Don't give an error for failing to compile the regexp.
相关文件: src/ex_docmd.c, src/misc1.c, src/vim.h

Patch 7.3.256
问题: Javascript indenting not sufficiently tested.
解决方案: Add more tests. (Luc Deschenaux) Mark the lines that are indented wrong.
相关文件: src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.257
问题: Not all completions are available to user commands.
解决方案: Add "color", "compiler", "file_in_path" and "locale". (Dominique Pelle)
相关文件: src/ex_docmd.c, runtime/doc/map.txt

Patch 7.3.258
问题: MS-Windows: The edit with existing vim context menu entries can be unwanted.
解决方案: Let a registry entry disable them. (Jerome Vuarand)
相关文件: src/GvimExt/gvimext.cpp

Patch 7.3.259
问题: Equivalence classes only work for latin characters.
解决方案: Add the Unicode equivalence characters. (Dominique Pelle)
相关文件: runtime/doc/pattern.txt, src/regexp.c, src/testdir/test44.in, src/testdir/test44.ok

Patch 7.3.260
问题: CursorHold triggers on an incomplete mapping. (Will Gray)
解决方案: Don't trigger CursorHold when there is typeahead.
相关文件: src/fileio.c

Patch 7.3.261
问题: G++ error message erroneously recognized as error.
解决方案: Ignore "In file included from" line also when it ends in a colon. (Fernando Castillo)
相关文件: src/option.h

Patch 7.3.262
问题: Photon code style doesn't match Vim style.
解决方案: Clean up some of it. (Elias Diem)
相关文件: src/gui_photon.c

Patch 7.3.263
问题: Perl and Tcl have a few code style problems.
解决方案: Clean it up. (Elias Diem)
相关文件: src/if_perl.xs, src/if_tcl.c

Patch 7.3.264
问题: When the current directory name contains wildcard characters, such as "foo[with]bar", the tags file can't be found. (Jeremy Erickson)
解决方案: When searching for matching files also match without expanding wildcards. This is a bit of a hack.
相关文件: src/vim.h, src/misc1.c, src/misc2.c

Patch 7.3.265
问题: When storing a pattern in search history there is no proper check for the separator character.
解决方案: Pass the separator character to in_history(). (Taro Muraoka)
相关文件: src/ex_getln.c

Patch 7.3.266
问题: In gvim with iBus typing space in Insert mode doesn't work.
解决方案: Clear xim_expected_char after checking it.
相关文件: src/mbyte.c

Patch 7.3.267
问题: Ruby on Mac OS X 10.7 may crash.
解决方案: Avoid alloc(0). (Bjorn Winckler)
相关文件: src/if_ruby.c

Patch 7.3.268
问题: Vim freezes when executing an external command with zsh.
解决方案: Use O_NOCTTY both in the master and slave. (Bjorn Winckler)

相关文件: src/os_unix.c

Patch 7.3.269
问题: 'shellcmdflag' only works with one flag.
解决方案: Split into multiple arguments. (Gary Johnson)
相关文件: src/os_unix.c

Patch 7.3.270
问题: Illegal memory access.
解决方案: Swap conditions. (Dominique Pelle)
相关文件: src/ops.c

Patch 7.3.271
问题: Code not following Vim coding style.
解决方案: Fix the style. (Elias Diem)
相关文件: src/gui_photon.c

Patch 7.3.272
问题: ":put =list" does not add an empty line for a trailing empty
item.
解决方案: Add a trailing NL when turning a list into a string.
相关文件: src/eval.c

Patch 7.3.273
问题: A BOM in an error file is seen as text. (Aleksey Baibarin)
解决方案: Remove the BOM from the text before evaluating. (idea by Christian
Brabandt)
相关文件: src/quickfix.c, src/mbyte.c, src/proto/mbyte.pro,
src/testdir/test10.in

Patch 7.3.274
问题: With concealed characters tabs do not have the right size.
解决方案: Use VCOL_HLC instead of vcol. (Eiichi Sato)
相关文件: src/screen.c

Patch 7.3.275
问题: MS-Windows: When using a black background some screen updates
cause the window to flicker.
解决方案: Add WS_CLIPCHILDREN to CreateWindow(). (René Aguirre)
相关文件: src/gui_w32.c

Patch 7.3.276
问题: GvimExt sets $LANG in the wrong way.
解决方案: Save the environment and use it for gvim. (Yasuhiro Matsumoto)
相关文件: src/GvimExt/gvimext.cpp

Patch 7.3.277
问题: MS-Windows: some characters do not show in dialogs.
解决方案: Use the wide methods when available. (Yanwei Jia)
相关文件: src/gui_w32.c, src/gui_w48.c, src/os_mswin.c, src/os_win32.c,
src/os_win32.h

Patch 7.3.278
问题: Passing the file name to open in VisVim doesn't work.

解决方案: Adjust the index and check for end of buffer. (Jiri Sedlak)
相关文件: src/VisVim/Commands.cpp

Patch 7.3.279
问题: With GTK, when gvim is full-screen and a tab is opened and using a specific monitor configuration the window is too big.
解决方案: Adjust the window size like on MS-Windows. (Yukihiro Nakadaira)
相关文件: src/gui.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro

Patch 7.3.280
问题: ":lmake" does not update the quickfix window title.
解决方案: Update the title. (Lech Lorens)
相关文件: src/quickfix.c, src/testdir/test10.in, src/testdir/test10.ok

Patch 7.3.281
问题: After using "expand('%:8')" the buffer name is changed.
解决方案: Make a copy of the file name before shortening it.
相关文件: src/eval.c

Patch 7.3.282
问题: When using input() and :echo in a loop the displayed text is incorrect. (Benjamin Fritz)
解决方案: Only restore the cursor position when there is a command line. (Ben Schmidt)
相关文件: src/ex_getln.c

Patch 7.3.283
问题: An expression mapping with a multi-byte character containing a 0x80 byte gets messed up. (ZyX)
解决方案: Unescape the expression before evaluating it (Yukihiro Nakadaira)
相关文件: src/getchar.c

Patch 7.3.284
问题: The str2special() function doesn't handle multi-byte characters properly.
解决方案: Recognize multi-byte characters. (partly by Vladimir Vichniakov)
相关文件: src/getchar.c, src/message.c, src/misc2.c

Patch 7.3.285 (after 7.3.284)
问题: Mapping <Char-123> no longer works.
解决方案: Properly check for "char-". Add a test for it.
相关文件: src/misc2.c, src/testdir/test75.in, src/testdir/test75.ok

Patch 7.3.286
问题: Crash when using "zd" on a large number of folds. (Sam King)
解决方案: Recompute pointer after reallocating array. Move fewer entries when making room.
相关文件: src/fold.c

Patch 7.3.287
问题: Can't compile with MSVC and tiny options.
解决方案: Move variables and #ifdefs. (Sergey Khorev)
相关文件: src/os_win32.c

Patch 7.3.288
问题: has('python') may give an error message for not being able to load the library after using python3.
解决方案: Only give the error when the verbose argument is true.
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.289
问题: Complete function isn't called when the leader changed.
解决方案: Call ins_compl_restart() when the leader changed. (Taro Muraoka)
相关文件: src/edit.c

Patch 7.3.290
问题: When a BufWriteCmd autocommand resets 'modified' this doesn't change older buffer states to be marked as 'modified' like ":write" does. (Yukihiro Nakadaira)
解决方案: When the BufWriteCmd resets 'modified' then adjust the undo information like ":write" does.
相关文件: src/fileio.c

Patch 7.3.291
问题: Configure doesn't work properly with Python3.
解决方案: Put -ldl before $LDFLAGS. Add PY3_NO_RTLD_GLOBAL. (Roland Puntaier)
相关文件: src/config.h.in, src/auto/configure, src/configure.in

Patch 7.3.292
问题: Crash when using fold markers and selecting a visual block that includes a folded line and goes to end of line. (Sam Lidder)
解决方案: Check for the column to be MAXCOL. (James Vega)
相关文件: src/screen.c

Patch 7.3.293
问题: MSVC compiler has a problem with non-ASCII characters.
解决方案: Avoid non-ASCII characters. (Hong Xu)
相关文件: src/ascii.h, src/spell.c

Patch 7.3.294 (after 7.3.289)
问题: Patch 289 causes more problems than it solves.
解决方案: Revert the patch until a better solution is found.
相关文件: src/edit.c

Patch 7.3.295
问题: When filtering text with an external command Vim may not read all the output.
解决方案: When select() is interrupted loop and try again. (James Vega)
相关文件: src/os_unix.c

Patch 7.3.296
问题: When writing to an external command a zombie process may be left behind.
解决方案: Wait on the process. (James Vega)
相关文件: src/os_unix.c

Patch 7.3.297

问题:　　Can't load Perl 5.14 dynamically.
解决方案:　Add code in #ifdefs. (Charles Cooper)
相关文件:　if_perl.xs

Patch 7.3.298
问题:　　Built-in colors are different from rgb.txt.
解决方案:　Adjust the color values. (Benjamin Haskell)
相关文件:　src/gui_photon.c, src/gui_w48.c

Patch 7.3.299
问题:　　Source code not in Vim style.
解决方案:　Adjust the style. (Elias Diem)
相关文件:　src/gui_photon.c

Patch 7.3.300
问题:　　Python doesn't parse multi-byte argument correctly.
解决方案:　Use "t" instead of "s". (lilydjwg)
相关文件:　src/if_py_both.h

Patch 7.3.301
问题:　　When 'smartindent' and 'copyindent' are set a Tab is used even
　　though 'expandtab' is set.
解决方案:　Do not insert Tabs. Add a test. (Christian Brabandt)
相关文件:　src/misc1.c, src/testdir/test19.in, src/testdir/test19.ok

Patch 7.3.302 (after 7.3.301)
问题:　　Test 19 fails without 'smartindent' and +eval.
解决方案:　Don't use ":exe". Source small.vim.
相关文件:　src/testdir/test19.in

Patch 7.3.303 (after 7.3.296)
问题:　　Compilation error.
解决方案:　Correct return type from int to pid_t. (Danek Duvall)
相关文件:　src/os_unix.c

Patch 7.3.304
问题:　　Strawberry Perl doesn't work on MS-Windows.
解决方案:　Use xsubpp if needed. (Yasuhiro Matsumoto)
相关文件:　src/Make_ming.mak, src/Make_mvc.mak

Patch 7.3.305
问题:　　Auto-loading a function while editing the command line causes
　　scrolling up the display.
解决方案:　Don't set msg_scroll when defining a function and the user is not
　　typing. (Yasuhiro Matsumoto)
相关文件:　src/eval.c

Patch 7.3.306
问题:　　When closing a window there is a chance that deleting a scrollbar
　　triggers a GUI resize, which uses the window while it is not in a
　　valid state.
解决方案:　Set the buffer pointer to NULL to be able to detect the invalid
　　situation. Fix a few places that used the buffer pointer
　　incorrectly.

相关文件: src/buffer.c, src/ex_cmds.c, src/term.c, src/window.c

Patch 7.3.307
问题: Python 3 doesn't support slice assignment.
解决方案: Implement slices. (Brett Overesch, Roland Puntaier)
相关文件: src/if_python3.c

Patch 7.3.308
问题: Writing to 'verbosefile' has problems, e.g. for :highlight.
解决方案: Do not use a separate verbose_write() function but write with the same code that does redirecting. (Yasuhiro Matsumoto)
相关文件: src/message.c

Patch 7.3.309 (after 7.3.307)
问题: Warnings for pointer types.
解决方案: Change PySliceObject to PyObject.
相关文件: src/if_python3.c

Patch 7.3.310
问题: Code not following Vim style.
解决方案: Fix the style. (Elias Diem)
相关文件: src/gui_photon.c

Patch 7.3.311 (replaces 7.3.289)
问题: Complete function isn't called when the leader changed.
解决方案: Allow the complete function to return a dictionary with a flag that indicates ins_compl_restart() is to be called when the leader changes. (Taro Muraoka)
相关文件: runtime/insert.txt, src/edit.c, src/eval.c, src/proto/eval.pro

Patch 7.3.312 (after 7.3.306)
问题: Can't compile with tiny features.
解决方案: Add #ifdef around win_valid().
相关文件: src/buffer.c

Patch 7.3.313 (after 7.3.307)
问题: One more warning when compiling with dynamic Python 3.
解决方案: Change PySliceObject to PyObject.
相关文件: src/if_python3.c

Patch 7.3.314 (after 7.3.304)
问题: Missing parenthesis.
解决方案: Add it. (Benjamin R. Haskell)
相关文件: src/Make_mvc.mak

Patch 7.3.315
问题: Opening a window before forking causes problems for GTK.
解决方案: Fork first, create the window in the child and report back to the parent process whether it worked. If successful the parent exits, if unsuccessful the child exits and the parent continues in the terminal. (Tim Starling)
相关文件: src/gui.c

Patch 7.3.316 (after 7.3.306)

问题:　　Crash when 'colorcolumn' is set and closing buffer.
解决方案:　Check for w_buffer to be NULL. (Yasuhiro Matsumoto)
相关文件:　src/option.c

Patch 7.3.317
问题:　　Calling debug.debug() in Lua may cause Vim to hang.
解决方案:　Add a better debug method. (Rob Hoelz, Luis Carvalho)
相关文件:　src/if_lua.c

Patch 7.3.318
问题:　　"C" on the last line deletes that line if it's blank.
解决方案:　Only delete the last line for a delete operation. (James Vega)
相关文件:　src/ops.c

Patch 7.3.319 (after 7.3.311)
问题:　　Redobuff doesn't always include changes of the completion leader.
解决方案:　Insert backspaces as needed. (idea by Taro Muraoka)
相关文件:　src/edit.c

Patch 7.3.320
问题:　　When a 0xa0 character is in a sourced file the error message for
　　unrecognized command does not show the problem.
解决方案:　Display 0xa0 as <a0>.
相关文件:　src/ex_docmd.c

Patch 7.3.321
问题:　　Code not following Vim style.
解决方案:　Fix the style. (Elias Diem)
相关文件:　src/os_qnx.c

Patch 7.3.322
问题:　　#ifdef for PDP_RETVAL doesn't work, INT_PTR can be a typedef.
解决方案:　Check the MSC version and 64 bit flags. (Sergiu Dotenco)
相关文件:　src/os_mswin.c

Patch 7.3.323
问题:　　The default 'errorformat' does not ignore some "included from"
　　lines.
解决方案:　Add a few more patterns. (Ben Boeckel)
相关文件:　src/option.h

Patch 7.3.324 (after 7.3.237)
问题:　　Completion for ":compiler" shows color scheme names.
解决方案:　Fix the directory name. (James Vega)
相关文件:　src/ex_getln.c

Patch 7.3.325
问题:　　A duplicated function argument gives an internal error.
解决方案:　Give a proper error message. (based on patch by Tyru)
相关文件:　src/eval.c

Patch 7.3.326
问题:　　MingW 4.6 no longer supports the -mno-cygwin option.
解决方案:　Split the Cygwin and MingW makefiles. (Matsushita Shougo)

相关文件: src/GvimExt/Make_cyg.mak, src/GvimExt/Make_ming.mak,
           src/Make_cyg.mak, src/Make_ming.mak, src/xxd/Make_ming.mak,
           Filelist

Patch 7.3.327
问题: When jumping to a help tag a closed fold doesn't open.
解决方案: Save and restore KeyTyped. (Yasuhiro Matsumoto)
相关文件: src/ex_cmds.c

Patch 7.3.328
问题: When command line wraps the cursor may be displayed wrong when
           there are multi-byte characters.
解决方案: Position the cursor before drawing the text. (Yasuhiro Matsumoto)
相关文件: src/ex_getln.c

Patch 7.3.329
问题: When skipping over code from ":for" to ":endfor" get an error for
           calling a dict function. (Yasuhiro Matsumoto)
解决方案: Ignore errors when skipping over :call command.
相关文件: src/ex_docmd.c, src/eval.c

Patch 7.3.330
问题: When longjmp() is invoked if the X server gives an error the state
           is not properly restored.
解决方案: Reset vgetc_busy. (Yukihiro Nakadaira)
相关文件: src/main.c

Patch 7.3.331
问题: "vit" selects wrong text when a tag name starts with the same text
           as an outer tag name. (Ben Fritz)
解决方案: Add "\>" to the pattern to check for word boundary.
相关文件: src/search.c

Patch 7.3.332 (after 7.3.202)
问题: Indent after "public:" is not increased in C++ code. (Lech Lorens)
解决方案: Check for namespace after the regular checks. (partly by Martin
           Gieseking)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.333
问题: Using "." to repeat a Visual delete counts the size in bytes, not
           characters. (Connor Lane Smith)
解决方案: Store the virtual column numbers instead of byte positions.
相关文件: src/normal.c

Patch 7.3.334
问题: Latest MingW about XSUBPP referencing itself. (Gongqian Li)
解决方案: Rename the first use to XSUBPPTRY.
相关文件: src/Make_ming.mak

Patch 7.3.335
问题: When 'imdisable' is reset from an autocommand in Insert mode it
           doesn't take effect.
解决方案: Call im_set_active() in Insert mode. (Taro Muraoka)

相关文件: src/option.c

Patch 7.3.336
问题: When a tags file specifies an encoding different from 'enc' it may hang and using a pattern doesn't work.
解决方案: Convert the whole line. Continue reading the header after the SORT tag. Add test83. (Yukihiro Nakadaira)
相关文件: src/tag.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test83-tags2, src/testdir/test83-tags3, src/testdir/test83.in, src/testdir/test83.ok

Patch 7.3.337 (after 7.3.295)
问题: Screen doesn't update after resizing the xterm until a character is typed.
解决方案: When the select call is interrupted check do_resize. (Taylor Hedberg)
相关文件: src/os_unix.c

Patch 7.3.338
问题: Using getchar() in an expression mapping doesn't work well.
解决方案: Don't save and restore the typeahead. (James Vega)
相关文件: src/getchar.c, src/testdir/test34.ok

Patch 7.3.339
问题: "make shadow" doesn't link all test files.
解决方案: Add a line in Makefile and Filelist.
相关文件: src/Makefile, Filelist

Patch 7.3.340
问题: When 'verbosefile' is set ftplugof.vim can give an error.
解决方案: Only remove filetypeplugin autocommands when they exist. (Yasuhiro Matsumoto)
相关文件: runtime/ftplugof.vim

Patch 7.3.341
问题: Local help files are only listed in help.txt, not in translated help files.
解决方案: Also find translated help files. (Yasuhiro Matsumoto)
相关文件: src/ex_cmds.c

Patch 7.3.342
问题: Code not in Vim style.
解决方案: Fix the style. (Elias Diem)
相关文件: src/os_amiga.c, src/os_mac_conv.c, src/os_win16.c

Patch 7.3.343
问题: No mouse support for urxvt.
解决方案: Implement urxvt mouse support, also for > 252 columns. (Yiding Jia)
相关文件: src/feature.h, src/keymap.h, src/option.h, src/os_unix.c, src/term.c, src/version.c

Patch 7.3.344
问题: Problem with GUI startup related to XInitThreads.
解决方案: Use read() and write() instead of fputs() and fread(). (James Vega)
相关文件: src/gui.c

Patch 7.3.345
问题: When switching language with ":lang" the window title doesn't change until later.
解决方案: Update the window title right away. (Dominique Pelle)
相关文件: src/ex_cmds2.c

Patch 7.3.346
问题: It's hard to test netbeans commands.
解决方案: Process netbeans commands after :sleep. (Xavier de Gaye)
相关文件: runtime/doc/netbeans.txt, src/ex_docmd.c, src/netbeans.c

Patch 7.3.347
问题: When dropping text from a browser on Vim it receives HTML even though "html" is excluded from 'clipboard'. (Andrei Avk)
解决方案: Fix the condition for TARGET_HTML.
相关文件: src/gui_gtk_x11.c

Patch 7.3.348
问题: "call range(1, 947948399)" causes a crash. (ZyX)
解决方案: Avoid a loop in the out of memory message.
相关文件: src/misc2.c

Patch 7.3.349
问题: When running out of memory during startup trying to open a swapfile will loop forever.
解决方案: Let findswapname() set dirp to NULL if out of memory.
相关文件: src/memline.c

Patch 7.3.350
问题: Block of code after ":lua << EOF" may not work. (Paul Isambert)
解决方案: Recognize the ":lua" command, skip to EOF.
相关文件: src/eval.c

Patch 7.3.351
问题: Text formatting uses start of insert position when it should not. (Peter Wagenaar)
解决方案: Do not use Insstart when intentionally formatting.
相关文件: src/edit.c

Patch 7.3.352
问题: When completing methods dict functions and script-local functions get in the way.
解决方案: Sort function names starting with "<" to the end. (Yasuhiro Matsumoto)
相关文件: src/ex_getln.c

Patch 7.3.353 (after 7.3.343)
问题: Missing part of the urxvt patch.

解决方案: Add the change in term.c
相关文件: src/term.c

Patch 7.3.354
问题: ":set backspace+=eol" doesn't work when 'backspace' has a
backwards compatible value of 2.
解决方案: Convert the number to a string. (Hirohito Higashi)
相关文件: src/option.c

Patch 7.3.355
问题: GTK warnings when using netrw.vim. (Ivan Krasilnikov)
解决方案: Do not remove the beval event handler twice.
相关文件: src/option.c

Patch 7.3.356
问题: Using "o" with 'cindent' set may freeze Vim. (lolilolicon)
解决方案: Skip over {} correctly. (Hari G)
相关文件: src/misc1.c

Patch 7.3.357
问题: Compiler warning in MS-Windows console build.
解决方案: Adjust return type of PrintHookProc(). (Mike Williams)
相关文件: src/os_mswin.c

Patch 7.3.358 (after 7.3.353)
问题: Mouse support doesn't work properly.
解决方案: Add HMT_URXVT. (lilydjwg, James McCoy)
相关文件: src/term.c

Patch 7.3.359
问题: Command line completion shows dict functions.
解决方案: Skip dict functions for completion. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.3.360
问题: Interrupting the load of an autoload function may cause a crash.
解决方案: Do not use the hashitem when not valid. (Yukihiro Nakadaira)
相关文件: src/eval.c

Patch 7.3.361
问题: Accessing memory after it is freed when EXITFREE is defined.
解决方案: Don't access curwin when firstwin is NULL. (Dominique Pelle)
相关文件: src/buffer.c

Patch 7.3.362
问题: ml_get error when using ":g" with folded lines.
解决方案: Adjust the line number for changed_lines(). (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.3.363
问题: C indenting is wrong after #endif followed by a semicolon.
解决方案: Add special handling for a semicolon in a line by itself. (Lech
Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.364 (after 7.3.353)
问题: Can't compile on HP-UX. (John Marriott)
解决方案: Only use TTYM_URXVT when it is defined.
相关文件: src/term.c

Patch 7.3.365
问题: Crash when using a large Unicode character in a file that has syntax highlighting. (ngollan)
解决方案: Check for going past the end of the utf tables. (Dominique Pelle)
相关文件: src/mbyte.c

Patch 7.3.366
问题: A tags file with an extremely long name causes errors.
解决方案: Ignore tags that are too long. (Arno Renevier)
相关文件: src/tag.c

Patch 7.3.367
问题: :wundo and :rundo use a wrong checksum.
解决方案: Include the last line when computing the hash. (Christian Brabandt)
相关文件: src/undo.c

Patch 7.3.368
问题: Gcc complains about redefining _FORTIFY_SOURCE.
解决方案: Undefine it before redefining it.
相关文件: src/Makefile, src/configure.in, src/auto/configure

Patch 7.3.369
问题: When compiled with Gnome get an error message when using --help.
解决方案: Don't fork. (Ivan Krasilnikov)
相关文件: src/main.c

Patch 7.3.370
问题: Compiler warns for unused variable in Lua interface.
解决方案: Remove the variable.
相关文件: src/if_lua.c

Patch 7.3.371
问题: Crash in autocomplete. (Greg Weber)
解决方案: Check not going over allocated buffer size.
相关文件: src/misc2.c

Patch 7.3.372
问题: When using a command line mapping to <Up> with file name completion to go one directory up, 'wildchar' is inserted. (Yasuhiro Matsumoto)
解决方案: Set the KeyTyped flag.
相关文件: src/ex_getln.c

Patch 7.3.373 (after 7.3.366)
问题: A tags file with an extremely long name may cause an infinite loop.
解决方案: When encountering a long name switch to linear search.
相关文件: src/tag.c

Patch 7.3.374
问题: ++encoding does not work properly.
解决方案: Recognize ++encoding before ++enc. (Charles Cooper)
相关文件: src/ex_docmd.c

Patch 7.3.375
问题: Duplicate return statement.
解决方案: Remove the superfluous one. (Dominique Pelle)
相关文件: src/gui_mac.c

Patch 7.3.376
问题: Win32: Toolbar repainting does not work when the mouse pointer
            hovers over a button.
解决方案: Call DefWindowProc() when not handling an event. (Sergiu Dotenco)
相关文件: src/gui_w32.c

Patch 7.3.377
问题: No support for bitwise AND, OR, XOR and invert.
解决方案: Add and(), or(), invert() and xor() functions.
相关文件: src/eval.c, src/testdir/test49.in, src/testdir/test65.in,
            src/testdir/test65.ok, runtime/doc/eval.txt

Patch 7.3.378
问题: When cross-compiling the check for uint32_t fails.
解决方案: Only give a warning message. (Maksim Melnikau)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.379
问题: C-indenting wrong for static enum.
解决方案: Skip over "static". (Lech Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.380
问题: C-indenting wrong for a function header.
解决方案: Skip to the start paren. (Lech Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.381
问题: Configure silently skips interfaces that won't work.
解决方案: Add the --enable-fail_if_missing argument. (Shlomi Fish)
相关文件: src/Makefile, src/configure.in, src/auto/configure

Patch 7.3.382 (after 7.3.376)
问题: IME characters are inserted twice.
解决方案: Do not call DefWindowProc() if the event was handled. (Yasuhiro
            Matsumoto)
相关文件: src/gui_w32.c

Patch 7.3.383
问题: For EBCDIC pound sign is defined as 't'.
解决方案: Correctly define POUND.
相关文件: src/ascii.h

Patch 7.3.384

问题:　　Mapping CTRL-K in Insert mode breaks CTRL-X CTRL-K for dictionary completion.
解决方案:　Add CTRL-K to the list of recognized keys. (James McCoy)
相关文件:　src/edit.c

Patch 7.3.385
问题:　　When using an expression mapping on the command line the cursor ends up in the wrong place. (Yasuhiro Matsumoto)
解决方案:　Save and restore msg_col and msg_row when evaluating the expression.
相关文件:　src/getchar.

Patch 7.3.386
问题:　　Test 83 fails when iconv does not support cp932. (raf)
解决方案:　Test if conversion works. (Yukihiro Nakadaira)
相关文件:　src/testdir/test83.in

Patch 7.3.387 (after 7.3.386)
问题:　　Test 83 may fail for some encodings.
解决方案:　Set 'encoding' to utf-8 earlier.
相关文件:　src/testdir/test83.in

Patch 7.3.388
问题:　　Crash on exit when EXITFREE is defined and using tiny features.
解决方案:　Check for NULL window pointer. (Dominique Pelle)
相关文件:　src/buffer.c

Patch 7.3.389
问题:　　After typing at a prompt the "MORE" message appears too soon.
解决方案:　reset lines_left in msg_end_prompt(). (Eswald)
相关文件:　src/message.c

Patch 7.3.390
问题:　　Using NULL buffer pointer in a window.
解决方案:　Check for w_buffer being NULL in more places. (Bjorn Winckler)
相关文件:　src/ex_cmds.c, src/quickfix.c, src/window.c

Patch 7.3.391
问题:　　Can't check if the XPM_W32 feature is enabled.
解决方案:　Add xpm_w32 to the list of features. (kat)
相关文件:　src/eval.c

Patch 7.3.392
问题:　　When setting 'undofile' while the file is already loaded but unchanged, try reading the undo file. (Andy Wokula)
解决方案:　Compute a checksum of the text when 'undofile' is set. (Christian Brabandt)
相关文件:　src/option.c, src/testdir/test72.in, src/testdir/test72.ok

Patch 7.3.393
问题:　　Win32: When resizing Vim it is always moved to the primary monitor if the secondary monitor is on the left.
解决方案:　Use the nearest monitor. (Yukihiro Nakadaira)
相关文件:　src/gui_w32.c

Patch 7.3.394
问题: When placing a mark while starting up a screen redraw messes up the screen. (lith)
解决方案: Don't redraw while still starting up. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.3.395 (after 7.3.251)
问题: "dv?bar" in the last line deletes too much and breaks undo.
解决方案: Only adjust the cursor position when it's after the last line of the buffer. Add a test. (Christian Brabandt)
相关文件: src/ops.c, src/testdir/test43.in, src/testdir/test43.ok

Patch 7.3.396
问题: After forcing an operator to be characterwise it can still become linewise when spanning whole lines.
解决方案: Don't make the operator linewise when motion_force was set. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.3.397
问题: ":helpgrep" does not work properly when 'encoding' is not utf-8 or latin1.
解决方案: Convert non-ascii lines to 'encoding'. (Yasuhiro Matsumoto)
相关文件: src/quickfix.c, src/spell.c, src/misc2.c, src/proto/misc2.pro

Patch 7.3.398
问题: When creating more than 10 location lists and adding items one by one a previous location may be used. (Audrius Kaukauskas)
解决方案: Clear the location list completely when adding the tenth one.
相关文件: src/quickfix.c

Patch 7.3.399
问题: ":cd" doesn't work when the path contains wildcards. (Yukihiro Nakadaira)
解决方案: Ignore wildcard errors when the EW_NOTWILD flag is used.
相关文件: src/misc1.c

Patch 7.3.400
问题: Compiler warnings for shadowed variables.
解决方案: Remove or rename the variables.
相关文件: src/charset.c, src/digraph.c, src/edit.c, src/eval.c, src/fold.c, src/getchar.c, src/message.c, src/misc2.c, src/move.c, src/netbeans.c, src/option.c, src/os_unix.c, src/screen.c, src/search.c, src/spell.c, src/syntax.c, src/tag.c, src/window.c

Patch 7.3.401
问题: A couple more shadowed variables.
解决方案: Rename the variables.
相关文件: src/netbeans.c

Patch 7.3.402
问题: When jumping to the first error a line of the buffer is sometimes redrawn on top of the list of errors.

解决方案: Do not call update_topline_redraw() if the display was scrolled
           up.
相关文件: src/quickfix.c

Patch 7.3.403
问题: ":helpgrep" does not trigger QuickFixCmd* autocommands.
解决方案: Trigger the autocommands. (Christian Brabandt)
相关文件: src/quickfix.c

Patch 7.3.404
问题: When a complete function uses refresh "always" redo will not work
           properly.
解决方案: Do not reset compl_leader when compl_opt_refresh_always is set.
           (Yasuhiro Matsumoto)
相关文件: src/edit.c

Patch 7.3.405
问题: When xterm gets back the function keys it may delete the urxvt
           mouse termcap code.
解决方案: Check for the whole code, not just the start. (Egmont Koblinger)
相关文件: src/keymap.h, src/misc2.c, src/term.c

Patch 7.3.406
问题: Multi-byte characters in b:browsefilter are not handled correctly.
解决方案: First use convert_filter() normally and then convert to wide
           characters. (Taro Muraoka)
相关文件: src/gui_w48.c

Patch 7.3.407
问题: ":12verbose call F()" may duplicate text while trying to truncate.
           (Thinca)
解决方案: Only truncate when there is not enough room. Also check the byte
           length of the buffer.
相关文件: src/buffer.c, src/eval.c, src/ex_getln.c, src/message.c,
           src/proto/message.pro

Patch 7.3.408 (after 7.3.406)
问题: Missing declaration.
解决方案: Add the declaration. (John Marriott)
相关文件: src/gui_w48.c

Patch 7.3.409
问题: The license in pty.c is unclear.
解决方案: Add a comment about the license.
相关文件: src/pty.c

Patch 7.3.410
问题: Compiler error for // comment. (Joachim Schmitz)
解决方案: Turn into /* comment */.
相关文件: src/message.c

Patch 7.3.411
问题: Pasting in Visual mode using the "" register does not work. (John
           Beckett)

解决方案: Detect that the write is overwriting the pasted register.
           (Christian Brabandt)
相关文件: src/normal.c

Patch 7.3.412
问题: Storing a float in a session file has an additional '&'.
解决方案: Remove the '&'. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.3.413
问题: Build warnings on MS-Windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/ex_getln.c, src/message.c, src/term.c

Patch 7.3.414
问题: Using CTRL-A on "000" drops the leading zero, while on "001" it
           doesn't.
解决方案: Detect "000" as an octal number. (James McCoy)
相关文件: src/charset.c

Patch 7.3.415 (after 7.3.359)
问题: Completion of functions stops once a dictionary is encountered.
           (James McCoy)
解决方案: Return an empty string instead of NULL.
相关文件: src/eval.c

Patch 7.3.416 (after 7.3.415)
问题: Compiler warning for wrong pointer.
解决方案: Add type cast.
相关文件: src/eval.c

Patch 7.3.417 (after 7.3.395)
问题: Test 43 fails with a tiny build.
解决方案: Only run test 43 with at least a small build.
相关文件: src/testdir/test43.in

Patch 7.3.418
问题: When a user complete function returns -1 an error message is
           given.
解决方案: When -2 is returned stop completion silently. (Yasuhiro Matsumoto)
相关文件: src/edit.

Patch 7.3.419
问题: DBCS encoding in a user command does not always work.
解决方案: Skip over DBCS characters. (Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c

Patch 7.3.420
问题: "it" and "at" don't work properly with a dash in the tag name.
解决方案: Require a space to match the tag name. (Christian Brabandt)
相关文件: src/search.c

Patch 7.3.421
问题: Get E832 when setting 'undofile' in vimrc and there is a file to

be edited on the command line. (Toothpik)
解决方案: Do not try reading the undo file for a file that wasn't loaded.
相关文件: src/option.c

Patch 7.3.422
问题: Python 3 does not have __members__.
解决方案: Add "name" and "number" in another way. (lilydjwg)
相关文件: src/if_py_both.h, src/if_python3.c

Patch 7.3.423
问题: Small mistakes in comments, proto and indent.
解决方案: Fix the mistakes.
相关文件: src/ex_cmds2.c, src/structs.h, src/ui.c, src/proto/ex_docmd.pro

Patch 7.3.424
问题: Win16 version missing some functions.
解决方案: Add #defines for the functions.
相关文件: src/gui_w16.c

Patch 7.3.425 (after 7.3.265)
问题: Search history lines are duplicated. (Edwin Steiner)
解决方案: Convert separator character from space to NUL.
相关文件: src/ex_getln.c

Patch 7.3.426
问题: With '$' in 'cpoptions' the $ is not displayed in the first column.
解决方案: Use -1 instead of 0 as a special value. (Hideki Eiraku and Hirohito Higashi)
相关文件: src/edit.c, src/globals.h, src/move.c, src/screen.c, src/search.c

Patch 7.3.427
问题: readfile() can be slow with long lines.
解决方案: Use realloc() instead of alloc(). (John Little)
相关文件: src/eval.c

Patch 7.3.428
问题: Win32: an xpm file without a mask crashes Vim.
解决方案: Fail when the mask is missing. (Dave Bodenstab)
相关文件: src/xpm_w32.c

Patch 7.3.429
问题: When 'cpoptions' includes "E" "c0" in the first column is an error. The redo register is then set to the erroneous command.
解决方案: Do not set the redo register if the command fails because of an empty region. (Hideki Eiraku)
相关文件: src/getchar.c, src/normal.c, src/proto/getchar.pro

Patch 7.3.430
问题: When a custom filetype detection uses "augroup END" the conf fileytpe detection does not have the filetypedetect group.
解决方案: Always end the group and include filetypedetect in the conf autocommand. (Lech Lorens)
相关文件: runtime/filetype.vim

Patch 7.3.431
问题: Fetching a key at a prompt may be confused by escape sequences.
Especially when getting a prompt at a VimEnter autocommand.
(Alex Efros)
解决方案: Properly handle escape sequences deleted by check_termcode().
相关文件: src/getchar.c, src/misc1.c, src/term.c, src/proto/term.pro

Patch 7.3.432
问题: ACLs are not supported for ZFS or NFSv4 on Solaris.
解决方案: Add configure check and code. (Danek Duvall)
相关文件: src/configure.in, src/auto/configure, src/config.h.in,
src/os_unix.c

Patch 7.3.433
问题: Using continued lines in a Vim script can be slow.
解决方案: Instead of reallocating for every line use a growarray. (Yasuhiro
Matsumoto)
相关文件: src/ex_cmds2.c

Patch 7.3.434
问题: Using join() can be slow.
解决方案: Compute the size of the result before allocation to avoid a lot of
allocations and copies. (Taro Muraoka)
相关文件: src/eval.c

Patch 7.3.435
问题: Compiler warning for unused variable.
解决方案: Move the variable inside #ifdef.
相关文件: src/ex_cmds2.c

Patch 7.3.436
问题: Compiler warnings for types on Windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/eval.c

Patch 7.3.437
问题: Continue looping inside FOR_ALL_TAB_WINDOWS even when already done.
解决方案: Use goto instead of break. (Hirohito Higashi)
相关文件: src/fileio.c, src/globals.h

Patch 7.3.438
问题: There is no way to avoid ":doautoall" reading modelines.
解决方案: Add the <nomodeline> argument. Adjust documentation.
相关文件: src/fileio.c, runtime/doc/autocmd.txt

Patch 7.3.439
问题: Compiler warnings to size casts in Perl interface.
解决方案: Use XS macros. (James McCoy)
相关文件: src/if_perl.xs, src/typemap

Patch 7.3.440
问题: Vim does not support UTF8_STRING for the X selection.
解决方案: Add UTF8_STRING atom support. (Alex Efros) Use it only when

'encoding' is set to Unicode.
相关文件: src/ui.c

Patch 7.3.441
问题: Newer versions of MzScheme (Racket) require earlier (trampolined) initialisation.
解决方案: Call mzscheme_main() early in main(). (Sergey Khorev)
相关文件: src/Make_mvc.mak, src/if_mzsch.c, src/main.c, src/proto/if_mzsch.pro

Patch 7.3.442 (after 7.3.438)
问题: Still read modelines for ":doautocmd".
解决方案: Move check for <nomodeline> to separate function.
相关文件: src/fileio.c, src/ex_docmd.c, src/proto/fileio.pro, runtime/doc/autocmd.txt

Patch 7.3.443
问题: MS-Windows: 'shcf' and 'shellxquote' defaults are not very good.
解决方案: Make a better guess when 'shell' is set to "cmd.exe". (Ben Fritz)
相关文件: src/option.c, runtime/doc/options.txt

Patch 7.3.444
问题: ":all!" and ":sall!" give error E477, even though the documentation says these are valid commands.
解决方案: Support the exclamation mark. (Hirohito Higashi)
相关文件: src/ex_cmds.h, src/testdir/test31.in, src/testdir/test31.ok

Patch 7.3.445 (after 7.3.443)
问题: Can't properly escape commands for cmd.exe.
解决方案: Default 'shellxquote' to '('. Append ')' to make '(command)'. No need to use "/s" for 'shellcmdflag'.
相关文件: src/misc2.c, src/option.c, src/os_win32.c

Patch 7.3.446 (after 7.3.445)
问题: Win32: External commands with special characters don't work.
解决方案: Add the 'shellxescape' option.
相关文件: src/misc2.c, src/option.c, src/option.h, runtime/doc/options.txt

Patch 7.3.447 (after 7.3.446)
问题: Win32: External commands with "start" do not work.
解决方案: Unescape part of the command. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.3.448 (after 7.3.447)
问题: Win32: Still a problem with "!start /b".
解决方案: Escape only '|'. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.3.449
问题: Crash when a BufWinLeave autocommand closes the only other window. (Daniel Hunt)
解决方案: Abort closing a buffer when it becomes the only one.
相关文件: src/buffer.c, src/proto/buffer.pro, src/ex_cmds.c, src/ex_getln.c, src/misc2.c, src/quickfix.c, src/window.c, src/proto/window.pro

Patch 7.3.450 (after 7.3.448)
问题: Win32: Still a problem with "!start /b".
解决方案: Fix pointer use. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.3.451
问题: Tcl doesn't work on 64 MS-Windows.
解决方案: Make it work. (Dave Bodenstab)
相关文件: src/Make_mvc.mak, src/if_tcl.c

Patch 7.3.452
问题: Undo broken when pasting close to the last line. (Andrey Radev)
解决方案: Use a flag to remember if the deleted included the last line.
(Christian Brabandt)
相关文件: src/ops.c

Patch 7.3.453
问题: Pasting in the command line is slow.
解决方案: Don't redraw if there is another character to read. (Dominique
Pelle)
相关文件: src/ex_getln.c

Patch 7.3.454
问题: Re-allocating memory slows Vim down.
解决方案: Use realloc() in ga_grow(). (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.3.455
问题: Using many continuation lines can be slow.
解决方案: Adjust the reallocation size to the current length.
相关文件: src/ex_cmds2.c

Patch 7.3.456
问题: ":tab drop file" has several problems, including moving the
current window and opening a new tab for a file that already has a
window.
解决方案: Refactor ":tab drop" handling. (Hirohito Higashi)
相关文件: src/buffer.c, src/testdir/test62.in, src/testdir/test62.ok

Patch 7.3.457
问题: When setting $VIMRUNTIME later the directory for fetching
translated messages is not adjusted.
解决方案: Put bindtextdomain() in vim_setenv().
相关文件: src/misc1.c

Patch 7.3.458
问题: Crash when calling smsg() during startup.
解决方案: Don't use 'shortmess' when it is not set yet.
相关文件: src/option.c

Patch 7.3.459
问题: Win32: Warnings for type conversion.
解决方案: Add type casts. (Mike Williams)

相关文件: src/misc2.c, src/os_win32.c

Patch 7.3.460
问题: Win32: UPX does not compress 64 bit binaries.
解决方案: Mention and add the alternative: mpress. (Dave Bodenstab)
相关文件: src/INSTALLpc.txt, src/Make_ming.mak

Patch 7.3.461
问题: The InsertCharPre autocommand event is not triggered during
completion and when typing several characters quickly.
解决方案: Also trigger InsertCharPre during completion. Do not read ahead
when an InsertCharPre autocommand is defined. (Yasuhiro Matsumoto)
相关文件: src/edit.c, src/fileio.c, src/proto/fileio.pro

Patch 7.3.462
问题: When using ":loadview" folds may be closed unexpectedly.
解决方案: Take into account foldlevel. (Xavier de Gaye)
相关文件: src/fold.c

Patch 7.3.463
问题: When using ":s///c" the cursor is moved away from the match.
(Lawman)
解决方案: Don't move the cursor when do_ask is set. (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.3.464
问题: Compiler warning for sprintf.
解决方案: Put the length in a variable. (Dominique Pelle)
相关文件: src/version.c

Patch 7.3.465
问题: Cannot get file name with newline from glob().
解决方案: Add argument to glob() and expand() to indicate they must return a
list. (Christian Brabandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/ex_getln.c, src/vim.h

Patch 7.3.466
问题: Get ml_get error hen ":behave mswin" was used and selecting
several lines. (A. Sinan Unur)
解决方案: Adjust the end of the operation. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.3.467
问题: Cursor positioned wrong at the command line when regaining focus
and using some input method.
解决方案: Do not position the cursor in command line mode.
相关文件: src/mbyte.c

Patch 7.3.468
问题: For some compilers the error file is not easily readable.
解决方案: Use QuickFixCmdPre for more commands. (Marcin Szamotulski)
相关文件: runtime/doc/autocmd.txt, src/quickfix.c

Patch 7.3.469

问题:　　Compiler warning for unused argument without some features.
解决方案:　Add UNUSED.
相关文件:　src/buffer.c

Patch 7.3.470
问题:　　Test 62 fails when compiled without GUI and X11.
解决方案:　Don't test :drop when it is not supported.
相关文件:　src/testdir/test62.in

Patch 7.3.471
问题:　　Can't abort listing placed signs.
解决方案:　Check "got_int". (Christian Brabandt)
相关文件:　src/buffer.c, src/ex_cmds.c

Patch 7.3.472
问题:　　Crash when using ":redraw" in a BufEnter autocommand and
　　switching to another tab. (驼峰)
解决方案:　Move triggering the autocommands to after correcting the
　　option values. Also check the row value to be out of bounds.
　　(Christian Brabandt, Sergey Khorev)
相关文件:　src/screen.c, src/window.c

Patch 7.3.473
问题:　　'cursorbind' does not work correctly in combination with
　　'virtualedit' set to "all".
解决方案:　Copy coladd. (Gary Johnson)
相关文件:　src/move.c

Patch 7.3.474
问题:　　Perl build with gcc 4 fails.
解决方案:　Remove XS() statements. (Yasuhiro Matsumoto)
相关文件:　src/if_perl.xs

Patch 7.3.475
问题:　　In a terminal with few colors the omnicomplete menu may be hard to
　　see when using the default colors.
解决方案:　Use more explicit colors. (suggested by Alex Henrie)
相关文件:　src/syntax.c

Patch 7.3.476
问题:　　When selecting a block, using "$" to include the end of each line
　　and using "A" and typing a backspace strange things happen.
　　(Yuangchen Xie)
解决方案:　Avoid using a negative length. (Christian Brabandt)
相关文件:　src/ops.c

Patch 7.3.477
问题:　　Using ":echo" to output enough lines to scroll, then using "j" and
　　"k" at the more prompt, displays the command on top of the output.
　　(Marcin Szamotulski)
解决方案:　Put the output below the command. (Christian Brabandt)
相关文件:　src/eval.c

Patch 7.3.478

问题:　　Memory leak using the ':rv!' command when reading dictionary or
　　list global variables i.e. with 'viminfo' containing !.
解决方案:　Free the typeval. (Dominique Pelle)
相关文件:　src/eval.c

Patch 7.3.479
问题:　　When 'cursorline' is set the line number highlighting can't be set
　　separately.
解决方案:　Add "CursorLineNr". (Howard Buchholz)
相关文件:　src/option.c, src/screen.c, src/syntax.c, src/vim.h

Patch 7.3.480
问题:　　When using ":qa" and there is a changed buffer picking the buffer
　　to jump to is not very good.
解决方案:　Consider current and other tab pages. (Hirohito Higashi)
相关文件:　src/ex_cmds2.c

Patch 7.3.481
问题:　　Changing 'virtualedit' in an operator function to "all" does not
　　have the desired effect. (Aaron Bohannon)
解决方案:　Save, reset and restore virtual_op when executing an operator
　　function.
相关文件:　src/normal.c

Patch 7.3.482
问题:　　With 'cursorbind' set moving up/down does not always keep the same
　　column.
解决方案:　Set curswant appropriately. (Gary Johnson)
相关文件:　src/move.c

Patch 7.3.483　(after 7.3.477)
问题:　　More prompt shows up too often.
解决方案:　Instead of adding a line break, only start a new line in the
　　message history. (Christian Brabandt)
相关文件:　src/eval.c, src/message.c, src/proto/message.pro

Patch 7.3.484
问题:　　The -E and --echo-wid command line arguments are not mentioned in
　　"vim --help".
解决方案:　Add the help lines. (Dominique Pelle)
相关文件:　src/main.c

Patch 7.3.485
问题:　　When building Vim LDFLAGS isn't passed on to building xxd.
解决方案:　Pass the LDFLAGS value. (James McCoy)
相关文件:　src/Makefile

Patch 7.3.486
问题:　　Build error with mingw64 on Windows 7.
解决方案:　Avoid the step of going through vimres.res. (Guopeng Wen)
相关文件:　src/Make_ming.mak

Patch 7.3.487
问题:　　When setting 'timeoutlen' or 'ttimeoutlen' the column for vertical

movement is reset unnecessarily.
解决方案: Do not set w_set_curswant for every option. Add a test for this.
(Kana Natsuno) Add the P_CURSWANT flag for options.
相关文件: src/option.c, src/testdir/test84.in, src/testdir/test84.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.3.488
问题: ":help!" in a help file does not work as documented.
解决方案: When in a help file don't give an error message. (thinca)
相关文件: src/ex_cmds.c

Patch 7.3.489
问题: CTRL-] in Insert mode does not expand abbreviation when used in a
mapping. (Yichao Zhou)
解决方案: Special case using CTRL-]. (Christian Brabandt)
相关文件: src/getchar.c, src/edit.c

Patch 7.3.490
问题: Member confusion in Lua interface.
解决方案: Fix it. Add luaeval(). (Taro Muraoka, Luis Carvalho)
相关文件: runtime/doc/if_lua.txt, src/eval.c, src/if_lua.c,
src/proto/if_lua.pro

Patch 7.3.491
问题: No tests for Lua.
解决方案: Add some simple tests for Lua. (Luis Carvalho)
相关文件: src/testdir/test1.in, src/testdir/test85.in, src/testdir/test85.ok
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.3.492
问题: Can't indent conditions separately from function arguments.
解决方案: Add the 'k' flag in 'cino'. (Lech Lorens)
相关文件: runtime/doc/indent.txt, src/misc1.c, src/testdir/test3.in,
src/testdir/test3.ok

Patch 7.3.493 (after 7.3.492)
问题: Two unused variables.
解决方案: Remove them. (Hong Xu)
相关文件: src/misc1.c

Patch 7.3.494 (after 7.3.491)
问题: Can't compile with Lua 5.1 or dynamic Lua.
解决方案: Fix dll_ methods. Fix luado(). (Muraoka Taro, Luis Carvalho)
相关文件: src/if_lua.c

Patch 7.3.495 (after 7.3.492)
问题: Compiler warnings.
解决方案: Add function declaration. Remove "offset" argument.
相关文件: src/misc1.c

Patch 7.3.496
问题:　　MS-DOS: When "diff" trips over difference in line separators some tests fail.
解决方案:　Make some .ok files use unix line separators. (David Pope)
相关文件:　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak

Patch 7.3.497
问题:　　Crash when doing ":python print" and compiled with gcc and the optimizer enabled.
解决方案:　Avoid the crash, doesn't really fix the problem. (Christian Brabandt)
相关文件:　src/if_py_both.h

Patch 7.3.498
问题:　　The behavior of the "- register changes depending on value of the 'clipboard' option. (Szamotulski)
解决方案:　Also set the "- register when the register is "*" or "+". (Christian Brabandt)
相关文件:　src/ops.c

Patch 7.3.499
问题:　　When using any interface language when Vim is waiting for a child process it gets confused by a child process started through the interface.
解决方案:　Always used waitpid() instead of wait(). (Yasuhiro Matsumoto)
相关文件:　src/os_unix.c

Patch 7.3.500
问题:　　Ming makefile unconditionally sets WINVER.
解决方案:　Only defined when not already defined. (Yasuhiro Matsumoto)
相关文件:　src/Make_ming.mak

Patch 7.3.501
问题:　　Error for "flush" not being defined when using Ruby command.
解决方案:　Defined "flush" as a no-op method. (Kent Sibilev)
相关文件:　src/if_ruby.c

Patch 7.3.502
问题:　　Netbeans insert halfway a line actually appends to the line.
解决方案:　Insert halfway the line. (Brian Victor)
相关文件:　src/netbeans.c

Patch 7.3.503 (after 7.3.501)
问题:　　Warning for unused argument.
解决方案:　Add UNUSED.
相关文件:　src/if_ruby.c

Patch 7.3.504
问题:　　Commands in help files are not highlighted.
解决方案:　Allow for commands in backticks. Adjust CTRL-] to remove the backticks.
相关文件:　src/ex_cmds.c

Patch 7.3.505

问题:　　Test 11 fails on MS-Windows in some versions.
解决方案:　Fix #ifdefs for whether filtering through a pipe is possible. Move setting b_no_eol_lnum back to where it was before patch 7.3.124. (David Pope)
相关文件:　src/feature.h, src/eval.c, src/ex_cmds.c, src/fileio.c

Patch 7.3.506
问题:　　GTK gives an error when selecting a non-existent file.
解决方案:　Add a handler to avoid the error. (Christian Brabandt)
相关文件:　src/gui_gtk.c

Patch 7.3.507
问题:　　When exiting with unsaved changes, selecting an existing file in the file dialog, there is no dialog to ask whether the existing file should be overwritten. (Felipe G. Nievinski)
解决方案:　Call check_overwrite() before writing. (Christian Brabandt)
相关文件:　src/ex_cmds.c, src/ex_cmds2.c, src/proto/ex_cmds.pro

Patch 7.3.508
问题:　　Default for v:register is not set.
解决方案:　Init v:register in eval_init(). Correct for 'clipboard' before the main loop. (Ingo Karkat)
相关文件:　src/eval.c, src/main.c

Patch 7.3.509
问题:　　":vimgrep" fails when 'autochdir' is set.
解决方案:　A more generic solution for changing directory. (Ben Fritz)
相关文件:　src/quickfix.c

Patch 7.3.510
问题:　　Test 77 fails on Solaris 7. (Michael Soyka)
解决方案:　Replace any tabs with spaces.
相关文件:　src/testdir/test77.in

Patch 7.3.511
问题:　　Using a FileReadCmd autocommand that does ":e! {file}" may cause a crash. (Christian Brabandt)
解决方案:　Properly restore curwin->w_s.
相关文件:　src/fileio.c

Patch 7.3.512
问题:　　undofile() returns a useless name when passed an empty string.
解决方案:　Return an empty string. (Christian Brabandt)
相关文件:　src/eval.c

Patch 7.3.513
问题:　　Cannot use CTRL-E and CTRL-Y with "r".
解决方案:　Make CTRL-E and CTRL-Y work like in Insert mode. (Christian Brabandt)
相关文件:　src/edit.c, src/normal.c, src/proto/edit.pro

Patch 7.3.514
问题:　　No completion for :history command.
解决方案:　Add the completion and update the docs. Also fix ":behave"

completion. (Dominique Pelle)
相关文件: runtime/doc/cmdline.txt, runtime/doc/map.txt, src/ex_docmd.c, src/ex_getln.c, src/vim.h

Patch 7.3.515
问题: 'wildignorecase' only applies to the last part of the path.
解决方案: Also ignore case for letters earlier in the path.
相关文件: src/misc1.c

Patch 7.3.516
问题: extend(o, o) may crash Vim.
解决方案: Fix crash and add test. (Thinca and Hirohito Higashi)
相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.3.517
问题: Crash when using "vipvv". (Alexandre Provencio)
解决方案: Don't let the text length become negative.
相关文件: src/ops.c

Patch 7.3.518
问题: When 'encoding' is a double-byte encoding ":helptags" may not find tags correctly.
解决方案: Use vim_strbyte() instead of vim_strchr(). (Yasuhiro Matsumoto)
相关文件: src/ex_cmds.c

Patch 7.3.519
问题: When completefunction returns it cannot indicate end of completion mode.
解决方案: Recognize completefunction returning -3. (Matsushita Shougo)
相关文件: src/edit.c

Patch 7.3.520
问题: gvim starts up slow on Ubuntu 12.04.
解决方案: Move the call to gui_mch_init_check() to after fork(). (Yasuhiro Matsumoto) Do check $DISPLAY being set.
相关文件: src/gui.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro

Patch 7.3.521
问题: Using "z=" on a multi-byte character may cause a crash.
解决方案: Don't use strlen() on an int pointer.
相关文件: src/spell.c

Patch 7.3.522
问题: Crash in vim_realloc() when using MEM_PROFILE.
解决方案: Avoid using a NULL argument. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.3.523
问题: ":diffupdate" doesn't check for files changed elsewhere.
解决方案: Add the ! flag. (Christian Brabandt)
相关文件: runtime/doc/diff.txt, src/diff.c, src/ex_cmds.h

Patch 7.3.524 (after 7.3.523)
问题: Missing comma.

解决方案: Add the comma.
相关文件: src/version.c

Patch 7.3.525
问题: Compiler warning on 64 bit MS-Windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/ex_getln.c

Patch 7.3.526
问题: Confusing indenting for #ifdef.
解决方案: Remove and add indent. (Elias Diem)
相关文件: src/normal.c

Patch 7.3.527
问题: Clang complains about non-ASCII characters in a string.
解决方案: Change to \x88 form. (Dominique Pelle)
相关文件: src/charset.c

Patch 7.3.528
问题: Crash when closing last window in a tab. (Alex Efros)
解决方案: Use common code in close_last_window_tabpage(). (Christian Brabandt)
相关文件: src/window.c

Patch 7.3.529
问题: Using a count before "v" and "V" does not work (Kikyous)
解决方案: Make the count select that many characters or lines. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.3.530 (after 7.3.520)
问题: gvim does not work when 'guioptions' includes "f". (Davido)
解决方案: Call gui_mch_init_check() when running GUI in the foreground. (Yasuhiro Matsumoto)
相关文件: src/gui.c

Patch 7.3.531 (after 7.3.530)
问题: GUI does not work on MS-Windows.
解决方案: Add the missing #ifdef. (Patrick Avery)
相关文件: src/gui.c

Patch 7.3.532
问题: Compiler warning from Clang.
解决方案: Use a different way to point inside a string. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.3.533
问题: Memory leak when writing undo file.
解决方案: Free the ACL. (Dominique Pelle)
相关文件: src/undo.c

Patch 7.3.534 (after 7.3.461)
问题: When using an InsertCharPre autocommand autoindent fails.
解决方案: Proper handling of v:char. (Alexey Radkov)

相关文件: src/edit.c

Patch 7.3.535
问题: Many #ifdefs for MB_MAXBYTES.
解决方案: Also define MB_MAXBYTES without the +multi_byte feature. Fix places where the buffer didn't include space for a NUL byte.
相关文件: src/arabic.c, src/edit.c, src/eval.c, src/getchar.c, src/mbyte.c, src/misc1.c, src/screen.c, src/spell.c, src/vim.h

Patch 7.3.536
问题: When spell checking the German sharp s is not seen as a word character. (Aexl Bender)
解决方案: In utf_islower() return true for the sharp s. Note: also need updated spell file for this to take effect.
相关文件: src/mbyte.c

Patch 7.3.537
问题: Unnecessary call to init_spell_chartab().
解决方案: Delete the call.
相关文件: src/spell.c

Patch 7.3.538
问题: 'efm' does not handle Tabs in pointer lines.
解决方案: Add Tab support. Improve tests. (Lech Lorens)
相关文件: src/quickfix.c, src/testdir/test10.in, src/testdir/test10.ok

Patch 7.3.539
问题: Redrawing a character on the command line does not work properly for multi-byte characters.
解决方案: Count the number of bytes in a character. (Yukihiro Nakadaira)
相关文件: src/ex_getln.c

Patch 7.3.540
问题: Cursor is left on the text instead of the command line.
解决方案: Don't call setcursor() in command line mode.
相关文件: src/getchar.c

Patch 7.3.541
问题: When joining lines comment leaders need to be removed manually.
解决方案: Add the 'j' flag to 'formatoptions'. (Lech Lorens)
相关文件: runtime/doc/change.txt, src/edit.c, src/ex_docmd.c, src/misc1.c, src/normal.c, src/ops.c, src/option.h, src/proto/misc1.pro, src/proto/ops.pro, src/search.c, src/testdir/test29.in, src/testdir/test29.ok

Patch 7.3.542 (after 7.3.506)
问题: Function is sometimes unused.
解决方案: Add #ifdef.
相关文件: src/gui_gtk.c

Patch 7.3.543
问题: The cursor is in the wrong line after using ":copen". (John Beckett)
解决方案: Invoke more drastic redraw method.

相关文件: src/eval.c

Patch 7.3.544
问题: There is no good way to close a quickfix window when closing the last ordinary window.
解决方案: Add the QuitPre autocommand.
相关文件: src/ex_docmd.c, src/fileio.c, src/vim.h

Patch 7.3.545
问题: When closing a window or buffer autocommands may close it too, causing problems for where the autocommand was invoked from.
解决方案: Add the w_closing and b_closing flags. When set disallow ":q" and ":close" to prevent recursive closing.
相关文件: src/structs.h, src/buffer.c, src/ex_docmd.c, src/window.c

Patch 7.3.546
问题: Bogus line break.
解决方案: Remove the line break.
相关文件: src/screen.c

Patch 7.3.547 (after 7.3.541)
问题: Compiler warning for uninitialized variable.
解决方案: Initialize it.
相关文件: src/ops.c

Patch 7.3.548
问题: Compiler warning on 64 bit Windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/ops.c

Patch 7.3.549
问题: In 'cinoptions' "0s" is interpreted as one shiftwidth. (David Pineau)
解决方案: Use the zero as zero. (Lech Lorens)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.550 (after 7.3.541)
问题: With "j" in 'formatoptions' a list leader is not removed. (Gary Johnson)
解决方案: Don't ignore the start of a three part comment. (Lech Lorens)
相关文件: src/ops.c, src/testdir/test29.in, src/testdir/test29.ok

Patch 7.3.551
问题: When using :tablose a TabEnter autocommand is triggered too early. (Karthick)
解决方案: Don't trigger *Enter autocommands before closing the tab. (Christian Brabandt)
相关文件: src/buffer.c, src/eval.c, src/ex_cmds2.c, src/fileio.c, src/proto/window.pro, src/window.c

Patch 7.3.552
问题: Formatting inside comments does not use the "2" flag in 'formatoptions'.
解决方案: Support the "2" flag. (Tor Perkins)

相关文件: src/vim.h, src/ops.c, src/edit.c, src/misc1.c,
           src/testdir/test68.in, src/testdir/test68.ok

Patch 7.3.553
问题: With double-width characters and 'listchars' containing "precedes"
           the text is displayed one cell off.
解决方案: Check for double-width character being overwritten by the
           "precedes" character. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 7.3.554 (after 7.3.551)
问题: Compiler warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/window.c

Patch 7.3.555
问题: Building on IBM z/OS fails.
解决方案: Adjust configure. Use the QUOTESED value from config.mk instead of
           the hard coded one in Makefile. (Stephen Bovy)
相关文件: src/configure.in, src/auto/configure, src/Makefile

Patch 7.3.556
问题: Compiler warnings on 64 bit Windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/misc1.c

Patch 7.3.557
问题: Crash when an autocommand wipes out a buffer when it is hidden.
解决方案: Restore the current window when needed. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.3.558
问题: Memory access error. (Gary Johnson)
解决方案: Allocate one more byte. (Dominique Pelle)
相关文件: src/misc1.c

Patch 7.3.559
问题: home_replace() does not work with 8.3 filename.
解决方案: Make ":p" expand 8.3 name to full path. (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/misc1.c

Patch 7.3.560
问题: Get an error for a locked argument in extend().
解决方案: Initialize the lock flag for a dictionary. (Yukihiro Nakadaira)
相关文件: src/eval.c

Patch 7.3.561
问题: Using refresh: always in a complete function breaks the "."
           command. (Val Markovic)
解决方案: Add match leader to the redo buffer. (Yasuhiro Matsumoto)
相关文件: src/edit.c

Patch 7.3.562
问题: ":profdel" should not work when the +profile feature is disabled.

解决方案: Call ex_ni(). (Yasuhiro Matsumoto)
相关文件: src/ex_cmds2.c

Patch 7.3.563 (after 7.3.557)
问题: Can't build with tiny features.
解决方案: Add #ifdef.
相关文件: src/buffer.c

Patch 7.3.564 (after 7.3.559)
问题: Warning for pointer conversion.
解决方案: Add type cast.
相关文件: src/misc1.c

Patch 7.3.565
问题: Can't generate proto file for Python 3.
解决方案: Add PYTHON3_CFLAGS to LINT_CFLAGS.
相关文件: src/Makefile

Patch 7.3.566 (after 7.3.561)
问题: Redo after completion does not work correctly when refresh: always
is not used. (Raymond Ko)
解决方案: Check the compl_opt_refresh_always flag. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.3.567
问题: Missing copyright notice.
解决方案: Add Vim copyright notice. (Taro Muraoka)
相关文件: src/dehqx.py

Patch 7.3.568
问题: Bad indents for #ifdefs.
解决方案: Add and remove spaces. (Elias Diem)
相关文件: src/globals.h

Patch 7.3.569
问题: Evaluating Vim expression in Python is insufficient.
解决方案: Add vim.bindeval(). Also add pyeval() and py3eval(). (ZyX)
相关文件: runtime/doc/eval.txt, runtime/doc/if_pyth.txt, src/eval.c,
src/if_lua.c, src/if_py_both.h, src/if_python.c, src/if_python3.c,
src/proto/eval.pro, src/proto/if_python.pro,
src/proto/if_python3.pro, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Makefile,
src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.570
问题: ":vimgrep" does not obey 'wildignore'.
解决方案: Apply 'wildignore' and 'suffixes' to ":vimgrep". (Ingo Karkat)
相关文件: src/ex_cmds2.c, src/proto/ex_cmds2.pro, src/quickfix.c, src/spell.c

Patch 7.3.571
问题: Duplicated condition.
解决方案: Remove one. (Dominique Pelle)

相关文件: src/os_win32.c

Patch 7.3.572
问题: Duplicate statement in if and else. (Dominique Pelle)
解决方案: Remove the condition and add a TODO.
相关文件: src/gui_xmebw.c

Patch 7.3.573
问题: Using array index before bounds checking.
解决方案: Swap the parts of the condition. (Dominique Pelle)
相关文件: src/ops.c

Patch 7.3.574
问题: When pasting a register in the search command line a CTRL-L
character is not pasted. (Dominique Pelle)
解决方案: Escape the CTRL-L. (Christian Brabandt)
相关文件: src/ex_getln.c

Patch 7.3.575
问题: "ygt" tries to yank instead of giving an error. (Daniel Mueller)
解决方案: Check for a pending operator.
相关文件: src/normal.c

Patch 7.3.576
问题: Formatting of lists inside comments is not right yet.
解决方案: Use another solution and add a test. (Tor Perkins)
相关文件: src/edit.c, src/misc1.c, src/testdir/test68.in,
src/testdir/test69.ok

Patch 7.3.577
问题: Size of memory does not fit in 32 bit unsigned.
解决方案: Use Kbyte instead of byte. Call GlobalMemoryStatusEx() instead of
GlobalMemoryStatus() when available.
相关文件: src/misc2.c, src/option.c, src/os_amiga.c, src/os_msdos.c,
src/os_win16.c, src/os_win32.c

Patch 7.3.578
问题: Misplaced declaration.
解决方案: Move declaration to start of block.
相关文件: src/if_py_both.h

Patch 7.3.579 (after 7.3.569)
问题: Can't compile with Python 2.5.
解决方案: Use PyCObject when Capsules are not available.
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.580
问题: Warning on 64 bit MS-Windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/if_py_both.h

Patch 7.3.581
问题: Problems compiling with Python.
解决方案: Pick UCS2 or UCS4 function at runtime. (lilydjwg)

相关文件: src/if_python.c

Patch 7.3.582 (after 7.3.576)
问题: Missing parts of the test OK file.
解决方案: Add the missing parts.
相关文件: src/testdir/test68.ok

Patch 7.3.583
问题: PyObject_NextNotImplemented is not defined before Python 2.7.
           (Danek Duvall)
解决方案: Add #ifdefs.
相关文件: src/if_python.c

Patch 7.3.584
问题: PyCObject is not always defined.
解决方案: Use PyObject instead.
相关文件: src/if_py_both.h, src/if_python.c

Patch 7.3.585
问题: Calling changed_bytes() too often.
解决方案: Move changed_bytes() out of a loop. (Tor Perkins)
相关文件: src/edit.c

Patch 7.3.586
问题: When compiling with Cygwin or MingW MEMORYSTATUSEX is not defined.
解决方案: Set the default for WINVER to 0x0500.
相关文件: src/Make_ming.mak, src/Make_cyg.mak

Patch 7.3.587
问题: Compiler warning for local var shadowing global var.
解决方案: Rename the var and move it to an inner block. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.3.588
问题: Crash on NULL pointer.
解决方案: Fix the immediate problem by checking for NULL. (Lech Lorens)
相关文件: src/window.c

Patch 7.3.589
问题: Crash when $HOME is not set.
解决方案: Check for a NULL pointer. (Chris Webb)
相关文件: src/misc1.c

Patch 7.3.590
问题: The '< and '> marks cannot be set directly.
解决方案: Allow setting '< and '>. (Christian Brabandt)
相关文件: src/mark.c

Patch 7.3.591
问题: Can only move to a tab by absolute number.
解决方案: Move a number of tabs to the left or the right. (Lech Lorens)
相关文件: runtime/doc/tabpage.txt, src/ex_cmds.h, src/ex_docmd.c,
           src/testdir/test62.in, src/testdir/test62.ok, src/window.c

Patch 7.3.592
问题:　　Vim on GTK does not support g:browsefilter.
解决方案:　Add a GtkFileFilter to the file chooser. (Christian Brabandt)
相关文件:　src/gui_gtk.c

Patch 7.3.593
问题:　　No easy way to decide if b:browsefilter will work.
解决方案:　Add the browsefilter feature.
相关文件:　src/gui_gtk.c, src/eval.c, src/vim.h

Patch 7.3.594
问题:　　The X command server doesn't work perfectly. It sends an empty
reply for as-keys requests.
解决方案:　Remove duplicate ga_init2(). Do not send a reply for as-keys
requests. (Brian Burns)
相关文件:　src/if_xcmdsrv.c

Patch 7.3.595
问题:　　The X command server responds slowly
解决方案:　Change the loop that waits for replies. (Brian Burns)
相关文件:　src/if_xcmdsrv.c

Patch 7.3.596
问题:　　Can't remove all signs for a file or buffer.
解决方案:　Support "*" for the sign id. (Christian Brabandt)
相关文件:　runtime/doc/sign.txt, src/buffer.c, src/ex_cmds.c,
src/proto/buffer.pro

Patch 7.3.597
问题:　　'clipboard' "autoselect" only applies to the * register. (Sergey
Vakulenko)
解决方案:　Make 'autoselect' work for the + register. (Christian Brabandt)
Add the "autoselectplus" option in 'clipboard' and the "P" flag in
'guioptions'.
相关文件:　runtime/doc/options.txt, src/normal.c, src/ops.c, src/screen.c,
src/ui.c, src/globals.h, src/proto/ui.pro, src/option.h, src/gui.c

Patch 7.3.598
问题:　　Cannot act upon end of completion. (Taro Muraoka)
解决方案:　Add an autocommand event that is triggered when completion has
finished. (Idea by Florian Klein)
相关文件:　src/edit.c, src/fileio.c, src/vim.h

Patch 7.3.599 (after 7.3.597)
问题:　　Missing change in one file.
解决方案:　Patch for changed clip_autoselect().
相关文件:　src/option.c

Patch 7.3.600
问题:　　<f-args> is not expanded properly with DBCS encoding.
解决方案:　Skip over character instead of byte. (Yukihiro Nakadaira)
相关文件:　src/ex_docmd.c

Patch 7.3.601

问题: Bad code style.
解决方案: Insert space, remove parens.
相关文件: src/farsi.c

Patch 7.3.602
问题: Missing files in distribution.
解决方案: Update the list of files.
相关文件: Filelist

Patch 7.3.603
问题: It is possible to add replace builtin functions by calling
            extend() on g:.
解决方案: Add a flag to a dict to indicate it is a scope. Check for
            existing functions. (ZyX)
相关文件: src/buffer.c, src/eval.c, src/proto/eval.pro, src/structs.h,
            src/testdir/test34.in, src/testdir/test34.ok, src/window.c

Patch 7.3.604
问题: inputdialog() doesn't use the cancel argument in the console.
            (David Fishburn)
解决方案: Use the third argument. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.3.605 (after 7.3.577)
问题: MS-Windows: Can't compile with older compilers. (Titov Anatoly)
解决方案: Add #ifdef for MEMORYSTATUSEX.
相关文件: src/os_win32.c

Patch 7.3.606
问题: CTRL-P completion has a problem with multi-byte characters.
解决方案: Check for next character being NUL properly. (Yasuhiro Matsumoto)
相关文件: src/search.c, src/macros.h

Patch 7.3.607
问题: With an 8 color terminal the selected menu item is black on black,
            because darkGrey as bg is the same as black.
解决方案: Swap fg and bg colors. (James McCoy)
相关文件: src/syntax.c

Patch 7.3.608
问题: winrestview() does not always restore the view correctly.
解决方案: Call win_new_height() and win_new_width(). (Lech Lorens)
相关文件: src/eval.c, src/proto/window.pro, src/window.c

Patch 7.3.609
问题: File names in :checkpath! output are garbled.
解决方案: Check for \zs in the pattern. (Lech Lorens)
相关文件: src/search.c, src/testdir/test17.in, src/testdir/test17.ok

Patch 7.3.610
问题: Cannot operate on the text that a search pattern matches.
解决方案: Add the "gn" and "gN" commands. (Christian Brabandt)
相关文件: runtime/doc/index.txt, runtime/doc/visual.txt, src/normal.c,
            src/proto/search.pro, src/search.c, src/testdir/test53.in,

src/testdir/test53.ok

Patch 7.3.611
问题: Can't use Vim dictionary as self argument in Python.
解决方案: Fix the check for the "self" argument. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.612
问题: Auto formatting messes up text when 'fo' contains "2". (ZyX)
解决方案: Decrement "less_cols". (Tor Perkins)
相关文件: src/misc1.c, src/testdir/test68.in, src/testdir/test68.ok

Patch 7.3.613
问题: Including Python's config.c in the build causes trouble. It is not clear why it was there.
解决方案: Omit the config file. (James McCoy)
相关文件: src/Makefile, src/auto/configure, src/configure.in

Patch 7.3.614
问题: Number argument gets turned into a number while it should be a string.
解决方案: Add flag to the call_vim_function() call. (Yasuhiro Matsumoto)
相关文件: src/edit.c, src/eval.c, src/proto/eval.pro

Patch 7.3.615
问题: Completion for a user command does not recognize backslash before a space.
解决方案: Recognize escaped characters. (Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c

Patch 7.3.616 (after 7.3.610)
问题: Can't compile without +visual.
解决方案: Add #ifdef.
相关文件: src/normal.c

Patch 7.3.617 (after 7.3.615)
问题: Hang on completion.
解决方案: Skip over the space. (Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c

Patch 7.3.618 (after 7.3.616)
问题: Still doesn't compile with small features.
解决方案: Move current_search() out of #ifdef. (Dominique Pelle)
相关文件: src/normal.c, src/search.c

Patch 7.3.619
问题: When executing a shell command Vim may become slow to respond.
解决方案: Don't wait after every processed message. (idea by Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.3.620
问题: Building with recent Ruby on Win32 doesn't work.
解决方案: Add a separate argument for the API version. (Yasuhiro Matsumoto)

相关文件: src/Make_ming.mak, src/Make_mvc.mak

Patch 7.3.621
问题: Compiler warnings on 64 bit windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/ex_docmd.c, src/search.c

Patch 7.3.622
问题: XPM library for Win32 can't be found.
解决方案: Suggest using the one from the Vim ftp site.
相关文件: src/Make_mvc.mak

Patch 7.3.623
问题: Perl 5.14 commands crash Vim on MS-Windows.
解决方案: Use perl_get_sv() instead of GvSV(). (Raymond Ko)
相关文件: src/if_perl.xs

Patch 7.3.624
问题: When cancelling input() it returns the third argument. That should only happen for inputdialog().
解决方案: Check if inputdialog() was used. (Hirohito Higashi)
相关文件: src/eval.c

Patch 7.3.625
问题: "gn" does not handle zero-width matches correctly.
解决方案: Handle zero-width patterns specially. (Christian Brabandt)
相关文件: src/search.c

Patch 7.3.626
问题: Python interface doesn't build with Python 2.4 or older.
解决方案: Define Py_ssize_t. (Benjamin Bannier)
相关文件: src/if_py_both.h

Patch 7.3.627
问题: When using the "n" flag with the ":s" command a \= substitution will not be evaluated.
解决方案: Do perform the evaluation, so that a function can be invoked at every matching position without changing the text. (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.3.628
问题: ":open" does not allow for a !, which results in a confusing error message. (Shawn Wilson)
解决方案: Allow ! on ":open". (Christian Brabandt)
相关文件: src/ex_cmds.h

Patch 7.3.629
问题: There is no way to make 'shiftwidth' follow 'tabstop'.
解决方案: When 'shiftwidth' is zero use the value of 'tabstop'. (Christian Brabandt)
相关文件: src/edit.c, src/ex_getln.c, src/fold.c, src/misc1.c, src/ops.c, src/option.c, src/proto/option.pro

Patch 7.3.630
问题: "|" does not behave correctly when 'virtualedit' is set.
解决方案: Call validate_virtcol(). (David Bürgin)
相关文件: src/normal.c

Patch 7.3.631
问题: Cannot complete user names.
解决方案: Add user name completion. (Dominique Pelle)
相关文件: runtime/doc/map.txt, src/auto/configure, src/config.h.in,
src/configure.in, src/ex_docmd.c, src/ex_getln.c, src/misc1.c,
src/misc2.c, src/proto/misc1.pro, src/vim.h

Patch 7.3.632
问题: Cannot select beyond 222 columns with the mouse in xterm.
解决方案: Add support for SGR mouse tracking. (Hayaki Saito)
相关文件: runtime/doc/options.txt, src/feature.h, src/keymap.h, src/misc2.c,
src/option.h, src/os_unix.c, src/term.c, src/version.c

Patch 7.3.633
问题: Selection remains displayed as selected after selecting another
text.
解决方案: Call xterm_update() before select(). (Andrew Pimlott)
相关文件: src/os_unix.c

Patch 7.3.634
问题: Month/Day format for undo is confusing. (Marcin Szamotulski)
解决方案: Always use Year/Month/Day, should work for everybody.
相关文件: src/undo.c

Patch 7.3.635
问题: Issue 21: System call during startup sets 'lines' to a wrong
value. (Karl Yngve)
解决方案: Don't set the shell size while the GUI is still starting up.
(Christian Brabandt)
相关文件: src/ui.c

Patch 7.3.636 (after 7.3.625)
问题: Not all zero-width matches handled correctly for "gn".
解决方案: Move zero-width detection to a separate function. (Christian
Brabandt)
相关文件: src/search.c

Patch 7.3.637
问题: Cannot catch the error caused by a foldopen when there is no fold.
(ZyX, Issue 48)
解决方案: Do not break out of the loop early when inside try/catch.
(Christian Brabandt) Except when there is a syntax error.
相关文件: src/ex_docmd.c, src/globals.h

Patch 7.3.638
问题: Unnecessary redraw of the previous character.
解决方案: Check if the character is double-width. (Jon Long)
相关文件: src/screen.c

Patch 7.3.639
问题: It's not easy to build Vim on Windows with XPM support.
解决方案: Include the required files, they are quite small. Update the MSVC makefile to use them. Binary files are in the next patch. (Sergey Khorev)
相关文件: src/xpm/COPYRIGHT, src/xpm/README.txt, src/xpm/include/simx.h, src/xpm/include/xpm.h, src/Make_mvc.mak, src/bigvim.bat, src/bigvim64.bat, Filelist

Patch 7.3.640
问题: It's not easy to build Vim on Windows with XPM support.
解决方案: Binary files for 7.3.639. (Sergey Khorev)
相关文件: src/xpm/x64/lib/libXpm.lib, src/xpm/x86/lib/libXpm.a, src/xpm/x86/lib/libXpm.lib

Patch 7.3.641
问题: ":mkview" uses ":normal" instead of ":normal!" for folds. (Dan)
解决方案: Add the bang. (Christian Brabandt)
相关文件: src/fold.c

Patch 7.3.642
问题: Segfault with specific autocommands. Was OK after 7.3.449 and before 7.3.545. (Richard Brown)
解决方案: Pass TRUE for abort_if_last in the call to close_buffer(). (Christian Brabandt)
相关文件: src/window.c

Patch 7.3.643 (after 7.3.635)
问题: MS-Windows: When starting gvim maximized 'lines' and 'columns' are wrong. (Christian Robinson)
解决方案: Move the check for gui.starting from ui_get_shellsize() to check_shellsize().
相关文件: src/ui.c, src/term.c

Patch 7.3.644
问题: Dead code for BeOS GUI.
解决方案: Remove unused __BEOS__ stuff.
相关文件: src/gui.c

Patch 7.3.645
问题: No tests for patch 7.3.625 and 7.3.637.
解决方案: Add more tests for the "gn" command and try/catch. (Christian Brabandt)
相关文件: src/testdir/test53.in, src/testdir/test53.ok, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.3.646
问题: When reloading a buffer the undo file becomes unusable unless ":w" is executed. (Dmitri Frank)
解决方案: After reloading the buffer write the undo file. (Christian Brabandt)
相关文件: src/fileio.c

Patch 7.3.647

问题:　　"gnd" doesn't work correctly in Visual mode.
解决方案:　Handle Visual mode differently in "gn". (Christian Brabandt)
相关文件:　src/search.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.3.648
问题:　　Crash when using a very long file name. (ZyX)
解决方案:　Properly check length of buffer space.
相关文件:　src/buffer.c

Patch 7.3.649
问题:　　When 'clipboard' is set to "unnamed" small deletes end up in the
　　numbered registers. (Ingo Karkat)
解决方案:　Use the original register name to decide whether to put a delete
　　in a numbered register. (Christian Brabandt)
相关文件:　src/ops.c

Patch 7.3.650
问题:　　Completion after ":help \{-" gives an error message and messes up
　　the command line.
解决方案:　Cancel the tag search if the pattern can't be compiled. (Yasuhiro
　　Matsumoto)
相关文件:　src/tag.c

Patch 7.3.651
问题:　　Completion after ":help \{-" gives an error message.
解决方案:　Prepend a backslash.
相关文件:　src/ex_cmds.c

Patch 7.3.652
问题:　　Workaround for Python crash isn't perfect.
解决方案:　Change the type of the length argument. (Sean Estabrooks)
相关文件:　src/if_py_both.h

Patch 7.3.653
问题:　　MingW needs build rule for included XPM files. Object directory
　　for 32 and 64 builds is the same, also for MSVC.
解决方案:　Add MingW build rule to use included XPM files. Add the CPU or
　　architecture to the object directory name. (Sergey Khorev)
相关文件:　src/Make_ming.mak, src/Make_mvc.mak, src/xpm/README.txt

Patch 7.3.654
问题:　　When creating a Vim dictionary from Python objects an empty key
　　might be used.
解决方案:　Do not use empty keys, throw an IndexError. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.655
问题:　　64 bit MingW xpm .a file is missing.
解决方案:　Add the file. (Sergey Khorev)
相关文件:　src/xpm/x64/lib/libXpm.a

Patch 7.3.656
问题:　　Internal error in :pyeval.
解决方案:　Handle failed object conversion. (ZyX)

相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.657
问题: Python bindings silently truncate string values containing NUL.
解决方案: Fail when a string contains NUL. (ZyX)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.658
问题: NUL bytes truncate strings when converted from Python.
解决方案: Handle truncation as an error. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c

Patch 7.3.659
问题: Recent Python changes are not tested.
解决方案: Add tests for Python bindings. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.660
问题: ":help !" jumps to help for ":!".
解决方案: Adjust check for tag header line. (Andy Wokula)
相关文件: src/tag.c

Patch 7.3.661 (after 7.3.652)
问题: SEGV in Python code.
解决方案: Initialize len to zero. Use the right function depending on
version. (Maxim Philippov)
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.662
问题: Can't build Ruby interface with Ruby 1.9.3.
解决方案: Add missing functions. (V. Ondruch)
相关文件: src/if_ruby.c

Patch 7.3.663
问题: End of color scheme name not clear in E185. (Aaron Lewis)
解决方案: Put the name in single quotes.
相关文件: src/ex_docmd.c

Patch 7.3.664
问题: Buffer overflow in unescaping text. (Raymond Ko)
解决方案: Limit check for multi-byte character to 4 bytes.
相关文件: src/mbyte.c

Patch 7.3.665
问题: MSVC 11 is not supported. (Raymond Ko)
解决方案: Recognize MSVC 11. (Gary Willoughby)
相关文件: src/Make_mvc.mak

Patch 7.3.666
问题: With MSVC 11 Win32.mak is not found.
解决方案: Add the SDK_INCLUDE_DIR variable. (Raymond Ko)
相关文件: src/Make_mvc.mak

Patch 7.3.667
问题:　　Unused variables in Perl interface.
解决方案:　Adjust #ifdefs.
相关文件:　src/if_perl.xs

Patch 7.3.668
问题:　　Building with Perl loaded dynamically still uses static library.
解决方案:　Adjust use of PL_thr_key. (Ken Takata)
相关文件:　src/if_perl.xs

Patch 7.3.669
问题:　　When building with Cygwin loading Python dynamically fails.
解决方案:　Use DLLLIBRARY instead of INSTSONAME. (Ken Takata)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.670
问题:　　Python: memory leaks when there are exceptions.
解决方案:　Add DICTKEY_UNREF in the right places. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.671
问题:　　More Python code can be shared between Python 2 and 3.
解决方案:　Move code to if_py_both.h. (ZyX)
相关文件:　src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.672
问题:　　Not possible to lock/unlock lists in Python interface.
解决方案:　Add .locked and .scope attributes. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python.c,
　　src/if_python3.c, src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.673
问题:　　Using "gN" while 'selection' is "exclusive" misses one character.
　　(Ben Fritz)
解决方案:　Check the direction when compensating for exclusive selection.
　　(Christian Brabandt)
相关文件:　src/search.c

Patch 7.3.674
问题:　　Can't compile with Lua/dyn on Cygwin.
解决方案:　Adjust configure to use the right library name. (Ken Takata)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.675
问题:　　Using uninitialized memory with very long file name.
解决方案:　Put NUL after text when it is truncated. (ZyX)
相关文件:　src/buffer.c

Patch 7.3.676
问题:　　Ruby compilation on Windows 32 bit doesn't work.
解决方案:　Only use some functions for 64 bit. (Ken Takata)
相关文件:　src/if_ruby.c

Patch 7.3.677
问题:　　buf_spname() is used inconsistently.
解决方案:　Make the return type a char_u pointer. Check the size of the
　　returned string.
相关文件:　src/buffer.c, src/proto/buffer.pro, src/ex_cmds2.c,
　　src/ex_docmd.c, src/memline.c, src/screen.c

Patch 7.3.678
问题:　　Ruby .so name may not be correct.
解决方案:　Use the LIBRUBY_SO entry from the config. (Vit Ondruch)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.679
问题:　　Ruby detection uses Config, newer Ruby versions use RbConfig.
解决方案:　Detect the need to use RbConfig. (Vit Ondruch)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.680
问题:　　Some files missing in the list of distributed files.
解决方案:　Add lines for new files.
相关文件:　Filelist

Patch 7.3.681 (after 7.3.680)
问题:　　List of distributed files picks up backup files.
解决方案:　Make tutor patterns more specific.
相关文件:　Filelist

Patch 7.3.682 (after 7.3.677)
问题:　　Compiler complains about incompatible types.
解决方案:　Remove type casts. (hint by Danek Duvall)
相关文件:　src/edit.c

Patch 7.3.683
问题:　　":python" may crash when vimbindeval() returns None.
解决方案:　Check for v_string to be NULL. (Yukihiro Nakadaira)
相关文件:　src/if_py_both.h

Patch 7.3.684
问题:　　"make test" does not delete lua.vim.
解决方案:　Add lua.vim to the clean target. (Simon Ruderich)
相关文件:　src/testdir/Makefile, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Make_vms.mms

Patch 7.3.685
问题:　　No test for what patch 7.3.673 fixes.
解决方案:　Add a test. (Christian Brabandt)
相关文件:　src/testdir/test53.in, src/testdir/test53.ok

Patch 7.3.686
问题:　　Using CTRL-\ e mappings is useful also when entering an
　　expression, but it doesn't work. (Marcin Szamotulski)
解决方案:　Allow using CTRL-\ e when entering an expression if it was not
　　typed.
相关文件:　src/ex_getln.c

Patch 7.3.687
问题: Test 16 fails when $DISPLAY is not set.
解决方案: Skip the test when $DISPLAY is not set.
相关文件: src/testdir/test16.in

Patch 7.3.688
问题: Python 3.3 is not supported.
解决方案: Add Python 3.3 support (Ken Takata)
相关文件: src/if_python3.c

Patch 7.3.689
问题: MzScheme and Lua may use a NULL string.
解决方案: Use an empty string instead of NULL. (Yukihiro Nakadaira)
相关文件: src/if_lua.c, src/if_mzsch.c

Patch 7.3.690
问题: When the current directory name is exactly the maximum path length
Vim may crash.
解决方案: Only add "/" when there is room. (Danek Duvall)
相关文件: src/os_unix.c

Patch 7.3.691
问题: State specific to the Python thread is discarded.
解决方案: Keep state between threads. (Paul)
相关文件: src/if_python.c

Patch 7.3.692
问题: Can't build GTK version with GTK 2.0.
解决方案: Put GtkFileFilter declaration in the right place. (Yegappan
Lakshmanan)
相关文件: src/gui_gtk.c

Patch 7.3.693
问题: Can't make 'softtabstop' follow 'shiftwidth'.
解决方案: When 'softtabstop' is negative use the value of 'shiftwidth'.
(so8res)
相关文件: src/edit.c, src/option.c, src/proto/option.pro

Patch 7.3.694
问题: Now that 'shiftwidth' may use the value of 'tabstop' it is not so
easy to use in indent files.
解决方案: Add the shiftwidth() function. (so8res)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.3.695
问题: Balloon cannot show multi-byte text.
解决方案: Properly deal with multi-byte characters. (Dominique Pelle)
相关文件: src/gui_beval.c, src/ui.c

Patch 7.3.696
问题: Message about added spell language can be wrong.
解决方案: Give correct message. Add g:menutrans_set_lang_to to allow for
translation. (Jiri Sedlak)

相关文件: runtime/menu.vim

Patch 7.3.697
问题: Leaking resources when setting GUI font.
解决方案: Free the font. (Ken Takata)
相关文件: src/syntax.c

Patch 7.3.698
问题: Python 3 does not preserve state between commands.
解决方案: Preserve the state. (Paul Ollis)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.699
问题: When 'ttymouse' is set to "sgr" manually, it is overruled by
automatic detection.
解决方案: Do not use automatic detection when 'ttymouse' was set manually.
(Hayaki Saito)
相关文件: src/term.c

Patch 7.3.700
问题: Cannot detect URXVT and SGR mouse support.
解决方案: add +mouse_urxvt and +mouse_sgr. (Hayaki Saito)
相关文件: src/feature.h, src/eval.c

Patch 7.3.701
问题: MS-Windows: Crash with stack overflow when setting 'encoding'.
解决方案: Handle that loading the iconv library may be called recursively.
(Jiri Sedlak)
相关文件: src/os_win32.c

Patch 7.3.702
问题: Nmake from VS6 service pack 6 is not recognized.
解决方案: Detect the version number. (Jiri Sedlak)
相关文件: src/Make_mvc.mak

Patch 7.3.703
问题: When 'undofile' is reset the hash is computed unnecessarily.
解决方案: Only compute the hash when the option was set. (Christian Brabandt)
相关文件: src/option.c

Patch 7.3.704
问题: Repeating "cgn" does not always work correctly.
解决方案: Also fetch the operator character. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.3.705
问题: Mouse features are not sorted properly. (Tony Mechelynck)
解决方案: Put the mouse features in alphabetical order.
相关文件: src/version.c

Patch 7.3.706 (after 7.3.697)
问题: Can't build Motif version.
解决方案: Fix wrongly named variable. (Ike Devolder)
相关文件: src/syntax.c

Patch 7.3.707 (after 7.3.701)
问题:　　Problems loading a library for a file name with non-latin
　　characters.
解决方案:　Use wide system functions when possible. (Ken Takata)
相关文件:　src/os_win32.c, src/os_win32.h

Patch 7.3.708
问题:　　Filler lines above the first line may be hidden when opening Vim.
解决方案:　Change how topfill is computed. (Christian Brabandt)
相关文件:　src/diff.c, src/testdir/test47.in, src/testdir/test47.ok

Patch 7.3.709
问题:　　Compiler warning for unused argument.
解决方案:　Add UNUSED.
相关文件:　src/eval.c

Patch 7.3.710 (after 7.3.704)
问题:　　Patch 7.3.704 breaks "fn".
解决方案:　Add check for ca.cmdchar. (Christian Brabandt)
相关文件:　src/normal.c

Patch 7.3.711 (after 7.3.688)
问题:　　vim.current.buffer is not available. (lilydjwg)
解决方案:　Use py3_PyUnicode_AsUTF8 instead of py3_PyUnicode_AsUTF8String.
　　(Ken Takata)
相关文件:　src/if_python3.c

Patch 7.3.712
问题:　　Nmake from VS2010 SP1 is not recognized.
解决方案:　Add the version number. (Ken Takata)
相关文件:　src/Make_mvc.mak

Patch 7.3.713
问题:　　printf() can only align to bytes, not characters.
解决方案:　Add the "S" item. (Christian Brabandt)
相关文件:　runtime/doc/eval.txt, src/message.c

Patch 7.3.714
问题:　　Inconsistency: :set can be used in the sandbox, but :setlocal and
　　:setglobal cannot. (Michael Henry)
解决方案:　Fix the flags for :setlocal and :setglobal. (Christian Brabandt)
相关文件:　src/ex_cmds.h

Patch 7.3.715
问题:　　Crash when calling setloclist() in BufUnload autocmd. (Marcin
　　Szamotulski)
解决方案:　Set w_llist to NULL when it was freed. Also add a test.
　　(Christian Brabandt)
相关文件:　src/quickfix.c, src/testdir/test49.ok, src/testdir/test49.vim

Patch 7.3.716
问题:　　Error on exit when using Python 3.
解决方案:　Remove PythonIO_Fini(). (Roland Puntaier)

相关文件: src/if_python3.c

Patch 7.3.717
问题: When changing the font size, only MS-Windows limits the window size.
解决方案: Also limit the window size on other systems. (Roland Puntaier)
相关文件: src/gui.c

Patch 7.3.718
问题: When re-using the current buffer the buffer-local options stay.
解决方案: Re-initialize the buffer-local options. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.3.719
问题: Cannot run new version of cproto, it fails on missing include files.
解决方案: Add lots of #ifndef PROTO
相关文件: src/os_amiga.c, src/os_amiga.h, src/gui_w16.c, src/gui_w48.c, src/gui_w32.c, src/vimio.h, src/os_msdos.c, src/os_msdos.h, src/os_win16.h, src/os_win16.c, src/os_win32.h, src/os_win32.c, src/os_mswin.c, src/gui_photon.c, src/os_unix.h, src/os_beos.c, src/os_beos.h

Patch 7.3.720
问题: Proto files are outdated.
解决方案: Update the newly generated proto files.
相关文件: src/proto/digraph.pro, src/proto/fold.pro, src/proto/misc1.pro, src/proto/move.pro, src/proto/screen.pro, src/proto/search.pro, src/proto/os_win32.pro, src/proto/os_mswin.pro, src/proto/os_beos.pro

Patch 7.3.721
问题: Ruby interface defines local functions globally.
解决方案: Make the functions static.
相关文件: src/if_ruby.c

Patch 7.3.722
问题: Perl flags may contain "-g", which breaks "make proto".
解决方案: Filter out the "-g" flag for cproto. (Ken Takata)
相关文件: src/Makefile

Patch 7.3.723
问题: Various tiny problems.
解决方案: Various tiny fixes.
相关文件: src/gui_mac.c, src/xpm_w32.c, src/netbeans.c, src/sha256.c, src/if_sniff.c, README.txt

Patch 7.3.724
问题: Building with Ruby and Tcl on MS-Windows 64 bit does not work.
解决方案: Remove Ruby and Tcl from the big MS-Windows build.
相关文件: src/bigvim64.bat

Patch 7.3.725
问题: :aboveleft and :belowright have no effect on :copen.

解决方案: Check for cmdmod.split. (Christian Brabandt)
相关文件: src/quickfix.c

Patch 7.3.726
问题: Typos and duplicate info in README.
解决方案: Fix the text.
相关文件: README.txt

Patch 7.3.727
问题: Can't always find Win32.mak when building GvimExt.
解决方案: Use same mechanism as in Make_mvc.mak. (Cade Foster)
相关文件: src/GvimExt/Makefile

Patch 7.3.728
问题: Cannot compile with MzScheme interface on Ubuntu 12.10.
解决方案: Find the collects directory under /usr/share.
相关文件: src/configure.in, src/auto/configure

Patch 7.3.729
问题: Building with Ruby fails on some systems.
解决方案: Remove "static" and add #ifndef PROTO. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.3.730
问题: Crash in PHP file when using syntastic. (Ike Devolder)
解决方案: Avoid using NULL pointer. (Christian Brabandt)
相关文件: src/quickfix.c

Patch 7.3.731
问题: Py3Init_vim() is exported unnecessarily.
解决方案: Make it static. (Ken Takata)
相关文件: src/if_python3.c

Patch 7.3.732
问题: Compiler warnings for function arguments.
解决方案: Use inteptr_t instead of long.
相关文件: src/if_mzsch.c, src/main.c

Patch 7.3.733
问题: Tests fail when including MzScheme.
解决方案: Change #ifdefs for vim_main2().
相关文件: src/main.c

Patch 7.3.734
问题: Cannot put help files in a sub-directory.
解决方案: Make :helptags work for sub-directories. (Charles Campbell)
相关文件: src/ex_cmds.c

Patch 7.3.735
问题: Cannot build Ruby 1.9 with MingW or Cygwin.
解决方案: Add another include directory. (Ken Takata)
相关文件: src/Make_cyg.mak, src/Make_ming.mak

Patch 7.3.736

问题:　　File name completion in input() escapes white space. (Frederic Hardy)
解决方案:　Do not escape white space. (Christian Brabandt)
相关文件:　src/ex_getln.c

Patch 7.3.737
问题:　　When using do_cmdline() recursively did_endif is not reset, causing messages to be overwritten.
解决方案:　Reset did_endif. (Christian Brabandt)
相关文件:　src/ex_docmd.c

Patch 7.3.738 (after 7.3.730)
问题:　　Unused function argument.
解决方案:　Remove it. (Christian Brabandt)
相关文件:　src/quickfix.c

Patch 7.3.739
问题:　　Computing number of lines may have an integer overflow.
解决方案:　Check for MAXCOL explicitly. (Dominique Pelle)
相关文件:　src/move.c

Patch 7.3.740
问题:　　IOC tool complains about undefined behavior for int.
解决方案:　Change to unsigned int. (Dominique Pelle)
相关文件:　src/hashtab.c, src/misc2.c

Patch 7.3.741 (after 7.3.737)
问题:　　Tiny build fails.
解决方案:　Move #ifdef. (Ike Devolder)
相关文件:　src/ex_docmd.c

Patch 7.3.742
问题:　　Leaking memory when :vimgrep restores the directory.
解决方案:　Free the allocated memory. (Christian Brabandt)
相关文件:　src/quickfix.c

Patch 7.3.743 (after 7.3.741)
问题:　　Tiny build still fails.
解决方案:　Add #else in the right place.
相关文件:　src/ex_docmd.c

Patch 7.3.744
问题:　　64 bit compiler warning.
解决方案:　Add type cast. (Mike Williams)
相关文件:　src/ex_cmds.c

Patch 7.3.745
问题:　　Automatically setting 'ttymouse' doesn't work.
解决方案:　Reset the "option was set" flag when using the default.
相关文件:　src/option.c, src/proto/option.pro, src/term.c

Patch 7.3.746
问题:　　Memory leaks when using location lists.
解决方案:　Set qf_title to something. (Christian Brabandt)

相关文件: src/eval.c, src/quickfix.c

Patch 7.3.747
问题: When characters are concealed text aligned with tabs are no longer aligned, e.g. at ":help :index".
解决方案: Compensate space for tabs for concealed characters. (Dominique Pelle)
相关文件: src/screen.c

Patch 7.3.748
问题: Cannot properly test conceal mode.
解决方案: Add the screencol() and screenrow() functions. Use them in test88. (Simon Ruderich)
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/screen.pro, src/screen.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms src/testdir/Makefile, src/testdir/test88.in, src/testdir/test88.ok,

Patch 7.3.749
问题: Python interface doesn't build without the multi-byte feature.
解决方案: Add #ifdef. (Ken Takata)
相关文件: src/if_py_both.h

Patch 7.3.750
问题: The justify macro does not always work correctly.
解决方案: Fix off-by-one error (James McCoy)
相关文件: runtime/macros/justify.vim

Patch 7.3.751
问题: Test 61 is flaky, it fails once in a while.
解决方案: When it fails retry once.
相关文件: src/testdir/Makefile

Patch 7.3.752
问题: Test 49 script file doesn't fold properly.
解决方案: Add a colon.
相关文件: src/testdir/test49.vim

Patch 7.3.753
问题: When there is a QuitPre autocommand using ":q" twice does not work for exiting when there are more files to edit.
解决方案: Do not decrement quitmore in an autocommand. (Techlive Zheng)
相关文件: src/ex_docmd.c, src/fileio.c, src/proto/fileio.pro

Patch 7.3.754
问题: Latest nmake is not recognized.
解决方案: Add nmake version 11.00.51106.1. (Raymond Ko)
相关文件: src/Make_mvc.mak

Patch 7.3.755
问题: Autoconf doesn't find Python 3 if it's called "python".
解决方案: Search for "python2" and "python3" first, then "python".

相关文件: src/configure.in, src/auto/configure

Patch 7.3.756
问题: A location list can get a wrong count in :lvimgrep.
解决方案: Check if the list was changed by autocommands. (mostly by
Christian Brabandt)
相关文件: src/quickfix.c

Patch 7.3.757
问题: Issue 96: May access freed memory when a put command triggers
autocommands. (Dominique Pelle)
解决方案: Call u_save() before getting y_array.
相关文件: src/ops.c

Patch 7.3.758
问题: Matchit plugin does not handle space in #ifdef.
解决方案: Change matching pattern to allow spaces. (Mike Morearty)
相关文件: runtime/macros/matchit.vim

Patch 7.3.759
问题: MS-Windows: Updating the tabline is slow when there are many tabs.
解决方案: Disable redrawing while performing the update. (Arseny Kapoulkine)
相关文件: src/gui_w48.c

Patch 7.3.760
问题: dv_ deletes the white space before the line.
解决方案: Move the cursor to the first non-white. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test19.in, src/testdir/test19.ok

Patch 7.3.761
问题: In Visual mode a "-p does not work. (Marcin Szamotulski)
解决方案: Avoid writing to "- before putting it. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test48.in, src/testdir/test48.ok

Patch 7.3.762 (after 7.3.759)
问题: On some systems the tabline is not redrawn.
解决方案: Call RedrawWindow(). (Charles Peacech)
相关文件: src/gui_w48.c

Patch 7.3.763
问题: Jumping to a mark does not open a fold if it is in the same line.
(Wiktor Ruben)
解决方案: Also compare the column after the jump. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.3.764
问题: Not all message translation files are installed.
解决方案: Also install the converted files.
相关文件: src/po/Makefile

Patch 7.3.765
问题: Segfault when doing "cclose" on BufUnload in a python function.
(Sean Reifschneider)
解决方案: Skip window with NULL buffer. (Christian Brabandt)

相关文件: src/main.c, src/window.c

Patch 7.3.766
问题: ":help cpo-*" jumps to the wrong place.
解决方案: Make it equivalent to ":help cpo-star".
相关文件: src/ex_cmds.c

Patch 7.3.767
问题: (Win32) The _errno used for iconv may be the wrong one.
解决方案: Use the _errno from iconv.dll. (Ken Takata)
相关文件: src/mbyte.c

Patch 7.3.768
问题: settabvar() and setwinvar() may move the cursor.
解决方案: Save and restore the cursor position when appropriate. (idea by
Yasuhiro Matsumoto)
相关文件: src/edit.c

Patch 7.3.769
问题: 'matchpairs' does not work with multi-byte characters.
解决方案: Make it work. (Christian Brabandt)
相关文件: src/misc1.c, src/option.c, src/proto/option.pro, src/search.c,
src/testdir/test69.in, src/testdir/test69.ok

Patch 7.3.770
问题: Vim.h indentation is inconsistent.
解决方案: Adjust the indentation. (Elias Diem)
相关文件: src/vim.h

Patch 7.3.771 (after 7.3.769)
问题: Uninitialized variable. (Yasuhiro Matsumoto)
解决方案: Set x2 to -1.
相关文件: src/option.c

Patch 7.3.772
问题: Cursor is at the wrong location and below the end of the file
after doing substitutions with confirm flag: %s/x/y/c
(Dominique Pelle)
解决方案: Update the cursor position. (Christian Brabandt & Dominique)
相关文件: src/ex_cmds.c

Patch 7.3.773 (after 7.3.767)
问题: Crash when OriginalFirstThunk is zero.
解决方案: Skip items with OriginalFirstThunk not set. (Ken Takata)
相关文件: src/mbyte.c

Patch 7.3.774
问题: Tiny GUI version misses console dialog feature.
解决方案: Define FEAT_CON_DIALOG when appropriate. (Christian Brabandt)
相关文件: src/feature.h, src/gui.h

Patch 7.3.775
问题: Cygwin and Mingw builds miss dependency on gui_w48.c.
解决方案: Add a build rule. (Ken Takata)

相关文件: src/Make_cyg.mak, src/Make_ming.mak

Patch 7.3.776
问题: ml_get error when searching, caused by curwin not matching curbuf.
解决方案: Avoid changing curbuf. (Lech Lorens)
相关文件: src/charset.c, src/eval.c, src/mark.c, src/proto/charset.pro,
src/proto/mark.pro, src/regexp.c, src/syntax.c,

Patch 7.3.777
问题: When building with Gnome locale gets reset.
解决方案: Set locale after gnome_program_init(). (Christian Brabandt)
相关文件: src/gui_gtk_x11.c

Patch 7.3.778
问题: Compiler error for adding up two pointers. (Titov Anatoly)
解决方案: Add a type cast. (Ken Takata)
相关文件: src/mbyte.c

Patch 7.3.779
问题: Backwards search lands in wrong place when started on a multibyte
character.
解决方案: Do not set extra_col for a backwards search. (Sung Pae)
相关文件: src/search.c, src/testdir/test44.in, src/testdir/test44.ok

Patch 7.3.780
问题: char2nr() and nr2char() always use 'encoding'.
解决方案: Add argument to use utf-8 characters. (Yasuhiro Matsumoto)
相关文件: runtime/doc/eval.txt, src/eval.c

Patch 7.3.781
问题: Drawing with 'guifontwide' can be slow.
解决方案: Draw multiple characters at a time. (Taro Muraoka)
相关文件: src/gui.c

Patch 7.3.782
问题: Windows: IME composition may use a wrong font.
解决方案: Use 'guifontwide' for IME when it is set. (Taro Muraoka)
相关文件: runtime/doc/options.txt, src/gui.c, src/gui_w48.c,
src/proto/gui_w16.pro, src/proto/gui_w32.pro

Patch 7.3.783
问题: Crash when mark is not set. (Dominique Pelle)
解决方案: Check for NULL.
相关文件: src/normal.c

Patch 7.3.784 (after 7.3.781)
问题: Error when 'guifontwide' has a comma.
解决方案: Use gui.wide_font. (Taro Muraoka)
相关文件: src/gui_w48.c

Patch 7.3.785 (after 7.3.776)
问题: Crash with specific use of search pattern.
解决方案: Initialize reg_buf to curbuf.
相关文件: src/regexp.c

Patch 7.3.786
问题: Python threads don't run in the background (issue 103).
解决方案: Move the statements to manipulate thread state.
相关文件: src/if_python.c

Patch 7.3.787
问题: With 'relativenumber' set it is not possible to see the absolute line number.
解决方案: For the cursor line show the absolute line number instead of a zero. (Nazri Ramliy)
相关文件: src/screen.c

Patch 7.3.788
问题: When only using patches build fails on missing nl.po.
解决方案: Create an empty nl.po file.
相关文件: src/po/Makefile

Patch 7.3.789 (after 7.3.776)
问题: "\k" in regexp does not work in other window.
解决方案: Use the right buffer. (Yukihiro Nakadaira)
相关文件: src/mbyte.c, src/proto/mbyte.pro, src/regexp.c

Patch 7.3.790
问题: After reloading a buffer the modelines are not processed.
解决方案: call do_modelines(). (Ken Takata)
相关文件: src/fileio.c

Patch 7.3.791
问题: MzScheme interface doesn't work properly.
解决方案: Make it work better. (Sergey Khorev)
相关文件: runtime/doc/if_mzsch.txt, src/configure.in, src/auto/configure, src/eval.c, src/if_mzsch.c, src/if_mzsch.h, src/Make_ming.mak, src/Make_mvc.mak, src/os_unix.c, src/proto/eval.pro, src/testdir/test70.in, src/testdir/test70.ok

Patch 7.3.792
问题: ":substitute" works differently without confirmation.
解决方案: Do not change the text when asking for confirmation, only display it.
相关文件: src/ex_cmds.c

Patch 7.3.793 (after 7.3.792)
问题: New interactive :substitute behavior is not tested.
解决方案: Add tests. (Christian Brabandt)
相关文件: src/testdir/test80.in, src/testdir/test80.ok

Patch 7.3.794
问题: Tiny build fails. (Tony Mechelynck)
解决方案: Adjust #ifdefs.
相关文件: src/charset.c

Patch 7.3.795
问题: MzScheme does not build with tiny features.

解决方案: Add #ifdefs. Also add UNUSED to avoid warnings. And change
           library ordering.
相关文件: src/if_mzsch.c, src/Makefile

Patch 7.3.796
问题: "/[^\n]" does match at a line break.
解决方案: Make it do the same as "/.". (Christian Brabandt)
相关文件: src/regexp.c, src/testdir/test79.in, src/testdir/test79.ok

Patch 7.3.797 (after 7.3.792)
问题: Compiler warning for size_t to int conversion. (Skeept)
解决方案: Add type casts.
相关文件: src/ex_cmds.c

Patch 7.3.798 (after 7.3.791)
问题: MzScheme: circular list does not work correctly.
解决方案: Separate Mac-specific code from generic code. (Sergey Khorev)
相关文件: src/if_mzsch.c, src/testdir/test70.in

Patch 7.3.799
问题: The color column is not correct when entering a buffer. (Ben
           Fritz)
解决方案: Call check_colorcolumn() if 'textwidth' changed. (Christian
           Brabandt)
相关文件: src/buffer.c

Patch 7.3.800
问题: The " mark is not adjusted when inserting lines. (Roland Eggner)
解决方案: Adjust the line number. (Christian Brabandt)
相关文件: src/mark.c

Patch 7.3.801
问题: ":window set nu?" displays the cursor line. (Nazri Ramliy)
解决方案: Do not update the cursor line when conceallevel is zero or the
           screen has scrolled. (partly by Christian Brabandt)
相关文件: src/window.c

Patch 7.3.802
问题: After setting 'isk' to a value ending in a comma appending to the
           option fails.
解决方案: Disallow a trailing comma for 'isk' and similar options.
相关文件: src/charset.c

Patch 7.3.803 (after 7.3.792)
问题: Substitute with confirmation and then "q" does not replace
           anything. (John McGowan)
解决方案: Do not break the loop, skip to the end.
相关文件: src/ex_cmds.c, src/testdir/test80.in, src/testdir/test80.ok

Patch 7.3.804 (after 7.3.799)
问题: Compiler warning for tiny build. (Tony Mechelynck)
解决方案: Add #ifdefs around variable.
相关文件: src/buffer.c

Patch 7.3.805
问题: Lua version 5.2 is not detected properly on Arch Linux.
解决方案: Adjust autoconf. (lilydjwg)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.806
问题: Compiler warnings in Perl code when building with Visual studio 2012. (skeept)
解决方案: Add type casts. (Christian Brabandt, 2013 Jan 30)
相关文件: src/if_perl.xs

Patch 7.3.807
问题: Popup menu does not work properly with the preview window, folds and 'cursorcolumn'.
解决方案: Redraw the popup menu after redrawing windows. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.3.808
问题: Python threads still do not work properly.
解决方案: Fix both Python 2 and 3. Add tests. (Ken Takata)
相关文件: src/if_python.c, src/if_python3.c, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.809
问题: The dosinst.c program has a buffer overflow. (Thomas Gwae)
解决方案: Ignore $VIMRUNTIME if it is too long.
相关文件: src/dosinst.c

Patch 7.3.810
问题: 'relativenumber' is reset unexpectedly. (Franois Ingelrest)
解决方案: After an option was reset also reset the global value. Add a test. (Christian Brabandt)
相关文件: src/option.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test89.in, src/testdir/test89.ok

Patch 7.3.811
问题: Useless termresponse parsing for SGR mouse.
解决方案: Skip the parsing. (Hayaki Saito)
相关文件: src/term.c

Patch 7.3.812
问题: When 'indentexpr' moves the cursor "curswant" not restored.
解决方案: Restore "curswant". (Sung Pae)
相关文件: src/misc1.c

Patch 7.3.813
问题: The CompleteDone event is not triggered when there are no pattern matches. (Jianjun Mao)
解决方案: Trigger the event. (Christian Brabandt)

相关文件: src/edit.c

Patch 7.3.814
问题: Can't input multibyte characters on Win32 console if 'encoding' is different from current codepage.
解决方案: Use convert_input_safe() instead of convert_input(). Make string_convert_ext() return an error for incomplete input. (Ken Takata)
相关文件: src/mbyte.c, src/os_win32.c

Patch 7.3.815
问题: Building with Cygwin and Ruby doesn't work.
解决方案: Copy some things from the MingW build file. (Ken Takata)
相关文件: src/Make_cyg.mak

Patch 7.3.816
问题: Can't compute a hash.
解决方案: Add the sha256() function. (Tyru, Hirohito Higashi)
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/sha256.pro, src/sha256.c, src/testdir/test90.in, src/testdir/test90.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.3.817
问题: Test 89 fails with tiny and small features.
解决方案: Add sourcing small.vim.
相关文件: src/testdir/test89.in

Patch 7.3.818
问题: When test 40 fails because of a bad build it may leave files behind that cause it to fail later.
解决方案: Let the file names start with "X".
相关文件: src/testdir/test40.in

Patch 7.3.819
问题: Compiling without +eval and with Python isn't working.
解决方案: Add the eval feature when building with Python.
相关文件: src/if_py_both.h, src/feature.h, src/eval.c, src/ex_docmd.c, src/normal.c, src/ex_docmd.c, src/gui_gtk_x11.c

Patch 7.3.820
问题: Build errors and warnings when building with small features and Lua, Perl or Ruby.
解决方案: Add #ifdefs and UNUSED.
相关文件: src/if_perl.xs, src/if_lua.c, src/if_ruby.c

Patch 7.3.821
问题: Build with OLE and Cygwin is broken. (Steve Hall)
解决方案: Select static or shared stdc library. (Ken Takata)
相关文件: src/Make_cyg.mak

Patch 7.3.822 (after 7.3.799)
问题: Crash when accessing freed buffer.

解决方案: Get 'textwidth' in caller of enter_buffer(). (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.3.823 (after 7.3.821)
问题: Building with Cygwin: '-lsupc++' is not needed.
解决方案: Remove it. (Ken Takata)
相关文件: src/Make_cyg.mak

Patch 7.3.824
问题: Can redefine builtin functions. (ZyX)
解决方案: Disallow adding a function to g:.
相关文件: src/eval.c

Patch 7.3.825
问题: With Python errors are not always clear.
解决方案: Print the stack trace, unless :silent is used. (ZyX)
相关文件: src/if_python3.c, src/if_python.c

Patch 7.3.826
问题: List of features in :version output is hard to read.
解决方案: Make columns. (Nazri Ramliy)
相关文件: src/version.c

Patch 7.3.827 (after 7.3.825)
问题: Python tests fail.
解决方案: Adjust the output for the stack trace.
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.ok

Patch 7.3.828
问题: Mappings are not aware of wildmenu mode.
解决方案: Add wildmenumode(). (Christian Brabandt)
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 7.3.829
问题: When compiled with the +rightleft feature 'showmatch' also shows a
match for the opening paren. When 'revins' is set the screen may
scroll.
解决方案: Only check the opening paren when the +rightleft feature was
enabled. Do not show a match that is not visible. (partly by
Christian Brabandt)
相关文件: src/search.c

Patch 7.3.830
问题: :mksession confuses bytes, columns and characters when positioning
the cursor.
解决方案: Use w_virtcol with "|" instead of w_cursor.col with "l".
相关文件: src/ex_docmd.c

Patch 7.3.831
问题: Clumsy to handle the situation that a variable does not exist.
解决方案: Add default value to getbufvar() et al. (Shougo Matsushita,
Hirohito Higashi)
相关文件: runtime/doc/eval.txt, src/eval.c src/testdir/test91.in,

src/testdir/test91.ok, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile

Patch 7.3.832
问题: Compiler warning.
解决方案: Add type cast. (Mike Williams)
相关文件: src/version.c

Patch 7.3.833
问题: In the terminal the scroll wheel always scrolls the active window.
解决方案: Scroll the window under the mouse pointer, like in the GUI.
(Bradie Rao)
相关文件: src/edit.c, src/normal.c

Patch 7.3.834
问题: Ruby 2.0 has a few API changes.
解决方案: Add handling of Ruby 2.0. (Yasuhiro Matsumoto)
相关文件: src/if_ruby.c

Patch 7.3.835
问题: "xxd -i" fails on an empty file.
解决方案: Do output the closing } for an empty file. (partly by Lawrence
Woodman)
相关文件: src/xxd/xxd.c

Patch 7.3.836
问题: Clipboard does not work on Win32 when compiled with Cygwin.
解决方案: Move the Win32 clipboard code to a separate file and use it when
building with os_unix.c. (Frodak Baksik, Ken Takata)
相关文件: src/Make_bc5.mak, src/Make_cyg.mak, src/Make_ivc.mak,
src/Make_ming.mak, src/Make_mvc.mak, src/Make_w16.mak,
src/Makefile, src/config.h.in, src/configure.in,
src/auto/configure, src/feature.h, src/globals.h, src/mbyte.c,
src/os_mswin.c, src/os_unix.c, src/os_win32.c, src/proto.h,
src/proto/os_mswin.pro, src/proto/winclip.pro, src/term.c,
src/vim.h, src/winclip.c

Patch 7.3.837 (after 7.3.826)
问题: Empty lines in :version output when 'columns' is 320.
解决方案: Simplify the logic of making columns. (Nazri Ramliy, Roland
Eggner)
相关文件: src/version.c

Patch 7.3.838 (after 7.3.830)
问题: Insufficient testing for mksession.
解决方案: Add tests. (mostly by Roland Eggner)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test92.in, src/testdir/test92.ok,
src/testdir/test93.in, src/testdir/test93.ok,
src/ex_docmd.c

Patch 7.3.839
问题:　　Some files missing in the list of distributed files.
解决方案:　Add lines for new files.
相关文件:　Filelist

Patch 7.3.840
问题:　　"\@<!" in regexp does not work correctly with multi-byte
　　characters, especially cp932.
解决方案:　Move column to start of multi-byte character. (Yasuhiro Matsumoto)
相关文件:　src/regexp.c

Patch 7.3.841
问题:　　When a "cond ? one : two" expression has a subscript it is not
　　parsed correctly. (Andy Wokula)
解决方案:　Handle a subscript also when the type is unknown. (Christian
　　Brabandt)
相关文件:　src/eval.c

Patch 7.3.842
问题:　　Compiler warning for signed/unsigned pointer.
解决方案:　Add type cast. (Christian Brabandt)
相关文件:　src/eval.c

Patch 7.3.843 (after 7.3.841)
问题:　　Missing test file changes.
解决方案:　Change the tests.
相关文件:　src/testdir/test49.vim, src/testdir/test49.ok

Patch 7.3.844
问题:　　Enum is not indented correctly with "public" etc.
解决方案:　Skip "public", "private" and "protected". (Hong Xu)
相关文件:　src/misc1.c

Patch 7.3.845 (after 7.3.844)
问题:　　Enum indenting is not tested.
解决方案:　Add tests. (Hong Xu)
相关文件:　src/testdir/test3.in, src/testdir/test3.ok

Patch 7.3.846
问题:　　Missing proto files.
解决方案:　Add the files.
相关文件:　Filelist, src/proto/os_beos.pro

Patch 7.3.847
问题:　　Test 55 fails when messages are translated.
解决方案:　Set language to C. (Ken Takata)
相关文件:　src/testdir/test55.in

Patch 7.3.848
问题:　　Can't build with Ruby 2.0 when using MinGW x64 or MSVC10.
解决方案:　Fix it. Also detect RUBY_PLATFORM and RUBY_INSTALL_NAME for x64.
　　(Ken Takata)
相关文件:　src/Make_cyg.mak, src/Make_ming.mak, src/if_ruby.c

Patch 7.3.849
问题: ":g//" gives "Pattern not found error" with E486. Should not use the error number, it's not a regular error message.
解决方案: Use a normal message. (David Bürgin)
相关文件: src/ex_cmds.c

Patch 7.3.850
问题: ":vimgrep //" matches everywhere.
解决方案: Make it use the previous search pattern. (David Bürgin)
相关文件: runtime/doc/quickfix.txt, src/quickfix.c

Patch 7.3.851
问题: Using an empty pattern with :sort silently continues when there is no previous search pattern.
解决方案: Give an error message. (David Bürgin)
相关文件: src/ex_cmds.c

Patch 7.3.852
问题: system() breaks clipboard text. (Yukihiro Nakadaira)
解决方案: Use Xutf8TextPropertyToTextList(). (Christian Brabandt)
Also do not put the text in the clip buffer if conversion fails.
相关文件: src/ui.c, src/ops.c

Patch 7.3.853
问题: Using "ra" in multiple lines on multi-byte characters leaves a few characters not replaced.
解决方案: Adjust the end column only in the last line. (Yasuhiro Matsumoto)
相关文件: src/testdir/test69.in, src/testdir/test69.ok, src/ops.c

Patch 7.3.854
问题: After using backspace in insert mode completion, CTRL-N and CTRL-P do not highlight the right entry. (Olivier Teuliere)
解决方案: Set the current item to the shown item after using backspace.
相关文件: src/edit.c

Patch 7.3.855
问题: Compiler warnings.
解决方案: Add type casts. (Mike Williams)
相关文件: src/misc1.c

Patch 7.3.856
问题: When calling system() multi-byte clipboard contents is garbled.
解决方案: Save and restore the clipboard contents. (Yukihiro Nakadaira)
相关文件: src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro, src/ops.c, src/proto/ops.pro, src/os_unix.c, src/proto/ui.pro, src/ui.c

Patch 7.3.857
问题: The QuitPre autocommand event does not trigger for :qa and :wq.
解决方案: Trigger the event. (Tatsuro Fujii)
相关文件: src/ex_docmd.c

Patch 7.3.858
问题: "gv" selects the wrong area after some operators.

解决方案: Save and restore the type of selection. (Christian Brabandt)
相关文件: src/testdir/test66.in, src/testdir/test66.ok, src/normal.c

Patch 7.3.859
问题: 'ambiwidth' must be set by the user.
解决方案: Detects East Asian ambiguous width (UAX #11) state of the terminal
at the start-up time and 'ambiwidth' accordingly. (Hayaki Saito)
相关文件: src/main.c, src/option.c, src/term.c, src/term.h,
src/proto/term.pro

Patch 7.3.860
问题: When using --remote-expr try/catch does not work. (Andrey Radev)
解决方案: Set emsg_silent instead of emsg_skip.
相关文件: src/main.c

Patch 7.3.861
问题: ":setlocal number" clears global value of 'relativenumber'.
解决方案: Do it properly. (Markus Heidelberg)
相关文件: src/testdir/test89.in, src/testdir/test89.ok, src/option.c

Patch 7.3.862
问题: Dragging the status line can be slow.
解决方案: Look ahead and drop the drag event if there is a next one.
相关文件: src/eval.c, src/misc1.c, src/proto/misc1.pro, src/normal.c

Patch 7.3.863 (after 7.3.859)
问题: Problem with 'ambiwidth' detection for ANSI terminal.
解决方案: Work around not recognizing a term response. (Hayaki Saito)
相关文件: src/term.c

Patch 7.3.864 (after 7.3.862)
问题: Can't build without the mouse feature.
解决方案: Add an #ifdef. (Ike Devolder)
相关文件: src/misc1.c

Patch 7.3.865 (after 7.3.862)
问题: Mouse position may be wrong.
解决方案: Let vungetc() restore the mouse position.
相关文件: src/getchar.c

Patch 7.3.866
问题: Not serving the X selection during system() isn't nice.
解决方案: When using fork() do not loose the selection, keep serving it.
Add a loop similar to handling I/O. (Yukihiro Nakadaira)
相关文件: src/os_unix.c

Patch 7.3.867
问题: Matchparen does not update match when using auto-indenting.
(Marc Aldorasi)
解决方案: Add the TextChanged and TextChangedI autocommand events.
相关文件: runtime/plugin/matchparen.vim, src/main.c, src/edit.c,
src/globals.h, src/vim.h, src/fileio.c, src/proto/fileio.pro,
runtime/doc/autocmd.txt

Patch 7.3.868
问题: When at the hit-return prompt and using "k" while no text has scrolled off screen, then using "j", an empty line is displayed.
解决方案: Only act on "k" when text scrolled off screen. Also accept page-up and page-down. (cptstubing)
相关文件: src/message.c

Patch 7.3.869
问题: bufwinnr() matches buffers in other tabs.
解决方案: For bufwinnr() and ? only match buffers in the current tab. (Alexey Radkov)
相关文件: src/buffer.c, src/diff.c, src/eval.c, src/ex_docmd.c, src/if_perl.xs, src/proto/buffer.pro

Patch 7.3.870
问题: Compiler warnings when using MingW 4.5.3.
解决方案: Do not use MAKEINTRESOURCE. Adjust #if. (Ken Takata)
相关文件: src/gui_w32.c, src/gui_w48.c, src/os_mswin.c, src/os_win32.c, src/os_win32.h

Patch 7.3.871
问题: search('^$', 'c') does not use the empty match under the cursor.
解决方案: Special handling of the 'c' flag. (Christian Brabandt) Add tests.
相关文件: src/search.c, src/testdir/test14.in, src/testdir/test14.ok

Patch 7.3.872
问题: On some systems case of file names is always ignored, on others never.
解决方案: Add the 'fileignorecase' option to control this at runtime. Implies 'wildignorecase'.
相关文件: src/buffer.c, src/edit.c, src/ex_cmds2.c, src/ex_getln.c, src/fileio.c, src/misc1.c, src/misc2.c, src/option.c, src/option.h, src/vim.h, runtime/doc/options.txt

Patch 7.3.873
问题: Cannot easily use :s to make title case.
解决方案: Have "\L\u" result in title case. (James McCoy)
相关文件: src/regexp.c, src/testdir/test79.in, src/testdir/test79.ok, src/testdir/test80.in, src/testdir/test80.ok

Patch 7.3.874
问题: Comparing file names does not handle multi-byte characters properly.
解决方案: Implement multi-byte handling.
相关文件: src/misc1.c, src/misc2.c

Patch 7.3.875 (after 7.3.866)
问题: Build problem with some combination of features.
解决方案: Use FEAT_XCLIPBOARD instead of FEAT_CLIPBOARD.
相关文件: src/os_unix.c

Patch 7.3.876
问题: #if indents are off.

解决方案: Insert a space where appropriate. (Taro Muraoka)
相关文件: src/gui.c

Patch 7.3.877 (after 7.3.871)
问题: Forward searching with search() is broken.
解决方案: Fix it and add tests. (Sung Pae)
相关文件: src/search.c, src/testdir/test14.in, src/testdir/test14.ok

Patch 7.3.878
问题: 'fileignorecase' is missing in options window and quickref.
解决方案: Add the option.
相关文件: runtime/optwin.vim, runtime/doc/quickref.txt

Patch 7.3.879
问题: When using an ex command in operator pending mode, using Esc to
abort the command still executes the operator. (David Bürgin)
解决方案: Clear the operator when the ex command fails. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.3.880
问题: When writing viminfo, old history lines may replace lines written
more recently by another Vim instance.
解决方案: Mark history entries that were read from viminfo and overwrite
them when merging with the current viminfo.
相关文件: src/ex_getln.c

Patch 7.3.881
问题: Python list does not work correctly.
解决方案: Fix it and add a test. (Yukihiro Nakadaira)
相关文件: src/testdir/test86.in, src/testdir/test86.ok, src/if_py_both.h

Patch 7.3.882
问题: CursorHold may trigger after receiving the termresponse.
解决方案: Set the did_cursorhold flag. (Hayaki Saito)
相关文件: src/term.c

Patch 7.3.883 (after 7.3.880)
问题: Can't build with some combination of features.
解决方案: Adjust #ifdefs.
相关文件: src/ex_getln.c

Patch 7.3.884
问题: Compiler warning for variable shadowing another. (John Little)
解决方案: Rename the variable. (Christian Brabandt)
相关文件: src/term.c

Patch 7.3.885
问题: Double free for list and dict in Lua. (Shougo Matsu)
解决方案: Do not unref list and dict. (Yasuhiro Matsumoto)
相关文件: src/if_lua.c

Patch 7.3.886
问题: Can't build with multi-byte on Solaris 10.
解决方案: Add #ifdef X_HAVE_UTF8_STRING. (Laurent Blume)

相关文件: src/ui.c

Patch 7.3.887
问题: No tests for Visual mode operators, what 7.3.879 fixes.
解决方案: Add a new test file. (David Bürgin)
相关文件: src/testdir/test94.in, src/testdir/test94.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.3.888
问题: Filename completion with 'fileignorecase' does not work for
multi-byte characters.
解决方案: Make 'fileignorecase' work properly. (Hirohito Higashi)
相关文件: src/misc2.c

Patch 7.3.889
问题: Can't build with Ruby 2.0 on a 64 bit system.
解决方案: Define rb_fix2int and rb_num2int. (Kohei Suzuki)
相关文件: src/if_ruby.c

Patch 7.3.890
问题: Test 79 fails on Windows. (Michael Soyka)
解决方案: Add comment below line causing an error.
相关文件: src/testdir/test79.in

Patch 7.3.891
问题: Merging viminfo history doesn't work well.
解决方案: Don't stop when one type of history is empty. Don't merge history
when writing viminfo.
相关文件: src/ex_getln.c

Patch 7.3.892 (after 7.3.891)
问题: Still merging problems for viminfo history.
解决方案: Do not merge lines when writing, don't write old viminfo lines.
相关文件: src/ex_getln.c, src/ex_cmds.c, src/proto/ex_getln.pro

Patch 7.3.893
问题: Crash when using b:, w: or t: after closing the buffer, window or
tabpage.
解决方案: Allocate the dictionary instead of having it part of the
buffer/window/tabpage struct. (Yukihiro Nakadaira)
相关文件: src/buffer.c, src/eval.c, src/fileio.c, src/structs.h,
src/window.c, src/proto/eval.pro

Patch 7.3.894
问题: Using wrong RUBY_VER causing Ruby build to break.
解决方案: Correct the RUBY_VER value. (Yongwei Wu)
相关文件: src/bigvim.bat

Patch 7.3.895
问题: Valgrind error in test 91. (Issue 128)
解决方案: Pass scope name to find_var_in_ht().
相关文件: src/eval.c

Patch 7.3.896
问题:　　Memory leaks in Lua interface.
解决方案:　Fix the leaks, add tests. (Yukihiro Nakadaira)
相关文件:　src/testdir/test85.in, src/testdir/test85.ok, src/if_lua.c

Patch 7.3.897
问题:　　Configure doesn't always find the shared library.
解决方案:　Change the configure script. (Ken Takata)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.898
问题:　　Memory leak reported by valgrind in test 91.
解决方案:　Only use default argument when needed.
相关文件:　src/eval.c, src/testdir/test91.in, src/testdir/test91.ok

Patch 7.3.899
问题:　　#if indents are off.
解决方案:　Fix the indents.
相关文件:　src/os_unix.c

Patch 7.3.900
问题:　　Not obvious that some mouse features are mutual-exclusive.
解决方案:　Add a comment.
相关文件:　src/feature.h

Patch 7.3.901
问题:　　Outdated comment, ugly condition.
解决方案:　Update a few comments, break line.
相关文件:　src/getchar.c, src/misc1.c, src/undo.c

Patch 7.3.902
问题:　　When deleting last buffer in other tab the tabline is not updated.
解决方案:　Set the redraw_tabline flag. (Yukihiro Nakadaira)
相关文件:　src/window.c

Patch 7.3.903 (after 7.3.892)
问题:　　Crash on exit writing viminfo. (Ron Aaron)
解决方案:　Check for the history to be empty.
相关文件:　src/ex_getln.c

Patch 7.3.904 (after 7.3.893)
问题:　　Using memory freed by the garbage collector.
解决方案:　Mark items in aucmd_win as used.
相关文件:　src/eval.c

Patch 7.3.905 (after 7.3.903)
问题:　　Crash when writing viminfo. (Ron Aaron)
解决方案:　Prevent freed history info to be used.
相关文件:　src/ex_getln.c

Patch 7.3.906
问题:　　The "sleep .2" for running tests does not work on Solaris.
解决方案:　Fall back to using "sleep 1". (Laurent Blume)

相关文件: src/testdir/Makefile

Patch 7.3.907
问题: Python uses IndexError when a dict key is not found.
解决方案: Use KeyError instead. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c,
src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.908
问题: Possible crash when using a list in Python.
解决方案: Return early if the list is NULL. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.909
问题: Duplicate Python code.
解决方案: Move more items to if_py_both.h. (ZyX) Also avoid compiler
warnings for missing initializers.
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.910
问题: Python code in #ifdef branches with only minor differences.
解决方案: Merge the #ifdef branches. (ZyX)
相关文件: src/if_py_both.h, src/if_python.c

Patch 7.3.911
问题: Python: Access to Vim variables is not so easy.
解决方案: Define vim.vars and vim.vvars. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/eval.c, src/globals.h,
src/if_py_both.h, src/if_python3.c, src/if_python.c,
src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.912
问题: Typing a ":" command at the hit-enter dialog does not work if the
"file changed" dialog happens next.
解决方案: Check for changed files before giving the hit-enter dialog.
相关文件: src/message.c

Patch 7.3.913 (after 7.3.905)
问题: Still a crash when writing viminfo.
解决方案: Add checks for NULL pointers. (Ron Aaron)
相关文件: src/ex_getln.c

Patch 7.3.914
问题: ~/.viminfo is messed up when running tests.
解决方案: Set the viminfo filename.
相关文件: src/testdir/test89.in, src/testdir/test94.in

Patch 7.3.915
问题: When reading a file with encoding conversion fails at the end the
next encoding in 'fencs' is not used.
解决方案: Retry with another encoding when possible. (Taro Muraoka)
相关文件: src/fileio.c

Patch 7.3.916
问题:　　Using freed memory when pasting with the mouse (Issue 130).
解决方案:　Get the byte value early. (hint by Dominique Pelle)
相关文件:　src/buffer.c

Patch 7.3.917
问题:　　When a path ends in a backslash appending a comma has the wrong
　　effect.
解决方案:　Replace a trailing backslash with a slash. (Nazri Ramliy)
相关文件:　src/misc1.c, src/testdir/test73.in, src/testdir/test73.ok

Patch 7.3.918
问题:　　Repeating an Ex command after using a Visual motion does not work.
解决方案:　Check for an Ex command being used. (David Bürgin)
相关文件:　src/normal.c

Patch 7.3.919 (after 7.3.788)
问题:　　An empty nl.po file does not work with an old msgfmt.
解决方案:　Put a single # in the file. (Laurent Blume)
相关文件:　src/po/Makefile

Patch 7.3.920
问题:　　Compiler warning for size_t to int.
解决方案:　Add a type cast. (Mike Williams)
相关文件:　src/misc1.c

Patch 7.3.921 (after 7.3.697)
问题:　　Trying to create a fontset handle when 'guifontset' is not set.
解决方案:　Add curly braces around the code block. (Max Kirillov)
相关文件:　src/syntax.c

Patch 7.3.922
问题:　　No test for what 7.3.918 fixes.
解决方案:　Add a test. (David Bürgin)
相关文件:　src/testdir/test94.in, src/testdir/test94.ok

Patch 7.3.923
问题:　　Check for X11 header files fails on Solaris.
解决方案:　Only use -Werror for gcc. (Laurent Blume)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.924
问题:　　Python interface can't easily access options.
解决方案:　Add vim.options, vim.window.options and vim.buffer.options. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/eval.c, src/if_py_both.h,
　　src/if_python.c, src/if_python3.c, src/option.c,
　　src/proto/eval.pro, src/proto/option.pro, src/testdir/test86.in,
　　src/testdir/test86.ok, src/testdir/test87.in,
　　src/testdir/test87.ok, src/vim.h

Patch 7.3.925
问题:　　Typos in source files.
解决方案:　Fix the typos. (Ken Takata)
相关文件:　runtime/plugin/matchparen.vim, runtime/tools/vim_vs_net.cmd,

src/GvimExt/gvimext.cpp, src/INSTALLvms.txt, src/Make_cyg.mak,
src/Make_mvc.mak, src/Make_sas.mak, src/Make_vms.mms,
src/Make_w16.mak, src/Makefile, src/VisVim/OleAut.cpp,
src/VisVim/README_VisVim.txt, src/auto/configure, src/buffer.c,
src/configure.in, src/diff.c, src/dosinst.c, src/edit.c,
src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c,
src/farsi.c, src/feature.h, src/fileio.c, src/glbl_ime.cpp,
src/gui.c, src/gui_athena.c, src/gui_beval.c, src/gui_gtk_x11.c,
src/gui_mac.c, src/gui_motif.c, src/gui_photon.c, src/gui_w16.c,
src/gui_w32.c, src/gui_w48.c, src/gui_xmebw.c, src/gui_xmebwp.h,
src/hardcopy.c, src/if_cscope.c, src/if_mzsch.c, src/if_ole.cpp,
src/if_perl.xs, src/if_py_both.h, src/if_python.c,
src/if_python3.c, src/if_ruby.c, src/main.aap, src/mbyte.c,
src/memfile.c, src/memline.c, src/misc1.c, src/misc2.c,
src/nbdebug.c, src/normal.c, src/ops.c, src/os_amiga.c,
src/os_mac.h, src/os_msdos.c, src/os_mswin.c, src/os_win16.h,
src/os_win32.c, src/os_win32.h, src/quickfix.c, src/screen.c,
src/search.c, src/spell.c, src/structs.h, src/syntax.c,
src/window.c, vimtutor.com

Patch 7.3.926
问题: Autocommands are triggered by setwinvar() et al. Missing BufEnter
on :tabclose. Duplicate WinEnter on :tabclose. Wrong order of
events for :tablose and :tabnew.
解决方案: Fix these autocommand events. (ZyX)
相关文件: runtime/doc/eval.txt, src/buffer.c, src/eval.c, src/ex_cmds2.c,
src/fileio.c, src/proto/window.pro, src/testdir/test62.in,
src/testdir/test62.ok, src/window.c

Patch 7.3.927
问题: Missing combining characters when putting text in a register.
解决方案: Include combining characters. (David Bürgin)
相关文件: src/getchar.c, src/testdir/test44.in, src/testdir/test44.ok

Patch 7.3.928 (after 7.3.924)
问题: Can't build with strict C compiler.
解决方案: Move declaration to start of block. (Taro Muraoka)
相关文件: src/if_py_both.h

Patch 7.3.929 (after 7.3.924)
问题: Compiler warning for unused variable. Not freeing unused string.
解决方案: Remove the variable. Clear the options.
相关文件: src/option.c

Patch 7.3.930
问题: MSVC 2012 update is not recognized.
解决方案: Update the version in the makefile. (Raymond Ko)
相关文件: src/Make_mvc.mak

Patch 7.3.931
问题: No completion for :xmap and :smap. (Yukihiro Nakadaira)
解决方案: Add the case statements. (Christian Brabandt)
相关文件: src/ex_docmd.c

Patch 7.3.932
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Initialize the variable.
相关文件: src/option.c

Patch 7.3.933
问题: Ruby on Mac crashes due to GC failure.
解决方案: Init the stack from main(). (Hiroshi Shirosaki)
相关文件: src/main.c, src/if_ruby.c, src/proto/if_ruby.pro

Patch 7.3.934
问题: E381 and E380 make the user think nothing happened.
解决方案: Display the message indicating what error list is now active.
(Christian Brabandt)
相关文件: src/quickfix.c

Patch 7.3.935 (after 7.3.933)
问题: Ruby: Init stack works differently on 64 bit systems.
解决方案: Handle 64 bit systems and also static library. (Yukihiro
Nakadaira)
相关文件: src/if_ruby.c

Patch 7.3.936 (after 7.3.935)
问题: Ruby 1.8: Missing piece for static linking on 64 bit systems.
解决方案: Define ruby_init_stack() (Hiroshi Shirosaki)
Also fix preprocessor indents.
相关文件: src/if_ruby.c

Patch 7.3.937
问题: More can be shared between Python 2 and 3.
解决方案: Move code to if_py_both.h. (ZyX)
相关文件: src/if_python.c, src/if_python3.c, src/if_py_both.h

Patch 7.3.938
问题: Python: not easy to get to window number.
解决方案: Add vim.window.number. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/proto/window.pro,
src/window.c

Patch 7.3.939
问题: Using Py_BuildValue is inefficient sometimes.
解决方案: Use PyLong_FromLong(). (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.940
问题: Python: Can't get position of window.
解决方案: Add window.row and window.col. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h

Patch 7.3.941
问题: Stuff in if_py_both.h is ordered badly.
解决方案: Reorder by type. (ZyX)
相关文件: src/if_py_both.h, src/if_python.c

Patch 7.3.942
问题:　　Python: SEGV in Buffer functions.
解决方案:　Call CheckBuffer() at the right time. (ZyX)
相关文件:　src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.943
问题:　　Python: Negative indices were failing.
解决方案:　Fix negative indices. Add tests. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/testdir/test86.in,
　　src/testdir/test86.ok, src/testdir/test87.in,
　　src/testdir/test87.ok

Patch 7.3.944
问题:　　External program receives the termresponse.
解决方案:　Insert a delay and discard input. (Hayaki Saito)
相关文件:　src/term.c

Patch 7.3.945
问题:　　Python: List of buffers is not very useful.
解决方案:　Make vim.buffers a map. No iterator yet. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python3.c,
　　src/if_python.c, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.946
问题:　　Sometimes get stuck in waiting for cursor position report,
　　resulting in keys starting with <Esc>[not working.
解决方案:　Only wait for more characters after <Esc>[if followed by '?', '>'
　　or a digit.
相关文件:　src/term.c

Patch 7.3.947
问题:　　Python: No iterator for vim.list and vim.bufferlist.
解决方案:　Add the iterators. Also fix name of FunctionType. Add tests for
　　vim.buffers. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/eval.c, src/if_py_both.h,
　　src/if_python3.c, src/if_python.c, src/proto/eval.pro,
　　src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.948
问题:　　Cannot build with Python 2.2
解决方案:　Make Python interface work with Python 2.2
　　Make 2.2 the first supported version. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.ok, src/configure.in, src/auto/configure

Patch 7.3.949
问题:　　Python: no easy access to tabpages.
解决方案:　Add vim.tabpages and vim.current.tabpage. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python3.c,
　　src/if_python.c, src/proto/if_python3.pro,
　　src/proto/if_python.pro, src/proto/window.pro, src/structs.h,

src/window.c

Patch 7.3.950
问题: Python: Stack trace printer can't handle messages.
解决方案: Make KeyErrors use PyErr_SetObject. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.951
问题: Python exceptions have problems.
解决方案: Change some IndexErrors to TypeErrors. Make "line number out of range" an IndexError. Make "unable to get option value" a RuntimeError. Make all PyErr_SetString messages start with lowercase letter and use _(). (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.952
问题: Python: It's not easy to change window/buffer/tabpage.
解决方案: Add ability to assign to vim.current.{tabpage,buffer,window}. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h

Patch 7.3.953
问题: Python: string exceptions are deprecated.
解决方案: Make vim.error an Exception subclass. (ZyX)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.954
问题: No check if PyObject_IsTrue fails.
解决方案: Add a check for -1 value. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.955
问题: Python: Not enough tests.
解决方案: Add tests for vim.{current,window*,tabpage*}. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.956
问题: Python vim.bindeval() causes SIGABRT.
解决方案: Make pygilstate a local variable. (Yukihiro Nakadaira)
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.957
问题: Python does not have a "do" command like Perl or Lua.
解决方案: Add the ":py3do" command. (Lilydjwg)
相关文件: runtime/doc/if_pyth.txt, src/ex_cmds.h, src/ex_docmd.c, src/if_python3.c, src/proto/if_python3.pro

Patch 7.3.958
问题: Python: Iteration destructor not set.
解决方案: Put IterDestructor to use. (ZyX)
相关文件: src/if_py_both.c

Patch 7.3.959 (after 7.3.957)
问题: Missing error number.
解决方案: Assign an error number.
相关文件: src/if_python3.c

Patch 7.3.960
问题: Compiler warning for unused variable.
解决方案: Put declaration in #ifdef.
相关文件: src/window.c

Patch 7.3.961
问题: Tests 86 and 87 fail when using another language than English.
解决方案: Set the language to C in the test. (Dominique Pelle)
相关文件: src/testdir/test86.in, src/testdir/test87.in,
            src/testdir/test87.ok

Patch 7.3.962
问题: Python tests are not portable.
解决方案: Use shiftwidth instead of iminsert. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
            src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.963
问题: Setting curbuf without curwin causes trouble.
解决方案: Add switch_buffer() and restore_buffer(). Block autocommands to
            avoid trouble.
相关文件: src/eval.c, src/proto/eval.pro, src/proto/window.pro,
            src/if_py_both.h, src/window.c, src/testdir/test86.ok

Patch 7.3.964
问题: Python: not so easy to access tab pages.
解决方案: Add window.tabpage, make window.number work with non-current tab
            pages. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python3.c,
            src/if_python.c, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.965
问题: Python garbage collection not working properly.
解决方案: Add support for garbage collection. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.966
问题: There is ":py3do" but no ":pydo".
解决方案: Add the ":pydo" command. (Lilydjwg)
相关文件: runtime/doc/if_pyth.txt, src/ex_cmds.h, src/ex_docmd.c,
            src/if_py_both.h, src/if_python.c, src/if_python3.c,
            src/proto/if_python.pro

Patch 7.3.967 (after 7.3.965)
问题: Build fails on Mac OSX. (Greg Novack)
解决方案: Undefine clear().
相关文件: src/if_py_both.h

Patch 7.3.968

问题:　　Multi-byte support is only available when compiled with "big"
　　features.
解决方案:　Include multi-byte by default, with "normal" features.
相关文件:　src/feature.h

Patch 7.3.969
问题:　　Can't build with Python 3 and without Python 2.
解决方案:　Adjust #ifdef. (Xavier de Gaye)
相关文件:　src/window.c

Patch 7.3.970
问题:　　Syntax highlighting can be slow.
解决方案:　Include the NFA regexp engine. Add the 'regexpengine' option to
　　select which one is used. (various authors, including Ken Takata,
　　Andrei Aiordachioaie, Russ Cox, Xiaozhou Liua, Ian Young)
相关文件:　src/Make_cyg.mak, src/Make_ming.mak, src/Make_mvc.mak,
　　src/Makefile, src/regexp.c, src/regexp.h, src/regexp_nfa.c,
　　src/structs.h, src/testdir/Makefile, src/testdir/test64.in,
　　src/testdir/test64.ok, Filelist, runtime/doc/pattern.txt,
　　runtime/doc/option.txt, src/option.c, src/option.h,
　　src/testdir/test95.in, src/testdir/test95.ok,
　　src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
　　src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.3.971
问题:　　No support for VS2012 static code analysis.
解决方案:　Add the ANALYZE option. (Mike Williams)
相关文件:　src/Make_mvc.mak

Patch 7.3.972
问题:　　Cursor not restored after InsertEnter autocommand if it moved to
　　another line.
解决方案:　Also restore if the saved line number is still valid. Allow
　　setting v:char to skip restoring.
相关文件:　src/edit.c, runtime/doc/autocmd.txt

Patch 7.3.973
问题:　　Compiler warnings. Crash on startup. (Tony Mechelynck)
解决方案:　Change EMSG2 to EMSGN. Make array one character longer.
相关文件:　src/regexp_nfa.c

Patch 7.3.974
问题:　　Can't build with ruby 1.8.5.
解决方案:　Only use ruby_init_stack() when RUBY_INIT_STACK is defined.
　　(Yukihiro Nakadaira)
相关文件:　src/if_ruby.c

Patch 7.3.975
问题:　　Crash in regexp parsing.
解决方案:　Correctly compute the end of allocated memory.
相关文件:　src/regexp_nfa.c

Patch 7.3.976

问题: Can't build on HP-UX.
解决方案: Remove modern initialization. (John Marriott)
相关文件: src/regexp_nfa.c

Patch 7.3.977
问题: Compiler warnings on 64 bit Windows.
解决方案: Add type casts. (Mike Williams) Also fix some white space and
            uncomment what was commented-out for testing.
相关文件: src/regexp_nfa.c

Patch 7.3.978
问题: Regexp debug logs don't have a good name.
解决方案: Use clear names and make it possible to write logs for the old and
            new engines separately. (Taro Muraoka)
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.979
问题: Complex NFA regexp doesn't work.
解决方案: Set actual state stack end instead of using an arbitrary number.
            (Yasuhiro Matsumoto)
相关文件: src/regexp_nfa.c

Patch 7.3.980
问题: Regexp logs may contain garbage. Character classes don't work
            correctly for multi-byte characters.
解决方案: Check for end of post list. Only use "is" functions for
            characters up to 255. (Ken Takata)
相关文件: src/regexp_nfa.c

Patch 7.3.981
问题: In the old regexp engine \i, \I, \f and \F don't work on
            multi-byte characters.
解决方案: Dereference pointer properly.
相关文件: src/regexp.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.982
问题: In the new regexp engine \p does not work on multi-byte
            characters.
解决方案: Don't point to an integer but the characters.
相关文件: src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.983
问题: Unnecessary temp variable.
解决方案: Remove the variable.
相关文件: src/regexp_nfa.c

Patch 7.3.984
问题: A Visual mapping that uses CTRL-G works differently when started
            from Insert mode. (Ein Brown)
解决方案: Reset old_mapped_len when handling typed text in Select mode.
相关文件: src/normal.c

Patch 7.3.985
问题: GTK vim not started as gvim doesn't set WM_CLASS property to a

useful value.
解决方案: Call g_set_prgname() on startup. (James McCoy)
相关文件: src/gui_gtk_x11.c

Patch 7.3.986
问题: Test 95 doesn't pass when 'encoding' isn't utf-8. (Yasuhiro Matsumoto)
解决方案: Force 'encoding' to be utf-8.
相关文件: src/testdir/test95.in

Patch 7.3.987
问题: No easy to run an individual test. Tests 64 fails when 'encoding' is not utf-8.
解决方案: Add individual test targets to the Makefile. Move some lines from test 64 to 95.
相关文件: src/Makefile, src/testdir/test64.in, src/testdir/test64.ok, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.988
问题: New regexp engine is slow.
解决方案: Break out of the loop when the state list is empty.
相关文件: src/regexp_nfa.c

Patch 7.3.989
问题: New regexp engine compares negative numbers to character.
解决方案: Add missing case statements.
相关文件: src/regexp_nfa.c

Patch 7.3.990
问题: Memory leak in new regexp engine.
解决方案: Jump to end of function to free memory. (Dominique Pelle)
相关文件: src/regexp_nfa.c

Patch 7.3.991
问题: More can be shared by Python 2 and 3.
解决方案: Move more stuff to if_py_both. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c, src/testdir/test87.ok

Patch 7.3.992
问题: Python: Too many type casts.
解决方案: Change argument types. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.993
问题: Python: Later patch does things slightly differently.
解决方案: Adjusted argument type changes. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.994
问题: Python: using magic constants.
解决方案: Use descriptive values for ml_flags. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c

Patch 7.3.995
问题:　　Python: Module initialization is duplicated.
解决方案:　Move to shared file. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.996
问题:　　Python: Can't check types of what is returned by bindeval().
解决方案:　Add vim.List, vim.Dictionary and vim.Function types. (ZyX)
相关文件:　runtime/doc/if_pyth.txt, src/if_py_both.h, src/testdir/test86.in,
　　src/testdir/test86.ok, src/testdir/test87.in,
　　src/testdir/test87.ok

Patch 7.3.997
问题:　　Vim and Python exceptions are different.
解决方案:　Make Vim exceptions be Python exceptions. (ZyX)
相关文件:　src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.998
问题:　　Python: garbage collection issues.
解决方案:　Fix the GC issues: Use proper DESTRUCTOR_FINISH: avoids negative
　　refcounts, use PyObject_GC_* for objects with tp_traverse and
　　tp_clear, add RangeTraverse and RangeClear, use Py_XDECREF in some
　　places. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.999
问题:　　New regexp engine sets curbuf temporarily.
解决方案:　Use reg_buf instead, like the old engine.
相关文件:　src/regexp_nfa.c

Patch 7.3.1000 (whoa!)
问题:　　Typo in char value causes out of bounds access.
解决方案:　Fix character value. (Klemens Baum)
相关文件:　src/regexp.c

Patch 7.3.1001
问题:　　Duplicate condition in if.
解决方案:　Remove one condition.
相关文件:　src/regexp_nfa.c

Patch 7.3.1002
问题:　　Valgrind errors for Python interface.
解决方案:　Fix memory leaks when running tests. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1003
问题:　　Python interface does not compile with Python 2.2
解决方案:　Fix thread issues and True/False. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1004

问题:	No error when option could not be set.
解决方案:	Report an error. (ZyX)
相关文件:	src/if_py_both.h, src/option.c, src/proto/option.pro,
src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1005
问题:	Get stuck on regexp "\n*" and on "%s/^\n\+/\r".
解决方案:	Fix handling of matching a line break. (idea by Hirohito Higashi)
相关文件:	src/regexp_nfa.c

Patch 7.3.1006
问题:	NFA engine not used for "_[0-9]".
解决方案:	Enable this, fixed in patch 1005.
相关文件:	src/regexp_nfa.c

Patch 7.3.1007
问题:	Can't build on Minix 3.2.1.
解决方案:	Add a condition to an #ifdef. (Gautam Tirumala)
相关文件:	src/memfile.c

Patch 7.3.1008
问题:	Test 95 fails on MS-Windows.
解决方案:	Set 'nomore'. Change \i to \f. Change multi-byte character to
something that is not matching \i. (Ken Takata)
相关文件:	src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1009
问题:	Compiler warning for ambiguous else.
解决方案:	Add curly braces.
相关文件:	src/if_py_both.h

Patch 7.3.1010
问题:	New regexp: adding \Z makes every character match.
解决方案:	Only apply ireg_icombine for composing characters.
Also add missing change from patch 1008. (Ken Takata)
相关文件:	src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1011
问题:	New regexp engine is inefficient with multi-byte characters.
解决方案:	Handle a character at a time instead of a byte at a time. Also
make \Z partly work.
相关文件:	src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1012
问题:	\Z does not work properly with the new regexp engine.
解决方案:	Make \Z work. Add tests.
相关文件:	src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1013
问题:	New regexp logging is a bit messy.
解决方案:	Consistently use #defines, add explanatory comment. (Taro Muraoka)
相关文件:	src/regexp_nfa.c

Patch 7.3.1014

问题:　　New regexp state dump is hard to read.
解决方案:　Make the state dump more pretty. (Taro Muraoka)
相关文件:　src/regexp_nfa.c

Patch 7.3.1015
问题:　　New regexp engine: Matching composing characters is wrong.
解决方案:　Fix matching composing characters.
相关文件:　src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1016
问题:　　Unused field in nfa_state.
解决方案:　Remove lastthread.
相关文件:　src/regexp.h, src/regexp_nfa.c

Patch 7.3.1017
问题:　　Zero width match changes length of match.
解决方案:　For a zero width match put new states in the current position in
　　the state list.
相关文件:　src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok,
　　src/regexp.h

Patch 7.3.1018
问题:　　New regexp engine wastes memory.
解决方案:　Allocate prog with actual number of states, not estimated maximum
　　number of sates.
相关文件:　src/regexp_nfa.c

Patch 7.3.1019
问题:　　These do not work with the new regexp engine: \%o123, \%x123,
　　\%d123, \%u123 and \%U123.
解决方案:　Implement these items.
相关文件:　src/regexp_nfa.c

Patch 7.3.1020
问题:　　Not all patterns are tested with auto / old / new engine.
解决方案:　Test patterns with three values of 'regexpengine'.
相关文件:　src/testdir/test64.in, src/testdir/test64.ok,
　　src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1021
问题:　　New regexp engine does not ignore order of composing chars.
解决方案:　Ignore composing chars order.
相关文件:　src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1022
问题:　　Compiler warning for shadowed variable. (John Little)
解决方案:　Move declaration, rename variables.
相关文件:　src/regexp_nfa.c

Patch 7.3.1023
问题:　　Searching for composing char only and using \Z has different
　　results.
解决方案:　Make it match the composing char, matching everything is not
　　useful.

相关文件: src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.3.1024
问题: New regexp: End of matching pattern not set correctly. (Cesar Romani)
解决方案: Quit the loop after finding the match. Store nfa_has_zend in the program.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok, src/regexp.h

Patch 7.3.1025
问题: New regexp: not matching newline in string. (Marc Weber)
解决方案: Check for "\n" character.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1026
问题: New regexp: pattern that includes a new-line matches too early. (John McGowan)
解决方案: Do not start searching in the second line.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1027
问题: New regexp performance: Calling no_Magic() very often.
解决方案: Remove magicness inline.
相关文件: src/regexp_nfa.c

Patch 7.3.1028
问题: New regexp performance: Copying a lot of position state.
解决方案: Only copy the sub-expressions that are being used.
相关文件: src/regexp_nfa.c, src/regexp.h

Patch 7.3.1029
问题: New regexp performance: Unused position state being copied.
解决方案: Keep track of which positions are actually valid.
相关文件: src/regexp_nfa.c

Patch 7.3.1030 (after 7.3.1028)
问题: Can't build for debugging.
解决方案: Fix struct member names.
相关文件: src/regexp_nfa.c

Patch 7.3.1031
问题: Compiler warnings for shadowed variable. (John Little)
解决方案: Move the variable declarations to the scope where they are used.
相关文件: src/regexp_nfa.c

Patch 7.3.1032
问题: "\ze" is not supported by the new regexp engine.
解决方案: Make "\ze" work.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1033
问题: "\1" .. "\9" are not supported in the new regexp engine.
解决方案: Implement them. Add a few more tests.

相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok,
           src/regexp.h

Patch 7.3.1034
问题: New regexp code using strange multi-byte code.
解决方案: Use the normal code to advance and backup pointers.
相关文件: src/regexp_nfa.c

Patch 7.3.1035
问题: Compiler warning on 64 bit windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/if_py_both.h

Patch 7.3.1036
问题: Can't build on HP-UX.
解决方案: Give the union a name. (John Marriott)
相关文件: src/regexp_nfa.c

Patch 7.3.1037
问题: Look-behind matching is very slow on long lines.
解决方案: Add a byte limit to how far back an attempt is made.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in,
           src/testdir/test64.ok

Patch 7.3.1038
问题: Crash when using Cscope.
解决方案: Avoid negative argument to vim_strncpy(). (Narendran
           Gopalakrishnan)
相关文件: src/if_cscope.c

Patch 7.3.1039
问题: New regexp engine does not support \%23c, \%<23c and the like.
解决方案: Implement them. (partly by Yasuhiro Matsumoto)
相关文件: src/regexp.h, src/regexp_nfa.c, src/testdir/test64.in,
           src/testdir/test64.ok

Patch 7.3.1040
问题: Python: Problems with debugging dynamic build.
解决方案: Python patch 1. (ZyX)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.1041
问题: Python: Invalid read valgrind errors.
解决方案: Python patch 2: defer DICTKEY_UNREF until key is no longer needed.
           (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1042
问题: Python: can't assign to vim.Buffer.name.
解决方案: Python patch 3. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/ex_cmds.c, src/if_py_both.h,
           src/if_python3.c, src/if_python.c, src/proto/ex_cmds.pro,
           src/testdir/test86.in, src/testdir/test86.ok,
           src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1043
问题:　　Python: Dynamic compilation with 2.3 fails.
解决方案:　Python patch 4. (ZyX)
相关文件:　src/if_python.c

Patch 7.3.1044
问题:　　Python: No {Buffer,TabPage,Window}.valid attributes.
解决方案:　Python patch 5: add .valid (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1045
问题:　　Python: No error handling for VimToPython function.
解决方案:　Python patch 6. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1046
问题:　　Python: Using Py_BuildValue for building strings.
解决方案:　Python patch 7 and 7.5: Replace Py_BuildValue with
　　PyString_FromString. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1047
问题:　　Python: dir() does not work properly.
解决方案:　Python patch 8. Add __dir__ method to all objects with custom
　　tp_getattr supplemented by __members__ attribute for at least
　　python-2* versions. __members__ is not mentioned in python-3*
　　dir() output even if it is accessible. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1048
问题:　　Python: no consistent naming.
解决方案:　Python patch 9: Rename d to dict and lookupDict to lookup_dict.
　　(ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1049
问题:　　Python: no consistent naming
解决方案:　Python patch 10: Rename DICTKEY_GET_NOTEMPTY to DICTKEY_GET. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1050
问题:　　Python: Typo in pyiter_to_tv.
解决方案:　Python patch 11. (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1051
问题:　　Python: possible memory leaks.
解决方案:　Python patch 12: fix the leaks (ZyX)
相关文件:　src/if_py_both.h

Patch 7.3.1052
问题: Python: possible SEGV and negative refcount.
解决方案: Python patch 13: Fix IterIter function. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1053
问题: Python: no flag for types with tp_traverse+tp_clear.
解决方案: Python patch 14: Add Py_TPFLAGS_HAVE_GC. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1054 (after 7.3.1042)
问题: Can't build without the +autocmd feature. (Elimar Riesebieter)
解决方案: Fix use of buf and curbuf.
相关文件: src/ex_cmds.c, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1055
问题: Negated collection does not match newline.
解决方案: Handle newline differently. (Hiroshi Shirosaki)
相关文件: src/regexp_nfa.c, src/testdir/test64.ok, src/testdir/test64.in

Patch 7.3.1056
问题: Python: possible memory leaks.
解决方案: Python patch 15. (ZyX) Fix will follow later.
相关文件: src/eval.c, src/if_py_both.h, src/proto/eval.pro

Patch 7.3.1057
问题: Python: not enough compatibility.
解决方案: Python patch 16: Make OutputWritelines support any sequence object
(ZyX) Note: tests fail
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1058
问题: Call of funcref does not succeed in other script.
解决方案: Python patch 17: add get_expanded_name(). (ZyX)
相关文件: src/eval.c, src/proto/eval.pro

Patch 7.3.1059
问题: Python: Using fixed size buffers.
解决方案: Python patch 18: Use python's own formatter. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.1060
问题: Python: can't repr() a function.
解决方案: Python patch 19: add FunctionRepr(). (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1061
问题: Python: Dictionary is not standard.
解决方案: Python patch 20: Add standard methods and fields. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/eval.c, src/if_py_both.h,
src/if_python3.c, src/if_python.c, src/proto/eval.pro,
src/testdir/test86.in, src/testdir/test86.ok,

src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1062
问题: Python: List is not standard.
解决方案: Python patch 21: Add standard methods and fields. (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1063
问题: Python: Function is not standard.
解决方案: Python patch 22: make Function subclassable. (ZyX)
相关文件: src/eval.c, src/if_py_both.h, src/proto/eval.pro,
src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1064
问题: Python: insufficient error checking.
解决方案: Python patch 23. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1065
问题: Python: key mapping is not standard.
解决方案: Python patch 24: use PyMapping_Keys. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.1066
问题: Python: Insufficient exception and error testing.
解决方案: Python patch 25. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1067
问题: Python: documentation lags behind.
解决方案: Python patch 26. (ZyX)
相关文件: runtime/doc/if_pyth.txt

Patch 7.3.1068
问题: Python: Script is auto-loaded on function creation.
解决方案: Python patch 27. (ZyX)
相关文件: src/eval.c, src/if_py_both.h, src/proto/eval.pro,
src/testdir/test86.ok, src/testdir/test87.ok, src/vim.h

Patch 7.3.1069
问题: Python: memory leaks.
解决方案: Python patch 28: Purge out DICTKEY_CHECK_EMPTY macros. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1070
问题: Vim crashes in Python tests. Compiler warning for unused function.
解决方案: Disable the tests for now. Move the function.
相关文件: src/if_py_both.h, src/if_python.c, src/testdir/test86.in,
src/testdir/test87.in

Patch 7.3.1071

问题: New regexp engine: backreferences don't work correctly.
解决方案: Add every possible start/end position on the state stack.
相关文件: src/regexp_nfa.c, src/regexp.h, src/testdir/test64.in,
            src/testdir/test64.ok

Patch 7.3.1072
问题: Compiler warning for uninitialized variable.
解决方案: Initialize it.
相关文件: src/regexp_nfa.c

Patch 7.3.1073
问题: New regexp engine may run out of states.
解决方案: Allocate states dynamically. Also make the test report errors.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok,
            src/testdir/test95.in

Patch 7.3.1074
问题: Compiler warning for printf format. (Manuel Ortega)
解决方案: Add type casts.
相关文件: src/if_py_both.h

Patch 7.3.1075
问题: Compiler warning for storing a long_u in an int.
解决方案: Declare the number as an int. (Mike Williams)
相关文件: src/regexp_nfa.c

Patch 7.3.1076
问题: New regexp engine: \@= and \& don't work.
解决方案: Make these items work. Add column info to logging.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1077
问题: Python: Allocating dict the wrong way, causing a crash.
解决方案: Use py_dict_alloc(). Fix some exception problems. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1078
问题: New regexp engine: \@! doesn't work.
解决方案: Implement the negated version of \@=.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1079
问题: Test 87 fails.
解决方案: Fix the test for Python 3.3. (ZyX) Make it pass on 32 bit systems.
相关文件: src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1080
问题: Test 86 fails.
解决方案: Comment out the parts that don't work. Make it pass on 32 bit
            systems.
相关文件: src/testdir/test86.in, src/testdir/test86.ok

Patch 7.3.1081
问题: Compiler warnings on 64-bit Windows.

解决方案: Change variable types. (Mike Williams)
相关文件: src/if_py_both.h, src/regexp_nfa.c

Patch 7.3.1082
问题: New regexp engine: Problem with \@= matching.
解决方案: Save and restore nfa_match.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1083
问题: New regexp engine: Does not support \%^ and \%$.
解决方案: Support matching start and end of file.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1084
问题: New regexp engine: only accepts up to \{,10}.
解决方案: Remove upper limit. Remove dead code with NFA_PLUS.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1085
问题: New regexp engine: Non-greedy multi doesn't work.
解决方案: Implement \{-}.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1086
问题: Old regexp engine accepts illegal range, new one doesn't.
解决方案: Also accept the illegal range with the new engine.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1087
问题: A leading star is not seen as a normal char when \{} follows.
解决方案: Save and restore the parse state properly.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in,
           src/testdir/test64.ok

Patch 7.3.1088
问题: New regexp engine: \@<= and \@<! are not implemented.
解决方案: Implement look-behind matching. Fix off-by-one error in old
           regexp engine.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in,
           src/testdir/test64.ok

Patch 7.3.1089
问题: Tests 86 and 87 fail on MS-Windows. (Ken Takata)
解决方案: Fix platform-specific stuff. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
           src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1090
问题: New regexp engine does not support \z1 .. \z9 and \z(.
解决方案: Implement the syntax submatches.
相关文件: src/regexp.h, src/regexp_nfa.c

Patch 7.3.1091
问题: New regexp engine: no error when using \z1 or \z(where it does

not work.
解决方案: Give an error message.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1092
问题: Can't build with regexp debugging. NFA debug output shows wrong pattern.
解决方案: Fix debugging code for recent changes. Add the pattern to the program.
相关文件: src/regexp_nfa.c, src/regexp.h

Patch 7.3.1093
问题: New regexp engine: When a sub expression is empty \1 skips a character.
解决方案: Make \1 try the current position when the match is empty.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1094
问题: New regexp engine: Attempts to match "^" at every character.
解决方案: Only try "^" at the start of a line.
相关文件: src/regexp_nfa.c

Patch 7.3.1095
问题: Compiler warnings for shadowed variables. (Christian Brabandt)
解决方案: Rename new_state() to alloc_state(). Remove unnecessary declaration.
相关文件: src/regexp_nfa.c

Patch 7.3.1096
问题: Python: popitem() was not defined in a standard way.
解决方案: Remove the argument from popitem(). (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1097
问题: Python: a few recently added items are not documented.
解决方案: Update the documentation. (ZyX)
相关文件: runtime/doc/if_pyth.txt

Patch 7.3.1098
问题: Python: Possible memory leaks
解决方案: Add Py_XDECREF() calls. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1099
问题: Python: Changing directory with os.chdir() causes problems for Vim's notion of directories.
解决方案: Add vim.chdir() and vim.fchdir(). (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/ex_docmd.c, src/if_py_both.h, src/if_python3.c, src/if_python.c, src/proto/ex_docmd.pro, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1100
问题: Python: a few more memory problems.
解决方案: Add and remove Py_XDECREF(). (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1101
问题: Configure doesn't find Python 3 on Ubuntu 13.04.
解决方案: First try distutils.sysconfig. Also fix some indents. (Ken Takata)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.1102
问题: Completion of ":py3do" and ":py3file" does not work after ":py3".
解决方案: Make completion work. (Taro Muraoka)
相关文件: src/ex_docmd.c

Patch 7.3.1103
问题: New regexp engine: overhead in saving and restoring.
解决方案: Make saving and restoring list IDs faster. Don't copy or check \z subexpressions when they are not used.
相关文件: src/regexp_nfa.c

Patch 7.3.1104
问题: New regexp engine does not handle "~".
解决方案: Add support for "~".
相关文件: src/regexp_nfa.c, src/testdir/test24.in, src/testdir/test24.ok

Patch 7.3.1105
问题: New regexp engine: too much code in one function. Dead code.
解决方案: Move the recursive nfa_regmatch call to a separate function. Remove the dead code.
相关文件: src/regexp_nfa.c

Patch 7.3.1106
问题: New regexp engine: saving and restoring lastlist in the states takes a lot of time.
解决方案: Use a second lastlist value for the first recursive call.
相关文件: src/regexp.h, src/regexp_nfa.c

Patch 7.3.1107
问题: Compiler warnings for unused variables.
解决方案: Put the variables inside #ifdef.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1108
问题: Error message for os.fchdir() (Charles Peacech)
解决方案: Clear the error. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1109
问题: Building on MS-Windows doesn't see changes in if_py_both.h.
解决方案: Add a dependency. (Ken Takata)
相关文件: src/Make_bc5.mak, src/Make_cyg.mak, src/Make_ming.mak,

src/Make_mvc.mak

Patch 7.3.1110
问题: New regexp matching: Using \@= and the like can be slow.
解决方案: Decide whether to first try matching the zero-width part or what follows, whatever is more likely to fail.
相关文件: src/regexp_nfa.c

Patch 7.3.1111
问题: nfa_recognize_char_class() implementation is inefficient.
解决方案: Use bits in an int instead of chars in a string. (Dominique Pelle)
相关文件: src/regexp_nfa.c, src/testdir/test36.in, src/testdir/test36.ok

Patch 7.3.1112
问题: New regexp engine: \%V not supported.
解决方案: Implement \%V. Add tests.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1113
问题: New regexp engine: \%'m not supported.
解决方案: Implement \%'m. Add tests.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1114 (after 7.3.1110)
问题: Can't build without the syntax feature.
解决方案: Add #ifdefs. (Erik Falor)
相关文件: src/regexp_nfa.c

Patch 7.3.1115
问题: Many users don't like the cursor line number when 'relativenumber' is set.
解决方案: Have four combinations with 'number' and 'relativenumber'. (Christian Brabandt)
相关文件: runtime/doc/options.txt, src/option.c, src/screen.c, src/testdir/test89.in, src/testdir/test89.ok

Patch 7.3.1116
问题: Can't build without Visual mode.
解决方案: Add #ifdefs.
相关文件: src/regexp_nfa.c

Patch 7.3.1117
问题: New regexp engine: \%[abc] not supported.
解决方案: Implement \%[abc]. Add tests.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1118
问题: Match failure rate is not very specific.
解决方案: Tune the failure rate for match items.
相关文件: src/regexp_nfa.c

Patch 7.3.1119

问题: Flags in 'cpo' are search for several times.
解决方案: Store the result and re-use the flags.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1120
问题: Crash when regexp logging is enabled.
解决方案: Avoid using NULL pointers. Advance over count argument.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1121
问题: New regexp engine: adding states that are not used.
解决方案: Don't add the states.
相关文件: src/regexp_nfa.c

Patch 7.3.1122
问题: New regexp engine: \@> not supported.
解决方案: Implement \%>.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1123
问题: Can't build tiny Vim on MS-Windows.
解决方案: Adjust #ifdef around using modif_fname(). (Mike Williams)
相关文件: src/misc1.c

Patch 7.3.1124
问题: Python: Crash on MS-Windows when os.fchdir() is not available.
解决方案: Check for _chdir to be NULL. (Ken Takata)
相关文件: src/if_py_both.h

Patch 7.3.1125
问题: Error for using \%V in a pattern in tiny Vim.
解决方案: Allow using \%V but never match. (Dominique Pelle)
相关文件: src/regexp_nfa.c

Patch 7.3.1126
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Assign something to the variable.
相关文件: src/regexp_nfa.c

Patch 7.3.1127
问题: No error for using empty \%[].
解决方案: Give error message.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1128
问题: Now that the NFA engine handles everything every failure is a
            syntax error.
解决方案: Remove the syntax_error flag.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1129
问题: Can't see what pattern in syntax highlighting is slow.
解决方案: Add the ":syntime" command.
相关文件: src/structs.h, src/syntax.c, src/ex_cmds.h, src/ex_docmd.c,

src/proto/syntax.pro, src/ex_cmds2.c, src/proto/ex_cmds2.pro,
runtime/doc/syntax.txt

Patch 7.3.1130 (after 7.3.1129)
问题: Can't build with anything but huge features.
解决方案: Check for FEAT_PROFILE. (Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c, src/structs.h, src/syntax.c

Patch 7.3.1131
问题: New regexp engine is a bit slow.
解决方案: Do not clear the state list. Don't copy syntax submatches when not used.
相关文件: src/regexp_nfa.c

Patch 7.3.1132
问题: Crash when debugging regexp.
解决方案: Do not try to dump subexpr that were not set. Skip over count of \% items.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.3.1133
问题: New regexp engine is a bit slow.
解决方案: Skip ahead to a character that must match. Don't try matching a "^" patter past the start of line.
相关文件: src/regexp_nfa.c, src/regexp.h

Patch 7.3.1134
问题: Running test 49 takes a long time.
解决方案: Don't have it grep all files.
相关文件: src/testdir/test49.vim

Patch 7.3.1135
问题: Compiler warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/syntax.c

Patch 7.3.1136
问题: ":func Foo" does not show attributes.
解决方案: Add "abort", "dict" and "range". (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.3.1137
问题: New regexp engine: collections are slow.
解决方案: Handle all characters in one go.
相关文件: src/regexp_nfa.c

Patch 7.3.1138
问题: New regexp engine: neglist no longer used.
解决方案: Remove the now unused neglist.
相关文件: src/regexp_nfa.c

Patch 7.3.1139
问题: New regexp engine: negated flag is hardly used.
解决方案: Add separate _NEG states, remove negated flag.

相关文件: src/regexp_nfa.c, src/regexp.h

Patch 7.3.1140
问题: New regexp engine: trying expensive match while the result is not
           going to be used.
解决方案: Check for output state already being in the state list.
相关文件: src/regexp_nfa.c

Patch 7.3.1141
问题: Win32: Check for available memory is not reliable and adds
           overhead.
解决方案: Remove mch_avail_mem(). (Mike Williams)
相关文件: src/os_win32.c, src/os_win32.h

Patch 7.3.1142
问题: Memory leak in ":syntime report".
解决方案: Clear the grow array. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.3.1143
问题: When mapping NUL it is displayed as an X.
解决方案: Check for KS_ZERO instead of K_ZERO. (Yasuhiro Matsumoto)
相关文件: src/message.c

Patch 7.3.1144
问题: "RO" is not translated everywhere.
解决方案: Put inside _(). (Sergey Alyoshin)
相关文件: src/buffer.c, src/screen.c

Patch 7.3.1145
问题: New regexp engine: addstate() is called very often.
解决方案: Optimize adding the start state.
相关文件: src/regexp_nfa.c

Patch 7.3.1146
问题: New regexp engine: look-behind match not checked when followed by
           zero-width match.
解决方案: Do the look-behind match before adding the zero-width state.
相关文件: src/regexp_nfa.c

Patch 7.3.1147
问题: New regexp engine: regstart is only used to find the first match.
解决方案: Use regstart whenever adding the start state.
相关文件: src/regexp_nfa.c

Patch 7.3.1148
问题: No command line completion for ":syntime".
解决方案: Implement the completion. (Dominique Pelle)
相关文件: runtime/doc/map.txt, src/ex_cmds.h, src/ex_docmd.c,
           src/ex_getln.c, src/proto/syntax.pro, src/syntax.c, src/vim.h

Patch 7.3.1149
问题: New regexp engine: Matching plain text could be faster.
解决方案: Detect a plain text match and handle it specifically. Add

vim_regfree().
相关文件: src/regexp.c, src/regexp.h, src/regexp_nfa.c,
src/proto/regexp.pro, src/buffer.c, src/edit.c, src/eval.c,
src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c,
src/ex_getln.c, src/fileio.c, src/gui.c, src/misc1.c, src/misc2.c,
src/option.c, src/syntax.c, src/quickfix.c, src/search.c,
src/spell.c, src/tag.c, src/window.c, src/screen.c, src/macros.h,
src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1150
问题: New regexp engine: Slow when a look-behind match does not have a
width specified.
解决方案: Try to compute the maximum width.
相关文件: src/regexp_nfa.c

Patch 7.3.1151
问题: New regexp engine: Slow when a look-behind match is followed by a
zero-width match.
解决方案: Postpone the look-behind match more often.
相关文件: src/regexp_nfa.c

Patch 7.3.1152
问题: In tiny build ireg_icombine is undefined. (Tony Mechelynck)
解决方案: Add #ifdef.
相关文件: src/regexp_nfa.c

Patch 7.3.1153
问题: New regexp engine: Some look-behind matches are very expensive.
解决方案: Postpone invisible matches further, until a match is almost found.
相关文件: src/regexp_nfa.c

Patch 7.3.1154
问题: New regexp_nfa engine: Unnecessary code.
解决方案: Remove unnecessary code.
相关文件: src/regexp_nfa.c

Patch 7.3.1155
问题: MS-DOS: "make test" uses external rmdir command.
解决方案: Rename "rmdir" to "rd". (Taro Muraoka)
相关文件: src/testdir/Make_dos.mak

Patch 7.3.1156
问题: Compiler warnings. (dv1445)
解决方案: Initialize variables, even when the value isn't really used.
相关文件: src/regexp_nfa.c, src/eval.c

Patch 7.3.1157
问题: New regexp engine fails on "\(\<command\)\@<=.*"
解决方案: Fix rule for postponing match. Further tune estimating whether
postponing works better. Add test.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1158
问题: Crash when running test 86. (Jun Takimoto)

解决方案: Define PY_SSIZE_T_CLEAN early. (Elimar Riesebieter)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.3.1159
问题: The round() function is not always available. (Christ van Willegen)
解决方案: Use the solution from f_round().
相关文件: src/ex_cmds2.c, src/eval.c, src/proto/eval.pro

Patch 7.3.1160
问题: Mixing long and pointer doesn't always work.
解决方案: Avoid cast to pointer.
相关文件: src/undo.c

Patch 7.3.1161
问题: Python: PyList_SetItem() is inefficient.
解决方案: Use PyList_SET_ITEM() (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1162
问题: Python: Memory leaks
解决方案: Add more Py_DECREF(). (ZyX)
相关文件: src/if_py_both.h, src/if_python.c

Patch 7.3.1163
问题: Not easy to load Python modules.
解决方案: Search "python2", "python3" and "pythonx" directories in 'runtimepath' for Python modules. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/configure.in, src/ex_cmds2.c, src/if_py_both.h, src/if_python.c, src/if_python3.c, src/testdir/test86.in, src/testdir/test87.in, src/auto/configure

Patch 7.3.1164
问题: Can't test what is actually displayed on screen.
解决方案: Add the screenchar() and screenattr() functions.
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 7.3.1165
问题: HP-UX compiler can't handle zero size array. (Charles Cooper)
解决方案: Make the array one item big.
相关文件: src/regexp.h, src/regexp_nfa.c

Patch 7.3.1166
问题: Loading Python modules is not tested.
解决方案: Enable commented-out tests, add missing files. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok, src/testdir/python2/module.py, src/testdir/python3/module.py, src/testdir/pythonx/module.py, src/testdir/pythonx/modulex.py, Filelist

Patch 7.3.1167
问题: Python configure check doesn't reject Python 2 when requesting Python 3. Some systems need -pthreads instead of -pthread.

解决方案: Adjust configure accordingly. (Andrei Olsen)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.1168
问题: Python "sane" configure checks give a warning message.
解决方案: Use single quotes instead of escaped double quotes. (Ben Fritz)
相关文件: src/configure.in, src/auto/configure

Patch 7.3.1169
问题: New regexp engine: some work is done while executing a pattern,
           even though the result is predictable.
解决方案: Do the work while compiling the pattern.
相关文件: src/regexp_nfa.c

Patch 7.3.1170
问题: Patch 7.3.1058 breaks backwards compatibility, not possible to use
           a function reference as a string. (lilydjwg)
解决方案: Instead of translating the function name only translate "s:".
相关文件: src/eval.c

Patch 7.3.1171
问题: Check for digits and ascii letters can be faster.
解决方案: Use a trick with one comparison. (Dominique Pelle)
相关文件: src/macros.h

Patch 7.3.1172
问题: Python 2: loading modules doesn't work well.
解决方案: Fix the code. Add more tests. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python.c,
           src/testdir/python2/module.py, src/testdir/python3/module.py,
           src/testdir/python_after/after.py,
           src/testdir/python_before/before.py, src/testdir/test86.in,
           src/testdir/test86.ok, src/testdir/test87.in,
           src/testdir/test87.ok, Filelist

Patch 7.3.1173
问题: Python 2 tests don't have the same output everywhere.
解决方案: Make the Python 2 tests more portable. (ZyX)
相关文件: src/testdir/test86.in, src/testdir/test86.ok

Patch 7.3.1174
问题: Python 2 and 3 use different ways to load modules.
解决方案: Use the same method. (ZyX)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python3.c,
           src/if_python.c

Patch 7.3.1175
问题: Using isalpha() and isalnum() can be slow.
解决方案: Use range checks. (Mike Williams)
相关文件: src/ex_docmd.c, src/macros.h

Patch 7.3.1176
问题: Compiler warnings on 64 bit system.
解决方案: Add type casts. (Mike Williams)

相关文件: src/eval.c, src/if_py_both.h

Patch 7.3.1177
问题: Wasting memory on padding.
解决方案: Reorder struct fields. (Dominique Pelle)
相关文件: src/structs.h, src/fileio.c

Patch 7.3.1178
问题: Can't put all Vim config files together in one directory.
解决方案: Load ~/.vim/vimrc if ~/.vimrc does not exist. (Lech Lorens)
相关文件: runtime/doc/gui.txt, runtime/doc/starting.txt, src/gui.c,
src/main.c, src/os_amiga.h, src/os_dos.h, src/os_unix.h

Patch 7.3.1179
问题: When a global mapping starts with the same characters as a
buffer-local mapping Vim waits for a character to be typed to find
out whether the global mapping is to be used. (Andy Wokula)
解决方案: Use the local mapping without waiting. (Michael Henry)
相关文件: runtime/doc/map.txt, src/getchar.c

Patch 7.3.1180
问题: When current directory changes, path from cscope may no longer be
valid. (AS Budden)
解决方案: Always store the absolute path. (Christian Brabandt)
相关文件: src/if_cscope.c

Patch 7.3.1181
问题: Wrong error message for 1.0[0].
解决方案: Check for funcref and float separately. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.3.1182
问题: 'backupcopy' default on MS-Windows does not work for hard and soft
links.
解决方案: Check for links. (David Pope, Ken Takata)
相关文件: src/fileio.c, src/os_win32.c, src/proto/os_win32.pro

Patch 7.3.1183
问题: Python tests 86 and 87 fail.
解决方案: Add "empty" files. (ZyX)
相关文件: src/testdir/python_before/before_1.py,
src/testdir/python_before/before_2.py

Patch 7.3.1184
问题: Highlighting is sometimes wrong. (Axel Bender)
解决方案: Fetch regline again when returning from recursive regmatch.
相关文件: src/regexp_nfa.c

Patch 7.3.1185
问题: New regexp engine: no match with ^ after \n. (SungHyun Nam)
解决方案: Fix it, add a test.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1186

问题:　　Python 3: test 87 may crash.
解决方案:　Use _PyArg_Parse_SizeT instead of PyArg_Parse. (Jun Takimoto)
相关文件:　src/if_python3.c

Patch 7.3.1187 (after 7.3.1170)
问题:　　"s:" is recognized but "<SID>" is not. (ZyX)
解决方案:　Translate "<SID>" like "s:".
相关文件:　src/eval.c

Patch 7.3.1188
问题:　　Newline characters messing up error message.
解决方案:　Remove the newlines. (Kazunobu Kuriyama)
相关文件:　src/gui_x11.c

Patch 7.3.1189 (after 7.3.1185)
问题:　　Highlighting is still wrong sometimes. (Dominique Pelle)
解决方案:　Also restore reginput properly.
相关文件:　src/regexp_nfa.c

Patch 7.3.1190
问题:　　Compiler warning for parentheses. (Christian Wellenbrock)
解决方案:　Change #ifdef.
相关文件:　src/ex_docmd.c

Patch 7.3.1191
问题:　　Backreference to previous line doesn't work. (Lech Lorens)
解决方案:　Implement looking in another line.
相关文件:　src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in,
　　src/testdir/test64.ok

Patch 7.3.1192
问题:　　Valgrind reports errors when using backreferences. (Dominique
　　Pelle)
解决方案:　Do not check the end of submatches.
相关文件:　src/regexp_nfa.c

Patch 7.3.1193
问题:　　fail_if_missing not used for Python 3.
解决方案:　Give an error when Python 3 can't be configured. (Andrei Olsen)
相关文件:　src/configure.in, src/auto/configure

Patch 7.3.1194
问题:　　Yaml highlighting is slow.
解决方案:　Tune the estimation of pattern failure chance.
相关文件:　src/regexp_nfa.c

Patch 7.3.1195
问题:　　Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案:　Set the length to the matching backref.
相关文件:　src/regexp.c

Patch 7.3.1196
问题:　　Old regexp engine does not match pattern with backref correctly.
　　(Dominique Pelle)

解决方案: Fix setting status. Test multi-line patterns better.
相关文件: src/regexp.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1197
问题: ":wviminfo!" does not write history previously read from a viminfo file. (Roland Eggner)
解决方案: When not merging history write all entries.
相关文件: src/ex_cmds.c, src/ex_getln.c, src/proto/ex_getln.pro

Patch 7.3.1198
问题: Build error when using Perl 5.18.0 and dynamic loading.
解决方案: Change #ifdefs for Perl_croak_xs_usage. (Ike Devolder)
相关文件: src/if_perl.xs

Patch 7.3.1199
问题: When evaluating 'foldexpr' causes an error this is silently ignored and evaluation is retried every time.
解决方案: Set emsg_silent instead of emsg_off. Stop evaluating 'foldexpr' is it is causing errors. (Christian Brabandt)
相关文件: src/fold.c

Patch 7.3.1200
问题: When calling setline() from Insert mode, using CTRL-R =, undo does not work properly. (Israel Chauca)
解决方案: Sync undo after evaluating the expression. (Christian Brabandt)
相关文件: src/edit.c, src/testdir/test61.in, src/testdir/test61.ok

Patch 7.3.1201
问题: When a startup script creates a preview window, it probably becomes the current window.
解决方案: Make another window the current one. (Christian Brabandt)
相关文件: src/main.c

Patch 7.3.1202 (after 7.3.660)
问题: Tags are not found in case-folded tags file. (Darren cole, Issue 90)
解决方案: Take into account that when case folding was used for the tags file "!rm" sorts before the "!_TAG" header lines.
相关文件: src/tag.c

Patch 7.3.1203
问题: Matches from matchadd() might be highlighted incorrectly when they are at a fixed position and inserting lines. (John Szakmeister)
解决方案: Redraw all lines below a change if there are highlighted matches. (idea by Christian Brabandt)
相关文件: src/screen.c

Patch 7.3.1204
问题: Calling gettabwinvar() in 'tabline' cancels Visual mode. (Hirohito Higashi)
解决方案: Don't always use goto_tabpage_tp().
相关文件: src/window.c, src/proto/window.pro, src/eval.c, src/if_py_both.h

Patch 7.3.1205

问题: logtalk.dict is not removed on uninstall.
解决方案: Remove the file. (Kazunobu Kuriyama)
相关文件: src/Makefile

Patch 7.3.1206
问题: Inconsistent function argument declarations.
解决方案: Use ANSI style.
相关文件: src/if_py_both.h

Patch 7.3.1207
问题: New regexp engine: no match found on "#if FOO". (Lech Lorens)
解决方案: When adding a state gets skipped don't adjust the index.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1208
问题: Compiler warnings on MS-Windows.
解决方案: Add type cast. Move variable declaration. (Mike Williams)
相关文件: src/option.c, src/os_mswin.c

Patch 7.3.1209
问题: No completion for ":tabdo".
解决方案: Add tabdo to the list of modifiers. (Dominique Pelle)
相关文件: src/ex_docmd.c

Patch 7.3.1210 (after 7.3.1182)
问题: 'backupcopy' default on MS-Windows is wrong when 'encoding' equals
            the current codepage.
解决方案: Change the #else block. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.3.1211
问题: MS-Windows: When 'encoding' differs from the current codepage
            ":hardcopy" does not work properly.
解决方案: Use TextOutW() and SetDlgItemTextW(). (Ken Takata)
相关文件: src/os_mswin.c, src/vim.rc

Patch 7.3.1212
问题: "make test" on MS-Windows does not report failure like Unix does.
解决方案: Make it work like on Unix. (Taro Muraoka)
相关文件: src/testdir/Make_dos.mak

Patch 7.3.1213
问题: Can't build with small features and Python.
解决方案: Adjust #ifdefs.
相关文件: src/eval.c, src/buffer.c, src/eval.c, src/window.c

Patch 7.3.1214
问题: Missing declaration for init_users() and realloc_post_list().
            (Salman Halim)
解决方案: Add the declarations.
相关文件: src/misc1.c, src/regexp_nfa.c

Patch 7.3.1215
问题: Compiler warning for function not defined.

解决方案: Add #ifdef.
相关文件: src/misc1.c

Patch 7.3.1216
问题: Configure can't find Motif on Ubuntu.
解决方案: Search for libXm in /usr/lib/*-linux-gnu.
相关文件: src/configure.in, src/auto/configure

Patch 7.3.1217
问题: New regexp engine: Can't handle \%[[ao]]. (Yukihiro Nakadaira)
解决方案: Support nested atoms inside \%[].
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1218
问题: "make test" on MS-Windows does not clean all temporary files and
gives some unnecessary message.
解决方案: Clean the right files. Create .failed files. (Ken Takata)
相关文件: src/testdir/Make_dos.mak

Patch 7.3.1219
问题: No test for using [[]] inside \%[].
解决方案: Add a test.
相关文件: src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1220
问题: MS-Windows: When using wide font italic and bold are not included.
解决方案: Support wide-bold, wide-italic and wide-bold-italic. (Ken Takata,
Taro Muraoka)
相关文件: src/gui.c, src/gui.h, src/gui_w48.c

Patch 7.3.1221
问题: When build flags change "make distclean" run into a configure
error.
解决方案: When CFLAGS changes delete auto/config.cache. Also avoid adding
duplicate text to flags. (Ken Takata)
相关文件: src/Makefile, src/configure.in, src/auto/configure

Patch 7.3.1222
问题: Cannot execute some tests from the src directly.
解决方案: Add missing targets.
相关文件: src/Makefile

Patch 7.3.1223
问题: Tests fail on MS-Windows.
解决方案: Avoid depending on OS version. Use DOS commands instead of Unix
commands. (Taro Muraoka, Ken Takata)
相关文件: src/testdir/test17.in, src/testdir/test50.in,
src/testdir/test71.in, src/testdir/test77.in

Patch 7.3.1224
问题: Clang gives warnings on xxd.
解决方案: Change how to use part of a string. (Dominique Pelle) Also avoid
warning for return not reached.
相关文件: src/xxd/xxd.c, src/regexp_nfa.c

Patch 7.3.1225
问题:　　Compiler warnings when building with Motif.
解决方案:　Change set_label() argument. (Kazunobu Kuriyama)
相关文件:　src/gui_motif.c

Patch 7.3.1226
问题:　　Python: duplicate code.
解决方案:　Share code between OutputWrite() and OutputWritelines(). (ZyX)
相关文件:　src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1227
问题:　　Inconsistent string conversion.
解决方案:　Use 'encoding' instead of utf-8. Use METH_O in place of
　　METH_VARARGS where appropriate. (ZyX)
相关文件:　src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1228
问题:　　Python: various inconsistencies and problems.
解决方案:　StringToLine now supports both bytes() and unicode() objects.
　　Make function names consistent. Fix memory leak fixed in
　　StringToLine. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c

Patch 7.3.1229
问题:　　Python: not so easy to delete/restore translating.
解决方案:　Make macros do translation of exception messages. (ZyX)
　　Note: this breaks translations!
相关文件:　src/if_py_both.h, src/if_python3.c

Patch 7.3.1230
问题:　　Python: Exception messages are not clear.
解决方案:　Make exception messages more verbose. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1231
问题:　　Python: use of numbers not consistent.
解决方案:　Add support for Number protocol. (ZyX)
相关文件:　src/if_py_both.h, src/if_python3.c, src/if_python.c,
　　src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1232
问题:　　Python: inconsistencies in variable names.
解决方案:　Rename variables. (ZyX)
相关文件:　src/eval.c, src/if_py_both.h

Patch 7.3.1233
问题:　　Various Python problems.
解决方案:　Fix VimTryEnd. Crash with debug build and PYTHONDUMPREFS=1. Memory
　　leaks in StringToLine(), BufferMark() and convert_dl. (ZyX)
相关文件:　src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
　　src/testdir/test87.in, src/testdir/test87.ok

Patch 7.3.1234 (after 7.3.1229)
问题: Python: Strings are not marked for translation.
解决方案: Add N_() where appropriate. (ZyX)
相关文件: src/if_py_both.h

Patch 7.3.1235
问题: In insert mode CTRL-] is not inserted, on the command-line it is.
解决方案: Don't insert CTRL-] on the command line. (Yukihiro Nakadaira)
相关文件: src/ex_getln.c

Patch 7.3.1236
问题: Python: WindowSetattr() missing support for NUMBER_UNSIGNED.
解决方案: Add NUMBER_UNSIGNED, add more tests. Various fixes. (ZyX)
相关文件: src/if_py_both.h, src/if_python3.c, src/if_python.c,
src/testdir/pythonx/failing.py,
src/testdir/pythonx/failing_import.py, src/testdir/test86.in,
src/testdir/test86.ok, src/testdir/test87.in,
src/testdir/test87.ok, src/testdir/pythonx/topmodule/__init__.py,
src/testdir/pythonx/topmodule/submodule/__init__.py,
src/testdir/pythonx/topmodule/submodule/subsubmodule/__init__.py,
src/testdir/pythonx/topmodule/submodule/subsubmodule/subsubsubmodule.py

Patch 7.3.1237
问题: Python: non-import errors not handled correctly.
解决方案: Let non-ImportError exceptions pass the finder. (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.3.1238
问题: Crash in Python interface on 64 bit machines.
解决方案: Change argument type of PyString_AsStringAndSize. (Taro Muraoka,
Jun Takimoto)
相关文件: src/if_python.c

Patch 7.3.1239
问题: Can't build with Python and MSVC10.
解决方案: Move #if outside of macro. (Taro Muraoka)
相关文件: src/if_py_both.h

Patch 7.3.1240
问题: Memory leak in findfile().
解决方案: Free the memory. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.3.1241 (after 7.3.1236)
问题: Some test files missing from the distribution.
解决方案: Update the list of files.
相关文件: Filelist

Patch 7.3.1242
问题: No failure when trying to use a number as a string.
解决方案: Give an error when StringToLine() is called with an instance of
the wrong type. (Jun Takimoto)
相关文件: src/if_py_both.h

Patch 7.3.1243
问题: New regexp engine: back references in look-behind match don't work. (Lech Lorens)
解决方案: Copy the submatches before a recursive match. Also fix function prototypes.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1244
问题: MS-Windows: confirm() dialog text may not fit.
解决方案: Use GetTextWidthEnc() instead of GetTextWidth(). (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 7.3.1245
问题: MS-Windows: confirm() dialog text may still not fit.
解决方案: Use GetTextWidthEnc() instead of GetTextWidth() in two more places. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 7.3.1246
问题: When setting 'winfixheight' and resizing the window causes the window layout to be wrong.
解决方案: Add frame_check_height() and frame_check_width() (Yukihiro Nakadaira)
相关文件: src/window.c

Patch 7.3.1247
问题: New regexp engine: '[ ]\@!\p\%([ ]\@!\p\)*:' does not always match.
解决方案: When there is a PIM add a duplicate state that starts at another position.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1248
问题: Still have old hacking code for Input Method.
解决方案: Add 'imactivatefunc' and 'imstatusfunc' as a generic solution to Input Method activation. (Yukihiro Nakadaira)
相关文件: runtime/doc/options.txt, src/fileio.c, src/mbyte.c, src/option.c, src/option.h, src/proto/fileio.pro

Patch 7.3.1249
问题: Modeline not recognized when using "Vim" instead of "vim".
解决方案: Also accept "Vim".
相关文件: src/buffer.c

Patch 7.3.1250
问题: Python tests fail on MS-Windows.
解决方案: Change backslashes to slashes. (Taro Muraoka)
相关文件: src/testdir/test86.in, src/testdir/test87.in

Patch 7.3.1251
问题: Test 61 messes up viminfo.
解决方案: Specify a separate viminfo file.
相关文件: src/testdir/test61.in

Patch 7.3.1252
问题: gvim does not find the toolbar bitmap files in ~/vimfiles/bitmaps if the corresponding menu command contains additional characters like the shortcut marker '&' or if you use a non-english locale.
解决方案: Use menu->en_dname or menu->dname. (Martin Gieseking)
相关文件: src/gui_w32.c

Patch 7.3.1253 (after 7.3.1200)
问题: Still undo problem after using CTRL-R = setline(). (Hirohito Higashi)
解决方案: Set the ins_need_undo flag.
相关文件: src/edit.c

Patch 7.3.1254 (after 7.3.1252)
问题: Can't build without the multi-lang feature. (John Marriott)
解决方案: Add #ifdef.
相关文件: src/gui_w32.c

Patch 7.3.1255
问题: Clang warnings when building with Athena.
解决方案: Add type casts. (Dominique Pelle)
相关文件: src/gui_at_fs.c

Patch 7.3.1256
问题: Can't build without eval or autocmd feature.
解决方案: Add #ifdefs.
相关文件: src/mbyte.c, src/window.c

Patch 7.3.1257
问题: With GNU gettext() ":lang de_DE.utf8" does not always result in German messages.
解决方案: Clear the $LANGUAGE environment variable.
相关文件: src/ex_cmds2.c

Patch 7.3.1258
问题: Using submatch() may crash Vim. (Ingo Karkat)
解决方案: Restore the number of subexpressions used.
相关文件: src/regexp_nfa.c

Patch 7.3.1259
问题: No test for patch 7.3.1258
解决方案: Add a test entry.
相关文件: src/testdir/test64.in, src/testdir/test64.ok

Patch 7.3.1260
问题: User completion does not get the whole command line in the command line window.
解决方案: Pass on the whole command line. (Daniel Thau)
相关文件: src/ex_getln.c, src/structs.h

Patch 7.3.1261 (after patch 7.3.1179)
问题: A buffer-local language mapping from a keymap stops a global insert mode mapping from working. (Ron Aaron)
解决方案: Do not wait for more characters to be typed only when the mapping

was defined with <nowait>.
相关文件: runtime/doc/map.txt, src/eval.c, src/getchar.c,
src/testdir/test75.in, src/testdir/test75.ok

Patch 7.3.1262
问题: Crash and compilation warnings with Cygwin.
解决方案: Check return value of XmbTextListToTextProperty(). Add type casts.
Adjust #ifdefs. (Lech Lorens)
相关文件: src/main.c, src/os_unix.c, src/ui.c

Patch 7.3.1263
问题: Typo in short option name.
解决方案: Change "imse" to "imsf".
相关文件: src/option.c

Patch 7.3.1264 (after 7.3.1261)
问题: Missing m_nowait.
解决方案: Include missing part of the patch.
相关文件: src/structs.h

Patch 7.3.1265 (after 7.3.1249)
问题: Accepting "Vim:" for a modeline causes errors too often.
解决方案: Require "Vim:" to be followed by "set".
相关文件: src/buffer.c

Patch 7.3.1266
问题: QNX: GUI fails to start.
解决方案: Remove the QNX-specific #ifdef. (Sean Boudreau)
相关文件: src/gui.c

Patch 7.3.1267
问题: MS-Windows ACL support doesn't work well.
解决方案: Implement more ACL support. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.3.1268
问题: ACL support doesn't work when compiled with MingW.
解决方案: Support ACL on MingW. (Ken Takata)
相关文件: src/os_win32.c, src/os_win32.h

Patch 7.3.1269
问题: Insert completion keeps entry selected even though the list has
changed. (Olivier Teuliere)
解决方案: Reset compl_shown_match and compl_curr_match. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.3.1270
问题: Using "Vp" in an empty buffer can't be undone. (Hauke Petersen)
解决方案: Save one line in an empty buffer. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.3.1271 (after 7.3.1260)
问题: Command line completion does not work.
解决方案: Move setting xp_line down. (Daniel Thau)

相关文件: src/ex_getln.c

Patch 7.3.1272
问题: Crash when editing Ruby file. (Aliaksandr Rahalevich)
解决方案: Reallocate the state list when necessary.
相关文件: src/regexp_nfa.c

Patch 7.3.1273
问题: When copying a location list the index might be wrong.
解决方案: Set the index to one when using the first entry. (Lech Lorens)
相关文件: src/quickfix.c

Patch 7.3.1274
问题: When selecting an entry from a location list it may pick an
arbitrary window or open a new one.
解决方案: Prefer using a window related to the location list. (Lech Lorens)
相关文件: src/quickfix.c

Patch 7.3.1275
问题: "gn" does not work when the match is a single character.
解决方案: Fix it, add a test. (Christian Brabandt)
相关文件: src/search.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.3.1276
问题: When using a cscope connection resizing the window may send
SIGWINCH to cscope and it quits.
解决方案: Call setpgid(0, 0) in the child process. (Narendran Gopalakrishnan)
相关文件: src/if_cscope.c

Patch 7.3.1277
问题: In diff mode 'cursorline' also draws in the non-active window.
When 'nu' and 'sbr' are set the 'sbr' string is not underlined.
解决方案: Only draw the cursor line in the current window. Combine the
'cursorline' and other highlighting attributes. (Christian
Brabandt)
相关文件: src/screen.c

Patch 7.3.1278
问题: When someone sets the screen size to a huge value with "stty" Vim
runs out of memory before reducing the size.
解决方案: Limit Rows and Columns in more places.
相关文件: src/gui.c, src/gui_gtk_x11.c, src/option.c, src/os_unix.c,
src/proto/term.pro, src/term.c

Patch 7.3.1279
问题: Compiler warning for variable uninitialized. (Tony Mechelynck)
解决方案: Add an init.
相关文件: src/ex_getln.c

Patch 7.3.1280
问题: Reading memory already freed since patch 7.3.1247. (Simon
Ruderich, Dominique Pelle)
解决方案: Copy submatches before reallocating the state list.
相关文件: src/regexp_nfa.c

Patch 7.3.1281
问题: When 'ttymouse' is set to "xterm2" clicking in column 123 moves the cursor to column 96. (Kevin Goodsell)
解决方案: Decode KE_CSI.
相关文件: src/term.c

Patch 7.3.1282 (after 7.3.1277)
问题: 'cursorline' not drawn in any other window. (Charles Campbell)
解决方案: Do draw the cursor line in other windows.
相关文件: src/screen.c

Patch 7.3.1283
问题: Test 71 fails on MS-Windows.
解决方案: Put the binary data in a separate file. (Ken Takata)
相关文件: src/testdir/test71.in, src/testdir/test71a.in

Patch 7.3.1284
问题: Compiler warnings in MS-Windows clipboard handling.
解决方案: Add type casts. (Ken Takata)
相关文件: src/winclip.c

Patch 7.3.1285
问题: No tests for picking a window when selecting an entry in a location list. Not picking the right window sometimes.
解决方案: Add test 96. Set usable_win appropriately. (Lech Lorens)
相关文件: src/quickfix.c, src/testdir/Makefile, src/testdir/test96.in, src/testdir/test96.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.3.1286
问题: Check for screen size missing for Athena and Motif.
解决方案: Add call to limit_screen_size().
相关文件: src/gui_x11.c

Patch 7.3.1287
问题: Python SystemExit exception is not handled properly.
解决方案: Catch the exception and give an error. (Yasuhiro Matsumoto, Ken Takata)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.3.1288
问题: The first ":echo 'hello'" command output doesn't show. Mapping for <S-F3> gets triggered during startup.
解决方案: Add debugging code for the termresponse. When receiving the "Co" entry and when setting 'ambiwidth' redraw right away if possible. Add redraw_asap(). Don't set 'ambiwidth' if it already had the right value. Do the 'ambiwidth' check in the second row to avoid confusion with <S-F3>.
相关文件: src/term.c, src/screen.c, src/proto/screen.pro

Patch 7.3.1289

问题:　　Get GLIB warning when removing a menu item.
解决方案:　Reference menu-id and also call gtk_container_remove(). (Ivan Krasilnikov)
相关文件:　src/gui_gtk.c

Patch 7.3.1290 (after 7.3.1253)
问题:　　CTRL-R = in Insert mode changes the start of the insert position. (Ingo Karkat)
解决方案:　Only break undo, don't start a new insert.
相关文件:　src/edit.c

Patch 7.3.1291 (after 7.3.1288)
问题:　　Compiler warnings for uninitialized variables. (Tony Mechelynck)
解决方案:　Initialize the variables.
相关文件:　src/screen.c

Patch 7.3.1292
问题:　　Possibly using invalid pointer when searching for window. (Raichoo)
解决方案:　Use "firstwin" instead of "tp_firstwin" for current tab.
相关文件:　src/window.c

Patch 7.3.1293
问题:　　Put in empty buffer cannot be undone.
解决方案:　Save one more line for undo. (Ozaki)
相关文件:　src/ops.c

Patch 7.3.1294
问题:　　":diffoff" resets options.
解决方案:　Save and restore option values. (Christian Brabandt)
相关文件:　src/diff.c, src/structs.h, src/option.c

Patch 7.3.1295
问题:　　glob() and globpath() do not handle escaped special characters properly.
解决方案:　Handle escaped characters differently. (Adnan Zafar)
相关文件:　src/testdir/Makefile, src/testdir/test97.in, src/testdir/test97.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/fileio.c, src/misc1.c

Patch 7.3.1296
问题:　　Only MS-Windows limits the GUI window size to what fits on the monitor.
解决方案:　Limit the size for all systems. (Daniel Harding)
相关文件:　src/ui.c

Patch 7.3.1297
问题:　　findfile() directory matching does not work when a star follows text. (Markus Braun)
解决方案:　Make a wildcard work properly. (Christian Brabandt)
相关文件:　src/misc2.c, src/testdir/test89.in, src/testdir/test89.ok

Patch 7.3.1298 (after 7.3.1297)

问题:　　Crash.
解决方案:　Use STRCPY() instead of STRCAT() and allocate one more byte.
相关文件:　src/misc2.c

Patch 7.3.1299
问题:　　Errors when doing "make proto". Didn't do "make depend" for a
　　while.
解决方案:　Add #ifdefs. Update dependencies. Update proto files.
相关文件:　src/if_python3.c, src/os_win32.c, src/Makefile,
　　src/proto/ex_docmd.pro, src/proto/if_python.pro,
　　src/proto/if_python3.pro, src/proto/gui_w16.pro,
　　src/proto/gui_w32.pro, src/proto/os_win32.pro

Patch 7.3.1300
问题:　　Mac: tiny and small build fails.
解决方案:　Don't include os_macosx.m in tiny build. Include mouse support in
　　small build. (Kazunobu Kuriyama)
相关文件:　src/configure.in, src/auto/configure, src/vim.h

Patch 7.3.1301
问题:　　Some tests fail on MS-Windows.
解决方案:　Fix path separators in test 89 and 96. Omit test 97, escaping
　　works differently. Make findfile() work on MS-Windows.
相关文件:　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
　　src/testdir/Make_os2.mak, src/testdir/test89.in,
　　src/testdir/test96.in, src/misc2.c

Patch 7.3.1302
问题:　　Test 17 fails on MS-Windows. Includes line break in file name
　　everywhere.
解决方案:　Fix 'fileformat'. Omit CR-LF from a line read from an included
　　file.
相关文件:　src/search.c, src/testdir/test17.in, src/testdir/test17.ok

Patch 7.3.1303 (after 7.3.1290)
问题:　　Undo is synced whenever CTRL-R = is called, breaking some plugins.
解决方案:　Only break undo when calling setline() or append().
相关文件:　src/globals.h, src/eval.c, src/edit.c, src/testdir/test61.in,
　　src/testdir/test61.ok

Patch 7.3.1304
问题:　　Test 89 still fails on MS-Windows.
解决方案:　Set 'shellslash'. (Taro Muraoka)
相关文件:　src/testdir/test89.in

Patch 7.3.1305
问题:　　Warnings from 64 bit compiler.
解决方案:　Add type casts.
相关文件:　src/misc2.c

Patch 7.3.1306
问题:　　When redrawing the screen during startup the intro message may be
　　cleared.
解决方案:　Redisplay the intro message when appropriate.

相关文件: src/screen.c, src/version.c, src/proto/version.pro

Patch 7.3.1307
问题: MS-Windows build instructions are outdated.
解决方案: Adjust for building on Windows 7. Drop Windows 95/98/ME support.
相关文件: Makefile, nsis/gvim.nsi

Patch 7.3.1308
问题: Typos in MS-Windows build settings and README.
解决方案: Minor changes to MS-Windows files.
相关文件: src/msvc2008.bat, src/msvc2010.bat, src/VisVim/README_VisVim.txt

Patch 7.3.1309
问题: When a script defines a function the flag to wait for the user to hit enter is reset.
解决方案: Restore the flag. (Yasuhiro Matsumoto) Except when the user was typing the function.
相关文件: src/eval.c

Patch 7.3.1310
问题: Typos in nsis script. Can use better compression.
解决方案: Fix typos. Use lzma compression. (Ken Takata)
相关文件: nsis/gvim.nsi

Patch 7.3.1311
问题: Compiler warnings on Cygwin.
解决方案: Add type casts. Add windows include files. (Ken Takata)
相关文件: src/mbyte.c, src/ui.c

Patch 7.3.1312 (after 7.3.1287)
问题: Not giving correct error messages for SystemExit().
解决方案: Move E858 into an else. (Ken Takata)
相关文件: src/if_py_both.h

Patch 7.3.1313
问题: :py and :py3 don't work when compiled with Cygwin or MingW with 64 bit.
解决方案: Add -DMS_WIN64 to the build command. (Ken Takata)
相关文件: src/Make_cyg.mak, src/Make_ming.mak

Patch 7.3.1314
问题: Test 87 fails with Python 3.3.
解决方案: Filter the error messages. (Taro Muraoka)
相关文件: src/testdir/test87.in

Patch 7.4a.001
问题: Script to update syntax menu is outdated.
解决方案: Add the missing items.
相关文件: runtime/makemenu.vim

Patch 7.4a.002
问题: Valgrind errors in test 89. (Simon Ruderich)
解决方案: Allocate one more byte. (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.4a.003
问题:　　Copyright year is outdated.
解决方案:　Only use the first year.
相关文件:　src/vim.rc, src/vim16.rc

Patch 7.4a.004
问题:　　MSVC 2012 Update 3 is not recognized.
解决方案:　Add the version number. (Raymond Ko)
相关文件:　src/Make_mvc.mak

Patch 7.4a.005
问题:　　Scroll binding causes unexpected scroll.
解决方案:　Store the topline after updating scroll binding. Add a test.
　　(Lech Lorens)
相关文件:　src/testdir/test98.in, src/testdir/test98a.in,
　　src/testdir/test98.ok, src/option.c, src/testdir/Make_amiga.mak,
　　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
　　src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
　　src/testdir/Makefile

Patch 7.4a.006
问题:　　Failure in po file check goes unnoticed.
解决方案:　Fail "make test" if the po file check fails.
相关文件:　src/Makefile

Patch 7.4a.007
问题:　　After "g$" with 'virtualedit' set, "k" moves to a different
　　column. (Dimitar Dimitrov)
解决方案:　Set w_curswant. (Christian Brabandt)
相关文件:　src/normal.c

Patch 7.4a.008
问题:　　Python 3 doesn't handle multibyte characters properly when
　　'encoding' is not utf-8.
解决方案:　Use PyUnicode_Decode() instead of PyUnicode_FromString(). (Ken
　　Takata)
相关文件:　src/if_python3.c

Patch 7.4a.009
问题:　　Compiler warnings for function prototypes.
解决方案:　Add "void". Move list_features() prototype. (Ken Takata)
相关文件:　src/gui_w48.c, src/if_py_both.h, src/version.c

Patch 7.4a.010
问题:　　Test 86 and 87 fail when building with Python or Python 3 and
　　using a static library.
解决方案:　Add configure check to add -fPIE compiler flag.
相关文件:　src/configure.in, src/auto/configure

Patch 7.4a.011
问题:　　Configure check for Python 3 config name isn't right.
解决方案:　Always include vi_cv_var_python3_version. (Tim Harder)
相关文件:　src/configure.in, src/auto/configure

Patch 7.4a.012
问题:　　"make test" fails when using a shadow directory.
解决方案:　Create links for files in src/po. (James McCoy)
相关文件:　src/Makefile

Patch 7.4a.013
问题:　　Setting/resetting 'lbr' in the main help file changes alignment
after a Tab. (Dimitar Dimitrov)
解决方案:　Also use the code for conceal mode where n_extra is computed for
'lbr'.
相关文件:　src/screen.c, src/testdir/test88.in, src/testdir/test88.ok

Patch 7.4a.014
问题:　　Test 86 and 89 have a problem with using a shadow dir.
解决方案:　Adjust for the different directory structure. (James McCoy)
相关文件:　src/testdir/test89.in, src/testdir/test86.in, src/Makefile

Patch 7.4a.015
问题:　　No Japanese man pages.
解决方案:　Add Japanese translations of man pages. (Ken Takata, Yukihiro
Nakadaira, et al.)
相关文件:　Filelist, src/Makefile, runtime/doc/evim-ja.UTF-8.1,
runtime/doc/vim-ja.UTF-8.1, runtime/doc/vimdiff-ja.UTF-8.1,
runtime/doc/vimtutor-ja.UTF-8.1, runtime/doc/xxd-ja.UTF-8.1

Patch 7.4a.016 (after 7.4a.014)
问题:　　Features enabled in Makefile.
解决方案:　Undo accidental changes.
相关文件:　src/Makefile

Patch 7.4a.017
问题:　　When 'foldmethod' is "indent", using ">>" on a line just above a
fold makes the cursor line folded. (Evan Laforge)
解决方案:　Call foldOpenCursor(). (Christian Brabandt)
相关文件:　src/ops.c

Patch 7.4a.018
问题:　　Compiler warning for code unreachable. (Charles Campbell)
解决方案:　Use "while" instead of endless loop. Change break to continue.
相关文件:　src/regexp_nfa.c, src/ui.c

Patch 7.4a.019
问题:　　Invalid closing parenthesis in test 62. Command truncated at
double quote.
解决方案:　Remove the parenthesis. Change double quote to ''. (ZyX)
相关文件:　src/testdir/test62.in, src/testdir/test62.ok

Patch 7.4a.020
问题:　　Superfluous mb_ptr_adv().
解决方案:　Remove the call. (Dominique Pelle)
相关文件:　src/regexp_nfa.c

Patch 7.4a.021

问题:　　Using feedkeys() doesn't always work.
解决方案:　Omit feedkeys(). (Ken Takata)
相关文件:　src/testdir/test98a.in

Patch 7.4a.022
问题:　　Using "d2g$" does not delete the last character. (ZyX)
解决方案:　Set the "inclusive" flag properly.
相关文件:　src/normal.c

Patch 7.4a.023 (after 7.4a.019)
问题:　　Still another superfluous parenthesis. (ZyX)
解决方案:　Remove it.
相关文件:　src/testdir/test62.in

Patch 7.4a.024
问题:　　X11 GUI: Checking icon height twice.
解决方案:　Check height and width. (Dominique Pelle)
相关文件:　src/gui_x11.c

Patch 7.4a.025
问题:　　Get the press-Enter prompt even after using :redraw.
解决方案:　Clear need_wait_return when executing :redraw.
相关文件:　src/ex_docmd.c

Patch 7.4a.026
问题:　　":diffoff" does not remove folds. (Ramel)
解决方案:　Do not restore 'foldenable' when 'foldmethod' is "manual".
相关文件:　src/diff.c

Patch 7.4a.027
问题:　　When Python adds lines to another buffer the cursor position is
　　wrong, it might be below the last line causing ml_get errors.
　　(Vlad Irnov)
解决方案:　Temporarily change the current window, so that marks are corrected
　　properly.
相关文件:　src/if_py_both.h, src/window.c, src/proto/buffer.pro

Patch 7.4a.028
问题:　　Crash when spell checking in new buffer.
解决方案:　Set the b_p_key field.　(Mike Williams)
相关文件:　src/spell.c, src/testdir/test58.in

Patch 7.4a.029
问题:　　Can't build with MzScheme on Ubuntu 13.04.
解决方案:　Add configure check for the "ffi" library.
相关文件:　src/configure.in, src/auto/configure

Patch 7.4a.030 (after 7.4.027)
问题:　　Missing find_win_for_buf(). (toothpik)
解决方案:　Add missing changes.
相关文件:　src/buffer.c

Patch 7.4a.031
问题:　　Compiler warnings. (Charles Campbell)

解决方案: Initialize variables even when not needed.
相关文件: src/regexp_nfa.c, src/search.c

Patch 7.4a.032
问题: New regexp engine: Does not match shorter alternative. (Ingo Karkat)
解决方案: Do not drop a new state when the PIM info is different.
相关文件: src/regexp_nfa.c

Patch 7.4a.033
问题: Test 98 always passes.
解决方案: Include test98a.in in test98.in, execute the crucial command in one line. (Yukihiro Nakadaira)
相关文件: src/testdir/test98.in, src/testdir/test98a.in

Patch 7.4a.034
问题: The tabline may flicker when opening a new tab after 7.3.759 on Win32.
解决方案: Move call to TabCtrl_SetCurSel(). (Ken Takata)
相关文件: src/gui_w48.c

Patch 7.4a.035
问题: Fix in patch 7.4a.032 is not tested.
解决方案: Add test.
相关文件: src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4a.036
问题: "\p" in a regexp does not match double-width characters. (Yukihiro Nakadaira)
解决方案: Don't count display cells, use vim_isprintc().
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.4a.037
问题: Win32: When mouse is hidden and in the toolbar, moving it won't make it appear. (Sami Salonen)
解决方案: Add tabline_wndproc() and toolbar_wndproc(). (Ken Takata)
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 7.4a.038
问题: When using MSVC 2012 there are various issues, including GUI size computations.
解决方案: Use SM_CXPADDEDBORDER. (Mike Williams)
相关文件: src/gui_w32.c, src/gui_w48.c, src/os_win32.h

Patch 7.4a.039
问题: New regexp engine doesn't match pattern. (Ingo Karkat)
解决方案: When adding a state also check for different PIM if the list of states has any state with a PIM.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4a.040
问题: Win32: using uninitialized variable.

解决方案: (Yukihiro Nakadaira)
相关文件: src/os_win32.c

Patch 7.4a.041
问题: When using ":new ++ff=unix" and "dos" is first in 'fileformats'
then 'ff' is set to "dos" instead of "unix". (Ingo Karkat)
解决方案: Create set_file_options() and invoke it from do_ecmd().
相关文件: src/fileio.c, src/proto/fileio.pro, src/ex_cmds.c,
src/testdir/test91.in, src/testdir/test91.ok

Patch 7.4a.042
问题: Crash when BufUnload autocommands close all buffers. (Andrew
Pimlott)
解决方案: Set curwin->w_buffer to curbuf to avoid NULL.
相关文件: src/window.c, src/testdir/test8.in, src/testdir/test8.ok

Patch 7.4a.043
问题: More ml_get errors when adding or deleting lines from Python.
(Vlad Irnov)
解决方案: Switch to a window with the buffer when possible.
相关文件: src/if_py_both.h

Patch 7.4a.044
问题: Test 96 sometimes fails.
解决方案: Clear window from b_wininfo in win_free(). (Suggestion by
Yukihiro Nakadaira)
相关文件: src/window.c

Patch 7.4a.045
问题: Configure does not always find the right library for Lua. Missing
support for LuaJit.
解决方案: Improve the configure detection of Lua. (Hiroshi Shirosaki)
相关文件: src/Makefile, src/configure.in, src/auto/configure

Patch 7.4a.046
问题: Can't build without mbyte feature.
解决方案: Add #ifdefs.
相关文件: src/ex_cmds.c

Patch 7.4a.047
问题: Some comments are not so nice.
解决方案: Change the comments.
相关文件: src/ex_docmd.c, src/message.c, src/ops.c, src/option.c

Patch 7.4b.001
问题: Win32: dialog may extend off-screen.
解决方案: Reduce the size, use correct borders. (Andrei Olsen)
相关文件: src/gui_w32.c

Patch 7.4b.002
问题: Crash searching for \%(\%(\ \d\ -\|\.\)*\|*\). (Marcin
Szamotulski) Also for \(\)*.
解决方案: Do add a state for opening parenthesis, so that we can check if it
was added before at the same position.

相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4b.003
问题: Regexp code is not nicely aligned.
解决方案: Adjust white space. (Ken Takata)
相关文件: src/regexp_nfa.c

Patch 7.4b.004
问题: Regexp crash on pattern "@\%[\w\-]*". (Axel Kielhorn)
解决方案: Add \%(\) around \%[] internally.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4b.005
问题: Finding %s in shellpipe and shellredir does not ignore %%s.
解决方案: Skip over %%. (lcd 47)
相关文件: src/ex_cmds.c

Patch 7.4b.006 (after 7.3.1295)
问题: Using \{n,m} in an autocommand pattern no longer works.
Specifically, mutt temp files are not recognized. (Gary Johnson)
解决方案: Make \\\{n,m\} work.
相关文件: runtime/doc/autocmd.txt, src/fileio.c

Patch 7.4b.007
问题: On 32 bit MS-Windows :perldo does not work.
解决方案: Make sure time_t uses 32 bits. (Ken Takata)
相关文件: src/if_perl.xs, src/vim.h

Patch 7.4b.008
问题: 'autochdir' causes setbufvar() to change the current directory.
(Ben Fritz)
解决方案: When disabling autocommands also reset 'acd' temporarily.
(Christian Brabandt)
相关文件: src/fileio.c

Patch 7.4b.009
问题: When setting the Visual area manually and 'selection' is
exclusive, a yank includes one character too much. (Ingo Karkat)
解决方案: Default the Visual operation to "v". (Christian Brabandt)
相关文件: src/mark.c

Patch 7.4b.010
问题: Win32: Tcl library load does not use standard mechanism.
解决方案: Call vimLoadLib() instead of LoadLibraryEx(). (Ken Takata)
相关文件: src/if_perl.xs, src/if_tcl.c

Patch 7.4b.011
问题: ":he \%(\)" does not work. (ZyX)
解决方案: Add an exception to the list.
相关文件: src/ex_cmds.c

Patch 7.4b.012
问题: Output from a shell command is truncated at a NUL. (lcd 47)
解决方案: Change every NUL into an SOH.

相关文件: src/misc1.c

Patch 7.4b.013
问题: Install dir for JP man pages is wrong.
解决方案: Remove ".UTF-8" from the directory name. (Ken Takata)
相关文件: src/Makefile

Patch 7.4b.014 (after 7.4b.012)
问题: Stupid mistake.
解决方案: Changle "len" to "i".
相关文件: src/misc1.c

Patch 7.4b.015 (after 7.4b.008)
问题: Can't compile without the 'acd' feature.
解决方案: Add #ifdefs. (Kazunobu Kuriyama)
相关文件: src/fileio.c

Patch 7.4b.016
问题: Ruby detection fails on Fedora 19.
解决方案: Use one way to get the Ruby version. (Michael Henry)
相关文件: src/configure.in, src/auto/configure

Patch 7.4b.017
问题: ":he \^x" gives a strange error message. (glts)
解决方案: Do not translate \^x to _CTRL-x.
相关文件: src/ex_cmds.c

Patch 7.4b.018 (after 7.4b.001)
问题: Win32: Dialog can still be too big.
解决方案: Move the check for height further down. (Andrei Olsen)
相关文件: src/gui_w32.c

Patch 7.4b.019 (after 7.4a.034)
问题: Tabline is not updated properly when closing a tab on Win32.
解决方案: Only reduce flickering when adding a tab. (Ken Takata)
相关文件: src/gui_w48.c

Patch 7.4b.020
问题: "g~ap" changes first character of next paragraph. (Manuel Ortega)
解决方案: Avoid subtracting (0 - 1) from todo. (Mike Williams)
相关文件: src/ops.c, src/testdir/test82.in, src/testdir/test82.ok

Patch 7.4b.021
问题: Pressing "u" after an external command results in multiple
press-enter messages. (glts)
解决方案: Don't call hit_return_msg() when we have K_IGNORE. (Christian
Brabandt)
相关文件: src/message.c

Patch 7.4b.022
问题: Not waiting for a character when the tick count overflows.
解决方案: Subtract the unsigned numbers and cast to int. (Ken Takata)
相关文件: src/os_win32.c

vim:tw=78:ts=8:noet:ft=help:norl:

version8.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

vim8 vim-8 version-8.0 version8.0

欢迎来到 Vim 版本 8！新版本修正了大量的漏洞，也增加了几个好用的特性。本文档列出 Vim 7.4 以来所有加入的新项目和已有特性的改动。Vim 7.4 及之前的补丁可见：vim-7.4 .

使用此命令可见正在使用的 Vim 程序的完整版本和特性信息：

```
:version
```


新 特 性 new-8
Vim 脚本增强 new-vim-script-8
若干新项目 new-items-8

不 兼 容 的 改 动 incompatible-8

改 进 improvements-8

编 译 时 的 改 变 compile-changes-8

补 丁 patches-8

8.1 版 本 version-8.1
改动 changed-8.1
新增 added-8.1
补丁 patches-8.1

8.2 版 本 version-8.2
改动 changed-8.2
新增 added-8.2
补丁 patches-8.2

vi_diff.txt 提供 Vi 和 Vim 8.0 的区别的一个概述。
version4.txt 、 version5.txt 、 version6.txt 和 version7.txt 说明其它版本间的区别。

新 特 性 new-8

首先给出最有意思的新特性的概述。下面再给出完整列表。

异步 I/O 支持，通道

Vim 现在可以在后台和其它进程交换信息。这样就可以让服务器完成工作并发送结果给 Vim。例见 channel-demo ，展示和一个 Python 服务器的交流。

和通道紧密相关的是 JSON 支持。JSON 受到广泛支持，可以方便地支持进程间通信，

支持任何语言编写的服务器。要用的函数是 json_encode() 和 json_decode() 。

这样就可以编写十分复杂的插件，使用任何语言并在单独的进程上运行。

作业

Vim 现在可以启动作业，与之交互，并停止作业。可用于运行进程来完成补全，拼写检查等等工作。使用通道机制和作业进行交互。作业也可以读入或写到缓冲区或文件。见 job_start() 。

定时器

定时器是另一种异步机制。可以单次或重复激活并调用函数来完成任何工作。例如：

```
let tempTimer = timer_start(4000, 'CheckTemp')
```

这会在四秒 (4000 毫秒) 后调用 CheckTemp() 函数。见 timer_start() 。

偏函数

Vim 已经有了函数引用，即对一个函数的引用。偏函数也引用函数，并附加对参数和/或字典的绑定。这对通道和定时器的回调特别有用。例如，上面定时器的示例，要给函数传递参数：

```
let tempTimer = timer_start(4000, function('CheckTemp', ['out']))
```

这会在四秒后调用 CheckTemp('out')。

匿名函数和闭包

加入函数创建的快捷方式：{args -> expr}。见 lambda 。
可用于 filter() 和 map() 之类的函数，它们现在也接受函数参数。例如：

```
:call filter(mylist, {idx, val -> val > 20})
```

匿名函数可以使用该函数定义所在的作用域内定义的变量。这通常被称为 closure 。

通过加入 "closure" 参数，用户定义函数也可以是闭包 :func-closure 。

软件包

插件持续增长，可用的数目前所未有。要方便管理诸多插件，引入软件包的支持。这是获得一个或更多插件的方便方式，它们在目录里存放并可一直保持更新。Vim 会自动或按需载入。见 packages 。

新风格测试

用于 Vim 开发者。目前为止，为 Vim 编写测试用例并不容易。Vim 8 加入 assert 函数和支持测试的框架。这样测试的编写和保持更新就简单很多。同时，新增了若干专为测试而引入的函数。见 test-functions 。

窗口 ID

之前窗口只能为它们的编号访问。每次窗口打开、关闭或移动时，此编号都会改变。现在每个窗口有了独一无二的 ID，这样方便它们的查找。见 win_getid() 和 win_id2win() 。

Viminfo 使用时间戳

之前，viminfo 保存上次 Vim 在那里写入的信息。现在使用时间戳，来确保保存的是最新的项目。见 viminfo-timestamp 。

回绕缩进行

加入 'breakindent' 选项以便回绕行而不改变缩进的数量。

Windows: DirectX 支持

加入 'renderoptions' 选项以便 MS-Windows 上切换 DirectX (DirectWrite) 的支持。

GTK+ 3 支持

除了很细微的技术上的差异，GTK+ 3 GUI 的工作方式和 GTK+ 2 类似。如果 2 和 3 同时存在，configure 只选择 GTK + 2。src/Makefile 说明如何选择 GTK+ 3 来代替。详见 gui-x11-compiling 。

Vim 脚本增强 new-vim-script-8

Vim 脚本加入以下新类型：

Special	v:false 、 v:true 、 v:none 和 v:null
Channel	到其它进程的连接，用于支持异步 I/O
Job	进程控制

加入许多函数和命令以支持新类型。

在新系统上，Vim 脚本使用的数值现在使用 64 位。可通过 +num64 特性来检查。

加入很多新项目以支持 new-style-testing 。

printf() 的 %s 现在接受任何类型的参数。用和 string() 类似的方式转换为字符串。

若干新项目 new-items-8

可视模式命令：

v_CTRL-A	CTRL-A	给高亮文本中的数值加上 N
v_CTRL-X	CTRL-X	给高亮文本中的数值减去 N

v_g_CTRL-A	g CTRL-A	给高亮文本中的数值加上 N
v_g_CTRL-X	g CTRL-X	给高亮文本中的数值减去 N

插入模式命令:

i_CTRL-G_U	CTRL-G U	下一个光标移动不打断撤销

命令行模式命令:

/_CTRL-G	CTRL-G	'incsearch' 激活时，转到下个匹配
/_CTRL-T	CTRL-T	'incsearch' 激活时，转到前个匹配

选项:

'belloff'	为这些原因不响铃
'breakindent'	回绕行重复缩进
'breakindentopt'	'breakindent' 的设置
'emoji'	表情字符视作全宽
'fixendofline'	确保文件的末行有 <EOL>
'langremap'	映射字符适用 'langmap'
'luadll'	Lua 动态库名
'packpath'	软件包使用的目录列表
'perldll'	Perl 动态库名
'pythondll'	Python 2 动态库名
'pythonthreedll'	Python 3 动态库名
'renderoptions'	Window 上文本渲染选项
'rubydll'	Ruby 动态库名
'signcolumn'	何时显示标号列
'tagcase'	标签文件查找时如何处理大小写
'tcldll'	Tcl 动态库名
'termguicolors'	终端使用 GUI 颜色

Ex 命令:

:cbottom	滚动到 quickfox 窗口底部
:cdo	为每个合法的错误列表项目执行命令
:cfdo	为每个错误列表的文件执行命令
:chistory	显示 quickfix 列表堆栈
:clearjumps	清除跳转列表
:filter	只输出匹配（或不匹配）的行
:helpclose	关闭帮助窗口
:lbottom	滚动到位置窗口的底部
:ldo	为每个合法位置列表项目执行命令
:lfdo	为每个位置列表的文件执行命令
:lhistory	显示位置列表堆栈
:noswapfile	跟随的若干命令不创建交换文件
:packadd	从 'packpath' 载入插件
:packloadall	从 'packpath' 载入所有包
:smile	让用户高兴一下

Ex 命令修饰符：

:keeppatterns	跟随的命令保持搜索模式历史不变
<mods>	提供用户自定义命令的命令修饰符

新函数及功能扩展的函数：

arglistid()	获取参数列表的编号
assert_equal()	断言两个表达式的值相等
assert_exception()	断言命令抛出例外
assert_fails()	断言函数调用失败
assert_false()	断言表达式为假
assert_inrange()	断言表达式在范围内
assert_match()	断言模式与值匹配
assert_notequal()	断言两个表达式的值不等
assert_notmatch()	断言模式不与值匹配
assert_true()	断言表达式为真
bufwinid()	获取特定缓冲区的窗口 ID
byteidxcomp()	类似于 byteidx()，但计算组合字符
ch_close()	关闭通道
ch_close_in()	关闭通道的 in 部分
ch_evalexpr()	通过通道计算表达式
ch_evalraw()	通过通道计算未经处理的表达式
ch_getbufnr()	获取通道的缓冲区号
ch_getjob()	获取通道相关的作业
ch_info()	获取通道信息
ch_log()	在通道日志文件写下信息
ch_logfile()	设置通道日志文件
ch_open()	打开通道
ch_read()	从通道读取信息
ch_readraw()	从通道读取未处理的信息
ch_sendexpr()	从通道读取 JSON 信息
ch_sendraw()	向通道发送未处理的信息
ch_setoptions()	设置通道的选项
ch_status()	获取通道的状态
execute()	执行 Ex 命令并获取输出
exepath()	可执行程序的完整路径
funcref()	返回函数 {name} 的引用
getbufinfo()	获取缓冲区信息的列表
getcharsearch()	返回字符搜索信息
getcmdwintype()	返回当前命令行窗口类型
getcompletion()	返回命令行补全匹配的列表
getcurpos()	获取光标位置
gettabinfo()	获取标签页信息的列表
getwininfo()	获取窗口信息的列表
glob2regpat()	转换 glob 模式到搜索模式
isnan()	检查非数
job_getchannel()	获取作业使用的通道
job_info()	获取作业信息
job_setoptions()	设置作业选项
job_start()	启动作业
job_status()	获取作业状态

job_stop()	停止作业
js_decode()	把 JSON 字符串解码为 Vim 类型
js_encode()	把表达式编码为 JSON 字符串
json_decode()	把 JSON 字符串解码为 Vim 类型
json_encode()	把表达式编码为 JSON 字符串
matchaddpos()	定义位置列表用于高亮
matchstrpos()	字符串中满足匹配的模式和位置
perleval()	计算 Perl 表达式
reltimefloat()	转换 reltime() 结果为浮点数
setcharsearch()	设置字符搜索信息
setfperm()	设置文件权限
strcharpart()	用字符索引获取字符串的子串
strgetchar()	用字符索引获取字符串里的字符
systemlist()	得到列表形式的外壳命令结果
test_alloc_fail()	使内存分配失败
test_autochdir()	测试 'autochdir' 功能
test_disable_char_avail()	无预输入的测试 (将要删除)
test_garbagecollect_now()	立即清理内存
test_null_channel()	返回 null 通道
test_null_dict()	返回 null 字典
test_null_job()	返回 null 作业
test_null_list()	返回 null 列表
test_null_partial()	返回 null 偏函数
test_null_string()	返回 null 字符串
test_settime()	设置 Vim 内部使用的时间
timer_info()	获取定时器信息
timer_pause()	暂停或继续定时器
timer_start()	建立定时器
timer_stop()	停止定时器
timer_stopall()	停止所有定时器
uniq()	删除重复邻接项目的备份
win_findbuf()	寻找包含某缓冲区的窗口
win_getid()	获取窗口的窗口 ID
win_gotoid()	给出 ID 转到指定窗口
win_id2tabwin()	给出窗口 ID 获取标签页号和窗口号
win_id2win()	把窗口 ID 转换为窗口号
wordcount()	获取缓冲区的字节/单词/字符计数

新 Vim 变量:

v:beval_winid	鼠标指针所在的窗口的窗口 ID
v:completed_item	用于最近经补全的单词的补全项目
v:errors	assert 函数找到的错误
v:false	值为零的数值
v:hlsearch	指示搜索高亮是否打开
v:mouse_winid	getchar() 获取的鼠标点击所在的窗口 ID
v:none	空字符串，用于 JSON
v:null	空字符串，用于 JSON
v:option_new	选项的新值，用于 OptionSet
v:option_old	选项的旧值，用于 OptionSet
v:option_oldlocal	选项的旧局部值，用于 OptionSet
v:option_oldglobal	选项的旧全局值，用于 OptionSet
v:option_type	set 命令的作用域，用于 OptionSet

v:option_command	选项设置所用的命令，用于 OptionSet
v:progpath	Vim 启动所用的命令
v:t_bool	布尔型的类型值
v:t_channel	通道的类型值
v:t_dict	字典的类型值
v:t_float	浮点数的类型值
v:t_func	函数引用的类型值
v:t_job	作业的类型值
v:t_list	列表的类型值
v:t_none	None 的类型值
v:t_number	数值的类型值
v:t_string	字符串的类型值
v:testing	必须在用 test_garbagecollect_now() 前设置
v:true	值为一的数值
v:vim_did_enter	就在 VimEnter 自动命令激活前设置

新自动命令事件：

CmdUndefined	使用未定义的用户命令
OptionSet	设置任何选项后
TabClosed	关闭标签页后
TabNew	建立新标签页后
TextChangedI	插入模式下改动文本后
TextChanged	普通模式下改动文本后
WinNew	建立新窗口后

新高亮组：

EndOfBuffer	缓冲区末行的填充行 (~)。 hl-EndOfBuffer

搜索模式的新项目：

/\%C	\%C	匹配任何组合字符

新语法/缩进/文件类型插件文件：

AVR Assembler (Avra) 语法
Arduino 语法
Bazel 语法、缩进和文件类型插件
Dockerfile 语法和文件类型插件
Eiffel 文件类型插件
Euphoria 3 和 4 语法
Go 语法、缩进和文件类型插件
Godoc 语法
Groovy 文件类型插件
HGcommit 文件类型插件
Hog 缩进和文件类型插件
Innovation Data Processing upstream.pt 语法
J 语法、缩进和文件类型插件
Jproperties 文件类型插件

Json 语法、缩进和文件类型插件
Kivy 语法
Less 语法和缩进
Mix 语法
Motorola S-Record 语法
R 文件类型插件
ReStructuredText 语法、缩进和文件类型插件
Registry 文件类型插件
Rhelp 缩进和文件类型插件
Rmd (markdown with R code chunks) 语法和缩进
Rmd 文件类型插件
Rnoweb 文件类型插件
Rnoweb 缩进
Scala 语法、缩进和文件类型插件
SystemVerilog 语法、缩进和文件类型插件
Systemd 语法、缩进和文件类型插件
Teraterm (TTL) 语法和缩进
Text 文件类型插件
Vroom 语法、缩进和文件类型插件

新键盘映射表:

东亚美尼亚语和西亚美尼亚语
俄罗斯语 JCUKEN Windows 变种的输入
越南语 telex 和 vni 输入

不 兼 容 的 改 动 incompatible-8

这些改动和以前版本不兼容。如果从 Vim 7.4 升级到 8.0 时出现问题，检查这个列表。

没有 vimrc 时更好的缺省处理

如果找不到 vimrc，截入 defaults.vim 脚本来设置更多对新用户有用的缺省值。这包括设置 'nocompatible'。这样，Vim 就不再以 Vi 兼容模式启动。如果你确实想要，建立 .vimrc 文件并设置 "set compatible"，或以 "vim -C" 启动 Vim。

删除支持

删除了 MS-DOS 的支持。有一段时间不能用了 (Vim 不能放进内存里)，它的删除清理了不少代码。

删除了 Windows 16 位 (Windows 95 和更旧版本) 的支持。

删除了 OS/2 的支持。因为没有人在用，可能有一段时间早就不能用了。

删除了 SNiFF+ 的支持。

少量不兼容:

可能吧...

改 进 improvements-8

已有的 blowfish 加密被发现比以前想象的要弱。为此，加入了 blowfish2 方法进行修正。注意 这还不是顶尖的加密方法，但对多数应用足够了。见 'cryptmethod'。

编 译 时 的 改 变 compile-changes-8

Vim 版本库从 Google code 转到了 gitub，因为 Google code 停止了服务。可在这里找到。 https://github.com/vim/vim。

函数现在使用 ANSI-C 声明。需要至少 C-89 兼容的编译器。

现在总是包含 +visual 特性。

补 丁 patches-8 bug-fixes-8

这里的补丁列表包含 7.4.0 开始的补丁。包含所有的新特性，但不包含 runtime 的文件改动（语法、缩进、帮助等等）

```
Patch 7.4.001
问题:       Character classes such as [a-z] do not react to 'ignorecase'.
            Breaks man page highlighting. (Mario Grgic)
解决方案:   Add separate items for classes that react to 'ignorecase'.  Clean
            up logic handling character classes.  Add more tests.
相关文件:   src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.002
问题:       Pattern with two alternative look-behind matches does not match.
            (Amadeus Demarzi)
解决方案:   When comparing PIMs also compare their state ID to see if they are
            different.
相关文件:   src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.003
问题:       Memory access error in Ruby syntax highlighting. (Christopher Chow)
解决方案:   Refresh stale pointer. (James McCoy)
相关文件:   src/regexp_nfa.c

Patch 7.4.004
问题:       When closing a window fails ":bwipe" may hang.
解决方案:   Let win_close() return FAIL and break out of the loop.
相关文件:   src/window.c, src/proto/window.pro, src/buffer.c

Patch 7.4.005
问题:       Using "vaB" while 'virtualedit' is set selects the wrong area.
            (Dimitar Dimitrov)
```

解决方案: Reset coladd when finding a match.
相关文件: src/search.c

Patch 7.4.006
问题: mkdir("foo/bar/", "p") gives an error message. (David Barnett)
解决方案: Remove the trailing slash. (lcd)
相关文件: src/eval.c

Patch 7.4.007
问题: Creating a preview window on startup leaves the screen layout in a
           messed up state. (Marius Gedminas)
解决方案: Don't change firstwin. (Christian Brabandt)
相关文件: src/main.c

Patch 7.4.008
问题: New regexp engine can't be interrupted.
解决方案: Check for CTRL-C pressed. (Yasuhiro Matsumoto)
相关文件: src/regexp_nfa.c, src/regexp.c

Patch 7.4.009
问题: When a file was not decrypted (yet), writing it may destroy the
           contents.
解决方案: Mark the file as readonly until decryption was done. (Christian
           Brabandt)
相关文件: src/fileio.c

Patch 7.4.010 (after 7.4.006)
问题: Crash with invalid argument to mkdir().
解决方案: Check for empty string. (lcd47)
相关文件: src/eval.c

Patch 7.4.011
问题: Cannot find out if "acl" and "xpm" features are supported.
解决方案: Add "acl" and "xpm" to the list of features. (Ken Takata)
相关文件: src/eval.c, src/version.c

Patch 7.4.012
问题: MS-Windows: resolving shortcut does not work properly with
           multi-byte characters.
解决方案: Use wide system functions. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.013
问题: MS-Windows: File name buffer too small for utf-8.
解决方案: Use character count instead of byte count. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.014
问题: MS-Windows: check for writing to device does not work.
解决方案: Fix #ifdefs. (Ken Takata)
相关文件: src/fileio.c

Patch 7.4.015
问题: MS-Windows: Detecting node type does not work for multi-byte

characters.
解决方案: Use wide character function when needed. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.016
问题: MS-Windows: File name case can be wrong.
解决方案: Add fname_casew(). (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.017
问题: ":help !!" does not find the "!!" tag in the help file. (Ben Fritz)
解决方案: When reading the start of the tags file do parse lines that are not header lines.
相关文件: src/tag.c

Patch 7.4.018
问题: When completing item becomes unselected. (Shougo Matsu)
解决方案: Revert patch 7.3.1269.
相关文件: src/edit.c

Patch 7.4.019
问题: MS-Windows: File name completion doesn't work properly with Chinese characters. (Yue Wu)
解决方案: Take care of multi-byte characters when looking for the start of the file name. (Ken Takata)
相关文件: src/edit.c

Patch 7.4.020
问题: NFA engine matches too much with \@>. (John McGowan)
解决方案: When a whole pattern match is found stop searching.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.021
问题: NFA regexp: Using \ze in one branch which doesn't match may cause end of another branch to be wrong. (William Fugh)
解决方案: Set end position if it wasn't set yet.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.022
问题: Deadlock while exiting, because of allocating memory.
解决方案: Do not use gettext() in deathtrap(). (James McCoy)
相关文件: src/os_unix.c, src/misc1.c

Patch 7.4.023
问题: Compiler warning on 64 bit windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/edit.c

Patch 7.4.024
问题: When root edits a file the undo file is owned by root while the edited file may be owned by another user, which is not allowed. (cac2s)
解决方案: Accept an undo file owned by the current user.

相关文件: src/undo.c

Patch 7.4.025 (after 7.4.019)
问题: Reading before start of a string.
解决方案: Do not call mb_ptr_back() at start of a string. (Dominique Pelle)
相关文件: src/edit.c

Patch 7.4.026
问题: Clang warning for int shift overflow.
解决方案: Use unsigned and cast back to int. (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.4.027 (after 7.4.025)
问题: Another valgrind error when using CTRL-X CTRL-F at the start of the line. (Dominique Pelle)
解决方案: Don't call mb_ptr_back() at the start of the line. Add a test.
相关文件: src/edit.c, src/testdir/test32.in

Patch 7.4.028
问题: Equivalence classes are not working for multi-byte characters.
解决方案: Copy the rules from the old to the new regexp engine. Add a test to check both engines.
相关文件: src/regexp_nfa.c, src/testdir/test44.in, src/testdir/test99.in, src/testdir/test99.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.029
问题: An error in a pattern is reported twice.
解决方案: Remove the retry with the backtracking engine, it won't work.
相关文件: src/regexp.c

Patch 7.4.030
问题: The -mno-cygwin argument is no longer supported by Cygwin.
解决方案: Remove the arguments. (Steve Hall)
相关文件: src/GvimExt/Make_cyg.mak, src/Make_cyg.mak, src/xxd/Make_cyg.mak

Patch 7.4.031
问题: ":diffoff!" resets options even when 'diff' is not set. (Charles Cooper)
解决方案: Only resets related options in a window where 'diff' is set.
相关文件: src/diff.c

Patch 7.4.032
问题: NFA engine does not match the NUL character. (Jonathon Merz)
解决方案: Use 0x0a instead of NUL. (Christian Brabandt)
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.033
问题: When the terminal has only 20 lines test 92 and 93 overwrite the input file.
解决方案: Explicitly write test.out. Check that the terminal is large enough to run the tests. (Hirohito Higashi)

相关文件: src/testdir/test92.in, src/testdir/test93.in,
           src/testdir/test1.in, src/testdir/Makefile

Patch 7.4.034
问题: Using "p" in Visual block mode only changes the first line.
解决方案: Repeat the put in all text in the block. (Christian Brabandt)
相关文件: runtime/doc/change.txt, src/ops.c, src/normal.c,
           src/testdir/test20.in, src/testdir/test20.ok

Patch 7.4.035
问题: MS-Windows: The mouse pointer flickers when going from command
           line mode to Normal mode.
解决方案: Check for WM_NCMOUSEMOVE. (Ken Takata)
相关文件: src/gui_w48.c

Patch 7.4.036
问题: NFA engine does not capture group correctly when using \@>. (ZyX)
解决方案: Copy submatches before doing the recursive match.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.037
问题: Using "\ze" in a sub-pattern does not result in the end of the
           match to be set. (Axel Bender)
解决方案: Copy the end of match position when a recursive match was
           successful.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.038
问题: Using "zw" and "zg" when 'spell' is off give a confusing error
           message. (Gary Johnson)
解决方案: Ignore the error when locating the word. Explicitly mention what
           word was added. (Christian Brabandt)
相关文件: src/normal.c, src/spell.c

Patch 7.4.039
问题: MS-Windows: MSVC10 and earlier can't handle symlinks to a
           directory properly.
解决方案: Add stat_symlink_aware() and wstat_symlink_aware(). (Ken Takata)
相关文件: src/os_mswin.c, src/os_win32.c, src/os_win32.h

Patch 7.4.040
问题: Valgrind error on exit when a script-local variable holds a
           reference to the scope of another script.
解决方案: First clear all variables, then free the scopes. (ZyX)
相关文件: src/eval.c

Patch 7.4.041 (after 7.4.034)
问题: Visual selection does not remain after being copied over. (Axel
           Bender)
解决方案: Move when VIsual_active is reset. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.4.042
问题: When using ":setlocal" for 'spell' and 'spelllang' then :spelldump

doesn't work. (Dimitar Dimitrov)
解决方案: Copy the option variables to the new window used to show the dump. (Christian Brabandt)
相关文件: src/spell.c

Patch 7.4.043
问题: VMS can't handle long function names.
解决方案: Shorten may_req_ambiguous_character_width. (Samuel Ferencik)
相关文件: src/main.c, src/term.c, src/proto/term.pro

Patch 7.4.044 (after 7.4.039)
问题: Can't build with old MSVC. (Wang Shoulin)
解决方案: Define OPEN_OH_ARGTYPE instead of using intptr_t directly.
相关文件: src/os_mswin.c

Patch 7.4.045
问题: substitute() does not work properly when the pattern starts with "\ze".
解决方案: Detect an empty match. (Christian Brabandt)
相关文件: src/eval.c, src/testdir/test80.in, src/testdir/test80.ok

Patch 7.4.046
问题: Can't use Tcl 8.6.
解决方案: Change how Tcl_FindExecutable is called. (Jan Nijtmans)
相关文件: src/if_tcl.c

Patch 7.4.047
问题: When using input() in a function invoked by a mapping it doesn't work.
解决方案: Temporarily reset ex_normal_busy. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.048
问题: Recent clang version complains about -fno-strength-reduce.
解决方案: Add a configure check for the clang version. (Kazunobu Kuriyama)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.049
问题: In Ex mode, when line numbers are enabled the substitute prompt is wrong.
解决方案: Adjust for the line number size. (Benoit Pierre)
相关文件: src/ex_cmds.c

Patch 7.4.050
问题: "gn" selects too much for the pattern "\d" when there are two lines with a single digit. (Ryan Carney)
解决方案: Adjust the logic of is_one_char(). (Christian Brabandt)
相关文件: src/search.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.4.051
问题: Syntax highlighting a Yaml file causes a crash. (Blake Preston)
解决方案: Copy the pim structure before calling addstate() to avoid it becoming invalid when the state list is reallocated.

相关文件: src/regexp_nfa.c

Patch 7.4.052
问题: With 'fo' set to "a2" inserting a space in the first column may cause the cursor to jump to the previous line.
解决方案: Handle the case when there is no comment leader properly. (Tor Perkins) Also fix that cursor is in the wrong place when spaces get replaced with a Tab.
相关文件: src/misc1.c, src/ops.c, src/testdir/test68.in, src/testdir/test68.ok

Patch 7.4.053
问题: Test75 has a wrong header. (ZyX)
解决方案: Fix the text and remove leading ".
相关文件: src/testdir/test75.in

Patch 7.4.054
问题: Reading past end of the 'stl' string.
解决方案: Don't increment pointer when already at the NUL. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.4.055
问题: Mac: Where availability macros are defined depends on the system.
解决方案: Add a configure check. (Felix Bünemann)
相关文件: src/config.h.in, src/configure.in, src/auto/configure, src/os_mac.h

Patch 7.4.056
问题: Mac: Compilation problem with OS X 10.9 Mavericks.
解决方案: Include AvailabilityMacros.h when available. (Kazunobu Kuriyama)
相关文件: src/os_unix.c

Patch 7.4.057
问题: byteidx() does not work for composing characters.
解决方案: Add byteidxcomp().
相关文件: src/eval.c, src/testdir/test69.in, src/testdir/test69.ok, runtime/doc/eval.txt

Patch 7.4.058
问题: Warnings on 64 bit Windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/ops.c

Patch 7.4.059
问题: set_last_cursor() may encounter w_buffer being NULL. (Matt Mkaniaris)
解决方案: Check for NULL.
相关文件: src/mark.c

Patch 7.4.060
问题: Declaration has wrong return type for PyObject_SetAttrString().
解决方案: Use int instead of PyObject. (Andreas Schwab)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.4.061 (after 7.4.055 and 7.4.056)
问题: Availability macros configure check in wrong place.
解决方案: Also check when not using Darwin. Remove version check.
相关文件: src/configure.in, src/auto/configure, src/os_unix.c

Patch 7.4.062 (after 7.4.061)
问题: Configure check for AvailabilityMacros.h is wrong.
解决方案: Use AC_CHECK_HEADERS().
相关文件: src/configure.in, src/auto/configure

Patch 7.4.063
问题: Crash when using invalid key in Python dictionary.
解决方案: Check for object to be NULL. Add tests. (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.064
问题: When replacing a character in Visual block mode, entering a CR does not cause a repeated line break.
解决方案: Recognize the situation and repeat the line break. (Christian Brabandt)
相关文件: src/normal.c, src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.065
问题: When recording, the character typed at the hit-enter prompt is recorded twice. (Urtica Dioica)
解决方案: Avoid recording the character twice. (Christian Brabandt)
相关文件: src/message.c

Patch 7.4.066
问题: MS-Windows: When there is a colon in the file name (sub-stream feature) the swap file name is wrong.
解决方案: Change the colon to "%". (Yasuhiro Matsumoto)
相关文件: src/fileio.c, src/memline.c, src/misc1.c, src/proto/misc1.pro

Patch 7.4.067
问题: After inserting comment leader, CTRL-\ CTRL-O does move the cursor. (Wiktor Ruben)
解决方案: Avoid moving the cursor. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.4.068
问题: Cannot build Vim on Mac with non-Apple compilers.
解决方案: Remove the -no-cpp-precomp flag. (Misty De Meo)
相关文件: src/configure.in, src/auto/configure, src/osdef.sh

Patch 7.4.069
问题: Cannot right shift lines starting with #.
解决方案: Allow the right shift when 'cino' contains #N with N > 0. (Christian Brabandt)
Refactor parsing 'cino', store the values in the buffer.
相关文件: runtime/doc/indent.txt, src/buffer.c, src/edit.c, src/eval.c,

src/ex_getln.c, src/fold.c, src/misc1.c, src/ops.c,
src/proto/misc1.pro, src/proto/option.pro, src/structs.h,
src/option.c

Patch 7.4.070 (after 7.4.069)
问题: Can't compile with tiny features. (Tony Mechelynck)
解决方案: Add #ifdef.
相关文件: src/buffer.c

Patch 7.4.071 (after 7.4.069)
问题: Passing limits around too often.
解决方案: Use limits from buffer.
相关文件: src/edit.c, src/misc1.c, src/proto/misc1.pro

Patch 7.4.072
问题: Crash when using Insert mode completion.
解决方案: Avoid going past the end of pum_array. (idea by Francisco Lopes)
相关文件: src/popupmnu.c

Patch 7.4.073
问题: Setting undolevels for one buffer changes undo in another.
解决方案: Make 'undolevels' a global-local option. (Christian Brabandt)
相关文件: runtime/doc/options.txt, src/buffer.c, src/option.c, src/option.h
src/structs.h, src/undo.c

Patch 7.4.074
问题: When undo'ing all changes and creating a new change the undo
structure is incorrect. (Christian Brabandt)
解决方案: When deleting the branch starting at the old header, delete the
whole branch, not just the first entry.
相关文件: src/undo.c

Patch 7.4.075
问题: Locally setting 'undolevels' is not tested.
解决方案: Add a test. (Christian Brabandt)
相关文件: src/testdir/test100.in, src/testdir/test100.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile, src/Makefile

Patch 7.4.076
问题: "cgn" does not wrap around the end of the file. (Dimitar Dimitrov)
解决方案: Restore 'wrapscan' earlier. (Christian Brabandt)
相关文件: src/search.c

Patch 7.4.077
问题: DOS installer creates shortcut without a path, resulting in the
current directory to be C:\Windows\system32.
解决方案: Use environment variables.
相关文件: src/dosinst.c

Patch 7.4.078
问题: MSVC 2013 is not supported.
解决方案: Recognize and support MSVC 2013. (Ed Brown)

相关文件: src/Make_mvc.mak

Patch 7.4.079
问题: A script cannot detect whether 'hlsearch' highlighting is actually
displayed.
解决方案: Add the "v:hlsearch" variable. (ZyX)
相关文件: src/eval.c, src/ex_docmd.c,
src/option.c, src/screen.c, src/search.c, src/tag.c, src/vim.h,
src/testdir/test101.in, src/testdir/test101.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.080 (after 7.4.079)
问题: Missing documentation for v:hlsearch.
解决方案: Include the right file in the patch.
相关文件: runtime/doc/eval.txt

Patch 7.4.081 (after 7.4.078)
问题: Wrong logic when ANALYZE is "yes".
解决方案: Use or instead of and. (KF Leong)
相关文件: src/Make_mvc.mak

Patch 7.4.082
问题: Using "gf" in a changed buffer suggests adding "!", which is not
possible. (Tim Chase)
解决方案: Pass a flag to check_changed() whether adding ! make sense.
相关文件: src/vim.h, src/ex_cmds2.c, src/proto/ex_cmds2.pro, src/globals.h,
src/ex_cmds.c, src/ex_docmd.c

Patch 7.4.083
问题: It's hard to avoid adding a used pattern to the search history.
解决方案: Add the ":keeppatterns" modifier. (Christian Brabandt)
相关文件: runtime/doc/cmdline.txt, src/ex_cmds.h, src/ex_docmd.c,
src/ex_getln.c, src/structs.h

Patch 7.4.084
问题: Python: interrupt not being properly discarded. (Yggdroot Chen)
解决方案: Discard interrupt in VimTryEnd. (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.085
问题: When inserting text in Visual block mode and moving the cursor the
wrong text gets repeated in other lines.
解决方案: Use the '[mark to find the start of the actually inserted text.
(Christian Brabandt)
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.086
问题: Skipping over an expression when not evaluating it does not work
properly for dict members.
解决方案: Skip over unrecognized expression. (ZyX)
相关文件: src/eval.c, src/testdir/test34.in, src/testdir/test34.ok

Patch 7.4.087
问题: Compiler warning on 64 bit Windows systems.
解决方案: Fix type cast. (Mike Williams)
相关文件: src/ops.c

Patch 7.4.088
问题: When spell checking is enabled Asian characters are always marked as error.
解决方案: When 'spelllang' contains "cjk" do not mark Asian characters as error. (Ken Takata)
相关文件: runtime/doc/options.txt, runtime/doc/spell.txt, src/mbyte.c, src/option.c, src/spell.c, src/structs.h

Patch 7.4.089
问题: When editing a file in a directory mounted through sshfs Vim doesn't set the security context on a renamed file.
解决方案: Add mch_copy_sec() to vim_rename(). (Peter Backes)
相关文件: src/fileio.c

Patch 7.4.090
问题: Win32: When a directory name contains an exclamation mark, completion doesn't complete the contents of the directory.
解决方案: Escape the exclamation mark. (Jan Stocker)
相关文件: src/ex_getln.c, src/testdir/test102.in, src/testdir/test102.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.091 (after 7.4.089)
问题: Missing semicolon.
解决方案: Add the semicolon.
相关文件: src/fileio.c

Patch 7.4.092 (after 7.4.088)
问题: Can't build small version.
解决方案: Add #ifdef where the b_cjk flag is used. (Ken Takata)
相关文件: src/spell.c

Patch 7.4.093
问题: Configure can't use LuaJIT on ubuntu 12.04.
解决方案: Adjust the configure regexp that locates the version number. (Charles Strahan)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.094
问题: Configure may not find that -lint is needed for gettext().
解决方案: Check for gettext() with empty $LIBS. (Thomas De Schampheleire)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.095 (after 7.4.093)
问题: Regexp for LuaJIT version doesn't work on BSD.
解决方案: Use "*" instead of "\+" and "\?". (Ozaki Kiichi)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.096
问题: Can't change directory to an UNC path.
解决方案: Use win32_getattrs() in mch_getperm(). (Christian Brabandt)
相关文件: src/os_win32.c

Patch 7.4.097 (after 7.4.034)
问题: Unexpected behavior change related to 'virtualedit'. (Ingo Karkat)
解决方案: Update the valid cursor position. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.4.098
问题: When using ":'<,'>del" errors may be given for the visual line numbers being out of range.
解决方案: Reset Visual mode in ":del". (Lech Lorens)
相关文件: src/ex_docmd.c, src/testdir/test103.in, src/testdir/test103.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.099
问题: Append in blockwise Visual mode with "$" is wrong.
解决方案: After "$" don't use the code that checks if the cursor was moved. (Hirohito Higashi, Ken Takata)
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.100
问题: NFA regexp doesn't handle backreference correctly. (Ryuichi Hayashida, Urtica Dioica)
解决方案: Always add NFA_SKIP, also when it already exists at the start position.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.101
问题: Using \1 in pattern goes one line too far. (Bohr Shaw, John Little)
解决方案: Only advance the match end for the matched characters in the last line.
相关文件: src/regexp.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.102
问题: Crash when interrupting "z=".
解决方案: Add safety check for word length. (Christian Brabandt, Dominique Pelle)
相关文件: src/spell.c

Patch 7.4.103
问题: Dos installer uses an old way to escape spaces in the diff command.
解决方案: Adjust the quoting to the new default shellxquote. (Ben Fritz)
相关文件: src/dosinst.c

Patch 7.4.104
问题: ":help s/_" reports an internal error. (John Beckett)
解决方案: Check for NUL and invalid character classes.

相关文件: src/regexp_nfa.c

Patch 7.4.105
问题: Completing a tag pattern may give an error for invalid pattern.
解决方案: Suppress the error, just return no matches.
相关文件: src/tag.c

Patch 7.4.106
问题: Can't build with Ruby using Cygwin.
解决方案: Fix library name in makefile. (Steve Hall)
相关文件: src/Make_cyg.mak

Patch 7.4.107
问题: Python: When vim.eval() encounters a Vim error, a try/catch in the
Python code doesn't catch it. (Yggdroot Chen)
解决方案: Throw exceptions on errors in vim.eval(). (ZyX)
相关文件: src/ex_eval.c, src/if_py_both.h, src/proto/ex_eval.pro,
src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.108
问题: "zG" and "zW" leave temp files around on MS-Windows.
解决方案: Delete the temp files when exiting. (Ken Takata)
相关文件: src/memline.c, src/proto/spell.pro, src/spell.c

Patch 7.4.109
问题: ColorScheme autocommand matches with the current buffer name.
解决方案: Match with the colorscheme name. (Christian Brabandt)
相关文件: runtime/doc/autocmd.txt, src/fileio.c, src/syntax.c

Patch 7.4.110
问题: "gUgn" cannot be repeated. (Dimitar Dimitrov)
解决方案: Don't put "gn" in a different order in the redo buffer. Restore
'wrapscan' when the pattern isn't found. (Christian Wellenbrock)
相关文件: src/normal.c, src/search.c, src/test53.in, src/test53.ok

Patch 7.4.111
问题: Memory leak in Python OptionsAssItem. (Ken Takata)
解决方案: Call Py_XDECREF() where needed. (ZyX)
相关文件: src/if_py_both.h

Patch 7.4.112
问题: The defaults for 'directory' and 'backupdir' on MS-Windows do not
include a directory that exists.
解决方案: Use $TEMP.
相关文件: src/os_dos.h

Patch 7.4.113
问题: MSVC static analysis gives warnings.
解决方案: Avoid the warnings and avoid possible bugs. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.114
问题: New GNU make outputs messages about changing directory in another

format.
解决方案: Recognize the new format.
相关文件: src/option.h

Patch 7.4.115
问题: When using Zsh expanding ~abc doesn't work when the result contains a space.
解决方案: Off-by-one error in detecting the NUL. (Pavol Juhas)
相关文件: src/os_unix.c

Patch 7.4.116
问题: When a mapping starts with a space, the typed space does not show up for 'showcmd'.
解决方案: Show "<20>". (Brook Hong)
相关文件: src/normal.c

Patch 7.4.117
问题: Can't build with Cygwin/MingW and Perl 5.18.
解决方案: Add a linker argument for the Perl library. (Cesar Romani)
Adjust CFLAGS and LIB. (Cesar Romani)
Move including inline.h further down. (Ken Takata)
相关文件: src/Make_cyg.mak, src/Make_ming.mak, src/if_perl.xs

Patch 7.4.118
问题: It's possible that redrawing the status lines causes win_redr_custom() to be called recursively.
解决方案: Protect against recursiveness. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 7.4.119
问题: Vim doesn't work well on OpenVMS.
解决方案: Fix various problems. (Samuel Ferencik)
相关文件: src/os_unix.c, src/os_unix.h, src/os_vms.c

Patch 7.4.120 (after 7.4.117)
问题: Can't build with Perl 5.18 on Linux. (Lcd 47)
解决方案: Add #ifdef. (Ken Takata)
相关文件: src/if_perl.xs

Patch 7.4.121
问题: Completion doesn't work for ":py3d" and ":py3f". (Bohr Shaw)
解决方案: Skip over letters after ":py3".
相关文件: src/ex_docmd.c

Patch 7.4.122
问题: Win32: When 'encoding' is set to "utf-8" and the active codepage is cp932 then ":grep" and other commands don't work for multi-byte characters.
解决方案: (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.4.123
问题: Win32: Getting user name does not use wide function.
解决方案: Use GetUserNameW() if possible. (Ken Takata)

相关文件: src/os_win32.c

Patch 7.4.124
问题: Win32: Getting host name does not use wide function.
解决方案: Use GetComputerNameW() if possible. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.125
问题: Win32: Dealing with messages may not work for multi-byte chars.
解决方案: Use pDispatchMessage(). (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.126
问题: Compiler warnings for "const" and incompatible types.
解决方案: Remove "const", add type cast. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.127
问题: Perl 5.18 on Unix doesn't work.
解决方案: Move workaround to after including vim.h. (Ken Takata)
相关文件: src/if_perl.xs

Patch 7.4.128
问题: Perl 5.18 for MSVC doesn't work.
解决方案: Add check in makefile and define __inline. (Ken Takata)
相关文件: src/Make_mvc.mak, src/if_perl.xs

Patch 7.4.129
问题: getline(-1) returns zero. (mvxxc)
解决方案: Return an empty string.
相关文件: src/eval.c

Patch 7.4.130
问题: Relative line numbers mix up windows when using folds.
解决方案: Use hasFoldingWin() instead of hasFolding(). (Lech Lorens)
相关文件: src/misc2.c

Patch 7.4.131
问题: Syncbind causes E315 errors in some situations. (Liang Li)
解决方案: Set and restore curbuf in ex_syncbind(). (Christian Brabandt)
相关文件: src/ex_docmd.c, src/testdir/test37.ok

Patch 7.4.132 (after 7.4.122)
问题: Win32: flags and inherit_handles arguments mixed up.
解决方案: Swap the argument. (cs86661)
相关文件: src/os_win32.c

Patch 7.4.133
问题: Clang warns for using NUL.
解决方案: Change NUL to NULL. (Dominique Pelle)
相关文件: src/eval.c, src/misc2.c

Patch 7.4.134
问题: Spurious space in MingW Makefile.

解决方案: Remove the space. (Michael Soyka)
相关文件: src/Make_ming.mak

Patch 7.4.135
问题: Missing dot in MingW test Makefile.
解决方案: Add the dot. (Michael Soyka)
相关文件: src/testdir/Make_ming.mak

Patch 7.4.136 (after 7.4.096)
问题: MS-Windows: When saving a file with a UNC path the file becomes
           read-only.
解决方案: Don't mix up Win32 attributes and Unix attributes. (Ken Takata)
相关文件: src/os_mswin.c, src/os_win32.c

Patch 7.4.137
问题: Cannot use IME with Windows 8 console.
解决方案: Change the user of ReadConsoleInput() and PeekConsoleInput().
           (Nobuhiro Takasaki)
相关文件: src/os_win32.c

Patch 7.4.138 (after 7.4.114)
问题: Directory change messages are not recognized.
解决方案: Fix using a character range literally. (Lech Lorens)
相关文件: src/option.h

Patch 7.4.139
问题: Crash when using :cd in autocommand. (Franois Ingelrest)
解决方案: Set w_localdir to NULL after freeing it. (Dominique Pelle)
相关文件: src/ex_docmd.c, src/window.c

Patch 7.4.140
问题: Crash when wiping out buffer triggers autocommand that wipes out
           only other buffer.
解决方案: Do not delete the last buffer, make it empty. (Hirohito Higashi)
相关文件: src/buffer.c

Patch 7.4.141
问题: Problems when building with Borland: st_mode is signed short;
           can't build with Python; temp files not ignored by Mercurial;
           building with DEBUG doesn't define _DEBUG.
解决方案: Fix the problems. (Ken Takata)
相关文件: src/Make_bc5.mak, src/if_py_both.h, src/os_win32.c

Patch 7.4.142 (after 7.4.137)
问题: On MS-Windows 8 IME input doesn't work correctly.
解决方案: Work around the problem. (Nobuhiro Takasaki)
相关文件: src/os_win32.c

Patch 7.4.143
问题: TextChangedI is not triggered.
解决方案: Reverse check for "ready". (lilydjwg)
相关文件: src/edit.c

Patch 7.4.144

问题: MingW also supports intptr_t for OPEN_OH_ARGTYPE.
解决方案: Adjust #ifdef. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.145
问题: getregtype() does not return zero for unknown register.
解决方案: Adjust documentation: return empty string for unknown register.
            Check the register name to be valid. (Yukihiro Nakadaira)
相关文件: runtime/doc/eval.txt, src/ops.c

Patch 7.4.146
问题: When starting Vim with "-u NONE" v:oldfiles is NULL.
解决方案: Set v:oldfiles to an empty list. (Yasuhiro Matsumoto)
相关文件: src/main.c

Patch 7.4.147
问题: Cursor moves to wrong position when using "gj" after "$" and
            virtual editing is active.
解决方案: Make "gj" behave differently when virtual editing is active.
            (Hirohito Higashi)
相关文件: src/normal.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.148
问题: Cannot build with Cygwin and X11.
解决方案: Include Xwindows.h instead of windows.h. (Lech Lorens)
相关文件: src/mbyte.c

Patch 7.4.149
问题: Get E685 error when assigning a function to an autoload variable.
            (Yukihiro Nakadaira)
解决方案: Instead of having a global no_autoload variable, pass an autoload
            flag down to where it is used. (ZyX)
相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok,
            src/testdir/test60.in, src/testdir/test60.ok,
            src/testdir/sautest/autoload/footest.vim

Patch 7.4.150
问题: :keeppatterns is not respected for :s.
解决方案: Check the keeppatterns flag. (Yasuhiro Matsumoto)
相关文件: src/search.c, src/testdir/test14.in, src/testdir/test14.ok

Patch 7.4.151
问题: Python: slices with steps are not supported.
解决方案: Support slices in Python vim.List. (ZyX)
相关文件: src/eval.c, src/if_py_both.h, src/if_python3.c, src/if_python.c,
            src/proto/eval.pro, src/testdir/test86.in, src/testdir/test86.ok,
            src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.152
问题: Python: Cannot iterate over options.
解决方案: Add options iterator. (ZyX)
相关文件: src/if_py_both.h, src/option.c, src/proto/option.pro,
            src/testdir/test86.in, src/testdir/test86.ok,
            src/testdir/test87.in, src/testdir/test87.ok, src/vim.h

Patch 7.4.153
问题:　　Compiler warning for pointer type.
解决方案:　Add type cast.
相关文件:　src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.4.154 (after 7.4.149)
问题:　　Still a problem with auto-loading.
解决方案:　Pass no_autoload to deref_func_name(). (Yukihiro Nakadaira)
相关文件:　src/eval.c

Patch 7.4.155
问题:　　":keeppatterns /pat" does not keep search pattern offset.
解决方案:　Restore the offset after doing the search.
相关文件:　src/search.c, src/testdir/test14.in, src/testdir/test14.ok

Patch 7.4.156
问题:　　Test file missing from distribution.
解决方案:　Add new directory to file list.
相关文件:　Filelist

Patch 7.4.157
问题:　　Error number used twice. (Yukihiro Nakadaira)
解决方案:　Change the one not referred in the docs.
相关文件:　src/undo.c

Patch 7.4.158 (after 7.4.045)
问题:　　Pattern containing \zs is not handled correctly by substitute().
解决方案:　Change how an empty match is skipped. (Yukihiro Nakadaira)
相关文件:　src/eval.c, src/testdir/test80.in, src/testdir/test80.ok

Patch 7.4.159
问题:　　Completion hangs when scanning the current buffer after doing
　　keywords. (Christian Brabandt)
解决方案:　Set the first match position when starting to scan the current
　　buffer.
相关文件:　src/edit.c

Patch 7.4.160
问题:　　Win32: Crash when executing external command.
解决方案:　Only close the handle when it was created. (Yasuhiro Matsumoto)
相关文件:　src/os_win32.c

Patch 7.4.161
问题:　　Crash in Python exception handling.
解决方案:　Only use exception variables if did_throw is set. (ZyX)
相关文件:　srcc/if_py_both.h

Patch 7.4.162
问题:　　Running tests in shadow dir doesn't work.
解决方案:　Add testdir/sautest to the shadow target. (James McCoy)
相关文件:　src/Makefile

Patch 7.4.163 (after 7.4.142)

问题:　　MS-Windows input doesn't work properly on Windows 7 and earlier.
解决方案:　Add a check for Windows 8. (Yasuhiro Matsumoto)
相关文件:　src/os_win32.c

Patch 7.4.164 (after 7.4.163)
问题:　　Problem with event handling on Windows 8.
解决方案:　Ignore duplicate WINDOW_BUFFER_SIZE_EVENTs. (Nobuhiro Takasaki)
相关文件:　src/os_win32.c

Patch 7.4.165
问题:　　By default, after closing a buffer changes can't be undone.
解决方案:　In the example vimrc file set 'undofile'.
相关文件:　runtime/vimrc_example.vim

Patch 7.4.166
问题:　　Auto-loading a function for code that won't be executed.
解决方案:　Do not auto-load when evaluation is off. (Yasuhiro Matsumoto)
相关文件:　src/eval.c

Patch 7.4.167 (after 7.4.149)
问题:　　Fixes are not tested.
解决方案:　Add a test for not autoloading on assignment. (Yukihiro Nakadaira)
相关文件:　src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
　　src/testdir/Make_vms.mms, src/testdir/Makefile,
　　src/testdir/sautest/autoload/Test104.vim, src/testdir/test104.in,
　　src/testdir/test104.ok

Patch 7.4.168
问题:　　Can't compile with Ruby 2.1.0.
解决方案:　Add support for new GC. (Kohei Suzuki)
相关文件:　src/if_ruby.c

Patch 7.4.169
问题:　　":sleep" puts cursor in the wrong column. (Liang Li)
解决方案:　Add the window offset. (Christian Brabandt)
相关文件:　src/ex_docmd.c

Patch 7.4.170
问题:　　Some help tags don't work with ":help". (Tim Chase)
解决方案:　Add exceptions.
相关文件:　src/ex_cmds.c

Patch 7.4.171
问题:　　Redo does not set v:count and v:count1.
解决方案:　Use a separate buffer for redo, so that we can set the counts when
　　performing redo.
相关文件:　src/getchar.c, src/globals.h, src/normal.c, src/proto/getchar.pro,
　　src/structs.h

Patch 7.4.172
问题:　　The blowfish code mentions output feedback, but the code is
　　actually doing cipher feedback.
解决方案:　Adjust names and comments.

相关文件: src/blowfish.c, src/fileio.c, src/proto/blowfish.pro,
           src/memline.c

Patch 7.4.173
问题: When using scrollbind the cursor can end up below the last line.
           (mvxxc)
解决方案: Reset w_botfill when scrolling up. (Christian Brabandt)
相关文件: src/move.c

Patch 7.4.174
问题: Compiler warnings for Python interface. (Tony Mechelynck)
解决方案: Add type casts, initialize variable.
相关文件: src/if_py_both.h

Patch 7.4.175
问题: When a wide library function fails, falling back to the non-wide
           function may do the wrong thing.
解决方案: Check the platform, when the wide function is supported don't fall
           back to the non-wide function. (Ken Takata)
相关文件: src/os_mswin.c, src/os_win32.c

Patch 7.4.176
问题: Dictionary.update() throws an error when used without arguments.
           Python programmers don't expect that.
解决方案: Make Dictionary.update() without arguments do nothing. (ZyX)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test87.in

Patch 7.4.177
问题: Compiler warning for unused variable. (Tony Mechelynck)
解决方案: Add #ifdef.
相关文件: src/move.c

Patch 7.4.178
问题: The J command does not update '[and '] marks. (William Gardner)
解决方案: Set the marks. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.4.179
问题: Warning for type-punned pointer. (Tony Mechelynck)
解决方案: Use intermediate variable.
相关文件: src/if_py_both.h

Patch 7.4.180 (after 7.4.174)
问题: Older Python versions don't support %ld.
解决方案: Use %d instead. (ZyX)
相关文件: src/if_py_both.h

Patch 7.4.181
问题: When using 'pastetoggle' the status lines are not updated. (Samuel
           Ferencik, Jan Christoph Ebersbach)
解决方案: Update the status lines. (Nobuhiro Takasaki)
相关文件: src/getchar.c

Patch 7.4.182

问题: Building with mzscheme and racket does not work. (David Chimay)
解决方案: Adjust autoconf. (Sergey Khorev)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.183
问题: MSVC Visual Studio update not supported.
解决方案: Add version number. (Mike Williams)
相关文件: src/Make_mvc.mak

Patch 7.4.184
问题: match() does not work properly with a {count} argument.
解决方案: Compute the length once and update it. Quit the loop when at the
            end. (Hirohito Higashi)
相关文件: src/eval.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.4.185
问题: Clang gives warnings.
解决方案: Adjust how bigness is set. (Dominique Pelle)
相关文件: src/ex_cmds.c

Patch 7.4.186 (after 7.4.085)
问题: Insert in Visual mode sometimes gives incorrect results.
            (Dominique Pelle)
解决方案: Remember the original insert start position. (Christian Brabandt,
            Dominique Pelle)
相关文件: src/edit.c, src/globals.h, src/ops.c, src/structs.h

Patch 7.4.187
问题: Delete that crosses line break splits multi-byte character.
解决方案: Advance a character instead of a byte. (Cade Foster)
相关文件: src/normal.c, src/testdir/test69.in, src/testdir/test69.ok

Patch 7.4.188
问题: SIZEOF_LONG clashes with similar defines in header files.
解决方案: Rename to a name starting with VIM_. Also for SIZEOF_INT.
相关文件: src/if_ruby.c, src/vim.h, src/configure.in, src/auto/configure,
            src/config.h.in, src/fileio.c, src/if_python.c, src/message.c,
            src/spell.c, src/feature.h, src/os_os2_cfg.h, src/os_vms_conf.h,
            src/os_win16.h, src/structs.h

Patch 7.4.189
问题: Compiler warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/eval.c

Patch 7.4.190
问题: Compiler warning for using %lld for off_t.
解决方案: Add type cast.
相关文件: src/fileio.c

Patch 7.4.191
问题: Escaping a file name for shell commands can't be done without a
            function.
解决方案: Add the :S file name modifier.

相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
           src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
           src/testdir/Make_vms.mms, src/testdir/Makefile,
           src/testdir/test105.in, src/testdir/test105.ok,
           runtime/doc/cmdline.txt, runtime/doc/eval.txt,
           runtime/doc/map.txt, runtime/doc/options.txt,
           runtime/doc/quickfix.txt, runtime/doc/usr_30.txt,
           runtime/doc/usr_40.txt, runtime/doc/usr_42.txt,
           runtime/doc/vi_diff.txt, src/eval.c, src/misc2.c, src/normal.c,
           src/proto/misc2.pro

Patch 7.4.192
问题: Memory leak when giving E853.
解决方案: Free the argument. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.4.193
问题: Typos in messages.
解决方案: "then" -> "than". (Dominique Pelle)
相关文件: src/if_py_both.h, src/spell.c

Patch 7.4.194
问题: Can't build for Android.
解决方案: Add #if condition. (Fredrik Fornwall)
相关文件: src/mbyte.c

Patch 7.4.195 (after 7.4.193)
问题: Python tests fail.
解决方案: Change "then" to "than" in more places. (Dominique Pelle, Taro
           Muraoka)
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
           src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.196
问题: Tests fail on Solaris 9 and 10.
解决方案: Use "test -f" instead of "test -e". (Laurent Blume)
相关文件: src/testdir/Makefile

Patch 7.4.197
问题: Various problems on VMS.
解决方案: Fix several VMS problems. (Zoltan Arpadffy)
相关文件: runtime/doc/os_vms.txt, src/Make_vms.mms, src/fileio.c,
           src/os_unix.c, src/os_unix.h, src/os_vms.c, src/os_vms_conf.h,
           src/proto/os_vms.pro, src/testdir/Make_vms.mms,
           src/testdir/test72.in, src/testdir/test77a.com,
           src/testdir/test77a.in, src/testdir/test77a.ok src/undo.c

Patch 7.4.198
问题: Can't build Vim with Perl when -Dusethreads is not specified for
           building Perl, and building Vim with --enable-perlinterp=dynamic.
解决方案: Adjust #ifdefs. (Yasuhiro Matsumoto)
相关文件: src/if_perl.xs

Patch 7.4.199

问题: (issue 197)]P doesn't paste over Visual selection.
解决方案: Handle Visual mode specifically. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.4.200
问题: Too many #ifdefs in the code.
解决方案: Enable FEAT_VISUAL always, await any complaints
相关文件: src/feature.h

Patch 7.4.201
问题: 'lispwords' is a global option.
解决方案: Make 'lispwords' global-local. (Sung Pae)
相关文件: runtime/doc/options.txt, runtime/optwin.vim, src/buffer.c,
src/misc1.c, src/option.c, src/option.h, src/structs.h,
src/testdir/test100.in, src/testdir/test100.ok

Patch 7.4.202
问题: MS-Windows: non-ASCII font names don't work.
解决方案: Convert between the current code page and 'encoding'. (Ken Takata)
相关文件: src/gui_w48.c, src/os_mswin.c, src/proto/winclip.pro,
src/winclip.c

Patch 7.4.203
问题: Parsing 'errorformat' is not correct.
解决方案: Reset "multiignore" at the start of a multi-line message. (Lcd)
相关文件: src/quickfix.c, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test106.in,
src/testdir/test106.ok

Patch 7.4.204
问题: A mapping where the second byte is 0x80 doesn't work.
解决方案: Unescape before checking for incomplete multi-byte char. (Nobuhiro
Takasaki)
相关文件: src/getchar.c, src/testdir/test75.in, src/testdir/test75.ok

Patch 7.4.205
问题: ":mksession" writes command to move to second argument while it
does not exist. When it does exist the order might be wrong.
解决方案: Use ":argadd" for each argument instead of using ":args" with a
list of names. (Nobuhiro Takasaki)
相关文件: src/ex_docmd.c

Patch 7.4.206
问题: Compiler warnings on 64 bit Windows.
解决方案: Add type casts. (Mike Williams)
相关文件: src/gui_w48.c, src/os_mswin.c

Patch 7.4.207
问题: The cursor report sequence is sometimes not recognized and results
in entering replace mode.
解决方案: Also check for the cursor report when not asked for.
相关文件: src/term.c

Patch 7.4.208
问题: Mercurial picks up some files that are not distributed.
解决方案: Add patterns to the ignore list. (Cade Forester)
相关文件: .hgignore

Patch 7.4.209
问题: When repeating a filter command "%" and "#" are expanded.
解决方案: Escape the command when storing for redo. (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.4.210
问题: Visual block mode plus virtual edit doesn't work well with tabs.
(Liang Li)
解决方案: Take coladd into account. (Christian Brabandt)
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.211
问题: ":lu" is an abbreviation for ":lua", but it should be ":lunmap".
(ZyX)
解决方案: Move "lunmap" to above "lua".
相关文件: src/ex_cmds.h

Patch 7.4.212 (after 7.4.200)
问题: Now that the +visual feature is always enabled the #ifdefs for it
are not useful.
解决方案: Remove the checks for FEAT_VISUAL.
相关文件: src/buffer.c, src/charset.c, src/edit.c, src/eval.c,
src/ex_cmds.c, src/ex_docmd.c, src/fold.c, src/getchar.c,
src/gui.c, src/gui_mac.c, src/gui_w48.c, src/main.c, src/mark.c,
src/menu.c, src/misc2.c, src/move.c, src/netbeans.c, src/normal.c,
src/ops.c, src/option.c, src/os_msdos.c, src/os_qnx.c,
src/quickfix.c, src/regexp.c, src/regexp_nfa.c, src/screen.c,
src/search.c, src/spell.c, src/syntax.c, src/term.c, src/ui.c,
src/undo.c, src/version.c, src/window.c, src/feature.h,
src/globals.h, src/option.h, src/os_win32.h, src/structs.h

Patch 7.4.213
问题: It's not possible to open a new buffer without creating a swap
file.
解决方案: Add the ":noswapfile" modifier. (Christian Brabandt)
相关文件: runtime/doc/recover.txt, src/ex_cmds.h, src/ex_docmd.c,
src/memline.c, src/structs.h

Patch 7.4.214
问题: Compilation problems on HP_nonStop (Tandem).
解决方案: Add #defines. (Joachim Schmitz)
相关文件: src/vim.h

Patch 7.4.215
问题: Inconsistency: ":sp foo" does not reload "foo", unless "foo" is
the current buffer. (Liang Li)
解决方案: Do not reload the current buffer on a split command.
相关文件: runtime/doc/windows.txt, src/ex_docmd.c

Patch 7.4.216
问题:　　Compiler warnings. (Tony Mechelynck)
解决方案:　Initialize variables, add #ifdef.
相关文件:　src/term.c, src/os_unix.h

Patch 7.4.217
问题:　　When src/auto/configure was updated, "make clean" would run
　　configure pointlessly.
解决方案:　Do not run configure for "make clean" and "make distclean" when
　　the make program supports $MAKECMDGOALS. (Ken Takata)
相关文件:　src/Makefile

Patch 7.4.218
问题:　　It's not easy to remove duplicates from a list.
解决方案:　Add the uniq() function. (Lcd)
相关文件:　runtime/doc/change.txt, runtime/doc/eval.txt,
　　runtime/doc/usr_41.txt, runtime/doc/version7.txt, src/eval.c,
　　src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.219
问题:　　When 'relativenumber' or 'cursorline' are set the window is
　　redrawn much to often. (Patrick Hemmer, Dominique Pelle)
解决方案:　Check the VALID_CROW flag instead of VALID_WROW.
相关文件:　src/move.c

Patch 7.4.220
问题:　　Test 105 does not work in a shadow dir. (James McCoy)
解决方案:　Omit "src/" from the checked path.
相关文件:　src/testdir/test105.in, src/testdir/test105.ok

Patch 7.4.221
问题:　　Quickfix doesn't resize on ":copen 20". (issue 199)
解决方案:　Resize the window when requested. (Christian Brabandt)
相关文件:　src/quickfix.c

Patch 7.4.222
问题:　　The Ruby directory is constructed from parts.
解决方案:　Use 'rubyarchhdrdir' if it exists. (James McCoy)
相关文件:　src/configure.in, src/auto/configure

Patch 7.4.223
问题:　　Still using an older autoconf version.
解决方案:　Switch to autoconf 2.69.
相关文件:　src/Makefile, src/configure.in, src/auto/configure

Patch 7.4.224
问题:　　/usr/bin/grep on Solaris does not support -F.
解决方案:　Add configure check to find a good grep. (Danek Duvall)
相关文件:　src/configure.in, src/auto/configure

Patch 7.4.225
问题:　　Dynamic Ruby doesn't work on Solaris.
解决方案:　Always use the stubs. (Danek Duvall, Yukihiro Nakadaira)

相关文件: src/if_ruby.c

Patch 7.4.226 (after 7.4.219)
问题: Cursorline highlighting not redrawn when scrolling. (John Marriott)
解决方案: Check for required redraw in two places.
相关文件: src/move.c

Patch 7.4.227 (after 7.4.225)
问题: Can't build with Ruby 1.8.
解决方案: Do include a check for the Ruby version. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.4.228
问题: Compiler warnings when building with Python 3.2.
解决方案: Make type cast depend on Python version. (Ken Takata)
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.4.229
问题: Using ":let" for listing variables and the second one is a curly braces expression may fail.
解决方案: Check for an "=" in a better way. (ZyX)
相关文件: src/eval.c, src/testdir/test104.in, src/testdir/test104.ok

Patch 7.4.230
问题: Error when using ":options".
解决方案: Fix the entry for 'lispwords'. (Kenichi Ito)
相关文件: runtime/optwin.vim

Patch 7.4.231
问题: An error in ":options" is not caught by the tests.
解决方案: Add a test for ":options". Set $VIMRUNTIME for the tests so that it uses the current runtime files instead of the installed ones.
相关文件: src/Makefile, src/testdir/Makefile, src/testdir/test_options.in, src/testdir/test_options.ok, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.4.232
问题: ":%s/\n//" uses a lot of memory. (Aidan Marlin)
解决方案: Turn this into a join command. (Christian Brabandt)
相关文件: src/ex_cmds.c, src/ex_docmd.c, src/proto/ex_docmd.pro

Patch 7.4.233
问题: Escaping special characters for using "%" with a shell command is inconsistent, parentheses are escaped but spaces are not.
解决方案: Only escape "!". (Gary Johnson)
相关文件: src/ex_docmd.c

Patch 7.4.234
问题: Can't get the command that was used to start Vim.
解决方案: Add v:progpath. (Viktor Kojouharov)
相关文件: runtime/doc/eval.txt, src/eval.c, src/main.c, src/vim.h

Patch 7.4.235
问题:　　It is not easy to get the full path of a command.
解决方案:　Add the exepath() function.
相关文件:　src/eval.c, src/misc1.c, src/os_amiga.c, src/os_msdos.c,
　　src/os_unix.c, src/os_vms.c, src/os_win32.c,
　　src/proto/os_amiga.pro, src/proto/os_msdos.pro,
　　src/proto/os_unix.pro, src/proto/os_win32.pro,
　　runtime/doc/eval.txt

Patch 7.4.236
问题:　　It's not that easy to check the Vim patch version.
解决方案:　Make has("patch-7.4.123") work. (partly by Marc Weber)
相关文件:　runtime/doc/eval.txt, src/eval.c, src/testdir/test60.in,
　　src/testdir/test60.ok

Patch 7.4.237 (after 7.4.236)
问题:　　When some patches were not included has("patch-7.4.123") may return
　　true falsely.
解决方案:　Check for the specific patch number.
相关文件:　runtime/doc/eval.txt, src/eval.c

Patch 7.4.238
问题:　　Vim does not support the smack library.
解决方案:　Add smack support (Jose Bollo)
相关文件:　src/config.h.in, src/configure.in, src/fileio.c, src/memfile.c,
　　src/os_unix.c, src/undo.c, src/auto/configure

Patch 7.4.239
问题:　　":e +" does not position cursor at end of the file.
解决方案:　Check for "+" being the last character (ZyX)
相关文件:　src/ex_docmd.c

Patch 7.4.240
问题:　　":tjump" shows "\n" as "\\n".
解决方案:　Skip over "\" that escapes a backslash. (Gary Johnson)
相关文件:　src/tag.c

Patch 7.4.241
问题:　　The string returned by submatch() does not distinguish between a
　　NL from a line break and a NL that stands for a NUL character.
解决方案:　Add a second argument to return a list. (ZyX)
相关文件:　runtime/doc/eval.txt, src/eval.c, src/proto/regexp.pro,
　　src/regexp.c, src/testdir/test79.in, src/testdir/test79.ok,
　　src/testdir/test80.in, src/testdir/test80.ok

Patch 7.4.242
问题:　　getreg() does not distinguish between a NL used for a line break
　　and a NL used for a NUL character.
解决方案:　Add another argument to return a list. (ZyX)
相关文件:　runtime/doc/eval.txt, src/eval.c src/ops.c, src/proto/ops.pro,
　　src/vim.h, src/Makefile, src/testdir/test_eval.in,
　　src/testdir/test_eval.ok, src/testdir/Make_amiga.mak,
　　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
　　src/testdir/Make_os2.mak, src/testdir/Make_vms.mms

Patch 7.4.243
问题:　　Cannot use setreg() to add text that includes a NUL.
解决方案:　Make setreg() accept a list.
相关文件:　runtime/doc/eval.txt, src/eval.c, src/ops.c, src/proto/ops.pro,
　　src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.244 (after 7.4.238)
问题:　　The smack feature causes stray error messages.
解决方案:　Remove the error messages.
相关文件:　src/os_unix.c

Patch 7.4.245
问题:　　Crash for "vim -u NONE -N -c '&&'".
解决方案:　Check for the pattern to be NULL. (Dominique Pelle)
相关文件:　src/ex_cmds.c

Patch 7.4.246
问题:　　Configure message for detecting smack are out of sequence.
解决方案:　Put the messages in the right place. (Kazunobu Kuriyama)
相关文件:　src/configure.in, src/auto/configure

Patch 7.4.247
问题:　　When passing input to system() there is no way to keep NUL and
　　NL characters separate.
解决方案:　Optionally use a list for the system() input. (ZyX)
相关文件:　runtime/doc/eval.txt, src/eval.c

Patch 7.4.248
问题:　　Cannot distinguish between NL and NUL in output of system().
解决方案:　Add systemlist(). (ZyX)
相关文件:　runtime/doc/eval.txt, src/eval.c, src/ex_cmds2.c, src/misc1.c,
　　src/proto/misc1.pro

Patch 7.4.249
问题:　　Using setreg() with a list of numbers does not work.
解决方案:　Use a separate buffer for numbers. (ZyX)
相关文件:　src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.250
问题:　　Some test files missing from distribution.
解决方案:　Add pattern for newly added tests.
相关文件:　Filelist

Patch 7.4.251
问题:　　Crash when BufAdd autocommand wipes out the buffer.
解决方案:　Check for buffer to still be valid. Postpone freeing the buffer
　　structure. (Hirohito Higashi)
相关文件:　src/buffer.c, src/ex_cmds.c, src/fileio.c, src/globals.h

Patch 7.4.252
问题:　　Critical error in GTK, removing timer twice.
解决方案:　Clear the timer after removing it. (James McCoy)
相关文件:　src/gui_gtk_x11.c

Patch 7.4.253
问题: Crash when using cpp syntax file with pattern using external match. (Havard Garnes)
解决方案: Discard match when end column is before start column.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.4.254
问题: Smack support detection is incomplete.
解决方案: Check for attr/xattr.h and specific macro.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.255
问题: Configure check for smack doesn't work with all shells. (David Larson)
解决方案: Remove spaces in set command.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.256 (after 7.4.248)
问题: Using systemlist() may cause a crash and does not handle NUL characters properly.
解决方案: Increase the reference count, allocate memory by length. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.257
问题: Compiler warning, possibly for mismatch in parameter name.
解决方案: Rename the parameter in the declaration.
相关文件: src/ops.c

Patch 7.4.258
问题: Configure fails if $CC contains options.
解决方案: Remove quotes around $CC. (Paul Barker)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.259
问题: Warning for misplaced "const".
解决方案: Move the "const". (Yukihiro Nakadaira)
相关文件: src/os_unix.c

Patch 7.4.260
问题: It is possible to define a function with a colon in the name. It is possible to define a function with a lower case character if a "#" appears after the name.
解决方案: Disallow using a colon other than with "s:". Ignore "#" after the name.
相关文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.261
问题: When updating the window involves a regexp pattern, an interactive substitute to replace a "\n" with a line break fails. (Ingo Karkat)
解决方案: Set reg_line_lbr in vim_regsub() and vim_regsub_multi().

相关文件: src/regexp.c, src/testdir/test79.in, src/testdir/test79.ok

Patch 7.4.262
问题: Duplicate code in regexec().
解决方案: Add line_lbr flag to regexec_nl().
相关文件: src/regexp.c, src/regexp_nfa.c, src/regexp.h

Patch 7.4.263
问题: GCC 4.8 compiler warning for hiding a declaration (Franois Gannaz)
解决方案: Remove the second declaration.
相关文件: src/eval.c

Patch 7.4.264 (after 7.4.260)
问题: Can't define a function starting with "g:". Can't assign a
funcref to a buffer-local variable.
解决方案: Skip "g:" at the start of a function name. Don't check for colons
when assigning to a variable.
相关文件: src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.265 (after 7.4.260)
问题: Can't call a global function with "g:" in an expression.
解决方案: Skip the "g:" when looking up the function.
相关文件: src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.266
问题: Test 62 fails.
解决方案: Set the language to C. (Christian Brabandt)
相关文件: src/testdir/test62.in

Patch 7.4.267 (after 7.4.178)
问题: The '[mark is in the wrong position after "gq". (Ingo Karkat)
解决方案: Add the setmark argument to do_join(). (Christian Brabandt)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test_autoformat_join.in,
src/testdir/test_autoformat_join.ok, src/Makefile, src/edit.c,
src/ex_cmds.c, src/ex_docmd.c, src/normal.c, src/ops.c,
src/proto/ops.pro

Patch 7.4.268
问题: Using exists() on a funcref for a script-local function does not
work.
解决方案: Translate <SNR> to the special byte sequence. Add a test.
相关文件: src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok,
src/testdir/test_eval_func.vim, Filelist

Patch 7.4.269
问题: CTRL-U in Insert mode does not work after using a cursor key.
(Pine Wu)
解决方案: Use the original insert start position. (Christian Brabandt)
相关文件: src/edit.c, src/testdir/test29.in, src/testdir/test29.ok

Patch 7.4.270

问题: Comparing pointers instead of the string they point to.
解决方案: Use strcmp(). (Ken Takata)
相关文件: src/gui_gtk_x11.c

Patch 7.4.271
问题: Compiler warning on 64 bit windows.
解决方案: Add type cast. (Mike Williams)
相关文件: src/ops.c

Patch 7.4.272
问题: Using just "$" does not cause an error message.
解决方案: Check for empty environment variable name. (Christian Brabandt)
相关文件: src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.273
问题: "make autoconf" and "make reconfig" may first run configure and
            then remove the output.
解决方案: Add these targets to the exceptions. (Ken Takata)
相关文件: src/Makefile

Patch 7.4.274
问题: When doing ":update" just before running an external command that
            changes the file, the timestamp may be unchanged and the file
            is not reloaded.
解决方案: Also check the file size.
相关文件: src/fileio.c

Patch 7.4.275
问题: When changing the type of a sign that hasn't been placed there is
            no error message.
解决方案: Add an error message. (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.4.276
问题: The fish shell is not supported.
解决方案: Use begin/end instead of () for fish. (Andy Russell)
相关文件: src/ex_cmds.c, src/misc1.c, src/option.c, src/proto/misc1.pro

Patch 7.4.277
问题: Using ":sign unplace *" may leave the cursor in the wrong position
            (Christian Brabandt)
解决方案: Update the cursor position when removing all signs.
相关文件: src/buffer.c

Patch 7.4.278
问题: list_remove() conflicts with function defined in Sun header file.
解决方案: Rename the function. (Richard Palo)
相关文件: src/eval.c, src/if_lua.c, src/if_py_both.h, src/proto/eval.pro

Patch 7.4.279
问题: globpath() returns a string, making it difficult to get a list of
            matches. (Greg Novack)
解决方案: Add an optional argument like with glob(). (Adnan Zafar)
相关文件: runtime/doc/eval.txt, src/eval.c, src/ex_getln.c, src/misc1.c,

src/misc2.c, src/proto/ex_getln.pro, src/proto/misc2.pro,
src/testdir/test97.in, src/testdir/test97.ok

Patch 7.4.280
问题: When using a session file the relative position of the cursor is not restored if there is another tab. (Nobuhiro Takasaki)
解决方案: Update w_wrow before calculating the fraction.
相关文件: src/window.c

Patch 7.4.281
问题: When a session file has more than one tabpage and 'showtabline' is one the positions may be slightly off.
解决方案: Set 'showtabline' to two while positioning windows.
相关文件: src/ex_docmd.c

Patch 7.4.282 (after 7.4.279)
问题: Test 97 fails on Mac.
解决方案: Do not ignore case in file names. (Jun Takimoto)
相关文件: src/testdir/test97.in

Patch 7.4.283 (after 7.4.276)
问题: Compiler warning about unused variable. (Charles Cooper)
解决方案: Move the variable inside the #if block.
相关文件: src/ex_cmds.c

Patch 7.4.284
问题: Setting 'langmap' in the modeline can cause trouble. E.g. mapping ":" breaks many commands. (Jens-Wolfhard Schicke-Uffmann)
解决方案: Disallow setting 'langmap' from the modeline.
相关文件: src/option.c

Patch 7.4.285
问题: When 'relativenumber' is set and deleting lines or undoing that, line numbers are not always updated. (Robert Arkwright)
解决方案: (Christian Brabandt)
相关文件: src/misc1.c

Patch 7.4.286
问题: Error messages are inconsistent. (ZyX)
解决方案: Change "Lists" to "list".
相关文件: src/eval.c

Patch 7.4.287
问题: Patches for .hgignore don't work, since the file is not in the distribution.
解决方案: Add .hgignore to the distribution. Will be effective with the next version.
相关文件: Filelist

Patch 7.4.288
问题: When 'spellfile' is set the screen is not redrawn.
解决方案: Redraw when updating the spelling info. (Christian Brabandt)
相关文件: src/spell.c

Patch 7.4.289
问题: Pattern with repeated backreference does not match with new regexp engine. (Urtica Dioica)
解决方案: Also check the end of a submatch when deciding to put a state in the state list.
相关文件: src/testdir/test64.in, src/testdir/test64.ok, src/regexp_nfa.c

Patch 7.4.290
问题: A non-greedy match followed by a branch is too greedy. (Ingo Karkat)
解决方案: Add NFA_MATCH when it is already in the state list if the position differs.
相关文件: src/testdir/test64.in, src/testdir/test64.ok, src/regexp_nfa.c

Patch 7.4.291
问题: Compiler warning for int to pointer of different size when DEBUG is defined.
解决方案: use smsg() instead of EMSG3().
相关文件: src/regexp.c

Patch 7.4.292
问题: Searching for "a" does not match accented "a" with new regexp engine, does match with old engine. (David Bürgin)
"ca" does not match "ca" with accented "a" with either engine.
解决方案: Change the old engine, check for following composing character also for single-byte patterns.
相关文件: src/regexp.c, src/testdir/test95.in, src/testdir/test95.ok

Patch 7.4.293
问题: It is not possible to ignore composing characters at a specific point in a pattern.
解决方案: Add the %C item.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test95.in, src/testdir/test95.ok, runtime/doc/pattern.txt

Patch 7.4.294 (7.4.293)
问题: Test files missing from patch.
解决方案: Patch the test files.
相关文件: src/testdir/test95.in, src/testdir/test95.ok

Patch 7.4.295
问题: Various typos, bad white space and unclear comments.
解决方案: Fix typos. Improve white space. Update comments.
相关文件: src/testdir/test49.in, src/macros.h, src/screen.c, src/structs.h, src/gui_gtk_x11.c, src/os_unix.c

Patch 7.4.296
问题: Can't run tests on Solaris.
解决方案: Change the way VIMRUNTIME is set. (Laurent Blume)
相关文件: src/testdir/Makefile

Patch 7.4.297
问题: Memory leak from result of get_isolated_shell_name().
解决方案: Free the memory. (Dominique Pelle)

相关文件: src/ex_cmds.c, src/misc1.c

Patch 7.4.298
问题: Can't have a funcref start with "t:".
解决方案: Add "t" to the list of accepted names. (Yukihiro Nakadaira)
相关文件: src/eval.c

Patch 7.4.299
问题: When running configure twice DYNAMIC_PYTHON_DLL may become empty.
解决方案: Use AC_CACHE_VAL. (Ken Takata)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.300
问题: The way config.cache is removed doesn't always work.
解决方案: Always remove config.cache. (Ken Takata)
相关文件: src/Makefile

Patch 7.4.301 (after 7.4.280)
问题: Still a scrolling problem when loading a session file.
解决方案: Fix off-by-one mistake. (Nobuhiro Takasaki)
相关文件: src/window.c

Patch 7.4.302
问题: Signs placed with 'foldcolumn' set don't show up after filler lines.
解决方案: Take filler lines into account. (Olaf Dabrunz)
相关文件: src/screen.c

Patch 7.4.303
问题: When using double-width characters the text displayed on the command line is sometimes truncated.
解决方案: Reset the string length. (Nobuhiro Takasaki)
相关文件: src/screen.c

Patch 7.4.304
问题: Cannot always use Python with Vim.
解决方案: Add the manifest to the executable. (Jacques Germishuys)
相关文件: src/Make_mvc.mak

Patch 7.4.305
问题: Making 'ttymouse' empty after the xterm version was requested causes problems. (Elijah Griffin)
解决方案: Do not check for DEC mouse sequences when the xterm version was requested. Also don't request the xterm version when DEC mouse was enabled.
相关文件: src/term.c, src/os_unix.c, src/proto/term.pro, src/globals.h

Patch 7.4.306
问题: getchar(0) does not return Esc.
解决方案: Do not wait for an Esc sequence to be complete. (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/getchar.c

Patch 7.4.307 (after 7.4.305)

问题:　　Can't build without the +termresponse feature.
解决方案:　Add proper #ifdefs.
相关文件:　src/os_unix.c, src/term.c

Patch 7.4.308
问题:　　When using ":diffsplit" on an empty file the cursor is displayed
　　on the command line.
解决方案:　Limit the value of w_topfill.
相关文件:　src/diff.c

Patch 7.4.309
问题:　　When increasing the size of the lower window, the upper window
　　jumps back to the top. (Ron Aaron)
解决方案:　Change setting the topline. (Nobuhiro Takasaki)
相关文件:　src/window.c

Patch 7.4.310
问题:　　getpos()/setpos() don't include curswant.
解决方案:　Add a fifth number when getting/setting the cursor.
相关文件:　src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok,
　　runtime/doc/eval.txt

Patch 7.4.311
问题:　　Can't use winrestview to only restore part of the view.
解决方案:　Handle missing items in the dict. (Christian Brabandt)
相关文件:　src/eval.c, runtime/doc/eval.txt

Patch 7.4.312
问题:　　Cannot figure out what argument list is being used for a window.
解决方案:　Add the arglistid() function. (Marcin Szamotulski)
相关文件:　runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/eval.c,
　　src/ex_docmd.c, src/globals.h, src/structs.h, src/main.c

Patch 7.4.313 (after 7.4.310)
问题:　　Changing the return value of getpos() causes an error. (Jie Zhu)
解决方案:　Revert getpos() and add getcurpos().
相关文件:　src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok,
　　runtime/doc/eval.txt

Patch 7.4.314
问题:　　Completion messages can get in the way of a plugin.
解决方案:　Add 'c' flag to 'shortmess' option. (Shougo Matsu)
相关文件:　runtime/doc/options.txt, src/edit.c, src/option.h, src/screen.c

Patch 7.4.315 (after 7.4.309)
问题:　　Fixes for computation of topline not tested.
解决方案:　Add test. (Hirohito Higashi)
相关文件:　src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
　　src/testdir/Make_vms.mms, src/testdir/Makefile,
　　src/testdir/test107.in, src/testdir/test107.ok

Patch 7.4.316
问题:　　Warning from 64-bit compiler.

解决方案: Add type cast. (Mike Williams)
相关文件: src/ex_getln.c

Patch 7.4.317
问题: Crash when starting gvim. Issue 230.
解决方案: Check for a pointer to be NULL. (Christian Brabandt)
相关文件: src/window.c

Patch 7.4.318
问题: Check for whether a highlight group has settings ignores fg and bg
           color settings.
解决方案: Also check cterm and GUI color settings. (Christian Brabandt)
相关文件: src/syntax.c

Patch 7.4.319
问题: Crash when putting zero bytes on the clipboard.
解决方案: Do not support the utf8_atom target when not using a Unicode
           encoding. (Naofumi Honda)
相关文件: src/ui.c

Patch 7.4.320
问题: Possible crash when an BufLeave autocommand deletes the buffer.
解决方案: Check for the window pointer being valid. Postpone freeing the
           window until autocommands are done. (Yasuhiro Matsumoto)
相关文件: src/buffer.c, src/fileio.c, src/globals.h, src/window.c

Patch 7.4.321
问题: Can't build with strawberry perl 5.20 + mingw-w64-4.9.0.
解决方案: Define save_strlen. (Ken Takata)
相关文件: src/if_perl.xs

Patch 7.4.322
问题: Using "msgfmt" is hard coded, cannot use "gmsgfmt".
解决方案: Use the msgfmt command found by configure. (Danek Duvall)
相关文件: src/config.mk.in, src/po/Makefile

Patch 7.4.323
问题: substitute() with zero width pattern breaks multi-byte character.
解决方案: Take multi-byte character size into account. (Yukihiro Nakadaira)
相关文件: src/eval.c src/testdir/test69.in, src/testdir/test69.ok

Patch 7.4.324
问题: In Ex mode, cyrillic characters are not handled. (Stas Malavin)
解决方案: Support multi-byte characters in Ex mode. (Yukihiro Nakadaira)
相关文件: src/ex_getln.c

Patch 7.4.325
问题: When starting the gui and changing the window size the status line
           may not be drawn correctly.
解决方案: Catch new_win_height() being called recursively. (Christian
           Brabandt)
相关文件: src/window.c

Patch 7.4.326

问题:　　Can't build Tiny version. (Elimar Riesebieter)
解决方案:　Add #ifdef.
相关文件:　src/window.c

Patch 7.4.327
问题:　　When 'verbose' is set to display the return value of a function,
　　may get E724 repeatedly.
解决方案:　Do not give an error for verbose messages. Abort conversion to
　　string after an error.
相关文件:　src/eval.c

Patch 7.4.328
问题:　　Selection of inner block is inconsistent.
解决方案:　Skip indent not only for '}' but all parens. (Tom McDonald)
相关文件:　src/search.c

Patch 7.4.329
问题:　　When moving the cursor and then switching to another window the
　　previous window isn't scrolled. (Yukihiro Nakadaira)
解决方案:　Call update_topline() before leaving the window. (Christian
　　Brabandt)
相关文件:　src/window.c

Patch 7.4.330
问题:　　Using a regexp pattern to highlight a specific position can be
　　slow.
解决方案:　Add matchaddpos() to highlight specific positions efficiently.
　　(Alexey Radkov)
相关文件:　runtime/doc/eval.txt, runtime/doc/usr_41.txt,
　　runtime/plugin/matchparen.vim, src/eval.c, src/ex_docmd.c,
　　src/proto/window.pro, src/screen.c, src/structs.h,
　　src/testdir/test63.in, src/testdir/test63.ok, src/window.c

Patch 7.4.331
问题:　　Relative numbering not updated after a linewise yank. Issue 235.
解决方案:　Redraw after the yank. (Christian Brabandt)
相关文件:　src/ops.c

Patch 7.4.332
问题:　　GTK: When a sign icon doesn't fit exactly there can be ugly gaps.
解决方案:　Scale the sign to fit when the aspect ratio is not too far off.
　　(Christian Brabandt)
相关文件:　src/gui_gtk_x11.c

Patch 7.4.333
问题:　　Compiler warning for unused function.
解决方案:　Put the function inside the #ifdef.
相关文件:　src/screen.c

Patch 7.4.334 (after 7.4.330)
问题:　　Uninitialized variables, causing some problems.
解决方案:　Initialize the variables. (Dominique Pelle)
相关文件:　src/screen.c, src/window.c

Patch 7.4.335
问题: No digraph for the new rouble sign.
解决方案: Add the digraphs =R and =P.
相关文件: src/digraph.c, runtime/doc/digraph.txt

Patch 7.4.336
问题: Setting 'history' to a big value causes out-of-memory errors.
解决方案: Limit the value to 10000. (Hirohito Higashi)
相关文件: runtime/doc/options.txt, src/option.c

Patch 7.4.337
问题: When there is an error preparing to edit the command line, the
command won't be executed. (Hirohito Higashi)
解决方案: Reset did_emsg before editing.
相关文件: src/ex_getln.c

Patch 7.4.338
问题: Cannot wrap lines taking indent into account.
解决方案: Add the 'breakindent' option. (many authors, final improvements by
Christian Brabandt)
相关文件: runtime/doc/eval.txt, runtime/doc/options.txt, runtime/optwin.vim,
src/buffer.c, src/charset.c, src/edit.c, src/ex_getln.c,
src/getchar.c, src/misc1.c, src/misc2.c, src/ops.c, src/option.c,
src/option.h, src/proto/charset.pro, src/proto/misc1.pro,
src/proto/option.pro, src/screen.c, src/structs.h,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test_breakindent.in, src/testdir/test_breakindent.ok,
src/ui.c, src/version.c

Patch 7.4.339
问题: Local function is available globally.
解决方案: Add "static".
相关文件: src/option.c, src/proto/option.pro

Patch 7.4.340
问题: Error from sed about illegal bytes when installing Vim.
解决方案: Prepend LC_ALL=C. (Itchyny)
相关文件: src/installman.sh

Patch 7.4.341
问题: sort() doesn't handle numbers well.
解决方案: Add an argument to specify sorting on numbers. (Christian Brabandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test55.in,
src/testdir/test55.ok

Patch 7.4.342
问题: Clang gives warnings.
解决方案: Add an else block. (Dominique Pelle)
相关文件: src/gui_beval.c

Patch 7.4.343
问题: matchdelete() does not always update the right lines.

解决方案: Fix off-by-one error. (Ozaki Kiichi)
相关文件: src/window.c

Patch 7.4.344
问题: Unnecessary initializations and other things related to
matchaddpos().
解决方案: Code cleanup. (Alexey Radkov)
相关文件: runtime/doc/eval.txt, src/screen.c, src/window.c

Patch 7.4.345 (after 7.4.338)
问题: Indent is not updated when deleting indent.
解决方案: Remember changedtick.
相关文件: src/misc1.c

Patch 7.4.346 (after 7.4.338)
问题: Indent is not updated when changing 'breakindentopt'. (itchyny)
解决方案: Do not cache "brishift". (Christian Brabandt)
相关文件: src/misc1.c

Patch 7.4.347
问题: test55 fails on some systems.
解决方案: Remove the elements that all result in zero and can end up in an
arbitrary position.
相关文件: src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.348
问题: When using "J1" in 'cinoptions' a line below a continuation line
gets too much indent.
解决方案: Fix parentheses in condition.
相关文件: src/misc1.c

Patch 7.4.349
问题: When there are matches to highlight the whole window is redrawn,
which is slow.
解决方案: Only redraw everything when lines were inserted or deleted.
Reset b_mod_xlines when needed. (Alexey Radkov)
相关文件: src/screen.c, src/window.c

Patch 7.4.350
问题: Using C indenting for Javascript does not work well for a {} block
inside parentheses.
解决方案: When looking for a matching paren ignore one that is before the
start of a {} block.
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.351
问题: sort() is not stable.
解决方案: When the items are identical, compare the pointers.
相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.352
问题: With 'linebreak' a tab causes a missing line break.
解决方案: Count a tab for what it's worth also for shorter lines.
(Christian Brabandt)

相关文件: src/charset.c

Patch 7.4.353
问题: 'linebreak' doesn't work with the 'list' option.
解决方案: Make it work. (Christian Brabandt)
相关文件: runtime/doc/options.txt, src/charset.c, src/screen.c,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test_listlbr.in, src/testdir/test_listlbr.ok

Patch 7.4.354
问题: Compiler warning.
解决方案: Change NUL to NULL. (Ken Takata)
相关文件: src/screen.c

Patch 7.4.355
问题: Several problems with Javascript indenting.
解决方案: Improve Javascript indenting.
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.356
问题: Mercurial does not ignore memfile_test. (Daniel Hahler)
解决方案: Add memfile_test to ignored files, remove trailing spaces.
相关文件: .hgignore

Patch 7.4.357
问题: After completion some characters are not redrawn.
解决方案: Clear the command line unconditionally. (Jacob Niehus)
相关文件: src/edit.c

Patch 7.4.358 (after 7.4.351)
问题: Sort is not always stable.
解决方案: Add an index instead of relying on the pointer to remain the same.
Idea by Jun Takimoto.
相关文件: src/eval.c

Patch 7.4.359
问题: When 'ttymouse' is set to 'uxterm' the xterm version is not
requested. (Tomas Janousek)
解决方案: Do not mark uxterm as a conflict mouse and add
resume_get_esc_sequence().
相关文件: src/term.c, src/os_unix.c, src/proto/term.pro

Patch 7.4.360
问题: In a regexp pattern a "$" followed by \v or \V is not seen as the
end-of-line.
解决方案: Handle the situation. (Ozaki Kiichi)
相关文件: src/regexp.c

Patch 7.4.361
问题: Lots of flickering when filling the preview window for 'omnifunc'.
解决方案: Disable redrawing. (Hirohito Higashi)
相关文件: src/popupmnu.c

Patch 7.4.362
问题: When matchaddpos() uses a length smaller than the number of bytes in the (last) character the highlight continues until the end of the line.
解决方案: Change condition from equal to larger-or-equal.
相关文件: src/screen.c

Patch 7.4.363
问题: In Windows console typing 0xCE does not work.
解决方案: Convert 0xCE to K_NUL 3. (Nobuhiro Takasaki et al.)
相关文件: src/os_win32.c, src/term.c

Patch 7.4.364
问题: When the viminfo file can't be renamed there is no error message. (Vladimir Berezhnoy)
解决方案: Check for the rename to fail.
相关文件: src/ex_cmds.c

Patch 7.4.365
问题: Crash when using ":botright split" when there isn't much space.
解决方案: Add a check for the minimum width/height. (Yukihiro Nakadaira)
相关文件: src/window.c

Patch 7.4.366
问题: Can't run the linebreak test on MS-Windows.
解决方案: Fix the output file name. (Taro Muraoka)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.367 (after 7.4.357)
问题: Other solution for redrawing after completion.
解决方案: Schedule a window redraw instead of just clearing the command line. (Jacob Niehus)
相关文件: src/edit.c

Patch 7.4.368
问题: Restoring the window sizes after closing the command line window doesn't work properly if there are nested splits.
解决方案: Restore the sizes twice. (Hirohito Higashi)
相关文件: src/window.c

Patch 7.4.369
问题: Using freed memory when exiting while compiled with EXITFREE.
解决方案: Set curwin to NULL and check for that. (Dominique Pelle)
相关文件: src/buffer.c, src/window.c

Patch 7.4.370
问题: Linebreak test fails when encoding is not utf-8. (Danek Duvall)
解决方案: Split the test in a single byte one and a utf-8 one. (Christian Brabandt)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_listlbr.in, src/testdir/test_listlbr.ok,

src/testdir/test_listlbr_utf8.in, src/testdir/test_listlbr_utf8.ok

Patch 7.4.371
问题: When 'linebreak' is set control characters are not correctly displayed. (Kimmy Lindvall)
解决方案: Set n_extra. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.4.372
问题: When 'winminheight' is zero there might not be one line for the current window.
解决方案: Change the size computations. (Yukihiro Nakadaira)
相关文件: src/window.c

Patch 7.4.373
问题: Compiler warning for unused argument and unused variable.
解决方案: Add UNUSED. Move variable inside #ifdef.
相关文件: src/charset.c, src/window.c

Patch 7.4.374
问题: Character after "fb" command not mapped if it might be a composing character.
解决方案: Don't disable mapping when looking for a composing character. (Jacob Niehus)
相关文件: src/normal.c

Patch 7.4.375
问题: Test 63 fails when run with GUI-only Vim.
解决方案: Add guibg attributes. (suggested by Mike Soyka)
相关文件: src/testdir/test63.in

Patch 7.4.376 (after 7.4.367)
问题: Popup menu flickers too much.
解决方案: Remove the forced redraw. (Hirohito Higashi)
相关文件: src/edit.c

Patch 7.4.377
问题: When 'equalalways' is set a split may report "no room" even though there is plenty of room.
解决方案: Compute the available room properly. (Yukihiro Nakadaira)
相关文件: src/window.c

Patch 7.4.378
问题: Title of quickfix list is not kept for setqflist(list, 'r').
解决方案: Keep the title. Add a test. (Lcd)
相关文件: src/quickfix.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_qf_title.in, src/testdir/test_qf_title.ok

Patch 7.4.379
问题: Accessing freed memory after using setqflist(list, 'r'). (Lcd)
解决方案: Reset qf_index.

相关文件: src/quickfix.c

Patch 7.4.380
问题: Loading python may cause Vim to exit.
解决方案: Avoid loading the "site" module. (Taro Muraoka)
相关文件: src/if_python.c

Patch 7.4.381
问题: Get u_undo error when backspacing in Insert mode deletes more than
           one line break. (Ayberk Ozgur)
解决方案: Also decrement Insstart.lnum.
相关文件: src/edit.c

Patch 7.4.382
问题: Mapping characters may not work after typing Esc in Insert mode.
解决方案: Fix the noremap flags for inserted characters. (Jacob Niehus)
相关文件: src/getchar.c

Patch 7.4.383
问题: Bad interaction between preview window and omnifunc.
解决方案: Avoid redrawing the status line. (Hirohito Higashi)
相关文件: src/popupmnu.c

Patch 7.4.384
问题: Test 102 fails when compiled with small features.
解决方案: Source small.vim. (Jacob Niehus)
相关文件: src/testdir/test102.in

Patch 7.4.385
问题: When building with tiny or small features building the .mo files
           fails.
解决方案: In autoconf do not setup for building the .mo files when it would
           fail.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.386
问题: When splitting a window the changelist position is wrong.
解决方案: Copy the changelist position. (Jacob Niehus)
相关文件: src/window.c, src/testdir/Make_amiga.mak,
           src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
           src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
           src/testdir/Makefile, src/testdir/test_changelist.in,
           src/testdir/test_changelist.ok

Patch 7.4.387
问题: "4gro" replaces one character then executes "ooo". (Urtica Dioica)
解决方案: Write the ESC in the second stuff buffer.
相关文件: src/getchar.c, src/proto/getchar.pro, src/edit.c,
           src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
           src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
           src/testdir/Make_vms.mms, src/testdir/Makefile,
           src/testdir/test_insertcount.in, src/testdir/test_insertcount.ok

Patch 7.4.388

问题:　　With 'linebreak' set and 'list' unset a Tab is not counted
properly. (Kent Sibilev)
解决方案:　Check the 'list' option. (Christian Brabandt)
相关文件:　src/screen.c, src/testdir/test_listlbr_utf8.in,
src/testdir/test_listlbr_utf8.ok

Patch 7.4.389
问题:　　Still sometimes Vim enters Replace mode when starting up.
解决方案:　Use a different solution in detecting the termresponse and
location response. (Hayaki Saito)
相关文件:　src/globals.h, src/os_unix.c, src/term.c, src/proto/term.pro

Patch 7.4.390
问题:　　Advancing pointer over end of a string.
解决方案:　Init quote character to -1 instead of zero. (Dominique Pelle)
相关文件:　src/misc1.c

Patch 7.4.391
问题:　　No 'cursorline' highlighting when the cursor is on a line with
diff highlighting. (Benjamin Fritz)
解决方案:　Combine the highlight attributes. (Christian Brabandt)
相关文件:　src/screen.c

Patch 7.4.392
问题:　　Not easy to detect type of command line window.
解决方案:　Add the getcmdwintype() function. (Jacob Niehus)
相关文件:　src/eval.c

Patch 7.4.393
问题:　　Text drawing on newer MS-Windows systems is suboptimal. Some
multi-byte characters are not displayed, even though the same font
in Notepad can display them. (Srinath Avadhanula)
解决方案:　Add the 'renderoptions' option to enable DirectX drawing. (Taro
Muraoka)
相关文件:　runtime/doc/eval.txt, runtime/doc/options.txt,
runtime/doc/various.txt, src/Make_cyg.mak, src/Make_ming.mak,
src/Make_mvc.mak, src/eval.c, src/gui_dwrite.cpp,
src/gui_dwrite.h, src/gui_w32.c, src/gui_w48.c, src/option.c,
src/option.h, src/version.c, src/vim.h, src/proto/gui_w32.pro

Patch 7.4.394 (after 7.4.393)
问题:　　When using DirectX last italic character is incomplete.
解决方案:　Add one to the number of cells. (Ken Takata)
相关文件:　src/gui_w32.c

Patch 7.4.395 (after 7.4.355)
问题:　　C indent is wrong below an if with wrapped condition followed by
curly braces. (Trevor Powell)
解决方案:　Make a copy of tryposBrace.
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.396
问题:　　When 'clipboard' is "unnamed", :g/pat/d is very slow. (Praful)
解决方案:　Only set the clipboard after the last delete. (Christian Brabandt)

相关文件: src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/globals.h,
           src/ops.c, src/proto/ui.pro, src/ui.c

Patch 7.4.397
问题: Matchparen only uses the topmost syntax item.
解决方案: Go through the syntax stack to find items. (James McCoy)
           Also use getcurpos() when possible.
相关文件: runtime/plugin/matchparen.vim

Patch 7.4.398 (after 7.4.393)
问题: Gcc error for the argument of InterlockedIncrement() and
           InterlockedDecrement(). (Axel Bender)
解决方案: Remove "unsigned" from the cRefCount_ declaration.
相关文件: src/gui_dwrite.cpp

Patch 7.4.399
问题: Encryption implementation is messy. Blowfish encryption has a
           weakness.
解决方案: Refactor the encryption, store the state in an allocated struct
           instead of using a save/restore mechanism. Introduce the
           "blowfish2" method, which does not have the weakness and encrypts
           the whole undo file. (largely by David Leadbeater)
相关文件: runtime/doc/editing.txt, runtime/doc/options.txt, src/Makefile,
           src/blowfish.c, src/crypt.c, src/crypt_zip.c, src/ex_docmd.c,
           src/fileio.c, src/globals.h, src/main.c, src/memline.c,
           src/misc2.c, src/option.c, src/proto.h, src/proto/blowfish.pro,
           src/proto/crypt.pro, src/proto/crypt_zip.pro,
           src/proto/fileio.pro, src/proto/misc2.pro, src/structs.h,
           src/undo.c, src/testdir/test71.in, src/testdir/test71.ok,
           src/testdir/test71a.in, src/testdir/test72.in,
           src/testdir/test72.ok

Patch 7.4.400
问题: List of distributed files is incomplete.
解决方案: Add recently added files.
相关文件: Filelist

Patch 7.4.401 (after 7.4.399)
问题: Can't build on MS-Windows.
解决方案: Include the new files in all the Makefiles.
相关文件: src/Make_bc3.mak, src/Make_bc5.mak, src/Make_cyg.mak,
           src/Make_dice.mak, src/Make_djg.mak, src/Make_ivc.mak,
           src/Make_manx.mak, src/Make_ming.mak, src/Make_morph.mak,
           src/Make_mvc.mak, src/Make_os2.mak, src/Make_sas.mak,
           Make_vms.mms

Patch 7.4.402
问题: Test 72 crashes under certain conditions. (Kazunobu Kuriyama)
解决方案: Clear the whole bufinfo_T early.
相关文件: src/undo.c

Patch 7.4.403
问题: Valgrind reports errors when running test 72. (Dominique Pelle)
解决方案: Reset the local 'cryptmethod' option before storing the seed.

Set the seed in the memfile even when there is no block0 yet.
相关文件: src/fileio.c, src/option.c, src/memline.c

Patch 7.4.404
问题: Windows 64 bit compiler warnings.
解决方案: Add type casts. (Mike Williams)
相关文件: src/crypt.c, src/undo.c

Patch 7.4.405
问题: Screen updating is slow when using matches.
解决方案: Do not use the ">=" as in patch 7.4.362, check the lnum.
相关文件: src/screen.c, src/testdir/test63.in, src/testdir/test63.ok

Patch 7.4.406
问题: Test 72 and 100 fail on MS-Windows.
解决方案: Set fileformat to unix in the tests. (Taro Muraoka)
相关文件: src/testdir/test72.in, src/testdir/test100.in

Patch 7.4.407
问题: Inserting text for Visual block mode, with cursor movement,
repeats the wrong text. (Aleksandar Ivanov)
解决方案: Reset the update_Insstart_orig flag. (Christian Brabandt)
相关文件: src/edit.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.408
问题: Visual block insert breaks a multi-byte character.
解决方案: Calculate the position properly. (Yasuhiro Matsumoto)
相关文件: src/ops.c, src/testdir/test_utf8.in, src/testdir/test_utf8.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.409
问题: Can't build with Perl on Fedora 20.
解决方案: Find xsubpp in another directory. (Michael Henry)
相关文件: src/Makefile, src/config.mk.in, src/configure.in,
src/auto/configure

Patch 7.4.410
问题: Fold does not open after search when there is a CmdwinLeave
autocommand.
解决方案: Restore KeyTyped. (Jacob Niehus)
相关文件: src/ex_getln.c

Patch 7.4.411
问题: "foo bar" sorts before "foo" with sort(). (John Little)
解决方案: Avoid putting quotes around strings before comparing them.
相关文件: src/eval.c

Patch 7.4.412
问题: Can't build on Windows XP with MSVC.
解决方案: Add SUBSYSTEM_VER to the Makefile. (Yongwei Wu)
相关文件: src/Make_mvc.mak, src/INSTALLpc.txt

Patch 7.4.413
问题: MS-Windows: Using US international keyboard layout, inserting dead key by pressing space does not always work. Issue 250.
解决方案: Let MS-Windows translate the message. (John Wellesz)
相关文件: src/gui_w48.c

Patch 7.4.414
问题: Cannot define a command only when it's used.
解决方案: Add the CmdUndefined autocommand event. (partly by Yasuhiro Matsumoto)
相关文件: runtime/doc/autocmd.txt, src/ex_docmd.c, src/fileio.c, src/proto/fileio.pro

Patch 7.4.415 (after 7.4.414)
问题: Cannot build. Warning for shadowed variable. (John Little)
解决方案: Add missing change. Remove declaration.
相关文件: src/vim.h, src/ex_docmd.c

Patch 7.4.416
问题: Problem with breakindent/showbreak and tabs.
解决方案: Handle tabs differently. (Christian Brabandt)
相关文件: src/testdir/test_breakindent.in, src/testdir/test_breakindent.ok, src/charset.c

Patch 7.4.417
问题: After splitting a window and setting 'breakindent' the default minimum with is not respected.
解决方案: Call briopt_check() when copying options to a new window.
相关文件: src/option.c, src/proto/option.pro, src/testdir/test_breakindent.in

Patch 7.4.418
问题: When leaving ":append" the cursor shape is like in Insert mode. (Jacob Niehus)
解决方案: Do not have State set to INSERT when calling getline().
相关文件: src/ex_cmds.c

Patch 7.4.419
问题: When part of a list is locked it's possible to make changes.
解决方案: Check if any of the list items is locked before make a change. (ZyX)
相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.420
问题: It's not obvious how to add a new test.
解决方案: Add a README file. (Christian Brabandt)
相关文件: src/testdir/README.txt

Patch 7.4.421
问题: Crash when searching for "\ze*". (Urtica Dioica)
解决方案: Disallow a multi after \ze and \zs.
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.422

问题:　　When using conceal with linebreak some text is not displayed
　　correctly. (Grüner Gimpel)
解决方案:　Check for conceal mode when using linebreak. (Christian Brabandt)
相关文件:　src/screen.c, src/testdir/test_listlbr.in,
　　src/testdir/test_listlbr.ok

Patch 7.4.423
问题:　　expand("$shell") does not work as documented.
解决方案:　Do not escape the $ when expanding environment variables.
相关文件:　src/os_unix.c, src/misc1.c, src/vim.h

Patch 7.4.424
问题:　　Get ml_get error when using Python to delete lines in a buffer
　　that is not in a window. issue 248.
解决方案:　Do not try adjusting the cursor for a different buffer.
相关文件:　src/if_py_both.h

Patch 7.4.425
问题:　　When 'showbreak' is used "gj" may move to the wrong position.
　　(Nazri Ramliy)
解决方案:　Adjust virtcol when 'showbreak' is set. (Christian Brabandt)
相关文件:　src/normal.c

Patch 7.4.426
问题:　　README File missing from list of files.
解决方案:　Update the list of files.
相关文件:　Filelist

Patch 7.4.427
问题:　　When an InsertCharPre autocommand executes system() typeahead may
　　be echoed and messes up the display. (Jacob Niehus)
解决方案:　Do not set cooked mode when invoked from ":silent".
相关文件:　src/eval.c, runtime/doc/eval.txt

Patch 7.4.428
问题:　　executable() may return a wrong result on MS-Windows.
解决方案:　Change the way SearchPath() is called. (Yasuhiro Matsumoto, Ken
　　Takata)
相关文件:　src/os_win32.c

Patch 7.4.429
问题:　　Build fails with fewer features. (Elimar Riesebieter)
解决方案:　Add #ifdef.
相关文件:　src/normal.c

Patch 7.4.430
问题:　　test_listlbr fails when compiled with normal features.
解决方案:　Check for the +conceal feature.
相关文件:　src/testdir/test_listlbr.in

Patch 7.4.431
问题:　　Compiler warning.
解决方案:　Add type cast. (Mike Williams)
相关文件:　src/ex_docmd.c

Patch 7.4.432
问题: When the startup code expands command line arguments, setting 'encoding' will not properly convert the arguments.
解决方案: Call get_cmd_argsW() early in main(). (Yasuhiro Matsumoto)
相关文件: src/os_win32.c, src/main.c, src/os_mswin.c

Patch 7.4.433
问题: Test 75 fails on MS-Windows.
解决方案: Use ":normal" instead of feedkeys(). (Michael Soyka)
相关文件: src/testdir/test75.in

Patch 7.4.434
问题: gettabvar() is not consistent with getwinvar() and getbufvar().
解决方案: Return a dict with all variables when the varname is empty. (Yasuhiro Matsumoto)
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test91.in, src/testdir/test91.ok

Patch 7.4.435
问题: Line formatting behaves differently when 'linebreak' is set. (mvxxc)
解决方案: Disable 'linebreak' temporarily. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.4.436
问题: ml_get error for autocommand that moves the cursor of the current window.
解决方案: Check the cursor position after switching back to the current buffer. (Christian Brabandt)
相关文件: src/fileio.c

Patch 7.4.437
问题: New and old regexp engine are not consistent.
解决方案: Also give an error for "\ze*" for the old regexp engine.
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.4.438
问题: Cached values for 'cino' not reset for ":set all&".
解决方案: Call parse_cino(). (Yukihiro Nakadaira)
相关文件: src/option.c

Patch 7.4.439
问题: Duplicate message in message history. Some quickfix messages appear twice. (Gary Johnson)
解决方案: Do not reset keep_msg too early. (Hirohito Higashi)
相关文件: src/main.c

Patch 7.4.440
问题: Omni complete popup drawn incorrectly.
解决方案: Call validate_cursor() instead of check_cursor(). (Hirohito Higashi)
相关文件: src/edit.c

Patch 7.4.441
问题: Endless loop and other problems when 'cedit' is set to CTRL-C.
解决方案: Do not call ex_window() when ex_normal_busy or got_int was set.
(Yasuhiro Matsumoto)
相关文件: src/ex_getln.c

Patch 7.4.442 (after 7.4.434)
问题: Using uninitialized variable.
解决方案: Pass the first window of the tabpage.
相关文件: src/eval.c

Patch 7.4.443
问题: Error reported by ubsan when running test 72.
解决方案: Add type cast to unsigned. (Dominique Pelle)
相关文件: src/undo.c

Patch 7.4.444
问题: Reversed question mark not recognized as punctuation. (Issue 258)
解决方案: Add the Supplemental Punctuation range.
相关文件: src/mbyte.c

Patch 7.4.445
问题: Clipboard may be cleared on startup.
解决方案: Set clip_did_set_selection to -1 during startup. (Christian
Brabandt)
相关文件: src/main.c, src/ui.c

Patch 7.4.446
问题: In some situations, when setting up an environment to trigger an
autocommand, the environment is not properly restored.
解决方案: Check the return value of switch_win() and call restore_win()
always. (Daniel Hahler)
相关文件: src/eval.c, src/misc2.c, src/window.c

Patch 7.4.447
问题: Spell files from Hunspell may generate a lot of errors.
解决方案: Add the IGNOREEXTRA flag.
相关文件: src/spell.c, runtime/doc/spell.txt

Patch 7.4.448
问题: Using ETO_IGNORELANGUAGE causes problems.
解决方案: Remove this flag. (Paul Moore)
相关文件: src/gui_w32.c

Patch 7.4.449
问题: Can't easily close the help window. (Chris Gaal)
解决方案: Add ":helpclose". (Christian Brabandt)
相关文件: runtime/doc/helphelp.txt, runtime/doc/index.txt, src/ex_cmds.c,
src/ex_cmds.h, src/proto/ex_cmds.pro

Patch 7.4.450
问题: Not all commands that edit another buffer support the +cmd
argument.
解决方案: Add the +cmd argument to relevant commands. (Marcin Szamotulski)

相关文件: runtime/doc/windows.txt, src/ex_cmds.h, src/ex_docmd.c

Patch 7.4.451
问题: Calling system() with empty input gives an error for writing the temp file.
解决方案: Do not try writing if the string length is zero. (Olaf Dabrunz)
相关文件: src/eval.c

Patch 7.4.452
问题: Can't build with tiny features. (Tony Mechelynck)
解决方案: Use "return" instead of "break".
相关文件: src/ex_cmds.c

Patch 7.4.453
问题: Still can't build with tiny features.
解决方案: Add #ifdef.
相关文件: src/ex_cmds.c

Patch 7.4.454
问题: When using a Visual selection of multiple words and doing CTRL-W_] it jumps to the tag matching the word under the cursor, not the selected text. (Patrick hemmer)
解决方案: Do not reset Visual mode. (idea by Christian Brabandt)
相关文件: src/window.c

Patch 7.4.455
问题: Completion for :buf does not use 'wildignorecase'. (Akshay H)
解决方案: Pass the 'wildignorecase' flag around.
相关文件: src/buffer.c

Patch 7.4.456
问题: 'backupcopy' is global, cannot write only some files in a different way.
解决方案: Make 'backupcopy' global-local. (Christian Brabandt)
相关文件: runtime/doc/options.txt, src/buffer.c, src/fileio.c, src/option.c, src/option.h, src/proto/option.pro, src/structs.h

Patch 7.4.457
问题: Using getchar() in an expression mapping may result in K_CURSORHOLD, which can't be recognized.
解决方案: Add the <CursorHold> key. (Hirohito Higashi)
相关文件: src/misc2.c

Patch 7.4.458
问题: Issue 252: Cursor moves in a zero-height window.
解决方案: Check for zero height. (idea by Christian Brabandt)
相关文件: src/move.c

Patch 7.4.459
问题: Can't change the icon after building Vim.
解决方案: Load the icon from a file on startup. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c, src/os_mswin.c, src/os_win32.c, src/proto/os_mswin.pro

Patch 7.4.460 (after 7.4.454)
问题: Can't build without the quickfix feature. (Erik Falor)
解决方案: Add a #ifdef.
相关文件: src/window.c

Patch 7.4.461
问题: MS-Windows: When collate is on the number of copies is too high.
解决方案: Only set the collated/uncollated count when collate is on. (Yasuhiro Matsumoto)
相关文件: src/os_mswin.c

Patch 7.4.462
问题: Setting the local value of 'backupcopy' empty gives an error. (Peter Mattern)
解决方案: When using an empty value set the flags to zero. (Hirohito Higashi)
相关文件: src/option.c

Patch 7.4.463
问题: Test 86 and 87 may hang on MS-Windows.
解决方案: Call inputrestore() after inputsave(). (Ken Takata)
相关文件: src/testdir/test86.in, src/testdir/test87.in

Patch 7.4.464 (after 7.4.459)
问题: Compiler warning.
解决方案: Add type cast. (Ken Takata)
相关文件: src/gui_w32.c

Patch 7.4.465 (after 7.4.016)
问题: Crash when expanding a very long string.
解决方案: Use wcsncpy() instead of wcscpy(). (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.466 (after 7.4.460)
问题: CTRL-W } does not open preview window. (Erik Falor)
解决方案: Don't set g_do_tagpreview for CTRL-W }.
相关文件: src/window.c

Patch 7.4.467
问题: 'linebreak' does not work well together with Visual mode.
解决方案: Disable 'linebreak' while applying an operator. Fix the test. (Christian Brabandt)
相关文件: src/normal.c, src/screen.c, src/testdir/test_listlbr.in, src/testdir/test_listlbr.ok

Patch 7.4.468
问题: Issue 26: CTRL-C does not interrupt after it was mapped and then unmapped.
解决方案: Reset mapped_ctrl_c. (Christian Brabandt)
相关文件: src/getchar.c

Patch 7.4.469 (after 7.4.467)
问题: Can't build with MSVC. (Ken Takata)
解决方案: Move the assignment after the declarations.

相关文件: src/normal.c

Patch 7.4.470
问题: Test 11 and 100 do not work properly on Windows.
解决方案: Avoid using feedkeys(). (Ken Takata)
相关文件: src/testdir/Make_dos.mak, src/testdir/test11.in,
src/testdir/test100.in

Patch 7.4.471
问题: MS-Windows: When printer name contains multi-byte, the name is
displayed as ???.
解决方案: Convert the printer name from the active codepage to 'encoding'.
(Yasuhiro Matsumoto)
相关文件: src/os_mswin.c

Patch 7.4.472
问题: The "precedes" entry in 'listchar' will be drawn when 'showbreak'
is set and 'list' is not.
解决方案: Only draw this character when 'list' is on. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.4.473
问题: Cursor movement is incorrect when there is a number/sign/fold
column and 'sbr' is displayed.
解决方案: Adjust the column for 'sbr'. (Christian Brabandt)
相关文件: src/charset.c

Patch 7.4.474
问题: AIX compiler can't handle // comment. Issue 265.
解决方案: Remove that line.
相关文件: src/regexp_nfa.c

Patch 7.4.475
问题: Can't compile on a system where Xutf8SetWMProperties() is not in
the X11 library. Issue 265.
解决方案: Add a configure check.
相关文件: src/configure.in, src/auto/configure, src/config.h.in,
src/os_unix.c

Patch 7.4.476
问题: MingW: compiling with "XPM=no" doesn't work.
解决方案: Check for the "no" value. (KF Leong) Also for Cygwin. (Ken
Takata)
相关文件: src/Make_ming.mak, src/Make_cyg.mak

Patch 7.4.477
问题: When using ":%diffput" and the other file is empty an extra empty
line remains.
解决方案: Set the buf_empty flag.
相关文件: src/diff.c

Patch 7.4.478
问题: Using byte length instead of character length for 'showbreak'.
解决方案: Compute the character length. (Marco Hinz)

相关文件: src/charset.c

Patch 7.4.479
问题: MS-Windows: The console title can be wrong.
解决方案: Take the encoding into account. When restoring the title use the right function. (Yasuhiro Matsumoto)
相关文件: src/os_mswin.c, src/os_win32.c

Patch 7.4.480 (after 7.4.479)
问题: MS-Windows: Can't build.
解决方案: Remove goto, use a flag instead.
相关文件: src/os_win32.c

Patch 7.4.481 (after 7.4.471)
问题: Compiler warning on MS-Windows.
解决方案: Add type casts. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.482
问题: When 'balloonexpr' results in a list, the text has a trailing newline. (Lcd)
解决方案: Remove one trailing newline.
相关文件: src/gui_beval.c

Patch 7.4.483
问题: A 0x80 byte is not handled correctly in abbreviations.
解决方案: Unescape special characters. Add a test. (Christian Brabandt)
相关文件: src/getchar.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_mapping.in, src/testdir/test_mapping.ok

Patch 7.4.484 (after 7.4.483)
问题: Compiler warning on MS-Windows. (Ken Takata)
解决方案: Add type cast.
相关文件: src/getchar.c

Patch 7.4.485 (after 7.4.484)
问题: Abbreviations don't work. (Toothpik)
解决方案: Move the length computation inside the for loop. Compare against the unescaped key.
相关文件: src/getchar.c

Patch 7.4.486
问题: Check for writing to a yank register is wrong.
解决方案: Negate the check. (Zyx). Also clean up the #ifdefs.
相关文件: src/ex_docmd.c, src/ex_cmds.h

Patch 7.4.487
问题: ":sign jump" may use another window even though the file is already edited in the current window.
解决方案: First check if the file is in the current window. (James McCoy)
相关文件: src/window.c, src/testdir/Make_amiga.mak,

src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_signs.in,
src/testdir/test_signs.ok

Patch 7.4.488
问题: test_mapping fails for some people.
解决方案: Set the 'encoding' option. (Ken Takata)
相关文件: src/testdir/test_mapping.in

Patch 7.4.489
问题: Cursor movement still wrong when 'lbr' is set and there is a
number column. (Hirohito Higashi)
解决方案: Add correction for number column. (Hiroyuki Takagi)
相关文件: src/charset.c

Patch 7.4.490
问题: Cannot specify the buffer to use for "do" and "dp", making them
useless for three-way diff.
解决方案: Use the count as the buffer number. (James McCoy)
相关文件: runtime/doc/diff.txt, src/diff.c, src/normal.c, src/proto/diff.pro

Patch 7.4.491
问题: When winrestview() has a negative "topline" value there are
display errors.
解决方案: Correct a negative value to 1. (Hirohito Higashi)
相关文件: src/eval.c

Patch 7.4.492
问题: In Insert mode, after inserting a newline that inserts a comment
leader, CTRL-O moves to the right. (ZyX) Issue 57.
解决方案: Correct the condition for moving the cursor back to the NUL.
(Christian Brabandt)
相关文件: src/edit.c, src/testdir/test4.in, src/testdir/test4.ok

Patch 7.4.493
问题: A TextChanged autocommand is triggered when saving a file.
(William Gardner)
解决方案: Update last_changedtick after calling unchanged(). (Christian
Brabandt)
相关文件: src/fileio.c

Patch 7.4.494
问题: Cursor shape is wrong after a CompleteDone autocommand.
解决方案: Update the cursor and mouse shape after ":normal" restores the
state. (Jacob Niehus)
相关文件: src/ex_docmd.c

Patch 7.4.495
问题: XPM isn't used correctly in the Cygwin Makefile.
解决方案: Include the rules like in Make_ming.mak. (Ken Takata)
相关文件: src/Make_cyg.mak

Patch 7.4.496

问题:　　Many lines are both in Make_cyg.mak and Make_ming.mak
解决方案:　Move the common parts to one file. (Ken Takata)
相关文件:　src/INSTALLpc.txt, src/Make_cyg.mak, src/Make_cyg_ming.mak,
src/Make_ming.mak, src/Make_mvc.mak, Filelist

Patch 7.4.497
问题:　　With some regexp patterns the NFA engine uses many states and
becomes very slow. To the user it looks like Vim freezes.
解决方案:　When the number of states reaches a limit fall back to the old
engine. (Christian Brabandt)
相关文件:　runtime/doc/options.txt, src/Makefile, src/regexp.c, src/regexp.h,
src/regexp_nfa.c, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Makefile, src/testdir/samples/re.freeze.txt,
src/testdir/bench_re_freeze.in, src/testdir/bench_re_freeze.vim,
Filelist

Patch 7.4.498 (after 7.4.497)
问题:　　Typo in DOS makefile.
解决方案:　Change exists to exist. (Ken Takata)
相关文件:　src/testdir/Make_dos.mak

Patch 7.4.499
问题:　　substitute() can be slow with long strings.
解决方案:　Store a pointer to the end, instead of calling strlen() every
time. (Ozaki Kiichi)
相关文件:　src/eval.c

Patch 7.4.500
问题:　　Test 72 still fails once in a while.
解决方案:　Don't set 'fileformat' to unix, reset it. (Ken Takata)
相关文件:　src/testdir/test72.in

Patch 7.4.501 (after 7.4.497)
问题:　　Typo in file pattern.
解决方案:　Insert a slash and remove a dot.
相关文件:　Filelist

Patch 7.4.502
问题:　　Language mapping also applies to mapped characters.
解决方案:　Add the 'langnoremap' option, when on 'langmap' does not apply to
mapped characters. (Christian Brabandt)
相关文件:　runtime/doc/options.txt, runtime/vimrc_example.vim, src/macros.h,
src/option.c, src/option.h

Patch 7.4.503
问题:　　Cannot append a list of lines to a file.
解决方案:　Add the append option to writefile(). (Yasuhiro Matsumoto)
相关文件:　runtime/doc/eval.txt, src/Makefile, src/eval.c,
src/testdir/test_writefile.in, src/testdir/test_writefile.ok

Patch 7.4.504
问题:　　Restriction of the MS-Windows installer that the path must end in
"Vim" prevents installing more than one version.

解决方案: Remove the restriction. (Tim Lebedkov)
相关文件: nsis/gvim.nsi

Patch 7.4.505
问题: On MS-Windows when 'encoding' is a double-byte encoding a file name longer than MAX_PATH bytes but shorter than that in characters causes problems.
解决方案: Fail on file names longer than MAX_PATH bytes. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.506
问题: MS-Windows: Cannot open a file with 259 characters.
解决方案: Fix off-by-one error. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.507 (after 7.4.496)
问题: Building with MingW and Perl.
解决方案: Remove quotes. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.508
问题: When generating ja.sjis.po the header is not correctly adjusted.
解决方案: Check for the right header string. (Ken Takata)
相关文件: src/po/sjiscorr.c

Patch 7.4.509
问题: Users are not aware their encryption is weak.
解决方案: Give a warning when prompting for the key.
相关文件: src/crypt.c, src/ex_docmd.c, src/fileio.c, src/main.c, src/proto/crypt.pro

Patch 7.4.510
问题: "-fwrapv" argument breaks use of cproto.
解决方案: Remove the alphabetic arguments in a drastic way.
相关文件: src/Makefile

Patch 7.4.511
问题: Generating proto for if_ruby.c uses type not defined elsewhere.
解决方案: Do not generate a prototype for rb_gc_writebarrier_unprotect_promoted()
相关文件: src/if_ruby.c

Patch 7.4.512
问题: Cannot generate prototypes for Win32 files and VMS.
解决方案: Add typedefs and #ifdef
相关文件: src/os_win32.c, src/gui_w32.c, src/os_vms.c

Patch 7.4.513
问题: Crash because reference count is wrong for list returned by getreg().
解决方案: Increment the reference count. (Kimmy Lindvall)
相关文件: src/eval.c

Patch 7.4.514 (after 7.4.492)

问题: Memory access error. (Dominique Pelle)
解决方案: Update tpos. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.4.515
问题: In a help buffer the global 'foldmethod' is used. (Paul Marshall)
解决方案: Reset 'foldmethod' when starting to edit a help file. Move the
            code to a separate function.
相关文件: src/ex_cmds.c

Patch 7.4.516
问题: Completing a function name containing a # does not work. Issue
            253.
解决方案: Recognize the # character. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.4.517
问题: With a wrapping line the cursor may not end up in the right place.
            (Nazri Ramliy)
解决方案: Adjust n_extra for a Tab that wraps. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.4.518
问题: Using status line height in width computations.
解决方案: Use one instead. (Hirohito Higashi)
相关文件: src/window.c

Patch 7.4.519 (after 7.4.497)
问题: Crash when using syntax highlighting.
解决方案: When regprog is freed and replaced, store the result.
相关文件: src/buffer.c, src/regexp.c, src/syntax.c, src/spell.c,
            src/ex_cmds2.c, src/fileio.c, src/proto/fileio.pro,
            src/proto/regexp.pro, src/os_unix.c

Patch 7.4.520
问题: Sun PCK locale is not recognized.
解决方案: Add PCK in the table. (Keiichi Oono)
相关文件: src/mbyte.c

Patch 7.4.521
问题: When using "vep" a mark is moved to the next line. (Maxi Padulo,
            Issue 283)
解决方案: Decrement the line number. (Christian Brabandt)
相关文件: src/ops.c

Patch 7.4.522
问题: Specifying wrong buffer size for GetLongPathName().
解决方案: Use the actual size. (Ken Takata)
相关文件: src/eval.c

Patch 7.4.523
问题: When the X11 server is stopped and restarted, while Vim is kept in
            the background, copy/paste no longer works. (Issue 203)
解决方案: Setup the clipboard again. (Christian Brabandt)

相关文件: src/os_unix.c

Patch 7.4.524
问题: When using ":ownsyntax" spell checking is messed up. (Issue 78)
解决方案: Use the window-local option values. (Christian Brabandt)
相关文件: src/option.c, src/syntax.c

Patch 7.4.525
问题: map() leaks memory when there is an error in the expression.
解决方案: Call clear_tv(). (Christian Brabandt)
相关文件: src/eval.c

Patch 7.4.526
问题: matchstr() fails on long text. (Daniel Hahler)
解决方案: Return NFA_TOO_EXPENSIVE from regexec_nl(). (Christian Brabandt)
相关文件: src/regexp.c

Patch 7.4.527
问题: Still confusing regexp failure and NFA_TOO_EXPENSIVE.
解决方案: NFA changes equivalent of 7.4.526.
相关文件: src/regexp_nfa.c

Patch 7.4.528
问题: Crash when using matchadd() (Yasuhiro Matsumoto)
解决方案: Copy the match regprog.
相关文件: src/screen.c

Patch 7.4.529
问题: No test for what 7.4.517 fixes.
解决方案: Adjust the tests for breakindent. (Christian Brabandt)
相关文件: src/testdir/test_breakindent.in, src/testdir/test_breakindent.ok

Patch 7.4.530
问题: Many commands take a count or range that is not using line
numbers.
解决方案: For each command specify what kind of count it uses. For windows,
buffers and arguments have "$" and "." have a relevant meaning.
(Marcin Szamotulski)
相关文件: runtime/doc/editing.txt, runtime/doc/tabpage.txt,
runtime/doc/windows.txt, src/Makefile, src/ex_cmds.h,
src/ex_docmd.c, src/testdir/Make_amiga.mak
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_argument_count.in,
src/testdir/test_argument_count.ok,
src/testdir/test_close_count.in, src/testdir/test_close_count.ok,
src/window.c

Patch 7.4.531
问题: Comments about parsing an Ex command are wrong.
解决方案: Correct the step numbers.
相关文件: src/ex_docmd.c

Patch 7.4.532

问题:　　When using 'incsearch' "2/pattern/e" highlights the first match.
解决方案:　Move the code to set extra_col inside the loop for count. (Ozaki Kiichi)
相关文件:　src/search.c

Patch 7.4.533
问题:　　":hardcopy" leaks memory in case of errors.
解决方案:　Free memory in all code paths. (Christian Brabandt)
相关文件:　src/hardcopy.c

Patch 7.4.534
问题:　　Warnings when compiling if_ruby.c.
解决方案:　Avoid the warnings. (Ken Takata)
相关文件:　src/if_ruby.c

Patch 7.4.535 (after 7.4.530)
问题:　　Can't build with tiny features.
解决方案:　Add #ifdefs and skip a test.
相关文件:　src/ex_docmd.c, src/testdir/test_argument_count.in

Patch 7.4.536
问题:　　Test 63 fails when using a black&white terminal.
解决方案:　Add attributes for a non-color terminal. (Christian Brabandt)
相关文件:　src/testdir/test63.in

Patch 7.4.537
问题:　　Value of v:hlsearch reflects an internal variable.
解决方案:　Make the value reflect whether search highlighting is actually displayed. (Christian Brabandt)
相关文件:　runtime/doc/eval.txt, src/testdir/test101.in, src/testdir/test101.ok, src/vim.h

Patch 7.4.538
问题:　　Tests fail with small features plus Python.
解决方案:　Disallow weird combination of options. Do not set "fdm" when folding is disabled.
相关文件:　src/option.c, src/ex_cmds.c, src/configure.in, src/auto/configure, src/feature.h

Patch 7.4.539 (after 7.4.530)
问题:　　Crash when computing buffer count. Problem with range for user commands. Line range wrong in Visual area.
解决方案:　Avoid segfault in compute_buffer_local_count(). Check for CMD_USER when checking type of range. (Marcin Szamotulski)
相关文件:　runtime/doc/windows.txt, src/ex_docmd.c

Patch 7.4.540 (after 7.4.539)
问题:　　Cannot build with tiny and small features. (Taro Muraoka)
解决方案:　Add #ifdef around CMD_USER.
相关文件:　src/ex_docmd.c

Patch 7.4.541
问题:　　Crash when doing a range assign.
解决方案:　Check for NULL pointer. (Yukihiro Nakadaira)

相关文件: src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.542
问题: Using a range for window and buffer commands has a few problems.
Cannot specify the type of range for a user command.
解决方案: Add the -addr argument for user commands. Fix problems. (Marcin
Szamotulski)
相关文件: src/testdir/test_command_count.in,
src/testdir/test_command_count.ok src/testdir/Make_amiga.mak
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, runtime/doc/map.txt, src/Makefile,
src/ex_cmds.h, src/ex_docmd.c, src/ex_getln.c,
src/proto/ex_docmd.pro, src/vim.h,

Patch 7.4.543
问题: Since patch 7.4.232 "1,3s/\n//" joins two lines instead of three.
(Eliseo Martínez) Issue 287
解决方案: Correct the line count. (Christian Brabandt)
Also set the last used search pattern.
相关文件: src/ex_cmds.c, src/search.c, src/proto/search.pro

Patch 7.4.544
问题: Warnings for unused arguments when compiling with a combination of
features.
解决方案: Add "UNUSED".
相关文件: src/if_cscope.c

Patch 7.4.545
问题: Highlighting for multi-line matches is not correct.
解决方案: Stop highlight at the end of the match. (Hirohito Higashi)
相关文件: src/screen.c

Patch 7.4.546
问题: Repeated use of vim_snprintf() with a number.
解决方案: Move these vim_snprintf() calls into a function.
相关文件: src/window.c

Patch 7.4.547
问题: Using "vit" does not select a multi-byte character at the end
correctly.
解决方案: Advance the cursor over the multi-byte character. (Christian
Brabandt)
相关文件: src/search.c

Patch 7.4.548
问题: Compilation fails with native version of MinGW-w64, because
it doesn't have x86_64-w64-mingw32-windres.exe.
解决方案: Use windres instead. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.549
问题: Function name not recognized correctly when inside a function.
解决方案: Don't check for an alpha character. (Ozaki Kiichi)

相关文件: src/eval.c, src/testdir/test_nested_function.in,
           src/testdir/test_nested_function.ok, src/testdir/Make_amiga.mak,
           src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
           src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
           src/testdir/Makefile

Patch 7.4.550
问题: curs_rows() function is always called with the second argument
           false.
解决方案: Remove the argument. (Christian Brabandt)
           validate_botline_win() can then also be removed.
相关文件: src/move.c

Patch 7.4.551
问题: "ygn" may yank too much. (Fritzophrenic) Issue 295.
解决方案: Check the width of the next match. (Christian Brabandt)
相关文件: src/search.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.4.552
问题: Langmap applies to Insert mode expression mappings.
解决方案: Check for Insert mode. (Daniel Hahler)
相关文件: src/getchar.c, src/testdir/test_mapping.in,
           src/testdir/test_mapping.ok

Patch 7.4.553
问题: Various small issues.
解决方案: Fix those issues.
相关文件: src/ex_cmds.h, src/gui.h, src/message.c, src/testdir/test39.in,
           src/proto/eval.pro, src/proto/misc1.pro, src/proto/ops.pro,
           src/proto/screen.pro, src/proto/window.pro. src/os_unix.c,
           src/Make_vms.mms, src/proto/os_vms.pro, src/INSTALL

Patch 7.4.554
问题: Missing part of patch 7.4.519.
解决方案: Copy back regprog after calling vim_regexec.
相关文件: src/quickfix.c

Patch 7.4.555
问题: test_close_count may fail for some combination of features.
解决方案: Require normal features.
相关文件: src/testdir/test_close_count.in

Patch 7.4.556
问题: Failed commands in Python interface not handled correctly.
解决方案: Restore window and buffer on failure.
相关文件: src/if_py_both.h

Patch 7.4.557
问题: One more small issue.
解决方案: Update function proto.
相关文件: src/proto/window.pro

Patch 7.4.558
问题: When the X server restarts Vim may get stuck.

解决方案: Destroy the application context and create it again. (Issue 203)
相关文件: src/os_unix.c

Patch 7.4.559
问题: Appending a block in the middle of a tab does not work correctly
when virtualedit is set.
解决方案: Decrement spaces and count, don't reset them. (James McCoy)
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.560
问题: Memory leak using :wviminfo. Issue 296.
解决方案: Free memory when needed. (idea by Christian Brabandt)
相关文件: src/ops.c

Patch 7.4.561
问题: Ex range handling is wrong for buffer-local user commands.
解决方案: Check for CMD_USER_BUF. (Marcin Szamotulski)
相关文件: src/ex_docmd.c, src/testdir/test_command_count.in,
src/testdir/test_command_count.ok

Patch 7.4.562
问题: Segfault with wide screen and error in 'rulerformat'. (Ingo Karkat)
解决方案: Check there is enough space. (Christian Brabandt)
相关文件: src/buffer.c, src/screen.c

Patch 7.4.563
问题: No test for replacing on a tab in Virtual replace mode.
解决方案: Add a test. (Elias Diem)
相关文件: src/testdir/test48.in, src/testdir/test48.ok

Patch 7.4.564
问题: FEAT_OSFILETYPE is used even though it's never defined.
解决方案: Remove the code. (Christian Brabandt)
相关文件: src/fileio.c

Patch 7.4.565
问题: Ranges for arguments, buffers, tabs, etc. are not checked to be
valid but limited to the maximum. This can cause the wrong thing
to happen.
解决方案: Give an error for an invalid value. (Marcin Szamotulski)
Use windows range for ":wincmd".
相关文件: src/ex_docmd.c, src/ex_cmds.h, src/testdir/test62.in,
src/testdir/test_argument_count.in,
src/testdir/test_argument_count.ok,
src/testdir/test_close_count.in,
src/testdir/test_command_count.in,
src/testdir/test_command_count.ok

Patch 7.4.566
问题: :argdo, :bufdo, :windo and :tabdo don't take a range.
解决方案: Support the range. (Marcin Szamotulski)
相关文件: runtime/doc/editing.txt, runtime/doc/tabpage.txt,
runtime/doc/windows.txt, src/ex_cmds.h, src/ex_cmds2.c,
src/testdir/test_command_count.in,

src/testdir/test_command_count.ok

Patch 7.4.567
问题: Non-ascii vertical separator characters are always redrawn.
解决方案: Compare only the one byte that's stored. (Thiago Padilha)
相关文件: src/screen.c

Patch 7.4.568
问题: Giving an error for ":0wincmd w" is a problem for some plugins.
解决方案: Allow the zero in the range. (Marcin Szamotulski)
相关文件: src/ex_docmd.c, src/testdir/test_command_count.ok

Patch 7.4.569 (after 7.4.468)
问题: Having CTRL-C interrupt or not does not check the mode of the mapping. (Ingo Karkat)
解决方案: Use a bitmask with the map mode. (Christian Brabandt)
相关文件: src/getchar.c, src/structs.h, src/testdir/test_mapping.in, src/testdir/test_mapping.ok, src/ui.c, src/globals.h

Patch 7.4.570
问题: Building with dynamic library does not work for Ruby 2.2.0
解决方案: Change #ifdefs and #defines. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.4.571 (after 7.4.569)
问题: Can't build with tiny features. (Ike Devolder)
解决方案: Add #ifdef.
相关文件: src/getchar.c

Patch 7.4.572
问题: Address type of :wincmd depends on the argument.
解决方案: Check the argument.
相关文件: src/ex_docmd.c, src/window.c, src/proto/window.pro

Patch 7.4.573 (after 7.4.569)
问题: Mapping CTRL-C in Visual mode doesn't work. (Ingo Karkat)
解决方案: Call get_real_state() instead of using State directly.
相关文件: src/ui.c, src/testdir/test_mapping.in, src/testdir/test_mapping.ok

Patch 7.4.574
问题: No error for eval('$').
解决方案: Check for empty name. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.575
问题: Unicode character properties are outdated.
解决方案: Update the tables with the latest version.
相关文件: src/mbyte.c

Patch 7.4.576
问题: Redrawing problem with 'relativenumber' and 'linebreak'.
解决方案: Temporarily reset 'linebreak' and restore it in more places. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.4.577
问题: Matching with a virtual column has a lot of overhead on very long lines. (Issue 310)
解决方案: Bail out early if there can't be a match. (Christian Brabandt) Also check for CTRL-C at every position.
相关文件: src/regexp_nfa.c

Patch 7.4.578
问题: Using getcurpos() after "$" in an empty line returns a negative number.
解决方案: Don't add one when this would overflow. (Hirohito Higashi)
相关文件: src/eval.c

Patch 7.4.579
问题: Wrong cursor positioning when 'linebreak' is set and lines wrap.
解决方案: Fix it. (Christian Brabandt)
相关文件: src/charset.c, src/screen.c

Patch 7.4.580
问题: ":52wincmd v" still gives an invalid range error. (Charles Campbell)
解决方案: Skip over white space.
相关文件: src/ex_docmd.c

Patch 7.4.581
问题: Compiler warnings for uninitialized variables. (John Little)
解决方案: Initialize the variables.
相关文件: src/ops.c

Patch 7.4.582 (after 7.4.577)
问题: Can't match "%>80v" properly. (Axel Bender)
解决方案: Correctly handle ">". (Christian Brabandt)
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.583
问题: With tiny features test 16 may fail.
解决方案: Source small.vim. (Christian Brabandt)
相关文件: src/testdir/test16.in

Patch 7.4.584
问题: With tiny features test_command_count may fail.
解决方案: Source small.vim. (Christian Brabandt)
相关文件: src/testdir/test_command_count.in

Patch 7.4.585
问题: Range for :bdelete does not work. (Ronald Schild)
解决方案: Also allow unloaded buffers.
相关文件: src/ex_cmds.h, src/testdir/test_command_count.in, src/testdir/test_command_count.ok

Patch 7.4.586
问题: Parallel building of the documentation html files is not reliable.
解决方案: Remove a cyclic dependency. (Reiner Herrmann)

相关文件: runtime/doc/Makefile

Patch 7.4.587
问题: Conceal does not work properly with 'linebreak'. (cs86661)
解决方案: Save and restore boguscols. (Christian Brabandt)
相关文件: src/screen.c, src/testdir/test_listlbr_utf8.in,
src/testdir/test_listlbr_utf8.ok

Patch 7.4.588
问题: ":0argedit foo" puts the new argument in the second place instead
of the first.
解决方案: Adjust the range type. (Ingo Karkat)
相关文件: src/ex_cmds.h, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_argument_0count.in,
src/testdir/test_argument_0count.ok

Patch 7.4.589
问题: In the MS-Windows console Vim can't handle greek characters when
encoding is utf-8.
解决方案: Escape K_NUL. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.4.590
问题: Using ctrl_x_mode as if it contains flags.
解决方案: Don't use AND with CTRL_X_OMNI. (Hirohito Higashi)
相关文件: src/edit.c

Patch 7.4.591 (after 7.4.587)
问题: test_listlbr_utf8 fails when the conceal feature is not available.
解决方案: Check for the conceal feature. (Kazunobu Kuriyama)
相关文件: src/testdir/test_listlbr_utf8.in

Patch 7.4.592
问题: When doing ":e foobar" when already editing "foobar" and 'buftype'
is "nofile" the buffer is cleared. (Xavier de Gaye)
解决方案: Do no clear the buffer.
相关文件: src/ex_cmds.c

Patch 7.4.593
问题: Crash when searching for "x\{0,90000}". (Dominique Pelle)
解决方案: Bail out from the NFA engine when the max limit is much higher
than the min limit.
相关文件: src/regexp_nfa.c, src/regexp.c, src/vim.h

Patch 7.4.594
问题: Using a block delete while 'breakindent' is set does not work
properly.
解决方案: Use "line" instead of "prev_pend" as the first argument to
lbr_chartabsize_adv(). (Hirohito Higashi)
相关文件: src/ops.c, src/testdir/test_breakindent.in,
src/testdir/test_breakindent.ok

Patch 7.4.595
问题:　　The test_command_count test fails when using Japanese.
解决方案:　Force the language to C. (Hirohito Higashi)
相关文件:　src/testdir/test_command_count.in

Patch 7.4.596 (after 7.4.592)
问题:　　Tiny build doesn't compile. (Ike Devolder)
解决方案:　Add #ifdef.
相关文件:　src/ex_cmds.c

Patch 7.4.597
问题:　　Cannot change the result of systemlist().
解决方案:　Initialize v_lock. (Yukihiro Nakadaira)
相关文件:　src/eval.c

Patch 7.4.598
问题:　　":tabdo windo echo 'hi'" causes "* register not to be changed.
　　(Salman Halim)
解决方案:　Change how clip_did_set_selection is used and add
　　clipboard_needs_update and global_change_count. (Christian
　　Brabandt)
相关文件:　src/main.c, src/ui.c, src/testdir/test_eval.in,
　　src/testdir/test_eval.ok

Patch 7.4.599
问题:　　Out-of-memory error.
解决方案:　Avoid trying to allocate a negative amount of memory, use size_t
　　instead of int. (Dominique Pelle)
相关文件:　src/regexp_nfa.c

Patch 7.4.600
问题:　　Memory wasted in struct because of aligning.
解决方案:　Split pos in lnum and col. (Dominique Pelle)
相关文件:　src/regexp_nfa.c

Patch 7.4.601
问题:　　It is not possible to have feedkeys() insert characters.
解决方案:　Add the 'i' flag.
相关文件:　src/eval.c, runtime/doc/eval.txt

Patch 7.4.602
问题:　　":set" does not accept hex numbers as documented.
解决方案:　Use vim_str2nr(). (ZyX)
相关文件:　src/option.c, runtime/doc/options.txt

Patch 7.4.603
问题:　　'foldcolumn' may be set such that it fills the whole window, not
　　leaving space for text.
解决方案:　Reduce the foldcolumn width when there is not sufficient room.
　　(idea by Christian Brabandt)
相关文件:　src/screen.c

Patch 7.4.604
问题:　　Running tests changes viminfo.

解决方案: Disable viminfo.
相关文件: src/testdir/test_breakindent.in

Patch 7.4.605
问题: The # register is not writable, it cannot be restored after
           jumping around.
解决方案: Make the # register writable. (Marcin Szamotulski)
相关文件: runtime/doc/change.txt, src/ops.c, src/buffer.c, src/globals.h

Patch 7.4.606
问题: May crash when using a small window.
解决方案: Avoid dividing by zero. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.4.607 (after 7.4.598)
问题: Compiler warnings for unused variables.
解决方案: Move them inside #ifdef. (Kazunobu Kuriyama)
相关文件: src/ui.c

Patch 7.4.608 (after 7.4.598)
问题: test_eval fails when the clipboard feature is missing.
解决方案: Skip part of the test. Reduce the text used.
相关文件: src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.609
问题: For complicated list and dict use the garbage collector can run
           out of stack space.
解决方案: Use a stack of dicts and lists to be marked, thus making it
           iterative instead of recursive. (Ben Fritz)
相关文件: src/eval.c, src/if_lua.c, src/if_py_both.h, src/if_python.c,
           src/if_python3.c, src/proto/eval.pro, src/proto/if_lua.pro,
           src/proto/if_python.pro, src/proto/if_python3.pro, src/structs.h

Patch 7.4.610
问题: Some function headers may be missing from generated .pro files.
解决方案: Add PROTO to the #ifdef.
相关文件: src/option.c, src/syntax.c

Patch 7.4.611 (after 7.4.609)
问题: Syntax error.
解决方案: Change statement to return.
相关文件: src/if_python3.c

Patch 7.4.612
问题: test_eval fails on Mac.
解决方案: Use the * register instead of the + register. (Jun Takimoto)
相关文件: src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.613
问题: The NFA engine does not implement the 'redrawtime' time limit.
解决方案: Implement the time limit.
相关文件: src/regexp_nfa.c

Patch 7.4.614

问题: There is no test for what patch 7.4.601 fixes.
解决方案: Add a test. (Christian Brabandt)
相关文件: src/testdir/test_mapping.in, src/testdir/test_mapping.ok

Patch 7.4.615
问题: Vim hangs when freeing a lot of objects.
解决方案: Do not go back to the start of the list every time. (Yasuhiro
            Matsumoto and Ariya Mizutani)
相关文件: src/eval.c

Patch 7.4.616
问题: Cannot insert a tab in front of a block.
解决方案: Correctly compute aop->start. (Christian Brabandt)
相关文件: src/ops.c, src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.617
问题: Wrong ":argdo" range does not cause an error.
解决方案: Reset "cmd" to NULL. (Marcin Szamotulski, Ingo Karkat)
相关文件: src/ex_docmd.c

Patch 7.4.618 (after 7.4.609)
问题: luaV_setref() is missing a return statement. (Ozaki Kiichi)
解决方案: Put the return statement back.
相关文件: src/if_lua.c

Patch 7.4.619 (after 7.4.618)
问题: luaV_setref() not returning the correct value.
解决方案: Return one.
相关文件: src/if_lua.c

Patch 7.4.620
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Initialize "did_free". (Ben Fritz)
相关文件: src/eval.c

Patch 7.4.621 (after 7.4.619)
问题: Returning 1 in the wrong function. (Raymond Ko)
解决方案: Return 1 in the right function (hopefully).
相关文件: src/if_lua.c

Patch 7.4.622
问题: Compiler warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/regexp_nfa.c

Patch 7.4.623
问题: Crash with pattern: \(\)\{80000} (Dominique Pelle)
解决方案: When the max limit is large fall back to the old engine.
相关文件: src/regexp_nfa.c

Patch 7.4.624
问题: May leak memory or crash when vim_realloc() returns NULL.
解决方案: Handle a NULL value properly. (Mike Williams)
相关文件: src/if_cscope.c, src/memline.c, src/misc1.c, src/netbeans.c

Patch 7.4.625
问题:　　Possible NULL pointer dereference.
解决方案:　Check for NULL before using it. (Mike Williams)
相关文件:　src/if_py_both.h

Patch 7.4.626
问题:　　MSVC with W4 gives useless warnings.
解决方案:　Disable more warnings. (Mike Williams)
相关文件:　src/vim.h

Patch 7.4.627
问题:　　The last screen cell is not updated.
解决方案:　Respect the "tn" termcap feature. (Hayaki Saito)
相关文件:　runtime/doc/term.txt, src/option.c, src/screen.c, src/term.c,
　　src/term.h

Patch 7.4.628
问题:　　Compiler warning for variable might be clobbered by longjmp.
解决方案:　Add volatile. (Michael Jarvis)
相关文件:　src/main.c

Patch 7.4.629
问题:　　Coverity warning for Out-of-bounds read.
解决方案:　Increase MAXWLEN to 254. (Eliseo Martínez)
相关文件:　src/spell.c

Patch 7.4.630
问题:　　When using Insert mode completion combined with autocommands the
　　redo command may not work.
解决方案:　Do not save the redo buffer when executing autocommands. (Yasuhiro
　　Matsumoto)
相关文件:　src/fileio.c

Patch 7.4.631
问题:　　The default conceal character is documented to be a space but it's
　　initially a dash. (Christian Brabandt)
解决方案:　Make the initial value a space.
相关文件:　src/globals.h

Patch 7.4.632 (after 7.4.592)
问题:　　7.4.592 breaks the netrw plugin, because the autocommands are
　　skipped.
解决方案:　Roll back the change.
相关文件:　src/ex_cmds.c

Patch 7.4.633
问题:　　After 7.4.630 the problem persists.
解决方案:　Also skip redo when calling a user function.
相关文件:　src/eval.c

Patch 7.4.634
问题:　　Marks are not restored after redo + undo.
解决方案:　Fix the way marks are restored. (Olaf Dabrunz)

相关文件: src/undo.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
           src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
           src/testdir/Make_vms.mms, src/testdir/Makefile,
           src/testdir/test_marks.in, src/testdir/test_marks.ok

Patch 7.4.635
问题: If no NL or CR is found in the first block of a file then the
           'fileformat' may be set to "mac". (Issue 77)
解决方案: Check if a CR was found. (eswald)
相关文件: src/fileio.c

Patch 7.4.636
问题: A search with end offset gets stuck at end of file. (Gary Johnson)
解决方案: When a search doesn't move the cursor repeat it with a higher
           count. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test44.in, src/testdir/test44.ok

Patch 7.4.637
问题: Incorrectly read the number of buffer for which an autocommand
           should be registered.
解决方案: Reverse check for "<buffer=abuf>". (Lech Lorens)
相关文件: src/fileio.c

Patch 7.4.638
问题: Can't build with Lua 5.3 on Windows.
解决方案: use luaL_optinteger() instead of LuaL_optlong(). (Ken Takata)
相关文件: src/if_lua.c

Patch 7.4.639
问题: Combination of linebreak and conceal doesn't work well.
解决方案: Fix the display problems. (Christian Brabandt)
相关文件: src/screen.c, src/testdir/test88.in, src/testdir/test88.ok,
           src/testdir/test_listlbr_utf8.in, src/testdir/test_listlbr_utf8.ok

Patch 7.4.640
问题: After deleting characters in Insert mode such that lines are
           joined undo does not work properly. (issue 324)
解决方案: Use Insstart instead of Insstart_orig. (Christian Brabandt)
相关文件: src/edit.c

Patch 7.4.641
问题: The tabline menu was using ":999tabnew" which is now invalid.
解决方案: Use ":$tabnew" instead. (Florian Degner)
相关文件: src/normal.c

Patch 7.4.642
问题: When using "gf" escaped spaces are not handled.
解决方案: Recognize escaped spaces.
相关文件: src/vim.h, src/window.c, src/misc2.c

Patch 7.4.643
问题: Using the default file format for Mac files. (Issue 77)
解决方案: Reset the try_mac counter in the right place. (Oswald)
相关文件: src/fileio.c, src/testdir/test30.in, src/testdir/test30.ok

Patch 7.4.644
问题: Stratus VOS doesn't have sync().
解决方案: Use fflush(). (Karli Aurelia)
相关文件: src/memfile.c

Patch 7.4.645
问题: When splitting the window in a BufAdd autocommand while still in the first, empty buffer the window count is wrong.
解决方案: Do not reset b_nwindows to zero and don't increment it.
相关文件: src/buffer.c, src/ex_cmds.c

Patch 7.4.646
问题: ":bufdo" may start at a deleted buffer.
解决方案: Find the first not deleted buffer. (Shane Harper)
相关文件: src/ex_cmds2.c, src/testdir/test_command_count.in, src/testdir/test_command_count.ok

Patch 7.4.647
问题: After running the tests on MS-Windows many files differ from their originals as they were checked out.
解决方案: Use a temp directory for executing the tests. (Ken Takata, Taro Muraoka)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.648 (after 7.4.647)
问题: Tests broken on MS-Windows.
解决方案: Delete wrong copy line. (Ken Takata)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.649
问题: Compiler complains about ignoring return value of fwrite(). (Michael Jarvis)
解决方案: Add (void).
相关文件: src/misc2.c

Patch 7.4.650
问题: Configure check may fail because the dl library is not used.
解决方案: Put "-ldl" in LIBS rather than LDFLAGS. (Ozaki Kiichi)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.651 (after 7.4.582)
问题: Can't match "%>80v" properly for multi-byte characters.
解决方案: Multiply the character number by the maximum number of bytes in a character. (Yasuhiro Matsumoto)
相关文件: src/regexp_nfa.c

Patch 7.4.652
问题: Xxd lacks a few features.
解决方案: Use 8 characters for the file position. Add the -e and -o arguments. (Vadim Vygonets)
相关文件: src/xxd/xxd.c, runtime/doc/xxd.1

Patch 7.4.653

问题:　　Insert mode completion with complete() may have CTRL-L work like CTRL-P.
解决方案:　Handle completion with complete() differently. (Yasuhiro Matsumoto, Christian Brabandt, Hirohito Higashi)
相关文件:　src/edit.c

Patch 7.4.654
问题:　　glob() and globpath() cannot include links to non-existing files. (Charles Campbell)
解决方案:　Add an argument to include all links with glob(). (James McCoy) Also for globpath().
相关文件:　src/vim.h, src/eval.c, src/ex_getln.c

Patch 7.4.655
问题:　　Text deleted by "dit" depends on indent of closing tag. (Jan Parthey)
解决方案:　Do not adjust oap->end in do_pending_operator(). (Christian Brabandt)
相关文件:　src/normal.c, src/search.c, src/testdir/test53.in, src/testdir/test53.ok

Patch 7.4.656 (after 7.4.654)
问题:　　Missing changes for glob() in one file.
解决方案:　Add the missing changes.
相关文件:　src/misc1.c

Patch 7.4.657 (after 7.4.656)
问题:　　Compiler warnings for pointer mismatch.
解决方案:　Add a typecast. (John Marriott)
相关文件:　src/misc1.c

Patch 7.4.658
问题:　　'formatexpr' is evaluated too often.
解决方案:　Only invoke it when beyond the 'textwidth' column, as it is documented. (James McCoy)
相关文件:　src/edit.c

Patch 7.4.659
问题:　　When 'ruler' is set the preferred column is reset. (Issue 339)
解决方案:　Don't set curswant when redrawing the status lines.
相关文件:　src/option.c

Patch 7.4.660
问题:　　Using freed memory when g:colors_name is changed in the colors script. (oni-link)
解决方案:　Make a copy of the variable value.
相关文件:　src/syntax.c

Patch 7.4.661
问题:　　Using "0 CTRL-D" in Insert mode may have CursorHoldI interfere. (Gary Johnson)
解决方案:　Don't store K_CURSORHOLD as the last character. (Christian Brabandt)
相关文件:　src/edit.c

Patch 7.4.662
问题: When 'M' is in the 'cpo' option then selecting a text object in parentheses does not work correctly.
解决方案: Keep 'M' in 'cpo' when finding a match. (Hirohito Higashi)
相关文件: src/search.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_textobjects.in, src/testdir/test_textobjects.ok

Patch 7.4.663
问题: When using netbeans a buffer is not found in another tab.
解决方案: When 'switchbuf' is set to "usetab" then switch to another tab when possible. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.4.664
问题: When 'compatible' is reset 'numberwidth' is set to 4, but the effect doesn't show until a change is made.
解决方案: Check if 'numberwidth' changed. (Christian Brabandt)
相关文件: src/screen.c, src/structs.h

Patch 7.4.665
问题: 'linebreak' does not work properly with multi-byte characters.
解决方案: Compute the pointer offset with mb_head_off(). (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 7.4.666
问题: There is a chance that Vim may lock up.
解决方案: Handle timer events differently. (Aaron Burrow)
相关文件: src/os_unix.c

Patch 7.4.667
问题: 'colorcolumn' isn't drawn in a closed fold while 'cursorcolumn' is. (Carlos Pita)
解决方案: Make it consistent. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.4.668
问题: Can't use a glob pattern as a regexp pattern.
解决方案: Add glob2regpat(). (Christian Brabandt)
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 7.4.669
问题: When netbeans is active the sign column always shows up.
解决方案: Only show the sign column once a sign has been added. (Xavier de Gaye)
相关文件: src/buffer.c, src/edit.c, src/move.c, src/netbeans.c, src/screen.c, src/structs.h

Patch 7.4.670
问题: Using 'cindent' for Javascript is less than perfect.

解决方案: Improve indenting of continuation lines. (Hirohito Higashi)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.671 (after 7.4.665)
问题: Warning for shadowing a variable.
解决方案: Rename off to mb_off. (Kazunobu Kuriyama)
相关文件: src/screen.c

Patch 7.4.672
问题: When completing a shell command, directories in the current
           directory are not listed.
解决方案: When "." is not in $PATH also look in the current directory for
           directories.
相关文件: src/ex_getln.c, src/vim.h, src/misc1.c, src/eval.c,
           src/os_amiga.c, src/os_msdos.c, src/os_unix.c, src/os_vms.c,
           src/proto/os_amiga.pro, src/proto/os_msdos.pro,
           src/proto/os_unix.pro, src/proto/os_win32.pro

Patch 7.4.673
问题: The first syntax entry gets sequence number zero, which doesn't
           work. (Clinton McKay)
解决方案: Start at number one. (Bjorn Linse)
相关文件: src/syntax.c

Patch 7.4.674 (after 7.4.672)
问题: Missing changes in one file.
解决方案: Also change the win32 file.
相关文件: src/os_win32.c

Patch 7.4.675
问题: When a FileReadPost autocommand moves the cursor inside a line it
           gets moved back.
解决方案: When checking whether an autocommand moved the cursor store the
           column as well. (Christian Brabandt)
相关文件: src/ex_cmds.c

Patch 7.4.676
问题: On Mac, when not using the default Python framework configure
           doesn't do the right thing.
解决方案: Use a linker search path. (Kazunobu Kuriyama)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.677 (after 7.4.676)
问题: Configure fails when specifying a python-config-dir. (Lcd)
解决方案: Check if PYTHONFRAMEWORKPREFIX is set.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.678
问题: When using --remote the directory may end up being wrong.
解决方案: Use localdir() to find out what to do. (Xaizek)
相关文件: src/main.c

Patch 7.4.679
问题: Color values greater than 255 cause problems on MS-Windows.

解决方案: Truncate to 255 colors. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.4.680
问题: CTRL-W in Insert mode does not work well for multi-byte
characters.
解决方案: Use mb_get_class(). (Yasuhiro Matsumoto)
相关文件: src/edit.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test_erasebackword.in,
src/testdir/test_erasebackword.ok,

Patch 7.4.681
问题: MS-Windows: When Vim is minimized the window height is computed
incorrectly.
解决方案: When minimized use the previously computed size. (Ingo Karkat)
相关文件: src/gui_w32.c

Patch 7.4.682
问题: The search highlighting and match highlighting replaces the
cursorline highlighting, this doesn't look good.
解决方案: Combine the highlighting. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 7.4.683
问题: Typo in the vimtutor command.
解决方案: Fix the typo. (Corey Farwell, github pull 349)
相关文件: vimtutor.com

Patch 7.4.684
问题: When starting several Vim instances in diff mode, the temp files
used may not be unique. (Issue 353)
解决方案: Add an argument to vim_tempname() to keep the file.
相关文件: src/diff.c, src/eval.c, src/ex_cmds.c, src/fileio.c,
src/hardcopy.c, src/proto/fileio.pro, src/if_cscope.c,
src/memline.c, src/misc1.c, src/os_unix.c, src/quickfix.c,
src/spell.c

Patch 7.4.685
问题: When there are illegal utf-8 characters the old regexp engine may
go past the end of a string.
解决方案: Only advance to the end of the string. (Dominique Pelle)
相关文件: src/regexp.c

Patch 7.4.686
问题: "zr" and "zm" do not take a count.
解决方案: Implement the count, restrict the fold level to the maximum
nesting depth. (Marcin Szamotulski)
相关文件: runtime/doc/fold.txt, src/normal.c

Patch 7.4.687
问题: There is no way to use a different in Replace mode for a terminal.
解决方案: Add t_SR. (Omar Sandoval)

相关文件: runtime/doc/options.txt, runtime/doc/term.txt,
           runtime/syntax/vim.vim, src/option.c, src/term.c, src/term.h

Patch 7.4.688
问题: When "$" is in 'cpo' the popup menu isn't undrawn correctly.
           (Issue 166)
解决方案: When using the popup menu remove the "$".
相关文件: src/edit.c

Patch 7.4.689
问题: On MS-Windows, when 'autochdir' is set, diff mode with files in
           different directories does not work. (Axel Bender)
解决方案: Remember the current directory and use it where needed. (Christian
           Brabandt)
相关文件: src/main.c

Patch 7.4.690
问题: Memory access errors when changing indent in Ex mode. Also missing
           redraw when using CTRL-U. (Knil Ino)
解决方案: Update pointers after calling ga_grow().
相关文件: src/ex_getln.c

Patch 7.4.691 (after 7.4.689)
问题: Can't build with MzScheme.
解决方案: Change "cwd" into the global variable "start_dir".
相关文件: src/main.c

Patch 7.4.692
问题: Defining SOLARIS for no good reason. (Danek Duvall)
解决方案: Remove it.
相关文件: src/os_unix.h

Patch 7.4.693
问题: Session file is not correct when there are multiple tab pages.
解决方案: Reset the current window number for each tab page. (Jacob Niehus)
相关文件: src/ex_docmd.c

Patch 7.4.694
问题: Running tests changes the .viminfo file.
解决方案: Disable viminfo in the text objects test.
相关文件: src/testdir/test_textobjects.in

Patch 7.4.695
问题: Out-of-bounds read, detected by Coverity.
解决方案: Remember the value of cmap for the first matching encoding. Reset
           cmap to that value if first matching encoding is going to be used.
           (Eliseo Martínez)
相关文件: src/hardcopy.c

Patch 7.4.696
问题: Not freeing memory when encountering an error.
解决方案: Free the stack before returning. (Eliseo Martínez)
相关文件: src/regexp_nfa.c

Patch 7.4.697
问题: The filename used for ":profile" must be given literally.
解决方案: Expand "~" and environment variables. (Marco Hinz)
相关文件: src/ex_cmds2.c

Patch 7.4.698
问题: Various problems with locked and fixed lists and dictionaries.
解决方案: Disallow changing locked items, fix a crash, add tests. (Olaf Dabrunz)
相关文件: src/structs.h, src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.699
问题: E315 when trying to delete a fold. (Yutao Yuan)
解决方案: Make sure the fold doesn't go beyond the last buffer line. (Christian Brabandt)
相关文件: src/fold.c

Patch 7.4.700
问题: Fold can't be opened after ":move". (Ein Brown)
解决方案: Delete the folding information and update it afterwards. (Christian Brabandt)
相关文件: src/ex_cmds.c, src/fold.c, src/testdir/test45.in, src/testdir/test45.ok

Patch 7.4.701
问题: Compiler warning for using uninitialized variable. (Yasuhiro Matsumoto)
解决方案: Initialize it.
相关文件: src/hardcopy.c

Patch 7.4.702
问题: Joining an empty list does unnecessary work.
解决方案: Let join() return early. (Marco Hinz)
相关文件: src/eval.c

Patch 7.4.703
问题: Compiler warning for start_dir unused when building unittests.
解决方案: Move start_dir inside the #ifdef.
相关文件: src/main.c

Patch 7.4.704
问题: Searching for a character matches an illegal byte and causes invalid memory access. (Dominique Pelle)
解决方案: Do not match an invalid byte when search for a character in a string. Fix equivalence classes using negative numbers, which result in illegal bytes.
相关文件: src/misc2.c, src/regexp.c, src/testdir/test44.in

Patch 7.4.705
问题: Can't build with Ruby 2.2.
解决方案: Add #ifdefs to handle the incompatible change. (Andrei Olsen)
相关文件: src/if_ruby.c

Patch 7.4.706
问题:　　Window drawn wrong when 'laststatus' is zero and there is a
　　command-line window. (Yclept Nemo)
解决方案:　Set the status height a bit later. (Christian Brabandt)
相关文件:　src/window.c

Patch 7.4.707
问题:　　Undo files can have their executable bit set.
解决方案:　Strip of the executable bit. (Mikael Berthe)
相关文件:　src/undo.c

Patch 7.4.708
问题:　　gettext() is called too often.
解决方案:　Do not call gettext() for messages until they are actually used.
　　(idea by Yasuhiro Matsumoto)
相关文件:　src/eval.c

Patch 7.4.709
问题:　　":tabmove" does not work as documented.
解决方案:　Make it work consistently. Update documentation and add tests.
　　(Hirohito Higashi)
相关文件:　src/window.c, runtime/doc/tabpage.txt, src/ex_docmd.c,
　　src/testdir/test62.in, src/testdir/test62.ok

Patch 7.4.710
问题:　　It is not possible to make spaces visible in list mode.
解决方案:　Add the "space" item to 'listchars'. (David Bürgin, issue 350)
相关文件:　runtime/doc/options.txt, src/globals.h, src/message.h,
　　src/screen.c, src/testdir/test_listchars.in,
　　src/testdir/test_listchars.ok, src/testdir/Make_amiga.mak,
　　src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
　　src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
　　src/testdir/Makefile

Patch 7.4.711 (after 7.4.710)
问题:　　Missing change in one file.
解决方案:　Also change option.c
相关文件:　src/option.c

Patch 7.4.712 (after 7.4.710)
问题:　　Missing change in another file.
解决方案:　Also change message.c
相关文件:　src/message.c

Patch 7.4.713
问题:　　Wrong condition for #ifdef.
解决方案:　Change USR_EXRC_FILE2 to USR_VIMRC_FILE2. (Mikael Fourrier)
相关文件:　src/os_unix.h

Patch 7.4.714
问题:　　Illegal memory access when there are illegal bytes.
解决方案:　Check the byte length of the character. (Dominique Pelle)
相关文件:　src/regexp.c

Patch 7.4.715
问题:　　Invalid memory access when there are illegal bytes.
解决方案:　Get the length from the text, not from the character. (Dominique Pelle)
相关文件:　src/regexp_nfa.c

Patch 7.4.716
问题:　　When using the 'c' flag of ":substitute" and selecting "a" or "l" at the prompt the flags are not remembered for ":&&". (Ingo Karkat)
解决方案:　Save the flag values and restore them. (Hirohito Higashi)
相关文件:　src/ex_cmds.c

Patch 7.4.717
问题:　　":let list += list" can change a locked list.
解决方案:　Check for the lock earlier. (Olaf Dabrunz)
相关文件:　src/eval.c, src/testdir/test55.in, src/testdir/test55.ok

Patch 7.4.718
问题:　　Autocommands triggered by quickfix cannot get the current title value.
解决方案:　Set w:quickfix_title earlier. (Yannick)
Also move the check for a title into the function.
相关文件:　src/quickfix.c

Patch 7.4.719
问题:　　Overflow when adding MAXCOL to a pointer.
解决方案:　Subtract pointers instead. (James McCoy)
相关文件:　src/screen.c

Patch 7.4.720
问题:　　Can't build with Visual Studio 2015.
解决方案:　Recognize the "version 14" numbers and omit /nodefaultlib when appropriate. (Paul Moore)
相关文件:　src/Make_mvc.mak

Patch 7.4.721
问题:　　When 'list' is set Visual mode does not highlight anything in empty lines. (mgaleski)
解决方案:　Check the value of lcs_eol in another place. (Christian Brabandt)
相关文件:　src/screen.c

Patch 7.4.722
问题:　　0x202f is not recognized as a non-breaking space character.
解决方案:　Add 0x202f to the list. (Christian Brabandt)
相关文件:　runtime/doc/options.txt, src/message.c, src/screen.c

Patch 7.4.723
问题:　　For indenting, finding the C++ baseclass can be slow.
解决方案:　Cache the result. (Hirohito Higashi)
相关文件:　src/misc1.c

Patch 7.4.724
问题:　　Vim icon does not show in Windows context menu. (issue 249)

解决方案: Load the icon in GvimExt.
相关文件: src/GvimExt/gvimext.cpp, src/GvimExt/gvimext.h

Patch 7.4.725
问题: ":call setreg('"', [])" reports an internal error.
解决方案: Make the register empty. (Yasuhiro Matsumoto)
相关文件: src/ops.c

Patch 7.4.726 (after 7.4.724)
问题: Cannot build GvimExt.
解决方案: Set APPVER to 5.0. (KF Leong)
相关文件: src/GvimExt/Makefile

Patch 7.4.727 (after 7.4.724)
问题: Cannot build GvimExt with MingW.
解决方案: Add -lgdi32. (KF Leong)
相关文件: src/GvimExt/Make_ming.mak

Patch 7.4.728
问题: Can't build with some version of Visual Studio 2015.
解决方案: Recognize another version 14 number. (Sinan)
相关文件: src/Make_mvc.mak

Patch 7.4.729 (after 7.4.721)
问题: Occasional crash with 'list' set.
解决方案: Fix off-by-one error. (Christian Brabandt)
相关文件: src/screen.c

Patch 7.4.730
问题: When setting the crypt key and using a swap file, text may be
encrypted twice or unencrypted text remains in the swap file.
(Issue 369)
解决方案: Call ml_preserve() before re-encrypting. Set correct index for
next pointer block.
相关文件: src/memfile.c, src/memline.c, src/proto/memline.pro, src/option.c

Patch 7.4.731
问题: The tab menu shows "Close tab" even when it doesn't work.
解决方案: Don't show "Close tab" for the last tab. (John Marriott)
相关文件: src/gui_w48.c, src/gui_gtk_x11.c, src/gui_mac.c, src/gui_motif.c

Patch 7.4.732
问题: The cursor line is not always updated for the "O" command.
解决方案: Reset the VALID_CROW flag. (Christian Brabandt)
相关文件: src/normal.c

Patch 7.4.733
问题: test_listchars breaks on MS-Windows. (Kenichi Ito)
解决方案: Set fileformat to "unix". (Christian Brabandt)
相关文件: src/testdir/test_listchars.in

Patch 7.4.734
问题: ml_get error when using "p" in a Visual selection in the last
line.

解决方案: Change the behavior at the last line. (Yukihiro Nakadaira)
相关文件: src/normal.c, src/ops.c, src/testdir/test94.in,
           src/testdir/test94.ok

Patch 7.4.735
问题: Wrong argument for sizeof().
解决方案: Use a pointer argument. (Chris Hall)
相关文件: src/eval.c

Patch 7.4.736
问题: Invalid memory access.
解决方案: Avoid going over the end of a NUL terminated string. (Dominique
           Pelle)
相关文件: src/regexp.c

Patch 7.4.737
问题: On MS-Windows vimgrep over arglist doesn't work (Issue 361)
解决方案: Only escape backslashes in ## expansion when it is not used as the
           path separator. (James McCoy)
相关文件: src/ex_docmd.c

Patch 7.4.738 (after 7.4.732)
问题: Can't compile without the syntax highlighting feature.
解决方案: Add #ifdef around use of w_p_cul. (Hirohito Higashi)
相关文件: src/normal.c, src/screen.c

Patch 7.4.739
问题: In a string "\U" only takes 4 digits, while after CTRL-V U eight
           digits can be used.
解决方案: Make "\U" also take eight digits. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.4.740
问题: ":1quit" works like ":.quit". (Bohr Shaw)
解决方案: Don't exit Vim when a range is specified. (Christian Brabandt)
相关文件: src/ex_docmd.c, src/testdir/test13.in, src/testdir/test13.ok

Patch 7.4.741
问题: When using += with ":set" a trailing comma is not recognized.
           (Issue 365)
解决方案: Don't add a second comma. Add a test. (partly by Christian
           Brabandt)
相关文件: src/option.c, src/testdir/test_set.in, src/testdir/test_set.ok,
           src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
           src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
           src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.742
问题: Cannot specify a vertical split when loading a buffer for a
           quickfix command.
解决方案: Add the "vsplit" value to 'switchbuf'. (Brook Hong)
相关文件: runtime/doc/options.txt, src/buffer.c, src/option.h

Patch 7.4.743

问题: "p" in Visual mode causes an unexpected line split.
解决方案: Advance the cursor first. (Yukihiro Nakadaira)
相关文件: src/ops.c, src/testdir/test94.in, src/testdir/test94.ok

Patch 7.4.744
问题: No tests for Ruby and Perl.
解决方案: Add minimal tests. (Ken Takata)
相关文件: src/testdir/test_perl.in, src/testdir/test_perl.ok,
            src/testdir/test_ruby.in, src/testdir/test_ruby.ok,
            src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
            src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
            src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.745
问题: The entries added by matchaddpos() are returned by getmatches()
            but can't be set with setmatches(). (Lcd)
解决方案: Fix setmatches(). (Christian Brabandt)
相关文件: src/eval.c, src/testdir/test63.in, src/testdir/test63.ok

Patch 7.4.746
问题: ":[count]tag" is not always working. (cs86661)
解决方案: Set cur_match a bit later. (Hirohito Higashi)
相关文件: src/tag.c,

Patch 7.4.747
问题: ":cnext" may jump to the wrong column when setting
            'virtualedit=all' (cs86661)
解决方案: Reset the coladd field. (Hirohito Higashi)
相关文件: src/quickfix.c

Patch 7.4.748 (after 7.4.745)
问题: Buffer overflow.
解决方案: Make the buffer larger. (Kazunobu Kuriyama)
相关文件: src/eval.c

Patch 7.4.749 (after 7.4.741)
问题: For some options two consecutive commas are OK. (Nikolai Pavlov)
解决方案: Add the P_ONECOMMA flag.
相关文件: src/option.c

Patch 7.4.750
问题: Cannot build with clang 3.5 on Cygwin with perl enabled.
解决方案: Strip "-fdebug-prefix-map" in configure. (Ken Takata)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.751
问题: It is not obvious how to enable the address sanitizer.
解决方案: Add commented-out flags in the Makefile. (Dominique Pelle)
            Also add missing test targets.
相关文件: src/Makefile

Patch 7.4.752
问题: Unicode 8.0 not supported.
解决方案: Update tables for Unicode 8.0. Avoid E36 when running the script.

(James McCoy)
相关文件: runtime/tools/unicode.vim, src/mbyte.c

Patch 7.4.753
问题: Appending in Visual mode with 'linebreak' set does not work properly. Also when 'selection' is "exclusive". (Ingo Karkat)
解决方案: Recalculate virtual columns. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test_listlbr.in, src/testdir/test_listlbr.ok, src/testdir/test_listlbr_utf8.in, src/testdir/test_listlbr_utf8.ok

Patch 7.4.754
问题: Using CTRL-A in Visual mode does not work well. (Gary Johnson)
解决方案: Make it increment all numbers in the Visual area. (Christian Brabandt)
相关文件: runtime/doc/change.txt, src/normal.c, src/ops.c, src/proto/ops.pro, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_increment.in, src/testdir/test_increment.ok

Patch 7.4.755
问题: It is not easy to count the number of characters.
解决方案: Add the skipcc argument to strchars(). (Hirohito Higashi, Ken Takata)
相关文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_utf8.in, src/testdir/test_utf8.ok

Patch 7.4.756
问题: Can't use strawberry Perl 5.22 x64 on MS-Windows.
解决方案: Add new defines and #if. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak, src/if_perl.xs

Patch 7.4.757
问题: Cannot detect the background color of a terminal.
解决方案: Add T_RBG to request the background color if possible. (Lubomir Rintel)
相关文件: src/main.c, src/term.c, src/term.h, src/proto/term.pro

Patch 7.4.758
问题: When 'conceallevel' is 1 and quitting the command-line window with CTRL-C the first character ':' is erased.
解决方案: Reset 'conceallevel' in the command-line window. (Hirohito Higashi)
相关文件: src/ex_getln.c

Patch 7.4.759
问题: Building with Lua 5.3 doesn't work, symbols have changed.
解决方案: Use the new names for the new version. (Felix Schnizlein)
相关文件: src/if_lua.c

Patch 7.4.760
问题: Spelling mistakes are not displayed after ":syn spell".

解决方案: Force a redraw after ":syn spell" command. (Christian Brabandt)
相关文件: src/syntax.c

Patch 7.4.761 (after 7.4.757)
问题: The request-background termcode implementation is incomplete.
解决方案: Add the missing pieces.
相关文件: src/option.c, src/term.c

Patch 7.4.762 (after 7.4.757)
问题: Comment for may_req_bg_color() is wrong. (Christ van Willegen)
解决方案: Rewrite the comment.
相关文件: src/term.c

Patch 7.4.763 (after 7.4.759)
问题: Building with Lua 5.1 doesn't work.
解决方案: Define lua_replace and lua_remove. (KF Leong)
相关文件: src/if_lua.c

Patch 7.4.764 (after 7.4.754)
问题: test_increment fails on MS-Windows. (Ken Takata)
解决方案: Clear Visual mappings. (Taro Muraoka)
相关文件: src/testdir/test_increment.in

Patch 7.4.765 (after 7.4.754)
问题: CTRL-A and CTRL-X in Visual mode do not always work well.
解决方案: Improvements for increment and decrement. (Christian Brabandt)
相关文件: src/normal.c, src/ops.c, src/testdir/test_increment.in,
           src/testdir/test_increment.ok

Patch 7.4.766 (after 7.4.757)
问题: Background color check does not work on Tera Term.
解决方案: Also recognize ST as a termination character. (Hirohito Higashi)
相关文件: src/term.c

Patch 7.4.767
问题: --remote-tab-silent can fail on MS-Windows.
解决方案: Use single quotes to avoid problems with backslashes. (Idea by
           Weiyong Mao)
相关文件: src/main.c

Patch 7.4.768
问题: :diffoff only works properly once.
解决方案: Also make :diffoff work when used a second time. (Olaf Dabrunz)
相关文件: src/diff.c

Patch 7.4.769 (after 7.4 768)
问题: Behavior of :diffoff is not tested.
解决方案: Add a bit of testing. (Olaf Dabrunz)
相关文件: src/testdir/test47.in, src/testdir/test47.ok

Patch 7.4.770 (after 7.4.766)
问题: Background color response with transparency is not ignored.
解决方案: Change the way escape sequences are recognized. (partly by
           Hirohito Higashi)

相关文件: src/ascii.h, src/term.c

Patch 7.4.771
问题: Search does not handle multi-byte character at the start position
correctly.
解决方案: Take byte size of character into account. (Yukihiro Nakadaira)
相关文件: src/search.c, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_search_mbyte.in,
src/testdir/test_search_mbyte.ok

Patch 7.4.772
问题: Racket 6.2 is not supported on MS-Windows.
解决方案: Check for the "racket" subdirectory. (Weiyong Mao)
相关文件: src/Make_mvc.mak, src/if_mzsch.c

Patch 7.4.773
问题: 'langmap' is used in command-line mode when checking for mappings.
Issue 376.
解决方案: Do not use 'langmap' in command-line mode. (Larry Velazquez)
相关文件: src/getchar.c, src/testdir/test_mapping.in,
src/testdir/test_mapping.ok

Patch 7.4.774
问题: When using the CompleteDone autocommand event it's difficult to
get to the completed items.
解决方案: Add the v:completed_items variable. (Shougo Matsu)
相关文件: runtime/doc/autocmd.txt, runtime/doc/eval.txt, src/edit.c,
src/eval.c, src/macros.h, src/proto/eval.pro, src/vim.h

Patch 7.4.775
问题: It is not possible to avoid using the first item of completion.
解决方案: Add the "noinsert" and "noselect" values to 'completeopt'. (Shougo
Matsu)
相关文件: runtime/doc/options.txt, src/edit.c, src/option.c

Patch 7.4.776
问题: Equivalence class for 'd' does not work correctly.
解决方案: Fix 0x1e0f and 0x1d0b. (Dominique Pelle)
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.4.777
问题: The README file doesn't look nice on github.
解决方案: Add a markdown version of the README file.
相关文件: Filelist, README.md

Patch 7.4.778
问题: Coverity warns for uninitialized variable.
解决方案: Change condition of assignment.
相关文件: src/ops.c

Patch 7.4.779
问题: Using CTRL-A in a line without a number moves the cursor. May

cause a crash when at the start of the line. (Urtica Dioica)
解决方案: Do not move the cursor if no number was changed.
相关文件: src/ops.c

Patch 7.4.780
问题: Compiler complains about uninitialized variable and clobbered variables.
解决方案: Add Initialization. Make variables static.
相关文件: src/ops.c, src/main.c

Patch 7.4.781
问题: line2byte() returns one less when 'bin' and 'noeol' are set.
解决方案: Only adjust the size for the last line. (Rob Wu)
相关文件: src/memline.c

Patch 7.4.782
问题: Still a few problems with CTRL-A and CTRL-X in Visual mode.
解决方案: Fix the reported problems. (Christian Brabandt)
相关文件: src/charset.c, src/eval.c, src/ex_cmds.c, src/ex_getln.c, src/misc2.c, src/normal.c, src/ops.c, src/option.c, src/proto/charset.pro, src/testdir/test_increment.in, src/testdir/test_increment.ok

Patch 7.4.783
问题: copy_chars() and copy_spaces() are inefficient.
解决方案: Use memset() instead. (Dominique Pelle)
相关文件: src/ex_getln.c, src/misc2.c, src/ops.c, src/proto/misc2.pro, src/screen.c

Patch 7.4.784
问题: Using both "noinsert" and "noselect" in 'completeopt' does not work properly.
解决方案: Change the ins_complete() calls. (Ozaki Kiichi)
相关文件: src/edit.c

Patch 7.4.785
问题: On some systems automatically adding the missing EOL causes problems. Setting 'binary' has too many side effects.
解决方案: Add the 'fixeol' option, default on. (Pavel Samarkin)
相关文件: src/buffer.c, src/fileio.c, src/memline.c, src/netbeans.c, src/ops.c, src/option.c, src/option.h, src/os_unix.c, src/os_win32.c, src/structs.h, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_fixeol.in, src/testdir/test_fixeol.ok, runtime/doc/options.txt, runtime/optwin.vim

Patch 7.4.786
问题: It is not possible for a plugin to adjust to a changed setting.
解决方案: Add the OptionSet autocommand event. (Christian Brabandt)
相关文件: runtime/doc/autocmd.txt, runtime/doc/eval.txt, src/eval.c, src/fileio.c, src/option.c, src/proto/eval.pro, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,

src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/test_autocmd_option.in,
src/testdir/test_autocmd_option.ok, src/vim.h

Patch 7.4.787 (after 7.4.786)
问题: snprintf() isn't available everywhere.
解决方案: Use vim_snprintf(). (Ken Takata)
相关文件: src/option.c

Patch 7.4.788 (after 7.4.787)
问题: Can't build without the crypt feature. (John Marriott)
解决方案: Add #ifdef's.
相关文件: src/option.c

Patch 7.4.789 (after 7.4.788)
问题: Using freed memory and crash. (Dominique Pelle)
解决方案: Correct use of pointers. (Hirohito Higashi)
相关文件: src/option.c

Patch 7.4.790 (after 7.4.786)
问题: Test fails when the autochdir feature is not available. Test
output contains the test script.
解决方案: Check for the autochdir feature. (Kazunobu Kuriyama) Only write
the relevant test output.
相关文件: src/testdir/test_autocmd_option.in,
src/testdir/test_autocmd_option.ok

Patch 7.4.791
问题: The buffer list can be very long.
解决方案: Add an argument to ":ls" to specify the type of buffer to list.
(Marcin Szamotulski)
相关文件: runtime/doc/windows.txt, src/buffer.c, src/ex_cmds.h

Patch 7.4.792
问题: Can only conceal text by defining syntax items.
解决方案: Use matchadd() to define concealing. (Christian Brabandt)
相关文件: runtime/doc/eval.txt, src/eval.c, src/ex_docmd.c,
src/proto/window.pro, src/screen.c, src/structs.h,
src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_match_conceal.in,
src/testdir/test_match_conceal.ok, src/window.c

Patch 7.4.793
问题: Can't specify when not to ring the bell.
解决方案: Add the 'belloff' option. (Christian Brabandt)
相关文件: runtime/doc/options.txt, src/edit.c, src/ex_getln.c,
src/hangulin.c, src/if_lua.c, src/if_mzsch.c, src/if_tcl.c,
src/message.c, src/misc1.c, src/normal.c, src/option.c,
src/option.h, src/proto/misc1.pro, src/search.c, src/spell.c

Patch 7.4.794

问题:　　Visual Studio 2015 is not recognized.
解决方案:　Add the version numbers to the makefile. (Taro Muraoka)
相关文件:　src/Make_mvc.mak

Patch 7.4.795
问题:　　The 'fixeol' option is not copied to a new window.
解决方案:　Copy the option value. (Yasuhiro Matsumoto)
相关文件:　src/option.c

Patch 7.4.796
问题:　　Warning from 64 bit compiler.
解决方案:　Add type cast. (Mike Williams)
相关文件:　src/ops.c

Patch 7.4.797
问题:　　Crash when using more lines for the command line than
　　'maxcombine'.
解决方案:　Use the correct array index. Also, do not try redrawing when
　　exiting. And use screen_Columns instead of Columns.
相关文件:　src/screen.c

Patch 7.4.798 (after 7.4.753)
问题:　　Repeating a change in Visual mode does not work as expected.
　　(Urtica Dioica)
解决方案:　Make redo in Visual mode work better. (Christian Brabandt)
相关文件:　src/normal.c, src/testdir/test_listlbr.in,
　　src/testdir/test_listlbr.ok

Patch 7.4.799
问题:　　Accessing memory before an allocated block.
解决方案:　Check for not going before the start of a pattern. (Dominique
　　Pelle)
相关文件:　src/fileio.c

Patch 7.4.800
问题:　　Using freed memory when triggering CmdUndefined autocommands.
解决方案:　Set pointer to NULL. (Dominique Pelle)
相关文件:　src/ex_docmd.c

Patch 7.4.801 (after 7.4.769)
问题:　　Test for ":diffoff" doesn't catch all potential problems.
解决方案:　Add a :diffthis and a :diffoff command. (Olaf Dabrunz)
相关文件:　src/testdir/test47.in

Patch 7.4.802
问题:　　Using "A" in Visual mode while 'linebreak' is set is not tested.
解决方案:　Add a test for this, verifies the problem is fixed. (Ingo Karkat)
相关文件:　src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.803
问题:　　C indent does not support C11 raw strings. (Mark Lodato)
解决方案:　Do not change indent inside the raw string.
相关文件:　src/search.c, src/misc1.c, src/edit.c, src/ops.c,
　　src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.804
问题:　　Xxd doesn't have a license notice.
解决方案:　Add license as indicated by Juergen.
相关文件:　src/xxd/xxd.c

Patch 7.4.805
问题:　　The ruler shows "Bot" even when there are only filler lines
missing. (Gary Johnson)
解决方案:　Use "All" when the first line and one filler line are visible.
相关文件:　src/buffer.c

Patch 7.4.806
问题:　　CTRL-A in Visual mode doesn't work properly with "alpha" in
'nrformats'.
解决方案:　Make it work. (Christian Brabandt)
相关文件:　src/ops.c, src/testdir/test_increment.in,
src/testdir/test_increment.ok

Patch 7.4.807 (after 7.4.798)
问题:　　After CTRL-V CTRL-A mode isn't updated. (Hirohito Higashi)
解决方案:　Clear the command line or update the displayed command.
相关文件:　src/normal.c

Patch 7.4.808
问题:　　On MS-Windows 8 IME input doesn't work correctly.
解决方案:　Read console input before calling MsgWaitForMultipleObjects().
(vim-jp, Nobuhiro Takasaki)
相关文件:　src/os_win32.c

Patch 7.4.809 (after 7.4.802)
问题:　　Test is duplicated.
解决方案:　Roll back 7.4.802.
相关文件:　src/testdir/test39.in, src/testdir/test39.ok

Patch 7.4.810
问题:　　With a sequence of commands using buffers in diff mode E749 is
given. (itchyny)
解决方案:　Skip unloaded buffer. (Hirohito Higashi)
相关文件:　src/diff.c

Patch 7.4.811
问题:　　Invalid memory access when using "exe 'sc'".
解决方案:　Avoid going over the end of the string. (Dominique Pelle)
相关文件:　src/ex_docmd.c

Patch 7.4.812
问题:　　Gcc sanitizer complains about using a NULL pointer to memmove().
解决方案:　Only call memmove when there is something to move. (Vittorio
Zecca)
相关文件:　src/memline.c

Patch 7.4.813
问题:　　It is not possible to save and restore character search state.

解决方案: Add getcharsearch() and setcharsearch(). (James McCoy)
相关文件: runtime/doc/eval.txt, src/eval.c, src/proto/search.pro,
           src/search.c, src/testdir/test_charsearch.in,
           src/testdir/test_charsearch.ok, src/testdir/Makefile,
           src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
           src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
           src/testdir/Make_vms.mms

Patch 7.4.814
问题: Illegal memory access with "sy match a fold".
解决方案: Check for empty string. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.4.815
问题: Invalid memory access when doing ":call g:".
解决方案: Check for an empty name. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.4.816
问题: Invalid memory access when doing ":fun X(".
解决方案: Check for missing ')'. (Dominique Pelle)
相关文件: src/eval.c

Patch 7.4.817
问题: Invalid memory access in file_pat_to_reg_pat().
解决方案: Use vim_isspace() instead of checking for a space only. (Dominique
           Pelle)
相关文件: src/fileio.c

Patch 7.4.818
问题: 'linebreak' breaks c% if the last Visual selection was block.
           (Chris Morganiser, Issue 389)
解决方案: Handle Visual block mode differently. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test_listlbr.in,
           src/testdir/test_listlbr.ok

Patch 7.4.819
问题: Beeping when running the tests.
解决方案: Fix 41 beeps. (Roland Eggner)
相关文件: src/testdir/test17.in, src/testdir/test29.in,
           src/testdir/test4.in, src/testdir/test61.in,
           src/testdir/test82.in, src/testdir/test83.in,
           src/testdir/test90.in, src/testdir/test95.in,
           src/testdir/test_autoformat_join.in

Patch 7.4.820
问题: Invalid memory access in file_pat_to_reg_pat.
解决方案: Avoid looking before the start of a string. (Dominique Pelle)
相关文件: src/fileio.c

Patch 7.4.821
问题: Coverity reports a few problems.
解决方案: Avoid the warnings. (Christian Brabandt)
相关文件: src/ex_docmd.c, src/option.c, src/screen.c

Patch 7.4.822
问题: More problems reported by coverity.
解决方案: Avoid the warnings. (Christian Brabandt)
相关文件: src/os_unix.c, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c,
src/ex_getln.c, src/fold.c, src/gui.c, src/gui_w16.c,
src/gui_w32.c, src/if_cscope.c, src/if_xcmdsrv.c, src/move.c,
src/normal.c, src/regexp.c, src/syntax.c, src/ui.c, src/window.c

Patch 7.4.823
问题: Cursor moves after CTRL-A on alphabetic character.
解决方案: (Hirohito Higashi, test by Christian Brabandt)
相关文件: src/testdir/test_increment.in, src/testdir/test_increment.ok,
src/ops.c

Patch 7.4.824 (after 7.4.813)
问题: Can't compile without the multi-byte feature. (John Marriott)
解决方案: Add #ifdef.
相关文件: src/eval.c

Patch 7.4.825
问题: Invalid memory access for ":syn keyword x a[".
解决方案: Do not skip over the NUL. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.4.826
问题: Compiler warnings and errors.
解决方案: Make it build properly without the multi-byte feature.
相关文件: src/eval.c, src/search.c

Patch 7.4.827
问题: Not all test targets are in the Makefile.
解决方案: Add the missing targets.
相关文件: src/Makefile

Patch 7.4.828
问题: Crash when using "syn keyword x c". (Dominique Pelle)
解决方案: Initialize the keyword table. (Raymond Ko, PR 397)
相关文件: src/syntax.c

Patch 7.4.829
问题: Crash when clicking in beval balloon. (Travis Lebsock)
解决方案: Use PostMessage() instead of DestroyWindow(). (Raymond Ko, PR 298)
相关文件: src/gui_w32.c

Patch 7.4.830
问题: Resetting 'encoding' when doing ":set all&" causes problems.
(Bjorn Linse) Display is not updated.
解决方案: Do not reset 'encoding'. Do a full redraw.
相关文件: src/option.c

Patch 7.4.831
问题: When expanding =expr on the command line and encountering an
error, the command is executed anyway.

解决方案: Bail out when an error is detected.
相关文件: src/misc1.c

Patch 7.4.832
问题: $HOME in ‘=$HOME . ’/.vimrc’‘ is expanded too early.
解决方案: Skip over =expr when expanding environment names.
相关文件: src/misc1.c

Patch 7.4.833
问题: More side effects of ":set all&" are missing. (Bjrn Linse)
解决方案: Call didset_options() and add didset_options2() to collect more
           side effects to take care of. Still not everything...
相关文件: src/option.c

Patch 7.4.834
问题: gettabvar() doesn’t work after Vim start. (Szymon Wrozynski)
解决方案: Handle first window in tab still being NULL. (Christian Brabandt)
相关文件: src/eval.c, src/testdir/test91.in, src/testdir/test91.ok

Patch 7.4.835
问题: Comparing utf-8 sequences does not handle different byte sizes
           correctly.
解决方案: Get the byte size of each character. (Dominique Pelle)
相关文件: src/misc2.c

Patch 7.4.836
问题: Accessing uninitialized memory.
解决方案: Add missing calls to init_tv(). (Dominique Pelle)
相关文件: src/eval.c

Patch 7.4.837
问题: Compiler warning with MSVC compiler when using +sniff.
解决方案: Use Sleep() instead of _sleep(). (Tux)
相关文件: src/if_sniff.c

Patch 7.4.838 (after 7.4.833)
问题: Can’t compile without the crypt feature. (John Marriott)
解决方案: Add #ifdef.
相关文件: src/option.c

Patch 7.4.839
问题: Compiler warning on 64-bit system.
解决方案: Add cast to int. (Mike Williams)
相关文件: src/search.c

Patch 7.4.840 (after 7.4.829)
问题: Tooltip window stays open.
解决方案: Send a WM_CLOSE message. (Jurgen Kramer)
相关文件: src/gui_w32.c

Patch 7.4.841
问题: Can’t compile without the multi-byte feature. (John Marriott)
解决方案: Add more #ifdef’s.
相关文件: src/option.c

Patch 7.4.842 (after 7.4.840)
问题:　　Sending too many messages to close the balloon.
解决方案:　Only send a WM_CLOSE message. (Jurgen Kramer)
相关文件:　src/gui_w32.c

Patch 7.4.843 (after 7.4.835)
问题:　　Still possible to go beyond the end of a string.
解决方案:　Check for NUL also in second string. (Dominique Pelle)
相关文件:　src/misc2.c

Patch 7.4.844
问题:　　When '#' is in 'isident' the is# comparator doesn't work.
解决方案:　Don't use vim_isIDc(). (Yasuhiro Matsumoto)
相关文件:　src/eval.c, src/testdir/test_comparators.in,
　　src/testdir/test_comparators.ok, src/testdir/Makefile,
　　src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
　　src/testdir/Make_vms.mms

Patch 7.4.845
问题:　　Compiler warning for possible loss of data.
解决方案:　Add a type cast. (Erich Ritz)
相关文件:　src/misc1.c

Patch 7.4.846
问题:　　Some GitHub users don't know how to use issues.
解决方案:　Add a file that explains the basics of contributing.
相关文件:　Filelist, CONTRIBUTING.md

Patch 7.4.847
问题:　　"vi)d" may leave a character behind.
解决方案:　Skip over multi-byte character. (Christian Brabandt)
相关文件:　src/search.c

Patch 7.4.848
问题:　　CTRL-A on hex number in Visual block mode is incorrect.
解决方案:　Account for the "0x". (Hirohito Higashi)
相关文件:　src/charset.c, src/testdir/test_increment.in,
　　src/testdir/test_increment.ok

Patch 7.4.849
问题:　　Moving the cursor in Insert mode starts new undo sequence.
解决方案:　Add CTRL-G U to keep the undo sequence for the following cursor
　　movement command. (Christian Brabandt)
相关文件:　runtime/doc/insert.txt, src/edit.c, src/testdir/test_mapping.in,
　　src/testdir/test_mapping.ok

Patch 7.4.850 (after 7.4.846)
问题:　　<Esc> does not show up.
解决方案:　Use > and <. (Kazunobu Kuriyama)
相关文件:　CONTRIBUTING.md

Patch 7.4.851

问题:　　Saving and restoring the console buffer does not work properly.
解决方案:　Instead of ReadConsoleOutputA/WriteConsoleOutputA use
CreateConsoleScreenBuffer and SetConsoleActiveScreenBuffer.
(Ken Takata)
相关文件:　src/os_win32.c

Patch 7.4.852
问题:　　On MS-Windows console Vim uses ANSI APIs for keyboard input and
console output, it cannot input/output Unicode characters.
解决方案:　Use Unicode APIs for console I/O. (Ken Takata, Yasuhiro Matsumoto)
相关文件:　src/os_win32.c, src/ui.c, runtime/doc/options.txt

Patch 7.4.853
问题:　　"zt" in diff mode does not always work properly. (Gary Johnson)
解决方案:　Don't count filler lines twice. (Christian Brabandt)
相关文件:　src/move.c

Patch 7.4.854 (after 7.4.850)
问题:　　Missing information about runtime files.
解决方案:　Add section about runtime files. (Christian Brabandt)
相关文件:　CONTRIBUTING.md

Patch 7.4.855
问题:　　GTK: font glitches for combining characters
解决方案:　Use pango_shape_full() instead of pango_shape(). (luchr, PR #393)
相关文件:　src/gui_gtk_x11.c

Patch 7.4.856
问题:　　"zt" still doesn't work well with filler lines. (Gary Johnson)
解决方案:　Check for filler lines above the cursor. (Christian Brabandt)
相关文件:　src/move.c

Patch 7.4.857
问题:　　Dragging the current tab with the mouse doesn't work properly.
解决方案:　Take the current tabpage index into account. (Hirohito Higashi)
相关文件:　src/normal.c

Patch 7.4.858
问题:　　It's a bit clumsy to execute a command on a list of matches.
解决方案:　Add the ":ldo", ":lfdo", ":cdo" and ":cfdo" commands. (Yegappan
Lakshmanan)
相关文件:　runtime/doc/cmdline.txt, runtime/doc/editing.txt,
runtime/doc/index.txt, runtime/doc/quickfix.txt,
runtime/doc/tabpage.txt, runtime/doc/windows.txt, src/ex_cmds.h,
src/ex_cmds2.c, src/ex_docmd.c, src/proto/quickfix.pro,
src/quickfix.c, src/testdir/Make_amiga.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
src/testdir/Make_os2.mak, src/testdir/Make_vms.mms,
src/testdir/Makefile, src/testdir/test_cdo.in,
src/testdir/test_cdo.ok

Patch 7.4.859
问题:　　Vim doesn't recognize all htmldjango files.
解决方案:　Recognize a comment. (Daniel Hahler, PR #410)

相关文件: runtime/filetype.vim

Patch 7.4.860
问题: Filetype detection is outdated.
解决方案: Include all recent and not-so-recent changes.
相关文件: runtime/filetype.vim

Patch 7.4.861 (after 7.4.855)
问题: pango_shape_full() is not always available.
解决方案: Add a configure check.
相关文件: src/configure.in, src/auto/configure, src/config.h.in,
src/gui_gtk_x11.c

Patch 7.4.862 (after 7.4.861)
问题: Still problems with pango_shape_full() not available.
解决方案: Change AC_TRY_COMPILE to AC_TRY_LINK.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.863 (after 7.4.856)
问题: plines_nofill() used without the diff feature.
解决方案: Define PLINES_NOFILL().
相关文件: src/macros.h, src/move.c

Patch 7.4.864 (after 7.4.858)
问题: Tiny build fails.
解决方案: Put qf_ items inside #ifdef.
相关文件: src/ex_docmd.c

Patch 7.4.865
问题: Compiler warning for uninitialized variable.
解决方案: Initialize.
相关文件: src/ex_cmds2.c

Patch 7.4.866
问题: Crash when changing the 'tags' option from a remote command.
(Benjamin Fritz)
解决方案: Instead of executing messages immediately, use a queue, like for
netbeans. (James Kolb)
相关文件: src/ex_docmd.c, src/getchar.c, src/gui_gtk_x11.c, src/gui_w48.c,
src/gui_x11.c, src/if_xcmdsrv.c, src/misc2.c, src/os_unix.c,
src/proto/if_xcmdsrv.pro, src/proto/misc2.pro, src/macros.h

Patch 7.4.867 (after 7.4.866)
问题: Can't build on MS-Windows. (Taro Muraoka)
解决方案: Adjust #ifdef.
相关文件: src/misc2.c

Patch 7.4.868
问题: 'smarttab' is also effective when 'paste' is enabled. (Alexander
Monakov)
解决方案: Disable 'smarttab' when 'paste' is set. (Christian Brabandt)
Do the same for 'expandtab'.
相关文件: src/option.c, src/structs.h

Patch 7.4.869
问题:　　MS-Windows: scrolling may cause text to disappear when using an
　　Intel GPU.
解决方案:　Call GetPixel(). (Yohei Endo)
相关文件:　src/gui_w48.c

Patch 7.4.870
问题:　　May get into an invalid state when using getchar() in an
　　expression mapping.
解决方案:　Anticipate mod_mask to change. (idea by Yukihiro Nakadaira)
相关文件:　src/getchar.c

Patch 7.4.871
问题:　　Vim leaks memory, when 'wildignore' filters out all matches.
解决方案:　Free the files array when it becomes empty.
相关文件:　src/misc1.c

Patch 7.4.872
问题:　　Not using CI services available.
解决方案:　Add configuration files for travis and appveyor. (Ken Takata,
　　vim-jp, PR #401)
相关文件:　.travis.yml, appveyor.yml, Filelist

Patch 7.4.873 (after 7.4.866)
问题:　　Compiler warning for unused variable. (Tony Mechelynck)
解决方案:　Remove the variable. Also fix int vs long_u mixup.
相关文件:　src/if_xcmdsrv.c

Patch 7.4.874
问题:　　MS-Windows: When Vim runs inside another application, the size
　　isn't right.
解决方案:　When in child mode compute the size differently. (Agorgianitis
　　Loukas)
相关文件:　src/gui_w48.c

Patch 7.4.875
问题:　　Not obvious how to contribute.
解决方案:　Add a remark about CONTRIBUTING.md to README.md
相关文件:　README.md

Patch 7.4.876
问题:　　Windows7: when using vim.exe with msys or msys2, conhost.exe
　　(console window provider on Windows7) will freeze or crash.
解决方案:　Make original screen buffer active, before executing external
　　program. And when the program is finished, revert to vim's one.
　　(Taro Muraoka)
相关文件:　src/os_win32.c

Patch 7.4.877 (after 7.4.843)
问题:　　":find" sometimes fails. (Excanoe)
解决方案:　Compare current characters instead of previous ones.
相关文件:　src/misc2.c

Patch 7.4.878

问题:　　Coverity error for clearing only one byte of struct.
解决方案:　Clear the whole struct. (Dominique Pelle)
相关文件:　src/ex_docmd.c

Patch 7.4.879
问题:　　Can't see line numbers in nested function calls.
解决方案:　Add line number to the file name. (Alberto Fanjul)
相关文件:　src/eval.c

Patch 7.4.880
问题:　　No build and coverage status.
解决方案:　Add links to the README file. (Christian Brabandt)
相关文件:　README.md

Patch 7.4.881 (after 7.4.879)
问题:　　Test 49 fails.
解决方案:　Add line number to check of call stack.
相关文件:　src/testdir/test49.vim

Patch 7.4.882
问题:　　When leaving the command line window with CTRL-C while a
　　completion menu is displayed the menu isn't removed.
解决方案:　Force a screen update. (Hirohito Higashi)
相关文件:　src/edit.c

Patch 7.4.883 (after 7.4.818)
问题:　　Block-mode replace works characterwise instead of blockwise after
　　column 147. (Issue #422)
解决方案:　Set Visual mode. (Christian Brabandt)
相关文件:　src/normal.c, src/testdir/test_listlbr.in,
　　src/testdir/test_listlbr.ok

Patch 7.4.884
问题:　　Travis also builds on a tag push.
解决方案:　Filter out tag pushes. (Kenichi Ito)
相关文件:　.travis.yml

Patch 7.4.885
问题:　　When doing an upwards search without wildcards the search fails if
　　the initial directory doesn't exist.
解决方案:　Fix the non-wildcard case. (Stefan Kempf)
相关文件:　src/misc2.c

Patch 7.4.886 (after 7.4.876)
问题:　　Windows7: Switching screen buffer causes flicker when using
　　system().
解决方案:　Instead of actually switching screen buffer, duplicate the handle.
　　(Yasuhiro Matsumoto)
相关文件:　src/os_win32.c

Patch 7.4.887
问题:　　Using uninitialized memory for regexp with back reference.
　　(Dominique Pelle)
解决方案:　Initialize end_lnum.

相关文件: src/regexp_nfa.c

Patch 7.4.888
问题: The OptionSet autocommands are not triggered from setwinvar().
解决方案: Do not use switch_win() when not needed. (Hirohito Higashi)
相关文件: src/eval.c

Patch 7.4.889
问题: Triggering OptionSet from setwinvar() isn't tested.
解决方案: Add a test. (Christian Brabandt)
相关文件: src/testdir/test_autocmd_option.in,
src/testdir/test_autocmd_option.ok

Patch 7.4.890
问题: Build failure when using dynamic python but not python3.
解决方案: Adjust the #if to also include DYNAMIC_PYTHON3 and UNIX.
相关文件: src/if_python3.c

Patch 7.4.891
问题: Indentation of array initializer is wrong.
解决方案: Avoid that calling find_start_rawstring() changes the position
returned by find_start_comment(), add a test. (Hirohito Higashi)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.892
问题: On MS-Windows the iconv DLL may have a different name.
解决方案: Also try libiconv2.dll and libiconv-2.dll. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 7.4.893
问题: C indenting is wrong below a "case (foo):" because it is
recognized as a C++ base class construct. Issue #38.
解决方案: Check for the case keyword.
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.894
问题: vimrun.exe is picky about the number of spaces before -s.
解决方案: Skip all spaces. (Cam Sinclair)
相关文件: src/vimrun.c

Patch 7.4.895
问题: Custom command line completion does not work for a command
containing digits.
解决方案: Skip over the digits. (suggested by Yasuhiro Matsumoto)
相关文件: src/ex_docmd.c

Patch 7.4.896
问题: Editing a URL, which netrw should handle, doesn't work.
解决方案: Avoid changing slashes to backslashes. (Yasuhiro Matsumoto)
相关文件: src/fileio.c, src/os_mswin.c

Patch 7.4.897
问题: Freeze and crash when there is a sleep in a remote command.
(Karl Yngve Lervg)

解决方案: Remove a message from the queue before dealing with it. (James Kolb)
相关文件: src/if_xcmdsrv.c

Patch 7.4.898
问题: The 'fixendofline' option is set on with ":edit".
解决方案: Don't set the option when clearing a buffer. (Yasuhiro Matsumoto)
相关文件: src/buffer.c

Patch 7.4.899
问题: README file is not optimal.
解决方案: Move buttons, update some text. (closes #460)
相关文件: README.txt, README.md

Patch 7.4.900 (after 7.4.899)
问题: README file can still be improved
解决方案: Add a couple of links. (Christian Brabandt)
相关文件: README.md

Patch 7.4.901
问题: When a BufLeave autocommand changes folding in a way it syncs undo, undo can be corrupted.
解决方案: Prevent undo sync. (Jacob Niehus)
相关文件: src/popupmnu.c

Patch 7.4.902
问题: Problems with using the MS-Windows console.
解决方案: Revert patches 7.4.851, 7.4.876 and 7.4.886 until we find a better solution. (suggested by Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.903
问题: MS-Windows: When 'encoding' differs from the current code page, expanding wildcards may cause illegal memory access.
解决方案: Allocate a longer buffer. (Ken Takata)
相关文件: src/misc1.c

Patch 7.4.904
问题: Vim does not provide .desktop files.
解决方案: Include and install .desktop files. (James McCoy, closes #455)
相关文件: Filelist, runtime/vim.desktop, runtime/gvim.desktop, src/Makefile

Patch 7.4.905
问题: Python interface can produce error "vim.message' object has no attribute 'isatty'".
解决方案: Add dummy isatty(), readable(), etc. (closes #464)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.906
问题: On MS-Windows the viminfo file is (always) given the hidden attribute. (raulnac)
解决方案: Check the hidden attribute in a different way. (Ken Takata)
相关文件: src/ex_cmds.c, src/os_win32.c, src/os_win32.pro

Patch 7.4.907
问题: Libraries for dynamically loading interfaces can only be defined at compile time.
解决方案: Add options to specify the dll names. (Kazuki Sakamoto, closes #452)
相关文件: runtime/doc/if_lua.txt, runtime/doc/if_perl.txt, runtime/doc/if_pyth.txt, runtime/doc/if_ruby.txt, runtime/doc/options.txt, src/if_lua.c, src/if_perl.xs, src/if_python.c, src/if_python3.c, src/if_ruby.c, src/option.c, src/option.h

Patch 7.4.908 (after 7.4.907)
问题: Build error with MingW compiler. (Cesar Romani)
解决方案: Change #if into #ifdef.
相关文件: src/if_perl.xs

Patch 7.4.909 (after 7.4.905)
问题: "make install" fails.
解决方案: Only try installing desktop files if the destination directory exists.
相关文件: src/Makefile

Patch 7.4.910 (after 7.4.905)
问题: Compiler complains about type punned pointer.
解决方案: Use another way to increment the ref count.
相关文件: src/if_py_both.h

Patch 7.4.911
问题: t_Ce and t_Cs are documented but not supported. (Hirohito Higashi)
解决方案: Define the options.
相关文件: src/option.c

Patch 7.4.912
问题: Wrong indenting for C++ constructor.
解决方案: Recognize ::. (Anhong)
相关文件: src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 7.4.913
问题: No utf-8 support for the hangul input feature.
解决方案: Add utf-8 support. (Namsh)
相关文件: src/gui.c, src/hangulin.c, src/proto/hangulin.pro, src/screen.c, src/ui.c, runtime/doc/hangulin.txt, src/feature.h

Patch 7.4.914
问题: New compiler warning: logical-not-parentheses
解决方案: Silence the warning.
相关文件: src/term.c

Patch 7.4.915
问题: When removing from 'path' and then adding, a comma may go missing. (Malcolm Rowe)
解决方案: Fix the check for P_ONECOMMA. (closes #471)
相关文件: src/option.c, src/testdir/test_options.in,

src/testdir/test_options.ok

Patch 7.4.916
问题: When running out of memory while copying a dict memory may be freed twice. (ZyX)
解决方案: Do not call the garbage collector when running out of memory.
相关文件: src/misc2.c

Patch 7.4.917
问题: Compiler warning for comparing signed and unsigned.
解决方案: Add a type cast.
相关文件: src/hangulin.c

Patch 7.4.918
问题: A digit in an option name has problems.
解决方案: Rename 'python3dll' to 'pythonthreedll'.
相关文件: src/option.c, src/option.h, runtime/doc/options.txt

Patch 7.4.919
问题: The dll options are not in the options window.
解决方案: Add the dll options. And other fixes.
相关文件: runtime/optwin.vim

Patch 7.4.920
问题: The rubydll option is not in the options window.
解决方案: Add the rubydll option.
相关文件: runtime/optwin.vim

Patch 7.4.921 (after 7.4.906)
问题: Missing proto file update. (Randall W. Morris)
解决方案: Add the missing line for mch_ishidden.
相关文件: src/proto/os_win32.pro

Patch 7.4.922
问题: Leaking memory with ":helpt {dir-not-exists}".
解决方案: Free dirname. (Dominique Pelle)
相关文件: src/ex_cmds.c

Patch 7.4.923
问题: Prototypes not always generated.
解决方案: Change #if to OR with PROTO.
相关文件: src/window.c

Patch 7.4.924
问题: DEVELOPER_DIR gets reset by configure.
解决方案: Do not reset DEVELOPER_DIR when there is no --with-developer-dir argument. (Kazuki Sakamoto, closes #482)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.925
问题: User may yank or put using the register being recorded in.
解决方案: Add the recording register in the message. (Christian Brabandt, closes #470)
相关文件: runtime/doc/options.txt, runtime/doc/repeat.txt, src/ops.c,

src/option.h, src/screen.c

Patch 7.4.926
问题: Completing the longest match doesn't work properly with multi-byte characters.
解决方案: When using multi-byte characters use another way to find the longest match. (Hirohito Higashi)
相关文件: src/ex_getln.c, src/testdir/test_utf8.in, src/testdir/test_utf8.ok

Patch 7.4.927
问题: Ruby crashes when there is a runtime error.
解决方案: Use ruby_options() instead of ruby_process_options(). (Damien)
相关文件: src/if_ruby.c

Patch 7.4.928
问题: A clientserver message interrupts handling keys of a mapping.
解决方案: Have mch_inchar() send control back to WaitForChar when it is interrupted by server message. (James Kolb)
相关文件: src/os_unix.c

Patch 7.4.929
问题: "gv" after paste selects one character less if 'selection' is "exclusive".
解决方案: Increment the end position. (Christian Brabandt)
相关文件: src/normal.c, src/testdir/test94.in, src/testdir/test94.ok

Patch 7.4.930
问题: MS-Windows: Most users appear not to like the window border.
解决方案: Remove WS_EX_CLIENTEDGE. (Ian Halliday)
相关文件: src/gui_w32.c

Patch 7.4.931 (after 7.4.929)
问题: Test 94 fails on some systems.
解决方案: Set 'encoding' to utf-8.
相关文件: src/testdir/test94.in

Patch 7.4.932 (after 7.4.926)
问题: test_utf8 has confusing dummy command.
解决方案: Use a real command instead of a colon.
相关文件: src/testdir/test_utf8.in

Patch 7.4.933 (after 7.4.926)
问题: Crash when using longest completion match.
解决方案: Fix array index.
相关文件: src/ex_getln.c

Patch 7.4.934
问题: Appveyor also builds on a tag push.
解决方案: Add a skip_tags line. (Kenichi Ito, closes #489)
相关文件: appveyor.yml

Patch 7.4.935 (after 7.4.932)
问题: test_utf8 fails on MS-Windows when executed with gvim.
解决方案: Use the insert flag on feedkeys() to put the string before the

":" that was already read when checking for available chars.
相关文件: src/testdir/test_utf8.in

Patch 7.4.936
问题: Crash when dragging with the mouse.
解决方案: Add safety check for NULL pointer. Check mouse position for valid value. (Hirohito Higashi)
相关文件: src/window.c, src/term.c

Patch 7.4.937
问题: Segfault reading uninitialized memory.
解决方案: Do not read match \z0, it does not exist. (Marius Gedminas, closes #497)
相关文件: src/regexp_nfa.c

Patch 7.4.938
问题: X11 and GTK have more mouse buttons than Vim supports.
解决方案: Recognize more mouse buttons. (Benoit Pierre, closes #498)
相关文件: src/gui_gtk_x11.c, src/gui_x11.c

Patch 7.4.939
问题: Memory leak when encountering a syntax error.
解决方案: Free the memory. (Dominique Pelle)
相关文件: src/ex_docmd.c

Patch 7.4.940
问题: vt52 terminal codes are not correct.
解决方案: Move entries outside of #if. (Random) Adjustments based on documented codes.
相关文件: src/term.c

Patch 7.4.941
问题: There is no way to ignore case only for tag searches.
解决方案: Add the 'tagcase' option. (Gary Johnson)
相关文件: runtime/doc/options.txt, runtime/doc/quickref.txt, runtime/doc/tagsrch.txt, runtime/doc/usr_29.txt, runtime/optwin.vim, src/Makefile, src/buffer.c, src/option.c, src/option.h, src/structs.h, src/tag.c, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile, src/testdir/test_tagcase.in, src/testdir/test_tagcase.ok

Patch 7.4.942 (after 7.4.941)
问题: test_tagcase breaks for small builds.
解决方案: Bail out of the test early. (Hirohito Higashi)
相关文件: src/testdir/test_tagcase.in

Patch 7.4.943
问题: Tests are not run.
解决方案: Add test_writefile to makefiles. (Ken Takata)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_os2.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.944
问题: Writing tests for Vim script is hard.
解决方案: Add assertEqual(), assertFalse() and assertTrue() functions. Add the v:errors variable. Add the runtest script. Add a first new style test script.
相关文件: src/eval.c, src/vim.h, src/misc2.c, src/testdir/Makefile, src/testdir/runtest.vim, src/testdir/test_assert.vim, runtime/doc/eval.txt

Patch 7.4.945 (after 7.4.944)
问题: New style testing is incomplete.
解决方案: Add the runtest script to the list of distributed files. Add the new functions to the function overview. Rename the functions to match Vim function style. Move undolevels testing into a new style test script.
相关文件: Filelist, runtime/doc/usr_41.txt, runtime/doc/eval.txt, src/testdir/test_assert.vim, src/testdir/Makefile, src/testdir/test_undolevels.vim, src/testdir/test100.in, src/testdir/test100.ok

Patch 7.4.946 (after 7.4.945)
问题: Missing changes in source file.
解决方案: Include changes to the eval.c file.
相关文件: src/eval.c

Patch 7.4.947
问题: Test_listchars fails with MingW. (Michael Soyka)
解决方案: Add the test to the ones that need the fileformat fixed. (Christian Brabandt)
相关文件: src/testdir/Make_ming.mak

Patch 7.4.948
问题: Can't build when the insert_expand feature is disabled.
解决方案: Add #ifdefs. (Dan Pasanen, closes #499)
相关文件: src/eval.c, src/fileio.c

Patch 7.4.949
问题: When using 'colorcolumn' and there is a sign with a fullwidth character the highlighting is wrong. (Andrew Stewart)
解决方案: Only increment vcol when in the right state. (Christian Brabandt)
相关文件: src/screen.c, src/testdir/test_listlbr_utf8.in, src/testdir/test_listlbr_utf8.ok

Patch 7.4.950
问题: v:errors is not initialized.
解决方案: Initialize it to an empty list. (Thinca)
相关文件: src/eval.c

Patch 7.4.951
问题: Sorting number strings does not work as expected. (Luc Hermitte)
解决方案: Add the "N" argument to sort()
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test_alot.vim, src/testdir/test_sort.vim, src/testdir/Makefile

Patch 7.4.952
问题: 'lispwords' is tested in the old way.
解决方案: Make a new style test for 'lispwords'.
相关文件: src/testdir/test_alot.vim, src/testdir/test_lispwords.vim,
src/testdir/test100.in, src/testdir/test100.ok,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.953
问题: When a test script navigates to another buffer the .res file is
created with the wrong name.
解决方案: Use the "testname" for the .res file. (Damien)
相关文件: src/testdir/runtest.vim

Patch 7.4.954
问题: When using Lua there may be a crash. (issue #468)
解决方案: Avoid using an uninitialized tv. (Yukihiro Nakadaira)
相关文件: src/if_lua.c

Patch 7.4.955
问题: Vim doesn't recognize .pl6 and .pod6 files.
解决方案: Recognize them as perl6 and pod6. (Mike Eve, closes #511)
相关文件: runtime/filetype.vim

Patch 7.4.956
问题: A few more file name extensions not recognized.
解决方案: Add .asciidoc, .bzl, .gradle, etc.
相关文件: runtime/filetype.vim

Patch 7.4.957
问题: Test_tagcase fails when using another language than English.
解决方案: Set the messages language to C. (Kenichi Ito)
相关文件: src/testdir/test_tagcase.in

Patch 7.4.958
问题: Vim checks if the directory "$TMPDIR" exists.
解决方案: Do not check if the name starts with "$".
相关文件: src/fileio.c

Patch 7.4.959
问题: When setting 'term' the clipboard ownership is lost.
解决方案: Do not call clip_init(). (James McCoy)
相关文件: src/term.c

Patch 7.4.960
问题: Detecting every version of nmake is clumsy.
解决方案: Use a tiny C program to get the version of _MSC_VER. (Ken Takata)
相关文件: src/Make_mvc.mak

Patch 7.4.961
问题: Test107 fails in some circumstances.
解决方案: When using "zt", "zb" and "z=" recompute the fraction.

相关文件: src/normal.c, src/window.c, src/proto/window.pro

Patch 7.4.962
问题: Cannot run the tests with gvim. Cannot run individual new tests.
解决方案: Add the -f flag. Add new test targets in Makefile.
相关文件: src/Makefile, src/testdir/Makefile

Patch 7.4.963
问题: test_listlbr_utf8 sometimes fails.
解决方案: Don't use a literal multibyte character but <C-V>uXXXX. Do not dump the screen highlighting. (Christian Brabandt, closes #518)
相关文件: src/testdir/test_listlbr_utf8.in, src/testdir/test_listlbr_utf8.ok

Patch 7.4.964
问题: Test 87 doesn't work in a shadow directory.
解决方案: Handle the extra subdirectory. (James McCoy, closes #515)
相关文件: src/testdir/test87.in

Patch 7.4.965
问题: On FreeBSD /dev/fd/ files are special.
解决方案: Use is_dev_fd_file() also for FreeBSD. (Derek Schrock, closes #521)
相关文件: src/fileio.c

Patch 7.4.966
问题: Configure doesn't work with a space in a path.
解决方案: Put paths in quotes. (James McCoy, closes #525)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.967
问题: Cross compilation on MS-windows doesn't work well.
解决方案: Tidy up cross compilation across architectures with Visual Studio. (Mike Williams)
相关文件: src/Make_mvc.mak

Patch 7.4.968
问题: test86 and test87 are flaky in Appveyor.
解决方案: Reduce the count from 8 to 7. (suggested by ZyX)
相关文件: src/testdir/test86.in, src/testdir/test87.in

Patch 7.4.969
问题: Compiler warnings on Windows x64 build.
解决方案: Add type casts. (Mike Williams)
相关文件: src/option.c

Patch 7.4.970
问题: Rare crash in getvcol(). (Timo Mihaljov)
解决方案: Check for the buffer being NULL in init_preedit_start_col. (Hirohito Higashi, Christian Brabandt)
相关文件: src/mbyte.c

Patch 7.4.971
问题: The asin() function can't be used.
解决方案: Sort the function table properly. (Watiko)
相关文件: src/eval.c

Patch 7.4.972
问题: Memory leak when there is an error in setting an option.
解决方案: Free the saved value (Christian Brabandt)
相关文件: src/option.c

Patch 7.4.973
问题: When pasting on the command line line breaks result in literal
<CR> characters. This makes pasting a long file name difficult.
解决方案: Skip the characters.
相关文件: src/ex_getln.c, src/ops.c

Patch 7.4.974
问题: When using :diffsplit the cursor jumps to the first line.
解决方案: Put the cursor on the line related to where the cursor was before
the split.
相关文件: src/diff.c

Patch 7.4.975
问题: Using ":sort" on a very big file sometimes causes text to be
corrupted. (John Beckett)
解决方案: Copy the line into a buffer before calling ml_append().
相关文件: src/ex_cmds.c

Patch 7.4.976
问题: When compiling Vim for MSYS2 (linked with msys-2.0.dll), the Win32
clipboard is not enabled.
解决方案: Recognize MSYS like CYGWIN. (Ken Takata)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.977
问题: 'linebreak' does not work properly when using "space" in
'listchars'.
解决方案: (Hirohito Higashi, Christian Brabandt)
相关文件: src/screen.c, src/testdir/test_listlbr.in,
src/testdir/test_listlbr.ok

Patch 7.4.978
问题: test_cdo fails when using another language than English.
解决方案: Set the language to C. (Dominique Pelle, Kenichi Ito)
相关文件: src/testdir/test_cdo.in

Patch 7.4.979
问题: When changing the crypt key the blocks read from disk are not
decrypted.
解决方案: Also call ml_decrypt_data() when mf_old_key is set. (Ken Takata)
相关文件: src/memfile.c

Patch 7.4.980
问题: Tests for :cdo, :ldo, etc. are outdated.
解决方案: Add new style tests for these commands. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,

src/testdir/test_cdo.in, src/testdir/test_cdo.ok,
src/testdir/test_cdo.vim

Patch 7.4.981
问题: An error in a test script goes unnoticed.
解决方案: Source the test script inside try/catch. (Hirohito Higashi)
相关文件: src/testdir/runtest.vim

Patch 7.4.982
问题: Keeping the list of tests updated is a hassle.
解决方案: Move the list to a separate file, so that it only needs to be
updated in one place.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/Make_all.mak

Patch 7.4.983
问题: Executing one test after "make testclean" doesn't work.
解决方案: Add a dependency on test1.out.
相关文件: src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile,
src/testdir/Make_all.mak

Patch 7.4.984
问题: searchpos() always starts searching in the first column, which is
not what some people expect. (Brett Stahlman)
解决方案: Add the 'z' flag: start at the specified column.
相关文件: src/vim.h, src/eval.c, src/search.c,
src/testdir/test_searchpos.vim, src/testdir/test_alot.vim,
runtime/doc/eval.txt

Patch 7.4.985
问题: Can't build with Ruby 2.3.0.
解决方案: Use the new TypedData_XXX macro family instead of Data_XXX. Use
TypedData. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.4.986
问题: Test49 doesn't work on MS-Windows. test70 is listed twice.
解决方案: Move test49 to the group not used on Amiga and MS-Windows.
Remove test70 from SCRIPTS_WIN32.
相关文件: src/testdir/Make_all.mak, src/testdir/Make_dos.mak

Patch 7.4.987 (after 7.4.985)
问题: Can't build with Ruby 1.9.2.
解决方案: Require Rub 2.0 for defining USE_TYPEDDATA.
相关文件: src/if_ruby.c

Patch 7.4.988 (after 7.4.982)
问题: Default test target is test49.out.
解决方案: Add a build rule before including Make_all.mak.
相关文件: src/testdir/Make_dos.mak, src/testdir/Make_amiga.mak,

src/testdir/Make_ming.mak, src/testdir/Make_os2.mak,
src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.989
问题: Leaking memory when hash_add() fails. Coverity error 99126.
解决方案: When hash_add() fails free the memory.
相关文件: src/eval.c

Patch 7.4.990
问题: Test 86 fails on AppVeyor.
解决方案: Do some registry magic. (Ken Takata)
相关文件: appveyor.yml

Patch 7.4.991
问题: When running new style tests the output is not visible.
解决方案: Add the testdir/messages file and show it. Update the list of
test names.
相关文件: src/Makefile, src/testdir/Makefile, src/testdir/runtest.vim

Patch 7.4.992
问题: Makefiles for MS-Windows in src/po are outdated.
解决方案: Make them work. (Ken Takata, Taro Muraoka)
相关文件: src/po/Make_cyg.mak, src/po/Make_ming.mak, src/po/Make_mvc.mak,
src/po/README_mingw.txt, src/po/README_mvc.txt

Patch 7.4.993
问题: Test 87 is flaky on AppVeyor.
解决方案: Reduce the minimum background thread count.
相关文件: src/testdir/test86.in, src/testdir/test87.in

Patch 7.4.994
问题: New style tests are not run on MS-Windows.
解决方案: Add the new style tests.
相关文件: src/testdir/Make_dos.mak

Patch 7.4.995
问题: gdk_pixbuf_new_from_inline() is deprecated.
解决方案: Generate auto/gui_gtk_gresources.c. (Kazunobu Kuriyama,
closes #507)
相关文件: src/Makefile, src/auto/configure, src/config.h.in,
src/config.mk.in, src/configure.in, src/gui_gtk.c,
src/gui_gtk_gresources.xml, src/gui_gtk_x11.c,
src/proto/gui_gtk_gresources.pro,
pixmaps/stock_vim_build_tags.png, pixmaps/stock_vim_find_help.png,
pixmaps/stock_vim_save_all.png,
pixmaps/stock_vim_session_load.png,
pixmaps/stock_vim_session_new.png,
pixmaps/stock_vim_session_save.png, pixmaps/stock_vim_shell.png,
pixmaps/stock_vim_window_maximize.png,
pixmaps/stock_vim_window_maximize_width.png,
pixmaps/stock_vim_window_minimize.png,
pixmaps/stock_vim_window_minimize_width.png,
pixmaps/stock_vim_window_split.png,
pixmaps/stock_vim_window_split_vertical.png

Patch 7.4.996
问题: New GDK files and testdir/Make_all.mak missing from distribution.
PC build instructions are outdated.
解决方案: Add the file to the list. Update PC build instructions.
相关文件: Filelist, Makefile

Patch 7.4.997
问题: "make shadow" was sometimes broken.
解决方案: Add a test for it. (James McCoy, closes #520)
相关文件: .travis.yml

Patch 7.4.998
问题: Running tests in shadow directory fails. Test 49 fails.
解决方案: Link more files for the shadow directory. Make test 49 ends up in
the right buffer.
相关文件: src/Makefile, src/testdir/test49.in

Patch 7.4.999
问题: "make shadow" creates a broken link. (Tony Mechelynck)
解决方案: Remove vimrc.unix from the list.
相关文件: src/Makefile

Patch 7.4.1000
问题: Test 49 is slow and doesn't work on MS-Windows.
解决方案: Start moving parts of test 49 to test_viml.
相关文件: src/Makefile, src/testdir/runtest.vim, src/testdir/test_viml.vim,
src/testdir/test49.vim, src/testdir/test49.ok

Patch 7.4.1001 (after 7.4.1000)
问题: test_viml isn't run.
解决方案: Include change in makefile.
相关文件: src/testdir/Make_all.mak

Patch 7.4.1002
问题: Cannot run an individual test on MS-Windows.
解决方案: Move the rule to run test1 downwards. (Ken Takata)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.1003
问题: Travis could check a few more things.
解决方案: Run autoconf on one of the builds. (James McCoy, closes #510)
Also build with normal features.
相关文件: .travis.yml

Patch 7.4.1004
问题: Using Makefile when auto/config.mk does not exist results in
warnings.
解决方案: Use default values for essential variables.
相关文件: src/Makefile

Patch 7.4.1005
问题: Vim users are not always happy.
解决方案: Make them happy.

相关文件: src/ex_cmds.h, src/ex_cmds.c, src/proto/ex_cmds.pro

Patch 7.4.1006
问题: The fix in patch 7.3.192 is not tested.
解决方案: Add a test, one for each regexp engine. (Elias Diem)
相关文件: src/testdir/test44.in, src/testdir/test44.ok,
src/testdir/test99.in, src/testdir/test99.ok

Patch 7.4.1007
问题: When a symbolic link points to a file in the root directory, the
swapfile is not correct.
解决方案: Do not try getting the full name of a file in the root directory.
(Milly, closes #501)
相关文件: src/os_unix.c

Patch 7.4.1008
问题: The OS/2 code pollutes the source while nobody uses it these days.
解决方案: Drop the support for OS/2.
相关文件: src/feature.h, src/globals.h, src/macros.h, src/option.h,
src/os_unix.c, src/os_unix.h, src/proto/os_unix.pro, src/vim.h,
src/digraph.c, src/eval.c, src/ex_cmds.c, src/ex_docmd.c,
src/ex_getln.c, src/fileio.c, src/getchar.c, src/memline.c,
src/misc1.c, src/misc2.c, src/netbeans.c, src/option.c,
src/term.c, src/ui.c, src/window.c, src/os_os2_cfg.h,
src/Make_os2.mak, src/testdir/Make_os2.mak, src/testdir/os2.vim,
src/INSTALL, runtime/doc/os_os2.txt

Patch 7.4.1009
问题: There are still #ifdefs for ARCHIE.
解决方案: Remove references to ARCHIE, the code was removed in Vim 5.
相关文件: src/ex_cmds.c, src/ex_docmd.c, src/fileio.c, src/main.c,
src/memline.c, src/option.c, src/term.c

Patch 7.4.1010
问题: Some developers are unhappy while running tests.
解决方案: Add a test and some color.
相关文件: src/ex_cmds.c, src/testdir/test_assert.vim

Patch 7.4.1011
问题: Can't build with Strawberry Perl.
解决方案: Include stdbool.h. (Ken Takata, closes #328)
相关文件: Filelist, src/Make_mvc.mak, src/if_perl_msvc/stdbool.h

Patch 7.4.1012
问题: Vim overwrites the value of $PYTHONHOME.
解决方案: Do not set $PYTHONHOME if it is already set. (Kazuki Sakamoto,
closes #500)
相关文件: src/if_python.c, src/if_python3.c

Patch 7.4.1013
问题: The local value of 'errorformat' is not used for ":lexpr" and
":cexpr".
解决方案: Use the local value if it exists. (Christian Brabandt) Adjust the
help for this.

相关文件: runtime/doc/quickfix.txt, src/quickfix.c

Patch 7.4.1014
问题: ‘fnamemodify('.', ':.')‘ returns an empty string in Cygwin.
解决方案: Use CCP_RELATIVE in the call to cygwin_conv_path. (Jacob Niehus, closes #505)
相关文件: src/os_unix.c

Patch 7.4.1015
问题: The column is not restored properly when the matchparen plugin is used in Insert mode and the cursor is after the end of the line.
解决方案: Set the curswant flag. (Christian Brabandt). Also fix highlighting the match of the character before the cursor.
相关文件: src/eval.c, runtime/plugin/matchparen.vim

Patch 7.4.1016
问题: Still a few OS/2 pieces remain.
解决方案: Delete more.
相关文件: Filelist, README_os2.txt, testdir/todos.vim, src/xxd/Make_os2.mak

Patch 7.4.1017
问题: When there is a backslash in an option ":set -=" doesn't work.
解决方案: Handle a backslash better. (Jacob Niehus) Add a new test, merge in old test.
相关文件: src/testdir/test_cdo.vim, src/testdir/test_set.vim, src/testdir/test_alot.vim, src/option.c, src/testdir/test_set.in, src/testdir/test_set.ok, src/Makefile

Patch 7.4.1018 (after 7.4.1017)
问题: Failure running tests.
解决方案: Add missing change to list of old style tests.
相关文件: src/testdir/Make_all.mak

Patch 7.4.1019
问题: Directory listing of "src" is too long.
解决方案: Rename the resources file to make it shorter.
相关文件: src/gui_gtk_gresources.xml, src/gui_gtk_res.xml, src/Makefile, Filelist

Patch 7.4.1020
问题: On MS-Windows there is no target to run tests with gvim.
解决方案: Add the testgvim target.
相关文件: src/Make_mvc.mak

Patch 7.4.1021
问题: Some makefiles are outdated.
解决方案: Add a note to warn developers.
相关文件: src/Make_manx.mak, src/Make_bc3.mak, src/Make_bc5.mak, src/Make_djg.mak, src/Make_w16.mak

Patch 7.4.1022
问题: The README file contains some outdated information.
解决方案: Update the information about supported systems.
相关文件: README.txt, README.md

Patch 7.4.1023
问题: The distribution files for MS-Windows use CR-LF, which is inconsistent with what one gets from github.
解决方案: Use LF in the distribution files.
相关文件: Makefile

Patch 7.4.1024
问题: Interfaces for MS-Windows are outdated.
解决方案: Use Python 2.7.10, Python 3.4.4, Perl 5.22, TCL 8.6.
相关文件: src/bigvim.bat

Patch 7.4.1025
问题: Version in installer needs to be updated manually.
解决方案: Generate a file with the version number. (Guopeng Wen)
相关文件: Makefile, nsis/gvim.nsi, nsis/gvim_version.nsh

Patch 7.4.1026
问题: When using MingW the tests do not clean up all files. E.g. test 17 leaves Xdir1 behind. (Michael Soyka)
解决方案: Also delete directories, like Make_dos.mak. Delete files after directories to reduce warnings.
相关文件: src/testdir/Make_ming.mak, src/testdir/Make_dos.mak

Patch 7.4.1027
问题: No support for binary numbers.
解决方案: Add "bin" to 'nrformats'. (Jason Schulz)
相关文件: runtime/doc/change.txt, runtime/doc/eval.txt, runtime/doc/version7.txt, src/charset.c, src/eval.c, src/ex_cmds.c, src/ex_getln.c, src/misc2.c, src/ops.c, src/option.c, src/proto/charset.pro, src/spell.c, src/testdir/test57.in, src/testdir/test57.ok, src/testdir/test58.in, src/testdir/test58.ok, src/testdir/test_increment.in, src/testdir/test_increment.ok, src/vim.h

Patch 7.4.1028
问题: Nsis version file missing from the distribution.
解决方案: Add the file to the list.
相关文件: Filelist

Patch 7.4.1029 (after 7.4.1027)
问题: test_increment fails on systems with 32 bit long.
解决方案: Only test with 32 bits.
相关文件: src/testdir/test_increment.in, src/testdir/test_increment.ok

Patch 7.4.1030
问题: test49 is still slow.
解决方案: Move more tests from old to new style.
相关文件: src/testdir/test_viml.vim, src/testdir/test49.vim, src/testdir/test49.ok, src/testdir/runtest.vim

Patch 7.4.1031
问题: Can't build with Python interface using MingW.

解决方案: Update the Makefile. (Yasuhiro Matsumoto)
相关文件: src/INSTALLpc.txt, src/Make_cyg_ming.mak

Patch 7.4.1032
问题: message from assert_false() does not look nice.
解决方案: Handle missing sourcing_name. Use right number of spaces. (Watiko)
Don't use line number if it's zero.
相关文件: src/eval.c

Patch 7.4.1033
问题: Memory use on MS-Windows is very conservative.
解决方案: Use the global memory status to estimate amount of memory.
(Mike Williams)
相关文件: src/os_win32.c, src/os_win32.h, src/proto/os_win32.pro

Patch 7.4.1034
问题: There is no test for the 'backspace' option behavior.
解决方案: Add a test. (Hirohito Higashi)
相关文件: src/testdir/test_alot.vim, src/testdir/test_backspace_opt.vim

Patch 7.4.1035
问题: An Ex range gets adjusted for folded lines even when the range is
not using line numbers.
解决方案: Only adjust line numbers for folding. (Christian Brabandt)
相关文件: runtime/doc/fold.txt, src/ex_docmd.c

Patch 7.4.1036
问题: Only terminals with up to 256 colors work properly.
解决方案: Use the 256 color behavior for all terminals with 256 or more
colors. (Robert de Bath, closes #504)
相关文件: src/syntax.c

Patch 7.4.1037
问题: Using "q!" when there is a modified hidden buffer does not unload
the current buffer, resulting in the need to abandon it again.
解决方案: When using "q!" unload the current buffer when needed. (Yasuhiro
Matsumoto, Hirohito Higashi)
相关文件: src/testdir/test31.in, src/testdir/test31.ok,
runtime/doc/editing.txt, src/ex_cmds2.c, src/ex_docmd.c,
src/gui.c, src/gui_gtk_x11.c, src/os_unix.c,
src/proto/ex_cmds2.pro

Patch 7.4.1038
问题: Still get a warning for a deprecated function with gdk-pixbuf
2.31.
解决方案: Change minimum minor version from 32 to 31.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.1039 (after 7.4.1037)
问题: Test 31 fails with small build.
解决方案: Bail out for small build. (Hirohito Higashi)
相关文件: src/testdir/test31.in

Patch 7.4.1040

问题: The tee command is not available on MS-Windows.
解决方案: Adjust tee.c for MSVC and add a makefile. (Yasuhiro Matsumoto)
相关文件: src/tee/tee.c, src/tee/Make_mvc.mak, src/Make_mvc.mak

Patch 7.4.1041
问题: Various small things.
解决方案: Add file to list of distributed files. Adjust README. Fix typo.
相关文件: Filelist, src/testdir/README.txt, src/testdir/test_charsearch.in,
            src/INSTALLmac.txt

Patch 7.4.1042
问题: g-CTRL-G shows the word count, but there is no way to get the word
            count in a script.
解决方案: Add the wordcount() function. (Christian Brabandt)
相关文件: runtime/doc/editing.txt, runtime/doc/eval.txt,
            runtime/doc/usr_41.txt, src/eval.c, src/normal.c, src/ops.c,
            src/proto/ops.pro, src/testdir/test_wordcount.in,
            src/testdir/test_wordcount.ok, src/testdir/Make_all.mak

Patch 7.4.1043
问题: Another small thing.
解决方案: Now really update the Mac install text.
相关文件: src/INSTALLmac.txt

Patch 7.4.1044 (after 7.4.1042)
问题: Can't build without the +eval feature.
解决方案: Add #ifdef.
相关文件: src/ops.c

Patch 7.4.1045
问题: Having shadow and coverage on the same build results in the source
            files not being available in the coverage view.
解决方案: Move using shadow to the normal build.
相关文件: .travis.yml

Patch 7.4.1046
问题: No test coverage for menus.
解决方案: Load the standard menus and check there is no error.
相关文件: src/testdir/test_menu.vim, src/testdir/test_alot.vim

Patch 7.4.1047 (after patch 7.4.1042)
问题: Tests fail on MS-Windows.
解决方案: Set 'selection' to inclusive.
相关文件: src/testdir/test_wordcount.in

Patch 7.4.1048 (after patch 7.4.1047)
问题: Wordcount test still fail on MS-Windows.
解决方案: Set 'fileformat' to "unix".
相关文件: src/testdir/test_wordcount.in

Patch 7.4.1049 (after patch 7.4.1048)
问题: Wordcount test still fails on MS-Windows.
解决方案: Set 'fileformats' to "unix".
相关文件: src/testdir/test_wordcount.in

Patch 7.4.1050
问题: Warning for unused var with tiny features. (Tony Mechelynck)
解决方案: Add #ifdef. Use vim_snprintf(). Reduce number of statements.
相关文件: src/ops.c

Patch 7.4.1051
问题: Segfault when unletting "count".
解决方案: Check for readonly and locked first. (Dominique Pelle)
            Add a test.
相关文件: src/eval.c, src/testdir/test_alot.vim, src/testdir/test_unlet.vim

Patch 7.4.1052
问题: Illegal memory access with weird syntax command. (Dominique Pelle)
解决方案: Check for column past end of line.
相关文件: src/syntax.c

Patch 7.4.1053
问题: Insufficient testing for quickfix commands.
解决方案: Add a new style quickfix test. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/test_quickfix.vim

Patch 7.4.1054
问题: Illegal memory access.
解决方案: Check for missing pattern. (Dominique Pelle)
相关文件: src/syntax.c

Patch 7.4.1055
问题: Running "make newtests" in src/testdir has no output.
解决方案: List the messages file when a test fails. (Christian Brabandt)
            Update the list of tests.
相关文件: src/Makefile, src/testdir/Makefile

Patch 7.4.1056
问题: Don't know why finding spell suggestions is slow.
解决方案: Add some code to gather profiling information.
相关文件: src/spell.c

Patch 7.4.1057
问题: Typos in the :options window.
解决方案: Fix the typos. (Dominique Pelle)
相关文件: runtime/optwin.vim

Patch 7.4.1058
问题: It is not possible to test code that is only reached when memory
            allocation fails.
解决方案: Add the alloc_fail() function. Try it out with :vimgrep.
相关文件: runtime/doc/eval.txt, src/globals.h, src/eval.c, src/quickfix.c,
            src/misc2.c, src/proto/misc2.pro, src/testdir/test_quickfix.vim

Patch 7.4.1059
问题: Code will never be executed.
解决方案: Remove the code.
相关文件: src/quickfix.c

Patch 7.4.1060
问题: Instructions for writing tests are outdated.
解决方案: Mention Make_all.mak. Add steps for new style tests.
相关文件: src/testdir/README.txt

Patch 7.4.1061
问题: Compiler warning for ignoring return value of fwrite().
解决方案: Do use the return value. (idea: Charles Campbell)
相关文件: src/misc2.c, src/proto/misc2.pro

Patch 7.4.1062
问题: Building with Ruby on MS-Windows requires a lot of arguments.
解决方案: Make it simpler. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 7.4.1063
问题: TCL_VER_LONG and DYNAMIC_TCL_VER are not set when building with
Cygwin and MingW.
解决方案: Add TCL_VER_LONG and DYNAMIC_TCL_VER to the makefile. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1064
问题: When a spell file has single letter compounding creating
suggestions takes an awful long time.
解决方案: Add the NOCOMPOUNDSUGS flag.
相关文件: runtime/doc/spell.txt, src/spell.c

Patch 7.4.1065
问题: Cannot use the "dll" options on MS-Windows.
解决方案: Support the options on all platforms. Use the built-in name as
the default, so that it's clear what Vim is looking for.
相关文件: src/if_python.c, src/if_python3.c, src/if_lua.c, src/if_perl.xs,
src/if_ruby.c, src/option.c, runtime/doc/options.txt, src/Makefile

Patch 7.4.1066 (after 7.4.1065)
问题: Build fails on MS-Windows.
解决方案: Adjust the #ifdefs for "dll" options.
相关文件: src/option.h

Patch 7.4.1067 (after 7.4.1065)
问题: Can't build with MingW and Python on MS-Windows.
解决方案: Move the build flags to CFLAGS.
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1068
问题: Wrong way to check for unletting internal variables.
解决方案: Use a better way. (Olaf Dabrunz)
相关文件: src/testdir/test_unlet.c, src/eval.c

Patch 7.4.1069
问题: Compiler warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/misc2.c

Patch 7.4.1070
问题: The Tcl interface can't be loaded dynamically on Unix.
解决方案: Make it possible to load it dynamically. (Ken Takata)
相关文件: runtime/doc/if_tcl.txt, runtime/doc/options.txt,
runtime/doc/quickref.txt, runtime/optwin.vim, src/Makefile,
src/config.h.in, src/configure.in, src/auto/configure,
src/if_tcl.c, src/option.c, src/option.h

Patch 7.4.1071
问题: New style tests are executed in arbitrary order.
解决方案: Sort the test function names. (Hirohito Higashi)
Fix the quickfix test that depended on the order.
相关文件: src/testdir/runtest.vim, src/testdir/test_quickfix.vim

Patch 7.4.1072
问题: Increment test is old style.
解决方案: Make the increment test a new style test. (Hirohito Higashi)
相关文件: src/Makefile, src/testdir/Make_all.mak,
src/testdir/test_increment.in, src/testdir/test_increment.ok,
src/testdir/test_increment.vim

Patch 7.4.1073
问题: Alloc_id depends on numbers, may use the same one twice. It's not
clear from the number what it's for.
解决方案: Use an enum. Add a function to lookup the enum value from the
name.
相关文件: src/misc2.c, src/vim.h, src/alloc.h, src/globals.h,
src/testdir/runtest.vim, src/proto/misc2.pro,
src/testdir/test_quickfix.vim

Patch 7.4.1074
问题: Warning from VC2015 compiler.
解决方案: Add a type cast. (Mike Williams)
相关文件: src/gui_dwrite.cpp

Patch 7.4.1075
问题: Crash when using an invalid command.
解决方案: Fix generating the error message. (Dominique Pelle)
相关文件: src/ex_docmd.c

Patch 7.4.1076
问题: CTRL-A does not work well in right-left mode.
解决方案: Remove reversing the line, add a test. (Hirohito Higashi)
相关文件: src/ops.c, src/testdir/test_increment.vim

Patch 7.4.1077
问题: The build instructions for MS-Windows are incomplete.
解决方案: Add explanations for how to build with various interfaces. (Ken
Takata)
相关文件: src/INSTALLpc.txt

Patch 7.4.1078
问题: MSVC: "make clean" doesn't cleanup in the tee directory.

解决方案: Add the commands to cleanup tee. (Erich Ritz)
相关文件: src/Make_mvc.mak

Patch 7.4.1079 (after 7.4.1073)
问题: New include file missing from distribution. Missing changes to
quickfix code.
解决方案: Add alloc.h to the list of distributed files. Use the enum in
quickfix code.
相关文件: Filelist, src/quickfix.c

Patch 7.4.1080
问题: VS2015 has a function HandleToLong() that is shadowed by the macro
that Vim defines.
解决方案: Do not define HandleToLong() for MSVC version 1400 and later.
(Mike Williams)
相关文件: src/gui_w32.c

Patch 7.4.1081
问题: No test for what previously caused a crash.
解决方案: Add test for unletting errmsg.
相关文件: src/testdir/test_unlet.vim

Patch 7.4.1082
问题: The Tcl interface is always skipping memory free on exit.
解决方案: Only skip for dynamically loaded Tcl.
相关文件: src/if_tcl.c

Patch 7.4.1083
问题: Building GvimExt with VS2015 may fail.
解决方案: Adjust the makefile. (Mike Williams)
相关文件: src/GvimExt/Makefile

Patch 7.4.1084
问题: Using "." to repeat CTRL-A in Visual mode increments the wrong
numbers.
解决方案: Append right size to the redo buffer. (Ozaki Kiichi)
相关文件: src/normal.c, src/testdir/test_increment.vim

Patch 7.4.1085
问题: The CTRL-A and CTRL-X commands do not update the '[and '] marks.
解决方案: (Yukihiro Nakadaira)
相关文件: src/ops.c, src/testdir/test_marks.in, src/testdir/test_marks.ok

Patch 7.4.1086
问题: Crash with an extremely long buffer name.
解决方案: Limit the return value of vim_snprintf(). (Dominique Pelle)
相关文件: src/buffer.c

Patch 7.4.1087
问题: CTRL-A and CTRL-X do not work properly with blockwise visual
selection if there is a mix of Tab and spaces.
解决方案: Add OP_NR_ADD and OP_NR_SUB. (Hirohito Higashi)
相关文件: src/testdir/test_increment.vim, src/normal.c, src/ops.c,
src/proto/ops.pro, src/vim.h

Patch 7.4.1088
问题: Coverity warns for uninitialized variables. Only one is an actual problem.
解决方案: Move the conditions. Don't use endpos if handling an error.
相关文件: src/ops.c

Patch 7.4.1089
问题: Repeating CTRL-A doesn't work.
解决方案: Call prep_redo_cmd(). (Hirohito Higashi)
相关文件: src/normal.c, src/testdir/test_increment.vim

Patch 7.4.1090
问题: No tests for :hardcopy and related options.
解决方案: Add test_hardcopy.
相关文件: src/testdir/test_hardcopy.vim, src/Makefile, src/testdir/Make_all.mak

Patch 7.4.1091
问题: When making a change while need_wait_return is set there is a two second delay.
解决方案: Do not assume the ATTENTION prompt was given when need_wait_return was set already.
相关文件: src/misc1.c

Patch 7.4.1092
问题: It is not simple to test for an exception and give a proper error message.
解决方案: Add assert_exception().
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 7.4.1093
问题: Typo in test goes unnoticed.
解决方案: Fix the typo. Give error for wrong arguments to cursor(). (partly by Hirohito Higashi) Add a test for cursor().
相关文件: src/testdir/test_searchpos.vim, src/testdir/test_cursor_func.vim, src/eval.c, src/testdir/test_alot.vim

Patch 7.4.1094
问题: Test for :hardcopy fails on MS-Windows.
解决方案: Check for the +postscript feature.
相关文件: src/testdir/test_hardcopy.vim

Patch 7.4.1095
问题: Can't build GvimExt with SDK 7.1.
解决方案: Support using setenv.bat instead of vcvars32.bat. (Ken Takata)
相关文件: src/Make_mvc.mak, src/GvimExt/Makefile

Patch 7.4.1096
问题: Need several lines to verify a command produces an error.
解决方案: Add assert_fails(). (suggested by Nikolai Pavlov) Make the quickfix alloc test actually work.
相关文件: src/testdir/test_quickfix.vim, src/eval.c, runtime/doc/eval.txt, src/misc2.c, src/alloc.h

Patch 7.4.1097
问题: Looking up the alloc ID for tests fails.
解决方案: Fix the line computation. Use assert_fails() for unlet test.
相关文件: src/testdir/runtest.vim, src/testdir/test_unlet.vim

Patch 7.4.1098
问题: Still using old style C function declarations.
解决方案: Always define __ARGS() to include types. Turn a few functions into ANSI style to find out if this causes problems for anyone.
相关文件: src/vim.h, src/os_unix.h, src/eval.c, src/main.c

Patch 7.4.1099
问题: It's not easy to know if Vim supports blowfish. (Smu Johnson)
解决方案: Add has('crypt-blowfish') and has('crypt-blowfish2').
相关文件: src/eval.c

Patch 7.4.1100
问题: Cygwin makefiles are unused.
解决方案: Remove them.
相关文件: src/GvimExt/Make_ming.mak, src/GvimExt/Make_cyg.mak, src/xxd/Make_ming.mak, src/xxd/Make_cyg.mak

Patch 7.4.1101
问题: With 'rightleft' and concealing the cursor may move to the wrong position.
解决方案: Compute the column differently when 'rightleft' is set. (Hirohito Higashi)
相关文件: src/screen.c

Patch 7.4.1102
问题: Debugger has no stack backtrace support.
解决方案: Add "backtrace", "frame", "up" and "down" commands. (Alberto Fanjul, closes #433)
相关文件: runtime/doc/repeat.txt, src/eval.c, src/ex_cmds2.c, src/globals.h, src/testdir/Make_all.mak, src/testdir/test108.in, src/testdir/test108.ok

Patch 7.4.1103 (after 7.4.1100)
问题: Removed file still in distribution.
解决方案: Remove Make_cyg.mak from the list of files.
相关文件: Filelist

Patch 7.4.1104
问题: Various problems building with MzScheme/Racket.
解决方案: Make it work with new versions of Racket. (Yukihiro Nakadaira, Ken Takata)
相关文件: runtime/doc/if_mzsch.txt, src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/auto/configure, src/configure.in, src/if_mzsch.c

Patch 7.4.1105
问题: When using slices there is a mixup of variable name and namespace.
解决方案: Recognize variables that can't be a namespace. (Hirohito Higashi)

相关文件: src/eval.c, src/testdir/test_eval.in, src/testdir/test_eval.ok

Patch 7.4.1106
问题: The nsis script can't be used from the appveyor build.
解决方案: Add "ifndef" to allow for variables to be set from the command line. Remove duplicate SetCompressor command. Support using other gettext binaries. (Ken Takata) Update build instructions to use libintl-8.dll.
相关文件: Makefile, nsis/gvim.nsi, src/os_win32.c, src/proto/os_win32.pro, src/main.c, os_w32exe.c

Patch 7.4.1107
问题: Vim can create a directory but not delete it.
解决方案: Add an argument to delete() to make it possible to delete a directory, also recursively.
相关文件: src/fileio.c, src/eval.c, src/proto/fileio.pro, src/testdir/test_delete.vim, src/testdir/test_alot.vim, runtime/doc/eval.txt

Patch 7.4.1108
问题: Expanding "~" halfway a file name.
解决方案: Handle the file name as one name. (Marco Hinz) Add a test. Closes #564.
相关文件: src/testdir/test27.in, src/testdir/test27.ok, src/testdir/test_expand.vim, src/testdir/test_alot.vim, src/Makefile, src/misc2.c

Patch 7.4.1109 (after 7.4.1107)
问题: MS-Windows doesn't have rmdir().
解决方案: Add mch_rmdir().
相关文件: src/os_win32.c, src/proto/os_win32.pro

Patch 7.4.1110
问题: Test 108 fails when language is French.
解决方案: Force English messages. (Dominique Pelle)
相关文件: src/testdir/test108.in

Patch 7.4.1111
问题: test_expand fails on MS-Windows.
解决方案: Always use forward slashes. Remove references to test27.
相关文件: src/testdir/runtest.vim, src/testdir/test_expand.vim, src/testdir/Make_dos.mak, src/testdir/Make_all.mak, src/testdir/Make_amiga.mak, src/testdir/Make_ming.mak

Patch 7.4.1112
问题: When using ":next" with an illegal file name no error is reported.
解决方案: Give an error message.
相关文件: src/ex_cmds2.c

Patch 7.4.1113 (after 7.4.1105)
问题: Using {ns} in variable name does not work. (lilydjwg)
解决方案: Fix recognizing colon. Add a test.
相关文件: src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1114 (after 7.4.1107)
问题: delete() does not work well with symbolic links.
解决方案: Recognize symbolic links.
相关文件: src/eval.c, src/fileio.c, src/os_unix.c, src/proto/os_unix.pro, src/testdir/test_delete.vim, runtime/doc/eval.txt

Patch 7.4.1115
问题: MS-Windows: make clean in testdir doesn't clean everything.
解决方案: Add command to delete X* directories. (Ken Takata)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.1116
问题: delete(x, 'rf') does not delete files starting with a dot.
解决方案: Also delete files starting with a dot.
相关文件: src/misc1.c, src/fileio.c, src/vim.h

Patch 7.4.1117 (after 7.4.1116)
问题: No longer get "." and ".." in directory list.
解决方案: Do not skip "." and ".." unless EW_DODOT is set.
相关文件: src/misc1.c

Patch 7.4.1118
问题: Tests hang in 24 line terminal.
解决方案: Set the 'more' option off.
相关文件: src/testdir/runtest.vim

Patch 7.4.1119
问题: argidx() has a wrong value after ":%argdelete". (Yegappan Lakshmanan)
解决方案: Correct the value of w_arg_idx. Add a test.
相关文件: src/ex_cmds2.c, src/testdir/test_arglist.vim, src/testdir/Make_all.mak

Patch 7.4.1120
问题: delete(x, 'rf') fails if a directory is empty. (Lcd)
解决方案: Ignore not finding matches in an empty directory.
相关文件: src/fileio.c, src/misc1.c, src/vim.h, src/testdir/test_delete.vim

Patch 7.4.1121
问题: test_expand leaves files behind.
解决方案: Edit another file before deleting, otherwise the swap file remains.
相关文件: src/testdir/test_expand.vim

Patch 7.4.1122
问题: Test 92 and 93 fail when using gvim on a system with a non utf-8 locale.
解决方案: Avoid using .gvimrc by adding -U NONE. (Yukihiro Nakadaira)
相关文件: src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_vms.mms, src/testdir/Makefile

Patch 7.4.1123
问题: Using ":argadd" when there are no arguments results in the second argument to be the current one. (Yegappan Lakshmanan)

解决方案: Correct the w_arg_idx value.
相关文件: src/ex_cmds2.c, src/testdir/test_arglist.vim

Patch 7.4.1124
问题: MS-Windows: dead key behavior is not ideal.
解决方案: Handle dead keys differently when not in Insert or Select mode.
           (John Wellesz, closes #399)
相关文件: src/gui_w48.c

Patch 7.4.1125
问题: There is no perleval().
解决方案: Add perleval(). (Damien)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/eval.c,
           src/if_perl.xs, src/proto/if_perl.pro, src/testdir/Make_all.mak,
           src/testdir/test_perl.vim

Patch 7.4.1126
问题: Can only get the directory of the current window.
解决方案: Add window and tab arguments to getcwd() and haslocaldir().
           (Thinca, Hirohito Higashi)
相关文件: src/Makefile, src/testdir/Make_all.mak,
           src/testdir/test_getcwd.in, src/testdir/test_getcwd.ok,
           runtime/doc/eval.txt, patching file src/eval.c

Patch 7.4.1127
问题: Both old and new style tests for Perl.
解决方案: Merge the old tests with the new style tests.
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_perl.in,
           src/testdir/test_perl.ok, src/testdir/test_perl.vim

Patch 7.4.1128
问题: MS-Windows: delete() does not recognize junctions.
解决方案: Add mch_isrealdir() for MS-Windows. Update mch_is_symbolic_link().
           (Ken Takata)
相关文件: src/fileio.c, src/os_win32.c, src/proto/os_win32.pro

Patch 7.4.1129
问题: Python None value can't be converted to a Vim value.
解决方案: Just use zero. (Damien)
相关文件: src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok,
           src/testdir/test87.in, src/testdir/test87.ok,

Patch 7.4.1130
问题: Memory leak in :vimgrep.
解决方案: Call FreeWild(). (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 7.4.1131
问题: New lines in the viminfo file are dropped.
解决方案: Copy lines starting with "|". Fix that when using :rviminfo in a
           function global variables were restored as function-local
           variables.
相关文件: src/eval.c, src/structs.h, src/ex_cmds.c, src/misc2.c,
           src/proto/misc2.pro, src/testdir/test_viminfo.vim,

src/testdir/Make_all.mak, src/testdir/test74.in,
src/testdir/test74.ok

Patch 7.4.1132
问题: Old style tests for the argument list.
解决方案: Add more new style tests. (Yegappan Lakshmanan)
相关文件: src/testdir/test_arglist.vim, src/testdir/test_argument_0count.in,
src/testdir/test_argument_0count.ok,
src/testdir/test_argument_count.in, src/Makefile,
src/testdir/test_argument_count.ok, src/testdir/Make_all.mak

Patch 7.4.1133
问题: Generated function prototypes still have __ARGS().
解决方案: Generate function prototypes without __ARGS().
相关文件: src/Makefile, src/if_ruby.c, src/os_win32.c,
src/proto/blowfish.pro, src/proto/buffer.pro,
src/proto/charset.pro, src/proto/crypt.pro,
src/proto/crypt_zip.pro, src/proto/diff.pro,
src/proto/digraph.pro, src/proto/edit.pro, src/proto/eval.pro,
src/proto/ex_cmds2.pro, src/proto/ex_cmds.pro,
src/proto/ex_docmd.pro, src/proto/ex_eval.pro,
src/proto/ex_getln.pro, src/proto/fileio.pro, src/proto/fold.pro,
src/proto/getchar.pro, src/proto/gui_athena.pro,
src/proto/gui_beval.pro, src/proto/gui_gtk_gresources.pro,
src/proto/gui_gtk.pro, src/proto/gui_gtk_x11.pro,
src/proto/gui_mac.pro, src/proto/gui_motif.pro,
src/proto/gui_photon.pro, src/proto/gui.pro,
src/proto/gui_w16.pro, src/proto/gui_w32.pro,
src/proto/gui_x11.pro, src/proto/gui_xmdlg.pro,
src/proto/hangulin.pro, src/proto/hardcopy.pro,
src/proto/hashtab.pro, src/proto/if_cscope.pro,
src/proto/if_lua.pro, src/proto/if_mzsch.pro,
src/proto/if_ole.pro, src/proto/if_perl.pro,
src/proto/if_perlsfio.pro, src/proto/if_python3.pro,
src/proto/if_python.pro, src/proto/if_ruby.pro,
src/proto/if_tcl.pro, src/proto/if_xcmdsrv.pro,
src/proto/main.pro, src/proto/mark.pro, src/proto/mbyte.pro,
src/proto/memfile.pro, src/proto/memline.pro, src/proto/menu.pro,
src/proto/message.pro, src/proto/misc1.pro, src/proto/misc2.pro,
src/proto/move.pro, src/proto/netbeans.pro, src/proto/normal.pro,
src/proto/ops.pro, src/proto/option.pro, src/proto/os_amiga.pro,
src/proto/os_beos.pro, src/proto/os_mac_conv.pro,
src/proto/os_msdos.pro, src/proto/os_mswin.pro,
src/proto/os_qnx.pro, src/proto/os_unix.pro, src/proto/os_vms.pro,
src/proto/os_win16.pro, src/proto/os_win32.pro,
src/proto/popupmnu.pro, src/proto/pty.pro, src/proto/quickfix.pro,
src/proto/regexp.pro, src/proto/screen.pro, src/proto/search.pro,
src/proto/sha256.pro, src/proto/spell.pro, src/proto/syntax.pro,
src/proto/tag.pro, src/proto/termlib.pro, src/proto/term.pro,
src/proto/ui.pro, src/proto/undo.pro, src/proto/version.pro,
src/proto/winclip.pro, src/proto/window.pro,
src/proto/workshop.pro

Patch 7.4.1134

问题: The arglist test fails on MS-Windows.
解决方案: Only check for failure of argedit on Unix.
相关文件: src/testdir/test_arglist.vim

Patch 7.4.1135
问题: One more arglist test fails on MS-Windows.
解决方案: Don't edit "Y" after editing "y".
相关文件: src/testdir/test_arglist.vim

Patch 7.4.1136
问题: Wrong argument to assert_exception() causes a crash. (reported by
            Coverity)
解决方案: Check for NULL pointer. Add a test.
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 7.4.1137
问题: Illegal memory access when using :copen and :cclose.
解决方案: Avoid that curbuf is invalid. (suggestion by Justin M. Keyes)
            Add a test.
相关文件: src/window.c, src/testdir/test_quickfix.vim

Patch 7.4.1138
问题: When running gvim in the foreground some icons are missing.
            (Taylor Venable)
解决方案: Move the call to gui_gtk_register_resource(). (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1139
问题: MS-Windows: getftype() returns "file" for symlink to directory.
解决方案: Make it return "dir". (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.1140
问题: Recognizing <sid> does not work when the language is Turkish.
            (Christian Brabandt)
解决方案: Use MB_STNICMP() instead of STNICMP().
相关文件: src/eval.c

Patch 7.4.1141
问题: Using searchpair() with a skip expression that uses syntax
            highlighting sometimes doesn't work. (David Fishburn)
解决方案: Reset next_match_idx. (Christian Brabandt)
相关文件: src/syntax.c

Patch 7.4.1142
问题: Cannot define keyword characters for a syntax file.
解决方案: Add the ":syn iskeyword" command. (Christian Brabandt)
相关文件: runtime/doc/options.txt, runtime/doc/syntax.txt, src/buffer.c,
            src/option.c, src/structs.h, src/syntax.c,
            src/testdir/Make_all.mak, src/testdir/test_syntax.vim

Patch 7.4.1143
问题: Can't sort on floating point numbers.
解决方案: Add the "f" flag to ":sort". (Alex Jakushev) Also add the "f"

flag to sort().
相关文件: runtime/doc/change.txt, src/ex_cmds.c, src/testdir/test_sort.vim, src/testdir/test57.in, src/testdir/test57.ok, src/eval.c

Patch 7.4.1144 (after 7.4.1143)
问题: Can't build on several systems.
解决方案: Include float.h. (Christian Robinson, closes #570 #571)
相关文件: src/ex_cmds.c

Patch 7.4.1145
问题: Default features are conservative.
解决方案: Make the default feature set for most of today's systems "huge".
相关文件: src/feature.h, src/configure.in, src/auto/configure

Patch 7.4.1146
问题: Can't build with Python 3 interface using MingW.
解决方案: Update the Makefile. (Yasuhiro Matsumoto, Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1147
问题: Conflict for "chartab". (Kazunobu Kuriyama)
解决方案: Rename the global one to something less obvious. Move it into src/chartab.c.
相关文件: src/macros.h, src/globals.h, src/charset.c, src/main.c, src/option.c, src/screen.c, src/vim.h

Patch 7.4.1148
问题: Default for MingW and Cygwin is still "normal".
解决方案: Use "huge" as default. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 7.4.1149 (after 7.4.1013)
问题: Using the local value of 'errorformat' causes more problems than it solves.
解决方案: Revert 7.4.1013.
相关文件: runtime/doc/quickfix.txt, src/quickfix.c

Patch 7.4.1150
问题: 'langmap' applies to the first character typed in Select mode. (David Watson)
解决方案: Check for SELECTMODE. (Christian Brabandt, closes #572)
Add the 'x' flag to feedkeys().
相关文件: src/getchar.c, src/normal.c, src/testdir/test_langmap.vim, src/ex_docmd.c, src/proto/ex_docmd.pro, src/testdir/Make_all.mak, runtime/doc/eval.txt

Patch 7.4.1151 (after 7.4.1150)
问题: Missing change to eval.c
解决方案: Also change feedkeys().
相关文件: src/eval.c

Patch 7.4.1152
问题: Langmap test fails with normal build.
解决方案: Check for +langmap feature.

相关文件: src/testdir/test_langmap.vim

Patch 7.4.1153
问题: Autocommands triggered by quickfix cannot always get the current
title value.
解决方案: Call qf_fill_buffer() later. (Christian Brabandt)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1154
问题: No support for JSON.
解决方案: Add jsonencode() and jsondecode(). Also add v:false, v:true,
v:null and v:none.
相关文件: src/json.c, src/eval.c, src/proto.h, src/structs.h, src/vim.h,
src/if_lua.c, src/if_mzsch.c, src/if_ruby.c, src/if_py_both.h,
src/globals.h, src/Makefile, src/Make_bc3.mak, src/Make_bc5.mak,
src/Make_cyg_ming.mak, src/Make_dice.mak, src/Make_ivc.mak,
src/Make_manx.mak, src/Make_morph.mak, src/Make_mvc.mak,
src/Make_sas.mak, src/Make_vms.mms, src/proto/json.pro,
src/proto/eval.pro, src/testdir/test_json.vim,
src/testdir/test_alot.vim, Filelist, runtime/doc/eval.txt

Patch 7.4.1155
问题: Build with normal features fails.
解决方案: Always define dict_lookup().
相关文件: src/eval.c

Patch 7.4.1156
问题: Coverity warns for NULL pointer and ignoring return value.
解决方案: Check for NULL pointer. When dict_add() returns FAIL free the item.
相关文件: src/json.c

Patch 7.4.1157
问题: type() does not work for v:true, v:none, etc.
解决方案: Add new type numbers.
相关文件: src/eval.c, src/testdir/test_json.vim, src/testdir/test_viml.vim

Patch 7.4.1158
问题: Still using __ARGS().
解决方案: Remove __ARGS() from eval.c
相关文件: src/eval.c

Patch 7.4.1159
问题: Automatically generated function prototypes use __ARGS.
解决方案: Remove __ARGS from osdef.sh.
相关文件: src/osdef.sh, src/osdef1.h.in, src/osdef2.h.in

Patch 7.4.1160
问题: No error for jsondecode('"').
解决方案: Give an error message for missing double quote.
相关文件: src/json.c

Patch 7.4.1161
问题: ":argadd" without argument is supposed to add the current buffer
name to the arglist.

解决方案: Make it work as documented. (Coot, closes #577)
相关文件: src/ex_cmds.h, src/ex_cmds2.c, src/testdir/test_arglist.vim

Patch 7.4.1162
问题: Missing error number in MzScheme. (Dominique Pelle)
解决方案: Add a proper error number.
相关文件: src/if_mzsch.c

Patch 7.4.1163
问题: Expressions "0 + v:true" and "'' . v:true" cause an error.
解决方案: Return something sensible when using a special variable as a
           number or as a string. (suggested by Damien)
相关文件: src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1164
问题: No tests for comparing special variables. Error in jsondecode()
           not reported. test_json does not work with Japanese system.
解决方案: Set scriptencoding. (Ken Takata) Add a few more tests. Add error.
相关文件: src/json.c, src/testdir/test_viml.vim, src/testdir/test_json.vim

Patch 7.4.1165
问题: When defining DYNAMIC_ICONV_DLL in the makefile, the build fails.
解决方案: Add #ifdef's. (Taro Muraoka) Try the newer version first.
相关文件: src/mbyte.c, src/os_win32.c

Patch 7.4.1166
问题: Can't encode a Funcref into JSON. jsonencode() doesn't handle the
           same list or dict twice properly. (Nikolai Pavlov)
解决方案: Give an error. Reset copyID when the list or dict is finished.
相关文件: src/json.c, src/proto/json.pro, src/testdir/test_json.vim

Patch 7.4.1167
问题: No tests for "is" and "isnot" with the new variables.
解决方案: Add tests.
相关文件: src/testdir/test_viml.vim

Patch 7.4.1168
问题: This doesn't give the right result: eval(string(v:true)). (Nikolai
           Pavlov)
解决方案: Make the string "v:true" instead of "true".
相关文件: src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1169
问题: The socket I/O is intertwined with the netbeans code.
解决方案: Start refactoring the netbeans communication to split off the
           socket I/O. Add the +channel feature.
相关文件: src/channel.c, src/netbeans.c, src/proto/channel.pro,
           src/proto/netbeans.pro, src/proto/gui_w32.pro, src/gui_w32.c,
           src/eval.c, src/os_mswin.c, src/ui.c, src/macros.h, Makefile,
           src/proto.h, src/feature.h, src/os_unix.c, src/vim.h,
           src/configure.in, src/auto/configure, src/config.mk.in,
           src/config.aap.in, src/config.h.in, src/Make_bc5.mak,
           src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 7.4.1170 (after 7.4.1169)
问题:　　Missing changes in src/Makefile, Filelist.
解决方案:　Add the missing changes.
相关文件:　Filelist, src/Makefile

Patch 7.4.1171
问题:　　Makefile dependencies are outdated.
解决方案:　Run "make depend". Add GTK resource dependencies.
相关文件:　src/Makefile

Patch 7.4.1172 (after 7.4.1169)
问题:　　Configure is overly positive.
解决方案:　Insert "test".
相关文件:　src/configure.in, src/auto/configure

Patch 7.4.1173 (after 7.4.1168)
问题:　　No test for new behavior of v:true et al.
解决方案:　Add a test.
相关文件:　src/testdir/test_viml.vim

Patch 7.4.1174
问题:　　Netbeans contains dead code inside #ifndef INIT_SOCKETS.
解决方案:　Remove the dead code.
相关文件:　src/netbeans.c

Patch 7.4.1175 (after 7.4.1169)
问题:　　Can't build with Mingw and Cygwin.
解决方案:　Remove extra "endif". (Christian J. Robinson)
相关文件:　src/Make_cyg_ming.mak

Patch 7.4.1176
问题:　　Missing change to proto file.
解决方案:　Update the proto file. (Charles Cooper)
相关文件:　src/proto/gui_w32.pro

Patch 7.4.1177
问题:　　The +channel feature is not in :version output. (Tony Mechelynck)
解决方案:　Add the feature string.
相关文件:　src/version.c

Patch 7.4.1178
问题:　　empty() doesn't work for the new special variables.
解决方案:　Make empty() work. (Damien)
相关文件:　src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1179
问题:　　test_writefile and test_viml do not delete the tempfile.
解决方案:　Delete the tempfile. (Charles Cooper) Add DeleteTheScript().
相关文件:　src/testdir/test_writefile.in, src/testdir/test_viml.vim

Patch 7.4.1180
问题:　　Crash with invalid argument to glob2regpat().
解决方案:　Check for NULL. (Justin M. Keyes, closes #596) Add a test.
相关文件:　src/eval.c, src/testdir/test_glob2regpat.vim,

src/testdir/test_alot.vim

Patch 7.4.1181
问题: free_tv() can't handle special variables. (Damien)
解决方案: Add the variable type.
相关文件: src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1182
问题: Still socket code intertwined with netbeans.
解决方案: Move code from netbeans.c to channel.c
相关文件: src/channel.c, src/netbeans.c, src/proto/channel.pro,
src/proto/netbeans.pro, src/gui.c, src/gui_w48.c

Patch 7.4.1183 (after 7.4.1182)
问题: MS-Windows build is broken.
解决方案: Remove init in wrong place.
相关文件: src/channel.c

Patch 7.4.1184 (after 7.4.1182)
问题: MS-Windows build is still broken.
解决方案: Change nbsock to ch_fd.
相关文件: src/channel.c

Patch 7.4.1185
问题: Can't build with TCL on some systems.
解决方案: Rename the channel_ functions.
相关文件: src/if_tcl.c

Patch 7.4.1186
问题: Error messages for security context are hard to translate.
解决方案: Use one string with %s. (Ken Takata)
相关文件: src/os_unix.c

Patch 7.4.1187
问题: MS-Windows channel code only supports one channel. Doesn't build
without netbeans support.
解决方案: Get the channel index from the socket in the message. Closes #600.
相关文件: src/channel.c, src/netbeans.c, src/gui_w48.c,
src/proto/channel.pro, src/proto/netbeans.pro

Patch 7.4.1188
问题: Using older JSON standard.
解决方案: Update the link. Adjust the text a bit.
相关文件: src/json.c, runtime/doc/eval.txt

Patch 7.4.1189 (after 7.4.1165)
问题: Using another language on MS-Windows does not work. (Yongwei Wu)
解决方案: Undo the change to try loading libintl-8.dll first.
相关文件: src/os_win32.c

Patch 7.4.1190
问题: On OSX the default flag for dlopen() is different.
解决方案: Add RTLD_LOCAL in the configure check. (sv99, closes #604)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.1191
问题: The channel feature isn't working yet.
解决方案: Add the connect(), disconnect(), sendexpr() and sendraw()
functions. Add initial documentation. Add a demo server.
相关文件: src/channel.c, src/eval.c, src/proto/channel.pro,
src/proto/eval.pro, runtime/doc/channel.txt, runtime/doc/eval.txt,
runtime/doc/Makefile, runtime/tools/demoserver.py

Patch 7.4.1192
问题: Can't build with FEAT_EVAL but without FEAT_MBYTE. (John
Marriott)
解决方案: Add #ifdef for FEAT_MBYTE.
相关文件: src/json.c

Patch 7.4.1193
问题: Can't build the channel feature on MS-Windows.
解决方案: Add #ifdef HAVE_POLL.
相关文件: src/channel.c

Patch 7.4.1194
问题: Compiler warning for not using return value of fwrite().
解决方案: Return OK/FAIL. (Charles Campbell)
相关文件: src/channel.c, src/proto/channel.pro

Patch 7.4.1195
问题: The channel feature does not work in the MS-Windows console.
解决方案: Add win32 console support. (Yasuhiro Matsumoto)
相关文件: src/channel.c, src/gui_w32.c, src/os_mswin.c, src/os_win32.c,
src/proto/gui_w32.pro, src/proto/os_mswin.pro, src/vim.h

Patch 7.4.1196
问题: Still using __ARGS.
解决方案: Remove __ARGS in several files. (script by Hirohito Higashi)
相关文件: src/arabic.c, src/buffer.c, src/charset.c, src/crypt_zip.c,
src/diff.c, src/digraph.c, src/edit.c, src/ex_cmds.c,
src/ex_cmds2.c, src/ex_docmd.c

Patch 7.4.1197
问题: Still using __ARGS.
解决方案: Remove __ARGS in several files. (script by Hirohito Higashi)
相关文件: src/ex_eval.c, src/ex_getln.c, src/farsi.c, src/fileio.c,
src/fold.c, src/getchar.c, src/gui.c, src/gui_at_fs.c,
src/gui_at_sb.c, src/gui_athena.c, src/gui_beval.c,
src/gui_motif.c, src/gui_w32.c, src/gui_w48.c

Patch 7.4.1198
问题: Still using __ARGS.
解决方案: Remove __ARGS in several files. (script by Hirohito Higashi)
Also remove use of HAVE_STDARG_H.
相关文件: src/gui_x11.c, src/hangulin.c, src/hardcopy.c, src/hashtab.c,
src/if_cscope.c, src/if_python3.c, src/if_sniff.c,
src/if_xcmdsrv.c, src/main.c, src/mark.c, src/mbyte.c,
src/memfile.c, src/memfile_test.c, src/memline.c, src/menu.c,

src/message.c, src/misc1.c, src/misc2.c, src/move.c,
src/netbeans.c, src/normal.c

Patch 7.4.1199
问题: Still using __ARGS.
解决方案: Remove __ARGS in several files. (script by Hirohito Higashi)
相关文件: src/ops.c, src/option.c, src/os_amiga.c, src/os_mac_conv.c,
src/os_unix.c, src/os_vms.c, src/os_w32exe.c, src/popupmnu.c,
src/pty.c, src/quickfix.c, src/regexp.c, src/regexp_nfa.c,
src/screen.c, src/search.c, src/sha256.c, src/spell.c,
src/syntax.c, src/tag.c, src/term.c, src/termlib.c, src/ui.c,
src/undo.c, src/version.c, src/window.c

Patch 7.4.1200
问题: Still using __ARGS.
解决方案: Remove __ARGS in several files. (script by Hirohito Higashi)
相关文件: src/blowfish.c, src/ex_cmds2.c, src/ex_getln.c, src/fold.c,
src/gui_beval.c, src/gui_w32.c, src/os_unix.c, src/os_win16.c,
src/pty.c, src/regexp.c, src/syntax.c, src/xpm_w32.c,
src/ex_cmds.h, src/globals.h, src/gui_at_sb.h, src/gui_beval.h,
src/if_cscope.h, src/if_sniff.h, src/nbdebug.h, src/os_unix.h,
src/proto.h, src/structs.h, src/vim.h, src/xpm_w32.h,
src/if_perl.xs, src/proto/if_lua.pro, src/proto/pty.pro,
runtime/tools/xcmdsrv_client.c,
src/Makefile

Patch 7.4.1201
问题: One more file still using __ARGS.
解决方案: Remove __ARGS in the last file. (script by Hirohito Higashi)
相关文件: src/gui_at_sb.c

Patch 7.4.1202
问题: Still one more file still using __ARGS.
解决方案: Remove __ARGS in the last file. (script by Hirohito Higashi)
(closes #612)
相关文件: src/proto/os_mac_conv.pro, src/os_mac_conv.c, src/Makefile

Patch 7.4.1203
问题: Still more files still using __ARGS.
解决方案: Remove __ARGS in really the last files.
相关文件: src/proto/if_mzsch.pro, src/if_mzsch.c, src/vim.h,
src/proto/gui_gtk_gresources.pro, src/proto/gui_mac.pro,
src/proto/if_ole.pro, src/proto/os_qnx.pro, src/Makefile

Patch 7.4.1204
问题: Latin1 characters cause encoding conversion.
解决方案: Remove the characters.
相关文件: src/gui_motif.c

Patch 7.4.1205
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito
Higashi)
相关文件: src/arabic.c, src/blowfish.c, src/buffer.c, src/channel.c,

src/charset.c, src/crypt.c, src/crypt_zip.c, src/diff.c,
src/digraph.c, src/edit.c, src/eval.c

Patch 7.4.1206
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c, src/farsi.c, src/fileio.c

Patch 7.4.1207
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/fold.c, src/getchar.c, src/gui_at_fs.c, src/gui_athena.c, src/gui_at_sb.c, src/gui_beval.c, src/gui.c, src/gui_gtk.c, src/gui_gtk_x11.c, src/gui_mac.c, src/gui_motif.c

Patch 7.4.1208
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/gui_photon.c, src/gui_w32.c, src/gui_w48.c, src/gui_x11.c, src/hangulin.c, src/hardcopy.c, src/hashtab.c, src/if_cscope.c, src/if_mzsch.c, src/if_perlsfio.c, src/if_python.c, src/if_python3.c, src/if_ruby.c, src/if_sniff.c, src/if_tcl.c, src/if_xcmdsrv.c, src/integration.c

Patch 7.4.1209 (after 7.4.1207)
问题: Can't build with Athena. (Elimar Riesebieter)
解决方案: Fix function declarations.
相关文件: src/gui_athena.c, src/gui_x11.c, src/gui_at_sb.c, src/gui_at_fs.c

Patch 7.4.1210
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/main.c, src/mark.c, src/mbyte.c, src/memfile.c, src/memfile_test.c, src/memline.c, src/menu.c, src/message.c

Patch 7.4.1211
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/misc1.c, src/misc2.c, src/move.c, src/netbeans.c, src/normal.c, src/ops.c, src/option.c

Patch 7.4.1212 (after 7.4.1207)
问题: Can't build with Motif.
解决方案: Fix function declaration.(Dominique Pelle)
相关文件: src/gui_motif.c

Patch 7.4.1213
问题: Using old style function declarations.

解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/os_amiga.c, src/os_mac_conv.c, src/os_msdos.d, src/os_mswin.c, src/os_qnx.c, src/os_unix.c, src/os_vms.c, src/os_win16.c, src/os_win32.c, src/popupmnu.c, src/pty.c, src/quickfix.c, src/regexp.c, src/regexp_nfa.c, src/screen.c

Patch 7.4.1214
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/search.c, src/sha256.c, src/spell.c, src/syntax.c, src/tag.c, src/term.c, src/termlib.c, src/ui.c, src/undo.c

Patch 7.4.1215
问题: Using old style function declarations.
解决方案: Change to new style function declarations. (script by Hirohito Higashi)
相关文件: src/version.c, src/winclip.c, src/window.c, src/workshop.c, src/xpm_w32.c, runtime/doc/doctags.c, runtime/tools/xcmdsrv_client.c, src/po/sjiscorr.c, src/xxd/xxd.c

Patch 7.4.1216
问题: Still using HAVE_STDARG_H.
解决方案: Assume it's always defined.
相关文件: src/eval.c, src/misc2.c, src/vim.h, src/proto.h, src/configure.in, src/auto/configure, config.h.in, src/os_amiga.h, src/os_msdos.h, src/os_vms_conf.h, src/os_win32.h

Patch 7.4.1217
问题: Execution of command on channel doesn't work yet.
解决方案: Implement the "ex" and "normal" commands.
相关文件: src/channel.c, src/proto/channel.pro, src/misc2.c, src/eval.c, src/ex_docmd.c, src/proto/ex_docmd.pro, src/feature.h

Patch 7.4.1218
问题: Missing change in configure. More changes for function style.
解决方案: Avoid the typos.
相关文件: src/configure.in, src/config.h.in, runtime/tools/ccfilter.c, src/os_msdos.c

Patch 7.4.1219
问题: Build fails with +channel but without +float.
解决方案: Add #ifdef.
相关文件: src/ex_cmds.c

Patch 7.4.1220
问题: Warnings for unused variables in tiny build. (Tony Mechelynck)
解决方案: Move declarations inside #ifdef. (Hirohito Higashi)
相关文件: src/ex_cmds.c

Patch 7.4.1221
问题: Including netbeans and channel support in small and tiny builds. Build fails with some interfaces.

解决方案: Only include these features in small build and above. Let
           configure fail if trying to enable an interface that won't build.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.1222
问题: ":normal" command and others missing in tiny build.
解决方案: Graduate FEAT_EX_EXTRA.
相关文件: src/feature.h, src/charset.c, src/eval.c, src/ex_cmds.c,
           src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/getchar.c,
           src/normal.c, src/ui.c, src/version.c, src/globals.h

Patch 7.4.1223
问题: Crash when setting v:errors to a number.
解决方案: Free the typval without assuming its type. (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 7.4.1224
问题: Build problems with GTK on BSD. (Mike Williams)
解决方案: Don't use "$<". Skip building gui_gtk_gresources.h when it doesn't
           work. (Kazunobu Kuriyama)
相关文件: src/Makefile

Patch 7.4.1225
问题: Still a few old style function declarations.
解决方案: Make them new style. (Hirohito Higashi)
相关文件: runtime/tools/blink.c, src/eval.c, src/ex_cmds2.c, src/ex_getln.c,
           src/fileio.c, src/gui_w32.c, src/gui_x11.c, src/if_perl.xs,
           src/os_unix.c, src/po/sjiscorr.c, src/pty.c

Patch 7.4.1226
问题: GRESOURCE_HDR is unused.
解决方案: Remove it. (Kazunobu Kuriyama)
相关文件: src/configure.in, src/auto/configure, src/config.mk.in

Patch 7.4.1227
问题: Compiler warnings.
解决方案: Add UNUSED. Add type cast. (Yegappan Lakshmanan)
相关文件: src/getchar.c, src/os_macosx.m

Patch 7.4.1228
问题: copy() and deepcopy() fail with special variables. (Nikolai
           Pavlov)
解决方案: Make it work. Add a test. Closes #614.
相关文件: src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1229
问题: "eval" and "expr" channel commands don't work yet.
解决方案: Implement them. Update the error numbers. Also add "redraw".
相关文件: src/channel.c, src/eval.c, src/json.c, src/ex_docmd.c,
           src/proto/channel.pro, src/proto/json.pro, src/proto/ex_docmd.pro,
           runtime/doc/channel.txt

Patch 7.4.1230
问题: Win32: opening a channel may hang. Not checking for messages

while waiting for characters.
解决方案: Add a zero timeout. Call parse_queued_messages(). (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.4.1231
问题: JSON messages are not parsed properly.
解决方案: Queue received messages.
相关文件: src/eval.c src/channel.c, src/json.c, src/proto/eval.pro, src/proto/channel.pro, src/proto/json.pro, src/structs.h

Patch 7.4.1232
问题: Compiler warnings when the Sniff feature is enabled.
解决方案: Add UNUSED.
相关文件: src/gui_gtk_x11.c

Patch 7.4.1233
问题: Channel command may cause a crash.
解决方案: Check for NULL argument. (Damien)
相关文件: src/channel.c

Patch 7.4.1234
问题: Demo server only runs with Python 2.
解决方案: Make it run with Python 3 as well. (Ken Takata)
相关文件: runtime/tools/demoserver.py

Patch 7.4.1235 (after 7.4.1231)
问题: Missing change to eval.c.
解决方案: Include that change.
相关文件: src/eval.c

Patch 7.4.1236
问题: When "syntax manual" was used switching between buffers removes the highlighting.
解决方案: Set the syntax option without changing the value. (Anton Lindqvist)
相关文件: runtime/syntax/manual.vim

Patch 7.4.1237
问题: Can't translate message without adding a line break.
解决方案: Join the two parts of the message.
相关文件: src/memline.c

Patch 7.4.1238
问题: Can't handle two messages right after each other.
解决方案: Find the end of the JSON. Read more when incomplete. Add a C test for the JSON decoding.
相关文件: src/channel.c, src/json.c, src/proto/json.pro, src/eval.c, src/Makefile, src/json_test.c, src/memfile_test.c, src/structs.h

Patch 7.4.1239
问题: JSON message after the first one is dropped.
解决方案: Put remainder of message back in the queue.
相关文件: src/channel.c

Patch 7.4.1240
问题: Visual studio tools are noisy.
解决方案: Suppress startup info. (Mike Williams)
相关文件: src/GvimExt/Makefile, src/Make_mvc.mak, src/tee/Make_mvc.mak

Patch 7.4.1241 (after 7.4.1238)
问题: Missing change in Makefile due to diff mismatch
解决方案: Update the list of object files.
相关文件: src/Makefile

Patch 7.4.1242 (after 7.4.1238)
问题: json_test fails without the eval feature.
解决方案: Add #ifdef.
相关文件: src/json_test.c

Patch 7.4.1243
问题: Compiler warning for uninitialized variable.
解决方案: Initialize it. (Elias Diem)
相关文件: src/json.c

Patch 7.4.1244
问题: The channel functions don't sort together.
解决方案: Use a common "ch_" prefix.
相关文件: src/eval.c, runtime/doc/eval.txt, runtime/tools/demoserver.py

Patch 7.4.1245
问题: File missing from distribution.
解决方案: Add json_test.c.
相关文件: Filelist

Patch 7.4.1246
问题: The channel functionality isn't tested.
解决方案: Add a test using a Python test server.
相关文件: src/channel.c, src/proto/channel.pro,
            src/testdir/test_channel.vim, src/testdir/test_channel.py,
            src/testdir/Make_all.mak

Patch 7.4.1247
问题: The channel test doesn't run on MS-Windows.
解决方案: Make it work on the MS-Windows console. (Ken Takata)
相关文件: src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1248
问题: Can't reliably stop the channel test server. Can't start the
            server if the python file is not executable.
解决方案: Use "pkill" instead of "killall". Run the python file as an
            argument instead of as an executable.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1249
问题: Crash when the process a channel is connected to exits.
解决方案: Use the file descriptor properly. Add a test. (Damien)
            Also add a test for eval().

相关文件: src/channel.c, src/testdir/test_channel.py,
           src/testdir/test_channel.vim

Patch 7.4.1250
问题: Running tests in shadow directory fails.
解决方案: Also link testdir/*.py
相关文件: src/Makefile

Patch 7.4.1251
问题: New test file missing from distribution.
解决方案: Add src/testdir/*.py.
相关文件: Filelist

Patch 7.4.1252
问题: The channel test server may receive two messages concatenated.
解决方案: Split the messages.
相关文件: src/testdir/test_channel.py

Patch 7.4.1253
问题: Python test server not displaying second of two commands.
           Solaris doesn't have "pkill --full".
解决方案: Also echo the second command. Use "pkill -f".
相关文件: src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1254
问题: Opening a second channel causes a crash. (Ken Takata)
解决方案: Don't re-allocate the array with channels.
相关文件: src/channel.c, src/testdir/test_channel.vim,
           src/testdir/test_channel.py

Patch 7.4.1255
问题: Crash for channel "eval" command without third argument.
解决方案: Check for missing argument.
相关文件: src/channel.c, src/testdir/test_channel.vim,
           src/testdir/test_channel.py

Patch 7.4.1256
问题: On Mac sys.exit(0) doesn't kill the test server.
解决方案: Use self.server.shutdown(). (Jun Takimoto)
相关文件: src/testdir/test_channel.py

Patch 7.4.1257
问题: Channel test fails in some configurations.
解决方案: Add check for the +channel feature.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1258
问题: The channel test can fail if messages arrive later.
解决方案: Add a short sleep. (Jun Takimoto)
相关文件: src/testdir/test_channel.vim

Patch 7.4.1259
问题: No test for what patch 7.3.414 fixed.
解决方案: Add a test. (Elias Diem)

相关文件: src/testdir/test_increment.vim

Patch 7.4.1260
问题: The channel feature doesn't work on Win32 GUI.
解决方案: Use WSAGetLastError(). (Ken Takata)
相关文件: src/channel.c, src/testdir/test_channel.vim, src/vim.h

Patch 7.4.1261
问题: Pending channel messages are garbage collected. Leaking memory in ch_sendexpr(). Leaking memory for a decoded JSON string.
解决方案: Mark the message list as used. Free the encoded JSON. Don't save the JSON string.
相关文件: src/eval.c, src/channel.c, src/json.c, src/proto/channel.pro

Patch 7.4.1262
问题: The channel callback is not invoked.
解决方案: Make a list of pending callbacks.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 7.4.1263
问题: ch_open() hangs when the server isn't running.
解决方案: Add a timeout. Use a dict to pass arguments. (Yasuhiro Matsumoto)
相关文件: runtime/doc/eval.txt, runtime/doc/channel.txt, src/channel.c, src/eval.c, src/netbeans.c, src/os_win32.c, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 7.4.1264
问题: Crash when receiving an empty array.
解决方案: Check for array with wrong number of arguments. (Damien)
相关文件: src/channel.c, src/eval.c, src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1265
问题: Not all channel commands are tested.
解决方案: Add a test for "normal", "expr" and "redraw".
相关文件: src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1266
问题: A BufAdd autocommand may cause an ml_get error (Christian Brabandt)
解决方案: Increment RedrawingDisabled earlier.
相关文件: src/ex_cmds.c

Patch 7.4.1267
问题: Easy to miss handling all types of variables.
解决方案: Change the variable type into an enum.
相关文件: src/structs.h, src/eval.c

Patch 7.4.1268
问题: Waittime is used as seconds instead of milliseconds. (Hirohito Higashi)
解决方案: Divide by 1000.
相关文件: src/channel.c

Patch 7.4.1269
问题: Encoding {'key':v:none} to JSON doesn't give an error (Tyru)
解决方案: Give an error.
相关文件: src/json.c, src/testdir/test_json.vim

Patch 7.4.1270
问题: Warnings for missing values in switch.
解决方案: Change switch to if-else or add values.
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 7.4.1271
问题: assert_false(v:false) reports an error. (Nikolai Pavlov)
解决方案: Recognize v:true and v:false. (Closes #625)
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 7.4.1272 (after 7.4.1270)
问题: Using future enum value.
解决方案: Remove it.
相关文件: src/if_python.c, src/if_python3.c

Patch 7.4.1273 (after 7.4.1271)
问题: assert_false(v:false) still fails.
解决方案: Fix the typo.
相关文件: src/eval.c

Patch 7.4.1274
问题: Cannot run a job.
解决方案: Add job_start(), job_status() and job_stop(). Currently only works
for Unix.
相关文件: src/eval.c, src/structs.h, runtime/doc/eval.txt, src/os_unix.c,
src/proto/os_unix.pro, src/feature.h, src/version.c,
src/testdir/test_channel.vim

Patch 7.4.1275 (after 7.4.1274)
问题: Build fails on MS-Windows.
解决方案: Fix wrong #ifdef.
相关文件: src/eval.c

Patch 7.4.1276
问题: Warning for not using return value of fcntl().
解决方案: Explicitly ignore the return value.
相关文件: src/fileio.c, src/channel.c, src/memfile.c, src/memline.c

Patch 7.4.1277
问题: Compiler can complain about missing enum value in switch with some
combination of features.
解决方案: Remove #ifdefs around case statements.
相关文件: src/eval.c

Patch 7.4.1278
问题: When jsonencode() fails it still returns something.
解决方案: Return an empty string on failure.
相关文件: src/json.c, src/channel.c, src/testdir/test_json.vim,

src/testdir/test_channel.vim, src/testdir/test_channel.py

Patch 7.4.1279
问题: jsonencode() is not producing strict JSON.
解决方案: Add jsencode() and jsdecode(). Make jsonencode() and jsondecode() strict.
相关文件: src/json.c, src/json_test.c, src/proto/json.pro, src/channel.c, src/proto/channel.pro, src/eval.c, src/vim.h, src/structs.h, runtime/doc/eval.txt, runtime/doc/channel.txt, src/testdir/test_json.vim

Patch 7.4.1280
问题: Missing case value.
解决方案: Add VAR_JOB.
相关文件: src/if_python.c, src/if_python3.c

Patch 7.4.1281
问题: No test for skipping over code that isn't evaluated.
解决方案: Add a test with code that would fail when not skipped.
相关文件: src/testdir/test_viml.vim

Patch 7.4.1282
问题: Crash when evaluating the pattern of ":catch" causes an error. (Dominique Pelle)
解决方案: Block error messages at this point.
相关文件: src/ex_eval.c

Patch 7.4.1283
问题: The job feature isn't available on MS-Windows.
解决方案: Add the job feature. Fix argument of job_stop(). (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/feature.h, src/os_win32.c, src/proto/os_win32.pro

Patch 7.4.1284 (after 7.4.1282)
问题: Test 49 fails.
解决方案: Check for a different error message.
相关文件: src/testdir/test49.vim

Patch 7.4.1285
问题: Cannot measure elapsed time.
解决方案: Add reltimefloat().
相关文件: src/ex_cmds2.c, src/eval.c, src/proto/ex_cmds2.pro, src/testdir/test_reltime.vim, src/testdir/test_alot.vim

Patch 7.4.1286
问题: ch_open() with a timeout doesn't work correctly.
解决方案: Change how select() is used. Don't give an error on timeout. Add a test for ch_open() failing.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1287 (after 7.4.1286)
问题: Channel test fails.
解决方案: Use reltimefloat().
相关文件: src/testdir/test_channel.vim

Patch 7.4.1288
问题: ch_sendexpr() does not use JS encoding.
解决方案: Use the encoding that fits the channel mode. Refuse using ch_sendexpr() on a raw channel.
相关文件: src/channel.c, src/proto/channel.pro, src/eval.c

Patch 7.4.1289
问题: Channel test fails on MS-Windows, connect() takes too long.
解决方案: Adjust the test for MS-Windows using "waittime".
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1290
问题: Coverity complains about unnecessary check for NULL.
解决方案: Remove the check.
相关文件: src/eval.c

Patch 7.4.1291
问题: On MS-Windows the channel test server doesn't quit.
解决方案: Use return instead of break. (Ken Takata)
相关文件: src/testdir/test_channel.py

Patch 7.4.1292
问题: Some compilers complain about uninitialized variable, even though all possible cases are handled. (Dominique Pelle)
解决方案: Add a default initialization.
相关文件: src/eval.c

Patch 7.4.1293
问题: Sometimes a channel may hang waiting for a message that was already discarded. (Ken Takata)
解决方案: Store the ID of the message blocking on in the channel.
相关文件: src/channel.c

Patch 7.4.1294
问题: job_stop() only kills the started process.
解决方案: Send the signal to the process group. (Olaf Dabrunz)
相关文件: src/os_unix.c

Patch 7.4.1295
问题: string(job) doesn't work well on MS-Windows.
解决方案: Use the process ID. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.1296
问题: Cursor changes column with up motion when the matchparen plugin saves and restores the cursor position. (Martin Kunev)
解决方案: Make sure curswant is updated before invoking the autocommand.
相关文件: src/edit.c

Patch 7.4.1297
问题: On Mac test_channel leaves python instances running.
解决方案: Use a small waittime to make ch_open() work. (Ozaki Kiichi)
相关文件: src/testdir/test_channel.vim

Patch 7.4.1298
问题: When the channel test fails in an unexpected way the server keeps running.
解决方案: Use try/catch. (Ozaki Kiichi)
相关文件: src/testdir/test_channel.vim

Patch 7.4.1299
问题: When the server sends a message with ID zero the channel handler is not invoked. (Christian J. Robinson)
解决方案: Recognize zero value for the request ID. Add a test for invoking the channel handler.
相关文件: src/channel.c, src/testdir/test_channel.vim, src/testdir/test_channel.py

Patch 7.4.1300
问题: Cannot test CursorMovedI because there is typeahead.
解决方案: Add disable_char_avail_for_testing().
相关文件: src/eval.c, src/getchar.c, src/globals.h, src/testdir/test_cursor_func.vim, src/testdir/README.txt

Patch 7.4.1301
问题: Missing options in ch_open().
解决方案: Add s:chopt like in the other calls. (Ozaki Kiichi)
相关文件: src/testdir/test_channel.vim

Patch 7.4.1302
问题: Typo in struct field name. (Ken Takata)
解决方案: Rename jf_pi to jv_pi.
相关文件: src/eval.c, src/os_win32.c, src/structs.h

Patch 7.4.1303
问题: A Funcref is not accepted as a callback.
解决方案: Make a Funcref work. (Damien)
相关文件: src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1304
问题: Function names are difficult to read.
解决方案: Rename jsonencode to json_encode, jsondecode to json_decode, jsencode to js_encode and jsdecode to js_decode.
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test_json.vim

Patch 7.4.1305
问题: "\%1l^#.*" does not match on a line starting with "#".
解决方案: Do not clear the start-of-line flag. (Christian Brabandt)
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test36.in, src/testdir/test36.ok

Patch 7.4.1306
问题: Job control doesn't work well on MS-Windows.
解决方案: Various fixes. (Ken Takata, Ozaki Kiichi, Yukihiro Nakadaira, Yasuhiro Matsumoto)
相关文件: src/Make_mvc.mak, src/eval.c, src/os_unix.c, src/os_win32.c, src/proto/os_unix.pro, src/proto/os_win32.pro, src/structs.h

Patch 7.4.1307
问题: Some channel tests fail on MS-Windows.
解决方案: Disable the failing tests temporarily.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1308 (after 7.4.1307)
问题: Typo in test.
解决方案: Change endf to endif.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1309
问题: When a test fails not all relevant info is listed.
解决方案: Add the errors to the messages.
相关文件: src/testdir/runtest.vim

Patch 7.4.1310
问题: Jobs don't open a channel.
解决方案: Create pipes and add them to the channel. Add ch_logfile().
            Only Unix for now.
相关文件: src/channel.c, src/eval.c, src/os_unix.c, src/structs.h,
            src/gui_w48.c, src/proto/channel.pro, src/testdir/test_channel.vim,
            src/testdir/test_channel_pipe.py, runtime/doc/eval.txt

Patch 7.4.1311 (after 7.4.1310)
问题: sock_T is defined too late.
解决方案: Move it up.
相关文件: src/vim.h

Patch 7.4.1312 (after 7.4.1311)
问题: sock_T is not defined without the +channel feature.
解决方案: Always define it.
相关文件: src/vim.h

Patch 7.4.1313
问题: MS-Windows: Using socket after it was closed causes an exception.
解决方案: Don't give an error when handling WM_NETBEANS. Re-enable tests
            for MS-Windows.
相关文件: src/gui_w48.c, src/testdir/test_channel.vim

Patch 7.4.1314
问题: Warning for uninitialized variable.
解决方案: Initialize it. (Dominique Pelle)
相关文件: src/channel.c

Patch 7.4.1315
问题: Using a channel handle does not allow for freeing it when unused.
解决方案: Add the Channel variable type.
相关文件: src/structs.h, src/channel.c, src/misc2.c, src/eval.c,
            src/if_python.c, src/if_python3.c, src/json.c, src/gui_w48.c,
            src/netbeans.c, src/proto/channel.pro, src/os_unix.c,
            src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1316

问题: Can't build MS-Windows console version. (Tux)
解决方案: Add #ifdefs.
相关文件: src/eval.c

Patch 7.4.1317
问题: MS-Windows: channel test fails.
解决方案: Temporarily disable Test_connect_waittime().
相关文件: src/testdir/test_channel.vim

Patch 7.4.1318
问题: Channel with pipes doesn't work in GUI.
解决方案: Register input handlers for pipes.
相关文件: src/structs.h, src/feature.h, src/channel.c, src/eval.c,
            src/os_unix.c, src/os_win32.c, src/gui_w48.c, src/proto/channel.pro

Patch 7.4.1319 (after 7.4.1318)
问题: Tests fail on MS-Windows and on Unix with GUI.
解决方案: Fix unregistering.
相关文件: src/structs.h, src/channel.c, src/os_unix.c, src/os_win32.c,
            src/proto/channel.pro

Patch 7.4.1320
问题: Building with Cygwin or MingW with channel but without Netbeans
            doesn't work.
解决方案: Set NETBEANS to "no" when not used.
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1321
问题: Compiler complains about missing statement.
解决方案: Add an empty statement. (Andrei Olsen)
相关文件: src/os_win32.c

Patch 7.4.1322
问题: Crash when unletting the variable that holds the channel in a
            callback function. (Christian Robinson)
解决方案: Increase the reference count while invoking the callback.
相关文件: src/eval.c, src/channel.c, src/proto/eval.pro,
            src/testdir/test_channel.vim

Patch 7.4.1323
问题: Do not get warnings when building with MingW.
解决方案: Remove the -w flag. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1324
问题: Channels with pipes don't work on MS-Windows.
解决方案: Add pipe I/O support. (Yasuhiro Matsumoto)
相关文件: src/channel.c, src/os_win32.c, src/proto/channel.pro,
            src/structs.h, src/vim.h, src/testdir/test_channel.vim

Patch 7.4.1325
问题: Channel test fails on difference between Unix and DOS line endings.
解决方案: Strip off CR. Make assert show difference better.
相关文件: src/eval.c, src/channel.c

Patch 7.4.1326
问题: Build rules are bit too complicated.
解决方案: Remove -lwsock32 from Netbeans, it's already added for the channel feature that it depends on. (Tony Mechelynck)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1327
问题: Channel test doesn't work if Python executable is python.exe.
解决方案: Find py.exe or python.exe. (Ken Takata)
相关文件: src/testdir/test_channel.vim

Patch 7.4.1328
问题: Can't compile with +job but without +channel. (John Marriott)
解决方案: Add more #ifdefs.
相关文件: src/os_unix.c

Patch 7.4.1329
问题: Crash when using channel that failed to open.
解决方案: Check for NULL. Update messages. (Yukihiro Nakadaira)
相关文件: src/channel.c, src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1330
问题: fd_read() has an unused argument.
解决方案: Remove the timeout. (Yasuhiro Matsumoto)
相关文件: src/channel.c

Patch 7.4.1331
问题: Crash when closing the channel in a callback. (Christian J. Robinson)
解决方案: Take the callback out of the list before invoking it.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1332
问题: Problem using Python3 when compiled with MingW.
解决方案: Define PYTHON3_HOME as a wide character string. (Yasuhiro Matsumoto)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1333
问题: Channel test fails on non-darwin builds.
解决方案: Add the "osx" feature and test for that. (Kazunobu Kuriyama)
相关文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1334
问题: Many compiler warnings with MingW.
解决方案: Add type casts. (Yasuhiro Matsumoto)
相关文件: src/channel.c, src/dosinst.h, src/eval.c, src/ex_cmds2.c, src/ex_getln.c, src/fileio.c, src/if_cscope.c, src/if_perl.xs, src/if_python.c, src/if_python3.c, src/if_ruby.c, src/main.c, src/mbyte.c, src/misc1.c, src/option.c, src/os_mswin.c, src/os_win32.c

Patch 7.4.1335

问题:　　Can't build on MS-Windows with +job but without +channel. (Cesar Romani)
解决方案:　Add #ifdefs. (Yasuhiro Matsumoto)
相关文件:　src/os_win32.c

Patch 7.4.1336
问题:　　Channel NL mode is not supported yet.
解决方案:　Add NL mode support to channels.
相关文件:　src/channel.c, src/netbeans.c, src/structs.h, src/os_win32.c, src/proto/channel.pro, src/proto/os_unix.pro, src/proto/os_win32.pro, src/testdir/test_channel.vim, src/testdir/test_channel_pipe.py

Patch 7.4.1337 (after 7.4.1336)
问题:　　Part of the change is missing.
解决方案:　Add changes to eval.c
相关文件:　src/eval.c

Patch 7.4.1338 (after 7.4.1336)
问题:　　Another part of the change is missing.
解决方案:　Type os_unix.c right this time.
相关文件:　src/os_unix.c

Patch 7.4.1339
问题:　　Warnings when building the GUI with MingW. (Cesar Romani)
解决方案:　Add type casts. (Yasuhiro Matsumoto)
相关文件:　src/edit.c, src/gui_w32.c, src/gui_w48.c, src/os_mswin.c, src/os_win32.c

Patch 7.4.1340 (after 7.4.1339)
问题:　　Merge left extra #endif behind.
解决方案:　Remove the #endif
相关文件:　src/os_win32.c

Patch 7.4.1341
问题:　　It's difficult to add more arguments to ch_sendraw() and ch_sendexpr().
解决方案:　Make the third option a dictionary.
相关文件:　src/eval.c, src/structs.h, src/channel.c, src/os_unix.c, src/os_win32.c, src/proto/channel.pro, src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1342
问题:　　On Mac OS/X the waittime must be > 0 for connect to work.
解决方案:　Use select() in a different way. (partly by Kazunobu Kuriyama) Always use a waittime of 1 or more.
相关文件:　src/eval.c, src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1343
问题:　　Can't compile with +job but without +channel. (Andrei Olsen)
解决方案:　Move get_job_options up and adjust #ifdef.
相关文件:　src/eval.c

Patch 7.4.1344
问题: Can't compile Win32 GUI with tiny features.
解决方案: Add #ifdef. (Christian Brabandt)
相关文件: src/gui_w32.c

Patch 7.4.1345
问题: A few more compiler warnings. (Axel Bender)
解决方案: Add type casts.
相关文件: src/gui_w32.c, src/gui_w48.c

Patch 7.4.1346
问题: Compiler warnings in build with -O2.
解决方案: Add initializations.
相关文件: src/eval.c

Patch 7.4.1347
问题: When there is any error Vim will use a non-zero exit code.
解决方案: When using ":silent!" do not set the exit code. (Yasuhiro
            Matsumoto)
相关文件: src/message.c

Patch 7.4.1348
问题: More compiler warnings. (John Marriott)
解决方案: Add type casts, remove unused variable.
相关文件: src/gui_w32.c

Patch 7.4.1349
问题: And some more MingW compiler warnings. (Cesar Romani)
解决方案: Add type casts.
相关文件: src/if_mzsch.c

Patch 7.4.1350
问题: When the test server fails to start Vim hangs.
解决方案: Check that there is actually something to read from the tty fd.
相关文件: src/os_unix.c

Patch 7.4.1351
问题: When the port isn't opened yet when ch_open() is called it may
            fail instead of waiting for the specified time.
解决方案: Loop when select() succeeds but when connect() failed. Also use
            channel logging for jobs. Add ch_log().
相关文件: src/channel.c, src/eval.c, src/netbeans.c, src/proto/channel.pro,
            src/testdir/test_channel.vim, src/testdir/test_channel.py

Patch 7.4.1352
问题: The test script lists all functions before executing them.
解决方案: Only list the function currently being executed.
相关文件: src/testdir/runtest.vim

Patch 7.4.1353
问题: Test_connect_waittime is skipped for MS-Windows.
解决方案: Add the test back, it works now.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1354
问题: MS-Windows: Mismatch between default compile options and what the code expects.
解决方案: Change the default WINVER from 0x0500 to 0x0501. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 7.4.1355
问题: Win32 console and GUI handle channels differently.
解决方案: Consolidate code between Win32 console and GUI.
相关文件: src/channel.c, src/eval.c, src/gui_w48.c, src/os_win32.c, src/proto/channel.pro

Patch 7.4.1356
问题: Job and channel options parsing is scattered.
解决方案: Move all option value parsing to get_job_options();
相关文件: src/channel.c, src/eval.c, src/structs.h, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 7.4.1357 (after 7.4.1356)
问题: Error for returning value from void function.
解决方案: Don't do that.
相关文件: src/eval.c

Patch 7.4.1358
问题: Compiler warning when not building with +crypt.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/undo.c

Patch 7.4.1359 (after 7.4.1356)
问题: Channel test ch_sendexpr() times out.
解决方案: Increase the timeout
相关文件: src/testdir/test_channel.vim

Patch 7.4.1360
问题: Can't remove a callback with ch_setoptions().
解决方案: When passing zero or an empty string remove the callback.
相关文件: src/channel.c, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 7.4.1361
问题: Channel test fails on Solaris.
解决方案: Use the 1 msec waittime for all systems.
相关文件: src/channel.c

Patch 7.4.1362 (after 7.4.1356)
问题: Using uninitialized value.
解决方案: Initialize jo_set.
相关文件: src/eval.c

Patch 7.4.1363
问题: Compiler warnings with tiny build.
解决方案: Add #ifdefs.
相关文件: src/gui_w48.c, src/gui_w32.c

Patch 7.4.1364

问题: The Win 16 code is not maintained and unused.
解决方案: Remove the Win 16 support.
相关文件: src/gui_w16.c, src/gui_w32.c, src/gui_w48.c, src/Make_w16.mak,
            src/Makefile, src/Make_cyg_ming.mak, src/Make_mvc.mak,
            src/proto/gui_w16.pro, src/proto/os_win16.pro, src/guiw16rc.h,
            src/vim16.rc, src/vim16.def, src/tools16.bmp, src/eval.c,
            src/gui.c, src/misc2.c, src/option.c, src/os_msdos.c,
            src/os_mswin.c, src/os_win16.c, src/os_win16.h, src/version.c,
            src/winclip.c, src/feature.h, src/proto.h, src/vim.h, Filelist

Patch 7.4.1365
问题: Cannot execute a single test function.
解决方案: Add an argument to filter the functions with. (Yasuhiro Matsumoto)
相关文件: src/testdir/runtest.vim

Patch 7.4.1366
问题: Typo in test and resulting error in test result.
解决方案: Fix the typo and correct the result. (James McCoy, closes #650)
相关文件: src/testdir/test_charsearch.in, src/testdir/test_charsearch.ok

Patch 7.4.1367
问题: Compiler warning for unreachable code.
解决方案: Remove a "break". (Danek Duvall)
相关文件: src/json.c

Patch 7.4.1368
问题: One more Win16 file remains.
解决方案: Delete it.
相关文件: src/proto/os_win16.pro

Patch 7.4.1369
问题: Channels don't have a queue for stderr.
解决方案: Have a queue for each part of the channel.
相关文件: src/channel.c, src/eval.c, src/structs.h, src/netbeans.c,
            src/gui_w32.c, src/proto/channel.pro

Patch 7.4.1370
问题: The Python test script may keep on running.
解决方案: Join the threads. (Yasuhiro Matsumoto)
相关文件: src/testdir/test_channel.py

Patch 7.4.1371
问题: X11 GUI callbacks don't specify the part of the channel.
解决方案: Pass the fd instead of the channel ID.
相关文件: src/channel.c

Patch 7.4.1372
问题: channel read implementation is incomplete.
解决方案: Add ch_read() and options for ch_readraw().
相关文件: src/channel.c, src/eval.c, src/structs.h, src/proto/channel.pro,
            src/testdir/test_channel.vim

Patch 7.4.1373
问题: Calling a Vim function over a channel requires turning the

arguments into a string.
解决方案: Add the "call" command. (Damien) Also merge "expr" and "eval" into one.
相关文件: src/channel.c, src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1374
问题: Channel test hangs on MS-Windows.
解决方案: Disable the ch_read() that is supposed to time out.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1375
问题: Still some Win16 code.
解决方案: Remove FEAT_GUI_W16.(Hirohito Higashi)
相关文件: src/eval.c, src/ex_cmds.h, src/feature.h, src/gui.h, src/menu.c, src/misc1.c, src/option.c, src/proto.h, src/structs.h, src/term.c, src/vim.h, runtime/doc/gui_w16.txt

Patch 7.4.1376
问题: ch_setoptions() cannot set all options.
解决方案: Support more options.
相关文件: src/channel.c, src/eval.c, src/structs.h, runtime/doc/channel.txt, src/testdir/test_channel.vim

Patch 7.4.1377
问题: Test_connect_waittime() is flaky.
解决方案: Ignore the "Connection reset by peer" error.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1378
问题: Can't change job settings after it started.
解决方案: Add job_setoptions() with the "stoponexit" flag.
相关文件: src/eval.c, src/main.c, src/structs.h, src/proto/eval.pro, src/testdir/test_channel.vim

Patch 7.4.1379
问题: Channel test fails on Win32 console.
解决方案: Don't sleep when timeout is zero. Call channel_wait() before channel_read(). Channels are not polled during ":sleep". (Yukihiro Nakadaira)
相关文件: src/channel.c, src/misc2.c, src/gui_w32.c, src/os_win32.c

Patch 7.4.1380
问题: The job exit callback is not implemented.
解决方案: Add the "exit-cb" option.
相关文件: src/structs.h, src/eval.c, src/channel.c, src/proto/eval.pro, src/misc2.c, src/macros.h, src/testdir/test_channel.vim

Patch 7.4.1381 (after 7.4.1380)
问题: Exit value not available on MS-Windows.
解决方案: Set the exit value.
相关文件: src/structs.h, src/os_win32.c

Patch 7.4.1382

问题:　　Can't get the job of a channel.
解决方案:　Add ch_getjob().
相关文件:　src/eval.c, runtime/doc/channel.txt, runtime/doc/eval.txt

Patch 7.4.1383
问题:　　GvimExt only loads the old libintl.dll.
解决方案:　Also try loading libint-8.dll. (Ken Takata, closes #608)
相关文件:　src/GvimExt/gvimext.cpp, src/GvimExt/gvimext.h

Patch 7.4.1384
问题:　　It is not easy to use a set of plugins and their dependencies.
解决方案:　Add packages, ":loadplugin", 'packpath'.
相关文件:　src/main.c, src/ex_cmds2.c, src/option.c, src/option.h,
　　src/ex_cmds.h, src/eval.c, src/version.c, src/proto/ex_cmds2.pro,
　　runtime/doc/repeat.txt, runtime/doc/options.txt,
　　runtime/optwin.vim

Patch 7.4.1385
问题:　　Compiler warning for using array.
解决方案:　Use the right member name. (Yegappan Lakshmanan)
相关文件:　src/eval.c

Patch 7.4.1386
问题:　　When the Job exit callback is invoked, the job may be freed too
　　soon. (Yasuhiro Matsumoto)
解决方案:　Increase refcount.
相关文件:　src/eval.c

Patch 7.4.1387
问题:　　Win16 docs still referenced.
解决方案:　Remove Win16 files from the docs Makefile. (Kenichi Ito)
相关文件:　runtime/doc/Makefile

Patch 7.4.1388
问题:　　Compiler warning. (Cesar Romani)
解决方案:　Initialize variable.
相关文件:　src/ex_cmds2.c

Patch 7.4.1389
问题:　　Incomplete function declaration.
解决方案:　Add "void". (Yasuhiro Matsumoto)
相关文件:　src/eval.c

Patch 7.4.1390
问题:　　When building with GTK and glib-compile-resources cannot be found
　　building Vim fails. (Michael Gehring)
解决方案:　Make GLIB_COMPILE_RESOURCES empty instead of leaving it at "no".
　　(nuko8, closes #655)
相关文件:　src/configure.in, src/auto/configure

Patch 7.4.1391
问题:　　Warning for uninitialized variable.
解决方案:　Set it to zero. (Christian Brabandt)
相关文件:　src/eval.c

Patch 7.4.1392
问题: Some tests fail for Win32 console version.
解决方案: Move the tests to SCRIPTS_MORE2. Pass VIMRUNTIME. (Christian Brabandt)
相关文件: src/testdir/Make_all.mak

Patch 7.4.1393
问题: Starting a job hangs in the GUI. (Takuya Fujiwara)
解决方案: Don't check if ch_job is NULL when checking for an error. (Yasuhiro Matsumoto)
相关文件: src/channel.c

Patch 7.4.1394
问题: Can't sort inside a sort function.
解决方案: Use a struct to store the sort parameters. (Jacob Niehus)
相关文件: src/eval.c, src/testdir/test_sort.vim

Patch 7.4.1395
问题: Using DETACH in quotes is not compatible with the Netbeans interface. (Xavier de Gaye)
解决方案: Remove the quotes, only use them for JSON and JS mode.
相关文件: src/netbeans.c, src/channel.c

Patch 7.4.1396
问题: Compiler warnings for conversions.
解决方案: Add type cast.
相关文件: src/ex_cmds2.c

Patch 7.4.1397
问题: Sort test fails on MS-Windows.
解决方案: Correct the compare function.
相关文件: src/testdir/test_sort.vim

Patch 7.4.1398
问题: The close-cb option is not implemented yet.
解决方案: Implement close-cb. (Yasuhiro Matsumoto)
相关文件: src/channel.c, src/eval.c, src/structs.h, src/proto/channel.pro, src/testdir/test_channel.py, src/testdir/test_channel.vim

Patch 7.4.1399
问题: The MS-DOS code does not build.
解决方案: Remove the old MS-DOS code.
相关文件: Filelist, src/Make_bc3.mak, src/Make_bc5.mak, src/Make_djg.mak, src/Makefile, src/blowfish.c, src/buffer.c, src/diff.c, src/digraph.c, src/dosinst.h, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/feature.h, src/fileio.c, src/getchar.c, src/globals.h, src/macros.h, src/main.c, src/mbyte.c, src/memfile.c, src/memline.c, src/misc1.c, src/misc2.c, src/netbeans.c, src/option.c, src/option.h, src/os_msdos.c, src/os_msdos.h, src/proto.h, src/proto/os_msdos.pro, src/regexp.c, src/screen.c, src/structs.h, src/syntax.c, src/term.c, src/undo.c, src/uninstal.c, src/version.c, src/vim.h, src/window.c, src/xxd/Make_bc3.mak,

src/xxd/Make_djg.mak

Patch 7.4.1400
问题: Perl eval doesn't work properly on 64-bit big-endian machine.
解决方案: Use 32 bit type for the key. (Danek Duvall)
相关文件: src/if_perl.xs

Patch 7.4.1401
问题: Having 'autochdir' set during startup and using diff mode doesn't work. (Axel Bender)
解决方案: Don't use 'autochdir' while still starting up. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.4.1402
问题: GTK 3 is not supported.
解决方案: Add GTK 3 support. (Kazunobu Kuriyama)
相关文件: runtime/doc/eval.txt, runtime/doc/gui.txt, runtime/doc/gui_x11.txt, src/auto/configure, src/channel.c, src/config.h.in, src/configure.in, src/eval.c, src/gui.h, src/gui_beval.c, src/gui_beval.h, src/gui_gtk.c, src/gui_gtk_f.c, src/gui_gtk_f.h, src/gui_gtk_x11.c, src/if_mzsch.c, src/mbyte.c, src/netbeans.c, src/structs.h, src/version.c

Patch 7.4.1403
问题: Can't build without the quickfix feature.
解决方案: Add #ifdefs. Call ex_ni() for unimplemented commands. (Yegappan Lakshmanan)
相关文件: src/ex_cmds2.c, src/popupmnu.c

Patch 7.4.1404
问题: ch_read() doesn't time out on MS-Windows.
解决方案: Instead of WM_NETBEANS use select(). (Yukihiro Nakadaira)
相关文件: src/channel.c, src/gui_w32.c, src/os_win32.c, src/structs.h, src/testdir/test_channel.vim, src/vim.h

Patch 7.4.1405
问题: Completion menu flickers.
解决方案: Delay showing the popup menu. (Shougo Matsu, Justin M. Keyes, closes #656)
相关文件: src/edit.c

Patch 7.4.1406
问题: Leaking memory in cs_print_tags_priv().
解决方案: Free tbuf. (idea by Forrest Fleming)
相关文件: src/if_cscope.c

Patch 7.4.1407
问题: json_encode() does not handle NaN and inf properly. (David Barnett)
解决方案: For JSON turn them into "null". For JS use "NaN" and "Infinity". Add isnan().
相关文件: src/eval.c, src/json.c, src/testdir/test_json.vim

Patch 7.4.1408
问题: MS-Windows doesn't have isnan() and isinf().
解决方案: Use _isnan() and _isinf().
相关文件: src/eval.c, src/json.c

Patch 7.4.1409 (after 7.4.1402)
问题: Configure includes GUI despite --disable-gui flag.
解决方案: Add SKIP_GTK3. (Kazunobu Kuriyama)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.1410
问题: Leaking memory in cscope interface.
解决方案: Free memory when no tab is found. (Christian Brabandt)
相关文件: src/if_cscope.c

Patch 7.4.1411
问题: Compiler warning for indent. (Ajit Thakkar)
解决方案: Indent normally.
相关文件: src/ui.c

Patch 7.4.1412
问题: Compiler warning for indent. (Dominique Pelle)
解决方案: Fix the indent.
相关文件: src/farsi.c

Patch 7.4.1413
问题: When calling ch_close() the close callback is invoked, even though
the docs say it isn't. (Christian J. Robinson)
解决方案: Don't call the close callback.
相关文件: src/eval.c, src/channel.c, src/netbeans.c, src/proto/channel.pro

Patch 7.4.1414
问题: Appveyor only builds one feature set.
解决方案: Build a combination of features and GUI/console. (Christian
Brabandt)
相关文件: appveyor.yml, src/appveyor.bat

Patch 7.4.1415 (after 7.4.1414)
问题: Dropped the skip-tags setting.
解决方案: Put it back.
相关文件: appveyor.yml

Patch 7.4.1416
问题: Using "u_char" instead of "char_u", which doesn't work everywhere.
(Jrg Plate)
解决方案: Use "char_u" always.
相关文件: src/integration.c, src/macros.h

Patch 7.4.1417 (after 7.4.1414)
问题: Missing appveyor.bat from the distribution.
解决方案: Add it to the list of files.
相关文件: Filelist

Patch 7.4.1418
问题: job_stop() on MS-Windows does not really stop the job.
解决方案: Make the default to stop the job forcefully. (Ken Takata)
            Make MS-Windows and Unix more similar.
相关文件: src/os_win32.c, src/os_unix.c, runtime/doc/eval.txt

Patch 7.4.1419
问题: Tests slowed down because of the "not a terminal" warning.
解决方案: Add the --not-a-term command line argument.
相关文件: src/main.c, src/testdir/Makefile, src/Make_all.mak,
            src/Make_amiga.mak, src/testdir/Make_dos.mak,
            src/testdir/Make_ming.mak, src/testdir/Make_vms.mms,
            runtime/doc/starting.txt

Patch 7.4.1420 (after 7.4.1419)
问题: Missing makefile.
解决方案: Type the path correctly.
相关文件: src/testdir/Make_all.mak

Patch 7.4.1421
问题: May free a channel when a callback may need to be invoked.
解决方案: Keep the channel when refcount is zero.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro

Patch 7.4.1422
问题: Error when reading fails uses wrong errno. Keeping channel open
            after job stops results in test failing.
解决方案: Move the error up. Add ch_job_killed.
相关文件: src/channel.c, src/eval.c, src/structs.h

Patch 7.4.1423
问题: Channel test fails on MS-Windows.
解决方案: Do not give an error message when reading fails, assume the other
            end exited.
相关文件: src/channel.c

Patch 7.4.1424
问题: Not using --not-a-term when running tests on MS-Windows.
解决方案: Use NO_PLUGIN. (Christian Brabandt)
相关文件: src/testdir/Make_dos.mak

Patch 7.4.1425
问题: There are still references to MS-DOS support.
解决方案: Remove most of the help txt and install instructions. (Ken Takata)
相关文件: src/INSTALLpc.txt, runtime/doc/os_msdos.txt, csdpmi4b.zip,
            Filelist

Patch 7.4.1426
问题: The "out-io" option for jobs is not implemented yet.
解决方案: Implement the "buffer" value: append job output to a buffer.
相关文件: src/eval.c, src/channel.c, src/structs.h, src/netbeans.c,
            runtime/doc/channel.txt

Patch 7.4.1427

问题: Trailing comma in enums is not ANSI C.
解决方案: Remove the trailing commas.
相关文件: src/alloc.h, src/gui_mac.c

Patch 7.4.1428
问题: Compiler warning for non-virtual destructor.
解决方案: Make it virtual. (Yasuhiro Matsumoto)
相关文件: src/gui_dwrite.cpp

Patch 7.4.1429
问题: On MS-Windows, when not use renderoptions=type:directx, drawing
            emoji will be broken.
解决方案: Fix usage of unicodepdy. (Yasuhiro Matsumoto)
相关文件: src/gui_w32.c

Patch 7.4.1430
问题: When encoding JSON, turning NaN and Infinity into null without
            giving an error is not useful.
解决方案: Pass NaN and Infinity on. If the receiver can't handle them it
            will generate the error.
相关文件: src/json.c, src/testdir/test_json.vim, runtime/doc/eval.txt

Patch 7.4.1431
问题: Including header files twice.
解决方案: Remove the extra includes.
相关文件: src/if_cscope.h

Patch 7.4.1432
问题: Typo in button text.
解决方案: Fix the typo. (Dominique Pelle)
相关文件: src/gui_gtk.c

Patch 7.4.1433
问题: The Sniff interface is no longer useful, the tool has not been
            available for may years.
解决方案: Delete the Sniff interface and related code.
相关文件: src/if_sniff.c, src/if_sniff.h, src/charset.c, src/edit.c,
            src/eval.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c,
            src/gui_gtk_x11.c, src/gui_w32.c, src/gui_x11.c, src/normal.c,
            src/os_unix.c, src/os_win32.c, src/term.c, src/ui.c,
            src/version.c, src/ex_cmds.h, src/feature.h, src/keymap.h,
            src/structs.h, src/vim.h, src/Make_mvc.mak, src/Make_vms.mms,
            src/Makefile, src/configure.in, src/auto/configure,
            src/config.h.in, src/config.mk.in, runtime/doc/if_sniff.txt,
            src/config.aap.in, src/main.aap

Patch 7.4.1434
问题: JSON encoding doesn't handle surrogate pair.
解决方案: Improve multi-byte handling of JSON. (Yasuhiro Matsumoto)
相关文件: src/json.c, src/testdir/test_json.vim

Patch 7.4.1435
问题: It is confusing that ch_sendexpr() and ch_sendraw() wait for a
            response.

解决方案: Add ch_evalexpr() and ch_evalraw().
相关文件: src/eval.c, runtime/doc/channel.txt, runtime/doc/eval.txt,
           src/testdir/test_channel.vim

Patch 7.4.1436 (after 7.4.1433)
问题: Sniff files still referenced in distribution.
解决方案: Remove sniff files from distribution.
相关文件: Filelist

Patch 7.4.1437
问题: Old system doesn't have isinf() and NAN. (Ben Fritz)
解决方案: Adjust #ifdefs. Detect isnan() and isinf() functions with
           configure. Use a replacement when missing. (Kazunobu Kuriyama)
相关文件: src/eval.c, src/json.c, src/macros.h, src/message.c,
           src/config.h.in, src/configure.in, src/auto/configure

Patch 7.4.1438
问题: Can't get buffer number of a channel.
解决方案: Add ch_getbufnr().
相关文件: src/eval.c, src/channel.c, src/testdir/test_channel.vim,
           runtime/doc/channel.txt, runtime/doc/eval.txt

Patch 7.4.1439 (after 7.4.1434)
问题: Using uninitialized variable.
解决方案: Initialize vc_type.
相关文件: src/json.c

Patch 7.4.1440 (after 7.4.1437)
问题: Can't build on Windows.
解决方案: Change #ifdefs. Only define isnan when used.
相关文件: src/macros.h, src/eval.c, src/json.c

Patch 7.4.1441
问题: Using empty name instead of no name for channel buffer.
解决方案: Remove the empty name.
相关文件: src/channel.c

Patch 7.4.1442
问题: MS-Windows: more compilation warnings for destructor.
解决方案: Add "virtual". (Ken Takata)
相关文件: src/if_ole.cpp

Patch 7.4.1443
问题: Can't build GTK3 with small features.
解决方案: Use gtk_widget_get_window(). Fix typos. (Dominique Pelle)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1444
问题: Can't build with JSON but without multi-byte.
解决方案: Fix pointer name.
相关文件: src/json.c

Patch 7.4.1445
问题: Memory corruption when 'encoding' is not utf-8.

解决方案: Convert decoded string later.
相关文件: src/json.c

Patch 7.4.1446
问题: Crash when using json_decode().
解决方案: Terminate string with a NUL byte.
相关文件: src/json.c

Patch 7.4.1447
问题: Memory leak when using ch_read(). (Dominique Pelle)
           No log message when stopping a job and a few other situations.
           Too many "Nothing to read" messages. Channels are not freed.
解决方案: Free the listtv. Add more log messages. Remove "Nothing to read"
           message. Remove the channel from the job when its refcount
           becomes zero.
相关文件: src/eval.c, src/channel.c

Patch 7.4.1448
问题: JSON tests fail if 'encoding' is not utf-8.
解决方案: Force encoding to utf-8.
相关文件: src/testdir/test_json.vim

Patch 7.4.1449
问题: Build fails with job feature but without channel feature.
解决方案: Add #ifdef.
相关文件: src/eval.c

Patch 7.4.1450
问题: Json encoding still fails when encoding is not utf-8.
解决方案: Set 'encoding' before :scriptencoding. Run the json test
           separately to avoid affecting other tests.
相关文件: src/testdir/test_json.vim, src/testdir/Make_all.mak,
           src/testdir/test_alot.vim

Patch 7.4.1451
问题: Vim hangs when a channel has a callback but isn't referenced.
解决方案: Have channel_unref() only return TRUE when the channel was
           actually freed.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro

Patch 7.4.1452
问题: When a callback adds a syntax item either the redraw doesn't
           happen right away or in the GUI the cursor is in the wrong
           position for a moment. (Jakson Alves de Aquino)
解决方案: Redraw after the callback was invoked.
相关文件: src/channel.c

Patch 7.4.1453
问题: Missing --not-a-term.
解决方案: Add the argument.
相关文件: src/testdir/Make_amiga.mak

Patch 7.4.1454
问题: The exit callback test is flaky.

解决方案: Loop to wait for a short time up to a second.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1455
问题: JSON decoding test for surrogate pairs is in the wrong place.
解决方案: Move the test lines. (Ken Takata)
相关文件: src/testdir/test_json.vim

Patch 7.4.1456
问题: Test 87 fails with Python 3.5.
解决方案: Work around difference. (Taro Muraoka)
相关文件: src/testdir/test87.in

Patch 7.4.1457
问题: Opening a channel with select() is not done properly.
解决方案: Also used read-fds. Use getsockopt() to check for errors. (Ozaki
           Kiichi)
相关文件: src/channel.c

Patch 7.4.1458
问题: When a JSON channel has a callback it may never be cleared.
解决方案: Do not write "DETACH" into a JS or JSON channel.
相关文件: src/channel.c

Patch 7.4.1459 (after 7.4.1457)
问题: MS-Windows doesn't know socklen_t.
解决方案: Use previous method for WIN32.
相关文件: src/channel.c

Patch 7.4.1460
问题: Syntax error in rarely used code.
解决方案: Fix the mch_rename() declaration. (Ken Takata)
相关文件: src/os_unix.c, src/proto/os_unix.pro

Patch 7.4.1461
问题: When starting job on MS-Windows all parts of the command are put
           in quotes.
解决方案: Only use quotes when needed. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.1462
问题: Two more rarely used functions with errors.
解决方案: Add proper argument types. (Dominique Pelle)
相关文件: src/misc2.c, src/termlib.c

Patch 7.4.1463
问题: Configure doesn't find isinf() and isnan() on some systems.
解决方案: Use a configure check that includes math.h.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.1464
问题: When the argument of sort() is zero or empty it fails.
解决方案: Make zero work as documented. (suggested by Yasuhiro Matsumoto)
相关文件: src/eval.c, src/testdir/test_sort.vim

Patch 7.4.1465
问题: Coverity reported possible use of NULL pointer when using buffer output with JSON mode.
解决方案: Make it actually possible to use JSON mode with a buffer. Re-encode the JSON to append it to the buffer.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1466
问题: Coverity reports dead code.
解决方案: Remove the two lines.
相关文件: src/channel.c

Patch 7.4.1467
问题: Can't build without the float feature.
解决方案: Add #ifdefs. (Nick Owens, closes #667)
相关文件: src/eval.c, src/json.c

Patch 7.4.1468
问题: Sort test doesn't test with "1" argument.
解决方案: Also test ignore-case sorting. (Yasuhiro Matsumoto)
相关文件: src/testdir/test_sort.vim

Patch 7.4.1469
问题: Channel test sometimes fails, especially on OS/X. (Kazunobu Kuriyama)
解决方案: Change the && into ||, call getsockopt() in more situations. (Ozaki Kiichi)
相关文件: src/channel.c

Patch 7.4.1470
问题: Coverity reports missing restore.
解决方案: Move json_encode() call up.
相关文件: src/channel.c

Patch 7.4.1471
问题: Missing out-of-memory check. And Coverity warning.
解决方案: Bail out when msg is NULL.
相关文件: src/channel.c

Patch 7.4.1472
问题: Coverity warning for not using return value.
解决方案: Add "(void)".
相关文件: src/os_unix.c

Patch 7.4.1473
问题: Can't build without the autocommand feature.
解决方案: Add #ifdefs. (Yegappan Lakshmanan)
相关文件: src/edit.c, src/main.c, src/syntax.c

Patch 7.4.1474
问题: Compiler warnings without the float feature.
解决方案: Move #ifdefs. (John Marriott)
相关文件: src/eval.c

Patch 7.4.1475
问题: When using hangulinput with utf-8 a CSI character is
            misinterpreted.
解决方案: Convert CSI to K_CSI. (SungHyun Nam)
相关文件: src/ui.c

Patch 7.4.1476
问题: Function arguments marked as unused while they are not.
解决方案: Remove UNUSED. (Yegappan Lakshmanan)
相关文件: src/diff.c, src/eval.c, src/ex_cmds2.c, src/ex_docmd.c,
            src/window.c

Patch 7.4.1477
问题: Test_reltime is flaky, it depends on timing.
解决方案: When it fails run it a second time.
相关文件: src/testdir/runtest.vim

Patch 7.4.1478
问题: ":loadplugin" doesn't take care of ftdetect files.
解决方案: Also load ftdetect scripts when appropriate.
相关文件: src/ex_cmds2.c

Patch 7.4.1479
问题: No testfor ":loadplugin".
解决方案: Add a test. Fix how option is being set.
相关文件: src/ex_cmds2.c, src/testdir/test_loadplugin.vim,
            src/testdir/Make_all.mak

Patch 7.4.1480
问题: Cannot add a pack directory without loading a plugin.
解决方案: Add the :packadd command.
相关文件: src/ex_cmds.h, src/ex_cmds2.c, src/proto/ex_cmds2.pro,
            src/testdir/test_loadplugin.vim, runtime/doc/repeat.txt

Patch 7.4.1481
问题: Can't build with small features.
解决方案: Add #ifdef.
相关文件: src/ex_cmds2.c

Patch 7.4.1482
问题: "timeout" option not supported on ch_eval*().
解决方案: Get and use the timeout option from the argument.
相关文件: src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1483
问题: A one-time callback is not used for a raw channel.
解决方案: Use a one-time callback when it exists.
相关文件: src/channel.c, src/testdir/test_channel.vim,
            src/testdir/test_channel.py

Patch 7.4.1484
问题: Channel "err-io" value "out" is not supported.
解决方案: Connect stderr to stdout if wanted.

相关文件: src/os_unix.c, src/os_win32.c, src/testdir/test_channel.vim,
           src/testdir/test_channel_pipe.py

Patch 7.4.1485
问题: Job input from buffer is not implemented.
解决方案: Implement it. Add "in-top" and "in-bot" options.
相关文件: src/structs.h, src/eval.c, src/channel.c, src/proto/channel.pro,
           src/os_unix.c, src/os_win32.c, src/testdir/test_channel.vim

Patch 7.4.1486
问题: ":loadplugin" is not optimal, some people find it confusing.
解决方案: Only use ":packadd" with an optional "!".
相关文件: src/ex_cmds.h, src/ex_cmds2.c, src/testdir/test_loadplugin.vim,
           src/testdir/test_packadd.vim, src/testdir/Make_all.mak,
           runtime/doc/repeat.txt

Patch 7.4.1487
问题: For WIN32 isinf() is defined as a macro.
解决方案: Define it as an inline function. (ZyX)
相关文件: src/macros.h

Patch 7.4.1488 (after 7.4.1475)
问题: Not using key when result from hangul_string_convert() is NULL.
解决方案: Fall back to not converted string.
相关文件: src/ui.c

Patch 7.4.1489 (after 7.4.1487)
问题: "inline" is not supported by old MSVC.
解决方案: use "__inline". (Ken Takata)
相关文件: src/macros.h

Patch 7.4.1490
问题: Compiler warning for unused function.
解决方案: Add #ifdef. (Dominique Pelle)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1491
问题: Visual-block shift breaks multi-byte characters.
解决方案: Compute column differently. (Yasuhiro Matsumoto) Add a test.
相关文件: src/ops.c, src/testdir/test_visual.vim, src/testdir/Make_all.mak

Patch 7.4.1492
问题: No command line completion for ":packadd".
解决方案: Implement completion. (Hirohito Higashi)
相关文件: src/ex_docmd.c, src/ex_getln.c, src/testdir/test_packadd.vim,
           src/vim.h

Patch 7.4.1493
问题: Wrong callback invoked for zero-id messages.
解决方案: Don't use the first one-time callback when the sequence number
           doesn't match.
相关文件: src/channel.c, src/testdir/test_channel.vim,
           src/testdir/test_channel.py

Patch 7.4.1494
问题: clr_history() does not work properly.
解决方案: Increment hisptr. Add a test. (Yegappan Lakshmanan)
相关文件: src/ex_getln.c, src/testdir/test_history.vim,
            src/testdir/Make_all.mak

Patch 7.4.1495
问题: Compiler warnings when building on Unix with the job feature but
            without the channel feature.
解决方案: Move #ifdefs. (Dominique Pelle)
相关文件: src/os_unix.c

Patch 7.4.1496
问题: Crash when built with GUI but it's not active. (Dominique Pelle)
解决方案: Check gui.in_use.
相关文件: src/channel.c

Patch 7.4.1497
问题: Cursor drawing problem with GTK 3.
解决方案: Handle blinking differently. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1498
问题: Error for locked item when using json_decode(). (Shougo Matsu)
解决方案: Initialize v_lock.
相关文件: src/json.c

Patch 7.4.1499
问题: No error message when :packadd does not find anything.
解决方案: Add an error message. (Hirohito Higashi)
相关文件: runtime/doc/repeat.txt, src/ex_cmds.h, src/ex_cmds2.c,
            src/globals.h, src/testdir/test_packadd.vim

Patch 7.4.1500
问题: Should_free flag set to FALSE.
解决方案: Set it to TRUE. (Neovim 4415)
相关文件: src/ex_eval.c

Patch 7.4.1501
问题: Garbage collection with an open channel is not tested.
解决方案: Call garbagecollect() in the test.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1502
问题: Writing last-but-one line of buffer to a channel isn't implemented
            yet.
解决方案: Implement it. Fix leaving a swap file behind.
相关文件: src/channel.c, src/structs.h, src/memline.c, src/proto/channel.pro

Patch 7.4.1503
问题: Crash when using ch_getjob(). (Damien)
解决方案: Check for a NULL job.
相关文件: src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1504 (after 7.4.1502)
问题: No test for reading last-but-one line.
解决方案: Add a test.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1505
问题: When channel log is enabled get too many "looking for messages"
            log entries.
解决方案: Only give the message after another message.
相关文件: src/channel.c

Patch 7.4.1506
问题: Job cannot read from a file.
解决方案: Implement reading from a file for Unix.
相关文件: src/eval.c, src/os_unix.c, src/os_win32.c,
            src/testdir/test_channel.vim

Patch 7.4.1507
问题: Crash when starting a job fails.
解决方案: Check for the channel to be NULL. (idea by Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.1508
问题: Can't build GvimExt with MingW.
解决方案: Adjust the makefile. (Ben Fritz)
相关文件: src/GvimExt/Make_ming.mak

Patch 7.4.1509
问题: Keeping both a variable for a job and the channel it refers to is
            a hassle.
解决方案: Allow passing the job where a channel is expected. (Damien)
相关文件: src/eval.c, src/testdir/test_channel.vim

Patch 7.4.1510
问题: Channel test fails on AppVeyor.
解决方案: Wait longer than 10 msec if needed.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1511
问题: Statusline highlighting is sometimes wrong.
解决方案: Check for Highlight type. (Christian Brabandt)
相关文件: src/buffer.c

Patch 7.4.1512
问题: Channel input from file not supported on MS-Windows.
解决方案: Implement it. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c, src/testdir/test_channel.vim

Patch 7.4.1513
问题: "J" fails if there are not enough lines. (Christian Neukirchen)
解决方案: Reduce the count, only fail on the last line.
相关文件: src/normal.c, src/testdir/test_join.vim, src/testdir/test_alot.vim

Patch 7.4.1514

问题: Channel output to file not implemented yet.
解决方案: Implement it for Unix.
相关文件: src/os_unix.c, src/testdir/test_channel.vim,
            src/testdir/test_channel_pipe.py

Patch 7.4.1515
问题: Channel test is a bit flaky.
解决方案: Instead of a fixed sleep time wait until an expression evaluates
            to true.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1516
问题: Cannot change file permissions.
解决方案: Add setfperm().
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test_alot.vim,
            src/testdir/test_file_perm.vim

Patch 7.4.1517
问题: Compiler warning with 64bit compiler.
解决方案: Add typecast. (Mike Williams)
相关文件: src/channel.c

Patch 7.4.1518
问题: Channel with disconnected in/out/err is not supported.
解决方案: Implement it for Unix.
相关文件: src/eval.c, src/os_unix.c, src/structs.h,
            src/testdir/test_channel.vim, src/testdir/test_channel_pipe.py

Patch 7.4.1519 (after 7.4.1514)
问题: Channel output to file not implemented for MS-Windows.
解决方案: Implement it. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c, src/testdir/test_channel.vim

Patch 7.4.1520
问题: Channel test: Waiting for a file to appear doesn't work.
解决方案: In waitFor() ignore errors.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1521 (after 7.4.1516)
问题: File permission test fails on MS-Windows.
解决方案: Expect a different permission.
相关文件: src/testdir/test_file_perm.vim

Patch 7.4.1522
问题: Cannot write channel err to a buffer.
解决方案: Implement it.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1523
问题: Writing channel to a file fails on MS-Windows.
解决方案: Disable it for now.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1524

问题: Channel test fails on BSD.
解决方案: Break out of the loop when connect() succeeds. (Ozaki Kiichi)
相关文件: src/channel.c

Patch 7.4.1525
问题: On a high resolution screen the toolbar icons are too small.
解决方案: Add "huge" and "giant" to 'toolbariconsize'. (Brian Gix)
相关文件: src/gui_gtk_x11.c, src/option.h

Patch 7.4.1526
问题: Writing to file and not connecting a channel doesn't work for
            MS-Windows.
解决方案: Make it work. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c, src/testdir/test_channel.vim

Patch 7.4.1527
问题: Channel test is flaky on MS-Windows.
解决方案: Limit the select() timeout to 50 msec and try with a new socket if
            it fails.
相关文件: src/channel.c

Patch 7.4.1528
问题: Using "ever" for packages is confusing.
解决方案: Use "start", as it's related to startup.
相关文件: src/ex_cmds2.c, runtime/doc/repeat.txt

Patch 7.4.1529
问题: Specifying buffer number for channel not implemented yet.
解决方案: Implement passing a buffer number.
相关文件: src/structs.h, src/channel.c, src/eval.c,
            src/testdir/test_channel.vim

Patch 7.4.1530
问题: MS-Windows job_start() closes wrong handle.
解决方案: Close hThread on the process info. (Ken Takata)
相关文件: src/os_win32.c

Patch 7.4.1531
问题: Compiler warning for uninitialized variable. (Dominique Pelle)
解决方案: Always give the variable a value.
相关文件: src/channel.c

Patch 7.4.1532
问题: MS-Windows channel leaks file descriptor.
解决方案: Use CreateFile with the right options. (Yasuhiro Matsumoto)
相关文件: src/os_win32.c

Patch 7.4.1533
问题: Using feedkeys() with an empty string disregards 'x' option.
解决方案: Make 'x' work with an empty string. (Thinca)
相关文件: src/eval.c, src/testdir/test_alot.vim,
            src/testdir/test_feedkeys.vim

Patch 7.4.1534

问题: Compiler warning for shadowed variable. (Kazunobu Kuriyama)
解决方案: Rename it.
相关文件: src/eval.c

Patch 7.4.1535
问题: The feedkeys test has a one second delay.
解决方案: Avoid need_wait_return() to delay. (Hirohito Higashi)
相关文件: src/eval.c

Patch 7.4.1536
问题: Cannot re-use a channel for another job.
解决方案: Add the "channel" option to job_start().
相关文件: src/channel.c, src/eval.c, src/structs.h, src/os_unix.c,
            src/os_win32.c, src/proto/channel.pro,
            src/testdir/test_channel.vim

Patch 7.4.1537
问题: Too many feature flags for pipes, jobs and channels.
解决方案: Only use FEAT_JOB_CHANNEL.
相关文件: src/structs.h, src/feature.h, src/configure.in,
            src/auto/configure, src/config.h.in, src/channel.c, src/eval.c,
            src/gui.c, src/main.c, src/memline.c, src/misc2.c, src/os_mswin.c,
            src/os_unix.c, src/os_win32.c, src/ui.c, src/version.c,
            src/macros.h, src/proto.h, src/vim.h, src/Make_cyg_ming.mak,
            src/Make_bc5.mak, src/Make_mvc.mak

Patch 7.4.1538
问题: Selection with the mouse does not work in command line mode.
解决方案: Use cairo functions. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1539
问题: Too much code in eval.c.
解决方案: Move job and channel code to channel.c.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro,
            src/proto/eval.pro

Patch 7.4.1540
问题: Channel test is a bit flaky.
解决方案: Increase expected wait time.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1541
问题: Missing job_info().
解决方案: Implement it.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro,
            src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1542
问题: job_start() with a list is not tested.
解决方案: Call job_start() with a list.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1543

问题: Channel log methods are not tested.
解决方案: Log job activity and check it.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1544
问题: On Win32 escaping the command does not work properly.
解决方案: Reset 'ssl' when escaping the command. (Yasuhiro Matsumoto)
相关文件: src/channel.c

Patch 7.4.1545
问题: GTK3: horizontal cursor movement in Visual selection not good.
解决方案: Make it work better. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1546
问题: Sticky type checking is more annoying than useful.
解决方案: Remove the error for changing a variable type.
相关文件: src/eval.c, src/testdir/test_assign.vim,
            src/testdir/test_alot.vim, runtime/doc/eval.txt

Patch 7.4.1547
问题: Getting a cterm highlight attribute that is not set results in the
            string "-1".
解决方案: Return an empty string. (Taro Muraoka)
相关文件: src/syntax.c, src/testdir/test_alot.vim,
            src/testdir/test_syn_attr.vim

Patch 7.4.1548 (after 7.4.1546)
问题: Two tests fail.
解决方案: Adjust the expected error number. Remove check for type.
相关文件: src/testdir/test101.ok, src/testdir/test55.in,
            src/testdir/test55.ok

Patch 7.4.1549 (after 7.4.1547)
问题: Test for syntax attributes fails in Win32 GUI.
解决方案: Use an existing font name.
相关文件: src/testdir/test_syn_attr.vim

Patch 7.4.1550
问题: Cannot load packages early.
解决方案: Add the ":packloadall" command.
相关文件: src/ex_cmds.h, src/ex_cmds2.c, src/main.c,
            src/proto/ex_cmds2.pro, src/testdir/test_packadd.vim

Patch 7.4.1551
问题: Cannot generate help tags in all doc directories.
解决方案: Make ":helptags ALL" work.
相关文件: src/ex_cmds2.c, src/proto/ex_cmds2.pro, src/ex_cmds.c, src/vim.h
            src/testdir/test_packadd.vim

Patch 7.4.1552
问题: ":colorscheme" does not use 'packpath'.
解决方案: Also use in "start" and "opt" directories in 'packpath'.
相关文件: src/ex_cmds2.c, src/gui.c, src/hardcopy.c, src/os_mswin.c,

src/spell.c, src/tag.c, src/if_py_both.h, src/vim.h,
src/digraph.c, src/eval.c, src/ex_docmd.c, src/main.c,
src/option.c, src/syntax.c, src/testdir/test_packadd.vim

Patch 7.4.1553
问题: ":runtime" does not use 'packpath'.
解决方案: Add "what" argument.
相关文件: src/ex_cmds2.c, src/vim.h, runtime/doc/repeat.txt,
src/testdir/test_packadd.vim

Patch 7.4.1554
问题: Completion for :colorscheme does not use 'packpath'.
解决方案: Make it work, add a test. (Hirohito Higashi)
相关文件: src/ex_getln.c, src/testdir/test_packadd.vim

Patch 7.4.1555
问题: List of test targets incomplete.
解决方案: Add newly added tests.
相关文件: src/Makefile

Patch 7.4.1556
问题: "make install" changes the help tags file, causing it to differ
from the repository.
解决方案: Move it aside and restore it.
相关文件: src/Makefile

Patch 7.4.1557
问题: Windows cannot be identified.
解决方案: Add a unique window number to each window and functions to use it.
相关文件: src/structs.h, src/window.c, src/eval.c, src/proto/eval.pro,
src/proto/window.pro, src/testdir/test_window_id.vim,
src/testdir/Make_all.mak, runtime/doc/eval.txt

Patch 7.4.1558
问题: It is not easy to find out what windows display a buffer.
解决方案: Add win_findbuf().
相关文件: src/eval.c, src/window.c, src/proto/window.pro,
src/testdir/test_window_id.vim, runtime/doc/eval.txt

Patch 7.4.1559
问题: Passing cookie to a callback is clumsy.
解决方案: Change function() to take arguments and return a partial.
相关文件: src/structs.h, src/channel.c, src/eval.c, src/if_python.c,
src/if_python3.c, src/if_py_both.h, src/json.c,
src/proto/eval.pro, src/testdir/test_partial.vim,
src/testdir/test_alot.vim, runtime/doc/eval.txt

Patch 7.4.1560
问题: Dict options with a dash are more difficult to use.
解决方案: Use an underscore, so that dict.err_io can be used.
相关文件: src/channel.c, src/structs.h, src/testdir/test_channel.vim,
runtime/doc/channel.txt

Patch 7.4.1561 (after 7.4.1559)

问题: Missing update to proto file.
解决方案: Change the proto file.
相关文件: src/proto/channel.pro

Patch 7.4.1562
问题: ":helptags ALL" crashes. (Lcd)
解决方案: Don't free twice.
相关文件: src/ex_cmds.c

Patch 7.4.1563
问题: Partial test fails on windows.
解决方案: Return 1 or -1 from compare function.
相关文件: src/testdir/test_partial.vim

Patch 7.4.1564
问题: An empty list in function() causes an error.
解决方案: Handle an empty list like there is no list of arguments.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1565
问题: Crash when assert_equal() runs into a NULL string.
解决方案: Check for NULL. (Dominique) Add a test.
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 7.4.1566
问题: Compiler warning for shadowed variable. (Kazunobu Kuriyama)
解决方案: Remove the inner one.
相关文件: src/eval.c

Patch 7.4.1567
问题: Crash in assert_fails().
解决方案: Check for NULL. (Dominique Pelle) Add a test.
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 7.4.1568
问题: Using CTRL-] in help on option in parentheses doesn't work.
解决方案: Skip the "(" in "('". (Hirohito Higashi)
相关文件: src/ex_cmds.c

Patch 7.4.1569
问题: Using old style tests for quickfix.
解决方案: Change them to new style tests. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/test106.in,
            src/testdir/test106.ok, src/testdir/test_qf_title.in,
            src/testdir/test_qf_title.ok, src/testdir/test_quickfix.vim

Patch 7.4.1570
问题: There is no way to avoid the message when editing a file.
解决方案: Add the "F" flag to 'shortmess'. (Shougo Matsu, closes #686)
相关文件: runtime/doc/options.txt, src/buffer.c, src/ex_cmds.c,
            src/option.h

Patch 7.4.1571
问题: No test for ":help".

解决方案: Add a test for what 7.4.1568 fixed. (Hirohito Higashi)
相关文件: src/testdir/test_alot.vim, src/testdir/test_help_tagjump.vim

Patch 7.4.1572
问题: Setting 'compatible' in test influences following tests.
解决方案: Turn 'compatible' off again.
相关文件: src/testdir/test_backspace_opt.vim

Patch 7.4.1573
问题: Tests get stuck at the more prompt.
解决方案: Move the backspace test out of test_alot.
相关文件: src/testdir/test_alot.vim, src/testdir/Make_all.mak

Patch 7.4.1574
问题: ":undo 0" does not work. (Florent Fayolle)
解决方案: Make it undo all the way. (closes #688)
相关文件: src/undo.c, src/testdir/test_undolevels.vim,
src/testdir/test_ex_undo.vim, src/testdir/test_alot.vim

Patch 7.4.1575
问题: Using wrong size for struct.
解决方案: Use the size for wide API. (Ken Takata)
相关文件: src/gui_w32.c

Patch 7.4.1576
问题: Write error of viminfo file is not handled properly. (Christian
Neukirchen)
解决方案: Check the return value of fclose(). (closes #682)
相关文件: src/ex_cmds.c

Patch 7.4.1577
问题: Cannot pass "dict.Myfunc" around as a partial.
解决方案: Create a partial when expected.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1578
问题: There is no way to invoke a function later or periodically.
解决方案: Add timer support.
相关文件: src/eval.c, src/ex_cmds2.c, src/screen.c, src/ex_docmd.c,
src/feature.h, src/gui.c, src/proto/eval.pro,
src/proto/ex_cmds2.pro, src/proto/screen.pro, src/structs.h,
src/version.c, src/testdir/test_alot.vim,
src/testdir/test_timers.vim, runtime/doc/eval.txt

Patch 7.4.1579 (after 7.4.1578)
问题: Missing changes in channel.c
解决方案: Include the changes.
相关文件: src/channel.c

Patch 7.4.1580
问题: Crash when using function reference. (Luchr)
解决方案: Set initial refcount. (Ken Takata, closes #690)
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1581
问题: Using ":call dict.func()" where the function is a partial does not work. Using "dict.func()" where the function does not take a Dictionary does not work.
解决方案: Handle partial properly in ":call". (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/testdir/test_partial.vim, src/testdir/test55.ok

Patch 7.4.1582
问题: Get E923 when using function(dict.func, [], dict). (Kent Sibilev) Storing a function with a dict in a variable drops the dict if the function is script-local.
解决方案: Translate the function name. Use dict arg if present.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1583
问题: Warning for uninitialized variable.
解决方案: Initialize it. (Dominique)
相关文件: src/ex_cmds2.c

Patch 7.4.1584
问题: Timers don't work for Win32 console.
解决方案: Add check_due_timer() in WaitForChar().
相关文件: src/os_win32.c

Patch 7.4.1585
问题: Partial is not recognized everywhere.
解决方案: Check for partial in trans_function_name(). (Yasuhiro Matsumoto) Add a test.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1586
问题: Nesting partials doesn't work.
解决方案: Append arguments. (Ken Takata)
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1587
问题: Compiler warnings with 64 bit compiler.
解决方案: Add type casts. (Mike Williams)
相关文件: src/ex_cmds2.c

Patch 7.4.1588
问题: Old style test for quickfix.
解决方案: Turn test 96 into a new style test.
相关文件: src/testdir/Make_all.mak, src/testdir/test96.in, src/testdir/test96.ok, src/testdir/test_quickfix.vim

Patch 7.4.1589
问题: Combining dict and args with partial doesn't always work.
解决方案: Use the arguments from the partial.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1590
问题: Warning for shadowed variable. (Christian Brabandt)
解决方案: Move the variable into a local block.

相关文件: src/eval.c

Patch 7.4.1591
问题: The quickfix title is truncated.
解决方案: Save the command before it is truncated. (Anton Lindqvist)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1592
问题: Quickfix code using memory after being freed. (Dominique Pelle)
解决方案: Detect that the window was closed. (Hirohito Higashi)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1593
问题: Using channel timeout instead of request timeout. (Coverity)
解决方案: Remove the extra assignment.
相关文件: src/channel.c

Patch 7.4.1594
问题: Timers don't work on Unix.
解决方案: Add missing code.
相关文件: src/os_unix.c

Patch 7.4.1595
问题: Not checking for failed open(). (Coverity)
解决方案: Check file descriptor not being negative.
相关文件: src/os_unix.c

Patch 7.4.1596
问题: Memory leak. (Coverity)
解决方案: Free the pattern.
相关文件: src/ex_cmds2.c

Patch 7.4.1597
问题: Memory leak when out of memory. (Coverity)
解决方案: Free the name.
相关文件: src/eval.c

Patch 7.4.1598
问题: When starting the GUI fails a swap file is left behind. (Joerg Plate)
解决方案: Preserve files before exiting. (closes #692)
相关文件: src/main.c, src/gui.c

Patch 7.4.1599
问题: No link to Coverity.
解决方案: Add Coverity badge in README.
相关文件: README.md

Patch 7.4.1600
问题: libs directory is not useful.
解决方案: Remove arp.library, it was only for very old Amiga versions.
相关文件: libs/arp.library, Filelist

Patch 7.4.1601

问题:　　README files take a lot of space in the top directory.
解决方案:　Move most of them to "READMEdir".
相关文件:　Filelist, Makefile, README.txt.info, README_ami.txt,
README_ami.txt.info, README_amibin.txt, README_amibin.txt.info,
README_amisrc.txt, README_amisrc.txt.info, README_bindos.txt,
README_dos.txt, README_extra.txt, README_mac.txt, README_ole.txt,
README_os2.txt, README_os390.txt, README_src.txt,
README_srcdos.txt, README_unix.txt, README_vms.txt,
README_w32s.txt, READMEdir/README.txt.info,
READMEdir/README_ami.txt, READMEdir/README_ami.txt.info,
READMEdir/README_amibin.txt, READMEdir/README_amibin.txt.info,
READMEdir/README_amisrc.txt, READMEdir/README_amisrc.txt.info,
READMEdir/README_bindos.txt, READMEdir/README_dos.txt,
READMEdir/README_extra.txt, READMEdir/README_mac.txt,
READMEdir/README_ole.txt, READMEdir/README_os2.txt,
READMEdir/README_os390.txt, READMEdir/README_src.txt,
READMEdir/README_srcdos.txt, READMEdir/README_unix.txt,
READMEdir/README_vms.txt, READMEdir/README_w32s.txt,

Patch 7.4.1602
问题:　　Info files take space in the top directory.
解决方案:　Move them to "READMEdir".
相关文件:　Filelist, src.info, Contents.info, runtime.info, vimdir.info,
Vim.info, Xxd.info, READMEdir/src.info, READMEdir/Contents.info,
READMEdir/runtime.info, READMEdir/vimdir.info, READMEdir/Vim.info,
READMEdir/Xxd.info

Patch 7.4.1603
问题:　　Timer with an ":echo" command messes up display.
解决方案:　Redraw depending on the mode. (Hirohito Higashi) Avoid the more
prompt being used recursively.
相关文件:　src/screen.c, src/message.c

Patch 7.4.1604
问题:　　Although emoji characters are ambiguous width, best is to treat
them as full width.
解决方案:　Update the Unicode character tables. Add the 'emoji' options.
(Yasuhiro Matsumoto)
相关文件:　runtime/doc/options.txt, runtime/optwin.vim,
runtime/tools/unicode.vim, src/mbyte.c, src/option.c, src/option.h

Patch 7.4.1605
问题:　　Catching exception that won't be thrown.
解决方案:　Remove try/catch.
相关文件:　src/testdir/test55.in

Patch 7.4.1606
问题:　　Having type() handle a Funcref that is or isn't a partial
differently causes problems for existing scripts.
解决方案:　Make type() return the same value. (Thinca)
相关文件:　src/eval.c, src/testdir/test_viml.vim

Patch 7.4.1607
问题:　　Comparing a function that exists on two dicts is not backwards

compatible. (Thinca)
解决方案: Only compare the function, not what the partial adds.
相关文件: src/eval.c, src/testdir/test_alot.vim, src/testdir/test_expr.vim

Patch 7.4.1608
问题: string() doesn't handle a partial.
解决方案: Make a string from a partial.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1609
问题: Contents file is only for Amiga distro.
解决方案: Move it to "README dir". Update some info.
相关文件: Filelist, Contents, README dir/Contents

Patch 7.4.1610
问题: Compiler warnings for non-virtual destructor.
解决方案: Mark the classes final. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/gui_dwrite.cpp, src/if_ole.cpp

Patch 7.4.1611
问题: The versplit feature makes the code unnecessary complicated.
解决方案: Remove FEAT_VERTSPLIT, always support vertical splits when
FEAT_WINDOWS is defined.
相关文件: src/buffer.c, src/charset.c, src/eval.c, src/ex_cmds.c,
src/ex_docmd.c, src/ex_getln.c, src/gui.c, src/if_lua.c,
src/if_mzsch.c, src/if_ruby.c, src/main.c, src/misc1.c,
src/misc2.c, src/move.c, src/normal.c, src/option.c,
src/quickfix.c, src/screen.c, src/syntax.c, src/term.c, src/ui.c,
src/window.c, src/globals.h, src/gui.h, src/if_py_both.h,
src/option.h, src/structs.h, src/term.h
src/feature.h, src/vim.h, src/version.c

Patch 7.4.1612 (after 7.4.1611)
问题: Can't build with small features.
解决方案: Move code and #ifdefs.
相关文件: src/ex_getln.c

Patch 7.4.1613 (after 7.4.1612)
问题: Still can't build with small features.
解决方案: Adjust #ifdefs.
相关文件: src/ex_getln.c

Patch 7.4.1614
问题: Still quickfix test in old style.
解决方案: Turn test 10 into a new style test.
相关文件: src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/main.aap, src/testdir/test10.in,
src/testdir/test10.ok, src/testdir/test_quickfix.vim,
src/testdir/test10a.in, src/testdir/test10a.ok

Patch 7.4.1615
问题: Build fails with tiny features.
解决方案: Adjust #ifdefs.
相关文件: src/normal.c, src/window.c

Patch 7.4.1616
问题: Malformed channel request causes a hang.
解决方案: Drop malformed message. (Damien)
相关文件: src/channel.c, src/testdir/test_channel.vim,
src/testdir/test_channel.py

Patch 7.4.1617
问题: When a JSON message is split it isn't decoded.
解决方案: Wait a short time for the rest of the message to arrive.
相关文件: src/channel.c, src/json.c, src/structs.h,
src/testdir/test_channel.vim, src/testdir/test_channel.py

Patch 7.4.1618
问题: Starting job with output to buffer changes options in the current
buffer.
解决方案: Set "curbuf" earlier. (Yasuhiro Matsumoto)
相关文件: src/channel.c

Patch 7.4.1619
问题: When 'fileformats' is set in the vimrc it applies to new buffers
but not the initial buffer.
解决方案: Set 'fileformat' when starting up. (Mike Williams)
相关文件: src/option.c

Patch 7.4.1620
问题: Emoji characters are not considered as a kind of word character.
解决方案: Give emoji characters a word class number. (Yasuhiro Matsumoto)
相关文件: src/mbyte.c

Patch 7.4.1621
问题: Channel test doesn't work with Python 2.6.
解决方案: Add number in formatting placeholder. (Wiredool)
相关文件: src/testdir/test_channel.py

Patch 7.4.1622
问题: Channel demo doesn't work with Python 2.6.
解决方案: Add number in formatting placeholder
相关文件: runtime/tools/demoserver.py

Patch 7.4.1623
问题: All Channels share the message ID, it keeps getting bigger.
解决方案: Use a message ID per channel.
相关文件: src/channel.c, src/proto/channel.pro, src/structs.h

Patch 7.4.1624
问题: Can't get info about a channel.
解决方案: Add ch_info().
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro,
src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1625
问题: Trying to close file descriptor that isn't open.
解决方案: Check for negative number.

相关文件: src/os_unix.c

Patch 7.4.1626 (after 7.4.1624)
问题: Missing changes to structs.
解决方案: Include the changes.
相关文件: src/structs.h

Patch 7.4.1627
问题: Channel out_cb and err_cb are not tested.
解决方案: Add a test.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1628
问题: 64-bit Compiler warning.
解决方案: Change type of variable. (Mike Williams)
相关文件: src/channel.c

Patch 7.4.1629
问题: Handling emoji characters as full width has problems with
backwards compatibility.
解决方案: Remove ambiguous and double width characters from the emoji table.
Use a separate table for the character class.
(partly by Yasuhiro Matsumoto)
相关文件: runtime/tools/unicode.vim, src/mbyte.c

Patch 7.4.1630
问题: Unicode table for double width is outdated.
解决方案: Update to the latest Unicode standard.
相关文件: src/mbyte.c

Patch 7.4.1631
问题: Compiler doesn't understand switch on all enum values. (Tony
Mechelynck)
解决方案: Initialize variable.
相关文件: src/channel.c

Patch 7.4.1632
问题: List of test targets is outdated.
解决方案: Update to current list of test targets.
相关文件: src/Makefile

Patch 7.4.1633
问题: If the help tags file was removed "make install" fails. (Tony
Mechelynck)
解决方案: Only try moving the file if it exists.
相关文件: src/Makefile

Patch 7.4.1634
问题: Vertical movement after CTRL-A ends up in the wrong column.
(Urtica Dioica)
解决方案: Set curswant when appropriate. (Hirohito Higashi)
相关文件: src/ops.c, src/testdir/test_increment.vim

Patch 7.4.1635

问题:　　Channel test is a bit flaky.
解决方案:　Remove 'DETACH' if it's there.
相关文件:　src/testdir/test_channel.vim

Patch 7.4.1636
问题:　　When 'F' is in 'shortmess' the prompt for the encryption key isn't displayed. (Toothpik)
解决方案:　Reset msg_silent.
相关文件:　src/ex_getln.c

Patch 7.4.1637
问题:　　Can't build with older MinGW compiler.
解决方案:　Change option from c++11 to gnu++11. (Ken Takata)
相关文件:　src/Make_cyg_ming.mak

Patch 7.4.1638
问题:　　When binding a function to a dict the reference count is wrong.
解决方案:　Decrement dict reference count, only reference the function when actually making a copy. (Ken Takata)
相关文件:　src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1639
问题:　　Invoking garbage collection may cause a double free.
解决方案:　Don't free the dict in a partial when recursive is FALSE.
相关文件:　src/eval.c

Patch 7.4.1640
问题:　　Crash when an autocommand changes a quickfix list. (Dominique)
解决方案:　Check whether an entry is still valid. (Yegappan Lakshmanan, Hirohito Higashi)
相关文件:　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1641
问题:　　Using unterminated string.
解决方案:　Add NUL before calling vim_strsave_shellescape(). (James McCoy)
相关文件:　src/eval.c, src/testdir/test105.in, src/testdir/test105.ok

Patch 7.4.1642
问题:　　Handling emoji characters as full width has problems with backwards compatibility.
解决方案:　Only put characters in the 1f000 range in the emoji table.
相关文件:　runtime/tools/unicode.vim, src/mbyte.c

Patch 7.4.1643 (after 7.4.1641)
问题:　　Terminating file name has side effects.
解决方案:　Restore the character. (mostly by James McCoy, closes #713)
相关文件:　src/eval.c, src/testdir/test105.in, src/testdir/test105.ok

Patch 7.4.1644
问题:　　Using string() on a partial that exists in the dictionary it binds results in an error. (Nikolai Pavlov)
解决方案:　Make string() not fail on a recursively nested structure. (Ken Takata)
相关文件:　src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1645
问题: When a dict contains a partial it can't be redefined as a function. (Nikolai Pavlov)
解决方案: Remove the partial when overwriting with a function.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1646
问题: Using Python vim.bindeval() on a partial doesn't work. (Nikolai Pavlov)
解决方案: Add VAR_PARTIAL support in Python.
相关文件: src/if_py_both.h, src/testdir/test_partial.vim

Patch 7.4.1647
问题: Using freed memory after setqflist() and ":caddbuffer". (Dominique)
解决方案: Set qf_ptr when adding the first item to the quickfix list.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1648
问题: Compiler has a problem copying a string into di_key[]. (Yegappan Lakshmanan)
解决方案: Add dictitem16_T.
相关文件: src/structs.h, src/eval.c

Patch 7.4.1649
问题: The matchit plugin needs to be copied to be used.
解决方案: Put the matchit plugin in an optional package.
相关文件: Filelist, runtime/macros/matchit.vim, runtime/macros/matchit.txt, runtime/macros/README.txt, src/Makefile, runtime/pack/dist/opt/matchit/plugin/matchit.vim, runtime/pack/dist/opt/matchit/doc/matchit.txt, runtime/pack/dist/opt/matchit/doc/tags, runtime/doc/usr_05.txt, runtime/doc/usr_toc.txt

Patch 7.4.1650
问题: Quickfix test fails.
解决方案: Accept any number of matches.
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1651
问题: Some dead (MSDOS) code remains.
解决方案: Remove the unused lines. (Ken Takata)
相关文件: src/misc1.c

Patch 7.4.1652
问题: Old style test for fnamemodify().
解决方案: Turn it into a new style test.
相关文件: src/testdir/test105.in, src/testdir/test105.ok, src/testdir/test_fnamemodify.vim, src/testdir/test_alot.vim, src/testdir/Make_all.mak

Patch 7.4.1653 (after 7.4.1649)
问题: Users who loaded matchit.vim manually have to change their startup. (Gary Johnson)

解决方案: Add a file in the old location that loads the package.
相关文件: runtime/macros/matchit.vim, Filelist

Patch 7.4.1654
问题: Crash when using expand('%:S') in a buffer without a name.
解决方案: Don't set a NUL. (James McCoy, closes #714)
相关文件: src/eval.c, src/testdir/test_fnamemodify.vim

Patch 7.4.1655
问题: remote_expr() hangs. (Ramel)
解决方案: Check for messages in the waiting loop.
相关文件: src/if_xcmdsrv.c

Patch 7.4.1656
问题: Crash when using partial with a timer.
解决方案: Increment partial reference count. (Hirohito Higashi)
相关文件: src/eval.c, src/testdir/test_timers.vim

Patch 7.4.1657
问题: On Unix in a terminal: channel messages are not handled right away.
           (Jackson Alves de Aquino)
解决方案: Break the loop for timers when something was received.
相关文件: src/os_unix.c

Patch 7.4.1658
问题: A plugin does not know when VimEnter autocommands were already
           triggered.
解决方案: Add the v:vim_did_enter variable.
相关文件: src/eval.c, src/main.c, src/vim.h, src/testdir/test_autocmd.vim,
           src/testdir/test_alot.vim, runtime/doc/autocmd.txt,
           runtime/doc/eval.txt

Patch 7.4.1659 (after 7.4.1657)
问题: Compiler warning for argument type. (Manuel Ortega)
解决方案: Remove "&".
相关文件: src/os_unix.c

Patch 7.4.1660
问题: has('patch-7.4.1') doesn't work.
解决方案: Fix off-by-one error. (Thinca)
相关文件: src/eval.c, src/testdir/test_expr.vim, src/testdir/test60.in,
           src/testdir/test60.ok

Patch 7.4.1661
问题: No test for special characters in channel eval command.
解决方案: Testing sending and receiving text with special characters.
相关文件: src/testdir/test_channel.vim, src/testdir/test_channel.py

Patch 7.4.1662
问题: No test for an invalid Ex command on a channel.
解决方案: Test handling an invalid command gracefully. Avoid getting an
           error message, do write it to the channel log.
相关文件: src/channel.c, src/testdir/test_channel.vim,
           src/testdir/test_channel.py

Patch 7.4.1663
问题: In tests it's often useful to check if a pattern matches.
解决方案: Add assert_match().
相关文件: src/eval.c, src/testdir/test_assert.vim,
src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1664
问题: Crash in :cgetexpr.
解决方案: Check for NULL pointer. (Dominique) Add a test.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1665
问题: Crash when calling job_start() with a NULL string. (Dominique)
解决方案: Check for an invalid argument.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1666
问题: When reading JSON from a channel all readahead is used.
解决方案: Use the fill function to reduce overhead.
相关文件: src/channel.c, src/json.c, src/structs.h

Patch 7.4.1667
问题: Win32: waiting on a pipe with fixed sleep time.
解决方案: Start with a short delay and increase it when looping.
相关文件: src/channel.c

Patch 7.4.1668
问题: channel_get_all() does multiple allocations.
解决方案: Compute the size and allocate once.
相关文件: src/channel.c

Patch 7.4.1669
问题: When writing buffer lines to a pipe Vim may block.
解决方案: Avoid blocking, write more lines later.
相关文件: src/channel.c, src/misc2.c, src/os_unix.c, src/structs.h,
src/vim.h, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 7.4.1670
问题: Completion doesn't work well for a variable containing "#".
解决方案: Recognize the "#". (Watiko)
相关文件: src/eval.c

Patch 7.4.1671
问题: When help exists in multiple languages, adding @ab while "ab" is
the default help language is unnecessary.
解决方案: Leave out "@ab" when not needed. (Ken Takata)
相关文件: src/ex_getln.c

Patch 7.4.1672
问题: The Dvorak support is a bit difficult to install.
解决方案: Turn it into an optional package.
相关文件: runtime/macros/dvorak, runtime/macros/README.txt,
runtime/pack/dist/opt/dvorak/plugin/dvorak.vim,

runtime/pack/dist/opt/dvorak/dvorak/enable.vim,
runtime/pack/dist/opt/dvorak/dvorak/disable.vim

Patch 7.4.1673
问题: The justify plugin has to be copied or sourced to be used.
解决方案: Turn it into a package.
相关文件: runtime/macros/justify.vim, runtime/macros/README.txt,
runtime/pack/dist/opt/justify/plugin/justify.vim, Filelist

Patch 7.4.1674
问题: The editexisting plugin has to be copied or sourced to be used.
解决方案: Turn it into a package.
相关文件: runtime/macros/editexisting.vim, runtime/macros/README.txt,
runtime/pack/dist/opt/editexisting/plugin/editexisting.vim,
Filelist

Patch 7.4.1675
问题: The swapmous plugin has to be copied or sourced to be used.
解决方案: Turn it into the swapmouse package.
相关文件: runtime/macros/swapmous.vim, runtime/macros/README.txt,
runtime/pack/dist/opt/swapmouse/plugin/swapmouse.vim, Filelist

Patch 7.4.1676
问题: The shellmenu plugin has to be copied or sourced to be used.
解决方案: Turn it into a package.
相关文件: runtime/macros/shellmenu.vim, runtime/macros/README.txt,
runtime/pack/dist/opt/shellmenu/plugin/shellmenu.vim, Filelist

Patch 7.4.1677
问题: A reference to the removed file_select plugin remains.
解决方案: Remove it.
相关文件: runtime/macros/README.txt

Patch 7.4.1678
问题: Warning for unused argument.
解决方案: Add UNUSED. (Dominique Pelle)
相关文件: src/if_mzsch.c

Patch 7.4.1679
问题: Coverity: copying value of v_lock without initializing it.
解决方案: Init v_lock in rettv_list_alloc() and rettv_dict_alloc().
相关文件: src/eval.c

Patch 7.4.1680
问题: Coverity warns for not checking name length (false positive).
解决方案: Only copy the characters we know are there.
相关文件: src/channel.c

Patch 7.4.1681
问题: Coverity warns for fixed size buffer length (false positive).
解决方案: Add a check for the name length.
相关文件: src/eval.c

Patch 7.4.1682

问题: Coverity: no check for NULL.
解决方案: Add check for invalid argument to assert_match().
相关文件: src/eval.c

Patch 7.4.1683
问题: Generated .bat files do not support --nofork.
解决方案: Add check for --nofork. Also add "setlocal". (Kevin Cantú,
            closes #659)
相关文件: src/dosinst.c

Patch 7.4.1684
问题: README text is slightly outdated.
解决方案: Mention the README_dir directory.
相关文件: README.md, README.txt

Patch 7.4.1685
问题: There is no easy way to get all the information about a match.
解决方案: Add matchstrpos(). (Ozaki Kiichi)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/eval.c,
            src/testdir/test_alot.vim, src/testdir/test_matchstrpos.vim

Patch 7.4.1686
问题: When running tests $HOME/.viminfo is written. (James McCoy)
解决方案: Add 'nviminfo' to the 'viminfo' option. (closes #722)
相关文件: src/testdir/test_backspace_opt.vim, src/testdir/test_viminfo.vim,
            src/testdir/runtest.vim.

Patch 7.4.1687
问题: The channel close_cb option does not work.
解决方案: Use jo_close_partial instead of jo_err_partial. (Damien)
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1688
问题: MzScheme does not support partial.
解决方案: Add minimal partial support. (Ken Takata)
相关文件: src/if_mzsch.c

Patch 7.4.1689
问题: Ruby interface has inconsistent coding style.
解决方案: Fix the coding style. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.4.1690
问题: Can't compile with the conceal feature but without multi-byte.
解决方案: Adjust #ifdef. (Owen Leibman)
相关文件: src/eval.c, src/window.c

Patch 7.4.1691
问题: When switching to a new buffer and an autocommand applies syntax
            highlighting an ml_get error may occur.
解决方案: Check "syn_buf" against the buffer in the window. (Alexander von
            Buddenbrock, closes #676)
相关文件: src/syntax.c

Patch 7.4.1692
问题: feedkeys('i', 'x') gets stuck, waits for a character to be typed.
解决方案: Behave like ":normal". (Yasuhiro Matsumoto)
相关文件: src/eval.c, src/testdir/test_feedkeys.vim

Patch 7.4.1693
问题: Building the Perl interface gives compiler warnings.
解决方案: Remove a pragma. Add noreturn attributes. (Damien)
相关文件: src/if_perl.xs

Patch 7.4.1694
问题: Win32 gvim doesn't work with "dvorakj" input method.
解决方案: Wait for QS_ALLINPUT instead of QS_ALLEVENTS. (Yukihiro Nakadaira)
相关文件: src/gui_w32.c

Patch 7.4.1695
问题: ":syn reset" clears the effect ":syn iskeyword". (James McCoy)
解决方案: Remove clearing the syntax keywords.
相关文件: src/syntax.c

Patch 7.4.1696
问题: When using :stopinsert in a silent mapping the "INSERT" message
isn't cleared. (Coacher)
解决方案: Always clear the message. (Christian Brabandt, closes #718)
相关文件: src/ex_docmd.c, src/proto/screen.pro, src/screen.c

Patch 7.4.1697
问题: Display problems when the 'ambiwidth' and 'emoji' options are not
set properly or the terminal doesn't behave as expected.
解决方案: After drawing an ambiguous width character always position the
cursor.
相关文件: src/mbyte.c, src/screen.c, src/proto/mbyte.pro

Patch 7.4.1698
问题: Two tests fail when running tests with MinGW. (Michael Soyka)
解决方案: Convert test_getcwd.ok test_wordcount.ok to unix fileformat.
相关文件: src/testdir/Make_ming.mak

Patch 7.4.1699
问题: :packadd does not work the same when used early or late.
解决方案: Always load plugins matching "plugin/**/*.vim".
相关文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 7.4.1700
问题: Equivalence classes are not properly tested.
解决方案: Add tests for multi-byte and latin1. Fix an error. (Owen Leibman)
相关文件: src/regexp.c, src/testdir/Make_all.mak,
src/testdir/test_alot_latin.vim, src/testdir/test_alot_utf8.vim,
src/testdir/test_regexp_latin.vim,
src/testdir/test_regexp_utf8.vim

Patch 7.4.1701
问题: Equivalence classes still tested in old style tests.
解决方案: Remove the duplicate.

相关文件: src/testdir/test44.in, src/testdir/test44.ok,
           src/testdir/test99.in, src/testdir/test99.ok

Patch 7.4.1702
问题: Using freed memory when parsing 'printoptions' fails.
解决方案: Save the old options and restore them in case of an error.
           (Dominique)
相关文件: src/hardcopy.c, src/testdir/test_hardcopy.vim

Patch 7.4.1703
问题: Can't assert for not equal and not matching.
解决方案: Add assert_notmatch() and assert_notequal().
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test_assert.vim

Patch 7.4.1704
问题: Using freed memory with "wincmd p". (Dominique Pelle)
解决方案: Also clear "prevwin" in other tab pages.
相关文件: src/window.c

Patch 7.4.1705
问题: The 'guifont' option does not allow for a quality setting.
解决方案: Add the "q" item, supported on MS-Windows. (Yasuhiro Matsumoto)
相关文件: runtime/doc/options.txt, src/gui_w32.c, src/os_mswin.c,
           src/proto/os_mswin.pro

Patch 7.4.1706
问题: Old style function declaration breaks build.
解决方案: Remove __ARGS().
相关文件: src/proto/os_mswin.pro

Patch 7.4.1707
问题: Cannot use empty dictionary key, even though it can be useful.
解决方案: Allow using an empty dictionary key.
相关文件: src/hashtab.c, src/eval.c, src/testdir/test_expr.vim

Patch 7.4.1708
问题: New regexp engine does not work properly with EBCDIC.
解决方案: Define equivalence class characters. (Owen Leibman)
相关文件: src/regexp_nfa.c

Patch 7.4.1709
问题: Mistake in #ifdef.
解决方案: Change PROOF_QUALITY to DRAFT_QUALITY. (Ken Takata)
相关文件: src/os_mswin.c

Patch 7.4.1710
问题: Not all output of an external command is read.
解决方案: Avoid timing out when the process has exited. (closes #681)
相关文件: src/os_unix.c

Patch 7.4.1711
问题: When using try/catch in 'statusline' it is still considered an
           error and the status line will be disabled.
解决方案: Check did_emsg instead of called_emsg. (haya14busa, closes #729)

相关文件: src/screen.c, src/testdir/test_statusline.vim,
           src/testdir/test_alot.vim

Patch 7.4.1712
问题: For plugins in packages, plugin authors need to take care of all
           dependencies.
解决方案: When loading "start" packages and for :packloadall, first add all
           directories to 'runtimepath' before sourcing plugins.
相关文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 7.4.1713
问题: GTK GUI doesn't work on Wayland.
解决方案: Specify that only the X11 backend is allowed. (Simon McVittie)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1714
问题: Non-GUI specific settings in the gvimrc_example file.
解决方案: Move some settings to the vimrc_example file. Remove setting
           'hlsearch' again. (suggested by Hirohito Higashi)
相关文件: runtime/vimrc_example.vim, runtime/gvimrc_example.vim

Patch 7.4.1715
问题: Double free when a partial is in a cycle with a list or dict.
           (Nikolai Pavlov)
解决方案: Do not free a nested list or dict used by the partial.
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1716
问题: 'autochdir' doesn't work for the first file. (Rob Hoelz)
解决方案: Call DO_AUTOCHDIR after startup. (Christian Brabandt, closes #704)
相关文件: src/main.c

Patch 7.4.1717
问题: Leaking memory when opening a channel fails.
解决方案: Unreference partials in job options.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro,
           src/testdir/test_channel.vim

Patch 7.4.1718
问题: Coverity: not using return value of set_ref_in_item().
解决方案: Use the return value.
相关文件: src/eval.c

Patch 7.4.1719
问题: Leaking memory when there is a cycle involving a job and a
           partial.
解决方案: Add a copyID to job and channel. Set references in items referred
           by them. Go through all jobs and channels to find unreferenced
           items. Also, decrement reference counts when garbage collecting.
相关文件: src/eval.c, src/channel.c, src/netbeans.c, src/globals.h,
           src/ops.c, src/regexp.c, src/tag.c, src/proto/channel.pro,
           src/proto/eval.pro, src/testdir/test_partial.vim, src/structs.h

Patch 7.4.1720

问题: Tests fail without the job feature.
解决方案: Skip tests when the job feature is not present.
相关文件: src/testdir/test_partial.vim

Patch 7.4.1721
问题: The vimtbar files are unused.
解决方案: Remove them. (Ken Takata)
相关文件: src/vimtbar.dll, src/vimtbar.h, src/vimtbar.lib, Filelist

Patch 7.4.1722
问题: Crash when calling garbagecollect() after starting a job.
解决方案: Set the copyID on job and channel. (Hirohito Higashi, Ozaki
            Kiichi)
相关文件: src/eval.c

Patch 7.4.1723
问题: When using try/catch in 'tabline' it is still considered an
            error and the tabline will be disabled.
解决方案: Check did_emsg instead of called_emsg. (haya14busa, closes #746)
相关文件: src/screen.c, src/testdir/test_tabline.vim,
            src/testdir/test_alot.vim

Patch 7.4.1724 (after 7.4.1723)
问题: Tabline test fails in GUI.
解决方案: Remove 'e' from 'guioptions'.
相关文件: src/testdir/test_tabline.vim

Patch 7.4.1725
问题: Compiler errors for non-ANSI compilers.
解决方案: Remove // comment. Remove comma at end of enum. (Michael Jarvis)
相关文件: src/eval.c

Patch 7.4.1726
问题: ANSI compiler complains about string length.
解决方案: Split long string in two parts. (Michael Jarvis)
相关文件: src/ex_cmds.c

Patch 7.4.1727
问题: Cannot detect a crash in tests when caused by garbagecollect().
解决方案: Add garbagecollect_for_testing(). Do not free a job if is still
            useful.
相关文件: src/channel.c, src/eval.c, src/getchar.c, src/main.c, src/vim.h,
            src/proto/eval.pro, src/testdir/runtest.vim,
            src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1728
问题: The help for functions require a space after the "(".
解决方案: Make CTRL-] on a function name ignore the arguments. (Hirohito
            Higashi)
相关文件: src/ex_cmds.c, src/testdir/test_help_tagjump.vim,
            runtime/doc/eval.txt

Patch 7.4.1729
问题: The Perl interface cannot use 'print' operator for writing

directly in standard IO.
解决方案: Add a minimal implementation of PerlIO Layer feature and try to use it for STDOUT/STDERR. (Damien)
相关文件: src/if_perl.xs, src/testdir/test_perl.vim

Patch 7.4.1730
问题: It is not easy to get a character out of a string.
解决方案: Add strgetchar() and strcharpart().
相关文件: src/eval.c, src/testdir/test_expr.vim

Patch 7.4.1731
问题: Python: turns partial into simple funcref.
解决方案: Use partials like partials. (Nikolai Pavlov, closes #734)
相关文件: runtime/doc/if_pyth.txt, src/eval.c, src/if_py_both.h, src/if_python.c, src/if_python3.c, src/proto/eval.pro, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.1732
问题: Folds may close when using autocomplete. (Anmol Sethi)
解决方案: Increment/decrement disable_fold. (Christian Brabandt, closes #643)
相关文件: src/edit.c, src/fold.c, src/globals.h

Patch 7.4.1733
问题: "make install" doesn't know about cross-compiling. (Christian Neukirchen)
解决方案: Add CROSS_COMPILING. (closes #740)
相关文件: src/configure.in, src/auto/configure, src/config.mk.in, src/Makefile

Patch 7.4.1734 (after 7.4.1730)
问题: Test fails when not using utf-8.
解决方案: Split test in regular and utf-8 part.
相关文件: src/testdir/test_expr.vim, src/testdir/test_expr_utf8.vim, src/testdir/test_alot_utf8.vim

Patch 7.4.1735
问题: It is not possible to only see part of the message history. It is not possible to clear messages.
解决方案: Add a count to ":messages" and a clear argument. (Yasuhiro Matsumoto)
相关文件: runtime/doc/message.txt, src/ex_cmds.h, src/message.c, src/testdir/test_messages.vim, src/testdir/test_alot.vim

Patch 7.4.1736 (after 7.4.1731)
问题: Unused variable.
解决方案: Remove it. (Yasuhiro Matsumoto)
相关文件: src/if_py_both.h

Patch 7.4.1737
问题: Argument marked as unused is used.
解决方案: Remove UNUSED.
相关文件: src/message.c

Patch 7.4.1738
问题: Count for ":messages" depends on number of lines.
解决方案: Add ADDR_OTHER address type.
相关文件: src/ex_cmds.h

Patch 7.4.1739
问题: Messages test fails on MS-Windows.
解决方案: Adjust the asserts. Skip the "messages maintainer" line if not
            showing all messages.
相关文件: src/message.c, src/testdir/test_messages.vim

Patch 7.4.1740
问题: syn-cchar defined with matchadd() does not appear if there are no
            other syntax definitions which matches buffer text.
解决方案: Check for startcol. (Ozaki Kiichi, haya14busa, closes #757)
相关文件: src/screen.c, src/testdir/Make_all.mak,
            src/testdir/test_alot_utf8.vim, src/testdir/test_match_conceal.in,
            src/testdir/test_match_conceal.ok,
            src/testdir/test_matchadd_conceal.vim,
            src/testdir/test_matchadd_conceal_utf8.vim,
            src/testdir/test_undolevels.vim

Patch 7.4.1741
问题: Not testing utf-8 characters.
解决方案: Move the right asserts to the test_expr_utf8 test.
相关文件: src/testdir/test_expr.vim, src/testdir/test_expr_utf8.vim

Patch 7.4.1742
问题: strgetchar() does not work correctly.
解决方案: use mb_cptr2len(). Add a test. (Naruhiko Nishino)
相关文件: src/eval.c, src/testdir/test_expr_utf8.vim

Patch 7.4.1743
问题: Clang warns for uninitialized variable. (Michael Jarvis)
解决方案: Initialize it.
相关文件: src/if_py_both.h

Patch 7.4.1744
问题: Python: Converting a sequence may leak memory.
解决方案: Decrement a reference. (Nikolai Pavlov)
相关文件: src/if_py_both.h

Patch 7.4.1745
问题: README file is not clear about where to get Vim.
解决方案: Add links to github, releases and the Windows installer.
            (Suggested by Christian Brabandt)
相关文件: README.md, README.txt

Patch 7.4.1746
问题: Memory leak in Perl.
解决方案: Decrement the reference count. Add a test. (Damien)
相关文件: src/if_perl.xs, src/testdir/test_perl.vim

Patch 7.4.1747
问题: Coverity: missing check for NULL pointer.
解决方案: Check for out of memory.
相关文件: src/if_py_both.h

Patch 7.4.1748
问题: "gD" does not find match in first column of first line. (Gary Johnson)
解决方案: Accept match at the cursor.
相关文件: src/normal.c, src/testdir/test_goto.vim, src/testdir/test_alot.vim

Patch 7.4.1749
问题: When using GTK 3.20 there are a few warnings.
解决方案: Use new functions when available. (Kazunobu Kuriyama)
相关文件: src/gui_beval.c src/gui_gtk_x11.c

Patch 7.4.1750
问题: When a buffer gets updated while in command line mode, the screen may be messed up.
解决方案: Postpone the redraw when the screen is scrolled.
相关文件: src/channel.c

Patch 7.4.1751
问题: Crash when 'tagstack' is off. (Dominique Pelle)
解决方案: Fix it. (Hirohito Higashi)
相关文件: src/tag.c, src/testdir/test_alot.vim, src/testdir/test_tagjump.vim

Patch 7.4.1752
问题: When adding to the quickfix list the current position is reset.
解决方案: Do not reset the position when not needed. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1753
问题: "noinsert" in 'completeopt' is sometimes ignored.
解决方案: Set the variables when the 'completeopt' was set. (Ozaki Kiichi)
相关文件: src/edit.c, src/option.c, src/proto/edit.pro

Patch 7.4.1754
问题: When 'filetype' was set and reloading a buffer which does not cause it to be set, the syntax isn't loaded. (KillTheMule)
解决方案: Remember whether the FileType event was fired and fire it if not. (Anton Lindqvist, closes #747)
相关文件: src/fileio.c, src/testdir/test_syntax.vim

Patch 7.4.1755
问题: When using getreg() on a non-existing register a NULL list is returned. (Bjorn Linse)
解决方案: Allocate an empty list. Add a test.
相关文件: src/eval.c, src/testdir/test_expr.vim

Patch 7.4.1756
问题: "dll" options are not expanded.
解决方案: Expand environment variables. (Ozaki Kiichi)
相关文件: src/option.c, src/testdir/test_alot.vim,

src/testdir/test_expand_dllpath.vim

Patch 7.4.1757
问题: When using complete() it may set 'modified' even though nothing was inserted.
解决方案: Use Down/Up instead of Next/Previous match. (Shougo Matsu, closes #745)
相关文件: src/edit.c

Patch 7.4.1758
问题: Triggering CursorHoldI when in CTRL-X mode causes problems.
解决方案: Do not trigger CursorHoldI in CTRL-X mode. Add "!" flag to feedkeys() (test with that didn't work though).
相关文件: src/edit.c, src/eval.c

Patch 7.4.1759
问题: When using feedkeys() in a timer the inserted characters are not used right away.
解决方案: Break the wait loop when characters have been added to typebuf. use this for testing CursorHoldI.
相关文件: src/gui.c, src/os_win32.c, src/os_unix.c, src/testdir/test_autocmd.vim

Patch 7.4.1760 (after 7.4.1759)
问题: Compiler warning for unused variable.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/os_win32.c

Patch 7.4.1761
问题: Coverity complains about ignoring return value.
解决方案: Add "(void)" to get rid of the warning.
相关文件: src/eval.c

Patch 7.4.1762
问题: Coverity: useless assignments.
解决方案: Remove them.
相关文件: src/search.c

Patch 7.4.1763
问题: Coverity: useless assignment.
解决方案: Add #if 0.
相关文件: src/spell.c

Patch 7.4.1764
问题: C++ style comment. (Ken Takata)
解决方案: Finish the work started here: don't call perror() when stderr isn't working.
相关文件: src/os_unix.c

Patch 7.4.1765
问题: Undo options are not together in the options window.
解决方案: Put them together. (Gary Johnson)
相关文件: runtime/optwin.vim

Patch 7.4.1766
问题: Building instructions for MS-Windows are outdated.
解决方案: Mention setting SDK_INCLUDE_DIR. (Ben Franklin, closes #771) Move outdated instructions further down.
相关文件: src/INSTALLpc.txt

Patch 7.4.1767
问题: When installing Vim on a GTK system the icon cache is not updated.
解决方案: Update the GTK icon cache when possible. (Kazunobu Kuriyama)
相关文件: src/Makefile, src/configure.in, src/config.mk.in, src/auto/configure

Patch 7.4.1768
问题: Arguments of setqflist() are not checked properly.
解决方案: Add better checks, add a test. (Nikolai Pavlov, Hirohito Higashi, closes #661)
相关文件: src/eval.c, src/testdir/test_quickfix.vim

Patch 7.4.1769
问题: No "closed", "errors" and "encoding" attribute on Python output.
解决方案: Add attributes and more tests. (Roland Puntaier, closes #622)
相关文件: src/if_py_both.h, src/if_python.c, src/if_python3.c, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.1770
问题: Cannot use true color in the terminal.
解决方案: Add the 'guicolors' option. (Nikolai Pavlov)
相关文件: runtime/doc/options.txt, runtime/doc/term.txt, runtime/doc/various.txt, src/auto/configure, src/config.h.in, src/configure.in, src/eval.c, src/globals.h, src/hardcopy.c, src/option.c, src/option.h, src/proto/term.pro, src/screen.c, src/structs.h, src/syntax.c, src/term.c, src/term.h, src/version.c, src/vim.h

Patch 7.4.1771 (after 7.4.1768)
问题: Warning for unused variable.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/eval.c

Patch 7.4.1772 (after 7.4.1767)
问题: Installation fails when $GTK_UPDATE_ICON_CACHE is empty.
解决方案: Add quotes. (Kazunobu Kuriyama)
相关文件: src/Makefile

Patch 7.4.1773 (after 7.4.1770)
问题: Compiler warnings. (Dominique Pelle)
解决方案: Add UNUSED. Add type cast. Avoid a buffer overflow.
相关文件: src/syntax.c, src/term.c

Patch 7.4.1774 (after 7.4.1770)
问题: Cterm true color feature has warnings.
解决方案: Add type casts.
相关文件: src/screen.c, src/syntax.c, src/term.c

Patch 7.4.1775
问题: The rgb.txt file is not installed.
解决方案: Install the file. (Christian Brabandt)
相关文件: src/Makefile

Patch 7.4.1776
问题: Using wrong buffer length.
解决方案: use the right name. (Kazunobu Kuriyama)
相关文件: src/term.c

Patch 7.4.1777
问题: Newly added features can escape the sandbox.
解决方案: Add checks for restricted and secure. (Yasuhiro Matsumoto)
相关文件: src/eval.c

Patch 7.4.1778
问题: When using the term truecolor feature, the t_8f and t_8b termcap
            options are not set by default.
解决方案: Move the values to before BT_EXTRA_KEYS. (Christian Brabandt)
相关文件: src/term.c

Patch 7.4.1779
问题: Using negative index in strcharpart(). (Yegappan Lakshmanan)
解决方案: Assume single byte when using a negative index.
相关文件: src/eval.c

Patch 7.4.1780
问题: Warnings reported by cppcheck.
解决方案: Fix the warnings. (Dominique Pelle)
相关文件: src/ex_cmds2.c, src/json.c, src/misc1.c, src/ops.c,
            src/regexp_nfa.c

Patch 7.4.1781
问题: synIDattr() does not respect 'guicolors'.
解决方案: Change the condition for the mode. (Christian Brabandt)
相关文件: src/eval.c

Patch 7.4.1782
问题: strcharpart() does not work properly with some multi-byte
            characters.
解决方案: Use mb_cptr2len() instead of mb_char2len(). (Hirohito Higashi)
相关文件: src/eval.c, src/testdir/test_expr_utf8.vim

Patch 7.4.1783
问题: The old regexp engine doesn't handle character classes correctly.
            (Manuel Ortega)
解决方案: Use regmbc() instead of regc(). Add a test.
相关文件: src/regexp.c, src/testdir/test_regexp_utf8.vim

Patch 7.4.1784
问题: The termtruecolor feature is enabled differently from many other
            features.
解决方案: Enable the termtruecolor feature for the big build, not through

configure.
相关文件: src/configure.in, src/config.h.in, src/auto/configure,
src/feature.h

Patch 7.4.1785 (after 7.4.1783)
问题: Regexp test fails on windows.
解决方案: set 'isprint' to the right value for testing.
相关文件: src/testdir/test_regexp_utf8.vim

Patch 7.4.1786
问题: Compiled-in colors do not match rgb.txt.
解决方案: Use the rgb.txt colors. (Kazunobu Kuriyama)
相关文件: src/term.c

Patch 7.4.1787
问题: When a job ends the close callback is invoked before other
callbacks. On Windows the close callback is not called.
解决方案: First invoke out/err callbacks before the close callback.
Make the close callback work on Windows.
相关文件: src/channel.c, src/proto/channel.pro,
src/testdir/test_channel.vim, src/testdir/test_channel_pipe.py

Patch 7.4.1788
问题: NSIS script is missing packages.
解决方案: Add the missing directories. (Ken Takata)
相关文件: nsis/gvim.nsi

Patch 7.4.1789
问题: Cannot use ch_read() in the close callback.
解决方案: Do not discard the channel if there is readahead. Do not discard
readahead if there is a close callback.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro,
src/testdir/test_channel.vim

Patch 7.4.1790
问题: Leading white space in a job command matters. (Andrew Stewart)
解决方案: Skip leading white space.
相关文件: src/os_unix.c

Patch 7.4.1791
问题: Channel could be garbage collected too early.
解决方案: Don't free a channel or remove it from a job when it is still
useful.
相关文件: src/channel.c

Patch 7.4.1792
问题: Color name decoding is implemented several times.
解决方案: Move it to term.c. (Christian Brabandt)
相关文件: src/gui_mac.c, src/gui_photon.c, src/gui_w32.c,
src/proto/term.pro, src/term.c

Patch 7.4.1793
问题: Some character classes may differ between systems. On OS/X the
regexp test fails.

解决方案: Make this less dependent on the system. (idea by Kazunobu Kuriyama)
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 7.4.1794 (after 7.4.1792)
问题: Can't build on MS-Windows.
解决方案: Add missing declaration.
相关文件: src/gui_w32.c

Patch 7.4.1795
问题: Compiler warning for redefining RGB. (John Marriott)
解决方案: Rename it to TORGB.
相关文件: src/term.c

Patch 7.4.1796 (after 7.4.1795)
问题: Colors are wrong on MS-Windows. (Christian Robinson)
解决方案: Use existing RGB macro if it exists. (Ken Takata)
相关文件: src/term.c

Patch 7.4.1797
问题: Warning from Windows 64 bit compiler.
解决方案: Change int to size_t. (Mike Williams)
相关文件: src/term.c

Patch 7.4.1798
问题: Still compiler warning for unused return value. (Charles Campbell)
解决方案: Assign to ignoredp.
相关文件: src/term.c

Patch 7.4.1799
问题: 'guicolors' is a confusing option name.
解决方案: Use 'termguicolors' instead. (Hirohito Higashi, Ken Takata)
相关文件: runtime/doc/options.txt, runtime/doc/term.txt,
           runtime/doc/various.txt, runtime/syntax/dircolors.vim, src/eval.c,
           src/feature.h, src/globals.h, src/hardcopy.c, src/option.c,
           src/option.h, src/proto/term.pro, src/screen.c, src/structs.h,
           src/syntax.c, src/term.c, src/version.c, src/vim.h

Patch 7.4.1800 (after 7.4.1799)
问题: Unnecessary #ifdef.
解决方案: Just use USE_24BIT. (Ken Takata)
相关文件: src/syntax.c

Patch 7.4.1801
问题: Make uninstall leaves file behind.
解决方案: Delete rgb.txt. (Kazunobu Kuriyama)
相关文件: src/Makefile

Patch 7.4.1802
问题: Quickfix doesn't handle long lines well, they are split.
解决方案: Drop characters after a limit. (Anton Lindqvist)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim,
           src/testdir/samples/quickfix.txt

Patch 7.4.1803

问题:　　GTK3 doesn't handle menu separators properly.
解决方案:　Use gtk_separator_menu_item_new(). (Kazunobu Kuriyama)
相关文件:　src/gui_gtk.c

Patch 7.4.1804
问题:　　Can't use Vim as MANPAGER.
解决方案:　Add manpager.vim. (Enno Nagel, closes #491)
相关文件:　runtime/doc/filetype.txt, runtime/plugin/manpager.vim

Patch 7.4.1805
问题:　　Running tests in shadow dir fails.
解决方案:　Link the samples directory
相关文件:　src/Makefile

Patch 7.4.1806
问题:　　'termguicolors' option missing from the options window.
解决方案:　Add the entry.
相关文件:　runtime/optwin.vim

Patch 7.4.1807
问题:　　Test_out_close_cb sometimes fails.
解决方案:　Always write DETACH to out, not err.
相关文件:　src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1808 (after 7.4.1806)
问题:　　Using wrong feature name to check for 'termguicolors'.
解决方案:　Use the right feature name. (Ken Takata)
相关文件:　runtime/optwin.vim

Patch 7.4.1809 (after 7.4.1808)
问题:　　Using wrong short option name for 'termguicolors'.
解决方案:　Use the option name.
相关文件:　runtime/optwin.vim

Patch 7.4.1810
问题:　　Sending DETACH after a channel was closed isn't useful.
解决方案:　Only add DETACH for a netbeans channel.
相关文件:　src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1811
问题:　　Netbeans channel gets garbage collected.
解决方案:　Set reference in nb_channel.
相关文件:　src/eval.c, src/netbeans.c, src/proto/netbeans.pro

Patch 7.4.1812
问题:　　Failure on startup with Athena and Motif.
解决方案:　Check for INVALCOLOR. (Kazunobu Kuriyama)
相关文件:　src/syntax.c, src/vim.h

Patch 7.4.1813
问题:　　Memory access error when running test_quickfix.
解决方案:　Allocate one more byte. (Yegappan Lakshmanan)
相关文件:　src/quickfix.c

Patch 7.4.1814
问题: A channel may be garbage collected while it's still being used by a job. (James McCoy)
解决方案: Mark the channel as used if the job is still used. Do the same for channels that are still used.
相关文件: src/eval.c, src/channel.c, src/proto/channel.pro

Patch 7.4.1815
问题: Compiler warnings for unused variables. (Ajit Thakkar)
解决方案: Add a dummy initialization. (Yasuhiro Matsumoto)
相关文件: src/quickfix.c

Patch 7.4.1816
问题: Looping over a null list throws an error.
解决方案: Skip over the for loop.
相关文件: src/eval.c, src/testdir/test_expr.vim

Patch 7.4.1817
问题: The screen is not updated if a callback is invoked when closing a channel.
解决方案: Invoke redraw_after_callback().
相关文件: src/channel.c

Patch 7.4.1818
问题: Help completion adds @en to all matches except the first one.
解决方案: Remove "break", go over all items.
相关文件: src/ex_getln.c

Patch 7.4.1819
问题: Compiler warnings when sprintf() is a macro.
解决方案: Don't interrupt sprintf() with an #ifdef. (Michael Jarvis, closes #788)
相关文件: src/fileio.c, src/tag.c, src/term.c

Patch 7.4.1820
问题: Removing language from help tags too often.
解决方案: Only remove @en when not needed. (Hirohito Higashi)
相关文件: src/ex_getln.c, src/testdir/test_help_tagjump.vim

Patch 7.4.1821 (after 7.4.1820)
问题: Test fails on MS-Windows.
解决方案: Sort the completion results.
相关文件: src/testdir/test_help_tagjump.vim

Patch 7.4.1822
问题: Redirecting stdout of a channel to "null" doesn't work. (Nicola)
解决方案: Correct the file descriptor number.
相关文件: src/os_unix.c

Patch 7.4.1823
问题: Warning from 64 bit compiler.
解决方案: Add type cast. (Mike Williams)
相关文件: src/quickfix.c

Patch 7.4.1824
问题: When a job is no longer referenced and does not have an exit callback the process may hang around in defunct state. (Nicola)
解决方案: Call job_status() if the job is running and won't get freed because it might still be useful.
相关文件: src/channel.c

Patch 7.4.1825
问题: When job writes to buffer nothing is written. (Nicola)
解决方案: Do not discard a channel before writing is done.
相关文件: src/channel.c

Patch 7.4.1826
问题: Callbacks are invoked when it's not safe. (Andrew Stewart)
解决方案: When a channel is to be closed don't invoke callbacks right away, wait for a safe moment.
相关文件: src/structs.h, src/channel.c

Patch 7.4.1827
问题: No error when invoking a callback when it's not safe.
解决方案: Add an error message. Avoid the error when freeing a channel.
相关文件: src/structs.h, src/channel.c

Patch 7.4.1828
问题: May try to access buffer that's already freed.
解决方案: When freeing a buffer remove it from any channel.
相关文件: src/buffer.c, src/channel.c, src/proto/channel.pro

Patch 7.4.1829 (after 7.4.1828)
问题: No message on channel log when buffer was freed.
解决方案: Log a message.
相关文件: src/channel.c

Patch 7.4.1830
问题: non-antialiased misnamed.
解决方案: Use NONANTIALIASED and NONANTIALIASED_QUALITY. (Kim Brouer, closes #793)
相关文件: src/os_mswin.c, runtime/doc/options.txt

Patch 7.4.1831
问题: When timer_stop() is called with a string there is no proper error message.
解决方案: Require getting a number. (Bjorn Linse)
相关文件: src/eval.c

Patch 7.4.1832
问题: Memory leak in debug commands.
解决方案: Free memory before overwriting the pointer. (hint by Justin Keyes)
相关文件: src/ex_cmds2.c

Patch 7.4.1833
问题: Cannot use an Ex command for 'keywordprg'.
解决方案: Accept an Ex command. (Nelo-Thara Wallus)
相关文件: src/normal.c, runtime/doc/options.txt

Patch 7.4.1834
问题: Possible crash when conceal is active.
解决方案: Check for the screen to be valid when redrawing a line.
相关文件: src/screen.c

Patch 7.4.1835
问题: When splitting and closing a window the status height changes.
解决方案: Compute the frame height correctly. (Hirohito Higashi)
相关文件: src/window.c, src/testdir/test_alot.vim,
src/testdir/test_window_cmd.vim

Patch 7.4.1836
问题: When using a partial on a dictionary it always gets bound to that
dictionary.
解决方案: Make a difference between binding a function to a dictionary
explicitly or automatically.
相关文件: src/structs.h, src/eval.c, src/testdir/test_partial.vim,
runtime/doc/eval.txt

Patch 7.4.1837
问题: The BufUnload event is triggered twice, when :bunload is used with
bufhidden set to unload or delete .
解决方案: Do not trigger the event when ml_mfp is NULL. (Hirohito Higashi)
相关文件: src/buffer.c, src/testdir/test_autocmd.vim

Patch 7.4.1838
问题: Functions specifically for testing do not sort together.
解决方案: Rename garbagecollect_for_testing() to test_garbagecollect_now().
Add test_null_list(), test_null_dict(), etc.
相关文件: src/eval.c, src/testdir/test_expr.vim,
src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 7.4.1839
问题: Cannot get the items stored in a partial.
解决方案: Support using get() on a partial.
相关文件: src/eval.c, src/testdir/test_partial.vim, runtime/doc/eval.txt

Patch 7.4.1840
问题: When using packages an "after" directory cannot be used.
解决方案: Add the "after" directory of the package to 'runtimepath' if it
exists.
相关文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 7.4.1841
问题: The code to reallocate the buffer used for quickfix is repeated.
解决方案: Move the code to a function. (Yegappan Lakshmanan, closes #831)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1842 (after 7.4.1839)
问题: get() works for Partial but not for Funcref.
解决方案: Accept Funcref. Also return the function itself. (Nikolai Pavlov)
相关文件: src/eval.c, src/testdir/test_partial.vim, runtime/doc/eval.txt

Patch 7.4.1843
问题: Tests involving Python are flaky.
解决方案: Set the pt_auto field. Add tests. (Nikolai Pavlov)
相关文件: runtime/doc/if_pyth.txt, src/if_py_both.h, src/testdir/test86.in, src/testdir/test86.ok, src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.1844
问题: Using old function name in comment. More functions should start with test_.
解决方案: Rename function in comment. (Hirohito Higashi) Rename disable_char_avail_for_testing() to test_disable_char_avail(). And alloc_fail() to test_alloc_fail().
相关文件: src/eval.c, src/getchar.c, src/testdir/runtest.vim, src/testdir/test_cursor_func.vim, src/testdir/test_quickfix.vim, runtime/doc/eval.txt

Patch 7.4.1845
问题: Mentioning NetBeans when reading from channel. (Ramel Eshed)
解决方案: Make the text more generic.
相关文件: src/channel.c

Patch 7.4.1846
问题: Ubsan detects a multiplication overflow.
解决方案: Don't use orig_mouse_time when it's zero. (Dominique Pelle)
相关文件: src/term.c

Patch 7.4.1847
问题: Getting an item from a NULL dict crashes. Setting a register to a NULL list crashes. (Nikolai Pavlov, issue #768) Comparing a NULL dict with a NULL dict fails.
解决方案: Properly check for NULL.
相关文件: src/eval.c, src/testdir/test_expr.vim

Patch 7.4.1848
问题: Can't build with Strawberry Perl 5.24.
解决方案: Define S_SvREFCNT_dec() if needed. (Damien, Ken Takata)
相关文件: src/if_perl.xs

Patch 7.4.1849
问题: Still trying to read from channel that is going to be closed. (Ramel Eshed)
解决方案: Check if ch_to_be_closed is set.
相关文件: src/channel.c

Patch 7.4.1850
问题: GUI freezes when using a job. (Shougo Matsu)
解决方案: Unregister the channel when there is an input error.
相关文件: src/channel.c

Patch 7.4.1851
问题: test_syn_attr fails when using the GUI. (Dominique Pelle)
解决方案: Escape the font name properly.
相关文件: src/testdir/test_syn_attr.vim

Patch 7.4.1852
问题: Unix: Cannot run all tests with the GUI.
解决方案: Add the "testgui" target.
相关文件: src/Makefile, src/testdir/Makefile

Patch 7.4.1853
问题: Crash when job and channel are in the same dict while using
partials. (Luc Hermitte)
解决方案: Do not decrement the channel reference count too early.
相关文件: src/channel.c

Patch 7.4.1854
问题: When setting 'termguicolors' the Ignore highlighting doesn't work.
(Charles Campbell)
解决方案: Handle the color names "fg" and "bg" when the GUI isn't running
and no colors are specified, fall back to black and white.
相关文件: src/syntax.c

Patch 7.4.1855
问题: Valgrind reports memory leak for job that is not freed.
解决方案: Free all jobs on exit. Add test for failing job.
相关文件: src/channel.c, src/misc2.c, src/proto/channel.pro,
src/testdir/test_partial.vim

Patch 7.4.1856 (after 7.4.1855)
问题: failing job test fails on MS-Windows.
解决方案: Expect "fail" status instead of "dead".
相关文件: src/testdir/test_partial.vim

Patch 7.4.1857
问题: When a channel appends to a buffer that is 'nomodifiable' there is
an error but appending is done anyway.
解决方案: Add the 'modifiable' option. Refuse to write to a 'nomodifiable'
when the value is 1.
相关文件: src/structs.h, src/channel.c, src/testdir/test_channel.vim,
runtime/doc/channel.txt

Patch 7.4.1858
问题: When a channel writes to a buffer it doesn't find a buffer by the
short name but re-uses it anyway.
解决方案: Find buffer also by the short name.
相关文件: src/channel.c, src/buffer.c, src/vim.h

Patch 7.4.1859
问题: Cannot use a function reference for "exit_cb".
解决方案: Use get_callback(). (Yegappan Lakshmanan)
相关文件: src/channel.c, src/structs.h

Patch 7.4.1860
问题: Using a partial for timer_start() may cause a crash.
解决方案: Set the copyID in timer objects. (Ozaki Kiichi)
相关文件: src/testdir/test_timers.vim, src/eval.c, src/ex_cmds2.c,
src/proto/ex_cmds2.pro

Patch 7.4.1861
问题: Compiler warnings with 64 bit compiler.
解决方案: Change int to size_t. (Mike Williams)
相关文件: src/ex_cmds2.c

Patch 7.4.1862
问题: string() with repeated argument does not give a result usable by eval().
解决方案: Refactor echo_string and tv2string(), moving the common part to echo_string_core(). (Ken Takata)
相关文件: src/eval.c, src/testdir/test_viml.vim, src/testdir/test86.ok, src/testdir/test87.ok

Patch 7.4.1863
问题: Compiler warnings on Win64.
解决方案: Adjust types, add type casts. (Ken Takata)
相关文件: src/if_mzsch.c, src/if_perl.xs, src/if_ruby.c, src/version.c

Patch 7.4.1864
问题: Python: encoding error with Python 2.
解决方案: Use "getcwdu" instead of "getcwd". (Ken Takata)
相关文件: src/if_py_both.h

Patch 7.4.1865
问题: Memory leaks in test49. (Dominique Pelle)
解决方案: Use NULL instead of an empty string.
相关文件: src/eval.c

Patch 7.4.1866
问题: Invalid memory access when exiting with EXITFREE defined. (Dominique Pelle)
解决方案: Set "really_exiting" and skip error messages.
相关文件: src/misc2.c, src/eval.c

Patch 7.4.1867
问题: Memory leak in test_matchstrpos.
解决方案: Free the string before overwriting. (Yegappan Lakshmanan)
相关文件: src/eval.c

Patch 7.4.1868
问题: Setting really_exiting causes memory leaks to be reported.
解决方案: Add the in_free_all_mem flag.
相关文件: src/globals.h, src/misc2.c, src/eval.c

Patch 7.4.1869
问题: Can't build with old version of Perl.
解决方案: Define PERLIO_FUNCS_DECL. (Tom G. Christensen)
相关文件: src/if_perl.xs

Patch 7.4.1870 (after 7.4.1863)
问题: One more Win64 compiler warning.
解决方案: Change declared argument type. (Ken Takata)
相关文件: src/if_mzsch.c

Patch 7.4.1871
问题: Appending to the quickfix list while the quickfix window is open is very slow.
解决方案: Do not delete all the lines, only append the new ones. Avoid using a window while updating the list. (closes #841)
相关文件: src/quickfix.c

Patch 7.4.1872
问题: Still build problem with old version of Perl.
解决方案: Also define SvREFCNT_inc_void_NN if needed. (Tom G. Christensen)
相关文件: src/if_perl.xs

Patch 7.4.1873
问题: When a callback adds a timer the GUI doesn't use it until later. (Ramel Eshed)
解决方案: Return early if a callback adds a timer.
相关文件: src/ex_cmds2.c, src/gui_gtk_x11.c, src/gui_w32.c, src/gui_x11.c, src/globals.h

Patch 7.4.1874
问题: Unused variable in Win32 code.
解决方案: Remove it. (Mike Williams)
相关文件: src/gui_w32.c

Patch 7.4.1875
问题: Comparing functions and partials doesn't work well.
解决方案: Add tests. (Nikolai Pavlov) Compare the dict and arguments in the partial. (closes #813)
相关文件: src/eval.c, src/testdir/test_partial.vim

Patch 7.4.1876
问题: Typing "k" at the hit-enter prompt has no effect.
解决方案: Don't assume recursive use of the prompt if a character was typed. (Hirohito Higashi)
相关文件: src/message.c

Patch 7.4.1877
问题: No test for invoking "close_cb" when writing to a buffer.
解决方案: Add using close_cb to a test case.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1878
问题: Whether a job has exited isn't detected until a character is typed. After calling exit_cb the cursor is in the wrong place.
解决方案: Don't wait forever for a character to be typed when there is a pending job. Update the screen if needed after calling exit_cb.
相关文件: src/os_unix.c, src/channel.c, src/proto/channel.pro

Patch 7.4.1879 (after 7.4.1877)
问题: Channel test is flaky.
解决方案: Wait for close_cb to be invoked.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1880
问题: MS-Windows console build defaults to not having +channel.
解决方案: Include the channel feature if building with huge features.
相关文件: src/Make_mvc.mak

Patch 7.4.1881
问题: Appending to a long quickfix list is slow.
解决方案: Add qf_last.
相关文件: src/quickfix.c

Patch 7.4.1882
问题: Check for line break at end of line wrong. (Dominique Pelle)
解决方案: Correct the logic.
相关文件: src/quickfix.c

Patch 7.4.1883
问题: Cppcheck found 2 incorrect printf formats.
解决方案: Use %ld and %lx. (Dominique Pelle)
相关文件: src/VisVim/Commands.cpp, src/gui_mac.c

Patch 7.4.1884
问题: Updating marks in a quickfix list is very slow when the list is long.
解决方案: Only update marks if the buffer has a quickfix entry.
相关文件: src/structs.h, src/quickfix.c

Patch 7.4.1885
问题: MinGW console build defaults to not having +channel.
解决方案: Include the channel feature if building with huge features. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.1886
问题: When waiting for a character is interrupted by receiving channel data and the first character of a mapping was typed, the mapping times out. (Ramel Eshed)
解决方案: When dealing with channel data don't return from mch_inchar().
相关文件: src/getchar.c, src/proto/getchar.pro, src/os_unix.c

Patch 7.4.1887
问题: When receiving channel data 'updatetime' is not respected.
解决方案: Recompute the waiting time after being interrupted.
相关文件: src/os_unix.c

Patch 7.4.1888
问题: Wrong computation of remaining wait time in RealWaitForChar()
解决方案: Remember the original waiting time.
相关文件: src/os_unix.c

Patch 7.4.1889
问题: When umask is set to 0177 Vim can't create temp files. (Lcd)
解决方案: Also correct umask when using mkdtemp().
相关文件: src/fileio.c

Patch 7.4.1890
问题: GUI: When channel data is received the cursor blinking is interrupted. (Ramel Eshed)
解决方案: Don't update the cursor when it is blinking.
相关文件: src/screen.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro, src/gui_mac.c, src/proto/gui_mac.pro, src/gui_photon.c, src/proto/gui_photon.pro, src/gui_w32.c, src/proto/gui_w32.pro, src/gui_x11.c, src/proto/gui_x11.pro

Patch 7.4.1891
问题: Channel reading very long lines is slow.
解决方案: Collapse multiple buffers until a NL is found.
相关文件: src/channel.c, src/netbeans.c, src/proto/channel.pro, src/structs.h

Patch 7.4.1892
问题: balloon eval only gets the window number, not the ID.
解决方案: Add v:beval_winid.
相关文件: src/eval.c, src/gui_beval.c, src/vim.h

Patch 7.4.1893
问题: Cannot easily get the window ID for a buffer.
解决方案: Add bufwinid().
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 7.4.1894
问题: Cannot get the window ID for a mouse click.
解决方案: Add v:mouse_winid.
相关文件: src/eval.c, src/vim.h, runtime/doc/eval.txt

Patch 7.4.1895
问题: Cannot use a window ID where a window number is expected.
解决方案: Add LOWEST_WIN_ID, so that the window ID can be used where a number is expected.
相关文件: src/window.c, src/eval.c, src/vim.h, runtime/doc/eval.txt, src/testdir/test_window_id.vim

Patch 7.4.1896
问题: Invoking mark_adjust() when adding a new line below the last line is pointless.
解决方案: Skip calling mark_adjust() when appending below the last line.
相关文件: src/misc1.c, src/ops.c

Patch 7.4.1897
问题: Various typos, long lines and style mistakes.
解决方案: Fix the typos, wrap lines, improve style.
相关文件: src/buffer.c, src/ex_docmd.c, src/getchar.c, src/option.c, src/main.aap, src/testdir/README.txt, src/testdir/test_reltime.vim, src/testdir/test_tagjump.vim, src/INSTALL, src/config.aap.in, src/if_mzsch.c

Patch 7.4.1898
问题: User commands don't support modifiers.
解决方案: Add the <mods> item. (Yegappan Lakshmanan, closes #829)

相关文件: runtime/doc/map.txt, src/ex_docmd.c, src/testdir/Make_all.mak,
           src/testdir/test_usercommands.vim

Patch 7.4.1899
问题: GTK 3: cursor blinking doesn't work well.
解决方案: Instead of gui_gtk_window_clear() use gui_mch_clear_block().
           (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 7.4.1900
问题: Using CTRL-] in the help on "{address}." doesn't work.
解决方案: Recognize an item in {}. (Hirohito Higashi, closes #814)
相关文件: src/ex_cmds.c, src/testdir/test_help_tagjump.vim

Patch 7.4.1901
问题: Win32: the "Disabled" menu items would appear enabled.
解决方案: Use submenu_id if there is a parent. (Shane Harper, closes #834)
相关文件: src/gui_w32.c

Patch 7.4.1902
问题: No test for collapsing buffers for a channel. Some text is lost.
解决方案: Add a simple test. Set rq_buflen correctly.
相关文件: src/channel.c, src/testdir/test_channel.vim,
           src/testdir/test_channel_pipe.py

Patch 7.4.1903
问题: When writing viminfo merging current history with history in
           viminfo may drop recent history entries.
解决方案: Add new format for viminfo lines, use it for history entries. Use
           a timestamp for ordering the entries. Add test_settime().
           Add the viminfo version. Does not do merging on timestamp yet.
相关文件: src/eval.c, src/ex_getln.c, src/ex_cmds.c, src/structs.h,
           src/globals.h, src/proto/ex_cmds.pro, src/proto/ex_getln.pro,
           src/testdir/test_viminfo.vim

Patch 7.4.1904 (after 7.4.1903)
问题: Build fails.
解决方案: Add missing changes.
相关文件: src/vim.h

Patch 7.4.1905 (after 7.4.1903)
问题: Some compilers can't handle a double semicolon.
解决方案: Remove one semicolon.
相关文件: src/ex_cmds.c

Patch 7.4.1906
问题: Collapsing channel buffers and searching for NL does not work
           properly. (Xavier de Gaye, Ramel Eshed)
解决方案: Do not assume the buffer contains a NUL or not. Change NUL bytes
           to NL to avoid the string is truncated.
相关文件: src/channel.c, src/netbeans.c, src/proto/channel.pro

Patch 7.4.1907
问题: Warnings from 64 bit compiler.

解决方案: Change type to size_t. (Mike Williams)
相关文件: src/ex_cmds.c

Patch 7.4.1908
问题: Netbeans uses uninitialized pointer and freed memory.
解决方案: Set "buffer" at the right place (hint by Ken Takata)
相关文件: src/netbeans.c

Patch 7.4.1909
问题: Doubled semicolons.
解决方案: Reduce to one. (Dominique Pelle)
相关文件: src/dosinst.c, src/fold.c, src/gui_gtk_x11.c, src/gui_w32.c,
           src/main.c, src/misc2.c

Patch 7.4.1910
问题: Tests using external command to delete directory.
解决方案: Use delete().
相关文件: src/testdir/test17.in, src/testdir/test73.in,
           src/testdir/test_getcwd.in

Patch 7.4.1911
问题: Recent history lines may be lost when exiting Vim.
解决方案: Merge history using the timestamp.
相关文件: src/ex_getln.c, src/ex_cmds.c, src/vim.h, src/proto/ex_getln.pro,
           src/testdir/test_viminfo.vim

Patch 7.4.1912
问题: No test for using setqflist() on an older quickfix list.
解决方案: Add a couple of tests.
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1913
问题: When ":doautocmd" is used modelines are used even when no
           autocommands were executed. (Daniel Hahler)
解决方案: Skip processing modelines. (closes #854)
相关文件: src/fileio.c, src/ex_cmds.c, src/ex_docmd.c, src/proto/fileio.pro

Patch 7.4.1914
问题: Executing autocommands while using the signal stack has a high
           chance of crashing Vim.
解决方案: Don't invoke autocommands when on the signal stack.
相关文件: src/os_unix.c

Patch 7.4.1915
问题: The effect of the PopupMenu autocommand isn't directly visible.
解决方案: Call gui_update_menus() before displaying the popup menu. (Shane
           Harper, closes #855)
相关文件: src/menu.c

Patch 7.4.1916 (after 7.4.1906)
问题: No proper test for what 7.4.1906 fixes.
解决方案: Add a test for reading many lines.
相关文件: src/testdir/test_channel.vim

Patch 7.4.1917
问题:　　History lines read from viminfo in different encoding than when
　　writing are not converted.
解决方案:　Convert the history lines.
相关文件:　src/ex_cmds.c, src/testdir/test_viminfo.vim

Patch 7.4.1918
问题:　　Not enough testing for parsing viminfo lines.
解决方案:　Add test with viminfo lines in bad syntax. Fix memory leak.
相关文件:　src/ex_cmds.c, src/ex_getln.c, src/testdir/test_viminfo.vim

Patch 7.4.1919
问题:　　Register contents is not merged when writing viminfo.
解决方案:　Use timestamps for register contents.
相关文件:　src/ops.c, src/ex_getln.c, src/ex_cmds.c, src/proto/ex_cmds.pro,
　　src/proto/ex_getln.pro, src/proto/ops.pro, src/vim.h

Patch 7.4.1920 (after 7.4.1919)
问题:　　Missing test changes.
解决方案:　Update viminfo test.
相关文件:　src/testdir/test_viminfo.vim

Patch 7.4.1921 (after 7.4.1919)
问题:　　vim_time() not included when needed.
解决方案:　Adjust #ifdef.
相关文件:　src/ex_cmds.c

Patch 7.4.1922
问题:　　Ruby 2.4.0 unifies Fixnum and Bignum into Integer.
解决方案:　Use rb_cInteger. (Weiyong Mao)
相关文件:　src/if_ruby.c

Patch 7.4.1923
问题:　　Command line editing is not tested much.
解决方案:　Add tests for expanding the file name and 'wildmenu'.
相关文件:　src/testdir/test_cmdline.vim, src/testdir/Make_all.mak

Patch 7.4.1924
问题:　　Missing "void" for functions without argument.
解决方案:　Add "void". (Hirohito Higashi)
相关文件:　src/channel.c, src/edit.c, src/ex_cmds2.c, src/ops.c, src/screen.c

Patch 7.4.1925
问题:　　Viminfo does not merge file marks properly.
解决方案:　Use a timestamp. Add the :clearjumps command.
相关文件:　src/mark.c, src/ex_cmds.c, src/ex_docmd.c, src/proto/mark.pro,
　　src/structs.h, src/vim.h, src/ex_cmds.h,
　　src/testdir/test_viminfo.vim

Patch 7.4.1926
问题:　　Possible crash with many history items.
解决方案:　Avoid the index going past the last item.
相关文件:　src/ex_getln.c

Patch 7.4.1927
问题: Compiler warning for signed/unsigned.
解决方案: Add type cast.
相关文件: src/if_mzsch.c

Patch 7.4.1928
问题: Overwriting pointer argument.
解决方案: Assign to what it points to. (Dominique Pelle)
相关文件: src/fileio.c

Patch 7.4.1929
问题: Inconsistent indenting and weird name.
解决方案: Fix indent, make name all upper case. (Ken Takata)
相关文件: src/if_ruby.c

Patch 7.4.1930
问题: Can't build without +spell but with +quickfix. (Charles)
解决方案: Add better #ifdef around ml_append_buf(). (closes #864)
相关文件: src/memline.c

Patch 7.4.1931
问题: Using both old and new style file mark lines from viminfo.
解决方案: Skip the old style lines if the viminfo file was written with a
            Vim version that supports the new style.
相关文件: src/ex_cmds.c

Patch 7.4.1932
问题: When writing viminfo the jumplist is not merged with the one in
            the viminfo file.
解决方案: Merge based on timestamp.
相关文件: src/mark.c, src/testdir/test_viminfo.vim

Patch 7.4.1933
问题: Compiler warning about uninitialized variable. (Yegappan)
解决方案: Give it a dummy value.
相关文件: src/ex_getln.c

Patch 7.4.1934
问题: New style tests not executed with MinGW compiler.
解决方案: Add new style test support. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_ming.mak

Patch 7.4.1935
问题: When using the GUI search/replace a second match right after the
            replacement is skipped.
解决方案: Add the SEARCH_START flag. (Mleddy)
相关文件: src/gui.c

Patch 7.4.1936
问题: Off-by-one error in bounds check. (Coverity)
解决方案: Check register number properly.
相关文件: src/ops.c

Patch 7.4.1937

问题: No test for directory stack in quickfix.
解决方案: Add a test. (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1938
问题: When writing viminfo numbered marks were duplicated.
解决方案: Check for duplicates between current numbered marks and the ones
            read from viminfo.
相关文件: src/mark.c

Patch 7.4.1939
问题: Memory access error when reading viminfo. (Dominique Pelle)
解决方案: Correct index in jumplist when at the end.
相关文件: src/mark.c, src/testdir/test_viminfo.vim

Patch 7.4.1940
问题: "gd" hangs in some situations. (Eric Biggers)
解决方案: Remove the SEARCH_START flag when looping. Add a test.
相关文件: src/normal.c, src/testdir/test_goto.vim

Patch 7.4.1941
问题: Not all quickfix tests are also done with the location lists.
解决方案: Test more quickfix code. Use user commands instead of "exe".
            (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1942
问题: Background is not drawn properly when 'termguicolors' is set.
解决方案: Check cterm_normal_bg_color. (Jacob Niehus, closes #805)
相关文件: src/screen.c

Patch 7.4.1943
问题: Coverity warns for unreachable code.
解决方案: Remove the code that won't do anything.
相关文件: src/mark.c

Patch 7.4.1944
问题: Win32: Cannot compile with XPM feature using VC2015
解决方案: Add XPM libraries compiled with VC2015, and enable to build
            gvim.exe which supports XPM using VC2015. (Ken Takata)
相关文件: src/Make_mvc.mak, src/xpm/x64/lib-vc14/libXpm.lib,
            src/xpm/x86/lib-vc14/libXpm.lib

Patch 7.4.1945
问题: The Man plugin doesn't work that well.
解决方案: Use "g:ft_man_open_mode" to be able open man pages in vert split
            or separate tab. Set nomodifiable for buffer with man content. Add
            a test. (Andrey Starodubtsev, closes #873)
相关文件: runtime/ftplugin/man.vim, src/testdir/test_man.vim,
            src/testdir/Make_all.mak

Patch 7.4.1946 (after 7.4.1944)
问题: File list does not include new XPM libraries.
解决方案: Add the file list entries.

相关文件: Filelist

Patch 7.4.1947
问题: Viminfo continuation line with wrong length isn't skipped. (Marius Gedminas)
解决方案: Skip a line when encountering an error, but not two lines.
相关文件: src/ex_cmds.c

Patch 7.4.1948
问题: Using Ctrl-A with double-byte encoding may result in garbled text.
解决方案: Skip to the start of a character. (Hirohito Higashi)
相关文件: src/ops.c

Patch 7.4.1949
问题: Minor problems with the quickfix code.
解决方案: Fix the problems. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1950
问题: Quickfix long lines test not executed for buffer.
解决方案: Call the function to test long lines. (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1951
问题: Ruby test is old style.
解决方案: Convert to a new style test. (Ken Takata)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_ruby.in, src/testdir/test_ruby.ok, src/testdir/test_ruby.vim

Patch 7.4.1952
问题: Cscope interface does not support finding assignments.
解决方案: Add the "a" command. (ppettina, closes #882)
相关文件: runtime/doc/if_cscop.txt, src/if_cscope.c

Patch 7.4.1953
问题: Not all parts of the quickfix code are tested.
解决方案: Add more tests. (Yegappan Lakshmanan)
相关文件: src/testdir/samples/quickfix.txt, src/testdir/test_quickfix.vim

Patch 7.4.1954 (after 7.4.1948)
问题: No test for what 7.4.1948 fixes.
解决方案: Add a test. (Hirohito Higashi, closes #880)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_increment_dbcs.vim

Patch 7.4.1955
问题: Using 32-bit Perl with 64-bit time_t causes memory corruption. (Christian Brabandt)
解决方案: Use time_T instead of time_t for global variables. (Ken Takata)
相关文件: src/ex_cmds.c, src/globals.h, src/misc2.c, src/proto/ex_cmds.pro, src/proto/misc2.pro, src/structs.h, src/vim.h

Patch 7.4.1956

问题:　　When using CTRL-W f and pressing "q" at the ATTENTION dialog the newly opened window is not closed.
解决方案:　Close the window and go back to the original one. (Norio Takagi, Hirohito Higashi)
相关文件:　src/window.c, src/testdir/test_window_cmd.vim

Patch 7.4.1957
问题:　　Perl interface has obsolete workaround.
解决方案:　Remove the workaround added by 7.3.623. (Ken Takata)
相关文件:　src/if_perl.xs

Patch 7.4.1958
问题:　　Perl interface preprocessor statements not nicely indented.
解决方案:　Improve the indenting. (Ken Takata)
相关文件:　src/if_perl.xs

Patch 7.4.1959
问题:　　Crash when running test_channel.vim on Windows.
解决方案:　Check for NULL pointer result from FormatMessage(). (Christian Brabandt)
相关文件:　src/channel.c

Patch 7.4.1960
问题:　　Unicode standard 9 was released.
解决方案:　Update the character property tables. (Christian Brabandt)
相关文件:　src/mbyte.c

Patch 7.4.1961
问题:　　When 'insertmode' is reset while doing completion the popup menu remains even though Vim is in Normal mode.
解决方案:　Ignore stop_insert_mode when the popup menu is visible. Don't set stop_insert_mode when 'insertmode' was already off. (Christian Brabandt)
相关文件:　src/edit.c, src/option.c, src/Makefile, src/testdir/test_alot.vim, src/testdir/test_popup.vim

Patch 7.4.1962
问题:　　Two test files for increment/decrement.
解决方案:　Move the old style test into the new style test. (Hirohito Higashi, closes #881)
相关文件:　src/Makefile, src/testdir/Make_all.mak, src/testdir/main.aap, src/testdir/test35.in, src/testdir/test35.ok, src/testdir/test_increment.vim

Patch 7.4.1963
问题:　　Running Win32 Vim in mintty does not work.
解决方案:　Detect mintty and give a helpful error message. (Ken Takata)
相关文件:　src/Make_cyg_ming.mak, src/Make_mvc.mak, src/iscygpty.c, src/iscygpty.h, src/main.c, Filelist

Patch 7.4.1964
问题:　　The quickfix init function is too big.
解决方案:　Factor out parsing 'errorformat' to a separate function. (Yegappan Lakshmanan)

相关文件: src/quickfix.c

Patch 7.4.1965
问题: When using a job in raw mode to append to a buffer garbage
characters are added.
解决方案: Do not replace the trailing NUL with a NL. (Ozaki Kiichi)
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.1966
问题: Coverity reports a resource leak.
解决方案: Close "fd" also when bailing out.
相关文件: src/quickfix.c

Patch 7.4.1967
问题: Falling back from NFA to old regexp engine does not work properly.
(fritzophrenic)
解决方案: Do not restore nfa_match. (Christian Brabandt, closes #867)
相关文件: src/regexp_nfa.c, src/testdir/test64.in, src/testdir/test64.ok

Patch 7.4.1968
问题: Invalid memory access with "\<C-">.
解决方案: Do not recognize this as a special character. (Dominique Pelle)
相关文件: src/misc2.c, src/testdir/test_expr.vim

Patch 7.4.1969
问题: When the netbeans channel is closed consuming the buffer may cause
a crash.
解决方案: Check for nb_channel not to be NULL. (Xavier de Gaye)
相关文件: src/netbeans.c

Patch 7.4.1970
问题: Using ":insert" in an empty buffer sets the jump mark. (Ingo
Karkat)
解决方案: Don't adjust marks when replacing the empty line in an empty
buffer. (closes #892)
相关文件: src/ex_cmds.c, src/testdir/test_jumps.vim,
src/testdir/test_alot.vim

Patch 7.4.1971
问题: It is not easy to see unrecognized error lines below the current
error position.
解决方案: Add ":clist +count".
相关文件: src/quickfix.c, runtime/doc/quickfix.txt

Patch 7.4.1972
问题: On Solaris select() does not work as expected when there is
typeahead.
解决方案: Add ICANON when sleeping. (Ozaki Kiichi)
相关文件: src/os_unix.c

Patch 7.4.1973
问题: On MS-Windows the package directory may be added at the end
because of forward/backward slash differences. (Matthew
Desjardins)

解决方案: Ignore slash differences.
相关文件: src/ex_cmds2.c

Patch 7.4.1974
问题: GUI has a problem with some termcodes.
解决方案: Handle negative numbers. (Kazunobu Kuriyama)
相关文件: src/gui.c

Patch 7.4.1975
问题: On MS-Windows large files (> 2Gbyte) cause problems.
解决方案: Use "off_T" instead of "off_t". Use "stat_T" instead of "struct stat". Use 64 bit system functions if available. (Ken Takata)
相关文件: src/Makefile, src/buffer.c, src/diff.c, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c, src/fileio.c, src/gui.c, src/gui_at_fs.c, src/if_cscope.c, src/main.c, src/memfile.c, src/memline.c, src/misc1.c, src/misc2.c, src/netbeans.c, src/os_mswin.c, src/os_win32.c, src/proto/fileio.pro, src/proto/memline.pro, src/proto/os_mswin.pro, src/pty.c, src/quickfix.c, src/spell.c, src/structs.h, src/tag.c, src/testdir/Make_all.mak, src/testdir/test_largefile.vim, src/testdir/test_stat.vim, src/undo.c, src/vim.h

Patch 7.4.1976
问题: Number variables are not 64 bits while they could be.
解决方案: Add the num64 feature. (Ken Takata, Yasuhiro Matsumoto)
相关文件: runtime/doc/eval.txt, runtime/doc/various.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/charset.c, src/eval.c, src/ex_cmds.c, src/ex_getln.c, src/feature.h, src/fileio.c, src/fold.c, src/json.c, src/message.c, src/misc1.c, src/misc2.c, src/ops.c, src/option.c, src/proto/charset.pro, src/proto/eval.pro, src/quickfix.c, src/structs.h, src/testdir/test_viml.vim, src/version.c

Patch 7.4.1977
问题: With 64 bit changes don't need three calls to sprintf().
解决方案: Simplify the code, use vim_snprintf(). (Ken Takata)
相关文件: src/fileio.c

Patch 7.4.1978 (after 7.4.1975)
问题: Large file test does not delete its output.
解决方案: Delete the output. Check size properly when possible. (Ken Takata)
相关文件: src/testdir/test_largefile.vim

Patch 7.4.1979 (after 7.4.1976)
问题: Getting value of binary option is wrong. (Kent Sibilev)
解决方案: Fix type cast. Add a test.
相关文件: src/option.c, src/testdir/test_expr.vim

Patch 7.4.1980
问题: 'errorformat' is parsed for every call to ":caddexpr". Can't add to two location lists asynchronously.
解决方案: Keep the previously parsed data when appropriate. (mostly by Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.1981
问题: No testing for Farsi code.
解决方案: Add a minimal test. Clean up Farsi code.
相关文件: src/farsi.c, src/Makefile, src/charset.c, src/normal.c,
            src/proto/main.pro, src/testdir/Make_all.mak,
            src/testdir/test_farsi.vim

Patch 7.4.1982
问题: Viminfo file contains duplicate change marks.
解决方案: Drop duplicate marks.
相关文件: src/mark.c

Patch 7.4.1983
问题: farsi.c and arabic.c are included in a strange way.
解决方案: Build them like other files.
相关文件: src/main.c, src/farsi.c, src/arabic.c, src/proto.h,
            src/proto/main.pro, src/proto/farsi.pro, src/proto/arabic.pro,
            src/Makefile, src/Make_bc5.mak, src/Make_cyg_ming.mak,
            src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
            src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
            Filelist

Patch 7.4.1984
问题: Not all quickfix features are tested.
解决方案: Add a few more tests. (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim

Patch 7.4.1985 (after 7.4.1983)
问题: Missing changes in VMS build file.
解决方案: Use the right file name.
相关文件: src/Make_vms.mms

Patch 7.4.1986
问题: Compiler warns for loss of data.
解决方案: Use size_t instead of int. (Christian Brabandt)
相关文件: src/ex_cmds2.c

Patch 7.4.1987
问题: When copying unrecognized lines for viminfo, end up with useless
            continuation lines.
解决方案: Skip continuation lines.
相关文件: src/ex_cmds.c

Patch 7.4.1988
问题: When updating viminfo with file marks there is no time order.
解决方案: Remember the time when a buffer was last used, store marks for
            the most recently used buffers.
相关文件: src/buffer.c, src/structs.h, src/mark.c, src/main.c,
            src/ex_cmds.c, src/proto/mark.pro, src/testdir/test_viminfo.vim

Patch 7.4.1989
问题: filter() and map() only accept a string argument.
解决方案: Implement using a Funcref argument (Yasuhiro Matsumoto, Ken

Takata)
相关文件: runtime/doc/eval.txt, src/Makefile, src/eval.c,
src/testdir/test_alot.vim, src/testdir/test_filter_map.vim,
src/testdir/test_partial.vim

Patch 7.4.1990 (after 7.4.1952)
问题: Cscope items are not sorted.
解决方案: Put the new "a" command first. (Ken Takata)
相关文件: src/if_cscope.c

Patch 7.4.1991
问题: glob() does not add a symbolic link when there are no wildcards.
解决方案: Remove the call to mch_getperm().
相关文件: src/misc1.c

Patch 7.4.1992
问题: Values for true and false can be confusing.
解决方案: Update the documentation. Add a test. Make v:true evaluate to
TRUE for a non-zero-arg.
相关文件: runtime/doc/eval.txt, src/eval.c, src/Makefile,
src/testdir/test_true_false.vim, src/testdir/test_alot.vim

Patch 7.4.1993
问题: Not all TRUE and FALSE arguments are tested.
解决方案: Add a few more tests.
相关文件: src/testdir/test_true_false.vim

Patch 7.4.1994 (after 7.4.1993)
问题: True-false test fails.
解决方案: Filter the dict to only keep the value that matters.
相关文件: src/testdir/test_true_false.vim

Patch 7.4.1995
问题: GUI: cursor drawn in wrong place if a timer callback causes a
screen update. (David Samvelyan)
解决方案: Also redraw the cursor when it's blinking and on.
相关文件: src/gui_gtk_x11.c, src/gui_mac.c, src/gui_photon.c, src/gui_w32.c,
src/gui_x11.c, src/screen.c, src/proto/gui_gtk_x11.pro,
src/proto/gui_mac.pro, src/proto/gui_photon.pro,
src/proto/gui_w32.pro, src/proto/gui_x11.pro

Patch 7.4.1996
问题: Capturing the output of a command takes a few commands.
解决方案: Add evalcmd().
相关文件: src/eval.c, runtime/doc/eval.txt, src/testdir/test_alot.vim,
src/Makefile, src/testdir/test_evalcmd.vim

Patch 7.4.1997
问题: Cannot easily scroll the quickfix window.
解决方案: Add ":cbottom".
相关文件: src/ex_cmds.h, src/quickfix.c, src/proto/quickfix.pro,
src/ex_docmd.c, src/testdir/test_quickfix.vim,
runtime/doc/quickfix.txt

Patch 7.4.1998
问题: When writing buffer lines to a job there is no NL to NUL conversion.
解决方案: Make it work symmetrical with writing lines from a job into a buffer.
相关文件: src/channel.c, src/proto/channel.pro, src/netbeans.c

Patch 7.4.1999
问题: evalcmd() doesn't work recursively.
解决方案: Use redir_evalcmd instead of redir_vname.
相关文件: src/message.c, src/eval.c, src/globals.h, src/proto/eval.pro, src/testdir/test_evalcmd.vim

Patch 7.4.2000 (after 7.4.1999)
问题: Evalcmd test fails.
解决方案: Add missing piece.
相关文件: src/ex_docmd.c

Patch 7.4.2001 (after 7.4.2000)
问题: Tiny build fails. (Tony Mechelynck)
解决方案: Add #ifdef.
相关文件: src/ex_docmd.c

Patch 7.4.2002
问题: Crash when passing number to filter() or map().
解决方案: Convert to a string. (Ozaki Kiichi)
相关文件: src/eval.c, src/testdir/test_filter_map.vim

Patch 7.4.2003
问题: Still cursor flickering when a callback updates the screen. (David Samvelyan)
解决方案: Put the cursor in the right position after updating the screen.
相关文件: src/screen.c

Patch 7.4.2004
问题: GUI: cursor displayed in the wrong position.
解决方案: Correct screen_cur_col and screen_cur_row.
相关文件: src/screen.c

Patch 7.4.2005
问题: After using evalcmd() message output is in the wrong position. (Christian Brabandt)
解决方案: Reset msg_col.
相关文件: src/eval.c

Patch 7.4.2006
问题: Crash when using tabnext in BufUnload autocmd. (Norio Takagi)
解决方案: First check that the current buffer is the right one. (Hirohito Higashi)
相关文件: src/buffer.c, src/testdir/test_autocmd.vim

Patch 7.4.2007
问题: Running the tests leaves a viminfo file behind.
解决方案: Make the viminfo option empty.

相关文件: src/testdir/runtest.vim

Patch 7.4.2008
问题: evalcmd() has a confusing name.
解决方案: Rename to execute(). Make silent optional. Support a list of commands.
相关文件: src/eval.c, src/ex_docmd.c, src/message.c, src/globals.h, src/proto/eval.pro, src/Makefile, src/testdir/test_evalcmd.vim, src/testdir/test_execute_func.vim, src/testdir/test_alot.vim, runtime/doc/eval.txt

Patch 7.4.2009 (after 7.4.2008)
问题: Messages test fails.
解决方案: Don't set redir_execute before returning. Add missing version number.
相关文件: src/eval.c

Patch 7.4.2010
问题: There is a :cbottom command but no :lbottom command.
解决方案: Add :lbottom. (Yegappan Lakshmanan)
相关文件: runtime/doc/index.txt, runtime/doc/quickfix.txt, src/ex_cmds.h, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.2011
问题: It is not easy to get a list of command arguments.
解决方案: Add getcompletion(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/eval.c, src/ex_docmd.c, src/proto/ex_docmd.pro, src/testdir/test_cmdline.vim

Patch 7.4.2012 (after 7.4.2011)
问题: Test for getcompletion() does not pass on all systems.
解决方案: Only test what is supported.
相关文件: src/testdir/test_cmdline.vim

Patch 7.4.2013
问题: Using "noinsert" in 'completeopt' breaks redo.
解决方案: Set compl_curr_match. (Shougo Matsu, closes #874)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 7.4.2014
问题: Using "noinsert" in 'completeopt' does not insert match.
解决方案: Set compl_enter_selects. (Shougo Matsu, closes #875)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 7.4.2015
问题: When a file gets a name when writing it 'acd' is not effective. (Dan Church)
解决方案: Invoke DO_AUTOCHDIR after writing the file. (Allen Haim, closes #777, closes #803) Add test_autochdir() to enable 'acd' before "starting" is reset.
相关文件: src/ex_cmds.c, src/buffer.c, src/eval.c, src/globals.h, src/Makefile, src/testdir/test_autochdir.vim, src/testdir/Make_all.mak

Patch 7.4.2016
问题:　　Warning from MinGW about _WIN32_WINNT redefined. (John Marriott)
解决方案:　First undefine it. (Ken Takata)
相关文件:　src/Make_cyg_ming.mak

Patch 7.4.2017
问题:　　When there are many errors adding them to the quickfix list takes
　　a long time.
解决方案:　Add BLN_NOOPT. Don't call buf_valid() in buf_copy_options().
　　Remember the last file name used. When going through the buffer
　　list start from the end of the list. Only call buf_valid() when
　　autocommands were executed.
相关文件:　src/buffer.c, src/option.c, src/quickfix.c, src/vim.h

Patch 7.4.2018
问题:　　buf_valid() can be slow when there are many buffers.
解决方案:　Add bufref_valid(), only go through the buffer list when a buffer
　　was freed.
相关文件:　src/structs.h, src/buffer.c, src/quickfix.c, src/proto/buffer.pro

Patch 7.4.2019
问题:　　When ignoring case utf_fold() may consume a lot of time.
解决方案:　Optimize for ASCII.
相关文件:　src/mbyte.c

Patch 7.4.2020
问题:　　Can't build without +autocmd feature.
解决方案:　Adjust #ifdefs.
相关文件:　src/buffer.c

Patch 7.4.2021
问题:　　Still too many buf_valid() calls.
解决方案:　Make au_new_curbuf a bufref. Use bufref_valid() in more places.
相关文件:　src/ex_cmds.c, src/buffer.c, src/globals.h

Patch 7.4.2022
问题:　　Warnings from 64 bit compiler.
解决方案:　Add type casts. (Mike Williams)
相关文件:　src/eval.c

Patch 7.4.2023
问题:　　buflist_findname_stat() may find a dummy buffer.
解决方案:　Set the BF_DUMMY flag after loading a dummy buffer. Start
　　finding buffers from the end of the list.
相关文件:　src/quickfix.c, src/buffer.c

Patch 7.4.2024
问题:　　More buf_valid() calls can be optimized.
解决方案:　Use bufref_valid() instead.
相关文件:　src/buffer.c, src/ex_cmds.c, src/structs.h, src/channel.c,
　　src/diff.c, src/eval.c, src/ex_cmds2.c, src/ex_docmd.c,
　　src/ex_getln.c, src/fileio.c, src/main.c, src/misc2.c,
　　src/netbeans.c, src/quickfix.c, src/spell.c, src/term.c,
　　src/if_py_both.h, src/window.c, src/proto/buffer.pro,

src/proto/window.pro

Patch 7.4.2025
问题: The cursor blinking stops or is irregular when receiving date over a channel and writing it in a buffer, and when updating the status line. (Ramel Eshed)
解决方案: Make it a bit better by flushing GUI output. Don't redraw the cursor after updating the screen if the blink state is off.
相关文件: src/gui_gtk_x11.c, src/screen.c

Patch 7.4.2026
问题: Reference counting for callbacks isn't right.
解决方案: Add free_callback(). (Ken Takata) Fix reference count.
相关文件: src/channel.c, src/eval.c, src/ex_cmds2.c, src/proto/eval.pro

Patch 7.4.2027
问题: Can't build with +eval but without +menu.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/eval.c

Patch 7.4.2028
问题: cppcheck warns for using index before limits check.
解决方案: Swap the expressions. (Dominique Pelle)
相关文件: src/mbyte.c

Patch 7.4.2029
问题: printf() does not work with 64 bit numbers.
解决方案: use the "L" length modifier. (Ken Takata)
相关文件: src/message.c, src/testdir/test_expr.vim

Patch 7.4.2030
问题: ARCH must be set properly when using MinGW.
解决方案: Detect the default value of ARCH from the current compiler. (Ken Takata)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.2031
问题: The list_lbr_utf8 test fails if ~/.vim/syntax/c.vim sets 'textwidth' to a non-zero value. (Oyvind A. Holm)
解决方案: Add a setup.vim file that sets 'runtimepath' and $HOME to a safe value. (partly by Christian Brabandt, closes #912)
相关文件: src/testdir/setup.vim, src/testdir/amiga.vim, src/testdir/dos.vim, src/testdir/unix.vim, src/testdir/vms.vim, src/testdir/runtest.vim

Patch 7.4.2032 (after 7.4.2030)
问题: Build fails with 64 bit MinGW. (Axel Bender)
解决方案: Handle dash vs. underscore. (Ken Takata, Hirohito Higashi)
相关文件: src/Make_cyg_ming.mak

Patch 7.4.2033
问题: 'cscopequickfix' option does not accept new value "a".
解决方案: Adjust list of command characters. (Ken Takata)
相关文件: src/option.h, src/Makefile, src/testdir/test_cscope.vim, src/testdir/Make_all.mak

Patch 7.4.2034 (after 7.4.2032)
问题:　　Build fails with some version of MinGW. (illusorypan)
解决方案:　Recognize mingw32. (Ken Takata, closes #921)
相关文件:　src/Make_cyg_ming.mak

Patch 7.4.2035
问题:　　On Solaris with ZFS the ACL may get removed.
解决方案:　Always restore the ACL for Solaris ZFS. (Danek Duvall)
相关文件:　src/fileio.c

Patch 7.4.2036
问题:　　Looking up a buffer by number is slow if there are many.
解决方案:　Use a hashtab.
相关文件:　src/structs.h, src/buffer.c

Patch 7.4.2037 (after 7.4.2036)
问题:　　Small build fails.
解决方案:　Adjust #ifdefs.
相关文件:　src/hashtab.c

Patch 7.4.2038 (after 7.4.2036)
问题:　　Small build still fails.
解决方案:　Adjust more #ifdefs.
相关文件:　src/globals.h, src/buffer.c

Patch 7.4.2039
问题:　　The Netbeans integration is not tested.
解决方案:　Add a first Netbeans test.
相关文件:　src/testdir/test_netbeans.vim, src/testdir/test_netbeans.py,
　　src/testdir/Make_all.mak, src/Makefile,
　　src/testdir/test_channel.vim, src/testdir/shared.vim

Patch 7.4.2040
问题:　　New files missing from distribution.
解决方案:　Add new test scripts.
相关文件:　Filelist

Patch 7.4.2041
问题:　　Netbeans file authentication not tested.
解决方案:　Add a test.
相关文件:　src/testdir/test_netbeans.vim

Patch 7.4.2042
问题:　　GTK: display updating is not done properly and can be slow.
解决方案:　Use gdk_display_flush() instead of gdk_display_sync(). Don't call
　　gdk_window_process_updates(). (Kazunobu Kuriyama)
相关文件:　src/gui_gtk_x11.c

Patch 7.4.2043
问题:　　setbuvfar() causes a screen redraw.
解决方案:　Only use aucmd_prepbuf() for options.
相关文件:　src/eval.c

Patch 7.4.2044
问题:　　filter() and map() either require a string or defining a function.
解决方案:　Support lambda, a short way to define a function that evaluates an
　　expression. (Yasuhiro Matsumoto, Ken Takata)
相关文件:　runtime/doc/eval.txt, src/eval.c, src/testdir/test_alot.vim,
　　src/Makefile, src/testdir/test_channel.vim,
　　src/testdir/test_lambda.vim

Patch 7.4.2045
问题:　　Memory leak when using a function callback.
解决方案:　Don't save the function name when it's in the partial.
相关文件:　src/channel.c

Patch 7.4.2046
问题:　　The qf_init_ext() function is too big.
解决方案:　Refactor it. (Yegappan Lakshmanan)
相关文件:　src/quickfix.c

Patch 7.4.2047
问题:　　Compiler warning for initializing a struct.
解决方案:　Initialize in another way. (Anton Lindqvist)
相关文件:　src/quickfix.c

Patch 7.4.2048
问题:　　There is still code and help for unsupported systems.
解决方案:　Remove the code and text. (Hirohito Higashi)
相关文件:　runtime/doc/eval.txt, runtime/lang/menu_sk_sk.vim,
　　runtime/menu.vim, runtime/optwin.vim, src/Make_bc5.mak,
　　src/ex_docmd.c, src/feature.h, src/fileio.c, src/globals.h,
　　src/main.c, src/memfile.c, src/memline.c, src/misc1.c,
　　src/misc2.c, src/option.c, src/option.h, src/os_unix.c,
　　src/os_unix.h, src/proto.h, src/term.c, src/undo.c, src/version.c,
　　src/vim.h, src/xxd/xxd.c

Patch 7.4.2049
问题:　　There is no way to get a list of the error lists.
解决方案:　Add ":chistory" and ":lhistory".
相关文件:　src/ex_cmds.h, src/quickfix.c, src/ex_docmd.c, src/message.c,
　　src/proto/quickfix.pro, src/testdir/test_quickfix.vim

Patch 7.4.2050
问题:　　When using ":vimgrep" may end up with duplicate buffers.
解决方案:　When adding an error list entry pass the buffer number if possible.
相关文件:　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.2051
问题:　　No proper testing of trunc_string().
解决方案:　Add a unittest for message.c.
相关文件:　src/Makefile, src/message.c, src/message_test.c, src/main.c,
　　src/proto/main.pro, src/structs.h

Patch 7.4.2052
问题:　　Coverage report is messed up by the unittests.
解决方案:　Add a separate test target for script tests. Use that when

collecting coverage information.
相关文件: src/Makefile

Patch 7.4.2053
问题: Can't run scripttests in the top directory.
解决方案: Add targets to the top Makefile.
相关文件: Makefile

Patch 7.4.2054 (after 7.4.2048)
问题: Wrong part of #ifdef removed.
解决方案: Use the right part. (Hirohito Higashi)
相关文件: src/os_unix.c

Patch 7.4.2055
问题: eval.c is too big
解决方案: Move Dictionary functions to dict.c
相关文件: src/eval.c, src/dict.c, src/vim.h, src/globals.h,
src/proto/eval.pro, src/proto/dict.pro, src/Makefile, Filelist

Patch 7.4.2056 (after 7.4.2055)
问题: Build fails.
解决方案: Add missing changes.
相关文件: src/proto.h

Patch 7.4.2057
问题: eval.c is too big.
解决方案: Move List functions to list.c
相关文件: src/eval.c, src/dict.c, src/list.c, src/proto.h, src/Makefile,
src/globals.h, src/proto/eval.pro, src/proto/list.pro, Filelist

Patch 7.4.2058
问题: eval.c is too big.
解决方案: Move user functions to userfunc.c
相关文件: src/userfunc.c, src/eval.c, src/vim.h, src/globals.h,
src/structs.h, src/proto.h, src/Makefile, src/proto/eval.pro,
src/proto/userfunc.pro, Filelist

Patch 7.4.2059
问题: Non-Unix builds fail.
解决方案: Update Makefiles for new files.
相关文件: src/Make_bc5.mak, src/Make_cyg_ming.mak, src/Make_dice.mak,
src/Make_ivc.mak, src/Make_manx.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_sas.mak

Patch 7.4.2060 (after 7.4.2059)
问题: Wrong file name.
解决方案: Fix typo.
相关文件: src/Make_mvc.mak

Patch 7.4.2061
问题: qf_init_ext() is too big.
解决方案: Move code to qf_parse_line() (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.2062
问题: Using dummy variable to compute struct member offset.
解决方案: Use offsetof().
相关文件: src/globals.h, src/macros.h, src/vim.h, src/spell.c

Patch 7.4.2063
问题: eval.c is still too big.
解决方案: Split off internal functions to evalfunc.c.
相关文件: src/eval.c, src/evalfunc.c, src/list.c, src/proto.h,
            src/globals.h, src/vim.h, src/proto/eval.pro,
            src/proto/evalfunc.pro, src/proto/list.pro, src/Makefile, Filelist,
            src/Make_bc5.mak, src/Make_cyg_ming.mak, src/Make_dice.mak,
            src/Make_ivc.mak, src/Make_manx.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_sas.mak

Patch 7.4.2064
问题: Coverity warns for possible buffer overflow.
解决方案: Use vim_strcat() instead of strcat().
相关文件: src/quickfix.c

Patch 7.4.2065
问题: Compiler warns for uninitialized variable. (John Marriott)
解决方案: Set lnum to the right value.
相关文件: src/evalfunc.c

Patch 7.4.2066
问题: getcompletion() not well tested.
解决方案: Add more testing.
相关文件: src/testdir/test_cmdline.vim

Patch 7.4.2067
问题: Compiler warning for char/char_u conversion. (Tony Mechelynck)
            Inefficient code.
解决方案: Use more lines to fill with spaces. (Nikolai Pavlov) Add type cast.
相关文件: src/quickfix.c

Patch 7.4.2068
问题: Not all arguments of trunc_string() are tested. Memory access
            error when running the message tests.
解决方案: Add another test case. (Yegappan Lakshmanan) Make it easy to run
            unittests with valgrind. Fix the access error.
相关文件: src/message.c, src/message_test.c, src/Makefile

Patch 7.4.2069
问题: spell.c is too big.
解决方案: Split it in spell file handling and spell checking.
相关文件: src/spell.c, src/spellfile.c, src/spell.h, src/Makefile,
            src/proto/spell.pro, src/proto/spellfile.pro, src/proto.h
            Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
            src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
            src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak

Patch 7.4.2070 (after 7.4.2069)
问题: Missing change to include file.

解决方案: Include the spell header file.
相关文件: src/vim.h

Patch 7.4.2071
问题: The return value of type() is difficult to use.
解决方案: Define v:t_ constants. (Ken Takata)
相关文件: runtime/doc/eval.txt, src/eval.c, src/evalfunc.c,
src/testdir/test_channel.vim, src/testdir/test_viml.vim, src/vim.h

Patch 7.4.2072
问题: substitute() does not support a Funcref argument.
解决方案: Support a Funcref like it supports a string starting with "\=".
相关文件: src/evalfunc.c, src/regexp.c, src/eval.c, src/proto/eval.pro,
src/proto/regexp.pro, src/testdir/test_expr.vim

Patch 7.4.2073
问题: rgb.txt is read for every color name.
解决方案: Load rgb.txt once. (Christian Brabandt) Add a test.
相关文件: runtime/rgb.txt, src/term.c, src/testdir/test_syn_attr.vim

Patch 7.4.2074
问题: One more place using a dummy variable.
解决方案: Use offsetof(). (Ken Takata)
相关文件: src/userfunc.c

Patch 7.4.2075
问题: No autocommand event to initialize a window or tab page.
解决方案: Add WinNew and TabNew events. (partly by Felipe Morales)
相关文件: src/fileio.c, src/window.c, src/vim.h,
src/testdir/test_autocmd.vim, runtime/doc/autocmd.txt

Patch 7.4.2076
问题: Syntax error when dict has '>' key.
解决方案: Check for endchar. (Ken Takata)
相关文件: src/userfunc.c, src/testdir/test_lambda.vim

Patch 7.4.2077
问题: Cannot update 'tabline' when a tab was closed.
解决方案: Add the TabClosed autocmd event. (partly by Felipe Morales)
相关文件: src/fileio.c, src/window.c, src/vim.h,
src/testdir/test_autocmd.vim, runtime/doc/autocmd.txt

Patch 7.4.2078
问题: Running checks in po directory fails.
解决方案: Add colors used in syntax.c to the builtin color table.
相关文件: src/term.c

Patch 7.4.2079
问题: Netbeans test fails on non-Unix systems.
解决方案: Only do the permission check on Unix systems.
相关文件: src/testdir/test_netbeans.vim

Patch 7.4.2080
问题: When using PERROR() on some systems assert_fails() does not see

the error.
解决方案: Make PERROR() always report the error.
相关文件: src/vim.h, src/message.c, src/proto/message.pro

Patch 7.4.2081
问题: Line numbers in the error list are not always adjusted.
解决方案: Set b_has_qf_entry properly. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/structs.h, src/testdir/test_quickfix.vim

Patch 7.4.2082
问题: Not much test coverage for digraphs.
解决方案: Add a new style digraph test. (Christian Brabandt)
相关文件: src/Makefile, src/testdir/test_alot.vim,
src/testdir/test_digraph.vim

Patch 7.4.2083
问题: Coverity complains about not restoring a value.
解决方案: Restore the value, although it's not really needed. Change return
to jump to cleanup, might leak memory.
相关文件: src/userfunc.c

Patch 7.4.2084
问题: New digraph test makes testing hang.
解决方案: Don't set "nocp".
相关文件: src/testdir/test_digraph.vim

Patch 7.4.2085
问题: Digraph tests fails on some systems.
解决方案: Run it separately and set 'encoding' early.
相关文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim,
src/testdir/test_digraph.vim

Patch 7.4.2086
问题: Using the system default encoding makes tests unpredictable.
解决方案: Always use utf-8 or latin1 in the new style tests. Remove setting
encoding and scriptencoding where it is not needed.
相关文件: src/testdir/runtest.vim, src/testdir/test_channel.vim,
src/testdir/test_digraph.vim, src/testdir/test_expand_dllpath.vim,
src/testdir/test_expr_utf8.vim, src/testdir/test_json.vim,
src/testdir/test_matchadd_conceal_utf8.vim,
src/testdir/test_regexp_utf8.vim, src/testdir/test_visual.vim,
src/testdir/test_alot_utf8.vim,

Patch 7.4.2087
问题: Digraph code test coverage is still low.
解决方案: Add more tests. (Christian Brabandt)
相关文件: src/testdir/test_digraph.vim

Patch 7.4.2088 (after 7.4.2087)
问题: Keymap test fails with normal features.
解决方案: Bail out if the keymap feature is not supported.
相关文件: src/testdir/test_digraph.vim

Patch 7.4.2089

问题: Color handling of X11 GUIs is too complicated.
解决方案: Simplify the code. Use RGBA where appropriate. (Kazunobu
            Kuriyama)
相关文件: src/gui.h, src/gui_beval.c, src/gui_gtk_x11.c, src/netbeans.c

Patch 7.4.2090
问题: Using submatch() in a lambda passed to substitute() is verbose.
解决方案: Use a static list and pass it as an optional argument to the
            function. Fix memory leak.
相关文件: src/structs.h, src/list.c, src/userfunc.c, src/channel.c,
            src/eval.c, src/evalfunc.c, src/ex_cmds2.c, src/regexp.c,
            src/proto/list.pro, src/proto/userfunc.pro,
            src/testdir/test_expr.vim, runtime/doc/eval.txt

Patch 7.4.2091
问题: Coverity reports a resource leak when out of memory.
解决方案: Close the file before returning.
相关文件: src/term.c

Patch 7.4.2092
问题: GTK 3 build fails with older GTK version.
解决方案: Check the pango version. (Kazunobu Kuriyama)
相关文件: src/gui_beval.c

Patch 7.4.2093
问题: Netbeans test fails once in a while. Leaving log file behind.
解决方案: Add it to the list of flaky tests. Disable logfile.
相关文件: src/testdir/runtest.vim, src/testdir/test_channel.vim

Patch 7.4.2094
问题: The color allocation in X11 is overly complicated.
解决方案: Remove find_closest_color(), XAllocColor() already does this.
            (Kazunobu Kuriyama)
相关文件: src/gui_x11.c

Patch 7.4.2095
问题: Man test fails when run with the GUI.
解决方案: Adjust for different behavior of GUI. Add assert_inrange().
相关文件: src/eval.c, src/evalfunc.c, src/proto/eval.pro,
            src/testdir/test_assert.vim, src/testdir/test_man.vim,
            runtime/doc/eval.txt

Patch 7.4.2096
问题: Lambda functions show up with completion.
解决方案: Don't show lambda functions. (Ken Takata)
相关文件: src/userfunc.c, src/testdir/test_cmdline.vim

Patch 7.4.2097
问题: Warning from 64 bit compiler.
解决方案: use size_t instead of int. (Mike Williams)
相关文件: src/message.c

Patch 7.4.2098
问题: Text object tests are old style.

解决方案: Turn them into new style tests. (James McCoy, closes #941)
相关文件: src/testdir/Make_all.mak, src/testdir/test_textobjects.in,
           src/testdir/test_textobjects.ok, src/testdir/test_textobjects.vim,
           src/Makefile

Patch 7.4.2099
问题: When a keymap is active only "(lang)" is displayed. (Ilya
           Dogolazky)
解决方案: Show the keymap name. (Dmitri Vereshchagin, closes #933)
相关文件: src/buffer.c, src/proto/screen.pro, src/screen.c

Patch 7.4.2100
问题: "cgn" and "dgn" do not work correctly with a single character
           match and the replacement includes the searched pattern. (John
           Beckett)
解决方案: If the match is found in the wrong column try in the next column.
           Turn the test into new style. (Christian Brabandt)
相关文件: src/search.c, src/testdir/Make_all.mak, src/Makefile,
           src/testdir/test53.in, src/testdir/test53.ok,
           src/testdir/test_gn.vim

Patch 7.4.2101
问题: Looping over windows, buffers and tab pages is inconsistent.
解决方案: Use FOR_ALL_ macros everywhere. (Yegappan Lakshmanan)
相关文件: src/buffer.c, src/diff.c, src/edit.c, src/eval.c, src/evalfunc.c,
           src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/fileio.c,
           src/globals.h, src/gui.c, src/gui_mac.c, src/if_lua.c,
           src/if_mzsch.c, src/if_perl.xs, src/if_ruby.c, src/if_tcl.c,
           src/main.c, src/mark.c, src/memfile.c, src/memline.c, src/misc1.c,
           src/move.c, src/netbeans.c, src/normal.c, src/option.c,
           src/quickfix.c, src/screen.c, src/spell.c, src/term.c,
           src/window.c, src/workshop.c

Patch 7.4.2102 (after 7.4.2101)
问题: Tiny build with GUI fails.
解决方案: Revert one FOR_ALL_ change.
相关文件: src/gui.c

Patch 7.4.2103
问题: Can't have "augroup END" right after ":au!".
解决方案: Check for the bar character before the command argument.
相关文件: src/fileio.c, src/testdir/test_autocmd.vim,
           runtime/doc/autocmd.txt

Patch 7.4.2104
问题: Code duplication when unreferencing a function.
解决方案: De-duplicate.
相关文件: src/userfunc.c

Patch 7.4.2105
问题: Configure reports default features to be "normal" while it is
           "huge".
解决方案: Change the default text. Build with newer autoconf.
相关文件: src/configure.in, src/auto/configure

Patch 7.4.2106
问题:　　Clang warns about missing field in initializer.
解决方案:　Define COMMA and use it.　(Kazunobu Kuriyama)
相关文件:　src/ex_cmds.c, src/globals.h, src/vim.h

Patch 7.4.2107 (after 7.4.2106)
问题:　　Misplaced equal sign.
解决方案:　Remove it.
相关文件:　src/globals.h

Patch 7.4.2108
问题:　　Netbeans test is flaky.
解决方案:　Wait for the cursor to be positioned.
相关文件:　src/testdir/test_netbeans.vim

Patch 7.4.2109
问题:　　Setting 'display' to "lastline" is a drastic change, while
　　omitting it results in lots of "@" lines.
解决方案:　Add "truncate" to show "@@@" for a truncated line.
相关文件:　src/option.h, src/screen.c, runtime/doc/options.txt

Patch 7.4.2110
问题:　　When there is an CmdUndefined autocmd then the error for a missing
　　command is E464 instead of E492. (Manuel Ortega)
解决方案:　Don't let the pointer be NULL.
相关文件:　src/ex_docmd.c, src/testdir/test_usercommands.vim

Patch 7.4.2111
问题:　　Defaults are very conservative.
解决方案:　Move settings from vimrc_example.vim to defaults.vim.　Load
　　defaults.vim if no .vimrc was found.
相关文件:　src/main.c, src/version.c, src/os_amiga.h, src/os_dos.h,
　　src/os_mac.h, src/os_unix.h, src/feature.h, src/Makefile,
　　runtime/vimrc_example.vim, runtime/defaults.vim,
　　runtime/evim.vim, Filelist, runtime/doc/starting.txt

Patch 7.4.2112
问题:　　getcompletion(.., 'dir') returns a match with trailing "*" when
　　there are no matches. (Chdiza)
解决方案:　Return an empty list when there are no matches.　Add a trailing
　　slash to directories. (Yegappan Lakshmanan)　Add tests for no
　　matches. (closes #947)
相关文件:　src/evalfunc.c, src/testdir/test_cmdline.vim

Patch 7.4.2113
问题:　　Test for undo is flaky.
解决方案:　Turn it into a new style test.　Use test_settime() to avoid
　　flakyness.
相关文件:　src/Makefile, src/undo.c, src/testdir/test61.in,
　　src/testdir/test61.ok, src/testdir/test_undo.vim,
　　src/testdir/test_undolevels.vim, src/testdir/Make_all.mak,
　　src/testdir/test_alot.vim

Patch 7.4.2114
问题: Tiny build fails.
解决方案: Always include vim_time().
相关文件: src/ex_cmds.c

Patch 7.4.2115
问题: Loading defaults.vim with -C argument.
解决方案: Don't load the defaults script with -C argument. Test sourcing the defaults script. Set 'display' to "truncate".
相关文件: src/main.c, src/Makefile, runtime/defaults.vim, src/testdir/test_startup.vim, src/testdir/Make_all.mak

Patch 7.4.2116
问题: The default vimrc for Windows is very conservative.
解决方案: Use the defaults.vim in the Windows installer.
相关文件: src/dosinst.c

Patch 7.4.2117
问题: Deleting an augroup that still has autocmds does not give a warning. The next defined augroup takes its place.
解决方案: Give a warning and prevent the index being used for another group name.
相关文件: src/fileio.c, src/testdir/test_autocmd.vim

Patch 7.4.2118
问题: Mac: can't build with tiny features.
解决方案: Don't define FEAT_CLIPBOARD unconditionally. (Kazunobu Kuriyama)
相关文件: src/vim.h

Patch 7.4.2119
问题: Closures are not supported.
解决方案: Capture variables in lambdas from the outer scope. (Yasuhiro Matsumoto, Ken Takata)
相关文件: runtime/doc/eval.txt, src/eval.c, src/ex_cmds2.c, src/globals.h, src/proto/eval.pro, src/proto/userfunc.pro, src/testdir/test_lambda.vim, src/userfunc.c

Patch 7.4.2120
问题: User defined functions can't be a closure.
解决方案: Add the "closure" argument. Allow using :unlet on a bound variable. (Yasuhiro Matsumoto, Ken Takata)
相关文件: runtime/doc/eval.txt, src/testdir/test_lambda.vim, src/userfunc.c, src/eval.c src/proto/userfunc.pro

Patch 7.4.2121
问题: No easy way to check if lambda and closure are supported.
解决方案: Add the +lambda feature.
相关文件: src/evalfunc.c, src/version.c, src/testdir/test_lambda.vim

Patch 7.4.2122 (after 7.4.2118)
问题: Mac: don't get +clipboard in huge build.
解决方案: Move #define down below including feature.h
相关文件: src/vim.h

Patch 7.4.2123
问题: No new style test for diff mode.
解决方案: Add a test. Check that folds are in sync.
相关文件: src/Makefile, src/testdir/test_diffmode.vim,
            src/testdir/Make_all.mak, src/testdir/test47.in,
            src/testdir/test47.ok

Patch 7.4.2124
问题: diffmode test leaves files behind, breaking another test.
解决方案: Delete the files.
相关文件: src/testdir/test_diffmode.vim

Patch 7.4.2125
问题: Compiler warning for loss of data.
解决方案: Add a type cast. (Christian Brabandt)
相关文件: src/message.c

Patch 7.4.2126
问题: No tests for :diffget and :diffput
解决方案: Add tests.
相关文件: src/testdir/test_diffmode.vim

Patch 7.4.2127
问题: The short form of ":noswapfile" is ":noswap" instead of ":nos".
            (Kent Sibilev)
解决方案: Only require three characters. Add a test for the short forms.
相关文件: src/ex_docmd.c, src/testdir/test_usercommands.vim

Patch 7.4.2128
问题: Memory leak when saving for undo fails.
解决方案: Free allocated memory. (Hirohito Higashi)
相关文件: src/ex_cmds.c

Patch 7.4.2129
问题: Memory leak when using timer_start(). (Dominique Pelle)
解决方案: Don't copy the callback when using a partial.
相关文件: src/evalfunc.c

Patch 7.4.2130
问题: Pending timers cause false memory leak reports.
解决方案: Free all timers on exit.
相关文件: src/ex_cmds2.c, src/proto/ex_cmds2.pro, src/misc2.c

Patch 7.4.2131
问题: More memory leaks when using partial, e.g. for "exit-cb".
解决方案: Don't copy the callback when using a partial.
相关文件: src/channel.c

Patch 7.4.2132
问题: test_partial has memory leaks reported.
解决方案: Add a note about why this happens.
相关文件: src/testdir/test_partial.vim

Patch 7.4.2133 (after 7.4.2128)

问题: Can't build with tiny features.
解决方案: Add #ifdef.
相关文件: src/ex_cmds.c

Patch 7.4.2134
问题: No error for using function() badly.
解决方案: Check for passing wrong function name. (Ken Takata)
相关文件: src/eval.c, src/evalfunc.c, src/proto/userfunc.pro,
            src/testdir/test_expr.vim, src/userfunc.c, src/vim.h

Patch 7.4.2135
问题: Various tiny issues.
解决方案: Update comments, white space, etc.
相关文件: src/diff.c, src/digraph.c, src/testdir/test80.in,
            src/testdir/test_channel.vim, src/testdir/Makefile,
            runtime/menu.vim, src/INSTALLpc.txt, src/xpm/README.txt

Patch 7.4.2136
问题: Closure function fails.
解决方案: Don't reset uf_scoped when it points to another funccal.
相关文件: src/userfunc.c, src/testdir/test_lambda.vim

Patch 7.4.2137
问题: Using function() with a name will find another function when it is
            redefined.
解决方案: Add funcref(). Refer to lambda using a partial. Fix several
            reference counting issues.
相关文件: src/vim.h, src/structs.h, src/userfunc.c, src/eval.c,
            src/evalfunc.c, src/channel.c, src/proto/eval.pro,
            src/proto/userfunc.pro, src/if_mzsch.c, src/regexp.c, src/misc2.c,
            src/if_py_both.h, src/testdir/test_expr.vim, runtime/doc/eval.txt

Patch 7.4.2138
问题: Test 86 and 87 fail.
解决方案: Call func_ref() also for regular functions.
相关文件: src/if_py_both.h

Patch 7.4.2139
问题: :delfunction causes illegal memory access.
解决方案: Correct logic when deciding to free a function.
相关文件: src/userfunc.c, src/testdir/test_lambda.vim

Patch 7.4.2140
问题: Tiny build fails.
解决方案: Add dummy typedefs.
相关文件: src/structs.h

Patch 7.4.2141
问题: Coverity reports bogus NULL check.
解决方案: When checking for a variable in the funccal scope don't pass the
            varname.
相关文件: src/userfunc.c, src/proto/userfunc.pro, src/eval.c

Patch 7.4.2142

问题:　　Leaking memory when redefining a function.
解决方案:　Don't increment the function reference count when it's found by name. Don't remove the wrong function from the hashtab. More reference counting fixes.
相关文件:　src/structs.h, src/userfunc.c

Patch 7.4.2143
问题:　　A funccal is garbage collected while it can still be used.
解决方案:　Set copyID in all referenced functions. Do not list lambda functions with ":function".
相关文件:　src/userfunc.c, src/proto/userfunc.pro, src/eval.c, src/testdir/test_lambda.vim

Patch 7.4.2144
问题:　　On MS-Windows quickfix does not handle a line with 1023 bytes ending in CR-LF properly.
解决方案:　Don't consider CR a line break. (Ken Takata)
相关文件:　src/quickfix.c

Patch 7.4.2145
问题:　　Win32: Using CreateThread/ExitThread is not safe.
解决方案:　Use _beginthreadex and return from the thread. (Ken Takata)
相关文件:　src/os_win32.c

Patch 7.4.2146
问题:　　Not enough testing for popup menu. CTRL-E does not always work properly.
解决方案:　Add more tests. When using CTRL-E check if the popup menu is visible. (Christian Brabandt)
相关文件:　src/edit.c, src/testdir/test_popup.vim

Patch 7.4.2147 (after 7.4.2146)
问题:　　test_alot fails.
解决方案:　Close window.
相关文件:　src/testdir/test_popup.vim

Patch 7.4.2148
问题:　　Not much testing for cscope.
解决方案:　Add a test that uses the cscope program. (Christian Brabandt)
相关文件:　src/testdir/test_cscope.vim

Patch 7.4.2149
问题:　　If a test leaves a window open a following test may fail.
解决方案:　Always close extra windows after running a test.
相关文件:　src/testdir/runtest.vim, src/testdir/test_popup.vim

Patch 7.4.2150
问题:　　Warning with MinGW 64. (John Marriott)
解决方案:　Change return type. (Ken Takata)
相关文件:　src/os_win32.c

Patch 7.4.2151
问题:　　Quickfix test fails on MS-Windows.
解决方案:　Close the help window. (Christian Brabandt)

相关文件: src/testdir/test_quickfix.vim

Patch 7.4.2152
问题: No proper translation of messages with a count.
解决方案: Use ngettext(). (Sergey Alyoshin)
相关文件: src/evalfunc.c, src/fold.c, src/os_win32.c, src/screen.c, src/vim.h

Patch 7.4.2153
问题: GUI test isn't testing much.
解决方案: Turn into a new style test. Execute a shell command.
相关文件: src/testdir/test_gui.vim, src/testdir/test16.in,
src/testdir/test16.ok, src/testdir/Make_all.mak, src/Makefile,
src/testdir/Make_vms.mms

Patch 7.4.2154
问题: Test_communicate() fails sometimes.
解决方案: Add it to the flaky tests.
相关文件: src/testdir/runtest.vim

Patch 7.4.2155
问题: Quotes make GUI test fail on MS-Windows.
解决方案: Remove quotes, strip white space.
相关文件: src/testdir/test_gui.vim

Patch 7.4.2156
问题: Compiler warning.
解决方案: Add type cast. (Ken Takata, Mike Williams)
相关文件: src/os_win32.c

Patch 7.4.2157
问题: Test_job_start_fails() is expected to report memory leaks, making
it hard to see other leaks in test_partial.
解决方案: Move Test_job_start_fails() to a separate test file.
相关文件: src/testdir/test_partial.vim, src/testdir/test_job_fails.vim,
src/Makefile, src/testdir/Make_all.mak

Patch 7.4.2158
问题: Result of getcompletion('', 'cscope') depends on previous
completion. (Christian Brabandt)
解决方案: Call set_context_in_cscope_cmd().
相关文件: src/evalfunc.c, src/testdir/test_cmdline.vim

Patch 7.4.2159
问题: Insufficient testing for cscope.
解决方案: Add more tests. (Dominique Pelle)
相关文件: src/testdir/test_cscope.vim

Patch 7.4.2160
问题: setmatches() mixes up values. (Nikolai Pavlov)
解决方案: Save the string instead of reusing a shared buffer.
相关文件: src/dict.c, src/evalfunc.c, src/testdir/test_expr.vim,

Patch 7.4.2161 (after 7.4.2160)
问题: Expression test fails without conceal feature.

解决方案: Only check "conceal" with the conceal feature.
相关文件: src/testdir/test_expr.vim

Patch 7.4.2162
问题: Result of getcompletion('', 'sign') depends on previous
completion.
解决方案: Call set_context_in_sign_cmd(). (Dominique Pelle)
相关文件: src/evalfunc.c, src/testdir/test_cmdline.vim

Patch 7.4.2163
问题: match() and related functions tested with old style test.
解决方案: Convert to new style test. (Hirohito Higashi)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test63.in,
src/testdir/test63.ok, src/testdir/test_alot.vim,
src/testdir/test_match.vim, src/testdir/test_matchstrpos.vim

Patch 7.4.2164
问题: It is not possible to use plugins in an "after" directory to tune
the behavior of a package.
解决方案: First load plugins from non-after directories, then packages and
finally plugins in after directories.
Reset 'loadplugins' before executing --cmd arguments.
相关文件: src/main.c, src/vim.h, src/ex_cmds2.c, src/testdir/Makefile,
src/testdir/shared.vim, src/testdir/test_startup.vim,
src/testdir/setup.vim, runtime/doc/starting.txt

Patch 7.4.2165 (after 7.4.2164)
问题: Startup test fails on MS-Windows.
解决方案: Don't check output if RunVim() returns zero.
相关文件: src/testdir/test_startup.vim

Patch 7.4.2166 (after 7.4.2164)
问题: Small build can't run startup test.
解决方案: Skip the test.
相关文件: src/testdir/test_startup.vim

Patch 7.4.2167 (after 7.4.2164)
问题: Small build can't run tests.
解决方案: Don't try setting 'packpath'.
相关文件: src/testdir/setup.vim

Patch 7.4.2168
问题: Not running the startup test on MS-Windows.
解决方案: Write vimcmd.
相关文件: src/testdir/Make_ming.mak, src/testdir/Make_dos.mak

Patch 7.4.2169 (after 7.4.2168)
问题: Startup test gets stuck on MS-Windows.
解决方案: Use double quotes.
相关文件: src/testdir/shared.vim, src/testdir/test_startup.vim

Patch 7.4.2170
问题: Cannot get information about timers.
解决方案: Add timer_info().

相关文件: src/evalfunc.c, src/ex_cmds2.c, src/proto/ex_cmds2.pro,
           runtime/doc/eval.txt

Patch 7.4.2171 (after 7.4.2170)
问题: MS-Windows build fails.
解决方案: Add QueryPerformanceCounter().
相关文件: src/ex_cmds2.c

Patch 7.4.2172
问题: No test for "vim --help".
解决方案: Add a test.
相关文件: src/testdir/test_startup.vim, src/testdir/shared.vim

Patch 7.4.2173 (after 7.4.2172)
问题: Can't test help on MS-Windows.
解决方案: Skip the test.
相关文件: src/testdir/test_startup.vim

Patch 7.4.2174
问题: Adding duplicate flags to 'whichwrap' leaves commas behind.
解决方案: Also remove the commas. (Naruhiko Nishino)
相关文件: src/Makefile, src/option.c, src/testdir/Make_all.mak,
           src/testdir/test_alot.vim, src/testdir/test_options.in,
           src/testdir/test_options.ok, src/testdir/test_options.vim

Patch 7.4.2175
问题: Insufficient testing of cscope.
解决方案: Add more tests. (Dominique Pelle)
相关文件: src/testdir/test_cscope.vim

Patch 7.4.2176
问题: #ifdefs in main() are complicated.
解决方案: Always define vim_main2(). Move params to the file level.
           (suggested by Ken Takata)
相关文件: src/main.c, src/structs.h, src/vim.h, src/if_mzsch.c,
           src/proto/if_mzsch.pro

Patch 7.4.2177
问题: No testing for -C and -N command line flags, file arguments,
           startuptime.
解决方案: Add tests.
相关文件: src/testdir/test_startup.vim, src/testdir/shared.vim

Patch 7.4.2178
问题: No test for reading from stdin.
解决方案: Add a test.
相关文件: src/testdir/test_startup.vim, src/testdir/shared.vim

Patch 7.4.2179 (after 7.4.2178)
问题: Reading from stdin test fails on MS-Windows.
解决方案: Strip the extra space.
相关文件: src/testdir/test_startup.vim

Patch 7.4.2180

问题: There is no easy way to stop all timers. There is no way to
            temporary pause a timer.
解决方案: Add timer_stopall() and timer_pause().
相关文件: src/evalfunc.c, src/ex_cmds2.c, src/proto/ex_cmds2.pro,
            src/structs.h, src/testdir/test_timers.vim,
            src/testdir/shared.vim, runtime/doc/eval.txt

Patch 7.4.2181
问题: Compiler warning for unused variable.
解决方案: Remove it. (Dominique Pelle)
相关文件: src/ex_cmds2.c

Patch 7.4.2182
问题: Color Grey40 used in startup but not in the short list.
解决方案: Add Grey40 to the builtin colors.
相关文件: src/term.c

Patch 7.4.2183
问题: Sign tests are old style.
解决方案: Turn them into new style tests. (Dominique Pelle)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_signs.in,
            src/testdir/test_signs.ok, src/testdir/test_signs.vim,

Patch 7.4.2184
问题: Tests that use RunVim() do not actually perform the test.
解决方案: Use "return" instead of "call". (Ken Takata)
相关文件: src/testdir/shared.vim

Patch 7.4.2185
问题: Test glob2regpat does not test much.
解决方案: Add a few more test cases. (Dominique Pelle)
相关文件: src/testdir/test_glob2regpat.vim

Patch 7.4.2186
问题: Timers test is flaky.
解决方案: Relax the sleep time check.
相关文件: src/testdir/test_timers.vim

Patch 7.4.2187 (after 7.4.2185)
问题: glob2regpat test fails on Windows.
解决方案: Remove the checks that use backslashes.
相关文件: src/testdir/test_glob2regpat.vim

Patch 7.4.2188 (after 7.4.2146)
问题: Completion does not work properly with some plugins.
解决方案: Revert the part related to typing CTRL-E. (closes #972)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 7.4.2189
问题: Cannot detect encoding in a fifo.
解决方案: Extend the stdin way of detecting encoding to fifo. Add a test
            for detecting encoding on stdin and fifo. (Ken Takata)
相关文件: src/buffer.c, src/fileio.c, src/Makefile,
            src/testdir/Make_all.mak, src/testdir/test_startup_utf8.vim,

src/vim.h

Patch 7.4.2190
问题: When startup test fails it's not easy to find out why.
GUI test fails with Gnome.
解决方案: Add the help entry matches to a list an assert that.
Set $HOME for Gnome to create .gnome2 directory.
相关文件: src/testdir/test_startup.vim, src/testdir/test_gui.vim

Patch 7.4.2191
问题: No automatic prototype for vim_main2().
解决方案: Move the #endif. (Ken Takata)
相关文件: src/main.c, src/vim.h, src/proto/main.pro

Patch 7.4.2192
问题: Generating prototypes with Cygwin doesn't work well.
解决方案: Change #ifdefs. (Ken Takata)
相关文件: src/gui.h, src/gui_w32.c, src/ops.c, src/proto/fileio.pro,
src/proto/message.pro, src/proto/normal.pro, src/proto/ops.pro,
src/vim.h

Patch 7.4.2193
问题: With Gnome when the GUI can't start test_startup hangs.
解决方案: Call gui_mch_early_init_check(). (Hirohito Higashi)
相关文件: src/gui.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro

Patch 7.4.2194
问题: Sign tests don't cover enough.
解决方案: Add more test cases. (Dominique Pelle)
相关文件: src/testdir/test_signs.vim

Patch 7.4.2195
问题: MS-Windows: The vimrun program does not support Unicode.
解决方案: Use GetCommandLineW(). Cleanup old #ifdefs. (Ken Takata)
相关文件: src/vimrun.c

Patch 7.4.2196
问题: glob2regpat test doesn't test everything on MS-Windows.
解决方案: Add patterns with backslash handling.
相关文件: src/testdir/test_glob2regpat.vim

Patch 7.4.2197
问题: All functions are freed on exit, which may hide leaks.
解决方案: Only free named functions, not reference counted ones.
相关文件: src/userfunc.c

Patch 7.4.2198
问题: Test alot sometimes fails under valgrind. (Dominique Pelle)
解决方案: Avoid passing a callback with the wrong number of arguments.
相关文件: src/testdir/test_partial.vim

Patch 7.4.2199
问题: In the GUI the cursor is hidden when redrawing any window,
causing flicker.

解决方案: Only undraw the cursor when updating the window it's in.
相关文件: src/screen.c, src/gui.c, src/proto/gui.pro, src/gui_gtk_x11.c

Patch 7.4.2200
问题: Cannot get all information about a quickfix list.
解决方案: Add an optional argument to get/set loc/qf list(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/proto/quickfix.pro, src/quickfix.c, src/tag.c, src/testdir/test_quickfix.vim

Patch 7.4.2201
问题: The sign column disappears when the last sign is deleted.
解决方案: Add the 'signcolumn' option. (Christian Brabandt)
相关文件: runtime/doc/options.txt, runtime/optwin.vim, src/edit.c, src/move.c, src/option.c, src/option.h, src/proto/option.pro, src/screen.c, src/structs.h, src/testdir/test_options.vim

Patch 7.4.2202
问题: Build fails with small features.
解决方案: Correct option initialization.
相关文件: src/option.c

Patch 7.4.2203
问题: Test fails with normal features.
解决方案: Check is signs are supported.
相关文件: src/testdir/test_options.vim

Patch 7.4.2204
问题: It is not easy to get information about buffers, windows and tabpages.
解决方案: Add getbufinfo(), getwininfo() and gettabinfo(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/dict.c, src/evalfunc.c, src/option.c, src/proto/dict.pro, src/proto/option.pro, src/proto/window.pro, src/testdir/Make_all.mak, src/testdir/test_bufwintabinfo.vim, src/window.c, src/Makefile

Patch 7.4.2205
问题: 'wildignore' always applies to getcompletion().
解决方案: Add an option to use 'wildignore' or not. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_cmdline.vim

Patch 7.4.2206
问题: Warning for unused function.
解决方案: Put the function inside #ifdef. (John Marriott)
相关文件: src/evalfunc.c

Patch 7.4.2207
问题: The +xpm feature is not sorted properly in :version output.
解决方案: Move it up. (Tony Mechelynck)
相关文件: src/version.c

Patch 7.4.2208

问题: Test for mappings is old style.
解决方案: Convert the test to new style.
相关文件: src/testdir/test_mapping.vim, src/testdir/test_mapping.in,
            src/testdir/test_mapping.ok, src/Makefile,
            src/testdir/test_alot.vim, src/testdir/Make_all.mak

Patch 7.4.2209
问题: Cannot map <M-">. (Stephen Riehm)
解决方案: Solve the memory access problem in another way. (Dominique Pelle)
            Allow for using <M-\"> in a string.
相关文件: src/eval.c, src/gui_mac.c, src/misc2.c, src/option.c,
            src/proto/misc2.pro, src/syntax.c, src/term.c,
            src/testdir/test_mapping.vim

Patch 7.4.2210
问题: On OSX configure mixes up a Python framework and the Unix layout.
解决方案: Make configure check properly. (Tim D. Smith, closes #980)
相关文件: src/configure.in, src/auto/configure

Patch 7.4.2211
问题: Mouse support is not automatically enabled with simple term.
解决方案: Recognize "st" and other names. (Manuel Schiller, closes #963)
相关文件: src/os_unix.c

Patch 7.4.2212
问题: Mark " is not set when closing a window in another tab. (Guraga)
解决方案: Check all tabs for the window to be valid. (based on patch by
            Hirohito Higashi, closes #974)
相关文件: src/window.c, src/proto/window.pro, src/buffer.c,
            src/testdir/test_viminfo.vim

Patch 7.4.2213
问题: Cannot highlight the "~" lines at the end of a window differently.
解决方案: Add the EndOfBuffer highlighting. (Marco Hinz, James McCoy)
相关文件: runtime/doc/options.txt, runtime/doc/syntax.txt, src/option.c,
            src/screen.c, src/syntax.c, src/vim.h

Patch 7.4.2214
问题: A font that uses ligatures messes up the screen display.
解决方案: Put spaces between characters when building the glyph table.
            (based on a patch from Manuel Schiller)
相关文件: src/gui_gtk_x11.c

Patch 7.4.2215
问题: It's not easy to find out if a window is a quickfix or location
            list window.
解决方案: Add "loclist" and "quickfix" entries to the dict returned by
            getwininfo(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_bufwintabinfo.vim

Patch 7.4.2216 (after 7.4.2215)
问题: Test fails without the +sign feature.
解决方案: Only check for signcolumn with the +sign feature.

相关文件: src/testdir/test_bufwintabinfo.vim

Patch 7.4.2217
问题: When using matchaddpos() a character after the end of the line can be highlighted.
解决方案: Only highlight existing characters. (Hirohito Higashi)
相关文件: src/screen.c, src/structs.h, src/testdir/test_match.vim

Patch 7.4.2218
问题: Can't build with +timers when +digraph is not included.
解决方案: Change #ifdef for e_number_exp. (Damien)
相关文件: src/globals.h

Patch 7.4.2219
问题: Recursive call to substitute gets stuck in sandbox. (Nikolai Pavlov)
解决方案: Handle the recursive call. (Christian Brabandt, closes #950) Add a test.
相关文件: src/ex_cmds.c, src/testdir/test_regexp_latin.vim

Patch 7.4.2220
问题: printf() gives an error when the argument for %s is not a string. (Ozaki Kiichi)
解决方案: Behave like invoking string() on the argument. (Ken Takata)
相关文件: runtime/doc/eval.txt, src/message.c, src/testdir/test_expr.vim

Patch 7.4.2221
问题: printf() does not support binary format.
解决方案: Add %b and %B. (Ozaki Kiichi)
相关文件: runtime/doc/eval.txt, src/message.c, src/testdir/test_expr.vim

Patch 7.4.2222
问题: Sourcing a script where a character has 0x80 as a second byte does not work. (Filipe L B Correia)
解决方案: Turn 0x80 into K_SPECIAL KS_SPECIAL KE_FILLER. (Christian Brabandt, closes #728) Add a test case.
相关文件: src/getchar.c, src/proto/getchar.pro, src/misc1.c, src/testdir/test_regexp_utf8.vim

Patch 7.4.2223
问题: Buffer overflow when using latin1 character with feedkeys().
解决方案: Check for an illegal character. Add a test.
相关文件: src/testdir/test_regexp_utf8.vim, src/testdir/test_source_utf8.vim, src/testdir/test_alot_utf8.vim, src/Makefile, src/getchar.c, src/macros.h, src/evalfunc.c, src/os_unix.c, src/os_win32.c, src/spell.c,

Patch 7.4.2224
问题: Compiler warnings with older compiler and 64 bit numbers.
解决方案: Add "LL" to large values. (Mike Williams)
相关文件: src/eval.c, src/evalfunc.c

Patch 7.4.2225
问题: Crash when placing a sign in a deleted buffer.

解决方案: Check for missing buffer name. (Dominique Pelle). Add a test.
相关文件: src/ex_cmds.c, src/testdir/test_signs.vim

Patch 7.4.2226
问题: The field names used by getbufinfo(), gettabinfo() and
getwininfo() are not consistent.
解决方案: Use bufnr, winnr and tabnr. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_bufwintabinfo.vim

Patch 7.4.2227
问题: Tab page tests are old style.
解决方案: Change into new style tests. (Hirohito Higashi)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test62.in,
src/testdir/test62.ok, src/testdir/test_alot.vim,
src/testdir/test_tabpage.vim

Patch 7.4.2228
问题: Test files have inconsistent modelines.
解决方案: Don't set 'tabstop' to 2, use 'sts' and 'sw'.
相关文件: src/testdir/README.txt, src/testdir/test_backspace_opt.vim,
src/testdir/test_digraph.vim, src/testdir/test_gn.vim
src/testdir/test_help_tagjump.vim,
src/testdir/test_increment_dbcs.vim,
src/testdir/test_increment.vim, src/testdir/test_match.vim,
src/testdir/test_tagjump.vim, src/testdir/test_window_cmd.vim,
src/testdir/test_regexp_latin.vim, src/testdir/test_timers.vim

Patch 7.4.2229
问题: Startup test fails on Solaris.
解决方案: Recognize a character device. (Danek Duvall)
相关文件: src/buffer.c, src/fileio.c, src/proto/fileio.pro, src/vim.h

Patch 7.4.2230
问题: There is no equivalent of 'smartcase' for a tag search.
解决方案: Add value "followscs" and "smart" to 'tagcase'. (Christian
Brabandt, closes #712) Turn tagcase test into new style.
相关文件: runtime/doc/options.txt, runtime/doc/tagsrch.txt, src/option.h,
src/tag.c, src/search.c, src/proto/search.pro,
src/testdir/test_tagcase.in, src/testdir/test_tagcase.ok,
src/testdir/test_tagcase.vim, src/Makefile,
src/testdir/Make_all.mak, src/testdir/test_alot.vim

Patch 7.4.2231
问题: ":oldfiles" output is a very long list.
解决方案: Add a pattern argument. (Coot, closes #575)
相关文件: runtime/doc/starting.txt, src/ex_cmds.h, src/eval.c,
src/ex_cmds.c, src/proto/eval.pro, src/proto/ex_cmds.pro,
src/testdir/test_viminfo.vim

Patch 7.4.2232
问题: The default ttimeoutlen is very long.
解决方案: Use "100". (Hirohito Higashi)
相关文件: runtime/defaults.vim

Patch 7.4.2233
问题: Crash when using funcref() with invalid name. (Dominique Pelle)
解决方案: Check for NULL translated name.
相关文件: src/evalfunc.c, src/testdir/test_expr.vim

Patch 7.4.2234
问题: Can't build with +eval but without +quickfix. (John Marriott)
解决方案: Move skip_vimgrep_pat() to separate #ifdef block.
相关文件: src/quickfix.c

Patch 7.4.2235
问题: submatch() does not check for a valid argument.
解决方案: Give an error if the argument is out of range. (Dominique Pelle)
相关文件: src/evalfunc.c, src/testdir/test_expr.vim

Patch 7.4.2236
问题: The 'langnoremap' option leads to double negatives. And it does not work for the last character of a mapping.
解决方案: Add 'langremap' with the opposite value. Keep 'langnoremap' for backwards compatibility. Make it work for the last character of a mapping. Make the test work.
相关文件: runtime/doc/options.txt, runtime/defaults.vim, src/option.c, src/option.h, src/macros.h, src/testdir/test_mapping.vim

Patch 7.4.2237
问题: Can't use "." and "$" with ":tab".
解决方案: Support a range for ":tab". (Hirohito Higashi)
相关文件: runtime/doc/tabpage.txt, src/ex_docmd.c, src/testdir/test_tabpage.vim

Patch 7.4.2238
问题: With SGR mouse reporting (suckless terminal) the mouse release and scroll up/down is confused.
解决方案: Don't see a release as a scroll up/down. (Ralph Eastwood)
相关文件: src/term.c

Patch 7.4.2239
问题: Warning for missing declaration of skip_vimgrep_pat(). (John Marriott)
解决方案: Move it to another file.
相关文件: src/quickfix.c, src/proto/quickfix.pro, src/ex_cmds.c, src/proto/ex_cmds.pro

Patch 7.4.2240
问题: Tests using the sleep time can be flaky.
解决方案: Use reltime() if available. (Partly by Shane Harper)
相关文件: src/testdir/shared.vim, src/testdir/test_timers.vim

Patch 7.4.2241 (after 7.4.2240)
问题: Timer test sometimes fails.
解决方案: Increase the maximum time for repeating timer.
相关文件: src/testdir/test_timers.vim

Patch 7.4.2242 (after 7.4.2240)
问题: Timer test sometimes fails.
解决方案: Increase the maximum time for callback timer test.
相关文件: src/testdir/test_timers.vim

Patch 7.4.2243
问题: Warning for assigning negative value to unsigned. (Danek Duvall)
解决方案: Make cterm_normal_fg_gui_color and _bg_ guicolor_T, cast to long_u
            only when an unsigned is needed.
相关文件: src/structs.h, src/globals.h, src/screen.c, src/term.c,
            src/syntax.c, src/gui_gtk_x11.c, src/gui.c, src/gui_mac.c,
            src/gui_photon.c, src/gui_w32.c, src/gui_x11.c,
            src/proto/term.pro, src/proto/gui_gtk_x11.pro,
            src/proto/gui_mac.pro, src/proto/gui_photon.pro,
            src/proto/gui_w32.pro, src/proto/gui_x11.pro

Patch 7.4.2244
问题: Adding pattern to ":oldfiles" is not a generic solution.
解决方案: Add the ":filter /pat/ cmd" command modifier. Only works for some
            commands right now.
相关文件: src/structs.h, src/ex_docmd.c, src/ex_cmds.h, src/message.c,
            src/proto/message.pro, runtime/doc/starting.txt,
            runtime/doc/various.txt, src/testdir/test_viminfo.vim,
            src/testdir/test_alot.vim, src/testdir/test_filter_cmd.vim,
            src/Makefile

Patch 7.4.2245 (after 7.4.2244)
问题: Filter test fails.
解决方案: Include missing changes.
相关文件: src/buffer.c

Patch 7.4.2246 (after 7.4.2244)
问题: Oldfiles test fails.
解决方案: Include missing changes.
相关文件: src/ex_cmds.c

Patch 7.4.2247 (after 7.4.2244)
问题: Tiny build fails. (Tony Mechelynck)
解决方案: Remove #ifdef.
相关文件: src/ex_cmds.c

Patch 7.4.2248
问题: When cancelling the :ptjump prompt a preview window is opened for
            a following command.
解决方案: Reset g_do_tagpreview. (Hirohito Higashi) Add a test. Avoid that
            the test runner gets stuck in trying to close a window.
相关文件: src/tag.c, src/testdir/test_tagjump.vim, src/testdir/runtest.vim

Patch 7.4.2249
问题: Missing colon in error message.
解决方案: Add the colon. (Dominique Pelle)
相关文件: src/userfunc.c

Patch 7.4.2250

问题: Some error messages cannot be translated.
解决方案: Enclose them in _() and N_(). (Dominique Pelle)
相关文件: src/channel.c, src/evalfunc.c, src/ex_cmds.c, src/spell.c,
            src/window.c

Patch 7.4.2251
问题: In rare cases diffing 4 buffers is not enough.
解决方案: Raise the limit to 8. (closes #1000)
相关文件: src/structs.h, runtime/doc/diff.txt

Patch 7.4.2252
问题: Compiler warnings for signed/unsigned in expression.
解决方案: Remove type cast. (Dominique Pelle)
相关文件: src/vim.h

Patch 7.4.2253
问题: Check for Windows 3.1 will always return false. (Christian
            Brabandt)
解决方案: Remove the dead code.
相关文件: src/gui_w32.c, src/evalfunc.c, src/ex_cmds.c, src/option.c,
            src/os_win32.c, src/version.c, src/proto/gui_w32.pro

Patch 7.4.2254
问题: Compiler warnings in MzScheme code.
解决方案: Add UNUSED. Remove unreachable code.
相关文件: src/if_mzsch.c

Patch 7.4.2255
问题: The script that checks translations can't handle plurals.
解决方案: Check for plural msgid and msgstr entries. Leave the cursor on
            the first error.
相关文件: src/po/check.vim

Patch 7.4.2256
问题: Coverity complains about null pointer check.
解决方案: Remove wrong and superfluous error check.
相关文件: src/eval.c

Patch 7.4.2257
问题: Coverity complains about not checking for NULL.
解决方案: Check for out of memory.
相关文件: src/if_py_both.h

Patch 7.4.2258
问题: Two JSON messages are sent without a separator.
解决方案: Separate messages with a NL. (closes #1001)
相关文件: src/json.c, src/channel.c, src/vim.h, src/testdir/test_channel.py,
            src/testdir/test_channel.vim, runtime/doc/channel.txt

Patch 7.4.2259
问题: With 'incsearch' can only see the next match.
解决方案: Make CTRL-N/CTRL-P move to the previous/next match. (Christian
            Brabandt)
相关文件: runtime/doc/cmdline.txt, src/ex_getln.c, src/testdir/Make_all.mak,

src/testdir/test_search.vim, src/Makefile

Patch 7.4.2260 (after 7.4.2258)
问题: Channel test is flaky.
解决方案: Add a newline to separate JSON messages.
相关文件: src/testdir/test_channel.vim

Patch 7.4.2261 (after 7.4.2259)
问题: Build fails with small features.
解决方案: Move "else" inside the #ifdef.
相关文件: src/ex_getln.c

Patch 7.4.2262
问题: Fail to read register content from viminfo if it is 438 characters
long. (John Chen)
解决方案: Adjust the check for line wrapping. (closes #1010)
相关文件: src/testdir/test_viminfo.vim, src/ex_cmds.c

Patch 7.4.2263
问题: :filter does not work for many commands. Can only get matching
messages.
解决方案: Make :filter work for :command, :map, :list, :number and :print.
Make ":filter!" show non-matching lines.
相关文件: src/getchar.c, src/ex_cmds.c, src/ex_cmds.h, src/ex_docmd.c,
src/message.c, src/structs.h, src/testdir/test_filter_cmd.vim

Patch 7.4.2264
问题: When adding entries to an empty quickfix list the title is reset.
解决方案: Improve handling of the title. (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim, src/quickfix.c

Patch 7.4.2265
问题: printf() isn't tested much.
解决方案: Add more tests for printf(). (Dominique Pelle)
相关文件: src/testdir/test_expr.vim

Patch 7.4.2266 (after 7.4.2265)
问题: printf() test fails on Windows. "-inf" is not used.
解决方案: Check for Windows-specific values for "nan". Add sign to "inf"
when appropriate.
相关文件: src/message.c, src/testdir/test_expr.vim

Patch 7.4.2267 (after 7.4.2266)
问题: Build fails on MS-Windows.
解决方案: Add define to get isinf().
相关文件: src/message.c

Patch 7.4.2268 (after 7.4.2259)
问题: Using CTRL-N and CTRL-P for incsearch shadows completion keys.
解决方案: Use CTRL-T and CTRL-G instead.
相关文件: runtime/doc/cmdline.txt, src/ex_getln.c,
src/testdir/test_search.vim

Patch 7.4.2269

问题:　　Using 'hlsearch' highlighting instead of matchpos if there is no search match.
解决方案:　Pass NULL as last item to next_search_hl() when searching for 'hlsearch' match. (Shane Harper, closes #1013)
相关文件:　src/screen.c, src/testdir/test_match.vim

Patch 7.4.2270
问题:　　Insufficient testing for NUL bytes on a raw channel.
解决方案:　Add a test for writing and reading.
相关文件:　src/testdir/test_channel.vim

Patch 7.4.2271
问题:　　Netbeans test doesn't read settings from file.
解决方案:　Use "-Xnbauth".
相关文件:　src/testdir/test_netbeans.vim

Patch 7.4.2272
问题:　　getbufinfo(), getwininfo() and gettabinfo() are inefficient.
解决方案:　Instead of making a copy of the variables dictionary, use a reference.
相关文件:　src/evalfunc.c

Patch 7.4.2273
问题:　　getwininfo() and getbufinfo() are inefficient.
解决方案:　Do not make a copy of all window/buffer-local options. Make it possible to get them with gettabwinvar() or getbufvar().
相关文件:　src/evalfunc.c, src/eval.c, src/testdir/test_bufwintabinfo.vim, runtime/doc/eval.txt

Patch 7.4.2274
问题:　　Command line completion on "find **/filename" drops sub-directory.
解决方案:　Handle this case separately. (Harm te Hennepe, closes #932, closes #939)
相关文件:　src/misc1.c, src/testdir/test_cmdline.vim

Patch 7.4.2275
问题:　　":diffoff!" does not remove filler lines.
解决方案:　Force a redraw and invalidate the cursor. (closes #1014)
相关文件:　src/diff.c, src/testdir/test_diffmode.vim

Patch 7.4.2276
问题:　　Command line test fails on Windows when run twice.
解决方案:　Wipe the buffer so that the directory can be deleted.
相关文件:　src/testdir/test_cmdline.vim

Patch 7.4.2277
问题:　　Memory leak in getbufinfo() when there is a sign. (Dominique Pelle)
解决方案:　Remove extra vim_strsave().
相关文件:　src/evalfunc.c

Patch 7.4.2278
问题:　　New users have no idea of the 'scrolloff' option.
解决方案:　Set 'scrolloff' in defaults.vim.

相关文件: runtime/defaults.vim

Patch 7.4.2279
问题: Starting diff mode with the cursor in the last line might end up
only showing one closed fold. (John Beckett)
解决方案: Scroll the window to show the same relative cursor position.
相关文件: src/diff.c, src/window.c, src/proto/window.pro

Patch 7.4.2280
问题: printf() doesn't handle infinity float values correctly.
解决方案: Add a table with possible infinity values. (Dominique Pelle)
相关文件: src/message.c, src/testdir/test_expr.vim

Patch 7.4.2281
问题: Timer test fails sometimes.
解决方案: Reduce minimum time by 1 msec.
相关文件: src/testdir/test_timers.vim

Patch 7.4.2282
问题: When a child process is very fast waiting 10 msec for it is
noticeable. (Ramel Eshed)
解决方案: Start waiting for 1 msec and gradually increase.
相关文件: src/os_unix.c

Patch 7.4.2283
问题: Part of ":oldfiles" command isn't cleared. (Lifepillar)
解决方案: Clear the rest of the line. (closes 1018)
相关文件: src/ex_cmds.c

Patch 7.4.2284
问题: Comment in scope header file is outdated. (KillTheMule)
解决方案: Point to the help instead. (closes #1017)
相关文件: src/if_cscope.h

Patch 7.4.2285
问题: Generated files are outdated.
解决方案: Generate the files. Avoid errors when generating prototypes.
相关文件: src/if_mzsch.h, src/Makefile, src/option.h, src/os_mac_conv.c,
src/os_amiga.c, src/vim.h, src/structs.h, src/os_win32.c,
src/if_lua.c, src/proto/mbyte.pro

Patch 7.4.2286
问题: The tee program isn't included. Makefile contains build
instructions that don't work.
解决方案: Update the Filelist and build instructions. Remove build
instructions for DOS and old Windows. Add the tee program.
相关文件: Filelist, Makefile, nsis/gvim.nsi

Patch 7.4.2287
问题: The callback passed to ch_sendraw() is not used.
解决方案: Pass the read part, not the send part. (haya14busa, closes #1019)
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.2288

问题:　　MS-Windows build instructions are clumsy. "dosbin" doesn't build.
解决方案:　Add rename.bat. Fix building "dosbin".
相关文件:　Makefile, Filelist, rename.bat

Patch 7.4.2289
问题:　　When installing and $DESTDIR is set the icons probably won't be
　　installed.
解决方案:　Create the icon directories if $DESTDIR is not empty. (Danek
　　Duvall)
相关文件:　src/Makefile

Patch 7.4.2290
问题:　　Compiler warning in tiny build. (Tony Mechelynck)
解决方案:　Add #ifdef around infinity_str().
相关文件:　src/message.c

Patch 7.4.2291
问题:　　printf() handles floats wrong when there is a sign.
解决方案:　Fix placing the sign. Add tests. (Dominique Pelle)
相关文件:　src/testdir/test_expr.vim, runtime/doc/eval.txt, src/message.c

Patch 7.4.2292 (after 7.4.2291)
问题:　　Not all systems understand %F in printf().
解决方案:　Use %f.
相关文件:　src/message.c

Patch 7.4.2293
问题:　　Modelines in source code are inconsistent.
解决方案:　Use the same line in most files. Add 'noet'. (Naruhiko Nishino)
相关文件:　src/alloc.h, src/arabic.c, src/arabic.h, src/ascii.h,
　　src/blowfish.c, src/buffer.c, src/channel.c, src/charset.c,
　　src/crypt.c, src/crypt_zip.c, src/dict.c, src/diff.c,
　　src/digraph.c, src/dosinst.c, src/dosinst.h, src/edit.c,
　　src/eval.c, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds.h,
　　src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c,
　　src/farsi.c, src/farsi.h, src/feature.h, src/fileio.c, src/fold.c,
　　src/getchar.c, src/glbl_ime.cpp, src/glbl_ime.h, src/globals.h,
　　src/gui.c, src/gui.h, src/gui_at_fs.c, src/gui_at_sb.c,
　　src/gui_at_sb.h, src/gui_athena.c, src/gui_beval.c,
　　src/gui_beval.h, src/gui_gtk.c, src/gui_gtk_f.c, src/gui_gtk_f.h,
　　src/gui_gtk_vms.h, src/gui_gtk_x11.c, src/gui_mac.c,
　　src/gui_motif.c, src/gui_photon.c, src/gui_w32.c, src/gui_x11.c,
　　src/gui_x11_pm.h, src/gui_xmdlg.c, src/gui_xmebw.c,
　　src/gui_xmebw.h, src/gui_xmebwp.h, src/hangulin.c, src/hardcopy.c,
　　src/hashtab.c, src/if_cscope.c, src/if_cscope.h, src/if_mzsch.c,
　　src/if_mzsch.h, src/if_ole.cpp, src/if_perl.xs, src/if_perlsfio.c,
　　src/if_python3.c, src/if_ruby.c, src/if_tcl.c, src/if_xcmdsrv.c,
　　src/integration.c, src/integration.h, src/iscygpty.c, src/json.c,
　　src/json_test.c, src/keymap.h, src/list.c, src/macros.h,
　　src/main.c, src/mark.c, src/mbyte.c, src/memfile.c,
　　src/memfile_test.c, src/memline.c, src/menu.c, src/message.c,
　　src/message_test.c, src/misc1.c, src/misc2.c, src/move.c,
　　src/nbdebug.c, src/nbdebug.h, src/netbeans.c, src/normal.c,
　　src/ops.c, src/option.c, src/option.h, src/os_amiga.c,

src/os_amiga.h, src/os_beos.c, src/os_beos.h, src/os_dos.h,
src/os_mac.h, src/os_mac_conv.c, src/os_macosx.m, src/os_mint.h,
src/os_mswin.c, src/os_qnx.c, src/os_qnx.h, src/os_unix.c,
src/os_unix.h, src/os_unixx.h, src/os_vms.c, src/os_w32dll.c,
src/os_w32exe.c, src/os_win32.c, src/os_win32.h, src/popupmnu.c,
src/proto.h, src/pty.c, src/quickfix.c, src/regexp.c,
src/regexp.h, src/regexp_nfa.c, src/screen.c, src/search.c,
src/sha256.c, src/spell.c, src/spell.h, src/spellfile.c,
src/structs.h, src/syntax.c, src/tag.c, src/term.c, src/term.h,
src/termlib.c, src/ui.c, src/undo.c, src/uninstal.c,
src/userfunc.c, src/version.c, src/version.h, src/vim.h,
src/vim.rc, src/vimio.h, src/vimrun.c, src/winclip.c,
src/window.c, src/workshop.c, src/workshop.h, src/wsdebug.c,
src/wsdebug.h, src/xpm_w32.c

Patch 7.4.2294
问题: Sign test fails on MS-Windows when using the distributed zip
archives.
解决方案: Create dummy files instead of relying on files in the pixmaps
directory.
相关文件: src/testdir/test_signs.vim

Patch 7.4.2295 (after 7.4.2293)
问题: Cscope test fails.
解决方案: Avoid checking for specific line and column numbers.
相关文件: src/testdir/test_cscope.vim

Patch 7.4.2296
问题: No tests for :undolist and "U" command.
解决方案: Add tests. (Dominique Pelle)
相关文件: src/testdir/test_undo.vim

Patch 7.4.2297
问题: When starting a job that reads from a buffer and reaching the end,
the job hangs.
解决方案: Close the pipe or socket when all lines were read.
相关文件: src/channel.c, src/testdir/test_channel.vim

Patch 7.4.2298
问题: It is not possible to close the "in" part of a channel.
解决方案: Add ch_close_in().
相关文件: src/evalfunc.c, src/channel.c, src/proto/channel.pro,
src/testdir/test_channel.vim, runtime/doc/eval.txt,
runtime/doc/channel.txt

Patch 7.4.2299
问题: QuickFixCmdPre and QuickFixCmdPost autocommands are not always
triggered.
解决方案: Also trigger on ":cexpr", ":cbuffer", etc. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 7.4.2300
问题: Get warning for deleting autocommand group when the autocommand
using the group is scheduled for deletion. (Pavol Juhas)

解决方案: Check for deleted autocommand.
相关文件: src/fileio.c, src/testdir/test_autocmd.vim

Patch 7.4.2301
问题: MS-Windows: some files remain after testing.
解决方案: Close the channel output file. Wait for the file handle to be closed before deleting the file.
相关文件: src/os_win32.c, src/testdir/test_channel.vim

Patch 7.4.2302
问题: Default interface versions for MS-Windows are outdated.
解决方案: Use Active Perl 5.24, Python 3.5.2. Could only make it work with Ruby 1.9.2.
相关文件: src/bigvim.bat, src/bigvim64.bat, src/Make_mvc.mak

Patch 7.4.2303
问题: When using "is" the mode isn't always updated.
解决方案: Redraw the command line. (Christian Brabandt)
相关文件: src/search.c

Patch 7.4.2304
问题: In a timer callback the timer itself can't be found or stopped. (Thinca)
解决方案: Do not remove the timer from the list, remember whether it was freed.
相关文件: src/ex_cmds2.c, src/testdir/test_timers.vim

Patch 7.4.2305
问题: Marks, writefile and nested function tests are old style.
解决方案: Turn them into new style tests. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/test_marks.in, src/testdir/test_marks.ok, src/testdir/test_marks.vim, src/testdir/test_nested_function.in, src/testdir/test_nested_function.ok, src/testdir/test_nested_function.vim, src/testdir/test_writefile.in, src/testdir/test_writefile.ok, src/testdir/test_writefile.vim, src/Makefile

Patch 7.4.2306
问题: Default value for 'langremap' is wrong.
解决方案: Set the right value. (Jürgen Krmer) Add a test.
相关文件: src/option.c, src/testdir/test_mapping.vim

Patch 7.4.2307
问题: Several tests are old style.
解决方案: Turn them into new style tests. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/test102.in, src/testdir/test102.ok, src/testdir/test46.in, src/testdir/test46.ok, src/testdir/test81.in, src/testdir/test81.ok, src/testdir/test_charsearch.in, src/testdir/test_charsearch.ok, src/testdir/test_charsearch.vim, src/testdir/test_fnameescape.vim, src/testdir/test_substitute.vim, src/Makefile

Patch 7.4.2308 (after 7.4.2307)
问题: Old charsearch test still listed in Makefile.
解决方案: Remove the line.
相关文件: src/testdir/Make_all.mak

Patch 7.4.2309
问题: Crash when doing tabnext in a BufUnload autocmd. (Dominique Pelle)
解决方案: When detecting that the tab page changed, don't just abort but delete the window where w_buffer is NULL.
相关文件: src/window.c, src/testdir/test_tabpage.vim

Patch 7.4.2310 (after 7.4.2304)
问题: Accessing freed memory when a timer does not repeat.
解决方案: Free after removing it. (Dominique Pelle)
相关文件: src/ex_cmds2.c

Patch 7.4.2311
问题: Appveyor 64 bit build still using Python 3.4
解决方案: Switch to Python 3.5. (Ken Takata, closes #1032)
相关文件: appveyor.yml, src/appveyor.bat

Patch 7.4.2312
问题: Crash when autocommand moves to another tab. (Dominique Pelle)
解决方案: When navigating to another window halfway the :edit command go back to the right window.
相关文件: src/buffer.c, src/ex_cmds.c, src/ex_getln.c, src/ex_docmd.c, src/window.c, src/proto/ex_getln.pro, src/testdir/test_tabpage.vim

Patch 7.4.2313
问题: Crash when deleting an augroup and listing an autocommand. (Dominique Pelle)
解决方案: Make sure deleted_augroup is valid.
相关文件: src/fileio.c, src/testdir/test_autocmd.vim

Patch 7.4.2314
问题: No error when deleting an augroup while it's the current one.
解决方案: Disallow deleting an augroup when it's the current one.
相关文件: src/fileio.c, src/testdir/test_autocmd.vim

Patch 7.4.2315
问题: Insufficient testing for Normal mode commands.
解决方案: Add a big test. (Christian Brabandt, closes #1029)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_normal.vim

Patch 7.4.2316
问题: Channel sort test is flaky.
解决方案: Add a check the output has been read.
相关文件: src/testdir/test_channel.vim

Patch 7.4.2317 (after 7.4.2315)
问题: Normal mode tests fail on MS-Windows.
解决方案: Do some tests only on Unix. Set 'fileformat' to "unix".
相关文件: src/testdir/test_normal.vim

Patch 7.4.2318
问题: When 'incsearch' is not set CTRL-T and CTRL-G are not inserted as before.
解决方案: Move #ifdef and don't use goto.
相关文件: src/ex_getln.c

Patch 7.4.2319
问题: No way for a system wide vimrc to stop loading defaults.vim. (Christian Hesse)
解决方案: Bail out of defaults.vim if skip_defaults_vim was set.
相关文件: runtime/defaults.vim

Patch 7.4.2320
问题: Redraw problem when using 'incsearch'.
解决方案: Save the current view when deleting characters. (Christian Brabandt) Fix that the '" mark is set in the wrong position. Don't change the search start when using BS.
相关文件: src/ex_getln.c, src/normal.c, src/testdir/test_search.vim

Patch 7.4.2321
问题: When a test is commented out we forget about it.
解决方案: Let a test throw an exception with "Skipped" and list skipped test functions. (Christian Brabandt)
相关文件: src/testdir/Makefile, src/testdir/runtest.vim, src/testdir/test_popup.vim, src/testdir/README.txt

Patch 7.4.2322
问题: Access memory beyond the end of the line. (Dominique Pelle)
解决方案: Adjust the cursor column.
相关文件: src/move.c, src/testdir/test_normal.vim

Patch 7.4.2323
问题: Using freed memory when using 'formatexpr'. (Dominique Pelle)
解决方案: Make a copy of 'formatexpr' before evaluating it.
相关文件: src/ops.c, src/testdir/test_normal.vim

Patch 7.4.2324
问题: Crash when editing a new buffer and BufUnload autocommand wipes out the new buffer. (Norio Takagi)
解决方案: Don't allow wiping out this buffer. (partly by Hirohito Higashi) Move old style test13 into test_autocmd. Avoid ml_get error when editing a file.
相关文件: src/structs.h, src/buffer.c, src/ex_cmds.c, src/ex_docmd.c, src/window.c, src/testdir/test13.in, src/testdir/test13.ok, src/testdir/test_autocmd.vim, src/testdir/Make_all.mak, src/Makefile

Patch 7.4.2325 (after 7.4.2324)
问题: Tiny build fails.
解决方案: Add #ifdef.
相关文件: src/buffer.c

Patch 7.4.2326

问题:　　Illegal memory access when Visual selection starts in invalid position. (Dominique Pelle)
解决方案:　Correct position when needed.
相关文件:　src/normal.c, src/misc2.c, src/proto/misc2.pro

Patch 7.4.2327
问题:　　Freeing a variable that is on the stack.
解决方案:　Don't free res_tv or err_tv. (Ozaki Kiichi)
相关文件:　src/channel.c

Patch 7.4.2328
问题:　　Crash when BufWinLeave autocmd goes to another tab page. (Hirohito Higashi)
解决方案:　Make close_buffer() go back to the right window.
相关文件:　src/buffer.c, src/testdir/test_autocmd.vim

Patch 7.4.2329
问题:　　Error for min() and max() contains %s. (Nikolai Pavlov)
解决方案:　Pass the function name. (closes #1040)
相关文件:　src/evalfunc.c, src/testdir/test_expr.vim

Patch 7.4.2330
问题:　　Coverity complains about not checking curwin to be NULL.
解决方案:　Use firstwin to avoid the warning.
相关文件:　src/buffer.c

Patch 7.4.2331
问题:　　Using CTRL-X CTRL-V to complete a command line from Insert mode does not work after entering an expression on the command line.
解决方案:　Don't use "ccline" when not actually using a command line. (test by Hirohito Higashi)
相关文件:　src/edit.c, src/ex_getln.c, src/proto/ex_getln.pro, src/testdir/test_popup.vim

Patch 7.4.2332
问题:　　Crash when stop_timer() is called in a callback of a callback. Vim hangs when the timer callback uses too much time.
解决方案:　Set tr_id to -1 when a timer is to be deleted. Don't keep calling callbacks forever. (Ozaki Kiichi)
相关文件:　src/evalfunc.c, src/ex_cmds2.c, src/structs.h, src/proto/ex_cmds2.pro, src/testdir/test_timers.vim

Patch 7.4.2333
问题:　　Outdated comments in test.
解决方案:　Cleanup normal mode test. (Christian Brabandt)
相关文件:　src/testdir/test_normal.vim

Patch 7.4.2334
问题:　　On MS-Windows test_getcwd leaves Xtopdir behind.
解决方案:　Set 'noswapfile'. (Michael Soyka)
相关文件:　src/testdir/test_getcwd.in

Patch 7.4.2335
问题:　　taglist() is slow. (Luc Hermitte)

解决方案: Check for CTRL-C less often when doing a linear search. (closes
           #1044)
相关文件: src/tag.c

Patch 7.4.2336
问题: Running normal mode tests leave a couple of files behind.
           (Yegappan Lakshmanan)
解决方案: Delete the files. (Christian Brabandt)
相关文件: src/testdir/test_normal.vim

Patch 7.4.2337
问题: taglist() is still slow. (Luc Hermitte)
解决方案: Check for CTRL-C less often when finding duplicates.
相关文件: src/tag.c

Patch 7.4.2338
问题: Can't build with small features. (John Marriott)
解决方案: Nearly always define FEAT_TAG_BINS.
相关文件: src/feature.h, src/tag.c

Patch 7.4.2339
问题: Tab page test fails when run as fake root.
解决方案: Check 'buftype' instead of 'filetype'. (James McCoy, closes #1042)
相关文件: src/testdir/test_tabpage.vim

Patch 7.4.2340
问题: MS-Windows: Building with Ruby uses old version.
解决方案: Update to 2.2.X. Use clearer name for the API version. (Ken
           Takata)
相关文件: Makefile, src/INSTALLpc.txt, src/Make_cyg_ming.mak,
           src/Make_mvc.mak, src/bigvim.bat

Patch 7.4.2341
问题: Tiny things. Test doesn't clean up properly.
解决方案: Adjust comment and white space. Restore option value.
相关文件: src/ex_cmds.c, src/message.c, src/testdir/test_autocmd.vim

Patch 7.4.2342
问题: Typo in MS-Windows build script.
解决方案: change "w2" to "22".
相关文件: src/bigvim.bat

Patch 7.4.2343
问题: Too many old style tests.
解决方案: Turn several into new style tests. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/test101.in,
           src/testdir/test101.ok, src/testdir/test18.in,
           src/testdir/test18.ok, src/testdir/test2.in, src/testdir/test2.ok,
           src/testdir/test21.in, src/testdir/test21.ok,
           src/testdir/test6.in, src/testdir/test6.ok,
           src/testdir/test_arglist.vim, src/testdir/test_charsearch.vim,
           src/testdir/test_fnameescape.vim, src/testdir/test_gf.vim,
           src/testdir/test_hlsearch.vim, src/testdir/test_smartindent.vim,
           src/testdir/test_tagjump.vim, src/Makefile

Patch 7.4.2344
问题: The "Reading from channel output..." message can be unwanted.
Appending to a buffer leaves an empty first line behind.
解决方案: Add the "out_msg" and "err_msg" options. Writing the first line
overwrites the first, empty line.
相关文件: src/structs.h, src/channel.c, src/testdir/test_channel.vim,
runtime/doc/channel.txt

Patch 7.4.2345 (after 7.4.2340)
问题: For MinGW RUBY_API_VER_LONG isn't set correctly. Many default
version numbers are outdated.
解决方案: Set RUBY_API_VER_LONG to RUBY_VER_LONG. Use latest stable releases
for defaults. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 7.4.2346
问题: Autocommand test fails when run directly, passes when run as part
of test_alot.
解决方案: Add command to make the cursor move. Close a tab page.
相关文件: src/testdir/test_autocmd.vim

Patch 7.4.2347
问题: Crash when closing a buffer while Visual mode is active.
(Dominique Pelle)
解决方案: Adjust the position before computing the number of lines.
When closing the current buffer stop Visual mode.
相关文件: src/buffer.c, src/normal.c, src/testdir/test_normal.vim

Patch 7.4.2348
问题: Crash on exit when EXITFREE is defined. (Dominique Pelle)
解决方案: Don't access curwin when exiting.
相关文件: src/buffer.c

Patch 7.4.2349
问题: Valgrind reports using uninitialized memory. (Dominique Pelle)
解决方案: Check the length before checking for a NUL.
相关文件: src/message.c

Patch 7.4.2350
问题: Test 86 and 87 fail with some version of Python.
解决方案: Unify "can't" and "cannot". Unify quotes.
相关文件: src/testdir/test86.in, src/testdir/test86.ok,
src/testdir/test87.in, src/testdir/test87.ok

Patch 7.4.2351
问题: Netbeans test fails when run from unpacked MS-Windows sources.
解决方案: Open README.txt instead of Makefile.
相关文件: src/testdir/test_netbeans.py, src/testdir/test_netbeans.vim

Patch 7.4.2352
问题: Netbeans test fails in shadow directory.
解决方案: Also copy README.txt to the shadow directory.
相关文件: src/Makefile

Patch 7.4.2353
问题: Not enough test coverage for Normal mode commands.
解决方案: Add more tests. (Christian Brabandt)
相关文件: src/testdir/test_normal.vim

Patch 7.4.2354
问题: The example that explains nested backreferences does not work
            properly with the new regexp engine. (Harm te Hennepe)
解决方案: Also save the end position when adding a state. (closes #990)
相关文件: src/regexp_nfa.c, src/testdir/test_regexp_latin.vim

Patch 7.4.2355
问题: Regexp fails to match when using "\>\)\?". (Ramel)
解决方案: When a state is already in the list, but addstate_here() is used
            and the existing state comes later, add the new state anyway.
相关文件: src/regexp_nfa.c, src/testdir/test_regexp_latin.vim

Patch 7.4.2356
问题: Reading past end of line when using previous substitute pattern.
            (Dominique Pelle)
解决方案: Don't set "pat" only set "searchstr".
相关文件: src/search.c, src/testdir/test_search.vim

Patch 7.4.2357
问题: Attempt to read history entry while not initialized.
解决方案: Skip when the index is negative.
相关文件: src/ex_getln.c

Patch 7.4.2358
问题: Compiler warnings with Solaris Studio when using GTK3. (Danek
            Duvall)
解决方案: Define FUNC2GENERIC depending on the system. (Kazunobu Kuriyama)
相关文件: src/gui.h, src/gui_beval.c, src/gui_gtk_f.c

Patch 7.4.2359
问题: Memory leak in timer_start().
解决方案: Check the right field to be NULL.
相关文件: src/evalfunc.c, src/testdir/test_timers.vim

Patch 7.4.2360
问题: Invalid memory access when formatting. (Dominique Pelle)
解决方案: Make sure cursor line and column are associated.
相关文件: src/misc1.c

Patch 7.4.2361
问题: Checking for last_timer_id to overflow is not reliable. (Ozaki
            Kiichi)
解决方案: Check for the number not going up.
相关文件: src/ex_cmds2.c

Patch 7.4.2362
问题: Illegal memory access with ":1@". (Dominique Pelle)
解决方案: Correct cursor column after setting the line number. Also avoid

calling end_visual_mode() when not in Visual mode.
相关文件: src/ex_docmd.c, src/buffer.c

Patch 7.4.2363
问题: Superfluous function prototypes.
解决方案: Remove them.
相关文件: src/regexp.c

Patch 7.4.2364
问题: Sort test sometimes fails.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 7.4.2365
问题: Needless line break. Confusing directory name.
解决方案: Remove line break. Prepend "../" to "tools".
相关文件: Makefile, src/normal.c

Patch 7.4.2366
问题: MS-Windows gvim.exe does not have DirectX support.
解决方案: Add the DIRECTX to the script.
相关文件: src/bigvim.bat

Patch 7.4.2367 (after 7.4.2364)
问题: Test runner misses a comma.
解决方案: Add the comma.
相关文件: src/testdir/runtest.vim

8.1 版本 version-8.1 version8.1 vim-8.1

本小节讨论 8.0 和 8.1 版本之间的改进。

这个版本有几百个漏洞修正，一个新特性和许多次要的改进。

终端窗口 new-terminal-window

你现在可以打开窗口来用作终端。可用它来:
- 在其它窗口编辑的同时，运行 "make" 这样的命令
- 运行外壳执行若干命令
- 使用终端调试器插件，见 terminal-debugger

所有这些，对在远程 (ssh) 连接上运行 Vim 尤其有用，那里不方便打开更多的终端。

更多详情可见 terminal-window 。

改动 changed-8.1

内部: 现在允许一些 C99 特性，如 // 注释和枚举的最后一个项目之后的逗号。见

style-compiler 。

8.0.0029 补丁开始，删除了旧 MS-Windows 系统的支持，只支持 MS-Windows XP 和之后的版本。

新增 added-8.1

新增若干语法、缩进和其它插件。

Quickfix 改进 (Yegappan Lakshmanan 提供):
新增 quickfix 栈对任意 quickfix/位置列表修改的支持。
新增每个 quickfix/位置列表的唯一标识符。
新增和 quickfix/位置列表关联任意 Vim 类型作为上下文的支持。
增强 getqflist()、getloclist()、setqflist() 和 setloclist()
获取和设置若干 quickfix/位置列表属性的函数。
新增 QuickFixLine 高亮组来高亮 quickfix 窗口的当前行。
quickfix 缓冲区 b:changedtick 变量用来为所在的 quickfix 列表的每个变动增量。
新增 quickfix/位置列表的 changedtick 变量，每次列表修改时加一。
新增用 'errorformat' 解析文本而不用建立新 quickfix 列表的支持。
新增 quickfix 项目的 "module" 项的支持，为了显示目的可用来代替很长的文件名。
新增释放所有 quickfix/位置栈列表的支持。
用 :copen/:cwindow 命令打开 quickfix 窗口时，使用提供的分割修饰符。

函数:
所有的 term_ 函数。

assert_beeps()
assert_equalfile()
assert_report()
balloon_show()
balloon_split()
ch_canread()
getchangelist()
getjumplist()
getwinpos()
pyxeval()
remote_startserver()
setbufline()
test_ignore_error()
test_override()
trim()
win_screenpos()

自动命令:
CmdlineChanged
CmdlineEnter
CmdlineLeave
ColorSchemePre
DirChanged
ExitPre

TerminalOpen
TextChangedP
TextYankPost

命令:

:pyx
:pythonx
:pyxdo
:pyxfile
:terminal
:tmapclear
:tmap
:tnoremap
:tunmap

选项:

'balloonevalterm'
'imstyle'
'mzschemedll'
'mzschemegcdll'
'makeencoding'
'pumwidth'
'pythonhome'
'pythonthreehome'
'pyxversion'
'termwinkey'
'termwinscroll'
'termwinsize'
'viminfofile'
'winptydll'

补丁 patches-8.1

Patch 8.0.0001
问题: Intro screen still mentions version7. (Paul)
解决方案: Change it to version8.
相关文件: src/version.c

Patch 8.0.0002
问题: The netrw plugin does not work.
解决方案: Make it accept version 8.0.
相关文件: runtime/autoload/netrw.vim

Patch 8.0.0003
问题: getwinvar() returns wrong Value of boolean and number options,
especially non big endian systems. (James McCoy)
解决方案: Cast the pointer to long or int. (closes #1060)
相关文件: src/option.c, src/testdir/test_bufwintabinfo.vim

Patch 8.0.0004
问题: A string argument for function() that is not a function name
results in an error message with NULL. (Christian Brabandt)

解决方案: Use the argument for the error message.
相关文件: src/evalfunc.c, src/testdir/test_expr.vim

Patch 8.0.0005
问题: Netbeans test fails with Python 3. (Jonathonf)
解决方案: Encode the string before sending it. (closes #1070)
相关文件: src/testdir/test_netbeans.py

Patch 8.0.0006
问题: ":lb" is interpreted as ":lbottom" while the documentation says it
           means ":lbuffer".
解决方案: Adjust the order of the commands. (haya14busa, closes #1093)
相关文件: src/ex_cmds.h

Patch 8.0.0007
问题: Vim 7.4 is still mentioned in a few places.
解决方案: Update to Vim 8. (Uncle Bill, closes #1094)
相关文件: src/INSTALLpc.txt, src/vimtutor, uninstal.txt

Patch 8.0.0008
问题: Popup complete test is disabled.
解决方案: Enable the test and change the assert. (Hirohito Higashi)
相关文件: src/testdir/test_popup.vim

Patch 8.0.0009
问题: Unnecessary workaround for AppVeyor.
解决方案: Revert patch 7.4.990. (Christian Brabandt)
相关文件: appveyor.yml

Patch 8.0.0010
问题: Crash when editing file that starts with crypt header. (igor2x)
解决方案: Check for length of text. (Christian Brabandt) Add a test.
相关文件: src/fileio.c, src/testdir/test_crypt.vim, src/Makefile,
           src/testdir/Make_all.mak

Patch 8.0.0011
问题: On OSX Test_pipe_through_sort_all() sometimes fails.
解决方案: Add the test to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0012
问题: Typos in comments.
解决方案: Change "its" to "it's". (Matthew Brener, closes #1088)
相关文件: src/evalfunc.c, src/main.aap, src/nbdebug.c, src/netbeans.c,
           src/quickfix.c, src/workshop.c, src/wsdebug.c

Patch 8.0.0013 (after 8.0.0011)
问题: Missing comma in list.
解决方案: Add the comma.
相关文件: src/testdir/runtest.vim

Patch 8.0.0014
问题: Crypt tests are old style.
解决方案: Convert to new style.

相关文件: src/testdir/test71.in, src/testdir/test71.ok,
           src/testdir/test71a.in, src/testdir/test_crypt.vim, src/Makefile,
           src/testdir/Make_all.mak

Patch 8.0.0015
问题: Can't tell which part of a channel has "buffered" status.
解决方案: Add an optional argument to ch_status(). Let ch_info() also
           return "buffered" for out_status and err_status.
相关文件: src/evalfunc.c, src/channel.c, src/proto/channel.pro,
           src/testdir/test_channel.vim, runtime/doc/eval.txt

Patch 8.0.0016 (after 8.0.0015)
问题: Build fails.
解决方案: Include missing change.
相关文件: src/eval.c

Patch 8.0.0017
问题: Cannot get the number of the current quickfix or location list.
解决方案: Use the current list if "nr" in "what" is zero. (Yegappan
           Lakshmanan) Remove debug command from test.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim,
           runtime/doc/eval.txt

Patch 8.0.0018
问题: When using ":sleep" channel input is not handled.
解决方案: When there is a channel check for input also when not in raw mode.
           Check every 100 msec.
相关文件: src/channel.c, src/proto/channel.pro, src/ui.c, src/proto/ui.pro,
           src/ex_docmd.c, src/os_amiga.c, src/proto/os_amiga.pro,
           src/os_unix.c, src/proto/os_unix.pro, src/os_win32.c,
           src/proto/os_win32.pro

Patch 8.0.0019
问题: Test_command_count is old style.
解决方案: Turn it into a new style test. (Naruhiko Nishino)
           Use more assert functions.
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_alot.vim,
           src/testdir/test_autocmd.vim, src/testdir/test_command_count.in,
           src/testdir/test_command_count.ok,
           src/testdir/test_command_count.vim

Patch 8.0.0020
问题: The regexp engines are not reentrant.
解决方案: Add regexec_T and save/restore the state when needed.
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test_expr.vim,
           runtime/doc/eval.txt, runtime/doc/change.txt

Patch 8.0.0021
问题: In the GUI when redrawing the cursor it may be on the second half
           of a double byte character.
解决方案: Correct the cursor column. (Yasuhiro Matsumoto)
相关文件: src/screen.c

Patch 8.0.0022

问题:　　If a channel in NL mode is missing the NL at the end the remaining characters are dropped.
解决方案:　When the channel is closed use the remaining text. (Ozaki Kiichi)
相关文件:　src/channel.c, src/testdir/test_channel.vim

Patch 8.0.0023
问题:　　"gd" and "gD" may find a match in a comment or string.
解决方案:　Ignore matches in comments and strings. (Anton Lindqvist)
相关文件:　src/normal.c, src/testdir/test_goto.vim

Patch 8.0.0024
问题:　　When the netbeans channel closes, "DETACH" is put in the output part. (Ozaki Kiichi)
解决方案:　Write "DETACH" in the socket part.
相关文件:　src/channel.c, src/testdir/test_netbeans.vim

Patch 8.0.0025
问题:　　Inconsistent use of spaces vs tabs in gd test.
解决方案:　Use tabs. (Anton Lindqvist)
相关文件:　src/testdir/test_goto.vim

Patch 8.0.0026
问题:　　Error format with %W, %C and %Z does not work. (Gerd Wachsmuth)
解决方案:　Skip code when qf_multiignore is set. (Lcd)
相关文件:　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0027
问题:　　A channel is closed when reading on stderr or stdout fails, but there may still be something to read on another part.
解决方案:　Turn ch_to_be_closed into a bitfield. (Ozaki Kiichi)
相关文件:　src/channel.c, src/eval.c, src/structs.h, src/proto/channel.pro, src/testdir/test_channel.vim

Patch 8.0.0028
问题:　　Superfluous semicolons.
解决方案:　Remove them. (Ozaki Kiichi)
相关文件:　src/ex_cmds2.c

Patch 8.0.0029
问题:　　Code for MS-Windows is complicated because of the exceptions for old systems.
解决方案:　Drop support for MS-Windows older than Windows XP. (Ken Takata)
相关文件:　runtime/doc/gui_w32.txt, runtime/doc/os_win32.txt, runtime/doc/todo.txt, src/GvimExt/Makefile, src/Make_mvc.mak, src/evalfunc.c, src/ex_cmds.c, src/ex_docmd.c, src/gui_w32.c, src/if_cscope.c, src/misc1.c, src/misc2.c, src/option.c, src/os_mswin.c, src/os_win32.c, src/os_win32.h, src/proto/os_mswin.pro, src/proto/os_win32.pro, src/version.c

Patch 8.0.0030
问题:　　Mouse mode is not automatically detected for tmux.
解决方案:　Check for 'term' to be "tmux". (Michael Henry)
相关文件:　src/os_unix.c

Patch 8.0.0031
问题:　　After ":bwipeout" 'fileformat' is not set to the right default.
解决方案:　Get the default from 'fileformats'. (Mike Williams)
相关文件:　src/option.c, src/Makefile, src/testdir/test_fileformat.vim,
　　src/testdir/test_alot.vim

Patch 8.0.0032
问题:　　Tests may change the input file when something goes wrong.
解决方案:　Avoid writing the input file.
相关文件:　src/testdir/test51.in, src/testdir/test67.in,
　　src/testdir/test97.in, src/testdir/test_tabpage.vim

Patch 8.0.0033
问题:　　Cannot use overlapping positions with matchaddpos().
解决方案:　Check end of match. (Ozaki Kiichi) Add a test (Hirohito Higashi)
相关文件:　src/screen.c, src/testdir/test_match.vim

Patch 8.0.0034
问题:　　No completion for ":messages".
解决方案:　Complete "clear" argument. (Hirohito Higashi)
相关文件:　src/ex_docmd.c, src/ex_getln.c, src/proto/ex_docmd.pro,
　　src/testdir/test_cmdline.vim, src/vim.h,
　　runtime/doc/eval.txt, runtime/doc/map.txt

Patch 8.0.0035 (after 7.4.2013)
问题:　　Order of matches for 'omnifunc' is messed up. (Danny Su)
解决方案:　Do not set compl_curr_match when called from complete_check().
　　(closes #1168)
相关文件:　src/edit.c, src/evalfunc.c, src/proto/edit.pro, src/search.c,
　　src/spell.c, src/tag.c, src/testdir/test76.in,
　　src/testdir/test76.ok, src/testdir/test_popup.vim, src/Makefile,
　　src/testdir/Make_all.mak

Patch 8.0.0036
问题:　　Detecting that a job has finished may take a while.
解决方案:　Check for a finished job more often (Ozaki Kiichi)
相关文件:　src/channel.c, src/os_unix.c, src/os_win32.c,
　　src/proto/os_unix.pro, src/proto/os_win32.pro,
　　src/testdir/test_channel.vim

Patch 8.0.0037
问题:　　Get E924 when switching tabs. ()
解决方案:　Use win_valid_any_tab() instead of win_valid(). (Martin Vuille,
　　closes #1167, closes #1171)
相关文件:　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0038
问题:　　OPEN_CHR_FILES not defined for FreeBSD using Debian userland
　　files.
解决方案:　Check for __FreeBSD_kernel__. (James McCoy, closes #1166)
相关文件:　src/vim.h

Patch 8.0.0039
问题:　　When Vim 8 reads an old viminfo and exits, the next time marks are

not read from viminfo. (Ned Batchelder)
解决方案: Set a mark when it wasn't set before, even when the timestamp is zero. (closes #1170)
相关文件: src/mark.c, src/testdir/test_viminfo.vim

Patch 8.0.0040 (after 8.0.0033)
问题: Whole line highlighting with matchaddpos() does not work.
解决方案: Check for zero length. (Hirohito Higashi)
相关文件: src/screen.c, src/testdir/test_match.vim

Patch 8.0.0041
问题: When using Insert mode completion but not actually inserting anything an undo item is still created. (Tommy Allen)
解决方案: Do not call stop_arrow() when not inserting anything.
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0042 (after 8.0.0041)
问题: When using Insert mode completion with 'completeopt' containing "noinsert" change is not saved for undo. (Tommy Allen)
解决方案: Call stop_arrow() before inserting for pressing Enter.
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0043 (after 8.0.0041)
问题: When using Insert mode completion with 'completeopt' containing "noinsert" with CTRL-N the change is not saved for undo. (Tommy Allen)
解决方案: Call stop_arrow() before inserting for any key.
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0044
问题: In diff mode the cursor may end up below the last line, resulting in an ml_get error.
解决方案: Check the line to be valid.
相关文件: src/move.c, src/diff.c, src/proto/diff.pro, src/testdir/test_diffmode.vim

Patch 8.0.0045
问题: Calling job_stop() right after job_start() does not work.
解决方案: Block signals while fork is still busy. (Ozaki Kiichi, closes #1155)
相关文件: src/auto/configure, src/config.h.in, src/configure.in, src/os_unix.c, src/testdir/test_channel.vim

Patch 8.0.0046
问题: Using NUL instead of NULL.
解决方案: Change to NULL. (Dominique Pelle)
相关文件: src/ex_cmds.c, src/json.c

Patch 8.0.0047
问题: Crash when using the preview window from an unnamed buffer. (lifepillar)
解决方案: Do not clear the wrong buffer. (closes #1200)
相关文件: src/popupmnu.c

Patch 8.0.0048
问题: On Windows job_stop() stops cmd.exe, not the processes it runs.
(Linwei)
解决方案: Iterate over all processes and terminate the one where the parent
is the job process. (Yasuhiro Matsumoto, closes #1184)
相关文件: src/os_win32.c, src/structs.h

Patch 8.0.0049
问题: When a match ends in part of concealed text highlighting, it might
mess up concealing by resetting prev_syntax_id.
解决方案: Do not reset prev_syntax_id and add a test to verify. (Christian
Brabandt, closes #1092)
相关文件: src/screen.c, src/testdir/test_matchadd_conceal.vim

Patch 8.0.0050
问题: An exiting job is detected with a large latency.
解决方案: Check for pending job more often. (Ozaki Kiichi) Change the
double loop in mch_inchar() into one.
相关文件: src/channel.c, src/os_unix.c, src/testdir/shared.vim,
src/testdir/test_channel.vim

Patch 8.0.0051 (after 8.0.0048)
问题: New code for job_stop() breaks channel test on AppVeyor.
解决方案: Revert the change.
相关文件: src/os_win32.c, src/structs.h

Patch 8.0.0052 (after 8.0.0049)
问题: Conceal test passes even without the bug fix.
解决方案: Add a redraw command. (Christian Brabandt)
相关文件: src/testdir/test_matchadd_conceal.vim

Patch 8.0.0053 (after 8.0.0047)
问题: No test for what 8.0.0047 fixes.
解决方案: Add a test. (Hirohito Higashi)
相关文件: src/testdir/test_popup.vim

Patch 8.0.0054 (after 8.0.0051)
问题: On Windows job_stop() stops cmd.exe, not the processes it runs.
(Linwei)
解决方案: Iterate over all processes and terminate the one where the parent
is the job process. Now only when there is no job object.
(Yasuhiro Matsumoto, closes #1203)
相关文件: src/os_win32.c

Patch 8.0.0055
问题: Minor comment and style deficiencies.
解决方案: Update comments and fix style.
相关文件: src/buffer.c, src/misc2.c, src/os_unix.c

Patch 8.0.0056
问题: When setting 'filetype' there is no check for a valid name.
解决方案: Only allow valid characters in 'filetype', 'syntax' and 'keymap'.
相关文件: src/option.c, src/testdir/test_options.vim

Patch 8.0.0057 (after 8.0.0056)
问题: Tests fail without the 'keymap' features.
解决方案: Check for feature in test.
相关文件: src/testdir/test_options.vim

Patch 8.0.0058
问题: Positioning of the popup menu is not good.
解决方案: Position it better. (Hirohito Higashi)
相关文件: src/popupmnu.c

Patch 8.0.0059
问题: Vim does not build on VMS systems.
解决方案: Various changes for VMS. (Zoltan Arpadffy)
相关文件: src/json.c, src/macros.h, src/Make_vms.mms, src/os_unix.c,
src/os_unix.h, src/os_vms.c, src/os_vms_conf.h,
src/proto/os_vms.pro, src/testdir/Make_vms.mms

Patch 8.0.0060
问题: When using an Ex command for 'keywordprg' it is escaped as with a
shell command. (Romain Lafourcade)
解决方案: Escape for an Ex command. (closes #1175)
相关文件: src/normal.c, src/testdir/test_normal.vim

Patch 8.0.0061 (after 8.0.0058)
问题: Compiler warning for unused variable.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/popupmnu.c

Patch 8.0.0062
问题: No digraph for HORIZONTAL ELLIPSIS.
解决方案: Use ",.". (Hans Ginzel, closes #1226)
相关文件: src/digraph.c, runtime/doc/digraph.txt

Patch 8.0.0063
问题: Compiler warning for comparing with unsigned. (Zoltan Arpadffy)
解决方案: Change <= to ==.
相关文件: src/undo.c

Patch 8.0.0064 (after 8.0.0060)
问题: Normal test fails on MS-Windows.
解决方案: Don't try using an illegal file name.
相关文件: src/testdir/test_normal.vim

Patch 8.0.0065 (after 8.0.0056)
问题: Compiler warning for unused function in tiny build. (Tony
Mechelynck)
解决方案: Add #ifdef.
相关文件: src/option.c

Patch 8.0.0066
问题: when calling an operator function when 'linebreak' is set, it is
internally reset before calling the operator function.
解决方案: Restore 'linebreak' before calling op_function(). (Christian
Brabandt)

相关文件: src/normal.c, src/testdir/test_normal.vim

Patch 8.0.0067
问题: VMS has a problem with infinity.
解决方案: Avoid an overflow. (Zoltan Arpadffy)
相关文件: src/json.c, src/macros.h

Patch 8.0.0068
问题: Checking did_throw after executing autocommands is wrong. (Daniel Hahler)
解决方案: Call aborting() instead, and only when autocommands were executed.
相关文件: src/quickfix.c, src/if_cscope.c, src/testdir/test_quickfix.vim

Patch 8.0.0069
问题: Compiler warning for self-comparison.
解决方案: Define ONE_WINDOW and add #ifdef.
相关文件: src/globals.h, src/buffer.c, src/ex_docmd.c, src/move.c, src/screen.c, src/quickfix.c, src/window.c

Patch 8.0.0070
问题: Tests referred in Makefile that no longer exist.
解决方案: Remove test71 and test74 entries. (Michael Soyka)
相关文件: src/testdir/Mak_ming.mak

Patch 8.0.0071
问题: Exit value from a shell command is wrong. (Hexchain Tong)
解决方案: Do not check for ended jobs while waiting for a shell command. (ichizok, closes #1196)
相关文件: src/os_unix.c

Patch 8.0.0072
问题: MS-Windows: Crash with long font name. (Henry Hu)
解决方案: Fix comparing with LF_FACESIZE. (Ken Takata, closes #1243)
相关文件: src/os_mswin.c

Patch 8.0.0073 (after 8.0.0069)
问题: More comparisons between firstwin and lastwin.
解决方案: Use ONE_WINDOW for consistency. (Hirohito Higashi)
相关文件: src/buffer.c, src/ex_cmds.c, src/ex_docmd.c, src/option.c, src/window.c

Patch 8.0.0074
问题: Cannot make Vim fail on an internal error.
解决方案: Add IEMSG() and IEMSG2(). (Dominique Pelle) Avoid reporting an internal error without mentioning where.
相关文件: src/globals.h, src/blowfish.c, src/dict.c, src/edit.c, src/eval.c, src/evalfunc.c, src/ex_eval.c, src/getchar.c, src/gui_beval.c, src/gui_w32.c, src/hangulin.c, src/hashtab.c, src/if_cscope.c, src/json.c, src/memfile.c, src/memline.c, src/message.c, src/misc2.c, src/option.c, src/quickfix.c, src/regexp.c, src/spell.c, src/undo.c, src/userfunc.c, src/vim.h, src/window.c, src/proto/misc2.pro, src/proto/message.pro, src/Makefile

Patch 8.0.0075

问题: Using number for exception type lacks type checking.
解决方案: Use an enum.
相关文件: src/structs.h, src/ex_docmd.c, src/ex_eval.c,
            src/proto/ex_eval.pro

Patch 8.0.0076
问题: Channel log has double parens ()().
解决方案: Remove () for write_buf_line. (Yasuhiro Matsumoto)
相关文件: src/channel.c

Patch 8.0.0077
问题: The GUI code is not tested by Travis.
解决方案: Install the virtual framebuffer.
相关文件: .travis.yml

Patch 8.0.0078
问题: Accessing freed memory in quickfix.
解决方案: Reset pointer when freeing 'errorformat'. (Dominique Pelle)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0079
问题: Accessing freed memory in quickfix. (Dominique Pelle)
解决方案: Do not free the current list when adding to it.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0080
问题: The OS X build fails on Travis.
解决方案: Skip the virtual framebuffer on OS X.
相关文件: .travis.yml

Patch 8.0.0081
问题: Inconsistent function names.
解决方案: Rename do_cscope to ex_cscope. Clean up comments.
相关文件: src/ex_cmds.h, src/if_cscope.c, src/ex_docmd.c,
            src/proto/if_cscope.pro

Patch 8.0.0082
问题: Extension for configure should be ".ac".
解决方案: Rename configure.in to configure.ac. (James McCoy, closes #1173)
相关文件: src/configure.in, src/configure.ac, Filelist, src/Makefile,
            src/blowfish.c, src/channel.c, src/config.h.in, src/main.aap,
            src/os_unix.c, src/INSTALL, src/mysign

Patch 8.0.0083
问题: Using freed memory with win_getid(). (Dominique Pelle)
解决方案: For the current tab use curwin.
相关文件: src/window.c, src/testdir/test_window_id.vim

Patch 8.0.0084
问题: Using freed memory when adding to a quickfix list. (Dominique
            Pelle)
解决方案: Clear the directory name.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0085
问题:　　Using freed memory with recursive function call. (Dominique Pelle)
解决方案:　Make a copy of the function name.
相关文件:　src/eval.c, src/testdir/test_nested_function.vim

Patch 8.0.0086
问题:　　Cannot add a comment after ":hide". (Norio Takagi)
解决方案:　Make it work, add a test. (Hirohito Higashi)
相关文件:　src/Makefile, src/ex_cmds.h, src/ex_docmd.c,
　　src/testdir/Make_all.mak, src/testdir/test_hide.vim

Patch 8.0.0087
问题:　　When the channel callback gets job info the job may already have
　　been deleted. (lifepillar)
解决方案:　Do not delete the job when the channel is still useful. (ichizok,
　　closes #1242, closes #1245)
相关文件:　src/channel.c, src/eval.c, src/os_unix.c, src/os_win32.c,
　　src/structs.h, src/testdir/test_channel.vim

Patch 8.0.0088
问题:　　When a test fails in Setup or Teardown the problem is not reported.
解决方案:　Add a try/catch. (Hirohito Higashi)
相关文件:　src/testdir/runtest.vim

Patch 8.0.0089
问题:　　Various problems with GTK 3.22.2.
解决方案:　Fix the problems, add #ifdefs. (Kazunobu Kuriyama)
相关文件:　src/gui_beval.c, src/gui_gtk.c, src/gui_gtk_x11.c

Patch 8.0.0090
问题:　　Cursor moved after last character when using 'breakindent'.
解决方案:　Fix the cursor positioning. Turn the breakindent test into new
　　style. (Christian Brabandt)
相关文件:　src/screen.c, src/testdir/Make_all.mak,
　　src/testdir/test_breakindent.in, src/testdir/test_breakindent.ok,
　　src/testdir/test_breakindent.vim, src/Makefile

Patch 8.0.0091
问题:　　Test_help_complete sometimes fails in MS-Windows console.
解决方案:　Use getcompletion() instead of feedkeys() and command line
　　completion. (Hirohito Higashi)
相关文件:　src/testdir/test_help_tagjump.vim

Patch 8.0.0092
问题:　　C indenting does not support nested namespaces that C++ 17 has.
解决方案:　Add check that passes double colon inside a name. (Pauli, closes
　　#1214)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 8.0.0093
问题:　　Not using multiprocess build feature.
解决方案:　Enable multiprocess build with MSVC 10. (Ken Takata)
相关文件:　src/Make_mvc.mak

Patch 8.0.0094
问题: When vimrun.exe is not found the error message is not properly encoded.
解决方案: Use utf-16 and MessageBoxW(). (Ken Takata)
相关文件: src/os_win32.c

Patch 8.0.0095
问题: Problems with GTK 3.22.2 fixed in 3.22.4.
解决方案: Adjust the #ifdefs. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 8.0.0096
问题: When the input or output is not a tty Vim appears to hang.
解决方案: Add the --ttyfail argument. Also add the "ttyin" and "ttyout" features to be able to check in Vim script.
相关文件: src/globals.h, src/structs.h, src/main.c, src/evalfunc.c, runtime/doc/starting.txt, runtime/doc/eval.txt

Patch 8.0.0097
问题: When a channel callback consumes a lot of time Vim becomes unresponsive. (skywind)
解决方案: Bail out of checking channel readahead after 100 msec.
相关文件: src/os_unix.c, src/misc2.c, src/vim.h, src/os_win32.c, src/channel.c

Patch 8.0.0098 (after 8.0.0097)
问题: Can't build on MS-Windows.
解决方案: Add missing parenthesis.
相关文件: src/vim.h

Patch 8.0.0099
问题: Popup menu always appears above the cursor when it is in the lower half of the screen. (Matt Gardner)
解决方案: Compute the available space better. (Hirohito Higashi, closes #1241)
相关文件: src/popupmnu.c

Patch 8.0.0100
问题: Options that are a file name may contain non-filename characters.
解决方案: Check for more invalid characters.
相关文件: src/option.c

Patch 8.0.0101
问题: Some options are not strictly checked.
解决方案: Add flags for stricter checks.
相关文件: src/option.c

Patch 8.0.0102 (after 8.0.0101)
问题: Cannot set 'dictionary' to a path.
解决方案: Allow for slash and backslash. Add a test (partly by Daisuke Suzuki, closes #1279, closes #1284)
相关文件: src/option.c, src/testdir/test_options.vim

Patch 8.0.0103

问题:　　May not process channel readahead. (skywind)
解决方案:　If there is readahead don't block on input.
相关文件:　src/channel.c, src/proto/channel.pro, src/os_unix.c,
　　src/os_win32.c, src/misc2.c

Patch 8.0.0104
问题:　　Value of 'thesaurus' option not checked properly.
解决方案:　Add P_NDNAME flag. (Daisuke Suzuki)
相关文件:　src/option.c, src/testdir/test_options.vim

Patch 8.0.0105
问题:　　When using ch_read() with zero timeout, can't tell the difference
　　between reading an empty line and nothing available.
解决方案:　Add ch_canread().
相关文件:　src/evalfunc.c, src/channel.c, src/proto/channel.pro,
　　src/testdir/test_channel.vim, src/testdir/shared.vim,
　　runtime/doc/eval.txt, runtime/doc/channel.txt

Patch 8.0.0106 (after 8.0.0100)
问题:　　Cannot use a semicolon in 'backupext'. (Jeff)
解决方案:　Allow for a few more characters when "secure" isn't set.
相关文件:　src/option.c

Patch 8.0.0107
问题:　　When reading channel output in a timer, messages may go missing.
　　(Skywind)
解决方案:　Add the "drop" option. Write error messages in the channel log.
　　Don't have ch_canread() check for the channel being open.
相关文件:　src/structs.h, src/channel.c, src/message.c, src/evalfunc.c,
　　src/proto/channel.pro, runtime/doc/channel.txt

Patch 8.0.0108 (after 8.0.0107)
问题:　　The channel "drop" option is not tested.
解决方案:　Add a test.
相关文件:　src/testdir/test_channel.vim

Patch 8.0.0109
问题:　　Still checking if memcmp() exists while every system should have
　　it now.
解决方案:　Remove vim_memcmp(). (James McCoy, closes #1295)
相关文件:　src/config.h.in, src/configure.ac, src/misc2.c, src/os_vms_conf.h,
　　src/osdef1.h.in, src/search.c, src/tag.c, src/vim.h

Patch 8.0.0110
问题:　　Drop command doesn't use existing window.
解决方案:　Check the window width properly. (Hirohito Higashi)
相关文件:　src/buffer.c, src/testdir/test_tabpage.vim

Patch 8.0.0111
问题:　　The :history command is not tested.
解决方案:　Add tests. (Dominique Pelle)
相关文件:　runtime/doc/cmdline.txt, src/testdir/test_history.vim

Patch 8.0.0112

问题:　　Tests 92 and 93 are old style.
解决方案:　Make test92 and test93 new style. (Hirohito Higashi, closes #1289)
相关文件:　src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test92.in, src/testdir/test92.ok,
src/testdir/test93.in, src/testdir/test93.ok,
src/testdir/test_mksession.vim,
src/testdir/test_mksession_utf8.vim

Patch 8.0.0113
问题:　　MS-Windows: message box to prompt for saving changes may appear on
the wrong monitor.
解决方案:　Adjust the CenterWindow function. (Ken Takata)
相关文件:　src/gui_w32.c

Patch 8.0.0114
问题:　　Coding style not optimal.
解决方案:　Add spaces. (Ken Takata)
相关文件:　src/gui_w32.c, src/os_mswin.c

Patch 8.0.0115
问题:　　When building with Cygwin libwinpthread isn't found.
解决方案:　Link winpthread statically. (jmmerz, closes #1255, closes #1256)
相关文件:　src/Make_cyg_ming.mak

Patch 8.0.0116
问题:　　When reading English help and using CTRl-] the language from
'helplang' is used.
解决方案:　Make help tag jumps keep the language. (Tatsuki, test by Hirohito
Higashi, closes #1249)
相关文件:　src/tag.c, src/testdir/test_help_tagjump.vim

Patch 8.0.0117
问题:　　Parallel make fails. (J. Lewis Muir)
解决方案:　Make sure the objects directory exists. (closes #1259)
相关文件:　src/Makefile

Patch 8.0.0118
问题:　　"make proto" adds extra function prototype.
解决方案:　Add #ifdef.
相关文件:　src/misc2.c

Patch 8.0.0119
问题:　　No test for using CTRL-R on the command line.
解决方案:　Add a test. (Dominique Pelle) And some more.
相关文件:　src/testdir/test_cmdline.vim

Patch 8.0.0120
问题:　　Channel test is still flaky on OS X.
解决方案:　Set the drop argument to "never".
相关文件:　src/testdir/test_channel.vim

Patch 8.0.0121
问题:　　Setting 'cursorline' changes the curswant column. (Daniel Hahler)
解决方案:　Add the P_RWINONLY flag. (closes #1297)

相关文件: src/option.c, src/testdir/test_goto.vim

Patch 8.0.0122
问题: Channel test is still flaky on OS X.
解决方案: Add a short sleep.
相关文件: src/testdir/test_channel.py

Patch 8.0.0123
问题: Modern Sun compilers define "__sun" instead of "sun".
解决方案: Use __sun. (closes #1296)
相关文件: src/mbyte.c, src/pty.c, src/os_unixx.h, src/vim.h

Patch 8.0.0124
问题: Internal error for assert_inrange(1, 1).
解决方案: Adjust number of allowed arguments. (Dominique Pelle)
相关文件: src/evalfunc.c, src/testdir/test_assert.vim

Patch 8.0.0125
问题: Not enough testing for entering Ex commands.
解决方案: Add test for CTRL-\ e {expr}. (Dominique Pelle)
相关文件: src/testdir/test_cmdline.vim

Patch 8.0.0126
问题: Display problem with 'foldcolumn' and a wide character.
(esiegerman)
解决方案: Don't use "extra" but an allocated buffer. (Christian Brabandt,
closes #1310)
相关文件: src/screen.c, src/testdir/Make_all.mak, src/Makefile,
src/testdir/test_display.vim

Patch 8.0.0127
问题: Cancelling completion still inserts text when formatting is done
for 'textwidth'. (lacygoill)
解决方案: Don't format when CTRL-E was typed. (Hirohito Higashi,
closes #1312)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0128 (after 8.0.0126)
问题: Display test fails on MS-Windows.
解决方案: Set 'isprint' to "@".
相关文件: src/testdir/test_display.vim

Patch 8.0.0129
问题: Parallel make still doesn't work. (Lewis Muir)
解决方案: Define OBJ_MAIN.
相关文件: src/Makefile

Patch 8.0.0130
问题: Configure uses "ushort" while the Vim code doesn't.
解决方案: Use "unsigned short" instead. (Fredrik Fornwall, closes #1314)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.0131
问题: Not enough test coverage for syntax commands.

解决方案: Add more tests. (Dominique Pelle)
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0132 (after 8.0.0131)
问题: Test fails because of using :finish.
解决方案: Change to return.
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0133
问题: "2;'(" causes ml_get errors in an empty buffer. (Dominique Pelle)
解决方案: Check the cursor line earlier.
相关文件: src/ex_docmd.c, src/testdir/test_cmdline.vim

Patch 8.0.0134
问题: Null pointer access reported by UBsan.
解决方案: Check curwin->w_buffer is not NULL. (Yegappan Lakshmanan)
相关文件: src/ex_cmds.c

Patch 8.0.0135
问题: An address relative to the current line, ":.,+3y", does not work
properly on a closed fold. (Efraim Yawitz)
解决方案: Correct for including the closed fold. (Christian Brabandt)
相关文件: src/ex_docmd.c, src/testdir/test_fold.vim,
src/testdir/Make_all.mak, src/Makefile

Patch 8.0.0136
问题: When using indent folding and changing indent the wrong fold is
opened. (Jonathan Fudger)
解决方案: Open the fold under the cursor a bit later. (Christian Brabandt)
相关文件: src/ops.c, src/testdir/test_fold.vim

Patch 8.0.0137
问题: When 'maxfuncdepth' is set above 200 the nesting is limited to
200. (Brett Stahlman)
解决方案: Allow for Ex command recursion depending on 'maxfuncdepth'.
相关文件: src/ex_docmd.c, src/testdir/test_nested_function.vim

Patch 8.0.0138 (after 8.0.0137)
问题: Small build fails.
解决方案: Add #ifdef.
相关文件: src/ex_docmd.c

Patch 8.0.0139 (after 8.0.0135)
问题: Warning for unused argument.
解决方案: Add UNUSED.
相关文件: src/ex_docmd.c

Patch 8.0.0140
问题: Pasting inserted text in Visual mode does not work properly.
(Matthew Malcomson)
解决方案: Stop Visual mode before stuffing the inserted text. (Christian
Brabandt, from neovim #5709)
相关文件: src/ops.c, src/testdir/test_visual.vim

Patch 8.0.0141 (after 8.0.0137)
问题:　　Nested function test fails on AppVeyor.
解决方案:　Disable the test on Windows for now.
相关文件:　src/testdir/test_nested_function.vim

Patch 8.0.0142
问题:　　Normal colors are wrong with 'termguicolors'.
解决方案:　Initialize to INVALCOLOR instead of zero. (Ben Jackson, closes #1344)
相关文件:　src/syntax.c

Patch 8.0.0143
问题:　　Line number of current buffer in getbufinfo() is wrong.
解决方案:　For the current buffer use the current line number. (Ken Takata)
相关文件:　src/evalfunc.c

Patch 8.0.0144
问题:　　When using MSVC the GvimExt directory is cleaned twice.
解决方案:　Remove the lines. (Ken Takata)
相关文件:　src/Make_mvc.mak

Patch 8.0.0145
问题:　　Running tests on MS-Windows is a little bit noisy.
解决方案:　Redirect some output to "nul". (Ken Takata)
相关文件:　src/testdir/Make_dos.mak

Patch 8.0.0146
问题:　　When using 'termguicolors' on MS-Windows the RGB definition causes the colors to be wrong.
解决方案:　Undefined RGB and use our own. (Gabriel Barta)
相关文件:　src/term.c

Patch 8.0.0147
问题:　　searchpair() does not work when 'magic' is off. (Chris Paul)
解决方案:　Add \m in the pattern. (Christian Brabandt, closes #1341)
相关文件:　src/evalfunc.c, src/testdir/test_search.vim

Patch 8.0.0148
问题:　　When a C preprocessor statement has two line continuations the following line does not have the right indent. (Ken Takata)
解决方案:　Add the indent of the previous continuation line. (Hirohito Higashi)
相关文件:　src/misc1.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 8.0.0149
问题:　　":earlier" and ":later" do not work after startup or reading the undo file.
解决方案:　Use absolute time stamps instead of relative to the Vim start time. (Christian Brabandt, Pavel Juhas, closes #1300, closes #1254)
相关文件:　src/testdir/test_undo.vim, src/undo.c

Patch 8.0.0150
问题:　　When the pattern of :filter does not have a separator then

completion of the command fails.
解决方案: Skip over the pattern. (Ozaki Kiichi, closes #1299)
相关文件: src/ex_docmd.c, src/testdir/test_filter_cmd.vim

Patch 8.0.0151
问题: To pass buffer content to system() and systemlist() one has to first create a string or list.
解决方案: Allow passing a buffer number. (LemonBoy, closes #1240)
相关文件: runtime/doc/eval.txt, src/Makefile, src/evalfunc.c, src/testdir/Make_all.mak, src/testdir/test_system.vim

Patch 8.0.0152
问题: Running the channel test creates channellog.
解决方案: Delete the debug line.
相关文件: src/testdir/test_channel.vim

Patch 8.0.0153 (after 8.0.0151)
问题: system() test fails on MS-Windows.
解决方案: Deal with extra space and CR.
相关文件: src/testdir/test_system.vim

Patch 8.0.0154 (after 8.0.0151)
问题: system() test fails on OS/X.
解决方案: Deal with leading spaces.
相关文件: src/testdir/test_system.vim

Patch 8.0.0155
问题: When sorting zero elements a NULL pointer is passed to qsort(), which ubsan warns for.
解决方案: Don't call qsort() if there are no elements. (Dominique Pelle)
相关文件: src/syntax.c

Patch 8.0.0156
问题: Several float functions are not covered by tests.
解决方案: Add float tests. (Dominique Pelle)
相关文件: src/Makefile, src/testdir/test_alot.vim, src/testdir/test_float_func.vim

Patch 8.0.0157
问题: No command line completion for ":syntax spell" and ":syntax sync".
解决方案: Implement the completion. (Dominique Pelle)
相关文件: src/syntax.c, src/testdir/test_syntax.vim

Patch 8.0.0158 (after 8.0.0156)
问题: On MS-Windows some float functions return a different value when passed unusual values. strtod() doesn't work for "inf" and "nan".
解决方案: Accept both results. Fix str2float() for MS-Windows. Also reorder assert function arguments.
相关文件: src/testdir/test_float_func.vim, src/eval.c

Patch 8.0.0159
问题: Using a NULL pointer when using feedkeys() to trigger drawing a tabline.
解决方案: Skip drawing a tabline if TabPageIdxs is NULL. (Dominique Pelle)

Also fix recursing into getcmdline() from the cmd window.
相关文件: src/screen.c, src/ex_getln.c

Patch 8.0.0160
问题: EMSG() is sometimes used for internal errors.
解决方案: Change them to IEMSG(). (Dominique Pelle) And a few more.
相关文件: src/regexp_nfa.c, src/channel.c, src/eval.c

Patch 8.0.0161 (after 8.0.0159)
问题: Build fails when using small features.
解决方案: Update #ifdef for using save_ccline. (Hirohito Higashi)
相关文件: src/ex_getln.c

Patch 8.0.0162
问题: Build error on Fedora 23 with small features and gnome2.
解决方案: Undefine ngettext(). (Hirohito Higashi)
相关文件: src/gui_gtk.c, src/gui_gtk_x11.c

Patch 8.0.0163
问题: Ruby 2.4 no longer supports rb_cFixnum.
解决方案: move rb_cFixnum into an #ifdef. (Kazuki Sakamoto, closes #1365)
相关文件: src/if_ruby.c

Patch 8.0.0164
问题: Outdated and misplaced comments.
解决方案: Fix the comments.
相关文件: src/charset.c, src/getchar.c, src/list.c, src/misc2.c,
src/testdir/README.txt

Patch 8.0.0165
问题: Ubsan warns for integer overflow.
解决方案: Swap two conditions. (Dominique Pelle)
相关文件: src/regexp_nfa.c

Patch 8.0.0166
问题: JSON with a duplicate key gives an internal error. (Lcd)
解决方案: Give a normal error. Avoid an error when parsing JSON from a
remote client fails.
相关文件: src/evalfunc.c, src/json.c, src/channel.c,
src/testdir/test_json.vim

Patch 8.0.0167
问题: str2nr() and str2float() do not always work with negative values.
解决方案: Be more flexible about handling signs. (LemonBoy, closes #1332)
Add more tests.
相关文件: src/evalfunc.c, src/testdir/test_float_func.vim,
src/testdir/test_functions.vim, src/testdir/test_alot.vim,
src/Makefile

Patch 8.0.0168
问题: Still some float functionality is not covered by tests.
解决方案: Add more tests. (Dominique Pelle, closes #1364)
相关文件: src/testdir/test_float_func.vim

Patch 8.0.0169
问题:　　For complicated string json_decode() may run out of stack space.
解决方案:　Change the recursive solution into an iterative solution.
相关文件:　src/json.c

Patch 8.0.0170 (after 8.0.0169)
问题:　　Channel test fails for using freed memory.
解决方案:　Fix memory use in json_decode().
相关文件:　src/json.c

Patch 8.0.0171
问题:　　JS style JSON does not support single quotes.
解决方案:　Allow for single quotes. (Yasuhiro Matsumoto, closes #1371)
相关文件:　src/json.c, src/testdir/test_json.vim, src/json_test.c,
　　runtime/doc/eval.txt

Patch 8.0.0172 (after 8.0.0159)
问题:　　The command selected in the command line window is not executed.
　　(Andrey Starodubtsev)
解决方案:　Save and restore the command line at a lower level. (closes #1370)
相关文件:　src/ex_getln.c, src/testdir/test_history.vim

Patch 8.0.0173
问题:　　When compiling with EBCDIC defined the build fails. (Yaroslav
　　Kuzmin)
解决方案:　Move sortFunctions() to the right file. Avoid warning for
　　redefining __SUSV3.
相关文件:　src/eval.c, src/evalfunc.c, src/os_unixx.h

Patch 8.0.0174
问题:　　For completion "locale -a" is executed on MS-Windows, even though
　　it most likely won't work.
解决方案:　Skip executing "locale -a" on MS-Windows. (Ken Takata)
相关文件:　src/ex_cmds2.c

Patch 8.0.0175
问题:　　Setting language in gvim on MS-Windows does not work when
　　libintl.dll is dynamically linked with msvcrt.dll.
解决方案:　Use putenv() from libintl as well. (Ken Takata, closes #1082)
相关文件:　src/mbyte.c, src/misc1.c, src/os_win32.c, src/proto/os_win32.pro,
　　src/vim.h

Patch 8.0.0176
问题:　　Using :change in between :function and :endfunction fails.
解决方案:　Recognize :change inside a function. (ichizok, closes #1374)
相关文件:　src/userfunc.c, src/testdir/test_viml.vim

Patch 8.0.0177
问题:　　When opening a buffer on a directory and inside a try/catch then
　　the BufEnter event is not triggered.
解决方案:　Return NOTDONE from readfile() for a directory and deal with the
　　three possible return values. (Justin M. Keyes, closes #1375,
　　closes #1353)
相关文件:　src/buffer.c, src/ex_cmds.c, src/ex_docmd.c, src/fileio.c,

src/memline.c

Patch 8.0.0178
问题: test_command_count may fail when a previous test interferes, seen on MS-Windows.
解决方案: Run it separately.
相关文件: src/testdir/test_alot.vim, src/testdir/Make_all.mak

Patch 8.0.0179
问题: 'formatprg' is a global option but the value may depend on the type of buffer. (Sung Pae)
解决方案: Make 'formatprg' global-local. (closes #1380)
相关文件: src/structs.h, src/option.h, src/option.c, src/normal.c, runtime/doc/options.txt, src/testdir/test_normal.vim

Patch 8.0.0180
问题: Error E937 is used both for duplicate key in JSON and for trying to delete a buffer that is in use.
解决方案: Rename the JSON error to E938. (Norio Takagi, closes #1376)
相关文件: src/json.c, src/testdir/test_json.vim

Patch 8.0.0181
问题: When 'cursorbind' and 'cursorcolumn' are both on, the column highlight in non-current windows is wrong.
解决方案: Add validate_cursor(). (Masanori Misono, closes #1372)
相关文件: src/move.c

Patch 8.0.0182
问题: When 'cursorbind' and 'cursorline' are set, but 'cursorcolumn' is not, then the cursor line highlighting is not updated. (Hirohito Higashi)
解决方案: Call redraw_later() with NOT_VALID.
相关文件: src/move.c

Patch 8.0.0183
问题: Ubsan warns for using a pointer that is not aligned.
解决方案: First copy the address. (Yegappan Lakshmanan)
相关文件: src/channel.c

Patch 8.0.0184
问题: When in Ex mode and an error is caught by try-catch, Vim still exits with a non-zero exit code.
解决方案: Don't set ex_exitval when inside a try-catch. (partly by Christian Brabandt)
相关文件: src/message.c, src/testdir/test_system.vim

Patch 8.0.0185 (after 8.0.0184)
问题: The system() test fails on MS-Windows.
解决方案: Skip the test on MS-Windows.
相关文件: src/testdir/test_system.vim

Patch 8.0.0186
问题: The error message from assert_notequal() is confusing.
解决方案: Only mention the expected value.

相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 8.0.0187
问题: Building with a new Ruby version fails.
解决方案: Use ruby_sysinit() instead of NtInitialize(). (Tomas Volf,
closes #1382)
相关文件: src/if_ruby.c

Patch 8.0.0188 (after 8.0.0182)
问题: Using NOT_VALID for redraw_later() to update the cursor
line/column highlighting is not efficient.
解决方案: Call validate_cursor() when 'cul' or 'cuc' is set.
相关文件: src/move.c

Patch 8.0.0189
问题: There are no tests for the :profile command.
解决方案: Add tests. (Dominique Pelle, closes #1383)
相关文件: src/Makefile, src/testdir/Make_all.mak,
src/testdir/test_profile.vim

Patch 8.0.0190
问题: Detecting duplicate tags uses a slow linear search.
解决方案: Use a much faster hash table solution. (James McCoy, closes #1046)
But don't add hi_keylen, it makes hash tables 50% bigger.
相关文件: src/tag.c

Patch 8.0.0191 (after 8.0.0187)
问题: Some systems do not have ruby_sysinit(), causing the build to
fail.
解决方案: Clean up how ruby_sysinit() and NtInitialize() are used. (Taro
Muraoka)
相关文件: src/if_ruby.c

Patch 8.0.0192 (after 8.0.0190)
问题: Build fails with tiny features.
解决方案: Change #ifdef for hash_clear(). Avoid warning for unused
argument.
相关文件: src/hashtab.c, src/if_cscope.c

Patch 8.0.0193 (after 8.0.0188)
问题: Accidentally removed #ifdef.
解决方案: Put it back. (Masanori Misono)
相关文件: src/move.c

Patch 8.0.0194 (after 8.0.0189)
问题: Profile tests fails if total and self time are equal.
解决方案: Make one time optional.
相关文件: src/testdir/test_profile.vim

Patch 8.0.0195 (after 8.0.0190)
问题: Jumping to a tag that is a static item in the current file fails.
(Kazunobu Kuriyama)
解决方案: Make sure the first byte of the tag key is not NUL. (Suggested by
James McCoy, closes #1387)

相关文件: src/tag.c, src/testdir/test_tagjump.vim

Patch 8.0.0196 (after 8.0.0194)
问题: The test for :profile is slow and does not work on MS-Windows.
解决方案: Use the "-es" argument. (Dominique Pelle) Swap single and double quotes for system()
相关文件: src/testdir/test_profile.vim

Patch 8.0.0197
问题: On MS-Windows the system() test skips a few parts.
解决方案: Swap single and double quotes for the command.
相关文件: src/testdir/test_system.vim

Patch 8.0.0198
问题: Some syntax arguments take effect even after "if 0". (Taylor Venable)
解决方案: Properly skip the syntax statements. Make "syn case" and "syn conceal" report the current state. Fix that "syn clear" didn't reset the conceal flag. Add tests for :syntax skipping properly.
相关文件: src/syntax.c, src/testdir/test_syntax.vim

Patch 8.0.0199
问题: Warning for an unused parameter when the libcall feature is disabled. Warning for a function type cast when compiling with -pedantic.
解决方案: Add UNUSED. Use a different type cast. (Damien Molinier)
相关文件: src/evalfunc.c, src/os_unix.c

Patch 8.0.0200
问题: Some syntax arguments are not tested.
解决方案: Add more syntax command tests.
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0201
问题: When completing a group name for a highlight or syntax command cleared groups are included.
解决方案: Skip groups that have been cleared.
相关文件: src/syntax.c, src/testdir/test_syntax.vim

Patch 8.0.0202
问题: No test for invalid syntax group name.
解决方案: Add a test for group name error and warning.
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0203
问题: Order of complication flags is sometimes wrong.
解决方案: Put interface-specific flags before ALL_CFLAGS. (idea by Yousong Zhou, closes #1100)
相关文件: src/Makefile

Patch 8.0.0204
问题: Compiler warns for uninitialized variable. (Tony Mechelynck)
解决方案: When skipping set "id" to -1.
相关文件: src/syntax.c

Patch 8.0.0205
问题: After :undojoin some commands don't work properly, such as :redo.
(Matthew Malcomson)
解决方案: Don't set curbuf->b_u_curhead. (closes #1390)
相关文件: src/undo.c, src/testdir/test_undo.vim

Patch 8.0.0206
问题: Test coverage for :retab insufficient.
解决方案: Add test for :retab. (Dominique Pelle, closes #1391)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_retab.vim

Patch 8.0.0207
问题: Leaking file descriptor when system() cannot find the buffer.
(Coverity)
解决方案: Close the file descriptor. (Dominique Pelle, closes #1398)
相关文件: src/evalfunc.c

Patch 8.0.0208
问题: Internally used commands for CTRL-Z and mouse click end up in
history. (Matthew Malcomson)
解决方案: Use do_cmdline_cmd() instead of stuffing them in the readahead
buffer. (James McCoy, closes #1395)
相关文件: src/edit.c, src/normal.c

Patch 8.0.0209
问题: When using :substitute with the "c" flag and 'cursorbind' is set
the cursor is not updated in other windows.
解决方案: Call do_check_cursorbind(). (Masanori Misono)
相关文件: src/ex_cmds.c

Patch 8.0.0210
问题: Vim does not support bracketed paste, as implemented by xterm and
other terminals.
解决方案: Add t_BE, t_BD, t_PS and t_PE.
相关文件: src/term.c, src/term.h, src/option.c, src/misc2.c, src/keymap.h,
src/edit.c, src/normal.c, src/evalfunc.c, src/getchar.c,
src/vim.h, src/proto/edit.pro, runtime/doc/term.txt

Patch 8.0.0211 (after 8.0.0210)
问题: Build fails if the multi-byte feature is disabled.
解决方案: Change #ifdef around ins_char_bytes.
相关文件: src/misc1.c

Patch 8.0.0212
问题: The buffer used to store a key name theoretically could be too
small. (Coverity)
解决方案: Count all possible modifier characters. Add a check for the
length just in case.
相关文件: src/keymap.h, src/misc2.c

Patch 8.0.0213
问题: The Netbeans "specialKeys" command does not check if the argument
fits in the buffer. (Coverity)

解决方案: Add a length check.
相关文件: src/netbeans.c

Patch 8.0.0214
问题: Leaking memory when syntax cluster id is unknown. (Coverity)
解决方案: Free the memory.
相关文件: src/syntax.c

Patch 8.0.0215
问题: When a Cscope line contains CTRL-L a NULL pointer may be used.
(Coverity)
解决方案: Don't check for an emacs tag in a cscope line.
相关文件: src/tag.c

Patch 8.0.0216
问题: When decoding JSON with a JS style object the JSON test may use a
NULL pointer. (Coverity)
解决方案: Check for a NULL pointer.
相关文件: src/json.c, src/json_test.c

Patch 8.0.0217 (after 8.0.0215)
问题: Build fails without the cscope feature.
解决方案: Add #ifdef.
相关文件: src/tag.c

Patch 8.0.0218
问题: No command line completion for :cexpr, :cgetexpr, :caddexpr, etc.
解决方案: Make completion work. (Yegappan Lakshmanan) Add a test.
相关文件: src/ex_docmd.c, src/testdir/test_cmdline.vim

Patch 8.0.0219
问题: Ubsan reports errors for integer overflow.
解决方案: Define macros for minimum and maximum values. Select an
expression based on the value. (Mike Williams)
相关文件: src/charset.c, src/eval.c, src/evalfunc.c, src/structs.h,
src/testdir/test_viml.vim

Patch 8.0.0220
问题: Completion for :match does not show "none" and other missing
highlight names.
解决方案: Skip over cleared entries before checking the index to be at the
end.
相关文件: src/syntax.c, src/testdir/test_cmdline.vim

Patch 8.0.0221
问题: Checking if PROTO is defined inside a function has no effect.
解决方案: Remove the check for PROTO. (Hirohito Higashi)
相关文件: src/misc1.c

Patch 8.0.0222
问题: When a multi-byte character ends in a zero byte, putting blockwise
text puts it before the character instead of after it.
解决方案: Use int instead of char for the character under the cursor.
(Luchr, closes #1403) Add a test.

相关文件: src/ops.c, src/testdir/test_put.vim, src/Makefile,
           src/testdir/test_alot.vim

Patch 8.0.0223
问题: Coverity gets confused by the flags passed to find_tags() and
           warns about uninitialized variable.
解决方案: Disallow using cscope and help tags at the same time.
相关文件: src/tag.c

Patch 8.0.0224
问题: When 'fileformats' is changed in a BufReadPre auto command, it
           does not take effect in readfile(). (Gary Johnson)
解决方案: Check the value of 'fileformats' after executing auto commands.
           (Christian Brabandt)
相关文件: src/fileio.c, src/testdir/test_fileformat.vim

Patch 8.0.0225
问题: When a block is visually selected and put is used on the end of
           the selection only one line is changed.
解决方案: Check for the end properly. (Christian Brabandt, neovim issue
           5781)
相关文件: src/ops.c, src/testdir/test_put.vim

Patch 8.0.0226
问题: The test for patch 8.0.0224 misses the CR characters and passes
           even without the fix. (Christian Brabandt)
解决方案: Use double quotes and \<CR>.
相关文件: src/testdir/test_fileformat.vim

Patch 8.0.0227
问题: Crash when 'fileformat' is forced to "dos" and the first line in
           the file is empty and does not have a CR character.
解决方案: Don't check for CR before the start of the buffer.
相关文件: src/fileio.c, src/testdir/test_fileformat.vim

Patch 8.0.0228 (after 8.0.0210)
问题: When pasting test in an xterm on the command line it is surrounded
           by <PasteStart> and <PasteEnd>. (Johannes Kaltenbach)
解决方案: Add missing changes.
相关文件: src/ex_getln.c, src/term.c

Patch 8.0.0229 (after 8.0.0179)
问题: When freeing a buffer the local value of the 'formatprg' option is
           not cleared.
解决方案: Add missing change.
相关文件: src/buffer.c

Patch 8.0.0230 (after 8.0.0210)
问题: When using bracketed paste line breaks are not respected.
解决方案: Turn CR characters into a line break if the text is being
           inserted. (closes #1404)
相关文件: src/edit.c

Patch 8.0.0231

问题:　　There are no tests for bracketed paste mode.
解决方案:　Add a test. Fix repeating with "normal .".
相关文件:　src/edit.c, src/testdir/test_paste.vim, src/Makefile,
　　src/testdir/Make_all.mak

Patch 8.0.0232
问题:　　Pasting in Insert mode does not work when bracketed paste is used
　　and 'esckeys' is off.
解决方案:　When 'esckeys' is off disable bracketed paste in Insert mode.
相关文件:　src/edit.c

Patch 8.0.0233 (after 8.0.0231)
问题:　　The paste test fails if the GUI is being used.
解决方案:　Skip the test in the GUI.
相关文件:　src/testdir/test_paste.vim

Patch 8.0.0234 (after 8.0.0225)
问题:　　When several lines are visually selected and one of them is short,
　　using put may cause a crash. (Axel Bender)
解决方案:　Check for a short line. (Christian Brabandt)
相关文件:　src/ops.c, src/testdir/test_put.vim

Patch 8.0.0235
问题:　　Memory leak detected when running tests for diff mode.
解决方案:　Free p_extra_free.
相关文件:　src/screen.c

Patch 8.0.0236 (after 8.0.0234)
问题:　　Gcc complains that a variable may be used uninitialized. Confusion
　　between variable and label name. (John Marriott)
解决方案:　Initialize it. Rename end to end_lnum.
相关文件:　src/ops.c

Patch 8.0.0237
问题:　　When setting wildoptions=tagfile the completion context is not set
　　correctly. (desjardins)
解决方案:　Check for EXPAND_TAGS_LISTFILES. (Christian Brabandt, closes #1399)
相关文件:　src/ex_getln.c, src/testdir/test_cmdline.vim

Patch 8.0.0238
问题:　　When using bracketed paste autoindent causes indent to be
　　increased.
解决方案:　Disable 'ai' and set 'paste' temporarily. (Ken Takata)
相关文件:　src/edit.c, src/testdir/test_paste.vim

Patch 8.0.0239
问题:　　The address sanitizer sometimes finds errors, but it needs to be
　　run manually.
解决方案:　Add an environment to Travis with clang and the address sanitizer.
　　(Christian Brabandt) Also include changes only on github.
相关文件:　.travis.yml

Patch 8.0.0240 (after 8.0.0239)
问题:　　The clang build on CI fails with one configuration.

解决方案: Redo a previous patch that was accidentally reverted.
相关文件: .travis.yml

Patch 8.0.0241
问题: Vim defines a mch_memmove() function but it doesn't work, thus is always unused.
解决方案: Remove the mch_memmove implementation. (suggested by Dominique Pelle)
相关文件: src/os_unix.h, src/misc2.c, src/vim.h

Patch 8.0.0242
问题: Completion of user defined functions is not covered by tests.
解决方案: Add tests. Also test various errors of user-defined commands. (Dominique Pelle, closes #1413)
相关文件: src/testdir/test_usercommands.vim

Patch 8.0.0243
问题: When making a character lower case with tolower() changes the byte count, it is not made lower case.
解决方案: Add strlow_save(). (Dominique Pelle, closes #1406)
相关文件: src/evalfunc.c, src/misc2.c, src/proto/misc2.pro, src/testdir/test_functions.vim

Patch 8.0.0244
问题: When the user sets t_BE empty after startup to disable bracketed paste, this has no direct effect.
解决方案: When t_BE is made empty write t_BD. When t_BE is made non-empty write the new value.
相关文件: src/option.c

Patch 8.0.0245
问题: The generated zh_CN.cp936.po message file is not encoded properly.
解决方案: Instead of using zh_CN.po as input, use zh_CN.UTF-8.po.
相关文件: src/po/Makefile

Patch 8.0.0246
问题: Compiler warnings for int to pointer conversion.
解决方案: Fix macro for mch_memmove(). (John Marriott)
相关文件: src/vim.h

Patch 8.0.0247
问题: Under some circumstances, one needs to type Ctrl-N or Ctrl-P twice to have a menu entry selected. (Lifepillar)
解决方案: call ins_compl_free(). (Christian Brabandt, closes #1411)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0248
问题: vim_strcat() cannot handle overlapping arguments.
解决方案: Use mch_memmove() instead of strcpy(). (Justin M. Keyes, closes #1415)
相关文件: src/misc2.c

Patch 8.0.0249
问题: When two submits happen quick after each other, the tests for the

first one may error out.
解决方案: Use a git depth of 10 instead of 1. (Christian Brabandt)
相关文件: .travis.yml

Patch 8.0.0250
问题: When virtcol() gets a column that is not the first byte of a multi-byte character the result is unpredictable. (Christian Ludwig)
解决方案: Correct the column to the first byte of a multi-byte character. Change the utf-8 test to new style.
相关文件: src/charset.c, src/testdir/test_utf8.in, src/testdir/test_utf8.ok, src/testdir/test_utf8.vim, src/Makefile, src/testdir/Make_all.mak, src/testdir/test_alot_utf8.vim

Patch 8.0.0251
问题: It is not so easy to write a script that works with both Python 2 and Python 3, even when the Python code works with both.
解决方案: Add 'pyxversion', :pyx, etc. (Marc Weber, Ken Takata)
相关文件: Filelist, runtime/doc/eval.txt, runtime/doc/if_pyth.txt, runtime/doc/index.txt, runtime/doc/options.txt, runtime/optwin.vim, runtime/doc/quickref.txt, runtime/doc/usr_41.txt, src/Makefile, src/evalfunc.c, src/ex_cmds.h, src/ex_cmds2.c, src/ex_docmd.c, src/if_python.c, src/if_python3.c, src/option.c, src/option.h, src/proto/ex_cmds2.pro, src/testdir/Make_all.mak, src/testdir/pyxfile/py2_magic.py, src/testdir/pyxfile/py2_shebang.py, src/testdir/pyxfile/py3_magic.py, src/testdir/pyxfile/py3_shebang.py, src/testdir/pyxfile/pyx.py, src/testdir/test_pyx2.vim, src/testdir/test_pyx3.vim src/userfunc.c

Patch 8.0.0252
问题: Characters below 256 that are not one byte are not always recognized as word characters.
解决方案: Make vim_iswordc() and vim_iswordp() work the same way. Add a test for this. (Ozaki Kiichi)
相关文件: src/Makefile, src/charset.c, src/kword_test.c, src/mbyte.c, src/proto/mbyte.pro

Patch 8.0.0253
问题: When creating a session when 'winminheight' is 2 or larger and loading that session gives an error.
解决方案: Also set 'winminheight' before setting 'winheight' to 1. (Rafael Bodill, neovim #5717)
相关文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.0.0254
问题: When using an assert function one can either specify a message or get a message about what failed, not both.
解决方案: Concatenate the error with the message.
相关文件: src/eval.c, src/testdir/test_assert.vim

Patch 8.0.0255

问题: When calling setpos() with a buffer argument it often is ignored.
            (Matthew Malcomson)
解决方案: Make the buffer argument work for all marks local to a buffer.
            (neovim #5713) Add more tests.
相关文件: src/mark.c, src/testdir/test_marks.vim, runtime/doc/eval.txt

Patch 8.0.0256 (after 8.0.0255)
问题: Tests fail because some changes were not included.
解决方案: Add changes to evalfunc.c
相关文件: src/evalfunc.c

Patch 8.0.0257 (after 8.0.0252)
问题: The keyword test file is not included in the archive.
解决方案: Update the list of files.
相关文件: Filelist

Patch 8.0.0258 (after 8.0.0253)
问题: mksession test leaves file behind.
解决方案: Delete the file. Rename files to start with "X".
相关文件: src/testdir/test_mksession.vim

Patch 8.0.0259
问题: Tab commands do not handle count correctly. (Ken Hamada)
解决方案: Add ADDR_TABS_RELATIVE. (Hirohito Higashi)
相关文件: runtime/doc/tabpage.txt, src/ex_cmds.h, src/ex_docmd.c,
            src/testdir/test_tabpage.vim

Patch 8.0.0260
问题: Build fails with tiny features.
解决方案: Move get_tabpage_arg() inside #ifdef.
相关文件: src/ex_docmd.c

Patch 8.0.0261
问题: Not enough test coverage for eval functions.
解决方案: Add more tests. (Dominique Pelle, closes #1420)
相关文件: src/testdir/test_functions.vim

Patch 8.0.0262
问题: Farsi support is barely tested.
解决方案: Add more tests for Farsi. Clean up the code.
相关文件: src/edit.c, src/farsi.c, src/testdir/test_farsi.vim

Patch 8.0.0263
问题: Farsi support is not tested enough.
解决方案: Add more tests for Farsi. Clean up the code.
相关文件: src/farsi.c, src/testdir/test_farsi.vim

Patch 8.0.0264
问题: Memory error reported by ubsan, probably for using the string
            returned by execute().
解决方案: NUL terminate the result of execute().
相关文件: src/evalfunc.c

Patch 8.0.0265

问题: May get ml_get error when :pydo deletes lines or switches to
            another buffer. (Nikolai Pavlov, issue #1421)
解决方案: Check the buffer and line every time.
相关文件: src/if_py_both.h, src/testdir/test_python2.vim,
            src/testdir/test_python3.vim, src/Makefile,
            src/testdir/Make_all.mak

Patch 8.0.0266
问题: Compiler warning for using uninitialized variable.
解决方案: Set tab_number also when there is an error.
相关文件: src/ex_docmd.c

Patch 8.0.0267
问题: A channel test sometimes fails on Mac.
解决方案: Add the test to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0268
问题: May get ml_get error when :luado deletes lines or switches to
            another buffer. (Nikolai Pavlov, issue #1421)
解决方案: Check the buffer and line every time.
相关文件: src/if_lua.c, src/testdir/test_lua.vim, src/Makefile,
            src/testdir/Make_all.mak

Patch 8.0.0269
问题: May get ml_get error when :perldo deletes lines or switches to
            another buffer. (Nikolai Pavlov, issue #1421)
解决方案: Check the buffer and line every time.
相关文件: src/if_perl.xs, src/testdir/test_perl.vim

Patch 8.0.0270
问题: May get ml_get error when :rubydo deletes lines or switches to
            another buffer. (Nikolai Pavlov, issue #1421)
解决方案: Check the buffer and line every time.
相关文件: src/if_ruby.c, src/testdir/test_ruby.vim

Patch 8.0.0271
问题: May get ml_get error when :tcldo deletes lines or switches to
            another buffer. (Nikolai Pavlov, closes #1421)
解决方案: Check the buffer and line every time.
相关文件: src/if_tcl.c, src/testdir/test_tcl.vim, src/Makefile,
            src/testdir/Make_all.mak

Patch 8.0.0272
问题: Crash on exit is not detected when running tests.
解决方案: Remove the dash before the command. (Dominique Pelle, closes
            #1425)
相关文件: src/testdir/Makefile

Patch 8.0.0273
问题: Dead code detected by Coverity when not using gnome.
解决方案: Rearrange the #ifdefs to avoid dead code.
相关文件: src/gui_gtk_x11.c

Patch 8.0.0274
问题: When update_single_line() is called recursively, or another screen update happens while it is busy, errors may occur.
解决方案: Check and update updating_screen. (Christian Brabandt)
相关文件: src/screen.c

Patch 8.0.0275
问题: When checking for CTRL-C typed the GUI may detect a screen resize and redraw the screen, causing trouble.
解决方案: Set updating_screen in ui_breakcheck().
相关文件: src/ui.c

Patch 8.0.0276
问题: Checking for FEAT_GUI_GNOME inside GTK 3 code is unnecessary.
解决方案: Remove the #ifdef. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 8.0.0277
问题: The GUI test may trigger fontconfig and take a long time.
解决方案: Set $XDG_CACHE_HOME. (Kazunobu Kuriyama)
相关文件: src/testdir/unix.vim, src/testdir/test_gui.vim

Patch 8.0.0278 (after 8.0.0277)
问题: GUI test fails on MS-Windows.
解决方案: Check that tester_HOME exists.
相关文件: src/testdir/test_gui.vim

Patch 8.0.0279
问题: With MSVC 2015 the dll name is vcruntime140.dll.
解决方案: Check the MSVC version and use the right dll name. (Ken Takata)
相关文件: src/Make_mvc.mak

Patch 8.0.0280
问题: On MS-Windows setting an environment variable with multi-byte strings does not work well.
解决方案: Use wputenv when possible. (Taro Muraoka, Ken Takata)
相关文件: src/misc1.c, src/os_win32.c, src/os_win32.h, src/proto/os_win32.pro, src/vim.h

Patch 8.0.0281
问题: MS-Windows files are still using ARGSUSED while most other files have UNUSED.
解决方案: Change ARGSUSED to UNUSED or delete it.
相关文件: src/os_win32.c, src/gui_w32.c, src/os_mswin.c, src/os_w32exe.c, src/winclip.c

Patch 8.0.0282
问题: When doing a Visual selection and using "I" to go to insert mode, CTRL-O needs to be used twice to go to Normal mode. (Coacher)
解决方案: Check for the return value of edit(). (Christian Brabandt, closes #1290)
相关文件: src/normal.c, src/ops.c

Patch 8.0.0283

问题: The return value of mode() does not indicate that completion is active in Replace and Insert mode. (Zhen-Huan (Kenny) Hu)
解决方案: Add "c" or "x" for two kinds of completion. (Yegappan Lakshmanan, closes #1397) Test some more modes.
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_functions.vim, src/testdir/test_mapping.vim

Patch 8.0.0284
问题: The Test_collapse_buffers() test failed once, looks like it is flaky.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0285 (after 8.0.0277)
问题: Tests fail with tiny build on Unix.
解决方案: Only set g:tester_HOME when build with the +eval feature.
相关文件: src/testdir/unix.vim

Patch 8.0.0286
问题: When concealing is active and the screen is resized in the GUI it is not immediately redrawn.
解决方案: Use update_prepare() and update_finish() from update_single_line().
相关文件: src/screen.c

Patch 8.0.0287
问题: Cannot access the arguments of the current function in debug mode. (Luc Hermitte)
解决方案: use get_funccal(). (LemonBoy, closes #1432, closes #1352)
相关文件: src/userfunc.c

Patch 8.0.0288 (after 8.0.0284)
问题: Errors reported while running tests.
解决方案: Put comma in the right place.
相关文件: src/testdir/runtest.vim

Patch 8.0.0289
问题: No test for "ga" and :ascii.
解决方案: Add a test. (Dominique Pelle, closes #1429)
相关文件: src/Makefile, src/testdir/test_alot.vim, src/testdir/test_ga.vim

Patch 8.0.0290
问题: If a wide character doesn't fit at the end of the screen line, and the line doesn't fit on the screen, then the cursor position may be wrong. (anliting)
解决方案: Don't skip over wide character. (Christian Brabandt, closes #1408)
相关文件: src/screen.c

Patch 8.0.0291 (after 8.0.0282)
问题: Visual block insertion does not insert in all lines.
解决方案: Don't bail out of insert too early. Add a test. (Christian Brabandt, closes #1290)
相关文件: src/ops.c, src/testdir/test_visual.vim

Patch 8.0.0292
问题:　　The stat test is a bit slow.
解决方案:　Remove a couple of sleep comments and reduce another.
相关文件:　src/testdir/test_stat.vim

Patch 8.0.0293
问题:　　Some tests have a one or three second wait.
解决方案:　Reset the 'showmode' option. Use a test time of one to disable
　　sleep after an error or warning message.
相关文件:　src/misc1.c, src/testdir/runtest.vim, src/testdir/test_normal.vim

Patch 8.0.0294
问题:　　Argument list is not stored correctly in a session file.
　　(lgpasquale)
解决方案:　Use "$argadd" instead of "argadd". (closes #1434)
相关文件:　src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.0.0295 (after 8.0.0293)
问题:　　test_viml hangs.
解决方案:　Put resetting 'more' before sourcing the script.
相关文件:　src/testdir/runtest.vim

Patch 8.0.0296
问题:　　Bracketed paste can only append, not insert.
解决方案:　When the cursor is in the first column insert the text.
相关文件:　src/normal.c, src/testdir/test_paste.vim, runtime/doc/term.txt

Patch 8.0.0297
问题:　　Double free on exit when using a closure. (James McCoy)
解决方案:　Split free_al_functions in two parts. (closes #1428)
相关文件:　src/userfunc.c, src/structs.h

Patch 8.0.0298
问题:　　Ex command range with repeated search does not work. (Bruce
　　DeVisser)
解决方案:　Skip over \/, \? and \&.
相关文件:　src/ex_docmd.c, src/testdir/test_cmdline.vim

Patch 8.0.0299
问题:　　When the GUI window is resized Vim does not always take over the
　　new size. (Luchr)
解决方案:　Reset new_p_guifont in gui_resize_shell(). Call
　　gui_may_resize_shell() in the main loop.
相关文件:　src/main.c, src/gui.c

Patch 8.0.0300
问题:　　Cannot stop diffing hidden buffers. (Daniel Hahler)
解决方案:　When using :diffoff! make the whole list if diffed buffers empty.
　　(closes #736)
相关文件:　src/diff.c, src/testdir/test_diffmode.vim

Patch 8.0.0301
问题:　　No tests for ":set completion" and various errors of the :set
　　command.

解决方案: Add more :set tests. (Dominique Pelle, closes #1440)
相关文件: src/testdir/test_options.vim

Patch 8.0.0302
问题: Cannot set terminal key codes with :let.
解决方案: Make it work.
相关文件: src/option.c, src/testdir/test_assign.vim

Patch 8.0.0303
问题: Bracketed paste does not work in Visual mode.
解决方案: Delete the text before pasting
相关文件: src/normal.c, src/ops.c, src/proto/ops.pro,
src/testdir/test_paste.vim

Patch 8.0.0304 (after 8.0.0302)
问题: Assign test fails in the GUI.
解决方案: Skip the test for setting t_k1.
相关文件: src/testdir/test_assign.vim

Patch 8.0.0305
问题: Invalid memory access when option has duplicate flag.
解决方案: Correct pointer computation. (Dominique Pelle, closes #1442)
相关文件: src/option.c, src/testdir/test_options.vim

Patch 8.0.0306
问题: mode() not sufficiently tested.
解决方案: Add more tests. (Yegappan Lakshmanan)
相关文件: src/testdir/test_functions.vim

Patch 8.0.0307
问题: Asan detects a memory error when EXITFREE is defined. (Dominique
Pelle)
解决方案: In getvcol() check for ml_get_buf() returning an empty string.
Also skip adjusting the scroll position. Set "exiting" in
mch_exit() for all systems.
相关文件: src/charset.c, src/window.c, src/os_mswin.c, src/os_win32.c,
src/os_amiga.c

Patch 8.0.0308
问题: When using a symbolic link, the package path will not be inserted
at the right position in 'runtimepath'. (Dugan Chen, Norio Takagi)
解决方案: Resolve symbolic links when finding the right position in
'runtimepath'. (Hirohito Higashi)
相关文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 8.0.0309
问题: Cannot use an empty key in json.
解决方案: Allow for using an empty key.
相关文件: src/json.c, src/testdir/test_json.vim

Patch 8.0.0310
问题: Not enough testing for GUI functionality.
解决方案: Add tests for v:windowid and getwinpos[xy](). (Kazunobu Kuriyama)
相关文件: src/testdir/test_gui.vim

Patch 8.0.0311
问题:　　Linebreak tests are old style.
解决方案:　Turn the tests into new style. Share utility functions. (Ozaki
　　Kiichi, closes #1444)
相关文件:　src/Makefile, src/testdir/Make_all.mak,
　　src/testdir/test_breakindent.vim, src/testdir/test_listlbr.in,
　　src/testdir/test_listlbr.ok, src/testdir/test_listlbr.vim,
　　src/testdir/test_listlbr_utf8.in,
　　src/testdir/test_listlbr_utf8.ok,
　　src/testdir/test_listlbr_utf8.vim, src/testdir/view_util.vim

Patch 8.0.0312
问题:　　When a json message arrives in pieces, the start is dropped and
　　the decoding fails.
解决方案:　Do not drop the start when it is still needed. (Kay Zheng) Add a
　　test. Reset the timeout when something is received.
相关文件:　src/channel.c, src/testdir/test_channel.vim, src/structs.h,
　　src/testdir/test_channel_pipe.py

Patch 8.0.0313 (after 8.0.0310)
问题:　　Not enough testing for GUI functionality.
解决方案:　Add tests for the GUI font. (Kazunobu Kuriyama)
相关文件:　src/testdir/test_gui.vim

Patch 8.0.0314
问题:　　getcmdtype(), getcmdpos() and getcmdline() are not tested.
解决方案:　Add tests. (Yegappan Lakshmanan)
相关文件:　src/testdir/test_cmdline.vim

Patch 8.0.0315
问题:　　":help :[range]" does not work. (Tony Mechelynck)
解决方案:　Translate to insert a backslash.
相关文件:　src/ex_cmds.c

Patch 8.0.0316
问题:　　":help z?" does not work. (Pavol Juhas)
解决方案:　Remove exception for z?.
相关文件:　src/ex_cmds.c

Patch 8.0.0317
问题:　　No test for setting 'guifont'.
解决方案:　Add a test for X11 GUIs. (Kazunobu Kuriyama)
相关文件:　src/testdir/test_gui.vim

Patch 8.0.0318
问题:　　Small mistake in 7x13 font name.
解决方案:　Use ISO 8859-1 name instead of 10646-1. (Kazunobu Kuriyama)
相关文件:　src/testdir/test_gui.vim

Patch 8.0.0319
问题:　　Insert mode completion does not respect "start" in 'backspace'.
解决方案:　Check whether backspace can go before where insert started.
　　(Hirohito Higashi)

相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0320
问题: Warning for unused variable with small build.
解决方案: Change #ifdef to exclude FEAT_CMDWIN. (Kazunobu Kuriyama)
相关文件: src/ex_getln.c

Patch 8.0.0321
问题: When using the tiny version trying to load the matchit plugin
gives an error. On MS-Windows some default mappings fail.
解决方案: Add a check if the command used is available. (Christian Brabandt)
相关文件: runtime/mswin.vim, runtime/macros/matchit.vim

Patch 8.0.0322
问题: Possible overflow with spell file where the tree length is
corrupted.
解决方案: Check for an invalid length (suggested by shqking)
相关文件: src/spellfile.c

Patch 8.0.0323
问题: When running the command line tests there is a one second wait.
解决方案: Change an Esc to Ctrl-C. (Yegappan Lakshmanan)
相关文件: src/testdir/test_cmdline.vim

Patch 8.0.0324
问题: Illegal memory access with "1;y".
解决方案: Call check_cursor() instead of check_cursor_lnum(). (Dominique
Pelle, closes #1455)
相关文件: src/ex_docmd.c, src/testdir/test_cmdline.vim

Patch 8.0.0325
问题: Packadd test does not clean up symlink.
解决方案: Delete the link. (Hirohito Higashi)
相关文件: src/testdir/test_packadd.vim

Patch 8.0.0326 (after 8.0.0325)
问题: Packadd test uses wrong directory name.
解决方案: Use the variable name value. (Hirohito Higashi)
相关文件: src/testdir/test_packadd.vim

Patch 8.0.0327
问题: The E11 error message in the command line window is not
translated.
解决方案: use _(). (Hirohito Higashi)
相关文件: src/ex_docmd.c

Patch 8.0.0328
问题: The "zero count" error doesn't have a number. (Hirohito Higashi)
解决方案: Give it a number and be more specific about the error.
相关文件: src/globals.h

Patch 8.0.0329
问题: Xfontset and guifontwide are not tested.
解决方案: Add tests. (Kazunobu Kuriyama)

相关文件: src/testdir/test_gui.vim

Patch 8.0.0330
问题: Illegal memory access after "vapo". (Dominique Pelle)
解决方案: Fix the cursor column.
相关文件: src/search.c, src/testdir/test_visual.vim

Patch 8.0.0331
问题: Restoring help snapshot accesses freed memory. (Dominique Pelle)
解决方案: Don't restore a snapshot when the window closes.
相关文件: src/window.c, src/Makefile, src/testdir/Make_all.mak,
src/testdir/test_help.vim

Patch 8.0.0332
问题: GUI test fails on some systems.
解决方案: Try different language settings. (Kazunobu Kuriyama)
相关文件: src/testdir/test_gui.vim

Patch 8.0.0333
问题: Illegal memory access when 'complete' ends in a backslash.
解决方案: Check for trailing backslash. (Dominique Pelle, closes #1478)
相关文件: src/option.c, src/testdir/test_options.vim

Patch 8.0.0334
问题: Can't access b:changedtick from a dict reference.
解决方案: Make changedtick a member of the b: dict. (inspired by neovim
#6112)
相关文件: src/structs.h, src/buffer.c, src/edit.c, src/eval.c,
src/evalfunc.c, src/ex_docmd.c, src/main.c, src/globals.h,
src/fileio.c, src/memline.c, src/misc1.c, src/syntax.c,
src/proto/eval.pro, src/testdir/test_changedtick.vim,
src/Makefile, src/testdir/test_alot.vim, src/testdir/test91.in,
src/testdir/test91.ok, src/testdir/test_functions.vim

Patch 8.0.0335 (after 8.0.0335)
问题: Functions test fails.
解决方案: Use the right buffer number.
相关文件: src/testdir/test_functions.vim

Patch 8.0.0336
问题: Flags of :substitute not sufficiently tested.
解决方案: Test up to two letter flag combinations. (James McCoy, closes
#1479)
相关文件: src/testdir/test_substitute.vim

Patch 8.0.0337
问题: Invalid memory access in :recover command.
解决方案: Avoid access before directory name. (Dominique Pelle,
closes #1488)
相关文件: src/Makefile, src/memline.c, src/testdir/test_alot.vim,
src/testdir/test_recover.vim

Patch 8.0.0338 (after 8.0.0337)
问题: :recover test fails on MS-Windows.

解决方案: Use non-existing directory on MS-Windows.
相关文件: src/testdir/test_recover.vim

Patch 8.0.0339
问题: Illegal memory access with vi'
解决方案: For quoted text objects bail out if the Visual area spans more than one line.
相关文件: src/search.c, src/testdir/test_visual.vim

Patch 8.0.0340
问题: Not checking return value of dict_add(). (Coverity)
解决方案: Handle a failure.
相关文件: src/buffer.c

Patch 8.0.0341
问题: When using complete() and typing a character undo is saved after the character was inserted. (Shougo)
解决方案: Save for undo before inserting the character.
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0342
问题: Double free when compiled with EXITFREE and setting 'ttytype'.
解决方案: Avoid setting P_ALLOCED on 'ttytype'. (Dominique Pelle, closes #1461)
相关文件: src/option.c, src/testdir/test_options.vim

Patch 8.0.0343
问题: b:changedtick can be unlocked, even though it has no effect. (Nikolai Pavlov)
解决方案: Add a check and error E940. (closes #1496)
相关文件: src/eval.c, src/testdir/test_changedtick.vim, runtime/doc/eval.txt

Patch 8.0.0344
问题: Unlet command leaks memory. (Nikolai Pavlov)
解决方案: Free the memory on error. (closes #1497)
相关文件: src/eval.c, src/testdir/test_unlet.vim

Patch 8.0.0345
问题: islocked('d.changedtick') does not work.
解决方案: Make it work.
相关文件: src/buffer.c, src/eval.c, src/evalfunc.c, src/vim.h, src/testdir/test_changedtick.vim,

Patch 8.0.0346
问题: Vim relies on limits.h to be included indirectly, but on Solaris 9 it may not be. (Ben Fritz)
解决方案: Always include limits.h.
相关文件: src/os_unixx.h, src/vim.h

Patch 8.0.0347
问题: When using CTRL-X CTRL-U inside a comment, the use of the comment leader may not work. (Klement)
解决方案: Save and restore did_ai. (Christian Brabandt, closes #1494)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0348
问题: When building with a shadow directory on macOS lacks the
+clipboard feature.
解决方案: Link *.m files, specifically os_macosx.m. (Kazunobu Kuriyama)
相关文件: src/Makefile

Patch 8.0.0349
问题: Redrawing errors with GTK 3.
解决方案: When updating, first clear all rectangles and then draw them.
(Kazunobu Kuriyama, Christian Ludwig, closes #848)
相关文件: src/gui_gtk_x11.c

Patch 8.0.0350
问题: Not enough test coverage for Perl.
解决方案: Add more Perl tests. (Dominique Pelle, closes #1500)
相关文件: src/testdir/test_perl.vim

Patch 8.0.0351
问题: No test for concatenating an empty string that results from out of
bounds indexing.
解决方案: Add a simple test.
相关文件: src/testdir/test_expr.vim

Patch 8.0.0352
问题: The condition for when a typval needs to be cleared is too
complicated.
解决方案: Init the type to VAR_UNKNOWN and always clear it.
相关文件: src/eval.c

Patch 8.0.0353
问题: If [RO] in the status line is translated to a longer string, it is
truncated to 4 bytes.
解决方案: Skip over the resulting string. (Jente Hidskes, closes #1499)
相关文件: src/screen.c

Patch 8.0.0354
问题: Test to check that setting termcap key fails sometimes.
解决方案: Check for "t_k1" to exist. (Christian Brabandt, closes #1459)
相关文件: src/testdir/test_assign.vim

Patch 8.0.0355
问题: Using uninitialized memory when 'isfname' is empty.
解决方案: Don't call getpwnam() without an argument. (Dominique Pelle,
closes #1464)
相关文件: src/misc1.c, src/testdir/test_options.vim

Patch 8.0.0356 (after 8.0.0342)
问题: Leaking memory when setting 'ttytype'.
解决方案: Get free_oldval from the right option entry.
相关文件: src/option.c

Patch 8.0.0357
问题: Crash when setting 'guicursor' to weird value.

解决方案: Avoid negative size. (Dominique Pelle, closes #1465)
相关文件: src/misc2.c, src/testdir/test_options.vim

Patch 8.0.0358
问题: Invalid memory access in C-indent code.
解决方案: Don't go over end of empty line. (Dominique Pelle, closes #1492)
相关文件: src/edit.c, src/testdir/test_options.vim

Patch 8.0.0359
问题: 'number' and 'relativenumber' are not properly tested.
解决方案: Add tests, change old style to new style tests. (Ozaki Kiichi,
closes #1447)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test89.in, src/testdir/test89.ok,
src/testdir/test_alot.vim, src/testdir/test_findfile.vim,
src/testdir/test_number.vim

Patch 8.0.0360
问题: Sometimes VimL is used, which is confusing.
解决方案: Consistently use "Vim script". (Hirohito Higashi)
相关文件: runtime/doc/if_mzsch.txt, runtime/doc/if_pyth.txt,
runtime/doc/syntax.txt, runtime/doc/usr_02.txt,
runtime/doc/version7.txt, src/Makefile, src/eval.c,
src/ex_getln.c, src/if_py_both.h, src/if_xcmdsrv.c,
src/testdir/Make_all.mak, src/testdir/runtest.vim,
src/testdir/test49.vim, src/testdir/test_vimscript.vim,
src/testdir/test_viml.vim

Patch 8.0.0361
问题: GUI initialisation is not sufficiently tested.
解决方案: Add the gui_init test. (Kazunobu Kuriyama)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Makefile,
src/testdir/gui_init.vim, src/testdir/setup_gui.vim,
src/testdir/test_gui.vim, src/testdir/test_gui_init.vim, Filelist

Patch 8.0.0362 (after 8.0.0361)
问题: Tests fail on MS-Windows.
解决方案: Use $*.vim instead of $<.
相关文件: src/testdir/Make_dos.mak

Patch 8.0.0363
问题: Travis is too slow to keep up with patches.
解决方案: Increase git depth to 20
相关文件: .travis.yml

Patch 8.0.0364
问题:]s does not move cursor with two spell errors in one line. (Manuel
Ortega)
解决方案: Don't stop search immediately when wrapped, search the line first.
(Ken Takata) Add a test.
相关文件: src/spell.c, src/Makefile, src/testdir/test_spell.vim,
src/testdir/Make_all.mak

Patch 8.0.0365
问题: Might free a dict item that wasn't allocated.
解决方案: Call dictitem_free(). (Nikolai Pavlov) Use this for b:changedtick.
相关文件: src/dict.c, src/structs.h, src/buffer.c, src/edit.c, src/evalfunc.c, src/ex_docmd.c, src/fileio.c, src/main.c, src/memline.c, src/misc1.c, src/syntax.c

Patch 8.0.0366 (after 8.0.0365)
问题: Build fails with tiny features.
解决方案: Add #ifdef.
相关文件: src/buffer.c

Patch 8.0.0367
问题: If configure defines _LARGE_FILES some include files are included before it is defined.
解决方案: Include vim.h first. (Sam Thursfield, closes #1508)
相关文件: src/gui_at_sb.c, src/gui_athena.c, src/gui_motif.c, src/gui_x11.c, src/gui_xmdlg.c

Patch 8.0.0368
问题: Not all options are tested with a range of values.
解决方案: Generate a test script from the source code.
相关文件: Filelist, src/gen_opt_test.vim, src/testdir/test_options.vim, src/Makefile

Patch 8.0.0369 (after 8.0.0368)
问题: The 'balloondelay', 'balooneval' and 'balloonexpr' options are not defined without the +balloon_eval feature. Testing that an option value fails does not work for unsupported options.
解决方案: Make the options defined but not supported. Don't test if setting unsupported options fails.
相关文件: src/option.c, src/gen_opt_test.vim

Patch 8.0.0370
问题: Invalid memory access when setting wildchar empty.
解决方案: Avoid going over the end of the option value. (Dominique Pelle, closes #1509) Make option test check all number options with empty value.
相关文件: src/gen_opt_test.vim, src/option.c, src/testdir/test_options.vim

Patch 8.0.0371 (after 8.0.0365)
问题: Leaking memory when setting v:completed_item.
解决方案: Or the flags instead of setting them.
相关文件: src/eval.c

Patch 8.0.0372
问题: More options are not always defined.
解决方案: Consistently define all possible options.
相关文件: src/option.c, src/testdir/test_expand_dllpath.vim

Patch 8.0.0373
问题: Build fails without +folding.
解决方案: Move misplaced #ifdef.

相关文件: src/option.c

Patch 8.0.0374
问题: Invalid memory access when using :sc in Ex mode. (Dominique Pelle)
解决方案: Avoid the column being negative. Also fix a hang in Ex mode.
相关文件: src/ex_getln.c, src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.0.0375
问题: The "+ register is not tested.
解决方案: Add a test using another Vim instance to change the "+ register.
(Kazunobu Kuriyama)
相关文件: src/testdir/test_gui.vim

Patch 8.0.0376
问题: Size computations in spell file reading are not exactly right.
解决方案: Make "len" a "long" and check with LONG_MAX.
相关文件: src/spellfile.c

Patch 8.0.0377
问题: Possible overflow when reading corrupted undo file.
解决方案: Check if allocated size is not too big. (King)
相关文件: src/undo.c

Patch 8.0.0378
问题: Another possible overflow when reading corrupted undo file.
解决方案: Check if allocated size is not too big. (King)
相关文件: src/undo.c

Patch 8.0.0379
问题: CTRL-Z and mouse click use CTRL-O unnecessary.
解决方案: Remove stuffing CTRL-O. (James McCoy, closes #1453)
相关文件: src/edit.c, src/normal.c

Patch 8.0.0380
问题: With 'linebreak' set and 'breakat' includes ">" a double-wide
character results in "<<" displayed.
解决方案: Check for the character not to be replaced. (Ozaki Kiichi,
closes #1456)
相关文件: src/screen.c, src/testdir/test_listlbr_utf8.vim

Patch 8.0.0381
问题: Diff mode is not sufficiently tested.
解决方案: Add more diff mode tests. (Dominique Pelle, closes #1515)
相关文件: src/testdir/test_diffmode.vim

Patch 8.0.0382 (after 8.0.0380)
问题: Warning in tiny build for unused variable. (Tony Mechelynck)
解决方案: Add #ifdefs.
相关文件: src/screen.c

Patch 8.0.0383 (after 8.0.0382)
问题: Misplaced #ifdef. (Christ van Willigen)
解决方案: Split assignment.
相关文件: src/screen.c

Patch 8.0.0384
问题: Timer test failed for no apparent reason.
解决方案: Mark the test as flaky.
相关文件: src/testdir/runtest.vim

Patch 8.0.0385
问题: No tests for arabic.
解决方案: Add a first test for arabic. (Dominique Pelle, closes #1518)
相关文件: src/Makefile, src/testdir/Make_all.mak,
            src/testdir/test_arabic.vim

Patch 8.0.0386
问题: Tiny build has a problem with generating the options test.
解决方案: Change the "if" to skip over statements.
相关文件: src/gen_opt_test.vim

Patch 8.0.0387
问题: compiler warnings
解决方案: Add type casts. (Christian Brabandt)
相关文件: src/channel.c, src/memline.c

Patch 8.0.0388
问题: filtering lines through "cat", without changing the line count,
            changes manual folds.
解决方案: Change how marks and folds are adjusted. (Matthew Malcomson, from
            neovim #6194).
相关文件: src/fold.c, src/testdir/test_fold.vim

Patch 8.0.0389
问题: Test for arabic does not check what is displayed.
解决方案: Improve what is asserted. (Dominique Pelle, closes #1523)
            Add a first shaping test.
相关文件: src/testdir/test_arabic.vim

Patch 8.0.0390
问题: When the window scrolls horizontally when the popup menu is
            displayed part of it may not be cleared. (Neovim issue #6184)
解决方案: Remove the menu when the windows scrolled. (closes #1524)
相关文件: src/edit.c

Patch 8.0.0391
问题: Arabic support is verbose and not well tested.
解决方案: Simplify the code. Add more tests.
相关文件: src/arabic.c, src/testdir/test_arabic.vim

Patch 8.0.0392
问题: GUI test fails with Athena and Motif.
解决方案: Add test_ignore_error(). Use it to ignore the "failed to create
            input context" error.
相关文件: src/message.c, src/proto/message.pro, src/evalfunc.c,
            src/testdir/test_gui.vim, runtime/doc/eval.txt

Patch 8.0.0393 (after 8.0.0190)

问题:　　When the same tag appears more than once, the order is
　　unpredictable. (Charles Campbell)
解决方案:　Besides using a dict for finding duplicates, use a grow array for
　　keeping the tags in sequence.
相关文件:　src/tag.c, src/testdir/test_tagjump.vim

Patch 8.0.0394
问题:　　Tabs are not aligned when scrolling horizontally and a Tab doesn't
　　fit. (Axel Bender)
解决方案:　Handle a Tab as a not fitting character. (Christian Brabandt)
　　Also fix that ":redraw" does not scroll horizontally to show the
　　cursor. And fix the test that depended on the old behavior.
相关文件:　src/screen.c, src/ex_docmd.c, src/testdir/test_listlbr.vim,
　　src/testdir/test_listlbr_utf8.vim,
　　src/testdir/test_breakindent.vim

Patch 8.0.0395 (after 8.0.0392)
问题:　　Testing the + register fails with Motif.
解决方案:　Also ignore the "failed to create input context" error in the
　　second gvim. Don't use msg() when it would result in a dialog.
相关文件:　src/message.c, src/testdir/test_gui.vim, src/testdir/setup_gui.vim

Patch 8.0.0396
问题:　　'balloonexpr' only works synchronously.
解决方案:　Add balloon_show(). (Jusufadis Bakamovic, closes #1449)
相关文件:　runtime/doc/eval.txt, src/evalfunc.c, src/os_unix.c,
　　src/os_win32.c

Patch 8.0.0397 (after 8.0.0392)
问题:　　Cannot build with the viminfo feature but without the eval
　　feature.
解决方案:　Adjust #ifdef. (John Marriott)
相关文件:　src/message.c, src/misc2.c

Patch 8.0.0398
问题:　　Illegal memory access with "t".
解决方案:　Use strncmp() instead of memcmp(). (Dominique Pelle, closes #1528)
相关文件:　src/search.c, src/testdir/test_search.vim

Patch 8.0.0399
问题:　　Crash when using balloon_show() when not supported. (Hirohito
　　Higashi)
解决方案:　Check for balloonEval not to be NULL. (Ken Takata)
相关文件:　src/evalfunc.c, src/testdir/test_functions.vim

Patch 8.0.0400
问题:　　Some tests have a one second delay.
解决方案:　Add --not-a-term in RunVim().
相关文件:　src/testdir/shared.vim

Patch 8.0.0401
问题:　　Test fails with missing balloon feature.
解决方案:　Add check for balloon feature.
相关文件:　src/testdir/test_functions.vim

Patch 8.0.0402
问题:　　:map completion does not have <special>. (Dominique Pelle)
解决方案:　Recognize <special> in completion. Add a test.
相关文件:　src/getchar.c, src/testdir/test_cmdline.vim

Patch 8.0.0403
问题:　　GUI tests may fail.
解决方案:　Ignore the E285 error better. (Kazunobu Kuriyama)
相关文件:　src/testdir/test_gui.vim, src/testdir/test_gui_init.vim

Patch 8.0.0404
问题:　　Not enough testing for quickfix.
解决方案:　Add some more tests. (Yegappan Lakshmanan)
相关文件:　src/testdir/test_quickfix.vim

Patch 8.0.0405
问题:　　v:progpath may become invalid after ":cd".
解决方案:　Turn v:progpath into a full path if needed.
相关文件:　src/main.c, src/testdir/test_startup.vim, runtime/doc/eval.txt

Patch 8.0.0406
问题:　　The arabic shaping code is verbose.
解决方案:　Shorten the code without changing the functionality.
相关文件:　src/arabic.c

Patch 8.0.0407 (after 8.0.0388)
问题:　　Filtering folds with marker method not tested.
解决方案:　Also set 'foldmethod' to "marker".
相关文件:　src/testdir/test_fold.vim

Patch 8.0.0408
问题:　　Updating folds does not work properly when inserting a file and a
　　few other situations.
解决方案:　Adjust the way folds are updated. (Matthew Malcomson)
相关文件:　src/fold.c, src/testdir/test_fold.vim

Patch 8.0.0409
问题:　　set_progpath is defined but not always used
解决方案:　Adjust #ifdef.
相关文件:　src/main.c

Patch 8.0.0410
问题:　　Newer gettext/iconv library has extra dll file.
解决方案:　Add the file to the Makefile and nsis script. (Christian Brabandt)
相关文件:　Makefile, nsis/gvim.nsi

Patch 8.0.0411
问题:　　We can't change the case in menu entries, it breaks translations.
解决方案:　Ignore case when looking up a menu translation.
相关文件:　src/menu.c, src/testdir/test_menu.vim

Patch 8.0.0412 (after 8.0.0411)
问题:　　Menu test fails on MS-Windows.

解决方案: Use a menu entry with only ASCII characters.
相关文件: src/testdir/test_menu.vim

Patch 8.0.0413 (after 8.0.0412)
问题: Menu test fails on MS-Windows using gvim.
解决方案: First delete the English menus.
相关文件: src/testdir/test_menu.vim

Patch 8.0.0414
问题: Balloon eval is not tested.
解决方案: Add a few balloon tests. (Kazunobu Kuriyama)
相关文件: src/testdir/test_gui.vim

Patch 8.0.0415 (after 8.0.0414)
问题: Balloon test fails on MS-Windows.
解决方案: Test with 0x7fffffff instead of 0xffffffff.
相关文件: src/testdir/test_gui.vim

Patch 8.0.0416
问题: Setting v:progpath is not quite right.
解决方案: On MS-Windows add the extension. On Unix use the full path for a
           relative directory. (partly by James McCoy, closes #1531)
相关文件: src/main.c, src/os_win32.c, src/os_unix.c

Patch 8.0.0417
问题: Test for the clipboard fails sometimes.
解决方案: Add it to the flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0418
问题: ASAN logs are disabled and don't cause a failure.
解决方案: Enable ASAN logs and fail if not empty. (James McCoy,
           closes #1425)
相关文件: .travis.yml

Patch 8.0.0419
问题: Test for v:progpath fails on MS-Windows.
解决方案: Expand to full path. Also add ".exe" when the path is an absolute
           path.
相关文件: src/os_win32.c, src/main.c

Patch 8.0.0420
问题: When running :make the output may be in the system encoding,
           different from 'encoding'.
解决方案: Add the 'makeencoding' option. (Ken Takata)
相关文件: runtime/doc/options.txt, runtime/doc/quickfix.txt,
           runtime/doc/quickref.txt, src/Makefile, src/buffer.c,
           src/if_cscope.c, src/main.c, src/option.c, src/option.h,
           src/proto/quickfix.pro, src/quickfix.c, src/structs.h,
           src/testdir/Make_all.mak, src/testdir/test_makeencoding.py,
           src/testdir/test_makeencoding.vim

Patch 8.0.0421
问题: Diff mode is displayed wrong when adding a line at the end of a

buffer.
解决方案: Adjust marks in diff mode. (James McCoy, closes #1329)
相关文件: src/misc1.c, src/ops.c, src/testdir/test_diffmode.vim

Patch 8.0.0422
问题: Python test fails with Python 3.6.
解决方案: Convert new exception messages to old ones. (closes #1359)
相关文件: src/testdir/test87.in

Patch 8.0.0423
问题: The effect of adding "#" to 'cinoptions' is not always removed.
(David Briscoe)
解决方案: Reset b_ind_hash_comment. (Christian Brabandt, closes #1475)
相关文件: src/misc1.c, src/Makefile, src/testdir/Make_all.mak,
src/testdir/test_cindent.vim, src/testdir/test3.in

Patch 8.0.0424
问题: Compiler warnings on MS-Windows. (Ajit Thakkar)
解决方案: Add type casts.
相关文件: src/os_win32.c

Patch 8.0.0425
问题: Build errors when building without folding.
解决方案: Add #ifdefs. (John Marriott)
相关文件: src/diff.c, src/edit.c, src/option.c, src/syntax.c

Patch 8.0.0426
问题: Insufficient testing for statusline.
解决方案: Add several tests. (Dominique Pelle, closes #1534)
相关文件: src/testdir/test_statusline.vim

Patch 8.0.0427
问题: 'makeencoding' missing from the options window.
解决方案: Add the entry.
相关文件: runtime/optwin.vim

Patch 8.0.0428
问题: Git and hg see new files after running tests. (Manuel Ortega)
解决方案: Add the generated file to .hgignore (or .gitignore). Delete the
resulting verbose file. (Christian Brabandt) Improve dependency
on opt_test.vim. Reset the 'more' option.
相关文件: .hgignore, src/gen_opt_test.vim, src/testdir/gen_opt_test.vim,
src/Makefile, src/testdir/Make_all.mak, src/testdir/Makefile,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
Filelist

Patch 8.0.0429
问题: Options test does not always test everything.
解决方案: Fix dependency for opt_test.vim. Give a message when opt_test.vim
was not found.
相关文件: src/testdir/test_options.vim, src/testdir/gen_opt_test.vim,
src/testdir/Makefile, src/testdir/Make_all.mak,
src/testdir/Make_dos.mak, src/testdir/Make_ming.mak

Patch 8.0.0430
问题:　　Options test fails or hangs on MS-Windows.
解决方案:　Run it separately instead of part of test_alot. Use "-S" instead
　　of "-u" to run the script. Fix failures.
相关文件:　src/testdir/Make_all.mak, src/testdir/test_alot.vim,
　　src/testdir/Makefile, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/gen_opt_test.vim

Patch 8.0.0431
问题:　　'cinoptions' cannot set indent for extern block.
解决方案:　Add the "E" flag in 'cinoptions'. (Hirohito Higashi)
相关文件:　runtime/doc/indent.txt, src/misc1.c, src/structs.h,
　　src/testdir/test_cindent.vim

Patch 8.0.0432
问题:　　"make shadow" creates an invalid link.
解决方案:　Don't link "*.vim". (Kazunobu Kuriyama)
相关文件:　src/Makefile

Patch 8.0.0433
问题:　　Quite a few beeps when running tests.
解决方案:　Set 'belloff' for these tests. (Christian Brabandt)
相关文件:　src/testdir/test103.in, src/testdir/test14.in,
　　src/testdir/test29.in, src/testdir/test30.in,
　　src/testdir/test32.in, src/testdir/test45.in,
　　src/testdir/test72.in, src/testdir/test73.in,
　　src/testdir/test77.in, src/testdir/test78.in,
　　src/testdir/test85.in, src/testdir/test94.in,
　　src/testdir/test_alot.vim, src/testdir/test_alot_utf8.vim,
　　src/testdir/test_close_count.in, src/testdir/test_cmdline.vim,
　　src/testdir/test_diffmode.vim, src/testdir/test_digraph.vim,
　　src/testdir/test_erasebackword.in, src/testdir/test_normal.vim,
　　src/testdir/test_packadd.vim, src/testdir/test_search.vim,
　　src/testdir/test_textobjects.vim, src/testdir/test_undo.vim,
　　src/testdir/test_usercommands.vim, src/testdir/test_visual.vim

Patch 8.0.0434
问题:　　Clang version not correctly detected.
解决方案:　Adjust the configure script. (Kazunobu Kuriyama)
相关文件:　src/configure.ac, src/auto/configure

Patch 8.0.0435
问题:　　Some functions are not tested.
解决方案:　Add more tests for functions. (Dominique Pelle, closes #1541)
相关文件:　src/testdir/test_functions.vim

Patch 8.0.0436
问题:　　Running the options test sometimes resizes the terminal.
解决方案:　Clear out t_WS.
相关文件:　src/testdir/gen_opt_test.vim

Patch 8.0.0437
问题:　　The packadd test does not create the symlink correctly and does
　　not test the right thing.

解决方案: Create the directory and symlink correctly.
相关文件: src/testdir/test_packadd.vim

Patch 8.0.0438
问题: The fnamemodify test changes 'shell' in a way later tests may not
           be able to use system().
解决方案: Save and restore 'shell'.
相关文件: src/testdir/test_fnamemodify.vim

Patch 8.0.0439
问题: Using ":%argdel" while the argument list is already empty gives an
           error. (Pavol Juhas)
解决方案: Don't give an error. (closes #1546)
相关文件: src/ex_cmds2.c, src/testdir/test_arglist.vim

Patch 8.0.0440
问题: Not enough test coverage in Insert mode.
解决方案: Add lots of tests. Add test_override(). (Christian Brabandt,
           closes #1521)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/edit.c,
           src/evalfunc.c, src/globals.h, src/screen.c,
           src/testdir/Make_all.mak, src/testdir/test_cursor_func.vim,
           src/testdir/test_edit.vim, src/testdir/test_search.vim,
           src/testdir/test_assert.vim, src/Makefile, src/testdir/runtest.vim

Patch 8.0.0441
问题: Dead code in #ifdef.
解决方案: Remove the #ifdef and #else part.
相关文件: src/option.c

Patch 8.0.0442
问题: Patch shell command uses double quotes around the argument, which
           allows for $HOME to be expanded. (Etienne)
解决方案: Use single quotes on Unix. (closes #1543)
相关文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.0.0443
问题: Terminal width is set to 80 in test3.
解决方案: Instead of setting 'columns' set 'wrapmargin' depending on
           'columns.
相关文件: src/testdir/test3.in

Patch 8.0.0444 (after 8.0.0442)
问题: Diffpatch fails when the file name has a quote.
解决方案: Escape the name properly. (zetzei)
相关文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.0.0445
问题: Getpgid is not supported on all systems.
解决方案: Add a configure check.
相关文件: src/configure.ac, src/auto/configure, src/config.h.in,
           src/os_unix.c

Patch 8.0.0446

问题:　　The ";" command does not work after characters with a lower byte
　　that is NUL.
解决方案:　Properly check for not having a previous character. (Hirohito
　　Higashi)
相关文件:　src/Makefile, src/search.c, src/testdir/test_alot_utf8.vim,
　　src/testdir/test_charsearch_utf8.vim

Patch 8.0.0447
问题:　　Getting font name does not work on X11.
解决方案:　Implement gui_mch_get_fontname() for X11. Add more GUI tests.
　　(Kazunobu Kuriyama)
相关文件:　src/gui_x11.c, src/syntax.c, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak, src/testdir/Makefile,
　　src/testdir/gui_init.vim, src/testdir/gui_preinit.vim,
　　src/testdir/test_gui.vim, src/testdir/test_gui_init.vim,
　　Filelist

Patch 8.0.0448
问题:　　Some macros are in lower case, which can be confusing.
解决方案:　Make a few lower case macros upper case.
相关文件:　src/macros.h, src/buffer.c, src/charset.c, src/ops.c, src/diff.c,
　　src/edit.c, src/evalfunc.c, src/ex_cmds.c, src/ex_getln.c,
　　src/fileio.c, src/fold.c, src/gui.c, src/gui_beval.c, src/main.c,
　　src/mark.c, src/misc1.c, src/move.c, src/normal.c,
　　src/option.c, src/popupmnu.c, src/regexp.c, src/screen.c,
　　src/search.c, src/spell.c, src/tag.c, src/ui.c, src/undo.c,
　　src/version.c, src/workshop.c, src/if_perl.xs

Patch 8.0.0449 (after 8.0.0448)
问题:　　Part of fold patch accidentally included.
解决方案:　Revert that part of the patch.
相关文件:　src/ex_cmds.c

Patch 8.0.0450
问题:　　v:progpath is not reliably set.
解决方案:　Read /proc/self/exe if possible. (idea by Michal Grochmal)
　　Also fixes missing #if.
相关文件:　src/main.c, src/config.h.in

Patch 8.0.0451
问题:　　Some macros are in lower case.
解决方案:　Make a few more macros upper case. Avoid lower case macros use an
　　argument twice.
相关文件:　src/macros.h, src/charset.c, src/misc2.c, src/proto/misc2.pro,
　　src/edit.c, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c,
　　src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/fold.c,
　　src/gui.c, src/gui_gtk.c, src/mark.c, src/memline.c, src/mbyte.c,
　　src/menu.c, src/message.c, src/misc1.c, src/ops.c, src/option.c,
　　src/os_amiga.c, src/os_mswin.c, src/os_unix.c, src/os_win32.c,
　　src/popupmnu.c, src/regexp.c, src/regexp_nfa.c, src/screen.c,
　　src/search.c, src/spell.c, src/spellfile.c, src/syntax.c,
　　src/tag.c, src/ui.c, src/undo.c, src/window.c

Patch 8.0.0452

问题:　　Some macros are in lower case.
解决方案:　Make a few more macros upper case.
相关文件:　src/vim.h, src/macros.h, src/evalfunc.c, src/fold.c,
　　src/gui_gtk.c, src/gui_gtk_x11.c, src/charset.c, src/diff.c,
　　src/edit.c, src/eval.c, src/ex_cmds.c, src/ex_cmds2.c,
　　src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/getchar.c,
　　src/gui.c, src/gui_w32.c, src/if_cscope.c, src/mbyte.c,
　　src/menu.c, src/message.c, src/misc1.c, src/misc2.c, src/normal.c,
　　src/ops.c, src/option.c, src/os_unix.c, src/os_win32.c,
　　src/quickfix.c, src/regexp.c, src/regexp_nfa.c, src/screen.c,
　　src/search.c, src/spell.c, src/syntax.c, src/tag.c, src/userfunc.c

Patch 8.0.0453
问题:　　Adding fold marker creates new comment.
解决方案:　Use an existing comment if possible. (LemonBoy, closes #1549)
相关文件:　src/ops.c, src/proto/ops.pro, src/fold.c,
　　src/testdir/test_fold.vim

Patch 8.0.0454
问题:　　Compiler warnings for comparing unsigned char with 256 always
　　being true. (Manuel Ortega)
解决方案:　Add type cast.
相关文件:　src/screen.c, src/charset.c

Patch 8.0.0455
问题:　　The mode test may hang in Test_mode(). (Michael Soyka)
解决方案:　Set 'complete' to only search the current buffer (as suggested by
　　Michael)
相关文件:　src/testdir/test_functions.vim

Patch 8.0.0456
问题:　　Typo in MinGW test makefile.
解决方案:　Change an underscore to a dot. (Michael Soyka)
相关文件:　src/testdir/Make_ming.mak

Patch 8.0.0457
问题:　　Using :move messes up manual folds.
解决方案:　Split adjusting marks and folds. Add foldMoveRange(). (neovim
　　patch #6221)
相关文件:　src/ex_cmds.c, src/fold.c, src/mark.c, src/proto/fold.pro,
　　src/proto/mark.pro src/testdir/test_fold.vim

Patch 8.0.0458
问题:　　Potential crash if adding list or dict to dict fails.
解决方案:　Make sure the reference count is correct. (Nikolai Pavlov, closes
　　#1555)
相关文件:　src/dict.c

Patch 8.0.0459 (after 8.0.0457)
问题:　　Old fix for :move messing up folding no longer needed, now that we
　　have a proper solution.
解决方案:　Revert patch 7.4.700. (Christian Brabandt)
相关文件:　src/ex_cmds.c

Patch 8.0.0460 (after 8.0.0452)
问题: Can't build on HPUX.
解决方案: Fix argument names in vim_stat(). (John Marriott)
相关文件: src/misc2.c

Patch 8.0.0461 (after 8.0.0457)
问题: Test 45 hangs on MS-Windows.
解决方案: Reset 'shiftwidth'. Also remove redundant function.
相关文件: src/fold.c, src/testdir/test45.in

Patch 8.0.0462
问题: If an MS-Windows tests succeeds at first and then fails in a way it does not produce a test.out file it looks like the test succeeded.
解决方案: Delete the previous output file.
相关文件: src/testdir/Make_dos.mak

Patch 8.0.0463
问题: Resetting 'compatible' in defaults.vim has unexpected side effects. (David Fishburn)
解决方案: Only reset 'compatible' if it was set.
相关文件: runtime/defaults.vim

Patch 8.0.0464
问题: Can't find executable name on Solaris and FreeBSD.
解决方案: Check for "/proc/self/path/a.out". (Danek Duvall) And for "/proc/curproc/file".
相关文件: src/config.h.in, src/configure.ac, src/main.c, src/auto/configure

Patch 8.0.0465
问题: Off-by-one error in using :move with folding.
解决方案: Correct off-by-one mistakes and add more tests. (Matthew Malcomson)
相关文件: src/fold.c, src/testdir/test_fold.vim

Patch 8.0.0466
问题: There are still a few macros that should be all-caps.
解决方案: Make a few more macros all-caps.
相关文件: src/buffer.c, src/edit.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/farsi.c, src/fileio.c, src/getchar.c, src/gui_beval.c, src/hardcopy.c, src/if_cscope.c, src/if_xcmdsrv.c, src/mark.c, src/memline.c, src/menu.c, src/message.c, src/misc1.c, src/normal.c, src/ops.c, src/option.c, src/quickfix.c, src/screen.c, src/search.c, src/syntax.c, src/tag.c, src/term.c, src/term.h, src/ui.c, src/undo.c, src/userfunc.c, src/version.c, src/vim.h

Patch 8.0.0467
问题: Using g< after :for does not show the right output. (Marcin Szamotulski)
解决方案: Call msg_sb_eol() in :echomsg.
相关文件: src/eval.c

Patch 8.0.0468
问题: After aborting an Ex command g< does not work. (Marcin Szamotulski)
解决方案: Postpone clearing scrollback messages to until the command line has been entered. Also fix that the screen isn't redrawn if after g< the command line is cancelled.
相关文件: src/message.c, src/proto/message.pro, src/ex_getln.c, src/misc2.c, src/gui.c

Patch 8.0.0469
问题: Compiler warnings on MS-Windows.
解决方案: Add type casts. (Christian Brabandt)
相关文件: src/fold.c

Patch 8.0.0470
问题: Not enough testing for help commands.
解决方案: Add a few more help tests. (Dominique Pelle, closes #1565)
相关文件: src/testdir/test_help.vim, src/testdir/test_help_tagjump.vim

Patch 8.0.0471
问题: Exit callback test sometimes fails.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0472
问题: When a test fails and test.log is created, Test_edit_CTRL_I matches it instead of test1.in.
解决方案: Match with runtest.vim instead.
相关文件: src/testdir/test_edit.vim

Patch 8.0.0473
问题: No test covering arg_all().
解决方案: Add a test expanding ##.
相关文件: src/testdir/test_arglist.vim

Patch 8.0.0474
问题: The client-server feature is not tested.
解决方案: Add a test.
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/shared.vim, src/testdir/test_clientserver.vim, src/os_mswin.c

Patch 8.0.0475
问题: Not enough testing for the client-server feature.
解决方案: Add more tests. Add the remote_startserver() function. Fix that a locally evaluated expression uses function-local variables.
相关文件: src/if_xcmdsrv.c, src/evalfunc.c, src/os_mswin.c, src/proto/main.pro, src/testdir/test_clientserver.vim, runtime/doc/eval.txt

Patch 8.0.0476 (after 8.0.0475)
问题: Missing change to main.c.
解决方案: Add new function.
相关文件: src/main.c

Patch 8.0.0477
问题: The client-server test may hang when failing.
解决方案: Set a timer. Add assert_report()
相关文件: src/testdir/test_clientserver.vim, src/testdir/runtest.vim,
            src/eval.c, src/evalfunc.c, src/proto/eval.pro, src/if_xcmdsrv.c,
            src/os_mswin.c, runtime/doc/eval.txt

Patch 8.0.0478
问题: Tests use assert_true(0) and assert_false(1) to report errors.
解决方案: Use assert_report().
相关文件: src/testdir/test_cscope.vim, src/testdir/test_expr.vim,
            src/testdir/test_perl.vim, src/testdir/test_channel.vim,
            src/testdir/test_cursor_func.vim, src/testdir/test_gui.vim,
            src/testdir/test_menu.vim, src/testdir/test_popup.vim,
            src/testdir/test_viminfo.vim, src/testdir/test_vimscript.vim,
            src/testdir/test_assert.vim

Patch 8.0.0479
问题: remote_peek() is not tested.
解决方案: Add a test.
相关文件: src/testdir/test_clientserver.vim, src/testdir/runtest.vim

Patch 8.0.0480
问题: The remote_peek() test fails on MS-Windows.
解决方案: Check for pending messages. Also report errors in the first run if
            a flaky test fails twice.
相关文件: src/os_mswin.c, src/testdir/runtest.vim

Patch 8.0.0481
问题: Unnecessary if statement.
解决方案: Remove the statement. Fix "it's" vs "its" mistakes. (Dominique
            Pelle, closes #1568)
相关文件: src/syntax.c

Patch 8.0.0482
问题: The setbufvar() function may mess up the window layout. (Kay Z.)
解决方案: Do not check the window to be valid if it is NULL.
相关文件: src/window.c, src/testdir/test_functions.vim

Patch 8.0.0483
问题: Illegal memory access when using :all. (Dominique Pelle)
解决方案: Adjust the cursor position right after setting "curwin".
相关文件: src/window.c, src/testdir/test_window_cmd.vim

Patch 8.0.0484
问题: Using :lhelpgrep with an argument that should fail does not
            produce an error if the previous :helpgrep worked.
解决方案: Use another way to detect that autocommands made the quickfix info
            invalid. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0485
问题: Not all windows commands are tested.
解决方案: Add more tests for windows commands. (Dominique Pelle,

closes #1575) Run test_autocmd separately, it interferes with other tests. Fix tests that depended on side effects.
相关文件: src/testdir/test_window_cmd.vim, src/testdir/test_alot.vim, src/testdir/test_autocmd.vim, src/testdir/test_fnamemodify.vim, src/testdir/test_functions.vim, src/testdir/test_delete.vim, src/testdir/Make_all.mak

Patch 8.0.0486
问题: Crash and endless loop when closing windows in a SessionLoadPost autocommand.
解决方案: Check for valid tabpage. (partly neovim #6308)
相关文件: src/testdir/test_autocmd.vim, src/fileio.c, src/proto/window.pro, src/window.c

Patch 8.0.0487
问题: The autocmd test hangs on MS-Windows.
解决方案: Skip the hanging tests for now.
相关文件: src/testdir/test_autocmd.vim

Patch 8.0.0488
问题: Running tests leaves an "xxx" file behind.
解决方案: Delete the 'verbosefile' after resetting the option.
相关文件: src/testdir/gen_opt_test.vim

Patch 8.0.0489
问题: Clipboard and "* register is not tested.
解决方案: Add a test for Mac and X11. (Kazunobu Kuriyama)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/test_quotestar.vim, src/testdir/runtest.vim

Patch 8.0.0490
问题: Splitting a 'winfixwidth' window vertically makes it one column smaller. (Dominique Pelle)
解决方案: Add one to the width for the separator.
相关文件: src/window.c, src/testdir/test_window_cmd.vim

Patch 8.0.0491
问题: The quotestar test fails when a required feature is missing.
解决方案: Prepend "Skipped" to the thrown exception.
相关文件: src/testdir/test_quotestar.vim

Patch 8.0.0492
问题: A failing client-server request can make Vim hang.
解决方案: Add a timeout argument to functions that wait.
相关文件: src/evalfunc.c, src/if_xcmdsrv.c, src/proto/if_xcmdsrv.pro, src/main.c, src/os_mswin.c, src/proto/os_mswin.pro, src/vim.h, runtime/doc/eval.txt, src/testdir/test_clientserver.vim

Patch 8.0.0493
问题: Crash with cd command with very long argument.
解决方案: Check for running out of space. (Dominique Pelle, closes #1576)
相关文件: src/testdir/test_alot.vim, src/testdir/test_cd.vim, src/Makefile, src/misc2.c

Patch 8.0.0494
问题: Build failure with older compiler on MS-Windows.
解决方案: Move declaration to start of block.
相关文件: src/evalfunc.c, src/main.c, src/os_mswin.c

Patch 8.0.0495
问题: The quotestar test uses a timer instead of a timeout, thus it
            cannot be rerun like a flaky test.
解决方案: Remove the timer and add a timeout. (Kazunobu Kuriyama)
相关文件: src/testdir/test_quotestar.vim

Patch 8.0.0496
问题: Insufficient testing for folding.
解决方案: Add a couple more fold tests. (Dominique Pelle, closes #1579)
相关文件: src/testdir/test_fold.vim

Patch 8.0.0497
问题: Arabic support is not fully tested.
解决方案: Add more tests for the untested functions. Comment out
            unreachable code.
相关文件: src/arabic.c, src/testdir/test_arabic.vim

Patch 8.0.0498
问题: Two autocmd tests are skipped on MS-Windows.
解决方案: Make the test pass on MS-Windows. Write the messages in a file
            instead of getting the output of system().
相关文件: src/testdir/test_autocmd.vim

Patch 8.0.0499
问题: taglist() does not prioritize tags for a buffer.
解决方案: Add an optional buffer argument. (Duncan McDougall, closes #1194)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/proto/tag.pro,
            src/Makefile, src/tag.c, src/testdir/test_alot.vim,
            src/testdir/test_taglist.vim

Patch 8.0.0500
问题: Quotestar test is still a bit flaky.
解决方案: Add a slower check for v:version.
相关文件: src/testdir/test_quotestar.vim

Patch 8.0.0501
问题: On MS-Windows ":!start" does not work as expected.
解决方案: When creating a process fails try passing the argument to
            ShellExecute(). (Katsuya Hino, closes #1570)
相关文件: runtime/doc/os_win32.txt, src/os_win32.c

Patch 8.0.0502
问题: Coverity complains about possible NULL pointer.
解决方案: Add an assert(), let's see if this works on all systems.
相关文件: src/window.c

Patch 8.0.0503
问题: Endless loop in updating folds with 32 bit ints.
解决方案: Subtract from LHS instead of add to the RHS. (Matthew Malcomson)

相关文件: src/fold.c

Patch 8.0.0504
问题: Looking up an Ex command is a bit slow.
解决方案: Instead of just using the first letter, also use the second letter to skip ahead in the list of commands. Generate the table with a Perl script. (Dominique Pelle, closes #1589)
相关文件: src/Makefile, src/create_cmdidxs.pl, src/ex_docmd.c, Filelist

Patch 8.0.0505
问题: Failed window split for :stag not handled. (Coverity CID 99204)
解决方案: If the split fails skip to the end. (bstaletic, closes #1577)
相关文件: src/tag.c

Patch 8.0.0506 (after 8.0.0504)
问题: Can't build with ANSI C.
解决方案: Move declarations to start of block.
相关文件: src/ex_docmd.c

Patch 8.0.0507
问题: Client-server tests fail when $DISPLAY is not set.
解决方案: Check for E240 before running the test.
相关文件: src/testdir/test_quotestar.vim, src/testdir/test_clientserver.vim

Patch 8.0.0508
问题: Coveralls no longer shows per-file coverage.
解决方案: Add coverage from codecov.io. (Christian Brabandt)
相关文件: .travis.yml

Patch 8.0.0509
问题: No link to codecov.io results.
解决方案: Add a badge to the readme file.
相关文件: README.md

Patch 8.0.0510 (after 8.0.0509)
问题: Typo in link to codecov.io results.
解决方案: Remove duplicate https:.
相关文件: README.md

Patch 8.0.0511
问题: Message for skipping client-server tests is unclear.
解决方案: Be more specific about what's missing (Hirohito Higashi, Kazunobu Kuriyama)
相关文件: src/testdir/test_quotestar.vim, src/testdir/test_clientserver.vim

Patch 8.0.0512
问题: Check for available characters takes too long.
解决方案: Only check did_start_blocking if wtime is negative. (Daisuke Suzuki, closes #1591)
相关文件: src/os_unix.c

Patch 8.0.0513 (after 8.0.0201)
问题: Getting name of cleared highlight group is wrong. (Matt Wozniski)
解决方案: Only skip over cleared names for completion. (closes #1592)

Also fix that a cleared group causes duplicate completions.
相关文件: src/syntax.c, src/proto/syntax.pro, src/evalfunc.c, src/ex_cmds.c, src/testdir/test_syntax.vim, src/testdir/test_cmdline.vim

Patch 8.0.0514
问题: Script for creating cmdidxs can be improved.
解决方案: Count skipped lines instead of collecting the lines. Add "const". (Dominique Pelle, closes #1594)
相关文件: src/create_cmdidxs.pl, src/ex_docmd.c

Patch 8.0.0515
问题: ml_get errors in silent Ex mode. (Dominique Pelle)
解决方案: Clear valid flags when setting the cursor. Set the topline when not in full screen mode.
相关文件: src/ex_docmd.c, src/move.c, src/testdir/test_startup.vim

Patch 8.0.0516
问题: A large count on a normal command causes trouble. (Dominique Pelle)
解决方案: Make "opcount" long.
相关文件: src/globals.h, src/testdir/test_normal.vim

Patch 8.0.0517
问题: There is no way to remove quickfix lists (for testing).
解决方案: Add the 'f' action to setqflist(). Add tests. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0518
问题: Storing a zero byte from a multi-byte character causes fold text to show up wrong.
解决方案: Avoid putting zero in ScreenLines. (Christian Brabandt, closes #1567)
相关文件: src/screen.c, src/testdir/test_display.vim

Patch 8.0.0519
问题: Character classes are not well tested. They can differ between platforms.
解决方案: Add tests. In the documentation make clear which classes depend on what library function. Only use :cntrl: and :graph: for ASCII. (Kazunobu Kuriyama, Dominique Pelle, closes #1560)
Update the documentation.
相关文件: src/regexp.c, src/regexp_nfa.c, runtime/doc/pattern.txt, src/testdir/test_regexp_utf8.vim

Patch 8.0.0520
问题: Using a function pointer instead of the actual function, which we know.
解决方案: Change mb_ functions to utf_ functions when already checked for Unicode. (Dominique Pelle, closes #1582)
相关文件: src/message.c, src/misc2.c, src/regexp.c, src/regexp_nfa.c, src/screen.c, src/spell.c

Patch 8.0.0521
问题: GtkForm handling is outdated.
解决方案: Get rid of event filter functions. Get rid of GtkForm.width and .height. Eliminate gtk_widget_size_request() calls. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_f.c, src/gui_gtk_f.h

Patch 8.0.0522
问题: MS-Windows: when 'clipboard' is "unnamed" yyp does not work in a :global command.
解决方案: When setting the clipboard was postponed, do not clear the register.
相关文件: src/ops.c, src/proto/ui.pro, src/ui.c, src/globals.h, src/testdir/test_global.vim, src/Makefile, src/testdir/test_alot.vim

Patch 8.0.0523
问题: dv} deletes part of a multi-byte character. (Urtica Dioica)
解决方案: Include the whole character.
相关文件: src/search.c, src/testdir/test_normal.vim

Patch 8.0.0524 (after 8.0.0518)
问题: Folds are messed up when 'encoding' is "utf-8".
解决方案: Also set the fold character when it's not multi-byte.
相关文件: src/screen.c, src/testdir/test_display.vim

Patch 8.0.0525
解决方案: Completion for user command argument not tested.
问题: Add a test.
相关文件: src/testdir/test_cmdline.vim

Patch 8.0.0526
问题: Coverity complains about possible negative value.
解决方案: Check return value of ftell() not to be negative.
相关文件: src/os_unix.c

Patch 8.0.0527
问题: RISC OS support was removed long ago, but one file is still included.
解决方案: Delete the file. (Thomas Dziedzic, closes #1603)
相关文件: Filelist, src/swis.s

Patch 8.0.0528
问题: When 'wildmenu' is set and 'wildmode' has "longest" then the first file name is highlighted, even though the text shows the longest match.
解决方案: Do not highlight the first match. (LemonBoy, closes #1602)
相关文件: src/ex_getln.c

Patch 8.0.0529
问题: Line in test commented out.
解决方案: Uncomment the lines for character classes that were failing before 8.0.0519. (Dominique Pelle, closes #1599)

相关文件: src/testdir/test_regexp_utf8.vim

Patch 8.0.0530
问题: Buffer overflow when 'columns' is very big. (Nikolai Pavlov)
解决方案: Correctly compute where to truncate. Fix translation.
(closes #1600)
相关文件: src/edit.c, src/testdir/test_edit.vim

Patch 8.0.0531 (after 8.0.0530)
问题: Test with long directory name fails on non-unix systems.
解决方案: Skip the test on non-unix systems.
相关文件: src/testdir/test_edit.vim

Patch 8.0.0532 (after 8.0.0531)
问题: Test with long directory name fails on Mac.
解决方案: Skip the test on Mac systems.
相关文件: src/testdir/test_edit.vim

Patch 8.0.0533
问题: Abbreviation doesn't work after backspacing newline. (Hkonrk)
解决方案: Set the insert start column. (closes #1609)
相关文件: src/testdir/test_mapping.vim, src/edit.c

Patch 8.0.0534
问题: Defaults.vim does not work well with tiny features. (crd477)
解决方案: When the +eval feature is not available always reset 'compatible'.
相关文件: runtime/defaults.vim

Patch 8.0.0535
问题: Memory leak when exiting from within a user function.
解决方案: Clear the function call stack on exit.
相关文件: src/userfunc.c

Patch 8.0.0536
问题: Quickfix window not updated when freeing quickfix stack.
解决方案: Update the quickfix window. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0537
问题: Illegal memory access with :z and large count.
解决方案: Check for number overflow, using long instead of int. (Dominique
Pelle, closes #1612)
相关文件: src/Makefile, src/ex_cmds.c, src/testdir/test_alot.vim,
src/testdir/test_ex_z.vim

Patch 8.0.0538
问题: No test for falling back to default term value.
解决方案: Add a test.
相关文件: src/testdir/test_startup.vim

Patch 8.0.0539 (after 8.0.0538)
问题: Startup test fails on Mac.
解决方案: Use another term name, "unknown" is known. Avoid a 2 second delay.
相关文件: src/testdir/test_startup.vim, src/main.c, src/proto/main.pro,

src/term.c

Patch 8.0.0540 (after 8.0.0540)
问题: Building unit tests fails.
解决方案: Move params outside of #ifdef.
相关文件: src/main.c, src/message_test.c

Patch 8.0.0541
问题: Compiler warning on MS-Windows.
解决方案: Add a type cast. (Mike Williams)
相关文件: src/edit.c

Patch 8.0.0542
问题: getpos() can return a negative line number. (haya14busa)
解决方案: Handle a zero topline and botline. (closes #1613)
相关文件: src/eval.c, runtime/doc/eval.txt

Patch 8.0.0543
问题: Test_edit causes older xfce4-terminal to close. (Dominique Pelle)
解决方案: Reduce number of columns to 2000. Try to restore the window position.
相关文件: src/testdir/test_edit.vim, src/evalfunc.c, src/term.c, src/proto/term.pro, src/term.h

Patch 8.0.0544
问题: Cppcheck warnings.
解决方案: Use temp variable. Change NUL to NULL. Swap conditions. (Dominique Pelle)
相关文件: src/channel.c, src/edit.c, src/farsi.c

Patch 8.0.0545
问题: Edit test may fail on some systems.
解决方案: If creating a directory with a very long path fails, bail out.
相关文件: src/testdir/test_edit.vim

Patch 8.0.0546
问题: Swap file exists briefly when opening the command window.
解决方案: Set the noswapfile command modifier before splitting the window. (James McCoy, closes #1620)
相关文件: src/ex_getln.c, src/option.c

Patch 8.0.0547
问题: Extra line break in verbosefile when using ":echomsg". (Ingo Karkat)
解决方案: Don't call msg_start(). (closes #1618)
相关文件: src/eval.c, src/testdir/test_cmdline.vim

Patch 8.0.0548
问题: Saving the redo buffer only works one time, resulting in the "." command not working well for a function call inside another function call. (Ingo Karkat)
解决方案: Save the redo buffer at every user function call. (closes #1619)
相关文件: src/getchar.c, src/proto/getchar.pro, src/structs.h, src/fileio.c, src/userfunc.c, src/testdir/test_functions.vim

Patch 8.0.0549
问题:　　No test for the 8g8 command.
解决方案:　Add a test. (Dominique Pelle, closes #1615)
相关文件:　src/testdir/test_normal.vim

Patch 8.0.0550
问题:　　Some etags format tags file use 0x01, breaking the parsing.
解决方案:　Use 0x02 for TAG_SEP. (James McCoy, closes #1614)
相关文件:　src/tag.c, src/testdir/test_taglist.vim

Patch 8.0.0551
问题:　　The typeahead buffer is reallocated too often.
解决方案:　Re-use the existing buffer if possible.
相关文件:　src/getchar.c

Patch 8.0.0552
问题:　　Toupper and tolower don't work properly for Turkish when 'casemap'
　　is empty. (Bjorn Linse)
解决方案:　Check the 'casemap' options when deciding how to upper/lower case.
相关文件:　src/charset.c, src/testdir/test_normal.vim

Patch 8.0.0553 (after 8.0.0552)
问题:　　Toupper/tolower test with Turkish locale fails on Mac.
解决方案:　Skip the test on Mac.
相关文件:　src/testdir/test_normal.vim

Patch 8.0.0554 (after 8.0.0552)
问题:　　Toupper and tolower don't work properly for Turkish when 'casemap'
　　contains "keepascii". (Bjorn Linse)
解决方案:　When 'casemap' contains "keepascii" use ASCII toupper/tolower.
相关文件:　src/charset.c, src/testdir/test_normal.vim

Patch 8.0.0555 (after 8.0.0552)
问题:　　Toupper/tolower test fails on OSX without Darwin.
解决方案:　Skip that part of the test also for OSX. (Kazunobu Kuriyama)
相关文件:　src/testdir/test_normal.vim

Patch 8.0.0556
问题:　　Getting the window position fails if both the GUI and term
　　code is built in.
解决方案:　Return after getting the GUI window position. (Kazunobu Kuriyama)
相关文件:　src/evalfunc.c

Patch 8.0.0557
问题:　　GTK: using static gravities is not useful.
解决方案:　Remove setting static gravities. (Kazunobu Kuriyama)
相关文件:　src/gui_gtk_f.c

Patch 8.0.0558
问题:　　The :ownsyntax command is not tested.
解决方案:　Add a test. (Dominique Pelle, closes #1622)
相关文件:　src/testdir/test_syntax.vim

Patch 8.0.0559
问题: Setting 'ttytype' to xxx does not always fail as expected. (Marvin Schmidt)
解决方案: Catch both possible errors. (closes #1601)
相关文件: src/testdir/test_options.vim

Patch 8.0.0560
问题: :windo allows for ! but it's not supported.
解决方案: Disallow passing !. (Hirohito Higashi)
相关文件: src/ex_cmds.h

Patch 8.0.0561
问题: Undefined behavior when using backslash after empty line.
解决方案: Check for an empty line. (Dominique Pelle, closes #1631)
相关文件: src/misc2.c, src/testdir/test_vimscript.vim

Patch 8.0.0562
问题: Not enough test coverage for syntax commands.
解决方案: Add a few more tests. (Dominique Pelle, closes #1624)
相关文件: src/testdir/test_cmdline.vim, src/testdir/test_syntax.vim

Patch 8.0.0563
问题: Crash when getting the window position in tmux. (Marvin Schmidt)
解决方案: Add t_GP to the list of terminal options. (closes #1627)
相关文件: src/option.c

Patch 8.0.0564
问题: Cannot detect Bazel BUILD files on some systems.
解决方案: Check for BUILD after script checks. (Issue #1340)
相关文件: runtime/filetype.vim

Patch 8.0.0565
问题: Using freed memory in :caddbuf after clearing quickfix list. (Dominique Pelle)
解决方案: Set qf_last to NULL.
相关文件: src/quickfix.c

Patch 8.0.0566
问题: Setting 'nocompatible' for the tiny version moves the cursor.
解决方案: Use another trick to skip commands when the +eval feature is present. (Christian Brabandt, closes #1630)
相关文件: runtime/defaults.vim

Patch 8.0.0567
问题: Call for requesting color and ambiwidth is too early. (Hirohito Higashi)
解决方案: Move the call down to below resetting "starting".
相关文件: src/main.c

Patch 8.0.0568
问题: "1gd" may hang.
解决方案: Don't get stuck in one position. (Christian Brabandt, closes #1643)
相关文件: src/testdir/test_goto.vim, src/normal.c

Patch 8.0.0569
问题: Bracketed paste is still enabled when executing a shell command.
(Michael Smith)
解决方案: Disable bracketed paste when going into cooked mode. (closes #1638)
相关文件: src/term.c

Patch 8.0.0570
问题: Can't run make with several jobs, creating directories has a race
condition.
解决方案: Use the MKDIR_P autoconf mechanism. (Eric N. Vander Weele,
closes #1639)
相关文件: src/configure.ac, src/auto/configure, src/Makefile,
src/config.mk.in, src/install-sh, src/mkinstalldirs, Filelist

Patch 8.0.0571
问题: The cursor line number becomes negative when using :z^ in an empty
buffer. (neovim #6557)
解决方案: Correct the line number. Also reset the column.
相关文件: src/testdir/test_ex_z.vim, src/ex_cmds.c

Patch 8.0.0572
问题: Building the command table requires Perl.
解决方案: Use a Vim script solution. (Dominique Pelle, closes #1641)
相关文件: src/Makefile, src/create_cmdidxs.pl, src/create_cmdidxs.vim,
src/ex_cmdidxs.h, src/ex_docmd.c, Filelist

Patch 8.0.0573
问题: Running parallel make after distclean fails. (Manuel Ortega)
解决方案: Instead of using targets "scratch config myself" use "reconfig".
相关文件: src/Makefile, src/config.mk.dist

Patch 8.0.0574
问题: Get only one quickfix list after :caddbuf.
解决方案: Reset qf_multiline. (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0575
问题: Using freed memory when resetting 'indentexpr' while evaluating
it. (Dominique Pelle)
解决方案: Make a copy of 'indentexpr'.
相关文件: src/misc1.c, src/testdir/test_options.vim

Patch 8.0.0576 (after 8.0.0570 and 8.0.0573)
问题: Can't build when configure chooses "install-sh". (Daniel Hahler)
解决方案: Always use install-sh. Fix remaining use of mkinstalldirs.
(closes #1647)
相关文件: src/installman.sh, src/installml.sh, src/config.mk.in,
src/configure.ac, src/auto/configure, src/Makefile

Patch 8.0.0577 (after 8.0.0575)
问题: Warning for uninitialized variable. (John Marriott)
解决方案: Initialize "indent".
相关文件: src/misc1.c

Patch 8.0.0578
问题: :simalt on MS-Windows does not work properly.
解决方案: Put something in the typeahead buffer. (Christian Brabandt)
相关文件: src/gui_w32.c

Patch 8.0.0579
问题: Duplicate test case for quickfix.
解决方案: Remove the function. (Yegappan Lakshmanan)
相关文件: src/testdir/test_quickfix.vim

Patch 8.0.0580
问题: Cannot set the valid flag with setqflist().
解决方案: Add the "valid" argument. (Yegappan Lakshmanan, closes #1642)
相关文件: runtime/doc/eval.txt, src/quickfix.c,
            src/testdir/test_quickfix.vim

Patch 8.0.0581
问题: Moving folded text is sometimes not correct.
解决方案: Bail out when "move_end" is zero. (Matthew Malcomson)
相关文件: src/fold.c, src/testdir/test_fold.vim

Patch 8.0.0582
问题: Illegal memory access with z= command. (Dominique Pelle)
解决方案: Avoid case folded text to be longer than the original text. Use
            MB_PTR2LEN() instead of MB_BYTE2LEN().
相关文件: src/spell.c, src/testdir/test_spell.vim

Patch 8.0.0583
问题: Fold test hangs on MS-Windows.
解决方案: Avoid overflow in compare.
相关文件: src/fold.c

Patch 8.0.0584
问题: Memory leak when executing quickfix tests.
解决方案: Free the list reference. (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.0585
问题: Test_options fails when run in the GUI.
解决方案: Also check the 'imactivatekey' value when the GUI is not running.
            Specify test values that work and that fail.
相关文件: src/option.c, src/testdir/gen_opt_test.vim

Patch 8.0.0586
问题: No test for mapping timing out.
解决方案: Add a test.
相关文件: src/testdir/test_mapping.vim

Patch 8.0.0587
问题: Configure check for return value of tgetent is skipped.
解决方案: Always perform the check. (Marvin Schmidt, closes #1664)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.0588

问题:　　job_stop() often assumes the channel will be closed, while the job may not actually be stopped. (Martin Gammelster)
解决方案:　Only assume the job stops on "kill". Don't send a signal if the job has already ended. (closes #1632)
相关文件:　src/channel.c

Patch 8.0.0589 (after 8.0.0578)
问题:　　:simalt still does not work.
解决方案:　Use K_NOP instead of K_IGNORE. (Christian Brabandt)
相关文件:　src/gui_w32.c

Patch 8.0.0590
问题:　　Cannot add a context to locations.
解决方案:　Add the "context" entry in location entries. (Yegappan Lakshmanan, closes #1012)
相关文件:　src/eval.c, src/proto/quickfix.pro, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0591
问题:　　Changes to eval functionality not documented.
解决方案:　Include all the changes.
相关文件:　runtime/doc/eval.txt

Patch 8.0.0592
问题:　　If a job writes to a buffer and the user is typing a command, the screen isn't updated. When a message is displayed the changed buffer may cause it to be cleared. (Ramel Eshed)
解决方案:　Update the screen and then the command line if the screen didn't scroll. Avoid inserting screen lines, as it clears any message. Update the status line when the buffer changed.
相关文件:　src/channel.c, src/screen.c, src/ex_getln.c, src/globals.h, src/vim.h, src/proto/ex_getln.pro, src/proto/screen.pro

Patch 8.0.0593
问题:　　Duplication of code for adding a list or dict return value.
解决方案:　Add rettv_dict_set() and rettv_list_set(). (Yegappan Lakshmanan)
相关文件:　src/dict.c, src/eval.c, src/evalfunc.c, src/if_perl.xs, src/list.c, src/proto/dict.pro, src/proto/list.pro

Patch 8.0.0594 (after 8.0.0592)
问题:　　Build failure when windows feature is missing.
解决方案:　Add #ifdef.
相关文件:　src/screen.c

Patch 8.0.0595 (after 8.0.0590)
问题:　　Coverity warning for not checking return value of dict_add().
解决方案:　Check the return value for FAIL.
相关文件:　src/quickfix.c

Patch 8.0.0596
问题:　　Crash when complete() is called after complete_add() in 'completefunc'. (Lifepillar)
解决方案:　Bail out if compl_pattern is NULL. (closes #1668)
Also avoid using freed memory.

相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0597
问题: Off-by-one error in buffer size computation.
解决方案: Use ">=" instead of ">". (LemonBoy, closes #1694)
相关文件: src/quickfix.c

Patch 8.0.0598
问题: Building with gcc 7.1 yields new warnings.
解决方案: Initialize result. (John Marriott)
相关文件: src/ex_docmd.c

Patch 8.0.0599
问题: diff mode is insufficiently tested
解决方案: Add more test cases. (Dominique Pelle, closes #1685)
相关文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.0.0600
问题: test_recover fails on some systems.
解决方案: Explicitly check if "/" is writable. (Ken Takata)
相关文件: src/testdir/test_recover.vim

Patch 8.0.0601
问题: No test coverage for :spellrepall.
解决方案: Add a test. (Dominique Pelle, closes #1717)
相关文件: src/testdir/test_spell.vim

Patch 8.0.0602
问题: When gF fails to edit the file the cursor still moves to the found
line number.
解决方案: Check the return value of do_ecmd(). (Michael Hwang)
相关文件: src/normal.c, src/testdir/test_gf.vim

Patch 8.0.0603 (after 8.0.0602)
问题: gF test fails on MS-Windows.
解决方案: Use @ instead of : before the line number
相关文件: src/testdir/test_gf.vim

Patch 8.0.0604 (after 8.0.0603)
问题: gF test still fails on MS-Windows.
解决方案: Use : before the line number and remove it from 'isfname'.
相关文件: src/testdir/test_gf.vim

Patch 8.0.0605
问题: The buffer that quickfix caches for performance may become
invalid. (Daniel Hahler)
解决方案: Reset qf_last_bufref in qf_init_ext(). (Daniel Hahler,
closes #1728, closes #1676)
相关文件: src/quickfix.c

Patch 8.0.0606
问题: Cannot set the context for a specified quickfix list.
解决方案: Use the list index instead of the current list. (Yegappan
Lakshmanan)

相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0607
问题: When creating a bufref, then using :bwipe and :new it might get
the same memory and bufref_valid() returns true.
解决方案: Add br_fnum to check the buffer number didn't change.
相关文件: src/structs.h, src/buffer.c, src/globals.h, src/if_py_both.h,
src/quickfix.c

Patch 8.0.0608
问题: Cannot manipulate other than the current quickfix list.
解决方案: Pass the list index to quickfix functions. (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.0609
问题: For some people the hint about quitting is not sufficient.
解决方案: Put <Enter> separately. Also use ":qa!" to get out even when
there are changes.
相关文件: src/normal.c

Patch 8.0.0610
问题: The screen is redrawn when t_BG is set and used to detect the
value for 'background'.
解决方案: Don't redraw when the value of 'background' didn't change.
相关文件: src/term.c

Patch 8.0.0611
问题: When t_u7 is sent a few characters in the second screen line are
overwritten and not redrawn later. (Rastislav Barlik)
解决方案: Move redrawing the screen to after overwriting the characters.
相关文件: src/main.c, src/term.c

Patch 8.0.0612
问题: Package directories are added to 'runtimepath' only after loading
non-package plugins.
解决方案: Split off the code to add package directories to 'runtimepath'.
(Ingo Karkat, closes #1680)
相关文件: src/ex_cmds2.c, src/globals.h, src/main.c, src/proto/ex_cmds2.pro,
src/testdir/test_startup.vim

Patch 8.0.0613
问题: The conf filetype detection is done before ftdetect scripts from
packages that are added later.
解决方案: Add the FALLBACK argument to :setfiletype. (closes #1679,
closes #1693)
相关文件: src/ex_docmd.c, runtime/filetype.vim, src/Makefile,
src/testdir/test_filetype.vim, src/testdir/test_alot.vim

Patch 8.0.0614
问题: float2nr() is not exactly right.
解决方案: Make float2nr() more accurate. Turn test65 into a new style test.
(Hirohito Higashi, closes #1688)
相关文件: src/Makefile, src/evalfunc.c, src/testdir/Make_all.mak,
src/testdir/Make_vms.mms, src/testdir/test65.in,

src/testdir/test65.ok, src/testdir/test_float_func.vim,
src/testdir/test_vimscript.vim, src/macros.h

Patch 8.0.0615
问题: Using % with :hardcopy wrongly escapes spaces. (Alexey Muranov)
解决方案: Expand % differently. (Christian Brabandt, closes #1682)
相关文件: src/ex_docmd.c, src/testdir/test_hardcopy.vim

Patch 8.0.0616
问题: When setting the cterm background with ":hi Normal" the value of
'background' may be set wrongly.
解决方案: Check that the color is less than 16. Don't set 'background' when
it was set explicitly. (LemonBoy, closes #1710)
相关文件: src/syntax.c, src/testdir/test_syntax.vim

Patch 8.0.0617 (after 8.0.0615)
问题: Hardcopy test hangs on MS-Windows.
解决方案: Check the postscript feature is supported.
相关文件: src/testdir/test_hardcopy.vim

Patch 8.0.0618
问题: NFA regex engine handles [0-z] incorrectly.
解决方案: Return at the right point. (James McCoy, closes #1703)
相关文件: src/regexp_nfa.c, src/testdir/test36.in, src/testdir/test36.ok

Patch 8.0.0619
问题: In the GUI, when a timer uses feedkeys(), it still waits for an
event. (Raymond Ko)
解决方案: Check tb_change_cnt in one more place.
相关文件: src/gui.c

Patch 8.0.0620
问题: Since we only support GTK versions that have it, the check for
HAVE_GTK_MULTIHEAD is no longer needed.
解决方案: Remove HAVE_GTK_MULTIHEAD. (Kazunobu Kuriyama)
相关文件: src/config.h.in, src/configure.ac, src/auto/configure,
src/gui_beval.c, src/gui_gtk_x11.c, src/mbyte.c

Patch 8.0.0621
问题: The ":stag" command does not respect 'switchbuf'.
解决方案: Check 'switchbuf' for tag commands that may open a new window.
(Ingo Karkat, closes #1681) Define macros for the return values
of getfile().
相关文件: src/tag.c, src/testdir/test_tagjump.vim, src/vim.h, src/buffer.c,
src/ex_cmds.c, src/search.c,

Patch 8.0.0622
问题: Using a text object to select quoted text fails when 'selection'
is set to "exclusive". (Guraga)
解决方案: Swap cursor and visual start position. (Christian Brabandt,
closes #1687)
相关文件: src/search.c, src/testdir/test_textobjects.vim

Patch 8.0.0623
问题:　　The message "Invalid range" is used for multiple errors.
解决方案:　Add two more specific error messages. (Itchyny, Ken Hamada)
相关文件:　src/regexp.c, src/regexp_nfa.c, src/testdir/test_regexp_utf8.vim

Patch 8.0.0624 (after 8.0.0623)
问题:　　Warning for unused variable in tiny build. (Tony Mechelynck)
解决方案:　Add an #ifdef.
相关文件:　src/regexp.c

Patch 8.0.0625
问题:　　shellescape() always escapes a newline, which does not work with
　　some shells. (Harm te Hennepe)
解决方案:　Only escape a newline when the "special" argument is non-zero.
　　(Christian Brabandt, closes #1590)
相关文件:　src/evalfunc.c, src/testdir/test_functions.vim

Patch 8.0.0626
问题:　　In the GUI the cursor may flicker.
解决方案:　Check the cmd_silent flag before updating the cursor shape.
　　(Hirohito Higashi, closes #1637)
相关文件:　src/getchar.c

Patch 8.0.0627
问题:　　When 'wrapscan' is off "gn" does not select the whole pattern when
　　it's the last one in the text. (KeyboardFire)
解决方案:　Check if the search fails. (Christian Brabandt, closes #1683)
相关文件:　src/search.c, src/testdir/test_gn.vim

Patch 8.0.0628 (after 8.0.0626
问题:　　Cursor disappears after silent mapping. (Ramel Eshed)
解决方案:　Do restore the cursor when it was changed, but don't change it in
　　the first place for a silent mapping.
相关文件:　src/getchar.c

Patch 8.0.0629 (after 8.0.0611)
问题:　　Checking for ambiguous width is not working. (Hirohito Higashi)
解决方案:　Reset "starting" earlier.
相关文件:　src/main.c

Patch 8.0.0630
问题:　　The :global command does not work recursively, which makes it
　　difficult to execute a command on a line where one pattern matches
　　and another does not match. (Miles Cranmer)
解决方案:　Allow for recursion if it is for only one line. (closes #1760)
相关文件:　src/ex_cmds.c, src/testdir/test_global.vim, runtime/doc/repeat.txt

Patch 8.0.0631
问题:　　Perl 5.26 also needs S_TOPMARK and S_POPMARK defined.
解决方案:　Define the functions when needed. (Jesin, closes #1748)
相关文件:　src/if_perl.xs

Patch 8.0.0632

问题:　　The quotestar test is still a bit flaky.
解决方案:　Kill any existing server to make the retry work. Wait for the
　　register to be filled.
相关文件:　src/testdir/test_quotestar.vim

Patch 8.0.0633
问题:　　The client-server test is still a bit flaky.
解决方案:　Wait a bit for the GUI to start. Check that the version number
　　can be obtained.
相关文件:　src/testdir/test_clientserver.vim

Patch 8.0.0634
问题:　　Cannot easily get to the last quickfix list.
解决方案:　Add "$" as a value for the "nr" argument of getqflist() and
　　setqflist(). (Yegappan Lakshmanan)
相关文件:　runtime/doc/eval.txt, src/quickfix.c,
　　src/testdir/test_quickfix.vim

Patch 8.0.0635
问题:　　When 'ignorecase' is set script detection is inaccurate.
解决方案:　Enforce matching case for text. (closes #1753)
相关文件:　runtime/scripts.vim

Patch 8.0.0636
问题:　　When reading the undo file fails may use uninitialized data.
解决方案:　Always clear the buffer on failure.
相关文件:　src/undo.c

Patch 8.0.0637
问题:　　Crash when using some version of GTK 3.
解决方案:　Add #ifdefs around incrementing the menu index. (Kazunobu
　　Kuriyama)
相关文件:　src/gui_gtk.c

Patch 8.0.0638
问题:　　Cannot build with new MSVC version VS2017.
解决方案:　Change the compiler arguments. (Leonardo Valeri Manera,
　　closes #1731, closes #1747)
相关文件:　src/GvimExt/Makefile, src/Make_mvc.mak

Patch 8.0.0639
问题:　　The cursor position is set to the last position in a new commit
　　message.
解决方案:　Don't set the position if the filetype matches "commit".
　　(Christian Brabandt)
相关文件:　runtime/defaults.vim

Patch 8.0.0640
问题:　　Mismatch between help and actual message for ":syn conceal".
解决方案:　Change the message to match the help. (Ken Takata)
相关文件:　src/syntax.c

Patch 8.0.0641
问题:　　Cannot set a separate highlighting for the current line in the

quickfix window.
解决方案: Add QuickFixLine. (anishsane, closes #1755)
相关文件: src/option.c, src/quickfix.c, src/screen.c, src/syntax.c, src/vim.h, runtime/doc/options.txt, runtime/doc/quickfix.txt

Patch 8.0.0642
问题: writefile() continues after detecting an error.
解决方案: Bail out as soon as an error is detected. (suggestions by Nikolai Pavlov, closes #1476)
相关文件: src/evalfunc.c, src/testdir/test_writefile.vim

Patch 8.0.0643
问题: When 'hlsearch' is set and matching with the last search pattern is very slow, Vim becomes unusable. Cannot quit search by pressing CTRL-C.
解决方案: When the search times out set a flag and don't try again. Check for timeout and CTRL-C in NFA loop that adds states.
相关文件: src/screen.c, src/ex_cmds.c, src/quickfix.c, src/regexp.c, src/proto/regexp.pro, src/regexp.h, src/search.c, src/proto/search.pro, src/syntax.c, src/regexp_nfa.c, src/spell.c, src/tag.c, src/gui.c, src/edit.c, src/evalfunc.c, src/ex_docmd.c, src/ex_getln.c, src/normal.c

Patch 8.0.0644
问题: There is no test for 'hlsearch' timing out.
解决方案: Add a test.
相关文件: src/testdir/test_hlsearch.vim

Patch 8.0.0645
问题: The new regexp engine does not give an error for using a back reference where it is not allowed. (Dominique Pelle)
解决方案: Check the back reference like the old engine. (closes #1774)
相关文件: src/regexp.c, src/regexp_nfa.c, src/testdir/test_hlsearch.vim, src/testdir/test_statusline.vim, src/testdir/test_regexp_latin1.vim

Patch 8.0.0646
问题: The hlsearch test fails on fast systems.
解决方案: Make the search pattern slower. Fix that the old regexp engine doesn't timeout properly.
相关文件: src/regexp.c, src/testdir/test_hlsearch.vim

Patch 8.0.0647
问题: Syntax highlighting can cause a freeze.
解决方案: Apply 'redrawtime' to syntax highlighting, per window.
相关文件: src/structs.h, src/screen.c, src/syntax.c, src/normal.c, src/regexp.c, src/proto/syntax.pro, src/testdir/test_syntax.vim, runtime/doc/options.txt

Patch 8.0.0648
问题: Possible use of NULL pointer if buflist_new() returns NULL. (Coverity)
解决方案: Check for NULL pointer in set_bufref().
相关文件: src/buffer.c

Patch 8.0.0649
问题:　　When opening a help file the filetype is set several times.
解决方案:　When setting the filetype to the same value from a modeline, don't
　　trigger FileType autocommands. Don't set the filetype to "help"
　　when it's already set correctly.
相关文件:　src/ex_cmds.c, src/option.c, runtime/filetype.vim

Patch 8.0.0650
问题:　　For extra help files the filetype is set more than once.
解决方案:　In *.txt files check that there is no help file modline.
相关文件:　runtime/filetype.vim

Patch 8.0.0651 (after 8.0.0649)
问题:　　Build failure without the auto command feature.
解决方案:　Add #ifdef. (closes #1782)
相关文件:　src/ex_cmds.c

Patch 8.0.0652
问题:　　Unicode information is outdated.
解决方案:　Update to Unicode 10. (Christian Brabandt)
相关文件:　runtime/tools/unicode.vim, src/mbyte.c

Patch 8.0.0653
问题:　　The default highlight for QuickFixLine does not work for several
　　color schemes. (Manas Thakur)
解决方案:　Make the default use the old color. (closes #1780)
相关文件:　src/syntax.c

Patch 8.0.0654
问题:　　Text found after :endfunction is silently ignored.
解决方案:　Give a warning if 'verbose' is set. When | or \n are used,
　　execute the text as a command.
相关文件:　src/testdir/test_vimscript.vim, src/userfunc.c,
　　runtime/doc/eval.txt

Patch 8.0.0655
问题:　　Not easy to make sure a function does not exist.
解决方案:　Add ! as an optional argument to :delfunc.
相关文件:　src/userfunc.c, src/ex_cmds.h, src/testdir/test_vimscript.vim

Patch 8.0.0656
问题:　　Cannot use ! after some user commands.
解决方案:　Properly check for existing command. (Hirohito Higashi)
相关文件:　src/ex_docmd.c, src/testdir/test_vimscript.vim

Patch 8.0.0657
问题:　　Cannot get and set quickfix list items.
解决方案:　Add the "items" argument to getqflist() and setqflist(). (Yegappan
　　Lakshmanan)
相关文件:　runtime/doc/eval.txt, src/quickfix.c,
　　src/testdir/test_quickfix.vim

Patch 8.0.0658

问题: Spell test is old style.
解决方案: Turn the spell test into a new style test (pschuh, closes #1778)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
            src/testdir/test58.in, src/testdir/test58.ok,
            src/testdir/test_spell.vim

Patch 8.0.0659
问题: No test for conceal mode.
解决方案: Add a conceal mode test. (Dominique Pelle, closes #1783)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_syntax.vim

Patch 8.0.0660
问题: Silent install on MS-Windows does show a dialog.
解决方案: Add /SD to the default choice. (allburov, closes #1772)
相关文件: nsis/gvim.nsi

Patch 8.0.0661
问题: Recognizing urxvt mouse codes does not work well.
解决方案: Recognize "Esc[*M" and "Esc[*m". (Maurice Bos, closes #1486)
相关文件: src/keymap.h, src/misc2.c, src/os_unix.c, src/term.c

Patch 8.0.0662 (after 8.0.0659)
问题: Stray FIXME for fixed problem.
解决方案: Remove the comment. (Dominique Pelle)
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0663
问题: Giving an error message only when 'verbose' set is unexpected.
解决方案: Give a warning message instead.
相关文件: src/message.c, src/proto/message.pro, src/userfunc.c,
            src/testdir/test_vimscript.vim, runtime/doc/eval.txt

Patch 8.0.0664 (after 8.0.0661)
问题: Mouse does not work in tmux. (lilydjwg)
解决方案: Add flag for SGR release being present.
相关文件: src/term.c

Patch 8.0.0665 (after 8.0.0661)
问题: Warning for uninitialized variable. (Tony Mechelynck)
解决方案: Initialize it.
相关文件: src/term.c

Patch 8.0.0666
问题: Dead for loop. (Coverity)
解决方案: Remove the for loop.
相关文件: src/term.c

Patch 8.0.0667
问题: Memory access error when command follows :endfunction. (Nikolai
            Pavlov)
解决方案: Make memory handling in :function straightforward. (closes #1793)
相关文件: src/userfunc.c, src/testdir/test_vimscript.vim

Patch 8.0.0668 (after 8.0.0660)

问题:　　Nsis installer script does not work. (Christian Brabandt)
解决方案:　Fix the syntax of /SD.
相关文件:　nsis/gvim.nsi

Patch 8.0.0669
问题:　　In Insert mode, CTRL-N at start of the buffer does not work
　　correctly. (zuloloxi)
解决方案:　Wrap around the start of the buffer. (Christian Brabandt)
相关文件:　src/edit.c, src/testdir/test_popup.vim

Patch 8.0.0670
问题:　　Can't use input() in a timer callback. (Cosmin Popescu)
解决方案:　Reset vgetc_busy and set timer_busy. (Ozaki Kiichi, closes #1790,
　　closes #1129)
相关文件:　src/evalfunc.c, src/ex_cmds2.c, src/globals.h,
　　src/testdir/test_timers.vim

Patch 8.0.0671
问题:　　When a function invoked from a timer calls confirm() and the user
　　types CTRL-C then Vim hangs.
解决方案:　Reset typebuf_was_filled. (Ozaki Kiichi, closes #1791)
相关文件:　src/getchar.c

Patch 8.0.0672
问题:　　Third item of synconcealed() changes too often. (Dominique Pelle)
解决方案:　Reset the sequence number at the start of each line.
相关文件:　src/syntax.c, src/testdir/test_syntax.vim, runtime/doc/eval.txt

Patch 8.0.0673 (after 8.0.0673)
问题:　　Build failure without conceal feature.
解决方案:　Add #ifdef.
相关文件:　src/syntax.c

Patch 8.0.0674 (after 8.0.0670)
问题:　　Cannot build with eval but without timers.
解决方案:　Add #ifdef (John Marriott)
相关文件:　src/evalfunc.c

Patch 8.0.0675
问题:　　'colorcolumn' has a higher priority than 'hlsearch', it should be
　　the other way around. (Nazri Ramliy)
解决方案:　Change the priorities. (LemonBoy, closes #1794)
相关文件:　src/screen.c, src/testdir/test_listlbr_utf8.vim

Patch 8.0.0676
问题:　　Crash when closing the quickfix window in a FileType autocommand
　　that triggers when the quickfix window is opened.
解决方案:　Save the new value before triggering the OptionSet autocommand.
　　Add the "starting" flag to test_override() to make the text work.
相关文件:　src/evalfunc.c, src/option.c, runtime/doc/eval.txt

Patch 8.0.0677
问题:　　Setting 'filetype' internally may cause the current buffer and
　　window to change unexpectedly.

解决方案: Set curbuf_lock. (closes #1734)
相关文件: src/quickfix.c, src/ex_cmds.c, src/ex_getln.c,
           src/testdir/test_quickfix.vim

Patch 8.0.0678
问题: When 'equalalways' is set and closing a window in a separate
           frame, not all window sizes are adjusted. (Glacambre)
解决方案: Resize all windows if the new current window is not in the same
           frame as the closed window. (closes #1707)
相关文件: src/window.c, src/testdir/test_window_cmd.vim

Patch 8.0.0679 (after 8.0.0678)
问题: Using freed memory.
解决方案: Get the parent frame pointer earlier.
相关文件: src/window.c

Patch 8.0.0680 (after 8.0.0612)
问题: Plugins in start packages are sourced twice. (mseplowitz)
解决方案: Use the unmodified runtime path when loading plugins (test by Ingo
           Karkat, closes #1801)
相关文件: src/testdir/test_startup.vim, src/main.c, src/ex_cmds2.c,
           src/proto/ex_cmds2.pro

Patch 8.0.0681
问题: Unnamed register only contains the last deleted text when
           appending deleted text to a register. (Wolfgang Jeltsch)
解决方案: Only set y_previous when not using y_append. (Christian Brabandt)
相关文件: src/ops.c, src/testdir/test_put.vim

Patch 8.0.0682
问题: No test for synIDtrans().
解决方案: Add a test. (Dominique Pelle, closes #1796)
相关文件: src/testdir/test_syntax.vim

Patch 8.0.0683
问题: When using a visual bell there is no delay, causing the flash to
           be very short, possibly unnoticeable. Also, the flash and the
           beep can lockup the UI when repeated often.
解决方案: Do the delay in Vim or flush the output before the delay. Limit the
           bell to once per half a second. (Ozaki Kiichi, closes #1789)
相关文件: src/misc1.c, src/proto/term.pro, src/term.c

Patch 8.0.0684
问题: Old style tests are not nice.
解决方案: Turn two tests into new style. (pschuh, closes #1797)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/test82.in, src/testdir/test82.ok,
           src/testdir/test90.in, src/testdir/test90.ok,
           src/testdir/test_sha256.vim, src/testdir/test_utf8_comparisons.vim

Patch 8.0.0685
问题: When making backups is disabled and conversion with iconv fails
           the written file is truncated. (Luo Chen)
解决方案: First try converting the file and write the file only when it did

not fail. (partly by Christian Brabandt)
相关文件: src/fileio.c, src/testdir/test_writefile.vim

Patch 8.0.0686
问题: When typing CTRL-L in a window that's not the first one, another redraw will happen later. (Christian Brabandt)
解决方案: Reset must_redraw after calling screenclear().
相关文件: src/screen.c

Patch 8.0.0687
问题: Minor issues related to quickfix.
解决方案: Set the proper return status for all cases in setqflist() and at test cases for this. Move the "adding" flag outside of FEAT_WINDOWS. Minor update to the setqflist() help text. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.0688
问题: Cannot resize the window in a FileType autocommand. (Ingo Karkat)
解决方案: Add the CMDWIN flag to :resize. (test by Ingo Karkat, closes #1804)
相关文件: src/ex_cmds.h, src/testdir/test_quickfix.vim

Patch 8.0.0689
问题: The ~ character is not escaped when adding to the search pattern with CTRL-L. (Ramel Eshed)
解决方案: Escape the character. (Christian Brabandt)
相关文件: src/ex_getln.c, src/testdir/test_search.vim

Patch 8.0.0690
问题: Compiler warning on non-Unix system.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/term.c

Patch 8.0.0691
问题: Compiler warning without the linebreak feature.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/edit.c

Patch 8.0.0692
问题: Using CTRL-G with 'incsearch' and ? goes in the wrong direction. (Ramel Eshed)
解决方案: Adjust search_start. (Christian Brabandt)
相关文件: src/ex_getln.c, src/testdir/test_search.vim

Patch 8.0.0693
问题: No terminal emulator support. Cannot properly run commands in the GUI. Cannot run a job interactively with an ssh connection.
解决方案: Very early implementation of the :terminal command. Includes libvterm converted to ANSI C. Many parts still missing.
相关文件: src/feature.h, src/Makefile, src/configure.ac, src/auto/configure, src/config.mk.in, src/config.h.in, src/terminal.c, src/structs.h, src/ex_cmdidxs.h, src/ex_docmd.c, src/option.c, src/option.h,

src/evalfunc.c, src/proto/terminal.pro, src/proto.h,
runtime/doc/terminal.txt, runtime/doc/Makefile, Filelist,
src/libvterm/.bzrignore, src/libvterm/.gitignore,
src/libvterm/LICENSE, src/libvterm/README, src/libvterm/Makefile,
src/libvterm/tbl2inc_c.pl, src/libvterm/vterm.pc.in,
src/libvterm/bin/unterm.c, src/libvterm/bin/vterm-ctrl.c,
src/libvterm/bin/vterm-dump.c, src/libvterm/doc/URLs,
src/libvterm/doc/seqs.txt, src/libvterm/include/vterm.h,
src/libvterm/include/vterm_keycodes.h,
src/libvterm/src/encoding.c,
src/libvterm/src/encoding/DECdrawing.inc,
src/libvterm/src/encoding/DECdrawing.tbl,
src/libvterm/src/encoding/uk.inc,
src/libvterm/src/encoding/uk.tbl, src/libvterm/src/keyboard.c,
src/libvterm/src/mouse.c, src/libvterm/src/parser.c,
src/libvterm/src/pen.c, src/libvterm/src/rect.h,
src/libvterm/src/screen.c, src/libvterm/src/state.c,
src/libvterm/src/unicode.c, src/libvterm/src/utf8.h,
src/libvterm/src/vterm.c, src/libvterm/src/vterm_internal.h,
src/libvterm/t/02parser.test, src/libvterm/t/03encoding_utf8.test,
src/libvterm/t/10state_putglyph.test,
src/libvterm/t/11state_movecursor.test,
src/libvterm/t/12state_scroll.test,
src/libvterm/t/13state_edit.test,
src/libvterm/t/14state_encoding.test,
src/libvterm/t/15state_mode.test,
src/libvterm/t/16state_resize.test,
src/libvterm/t/17state_mouse.test,
src/libvterm/t/18state_termprops.test,
src/libvterm/t/20state_wrapping.test,
src/libvterm/t/21state_tabstops.test,
src/libvterm/t/22state_save.test,
src/libvterm/t/25state_input.test,
src/libvterm/t/26state_query.test,
src/libvterm/t/27state_reset.test,
src/libvterm/t/28state_dbl_wh.test,
src/libvterm/t/29state_fallback.test, src/libvterm/t/30pen.test,
src/libvterm/t/40screen_ascii.test,
src/libvterm/t/41screen_unicode.test,
src/libvterm/t/42screen_damage.test,
src/libvterm/t/43screen_resize.test,
src/libvterm/t/44screen_pen.test,
src/libvterm/t/45screen_protect.test,
src/libvterm/t/46screen_extent.test,
src/libvterm/t/47screen_dbl_wh.test,
src/libvterm/t/48screen_termprops.test,
src/libvterm/t/90vttest_01-movement-1.test,
src/libvterm/t/90vttest_01-movement-2.test,
src/libvterm/t/90vttest_01-movement-3.test,
src/libvterm/t/90vttest_01-movement-4.test,
src/libvterm/t/90vttest_02-screen-1.test,
src/libvterm/t/90vttest_02-screen-2.test,
src/libvterm/t/90vttest_02-screen-3.test,
src/libvterm/t/90vttest_02-screen-4.test,

src/libvterm/t/92lp1640917.test, src/libvterm/t/harness.c,
src/libvterm/t/run-test.pl

Patch 8.0.0694
问题: Building in shadow directory does not work. Running Vim fails.
解决方案: Add the new libvterm directory. Add missing change in command
list.
相关文件: src/Makefile, src/ex_cmds.h

Patch 8.0.0695
问题: Missing dependencies breaks parallel make.
解决方案: Add dependencies for terminal.o.
相关文件: src/Makefile

Patch 8.0.0696
问题: The .inc files are missing in git. (Nazri Ramliy)
解决方案: Remove the .inc line from .gitignore.
相关文件: src/libvterm/.gitignore

Patch 8.0.0697
问题: Recorded key sequences may become invalid.
解决方案: Add back KE_SNIFF removed in 7.4.1433. Use fixed numbers for the
key_extra enum.
相关文件: src/keymap.h

Patch 8.0.0698
问题: When a timer uses ":pyeval" or another Python command and it
happens to be triggered while exiting a Crash may happen.
(Ricky Zhou)
解决方案: Avoid running a Python command after python_end() was called.
Do not trigger timers while exiting. (closes #1824)
相关文件: src/if_python.c, src/if_python3.c, src/ex_cmds2.c

Patch 8.0.0699
问题: Checksum tests are not actually run.
解决方案: Add the tests to the list. (Dominique Pelle, closes #1819)
相关文件: src/testdir/test_alot.vim, src/testdir/test_alot_utf8.vim

Patch 8.0.0700
问题: Segfault with QuitPre autocommand closes the window. (Marek)
解决方案: Check that the window pointer is still valid. (Christian Brabandt,
closes #1817)
相关文件: src/testdir/test_tabpage.vim, src/ex_docmd.c

Patch 8.0.0701
问题: System test failing when using X11 forwarding.
解决方案: Set $XAUTHORITY before changing $HOME. (closes #1812)
Also use a better check for the exit value.
相关文件: src/testdir/setup.vim, src/testdir/test_system.vim

Patch 8.0.0702
问题: An error in a timer can make Vim unusable.
解决方案: Don't set the error flag or exception from a timer. Stop a timer
if it causes an error 3 out of 3 times. Discard an exception

caused inside a timer.
相关文件: src/ex_cmds2.c, src/structs.h, src/testdir/test_timers.vim, runtime/doc/eval.txt

Patch 8.0.0703
问题: Illegal memory access with empty :doau command.
解决方案: Check the event for being out of range. (James McCoy)
相关文件: src/testdir/test_autocmd.vim, src/fileio.c

Patch 8.0.0704
问题: Problems with autocommands when opening help.
解决方案: Avoid using invalid "varp" value. Allow using :wincmd if buffer is locked. (closes #1806, closes #1804)
相关文件: src/option.c, src/ex_cmds.h

Patch 8.0.0705 (after 8.0.0702)
问题: Crash when there is an error in a timer callback. (Aron Griffis, Ozaki Kiichi)
解决方案: Check did_throw before discarding an exception. NULLify current_exception when no longer valid.
相关文件: src/ex_eval.c, src/ex_cmds2.c

Patch 8.0.0706
问题: Crash when cancelling the cmdline window in Ex mode. (James McCoy)
解决方案: Do not set cmdbuff to NULL, make it empty.
相关文件: src/ex_getln.c

Patch 8.0.0707
问题: Freeing wrong memory when manipulating buffers in autocommands. (James McCoy)
解决方案: Also set the w_s pointer if w_buffer was NULL.
相关文件: src/ex_cmds.c

Patch 8.0.0708
问题: Some tests are old style.
解决方案: Change a few tests from old style to new style. (pschuh, closes #1813)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_ming.mak, src/testdir/Make_vms.mms, src/testdir/main.aap, src/testdir/test23.in, src/testdir/test23.ok, src/testdir/test24.in, src/testdir/test24.ok, src/testdir/test26.in, src/testdir/test26.ok, src/testdir/test67.in, src/testdir/test67.ok, src/testdir/test75.in, src/testdir/test75.ok, src/testdir/test97.in, src/testdir/test97.ok, src/testdir/test_comparators.in, src/testdir/test_comparators.ok, src/testdir/test_comparators.vim, src/testdir/test_escaped_glob.vim, src/testdir/test_exec_while_if.vim, src/testdir/test_exists_autocmd.vim, src/testdir/test_getcwd.in, src/testdir/test_getcwd.ok, src/testdir/test_getcwd.vim, src/testdir/test_maparg.vim, src/testdir/test_plus_arg_edit.vim, src/testdir/test_regex_char_classes.vim

Patch 8.0.0709
问题:　　Libvterm cannot use vsnprintf(), it does not exist in C90.
解决方案:　Use vim_vsnprintf() instead.
相关文件:　src/message.c, src/Makefile, src/proto.h, src/evalfunc.c,
　　src/netbeans.c, src/libvterm/src/vterm.c

Patch 8.0.0710
问题:　　A job that writes to a buffer clears command line completion.
　　(Ramel Eshed)
解决方案:　Do not redraw while showing the completion menu.
相关文件:　src/screen.c

Patch 8.0.0711 (after 8.0.0710)
问题:　　Cannot build without the wildmenu feature.
解决方案:　Add #ifdef
相关文件:　src/screen.c

Patch 8.0.0712
问题:　　The terminal implementation is incomplete.
解决方案:　Add the 'termkey' option.
相关文件:　src/option.c, src/option.h, src/structs.h

Patch 8.0.0713 (after 8.0.0712)
问题:　　'termkey' option not fully implemented.
解决方案:　Add initialisation.
相关文件:　src/option.c

Patch 8.0.0714
问题:　　When a timer causes a command line redraw the " that is displayed
　　for CTRL-R goes missing.
解决方案:　Remember an extra character to display.
相关文件:　src/ex_getln.c

Patch 8.0.0715
问题:　　Writing to the wrong buffer if the buffer that a channel writes to
　　was closed.
解决方案:　Do not write to a buffer that was unloaded.
相关文件:　src/channel.c, src/testdir/test_channel.vim,
　　src/testdir/test_channel_write.py

Patch 8.0.0716
问题:　　Not easy to start Vim cleanly without changing the viminfo file.
　　Not possible to know whether the -i command line flag was used.
解决方案:　Add the --clean command line argument. Add the 'viminfofile'
　　option. Add "-u DEFAULTS".
相关文件:　src/main.c, runtime/doc/starting.txt, src/option.c, src/option.h,
　　src/ex_cmds.c, src/globals.h, runtime/doc/options.txt

Patch 8.0.0717
问题:　　Terminal feature not included in :version output.
解决方案:　Add +terminal or -terminal.
相关文件:　src/version.c, src/terminal.c

Patch 8.0.0718

问题: Output of job in terminal is not displayed.
解决方案: Connect the job output to the terminal.
相关文件: src/channel.c, src/proto/channel.pro, src/terminal.c,
            src/proto/terminal.pro, src/channel.c, src/proto/channel.pro,
            src/evalfunc.c, src/screen.c, src/proto/screen.pro

Patch 8.0.0719
问题: Build failure without +terminal feature.
解决方案: Add #ifdefs.
相关文件: src/screen.c, src/channel.c

Patch 8.0.0720
问题: Unfinished mapping not displayed when running timer.
解决方案: Also use the extra_char while waiting for a mapping and digraph.
            (closes #1844)
相关文件: src/ex_getln.c

Patch 8.0.0721
问题: :argedit can only have one argument.
解决方案: Allow for multiple arguments. (Christian Brabandt)
相关文件: runtime/doc/editing.txt, src/ex_cmds.h, src/ex_cmds2.c,
            src/testdir/test_arglist.vim

Patch 8.0.0722
问题: Screen is messed by timer up at inputlist() prompt.
解决方案: Set state to ASKMORE. (closes #1843)
相关文件: src/misc1.c

Patch 8.0.0723 (after 8.0.0721)
问题: Arglist test fails if file name case is ignored.
解决方案: Wipe existing buffers, check for fname_case property.
相关文件: src/testdir/test_arglist.vim

Patch 8.0.0724
问题: The message for yanking doesn't indicate the register.
解决方案: Show the register name in the "N lines yanked" message. (LemonBoy,
            closes #1803, closes #1809)
相关文件: src/ops.c, src/Makefile, src/testdir/test_registers.vim,
            src/testdir/Make_all.mak

Patch 8.0.0725
问题: A terminal window does not handle keyboard input.
解决方案: Add terminal_loop(). ":term bash -i" sort of works now.
相关文件: src/main.c, src/terminal.c, src/proto/terminal.pro, src/normal.c

Patch 8.0.0726
问题: Translations cleanup script is too conservative.
解决方案: Also delete untranslated messages.
相关文件: src/po/cleanup.vim

Patch 8.0.0727
问题: Message about what register to yank into is not translated.
            (LemonBoy)
解决方案: Add _().

相关文件: src/ops.c

Patch 8.0.0728
问题: The terminal structure is never freed.
解决方案: Free the structure and unreference what it contains.
相关文件: src/terminal.c, src/buffer.c, src/proto/terminal.pro,
src/channel.c, src/proto/channel.pro, src/evalfunc.c

Patch 8.0.0729
问题: The help for the terminal configure option is wrong.
解决方案: Change "Disable" to "Enable". (E Kawashima, closes #1849)
Improve alignment.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.0730
问题: Terminal feature only supports Unix-like systems.
解决方案: Prepare for adding an MS-Windows implementation.
相关文件: src/terminal.c

Patch 8.0.0731
问题: Cannot build the terminal feature on MS-Windows.
解决方案: Add the Makefile changes. (Yasuhiro Matsumoto, closes #1851)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.0732
问题: When updating a buffer for a callback the modeless selection is
lost.
解决方案: Do not insert or delete screen lines when redrawing for a callback
and there is a modeless selection.
相关文件: src/screen.c

Patch 8.0.0733
问题: Can only add entries to one list in the quickfix stack.
解决方案: Move state variables from qf_list_T to qf_list_T. (Yegappan
Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.0734
问题: The script to check translations can be improved.
解决方案: Restore the view when no errors are found. Check for matching
line break at the end of the message. (Christian Brabandt)
相关文件: src/po/check.vim

Patch 8.0.0735
问题: There is no way to notice that the quickfix window contents has
changed.
解决方案: Increment b:changedtick when updating the quickfix window.
(Yegappan Lakshmanan)
相关文件: runtime/doc/quickfix.txt, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.0.0736
问题: The OptionSet autocommand event is not triggered when entering
diff mode.

解决方案: use set_option_value() instead of setting the option directly.
           Change the tests from old to new style. (Christian Brabandt)
相关文件: src/diff.c, src/testdir/Make_all.mak, src/Makefile,
           src/testdir/test_autocmd.vim, src/testdir/test_autocmd_option.in,
           src/testdir/test_autocmd_option.ok

Patch 8.0.0737
问题: Crash when X11 selection is very big.
解决方案: Use static items instead of allocating them. Add callbacks.
           (Ozaki Kiichi)
相关文件: src/testdir/shared.vim, src/testdir/test_quotestar.vim,
           src/ui.c

Patch 8.0.0738
问题: Cannot use the mouse to resize window while the focus is in a
           terminal window.
解决方案: Recognize nice mouse events in the terminal window. A few more
           fixes for the terminal window.
相关文件: src/terminal.c

Patch 8.0.0739
问题: Terminal resizing doesn't work well.
解决方案: Resize the terminal to the Vim window and the other way around.
           Avoid mapping typed keys. Set the environment properly.
相关文件: src/terminal.c, src/os_unix.c, src/structs.h

Patch 8.0.0740
问题: Cannot resize a terminal window by the command running in it.
解决方案: Add support for the window size escape sequence. Make BS work.
相关文件: src/terminal.c, src/libvterm/src/state.c

Patch 8.0.0741
问题: Cannot build with HPUX.
解决方案: Rename envbuf_TERM to envbuf_Term. (John Marriott)
相关文件: src/os_unix.c

Patch 8.0.0742
问题: Terminal feature does not work on MS-Windows.
解决方案: Use libvterm and libwinpty on MS-Windows. (Yasuhiro Matsumoto)
相关文件: src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/channel.c,
           src/proto/channel.pro, src/terminal.c

Patch 8.0.0743
问题: The 'termsize' option can be set to an invalid value.
解决方案: Check the 'termsize' option to be valid.
相关文件: src/option.c, src/testdir/gen_opt_test.vim

Patch 8.0.0744
问题: A terminal window uses pipes instead of a pty.
解决方案: Add pty support.
相关文件: src/structs.h, src/os_unix.c, src/terminal.c, src/channel.c,
           src/proto/os_unix.pro, src/os_win32.c, src/proto/os_win32.pro

Patch 8.0.0745

问题:　　multi-byte characters in a terminal window are not displayed
properly.
解决方案:　Set the unused screen characters. (Yasuhiro Matsumoto, closes
#1857)
相关文件:　src/terminal.c

Patch 8.0.0746
问题:　　When :term fails the job is not properly cleaned up.
解决方案:　Free the terminal. Handle a job that failed to start. (closes
#1858)
相关文件:　src/os_unix.c, src/channel.c, src/terminal.c

Patch 8.0.0747
问题:　　:terminal without an argument doesn't work.
解决方案:　Use the 'shell' option. (Yasuhiro Matsumoto, closes #1860)
相关文件:　src/terminal.c

Patch 8.0.0748
问题:　　When running Vim in a terminal window it does not detect the right
number of colors available.
解决方案:　Detect the version string that libvterm returns. Pass the number
of colors in $COLORS.
相关文件:　src/term.c, src/os_unix.c

Patch 8.0.0749
问题:　　Some unicode digraphs are hard to remember.
解决方案:　Add alternatives with a backtick. (Chris Harding, closes #1861)
相关文件:　src/digraph.c

Patch 8.0.0750
问题:　　OpenPTY missing in non-GUI build.
解决方案:　Always include pty.c, add an #ifdef to skip over the contents.
相关文件:　src/pty.c, src/Makefile

Patch 8.0.0751 (after 8.0.0750)
问题:　　OpenPTY missing with some combination of features. (Kazunobu
Kuriyama)
解决方案:　Adjust #ifdef. Also include pty.pro when needed.
相关文件:　src/pty.c, src/misc2.c, src/proto.h

Patch 8.0.0752
问题:　　Build fails on MS-Windows.
解决方案:　Change #ifdef for set_color_count().
相关文件:　src/term.c

Patch 8.0.0753
问题:　　A job running in a terminal does not get notified of changes in
the terminal size.
解决方案:　Use ioctl() and SIGWINCH to report the terminal size.
相关文件:　src/terminal.c, src/os_unix.c, src/proto/os_unix.pro

Patch 8.0.0754
问题:　　Terminal window does not support colors.
解决方案:　Lookup the color attribute.

相关文件: src/terminal.c, src/syntax.c, src/proto/syntax.pro

Patch 8.0.0755
问题: Terminal window does not have colors in the GUI.
解决方案: Lookup the GUI color.
相关文件: src/terminal.c, src/syntax.c, src/proto/syntax.pro, src/term.c,
src/proto/term.pro, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro,
src/gui_x11.c, src/proto/gui_x11.pro, src/gui_mac.c,
src/proto/gui_mac.pro, src/gui_photon.c, src/proto/gui_photon.pro,
src/gui_w32.c, src/proto/gui_w32.pro,

Patch 8.0.0756
问题: Cannot build libvterm with MSVC.
解决方案: Add an MSVC Makefile to libvterm. (Yasuhiro Matsumoto, closes
#1865)
相关文件: src/INSTALLpc.txt, src/Make_mvc.mak, src/libvterm/Makefile.msc

Patch 8.0.0757
问题: Libvterm MSVC Makefile not included in the distribution.
解决方案: Add the file to the list.
相关文件: Filelist

Patch 8.0.0758
问题: Possible crash when using a terminal window.
解决方案: Check for NULL pointers. (Yasuhiro Matsumoto, closes #1864)
相关文件: src/terminal.c

Patch 8.0.0759
问题: MS-Windows: terminal does not adjust size to the Vim window size.
解决方案: Add a call to winpty_set_size(). (Yasuhiro Matsumoto, closes #1863)
相关文件: src/terminal.c

Patch 8.0.0760
问题: Terminal window colors wrong with 'termguicolors'.
解决方案: Add 'termguicolors' support.
相关文件: src/terminal.c, src/syntax.c, src/proto/syntax.pro

Patch 8.0.0761
问题: Options of a buffer for a terminal window are not set properly.
解决方案: Add "terminal" value for 'buftype'. Make 'buftype' and
'bufhidden' not depend on the quickfix feature.
Also set the buffer name and show "running" or "finished" in the
window title.
相关文件: src/option.c, src/terminal.c, src/proto/terminal.pro,
runtime/doc/options.txt, src/quickfix.c, src/proto/quickfix.pro,
src/structs.h, src/buffer.c, src/ex_docmd.c, src/fileio.c,
src/channel.c

Patch 8.0.0762
问题: ml_get error with :psearch in buffer without a name. (Dominique
Pelle)
解决方案: Use the buffer number instead of the file name. Check the cursor
position.
相关文件: src/search.c, src/testdir/test_preview.vim, src/Makefile,

src/testdir/Make_all.mak

Patch 8.0.0763
问题: Libvterm can be improved.
解决方案: Various small improvements, more comments.
相关文件: src/libvterm/README, src/libvterm/include/vterm.h,
src/libvterm/include/vterm_keycodes.h,
src/libvterm/src/keyboard.c, src/libvterm/src/parser.c,
src/libvterm/src/screen.c, src/libvterm/src/state.c

Patch 8.0.0764
问题: 'termkey' does not work yet.
解决方案: Implement 'termkey'.
相关文件: src/terminal.c, src/option.c, src/proto/option.pro

Patch 8.0.0765
问题: Build fails with tiny features.
解决方案: Adjust #ifdef. (John Marriott)
相关文件: src/option.c, src/option.h

Patch 8.0.0766
问题: Option test fails with +terminal feature.
解决方案: Fix using the right option when checking the value.
相关文件: src/option.c

Patch 8.0.0767
问题: Build failure with Athena and Motif.
解决方案: Move local variable declarations. (Kazunobu Kuriyama)
相关文件: src/gui_x11.c

Patch 8.0.0768
问题: Terminal window status shows "[Scratch]".
解决方案: Show "[Terminal]" when no title was set. (Yasuhiro Matsumoto)
Store the terminal title that vterm sends and use it. Update the
special buffer name. (closes #1869)
相关文件: src/terminal.c, src/proto/terminal.pro, src/buffer.c

Patch 8.0.0769
问题: Build problems with terminal on MS-Windows using MSVC.
解决方案: Remove stdbool.h dependency. Only use ScreenLinesUC when it was
allocated. Fix typos. (Ken Takata)
相关文件: src/libvterm/bin/vterm-ctrl.c, runtime/doc/terminal.txt,
src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak,
src/libvterm/Makefile.msc, src/terminal.c

Patch 8.0.0770
问题: Compiler warning for missing field initializer.
解决方案: Add two more values. (Yegappan Lakshmanan)
相关文件: src/libvterm/src/encoding.c

Patch 8.0.0771
问题: Cursor in a terminal window not always updated in the GUI.
解决方案: Call gui_update_cursor(). (Yasuhiro Matsumoto, closes #1868)
相关文件: src/terminal.c

Patch 8.0.0772
问题:	Other stdbool.h dependencies in libvterm.
解决方案:	Remove the dependency and use TRUE/FALSE/int. (Ken Takata)
相关文件:	src/libvterm/include/vterm.h, src/libvterm/src/mouse.c,
src/libvterm/src/pen.c, src/libvterm/t/harness.c,
src/libvterm/bin/unterm.c

Patch 8.0.0773
问题:	Mixing 32 and 64 bit libvterm builds fails.
解决方案:	Use OUTDIR. (Ken Takata)
相关文件:	src/Make_cyg_ming.mak, src/Make_mvc.mak, src/libvterm/Makefile.msc

Patch 8.0.0774
问题:	Build failure without the multi-byte feature on HPUX.
解决方案:	Move #ifdefs. (John Marriott)
相关文件:	src/term.c

Patch 8.0.0775
问题:	In a terminal the cursor is updated too often.
解决方案:	Only flush when needed. (Yasuhiro Matsumoto). Remember whether the
cursor is visible. (closes #1873)
相关文件:	src/terminal.c

Patch 8.0.0776
问题:	Function prototypes missing without the quickfix feature. (Tony
Mechelynck)
解决方案:	Move non-quickfix functions to buffer.c.
相关文件:	src/buffer.c, src/proto/buffer.pro, src/quickfix.c,
src/proto/quickfix.pro

Patch 8.0.0777
问题:	Compiler warnings with 64 bit compiler.
解决方案:	Add type casts. (Mike Williams)
相关文件:	src/libvterm/src/pen.c, src/libvterm/src/state.c, src/terminal.c

Patch 8.0.0778
问题:	In a terminal the cursor may be hidden and screen updating lags
behind. (Nazri Ramliy)
解决方案:	Switch the cursor on and flush output when needed. (Ozaki Kiichi)
相关文件:	src/terminal.c

Patch 8.0.0779
问题:	:term without an argument uses empty buffer name but runs the
shell.
解决方案:	Change the command to the shell earlier.
相关文件:	src/terminal.c

Patch 8.0.0780
问题:	Build failure on Travis.
解决方案:	Set distribution explicitly. Use Lua and Ruby dev. (Ken Takata,
closes #1884)
相关文件:	.travis.yml

Patch 8.0.0781
问题:　　MS-Windows: Memory leak when using :terminal.
解决方案:　Handle failures properly. (Ken Takata)
相关文件:　src/terminal.c

Patch 8.0.0782
问题:　　Using freed memory in quickfix code. (Dominique Pelle)
解决方案:　Handle a help window differently. (Yegappan Lakshmanan)
相关文件:　src/buffer.c, src/proto/buffer.pro, src/quickfix.c,
　　src/testdir/test_quickfix.vim, src/ex_cmds.c, src/window.c

Patch 8.0.0783
问题:　　Job of terminal may be freed too early.
解决方案:　Increment job refcount. (Yasuhiro Matsumoto)
相关文件:　src/terminal.c

Patch 8.0.0784
问题:　　Job of terminal may be garbage collected.
解决方案:　Set copyID on job in terminal. (Ozaki Kiichi)
相关文件:　src/terminal.c, src/eval.c, src/proto/terminal.pro

Patch 8.0.0785
问题:　　Wildcards are not expanded for :terminal.
解决方案:　Add FILES to the command flags. (Yasuhiro Matsumoto, closes #1883)
　　Also complete commands.
相关文件:　src/ex_cmds.h, src/ex_docmd.c

Patch 8.0.0786
问题:　　Build failures on Travis.
解决方案:　Go back to precise temporarily. Disable coverage with clang.
相关文件:　.travis.yml

Patch 8.0.0787
问题:　　Cannot send CTRL-W command to terminal job.
解决方案:　Make CTRL-W . a prefix for sending a key to the job.
相关文件:　src/terminal.c, runtime/doc/terminal.txt, src/option.c

Patch 8.0.0788
问题:　　MS-Windows: cannot build with terminal feature.
解决方案:　Move set_ref_in_term(). (Ozaki Kiichi)
相关文件:　src/terminal.c

Patch 8.0.0789
问题:　　When splitting a terminal window where the terminal follows the
　　size of the window doesn't work.
解决方案:　Use the size of the smallest window. (Yasuhiro Matsumoto, closes
　　#1885)
相关文件:　src/terminal.c

Patch 8.0.0790
问题:　　MSVC compiler warning for strncpy in libvterm.
解决方案:　Add a define to stop the warnings. (Mike Williams)
相关文件:　src/Make_mvc.mak

Patch 8.0.0791
问题:　　Terminal colors depend on the system.
解决方案:　Use the highlight color lookup tables.
相关文件:　src/syntax.c, src/proto/syntax.pro, src/terminal.c

Patch 8.0.0792
问题:　　Spell test leaves files behind.
解决方案:　Delete the files.
相关文件:　src/testdir/test_spell.vim

Patch 8.0.0793
问题:　　Using wrong terminal name for terminal window.
解决方案:　When 'term' starts with "xterm" use it for $TERM in a terminal
　　window.
相关文件:　src/os_unix.c

Patch 8.0.0794
问题:　　The script to check translations fails if there is more than one
　　NL in one line.
解决方案:　Count the number of NL characters. Make count() accept a string.
相关文件:　src/po/check.vim, src/evalfunc.c, runtime/doc/eval.txt,
　　src/testdir/test_functions.vim

Patch 8.0.0795
问题:　　Terminal feature does not build with older MSVC.
解决方案:　Do not use stdint.h.
相关文件:　src/libvterm/include/vterm.h

Patch 8.0.0796
问题:　　No coverage on Travis with clang.
解决方案:　Use a specific coveralls version. (Ozaki Kiichi, closes #1888)
相关文件:　.travis.yml

Patch 8.0.0797
问题:　　Finished job in terminal window is not handled.
解决方案:　Add the scrollback buffer. Use it to fill the buffer when the job
　　has ended.
相关文件:　src/terminal.c, src/screen.c, src/proto/terminal.pro,
　　src/channel.c, src/os_unix.c, src/buffer.c

Patch 8.0.0798
问题:　　No highlighting in a terminal window with a finished job.
解决方案:　Highlight the text.
相关文件:　src/terminal.c, src/proto/terminal.pro, src/screen.c, src/undo.c

Patch 8.0.0799
问题:　　Missing semicolon.
解决方案:　Add it.
相关文件:　src/terminal.c

Patch 8.0.0800
问题:　　Terminal window scrollback contents is wrong.
解决方案:　Fix handling of multi-byte characters (Yasuhiro Matsumoto) Handle
　　empty lines correctly. (closes #1891)

相关文件: src/terminal.c

Patch 8.0.0801
问题: The terminal window title sometimes still says "running" even
       though the job has finished.
解决方案: Also consider the job finished when the channel has been closed.
相关文件: src/terminal.c

Patch 8.0.0802
问题: After a job exits the last line in the terminal window does not
       get color attributes.
解决方案: Fix off-by-one error.
相关文件: src/terminal.c

Patch 8.0.0803
问题: Terminal window functions not yet implemented.
解决方案: Implement several functions. Add a first test. (Yasuhiro
       Matsumoto, closes #1871)
相关文件: runtime/doc/eval.txt, src/Makefile, src/evalfunc.c,
       src/proto/evalfunc.pro, src/proto/terminal.pro, src/terminal.c,
       src/testdir/Make_all.mak, src/testdir/test_terminal.vim

Patch 8.0.0804
问题: Running tests fails when stdin is /dev/null. (James McCoy)
解决方案: Do not bail out from getting input if the --not-a-term argument
       was given. (closes #1460)
相关文件: src/eval.c, src/evalfunc.c

Patch 8.0.0805
问题: GUI test fails with gnome2.
解决方案: Set $HOME to an existing directory.
相关文件: src/testdir/setup.vim, src/testdir/runtest.vim

Patch 8.0.0806
问题: Tests may try to create XfakeHOME twice.
解决方案: Avoid loading setup.vim twice.
相关文件: src/testdir/setup.vim

Patch 8.0.0807
问题: Terminal window can't handle mouse buttons. (Hirohito Higashi)
解决方案: Implement mouse buttons and many other keys. Ignore the ones that
       are not implemented.
相关文件: src/terminal.c

Patch 8.0.0808
问题: Cannot build with terminal feature and DEBUG defined. (Christian
       Brabandt)
解决方案: Use DEBUG_LOG3().
相关文件: src/libvterm/src/pen.c

Patch 8.0.0809
问题: MS-Windows: tests hang.
解决方案: Delete the XfakeHOME directory.
相关文件: src/testdir/Make_dos.mak, src/testdir/Make_ming.mak

Patch 8.0.0810
问题:　　MS-Windows: tests still hang.
解决方案:　Only create the XfakeHOME directory if it does not exist yet.
相关文件:　src/testdir/setup.vim

Patch 8.0.0811
问题:　　MS-Windows: test_expand_dllpath fails.
解决方案:　Change backslashes to forward slashes
相关文件:　src/testdir/test_expand_dllpath.vim

Patch 8.0.0812
问题:　　Terminal window colors shift when 'number' is set. (Nazri Ramliy)
解决方案:　Use vcol instead of col.
相关文件:　src/screen.c

Patch 8.0.0813
问题:　　Cannot use Vim commands in a terminal window while the job is
　　running.
解决方案:　Implement Terminal Normal mode.
相关文件:　src/terminal.c, src/proto/terminal.pro, src/main.c, src/screen.c,
　　src/normal.c, src/option.c, runtime/doc/terminal.txt

Patch 8.0.0814 (after 8.0.0757)
问题:　　File in Filelist does not exist.
解决方案:　Remove the line.
相关文件:　Filelist

Patch 8.0.0815
问题:　　Terminal window not correctly updated when 'statusline' invokes
　　":sleep". (NIkolay Pavlov)
解决方案:　Clear got_int. Repeat redrawing when needed.
相关文件:　src/terminal.c

Patch 8.0.0816
问题:　　Crash when using invalid buffer number.
解决方案:　Check for NULL buffer. (Yasuhiro Matsumoto, closes #1899)
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0817
问题:　　Cannot get the line of a terminal window at the cursor.
解决方案:　Make the row argument optional. (Yasuhiro Matsumoto, closes #1898)
相关文件:　runtime/doc/eval.txt, src/evalfunc.c, src/terminal.c

Patch 8.0.0818
问题:　　Cannot get the cursor position of a terminal.
解决方案:　Add term_getcursor().
相关文件:　runtime/doc/eval.txt, src/evalfunc.c, src/terminal.c,
　　src/proto/terminal.pro

Patch 8.0.0819
问题:　　After changing current window the cursor position in the terminal
　　window is not updated.
解决方案:　Set w_wrow, w_wcol and w_valid.

相关文件: src/terminal.c

Patch 8.0.0820
问题: GUI: cursor in terminal window lags behind.
解决方案: call gui_update_cursor() under different conditions. (Ozaki Kiichi, closes #1893)
相关文件: src/terminal.c

Patch 8.0.0821
问题: Cannot get the title and status of a terminal window.
解决方案: Implement term_gettitle() and term_getstatus().
相关文件: src/evalfunc.c, src/terminal.c, src/proto/terminal.pro, runtime/doc/eval.txt

Patch 8.0.0822
问题: Test_with_partial_callback is a tiny bit flaky.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.0823
问题: Cannot paste text into a terminal window.
解决方案: Make CTRL-W " work.
相关文件: src/terminal.c

Patch 8.0.0824
问题: In Terminal mode the cursor and screen gets redrawn when the job produces output.
解决方案: Check for tl_terminal_mode. (partly by Yasuhiro Matsumoto, closes #1904)
相关文件: src/terminal.c

Patch 8.0.0825
问题: Not easy to see that a window is a terminal window.
解决方案: Add StatusLineTerm highlighting.
相关文件: src/option.c, src/vim.h, src/screen.c, src/syntax.c

Patch 8.0.0826
问题: Cannot use text objects in Terminal mode.
解决方案: Check for pending operator and Visual mode first. (Yasuhiro Matsumoto, closes #1906)
相关文件: src/normal.c

Patch 8.0.0827
问题: Coverity: could leak pty file descriptor, theoretically.
解决方案: If channel is NULL, free the file descriptors.
相关文件: src/os_unix.c

Patch 8.0.0828
问题: Coverity: may dereference NULL pointer.
解决方案: Bail out if calloc_state() returns NULL.
相关文件: src/regexp_nfa.c

Patch 8.0.0829
问题: A job running in a terminal window cannot easily communicate with

the Vim it is running in.
解决方案: Pass v:servername in an environment variable. (closes #1908)
相关文件: src/os_unix.c

Patch 8.0.0830
问题: Translating messages is not ideal.
解决方案: Add a remark about obsolete messages. Use msgfmt in the check script. (Christian Brabandt)
相关文件: src/po/README.txt, src/po/check.vim

Patch 8.0.0831 (after 8.0.0791)
问题: With 8 colors the bold attribute is not set properly.
解决方案: Move setting HL_TABLE() out of lookup_color. (closes #1901)
相关文件: src/syntax.c, src/proto/syntax.pro, src/terminal.c

Patch 8.0.0832
问题: Terminal function arguments are not consistent.
解决方案: Use one-based instead of zero-based rows and cols. Use "." for the current row.
相关文件: src/terminal.c, runtime/doc/eval.txt

Patch 8.0.0833
问题: Terminal test fails.
解决方案: Update the row argument to one based.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0834
问题: Can't build without the client-server feature.
解决方案: Add #ifdef.
相关文件: src/os_unix.c

Patch 8.0.0835
问题: Translations check with msgfmt does not work.
解决方案: Add a space before the file name.
相关文件: src/po/check.vim

Patch 8.0.0836
问题: When a terminal buffer is changed it can still be accidentally abandoned.
解决方案: When making a change reset the 'buftype' option.
相关文件: src/terminal.c, src/testdir/test_terminal.vim, src/option.c

Patch 8.0.0837
问题: Signs can be drawn on top of console messages.
解决方案: don't redraw at a prompt or when scrolled up. (Christian Brabandt, closes #1907)
相关文件: src/screen.c

Patch 8.0.0838
问题: Buffer hangs around when terminal window is closed.
解决方案: When the job has ended wipe out a terminal buffer when the window is closed.
相关文件: src/buffer.c, src/terminal.c, src/proto/terminal.pro, src/testdir/test_terminal.vim

Patch 8.0.0839
问题:　　Cannot kill a job in a terminal with CTRL-C.
解决方案:　Set the controlling tty and send SIGINT. (closes #1910)
相关文件:　src/os_unix.c, src/terminal.c, src/proto/os_unix.pro

Patch 8.0.0840
问题:　　MS-Windows: fopen() and open() prototypes do not match the ones in
　　the system header file. Can't build without FEAT_MBYTE.
解决方案:　Add "const". Move macro to after including protoo.h.
相关文件:　src/os_win32.c, src/proto/os_win32.pro, src/macros.h, src/vim.h

Patch 8.0.0841
问题:　　term_getline() may cause a crash.
解决方案:　Check that the row is valid. (Hirohito Higashi)
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0842
问题:　　Using slave pty after closing it.
解决方案:　Do the ioctl() before dup'ing it.
相关文件:　src/os_unix.c

Patch 8.0.0843
问题:　　MS-Windows: compiler warning for signed/unsigned.
解决方案:　Add type cast. (Yasuhiro Matsumoto, closes #1912)
相关文件:　src/terminal.c

Patch 8.0.0844
问题:　　Wrong function prototype because of missing static.
解决方案:　Add "static".
相关文件:　src/os_win32.c, src/proto/os_win32.pro

Patch 8.0.0845
问题:　　MS-Windows: missing semicolon in terminal code.
解决方案:　Add it. (Naruhiko Nishino, closes #1923)
相关文件:　src/terminal.c

Patch 8.0.0846
问题:　　Cannot get the name of the pty of a job.
解决方案:　Add the "tty" entry to the job info. (Ozaki Kiichi, closes #1920)
　　Add the term_gettty() function.
相关文件:　runtime/doc/eval.txt, src/channel.c, src/os_unix.c, src/structs.h,
　　src/terminal.c, src/proto/terminal.pro, src/evalfunc.c,
　　src/testdir/test_terminal.vim

Patch 8.0.0847
问题:　　:argadd without argument can't handle space in file name. (Harm te
　　Hennepe)
解决方案:　Escape the space. (Yasuhiro Matsumoto, closes #1917)
相关文件:　src/ex_cmds2.c, src/proto/ex_cmds2.pro,
　　src/testdir/test_arglist.vim

Patch 8.0.0848
问题:　　Using multiple ch_log functions is clumsy.

解决方案: Use variable arguments. (Ozaki Kiichi, closes #1919)
相关文件: src/channel.c, src/message.c, src/proto/channel.pro,
           src/terminal.c

Patch 8.0.0849
问题: Crash when job exit callback wipes the terminal.
解决方案: Check for b_term to be NULL. (Yasuhiro Matsumoto, closes #1922)
           Implement options for term_start() to be able to test.
           Make term_wait() more reliable.
相关文件: src/terminal.c, src/testdir/test_terminal.vim, src/channel.c

Patch 8.0.0850
问题: MS-Windows: Depending on the console encoding, an error message
           that is given during startup may be broken.
解决方案: Convert the message to the console codepage. (Yasuhiro Matsumoto,
           closes #1927)
相关文件: src/message.c

Patch 8.0.0851
问题: 'smartindent' is used even when 'indentexpr' is set.
解决方案: Ignore 'smartindent' when 'indentexpr' is set. (Hirohito Higashi)
相关文件: src/misc1.c, src/testdir/test_smartindent.vim

Patch 8.0.0852 (after 8.0.0850)
问题: MS-Windows: possible crash when giving a message on startup.
解决方案: Initialize length. (Yasuhiro Matsumoto, closes #1931)
相关文件: src/message.c

Patch 8.0.0853
问题: Crash when running terminal with unknown command.
解决方案: Check "term" not to be NULL. (Yasuhiro Matsumoto, closes #1932)
相关文件: src/terminal.c

Patch 8.0.0854
问题: No redraw after terminal was closed.
解决方案: Set typebuf_was_filled. (Yasuhiro Matsumoto, closes #1925, closes
           #1924) Add function to check for messages even when input is
           available.
相关文件: src/terminal.c, src/os_unix.c, src/proto/os_unix.pro,
           src/os_win32.c, src/proto/os_win32.pro, src/os_mswin.c

Patch 8.0.0855
问题: MS-Windows: can't get tty name of terminal.
解决方案: Use the winpty process number. (Yasuhiro Matsumoto, closes #1929)
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0856
问题: MS-Windows: terminal job doesn't take options.
解决方案: Call job_set_options(). (Yasuhiro Matsumoto)
相关文件: src/terminal.c

Patch 8.0.0857
问题: Terminal test fails on MS-Windows.
解决方案: Sleep a fraction of a second.

相关文件: src/testdir/test_terminal.vim

Patch 8.0.0858
问题: Can exit while a terminal is still running a job.
解决方案: Consider a buffer with a running job like a changed file.
相关文件: src/undo.c, src/terminal.c, src/option.h, src/buffer.c,
src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/normal.c,
src/window.c, src/testdir/test_terminal.vim

Patch 8.0.0859
问题: NULL pointer access when term_free_vterm called twice.
解决方案: Return when tl_vterm is NULL. (Yasuhiro Matsumoto, closes #1934)
相关文件: src/terminal.c

Patch 8.0.0860
问题: There may be side effects when a channel appends to a buffer that
is not the current buffer.
解决方案: Properly switch to another buffer before appending. (Yasuhiro
Matsumoto, closes #1926, closes #1937)
相关文件: src/channel.c, src/buffer.c, src/proto/buffer.pro,
src/if_py_both.h

Patch 8.0.0861
问题: Still many old style tests.
解决方案: Convert several tests to new style. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/main.aap, src/testdir/test104.in,
src/testdir/test104.ok, src/testdir/test22.in,
src/testdir/test22.ok, src/testdir/test77.in,
src/testdir/test77.ok, src/testdir/test84.in,
src/testdir/test84.ok, src/testdir/test9.in, src/testdir/test9.ok,
src/testdir/test98.in, src/testdir/test98.ok,
src/testdir/test_autocmd.vim, src/testdir/test_curswant.vim,
src/testdir/test_file_size.vim, src/testdir/test_let.vim,
src/testdir/test_lineending.vim, src/testdir/test_scrollbind.vim,
src/Makefile

Patch 8.0.0862 (after 8.0.0862)
问题: File size test fails on MS-Windows.
解决方案: Set fileformat after opening new buffer. Strip CR.
相关文件: src/testdir/test_file_size.vim

Patch 8.0.0863
问题: A remote command starting with CTRL-\ CTRL-N does not work in the
terminal window. (Christian J. Robinson)
解决方案: Use CTRL-\ CTRL-N as a prefix or a Normal mode command.
相关文件: src/terminal.c, runtime/doc/terminal.txt

Patch 8.0.0864
问题: Cannot specify the name of a terminal.
解决方案: Add the "term_name" option. (Yasuhiro Matsumoto, closes #1936)
相关文件: src/channel.c, src/structs.h, src/terminal.c, runtime/doc/eval.txt

Patch 8.0.0865

问题:　　Cannot build with channel but without terminal feature.
解决方案:　Add #ifdef
相关文件:　src/channel.c

Patch 8.0.0866
问题:　　Solaris also doesn't have MIN and MAX.
解决方案:　Define MIN and MAX whenever they are not defined. (Ozaki Kiichi,
closes #1939)
相关文件:　src/terminal.c

Patch 8.0.0867
问题:　　When using a job or channel value as a dict value, when turning it
into a string the quotes are missing.
解决方案:　Add quotes to the job and channel values. (Yasuhiro Matsumoto,
closes #1930)
相关文件:　src/list.c, src/eval.c, src/testdir/test_terminal.vim

Patch 8.0.0868
问题:　　Cannot specify the terminal size on the command line.
解决方案:　Use the address range for the terminal size. (Yasuhiro Matsumoto,
closes #1941)
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0869
问题:　　Job output is sometimes not displayed in a terminal.
解决方案:　Flush output before closing the channel.
相关文件:　src/channel.c, src/terminal.c

Patch 8.0.0870
问题:　　Mouse escape codes sent to terminal unintentionally.
解决方案:　Fix libvterm to send mouse codes only when enabled.
相关文件:　src/terminal.c, src/libvterm/src/mouse.c

Patch 8.0.0871
问题:　　The status line for a terminal window always has "[+]".
解决方案:　Do make the status line include "[+]" for a terminal window.
相关文件:　src/screen.c

Patch 8.0.0872
问题:　　Using mouse scroll while a terminal window has focus and the mouse
pointer is on another window does not work. Same for focus in a
non-terminal window and the mouse pointer is over a terminal
window.
解决方案:　Send the scroll action to the right window.
相关文件:　src/terminal.c, src/normal.c, src/proto/terminal.pro

Patch 8.0.0873
问题:　　In a terminal window cannot use CTRL-\ CTRL-N to start Visual
mode.
解决方案:　After CTRL-\ CTRL-N enter Terminal-Normal mode for one command.
相关文件:　src/main.c, src/terminal.c, src/proto/terminal.pro

Patch 8.0.0874 (after 8.0.0873)
问题:　　Can't build with terminal feature.

解决方案: Include change to term_use_loop(). (Dominique Pelle)
相关文件: src/normal.c

Patch 8.0.0875
问题: Crash with weird command sequence. (Dominique Pelle)
解决方案: Use vim_snprintf() instead of STRCPY().
相关文件: src/misc1.c

Patch 8.0.0876
问题: MS-Windows: Backslashes and wildcards in backticks don't work.
解决方案: Do not handle backslashes inside backticks in the wrong place.
           (Yasuhiro Matsumoto, closes #1942)
相关文件: src/os_mswin.c, src/os_win32.c

Patch 8.0.0877
问题: Using CTRL-\ CTRL-N in terminal is inconsistent.
解决方案: Stay in Normal mode.
相关文件: src/terminal.c, src/proto/terminal.pro, src/main.c, src/normal.c,
           src/option.c

Patch 8.0.0878
问题: No completion for :mapclear.
解决方案: Add completion (Nobuhiro Takasaki et al. closes #1943)
相关文件: runtime/doc/eval.txt, runtime/doc/map.txt, src/ex_docmd.c,
           src/ex_getln.c, src/proto/ex_docmd.pro,
           src/testdir/test_cmdline.vim, src/vim.h

Patch 8.0.0879
问题: Crash when shifting with huge number.
解决方案: Check for overflow. (Dominique Pelle, closes #1945)
相关文件: src/ops.c, src/testdir/test_visual.vim

Patch 8.0.0880
问题: Travis uses an old Ubuntu version.
解决方案: Switch from precise to trusty. (Ken Takata, closes #1897)
相关文件: .travis.yml, Filelist, src/testdir/if_ver-1.vim,
           src/testdir/if_ver-2.vim, src/testdir/lsan-suppress.txt

Patch 8.0.0881
问题: win32.mak no longer included in Windows SDK.
解决方案: Do not include win32.mak. (Ken Takata)
相关文件: src/GvimExt/Makefile, src/Make_mvc.mak

Patch 8.0.0882
问题: term_scrape() and term_getline() require two arguments but it is
           not enforced.
解决方案: Correct minimal number of arguments. (Hirohito Higashi) Update
           documentation. (Ken Takata)
相关文件: src/evalfunc.c, runtime/doc/eval.txt

Patch 8.0.0883
问题: Invalid memory access with nonsensical script.
解决方案: Check "dstlen" being positive. (Dominique Pelle)
相关文件: src/misc1.c

Patch 8.0.0884
问题: Can't specify the wait time for term_wait().
解决方案: Add an optional second argument.
相关文件: src/evalfunc.c, src/terminal.c, runtime/doc/eval.txt

Patch 8.0.0885
问题: Terminal window scrollback is stored inefficiently.
解决方案: Store the text in the Vim buffer.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0886
问题: Crash when using ":term ls".
解决方案: Fix line number computation. Add a test for this.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0887
问题: Can create a logfile in the sandbox.
解决方案: Disable ch_logfile() in the sandbox. (Yasuhiro Matsumoto)
相关文件: src/evalfunc.c

Patch 8.0.0888
问题: Compiler warnings with 64 bit build.
解决方案: Add type cast of change the type. (Mike Williams)
相关文件: src/message.c, src/os_mswin.c, src/os_win32.c

Patch 8.0.0889
问题: Gcc gives warnings for uninitialized variables. (Tony Mechelynck)
解决方案: Initialize variables even though they are not used.
相关文件: src/terminal.c

Patch 8.0.0890
问题: Still many old style tests.
解决方案: Convert several tests to new style. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
            src/testdir/test103.in, src/testdir/test103.ok,
            src/testdir/test107.in, src/testdir/test107.ok,
            src/testdir/test51.in, src/testdir/test51.ok,
            src/testdir/test91.in, src/testdir/test91.ok,
            src/testdir/test_getvar.vim, src/testdir/test_highlight.vim,
            src/testdir/test_visual.vim, src/testdir/test_window_cmd.vim,
            src/Makefile

Patch 8.0.0891
问题: Uninitialized memory use with empty line in terminal.
解决方案: Initialize growarray earlier. (Yasuhiro Matsumoto, closes #1949)
相关文件: src/terminal.c

Patch 8.0.0892
问题: When opening a terminal the pty size doesn't always match.
解决方案: Update the pty size after opening the terminal. (Ken Takata)
相关文件: src/terminal.c

Patch 8.0.0893

问题: Cannot get the scroll count of a terminal window.
解决方案: Add term_getscrolled().
相关文件: src/terminal.c, src/proto/terminal.pro, src/evalfunc.c,
            runtime/doc/eval.txt, src/testdir/test_terminal.vim

Patch 8.0.0894
问题: There is no test for runtime filetype detection.
解决方案: Test a list of filetypes from patterns.
相关文件: src/testdir/test_filetype.vim, runtime/filetype.vim

Patch 8.0.0895 (after 8.0.0894)
问题: Filetype test fails on MS-Windows.
解决方案: Fix file names.
相关文件: src/testdir/test_filetype.vim

Patch 8.0.0896
问题: Cannot automatically close a terminal window when the job ends.
解决方案: Add the ++close argument to :term. Add the term_finish option to
            term_start(). (Yasuhiro Matsumoto, closes #1950) Also add
            ++open.
相关文件: runtime/doc/eval.txt, runtime/doc/terminal.txt, src/channel.c,
            src/structs.h, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0897 (after 8.0.0896)
问题: Wrong error message for invalid term_finish value
解决方案: Pass the right argument to emsg().
相关文件: src/channel.c

Patch 8.0.0898
问题: Can't use the alternate screen in a terminal window.
解决方案: Initialize the alternate screen. (Yasuhiro Matsumoto, closes
            #1957) Add term_getaltscreen().
相关文件: src/libvterm/include/vterm.h, src/terminal.c,
            src/proto/terminal.pro, src/evalfunc.c, runtime/doc/eval.txt

Patch 8.0.0899
问题: Function name mch_stop_job() is confusing.
解决方案: Rename to mch_signal_job().
相关文件: src/channel.c, src/os_unix.c, src/proto/os_unix.pro,
            src/os_win32.c, src/proto/os_win32.pro, src/terminal.c

Patch 8.0.0900
问题: :tab options doesn't open a new tab page. (Aviany)
解决方案: Support the :tab modifier. (closes #1960)
相关文件: src/ex_cmds2.c, runtime/optwin.vim

Patch 8.0.0901
问题: Asan suppress file missing from distribution.
解决方案: Add the file.
相关文件: Filelist

Patch 8.0.0902
问题: Cannot specify directory or environment for a job.
解决方案: Add the "cwd" and "env" arguments to job options. (Yasuhiro

Matsumoto, closes #1160)
相关文件: runtime/doc/channel.txt, src/channel.c, src/terminal.c,
src/os_unix.c, src/os_win32.c, src/structs.h,
src/testdir/test_channel.vim, src/testdir/test_terminal.vim

Patch 8.0.0903 (after 8.0.0902)
问题: Early return from test function.
解决方案: Remove the return.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0904
问题: Cannot set a location list from text.
解决方案: Add the "text" argument to setqflist(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.0.0905
问题: MS-Windows: broken multi-byte characters in the console.
解决方案: Restore all regions of the console buffer. (Ken Takata)
相关文件: src/os_win32.c

Patch 8.0.0906
问题: Don't recognize Couchbase files.
解决方案: Add filetype detection. (Eugene Ciurana, closes #1951)
相关文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.0.0907
问题: With cp932 font names might be misinterpreted.
解决方案: Do not see "_" as a space when it is the second byte of a double
byte character. (Ken Takata)
相关文件: src/os_win32.c

Patch 8.0.0908
问题: Cannot set terminal size with options.
解决方案: Add "term_rows", "term_cols" and "vertical".
相关文件: src/terminal.c, runtime/doc/eval.txt, src/channel.c,
src/proto/channel.pro, src/structs.h, src/evalfunc.c,
src/testdir/test_terminal.vim

Patch 8.0.0909
问题: Channel test fails.
解决方案: Allow for "cwd" and "env" arguments.
相关文件: src/channel.c

Patch 8.0.0910
问题: Cannot create a terminal in the current window.
解决方案: Add option "curwin" and ++curwin.
相关文件: src/terminal.c, runtime/doc/eval.txt, src/channel.c,
src/structs.h, src/ex_cmds.h, src/testdir/test_terminal.vim

Patch 8.0.0911
问题: Terminal test takes too long.
解决方案: Instead of "sleep 1" use a Python program to briefly sleep.
相关文件: src/testdir/test_terminal.vim, src/testdir/test_short_sleep.py

Patch 8.0.0912
问题:　　Cannot run a job in a hidden terminal.
解决方案:　Add option "hidden" and ++hidden.
相关文件:　src/terminal.c, src/structs.h, src/channel.c, src/fileio.c,
　　runtime/doc/terminal.txt, src/testdir/test_terminal.vim

Patch 8.0.0913
问题:　　MS-Windows: CTRL-C kills shell in terminal window instead of the
　　command running in the shell.
解决方案:　Make CTRL-C only send a CTRL_C_EVENT and have CTRL-BREAK kill the
　　job. (partly by Yasuhiro Matsumoto, closes #1962)
相关文件:　src/os_win32.c, src/gui_w32.c, src/terminal.c, src/globals.h

Patch 8.0.0914
问题:　　Highlight attributes are always combined.
解决方案:　Add the 'nocombine' value to replace attributes instead of
　　combining them. (scauligi, closes #1963)
相关文件:　runtime/doc/syntax.txt, src/syntax.c, src/vim.h

Patch 8.0.0915
问题:　　Wrong initialisation of global.
解决方案:　Use INIT().
相关文件:　src/globals.h

Patch 8.0.0916
问题:　　Cannot specify properties of window for when opening a window for
　　a finished terminal job.
解决方案:　Add "term_opencmd".
相关文件:　src/channel.c, src/structs.h, src/terminal.c,
　　runtime/doc/eval.txt, src/testdir/test_terminal.vim

Patch 8.0.0917
问题:　　MS-Windows:CTRL-C handling in terminal window is wrong
解决方案:　Pass CTRL-C as a key. Turn CTRL-BREAK into a key stroke. (Yasuhiro
　　Matsumoto, closes #1965)
相关文件:　src/os_win32.c, src/terminal.c

Patch 8.0.0918
问题:　　Cannot get terminal window cursor shape or attributes.
解决方案:　Support cursor shape, attributes and color.
相关文件:　src/terminal.c, runtime/doc/eval.txt,
　　src/libvterm/include/vterm.h, src/libvterm/src/state.c,
　　src/libvterm/src/vterm.c, src/feature.h, src/ui.c,
　　src/proto/ui.pro, src/term.c, src/proto/term.pro,
　　src/option.c, src/term.h

Patch 8.0.0919
问题:　　Cursor color isn't set on startup.
解决方案:　Initialize showing_mode to invalid value.
相关文件:　src/term.c

Patch 8.0.0920
问题:　　The cursor shape is wrong after switch back from an alternate

screen in a terminal window. (Marius Gedminas)
解决方案: Change bitfield to unsigned. Set flag that cursor shape was set.
相关文件: src/terminal.c, src/libvterm/src/vterm_internal.h

Patch 8.0.0921
问题: Terminal window cursor shape not supported in the GUI.
解决方案: Use the terminal window cursor shape in the GUI.
相关文件: src/terminal.c, src/proto/terminal.pro, src/gui.c, src/syntax.c,
src/proto/syntax.pro

Patch 8.0.0922
问题: Quickfix list always added after current one.
解决方案: Make it possible to add a quickfix list after the last one.
(Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.0.0923
问题: Crash in GUI when terminal job exits. (Kazunobu Kuriyama)
解决方案: reset in_terminal_loop when a terminal is freed.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.0924
问题: Terminal window not updated after using term_sendkeys().
解决方案: Call redraw_after_callback().
相关文件: src/terminal.c

Patch 8.0.0925
问题: MS-Windows GUI: channel I/O not handled right away.
解决方案: Don't call process_message() unless a message is available.
(Yasuhiro Matsumoto, closes #1969)
相关文件: src/gui_w32.c

Patch 8.0.0926
问题: When job in terminal window ends topline may be wrong.
解决方案: When the job ends adjust topline so that the active part of the
terminal is displayed.
相关文件: src/terminal.c

Patch 8.0.0927
问题: If a terminal job sends a blank title "running" is not shown.
解决方案: When the title is blank make it empty.
相关文件: src/terminal.c

Patch 8.0.0928
问题: MS-Windows: passing arglist to job has escaping problems.
解决方案: Improve escaping. (Yasuhiro Matsumoto, closes #1954)
相关文件: src/testdir/test_channel.vim, src/testdir/test_terminal.vim,
src/channel.c, src/proto/channel.pro, src/terminal.c

Patch 8.0.0929
问题: :term without argument does not work.
解决方案: Use shell for empty command. (Yasuhiro Matsumoto, closes #1970)
相关文件: src/terminal.c

Patch 8.0.0930
问题: Terminal buffers are stored in the viminfo file while they can't be useful.
解决方案: Skip terminal buffers for file marks and buffer list
相关文件: src/buffer.c, src/mark.c

Patch 8.0.0931
问题: getwininfo() does not indicate a terminal window.
解决方案: Add "terminal" to the dictionary.
相关文件: runtime/doc/eval.txt, src/evalfunc.c

Patch 8.0.0932
问题: Terminal may not use right characters for BS and Enter.
解决方案: Get the characters from the tty.
相关文件: src/os_unix.c, src/proto/os_unix.pro, src/terminal.c

Patch 8.0.0933
问题: Terminal test tries to start GUI when it's not possible.
解决方案: Check if the GUI can run. (James McCoy, closes #1971)
相关文件: src/testdir/shared.vim, src/testdir/test_terminal.vim, src/testdir/test_gui.vim, src/testdir/test_gui_init.vim

Patch 8.0.0934 (after 8.0.0932)
问题: Change to struts.h missing in patch.
解决方案: Include adding ttyinfo_T.
相关文件: src/structs.h

Patch 8.0.0935
问题: Cannot recognize a terminal buffer in :ls output.
解决方案: Use R for a running job and F for a finished job.
相关文件: src/buffer.c

Patch 8.0.0936
问题: mode() returns wrong value for a terminal window.
解决方案: Return 't' when typed keys go to a job.
相关文件: src/evalfunc.c, src/testdir/test_terminal.vim

Patch 8.0.0937
问题: User highlight groups are not adjusted for StatusLineTerm.
解决方案: Combine attributes like for StatusLineNC.
相关文件: src/syntax.c, src/globals.h, src/screen.c

Patch 8.0.0938
问题: Scrolling in terminal window is inefficient.
解决方案: Use win_del_lines().
相关文件: src/terminal.c

Patch 8.0.0939
问题: Test_terminal_env is flaky. (James McCoy)
解决方案: Use WaitFor() instead of term_wait().
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0940

问题:　　Test_terminal_scrape_multibyte is flaky. (James McCoy)
解决方案:　Use WaitFor() instead of term_wait().
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.0941
问题:　　Existing color schemes don't work well with StatusLineTerm.
解决方案:　Don't use "reverse", use fg and bg colors. Also add
　　StatusLineTermNC.
相关文件:　src/syntax.c, src/vim.h, src/screen.c, src/globals.h, src/option.c

Patch 8.0.0942
问题:　　Using freed memory with ":terminal" if an autocommand changes
　　'shell' when splitting the window. (Marius Gedminas)
解决方案:　Make a copy of 'shell'. (closes #1974)
相关文件:　src/terminal.c

Patch 8.0.0943
问题:　　Test_terminal_scrape_multibyte fails if the codepage is not utf-8.
解决方案:　Start "cmd" with the utf-8 codepage. (micbou, closes #1975)
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.0944
问题:　　Test_profile is a little bit flaky.
解决方案:　Accept a match when self and total time are the same. (James
　　McCoy, closes #1972)
相关文件:　src/testdir/test_profile.vim

Patch 8.0.0945
问题:　　64-bit compiler warnings.
解决方案:　Use "size_t" instead of "int". (Mike Williams)
相关文件:　src/os_win32.c

Patch 8.0.0946
问题:　　Using PATH_MAX does not work well on some systems.
解决方案:　use MAXPATHL instead. (James McCoy, closes #1973)
相关文件:　src/main.c

Patch 8.0.0947
问题:　　When in Insert mode and using CTRL-O CTRL-W CTRL-W to move to a
　　terminal window, get in a weird Insert mode.
解决方案:　Don't go to Insert mode in a terminal window. (closes #1977)
相关文件:　src/normal.c

Patch 8.0.0948
问题:　　Crash if timer closes window while dragging status line.
解决方案:　Check if the window still exists. (Yasuhiro Matsumoto, closes
　　#1979)
相关文件:　src/edit.c, src/evalfunc.c, src/gui.c, src/normal.c, src/ui.c

Patch 8.0.0949
问题:　　winpty.dll name is fixed.
解决方案:　Add the 'winptydll' option. Make the default name depend on
　　whether it is a 32-bit or 64-bit build. (idea by Yasuhiro
　　Matsumoto, closes #1978)

相关文件: src/option.c, src/option.h, src/terminal.c,
           runtime/doc/options.txt

Patch 8.0.0950
问题: MS-Windows: wrong #ifdef, compiler warnings for signed/unsigned.
解决方案: Change variable type. Change TERMINAL to FEAT_TERMINAL.
相关文件: src/os_win32.c, src/option.h

Patch 8.0.0951
问题: Another wrong #ifdef.
解决方案: Change TERMINAL to FEAT_TERMINAL. (closes #1981)
相关文件: src/option.c

Patch 8.0.0952
问题: MS-Windows: has('terminal') does not check existence of dll file.
解决方案: Check if the winpty dll file can be loaded. (Ken Takata)
相关文件: src/evalfunc.c, src/proto/terminal.pro, src/terminal.c

Patch 8.0.0953
问题: Get "no write since last change" error in terminal window.
解决方案: Use another message when closing a terminal window. Make ":quit!"
           also end the job.
相关文件: src/globals.h, src/buffer.c, src/proto/buffer.pro, src/ex_cmds.c,
           src/ex_cmds2.c, src/ex_docmd.c, src/quickfix.c, src/terminal.c

Patch 8.0.0954
问题: /proc/self/exe might be a relative path.
解决方案: Make the path a full path. (James McCoy, closes #1983)
相关文件: src/main.c

Patch 8.0.0955
问题: Test_existent_file() fails on some file systems.
解决方案: Run the test again with a sleep when the test fails without a
           sleep. (James McCoy, closes #1984)
相关文件: src/testdir/test_stat.vim

Patch 8.0.0956
问题: Scrolling in a terminal hwindow as flicker when the Normal
           background differs from the terminal window background.
解决方案: Set the attribute to clear with.
相关文件: src/terminal.c, src/screen.c, src/proto/screen.pro, src/message.c,
           src/move.c

Patch 8.0.0957
问题: When term_sendkeys() sends many keys it may get stuck in writing
           to the job.
解决方案: Make the write non-blocking, buffer keys to be sent.
相关文件: src/terminal.c, src/channel.c, src/proto/channel.pro,
           src/structs.h src/testdir/test_terminal.vim

Patch 8.0.0958
问题: The terminal test fails on MS-Windows when compiled with the
           terminal feature but the winpty DLL is missing.
解决方案: Check if the terminal feature works. (Ken Takata)

相关文件: src/testdir/test_terminal.vim

Patch 8.0.0959
问题: Build failure on MS-Windows.
解决方案: Use ioctlsocket() instead of fcntl().
相关文件: src/channel.c

Patch 8.0.0960
问题: Job in terminal does not get CTRL-C, we send a SIGINT instead.
解决方案: Don't call may_send_sigint() on CTRL-C. Make CTRL-W CTRL-C end
the job.
相关文件: src/terminal.c, runtime/doc/terminal.txt

Patch 8.0.0961
问题: The script to build the installer does not include winpty.
解决方案: Add winpty32.dll and winpty-agent.exe like diff.exe
相关文件: nsis/gvim.nsi

Patch 8.0.0962
问题: Crash with virtualedit and joining lines. (Joshua T Corbin, Neovim
#6726)
解决方案: When using a mark check that coladd is valid.
相关文件: src/normal.c, src/misc2.c, src/Makefile,
src/testdir/test_virtualedit.vim, src/testdir/test_alot.vim

Patch 8.0.0963
问题: Terminal test fails on MacOS. (chdiza)
解决方案: Wait for the shell to echo the characters. (closes #1991)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0964
问题: Channel write buffer does not work with poll().
解决方案: Use the same mechanism as with select().
相关文件: src/channel.c

Patch 8.0.0965
问题: The cursor shape is not reset after it was changed in a terminal.
解决方案: Request the original cursor shape and restore it. Add t_RS.
Do not add t_SH for now, it does not work properly.
相关文件: src/term.c, src/term.h, src/option.c, src/terminal.c

Patch 8.0.0966 (after 8.0.0965)
问题: Build failure without terminal feature.
解决方案: Move #endif.
相关文件: src/term.c

Patch 8.0.0967
问题: Using a terminal may cause the cursor to blink.
解决方案: Do not set t_vs, since we cannot restore the old blink state.
相关文件: src/term.c

Patch 8.0.0968
问题: Crash when switching terminal modes. (Nikolai Pavlov)
解决方案: Check that there are scrollback lines.

相关文件: src/terminal.c

Patch 8.0.0969
问题: Coverity warning for unused return value.
解决方案: Add (void) to avoid the warning.
相关文件: src/channel.c

Patch 8.0.0970
问题: if there is no StatusLine highlighting and there is StatusLineNC
or StatusLineTermNC highlighting then an invalid highlight id is
passed to combine_stl_hlt(). (Coverity)
解决方案: Check id_S to be -1 instead of zero.
相关文件: src/syntax.c

Patch 8.0.0971
问题: 'winptydll' missing from :options.
解决方案: Add the entry.
相关文件: runtime/optwin.vim

Patch 8.0.0972
问题: Compiler warnings for unused variables. (Tony Mechelynck)
解决方案: Add #ifdefs.
相关文件: src/term.c

Patch 8.0.0973
问题: initial info about blinking cursor is wrong
解决方案: Invert the blink flag. Add t_VS to stop a blinking cursor.
相关文件: src/term.c, src/proto/term.pro, src/term.h, src/option.c,
src/terminal.c

Patch 8.0.0974
问题: Resetting a string option does not trigger OptionSet. (Rick Howe)
解决方案: Set the origval.
相关文件: src/option.c, src/testdir/test_autocmd.vim

Patch 8.0.0975
问题: Using freed memory when setting 'backspace'.
解决方案: When changing oldval also change origval.
相关文件: src/option.c

Patch 8.0.0976
问题: Cannot send lines to a terminal job.
解决方案: Make [range]terminal send selected lines to the job.
Use ++rows and ++cols for the terminal size.
相关文件: src/ex_cmds.h, src/terminal.c, src/os_unix.c,
src/testdir/test_terminal.vim

Patch 8.0.0977
问题: Cannot send lines to a terminal job on MS-Windows.
解决方案: Set jv_in_buf. Command doesn't get EOF yet though.
相关文件: src/terminal.c

Patch 8.0.0978
问题: Writing to terminal job is not tested.

解决方案: Add a test.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0979
问题: Terminal noblock test fails on MS-Windows. (Christian Brabandt)
解决方案: Ignore empty line below "done".
相关文件: src/testdir/test_terminal.vim

Patch 8.0.0980
问题: Coverity warning for failing to open /dev/null.
解决方案: When /dev/null can't be opened exit the child.
相关文件: src/os_unix.c

Patch 8.0.0981
问题: Cursor in terminal window blinks by default, while in a real xterm
it does not blink, unless the -bc argument is used.
解决方案: Do not use a blinking cursor by default.
相关文件: src/terminal.c

Patch 8.0.0982
问题: When 'encoding' is set to a multi-byte encoding other than utf-8
the characters from their terminal are messed up.
解决方案: Convert displayed text from utf-8 to 'encoding' for MS-Windows.
(Yasuhiro Matsumoto, close #2000)
相关文件: src/terminal.c

Patch 8.0.0983
问题: Unnecessary check for NULL pointer.
解决方案: Remove the NULL check in dialog_changed(), it already happens in
dialog_msg(). (Ken Takata)
相关文件: src/ex_cmds2.c

Patch 8.0.0984
问题: Terminal blinking cursor not correct in the GUI.
解决方案: Set blinkoff correctly. Also make the cursor blink on MS-Windows
by default. (Ken Takata)
相关文件: src/terminal.c

Patch 8.0.0985
问题: Libvterm has its own idea of character width.
解决方案: Use the Vim functions for character width and composing to avoid a
mismatch. (idea by Yasuhiro Matsumoto)
相关文件: src/Makefile, src/libvterm/src/unicode.c, src/mbyte.c,
src/proto/mbyte.pro, src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.0986
问题: Terminal feature always requires multi-byte feature.
解决方案: Remove #ifdef FEAT_MBYTE, disable terminal without multi-byte.
相关文件: src/terminal.c, src/feature.h

Patch 8.0.0987
问题: terminal: second byte of double-byte char wrong
解决方案: Set the second byte to NUL only for utf-8 and non-multibyte.
相关文件: src/terminal.c

Patch 8.0.0988
问题:　　Warning from Covscan about using NULL pointer.
解决方案:　Add extra check for NULL. (zdohnal)
相关文件:　src/fileio.c, src/undo.c

Patch 8.0.0989
问题:　　ActiveTcl dll name has changed in 8.6.6.
解决方案:　Adjust the makefile. (Ken Takata)
相关文件:　src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.0990
问题:　　When 'encoding' is a double-byte encoding, pasting a register into
a terminal ends up with the wrong characters.
解决方案:　Convert from 'encoding' to utf-8. (Yasuhiro Matsumoto, closes
#2007)
相关文件:　src/terminal.c

Patch 8.0.0991
问题:　　Using wrong character conversion for DBCS.
解决方案:　Use utf_char2bytes instead of mb_char2bytes. (Yasuhiro Matsumoto,
closes #2012)
相关文件:　src/terminal.c

Patch 8.0.0992
问题:　　Terminal title is wrong when 'encoding' is DBCS.
解决方案:　Convert the title from DBCS to utf-8. (Yasuhiro Matsumoto, closes
#2009)
相关文件:　src/terminal.c

Patch 8.0.0993
问题:　　Sometimes an xterm sends an extra CTRL-X after the response for
the background color. Related to t_RS.
解决方案:　Check for the CTRL-X after the terminating 0x7.
相关文件:　src/term.c

Patch 8.0.0994
问题:　　MS-Windows: cursor in terminal blinks even though the blinking
cursor was disabled on the system.
解决方案:　Use GetCaretBlinkTime(). (Ken Takata)
相关文件:　src/terminal.c

Patch 8.0.0995
问题:　　Terminal tests fail on Mac.
解决方案:　Add workaround: sleep a moment in between sending keys.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.0996
问题:　　Mac: t_RS is echoed on the screen in Terminal.app. Even though
$TERM is set to "xterm-256colors" it cannot handle this xterm
escape sequence.
解决方案:　Recognize Terminal.app from the termresponse and skip sending t_RS
if it looks like Terminal.app.
相关文件:　src/term.c

Patch 8.0.0997 (after 8.0.0996)
问题:　　Libvterm and Terminal.app not recognized from termresponse.
解决方案:　Adjust string compare.
相关文件:　src/term.c

Patch 8.0.0998
问题:　　Strange error when using K while only spaces are selected.
(Christian J. Robinson)
解决方案:　Check for blank argument.
相关文件:　src/normal.c, src/testdir/test_help.vim

Patch 8.0.0999
问题:　　Indenting raw C++ strings is wrong.
解决方案:　Add special handling of raw strings. (Christian Brabandt)
相关文件:　src/misc1.c, src/testdir/test_cindent.vim

Patch 8.0.1000
问题:　　Cannot open a terminal without running a job in it.
解决方案:　Make ":terminal NONE" open a terminal with a pty.
相关文件:　src/terminal.c, src/os_unix.c, src/proto/os_unix.pro,
src/channel.c, src/proto/channel.pro, src/structs.h,
src/testdir/test_terminal.c, src/misc2.c, src/gui_gtk_x11.c

Patch 8.0.1001
问题:　　Setting 'encoding' makes 'printheader' invalid.
解决方案:　Do not translate the default value of 'printheader'. (Yasuhiro
Matsumoto, closes #2026)
相关文件:　src/option.c

Patch 8.0.1002
问题:　　Unnecessarily updating screen after timer callback.
解决方案:　Check if calling the timer sets must_redraw.
相关文件:　src/ex_cmds2.c, src/channel.c, src/screen.c, src/proto/screen.pro,
src/terminal.c

Patch 8.0.1003
问题:　　64 bit compiler warning
解决方案:　Add type cast. (Mike Williams)
相关文件:　src/channel.c

Patch 8.0.1004
问题:　　matchstrpos() without a match returns too many items.
解决方案:　Also remove the second item when the position is beyond the end of
the string. (Hirohito Higashi) Use an enum for the type.
相关文件:　src/evalfunc.c, src/testdir/test_match.vim

Patch 8.0.1005
问题:　　Terminal without job updates slowly in GUI.
解决方案:　Poll for input when a channel has the keep_open flag.
相关文件:　src/channel.c, src/proto/channel.pro, src/gui_gtk_x11.c

Patch 8.0.1006
问题:　　Cannot parse text with 'errorformat' without changing a quickfix

list.
解决方案: Add the "text" argument to getqflist(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/proto/quickfix.pro, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1007
问题: No test for filetype detection for scripts.
解决方案: Add a first test file script filetype detection.
相关文件: src/testdir/test_filetype.vim, runtime/scripts.vim

Patch 8.0.1008
问题: Slow updating of terminal window in Motif.
解决方案: Add a timeout to the wait-for-character loop.
相关文件: src/gui_x11.c

Patch 8.0.1009
问题: Xterm cursor blinking status may be inverted.
解决方案: Use another request to get the blink status and compare with the cursor style report
相关文件: src/term.c, src/proto/term.pro, src/term.h, src/option.c, src/terminal.c

Patch 8.0.1010 (after 8.0.1009)
问题: Build failure without termresponse feature.
解决方案: Add #ifdef.
相关文件: src/term.c

Patch 8.0.1011
问题: Terminal test fails with Athena and Motif.
解决方案: Ignore the error for the input context. (Kazunobu Kuriyama)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1012
问题: MS-Windows: Problem with $HOME when it was set internally.
解决方案: Only use the $HOME default internally. (Yasuhiro Matsumoto, closes #2013)
相关文件: src/misc1.c, src/testdir/Make_all.mak, src/Makefile, src/testdir/test_windows_home.vim

Patch 8.0.1013
问题: A terminal window with a running job behaves different from a window containing a changed buffer.
解决方案: Do not set 'bufhidden' to "hide". Fix that a buffer where a terminal used to run is listed as "[Scratch]".
相关文件: src/terminal.c, runtime/doc/terminal.txt, src/buffer.c

Patch 8.0.1014
问题: Old compiler doesn't know uint32_t. Warning for using NULL instead of NUL.
解决方案: Use UINT32_T. Use NUL instead of NULL.
相关文件: src/mbyte.c, src/proto/mbyte.pro, src/misc1.c

Patch 8.0.1015 (after 8.0.1013)
问题: Missing update to terminal test.

解决方案: Add the changes to the test.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1016
问题: Gnome terminal echoes t_RC.
解决方案: Detect Gnome terminal by the version string. Add v: variables for
           all the term responses.
相关文件: src/term.c, src/eval.c, src/vim.h, runtime/doc/eval.txt

Patch 8.0.1017
问题: Test for MS-Windows $HOME always passes.
解决方案: Rename the test function. Make the test pass.
相关文件: src/testdir/test_windows_home.vim

Patch 8.0.1018
问题: Warnings from 64-bit compiler. (Christian Brabandt)
解决方案: Add type casts.
相关文件: src/terminal.c

Patch 8.0.1019
问题: Pasting in virtual edit happens in the wrong place.
解决方案: Do not adjust coladd when after the end of the line (closes #2015)
相关文件: src/testdir/test_virtualedit.vim, src/misc2.c

Patch 8.0.1020
问题: When a timer calls getchar(1) input is overwritten.
解决方案: Increment tb_change_cnt in inchar(). (closes #1940)
相关文件: src/getchar.c

Patch 8.0.1021
问题: Older Gnome terminal still echoes t_RC. (Franois Ingelrest)
解决方案: Check for version > 3000 instead of 4000.
相关文件: src/term.c

Patch 8.0.1022
问题: Test 80 is old style.
解决方案: Turn it into a new style test. (Yegappan Lakshmanan)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/test80.in, src/testdir/test80.ok,
           src/testdir/test_substitute.vim

Patch 8.0.1023
问题: It is not easy to identify a quickfix list.
解决方案: Add the "id" field. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/quickfix.c,
           src/testdir/test_quickfix.vim

Patch 8.0.1024
问题: Manual folds are lost when a session file has the same buffer in
           two windows. (Jeansen)
解决方案: Use ":edit" only once. (Christian Brabandt, closes #1958)
相关文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.0.1025

问题: Stray copy command in test.
解决方案: Remove the copy command.
相关文件: src/testdir/test_mksession.vim

Patch 8.0.1026
问题: GTK on-the-spot input has problems. (Gerd Wachsmuth)
解决方案: Support over-the-spot. (Yukihiro Nakadaira, Ken Takata, closes
            #1215)
相关文件: runtime/doc/mbyte.txt, runtime/doc/options.txt, src/edit.c,
            src/ex_getln.c, src/mbyte.c, src/misc1.c, src/option.c,
            src/option.h, src/screen.c, src/undo.c,
            src/testdir/gen_opt_test.vim

Patch 8.0.1027
问题: More terminals can't handle requesting cursor mode.
解决方案: Recognize Putty. (Hirohito Higashi) Also include Xfce in the
            version check. (Dominique Pelle) Recognize Konsole.
相关文件: src/term.c

Patch 8.0.1028
问题: MS-Windows: viminfo uses $VIM/_viminfo if $HOME not set. (Yongwei
            Wu)
解决方案: Use vim_getenv() but check it's returning the default "C:/".
相关文件: src/ex_cmds.c

Patch 8.0.1029
问题: Return value of getqflist() is inconsistent. (Lcd47)
解决方案: Always return an "items" entry.
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1030
问题: MS-Windows: wrong size computation in is_cygpty().
解决方案: Compute the size properly. (Ken Takata)
相关文件: src/iscygpty.c, src/iscygpty.h

Patch 8.0.1031
问题: "text" argument for getqflist() is confusing. (Lcd47)
解决方案: Use "lines" instead. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c,
            src/testdir/test_quickfix.vim

Patch 8.0.1032
问题: "make tags" doesn't work well on MS-Windows.
解决方案: Add or fix tags target. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.1033
问题: Detecting background color does not work in screen, even when it
            is working like an xterm.
解决方案: Make "screen.xterm" use termcap entries like an xterm. (Lubomir
            Rintel, closes #2048) When termresponse version is huge also
            recognize as not being an xterm.
相关文件: src/os_unix.c, src/term.c

Patch 8.0.1034
问题: Sending buffer lines to terminal doesn't work on MS-Windows.
解决方案: Send CTRL-D to mark the end of the text. (Yasuhiro Matsumoto, closes #2043) Add the "eof_chars" option.
相关文件: src/channel.c, src/proto/terminal.pro, src/terminal.c, src/testdir/test_terminal.vim, src/structs.h

Patch 8.0.1035
问题: Sending buffer lines to terminal doesn't work on MS-Windows.
解决方案: Use CR instead of NL after every line. Make the EOF text work properly. Add the ++eof argument to :terminal.
相关文件: src/structs.h, src/channel.c, src/terminal.c, runtime/doc/terminal.txt, runtime/doc/eval.txt

Patch 8.0.1036
问题: ++eof argument for terminal only available on MS-Windows.
解决方案: Also support ++eof on Unix. Add a test.
相关文件: src/channel.c, src/terminal.c, src/structs.h, src/testdir/test_terminal.vim

Patch 8.0.1037
问题: "icase" of 'diffopt' is not used for highlighting differences.
解决方案: Also use "icase". (Rick Howe)
相关文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.0.1038
问题: Strike-through text not supported.
解决方案: Add support for the "strikethrough" attribute. (Christian Brabandt, Ken Takata)
相关文件: runtime/doc/eval.txt, runtime/doc/options.txt, runtime/doc/syntax.txt, runtime/doc/term.txt, src/evalfunc.c, src/gui.c, src/gui.h, src/gui_gtk_x11.c, src/gui_mac.c, src/gui_w32.c, src/gui_x11.c, src/option.c, src/screen.c, src/syntax.c, src/term.c, src/term.h, src/terminal.c, src/vim.h

Patch 8.0.1039
问题: Cannot change a line in a buffer other than the current one.
解决方案: Add setbufline(). (Yasuhiro Matsumoto, Ozaki Kiichi, closes #1953)
相关文件: src/evalfunc.c, runtime/doc/eval.txt, src/Makefile, src/testdir/test_bufline.vim, src/testdir/test_alot.vim

Patch 8.0.1040
问题: Cannot use another error format in getqflist().
解决方案: Add the "efm" argument to getqflist(). (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1041
问题: Bogus characters appear when indenting kicks in while doing a visual-block append.
解决方案: Recompute when indenting is done. (Christian Brabandt)
相关文件: runtime/doc/visual.txt, src/charset.c, src/edit.c, src/misc1.c, src/ops.c, src/proto/charset.pro, src/proto/misc1.pro,

src/screen.c, src/spell.c, src/testdir/test_cindent.vim

Patch 8.0.1042 (after 8.0.1038)
问题: Without the syntax feature highlighting doesn't work.
解决方案: Always use unsigned short to store attributes.
相关文件: src/vim.h

Patch 8.0.1043
问题: Warning for uninitialized variable. (John Marriott)
解决方案: Move code to check indent inside "if".
相关文件: src/ops.c

Patch 8.0.1044
问题: Warning for uninitialized variable. (John Marriott)
解决方案: Initialize ind_pre.
相关文件: src/ops.c

Patch 8.0.1045
问题: Running tests may pollute shell history. (Manuel Ortega)
解决方案: Make $HISTFILE empty.
相关文件: src/testdir/setup.vim

Patch 8.0.1046
问题: Code duplication in diff mode.
解决方案: Use diff_equal_char() also in diff_cmp(). (Rick Howe)
相关文件: src/diff.c

Patch 8.0.1047
问题: Buffer overflow in Ruby.
解决方案: Allocate one more byte. (Dominique Pelle)
相关文件: src/if_ruby.c

Patch 8.0.1048
问题: No test for what 8.0.1020 fixes.
解决方案: Add test_feedinput(). Add a test. (Ozaki Kiichi, closes #2046)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_timers.vim, src/ui.c

Patch 8.0.1049
问题: Shell on Mac can't handle long text, making terminal test fail.
解决方案: Only write 1000 characters instead of 5000.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1050
问题: Terminal window feature not included by default.
解决方案: Include the terminal feature for the "huge" build.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1051
问题: Cannot run terminal with spaces in argument.
解决方案: Accept backslash to escape space and other characters. (closes #1999)
相关文件: src/os_unix.c, src/testdir/test_terminal.vim

Patch 8.0.1052
问题:　　term_start() does not allow in_io, out_io and err_io options.
解决方案:　Add JO_OUT_IO to get_job_options().
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1053
问题:　　setline() does not work on startup. (Manuel Ortega)
解决方案:　Do not check for ml_mfp to be set for the current buffer.
(Christian Brabandt)
相关文件:　src/testdir/shared.vim, src/testdir/test_alot.vim,
src/testdir/test_bufline.vim, src/testdir/test_timers.vim,
src/evalfunc.c

Patch 8.0.1054
问题:　　Terminal test fails on MS-Windows.
解决方案:　Disable the redirection test for now. Improve scrape test to make
it less flaky.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1055
问题:　　Bufline test hangs on MS-Windows.
解决方案:　Avoid message for writing file. Source shared.vim when running
test individually.
相关文件:　src/testdir/test_bufline.vim, src/testdir/test_timers.vim

Patch 8.0.1056
问题:　　Cannot build with the diff feature but without the multi-byte
feature.
解决方案:　Remove #ifdefs. (John Marriott)
相关文件:　src/diff.c

Patch 8.0.1057
问题:　　Terminal scrape test waits too long, it checks for one instead of
three.
解决方案:　Check there are three characters. (micbou)
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1058
问题:　　Terminal redirection test is flaky.
解决方案:　Wait for job to finish.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1059
问题:　　older Gnome terminal returns smaller version number. (antarestrue)
解决方案:　Lower version limit from 2800 to 2500. (#2032)
相关文件:　src/term.c

Patch 8.0.1060
问题:　　When imstyle is zero, mapping <Left> breaks preediting.
解决方案:　Pass though preediting key-events. (Yasuhiro Matsumoto, closes
#2064, closes #2063)
相关文件:　src/getchar.c, src/mbyte.c

Patch 8.0.1061

问题:　　Coverity: no check for NULL command.
解决方案:　Check for NULL list item.
相关文件:　src/terminal.c

Patch 8.0.1062
问题:　　Coverity warnings in libvterm.
解决方案:　Add (void) to avoid warning for not checking return value.
Add "break" before "case".
相关文件:　src/libvterm/src/screen.c, src/libvterm/src/state.c

Patch 8.0.1063
问题:　　Coverity warns for NULL check and using variable pointer as an
array.
解决方案:　Remove the NULL check. Make "argvar" an array.
相关文件:　src/terminal.c

Patch 8.0.1064
问题:　　Coverity warns for leaking resource.
解决方案:　Free pty_master_fd on failure.
相关文件:　src/os_unix.c

Patch 8.0.1065
问题:　　Not all macro examples are included in the self-installing
executable. (lkintact)
解决方案:　Add the directories to the NSIS script. (closes #2065)
相关文件:　nsis/gvim.nsi

Patch 8.0.1066
问题:　　Some terminals can't handle requesting cursor mode. (Steven
Hartland)
解决方案:　Recognize vandyke SecureCRT. (closes #2008)
相关文件:　src/term.c

Patch 8.0.1067
问题:　　Using try/catch in timer does not prevent it from being stopped.
解决方案:　Reset the exception context and use did_emsg instead of
called_emsg.
相关文件:　src/ex_cmds2.c, src/testdir/test_timers.vim, src/globals.h,
src/message.c

Patch 8.0.1068 (after 8.0.1066)
问题:　　Vandyke SecureCRT terminal can't handle cursor mode request.
(Steven Hartland)
解决方案:　Fix pointer computation. (closes #2008)
相关文件:　src/term.c

Patch 8.0.1069
问题:　　Still get CTRL-X sometimes for t_RS request.
解决方案:　Also skip 0x18 after a key code response.
相关文件:　src/term.c

Patch 8.0.1070
问题:　　Terminal test is flaky on Mac.
解决方案:　Add Test_terminal_noblock() to list of flaky tests.

相关文件: src/testdir/runtest.vim

Patch 8.0.1071
问题: $TERM names starting with "putty" and "cygwin" are likely to have a dark background, but are not recognized.
解决方案: Only check the first few characters of $TERM to match "putty" or "cygwin". (Christian Brabandt)
相关文件: src/option.c

Patch 8.0.1072
问题: The :highlight command causes a redraw even when nothing changed.
解决方案: Only set "need_highlight_changed" when an attribute changed.
相关文件: src/syntax.c

Patch 8.0.1073
问题: May get an endless loop if 'statusline' changes a highlight.
解决方案: Do not let evaluating 'statusline' trigger a redraw.
相关文件: src/buffer.c

Patch 8.0.1074
问题: ":term NONE" does not work on MS-Windows.
解决方案: Make it work. Split "pty" into "pty_in" and "pty_out". (Yasuhiro Matsumoto, closes #2058, closes #2045)
相关文件: runtime/doc/eval.txt, runtime/pack/dist/opt/termdebug/plugin/termdebug.vim, src/channel.c, src/evalfunc.c, src/os_unix.c, src/structs.h, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1075
问题: MS-Windows: mouse does not work in terminal.
解决方案: Force the winpty mouse on. (Yasuhiro Matsumoto, closes #2072)
相关文件: src/terminal.c

Patch 8.0.1076
问题: term_start() does not take callbacks. When using two terminals without a job only one is read from. A terminal without a window returns the wrong pty.
解决方案: Support "callback", "out_cb" and "err_cb". Fix terminal without a window. Fix reading from multiple channels.
相关文件: src/terminal.c, src/proto/terminal.pro, src/channel.c

Patch 8.0.1077
问题: No debugger making use of the terminal window.
解决方案: Add the term debugger plugin. So far only displays the current line when stopped.
相关文件: Filelist, runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1078
问题: Using freed memory with ":hi Normal".
解决方案: Get "item" again after updating the table.
相关文件: src/syntax.c

Patch 8.0.1079
问题: Memory leak when remote_foreground() fails.

解决方案: Free the error message.
相关文件: src/evalfunc.c, src/if_xcmdsrv.c

Patch 8.0.1080
问题: Memory leak for eof_chars terminal option and buffer name.
解决方案: Free job options. Free the buffer name
相关文件: src/terminal.c

Patch 8.0.1081
问题: Memory leak for the channel write queue.
解决方案: Free the write queue when clearing a channel.
相关文件: src/channel.c

Patch 8.0.1082
问题: Tests fail when run under valgrind.
解决方案: Increase waiting times.
相关文件: src/testdir/test_clientserver.vim, src/testdir/test_terminal.vim

Patch 8.0.1083
问题: Leaking memory in input part of channel.
解决方案: Clear the input part of channel. Free the entry. Move failing
           command test to a separate file to avoid bogus leak reports
           clouding tests that should not leak.
相关文件: src/channel.c, src/testdir/test_terminal.vim, src/Makefile,
           src/testdir/test_terminal_fail.vim, src/testdir/Make_all.mak

Patch 8.0.1084
问题: GTK build has compiler warnings. (Christian Brabandt)
解决方案: Get screen size with a different function. (Ken Takata, Yasuhiro
           Matsumoto)
相关文件: src/mbyte.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro,
           src/gui_beval.c

Patch 8.0.1085
问题: The terminal debugger can't set breakpoints.
解决方案: Add :Break and :Delete commands. Also commands for stepping
           through code.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
           runtime/doc/terminal.txt

Patch 8.0.1086 (after 8.0.1084)
问题: Can't build with GTK 3.
解决方案: Rename function argument. (Kazunobu Kuriyama)
相关文件: src/gui_gtk_x11.c

Patch 8.0.1087
问题: Test_terminal_cwd is flaky. MS-Windows: term_start() "cwd"
           argument does not work.
解决方案: Wait for the condition to be true instead of using a sleep.
           Pass the directory to winpty.
相关文件: src/testdir/test_terminal.vim, src/terminal.c

Patch 8.0.1088
问题: Occasional memory use after free.

解决方案: Use the highlight table directly, don't keep a pointer.
相关文件: src/syntax.c

Patch 8.0.1089
问题: Cannot get range count in user command.
解决方案: Add <range> argument.
相关文件: src/ex_docmd.c, runtime/doc/map.txt

Patch 8.0.1090
问题: cannot get the text under the cursor like v:beval_text
解决方案: Add <cexpr>.
相关文件: src/ex_docmd.c, src/testdir/test_normal.vim,
           runtime/doc/cmdline.txt

Patch 8.0.1091 (after 8.0.1090)
问题: Test for <cexpr> fails without +balloon_eval feature.
解决方案: Remove #ifdefs.
相关文件: src/normal.c

Patch 8.0.1092
问题: Terminal debugger can't evaluate expressions.
解决方案: Add :Evaluate and K. Various other improvements.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
           runtime/doc/terminal.txt

Patch 8.0.1093
问题: Various small quickfix issues.
解决方案: Remove ":" prefix from title set by a user. Add the qf_id2nr().
           function. Add a couple more tests. Update documentation.
           (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/evalfunc.c,
           src/proto/quickfix.pro, src/quickfix.c,
           src/testdir/test_quickfix.vim

Patch 8.0.1094
问题: Using ssh from Terminal.app runs into xterm incompatibility.
解决方案: Also detect Terminal.app on non-Mac systems.
相关文件: src/term.c

Patch 8.0.1095
问题: Terminal multibyte scrape test is flaky.
解决方案: Add another condition to wait for.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1096
问题: Terminal window in Normal mode has wrong background.
解决方案: Store the default background and use it for clearing until the
           end of the line. Not for below the last line, since there is no
           text there.
相关文件: src/screen.c, src/terminal.c

Patch 8.0.1097 (after 8.0.1096)
问题: Background color wrong if job changes background color.
解决方案: Get the background color from vterm.

相关文件: src/terminal.c, src/screen.c

Patch 8.0.1098
问题: Build failure if libvterm installed on the system. (Oleh Hushchenkov)
解决方案: Change the CCCTERM argument order. (Ken Takata, closes #2080)
相关文件: src/Makefile

Patch 8.0.1099
问题: Warnings for GDK calls.
解决方案: Use other calls for GTK 3 and fix a few problems. (Kazunobu Kuriyama)
相关文件: src/mbyte.c

Patch 8.0.1100
问题: Stuck in redraw loop when 'lazyredraw' is set.
解决方案: Don't loop on update_screen() when not redrawing. (Yasuhiro Matsumoto, closes #2082)
相关文件: src/terminal.c, src/screen.c, src/proto/screen.pro

Patch 8.0.1101
问题: Channel write fails if writing to log fails.
解决方案: Ignore return value of fwrite(). (Ozaki Kiichi, closes #2081)
相关文件: src/channel.c

Patch 8.0.1102
问题: Terminal window does not use Normal colors.
解决方案: For the GUI and when 'termguicolors' is enabled, use the actual foreground and background colors for the terminal. (Yasuhiro Matsumoto, closes #2067)
Use the "Terminal" highlight group if defined.
相关文件: src/terminal.c, src/syntax.c, src/proto/syntax.pro

Patch 8.0.1103 (after 8.0.1102)
问题: Converting cterm color fails for grey ramp.
解决方案: Use index instead of number.
相关文件: src/terminal.c

Patch 8.0.1104
问题: The qf_jump() function is too long.
解决方案: Split of parts to separate functions. (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.1105
问题: match() and matchend() are not tested.
解决方案: Add tests. (Ozaki Kiichi, closes #2088)
相关文件: src/testdir/test_functions.vim, src/testdir/test_match.vim

Patch 8.0.1106
问题: Terminal colors on an MS-Windows console are not matching the normal colors.
解决方案: Use the normal colors for the terminal. (Yasuhiro Matsumoto, closes #2087)
相关文件: src/terminal.c

Patch 8.0.1107
问题: Terminal debugger jumps to non-existing file.
解决方案: Check that the file exists. Add an option to make the Vim width wide. Fix removing highlight groups.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim, runtime/doc/terminal.txt

Patch 8.0.1108
问题: Cannot specify mappings for the terminal window.
解决方案: Add the :tmap command and associated code. (Jacob Askeland, closes #2073)
相关文件: runtime/doc/map.txt, runtime/doc/terminal.txt, src/ex_cmdidxs.h, src/ex_cmds.h, src/ex_docmd.c, src/getchar.c, src/gui.c, src/terminal.c, src/testdir/test_terminal.vim, src/vim.h, src/proto/terminal.pro, src/main.c, src/evalfunc.c

Patch 8.0.1109
问题: Timer causes error on exit from Ex mode. (xtal8)
解决方案: save and restore the ex_pressedreturn flag. (Christian Brabandt, closes #2079)
相关文件: src/ex_docmd.c, src/proto/ex_docmd.pro, src/ex_cmds2.c, src/testdir/test_timers.vim

Patch 8.0.1110
问题: FORTIFY_SOURCE from Perl causes problems. (Scott Baker)
解决方案: Filter out the flag. (Christian Brabandt, closes #2068)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1111
问题: Syntax error in configure when using Perl.
解决方案: Add missing quote
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1112
问题: Can't get size or current index from quickfix list.
解决方案: Add "idx" and "size" options. (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1113
问题: Can go to Insert mode from Terminal-Normal mode.
解决方案: Prevent :startinsert and "VA" to enter Insert mode. (Yasuhiro Matsumoto, closes #2092)
相关文件: src/normal.c

Patch 8.0.1114
问题: Default for 'iminsert' is annoying.
解决方案: Make the default always zero. (Yasuhiro Matsumoto, closes #2071)
相关文件: src/option.c, runtime/doc/options.txt

Patch 8.0.1115
问题: Crash when using foldtextresult() recursively.
解决方案: Avoid recursive calls. (Yasuhiro Matsumoto, closes #2098)

相关文件: src/evalfunc.c, src/testdir/test_fold.vim

Patch 8.0.1116
问题: Terminal test fails on MS-Windows.
解决方案: Wait for the text to appear. (micbou, closes #2097)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1117
问题: Test_terminal_no_cmd hangs on MS-Windows with GUI. (Christian Brabandt)
解决方案: Run the command with "start" and wait for the text to appear. (micbou, closes #2096)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1118
问题: FEAT_WINDOWS adds a lot of #ifdefs while it is nearly always enabled and only adds 7% to the binary size of the tiny build.
解决方案: Graduate FEAT_WINDOWS.
相关文件: src/feature.h, src/window.c, src/vim.h, src/structs.h, src/globals.h, src/gui.h, src/if_py_both.h, src/option.h, src/term.h, src/buffer.c, src/charset.c, src/digraph.c, src/edit.c, src/eval.c, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/fold.c, src/getchar.c, src/gui.c, src/gui_athena.c, src/gui_beval.c, src/gui_gtk.c, src/gui_motif.c, src/gui_w32.c, src/if_cscope.c, src/if_lua.c, src/if_mzsch.c, src/if_python.c, src/if_python3.c, src/if_ruby.c, src/if_tcl.c, src/main.c, src/mark.c, src/memline.c, src/misc1.c, src/misc2.c, src/move.c, src/netbeans.c, src/normal.c, src/option.c, src/popupmnu.c, src/quickfix.c, src/screen.c, src/search.c, src/spell.c, src/syntax.c, src/tag.c, src/term.c, src/ui.c, src/version.c, src/workshop.c, src/if_perl.xs, src/testdir/test_normal.vim

Patch 8.0.1119
问题: Quitting a split terminal window kills the job. (Yasuhiro Matsumoto)
解决方案: Only stop terminal job if it is the last window.
相关文件: src/buffer.c, src/testdir/test_terminal.vim

Patch 8.0.1120 (after 8.0.1108)
问题: :tm means :tmap instead of :tmenu. (Taro Muraoka)
解决方案: Move the new entry below the old entry. (closes #2102)
相关文件: src/ex_cmds.h, runtime/doc/map.txt

Patch 8.0.1121
问题: Can uncheck executables in MS-Windows installer.
解决方案: Make the choice read-only. (Ken Takata, closes #2106)
相关文件: nsis/gvim.nsi

Patch 8.0.1122
问题: vimtutor.bat doesn't work well with vim.bat.
解决方案: Use "call vim". (Ken Takata, closes #2105)
相关文件: vimtutor.bat

Patch 8.0.1123
问题:　　Cannot define a toolbar for a window.
解决方案:　Add a window-local toolbar.
相关文件:　src/syntax.c, src/proto/syntax.pro, src/structs.h, src/menu.c,
src/proto/menu.pro, src/testdir/test_winbar.vim, src/Makefile,
src/normal.c, src/testdir/Make_all.mak, src/if_perl.xs,
src/eval.c, src/evalfunc.c, src/window.c, src/ui.c,
src/terminal.c, src/screen.c,
runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
runtime/doc/gui.txt, runtime/doc/terminal.txt

Patch 8.0.1124
问题:　　Use of MZSCHEME_VER is unclear.
解决方案:　Add a comment. (Ken Takata)
相关文件:　src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.1125
问题:　　Wrong window height when splitting window with window toolbar.
解决方案:　Add or subtract the window toolbar height.
相关文件:　src/window.c

Patch 8.0.1126
问题:　　Endless resize when terminal showing in two buffers. (Hirohito
Higashi)
解决方案:　Set a flag to prevent resizing the window.
相关文件:　src/terminal.c

Patch 8.0.1127
问题:　　Test_peek_and_get_char fails on 32 bit system. (Elimar
Riesebieter)
解决方案:　Avoid an integer overflow. (James McCoy, closes #2116)
相关文件:　src/ex_cmds2.c

Patch 8.0.1128
问题:　　Old xterm sends CTRL-X in response to t_RS.
解决方案:　Only send t_RS for xterm 279 and later. Remove the workaround to
ignore CTRL-X.
相关文件:　src/term.c

Patch 8.0.1129
问题:　　Window toolbar missing a part of the patch.
解决方案:　Add change in vim.h.
相关文件:　src/vim.h

Patch 8.0.1130
问题:　　The qf_jump() function is still too long.
解决方案:　Split of parts to separate functions. (Yegappan Lakshmanan)
相关文件:　src/quickfix.c

Patch 8.0.1131
问题:　　It is not easy to trigger an autocommand for new terminal window.
(Marco Restelli)
解决方案:　Trigger BufWinEnter after setting 'buftype'.
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1132
问题: #if condition is not portable.
解决方案: Add defined(). (Zuloloxi, closes #2136)
相关文件: src/libvterm/src/vterm.c

Patch 8.0.1133
问题: Syntax timeout not used correctly.
解决方案: Do not pass the timeout to syntax_start() but set it explicitly.
            (Yasuhiro Matsumoto, closes #2139)
相关文件: src/proto/syntax.pro, src/screen.c, src/syntax.c

Patch 8.0.1134
问题: Superfluous call to syn_get_final_id().
解决方案: Remove it. (Ken Takata)
相关文件: src/syntax.c

Patch 8.0.1135
问题: W_WINCOL() is always the same.
解决方案: Expand the macro.
相关文件: src/edit.c, src/ex_docmd.c, src/gui_gtk.c, src/gui_w32.c,
            src/netbeans.c, src/popupmnu.c, src/screen.c, src/term.c,
            src/terminal.c, src/ui.c, src/window.c, src/if_py_both.h,
            src/structs.h, src/vim.h

Patch 8.0.1136
问题: W_WIDTH() is always the same.
解决方案: Expand the macro.
相关文件: src/charset.c, src/edit.c, src/evalfunc.c, src/ex_cmds.c,
            src/ex_docmd.c, src/getchar.c, src/gui.c, src/gui_beval.c,
            src/gui_mac.c, src/if_lua.c, src/if_mzsch.c, src/if_py_both.h,
            src/if_ruby.c, src/misc1.c, src/misc2.c, src/move.c, src/normal.c,
            src/popupmnu.c, src/quickfix.c, src/screen.c, src/search.c,
            src/structs.h, src/ui.c, src/vim.h, src/window.c

Patch 8.0.1137 (after 8.0.1136)
问题: Cannot build with Ruby.
解决方案: Fix misplaced brace.
相关文件: src/if_ruby.c

Patch 8.0.1138
问题: Click in window toolbar starts Visual mode.
解决方案: Add the MOUSE_WINBAR flag.
相关文件: src/ui.c, src/vim.h, src/normal.c

Patch 8.0.1139
问题: Using window toolbar changes state.
解决方案: Always execute window toolbar actions in Normal mode.
相关文件: runtime/doc/gui.txt, src/structs.h, src/ex_docmd.c,
            src/proto/ex_docmd.pro, src/menu.c

Patch 8.0.1140
问题: Still old style tests.
解决方案: Convert two tests to new style. (Yegappan Lakshmanan)

相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/test56.in, src/testdir/test56.ok,
           src/testdir/test57.in, src/testdir/test57.ok,
           src/testdir/test_sort.vim, src/testdir/test_vimscript.vim

Patch 8.0.1141
问题: MS-Windows build dependencies are incomplete.
解决方案: Fix the dependencies. (Ken Takata)
相关文件: src/Make_cyg.mak, src/Make_cyg_ming.mak, src/Make_ming.mak,
           src/Make_mvc.mak

Patch 8.0.1142
问题: Window toolbar menu gets a tear-off item.
解决方案: Recognize the window toolbar.
相关文件: src/menu.c

Patch 8.0.1143
问题: Macros always expand to the same thing.
解决方案: Remove W_VSEP_WIDTH() and W_STATUS_HEIGHT().
相关文件: src/vim.h, src/structs.h, src/gui.c, src/ex_getln.c, src/screen.c

Patch 8.0.1144
问题: Using wrong #ifdef for computing length.
解决方案: use BACKSLASH_IN_FILENAME instead of COLON_IN_FILENAME. (Yasuhiro
           Matsumoto, closes #2153)
相关文件: src/quickfix.c

Patch 8.0.1145
问题: Warning when compiling with Perl.
解决方案: Remove unused variable. (Ken Takata)
相关文件: src/if_perl.xs

Patch 8.0.1146
问题: Redraw when highlight is set with same names. (Ozaki Kiichi)
解决方案: Only free and save a name when it changed. (closes #2120)
相关文件: src/syntax.c

Patch 8.0.1147
问题: Fail to build with tiny features. (Tony Mechelynck)
解决方案: Move #ifdefs.
相关文件: src/syntax.c

Patch 8.0.1148
问题: "gN" doesn't work on last match with 'wrapscan' off. (fcpg)
解决方案: Adjust for searching backward. (Christian Brabandt)
相关文件: src/search.c, src/testdir/test_gn.vim

Patch 8.0.1149
问题: libvterm colors differ from xterm.
解决方案: Use the xterm colors for libvterm.
相关文件: src/terminal.c, src/libvterm/src/pen.c,
           src/testdir/xterm_ramp.vim, Filelist

Patch 8.0.1150

问题:	MS-Windows GUI: dialog font size is incorrect.
解决方案:	Pass flag to indicate 'encoding' or active codepage. (Yasuhiro Matsumoto, closes #2160)
相关文件:	src/gui_w32.c

Patch 8.0.1151
问题:	"vim -c startinsert!" doesn't append.
解决方案:	Correct line number on startup. (Christian Brabandt, closes #2117)
相关文件:	src/ex_docmd.c, src/testdir/test_startup.vim

Patch 8.0.1152
问题:	Encoding of error message wrong in Cygwin terminal.
解决方案:	Get locale from environment variables. (Ken Takata)
相关文件:	src/main.c, src/mbyte.c, src/proto/mbyte.pro

Patch 8.0.1153
问题:	No tests for diff_hlID() and diff_filler().
解决方案:	Add tests. (Dominique Pelle, closes #2156)
相关文件:	src/testdir/test_diffmode.vim

Patch 8.0.1154
问题:	'indentkeys' does not work properly. (Gary Johnson)
解决方案:	Get the cursor line again. (Christian Brabandt, closes #2151)
相关文件:	src/edit.c, src/testdir/test_edit.vim

Patch 8.0.1155
问题:	Ruby command triggers a warning when RUBYOPT is set to "-w".
解决方案:	use "-e_=0" instead of "-e0". (Masataka Pocke Kuwabara, closes #2143)
相关文件:	src/if_ruby.c

Patch 8.0.1156
问题:	Removing one -W argument from Perl CFLAGS may cause trouble.
解决方案:	Remove all -W flags. (Christian Brabandt)
相关文件:	src/configure.ac, src/auto/configure

Patch 8.0.1157
问题:	Compiler warning on MS-Windows.
解决方案:	Add type cast. (Yasuhiro Matsumoto)
相关文件:	src/main.c

Patch 8.0.1158
问题:	Still old style tests.
解决方案:	Convert several tests to new style. (Yegappan Lakshmanan)
相关文件:	src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms, src/testdir/main.aap, src/testdir/test33.in, src/testdir/test33.ok, src/testdir/test41.in, src/testdir/test41.ok, src/testdir/test43.in, src/testdir/test43.ok, src/testdir/test53.in, src/testdir/test53.ok, src/testdir/test_file_size.vim, src/testdir/test_lispwords.vim, src/testdir/test_search.vim, src/testdir/test_textobjects.vim

Patch 8.0.1159

问题: Typo in #ifdef.
解决方案: Change "PROT" to "PROTO". (Nobuhiro Takasaki, closes #2165)
相关文件: src/syntax.c

Patch 8.0.1160
问题: Getting tab-local variable fails after closing window.
解决方案: set tp_firstwin and tp_lastwin. (Jason Franklin, closes #2170)
相关文件: src/window.c, src/evalfunc.c, src/testdir/test_getvar.vim

Patch 8.0.1161
问题: Popup menu drawing problem when resizing terminal.
解决方案: Redraw after resizing also when a popup menu is visible. (Ozaki
            Kiichi, closes #2110)
相关文件: src/popupmnu.c, src/term.c, src/testdir/shared.vim,
            src/testdir/test_popup.vim

Patch 8.0.1162
问题: Shared script for tests cannot be included twice.
解决方案: Include it where needed, it will "finish" if loaded again.
相关文件: src/testdir/test_alot.vim, src/testdir/test_bufline.vim,
            src/testdir/test_timers.vim

Patch 8.0.1163
问题: Popup test is flaky.
解决方案: Add a WaitFor() and fix another.
相关文件: src/testdir/test_popup.vim

Patch 8.0.1164
问题: Changing StatusLine highlight while evaluating 'statusline' may
            not change the status line color.
解决方案: When changing highlighting while redrawing don't cause another
            redraw. (suggested by Ozaki Kiichi, closes #2171, closes #2120)
相关文件: src/buffer.c, src/syntax.c

Patch 8.0.1165
问题: Popup test is still flaky.
解决方案: Add a term_wait() call. (Ozaki Kiichi)
相关文件: src/testdir/test_popup.vim

Patch 8.0.1166
问题: :terminal doesn't work on Mac High Sierra.
解决方案: Change #ifdef for OpenPTY(). (Ozaki Kiichi, Kazunobu Kuriyama,
            closes #2162)
相关文件: src/pty.c

Patch 8.0.1167
问题: Motif: typing in terminal window is slow.
解决方案: Do not redraw the whole terminal window but only what was changed.
相关文件: src/terminal.c

Patch 8.0.1168
问题: wrong highlighting with combination of match and 'cursorline'.
解决方案: Use "line_attr" when appropriate. (Ozaki Kiichi, closes #2111)
            But don't highlight more than one character.

相关文件: src/screen.c, src/testdir/test_highlight.vim,
           src/testdir/view_util.vim

Patch 8.0.1169
问题: Highlighting one char too many with 'list' and 'cul'.
解决方案: Check for 'list' being active. (Ozaki Kiichi, closes #2177)
相关文件: src/screen.c, src/testdir/test_highlight.vim

Patch 8.0.1170
问题: Using termdebug results in 100% CPU time. (tomleb)
解决方案: Use polling instead of select().
相关文件: src/os_unix.c, src/channel.c, src/proto/channel.pro

Patch 8.0.1171
问题: Popup test is still a bit flaky.
解决方案: Change term_wait() calls. (Ozaki Kiichi)
相关文件: src/testdir/test_popup.vim

Patch 8.0.1172
问题: When E734 is given option is still set.
解决方案: Assign NULL to "s". (Christian Brabandt)
相关文件: src/eval.c, src/testdir/test_assign.vim

Patch 8.0.1173
问题: Terminal window is not redrawn after CTRL-L. (Marcin Szamotulski)
解决方案: Redraw the whole terminal when w_redr_type is NOT_VALID.
相关文件: src/terminal.c

Patch 8.0.1174
问题: Mac Terminal.app has wrong color for white.
解决方案: Use white from the color cube.
相关文件: src/globals.h, src/term.c, src/syntax.c

Patch 8.0.1175 (after 8.0.1174)
问题: Build failure without +termresponse.
解决方案: Add #ifdef.
相关文件: src/syntax.c

Patch 8.0.1176
问题: Job_start() does not handle quote and backslash correctly.
解决方案: Remove quotes, recognize and remove backslashes.
相关文件: src/testdir/test_channel.vim, src/os_unix.c

Patch 8.0.1177
问题: In a terminal window the popup menu is not cleared. (Gerry
           Agbobada)
解决方案: Redraw when SOME_VALID is used instead of NOT_VALID. (closes
           #2194)
相关文件: src/terminal.c

Patch 8.0.1178
问题: Using old compiler on MS-Windows.
解决方案: Switch default build on MS-Windows to use MSVC 2015. (Ken Takata)
相关文件: src/msvc2015.bat, src/INSTALLpc.txt, src/GvimExt/Makefile,

src/Make_mvc.mak, src/tee/Make_mvc.mak, src/xxd/Make_mvc.mak

Patch 8.0.1179
问题: Test_popup_and_window_resize() does not always pass.
解决方案: Do not use $VIMPROG, pass the Vim executable in the vimcmd file.
(Ozaki Kiichi, closes #2186)
相关文件: src/testdir/Makefile, src/testdir/shared.vim,
src/testdir/test_popup.vim

Patch 8.0.1180
问题: MS-Windows testclean target deletes the color script.
解决方案: Rename the script file.
相关文件: src/testdir/xterm_ramp.vim, src/testdir/color_ramp.vim

Patch 8.0.1181
问题: Tests using Vim command fail on MS-Windows.
解决方案: Do not add quotes around the Vim command.
相关文件: src/testdir/Make_dos.mak, src/testdir/Make_ming.mak

Patch 8.0.1182
问题: Cannot see or change mzscheme dll name.
解决方案: Add 'mzschemedll' and 'mzschemegcdll'.
相关文件: src/if_mzsch.c, src/option.h, src/option.c,
runtime/doc/if_mzsch.txt

Patch 8.0.1183
问题: MS-Windows build instructions are outdated.
解决方案: Update instructions for MSVC 2015. Update the build script.
相关文件: Filelist, Makefile, src/INSTALLpc.txt, src/bigvim.bat

Patch 8.0.1184
问题: The :marks command is not tested.
解决方案: Add a test. (Dominique Pelle, closes #2197)
相关文件: src/testdir/test_marks.vim

Patch 8.0.1185
问题: Ruby library includes minor version number.
解决方案: Only use the API version number. (Ben Boeckel, closes #2199)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1186
问题: Still quite a few old style tests.
解决方案: Convert old to new style tests. (Yegappan Lakshmanan)
Avoid ringing the bell while running tests.
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_ming.mak,
src/testdir/Make_vms.mms, src/testdir/main.aap,
src/testdir/test31.in, src/testdir/test31.ok,
src/testdir/test4.in, src/testdir/test4.ok, src/testdir/test5.in,
src/testdir/test5.ok, src/testdir/test60.in,
src/testdir/test60.ok, src/testdir/test60.vim,
src/testdir/test7.in, src/testdir/test7.ok, src/testdir/test78.in,
src/testdir/test78.ok, src/testdir/test_autocmd.vim,
src/testdir/test_exists.vim, src/testdir/test_recover.vim,
src/testdir/test_winbuf_close.vim, src/testdir/runtest.vim

Patch 8.0.1187
问题: Building with lua fails for OSX on Travis.
解决方案: Separate brew-update and brew-install. (Ozaki Kiichi, closes #2203)
相关文件: .travis.yml

Patch 8.0.1188
问题: Autocmd test fails on MS-Windows.
解决方案: Give the buffer a name and find the buffer to be wiped out by name.
相关文件: src/testdir/test_autocmd.vim

Patch 8.0.1189
问题: E172 is not actually useful, it's only on Unix anyway.
解决方案: Remove the check and the error.
相关文件: src/ex_docmd.c, runtime/doc/message.txt

Patch 8.0.1190
问题: Vim becomes unusable after opening new window in BufWritePre event.
解决方案: Call not_exiting(). (Martin Tournoij, closes #2205)
Also for "2q" when a help window is open. Add a test.
相关文件: src/ex_docmd.c, src/testdir/test_writefile.vim

Patch 8.0.1191
问题: MS-Windows: missing 32 and 64 bit files in installer.
解决方案: Include both 32 and 64 bit GvimExt and related dll files. Remove old Windows code from the installer. (Ken Takata, closes #2144)
相关文件: nsis/README.txt, nsis/gvim.nsi, src/GvimExt/gvimext.cpp, src/dosinst.c, src/dosinst.h, src/uninstal.c, Makefile

Patch 8.0.1192
问题: MS-Windows: terminal feature not enabled by default.
解决方案: Enable it. (Ken Takata)
相关文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.0.1193
问题: Crash when wiping out a buffer after using getbufinfo(). (Yegappan Lakshmanan)
解决方案: Remove b:changedtick from the buffer variables.
相关文件: src/buffer.c, src/testdir/test_autocmd.vim

Patch 8.0.1194
问题: Actual fg and bg colors of terminal are unknown.
解决方案: Add t_RF. Store response to t_RB and t_RF, use for terminal.
相关文件: src/term.c, src/term.h, src/proto/term.pro, src/terminal.c, src/vim.h, src/eval.c, runtime/doc/eval.txt

Patch 8.0.1195 (after 8.0.1194)
问题: Can't build on MS-Windows.
解决方案: Adjust #ifdef and add #ifdefs.
相关文件: src/term.c, src/terminal.c

Patch 8.0.1196 (after 8.0.1194)

问题:　　Crash when t_RF is not set. (Brian Pina)
解决方案:　Add t_RF to the list of terminal options. (Hirohito Higashi)
相关文件:　src/option.c

Patch 8.0.1197
问题:　　MS-Windows build instructions are not up to date.
解决方案:　Adjust the instructions. Fix the nsis script.
相关文件:　Makefile, nsis/gvim.nsi

Patch 8.0.1198
问题:　　Older compilers don't know uint8_t.
解决方案:　Use char_u instead.
相关文件:　src/term.c, src/proto/term.pro

Patch 8.0.1199
问题:　　When 'clipboard' is "autoselectplus" the star register is also
　　set. (Gilles Moris)
解决方案:　Don't set the star register in this situation.
相关文件:　src/ops.c

Patch 8.0.1200
问题:　　Tests switch the bell off twice.
解决方案:　Don't set 'belloff' in individual tests. (Christian Brabandt)
相关文件:　src/testdir/test_alot.vim, src/testdir/test_alot_utf8.vim,
　　src/testdir/test_autocmd.vim, src/testdir/test_cmdline.vim,
　　src/testdir/test_diffmode.vim, src/testdir/test_digraph.vim,
　　src/testdir/test_edit.vim, src/testdir/test_file_size.vim,
　　src/testdir/test_gn.vim, src/testdir/test_normal.vim,
　　src/testdir/test_packadd.vim, src/testdir/test_popup.vim,
　　src/testdir/test_recover.vim, src/testdir/test_search.vim,
　　src/testdir/test_textobjects.vim, src/testdir/test_undo.vim,
　　src/testdir/test_usercommands.vim, src/testdir/test_visual.vim

Patch 8.0.1201
问题:　　"yL" is affected by 'scrolloff'. (Eli the Bearded)
解决方案:　Don't use 'scrolloff' when an operator is pending.
相关文件:　src/normal.c, runtime/doc/motion.txt

Patch 8.0.1202
问题:　　:wall gives an error for a terminal window. (Marius Gedminas)
解决方案:　Don't try writing a buffer that can't be written. (Yasuhiro
　　Matsumoto, closes #2190)
相关文件:　src/ex_cmds.c, src/testdir/test_terminal.vim

Patch 8.0.1203
问题:　　Terminal window mistreats composing characters.
解决方案:　Count composing characters with the base character. (Ozaki Kiichi,
　　closes #2195)
相关文件:　src/mbyte.c, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1204
问题:　　A QuitPre autocommand may get the wrong file name.
解决方案:　Pass the buffer being closed to apply_autocmds(). (Rich Howe)
相关文件:　src/ex_docmd.c, src/testdir/test_autocmd.vim

Patch 8.0.1205
问题: Using "1q" it is possible to unload a changed buffer. (Rick Howe)
解决方案: Check the right window for changes.
相关文件: src/testdir/test_edit.vim, src/ex_docmd.c

Patch 8.0.1206
问题: No autocmd for entering or leaving the command line.
解决方案: Add CmdlineEnter and CmdlineLeave.
相关文件: runtime/doc/autocmd.txt, src/ex_getln.c, src/fileio.c, src/vim.h, src/testdir/test_autocmd.vim

Patch 8.0.1207
问题: Profiling skips the first and last script line.
解决方案: Check for BOM after setting script ID. (LemonBoy, closes #2103, closes #2112) Add a test. List the trailing script lines.
相关文件: src/testdir/test_profile.vim, src/ex_cmds2.c

Patch 8.0.1208
问题: 'statusline' drops empty group with highlight change.
解决方案: Do not drop an empty group if it changes highlighting. (Marius Gedminas, closes #2228)
相关文件: src/buffer.c, src/testdir/test_statusline.vim

Patch 8.0.1209
问题: Still too many old style tests.
解决方案: Convert a few more tests to new style. (Yegappan Lakshmanan, closes #2230)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_ming.mak, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Makefile, src/testdir/Make_vms.mms, src/testdir/main.aap, src/testdir/test34.in, src/testdir/test34.ok, src/testdir/test54.in, src/testdir/test54.ok, src/testdir/test8.in, src/testdir/test8.ok, src/testdir/test_autocmd.vim, src/testdir/test_autoformat_join.in, src/testdir/test_autoformat_join.ok, src/testdir/test_join.vim, src/testdir/test_user_func.vim

Patch 8.0.1210
问题: When typing a search pattern CTRL-G and CTRL-T are ignored when there is typeahead.
解决方案: Don't pass SEARCH_PEEK and don't call char_avail(). (haya14busa, closes #2233)
相关文件: src/ex_getln.c, src/testdir/test_search.vim

Patch 8.0.1211
问题: Cannot reorder tab pages with drag & drop.
解决方案: Support drag & drop for GTK and MS-Windows. (Ken Takata, Masamichi Abe)
相关文件: src/gui_gtk_x11.c, src/gui_w32.c

Patch 8.0.1212
问题: MS-Windows: tear-off menu does not work on 64 bit. (shaggyaxe)
解决方案: Change how the menu handle is looked up. (Ken Takata, closes

#1205)
相关文件: src/gui_w32.c

Patch 8.0.1213
问题: Setting 'mzschemedll' has no effect.
解决方案: Move loading .vimrc to before call to mzscheme_main().
相关文件: src/main.c

Patch 8.0.1214
问题: Accessing freed memory when EXITFREE is set and there is more than one tab and window. (Dominique Pelle)
解决方案: Free options later. Skip redraw when exiting.
相关文件: src/screen.c, src/misc2.c

Patch 8.0.1215
问题: Newer gcc warns for implicit fallthrough.
解决方案: Consistently use a FALLTHROUGH comment. (Christian Brabandt)
相关文件: src/buffer.c, src/edit.c, src/eval.c, src/ex_docmd.c, src/ex_getln.c, src/main.c, src/message.c, src/normal.c, src/regexp.c, src/regexp_nfa.c, src/spell.c, src/window.c, src/if_perl.xs

Patch 8.0.1216
问题: Tabline is not always updated for :file command. (Norio Takagi)
解决方案: Set redraw_tabline. (Hirohito Higashi)
相关文件: src/ex_cmds.c

Patch 8.0.1217
问题: Can't use remote eval to inspect vars in debug mode.
解决方案: Don't discard the call stack in debug mode. (closes #2237, #2247)
相关文件: src/globals.h, src/ex_cmds2.c, src/main.c

Patch 8.0.1218
问题: Writing to freed memory in autocmd.
解决方案: Make a copy of the tag line. (Dominique Pelle, closes #2245)
相关文件: src/tag.c, src/testdir/test_autocmd.vim

Patch 8.0.1219
问题: Terminal test is flaky.
解决方案: Add test function to list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.1220
问题: Skipping empty statusline groups is not correct.
解决方案: Also set group_end_userhl. (itchyny)
相关文件: src/buffer.c, src/testdir/test_statusline.vim

Patch 8.0.1221
问题: Still too many old style tests.
解决方案: Convert a few more tests to new style. (Yegappan Lakshmanan, closes #2256)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak, src/testdir/Make_ming.mak, src/testdir/Make_vms.mms,

src/testdir/main.aap, src/testdir/test19.in,
src/testdir/test19.ok, src/testdir/test20.in,
src/testdir/test20.ok, src/testdir/test25.in,
src/testdir/test25.ok, src/testdir/test28.in,
src/testdir/test28.ok, src/testdir/test32.in,
src/testdir/test32.ok, src/testdir/test38.in,
src/testdir/test38.ok, src/testdir/test66.in,
src/testdir/test66.ok, src/testdir/test79.in,
src/testdir/test79.ok, src/testdir/test_ins_complete.vim,
src/testdir/test_source_utf8.vim, src/testdir/test_substitute.vim,
src/testdir/test_tab.vim, src/testdir/test_tagjump.vim,
src/testdir/test_undo.vim, src/testdir/test_visual.vim,
src/testdir/test79.ok, src/testdir/test79.in,
src/testdir/test28.in

Patch 8.0.1222
问题: Test functions interfere with each other.
解决方案: Cleanup tab pages, windows and buffers. Reset option.
相关文件: src/testdir/runtest.vim, src/testdir/test_filetype.vim,
src/testdir/test_tabpage.vim, src/testdir/test_lispwords.vim

Patch 8.0.1223
问题: Crash when using autocomplete and tab pages.
解决方案: Check if the current tab changed. (Christian Brabandt, closes
#2239)
相关文件: src/popupmnu.c, src/testdir/test_popup.vim, src/misc1.c,

Patch 8.0.1224
问题: Still interference between test functions.
解决方案: Clear autocommands. Wipe all buffers. Fix tests that depend on a
specific start context.
相关文件: src/testdir/runtest.vim, src/testdir/test_autocmd.vim,
src/testdir/test_arglist.vim, src/testdir/test_bufwintabinfo.vim,
src/testdir/test_command_count.vim, src/testdir/test_quickfix.vim,
src/testdir/test_hardcopy.vim, src/testdir/test_ins_complete.vim,
src/testdir/test_packadd.vim, src/testdir/test_signs.vim,
src/testdir/test_autochdir.vim

Patch 8.0.1225
问题: No check for spell region being zero. (geeknik)
解决方案: Check for zero. (closes #2252)
相关文件: src/spellfile.c, src/testdir/test_spell.vim

Patch 8.0.1226
问题: Edit and popup tests failing.
解决方案: Make the tests pass.
相关文件: src/testdir/test_edit.vim, src/testdir/test_popup.vim

Patch 8.0.1227
问题: Undefined left shift in readfile(). (Brian 'geeknik' Carpenter)
解决方案: Add cast to unsigned. (Dominique Pelle, closes #2253)
相关文件: src/fileio.c

Patch 8.0.1228

问题:　　Invalid memory access in GUI test.
解决方案:　Check that the row is not outside of the screen.
相关文件:　src/screen.c

Patch 8.0.1229
问题:　　Condition in vim_str2nr() is always true. (Nikolai Pavlov)
解决方案:　Remove the condition. (Closes #2259)
相关文件:　src/charset.c

Patch 8.0.1230
问题:　　CTRL-A in Visual mode uses character after selection. (Nikolai Pavlov)
解决方案:　Check the length before using a character.
相关文件:　src/charset.c

Patch 8.0.1231
问题:　　Expanding file name drops dash. (stucki)
解决方案:　Use the right position. (Christian Brabandt, closes #2184)
相关文件:　src/ex_docmd.c, src/testdir/test_cmdline.vim

Patch 8.0.1232
问题:　　MS-Windows users are confused about default mappings.
解决方案:　Don't map keys in the console where they don't work. Add a choice in the installer to use MS-Windows key bindings or not. (Christian Brabandt, Ken Takata, closes #2093)
相关文件:　Filelist, nsis/gvim.nsi, nsis/vimrc.ini, src/dosinst.c, runtime/mswin.vim

Patch 8.0.1233
问题:　　Typo in dos installer.
解决方案:　Remove comma.
相关文件:　src/dosinst.c

Patch 8.0.1234
问题:　　MS-Windows: composing characters are not shown properly.
解决方案:　Pass base character and composing characters to the renderer at once. (Ken Takata, closes #2206)
相关文件:　src/gui.c, src/gui_w32.c

Patch 8.0.1235
问题:　　Cannot disable the terminal feature in a huge build. (lindhobe)
解决方案:　Adjust the autoconf check. (Kazunobu Kuriyama, closes #2242)
相关文件:　src/configure.ac, src/auto/configure, src/Makefile

Patch 8.0.1236
问题:　　Mac features are confusing.
解决方案:　Make feature names more consistent, add "osxdarwin". Rename feature flags, cleanup Mac code. (Kazunobu Kuriyama, closes #2178) Also includes a fix for when Ruby throws an exception inside :rubyfile. (ujihisa)
相关文件:　runtime/doc/eval.txt, runtime/doc/os_mac.txt, src/auto/configure, src/config.h.in, src/configure.ac, src/digraph.c, src/edit.c, src/evalfunc.c, src/feature.h, src/fileio.c, src/getchar.c, src/globals.h, src/gui.c, src/gui_mac.c, src/if_python.c,

src/if_python3.c, src/if_ruby.c, src/keymap.h, src/macros.h,
src/main.c, src/mbyte.c, src/message.c, src/misc1.c, src/misc2.c,
src/option.c, src/os_mac.h, src/os_macosx.m, src/os_unix.c,
src/proto.h, src/pty.c, src/structs.h, src/term.c, src/termlib.c,
src/ui.c, src/undo.c, src/version.c, src/vim.h, src/window.c

Patch 8.0.1237
问题: ":set scroll&" often gives an error.
解决方案: Don't use a fixed default value, use half the window height. Add a test. (Ozaki Kiichi, closes #2104)
相关文件: src/Makefile, src/option.c, src/testdir/test_alot.vim, src/testdir/test_scroll_opt.vim

Patch 8.0.1238
问题: Incremental search only shows one match.
解决方案: When 'incsearch' and 'hlsearch' are both set highlight all matches. (haya14busa, itchyny, closes #2198)
相关文件: runtime/doc/options.txt, src/ex_getln.c, src/proto/search.pro, src/search.c, src/testdir/test_search.vim

Patch 8.0.1239
问题: Cannot use a lambda for the skip argument to searchpair().
解决方案: Evaluate a partial, funcref and lambda. (LemonBoy, closes #1454, closes #2265)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/proto/evalfunc.pro, src/eval.c, src/proto/eval.pro, src/search.c, src/testdir/test_search.vim

Patch 8.0.1240
问题: MS-Windows: term_start() does not support environment.
解决方案: Implement the environment argument. (Yasuhiro Matsumoto, closes #2264)
相关文件: src/os_win32.c, src/proto/os_win32.pro, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1241
问题: Popup test is flaky. (James McCoy)
解决方案: Increase the wait time. (Dominique Pelle)
相关文件: src/testdir/test_popup.vim

Patch 8.0.1242
问题: Function argument with only dash is seen as number zero. (Wang Shidong)
解决方案: See a dash as a string. (Christian Brabandt)
相关文件: src/testdir/test_ins_complete.vim, src/Makefile, src/eval.c

Patch 8.0.1243
问题: No test for what 8.0.1227 fixes.
解决方案: Add a test that triggers the problem. (Christian Brabandt)
相关文件: src/testdir/test_normal.vim, src/testdir/test_search.vim

Patch 8.0.1244
问题: Search test does not work correctly on MS-Windows.
解决方案: Put text in a file instead of sending it to the terminal.

(Christian Brabandt)
相关文件: src/testdir/test_search.vim

Patch 8.0.1245
问题: When WaitFor() has a wrong expression it just waits a second, which goes unnoticed. (James McCoy)
解决方案: When WaitFor() times out throw an exception. Fix places where the expression was wrong.
相关文件: src/testdir/shared.vim, src/testdir/test_channel.vim, src/testdir/test_netbeans.vim, src/testdir/test_terminal.vim

Patch 8.0.1246
问题: Popup test has an arbitrary delay.
解决方案: Wait for the ruler to show. (James McCoy)
相关文件: src/testdir/test_popup.vim

Patch 8.0.1247
问题: Not easy to find Debian build info.
解决方案: Add a badge in the README file. (Dominique Pelle)
相关文件: README.md

Patch 8.0.1248 (after 8.0.1247)
问题: Stray + in README file.
解决方案: Remove the +. Add a line break.
相关文件: README.md

Patch 8.0.1249
问题: No error when WaitFor() gets an invalid wrong expression.
解决方案: Do not ignore errors in evaluation of the expression. Fix places where the expression was wrong.
相关文件: src/testdir/shared.vim, src/testdir/test_netbeans.vim

Patch 8.0.1250
问题: 'hlsearch' highlighting not removed after incsearch (lacygoill)
解决方案: Redraw all windows. Start search at the end of the match. Improve how CTRL-G works with incremental search. Add tests. (Christian Brabandt, Hirohito Higashi, haya14busa, closes #2267)
相关文件: runtime/doc/options.txt, src/ex_getln.c, src/testdir/test_search.vim

Patch 8.0.1251 (after 8.0.1249)
问题: Invalid expression passed to WaitFor().
解决方案: Check if the variable exists.
相关文件: src/testdir/test_clientserver.vim

Patch 8.0.1252
问题: Incomplete translations makefile for MinGW/Cygwin.
解决方案: Add missing source files. Make it work with msys2's bash. (Ken Takata)
相关文件: src/po/Make_cyg.mak, src/po/Make_ming.mak, src/po/Make_mvc.mak

Patch 8.0.1253
问题: Still too many old style tests.
解决方案: Convert a few more tests to new style. (Yegappan Lakshmanan,

closes #2272)
相关文件: src/Makefile, src/testdir/Make_all.mak,
src/testdir/Make_amiga.mak, src/testdir/Make_dos.mak,
src/testdir/Make_ming.mak, src/testdir/Make_vms.mms,
src/testdir/main.aap, src/testdir/test12.in,
src/testdir/test12.ok, src/testdir/test40.in,
src/testdir/test40.ok, src/testdir/test45.in,
src/testdir/test45.ok, src/testdir/test83.in,
src/testdir/test83.ok, src/testdir/test_autocmd.vim,
src/testdir/test_fold.vim, src/testdir/test_swap.vim,
src/testdir/test_tagjump.vim

Patch 8.0.1254
问题: Undefined left shift in gethexchrs(). (geeknik)
解决方案: Use unsigned long. (idea by Christian Brabandt, closes #2255)
相关文件: src/regexp.c, src/regexp_nfa.c

Patch 8.0.1255 (after 8.0.1248)
问题: duplicate badge README file.
解决方案: Remove one. (Dominique Pelle)
相关文件: README.md

Patch 8.0.1256
问题: Typo in configure variable vim_cv_tgent. (Matthieu Guillard)
解决方案: Rename the variable. (closes #2281)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1257 (after 8.0.1254)
问题: No test for fix of undefined behavior.
解决方案: Add a test. (closes #2255)
相关文件: src/testdir/test_search.vim

Patch 8.0.1258
问题: 'ttymouse' is set to "sgr" even though it's not supported. (Gary
Johnson)
解决方案: Adjust #ifdef
相关文件: src/term.c

Patch 8.0.1259
问题: Search test can be flaky.
解决方案: Use WaitFor() instead of a delay. Make it possible to pass a
funcref to WaitFor() to avoid the need for global variables.
(James McCoy, closes #2282)
相关文件: src/testdir/shared.vim, src/testdir/test_search.vim

Patch 8.0.1260 (after 8.0.1259)
问题: Using global variables for WaitFor().
解决方案: Use a lambda function instead. Don't check a condition if
WaitFor() already checked it.
相关文件: src/testdir/test_popup.vim, src/testdir/test_terminal.vim,
src/testdir/test_channel.vim, src/testdir/test_clientserver.vim,
src/testdir/test_job_fails.vim, src/testdir/test_quotestar.vim

Patch 8.0.1261
问题: Program in terminal window gets NL instead of CR. (Lifepillar)
解决方案: Check the tty setup more often. (closes #1998)
相关文件: src/terminal.c

Patch 8.0.1262
问题: Terminal redir test is flaky.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.1263
问题: Others can read the swap file if a user is careless with his
            primary group.
解决方案: If the group permission allows for reading but the world
            permissions doesn't, make sure the group is right.
相关文件: src/fileio.c, src/testdir/test_swap.vim, src/Makefile

Patch 8.0.1264
问题: Terminal debugger gets stuck in small window.
解决方案: Add "-quiet" to the gdb command. (Christian Brabandt, closes #2154)
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1265 (after 8.0.1263)
问题: Swap test not skipped when there is one group.
解决方案: Convert list to string for the message.
相关文件: src/testdir/test_swap.vim

Patch 8.0.1266 (after 8.0.1263)
问题: Test_swap_directory was accidentally commented out.
解决方案: Uncomment the test.
相关文件: src/testdir/test_swap.vim

Patch 8.0.1267 (after 8.0.1263)
问题: Test_swap_group may leave file behind.
解决方案: Add a try/finally.
相关文件: src/testdir/test_swap.vim, src/testdir/test_undo.vim

Patch 8.0.1268
问题: PC install instructions are incomplete.
解决方案: Update the instructions. (Ken Takata)
相关文件: src/INSTALLpc.txt

Patch 8.0.1269
问题: Effect of autocommands on marks is not tested.
解决方案: Add a couple of tests. (James McCoy, closes #2271)
相关文件: src/testdir/test_autocmd.vim

Patch 8.0.1270
问题: Mismatching file name with Filelist.
解决方案: Rename color_ramp.vim to xterm_ramp.vim
相关文件: src/testdir/color_ramp.vim, src/testdir/xterm_ramp.vim

Patch 8.0.1271
问题: Still too many old style tests.

解决方案: Convert a few more tests to new style. (Yegappan Lakshmanan,
           closes #2290)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/sautest/autoload/footest.vim, src/testdir/test55.in,
           src/testdir/test55.ok, src/testdir/test_changelist.in,
           src/testdir/test_changelist.ok, src/testdir/test_fold.vim,
           src/testdir/test_ins_complete.vim,
           src/testdir/test_insertcount.in, src/testdir/test_insertcount.ok,
           src/testdir/test_listdict.vim, src/testdir/test_normal.vim,
           src/testdir/test_search.vim, src/testdir/test_search_mbyte.in

Patch 8.0.1272
问题: Warnings for unused variables in tiny build.
解决方案: Add #ifdef. (Dominique Pelle, closes #2288)
相关文件: src/term.c

Patch 8.0.1273 (after 8.0.1271)
问题: Old test file remaining.
解决方案: Delete it.
相关文件: src/testdir/test_search_mbyte.ok

Patch 8.0.1274
问题: setbufline() fails when using folding.
解决方案: Set "curwin" if needed. (Ozaki Kiichi, closes #2293)
相关文件: src/evalfunc.c, src/testdir/test_bufline.vim

Patch 8.0.1275
问题: CmdlineLeave autocmd prevents fold from opening. (Waivek)
解决方案: Save and restore KeyTyped. (closes #2305)
相关文件: src/fileio.c

Patch 8.0.1276
问题: Typed key is lost when the terminal window is closed in exit
           callback. (Gabriel Barta)
解决方案: When the current window changes bail out of the wait loop. (closes
           #2302)
相关文件: src/misc2.c, src/terminal.c

Patch 8.0.1277
问题: Terminal window CR-NL conversions may cause problems.
解决方案: Avoid most conversions, only fetch the current backspace key value
           from the tty. (mostly by Ozaki Kiichi, closes #2278)
相关文件: src/terminal.c

Patch 8.0.1278
问题: GUI window always resizes when adding/removing a scrollbar,
           toolbar, etc.
解决方案: Add the 'k' flag in 'guioptions' to keep the GUI window size and
           change the number of lines/columns instead. (Ychin, closes #703)
相关文件: runtime/doc/options.txt, src/gui.c, src/gui_gtk_x11.c,
           src/gui_w32.c, src/option.h

Patch 8.0.1279
问题: Initializing menus can be slow, especially when there are many

keymaps, color schemes, etc.
解决方案: Do the globbing for runtime files lazily. (Ken Takata)
相关文件: runtime/doc/gui.txt, runtime/menu.vim

Patch 8.0.1280
问题: Python None cannot be converted to a Vim type.
解决方案: Convert it to v:none. (Ken Takata)
相关文件: src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok,
runtime/doc/if_pyth.txt

Patch 8.0.1281
问题: Loading file type detection slows down startup.
解决方案: Move functions to an autoload script.
相关文件: runtime/filetype.vim, runtime/autoload/filetype.vim,
runtime/scripts.vim

Patch 8.0.1282 (after 8.0.1281)
问题: script-local variable defined in the wrong script
解决方案: Move variable to autoload/filetype.vim.
相关文件: runtime/filetype.vim, runtime/autoload/filetype.vim

Patch 8.0.1283
问题: Test 86 fails under ASAN.
解决方案: Fix that an item was added to a dictionary twice.
相关文件: src/if_py_both.h

Patch 8.0.1284
问题: Loading file type detection slows down startup.
解决方案: Store the last pattern of an autocommand event to make appending
quicker.
相关文件: src/fileio.c

Patch 8.0.1285
问题: Distributed autoload files may clash with user files. (Andy
Wokula)
解决方案: Use the "autoload/dist" directory.
相关文件: runtime/filetype.vim, runtime/autoload/filetype.vim,
runtime/autoload/dist/ft.vim, runtime/scripts.vim, Filelist,
src/Makefile, nsis/gvim.nsi

Patch 8.0.1286
问题: Occasional crash when using a channel. (Marek)
解决方案: Decrement reference count later. (closes #2315)
相关文件: src/channel.c

Patch 8.0.1287
问题: The temp file used when updating the viminfo file may have the
wrong permissions if setting the group fails.
解决方案: Check if the group matches and reduce permissions if not.
相关文件: src/ex_cmds.c

Patch 8.0.1288
问题: GUI: cannot drag the statusline of a terminal window.
解决方案: Handle the TERMINAL state. (Hirohito Higashi)

相关文件: src/gui.c

Patch 8.0.1289
问题: Mkview always includes the local directory.
解决方案: Add the "curdir" value in 'viewoptions'. (Eric Roberts, closes #2316)
相关文件: runtime/doc/options.txt, runtime/doc/starting.txt, src/ex_docmd.c, src/option.c

Patch 8.0.1290
问题: seq_cur of undotree() wrong after undo.
解决方案: Get the actual sequence number instead of decrementing the current one. (Ozaki Kiichi, closes #2319)
相关文件: src/undo.c, src/testdir/test_undo.vim

Patch 8.0.1291
问题: C indent wrong when * immediately follows comment. (John Bowler)
解决方案: Do not see "/*" after "*" as a comment start. (closes #2321)
相关文件: src/search.c, src/testdir/test3.in, src/testdir/test3.ok

Patch 8.0.1292
问题: Quick clicks in the WinBar start Visual mode.
解决方案: Use a double click in the WinBar like a normal click.
相关文件: src/ui.c

Patch 8.0.1293
问题: Setting a breakpoint in the terminal debugger sometimes fails.
解决方案: Interrupt the program if needed. Set the interface to async.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim, runtime/doc/terminal.txt

Patch 8.0.1294
问题: GUI: get stuck when splitting a terminal window.
解决方案: Stop blinking when values become zero. (Hirohito Higashi)
相关文件: src/gui.c

Patch 8.0.1295
问题: Cannot automatically get a server name in a terminal.
解决方案: Add the --enable-autoservername flag to configure. (Cimbali, closes #2317)
相关文件: runtime/doc/eval.txt, runtime/doc/various.txt, src/config.h.in, src/configure.ac, src/auto/configure, src/evalfunc.c, src/feature.h, src/main.c, src/version.c, src/Makefile

Patch 8.0.1296 (after 8.0.1294)
问题: Checking the same condition twice. (John Marriott)
解决方案: Check blinkwait.
相关文件: src/gui.c

Patch 8.0.1297
问题: +autoservername does not show enabled on MS-Windows.
解决方案: Always define the flag on MS-Windows. (Ken Takata)
相关文件: src/feature.h

Patch 8.0.1298
问题:　　Missing test file.
解决方案:　Add samples/test000. (Christian Brabandt)
相关文件:　src/testdir/samples/test000, Filelist

Patch 8.0.1299
问题:　　Bracketed paste does not work well in terminal window.
解决方案:　Send translated string to job right away. (Ozaki Kiichi, closes
#2341)
相关文件:　src/terminal.c

Patch 8.0.1300
问题:　　File permissions may end up wrong when writing.
解决方案:　Use fchmod() instead of chmod() when possible. Don't truncate
until we know we can change the file.
相关文件:　src/os_unix.c, src/proto/os_unix.pro, src/configure.ac,
src/auto/configure, src/config.h.in, src/fileio.c

Patch 8.0.1301
问题:　　Generated license file for NSIS has a modeline.
解决方案:　Adjust the pattern for sed. (Ken Takata)
相关文件:　runtime/doc/Makefile

Patch 8.0.1302
问题:　　Still too many old style tests.
解决方案:　Convert a few more tests to new style. (Yegappan Lakshmanan,
closes #2326)
相关文件:　src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_ming.mak,
src/testdir/Make_vms.mms, src/testdir/runtest.vim,
src/testdir/test68.in, src/testdir/test68.ok,
src/testdir/test73.in, src/testdir/test73.ok,
src/testdir/test_close_count.in, src/testdir/test_close_count.ok,
src/testdir/test_close_count.vim,
src/testdir/test_erasebackword.in,
src/testdir/test_erasebackword.ok,
src/testdir/test_erasebackword.vim,
src/testdir/test_find_complete.vim, src/testdir/test_fixeol.in,
src/testdir/test_fixeol.ok, src/testdir/test_fixeol.vim,
src/testdir/test_listchars.in, src/testdir/test_listchars.ok,
src/testdir/test_listchars.vim, src/testdir/test_textformat.vim

Patch 8.0.1303
问题:　　'ttymouse' is not set to "sgr" for Terminal.app and Iterm2.
解决方案:　Recognize Iterm2 by the termresponse.
相关文件:　src/term.c

Patch 8.0.1304
问题:　　CTRL-G/CTRL-T don't work with incsearch and empty pattern.
解决方案:　Use the last search pattern. (Christian Brabandt, closes #2292)
相关文件:　src/ex_getln.c, src/proto/search.pro, src/search.c,
src/testdir/test_search.vim

Patch 8.0.1305
问题:　　writefile() never calls fsync().

解决方案: Follow the 'fsync' option with override to enable or disable.
相关文件: src/fileio.c, src/evalfunc.c, runtime/doc/eval.txt, src/globals.h,
           src/testdir/test_writefile.vim

Patch 8.0.1306
问题: ASAN error stack trace is not useful.
解决方案: Add "asan_symbolize". (James McCoy, closes #2344)
相关文件: .travis.yml

Patch 8.0.1307 (after 8.0.1300)
问题: Compiler warning for ignoring return value of ftruncate(). (Tony
           Mechelynck)
解决方案: Assign returned value to "ignore".
相关文件: src/fileio.c

Patch 8.0.1308
问题: The "Reading from stdin" message may be undesired and there is no
           easy way to skip it.
解决方案: Don't show the message with --not-a-term was used.
相关文件: src/fileio.c

Patch 8.0.1309
问题: Cannot use 'balloonexpr' in a terminal.
解决方案: Add 'balloonevalterm' and add code to handle mouse movements in a
           terminal. Initial implementation for Unix with GUI.
相关文件: src/option.c, src/option.h, src/os_unix.c, src/proto/os_unix.pro,
           src/feature.h, src/misc2.c, src/keymap.h, src/edit.c,
           src/ex_getln.c, src/message.c, src/misc1.c, src/normal.c,
           src/terminal.c, src/getchar.c, src/ex_cmds2.c, src/gui_beval.c,
           src/proto/gui_beval.pro, src/evalfunc.c, src/popupmnu.c,
           src/proto/popupmnu.pro, src/version.c, src/globals.h, src/gui.c,
           runtime/doc/options.txt, src/term.c,
           runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1310
问题: Cproto generates errors because of missing type.
解决方案: Define _Float128 when generating prototypes.
相关文件: src/vim.h

Patch 8.0.1311
问题: No test for strpart().
解决方案: Add a test. (Dominique Pelle, closes #2347)
相关文件: src/testdir/test_functions.vim

Patch 8.0.1312 (after 8.0.1309)
问题: balloon_show() only works in terminal when compiled with the GUI.
解决方案: Add FEAT_BEVAL_GUI and refactor to move common code out of the GUI
           specific file.
相关文件: src/feature.h, src/evalfunc.c, src/gui.c, src/gui_athena.c,
           src/gui_beval.c, src/proto/gui_beval.pro, src/beval.c,
           src/proto/beval.pro, src/gui_motif.c, src/gui_w32.c,
           src/gui_x11.c, src/integration.c, src/workshop.c, src/menu.c,
           src/netbeans.c, src/option.c, src/os_unix.c, src/os_win32.c,
           src/syntax.c, src/version.c, src/gui.h, src/gui_beval.h,

src/vim.h, src/beval.h, src/option.h, src/ex_cmds2.c, src/ui.c,
src/getchar.c, src/normal.c, src/popupmnu.c, src/globals.h,
src/Makefile, src/Make_cyg_ming.mak, src/Make_mvc.mak,
src/Make_vms.mms, Filelist

Patch 8.0.1313 (after 8.0.1312)
问题: Missing dependencies cause parallel make to fail.
解决方案: Update dependencies.
相关文件: src/Makefile

Patch 8.0.1314 (after 8.0.1312)
问题: Build fails on Mac. (chdiza)
解决方案: Add #ifdef around GUI fields.
相关文件: src/beval.h

Patch 8.0.1315 (after 8.0.1312)
问题: Build still fails on Mac. (chdiza)
解决方案: Remove bogus typedef.
相关文件: src/os_macosx.m

Patch 8.0.1316 (after 8.0.1312)
问题: Build still still fails on Mac. (chdiza)
解决方案: Remove another bogus typedef.
相关文件: src/os_mac_conv.c

Patch 8.0.1317
问题: Accessing freed memory in term_wait(). (Dominique Pelle)
解决方案: Check that the buffer still exists.
相关文件: src/terminal.c

Patch 8.0.1318
问题: Terminal balloon only shows one line.
解决方案: Split into several lines in a clever way. Add balloon_split().
Make balloon_show() accept a list in the terminal.
相关文件: src/popupmnu.c, src/proto/popupmnu.pro, src/evalfunc.c,
src/beval.c, src/proto/beval.pro, src/testdir/test_popup.vim,
runtime/doc/eval.txt,
runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1319
问题: Can't build GUI on MS-Windows.
解决方案: Don't define the balloon_split() function in a GUI-only build.
相关文件: src/evalfunc.c, runtime/doc/eval.txt

Patch 8.0.1320
问题: Popup test fails on GUI-only build.
解决方案: Don't test balloon_split() when it's not available.
相关文件: src/testdir/test_popup.vim

Patch 8.0.1321
问题: Can't build huge version with Athena. (Mark Kelly)
解决方案: Move including beval.h to before structs.h. Include beval.pro like
other proto files.
相关文件: src/vim.h, src/beval.h, src/proto.h

Patch 8.0.1322
问题: Textformat test isn't run. (Yegappan Lakshmanan)
解决方案: Add target to the list of tests.
相关文件: src/testdir/Make_all.mak

Patch 8.0.1323
问题: Mouse events in a terminal window may cause endless loop.
解决方案: Adjust position computation. Don't stuff a mouse event when
            coming from normal_cmd().
相关文件: src/normal.c, src/terminal.c

Patch 8.0.1324
问题: Some xterm sends different mouse move codes.
解决方案: Also accept 0x80 as a move event.
相关文件: src/term.c

Patch 8.0.1325
问题: More tests are not run.
解决方案: Add targets to the list of tests. (Yegappan Lakshmanan)
相关文件: src/testdir/Make_all.mak

Patch 8.0.1326
问题: Largefile test fails on CI, glob test on MS-Windows.
解决方案: Remove largefile test from list of all tests. Don't run
            Test_glob() on non-unix systems. More cleanup. (Yegappan
            Lakshmanan, closes #2354)
相关文件: src/testdir/Make_all.mak, src/testdir/test_escaped_glob.vim,
            src/testdir/test_plus_arg_edit.vim

Patch 8.0.1327
问题: New proto file missing from distribution.
解决方案: Add it. (closes #2355)
相关文件: Filelist

Patch 8.0.1328
问题: Trouble when using ":term ++close" with autocmd. (Gabriel Barta)
解决方案: Use aucmd_prepbuf() and aucmd_restbuf() instead of setting curbuf.
            (closes #2339)
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1329
问题: When a flaky test fails it also often fails the second time.
解决方案: Sleep a couple of seconds before the second try.
相关文件: src/testdir/runtest.vim

Patch 8.0.1330
问题: MS-Windows: job in terminal can't get back to Vim.
解决方案: set VIM_SERVERNAME in the environment. (Yasuhiro Matsumoto, closes
            #2360)
相关文件: runtime/doc/terminal.txt, src/os_win32.c, src/proto/os_win32.pro,
            src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1331

问题:　　Possible crash when window can be zero lines high. (Joseph Dornisch)
解决方案:　Only set w_fraction if the window is at least two lines high.
相关文件:　src/window.c

Patch 8.0.1332
问题:　　Highlighting in quickfix window could be better. (Axel Bender)
解决方案:　Use the qfSeparator highlight item. (Yegappan Lakshmanan)
相关文件:　src/quickfix.c

Patch 8.0.1333
问题:　　Some tests are run twice.
解决方案:　Invoked most utf8 tests only from test_alot_utf8. (Yegappan Lakshmanan, closes #2369)
相关文件:　src/testdir/Make_all.mak, src/testdir/test_alot_utf8.vim, src/testdir/test_mksession_utf8.vim

Patch 8.0.1334
问题:　　Splitting a window with a WinBar damages window layout. (Lifepillar)
解决方案:　Take the winbar into account when computing the new window position. Add WINBAR_HEIGHT().
相关文件:　src/vim.h, src/window.c

Patch 8.0.1335
问题:　　writefile() using fsync() may give an error for a device. (Yasuhiro Matsumoto)
解决方案:　Ignore fsync() failing. (closes #2373)
相关文件:　src/evalfunc.c

Patch 8.0.1336
问题:　　Cannot use imactivatefunc() unless compiled with +xim.
解决方案:　Allow using imactivatefunc() when not compiled with +xim. (Yasuhiro Matsumoto, closes #2349)
相关文件:　runtime/doc/options.txt, runtime/doc/mbyte.txt, src/mbyte.c, src/option.c, src/option.h, src/structs.h, src/testdir/test_iminsert.vim, src/Makefile, src/testdir/Make_all.mak, src/vim.h

Patch 8.0.1337 (after 8.0.1336)
问题:　　Typo in #ifdef.
解决方案:　Fix the #if line.
相关文件:　src/mbyte.c

Patch 8.0.1338 (after 8.0.1337)
问题:　　USE_IM_CONTROL is confusing and incomplete.
解决方案:　Just use FEAT_MBYTE. Call 'imactivatefunc' also without GUI.
相关文件:　src/vim.h, src/edit.c, src/ex_getln.c, src/getchar.c, src/gui.c, src/gui_mac.c, src/gui_w32.c, src/mbyte.c, src/normal.c, src/option.c, src/ui.c, src/globals.h, src/option.h

Patch 8.0.1339
问题:　　No test for what 8.0.1335 fixes.
解决方案:　Add a test. (Yasuhiro Matsumoto, closes #2373)

相关文件: src/testdir/test_writefile.vim

Patch 8.0.1340
问题: MS-Windows: cannot build GUI without IME.
解决方案: Define im_get_status() and im_set_active() when IME is not used.
相关文件: src/mbyte.c

Patch 8.0.1341
问题: 'imactivatefunc' test fails on MS-Windows.
解决方案: Skip the text.
相关文件: src/testdir/test_iminsert.vim, runtime/doc/options.txt

Patch 8.0.1342
问题: Cannot build with Motif and multi-byte. (Mohamed Boughaba)
解决方案: Use the right input method status flag. (closes #2374)
相关文件: src/mbyte.c

Patch 8.0.1343
问题: MS-Windows: does not show colored emojis.
解决方案: Implement colored emojis. Improve drawing speed. Make 'taamode'
work. (Taro Muraoka, Yasuhiro Matsumoto, Ken Takata, close #2375)
相关文件: appveyor.yml, runtime/doc/options.txt, src/gui_dwrite.cpp,
src/gui_dwrite.h, src/gui_w32.c, src/proto/gui_w32.pro

Patch 8.0.1344
问题: Using 'imactivatefunc' in the GUI does not work.
解决方案: Do not use 'imactivatefunc' and 'imstatusfunc' in the GUI.
相关文件: runtime/doc/options.txt, src/mbyte.c,
src/testdir/test_iminsert.vim

Patch 8.0.1345
问题: Race condition between stat() and open() for the viminfo temp
file. (Simon Ruderich)
解决方案: use open() with O_EXCL to atomically check if the file exists.
Don't try using a temp file, renaming it will fail anyway.
相关文件: src/ex_cmds.c

Patch 8.0.1346
问题: Crash when passing 50 char string to balloon_split().
解决方案: Fix off-by-one error.
相关文件: src/testdir/test_popup.vim, src/popupmnu.c

Patch 8.0.1347
问题: MS-Windows: build broken by misplaced curly.
解决方案: Move curly after #endif.
相关文件: src/ex_cmds.c

Patch 8.0.1348
问题: Make testclean deletes script file on MS-Windows.
解决方案: Rename file to avoid it starting with an "x".
相关文件: src/testdir/xterm_ramp.vim, src/testdir/color_ramp.vim, Filelist

Patch 8.0.1349
问题: Options test fails when using Motif or GTK GUI.

解决方案: Use "fixed" instead of "fixedsys" for Unix. Don't try "xxx" for guifonteset. Don't set 'termencoding' to anything but "utf-8" for GTK. Give an error if 'termencoding' can't be converted.
相关文件: src/testdir/gen_opt_test.vim, src/option.c

Patch 8.0.1350
问题: Cannot build with +eval and -multi_byte.
解决方案: Adjust #ifdefs. (John Marriott) Always include the multi_byte feature when an input method feature is enabled.
相关文件: src/mbyte.c, src/feature.h

Patch 8.0.1351
问题: Warning for unused variables building with MinGW.
解决方案: Change a few #ifdefs (suggested by John Marriott). Remove superfluous checks of FEAT_MBYTE.
相关文件: src/gui_w32.c

Patch 8.0.1352
问题: Dead URLs in the help go unnoticed.
解决方案: Add a script to check URLs in the help files. (Christian Brabandt)
相关文件: runtime/doc/Makefile, runtime/doc/test_urls.vim, Filelist

Patch 8.0.1353
问题: QuickFixCmdPost is not used consistently.
解决方案: Invoke QuickFixCmdPost consistently after QuickFixCmdPre. (Yegappan Lakshmanan, closes #2377)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1354
问题: Shift-Insert doesn't always work in MS-Windows console.
解决方案: Handle K_NUL differently. (Yasuhiro Matsumoto, closes #2381)
相关文件: src/os_win32.c

Patch 8.0.1355 (after 8.0.1354)
问题: Cursor keys don't work in MS-Windows console.
解决方案: Revert the previous patch. Also delete dead code.
相关文件: src/os_win32.c

Patch 8.0.1356
问题: Using simalt in a GUIEnter autocommand inserts strange characters. (Chih-Long Chang)
解决方案: Ignore K_NOP in Insert mode. (closes #2379)
相关文件: src/edit.c, src/ex_getln.c

Patch 8.0.1357
问题: Startup test fails on OpenBSD. (Edd Barrett)
解决方案: Check for "BSD" instead of "FreeBSD" being defined. (James McCoy, closes #2376, closes #2378)
相关文件: src/vim.h

Patch 8.0.1358
问题: Undercurl is not used in the terminal. (Kovid Goyal)
解决方案: Only fall back to underline when undercurl highlighting is not defined. (closes #1306)

相关文件: src/screen.c

Patch 8.0.1359
问题: Libvterm ANSI colors can not always be recognized from the RGB values. The default color is wrong when t_RB is empty.
解决方案: Add the ANSI color index to VTermColor.
相关文件: src/libvterm/include/vterm.h, src/libvterm/src/pen.c, src/terminal.c

Patch 8.0.1360
问题: The Terminal highlighting doesn't work in a terminal. (Ozaki Kiichi)
解决方案: Use the Terminal highlighting when the cterm index is zero.
相关文件: src/terminal.c

Patch 8.0.1361
问题: Some users don't want to diff with hidden buffers.
解决方案: Add the "hiddenoff" item to 'diffopt'. (Alisue, closes #2394)
相关文件: runtime/doc/options.txt, src/buffer.c, src/diff.c, src/proto/diff.pro, src/testdir/test_diffmode.vim

Patch 8.0.1362
问题: Terminal window colors wrong when using Terminal highlighting.
解决方案: Set ansi_index when setting the default color. Also cache the color index for Terminal. (Ozaki Kiichi, closes #2393)
相关文件: src/libvterm/src/pen.c, src/proto/terminal.pro, src/syntax.c, src/terminal.c

Patch 8.0.1363
问题: Recovering does not work when swap file ends in .stz.
解决方案: Check for all possible swap file names. (Elfling, closes #2395, closes #2396)
相关文件: src/memline.c

Patch 8.0.1364
问题: There is no easy way to get the window position.
解决方案: Add win_screenpos().
相关文件: src/evalfunc.c, src/testdir/test_window_cmd.vim, runtime/doc/eval.txt

Patch 8.0.1365
问题: When one channel test fails others fail as well.
解决方案: Stop the job after a failure. Also add a couple of tests to the list of flaky tests.
相关文件: src/testdir/test_channel.vim, src/testdir/runtest.vim

Patch 8.0.1366
问题: Balloon shows when cursor is in WinBar.
解决方案: Don't show the balloon when row is negative.
相关文件: src/beval.c

Patch 8.0.1367
问题: terminal test hangs, executing abcde. (Stucki)
解决方案: Rename abcde to abxde.

相关文件: src/testdir/test_terminal.vim

Patch 8.0.1368
问题: Cannot drag status line or vertical separator of new terminal window. (UncleBill)
解决方案: Adjust mouse row and column computation. (Yasuhiro Matsumoto, closes #2410)
相关文件: src/terminal.c

Patch 8.0.1369
问题: MS-Windows: drawing underline, curl and strikethrough is slow, mFallbackDC not properly updated.
解决方案: Several performance improvements. (Ken Takata, Taro Muraoka, Yasuhiro Matsumoto, closes #2401)
相关文件: runtime/doc/options.txt, src/gui_dwrite.cpp, src/gui_dwrite.h, src/gui_w32.c

Patch 8.0.1370
问题: Channel test for callback is flaky.
解决方案: Add the test to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.1371
问题: Shift-Insert doesn't always work in MS-Windows console.
解决方案: Handle K_NUL differently if the second character is more than one byte. (Yasuhiro Matsumoto, closes #2381)
相关文件: src/os_win32.c

Patch 8.0.1372
问题: Profile log may be truncated halfway a character.
解决方案: Find the start of the character. (Ozaki Kiichi, closes #2385)
相关文件: src/ex_cmds2.c, src/testdir/test_profile.vim

Patch 8.0.1373
问题: No error when setting 'renderoptions' to an invalid value before starting the GUI.
解决方案: Always check the value. (Ken Takata, closes #2413)
相关文件: src/gui_w32.c, src/option.c

Patch 8.0.1374
问题: CTRL-A does not work with an empty line. (Alex)
解决方案: Decrement the end only once. (Hirohito Higashi, closes #2387)
相关文件: src/ops.c, src/testdir/test_increment.vim

Patch 8.0.1375
问题: Window size wrong after maximizing with WinBar. (Lifepillar)
解决方案: Fix height computations. Redraw window when it is zero height but has a WinBar. (closes #2356)
相关文件: src/window.c, src/screen.c, src/vim.h

Patch 8.0.1376
问题: Cursor in terminal not always updated.
解决方案: Call gui_mch_flush(). (Ken Takata)
相关文件: src/terminal.c

Patch 8.0.1377
问题: Cannot call a dict function in autoloaded dict.
解决方案: Call get_lval() passing the read-only flag.
相关文件: src/userfunc.c, src/eval.c, src/testdir/sautest/autoload/foo.vim,
src/testdir/sautest/autoload/globone.vim,
src/testdir/sautest/autoload/globtwo.vim,
src/testdir/test_escaped_glob.vim, src/Makefile,
src/testdir/test_autoload.vim, src/Makefile,
src/testdir/Make_all.mak

Patch 8.0.1378
问题: Autoload script sources itself when defining function.
解决方案: Pass TFN_NO_AUTOLOAD to trans_function_name(). (Yasuhiro
Matsumoto, closes #2423)
相关文件: src/userfunc.c, src/testdir/test_autoload.vim,
src/testdir/sautest/autoload/sourced.vim

Patch 8.0.1379
问题: Configure check for selinux does not check for header file.
解决方案: Add an AC_CHECK_HEADER(). (Benny Siegert)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1380
问题: When recovering a file with "vim -r swapfile" the hit-enter prompt
is at the top of the window.
解决方案: Invalidate the cursor position.
相关文件: src/term.c

Patch 8.0.1381
问题: ch_readraw() waits for NL if channel mode is NL.
解决方案: Pass a "raw" flag to channel_read_block(). (Yasuhiro Matsumoto)
相关文件: src/channel.c, src/proto/channel.pro,
src/testdir/test_channel.vim, src/testdir/test_channel_pipe.py

Patch 8.0.1382
问题: Get "no write since last change" message if a terminal is open.
(Fritz mehner)
解决方案: Don't consider a buffer changed if it's a terminal window.
相关文件: src/ex_cmds.c, src/undo.c, src/proto/undo.pro

Patch 8.0.1383
问题: Local additions in help skips some files. (joshklod)
解决方案: Check the base file name length equals.
相关文件: src/ex_cmds.c, src/testdir/test_help.vim

Patch 8.0.1384
问题: Not enough quickfix help; confusing winid.
解决方案: Add more examples in the help. When the quickfix window is not
present, return zero for getqflist() with 'winid'. Add more tests
for jumping to quickfix list entries. (Yegappan Lakshmanan, closes
#2427)
相关文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.0.1385
问题: Python 3.5 is getting old.
解决方案: Make Python 3.6 the default. (Ken Takata, closes #2429)
相关文件: runtime/doc/if_pyth.txt, src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/bigvim.bat

Patch 8.0.1386
问题: Cannot select modified buffers with getbufinfo().
解决方案: Add the "bufmodified" flag. (Yegappan Lakshmanan, closes #2431)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_bufwintabinfo.vim

Patch 8.0.1387
问题: Wordcount test is old style.
解决方案: Change into a new style test. (Yegappan Lakshmanan, closes #2434)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_ming.mak, src/testdir/Make_vms.mms, src/testdir/test_wordcount.in, src/testdir/test_wordcount.ok, src/testdir/test_wordcount.vim

Patch 8.0.1388
问题: Char not overwritten with ambiguous width char, if the ambiguous char is single width but we reserve double-width space.
解决方案: First clear the screen cells. (Ozaki Kiichi, closes #2436)
相关文件: src/screen.c

Patch 8.0.1389
问题: getqflist() items are missing if not set, that makes it more difficult to handle the values.
解决方案: When a value is not available return zero or another invalid value. (Yegappan Lakshmanan, closes #2430)
相关文件: runtime/doc/eval.txt, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1390
问题: DirectX scrolling can be slow, vertical positioning is off.
解决方案: Make scroll slightly faster when using "scrlines:1". Fix y position of displayed text. Fix DirectX with non-utf8 encoding. (Ken Takata, closes #2440)
相关文件: src/INSTALLpc.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/gui_dwrite.cpp, src/gui_w32.c

Patch 8.0.1391
问题: Encoding empty string to JSON sometimes gives "null".
解决方案: Handle NULL string as empty string. (closes #2446)
相关文件: src/testdir/test_json.vim, src/json.c

Patch 8.0.1392
问题: Build fails with --with-features=huge --disable-channel.
解决方案: Don't enable the terminal feature when the channel feature is missing. (Dominique Pelle, closes #2453)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1393

问题:　　Too much highlighting with 'hlsearch' and 'incsearch' set.
解决方案:　Do not highlight matches when the pattern matches everything.
相关文件:　src/ex_getln.c

Patch 8.0.1394
问题:　　Cannot intercept a yank command.
解决方案:　Add the TextYankPost autocommand event. (Philippe Vaucher et al.,
closes #2333)
相关文件:　runtime/doc/autocmd.txt, runtime/doc/eval.txt, src/dict.c,
src/eval.c, src/fileio.c, src/ops.c, src/proto/dict.pro,
src/proto/eval.pro, src/proto/fileio.pro,
src/testdir/test_autocmd.vim, src/vim.h

Patch 8.0.1395
问题:　　It is not easy to see if a colorscheme is well written.
解决方案:　Add a script that checks for common mistakes. (Christian Brabandt)
相关文件:　runtime/colors/check_colors.vim, runtime/colors/README.txt

Patch 8.0.1396
问题:　　Memory leak when CTRL-G in search command line fails.
解决方案:　Move restore_last_search_pattern to after "if".
相关文件:　src/ex_getln.c

Patch 8.0.1397
问题:　　Pattern with \& following nothing gives an error.
解决方案:　Emit an empty node when needed.
相关文件:　src/regexp_nfa.c, src/testdir/test_search.vim

Patch 8.0.1398
问题:　　:packadd does not load packages from the "start" directory.
(Alejandro Hernandez)
解决方案:　Make :packadd look in the "start" directory if those packages were
not loaded on startup.
相关文件:　src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 8.0.1399
问题:　　Warnings and errors when building tiny version. (Tony Mechelynck)
解决方案:　Add #ifdefs.
相关文件:　src/ex_getln.c, src/ops.c

Patch 8.0.1400
问题:　　Color scheme check script shows up as color scheme.
解决方案:　Move it to the "tools" subdirectory. (closes #2457)
相关文件:　Filelist, runtime/colors/check_colors.vim,
runtime/colors/tools/check_colors.vim, runtime/colors/README.txt

Patch 8.0.1401
问题:　　Cannot build with GTK but without XIM. (Guido)
解决方案:　Adjust #ifdef. (closes #2461)
相关文件:　src/gui.c

Patch 8.0.1402
问题:　　Crash with nasty autocommand. (gy741, Dominique Pelle)
解决方案:　Check that the new current buffer isn't wiped out. (closes #2447)

相关文件: src/buffer.c, src/testdir/test_autocmd.vim

Patch 8.0.1403
问题: Using freed buffer in grep command. (gy741, Dominique Pelle)
解决方案: Lock the dummy buffer to avoid autocommands wiping it out.
相关文件: src/quickfix.c, src/testdir/test_autocmd.vim

Patch 8.0.1404
问题: Invalid memory access on exit when autocommands wipe out a buffer.
(gy741, Dominique Pelle)
解决方案: Check if the buffer is still valid. (closes #2449)
相关文件: src/main.c

Patch 8.0.1405
问题: Duplicated code for getting a typed character. CursorHold is
called too often in the GUI. (lilydjwg)
解决方案: Refactor code to move code up from mch_inchar(). Don't fire
CursorHold if feedkeys() was used. (closes #2451)
相关文件: src/gui.c, src/proto/gui.pro, src/main.c, src/ui.c,
src/proto/ui.pro, src/os_unix.c

Patch 8.0.1406
问题: Difficult to track changes to a quickfix list.
解决方案: Add a "changedtick" value. (Yegappan Lakshmanan, closes #2460)
相关文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.0.1407
问题: GUI: CursorHold may trigger before 'updatetime' when using timers.
解决方案: Check that 'updatetime' has passed.
相关文件: src/gui.c

Patch 8.0.1408
问题: Crash in setqflist().
解决方案: Check for string to be NULL. (Dominique Pelle, closes #2464)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1409
问题: Buffer overflow in :tags command.
解决方案: Use vim_snprintf(). (Dominique Pelle, closes #2471, closes #2475)
Add a test.
相关文件: src/testdir/test_taglist.vim, src/tag.c

Patch 8.0.1410
问题: Hang when using count() with an empty string.
解决方案: Return zero for an empty string. (Dominique Pelle, closes #2465)
相关文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_functions.vim

Patch 8.0.1411
问题: Reading invalid memory with CTRL-W :.
解决方案: Correct the command characters. (closes #2469)
相关文件: src/normal.c, src/testdir/test_window_cmd.vim, src/ops.c

Patch 8.0.1412
问题: Using free memory using setloclist(). (Dominique Pelle)
解决方案: Mark location list context as still in use when needed. (Yegappan Lakshmanan, closes #2462)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1413
问题: Accessing freed memory in :cbuffer.
解决方案: Get quickfix list after executing autocmds. (closes #2470)
相关文件: src/quickfix.c, src/testdir/test_autocmd.vim

Patch 8.0.1414
问题: Accessing freed memory in :lfile.
解决方案: Get the current window after executing autocommands. (Yegappan Lakshmanan, closes #2473)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1415
问题: Warning for unused function without timers feature.
解决方案: Add #ifdef. (John Marriott)
相关文件: src/gui.c

Patch 8.0.1416
问题: Crash when searching for a sentence.
解决方案: Return NUL when getting character at MAXCOL. (closes #2468)
相关文件: src/misc1.c, src/misc2.c, src/testdir/test_search.vim, src/ex_docmd.c

Patch 8.0.1417
问题: Test doesn't search for a sentence. Still fails when searching for start of sentence. (Dominique Pelle)
解决方案: Add paren. Check for MAXCOL in dec().
相关文件: src/testdir/test_search.vim, src/misc2.c

Patch 8.0.1418
问题: No test for expanding backticks.
解决方案: Add a test. (Dominique Pelle, closes #2479)
相关文件: src/testdir/test_normal.vim

Patch 8.0.1419
问题: Cursor column is not updated after]s. (Gary Johnson)
解决方案: Set the curswant flag.
相关文件: src/testdir/test_spell.vim, src/normal.c, src/evalfunc.c

Patch 8.0.1420
问题: Accessing freed memory in vimgrep.
解决方案: Check that the quickfix list is still valid. (Yegappan Lakshmanan, closes #2474)
相关文件: src/quickfix.c, src/testdir/test_autocmd.vim, src/testdir/test_quickfix.vim

Patch 8.0.1421
问题: Accessing invalid memory with overlong byte sequence.
解决方案: Check for NUL character. (test by Dominique Pelle, closes #2485)

相关文件: src/misc2.c, src/testdir/test_functions.vim

Patch 8.0.1422
问题: No fallback to underline when undercurl is not set. (Ben Jackson)
解决方案: Check for the value to be empty instead of NULL. (closes #2424)
相关文件: src/screen.c

Patch 8.0.1423
问题: Error in return not caught by try/catch.
解决方案: Call update_force_abort(). (Yasuhiro Matsumoto, closes #2483)
相关文件: src/testdir/test_eval.in, src/testdir/test_eval_stuff.vim,
src/Makefile, src/testdir/Make_all.mak, src/userfunc.c

Patch 8.0.1424
问题: The timer_pause test is flaky on Travis.
解决方案: Accept a longer sleep time on Mac.
相关文件: src/testdir/test_timers.vim

Patch 8.0.1425
问题: execute() does not work in completion of user command. (thinca)
解决方案: Switch off redir_off and restore it. (Ozaki Kiichi, closes #2492)
相关文件: src/evalfunc.c, src/testdir/test_usercommands.vim

Patch 8.0.1426
问题: "gf" and <cfile> don't accept ? and & in URL. (Dmitrii Tcyganok)
解决方案: Check for a URL and allow for extra characters. (closes #2493)
相关文件: src/window.c, src/testdir/test_gf.vim

Patch 8.0.1427
问题: The :leftabove modifier doesn't work for :copen.
解决方案: Respect the split modifier. (Yegappan Lakshmanan, closes #2496)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1428
问题: Compiler warning on 64 bit MS-Windows system.
解决方案: Change type from "int" to "size_t". (Mike Williams)
相关文件: src/ex_getln.c

Patch 8.0.1429
问题: Crash when calling term_start() with empty argument.
解决方案: Check for invalid argument. (Yasuhiro Matsumoto, closes #2503)
Fix memory leak.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1430 (after 8.0.1429)
问题: Crash when term_start() fails.
解决方案: Initialize winpty_err.
相关文件: src/terminal.c

Patch 8.0.1431
问题: MS-Windows: vimtutor fails if %TMP% has special chars.
解决方案: Add quotes. (Tamce, closes #2561)
相关文件: vimtutor.bat

Patch 8.0.1432
问题: After ":copen" can't get the window-ID of the quickfix window.
(FalacerSelene)
解决方案: Make it work without a quickfix list. Add a test. (Yegappan
Lakshmanan, closes #2541)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1433
问题: Illegal memory access after undo. (Dominique Pelle)
解决方案: Avoid the column becomes negative. (Christian Brabandt,
closes #2533)
相关文件: src/mbyte.c, src/testdir/test_undo.vim

Patch 8.0.1434
问题: GTK: :promtfind does not put focus on text input. (Adam Novak)
解决方案: When re-opening the dialog put focus on the text input. (Kazunobu
Kuriyama, closes #2563)
相关文件: src/gui_gtk.c

Patch 8.0.1435
问题: Memory leak in test_arabic.
解决方案: Free the from and to parts. (Christian Brabandt, closes #2569)
相关文件: src/buffer.c, src/digraph.c, src/proto/digraph.pro

Patch 8.0.1436
问题: Not enough information about what Python version may work.
解决方案: Add "python_compiled", "python3_compiled", "python_dynamic" and
"python3_dynamic" values for has().
相关文件: src/evalfunc.c, runtime/doc/eval.txt

Patch 8.0.1437
问题: Pkg-config doesn't work with cross compiling.
解决方案: Use AC_PATH_TOOL() instead of AC_PATH_PROG(). (James McCoy,
closes #2513)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1438
问题: Filetype detection test not updated for change.
解决方案: Update the test.
相关文件: src/testdir/test_filetype.vim

Patch 8.0.1439
问题: If cscope fails a search Vim may hang.
解决方案: Bail out when a search error is encountered. (Safouane Baroudi,
closes #2598)
相关文件: src/if_cscope.c

Patch 8.0.1440
问题: Terminal window: some vterm responses are delayed.
解决方案: After writing input. check if there is output to read. (Ozaki
Kiichi, closes #2594)
相关文件: src/terminal.c, src/testdir/test_search.vim,
src/testdir/test_terminal.vim

Patch 8.0.1441
问题: Using ":undo 0" leaves undo in wrong state.
解决方案: Instead of searching for state 1 and go above, just use the start. (Ozaki Kiichi, closes #2595)
相关文件: src/undo.c, src/testdir/test_undo.vim

Patch 8.0.1442 (after 8.0.1439)
问题: Using pointer before it is set.
解决方案: Search in whole buffer instead of next token.
相关文件: src/if_cscope.c

Patch 8.0.1443 (after 8.0.1441)
问题: Compiler complains about uninitialized variable. (Tony Mechelynck)
解决方案: Assign a value to the variable.
相关文件: src/undo.c

Patch 8.0.1444
问题: Missing -D_FILE_OFFSET_BITS=64 may cause problems if a library is compiled with it.
解决方案: Include -D_FILE_OFFSET_BITS if some CFLAGS has it. (James McCoy, closes #2600)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1445
问题: Cannot act on edits in the command line.
解决方案: Add the CmdlineChanged autocommand event. (xtal8, closes #2603, closes #2524)
相关文件: runtime/doc/autocmd.txt, src/ex_getln.c, src/fileio.c, src/testdir/test_autocmd.vim, src/vim.h

Patch 8.0.1446
问题: Accessing freed memory after window command in auto command. (gy741)
解决方案: Adjust the pointer in the parent frame. (Christian Brabandt, closes #2467)
相关文件: src/window.c, src/testdir/test_window_cmd.vim

Patch 8.0.1447
问题: Still too many old style tests.
解决方案: Turn a few tests into new style. (Yegappan Lakshmanan, closes #2509)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms, src/testdir/main.aap, src/testdir/test15.in, src/testdir/test15.ok, src/testdir/test36.in, src/testdir/test36.ok, src/testdir/test50.in, src/testdir/test50.ok, src/testdir/test_regex_char_classes.vim, src/testdir/test_shortpathname.vim, src/testdir/test_textformat.vim

Patch 8.0.1448
问题: Segmentation fault when Ruby throws an exception inside :rubyfile command.
解决方案: Use rb_protect() instead of rb_load_protect(). (ujihisa, closes #2147, greywolf, closes #2512, #2511)

相关文件: src/if_ruby.c, src/testdir/test_ruby.vim

Patch 8.0.1449
问题: Slow redrawing with DirectX.
解决方案: Avoid calling gui_mch_flush() unnecessarily, especially when updating the cursor. (Ken Takata, closes #2560)
相关文件: runtime/doc/options.txt, src/channel.c, src/edit.c, src/getchar.c, src/gui.c, src/gui_dwrite.cpp, src/gui_dwrite.h, src/gui_w32.c, src/macros.h, src/main.c, src/message.c, src/netbeans.c, src/proto/gui.pro, src/proto/term.pro, src/screen.c, src/search.c, src/term.c, src/ui.c

Patch 8.0.1450
问题: Endless loop when gui_mch_stop_blink() is called while blink_state is BLINK_OFF. (zdohnal)
解决方案: Avoid calling gui_update_cursor() recursively.
相关文件: src/gui.c, src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro, src/gui_mac.c, src/proto/gui_mac.pro, src/gui_photon.c, src/proto/gui_photon.pro, src/gui_w32.c, src/proto/gui_w32.pro, src/gui_x11.c, src/proto/gui_x11.pro

Patch 8.0.1451
问题: It is difficult to set the python home directory properly for Python 2.7 and 3.5 since both use $PYTHONHOME.
解决方案: Add the 'pythonhome' and 'pythonthreehome' options. (Kazuki Sakamoto, closes #1266)
相关文件: runtime/doc/options.txt, runtime/doc/quickref.txt, runtime/optwin.vim, src/if_python.c, src/if_python3.c, src/option.c, src/option.h

Patch 8.0.1452
问题: Terminal test fails on some systems. (jonathonf)
解决方案: Use "cat" instead of Python to produce the input. Add a delay. (closes #2607)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1453
问题: Terminal test fails on some slow terminals.
解决方案: Increase timeout to 10 seconds.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1454
问题: When in silent mode too much output is buffered.
解决方案: Use line buffering instead of fully buffered. (Brian M. Carlson, closes #2537)
相关文件: src/main.c

Patch 8.0.1455
问题: If $SHELL contains a space then the default value of 'shell' is incorrect. (Matthew Horan)
解决方案: Escape spaces in $SHELL. (Christian Brabandt, closes #459)
相关文件: src/option.c, runtime/doc/options.txt, src/testdir/test_startup.vim

Patch 8.0.1456
问题:　　Timer test on travis Mac is still flaky.
解决方案:　Increase time range a bit more.
相关文件:　src/testdir/test_timers.vim

Patch 8.0.1457
问题:　　Clojure now supports a shebang line.
解决方案:　Detect clojure script from the shebang line. (David Burgin,
closes #2570)
相关文件:　runtime/scripts.vim

Patch 8.0.1458
问题:　　Filetype detection test does not check all scripts.
解决方案:　Add most scripts to the test
相关文件:　src/testdir/test_filetype.vim

Patch 8.0.1459
问题:　　Cannot handle change of directory.
解决方案:　Add the DirChanged autocommand event. (Andy Massimino,
closes #888) Avoid changing directory for 'autochdir' too often.
相关文件:　runtime/doc/autocmd.txt, src/buffer.c, src/ex_docmd.c,
src/fileio.c, src/main.c, src/vim.h, src/proto/misc2.pro,
src/gui_mac.c, src/netbeans.c, src/os_win32.c,
src/testdir/test_autocmd.vim

Patch 8.0.1460 (after 8.0.1459)
问题:　　Missing file in patch.
解决方案:　Add changes to missing file.
相关文件:　src/misc2.c

Patch 8.0.1461 (after 8.0.1459)
问题:　　Missing another file in patch.
解决方案:　Add changes to missing file.
相关文件:　src/ex_cmds.c

Patch 8.0.1462 (after 8.0.1459)
问题:　　Missing yet another file in patch.
解决方案:　Add changes to missing file.
相关文件:　src/gui.c

Patch 8.0.1463
问题:　　Test fails without 'autochdir' option.
解决方案:　Skip test if 'autochdir' is not supported.
相关文件:　src/testdir/test_autocmd.vim

Patch 8.0.1464
问题:　　Completing directory after :find does not add slash.
解决方案:　Adjust the flags for globpath(). (Genki Sky)
相关文件:　src/misc1.c, src/testdir/test_find_complete.vim

Patch 8.0.1465
问题:　　Python2 and python3 detection not tested. (Matej Cepl)
解决方案:　Add test for detecting python2 and python3. Also detect a script
using "js" as javascript.

相关文件: runtime/scripts.vim, src/testdir/test_filetype.vim

Patch 8.0.1466
问题: Older GTK versions don't have gtk_entry_get_text_length().
解决方案: Add a function with #ifdefs to take care of GTK version differences. (Kazunobu Kuriyama, closes #2605)
相关文件: src/gui_gtk.c

Patch 8.0.1467
问题: Libvterm doesn't handle illegal byte sequence correctly.
解决方案: After the invalid code check if there is space to store another character. Allocate one more character. (zhykzhykzhyk, closes #2614, closes #2613)
相关文件: src/libvterm/src/encoding.c, src/libvterm/src/state.c

Patch 8.0.1468
问题: Illegal memory access in del_bytes().
解决方案: Check for negative byte count. (Christian Brabandt, closes #2466)
相关文件: src/message.c, src/misc1.c

Patch 8.0.1469
问题: When package path is a symlink adding it to 'runtimepath' happens at the end.
解决方案: Do not resolve symlinks before locating the position in 'runtimepath'. (Ozaki Kiichi, closes #2604)
相关文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 8.0.1470
问题: Integer overflow when using regexp pattern. (geeknik)
解决方案: Use a long instead of int. (Christian Brabandt, closes #2251)
相关文件: src/regexp_nfa.c

Patch 8.0.1471 (after 8.0.1401)
问题: On MS-Windows CursorIM highlighting no longer works.
解决方案: Adjust #if statements. (Ken Takata)
相关文件: src/gui.c

Patch 8.0.1472
问题: MS-Windows: nsis installer is a bit slow.
解决方案: Use ReserveFile for vimrc.ini. (Ken Takata, closes #2522)
相关文件: nsis/gvim.nsi

Patch 8.0.1473
问题: MS-Windows: D&D fails between 32 and 64 bit apps.
解决方案: Add the /HIGHENTROPYVA:NO linker option. (Ken Takata, closes #2504)
相关文件: src/Make_mvc.mak

Patch 8.0.1474
问题: Visual C 2017 has multiple MSVCVER numbers.
解决方案: Assume the 2017 version if MSVCVER >= 1910. (Leonardo Valeri Manera, closes #2619)
相关文件: src/Make_mvc.mak

Patch 8.0.1475

问题:　　Invalid memory access in read_redo(). (gy741)
解决方案:　Convert the replacement character back from a negative number to
　　CR or NL. (hint by Dominique Pelle, closes #2616)
相关文件:　src/testdir/test_undo.vim, src/normal.c, src/vim.h, src/ops.c

Patch 8.0.1476
问题:　　Screen isn't always updated right away.
解决方案:　Adjust #ifdef: Call out_flush() when not running the GUI.
相关文件:　src/screen.c

Patch 8.0.1477
问题:　　Redraw flicker when moving the mouse outside of terminal window.
解决方案:　Instead of updating the cursor color and shape every time leaving
　　and entering a terminal window, only update when different from
　　the previously used cursor.
相关文件:　src/terminal.c

Patch 8.0.1478
问题:　　Unnecessary condition for "len" being zero.
解决方案:　Remove the condition. (Dominique Pelle)
相关文件:　src/regexp_nfa.c

Patch 8.0.1479
问题:　　Insert mode completion state is confusing.
解决方案:　Move ctrl_x_mode into edit.c. Add CTRL_X_NORMAL for zero.
相关文件:　src/edit.c, src/globals.h, src/proto/edit.pro, src/search.c,
　　src/getchar.c

Patch 8.0.1480 (after 8.0.1479)
问题:　　Patch missing change.
解决方案:　Add missing change.
相关文件:　src/evalfunc.c

Patch 8.0.1481
问题:　　Clearing a pointer takes two lines.
解决方案:　Add vim_clear() to free and clear the pointer.
相关文件:　src/misc2.c, src/proto/misc2.pro, src/edit.c

Patch 8.0.1482
问题:　　Using feedkeys() does not work to test Insert mode completion.
　　(Lifepillar)
解决方案:　Do not check for typed keys when executing :normal or feedkeys().
　　Fix thesaurus completion not working when 'complete' is empty.
相关文件:　src/edit.c, src/testdir/test_ins_complete.vim,
　　src/testdir/test_popup.vim, src/testdir/test_edit.vim

Patch 8.0.1483
问题:　　searchpair() might return an invalid value on timeout.
解决方案:　When the second search times out, do not accept a match from the
　　first search. (Daniel Hahler, closes #2552)
相关文件:　src/search.c

Patch 8.0.1484
问题:　　Redundant conditions.

解决方案: Remove them. (Dominique Pelle)
相关文件: src/terminal.c

Patch 8.0.1485
问题: Weird autocmd may cause arglist to be changed recursively.
解决方案: Prevent recursively changing the argument list. (Christian Brabandt, closes #2472)
相关文件: src/ex_docmd.c, src/globals.h

Patch 8.0.1486
问题: Accessing invalid memory with "it". (Dominique Pelle)
解决方案: Avoid going over the end of the line. (Christian Brabandt, closes #2532)
相关文件: src/search.c, src/testdir/test_textobjects.vim

Patch 8.0.1487 (after 8.0.1486)
问题: Test 14 fails.
解决方案: Fix of-by-one error.
相关文件: src/search.c

Patch 8.0.1488 (after 8.0.1218)
问题: Emacs tags no longer work. (zdohnal)
解决方案: Do not skip over end of line.
相关文件: src/tag.c, src/testdir/test_tagjump.vim

Patch 8.0.1489
问题: There is no easy way to get the global directory, esp. if some windows have a local directory.
解决方案: Make getcwd(-1) return the global directory. (Andy Massimino, closes #2606)
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_getcwd.vim

Patch 8.0.1490
问题: Number of spell regions is spread out through the code.
解决方案: Define MAXREGIONS.
相关文件: src/spell.h, src/spellfile.c

Patch 8.0.1491
问题: The minimum width of the popup menu is hard coded.
解决方案: Add the 'pumwidth' option. (Christian Brabandt, James McCoy, closes #2314)
相关文件: runtime/doc/options.txt, src/option.c, src/option.h, src/popupmnu.c

Patch 8.0.1492
问题: Memory leak in balloon_split().
解决方案: Free the balloon lines. Free the balloon when exiting.
相关文件: src/misc2.c, src/evalfunc.c

Patch 8.0.1493
问题: Completion items cannot be annotated.
解决方案: Add a "user_data" entry to the completion item. (Ben Jackson, closes #2608, closes #2508)
相关文件: runtime/doc/insert.txt, src/edit.c, src/structs.h,

src/testdir/test_ins_complete.vim

Patch 8.0.1494
问题: No autocmd triggered in Insert mode with visible popup menu.
解决方案: Add TextChangedP. (Prabir Shrestha, Christian Brabandt,
closes #2372, closes #1691)
Fix that the TextChanged autocommands are not always triggered
when sourcing a script.
相关文件: runtime/doc/autocmd.txt, src/edit.c, src/globals.h, src/structs.h,
src/fileio.c, src/proto/fileio.pro, src/vim.h, src/main.c,
src/testdir/test_autocmd.vim

Patch 8.0.1495
问题: Having 'pumwidth' default to zero has no merit.
解决方案: Make the default 15, as the actual default value.
相关文件: src/popupmnu.c, src/option.c

Patch 8.0.1496
问题: Clearing a pointer takes two lines.
解决方案: Add VIM_CLEAR() and replace vim_clear(). (Hirohito Higashi,
closes #2629)
相关文件: src/buffer.c, src/channel.c, src/crypt.c, src/edit.c, src/eval.c,
src/evalfunc.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c,
src/ex_getln.c, src/fileio.c, src/gui_gtk_x11.c, src/gui_photon.c,
src/gui_w32.c, src/gui_x11.c, src/hardcopy.c, src/if_cscope.c,
src/macros.h, src/main.c, src/mark.c, src/mbyte.c, src/memfile.c,
src/memline.c, src/menu.c, src/message.c, src/misc1.c,
src/misc2.c, src/netbeans.c, src/normal.c, src/ops.c,
src/option.c, src/os_amiga.c, src/os_mac_conv.c, src/os_mswin.c,
src/os_unix.c, src/os_win32.c, src/popupmnu.c,
src/proto/misc2.pro, src/quickfix.c, src/regexp.c,
src/regexp_nfa.c, src/screen.c, src/search.c, src/spell.c,
src/spellfile.c, src/syntax.c, src/tag.c, src/term.c,
src/terminal.c, src/ui.c, src/undo.c, src/userfunc.c, src/window.c

Patch 8.0.1497
问题: Getting the jump list requires parsing the output of :jumps.
解决方案: Add getjumplist(). (Yegappan Lakshmanan, closes #2609)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/Makefile,
src/evalfunc.c, src/list.c, src/proto/list.pro,
src/testdir/Make_all.mak, src/testdir/test_jumplist.vim

Patch 8.0.1498 (after 8.0.1497)
问题: getjumplist() returns duplicate entries. (lacygoill)
解决方案: Call cleanup_jumplist(). (Yegappan Lakshmanan)
相关文件: src/evalfunc.c, src/mark.c, src/proto/mark.pro,
src/testdir/test_jumplist.vim

Patch 8.0.1499
问题: Out-of-memory situation not correctly handled. (Coverity)
解决方案: Check for NULL value.
相关文件: src/terminal.c

Patch 8.0.1500

问题: Possible NULL pointer dereference. (Coverity)
解决方案: Check for the pointer not being NULL.
相关文件: src/quickfix.c

Patch 8.0.1501
问题: Out-of-memory situation not correctly handled. (Coverity)
解决方案: Check for NULL value.
相关文件: src/ops.c

Patch 8.0.1502
问题: In out-of-memory situation character is not restored. (Coverity)
解决方案: Restore the character in all situations.
相关文件: src/ex_getln.c

Patch 8.0.1503
问题: Access memory beyond end of string. (Coverity)
解决方案: Keep allocated memory in separate pointer. Avoid outputting the
            NUL character.
相关文件: src/hardcopy.c

Patch 8.0.1504
问题: Win32: the screen may be cleared on startup.
解决方案: Only call shell_resized() when the size actually changed. (Ken
            Takata, closes #2527)
相关文件: src/os_win32.c

Patch 8.0.1505
问题: Debugger can't break on a condition. (Charles Campbell)
解决方案: Add ":breakadd expr". (Christian Brabandt, closes #859)
相关文件: runtime/doc/repeat.txt, src/eval.c, src/evalfunc.c,
            src/userfunc.c, src/ex_cmds2.c, src/ex_docmd.c,
            src/proto/eval.pro, src/proto/ex_cmds2.pro, src/structs.h

Patch 8.0.1506
问题: New version of HP NonStop (Tandem) doesn't like the default header
            for setenv().
解决方案: Put a #ifdef around the setenv() entry. (Joachim Schmitz)
相关文件: src/osdef2.h.in

Patch 8.0.1507
问题: Timer test is a bit flaky.
解决方案: Add it to the list of flaky tests.
相关文件: src/testdir/runtest.vim

Patch 8.0.1508
问题: The :drop command is not always available.
解决方案: Include :drop in all builds. (Yasuhiro Matsumoto, closes #2639)
相关文件: runtime/doc/windows.txt, src/ex_cmds.c, src/ex_cmds2.c,
            src/ex_docmd.c, src/testdir/test_normal.vim,
            src/testdir/test_tabpage.vim

Patch 8.0.1509 (after 8.0.1508)
问题: Test for failing drag-n-drop command no longer fails.
解决方案: Check for the "dnd" feature.

相关文件: src/testdir/test_normal.vim

Patch 8.0.1510
问题: Cannot test if a command causes a beep.
解决方案: Add assert_beeps().
相关文件: runtime/doc/eval.txt, src/evalfunc.c, src/eval.c,
src/proto/eval.pro, src/misc1.c, src/globals.h,
src/testdir/test_normal.vim, src/testdir/test_assert.vim

Patch 8.0.1511 (after 8.0.1505)
问题: Some code for the debugger watch expression is clumsy.
解决方案: Clean up the code.
相关文件: src/ex_cmds2.c, src/eval.c, src/proto/eval.pro

Patch 8.0.1512
问题: Warning for possibly using NULL pointer. (Coverity)
解决方案: Skip using the pointer if it's NULL.
相关文件: src/ex_cmds.c

Patch 8.0.1513
问题: The jumplist is not always properly cleaned up.
解决方案: Call fname2fnum() before cleanup_jumplist(). (Yegappan Lakshmanan)
相关文件: src/evalfunc.c, src/mark.c, src/proto/mark.pro

Patch 8.0.1514
问题: Getting the list of changes is not easy.
解决方案: Add the getchangelist() function. (Yegappan Lakshmanan,
closes #2634)
相关文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
src/testdir/Make_all.mak, src/testdir/test_changelist.vim,
src/Makefile

Patch 8.0.1515
问题: BufWinEnter event fired when opening hidden terminal.
解决方案: Do not fire BufWinEnter when the terminal is hidden and does not
open a window. (Kenta Sato, closes #2636)
相关文件: src/terminal.c

Patch 8.0.1516
问题: Errors for job options are not very specific.
解决方案: Add more specific error messages.
相关文件: src/channel.c, src/globals.h

Patch 8.0.1517
问题: Invalid memory access with pattern using look-behind match.
(Dominique Pelle)
解决方案: Get a pointer to the right line.
相关文件: src/regexp.c

Patch 8.0.1518
问题: Error messages suppressed after ":silent! try". (Ben Reilly)
解决方案: Restore emsg_silent before executing :try. (closes #2531)
相关文件: src/ex_docmd.c, src/testdir/test_eval_stuff.vim

Patch 8.0.1519
问题: getchangelist() does not use argument as bufname().
解决方案: Use get_buf_tv(). (Yegappan Lakshmanan, closes #2641)
相关文件: src/evalfunc.c, src/testdir/test_changelist.vim

Patch 8.0.1520
问题: Cursor is in the wrong line when using a WinBar in a Terminal
            window.
解决方案: Adjust the row number. (Christian Brabandt, closes #2362)
相关文件: src/screen.c, src/terminal.c

Patch 8.0.1521
问题: Shift-Tab does not work in a terminal window.
解决方案: Recognize Shift-Tab key press. (Jsees Luehrs, closes #2644)
相关文件: src/terminal.c

Patch 8.0.1522 (after 8.0.1491)
问题: Popup menu is positioned in the wrong place. (Davit Samvelyan,
            Boris Staletic)
解决方案: Correct computation of the column and the conditions for that.
            (Hirohito Higashi, closes #2640)
相关文件: src/popupmnu.c

Patch 8.0.1523
问题: Cannot write and read terminal screendumps.
解决方案: Add term_dumpwrite(), term_dumpread() and term_dumpdiff().
            Also add assert_equalfile().
相关文件: src/terminal.c, src/proto/terminal.pro, src/evalfunc.c,
            src/normal.c, src/eval.c, src/proto/eval.pro,
            runtime/doc/eval.txt, src/testdir/test_assert.vim

Patch 8.0.1524 (after 8.0.1523)
问题: Compiler warnings for uninitialized variables. (Tony Mechelynck)
解决方案: Initialize variables.
相关文件: src/terminal.c

Patch 8.0.1525
问题: Using :wqa exits even if a job runs in a terminal window. (Jason
            Felice)
解决方案: Check if a terminal has a running job. (closes #2654)
相关文件: src/ex_cmds2.c, src/buffer.c, src/proto/buffer.pro, src/ex_cmds.c,
            src/testdir/test_terminal.vim

Patch 8.0.1526
问题: No test using a screen dump yet.
解决方案: Add a test for C syntax highlighting. Add helper functions.
相关文件: src/terminal.c, src/testdir/test_syntax.vim,
            src/testdir/shared.vim, src/testdir/screendump.vim,
            src/testdir/dumps/Test_syntax_c_01.dump, runtime/doc/terminal.txt,
            src/testdir/README.txt

Patch 8.0.1527 (after 8.0.1526)
问题: Screen dump test fails on MS-Windows.
解决方案: Skip dump test on MS-Windows for now.

相关文件: src/testdir/test_syntax.vim

Patch 8.0.1528
问题: Dead code found.
解决方案: Remove the useless lines. (CodeAi, closes #2656)
相关文件: src/screen.c, src/spell.c, src/syntax.c, src/window.c

Patch 8.0.1529
问题: Assert_equalfile() does not close file descriptors. (Coverity)
解决方案: Close the file descriptors.
相关文件: src/eval.c

Patch 8.0.1530
问题: Dump test fails when using a shadow directory.
解决方案: Add the directory to the list of symlinks to make (Elimar
Riesebieter)
相关文件: src/Makefile

Patch 8.0.1531
问题: Cannot use 24 bit colors in MS-Windows console.
解决方案: Add support for vcon. (Nobuhiro Takasaki, Ken Takata,
fixes #1270, fixes #2060)
相关文件: runtime/doc/options.txt, src/misc1.c, src/option.c,
src/evalfunc.c, src/os_win32.c, src/proto/os_win32.pro,
src/feature.h, src/proto/term.pro, src/screen.c, src/syntax.c,
src/term.c, src/testdir/gen_opt_test.vim, src/version.c

Patch 8.0.1532
问题: Compiler warnings without termguicolors feature.
解决方案: Add #ifdef. (John Marriott) Cleanup the code a bit.
相关文件: src/term.c

Patch 8.0.1533
问题: Libterm doesn't support requesting fg and bg color.
解决方案: Implement t_RF and t_RB.
相关文件: src/libvterm/src/vterm_internal.h, src/libvterm/src/state.c,
src/libvterm/src/vterm.c

Patch 8.0.1534
问题: C syntax test fails when using gvim
解决方案: Force running in a terminal. Check that 'background' is correct
even when $COLORFGBG is set.
相关文件: src/testdir/test_syntax.vim, src/testdir/screendump.vim

Patch 8.0.1535 (after 8.0.1534)
问题: C syntax test still fails when using gvim.
解决方案: Clear Normal cterm highlighting instead of setting it.
相关文件: src/testdir/test_syntax.vim, src/testdir/screendump.vim,
src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.0.1536
问题: Quotestar test is flaky when using the GUI.
解决方案: Add check that the star register arrived at the server. Increase
timeouts.

相关文件: src/testdir/test_quotestar.vim

Patch 8.0.1537
问题: Xxd does not skip NUL lines when using ebcdic.
解决方案: Check for a NUL before converting a character for ebcdic. (Tim Sell, closes #2668)
相关文件: src/xxd/xxd.c

Patch 8.0.1538
问题: Popupmenu is too far left when completion is long. (Linwei)
解决方案: Adjust column computations. (Hirohito Higashi, closes #2661)
相关文件: src/popupmnu.c

Patch 8.0.1539
问题: No test for the popup menu positioning.
解决方案: Add a screendump test for the popup menu.
相关文件: src/terminal.c, src/testdir/test_syntax.vim,
src/testdir/screendump.vim,
src/testdir/test_popup.vim,
src/testdir/dumps/Test_popup_position_01.dump,
src/testdir/dumps/Test_popup_position_02.dump,
src/testdir/dumps/Test_popup_position_03.dump,
runtime/doc/eval.txt

Patch 8.0.1540
问题: Popup menu positioning fails with longer string.
解决方案: Only align with right side of window when width is less than 'pumwidth' (closes #2661)
相关文件: src/popupmnu.c, src/testdir/screendump.vim,
src/testdir/test_popup.vim,
src/testdir/dumps/Test_popup_position_04.dump

Patch 8.0.1541
问题: synpat_T is taking too much memory.
解决方案: Reorder members to reduce padding. (Dominique Pelle, closes #2671)
相关文件: src/syntax.c

Patch 8.0.1542
问题: Terminal screen dump does not include cursor position.
解决方案: Mark the cursor position in the dump.
相关文件: src/terminal.c,
src/testdir/dumps/Test_popup_position_01.dump,
src/testdir/dumps/Test_popup_position_02.dump,
src/testdir/dumps/Test_popup_position_03.dump,
src/testdir/dumps/Test_popup_position_04.dump,
src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.0.1543
问题: With 'termguicolors' Normal color doesn't work correctly.
解决方案: Set cterm_normal_bg_gui_color and cterm_normal_fg_color always. (Kazunobu Kuriyama, closes #981, closes #2332)
相关文件: src/syntax.c

Patch 8.0.1544

问题:　　When using 'termguicolors' SpellBad doesn't show.
解决方案:　When the GUI colors are not set fall back to the cterm colors.
相关文件:　src/syntax.c, src/screen.c, src/gui.h, src/structs.h

Patch 8.0.1545
问题:　　Screen dumps not included in distribution.
解决方案:　Add dumps to the list of distributed files.
相关文件:　Filelist

Patch 8.0.1546
问题:　　Using feedkeys() in a terminal window may trigger mappings.
(Charles Sheridan)
解决方案:　Avoid triggering a mapping when peeking for a key.
相关文件:　src/getchar.c, src/terminal.c

Patch 8.0.1547
问题:　　Undo in the options window makes it empty.
解决方案:　Set 'undolevels' while filling the buffer. (Yasuhiro Matsumoto,
closes #2645)
相关文件:　runtime/optwin.vim

Patch 8.0.1548
问题:　　Screen dump test script not included in distribution.
解决方案:　Add the script to the list of distributed files.
相关文件:　Filelist

Patch 8.0.1549
问题:　　Various small problems in test files.
解决方案:　Include small changes.
相关文件:　src/testdir/test_channel.py, src/testdir/shared.vim,
src/testdir/test_gui.vim, src/testdir/test_gui_init.vim

Patch 8.0.1550
问题:　　Various small problems in source files.
解决方案:　Fix the problems.
相关文件:　src/README.txt, src/beval.c, src/json_test.c, src/mbyte.c,
src/libvterm/include/vterm_keycodes.h, src/Makefile,
src/gui_gtk.c, src/if_xcmdsrv.c, src/pty.c, src/if_python.c,
src/if_py_both.h, uninstal.txt, src/dosinst.c, src/iscygpty.c,
src/vimrun.c, src/os_vms.c

Patch 8.0.1551
问题:　　On Mac 'maxmemtot' is set to a weird value.
解决方案:　For Mac use total memory and subtract system memory. For other
systems accept both a 32 bit and 64 bit result. (Ozaki Kiichi,
closes #2646)
相关文件:　src/os_unix.c

Patch 8.0.1552
问题:　　May leak file descriptors when executing job.
解决方案:　Close more file descriptors. (Ozaki Kiichi, closes #2651)
相关文件:　src/os_unix.c, src/testdir/test_channel.vim

Patch 8.0.1553

问题: Cannot see what digraph is used to insert a character.
解决方案: Show the digraph with the "ga" command. (Christian Brabandt)
相关文件: runtime/doc/various.txt, src/digraph.c, src/ex_cmds.c,
            src/proto/digraph.pro, src/testdir/shared.vim,
            src/testdir/test_matchadd_conceal.vim,
            src/testdir/test_digraph.vim, src/testdir/test_ga.vim,
            src/testdir/test_arabic.vim

Patch 8.0.1554
问题: Custom plugins loaded with --clean.
解决方案: Do not include the home directory in 'runtimepath'.
相关文件: src/option.c, src/main.c, src/proto/option.pro, src/structs.h,
            src/os_unix.h, src/os_amiga.h, src/os_dos.h, src/os_mac.h,
            runtime/doc/starting.txt

Patch 8.0.1555
问题: Build error for some combination of features.
解决方案: Declare variable in more situations.
相关文件: src/main.c

Patch 8.0.1556
问题: May not parse the t_RS response correctly, resulting in wrong
            characters in the input stream.
解决方案: When the t_RS response is partly received wait for more
            characters.
相关文件: src/term.c

Patch 8.0.1557
问题: printf() does not work with only one argument. (Daniel Hahler)
解决方案: Allow using just the format. (Ken Takata, closes #2687)
相关文件: src/evalfunc.c, src/testdir/test_expr.vim

Patch 8.0.1558
问题: No right-click menu in a terminal.
解决方案: Implement the right click menu for the terminal.
相关文件: src/popupmnu.c, src/proto/popupmnu.pro, src/normal.c, src/menu.c,
            src/proto/menu.pro, src/feature.h

Patch 8.0.1559
问题: Build failure without GUI.
解决方案: Adjust #ifdef for get_fpos_of_mouse().
相关文件: src/ui.c

Patch 8.0.1560
问题: Build failure without GUI on MS-Windows.
解决方案: Adjust #ifdef for vcol2col().
相关文件: src/ui.c

Patch 8.0.1561
问题: Crash with rust syntax highlighting. (Edd Barrett)
解决方案: Avoid going past the end of an empty line.
相关文件: src/syntax.c

Patch 8.0.1562

问题: The terminal debugger can't set a breakpoint with the mouse.
解决方案: Add popup menu entries.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
            runtime/doc/terminal.txt

Patch 8.0.1563
问题: Timeout of getwinposx() can be too short. (lilydjwg)
解决方案: Add getwinpos(). (closes #2689)
相关文件: src/evalfunc.c, src/term.c, src/proto/term.pro, runtime/doc/eval.txt

Patch 8.0.1564
问题: Too many #ifdefs.
解决方案: Graduate the +autocmd feature. Takes away 450 #ifdefs and
            increases code size of tiny Vim by only 40 Kbyte.
相关文件: src/buffer.c, src/diff.c, src/edit.c, src/eval.c, src/evalfunc.c,
            src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c,
            src/fileio.c, src/getchar.c, src/globals.h, src/gui.c,
            src/if_cscope.c, src/if_xcmdsrv.c, src/main.c, src/mbyte.c,
            src/memline.c, src/menu.c, src/misc1.c, src/gui_mac.c,
            src/misc2.c, src/move.c, src/netbeans.c, src/normal.c, src/ops.c,
            src/option.c, src/option.h, src/feature.h, src/vim.h,
            src/os_amiga.c, src/os_mswin.c, src/os_unix.c, src/os_win32.c,
            src/quickfix.c, src/screen.c, src/search.c, src/spell.c,
            src/structs.h, src/syntax.c, src/tag.c, src/term.c,
            src/terminal.c, src/ui.c, src/undo.c, src/userfunc.c,
            src/version.c, src/window.c

Patch 8.0.1565
问题: Can't build Mac version without GUI.
解决方案: Adjust when IME_WITHOUT_XIM is defined.
相关文件: src/vim.h

Patch 8.0.1566
问题: Too many #ifdefs.
解决方案: Graduate FEAT_SCROLLBIND and FEAT_CURSORBIND.
相关文件: src/buffer.c, src/diff.c, src/edit.c, src/evalfunc.c,
            src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/gui.c,
            src/main.c, src/move.c, src/normal.c, src/option.c, src/term.c,
            src/version.c, src/window.c, src/globals.h, src/macros.h,
            src/option.h, src/structs.h

Patch 8.0.1567
问题: Cannot build Win32 GUI without IME. (John Marriott)
解决方案: Adjust when IME_WITHOUT_XIM and HAVE_INPUT_METHOD are defined and
            use it in a few more places.
相关文件: src/vim.h, src/gui.c

Patch 8.0.1568
问题: Can't build on older Mac, header file is missing.
解决方案: Remove the header file. (Ozaki Kiichi, closes #2691)
相关文件: src/os_unix.c

Patch 8.0.1569
问题: Warning for uninitialized variable from gcc.

解决方案: Initialize the variable.
相关文件: src/quickfix.c

Patch 8.0.1570
问题: Can't use :popup for a menu in the terminal. (Wei Zhang)
解决方案: Make :popup work in the terminal. Also fix that entries were
included that don't work in the current state.
相关文件: src/ex_docmd.c, src/popupmnu.c, src/proto/popupmnu.pro,
src/menu.c, src/proto/menu.pro

Patch 8.0.1571 (after 8.0.1571)
问题: Can't build without GUI.
解决方案: Adjust #ifdef for gui_find_menu().
相关文件: src/menu.c

Patch 8.0.1572
问题: Mac: getting memory size doesn't work everywhere.
解决方案: Use MACOS_X instead of MACOS_X_DARWIN. (Kazunobu Kuriyama)
相关文件: src/os_unix.c

Patch 8.0.1573
问题: getwinpos(1) may cause response to be handled as command.
解决方案: Handle any cursor position report once one was requested. (partly
by Hirohito Higashi)
相关文件: src/term.c

Patch 8.0.1574
问题: Show cursor in wrong place when using popup menu. (Wei Zhang)
解决方案: Force updating the cursor position. Fix skipping over unused
entries.
相关文件: src/screen.c, src/proto/screen.pro, src/popupmnu.c

Patch 8.0.1575
问题: Crash when using virtual replace.
解决方案: Adjust orig_line_count. Add more tests. (Christian Brabandt)
相关文件: src/edit.c, src/testdir/test_visual.vim

Patch 8.0.1576
问题: Perl VIM::Buffers() does not find every buffer.
解决方案: Also find unlisted buffer by number or name. (Chris Weyl,
closes #2692)
相关文件: src/if_perl.xs

Patch 8.0.1577
问题: Virtual replace test fails on MS-Windows.
解决方案: Make adding a termcap entry work for a builtin terminal.
Restore terminal keys in a better way.
相关文件: src/term.c, src/testdir/test_visual.vim

Patch 8.0.1578
问题: No test for :popup in terminal.
解决方案: Add a screen dump test.
相关文件: src/testdir/test_popup.vim,
src/testdir/dumps/Test_popup_command_01.dump,

src/testdir/dumps/Test_popup_command_02.dump,
src/testdir/dumps/Test_popup_command_03.dump

Patch 8.0.1579
问题: Virtual replace test fails in GUI.
解决方案: Don't save key options if they were not set.
相关文件: src/testdir/test_visual.vim

Patch 8.0.1580
问题: FEAT_CURSORBIND and FEAT_SCROLLBIND are unused.
解决方案: Delete them.
相关文件: src/feature.h

Patch 8.0.1581
问题: Cannot build Win32 GUI without +eval.
解决方案: Define HAVE_INPUT_METHOD without +eval. (Ken Takata)
相关文件: src/vim.h

Patch 8.0.1582
问题: In the MS-Windows console mouse movement is not used.
解决方案: Pass mouse movement events when useful.
相关文件: src/os_win32.c, src/proto/os_win32.pro, src/feature.h

Patch 8.0.1583
问题: Using C99 comment.
解决方案: Use old style comment. (Kazunobu Kuriyama)
相关文件: src/quickfix.c

Patch 8.0.1584
问题: Using C99 in Mac file gives compiler warning messages.
解决方案: Add #pragmas to avoid the warnings. (Kazunobu Kuriyama)
相关文件: src/os_macosx.m

Patch 8.0.1585
问题: Enabling beval_term feature in Win32 GUI.
解决方案: Only enable beval_term in Win32 console.
相关文件: src/feature.h

Patch 8.0.1586
问题: Imactivatefunc does not work on non-GUI Mac.
解决方案: Fix logic in #ifdef.
相关文件: src/vim.h

Patch 8.0.1587
问题: inserting from the clipboard doesn't work literally
解决方案: When pasting from the * or + register always assume literally.
相关文件: src/ops.c, src/proto/ops.pro, src/testdir/test_paste.vim

Patch 8.0.1588
问题: Popup menu hangs after typing CTRL-C.
解决方案: Make CTRL-C exit the loop. (Ozaki Kiichi, closes #2697)
相关文件: src/popupmnu.c

Patch 8.0.1589

问题: Error for setting 'modifiable' when resetting it.
解决方案: Check if 'modifiable' was actually set.
相关文件: src/option.c

Patch 8.0.1590
问题: Padding in list type wastes memory.
解决方案: Reorder struct members to optimize padding. (Dominique Pelle,
            closes #2704)
相关文件: src/structs.h

Patch 8.0.1591
问题: MS-Windows: when reparsing the arguments 'wildignore' matters.
解决方案: Save and reset 'wildignore'. (Yasuhiro Matsumoto, closes #2702)
相关文件: src/os_win32.c

Patch 8.0.1592
问题: Terminal windows in a session are not properly restored.
解决方案: Add "terminal" in 'sessionoptions'. When possible restore the
            command running in a terminal.
相关文件: src/option.c, src/option.h, src/ex_docmd.c, src/terminal.c,
            src/proto/terminal.pro, src/evalfunc.c, src/structs.h,
            src/channel.c, src/testdir/test_terminal.vim,
            src/testdir/shared.vim, src/testdir/test_mksession.vim

Patch 8.0.1593
问题: :qall never exits with an active terminal window.
解决方案: Add a way to kill a job in a terminal window.
相关文件: src/ex_cmds2.c, src/terminal.c, src/proto/terminal.pro,
            src/structs.h, src/channel.c, src/evalfunc.c,
            src/testdir/test_terminal.vim, runtime/doc/terminal.txt,
            runtime/doc/eval.txt

Patch 8.0.1594
问题: :confirm qall not tested with active terminal window.
解决方案: Add a test.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1595
问题: No autocommand triggered before exiting.
解决方案: Add the ExitPre autocommand event.
相关文件: src/ex_docmd.c, src/fileio.c, src/vim.h,
            src/testdir/test_exit.vim, src/Makefile, src/testdir/Make_all.mak,
            runtime/doc/autocmd.txt

Patch 8.0.1596
问题: No autocommand specifically for opening a terminal window.
解决方案: Add TerminalOpen. (Yasuhiro Matsumoto, closes #2484)
相关文件: runtime/doc/autocmd.txt, src/fileio.c, src/terminal.c,
            src/testdir/test_terminal.vim, src/vim.h

Patch 8.0.1597
问题: Autocommand events are not sorted.
解决方案: Sort the autocommand events.
相关文件: src/vim.h

Patch 8.0.1598
问题:　　Cannot select text in a terminal with the mouse.
解决方案:　When a job in a terminal is not consuming mouse events, use them
　　for modeless selection. Also stop Insert mode when clicking in a
　　terminal window.
相关文件:　src/libvterm/include/vterm.h, src/libvterm/src/state.c,
　　src/libvterm/src/vterm_internal.h, src/terminal.c,
　　src/proto/terminal.pro, src/ui.c

Patch 8.0.1599
问题:　　No error message when gdb does not support the terminal debugger.
解决方案:　Check for the response to open the Machine Interface.
相关文件:　runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1600
问题:　　Crash when setting t_Co to zero when 'termguicolors' is set.
解决方案:　Use IS_CTERM instead of checking the number of colors.
　　(closes #2710)
相关文件:　src/screen.c, src/testdir/test_highlight.vim

Patch 8.0.1601
问题:　　Highlight test fails on Win32.
解决方案:　Check for vtp and vcon support.
相关文件:　src/evalfunc.c, src/testdir/test_highlight.vim

Patch 8.0.1602
问题:　　Crash in parsing JSON.
解决方案:　Fail when using array or dict as dict key. (Damien)
相关文件:　src/json.c, src/testdir/test_json.vim

Patch 8.0.1603
问题:　　Cannot build with +terminal but without +menu.
解决方案:　Add #ifdef. (Damien)
相关文件:　src/terminal.c

Patch 8.0.1604
问题:　　Paste test may fail if $DISPLAY is not set.
解决方案:　Add WorkingClipboard() and use it in the paste test.
相关文件:　src/testdir/shared.vim, src/testdir/test_paste.vim

Patch 8.0.1605
问题:　　Terminal test is a bit flaky.
解决方案:　Check for the shell prompt. Use more lambda functions.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1606
问题:　　Singular/plural variants not translated.
解决方案:　Add NGETTEXT argument to xgettext. (Sergey Alyoshin)
相关文件:　src/po/Make_cyg.mak, src/po/Make_ming.mak, src/po/Make_mvc.mak,
　　src/po/Makefile

Patch 8.0.1607
问题:　　--clean loads user settings from .gvimrc.

解决方案: Behave like "-U NONE" was used. (Ken Takata)
相关文件: src/main.c, runtime/doc/starting.txt

Patch 8.0.1608
问题: Win32: directx not enabled by default.
解决方案: Change Makefile to enable directx by default. (Ken Takata)
相关文件: runtime/doc/various.txt, src/Make_cyg_ming.mak,
src/Make_mvc.mak

Patch 8.0.1609
问题: Shell commands in the GUI use a dumb terminal.
解决方案: Add the "!" flag to 'guioptions' to execute system commands in a
special terminal window. Only for Unix now.
相关文件: src/os_unix.c, src/option.h, src/evalfunc.c, src/terminal.c,
src/proto/terminal.pro, src/channel.c, src/proto/channel.pro,
src/vim.h, runtime/doc/options.txt

Patch 8.0.1610 (after 8.0.1609)
问题: Cannot build without GUI.
解决方案: Add #ifdef.
相关文件: src/terminal.c

Patch 8.0.1611
问题: CTRL-W in system terminal does not go to job.
解决方案: Do not use CTRL-W as a terminal command in a system terminal.
相关文件: src/terminal.c

Patch 8.0.1612
问题: Need to close terminal after shell stopped.
解决方案: Make :terminal without argument close the window by default.
相关文件: src/terminal.c, src/testdir/test_terminal.vim,
runtime/doc/terminal.txt

Patch 8.0.1613
问题: Warning for unused variable in tiny build. (Tony Mechelynck)
解决方案: Move declaration to inner block.
相关文件: src/os_unix.c

Patch 8.0.1614
问题: "make tags" doesn't include libvterm.
解决方案: Add the libvterm sources to the tags command.
相关文件: src/Makefile

Patch 8.0.1615
问题: term_dumpload() does not use the right colors.
解决方案: Initialize colors when not using create_vterm().
相关文件: src/terminal.c

Patch 8.0.1616
问题: Win32: shell commands in the GUI open a new console.
解决方案: Use a terminal window for interactive use when 'guioptions'
contains "!".
相关文件: src/os_win32.c

Patch 8.0.1617 (after 8.0.1616)
问题:　　Win32: :shell command in the GUI crashes.
解决方案:　Handle the situation that "cmd" is NULL. (Yasuhiro Matsumoto,
　　closes #2721)
相关文件:　src/os_win32.c

Patch 8.0.1618
问题:　　Color Grey50, used for ToolbarLine, is missing in the compiled-in
　　table.
解决方案:　Add the color to the list. (Kazunobu Kuriyama)
相关文件:　src/term.c

Patch 8.0.1619
问题:　　Win32 GUI: crash when winpty is not installed and trying to use
　　:shell in a terminal window.
解决方案:　Check for NULL return form term_start(). (Yasuhiro Matsumoto,
　　closes #2727)
相关文件:　src/os_win32.c

Patch 8.0.1620
问题:　　Reading spell file has no good EOF detection.
解决方案:　Check for EOF at every character read for a length field.
相关文件:　src/misc2.c

Patch 8.0.1621
问题:　　Using invalid default value for highlight attribute.
解决方案:　Use zero instead of -1.
相关文件:　src/syntax.c

Patch 8.0.1622
问题:　　Possible NULL pointer dereference. (Coverity)
解决方案:　Reverse the check for a NULL pointer.
相关文件:　src/quickfix.c

Patch 8.0.1623
问题:　　Terminal kill tests are flaky.
解决方案:　Instead of running Vim in a terminal, run it as a normal command.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1624
问题:　　Options for term_dumpdiff() and term_dumpload() not implemented
　　yet.
解决方案:　Implement the relevant options.
相关文件:　src/terminal.c, runtime/doc/eval.txt

Patch 8.0.1625
问题:　　Test_quotestar is flaky when run in GTK GUI.
解决方案:　Do not call lose_selection when invoked from
　　selection_clear_event().
相关文件:　src/gui_gtk_x11.c

Patch 8.0.1626
问题:　　Compiler warning for possible loss of data.
解决方案:　Use size_t instead of int. (Christian Brabandt)

相关文件: src/terminal.c

Patch 8.0.1627
问题: Compiler warning for visibility attribute not supported on MinGW builds.
解决方案: Don't add the attribute when we don't expect it to work. (Christian Brabandt)
相关文件: src/libvterm/src/vterm_internal.h

Patch 8.0.1628
问题: Channel log doesn't mention exiting.
解决方案: Add a ch_log() call in getout().
相关文件: src/main.c

Patch 8.0.1629
问题: Mac: getpagesize() is deprecated.
解决方案: Use sysconf() instead. (Ozaki Kiichi, closes #2741)
相关文件: src/os_unix.c

Patch 8.0.1630
问题: Trimming white space is not that easy.
解决方案: Add the trim() function. (Bukn, Yasuhiro Matsumoto, closes #1280)
相关文件: src/evalfunc.c, runtime/doc/eval.txt, src/testdir/test_functions.vim

Patch 8.0.1631
问题: Testing with Vim running in terminal is a bit flaky.
解决方案: Delete any .swp file so that later tests don't fail.
相关文件: src/testdir/screendump.vim

Patch 8.0.1632
问题: In a terminal dump NUL and space considered are different, although they are displayed the same.
解决方案: When encountering NUL handle it like space.
相关文件: src/terminal.c

Patch 8.0.1633
问题: A TextChanged autocmd triggers when it is defined after creating a buffer.
解决方案: Set b_last_changedtick when opening a buffer. (Hirohito Higashi, closes #2742)
相关文件: src/buffer.c, src/testdir/test_autocmd.vim

Patch 8.0.1634
问题: The ex_vimgrep() function is too long.
解决方案: Split it in smaller functions. (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.1635
问题: Undefining _POSIX_THREADS causes problems with Python 3. (Micah Bucy, closes #2748)
解决方案: Remove the lines.
相关文件: src/if_python3.c

Patch 8.0.1636
问题:　　No test for term_dumpload() and term_dumpdiff().
解决方案:　Add tests.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1637
问题:　　No test for term_dumpdiff() options argument.
解决方案:　Add a test.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1638
问题:　　Popup test fails depending on environment variable.
解决方案:　Reset $COLORFGBG when running Vim in a terminal. (closes #2693)
相关文件:　src/testdir/screendump.vim

Patch 8.0.1639
问题:　　Libvterm code lags behind master.
解决方案:　Sync to head, solve merge problems.
相关文件:　src/libvterm/README, src/libvterm/bin/unterm.c,
　　src/libvterm/bin/vterm-ctrl.c, src/libvterm/bin/vterm-dump.c,
　　src/libvterm/doc/URLs, src/libvterm/doc/seqs.txt,
　　src/libvterm/include/vterm.h,
　　src/libvterm/include/vterm_keycodes.h, src/libvterm/src/mouse.c,
　　src/libvterm/src/parser.c, src/libvterm/src/pen.c,
　　src/libvterm/src/screen.c, src/libvterm/src/state.c,
　　src/libvterm/src/vterm.c, src/libvterm/src/vterm_internal.h,
　　src/libvterm/t/10state_putglyph.test,
　　src/libvterm/t/25state_input.test, src/libvterm/t/harness.c,
　　src/libvterm/t/26state_query.test

Patch 8.0.1640
问题:　　Test_cwd() is flaky.
解决方案:　Add to list of flaky tests.
相关文件:　src/testdir/runtest.vim

Patch 8.0.1641
问题:　　Job in terminal can't communicate with Vim.
解决方案:　Add the terminal API.
相关文件:　src/terminal.c, src/buffer.c, src/testdir/test_terminal.vim,
　　src/testdir/screendump.vim, runtime/doc/terminal.txt

Patch 8.0.1642
问题:　　Running Vim in terminal fails with two windows.
解决方案:　Pass the number of rows to RunVimInTerminal().
相关文件:　src/testdir/screendump.vim, src/testdir/test_terminal.vim

Patch 8.0.1643
问题:　　Terminal API tests fail.
解决方案:　Explicitly set 'title'.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1644
问题:　　Terminal API tests still fail.
解决方案:　Explicitly set 'title' in the terminal job. (Ozaki Kiichi,

closes #2750)
相关文件: src/testdir/test_terminal.vim, src/testdir/screendump.vim

Patch 8.0.1645
问题: Test for terminal response to escape sequence fails for some peoples. (toothpik)
解决方案: Run "cat" and let it echo the characters.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1646
问题: MS-Windows: executable contains unreferenced functions and data.
解决方案: Add /opt:ref to the compiler command. (Ken Takata)
相关文件: src/Make_mvc.mak

Patch 8.0.1647
问题: Terminal API may call a function not meant to be called by this API.
解决方案: Require the function to start with Tapi_.
相关文件: runtime/doc/terminal.txt, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1648
问题: Resource fork tool doesn't work on Python 3.
解决方案: Use "print()" instead of "print". (Marius Gedminas)
相关文件: src/dehqx.py

Patch 8.0.1649
问题: No completion for argument list commands.
解决方案: Add arglist completion. (Yegappan Lakshmanan, closes #2706)
相关文件: runtime/doc/eval.txt, runtime/doc/map.txt, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/proto/ex_cmds2.pro, src/testdir/test_cmdline.vim, src/vim.h

Patch 8.0.1650
问题: Too many #ifdefs.
解决方案: Graduate FEAT_LISTCMDS, no reason to leave out buffer commands.
相关文件: runtime/doc/various.txt, src/buffer.c, src/charset.c, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/version.c, src/feature.h

Patch 8.0.1651
问题: Cannot filter :ls output for terminal buffers.
解决方案: Add flags for terminal buffers. (Marcin Szamotulski, closes #2751)
相关文件: runtime/doc/windows.txt, src/buffer.c, src/testdir/test_terminal.vim

Patch 8.0.1652
问题: term_dumpwrite() does not output composing characters.
解决方案: Use the cell index.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1653
问题: Screen dump is made too soon.
解决方案: Wait until the ruler is displayed. (Ozaki Kiichi, closes #2755)

相关文件: src/testdir/dumps/Test_popup_command_01.dump,
           src/testdir/dumps/Test_popup_command_02.dump,
           src/testdir/screendump.vim, src/testdir/test_autocmd.vim,
           src/testdir/test_terminal.vim

Patch 8.0.1654
问题: Warnings for conversion of void to function pointer.
解决方案: Use a temp variable that is a function pointer.
相关文件: src/if_python.c, src/if_python3.c

Patch 8.0.1655
问题: Outdated gdb message in terminal debugger unclear.
解决方案: Specifically mention the required gdb version. Avoid getting
           stuck on pagination.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.0.1656
问题: No option to have xxd produce upper case variable names.
解决方案: Add the -C argument. (Matt Panaro, closes #2772)
相关文件: src/xxd/xxd.c

Patch 8.0.1657
问题: Crash when reading a channel.
解决方案: Clear the write flag before writing. (idea by Shinya Ohyanagi,
           closes #2769).
相关文件: src/channel.c

Patch 8.0.1658
问题: Capitalize argument not available in long form.
解决方案: Recognize -capitalize. Update man page.
相关文件: src/xxd/xxd.c, runtime/doc/xxd.1, runtime/doc/xxd.man

Patch 8.0.1659
问题: Scroll events not recognized for some xterm emulators.
解决方案: Recognize mouse codes 0x40 and 0x41 as scroll events.
相关文件: src/term.c

Patch 8.0.1660
问题: The terminal API "drop" command doesn't support options.
解决方案: Implement the options.
相关文件: src/terminal.c, src/ex_docmd.c, src/proto/ex_docmd.pro,
           src/ex_cmds.h, src/eval.c, src/misc2.c, src/fileio.c,
           src/testdir/test_terminal.vim, runtime/doc/terminal.txt

Patch 8.0.1661
问题: Warnings from 64 bit compiler.
解决方案: Add type casts. (Mike Williams)
相关文件: src/terminal.c

Patch 8.0.1662
问题: Showing dump diff doesn't mention both file names.
解决方案: Add the file name in the separator line.
相关文件: src/terminal.c

Patch 8.0.1663 (after 8.0.1660)
问题:　　Cannot build without multi-byte feature.
解决方案:　Add #ifdef.
相关文件:　src/ex_docmd.c

Patch 8.0.1664
问题:　　Test failure because of not allocating enough space.
解决方案:　Allocate more bytes.
相关文件:　src/terminal.c

Patch 8.0.1665
问题:　　When running a terminal from the GUI 'term' is not useful.
解决方案:　Use $TERM in the GUI if it starts with "xterm". (closes #2776)
相关文件:　src/os_unix.c, runtime/doc/terminal.txt

Patch 8.0.1666
问题:　　% argument in ch_log() causes trouble.
解决方案:　Use string as third argument in internal ch_log(). (Dominique
　　Pelle, closes #2784)
相关文件:　src/evalfunc.c, src/testdir/test_channel.vim

Patch 8.0.1667
问题:　　Terminal window tests are flaky.
解决方案:　Increase the waiting time for Vim to start.
相关文件:　src/testdir/screendump.vim

Patch 8.0.1668
问题:　　Terminal debugger: can't re-open source code window.
解决方案:　Add the :Source command. Also create the window if needed when
　　gdb stops at a source line.
相关文件:　runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
　　runtime/doc/terminal.txt

Patch 8.0.1669
问题:　　:vimgrep may add entries to the wrong quickfix list.
解决方案:　Use the list identifier. (Yegappan Lakshmanan)
相关文件:　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1670
问题:　　Terminal window tests are still a bit flaky.
解决方案:　Increase the waiting time for the buffer to be created.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1671
问题:　　Crash when passing non-dict argument as env to job_start().
解决方案:　Check for valid argument. (Ozaki Kiichi, closes #2765)
相关文件:　src/channel.c, src/testdir/test_channel.vim

Patch 8.0.1672
问题:　　Error during completion causes command to be cancelled.
解决方案:　Reset did_emsg before waiting for another character. (Tom M.)
相关文件:　src/ex_getln.c, src/testdir/test_cmdline.vim

Patch 8.0.1673

问题: Terminal window tests are still a bit flaky.
解决方案: Increase the waiting time even more. (Elimar Riesebieter)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1674
问题: Libvterm can't handle a long OSC string that is split.
解决方案: When an incomplete OSC string is received copy it to the parser
            buffer. Increase the size of the parser buffer to be able to
            handle longer strings.
相关文件: src/libvterm/src/parser.c, src/libvterm/src/vterm.c

Patch 8.0.1675
问题: Unused macro argument in libvterm. (Randall W. Morris)
解决方案: Remove the argument.
相关文件: src/libvterm/src/parser.c

Patch 8.0.1676
问题: No compiler warning for wrong printf format.
解决方案: Add a printf attribute for gcc. Fix reported problems. (Dominique
            Pelle, closes #2789)
相关文件: src/channel.c, src/vim.h, src/proto/channel.pro

Patch 8.0.1677
问题: No compiler warning for wrong format in vim_snprintf().
解决方案: Add printf attribute for gcc. Fix reported problems.
相关文件: src/vim.h, src/proto.h, src/eval.c, src/fileio.c, src/mbyte.c,
            src/ops.c, src/spellfile.c, src/undo.c, src/json.c

Patch 8.0.1678
问题: Errorformat "%r" implies "%>". (Jan Gosmann)
解决方案: Jump to before setting fmt_ptr. (Yegappan Lakshmanan,
            closes #2785)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1679
问题: Compiler warning for printf format. (Chdiza)
解决方案: Change type to "long long". (closes #2791)
相关文件: src/ops.c

Patch 8.0.1680
问题: Memory allocated by libvterm does not show up in profile.
解决方案: Pass allocator functions to vterm_new().
相关文件: src/terminal.c

Patch 8.0.1681
问题: The format attribute fails with MinGW. (John Marriott)
解决方案: Don't use the format attribute with MinGW.
相关文件: src/vim.h, src/proto.h, src/channel.c

Patch 8.0.1682
问题: Auto indenting breaks inserting a block.
解决方案: Do not check for cursor movement if indent was changed. (Christian
            Brabandt, closes #2778)
相关文件: src/testdir/test_blockedit.vim, src/testdir/Make_all.mak,

src/Makefile, src/ops.c

Patch 8.0.1683
问题: Python upgrade breaks Vim when defining PYTHON_HOME.
解决方案: Do not define PYTHON_HOME and PYTHON3_HOME in configure. (Naoki Inada, closes #2787)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1684
问题: ml_get errors when using terminal window for shell command. (Blay263)
解决方案: Do not change the size of the current window.
相关文件: src/terminal.c

Patch 8.0.1685
问题: Can't set ANSI colors of a terminal window.
解决方案: Add term_setansicolors(), term_getansicolors() and g:term_ansi_colors. (Andy Massimino, closes #2747)
相关文件: runtime/doc/eval.txt, runtime/doc/terminal.txt, src/channel.c, src/evalfunc.c, src/proto/terminal.pro, src/structs.h, src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1686 (after 8.0.1683)
问题: Python does not work when configuring with specific dir. (Rajdeep)
解决方案: Do define PYTHON_HOME and PYTHON3_HOME in configure if the Python config dir was specified.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1687
问题: 64 bit compiler warnings.
解决方案: change type, add type cast. (Mike Williams)
相关文件: src/terminal.c

Patch 8.0.1688
问题: Some macros are used without a semicolon, causing auto-indent to be wrong.
解决方案: Use the do-while(0) trick. (Ozaki Kiichi, closes #2729)
相关文件: src/buffer.c, src/dosinst.c, src/ex_cmds.c, src/gui_at_sb.c, src/macros.h, src/main.c, src/memline.c, src/option.c, src/os_vms.c, src/screen.c, src/window.c

Patch 8.0.1689
问题: No tests for xxd.
解决方案: Add a test. (Christian Brabandt)
相关文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Makefile, src/testdir/test_xxd.vim, src/testdir/runtest.vim

Patch 8.0.1690
问题: Not easy to run one test with gvim instead of vim.
解决方案: Add VIMTESTTARGET in Makefile.
相关文件: src/Makefile

Patch 8.0.1691
问题: Xxd test sometimes fails.

解决方案: Wipe out the XXDfile buffer.
相关文件: src/testdir/test_xxd.vim

Patch 8.0.1692 (after 8.0.1686)
问题: Python may not work when using statically linked library.
解决方案: Do not define PYTHON_HOME and PYTHON3_HOME in configure if the
           Python library is linked statically.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1693
问题: Xxd is excluded from coverage statistics.
解决方案: Don't skip the xxd directory. (Christian Brabandt)
相关文件: .travis.yml

Patch 8.0.1694
问题: Terminal API test is a bit flaky.
解决方案: Wait longer for Vim to stop.
相关文件: src/testdir/screendump.vim

Patch 8.0.1695
问题: Xxd test not run on MS-Windows.
解决方案: Use xxd.exe if it exists.
相关文件: src/testdir/test_xxd.vim

Patch 8.0.1696
问题: Coverage statistics don't work.
解决方案: Include the xxd directory. (Christian Brabandt)
相关文件: .travis.yml

Patch 8.0.1697
问题: Various tests are still a bit flaky.
解决方案: Increase the default wait time to five seconds.
相关文件: src/testdir/shared.vim, src/testdir/screendump.vim,
           src/testdir/test_channel.vim, src/testdir/test_clientserver.vim,
           src/testdir/test_quotestar.vim, src/testdir/test_terminal.vim

Patch 8.0.1698
问题: Coverage statistics don't work on coveralls.
解决方案: Use curly braces for $SRCDIR.
相关文件: .travis.yml

Patch 8.0.1699
问题: Leftover stuff for Python 1.4.
解决方案: Remove outdated Python 1.4 stuff. (Naoki Inada, closes #2794)
相关文件: src/Makefile, src/config.aap.in, src/config.mk.in,
           src/configure.ac, src/auto/configure

Patch 8.0.1700
问题: Coverage statistics still don't work on coveralls.
解决方案: Exclude the xxd directory again.
相关文件: .travis.yml

Patch 8.0.1701
问题: Can disable COLOR_EMOJI with MSVC but not MinGW.

解决方案: Add COLOR_EMOJI flag. Also add some empty lines for readability.
相关文件: src/Make_cyg_ming.mak

Patch 8.0.1702
问题: Leaking memory when autocommands make a quickfix list invalid.
解决方案: Call FreeWild(). (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.1703
问题: In the tutor 'showcmd' is not set.
解决方案: Set 'showcmd' in the vimtutor script. (Ken Takata, closes #2792)
相关文件: src/vimtutor

Patch 8.0.1704
问题: 'backupskip' default doesn't work for Mac.
解决方案: Use "/private/tmp". (Rainer Müller, closes #2793)
相关文件: src/option.c, src/testdir/test_options.vim,
runtime/doc/options.txt

Patch 8.0.1705
问题: When making a vertical split the mode message isn't always
updated, "VISUAL" remains. (Alexei Averchenko)
解决方案: Only reset clear_cmdline when filling all columns of the last
screen line. (Tom M. closes #2611)
相关文件: src/screen.c, src/testdir/test_window_cmd.vim

Patch 8.0.1706
问题: Cannot send CTRL-\ to a terminal window.
解决方案: Make CTRL-W CTRL-\ send CTRL-\ to a terminal window.
相关文件: src/terminal.c, runtime/doc/terminal.txt

Patch 8.0.1707
问题: When 'wfh' is set ":bel 10new" scrolls window. (Andrew Pyatkov)
解决方案: Set the fraction before changing the window height. (closes #2798)
相关文件: src/window.c

Patch 8.0.1708
问题: Mkdir with 'p' flag fails on existing directory, which is
different from the mkdir shell command.
解决方案: Don't fail if the directory already exists. (James McCoy,
closes #2775)
相关文件: src/evalfunc.c, src/testdir/test_eval_stuff.vim,
runtime/doc/eval.txt

Patch 8.0.1709
问题: Some non-C89 code may slip through.
解决方案: Enforce C89 in configure. Fix detected problems. (James McCoy,
closes #2735)
相关文件: src/channel.c, src/configure.ac, src/auto/configure,
src/gui_gtk_x11.c, src/if_python3.c

Patch 8.0.1710
问题: Building with Ruby fails.
解决方案: Don't add -ansi when building with Ruby.

相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1711
问题: Term_setsize() is not implemented yet.
解决方案: Implement it.
相关文件: src/evalfunc.c, src/terminal.c, src/proto/terminal.pro,
src/testdir/test_terminal.vim, runtime/doc/eval.txt

Patch 8.0.1712
问题: Terminal scrollback is not limited.
解决方案: Add the 'terminalscroll' option.
相关文件: src/terminal.c, src/option.h, src/option.c,
runtime/doc/options.txt, runtime/doc/terminal.txt

Patch 8.0.1713
问题: Terminal debugger doesn't handle arguments.
解决方案: Use <f-args> and pass all the arguments to gdb, e.g. the core file
or process number. (suggested by Christian Brabandt) Disallow
starting the debugger twice.
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
runtime/doc/terminal.txt

Patch 8.0.1714
问题: Term_setsize() does not give an error in a normal buffer.
解决方案: Add an error message.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1715
问题: Terminal buffer can be 1 more than 'terminalscroll' lines.
解决方案: Change > to >=.
相关文件: src/terminal.c

Patch 8.0.1716
问题: Test for term_setsize() does not give a good error message.
解决方案: use assert_inrange().
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1717
问题: C89 check causes too much trouble.
解决方案: Remove enforcing C89 for now.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1718
问题: Terminal scrollback test fails on MS-Windows.
解决方案: Check for the last line of output anticipating there might be an
empty line below it.
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1719
问题: Cannot specify which Python executable configure should use.
解决方案: Add --with-python-command and --with-python3-command.
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1720

问题: When a timer is running a terminal window may not close after a
            shell has exited.
解决方案: Call job_status() more often.
相关文件: src/terminal.c

Patch 8.0.1721
问题: No test for using the 'termsize' option.
解决方案: Add a test.
相关文件: src/testdir/screendump.vim, src/testdir/test_terminal.vim

Patch 8.0.1722
问题: Cannot specify a minimal size for a terminal window.
解决方案: Support the "rows*cols" format for 'winsize'.
相关文件: src/terminal.c, src/testdir/test_terminal.vim, src/option.c,
            runtime/doc/options.txt

Patch 8.0.1723
问题: Using one item array size declaration is misleading.
解决方案: Instead of using "[1]" and actually using a larger array, use
            "[]". This is to verify that this C99 feature works for all
            compilers.
相关文件: src/structs.h, src/getchar.c

Patch 8.0.1724
问题: Declarations cannot be halfway a block.
解决方案: Move one declaration to check if this works for all compilers.
相关文件: src/main.c

Patch 8.0.1725
问题: Terminal debugger doesn't handle command arguments.
解决方案: Add the :TermdebugCommand command. Use a ! to execute right away.
            (Christian Brabandt)
相关文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim,
            runtime/doc/terminal.txt

Patch 8.0.1726 (after 8.0.1724)
问题: Older MSVC doesn't support declarations halfway a block.
解决方案: Move the declaration back to the start of the block.
相关文件: src/main.c

Patch 8.0.1727
问题: qf_get_properties() function is too long.
解决方案: Refactor the code. (Yegappan Lakshmanan, closes #2807)
相关文件: src/quickfix.c

Patch 8.0.1728
问题: Condition always false, useless code.
解决方案: Remove the code. (Nikolai Pavlov, closes #2808)
相关文件: src/message.c

Patch 8.0.1729
问题: No comma after last enum item.
解决方案: Add a few commas to check if this works for all compilers. Also
            add a few // comments.

相关文件: src/structs.h

Patch 8.0.1730
问题: No configure check for the used C99 features.
解决方案: Add a compilation check. Tentatively document C99 features.
相关文件: src/configure.ac, src/auto/configure, runtime/doc/develop.txt

Patch 8.0.1731
问题: Characters deleted on completion. (Adrià Farrés)
解决方案: Also check the last item for the ORIGINAL_TEXT flag. (Christian Brabandt, closes #1645)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.1732
问题: Crash when terminal API call deletes the buffer.
解决方案: Lock the buffer while calling a function. (closes #2813)
相关文件: src/buffer.c, src/terminal.c, src/testdir/test_terminal.vim, src/testdir/test_autocmd.vim

Patch 8.0.1733
问题: Incomplete testing for completion fix. (Lifepillar)
解决方案: Add a test with CTRL-P.
相关文件: src/testdir/test_popup.vim

Patch 8.0.1734
问题: Package directory not added to 'rtp' if prefix matches.
解决方案: Check the match is a full match. (Ozaki Kiichi, closes #2817) Also handle different ways of spelling a path.
相关文件: src/testdir/test_packadd.vim, src/ex_cmds2.c

Patch 8.0.1735 (after 8.0.1723 and 8.0.1730)
问题: Flexible array member feature not supported by HP-UX. (John Marriott)
解决方案: Do not use the flexible array member feature of C99.
相关文件: src/configure.ac, src/auto/configure, src/structs.h, src/getchar.c, runtime/doc/develop.txt

Patch 8.0.1736
问题: Check for C99 features is incomplete.
解决方案: Use AC_PROG_CC_C99 and when C99 isn't fully supported check the features we need. (James McCoy, closes #2820)
相关文件: src/configure.ac, src/auto/configure

Patch 8.0.1737
问题: fchown() used when it is not supported.
解决方案: Add #ifdef.
相关文件: src/fileio.c

Patch 8.0.1738
问题: ":args" output is hard to read.
解决方案: Make columns with the names if the output is more than one line.
相关文件: src/ex_cmds2.c, src/version.c, src/proto/version.pro, src/testdir/test_arglist.vim

Patch 8.0.1739
问题: MS-Windows with msys2 cannot build Ruby statically.
解决方案: Define RUBY_VERSION. (Gray Wolf, closes #2826)
相关文件: src/Make_cyg_ming.mak

Patch 8.0.1740
问题: Warning for signed-unsigned incompatibility.
解决方案: Change type from "char *" to "char_u *". (John Marriott)
相关文件: src/ex_cmds2.c

Patch 8.0.1741
问题: MS-Windows with msys2 cannot build Ruby statically.
解决方案: Add RUBY_VERSION to CFLAGS later. (Gray Wolf, closes #2833)
相关文件: src/Make_cyg_ming.mak

Patch 8.0.1742
问题: Cannot get a list of all the jobs. Cannot get the command of
            the job.
解决方案: When job_info() is called without an argument return a list of
            jobs. Otherwise, include the command that the job is running.
            (Yegappan Lakshmanan)
相关文件: runtime/doc/eval.txt, src/channel.c, src/evalfunc.c,
            src/proto/channel.pro, src/structs.h, src/testdir/test_channel.vim

Patch 8.0.1743
问题: Terminal window options are named inconsistently.
解决方案: prefix terminal window options with "termwin". Keep the old names
            for now as an alias.
相关文件: src/option.c, src/option.h, src/structs.h, src/terminal.c,
            src/testdir/test_terminal.vim, src/testdir/gen_opt_test.vim,
            runtime/doc/options.txt, runtime/doc/quickref.txt,
            runtime/doc/terminal.txt, runtime/optwin.vim

Patch 8.0.1744
问题: On some systems /dev/stdout isn't writable.
解决方案: Skip test if writing is not possible. (James McCoy, closes #2830)
相关文件: src/testdir/test_writefile.vim

Patch 8.0.1745
问题: Build failure on MS-Windows.
解决方案: Build job arguments for MS-Windows. Fix allocating job twice.
相关文件: src/structs.h, src/channel.c, src/os_unix.c, src/misc2.c,
            src/terminal.c, src/proto/misc2.pro

Patch 8.0.1746
问题: MS-Windows: channel tests fail.
解决方案: Make a copy of the command before splitting it.
相关文件: src/channel.c

Patch 8.0.1747
问题: MS-Windows: term_start() does not set job_info() cmd.
解决方案: Share the code from job_start() to set jv_argv.
相关文件: src/testdir/test_terminal.vim, src/channel.c, src/misc2.c,
            src/proto/misc2.pro, src/terminal.c

Patch 8.0.1748
问题: CmdlineEnter command uses backslash instead of slash.
解决方案: Don't treat the character as a file name. (closes #2837)
相关文件: src/fileio.c, src/testdir/test_autocmd.vim

Patch 8.0.1749
问题: VMS: 100% CPU use, redefining mch_open() and mch_fopen() fails.
解决方案: Do not wait indefinitely in RealWaitForChar(). (Neil Rieck)
Do not redefine mch_open() and mch_fopen() on VMS. (Zoltan Arpadffy)
相关文件: src/os_vms.c, src/vim.h

Patch 8.0.1750
问题: Crash when clearing location list in autocommand.
解决方案: Check if "qi" equals "ql_info". (Yegappan Lakshmanan)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1751
问题: #ifdef causes bad highlighting.
解决方案: Move code around. (Ozaki Kiichi, closes #2731)
相关文件: src/ui.c

Patch 8.0.1752
问题: qf_set_properties() is to long.
解决方案: Refactor the function. Define INVALID_QFIDX. (Yegappan Lakshmanan, closes #2812)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1753
问题: Various warnings from a static analyser
解决方案: Add type casts, remove unneeded conditions. (Christian Brabandt, closes #2770)
相关文件: src/evalfunc.c, src/ex_cmds2.c, src/fileio.c, src/getchar.c, src/normal.c, src/os_unix.c, src/search.c, src/term.c

Patch 8.0.1754
问题: ex_helpgrep() is too long.
解决方案: Refactor the function. (Yegappan Lakshmanan, closes #2766)
相关文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1755
问题: MS-Windows GUI: high unicode char received as two utf-16 words.
解决方案: Keep the first word until the second word is received. (Chris Morgan, closes #2800)
相关文件: src/gui_w32.c

Patch 8.0.1756
问题: GUI: after prompting for a number the mouse shape is sometimes wrong.
解决方案: Call setmouse() after setting "State". (Hirohito Higashi, closes #2709)
相关文件: src/misc1.c

Patch 8.0.1757
问题: Unnecessary changes in libvterm.
解决方案: Bring back // comments and trailing comma in enums.
相关文件: src/libvterm/bin/unterm.c, src/libvterm/bin/vterm-ctrl.c,
src/libvterm/bin/vterm-dump.c, src/libvterm/include/vterm.h,
src/libvterm/include/vterm_keycodes.h,
src/libvterm/src/encoding.c, src/libvterm/src/keyboard.c,
src/libvterm/src/parser.c, src/libvterm/src/pen.c,
src/libvterm/src/screen.c, src/libvterm/src/state.c,
src/libvterm/src/unicode.c, src/libvterm/src/utf8.h,
src/libvterm/src/vterm.c, src/libvterm/src/vterm_internal.h

Patch 8.0.1758
问题: open_line() returns TRUE/FALSE for success/failure.
解决方案: Return OK or FAIL.
相关文件: src/misc1.c, src/normal.c, src/edit.c

Patch 8.0.1759
问题: Memory leak from duplicate options. (Yegappan Lakshmanan)
解决方案: Don't set the default value twice.
相关文件: src/option.c

Patch 8.0.1760
问题: Wrong number of arguments to vms_read().
解决方案: Drop the first argument. (Ozaki Kiichi)
相关文件: src/ui.c

Patch 8.0.1761
问题: Job in terminal window with no output channel is killed.
解决方案: Keep the job running when the input is a tty. (Ozaki Kiichi,
closes #2734)
相关文件: src/channel.c, src/os_unix.c, src/testdir/test_channel.vim

Patch 8.0.1762
问题: Terminal debug logging is a bit complicated.
解决方案: Make log_tr() use variable arguments (Ozaki Kiichi, closes #2730)
相关文件: src/term.c

Patch 8.0.1763
问题: :argedit does not reuse an empty unnamed buffer.
解决方案: Add the BLN_CURBUF flag and fix all the side effects. (Christian
Brabandt, closes #2713)
相关文件: src/buffer.c, src/ex_cmds2.c, src/proto/buffer.pro,
src/testdir/test_arglist.vim, src/testdir/test_command_count.vim

Patch 8.0.1764
问题: Lgtm considers tutor.es to be EcmaScript.
解决方案: Add a config file for lgtm. (Bas van Schaik, closes #2844)
相关文件: .lgtm.yml, Filelist

Patch 8.0.1765
问题: CTRL-G j in Insert mode is incorrect when 'virtualedit' is set.
解决方案: Take coladd into account. (Christian Brabandt, closes #2743)
相关文件: src/charset.c, src/testdir/test_virtualedit.vim

Patch 8.0.1766 (after 8.0.1758)
问题: Expanding abbreviation doesn't work. (Tooth Pik)
解决方案: Return OK instead of FALSE and FAIL instead of TRUE. (Christian Brabandt)
相关文件: src/edit.c, src/testdir/test_mapping.vim

Patch 8.0.1767
问题: With 'incsearch' text may jump up and down. ()
解决方案: Besides w_botline also save and restore w_empty_rows. (closes #2530)
相关文件: src/ex_getln.c, src/testdir/test_search.vim, src/testdir/dumps/Test_incsearch_scrolling_01.dump

Patch 8.0.1768
问题: SET_NO_HLSEARCH() used in a wrong way.
解决方案: Make it a function. (suggested by Dominique Pelle, closes #2850)
相关文件: src/vim.h, src/ex_docmd.c, src/proto/ex_docmd.pro, src/search.c, src/ex_getln.c, src/option.c, src/screen.c, src/tag.c

Patch 8.0.1769
问题: Repeated saving and restoring viewstate for 'incsearch'.
解决方案: Use a structure.
相关文件: src/ex_getln.c

Patch 8.0.1770
问题: Assert functions don't return anything.
解决方案: Return non-zero when the assertion fails.
相关文件: src/evalfunc.c, src/eval.c, src/proto/eval.pro, src/testdir/test_assert.vim, runtime/doc/eval.txt

Patch 8.0.1771
问题: In tests, when WaitFor() fails it doesn't say why. (James McCoy)
解决方案: Add WaitForAssert(), which produces an assert error when it fails.
相关文件: src/testdir/shared.vim, src/testdir/test_terminal.vim, src/testdir/screendump.vim, src/testdir/test_autocmd.vim, src/testdir/test_channel.vim, src/testdir/test_clientserver.vim, src/testdir/test_job_fails.vim

Patch 8.0.1772
问题: Quickfix: mixup of FALSE and FAIL, returning -1.
解决方案: Use FAIL and INVALID_QFIDX. (Yegappan Lakshmanan)
相关文件: src/quickfix.c

Patch 8.0.1773
问题: Dialog messages are not translated.
解决方案: Add N_() and _() where needed. (Sergey Alyoshin)
相关文件: src/diff.c, src/ex_cmds2.c, src/ex_docmd.c, src/message.c, src/po/Make_cyg.mak, src/po/Make_ming.mak, src/po/Make_mvc.mak, src/po/Makefile, src/quickfix.c, src/vim.h

Patch 8.0.1774
问题: Reading very long lines can be slow.

解决方案: Read up to 1 Mbyte at a time to avoid a lot of copying. Add a
           check for going over the column limit.
相关文件: src/fileio.c

Patch 8.0.1775
问题: MS-Windows: warning for unused variable.
解决方案: Move declaration inside #ifdef. (Mike Williams)
相关文件: src/channel.c

Patch 8.0.1776
问题: In tests, when WaitFor() fails it doesn't say why.
解决方案: Turn a few more WaitFor() into WaitForAssert().
相关文件: src/testdir/test_popup.vim, src/testdir/test_quotestar.vim,
           src/testdir/test_search.vim, src/testdir/test_terminal.vim,
           src/testdir/test_timers.vim

Patch 8.0.1777
问题: Cannot cleanup before loading another colorscheme.
解决方案: Add the ColorSchemePre autocommand event.
相关文件: src/fileio.c, src/syntax.c, src/vim.h, src/testdir/test_gui.vim,
           runtime/colors/README.txt

Patch 8.0.1778
问题: Script to check translations does not always work.
解决方案: Go to first line before searching for MIME.
相关文件: src/po/check.vim

Patch 8.0.1779
问题: Deleting in a block selection causes problems.
解决方案: Check the length of the line before adding bd.textcol and
           bd.textlen. (Christian Brabandt, closes #2825)
相关文件: src/ops.c, src/testdir/test_blockedit.vim

Patch 8.0.1780
问题: Test fails because Vim in a terminal uses wrong 'encoding'.
解决方案: Set encoding in the test where it matters. (James McCoy,
           closes #2847)
相关文件: src/testdir/test_terminal.vim

Patch 8.0.1781
问题: File names in quickfix window are not always shortened.
解决方案: Shorten the file name when opening the quickfix window. (Yegappan
           Lakshmanan, closes #2851, closes #2846)
相关文件: src/testdir/test_quickfix.vim, src/fileio.c, src/proto/fileio.pro,
           src/quickfix.c

Patch 8.0.1782
问题: No simple way to label quickfix entries.
解决方案: Add the "module" item, to be used instead of the file name for
           display purposes. (Marcin Szamotulski, closes #1757)
相关文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/alloc.h,
           src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1783

问题:　　Cannot use 256 colors in a MS-Windows console.
解决方案:　Add 256 color support. (Nobuhiro Takasaki, closes #2821)
相关文件:　src/misc1.c, src/option.c, src/os_win32.c, src/proto/os_win32.pro,
　　src/term.c, src/proto/term.pro, src/terminal.c

Patch 8.0.1784 (after 8.0.1782)
问题:　　Gvim test gets stuck in dialog.
解决方案:　Rename the file used.
相关文件:　src/testdir/test_quickfix.vim

Patch 8.0.1785 (after 8.0.1783)
问题:　　Missing symbol in Win32 small build.
解决方案:　Define VTERM_ANSI_INDEX_NONE without the terminal feature. Also
　　fix unused function with #ifdef.
相关文件:　src/term.c, src/os_win32.c

Patch 8.0.1786
问题:　　No test for 'termwinkey'.
解决方案:　Add a test. Make feedkeys() handle terminal_loop() returning
　　before characters are consumed.
相关文件:　src/testdir/test_terminal.vim, src/terminal.c, src/evalfunc.c,
　　src/ex_docmd.c, src/getchar.c, src/keymap.h

Patch 8.0.1787
问题:　　Cannot insert the whole cursor line.
解决方案:　Make CTRL-R CTRL-L work. (Andy Massimino, closes #2857)
相关文件:　runtime/doc/cmdline.txt, src/ex_getln.c, src/ops.c,
　　src/testdir/test_cmdline.vim

Patch 8.0.1788
问题:　　Tool to check a color scheme is not installed.
解决方案:　Update the install rule. (Christian Brabandt)
相关文件:　src/Makefile

Patch 8.0.1789
问题:　　BufWinEnter does not work well for a terminal window.
解决方案:　Do not trigger BufWinEnter when opening a terminal window.
相关文件:　src/terminal.c, runtime/doc/autocmd.txt,
　　src/testdir/test_terminal.vim

Patch 8.0.1790
问题:　　'winfixwidth' is not always respected by :close.
解决方案:　Prefer a frame without 'winfixwidth' or 'winfixheight'. (Jason
　　Franklin)
相关文件:　src/window.c, src/testdir/test_winbuf_close.vim

Patch 8.0.1791
问题:　　Using uint8_t does not work everywhere.
解决方案:　Use char_u instead.
相关文件:　src/term.c, src/proto/term.pro, src/os_win32.c

Patch 8.0.1792
问题:　　MS-Windows users expect -? to work like --help.
解决方案:　Add -?. (Christian Brabandt, closes #2867)

相关文件: src/main.c

Patch 8.0.1793
问题: No test for "vim -g".
解决方案: Add a test for "-g" and "-y".
相关文件: src/testdir/shared.vim, src/testdir/test_gui.vim

Patch 8.0.1794
问题: Duplicate term options after renaming.
解决方案: Remove the old names 'termkey', 'termsize' and 'terminalscroll'.
相关文件: src/option.c, src/terminal.c, src/option.h,
src/testdir/gen_opt_test.vim, src/testdir/screendump.vim

Patch 8.0.1795
问题: Lose contact with jobs when :gui forks.
解决方案: Don't fork when there is a running job. Make log message for a
died job clearer. Also close the terminal when stderr and stdout
are the same FD.
相关文件: src/gui.h, src/gui.c, src/channel.c, src/proto/channel.pro,
src/os_unix.c, src/terminal.c

Patch 8.0.1796
问题: GUI: click on tab fails when the focus is in a terminal window.
解决方案: Handle K_TABLINE.
相关文件: src/terminal.c

Patch 8.0.1797
问题: Terminal window is redrawn too often and scrolling is repeated.
解决方案: Don't scroll immediately but only when redrawing. Avoid redrawing
the whole terminal window on every change.
相关文件: src/terminal.c, src/screen.c, src/proto/terminal.pro

Patch 8.0.1798
问题: MS-Windows: file considered read-only when another program has
opened it.
解决方案: Pass file sharing flag to CreateFile(). (Linwei, closes #2860)
相关文件: src/os_win32.c

Patch 8.0.1799
问题: No test for :registers command.
解决方案: Add a test. (Dominique Pelle, closes #2880)
相关文件: src/testdir/test_registers.vim

Patch 8.0.1800
问题: X11: getting color is slow.
解决方案: Avoid using sprintf() and XParseColor(), put the RGB values in
XColor directly.
相关文件: src/gui_x11.c

Patch 8.0.1801
问题: MS-Windows: redirecting terminal output does not work.
解决方案: Intercept the text written to the terminal and write it to the
file.
相关文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1802 (after 8.0.1802)
问题: MS-Windows: terminal test fails.
解决方案: Close redirected output file earlier.
相关文件: src/terminal.c

Patch 8.0.1803
问题: Warning for uninitialized variable. (Tony Mechelynck)
解决方案: Initialize it.
相关文件: src/terminal.c

Patch 8.0.1804
问题: Using :normal in terminal window causes problems. (Dominique Pelle)
解决方案: Don't call terminal_loop() for :normal. (closes #2886)
相关文件: src/ex_docmd.c, src/proto/ex_docmd.pro, src/evalfunc.c

Patch 8.0.1805
问题: qf_parse_line() is too long.
解决方案: Split it in parts. Properly handle vim_realloc() failing. (Yegappan Lakshmanan, closes #2881)
相关文件: src/quickfix.c

Patch 8.0.1806
问题: InsertCharPre causes problems for autocomplete. (Lifepillar)
解决方案: Check for InsertCharPre before calling vpeekc(). (Christian Brabandt, closes #2876)
相关文件: src/edit.c, src/testdir/test_popup.vim

Patch 8.0.1807
问题: Function to set terminal name is too long.
解决方案: Refactor the function. Fix typo in test.
相关文件: src/term.c, src/testdir/test_options.vim

Patch 8.0.1808 (after 8.0.1807)
问题: Can't build without TGETENT.
解决方案: Add #ifdef
相关文件: src/term.c

Patch 8.0.1809
问题: Various typos.
解决方案: Correct the mistakes, change "cursur" to "cursor". (closes #2887)
相关文件: src/edit.c, src/normal.c, src/screen.c, src/proto/screen.pro, src/ui.c

Patch 8.0.1810
问题: Buffer of a terminal only updated in Terminal-Normal mode.
解决方案: Copy the terminal window content to the buffer when in Terminal-Job mode.
相关文件: src/terminal.c, src/proto/terminal.pro, src/ex_cmds2.c, src/proto/ex_cmds2.pro

Patch 8.0.1811
问题: No test for winrestcmd().

解决方案: Add a test. (Dominique Pelle, closes #2894)
相关文件: src/testdir/test_window_cmd.vim

Patch 8.0.1812
问题: The qf_jump_to_usable_window() function is too long.
解决方案: Split it in parts. (Yegappan Lakshmanan, closes #2891)
相关文件: src/quickfix.c

Patch 8.0.1813
问题: Windows installer doesn't install terminal debugger.
解决方案: Add the package to the list of files to install.
相关文件: nsis/gvim.nsi

Patch 8.0.1814
问题: Crash with terminal window and with 'lazyredraw' set. (Antoine)
解决方案: Check the terminal still exists after update_screen().
相关文件: src/terminal.c

Patch 8.0.1815 (after 8.0.1814)
问题: Still a crash with terminal window and with 'lazyredraw' set.
(Antoine)
解决方案: Do not wipe out the buffer when updating the screen.
相关文件: src/terminal.c, src/proto/terminal.pro, src/screen.c,
src/proto/screen.pro, src/ui.c

Patch 8.0.1816
问题: No test for setcmdpos().
解决方案: Add a test. (Dominique Pelle, closes #2901)
相关文件: src/testdir/test_cmdline.vim

Patch 8.0.1817
问题: A timer may change v:count unexpectedly.
解决方案: Save and restore v:count and similar variables when a timer
callback is invoked. (closes #2897)
相关文件: src/eval.c, src/proto/eval.pro, src/ex_cmds2.c, src/structs.h,
src/testdir/test_timers.vim

Patch 8.0.1818 (after 8.0.1810)
问题: Lines remove from wrong buffer when using terminal window.
解决方案: Make sure to use tl_buffer.
相关文件: src/terminal.c

Patch 8.0.1819
问题: Swap file warning for a file in a non-existing directory, if there
is another with the same file name. (Juergen Weigert)
解决方案: When expanding the file name fails compare the file names.
相关文件: src/testdir/test_swap.vim, src/memline.c

Patch 8.0.1820
问题: Terminal window redirecting stdout does not show stderr. (Matéo
Zanibelli)
解决方案: When stdout is not connected to pty_master_fd then use it for
stderr. (closes #2903)
相关文件: src/os_unix.c, src/testdir/test_terminal.vim

Patch 8.0.1821
问题:　　Cursor in terminal window moves when pressing CTRL-W. (Dominique Pelle)
解决方案:　Do not more the cursor or redraw when not in Terminal-Normal mode. (closes #2904)
相关文件:　src/terminal.c

Patch 8.0.1822
问题:　　Make uninstall does not remove colors/tools.
解决方案:　Add a line to delete the tools directory. (Kazunobu Kuriyama)
相关文件:　src/Makefile

Patch 8.0.1823
问题:　　Test for terminal stdout redirection is flaky.
解决方案:　Wait for the job to finish.
相关文件:　src/testdir/test_terminal.vim

Patch 8.0.1824
问题:　　Coverity warns for variable that may be uninitialized.
解决方案:　Initialize the variable.
相关文件:　src/terminal.c

Patch 8.0.1825
问题:　　Might use NULL pointer when out of memory. (Coverity)
解决方案:　Handle NULL pointer better.
相关文件:　src/getchar.c

Patch 8.0.1826
问题:　　Configure uses old compiler flag.
解决方案:　Remove _DARWIN_C_SOURCE. (Kazunobu Kuriyama)
相关文件:　src/configure.ac, src/auto/configure

Patch 8.0.1827
问题:　　Compiler warning for signed/unsigned char pointers. (Cesar Romani)
解决方案:　Change the type of jv_argv.
相关文件:　src/channel.c, src/structs.h

Patch 8.0.1828
问题:　　Get no clue why :gui does not fork.
解决方案:　Add a channel log message.
相关文件:　src/channel.c

Patch 8.0.1829
问题:　　MS-Windows: script for vimdiff can't handle ! chars.
解决方案:　Escape the ! chars. (Hans Ginzel, closes #2896)
相关文件:　src/dosinst.c

Patch 8.0.1830
问题:　　Switching to Terminal-Normal mode does not redraw. (Dominique Pelle)
解决方案:　Also redraw when not updating the snapshot. (closes #2904)
相关文件:　src/terminal.c

Patch 8.0.1831
问题: Sometimes the quickfix title is incorrectly prefixed with ':'.
解决方案: Prepend the colon in another way. (Yegappan Lakshmanan, closes #2905)
相关文件: src/evalfunc.c, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.0.1832
问题: Cannot use :unlet for an environment variable.
解决方案: Make it work. Use unsetenv() if available. (Yasuhiro Matsumoto, closes #2855)
相关文件: runtime/doc/eval.txt, src/config.h.in, src/configure.ac, src/auto/configure, src/eval.c, src/misc1.c, src/proto/misc1.pro, src/testdir/test_unlet.vim

Patch 8.0.1833
问题: X11: ":echo 3.14" gives E806.
解决方案: set LC_NUMERIC to "C". (Dominique Pelle, closes #2368)
相关文件: src/gui_x11.c

Patch 8.0.1834
问题: GUI: find/replace dialog does not handle some chars properly.
解决方案: Escape '?' when needed. Always escape backslash. (closes #2418, closes #2435)
相关文件: src/gui.c

Patch 8.0.1835
问题: Print document name does not support multi-byte.
解决方案: Use StartDocW() if needed. (Yasuhiro Matsumoto, closes #2478)
相关文件: src/os_mswin.c

Patch 8.0.1836
问题: Buffer-local window options may not be recent if the buffer is still open in another window.
解决方案: Copy the options from the window instead of the outdated window options. (Bjorn Linse, closes #2336)
相关文件: src/buffer.c, src/testdir/test_options.vim

Patch 8.0.1837
问题: One character cmdline abbreviation not triggered after '<,'>.
解决方案: Skip over the special range. (Christian Brabandt, closes #2320)
相关文件: src/ex_getln.c, src/testdir/test_mapping.vim

Patch 8.0.1838
问题: Cursor in wrong position when switching to Terminal-Normal mode. (Dominique Pelle)
解决方案: Move to the end of the line if coladvance() fails. Do not take a snapshot a second time.
相关文件: src/terminal.c

Patch 8.0.1839
问题: Script to check .po file doesn't check for plural header.
解决方案: Add a check that the plural header is present when needed.
相关文件: src/po/check.vim

Patch 8.0.1840
问题:　　getwinpos() is not tested.
解决方案:　Add a test. (Dominique Pelle, closes #2911)
相关文件:　src/testdir/test_gui.vim

Patch 8.0.1841
问题:　　HP-UX does not have setenv().
解决方案:　Use vim_setenv(). (John Marriott)
相关文件:　src/misc1.c

Patch 8.0.1842
问题:　　Popup menu inside terminal window isn't cleared.
解决方案:　Use NOT_VALID in pum_undisplay(). (suggested by Christian
　　Brabandt, closes #2908)
相关文件:　src/popupmnu.c

Patch 8.0.1843
问题:　　Entry for 'wrap' in options window is wrong. (John Little)
解决方案:　Make the change apply locally.
相关文件:　runtime/optwin.vim

Patch 8.0.1844
问题:　　Superfluous quickfix code, missing examples.
解决方案:　Remove unneeded code. Add a few examples. Add a bit more
　　testing. (Yegappan Lakshmanan, closes #2916)
相关文件:　runtime/doc/quickfix.txt, src/quickfix.c,
　　src/testdir/test_quickfix.vim

Patch 8.0.1845
问题:　　Various comment updates needed, missing white space.
解决方案:　Update comments, add white space.
相关文件:　src/getchar.c, src/testdir/test_cscope.vim, src/gui_mac.c

Patch 8.0.1846
问题:　　Python interface is incompatible with lldb.
解决方案:　For OutputType set the base to be PyFile_Type. (Boxu Zhang)
　　Partly disabled to avoid a crash.
相关文件:　src/if_py_both.h, src/if_python.c, src/if_python3.c

Patch 8.0.1847
问题:　　Some build options don't have an example.
解决方案:　Add a couple more examples and compiler flags.
相关文件:　src/Makefile

Patch 8.0.1848
问题:　　'termwinscroll' does not work properly. (Dominique Pelle)
解决方案:　Subtract removed scrollback from the scrollback count. Add a test
　　for 'termwinscroll'. (closes #2909)
相关文件:　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.0.1849
问题:　　Compiler warning for unused arguments and missing prototype.
解决方案:　Add UNUSED. Add static.
相关文件:　src/mbyte.c, src/if_ruby.c

Patch 8.0.1850
问题:　　Todo items in source code not visible for users.
解决方案:　Move the todo items to the help file.
相关文件:　src/terminal.c

8.2 版 本　　version-8.2　version8.2　vim-8.2

本小节讨论 8.1 和 8.2 版本之间的改进。

这个版本有几百个漏洞修正，若干新特性和许多次要的改进。

弹出窗口　　new-popup-window

弹出窗口可用来在其它窗口之上显示文本。可以是简单的信息，如 "Build finished successfully"，编辑函数调用时函数的原型，可变的弹出菜单，还有许多其它用途。见 popup-window 。

弹出窗口非常灵活：定位可以相对于文本，绝对位置或在屏幕的中间。大小可以固定也可以进行调整以放进文本。"zindex" 值指定弹出窗口之间谁在谁之上。

新的 'wincolor' 选项可为整个弹出窗口设置颜色。也可用于普通窗口。

文本属性　　new-text-properties

文本属性给插件作者如何高亮的灵活性。可用于外部的处理语法高亮的异步解析器。或者在弹出窗口中高亮文本。删除或插入字符时，文本属性与文本捆绑，这使它们可用于文本标记。见 text-properties 。

加入监听器函数，用来向服务器报告文本的改动，以便服务器能动态地刷新高亮，标记语法错误，诸如此类。见 listener_add() 。

Vim 脚本改进　　new-vimscript-8.2

函数现在可以用 "->" 进行链式调用：

```
mylist->filter(filterexpr)->map(mapexpr)->sort()->join()
```

如果调用链没有结果，可用新的 :eval 命令。

加入 :scriptversion 命令以支持后向不兼容的改动。比如，仅支持 ".." 来连接字符串，从而可用 "." 来更统一地访问字典成员。

加入 :const 以支持不能改变的变量的声明：

```
const TIMER_DELAY = 400
```

加入 heredoc-风格的赋值，从而能方便地把行的列表赋给变量，而无需引号或继续行：

```
let lines =<< trim END
   line one
   line two
END
```

加入 Blob 类型。可以方便地处理二进制数据。

加入 /= 和 %= 赋值操作符。

字典可以用 #{} 来定义按本义出现的键。这避免了许多引号的使用：

```
let options = #{width: 30, height: 24}
```

其它改进 new-other-8.2

- 'incsearch' 的置位也应用于 :substitute 。
- 加入 modifyOtherKeys 以支持更多键组合的映射。
- 加入 Windows 10 的 ConPTY 支持，支持终端的全色彩。
- MS-Windows 安装器支持翻译，安静安装，外观也好很多。

改动 changed-8.2

包含了 xdiff 库，避免了外部 diff 程序的需要，并使差异的更新快许多。

代码使用了更多现代的 C 特性，如 // 注释。

放弃了旧编译器的支持：Borland C++、MSVC 2008。

删除 Hangul 输入支持，实际上已经不能用了。

删除旧的 Amiga 编译器的 Makefiles：Dice、Manx 和 SAS。

如果找到没有任何改动的交换文件，会被自动删除。

删除了 FEAT_TAG_OLDSTATIC 代码，它减慢了标签的搜索。
删除了 FEAT_TAG_ANYWHITE 代码，它在任何编译版本中都没有打开。
删除了 UNICODE16 代码，没有用。
删除了 Workshop 支持，没人再用了。
删除了 Aap build 文件，过时了。
删除了波斯语支持。过时且没人用。

重新实现了 VIMDLL，它共享了 vim 和 gvim 的共用部分，以减少总安装大小。

现在在所有版本上都包含以下特性： +multi_byte 、 +virtualedit 、 +vreplace 、 +localmap 、 +cmdline_hist 、 +cmdline_compl 、 +insert_expand 、 +modify_fname 、 +comments

新增 added-8.2

新增函数:

所有的 popup_ 函数。
所有的 prop_ 函数。
所有的 sign_ 函数。
所有的 sound_ 函数。

appendbufline()
balloon_gettext()
bufadd()
bufload()
ch_readblob()
chdir()
debugbreak()
deletebufline()
environ()
expandcmd()
getenv()
getimstatus()
getmousepos()
gettagstack()
interrupt()
isinf()
list2str()
listener_add()
listener_flush()
listener_remove()
prompt_setcallback()
prompt_setinterrupt()
prompt_setprompt()
pum_getpos()
rand()
readdir()
reg_executing()
reg_recording()
rubyeval()
screenchars()
screenpos()
screenstring()
setenv()
settagstack()
srand()
state()
str2list()
strptime()
swapinfo()
swapname()
term_setapi()
test_getvalue()
test_null_blob()
test_refcount()
test_scrollbar()
test_setmouse()
win_execute()

win_splitmove()
winlayout()

新增自动命令：
CompleteChanged
DiffUpdated
SafeState
SafeStateAgain
SourcePost
TerminalWinOpen

新增命令：
跳转到相对于光标位置的错误：
:cabove
:cafter
:cbefore
:cbelow
:labove
:lbefore
:lbelow
:lafter
局部于标签页的目录：
:tcd
:tchdir
其它：
:const
:eval
:redrawtabline
:scriptversion
:spellrare
:tlmenu
:tlnoremenu
:tlunmenu
:wsverb
:xrestore

新增选项：
'completepopup'
'completeslash'
'cursorlineopt'
'modelineexpr'
'previewpopup'
'scrollfocus'
'tagfunc'
'termwintype'
'varsofttabstop'
'vartabstop'
'wincolor'

补丁 patches-8.2

这些补丁在 8.1 发布后被加入，在 8.2 中发布。

Patch 8.1.0001
问题: The netrw plugin does not work.
解决方案: Make it accept version 8.x.
文件: runtime/autoload/netrw.vim

Patch 8.1.0002
问题: :stopinsert changes the message position.
解决方案: Save and restore msg_col and msg_row in clearmode(). (Jason
            Franklin)
文件: src/screen.c, src/testdir/test_messages.vim

Patch 8.1.0003
问题: The :compiler command is not tested.
解决方案: Add a test. (Dominique Pelle, closes #2930)
文件: src/Makefile, src/testdir/test_alot.vim,
            src/testdir/test_compiler.vim

Patch 8.1.0004
问题: Test for :compiler command sometimes fails.
解决方案: Be less strict about the error message. (Dominique Pelle)
文件: src/testdir/test_compiler.vim

Patch 8.1.0005
问题: Test for :compiler command fails on MS-Windows.
解决方案: Ignore difference in path.
文件: src/testdir/test_compiler.vim

Patch 8.1.0006
问题: syn_id2cterm_bg() may be undefined. (Axel Bender)
解决方案: Adjust #ifdef.
文件: src/syntax.c

Patch 8.1.0007
问题: No test for "o" and "O" in Visual block mode.
解决方案: Add a test. (Dominique Pelle, closes #2932)
文件: src/testdir/test_visual.vim

Patch 8.1.0008
问题: No test for strwidth().
解决方案: Add a test. (Dominique Pelle, closes #2931)
文件: src/testdir/test_functions.vim

Patch 8.1.0009
问题: Tabpages insufficiently tested.
解决方案: Add more test coverage. (Dominique Pelle, closes #2934)
文件: src/testdir/test_tabpage.vim

Patch 8.1.0010
问题: efm_to_regpat() is too long.
解决方案: Split off three functions. (Yegappan Lakshmanan, closes #2924)
文件: src/quickfix.c

Patch 8.1.0011

问题: maparg() and mapcheck() confuse empty and non-existing.
解决方案: Return <Nop> for an existing non-empty mapping. (closes #2940)
文件: src/evalfunc.c, src/testdir/test_maparg.vim

Patch 8.1.0012
问题: Misplaced #endif.
解决方案: Move the #endif to after the expression. (David Binderman)
文件: src/fileio.c

Patch 8.1.0013
问题: Using freed memory when changing terminal cursor color.
解决方案: Make a copy of the color. (Dominique Pelle, closes #2938,
            closes #2941)
文件: src/terminal.c

Patch 8.1.0014
问题: qf_init_ext() is too long.
解决方案: Split it into multiple functions. (Yegappan Lakshmanan,
            closes #2939)
文件: src/quickfix.c

Patch 8.1.0015
问题: Cursor color wrong when closing a terminal window, ending up in
            another terminal window. (Dominique Pelle)
解决方案: Bail out of terminal_loop() when the buffer changes.
            (closes #2942)
文件: src/terminal.c

Patch 8.1.0016
问题: Possible crash in term_wait(). (Dominique Pelle)
解决方案: Check for a valid buffer after ui_delay(). (closes #2944)
文件: src/terminal.c

Patch 8.1.0017
问题: Shell command completion has duplicates. (Yegappan Lakshmanan)
解决方案: Use a hash table to avoid duplicates. (Ozaki Kiichi, closes #539,
            closes #2733)
文件: src/ex_getln.c, src/testdir/test_cmdline.vim

Patch 8.1.0018
问题: Using "gn" may select wrong text when wrapping.
解决方案: Avoid wrapping when searching forward. (Christian Brabandt)
文件: src/search.c, src/testdir/test_gn.vim

Patch 8.1.0019
问题: Error when defining a Lambda with index of a function result.
解决方案: When not evaluating an expression and skipping a function call,
            set the return value to VAR_UNKNOWN.
文件: src/userfunc.c, src/testdir/test_lambda.vim

Patch 8.1.0020
问题: Cannot tell whether a register is being used for executing or
            recording.
解决方案: Add reg_executing() and reg_recording(). (Hirohito Higashi,

closes #2745) Rename the global variables for consistency. Store the register name in reg_executing.
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c, src/testdir/test_functions.vim, src/getchar.c, src/normal.c, src/ops.c, src/globals.h, src/edit.c, src/fileio.c, src/message.c, src/screen.c

Patch 8.1.0021
问题: Clang warns for undefined behavior.
解决方案: Move #ifdef outside of sprintf() call. (suggestion by Michael Jarvis, closes #2946)
文件: src/term.c

Patch 8.1.0022
问题: Repeating put from expression register fails.
解决方案: Re-evaluate the expression register. (Andy Massimino, closes #2945)
文件: src/getchar.c, src/testdir/test_put.vim

Patch 8.1.0023
问题: gcc 8.1 warns for use of strncpy(). (John Marriott)
解决方案: Use mch_memmove() instead of STRNCPY().
文件: src/memline.c

Patch 8.1.0024
问题: % command not tested on #ifdef and comment.
解决方案: Add tests. (Dominique Pelle, closes #2956)
文件: src/testdir/test_goto.vim

Patch 8.1.0025
问题: No test for the undofile() function.
解决方案: Add test. (Dominique Pelle, closes #2958)
文件: src/testdir/test_undo.vim

Patch 8.1.0026
问题: Terminal test fails with very tall terminal. (Tom)
解决方案: Fix the terminal window size in the test.
文件: src/testdir/test_terminal.vim

Patch 8.1.0027
问题: Difficult to make a plugin that feeds a line to a job.
解决方案: Add the initial code for the "prompt" buftype.
文件: runtime/doc/channel.txt, runtime/doc/eval.txt, runtime/doc/options.txt, runtime/doc/tags, runtime/doc/todo.txt, src/Makefile, src/buffer.c, src/channel.c, src/diff.c, src/edit.c, src/evalfunc.c, src/normal.c, src/ops.c, src/option.c, src/proto/buffer.pro, src/proto/channel.pro, src/proto/edit.pro, src/proto/ops.pro, src/structs.h, src/testdir/Make_all.mak, src/testdir/screendump.vim, src/testdir/test_prompt_buffer.vim

Patch 8.1.0028 (after 8.1.0027)
问题: Prompt buffer test fails on MS-Windows.
解决方案: Disable the test for now. Remove stray assert.
文件: src/testdir/test_prompt_buffer.vim

Patch 8.1.0029
问题:　　Terminal test fails on MS-Windows when "wc" exists.
解决方案:　Skip test with redirection on MS-Windows.
文件:　　src/testdir/test_terminal.vim

Patch 8.1.0030
问题:　　Stopping Vim running in a terminal may not work.
解决方案:　Instead of sending <Esc> send CTRL-O.
文件:　　src/testdir/screendump.vim, src/testdir/test_prompt_buffer.vim

Patch 8.1.0031
问题:　　Terminal test aucmd_on_close is flaky.
解决方案:　Wait a bit longer.
文件:　　src/testdir/test_terminal.vim

Patch 8.1.0032
问题:　　BS in prompt buffer starts new line.
解决方案:　Do not allow BS over the prompt. Make term_sendkeys() handle
　　special keys. Add a test.
文件:　　src/option.c, src/terminal.c, src/testdir/test_prompt_buffer.vim

Patch 8.1.0033
问题:　　Keys to stop Vim in terminal are wrong. (Marius Gedminas)
解决方案:　Move ":" to before CTRL-U.
文件:　　src/testdir/screendump.vim

Patch 8.1.0034
问题:　　Cursor not restored with ":edit #".
解决方案:　Don't assume autocommands moved the cursor when it was moved to
　　the first non-blank.
文件:　　src/ex_cmds.c, src/testdir/test_edit.vim

Patch 8.1.0035
问题:　　Not easy to switch between prompt buffer and other windows.
解决方案:　Accept CTRL-W commands in Insert mode. Start and stop Insert mode
　　as one would expect.
文件:　　src/edit.c, src/ex_docmd.c, src/structs.h, src/window.c

Patch 8.1.0036
问题:　　Not restoring Insert mode if leaving a prompt buffer by using a
　　mouse click.
解决方案:　Set b_prompt_insert appropriately. Also correct cursor position
　　when moving cursor to last line.
文件:　　src/buffer.c, src/edit.c, src/window.c

Patch 8.1.0037
问题:　　Cannot easily append lines to another buffer.
解决方案:　Add appendbufline().
文件:　　runtime/doc/eval.txt, src/evalfunc.c,
　　src/testdir/test_bufline.vim, src/testdir/test_edit.vim

Patch 8.1.0038
问题:　　Popup test causes Vim to exit.

解决方案: Disable the broken part of the test for now.
文件: src/testdir/test_popup.vim

Patch 8.1.0039
问题: Cannot easily delete lines in another buffer.
解决方案: Add deletebufline().
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_bufline.vim

Patch 8.1.0040
问题: Warnings from 64-bit compiler.
解决方案: Add type casts. (Mike Williams)
文件: src/edit.c

Patch 8.1.0041
问题: Attribute "width" missing from python window attribute list.
解决方案: Add the item. (Ken Takata) Order the list like the items are used
           in the WindowAttr() function.
文件: src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok

Patch 8.1.0042
问题: If omni completion opens a window Insert mode is stopped.
           (Hirohito Higashi)
解决方案: Only set stop_insert_mode in a prompt buffer window.
文件: src/window.c

Patch 8.1.0043
问题: ++bad argument of :edit does not work properly.
解决方案: Return FAIL from get_bad_opt() only when there is no valid
           argument. (Dominique Pelle, Christian Brabandt, closes #2966,
           closes #2947)
文件: src/ex_docmd.c, src/testdir/test_plus_arg_edit.vim

Patch 8.1.0044
问题: If a test function exits Vim this may go unnoticed.
解决方案: Check for a test function quitting Vim. Fix tests that did exit
           Vim.
文件: src/testdir/runtest.vim, src/testdir/test_assert.vim

Patch 8.1.0045 (after 8.1.0038)
问题: Popup test isn't run completely.
解决方案: Remove "finish". Clean up function definitions.
文件: src/testdir/test_popup.vim

Patch 8.1.0046
问题: Loading a session file fails if 'winheight' is a big number.
解决方案: Set 'minwinheight' to zero at first. Don't give an error when
           setting 'minwinheight' while 'winheight' is a big number.
           Fix using vertical splits. Fix setting 'minwinwidth'.
           (closes #2970)
文件: src/testdir/test_mksession.vim, src/option.c, src/window.c,
           src/proto/window.pro

Patch 8.1.0047
问题: No completion for :unlet $VAR.

解决方案: Add completion. (Jason Franklin)
文件: src/ex_docmd.c, src/testdir/test_unlet.vim

Patch 8.1.0048
问题: vim_str2nr() does not handle numbers close to the maximum.
解决方案: Check for overflow more precisely. (Ken Takata, closes #2746)
文件: src/charset.c

Patch 8.1.0049
问题: Shell cannot tell running in a terminal window.
解决方案: Add the VIM_TERMINAL environment variable. (Christian Brabandt)
文件: runtime/doc/terminal.txt, src/os_unix.c, src/os_win32.c,
src/testdir/test_terminal.vim

Patch 8.1.0050 (after 8.1.0049)
问题: $VIM_TERMINAL is also set when not in a terminal window.
解决方案: Pass a flag to indicate whether the job runs in a terminal.
文件: src/channel.c, src/proto/channel.pro, src/evalfunc.c,
src/terminal.c, src/os_unix.c, src/proto/os_unix.pro,
src/os_win32.c

Patch 8.1.0051 (after 8.1.0050)
问题: MS-Windows: missing #endif.
解决方案: Add the #endif.
文件: src/os_win32.c

Patch 8.1.0052
问题: When a mapping to <Nop> times out the next mapping is skipped.
解决方案: Reset "timedout" when waiting for a character. (Christian
Brabandt, closes #2921)
文件: src/getchar.c

Patch 8.1.0053
问题: The first argument given to 'completefunc' can be Number or
String, depending on the value.
解决方案: Avoid guessing the type of an argument, use typval_T in the
callers of call_vim_function(). (Ozaki Kiichi, closes #2993)
文件: src/edit.c, src/eval.c, src/ex_getln.c, src/mbyte.c, src/normal.c,
src/proto/eval.pro, src/testdir/test_ins_complete.vim

Patch 8.1.0054
问题: Compiler warning for using %ld for "long long".
解决方案: Add a type cast. (closes #3002)
文件: src/os_unix.c

Patch 8.1.0055 (after 8.1.0053)
问题: Complete test has wrong order of arguments. Wrong type for
sentinel variable.
解决方案: Swap arguments, use VAR_UNKNOWN. (Ozaki Kiichi)
文件: src/mbyte.c, src/testdir/test_ins_complete.vim

Patch 8.1.0056
问题: Crash when using :hardcopy with illegal byte.
解决方案: Check for string_convert() returning NULL. (Dominique Pelle)

文件: src/hardcopy.c, src/testdir/test_hardcopy.vim

Patch 8.1.0057
问题: Popup menu displayed wrong when using autocmd.
解决方案: Use aucmd_prepbuf(). Force updating status line if the popup menu is going to be redrawn anyway. (Christian Brabandt, closes #3009)
文件: src/edit.c, src/screen.c, src/proto/screen.pro

Patch 8.1.0058
问题: Display problem with margins and scrolling.
解决方案: Place the cursor in the right column. (Kouichi Iwamoto, closes #3016)
文件: src/screen.c

Patch 8.1.0059
问题: Displayed digraph for "ga" wrong with 'encoding' "cp1251".
解决方案: Convert from 'encoding' to "utf-8" if needed. (closes #3015)
文件: src/digraph.c, src/testdir/test_digraph.vim

Patch 8.1.0060
问题: Crash when autocommands delete the current buffer. (Dominique Pelle)
解决方案: Check that autocommands don't change the buffer.
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0061
问题: Window title is wrong after resetting and setting 'title'.
解决方案: Move resetting the title into maketitle(). (Jason Franklin)
文件: src/option.c, src/buffer.c

Patch 8.1.0062
问题: Popup menu broken if a callback changes the window layout. (Qiming Zhao)
解决方案: Recompute the popup menu position if needed. Redraw the ruler even when the popup menu is displayed.
文件: src/popupmnu.c, src/proto/popupmnu.pro, src/screen.c

Patch 8.1.0063
问题: Mac: NSStringPboardType is deprecated.
解决方案: Use NSPasteboardTypeString. (Akshay Hegde, closes #3022)
文件: src/os_macosx.m

Patch 8.1.0064
问题: Typing CTRL-W in a prompt buffer shows mode "-- --".
解决方案: Set restart_edit to 'A' and check for it.
文件: src/edit.c, src/window.c, src/screen.c

Patch 8.1.0065 (after 8.1.0062)
问题: Balloon displayed at the wrong position.
解决方案: Do not reposition the popup menu at the cursor position.
文件: src/popupmnu.c

Patch 8.1.0066
问题: Nasty autocommand causes using freed memory. (Dominique Pelle)

解决方案: Do not force executing autocommands if the value of 'syntax' or
           'filetype' did not change.
文件: src/option.c

Patch 8.1.0067
问题: Syntax highlighting not working when re-entering a buffer.
解决方案: Do force executing autocommands when not called recursively.
文件: src/option.c

Patch 8.1.0068
问题: Nasty autocommands can still cause using freed memory.
解决方案: Disallow using setloclist() and setqflist() recursively.
文件: src/evalfunc.c

Patch 8.1.0069
问题: Cannot handle pressing CTRL-C in a prompt buffer.
解决方案: Add prompt_setinterrupt().
文件: runtime/doc/eval.txt, src/edit.c, src/evalfunc.c, src/channel.c,
           src/proto/channel.pro

Patch 8.1.0070
问题: Missing part of the changes for prompt_setinterrupt().
解决方案: Add the missing changes.
文件: src/structs.h

Patch 8.1.0071
问题: Terminal debugger only works with the terminal feature.
解决方案: Make it also work with a prompt buffer. Makes it possible to use
           on MS-Windows. Various other improvements. (closes #3012)
文件: runtime/doc/terminal.txt,
           runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0072
问题: Use of 'termwinkey' is inconsistent.
解决方案: Change the documentation and the behavior. (Ken Takata)
文件: src/terminal.c, runtime/doc/terminal.txt

Patch 8.1.0073
问题: Crash when autocommands call setloclist(). (Dominique Pelle)
解决方案: If the quickfix list changes then don't jump to the error.
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0074 (after 8.1.0073)
问题: Crash when running quickfix tests.
解决方案: Do not alloc a new location list when checking for the reference
           to be still valid.
文件: src/quickfix.c

Patch 8.1.0075
问题: No Vim logo in README file.
解决方案: Add one. (rni Dagur, closes #3024)
文件: README.md

Patch 8.1.0076

问题:　　Command getting cleared with CTRL-W : in a terminal window. (Jason Franklin)
解决方案:　Call redraw_after_callback() when editing the command line.
文件:　　src/terminal.c

Patch 8.1.0077
问题:　　Header of README file is not nice.
解决方案:　Move text to the bottom.
文件:　　 README.md

Patch 8.1.0078
问题:　　"..." used inconsistently in messages.
解决方案:　Drop the space before " ...".
文件:　　src/spellfile.c, src/regexp_nfa.c

Patch 8.1.0079
问题:　　Superfluous space in messages.
解决方案:　Remove the spaces. (closes #3030)
文件:　　src/gui_w32.c

Patch 8.1.0080
问题:　　Can't see the breakpoint number in the terminal debugger.
解决方案:　Use the breakpoint number for the sign. (Christian Brabandt)
文件:　　runtime/doc/terminal.txt,
　　runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0081
问题:　　The terminal debugger doesn't adjust to changed 'background'.
解决方案:　Add an OptionSet autocommand. (Christian Brabandt)
文件:　　runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0082
问题:　　In terminal window, typing : at more prompt, inserts ':' instead of starting another Ex command.
解决方案:　Add skip_term_loop and set it when putting ':' in the typeahead buffer.
文件:　　src/globals.h, src/main.c, src/message.c

Patch 8.1.0083
问题:　　"is" and "as" have trouble with quoted punctuation.
解决方案:　Check for punctuation before a quote. (Jason Franklin)
文件:　　src/search.c, src/testdir/test_textobjects.vim

Patch 8.1.0084
问题:　　User name completion does not work on MS-Windows.
解决方案:　Use NetUserEnum() to get user names. (Yasuhiro Matsumoto)
文件:　　src/Make_ivc.mak, src/Make_cyg_ming.mak, src/Make_mvc.mak,
　　src/misc1.c

Patch 8.1.0085
问题:　　No test for completing user name and language.
解决方案:　Add tests. (Dominique Pelle, closes #2978)
文件:　　src/testdir/test_cmdline.vim

Patch 8.1.0086
问题:	No tests for libcall() and libcallnr().
解决方案:	Add tests. (Dominique Pelle, closes #2982)
文件:	src/testdir/test_functions.vim

Patch 8.1.0087
问题:	v:shell_error is always zero when using terminal for "!cmd".
解决方案:	Use "exitval" of terminal-job. (Ozaki Kiichi, closes #2994)
文件:	src/os_unix.c, src/os_win32.c, src/proto/terminal.pro,
src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0088
问题:	Terminal test for stdout and stderr is a bit flaky.
解决方案:	Wait for both stdout and stderr to have been processed. (Ozaki
Kiichi, closes #2991)
文件:	src/testdir/test_terminal.vim

Patch 8.1.0089
问题:	error when ending the terminal debugger
解决方案:	Fix deleting defined signs for breakpoints. Make the debugger
work better on MS-Windows.
文件:	runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0090
问题:	"..." used inconsistently in a message.
解决方案:	Define the message with " ..." once. (hint by Ken Takata)
文件:	src/regexp_nfa.c

Patch 8.1.0091
问题:	MS-Windows: Cannot interrupt gdb when program is running.
解决方案:	Add debugbreak() and use it in the terminal debugger.
Respect 'modified' in a prompt buffer.
文件:	src/evalfunc.c, runtime/doc/eval.txt, src/undo.c,
runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0092 (after 8.1.0091)
问题:	Prompt buffer test fails.
解决方案:	Set 'nomodified' before closing the window. (Ozaki Kiichi,
closes #3051)
文件:	src/testdir/test_prompt_buffer.vim

Patch 8.1.0093
问题:	non-MS-Windows: Cannot interrupt gdb when program is running.
解决方案:	Only use debugbreak() on MS-Windows.
文件:	runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0094
问题:	Help text "usage:" is not capitalized.
解决方案:	Make it "Usage:". (closes #3044)
文件:	src/main.c

Patch 8.1.0095
问题:	Dialog for ":browse tabnew" says "new window".
解决方案:	Use "new tab page". (closes #3053)

文件: src/ex_docmd.c

Patch 8.1.0096
问题: Inconsistent use of the word autocommands.
解决方案: Don't use auto-commands or "auto commands".
文件: src/fileio.c

Patch 8.1.0097
问题: Superfluous space before exclamation mark.
解决方案: Remove the space. Don't translate debug message.
文件: src/regexp_nfa.c

Patch 8.1.0098
问题: Segfault when pattern with \z() is very slow.
解决方案: Check for NULL regprog. Add "nfa_fail" to test_override() to be
            able to test this. Fix that 'searchhl' resets called_emsg.
文件: src/syntax.c, runtime/doc/eval.txt, src/evalfunc.c, src/vim.h,
            src/testdir/test_syntax.vim, src/globals.h, src/screen.c,
            src/regexp.c, src/regexp_nfa.c

Patch 8.1.0099
问题: Exclamation mark in error message not needed.
解决方案: Remove the exclamation mark.
文件: src/regexp_nfa.c

Patch 8.1.0100
问题: Terminal debugger: error when setting a watch point.
解决方案: Don't try defining a sign for a watch point.
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0101
问题: No test for getcmdwintype().
解决方案: Add a test. (Dominique Pelle, closes #3068)
文件: src/testdir/test_cmdline.vim

Patch 8.1.0102
问题: Cannot build without syntax highlighting.
解决方案: Add #ifdef around using reg_do_extmatch.
文件: src/regexp.c

Patch 8.1.0103
问题: Long version string cannot be translated.
解决方案: Build the string in init_longVersion().
文件: src/globals.h, src/version.h, src/version.c,
            src/proto/version.pro, src/main.c

Patch 8.1.0104
问题: Can't build without the +eval feature.
解决方案: Add #ifdef.
文件: src/regexp_nfa.c

Patch 8.1.0105
问题: All tab stops are the same.
解决方案: Add the variable tabstop feature. (Christian Brabandt,

closes #2711)
文件: runtime/doc/change.txt, runtime/doc/options.txt,
runtime/doc/various.txt, runtime/optwin.vim, src/beval.c,
src/beval.h, src/buffer.c, src/charset.c, src/edit.c,
src/evalfunc.c, src/ex_cmds.c, src/feature.h, src/gui_beval.c,
src/gui_w32.c, src/hardcopy.c, src/message.c, src/misc1.c,
src/ops.c, src/option.c, src/option.h, src/proto/misc1.pro,
src/proto/option.pro, src/screen.c, src/structs.h,
src/testdir/Make_all.mak, src/testdir/gen_opt_test.vim,
src/testdir/test_breakindent.vim, src/testdir/test_vartabs.vim,
src/version.c, src/workshop.c, src/Makefile

Patch 8.1.0106 (after 8.1.0103)
问题: Build fails when HAVE_DATE_TIME is undefined.
解决方案: Always define init_longVersion(). (Christian Brabandt,
closes #3075)
文件: src/version.c

Patch 8.1.0107
问题: Python: getting buffer option clears message. (Jacob Niehus)
解决方案: Don't use aucmd_prepbuf(). (closes #3079)
文件: src/option.c

Patch 8.1.0108
问题: No Danish translations.
解决方案: Add Danish message translations. (closes #3073) Move list of
languages to a common makefile.
文件: src/po/Makefile, src/po/Make_cyg.mak, src/po/Make_mvc.mak,
src/po/Make_ming.mak, src/po/Make_all.mak, src/po/da.po

Patch 8.1.0109
问题: New po makefile missing from distribution.
解决方案: Add it to the file list.
文件: Filelist

Patch 8.1.0110
问题: File name not displayed with ":file" when 'F' is in 'shortmess'.
解决方案: Always display the file name when there is no argument (Christian
Brabandt, closes #3070)
文件: src/ex_cmds.c, src/testdir/test_options.vim

Patch 8.1.0111
问题: .po files do not use recommended names.
解决方案: Give a warning if the recommended name is not used. Accept the
recommended name for conversion. (Christian Brabandt, Ken Takata)
文件: src/po/Makefile, src/po/sjiscorr.c, src/po/check.vim

Patch 8.1.0112
问题: No error when using bad arguments with searchpair().
解决方案: Add error messages.
文件: src/evalfunc.c, src/testdir/test_search.vim

Patch 8.1.0113
问题: Compiler warning for unused variable. (Yegappan Lakshmanan)

解决方案: Add UNUSED. (Christian Brabandt)
文件: src/screen.c

Patch 8.1.0114
问题: Confusing variable name.
解决方案: Rename new_ts to new_vts_array. Change zero to NULL.
文件: src/ex_cmds.c, src/option.c

Patch 8.1.0115
问题: The matchparen plugin may throw an error.
解决方案: Change the skip argument from zero to "0".
文件: runtime/plugin/matchparen.vim

Patch 8.1.0116
问题: Display problem with 'vartabstop' and 'linebreak'. (Chauca
Fuentes)
解决方案: Call tabstop_padding(). (Christian Brabandt, closes #3076)
文件: src/screen.c, src/testdir/test_vartabs.vim

Patch 8.1.0117
问题: URL in install program still points to SourceForge.
解决方案: Change it to www.vim.org. (closes #3100)
文件: src/dosinst.c

Patch 8.1.0118
问题: Duplicate error message for put command.
解决方案: Check return value of u_save(). (Jason Franklin)
文件: src/ops.c, src/testdir/test_messages.vim src/testdir/test_put.vim

Patch 8.1.0119
问题: Failing test goes unnoticed because testdir/messages is not
written.
解决方案: Set 'nomodifiable' only local to the buffer.
文件: src/testdir/test_put.vim

Patch 8.1.0120
问题: Buffer 'modified' set even when :sort has no changes.
解决方案: Only set 'modified' when lines are moved. (Jason Franklin)
文件: src/ex_cmds.c, src/testdir/test_sort.vim

Patch 8.1.0121
问题: Crash when using balloneval related to 'vartabstop'.
解决方案: Initialize balloonEval->vts to NULL. (Markus Braun)
文件: src/ex_cmds2.c, src/gui_beval.c, src/gui_w32.c, src/gui.c

Patch 8.1.0122
问题: Translators don't always understand the maintainer message.
解决方案: Add a comment that ends up in the generated po file. (Christian
Brabandt, closes #3037)
文件: src/message.c

Patch 8.1.0123
问题: MS-Windows: colors are wrong after setting 'notgc'.
解决方案: Only call control_console_color_rgb() for the win32 terminal.

(Nobuhiro Takasaki, closes #3107)
文件: src/option.c

Patch 8.1.0124
问题: has('vcon') returns true even for non-win32 terminal.
解决方案: Check the terminal type. (Nobuhiro Takasaki, closes #3106)
文件: src/evalfunc.c

Patch 8.1.0125
问题: Virtual edit replace with multi-byte fails at end of line. (Lukas Werling)
解决方案: use ins_char() to add the character. (Christian Brabandt, closes #3114) Rename PCHAR() to PBYTE() to avoid mistakes like this.
文件: src/ops.c, src/testdir/test_virtualedit.vim, src/macros.h

Patch 8.1.0126
问题: Various problems with 'vartabstop'.
解决方案: Fix memory leak. Fix crash. Add a few more tests. (Christian Brabandt, closes #3076)
文件: src/ex_cmds.c, src/option.c, src/screen.c, src/testdir/test_vartabs.vim

Patch 8.1.0127
问题: Build failure when disabling the session feature. (Pawel Slowik)
解决方案: Adjust #ifdef for vim_chdirfile().
文件: src/misc2.c

Patch 8.1.0128
问题: Building with MinGW does not work out-of-the-box.
解决方案: Add instructions for MSYS2. Set default WINVER. Add batch files to set $PATH for MSYS2.
文件: src/Make_cyg_ming.mak, src/INSTALLpc.txt, src/msys32.bat, src/msys64.bat, Filelist

Patch 8.1.0129
问题: Still some xterm-like terminals get a stray "p" on startup.
解决方案: Consider all terminals that reply with a version smaller than 95 as not an xterm. (James McCoy)
文件: src/term.c

Patch 8.1.0130
问题: ":profdel func" does not work if func was called already. (Dominique Pelle)
解决方案: Reset uf_profiling and add a flag to indicate initialization was done.
文件: src/structs.h, src/userfunc.c

Patch 8.1.0131
问题: :profdel is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3123)
文件: src/testdir/test_profile.vim

Patch 8.1.0132

问题:	Lua tests are old style.
解决方案:	Convert to new style tests. Improve coverage. (Dominique Pelle, closes #3091)
文件:	src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_amiga.mak, src/testdir/Make_vms.mms, src/testdir/test85.in, src/testdir/test_lua.vim

Patch 8.1.0133
问题:	tagfiles() can have duplicate entries.
解决方案:	Simplify the filename to make checking for duplicates work better. Add a test. (Dominique Pelle, closes #2979)
文件:	src/tag.c, src/testdir/test_taglist.vim

Patch 8.1.0134
问题:	Lua interface does not support funcref.
解决方案:	Add funcref support. (Luis Carvalho)
文件:	src/if_lua.c, src/testdir/test_lua.vim

Patch 8.1.0135
问题:	Undo message delays screen update for CTRL-O u.
解决方案:	Add smsg_attr_keep(). (closes #3125)
文件:	src/message.c, src/proto.h, src/undo.c

Patch 8.1.0136
问题:	Lua tests don't cover new features.
解决方案:	Add more tests. (Dominique Pelle, closes #3130)
文件:	runtime/doc/if_lua.txt, src/testdir/test_lua.vim

Patch 8.1.0137
问题:	CI does not run with TCL.
解决方案:	Add TCL to the travis config. (Dominique Pelle, closes #3133)
文件:	.travis.yml

Patch 8.1.0138
问题:	Negative value of 'softtabstop' not used correctly.
解决方案:	Use get_sts_value(). (Tom Ryder)
文件:	src/edit.c, src/option.c, src/Makefile, src/testdir/test_tab.vim

Patch 8.1.0139
问题:	Lua tests fail on some platforms.
解决方案:	Accept a hex number with and without "0x". (Ken Takata, closes #3137)
文件:	src/testdir/test_lua.vim

Patch 8.1.0140
问题:	Recording into a register has focus events. (Michael Naumann)
解决方案:	Don't record K_FOCUSGAINED and K_FOCUSLOST. (closes #3143)
文件:	src/getchar.c

Patch 8.1.0141
问题:	:cexpr no longer jumps to the first error.
解决方案:	Use the quickfix list identifier. (Yegappan Lakshmanan, closes #3092)
文件:	src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0142
问题: Xterm and vt320 builtin termcap missing keypad keys.
解决方案: Add the escape sequences. (Kouichi Iwamoto, closes #2973)
文件: src/term.c

Patch 8.1.0143
问题: Matchit and matchparen don't handle E363.
解决方案: Catch the E363 error. (Christian Brabandt)
文件: runtime/pack/dist/opt/matchit/plugin/matchit.vim,
            runtime/plugin/matchparen.vim

Patch 8.1.0144
问题: The :cd command does not have good test coverage.
解决方案: Add more tests. (Dominique Pelle, closes #2972)
文件: src/testdir/test_cd.vim

Patch 8.1.0145
问题: Test with grep is failing on MS-Windows.
解决方案: Skip the test.
文件: src/testdir/test_quickfix.vim

Patch 8.1.0146
问题: When $LANG is set the compiler test may fail.
解决方案: Unset $LANG.
文件: src/testdir/test_compiler.vim

Patch 8.1.0147
问题: Compiler warning when building with Python 3.7.
解决方案: #undef PySlice_GetIndicesEx before redefining it. (Ozaki Kiichi,
            closes #3153)
文件: src/if_python3.c

Patch 8.1.0148
问题: Memory leak when using :tcl expr command.
解决方案: Free the result of expression evaluation. (Dominique Pelle,
            closes #3150)
文件: src/if_tcl.c

Patch 8.1.0149
问题: The generated sessions file does not restore tabs properly if :lcd
            was used in one of them.
解决方案: Create the tab pages before setting the directory. (Yee Cheng
            Chin, closes #3152)
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0150
问题: Insufficient test coverage for Tcl.
解决方案: Add more tests. (Dominique Pelle, closes #3140)
文件: src/testdir/test_tcl.vim

Patch 8.1.0151
问题: Mksession test fails on MS-Windows.
解决方案: Always use an argument for :lcd.

文件:　　src/testdir/test_mksession.vim

Patch 8.1.0152
问题:　　Cannot easily run individual tests on MS-Windows.
解决方案:　Move the list of tests to a separate file. Add a build rule in
　　the MSVC makefile.
文件:　　Filelist, src/Makefile, src/Make_all.mak, src/Make_mvc.mak

Patch 8.1.0153 (after 8.1.0152)
问题:　　Build with SHADOWDIR fails. (Elimar Riesebieter)
解决方案:　Create a link for Make_all.mak. (Tony Mechelynck)
文件:　　src/Makefile

Patch 8.1.0154
问题:　　Crash with "set smarttab shiftwidth=0 softtabstop=-1".
解决方案:　Fall back to using 'tabstop'. (closes #3155)
文件:　　src/edit.c, src/testdir/test_tab.vim

Patch 8.1.0155
问题:　　Evim.man missing from the distribution.
解决方案:　Add it to the list.
文件:　　Filelist

Patch 8.1.0156
问题:　　MS-Windows compiler warning.
解决方案:　Add a type cast. (Mike Williams)
文件:　　src/version.c

Patch 8.1.0157
问题:　　Old iTerm2 is not recognized, resulting in stray output.
解决方案:　Recognize the termresponse.
文件:　　src/term.c

Patch 8.1.0158
问题:　　GUI: input() fails if CTRL-C was pressed before. (Michael Naumann)
解决方案:　call vpeekc() to drop the CTRL-C from the input stream.
文件:　　src/ex_docmd.c

Patch 8.1.0159
问题:　　Completion for user names does not work if a prefix is also a full
　　matching name. (Nazri Ramliy)
解决方案:　Accept both full and partial matches. (Dominique Pelle)
文件:　　src/misc1.c, src/ex_docmd.c

Patch 8.1.0160
问题:　　No Danish manual translations.
解决方案:　Add the Danish manual translations to the file list.
文件:　　Filelist

Patch 8.1.0161
问题:　　Buffer not updated with 'autoread' set if file was deleted.
　　(Michael Naumann)
解决方案:　Don't set the timestamp to zero. (closes #3165)
文件:　　src/fileio.c, src/testdir/test_stat.vim

Patch 8.1.0162
问题: Danish and German man pages are not installed. (Tony Mechelynck)
解决方案: Adjust the makefile
文件: src/Makefile

Patch 8.1.0163
问题: Insufficient testing for Tcl.
解决方案: Add a few more tests. (Dominique Pelle, closes #3166)
文件: src/testdir/test_tcl.vim

Patch 8.1.0164
问题: luaeval('vim.buffer().name') returns an error.
解决方案: Return an empty string. (Dominique Pelle, closes #3167)
文件: src/if_lua.c, src/testdir/test_lua.vim

Patch 8.1.0165
问题: :clist output can be very long.
解决方案: Support filtering :clist entries. (Yegappan Lakshmanan)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0166
问题: Using dict_add_nr_str() is clumsy.
解决方案: Split into two functions. (Ozaki Kiichi, closes #3154)
文件: src/channel.c, src/dict.c, src/edit.c, src/evalfunc.c,
            src/ex_cmds2.c, src/ops.c, src/option.c, src/proto/dict.pro,
            src/quickfix.c, src/tag.c, src/terminal.c, src/undo.c

Patch 8.1.0167
问题: Lock flag in new dictitem is reset in many places.
解决方案: Always reset the lock flag.
文件: src/dict.c, src/channel.c, src/ex_cmds2.c, src/userfunc.c,
            src/if_perl.xs, src/if_py_both.h

Patch 8.1.0168
问题: Output of :marks is too short with multi-byte chars. (Tony
            Mechelynck)
解决方案: Get more bytes from the text line.
文件: src/mark.c, src/testdir/test_marks.vim

Patch 8.1.0169 (after 8.1.0165)
问题: Calling message_filtered() a bit too often.
解决方案: Only call message_filtered() when filtering is already false.
文件: src/quickfix.c, runtime/doc/quickfix.txt

Patch 8.1.0170
问题: Invalid memory use with complicated pattern. (Andy Massimino)
解决方案: Reallocate the list of listids when needed. (closes #3175)
            Remove unnecessary function prototypes.
文件: src/regexp_nfa.c

Patch 8.1.0171
问题: Typing CTRL-W n in a terminal window causes ml_get error.
解决方案: When resizing the terminal outside of terminal_loop() make sure

the snapshot is complete.
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0172
问题: 'viminfofile' option does not behave like a file name.
解决方案: Add the P_EXPAND flag. (closes #3178)
文件: src/option.c

Patch 8.1.0173
问题: Compiler warning on MS-Windows.
解决方案: Add type cast. (Mike Williams)
文件: src/libvterm/src/state.c

Patch 8.1.0174
问题: After paging up and down fold line is wrong.
解决方案: Correct the computation of w_topline and w_botline. (Hirohito Higashi)
文件: src/move.c, src/testdir/test_fold.vim

Patch 8.1.0175
问题: Marks test fails in very wide window. (Vladimir Lomov)
解决方案: Extend the text to match 'columns'. (closes #3180, closes #3181)
文件: src/testdir/test_marks.vim

Patch 8.1.0176
问题: Overlapping string argument for strcpy(). (Coverity)
解决方案: Use STRMOVE() instead of STRCPY(). (Dominique Pelle, closes #3187)
文件: src/term.c

Patch 8.1.0177
问题: Defining function in sandbox is inconsistent, cannot use :function but can define a lambda.
解决方案: Allow defining a function in the sandbox, but also use the sandbox when executing it. (closes #3182)
文件: src/userfunc.c, src/ex_cmds.h

Patch 8.1.0178
问题: Warning for passing pointer to non-pointer argument.
解决方案: Use zero instead of NULL.
文件: src/if_ole.cpp

Patch 8.1.0179
问题: Redundant condition for boundary check.
解决方案: Remove the condition. (Dominique Pelle). Change FALSE to FAIL.
文件: src/undo.c

Patch 8.1.0180
问题: Static analysis errors in Lua interface. (Coverity)
解决方案: Check for NULL pointers.
文件: src/if_lua.c

Patch 8.1.0181
问题: Memory leak with trailing characters in skip expression.
解决方案: Free the return value.

文件: src/eval.c, src/testdir/test_search.vim

Patch 8.1.0182
问题: Unicode standard was updated.
解决方案: Include the changes. (Christian Brabandt)
文件: src/mbyte.c

Patch 8.1.0183
问题: Lua API changed, breaking the build.
解决方案: Adjust prototype of lua_rawgeti(). (Ken Takata,
            closes #3157, closes #3144)
文件: src/if_lua.c

Patch 8.1.0184
问题: Not easy to figure out the window layout.
解决方案: Add "wincol" and "winrow" to what getwininfo() returns.
文件: src/evalfunc.c, src/testdir/test_bufwintabinfo.vim,
            runtime/doc/eval.txt

Patch 8.1.0185
问题: Running tests writes lua.vim even though it is not used.
解决方案: Stop writing lua.vim.
文件: src/testdir/test1.in, src/testdir/Make_dos.mak,
            src/testdir/Make_ming.mak, src/testdir/Make_vms.mms,
            src/testdir/Makefile

Patch 8.1.0186
问题: Test for getwininfo() fails in GUI.
解决方案: Account for missing tabline.
文件: src/testdir/test_bufwintabinfo.vim

Patch 8.1.0187 (after 8.1.0184)
问题: getwininfo() and win_screenpos() return different numbers.
解决方案: Add one to "wincol" and "winrow" from getwininfo().
文件: src/evalfunc.c, src/testdir/test_bufwintabinfo.vim,
            runtime/doc/eval.txt

Patch 8.1.0188
问题: No test for ":cscope add".
解决方案: Add a test. (Dominique Pelle, closes #3212)
文件: src/testdir/test_cscope.vim

Patch 8.1.0189
问题: Function defined in sandbox not tested.
解决方案: Add a text.
文件: src/testdir/test_functions.vim

Patch 8.1.0190
问题: Perl refcounts are wrong.
解决方案: Improve refcounting. Add a test. (Damien)
文件: src/if_perl.xs, src/testdir/test_perl.vim

Patch 8.1.0191 (after 8.1.0190)
问题: Perl test fails in 24 line terminal.

解决方案: Create fewer windows.
文件: src/testdir/test_perl.vim

Patch 8.1.0192
问题: Executing regexp recursively fails with a crash.
解决方案: Move global variables into "rex".
文件: src/regexp.c, src/regexp.h, src/regexp_nfa.c

Patch 8.1.0193
问题: Terminal debugger buttons don't always work. (Dominique Pelle)
解决方案: Set 'cpo' to its default value.
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0194
问题: Possibly use of NULL pointer. (Coverity)
解决方案: Reset the re_in_use flag earlier.
文件: src/regexp.c

Patch 8.1.0195
问题: Terminal debugger commands don't always work. (Dominique Pelle)
解决方案: Set 'cpo' to its default value when defining commands. (Christian Brabandt)
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0196
问题: Terminal debugger error with .gdbinit file.
解决方案: Check two lines for the "new ui" response. (hint from Hirohito Higashi)
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0197
问题: Windows GUI: title for search/replace is wrong.
解决方案: Remove remark about doubling backslash. (closes #3230)
文件: src/gui_win32.c

Patch 8.1.0198
问题: There is no hint that syntax is disabled for 'redrawtime'.
解决方案: Add a message.
文件: src/syntax.c

Patch 8.1.0199
问题: spellbadword() does not check for caps error. (Dominique Pelle)
解决方案: Adjust capcol when advancing.
文件: src/userfunc.c

Patch 8.1.0200
问题: spellbadword() not tested.
解决方案: Add a test. (Dominique Pelle, closes #3235)
文件: src/testdir/test_spell.vim

Patch 8.1.0201
问题: Newer Python uses "importlib" instead of "imp".
解决方案: Use "importlib" for newer Python versions. (Ozaki Kiichi, closes #3163)

文件: src/if_py_both.h, src/testdir/test87.in

Patch 8.1.0202
问题: :version always shows +packages. (Takuya Fujiwara)
解决方案: Add #ifdef (closes #3198) Also for has().
文件: src/version.c, src/evalfunc.c

Patch 8.1.0203
问题: Building with Perl 5.28 fails on Windows.
解决方案: Define Perl_mg_get. (closes #3196)
文件: src/if_perl.xs

Patch 8.1.0204
问题: inputlist() is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3240)
文件: src/testdir/test_functions.vim

Patch 8.1.0205
问题: Invalid memory access with invalid modeline.
解决方案: Pass pointer limit. Add a test. (closes #3241)
文件: src/Make_all.mak, src/testdir/test_alot.vim,
            src/testdir/test_modeline.vim, src/option.c

Patch 8.1.0206 (after 8.1.0205)
问题: Duplicate test function name.
解决方案: Rename both functions.
文件: src/testdir/test_modeline.vim, src/testdir/test_glob2regpat.vim

Patch 8.1.0207
问题: Need many menu translation files to cover regions.
解决方案: When there is no region match, try without. (Christian Brabandt)
文件: runtime/menu.vim

Patch 8.1.0208 (after 8.1.0205)
问题: File left behind after running individual test.
解决方案: Delete the file.
文件: src/testdir/test_modeline.vim

Patch 8.1.0209
问题: Stderr output from Ruby messes up display.
解决方案: Turn the stderr output into a Vim message. (Masataka Pocke
            Kuwabara, closes #3238)
文件: src/if_ruby.c

Patch 8.1.0210
问题: Still a few K&R function declarations.
解决方案: Use ANSI function declarations (Hirohito Higashi)
文件: src/eval.c, src/evalfunc.c, src/list.c

Patch 8.1.0211
问题: Expanding a file name "~" results in $HOME. (Aidan Shafran)
解决方案: Change "~" to "./~" before expanding. (closes #3072)
文件: src/testdir/test_expand.vim, src/ex_docmd.c, src/eval.c,
            src/proto/eval.pro, src/evalfunc.c, src/if_cscope.c, src/misc1.c

Patch 8.1.0212
问题:　　Preferred cursor column not set in interfaces.
解决方案:　Set w_set_curswant when setting the cursor. (David Hotham,
　　closes #3060)
文件:　　src/if_lua.c, src/if_mzsch.c, src/if_perl.xs, src/if_py_both.h,
　　src/if_ruby.c, src/if_tcl.c, src/testdir/test_lua.vim,
　　src/testdir/test_perl.vim, src/testdir/test_python2.vim,
　　src/testdir/test_python3.vim, src/testdir/test_ruby.vim,
　　src/testdir/test_tcl.vim

Patch 8.1.0213
问题:　　CTRL-W CR does not work properly in a quickfix window.
解决方案:　Split the window if needed. (Jason Franklin)
文件:　　src/normal.c, src/proto/quickfix.pro, src/quickfix.c,
　　src/testdir/test_quickfix.vim, src/window.c

Patch 8.1.0214
问题:　　+autochdir feature not reported by has() or :version.
解决方案:　Add the feature in the list.
文件:　　src/evalfunc.c, src/version.c

Patch 8.1.0215
问题:　　No error if configure --with-x cannot configure X.
解决方案:　Check that when --with-x is used X can be configured.
文件:　　src/configure.ac, src/auto/configure

Patch 8.1.0216
问题:　　Part of file not indented properly.
解决方案:　Adjust the indent. (Ken Takata)
文件:　　src/getchar.c

Patch 8.1.0217
问题:　　Compiler warning for variable set but not used.
解决方案:　Move tilde_file inside #ifdef. (Hirohito Higashi, closes #3255)
文件:　　src/ex_docmd.c

Patch 8.1.0218
问题:　　Cannot add matches to another window. (Qiming Zhao)
解决方案:　Add the "window" argument to matchadd() and matchaddpos().
　　(closes #3260)
文件:　　src/evalfunc.c, runtime/doc/eval.txt, src/testdir/test_match.vim

Patch 8.1.0219
问题:　　Expanding ## fails to escape backtick.
解决方案:　Escape a backtick in a file name. (closes #3257)
文件:　　src/ex_docmd.c, src/testdir/test_edit.vim

Patch 8.1.0220
问题:　　Ruby converts v:true and v:false to a number.
解决方案:　Use Qtrue and Qfalse instead. (Masataka Pocke Kuwabara,
　　closes #3259)
文件:　　src/if_ruby.c, src/testdir/test_ruby.vim

Patch 8.1.0221
问题:　　Not enough testing for the Ruby interface.
解决方案:　Add more tests. (Dominique Pelle, closes #3252)
文件:　　runtime/doc/if_ruby.txt, src/testdir/test_ruby.vim

Patch 8.1.0222
问题:　　Errors are reported for "make install".
解决方案:　Skip missing language files. (Christian Brabandt, closes #3254)
文件:　　src/installman.sh

Patch 8.1.0223
问题:　　Completing shell command finds sub-directories in $PATH.
解决方案:　Remove EW_DIR when completing an item in $PATH. (Jason Franklin)
文件:　　src/ex_getln.c, src/testdir/test_cmdline.vim

Patch 8.1.0224
问题:　　Hang in bracketed paste mode when t_PE not encountered.
解决方案:　Break out of the loop when got_int is set. (suggested by Christian
　　Brabandt, closes #3146)
文件:　　src/edit.c

Patch 8.1.0225
问题:　　Mode() does not indicate using CTRL-O from Insert mode.
解决方案:　Add "niI", "niR" and "niV" to mode() result. (closes #3000)
文件:　　runtime/doc/eval.txt, src/evalfunc.c,
　　src/testdir/test_functions.vim

Patch 8.1.0226
问题:　　Too many #ifdefs.
解决方案:　Graduate the +vreplace feature, it's not much code and quite a few
　　#ifdefs.
文件:　　runtime/doc/change.txt, runtime/doc/various.txt, src/edit.c,
　　src/evalfunc.c, src/gui.c, src/misc1.c, src/misc2.c, src/normal.c,
　　src/ops.c, src/screen.c, src/version.c, src/feature.h,
　　src/globals.h, src/macros.h, src/vim.h

Patch 8.1.0227
问题:　　Spaces instead of tabs in makefile.
解决方案:　Use tabs and fix sorting. (Ken Takata)
文件:　　src/po/Make_all.mak

Patch 8.1.0228
问题:　　Dropping files is ignored while Vim is busy.
解决方案:　Postpone the effect of dropping files until it's safe.
文件:　　src/ex_docmd.c, src/proto/ex_docmd.pro, src/gui.c, src/gui.h,
　　src/screen.c, src/main.c, src/gui_mac.c

Patch 8.1.0229
问题:　　Crash when dumping profiling data.
解决方案:　Reset flag indicating that initialization was done.
文件:　　src/userfunc.c

Patch 8.1.0230
问题:　　Directly checking 'buftype' value.

解决方案: Add the bt_normal() function. (Yegappan Lakshmanan)
文件: src/buffer.c, src/ex_docmd.c, src/fileio.c, src/proto/buffer.pro,
src/quickfix.c

Patch 8.1.0231
问题: :help -? goes to help for -+.
解决方案: Add -? to list of special cases. (Hirohito Higashi)
文件: src/ex_cmds.c, src/testdir/test_help_tagjump.vim

Patch 8.1.0232
问题: Ruby error does not include backtrace.
解决方案: Add an error backtrace. (Masataka Pocke Kuwabara, closes #3267)
文件: src/if_ruby.c

Patch 8.1.0233
问题: "safe" argument of call_vim_function() is always FALSE.
解决方案: Remove the argument.
文件: src/eval.c, src/proto/eval.pro, src/edit.c, src/mbyte.c,
src/normal.c, src/ex_getln.c

Patch 8.1.0234
问题: Incorrect reference counting in Perl interface.
解决方案: Call SvREFCNT_inc more often, add a test. (Damien)
文件: src/if_perl.xs, src/testdir/test_perl.vim

Patch 8.1.0235 (after 8.1.0231)
问题: More help tags that jump to the wrong location.
解决方案: Add more exceptions and a table for "expr-" tags. (Hirohito
Higashi)
文件: src/ex_cmds.c, src/testdir/test_help_tagjump.vim

Patch 8.1.0236 (after 8.1.0232)
问题: Ruby build fails when ruby_intern is missing.
解决方案: Do not use ruby_intern2. (Ken Takata)
文件: src/if_ruby.c

Patch 8.1.0237
问题: Ruby on Cygwin doesn't always work.
解决方案: Use LIBRUBY_SO if LIBRUBY_ALIASES isn't set. (Ken Takata)
文件: src/configure.ac, src/auto/configure

Patch 8.1.0238
问题: 'buftype' is cleared when using ":term ++hidden cat". (Marcin
Szamotulski)
解决方案: Set the "options initialized" flag earlier. (closes #3278)
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0239 (after 8.1.0236)
问题: Now Ruby build fails on other systems.
解决方案: Always define rb_intern. (Ken Takata, closes #3275)
文件: src/if_ruby.c

Patch 8.1.0240
问题: g:actual_curbuf set in wrong scope. (Daniel Hahler)

解决方案: Prepend the "g:" name space. (closes #3279)
文件: src/buffer.c

Patch 8.1.0241
问题: Effect of ":tabmove N" is not clear.
解决方案: Add a test that shows the behavior. (Christian Brabandt,
closes #3288)
文件: src/testdir/test_tabpage.vim

Patch 8.1.0242
问题: Insert mode completion may use an invalid buffer pointer. (Akib
Nizam)
解决方案: Check for ins_buf to be valid. (closes #3290)
文件: src/edit.c

Patch 8.1.0243
问题: Using :term ++close ++hidden closes a window. (Marcin Szamotulski)
解决方案: Don't close the window if only using it temporarily for unloading
the terminal buffer. (closes #3287)
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0244
问题: No redraw when using a STOP signal on Vim and then a CONT signal.
解决方案: Catch the CONT signal and force a redraw. (closes #3285)
文件: src/os_unix.c, src/term.c, src/proto/term.pro

Patch 8.1.0245
问题: Calling setline() in TextChangedI autocmd breaks undo. (Jason
Felice)
解决方案: Don't save lines for undo when already saved. (closes #3291)
文件: src/edit.c, src/testdir/test_autocmd.vim

Patch 8.1.0246 (after 8.1.0245)
问题: Build failure without the +eval feature.
解决方案: Add #ifdef
文件: src/edit.c

Patch 8.1.0247
问题: Python: error message for failing import is incorrect.
解决方案: Adjust how modules are loaded. (Ozaki Kiichi, closes #3162)
文件: src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok

Patch 8.1.0248
问题: duplicated quickfix code.
解决方案: Move the code to a function.
文件: src/quickfix.c

Patch 8.1.0249
问题: GTK: when screen DPI changes Vim does not handle it.
解决方案: Handle the gtk-xft-dpi signal. (Roel van de Kraats,
closes #2357)
文件: src/gui_gtk_x11.c

Patch 8.1.0250

问题:　　MS-Windows using VTP: windows size change incorrect.
解决方案:　Call SetConsoleScreenBufferSize() first. (Nobuhiro Takasaki,
closes #3164)
文件:　　src/os_win32.c

Patch 8.1.0251
问题:　　Using a full path is supported for 'directory' but not for
'backupdir'. (Mikolaj Machowski)
解决方案:　Support 'backupdir' as well. (Christian Brabandt, closes #179)
文件:　　runtime/doc/options.txt, src/fileio.c, src/memline.c,
src/proto/memline.pro, src/testdir/test_alot.vim,
src/testdir/test_backup.vim, src/Make_all.mak

Patch 8.1.0252
问题:　　Quickfix functions are too long.
解决方案:　Refactor. (Yegappan Lakshmanan, closes #2950)
文件:　　src/quickfix.c

Patch 8.1.0253
问题:　　Saving and restoring window title does not always work.
解决方案:　Use the stack push and pop commands. (Kouichi Iwamoto,
closes #3059)
文件:　　runtime/doc/term.txt, src/main.c, src/option.c, src/os_unix.c,
src/proto/term.pro, src/term.c, src/term.h, src/vim.h,
src/buffer.c, src/ex_docmd.c, src/option.c, src/os_amiga.c,
src/os_mswin.c, src/os_win32.c

Patch 8.1.0254 (after 8.1.0253)
问题:　　Cannot build on MS-Windows; Unused macro HAVE_HANDLE_DROP.
解决方案:　Adjust #ifdef. Delete the macro.
文件:　　src/main.c, src/vim.h

Patch 8.1.0255 (after 8.1.0251)
问题:　　Backup test fails when using shadow directory.
解决方案:　Remove check for "src".
文件:　　src/testdir/test_backup.vim

Patch 8.1.0256 (after 8.1.0245)
问题:　　Using setline() in TextChangedI splits undo.
解决方案:　Use another solution for undo not working properly.
文件:　　src/edit.c, src/testdir/test_autocmd.vim

Patch 8.1.0257
问题:　　No test for pathshorten().
解决方案:　Add a test. (Dominique Pelle, closes #3295)
文件:　　src/testdir/test_functions.vim

Patch 8.1.0258
问题:　　Not enough testing for the CompleteDone event.
解决方案:　Add a test. (closes #3297)
文件:　　src/testdir/test_ins_complete.vim

Patch 8.1.0259
问题:　　No test for fixed quickfix issue.

解决方案: Add a test. Clean up the code a bit. (Yegappan Lakshmanan)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0260
问题: No LGTM logo in README file.
解决方案: Add one. (Bas van Schaik, closes #3305)
文件: README.md

Patch 8.1.0261
问题: Coverity complains about a negative array index.
解决方案: When qf_id2nr() cannot find the list then don't set qf_curlist.
文件: src/quickfix.c

Patch 8.1.0262
问题: Not enough testing for getftype().
解决方案: Add a test. (Dominique Pelle, closes #3300)
文件: src/evalfunc.c, src/testdir/test_stat.vim

Patch 8.1.0263
问题: Channel log doesn't show part of channel.
解决方案: Add "sock", "out", "err" or "in". (Ozaki Kiichi, closes #3303)
文件: src/channel.c

Patch 8.1.0264
问题: Backup tests fail when CWD is in /tmp.
解决方案: Make 'backupskip' empty. (Christian Brabandt, closes #3301)
文件: src/testdir/test_backup.vim

Patch 8.1.0265
问题: The getcmdline() function is way too big.
解决方案: Factor out the incremental search highlighting.
文件: src/ex_getln.c

Patch 8.1.0266
问题: Parsing Ex address range is not a separate function.
解决方案: Refactor do_one_cmd() to separate address parsing.
文件: src/ex_docmd.c, src/proto/ex_docmd.pro

Patch 8.1.0267
问题: No good check if restoring quickfix list worked.
解决方案: Let qf_restore_list() return OK/FAIL. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0268
问题: File type checking has too many #ifdef.
解决方案: Always define the S_IF macros. (Ken Takata, closes #3306)
文件: src/buffer.c, src/evalfunc.c, src/fileio.c, src/if_cscope.c,
           src/os_unix.c, src/os_unix.h, src/vim.h

Patch 8.1.0269
问题: Ruby Kernel.#p method always returns nil.
解决方案: Copy p method implementation from Ruby code. (Masataka Pocke
           Kuwabara, closes #3315)
文件: src/if_ruby.c, src/testdir/test_ruby.vim

Patch 8.1.0270
问题:　　Checking for a Tab in a line could be faster.
解决方案:　Use strchr() instead of strrchr(). (closes #3312)
文件:　　src/ex_cmds.c

Patch 8.1.0271
问题:　　'incsearch' doesn't work for :s, :g or :v.
解决方案:　Also use 'incsearch' for other commands that use a pattern.
文件:　　src/ex_getln.c, src/globals.h, src/screen.c,
　　src/testdir/test_search.vim

Patch 8.1.0272
问题:　　Options test fails if temp var ends in slash. (Tom Briden)
解决方案:　Check for optional slash. (closes #3308)
文件:　　src/testdir/test_options.vim

Patch 8.1.0273
问题:　　Invalid memory access when using 'incsearch'.
解决方案:　Reset "patlen" when using previous search pattern.
文件:　　src/ex_getln.c

Patch 8.1.0274
问题:　　'incsearch' triggers on ":source".
解决方案:　Check for the whole command name.
文件:　　src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0275
问题:　　'incsearch' with :s doesn't start at cursor line.
解决方案:　Set cursor before parsing address. (closes #3318)
　　Also accept a match at the start of the first line.
文件:　　src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0276
问题:　　No test for 'incsearch' highlighting with :s.
解决方案:　Add a screendump test.
文件:　　src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_01.dump

Patch 8.1.0277
问题:　　'incsearch' highlighting wrong in a few cases.
解决方案:　Fix using last search pattern. Restore highlighting when changing
　　command. (issue #3321)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_02.dump,
　　src/testdir/dumps/Test_incsearch_substitute_03.dump

Patch 8.1.0278
问题:　　'incsearch' highlighting does not accept reverse range.
解决方案:　Swap the range when needed. (issue #3321)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_04.dump

Patch 8.1.0279

问题:　　'incsearch' highlighting does not skip white space.
解决方案:　Skip white space after the command. (issue #3321)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_05.dump

Patch 8.1.0280
问题:　　'incsearch' highlighting does not work for ":g!/".
解决方案:　Skip the exclamation mark. (Hirohito Higashi)
文件:　　src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0281
问题:　　Parsing command modifiers is not separated.
解决方案:　Move command modifier parsing to a separate function.
文件:　　src/ex_docmd.c, src/proto/ex_docmd.pro, src/ex_cmds.h,
　　src/globals.h, src/feature.h

Patch 8.1.0282
问题:　　'incsearch' does not work with command modifiers.
解决方案:　Skip command modifiers.
文件:　　src/ex_docmd.c, src/proto/ex_docmd.pro, src/ex_getln.c,
　　src/testdir/test_search.vim

Patch 8.1.0283 (after 8.1.0282)
问题:　　Missing test dump.
解决方案:　Add the dump file
文件:　　src/testdir/dumps/Test_incsearch_substitute_06.dump

Patch 8.1.0284
问题:　　'cursorline' highlighting wrong with 'incsearch'.
解决方案:　Move the cursor back if the match is outside the range.
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_07.dump
　　src/testdir/dumps/Test_incsearch_substitute_08.dump

Patch 8.1.0285
问题:　　Compiler warning for conversion.
解决方案:　Add a type cast. (Mike Williams)
文件:　　src/ex_getln.c

Patch 8.1.0286
问题:　　'incsearch' does not apply to :smagic and :snomagic.
解决方案:　Add support. (Hirohito Higashi)
文件:　　src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0287
问题:　　MAX is not defined everywhere.
解决方案:　Define MAX where needed.
文件:　　src/ex_getln.c

Patch 8.1.0288
问题:　　Quickfix code uses cmdidx too often.
解决方案:　Add is_loclist_cmd(). (Yegappan Lakshmanan)
文件:　　src/ex_docmd.c, src/proto/ex_docmd.pro, src/quickfix.c

Patch 8.1.0289
问题:　　Cursor moves to wrong column after quickfix jump.
解决方案:　Set the curswant flag. (Andy Massimino, closes #3331)
文件:　　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0290
问题:　　"cit" on an empty HTML tag changes the whole tag.
解决方案:　Only adjust the area in Visual mode. (Andy Massimino,
　　closes #3332)
文件:　　src/search.c, src/testdir/test_textobjects.vim

Patch 8.1.0291
问题:　　'incsearch' highlighting not used for :sort.
解决方案:　Handle pattern in :sort command.
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_sort_01.dump

Patch 8.1.0292
问题:　　MS-Windows: the text "self-installing" confuses some users.
解决方案:　Remove the text from the uninstall entry. (closes #3337)
文件:　　src/dosinst.c

Patch 8.1.0293
问题:　　Checks for type of stack is cryptic.
解决方案:　Define IS_QF_STACK() and IS_LL_STACK(). (Yegappan Lakshmanan)
文件:　　src/quickfix.c

Patch 8.1.0294
问题:　　MS-Windows: sometimes uses short directory name.
解决方案:　Expand to long file name with correct caps. (Nobuhiro Takasaki,
　　closes #3334)
文件:　　src/os_win32.c

Patch 8.1.0295
问题:　　No 'incsearch' highlighting for :vimgrep and similar commands.
解决方案:　Parse the :vimgrep command and similar ones to locate the search
　　pattern. (Hirohito Higashi, closes #3344)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_vimgrep_01.dump,
　　src/testdir/dumps/Test_incsearch_vimgrep_02.dump,
　　src/testdir/dumps/Test_incsearch_vimgrep_03.dump,
　　src/testdir/dumps/Test_incsearch_vimgrep_04.dump,
　　src/testdir/dumps/Test_incsearch_vimgrep_05.dump

Patch 8.1.0296
问题:　　Command parsing for 'incsearch' is a bit ugly.
解决方案:　Return when there is no pattern. Put common checks together.
文件:　　src/ex_getln.c

Patch 8.1.0297 (after 8.1.0294)
问题:　　MS-Windows: tests fail, Vim crashes.
解决方案:　Fix long file name handling.
文件:　　src/os_win32.c

Patch 8.1.0298
问题:　　Window resize test sometimes fails on Mac.
解决方案:　Add Test_popup_and_window_resize() to flaky tests.
文件:　　src/testdir/runtest.vim

Patch 8.1.0299 (after 8.1.0298)
问题:　　misplaced comment
解决方案:　Remove comment
文件:　　src/testdir/runtest.vim

Patch 8.1.0300
问题:　　The old window title might be freed twice. (Dominique Pelle)
解决方案:　Do not free "oldtitle" in a signal handler but set a flag to have
　　it freed later.
文件:　　src/os_unix.c

Patch 8.1.0301
问题:　　GTK: Input method popup displayed on wrong screen.
解决方案:　Add the screen position offset. (Ken Takata, closes #3268)
文件:　　src/gui_beval.c, src/gui_gtk_x11.c, src/mbyte.c,
　　src/proto/gui_gtk_x11.pro

Patch 8.1.0302
问题:　　Crash when using :suspend and "fg".
解决方案:　Undo patch 8.1.0244.
文件:　　src/os_unix.c, src/term.c, src/proto/term.pro

Patch 8.1.0303
问题:　　line2byte() is wrong for last line with 'noeol' and 'nofixeol'.
解决方案:　Fix off-by-one error. (Shane Harper, closes #3351)
文件:　　src/memline.c, src/testdir/test_functions.vim

Patch 8.1.0304
问题:　　No redraw when using a STOP signal on Vim and then a CONT signal.
解决方案:　Catch the CONT signal and set the terminal to raw mode. This is
　　like 8.1.0244 but without the screen redraw and a fix for
　　multi-threading suggested by Dominique Pelle.
文件:　　src/os_unix.c, src/term.c, src/proto/term.pro

Patch 8.1.0305
问题:　　Missing support for Lua 5.4 32 bits on Unix.
解决方案:　Define lua_newuserdatauv. (Kazunobu Kuriyama)
文件:　　src/if_lua.c

Patch 8.1.0306
问题:　　Plural messages are not translated properly.
解决方案:　Add more usage of NGETTEXT(). (Sergey Alyoshin)
文件:　　src/vim.h, src/buffer.c, src/ex_cmds.c, src/ex_docmd.c,
　　src/fileio.c, src/misc1.c, src/ops.c

Patch 8.1.0307
问题:　　There is no good way to get the window layout.
解决方案:　Add the winlayout() function. (Yegappan Lakshmanan)
文件:　　runtime/doc/eval.txt, src/evalfunc.c, src/proto/window.pro,

src/window.c, src/testdir/test_window_id.vim

Patch 8.1.0308
问题: A quick undo shows "1 seconds ago". (Tony Mechelynck)
解决方案: Add singular/plural message.
文件: src/undo.c

Patch 8.1.0309
问题: Profiling does not show a count for condition lines. (Daniel Hahler)
解决方案: Count lines when not skipping. (Ozaki Kiichi, closes #2499)
文件: src/ex_docmd.c, src/testdir/test_profile.vim

Patch 8.1.0310
问题: File info message not always suppressed with 'F' in 'shortmess'. (Asheq Imran)
解决方案: Save and restore msg_silent. (Christian Brabandt, closes #3221)
文件: src/buffer.c, src/memline.c, src/testdir/test_options.vim

Patch 8.1.0311
问题: Filtering entries in a quickfix list is not easy.
解决方案: Add the cfilter plugin. (Yegappan Lakshmanan)
文件: runtime/pack/dist/opt/cfilter/plugin/cfilter.vim, runtime/doc/quickfix.txt

Patch 8.1.0312
问题: Wrong type for flags used in signal handlers.
解决方案: Use sig_atomic_t. (Dominique Pelle, closes #3356)
文件: src/globals.h, src/os_unix.c, src/os_win32.h

Patch 8.1.0313
问题: Information about a swap file is unavailable.
解决方案: Add swapinfo(). (Enzo Ferber)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/memline.c, src/proto/memline.pro, src/testdir/test_swap.vim

Patch 8.1.0314 (after 8.1.0313)
问题: Build failure without the +eval feature. (Brenton Horne)
解决方案: Add #ifdef. Also add the "dirty" item.
文件: src/memline.c, runtime/doc/eval.txt, src/testdir/test_swap.vim

Patch 8.1.0315
问题: Helpgrep with language doesn't work properly. (Takuya Fujiwara)
解决方案: Check for the language earlier. (Hirohito Higashi)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0316
问题: swapinfo() test fails on Travis.
解决方案: Handle a long host name. (Ozaki Kiichi, closes #3361)
Also make the version check flexible. (James McCoy)
文件: src/testdir/test_swap.vim

Patch 8.1.0317
问题: Cscope test fails when using shadow directory.

解决方案: Resolve symlink in Vim. (James McCoy, closes #3364)
文件: src/testdir/test_cscope.vim

Patch 8.1.0318
问题: The getftype() test may fail for char devices if the file
disappeared in between the listing and the getftype() call.
解决方案: Ignore empty result. (Ozaki Kiichi, closes #3360)
文件: src/testdir/test_stat.vim

Patch 8.1.0319
问题: bzero() function prototype doesn't work for Android.
解决方案: Add an #ifdef. (Elliott Hughes, closes #3365)
文件: src/osdef1.h.in

Patch 8.1.0320
问题: Too much 'incsearch' highlight for pattern matching everything.
解决方案: Add the skiplen to the command and remove the line range.
(Christian Brabandt) Check for empty pattern earlier.
文件: src/ex_getln.c, src/testdir/test_search.vim,
src/testdir/dumps/Test_incsearch_substitute_09.dump

Patch 8.1.0321 (after 8.1.0320)
问题: 'incsearch' regression: /\v highlights everything.
解决方案: Put back the empty_pattern() check.
文件: src/ex_getln.c, src/testdir/test_search.vim,
src/testdir/dumps/Test_incsearch_search_01.dump,
src/testdir/dumps/Test_incsearch_search_02.dump

Patch 8.1.0322
问题: Test_copy_winopt() does not restore 'hidden'.
解决方案: Restore the option, fix indent. (Ozaki Kiichi, closes #3367)
文件: src/testdir/test_options.vim

Patch 8.1.0323
问题: Reverse order of VTP calls only needed the first time.
解决方案: Add a flag to remember the state. (Nobuhiro Takasaki, closes #3366)
文件: src/os_win32.c

Patch 8.1.0324
问题: Off-by-one error in cmdidx check. (Coverity)
解决方案: Use ">=" instead of ">".
文件: src/ex_docmd.c

Patch 8.1.0325
问题: Strings in swap file may not be NUL terminated. (Coverity)
解决方案: Limit the length of the used string.
文件: src/memline.c

Patch 8.1.0326
问题: Screen dump does not consider NUL and space equal.
解决方案: Use temp variables instead of character from cell.
文件: src/terminal.c, src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.0327

问题: The "g CTRL-G" command isn't tested much.
解决方案: Add more tests. (Dominique Pelle, closes #3369)
文件: src/testdir/test_normal.vim

Patch 8.1.0328
问题: inputlist() doesn't work with a timer. (Dominique Pelle)
解决方案: Don't redraw when cmdline_row is zero. (Hirohito Higashi,
            closes #3239)
文件: src/misc1.c, src/screen.c

Patch 8.1.0329
问题: Using inputlist() during startup results in garbage. (Dominique
            Pelle)
解决方案: Make sure the xterm tracing is stopped when disabling the mouse.
文件: src/os_unix.c

Patch 8.1.0330
问题: The qf_add_entries() function is too long.
解决方案: Split in two parts. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0331
问题: Insufficient test coverage for :mkview and :loadview.
解决方案: Add tests. (Dominique Pelle, closes #3385)
文件: src/testdir/test_mksession.vim

Patch 8.1.0332
问题: Get Gdk-Critical error on first balloon show.
解决方案: Get screen geometry using the draw area widget. (Davit Samvelyan,
            closes #3386)
文件: src/gui_beval.c

Patch 8.1.0333
问题: :mkview does not restore cursor properly after "$". (Dominique
            Pelle)
解决方案: Position the cursor with "normal! $".
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0334
问题: 'autowrite' takes effect when buffer is not to be written.
解决方案: Don't write buffers that are not supposed to be written. (Even Q
            Jones, closes #3391) Add tests for 'autowrite'.
文件: src/ex_cmds2.c, src/testdir/test_writefile.vim

Patch 8.1.0335
问题: mkview test fails on CI.
解决方案: Attempt to force recomputing curswant after folding.
文件: src/testdir/test_mksession.vim

Patch 8.1.0336
问题: mkview test still fails on CI.
解决方案: Ignore curswant, don't see another solution.
文件: src/testdir/test_mksession.vim

Patch 8.1.0337
问题:　　:file fails in quickfix command.
解决方案:　Allow :file without argument when curbuf_lock is set. (Jason
　　Franklin)
文件:　　src/ex_docmd.c, src/testdir/test_quickfix.vim

Patch 8.1.0338
问题:　　MS-Windows: VTP doesn't work properly with PowerShell.
解决方案:　Adjust the color index. (Nobuhiro Takasaki, closes #3347)
文件:　　src/os_win32.c

Patch 8.1.0339
问题:　　Wrong highlight when 'incsearch' set and cancelling :s.
解决方案:　Reset search line range. (Hirohito Higashi, Masamichi Abe)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_10.dump

Patch 8.1.0340
问题:　　No test for :spellinfo.
解决方案:　Add a test. (Dominique Pelle, closes #3394)
文件:　　src/testdir/test_spell.vim

Patch 8.1.0341
问题:　　:argadd in empty buffer changes the buffer name. (Pavol Juhas)
解决方案:　Don't re-use the current buffer when not going to edit the file.
　　(closes #3397)　Do re-use the current buffer for :next.
文件:　　src/ex_cmds2.c, src/testdir/test_arglist.vim,
　　src/testdir/test_command_count.vim

Patch 8.1.0342
问题:　　Crash when a callback deletes a window that is being used. (Ozaki
　　Kiichi)
解决方案:　Do not unload a buffer that is being displayed while redrawing the
　　screen. Also avoid invoking callbacks while redrawing.
　　(closes #2107)
文件:　　src/buffer.c, src/misc2.c

Patch 8.1.0343
问题:　　'shellslash' is not used for getcwd() with local directory.
　　(Daniel Hahler)
解决方案:　Call slash_adjust() later. (closes #3399)
文件:　　src/evalfunc.c

Patch 8.1.0344
问题:　　'hlsearch' highlighting has a gap after /$.
解决方案:　Remove suspicious code. (Ricky Zhou, closes #3400)
文件:　　src/screen.c, src/testdir/test_hlsearch.vim

Patch 8.1.0345
问题:　　Cannot get the window id associated with the location list.
解决方案:　Add the "filewinid" argument to getloclist(). (Yegappan
　　Lakshmanan, closes #3202)
文件:　　runtime/doc/eval.txt, src/quickfix.c,
　　src/testdir/test_quickfix.vim

Patch 8.1.0346
问题: Building with Aap is outdated and unused.
解决方案: Remove the Aap build files.
文件: Filelist, src/main.aap, src/testdir/main.aap, src/config.aap.in, runtime/macros/maze/main.aap

Patch 8.1.0347
问题: Some tests fail on Solaris.
解决方案: Skip writefile test. Fix path to libc.so. Improve test for Turkish case change. (Libor Bukata, Bjorn Linse, closes #3403)
文件: src/testdir/test_functions.vim, src/testdir/test_normal.vim, src/testdir/test_writefile.vim

Patch 8.1.0348
问题: On Travis the slowest build is run last. (Dominique Pelle)
解决方案: Reorder the build entries.
文件: .travis.yml

Patch 8.1.0349
问题: Crash when wiping buffer in a callback.
解决方案: Do not handle messages when only peeking for a character. (closes #2107) Add "redraw_flag" to test_override().
文件: src/os_unix.c, src/os_win32.c, src/screen.c, src/evalfunc.c, src/globals.h, runtime/doc/eval.txt

Patch 8.1.0350
问题: Vim may block on ch_sendraw() when the job is sending data back to Vim, which isn't read yet. (Nate Bosch)
解决方案: Add the "noblock" option to job_start(). (closes #2548)
文件: src/channel.c, src/structs.h, src/testdir/test_channel.vim, runtime/doc/channel.txt

Patch 8.1.0351
问题: 'incsearch' for :/foo/s//<Esc> changes last search pattern.
解决方案: Save the last search pattern earlier.
文件: src/ex_docmd.c, src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0352
问题: Browsing compressed tar files does not always work.
解决方案: Use the "file" command to get the compression type.
文件: runtime/autoload/tar.vim

Patch 8.1.0353
问题: An "after" directory of a package is appended to 'rtp', which will be after the user's "after" directory. ()
解决方案: Insert the package "after" directory before any other "after" directory in 'rtp'. (closes #3409)
文件: src/ex_cmds2.c, src/testdir/test_packadd.vim

Patch 8.1.0354 (after 8.1.0353)
问题: Packadd test fails on MS-Windows.
解决方案: Ignore difference between forward and backward slashes.
文件: src/testdir/test_packadd.vim

Patch 8.1.0355
问题:　　Incorrect adjusting the popup menu for the preview window.
解决方案:　Compute position and height properly. (Ronan Pigott) Also show at least ten items. (closes #3414)
文件:　　src/popupmnu.c

Patch 8.1.0356
问题:　　Using :s with 'incsearch' prevents CTRL-R CTRL-W. (Boris Staletic)
解决方案:　When past the pattern put cursor back in the start position. (closes #3413)
文件:　　src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.0357
问题:　　Instructions for tests are outdated. (Jason Franklin)
解决方案:　Update the text.
文件:　　src/testdir/README.txt

Patch 8.1.0358
问题:　　Crash when using term_dumpwrite() after the job finished.
解决方案:　Check for a finished job and give an error message.
文件:　　src/terminal.c

Patch 8.1.0359
问题:　　No clue what test failed when using a screendump twice.
解决方案:　Add an extra argument to VerifyScreenDump().
文件:　　src/testdir/screendump.vim

Patch 8.1.0360
问题:　　Using an external diff program is slow and inflexible.
解决方案:　Include the xdiff library. (Christian Brabandt, closes #2732)
Use it by default.
文件:　　Filelist, runtime/doc/diff.txt, runtime/doc/options.txt,
src/Make_cyg_ming.mak, src/Make_mvc.mak, src/Makefile, src/diff.c,
src/structs.h, src/testdir/dumps/Test_diff_01.dump,
src/testdir/dumps/Test_diff_02.dump,
src/testdir/dumps/Test_diff_03.dump,
src/testdir/dumps/Test_diff_04.dump,
src/testdir/dumps/Test_diff_05.dump,
src/testdir/dumps/Test_diff_06.dump,
src/testdir/dumps/Test_diff_07.dump,
src/testdir/dumps/Test_diff_08.dump,
src/testdir/dumps/Test_diff_09.dump,
src/testdir/dumps/Test_diff_10.dump,
src/testdir/dumps/Test_diff_11.dump,
src/testdir/dumps/Test_diff_12.dump,
src/testdir/dumps/Test_diff_13.dump,
src/testdir/dumps/Test_diff_14.dump,
src/testdir/dumps/Test_diff_15.dump,
src/testdir/dumps/Test_diff_16.dump,
src/testdir/test_diffmode.vim, src/xdiff/COPYING,
src/xdiff/xdiff.h, src/xdiff/xdiffi.c, src/xdiff/xdiffi.h,
src/xdiff/xemit.c, src/xdiff/xemit.h, src/xdiff/xhistogram.c,
src/xdiff/xinclude.h, src/xdiff/xmacros.h, src/xdiff/xpatience.c,

src/xdiff/xprepare.c, src/xdiff/xprepare.h, src/xdiff/xtypes.h,
src/xdiff/xutils.c, src/xdiff/xutils.h, src/xdiff/README.txt

Patch 8.1.0361
问题: Remote user not used for completion. (Stucki)
解决方案: Use $USER too. (Dominique Pelle, closes #3407)
文件: src/misc1.c

Patch 8.1.0362
问题: Cannot get the script line number when executing a function.
解决方案: Store the line number besides the script ID. (Ozaki Kiichi,
closes #3362) Also display the line number with ":verbose set".
文件: runtime/doc/cmdline.txt, runtime/doc/eval.txt, src/Make_all.mak,
src/buffer.c, src/eval.c, src/evalfunc.c, src/ex_cmds2.c,
src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/getchar.c,
src/globals.h, src/main.c, src/menu.c, src/option.c,
src/proto/eval.pro, src/structs.h, src/syntax.c,
src/testdir/test_alot.vim, src/testdir/test_expand_func.vim,
src/testdir/test_maparg.vim, src/term.c src/userfunc.c

Patch 8.1.0363
问题: Internal diff isn't used by default as advertised.
解决方案: Add "internal" to the default value of 'diffopt'.
Also add couple of files missing from the distribution.
文件: src/option.c, runtime/doc/options.txt, Filelist

Patch 8.1.0364
问题: Compiler warning in xdiff code. (Yegappan Lakshmanan)
解决方案: Initialize directly.
文件: src/xdiff/xemit.c, src/xdiff/README.txt

Patch 8.1.0365
问题: Function profile doesn't specify where it was defined.
解决方案: Show the script name and line number.
文件: src/userfunc.c, src/testdir/test_profile.vim

Patch 8.1.0366
问题: Pieces of the xdiff code are not used.
解决方案: Add "#if 0" to omit unused code.
文件: src/xdiff/xemit.c

Patch 8.1.0367
问题: getchar(1) no longer processes pending messages. (Yasuhiro
Matsumoto)
解决方案: Call parse_queued_messages().
文件: src/evalfunc.c

Patch 8.1.0368
问题: GTK code has too many #ifdefs and building fails with GTK 2.10.
解决方案: Always use gtk_widget_get_window() and define it for older GTK
versions. (Ken Takata, closes #3421)
文件: src/gui_beval.c, src/gui_gtk.c, src/gui_gtk_f.c,
src/gui_gtk_x11.c, src/mbyte.c, src/vim.h

Patch 8.1.0369
问题:　　Continuation lines cannot contain comments.
解决方案:　Support using "\ .
文件:　　src/ex_cmds2.c, src/testdir/test_eval_stuff.vim,
　　runtime/indent/vim.vim, runtime/doc/repeat.txt

Patch 8.1.0370
问题:　　Not using internal diff if 'diffopt' is not changed.
解决方案:　Correct initialization of diff_flags. (Christian Brabandt)
文件:　　src/diff.c

Patch 8.1.0371
问题:　　Argument types for select() may be wrong.
解决方案:　Use a configure macro. (Tobias Ulmer)
文件:　　src/config.h.in, src/configure.ac, src/auto/configure,
　　src/os_unix.c

Patch 8.1.0372
问题:　　Screen updating slow when 'cursorline' is set.
解决方案:　Only redraw the old and new cursor line, not all lines.
文件:　　src/edit.c, src/move.c, src/screen.c, src/proto/screen.pro

Patch 8.1.0373 (after 8.1.0372)
问题:　　Screen updating still slow when 'cursorline' is set.
解决方案:　Fix setting last_cursorline.
文件:　　src/move.c

Patch 8.1.0374
问题:　　Moving the cursor is slow when 'relativenumber' is set.
解决方案:　Only redraw the number column, not all lines.
文件:　　src/screen.c, src/move.c

Patch 8.1.0375
问题:　　Cannot use diff mode with Cygwin diff.exe. (Igor Forca)
解决方案:　Skip over unrecognized lines in the diff output.
文件:　　src/diff.c, src/testdir/test_diffmode.vim

Patch 8.1.0376
问题:　　Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案:　Initialize the variable.
文件:　　src/screen.c

Patch 8.1.0377
问题:　　Xdiff doesn't use the Vim memory allocation functions.
解决方案:　Change the xdl_ defines. Check for out-of-memory. Rename
　　"ignored" to "vim_ignored".
文件:　　src/xdiff/xdiff.h, src/xdiff/xpatience.c, src/xdiff/xdiffi.c,
　　src/channel.c, src/diff.c, src/evalfunc.c, src/ex_cmds.c,
　　src/fileio.c, src/main.c, src/mbyte.c, src/netbeans.c,
　　src/os_unix.c, src/os_win32.c, src/ui.c, src/window.c,
　　src/globals.h, src/term.c

Patch 8.1.0378
问题:　　CI build failure.

解决方案: Include vim.h as ../vim.h. Fix compiler warning.
文件: src/xdiff/xdiff.h, src/xdiff/xpatience.c

Patch 8.1.0379
问题: Build dependencies are incomplete.
解决方案: Update the build dependencies, mainly for xdiff. Adjust object
           directory for libvterm and xdiff.
文件: src/Makefile, src/configure.ac, src/auto/configure,
           src/libvterm/src/screen.c, src/libvterm/src/termscreen.c,
           src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.0380
问题: "make proto" doesn't work well.
解决方案: Define a few more types for cproto. Update proto files. Fix that
           workshop didn't build.
文件: src/vim.h, src/protodef.h, src/if_ruby.c, src/workshop.c,
           src/proto/digraph.pro, src/hardcopy.pro, src/proto/option.pro,
           src/proto/window.pro

Patch 8.1.0381
问题: Variable declaration not at start of block.
解决方案: Fix line ordering.
文件: src/xdiff/xpatience.c

Patch 8.1.0382
问题: Some make programs can't handle dependency on "xdiff/../".
解决方案: Strip it out.
文件: src/Makefile

Patch 8.1.0383
问题: Missing source file rename.
解决方案: Update the dependency.
文件: src/Make_mvc.mak

Patch 8.1.0384
问题: Sign ordering depends on +netbeans feature.
解决方案: Also order signs without +netbeans. (Christian Brabandt,
           closes #3224)
文件: src/structs.h, src/buffer.c

Patch 8.1.0385
问题: Coveralls badge doesn't update.
解决方案: Update the URL
文件: README.md

Patch 8.1.0386
问题: Cannot test with non-default option value.
解决方案: Add test_option_not_set().
文件: runtime/doc/eval.txt, src/option.c, src/proto/option.pro,
           src/evalfunc.c

Patch 8.1.0387
问题: No test for 'ambiwidth' detection.
解决方案: Add a test.

文件: src/testdir/test_startup_utf8.vim

Patch 8.1.0388
问题: Coverity complains about possible NULL pointer use.
解决方案: Use get_tv_string() instead of get_tv_string_chk().
文件: src/evalfunc.c

Patch 8.1.0389
问题: :behave command is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3429)
文件: src/Make_all.mak, src/testdir/test_alot.vim,
            src/testdir/test_behave.vim

Patch 8.1.0390
问题: Scrollbars are not tested.
解决方案: Add test_scrollbar() and a test.
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_gui.vim

Patch 8.1.0391
问题: Building in a shadow directory fails.
解决方案: Don't link the xdiff directory but what's in it. (closes #3428)
文件: src/Makefile

Patch 8.1.0392
问题: Error while typing :/foo/s// with 'incsearch' enabled.
解决方案: Do not give search errors when highlighting matches.
文件: src/ex_docmd.c, src/proto/ex_docmd.pro, src/ex_getln.c,
            src/testdir/test_search.vim

Patch 8.1.0393
问题: Not all white space difference options available.
解决方案: Add "iblank", "iwhiteall" and "iwhiteeol" to 'diffopt'.
文件: src/diff.c, src/testdir/test_diffmode.vim,
            src/testdir/dumps/Test_diff_17.dump,
            src/testdir/dumps/Test_diff_18.dump,
            src/testdir/dumps/Test_diff_19.dump,
            src/testdir/dumps/Test_diff_20.dump

Patch 8.1.0394
问题: Diffs are not always updated correctly.
解决方案: When using internal diff update for any changes properly.
文件: src/structs.h, src/diff.c, src/proto/diff.pro, src/misc1.c,
            src/main.c

Patch 8.1.0395
问题: Compiler warning on 64-bit MS-Windows.
解决方案: Add type cast. (Mike Williams)
文件: src/diff.c

Patch 8.1.0396
问题: Another compiler warning on 64-bit MS-Windows.
解决方案: Add type cast. (Mike Williams)
文件: src/xdiff/xutils.c

Patch 8.1.0397
问题:　　No event triggered after updating diffs.
解决方案:　Add the DiffUpdated event.
文件:　　src/vim.h, src/diff.c, src/fileio.c,
　　src/testdir/test_diffmode.vim, runtime/doc/autocmd.txt

Patch 8.1.0398
问题:　　No test for -o and -O command line arguments.
解决方案:　Add a test. (Dominique Pelle, closes #3438)
文件:　　src/testdir/test_startup.vim

Patch 8.1.0399
问题:　　'hlsearch' highlight remains in other window after cancelling
　　command.
解决方案:　Redraw all windows. Also remove unnecessary delays. (closes #3437)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_11.dump,
　　src/testdir/dumps/Test_incsearch_substitute_12.dump,
　　src/testdir/dumps/Test_incsearch_substitute_13.dump

Patch 8.1.0400
问题:　　Using freed memory with :diffget.
解决方案:　Skip ex_diffupdate() while updating diffs. (closes #3442)
文件:　　src/diff.c

Patch 8.1.0401
问题:　　Can't get swap name of another buffer.
解决方案:　Add swapname(). (Ozaki Kiichi, closes #3441)
文件:　　runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_swap.vim

Patch 8.1.0402
问题:　　The DiffUpdate event isn't triggered for :diffput.
解决方案:　Also trigger DiffUpdate for :diffget and :diffput.
文件:　　src/diff.c

Patch 8.1.0403
问题:　　Header file missing from distribution.
解决方案:　Add src/protodef.h.
文件:　　Filelist

Patch 8.1.0404
问题:　　Accessing invalid memory with long argument name.
解决方案:　Use item_count instead of checking for a terminating NULL.
　　(Dominique Pelle, closes #3444)
文件:　　src/testdir/test_arglist.vim, src/version.c

Patch 8.1.0405
问题:　　Too many #ifdefs for GTK.
解决方案:　Define macros instead of using #ifdef. (Ken Takata, closes #3436)
文件:　　src/gui_beval.c, src/gui_gtk.c, src/gui_gtk_f.c,
　　src/gui_gtk_x11.c, src/vim.h

Patch 8.1.0406
问题:　　Several command line arguments are not tested.

解决方案: Add tests for -A, -F, -H, -p and -V. (Dominique Pelle,
           closes #3446)
文件: src/testdir/test_startup.vim

Patch 8.1.0407
问题: Quickfix code mixes using the stack and a list pointer.
解决方案: Use a list pointer in more places. (Yegappan Lakshmanan,
           closes #3443)
文件: src/quickfix.c

Patch 8.1.0408
问题: MSVC: cannot use the "x64" native compiler option.
解决方案: Ignore case for %Platform%. Improve documentation. (Ken Takata)
文件: src/INSTALLpc.txt, src/msvc2015.bat

Patch 8.1.0409 (after 8.1.0406)
问题: Startup test fails on MS-Windows.
解决方案: Do the Arabic test in silent Ex mode. Loosen the check for -V2.
文件: src/testdir/test_startup.vim

Patch 8.1.0410
问题: The ex_copen() function is too long.
解决方案: Refactor to split off two functions. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0411
问题: Renamed file missing from distribution.
解决方案: Rename screen.c to termscreen.c (Zdenek Dohnal, closes #3449)
文件: Filelist

Patch 8.1.0412
问题: Cannot build with GTK 2.4.
解决方案: Add back a few #ifdefs. (Ken Takata, closes #3447)
           Also support older GTK. (Tom Christensen)
文件: src/gui_gtk_x11.c

Patch 8.1.0413
问题: Test output is duplicated or missing.
解决方案: Adjust the MS-Windows and Unix test makefiles. (Ken Takata,
           closes #3452)
文件: src/testdir/Make_dos.mak, src/testdir/Makefile

Patch 8.1.0414
问题: v:option_old and v:option_new are cleared when using :set in
           OptionSet autocmd. (Gary Johnson)
解决方案: Don't trigger OptionSet recursively.
文件: src/option.c

Patch 8.1.0415
问题: Not actually using 16 colors with vtp.
解决方案: Always use 256 colors when vtp is used. (Nobuhiro Takasaki,
           closes #3432)
文件: src/option.c, src/term.c

Patch 8.1.0416
问题: Sort doesn't report deleted lines.
解决方案: Call msgmore(). (Christian Brabandt, closes #3454)
文件: src/ex_cmds.c, src/testdir/test_sort.vim

Patch 8.1.0417
问题: Several command line arguments are not tested.
解决方案: Add tests for -m, -M, -R and -Vfile. (Dominique Pelle,
            closes #3458)
文件: src/testdir/test_startup.vim

Patch 8.1.0418
问题: MS-Windows: cannot separate Lua include and library directories.
解决方案: Add LUA_LIBDIR and LUA_INCDIR. (Ken Takata, closes #3464)
文件: src/Make_cyg_ming.mak

Patch 8.1.0419
问题: Cygwin: running cproto fails with -O2.
解决方案: Strip -O2 for cproto. (Ken Takata, closes #3465)
文件: src/Makefile

Patch 8.1.0420
问题: Generating vim.lib when using ActivePerl 5.20.3 or later.
解决方案: Redefine XS_EXTERNAL(). (Ken Takata, closes #3462)
文件: src/if_perl.xs

Patch 8.1.0421
问题: MS-Windows: Ruby path is wrong for Ruby 1.9 and later.
解决方案: Let -I argument depend on Ruby version. (Ken Takata, closes #3461)
文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.0422
问题: Cannot create map file with MinGW.
解决方案: Add support for $MAP. (Ken Takata, closes #3460)
文件: src/Make_cyg_ming.mak

Patch 8.1.0423
问题: MS-Windows: using dup-close for flushing a file.
解决方案: Use _commit(). (Ken Takata, closes #3463)
文件: src/memfile.c, src/os_mac.h, src/os_win32.h

Patch 8.1.0424
问题: Test output is very verbose, loading CI log is slow.
解决方案: Redirect output to /dev/null. (Ken Takata, closes #3456)
文件: src/testdir/Makefile

Patch 8.1.0425
问题: ml_get error and crash with appendbufline(). (Masashi Iizuka)
解决方案: Set per-window buffer info. (Hirohito Higashi, closes #3455)
文件: src/buffer.c, src/testdir/test_bufline.vim

Patch 8.1.0426
问题: Accessing invalid memory in SmcOpenConnection().
解决方案: Reduce size of errorstring by one. (Dominique Pelle, closes #3469)

文件:　　src/os_unix.c, src/testdir/test_startup.vim

Patch 8.1.0427
问题:　　MS-Windows GUI: using invalid encoded file name.
解决方案:　Drop the file name and return NULL. (Ken Takata, closes #3467)
文件:　　src/gui_w32.c

Patch 8.1.0428
问题:　　The :suspend command is not tested.
解决方案:　Add a test. (Dominique Pelle, closes #3472)
文件:　　src/Make_all.mak, src/testdir/test_alot.vim,
　　src/testdir/test_suspend.vim

Patch 8.1.0429 (after 8.1.0343)
问题:　　No test for :lcd with 'shellslash'.
解决方案:　Add a test. (Daniel Hahler, closes #3475)
文件:　　src/testdir/test_getcwd.vim

Patch 8.1.0430
问题:　　Xargadd file left behind after running test.
解决方案:　Delete the file. (Dominique Pelle)
文件:　　src/testdir/test_arglist.vim

Patch 8.1.0431
问题:　　The qf_jump() function is too long.
解决方案:　Refactor to split it into several functions. (Yegappan Lakshmanan)
文件:　　src/quickfix.c

Patch 8.1.0432
问题:　　Compiler warning for signed/unsigned.
解决方案:　Add type cast. (Mike Williams)
文件:　　src/xdiff/xemit.c

Patch 8.1.0433
问题:　　Mapping can obtain text from inputsecret(). (Tommy Allen)
解决方案:　Disallow CTRL-R = and CTRL-\ e when using inputsecret().
文件:　　src/ex_getln.c

Patch 8.1.0434
问题:　　copy_loclist() is too long.
解决方案:　Split in multiple functions. (Yegappan Lakshmanan)
文件:　　src/proto/quickfix.pro, src/quickfix.c, src/window.c

Patch 8.1.0435
问题:　　Cursorline highlight not removed in some situation. (Vitaly
　　Yashin)
解决方案:　Reset last_cursorline when resetting 'cursorline'. (Christian
　　Brabandt, closes #3481)
文件:　　src/move.c, src/proto/move.pro, src/option.c

Patch 8.1.0436
问题:　　Can get the text of inputsecret() with getcmdline(). (Tommy Allen)
解决方案:　Don't return the text.
文件:　　src/ex_getln.c

Patch 8.1.0437
问题: May access freed memory when syntax HL times out. (Philipp Gesang)
解决方案: Clear b_sst_first when clearing b_sst_array.
文件: src/syntax.c

Patch 8.1.0438
问题: The ex_make() function is too long.
解决方案: Split it into several functions. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0439
问题: Recursive use of getcmdline() still not protected.
解决方案: Instead of saving the command buffer when making a call which may
            cause recursiveness, save the buffer when actually being called
            recursively.
文件: src/ex_getln.c, src/proto/ex_getln.pro, src/getchar.c, src/main.c

Patch 8.1.0440
问题: remove() with a range not sufficiently tested.
解决方案: Add a test. (Dominique Pelle, closes #3497)
文件: src/testdir/test_listdict.vim

Patch 8.1.0441
问题: Build failure without command line history.
解决方案: Move cmdline_init() outside of #ifdef.
文件: src/ex_getln.c

Patch 8.1.0442
问题: GUI: Cursor not drawn after ":redraw | sleep".
解决方案: Flush the output. (closes #3496)
文件: src/ex_docmd.c

Patch 8.1.0443
问题: Unnecessary static function prototypes.
解决方案: Remove unnecessary prototypes.
文件: src/arabic.c, src/blowfish.c, src/buffer.c, src/charset.c,
            src/crypt_zip.c, src/digraph.c, src/edit.c, src/eval.c,
            src/evalfunc.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c,
            src/ex_eval.c, src/ex_getln.c, src/fileio.c, src/getchar.c,
            src/gui.c, src/gui_at_fs.c, src/gui_athena.c, src/gui_gtk_x11.c,
            src/gui_mac.c, src/gui_motif.c, src/gui_photon.c, src/gui_w32.c,
            src/gui_x11.c, src/hangulin.c, src/hardcopy.c, src/if_cscope.c,
            src/if_mzsch.c, src/if_python3.c, src/if_xcmdsrv.c,
            src/integration.c, src/json.c, src/main.c, src/mbyte.c,
            src/memline.c, src/message.c, src/misc1.c, src/misc2.c,
            src/move.c, src/netbeans.c, src/normal.c, src/ops.c, src/option.c,
            src/os_unix.c, src/os_win32.c, src/pty.c, src/regexp.c,
            src/screen.c, src/search.c, src/sha256.c, src/spell.c,
            src/spellfile.c, src/syntax.c, src/tag.c, src/term.c, src/ui.c,
            src/undo.c, src/version.c, src/window.c, src/workshop.c

Patch 8.1.0444
问题: Unnecessary check for NULL pointer.

解决方案: Remove check and call vim_free() directly.
文件: src/beval.c

Patch 8.1.0445
问题: Setting 'term' does not store location for termcap options.
解决方案: Set the script context for termcap options that are changed when 'term' is set.
文件: src/option.c, src/proto/option.pro, src/term.c, src/testdir/test_options.vim

Patch 8.1.0446
问题: Options test fails in the GUI.
解决方案: Don't try changing 'term' in the GUI.
文件: src/testdir/test_options.vim

Patch 8.1.0447
问题: GUI scrollbar test fails with Athena and Motif.
解决方案: When not using on-the-fly scrolling call normal_cmd().
文件: src/evalfunc.c, src/ex_docmd.c, src/proto/ex_docmd.pro

Patch 8.1.0448
问题: Cursorline not removed when using 'cursorbind'. (Justin Keyes)
解决方案: Store the last cursor line per window. (closes #3488)
文件: src/testdir/test_diffmode.vim, src/testdir/dumps/Test_diff_with_cursorline_01.dump, src/testdir/dumps/Test_diff_with_cursorline_02.dump, src/testdir/dumps/Test_diff_with_cursorline_03.dump, src/structs.h, src/move.c

Patch 8.1.0449
问题: When 'rnu' is set folded lines are not displayed correctly. (Vitaly Yashin)
解决方案: When only redrawing line numbers do draw folded lines. (closes #3484)
文件: src/screen.c, src/testdir/test_fold.vim, src/testdir/dumps/Test_folds_with_rnu_01.dump, src/testdir/dumps/Test_folds_with_rnu_02.dump

Patch 8.1.0450 (after patch 8.1.0449)
问题: Build failure without the +fold feature.
解决方案: Add #ifdef.
文件: src/screen.c

Patch 8.1.0451
问题: Win32 console: keypad keys don't work.
解决方案: Use numbers instead of characters to avoid the value becoming negative. (Mike Williams)
文件: src/os_win32.c

Patch 8.1.0452
问题: MS-Windows: not finding intl.dll.
解决方案: Also find intl.dll next to libintl.dll. (Ken Takata)
文件: src/os_win32.c, runtime/doc/mlang.txt

Patch 8.1.0453
问题: MS-Windows: executable() is not reliable.
解决方案: Use $PATHEXT properly. (Yasuhiro Matsumoto, closes #3512)
文件: src/os_win32.c, src/testdir/test_functions.vim

Patch 8.1.0454
问题: resolve() was not tested with a symlink cycle.
解决方案: Add a test. (Dominique Pelle, closes #3513)
文件: src/testdir/test_functions.vim

Patch 8.1.0455
问题: Checking for empty quickfix stack is not consistent.
解决方案: Use qf_stack_empty(). (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0456
问题: Running test hangs when the input file is being edited.
解决方案: Use a SwapExists autocommand to ignore editing the test script.
文件: src/testdir/Makefile, src/testdir/runtest.vim

Patch 8.1.0457 (after 8.1.0451)
问题: Win32 console: key mappings don't work.
解决方案: Use another solution for the keypad keys that doesn't break
            mappings. Some values will be negative. (Mike Williams)
文件: src/os_win32.c

Patch 8.1.0458
问题: Ml_get error and crash when using "do".
解决方案: Adjust cursor position also when diffupdate is not needed.
            (Hirohito Higashi)
文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.1.0459
问题: Test_executable fails when there is a dog in the system.
解决方案: Rename the dog. (Hirohito Higashi)
文件: src/testdir/test_functions.vim

Patch 8.1.0460
问题: assert_fails() does not take a message argument
解决方案: Add the argument.
文件: src/evalfunc.c, src/eval.c, src/testdir/test_assert.vim

Patch 8.1.0461
问题: Quickfix code uses too many /* */ comments.
解决方案: Change to // comments. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0462
问题: When using ConPTY Vim can be a child process.
解决方案: To find a Vim window use both EnumWindows() and
            EnumChildWindows(). (Nobuhiro Takasaki, closes #3521)
文件: src/os_mswin.c

Patch 8.1.0463

问题: "simalt ~x" in .vimrc blocks swap file prompt.
解决方案: Flush buffers before prompting. (Yasuhiro Matsumoto,
            closes #3518, closes #2192)
文件: src/memline.c

Patch 8.1.0464
问题: MS-Windows: job_info() has cmd without backslashes. (Daniel
            Hahler)
解决方案: Use rem_backslash(). (closes #3517, closes #3404) Add a test.
            (Yasuhiro Matsumoto)
文件: src/misc2.c, src/testdir/test_channel.vim

Patch 8.1.0465 (after 8.1.0452)
问题: Client-server test fails.
解决方案: Change logic in EnumWindows().
文件: src/os_mswin.c

Patch 8.1.0466 (after 8.1.0463)
问题: Autocmd test fails.
解决方案: Do call inchar() when flushing typeahead.
文件: src/vim.h, src/getchar.c, src/proto/getchar.pro, src/memline.c,
            src/message.c, src/misc1.c

Patch 8.1.0467 (after 8.1.0063)
问题: Cannot build with Mac OS X 10.5.
解决方案: Change #ifdef into #if. (Akshay Hegde, closes #3022)
文件: src/os_macosx.m

Patch 8.1.0468
问题: MS-Windows: Filter command with pipe character fails. (Johannes
            Riecken)
解决方案: Find the pipe character outside of quotes. (Yasuhiro Matsumoto,
            closes #1743, closes #3523)
文件: src/ex_cmds.c, src/testdir/test_filter_cmd.vim

Patch 8.1.0469
问题: Too often indexing in qf_lists[].
解决方案: Use a qf_list_T pointer. (Yegappan Lakshmanan)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0470
问题: Pointer ownership around fname_expand() is unclear.
解决方案: Allow b_ffname and b_sfname to point to the same allocated memory,
            only free one. Update comments.
文件: src/buffer.c, src/structs.h, src/fileio.c, src/ex_cmds.c

Patch 8.1.0471
问题: Some tests are flaky or fail on some systems.
解决方案: Increase waiting time for port number. Use "cmd /c" to execute
            "echo" on win32. (Ken Takata, closes #3534)
文件: src/testdir/shared.vim, src/testdir/test_channel.vim

Patch 8.1.0472
问题: Dosinst command has a few flaws.

解决方案: Register DisplayIcon, DisplayVersion and Publisher for the
uninstaller. (closes #3485) Don't set 'diffexpr' if internal diff
is supported. Allow for using Vi compatible from the command line.
Remove needless sleeps. Add comments in the generated _vimrc.
(Ken Takata, closes #3525)
文件: src/dosinst.c

Patch 8.1.0473
问题: User doesn't notice file does not exist when swap file does.
解决方案: Add a note that the file cannot be found. Make the "still
running" notice stand out.
文件: src/memline.c

Patch 8.1.0474
问题: Directory where if_perl.c is written is inconsistent.
解决方案: use auto/if_perl.c for MS-Windows. (Ken Takata, closes #3540)
文件: src/Make_bc5.mak, src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.0475
问题: Memory not freed on exit when quit in autocmd.
解决方案: Remember funccal stack when executing autocmd.
文件: src/structs.h, src/userfunc.c, src/proto/userfunc.pro,
src/fileio.c, src/eval.c, src/ex_cmds2.c, src/main.c

Patch 8.1.0476
问题: Memory leaks in test_escaped_glob.
解决方案: Avoid failure when running the shell, use the sandbox.
文件: src/testdir/test_escaped_glob.vim

Patch 8.1.0477 (after 8.1.0475)
问题: Tiny build fails.
解决方案: Add a dummy declaration for funccal_entry_T.
文件: src/structs.h

Patch 8.1.0478
问题: Cannot build with perl using MinGW.
解决方案: Add -I. (Ken Takata, Cesar Romani)
文件: src/Make_cyg_ming.mak

Patch 8.1.0479
问题: Failure when setting 'varsofttabstop' to end in a comma. (Ralf
Schandl)
解决方案: Reject value with trailing comma. Add test for invalid values
(closes #3544)
文件: src/testdir/test_vartabs.vim, src/option.c

Patch 8.1.0480
问题: MinGW build file uses different -I flags than MVC.
解决方案: Add -I to $CFLAGS. (Ken Takata)
文件: src/Make_cyg_ming.mak

Patch 8.1.0481
问题: When "Terminal" highlight is reverted cursor doesn't show.
解决方案: Get the colors of the "Terminal" group. (closes #3546)

文件: src/terminal.c

Patch 8.1.0482
问题: MinGW "make clean" deletes all .exe files.
解决方案: Only delete .exe files that it builds. (Ken Takata)
文件: src/Make_cyg_ming.mak

Patch 8.1.0483
问题: MinGW does not build tee.exe.
解决方案: Add build instructions. (Yasuhiro Matsumoto, closes #3548)
文件: src/Make_cyg_ming.mak, src/tee/Makefile

Patch 8.1.0484
问题: Some file types are not recognized.
解决方案: Update the file type detection.
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.0485
问题: term_start() does not check if directory is accessible.
解决方案: Add mch_access() call. (Jason Franklin)
文件: src/channel.c, src/testdir/test_terminal.vim

Patch 8.1.0486 (after 8.1.0485)
问题: Can't build in MS-Windows.
解决方案: Put mch_access() call inside #ifdef
文件: src/channel.c

Patch 8.1.0487
问题: No menus specifically for the terminal window.
解决方案: Add :tlmenu. (Yee Cheng Chin, closes #3439) Add a menu test.
文件: runtime/delmenu.vim, runtime/doc/autocmd.txt, runtime/doc/gui.txt,
            runtime/doc/index.txt, runtime/doc/terminal.txt,
            runtime/doc/usr_42.txt, runtime/menu.vim, src/ex_cmdidxs.h,
            src/ex_cmds.h, src/ex_docmd.c, src/menu.c, src/proto/menu.pro,
            src/popupmnu.c, src/structs.h, src/testdir/test_menu.vim

Patch 8.1.0488
问题: Using freed memory in quickfix code. (Dominique Pelle)
解决方案: Add the quickfix_busy() flag to postpone deleting quickfix lists
            until it is safe. (Yegappan Lakshmanan, closes #3538)
文件: src/quickfix.c, src/proto/quickfix.pro, src/misc2.c,
            src/testdir/test_quickfix.vim

Patch 8.1.0489
问题: Crash when autocmd clears vimpgrep location list.
解决方案: Return from qf_jump_edit_buffer() early. (Yegappan Lakshmanan)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0490
问题: MS-Windows: doesn't handle missing libwinpthread-1.dll.
解决方案: Adjust Cygwin/MinGW build file. (Ken Takata, closes #2827)
文件: src/Make_cyg_ming.mak

Patch 8.1.0491

问题: If a terminal dump has CR it is considered corrupt.
解决方案: Ignore CR characters. (Nobuhiro Takasaki, closes #3558)
文件: src/terminal.c

Patch 8.1.0492
问题: "Edit with existing Vim" list can get long.
解决方案: Move the list to a submenu. (Ken Takata, closes #3561)
文件: src/GvimExt/gvimext.cpp

Patch 8.1.0493
问题: argv() and argc() only work on the current argument list.
解决方案: Add a window ID argument. (Yegappan Lakshmanan, closes #832)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_arglist.vim,
            src/eval.c, src/proto/eval.pro

Patch 8.1.0494
问题: Functions do not check for a window ID in other tabs.
解决方案: Also find the window ID in other than the current tab.
文件: src/evalfunc.c

Patch 8.1.0495
问题: :filter only supports some commands.
解决方案: Add :filter support for more commands. (Marcin Szamotulski,
            closes #2856)
文件: runtime/doc/various.txt, src/eval.c, src/mark.c, src/option.c,
            src/syntax.c, src/testdir/test_filter_cmd.vim, src/userfunc.c

Patch 8.1.0496
问题: No tests for indent files.
解决方案: Add a mechanism for running indent file tests. Add a first test
            for Vim indenting.
文件: runtime/indent/Makefile, runtime/indent/testdir/runtest.vim,
            runtime/indent/testdir/cleantest.vim, runtime/indent/README.txt,
            runtime/indent/testdir/README.txt, runtime/indent/testdir/vim.in,
            runtime/indent/testdir/vim.ok, Filelist

Patch 8.1.0497
问题: :%diffput changes order of lines. (Markus Braun)
解决方案: Do adjust marks when using internal diff.
文件: src/diff.c, src/testdir/test_diffmode.vim

Patch 8.1.0498
问题: /etc/gitconfig not recognized at a gitconfig file.
解决方案: Add pattern to filetype detection. (closes #3568)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.0499
问题: :2vimgrep causes an ml_get error
解决方案: Pass tomatch pointer instead of value. (Yegappan Lakshmanan)
文件: src/ex_getln.c, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0500
问题: Cleaning up in src/tee may not always work.
解决方案: Use "rm" when appropriate. (Michael Soyka, closes #3571)

文件: src/tee/Makefile

Patch 8.1.0501
问题: Cppcheck warns for using array index before bounds check.
解决方案: Swap the conditions. (Dominique Pelle)
文件: src/memline.c

Patch 8.1.0502
问题: Internal diff fails when diffing a context diff. (Hirohito Higashi)
解决方案: Only use callback calls with one line. (closes #3581)
文件: src/diff.c, src/testdir/dumps/test_diff_of_diff_01.dump

Patch 8.1.0503
问题: Missing change to diff test. (Hirohito Higashi)
解决方案: Add the missing test function.
文件: src/testdir/test_diffmode.vim

Patch 8.1.0504
问题: When CTRL-C is mapped it triggers InsertLeave.
解决方案: Make CTRL-C behave the same way when typed or used in a mapping.
文件: src/edit.c, src/testdir/test_edit.vim

Patch 8.1.0505
问题: Filter command test may fail if helplang is not set.
解决方案: Set 'helplang' for the test. (James McCoy, closes #3591)
文件: src/testdir/test_filter_cmd.vim

Patch 8.1.0506
问题: Modeline test fails when run by root.
解决方案: Set 'modeline' for the test. (James McCoy, closes #3592)
文件: src/testdir/test_modeline.vim

Patch 8.1.0507
问题: .raml files not properly detected.
解决方案: Recognize .raml as raml instead of yaml. (closes #3594)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.0508
问题: Suspend test fails when run by root.
解决方案: Accept both '$' and '#' for the prompt. (James McCoy, closes #3590)
文件: src/testdir/test_suspend.vim

Patch 8.1.0509
问题: Checking cwd not accessible fails for root. (James McCoy)
解决方案: Skip this part of the test for root. (closes #3595)
文件: src/testdir/test_terminal.vim

Patch 8.1.0510
问题: Filter test fails when $LANG is C.UTF-8.
解决方案: Set 'helplang' to "en" for any C language. (Christian Brabandt,
            closes #3577)
文件: src/option.c

Patch 8.1.0511

问题:　　ml_get error when calling a function with a range.
解决方案:　Don't position the cursor after the last line.
文件:　　src/userfunc.c, src/testdir/test_functions.vim

Patch 8.1.0512
问题:　　'helplang' default is inconsistent for C and C.UTF-8.
解决方案:　Don't accept a value unless it starts with two letters.
文件:　　src/ex_cmds2.c

Patch 8.1.0513
问题:　　No error for set diffopt+=algorithm:.
解决方案:　Check for missing argument. (Hirohito Higashi, closes #3598)
文件:　　src/diff.c, src/testdir/gen_opt_test.vim

Patch 8.1.0514
问题:　　CTRL-W ^ does not work when alternate buffer has no name.
解决方案:　Use another method to split and edit the alternate buffer. (Jason
　　Franklin)
文件:　　src/testdir/test_normal.vim, src/testdir/test_window_cmd.vim,
　　src/normal.c, src/window.c, runtime/doc/windows.txt

Patch 8.1.0515
问题:　　Reloading a script gives errors for existing functions.
解决方案:　Allow redefining a function once when reloading a script.
文件:　　src/testdir/test_functions.vim, src/userfunc.c, src/structs.h,
　　src/globals.h, src/buffer.c, src/ex_cmds2.c, src/main.c,
　　src/option.c, runtime/doc/eval.txt

Patch 8.1.0516
问题:　　:move command marks buffer modified when nothing changed.
解决方案:　Do not set 'modified'. Add a test. (Jason Franklin)
文件:　　src/Make_all.mak, src/testdir/test_alot.vim,
　　src/testdir/test_move.vim, src/ex_cmds.c

Patch 8.1.0517
问题:　　Test_window_split_edit_alternate() fails on AppVeyor.
解决方案:　Disable the failing part for now.
文件:　　src/testdir/test_window_cmd.vim

Patch 8.1.0518
问题:　　Test_window_split_edit_bufnr() fails on AppVeyor.
解决方案:　Disable the failing part for now.
文件:　　src/testdir/test_window_cmd.vim

Patch 8.1.0519
问题:　　Cannot save and restore the tag stack.
解决方案:　Add gettagstack() and settagstack(). (Yegappan Lakshmanan,
　　closes #3604)
文件:　　runtime/doc/eval.txt, runtime/doc/tagsrch.txt,
　　runtime/doc/usr_41.txt, src/alloc.h, src/dict.c, src/evalfunc.c,
　　src/list.c, src/misc2.c, src/proto/dict.pro, src/proto/list.pro,
　　src/proto/misc2.pro, src/proto/tag.pro, src/tag.c,
　　src/testdir/test_tagjump.vim

Patch 8.1.0520
问题: Screen diff test sometimes fails.
解决方案: Add to list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0521
问题: Cannot build with +eval but without +quickfix.
解决方案: Remove #ifdef for e_stringreq. (John Marriott)
文件: src/evalfunc.c

Patch 8.1.0522
问题: :terminal does not show trailing empty lines.
解决方案: Add empty lines. (Hirohito Higashi, closes #3605)
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0523
问题: Opening window from quickfix leaves empty buffer behind.
解决方案: Add qf_jump_newwin(). (Yegappan Lakshmanan, closes #2574)
文件: src/proto/quickfix.pro, src/quickfix.c,
            src/testdir/test_quickfix.vim

Patch 8.1.0524 (after 8.1.0522)
问题: Terminal test fails on Windows.
解决方案: Skip Test_terminal_does_not_truncate_last_newlines() for now.
文件: src/testdir/test_terminal.vim

Patch 8.1.0525 (after 8.1.0524)
问题: Terminal test skips part on Windows.
解决方案: Fix Test_terminal_does_not_truncate_last_newlines(). (Hirohito
            Higashi, closes #3606)
文件: src/Make_mvc.mak, src/testdir/test_terminal.vim

Patch 8.1.0526
问题: Running out of signal stack in RealWaitForChar. (Vladimir Marek)
解决方案: Make the fd_set variables static.
文件: src/os_unix.c

Patch 8.1.0527
问题: Using 'shiftwidth' from wrong buffer for folding.
解决方案: Use "buf" instead of "curbuf". (Christian Brabandt)
文件: src/fold.c

Patch 8.1.0528
问题: Various typos in comments.
解决方案: Fix the typos.
文件: src/fileio.c, src/gui.c, src/macros.h, src/screen.c, src/search.c,
            src/spell.c, src/spellfile.c, src/vim.h, src/testdir/README.txt,
            src/INSTALL, src/gui_athena.c, src/gui_gtk.c, src/gui_gtk_x11.c,
            src/gui_motif.c, src/gui_xmebw.c, src/if_tcl.c, src/os_amiga.c,
            src/gui_w32.c, src/os_win32.c, src/gui_mac.c, src/os_vms_fix.com

Patch 8.1.0529
问题: Flaky test sometimes fails in different ways.
解决方案: When the second run gives a different error, try running the test

again, up to five times.
文件: src/testdir/runtest.vim

Patch 8.1.0530
问题: Channel and terminal tests that start a server can be flaky.
解决方案: Add all channel and terminal tests that start a server to the list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0531
问题: Flaky tests often fail with a common error message.
解决方案: Add a pattern to match an error message indicating a flaky test.
文件: src/testdir/runtest.vim

Patch 8.1.0532
问题: Cannot distinguish between quickfix and location list.
解决方案: Add an explicit type variable. (Yegappan Lakshmanan)
文件: src/quickfix.c

Patch 8.1.0533
问题: Screendump tests can be flaky.
解决方案: Add VerifyScreenDump to the pattern of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0534
问题: MS-Windows installer uses different $HOME than Vim.
解决方案: Use the Vim logic also in the MS-Windows installer. (Ken Takata, closes #3564)
文件: src/dosinst.c, src/misc1.c

Patch 8.1.0535
问题: Increment/decrement might get interrupted by updating folds.
解决方案: Disable fold updating for a moment. (Christian Brabandt, closes #3599)
文件: src/ops.c

Patch 8.1.0536
问题: File time test fails when using NFS.
解决方案: Use three file times instead of localtim(). (James McCoy, closes #3618)
文件: src/testdir/test_stat.vim

Patch 8.1.0537
问题: ui_breakcheck() may be called recursively, which doesn't work.
解决方案: When called recursively, just return. (James McCoy, closes #3617)
文件: src/ui.c

Patch 8.1.0538
问题: Evaluating a modeline might invoke using a shell command. (Paul Huber)
解决方案: Set the sandbox flag when setting options from a modeline.
文件: src/buffer.c

Patch 8.1.0539

问题:　　Cannot build without the sandbox.
解决方案:　Set the secure option instead of using the sandbox. Also restrict
the characters from 'spelllang' that are used for LANG.vim.
(suggested by Yasuhiro Matsumoto)
文件:　　runtime/doc/options.txt, src/buffer.c, src/option.c

Patch 8.1.0540
问题:　　May evaluate insecure value when appending to option.
解决方案:　Set the secure flag when changing an option that was previously
set insecurely. Also allow numbers for the characters from
'spelllang' that are used for LANG.vim. (closes #3623)
文件:　　src/option.c

Patch 8.1.0541
问题:　　Help message in dosinst.c is outdated.
解决方案:　Update the comment. (Ken Takata, closes #3626)
文件:　　src/dosinst.c

Patch 8.1.0542
问题:　　shiftwidth() does not take 'vartabstop' into account.
解决方案:　Use the cursor position or a position explicitly passed.
Also make >> and << work better with 'vartabstop'. (Christian
Brabandt)
文件:　　runtime/doc/change.txt, runtime/doc/eval.txt, src/edit.c,
src/evalfunc.c, src/normal.c, src/ops.c, src/option.c,
src/proto/edit.pro, src/proto/option.pro,
src/testdir/test_vartabs.vim

Patch 8.1.0543
问题:　　Coverity warns for leaking memory and using wrong struct.
解决方案:　Free pointer when allocation fails. Change "boff" to "loff".
(closes #3634)
文件:　　src/ex_getln.c, src/move.c

Patch 8.1.0544 (after 8.1.0540)
问题:　　Setting 'filetype' in a modeline causes an error (Hirohito
Higashi).
解决方案:　Don't add the P_INSECURE flag when setting 'filetype' from a
modeline. Also for 'syntax'.
文件:　　src/option.c, src/testdir/test_modeline.vim

Patch 8.1.0545
问题:　　When executing indent tests user preferences interfere.
解决方案:　Add "--clean".
文件:　　runtime/indent/Makefile, runtime/indent/testdir/runtest.vim

Patch 8.1.0546
问题:　　Modeline test with keymap fails.
解决方案:　Check that the keymap feature is available.
文件:　　src/testdir/test_modeline.vim

Patch 8.1.0547
问题:　　Modeline test with keymap still fails.
解决方案:　Check that the keymap feature is available for the failure assert.

文件: src/testdir/test_modeline.vim

Patch 8.1.0548
问题: Crash when job callback unloads a buffer. (James McCoy)
解决方案: Don't round up the wait time to 10 msec in ui_inchar().
文件: src/ui.c

Patch 8.1.0549
问题: Netbeans test depends on README.txt contents.
解决方案: Use a generated file instead.
文件: src/testdir/test_netbeans.vim, src/testdir/test_netbeans.py

Patch 8.1.0550
问题: Expression evaluation may repeat an error message. (Jason
            Franklin)
解决方案: Increment did_emsg and check for the value when giving an error
            for the echo command.
文件: src/message.c, src/eval.c, src/testdir/test108.ok

Patch 8.1.0551 (after 8.1.0550)
问题: Expression evaluation may repeat an error message. (Jason
            Franklin)
解决方案: Check for the value of did_emsg when giving an error
            for the :execute command.
文件: src/eval.c

Patch 8.1.0552
问题: Saved last search pattern may not be restored.
解决方案: Call restore_last_search_pattern(). Add a check for balancing
            saving and restoring the last search pattern.
文件: src/ex_getln.c, src/search.c

Patch 8.1.0553
问题: It is not easy to edit a script that was sourced.
解决方案: Add a count to ":scriptnames", so that ":script 40" edits the
            script with script ID 40.
文件: src/ex_cmds.h, src/ex_cmds2.c, src/testdir/test_scriptnames.vim,
            src/Make_all.mak, src/testdir/Make_all.mak, runtime/doc/repeat.txt

Patch 8.1.0554
问题: Popup menu overlaps with preview window.
解决方案: Adjust the height computation. (Hirohito Higashi, closes #3414)
文件: src/popupmnu.c, src/testdir/test_popup.vim,
            src/testdir/dumps/Test_popup_and_previewwindow_01.dump

Patch 8.1.0555
问题: Crash when last search pat is set but not last substitute pat.
解决方案: Do not mix up last search pattern and last substitute pattern.
            (closes #3647)
文件: src/search.c, src/testdir/test_search.vim

Patch 8.1.0556
问题: Saving/restoring search patterns share saved last_idx.
解决方案: Use a separate saved last_idx for saving search patterns for

functions and incremental search.
文件: src/search.c

Patch 8.1.0557
问题: Termdebug: gdb may use X.Y for breakpoint number. (Ryou Ezoe)
解决方案: Handle X.Y breakpoint numbers. (Yasuhiro Matsumoto, close #3641)
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0558
问题: Some MS-Windows instructions are outdated.
解决方案: Update the uninstall instructions and the NSIS README. (Ken Takata, closes #3614) Also update remark about diff.exe.
文件: nsis/README.txt, uninstal.txt

Patch 8.1.0559
问题: Command line completion not sufficiently tested.
解决方案: Add more tests. (Dominique Pelle, closes #3622)
文件: src/testdir/test_arglist.vim, src/testdir/test_filetype.vim, src/testdir/test_history.vim, src/testdir/test_messages.vim, src/testdir/test_syntax.vim

Patch 8.1.0560
问题: Cannot use address type "other" with user command.
解决方案: Add "other" to the list. (Daniel Hahler, closes #3655) Also reject "%" for commands with "other". Add some more tests.
文件: src/ex_docmd.c, src/testdir/test_usercommands.vim

Patch 8.1.0561
问题: MSVC error format has changed.
解决方案: Make the space between the line number and colon optional.
文件: src/option.h

Patch 8.1.0562
问题: Parsing of 'diffopt' is slightly wrong.
解决方案: Fix the parsing and add a test. (Jason Franklin, Christian Brabandt)
文件: src/diff.c, src/testdir/test_diffmode.vim, src/testdir/dumps/Test_diff_09.dump, src/testdir/dumps/Test_diff_11.dump, src/testdir/screendump.vim

Patch 8.1.0563
问题: Setting v:errors to a string give confusing error. (Christian Brabandt)
解决方案: Change internal error into normal error message.
文件: src/eval.c

Patch 8.1.0564
问题: Setting v:errors to wrong type still possible.
解决方案: Return after giving an error message. (Christian Brabandt)
文件: src/eval.c, src/testdir/test_eval_stuff.vim

Patch 8.1.0565
问题: Asan complains about reading before allocated block.
解决方案: Workaround: Avoid offset from becoming negative.

文件:　　src/gui.c

Patch 8.1.0566
问题:　　SGR not enabled for mintty because $TERM is "xterm".
解决方案:　Detect mintty by the termresponse. (Ken Takata, closes #3667)
文件:　　src/term.c

Patch 8.1.0567 (after 8.1.0565)
问题:　　Error for NUL byte in ScreenLines goes unnoticed.
解决方案:　Add an internal error message.
文件:　　src/gui.c

Patch 8.1.0568 (after 8.1.0567)
问题:　　Error message for NUL byte in ScreenLines breaks Travis CI.
解决方案:　Use a normal message fornow.
文件:　　src/gui.c

Patch 8.1.0569
问题:　　Execute() always resets display column to zero. (Sha Liu)
解决方案:　Don't reset it to zero, restore the previous value. (closes #3669)
文件:　　src/evalfunc.c, src/testdir/test_execute_func.vim

Patch 8.1.0570
问题:　　'commentstring' not used when adding fold marker. (Maxim Kim)
解决方案:　Only use empty 'comments' middle when leader is empty. (Christian
　　Brabandt, closes #3670)
文件:　　src/misc1.c, src/testdir/test_fold.vim

Patch 8.1.0571 (after 8.1.0569)
问题:　　Non-silent execute() resets display column to zero.
解决方案:　Keep the display column as-is.
文件:　　src/evalfunc.c, src/testdir/test_execute_func.vim

Patch 8.1.0572
问题:　　Stopping a job does not work properly on OpenBSD.
解决方案:　Do not use getpgid() to check the process group of the job
　　process ID, always pass the negative process ID to kill().
　　(George Koehler, closes #3656)
文件:　　src/os_unix.c

Patch 8.1.0573
问题:　　Cannot redefine user command without ! in same script
解决方案:　Allow redefining user command without ! in same script, like with
　　functions.
文件:　　src/ex_docmd.c, src/testdir/test_usercommands.vim,
　　runtime/doc/map.txt

Patch 8.1.0574
问题:　　'commentstring' not used when adding fold marker in C.
解决方案:　Require white space before middle comment part. (mostly by
　　Hirohito Higashi)
文件:　　src/misc1.c, src/testdir/test_fold.vim

Patch 8.1.0575

问题: Termdebug: clearing multi-breakpoint does not work.
解决方案: Delete all X.Y breakpoints. Keep more information about placed
            breakpoints. (Ozaki Kiichi, closes #3641)
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0576
问题: Indent script tests pick up installed scripts.
解决方案: Use current runtime indent scripts.
文件: runtime/indent/Makefile

Patch 8.1.0577
问题: Tabpage right-click menu never shows "Close tab".
解决方案: Always create the "Close tab" item but ignore the event if there
            is only one tab.
文件: src/gui_gtk_x11.c, src/gui_mac.c, src/gui_motif.c, src/gui.c

Patch 8.1.0578
问题: Cannot disable arabic, rightleft and farsi in configure.
解决方案: Add configure flags. (Diego Fernando Carrión, closes #1867)
文件: src/configure.ac, src/auto/configure, src/config.h.in,
            src/feature.h, src/Makefile

Patch 8.1.0579
问题: Cannot attach properties to text.
解决方案: First part of adding text properties.
文件: Filelist, runtime/doc/Makefile, runtime/doc/eval.txt,
            runtime/doc/textprop.txt, src/Make_all.mak, src/Make_cyg_ming.mak,
            src/Make_mvc.mak, src/Makefile, src/buffer.c, src/edit.c,
            src/evalfunc.c, src/feature.h, src/memline.c, src/misc1.c,
            src/misc2.c, src/proto.h, src/proto/memline.pro,
            src/proto/textprop.pro, src/screen.c, src/structs.h,
            src/testdir/Make_all.mak, src/testdir/test_textprop.vim,
            src/textprop.c, src/userfunc.c, src/version.c

Patch 8.1.0580
问题: Invalid memory access when using text properties.
解决方案: Disable text properties for now.
文件: src/feature.h

Patch 8.1.0581
问题: Double free without the text properties feature.
解决方案: Reset the dirty flag.
文件: src/memline.c

Patch 8.1.0582
问题: Text properties are not enabled.
解决方案: Fix sizeof argument and re-enable the text properties feature.
            Fix memory leak.
文件: src/feature.h, src/textprop.c

Patch 8.1.0583
问题: Using illogical name for get_dict_number()/get_dict_string().
解决方案: Rename to start with dict_.
文件: src/dict.c, src/proto/dict.pro, src/edit.c, src/eval.c,

src/evalfunc.c, src/quickfix.c, src/tag.c, src/terminal.c,
src/textprop.c

Patch 8.1.0584
问题: With search CTRL-L does not pick up composing characters.
解决方案: Check for composing characters. (Christian Brabandt, closes #3682)
[code change was accidentally included in 8.1.0579]
文件: src/testdir/test_search.vim

Patch 8.1.0585
问题: Undo test may fail on MS-Windows.
解决方案: Also handle lower case drive letters.
文件: src/testdir/test_undo.vim

Patch 8.1.0586
问题: :digraph output is not easy to read.
解决方案: Add highlighting for :digraphs. (Marcin Szamotulski, closes #3572)
Also add section headers for :digraphs!.
文件: src/ex_docmd.c, src/digraph.c, src/proto/digraph.pro,
src/ex_cmds.h, runtime/doc/digraph.txt

Patch 8.1.0587
问题: GvimExt: realloc() failing is not handled properly.
解决方案: Check for NULL return. (Jan-Jaap Korpershoek, closes #3689)
文件: src/GvimExt/gvimext.cpp

Patch 8.1.0588
问题: Cannot define a sign with space in the text.
解决方案: Allow for escaping characters. (Ben Jackson, closes #2967)
文件: src/ex_cmds.c, src/testdir/test_signs.vim

Patch 8.1.0589
问题: Compilation error in gvimext.cpp.
解决方案: Return a value. Also fix using uninitialized variable.
文件: src/GvimExt/gvimext.cpp, src/dosinst.c

Patch 8.1.0590
问题: When a job ends the closed channels are not handled.
解决方案: When a job is detected to have ended, check the channels again.
(closes #3530)
文件: src/channel.c, src/proto/channel.pro, src/misc2.c

Patch 8.1.0591
问题: Channel sort test is flaky.
解决方案: Do not check if the job is running, it may have be done very fast.
文件: src/testdir/test_channel.vim

Patch 8.1.0592
问题: The libvterm tests are not run as part of Vim tests.
解决方案: Add testing libvterm.
文件: src/Makefile, src/libvterm/Makefile

Patch 8.1.0593
问题: Illegal memory access in libvterm test.

解决方案: Fix off-by-one error.
文件: src/libvterm/src/vterm.c, src/libvterm/Makefile,
           src/libvterm/t/run-test.pl

Patch 8.1.0594
问题: Libvterm tests fail to run on Mac.
解决方案: Only run libvterm tests on Linux.
文件: src/Makefile

Patch 8.1.0595
问题: Libvterm tests are not run with coverage.
解决方案: Adjust the Travis config. Show the actually run commands.
文件: .travis.yml, src/libvterm/Makefile

Patch 8.1.0596
问题: Not all parts of printf() are tested.
解决方案: Add a few more test cases. (Dominique Pelle, closes #3691)
文件: src/testdir/test_expr.vim

Patch 8.1.0597
问题: Cannot run test_libvterm from the top directory.
解决方案: Add test target in toplevel Makefile.
文件: Makefile

Patch 8.1.0598
问题: Indent tests may use the wrong Vim binary.
解决方案: Pass in the just built Vim binary.
文件: Makefile

Patch 8.1.0599
问题: Without the +eval feature the indent tests don't work.
解决方案: Skip the body of the tests.
文件: runtime/indent/testdir/cleantest.vim,
           runtime/indent/testdir/runtest.vim

Patch 8.1.0600
问题: Channel test is flaky.
解决方案: Add test to list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0601
问题: A few compiler warnings.
解决方案: Add type casts. (Mike Williams)
文件: src/GvimExt/gvimext.cpp, src/memline.c, src/textprop.c

Patch 8.1.0602
问题: DirChanged is also triggered when the directory didn't change.
           (Daniel Hahler)
解决方案: Compare the current with the new directory. (closes #3697)
文件: src/ex_docmd.c, src/testdir/test_autocmd.vim, src/misc2.c,
           src/testdir/test_autochdir.vim

Patch 8.1.0603
问题: The :stop command is not tested.

解决方案: Test :stop using a terminal window.
文件: src/testdir/test_terminal.vim, src/testdir/shared.vim

Patch 8.1.0604
问题: Autocommand test fails on MS-Windows.
解决方案: Use pathcmp() instead of strcmp() to check if a directory differs.
文件: src/ex_docmd.c, src/misc2.c

Patch 8.1.0605
问题: Running make in the top directory echoes a comment.
解决方案: Prefix with @. (closes #3698)
文件: Makefile

Patch 8.1.0606
问题: 'cryptmethod' defaults to a very old method.
解决方案: Default to "blowfish2", it is now widely available.
文件: src/option.c, runtime/doc/options.txt

Patch 8.1.0607
问题: Proto files are not in sync with the source code.
解决方案: Update the proto files.
文件: src/os_mswin.c, src/proto/buffer.pro, src/proto/ex_cmds.pro,
           src/proto/ex_getln.pro, src/proto/misc2.pro,
           src/proto/userfunc.pro

Patch 8.1.0608
问题: Coveralls is not updating.
解决方案: Adjust path in Travis config.
文件: .travis.yml

Patch 8.1.0609
问题: MS-Windows: unused variable, depending on the Ruby version.
解决方案: Put ruby_sysinit and NtInitialize inside #ifdef and make them
           consistent. (Ken Takata)
文件: src/if_ruby.c

Patch 8.1.0610
问题: MS-Windows ctags file list differs from Unix.
解决方案: Define TAGS_FILES in the common makefile. (partly by Ken Takata)
文件: src/Make_all.mak, src/Makefile, src/Make_mvc.mak,
           src/Make_cyg_ming.mak

Patch 8.1.0611
问题: Crash when using terminal with long composing characters.
解决方案: Make space for all characters. (Yasuhiro Matsumoto, closes #3619,
           closes #3703)
文件: src/terminal.c

Patch 8.1.0612
问题: Cannot use two global runtime dirs with configure.
解决方案: Support a comma in --with-global-runtime. (James McCoy,
           closes #3704)
文件: src/config.h.in, src/configure.ac, src/feature.h, src/os_unix.h,
           src/auto/configure, src/Makefile

Patch 8.1.0613
问题: When executing an insecure function the secure flag is stuck.
(Gabriel Barta)
解决方案: Restore "secure" instead of decrementing it. (closes #3705)
文件: src/testdir/test_autocmd.vim, src/option.c, src/buffer.c

Patch 8.1.0614
问题: Placing signs can be complicated.
解决方案: Add functions for defining and placing signs. Introduce a group
name to avoid different plugins using the same signs. (Yegappan
Lakshmanan, closes #3652)
文件: runtime/doc/eval.txt, runtime/doc/sign.txt,
runtime/doc/usr_41.txt, src/alloc.h, src/buffer.c, src/evalfunc.c,
src/ex_cmds.c, src/globals.h, src/list.c, src/misc2.c,
src/netbeans.c, src/proto/buffer.pro, src/proto/ex_cmds.pro,
src/proto/list.pro, src/proto/misc2.pro, src/structs.h,
src/testdir/test_signs.vim, src/workshop.c

Patch 8.1.0615
问题: Get_tv function names are not consistent.
解决方案: Rename to tv_get.
文件: src/eval.c, src/proto/eval.pro, src/channel.c, src/dict.c,
src/evalfunc.c, src/list.c, src/message.c, src/tag.c,
src/terminal.c, src/textprop.c, src/window.c, src/ex_cmds.c,
src/os_unix.c, src/os_win32.c, src/json.c, src/regexp.c,
src/edit.c, src/misc2.c, src/popupmnu.c

Patch 8.1.0616
问题: NSIS installer is outdated.
解决方案: Use modern syntax, MUI2 and make it work better. Add translations.
(Guopeng Wen, Ken Takata, closes #3501)
文件: Filelist, nsis/gvim.nsi, nsis/icons/header.svg,
nsis/icons/welcome.svg, nsis/icons/header.bmp,
nsis/icons/un_header.bmp, nsis/icons/uninstall.bmp,
nsis/icons/welcome.bmp, nsis/lang/danish.nsi, nsis/lang/dutch.nsi,
nsis/lang/english.nsi, nsis/lang/german.nsi,
nsis/lang/italian.nsi, nsis/lang/japanese.nsi,
nsis/lang/simpchinese.nsi, nsis/lang/tradchinese.nsi,
src/dosinst.c

Patch 8.1.0617 (after 8.1.0616)
问题: NSIS installer gets two files from the wrong directory.
解决方案: Change ${VIMRT} to "..\".
文件: nsis/gvim.nsi

Patch 8.1.0618
问题: term_getjob() does not return v:null as documented.
解决方案: Do return v:null. (Damien) Add a test.
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0619
问题: :echomsg and :echoerr do not handle List and Dict like :echo does.
(Daniel Hahler)

解决方案: Be more tolerant about the expression result type.
文件: src/eval.c, src/proto/eval.pro, src/evalfunc.c,
src/proto/evalfunc.pro, runtime/doc/eval.txt,
src/testdir/test_messages.vim, src/message.c

Patch 8.1.0620
问题: Overruling CONF_ARGS from the environment no longer works. (Tony
Mechelynck)
解决方案: Do not define any CONF_ARGS by default.
文件: src/Makefile

Patch 8.1.0621
问题: Terminal debugger does not handle unexpected debugger exit.
解决方案: Check for debugger job ended and close unused buffers. (Damien)
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.0622
问题: Adding quickfix items marks items as valid errors. (Daniel Hahler)
解决方案: Check when items are valid. (Yegappan Lakshmanan, closes #3683,
closes #3633)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0623
问题: Iterating through window frames is repeated.
解决方案: Define FOR_ALL_FRAMES. (Yegappan Lakshmanan)
文件: src/ex_docmd.c, src/globals.h, src/screen.c, src/window.c

Patch 8.1.0624 (after 8.1.0620)
问题: Overruling CONF_ARGS from the environment still does not work.
(Tony Mechelynck)
解决方案: Add back CONF_ARGS next to the new numbered ones.
文件: src/Makefile

Patch 8.1.0625
问题: MS-Windows: terminal test fails in white console.
解决方案: Accept both white and black background colors.
文件: src/testdir/test_terminal.vim

Patch 8.1.0626
问题: MS-Windows: no resize to fit parent when using --windowid.
解决方案: Pass FALSE for "mustset" in gui_set_shellsize(). (Agorgianitis
Loukas, closes #3616)
文件: src/gui.c

Patch 8.1.0627
问题: Python cannot handle function name of script-local function.
解决方案: Use <SNR> instead of the special byte code. (Ozaki Kiichi, closes
#3681)
文件: src/if_py_both.h, src/testdir/test_python2.vim,
src/testdir/test_python3.vim

Patch 8.1.0628
问题: Compiler warning on MS-Windows.
解决方案: Add type cast. (Mike Williams)

文件: src/if_py_both.h

Patch 8.1.0629
问题: "gn" selects the wrong text with a multi-line match.
解决方案: Get the end position from searchit() directly. (closes #3695)
文件: src/testdir/test_gn.vim, src/search.c, src/proto/search.pro,
            src/edit.c, src/evalfunc.c, src/ex_docmd.c, src/ex_getln.c,
            src/normal.c

Patch 8.1.0630
问题: "wincmd p" does not work after using an autocmd window.
解决方案: Store "prevwin" in aco_save_T. (Christian Brabandt, closes #3690)
文件: src/fileio.c, src/structs.h, src/testdir/test_window_cmd.vim

Patch 8.1.0631
问题: Test for :stop fails on Arch.
解决方案: Check five lines for the expected output. (closes #3714)
文件: src/testdir/test_terminal.vim

Patch 8.1.0632
问题: Using sign group names is inefficient.
解决方案: Store group names in a hash table and use a reference to them.
            Also remove unnecessary use of ":exe" from the tests. (Yegappan
            Lakshmanan, closes #3715)
文件: src/buffer.c, src/ex_cmds.c, src/structs.h,
            src/testdir/test_signs.vim

Patch 8.1.0633
问题: Crash when out of memory while opening a terminal window.
解决方案: Handle out-of-memory more gracefully.
文件: src/terminal.c, src/libvterm/src/vterm.c,
            src/libvterm/src/state.c, src/libvterm/src/termscreen.c

Patch 8.1.0634
问题: Text properties cannot cross line boundaries.
解决方案: Support multi-line text properties.
文件: src/textprop.c, src/testdir/test_textprop.vim,
            runtime/doc/eval.txt

Patch 8.1.0635
问题: Coverity complains about null pointer use.
解决方案: Avoid using a null pointer.
文件: src/evalfunc.c

Patch 8.1.0636
问题: line2byte() gives wrong values with text properties. (Bjorn Linse)
解决方案: Compute byte offsets differently when text properties were added.
            (closes #3718)
文件: src/structs.h, src/textprop.c, src/proto/textprop.pro,
            src/memline.c, src/testdir/test_textprop.vim

Patch 8.1.0637
问题: Nsis file no longer used.
解决方案: Remove the file. (Ken Takata)

文件: nsis/vimrc.ini, Filelist

Patch 8.1.0638
问题: Text property highlighting is off by one column. (Bjorn Linse)
解决方案: Update text property highlighting earlier. Let it overrule syntax
            highlighting.
文件: src/structs.h, src/screen.c

Patch 8.1.0639
问题: text properties test fails on MS-Windows
解决方案: Set fileformat to "unix".
文件: src/testdir/test_textprop.vim

Patch 8.1.0640
问题: Get E14 while typing command :tab with 'incsearch' set.
解决方案: Do not give an error when looking for the command. (Hirohito
            Higashi)
文件: src/testdir/test_search.vim, src/ex_docmd.c

Patch 8.1.0641
问题: No check for out-of-memory when converting regexp.
解决方案: Bail out when lalloc() returns NULL. (John Marriott)
文件: src/regexp_nfa.c

Patch 8.1.0642
问题: swapinfo() leaks memory. (Christian Brabandt)
解决方案: Avoid allocating the strings twice.
文件: src/memline.c, src/dict.c, src/proto/dict.pro

Patch 8.1.0643
问题: Computing byte offset wrong. (Bjorn Linse)
解决方案: Use the right variable for array index.
文件: src/memline.c, src/testdir/test_textprop.vim

Patch 8.1.0644
问题: Finding next sign ID is inefficient.
解决方案: Add next_sign_id. (Yegappan Lakshmanan, closes #3717)
文件: runtime/doc/eval.txt, src/buffer.c, src/evalfunc.c, src/ex_cmds.c,
            src/globals.h, src/main.c, src/proto/buffer.pro, src/structs.h,
            src/testdir/test_signs.vim

Patch 8.1.0645
问题: Coverity warns for possible use of NULL pointer.
解决方案: Check return value of vterm_obtain_screen().
文件: src/terminal.c

Patch 8.1.0646
问题: Cannot build with Ruby 2.6.0.
解决方案: Add rb_ary_detransient(). (Ozaki Kiichi, closes #3724)
文件: src/if_ruby.c

Patch 8.1.0647
问题: MS-Windows: balloon_show() does not handle wide characters.
解决方案: Use CreateWindowExW(). (Yasuhiro Matsumoto, closes #3708)

文件: src/gui_w32.c

Patch 8.1.0648
问题: Custom operators can't act upon a forced motion. (Christian
            Wellenbrock)
解决方案: Add the forced motion to the mode() result. (Christian Brabandt,
            closes #3490)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/globals.h, src/normal.c,
            src/testdir/test_mapping.vim

Patch 8.1.0649
问题: setjmp() variables defined globally are used in one file.
解决方案: Move the declarations to that file.
文件: src/globals.h, src/os_unix.c

Patch 8.1.0650
问题: Command line argument -q [errorfile] is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3730)
文件: src/testdir/test_startup.vim

Patch 8.1.0651
问题: :args \"foo works like :args without argument.
解决方案: Fix check for empty argument. (closes #3728)
文件: src/ex_cmds2.c, src/testdir/test_arglist.vim

Patch 8.1.0652
问题: Freeing memory for balloon eval too early.
解决方案: Store the pointer in BalloonEval and free it later. (Yasuhiro
            Matsumoto, closes #3725)
文件: src/beval.h, src/gui_w32.c

Patch 8.1.0653 (after 8.1.0651)
问题: Arglist test fails on MS-windows.
解决方案: Only use a file name with a double quote on Unix.
文件: src/testdir/test_arglist.vim

Patch 8.1.0654
问题: When deleting a line text property flags are not adjusted.
解决方案: Adjust text property flags in preceding and following lines.
文件: src/memline.c, src/misc2.c, src/proto/misc2.pro,
            src/testdir/test_textprop.vim

Patch 8.1.0655
问题: When appending a line text property flags are not added.
解决方案: Add text properties to a newly added line.
文件: src/memline.c, src/testdir/test_textprop.vim, src/textprop.c

Patch 8.1.0656
问题: Trying to reconnect to X server may cause problems.
解决方案: Do no try reconnecting when exiting. (James McCoy)
文件: src/os_unix.c

Patch 8.1.0657 (after 8.1.0656)
问题: Get error for using regexp recursively. (Dominique Pelle)

解决方案: Do no check if connection is desired.
文件: src/os_unix.c

Patch 8.1.0658
问题: Deleting signs and completion for :sign is insufficient.
解决方案: Add deleting signs in a specified or any group from the current cursor location. Add group and priority to sign command completion. Add tests for different sign unplace commands. Update help text. Add tests for sign jump with group. Update help for sign jump. (Yegappan Lakshmanan, closes #3731)
文件: runtime/doc/sign.txt, src/buffer.c, src/evalfunc.c, src/ex_cmds.c, src/netbeans.c, src/proto/buffer.pro, src/proto/ex_cmds.pro, src/testdir/test_signs.vim

Patch 8.1.0659 (after 8.1.0658)
问题: Build failure without the sign feature.
解决方案: Put the sign struct declarations outside of the #ifdef.
文件: src/structs.h

Patch 8.1.0660
问题: sign_unplace() may leak memory.
解决方案: Free the group name before returning. Add a few more tests. (Yegappan Lakshmanan)
文件: src/evalfunc.c, src/testdir/test_signs.vim

Patch 8.1.0661
问题: Clipboard regexp might be used recursively.
解决方案: Check for recursive use and bail out.
文件: src/regexp.c, src/proto/regexp.pro, src/os_unix.c

Patch 8.1.0662
问题: Needlessly searching for tilde in string.
解决方案: Only check the first character. (James McCoy, closes #3734)
文件: src/misc1.c

Patch 8.1.0663
问题: Text property display wrong when 'number' is set. (Dominique Pelle)
解决方案: Compare with "vcol" instead of "col".
文件: src/screen.c

Patch 8.1.0664
问题: Configure "fail-if-missing" does not apply to the enable-gui argument. (Rhialto)
解决方案: Make configure fail if a GUI was specified and "fail-if-missing" is enabled and the GUI test fails.
文件: src/configure.ac, src/auto/configure

Patch 8.1.0665
问题: Text property display wrong when 'spell' is set. (Dominique Pelle)
解决方案: Remove unnecessary assignment to char_attr. Combine attributes if needed. Add a screenshot test.
文件: src/screen.c, src/testdir/test_textprop.vim, src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.0666 (after 8.1.0665)
问题: Text property test fails.
解决方案: Update screenshot.
文件: src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.0667 (after 8.1.0665)
问题: Textprop test leaves file behind.
解决方案: Delete the file. (Dominique Pelle, closes #3743)
文件: src/testdir/test_textprop.vim

Patch 8.1.0668
问题: No test for overstrike mode in the command line.
解决方案: Add a test. (Dominique Pelle, closes #3742)
文件: src/testdir/test_cmdline.vim

Patch 8.1.0669
问题: The ex_sign() function is too long.
解决方案: Refactor the function. Add a bit more testing. (Yegappan
            Lakshmanan, closes #3745)
文件: src/testdir/test_signs.vim, src/ex_cmds.c

Patch 8.1.0670
问题: Macro for popup menu width is unused.
解决方案: Remove it. (Hirohito Higashi)
文件: src/popupmnu.c

Patch 8.1.0671
问题: Cursor in the wrong column after auto-formatting.
解决方案: Check for deleting more spaces than adding. (closes #3748)
文件: src/ops.c, src/testdir/test_textformat.vim, src/mark.c,
            src/proto/mark.pro, src/misc1.c

Patch 8.1.0672
问题: The Lua interface doesn't know about v:null.
解决方案: Add Lua support for v:null. (Uji, closes #3744)
文件: src/if_lua.c, src/testdir/test_lua.vim

Patch 8.1.0673
问题: Functionality for signs is spread out over several files.
解决方案: Move most of the sign functionality into sign.c. (Yegappan
            Lakshmanan, closes #3751)
文件: Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
            src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
            src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
            src/Make_vms.mms, src/Makefile, src/README.txt, src/buffer.c,
            src/evalfunc.c, src/ex_cmds.c, src/proto.h, src/proto/buffer.pro,
            src/proto/ex_cmds.pro, src/proto/sign.pro, src/sign.c

Patch 8.1.0674
问题: Leaking memory when updating a single line.
解决方案: Do not call start_search_hl() twice.
文件: src/screen.c

Patch 8.1.0675
问题: Text property column is screen columns is not practical.
解决方案: Use byte values for the column.
文件: src/structs.h, src/textprop.c, src/proto/textprop.pro,
            runtime/doc/eval.txt, runtime/doc/textprop.txt,
            src/testdir/test_textprop.vim,
            src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.0676
问题: Textprop screendump test fails.
解决方案: Add missing changes.
文件: src/screen.c

Patch 8.1.0677
问题: Look-behind match may use the wrong line number. (Dominique Pelle)
解决方案: Use the line number in regsave instead of the one in behind_pos,
            we may be looking at the previous line. (closes #3749)
文件: src/regexp.c

Patch 8.1.0678
问题: Text properties as not adjusted for inserted text.
解决方案: Adjust text properties when inserting text.
文件: src/misc1.c, src/proto/misc1.pro, src/textprop.c,
            src/testdir/test_textprop.vim,
            src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.0679
问题: Sign functions do not take buffer argument as documented.
解决方案: Use get_buf_tv(). (Yegappan Lakshmanan, closes #3755)
文件: src/evalfunc.c, src/testdir/test_signs.vim

Patch 8.1.0680
问题: Not easy to see what features are unavailable.
解决方案: Highlight disabled features in the :version output. (Nazri Ramliy,
            closes #3756)
文件: src/version.c

Patch 8.1.0681
问题: Text properties as not adjusted for deleted text.
解决方案: Adjust text properties when backspacing to delete text.
文件: src/edit.c, src/misc1.c, src/testdir/test_textprop.vim,
            src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.0682
问题: Text properties are not adjusted when backspacing replaced text.
解决方案: Keep text properties on text restored in replace mode.
文件: src/edit.c, src/textprop.c, src/globals.h,
            src/testdir/test_textprop.vim

Patch 8.1.0683
问题: Spell highlighting does not always end. (Gary Johnson)
解决方案: Also reset char_attr when spell errors are highlighted.
文件: src/screen.c

Patch 8.1.0684
问题: Warnings from 64-bit compiler.
解决方案: Add type casts. (Mike Williams)
文件: src/memline.c, src/textprop.c

Patch 8.1.0685
问题: get_buf_tv() is named inconsistently.
解决方案: Rename it to tv_get_buf(). (Yegappan Lakshmanan, closes #3759)
文件: src/evalfunc.c, src/proto/evalfunc.pro, src/terminal.c,
            src/textprop.c

Patch 8.1.0686
问题: When 'y' is in 'cpoptions' yanking for the clipboard changes redo.
解决方案: Do not use the 'y' flag when "gui_yank" is TRUE. (Andy Massimino,
            closes #3760)
文件: src/normal.c

Patch 8.1.0687
问题: Sentence text object in Visual mode is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3758)
文件: src/testdir/test_visual.vim

Patch 8.1.0688
问题: Text properties are not restored by undo.
解决方案: Also save text properties for undo.
文件: src/structs.h, src/undo.c, src/memline.c, src/proto/memline.pro

Patch 8.1.0689 (after 8.1.0688)
问题: Undo with text properties not tested.
解决方案: Add a test function.
文件: src/testdir/test_textprop.vim

Patch 8.1.0690
问题: setline() and setbufline() do not clear text properties.
解决方案: Clear text properties when setting the text.
文件: src/evalfunc.c, src/testdir/test_textprop.vim

Patch 8.1.0691
问题: Text properties are not adjusted for :substitute.
解决方案: Adjust text properties as well as possible.
文件: src/ex_cmds.c, src/textprop.c, src/proto/textprop.pro,
            src/testdir/test_textprop.vim

Patch 8.1.0692
问题: If a buffer was deleted a channel can't write to it.
解决方案: When the buffer exists but was unloaded, prepare it for writing.
            (closes #3764)
文件: src/channel.c, src/testdir/test_channel.vim

Patch 8.1.0693 (after 8.1.0692)
问题: Channel test fails sometimes.
解决方案: Avoid race condition.
文件: src/testdir/test_channel.vim

Patch 8.1.0694
问题: When using text props may free memory that is not allocated.
(Andy Massimino)
解决方案: Allocate the line when adjusting text props. (closes #3766)
文件: src/textprop.c

Patch 8.1.0695
问题: Internal error when using :popup.
解决方案: When a menu only exists in Terminal mode give an error. (Naruhiko
Nishino, closes #3765)
文件: runtime/doc/gui.txt, src/globals.h, src/menu.c, src/popupmnu.c,
src/testdir/test_popup.vim

Patch 8.1.0696
问题: When test_edit fails 'insertmode' may not be reset and the next
test may get stuck. (James McCoy)
解决方案: Always reset 'insertmode' after executing a test. Avoid that an
InsertCharPre autocommand or a 'complete' function can change the
state. (closes #3768)
文件: src/testdir/runtest.vim, src/edit.c

Patch 8.1.0697
问题: ":sign place" requires the buffer argument.
解决方案: Make the argument optional. Also update the help and clean up the
sign test. (Yegappan Lakshmanan, closes #3767)
文件: runtime/doc/eval.txt, runtime/doc/sign.txt, src/sign.c,
src/testdir/test_signs.vim

Patch 8.1.0698
问题: Clearing the window is used too often, causing the command line
to be cleared when opening a tab. (Miroslav Kokár)
解决方案: Use NOT_VALID instead of CLEAR. (suggested by Jason Franklin,
closes #630) Also do this for a few other places where clearing
the screen isn't really needed.
文件: src/window.c

Patch 8.1.0699
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Add a dummy init.
文件: src/edit.c

Patch 8.1.0700 (after 8.1.0698)
问题: Using "gt" sometimes does not redraw a tab. (Jason Franklin)
解决方案: Always set must_redraw in redraw_all_later().
文件: src/screen.c

Patch 8.1.0701
问题: Sign message not translated and inconsistent spacing.
解决方案: Add _() for translation. Add a space. (Ken Takata) Also use
MSG_BUF_LEN instead of BUFSIZ.
文件: src/sign.c, src/testdir/test_signs.vim

Patch 8.1.0702
问题: ":sign place" only uses the current buffer.

解决方案: List signs for all buffers when there is no buffer argument.
           Fix error message for invalid buffer name in sign_place().
           (Yegappan Lakshmanan, closes #3774)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/sign.c,
           src/testdir/test_signs.vim

Patch 8.1.0703
问题: Compiler warnings with 64-bit compiler.
解决方案: Change types, add type casts. (Mike Williams)
文件: src/textprop.c, src/undo.c

Patch 8.1.0704
问题: Building with Ruby 2.6 gives compiler warnings.
解决方案: Define a stub for rb_ary_detransient. (Ozaki Kiichi, closes #3779)
文件: src/if_ruby.c

Patch 8.1.0705
问题: :colorscheme isn't tested enough
解决方案: Improve test coverage of :colorscheme. (Dominique Pelle, closes
           #3777) Remove unnecessary sleep.
文件: src/testdir/test_gui.vim

Patch 8.1.0706
问题: Tabline is not always redrawn when something that is used in
           'tabline' changes.
解决方案: Add ":redrawtabline" so that a plugin can at least cause the
           redraw when needed.
文件: runtime/doc/various.txt, runtime/doc/options.txt, src/ex_docmd.c,
           src/ex_cmds.h, src/screen.c, src/proto/screen.pro,
           src/ex_cmdidxs.h, src/testdir/test_tabline.vim

Patch 8.1.0707
问题: Text property columns are not adjusted for changed indent.
解决方案: Adjust text properties.
文件: src/misc1.c, src/testdir/test_textprop.vim

Patch 8.1.0708
问题: Third argument for redrawWinline() is always FALSE.
解决方案: Drop the argument. (neovim #9479)
文件: src/edit.c, src/move.c, src/screen.c, src/proto/screen.pro

Patch 8.1.0709
问题: Windows are updated for every added/deleted sign.
解决方案: Do not call update_debug_sign(). Only redraw when the line with
           the sign is visible. (idea from neovim #9479)
文件: src/sign.c, src/screen.c, src/proto/screen.pro

Patch 8.1.0710
问题: When using timers may wait for job exit quite long.
解决方案: Return from ui_wait_for_chars_or_timer() when a job or channel
           needs to be handled. (Ozaki Kiichi, closes #3783)
文件: src/ui.c, src/testdir/test_channel.vim

Patch 8.1.0711

问题:	Test files still use function!.
解决方案:	Remove the exclamation mark. Fix overwriting a function.
文件:	src/testdir/test49.vim, src/testdir/test_autocmd.vim,
src/testdir/test_charsearch.vim,
src/testdir/test_charsearch_utf8.vim,
src/testdir/test_display.vim, src/testdir/test_edit.vim,
src/testdir/test_eval_func.vim, src/testdir/test_fnameescape.vim,
src/testdir/test_getcwd.vim, src/testdir/test_highlight.vim,
src/testdir/test_hlsearch.vim, src/testdir/test_ins_complete.vim,
src/testdir/test_lambda.vim, src/testdir/test_listdict.vim,
src/testdir/test_listlbr.vim, src/testdir/test_listlbr_utf8.vim,
src/testdir/test_marks.vim, src/testdir/test_matchadd_conceal.vim,
src/testdir/test_matchadd_conceal_utf8.vim,
src/testdir/test_messages.vim, src/testdir/test_number.vim,
src/testdir/test_options.vim, src/testdir/test_partial.vim,
src/testdir/test_smartindent.vim, src/testdir/test_substitute.vim,
src/testdir/test_system.vim, src/testdir/test_terminal.vim,
src/testdir/test_textobjects.vim, src/testdir/test_utf8.vim,
src/testdir/test_utf8_comparisons.vim,
src/testdir/test_vartabs.vim, src/testdir/test_vimscript.vim,
src/testdir/test_window_cmd.vim, src/testdir/test_xxd.vim

Patch 8.1.0712
问题:	MS-Windows build instructions are a bit outdated.
解决方案:	Update the instructions. (Ken Takata)
文件:	src/INSTALLpc.txt

Patch 8.1.0713
问题:	Images for NSIS take up too much space.
解决方案:	Put the images in a zip file.
文件:	nsis/icons.zip, nsis/icons/disabled.bmp, nsis/icons/enabled.bmp,
nsis/icons/header.bmp, nsis/icons/header.svg,
nsis/icons/un_header.bmp, nsis/icons/uninstall.bmp,
nsis/icons/vim_16c.ico, nsis/icons/vim_uninst_16c.ico,
nsis/icons/welcome.bmp, nsis/icons/welcome.svg,
nsis/README.txt, Filelist, Makefile

Patch 8.1.0714
问题:	Unnecessary #if lines in GTK code.
解决方案:	Remove the #if. (Ken Takata, closes #3785)
文件:	src/gui_beval.c, src/if_mzsch.c

Patch 8.1.0715
问题:	Superfluous call to redraw_win_later().
解决方案:	Remove the call.
文件:	src/move.c

Patch 8.1.0716
问题:	Get warning message when 'completefunc' returns nothing.
解决方案:	Allow for returning v:none to suppress the warning message.
(Yasuhiro Matsumoto, closes #3789)
文件:	runtime/doc/insert.txt, src/edit.c,
src/testdir/test_ins_complete.vim

Patch 8.1.0717
问题:　　There is no function for the ":sign jump" command.
解决方案:　Add the sign_jump() function. (Yegappan Lakshmanan, closes #3780)
文件:　　runtime/doc/eval.txt, runtime/doc/sign.txt,
　　runtime/doc/usr_41.txt, src/evalfunc.c, src/proto/sign.pro,
　　src/sign.c, src/testdir/test_signs.vim

Patch 8.1.0718
问题:　　A couple compiler warnings.
解决方案:　Rename shadowed variables. Add UNUSED.
文件:　　src/misc1.c

Patch 8.1.0719
问题:　　Too many #ifdefs.
解决方案:　Always build with the +visualextra feature.
文件:　　src/evalfunc.c, src/version.c, src/normal.c, src/ops.c,
　　src/feature.h, runtime/doc/various.txt

Patch 8.1.0720
问题:　　Cannot easily change the current quickfix list index.
解决方案:　Add the "idx" argument to setqflist(). (Yegappan Lakshmanan,
　　closes #3701)
文件:　　runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/quickfix.c,
　　src/testdir/test_quickfix.vim

Patch 8.1.0721
问题:　　Conceal mode is not sufficiently tested.
解决方案:　Add screendump tests. Check all 'concealcursor' values.
文件:　　src/testdir/test_conceal.vim, src/Make_all.mak,
　　src/testdir/Make_all.mak
　　src/testdir/dumps/Test_conceal_two_windows_01.dump,
　　src/testdir/dumps/Test_conceal_two_windows_02.dump,
　　src/testdir/dumps/Test_conceal_two_windows_03.dump,
　　src/testdir/dumps/Test_conceal_two_windows_04.dump,
　　src/testdir/dumps/Test_conceal_two_windows_05.dump,
　　src/testdir/dumps/Test_conceal_two_windows_06i.dump,
　　src/testdir/dumps/Test_conceal_two_windows_06v.dump,
　　src/testdir/dumps/Test_conceal_two_windows_06c.dump,
　　src/testdir/dumps/Test_conceal_two_windows_06n.dump,
　　src/testdir/dumps/Test_conceal_two_windows_07i.dump,
　　src/testdir/dumps/Test_conceal_two_windows_07v.dump,
　　src/testdir/dumps/Test_conceal_two_windows_07c.dump,
　　src/testdir/dumps/Test_conceal_two_windows_07n.dump,
　　src/testdir/dumps/Test_conceal_two_windows_08i.dump,
　　src/testdir/dumps/Test_conceal_two_windows_08v.dump,
　　src/testdir/dumps/Test_conceal_two_windows_08c.dump,
　　src/testdir/dumps/Test_conceal_two_windows_08n.dump,
　　src/testdir/dumps/Test_conceal_two_windows_09i.dump,
　　src/testdir/dumps/Test_conceal_two_windows_09v.dump,
　　src/testdir/dumps/Test_conceal_two_windows_09c.dump,
　　src/testdir/dumps/Test_conceal_two_windows_09n.dump

Patch 8.1.0722
问题:　　Cannot build without the virtualedit feature.

解决方案: Make getviscol2() always available.
文件: src/misc2.c, src/proto/misc2.pro, src/ops.c

Patch 8.1.0723
问题: Cannot run specific test when in src/testdir the same was as in
           the src directory.
解决方案: Move build rule to src/testdir/Makefile.
文件: src/testdir/Make_all.mak, src/testdir/Make_amiga.mak,
           src/testdir/Make_dos.mak, src/testdir/Make_ming.mak,
           src/Makefile, src/Make_all.mak, src/testdir/Makefile,
           src/testdir/README.txt, src/Make_mvc.mak

Patch 8.1.0724
问题: Build for MinGW fails.
解决方案: Avoid specifying dependencies in included makefile.
文件: src/testdir/Make_all.mak, src/testdir/Makefile,
           src/testdir/Make_dos.mak, src/testdir/Make_ming.mak

Patch 8.1.0725
问题: Conceal mode is not completely tested.
解决方案: Add tests for moving the cursor in Insert mode.
文件: src/testdir/test_conceal.vim,
           src/testdir/dumps/Test_conceal_two_windows_10.dump,
           src/testdir/dumps/Test_conceal_two_windows_11.dump,
           src/testdir/dumps/Test_conceal_two_windows_12.dump,
           src/testdir/dumps/Test_conceal_two_windows_13.dump

Patch 8.1.0726
问题: Redrawing specifically for conceal feature.
解决方案: Use generic redrawing methods.
文件: src/edit.c, src/gui.c, src/main.c, src/normal.c, src/screen.c,
           src/proto/screen.pro, src/window.c

Patch 8.1.0727
问题: Compiler warning for sprintf() argument.
解决方案: Add type cast.
文件: src/dosinst.c

Patch 8.1.0728
问题: Cannot avoid breaking after a single space.
解决方案: Add the 'p' flag to 'formatoptions'. (Tom Ryder)
文件: runtime/doc/change.txt, src/edit.c, src/option.h,
           src/testdir/test_textformat.vim

Patch 8.1.0729
问题: There is a SourcePre autocommand event but not a SourcePost.
解决方案: Add the SourcePost autocommand event. (closes #3739)
文件: src/vim.h, src/fileio.c, src/ex_cmds2.c, runtime/doc/autocmd.txt,
           src/testdir/test_source.vim, src/testdir/Make_all.mak

Patch 8.1.0730
问题: Compiler warning for get_buf_arg() unused.
解决方案: Add #ifdef. (John Marriott)
文件: src/evalfunc.c

Patch 8.1.0731
问题: JS encoding does not handle negative infinity.
解决方案: Add support for negative infinity for JS encoding. (Dominique
            Pelle, closes #3792)
文件: runtime/doc/eval.txt, src/json.c, src/testdir/test_json.vim

Patch 8.1.0732
问题: Cannot build without the eval feature.
解决方案: Make a copy of the sourced file name.
文件: src/ex_cmds2.c

Patch 8.1.0733
问题: Too many #ifdefs for the multi-byte feature.
解决方案: Tentatively always enable the multi-byte feature. If you have a
            problem with this, please discuss on the Vim maillist.
文件: src/configure.ac, src/auto/configure, src/feature.h, src/Makefile,
            src/Make_bc5.mak, src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.0734
问题: The hlsearch state is not stored in a session file.
解决方案: Add "nohlsearch" if appropriate. (Jason Franklin)
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0735
问题: Cannot handle binary data.
解决方案: Add the Blob type. (Yasuhiro Matsumoto, closes #3638)
文件: runtime/doc/eval.txt, runtime/doc/if_perl.txt,
            runtime/doc/if_ruby.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak,
            src/Makefile, src/blob.c, src/channel.c, src/eval.c,
            src/evalfunc.c, src/if_perl.xs, src/if_py_both.h, src/if_python.c,
            src/if_python3.c, src/if_ruby.c, src/json.c, src/netbeans.c,
            src/proto.h, src/proto/blob.pro, src/proto/channel.pro,
            src/structs.h, src/testdir/Make_all.mak, src/vim.h, src/globals.h,
            src/testdir/test_blob.vim, src/testdir/test_channel.vim

Patch 8.1.0736
问题: Code for Blob not sufficiently tested.
解决方案: Add more tests. Fix uncovered crash. Add test_null_blob().
文件: src/testdir/test_blob.vim, src/testdir/test_assign.vim, src/eval.c,
            src/testdir/test_eval_stuff.vim, src/testdir/test_lambda.vim,
            runtime/doc/eval.txt, src/evalfunc.c, src/blob.c,
            src/testdir/test49.vim

Patch 8.1.0737
问题: Compiler warning for uninitialized variable.
解决方案: Add initialization. (John Marriott)
文件: src/eval.c

Patch 8.1.0738
问题: Using freed memory, for loop over blob leaks memory.
解决方案: Clear pointer after freeing memory. Decrement reference count
            after for loop over blob.
文件: src/eval.c

Patch 8.1.0739
问题: Text objects in not sufficiently tested.
解决方案: Add a few more test cases. (Dominique Pelle, closes #3795)
文件: src/testdir/test_visual.vim

Patch 8.1.0740
问题: Tcl test fails.
解决方案: When the argument is empty don't give an error, instead rely on the error reporting higher up.
文件: src/eval.c

Patch 8.1.0741
问题: Viminfo with Blob is not tested.
解决方案: Extend the viminfo test. Fix reading a blob. Fixed storing a special variable value.
文件: src/testdir/test_viminfo.vim, src/eval.c, src/blob.c, src/proto/blob.pro

Patch 8.1.0742
问题: Not all Blob operations are tested.
解决方案: Add more testing for Blob.
文件: src/testdir/test_blob.vim, src/evalfunc.c, src/testdir/test_eval_stuff.vim

Patch 8.1.0743
问题: Giving error messages is not flexible.
解决方案: Add semsg(). Change argument from "char_u *" to "char *", also for msg() and get rid of most MSG macros. (Ozaki Kiichi, closes #3302) Also make emsg() accept a "char *" argument. Get rid of an enormous number of type casts.
文件: src/blob.c, src/blowfish.c, src/buffer.c, src/channel.c, src/crypt.c, src/dict.c, src/diff.c, src/digraph.c, src/edit.c, src/eval.c, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds.h, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c, src/farsi.h, src/fileio.c, src/fold.c, src/getchar.c, src/globals.h, src/gui.c, src/gui_at_fs.c, src/gui_at_sb.c, src/gui_beval.c, src/gui_gtk_x11.c, src/gui_mac.c, src/gui_photon.c, src/gui_w32.c, src/gui_x11.c, src/hangulin.c, src/hardcopy.c, src/hashtab.c, src/if_cscope.c, src/if_lua.c, src/if_mzsch.c, src/if_perl.xs, src/if_py_both.h, src/if_python.c, src/if_python3.c, src/if_ruby.c, src/if_tcl.c, src/if_xcmdsrv.c, src/json.c, src/list.c, src/main.c, src/mark.c, src/mbyte.c, src/memfile.c, src/memline.c, src/menu.c, src/message.c, src/misc1.c, src/misc2.c, src/netbeans.c, src/normal.c, src/ops.c, src/option.c, src/os_amiga.c, src/os_mswin.c, src/os_unix.c, src/os_win32.c, src/popupmnu.c, src/proto.h, src/proto/buffer.pro, src/proto/digraph.pro, src/proto/ex_docmd.pro, src/proto/ex_eval.pro, src/proto/ex_getln.pro, src/proto/hardcopy.pro, src/proto/mbyte.pro, src/proto/message.pro, src/proto/misc2.pro, src/proto/option.pro, src/proto/spell.pro, src/quickfix.c, src/regexp.c, src/regexp_nfa.c, src/search.c, src/sign.c, src/spell.c, src/spellfile.c, src/structs.h, src/syntax.c, src/tag.c,

src/term.c, src/terminal.c, src/textprop.c, src/ui.c, src/undo.c,
src/userfunc.c, src/version.c, src/vim.h, src/window.c,

Patch 8.1.0744 (after 8.1.0743)
问题: Compiler warnings for signed/unsigned strings.
解决方案: A few more type cast fixes.
文件: src/option.c, src/if_perl.xs, src/if_py_both.h, src/integration.c

Patch 8.1.0745
问题: Compiler warnings for signed/unsigned string.
解决方案: Remove type casts. (John Marriott)
文件: src/ex_docmd.c, src/mbyte.c

Patch 8.1.0746
问题: Highlighting not updated with conceal and 'cursorline'. (Jason
Franklin)
解决方案: Do not use a zero line number. Check if 'conceallevel' is set for
the current window.
文件: src/main.c, src/testdir/test_conceal.vim,
src/testdir/dumps/Test_conceal_cul_01.dump,
src/testdir/dumps/Test_conceal_cul_02.dump,
src/testdir/dumps/Test_conceal_cul_03.dump

Patch 8.1.0747
问题: map() with a bad expression doesn't give an error. (Ingo Karkat)
解决方案: Check for giving an error message. (closes #3800)
文件: src/eval.c, src/testdir/test_filter_map.vim

Patch 8.1.0748
问题: Using sprintf() instead of semsg().
解决方案: Use semsg(). Fix bug with E888. (Ozaki Kiichi, closes #3801)
文件: src/regexp.c

Patch 8.1.0749 (after 8.1.0747)
问题: Error message contains garbage. (Dominique Pelle)
解决方案: Use correct pointer to failed expression.
文件: src/eval.c

Patch 8.1.0750
问题: When the last sign is deleted the signcolumn may not be removed
even though 'signcolumn' is "auto".
解决方案: When deleting the last sign redraw the buffer. (Dominique Pelle,
closes #3803, closes #3804)
文件: src/sign.c

Patch 8.1.0751
问题: Some regexp errors are not tested.
解决方案: Add a test function.
文件: src/testdir/test_regexp_latin.vim

Patch 8.1.0752
问题: One more compiler warning for signed/unsigned string. (Tony
Mechelynck)
解决方案: Remove type cast.

文件: src/ex_docmd.c

Patch 8.1.0753
问题: printf format not checked for semsg().
解决方案: Add GNUC attribute and fix reported problems. (Dominique Pelle,
            closes #3805)
文件: src/buffer.c, src/diff.c, src/eval.c, src/evalfunc.c,
            src/ex_docmd.c, src/if_cscope.c, src/netbeans.c, src/proto.h,
            src/proto/message.pro, src/quickfix.c, src/regexp_nfa.c,
            src/sign.c, src/spellfile.c, src/window.c, src/gui_x11.c

Patch 8.1.0754
问题: Preferred column is lost when setting 'cursorcolumn'.
解决方案: Change option flag to P_RWINONLY. (Takayuki Kurosawa,
            closes #3806)
文件: src/option.c, src/testdir/test_cursor_func.vim

Patch 8.1.0755
问题: Error message for get() on a Blob with invalid index.
解决方案: Return an empty Blob, like get() on a List does.
文件: src/evalfunc.c, src/testdir/test_blob.vim

Patch 8.1.0756
问题: copy() does not make a copy of a Blob.
解决方案: Make a copy.
文件: src/eval.c, src/testdir/test_blob.vim

Patch 8.1.0757
问题: Not enough documentation for Blobs.
解决方案: Add a section about Blobs.
文件: runtime/doc/eval.txt

Patch 8.1.0758
问题: Font number is always one instead of the actual.
解决方案: Use "%d" instead of "1". (Ken Takata)
文件: src/gui_x11.c

Patch 8.1.0759
问题: Showing two characters for tab is limited.
解决方案: Allow for a third character for "tab:" in 'listchars'. (Nathaniel
            Braun, Ken Takata, closes #3810)
文件: runtime/doc/options.txt, src/globals.h, src/message.c,
            src/option.c, src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.0760
问题: No proper test for using 'termencoding'.
解决方案: Add a screendump test. Fix using double width characters in a
            screendump.
文件: src/terminal.c, src/testdir/test_termencoding.vim,
            src/testdir/Make_all.mak,
            src/testdir/dumps/Test_tenc_euc_jp_01.dump

Patch 8.1.0761
问题: Default value for brief_wait is wrong.

解决方案: Make the default FALSE. (Ozaki Kiichi, closes #3812, closes #3799)
文件: src/ui.c

Patch 8.1.0762
问题: Compiler warning.
解决方案: Add type cast. (Mike Williams)
文件: src/channel.c

Patch 8.1.0763
问题: Nobody is using the Sun Workshop support.
解决方案: Remove the Workshop support.
文件: runtime/doc/workshop.txt, runtime/doc/help.txt,
           runtime/doc/netbeans.txt, src/Makefile, src/auto/configure,
           src/beval.c, src/buffer.c, src/config.h.in, src/config.mk.in,
           src/configure.ac, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds.h,
           src/ex_docmd.c, src/feature.h, src/fileio.c, src/globals.h,
           src/gui.c, src/gui_beval.c, src/gui_motif.c, src/gui_x11.c,
           src/integration.c, src/integration.h, src/main.c, src/misc2.c,
           src/nbdebug.c, src/netbeans.c, src/proto.h,
           src/proto/workshop.pro, src/ui.c, src/version.c, src/vim.h,
           src/workshop.c, src/workshop.h, src/wsdebug.c, src/wsdebug.h,
           src/ex_cmdidxs.h

Patch 8.1.0764
问题: List of distributed files is outdated.
解决方案: Remove workshop files. Add blob files.
文件: Filelist

Patch 8.1.0765
问题: String format of a Blob can't be parsed back.
解决方案: Use 0z format.
文件: src/blob.c, src/eval.c, src/testdir/test_blob.vim

Patch 8.1.0766
问题: Various problems when using Vim on VMS.
解决方案: Various fixes. Define long_long_T. (Zoltan Arpadffy)
文件: src/eval.c, src/feature.h, src/fileio.c, src/gui_motif.c,
           src/gui_x11.c, src/gui_xmebw.c, src/json.c, src/Make_vms.mms,
           src/ops.c, src/os_vms_conf.h, src/vim.h, src/xdiff/xdiff.h,
           src/xdiff/xinclude.h

Patch 8.1.0767
问题: When deleting lines at the bottom signs are misplaced.
解决方案: Properly update the line number of signs at the end of a buffer
           after a delete/undo operation. (Yegappan Lakshmanan, closes #3798)
文件: src/sign.c, src/testdir/test_signs.vim

Patch 8.1.0768
问题: Updating completions may cause the popup menu to flicker.
解决方案: Avoid updating the text below the popup menu before drawing the
           popup menu.
文件: src/popupmnu.c, src/proto/popupmnu.pro, src/edit.c, src/screen.c

Patch 8.1.0769

问题: :stop is covered in two tests.
解决方案: Remove Test_stop_in_terminal(). Make other test exit Vim cleanly.
            (Ozaki Kiichi, closes #3814)
文件: src/testdir/test_terminal.vim, src/testdir/test_suspend.vim

Patch 8.1.0770
问题: Inconsistent use of ELAPSED_FUNC.
解决方案: Consistently use ELAPSED_FUNC. Also turn ELAPSED_TYPE into a
            typedef. (Ozaki Kiichi, closes #3815)
文件: src/channel.c, src/gui.c, src/misc1.c, src/os_unix.c, src/vim.h

Patch 8.1.0771
问题: Some shell filetype patterns end in a star.
解决方案: Make sure that patterns not ending in a star are preferred.
文件: runtime/filetype.vim, runtime/autoload/dist/ft.vim

Patch 8.1.0772
问题: The sign_define_by_name() function is too long.
解决方案: Split it into smaller functions. (Yegappan Lakshmanan,
            closes #3819)
文件: src/sign.c

Patch 8.1.0773
问题: Not all crypt code is tested.
解决方案: Disable unused crypt code. Add more test coverage.
文件: src/structs.h, src/crypt.c, src/testdir/test_crypt.vim,
            src/proto/crypt.pro, src/fileio.c

Patch 8.1.0774
问题: VMS build is missing the blob file.
解决方案: Add the blob file to the build rules. (Zoltan Arpadffy)
文件: src/Make_vms.mms, runtime/doc/os_vms.txt

Patch 8.1.0775
问题: Matching too many files as zsh. (Danek Duvall)
解决方案: Be more specific with zsh filetype patterns.
文件: runtime/filetype.vim

Patch 8.1.0776
问题: Travis does not build a version without GUI on Linux.
解决方案: Add an environment for tiny features without GUI.
文件: .travis.yml

Patch 8.1.0777
问题: Win32: using pipes for channel does not work well.
解决方案: Use a larger buffer and handle overlaps. (Yasuhiro Matsumoto,
            closes #3782)
文件: src/channel.c, src/os_win32.c

Patch 8.1.0778
问题: Terminal test fails on MS-Windows.
解决方案: Temporarily skip the test on MS-Windows. Do run it both in
            terminal and GUI on other systems.
文件: src/testdir/test_terminal.vim

Patch 8.1.0779
问题: Argument for message functions is inconsistent.
解决方案: Make first argument to msg() "char *".
文件: src/buffer.c, src/crypt.c, src/edit.c, src/ex_cmds.c, src/eval.c,
            src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/farsi.c,
            src/if_cscope.c, src/fileio.c, src/getchar.c, src/globals.h,
            src/gui.c, src/if_perl.xs, src/netbeans.c, src/gui_w32.c,
            src/hardcopy.c, src/if_mzsch.c, src/if_py_both.h, src/if_ruby.c,
            src/if_tcl.c, src/mark.c, src/mbyte.c, src/menu.c, src/memline.c,
            src/message.c, src/misc1.c, src/misc2.c, src/normal.c, src/ops.c,
            src/option.c, src/os_amiga.c, src/os_unix.c, src/os_win32.c,
            src/proto/message.pro, src/quickfix.c, src/sign.c, src/regexp.c,
            src/ui.c, src/screen.c, src/search.c, src/spell.c,
            src/spellfile.c, src/syntax.c, src/tag.c, src/term.c, src/undo.c,
            src/userfunc.c, src/version.c, src/vim.h, src/window.c,
            src/proto/eval.pro, src/evalfunc.c, src/ex_eval.c, src/farsi.h

Patch 8.1.0780
问题: Terminal test fails on Mac.
解决方案: Skip the test on Mac.
文件: src/testdir/test_terminal.vim

Patch 8.1.0781
问题: Build error when using if_xcmdsrv.c.
解决方案: Add missing part of 8.1.0779.
文件: src/if_xcmdsrv.c

Patch 8.1.0782
问题: Win32: cursor blinks when Vim is not active.
解决方案: Remove call to setActiveWindow(). (Yasuhiro Matsumoto,
            closes #3778)
文件: src/gui_w32.c, src/proto/gui_w32.pro, src/menu.c

Patch 8.1.0783
问题: Compiler warning for signed/unsigned.
解决方案: Add type cast. Change type of buffer. (Ozaki Kiichi, closes #3827)
文件: src/main.c, src/message.c

Patch 8.1.0784
问题: Messy indent in if statement.
解决方案: Improve structure of if statement. (Ozaki Kiichi, closes #3826)
文件: src/os_win32.c

Patch 8.1.0785
问题: Depending on the configuration some functions are unused.
解决方案: Add more #ifdefs, remove unused functions. (Dominique Pelle,
            closes #3822)
文件: src/buffer.c, src/channel.c, src/ex_cmds2.c, src/ex_docmd.c,
            src/fileio.c, src/getchar.c, src/gui_gtk_x11.c, src/hashtab.c,
            src/json.c, src/mbyte.c, src/message.c, src/misc1.c, src/misc2.c,
            src/ops.c, src/option.c, src/os_unix.c, src/proto/os_unix.pro,
            src/proto/regexp.pro, src/proto/terminal.pro, src/regexp.c,
            src/screen.c, src/search.c, src/syntax.c, src/term.c,

src/terminal.c, src/ui.c, src/userfunc.c

Patch 8.1.0786
问题: ml_get error when updating the status line and a terminal had its scrollback cleared. (Chris Patuzzo)
解决方案: Check the cursor position when drawing the status line. (closes #3830)
文件: src/buffer.c, src/testdir/test_terminal.vim

Patch 8.1.0787
问题: Compiler warning for unused function. (Tony Mechelynck)
解决方案: Tune #ifdef around setjmp functions.
文件: src/os_unix.c

Patch 8.1.0788
问题: Cannot build with tiny features.
解决方案: Adjust #ifdefs.
文件: src/os_unix.c

Patch 8.1.0789
问题: Sourcing a session sets v:errmsg.
解决方案: Use "%argdel" instead of "argdel *". (Jason Franklin)
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0790
问题: Code for creating tabpages in session is too complex.
解决方案: Simplify the code. (Jason Franklin)
文件: src/ex_docmd.c

Patch 8.1.0791
问题: A few compiler warnings on VMS.
解决方案: Remove type cast. Adjust #ifdef. (Zoltan Arpadffy)
文件: src/os_unix.c, src/proto.h

Patch 8.1.0792
问题: Popup menu is displayed on top of the cmdline window if it is opened from Insert completion. (Bjorn Linse)
解决方案: Remove the popup menu. Restore the cursor position. (closes #3838)
文件: src/edit.c, src/ex_getln.c

Patch 8.1.0793
问题: Incorrect error messages for functions that now take a Blob argument.
解决方案: Adjust the error messages. (Dominique Pelle, closes #3846)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/globals.h, src/testdir/test_blob.vim, src/testdir/test_listdict.vim

Patch 8.1.0794
问题: White space before " -Ntabmove" causes problems.
解决方案: Skip whitespace. (Ozaki Kiichi, closes #3841)
文件: src/ex_docmd.c, src/testdir/test_tabpage.vim

Patch 8.1.0795 (after 8.1.0792)

问题: Cannot build without popup menu.
解决方案: Add #ifdef
文件: src/ex_getln.c

Patch 8.1.0796
问题: MS-Windows 7: problem with named pipe on channel.
解决方案: Put back the disconnect/connect calls. (Yasuhiro Matsumoto,
            closes #3833)
文件: src/channel.c, src/testdir/test_terminal.vim

Patch 8.1.0797
问题: Error E898 is used twice.
解决方案: Rename the Blob error to E899. (closes #3853)
文件: src/evalfunc.c, runtime/doc/eval.txt,
            src/testdir/test_listdict.vim

Patch 8.1.0798
问题: Changing a blob while iterating over it works strangely.
解决方案: Make a copy of the Blob before iterating.
文件: src/blob.c, src/proto/blob.pro, src/eval.c,
            src/testdir/test_blob.vim

Patch 8.1.0799
问题: Calling deleted function; test doesn't work on Mac.
解决方案: Wait for the function to be called before deleting it. Use a job
            to write to the pty, unless in the GUI. (Ozaki Kiichi,
            closes #3854)
文件: src/testdir/test_channel.vim, src/testdir/test_terminal.vim

Patch 8.1.0800
问题: May use a lot of memory when a function creates a cyclic
            reference.
解决方案: After saving a funccal many times, invoke the garbage collector.
            (closes #3835)
文件: src/userfunc.c

Patch 8.1.0801
问题: MinGW: no hint that tests fail because of small terminal.
解决方案: Add a rule for test1 that checks for "wrongtermsize".
            (msoyka-of-wharton)
文件: src/testdir/Make_ming.mak

Patch 8.1.0802
问题: Negative index doesn't work for Blob.
解决方案: Make it work, add a test. (closes #3856)
文件: src/blob.c, src/proto/blob.pro, src/eval.c,
            src/testdir/test_blob.vim

Patch 8.1.0803
问题: Session file has problem with single quote in file name. (Jon
            Crowe)
解决方案: Use a double quoted string. Add a test.
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0804
问题: Crash when setting v:errmsg to empty list. (Jaon Franklin)
解决方案: Separate getting value and assigning result.
文件: src/eval.c, src/testdir/test_eval_stuff.vim

Patch 8.1.0805
问题: Too many #ifdefs.
解决方案: Graduate FEAT_MBYTE, part 1.
文件: src/buffer.c, src/charset.c, src/diff.c, src/digraph.c,
            src/edit.c, src/eval.c, src/evalfunc.c, src/ex_cmds.c,
            src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c,
            src/fold.c, src/gui.c, src/gui_mac.c, src/gui_photon.c,
            src/gui_w32.c

Patch 8.1.0806
问题: Too many #ifdefs.
解决方案: Graduate FEAT_MBYTE, part 2.
文件: src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/gui_w32.c,
            src/gui_x11.c, src/hardcopy.c, src/if_xcmdsrv.c, src/json.c,
            src/kword_test.c, src/main.c, src/mbyte.c, src/memline.c,
            src/message.c, src/misc1.c, src/misc2.c, src/move.c, src/normal.c,
            src/ops.c, src/option.c, src/charset.c

Patch 8.1.0807
问题: Session test fails on MS-Windows.
解决方案: Don't try creating file with illegal name.
文件: src/testdir/test_mksession.vim

Patch 8.1.0808
问题: MS-Windows: build error with GUI.
解决方案: Remove "static".
文件: src/gui_w32.c

Patch 8.1.0809
问题: Too many #ifdefs.
解决方案: Graduate FEAT_MBYTE, part 3.
文件: src/os_amiga.c, src/os_mswin.c, src/os_unix.c, src/os_w32exe.c,
            src/os_win32.c, src/quickfix.c, src/regexp.c, src/regexp_nfa.c,
            src/screen.c

Patch 8.1.0810
问题: Too many #ifdefs.
解决方案: Graduate FEAT_MBYTE, part 4.
文件: src/getchar.c, src/search.c, src/sign.c, src/spell.c,
            src/spellfile.c, src/syntax.c, src/tag.c, src/term.c, src/ui.c,
            src/version.c, src/winclip.c, src/window.c, src/glbl_ime.cpp,
            src/ex_cmds.h, src/globals.h, src/gui.h, src/if_py_both.h,
            src/macros.h, src/option.h, src/os_mac.h, src/os_win32.h,
            src/proto.h, src/spell.h, src/structs.h, src/vim.h

Patch 8.1.0811
问题: Too many #ifdefs.
解决方案: Graduate FEAT_MBYTE, the final chapter.
文件: src/feature.h, src/vim.h, src/crypt_zip.c, src/fileio.c,

src/message.c, src/spell.h, src/structs.h, src/config.h.in,
src/configure.ac, src/auto/configure, src/testdir/runtest.vim,
src/testdir/test_alot_utf8.vim, src/testdir/test_arabic.vim,
src/testdir/test_charsearch_utf8.vim,
src/testdir/test_cmdline.vim, src/testdir/test_digraph.vim,
src/testdir/test_display.vim, src/testdir/test_edit.vim,
src/testdir/test_erasebackword.vim,
src/testdir/test_expr_utf8.vim, src/testdir/test_functions.vim,
src/testdir/test_ga.vim, src/testdir/test_iminsert.vim,
src/testdir/test_increment_dbcs.vim, src/testdir/test_json.vim,
src/testdir/test_makeencoding.vim, src/testdir/test_maparg.vim,
src/testdir/test_mapping.vim, src/testdir/test_marks.vim,
src/testdir/test_match.vim,
src/testdir/test_matchadd_conceal_utf8.vim,
src/testdir/test_mksession_utf8.vim, src/testdir/test_normal.vim,
src/testdir/test_plus_arg_edit.vim, src/testdir/test_profile.vim,
src/testdir/test_put.vim, src/testdir/test_regex_char_classes.vim,
src/testdir/test_regexp_utf8.vim, src/testdir/test_search.vim,
src/testdir/test_source_utf8.vim, src/testdir/test_spell.vim,
src/testdir/test_startup_utf8.vim,
src/testdir/test_termencoding.vim, src/testdir/test_terminal.vim,
src/testdir/test_utf8.vim, src/testdir/test_utf8_comparisons.vim,
src/testdir/test_viminfo.vim, src/testdir/test_virtualedit.vim,
src/testdir/test_visual.vim, src/testdir/test_wordcount.vim,
src/testdir/test_writefile.vim, src/appveyor.bat, src/os_macosx.m

Patch 8.1.0812
问题: Unicode 16 feature is not useful and cannot be detected.
解决方案: Remove UNICODE16.
文件: src/screen.c, src/vim.h, src/feature.h

Patch 8.1.0813
问题: FileChangedShell not sufficiently tested.
解决方案: Add a more comprehensive test case.
文件: src/testdir/test_autocmd.vim

Patch 8.1.0814
问题: :mksession cannot handle a very long 'runtimepath'. (Timothy
Madden)
解决方案: Expand each part separately, instead of the whole option at once.
(Christian Brabandt, closes #3466)
文件: src/option.c, src/testdir/test_mksession.vim

Patch 8.1.0815
问题: Dialog for file changed outside of Vim not tested.
解决方案: Add a test. Move FileChangedShell test. Add 'L' flag to
feedkeys().
文件: src/testdir/test_autocmd.vim, src/testdir/test_filechanged.vim,
src/testdir/Make_all.mak, src/evalfunc.c, runtime/doc/eval.txt

Patch 8.1.0816
问题: Test for 'runtimepath' in session fails on MS-Windows.
解决方案: Skip the test for now.
文件: src/testdir/test_mksession.vim

Patch 8.1.0817
问题: ":=" command is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3859)
文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim,
src/testdir/test_ex_equal.vim

Patch 8.1.0818
问题: MS-Windows: cannot send large data with ch_sendraw().
解决方案: Split write into several WriteFile() calls. (Yasuhiro Matsumoto,
closes #3823)
文件: src/channel.c, src/os_win32.c, src/testdir/test_channel.vim,
src/testdir/test_channel_pipe.py, src/vim.h

Patch 8.1.0819
问题: A failed assert with a long string is hard to read.
解决方案: Shorten the assert message.
文件: src/eval.c, src/testdir/test_assert.vim

Patch 8.1.0820
问题: Test for sending large data over channel sometimes fails.
解决方案: Handle that the job may have finished early. Also fix that file
changed test doesn't work in the GUI and reduce flakyness. (Ozaki
Kiichi, closes #3861)
文件: src/testdir/test_channel.vim, src/testdir/test_filechanged.vim

Patch 8.1.0821
问题: Xxd "usage" output and other arguments not tested.
解决方案: Add a test to trigger the usage output in various ways. Fix
uncovered problem.
文件: src/testdir/test_xxd.vim, src/xxd/xxd.c

Patch 8.1.0822
问题: Peeking and flushing output slows down execution.
解决方案: Do not update the mode message when global_busy is set. Do not
flush when only peeking for a character. (Ken Takata)
文件: src/getchar.c, src/screen.c, src/proto/screen.pro, src/edit.c

Patch 8.1.0823
问题: Not sufficient testing of xxd.
解决方案: Add some more test coverage.
文件: src/testdir/test_xxd.vim

Patch 8.1.0824
问题: SunOS/Solaris has a problem with ttys.
解决方案: Add mch_isatty() with extra handling for SunOS. (Ozaki Kiichi,
closes #3865)
文件: src/auto/configure, src/channel.c, src/config.h.in,
src/configure.ac, src/os_unix.c, src/proto/pty.pro, src/pty.c,
src/terminal.c

Patch 8.1.0825
问题: Code for autocommands is mixed with file I/O code.
解决方案: Move autocommand code to a separate file. (Yegappan Lakshmanan,

closes #3863)
文件: Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
src/Make_vms.mms, src/Makefile, src/README.txt, src/autocmd.c,
src/fileio.c, src/globals.h, src/proto.h, src/proto/autocmd.pro,
src/proto/fileio.pro

Patch 8.1.0826
问题: Too many #ifdefs.
解决方案: Graduate FEAT_VIRTUALEDIT. Adds about 10Kbyte to the code.
文件: src/buffer.c, src/charset.c, src/edit.c, src/eval.c,
src/evalfunc.c, src/ex_cmds.c, src/ex_docmd.c, src/feature.h,
src/globals.h, src/gui.c, src/if_py_both.h, src/macros.h,
src/mark.c, src/mbyte.c, src/memline.c, src/menu.c, src/misc1.c,
src/misc2.c, src/move.c, src/netbeans.c, src/normal.c, src/ops.c,
src/option.c, src/option.h, src/screen.c, src/search.c,
src/spell.c, src/structs.h, src/tag.c, src/ui.c, src/undo.c,
src/userfunc.c, src/version.c, src/vim.h, src/window.c

Patch 8.1.0827 (after 8.1.0825)
问题: Missing dependency in Makefile.
解决方案: Add dependency from autocmd.o on auto/osdef.h
文件: src/Makefile

Patch 8.1.0828
问题: Still using FEAT_VIRTUALEDIT.
解决方案: Remove last use of FEAT_VIRTUALEDIT.
文件: src/quickfix.c

Patch 8.1.0829
问题: When 'hidden' is set session creates extra buffers.
解决方案: Move :badd commands to the end. (Jason Franklin)
文件: src/ex_docmd.c, src/testdir/test_mksession.vim

Patch 8.1.0830
问题: Test leaves directory behind on MS-Windows.
解决方案: Close buffer before deleting directory.
文件: src/testdir/test_swap.vim

Patch 8.1.0831
问题: Xxd test fails if man page has dos fileformat.
解决方案: Make a copy with unix fileformat.
文件: src/testdir/test_xxd.vim

Patch 8.1.0832
问题: confirm() is not tested.
解决方案: Add a test. (Dominique Pelle, closes #3868)
文件: src/testdir/test_functions.vim

Patch 8.1.0833
问题: Memory leak when jumps output is filtered.
解决方案: Free the filtered name. (Dominique Pelle, closes #3869)
文件: src/mark.c

Patch 8.1.0834
问题: GUI may wait too long before dealing with messages. Returning
            early may cause a mapping to time out.
解决方案: Use the waiting loop from Unix also for the GUI.
            (closes #3817, closes #3824)
文件: src/ui.c, src/proto/ui.pro, src/os_unix.c, src/gui.c,
            src/testdir/screendump.vim

Patch 8.1.0835
问题: GUI build fails on MS-Windows.
解决方案: Adjust #ifdef.
文件: src/ui.c

Patch 8.1.0836
问题: User completion test can fail on MS-Windows.
解决方案: Allow for other names before "Administrator".
文件: src/testdir/test_cmdline.vim

Patch 8.1.0837
问题: Timer interrupting cursorhold and mapping not tested.
解决方案: Add tests with timers. (Ozaki Kiichi, closes #3871)
文件: src/testdir/test_autocmd.vim, src/testdir/test_mapping.vim

Patch 8.1.0838
问题: Compiler warning for type conversion.
解决方案: Add a type cast. (Mike Williams)
文件: src/channel.c

Patch 8.1.0839
问题: When using VTP wrong colors after a color scheme change.
解决方案: When VTP is active always clear after a color scheme change.
            (Nobuhiro Takasaki, closes #3872)
文件: src/ex_docmd.c

Patch 8.1.0840
问题: getchar(0) never returns a character in the terminal.
解决方案: Call wait_func() at least once.
文件: src/ui.c, src/testdir/test_timers.vim, src/gui_gtk_x11.c,
            src/gui_w32.c, src/gui_photon.c, src/gui_x11.c

Patch 8.1.0841
问题: Travis config to get Lua on MacOS is too complicated.
解决方案: Use an addons entry. (Ozaki Kiichi, closes #3876)
文件: .travis.yml

Patch 8.1.0842
问题: getchar_zero test fails on MS-Windows.
解决方案: Disable the test for now.
文件: src/testdir/test_timers.vim

Patch 8.1.0843
问题: Memory leak when running "make test_cd".
解决方案: Free the stack element when failing. (Dominique Pelle,

closes #3877)
文件: src/misc2.c

Patch 8.1.0844
问题: When timer fails test will hang forever.
解决方案: Use reltime() to limit waiting time. (Ozaki Kiichi, closes #3878)
文件: src/testdir/test_timers.vim

Patch 8.1.0845
问题: Having job_status() free the job causes problems.
解决方案: Do not actually free the job or terminal yet, put it in a list and free it a bit later. Do not use a terminal after checking the job status. (closes #3873)
文件: src/channel.c, src/terminal.c, src/proto/terminal.pro, src/misc2.c

Patch 8.1.0846
问题: Not easy to recognize the system Vim runs on.
解决方案: Add more items to the features list. (Ozaki Kiichi, closes #3855)
文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_channel.vim, src/testdir/test_functions.vim,
src/testdir/test_terminal.vim, src/testdir/test_writefile.vim

Patch 8.1.0847
问题: May use terminal after it was cleaned up.
解决方案: Use the job pointer.
文件: src/terminal.c

Patch 8.1.0848
问题: Cannot build with Ruby 1.8. (Tom G. Christensen)
解决方案: Use rb-str_new2(). (Yasuhiro Matsumoto, closes #3883,
closes #3884)
文件: src/if_ruby.c

Patch 8.1.0849
问题: Cursorline highlight is not always updated.
解决方案: Set w_last_cursorline when redrawing. Fix resetting cursor flags when using the popup menu.
文件: src/screen.c, src/popupmnu.c, src/testdir/test_highlight.vim,
src/testdir/dumps/Test_cursorline_yank_01.dump

Patch 8.1.0850
问题: Test for 'backupskip' is not correct.
解决方案: Split the option in parts and use expand(). (Michael Soyka)
文件: src/testdir/test_options.vim

Patch 8.1.0851
问题: feedkeys() with "L" does not work properly.
解决方案: Do not set typebuf_was_filled when using "L". (Ozaki Kiichi, closes #3885)
文件: src/evalfunc.c, src/testdir/test_autocmd.vim,
src/testdir/test_mapping.vim, src/testdir/test_timers.vim

Patch 8.1.0852
问题: findfile() and finddir() are not properly tested.

解决方案: Extend the test and add more. (Dominique Pelle, closes #3880)
文件: src/testdir/test_findfile.vim

Patch 8.1.0853 (after 8.1.0850)
问题: Options test fails on Mac.
解决方案: Remove a trailing slash from $TMPDIR.
文件: src/testdir/test_options.vim

Patch 8.1.0854
问题: xxd does not work with more than 32 bit addresses.
解决方案: Add support for 64 bit addresses. (Christer Jensen, closes #3791)
文件: src/xxd/xxd.c

Patch 8.1.0855
问题: Cannot build xxd with MSVC 10.
解决方案: Move declaration to start of block.
文件: src/xxd/xxd.c

Patch 8.1.0856
问题: When scrolling a window other than the current one the cursorline
highlighting is not always updated. (Jason Franklin)
解决方案: Call redraw_for_cursorline() after scrolling. Only set
w_last_cursorline when drawing the cursor line. Reset the lines
to be redrawn also when redrawing the whole window.
文件: src/move.c, src/proto/move.pro, src/normal.c

Patch 8.1.0857
问题: Indent functionality is not separated.
解决方案: Move indent functionality into a new file. (Yegappan Lakshmanan,
closes #3886)
文件: Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
src/Make_vms.mms, src/Makefile, src/edit.c, src/indent.c,
src/misc1.c, src/proto.h, src/proto/edit.pro,
src/proto/indent.pro, src/proto/misc1.pro

Patch 8.1.0858
问题: 'indentkeys' and 'cinkeys' defaults are different.
解决方案: Make them the same, update docs. (close #3882)
文件: src/option.c, runtime/doc/options.txt, runtime/doc/indent.txt

Patch 8.1.0859
问题: "%v" in 'errorformat' does not handle multi-byte characters.
解决方案: Handle multi-byte characters. (Yegappan Lakshmanan, closes #3700)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0860
问题: Debug lines left in the code.
解决方案: Delete the lines.
文件: src/edit.c

Patch 8.1.0861
问题: Building with MinGW and static libc doesn't work.

解决方案: Change the LIB argument. (Ken Takata)
文件: src/Make_cyg_ming.mak

Patch 8.1.0862
问题: No verbose version of character classes.
解决方案: Add [:ident:], [:keyword:] and [:fname:]. (Ozaki Kiichi,
closes #1373)
文件: runtime/doc/pattern.txt, src/regexp.c, src/regexp_nfa.c,
src/testdir/test_regexp_utf8.vim

Patch 8.1.0863
问题: Cannot see what signal caused a job to end.
解决方案: Add "termsig" to job_info(). (Ozaki Kiichi, closes #3786)
文件: runtime/doc/eval.txt, src/channel.c, src/os_unix.c, src/structs.h,
src/testdir/test_channel.vim

Patch 8.1.0864
问题: Cannot have a local value for 'scrolloff' and 'sidescrolloff'.
(Gary Holloway)
解决方案: Make 'scrolloff' and 'sidescrolloff' global-local. (mostly by
Aron Widforss, closes #3539)
文件: runtime/doc/options.txt, src/edit.c, src/ex_cmds.c,
src/ex_docmd.c, src/gui.c, src/misc2.c, src/move.c, src/normal.c,
src/option.c, src/proto/option.pro, src/option.h, src/search.c,
src/structs.h, src/window.c, src/testdir/test_options.vim

Patch 8.1.0865
问题: When 'listchars' only contains "nbsp:X" it does not work.
解决方案: Set extra_check when lcs_nbsp is set. (Ralf Schandl, closes #3889)
文件: src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.0866
问题: Build file dependencies are outdated. (John Little)
解决方案: Run "make proto" and "make depend".
文件: src/vim.h, src/Makefile, src/proto/sign.pro, src/proto/gui_w32.pro

Patch 8.1.0867
问题: Cannot build Python interface with Python 2.4. (Tom G. Christensen)
解决方案: Define PyBytes_FromStringAndSize. (Ken Takata, closes #3888)
文件: src/if_python.c

Patch 8.1.0868
问题: Crash if triggering garbage collector after a function call.
(Michael Henry)
解决方案: Don't call the garbage collector right away, do it later.
(closes #3894)
文件: src/userfunc.c

Patch 8.1.0869
问题: Travis CI script is too complicated.
解决方案: Add names to environments. Move appveyor script outside of src
directory. (Ozaki Kiichi, closes #3890)
文件: .travis.yml, appveyor.yml, ci/appveyor.bat, src/appveyor.bat,
Filelist

Patch 8.1.0870
问题: Vim doesn't use the new ConPTY support in Windows 10.
解决方案: Use ConPTY support, if available. (Nobuhiro Takasaki, closes #3794)
文件: runtime/doc/eval.txt, runtime/doc/options.txt,
runtime/doc/terminal.txt, src/channel.c, src/evalfunc.c,
src/globals.h, src/option.c, src/option.h, src/os_win32.c,
src/proto/terminal.pro, src/structs.h, src/terminal.c,
src/testdir/gen_opt_test.vim, src/testdir/test_autocmd.vim,
src/testdir/test_mksession.vim, src/testdir/test_terminal.vim

Patch 8.1.0871
问题: Build error when building with Ruby 2.6.0.
解决方案: Change argument of rb_int2big_stub(). (Android Baumann,
closes #3899)
文件: src/if_ruby.c

Patch 8.1.0872
问题: Confusing condition.
解决方案: Use "==" instead of "<=".
文件: src/gui_gtk_x11.c

Patch 8.1.0873
问题: List if distributed files does not include the matchit autoload
directory.
解决方案: Add the directory.
文件: src/Filelist

Patch 8.1.0874
问题: Using old style comments in new file.
解决方案: Convert to // comments in new file. (Yegappan Lakshmanan)
文件: src/indent.c

Patch 8.1.0875
问题: Not all errors of marks and findfile()/finddir() are tested.
解决方案: Add more test coverage. (Dominique Pelle)
文件: src/testdir/test_findfile.vim, src/testdir/test_marks.vim

Patch 8.1.0876
问题: Completion match not displayed when popup menu is not shown.
解决方案: Call update_screen() when not displaying the popup menu to show
the inserted match. (Ken Takata, Hirohito Higashi)
文件: src/edit.c

Patch 8.1.0877
问题: New buffer used every time the quickfix window is opened.
解决方案: Reuse the buffer. (Yegappan Lakshmanan, closes #3902)
文件: src/buffer.c, src/proto/quickfix.pro, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.1.0878
问题: Test for has('bsd') fails on some BSD systems.
解决方案: Adjust the uname match. (James McCoy, closes #3909)
文件: src/testdir/test_functions.vim

Patch 8.1.0879
问题: MS-Windows: temp name encoding can be wrong.
解决方案: Convert from active code page to 'encoding'. (Yasuhiro Matsumoto, closes #3520, closes #1698)
文件: src/fileio.c

Patch 8.1.0880
问题: MS-Windows: inconsistent selection of winpty/conpty.
解决方案: Name option 'termwintype', use ++type argument and "term_pty" for term_start(). (Hirohito Higashi, closes #3915)
文件: runtime/doc/eval.txt, runtime/doc/options.txt, runtime/doc/terminal.txt, src/channel.c, src/option.c, src/option.h, src/structs.h, src/terminal.c, src/testdir/gen_opt_test.vim, runtime/optwin.vim, runtime/doc/quickref.txt

Patch 8.1.0881
问题: Can execute shell commands in rvim through interfaces.
解决方案: Disable using interfaces in restricted mode. Allow for writing file with writefile(), histadd() and a few others.
文件: runtime/doc/starting.txt, src/if_perl.xs, src/if_cmds.h, src/ex_cmds.c, src/ex_docmd.c, src/evalfunc.c, src/testdir/test_restricted.vim, src/testdir/Make_all.mak

Patch 8.1.0882 (after 8.1.0879)
问题: Checking for FEAT_MBYTE which doesn't exist anymore. (Christ van Willegen)
解决方案: Remove it.
文件: src/fileio.c

Patch 8.1.0883
问题: Missing some changes for Ex commands.
解决方案: Add missing changes in header file.
文件: src/ex_cmds.h

Patch 8.1.0884
问题: Double check for bsd systems.
解决方案: Delete the old line.
文件: src/testdir/test_functions.vim

Patch 8.1.0885
问题: Test for restricted hangs on MS-Windows GUI.
解决方案: Skip the test.
文件: src/testdir/test_restricted.vim

Patch 8.1.0886
问题: Compiler warning for adding to NULL pointer and a condition that is always true.
解决方案: Check for NULL pointer before adding. Remove useless "if". (Friedirch, closes #3913)
文件: src/dosinst.c, src/search.c

Patch 8.1.0887

问题: The 'l' flag in :substitute is sticky.
解决方案: Reset the flag. (Dominique Pelle, closes #3925)
文件: src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.1.0888
问题: The a: dict is not immutable as documented.
解决方案: Make the a:dict immutable, add a test. (Ozaki Kiichi, Yasuhiro
            Matsumoto, closes #3929)
文件: src/eval.c, src/userfunc.c, src/testdir/test_let.vim,
            src/testdir/test_listdict.vim

Patch 8.1.0889
问题: MS-Windows: a channel write may hang.
解决方案: Check for WriteFile() not writing anything. (Yasuhiro Matsumoto,
            closes #3920)
文件: src/channel.c, src/testdir/test_channel.vim,
            src/testdir/test_channel_pipe.py

Patch 8.1.0890
问题: Pty allocation wrong if using file for out channel and using null
            for in channel and null for error channel.
解决方案: Correct using use_file_for_out in condition. (Ozaki Kiichi, closes
            #3917)
文件: src/os_unix.c, src/testdir/test_channel.vim

Patch 8.1.0891
问题: Substitute command insufficiently tested.
解决方案: Add more test coverage. (Dominique Pelle)
文件: src/testdir/test_substitute.vim

Patch 8.1.0892
问题: Failure when closing a window when location list is in use.
解决方案: Handle the situation gracefully. Make sure memory for 'switchbuf'
            is not freed at the wrong time. (Yegappan Lakshmanan,
            closes #3928)
文件: src/eval.c, src/evalfunc.c, src/proto/window.pro, src/quickfix.c,
            src/testdir/test_quickfix.vim, src/window.c

Patch 8.1.0893
问题: Terminal test is a bit flaky.
解决方案: Add test_terminal_no_cmd() to list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0894
问题: MS-Windows: resolve() does not return a reparse point.
解决方案: Improve resolve(). (Yasuhiro Matsumoto, closes #3896)
文件: runtime/doc/eval.txt, src/buffer.c, src/evalfunc.c,
            src/os_mswin.c, src/proto/os_mswin.pro,
            src/testdir/test_functions.vim

Patch 8.1.0895 (after 8.1.0879)
问题: MS-Windows: dealing with temp name encoding not quite right.
解决方案: Use more wide functions. (Ken Takata, closes #3921)
文件: src/fileio.c

Patch 8.1.0896
问题: Tests for restricted mode not run for MS-Windows GUI.
解决方案: Make tests also work in MS-Windows GUI.
文件: src/testdir/test_restricted.vim

Patch 8.1.0897
问题: Can modify a:000 when using a reference.
解决方案: Make check for locked variable stricter. (Ozaki Kiichi,
            closes #3930)
文件: src/dict.c, src/eval.c, src/evalfunc.c, src/proto/eval.pro,
            src/testdir/test_channel.vim, src/testdir/test_let.vim,
            src/userfunc.c

Patch 8.1.0898
问题: A messed up rgb.txt can crash Vim. (Pavel Cheremushkin)
解决方案: Limit to 10000 entries. Also don't retry many times when the file
            cannot be read.
文件: src/term.c

Patch 8.1.0899
问题: No need to check restricted mode for setwinvar().
解决方案: Remove check_restricted().
文件: src/eval.c

Patch 8.1.0900
问题: ConPTY may crash with 32-bit build.
解决方案: Fix function declarations. (Ken Takata, closes #3943)
文件: src/terminal.c

Patch 8.1.0901
问题: Index in getjumplist() may be wrong. (Epheien)
解决方案: Call cleanup_jumplist() earlier. (Yegappan Lakshmanan,
            closes #3942)
文件: src/evalfunc.c, src/testdir/test_jumplist.vim

Patch 8.1.0902
问题: Incomplete set of assignment operators.
解决方案: Add /=, *= and %=. (Ozaki Kiichi, closes #3931)
文件: runtime/doc/eval.txt src/eval.c src/testdir/test_vimscript.vim

Patch 8.1.0903
问题: Struct uses more bytes than needed.
解决方案: Reorder members of regitem_S. (Dominique Pelle, closes #3936)
文件: src/regexp.c

Patch 8.1.0904
问题: USE_LONG_FNAME never defined.
解决方案: Remove using USE_LONG_FNAME. (Ken Takata, closes #3938)
文件: src/buffer.c, src/ex_cmds.c, src/fileio.c

Patch 8.1.0905
问题: Complicated regexp causes a crash. (Kuang-che Wu)
解决方案: Limit the recursiveness of addstate(). (closes #3941)

文件:　　src/regexp_nfa.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0906
问题:　　Using clumsy way to get console window handle.
解决方案:　Use GetConsoleWindow(). (Ken Takata, closes #3940)
文件:　　src/os_mswin.c

Patch 8.1.0907
问题:　　CI tests on AppVeyor are failing.
解决方案:　Reduce the recursiveness limit for regexp.
文件:　　src/regexp_nfa.c

Patch 8.1.0908
问题:　　Can't handle large value for %{nr}v in regexp. (Kuang-che Wu)
解决方案:　Give an error if the value is too large. (closes #3948)
文件:　　src/regexp_nfa.c

Patch 8.1.0909
问题:　　MS-Windows: using ConPTY even though it is not stable.
解决方案:　When ConPTY version is unstable, prefer using winpty. (Ken Takata,
　　closes #3949)
文件:　　runtime/doc/options.txt, src/os_win32.c, src/proto/os_win32.pro,
　　src/terminal.c

Patch 8.1.0910
问题:　　Crash with tricky search pattern. (Kuang-che Wu)
解决方案:　Check for running out of memory. (closes #3950)
文件:　　src/regexp_nfa.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0911
问题:　　Tag line with Ex command cannot have extra fields.
解决方案:　Recognize |;" as the end of the command. (closes #2402)
文件:　　runtime/doc/tagsrch.txt, src/tag.c, src/testdir/test_taglist.vim

Patch 8.1.0912
问题:　　MS-Windows: warning for signed/unsigned.
解决方案:　Add type cast. (Nobuhiro Takasaki, closes #3945)
文件:　　src/terminal.c

Patch 8.1.0913
问题:　　CI crashes when running out of memory.
解决方案:　Apply 'maxmempattern' also to new regexp engine.
文件:　　src/regexp_nfa.c

Patch 8.1.0914
问题:　　Code related to findfile() is spread out.
解决方案:　Put findfile() related code into a new source file. (Yegappan
　　Lakshmanan, closes #3934)
文件:　　Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
　　src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
　　src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
　　src/Make_vms.mms, src/Makefile, src/README.txt, src/findfile.c,
　　src/misc1.c, src/misc2.c, src/proto.h, src/proto/findfile.pro,
　　src/proto/misc1.pro, src/proto/misc2.pro, src/proto/window.pro,

src/window.c

Patch 8.1.0915
问题: fsync() may not work properly on Mac.
解决方案: Use fcntl() with F_FULLFSYNC. (suggested by Justin M. Keyes)
文件: src/fileio.c, src/proto/fileio.pro, src/evalfunc.c, src/memfile.c

Patch 8.1.0916
问题: With Python 3.7 "find_module" is not made available.
解决方案: Also add "find_module" with Python 3.7. (Joel Frederico,
closes #3954)
文件: src/if_py_both.h

Patch 8.1.0917
问题: Double free when running out of memory.
解决方案: Remove one free. (Ken Takata, closes #3955)
文件: src/userfunc.c

Patch 8.1.0918
问题: MS-Windows: startup messages are not converted.
解决方案: Convert messages when the current codepage differs from
'encoding'. (Yasuhiro Matsumoto, closes #3914)
文件: src/message.c, src/os_mswin.c, src/vim.h

Patch 8.1.0919
问题: Compiler warnings.
解决方案: Add type casts. (Mike Williams)
文件: src/message.c, src/regexp_nfa.c

Patch 8.1.0920
问题: In Terminal-Normal mode job output messes up the window.
解决方案: Postpone scrolling and updating the buffer when in Terminal-Normal
mode.
文件: src/terminal.c, src/testdir/test_terminal.vim,
src/testdir/dumps/Test_terminal_01.dump,
src/testdir/dumps/Test_terminal_02.dump,
src/testdir/dumps/Test_terminal_03.dump

Patch 8.1.0921
问题: Terminal test sometimes fails; using memory after free.
解决方案: Fee memory a bit later. Add test to cover this. Disable flaky
screenshot test. (closes #3956)
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0922
问题: Terminal scrollback test is flaky.
解决方案: Wait a bit before running the tail command.
文件: src/testdir/test_terminal.vim,
src/testdir/dumps/Test_terminal_01.dump,
src/testdir/dumps/Test_terminal_02.dump,
src/testdir/dumps/Test_terminal_03.dump

Patch 8.1.0923
问题: Terminal dump diff swap does not update file names.

解决方案: Also swap the file name. Add a test.
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.0924
问题: Terminal scrollback test still flaky.
解决方案: Wait a bit longer before running the tail command.
文件: src/testdir/test_terminal.vim

Patch 8.1.0925
问题: Terminal scrollback test still still flaky.
解决方案: Explicitly set the shell. Disable ruler. (Ozaki Kiichi,
           closes #3966)
文件: src/testdir/test_terminal.vim,
           src/testdir/dumps/Test_terminal_01.dump,
           src/testdir/dumps/Test_terminal_02.dump,
           src/testdir/dumps/Test_terminal_03.dump

Patch 8.1.0926
问题: No test for :wnext, :wNext and :wprevious.
解决方案: Add a test. (Dominique Pelle, closes #3963)
文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim,
           src/testdir/test_wnext.vim

Patch 8.1.0927
问题: USE_CR is never defined.
解决方案: Remove usage of USE_CR. (Ken Takata, closes #3958)
文件: runtime/doc/options.txt, src/diff.c, src/evalfunc.c,
           src/ex_cmds2.c, src/fileio.c, src/message.c, src/ops.c,
           src/option.h, src/proto/ex_cmds2.pro, src/proto/fileio.pro,
           src/tag.c

Patch 8.1.0928 (after 8.1.0927)
问题: Stray log function call.
解决方案: Remove the log function call.
文件: src/ex_cmds2.c

Patch 8.1.0929
问题: No error when requesting ConPTY but it's not available.
解决方案: Add an error message. (Hirohito Higashi, closes #3967)
文件: runtime/doc/terminal.txt, src/terminal.c

Patch 8.1.0930
问题: Typo in Makefile.
解决方案: Change ABORT_CLFAGS to ABORT_CFLAGS. (Kuang-che Wu, closes #3977)
文件: src/Makefile

Patch 8.1.0931
问题: vtp_working included in GUI build but unused.
解决方案: Adjust #ifdefs. (Ken Takata, closes #3971)
文件: src/os_win32.c

Patch 8.1.0932
问题: Farsi support is outdated and unused.
解决方案: Delete the Farsi support.

文件: Filelist, src/farsi.c, src/proto/farsi.pro, src/farsi.h, src/edit.c,
src/main.c, src/normal.c, src/option.c, src/getchar.c,
src/ex_cmds.c, src/search.c, src/ex_getln.c, src/charset.c,
src/evalfunc.c, src/screen.c, src/window.c, src/globals.h,
src/proto.h, farsi/README.txt, src/structs.h,
farsi/fonts/DOS/far-a01.com, farsi/fonts/SunOs/far-a01.fb,
farsi/fonts/UNIXs/far-a01.f16, farsi/fonts/UNIXs/far-a01.pcf.gz,
farsi/fonts/UNIXs/far-a01.pcf.Z, farsi/fonts/WINDOWS/far-a01.fon,
src/Makefile, src/Make_bc5.mak, src/Make_cyg_ming.mak,
src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
src/Make_vms.mms, src/configure.ac, src/auto/configure,
src/config.h.in, src/testdir/test_farsi.vim, src/version.c,
src/testdir/Make_all.mak, runtime/doc/options.txt,
runtime/doc/starting.txt, runtime/doc/quickref.txt,
runtime/doc/farsi.txt

Patch 8.1.0933
问题: When using VTP scroll region isn't used properly.
解决方案: Make better use of the scroll region. (Nobuhiro Takasaki,
closes #3974)
文件: src/os_win32.c, src/term.c

Patch 8.1.0934
问题: Invalid memory access in search pattern. (Kuang-che Wu)
解决方案: Check for incomplete equivalence class. (closes #3970)
文件: src/regexp.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0935
问题: Old regexp engine may use invalid buffer for 'iskeyword' or
uninitialized buffer pointer. (Kuang-che Wu)
解决方案: Set rex.reg_buf when compiling the pattern. (closes #3972)
文件: src/regexp.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0936
问题: May leak memory when using 'vartabstop'. (Kuang-che Wu)
解决方案: Fix handling allocated memory for 'vartabstop'. (closes #3976)
文件: src/option.c, src/buffer.c

Patch 8.1.0937
问题: Invalid memory access in search pattern. (Kuang-che Wu)
解决方案: Check for incomplete collation element. (Dominique Pelle,
closes #3985)
文件: src/regexp.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0938
问题: Background color is wrong in MS-Windows console when not using VTP.
解决方案: Use g_attrCurrent. (Nobuhiro Takasaki, closes #3987)
文件: src/os_win32.c

Patch 8.1.0939
问题: No completion for sign group names.
解决方案: Add completion for sign group names and buffer names. (Yegappan
Lakshmanan, closes #3980)

文件: src/sign.c, src/testdir/test_signs.vim

Patch 8.1.0940
问题: MS-Windows console resizing not handled properly.
解决方案: Handle resizing the console better. (Nobuhiro Takasaki, Ken
            Takata, closes #3968, closes #3611)
文件: src/ex_docmd.c, src/normal.c, src/os_win32.c,
            src/proto/os_win32.pro

Patch 8.1.0941
问题: Macros for MS-Windows are inconsistent, using "32", "3264" and
            others.
解决方案: Use MSWIN for all MS-Windows builds. Use FEAT_GUI_MSWIN for the
            GUI build. (Hirohito Higashi, closes #3932)
文件: src/GvimExt/gvimext.h, src/Make_bc5.mak, src/Make_cyg_ming.mak,
            src/Make_ivc.mak, src/Make_mvc.mak, src/beval.h, src/blowfish.c,
            src/channel.c, src/edit.c, src/eval.c, src/evalfunc.c,
            src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c,
            src/feature.h, src/fileio.c, src/getchar.c, src/glbl_ime.cpp,
            src/globals.h, src/gui.c, src/gui.h, src/gui_beval.c,
            src/gui_gtk.c, src/gui_gtk_f.c, src/gui_gtk_x11.c,
            src/if_cscope.c, src/if_cscope.h, src/if_lua.c, src/if_mzsch.c,
            src/if_ole.cpp, src/if_perl.xs, src/if_python.c, src/if_python3.c,
            src/if_ruby.c, src/if_tcl.c, src/macros.h, src/main.c,
            src/mbyte.c, src/memfile.c, src/memline.c, src/menu.c,
            src/message.c, src/misc1.c, src/misc2.c, src/nbdebug.c,
            src/netbeans.c, src/normal.c, src/option.c, src/option.h,
            src/os_mswin.c, src/os_unix.c, src/os_w32exe.c, src/os_win32.c,
            src/os_win32.h, src/proto.h, src/screen.c, src/search.c,
            src/structs.h, src/syntax.c, src/term.c, src/terminal.c, src/ui.c,
            src/undo.c, src/version.c, src/vim.h, src/vim.rc, src/winclip.c

Patch 8.1.0942
问题: Options window still checks for the multi_byte feature.
解决方案: Remove the unnecessary check. (Dominique Pelle, closes #3990)
文件: runtime/optwin.vim

Patch 8.1.0943
问题: Still a trace of Farsi support.
解决方案: Remove defining macros.
文件: src/feature.h

Patch 8.1.0944
问题: Format of nbdbg() arguments is not checked.
解决方案: Add format attribute. Fix reported problems. (Dominique Pelle,
            closes #3992)
文件: src/nbdebug.h, src/netbeans.c

Patch 8.1.0945
问题: Internal error when using pattern with NL in the range.
解决方案: Use an actual newline for the range. (closes #3989) Also fix
            error message. (Dominique Pelle)
文件: src/regexp_nfa.c, src/testdir/test_regexp_latin.vim

Patch 8.1.0946
问题:　　Coveralls is not very useful.
解决方案:　Remove Coveralls badge, add badge for packages.
文件:　　README.md

Patch 8.1.0947
问题:　　Using MSWIN before it is defined. (Cesar Romani)
解决方案:　Move the block that uses MSWIN to below including vim.h. (Ken
Takata)
文件:　　src/if_ruby.c

Patch 8.1.0948
问题:　　When built without +eval "Vim --clean" produces errors. (James
McCoy)
解决方案:　Do not enable filetype detection.
文件:　　runtime/defaults.vim

Patch 8.1.0949
问题:　　MS-Windows defines GUI macros different than other systems.
解决方案:　Swap FEAT_GUI and FEAT_GUI_MSWIN. (Hirohito Higashi, closes #3996)
文件:　　src/Make_bc5.mak, src/Make_cyg_ming.mak, src/Make_ivc.mak,
src/Make_mvc.mak, src/if_ole.cpp, src/vim.h, src/vim.rc

Patch 8.1.0950
问题:　　Using :python sets 'pyxversion' even when not executed.
解决方案:　Check the "skip" flag. (Shane Harper, closes #3995)
文件:　　src/if_python.c, src/if_python3.c, src/testdir/test_python2.vim,
src/testdir/test_python3.vim

Patch 8.1.0951
问题:　　Using WIN64 even though it is never defined.
解决方案:　Only use _WIN64. (Ken Takata, closes #3997)
文件:　　src/evalfunc.c

Patch 8.1.0952
问题:　　Compilation warnings when building the MS-Windows installer.
解决方案:　Fix buffer sizes. (Yasuhiro Matsumoto, closes #3999)
文件:　　src/dosinst.c, src/dosinst.h, src/uninstal.c

Patch 8.1.0953
问题:　　A very long file is truncated at 2^31 lines.
解决方案:　Use LONG_MAX for MAXLNUM. (Dominique Pelle, closes #4011)
文件:　　src/vim.h

Patch 8.1.0954
问题:　　Arguments of semsg() and siemsg() are not checked.
解决方案:　Add function prototype with __attribute__.
文件:　　src/message.c, src/proto/message.pro, src/proto.h

Patch 8.1.0955
问题:　　Matchit autoload directory not in installer. (Chris Morgan)
解决方案:　Adjust the NSIS script. (Christian Brabandt, closes #4006)
文件:　　nsis/gvim.nsi

Patch 8.1.0956
问题: Using context:0 in 'diffopt' does not work well.
解决方案: Make zero context do the same as one line context. (closes #4005)
文件: src/diff.c, src/testdir/test_diffmode.vim,
            src/testdir/dumps/Test_diff_06.0.dump,
            src/testdir/dumps/Test_diff_06.1.dump,
            src/testdir/dumps/Test_diff_06.2.dump

Patch 8.1.0957 (after 8.1.0915)
问题: Mac: fsync fails on network share.
解决方案: Check for ENOTSUP. (Yee Cheng Chin, closes #4016)
文件: src/fileio.c

Patch 8.1.0958
问题: Compiling weird regexp pattern is very slow.
解决方案: When reallocating post list increase size by 50%. (Kuang-che Wu,
            closes #4012) Make assert_inrange() accept float values.
文件: src/regexp_nfa.c, src/eval.c, src/testdir/test_regexp_latin.vim,
            src/testdir/test_assert.vim

Patch 8.1.0959
问题: Sorting large numbers is not tested and does not work properly.
解决方案: Add test. Fix comparing lines with and without a number.
            (Dominique Pelle, closes #4017)
文件: src/ex_cmds.c, src/testdir/test_sort.vim

Patch 8.1.0960
问题: When using ConPTY garbage collection has undefined behavior.
解决方案: Free the channel in a better way. (Nobuhiro Takasaki, closes #4020)
文件: src/channel.c

Patch 8.1.0961 (after 8.1.0957)
问题: Mac: fsync may fail sometimes.
解决方案: Do not check errno. (Yee Cheng Chin, closes #4025)
文件: src/fileio.c

Patch 8.1.0962
问题: Building with MinGW and static libs doesn't work. (Salman Halim)
解决方案: Add -lgcc. (Ken Takata)
文件: src/Make_cyg_ming.mak

Patch 8.1.0963
问题: Illegal memory access when using 'incsearch'.
解决方案: Reset highlight_match when changing text. (closes #4022)
文件: src/testdir/test_search.vim, src/misc1.c,
            src/testdir/dumps/Test_incsearch_change_01.dump

Patch 8.1.0964
问题: Cannot see in CI why a screenshot test failed.
解决方案: Add info about the failure.
文件: src/testdir/screendump.vim

Patch 8.1.0965
问题: Search test fails.

解决方案: Wait a bit longer for the 'ambiwidth' redraw.
文件: src/testdir/test_search.vim,
           src/testdir/dumps/Test_incsearch_change_01.dump

Patch 8.1.0966
问题: One terminal test is flaky.
解决方案: Add to list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.0967
问题: Stray dependency in test Makefile.
解决方案: Remove it. (Masato Nishihata, closes #4018)
文件: src/testdir/Makefile

Patch 8.1.0968
问题: Crash when using search pattern \%Uffffffc23.
解决方案: Limit character to INT_MAX. (closes #4009)
文件: src/regexp_nfa.c, src/testdir/test_search.vim

Patch 8.1.0969
问题: Message written during startup is truncated.
解决方案: Restore message after truncating. (closes #3969) Add a test.
           (Yasuhiro Matsumoto)
文件: src/message.c, src/testdir/test_startup.vim

Patch 8.1.0970
问题: Text properties test fails when 'encoding' is not utf-8.
解决方案: Compare with original value of 'encoding'. (Christian Brabandt,
           closes #3986)
文件: src/testdir/runtest.vim, src/testdir/test_textprop.vim

Patch 8.1.0971
问题: Failure for selecting quoted text object moves cursor.
解决方案: Restore the Visual selection on failure. (Christian Brabandt,
           closes #4024)
文件: src/search.c, src/testdir/test_textobjects.vim

Patch 8.1.0972
问题: Cannot switch from terminal window to next tabpage.
解决方案: Make CTRL-W gt move to next tabpage.
文件: src/window.c, src/testdir/test_terminal.vim,
           runtime/doc/terminal.txt

Patch 8.1.0973
问题: Pattern with syntax error gives three error messages. (Kuang-che
           Wu)
解决方案: Remove outdated internal error. Don't fall back to other engine
           after an error.(closes #4035)
文件: src/regexp_nfa.c, src/testdir/test_search.vim, src/regexp.c

Patch 8.1.0974
问题: Cannot switch from terminal window to previous tabpage.
解决方案: Make CTRL-W gT move to previous tabpage.
文件: src/window.c, src/testdir/test_terminal.vim,

runtime/doc/terminal.txt

Patch 8.1.0975
问题: Using STRNCPY() wrongly. Warning for uninitialized variable.
解决方案: Use mch_memmove(). Initialize variable. (Yasuhiro Matsumoto, closes #3979)
文件: src/screen.c, src/textprop.c

Patch 8.1.0976
问题: Dosinstall still has buffer overflow problems.
解决方案: Adjust buffer sizes. (Yasuhiro Matsumoto, closes #4002)
文件: src/dosinst.c, src/dosinst.h, src/uninstal.c

Patch 8.1.0977
问题: Blob not tested with Ruby.
解决方案: Add more test coverage. Fixes a crash. (Dominique Pelle, closes #4036)
文件: src/if_ruby.c, src/testdir/test_ruby.vim

Patch 8.1.0978
问题: Blob not tested with Perl.
解决方案: Add more test coverage. Fixes a crash. (Dominique Pelle, closes #4037)
文件: src/if_perl.c, src/testdir/test_ruby.vim

Patch 8.1.0979
问题: Compiler warning for unused functions. (Yasuhiro Matsumoto)
解决方案: Adjust #ifdef.
文件: src/screen.c

Patch 8.1.0980
问题: extend() insufficiently tested.
解决方案: Add more tests. (Dominique Pelle, closes #4040)
文件: src/testdir/test_listdict.vim

Patch 8.1.0981
问题: Pasting in terminal insufficiently tested.
解决方案: Add more tests. (Dominique Pelle, closes #4040)
文件: src/testdir/test_terminal.vim

Patch 8.1.0982
问题: update_cursor() called twice in :shell.
解决方案: Remove one of the calls. (Yasuhiro Matsumoto, closes #4039)
文件: src/terminal.c

Patch 8.1.0983
问题: Checking __CYGWIN32__ unnecessarily.
解决方案: Remove the checks. (Ken Takata)
文件: src/evalfunc.c, src/os_unix.c, src/os_win32.c

Patch 8.1.0984
问题: Unnecessary #ifdefs.
解决方案: Remove the #ifdefs. (Ken Takata)
文件: src/winclip.c

Patch 8.1.0985
问题: Crash with large number in regexp. (Kuang-che Wu)
解决方案: Check for long becoming negative int. (closes #4042)
文件: src/regexp.c, src/testdir/test_search.vim

Patch 8.1.0986
问题: rename() is not properly tested.
解决方案: Add tests. (Dominique Pelle, closes #4061)
文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim,
            src/testdir/test_rename.vim

Patch 8.1.0987
问题: Unnecessary condition in #ifdef.
解决方案: Remove using CYGWIN32. (Ken Takata)
文件: src/os_unix.h, src/xxd/xxd.c

Patch 8.1.0988
问题: Deleting a location list buffer breaks location list window
            functionality.
解决方案: (Yegappan Lakshmanan, closes #4056)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.0989
问题: Various small code ugliness.
解决方案: Remove pointless NULL checks. Fix function calls. Fix typos.
            (Dominique Pelle, closes #4060)
文件: src/buffer.c, src/crypt.c, src/evalfunc.c, src/ex_cmds2.c,
            src/globals.h, src/gui_gtk_f.c, src/gui_gtk_x11.c, src/gui_mac.c,
            src/ops.c, src/option.h, src/os_unix.c, src/os_win32.c,
            src/popupmnu.c, src/regexp.c, src/ui.c, src/version.c

Patch 8.1.0990
问题: Floating point exception with "%= 0" and "/= 0".
解决方案: Avoid dividing by zero. (Dominique Pelle, closes #4058)
文件: src/eval.c, src/testdir/test_vimscript.vim

Patch 8.1.0991
问题: Cannot build with FEAT_EVAL defined and FEAT_SEARCH_EXTRA
            undefined, and with FEAT_DIFF defined and FEAT_EVAL undefined.
解决方案: Add a couple of #ifdefs. (closes #4067)
文件: src/diff.c, src/search.c

Patch 8.1.0992
问题: A :normal command while executing a register resets the
            reg_executing() result.
解决方案: Save and restore reg_executing. (closes #4066)
文件: src/ex_docmd.c, src/structs.h, src/testdir/test_functions.vim

Patch 8.1.0993
问题: ch_read() may return garbage if terminating NL is missing.
解决方案: Add terminating NUL. (Ozaki Kiichi, closes #4065)
文件: src/channel.c, src/testdir/test_channel.vim

Patch 8.1.0994
问题:　　Relative cursor position is not calculated correctly.
解决方案:　Always set topline, also when window is one line only.
　　(Robert Webb) Add more info to getwininfo() for testing.
文件:　　src/window.c, src/evalfunc.c, runtime/doc/eval.txt,
　　src/testdir/test_window_cmd.vim

Patch 8.1.0995
问题:　　A getchar() call while executing a register resets the
　　reg_executing() result.
解决方案:　Save and restore reg_executing. (closes #4066)
文件:　　src/evalfunc.c, src/testdir/test_functions.vim

Patch 8.1.0996 (after 8.1.0994)
问题:　　A few screendump tests fail because of scrolling.
解决方案:　Update the screendumps.
文件:　　src/testdir/dumps/Test_incsearch_substitute_11.dump,
　　src/testdir/dumps/Test_incsearch_substitute_12.dump,
　　src/testdir/dumps/Test_incsearch_substitute_13.dump

Patch 8.1.0997
问题:　　Using GUI colors in vim.exe when 'termguicolors' is off.
解决方案:　Add condition for 'termguicolors' set. (Ken Takata, closes #4078)
文件:　　src/os_win32.c

Patch 8.1.0998
问题:　　getcurpos() unexpectedly changes "curswant".
解决方案:　Save and restore "curswant". (closes #4069)
文件:　　src/evalfunc.c, src/testdir/test_visual.vim

Patch 8.1.0999
问题:　　Use register one too often and not properly tested.
解决方案:　Do not always use register one when specifying a register.
　　(closes #4085) Add more tests.
文件:　　src/ops.c, src/testdir/test_registers.vim

Patch 8.1.1000
问题:　　Indenting is off.
解决方案:　Make indenting consistent and update comments. (Ozaki Kiichi,
　　closes #4079)
文件:　　src/getchar.c, src/ops.c

Patch 8.1.1001
问题:　　Visual area not correct when using 'cursorline'.
解决方案:　Update w_last_cursorline also in Visual mode. (Hirohito Higashi,
　　closes #4086)
文件:　　src/screen.c, src/testdir/test_highlight.vim,
　　src/testdir/dumps/Test_cursorline_with_visualmode_01.dump

Patch 8.1.1002
问题:　　"gf" does not always work when URL has a port number. (Jakob
　　Schttl)
解决方案:　When a URL is recognized also accept ":". (closes #4082)
文件:　　src/findfile.c, src/testdir/test_gf.vim

Patch 8.1.1003
问题: Playing back recorded key sequence mistakes key code.
解决方案: Insert a <Nop> after the <Esc>. (closes #4068)
文件: src/getchar.c, src/testdir/test_registers.vim

Patch 8.1.1004
问题: Function "luaV_setref()" not covered with tests.
解决方案: Add a test. (Dominique Pelle, closes #4089)
文件: src/testdir/test_lua.vim

Patch 8.1.1005 (after 8.1.1003)
问题: Test fails because t_F2 is not set.
解决方案: Add try-catch.
文件: src/testdir/test_registers.vim

Patch 8.1.1006
问题: Repeated code in quickfix support.
解决方案: Move code to functions. (Yegappan Lakshmanan, closes #4091)
文件: src/quickfix.c

Patch 8.1.1007
问题: Using closure may consume a lot of memory.
解决方案: unreference items that are no longer needed. Add a test. (Ozaki
            Kiichi, closes #3961)
文件: src/testdir/Make_all.mak, src/testdir/test_memory_usage.vim,
            src/userfunc.c

Patch 8.1.1008
问题: MS-Windows: HAVE_STDINT_H only defined for non-debug version.
解决方案: Move definition of HAVE_STDINT_H up. (Taro Muraoka, closes #4109)
文件: src/Make_mvc.mak

Patch 8.1.1009
问题: MS-Windows: some text is not baseline aligned.
解决方案: Use bottom alignment. (Taro Muraoka, closes #4116, closes #1520)
文件: src/gui_dwrite.cpp

Patch 8.1.1010
问题: Lua interface leaks memory.
解决方案: Clear typeval after copying it.
文件: src/if_lua.c

Patch 8.1.1011
问题: Indent from autoindent not removed from blank line. (Daniel Hahler)
解决方案: Do not reset did_ai when text follows. (closes #4119)
文件: src/misc1.c, src/testdir/test_edit.vim

Patch 8.1.1012
问题: Memory leak with E461.
解决方案: Clear the typeval. (Dominique Pelle, closes #4111)
文件: src/eval.c

Patch 8.1.1013

问题: MS-Windows: Scrolling fails when dividing the screen.
解决方案: Position the cursor before calling ScrollConsoleScreenBuffer().
            (Nobuhiro Takasaki, closes #4115)
文件: src/os_win32.c

Patch 8.1.1014
问题: MS-Windows: /analyze only defined for non-debug version.
解决方案: Move adding of /analyze up. (Ken Takata, closes #4114)
文件: src/Make_mvc.mak

Patch 8.1.1015
问题: Quickfix buffer shows up in list, can't get buffer number.
解决方案: Make the quickfix buffer unlisted when the quickfix window is
            closed. get the quickfix buffer number with getqflist().
            (Yegappan Lakshmanan, closes #4113)
文件: runtime/doc/eval.txt, runtime/doc/quickfix.txt, src/quickfix.c,
            src/testdir/test_quickfix.vim, src/window.c

Patch 8.1.1016
问题: MS-Windows: No color in shell when using "!" in 'guioptions'.
解决方案: Don't stop termcap when using a terminal window for the shell.
            (Nobuhiro Takasaki, vim-jp, closes #4117)
文件: src/ex_cmds.c

Patch 8.1.1017
问题: Off-by-one error in filetype detection.
解决方案: Also check the last line of the file.
文件: runtime/autoload/dist/ft.vim

Patch 8.1.1018
问题: Window cleared when entering Terminal-Normal twice. (Epheien)
解决方案: Don't cleanup scrollback when there is no postponed scrollback.
            (Christian Brabandt, closes #4126)
文件: src/terminal.c

Patch 8.1.1019
问题: Lua: may garbage collect function reference in use.
解决方案: Keep the function name instead of the typeval. Make luaV_setref()
            handle funcref objects. (Ozaki Kiichi, closes #4127)
文件: src/if_lua.c, src/testdir/test_lua.vim

Patch 8.1.1020
问题: Compiler warning for Python3 interface.
解决方案: Add type cast. (Ozaki Kiichi, closes #4128, closes #4103)
文件: src/if_python3.c

Patch 8.1.1021
问题: pyeval() and py3eval() leak memory.
解决方案: Do not increase the reference count twice. (Ozaki Kiichi,
            closes #4129)
文件: src/if_python.c, src/if_python3.c

Patch 8.1.1022
问题: May use NULL pointer when out of memory. (Coverity)

解决方案: Check for blob_alloc() returning NULL.
文件: src/blob.c

Patch 8.1.1023
问题: May use NULL pointer when indexing a blob. (Coverity)
解决方案: Break out of loop after using index on blob
文件: src/eval.c

Patch 8.1.1024
问题: Stray log calls in terminal code. (Christian Brabandt)
解决方案: Remove the calls.
文件: src/terminal.c

Patch 8.1.1025
问题: Checking NULL pointer after addition. (Coverity)
解决方案: First check for NULL, then add the column.
文件: src/regexp.c

Patch 8.1.1026
问题: Unused condition. (Coverity)
解决方案: Remove the condition. Also remove unused #define.
文件: src/move.c

Patch 8.1.1027
问题: Memory usage test sometimes fails.
解决方案: Use 80% of before.last as the lower limit. (Christian Brabandt)
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1028
问题: MS-Windows: memory leak when creating terminal fails.
解决方案: Free the command. (Ken Takata, closes #4138)
文件: src/os_win32.c

Patch 8.1.1029
问题: DirectWrite doesn't take 'linespace' into account.
解决方案: Include 'linespace' in the position. (Ken Takata, closes #4137)
文件: src/gui_dwrite.cpp, src/gui_w32.c

Patch 8.1.1030
问题: Quickfix function arguments are inconsistent.
解决方案: Pass a list pointer instead of info and index. (Yegappan
           Lakshmanan, closes #4135)
文件: src/quickfix.c

Patch 8.1.1031
问题: Memory usage test may still fail.
解决方案: Drop the unused min value. (Christian Brabandt)
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1032
问题: Warnings from clang static analyzer. (Yegappan Lakshmanan)
解决方案: Fix relevant warnings.
文件: src/arabic.c, src/edit.c, src/eval.c, src/fileio.c, src/normal.c,
           src/option.c, src/os_unix.c, src/regexp.c, src/screen.c,

src/channel.c, src/charset.c, src/message.c

Patch 8.1.1033
问题: Memory usage test may still fail on some systems. (Elimar Riesebieter)
解决方案: Increase tolerance from 1% to 3%.
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1034
问题: Too many #ifdefs.
解决方案: Merge FEAT_MOUSE_SGR into FEAT_MOUSE_XTERM / FEAT_MOUSE_TTY.
文件: src/evalfunc.c, src/misc2.c, src/os_unix.c, src/term.c, src/version.c, src/feature.h

Patch 8.1.1035
问题: prop_remove() second argument is not optional.
解决方案: Fix argument count. Use "buf" instead of "curbuf". (closes #4147)
文件: src/evalfunc.c, src/testdir/test_textprop.vim, src/textprop.c

Patch 8.1.1036
问题: Quickfix function arguments are inconsistent.
解决方案: Pass a list pointer to more functions. (Yegappan Lakshmanan, closes #4149)
文件: src/quickfix.c

Patch 8.1.1037
问题: Memory usage test may still fail on some systems.
解决方案: Increase tolerance from 3% to 20%.
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1038
问题: Arabic support excludes Farsi.
解决方案: Add Farsi support to the Arabic support. (Ali Gholami Rudi, Ameretat Reith)
文件: Filelist, src/arabic.c, src/arabic.h, src/globals.h, src/macros.h, src/mbyte.c, src/proto/arabic.pro, src/proto/mbyte.pro, src/Makefile, src/testdir/test_arabic.vim

Patch 8.1.1039
问题: MS-Windows build fails.
解决方案: Remove dependency on arabic.h
文件: src/Make_cyg_ming.mak, src/Make_mvc.mak, src/Make_vms.mms

Patch 8.1.1040
问题: FEAT_TAG_ANYWHITE is not enabled in any build.
解决方案: Remove the feature.
文件: src/feature.h, src/tag.c, src/evalfunc.c, src/version.c, src/Make_vms.mms

Patch 8.1.1041
问题: Test for Arabic no longer needed.
解决方案: Remove the test for something that was intentionally left out.
文件: src/testdir/test_arabic.vim

Patch 8.1.1042
问题: The paste test doesn't work properly in the Windows console.
解决方案: Disable the test.
文件: src/testdir/test_paste.vim

Patch 8.1.1043
问题: Lua interface does not support Blob.
解决方案: Add support to Blob. (Ozaki Kiichi, closes #4151)
文件: runtime/doc/if_lua.txt, src/if_lua.c, src/testdir/test_lua.vim

Patch 8.1.1044
问题: No way to check the reference count of objects.
解决方案: Add test_refcount(). (Ozaki Kiichi, closes #4124)
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_vimscript.vim

Patch 8.1.1045
问题: E315 ml_get error when using Python and hidden buffer.
解决方案: Make sure the cursor position is valid. (Ben Jackson,
            closes #4153, closes #4154)
文件: src/if_py_both.h, src/testdir/test_python2.vim,
            src/testdir/test_python3.vim

Patch 8.1.1046
问题: the "secure" variable is used inconsistently. (Justin M. Keyes)
解决方案: Set it to one instead of incrementing.
文件: src/buffer.c, src/option.c

Patch 8.1.1047
问题: WINCH signal is not tested.
解决方案: Add a test. (Dominique Pelle, closes #4158)
文件: src/testdir/Make_all.mak, src/testdir/test_signals.vim

Patch 8.1.1048
问题: Minor issues with tests.
解决方案: Delete unused test OK file. Add missing entries in list of tests.
            Fix readme file. (Masato Nishihata, closes #4160)
文件: src/testdir/test85.ok, src/testdir/Make_all.mak,
            src/testdir/README.txt

Patch 8.1.1049
问题: When user tries to exit with CTRL-C message is confusing.
解决方案: Only mention ":qa!" when there is a changed buffer. (closes #4163)
文件: src/undo.c, src/proto/undo.pro, src/normal.c,
            src/testdir/test_normal.vim

Patch 8.1.1050
问题: Blank screen when DirectWrite failed.
解决方案: Call redraw_later_clear() after recreating the Direct2D render
            target. (Ken Takata, closes #4172)
文件: src/gui_dwrite.cpp

Patch 8.1.1051
问题: Not all ways to switch terminal mode are tested.

解决方案: Add more test cases.
文件: src/testdir/test_terminal.vim

Patch 8.1.1052
问题: test for CTRL-C message sometimes fails
解决方案: Make sure there are no changed buffers.
文件: src/testdir/test_normal.vim

Patch 8.1.1053
问题: Warning for missing return statement. (Dominique Pelle)
解决方案: Add return statement.
文件: src/undo.c

Patch 8.1.1054
问题: Not checking return value of ga_grow(). (Coverity)
解决方案: Only append when ga_grow() returns OK.
文件: src/if_lua.c

Patch 8.1.1055
问题: CTRL-G U in Insert mode doesn't work to avoid splitting the undo sequence for shift-left and shift-right.
解决方案: Also check dont_sync_undo for shifted cursor keys. (Christian Brabandt)
文件: src/edit.c, src/testdir/test_mapping.vim

Patch 8.1.1056
问题: No eval function for Ruby.
解决方案: Add rubyeval(). (Ozaki Kiichi, closes #4152)
文件: runtime/doc/eval.txt, runtime/doc/if_ruby.txt, src/evalfunc.c, src/if_ruby.c, src/proto/if_ruby.pro, src/testdir/test_ruby.vim

Patch 8.1.1057
问题: Nsis config is too complicated.
解决方案: Use "File /r" for the macros and pack directories. (Ken Takata, closes #4169)
文件: nsis/gvim.nsi

Patch 8.1.1058
问题: Memory usage test may still fail on some systems.
解决方案: Use 98% of the lower limit. (Christian Brabandt)
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1059
问题: MS-Windows: PlatformId() is called unnecessarily.
解决方案: Remove calls to PlatformId(). (Ken Takata, closes #4170)
文件: src/os_win32.c

Patch 8.1.1060
问题: MS-Windows: get_cmd_args() is no longer needed, get_cmd_argsW() is always used.
解决方案: Remove get_cmd_args(). (Ken Takata, closes #4171)
文件: src/gui_w32.c, src/os_w32exe.c

Patch 8.1.1061

问题: When substitute string throws error, substitute happens anyway.
解决方案: Skip substitution when aborting. (closes #4161)
文件: src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.1.1062
问题: Quickfix code is repeated.
解决方案: Define FOR_ALL_QFL_ITEMS(). Move some code to separate functions.
            (Yegappan Lakshmanan, closes #4166)
文件: src/quickfix.c

Patch 8.1.1063
问题: Insufficient testing for wildmenu completion.
解决方案: Extend the test case. (Dominique Pelle, closes #4182)
文件: src/testdir/test_cmdline.vim

Patch 8.1.1064
问题: No test for output conversion in the GTK GUI.
解决方案: Add a simplistic test.
文件: src/testdir/test_gui.vim

Patch 8.1.1065
问题: No test for using and deleting menu in the GUI.
解决方案: Add a test.
文件: src/testdir/test_gui.vim

Patch 8.1.1066
问题: VIMDLL isn't actually used.
解决方案: Remove VIMDLL support.
文件: src/gui_w32.c, src/main.c, src/os_w32exe.c, src/Make_bc5.mak,
            src/os_w32dll.c

Patch 8.1.1067
问题: Issues added on github are unstructured.
解决方案: Add a bug and feature request template. (Ken Takata, closes #4183)
文件: .github/ISSUE_TEMPLATE/feature_request.md,
            .github/ISSUE_TEMPLATE/bug_report.md

Patch 8.1.1068
问题: Cannot get all the information about current completion.
解决方案: Add complete_info(). (Shougo, Hirohito Higashi, closes #4106)
文件: runtime/doc/eval.txt, runtime/doc/insert.txt,
            runtime/doc/usr_41.txt, src/edit.c, src/evalfunc.c,
            src/proto/edit.pro, src/testdir/test_popup.vim

Patch 8.1.1069
问题: Source README file doesn't look nice on github.
解决方案: Turn it into markdown, still readable as plain text.
            (WenxuanHuang, closes #4141)
文件: src/README.txt, src/README.md, Filelist

Patch 8.1.1070
问题: Issue templates are not good enough.
解决方案: Rephrase to anticipate unexperienced users.
文件: .github/ISSUE_TEMPLATE/feature_request.md,

.github/ISSUE_TEMPLATE/bug_report.md

Patch 8.1.1071
问题: Cannot get composing characters from the screen.
解决方案: Add screenchars() and screenstring(). (partly by Ozaki Kiichi,
closes #4059)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
src/testdir/test_utf8.vim, src/testdir/view_util.vim

Patch 8.1.1072
问题: Extending sign and foldcolumn below the text is confusing.
解决方案: Let the sign and foldcolumn stop at the last text line, just like
the line number column. Also stop the command line window leader.
(Christian Brabandt, closes #3964)
文件: src/screen.c, src/testdir/test_diffmode.vim,
src/testdir/dumps/Test_diff_of_diff_01.dump,
src/testdir/dumps/Test_diff_01.dump,
src/testdir/dumps/Test_diff_02.dump,
src/testdir/dumps/Test_diff_03.dump,
src/testdir/dumps/Test_diff_04.dump,
src/testdir/dumps/Test_diff_05.dump,
src/testdir/dumps/Test_diff_06.dump,
src/testdir/dumps/Test_diff_06.0.dump,
src/testdir/dumps/Test_diff_06.1.dump,
src/testdir/dumps/Test_diff_06.2.dump,
src/testdir/dumps/Test_diff_10.dump,
src/testdir/dumps/Test_diff_11.dump,
src/testdir/dumps/Test_diff_12.dump,
src/testdir/dumps/Test_diff_13.dump,
src/testdir/dumps/Test_diff_14.dump,
src/testdir/dumps/Test_diff_15.dump,
src/testdir/dumps/Test_diff_16.dump,
src/testdir/dumps/Test_diff_17.dump,
src/testdir/dumps/Test_diff_18.dump,
src/testdir/dumps/Test_diff_19.dump,
src/testdir/dumps/Test_diff_20.dump,
src/testdir/dumps/Test_diff_with_cursorline_01.dump,
src/testdir/dumps/Test_diff_with_cursorline_02.dump,
src/testdir/dumps/Test_diff_with_cursorline_03.dump,
src/testdir/dumps/Test_folds_with_rnu_01.dump,
src/testdir/dumps/Test_folds_with_rnu_02.dump

Patch 8.1.1073
问题: Space in number column is on wrong side with 'rightleft' set.
解决方案: Move the space to the text side. Add a test.
文件: src/screen.c, src/testdir/test_diffmode.vim,
src/testdir/dumps/Test_diff_of_diff_02.dump

Patch 8.1.1074
问题: Python test doesn't wipe out hidden buffer.
解决方案: Wipe out the buffer. (Ben Jackson, closes #4189)
文件: src/testdir/test_python2.vim, src/testdir/test_python3.vim

Patch 8.1.1075

问题:	Function reference count wrong in Python code.
解决方案:	Use "O" instead of "N" for the arguments. (Ben Jackson,
closes #4188)
文件:	src/if_py_both.h

Patch 8.1.1076
问题:	File for Insert mode is much too big.
解决方案:	Split off the code for Insert completion. (Yegappan Lakshmanan,
closes #4044)
文件:	Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
src/Make_vms.mms, src/Makefile, src/edit.c, src/evalfunc.c,
src/globals.h, src/insexpand.c, src/misc2.c, src/proto.h,
src/proto/edit.pro, src/proto/insexpand.pro, src/search.c,
src/spell.c, src/structs.h, src/tag.c, src/vim.h

Patch 8.1.1077
问题:	reg_executing() is reset by calling input().
解决方案:	Implement a more generic way to save and restore reg_executing.
(Ozaki Kiichi, closes #4192)
文件:	src/evalfunc.c, src/ex_docmd.c, src/testdir/test_functions.vim

Patch 8.1.1078
问题:	When 'listchars' is set a composing char on a space is wrong.
解决方案:	Separate handling a non-breaking space and a space. (Yasuhiro
Matsumoto, closes #4046)
文件:	src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.1079
问题:	No need for a separate ScreenLinesUtf8() test function.
解决方案:	Get the composing characters with ScreenLines().
文件:	src/testdir/view_util.vim, src/testdir/test_listchars.vim,
src/testdir/test_utf8.vim

Patch 8.1.1080
问题:	When a screendump test fails, moving the file is a hassle.
解决方案:	Instead of appending ".failed" to the file name, keep the same
file name but put the screendump in the "failed" directory.
Then the file name only needs to be typed once when moving a
screendump.
文件:	src/testdir/screendump.vim

Patch 8.1.1081
问题:	MS-Windows: cannot use fonts whose name cannot be represented in
the current code page.
解决方案:	Use wide font functions. (Ken Takata, closes #4000)
文件:	src/gui_w32.c, src/os_mswin.c, src/proto/gui_w32.pro,
src/proto/os_mswin.pro

Patch 8.1.1082
问题:	"Conceal" match is mixed up with 'hlsearch' match.
解决方案:	Check that a match is found, not a 'hlsearch' item. (Andy
Massimino, closes #4073)

文件:　　src/screen.c

Patch 8.1.1083
问题:　　MS-Windows: hang when opening a file on network share.
解决方案:　Avoid using FindFirstFile(), use GetLongPathNameW(). (Ken Takata,
closes #3923)
文件:　　src/os_win32.c

Patch 8.1.1084
问题:　　Cannot delete a match from another window. (Paul Jolly)
解决方案:　Add window ID argument to matchdelete(), clearmatches(),
getmatches() and setmatches(). (Andy Massimino, closes #4178)
文件:　　runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_match.vim

Patch 8.1.1085
问题:　　Compiler warning for possibly uninitialized variable. (Tony
Mechelynck)
解决方案:　Make conditions more logical.
文件:　　src/arabic.c

Patch 8.1.1086
问题:　　Too many curly braces.
解决方案:　Remove curly braces where they are not needed. (Hirohito Higashi,
closes #3982)
文件:　　src/autocmd.c, src/buffer.c, src/crypt_zip.c, src/dosinst.c,
src/edit.c, src/insexpand.c, src/evalfunc.c, src/ex_cmds.c,
src/ex_docmd.c, src/ex_getln.c, src/getchar.c, src/gui.c,
src/gui_gtk.c, src/gui_mac.c, src/gui_motif.c, src/gui_photon.c,
src/gui_w32.c, src/gui_x11.c, src/if_mzsch.c, src/if_python3.c,
src/if_ruby.c, src/if_tcl.c, src/indent.c, src/libvterm/src/pen.c,
src/macros.h, src/memline.c, src/menu.c, src/misc1.c, src/move.c,
src/netbeans.c, src/normal.c, src/ops.c, src/option.c,
src/os_mswin.c, src/os_qnx.c, src/os_unix.c, src/os_win32.c,
src/regexp_nfa.c, src/screen.c, src/spell.c, src/terminal.c

Patch 8.1.1087
问题:　　tag stack is incorrect after CTRL-T and then :tag
解决方案:　Handle DT_TAG differently. (test by Andy Massimino, closes #3944,
closes #4177)
文件:　　src/tag.c, src/testdir/test_tagjump.vim

Patch 8.1.1088
问题:　　Height of quickfix window not retained with vertical split.
解决方案:　Use frame_fixed_height() and frame_fixed_width(). (Hongbo Liu,
closes #4013, closes #2998)
文件:　　src/testdir/test_winbuf_close.vim, src/window.c

Patch 8.1.1089
问题:　　Tutor does not check $LC_MESSAGES.
解决方案:　Let $LC_MESSAGES overrule $LANG. (Miklos Vajna, closes #4112)
文件:　　runtime/tutor/tutor.vim

Patch 8.1.1090
问题:　　MS-Windows: modify_fname() has problems with some 'encoding'.

解决方案: Use GetLongPathNameW() instead of GetLongPathName(). (Ken Takata,
           closes #4007)
文件: src/eval.c

Patch 8.1.1091
问题: MS-Windows: cannot use multi-byte chars in environment var.
解决方案: Use the wide API. (Ken Takata, closes #4008)
文件: src/misc1.c, src/testdir/test_let.vim

Patch 8.1.1092
问题: Setting 'guifont' when maximized resizes the Vim window. When
           'guioptions' contains "k" gvim may open with a tiny window.
解决方案: Avoid un-maximizing when setting 'guifont'. (Yee Cheng Chin,
           closes #3808)
文件: src/gui.c

Patch 8.1.1093
问题: Support for outdated tags format slows down tag parsing.
解决方案: Remove FEAT_TAG_OLDSTATIC.
文件: runtime/doc/tagsrch.txt, src/feature.h, src/tag.c, src/version.c

Patch 8.1.1094
问题: Long line in tags file causes error.
解决方案: Check for overlong line earlier. (Andy Massimino, closes #4051,
           closes #4084)
文件: src/tag.c, src/testdir/test_tagjump.vim

Patch 8.1.1095
问题: MS-Windows: executable() fails on very long filename.
解决方案: Use much bigger buffer. (Ken Takata, closes #4015)
文件: src/os_win32.c, src/testdir/test_functions.vim

Patch 8.1.1096
问题: MS-Windows: cannot distinguish BS and CTRL-H.
解决方案: Add code for VK_BACK. (Linwei, closes #1833)
文件: src/term.c, src/os_win32.c

Patch 8.1.1097 (after 8.1.1092)
问题: Motif build fails. (Paul Jolly)
解决方案: Only use gui_mch_maximized() for MS-Windows. (closes #4194)
文件: src/gui.c

Patch 8.1.1098
问题: Quickfix code duplication.
解决方案: Refactor the qf_init_ext() function. (Yegappan Lakshmanan,
           closes #4193)
文件: src/README.md, src/quickfix.c

Patch 8.1.1099
问题: The do_tag() function is too long.
解决方案: Factor parts out to separate functions. Move simplify_filename()
           to a file where it fits better. (Andy Massimino, closes #4195)
文件: src/tag.c, src/proto/tag.pro, src/findfile.c,
           src/proto/findfile.pro

Patch 8.1.1100
问题:　　Tag file without trailing newline no longer works. (Marco Hinz)
解决方案:　Don't expect a newline at the end of the file. (closes #4200)
文件:　　src/tag.c, src/testdir/test_taglist.vim

Patch 8.1.1101
问题:　　Signals test may fail in the GUI.
解决方案:　Skip the test for the GUI. (Yee Checng Chin, closes #4202)
文件:　　src/testdir/test_signals.vim

Patch 8.1.1102
问题:　　Win32 exe file contains unused code.
解决方案:　Remove unused #ifdefs and code. (Ken Takata, closes #4198)
文件:　　src/os_w32exe.c

Patch 8.1.1103
问题:　　MS-Windows: old API calls are no longer needed.
解决方案:　Always use the wide functions. (Ken Takata, closes #4199)
文件:　　src/glbl_ime.cpp, src/globals.h, src/gui_w32.c, src/misc1.c,
　　src/os_mswin.c, src/os_win32.c, src/vim.h,

Patch 8.1.1104
问题:　　MS-Windows: not all environment variables can be used.
解决方案:　Use the wide version of WinMain() and main(). (Ken Takata,
　　closes #4206)
文件:　　src/Make_cyg.mak, src/Make_cyg_ming.mak, src/Make_mvc.mak,
　　src/main.c, src/os_w32exe.c

Patch 8.1.1105
问题:　　Long escape sequences may be split up.
解决方案:　Assume escape sequences can be up to 80 bytes long. (Nobuhiro
　　Takasaki, closes #4196)
文件:　　src/term.c

Patch 8.1.1106
问题:　　No test for 'writedelay'.
解决方案:　Add a test.
文件:　　src/testdir/test_options.vim

Patch 8.1.1107
问题:　　No test for 'visualbell'.
解决方案:　Add a test.
文件:　　src/testdir/test_options.vim

Patch 8.1.1108
问题:　　Test for 'visualbell' doesn't work.
解决方案:　Make 'belloff' empty.
文件:　　src/testdir/test_options.vim

Patch 8.1.1109
问题:　　Deleted file still in list of distributed files.
解决方案:　Remove the src/os_w32dll.c entry.
文件:　　Filelist

Patch 8.1.1110
问题:　　Composing chars on space wrong when 'listchars' is set.
解决方案:　Do not use "space" and "nbsp" entries of 'listchars' when there is
　　a composing character.　(Yee Cheng Chin, closes #4197)
文件:　　src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.1111
问题:　　It is not easy to check for infinity.
解决方案:　Add isinf(). (Ozaki Kiichi, closes #3787)
文件:　　runtime/doc/eval.txt, src/evalfunc.c,
　　src/testdir/test_float_func.vim

Patch 8.1.1112
问题:　　Duplicate code in quickfix file.
解决方案:　Move code into functions. (Yegappan Lakshmanan, closes #4207)
文件:　　src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.1113
问题:　　Making an autocommand trigger once is not so easy.
解决方案:　Add the ++once argument.　Also add ++nested as an alias for
　　"nested". (Justin M. Keyes, closes #4100)
文件:　　runtime/doc/autocmd.txt, src/autocmd.c,
　　src/testdir/test_autocmd.vim, src/globals.h

Patch 8.1.1114
问题:　　Confusing overloaded operator "." for string concatenation.
解决方案:　Add ".." for string concatenation.　Also "let a ..= b".
文件:　　src/eval.c, src/testdir/test_eval_stuff.vim, runtime/doc/eval.txt

Patch 8.1.1115
问题:　　Cannot build with older C compiler.
解决方案:　Move variable declaration to start of block.
文件:　　src/autocmd.c

Patch 8.1.1116
问题:　　Cannot enforce a Vim script style.
解决方案:　Add the :scriptversion command. (idea by Yasuhiro Matsumoto,
　　closes #3857)
文件:　　runtime/doc/repeat.txt, runtime/doc/eval.txt, src/eval.c,
　　src/ex_cmds.h, src/evalfunc.c, src/ex_cmds2.c,
　　src/proto/ex_cmds2.pro, src/structs.h, src/buffer.c, src/main.c,
　　src/option.c, src/ex_cmdidxs.h, src/testdir/test_eval_stuff.vim

Patch 8.1.1117
问题:　　Build failure without the +eval feature.
解决方案:　Add #ifdef.
文件:　　src/ex_cmds2.c

Patch 8.1.1118
问题:　　A couple of conditions are hard to understand.
解决方案:　Split the conditions into pieces. (Ozaki Kiichi, closes #3879)
文件:　　src/getchar.c, src/os_unix.c

Patch 8.1.1119
问题: No support for Windows on ARM64.
解决方案: Add ARM64 support (Leendert van Doorn)
文件: src/GvimExt/Makefile, src/Make_mvc.mak, src/dosinst.c,
            src/xpm/arm64/lib-vc14/libXpm.lib, Filelist, src/INSTALLpc.txt

Patch 8.1.1120
问题: Cannot easily get directory entry matches.
解决方案: Add the readdir() function. (Yasuhiro Matsumoto, closes #2439)
文件: runtime/doc/eval.txt, src/eval.c, src/evalfunc.c, src/misc1.c,
            src/proto/eval.pro, src/testdir/test_functions.vim

Patch 8.1.1121
问题: Test for term_gettitle() was disabled.
解决方案: Enable the test and bail out only when it doesn't work. (Dominique
            Pelle, closes #3776)
文件: src/testdir/test_terminal.vim

Patch 8.1.1122
问题: char2nr() does not handle composing characters.
解决方案: Add str2list() and list2str(). (Ozaki Kiichi, closes #4190)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
            src/testdir/test_utf8.vim

Patch 8.1.1123
问题: No way to avoid filtering for autocomplete function, causing
            flickering of the popup menu.
解决方案: Add the "equal" field to complete items. (closes #3887)
文件: runtime/doc/insert.txt, src/insexpand.c,
            src/testdir/test_popup.vim

Patch 8.1.1124
问题: Insert completion flags are mixed up.
解决方案: Clean up flags use of ins_compl_add() and cp_flags.
文件: src/insexpand.c, src/proto/insexpand.pro, src/search.c, src/spell.c

Patch 8.1.1125
问题: Libvterm does not handle the window position report.
解决方案: Let libvterm call the fallback CSI handler when not handling CSI
            sequence. Handle the window position report in Vim.
文件: src/libvterm/src/state.c, src/terminal.c, src/ui.c,
            src/proto/ui.pro, src/evalfunc.c, src/testdir/test_terminal.vim

Patch 8.1.1126
问题: Build failure with +terminal but without tgetent.
解决方案: Adjust #ifdef.
文件: src/ui.c

Patch 8.1.1127
问题: getwinpos() doesn't work in terminal on MS-Windows console.
解决方案: Adjust #ifdefs. Disable test for MS-Windows console.
文件: src/ui.c, src/term.c, src/terminal.c,
            src/testdir/test_terminal.vim

Patch 8.1.1128
问题: getwinpos() test does not work on MS-Windows.
解决方案: Skip the test.
文件: src/testdir/test_terminal.vim

Patch 8.1.1129
问题: When making a new screendump test have to create the file.
解决方案: Continue creating the failed screendump, so it can be moved once
            it is correct.
文件: src/testdir/screendump.vim

Patch 8.1.1130
问题: MS-Windows: warning for unused variable.
解决方案: Remove the variable.
文件: src/evalfunc.c

Patch 8.1.1131
问题: getwinpos() does not work in the MS-Windows console.
解决方案: Implement getwinpos().
文件: src/ui.c, src/evalfunc.c, src/terminal.c,
            src/testdir/test_terminal.vim

Patch 8.1.1132
问题: getwinpos() test fails on MS-Windows.
解决方案: Don't try running this test.
文件: src/testdir/test_terminal.vim

Patch 8.1.1133
问题: Compiler warning for uninitialized struct member. (Yegappan
            Lakshmanan)
解决方案: Add initializer field.
文件: src/globals.h

Patch 8.1.1134
问题: Buffer for quickfix window is reused for another file.
解决方案: Don't reuse the quickfix buffer. (Yegappan Lakshmanan)
文件: src/buffer.c, src/testdir/test_quickfix.vim

Patch 8.1.1135 (after 8.1.1134)
问题: Build failure for small version. (Tony Mechelynck)
解决方案: Add #ifdef.
文件: src/buffer.c

Patch 8.1.1136
问题: Decoding of mouse click escape sequence is not tested.
解决方案: Add a test for xterm and SGR using low-level input. Make
            low-level input execution with feedkeys() work.
文件: src/testdir/test_termcodes.vim, src/testdir/Make_all.mak,
            src/evalfunc.c, src/ex_docmd.c

Patch 8.1.1137
问题: Xterm mouse wheel escape sequence is not tested.
解决方案: Add a test using low-level input. (Dominique Pelle, closes #4221)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1138
问题: Plugins don't get notified when the popup menu changes.
解决方案: Add the CompleteChanged event. (Qiming Zhao, Andy Massimino, closes #4176)
文件: runtime/doc/autocmd.txt, src/autocmd.c, src/dict.c, src/insexpand.c, src/popupmnu.c, src/proto/autocmd.pro, src/proto/dict.pro, src/proto/popupmnu.pro, src/testdir/test_popup.vim, src/vim.h

Patch 8.1.1139
问题: No test for what is fixed in patch 8.1.0716.
解决方案: Add a test. (Yasuhiro Matsumoto, closes #3797)
文件: src/testdir/test_ins_complete.vim

Patch 8.1.1140
问题: Not easy to find out what neighbors a window has.
解决方案: Add more arguments to winnr(). (Yegappan Lakshmanan, closes #3993)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/proto/window.pro, src/testdir/test_window_cmd.vim, src/window.c

Patch 8.1.1141
问题: Terminal winpos test fails with very large terminal. (Dominique Pelle)
解决方案: Compute the expected size more accurately. (closes #4228)
文件: src/testdir/test_terminal.vim

Patch 8.1.1142
问题: No test for dragging the window separators with the mouse.
解决方案: Add a test. (Dominique Pelle, closes #4226)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1143
问题: May pass weird strings to file name expansion.
解决方案: Check for matching characters. Disallow control characters.
文件: src/misc1.c, src/testdir/test_spell.vim, src/option.c, src/proto/option.pro, src/spell.c, src/testdir/test_escaped_glob.vim

Patch 8.1.1144 (after 8.1.1143)
问题: Too strict checking of the 'spellfile' option.
解决方案: Allow for a path.
文件: src/option.c, src/testdir/test_spell.vim

Patch 8.1.1145
问题: Compiler warning for unused function. (Tony Mechelynck)
解决方案: Add #ifdef.
文件: src/option.c

Patch 8.1.1146
问题: In MS-Windows console colors in a terminal window are wrong.
解决方案: Use the ansi index also for 16 colors. (Ken Takata)
文件: src/terminal.c

Patch 8.1.1147
问题:	Desktop file translations are requiring manual updates.
解决方案:	Use the .po files for desktop file translations. (Christian Brabandt)
文件:	src/po/Makefile, src/po/gvim.desktop.in, src/po/vim.desktop.in, CONTRIBUTING.md, Filelist, runtime/vim.desktop, runtime/gvim.desktop

Patch 8.1.1148
问题:	CTRL-L with 'incsearch' does not pick up char under cursor. (Smylers)
解决方案:	Do not compare the position with the cursor position. (Hirohito Higashi, closes #3620)
文件:	src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.1149
问题:	Building desktop files fails with older msgfmt.
解决方案:	Add autoconf check. Avoid always building the desktop files.
文件:	src/configure.ac, src/auto/configure, src/po/Makefile, src/po/Make_all.mak, src/config.mk.in

Patch 8.1.1150
问题:	Generating desktop files not tested on Travis.
解决方案:	Install a newer msgfmt package. (Christian Brabandt)
文件:	.travis.yml

Patch 8.1.1151
问题:	Build fails when using shadow directory.
解决方案:	Link the desktop.in files.
文件:	src/Makefile

Patch 8.1.1152
问题:	Compiler warning with VS2019.
解决方案:	Specify different offset for "AMD64". (Ken Takata, closes #4235)
文件:	src/GvimExt/Makefile

Patch 8.1.1153
问题:	Msgfmt complains about missing LINGUAS file. (Tony Mechelynck)
解决方案:	Add command to generate LINGUAS.
文件:	src/po/Makefile

Patch 8.1.1154
问题:	Getting a newer msgfmt on Travis is too complicated.
解决方案:	Use a "sourceline" entry. (Ozaki Kiichi, closes #4236)
文件:	.travis.yml

Patch 8.1.1155
问题:	Termcodes tests can be improved.
解决方案:	Add helper functions to simplify tests. Dragging statusline for xterm and sgr. (Dominique Pelle, closes #4237)
文件:	src/testdir/test_termcodes.vim

Patch 8.1.1156
问题:	Unicode emoji and other image characters not recognized.

解决方案: Add ranges for musical notation, game pieces, etc. (Martin
           Tournoij, closes #4238)
文件: src/mbyte.c

Patch 8.1.1157
问题: Unicode tables are out of date.
解决方案: Update to Unicode 12. (Christian Brabandt, closes #4240)
文件: src/mbyte.c

Patch 8.1.1158
问题: Json encoded string is sometimes missing the final NUL.
解决方案: Add the NUL. Also for log messages.
文件: src/json.c, src/channel.c, src/testdir/test_json.vim

Patch 8.1.1159
问题: MS-Windows: with a silent (un)install $VIM/_vimrc is removed.
解决方案: Don't delete _vimrc in silent mode. (Ken Takata, closes #4242)
文件: nsis/gvim.nsi

Patch 8.1.1160
问题: Termcodes test would fail in a very big terminal.
解决方案: Bail out when the row is larger than what will work. (Dominique
           Pelle, closes #4246)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1161
问题: Unreachable code.
解决方案: Remove condition that will never be true. Add tests for all ANSI
           colors.
文件: src/terminal.c, src/testdir/test_terminal.vim,
           src/testdir/dumps/Test_terminal_all_ansi_colors.dump

Patch 8.1.1162
问题: Incorrect coverage information; typo in color name.
解决方案: Fix the typo. Set environment variables to have a nested Vim
           write the coverage info in another directory.
文件: src/testdir/test_terminal.vim, src/testdir/screendump.vim,
           src/testdir/dumps/Test_terminal_all_ansi_colors.dump

Patch 8.1.1163
问题: Codecov does not report all the coverage information.
解决方案: Make a second run with the nested execution output, expect that
           Codecov will merge the results.
文件: .travis.yml

Patch 8.1.1164
问题: Gettitle test is failing when server name differs. (Kenta Sato)
解决方案: Accept "VIM1" when 'autoservername' is used. (Dominique Pelle,
           closes #4250, closes #4249)
文件: src/testdir/test_terminal.vim

Patch 8.1.1165
问题: No test for mouse clicks in the terminal tabpage line.
解决方案: Add a test. (Dominique Pelle, closes #4247). Also init

TabPageIdxs[], in case it's used before a redraw.
文件: src/screen.c, src/testdir/test_termcodes.vim

Patch 8.1.1166 (after 8.1.1164)
问题: Gettitle test can still fail when another Vim is running.
解决方案: Accept any server name number. (Dominique Pelle, closes #4252)
文件: src/testdir/test_terminal.vim

Patch 8.1.1167
问题: No test for closing tab by click in tabline.
解决方案: Add a test. Also fix that dragging window separator could fail in a large terminal. (Dominique Pelle, closes #4253)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1168
问题: Not all screen update code of the terminal window is executed in tests.
解决方案: Redraw before taking a screenshot.
文件: src/testdir/screendump.vim

Patch 8.1.1169
问题: Writing coverage info in a separate dir is not needed.
解决方案: Revert the changes to use a separate directory.
文件: .travis.yml, src/testdir/screendump.vim

Patch 8.1.1170
问题: Terminal ANSI color test does not cover all colors.
解决方案: Use the color number, the name is not always resulting in an ANSI color when t_Co is 256.
文件: src/testdir/test_terminal.vim,
src/testdir/dumps/Test_terminal_all_ansi_colors.dump

Patch 8.1.1171
问题: Statusline test could fail in large terminal.
解决方案: Make the test work on a huge terminal. (Dominique Pelle, closes #4255)
文件: src/testdir/test_statusline.vim

Patch 8.1.1172
问题: Cursor properties were not fully tested.
解决方案: Add a test. (Dominique Pelle, closes #4256)
文件: src/testdir/test_terminal.vim

Patch 8.1.1173
问题: Suspend test has duplicated lines.
解决方案: Use a function.
文件: src/testdir/test_suspend.vim

Patch 8.1.1174
问题: Cannot build with Ruby 1.8. (Tom G. Christensen)
解决方案: Include ruby/st.h. (Ozaki Kiichi, closes #4257)
文件: src/if_ruby.c

Patch 8.1.1175

问题:　　No test for dragging a tab with the mouse and for creating a new
　　tab by double clicking in the tabline.
解决方案:　Add two tests. (Dominique Pelle, closes #4258)
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.1176 (after 8.1.1175)
问题:　　Test for dragging a tab is flaky.
解决方案:　Add a brief sleep.
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.1177
问题:　　.ts files are recognized as xml, while typescript is more common.
解决方案:　Recognize .ts files as typescript. (closes #4264)
文件:　　runtime/filetype.vim src/testdir/test_filetype.vim

Patch 8.1.1178
问题:　　When mouse click tests fails value of 'ttymouse' is unknown.
解决方案:　Add a message to the assert.
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.1179
问题:　　No test for mouse clicks in the fold column.
解决方案:　Add a test. (Dominique Pelle, closes #4261)
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.1180
问题:　　Vim script debugger tests are old style.
解决方案:　Turn into new style tests. (Yegappan Lakshmanan, closes #4259)
文件:　　src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
　　src/testdir/test108.in, src/testdir/test108.ok,
　　src/testdir/test_debugger.vim

Patch 8.1.1181
问题:　　Tests for mouse clicks are a bit flaky when run in an interactive
　　terminal.
解决方案:　Use "xterm2" instead of "xterm" for 'ttymouse' to avoid spurious
　　drag events.
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.1182
问题:　　Some function prototypes are outdated.
解决方案:　Update function prototypes. (Ken Takata, closes #4267)
文件:　　src/os_mswin.c, src/proto/ex_getln.pro, src/proto/gui_w32.pro,
　　src/terminal.c, src/proto/terminal.pro, src/proto/window.pro,
　　src/window.c

Patch 8.1.1183
问题:　　Typos in VisVim comments.
解决方案:　Correct the typos. (Christ van Willegen)
文件:　　src/VisVim/Commands.cpp, src/VisVim/OleAut.cpp,
　　src/VisVim/README_VisVim.txt

Patch 8.1.1184
问题:　　Undo file left behind after running test.

解决方案: Delete the undo file. (Dominique Pelle, closes #4279)
文件: src/testdir/test_filechanged.vim

Patch 8.1.1185
问题: Mapping for CTRL-X is inconsistent.
解决方案: Map CTRL-X to "*d also for the MS-Windows console. (Ken Takata,
           closes #4265)
文件: src/getchar.c

Patch 8.1.1186
问题: readdir() allocates list twice.
解决方案: Remove second allocation. Also check for zero length.
文件: src/evalfunc.c

Patch 8.1.1187
问题: Cannot recognize Pipfile.
解决方案: Use existing filetypes. (Charles Ross, closes #4280)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.1188
问题: Not all Vim variables require the v: prefix.
解决方案: When scriptversion is 3 all Vim variables can only be used with
           the v: prefix. (Ken Takata, closes #4274)
文件: src/eval.c, src/ex_cmds2.c, src/testdir/test_eval_stuff.vim,
           runtime/doc/eval.txt

Patch 8.1.1189
问题: Mode is not cleared when leaving Insert mode.
解决方案: Clear the mode when got_int is set. (Ozaki Kiichi, closes #4270)
文件: src/edit.c, src/testdir/test_bufline.vim,
           src/testdir/test_messages.vim

Patch 8.1.1190
问题: has('vimscript-3') does not work.
解决方案: Add "vimscript-3" to the list of features. (partly by Ken Takata)
文件: src/evalfunc.c, src/testdir/test_eval_stuff.vim

Patch 8.1.1191
问题: Not all debug commands are covered by a test.
解决方案: Add more tests. (Yegappan Lakshmanan, closes #4282)
文件: src/testdir/test_debugger.vim

Patch 8.1.1192
问题: Mode is not cleared when leaving Insert mode with mapped Esc.
解决方案: Clear the mode when redraw_cmdline is set. (closes #4269)
文件: src/globals.h, src/screen.c, src/testdir/test_messages.vim

Patch 8.1.1193
问题: Typos and small problems in test files.
解决方案: Small improvements.
文件: src/testdir/test_gn.vim, src/testdir/test_quotestar.vim,
           src/testdir/test_registers.vim, src/testdir/test_syntax.vim,
           src/testdir/test_tabpage.vim, src/testdir/test_vartabs.vim

Patch 8.1.1194
问题:　　Typos and small problems in source files.
解决方案:　Small fixes.
文件:　　src/channel.c, src/crypt.c, src/edit.c, src/regexp.h, src/tag.c,
　　src/term.c, src/terminal.c, src/userfunc.c, src/installman.sh

Patch 8.1.1195
问题:　　Vim script debugger functionality needs cleanup.
解决方案:　Move debugger code to a separate file. Add more tests. (Yegappan
　　Lakshmanan, closes #4285)
文件:　　Filelist, src/Make_bc5.mak, src/Make_cyg_ming.mak,
　　src/Make_dice.mak, src/Make_ivc.mak, src/Make_manx.mak,
　　src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
　　src/Make_vms.mms, src/Makefile, src/debugger.c, src/ex_cmds2.c,
　　src/proto.h, src/proto/debugger.pro, src/proto/ex_cmds2.pro

Patch 8.1.1196
问题:　　Parallel build may fail.
解决方案:　Update dependencies.
文件:　　src/Makefile

Patch 8.1.1197
问题:　　When starting with multiple tabs file messages is confusing.
解决方案:　Set 'shortmess' when loading the other tabs. (Christian Brabandt)
文件:　　src/main.c, src/testdir/test_startup.vim,
　　src/testdir/dumps/Test_start_with_tabs.dump

Patch 8.1.1198
问题:　　Bracketed paste may remain active after Vim exists, because the
　　terminal emulator restores the setting.
解决方案:　Set/reset bracketed paste mode before setting the terminal mode.
　　(closes #3579)
文件:　　src/term.c

Patch 8.1.1199
问题:　　No test for :abclear.
解决方案:　Add a test. (Dominique Pelle, closes #4292)
文件:　　src/testdir/test_mapping.vim

Patch 8.1.1200
问题:　　Old style comments in debugger source.
解决方案:　Use new style comments. (Yegappan Lakshmanan, closes #4286)
文件:　　src/README.md, src/debugger.c

Patch 8.1.1201
问题:　　Output of :command is hard to read.
解决方案:　Make some columns wider, some narrower. Truncate the command when
　　listing all.
文件:　　src/ex_docmd.c, src/message.c, src/proto/message.pro,
　　src/getchar.c, src/menu.c

Patch 8.1.1202
问题:　　Always get regexp debugging logs when building with -DDEBUG.

解决方案: By default do not create regexp debugging logs. (Ken Takata)
文件: src/regexp.c

Patch 8.1.1203
问题: Some autocmd tests are old style.
解决方案: Turn the tests into new style. (Yegappan Lakshmanan, closes #4295)
文件: src/Makefile, src/testdir/Make_all.mak,
           src/testdir/Make_amiga.mak, src/testdir/Make_vms.mms,
           src/testdir/test11.in, src/testdir/test11.ok,
           src/testdir/test_autocmd.vim

Patch 8.1.1204
问题: Output of :command with address completion is not nice.
解决方案: Shorten the address completion names.
文件: src/ex_docmd.c, runtime/doc/map.txt

Patch 8.1.1205
问题: A BufReadPre autocommand may cause the cursor to move.
解决方案: Restore the cursor position after executing the autocommand,
           unless the autocommand moved it. (Christian Brabandt,
           closes #4302, closes #4294)
文件: src/autocmd.c, src/proto/window.pro, src/structs.h,
           src/testdir/test_autocmd.vim, src/window.c

Patch 8.1.1206
问题: User command parsing and listing not properly tested.
解决方案: Add more tests. (Dominique Pelle, closes #4296)
文件: src/testdir/test_usercommands.vim

Patch 8.1.1207
问题: Some compilers give warning messages.
解决方案: Initialize variables, change printf() argument. (Christian
           Brabandt, closes #4305)
文件: src/eval.c, src/screen.c, src/undo.c, src/window.c

Patch 8.1.1208
问题: Links to repository use wrong file name.
解决方案: Swap the file names. (Nahuel Ourthe, closes #4304)
文件: src/README.md

Patch 8.1.1209
问题: Clever compiler warns for buffer being too small.
解决方案: Make the buffer bigger (even though it's not really needed).
文件: src/evalfunc.c, src/syntax.c

Patch 8.1.1210
问题: Support for user commands is spread out. No good reason to make
           user commands optional.
解决方案: Move user command support to usercmd.c. Always enable the
           user_commands feature.
文件: src/usercmd.c, src/proto/usercmd.pro, Filelist, src/Make_bc5.mak,
           src/Make_cyg_ming.mak, src/Make_dice.mak, src/Make_ivc.mak,
           src/Make_manx.mak, src/Make_morph.mak, src/Make_mvc.mak,
           src/Make_sas.mak, src/Make_vms.mms, src/Makefile, src/README.md,

src/buffer.c, src/eval.c, src/evalfunc.c, src/ex_cmds.h,
src/ex_docmd.c, src/proto/ex_docmd.pro, src/ex_getln.c,
src/feature.h, src/macros.h, src/misc2.c, src/proto.h,
src/structs.h, src/version.c, runtime/doc/eval.txt,
runtime/doc/various.txt

Patch 8.1.1211
问题: Not all user command code is tested.
解决方案: Add more tests.
文件: src/testdir/test_usercommands.vim

Patch 8.1.1212
问题: Signal PWR is not tested.
解决方案: Test that PWR updates the swap file. (Dominique Pelle,
closes #4312)
文件: src/testdir/test_signals.vim

Patch 8.1.1213
问题: "make clean" in top dir does not cleanup indent test output.
解决方案: Clean the indent test output. Do not rely on the vim executable
for that. (closes #4307)
文件: Makefile, runtime/indent/Makefile,
runtime/indent/testdir/cleantest.vim

Patch 8.1.1214
问题: Old style tests.
解决方案: Move tests from test14 to new style test files. (Yegappan
Lakshmanan, closes #4308)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test14.in, src/testdir/test14.ok,
src/testdir/test_edit.vim, src/testdir/test_normal.vim,
src/testdir/test_search.vim, src/testdir/test_substitute.vim,
src/testdir/test_visual.vim

Patch 8.1.1215
问题: "make clean" does not remove generated src/po files.
解决方案: Remove the files for "make clean". (Christian Brabandt)
文件: src/po/Makefile

Patch 8.1.1216
问题: Mouse middle click is not tested.
解决方案: Add a test. (Dominique Pelle, closes #4310)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1217
问题: MS-Windows: no space reserved for font quality name.
解决方案: Add quality_name length if present. (Ken Takata, closes #4311)
文件: src/gui_w32.c

Patch 8.1.1218
问题: Cannot set a directory for a tab page.
解决方案: Add the tab-local directory. (Yegappan Lakshmanan, closes #4212)
文件: runtime/doc/autocmd.txt, runtime/doc/editing.txt,
runtime/doc/eval.txt, runtime/doc/index.txt,

runtime/doc/options.txt, runtime/doc/usr_22.txt,
runtime/doc/usr_41.txt, src/eval.c, src/evalfunc.c,
src/ex_cmdidxs.h, src/ex_cmds.h, src/ex_docmd.c, src/if_py_both.h,
src/proto/eval.pro, src/proto/ex_docmd.pro, src/structs.h,
src/testdir/test_getcwd.vim, src/testdir/test_mksession.vim,
src/window.c

Patch 8.1.1219
问题: Not checking for NULL return from alloc().
解决方案: Add checks. (Martin Kunev, closes #4303, closes #4174)
文件: src/beval.c, src/blowfish.c, src/crypt.c, src/crypt_zip.c,
src/ops.c, src/option.c, src/popupmnu.c, src/proto/blowfish.pro,
src/proto/crypt_zip.pro, src/gui_gtk_f.c, src/gui_gtk_x11.c,
src/libvterm/src/state.c, src/libvterm/src/termscreen.c

Patch 8.1.1220 (after 8.1.1219)
问题: Build fails on MS-Windows.
解决方案: Move declaration to start of block.
文件: src/libvterm/src/state.c

Patch 8.1.1221
问题: Filtering does not work when listing marks.
解决方案: Implement filtering marks. (Marcin Szamotulski, closes #3895)
文件: runtime/doc/various.txt, src/mark.c,
src/testdir/test_filter_cmd.vim

Patch 8.1.1222 (after 8.1.1219)
问题: Build still fails on MS-Windows.
解决方案: Move another declaration to start of block.
文件: src/libvterm/src/state.c

Patch 8.1.1223
问题: Middle mouse click test fails without a clipboard.
解决方案: Check if the clipboard can be used. (Dominique Pelle, Christian
Brabandt) Also use WorkingClipboard() instead of checking for the
"clipboard" feature.
文件: src/testdir/test_termcodes.vim, src/testdir/test_quotestar.vim

Patch 8.1.1224
问题: MS-Windows: cannot specify font weight.
解决方案: Add the "W" option to 'guifont'. (closes #4309) Move GUI font
explanation out of options.txt.
文件: runtime/doc/options.txt, runtime/doc/gui.txt,
runtime/doc/mbyte.txt, src/gui_w32.c, src/os_mswin.c

Patch 8.1.1225
问题: Cannot create a pty to use with :terminal on FreeBSD.
解决方案: Add support for posix_openpt(). (Ozaki Kiichi, closes #4306,
closes #4289)
文件: src/configure.ac, src/config.h.in, src/auto/configure, src/pty.c

Patch 8.1.1226
问题: {not in Vi} remarks get in the way of useful help text.
解决方案: Make a list of all Vi options, instead of mentioning what Vi does

not have. Update the help text for options.
文件: runtime/doc/vi_diff.txt, runtime/doc/options.txt

Patch 8.1.1227
问题: Duplicate entries in the generated .desktop files. (Ralf Schandl)
解决方案: Remove translated entries from the .in files. (closes #4313)
文件: src/po/gvim.desktop.in, src/po/vim.desktop.in

Patch 8.1.1228
问题: Not possible to process tags with a function.
解决方案: Add tagfunc() (Christian Brabandt, Andy Massimino, closes #4010)
文件: runtime/doc/options.txt, runtime/doc/tagsrch.txt,
runtime/optwin.vim, src/buffer.c, src/dict.c, src/ex_cmds.c,
src/globals.h, src/insexpand.c, src/normal.c, src/option.c,
src/option.h, src/proto/dict.pro, src/structs.h, src/tag.c,
src/testdir/Make_all.mak, src/testdir/test_alot.vim,
src/testdir/test_tagfunc.vim, src/vim.h, src/window.c

Patch 8.1.1229
问题: Warning for posix_openpt() not declared. (Tony Mechelynck)
解决方案: Add declaration.
文件: src/pty.c

Patch 8.1.1230
问题: A lot of code is shared between vim.exe and gvim.exe.
解决方案: Optionally put the shared code in vim.dll. (Ken Takata,
closes #4287)
文件: Filelist, nsis/gvim.nsi, runtime/doc/gui_w32.txt,
src/Make_cyg_ming.mak, src/Make_mvc.mak, src/channel.c,
src/evalfunc.c, src/ex_cmds.c, src/ex_docmd.c, src/feature.h,
src/fileio.c, src/getchar.c, src/globals.h, src/gui.c, src/gui.h,
src/gui_gtk_x11.c, src/gui_w32.c, src/if_mzsch.c, src/main.c,
src/mbyte.c, src/memline.c, src/message.c, src/misc2.c,
src/normal.c, src/option.c, src/os_mswin.c, src/os_w32dll.c,
src/os_w32exe.c, src/os_win32.c, src/os_win32.h,
src/proto/gui.pro, src/proto/gui_w32.pro, src/proto/misc2.pro,
src/proto/os_mswin.pro, src/proto/os_win32.pro, src/syntax.c,
src/term.c, src/terminal.c, src/ui.c, src/version.c, src/vim.rc

Patch 8.1.1231
问题: Asking about existing swap file unnecessarily.
解决方案: When it is safe, delete the swap file. Remove
HAS_SWAP_EXISTS_ACTION, it is always defined. (closes #1237)
文件: src/memline.c, src/globals.h, src/buffer.c, src/ex_cmds.c,
src/fileio.c, src/main.c, src/testdir/test_swap.vim,
runtime/doc/usr_11.txt, src/os_win32.c, src/proto/os_win32.pro,
src/os_unix.c, src/proto/os_unix.pro

Patch 8.1.1232
问题: Can't build on MS-Windows.
解决方案: Define process_still_running.
文件: src/memline.c, src/os_win32.c, src/proto/os_win32.pro,
src/os_unix.c, src/proto/os_unix.pro

Patch 8.1.1233
问题:　　Cannot build tiny version.
解决方案:　Remove #ifdef for verb_msg().
文件:　　src/message.c

Patch 8.1.1234
问题:　　Swap file test fails on MS-Windows.
解决方案:　Only compare the tail of the file names.
文件:　　src/testdir/test_swap.vim

Patch 8.1.1235
问题:　　Compiler warnings for using STRLEN() value.
解决方案:　Cast to int. (Christian Brabandt, Mike Williams)
文件:　　src/tag.c

Patch 8.1.1236
问题:　　sjiscorr.c not found in shadow directory. (Tony Mechelynck)
解决方案:　Link po/*.c files with "make shadow".
文件:　　src/Makefile

Patch 8.1.1237
问题:　　Error for using "compl", reserved word in C++.
解决方案:　Rename to "complp". (suggestion by Ken Takata)
文件:　　src/usercmd.c, src/proto/usercmd.pro

Patch 8.1.1238
问题:　　MS-Windows: compiler warning for sprintf() format.
解决方案:　Change %d to %ld. (Ken Takata)
文件:　　src/gui_w32.c

Patch 8.1.1239
问题:　　Key with byte sequence containing CSI does not work.
解决方案:　Do not recognize CSI as special unless the GUI is active. (Ken
　　Takata, closes #4318)
文件:　　src/getchar.c

Patch 8.1.1240
问题:　　Runtime desktop files are overwritten by build. (Tony Mechelynck)
解决方案:　Instead of copying the files find them with "make install".
文件:　　src/Makefile, src/po/Makefile

Patch 8.1.1241
问题:　　Ex command info contains confusing information.
解决方案:　When using the NOTADR flag use ADDR_OTHER for the address type.
　　Cleanup code using NOTADR. Check for errors in
　　create_cmdidxs.vim. Adjust Makefile to see the errors.
文件:　　src/ex_cmds.h, src/ex_docmd.c, src/Makefile,
　　src/create_cmdidxs.vim, src/usercmd.c, src/ex_cmds.c,
　　src/window.c, src/testdir/test_usercommands.vim

Patch 8.1.1242
问题:　　No cmdline redraw when tabpages have different 'cmdheight'.
解决方案:　redraw the command line when 'cmdheight' changes when switching
　　tabpages. (closes #4321)

文件: src/testdir/test_tabpage.vim, src/window.c,
            src/testdir/dumps/Test_tabpage_cmdheight.dump,
            src/testdir/screendump.vim

Patch 8.1.1243 (after 8.1.1241)
问题: Compiler warnings for incomplete switch statement. (Tony
            Mechelynck)
解决方案: Add ADDR_QUICKFIX to the list.
文件: src/ex_docmd.c

Patch 8.1.1244
问题: No tests for CTRL-mouse-click.
解决方案: Add a few tests. (Dominique Pelle, closes #4323)
文件: src/testdir/test_termcodes.vim

Patch 8.1.1245
问题: ":copen 10" sets height in full-height window. (Daniel Hahler)
解决方案: Don't set the height if the quickfix window is full height.
            (closes #4325)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.1246
问题: Cannot handle negative mouse coordinate from urxvt.
解决方案: Accept '-' where a digit is expected. (Vincent Vinel,
            closes #4326)
文件: src/term.c

Patch 8.1.1247
问题: Urxvt mouse codes are not tested.
解决方案: Also set 'ttymouse' to "urxvt" in the termcodes test.
文件: src/testdir/test_termcodes.vim

Patch 8.1.1248
问题: No test for dec mouse.
解决方案: Add some tests for dec mouse. Add "no_query_mouse".
文件: src/evalfunc.c, src/globals.h, src/os_unix.c,
            src/testdir/test_termcodes.vim, runtime/doc/eval.txt

Patch 8.1.1249
问题: Compiler warning for uninitialized variable.
解决方案: Initialize it. (Christian Brabandt)
文件: src/regexp_nfa.c

Patch 8.1.1250
问题: No test for netterm mouse.
解决方案: Add some tests for netterm mouse.
文件: src/testdir/test_termcodes.vim

Patch 8.1.1251
问题: No test for completion of mapping keys.
解决方案: Add a test. Also clean up the code.
文件: src/getchar.c, src/term.c, src/proto/term.pro,
            src/testdir/test_cmdline.vim

Patch 8.1.1252
问题: Not all mapping completion is tested.
解决方案: Add a few more mapping completion tests.
文件: src/testdir/test_cmdline.vim

Patch 8.1.1253 (after 8.1.1252)
问题: Mapping completion test fails.
解决方案: Fix expected output.
文件: src/testdir/test_cmdline.vim

Patch 8.1.1254
问题: Mapping completion contains dead code.
解决方案: Remove the code.
文件: src/term.c, src/testdir/test_cmdline.vim

Patch 8.1.1255
问题: Building desktop files fails on FreeBSD. (Adam Weinberger)
解决方案: Avoid using non-portable construct in Makefile. (closes #4332)
文件: src/po/Makefile

Patch 8.1.1256
问题: Cannot navigate through errors relative to the cursor.
解决方案: Add :cabove, :cbelow, :labove and :lbelow. (Yegappan Lakshmanan,
            closes #4316)
文件: runtime/doc/index.txt, runtime/doc/quickfix.txt, src/ex_cmdidxs.h,
            src/ex_cmds.h, src/ex_docmd.c, src/proto/quickfix.pro,
            src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.1257
问题: MSVC: name of object directory not always right.
解决方案: Adjust comment. Don't use different directory for DIRECTX. Do
            use different directory for USE_MSVCRT. (Ken Takata, closes #4333)
文件: src/Make_mvc.mak

Patch 8.1.1258
问题: The "N files to edit" message can not be suppressed.
解决方案: Suppress the message with --not-a-term. (closes #4320)
文件: src/main.c

Patch 8.1.1259
问题: Crash when exiting early. (Ralf Schandl)
解决方案: Only pop/push the title when it was set. (closes #4334)
文件: src/os_unix.c, src/misc2.c, src/usercmd.c, src/tag.c

Patch 8.1.1260
问题: Comparing with pointer instead of value.
解决方案: Add a "*". (Ken Takata, closes #4336)
文件: src/usercmd.c

Patch 8.1.1261
问题: No error for quickfix commands with negative range.
解决方案: Add ADDR_UNSIGNED and use it for quickfix commands. Make
            assert_fails() show the command if the error doesn't match.
文件: src/ex_cmds.h, src/ex_docmd.c, src/testdir/test_quickfix.vim,

runtime/doc/quickfix.txt, src/eval.c, src/quickfix.c,
src/proto/quickfix.pro, src/ex_cmds2.c

Patch 8.1.1262
问题: Cannot simulate a mouse click in a test.
解决方案: Add test_setmouse().
文件: src/evalfunc.c, runtime/doc/eval.txt, runtime/doc/usr_41.txt

Patch 8.1.1263
问题: Mouse clicks in WinBar not tested.
解决方案: Add a test for clicking on the WinBar entries.
文件: src/testdir/test_winbar.vim

Patch 8.1.1264
问题: Crash when closing window from WinBar click. (Ben Jackson)
解决方案: Check that window pointer is still valid. (closes #4337)
文件: src/menu.c

Patch 8.1.1265
问题: When GPM mouse support is enabled double clicks in xterm do not
work.
解决方案: Use KS_GPM_MOUSE for GPM mouse events.
文件: src/term.c, src/os_unix.c, src/keymap.h

Patch 8.1.1266
问题: Winbar test doesn't test enough.
解决方案: Check that the WinBar actually shows up. Correct check for clicks
with no effect. (Ben Jackson, closes #4338)
文件: src/testdir/test_winbar.vim

Patch 8.1.1267
问题: Cannot check if GPM mouse support is working.
解决方案: Add the "mouse_gpm_enable" feature.
文件: src/evalfunc.c, src/os_unix.c, src/proto/os_unix.pro,
runtime/doc/eval.txt

Patch 8.1.1268
问题: Map completion test fails in GUI.
解决方案: Skip the test that fails.
文件: src/testdir/test_cmdline.vim

Patch 8.1.1269
问题: MS-Windows GUI: multibyte chars with a 0x80 byte do not work when
compiled with VIMDLL.
解决方案: Adjust the condition for fixing the input buffer. (Ken Takata,
closes #4330)
文件: src/getchar.c

Patch 8.1.1270
问题: Cannot see current match position.
解决方案: Show "3/44" when using the "n" command and "S" is not in
'shortmess'. (Christian Brabandt, closes #4317)
文件: runtime/doc/options.txt, runtime/doc/pattern.txt, src/option.c,
src/option.h, src/search.c, src/testdir/Make_all.mak,

src/testdir/test_search_stat.vim

Patch 8.1.1271 (after 8.1.1270)
问题: Compiler warnings for use of STRNCPY(). (John Marriott)
解决方案: Use mch_memmove() instead of STRNCPY().
文件: src/search.c

Patch 8.1.1272
问题: Click on WinBar of other window not tested.
解决方案: Add a test case.
文件: src/testdir/test_winbar.vim

Patch 8.1.1273
问题: Compiler warning in direct write code.
解决方案: Add a type cast.
文件: src/gui_dwrite.cpp

Patch 8.1.1274
问题: After :unmenu can still execute the menu with :emenu.
解决方案: Do not execute a menu that was disabled for the specified mode.
文件: src/menu.c, src/testdir/test_menu.vim

Patch 8.1.1275
问题: Cannot navigate to errors before/after the cursor.
解决方案: Add the :cbefore and :cafter commands. (Yegappan Lakshmanan,
closes #4340)
文件: runtime/doc/index.txt, runtime/doc/quickfix.txt, src/ex_cmdidxs.h,
src/ex_cmds.h, src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.1276
问题: Cannot combine text properties with syntax highlighting.
解决方案: Add the "combine" field to prop_type_add(). (closes #4343)
文件: runtime/doc/eval.txt, runtime/doc/textprop.txt, src/screen.c,
src/structs.h, src/testdir/test_textprop.vim

Patch 8.1.1277 (after 8.1.1276)
问题: Missing screenshot update.
解决方案: Update the screenshot.
文件: src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.1278 (after 8.1.1276)
问题: Missing change for "combine" field.
解决方案: Also change the textprop implementation.
文件: src/textprop.c

Patch 8.1.1279
问题: Cannot set 'spellang' to "sr@latin". (Bojan Stipic)
解决方案: Allow using '@' in 'spellang'. (closes #4342)
文件: src/option.c, src/testdir/gen_opt_test.vim

Patch 8.1.1280
问题: Remarks about functionality not in Vi clutters the help.
解决方案: Move all info about what is new in Vim or already existed in Vi to
vi_diff.txt. Remove {not in Vi} remarks. (closes #4268) Add

"noet" to the help files modeline. Also include many other help
file improvements.

文件: runtime/doc/vi_diff.txt, runtime/doc/arabic.txt,
runtime/doc/autocmd.txt, runtime/doc/change.txt,
runtime/doc/channel.txt, runtime/doc/cmdline.txt,
runtime/doc/debugger.txt, runtime/doc/debug.txt,
runtime/doc/develop.txt, runtime/doc/diff.txt,
runtime/doc/digraph.txt, runtime/doc/editing.txt,
runtime/doc/eval.txt, runtime/doc/farsi.txt,
runtime/doc/filetype.txt, runtime/doc/fold.txt,
runtime/doc/ft_ada.txt, runtime/doc/ft_rust.txt,
runtime/doc/ft_sql.txt, runtime/doc/gui.txt,
runtime/doc/gui_w32.txt, runtime/doc/gui_x11.txt,
runtime/doc/hangulin.txt, runtime/doc/hebrew.txt,
runtime/doc/helphelp.txt, runtime/doc/help.txt,
runtime/doc/howto.txt, runtime/doc/if_cscop.txt,
runtime/doc/if_lua.txt, runtime/doc/if_mzsch.txt,
runtime/doc/if_ole.txt, runtime/doc/if_perl.txt,
runtime/doc/if_pyth.txt, runtime/doc/if_ruby.txt,
runtime/doc/if_sniff.txt, runtime/doc/if_tcl.txt,
runtime/doc/indent.txt, runtime/doc/index.txt,
runtime/doc/insert.txt, runtime/doc/intro.txt,
runtime/doc/map.txt, runtime/doc/mbyte.txt,
runtime/doc/message.txt, runtime/doc/mlang.txt,
runtime/doc/motion.txt, runtime/doc/netbeans.txt,
runtime/doc/options.txt, runtime/doc/os_390.txt,
runtime/doc/os_amiga.txt, runtime/doc/os_beos.txt,
runtime/doc/os_dos.txt, runtime/doc/os_mac.txt,
runtime/doc/os_mint.txt, runtime/doc/os_msdos.txt,
runtime/doc/os_os2.txt, runtime/doc/os_qnx.txt,
runtime/doc/os_risc.txt, runtime/doc/os_unix.txt,
runtime/doc/os_vms.txt, runtime/doc/os_win32.txt,
runtime/doc/pattern.txt, runtime/doc/pi_getscript.txt,
runtime/doc/pi_gzip.txt, runtime/doc/pi_logipat.txt,
runtime/doc/pi_netrw.txt, runtime/doc/pi_paren.txt,
runtime/doc/pi_spec.txt, runtime/doc/pi_tar.txt,
runtime/doc/pi_vimball.txt, runtime/doc/pi_zip.txt,
runtime/doc/print.txt, runtime/doc/quickfix.txt,
runtime/doc/quickref.txt, runtime/doc/quotes.txt,
runtime/doc/recover.txt, runtime/doc/remote.txt,
runtime/doc/repeat.txt, runtime/doc/rileft.txt,
runtime/doc/russian.txt, runtime/doc/scroll.txt,
runtime/doc/sign.txt, runtime/doc/spell.txt,
runtime/doc/sponsor.txt, runtime/doc/starting.txt,
runtime/doc/syntax.txt, runtime/doc/tabpage.txt,
runtime/doc/tagsrch.txt, runtime/doc/terminal.txt,
runtime/doc/term.txt, runtime/doc/textprop.txt,
runtime/doc/tips.txt, runtime/doc/todo.txt,
runtime/doc/uganda.txt, runtime/doc/undo.txt,
runtime/doc/usr_01.txt, runtime/doc/usr_02.txt,
runtime/doc/usr_03.txt, runtime/doc/usr_04.txt,
runtime/doc/usr_05.txt, runtime/doc/usr_06.txt,
runtime/doc/usr_07.txt, runtime/doc/usr_08.txt,
runtime/doc/usr_09.txt, runtime/doc/usr_10.txt,

runtime/doc/usr_11.txt, runtime/doc/usr_12.txt,
runtime/doc/usr_20.txt, runtime/doc/usr_21.txt,
runtime/doc/usr_22.txt, runtime/doc/usr_23.txt,
runtime/doc/usr_24.txt, runtime/doc/usr_25.txt,
runtime/doc/usr_26.txt, runtime/doc/usr_27.txt,
runtime/doc/usr_28.txt, runtime/doc/usr_29.txt,
runtime/doc/usr_30.txt, runtime/doc/usr_31.txt,
runtime/doc/usr_32.txt, runtime/doc/usr_40.txt,
runtime/doc/usr_41.txt, runtime/doc/usr_43.txt,
runtime/doc/usr_44.txt, runtime/doc/usr_45.txt,
runtime/doc/usr_90.txt, runtime/doc/usr_toc.txt,
runtime/doc/various.txt, runtime/doc/version4.txt,
runtime/doc/version5.txt, runtime/doc/version6.txt,
runtime/doc/version7.txt, runtime/doc/version8.txt,
runtime/doc/visual.txt, runtime/doc/windows.txt, runtime/doc/tags

Patch 8.1.1281
问题: Cannot specify a count with :chistory.
解决方案: Add a count to :chistory and :lhistory. (Yegappan Lakshmanan,
closes #4344)
文件: runtime/doc/quickfix.txt, src/ex_cmds.h, src/quickfix.c,
src/testdir/test_quickfix.vim

Patch 8.1.1282
问题: Running make in src/po leaves LINGUAS file behind. (Ken Takata)
解决方案: Delete LINGUAS after running msgfmt.
文件: src/po/Makefile

Patch 8.1.1283
问题: Delaying half a second after the top-bot message.
解决方案: Instead of the delay add "W" to the search count.
文件: src/search.c, src/testdir/test_search_stat.vim

Patch 8.1.1284
问题: Detecting *.tmpl as htmlcheetah is outdated.
解决方案: Use the generic name "template". (closes #4348)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.1285
问题: Test17 is old style.
解决方案: Turn into new style test. (Yegappan Lakshmanan, closes #4347)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test17.in, src/testdir/test17.ok,
src/testdir/test17a.in, src/testdir/test_checkpath.vim,
src/testdir/test_gf.vim

Patch 8.1.1286
问题: Running tests leaves XTest_tabpage_cmdheight file behind.
解决方案: Delete the right file. (closes #4350)
文件: src/testdir/test_tabpage.vim

Patch 8.1.1287
问题: Cannot build with +eval but without +mouse.
解决方案: Add #ifdefs around f_test_setmouse(). (John Marriott)

文件: src/evalfunc.c

Patch 8.1.1288
问题: Search stats don't show for mapped command.
解决方案: Remove SEARCH_PEEK from searchit flags. Add a test. (Christian
            Brabandt)
文件: src/search.c, src/testdir/test_search_stat.vim

Patch 8.1.1289
问题: May not have enough space to add "W" to search stats.
解决方案: Reserve a bit more space. (Christian Brabandt)
文件: src/search.c

Patch 8.1.1290
问题: .hgignore and .gitignore are either distributed or in git, not
            both.
解决方案: Add .gitignore to the distribution and .hgignore to git. Update
            the entries. (Christian Brabandt, Ken Takata)
文件: .gitignore, .hgignore, Filelist

Patch 8.1.1291
问题: Not easy to change directory and restore.
解决方案: Add the chdir() function. (Yegappan Lakshmanan, closes #4358)
文件: runtime/doc/eval.txt, runtime/doc/todo.txt,
            runtime/doc/usr_41.txt, src/evalfunc.c, src/ex_docmd.c,
            src/if_py_both.h, src/proto/ex_docmd.pro, src/structs.h,
            src/testdir/test_cd.vim

Patch 8.1.1292
问题: Invalid command line arguments not tested.
解决方案: Add a test. (Dominique Pelle, closes #4346)
文件: src/testdir/test_startup.vim

Patch 8.1.1293
问题: MSVC files are no longer useful for debugging. Newer Visual
            Studio versions cannot read them.
解决方案: Delete the files. (Ken Takata, closes #4357)
文件: Filelist, src/Make_dvc.mak, src/Make_ivc.mak,
            runtime/doc/debug.txt, src/INSTALLpc.txt, src/Make_mvc.mak

Patch 8.1.1294
问题: MS-Windows: Some fonts return wrong average char width.
解决方案: Compute the average ourselves. (Ken Takata, closes #4356)
文件: src/gui_w32.c

Patch 8.1.1295
问题: When vimrun.exe does not exist external command may fail.
解决方案: Use "cmd /c" twice to get the same behavior. (Ken Takata,
            closes #4355)
文件: src/os_win32.c

Patch 8.1.1296
问题: Crash when using invalid command line argument.
解决方案: Check for options not being initialized.

文件: src/term.c, src/testdir/test_startup.vim

Patch 8.1.1297
问题: Invalid argument test fails without GTK.
解决方案: Test -display and --display separately.
文件: src/testdir/test_startup.vim

Patch 8.1.1298
问题: Invalid argument test fails without X clipboard.
解决方案: Test -display only with the +xterm_clipboard feature.
文件: src/testdir/test_startup.vim

Patch 8.1.1299
问题: "extends" from 'listchars' is used when 'list' is off. (Hiroyuki Yoshinaga)
解决方案: Only use the "extends" character when 'list' is on. (Hirohito Higashi, closes #4360)
文件: src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.1300
问题: In a terminal 'balooneval' does not work right away.
解决方案: Flush output after drawing the balloon. Add the <Ignore> key code. Add a test.
文件: src/ex_cmds2.c, src/testdir/test_balloon.vim, src/misc2.c, src/testdir/Make_all.mak, src/testdir/dumps/Test_balloon_eval_term_01.dump

Patch 8.1.1301
问题: When compiled with VIMDLL some messages are not shown.
解决方案: Set/reset gui.in_use and gui.starting as needed. (Ken Takata, closes #4361)
文件: src/gui_w32.c, src/main.c, src/message.c

Patch 8.1.1302
问题: v:beval_text is not tested in Visual mode.
解决方案: Add a screenshot of the balloon in Visual mode.
文件: src/testdir/test_balloon.vim, src/normal.c, src/testdir/dumps/Test_balloon_eval_term_01.dump, src/testdir/dumps/Test_balloon_eval_term_02.dump

Patch 8.1.1303
问题: Not possible to hide a balloon.
解决方案: Hide the balloon when balloon_show() is called with an empty string or list. Add balloon_gettext().
文件: src/evalfunc.c, src/popupmnu.c, src/gui_beval.c, src/gui_w32.c, src/beval.h, src/testdir/test_balloon.vim, runtime/doc/eval.txt

Patch 8.1.1304
问题: MS-Windows: compiler warning for unused value.
解决方案: Adjust #ifdefs. (Ken Takata, closes #4363)
文件: src/gui.c

Patch 8.1.1305
问题: There is no easy way to manipulate environment variables.

解决方案: Add environ(), getenv() and setenv(). (Yasuhiro Matsumoto,
           closes #2875)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
           src/testdir/Make_all.mak, src/testdir/test_environ.vim

Patch 8.1.1306
问题: Borland support is outdated and doesn't work.
解决方案: Remove Borland support, there are other (free) compilers
           available. (Thomas Dziedzic, Ken Takata, closes #4364)
文件: .gitignore, .hgignore, Filelist, runtime/doc/debug.txt,
           runtime/doc/develop.txt, runtime/doc/usr_90.txt,
           src/GvimExt/Make_bc5.mak, src/GvimExt/gvimext.cpp,
           src/GvimExt/gvimext.rc, src/INSTALLpc.txt, src/Make_bc5.mak,
           src/dosinst.c, src/dosinst.h, src/evalfunc.c, src/ex_cmds.c,
           src/ex_getln.c, src/gui_w32.c, src/if_ole.cpp, src/if_py_both.h,
           src/main.c, src/mark.c, src/message.c, src/misc1.c, src/misc2.c,
           src/normal.c, src/option.c, src/os_mswin.c, src/os_w32exe.c,
           src/os_win32.c, src/os_win32.h, src/proto.h, src/screen.c,
           src/spell.c, src/spellfile.c, src/syntax.c, src/userfunc.c,
           src/vim.h, src/vim.rc, src/vimrun.c, src/xxd/Make_bc5.mak,
           src/xxd/xxd.c

Patch 8.1.1307
问题: Cannot reconnect to the X server after it restarted.
解决方案: Add the :xrestore command. (Adrian Kocis, closes #844)
文件: runtime/doc/index.txt, runtime/doc/various.txt, src/os_unix.c,
           src/proto/os_unix.pro, src/globals.h, src/ex_cmds.h,
           src/ex_cmdidxs.h, src/ex_docmd.c, src/testdir/test_paste.vim

Patch 8.1.1308
问题: The Normal highlight is not defined when compiled with GUI.
解决方案: Always define Normal. (Christian Brabandt, closes #4072)
文件: runtime/doc/syntax.txt, src/syntax.c,
           src/testdir/test_highlight.vim

Patch 8.1.1309 (after 8.1.1308)
问题: Test for Normal highlight fails on MS-Windows GUI.
解决方案: Skip the test for MS-Windows GUI.
文件: src/testdir/test_highlight.vim

Patch 8.1.1310
问题: Named function arguments are never optional.
解决方案: Support optional function arguments with a default value. (Andy
           Massimino, closes #3952)
文件: runtime/doc/eval.txt, src/structs.h,
           src/testdir/test_user_func.vim, src/userfunc.c

Patch 8.1.1311
问题: Aborting an autocmd with an exception is not tested.
解决方案: Add a test. Also shows how to abort a command by throwing an
           exception.
文件: src/testdir/test_autocmd.vim

Patch 8.1.1312

问题: Coverity warning for using uninitialized variable.
解决方案: Clear exarg_T.
文件: src/quickfix.c, src/channel.c, src/ex_cmds2.c

Patch 8.1.1313
问题: Warnings for using localtime() and ctime().
解决方案: Use localtime_r() if available. Avoid using ctime().
文件: src/configure.ac, src/auto/configure, src/config.h.in,
            src/evalfunc.c, src/nbdebug.c, src/undo.c, src/memline.c,
            src/proto/memline.pro, src/hardcopy.c

Patch 8.1.1314
问题: MSVC makefile is not nicely indented.
解决方案: Adjust spaces in preprocessor directives. (Ken Takata)
文件: src/Make_mvc.mak

Patch 8.1.1315
问题: There is always a delay if a termrequest is never answered.
解决方案: When the response is not received within two seconds consider the
            request to have failed.
文件: src/term.c

Patch 8.1.1316
问题: Duplicated localtime() call.
解决方案: Delete one.
文件: src/undo.c

Patch 8.1.1317
问题: Output from Travis can be improved.
解决方案: Add section headers. Handle errors better. (Ozaki Kiichi,
            closes #4098)
文件: .travis.yml, configure

Patch 8.1.1318
问题: Code for text changes is in a "misc" file.
解决方案: Move the code to change.c.
文件: src/misc1.c, src/proto/misc1.pro, src/change.c,
            src/proto/change.pro, src/proto.h, src/memline.c, Filelist,
            src/Make_cyg_ming.mak, src/Make_dice.mak, src/Make_manx.mak,
            src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak,
            src/Make_vms.mms, src/Makefile, src/README.md

Patch 8.1.1319
问题: Computing function length name in many places.
解决方案: compute name length in call_func().
文件: src/eval.c, src/userfunc.c, src/channel.c, src/evalfunc.c,
            src/ex_cmds2.c, src/regexp.c, src/terminal.c

Patch 8.1.1320
问题: It is not possible to track changes to a buffer.
解决方案: Add listener_add() and listener_remove(). No docs or tests yet.
文件: src/structs.h, src/change.c, src/proto/change.pro

Patch 8.1.1321

问题:	No docs or tests for listener functions.
解决方案:	Add help and tests for listener_add() and listener_remove().
Invoke the callbacks before redrawing.
文件:	runtime/doc/eval.txt, runtime/doc/usr_41.txt,
src/testdir/test_listener.vim, src/testdir/Make_all.mak,
src/change.c, src/screen.c, src/evalfunc.c, src/proto/evalfunc.pro

Patch 8.1.1322
问题:	Cygwin makefile is not nicely indented.
解决方案:	Adjust spaces in preprocessor directives. (Ken Takata)
文件:	src/Make_cyg_ming.mak

Patch 8.1.1323
问题:	'mouse' option is reset when using GPM mouse.
解决方案:	Add flag for GPM mouse.
文件:	src/term.c

Patch 8.1.1324
问题:	Stray comma in VMS makefile.
解决方案:	Remove the comma. (Naruhiko Nishino, closes #4368)
文件:	src/Make_vms.mms

Patch 8.1.1325
问题:	Cannot build with +eval but without +channel and +timers. (John
Marriott)
解决方案:	Adjust #ifdef for get_callback().
文件:	src/evalfunc.c, src/testdir/test_autocmd.vim

Patch 8.1.1326
问题:	No test for listener with partial.
解决方案:	Add a test. Add example to help.
文件:	src/testdir/test_listener.vim, runtime/doc/eval.txt

Patch 8.1.1327
问题:	Unnecessary scroll after horizontal split.
解决方案:	Don't adjust to fraction if all the text fits in the window.
(Martin Kunev, closes #4367)
文件:	src/testdir/test_window_cmd.vim, src/window.c

Patch 8.1.1328
问题:	No test for listener with undo operation.
解决方案:	Add a test.
文件:	src/testdir/test_listener.vim

Patch 8.1.1329
问题:	Plans for popup window support are spread out.
解决方案:	Add a first version of the popup window help.
文件:	runtime/doc/popup.txt, runtime/doc/Makefile, runtime/doc/help.txt

Patch 8.1.1330
问题:	Using bold attribute in terminal changes the color. (Jason
Franklin)
解决方案:	Don't set the "bold-highbright" flag in vterm unless the terminal
supports less than 16 colors.

文件:　　src/terminal.c, src/testdir/test_terminal.vim,
　　src/testdir/dumps/Test_terminal_all_ansi_colors.dump

Patch 8.1.1331
问题:　　Test 29 is old style.
解决方案:　Turn it into a new style test. (Yegappan Lakshmanan, closes #4370)
文件:　　src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
　　src/testdir/test29.in, src/testdir/test29.ok,
　　src/testdir/test_backspace_opt.vim, src/testdir/test_join.vim

Patch 8.1.1332
问题:　　Cannot flush change listeners without also redrawing. The line
　　numbers in the list of changes may become invalid.
解决方案:　Add listener_flush(). Invoke listeners before adding a change
　　that makes line numbers invalid.
文件:　　src/evalfunc.c, src/change.c, src/proto/change.pro,
　　src/screen.c, runtime/doc/eval.txt, src/testdir/test_listener.vim

Patch 8.1.1333
问题:　　Text properties don't always move after changes.
解决方案:　Update properties before reporting changes to listeners. Move text
　　property when splitting a line.
文件:　　src/change.c, src/ex_cmds.c, src/textprop.c,
　　src/proto/textprop.pro, src/testdir/test_textprop.vim

Patch 8.1.1334
问题:　　When buffer is hidden "F" in 'shortmess' is not used.
解决方案:　Check the "F" flag in 'shortmess' when the buffer is already
　　loaded. (Jason Franklin) Add test_getvalue() to be able to test
　　this.
文件:　　src/buffer.c, src/evalfunc.c, src/testdir/test_options.vim,
　　runtime/doc/eval.txt

Patch 8.1.1335
问题:　　Listener callback is called after inserting text.
解决方案:　Flush the changes before inserting or deleting a line. Store
　　changes per buffer.
文件:　　src/change.c, src/proto/change.pro, src/memline.c,
　　src/structs.h, src/testdir/test_listener.vim

Patch 8.1.1336
问题:　　Some eval functionality is not covered by tests.
解决方案:　Add a few more test cases. (Masato Nishihata, closes #4374)
文件:　　src/testdir/test_bufline.vim, src/testdir/test_cindent.vim,
　　src/testdir/test_cursor_func.vim, src/testdir/test_delete.vim,
　　src/testdir/test_expand_func.vim, src/testdir/test_float_func.vim,
　　src/testdir/test_fnamemodify.vim, src/testdir/test_functions.vim

Patch 8.1.1337
问题:　　Get empty text prop when splitting line just after text prop.
解决方案:　Do not create an empty text prop at the start of the line.
文件:　　src/textprop.c, src/testdir/test_textprop.vim

Patch 8.1.1338

问题:	Hang when concealing the '>' shown for a wide char that doesn't
	fit in the last cell.
解决方案:	Put back the pointer when the '>' is not going to be displayed.
	(closes #4377)
文件:	src/screen.c

Patch 8.1.1339
问题:	Installer needs to product name et al.
解决方案:	Add a few lines to the NSIS installer script. (Ken Takata)
文件:	nsis/gvim.nsi

Patch 8.1.1340
问题:	Attributes from 'cursorline' overwrite textprop.
解决方案:	Combine the attributes. (closes #3912)
文件:	src/screen.c, src/textprop.c, src/testdir/test_textprop.vim,
	src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.1341
问题:	Text properties are lost when joining lines.
解决方案:	Move the text properties to the joined line.
文件:	src/ops.c, src/textprop.c, src/proto/textprop.pro,
	src/testdir/test_textprop.vim,
	src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.1342
问题:	Using freed memory when joining line with text property.
解决方案:	Use already computed length.
文件:	src/ops.c

Patch 8.1.1343
问题:	Text properties not adjusted for Visual block mode delete.
解决方案:	Call adjust_prop_columns(). (closes #4384)
文件:	src/ops.c, src/textprop.c, src/testdir/test_textprop.vim,
	src/misc1.c, src/testdir/dumps/Test_textprop_vis_01.dump,
	src/testdir/dumps/Test_textprop_vis_02.dump

Patch 8.1.1344
问题:	Coverity complains about possibly using a NULL pointer and copying
	a string into a fixed size buffer.
解决方案:	Check for NULL, even though it should not happen. Use
	vim_strncpy() instead of strcpy().
文件:	src/change.c, src/memline.c

Patch 8.1.1345
问题:	Stuck in sandbox with ":s/../\=Function/gn".
解决方案:	Don't skip over code to restore sandbox. (Christian Brabandt)
文件:	src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.1.1346
问题:	Error for Python exception does not show useful info.
解决方案:	Show the last line instead of the first one. (Ben Jackson,
	closes #4381)
文件:	src/if_py_both.h, src/testdir/test86.ok, src/testdir/test87.ok,
	src/testdir/test_python2.vim, src/testdir/test_python3.vim,

src/testdir/test_pyx2.vim, src/testdir/test_pyx3.vim

Patch 8.1.1347 (after 8.1.1327)
问题: Fractional scroll position not restored after closing window.
解决方案: Do restore fraction if topline is not one.
文件: src/window.c, src/testdir/test_window_cmd.vim

Patch 8.1.1348
问题: Running tests may cause the window to move.
解决方案: Correct the reported window position for the offset with the position after ":winpos". Works around an xterm bug.
文件: src/testdir/test_edit.vim

Patch 8.1.1349
问题: If writing runs into a conversion error the backup file is deleted. (Arseny Nasokin)
解决方案: Don't delete the backup file is the file was overwritten and a conversion error occurred. (Christian Brabandt, closes #4387)
文件: src/fileio.c, src/testdir/test_writefile.vim

Patch 8.1.1350
问题: "W" for wrapping not shown when more than 99 matches.
解决方案: Adjust check for length. (Masato Nishihata, closes #4388)
文件: src/search.c, src/testdir/test_search_stat.vim

Patch 8.1.1351
问题: Text property wrong after :substitute.
解决方案: Save for undo before changing any text properties.
文件: src/testdir/test_textprop.vim, src/ex_cmds.c, src/textprop.c, src/proto/textprop.pro, src/change.c, src/edit.c, src/misc1.c, src/ops.c

Patch 8.1.1352
问题: Undofile() reports wrong name. (Francisco Giordano)
解决方案: Clean up the name before changing path separators. (closes #4392, closes #4394)
文件: src/evalfunc.c, src/testdir/test_undo.vim

Patch 8.1.1353 (after 8.1.1352)
问题: Undo test fails on Mac.
解决方案: Expect "private" on the Mac.
文件: src/testdir/test_undo.vim

Patch 8.1.1354
问题: Getting a list of text lines is clumsy.
解决方案: Add the =<< assignment. (Yegappan Lakshmanan, closes #4386)
文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_let.vim

Patch 8.1.1355
问题: Obvious mistakes are accepted as valid expressions.
解决方案: Be more strict about parsing numbers. (Yasuhiro Matsumoto, closes #3981)
文件: src/charset.c, src/eval.c, src/evalfunc.c, src/ex_cmds.c, src/ex_getln.c, src/json.c, src/misc2.c, src/ops.c, src/option.c,

src/proto/charset.pro, src/testdir/test_expr.vim,
src/testdir/test_json.vim

Patch 8.1.1356
问题: Some text in heredoc assignment ends the text. (Ozaki Kiichi)
解决方案: Recognize "let v =<<" and skip until the end.
文件: src/userfunc.c, src/testdir/test_let.vim

Patch 8.1.1357
问题: Test 37 is old style.
解决方案: Turn it into a new style test. (Yegappan Lakshmanan, closes #4398)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test37.in, src/testdir/test37.ok,
src/testdir/test_scrollbind.vim

Patch 8.1.1358
问题: Cannot enter character with a CSI byte.
解决方案: Only check "gui.in_use" when VIMDLL is defined. (Ken Takata,
closes #4396)
文件: src/getchar.c

Patch 8.1.1359
问题: Text property wrong after :substitute with backslash.
解决方案: Adjust text property columns when removing backslashes.
(closes #4397)
文件: src/ex_cmds.c, src/testdir/test_textprop.vim, src/vim.h,
src/textprop.c, src/proto/textprop.pro, src/change.c, src/edit.c,
src/misc1.c, src/ops.c

Patch 8.1.1360 (after Patch 8.1.1345)
问题: Buffer left 'nomodifiable' after :substitute. (Ingo Karkat)
解决方案: Save the value of 'modifiable' earlier. (Christian Brabandt,
closes #4403)
文件: src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.1.1361
问题: Python setuptools don't work with Python 3.
解决方案: Add dummy implementation for find_module. (Joel Frederico,
closes #4402, closes #3984)
文件: src/if_py_both.h

Patch 8.1.1362
问题: Code and data in tests can be hard to read.
解决方案: Use the new heredoc style. (Yegappan Lakshmanan, closes #4400)
文件: src/testdir/test_autocmd.vim, src/testdir/test_balloon.vim,
src/testdir/test_bufline.vim, src/testdir/test_cindent.vim,
src/testdir/test_conceal.vim, src/testdir/test_exit.vim,
src/testdir/test_fold.vim, src/testdir/test_goto.vim,
src/testdir/test_join.vim, src/testdir/test_mksession_utf8.vim,
src/testdir/test_normal.vim, src/testdir/test_profile.vim,
src/testdir/test_quickfix.vim, src/testdir/test_startup.vim,
src/testdir/test_terminal.vim, src/testdir/test_xxd.vim

Patch 8.1.1363

问题:　　":vert options" does not make a vertical split.
解决方案:　Pass the right modifiers in $OPTWIN_CMD. (Ken Takata,
　　closes #4401)
文件:　　src/ex_cmds2.c, src/testdir/test_options.vim

Patch 8.1.1364
问题:　　Design for popup window support needs more details.
解决方案:　Add details about using a window and buffer. Rename popup_show()
　　to popup_create() and add popup_show() and popup_hide().
文件:　　runtime/doc/popup.txt

Patch 8.1.1365
问题:　　Source command doesn't check for the sandbox. (Armin Razmjou)
解决方案:　Check for the sandbox when sourcing a file.
文件:　　src/getchar.c, src/testdir/test_source.vim

Patch 8.1.1366
问题:　　Using expressions in a modeline is unsafe.
解决方案:　Disallow using expressions in a modeline, unless the
　　'modelineexpr' option is set. Update help, add more tests.
文件:　　runtime/doc/options.txt, src/option.c, src/option.h,
　　src/testdir/test_modeline.vim, src/testdir/test49.in

Patch 8.1.1367 (after 8.1.1366)
问题:　　can set 'modelineexpr' in modeline.
解决方案:　Add P_SECURE flag.
文件:　　src/option.c, src/testdir/test_modeline.vim

Patch 8.1.1368 (after 8.1.1366)
问题:　　Modeline test fails with python but without pythonhome.
解决方案:　Correct test argument.
文件:　　src/testdir/test_modeline.vim

Patch 8.1.1369
问题:　　Get E484 when using system() during GUI startup.
解决方案:　Check "gui.starting". (Ken Takata)
文件:　　src/os_win32.c

Patch 8.1.1370
问题:　　Not using the new github feature for donations.
解决方案:　Add a Sponsor button. (closes #4417)
文件:　　.github/FUNDING.yml

Patch 8.1.1371
问题:　　Cannot recover from a swap file.
解决方案:　Do not expand environment variables in the swap file name.
　　Do not check the extension when we already know a file is a swap
　　file. (Ken Takata, closes #4415, closes #4369)
文件:　　src/buffer.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c,
　　src/gui.c, src/if_cscope.c, src/main.c, src/memline.c,
　　src/misc1.c, src/proto/memline.pro, src/proto/misc1.pro,
　　src/search.c, src/spell.c, src/spellfile.c, src/tag.c,
　　src/testdir/test_swap.vim, src/vim.h

Patch 8.1.1372
问题:　　When evaluating 'statusline' the current window is unknown.
　　(Daniel Hahler)
解决方案:　Set "g:actual_curwin" for %{} items. Set "g:statusline_winid"
　　when evaluating %!. (closes #4406, closes #3299)
文件:　　src/buffer.c, runtime/doc/options.txt,
　　src/testdir/test_statusline.vim

Patch 8.1.1373
问题:　　"[p" in Visual mode puts in wrong line.
解决方案:　Call nv_put() instead of duplicating the functionality.
　　(closes #4408)
文件:　　src/normal.c, src/testdir/test_put.vim

Patch 8.1.1374
问题:　　Check for file changed triggers too often.
解决方案:　Don't use "b_p_ar" when it is negative.
文件:　　src/fileio.c

Patch 8.1.1375
问题:　　Without "TS" in 'shortmess' get a hit-enter prompt often.
解决方案:　Always truncate the search message. Also avoid putting it in the
　　message history. (closes #4413)
文件:　　src/search.c, src/main.c, src/testdir/test_search_stat.vim

Patch 8.1.1376
问题:　　Warnings for size_t/int mixups.
解决方案:　Change types, add type casts. (Mike Williams)
文件:　　src/search.c, src/textprop.c

Patch 8.1.1377
问题:　　MS-Windows GUI uses wrong shell command for bash. (Robert Bogomip)
解决方案:　Check that 'shellcmdflag' is "/c". (Ken Takata, closes #4418)
文件:　　src/os_win32.c

Patch 8.1.1378
问题:　　Delete() can not handle a file name that looks like a pattern.
解决方案:　Use readdir() instead of appending "/*" and expanding wildcards.
　　(Ken Takata, closes #4424, closes #696)
文件:　　src/testdir/test_functions.vim, src/evalfunc.c, src/fileio.c,
　　src/proto/fileio.pro

Patch 8.1.1379 (after 8.1.1374)
问题:　　Filechanged test hangs.
解决方案:　Do not check 'autoread'.
文件:　　src/fileio.c, src/testdir/test_filechanged.vim

Patch 8.1.1380
问题:　　MS-Windows building VIMDLL with MSVC: SUBSYSTEM is not set.
解决方案:　Invert condition. (Ken Takata, closes #4422)
文件:　　src/Make_mvc.mak

Patch 8.1.1381
问题:　　MS-Windows: missing build dependency.

解决方案: Make gui_dwrite.cpp depend on gui_dwrite.h. (Ken Takata,
           closes #4423)
文件: src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.1382
问题: Error when editing test file.
解决方案: Remove part of modeline.
文件: src/testdir/test_vimscript.vim, src/testdir/test49.vim,
           src/testdir/test49.in

Patch 8.1.1383
问题: Warning for size_t/int mixup.
解决方案: Change type. (Mike Williams)
文件: src/search.c

Patch 8.1.1384
问题: Using "int" for alloc() often results in compiler warnings.
解决方案: Use "size_t" and remove type casts. Remove alloc_check(), Vim
           only works with 32 bit ints anyway.
文件: src/misc2.c, src/proto/misc2.pro, src/change.c, src/ex_cmds.c,
           src/netbeans.c, src/autocmd.c, src/buffer.c, src/change.c,
           src/channel.c, src/charset.c, src/debugger.c, src/dict.c,
           src/diff.c, src/digraph.c, src/edit.c, src/eval.c, src/evalfunc.c,
           src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c,
           src/ex_getln.c, src/fileio.c, src/findfile.c, src/fold.c,
           src/getchar.c, src/gui.c, src/gui_at_fs.c, src/gui_gtk.c,
           src/gui_gtk_x11.c, src/gui_motif.c, src/gui_w32.c, src/hashtab.c,
           src/if_cscope.c, src/if_perlsfio.c, src/if_python3.c,
           src/if_xcmdsrv.c, src/indent.c, src/insexpand.c, src/main.c,
           src/mbyte.c, src/memfile.c, src/memline.c, src/menu.c,
           src/message.c, src/misc1.c, src/misc2.c, src/netbeans.c,
           src/ops.c, src/option.c, src/os_amiga.c, src/os_mswin.c,
           src/os_unix.c, src/os_vms.c, src/os_win32.c, src/quickfix.c,
           src/regexp.c, src/screen.c, src/spell.c, src/spellfile.c,
           src/syntax.c, src/term.c, src/undo.c, src/usercmd.c,
           src/userfunc.c, src/version.c, src/winclip.c

Patch 8.1.1385
问题: Signed/unsigned compiler warning.
解决方案: Use STRLEN() instead of strlen().
文件: src/fileio.c

Patch 8.1.1386
问题: Unnecessary type casts for lalloc().
解决方案: Remove type casts. Change lalloc(size, TRUE) to alloc(size).
文件: src/buffer.c, src/change.c, src/channel.c, src/diff.c, src/edit.c,
           src/eval.c, src/ex_cmds.c, src/ex_getln.c, src/fileio.c,
           src/getchar.c, src/gui_mac.c, src/insexpand.c, src/gui_w32.c,
           src/gui_x11.c, src/menu.c, src/netbeans.c, src/ops.c,
           src/os_mswin.c, src/os_amiga.c, src/os_qnx.c, src/os_unix.c,
           src/os_win32.c, src/popupmnu.c, src/quickfix.c, src/regexp.c,
           src/regexp_nfa.c, src/screen.c, src/search.c, src/sign.c,
           src/spell.c, src/spellfile.c, src/syntax.c, src/tag.c,
           src/terminal.c, src/textprop.c, src/ui.c, src/undo.c,

src/userfunc.c, src/winclip.c, src/window.c

Patch 8.1.1387
问题: Calling prop_add() in an empty buffer doesn't work. (Dominique Pelle)
解决方案: Open the memline before adding a text property. (closes #4412)
文件: src/textprop.c, src/testdir/test_textprop.vim

Patch 8.1.1388
问题: Errors when calling prop_remove() for an unloaded buffer.
解决方案: Bail out when the buffer is not loaded. Add a few more tests for failing when the buffer number is invalid.
文件: src/textprop.c, src/testdir/test_textprop.vim

Patch 8.1.1389
问题: Changes are not flushed when end and start overlap. (Paul Jolly)
解决方案: When end of a previous changes overlaps with start of a new change, first flush listeners.
文件: src/change.c, src/testdir/test_listener.vim

Patch 8.1.1390
问题: Search stats are off when using count or offset.
解决方案: Recompute the stats when needed. (Masato Nishihata, closes #4410)
文件: src/testdir/test_search_stat.vim, src/search.c

Patch 8.1.1391
问题: No popup window support.
解决方案: Add initial code for popup windows. Add the 'wincolor' option.
文件: Filelist, runtime/doc/popup.txt, runtime/doc/options.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/autocmd.c, src/buffer.c, src/ex_cmds.h, src/ex_cmdidxs.h, src/proto/buffer.pro, src/eval.c src/evalfunc.c src/feature.h, src/globals.h, src/option.c, src/option.h, src/popupwin.c, src/proto.h, src/proto/popupwin.pro, src/proto/window.pro, src/screen.c, src/structs.h, src/terminal.c, src/testdir/Make_all.mak, src/testdir/dumps/Test_popupwin_01.dump, src/testdir/test_popupwin.vim, src/vim.h, src/window.c

Patch 8.1.1392 (after 8.1.1391)
问题: Build failure in tiny version.
解决方案: Define ex_popupclear to ex_ni if not implemented. Add UNUSED.
文件: src/ex_docmd.c, src/window.c

Patch 8.1.1393
问题: Unnecessary type casts.
解决方案: Remove type casts from alloc() and lalloc() calls. (Mike Williams)
文件: src/channel.c, src/crypt.c, src/dict.c, src/dosinst.c, src/evalfunc.c, src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c, src/findfile.c, src/if_ole.cpp, src/if_py_both.h, src/list.c, src/message.c, src/misc1.c, src/misc2.c, src/ops.c, src/os_vms.c, src/os_win32.c, src/quickfix.c, src/regexp_nfa.c, src/screen.c, src/search.c, src/sign.c, src/syntax.c, src/tag.c, src/term.c, src/terminal.c, src/textprop.c

Patch 8.1.1394
问题:　　Not restoring t_F2 in registers test.
解决方案:　Assign to &t_F2 instead of t_F2. (Andy Massimino, closes #4434)
文件:　　src/testdir/test_registers.vim

Patch 8.1.1395
问题:　　Saving for undo may access invalid memory. (Dominique Pelle)
解决方案:　Set ml_line_len also when returning a constant string.
文件:　　src/memline.c, src/testdir/test_textprop.vim

Patch 8.1.1396
问题:　　'wincolor' does not apply to lines below the buffer.
解决方案:　Also apply 'wincolor' to the "~" lines and the number column.
文件:　　src/screen.c, src/testdir/test_highlight.vim,
　　src/testdir/dumps/Test_wincolor_01.dump

Patch 8.1.1397
问题:　　Build fails in tiny version.
解决方案:　Always define hl_combine_attr().
文件:　　src/syntax.c

Patch 8.1.1398
问题:　　Duplicate line in MSVC build file.
解决方案:　Remove the line. (Ken Takata, closes #4436)
文件:　　src/Make_mvc.mak

Patch 8.1.1399
问题:　　Popup windows not adjusted when switching tabs.
解决方案:　Save and restore first_tab_popupwin. Fix closing a tabpage.
文件:　　src/window.c, src/popupwin.c, src/proto/popupwin.pro,
　　src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_02.dump,
　　src/testdir/dumps/Test_popupwin_03.dump,
　　src/testdir/dumps/Test_popupwin_04.dump

Patch 8.1.1400
问题:　　Using global pointer for tab-local popups is clumsy.
解决方案:　Use the pointer in tabpage_T.
文件:　　src/popupwin.c, src/globals.h, src/eval.c, src/screen.c,
　　src/window.c

Patch 8.1.1401
问题:　　Misspelled mkspellmem as makespellmem.
解决方案:　Drop duplicate help entry, fix test. (Naruhiko Nishino, Yasuhiro
　　Matsumoto, closes #4437)
文件:　　runtime/doc/options.txt, src/testdir/test_modeline.vim

Patch 8.1.1402
问题:　　"timer" option of popup windows not supported.
解决方案:　Implement the "timer" option. (Yasuhiro Matsumoto, closes #4439)
文件:　　src/structs.h, src/testdir/test_popupwin.vim, src/popupwin.c,
　　src/window.c, runtime/doc/popup.txt

Patch 8.1.1403
问题: Cannot build without the timer feature.
解决方案: Add #ifdef.
文件: src/structs.h, src/window.c, src/popupwin.c,
            src/testdir/test_popupwin.vim

Patch 8.1.1404
问题: Cannot change the patch level when building with NSIS.
解决方案: Use $PATCHLEVEL if defined. (Christian Brabandt)
文件: nsis/gvim.nsi

Patch 8.1.1405
问题: "highlight" option of popup windows not supported.
解决方案: Implement the "highlight" option.
文件: src/option.c, src/proto/option.pro, src/diff.c src/popupwin.c,
            runtime/doc/popup.txt, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_01.dump,
            src/testdir/dumps/Test_popupwin_03.dump

Patch 8.1.1406
问题: popup_hide() and popup_show() not implemented yet.
解决方案: Implement the functions.
文件: src/popupwin.c, src/proto/popupwin.pro, src/evalfunc.c,
            src/structs.h, runtime/doc/popup.txt, src/screen.c, src/vim.h,
            src/testdir/test_popupwin.vim

Patch 8.1.1407
问题: Popup_create() does not support text properties.
解决方案: Support the third form of the text argument.
文件: src/textprop.c, src/proto/textprop.pro, src/popupwin.c,
            src/testdir/test_popupwin.vim, src/screen.c,
            src/testdir/dumps/Test_popupwin_02.dump,
            src/testdir/dumps/Test_popupwin_03.dump,
            src/testdir/dumps/Test_popupwin_04.dump,
            runtime/doc/popup.txt

Patch 8.1.1408
问题: PFL_HIDDEN conflicts with system header file. (Ken Takata)
解决方案: Rename to POPF_HIDDEN.
文件: src/popupwin.c, src/screen.c, src/vim.h

Patch 8.1.1409
问题: Coverity warns for using uninitialized memory.
解决方案: Add a condition to clearing the growarray.
文件: src/json.c

Patch 8.1.1410
问题: Popup_move() is not implemented yet.
解决方案: Implement it. (Yasuhiro Matsumoto, closes #4441) Improve the
            positioning and resizing.
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/screen.c, src/structs.h, src/proto/popupwin.pro,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_05.dump

Patch 8.1.1411
问题: Coverity warns for divide by zero.
解决方案: Make sure width is larger than zero.
文件: src/charset.c

Patch 8.1.1412
问题: Test 30 is old style.
解决方案: Turn it into a new style test. (Yegappan Lakshmanan, closes #4440)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
            src/testdir/test30.in, src/testdir/test30.ok,
            src/testdir/test_fileformat.vim

Patch 8.1.1413
问题: Error when the drive of the swap file was disconnected.
解决方案: Try closing and re-opening the swap file. (partly by Joe Orost,
            closes #4378)
文件: src/memfile.c, src/structs.h, src/testdir/test_startup.vim

Patch 8.1.1414
问题: Alloc() returning "char_u *" causes a lot of type casts.
解决方案: Have it return "void *". (Mike Williams) Define ALLOC_ONE() to
            check the simple allocations.
文件: src/autocmd.c, src/blob.c, src/blowfish.c, src/buffer.c,
            src/change.c, src/channel.c, src/crypt.c, src/crypt_zip.c,
            src/dict.c, src/diff.c, src/eval.c, src/evalfunc.c, src/ex_cmds.c,
            src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c, src/ex_getln.c,
            src/fileio.c, src/findfile.c, src/getchar.c, src/gui_gtk.c,
            src/gui_gtk_x11.c, src/gui_mac.c, src/gui_motif.c,
            src/gui_photon.c, src/gui_w32.c, src/gui_x11.c, src/hardcopy.c,
            src/hashtab.c, src/if_cscope.c, src/if_mzsch.c, src/if_perlsfio.c,
            src/if_py_both.h, src/if_python3.c, src/if_xcmdsrv.c,
            src/insexpand.c, src/list.c, src/mark.c, src/mbyte.c,
            src/memfile.c, src/memfile_test.c, src/memline.c, src/message.c,
            src/misc2.c, src/netbeans.c, src/normal.c, src/ops.c,
            src/option.c, src/os_amiga.c, src/os_mac_conv.c, src/os_mswin.c,
            src/os_unix.c, src/os_vms.c, src/os_win32.c, src/popupmnu.c,
            src/proto/misc2.pro, src/quickfix.c, src/regexp.c,
            src/regexp_nfa.c, src/screen.c, src/search.c, src/sign.c,
            src/spell.c, src/spellfile.c, src/syntax.c, src/tag.c, src/term.c,
            src/terminal.c, src/textprop.c, src/ui.c, src/undo.c,
            src/userfunc.c, src/version.c, src/winclip.c, src/window.c,
            src/vim.h, src/testdir/test_cscope.vim

Patch 8.1.1415 (after 8.1.1414)
问题: Build error in MS-Windows GUI.
解决方案: Fix the LALLOC_MULT() argument.
文件: src/gui_w32.c

Patch 8.1.1416
问题: Popup_getposition() not implemented yet.
解决方案: Implement it. (Yasuhiro Matsumoto, closes #4449)
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim

Patch 8.1.1417
问题: MS-Windows: resolve() does not resolve all components of the path.
            (David Briscoe)
解决方案: Do not bail out for a reparse point. (Yasuhiro Matsumoto,
            closes #4211, closes #4447)
文件: src/os_mswin.c, src/testdir/test_functions.vim

Patch 8.1.1418
问题: Win_execute() is not implemented yet.
解决方案: Implement it.
文件: src/evalfunc.c, src/popupwin.c, src/testdir/test_execute_func.vim,
            runtime/doc/popup.txt, runtime/doc/eval.txt

Patch 8.1.1419
问题: Listener callbacks may be called recursively.
解决方案: Set "updating_screen" while listener callbacks are invoked.
文件: src/change.c, src/screen.c, src/proto/screen.pro, src/ui.c

Patch 8.1.1420
问题: Popup window size only uses first line length.
解决方案: Use the longest line. (Ben Jackson, closes #4451) Also deal with
            wrapping lines.
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1421
问题: Drawing "~" line in popup window.
解决方案: Just draw text in the last line of the popup window.
文件: src/screen.c, src/structs.h, src/popupwin.c,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_05.dump,
            src/testdir/dumps/Test_popupwin_06.dump

Patch 8.1.1422
问题: Popup_getoptions() not implemented yet.
解决方案: Implement it. (closes #4452)
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim

Patch 8.1.1423
问题: Popup windows use options from current window and buffer.
解决方案: Clear all local options when creating a popup window.
文件: src/popupwin.c, src/option.c, src/proto/option.pro,
            src/testdir/test_popupwin.vim

Patch 8.1.1424
问题: Crash when popup menu is deleted while waiting for char.
解决方案: Bail out when pum_array was cleared.
文件: src/popupmnu.c

Patch 8.1.1425
问题: Win_execute() does not set window pointers properly.
解决方案: Use switch_win_noblock(). Also execute autocommands in a popup
            window.

文件: src/window.c, src/proto/window.pro, src/evalfunc.c, src/autocmd.c

Patch 8.1.1426
问题: No test for syntax highlight in popup window.
解决方案: Add a screenshot test. Update associated documentation. Avoid
            'buftype' being reset by setbufvar().
文件: runtime/doc/eval.txt, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_10.dump,
            src/testdir/dumps/Test_popupwin_11.dump

Patch 8.1.1427 (after 8.1.1426)
问题: Popup window screenshot test fails.
解决方案: Add missing change to popup window code.
文件: src/popupwin.c

Patch 8.1.1428
问题: Popup_atcursor() not implemented yet.
解决方案: Implement it. (Yasuhiro Matsumoto, closes #4456)
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim

Patch 8.1.1429
问题: "pos" option of popup window not supported yet.
解决方案: Implement the option. Rename popup_getposition() to
            popup_getpos().
文件: src/structs.h, src/popupwin.c, src/proto/popupwin.pro,
            runtime/doc/popup.txt

Patch 8.1.1430
问题: Popup window option "wrap" not supported.
解决方案: Implement it.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_wrap.dump,
            src/testdir/dumps/Test_popupwin_nowrap.dump

Patch 8.1.1431
问题: Popup window listed as "Scratch".
解决方案: List them as "Popup".
文件: src/buffer.c, src/popupwin.c, src/testdir/test_popupwin.vim,
            runtime/doc/popup.txt, runtime/doc/windows.txt

Patch 8.1.1432 (after 8.1.1429)
问题: Can't build with eval feature.
解决方案: Add missing rename.
文件: src/evalfunc.c

Patch 8.1.1433
问题: Win_execute() may leave popup window focused, eventually leading
            to a crash. (Bjorn Linse)
解决方案: When previous window was closed, go to the first window.
文件: src/window.c, src/testdir/test_popupwin.vim

Patch 8.1.1434
问题: Test 3 is old style.

解决方案: Turn into a new style test. (Yegappan Lakshmanan, closes #4460)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/test3.in, src/testdir/test3.ok,
           src/testdir/test_cindent.vim

Patch 8.1.1435
问题: Memory usage test is a bit too flaky.
解决方案: Adjust the tolerances a bit. (Christian Brabandt)
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1436
问题: Writefile test fails when run under /tmp.
解决方案: Adjust 'backupskip'. (Kenta Sato, closes #4462)
文件: src/testdir/test_writefile.vim

Patch 8.1.1437
问题: Code to handle callbacks is duplicated.
解决方案: Add callback_T and functions to deal with it.
文件: src/structs.h, src/evalfunc.c, src/proto/evalfunc.pro,
           src/change.c, src/channel.c, src/proto/channel.pro, src/buffer.c,
           src/userfunc.c, src/proto/userfunc.pro, src/eval.c,
           src/ex_cmds2.c, src/popupwin.c

Patch 8.1.1438
问题: Some commands cause trouble in a popup window.
解决方案: Add NOT_IN_POPUP_WINDOW.
文件: src/macros.h, src/popupwin.c, src/proto/popupwin.pro,
           src/ex_docmd.c, src/ex_cmds2.c, src/window.c,
           src/testdir/test_popupwin.vim

Patch 8.1.1439
问题: Json_encode() is very slow for large results.
解决方案: In the growarray use a growth of at least 50%. (Ken Takata,
           closes #4461)
文件: src/misc2.c

Patch 8.1.1440
问题: Win_execute() test fails.
解决方案: Adjust the expected error number. Move to popup test.
文件: src/testdir/test_execute_func.vim, src/testdir/test_popupwin.vim

Patch 8.1.1441
问题: Popup window filter not yet implemented.
解决方案: Implement the popup filter.
文件: src/structs.h, runtime/doc/popup.txt, src/popupwin.c,
           src/proto/popupwin.pro, src/window.c, src/getchar.c, src/screen.c,
           src/misc2.c, src/proto/misc2.pro, src/vim.h,
           src/testdir/test_popupwin.vim

Patch 8.1.1442
问题: Popup windows not considered when the Vim window is resized.
           (Ben Jackson)
解决方案: Reallocate the w_lines structure. (closes #4467)
文件: src/screen.c

Patch 8.1.1443
问题: Popup window padding and border not implemented yet.
解决方案: Implement padding and border. Add core position and size to
            popup_getpos().
文件: src/structs.h, src/popupwin.c, src/screen.c,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_20.dump, runtime/doc/popup.txt

Patch 8.1.1444
问题: Not using double line characters for popup border.
解决方案: Use double line characters if using utf-8.
文件: src/screen.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_21.dump

Patch 8.1.1445
问题: Popup window border highlight not implemented yet.
解决方案: Implement the "borderhighlight" option.
文件: src/structs.h, src/popupwin.c, src/window.c, src/screen.c,
            src/testdir/test_popupwin.vim, runtime/doc/popup.txt,
            src/testdir/dumps/Test_popupwin_22.dump

Patch 8.1.1446
问题: Popup window callback not implemented yet.
解决方案: Implement the callback.
文件: runtime/doc/popup.txt, src/popupwin.c, src/structs.h,
            src/evalfunc.c, src/window.c, src/testdir/test_popupwin.vim

Patch 8.1.1447
问题: Not allowed to create an empty popup.
解决方案: Remove restriction that there is some text. (closes #4470)
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1448
问题: Statusline is sometimes drawn on top of popup.
解决方案: Redraw popups after the statusline. (Naruhiko Nishino,
            closes #4468)
文件: src/screen.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_behind.dump

Patch 8.1.1449
问题: Popup text truncated at end of screen.
解决方案: Move popup left if needed. Add the "fixed" property to disable
            that. (Ben Jackson, closes #4466)
文件: runtime/doc/popup.txt, src/popupwin.c, src/structs.h,
            src/testdir/test_popupwin.vim

Patch 8.1.1450
问题: Popup window positioning wrong when using padding or borders.
解决方案: Fix computing the position.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_corners.dump

Patch 8.1.1451

问题:　　CTRL-L does not clear screen with a popup window.
解决方案:　Do not change the type to NOT_VALID. Redraw all windows.
　　(closes #4471)
文件:　　src/screen.c

Patch 8.1.1452
问题:　　Line and col property of popup windows not properly checked.
解决方案:　Check for "+" or "-" sign.
文件:　　src/popupwin.c, src/dict.c, src/proto/dict.pro,
　　src/window.c, src/testdir/test_popupwin.vim

Patch 8.1.1453
问题:　　Popup window "moved" property not implemented yet.
解决方案:　Implement it.
文件:　　src/main.c, src/edit.c, src/gui.c, src/globals.h, src/structs.h,
　　src/screen.c, src/popupwin.c, src/proto/popupwin.pro,
　　src/testdir/test_popupwin.vim, runtime/doc/popup.txt

Patch 8.1.1454
问题:　　Build failure without the conceal feature.
解决方案:　Remove #ifdef.
文件:　　src/autocmd.c

Patch 8.1.1455
问题:　　Popup_atcursor() not completely implemented.
解决方案:　Add the default for the "moved" property.
文件:　　src/popupwin.c, src/normal.c, src/vim.h,
　　src/testdir/test_popupwin.vim

Patch 8.1.1456
问题:　　WinBar not redrawn after scrolling one line.
解决方案:　Exclude the winbar height when deciding what to redraw.
　　(closes #4473)
文件:　　src/screen.c, src/testdir/test_winbar.vim

Patch 8.1.1457
问题:　　Cannot reuse a buffer when loading a screen dump.
解决方案:　Add the "bufnr" option.
文件:　　runtime/doc/eval.txt, src/structs.h, src/channel.c,
　　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.1458
问题:　　Crash when using gtags. (issue #4102)
解决方案:　Check for negative row or col in screen_puts_len(). (Christian
　　Brabandt)
文件:　　src/screen.c

Patch 8.1.1459
问题:　　Popup window border looks bad when 'ambiwidth' is "double".
　　(Yasuhiro Matsumoto)
解决方案:　Only use line drawing characters when 'ambiwidth' is "single".
　　(Ken Takata, closes #4477)
文件:　　src/screen.c

Patch 8.1.1460
问题:　　Popup window border characters may be wrong.
解决方案:　Reset the border characters for each popup. Correct use of
'ambiwidth'.
文件:　　src/screen.c

Patch 8.1.1461
问题:　　Tests do not run or are not reliable on some systems.
解决方案:　Use "findstr" instead of "grep" on MS-Windows. Clear
PROMPT_COMMAND in the terminal test. Delete temp file. Wait for
output after executing a debug command. (Yegappan Lakshmanan,
closes #4479)
文件:　　src/testdir/test_debugger.vim, src/testdir/test_environ.vim,
src/testdir/test_filetype.vim, src/testdir/test_source.vim,
src/testdir/test_terminal.vim

Patch 8.1.1462
问题:　　MS-Windows: using special character requires quoting.
解决方案:　Add quotes. (Ken Takata)
文件:　　src/testdir/test_environ.vim

Patch 8.1.1463
问题:　　Gcc warns for uninitialized variable.
解决方案:　Put usage inside "if". (Ken Takata)
文件:　　src/textprop.c

Patch 8.1.1464
问题:　　Only 4-digit rgb termresponse is recognized.
解决方案:　Also recognize 2-digit rgb response. (closes #4486)
文件:　　src/term.c, src/test_termcodes.vim

Patch 8.1.1465
问题:　　Allocating wrong amount of memory. (Yegappan Lakshmanan)
解决方案:　Use sizeof() for right type of struct.
文件:　　src/memfile_test.c

Patch 8.1.1466
问题:　　Not updating priority on existing sign.
解决方案:　Set the sign priority. Add a test. (Yegappan Lakshmanan)
文件:　　src/sign.c, src/testdir/test_signs.vim, runtime/doc/eval.txt,
runtime/doc/sign.txt

Patch 8.1.1467 (after 8.1.1465)
问题:　　Cscope test fails.
解决方案:　Update expected text.
文件:　　src/testdir/test_cscope.vim

Patch 8.1.1468
问题:　　The generated desktop files may be invalid.
解决方案:　Check validity with desktop-file-validate. (Christian Brabandt,
Will Thompson, closes #4480)
文件:　　src/po/Makefile

Patch 8.1.1469

问题: No test for checking the cursor style response.
解决方案: Add a simple test. Also include the missing part of 8.1.1464.
文件: src/term.c, src/testdir/test_termcodes.vim

Patch 8.1.1470
问题: New Unicode character U+32FF missing from double-width table.
解决方案: Add the character.
文件: src/mbyte.c

Patch 8.1.1471
问题: 'background' not correctly set for 2-digit rgb termresponse.
解决方案: Adjust what digit to use. (closes #4495)
文件: src/term.c, src/testdir/test_termcodes.vim

Patch 8.1.1472
问题: Add_termcap_entry() is not tested.
解决方案: Add a simple test.
文件: src/testdir/test_termcodes.vim

Patch 8.1.1473
问题: New resolve() implementation causes problem for plugins.
解决方案: Only resolve a reparse point after checking it is needed. (Ken
            Takata, closes #4492)
文件: src/os_mswin.c, src/testdir/test_functions.vim

Patch 8.1.1474
问题: 'ttybuiltin' is not tested.
解决方案: At least test that it doesn't break things.
文件: src/testdir/test_termcodes.vim

Patch 8.1.1475
问题: Search string not displayed when 'rightleft' is set.
解决方案: Clear the right part of the old text. (closes #4488, closes #4489)
文件: src/search.c, src/testdir/test_search.vim

Patch 8.1.1476
问题: No statistics displayed after running tests.
解决方案: Summarize the test results. (Christian Brabandt, closes #4391)
            Also make it possible to report a skipped file.
文件: src/Makefile, src/testdir/Makefile, src/testdir/summarize.vim,
            src/testdir/runtest.vim, src/testdir/test_arabic.vim,
            src/testdir/test_autochdir.vim, src/testdir/test_balloon.vim

Patch 8.1.1477
问题: Test summary fails in the tiny version.
解决方案: set 'nocompatible'.
文件: Filelist, src/testdir/summarize.vim

Patch 8.1.1478
问题: Still an error when running tests with the tiny version.
解决方案: Do not try reading test.log
文件: src/testdir/Makefile, src/testdir/summarize.vim

Patch 8.1.1479

问题: Change included for debugging only.
解决方案: Restore the REDIR_TEST_TO_NULL line.
文件: src/testdir/Makefile

Patch 8.1.1480
问题: Desktop file check doesn't run on CI.
解决方案: Install the desktop-file-utils packages. (Christian Brabandt,
            closes #4498)
文件: .travis.yml

Patch 8.1.1481
问题: Length for two-digit rgb termresponse is off by one.
解决方案: Adjust the length. (closes #4494)
文件: src/term.c

Patch 8.1.1482
问题: No test for wincol() depending on the 'number' option.
解决方案: Add a couple of tests. (Christian Brabandt, closes #4500)
文件: src/testdir/test_gui.vim

Patch 8.1.1483
问题: Skipped tests are not properly listed.
解决方案: Throw a "Skipped" exception instead of using ":finish" or ":return".
文件: src/testdir/test_breakindent.vim, src/testdir/test_cdo.vim,
            src/testdir/test_channel.vim, src/testdir/test_clientserver.vim,
            src/testdir/test_balloon.vim, src/testdir/test_conceal.vim,
            src/testdir/test_debugger.vim, src/testdir/test_diffmode.vim,
            src/testdir/test_fold.vim, src/testdir/test_highlight.vim,
            src/testdir/test_popup.vim, src/testdir/test_popupwin.vim,
            src/testdir/test_search.vim, src/testdir/test_startup.vim,
            src/testdir/test_startup_utf8.vim, src/testdir/test_syntax.vim,
            src/testdir/test_tabpage.vim, src/testdir/test_termencoding.vim,
            src/testdir/test_terminal.vim, src/testdir/test_textprop.vim,
            src/testdir/test_timers.vim

Patch 8.1.1484
问题: Some tests are slow.
解决方案: Add timing to the test messages. Fix double free when quitting in
            VimLeavePre autocmd.
文件: src/testdir/runtest.vim, src/eval.c

Patch 8.1.1485
问题: Double free when garbage_collect() is used in autocommand.
解决方案: Have garbage collection also set the copyID in funccal_stack.
文件: src/eval.c, src/userfunc.c

Patch 8.1.1486
问题: A listener change is merged even when it adds a line. (Paul Jolly)
解决方案: Do not merge a change that adds or removes a line. (closes #4490)
文件: src/change.c, src/testdir/test_listener.vim

Patch 8.1.1487
问题: Older msgfmt cannot generate proper .desktop file.
解决方案: Add a configure check to not use this msgfmt version. (Ken Takata)

文件: src/configure.ac, src/auto/configure

Patch 8.1.1488
问题: Summary of tests has incorrect failed count.
解决方案: Add to the failed count instead of setting it. (Christian Brabandt)
文件: src/testdir/summarize.vim

Patch 8.1.1489
问题: Sign order wrong when priority was changed.
解决方案: Reorder signs when priority is changed. (Yegappan Lakshmanan,
            closes #4502)
文件: src/quickfix.c, src/sign.c, src/testdir/test_signs.vim

Patch 8.1.1490
问题: When a single test fails the exit code is not set. (Daniel Hahler)
解决方案: Add an exit command. (closes #4506)
文件: src/testdir/Makefile

Patch 8.1.1491
问题: When skipping over code after an exception was thrown expression
            evaluation is aborted after a function call. (Ingo Karkat)
解决方案: Do not fail if not executing the expression. (closes #4507)
文件: src/eval.c, src/testdir/test_eval_stuff.vim

Patch 8.1.1492
问题: MS-Windows: when "!" is in 'guioptions' ":!start" fails.
解决方案: Do not use a terminal window when the shell command begins with
            "!start". (Yasuhiro Matsumoto, closes #4504)
文件: src/misc2.c, src/os_win32.c

Patch 8.1.1493
问题: Redrawing with popups is slow and causes flicker.
解决方案: Avoid clearing and redrawing using a zindex mask.
文件: src/globals.h, src/screen.c, src/proto/screen.pro, src/popupwin.c,
            src/popupmnu.c

Patch 8.1.1494 (after 8.1.1493)
问题: Build failure.
解决方案: Add missing changes.
文件: src/structs.h

Patch 8.1.1495 (after 8.1.1494)
问题: Memory access error.
解决方案: Use the correct size for clearing the popup mask.
文件: src/screen.c

Patch 8.1.1496
问题: Popup window height is not recomputed.
解决方案: Recompute the height when needed.
文件: src/popupwin.c, src/testdir/dumps/Test_popupwin_06.dump

Patch 8.1.1497
问题: Accessing memory beyond allocated space.
解决方案: Check column before accessing popup mask.

文件: src/screen.c

Patch 8.1.1498
问题: ":write" increments b:changedtick even though nothing changed.
            (Daniel Hahler)
解决方案: Only increment b:changedtick if the modified flag is reset.
文件: src/change.c, src/proto/change.pro, runtime/doc/eval.txt,
            src/buffer.c, src/ex_cmds2.c, src/fileio.c, src/memline.c,
            src/undo.c

Patch 8.1.1499
问题: Ruler not updated after popup window was removed.
解决方案: use popup_mask in screen_puts().
文件: src/screen.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_07.dump,
            src/testdir/dumps/Test_popupwin_08.dump

Patch 8.1.1500
问题: Wrong shell command when building with VIMDLL and "!" in
            'guioptions'.
解决方案: Add check for GUI in use. (Ken Takata)
文件: src/misc2.c

Patch 8.1.1501
问题: New behavior of b:changedtick not tested.
解决方案: Add a few test cases. (Daniel Hahler)
文件: src/testdir/test_changedtick.vim

Patch 8.1.1502
问题: Cannot play any sound.
解决方案: Use libcanberra if available. Add sound functions.
文件: src/configure.ac, src/auto/configure, src/config.h.in,
            src/Makefile, src/sound.c, src/proto/sound.pro, src/proto.h,
            src/evalfunc.c, src/feature.h, runtime/doc/eval.txt, Filelist,
            src/version.c, src/testdir/test_sound.vim, src/testdir/silent.wav,
            src/testdir/Make_all.mak, .travis.yml

Patch 8.1.1503
问题: Sound test fails on Travis.
解决方案: Set AUDIODEV to "null".
文件: .travis.yml

Patch 8.1.1504
问题: Sound test still fails on Travis.
解决方案: Add more lines to the install section.
文件: .travis.yml

Patch 8.1.1505
问题: Running "make clean" twice gives errors.
解决方案: Add "-f" to "rm". (closes #4516)
文件: src/testdir/Makefile

Patch 8.1.1506
问题: Syntax error in Travis config.

解决方案: Set AUDIODEV in another section.
文件: .travis.yml

Patch 8.1.1507
问题: Sound test still fails on Travis.
解决方案: Try another dummy sound approach.
文件: .travis.yml

Patch 8.1.1508
问题: Sound keeps failing on Travis.
解决方案: Throw a skipped exception in the test.
文件: src/testdir/test_sound.vim

Patch 8.1.1509
问题: Cmdline_row can become negative, causing a crash.
解决方案: Make sure cmdline_row does not become negative. (closes #4102)
文件: src/misc1.c

Patch 8.1.1510
问题: A plugin cannot easily expand a command like done internally.
解决方案: Add the expandcmd() function. (Yegappan Lakshmanan, closes #4514)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
           src/testdir/test_expand.vim

Patch 8.1.1511
问题: Matches in a popup window are not displayed properly.
解决方案: Do display matches in a popup window. (closes #4517)
文件: src/screen.c, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_matches.dump

Patch 8.1.1512
问题: ch_evalexpr() hangs when used recursively. (Paul Jolly)
解决方案: Change ch_block_id from a single number to a list of IDs to wait
           on.
文件: src/channel.c, src/structs.h

Patch 8.1.1513
问题: All popup functionality is in functions, except :popupclear.
解决方案: Add popup_clear() for consistency. Also rename sound_stopall() to
           sound_clear().
文件: src/ex_cmds.h, src/ex_cmdidxs.h, src/evalfunc.c, src/popupwin.c,
           src/proto/popupwin.pro, src/sound.c, src/proto/sound.pro,
           src/testdir/test_popupwin.vim src/testdir/test_sound.vim,
           runtime/doc/eval.txt runtime/doc/popup.txt

Patch 8.1.1514 (after 8.1.1492)
问题: MS-Windows: wrong shell command with ! in 'guioptions'.
解决方案: Do not check for ! in 'guioptions' when applying 'shellxquote'.
           (Yasuhiro Matsumoto, closes #4519)
文件: src/misc2.c

Patch 8.1.1515
问题: Memory leak reported for sound when build with EXITFREE.
解决方案: Free sound stuff when exiting.

文件: src/misc2.c

Patch 8.1.1516
问题: Time reported for a test measured wrong.
解决方案: Move the computation to the end of RunTheTest(). (Ozaki Kiichi,
            closes #4520)
文件: src/testdir/runtest.vim

Patch 8.1.1517
问题: When a popup changes all windows are redrawn.
解决方案: Only update the lines that were affected. Add a file for
            profiling popup windows efficiency.
文件: src/screen.c, src/proto/screen.pro, src/ui.c, src/popupwin.c,
            src/globals.h, src/testdir/popupbounce.vim, Filelist

Patch 8.1.1518
问题: Crash when setting 'columns' while a popup is visible.
解决方案: Recompute all positions when clearing the screen. (closes #4467)
文件: src/screen.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_04a.dump

Patch 8.1.1519
问题: 'backupskip' may contain duplicates.
解决方案: Add the P_NODUP flag. (Tom Ryder)
文件: src/option.c, src/testdir/test_options.vim

Patch 8.1.1520
问题: Popup windows are ignored when dealing with mouse position
解决方案: Find the mouse position inside a popup window. Allow for modeless
            selection.
文件: src/ui.c, src/proto/ui.pro, src/popupwin.c,
            src/proto/popupwin.pro, src/screen.c, src/beval.c, src/edit.c,
            src/evalfunc.c, src/gui.c, src/normal.c, src/structs.h

Patch 8.1.1521
问题: When a popup window is closed the buffer remains.
解决方案: Wipe out the buffer.
文件: src/window.c, src/testdir/test_popupwin.vim

Patch 8.1.1522
问题: Popup_notification() not implemented yet.
解决方案: Implement it.
文件: src/popupwin.c, src/proto/popupwin.pro, src/evalfunc.c,
            src/structs.h, src/testdir/test_popupwin.vim,
            runtime/doc/popup.txt
            src/testdir/dumps/Test_popupwin_notify_01.dump,
            src/testdir/dumps/Test_popupwin_notify_02.dump

Patch 8.1.1523
问题: Cannot show range of buffer lines in popup window.
解决方案: Add the "firstline" property. (closes #4523)
文件: src/popupwin.c, src/structs.h, runtime/doc/popup.txt,
            src/testdir/test_popupwin.vim,
            testdir/dumps/Test_popupwin_firstline.dump

Patch 8.1.1524
问题:　　Tests are silently skipped.
解决方案:　Throw an exception for skipped tests in more places.
文件:　　src/testdir/test_assert.vim, src/testdir/test_paste.vim,
　　src/testdir/shared.vim, src/testdir/test_crypt.vim,
　　src/testdir/test_cscope.vim, src/testdir/test_digraph.vim,
　　src/testdir/test_float_func.vim, src/testdir/test_gui.vim,
　　src/testdir/test_gui_init.vim, src/testdir/test_history.vim,
　　src/testdir/test_langmap.vim, src/testdir/test_listlbr.vim,
　　src/testdir/test_listlbr_utf8.vim, src/testdir/test_lua.vim,
　　src/testdir/test_makeencoding.vim,
　　src/testdir/test_matchadd_conceal.vim,
　　src/testdir/test_matchadd_conceal_utf8.vim,
　　src/testdir/test_memory_usage.vim, src/testdir/test_menu.vim,
　　src/testdir/test_mksession.vim,
　　src/testdir/test_mksession_utf8.vim,
　　src/testdir/test_netbeans.vim, src/testdir/test_paste.vim,
　　src/testdir/test_perl.vim, src/testdir/test_profile.vim,
　　src/testdir/test_prompt_buffer.vim, src/testdir/test_python2.vim,
　　src/testdir/test_python3.vim, src/testdir/test_pyx2.vim,
　　src/testdir/test_pyx3.vim, src/testdir/test_quickfix.vim,
　　src/testdir/test_quotestar.vim, src/testdir/test_reltime.vim,
　　src/testdir/test_ruby.vim, src/testdir/test_sha256.vim,
　　src/testdir/test_shortpathname.vim, src/testdir/test_signals.vim,
　　src/testdir/test_signs.vim, src/testdir/test_spell.vim,
　　src/testdir/test_syntax.vim, src/testdir/test_tcl.vim,
　　src/testdir/test_termcodes.vim, src/testdir/test_terminal.vim,
　　src/testdir/test_terminal_fail.vim,
　　src/testdir/test_textobjects.vim, src/testdir/test_textprop.vim,
　　src/testdir/test_timers.vim, src/testdir/test_vartabs.vim,
　　src/testdir/test_winbar.vim, src/testdir/test_windows_home.vim,
　　src/testdir/test_xxd.vim

Patch 8.1.1525
问题:　　Cannot move a popup window with the mouse.
解决方案:　Add the "drag" property and make it possible to drag a popup
　　window by its border.
文件:　　src/popupwin.c, src/proto/popupwin.pro, src/structs.h, src/ui.c,
　　src/window.c, src/proto/window.pro, runtime/doc/popup.txt

Patch 8.1.1526
问题:　　No numerical value for the patchlevel.
解决方案:　Add v:versionlong.
文件:　　src/version.c, src/eval.c, src/vim.h, runtime/doc/eval.txt,
　　src/testdir/test_eval_stuff.vim

Patch 8.1.1527
问题:　　When moving a popup window over the command line it is not
　　redrawn.
解决方案:　Redraw the command line. Move popup redrawing code to the popupwin
　　file.
文件:　　src/screen.c, src/proto/screen.pro, src/popupwin.c,
　　src/proto/popupwin.pro, src/testdir/test_popupwin.vim,

src/testdir/dumps/Test_popupwin_drag_01.dump,
src/testdir/dumps/Test_popupwin_drag_02.dump

Patch 8.1.1528
问题: Popup_any_visible() is unused.
解决方案: Remove it.
文件: src/popupwin.c, src/proto/popupwin.pro

Patch 8.1.1529
问题: Libcanberra is linked with even when not used.
解决方案: Have configure check for libcanberra only when wanted.
(suggestions by Libor Bukata)
文件: src/feature.h, src/configure.ac, src/auto/configure, src/Makefile

Patch 8.1.1530
问题: Travis config is not optimal.
解决方案: Remove system conditions. Do not use excluding matrix. Cache OSX
results. (Ozaki Kiichi, closes #4521)
文件: .travis.yml

Patch 8.1.1531
问题: Clipboard type name is inconsistent.
解决方案: Rename VimClipboard to Clipboard_T.
文件: src/gui_gtk_x11.c, src/proto/gui_gtk_x11.pro, src/gui_mac.c,
src/proto/gui_mac.pro, src/gui_x11.c, src/proto/gui_x11.pro,
src/ops.c, src/proto/ops.pro, src/os_qnx.c, src/proto/os_qnx.pro,
src/os_unix.c, src/proto/os_unix.pro, src/ui.c, src/proto/ui.pro,
src/winclip.c, src/proto/winclip.pro, src/globals.h, src/proto.h

Patch 8.1.1532 (after 8.1.1531)
问题: Build fails.
解决方案: Add missing changes.
文件: src/vim.h

Patch 8.1.1533
问题: GUI build fails on Mac.
解决方案: Change VimClipboard type in non-C file.
文件: src/os_macosx.m

Patch 8.1.1534
问题: Modeless selection in popup window selects too much.
解决方案: Restrict the selection to inside of the popup window.
文件: src/vim.h, src/ui.c, src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_select_01.dump,
src/testdir/dumps/Test_popupwin_select_02.dump

Patch 8.1.1535 (after 8.1.1534)
问题: Popup select test fails on Mac.
解决方案: Skip test if clipboard feature not available.
文件: src/testdir/test_popupwin.vim

Patch 8.1.1536 (after 8.1.1534)
问题: Popup select test still fails on Mac.
解决方案: Set 'clipboard' to "autoselect"

文件: src/testdir/test_popupwin.vim

Patch 8.1.1537
问题: Using "tab" for popup window can be confusing.
解决方案: Use "tabpage". (Hirohito Higashi, closes #4532)
文件: runtime/doc/popup.txt, src/popupwin.c,
            src/testdir/test_popupwin.vim

Patch 8.1.1538
问题: Cannot specify highlighting for notifications.
解决方案: Use the PopupNotification group if it exists. Add a minimal width
            to notifications.
文件: runtime/doc/popup.txt, src/popupwin.c,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_notify_01.dump,
            src/testdir/dumps/Test_popupwin_notify_02.dump

Patch 8.1.1539
问题: Not easy to define a variable and lock it.
解决方案: Add ":const". (Ryuichi Hayashida, closes #4541)
文件: runtime/doc/eval.txt, src/eval.c, src/ex_cmdidxs.h, src/ex_cmds.h,
            src/proto/eval.pro, src/testdir/Make_all.mak,
            src/testdir/test_const.vim

Patch 8.1.1540 (after 8.1.1539)
问题: Cannot build without the +eval feature.
解决方案: Define ex_const if needed.
文件: src/ex_docmd.c

Patch 8.1.1541
问题: Check for ASAN is not reliable.
解决方案: Check the version output. (Dominique Pelle, closes #4543)
文件: src/testdir/test_memory_usage.vim

Patch 8.1.1542
问题: An OptionSet autocommand does not get enough info.
解决方案: Add v:option_command, v:option_oldlocal and v:option_oldglobal.
            (Latrice Wilgus, closes #4118)
文件: runtime/doc/autocmd.txt, runtime/doc/eval.txt,
            runtime/doc/version8.txt, src/eval.c, src/option.c, src/structs.h,
            src/testdir/test_autocmd.vim, src/vim.h

Patch 8.1.1543
问题: Const test fails with small features.
解决方案: Don't unlet non-existing variables.
文件: src/testdir/test_const.vim

Patch 8.1.1544
问题: Some balloon tests don't run when they can.
解决方案: Split GUI balloon tests off into a separate file. (Ozaki Kiichi,
            closes #4538) Change the feature check into a command for
            consistency.
文件: Filelist, src/testdir/Make_all.mak, src/testdir/check.vim,
            src/testdir/test_arabic.vim, src/testdir/test_balloon.vim,

src/testdir/test_balloon_gui.vim, src/testdir/test_crypt.vim,
src/testdir/test_cscope.vim, src/testdir/test_digraph.vim,
src/testdir/test_float_func.vim, src/testdir/test_gui.vim,
src/testdir/test_gui_init.vim, src/testdir/test_history.vim,
src/testdir/test_langmap.vim, src/testdir/test_listlbr.vim,
src/testdir/test_listlbr_utf8.vim, src/testdir/test_lua.vim,
src/testdir/test_makeencoding.vim,
src/testdir/test_matchadd_conceal.vim,
src/testdir/test_matchadd_conceal_utf8.vim,
src/testdir/test_memory_usage.vim, src/testdir/test_menu.vim,
src/testdir/test_mksession.vim,
src/testdir/test_mksession_utf8.vim,
src/testdir/test_netbeans.vim, src/testdir/test_paste.vim,
src/testdir/test_perl.vim, src/testdir/test_popupwin.vim,
src/testdir/test_profile.vim, src/testdir/test_prompt_buffer.vim,
src/testdir/test_python2.vim, src/testdir/test_python3.vim,
src/testdir/test_pyx2.vim, src/testdir/test_pyx3.vim,
src/testdir/test_quickfix.vim, src/testdir/test_quotestar.vim,
src/testdir/test_reltime.vim, src/testdir/test_ruby.vim,
src/testdir/test_sha256.vim, src/testdir/test_shortpathname.vim,
src/testdir/test_signals.vim, src/testdir/test_signs.vim,
src/testdir/test_spell.vim, src/testdir/test_syntax.vim,
src/testdir/test_tcl.vim, src/testdir/test_termcodes.vim,
src/testdir/test_terminal.vim, src/testdir/test_terminal_fail.vim,
src/testdir/test_textobjects.vim, src/testdir/test_textprop.vim,
src/testdir/test_timers.vim, src/testdir/test_vartabs.vim,
src/testdir/test_winbar.vim, src/testdir/test_windows_home.vim,
src/testdir/test_xxd.vim

Patch 8.1.1545
问题: When the screen is to small there is no message about that.
(Daniel Hahler)
解决方案: Do not use :cquit. (closes #4534)
文件: src/testdir/runtest.vim

Patch 8.1.1546
问题: In some tests 'tags' is set but not restored. (Daniel Hahler)
解决方案: Restore 'tags'. (closes #4535)
文件: src/testdir/test_autocmd.vim, src/testdir/test_cmdline.vim,
src/testdir/test_options.vim, src/testdir/test_tagcase.vim,
src/testdir/test_tagjump.vim, src/testdir/test_taglist.vim

Patch 8.1.1547
问题: Functionality of bt_nofile() is confusing.
解决方案: Split into bt_nofile() and bt_nofilename().
文件: src/buffer.c, src/proto/buffer.pro, src/evalfunc.c, src/ex_cmds.c,
src/ex_docmd.c, src/fileio.c, src/popupmnu.c, src/quickfix.c

Patch 8.1.1548
问题: Popup_dialog() is not implemented.
解决方案: Implement popup_dialog() and popup_filter_yesno().
文件: src/popupwin.c, src/proto/popupwin.pro, src/evalfunc.c,
src/structs.h, src/globals.h, src/testdir/test_popupwin.vim,
runtime/doc/popup.txt

Patch 8.1.1549 (after 8.1.1547)
问题: Quickfix test fails.
解决方案: Negate result of bt_quickfix().
文件: src/quickfix.c

Patch 8.1.1550
问题: When a popup has left padding text may be cut off.
解决方案: Add the border and padding when computing the size.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_20.dump,
            src/testdir/dumps/Test_popupwin_21.dump

Patch 8.1.1551
问题: Warning for shadowing popup_dragwin. (Dominique Pelle)
解决方案: Add missing change.
文件: src/ui.c

Patch 8.1.1552
问题: Cursor position is wrong after sign column appears or disappears.
            (Yegappan Lakshmanan)
解决方案: Call changed_line_abv_curs() instead of changed_cline_bef_curs().
文件: src/sign.c, src/testdir/test_signs.vim,
            src/testdir/dumps/Test_sign_cursor_01.dump,
            src/testdir/dumps/Test_sign_cursor_02.dump

Patch 8.1.1553
问题: Not easy to change the text in a popup window.
解决方案: Add popup_settext(). (Ben Jackson, closes #4549)
            Also display a space for an empty popup.
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/proto/popupwin.pro,
            src/testdir/dumps/Test_popup_settext_01.dump,
            src/testdir/dumps/Test_popup_settext_02.dump,
            src/testdir/dumps/Test_popup_settext_03.dump,
            src/testdir/dumps/Test_popup_settext_04.dump,
            src/testdir/dumps/Test_popup_settext_05.dump,
            src/testdir/dumps/Test_popup_settext_06.dump,
            src/testdir/test_popupwin.vim

Patch 8.1.1554 (after 8.1.1539)
问题: Docs and tests for :const can be improved.
解决方案: Improve documentation, add a few more tests. (Ryuichi Hayashida,
            closes #4551)
文件: runtime/doc/eval.txt, src/testdir/test_const.vim

Patch 8.1.1555
问题: NOT_IN_POPUP_WINDOW is confusing. (Andy Massimino)
解决方案: Rename to ERROR_IF_POPUP_WINDOW().
文件: src/popupwin.c, src/proto/popupwin.pro, src/macros.h,
            src/ex_cmds2.c, src/ex_docmd.c, src/window.c

Patch 8.1.1556
问题: The command displayed to show a failing screenshot does not include

the "testdir" directory.
解决方案: Prefix the directory name so that it can be copy-pasted.
文件: src/testdir/screendump.vim

Patch 8.1.1557
问题: Compiler warning for unused variables in tiny version. (Tony Mechelynck)
解决方案: Add #ifdef.
文件: src/option.c

Patch 8.1.1558
问题: Popup_menu() and popup_filter_menu() are not implemented yet.
解决方案: Implement the functions. Fix that centering didn't take the border and padding into account.
文件: runtime/doc/popup.txt, src/popupwin.c, src/proto/popupwin.pro, src/evalfunc.c, src/screen.c, src/testdir/test_popupwin.vim, src/testdir/dumps/Test_popupwin_menu_01.dump, src/testdir/dumps/Test_popupwin_menu_02.dump, src/testdir/dumps/Test_popupwin_menu_03.dump, src/testdir/dumps/Test_popupwin_drag_01.dump, src/testdir/dumps/Test_popupwin_drag_02.dump

Patch 8.1.1559
问题: Popup window title property not implemented yet.
解决方案: Implement the title property.
文件: runtime/doc/popup.txt, src/popupwin.c, src/structs.h src/window.c, src/testdir/test_popupwin.vim, src/testdir/dumps/Test_popupwin_menu_01.dump, src/testdir/dumps/Test_popupwin_menu_02.dump, src/testdir/dumps/Test_popupwin_title.dump

Patch 8.1.1560
问题: Popup window hidden option not implemented yet.
解决方案: Implement the hidden option.
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1561
问题: Popup_setoptions() is not implemented yet.
解决方案: Implement popup_setoptions(). Also add more fields to popup_getoptions().
文件: runtime/doc/popup.txt, src/popupwin.c, src/proto/popupwin.pro, src/dict.c, src/proto/dict.pro, src/evalfunc.c, src/testdir/test_popupwin.vim, src/testdir/runtest.vim

Patch 8.1.1562
问题: Popup window not always redrawn after popup_setoptions().
解决方案: Force a redraw.
文件: src/popupwin.c, src/testdir/test_popupwin.vim, src/testdir/dumps/Test_popupwin_23.dump

Patch 8.1.1563
问题: Crash when using closures.
解决方案: Set reference in varlist of funccal when running the garbage collector. (Ozaki Kiichi, closes #4554, closes #4547)

文件: src/testdir/test_vimscript.vim, src/userfunc.c

Patch 8.1.1564
问题: Sign column takes up space. (Adam Stankiewicz)
解决方案: Optionally put signs in the number column. (Yegappan Lakshmanan,
            closes #4555, closes #4515)
文件: runtime/doc/options.txt, src/option.c, src/screen.c,
            src/testdir/test_signs.vim

Patch 8.1.1565
问题: MS-Windows: no sound support.
解决方案: Add sound support for MS-Windows. (Yasuhiro Matsumoto, Ken Takata,
            closes #4522)
文件: runtime/doc/eval.txt, src/Make_cyg_ming.mak, src/Make_mvc.mak,
            src/sound.c, src/testdir/test_sound.vim

Patch 8.1.1566
问题: Error message when terminal closes while it is not in the current
            tab.
解决方案: Also set "do_set_w_closing" when using the special autocommand
            window. (closes #4552)
文件: src/terminal.c

Patch 8.1.1567
问题: Localtime_r() does not respond to $TZ changes.
解决方案: If $TZ changes then call tzset(). (Tom Ryder)
文件: src/auto/configure, src/config.h.in, src/configure.ac,
            src/evalfunc.c, src/memline.c, src/proto/memline.pro,
            src/testdir/test_functions.vim, src/undo.c

Patch 8.1.1568 (after 8.1.1567)
问题: Strftime() test fails on MS-Windows.
解决方案: Skip the check for using the $TZ environment variable.
文件: src/testdir/test_functions.vim

Patch 8.1.1569
问题: Cannot build with signs but without diff feature.
解决方案: Move #ifdef. (Tom Ryder)
文件: src/screen.c

Patch 8.1.1570
问题: Icon signs not displayed properly in the number column.
解决方案: Display them properly. (Yegappan Lakshmanan, closes #4559)
文件: src/gui.c, src/screen.c, src/testdir/test_signs.vim

Patch 8.1.1571
问题: textprop highlight starts too early if just after a tab.
解决方案: Check if still drawing a previous character. (closes #4558)
文件: src/screen.c, src/testdir/test_textprop.vim,
            src/testdir/dumps/Test_textprop_tab.dump

Patch 8.1.1572 (after 8.1.1569)
问题: Compiler warnings with tiny build. (Tony Mechelynck)
解决方案: Add #ifdef.

文件: src/screen.c

Patch 8.1.1573 (after 8.1.1571)
问题: Textprop test fails if screenshots do not work.
解决方案: Add check for screenshots working.
文件: src/testdir/test_textprop.vim

Patch 8.1.1574
问题: Tabpage option not yet implemented for popup window.
解决方案: Implement tabpage option, also for popup_getoptions().
文件: runtime/doc/popup.txt, src/popupwin.c,
            src/testdir/test_popupwin.vim

Patch 8.1.1575
问题: Callbacks may be garbage collected.
解决方案: Set reference in callbacks. (Ozaki Kiichi, closes #4564)
文件: src/buffer.c, src/channel.c, src/eval.c, src/ex_cmds2.c,
            src/popupwin.c, src/proto/buffer.pro, src/proto/popupwin.pro,
            src/terminal.c, src/testdir/test_listener.vim,
            src/testdir/test_popupwin.vim, src/testdir/test_prompt_buffer.vim,
            src/userfunc.c

Patch 8.1.1576
问题: Compiler warning for unused argument.
解决方案: Add "UNUSED" annotation. (Dominique Pelle, closes #4570)
文件: src/ui.c

Patch 8.1.1577
问题: Command line redrawn for +arabic without Arabic characters.
            (Dominique Pelle)
解决方案: Check if there actually are any Arabic characters. Do redraw
            after displaying incsearch. (closes #4569)
文件: src/ex_getln.c

Patch 8.1.1578
问题: MS-Windows: pathdef.c should depend on build options.
解决方案: Generate pathdef.c in the object directory. Fix dependencies.
            (Ken Takata, closes #4565)
文件: .gitignore, .hgignore, src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.1579
问题: Dict and list could be GC'ed while displaying error in a timer.
            (Yasuhiro Matsumoto)
解决方案: Block garbage collection when executing a timer. Add
            test_garbagecollect_soon(). Add "no_wait_return" to
            test_override(). (closes #4571)
文件: src/dict.c, src/testdir/test_timers.vim, src/evalfunc.c,
            runtime/doc/eval.txt

Patch 8.1.1580
问题: Cannot make part of a popup transparent.
解决方案: Add the "mask" option.
文件: runtime/doc/popup.txt, src/popupwin.c, src/screen.c,
            src/structs.h, src/window.c, src/ui.c, src/vim.h, src/globals.h,

src/testdir/dumps/Test_popupwin_mask_1.dump,
src/testdir/dumps/Test_popupwin_mask_2.dump

Patch 8.1.1581
问题: Shared functions for testing are disorganised.
解决方案: Group functions in script files. (Ozaki Kiichi, closes #4573)
文件: Filelist, src/testdir/screendump.vim, src/testdir/shared.vim,
src/testdir/term_util.vim, src/testdir/test_mksession.vim,
src/testdir/test_suspend.vim, src/testdir/test_terminal.vim,
src/testdir/test_timers.vim, src/testdir/view_util.vim

Patch 8.1.1582
问题: Cannot build with +textprop but without +timers.
解决方案: Add #ifdef. (Ola Sder, closes #4574)
文件: src/popupwin.c

Patch 8.1.1583
问题: Set_ref_in_list() only sets ref in items.
解决方案: Rename to set_ref_in_list_items() to avoid confusion.
文件: src/eval.c, src/proto/eval.pro, src/if_lua.c, src/popupwin.c,
src/userfunc.c, src/if_py_both.h

Patch 8.1.1584
问题: The evalfunc.c file is getting too big.
解决方案: Move channel and job related functions to channel.c.
文件: src/channel.c, src/evalfunc.c, src/proto/channel.pro

Patch 8.1.1585
问题: :let-heredoc does not trim enough.
解决方案: Trim indent from the contents based on the indent of the first
line. Use let-heredoc in more tests.
文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_balloon.vim,
src/testdir/test_cindent.vim, src/testdir/test_const.vim,
src/testdir/test_debugger.vim, src/testdir/test_functions.vim,
src/testdir/test_goto.vim, src/testdir/test_gui.vim,
src/testdir/test_highlight.vim, src/testdir/test_join.vim,
src/testdir/test_let.vim, src/testdir/test_memory_usage.vim,
src/testdir/test_messages.vim,
src/testdir/test_mksession_utf8.vim, src/testdir/test_normal.vim,
src/testdir/test_popup.vim, src/testdir/test_popupwin.vim,
src/testdir/test_profile.vim, src/testdir/test_quickfix.vim,
src/testdir/test_xxd.vim

Patch 8.1.1586
问题: Error number used in two places.
解决方案: Renumber one. (Ken Takata)
文件: runtime/doc/popup.txt, src/popupwin.c

Patch 8.1.1587
问题: Redraw problem when sign icons in the number column.
解决方案: Clear and redraw when changing related options. Right align the
sign icon in the GUI. (Yegappan Lakshmanan, closes #4578)
文件: src/gui.c, src/option.c

Patch 8.1.1588
问题: In :let-heredoc line continuation is recognized.
解决方案: Do not consume line continuation. (Ozaki Kiichi, closes #4580)
文件: src/autocmd.c, src/digraph.c, src/eval.c, src/evalfunc.c,
            src/ex_cmds.c, src/ex_cmds.h, src/ex_cmds2.c, src/ex_docmd.c,
            src/ex_getln.c, src/normal.c, src/ops.c, src/proto/autocmd.pro,
            src/proto/ex_cmds2.pro, src/proto/ex_docmd.pro,
            src/proto/ex_getln.pro, src/proto/userfunc.pro,
            src/testdir/test_let.vim, src/testdir/test_startup.vim,
            src/userfunc.c

Patch 8.1.1589
问题: Popup window does not indicate scroll position.
解决方案: Add a scrollbar.
文件: runtime/doc/popup.txt, src/popupwin.c, src/structs.h,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_firstline.dump,
            src/testdir/dumps/Test_popupwin_scroll_1.dump,
            src/testdir/dumps/Test_popupwin_scroll_2.dump,
            src/testdir/dumps/Test_popupwin_scroll_3.dump,
            src/testdir/dumps/Test_popupwin_scroll_4.dump

Patch 8.1.1590
问题: Popup window test fails.
解决方案: Add "scrollbar" to expected result.
文件: src/testdir/test_popupwin.vim

Patch 8.1.1591
问题: On error garbage collection may free memory in use.
解决方案: Reset may_garbage_collect when evaluating expression mapping.
            Add tests. (Ozaki Kiichi, closes #4579)
文件: src/ex_cmds2.c, src/getchar.c, src/testdir/test_mapping.vim,
            src/testdir/test_timers.vim, src/testdir/test_vimscript.vim

Patch 8.1.1592
问题: May start file dialog while exiting.
解决方案: Ignore the "browse" modifier when exiting. (Ozaki Kiichi,
            closes #4582
文件: src/ex_cmds.c, src/terminal.c

Patch 8.1.1593
问题: Filetype not detected for C++ header files without extension.
解决方案: Recognize the file by the Emacs file mode. (Dmitry Ilyin,
            closes #4593)
文件: runtime/scripts.vim, src/testdir/test_filetype.vim

Patch 8.1.1594
问题: May still start file dialog while exiting.
解决方案: Ignore the "browse" modifier in another place when exiting.
            (Ozaki Kiichi, closes #4582)
文件: src/ex_cmds.c

Patch 8.1.1595
问题: MS-Windows with VIMDLL: colors wrong in the GUI.

解决方案: Do not set the terminal colors when not using the GUI. (Ken
           Takata, closes #4588)
文件: src/syntax.c

Patch 8.1.1596
问题: When resizing the screen may draw popup in wrong position. (Masato
           Nishihata)
解决方案: Check the popup is not outside of the screen. (fixes #4592)
文件: src/popupwin.c

Patch 8.1.1597
问题: Cannot scroll a popup window with the mouse.
解决方案: If the popup window has a scrollbar let the mouse scroll wheel
           scroll the window.
文件: runtime/doc/popup.txt, src/normal.c, src/popupwin.c, src/screen.c,
           src/testdir/dumps/Test_popupwin_firstline.dump,
           src/testdir/dumps/Test_popupwin_scroll_1.dump,
           src/testdir/dumps/Test_popupwin_scroll_2.dump,
           src/testdir/dumps/Test_popupwin_scroll_3.dump,
           src/testdir/dumps/Test_popupwin_scroll_5.dump,
           src/testdir/dumps/Test_popupwin_scroll_6.dump,
           src/testdir/dumps/Test_popupwin_scroll_7.dump

Patch 8.1.1598
问题: Update to test file missing.
解决方案: Update the popup window test file.
文件: src/testdir/test_popupwin.vim

Patch 8.1.1599
问题: Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案: Add a dummy assignment.
文件: src/popupwin.c, src/normal.c

Patch 8.1.1600
问题: Cannot specify highlighting for popup window scrollbar.
解决方案: Add "scrollbarhighlight" and "thumbhighlight" options.
文件: src/popupwin.c, src/structs.h, src/window.c,
           src/testdir/dumps/Test_popupwin_scroll_5.dump,
           src/testdir/dumps/Test_popupwin_scroll_6.dump,
           src/testdir/dumps/Test_popupwin_scroll_7.dump

Patch 8.1.1601
问题: Missing changes to popup window test file.
解决方案: Add those changes.
文件: src/testdir/test_popupwin.vim

Patch 8.1.1602
问题: Popup window cannot overflow on the left or right.
解决方案: Only set the "fixed" option when it is in the dict. Set w_leftcol
           to allow for the popup overflowing on the left and use it when
           applying the mask.
文件: src/popupwin.c

Patch 8.1.1603

问题: Crash when using unknown highlighting in text property.
解决方案: Check for zero highlight ID.
文件: src/screen.c, src/testdir/test_textprop.vim

Patch 8.1.1604
问题: Popup window scroll test is flaky.
解决方案: Add a delay between scroll events.
文件: src/testdir/test_popupwin.vim

Patch 8.1.1605
问题: Vim may delay processing messages on a json channel. (Pontus
            Leitzler)
解决方案: Try parsing json when checking if there is readahead.
文件: src/channel.c

Patch 8.1.1606
问题: On a narrow screen ":hi" output is confusing.
解决方案: Insert a space between highlight group name and "xxx". (Masato
            Nishihaga, closes #4599)
文件: src/syntax.c, src/testdir/test_highlight.vim

Patch 8.1.1607
问题: Popup window scrollbar does not respond to click.
解决方案: Mouse click in scrollbar scrolls by one line.
文件: src/popupwin.c, src/proto/popupwin.pro, src/structs.h, src/ui.c,
            src/normal.c, runtime/doc/popup.txt,
            src/testdir/dumps/Test_popupwin_scroll_8.dump,
            src/testdir/dumps/Test_popupwin_scroll_9.dump

Patch 8.1.1608
问题: The evalfunc.c file is too big.
解决方案: Move sign functionality to sign.c.
文件: src/evalfunc.c, src/proto/evalfunc.pro, src/sign.c,
            src/proto/sign.pro

Patch 8.1.1609
问题: The user cannot easily close a popup window.
解决方案: Add the "close" property. (mostly by Masato Nishihata,
            closes #4601)
文件: runtime/doc/popup.txt, src/popupwin.c, src/proto/popupwin.pro,
            src/structs.h, src/testdir/dumps/Test_popupwin_close_01.dump,
            src/testdir/dumps/Test_popupwin_close_02.dump,
            src/testdir/dumps/Test_popupwin_close_03.dump,
            src/testdir/test_popupwin.vim, src/ui.c

Patch 8.1.1610
问题: There is no way to add or load a buffer without side effects.
解决方案: Add the bufadd() and bufload() functions.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_functions.vim

Patch 8.1.1611
问题: Bufadd() reuses existing buffer without a name.
解决方案: When the name is empty always create a new buffer.

文件: src/evalfunc.c, src/testdir/test_functions.vim

Patch 8.1.1612
问题: Cannot show an existing buffer in a popup window.
解决方案: Support buffer number argument in popup_create().
文件: src/buffer.c, src/proto/buffer.pro, src/evalfunc.c,
            src/popupwin.c, src/vim.h, src/normal.c, src/screen.c, src/ui.c,
            src/window.c, src/testdir/test_popupwin.vim, runtime/doc/popup.txt

Patch 8.1.1613
问题: Popup window test fails with Athena and Motif.
解决方案: Compute the highlight attribute when the GUI is not active.
文件: src/syntax.c

Patch 8.1.1614
问题: 'numberwidth' can only go up to 10.
解决方案: Allow up to 20. (Charlie Stanton, closes #4584)
文件: runtime/doc/options.txt, src/option.c, src/screen.c,
            src/testdir/gen_opt_test.vim, src/testdir/test_options.vim

Patch 8.1.1615
问题: Crash when passing buffer number to popup_create(). (Yasuhiro
            Matsumoto)
解决方案: Initialize the window properly.
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1616
问题: Build failure with gcc on Amiga.
解决方案: Add missing header includes. (Ola Sder, closes #4603)
文件: src/os_amiga.h

Patch 8.1.1617
问题: No test for popup window with mask and position fixed.
解决方案: Add a couple of screenshots. Fix detected problems.
文件: src/popupwin.c, src/structs.h, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_mask_1.dump,
            src/testdir/dumps/Test_popupwin_mask_2.dump,
            src/testdir/dumps/Test_popupwin_mask_3.dump,
            src/testdir/dumps/Test_popupwin_mask_4.dump

Patch 8.1.1618
问题: Amiga-like systems quickly run out of stack.
解决方案: Reserve a Megabyte stack. (Ola Sder, closes #4608)
文件: src/os_amiga.c

Patch 8.1.1619
问题: Tests are not run with GUI on Travis.
解决方案: Add a testgui job. (Ozaki Kiichi, closes #4609)
文件: .travis.yml, src/testdir/test_highlight.vim,
            src/testdir/test_mapping.vim, src/testdir/test_timers.vim

Patch 8.1.1620
问题: No test for popup window with border and mask.
解决方案: Add this popup window, fix problems.

文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_mask_1.dump,
            src/testdir/dumps/Test_popupwin_mask_2.dump,
            src/testdir/dumps/Test_popupwin_mask_3.dump,
            src/testdir/dumps/Test_popupwin_mask_4.dump

Patch 8.1.1621
问题: Amiga: time.h included twice.
解决方案: Remove include from evalfunc.c, move outside of #ifdef in
            os_amiga.h. (Ola Sder, closes #4607)
文件: src/evalfunc.c, src/os_amiga.h

Patch 8.1.1622
问题: Wrong width if displaying a lot of lines in a popup window.
解决方案: Accurately compute the line overflow.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_firstline.dump

Patch 8.1.1623
问题: Display wrong with signs in narrow number column.
解决方案: Increase the numbercolumn width if needed. (Yegappan Lakshmanan,
            closes #4606)
文件: src/option.c, src/screen.c, src/sign.c, src/testdir/test_signs.vim

Patch 8.1.1624
问题: When testing in the GUI may try to run gvim in a terminal.
解决方案: Add the -v argument. (Yee Cheng Chin, closes #4605) Don't skip
            tests that work now.
文件: src/testdir/shared.vim, src/testdir/term_util.vim,
            src/testdir/test_mapping.vim, src/testdir/test_timers.vim

Patch 8.1.1625
问题: Script line numbers are not exactly right.
解决方案: Handle heredoc and continuation lines better. (Ozaki Kiichi,
            closes #4611, closes #4511)
文件: src/ex_cmds2.c, src/proto/ex_cmds2.pro,
            src/testdir/test_vimscript.vim, src/userfunc.c

Patch 8.1.1626
问题: No test for closing a popup window with a modified buffer.
解决方案: Add a test. Add "popups" to getbufinfo().
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_popupwin.vim

Patch 8.1.1627
问题: Header file contains mixed comment style.
解决方案: Use // style comments.
文件: src/structs.h

Patch 8.1.1628
问题: Popup window functions not in list of functions.
解决方案: Add popup window functions to the list of functions. Reorganise
            the popup window help.
文件: runtime/doc/eval.txt, runtime/doc/popup.txt,

runtime/doc/usr_41.txt

Patch 8.1.1629
问题: Terminal function help is in the wrong file.
解决方案: Move the function details to terminal.txt.
文件: runtime/doc/eval.txt, runtime/doc/terminal.txt

Patch 8.1.1630
问题: Various small problems.
解决方案: Various small improvements.
文件: src/gui_beval.c, src/list.c, src/menu.c, src/message.c,
src/misc2.c, src/testdir/test_terminal.vim, src/os_vms_conf.h,
src/testdir/Make_vms.mms

Patch 8.1.1631
问题: Displaying signs is inefficient.
解决方案: Avoid making multiple calls to get information about a placed
sign. (Yegappan Lakshmanan, closes #4586)
文件: src/proto/sign.pro, src/screen.c, src/sign.c, src/structs.h

Patch 8.1.1632
问题: Build with EXITFREE but without +arabic fails.
解决方案: Rename the function and adjust #ifdefs. (closes #4613)
文件: src/ex_getln.c, src/proto/ex_getln.pro, src/misc2.c

Patch 8.1.1633
问题: Cannot generate prototypes with X11 but without GUI.
解决方案: Include X11/Intrinsic.h.
文件: src/gui.h

Patch 8.1.1634
问题: Terminal test fails when term_getansicolors() is missing.
Diff test fails without +rightleft. (Dominique Pelle)
解决方案: Check if term_getansicolors() is supported. (closes #4597)
文件: src/testdir/test_terminal.vim, src/testdir/test_diffmode.vim

Patch 8.1.1635
问题: Warnings for unused variables in small version. (John Marriott)
解决方案: Adjust #ifdefs.
文件: src/screen.c

Patch 8.1.1636
问题: Crash when popup has fitting scrollbar. (Trygve Aaberge)
解决方案: Don't divide by zero if the scrollbar just fits. (closes #4615)
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1637
问题: After running tests and clean the XfakeHOME directory remains.
解决方案: Use "rm -rf". (Hirohito Higashi)
文件: src/testdir/Makefile, src/testdir/Make_amiga.mak

Patch 8.1.1638
问题: Running tests leaves some files behind.
解决方案: Delete the files. (Ozaki Kiichi, closes #4617)

文件: src/testdir/test_functions.vim, src/testdir/test_popupwin.vim

Patch 8.1.1639
问题: Changing an autoload name into a script file name is inefficient.
解决方案: Remember the last replaced #. (Ozaki Kiichi, closes #4618)
文件: src/eval.c

Patch 8.1.1640
问题: The CursorHold autocommand takes down a balloon. (Paul Jolly)
解决方案: Ignore the CursorHold pseudo-key.
文件: src/getchar.c, src/testdir/test_balloon.vim,
            src/testdir/dumps/Test_balloon_eval_term_01.dump,
            src/testdir/dumps/Test_balloon_eval_term_01a.dump

Patch 8.1.1641
问题: Garbage collection may run at a wrong moment. (Trygve Aaberge)
解决方案: Postpone garbage collection while parsing messages. (closes #4620)
文件: src/misc2.c

Patch 8.1.1642 (after 8.1.0374)
问题: May use uninitialized variable. (Patrick Palka)
解决方案: Initialize variables earlier. (closes #4623)
文件: src/screen.c, src/testdir/test_number.vim

Patch 8.1.1643
问题: Sign placement is wrong when 'foldcolumn' is set.
解决方案: Adjust the column computation. (Yee Cheng Chin, closes #4627)
文件: src/gui.c

Patch 8.1.1644
问题: Sound test does not work on Travis.
解决方案: Use "sg" command to enable audio. (Ozaki Kiichi, closes #4624)
文件: .travis.yml

Patch 8.1.1645
问题: Cannot use a popup window for a balloon.
解决方案: Add popup_beval(). Add the "mousemoved" property. Add the
            screenpos() function.
文件: src/popupwin.c, src/proto/popupwin.pro, src/move.c,
            src/proto/move.pro, src/beval.c, src/proto/beval.pro,
            src/evalfunc.c, src/popupmnu.c, src/normal.c,
            src/testdir/test_popupwin.vim, src/testdir/test_cursor_func.vim,
            runtime/doc/popup.txt, runtime/doc/eval.txt,
            runtime/doc/usr_41.txt,
            src/testdir/dumps/Test_popupwin_beval_1.dump,
            src/testdir/dumps/Test_popupwin_beval_2.dump,
            src/testdir/dumps/Test_popupwin_beval_3.dump

Patch 8.1.1646 (after 8.1.1645)
问题: build failure
解决方案: Add changes to structure.
文件: src/structs.h

Patch 8.1.1647

问题:　　Build error with GTK and hangulinput feature, im_get_status()
　　defined twice. (Dominique Pelle)
解决方案:　Adjust im_get_status(). (closes #4628)
文件:　　src/hangulin.c, src/mbyte.c

Patch 8.1.1648
问题:　　MS-Windows: build error with normal features.
解决方案:　Adjust #ifdef for find_word_under_cursor().
文件:　　src/beval.c, src/proto/beval.pro

Patch 8.1.1649
问题:　　Illegal memory access when closing popup window.
解决方案:　Get w_next before closing the window.
文件:　　src/popupwin.c

Patch 8.1.1650
问题:　　Warning for using uninitialized variable. (Tony Mechelynck)
解决方案:　Simplify the code by always using the mouse coordinates.
文件:　　src/beval.c

Patch 8.1.1651
问题:　　Suspend test is flaky on some systems.
解决方案:　Wait for the shell prompt to show. (Yee Cheng Chin, closes #4632)
文件:　　src/testdir/test_suspend.vim

Patch 8.1.1652
问题:　　GUI: popup window doesn't close on mouse movement. (Paul Jolly)
解决方案:　Generate mouse-move events when a popup window is visible.
文件:　　src/gui.c, src/globals.h

Patch 8.1.1653
问题:　　Ubsan warns for possibly passing NULL pointer.
解决方案:　Skip code when length is zero. (Dominique Pelle, closes #4631)
文件:　　src/channel.c

Patch 8.1.1654
问题:　　GUI: screen updates from 'balloonexpr' are not displayed.
解决方案:　Update the screen if needed. Also avoid the cursor being
　　displayed in the wrong position.
文件:　　src/beval.c

Patch 8.1.1655
问题:　　Popup window border drawn wrong with multi-byte char. (Marcin
　　Szamotulski)
解决方案:　Correct check in mb_fix_col(). (closes #4635)
文件:　　src/mbyte.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_24.dump

Patch 8.1.1656
问题:　　Popup window width is wrong when using Tabs. (Paul Jolly)
解决方案:　Count tabs correctly. (closes #4637)
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_11.dump

Patch 8.1.1657
问题:　　Terminal: screen updates from 'balloonexpr' are not displayed.
解决方案:　Update the screen if needed. Fix the word position for
"mousemoved".
文件:　　src/beval.c, src/proto/beval.pro, src/popupwin.c, src/normal.c,
src/proto/normal.pro

Patch 8.1.1658
问题:　　Debug statements included in patch.
解决方案:　Remove the debug statements.
文件:　　src/normal.c, src/popupwin.c

Patch 8.1.1659
问题:　　Popup window "mousemoved" values not correct.
解决方案:　Convert text column to mouse column.
文件:　　src/popupwin.c, runtime/doc/popup.txt

Patch 8.1.1660
问题:　　Assert_fails() does not fail inside try/catch.
解决方案:　Set trylevel to zero. (Ozaki Kiichi, closes #4639)
文件:　　src/eval.c, src/testdir/test_assert.vim

Patch 8.1.1661
问题:　　Cannot build with +textprop but without +balloon_eval.
解决方案:　Adjust #ifdefs. (closes #4645)
文件:　　src/proto.h

Patch 8.1.1662
问题:　　Cannot build uninstal.exe with some version of MinGW.
解决方案:　Add -lole32. (Rene Nyffenegger, closes #4646)
文件:　　src/Make_cyg_ming.mak

Patch 8.1.1663
问题:　　Compiler warning for using size_t.
解决方案:　Add type cast. (Mike Williams)
文件:　　src/popupwin.c

Patch 8.1.1664
问题:　　GUI resize may cause changing Rows at a bad time. (Dominique
Pelle)
解决方案:　Postpone resizing while updating the screen.
文件:　　src/term.c

Patch 8.1.1665
问题:　　Crash when popup window with mask is below the screen.
解决方案:　Correct boundary check.
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_mask_5.dump

Patch 8.1.1666
问题:　　Click in popup window scrollbar with border doesn't scroll.
解决方案:　Correct column for the border. (Naruhiko Nishino, closes #4650)
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_scroll_9.dump

Patch 8.1.1667
问题: Flags for Ex commands may clash with other symbols.
解决方案: Prepend with EX_.
文件: src/ex_cmds.h, src/evalfunc.c, src/ex_docmd.c, src/ex_getln.c,
            src/usercmd.c, src/syntax.c

Patch 8.1.1668
问题: Popup window test is a bit flaky on some systems.
解决方案: Clear the command line. (Naruhiko Nishino, closes #4656)
文件: src/testdir/test_popupwin.vim

Patch 8.1.1669
问题: Travis: test results section is closed even when some tests
            failed.
解决方案: Only close the section on success. (Daniel Hahler, closes #4659)
文件: .travis.yml

Patch 8.1.1670
问题: Sign column not always properly aligned.
解决方案: Use "col" only after it was calculated. (Yee Cheng Chin,
            closes #4649)
文件: src/gui.c

Patch 8.1.1671
问题: Copying a blob may result in it being locked.
解决方案: Reset v_lock. (Ken Takata, closes #4648)
文件: src/blob.c, src/testdir/test_blob.vim

Patch 8.1.1672 (after 8.1.1667)
问题: "make cmdidxs" doesn't work.
解决方案: Update macro names. (Naruhiko Nishino, closes #4660)
文件: src/create_cmdidxs.vim

Patch 8.1.1673
问题: Cannot easily find the popup window at a certain position.
解决方案: Add popup_locate().
文件: runtime/doc/popup.txt, src/evalfunc.c, src/popupwin.c,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim

Patch 8.1.1674
问题: Script to check a colorscheme can be improved.
解决方案: Match the whole group name. Don't warn for what is usually omitted.
文件: runtime/colors/tools/check_colors.vim

Patch 8.1.1675
问题: Listener list not correctly updated on listener_remove().
解决方案: Only set "prev" when not removing a listener. Return one if the
            listener was found and removed.
文件: src/change.c

Patch 8.1.1676
问题: "maxwidth" of popup window does not always work properly.
解决方案: Adjust the computation. (Naruhiko Nishino, closes #4653)

文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_menu_maxwidth_1.dump

Patch 8.1.1677
问题: Tests get stuck when running into an existing swapfile.
解决方案: Set v:swapchoice to "q" and report an error. (Daniel Hahler,
            closes #4644)
文件: src/testdir/runtest.vim

Patch 8.1.1678
问题: When using popup_menu() does not scroll to show the selected line.
解决方案: Scroll the text. (Naruhiko Nishino, closes #4651)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_menu_scroll_1.dump,
            src/testdir/dumps/Test_popupwin_menu_scroll_2.dump,
            src/testdir/dumps/Test_popupwin_menu_scroll_3.dump,
            src/testdir/dumps/Test_popupwin_menu_scroll_4.dump,
            src/testdir/dumps/Test_popupwin_menu_scroll_5.dump,
            src/testdir/dumps/Test_popupwin_menu_scroll_6.dump

Patch 8.1.1679
问题: Test using SwapExists autocommand file may fail.
解决方案: Remove the SwapExists autocommand.
文件: src/testdir/test_window_cmd.vim

Patch 8.1.1680
问题: The command table is not well aligned.
解决方案: Adjust indent.
文件: src/ex_cmds.h

Patch 8.1.1681
问题: Insert stray "{" when listener gets buffer line. (Paul Jolly)
解决方案: Flush the cached line after invoking listeners. (closes #4455)
文件: src/memline.c, src/testdir/test_listener.vim

Patch 8.1.1682
问题: Placing a larger number of signs is slow.
解决方案: Add functions for dealing with a list of signs. (Yegappan
            Lakshmanan, closes #4636)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/evalfunc.c,
            src/proto/sign.pro, src/sign.c, src/testdir/test_signs.vim

Patch 8.1.1683
问题: Dictionary with string keys is longer than needed.
解决方案: Use *{key: val} for literal keys.
文件: runtime/doc/eval.txt, src/eval.c, src/dict.c, src/proto/dict.pro,
            src/testdir/test_listdict.vim, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_07.dump,
            src/testdir/dumps/Test_popupwin_mask_2.dump,
            src/testdir/dumps/Test_popupwin_mask_3.dump,
            src/testdir/dumps/Test_popupwin_mask_4.dump,
            src/testdir/dumps/Test_popupwin_mask_5.dump,
            src/testdir/dumps/Test_popupwin_scroll_2.dump,
            src/testdir/dumps/Test_popupwin_scroll_3.dump,

src/testdir/dumps/Test_popupwin_scroll_4.dump

Patch 8.1.1684
问题: Profiling functionality is spread out.
解决方案: Put profiling functionality in profiler.c. (Yegappan Lakshmanan, closes #4666)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_dice.mak, src/Make_manx.mak, src/Make_morph.mak, src/Make_mvc.mak, src/Make_sas.mak, src/Make_vms.mms, src/Makefile, src/README.md, src/ex_cmds2.c, src/globals.h, src/profiler.c, src/proto.h, src/proto/ex_cmds2.pro, src/proto/profiler.pro, src/proto/userfunc.pro, src/structs.h, src/userfunc.c

Patch 8.1.1685
问题: Missing file in distributed file list.
解决方案: Add profiler.pro
文件: Filelist

Patch 8.1.1686
问题: "*" of "*{" is recognized as multiply operator. (Yasuhiro Matsumoto)
解决方案: Check for the "{".
文件: src/eval.c, src/testdir/test_listdict.vim

Patch 8.1.1687
问题: The evalfunc.c file is too big.
解决方案: Move testing support to a separate file.
文件: Filelist, src/evalfunc.c, src/eval.c, src/proto/eval.pro, src/testing.c, src/proto/testing.pro, src/Make_cyg_ming.mak, src/Make_morph.mak, src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md, src/proto.h

Patch 8.1.1688
问题: Old makefiles are no longer useful.
解决方案: Delete the makefiles, they most likely don't work anyway.
文件: Filelist, src/Make_dice.mak, src/Make_manx.mak, src/Make_sas.mak

Patch 8.1.1689
问题: Profiling code is spread out.
解决方案: Move more profiling code to profiler.c. (Yegappan Lakshmanan, closes #4668)
文件: src/ex_cmds2.c, src/profiler.c, src/proto/ex_cmds2.pro, src/proto/profiler.pro, src/proto/userfunc.pro, src/structs.h, src/userfunc.c

Patch 8.1.1690
问题: Default padding for popup window menu is too much.
解决方案: Only add padding left and right.
文件: runtime/doc/popup.txt, src/popupwin.c, src/testdir/dumps/Test_popupwin_menu_01.dump, src/testdir/dumps/Test_popupwin_menu_02.dump, src/testdir/dumps/Test_popupwin_menu_maxwidth_1.dump, src/testdir/dumps/Test_popupwin_menu_scroll_1.dump, src/testdir/dumps/Test_popupwin_menu_scroll_2.dump,

src/testdir/dumps/Test_popupwin_menu_scroll_3.dump,
src/testdir/dumps/Test_popupwin_menu_scroll_4.dump,
src/testdir/dumps/Test_popupwin_menu_scroll_5.dump,
src/testdir/dumps/Test_popupwin_menu_scroll_6.dump

Patch 8.1.1691
问题: Diff test fails on some systems. (Elimar Riesebieter)
解决方案: Add a term_wait() call.
文件: src/testdir/test_diffmode.vim

Patch 8.1.1692
问题: Using *{} for literal dict is not backwards compatible. (Yasuhiro Matsumoto)
解决方案: Use ~{} instead.
文件: runtime/doc/eval.txt runtime/doc/popup.txt, src/eval.c,
src/testdir/test_listdict.vim src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_07.dump,
src/testdir/dumps/Test_popupwin_mask_2.dump,
src/testdir/dumps/Test_popupwin_mask_3.dump,
src/testdir/dumps/Test_popupwin_mask_4.dump,
src/testdir/dumps/Test_popupwin_mask_5.dump,
src/testdir/dumps/Test_popupwin_scroll_2.dump,
src/testdir/dumps/Test_popupwin_scroll_3.dump,
src/testdir/dumps/Test_popupwin_scroll_4.dump

Patch 8.1.1693
问题: Syntax coloring and highlighting is in one big file.
解决方案: Move the highlighting to a separate file. (Yegappan Lakshmanan, closes #4674)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
src/globals.h, src/highlight.c, src/proto.h,
src/proto/highlight.pro, src/proto/syntax.pro, src/structs.h,
src/syntax.c

Patch 8.1.1694
问题: The RUN_VIM variable is longer than needed.
解决方案: Shorten RUN_VIM. (Daniel Hahler, closes #4643)
文件: src/testdir/Makefile, src/testdir/shared.vim

Patch 8.1.1695
问题: Windows 10: crash when cursor is at bottom of terminal.
解决方案: Position the cursor before resizing. (Yasuhiro Matsumoto, closes #4679)
文件: src/os_win32.c

Patch 8.1.1696
问题: MSVC: link command line is too long.
解决方案: Use the @<< mechanism to pass the arguments via a file. (Christian Brabandt)
文件: src/Make_mvc.mak

Patch 8.1.1697
问题: Cannot build with MSVC.

解决方案: Remove the backslashes after the @<< mechanism.
文件: src/Make_mvc.mak

Patch 8.1.1698
问题: Appveyor build with MSVC fails.
解决方案: Remove the sed command
文件: ci/appveyor.bat

Patch 8.1.1699
问题: Highlight_ga can be local instead of global.
解决方案: Move highlight_ga into highlight.c. (Yegappan Lakshmanan,
           closes #4675)
文件: src/globals.h, src/highlight.c, src/proto/highlight.pro,
           src/structs.h, src/syntax.c

Patch 8.1.1700
问题: Listener callback called for the wrong buffer.
解决方案: Invoke listeners before calling ml_append_int().
文件: src/memline.c

Patch 8.1.1701
问题: Appveyor build with MSVC fails puts progress bar in log.
解决方案: Adjust the sed command. (Ken Takata)
文件: ci/appveyor.bat

Patch 8.1.1702
问题: Compiler warning for uninitialized variable.
解决方案: Initialize it. (Christian Brabandt)
文件: src/gui.c

Patch 8.1.1703
问题: Breaking out of loop by checking window pointer is insufficient.
解决方案: Check the window ID and the buffer number. (closes #4683)
文件: src/misc2.c

Patch 8.1.1704
问题: C-R C-W does not work after C-G when using 'incsearch'.
解决方案: Put cursor at end of the match. (Yasuhiro Matsumoto, closes #4664)
文件: src/ex_getln.c, src/testdir/test_search.vim

Patch 8.1.1705
问题: Using ~{} for a literal dict is not nice.
解决方案: Use #{} instead.
文件: runtime/doc/eval.txt runtime/doc/popup.txt, src/eval.c,
           src/testdir/test_listdict.vim src/testdir/test_popupwin.vim

Patch 8.1.1706
问题: Typo in #ifdef.
解决方案: Change PROT to PROTO.
文件: src/beval.c

Patch 8.1.1707
问题: Coverity warns for possibly using a NULL pointer.
解决方案: Change the logic to make sure no NULL pointer is used.

文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1708
问题: Coverity warns for using uninitialized variable.
解决方案: Set the start col when col is set.
文件: src/beval.c

Patch 8.1.1709
问题: Coverity warns for possibly using a NULL pointer.
解决方案: Make sure no NULL pointer is used.
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1710
问题: Coverity found dead code.
解决方案: Remove merging of listener changes.
文件: src/change.c

Patch 8.1.1711
问题: Listener callback called at the wrong moment
解决方案: Invoke listeners before calling ml_delete_int(). (closes #4657)
文件: src/memline.c

Patch 8.1.1712
问题: Signs in number column cause text to be misaligned.
解决方案: Improve alignment. (Yasuhiro Matsumoto, closes #4694)
文件: src/screen.c, src/testdir/test_signs.vim

Patch 8.1.1713
问题: Highlighting cursor line only works with popup_menu().
解决方案: Add the "cursorline" property. (Naruhiko Nishino, closes #4671)
文件: runtime/doc/popup.txt, src/popupwin.c,
            src/testdir/dumps/Test_popupwin_cursorline_1.dump,
            src/testdir/dumps/Test_popupwin_cursorline_2.dump,
            src/testdir/dumps/Test_popupwin_cursorline_3.dump,
            src/testdir/dumps/Test_popupwin_cursorline_4.dump,
            src/testdir/dumps/Test_popupwin_cursorline_5.dump,
            src/testdir/dumps/Test_popupwin_cursorline_6.dump,
            src/testdir/dumps/Test_popupwin_menu_filter_1.dump,
            src/testdir/dumps/Test_popupwin_menu_filter_2.dump,
            src/testdir/dumps/Test_popupwin_menu_filter_3.dump,
            src/testdir/dumps/Test_popupwin_menu_filter_4.dump,
            src/testdir/test_popupwin.vim, src/vim.h

Patch 8.1.1714
问题: Cannot preview a file in a popup window.
解决方案: Add the 'previewpopup' option.
文件: runtime/doc/windows.txt, runtime/doc/options.txt, src/popupwin.c,
            src/proto/popupwin.pro, src/option.c, src/option.h, src/ex_cmds.c,
            src/testdir/dumps/Test_popupwin_previewpopup_1.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_2.dump,
            src/ex_docmd.c, src/testdir/gen_opt_test.vim

Patch 8.1.1715
问题: Emoji characters are seen as word characters for spelling. (Gautam

Iyer)
解决方案: Exclude class 3 from word characters.
文件: src/spell.c

Patch 8.1.1716
问题: Old style comments are wasting space
解决方案: Use new style comments in option header file. (closes #4702)
文件: src/option.h

Patch 8.1.1717
问题: Last char in menu popup window highlighted.
解决方案: Do not highlight an extra character twice.
文件: src/screen.c, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_menu_04.dump

Patch 8.1.1718
问题: Popup menu highlighting does not look good.
解决方案: Highlight the whole window line. Fix that sign line HL is not
           displayed in a window with a background color.
文件: src/popupwin.c, src/sign.c, src/proto/sign.pro, src/screen.c,
           src/testdir/dumps/Test_popupwin_menu_scroll_1.dump,
           src/testdir/dumps/Test_popupwin_menu_scroll_2.dump,
           src/testdir/dumps/Test_popupwin_menu_scroll_3.dump,
           src/testdir/dumps/Test_popupwin_menu_scroll_4.dump,
           src/testdir/dumps/Test_popupwin_menu_scroll_5.dump,
           src/testdir/dumps/Test_popupwin_menu_scroll_6.dump,
           src/testdir/dumps/Test_popupwin_menu_01.dump,
           src/testdir/dumps/Test_popupwin_menu_02.dump,
           src/testdir/dumps/Test_popupwin_menu_04.dump

Patch 8.1.1719
问题: Popup too wide when 'showbreak' is set.
解决方案: Set window width when computing line length. (closes #4701)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_showbreak.dump

Patch 8.1.1720
问题: Crash with very long %[] pattern. (Reza Mirzazade farkhani)
解决方案: Check for reg_toolong. (closes #4703)
文件: src/regexp.c, src/testdir/test_regexp_utf8.vim

Patch 8.1.1721
问题: Build failure with normal features without netbeans interface.
解决方案: Enable signs when using the text properties feature.
文件: src/feature.h

Patch 8.1.1722
问题: Error when scriptversion is 2 a making a dictionary access.
解决方案: Parse the subscript even when not evaluating the sub-expression.
           (closes #4704)
文件: src/eval.c, src/testdir/test_eval_stuff.vim

Patch 8.1.1723
问题: Heredoc assignment has no room for new features. (FUJIWARA Takuya)

解决方案: Require the marker does not start with a lower case character.
           (closes #4705)
文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_let.vim

Patch 8.1.1724
问题: Too much overhead checking for CTRL-C while processing text.
解决方案: Increase BREAKCHECK_SKIP. Remove the difference for when built
           with the GUI. (suggested by Andy Massimino, closes #4708)
文件: src/misc1.c, src/screen.c, src/feature.h

Patch 8.1.1725
问题: MS-Windows: E325 message may use incorrect date format.
解决方案: Convert strftime() result to 'encoding'. Also make the message
           translatable. (Ken Takata, closes #4685, closes #4681)
文件: src/memline.c

Patch 8.1.1726
问题: The eval.txt help file is too big.
解决方案: Split off testing support to testing.txt. Move function details
           to where the functionality is explained.
文件: runtime/doc/Makefile, runtime/doc/eval.txt,
           runtime/doc/testing.txt, runtime/doc/sign.txt,
           runtime/doc/textprop.txt, runtime/doc/help.txt,
           runtime/doc/channel.txt, runtime/doc/tags

Patch 8.1.1727
问题: Code for viminfo support is spread out.
解决方案: Move to code to viminfo.c. (Yegappan Lakshmanan, closes #4686)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
           src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/buffer.c,
           src/eval.c, src/ex_cmds.c, src/ex_docmd.c, src/globals.h,
           src/proto.h, src/proto/buffer.pro, src/proto/eval.pro,
           src/proto/ex_cmds.pro, src/proto/viminfo.pro, src/structs.h,
           src/viminfo.c

Patch 8.1.1728
问题: Wrong place for command line history viminfo support.
解决方案: Move it to viminfo.c.
文件: src/ex_getln.c, src/proto/ex_getln.pro, src/viminfo.c,
           src/structs.h

Patch 8.1.1729
问题: Heredoc with trim not properly handled in function.
解决方案: Allow for missing indent. (FUJIWARA Takuya, closes #4713)
文件: src/userfunc.c, src/testdir/test_let.vim

Patch 8.1.1730
问题: Wrong place for mark viminfo support.
解决方案: Move it to viminfo.c. (Yegappan Lakshmanan, closes #4716)
文件: src/README.md, src/mark.c, src/proto/mark.pro,
           src/proto/viminfo.pro, src/structs.h, src/viminfo.c

Patch 8.1.1731
问题: Command line history not read from viminfo on startup.

解决方案: Get history length after initializing it.
文件: src/viminfo.c, src/testdir/test_viminfo.vim

Patch 8.1.1732
问题: Completion in cmdwin does not work for buffer-local commands.
解决方案: Use the right buffer. (closes #4711)
文件: src/usercmd.c, src/testdir/test_ins_complete.vim

Patch 8.1.1733
问题: The man ftplugin leaves an empty buffer behind.
解决方案: Don't make new window and edit, use split. (Jason Franklin)
文件: runtime/ftplugin/man.vim, src/testdir/test_man.vim

Patch 8.1.1734
问题: The evalfunc.c file is too big.
解决方案: Move some functions to other files.
文件: src/evalfunc.c, src/proto/evalfunc.pro, src/json.c,
           src/proto/json.pro src/window.c, src/proto/window.pro,
           src/highlight.c, src/proto/highlight.pro, src/globals.h

Patch 8.1.1735 (after 8.1.1734)
问题: Can't build with tiny features.
解决方案: Add missing #ifdefs.
文件: src/json.c, src/highlight.c

Patch 8.1.1736
问题: Viminfo support is spread out.
解决方案: Move more viminfo code to viminfo.c. (Yegappan Lakshmanan,
           closes #4717) Reorder code to make most functions static.
文件: src/fileio.c, src/ops.c, src/option.c, src/proto/ops.pro,
           src/proto/option.pro, src/proto/search.pro, src/proto/viminfo.pro,
           src/search.c, src/structs.h, src/viminfo.c, src/ex_cmds.c,
           src/proto/ex_cmds.pro

Patch 8.1.1737
问题: :args command that outputs one line gives more prompt.
解决方案: Only output line break if needed. (Daniel Hahler, closes #4715)
文件: src/version.c, src/testdir/test_arglist.vim

Patch 8.1.1738
问题: Testing lambda with timer is slow.
解决方案: Do not test timer accuracy, only that it works. (Daniel Hahler,
           closes #4723)
文件: src/testdir/test_lambda.vim

Patch 8.1.1739
问题: Deleted match highlighting not updated in other window.
解决方案: Mark the window for refresh. (closes #4720) Also fix that
           ambi-width check clears with wrong attributes.
文件: src/term.c, src/highlight.c, src/testdir/test_match.vim,
           src/testdir/dumps/Test_matchdelete_1.dump

Patch 8.1.1740
问题: Exepath() doesn't work for "bin/cat".

解决方案: Check for any path separator. (Daniel Hahler, closes #4724,
           closes #4710)
文件: src/evalfunc.c, src/os_unix.c, src/testdir/test_functions.vim

Patch 8.1.1741
问题: Cleared/added match highlighting not updated in other window.
           (Andy Massimino)
解决方案: Mark the right window for refresh.
文件: src/highlight.c, src/testdir/test_match.vim,
           src/testdir/dumps/Test_matchclear_1.dump,
           src/testdir/dumps/Test_matchadd_1.dump

Patch 8.1.1742
问题: Still some match functions in evalfunc.c.
解决方案: Move them to highlight.c.
文件: src/evalfunc.c, src/highlight.c, src/proto/highlight.pro,
           src/ex_docmd.c

Patch 8.1.1743
问题: 'hlsearch' and match highlighting in the wrong place.
解决方案: Move highlighting from inside screen functions to highlight.c.
文件: src/screen.c, src/highlight.c, src/proto/highlight.pro

Patch 8.1.1744
问题: Build error without the conceal feature.
解决方案: Define variables also without the conceal feature.
文件: src/screen.c

Patch 8.1.1745
问题: Compiler warning for unused argument.
解决方案: Add UNUSED. Change comments to new style.
文件: src/highlight.c

Patch 8.1.1746
问题: ":dl" is seen as ":dlist" instead of ":delete".
解决方案: Do not use cmdidxs2[] if the length is 1. (closes #4721)
文件: src/ex_docmd.c, src/testdir/test_excmd.vim,
           src/testdir/Make_all.mak

Patch 8.1.1747
问题: Compiler warning for unused variables. (Tony Mechelynck)
解决方案: Add #ifdef.
文件: src/screen.c

Patch 8.1.1748 (after 8.1.1737)
问题: :args output is not aligned.
解决方案: Output a line break after the last item in a row.
文件: src/version.c

Patch 8.1.1749
问题: Coverity warns for using negative index.
解决方案: Move using index inside "if".
文件: src/viminfo.c

Patch 8.1.1750
问题: Depending on the terminal width :version may miss a line break.
解决方案: Add a line break when needed.
文件: src/version.c

Patch 8.1.1751
问题: When redrawing popups plines_win() may be called often.
解决方案: Pass a cache to mouse_comp_pos().
文件: src/ui.c, src/proto/ui.pro, src/beval.c, src/evalfunc.c,
            src/popupwin.c

Patch 8.1.1752
问题: Resizing hashtable is inefficient.
解决方案: Avoid resizing when the final size is predictable.
文件: src/hashtab.c, src/proto/hashtab.pro, src/popupwin.c

Patch 8.1.1753
问题: Use of popup window mask is inefficient.
解决方案: Precompute and cache the mask.
文件: src/popupwin.c

Patch 8.1.1754 (after 8.1.1753)
问题: Build failure.
解决方案: Add missing change to window struct.
文件: src/structs.h

Patch 8.1.1755
问题: Leaking memory when using a popup window mask.
解决方案: Free the cached mask.
文件: src/window.c

Patch 8.1.1756
问题: Autocommand that splits window messes up window layout.
解决方案: Disallow splitting a window while closing one. In ":all" give an
            error when moving a window will not work.
文件: src/buffer.c, src/window.c, src/testdir/test_window_cmd.vim

Patch 8.1.1757
问题: Text added with appendbufline() to another buffer isn't displayed.
解决方案: Update topline. (partly by Christian Brabandt, closes #4718)
文件: src/evalfunc.c, src/testdir/test_bufline.vim,
            src/testdir/dumps/Test_appendbufline_1.dump

Patch 8.1.1758
问题: Count of g$ not used correctly when text is not wrapped.
解决方案: Do use the count. (Christian Brabandt, closes #4729, closes #4566)
文件: src/normal.c, src/testdir/test_normal.vim

Patch 8.1.1759
问题: No mode char for terminal mapping from maparg().
解决方案: Check for TERMINAL mode. (closes #4735)
文件: src/getchar.c, src/testdir/test_maparg.vim

Patch 8.1.1760

问题: Extra line break for wrapping output of :args.
解决方案: Avoid the extra line break. (Daniel Hahler, closes #4737)
文件: src/version.c, src/testdir/test_arglist.vim

Patch 8.1.1761
问题: Filetype "vuejs" causes problems for some users.
解决方案: Rename to "vue".
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.1762
问题: Some filetype rules are in the wrong place.
解决方案: Move to the right place. Add a few more tests.
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.1763
问题: Evalfunc.c is still too big.
解决方案: Move dict and list functions to a better place.
文件: src/evalfunc.c, src/dict.c, src/proto/dict.pro, src/list.c,
            src/proto/list.pro, src/blob.c, src/proto/blob.pro

Patch 8.1.1764
问题: ":browse oldfiles" is not tested.
解决方案: Add a test.
文件: src/testdir/test_viminfo.vim

Patch 8.1.1765
问题: get(func, dict, def) does not work properly.
解决方案: Handle NULL dict better. (Takuya Fujiwara, closes #4734)
文件: src/evalfunc.c, src/testdir/test_getvar.vim,
            src/testdir/test_partial.vim

Patch 8.1.1766
问题: Code for writing session file is spread out.
解决方案: Put it in one file. (Yegappan Lakshmanan, closes #4728)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
            src/eval.c, src/ex_docmd.c, src/misc2.c, src/proto.h,
            src/proto/eval.pro, src/proto/misc2.pro, src/proto/session.pro,
            src/session.c

Patch 8.1.1767
问题: FEAT_SESSION defined separately.
解决方案: Make FEAT_SESSION depend on FEAT_EVAL.
文件: src/feature.h, src/session.c, src/evalfunc.c, src/ex_docmd.c,
            src/gui_gtk_x11.c, src/proto.h

Patch 8.1.1768
问题: Man plugin changes setting in current window.
解决方案: Set options later. (Jason Franklin)
文件: runtime/ftplugin/man.vim, src/testdir/test_man.vim

Patch 8.1.1769
问题: 'shellslash' is also used for completion.
解决方案: Add the 'completeslash' option. (Yasuhiro Matsumoto, closes #3612)

文件: runtime/doc/options.txt, src/ex_getln.c, src/insexpand.c,
            src/option.c, src/option.h, src/structs.h,
            src/testdir/test_ins_complete.vim

Patch 8.1.1770
问题: Cannot get the window ID of the popup preview window.
解决方案: Add popup_getpreview().
文件: src/evalfunc.c, src/popupwin.c, src/proto/popupwin.pro,
            runtime/doc/eval.txt, runtime/doc/popup.txt,
            src/testdir/dumps/Test_popupwin_previewpopup_3.dump

Patch 8.1.1771
问题: Options test fails on MS-Windows.
解决方案: Add correct and incorrect values for 'completeslash'.
文件: src/testdir/gen_opt_test.vim

Patch 8.1.1772
问题: Options test still fails on MS-Windows.
解决方案: Check buffer-local value of 'completeslash'.
文件: src/option.c

Patch 8.1.1773
问题: The preview popup window may be too far to the right.
解决方案: Keep it inside the screen. Also keep the close button and
            scrollbar visible if possible.
文件: src/popupwin.c, src/proto/popupwin.pro, src/ex_cmds.c,
            src/screen.c, src/vim.h, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_previewpopup_1.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_2.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_3.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_4.dump

Patch 8.1.1774
问题: Test is silently skipped.
解决方案: Throw "Skipped".
文件: src/testdir/test_ins_complete.vim

Patch 8.1.1775
问题: Error message may be empty in filetype test.
解决方案: Use v:exception instead. (Daniel Hahler, closes #4744)
文件: src/testdir/test_filetype.vim

Patch 8.1.1776
问题: Text added with a job to another buffer isn't displayed.
解决方案: Update topline after adding a line. (closes #4745)
文件: src/channel.c, src/testdir/test_channel.vim, src/testdir/check.vim,
            src/testdir/dumps/Test_job_buffer_scroll_1.dump

Patch 8.1.1777
问题: Useless checks for job feature in channel test.
解决方案: Remove the checks. Remove ch_log() calls.
文件: src/testdir/test_channel.vim

Patch 8.1.1778

问题:　　Not showing the popup window right border is confusing.
解决方案:　Also show the border when there is no close button. (closes #4747)
文件:　　src/popupwin.c, src/testdir/dumps/Test_popupwin_20.dump,
　　src/testdir/dumps/Test_popupwin_21.dump

Patch 8.1.1779
问题:　　Not showing the popup window right border is confusing.
解决方案:　Also show the border when 'wrap' is off. (closes #4747)
文件:　　src/popupwin.c, src/testdir/dumps/Test_popupwin_20.dump,
　　src/testdir/dumps/Test_popupwin_21.dump

Patch 8.1.1780
问题:　　Warning for file no longer available is repeated every time Vim is
　　focused. (Brian Armstrong)
解决方案:　Only give the message once. (closes #4748)
文件:　　src/fileio.c

Patch 8.1.1781
问题:　　Amiga: no builtin OS readable version info.
解决方案:　Add a "version" variable. (Ola Sder, closes #4753)
文件:　　src/os_amiga.c

Patch 8.1.1782
问题:　　MS-Windows: system() has temp file error with 'noshelltemp'.
解决方案:　Check s_dont_use_vimrun. (Ken Takata, closes #4754)
文件:　　src/os_win32.c

Patch 8.1.1783
问题:　　MS-Windows: compiler test may fail when using %:S.
解决方案:　Reset 'shellslash'.
文件:　　src/testdir/test_compiler.vim

Patch 8.1.1784
问题:　　MS-Windows: resolve() does not work if serial nr duplicated.
解决方案:　Use another method to get the full path. (Ken Takata, closes #4661)
文件:　　src/os_mswin.c

Patch 8.1.1785
问题:　　Map functionality mixed with character input.
解决方案:　Move the map functionality to a separate file. (Yegappan
　　Lakshmanan, closes #4740) Graduate the +localmap feature.
文件:　　Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
　　src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
　　src/buffer.c, src/feature.h, src/evalfunc.c, src/ex_docmd.c,
　　src/getchar.c, src/map.c, src/proto.h, src/proto/getchar.pro,
　　src/proto/map.pro, src/version.c, src/structs.h

Patch 8.1.1786
问题:　　Double click in popup scrollbar starts selection.
解决方案:　Ignore the double click.
文件:　　src/ui.c, src/popupwin.c, src/proto/popupwin.pro

Patch 8.1.1787
问题:　　Cannot resize a popup window.

解决方案: Allow for resizing by dragging the lower right corner.
文件: runtime/doc/popup.txt, src/popupwin.c, src/structs.h, src/vim.h,
           src/ui.c src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_drag_01.dump,
           src/testdir/dumps/Test_popupwin_drag_02.dump,
           src/testdir/dumps/Test_popupwin_drag_03.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_1.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_2.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_3.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_4.dump

Patch 8.1.1788 (after 8.1.1787)
问题: missing changes in proto file
解决方案: Update proto file.
文件: src/proto/popupwin.pro

Patch 8.1.1789
问题: Cannot see file name of preview popup window.
解决方案: Add the file name as the title.
文件: src/ex_cmds.c, src/popupwin.c, src/proto/popupwin.pro,
           src/fileio.c,
           src/testdir/dumps/Test_popupwin_previewpopup_1.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_2.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_3.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_4.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_5.dump

Patch 8.1.1790
问题: :mkvimrc is not tested.
解决方案: Add a test.
文件: src/testdir/test_mksession.vim

Patch 8.1.1791
问题: 'completeslash' also applies to globpath().
解决方案: Add the WILD_IGNORE_COMPLETESLASH flag. (test by Yasuhiro
           Matsumoto, closes #4760)
文件: src/testdir/test_ins_complete.vim, src/ex_getln.c, src/evalfunc.c,
           src/vim.h

Patch 8.1.1792
问题: The vgetorpeek() function is too long.
解决方案: Split off the part that handles mappings.
文件: src/getchar.c

Patch 8.1.1793
问题: Mixed comment style in globals.
解决方案: Use // comments where appropriate.
文件: src/globals.h

Patch 8.1.1794 (after 8.1.1792)
问题: Tests are flaky.
解决方案: Undo the change to vgetorpeek().
文件: src/getchar.c

Patch 8.1.1795
问题: No syntax HL after splitting windows with :bufdo. (Yasuhiro
            Matsumoto)
解决方案: Trigger Syntax autocommands in buffers that are active.
            (closes #4761)
文件: src/vim.h, src/option.c, src/ex_cmds2.c,
            src/testdir/test_syntax.vim

Patch 8.1.1796
问题: :argdo is not tested
解决方案: Add a test.
文件: src/testdir/test_arglist.vim

Patch 8.1.1797 (after 8.1.1794)
问题: The vgetorpeek() function is too long.
解决方案: Split off the part that handles mappings, with fix.
文件: src/getchar.c

Patch 8.1.1798
问题: Warning for unused variable in tiny version. (Tony Mechelynck)
解决方案: Move inside #ifdef. Reformat code.
文件: src/getchar.c

Patch 8.1.1799
问题: Cannot avoid mapping for a popup window.
解决方案: Add the "mapping" property, default TRUE.
文件: runtime/doc/popup.txt, src/getchar.c, src/popupwin.c, src/vim.h,
            src/proto/popupwin.pro, src/testdir/test_popupwin.vim

Patch 8.1.1800
问题: Function call functions have too many arguments.
解决方案: Pass values in a funcexe_T struct.
文件: src/eval.c, src/structs.h, src/userfunc.c, src/proto/userfunc.pro,
            src/list.c, src/regexp.c, src/terminal.c, src/change.c,
            src/ex_cmds2.c, src/popupwin.c, src/channel.c

Patch 8.1.1801
问题: Cannot build without the +eval feature.
解决方案: Always define funcexe_T.
文件: src/structs.h

Patch 8.1.1802
问题: Missing change to call_callback().
解决方案: Add missing change.
文件: src/sound.c

Patch 8.1.1803
问题: All builtin functions are global.
解决方案: Add the method call operator ->. Implemented for a limited number
            of functions.
文件: runtime/doc/eval.txt, src/eval.c, src/structs.h, src/userfunc.c,
            src/globals.h, src/evalfunc.c, src/proto/evalfunc.pro,
            src/testdir/test_method.vim, src/testdir/Make_all.mak

Patch 8.1.1804
问题: No test for display updating without a scroll region.
解决方案: Add a test.
文件: src/testdir/test_display.vim, src/testdir/check.vim,
            src/testdir/test_diffmode.vim,
            src/testdir/dumps/Test_scroll_no_region_1.dump,
            src/testdir/dumps/Test_scroll_no_region_2.dump,
            src/testdir/dumps/Test_scroll_no_region_3.dump

Patch 8.1.1805
问题: Au_did_filetype is declared twice.
解决方案: Remove it from autocmd.c. (closes #4767)
文件: src/globals.h, src/autocmd.c

Patch 8.1.1806
问题: Test for display updating doesn't check without statusline.
解决方案: Add screenshots without a status line.
文件: src/testdir/test_display.vim,
            src/testdir/dumps/Test_scroll_no_region_4.dump,
            src/testdir/dumps/Test_scroll_no_region_5.dump,
            src/testdir/dumps/Test_scroll_no_region_6.dump

Patch 8.1.1807
问题: More functions can be used as a method.
解决方案: Add append(), appendbufline(), assert_equal(), etc.
            Also add the :eval command.
文件: runtime/doc/eval.txt, runtime/doc/testing.txt, src/evalfunc.c,
            src/testdir/test_method.vim, src/ex_cmds.h, src/ex_eval.c,
            src/proto/ex_eval.pro, src/ex_cmdidxs.h

Patch 8.1.1808
问题: Build failure for tiny version.
解决方案: Define ex_eval to ex_ni. Clean up the ordering a bit.
文件: src/ex_docmd.c

Patch 8.1.1809
问题: More functions can be used as a method.
解决方案: Add has_key(), split(), str2list(), etc.
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_method.vim,
            src/testdir/test_diffmode.vim, src/testdir/test_syntax.vim,
            src/testdir/test_system.vim

Patch 8.1.1810
问题: Popup_getoptions() is missing an entry for "mapping".
解决方案: Add the entry.
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1811
问题: Popup window color cannot be set to "Normal".
解决方案: Check for non-empty 'wincolor' instead of zero attribute.
            (closes #4772)
文件: src/screen.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_20.dump,
            src/testdir/dumps/Test_popupwin_21.dump

Patch 8.1.1812
问题: Reading a truncated undo file hangs Vim.
解决方案: Check for reading EOF. (closes #4769)
文件: src/undo.c, src/testdir/test_undo.vim

Patch 8.1.1813
问题: ATTENTION prompt for a preview popup window.
解决方案: Close the popup window if aborting the buffer load. Avoid getting
            the ATTENTION dialog.
文件: src/tag.c, src/ex_cmds.c, src/memline.c, src/vim.h,
            runtime/doc/windows.txt

Patch 8.1.1814
问题: A long title in a popup window overflows.
解决方案: Truncate the title. (closes #4770)
文件: src/testdir/test_popupwin.vim, src/popupwin.c,
            src/testdir/dumps/Test_popupwin_longtitle_1.dump,
            src/testdir/dumps/Test_popupwin_longtitle_2.dump

Patch 8.1.1815
问题: Duplicating info for internal functions.
解决方案: Use one table to list internal functions.
文件: src/evalfunc.c

Patch 8.1.1816
问题: Cannot use a user defined function as a method.
解决方案: Pass the base as the first argument to the user defined function
            after "->". (partly by FUJIWARA Takuya)
文件: src/eval.c, src/userfunc.c, src/testdir/test_user_func.vim,
            src/testdir/test_autoload.vim,
            src/testdir/sautest/autoload/foo.vim

Patch 8.1.1817
问题: Github contribution text is incomplete.
解决方案: Update the text.
文件: CONTRIBUTING.md

Patch 8.1.1818
问题: Unused variable.
解决方案: Remove the variable. (Mike Williams)
文件: src/sound.c

Patch 8.1.1819
问题: :pedit does not work with a popup preview window.
解决方案: Avoid aborting with an error. (fixes #4777) Also double check
            that after prepare_tagpreview() the current window is not a
            popup window.
文件: src/ex_docmd.c, src/popupmenu.c, src/search.c, src/tag.c,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_previewpopup_6.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_7.dump,
            src/testdir/dumps/Test_popupwin_previewpopup_8.dump

Patch 8.1.1820
问题: Using expr->FuncRef() does not work.
解决方案: Make FuncRef work as a method.
文件: src/eval.c, src/userfunc.c, src/testdir/test_method.vim

Patch 8.1.1821
问题: No test for wrong number of method arguments.
解决方案: Add a test.
文件: src/testdir/test_method.vim

Patch 8.1.1822
问题: Confusing error message when range is not allowed.
解决方案: With ADDR_NONE give e_norange. Change e_invaddr to e_invrange for
            consistency.
文件: src/ex_docmd.c, src/globals.h, src/testdir/test_excmd.vim

Patch 8.1.1823
问题: Command line history code is spread out.
解决方案: Put the code in a new file. (Yegappan Lakshmanan, closes #4779)
            Also graduate the +cmdline_hist feature.
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
            src/cmdhist.c, src/ex_getln.c, src/proto.h, src/proto/cmdhist.pro,
            src/proto/ex_getln.pro, src/version.c, src/evalfunc.c,
            src/ex_cmds.c, src/ex_docmd.c, src/misc2.c, src/normal.c,
            src/ops.c, src/option.c, src/search.c, src/tag.c, src/usercmd.c,
            src/viminfo.c, src/feature.h, src/globals.h

Patch 8.1.1824
问题: Crash when correctly spelled word is very long. (Ben Kraft)
解决方案: Check word length before copying. (closes #4778)
文件: src/spell.c, src/testdir/test_spell.vim

Patch 8.1.1825
问题: Allocating more memory than needed for extended structs.
解决方案: Use offsetof() instead of sizeof(). (Dominique Pelle,
            closes #4785)
文件: src/dict.c

Patch 8.1.1826
问题: Tests use hand coded feature and option checks.
解决方案: Use the commands from check.vim in more tests.
文件: src/testdir/check.vim, src/testdir/shared.vim,
            src/testdir/test_autochdir.vim, src/testdir/test_autocmd.vim,
            src/testdir/test_balloon.vim, src/testdir/test_breakindent.vim,
            src/testdir/test_bufline.vim, src/testdir/test_cdo.vim,
            src/testdir/test_channel.vim, src/testdir/test_clientserver.vim,
            src/testdir/test_conceal.vim, src/testdir/test_cscope.vim,
            src/testdir/test_debugger.vim, src/testdir/test_filechanged.vim,
            src/testdir/test_fold.vim, src/testdir/test_functions.vim,
            src/testdir/test_gui.vim, src/testdir/test_gui_init.vim,
            src/testdir/test_highlight.vim, src/testdir/test_mapping.vim,
            src/testdir/test_match.vim, src/testdir/test_memory_usage.vim,
            src/testdir/test_options.vim, src/testdir/test_paste.vim,

src/testdir/test_popup.vim, src/testdir/test_search.vim,
src/testdir/test_signals.vim, src/testdir/test_startup.vim,
src/testdir/test_syntax.vim, src/testdir/test_termcodes.vim,
src/testdir/test_terminal.vim, src/testdir/test_timers.vim,
src/testdir/test_vimscript.vim

Patch 8.1.1827
问题: Allocating more memory than needed for extended structs.
解决方案: Use offsetof() instead of sizeof(). (Dominique Pelle,
closes #4786)
文件: src/getchar.c, src/regexp.c, src/sign.c, src/structs.h,
src/syntax.c, src/textprop.c, src/userfunc.c

Patch 8.1.1828
问题: Not strict enough checking syntax of method invocation.
解决方案: Check there is no white space inside ->method(.
文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_method.vim

Patch 8.1.1829
问题: Difference in screenshots.
解决方案: Update screenshots. Change checks in a few more tests.
(closes #4789)
文件: src/testdir/test_balloon_gui.vim,
src/testdir/test_shortpathname.vim,
src/testdir/test_windows_home.vim,
src/testdir/dumps/Test_popupwin_previewpopup_1.dump,
src/testdir/dumps/Test_popupwin_previewpopup_2.dump,
src/testdir/dumps/Test_popupwin_previewpopup_3.dump,
src/testdir/dumps/Test_popupwin_previewpopup_4.dump,
src/testdir/dumps/Test_popupwin_previewpopup_5.dump

Patch 8.1.1830
问题: Travis does not report error when tests fail.
解决方案: Explicitly do "exit 1".
文件: .travis.yml

Patch 8.1.1831
问题: Confusing skipped message.
解决方案: Drop "run" from "run start the GUI".
文件: src/testdir/check.vim

Patch 8.1.1832
问题: Win_execute() does not work in other tab. (Rick Howe)
解决方案: Take care of the tab. (closes #4792)
文件: src/testdir/test_execute_func.vim, src/evalfunc.c, src/window.c,
src/proto/window.pro

Patch 8.1.1833
问题: Allocating a bit too much when spellbadword() does not find a bad
word.
解决方案: Reset "len" when going to the next word. (Daniel Hahler,
closes #4788)
文件: src/evalfunc.c

Patch 8.1.1834
问题: Cannot use a lambda as a method.
解决方案: Implement ->{lambda}(). (closes #4768)
文件: runtime/doc/eval.txt, src/eval.c, src/testdir/test_method.vim

Patch 8.1.1835
问题: Cannot use printf() as a method.
解决方案: Pass the base as the second argument to printf().
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_method.vim

Patch 8.1.1836
问题: Inaccurate memory estimate for Amiga-like OS.
解决方案: Adjust #ifdef for AvailMem(). (Ola Sder, closes #4797)
文件: src/os_amiga.c

Patch 8.1.1837
问题: Popup test fails if clipboard is supported but not working.
解决方案: Add the "clipboard_working" feature. Also use Check commands
            instead of "if" and "throw". And remove stray ch_logfile().
文件: src/testdir/test_popupwin.vim, src/evalfunc.c,
            runtime/doc/eval.txt

Patch 8.1.1838
问题: There is :spellwrong and :spellgood but not :spellrare.
解决方案: Add :spellrare. (Martin Tournoij, closes #4291)
文件: runtime/doc/spell.txt, src/ex_cmdidxs.h, src/ex_cmds.h,
            src/normal.c, src/proto/spellfile.pro, src/spellfile.c,
            src/spell.h, src/testdir/Make_all.mak,
            src/testdir/test_normal.vim, src/testdir/test_spellfile.vim

Patch 8.1.1839
问题: Insufficient info when test fails because of screen size.
解决方案: Report the detected screen size.
文件: src/testdir/runtest.vim

Patch 8.1.1840
问题: Testing: WorkingClipboard() is not accurate.
解决方案: Check feature clipboard_working instead.
文件: src/testdir/shared.vim, src/testdir/test_paste.vim,
            src/testdir/test_quotestar.vim, src/testdir/test_termcodes.vim

Patch 8.1.1841
问题: No test for Ex shift commands.
解决方案: Add a test. (Dominique Pelle, closes #4801)
文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim,
            src/testdir/test_shift.vim

Patch 8.1.1842
问题: Test listed as flaky should no longer be flaky.
解决方案: Remove Test_popup_and_window_resize from the list of flaky tests.
            (Daniel Hahler, close #4807)
文件: src/testdir/runtest.vim

Patch 8.1.1843

问题:　　Might be freeing memory that was not allocated.
解决方案:　Have next_fenc() set the fenc_alloced flag. (closes #4804)
文件:　　src/fileio.c

Patch 8.1.1844
问题:　　Buffer no longer unloaded when adding text properties to it.
解决方案:　Do not create the memfile. (closes #4808)
文件:　　runtime/doc/textprop.txt, src/testdir/test_textprop.vim,
　　src/textprop.c

Patch 8.1.1845
问题:　　May use NULL pointer when running out of memory.
解决方案:　Do not clear popup buffers when NULL. (closes #4802)
文件:　　src/screen.c

Patch 8.1.1846
问题:　　Inconsistently using GetVimCommand() and v:progpath. (Daniel
　　Hahler)
解决方案:　Use GetVimCommand(). (closes #4806)
文件:　　src/testdir/test_autocmd.vim, src/testdir/test_gui.vim,
　　src/testdir/test_normal.vim, src/testdir/test_profile.vim,
　　src/testdir/test_suspend.vim, src/testdir/test_system.vim,
　　src/testdir/test_vimscript.vim

Patch 8.1.1847
问题:　　Suspend test is failing.
解决方案:　Do not use GetVimCommandClean().
文件:　　src/testdir/test_suspend.vim

Patch 8.1.1848
问题:　　'langmap' is not used for CTRL-W command in terminal.
解决方案:　Push the command in the typeahead buffer instead of the stuff
　　buffer. (closes #4814)
文件:　　src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.1849
problem:　Some insert complete functions in the wrong file.
解决方案:　Move complete functions to insexpand.c. (Yegappan Lakshmanan,
　　closes #4815)
文件:　　src/evalfunc.c, src/insexpand.c, src/proto/insexpand.pro

Patch 8.1.1850
问题:　　Focus may remain in popup window.
解决方案:　Change focus if needed.
文件:　　src/popupmnu.c

Patch 8.1.1851
问题:　　Crash when sound_playfile() callback plays sound.
解决方案:　Invoke callback later from event loop.
文件:　　src/testdir/test_sound.vim, src/ui.c, src/sound.c,
　　src/proto/sound.pro, src/feature.h, src/os_unix.c, src/ex_docmd.c,
　　src/misc2.c

Patch 8.1.1852

问题: Timers test is flaky.
解决方案: Accept a larger count. Add test to list of flaky tests.
文件: src/testdir/test_timers.vim, src/testdir/runtest.vim

Patch 8.1.1853
问题: Timers test is still flaky.
解决方案: Compute the time to sleep more accurately.
文件: src/ex_docmd.c

Patch 8.1.1854
问题: Now another timer test is flaky.
解决方案: Add test to list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.1855
问题: Another failing timer test.
解决方案: Assert that timers are finished by the end of the test. Rename
            test functions to make them easier to find.
文件: src/testdir/test_timers.vim, src/testdir/runtest.vim

Patch 8.1.1856
问题: popup preview test fails sometimes. (Christian Brabandt)
解决方案: Clear the command line.
文件: src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_previewpopup_6.dump

Patch 8.1.1857
问题: Cannot use modifier with multi-byte character.
解决方案: Allow using a multi-byte character, although it doesn't work
            everywhere.
文件: src/misc2.c, src/testdir/test_mapping.vim

Patch 8.1.1858
问题: Test for multi-byte mapping fails on some systems.
解决方案: Test in another way.
文件: src/testdir/test_mapping.vim

Patch 8.1.1859
问题: Timer test sometimes fails on Mac.
解决方案: Show more info when it fails.
文件: src/testdir/test_timers.vim

Patch 8.1.1860
问题: Map timeout test is flaky.
解决方案: Add test to list of flaky tests. Increase timeout.
文件: src/testdir/runtest.vim, src/testdir/test_mapping.vim

Patch 8.1.1861
问题: Only some assert functions can be used as a method.
解决方案: Allow using most assert functions as a method.
文件: runtime/doc/testing.txt, src/evalfunc.c,
            src/testdir/test_assert.vim

Patch 8.1.1862

问题:　　Coverity warns for not using return value.
解决方案:　Add "(void)" to avoid the warning.
文件:　　src/normal.c

Patch 8.1.1863
问题:　　Confusing error when using a builtin function as method while it
　　does not support that.
解决方案:　Add a specific error message.
文件:　　src/vim.h, src/evalfunc.c, src/userfunc.c,
　　src/testdir/test_method.vim

Patch 8.1.1864
问题:　　Still a timer test that is flaky on Mac.
解决方案:　Adjust the sleep times.
文件:　　src/testdir/test_timers.vim

Patch 8.1.1865
问题:　　Spellrare and spellrepall in the wrong order.
解决方案:　Put spellrare below spellrepall. (closes #4820)
文件:　　runtime/doc/spell.txt, src/ex_cmds.h

Patch 8.1.1866
问题:　　Modeless selection in GUI does not work properly.
解决方案:　Avoid going beyond the end of the line. (closes #4783)
文件:　　src/ui.c

Patch 8.1.1867
问题:　　Still a timer test that is flaky on Mac.
解决方案:　Loop with a sleep instead of one fixed sleep.
文件:　　src/testdir/test_timers.vim

Patch 8.1.1868
问题:　　Multibyte characters in 'listchars' don't work correctly if
　　'linebreak' is also enabled. (Martin Tournoij)
解决方案:　Make it work correctly. (Christian Brabandt, closes #4822,
　　closes #4812)
文件:　　src/screen.c, src/testdir/test_listchars.vim

Patch 8.1.1869
问题:　　Code for the argument list is spread out.
解决方案:　Put argument list code in arglist.c. (Yegappan Lakshmanan,
　　closes #4819)
文件:　　Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
　　src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
　　src/arglist.c, src/buffer.c, src/evalfunc.c, src/ex_cmds2.c,
　　src/ex_docmd.c, src/proto.h, src/proto/arglist.pro,
　　src/proto/buffer.pro, src/proto/ex_cmds2.pro,
　　src/proto/ex_docmd.pro

Patch 8.1.1870
问题:　　Using :pedit from a help file sets the preview window to help
　　filetype. (Wang Shidong)
解决方案:　Do not set "keep_help_flag". (closes #3536)
文件:　　src/ex_docmd.c, src/testdir/test_window_cmd.vim

Patch 8.1.1871 (after 8.1.1866)
问题: Modeless selection in GUI still not correct.
解决方案: Fix max_col.
文件: src/ui.c

Patch 8.1.1872
问题: When Vim exits because of a signal, VimLeave is not triggered.
            (Daniel Hahler)
解决方案: Unblock autocommands when triggering VimLeave. (closes #4818)
文件: src/main.c

Patch 8.1.1873 (after 8.1.1872)
问题: Cannot build tiny version.
解决方案: Remove #ifdef for is_autocmd_blocked().
文件: src/autocmd.c

Patch 8.1.1874
问题: Modeless selection in popup window overlaps scrollbar.
解决方案: Subtract scrollbar from max_col. (closes #4773)
文件: src/ui.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_select_01.dump

Patch 8.1.1875
问题: Cannot get size and position of the popup menu.
解决方案: Add pum_getpos(). (Ben Jackson, closes #4827)
文件: runtime/doc/autocmd.txt, runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_popup.vim

Patch 8.1.1876
问题: proto file missing from distribution
解决方案: Add the file.
文件: Filelist

Patch 8.1.1877
问题: Graduated features scattered.
解决方案: Put graduated and obsolete features together.
文件: src/feature.h

Patch 8.1.1878
问题: Negative float before method not parsed correctly.
解决方案: Apply "!" and "-" in front of expression before using ->.
文件: src/eval.c, src/proto/eval.pro, src/userfunc.c,
            src/testdir/test_method.vim

Patch 8.1.1879
问题: More functions can be used as methods.
解决方案: Make float functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_float_func.vim

Patch 8.1.1880
问题: Cannot show extra info for completion in a popup window.
解决方案: Add the "popup" entry in 'completeopt'.

文件:　　runtime/doc/options.txt, src/popupmnu.c, src/ex_cmds.c,
src/proto/ex_cmds.pro, src/ex_docmd.c, src/search.c, src/tag.c,
src/popupwin.c, src/proto/popupwin.pro, src/option.c, src/vim.h,
src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_infopopup_1.dump,
src/testdir/dumps/Test_popupwin_infopopup_2.dump,
src/testdir/dumps/Test_popupwin_infopopup_3.dump,
src/testdir/dumps/Test_popupwin_infopopup_4.dump

Patch 8.1.1881
问题:　　Popup window test fails in some configurations.
解决方案:　Check that screendumps can be made.
文件:　　src/testdir/test_popupwin.vim

Patch 8.1.1882
问题:　　Cannot specify properties of the info popup window.
解决方案:　Add the 'completepopup' option. Default to PmenuSel highlight.
文件:　　runtime/doc/options.txt, runtime/doc/insert.txt, src/option.c,
src/popupwin.c, src/proto/popupwin.pro, src/option.h,
src/testdir/test_popupwin.vim, src/screen.c,
src/testdir/dumps/Test_popupwin_infopopup_1.dump,
src/testdir/dumps/Test_popupwin_infopopup_2.dump,
src/testdir/dumps/Test_popupwin_infopopup_3.dump

Patch 8.1.1883
问题:　　Options test fails.
解决方案:　Add entry for 'completepopup'.
文件:　　src/testdir/gen_opt_test.vim

Patch 8.1.1884
问题:　　Cannot use mouse scroll wheel in popup in Insert mode. Mouse
clicks in popup close the popup menu.
解决方案:　Check if the mouse is in a popup window. Do not let mouse events
close the popup menu. (closes #4544)
文件:　　src/edit.c, src/popupmnu.c, src/insexpand.c

Patch 8.1.1885
问题:　　Comments in libvterm are inconsistent.
解决方案:　Use // comments. Also update the table of combining characters.
文件:　　src/libvterm/bin/unterm.c, src/libvterm/bin/vterm-ctrl.c,
src/libvterm/bin/vterm-dump.c, src/libvterm/include/vterm.h,
src/libvterm/include/vterm_keycodes.h,
src/libvterm/src/encoding.c, src/libvterm/src/keyboard.c,
src/libvterm/src/mouse.c, src/libvterm/src/parser.c,
src/libvterm/src/pen.c, src/libvterm/src/rect.h,
src/libvterm/src/state.c, src/libvterm/src/unicode.c,
src/libvterm/src/utf8.h, src/libvterm/src/vterm.c,
src/libvterm/src/vterm_internal.h, src/libvterm/src/termscreen.c

Patch 8.1.1886
问题:　　Command line expansion code is spread out.
解决方案:　Move the code to cmdexpand.c. (Yegappan Lakshmanan, closes #4831)
文件:　　Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,

src/cmdexpand.c, src/evalfunc.c, src/ex_getln.c, src/proto.h,
src/proto/cmdexpand.pro, src/proto/ex_getln.pro, src/structs.h

Patch 8.1.1887
问题: The +cmdline_compl feature is not in the tiny version.
解决方案: Graduate the +cmdline_compl feature.
文件: src/cmdexpand.c, src/arglist.c, src/autocmd.c, src/buffer.c,
src/cmdhist.c, src/eval.c, src/evalfunc.c, src/ex_cmds2.c,
src/ex_docmd.c, src/ex_getln.c, src/feature.h, src/highlight.c,
src/if_cscope.c, src/map.c, src/menu.c, src/misc1.c, src/misc2.c,
src/option.c, src/sign.c, src/syntax.c, src/tag.c, src/term.c,
src/usercmd.c, src/userfunc.c, src/version.c, src/globals.h,
src/option.h, src/structs.h, runtime/doc/cmdline.txt

Patch 8.1.1888
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_vimscript.vim, src/testdir/test_balloon_gui.vim,
src/testdir/test_popup.vim, src/testdir/test_functions.vim,
src/testdir/test_hide.vim, src/testdir/test_arglist.vim

Patch 8.1.1889
问题: Coverity warns for using a NULL pointer.
解决方案: Use zero for column if pos is NULL.
文件: src/netbeans.c

Patch 8.1.1890
问题: Ml_get error when deleting fold marker.
解决方案: Check that the line number is not below the last line. Adjust the
fold when deleting the empty line. (Christian Brabandt,
closes #4834)
文件: src/fold.c, src/normal.c, src/testdir/test_fold.vim

Patch 8.1.1891
问题: Functions used in one file are global.
解决方案: Add "static". (Yegappan Lakshmanan, closes #4840)
文件: src/autocmd.c, src/buffer.c, src/change.c, src/channel.c,
src/charset.c, src/dict.c, src/digraph.c, src/eval.c,
src/ex_cmds.c, src/ex_eval.c, src/fileio.c, src/findfile.c,
src/getchar.c, src/gui.c, src/indent.c, src/json.c, src/list.c,
src/mark.c, src/menu.c, src/message.c, src/misc1.c, src/misc2.c,
src/ops.c, src/option.c, src/popupwin.c, src/profiler.c,
src/proto/autocmd.pro, src/proto/buffer.pro, src/proto/change.pro,
src/proto/channel.pro, src/proto/charset.pro, src/proto/dict.pro,
src/proto/eval.pro, src/proto/ex_cmds.pro, src/proto/ex_eval.pro,
src/proto/fileio.pro, src/proto/findfile.pro,
src/proto/getchar.pro, src/proto/gui.pro, src/proto/indent.pro,
src/proto/json.pro, src/proto/list.pro, src/proto/mark.pro,
src/proto/menu.pro, src/proto/message.pro, src/proto/misc1.pro,
src/proto/misc2.pro, src/proto/ops.pro, src/proto/option.pro,
src/proto/popupwin.pro, src/proto/profiler.pro,
src/proto/quickfix.pro, src/proto/spell.pro, src/proto/term.pro,
src/proto/textprop.pro, src/proto/ui.pro, src/proto/undo.pro,

src/proto/window.pro, src/quickfix.c, src/regexp.c, src/spell.c,
src/term.c, src/textprop.c, src/ui.c, src/undo.c, src/window.c

Patch 8.1.1892
问题: Missing index entry and option menu for 'completepopup'.
解决方案: Add the entries. Adjust #ifdefs to avoid dead code.
文件: runtime/doc/quickref.txt, runtime/optwin.vim, src/option.c,
src/option.h, src/popupwin.c

Patch 8.1.1893
问题: Script to summarize test results can be improved.
解决方案: Use "silent" for substitute to avoid reporting number of matches.
Remove duplicate "set nocp". (Daniel Hahler, closes #4845)
文件: src/testdir/summarize.vim

Patch 8.1.1894
问题: Not checking for out-of-memory of autoload_name().
解决方案: Check for NULL. (Dominique Pelle, closes #4846)
文件: src/eval.c

Patch 8.1.1895
问题: Using NULL pointer when out of memory.
解决方案: Bail out or skip the code using the pointer. (Zu-Ming Jiang,
closes #4805, closes #4843, closes #4939, closes #4844)
文件: src/message.c, src/highlight.c, src/buffer.c, src/ops.c

Patch 8.1.1896
问题: Compiler warning for unused variable.
解决方案: Add #ifdef. (John Marriott) Missing part of 8.1.1892.
文件: src/popupmnu.c

Patch 8.1.1897
问题: May free memory twice when out of memory.
解决方案: Check that backslash_halve_save() returns a different pointer.
(Dominique Pelle, closes #4847)
文件: src/cmdexpand.c, src/misc1.c

Patch 8.1.1898
问题: Crash when out of memory during startup.
解决方案: When out of memory message given during initialisation bail out.
(closes #4842)
文件: src/misc2.c

Patch 8.1.1899
问题: sign_place() does not work as documented.
解决方案: Make it accept line numbers like line(). (Yegappan Lakshmanan,
closes #4848)
文件: src/sign.c, src/testdir/test_signs.vim

Patch 8.1.1900
问题: Sign test fails in the GUI.
解决方案: Catch and ignore the exception.
文件: src/testdir/test_signs.vim

Patch 8.1.1901
问题: The +insert_expand feature is not always available.
解决方案: Graduate the +insert_expand feature.
文件: src/feature.h, src/autocmd.c, src/buffer.c, src/change.c,
            src/charset.c, src/edit.c, src/evalfunc.c, src/ex_cmds.c,
            src/ex_getln.c, src/getchar.c, src/gui.c, src/highlight.c,
            src/indent.c, src/insexpand.c, src/misc2.c, src/move.c,
            src/option.c, src/popupmnu.c, src/screen.c, src/search.c,
            src/spell.c, src/tag.c, src/term.c, src/userfunc.c, src/version.c,
            src/globals.h, src/option.h, src/proto.h, src/structs.h,
            src/vim.h, runtime/doc/change.txt, runtime/doc/index.txt,
            runtime/doc/insert.txt, runtime/doc/options.txt

Patch 8.1.1902
问题: Cannot have an info popup without a border.
解决方案: Add the "border" item to 'completepopup'.
文件: src/popupwin.c, src/proto/popupwin.pro, src/popupmnu.c,
            src/testdir/test_popupwin.vim, src/testdir/gen_opt_test.vim,
            src/testdir/dumps/Test_popupwin_infopopup_nb_1.dump

Patch 8.1.1903
问题: Cannot build without the +eval feature.
解决方案: Add missing #ifdefs
文件: src/insexpand.c, src/popupmnu.c

Patch 8.1.1904
问题: Cannot have an info popup align with the popup menu.
解决方案: Add the "align" item to 'completepopup'.
文件: src/popupwin.c, src/popupmnu.c, src/vim.h,
            runtime/doc/insert.txt, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_infopopup_align_1.dump,
            src/testdir/dumps/Test_popupwin_infopopup_align_2.dump,
            src/testdir/dumps/Test_popupwin_infopopup_align_3.dump

Patch 8.1.1905
问题: Cannot set all properties of the info popup.
解决方案: Add popup_findinfo(). Rename popup_getpreview() to
            popup_findpreview().
文件: src/popupwin.c, src/proto/popupwin.pro, src/ex_cmds.c,
            src/ex_docmd.c, src/popupmnu.c, src/evalfunc.c,
            runtime/doc/popup.txt, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_infopopup_align_3.dump

Patch 8.1.1906
问题: Info popup size is sometimes incorrect.
解决方案: Compute the position and size after setting the content.
文件: src/popupmnu.c

Patch 8.1.1907
问题: Wrong position for info popup with scrollbar on the left.
解决方案: Take the scrollbar into account.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_infopopup_5.dump,
            src/testdir/dumps/Test_popupwin_cursorline_3.dump,

src/testdir/dumps/Test_popupwin_cursorline_4.dump,
src/testdir/dumps/Test_popupwin_cursorline_5.dump,
src/testdir/dumps/Test_popupwin_cursorline_6.dump,
src/testdir/dumps/Test_popupwin_menu_filter_1.dump,
src/testdir/dumps/Test_popupwin_menu_filter_2.dump,
src/testdir/dumps/Test_popupwin_menu_filter_3.dump,
src/testdir/dumps/Test_popupwin_menu_filter_4.dump

Patch 8.1.1908
问题: Every popup window consumes a buffer number.
解决方案: Recycle buffers only used for popup windows. Do not list popup window buffers.
文件: src/popupwin.c, src/window.c, src/vim.h, src/buffer.c, src/proto/buffer.pro, src/ex_docmd.c, src/testdir/test_popupwin.vim

Patch 8.1.1909
问题: More functions can be used as methods.
解决方案: Make a few more functions usable as a method.
文件: runtime/doc/eval.txt, runtime/doc/testing.txt, src/evalfunc.c, src/testdir/test_popupwin.vim, src/testdir/test_bufwintabinfo.vim, src/testdir/test_bufline.vim, src/testdir/test_assert.vim

Patch 8.1.1910
问题: Redrawing too much when toggling 'relativenumber'.
解决方案: Only clear when 'signcolumn' is set to "number". (Yegappan Lakshmanan, closes #4852)
文件: src/option.c

Patch 8.1.1911
问题: More functions can be used as methods.
解决方案: Make a few more functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test69.in, src/testdir/test69.ok, src/testdir/test_functions.vim

Patch 8.1.1912
问题: More functions can be used as methods.
解决方案: Make channel and job functions usable as a method.
文件: runtime/doc/channel.txt, src/evalfunc.c, src/testdir/test_channel.vim

Patch 8.1.1913
问题: Not easy to compute the space on the command line.
解决方案: Add v:echospace. (Daniel Hahler, closes #4732)
文件: src/vim.h, src/eval.c, src/option.c, runtime/doc/eval.txt, src/testdir/test_messages.vim

Patch 8.1.1914
问题: Command line expansion code is spread out.
解决方案: Move set_one_cmd_context(). (Yegappan Lakshmanan, closes #4855)
文件: src/cmdexpand.c, src/ex_docmd.c, src/proto/ex_docmd.pro

Patch 8.1.1915
问题: More functions can be used as methods.

解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
           src/testdir/test_functions.vim, src/testdir/test_cd.vim,
           src/testdir/test_cindent.vim, src/testdir/test_match.vim,
           src/testdir/test_popup.vim, src/testdir/test_cursor_func.vim,
           src/testdir/test_method.vim, src/testdir/test_bufline.vim,
           src/testdir/test_diffmode.vim

Patch 8.1.1916
问题: Trying to allocate negative amount of memory when closing a popup.
解决方案: Check the rows are not out of bounds. Don't finish a selection if
           it was never started.
文件: src/ui.c

Patch 8.1.1917
问题: Non-current window is not redrawn when moving popup. (Ben Jackson)
解决方案: Redraw all windows under a popup. (closes #4860)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_drag_01.dump,
           src/testdir/dumps/Test_popupwin_drag_02.dump,
           src/testdir/dumps/Test_popupwin_drag_03.dump

Patch 8.1.1918
问题: Redrawing popups is inefficient.
解决方案: Fix the logic to compute what window lines to redraw. Make it
           work below the last line. Remove redrawing all windows.
文件: src/popupwin.c

Patch 8.1.1919
问题: Using current window option values when passing a buffer to
           popup_create().
解决方案: Clear the window-local options. (closes #4857)
文件: src/option.c, src/proto/option.pro, src/popupwin.c,
           src/testdir/test_popupwin.vim

Patch 8.1.1920
问题: Cannot close a popup by the X when a filter consumes all events.
解决方案: Check for a click on the close button before invoking filters.
           (closes #4858)
文件: src/popupwin.c, src/proto/popupwin.pro, src/ui.c,
           src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_close_04.dump,
           src/testdir/dumps/Test_popupwin_close_05.dump

Patch 8.1.1921
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_expand.vim,
           src/testdir/test_expand_func.vim, src/testdir/test_expr.vim,
           src/testdir/test_findfile.vim, src/testdir/test_fnameescape.vim,
           src/testdir/test_fnamemodify.vim, src/testdir/test_fold.vim,
           src/testdir/test_functions.vim, src/testdir/test_search.vim,
           src/testdir/test_vimscript.vim

Patch 8.1.1922
问题: In diff mode global operations can be very slow.
解决方案: Do not call diff_redraw() many times, call it once when redrawing.
And also don't update folds multiple times.
文件: src/globals.h, src/diff.c, src/proto/diff.pro, src/screen.c,
src/fold.c

Patch 8.1.1923
问题: Some source files are not in a normal encoding.
解决方案: Convert hangulin.c from euc-kr to utf-8 and digraph.c from latin1
to utf-8. (Daniel Hahler, closes #4731)
文件: src/hangulin.c, src/digraph.c, .travis.yml

Patch 8.1.1924
问题: Using empty string for current buffer is unexpected.
解决方案: Make the argument optional for bufname() and bufnr().
文件: src/evalfunc.c, src/testdir/test_arglist.vim, runtime/doc/eval.txt

Patch 8.1.1925
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_bufline.vim, src/testdir/test_bufwintabinfo.vim,
src/testdir/test_cd.vim, src/testdir/test_changelist.vim,
src/testdir/test_cmdline.vim, src/testdir/test_edit.vim,
src/testdir/test_environ.vim, src/testdir/test_file_perm.vim,
src/testdir/test_getvar.vim, src/testdir/test_jumplist.vim,
src/testdir/test_put.vim, src/testdir/test_stat.vim

Patch 8.1.1926
问题: Cursorline not redrawn when putting a line above the cursor.
解决方案: Redraw when the cursor line is below a change. (closes #4862)
文件: src/change.c

Patch 8.1.1927
问题: Code for dealing with script files is spread out.
解决方案: Move the code to scriptfile.c. (Yegappan Lakshmanan, closes #4861)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
src/cmdexpand.c, src/ex_cmds2.c, src/proto.h,
src/proto/ex_cmds2.pro, src/proto/scriptfile.pro, src/scriptfile.c

Patch 8.1.1928
问题: Popup windows don't move with the text when making changes.
解决方案: Add the 'textprop' property to the popup window options, position
the popup relative to a text property. (closes #4560)
No tests yet.
文件: runtime/doc/popup.txt, src/textprop.c, src/proto/textprop.pro,
src/structs.h, src/popupwin.c, src/proto/popupwin.pro, src/move.c,
src/proto/move.pro, src/window.c

Patch 8.1.1929
问题: No tests for text property popup window.
解决方案: Add a few tests.

文件: src/testdir/Make_all.mak, src/textprop.c,
            src/testdir/test_popupwin_textprop.vim,
            src/testdir/dumps/Test_popup_textprop_01.dump,
            src/testdir/dumps/Test_popup_textprop_02.dump,
            src/testdir/dumps/Test_popup_textprop_03.dump,
            src/testdir/dumps/Test_popup_textprop_04.dump,
            src/testdir/dumps/Test_popup_textprop_05.dump,
            src/testdir/dumps/Test_popup_textprop_06.dump

Patch 8.1.1930
问题: Cannot recognize .jsx and .tsx files.
解决方案: Recognize them as javascriptreact and typescriptreact.
            (closes #4830)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim,
            runtime/syntax/javascriptreact.vim,
            runtime/indent/javascriptreact.vim,
            runtime/ftplugin/javascriptreact.vim

Patch 8.1.1931 (after 8.1.1930)
问题: Syntax test fails.
解决方案: Add new javascriptreact type to completions.
文件: src/testdir/test_syntax.vim

Patch 8.1.1932
问题: Ml_get errors after using append(). (Alex Genco)
解决方案: Do not update the cursor twice. (closes #1737)
文件: src/evalfunc.c, src/testdir/test_functions.vim

Patch 8.1.1933
问题: The eval.c file is too big.
解决方案: Move code related to variables to evalvars.c. (Yegappan
            Lakshmanan, closes #4868)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
            src/eval.c, src/evalfunc.c, src/evalvars.c, src/globals.h,
            src/proto.h, src/proto/eval.pro, src/proto/evalvars.pro, src/vim.h

Patch 8.1.1934
问题: Not enough tests for text property popup window.
解决方案: Add a few more tests.
文件: src/popupwin.c, src/testdir/test_popupwin_textprop.vim,
            src/testdir/dumps/Test_popup_textprop_corn_1.dump,
            src/testdir/dumps/Test_popup_textprop_corn_2.dump,
            src/testdir/dumps/Test_popup_textprop_corn_3.dump,
            src/testdir/dumps/Test_popup_textprop_corn_4.dump

Patch 8.1.1935 (after 8.1.1934)
问题: Test for text property popup window is flaky.
解决方案: Remove the undo message
文件: src/testdir/test_popupwin_textprop.vim,
            src/testdir/dumps/Test_popup_textprop_corn_4.dump

Patch 8.1.1936
问题: Not enough tests for text property popup window.

解决方案: Add a few more tests. Make negative offset work. Close all
           popups when window closes.
文件: src/popupwin.c, src/testdir/test_popupwin_textprop.vim,
           src/testdir/dumps/Test_popup_textprop_07.dump,
           src/testdir/dumps/Test_popup_textprop_off_1.dump,
           src/testdir/dumps/Test_popup_textprop_off_2.dump,
           src/testdir/dumps/Test_popup_textprop_corn_5.dump,
           src/testdir/dumps/Test_popup_textprop_corn_6.dump

Patch 8.1.1937 (after 8.1.1930)
问题: Errors when using javascriptreact.
解决方案: Use ":runtime" instead of ":source". (closes #4875)
文件: runtime/syntax/javascriptreact.vim,
           runtime/indent/javascriptreact.vim,
           runtime/ftplugin/javascriptreact.vim

Patch 8.1.1938
问题: May crash when out of memory.
解决方案: Initialize v_type to VAR_UNKNOWN. (Dominique Pelle, closes #4871)
文件: src/userfunc.c

Patch 8.1.1939
问题: Code for handling v: variables in generic eval file.
解决方案: Move v: variables to evalvars.c. (Yegappan Lakshmanan,
           closes #4872)
文件: src/eval.c, src/evalvars.c, src/proto/eval.pro,
           src/proto/evalvars.pro

Patch 8.1.1940 (after 8.1.1939)
问题: Script tests fail.
解决方案: Don't set vimvars type in set_vim_var_nr().
文件: src/eval.c, src/evalvars.c, src/proto/evalvars.pro

Patch 8.1.1941
问题: getftype() test fails on Mac.
解决方案: Skip /dev/fd/.
文件: src/testdir/test_stat.vim

Patch 8.1.1942
问题: Shadow directory gets outdated when files are added.
解决方案: Add the "shadowupdate" target and add a few comments.
文件: src/Makefile

Patch 8.1.1943
问题: More code can be moved to evalvars.c.
解决方案: Move it, clean up comments. Also move some window related
           functions to window.c. (Yegappan Lakshmanan, closes #4874)
文件: src/eval.c, src/evalfunc.c, src/evalvars.c, src/proto/eval.pro,
           src/proto/evalvars.pro, src/proto/window.pro, src/window.c

Patch 8.1.1944
问题: Leaking memory when using sound callback.
解决方案: Free the callback queue item.
文件: src/sound.c

Patch 8.1.1945
问题: Popup window "firstline" cannot be reset.
解决方案: Allow for setting "firstline" to zero. Fix that the text jumps to
            the top when using win_execute(). (closes #4876)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_scroll_5.dump,
            src/testdir/dumps/Test_popupwin_scroll_6.dump

Patch 8.1.1946
问题: Memory error when profiling a function without a script ID.
解决方案: Check for missing script ID. (closes #4877)
文件: src/testdir/test_profile.vim, src/profiler.c

Patch 8.1.1947
问题: When executing one test the report doesn't show it.
解决方案: Adjust the regexp. (Daniel Hahler, closes #4879)
文件: src/testdir/summarize.vim

Patch 8.1.1948
问题: Mouse doesn't work in Linux console and causes 100% CPU. (James P.
            Harvey)
解决方案: Loop in WaitForCharOrMouse() when gpm_process_wanted is set.
            (closes #4828)
文件: src/os_unix.c

Patch 8.1.1949
问题: Cannot scroll a popup window to the very bottom.
解决方案: Scroll to the bottom when the "firstline" property was set to -1.
            (closes #4577) Allow resetting min/max width/height.
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/dict.c, src/proto/dict.pro,
            src/testdir/dumps/Test_popupwin_firstline.dump,
            src/testdir/dumps/Test_popupwin_firstline_1.dump,
            src/testdir/dumps/Test_popupwin_firstline_2.dump,
            src/testdir/dumps/Test_popupwin_scroll_10.dump

Patch 8.1.1950
问题: Using NULL pointer after an out-of-memory.
解决方案: Check for NULL pointer. (Dominique Pelle, closes #4881)
文件: src/syntax.c

Patch 8.1.1951
问题: Mouse double click test is a bit flaky.
解决方案: Add to list of flaky tests. Update a couple of comments.
文件: src/testdir/runtest.vim, src/testdir/shared.vim,
            src/testdir/test_substitute.vim

Patch 8.1.1952
问题: More functions can be used as a method.
解决方案: Allow more functions to be used as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_tagjump.vim, src/testdir/test_bufwintabinfo.vim,
            src/testdir/test_terminal.vim, src/testdir/test_getvar.vim,

src/testdir/test_escaped_glob.vim,
src/testdir/test_glob2regpat.vim

Patch 8.1.1953
问题: More functions can be used as a method.
解决方案: Allow more functions to be used as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_blob.vim,
src/testdir/test_breakindent.vim, src/testdir/test_delete.vim,
src/testdir/test_functions.vim, src/testdir/test_getcwd.vim,
src/testdir/test_history.vim, src/testdir/test_listdict.vim,
src/testdir/test_syn_attr.vim, src/testdir/test_termcodes.vim,
src/testdir/test_true_false.vim

Patch 8.1.1954
问题: More functions can be used as a method.
解决方案: Allow more functions to be used as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_arglist.vim, src/testdir/test_functions.vim,
src/testdir/test_json.vim, src/testdir/test_lispwords.vim,
src/testdir/test_listener.vim, src/testdir/test_lua.vim,
src/testdir/test_utf8.vim

Patch 8.1.1955
问题: Tests contain typos.
解决方案: Correct the typos. (Dominique Pelle)
文件: src/testdir/popupbounce.vim, src/testdir/runtest.vim,
src/testdir/screendump.vim, src/testdir/test49.vim,
src/testdir/test_autocmd.vim, src/testdir/test_cindent.vim,
src/testdir/test_const.vim, src/testdir/test_popupwin.vim,
src/testdir/test_quickfix.vim, src/testdir/test_search.vim,
src/testdir/test_tabpage.vim, src/testdir/test_tcl.vim

Patch 8.1.1956
问题: Screenshot tests may use a different encoding. (Dominique Pelle)
解决方案: Always set 'encoding' to "utf-8" when running Vim in a terminal.
(closes #4884)
文件: src/testdir/shared.vim, src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_behind.dump

Patch 8.1.1957
问题: More code can be moved to evalvars.c.
解决方案: Move code to where it fits better. (Yegappan Lakshmanan,
closes #4883)
文件: src/eval.c, src/evalvars.c, src/ex_getln.c, src/globals.h,
src/if_py_both.h, src/proto/eval.pro, src/proto/evalvars.pro,
src/proto/ex_getln.pro, src/proto/scriptfile.pro,
src/scriptfile.c, src/session.c, src/viminfo.c

Patch 8.1.1958
问题: Old style comments taking up space.
解决方案: Change to new style comments.
文件: src/vim.h

Patch 8.1.1959

问题:　　When using "firstline" in popup window text may jump when
redrawing it. (Nick Jensen)
解决方案:　Set 'scrolloff' to zero in a popup window. (closes #4882)
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim,
src/testdir/dumps/Test_popupwin_scroll_5.dump,
src/testdir/dumps/Test_popupwin_scroll_6.dump

Patch 8.1.1960
问题:　　Fold code is spread out.
解决方案:　Move fold functions to fold.c.
文件:　　src/evalfunc.c, src/fold.c, src/proto/fold.pro

Patch 8.1.1961
问题:　　More functions can be used as a method.
解决方案:　Allow more functions to be used as a method. Add a test for
mapcheck().
文件:　　runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test70.in,
src/testdir/test_functions.vim, src/testdir/test_getcwd.vim,
src/testdir/test_maparg.vim, src/testdir/test_match.vim

Patch 8.1.1962
问题:　　Leaking memory when using tagfunc().
解决方案:　Free the user_data. (Dominique Pelle, closes #4886)
文件:　　src/window.c

Patch 8.1.1963
问题:　　Popup window filter may be called recursively when using a Normal
mode command. (Nick Jensen)
解决方案:　Prevent recursiveness. (closes #4887) Also restore KeyTyped.
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.1964
问题:　　Crash when using nested map() and filter().
解决方案:　Do not set the v:key type to string without clearing the pointer.
(closes #4888)
文件:　　src/eval.c, src/testdir/test_filter_map.vim

Patch 8.1.1965
问题:　　The search count message is not displayed when using a mapping.
(Gary Johnson)
解决方案:　Ignore cmd_silent for showing the search count. (Christian
Brabandt)
文件:　　src/search.c

Patch 8.1.1966
问题:　　Some code in options.c fits better elsewhere.
解决方案:　Move functions from options.c to other files. (Yegappan
Lakshmanan, closes #4889)
文件:　　src/evalfunc.c, src/globals.h, src/indent.c, src/map.c,
src/option.c, src/proto/map.pro, src/proto/option.pro,
src/proto/quickfix.pro, src/proto/screen.pro, src/proto/spell.pro,
src/proto/window.pro, src/quickfix.c, src/screen.c, src/spell.c,
src/window.c

Patch 8.1.1967
问题: Line() only works for the current window.
解决方案: Add an optional argument for the window to use.
文件: runtime/eval.txt, src/evalfunc.c, src/testdir/test_popupwin.vim

Patch 8.1.1968
问题: Crash when using nested map().
解决方案: Clear the pointer in prepare_vimvar(). (Ozaki Kiichi,
            closes #4890, closes #4891)
文件: src/evalvars.c, src/testdir/test_filter_map.vim,
            src/testdir/test_functions.vim

Patch 8.1.1969
问题: Popup window filter is used in all modes.
解决方案: Add the "filtermode" property.
文件: src/popupwin.c, src/vim.h, src/map.c, src/proto/map.pro,
            src/structs.h, runtime/doc/popup.txt,
            src/testdir/test_popupwin.vim

Patch 8.1.1970
问题: Search stat space wrong, no test for 8.1.1965.
解决方案: Fix check for cmd_silent. Add a test. (Christian Brabandt)
文件: src/search.c, src/testdir/test_search_stat.vim

Patch 8.1.1971
问题: Manually enabling features causes build errors. (John Marriott)
解决方案: Adjust #ifdefs.
文件: src/proto.h, src/popupmnu.c, src/buffer.c, src/quickfix.c,
            src/ui.c

Patch 8.1.1972
问题: No proper test for getchar().
解决方案: Add a test with special characters.
文件: src/testdir/test_functions.vim

Patch 8.1.1973
问题: Cannot build without the quickfix feature.
解决方案: Remove #ifdef for qf_info_T.
文件: src/structs.h

Patch 8.1.1974
问题: Coverity warns for using pointer as array.
解决方案: Call var2fpos() directly instead of using f_line().
文件: src/evalfunc.c

Patch 8.1.1975
问题: MS-Windows GUI responds slowly to timer.
解决方案: Break out of wait loop when timer was added or input is available.
            (closes #4893)
文件: src/gui_w32.c

Patch 8.1.1976
问题: Travis log always shows test output.
解决方案: Change script to avoid if/else. (Ozaki Kiichi, closes #4892)

文件: .travis.yml

Patch 8.1.1977
问题: Terminal debugger plugin may hang.
解决方案: Wait longer when still reading symbols.
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.1978
问题: The eval.c file is too big.
解决方案: Move filter() and map() to list.c.
文件: src/eval.c, src/proto/eval.pro, src/list.c, src/proto/list.pro,
            src/evalfunc.c

Patch 8.1.1979
问题: Code for handling file names is spread out.
解决方案: Move code to new filepath.c file. Graduate FEAT_MODIFY_FNAME.
文件: src/filepath.c, Filelist, src/Make_cyg_ming.mak,
            src/Make_morph.mak, src/Make_mvc.mak, src/Make_vms.mms,
            src/Makefile, src/README.md, src/eval.c, src/evalfunc.c,
            src/ex_docmd.c, src/feature.h, src/findfile.c, src/if_cscope.c,
            src/message.c, src/misc1.c, src/proto.h, src/proto/eval.pro,
            src/proto/evalvars.pro src/proto/filepath.pro,
            src/proto/findfile.pro, src/proto/message.pro, src/regexp.c,
            src/version.c

Patch 8.1.1980
问题: Fix for search stat not tested.
解决方案: Add a screenshot test. (Christian Brabandt)
文件: src/testdir/test_search_stat.vim,
            src/testdir/dumps/Test_searchstat_1.dump,
            src/testdir/dumps/Test_searchstat_2.dump

Patch 8.1.1981
问题: The evalfunc.c file is too big.
解决方案: Move undo functions to undo.c. Move cmdline functions to
            ex_getln.c. Move some container functions to list.c.
文件: src/evalfunc.c, src/undo.c, src/proto/undo.pro, src/ex_getln.c,
            src/proto/ex_getln.pro, src/list.c, src/proto/list.pro

Patch 8.1.1982
问题: More functions can be used as methods.
解决方案: Make popup functions usable as a method.
文件: runtime/doc/popup.txt, src/evalfunc.c,
            src/testdir/test_popupwin.vim

Patch 8.1.1983
问题: Compiler nags for uninitialized variable and unused function.
解决方案: Add unnecessary initialization. Move function inside #ifdef.
文件: src/memline.c, src/channel.c

Patch 8.1.1984
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,

src/testdir/test_functions.vim, src/testdir/test_perl.vim,
src/testdir/test_prompt_buffer.vim, src/testdir/test_python2.vim,
src/testdir/test_python3.vim, src/testdir/test_pyx2.vim

Patch 8.1.1985
问题: Code for dealing with paths is spread out.
解决方案: Move path related functions from misc1.c to filepath.c.
Remove NO_EXPANDPATH.
文件: src/misc1.c, src/proto/misc1.pro, src/filepath.c,
src/evalfunc.c, src/globals.h, src/misc2.c, src/os_unix.c,
src/os_unix.h, src/proto/filepath.pro, src/scriptfile.c,
src/proto/misc2.pro, src/proto/scriptfile.pro, src/vim.h

Patch 8.1.1986
问题: More functions can be used as methods.
解决方案: Make textprop functions usable as a method.
文件: runtime/doc/textprop.txt, src/evalfunc.c,
src/testdir/test_textprop.vim

Patch 8.1.1987
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
src/testdir/test_clientserver.vim,
src/testdir/test_eval_stuff.vim, src/testdir/test_functions.vim,
src/testdir/test_reltime.vim, src/testdir/test_rename.vim

Patch 8.1.1988
问题: :startinsert! does not work the same way as "A".
解决方案: Use the same code to move the cursor. (closes #4896)
文件: src/ex_docmd.c, src/normal.c, src/proto/normal.pro,
src/testdir/test_edit.vim

Patch 8.1.1989
问题: The evalfunc.c file is still too big.
解决方案: Move f_pathshorten() to filepath.c. Move f_cscope_connection() to
if_cscope.c. Move diff_ functions to diff.c. Move timer_
functions to ex_cmds2.c. move callback functions to evalvars.c.
文件: src/evalfunc.c, src/proto/evalfunc.pro, src/filepath.c,
src/proto/filepath.pro, src/if_cscope.c, src/proto/if_cscope.pro,
src/diff.c, src/proto/diff.pro, src/ex_cmds2.c,
src/proto/ex_cmds2.pro, src/evalvars.c, src/proto/evalvars.pro

Patch 8.1.1990
问题: Cannot build with eval but without cscope.
解决方案: Always include if_cscope.pro.
文件: src/proto.h

Patch 8.1.1991
问题: Still cannot build with eval but without cscope.
解决方案: Move f_cscope_connection() outside of #ifdef.
文件: src/if_cscope.c

Patch 8.1.1992

问题: The search stat moves when wrapping at the end of the buffer.
解决方案: Put the "W" in front instead of at the end.
文件: src/search.c, src/testdir/test_search_stat.vim

Patch 8.1.1993
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_bufline.vim, src/testdir/test_charsearch.vim,
            src/testdir/test_clientserver.vim, src/testdir/test_cmdline.vim,
            src/testdir/test_cursor_func.vim, src/testdir/test_diffmode.vim,
            src/testdir/test_environ.vim, src/testdir/test_functions.vim,
            src/testdir/test_matchadd_conceal_utf8.vim,
            src/testdir/test_popupwin.vim, src/testdir/test_search.vim,
            src/testdir/test_searchpos.vim, src/testdir/test_utf8.vim

Patch 8.1.1994
问题: MS-Windows: cannot build with eval but without cscope
解决方案: Adjust the makefiles to always build if_cscope.obj.
文件: src/Make_mvc.mak, src/Make_cyg_ming.mak

Patch 8.1.1995
问题: More functions can be used as methods.
解决方案: Make sign functions usable as a method.
文件: runtime/doc/sign.txt, src/evalfunc.c, src/testdir/test_signs.vim

Patch 8.1.1996
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_bufwintabinfo.vim,
            src/testdir/test_cursor_func.vim, src/testdir/test_expr.vim,
            src/testdir/test_functions.vim, src/testdir/test_put.vim,
            src/testdir/test_quickfix.vim, src/testdir/test_sha256.vim,
            src/testdir/test_tabpage.vim, src/testdir/test_tagjump.vim,
            src/testdir/test_vartabs.vim

Patch 8.1.1997
问题: No redraw after a popup window filter is invoked.
解决方案: Redraw if needed.
文件: src/popupwin.c, src/testdir/test_popupwin.vim
            src/testdir/dumps/Test_popupwin_menu_filter_5.dump

Patch 8.1.1998
问题: Redraw even when no popup window filter was invoked.
解决方案: Only redraw when must_redraw was set to a larger value.
文件: src/popupwin.c

Patch 8.1.1999
问题: Calling both PlaySoundW() and PlaySoundA().
解决方案: Only use PlaySoundW(). (Dan Thompson, closes #4903)
文件: src/sound.c

Patch 8.1.2000

问题: Plugin cannot get the current IME status.
解决方案: Add the getimstatus() function. (closes #4904)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/mbyte.c,
            src/proto/mbyte.pro, src/testdir/test_iminsert.vim

Patch 8.1.2001
问题: Some source files are too big.
解决方案: Move buffer and window related functions to evalbuffer.c and
            evalwindow.c. (Yegappan Lakshmanan, closes #4898)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
            src/buffer.c, src/channel.c, src/evalbuffer.c, src/evalfunc.c,
            src/evalwindow.c, src/proto.h, src/proto/buffer.pro,
            src/proto/evalbuffer.pro, src/proto/evalfunc.pro,
            src/proto/evalwindow.pro, src/proto/window.pro, src/window.c

Patch 8.1.2002
问题: Version number 2000 missing.
解决方案: Add the number in the list of patches.
文件: src/version.c

Patch 8.1.2003
问题: MS-Windows: code page 65001 is not recognized.
解决方案: Use utf-8 for code page 65001. (Dan Thompson, closes #4902)
文件: src/mbyte.c

Patch 8.1.2004
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
            src/testdir/test_breakindent.vim, src/testdir/test_expr.vim,
            src/testdir/test_functions.vim, src/testdir/test_sound.vim,
            src/testdir/test_spell.vim, src/testdir/test_substitute.vim,
            src/testdir/test_swap.vim, src/testdir/test_utf8.vim

Patch 8.1.2005
问题: The regexp.c file is too big.
解决方案: Move the backtracking engine to a separate file. (Yegappan
            Lakshmanan, closes #4905)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_mvc.mak, src/Makefile,
            src/regexp.c, src/regexp_bt.c

Patch 8.1.2006
问题: Build failure with huge features but without channel feature.
解决方案: Add #ifdef. (Dominique Pelle, closes #4906)
文件: src/ui.c

Patch 8.1.2007
问题: No test for what 8.1.1926 fixes.
解决方案: Add a test case.
文件: src/testdir/test_highlight.vim

Patch 8.1.2008
问题: Error for invalid range when using listener and undo. (Paul Jolly)

解决方案: Do not change the cursor before the lines are restored.
           (closes #4908)
文件: src/undo.c, src/testdir/test_listener.vim

Patch 8.1.2009
问题: Cursorline highlighting not updated in popup window. (Marko
           Mahni)
解决方案: Check if the cursor position changed. (closes #4912)
文件: src/popupwin.c, src/structs.h, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_cursorline_7.dump

Patch 8.1.2010
问题: New file uses old style comments.
解决方案: Change to new style comments. (Yegappan Lakshmanan, closes #4910)
文件: src/regexp_bt.c

Patch 8.1.2011
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method. Make the window
           command test faster.
文件: runtime/doc/eval.txt, runtime/doc/testing.txt, src/evalfunc.c,
           src/testdir/test_assert.vim, src/testdir/test_gui.vim,
           src/testdir/test_messages.vim, src/testdir/test_options.vim,
           src/testdir/test_quickfix.vim, src/testdir/test_taglist.vim,
           src/testdir/test_termcodes.vim, src/testdir/test_timers.vim,
           src/testdir/test_vimscript.vim, src/testdir/test_viminfo.vim,
           src/testdir/test_window_cmd.vim

Patch 8.1.2012
问题: More functions can be used as methods.
解决方案: Make terminal functions usable as a method. Fix term_getattr().
文件: runtime/doc/terminal.txt, src/evalfunc.c, src/terminal.c
           src/testdir/test_mksession.vim, src/testdir/test_terminal.vim

Patch 8.1.2013
问题: More functions can be used as methods.
解决方案: Make various functions usable as a method.
文件: runtime/doc/eval.txt, src/evalfunc.c,
           src/testdir/test_cursor_func.vim,
           src/testdir/test_execute_func.vim, src/testdir/test_functions.vim,
           src/testdir/test_listchars.vim, src/testdir/test_timers.vim,
           src/testdir/test_undo.vim, src/testdir/test_window_cmd.vim,
           src/testdir/test_window_id.vim

Patch 8.1.2014
问题: Terminal altscreen test fails sometimes.
解决方案: Use WaitFor().
文件: src/testdir/test_terminal.vim

Patch 8.1.2015
问题: Terminal altscreen test still fails sometimes.
解决方案: Write the escape sequence in a file.
文件: src/testdir/test_terminal.vim

Patch 8.1.2016
问题:　　Terminal altscreen test now fails on MS-Windows.
解决方案:　Skip the test on MS-Windows
文件:　　src/testdir/test_terminal.vim

Patch 8.1.2017
问题:　　Cannot execute commands after closing the cmdline window.
解决方案:　Also trigger BufEnter and WinEnter. (closes #4762)
文件:　　runtime/doc/autocmd.txt, runtime/doc/cmdline.txt, src/ex_getln.c,
　　src/testdir/test_cmdline.vim

Patch 8.1.2018
问题:　　Using freed memory when out of memory and displaying message.
解决方案:　Make a copy of the message first.
文件:　　src/main.c, src/message.c, src/normal.c

Patch 8.1.2019
问题:　　'cursorline' always highlights the whole line.
解决方案:　Add 'cursorlineopt' to specify what is highlighted.
　　(Ozaki Kiichi, closes #4693)
文件:　　runtime/doc/options.txt, runtime/doc/quickref.txt,
　　runtime/doc/syntax.txt, runtime/optwin.vim, src/option.c,
　　src/option.h, src/screen.c, src/structs.h,
　　src/testdir/Make_all.mak, src/testdir/gen_opt_test.vim,
　　src/testdir/test_alot.vim, src/testdir/test_cursorline.vim

Patch 8.1.2020
问题:　　It is not easy to change the window layout.
解决方案:　Add win_splitmove(). (Andy Massimino, closes #4561)
文件:　　runtime/doc/eval.txt, src/evalfunc.c, src/evalwindow.c,
　　src/proto/evalwindow.pro, src/testdir/test_window_cmd.vim

Patch 8.1.2021
问题:　　Some global functions can be local to the file.
解决方案:　Add "static". (Yegappan Lakshmanan, closes #4917)
文件:　　src/ex_cmds2.c, src/filepath.c, src/hangulin.c, src/mbyte.c,
　　src/misc1.c, src/os_unix.c, src/proto/ex_cmds2.pro,
　　src/proto/filepath.pro, src/proto/hangulin.pro,
　　src/proto/mbyte.pro, src/proto/misc1.pro, src/proto/os_unix.pro,
　　src/proto/terminal.pro, src/proto/undo.pro, src/pty.c,
　　src/terminal.c, src/undo.c

Patch 8.1.2022
问题:　　The option.c file is too big.
解决方案:　Move option definitions to a separate file. (Yegappan Lakshmanan,
　　closes #4918)
文件:　　Filelist, src/Make_mvc.mak, src/Make_vms.mms, src/Makefile,
　　src/option.c, src/optiondefs.h

Patch 8.1.2023
问题:　　No test for synIDattr() returning "strikethrough".
解决方案:　Extend the synIDattr() test. (Jaskaran Singh, closes #4929)
文件:　　src/testdir/test_syn_attr.vim

Patch 8.1.2024
问题:　　Delete call commented out for debugging.
解决方案:　Restore the delete call. (Christian Brabandt)
文件:　　src/testdir/test_undo.vim

Patch 8.1.2025
问题:　　MS-Windows: Including shlguid.h causes problems for msys2.
解决方案:　Do not include shlguid.h. (closes #4913)
文件:　　src/GvimExt/gvimext.h

Patch 8.1.2026
问题:　　Possibly using uninitialized memory.
解决方案:　Check if "dict" is NULL. (closes #4925)
文件:　　src/ops.c

Patch 8.1.2027
问题:　　MS-Windows: problem with ambiwidth characters.
解决方案:　handle ambiguous width characters in ConPTY on Windows 10 (1903).
　　(Nobuhiro Takasaki, closes #4411)
文件:　　src/Make_mvc.mak, src/Make_cyg_ming.mak, src/libvterm/src/parser.c,
　　src/libvterm/src/state.c, src/libvterm/src/termscreen.c,
　　src/libvterm/src/unicode.c, src/libvterm/src/vterm_internal.h,
　　src/misc2.c, src/os_win32.c, src/proto/misc2.pro,
　　src/proto/os_win32.pro

Patch 8.1.2028
问题:　　Options test script does not work.
解决方案:　Use optiondefs.h for input.
文件:　　src/testdir/Makefile, src/testdir/Make_dos.mak,
　　src/testdir/Make_ming.mak

Patch 8.1.2029
问题:　　Cannot control 'cursorline' highlighting well.
解决方案:　Add "screenline". (Christian Brabandt, closes #4933)
文件:　　runtime/doc/options.txt, src/change.c, src/main.c, src/option.c,
　　src/option.h, src/optiondefs.h, src/screen.c, src/structs.h,
　　src/highlight.c, src/testdir/dumps/Test_Xcursorline_1.dump,
　　src/testdir/dumps/Test_Xcursorline_2.dump,
　　src/testdir/dumps/Test_Xcursorline_3.dump,
　　src/testdir/dumps/Test_Xcursorline_4.dump,
　　src/testdir/dumps/Test_Xcursorline_5.dump,
　　src/testdir/dumps/Test_Xcursorline_6.dump,
　　src/testdir/dumps/Test_Xcursorline_7.dump,
　　src/testdir/dumps/Test_Xcursorline_8.dump,
　　src/testdir/dumps/Test_Xcursorline_9.dump,
　　src/testdir/dumps/Test_Xcursorline_10.dump,
　　src/testdir/dumps/Test_Xcursorline_11.dump,
　　src/testdir/dumps/Test_Xcursorline_12.dump,
　　src/testdir/dumps/Test_Xcursorline_13.dump,
　　src/testdir/dumps/Test_Xcursorline_14.dump,
　　src/testdir/dumps/Test_Xcursorline_15.dump,
　　src/testdir/dumps/Test_Xcursorline_16.dump,
　　src/testdir/dumps/Test_Xcursorline_17.dump,
　　src/testdir/dumps/Test_Xcursorline_18.dump,

src/testdir/gen_opt_test.vim, src/testdir/test_cursorline.vim,
src/testdir/dumps/Test_cursorline_yank_01.dump,
src/testdir/dumps/Test_wincolor_01.dump,
src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.2030
问题: Tests fail when build with normal features and terminal.
(Dominique Pelle)
解决方案: Disable tests that won't work. (closes #4932)
文件: src/testdir/test_popupwin.vim, src/testdir/test_terminal.vim

Patch 8.1.2031
问题: Cursor position wrong when resizing and using conceal.
解决方案: Set the flags that the cursor position is valid when setting the
row and column during redrawing. (closes #4931)
文件: src/screen.c, src/testdir/test_conceal.vim,
src/testdir/dumps/Test_conceal_resize_01.dump,
src/testdir/dumps/Test_conceal_resize_02.dump

Patch 8.1.2032
问题: Scrollbar thumb wrong in popup window.
解决方案: Adjust thumb size and position when scrolled.
文件: src/popupwin.c, src/testdir/dumps/Test_popupwin_scroll_2.dump

Patch 8.1.2033
问题: Cannot build with tiny features.
解决方案: Add #ifdef.
文件: src/screen.c

Patch 8.1.2034
问题: Dark theme of GTK 3 not supported.
解决方案: Add the "d" flag in 'guioptions'. (Jonathan Conder, closes #4934)
文件: runtime/doc/options.txt, src/feature.h, src/gui.c,
src/gui_gtk_x11.c, src/option.h, src/proto/gui_gtk_x11.pro,
src/testdir/test_gui.vim

Patch 8.1.2035
问题: Recognizing octal numbers is confusing.
解决方案: Introduce scriptversion 4: do not use octal and allow for single
quote inside numbers.
文件: runtime/doc/eval.txt, src/vim.h, src/eval.c, src/scriptfile.c,
src/evalfunc.c, src/testdir/test_eval_stuff.vim,
src/testdir/test_functions.vim

Patch 8.1.2036 (after 8.1.2035)
问题: The str2nr() tests fail.
解决方案: Add missing part of patch.
文件: src/charset.c

Patch 8.1.2037
问题: Can call win_gotoid() in cmdline window.
解决方案: Disallow switching windows. (Yasuhiro Matsumoto, closes #4940)
文件: src/evalwindow.c, src/testdir/test_cmdline.vim

Patch 8.1.2038
问题:　　has('vimscript-4') is always 0.
解决方案:　Add "vimscript-4" to the feature table. (Naruhiko Nishino,
　　closes #4941)
文件:　　src/evalfunc.c, src/testdir/test_eval_stuff.vim

Patch 8.1.2039
问题:　　Character from 'showbreak' does not use 'wincolor'. (Nick Jensen)
解决方案:　Mix with 'wincolor'. (closes #4938)
文件:　　src/screen.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_showbreak.dump

Patch 8.1.2040
问题:　　No highlighting of current line in quickfix window.
解决方案:　Combine with line_attr.
文件:　　src/screen.c, src/testdir/test_quickfix.vim,
　　src/testdir/dumps/Test_quickfix_cwindow_1.dump,
　　src/testdir/dumps/Test_quickfix_cwindow_2.dump

Patch 8.1.2041 (after 8.1.2040)
问题:　　No test for diff mode with syntax highlighting.
解决方案:　Add a test case.
文件:　　src/testdir/test_diffmode.vim,
　　src/testdir/dumps/Test_diff_syntax_1.dump

Patch 8.1.2042
问题:　　The evalfunc.c file is too big.
解决方案:　Move getchar() and parse_queued_messages() to getchar.c.
文件:　　src/getchar.c, src/proto/getchar.pro, src/evalfunc.c, src/misc2.c,
　　src/proto/misc2.pro

Patch 8.1.2043
问题:　　Not sufficient testing for quoted numbers.
解决方案:　Add a few more test cases.
文件:　　src/testdir/test_functions.vim, src/testdir/test_eval_stuff.vim

Patch 8.1.2044
问题:　　No easy way to process postponed work. (Paul Jolly)
解决方案:　Add the SafeState autocommand event.
文件:　　runtime/doc/autocmd.txt, src/main.c, src/proto/main.pro,
　　src/vim.h, src/autocmd.c, src/channel.c, src/edit.c,
　　src/ex_getln.c

Patch 8.1.2045
问题:　　The option.c file is too big.
解决方案:　Split off the code dealing with strings. (Yegappan Lakshmanan,
　　closes #4937)
文件:　　Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
　　src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
　　src/option.c, src/option.h, src/optiondefs.h, src/optionstr.c,
　　src/ops.c, src/os_unix.c, src/proto.h, src/proto/option.pro,
　　src/proto/optionstr.pro

Patch 8.1.2046

问题: SafeState may be triggered at the wrong moment.
解决方案: Move it up higher to after where messages are processed. Add a
            SafeStateAgain event to trigger there.
文件: runtime/doc/autocmd.txt, src/main.c, src/proto/main.pro,
            src/getchar.c, src/channel.c, src/autocmd.c, src/vim.h

Patch 8.1.2047
问题: Cannot check the current state.
解决方案: Add the state() function.
文件: runtime/doc/eval.txt, src/misc1.c, src/proto/misc1.pro,
            src/evalfunc.c, src/proto/evalfunc.pro, src/main.c,
            src/proto/main.pro, src/channel.c, src/proto/channel.pro,
            src/userfunc.c, src/proto/userfunc.pro

Patch 8.1.2048
问题: Not clear why SafeState and SafeStateAgain are not triggered.
解决方案: Add log statements.
文件: src/getchar.c, src/main.c, src/proto/main.pro

Patch 8.1.2049 (after 8.1.2048)
问题: Cannot build tiny version.
解决方案: Add #ifdefs.
文件: src/main.c

Patch 8.1.2050
问题: Popup window test fails in some configurations. (James McCoy)
解决方案: Clear the command line.
文件: src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_scroll_10.dump

Patch 8.1.2051
问题: Double-click test is a bit flaky.
解决方案: Correct entry in list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.2052
问题: Using "x" before a closed fold may delete that fold.
解决方案: Do not translate 'x' do "dl". (Christian Brabandt, closes #4927)
文件: src/normal.c, src/testdir/test_fold.vim

Patch 8.1.2053
问题: SafeStateAgain not triggered if callback uses feedkeys().
解决方案: Check for safe state in the input loop. Make log messages easier
            to find. Add 'S' flag to state().
文件: src/main.c, src/proto/main.pro, src/getchar.c,
            runtime/doc/eval.txt

Patch 8.1.2054
问题: Compiler test for Perl may fail.
解决方案: Accept any error line number. (James McCoy, closes #4944)
文件: src/testdir/test_compiler.vim

Patch 8.1.2055
问题: Not easy to jump to function line from profile.

解决方案: Use "file:99" instead of "file line 99" so that "gf" works.
           (Daniel Hahler, closes #4951)
文件: src/profiler.c, src/testdir/test_profile.vim

Patch 8.1.2056
问题: "make test" for indent files doesn't cause make to fail.
解决方案: Exit the script with ":cquit". (Daniel Hahler, closes #4949)
文件: runtime/indent/testdir/runtest.vim, .gitignore

Patch 8.1.2057
问题: The screen.c file is much too big.
解决方案: Split it in three parts. (Yegappan Lakshmanan, closes #4943)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
           src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
           src/drawline.c, src/drawscreen.c, src/globals.h, src/proto.h,
           src/proto/drawline.pro, src/proto/drawscreen.pro,
           src/proto/screen.pro, src/screen.c, src/vim.h

Patch 8.1.2058
问题: Function for ex command is named inconsistently.
解决方案: Rename do_marks() to ex_marks().
文件: src/mark.c, src/proto/mark.pro, src/ex_cmds.h

Patch 8.1.2059
问题: Fix for "x" deleting a fold has side effects.
解决方案: Fix it where the fold is included.
文件: src/normal.c

Patch 8.1.2060
问题: "precedes" in 'listchars' not used properly.
解决方案: Correctly handle the "precedes" char in list mode for long lines.
           (Zach Wegner, Christian Brabandt, closes #4953)
文件: runtime/doc/options.txt, src/drawline.c,
           src/testdir/test_display.vim, src/testdir/view_util.vim

Patch 8.1.2061
问题: MS-Windows GUI: ":sh" crashes when trying to use a terminal.
解决方案: Check for a NULL command. (Yasuhiro Matsumoto, closes #4958)
文件: src/os_win32.c

Patch 8.1.2062
问题: The mouse code is spread out.
解决方案: Move all the mouse code to mouse.c. (Yegappan Lakshmanan,
           closes #4959)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
           src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
           src/auto/configure, src/configure.ac, src/edit.c, src/ex_cmds.c,
           src/ex_docmd.c, src/ex_getln.c, src/insexpand.c,
           src/libvterm/src/termmouse.c, src/libvterm/src/mouse.c,
           src/main.c, src/message.c, src/misc1.c, src/misc2.c, src/mouse.c,
           src/normal.c, src/proto.h, src/proto/edit.pro,
           src/proto/misc1.pro, src/proto/misc2.pro, src/proto/mouse.pro,
           src/proto/normal.pro, src/proto/term.pro, src/proto/ui.pro,
           src/search.c, src/term.c, src/ui.c, src/vim.h, src/window.c

Patch 8.1.2063
问题:　　Some tests fail when +balloon_eval_term is missing but
　　_balloon_eval is present. (Dominique Pelle)
解决方案:　Check the right feature in the test. (closes #4962)
文件:　　src/testdir/test_popupwin.vim, src/testdir/test_terminal.vim

Patch 8.1.2064
问题:　　MS-Windows: compiler warnings for unused arguments.
解决方案:　Add UNUSED. (Yegappan Lakshmanan, closes #4963)
文件:　　src/channel.c, src/ex_docmd.c, src/ex_getln.c, src/fileio.c,
　　src/gui_w32.c, src/main.c, src/memline.c, src/os_mswin.c,
　　src/os_win32.c, src/terminal.c, src/ui.c, src/undo.c

Patch 8.1.2065
问题:　　Compiler warning building non-GUI with MinGW.
解决方案:　Adjust #ifdefs. (Yegappan Lakshmanan, closes #4964)
文件:　　sre/mouse.c

Patch 8.1.2066
问题:　　No tests for state().
解决方案:　Add tests. Clean up some feature checks. Make "a" flag work.
文件:　　src/testdir/test_functions.vim, src/testdir/test_terminal.vim,
　　src/misc1.c

Patch 8.1.2067
问题:　　No tests for SafeState and SafeStateAgain.
解决方案:　Add tests.
文件:　　src/testdir/test_autocmd.vim

Patch 8.1.2068 (after 8.1.2067)
问题:　　Test for SafeState and SafeStateAgain may fail.
解决方案:　Accept more possible responses
文件:　　src/testdir/test_autocmd.vim

Patch 8.1.2069 (after 8.1.2068)
问题:　　Test for SafeStateAgain may still fail.
解决方案:　Send another message to trigger SafeStateAgain.
文件:　　src/testdir/test_autocmd.vim

Patch 8.1.2070
问题:　　Mouse code is spread out.
解决方案:　Move mouse terminal code parsing to mouse.c. (Yegappan Lakshmanan,
　　closes #4966)
文件:　　src/mouse.c, src/proto/mouse.pro, src/proto/term.pro, src/term.c

Patch 8.1.2071
问题:　　When 'wincolor' is set text property changes highlighting. (Andy
　　Stewart)
解决方案:　Fix combining colors. (closes #4968)
文件:　　src/drawline.c, src/testdir/test_highlight.vim,
　　src/testdir/dumps/Test_wincolor_01.dump

Patch 8.1.2072

问题:　　"gk" moves to start of line instead of upwards.
解决方案:　Fix off-by-one error. (Christian Brabandt, closes #4969)
文件:　　src/normal.c, src/testdir/test_normal.vim

Patch 8.1.2073
问题:　　When editing a buffer 'colorcolumn' may not work.
解决方案:　Set the buffer before copying option values. Call
check_colorcolumn() after copying window options.
文件:　　src/buffer.c, src/option.c, src/proto/option.pro,
src/proto/indent.pro, src/testdir/test_highlight.vim,
src/testdir/dumps/Test_colorcolumn_1.dump

Patch 8.1.2074
问题:　　Test for SafeState autocommand is a bit flaky.
解决方案:　Add to list of flaky tests.
文件:　　src/testdir/runtest.vim

Patch 8.1.2075
问题:　　Get many log messages when waiting for a typed character.
解决方案:　Do not repeat the repeated messages when nothing happens.
文件:　　src/globals.h, src/channel.c, src/main.c

Patch 8.1.2076
问题:　　Crash when trying to put a terminal buffer in a popup window.
解决方案:　Check for NULL buffer. Do not allow putting a terminal in a popup
window.
文件:　　src/libvterm/src/termscreen.c, src/terminal.c, src/popupwin.c,
runtime/doc/popup.txt, src/testdir/test_popupwin.vim

Patch 8.1.2077
问题:　　The ops.c file is too big.
解决方案:　Move code for dealing with registers to a new file. (Yegappan
Lakshmanan, closes #4982)
文件:　　Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_vms.mms src/Makefile, src/README.md,
src/ops.c, src/proto.h, src/proto/ops.pro, src/proto/register.pro,
src/register.c, src/structs.h

Patch 8.1.2078
问题:　　Build error with +textprop but without +terminal. (Tony Mechelynck)
解决方案:　Add #ifdef.
文件:　　src/popupwin.c

Patch 8.1.2079
问题:　　Popup window test fails without +terminal.
解决方案:　Check for the +terminal feature.
文件:　　src/testdir/test_popupwin.vim

Patch 8.1.2080
问题:　　The terminal API is limited and can't be disabled.
解决方案:　Add term_setapi() to set the function prefix. (Ozaki Kiichi,
closes #2907)
文件:　　runtime/doc/eval.txt, runtime/doc/terminal.txt, src/channel.c,
src/evalfunc.c, src/proto/terminal.pro, src/structs.h,

src/terminal.c, src/testdir/term_util.vim,
src/testdir/test_terminal.vim

Patch 8.1.2081
问题: The spell.c file is too big.
解决方案: Move the code for spell suggestions to a separate file. (Yegappan Lakshmanan, closes #4988)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak, src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md, src/proto.h, src/proto/spell.pro, src/proto/spellsuggest.pro, src/spell.c, src/spell.h, src/spellsuggest.c

Patch 8.1.2082
问题: Some files have a weird name to fit in 8.3 characters.
解决方案: Use a nicer names.
文件: Filelist, Makefile, src/popupmnu.c, src/popupmenu.c, src/proto/popupmnu.pro, src/proto/popupmenu.pro, src/Make_cyg_ming.mak, src/Make_morph.mak, src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/proto.h, src/README.md, src/uninstal.c, src/uninstall.c, uninstal.txt, uninstall.txt, nsis/gvim.nsi, src/INSTALLpc.txt, src/dosinst.c, src/dosinst.h

Patch 8.1.2083
问题: Multi-byte chars do not work properly with "%.*S" in printf().
解决方案: Use mb_ptr2cells(). Daniel Hahler, closes #4989)
文件: src/testdir/test_expr.vim, src/message.c

Patch 8.1.2084
问题: Amiga: cannot get the user name.
解决方案: Use getpwuid() if available. (Ola Sder, closes #4985)
文件: src/os_amiga.c, src/os_amiga.h

Patch 8.1.2085
问题: MS-Windows: draw error moving cursor over double-cell character.
解决方案: Move the cursor to the left edge if needed. (Nobuhiro Takasaki, closes #4986)
文件: src/os_win32.c

Patch 8.1.2086 (after 8.1.2082)
问题: Missing a few changes for the renamed files.
解决方案: Rename in a few more places. (Ken Takata)
文件: nsis/README.txt, runtime/doc/gui_w32.txt, runtime/doc/usr_90.txt, src/GvimExt/GvimExt.reg, src/GvimExt/README.txt, src/proto/popupmenu.pro, src/proto/popupmnu.pro

Patch 8.1.2087
问题: Cannot easily select one test function to execute.
解决方案: Support the $TEST_FILTER environment variable. (Ozaki Kiichi, closes #2695)
文件: src/Makefile, src/testdir/runtest.vim, src/testdir/summarize.vim

Patch 8.1.2088
问题: Renamed libvterm mouse.c file not in distributed file list.
解决方案: Rename the file in the file list.

文件: Filelist

Patch 8.1.2089 (after 8.1.2087)
问题: Do not get a hint that $TEST_FILTER was active.
解决方案: Mention $TEST_FILTER if no functions were executed.
文件: src/testdir/runtest.vim

Patch 8.1.2090
问题: Not clear why channel log file ends.
解决方案: Add a "closing" line.
文件: src/channel.c

Patch 8.1.2091
问题: Double free when memory allocation fails. (Zu-Ming Jiang)
解决方案: Use VIM_CLEAR() instead of vim_free(). (closes #4991)
文件: src/getchar.c

Patch 8.1.2092
问题: MS-Windows: redirect in system() does not work.
解决方案: Handle 'shellxescape' and 'shellxquote' better. (Yasuhiro
            Matsumoto, closes #2054)
文件: src/ex_cmds.c, src/misc2.c, src/testdir/test_system.vim

Patch 8.1.2093
问题: MS-Windows: system() test fails.
解决方案: Expect CR when using systemlist().
文件: src/testdir/test_system.vim

Patch 8.1.2094
问题: The fileio.c file is too big.
解决方案: Move buf_write() to bufwrite.c. (Yegappan Lakshmanan,
            closes #4990)
文件: Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
            src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
            src/bufwrite.c, src/fileio.c, src/option.c, src/proto.h,
            src/proto/bufwrite.pro, src/proto/fileio.pro, src/structs.h

Patch 8.1.2095
问题: Leaking memory when getting item from dict.
解决方案: Also free the key when not evaluating.
文件: src/dict.c

Patch 8.1.2096
问题: Too many #ifdefs.
解决方案: Graduate FEAT_COMMENTS.
文件: src/feature.h, src/buffer.c, src/change.c, src/edit.c,
            src/evalfunc.c, src/fold.c, src/insexpand.c, src/misc1.c,
            src/normal.c, src/ops.c, src/option.c, src/optionstr.c,
            src/search.c, src/version.c, src/globals.h, src/option.h,
            src/optiondefs.h, src/structs.h, runtime/doc/change.txt,
            runtime/doc/options.txt, runtime/doc/various.txt

Patch 8.1.2097
问题: :mksession is not sufficiently tested.

解决方案: Add more test cases. (Yegappan Lakshmanan, closes #4992)
文件: src/testdir/test_mksession.vim

Patch 8.1.2098 (after 8.1.2097)
问题: mksession test fails on MS-Windows.
解决方案: Skip testing with backslashes on MS-Windows.
文件: src/testdir/test_mksession.vim

Patch 8.1.2099
问题: state() test fails on some Mac systems.
解决方案: Increase the wait time. (closes #4983)
文件: src/testdir/test_functions.vim

Patch 8.1.2100
问题: :mksession is not sufficiently tested.
解决方案: Add more test cases. (Yegappan Lakshmanan, closes #4993)
文件: src/testdir/test_mksession.vim

Patch 8.1.2101
问题: write_session_file() often defined but not used.
解决方案: Adjust the #ifdef. (Yegappan Lakshmanan, closes #4998)
文件: src/session.c

Patch 8.1.2102
问题: Can't build with GTK and FEAT_GUI_GNOME. (Tony Mechelynck)
解决方案: Adjust the #ifdef. (Yegappan Lakshmanan)
文件: src/session.c

Patch 8.1.2103
问题: wrong error message if "termdebugger" is not executable.
解决方案: Check if "termdebugger" is executable and give a clear error
           message. (Ozaki Kiichi, closes #5000) Fix indents.
文件: runtime/pack/dist/opt/termdebug/plugin/termdebug.vim

Patch 8.1.2104
问题: The normal.c file is too big.
解决方案: Move do_pending_operator() to ops.c. (Yegappan Lakshmanan,
           closes #4999).
文件: src/normal.c, src/ops.c, src/proto/normal.pro, src/proto/ops.pro,
           src/globals.h

Patch 8.1.2105
问题: MS-Windows: system() may crash.
解决方案: Do not use "itmp" when it is NULL. (Yasuhiro Matsumoto,
           closes #5005)
文件: src/ex_cmds.c

Patch 8.1.2106
问题: No tests for dragging the mouse beyond the window.
解决方案: Add a test. (Dominique Pelle, closes #5004)
文件: src/testdir/test_termcodes.vim

Patch 8.1.2107
问题: Various memory leaks reported by asan.

解决方案: Free the memory. (Ozaki Kiichi, closes #5003)
文件: src/buffer.c, src/change.c, src/eval.c, src/evalfunc.c,
src/option.c, src/popupwin.c, src/proto/change.pro,
src/scriptfile.c, src/terminal.c, src/testdir/test_method.vim

Patch 8.1.2108
问题: Cannot close the cmdline window from CmdWinEnter. (George Brown)
解决方案: Reset cmdwin_result earlier. (Christian Brabandt, closes #4980)
文件: src/ex_getln.c, src/testdir/test_autocmd.vim

Patch 8.1.2109
问题: popup_getoptions() hangs with tab-local popup.
解决方案: Correct pointer name. (Marko Mahni, closes #5006)
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.2110
问题: CTRL-C closes two popups instead of one.
解决方案: Reset got_int when the filter consumed the key.
文件: src/getchar.c, src/testdir/test_popupwin.vim

Patch 8.1.2111
问题: Viminfo file not sufficiently tested.
解决方案: Add more tests. (Yegappan Lakshmanan, closes #5009)
文件: src/testdir/test_viminfo.vim

Patch 8.1.2112
问题: Build number for ConPTY is outdated.
解决方案: Update to new build number. (Nobuhiro Takasaki, closes #5014)
文件: src/os_win32.c

Patch 8.1.2113
问题: ":help expr-!~?" only works after searching.
解决方案: Escape "~" after "expr-". (closes #5015)
文件: src/ex_cmds.c, src/testdir/test_help.vim

Patch 8.1.2114
问题: When a popup is closed with CTRL-C the callback aborts.
解决方案: Reset got_int when invoking the callback. (closes #5008)
文件: src/popupwin.c

Patch 8.1.2115
问题: MS-Windows: shell commands fail if &shell contains a space.
解决方案: Use quotes instead of escaping. (closes #4920)
文件: src/option.c, src/os_win32.c, src/testdir/test_startup.vim,
src/testdir/test_system.vim, src/vimrun.c,

Patch 8.1.2116
问题: No check for out of memory.
解决方案: Check for NULL pointer.
文件: src/option.c

Patch 8.1.2117
问题: CursorLine highlight used while 'cursorline' is off.
解决方案: Check 'cursorline' is set. (cloes #5017)

文件:　　src/drawline.c, src/testdir/test_cursorline.vim

Patch 8.1.2118
问题:　　Termcodes test fails when $TERM is "dumb".
解决方案:　Skip the test. (James McCoy, closes #5019)
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.2119
问题:　　memory access error for empty string when 'encoding' is a single
byte encoding.
解决方案:　Check for empty string when getting the length. (Dominique Pelle,
closes #5021, closes #5007)
文件:　　src/macros.h

Patch 8.1.2120
问题:　　Some MB_ macros are more complicated than necessary. (Dominique
Pelle)
解决方案:　Simplify the macros. Expand inline.
文件:　　src/macros.h, src/beval.c, src/diff.c src/eval.c src/evalfunc.c
src/ex_getln.c, src/filepath.c, src/findfile.c, src/getchar.c,
src/highlight.c, src/ops.c, src/os_mswin.c, src/popupmenu.c,
src/search.c, src/spell.c, src/spellsuggest.c, src/terminal.c

Patch 8.1.2121
问题:　　Mode is not updated when switching to terminal in Insert mode.
解决方案:　Redraw the mode when entering a terminal window. (Jason Franklin)
文件:　　src/window.c, src/testdir/test_window_cmd.vim

Patch 8.1.2122 (after 8.1.2121)
问题:　　Cannot build without terminal feature.
解决方案:　Add #ifdef.
文件:　　src/window.c

Patch 8.1.2123
问题:　　Parsing CSI sequence is messy.
解决方案:　Generalize parsing a CSI sequence.
文件:　　src/term.c

Patch 8.1.2124
问题:　　Ruler is not updated if win_execute() moves cursor.
解决方案:　Update the status line. (closes #5022)
文件:　　src/evalwindow.c, src/testdir/test_execute_func.vim

Patch 8.1.2125
问题:　　Fnamemodify() fails when repeating :e.
解决方案:　Do not go before the tail. (Rob Pilling, closes #5024)
文件:　　src/filepath.c, src/testdir/test_fnamemodify.vim

Patch 8.1.2126
问题:　　Viminfo not sufficiently tested.
解决方案:　Add more test cases. Clean up comments. (Yegappan Lakshmanan,
closes #5032)
文件:　　src/search.c, src/structs.h, src/testdir/test_viminfo.vim,
src/viminfo.c

Patch 8.1.2127
问题:	The indent.c file is a bit big.
解决方案:	Move C-indent code to a new cindent.c file. Move other
indent-related code to indent.c. (Yegappan Lakshmanan,
closes #5031)
文件:	Filelist, src/Make_cyg_ming.mak, src/Make_morph.mak,
src/Make_mvc.mak, src/Make_vms.mms, src/Makefile, src/README.md,
src/change.c, src/cindent.c, src/edit.c, src/evalfunc.c,
src/ex_cmds.c, src/globals.h, src/indent.c, src/misc1.c,
src/ops.c, src/proto.h, src/proto/cindent.pro, src/proto/edit.pro,
src/proto/ex_cmds.pro, src/proto/indent.pro, src/proto/misc1.pro,
src/proto/ops.pro, src/userfunc.c

Patch 8.1.2128
问题:	Renamed libvterm sources makes merging difficult.
解决方案:	Rename back to the original name and only rename the .o files.
Also clean the libvterm build artifacts. (James McCoy,
closes #5027)
文件:	src/libvterm/src/termmouse.c, src/libvterm/src/mouse.c,
src/libvterm/src/termscreen.c, src/libvterm/src/screen.c,
src/Makefile, src/configure.ac, src/auto/configure,
src/Make_cyg_ming.mak, src/Make_mvc.mak

Patch 8.1.2129
问题:	Using hard coded executable path in test.
解决方案:	Use v:progpath. Use $VIMRUNTIME instead of "runtime". (James
McCoy, closes #5025)
文件:	src/testdir/test49.vim, src/testdir/test_compiler.vim,
src/testdir/test_spell.vim

Patch 8.1.2130 (after 8.1.2128)
问题:	MSVC build fails.
解决方案:	Add the source file name explicitly.
文件:	src/Make_mvc.mak

Patch 8.1.2131 (after 8.1.2129)
问题:	MSVC tests fail.
解决方案:	Replace backslashes with slashes.
文件:	src/testdir/test_compiler.vim, src/testdir/test_spell.vim

Patch 8.1.2132
问题:	MS-Windows: screen mess when not recognizing insider build.
解决方案:	Always move the cursor to the first column first. (Nobuhiro
Takasaki, closes #5036)
文件:	src/os_win32.c

Patch 8.1.2133
问题:	Some tests fail when run as root.
解决方案:	Add CheckNotRoot and use it. (James McCoy, closes #5020)
文件:	src/testdir/check.vim, src/testdir/shared.vim,
src/testdir/test_rename.vim, src/testdir/test_swap.vim,
src/testdir/test_terminal.vim, src/testdir/test_viminfo.vim

Patch 8.1.2134
问题:　　Modifier keys are not always recognized.
解决方案:　Handle key codes generated by xterm with modifyOtherKeys set.
　　Add this to libvterm so we can debug it.
文件:　　src/term.c, src/getchar.c, src/libvterm/src/vterm_internal.h,
　　src/libvterm/src/state.c, src/libvterm/src/keyboard.c,
　　src/libvterm/include/vterm.h, src/globals.h, src/terminal.c

Patch 8.1.2135
问题:　　With modifyOtherKeys Alt-a does not work properly.
解决方案:　Remove the ALT modifier. Get multi-byte after applying ALT.
文件:　　src/getchar.c

Patch 8.1.2136
问题:　　using freed memory with autocmd from fuzzer. (Dhiraj Mishra,
　　Dominique Pelle)
解决方案:　Avoid using "wp" after autocommands. (closes #5041)
文件:　　src/window.c, src/testdir/test_autocmd.vim

Patch 8.1.2137
问题:　　Parsing the termresponse is not tested.
解决方案:　Add a first test. (related to #5042)
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.2138
问题:　　Including the build number in the Win32 binary is confusing.
解决方案:　Only use the patchlevel.
文件:　　src/vim.rc

Patch 8.1.2139
问题:　　The modifyOtherKeys codes are not tested.
解决方案:　Add a test case.
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.2140
问题:　　"gk" and "gj" do not work correctly in number column.
解决方案:　Allow for a negative "curswant". (Zach Wegner, closes #4969)
文件:　　src/testdir/test_normal.vim, src/misc2.c, src/normal.c

Patch 8.1.2141
问题:　　:tselect has an extra hit-enter prompt.
解决方案:　Do not set need_wait_return when only moving the cursor.
　　(closes #5040)
文件:　　src/message.c, src/testdir/test_tagjump.vim,
　　src/testdir/dumps/Test_tselect_1.dump

Patch 8.1.2142
问题:　　Some key mappings do not work with modifyOtherKeys.
解决方案:　Remove the Shift modifier if it is already included in the key.
文件:　　src/term.c, src/testdir/test_termcodes.vim

Patch 8.1.2143
问题:　　Cannot see each command even when 'verbose' is set.
解决方案:　List each command when 'verbose' is at least 16.

文件:　　src/ex_docmd.c src/testdir/test_tagjump.vim,
　　src/testdir/test_cmdline.vim,
　　src/testdir/dumps/Test_verbose_option_1.dump

Patch 8.1.2144
问题:　　Side effects when using t_ti to enable modifyOtherKeys.
解决方案:　Add t_TI and t_TE.
文件:　　runtime/doc/term.txt, src/term.c, src/optiondefs.h, src/term.h,

Patch 8.1.2145
问题:　　Cannot map <C-H> when modifyOtherKeys is enabled.
解决方案:　Add the <C-H> mapping twice, both with modifier and as 0x08. Use
　　only the first one when modifyOtherKeys has been detected.
文件:　　src/term.c, src/eval.c, src/getchar.c, src/globals.h,
　　src/gui_mac.c, src/gui_w32.c, src/highlight.c, src/if_ole.cpp,
　　src/main.c, src/map.c, src/menu.c, src/misc2.c, src/option.c,
　　src/proto/misc2.pro, src/proto/term.pro,
　　src/testdir/test_termcodes.vim, src/structs.h, src/terminal.c,
　　src/usercmd.c, src/vim.h

Patch 8.1.2146 (after 8.1.2145)
问题:　　Build failure.
解决方案:　Include omitted changed file.
文件:　　src/optionstr.c

Patch 8.1.2147
问题:　　Crash when allocating memory fails. (Zu-Ming Jiang)
解决方案:　Check that 'spellcapcheck' is not NULL. (closes #5048)
文件:　　src/spell.c

Patch 8.1.2148
问题:　　No test for right click extending Visual area.
解决方案:　Add a test. (Dominique Pelle, closes #5018)
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.2149
问题:　　Crash when running out of memory very early.
解决方案:　Do not use IObuff when it's NULL. (closes #5052)
文件:　　src/message.c

Patch 8.1.2150
问题:　　No test for 'ttymouse' set from xterm version response.
解决方案:　Test the three possible values.
文件:　　src/testdir/test_termcodes.vim

Patch 8.1.2151
问题:　　State test is a bit flaky.
解决方案:　Add to the list of flaky tests.
文件:　　src/testdir/runtest.vim

Patch 8.1.2152
问题:　　Problems navigating tags file on MacOS Catalina.
解决方案:　Use fseek instead of lseek. (John Lamb, fixes #5061)
文件:　　src/tag.c

Patch 8.1.2153
问题: Combining text property and syntax highlight is wrong. (Nick Jensen)
解决方案: Compute the syntax highlight attribute much earlier. (closes #5057)
文件: src/drawline.c, src/testdir/test_textprop.vim, src/testdir/dumps/Test_textprop_syn_1.dump

Patch 8.1.2154
问题: Quickfix window height wrong when there is a tabline. (Daniel Hahler)
解决方案: Take the tabline height into account. (closes #5058)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.2155
问题: In a terminal window 'cursorlineopt' does not work properly.
解决方案: Check the 'cursorlineopt' value. (closes #5055)
文件: src/drawline.c, src/testdir/test_terminal.vim, src/testdir/dumps/Test_terminal_normal_1.dump, src/testdir/dumps/Test_terminal_normal_2.dump, src/testdir/dumps/Test_terminal_normal_3.dump

Patch 8.1.2156
问题: First character after Tab is not highlighted.
解决方案: Remember the syntax attribute for a column.
文件: src/drawline.c, src/testdir/test_syntax.vim, src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.2157
问题: Libvterm source files missing from distribution.
解决方案: Rename source files. (closes #5065)
文件: Filelist

Patch 8.1.2158
问题: Terminal attributes missing in Terminal-normal mode.
解决方案: Use "syntax_attr".
文件: src/drawline.c, src/testdir/test_terminal.vim, src/testdir/dumps/Test_terminal_dumpload.dump

Patch 8.1.2159
问题: Some mappings are listed twice.
解决方案: Skip mappings duplicated for modifyOtherKeys. (closes #5064)
文件: src/map.c, src/testdir/test_mapping.vim

Patch 8.1.2160
问题: Cannot build with +syntax but without +terminal.
解决方案: Add #ifdef.
文件: src/drawline.c

Patch 8.1.2161
问题: Mapping test fails.
解决方案: Run the test separately.
文件: src/testdir/Make_all.mak, src/testdir/test_alot.vim

Patch 8.1.2162
问题: Popup resize test is flaky. (Christian Brabandt)
解决方案: Add the function to the list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.2163
问题: Cannot build with +spell but without +syntax.
解决方案: Add #ifdef. (John Marriott)
文件: src/drawline.c

Patch 8.1.2164
问题: Stuck when using "j" in a popupwin with popup_filter_menu if a
            line wraps.
解决方案: Check the cursor line is visible. (closes #4577)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_wrap_1.dump,
            src/testdir/dumps/Test_popupwin_wrap_2.dump

Patch 8.1.2165
问题: Mapping test fails on Mac.
解决方案: Remove the default Mac mapping.
文件: src/testdir/test_mapping.vim

Patch 8.1.2166
问题: Rubyeval() not tested as a method.
解决方案: Change a test case.
文件: src/testdir/test_ruby.vim

Patch 8.1.2167
问题: Mapping test fails on MS-Windows.
解决方案: Remove all the existing Insert-mode mappings.
文件: src/testdir/test_mapping.vim

Patch 8.1.2168
问题: Heredoc assignment not skipped in if block.
解决方案: Check if "skip" is set. (closes #5063)
文件: src/evalvars.c, src/testdir/test_let.vim

Patch 8.1.2169
问题: Terminal flags are never reset.
解决方案: Reset the flags when setting 'term'.
文件: src/term.c, src/testdir/test_termcodes.vim

Patch 8.1.2170 (after 8.1.2169)
问题: Cannot build without the +termresponse feature.
解决方案: Add #ifdef.
文件: src/term.c

Patch 8.1.2171
问题: Mouse support not always available.
解决方案: Enable mouse support also in tiny version. Do not define
            FEAT_MOUSE_XTERM on MS-Windows (didn't really work).
文件: src/feature.h, src/edit.c, src/evalfunc.c, src/ex_getln.c,

src/getchar.c, src/message.c, src/misc1.c, src/mouse.c,
src/move.c, src/normal.c, src/ops.c, src/option.c,
src/optionstr.c, src/os_unix.c, src/os_win32.c, src/register.c,
src/term.c, src/testing.c, src/window.c, src/globals.h,
src/option.h, src/optiondefs.h, src/os_win32.h, src/vim.h,
src/version.c

Patch 8.1.2172
问题: Spell highlight is wrong at start of the line.
解决方案: Fix setting the "v" variable. (closes #5078)
文件: src/drawline.c, src/testdir/test_spell.vim,
src/testdir/dumps/Test_spell_1.dump

Patch 8.1.2173
问题: Searchit() has too many arguments.
解决方案: Move optional arguments to a struct. Add the "wrapped" argument.
文件: src/search.c, src/proto/search.pro, src/structs.h, src/evalfunc.c,
src/ex_docmd.c, src/gui.c, src/quickfix.c, src/spell.c, src/tag.c,
src/ex_getln.c, src/insexpand.c, src/normal.c

Patch 8.1.2174
问题: Screen not recognized as supporting "sgr" mouse codes.
解决方案: Recognize screen 4.7. (Jordan Christiansen, closes #5042)
文件: src/term.c, src/testdir/test_termcodes.vim

Patch 8.1.2175
问题: Meson files are not recognized.
解决方案: Add the meson filetype. (Liam Beguin, Nirbheek Chauhan,
closes #5056) Also recognize hollywood.
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.2176
问题: Syntax attributes not combined with Visual highlighting. (Arseny
Nasokin)
解决方案: Combine the attributes. (closes #5083)
文件: src/drawline.c, src/testdir/test_syntax.vim,
src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.2177
问题: Dart files are not recognized.
解决方案: Add a filetype rule. (Eugene Ciurana, closes #5087)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.2178
问题: Accessing uninitialized memory in test.
解决方案: Check if there was a match before using the match position.
(Dominique Pelle, closes #5088)
文件: src/search.c

Patch 8.1.2179
问题: Pressing "q" at the more prompt doesn’t stop Python output. (Daniel
Hahler)
解决方案: Check for got_int in writer(). (closes #5053)
Also do this for Lua.

文件:　　src/if_py_both.h, src/if_lua.c

Patch 8.1.2180
问题:　　Error E303 is not useful when 'directory' is empty.
解决方案:　Skip the error message. (Daniel Hahler, #5067)
文件:　　src/memline.c, src/testdir/test_recover.vim,
　　runtime/doc/options.txt, runtime/doc/message.txt

Patch 8.1.2181
问题:　　Highlighting wrong when item follows tab.
解决方案:　Don't use syntax attribute when n_extra is non-zero.
　　(Christian Brabandt, closes #5076)
文件:　　src/drawline.c, src/feature.h,
　　src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.2182
问题:　　Test42 seen as binary by git diff.
解决方案:　Add .gitattributes file. Make explicit that 'cpo' does not
　　contain 'S'. (Daniel Hahler, closes #5072)
文件:　　.gitattributes, Filelist, src/testdir/test42.in

Patch 8.1.2183
问题:　　Running a test is a bit verbose.
解决方案:　Silence some messages. (Daniel Hahler, closes #5070)
文件:　　src/testdir/Makefile

Patch 8.1.2184
问题:　　Option context is not copied when splitting a window. (Daniel
　　Hahler)
解决方案:　Copy the option context, so that ":verbose set" works.
　　(closes #5066)
文件:　　src/option.c, src/testdir/test_options.vim

Patch 8.1.2185 (after 8.1.2181)
问题:　　Syntax test fails.
解决方案:　Add missing file patch.
文件:　　src/testdir/test_syntax.vim

Patch 8.1.2186 (after 8.1.2184)
问题:　　Cannot build without the +eval feature.
解决方案:　Move line inside #ifdef.
文件:　　src/option.c

Patch 8.1.2187
问题:　　Error for bad regexp even though regexp is not used when writing
　　a file. (Arseny Nasokin)
解决方案:　Ignore regexp errors. (closes #5059)
文件:　　src/cmdexpand.c, src/ex_docmd.c, src/testdir/test_writefile.vim

Patch 8.1.2188 (after 8.1.2187)
问题:　　Build error for missing define.
解决方案:　Add missing change.
文件:　　src/vim.h

Patch 8.1.2189
问题:　　Syntax highlighting wrong for tab.
解决方案:　Don't clear syntax attribute n_extra is non-zero.
文件:　　src/drawline.c, src/testdir/test_syntax.vim,
　　src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.2190
问题:　　Syntax test fails on Mac.
解决方案:　Limit the window size to 20 rows.
文件:　　src/testdir/test_syntax.vim,
　　src/testdir/dumps/Test_syntax_c_01.dump

Patch 8.1.2191
问题:　　When using modifyOtherKeys CTRL-X mode may not work.
解决方案:　Recognize a control character also in the form with a modifier.
文件:　　src/getchar.c

Patch 8.1.2192
问题:　　Cannot easily fill the info popup asynchronously.
解决方案:　Add the "popuphidden" value to 'completeopt'. (closes #4924)
文件:　　src/popupmenu.c, src/proto/popupmenu.pro, src/popupwin.c,
　　src/proto/popupwin.pro, src/vim.h, runtime/doc/options.txt,
　　runtime/doc/insert.txt, src/ex_cmds.c, src/proto/ex_cmds.pro,
　　src/optionstr.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_infopopup_hidden_1.dump,
　　src/testdir/dumps/Test_popupwin_infopopup_hidden_2.dump,
　　src/testdir/dumps/Test_popupwin_infopopup_hidden_3.dump

Patch 8.1.2193
问题:　　Popup_setoptions(popup_getoptions()) does not work.
解决方案:　Also accept a list with three entries for "moved" and
　　"mousemoved". (closes #5081)
文件:　　runtime/doc/popup.txt, src/popupwin.c,
　　src/testdir/test_popupwin.vim

Patch 8.1.2194
问题:　　ModifyOtherKeys is not enabled by default.
解决方案:　Add t_TI and t_TE to the builtin xterm termcap.
文件:　　runtime/doc/map.txt, src/term.c

Patch 8.1.2195
问题:　　Vim does not exit when closing a terminal window and it is the
　　last window.
解决方案:　Exit Vim if the closed terminal window is the last one.
　　(closes #4539)
文件:　　runtime/doc/terminal.txt, src/terminal.c, src/ex_docmd.c,
　　src/proto/ex_docmd.pro, src/testdir/test_terminal.vim

Patch 8.1.2196
问题:　　MS-Windows: running tests with MSVC lacks updates.
解决方案:　Improve running individual tests on MS-Windows. (closes #4922)
文件:　　src/Make_mvc.mak, src/testdir/Make_dos.mak

Patch 8.1.2197

问题: ExitPre autocommand may cause accessing freed memory.
解决方案: Check the window pointer is still valid. (closes #5093)
文件: src/testdir/test_exit.vim, src/ex_docmd.c

Patch 8.1.2198
问题: Crash when using :center in autocommand.
解决方案: Bail out early for an empty line. (Dominique Pelle, closes #5095)
文件: src/ex_cmds.c, src/testdir/test_textformat.vim

Patch 8.1.2199
问题: Build failure when using normal features without GUI and EXITFREE
            defined.
解决方案: Add #ifdef. (Dominique Pelle, closes #5106)
文件: src/scriptfile.c

Patch 8.1.2200
问题: Crash when memory allocation fails.
解决方案: Check for NULL curwin and curbuf. (Christian Brabandt,
            closes #4839)
文件: src/getchar.c

Patch 8.1.2201
问题: Cannot build with dynamically linked Python 3.8.
解决方案: Implement py3__Py_DECREF() and py3__Py_XDECREF(). (Ken Takata,
            closes #4080)
文件: src/if_python3.c

Patch 8.1.2202
问题: MS-Windows: build failure with GUI and small features.
解决方案: Add #ifdef. (Michael Soyka, closes #5097)
文件: src/gui_w32.c

Patch 8.1.2203
问题: Running libvterm tests without the +terminal feature.
解决方案: Only add the libvterm test target when building libvterm.
文件: src/configure.ac, src/auto/configure, src/config.mk.in,
            src/Makefile

Patch 8.1.2204
问题: Crash on exit when closing terminals. (Corey Hickey)
解决方案: Actually wait for the job to stop. (closes #5100)
文件: src/terminal.c

Patch 8.1.2205
问题: Sign entry structure has confusing name.
解决方案: Rename signlist_T to sign_entry_T and prefix se_ to the fields.
文件: src/structs.h, src/netbeans.c, src/sign.c, src/globals.h,
            src/drawline.c

Patch 8.1.2206
问题: No test for fixed issue #3893.
解决方案: Add a test. (Christian Brabandt, #3893)
文件: src/testdir/test_display.vim,
            src/testdir/dumps/Test_winline_rnu.dump

Patch 8.1.2207
问题:　　"gn" doesn't work quite right. (Jaehwang Jerry Jung)
解决方案:　Improve and simplify the search logic. (Christian Brabandt,
　　closes #5103, closes #5075)
文件:　　src/search.c, src/testdir/test_gn.vim

Patch 8.1.2208
问题:　　Unix: Tabs in output might be expanded to spaces.
解决方案:　Reset the XTABS flag. (closes #5108)
文件:　　src/os_unix.c

Patch 8.1.2209
问题:　　LF in escape codes may be expanded to CR-LF.
解决方案:　Do not expand LF in escape codes to CR-LF. (closes #5107)
文件:　　src/term.c

Patch 8.1.2210
问题:　　Using negative offset for popup_create() does not work.
解决方案:　Use -1 instead of zero. (closes #5111)
文件:　　src/popupwin.c, src/popupwin.vim, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_corners.dump

Patch 8.1.2211
问题:　　Listener callback "added" argument is not the total. (Andy
　　Massimino)
解决方案:　Compute the total. (closes #5105)
文件:　　src/change.c, src/testdir/test_listener.vim

Patch 8.1.2212
问题:　　Cannot see the selection type in :reg output. (Ayberk Aydn)
解决方案:　Add c/l/b. (Christian Brabandt, closes #5110, closes #4546)
文件:　　runtime/doc/change.txt, src/register.c,
　　src/testdir/test_registers.vim

Patch 8.1.2213
问题:　　Popup_textprop tests fail.
解决方案:　Adjust the column and line positioning.
文件:　　src/popupwin.c

Patch 8.1.2214
问题:　　Too much is redrawn when 'cursorline' is set.
解决方案:　Don't do a complete redraw. (closes #5079)
文件:　　src/main.c, src/change.c, src/drawscreen.c,
　　src/testdir/dumps/Test_Xcursorline_13.dump,
　　src/testdir/dumps/Test_Xcursorline_14.dump,
　　src/testdir/dumps/Test_Xcursorline_15.dump,
　　src/testdir/dumps/Test_Xcursorline_16.dump,
　　src/testdir/dumps/Test_Xcursorline_17.dump,
　　src/testdir/dumps/Test_Xcursorline_18.dump

Patch 8.1.2215
问题:　　Unreachable code in adjusting text prop columns.
解决方案:　Remove the code. (Christian Brabandt)

文件: src/textprop.c

Patch 8.1.2216
问题: Text property in wrong place after :substitute.
解决方案: Pass the new column instead of the old one. (Christian Brabandt,
            closes #4427)
文件: src/ex_cmds.c, src/testdir/test_textprop.vim

Patch 8.1.2217
问题: Compiler warning for unused variable.
解决方案: Move variable inside #ifdef. (John Marriott)
文件: src/ex_cmds.c

Patch 8.1.2218
问题: "gN" is off by one in Visual mode.
解决方案: Check moving forward. (Christian Brabandt, #5075)
文件: src/search.c, src/testdir/test_gn.vim

Patch 8.1.2219
问题: No autocommand for open window with terminal.
解决方案: Add TerminalWinOpen. (Christian Brabandt)
文件: runtime/doc/autocmd.txt, src/autocmd.c, src/terminal.c,
            src/testdir/test_terminal.vim, src/vim.h

Patch 8.1.2220
问题: :cfile does not abort like other quickfix commands.
解决方案: Abort when desired. Add tests for aborting. (Yegappan Lakshmanan,
            closes #5121)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.2221
问题: Cannot filter :disp output.
解决方案: Support filtering :disp output. (Andy Massimino, closes #5117)
文件: runtime/doc/various.txt, src/register.c,
            src/testdir/test_filter_cmd.vim

Patch 8.1.2222
问题: Accessing invalid memory. (Dominique Pelle)
解决方案: Reset highlight_match every time. (closes #5125)
文件: src/ex_getln.c

Patch 8.1.2223
问题: Cannot see what buffer an ml_get error is for.
解决方案: Add the buffer number and name in the message
文件: src/memline.c

Patch 8.1.2224
问题: Cannot build Amiga version.
解决方案: Add dummy mch_setmouse(). (Ola Sder, closes #5126)
文件: src/os_amiga.c, src/proto/os_amiga.pro

Patch 8.1.2225
问题: The "last used" info of a buffer is under used.
解决方案: Add "lastused" to getbufinfo(). List buffers sorted by last-used

field. (Andy Massimino, closes #4722)
文件: runtime/doc/eval.txt, runtime/doc/options.txt,
runtime/doc/windows.txt, src/buffer.c, src/evalbuffer.c,
src/ex_getln.c, src/misc1.c, src/option.c, src/option.h,
src/proto/misc1.pro, src/proto/viminfo.pro,
src/testdir/test_bufwintabinfo.vim, src/testdir/test_cmdline.vim,
src/testdir/test_excmd.vim, src/undo.c, src/vim.h, src/viminfo.c

Patch 8.1.2226
问题: Cannot use system copy/paste in non-xterm terminals.
解决方案: Instead of setting 'mouse' to "a" set it to "nvi" in defaults.vim.
文件: runtime/defaults.vim, runtime/doc/term.txt,
runtime/doc/options.txt

Patch 8.1.2227
问题: Layout wrong if 'lines' changes while cmdline window is open.
解决方案: Do not restore the window layout if 'lines' changed.
(closes #5130)
文件: src/window.c, src/testdir/test_cmdline.vim,
src/testdir/dumps/Test_cmdwin_restore_1.dump,
src/testdir/dumps/Test_cmdwin_restore_2.dump,
src/testdir/dumps/Test_cmdwin_restore_3.dump

Patch 8.1.2228
问题: screenpos() returns wrong values when 'number' is set. (Ben
Jackson)
解决方案: Compare the column with the window width. (closes #5133)
文件: src/move.c, src/testdir/test_cursor_func.vim

Patch 8.1.2229
问题: Cannot color number column above/below cursor differently.
解决方案: Add LineNrAbove and LineNrBelow. (Shaun Brady, closes #624)
文件: runtime/doc/syntax.txt, runtime/doc/options.txt, src/optiondefs.h,
src/drawline.c, src/vim.h, src/testdir/test_number.vim,
src/testdir/dumps/Test_relnr_colors_1.dump,
src/testdir/dumps/Test_relnr_colors_2.dump,
src/testdir/dumps/Test_relnr_colors_3.dump,
src/testdir/dumps/Test_relnr_colors_4.dump

Patch 8.1.2230
问题: MS-Windows: testing external commands can be improved.
解决方案: Adjust tests, remove duplicate test. (closes #4928)
文件: src/testdir/test_normal.vim, src/testdir/test_system.vim,
src/testdir/test_terminal.vim, src/testdir/test_undo.vim

Patch 8.1.2231
问题: Not easy to move to the middle of a text line.
解决方案: Add the gM command. (Yasuhiro Matsumoto, closes #2070)
文件: runtime/doc/index.txt, runtime/doc/motion.txt,
runtime/doc/quickref.txt, runtime/doc/usr_25.txt, src/normal.c,
src/testdir/test_normal.vim

Patch 8.1.2232
问题: MS-Windows: compiler warning for int size.

解决方案: Add type cast. (Mike Williams)
文件: src/normal.c

Patch 8.1.2233
问题: Cannot get the Vim command line arguments.
解决方案: Add v:argv. (Dmitri Vereshchagin, closes #1322)
文件: runtime/doc/eval.txt, src/evalvars.c, src/vim.h,
           src/proto/evalvars.pro, src/main.c, src/testdir/test_startup.vim

Patch 8.1.2234
问题: get_short_pathname() fails depending on encoding.
解决方案: Use the wide version of the library function. (closes #5129)
文件: src/filepath.c, src/testdir/test_shortpathname.vim

Patch 8.1.2235
问题: "C" with 'virtualedit' set does not include multi-byte char.
解决方案: Include the whole multi-byte char. (Nobuhiro Takasaki,
           closes #5152)
文件: src/ops.c, src/testdir/test_virtualedit.vim

Patch 8.1.2236
问题: Ml_get error if pattern matches beyond last line.
解决方案: Adjust position if needed. (Christian Brabandt, closes #5139)
文件: src/ex_cmds.c, src/testdir/test_substitute.vim

Patch 8.1.2237
问题: Mode() result after using "r" depends on whether CURSOR_SHAPE is
           defined. (Christian Brabandt)
解决方案: Move the #ifdef to only skip ui_cursor_shape().
文件: src/normal.c

Patch 8.1.2238
问题: Error in docs tags goes unnoticed.
解决方案: Adjust tags build command. (Ken Takata, closes #5158)
文件: Filelist, .travis.yml, runtime/doc/Makefile,
           runtime/doc/doctags.vim

Patch 8.1.2239
问题: CI fails when running tests without building Vim.
解决方案: Skip creating doc tags if the execute does not exist.
文件: runtime/doc/Makefile

Patch 8.1.2240
问题: Popup window width changes when scrolling.
解决方案: Also adjust maxwidth when applying minwidth and there is a
           scrollbar. Fix off-by-one error. (closes #5162)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
           src/testdir/dumps/Test_popupwin_scroll_11.dump,
           src/testdir/dumps/Test_popupwin_scroll_12.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_4.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_5.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_7.dump,
           src/testdir/dumps/Test_popupwin_previewpopup_8.dump,

Patch 8.1.2241
问题:　　Match highlight does not combine with 'wincolor'.
解决方案:　Apply 'wincolor' last on top of any other attribute. (closes #5159)
文件:　　src/drawline.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_matches.dump
　　src/testdir/dumps/Test_popupwin_menu_01.dump
　　src/testdir/dumps/Test_popupwin_menu_02.dump
　　src/testdir/dumps/Test_popupwin_menu_04.dump

Patch 8.1.2242
问题:　　Creating docs tags uses user preferences. (Tony Mechelynck)
解决方案:　Add "--clean".
文件:　　runtime/doc/Makefile

Patch 8.1.2243
问题:　　Typos in comments.
解决方案:　Fix the typos. (Dominique Pelle, closes #5160) Also adjust
　　formatting a bit.
文件:　　src/autocmd.c, src/buffer.c, src/cindent.c, src/crypt.c,
　　src/diff.c, src/getchar.c, src/globals.h, src/gui_gtk_x11.c,
　　src/highlight.c, src/insexpand.c, src/macros.h, src/map.c,
　　src/memline.c, src/message.c, src/option.c, src/os_unix.c,
　　src/pty.c, src/quickfix.c, src/regexp_nfa.c, src/register.c,
　　src/spellsuggest.c, src/structs.h, src/textprop.c, src/ui.c,
　　src/undo.c, src/vim.h, src/viminfo.c

Patch 8.1.2244
问题:　　'wrapscan' is not used for "gn".
解决方案:　Only reset 'wrapscan' for the first search round. (closes #5164)
文件:　　src/search.c, src/testdir/test_gn.vim

Patch 8.1.2245
问题:　　Third character of 'listchars' tab shows in wrong place when
　　'breakindent' is set.
解决方案:　Set c_final to NUL. (Naruhiko Nishino, closes #5165)
文件:　　src/drawline.c, src/testdir/test_breakindent.vim

Patch 8.1.2246
问题:　　Some tests are still in old style.
解决方案:　Change a few tests to new style. (Yegappan Lakshmanan)
文件:　　src/testdir/Make_all.mak, src/testdir/test49.ok,
　　src/testdir/test49.vim, src/testdir/test_trycatch.vim,
　　src/testdir/test_vimscript.vim

Patch 8.1.2247
问题:　　"make vimtags" does not work in runtime/doc.
解决方案:　Test existence with "which" instead of "test -x". (Ken Takata)
文件:　　runtime/doc/Makefile

Patch 8.1.2248
问题:　　CTRL-W dot does not work in a terminal when modifyOtherKeys is
　　enabled.
解决方案:　Use the modifier when needed. Pass the modifier along with the
　　key to avoid mistakes.

文件: src/terminal.c, src/proto/terminal.pro, src/mouse.c

Patch 8.1.2249
问题: "make vimtags" does not print any message.
解决方案: Add a message that the tags have been updated.
文件: runtime/doc/Makefile

Patch 8.1.2250
问题: CTRL-U and CTRL-D don't work in popup window.
解决方案: Initialize 'scroll'. Add "lastline" in popup_getpos().
            (closes #5170)
文件: src/popupwin.c, src/testdir/test_popupwin.vim,
            runtime/doc/popup.txt

Patch 8.1.2251
问题: ":term command" may not work without a shell.
解决方案: Add the ++shell option to :term. (closes #3340)
文件: runtime/doc/terminal.txt, src/terminal.c,
            src/os_unix.c, src/proto/os_unix.pro,
            src/testdir/test_terminal.vim

Patch 8.1.2252
问题: Compiler warning for int size.
解决方案: Add type cast. (Mike Williams)
文件: src/filepath.c

Patch 8.1.2253
问题: Using "which" to check for an executable is not reliable.
解决方案: Use "command -v" instead. Also exit with error code when
            generating tags has an error. (closes #5174)
文件: runtime/doc/Makefile

Patch 8.1.2254
问题: MS-Windows: mouse scroll wheel doesn't work in popup.
解决方案: Handle mouse wheel events separately. (closes #5138)
文件: src/gui_w32.c, src/gui.c, src/proto/gui.pro

Patch 8.1.2255
问题: ":term ++shell" does not work on MS-Windows.
解决方案: Add MS-Windows support.
文件: src/terminal.c, src/testdir/test_terminal.vim

Patch 8.1.2256 (after 8.1.2255)
问题: Test for ":term ++shell" fails on MS-Windows.
解决方案: Accept failure of "dir" executable.
文件: src/testdir/test_terminal.vim

Patch 8.1.2257
问题: MS-Windows GUI: scroll wheel always uses current window.
解决方案: Add the 'scrollfocus' option for MS-Windows.
文件: runtime/doc/options.txt, src/gui_w32.c, src/optiondefs.h,
            src/option.h

Patch 8.1.2258

问题:	May get hit-enter prompt after entering a number. (Malcolm Rowe)
解决方案:	Put back accidentally deleted lines. (closes #5176)
文件:	src/misc1.c

Patch 8.1.2259
问题:	Running tests may leave XfakeHOME behind.
解决方案:	Source summarize.vim without using setup.vim. (closes #5177)
Also fix that on MS-Windows the test log isn't echoed.
文件:	src/testdir/Makefile, src/testdir/Make_dos.mak

Patch 8.1.2260
问题:	Terminal test may fail on MS-Windows.
解决方案:	Catch the situation that "term dir" fails with a CreateProcess error.
文件:	src/testdir/test_terminal.vim

Patch 8.1.2261
问题:	With modifyOtherKeys set 'noesckeys' doesn't work. (James McCoy)
解决方案:	Disable modifyOtherKeys while in Insert mode when 'noesckeys' is set. (closes #5180)
文件:	src/edit.c, src/testdir/test_edit.vim

Patch 8.1.2262
问题:	Unpack assignment in function not recognized.
解决方案:	Skip over "[a, b]". (closes #5051)
文件:	src/userfunc.c, src/testdir/test_let.vim

Patch 8.1.2263
问题:	'noesckeys' test fails in GUI.
解决方案:	Skip the test in the GUI.
文件:	src/testdir/test_edit.vim

Patch 8.1.2264
问题:	There are two test files for :let.
解决方案:	Merge the two files.
文件:	src/testdir/test_assign.vim, src/testdir/test_let.vim, src/testdir/Make_all.mak, src/testdir/test_alot.vim

Patch 8.1.2265
问题:	When popup with "botleft" does not fit it flips incorrectly.
解决方案:	Only flip when there is more space on the other side. Add the "posinvert" option to disable flipping and do it in both directions if enabled. (closes #5151)
文件:	src/popupwin.c, src/testdir/test_popupwin.vim, src/vim.h, src/testdir/dumps/Test_popupwin_nospace.dump

Patch 8.1.2266
问题:	Position unknown for a mouse click in a popup window.
解决方案:	Set v:mouse_col and v:mouse_lnum. (closes #5171)
文件:	src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.2267
问题:	Compiler warning for uninitialized variable. (Tony Mechelynck)
解决方案:	Rearrange the code.

文件: src/buffer.c

Patch 8.1.2268
问题: Spell file flag zero is not recognized.
解决方案: Use -1 as an error value, so that zero can be used as a valid flag
            number.
文件: src/spellfile.c, src/testdir/test_spell.vim

Patch 8.1.2269
问题: Tags file with very long line stops using binary search.
解决方案: Reallocate the buffer if needed.
文件: src/tag.c, src/testdir/test_tagjump.vim

Patch 8.1.2270
问题: "gf" is not tested in Visual mode.
解决方案: Add Visual mode test and test errors. (Dominique Pelle,
            closes #5197)
文件: src/testdir/test_gf.vim

Patch 8.1.2271
问题: Build error if FEAT_TAG_BINS is not defined. (John Marriott)
解决方案: Add #ifdef.
文件: src/tag.c

Patch 8.1.2272
问题: Test may hang at more prompt.
解决方案: Reset 'more' after resetting 'compatible'. (Michael Soyka)
文件: src/testdir/test_vimscript.vim

Patch 8.1.2273
问题: Wrong default when "pos" is changed with popup_atcursor().
解决方案: Adjust the default line and col when "pos" is not the default
            value. (#5151)
文件: runtime/doc/popup.txt, src/structs.h, src/popupwin.c,
            src/proto/popupwin.pro, src/ex_cmds.c,
            src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_atcursor_pos.dump

Patch 8.1.2274
问题: Newlines in 'balloonexpr' result only work in the GUI.
解决方案: Also recognize newlines in the terminal. (closes #5193)
文件: src/popupmenu.c, src/testdir/test_balloon.vim,
            src/testdir/dumps/Test_balloon_eval_term_01.dump,
            src/testdir/dumps/Test_balloon_eval_term_01a.dump,
            src/testdir/dumps/Test_balloon_eval_term_02.dump

Patch 8.1.2275
问题: Using "seesion" looks like a mistake.
解决方案: Use an underscore to make the function sort first.
文件: src/testdir/test_mksession.vim

Patch 8.1.2276
问题: MS-Windows: session test leaves files behind.
解决方案: Wipe out buffers before deleting the directory. (closes #5187)

文件: src/testdir/test_mksession.vim

Patch 8.1.2277
问题: Terminal window is not updated when info popup changes.
解决方案: Redraw windows when re-using an info popup. (closes #5192)
文件: src/ex_cmds.c

Patch 8.1.2278
问题: Using "cd" with "exe" may fail.
解决方案: Use chdir() instead.
文件: src/testdir/test_autochdir.vim, src/testdir/test_autocmd.vim,
            src/testdir/test_cd.vim, src/testdir/test_expand.vim,
            src/testdir/test_find_complete.vim, src/testdir/test_findfile.vim,
            src/testdir/test_getcwd.vim, src/testdir/test_shortpathname.vim

Patch 8.1.2279
问题: Computation of highlight attributes is too complicated.
解决方案: Simplify the attribute computation and make it more consistent.
            (closes #5190) Fix that 'combine' set to zero doesn't work.
文件: src/drawline.c, src/testdir/test_textprop.vim,
            src/testdir/dumps/Test_textprop_01.dump

Patch 8.1.2280
问题: Crash when passing partial to substitute().
解决方案: Take extra arguments into account. (closes #5186)
文件: src/userfunc.c, src/structs.h, src/regexp.c, src/proto/regexp.pro,
            src/testdir/test_substitute.vim

Patch 8.1.2281
问题: 'showbreak' cannot be set for one window.
解决方案: Make 'showbreak' global-local.
文件: src/optiondefs.h, src/option.c, src/option.h,
            src/proto/option.pro, src/structs.h, src/charset.c,
            src/drawline.c, src/edit.c, src/move.c, src/normal.c, src/ops.c,
            src/optionstr.c, src/testdir/test_highlight.vim,
            src/testdir/test_breakindent.vim, runtime/doc/options.txt

Patch 8.1.2282
问题: Crash when passing many arguments through a partial. (Andy
            Massimino)
解决方案: Check the number of arguments. (closes #5186)
文件: src/userfunc.c, src/proto/userfunc.pro, src/evalfunc.c,
            src/regexp.c, src/testdir/test_expr.vim,
            src/testdir/test_substitute.vim

Patch 8.1.2283
问题: Missed one use of p_sbr.
解决方案: Add missing p_sbr change.
文件: src/indent.c

Patch 8.1.2284
问题: Compiler warning for unused variable. (Tony Mechelynck)
解决方案: Add #ifdef.
文件: src/move.c

Patch 8.1.2285
问题:　　Padding in structures wastes memory.
解决方案:　Move fields to avoid padding. (Dominique Pelle, closes #5202)
文件:　　src/structs.h

Patch 8.1.2286
问题:　　Using border highlight in popup window leaks memory.
解决方案:　Free memory before overwriting. (Dominique Pelle, closes #5203)
文件:　　src/popupwin.c

Patch 8.1.2287
问题:　　Using EndOfBuffer highlight in popup does not look good.
解决方案:　Do not EndOfBuffer highlight. (closes #5204)
文件:　　src/drawscreen.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_02.dump,
　　src/testdir/dumps/Test_popupwin_04.dump,
　　src/testdir/dumps/Test_popupwin_04a.dump,
　　src/testdir/dumps/Test_popupwin_05.dump,
　　src/testdir/dumps/Test_popupwin_06.dump,
　　src/testdir/dumps/Test_popupwin_07.dump,
　　src/testdir/dumps/Test_popupwin_08.dump

Patch 8.1.2288
问题:　　Not using all space when popup with "topleft" flips to above.
解决方案:　Recompute the height when a popup flips from below to above.
　　(closes #5151)
文件:　　src/popupwin.c, src/testdir/test_popupwin.vim,
　　src/testdir/dumps/Test_popupwin_nospace.dump

Patch 8.1.2289
问题:　　After :diffsplit closing the window does not disable diff.
解决方案:　Add "closeoff" to 'diffopt' and add it to the default.
文件:　　runtime/doc/options.txt, src/optiondefs.h, src/diff.c,
　　src/proto/diff.pro, src/window.c, src/testdir/test_diffmode.vim

Patch 8.1.2290
问题:　　Autocommand test fails.
解决方案:　Remove 'closeoff' from 'diffopt'.
文件:　　src/testdir/test_autocmd.vim

Patch 8.1.2291
问题:　　Memory leak when executing command in a terminal.
解决方案:　Free "argv". (Dominique Pelle, closes #5208)
文件:　　src/terminal.c

Patch 8.1.2292
问题:　　v:mouse_winid not set on click in popup window.
解决方案:　Set v:mouse_winid. (closes #5171)
文件:　　runtime/doc/popup.txt, src/popupwin.c,
　　src/testdir/test_popupwin.vim

Patch 8.1.2293
问题:　　Join adds trailing space when second line is empty. (Brennan

Vincent)
解决方案: Do not add a trailing space.
文件: src/ops.c, src/testdir/test_join.vim

Patch 8.1.2294
问题: Cursor position wrong when characters are concealed and a search causes a scroll.
解决方案: Fix the cursor column in a concealed line after window scroll. (closes #5215, closes #5012)
文件: src/drawscreen.c, src/testdir/test_matchadd_conceal.vim

Patch 8.1.2295
问题: If buffer of popup is in another window cursorline sign shows.
解决方案: Check the group of the sign.
文件: src/option.c, src/proto/option.pro, src/sign.c, src/proto/sign.pro, src/screen.c, src/drawline.c, src/testdir/test_popupwin.vim, src/testdir/dumps/Test_popupwin_cursorline_8.dump

Patch 8.1.2296
问题: Text properties are not combined with syntax by default.
解决方案: Make it work as documented. (closes #5190)
文件: src/testprop.c, src/testdir/test_textprop.vim

Patch 8.1.2297
问题: The ex_vimgrep() function is too long.
解决方案: Split it in three parts. (Yegappan Lakshmanan, closes #5211)
文件: src/quickfix.c

Patch 8.1.2298 (after 8.1.2296)
问题: Missing part of 8.1.2296.
解决方案: s/test/text/
文件: src/textprop.c

Patch 8.1.2299
问题: ConPTY in MS-Windows 1909 is still wrong.
解决方案: Use same solution as for 1903. (Nobuhiro Takasaki, closes #5217)
文件: src/misc2.c, src/os_win32.c

Patch 8.1.2300
问题: Redraw breaks going through list of popup windows.
解决方案: Use different flags for popup_reset_handled(). (closes #5216)
文件: src/popupwin.c, src/proto/popupwin.pro, src/structs.h, src/vim.h, src/mouse.c, src/testdir/test_popupwin.vim

Patch 8.1.2301
问题: MS-Windows GUI: drawing error when background color changes.
解决方案: Implement gui_mch_new_colors(). (Simon Sadler)
文件: src/gui_w32.c

Patch 8.1.2302
问题: :lockmarks does not work for '[and '].
解决方案: save and restore '[and '] marks. (James McCoy, closes #5222)
文件: runtime/doc/motion.txt, src/bufwrite.c, src/diff.c, src/ex_cmds.c,

src/fileio.c, src/indent.c, src/ops.c, src/register.c,
src/testdir/test_autocmd.vim, src/testdir/test_diffmode.vim

Patch 8.1.2303
问题: Cursor in wrong position after horizontal scroll.
解决方案: Set w_valid_leftcol. (closes #5214, closes #5224)
文件: src/move.c, src/testdir/test_matchadd_conceal.vim

Patch 8.1.2304
问题: Cannot get the mouse position when getting a mouse click.
解决方案: Add getmousepos().
文件: runtime/doc/eval.txt, runtime/doc/popup.txt, src/mouse.c
src/proto/mouse.pro, src/evalfunc.c, src/popupwin.c,
src/popupwin.pro, src/testdir/test_popupwin.vim,
src/testdir/test_functions.vim

Patch 8.1.2305
问题: No warning for wrong entry in translations.
解决方案: Check semicolons in keywords entry of desktop file.
文件: src/po/check.vim

Patch 8.1.2306
问题: Double and triple clicks are not tested.
解决方案: Test mouse clicks to select text. (closes #5226)
文件: src/testdir/test_termcodes.vim

Patch 8.1.2307
问题: Positioning popup doesn't work for buffer-local textprop.
解决方案: Make it work. (closes #5225)
文件: src/popupwin.c, src/testdir/test_popupwin_textprop.vim

Patch 8.1.2308
问题: Deleting text before zero-width textprop removes it.
解决方案: Keep zero-width textprop when deleting text.
文件: src/textprop.c, src/testdir/test_textprop.vim

Patch 8.1.2309
问题: Compiler warning for argument type.
解决方案: Use linenr_T and cast to varnumber_T. (John Marriott)
文件: src/mouse.c

Patch 8.1.2310
问题: No proper test for directory changes in quickfix.
解决方案: Add a test that uses multiple directories. (Yegappan Lakshmanan,
closes #5230)
文件: src/testdir/test_quickfix.vim

Patch 8.1.2311
问题: Warning for missing function prototype.
解决方案: Add the proto. (Dominique Pelle, closes #5233)
文件: src/proto/popupwin.pro

Patch 8.1.2312
问题: "line:" field in tags file not used.

解决方案: Recognize the field and use the value. (Andy Massimino, Daniel
           Hahler, closes #5232, closes #2546, closes #1057)
文件: src/tag.c, src/testdir/test_tagjump.vim

Patch 8.1.2313
问题: Debugging where a delay comes from is not easy.
解决方案: Use different values when calling ui_delay().
文件: src/buffer.c, src/change.c, src/fileio.c, src/gui.c,
           src/if_xcmdsrv.c, src/insexpand.c, src/main.c, src/normal.c,
           src/screen.c, src/search.c, src/tag.c, src/term.c, src/ui.c

Patch 8.1.2314
问题: vi' sometimes does not select anything.
解决方案: Recognize an empty selection. (Christian Brabandt, closes #5183)
文件: src/search.c, src/testdir/test_textobjects.vim

Patch 8.1.2315
问题: Not always using the right window when jumping to an error.
解决方案: Add the "uselast" flag in 'switchbuf'. (closes #1652)
文件: runtime/doc/options.txt, src/option.h, src/optionstr.c,
           src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.2316
问题: FORTIFY_SOURCE can also be present in CPPFLAGS.
解决方案: Remove it in configure. (Benedikt Morbach, closes #2786)
文件: src/configure.ac, src/auto/configure

Patch 8.1.2317
问题: No test for spell affix file with flag on suffix.
解决方案: Add a test case.
文件: src/testdir/test_spell.vim

Patch 8.1.2318 (after 8.1.2301)
问题: MS-Windows GUI: main background shows in toolbar.
解决方案: Remove transparency from the toolbar. (Simon Sadler)
文件: src/gui_w32.c

Patch 8.1.2319
问题: Compiler warning for int size.
解决方案: Add typecast. (Mike Williams)
文件: src/mouse.c

Patch 8.1.2320
问题: Insufficient test coverage for quickfix.
解决方案: Add more tests. Fix uncovered problem. (Yegappan Lakshmanan,
           closes #5238)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.2321
问题: Cannot select all text with the mouse. (John Marriott)
解决方案: Move limiting the mouse column to f_getmousepos(). (closes #5242)
文件: src/mouse.c

Patch 8.1.2322 (after 8.1.2320)

问题: Quickfix test fails in very big terminal.
解决方案: Adjust the expected result for the width. (Masato Nishihata,
            closes #5244)
文件: src/testdir/test_quickfix.vim

Patch 8.1.2323
问题: Old MSVC version no longer tested.
解决方案: Drop support for MSVC 2008 and older. (Ken Takata, closes #5248)
文件: src/INSTALLpc.txt, src/Make_mvc.mak, src/gui_w32.c, src/os_win32.c

Patch 8.1.2324
问题: Width of scrollbar in popup menu not taken into account.
解决方案: Add the width of the scrollbar.
文件: src/popupmenu.c, src/testdir/dumps/Test_popupwin_infopopup_6.dump,
            src/testdir/test_popupwin.vim

Patch 8.1.2325
问题: Crash when using balloon with empty line.
解决方案: Handle empty lines. (Markus Braun)
文件: src/popupmenu.c, src/testdir/test_popup.vim

Patch 8.1.2326
问题: Cannot parse a date/time string.
解决方案: Add strptime(). (Stephen Wall, closes #5250)
文件: runtime/doc/eval.txt, runtime/doc/usr_41.txt, src/auto/configure,
            src/config.h.in, src/configure.ac, src/evalfunc.c, src/os_unix.h,
            src/testdir/test_functions.vim

Patch 8.1.2327
问题: Cannot build with Hangul input.
解决方案: Remove Hangul input support.
文件: Filelist, src/Makefile, runtime/doc/hangulin.txt, src/feature.h,
            src/gui_gtk_x11.c, src/gui_x11.c, src/gui.c, src/edit.c,
            src/mbyte.c, src/screen.c, src/ui.c, src/hangulin.c,
            src/globals.h, src/proto/hangulin.pro, src/proto.h,
            src/evalfunc.c, src/version.c, src/configure.ac,
            src/auto/configure, src/config.h.in, src/config.mk.in

Patch 8.1.2328
问题: A few hangul input pieces remain.
解决方案: Update VMS makefile.
文件: src/Make_vms.mms

Patch 8.1.2329
问题: Mouse multiple click test is a bit flaky.
解决方案: Add it to the list of flaky tests.
文件: src/testdir/runtest.vim

Patch 8.1.2330 (after 8.1.2314)
问题: vi' does not always work when 'selection' is exclusive.
解决方案: Adjust start position.
文件: src/search.c, src/testdir/test_textobjects.vim

Patch 8.1.2331

问题:　　The option.c file is still very big.
解决方案:　Move a few functions to where they fit better. (Yegappan
　　Lakshmanan, closes #4895)
文件:　　src/option.c, src/proto/option.pro, src/change.c,
　　src/proto/change.pro, src/ex_getln.c, src/proto/ex_getln.pro,
　　src/globals.h, src/gui.c, src/proto/gui.pro, src/ui.c,
　　src/proto/ui.pro, src/term.c, src/proto/term.pro, src/indent.c,
　　src/proto/indent.pro

Patch 8.1.2332 (after 8.1.2331)
问题:　　Missing file in refactoring.
解决方案:　Update missing file.
文件:　　src/search.c

Patch 8.1.2333
问题:　　With modifyOtherKeys CTRL-^ doesn't work.
解决方案:　Handle the exception.
文件:　　src/getchar.c, src/testdir/test_termcodes.vim

Patch 8.1.2334
问题:　　Possible NULL pointer dereference in popup_locate(). (Coverity)
解决方案:　Check for NULL pointer.
文件:　　src/popupwin.c

Patch 8.1.2335
问题:　　Error message for function arguments may use NULL pointer.
　　(Coverity)
解决方案:　Use the original function name.
文件:　　src/evalfunc.c

Patch 8.1.2336
问题:　　When an expr mapping moves the cursor it is not restored.
解决方案:　Position the cursor after an expr mapping. (closes #5256)
文件:　　src/getchar.c, src/testdir/test_mapping.vim,
　　src/testdir/dumps/Test_map_expr_1.dump

Patch 8.1.2337
问题:　　Double-click time sometimes miscomputed.
解决方案:　Correct time computation. (Dominique Pelle, closes #5259)
文件:　　src/mouse.c, src/testdir/runtest.vim

Patch 8.1.2338
问题:　　Using Visual mark with :s gives E20 if not set.
解决方案:　Ignore errors when handling 'incsearch'. (closes #3837)
文件:　　src/ex_getln.c, src/testdir/test_search.vim,
　　src/testdir/dumps/Test_incsearch_substitute_14.dump

Patch 8.1.2339
问题:　　Insufficient testing for quickfix.
解决方案:　Add a few more tests. (Yegappan Lakshmanan, closes #5261)
文件:　　src/testdir/test_quickfix.vim

Patch 8.1.2340
问题:　　Quickfix test fails under valgrind and asan.

解决方案: Make sure long line does not overflow IObuff. (Dominique Pelle,
           closes #5263) Put back fix for large terminals. (Yegappan
           Lakshmanan, closes #5264)
文件: src/quickfix.c, src/testdir/test_quickfix.vim

Patch 8.1.2341
问题: Not so easy to interrupt a script programmatically.
解决方案: Add the interrupt() function. (Yasuhiro Matsumoto, closes #2834)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/ex_eval.c,
           src/testdir/Make_all.mak, src/testdir/test_interrupt.vim

Patch 8.1.2342
问题: Random number generator in Vim script is slow.
解决方案: Add rand() and srand(). (Yasuhiro Matsumoto, closes #1277)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/Make_all.mak,
           src/testdir/test_random.vim

Patch 8.1.2343
问题: Using time() for srand() is not very random.
解决方案: use /dev/urandom if available
文件: src/evalfunc.c, src/testdir/test_random.vim

Patch 8.1.2344
问题: Cygwin: warning for using strptime().
解决方案: Move defining _XOPEN_SOURCE and __USE_XOPEN to vim.h. (Ken Takata,
           closes #5265) Use 700 for _XOPEN_SOURCE for mkdtemp().
文件: src/os_unix.h, src/vim.h

Patch 8.1.2345
问题: .cjs files are not recognized as Javascript.
解决方案: Add the *.cjs pattern. (closes #5268)
文件: runtime/filetype.vim, src/testdir/test_filetype.vim

Patch 8.1.2346
问题: CTRL-R CTRL-R doesn't work with modifyOtherKeys.
解决方案: Allow key codes when fetching argument for CTRL-R. (closes #5266)
           Also fix CTRL-G in Insert mode.
文件: src/edit.c, src/ex_getln.c, src/testdir/test_termcodes.vim

Patch 8.1.2347 (after 8.1.2344)
问题: MacOS: build fails.
解决方案: Don't define _XOPEN_SOURCE for Mac.
文件: src/vim.h

Patch 8.1.2348
问题: :const cannot be followed by "| endif".
解决方案: Check following command for :const. (closes #5269)
           Also fix completion after :const.
文件: src/testdir/test_let.vim, src/testdir/test_const.vim,
           src/ex_docmd.c, src/cmdexpand.c, src/eval.c,
           src/testdir/test_cmdline.vim

Patch 8.1.2349
问题: :lockvar and :unlockvar cannot be followed by "| endif".

解决方案: Check for following commands. (closes #5269)
文件: src/testdir/test_const.vim, src/ex_docmd.c

Patch 8.1.2350
问题: Other text for CTRL-V in Insert mode with modifyOtherKeys.
解决方案: Convert the Escape sequence back to key as if modifyOtherKeys is not set, and use CTRL-SHIFT-V to get the Escape sequence itself. (closes #5254)
文件: runtime/doc/insert.txt, runtime/doc/cmdline.txt, src/edit.c, src/proto/edit.pro, src/term.c, src/proto/term.pro, src/getchar.c, src/proto/getchar.pro, src/testdir/test_termcodes.vim, src/ex_getln.c

Patch 8.1.2351
问题: 'wincolor' not used for > for not fitting double width char. Also: popup drawn on right half of double width character looks wrong.
解决方案: Adjust color for > character. Clear left half of double width character if right half is being overwritten.
文件: src/drawline.c, src/screen.c, src/testdir/dumps/Test_popupwin_doublewidth_1.dump

Patch 8.1.2352
问题: CI doesn't cover FreeBSD.
解决方案: Configure Cirrus-CI. (Christian Brabandt, closes #5273)
文件: .cirrus.yml, README.md

Patch 8.1.2353
问题: Build failure on FreeBSD.
解决方案: Change #ifdef to only check for Linux-like systems.
文件: src/vim.h

Patch 8.1.2354
问题: Cirrus CI runs on another repository.
解决方案: Run Cirrus CI on vim/vim.
文件: .cirrus.yml, README.md

Patch 8.1.2355
问题: Test with "man" fails on FreeBSD.
解决方案: Use "-P" instead of "--pager".
文件: src/testdir/test_normal.vim

Patch 8.1.2356
问题: rand() does not use the best algorithm.
解决方案: use xoshiro128** instead of xorshift. (Kaito Udagawa, closes #5279)
文件: runtime/doc/eval.txt, src/evalfunc.c, src/testdir/test_random.vim

Patch 8.1.2357
问题: No test with wrong argument for rand().
解决方案: Add a test case.
文件: src/testdir/test_random.vim

Patch 8.1.2358

问题: Tests fail on Cirrus CI for FreeBSD.
解决方案: Fix a test and skip some. (Christian Brabandt, closes #5281)
文件: Filelist, .cirrus.yml, src/testdir/check.vim,
            src/testdir/test_normal.vim, src/testdir/test_quickfix.vim,
            src/testdir/test_source_utf8.vim, src/testdir/test_terminal.vim,
            src/testdir/test_utf8_comparisons.vim

Patch 8.1.2359
问题: Cannot build without FEAT_FLOAT. (John Marriott)
解决方案: Fix #ifdefs around f_srand().
文件: src/evalfunc.c

Patch 8.1.2360
问题: Quickfix test coverage can still be improved.
解决方案: Add more test cases. (Yegappan Lakshmanan, closes #5276)
文件: src/testdir/test_quickfix.vim

Patch 8.1.2361
问题: MS-Windows: test failures related to VIMDLL.
解决方案: Adjust code and tests. (Ken Takata, closes #5283)
文件: src/evalfunc.c, src/ex_cmds.c, src/gui_w32.c, src/mbyte.c,
            src/menu.c, src/proto.h, src/testdir/test_highlight.vim

Patch 8.1.2362
问题: Cannot place signs in a popup window. (Maxim Kim)
解决方案: Use the group prefix "PopUp" to specify which signs should show up
            in a popup window. (closes #5277)
文件: runtime/doc/sign.txt, src/popupwin.c, src/sign.c,
            src/testdir/dumps/Test_popupwin_sign_1.dump

Patch 8.1.2363
问题: ml_get error when accessing Visual area in 'statusline'.
解决方案: Disable Visual mode when using another window. (closes #5278)
文件: src/testdir/test_statusline.vim, src/buffer.c

Patch 8.1.2364
问题: Termwinscroll test is flaky on FreeBSD.
解决方案: Add to list of flaky tests. Rename function.
文件: src/testdir/runtest.vim, src/testdir/test_terminal.vim

Patch 8.1.2365
问题: Missing tests for recent popupwin changes.
解决方案: Add test cases.
文件: src/testdir/test_popupwin.vim

Patch 8.1.2366
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/ascii.h, src/beval.h, src/dosinst.h, src/feature.h,
            src/glbl_ime.h, src/globals.h, src/gui_at_sb.h, src/gui_gtk_f.h,
            src/gui_gtk_vms.h, src/gui.h, src/gui_x11_pm.h, src/gui_xmebwp.h,
            src/if_cscope.h, src/if_mzsch.h, src/if_ole.h, src/if_py_both.h,
            src/iscygpty.h, src/keymap.h, src/macros.h, src/nbdebug.h,
            src/option.h, src/os_amiga.h, src/os_beos.h, src/os_dos.h,

src/os_mac.h, src/os_qnx.h, src/os_unix.h, src/os_unixx.h,
src/os_vms_conf.h, src/os_win32.h, src/proto.h, src/regexp.h,
src/spell.h, src/structs.h, src/term.h, src/version.h, src/vimio.h

Patch 8.1.2367
问题: Registers are not sufficiently tested.
解决方案: Add a few more test cases. (Yegappan Lakshmanan, closes #5288)
文件: src/testdir/test_registers.vim

Patch 8.1.2368
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/autocmd.c, src/beval.c, src/blob.c, src/blowfish.c,
src/buffer.c, src/change.c, src/channel.c, src/charset.c,
src/cindent.c, src/crypt.c, src/crypt_zip.c

Patch 8.1.2369
问题: Cannot build with quickfix and without text properties.
解决方案: Fix typo. (Naruhiko Nishino)
文件: src/popupmenu.c

Patch 8.1.2370
问题: Build problems on VMS.
解决方案: Adjust the build file. (Zoltan Arpadffy)
文件: src/Make_vms.mms, src/os_unix.c, src/os_vms.c

Patch 8.1.2371
问题: FEAT_TEXT_PROP is a confusing name.
解决方案: Use FEAT_PROP_POPUP. (Naruhiko Nishino, closes #5291)
文件: runtime/doc/popup.txt, src/beval.c, src/buffer.c, src/change.c,
src/drawline.c, src/drawscreen.c, src/edit.c, src/eval.c,
src/evalbuffer.c, src/evalfunc.c, src/evalwindow.c, src/ex_cmds.c,
src/ex_docmd.c, src/feature.h, src/fileio.c, src/getchar.c,
src/globals.h, src/gui.c, src/gui_w32.c, src/indent.c,
src/insexpand.c, src/macros.h, src/main.c, src/memline.c,
src/misc2.c, src/mouse.c, src/move.c, src/ops.c, src/option.h,
src/optiondefs.h, src/optionstr.c, src/popupmenu.c,
src/popupwin.c, src/proto.h, src/screen.c, src/search.c,
src/sign.c, src/structs.h, src/tag.c, src/testdir/runtest.vim,
src/testdir/test_execute_func.vim, src/testdir/test_popupwin.vim,
src/testdir/test_popupwin_textprop.vim, src/textprop.c, src/ui.c,
src/version.c, src/vim.h, src/window.c

Patch 8.1.2372
问题: VMS: failing realloc leaks memory. (Chakshu Gupta)
解决方案: Free the memory. (partly fixes #5292)
文件: src/os_vms.c

Patch 8.1.2373
问题: Cannot build with +popupwin but without +quickfix. (John Marriott)
解决方案: Adjust #ifdefs.
文件: src/ex_cmds.c, src/popupmenu.c, src/popupwin.c, src/fileio.c,
src/testdir/test_compiler.vim, src/testdir/test_tagjump.vim,
src/testdir/test86.in, src/testdir/test87.in,

src/testdir/test_autocmd.vim, src/testdir/test_bufwintabinfo.vim,
src/testdir/test_channel.vim, src/testdir/test_edit.vim,
src/testdir/test_execute_func.vim,
src/testdir/test_filter_cmd.vim, src/testdir/test_gui.vim,
src/testdir/test_makeencoding.vim, src/testdir/test_mksession.vim,
src/testdir/test_normal.vim, src/testdir/test_popup.vim,
src/testdir/test_popupwin.vim, src/testdir/test_preview.vim,
src/testdir/test_startup.vim, src/testdir/test_statusline.vim,
src/testdir/test_tabpage.vim, src/testdir/test_window_cmd.vim,
src/testdir/test_window_id.vim

Patch 8.1.2374
问题: Unused parts of libvterm are included.
解决方案: Delete the unused files.
文件: Filelist, src/libvterm/bin/vterm-ctrl.c,
src/libvterm/bin/unterm.c, src/libvterm/bin/vterm-dump.c

Patch 8.1.2375
问题: No sufficient testing for registers.
解决方案: Add more test cases. (Yegappan Lakshmanan, closes #5296)
Fix that "p" on last virtual column of tab inserts spaces.
文件: src/register.c, src/testdir/test_registers.vim,
src/testdir/test_virtualedit.vim, src/testdir/test_visual.vim

Patch 8.1.2376
问题: Preprocessor indents are incorrect.
解决方案: Fix the indents. (Ken Takata, closes #5298)
文件: src/drawline.c, src/gui_w32.c, src/os_mswin.c, src/os_win32.c,
src/proto.h

Patch 8.1.2377
问题: GUI: when losing focus a pending operator is executed.
解决方案: Do not execute an operator when getting K_IGNORE. (closes #5300)
文件: src/normal.c

Patch 8.1.2378
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/dict.c, src/diff.c, src/digraph.c, src/dosinst.c, src/edit.c,
src/eval.c, src/evalbuffer.c, src/evalfunc.c

Patch 8.1.2379
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/ex_cmds.c, src/ex_cmds2.c, src/ex_docmd.c, src/ex_eval.c,
src/ex_getln.c, src/fileio.c, src/filepath.c, src/findfile.c,
src/fold.c

Patch 8.1.2380
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/getchar.c, src/gui.c, src/gui_at_fs.c, src/gui_at_sb.c,
src/gui_athena.c, src/gui_beval.c, src/gui_gtk.c, src/gui_gtk_f.c,
src/gui_gtk_x11.c

Patch 8.1.2381
问题: Not all register related code is covered by tests.
解决方案: Add more test cases. (Yegappan Lakshmanan, closes #5301)
文件: src/testdir/test_marks.vim, src/testdir/test_registers.vim,
src/testdir/test_virtualedit.vim

Patch 8.1.2382
问题: MS-Windows: When using VTP bold+inverse doesn't work.
解决方案: Compare with the default colors. (Nobuhiro Takasaki, closes #5303)
文件: src/os_win32.c, src/proto/os_win32.pro, src/screen.c

Patch 8.1.2383
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/gui_mac.c, src/gui_motif.c, src/gui_photon.c, src/gui_w32.c,
src/gui_x11.c, src/gui_xmdlg.c, src/gui_xmebw.c

Patch 8.1.2384
问题: Test 48 is old style.
解决方案: Merge test cases into new style test. (Yegappan Lakshmanan,
closes #5307)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
src/testdir/test48.in, src/testdir/test48.ok,
src/testdir/test_virtualedit.vim

Patch 8.1.2385
问题: Opening cmdline window with feedkeys() does not work. (Yegappan
Lakshmanan)
解决方案: Recognize K_CMDWIN also when ex_normal_busy is set.
文件: src/ex_getln.c, src/testdir/test_cmdline.vim

Patch 8.1.2386
问题: 'wincolor' is not used for 'listchars'.
解决方案: Combine the colors. (closes #5308)
文件: src/drawline.c, src/testdir/test_highlight.vim,
src/testdir/dumps/Test_wincolor_lcs.dump

Patch 8.1.2387
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/hardcopy.c, src/hashtab.c, src/if_cscope.c, src/if_lua.c,
src/if_mzsch.c, src/if_perlsfio.c, src/if_python.c,
src/if_python3.c, src/if_ruby.c, src/if_tcl.c, src/if_xcmdsrv.c

Patch 8.1.2388
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/json.c, src/json_test.c, src/kword_test.c, src/list.c,
src/main.c, src/mark.c, src/mbyte.c, src/memfile.c,
src/memfile_test.c, src/memline.c, src/menu.c

Patch 8.1.2389
问题: Using old C style comments.

解决方案: Use // comments where appropriate.
文件: src/libvterm/src/screen.c, src/libvterm/src/unicode.c,
           src/libvterm/src/vterm.c, src/libvterm/t/harness.c,
           src/libvterm/include/vterm.h, src/xdiff/xdiffi.c,
           src/xdiff/xemit.c, src/xdiff/xhistogram.c, src/xdiff/xpatience.c,
           src/xdiff/xutils.c, src/xdiff/xdiff.h, src/xdiff/xdiffi.h,
           src/xdiff/xemit.h, src/xdiff/xinclude.h, src/xdiff/xmacros.h,
           src/xdiff/xprepare.h, src/xdiff/xtypes.h, src/xdiff/xutils.h

Patch 8.1.2390
问题: Test94 is old style, fix 7.4.441 not tested.
解决方案: Turn test94 into a new style test. Add tests for the fix in patch
           7.4.441. (Yegappan Lakshmanan, closes #5316)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
           src/testdir/test94.in, src/testdir/test94.ok,
           src/testdir/test_cmdline.vim, src/testdir/test_visual.vim

Patch 8.1.2391
问题: Cannot build when __QNXNTO__ is defined. (Ian Wayne Larson)
解决方案: Move the check for "qansi". (Ken Takata, closes #5317)
文件: src/highlight.c

Patch 8.1.2392
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/nbdebug.c, src/netbeans.c, src/normal.c, src/ops.c,
           src/option.c

Patch 8.1.2393
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/os_amiga.c, src/os_beos.c, src/os_mac_conv.c, src/os_mswin.c,
           src/os_qnx.c, src/os_unix.c, src/os_vms.c, src/os_win32.c

Patch 8.1.2394
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/popupmenu.c, src/pty.c, src/quickfix.c, src/regexp.c,
           src/regexp_nfa.c, src/screen.c, src/search.c, src/sha256.c,
           src/sign.c

Patch 8.1.2395
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/spell.c, src/spellfile.c, src/syntax.c, src/tag.c, src/term.c,
           src/terminal.c, src/termlib.c, src/testing.c

Patch 8.1.2396
问题: Using old C style comments.
解决方案: Use // comments where appropriate.
文件: src/ui.c, src/undo.c, src/uninstall.c, src/usercmd.c,
           src/userfunc.c, src/winclip.c, src/window.c, src/xpm_w32.c

Patch 8.1.2397

问题: Should not define __USE_XOPEN. _XOPEN_SOURCE is not needed for
            Android.
解决方案: Remove __USE_XOPEN and adjust #ifdefs. (Ozaki Kiichi,
            closes #5322)
文件: src/vim.h

Patch 8.1.2398
问题: strptime() test fails on Japanese Mac.
解决方案: Use %T instead of %X.
文件: src/testdir/test_functions.vim

Patch 8.1.2399
问题: Info popup on top of cursor if it doesn't fit.
解决方案: Hide the popup if it doesn't fit.
文件: src/popupmenu.c, src/testdir/test_popupwin.vim,
            src/testdir/dumps/Test_popupwin_infopopup_wide_1.dump

Patch 8.1.2400
问题: Test39 is old style.
解决方案: Convert the test cases into new style. (Yegappan Lakshmanan,
            closes #5324)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
            src/testdir/test39.in, src/testdir/test39.ok,
            src/testdir/test_blockedit.vim, src/testdir/test_visual.vim

Patch 8.1.2401
问题: :cexpr does not handle | in expression.
解决方案: Remove EX_TRLBAR and set nextcmd pointer.
文件: src/testdir/test_quickfix.vim, src/ex_cmds.h, src/quickfix.c

Patch 8.1.2402
问题: Typos and other small things.
解决方案: Small fixes.
文件: .gitignore, src/testdir/shared.vim, src/testdir/test49.vim,
            src/message.c, src/Makefile

Patch 8.1.2403
问题: Autocmd test fails under valgrind.
解决方案: Wait a bit longer.
文件: src/testdir/test_autocmd.vim

Patch 8.1.2404
问题: Channel test fails under valgrind.
解决方案: Sleep a bit longer.
文件: src/testdir/test_channel.vim

Patch 8.1.2405
问题: matchadd_conceal test fails under valgrind.
解决方案: Use WaitForAssert() and wait a bit longer.
文件: src/testdir/test_matchadd_conceal.vim

Patch 8.1.2406
问题: Leaking memory in test_paste and test_registers.
解决方案: Free the old title. Don't copy expr_line.

文件:　　src/term.c, src/os_unix.c, src/register.c

Patch 8.1.2407
问题:　　proto file and dependencies outdated.
解决方案:　Update proto files and dependencies.
文件:　　src/Makefile, src/proto/bufwrite.pro, src/proto/cmdhist.pro,
　　src/proto/optionstr.pro, src/proto/popupwin.pro,
　　src/proto/viminfo.pro, src/proto/if_cscope.pro

Patch 8.1.2408
问题:　　Syntax menu and build instructions outdated.
解决方案:　Update build instructions and syntax menu.
文件:　　Makefile, runtime/makemenu.vim, runtime/synmenu.vim

Patch 8.1.2409
问题:　　Creating the distribution doesn't work as documented.
解决方案:　Adjust name of uninstall binary. Create src/auto directory if
　　needed.
文件:　　tools/rename.bat, src/Make_mvc.mak

Patch 8.1.2410
问题:　　MS-Windows: test_iminsert fails without IME support.
解决方案:　Skip the test when imgetstatus() doesn't work.
文件:　　src/testdir/test_iminsert.vim

Patch 8.1.2411
问题:　　Function argument copied unnecessarily.
解决方案:　Use the argument directly.
文件:　　src/ex_docmd.c

Patch 8.1.2412
问题:　　Crash when evaluating expression with error. (Dhiraj Mishra)
解决方案:　Check parsing failed. (closes #5329)
文件:　　src/eval.c, src/testdir/test_lambda.vim

Patch 8.1.2413
问题:　　Cannot update ex_cmdidxs.h on MS-Windows.
解决方案:　Add build rules and dependencies. (Ken Takata, closes #5337)
文件:　　src/Make_cyg_ming.mak, src/Make_mvc.mak, src/Make_vms.mms

Patch 8.1.2414
问题:　　MS-Windows: properties dialog box shows wrong character.
解决方案:　Explicitly specify encoding. (Ken Takata, closes #5338)
文件:　　src/vim.rc

Patch 8.1.2415
问题:　　Popup menu flickers if an info popup is used. (Nick Jensen)
解决方案:　Set the pum_skip_redraw flag.
文件:　　src/popupmenu.c

Patch 8.1.2416
问题:　　Loading menus sets v:errmsg.
解决方案:　Avoid setting v:errmsg and add a test for that. (Jason Franklin)
文件:　　runtime/delmenu.vim, runtime/menu.vim, src/testdir/test_menu.vim

Patch 8.1.2417
问题: MinGW/Cygwin build does not clean up all files.
解决方案: Delete *.map files. (Michael Soyka)
文件: src/Make_cyg_ming.mak

Patch 8.1.2418
问题: bufnr('$') is wrong after recycling popup buffer.
解决方案: Sort the buffer list by buffer number. (closes #5335)
文件: src/buffer.c, src/testdir/test_popupwin.vim

Patch 8.1.2419
问题: With a long file name the hit-enter prompt appears. (J. Lewis
            Muir)
解决方案: When checking for text to wrap don't do this when outputing a CR.
文件: src/message.c, src/testdir/test_display.vim,
            src/testdir/dumps/Test_long_file_name_1.dump

Patch 8.1.2420
问题: Crash when calling popup_close() in win_execute().
解决方案: Disallow popup_close() in popup window. (Yasuhiro Matsumoto,
            closes #5345)
文件: src/popupwin.c, src/testdir/test_popupwin.vim

Patch 8.1.2421
问题: Test88 is old style.
解决方案: Turn into a new style test. (Yegappan Lakshmanan, closes #5347)
文件: src/Makefile, src/testdir/Make_all.mak, src/testdir/Make_vms.mms,
            src/testdir/test88.in, src/testdir/test88.ok,
            src/testdir/test_conceal.vim, src/testdir/test_python2.vim
            src/testdir/test_python3.vim

Patch 8.1.2422
问题: "make depend" does not work correctly for libvterm.
解决方案: Fix build dependencies. And a few minor improvements.
文件: src/Makefile, src/filepath.c, src/insexpand.c, src/main.c

Patch 8.1.2423
问题: MS-Windows properties shows version as "8, 1, 0".
解决方案: Use "8.1.0". (Ken Takata, closes #5342)
文件: src/vim.rc

Patch 8.1.2424
问题: MS-Windows: console buffer is resized unnecessarily.
解决方案: Only call ResizeConBuf() when the size differs. (Nobuhiro
            Takasaki, closes #5343)
文件: src/os_win32.c

 vim:tw=78:ts=8:noet:ft=help:norl:

os_390.txt For Vim version 8.2. 最近更新：2017年8月

VIM 参考手册 by Ralf Schandl
译者：Willis
http://vimcdoc.sf.net

zOS z/OS OS390 os390 MVS

本文包含了 VIM 的 z/OS UNIX 版本的特点。

1. 依赖于 ASCII/EBCDIC 编码的脚本 zOS-has-ebcdic
2. Putty 和色彩 zOS-PuTTY
3. Motif 的问题 zOS-Motif
4 漏洞 zOS-Bugs
5. 限制 zOS-limitations
6. z/OS UNIX 上的开源软件 zOS-open-source

贡献者：
本 z/OS UNIX 的移植是由 Ralf Schandl 为了下面提到的红皮书完成的。

提供修改、报告漏洞、或者两者都有者：

David Moore
Anthony Giorgio
还有其他

---

1. 依赖于 ASCII/EBCDIC 的脚本 OS390-has-ebcdic zOS-has-ebcdic

为了内部脚本语言的编写方便，增加了 "ebcdic" 特性。所以你可以用以下方式修正依赖于ASCII 的脚本：

```
if has("ebcdic")
    let space = 64
else
    let space = 32
endif
```

---

2. PuTTY 和色彩 OS390-PuTTY zOS-PuTTY

如果使用 PuTTY 连接到 z/OS 有语法高亮的问题或者显示不正常，可以：

- 配置 Putty 使用 "vt220" 终端 (Connection->Data)
- 在 vimrc 加入以下 3 行：

```
set t_AB=[4%p1%dm
set t_AF=[3%p1%dm
set t_CO=8
```

注意： 是一个字符，用 <C-V><Esc> 可以输入。

3. Motif 问题 OS390-Motif zOS-Motif

备注：Vim 6.* 发现过，此后没有测试过。

看起来，在移植 Motif 库到 z/OS 的时候，忘记把下拉菜单的快捷键从 EBCDIC 翻译成 ASCII 了。即使在我手动修改菜单以后，快捷键也只能打开菜单（比如 <Alt-F> 打开文件菜单），但是不能打开菜单项（比如 <Alt-F>O 打开文件浏览器）。

现在还没有这个问题的解决方法。

4. 漏洞 OS390-bugs zOS-Bugs

- 在可视块模式，如果选择了大量的文本，Vim 一定会挂起。这可能是由一个内存错误引起的。 注意 终端和 GUI 版本都存在这个问题。

5. 限制 OS390-limitations zOS-limitations

- tag 文件不能进行二进制查找。
  /bin/sort 程序缺省按照 ASCII 值排序。通常，ctags 调用这个程序来为标签（tag）排序。也许有一个 ctags 版本能够做正确的事情，不过我们不能肯定。看起来，这是个永久的限制。

- 在我们的大型机上测试的版本上，cscope 接口（ cscope ）不能工作。我们有一个 15.0b12 版本的。在用 "cscope add" 命令的时候，它使得 Vim 挂起。我猜这是因为 cscope 数据库的二进制格式和 Vim 期待的不符所致。我试图把最新的 cscope（15.3）移植到 z/OS 上，看来成效不大。如果有人有兴趣尝试并有进展，请给我个消息。

- 没有 glib/gtk 支持。我不能成功在 z/OS UNIX 上编译 glib。这意味着你没法用漂亮的 gtk 工具栏了。

编译时关闭的功能：

- 多字节支持	(multibyte)
- 从右到左模式	(rileft)
- 波斯键盘映射	(Farsi)
- 阿拉伯语言支持	(Arabic)
- 拼写检查	(spell)

还未测试：

- Perl 接口	(perl)
- 韩语（Hangul）输入	(hangul)
- 加密支持	(encryption)
- 语言映射	('langmap')
- Python 支持	(Python)
- 从右到左模式	('rightleft')
- TCL 接口	(tcl)
...	

如果你尝试过这些特性而且成功了，告诉我们！

6. z/OS UNIX 上的开源软件 OS390-open-source zOS-open-source

如果你对 z/OS UNIX 上的其它开源软件感兴趣，请参考以下的红皮书：

```
Mike MacIsaac et al
"Open Source Software for z/OS and OS/390 UNIX"
IBM Form Number: SG24-5944-01
ISBN: 0738424633
http://www-03.ibm.com/systems/resources/servers_eserver_zseries_zos_unix_redbook_sg245944.pdf
```

另见：

```
http://www.redbooks.ibm.com
http://www-03.ibm.com/systems/z/os/zos/features/unix/
http://www-03.ibm.com/systems/z/os/zos/features/unix/library/IBM+Redbooks/index.html
```

---

vim:tw=78:ts=8:noet:ft=help:norl:

os_amiga.txt For Vim version 8.2. 最近更新: 2010年8月

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

本文档描述了 Vim 的 Amiga 版本的特点。有一节还专门讨论了 MorphOS 。

备注: Amiga 的代码还在，但没有人维护也没有经过测试。

Amiga 上的安装:
- Vim "doc" 目录赋予 "VIM:"。Vim 会查找 "VIM:doc/help.txt" 文件 (为了 help 命令)。设置环境变量 $VIM 也可以。反过来，如果用到 $VIM 但它没有定义，就用 "VIM:"。
- DOS 1.3 或更早的版本: 把 "arp.library" 放在 "libs:" 里。Vim 必须在编译时加入 +ARP 特性。确保 newcli 和 run 在 "C:" 里 (为了执行外部命令)。
- 在 "c:" 或者任何在搜索路径的目录里提供一个能接受带 "-c" 的命令的外壳程序 (例如，Fish disk 624 的 "Csh") (为了执行外部命令)。

如果你有足够内存，你可以使 Vim 和 csh 驻留来避免启动延迟。方法是执行命令 "rez csh vim"。你要把 "rezlib.library" 放在你的 "libs:" 目录。在 2.0 下你需要 rez 0.5 版本。

如果你不用二合字母 (digraphs)，去掉 +digraphs 特性并重新编译可以减少一些内存占用。如果你要在其它终端下使用 Vim，可以在重新编译时加上 TERMCAP 选项。Vim 在 Manx 5.x 和 SAS 6.x 下能编译通过。见 makefiles 和 feature.h。

如果你发现当打开语法高亮或者使用嵌套的通配符搜索时，Vim 在某些文件上崩溃，那可能是由于堆栈太小。试着增加堆栈大小。在命令行上，启动 Vim 之前，使用 Stack 命令。而在 Workbench 上，选择 Vim 图标，打开 workbench 的 "Info" 菜单，然后改变表单里的 Stack 字段。

如果你想要另外的颜色，设置如下的 termcap 代码:

```
        t_mr (反白文本)
        t_md (粗体文本)
        t_me (t_mr 和 t_md 后的正常文本)
        t_so (突出模式的文本)
        t_se (t_so 后的正常文本)
        t_us (加下划线的文本)
        t_ue (t_us 后的正常文本)
        t_ZH (斜体文本)
        t_ZR (t_ZH 后的正常文本)
```

可以使用标准 ANSI 的转换序列。这些代码是:

```
30 灰字  40 灰底  >0 灰色背景   0 关闭所有的属性
31 黑字  41 黑底  >1 黑色背景   1 粗体
32 白字  42 白底  >2 白色背景   2 变淡
33 蓝字  43 蓝底  >3 蓝色背景   3 斜体
34 灰字  44 灰底  >4 灰色背景   4 下划线
35 黑字  45 黑底  >5 黑色背景   7 反白
36 白字  46 白底  >6 白色背景   8 不可见
```

37 蓝字 47 蓝底 >7 蓝色背景

带 '>' 的代码必须在最后。底色和背景颜色应该一致。这些代码可以一起用，只要用分号分开。例如，要在蓝色背景上显示白色文字：

```
:set t_me=^V<Esc>[0;32;43;>3m
:set t_se=^V<Esc>[0;32;43;>3m
:set t_ue=^V<Esc>[0;32;43;>3m
:set t_ZR=^V<Esc>[0;32;43;>3m
:set t_md=^V<Esc>[1;32;43;>3m
:set t_mr=^V<Esc>[7;32;43;>3m
:set t_so=^V<Esc>[0;31;43;>3m
:set t_us=^V<Esc>[4;32;43;>3m
:set t_ZH=^V<Esc>[3;32;43;>3m
```

如果在一个过滤命令中使用多个命令，例如，

```
:r! echo this; echo that
```

只有最后一个命令的输出能看到。要补救这一点，你可以把命令组合起来。具体的做法决定于你用的外壳程序 (这就是为何 Vim 不能自己这么做)。例如：

```
:r! (echo this; echo that)
:r! {echo this; echo that}
```

接受单个文件名的命令允许文件名的空格。不过能接受多个文件的命令就不行了。文件名内含的空格必须要用反斜杠来转换。

MorphOS 下的 Vim MorphOS

[本节主要由 Ali Akcaagac 负责]

关于 MorphOS 版本的最新消息：

    http://www.akcaagac.com/index_vim.html

问题

有些问题不是 MorphOS 的，而更多是和 Vim 和 UN*X 有关。在随机存储器里启动 Vim 时，它抱怨 MorphOS 给出的一个 nag requestor，只要忽略它就行。另一个问题是运行 Vim 时，有些插件会导致一些问题，这些都可以忽略。希望将来有人能够修正这些问题。

目前，要不理这些问题，你或者可以运行：

```
vim <要编辑的文件名>
```

或者，如果你想简单运行 Vim 而且还能享受到帮助文件等的助力，最好这么用：

```
vim --noplugins <当然你也可以加文件名>
```

安装

1) 请将 'VIM' 二进制码复制到 c:
2) 从以下位置得到 Vim 运行时代码包：

```
ftp://ftp.vim.org/pub/vim/amiga/vim62rt.tgz
```

然后把它解压到 MorphOS 安装的 'Apps' 目录。对我而言，这会建立如下的目录结构：

```
MorphOS:Apps/Vim/Vim62/...
```

3) (非常重要的是) 把下面代码加到你的 s:shell-startup。

```
;Begin VIM
Set VIM=MorphOS:Apps/Vim/Vim62
Assign HOME: ""
;End VIM
```

4) 把 '.vimrc' 文件复制到 s:

5) 在这个包里还有一个叫做 'color-sequence' 的文件。它会将 MorphOS 外壳设成能显示 ANSI 颜色。请将该文件复制到 s: 并且修改 s:shell-startup 成：

```
;Begin VIM
Set VIM=MorphOS:Apps/Vim/Vim62
Assign HOME: ""
Execute S:Color-Sequence
Cls
;End VIM
```

vim:tw=78:ts=8:noet:ft=help:norl:

os_beos.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar
译者：Willis
http://vimcdoc.sf.net

BeOS BeBox

这是一个适用于 BeOS 预览版本 2（也被称为 PR2）或以后版本的 Vim 5.1 移植版本。

本文包含了 Vim 的 BeBox/BeOS 版本的特殊之处。除了本文提到的之外，和一般的 Unix 版本的 Vim 行为大致相当。 os_unix.txt

1. 通用 beos-general
2. 编译 Vim beos-compiling
3. 终端的超时问题 beos-timeout
4. Unicode 还是 Latin1 beos-unicode
5. BeOS GUI beos-gui
6. $VIM 目录 beos-vimdir
7. 拖和放 beos-dragndrop
8. 单次启动还是多次启动 beos-launch
9. 字体 beos-fonts
10. meta 键修饰符 beos-meta
11. 鼠标键映射 beos-mouse
12. 色彩名称 beos-colors
13. 带 Perl 的编译 beos-perl

1. 通用 beos-general

一般，语法高亮用不同的前景色高亮不同的项目。这特别适用于深色背景和浅色前景的终端窗口。有些中等灰色的背景（比如 (r,g,b)=(168,168,168)）也不错。如果你用缺省的浅色背景和深色字符，那么倒转前景和背景的设置概念也许看起来会舒服许多。要这么做，把以下的内容加到你的 .vimrc 文件里（其中的 <Esc> 需要被换成实际的 escape 字符）。

```
:if &term == "beos-ansi"
:    set t_AB=<Esc>[3%dm
:    set t_AF=<Esc>[4%dm
:endif
```

2. 编译 Vim beos-compiling

从 Advanced Access Preview Release (AAPR) 预览版本开始，Vim 可以用标准的 configure 脚本进行配置。要设置正确的编译器及其标志位，在外壳上应用下面的命令行（你可以剪贴到命令行上）：

```
CC=$BE_C_COMPILER CFLAGS="$BE_DEFAULT_C_FLAGS -O7" \
    ./configure --prefix=/boot/home/config
```

$BE_C_COMPILER 通常是 "mwcc"，$BE_DEFAULT_C_FLAGS 通常是 "-I- -I."

如果已经运行过 configure，你又想加上 GUI 支持，那你必须编辑 config.mk 文件，使得含有 GUI_xxx 的行指向 $(BEOSGUI_xxx) 而不是 $(NONE_xxx)。
或者，你也可以在 Makefile 里作此改动；这样的效果更持久一点。找找看 "NONE_"。

编译以后你需要把资源加进你的可执行程序。把以下几行加到 link.sh 脚本接近尾部的地方（在 "exit $exit_value" 一行之前），就可以自动执行这一任务。

```
rmattr BEOS:TYPE vim
copyres os_beos.rsrc vim
mimeset vim
```

同时，建立一个不干任何事的文件 "strip":

```
#!/bin/sh
mimeset $1
exit 0
```

用 "make install" 安装 Vim 的时候你需要它，

现在，输入 "make" 来编译 Vim，然后 "make install" 来安装它。

如果你想手动安装 Vim，你需要把 Vim 复制到 $HOME/config/bin 里，然后建立若干指向它的符号链接（{g,r,rg}{vim,ex,view}）。进一步，你需要把 Vim 的配置文件 vim-5.0s/{*.vim,doc,syntax} 复制到 $HOME/config/share/vim。为了完整起见，你还需要把 nroff 的手册页面复制到 $HOME/config/man/man1。不要忘了 ctags/ctags 和 xxd/xxd！

明显地，你需要用实际没有限制的连接器实际连接 Vim。参见 http://www.metrowerks.com 了解如果购买 BeOS 的 CodeWarrior 编译器。目前还没有其他可用的连接器。

即使你有合适的文件，以上步骤并不能包含 Perl 或者 Python 接口。 beos-perl

3. 终端的超时问题 beos-timeout

因为有的 POSIX/UNIX 功能还未实现[1]，终端里的带超时的读操作还没有直接的操作系统支持。这意味着除非你设置了 :set notimeout，你不能用 :mapping 等映射多于一个字符。 'timeout'

要避免这个问题，我提供了一个临时解决办法。用一个额外的线程预读一个字符，从而提供必需的带超时的输入。不过，有一个附带的后果就是 Vim 会识别到终端窗口的大小调整。

终端里不支持功能键，因为它们产生无法识别的字符序列。

这些问题在 GUI 里并不存在。

[1]: 没有文件描述符上的 select() 例程；另外 终端输入/输出接口 VMIN 和 VTIME 的设置也不正确。这一问题从 DR7 开始就存在，至少在 PR2 里还没有解决。

4. Unicode 还是 Latin1 beos-unicode beos-utf8

BeOS 用 Unicode 和 UTF-8 来编码文本字符串 (16-位字符编码成 8-位字符)。Vim 假定你用 ISO-Latin1 或者其他 8-位 字符代码。这对非 ASCII 的字符会产生意外的结果。试试命令 :digraphs 你就知道了。如果看起来一团乱，用 :set isprint=@ 可以少许改进对 ISO-Latin1 128-255 字符的显示。如果你有合适的字体 (见后)，GUI 的情况会好些。

你可以用 /boot/bin/xtou 命令把 ISO-Latin1 文件 转换成 UTF-8 文件 (xtou -f iso1 filename) 或者相反 (xtou -t iso1 filename)。

5. BeOS GUI beos-gui

不再包含 BeOS GUI 的代码。有相当长的时间没有人维护，很有可能已经不能用。如果你需要，从 Vim 6.x 版本中合并进相应的代码。

6。$VIM 目录 beos-vimdir

$VIM 代表 Vim 支持文件所在的位置。它的缺省值在编译时被确定，可以如下得到

```
:version
```

一般，它的值是 /boot/home/config/share/vim。如果你不喜欢，你可以设定 VIM 环境变量来改变缺省的设定，或者在你的 .vimrc 里设定 'helpfile':

```
:if version >= 500
:    set helpfile=~/vim/vim54/doc/help.txt
:    syntax on
:endif
```

7. 拖和放 beos-dragndrop

你可以把文件和目录拖放在 Vim 图标上 (从而启动一个新的 Vim 会话，除非你用文件类型应用把 Vim 设为 "单次启动") 或者放在 Vim 窗口上 (从而开始编辑文件)。拖放一个目录则改变 Vim 的当前工作目录为此目录。 :cd :pwd 如果你在拖放的时候按住 SHIFT，Vim 把当前目录改变为放置的第一个项目所在的目录。如果启动新的 Vim，那就不需要按 shift 了: Vim 会做同样的事情。

放置文件时设置当前的参数列表。 argument-list

8. 单次启动还是多次启动 beos-launch

发布时，Vim 的应用程序标志位 (在文件类型设定里) 设为多次启动。如果你愿意，你可以把它们改为单次启动。这样，试图启动第二个 Vim 程序时会用第一个 Vim 打开相应的文件。这在 Tracker 和命令行下都适用。后者不支持非文件 (选项) 的参数。

NB: 只有 GUI 版本 有 BApplication (从而有应用程序标志位)。这一节不适用于没有 GUI 的版本，如果你自己编译一个的话。

9. 字体 beos-fonts

以下方法可以设定字体

```
:set guifont=Courier10_BT/Roman/10
```

其中，第一部分是字体家族，第二部分是风格，第三部分是字体大小。在字体家族和风格里，你可以用下划线代替空格。

等宽字体（例如 Courier）提供最好的效果。Vim 试图对所有的字体应用 B_FIXED_SPACING 模式，不过看起来这对比例字体没有效果（即使 BeBook 说应该有）。

Vim 也试图用 B_ISO8859_1 编码，也被称为 ISO Latin 1 编码。这并不对所有字体适用。比如，它适用于 Courier，但不适用于 ProFontISOLatin1/Regular（很奇怪）。你可以通过以下命令验证这点，

```
:digraphs
```

它列出一些字符和它们的 ISO Latin 1 编码。如果，其中有 "方框" 字符，或者最后一个字符不是加点的 y，那么这个编码不适用于这个字体。

如果你指定的字体不存在，会用系统的等宽字体代替。

标准等宽的系统字体是：

```
ProFontISOLatin1/Regular
    Courier10_BT/Roman
    Courier10_BT/Italic
    Courier10_BT/Bold
    Courier10_BT/Bold_Italic
```

标准比例的系统字体是：

```
      Swis721_BT/Roman
      Swis721_BT/Italic
      Swis721_BT/Bold
      Swis721_BT/Bold_Italic
Dutch801_Rm_BT/Roman
Dutch801_Rm_BT/Italic
Dutch801_Rm_BT/Bold
Dutch801_Rm_BT/Bold_Italic
   Baskerville/Roman
   Baskerville/Italic
   Baskerville/Bold
   Baskerville/Bold_Italic
 SymbolProp_BT/Regular
```

为了好玩，试试其中的一些字体吧。

10. meta 键修饰符 beos-meta

你可以用左右的 OPTION 键来模拟 META 键修饰符。这是因为 ALT（也就是 COMMAND）键不会传给应用程序。

11. 鼠标键映射 beos-mouse

Vim 把鼠标按钮分别称为 LeftMouse、MiddleMouse 和 RightMouse。如果你用缺省的鼠标设置，这些名字反映的就是事实。Vim 用以下的映射：

```
Button 1 -> LeftMouse,
Button 2 -> RightMouse,
Button 3 -> MiddleMouse。
```

如果你的鼠标没有三个按钮，你可以用鼠标键击和若干修饰符来定义自己的鼠标按钮。例见 swapmouse 包： gui-mouse-mapping
$VIMRUNTIME/pack/dist/opt/swapmouse/plugin/swapmouse.vim

12. 色彩名称 beos-colors

Vim 有一些内建的色彩名称。附加的名字则从 $VIMRUNTIME/rgb.txt 里取得。这个文件基本上是 X 里的色彩数据库。为了提高效率，这个文件里的名字被缓冲存贮。

13. 带 Perl 的编译 beos-perl

带 Perl 支持的编译需要一点技巧。Metrowerks 编译器在找寻包含文件的位置上想法有些与众不同。因为有些 Perl 的包含文件和有些 Vim 的头文件重名，错误的文件被包含了。要修正这个问题，运行如下在 vim-5.0/src 目录的 Perl 脚本：

```
preproc.pl > perl.h
```

```
#!/bin/env perl
# Simple #include expander, just good enough for the Perl header files.

use strict;
use IO::File;
use Config;

sub doinclude
{
    my $filename = $_[0];
    my $fh = new IO::File($filename, "r");
    if (defined $fh) {
        print "/* Start of $filename */\n";

        while (<$fh>) {
            if (/^#include "(.*)"/) {
                doinclude($1);
                print "/* Back in $filename */\n";
            } else {
                print $_;
            }
        }
        print "/* End of $filename */\n";

        undef $fh;
    } else {
```

```
            print "/* Cannot open $filename */\n";
            print "#include \"$filename\"\n";
        }
    }

    chdir     $Config{installarchlib}."/CORE";
    doinclude "perl.h";
```

它会只用其他的 Perl 头文件来扩展 "perl.h" 头文件。

现在，你可以用 --enable-perlinterp 选项 configure 和 make Vim。不过，要提醒的是，这会给 Vim 增加 616 千字节的大小！没有 Perl 的支持，带着缺省的特性和 GUI 的 Vim 大概是 575K 大小，而带 Perl 支持则有 1191K 之大！

-Olaf Seibert

[注意：以下地址已经不能用了:]
<rhialto@polder.ubc.kun.nl>
http://polder.ubc.kun.nl/~rhialto/be

vim:tw=78:ts=8:noet:ft=help:norl:

os_dos.txt For Vim version 8.2. 最近更新：2017年8月

VIM 参考手册 by Bram Moolenaar
译者：Dasn
http://vimcdoc.sf.net

dos DOS

本文描述 MS-DOS 和 Win32 版本共有的特性。你还可以参考
os_win32.txt 和 os_msdos.txt 。

1. 文件位置 dos-locations
2. 使用反斜杠 dos-backslash
3. 标准映射 dos-standard-mappings
4. 屏幕的输出与颜色 dos-colors
5. 文件格式 dos-file-formats
6. :cd 命令 dos-:cd
7. 中止键 dos-CTRL-Break
8. 临时文件 dos-temp-files
9. 默认的外壳选项 dos-shell

---

1. 文件位置 dos-locations

只要你把 Vim 的可执行文件 (vim.exe)、帮助文件的目录 (doc) 以及语法文件的目录 (syntax) 放到同一个目录下，Vim 就可以工作了，不需要额外的配置，也不需要修改注册表或添加环境变量，只要保证 Vim 目录包含在路径中，或者在桌面上做一个快捷方式也可以。

通常 Vim 资源文件 ("_vimrc" 和 "_gvimrc") 放在 Vim 运行目录的上一级目录。如果你想把它放在别的地方，可以设置环境变量指向资源文件所在的目录，例如：

```
set VIM=C:\user\piet
```

Vim 会在 "c:\user\piet\" 下寻找资源文件。

备注：多用户计算机需要这种功能，否则还是把 _vimrc 放在原来的地方简单些。

如果你改变了 vim.exe 的位置，就需要设置一下 $VIM 变量，这样相应的运行目录 (runtime directory) 就是 "$VIM/vim{version}"。例如：

```
set VIM=E:\vim
```

Vim 会在 "e:\vim\vim54" 寻找 5.4 版本运行时需要的文件。

备注：强烈建议不要这样做。最好把可执行文件和运行时需要的文件放到同一个目录里。

如果你把可执行文件和资源文件都移动了，那么你必须用 $VIM 和 $VIMRUNTIME 两个环境变量来分别指定资源文件 ("_vimrc" 和 "_gvimrc") 和运行目录的位置。例如：

```
set VIM=C:\usr\piet
set VIMRUNTIME=E:\vim\vim54
```

这样，Vim 可以在 "c:\user\piet\" 目录下找到资源文件，在 "E:\vim\vim54" 目录下找到运行时所需的文件。

要想了解更多内容，见 $VIM 和 $VIMRUNTIME 。

在 Windows 95 上，你可以在 C:\autoexec.bat 里设置 $VIM。例如：

```
set VIM=D:\vim
```

在 Windows NT 上，你可以在 "控制面板－系统" 或者右键单击 "我的电脑"－"属性"，

"环境变量" 选项卡里为每一个用户设置环境变量。

2.使用反斜杠 dos-backslash

在文件名（这里是指带路径的文件名 译者）里使用反斜杠（\）可能会出问题。在某些命令里，VI 要用两个反斜杠表示反斜杠。Vim 在这一问题上宽容一些，它不会从文件名中删除反斜杠，所以直接用 ":e c:\foo\bar" 就可以工作。但是当反斜杠出现在一些特殊字符（空格、逗号、反斜杠，等等）前面时，Vim 就会删除反斜杠。用斜杠来避免问题: ":e c:/foo/bar"，同样可以。Vim 会自动把斜杠替换成反斜杠以避免一些 MS-DOS 或 Win32 程序产生问题。

如果你喜欢用斜杠，就设置 'shellslash' 选项。Vim 就会在自动补全文件名的时候，用斜杠代替反斜杠。这在你使用 UNIX 风格的 'shell' 时，特别有用。

---

3. 标准映射 dos-standard-mappings

CTRL-PageUp 和 CTRL-PageDown 的原有映射已经被取消了，现在它们的功能是跳转到下一个或者前一个标签页 <C-PageUp> <C-PageDown>

如果你想要让它们移动到屏幕行的首行和末行，这样映射:

键	键码	正常/可视模式	插入模式
CTRL-PageUp	<M-N><M-C-D>	H	<C-O>H
CTRL-PageDown	<M-N>v	L$	<C-O>L<C-O>$

此外，下面这些键用于复制/剪切/粘贴。Win32 版本和 DJGPP 版本使用剪贴板。

Shift-Insert	粘贴文本（从剪贴板）	<S-Insert>
CTRL-Insert	复制选择的文本（到剪贴板）	<C-Insert>
CTRL-Del	剪切选择的文本（到剪贴板）	<C-Del>
Shift-Del	剪切选择的文本（到剪贴板）	<S-Del>
CTRL-X	剪切可视文本（到剪贴板）	

这些键是这样映射的（Win32 和 DJGPP 版本):

键	键码	正常	可视	插入
Shift-Insert	<M-N><M-T>	"*P	"-d"*P	<C-R><C-O>*
CTRL-Insert	<M-N><M-U>		"*y	
Shift-Del	<M-N><M-W>		"*d	
CTRL-Del	<M-N><M-X>		"*d	

或这样映射（非 Win32 版本):

键	键码	正常	可视	插入
Shift-Insert	<M-N><M-T>	P	"-dP	<C-R><C-O>"
CTRL-Insert	<M-N><M-U>		y	
Shift-Del	<M-N><M-W>		d	
CTRL-Del	<M-N><M-X>		d	
CTRL-X	<C-X>		"*d	

如果系统支持剪贴板，Vim 就会使用 "* 寄存器。

4. 屏幕的输出与颜色 dos-colors

默认的写屏操作是直接调用 bios 完成的，这在大多数系统上可以工作。你不需要用 ansi.sys。你可以使用 ":mode" 命令设置当前屏幕模式。见 :mode 。

要想改变 Vim 的颜色，可以使用 :highlight 命令。Normal 高亮组 (highlight group) 决定了 Vim 用什么颜色显示普通的文本。例如，要设置灰色字体和蓝色背景：

```
:hi Normal ctermbg=Blue ctermfg=grey
```

要了解其它可用的高亮组，参见 highlight-groups 。

DOS 控制台不支持 "粗体"、"下划线" 之类的属性。你可以用九个终端选项来设置五种属性值的颜色。备注：因为你可以用 ":highlight" 命令直接设置颜色，这些选项并没有必要。它们是为了兼容老版本的 Vim 而保留的。 'highlight' 选项指定每种行为使用五种模式中的哪一种。九个终端选项如下：

```
:set t_mr=^V^[\|xxm          反转模式开始
:set t_md=^V^[\|xxm          粗体模式开始
:set t_me=^V^[\|xxm          返回正常文本

:set t_so=^V^[\|xxm          突出模式开始
:set t_se=^V^[\|xxm          返回正常文本

:set t_us=^V^[\|xxm          下划线模式开始
:set t_ue=^V^[\|xxm          返回正常文本

:set t_ZH=^V^[\|xxm          斜体模式开始
:set t_ZR=^V^[\|xxm          返回正常文本
```

^V 代表 CTRL-V
^[代表 <Esc>

(在输入上面的命令时，按下 CTRL-V Vim 只会显示 ^，这时不要输入 V，直接输入 <Esc> 即可。译者)

你必须把上面命令里的 xx 换成相应的十进制数，这个数是前景色与背景色相加的和：

颜色	前景色	背景色
黑色	0	0
暗蓝色	1	16
暗绿色	2	32
灰绿色	3	48
暗红色	4	64
暗紫红色	5	80
棕色，暗黄色	6	96
亮灰色	7	112
暗灰色	8	128 *
蓝色，天蓝色	9	144 *
绿色，浅绿色	10	160 *
青绿色，兰绿色	11	176 *
红色，亮红色	12	192 *
紫红色，亮紫红色	13	208 *
黄色，浅黄色	14	224 *
白色	15	240 *

* 在某些显示模式下，大于 128 的颜色代码可能不起作用，并且代码 128 会使字体闪烁。

代码 0 表示颜色恢复到起始状态（起始状态的值通常是 7，亮灰色文字，黑色背景，但你可以修改。如果你在命令行提示上修改了缺省的颜色，你可能需要在 vimrc 文件里调整一些高亮的颜色--见下面）。
这就是 't_me' 的初始值。

下面是 Vim 默认高亮模式：

```
        t_mr    112     反转模式：黑色文字 (0)，亮灰色背景 (112)
        t_md     15     粗体模式：白色文字 (15)，黑色背景 (0)
        t_me      0     正常模式：正常模式 (返回默认状态)

        t_so     31     突出模式：白色文字 (15)，暗蓝色背景 (16)
        t_se      0     突出模式结束 (返回默认状态)

        t_czh   225     斜体模式：暗蓝色文字 (1)，黄色背景 (224)
        t_czr     0     斜体模式结束 (返回默认状态)

        t_us     67     下划线模式：灰绿色文字 (3)，暗红色背景 (64)
        t_ue      0     下划线模式结束 (返回默认状态)
```

之所以选择这些颜色，是因为它们在使用反转显示的时候看起来比较舒服，但是你可以改成你喜欢的颜色。

例如：

```
:set t_mr=^V^[\|97m   " 启动反转模式：暗蓝色文字 (1) 棕色背景 (96)
:set t_md=^V^[\|67m   " 启动粗体模式：灰绿色文字 (3) 暗红色背景 (64)
:set t_me=^V^[\|112m  " 返回正常模式：黑色文字 (0) 亮灰色背景 (112)
:set t_so=^V^[\|37m   " 启动突出模式：暗紫红色文字 (5) 暗绿色背景 (32)
:set t_se=^V^[\|112m  " 返回正常模式：黑色文字 (0) 亮灰色背景 (112)
```

---

5. 文件格式 dos-file-formats

如果 'fileformat' 设置为 "dos" (这是默认设置)，Vim 把 <NL> 或 <CR><NL> 当作行尾符 (<EOL>)。写入文件的时候，Vim 使用 <CR><NL>。这样，如果你编辑一个文件并且保存了，那么 Vim 会把所有的 <NL> 替换成 <CR><NL>。

如果 'fileformat' 设置为 "unix"，Vim 则使用一个 <NL> 作为 <EOL>，并且把 <CR> 显示为 ^M。

你可以用 Vim 把 <NL> 替换成 <CR><NL>：在任何模式下读，保存成 Dos 格式 (":set ff=dos")。

你可以用 Vim 把 <CR><NL> 替换成 <NL>：在 Dos 模式下读，保存成 Unix 格式 (":set ff=unix")。

如果 'fileformats' 非空 (这是默认值)，Vim 会自动设置 'fileformat' 的值，因此你没有必要担心自己究竟在做什么。

'fileformat' 'fileformats'

如果你要编辑一个脚本文件或者一个二进制文件，那么你应该在装载这个文件之前预先设定 'binary' 选项。脚本和二进制文件可能包含 <NL>，Vim 要把它替换成 <CR><NL>。你
可以在启动时加上 "-b" 参数，这样可以自动置位 'binary' 选项。

---

6. :cd 命令 dos-:cd

":cd" 命令能识别驱动器符号，并且可以改变当前的驱动器。命令 ":cd c:" 使驱动器 C 成为当前驱动器。命令 ":cd d:\foo"，切换当前目录到 "D 盘的 foo 目录下"。Vim 也能识别 UNC 路径。例如: ":cd \\server\share\dir"。 :cd

---

7. 中止键 dos-CTRL-Break

用 CTRL-Break 来代替 CTRL-C 中止查找操作。Vim 只有在接受输入时才能检测到 CTRL-C。

---

8. 临时文件 dos-temp-files

对于 16 位和 32 位的 DOS 版本:
Vim 把临时文件 (用于过滤) 存放于下列目录中 (按顺序查找下列目录，一旦发现该目录存在并且可以创建文件，就用它存放临时文件):

```
$TMP
$TEMP
C:\TMP
C:\TEMP
当前目录
```

对于 Win32 版本 (控制台和 GUI):
Vim 用标准的 Windows API 函数来获取临时文件名 (用于过滤)。按顺序查找下列目录，一旦发现该目录存在并且可以创建文件，就用它存放临时文件:

```
$TMP
$TEMP
当前目录
```

---

9. 默认的外壳选项 dos-shell

在 Windows 95 上，默认的 'sh' ('shell') 选项是 "command.com"，而在 Windows NT 上是 "cmd.exe"。如果设置了环境变量 SHELL，Vim 就使用 SHELL 指定的程序，如果没有设置 SHELL，而设置了环境变量 COMSPEC，Vim 就使用 COMSPEC。Vim 用 "<shell> /c <command_name>" 执行外部命令。输入 CTRL-Z 启动一个命令子外壳。要返回 Vim 输入 "exit"。 'shell' CTRL-Z

如果你运行一个第三方的外壳，你可能需要自己设置一下 'shellcmdflag' ('shcf') 和 'shellquote' ('shq') 或者 'shellxquote' ('sxq') 选项。不幸的是，这些选项依赖于你所使用的 Vim 的版本。例如，要用 MKS Korn shell 或 bash，这些属性的值应该设置成:

DOS 16 位 DOS 32 位 Win32

'shellcmdflag'	-c	-c	-c
'shellquote'	"		
'shellxquote'			"

16 位的 Dos 版本这样启动外壳:

```
        <shell> -c "命令" >file
```

Win32 版本:

```
        <shell> -c "命令 >file"
```

32 位 DOS 版本，DJGPP 在内部执行。

Vim 启动的时候检查 'shell' 选项的值是否能匹配 "sh" 子串 (在任何位置)。如果是，Vim 就按照上面的描述设置 'shellcmdflag' 和 'shellquote' 或 'shellxquote' 选项。

vim:tw=78:ts=8:noet:ft=help:norl:

os_mac.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by Bram Moolenaar et al.
译者：iCrazy<icrazy@ustc.edu>，Willis，tocer
http://vimcdoc.sf.net

mac Mac macintosh Macintosh

本文件阐述 Vim 的 Macintonsh 版本的特性。

注意：本文档已经略有过时。在这里你可以找到更多有用的信息：
http://macvim.org

1. 文件名约定 mac-filename
2. .vimrc 和 .vim 文件 mac-vimfile
3. 标准映射 mac-standard-mappings
4. FAQ mac-faq
5. 已知的不足之处 mac-lack
6. Mac 错误报告 mac-bug
7. 编译 Vim mac-compile
8. Darwin 特性 mac-darwin-feature

Vim 的 3.0 版有一个移植到 Mac 的版本。这是当时文档中开始的几句话：

VIM Release Notes
Initial Macintosh release, VIM version 3.0
19 October 1994

Eric Fischer
<enf1@midway.uchicago.edu>, <eric@jcp.uchicago.edu>, <etaoin@uchicago.edu>
5759 N. Guilford Ave
Indianapolis IN 46220 USA

---

1. 文件名约定 mac-filename

从 Vim 版本 7 开始，你只能用 unix 的路径分隔符。为了确定指定的文件名是与当前目录相关（相对路径），还是绝对路径（即：和 "桌面" 相关），我们使用了如下的算法：

如果路径以 "/" 开始，则为绝对路径
如果路径以 ":" 开始，则为相对路径
如果路径不以 "/" 和 ":" 开始，但是 ":" 在 "/" 之前出现，则为绝对路径

```
:e /HD/text
:e HD:text
```
编辑磁盘 "HD" 中的文件 "text"
```
:e :src:main.c
:e src/main.c
```
编辑当前目录下子目录 "src" 中的文件 "main.c"
```
:e os_mac.c
```
编辑当前目录下的文件 "os_mac.c"。

您可以使用变量 $VIM 和 $VIMRUNTIME

```
:so $VIMRUNTIME:syntax:syntax.vim
```

---

2. .vimrc 和 .vim 文件 mac-vimfile

对于 Vim 脚本，建议使用 Unix 风格的换行符，也就是单个 NL 字符。

当 Vim 启动时，它会载入 $VIMRUNTIME/macmap.vim 脚本以定义默认的命令键 (command-key) 映射。

在旧版系统上，一般应避免使用以 "." 开头的文件，所以那些 rc 文件被命名为 "vimrc" 或 "_vimrc"，以及 "gvimrc" 或 "_gvimrc"。这些文件可以是任何格式 (mac、dos 或 unix)。如果 'nocompatible' 选项打开，Vim 可以处理任何文件格式，否则它只能处理 mac 格式的文件。

---

3. 标准映射 mac-standard-mappings

以下映射可用于剪切/复制/粘贴到剪贴板。

键	普通	可视	插入	描述	
Command-v	"*P	"-d"*P	<C-R>*	粘贴文本	<D-v>
Command-c		"*y		复制可视文本	<D-c>
Command-x		"*d		剪切可视文本	<D-x>
Backspace		"*d		剪切可视文本	

---

4. Mac FAQ mac-faq

在互联网上: http://macvim.org/OSX/index.php#FAQ

问题: 我不能在 Apple 终端输入非 ASCII 的字符。
回答: 在 Windows Setting，Emulation，确信没有选上 "Escape non-ASCII characters"。

问题: 怎么从命令行启动 GUI?
回答: 假设 Vim.app 存放在 /Applications 中:

```
open /Applications/Vim.app
```

或:

```
/Applications/Vim.app/Contents/MacOS/Vim -g {arguments}
```

问题: 如果我用 GUI 或 open 启动 Vim.app，如何合理设置 $PATH?
回答: 下面这个窍门能用于多数外壳。把它放到 vimrc 文件中。macvim.org 发布的二进制发行包中带有的系统 vimrc 文件已经包含。

```
let s:path = system("echo echo VIMPATH'${PATH}' | $SHELL -l")
let $PATH = matchstr(s:path, 'VIMPATH\zs.\{-}\ze\n')
```

---

5. 已知的不足之处 mac-lack

在终端中，CTRL-^ 需要用 Shift-Control-6 来输入。而 CTRL-@ 则用 Shift-Control-2 输入。

---

6. Mac 错误报告 mac-bug

当报告任何与 Mac 平台相关的 bug 或特性时，请使用 vim-mac 邮件列表 vim-mac 。然而，您必须先订阅它。另一个办法就是发邮件给当前的 MacVim 维护者：

mac@vim.org

---

7. 编译 Vim mac-compile

见伴随源程序的文件 "src/INSTALLmac.txt"。

---

8. Darwin 特性 mac-darwin-feature

如果你的 Mac 不是太老，你运行的会是 OS X，也叫 Darwin。Darwin 之前的最后的 OS 版本是 Mac OS 9。Darwin 特性意味着 Vim 能使用 Darwin 特定的属性。

此特性可以完成两重目的：

- Vim 可以和系统剪贴板互动。
- 给 Vim 引入转换模块，为一些平台特定的字符编码和 Vim 所知编码之间弥合差异。

不需要讲，任何像样的文本编辑器要在同一桌面环境下和其它应用友好互动，没有这两条是不行的。

Vim 不是专为 macOS 设计的应用，需要额外的特性来使 macOS 的用户享受其它平台用户相同的用户体验。

为简明起见，此特性称为 "darwin"，用来表示它是专为此特定平台提供的 Vim 特性。

此特性是个配置选项。亦即，它的打开与否是在编译时间决定的；一旦选择打开，编译时带入就不能在运行时关闭。

缺省打开此特性。对于多数 macOS 用户，这就足够了，除非他们有以下简要说明的特殊需要。

要关闭此特性，给 configure 脚本传递 --disable-darwin ：

```
./configure --disable-darwin <other options>
```

然后运行 make 来编译 Vim。选项的顺序无关紧要。

要在运行时确定是否编译进 darwin 特性，可用 'has('osxdarwin')'，如果编译时有此特性，返回 1；否则返回 0。为了后向兼容，继续可以用 'macunix' 代替 'osxdarwin'。

值得提出的 --disable-darwin 实际有用的用例包括：

- 想用 `x11-selection` 而不是系统剪贴板。
- 想用 `x11-clientserver` 。

因为两者都需要 `X11` 客户间通信才能正常工作，而该通信机制和系统剪贴板时有冲突，可以通过关闭 `darwin` 特性以防止 `Vim` 运行时挂起。

vim:tw=78:ts=8:noet:ft=help:norl:

os_mint.txt For Vim version 8.2. 最近更新: 2005年3月

VIM 参考手册 by Jens M.Felderhoff
译者: iCrazy<icrazy@ustc.edu> http://vimcdoc.sf.net

MiNT Atari

本文包含了 Atari MiNT 的 Vim 版本的特性。

如果您要为运行 MiNT 的 Atari 机器编译 Vim，您需要首先看一下 src 目录下的 "INSTALL" 和 "Makefile" 目录。

Vim MiNT 版本的行为和它的 Unix 版本几乎是一样的。
如果没有明确声明，文档描述的 Unix 上 Vim 的行为也适用于 MiNT 上的 Vim 版本。

为了通配符 <~> (主目录) 的扩展，你需要一个支持波浪符扩展的外壳。原始的 Bourne Shell 不能识别，而 "csh" 和 "ksh" 都没问题。

Vim 的 MINT 版本需要支持你的终端的终端能力的 termcap 文件 /etc/termcap。内建的 termcap 支持 vt52 终端。而支持 TOSWIN 窗口管理器和虚拟控制台终端的 termcap 入口项已经被附加到该 termcap 文件中，并随着 Vim 一起发布。

万一您遇到了 <BS> 和 <Del> 键被互换的问题，请看 :fixdel 。

MiNT 上的终端刷新常常比较慢 (比如串行终端)，所以默认情况下的 'showcmd' 选项和 'ruler' 选项都是关闭的。如果你所使用的终端比较快的话，可以把它们打开。甚至可以试试 'ttyfast' 选项。

如果您发现了错误，请发送报告至

Jens M. Felderhoff，电邮: <jmf@infko.uni-koblenz.de>

vim:tw=78:ts=8:noet:ft=help:norl:

os_msdos.txt For Vim version 8.2. 最近更新: 2017年8月

VIM 参考手册 by Bram Moolenaar
译者: Dasn, tocer
http://vimcdoc.sf.net

msdos ms-dos MSDOS MS-DOS

本文过去描述 MS-DOS 版本 Vim 的特点。在 7.4.1399 补丁中删除了 MS-DOS 支持。如果你还想要用，需要在此之前的版本。注意 MS-DOS 版本不能工作，内存不够。DOS32 版本（用 DJGPP）在较旧的系统上可能还能用。

vim:tw=78:ts=8:noet:ft=help:norl:

os_os2.txt For Vim version 8.2. 最近更新: 2017年8月

VIM 参考手册 by Paul Slootman
译者: Willis
http://vimcdoc.sf.net

os2 OS2 OS/2

本文过去讨论 OS/2 版本的 Vim 的特点。

patch 7.4.1008 删除了 OS/2 的支持。

vim:tw=78:ts=8:noet:ft=help:norl:

os_qnx.txt For Vim version 8.2. 最近更新: 2006年2月

VIM 参考手册 by Julian Kinraid
译者: Willis
http://vimcdoc.sf.net

QNX qnx

1. 概述 qnx-general
2. 编译 Vim qnx-compiling
3. 终端支持 qnx-terminal
4. Photon GUI photon-gui
5. Photon 字体 photon-fonts
6. 漏洞和待做的事情

---

1. 概述 qnx-general

QNX 上的 Vim 和其他的 unix 版本行为大致相同。 os_unix.txt

2. 编译 Vim qnx-compiling

Vim 可以用通常的 configure/make 方法编译。如果你要编译 X11 的版本，在配置 (configure) 时加入 --with-x 选项。否则，不需要任何参数。或者，加入 --enable-gui=photon 以得到 Photon GUI 的支持。执行 ./configure --help，看看还有什么特性你希望打开或者关闭。

3. 终端支持 qnx-terminal

如果鼠标和剪贴板选项在编译时加入的话 (通常如此)，Vim 可以提供 pterm 这两方面的支持。

影响鼠标设置的选项有 'mouse' 和 'ttymouse'。在使用鼠标时，如果你按住 shift、ctrl、或是 alt，pterm 自己会处理鼠标事件。它会选择文本，和 Vim 所做的无关。

如果鼠标可以使用，你按住 Alt-RightMouse (Alt + 鼠标右键) 可以打开 pterm 菜单。要关闭 Vim 的鼠标支持，把 mouse 选项置空，即 set mouse=

4. Photon GUI photon-gui

要启动 Vim 的 GUI 版本，你可以运行 gvim 或者 vim -g。否则运行的是终端版本。更多的信息可见 - gui-x11-start

支持的特性有:

:browse 命令	:browse
:confirm 命令	:confirm
光标闪烁	'guicursor'

菜单、弹出式菜单和菜单优先顺序 :menu
popup-menu
menu-priority

工具栏 gui-toolbar
'toolbar'

字体选择工具 (:set guifont=*) photon-fonts
鼠标聚焦 'mousefocus'
鼠标隐藏 'mousehide'
鼠标图标形状 'mouseshape'
剪贴板 gui-clipboard

还未完成的特性:
若干国际化支持，包括波斯语和希伯来文字的支持，
不同的编码，等等。

本帮助文件

不支持的特性:
查找和替换窗口 :promptfind
可卸的菜单

其他我还没有想及的恕难加入

5. Photon 字体 photon-fonts

你可以在 gui 中用 guifont 选项设置字体

```
:set guifont=Lucida\ Terminal
```

该字体须是等宽字体，字体名如有空格，应以 '\' 转义之。缺省字体是 PC Terminal, 8 号字。用 '*' 作为字体名会打开一个标准的 Photon 字体选择工具，然后你可以从中选择一个字体。

在字体名后面，你可以附加可选的控制字体大小和风格的选项，设置之间用 ':' 分隔。不是所有的字体都支持所有的风格。

这些选项是，

s{size}	字体大小 {size}
b	粗体字
a	抗锯齿字体
i	斜体字

示例:

设定 monospace 10号抗锯齿字体

```
:set guifont=monospace:s10:a
```

设定 Courier 12号粗体斜体字

```
:set guifont=Courier:s12:b:i
```

从选择工具挑选字体

```
:set guifont=*
```

6. 漏洞和待做的事情

已知问题:

- Vim 有时在执行外部程序时不响应。绕开的方法是把下面这行加入到你的 vimrc 文件中:

```
set noguipty
```

漏洞:

- 菜单高亮还有些许问题
- 在使用 phiditto/phinows 等的时候，如果你用的字体不支持粗体属性而 Vim 要显示粗体字，情况就会弄乱。
- 光标有时候很难看得到。
- 一些我可以解决的小问题 :)

待做:

- 改进多语言支持。
- 在菜单和工具栏中加入设置字体的选项。
- 查找和替换对话框。
- 客户/服务器方面的功能。
- 也许要做可卸下的菜单。

- 在启动外部程序时，不要使用带 spawn() 的 fork()。

vim:tw=78:sw=4:ts=8:noet:ft=help:norl:

os_risc.txt For Vim version 8.2. 最近更新: 2011年5月

VIM 参考手册 by Thomas Leonard
译者: Willis
http://vimcdoc.sf.net

riscos RISCOS RISC-OS

RISC OS 的支持在 Vim patch 7.3.187 中被删除。
如果你想在 RISC OS 中使用 Vim，请获取该补丁之前的版本。

vim:tw=78:ts=8:noet:ft=help:norl:

VIM 参考手册 by Bram Moolenaar

译者: yemao http://vimcdoc.sf.net

unix Unix

本文件介绍了 Unix 版本的 vim 的一些特殊之处。

首先，要在 Unix 上编译 vim，请参考 src 目录下的 "INSTALL" 文件和 "Makefile" 文件。

默认帮助文件名为 "/usr/local/lib/vim/help.txt"
同时，vim 会使用 "$HOME/.vimrc" 和 "$HOME/.exrc" 而不是 "s:.vimrc" 和 "s:.exrc"。另外它会首先使用 /usr/local/etc/vimrc 文件。注意，如果 "/usr/local/share" 已经存在，就会使用它来代替 "/usr/local/lib"。

临时文件（用于过滤程序）默认会放在 "/tmp" 下，如果你想把它们放在别处，那么必须设置环境变量 $TMPDIR 为你放临时文件的目录。

通过扩展通配符，你可以使用 '~'（用户主目录）及 '$'（系统环境变量）。

fork spoon

执行外部命令时，会尽可能调用 fork()/exec()，不然会调用 system()。使用后者速度会有所减慢。正因为有两个函数可以被调用，":version" 输出信息就有两种，当调用 fork()/exec() 时，输出 +fork，否则输出 +system()。编译时可以修改。
(GUI 上关于 fork（进程分叉）的详细信息参考 gui-fork。)

Unix 上的终端刷新常常比较慢（比如串行终端和 suntools 下的外壳窗口），所以默认情况下的 'showcmd' 选项和 'ruler' 选项都是关闭的。如果你所使用的终端比较快的话，可以把它们打开。甚至可以试试 'ttyfast' 选项。

在 xterm 下使用 Vim，只要把 'mouse' 选项设成 "a" 就可以使用鼠标了。如果你有权限使用 X-server，当你用鼠标拖动的时候，可以进行 gui 风格的复制和粘贴，而你还会看到可视反馈。如果你还是想用 xterm 的复制和粘贴，使用鼠标时按住 shift 键，具体请参考 mouse-using。只要你的 xterm 版本够新，设置一下 'ttymouse' 选项就可以在拖动的时候得到可视反馈。

terminal-colors

要在 Vim 里显示彩色，可以参考以下例子（假定你的终端支持色彩显示，而 "T_Co" 为空或零）:

```
:set t_me=^[[0;1;36m     " 正常模式 (撤销 t_mr 和 t_md)
:set t_mr=^[[0;1;33;44m  " 反转 (逆转) 模式
:set t_md=^[[1;33;41m    " 粗体模式
:set t_se=^[[1;36;40m    " 突出模式结束
:set t_so=^[[1;32;45m    " 突出模式开始
:set t_ue=^[[0;1;36m     " 下划线模式结束
:set t_us=^[[1;32m       " 下划线模式开始
```

[注意，^[是 <Esc>，可以用 CTRL-V <Esc> 来输入。]

而在真正的彩色终端下，可以使用 ":highlight" 命令。

另外，文件 "tools/vim132" 是一个外壳脚本。在 vt100 和与其类似的终端下，它可以使 Vim 使用 132 列模式。

vim:tw=78:ts=8:noet:ft=help:norl:

os_vms.txt For Vim version 8.2. 最近更新：2020年1月

VIM 参考手册

译者：Willis
http://vimcdoc.sf.net

VMS vms

本文件讨论 Vim 的 VMS 版本的特点。你可以在 Vim 命令提示下输入 :help VMS 得到本帮助文件。

1. 准备开始 vms-started
2. 下载文件 vms-download
3. 编译 vms-compiling
4. 问题 vms-problems
5. 布署 vms-deploy
6. 实际使用方法 vms-usage
7. GUI 模式的问题 vms-gui
8. 有用的提示 vms-notes
9. VMS 相关改动 vms-changes
10. 作者 vms-authors

---

1. 准备开始 vms-started

Vim (Vi IMproved) 是一个 Vi-兼容的文本编辑器，它能在人类知道的差不多一切操作系统上运行。现在在 OpenVMS 上也可以了，不管是文本方式或者 X/Motif 方式。它拥有完整的功能，和其他操作系统上的 Vim 也绝对兼容。

---

2. 下载文件 vms-download

你可以从官方 Vim 站点用 ftp 下载 Vim 源代码：

```
ftp://ftp.vim.org/pub/vim/
```

或者使用某个镜像站点：

```
ftp://ftp.vim.org/pub/vim/MIRRORS
```

你可以从这里得到编译好的二进制文件：

```
http://www.polarhome.com/vim/
ftp://ftp.polarhome.com/pub/vim/
```

要使用预编译的二进制版本，你需要如下档案之一：

vim-XX-exe-ia64-gui.zip	IA64 GUI/Motif 可执行文件
vim-XX-exe-ia64-gtk.zip	IA64 GUI/GTK 可执行文件
vim-XX-exe-ia64-term.zip	IA64 控制台可执行文件
vim-XX-exe-axp-gui.zip	Alpha GUI/Motif 可执行文件
vim-XX-exe-axp-gtk.zip	Alpha GUI/GTK 可执行文件
vim-XX-exe-axp-term.zip	Alpha 控制台可执行文件
vim-XX-exe-vax-gui.zip	VAX GUI 可执行文件
vim-XX-exe-vax-term.zip	VAX 控制台可执行文件

当然还有（可选）
  vim-XX-runtime.zip　　运行时文件

二进制档案包括：vim.exe、ctags.exe、xxd.exe 等文件。

GTK 可执行文件需要 GTKLIB，后者在 Alpha 和 IA64 平台上可以得到。

---

3. 编译　　vms-compiling

参见 [.SRC]INSTALLVMS.TXT 文件。

---

4. 问题　　vms-problems

代码经过 Open VMS 6.2 - 8.2 在 Alpha、VAX 和 IA64 平台以及 DEC C 编译器测试。应该不会有大问题。如果你的系统缺少一些库，你可以调整 OS_VMS_CONF.H 文件。

如果你决定编译 Vim 时加上 +perl，+python 等选项，首先你需要下载 OpenVMS 的 Perl 和 Python 的发布版本。编译并布署需要的库，并修改 MAKE_VMS.MMS 文件的若干行。在 Vim 这里，应该没有问题。

GTK、XPM 库的路径也应该在 MAKE_VMS.MMS 里配置。

注意：在 VAX 上，用 DEC C 编译器是没有问题的。VAX C 编译器在预处理指令方面和 ANSI C 不完全兼容，因而你需要使用转换程序，来完整这项麻烦的工作。详细的指令请阅读 INSTALLvms.txt 文件。

MMS_VIM.EXE 是和 VIM.EXE 一起编译的，但是 XXD.EXE 需要切换到另一个目录并单独编译。

CTAGS 不再是 Vim 源程序发布的一部分，但是 OpenVMS 专用的源程序可能还是如上所述那样包含 CTAGS 的源代码。关于 VMS 上的 CTAGS 的更多信息可以参考
http://www.polarhome.com/ctags/

高级的用户可能想试试修改 FEATURE.H。

你也可以编译时加上 +xfontset +xim 选项，但需要你做好 GUI 字体等设置。参见 Vim 命令行提示下的 :help xim。

你还可以使用带有 GTK 图标的 GUI。为此，你需要下载和安装 OpenVMS 版本的 GTK，或至少是运行库里共享的图片：polarhome.com 里有 LIBGTK。
7.2 版本后 Vim 使用 GTK2+，而 OpenVMS 最后的 GTK 版本是 1.2.10，所以 GTK 版本不再可用。

如果你有更多更高级的问题，请把你的问题发送到 Vim on VMS 邮件列表
<vim-vms@polarhome.com>
关于 vim-vms 列表的更多情况可以在这里找到：
http://www.polarhome.com/mailman/listinfo/vim-vms

---

5. 布署　　vms-deploy

Vim 使用一个特定的目录结构来保存文档和运行时文件：

```
vim (或者随便什么名字)
 |- tmp
 |- vim57
 |----- doc
 |----- syntax
 |- vim62
 |----- doc
 |----- syntax
 |- vim64
 |----- doc
 |----- syntax
 vimrc    (系统 rc 文件)
 gvimrc
```

使用：

```
define/nolog VIM        device:[path.vim]
define/nolog VIMRUNTIME device:[path.vim.vim60]
define/nolog TMP        device:[path.tmp]
```

可以使 vim.exe 找到它的文档，文件类型，语法文件，并指定哪里可以存放临时文件。把 Vim 发布版本的 "runtime" 子目录复制到 vimruntime 目录里。

逻辑名 $VIMRUNTIME 和 $TMP 是可选的。

如果 $VIMRUNTIME 没有设置，Vim 会猜测并试图自动设置。更多情况可以查阅 :help runtime。

如果 $TMP 没有设置，你就不能使用部分的功能，如 CTAGS，XXD，打印，等等，因为它们需要临时目录来进行正常的操作。$TMP 目录应该对该用户可读可写。最简单的设置 $TMP 的方法是定义逻辑名：

```
define/nolog TMP SYS$SCRATCH
```

或者：

```
define/nolog TMP SYS$LOGIN
```

---

6. 实际使用方法 vms-usage

通常，你只需要运行 Vim 的一个版本，因而为 Vim 提供一个专用的目录就够了。把所有的 Vim 运行时目录结构复制到要布署的位置。在你的 LOGIN.COM (在 SYS$LOGIN 目录下) 里加入如下一行，把逻辑名 $VIM 设置为：

```
$ define VIM device:<path>
```

设置一些符号：

```
$ ! vi 文本方式启动 Vim。
$ vi*m  :== mcr VIM:VIM.EXE

$ !gvi GUI 方式启动 Vim。
```

```
$ gv*im :== spawn/nowait mcr VIM:VIM.EXE -g
```

请查阅如何定制和配置符号的相关内容。

你也许想要在你的主目录 (SYS$LOGIN) 下创建 .vimrc 和 .gvimrc 文件，以覆盖缺省的设置。

最简单的方法就是简单的使用示例文件并换名。你需要把菜单文件 (MENU.VIM) 和 vimrc 和 gvimrc 文件留在原来的 $VIM 目录。这些是所有用户的缺省设置，个人用户只需要把他们自己的添加或者改变的设置保存在主目录下的 .vimrc 和 .gvimrc 文件就可以。这样就没有问题了。

注意：记住，系统 rc 文件 (所有用户的缺省) 没有开头的 "."。所以，系统文件是：

```
$VIM:vimrc
$VIM:gvimrc
$VIM:menu.vim
```

而用户自己的 rc 文件是：

```
sys$login:.vimrc
sys$login:.gvimrc
```

你可以用 :version 查看是否一切文件位置都正确。

LOGIN.COM 示例：

```
$ define/nolog VIM DKA0:[UTIL.VIM81]
$ vi*m :== mcr VIM:VIM.EXE
$ gv*im:== spawn/nowait/input=NLA0 mcr VIM:VIM.EXE -g -GEOMETRY 80x40
$ set disp/create/node=192.168.10.202/trans=tcpip
```

注意：这样的设置就可以了。如果你在单独的服务器或者簇环境下工作，但是你想使用 Vim 作为 DECNET 环境下的中继编辑器，它也够用了。你只要加上 "完整的" 路径：

```
$ define VIM "<服务器名>[""用户名 密码""]::设备:<路径>"
$ vi*m :== "mcr VIM:VIM.EXE"
```

例如：

```
$ define VIM "PLUTO::RF10:[UTIL.VIM]"
$ define VIM "PLUTO""ZAY mypass"""::RF10:[UTIL.VIM]" ! 如果密码需要的话
```

如果你同时安装了多个版本，你可以用 $VIMRUNTIME 逻辑名指向正确的 Vim 版本。如果 $VIMRUNTIME 没有定义，Vim 会使用 $VIM 逻辑名。你可以执行 Vim 命令 :help runtime 来了解关于 $VIMRUNTIME 的更多信息。

系统管理员可能希望设立一个系统范围的 Vim 安装版本。那就加到 SYS$STARTUP:SYLOGICALS.COM

```
$ define/nolog/sys VIM 设备:<路径>
$ define/nolog/sys TMP SYS$SCRATCH
```

并加到 SYS$STARTUP:SYLOGIN.COM

```
$ vi*m :== mcr VIM:VIM.EXE
$ gv*im:== spawn/nowait/input=NLA0 mcr VIM:VIM.EXE -g -GEOMETRY 80x40
```

这样就能为系统的每个用户设立正常的 Vim 工作环境了。

重 要: OpenVMS 上的 Vim (和其它对大小写不敏感的系统) 的命令行参数假设使用小写。要说明使用的命令行参数是大写, 必须使用 "/" 符号。

示例:

```
vim -R  filename  ! 意味着: -r 列出交换文件并退出
vim -/r filename  ! 意味着: -R 只读模式 (类似于 "view")
vim -u  <vimrc>   ! 意味着: -u 使用 <vimrc> 代替其它的 .vimrc
vim -/u <gvimrc>  ! 意味着: -U 使用 <gvimrc> 代替其它的 .gvimrc
```

---

7. GUI 模式的问题 vms-gui

OpenVMS 是一个真正的大型机操作系统。因此, 即使它有一个 GUI 控制台, 大多数用户也不会在普通的操作中使用一个本地的 X/Window 环境。从而不可能很简单的 "就那样" 使用 GUI 模式启动 Vim。但是, 也难不到哪儿去。

首先: 你需要一个内建支持 GUI 的可执行版本。

其次: 你需要在你的 VMS 服务器上安装 DECW/Motif, 不然你会得到错误报告, 提示你一些共享库找不到。

其三: 如果你选择执行带有附加功能, 如 GUI/GTK 的 Vim, 你需要安装 GTK 或至少有 GTK 运行环境 (LIBGTK 可在 http://www.polarhome.com/vim/ 下载)。

1) 如果你使用 VMS X/Motif 控制台工作:
   这样启动 Vim:

   ```
   $ mc device:<path>VIM.EXE -g
   ```

   或者在 Vim 命令行提示下输入 :gui 命令。详见 :help gui

2) 如果你用其它的 X/Window 环境, 比如 Unix 或者远程的 X VMS 控制台。这样设置连接到你的终端的显示:

   ```
   $ set disp/create/node=<你的 IP 地址>/trans=<传输层-名字>
   ```

   然后像地一点那样启动 Vim。你可以从 VMS 文档里查找到更多的帮助, 或者从 VMS 提示符下输入 :help set disp。
   示例:

   ```
   $ set disp/create/node=192.168.5.159            ! 缺省传输层是 DECnet
   $ set disp/create/node=192.168.5.159/trans=tcpip ! TCP/IP 网络
   $ set disp/create/node=192.168.5.159/trans=local ! 显示在相同的机器上
   ```

注意: 你只能定义其中的一个。

更多的信息可以通过在 VMS 提示符下输入 $help set disp 得到。

3) 另外一个很好的方法是 XDM，如果你在 OpenVMS 机器上安装了的话。XDM 客户端可以如同 GUI 控制台一样的工作。

4) 如果你在 MS Windows 或者其它非 X/Window 环境上工作，你需要设置 X 服务器，并像第二点那样运行 Vim。MS Windows 上有免费的 X 服务器，例如 MIX，Omni X 等。也有很好的商业软件，如 eXcursion 或者 ReflectionX，它们都有内建的 DEC 支持。

请 注意，没有 GUI 支持的可执行程序在文本模式下启动时，比带有 GUI 支持的稍快。因此，如果你不打算使用 GUI 特性，可以考虑使用非 GUI 版本的可执行程序。

---

8. 有用的提示 vms-notes

8.1 退格/删除键
8.2 过滤程序
8.3 VMS 文件版本号
8.4 目录名转换
8.5 远程宿主调用
8.6 终端问题
8.7 十六进制编辑和其他外部工具
8.8 执行 vimrc 和 gvimrc 脚本
8.9 从 Vim 里打印
8.10 设置符号
8.11 diff 和其他 GNU 程序
8.12 diff-模式
8.13 允许 C 关键字里出现 '$'
8.14 为初学者提供的 VIMTUTOR (VIM 教程)
8.15 控制台模式启动太慢的问题
8.16 公用 VIM 目录 - 不同的体系结构

8.1 退格/删除键

VMS 存在退格/删除键的不一致问题。:fixdel 不能解决这个问题，你应该用:

```
:inoremap ^? ^H         " 在终端模式下
:inoremap <Del> ^H       " 在 gui 模式下
```

详见 8.6 章 (终端问题)。
(Bruce Hunsaker <BNHunsaker@chq.byu.edu> Vim 5.3 版本)

8.2 过滤程序

Vim 支持过滤程序；例如，如果你有 sort 程序，并能像 Unix 那样支持输入/输出重定向 (<输入文件 >输出文件)，你可以用

```
:map \s 0!'aqsort<CR>
```

(Charles E. Campbell, Jr, <cec@gryphon.gsfc.nasa.gov> Vim 5.4 版本)

8.3 VMS 文件版本号

Vim 保存文件会保存到带有更高一级的文件版本号的新文件。试试如下的设置。

```
:set nobackup          " 不要创建 *.*_ 备份文件
:set nowritebackup     " 在 VMS 没有任何意义。这是缺省行为。
```

从缺省的交换文件恢复文件没有任何问题。详见 :help swapfile

(Claude Marinier <ClaudeMarinier@xwavesolutions.com> Vim 5.5 版本，Zoltan Arpadffy Vim 5.6 版本)

8.4 目录名转换

Vim 会在内部转换任何 unix 风格的路径名，甚至包括混合 unix/VMS 的路径名，成为 VMS 风格的路径。一些典型的转换类似于：

```
/abc/def/ghi            -> abc:[def]ghi.
/abc/def/ghi.j          -> abc:[def]ghi.j
/abc/def/ghi.j;2        -> abc:[def]ghi.j;2
/abc/def/ghi/jkl/mno    -> abc:[def.ghi.jkl]mno.
abc:[def.ghi]jkl/mno    -> abc:[def.ghi.jkl]mno.
  ./                    -> 当前目录
  ../                   -> 相对的父目录
  [.def.ghi]            -> 相对的子目录
   ./def/ghi            -> 相对的子目录
```

注意：你可能要用 <、> 尖括号 (设备:<路径>文件名.扩展名;版本)。比如，rf10:<user.zay.work>test.c;1

(David Elins <delins@foliage.com>，Jerome Lauret <JLAURET@mail.chem.sunysb.edu> Vim 5.6 版本)

8.5 远程宿主调用

可以使用 Vim 作为中继编辑器。
1. 从远端编辑若干文件：

```
vi "<服务器>""用户名 密码""::<设备>:<路径><文件名>;<版本>"
```

例如：

```
vi "pluto""zay passwd""::RF10:<USER.ZAY.WORK>TEST.C;1"
```

注意：语法非常重要，不然 VMS 会识别为多个参数，而不是一个 (结果是：文件找不到)。

2. 设置 Vim 成为中继编辑器。如果 Vim 没有在你的本地上安装，只要设置你的 IP 地址，包含服务器名字的完整的 Vim 路径，并运行如下的程序：

```
$ if (p1 .eqs. "") .OR. (p2 .eqs. "") then goto usage
$ set disp/create/node=<你的 IP 地址>/trans=tcpip
```

```
$ define "VIM "<vim_服务器>""''p1' ''p2'"":<设备>:<vim 路径>"
$  vi*m :== "mcr VIM:VIM.EXE"
$ gv*im :== "spawn/nowait mcr VIM:VIM.EXE -g"
$ goto end
$ usage:
$ write sys$output " Please enter username and password as a parameter."
$ write sys$output " Example: @SETVIM.COM username passwd"
$ end:
```

注意：不要在簇环境下使用（你也不需要），不然速度会非常非常慢，不过也许比本地的 Emacs 还要快些。:-)

(Zoltan Arpadffy，Vim 5.6 版本)

8.6 终端问题

如果你的终端名称不为 Vim 所知，那么 Vim 就会尝试缺省的名字，而你会在启动时得到如下的提示：

```
Terminal entry not found in termcap
'unknown-terminal' not known. Available built-in terminals are:
    builtin_gui
    builtin_riscos
    builtin_amiga
    builtin_beos-ansi
    builtin_ansi
    builtin_vt320
    builtin_vt52
    builtin_pcansi
    builtin_win32
    builtin_xterm
    builtin_iris-ansi
    builtin_debug
    builtin_dumb
defaulting to 'vt320'
```

解决方法是定义合适的缺省终端名：

```
$ ! 未知的终端名字。让我们使用 vt320 或者 ansi。
$ ! 注意：大小写是敏感的。
$ define term "vt320"
```

从 VT100 到 VT320（如 V300，VT220，VT200）等终端不需要特别的键盘映射。它们应该正常工作，例如光标移动，插入，删除等键。但 GUI 模式下的退格键除外。要解决这个问题，在 .gvimrc 下加入：

```
inoremap <Del> <BS>
```

Vim 也会识别它们为快速终端。

如果屏幕在窗口切换时出现一些烦人的行，在你的 .vimrc 文件里加入：

```
set ttyfast     " 设置快速终端
```

注意：如果你通过远程或者慢速的连接访问 Vim，建议你避免快速终端选项：

```
set nottyfast   " 设置慢速终端
```

(Zoltan Arpadffy，Vim 5.6 版本)

8.7 十六进制编辑和其他外部工具

OpenVMS 和其他系统一个很重要的区别就是 VMS 使用特殊的命令来执行可执行文件：

```
RUN <路径>文件名
MCR <路径>文件名 <参数>
```

OpenVMS 用户不得不经常注意到，Vim 命令 :! 只是把他们推到 DCL 提示符下。如果你想以此执行 DCL 的命令，那没有问题。但如果我们想执行诸如 XXD，CTAGS，JTAGS 等程序，我们就会发现依照 Vim 文档进行操作会出现问题（比如，:help xxd）。

解决方法：执行 MC 命令，并加上可执行文件的全路径。例如，执行 :%!xxd 命令可以用：

```
:%!mc vim:xxd
```

更一般的：

```
:!mc <路径>文件 <参数>
```

注意：你可以使用 XXD 和 GUI 菜单下的 CTAGS。

要定制 ctags，可以定义逻辑名 $CTAGS 并使用标准的参数。如：

```
define/nolog CTAGS "--totals -o sys$login:tags"
```

要知道更多的信息，请阅读 :help tagsearch 和
http://ctags.sourceforge.net/ctags.html 提供的 CTAGS 文档。

(Zoltan Arpadffy，Vim 5.6-70 版本)

8.8 执行 vimrc 和 gvimrc 脚本

如果你想使用用从别的平台上通过 ftp 等方式带来的 .vimrc 和 .gvimrc（例如，MS Windows），可能会有问题。因为 VMS 使用不同的行尾表示方式。
具体症状就是， Vim 不能执行你的 .vimrc/.gvimrc，即使你说：

```
:so sys$login:.vimrc
```

一个技巧就是压缩（例如，zip）其他平台的文件，并在 VMS 上解压；如果你还有同样的问题，那么尝试复制-粘贴（为此，你需要两个系统在一台机器上同时使用。例如，Windows 上的 Xterm 窗口，或者从 VMS telnet 到 Windows 上）。

(Sandor Kopanyi，<sandor.kopanyi@mailbox.hu> Vim 6.0a 版本)

8.9 从 Vim 里打印

要使得 Vim 能够在 VMS 环境下支持打印 (GUI 模式的版本)，你需要设置 $TMP 逻辑名并指向某个临时目录，还要设置 SYS$PRINT 逻辑名指向你的缺省打印队列。
示例：

```
$define SYS$PRINT HP5ANSI
```

你可以打印整个缓冲区，或者只是标记的区域。详情参见 :help 的内容。

(Zoltan Arpadffy，Vim 6.0c 版本)

8.10 设置符号

如果我这样设置 gvim，在父终端按 CTRL-Y 就会关闭 gvim。通过设置不同的符号，看来就能解决这个问题。我的建议是：

```
$ GV*IM:==SPAWN/NOWAIT/INPUT=NLA0: MCR VIM:VIM.EXE -G -GEOMETRY 80X40
```

其中，/INPUT=NLA0: 分隔 gvim 和父终端的标准输入，从而隔绝了父窗口的信号。如果没有 -GEOMETRY，gvim 窗口会很小，从而菜单很容易在窗口调整大小后引起混淆。

(Carlo Mekenkamp，Coen Engelbarts，Vim 6.0ac 版本)

8.11 diff 和其他 GNU 程序

6.0 版本开始实现了 diff 功能。但是 OpenVMS 并不使用 GNU/Unix 类似的 diff，而内建的 diff 不能工作。以下是一个简单的解决此问题的方法。安装一个 Unix 兼容的 diff，则 Vim 就能正常在比较模式下工作了。只要你重定义 diff 程序如下：

```
define /nolog diff <GNU_PATH>diff.exe
```

另外一个更复杂的方法参见下面 (8.12 diff-模式)。此外，还有其他的程序，如 patch、make 等等，都会有相同的问题。www.polarhome.com 提供了 Alpha 和 VAX 机器上的 GNU 程序包，可以用来解决 OpenVMS 上的 GNU 问题。
(Zoltan Arpadffy，Vim 6.1 版本)

8.12 diff-模式

Vim 6.0 或更高版本提供 Vim 的 diff-模式 (见 new-diff-mode 、 diff-mode 和 08.7)。它们使用外部程序 'diff' 并希望看到 Unix 兼容的 diff 输出结果。不过，标准的 VMS diff 程序输出的格式与之不兼容。要在 VMS 上使用 Vim 的 diff-模式，你需要：

1 安装 Unix 兼容的 diff 程序，例如 GNU diff
2 告诉 Vim 在 diff-模式下使用 Unix 兼容的 diff。

你可以从 VIM-VMS 网站下载 GNU diff，它是 GNU 工具的一部分。链接是：
http://www.polarhome.com/vim/files/gnu_tools.zip。我建议把它解压到一个单独的 "GNU" 目录下，并建立一个逻辑名 GNU: 指向该目录，例如：

```
DEFINE GNU    <DISK>:[<DIRECTORY>.BIN.GNU]
```

你可以定义 GDIFF 符号，以便在 DCL 提示下使用 GNU diff:

```
GDIFF :==     $GNU:DIFF.EXE
```

现在你可以告诉 Vim 使用新的 diff 程序了。以下程序取自 diff-diffexpr ，但把外部 diff 程序的调用指向新的 VMS 上的 diff。把它加到你的 .vimrc 文件里:

```
" Set up vimdiff options
  if v:version >= 600
   " Use GNU diff on VMS
   set diffexpr=MyDiff()
   function MyDiff()
      let opt = ""
      if &diffopt =~ "icase"
        let opt = opt . "-i "
      endif
      if &diffopt =~ "iwhite"
        let opt = opt . "-b "
      endif
      silent execute "!mc GNU:diff.exe -a " . opt . v:fname_in . " " .  v:fname_new .
           \  " > " . v:fname_out
  endfunction
 endif
```

你现在可以使用 Vim 的 diff-模式，比如，只读方式下比较两个文件:

```
$ VIM -D/R <FILE1> <FILE2>
```

你可以定义新的符号来执行 vimdiff，例如:

```
$ VIMDIFF     :== 'VIM' -D/R
$ GVIMDIFF    :== 'GVIM' -D/R
```

你现在可以用四种方式比较文件:

```
1. VMS  diff:  $ DIFF     <FILE1> <FILE2>
2. GNU  diff:  $ GDIFF    <FILE1> <FILE2>
3. VIM  diff:  $ VIMDIFF  <FILE1> <FILE2>
4. GVIM diff:  $ GVIMDIFF <FILE1> <FILE2>
```

(Coen Engelbarts，Vim 6.1 版本)

8.13 允许 C 关键字里出现 '$'

DEC C 使用很多带有 '$' 的标识符。ANSI C 则不允许如此，而 Vim 因而把 '$' 当作标识符结束的标记。通过 'iskeyword' 选项，你可以改变这一行为。
在你的 .vimrc 文件里加入如下的命令:

```
autocmd FileType c,cpp,cs  set iskeyword+=$
```

你也可以创建如下的文件 $VIM/FTPLUGIN/C.VIM (和/或 CPP.VIM 和 CS.VIM)，并加入如

下的命令：

```
set iskeyword+=$
```

这样，基于单词的命令，如 '*'-搜索命令和 CTRL-] 标签搜索等，都能作用于整个标识符上了。(VMS 上的 Ctags 从 5.1 版本开始，也支持带 '$' 的 C 关键字)。

(Coen Engelbarts，Vim 6.1 版本)

8.14 为初学者提供的 VIMTUTOR (VIM 教程)

VIMTUTOR.COM DCL 脚本可以帮助 Vim 初学者学习/掌握在 OpenVMS 上运行 Vim 的基本步骤。在某些二进制版本中，你可以如下方式开始：

```
@vim:vimtutor
```

(Thomas.R.Wyant III，Vim 6.1 版本)

8.15 控制台模式启动太慢的问题

因为 GUI/GTK Vim 在控制台模式中也能很好的工作，很多管理员为整个系统安装了这些版本的可执行程序。
不幸的是，在远端慢速连接上，GUI/GTK 版本对只想使用控制台的用户会相当慢，这是因为该版本检测 X 环境的超时关系。

幸运的是，有个简单的方法。管理员只需要同时安装 GUI/GTK 版本和纯控制台版本，如下：

```
|- vim73
|----- doc
|----- syntax
   vimrc    (系统 rc 文件)
   gvimrc
   gvim.exe (换名的 GUI 或 GTK 版本的 vim.exe)
   vim.exe  (纯控制台版本)
```

在 LOGIN.COM 或 SYSLOGIN.COM 为 ex 定义系统符号：

```
$ define/nolog VIM RF10:[UTIL.VIM73] ! VIM 所在的目录
$ vi*m  :== mcr VIM:VIM.EXE
$ gvi*m :== mcr VIM:GVIM.EXE
$ ! 或者用以下方式启动
$ gv*im :== spawn/nowait/input=NLA0 mcr VIM:GVIM.EXE -g -GEOMETRY 80x40
```

像这样，没有 X 环境而只想用纯控制台模式的用户就可以避免效率太低的问题了。

(Zoltan Arpadffy，Vim 7.2)

8.16 公用 VIM 目录 - 不同的体系结构

在一个下面这样包含不同体系结构结点的簇里：

```
$show cluster
```

```
View of Cluster from system ID 11655  node: TOR
+---------------------------------+
|        SYSTEMS        | MEMBERS |
+-----------------------+---------|
|  NODE  |   SOFTWARE   |  STATUS |
+--------+--------------+---------|
| TOR    | VMS V7.3-2   | MEMBER  |
| TITAN2 | VMS V8.3     | MEMBER  |
| ODIN   | VMS V7.3-2   | MEMBER  |
+---------------------------------+
```

如果能够有一个公共的 VIM 目录，但分别执行不同的可执行程序，会很方便。
有若干实现方案：

方法 1。所有可执行程序在相同目录，但使用不同的名字。
在 login.com 或 sylogin.com 里使用以下的脚本可以很容易做到：

```
$ if f$getsyi("NODE_HWTYPE") .eqs. "VAX"
$ then
$       say "VAX platform"
$       vi*m:== mcr vim:VIM.EXE_VAX
$ endif
$ if f$getsyi("NODE_HWTYPE") .eqs. "ALPH"
$ then
$       say "ALPHA platform"
$       vi*m :== mcr vim:VIM.EXE_AXP
$ endif
$ if f$getsyi("ARCH_NAME") .eqs. "IA64"
$ then
$      say "IA64 platform"
$      vi*m :== mcr vim:VIM.EXE_IA64
$ endif
```

方法 2. 不同目录：

```
$ if f$getsyi("NODE_HWTYPE") .eqs. "VAX"
$ then
$       say "VAX platform"
$       define/nolog VIM RF10:[UTIL.VAX_EXE] ! VAX executables
$ endif
$ if f$getsyi("NODE_HWTYPE") .eqs. "ALPH"
$ then
$       say "ALPHA platform"
$       define/nolog VIM RF10:[UTIL.AXP_EXE] ! AXP executables
$ endif
$ if f$getsyi("ARCH_NAME") .eqs. "IA64"
$ then
$      say "IA64 platform"
$      define/nolog VIM RF10:[UTIL.IA64_EXE] ! IA64 executables
$ endif
$! VIMRUNTIME must be defined in order to find runtime files
$ define/nolog VIMRUNTIME RF10:[UTIL.VIM73]
```

使用这种方法一个好的示例可见来自 GNU_TOOLS.ZIP 包的 [GNU]gnu_tools.com 脚本。

可以从这里下载 http://www.polarhome.com/vim/

(Zoltan Arpadffy, Vim 7.2)

---

9. VMS 相关修改记录 (英文) vms-changes

```
Version 8.1
- make necessary changes to build v8.1 on VMS

Version 8.0
- solve the 100% cpu usage issue while waiting for a keystroke
- correct the VMS warnings and errors around handling the INFINITY (used in json.c)
- minor VMS port related changes
- correct the make_vms.mms file for 8.0
- fix [.TESTDIR]make_vms.mms for 8.0

Version 7.4
- Undo: VMS can not handle more than one dot in the filenames use "dir/name" -> "dir/_un_name"
  add _un_ at the beginning to keep the extension
- correct swap file name wildcard handling
- handle iconv usage correctly
- do not optimize on vax - otherwise it hangs compiling crypto files
- fileio.c fix the comment
- correct RealWaitForChar
- after 7.4-119 use different functions lib$cvtf_to_internal_time because Alpha and VAX have
  G_FLOAT but IA64 uses IEEE float otherwise Vim crashes
- guard against crashes that are caused by mixed filenames
- [TESTDIR]make_vms.mms changed to see the output files
- Improve tests, update known issues
- minor compiler warnings fixed
- CTAGS 5.8 +regex included

Version 7.3
- CTAGS 5.8 included
- VMS compile warnings fixed - floating-point overflow warning corrected on VAX
- filepath completion corrected - too many chars were escaped in filename
  and shell commands
- the following plugins are included into VMS runtime:
        genutils 2.4, multiselect 2.2, multvals 3.1, selectbuf 4.3,
        bufexplorer 7.1.7, taglist 4.5
- minor changes in vimrc (just in VMS runtime)
- make_vms.mms - HUGE model is the default
- [TESTDIR]make_vms.mms include as many tests possible
- modify test30 and test54 for VMS
- enable FLOAT feature in VMS port
- os_vms.txt updated

Version 7.2 (2008 Aug 9)
- VCF files write corrected
- CTAGS 5.7 included
- corrected make_vms.mms (on VAX gave syntax error)
```

```
Version 7.1 (2007 Jun 15)
- create TAGS file from menu

Version 7
- Improved low level char input (affects just console mode)
- Fixed plugin bug
- CTAGS 5.6 included

Version 6.4 (2005 Oct 15)
- GTKLIB and Vim build on IA64
- colors in terminal mode
- syntax highlighting in terminal mode
- write problem fixed (extra CR)
- ESC and ESC sequence recognition in terminal mode
- make file changed to support new MMS version
- env variable expansion in path corrected
- printing problems corrected
- help text added for case insensitive arguments

Version 6.3 (2004 May 10)
- Improved vms_read function
- CTAGS v5.5.4 included
- Documentation corrected and updated

Version 6.2 (2003 May 7)
- Corrected VMS system call results
- Low level character input is rewritten
- Correction in tag and quickfix handling
- First GTK build
- Make file changes
    - GTK feature added
    - Define for OLD_VMS
    - OpenVMS version 6.2 or older
- Documentation updated with GTK features
- CTAGS v5.5 included
- VMS VIM tutor created

Version 6.1 (2002 Mar 25)
- TCL init_tcl() problem fixed
- CTAGS v5.4 included
- GNU tools binaries for OpenVMS
- Make file changes
    - PERL, PYTHON and TCL support improved
    - InstallVMS.txt has a detailed description HOWTO build
- VMS/Unix file handling rewritten
- Minor casting and bug fixes

Version 6.0 (2001 Sep 28)
- Unix and VMS code has been merged
        - separated "really" VMS related code
        - included all possible Unix functionality
        - simplified or deleted the configuration files
        - makefile MAKE_VMS.MMS reviewed
- menu changes (fixed printing, CTAGS and XXD usage)
```

- fixed variable RMS record format handling anomaly
- corrected syntax, ftplugin etc files load
- changed expand_wildcards and expandpath functions to work more general
- created OS_VMS_FILTER.COM - DECC->VAXC pre-processor directive convert
  script.
- Improved code's VAXC and new DECC compilers compatibility
- changed quickfix parameters:
        - errormessage format to suite DECC
        - search, make and other commands to suite VMS system
- updated and renamed MMS make files for Vim and CTAGS.
- CTAGS has been removed from source distribution of Vim but it will remain
  in OpenVMS binary distributions.
- simplified build/configuration procedure
- created INSTALLvms.txt - detailed compiling instructions under VMS.
- updated test scripts.

Version 5.8 (2001 Jun 1)
- OS_VMS.TXT updated with new features.
- other minor fixes.
- documentation updated
- this version had been tested much more than any other OpenVMS version
  earlier

Version 5.7 (2000 Jun 24)
- New CTAGS v5.0 in distribution
- Documentation updated

Version 5.6 (2000 Jan 17)
- VMS filename related changes:
        - version handling (open everything, save to new version)
        - correct file extension matching for syntax (version problem)
        - handle <,> characters and passwords in directory definition
        - handle internode/remote invocation and editing with passwords
        - OpenVMS files will be treated case insensitive from now
        - corrected response of expand("%:.") etc path related functions
        (in one word: VMS directory handling internally)
- version command
        - corrected (+,-) information data
        - added compiler and OS version
        - added user and host information
        - resolving $VIM and $VIMRUNTIME logicals
- VMS port is in MAX_FEAT (maximum features) club with Unix, Win32 and OS/2.
        - enabled farsi, rightleft etc. features
        - undo level raised up to 1000
- Updated OS_VMS.MMS file.
        - maximum features ON is default
        - Vim is compilable with +perl, +python and +tcl features.
        - improved MMK compatibility
- Created MAKEFILE_VMS.MMS, makefile for testing Vim during development.
- Defined DEC terminal VT320
        - compatibility for VT3*0, VT2*0 and VT1*0 - ANSI terminals
          backwards, but not VT340 and newer with colour capability.
        - VT320 is default terminal for OpenVMS
        - these new terminals are also fast ttys (default for OpenVMS).

```
        - allowed dec_mouse ttym
- Updated files vimrc and gvimrc with VMS specific suggestions.
- OS_VMS.TXT updated with new features.

Version 5.5 (1999 Dec 3)
- Popup menu line crash corrected.
- Handle full file names with version numbers.
- Directory handling (CD command etc.)
- Corrected file name conversion VMS to Unix and v.v.
- Correct response of expand wildcards
- Recovery is working from this version under VMS as well.
- Improved terminal and signal handing.
- Improved OS_VMS.TXT

Version 5.4 (1999 Sep 9)
- Cut and paste mismatch corrected.
- Motif directories during open and save are corrected.

Version 5.3 (1998 Oct 12)
- Minor changes in the code
- Standard distribution with +GUI option

Version 5.1 (1998 Apr 21)
- Syntax and DEC C changes in the code
- Fixing problems with the /doc subdirectory
- Improve OS_VMS.MMS

Version 4.5 (1996 Dec 16)
- First VMS port by Henk Elbers <henk@xs4all.nl>
```

---

10. 作者 vms-authors

OpenVMS 文档和执行文件的维护者是:
Zoltan Arpadffy <arpadffy@polarhome.com>
OpenVMS Vim page: http://www.polarhome.com/vim/

本文档采用了较早 OS_VMS.TXT 版本的作者和贡献者提供的部分内容和 备注 。他们是:

```
        Charles E. Campbell, Jr. <cec@gryphon.gsfc.nasa.gov>
        Bruce Hunsaker <BNHunsaker@chq.byu.edu>
        Sandor Kopanyi <sandor.kopanyi@mailbox.hu>

 vim:tw=78:ts=8:noet:ft=help:norl:
```

os_win32.txt For Vim version 8.2. 最近更新：2019年12月

VIM 参考手册 by George Reilly
译者：Dasn, tocer
http://vimcdoc.sf.net

win32 Win32 MS-Windows

本文描述 Vim Win32 版本的一些特性。

Vim 的 Win32 版本可以在 Windows XP、Vista、7、8 和 10 版本 上运行，并且有 console（控制 台，就是命令提示符窗口 译者）和 GUI（图形界面）两个版本。

32 位版本也可运行在 64 位 MS-Windows 系统上。

1. 一些已知的问题 win32-problems
2. 启动 win32-startup
3. 恢复屏幕内容 win32-restore
4. 使用鼠标 win32-mouse
5. 在 Windows 95 下运行 win32-win95
6. 在 Windows 3.1 下运行 win32-win3.1
7. Win32 常见问题解答（迷你版） win32-faq

此外，还有一些 Win32 和 DOS 版本通用的条目：

文件位置 dos-locations
使用反斜杠 dos-backslash
标准映射 dos-standard-mappings
屏幕的输出与颜色 dos-colors
文件格式 dos-file-formats
:cd 命令 dos-:cd
中止键 dos-CTRL-Break
临时文件 dos-temp-files
默认的外壳选项 dos-shell

Win32 用户图形界面 gui-w32

作者：
Win32 版本的作者是 George V. Reilly <george@reilly.org>。
最初 Windows NT 版本的移植工作是由 Roger Knobbe <RogerK@wonderware.com> 完成的。
GUI 版本的作者是 George V. Reilly 和 Robert Webb。

有关编译的讨论请参考 "src/INSTALLpc.txt"。 win32-compiling

---

1. 一些已知的问题 win32-problems

自动补全文件名的时候，虽然 Vim 会匹配短的文件名，但是却会寻找并使用对应的相应的长文件名。例如，如果有一个长文件名 "this_is_a_test"，其短文件名是 "this_i~1"，命令 ":e *1" 编辑文件 "this_is_a_test"。

---

2. 启动 win32-startup

当前目录 win32-curdir

如果启动 Vim 时后面带一个文件名作为参数，并且指定了该文件的完整路径（例如：以 "x:\" 开头的路径），Vim 会认为该文件是从文件管理器启动，并把当前目录（:cd 命令查看 译者）切换到文件所在目录。如果不想这样，只要把路径里的反斜杠（"\"）换成斜杠（"/"）即可。例如：

```
vim c:\text\files\foo.txt
```

当前目录切换到 "C:\text\files"。

```
vim c:/text\files\foo.txt
```

不改变当前目录。

term 选项 win32-term

Vim 的 Win32 版本仅有的终端类型是 "win32"，它是内置的。如果你把 'term' 设置成其它的值，Vim 的行为就可能变的很奇怪。所以 Vim 不会从环境变量 "TERM" 里获取 'term' 的值。

$PATH 环境变量 win32-PATH

Vim 可执行程序的路径被加入到变量 $PATH 中。这主要使 "!xxd" 能够运行，因为它在工具菜单中。这也意味着当 executable() 返回 1 时，该可执行程序确实能够执行。

命令行参数 win32-cmdargs

对 MS Windows 把命令行分解为参数的分析没有一个标准。Vim 和 gvim 过去使用不同的逻辑来处理命令行（7.4.432 之前），而且这些逻辑也取决于如何地编译。现在 Vim 和 gvim 两者都使用 CommandLineToArgvW() Win32 API，从而行为一致。

基本规则是： win32-backslashes

a) 参数是图形字符的序列。
b) 参数似空白分隔。
c) 参数可以双引号包围，其中可包含空白。
d) 零和多个反斜杠（\）和一个双引号（"）是特别的。有效的反斜杠减半往下取整。偶数反斜杠反转空格和制表的可接受性，奇数反斜杠生成按本义出现的双引号。

因而：

```
"       是特殊双引号
\"      是按本义出现的双引号
\\"     是一个按本义出现的反引号和一个特殊双引号
\\\"    是一个按本义出现的反引号和一个按本义出现的双引号
\\\\"   是两个按本义出现的反引号和一个特殊双引号
\\\\\"  是两个按本义出现的反引号和一个按本义出现的双引号
等等
```

示例：

```
vim "C:\My Music\freude" +"set ignorecase" +/"\"foo\\" +\"bar\\\"
```

打开 "C:\My Music\freude" 并执行行模式命令:

```
set ignorecase; /"foo\ and /bar\"
```

CommandLineToArgvW API 的参考中也有这些规则的描述:

https://msdn.microsoft.com/en-us/library/windows/desktop/bb776391.aspx

win32-quotes

引号还有额外的规则 (这里文档不甚完整)。如上所述, 文件名 (或任何其它命令行参数) 内的引号可以用反斜杠转义。例如

```
vim -c "echo 'foo\"bar'"
```

也可要用三个引号来表示一个实际一个引号:

```
vim -c "echo 'foo"""bar'"
```

引号规则是:

1. " 开始引用。
2. 另一个 " 或 "" 结束引用。如果引用以 "" 结束, 在引用的字符串之后生成 " 。

示例, [] 围绕参数:

```
"foo"           -> [foo]
"foo""          -> [foo"]
"foo"bar        -> [foobar]
"foo" bar       -> [foo], [bar]
"foo""bar       -> [foo"bar]
"foo"" bar      -> [foo"], [bar]
"foo"""bar"     -> [foo"bar]
```

---

3. 恢复屏幕内容

win32-restore

如果你设定了 'restorescreen' 选项 (默认设置), 当你退出 Vim 或者执行外部命令时, Vim 能恢复控制台原来的内容。可以用 ":ser nors" 命令关掉这个选项。

'restorescreen'

---

4. 使用鼠标

win32-mouse

Vim 的 Win32 版本支持鼠标。如果你使用两键鼠标, 可以用同时按下左右两键的办法来模拟第三键 (中键) — 但是, 请注意: 在 Win32 GUI 上, 如果你的右键用来呼叫弹出菜单 (参见 'mouse'), 那么你最好先按下鼠标左键 (左右两键同时按的时候, 左键略快于右键 译者)。

mouse-using

如果鼠标不好用, 你可以试着禁用控制台的 "快速编辑模式"。

---

5. 在 Windows 95 下运行

win32-win95

windows95 windows98 windowsme

8.0.0029 补丁删除了 Windows 95/98/ME 的支持。如果你还想用, 请用之前的版本。

6. 在 Windows 3.1 下运行 win32-win3.1

win32s windows-3.1 gui-w32s win16

有一个特殊的 Win32 版本的 Gvim 可以在 Windows 3.1 和 3.11 运行。
7.4.1363 补丁删除了此支持。

7. Win32 常见问题解答（迷你版） win32-faq

问：怎样改变字体?
答：在 GUI 版本里，可以用 'guifont' 选项。例如：

```
:set guifont=Lucida_Console:h15:cDEFAULT
```

在控制台版本里，你需要修改控制台窗口的字体，不能在 Vim 里修改字体。

问：在 Windows NT 上怎样输入死键?
答：NT 3.51 上可以正常输入死键，和在别的程序里的输入方法一样。
NT 4.0 上，你必须保证你的区域设置（控制面板键盘）和你当前使用的区域设置相同。否则 NT 会崩溃。这是 NT 的问题，不是 Vim 的问题。

问：我用 Vim 编辑一个符号链接，这个链接指向 UNIX NFS 文件服务器上的一个文件。当我保存这个文件时，Vim 不能通过链接保存到服务器上，而是删除了这个链接，创建了一个新的文件。为什么?
答：在 UNIX 系统上，Vim 可以处理链接（符号链接或硬链接）。Vim 先把文件写入一个备份文件中，然后用备份文件覆盖原文件。这样做是为了保证所有属性都和原文件相同。在非 UNIX 系统上，首先原文件被改名，然后写入新文件。只有保护位和原文件相同。但是，这个方法不能处理 NFS 文件系统，那里除了链接还有一些其它问题。
唯一的解决办法是禁用备份文件，可以通过命令 ":set nobackup nowritebackup" 实现 'writebackup'

问：我正通过 Samba 使用 Vim 编辑 Unix 系统上的一个文件。当我保存文件时，文件所有者被改变了。为什么?
答：当保存文件时，Vim 会给原来文件换名，使之成为一个备份（以防写入文件过程中失败）。然后该文件作为一个新文件被保存。Samba 这时把它赋予文件系统上默认的所有者，可能会和原所有者不一样。
为避免此问题，可以设置 'backupcopy' 选项为 "yes"。Vim 会用复制的方式建立备份，然后再覆盖原文件。这样文件所有者就不会改变了。

问：执行 ":make" 的时候怎样看到输出?
答：通常，你需要一个 tee 程序，它可以把输入（make 的输出）复制到标准输出和错误信息文件里。你可以在以下地址找到 tee（还有其它 GNU 工具）。
http://gnuwin32.sourceforge.net 或 http://unxutils.sourceforge.net
或者你也可以在 http://www.cygwin.com/ 下载 Cygnus 版本的 GNU 工具。
其它 Win32 上运行的 Unix 风格的工具可以在以下地址找到。
http://directory.google.com/Top/Computers/Software/Operating_Systems/Unix/Win32/
得到了 tee 程序之后，你需要把下面的命令添加到你的 _vimrc 文件中：

```
:set shellpipe=\|\ tee
```

问：我把文件保存到一台运行 VisionFS 的远程计算机上，但是文件找不到了！
答：VisionFS 不能处理特定的句号（.）加三个字母的扩展名，SCO 声称这是为了兼容 16 位的 DOS/Windows 环境。下面的两个命令可以说明问题：

```
echo Hello > file.bat~
```

```
dir > file.bat
```

"dir" 命令的输出被定向到 "file.bat~" 文件，而并没有新建一个 "file.bat" 文件。这种行为在 Vim 里的体现就是，当你编辑一个名为 "foo.bat" 的文件时，Vim 会自动创建名为文件名加上 '~' 字符的临时文件，也就是 "foo.bat~"。写入 "foo.bat" 文件时，最后它倒被删除了 (译者注：Vim 想删除的本来是临时文件)。

解决办法：在你的 _vimrc 文件里加入：

```
:set backupext=.temporary
```

问：怎样改变光标闪烁的频率?
答：不能改。这是 NT 控制台的一个限制，据报告，NT 5.0 可以一次改变所有控制台的闪烁频率。

:!start

问：如何异步地执行一个外部命令或程序，或用缺省程序打开文档或 URL?
答：执行外部命令的时候，你可以用 ":!start" 代替 ":!"。例如，要运行 notepad：

```
:!start notepad
```

要用缺省图像查看器打开 "image.jpg"：

```
:!start image.jpg
```

要用 Windows Explorer 打开当前文件所在的目录：

```
:!start %:h
```

要用缺省的浏览器打开 Vim 主页：

```
:!start http://www.vim.org/
```

用 "start" 命令不像 ":!" 那样 (切换到另一个屏幕、调用一个新的控制台，然后等待程序执行结束)；该命令的使用，意味着你所运行的程序不会影响到当前编辑的文件。用 ":!start" 命令运行程序不会得到 Vim 的打开文件句柄，这意味着他们不需要在 Vim 之前退出。
要避免这种特殊照顾，可以用 ":! start"。
有两个可选的参数 (见下个问答)：

```
/min   最小化方式打开窗口
/b     不打开控制台窗口
```

一次只能使用一个参数。第二个参数会被当作命令的一部分。

问：如何避免异步远行的程序打开窗口?
答：取决于你实际需要，有两个方案：

1) 用 /min 标志位，以最小化方式运行，其他不作改变。对控制台和 GUI 应用一样有效。
2) 用 /b 标志位以运行控制台程序，而不建立控制台窗口 (GUI 应用不受影响)。只能用于不需要输入的应用。否则会得到 EOF 错误，因为输入流 (stdin) 会重定向到 \\.\NUL (stdout 和 stderr 也是如此)。

运行 Exuberant ctags 的控制台示例：

```
:!start /min ctags -R .
```

程序结束时你会看到名为 "tags" 文件在当前目录出现。你会注意到底下任务栏上窗口标题在闪烁。尤其对需时较长的命令更容易看到。
现在删除 "tags" 文件，然后执行：

```
:!start /b ctags -R .
```

还是会产生 "tags" 文件，但这次任务栏不会闪烁了。
GUI 应用的示例：

```
:!start /min notepad
:!start /b notepad
```

前一个命令以最小化方式运行 notepad，后一个命令以正常方式运行。

windows-icon

问：我不喜欢 Vim 图标，如何改变?
答：是的，把你喜欢的图标放在 'runtimepath' 某个目录下的 bitmaps/vim.ico。例如 ~/vimfiles/bitmaps/vim.ico。

vim:tw=78:ts=8:noet:ft=help:norl:

pi_getscript.txt For Vim version 8.2. 最近更新：2019年12月

GETSCRIPT 参考手册 作者：Charles E. Campbell, Jr.

译者：Willis

http://vimcdoc.sf.net

Authors: Charles E. Campbell <NdrOchip@ScampbellPfamilyA.Mbiz>
(remove NOSPAM from the email address)

GetLatestVimScripts-copyright

Copyright: (c) 2004-2012 by Charles E. Campbell glvs-copyright
The VIM LICENSE (see copyright) applies to the files in this
package, including getscriptPlugin.vim, getscript.vim,
GetLatestVimScripts.dist, and pi_getscript.txt, except use "getscript"
instead of "Vim". Like anything else that's free, getscript and its
associated files are provided *as is* and comes with no warranty of
any kind, either expressed or implied. No guarantees of
merchantability. No guarantees of suitability for any purpose. By
using this plugin, you agree that in no event will the copyright
holder be liable for any damages resulting from the use of this
software. Use at your own risk!

getscript 是一个插件，它简化了获取你自己使用的脚本的最新版本的步骤！输入 :GLVS 会启动 getscript；它会接着使用 <GetLatestVimScripts.dat> 文件（见 GetLatestVimScripts_dat ）来从 http//vim.sf.net/ 得到该文件列出的脚本的最新版本。

---

1. 内容 glvs-contents glvs getscript GetLatestVimScripts

1. 内容.....: glvs-contents
2. GetLatestVimScripts -- 入门.....: glvs-install
3. GetLatestVimScripts 用法.....: glvs-usage
4. GetLatestVimScripts 数据文件.....: glvs-data
5. GetLatestVimScripts 友好插件.....: glvs-plugins
6. GetLatestVimScripts 自动安装.....: glvs-autoinstall
7. GetLatestViMScripts 选项.....: glvs-options
8. GetLatestVimScripts 算法.....: glvs-alg
9. GetLatestVimScripts 历史.....: glvs-hist

---

2. GetLatestVimScripts -- 入门 getscript-start getlatestvimscripts-install

VIM 发布提供的版本 glvs-dist-install

Vim 7.0 没有包含 GetLatestVimScripts.dist 文件，该文件提供了示例和样板。所以，你需要自己建立一个。参见 GetLatestVimScripts_dat 。

VIM SF NET 提供的版本 glvs-install

备注：最后一步，即为 GetLatestVimScripts.dist 文件换名或移动，适用于那些第一次

刚刚下载 GetLatestVimScripts.tar.bz2 的用户。

GetLatestVimScripts.dist 文件提供了示例和样板，你可以参照来建立自己的列表。你尽可以删掉所有那里提到的所有脚本，"紧要的" 部分是头两行。

你的计算机需要带有 wget 或 curl，这样 GetLatestVimScripts 才能工作。

1. 如果经过压缩： gunzip getscript.vba.gz
2. Unix:
```
vim getscript.vba
:so %
:q
cd ~/.vim/GetLatest
mv GetLatestVimScripts.dist GetLatestVimScripts.dat
(编辑 GetLatestVimScripts.dat，以安装你自己需要的插件列表 --
见  GetLatestVimScripts_dat )
```
3. Windows:
```
vim getscript.vba
:so %
:q
cd **path-to-vimfiles**/GetLatest
mv GetLatestVimScripts.dist GetLatestVimScripts.dat
(编辑 GetLatestVimScripts.dat，以安装你自己需要的插件列表 --
见  GetLatestVimScripts_dat )
```

---

3. GetLatestVimScripts 用法 glvs-usage :GLVS

除非另有定义，

```
:GLVS
```

会调用 GetLatestVimScripts()。如果其它脚本定义了该命令，那么你可以输入

```
:GetLatestVimScripts
```

该脚本试图从 http://vim.sourceforge.net/ 更新并，如果允许，为你自动安装脚本。为此它会审视如下文件

```
.vim/GetLatest/GetLatestVimScripts.dat                    (unix)
```

或

```
..wherever..\vimfiles\GetLatest\GetLatestVimScripts.dat   (windows)
```

(见 glvs-data) 并检查 [.vim|vimfiles]/plugin 目录里的插件 (见 glvs-plugins)。

下载的脚本会出现在 ~/.vim/GetLatest (unix) 或 ..wherever..\vimfiles\GetLatest (windows) 子目录。如果 <.vimrc> 里包含以下的行，GetLatestVimScripts 会尝试自动安装它们：

```
let g:GetLatestVimScripts_allowautoinstall=1
```

GetLatestVimScripts.dat> 文件会自动更新，以反映脚本的最新下载版本。(也见
glvs-options)

---

4. GetLatestVimScripts 数据文件 getscript-data glvs-data
:GetLatestVimScripts_dat

数据文件 <GetLatestVimScripts.dat> 开始的两行必须包含下面的文字:

```
ScriptID SourceID Filename
```

<
这两行之后的内容包含三列数据，前两列是数值列，然后是一列文本列。
GetLatest/GetLatestVimScripts.dist 文件包含了这样的数据文件的一个示例。任何
#... 之后的内容都被忽略，以便你在文件中嵌入注释。

每行第一个数值给出脚本的 ScriptID。如果用浏览器察看 http://vim.sf.net/ 里的脚本，点击脚本的链接之前，你会看到该链接的形式如下:

```
http://vim.sourceforge.net/scripts/script.php?script_id=40
```

这里 "40" 就是 ScriptID，GetLatestVimScripts 用它来下载相关页面。该 id 是
vim.sf.net 在插件初始上传的时候分配的。

每行第二个数值给出脚本的 SourceID。SourceID 记录由 vim.sf.net 决定的脚本上传次数；这样，它起到了指示脚本 "何时" 上传的作用。设置 SourceID 为 1 就可使
GetLatestVimScripts 认为它拥有的脚本版本已经过时。

GetLatestVimScripts 从 vim.sf.net 的相应脚本页提取 SourceID。一旦该值大于
GetLatestVimScripts.dat 文件保存的值，该脚本就会被下载 (见
GetLatestVimScripts_dat)。

如果脚本作者在脚本中包含了某特殊注释行，GetLatestVimScripts 会用该脚本本身和
(如果存在的话) 它依赖的其它脚本来构造 <GetLatestVimScripts.dat> 文件。例如，考虑:

```
" GetLatestVimScripts: 884  1 :AutoInstall: AutoAlign.vim
```

该注释行告诉 getscript.vim 检查 #884 vim 脚本，并且说明该脚本可自动安装。
Getscript 也会使用该行来帮助构造 GetLatestVimScripts.dat 文件，如果该文件中还
没有包含下面类似的行的话，会在其中自动加入这样的行:

```
884 1 :AutoInstall: AutoAlign.vim
```

详见 glvs-plugins 。这样，GetLatestVimScripts 提供了全面的让你脚本随时保持更新的完整功能!

摘要:

* 可选地，允许 getscript 根据已安装的插件来构造或附加到
  GetLatestVimScripts.dat 文件:
  let g:GetLatestVimScripts_allowautoinstall=1

* 已下载的插件中，如果有形如这样的行
  `" GetLatestVimScripts: 884  1 :AutoInstall: AutoAlign.vim`
  代表着插件作者同意 getscript 可以自动安装。不是所有插件都可以自动安装的，插件的作者最能了解自己插件的情况。

* GetLatestVimScripts.dat 中，形如这样的行
  `884 1 :AutoInstall: AutoAlign.vim`
  代表你同意 getscript 进行自动安装。自动安装需要你本人和插件作者双方的同意。见 GetLatestVimScripts_dat 。

GetLatestVimScripts_dat

一个 <GetLatestVimScripts.dat> 文件的示例：

```
ScriptID SourceID Filename
---------------------------------------
294 1 :AutoInstall: Align.vim
120 2 Decho.vim
 40 3 DrawIt.tar.gz
451 4 EasyAccents.vim
195 5 engspchk.vim
642 6 GetLatestVimScripts.vim
489 7 Manpageview.vim
```

备注：必须提供开始的两行，但它们基本上只起注释的作用。

---

5. GetLatestVimScripts 友好插件 getscript-plugins glvs-plugins

(这一节适用于插件作者们)

如果插件作者在插件的任何地方包含了下面形式的注释，GetLatestVimScripts 会找到它并用来构造用户 GetLatestVimScripts.dat 文件：

```
                    src_id
                      v
" GetLatestVimScripts: ### ### yourscriptname
                        ^
                    scriptid
```

作为作者，加上该行就可以引用你自己的脚本，如果需要，也可以附加注释行来描述依赖的其它脚本。当然，使用相同格式！

多数脚本都可以自动安装。如果你的命令也可以（见 glvs-autoinstall ），那么可在 "yourscriptname" 开始的位置之前加上 :AutoInstall: 。

```
                    src_id
                      v
" GetLatestVimScripts: ### ### :AutoInstall: yourscriptname
                        ^
                    scriptid
```

备注：:AutoInstall: 功能需要插件作者和用户双方的同意才能进行！

如果还没有加入的话，此时就会在用户 GetLatest/GetLatestVimScripts.dat 文件里加入这些脚本的 GetLatestVimScript 命令。这是一个相对简单的自动获取所有你的插件所依赖的脚本的方法。

不过，作为作者，你可能不想让 GetLatestVimScripts 下载你自己的脚本，因为这样会覆盖你还没有发布的工作。GetLatestVimScripts 提供了解决这个问题的方法： 把

```
0 0 yourscriptname
```

放入 <GetLatestVimScripts.dat> 文件。这样 GetLatestVimScripts 就会跳过 "yourscriptname" 脚本的 GetLatestVimScript 注释行检查。从而，这些行就不会加到 <GetLatestVimScripts.dat> 文件里并进而下载你自己的脚本。如果包含了 :AutoInstall: 选项，这一点尤其重要。

确定 "0 0 yourscriptname" 用的 "yourscriptname" 和 GetLatestVimScript 注释里的相同！

---

6. GetLatestVimScripts 自动安装

getscript-autoinstall
glvs-autoinstall

GetLatestVimScripts 现在支持了 "AutoInstall"（自动安装）。并非所有脚本都支持自动安装。若干脚本或有特殊的安装需求（请参阅这些脚本的 "install"（安装）指示）。换一角度讲，可以自动安装的脚本还是占多数的。

要让 GetLatestVimScripts 进行自动安装，数据文件的注释域须以下列文字开始（前后有空白无妨）：

```
:AutoInstall:
```

必须给出前后两个冒号，也必须放在注释（yourscriptname）域的开始处。

如果你不想进行任何自动安装，在 <.vimrc> 里放入：

```
let g:GetLatestVimScripts_allowautoinstall= 0
```

缺省打开 :AutoInstall:。此时，名字里包含如下后缀的文件会被相应地

```
---.tar.bz2  : 解压和 untar 解包到 .vim /目录
---.vba.bz2  : 解压到 .vim/ 目录，然后由 vimball 处理
---.vim.bz2  : 解压并移到 .vim/plugin 目录
---.tar.gz   : 解压和 untar 解包到 .vim/ 目录
---.vba.gz   : 解压到 .vim/ 目录，然后由 vimball 处理
---.vim.gz   : 解压并移到 .vim/plugin 目录
---.vba      : 解压到 .vim/ 目录
---.vim      : 移到 .vim/plugin 目录
---.zip      : unzip 解压到 .vim/ 目录
```

如果只要通过 untar/gunzip 或 "移到插件目录" 过程来放置各个部件就可以使用，那么这个脚本就称之为可以自动安装。当然，Vimball 包总是可以自动安装的。

什么时候脚本不能自动安装？让我举个例子：

[.vim|vimfiles]/after/syntax/blockhl.vim

<blockhl.vim> 脚本提供 C/C++ 程序的块高亮；它可以这样得到：

http://vim.sourceforge.net/scripts/script.php?script_id=104

目前，vim 的 after/syntax 只支持按文件类型命名的脚本 (blockhl.vim 的情况应该使用 after/syntax/c.vim)。所以，如果使用自动安装，可能会覆盖当前用户自己的 after/syntax/c.vim 文件。

我自己的例子。我用 <aftersyntax.vim> (换名到 after/syntax/c.vim) 以支持 after/syntax/c/ 目录：

http://vim.sourceforge.net/scripts/script.php?script_id=1023

该脚本允许多个语法文件在 after/syntax/c 子目录同时存在。我不能把 aftersyntax.vim 捆绑进来并构造一个合适的自动安装用 tar 包，因为它包含的 after/syntax/c.vim 可能会覆盖用户自己的 c.vim。

---

7. GetLatestVimScripts 选项 glvs-options

g:GetLatestVimScripts_wget
缺省= "wget"
该变量记住获取脚本命令名。

g:GetLatestVimScripts_options
缺省= "-q -O"
该变量记住用于 g:GetLatestVimScripts_wget 命令的选项。

g:GetLatestVimScripts_allowautoinstall
缺省= 1
该变量指示是否允许 GetLatestVimScripts 尝试自动安装脚本。更进一步地，插件作者必须显式地指明他/她的插件是否可以自动安装 (通过 GetLatestVimScripts 注释行的 :AutoInstall: 关键字)。

g:GetLatestVimScripts_autoinstalldir
缺省= $HOME/.vim (linux)
缺省= $HOME/vimfiles (windows)
覆盖 :AutoInstall: 脚本安装的位置。不用于覆盖 vimball 安装的位置。

g:GetLatestVimScripts_scriptaddr
缺省='http://vim.sourceforge.net/script.php?script_id='
如果你的系统需要，可以覆盖之
... ='http://vim.sourceforge.net/script/script.php?script_id='

---

8. GetLatestVimScripts 算法 glvs-algorithm glvs-alg

Vim sourceforge 页面用所谓的 script-id 键动态建立页面。以下网页

        http://vim.sourceforge.net/scripts/script.php?script_id=40

指定最新的 source-id (src_id)。该源标识符 (source-id) 总是不断递增，因此如果 src_id 大于 GetLatestVimScripts 脚本记录的值，那就是时候下载该脚本的新版本了。

此时，GetLatestVimScripts 下载脚本并更新内部数据库里的 script id、 source id 和 scriptname。

然后，AutoInstall 过程会:

        从 GetLatest/ 把文件移动到下面的目录
                Unix   : $HOME/.vim
                Windows: $HOME\vimfiles
        如果下载文件以 ".bz2" 结尾
                进行 bunzip2 解压
        否则如果下载文件以 ".gz" 结尾
                进行 gunzip 解压
        如果产生的文件以 ".zip" 结尾
                进行 unzip 解压
        否则如果产生的文件以 ".tar" 结尾
                进行 tar -oxvf 解包
        否则如果产生的文件以 ".vim" 结尾
                把它移动到 plugin 子目录

---

9. GetLatestVimScripts 历史 (英文) getscript-history glvs-hist {{{1

```
v36 Apr 22, 2013 : * (glts) suggested use of plugin/**/*.vim instead of
                     plugin/*.vim in globpath() call.
                   * (Andy Wokula) got warning message when setting
                     g:loaded_getscriptPlugin
v35 Apr 07, 2012 : * (MengHuan Yu) pointed out that the script URL has
                     changed (somewhat).  However, it doesn't work, and
                     the original one does (under Linux). I'll make it
                     yet-another-option.
v34 Jun 23, 2011 : * handles additional decompression options for tarballs
v33 May 31, 2011 : * using fnameescape() instead of escape()
                   * *.xz support
v32 Jun 19, 2010 : * (Jan Steffens) added support for xz compression
v31 Jun 29, 2008 : * (Bill McCarthy) fixed having hls enabled with getscript
                   * (David Schaefer) the acd option interferes with vimballs
                     Solution: bypass the acd option
v30 Jun 13, 2008 : * GLVS now checks for existence of fnameescape() and will
                     issue an error message if it is not supported
v29 Jan 07, 2008 : * Bram M pointed out that cpo is a global option and that
                     getscriptPlugin.vim was setting it but not restoring it.
v28 Jan 02, 2008 : * improved shell quoting character handling, cygwin
                     interface, register-a bypass
    Oct 29, 2007   * Bill McCarthy suggested a change to getscript that avoids
                     creating pop-up windows
v24 Apr 16, 2007 : * removed save&restore of the fo option during script
```

```
                    loading
v23 Nov 03, 2006 : * ignores comments (#...)
                   * handles vimballs
v22 Oct 13, 2006 : * supports automatic use of curl if wget is not
                    available
v21 May 01, 2006 : * now takes advantage of autoloading.
v20 Dec 23, 2005 : * Eric Haarbauer found&fixed a bug with unzip use;
                    unzip needs the -o flag to overwrite.
v19 Nov 28, 2005 : * v18's GetLatestVimScript line accessed the wrong
                    script! Fixed.
v18 Mar 21, 2005 : * bugfix to automatic database construction
                   * bugfix - nowrapscan caused an error
                    (tnx to David Green for the fix)
    Apr 01, 2005   * if shell is bash, "mv" instead of "ren" used in
                    :AutoInstall:s, even though its o/s is windows
    Apr 01, 2005   * when downloading errors occurred, GLVS was
                    terminating early.  It now just goes on to trying
                    the next script (after trying three times to
                    download a script description page)
    Apr 20, 2005   * bugfix - when a failure to download occurred,
                    GetLatestVimScripts would stop early and claim that
                    everything was current.  Fixed.
v17 Aug 25, 2004 : * g:GetLatestVimScripts_allowautoinstall, which
                    defaults to 1, can be used to prevent all
                    :AutoInstall:
v16 Aug 25, 2004 : * made execution of bunzip2/gunzip/tar/zip silent
                   * fixed bug with :AutoInstall: use of helptags
v15 Aug 24, 2004 : * bugfix: the "0 0 comment" download prevention wasn't
                    always preventing downloads (just usually).  Fixed.
v14 Aug 24, 2004 : * bugfix -- helptags was using dotvim, rather than
                    s:dotvim.  Fixed.
v13 Aug 23, 2004 : * will skip downloading a file if its scriptid or srcid
                    is zero.  Useful for script authors; that way their
                    own GetLatestVimScripts activity won't overwrite
                    their scripts.
v12 Aug 23, 2004 : * bugfix - a "return" got left in the distribution that
                    was intended only for testing.  Removed, now works.
                   * :AutoInstall: implemented
v11 Aug 20, 2004 : * GetLatestVimScripts is now a plugin:
                   * :GetLatestVimScripts command
                   * (runtimepath)/GetLatest/GetLatestVimScripts.dat
                    now holds scripts that need updating
v10 Apr 19, 2004 : * moved history from script to doc
v9  Jan 23, 2004 :   windows (win32/win16/win95) will use
                    double quotes ("") whereas other systems will use
                    single quotes ('') around the urls in calls via wget
v8  Dec 01, 2003 :   makes three tries at downloading
v7  Sep 02, 2003 :   added error messages if "Click on..." or "src_id="
                    not found in downloaded webpage
                    Uses t_ti, t_te, and rs to make progress visible
v6  Aug 06, 2003 :   final status messages now display summary of work
                    ( "Downloaded someqty scripts" or
                      "Everything was current")
                    Now GetLatestVimScripts is careful about downloading
```

```
                    GetLatestVimScripts.vim itself!
                    (goes to <NEW_GetLatestVimScripts.vim>)
v5  Aug 04, 2003 :  missing an endif near bottom
v4  Jun 17, 2003 :  redraw! just before each "considering" message
v3  May 27, 2003 :  Protects downloaded files from errant shell
                    expansions with single quotes: '...'
v2  May 14, 2003 :  extracts name of item to be obtained from the
                    script file.  Uses it instead of comment field
                    for output filename; comment is used in the
                    "considering..." line and is now just a comment!
                  * Fixed a bug: a string-of-numbers is not the
                    same as a number, so I added zero to them
                    and they became numbers.  Fixes comparison.
```

```
vim:tw=78:ts=8:noet:ft=help:fdm=marker
```

VIM 参考手册 by Bram Moolenaar
译者: iCrazy <icrazy@ustc.edu> http://vimcdoc.sf.net

用 Vim 编辑压缩的文件 gzip bzip2 compress

1. 自动命令 gzip-autocmd

这里涉及到的功能是一个 standard-plugin 。
这个插件仅在 'compatible' 选项没有被置位的情况下有效。
您可以通过设置变量 "loaded_gzip" 来避免装载这个插件:

```
:let loaded_gzip = 1
```

---

1. 自动命令 gzip-autocmd

这个插件建立自动命令来截获对具有以下扩展名的文件的读写:

扩展名	压缩格式
*.Z	compress (Lempel-Ziv)
*.gz	gzip
*.bz2	bzip2
*.lzma	lzma
*.xz	xz
*.lz	lzip
*.zst	zstd

您仅仅知道这些就已经足够了，此插件没有选项。

在解压缩一个文件之后，会自动重新检测文件格式。这样可能会使一个类似 "foo.c.gz" 的文件具有 "c" 的文件格式。

如果您设置了 'patchmode'，在进行压缩后该值会被自动加到文件的扩展名后面。因而编辑 patchmode 文件不会产生自动解压缩的效果。如果您希望这样，您需要改变文件名。

译者: Willis
http://vimcdoc.sf.net

Author: Charles E. Campbell <NdrOchip@ScampbellPfamily.AbizM>
Copyright: (c) 2004-2015 by Charles E. Campbell logiPat-copyright
The VIM LICENSE applies to LogiPat.vim and LogiPat.txt
(see copyright) except use "LogiPat" instead of "Vim"
No warranty, express or implied. Use At-Your-Own-Risk.

---

1. 内容 logiPat logiPat-contents

1. 内容.....: logiPat-contents
2. LogiPat 手册.....: logiPat-manual
3. LogiPat 示例.....: logiPat-examples
4. 警告.....: logiPat-caveat
5. LogiPat 历史.....: logiPat-history

---

2. LogiPat 手册 logiPat-manual logiPat-man

logiPat-arg logiPat-input logiPat-pattern logiPat-operators
布尔逻辑模式的组成是

```
operators  ! = 非
           | = 逻辑或
           & = 逻辑与
grouping   ( ... )
patterns   "pattern"
```

:LogiPat {布尔逻辑模式} :LogiPat
:LogiPat 是一个接受布尔逻辑参数的命令 (logiPat-arg)。

:LP {布尔逻辑模式} :LP
:LP 是 :LogiPat 的简写命令版本

:LPE {布尔逻辑模式} :LPE
不执行搜索，但进行布尔逻辑模式到正规表达式的转换，并在屏幕上回显。

:LogiPatFlags {搜索标志位} LogiPat-flags
LogiPat 使用 search() 命令。:LogiPatFlags 命令可用来指定传递给 search() 调用的标志位。

:LPF {搜索标志位} :LPF
:LPF 是 :LogiPatFlags 的简写版本。

:let pat=LogiPat({布尔逻辑模式}) LogiPat()
如果直接调用 LogiPat()，不执行搜索，但进行布尔逻辑模式到正规表达式的转换并返回之。

要在模式里指定 " 而不是用作模式的定界，给双引号加倍。

---

3. LogiPat 示例 logiPat-examples

LogiPat 接受布尔逻辑参数，产生正规表达式来实现选择。下面提供一系列的例子 :

:LogiPat "abc"
会搜索包含字符串 :abc: 的行

:LogiPat "ab""cd"
会搜索包含字符串 :ab"c: 的行

:LogiPat !"abc"
会搜索不包含字符串 :abc: 的行

:LogiPat "abc"|"def"
会搜索包含字符串 :abc: 或 :def: 的行

:LogiPat !("abc"|"def")
会搜索既不包含字符串 :abc: 也不包含 :def: 的行

:LogiPat "abc"&"def"
会搜索同时包含字符串 :abc: 和 :def: 的行

:let pat= LogiPat('!"abc"')
会返回正规表达式，可用来匹配所有不包含 :abc: 的行。双倍的引号是必要的，它可以把正常的匹配传递给 LogiPat，并把这种模式和布尔逻辑的操作符区别开来。

---

4. 警告 logiPat-caveat

"not" 操作符可能比较脆弱；也就是，它并不总是能和 & (逻辑与) 和 | (逻辑或) 操作符合作得很好。请先试试你的模式，可能的话用 :set hls，以确保匹配符合你的期望。

---

5. LogiPat 历史 (英文) logiPat-history

```
v4 Jun 22, 2015 * LogiPat has been picked up by Bram M for standard
                  plugin distribution; hence the name change
v3 Sep 25, 2006 * LP_Or() fixed; it now encapsulates its output
                  in \%(...\) parentheses
   Dec 12, 2011 *  :LPE  added
                * "" is mapped to a single " and left inside patterns
v2 May 31, 2005 * LPF and LogiPatFlags commands weren't working
v1 May 23, 2005 * initial release
```

vim:tw=78:ts=8:noet:ft=help

pi_netrw.txt For Vim version 8.2. 最近更新：2020年1月

NETRW 参考手册 作者：Charles E. Campbell

译者：Willis
http://vimcdoc.sf.net

Author: Charles E. Campbell <NdrOchip@ScampbellPfamily.AbizM>
(remove NOSPAM from Campbell's email first)

Copyright: Copyright (C) 2017 Charles E Campbell netrw-copyright
The VIM LICENSE applies to the files in this package, including
netrw.vim, pi_netrw.txt, netrwFileHandlers.vim, netrwSettings.vim, and
syntax/netrw.vim. Like anything else that's free, netrw.vim and its
associated files are provided *as is* and comes with no warranty of
any kind, either expressed or implied. No guarantees of
merchantability. No guarantees of suitability for any purpose. By
using this plugin, you agree that in no event will the copyright
holder be liable for any damages resulting from the use of this
software. Use at your own risk!

netrw
dav ftp netrw-file rcp scp
davs http netrw.vim rsync sftp
fetch network

---

1. 内容 netrw-contents {{{1

1. 内容 ..... netrw-contents
2. netrw 初步 ..... netrw-start
3. netrw 参考手册 ..... netrw-ref
外部程序及协议 ..... netrw-externapp
读取 ..... netrw-read
写入 ..... netrw-write
执行 ..... netrw-source
目录列表 ..... netrw-dirlist
改变用户名和密码 ..... netrw-chgup
变量与设置 ..... netrw-variables
路径 ..... netrw-path
4. 基于网络的文件传输 ..... netrw-xfer
NETRC ..... netrw-netrc
密码 ..... netrw-passwd
5. 激活 ..... netrw-activate
6. 透明远程文件编辑 ..... netrw-transparent
7. Ex 命令 ..... netrw-ex
8. 变量和选项 ..... netrw-var
9. 目录浏览 ..... netrw-browse
浏览简介 ..... netrw-intro-browse
快速参考：映射 ..... netrw-browse-maps

快速参考：命令 ..... netrw-browse-cmds
显示横幅 ..... netrw-I
把目录加入书签 ..... netrw-mb
浏览 ..... netrw-cr
压缩当前树列表目录 ..... netrw-s-cr
水平分割窗口方式浏览 ..... netrw-o
在新标签页中浏览 ..... netrw-t
垂直分割窗口方式浏览 ..... netrw-v
改变列表方式.(瘦 宽 长 树) ..... netrw-i
切换到收入书签的目录 ..... netrw-gb
切换到较早访问的目录 ..... netrw-u
切换到较迟访问的目录 ..... netrw-U
通过特殊处理程序定制浏览 ..... netrw-x
删除书签 ..... netrw-mB
删除文件或目录 ..... netrw-D
目录探索命令 ..... netrw-explore
使用星号和模式进行探索 ..... netrw-star
显示文件信息 ..... netrw-qf
编辑文件或目录隐藏列表 ..... netrw-ctrl-h
编辑排序序列 ..... netrw-S
强制作为文件或目录对待 ..... netrw-gd netrw-gf
向上一层 ..... netrw--
隐藏文件或目录 ..... netrw-a
改进浏览 ..... netrw-ssh-hack
列出书签和历史 ..... netrw-qb
建立新目录 ..... netrw-d
使浏览目录成为当前目录 ..... netrw-cd
标记文件 ..... netrw-mf
撤销文件标记 ..... netrw-mF
用位置列表标记文件 ..... netrw-qL
用 QuickFix 列表标记文件 ..... netrw-qF
用正规表达式标记文件 ..... netrw-mr
带标记文件：任意外壳命令 ..... netrw-mx
带标记文件：任意外壳命令，整块 ..... netrw-mX
带标记文件：任意命令 ..... netrw-mv
带标记文件：参数列表 ..... netrw-ma netrw-mA
带标记文件：缓冲区列表 ..... netrw-cb netrw-cB
带标记文件：压缩及解压缩 ..... netrw-mz
带标记文件：复制 ..... netrw-mc
带标记文件：比较 ..... netrw-md
带标记文件：编辑 ..... netrw-me
带标记文件：grep ..... netrw-mg
带标记文件：按后缀隐藏或显示 ..... netrw-mh
带标记文件：移动 ..... netrw-mm
带标记文件：打印 ..... netrw-mp
带标记文件：执行 ..... netrw-ms
带标记文件：设置目标目录 ..... netrw-mt
带标记文件：建立标签 ..... netrw-mT
带标记文件：用书签作为目标目录 ..... netrw-Tb
带标记文件：用历史作为目标目录 ..... netrw-Th
带标记文件：撤销标记 ..... netrw-mu
netrw 浏览器变量 ..... netrw-browser-var
netrw 浏览和选项不兼容性 ..... netrw-incompatible
netrw 设置窗口 ..... netrw-settings-window

获取文件..... netrw-O
预览窗口..... netrw-p
前次窗口..... netrw-P
刷新列表..... netrw-ctrl-l
反转排序顺序..... netrw-r
给文件或目录换名..... netrw-R
选择排序风格..... netrw-s
选择编辑窗口..... netrw-C
10. 问题和修正..... netrw-problems
11. 调试 netrw 自身..... netrw-debug
12. 历史..... netrw-history
13. TODO ..... netrw-todo
14. 鸣谢..... netrw-credits

---

2. netrw 初步 netrw-start {{{1

netrw 使得读写文件和通过本地或网络进行和浏览变得容易!
首先，确信你打开了插件。这样，你需要至少在 <.vimrc> 里设置: (或者参见 netrw-activate)

```
set nocp                    " 不设置 'compatible'
filetype plugin on          " 允许使用插件
```

(见 'cp' 和 :filetype-plugin-on)

netrw 支持用 url "透明" 地编辑其它机器上的文件 (见 netrw-transparent)。考虑一个例子，假设你在其它机器上有一个帐号; 如果 scp 可用的话，尝试

```
vim scp://hostname/path/to/file
```

想使 ssh/scp 更容易使用么? 察看 netrw-ssh-hack !

如果你只有 ftp，没有 ssh/scp 怎么办? 也很容易; 尝试

```
vim ftp://hostname/path/to/file
```

想使 ftp 更容易使用么? 检查你的 ftp 是不是支持叫 <.netrc> 的文件 -- 通常，它在你的主目录里，读写权限的设置只限用户自己可读 (也就是，没有组、世界、其它用户等的权限)，其形式是以下这样的行

```
machine HOSTNAME login USERID password "PASSWORD"
machine HOSTNAME login USERID password "PASSWORD"
...
default          login USERID password "PASSWORD"
```

Windows 的 ftp 不支持 .netrc; 不过，可以在 .vimrc 里这样做:

```
let g:netrw_ftp_cmd= 'c:\Windows\System32\ftp -s:C:\Users\MyUserName\MACHINE'
```

Netrw 会用 URL 的主机名来代替 "MACHINE"，从而，你可以为每个网站分别提供一个文件: c:\Users\MyUserName\MachineName，来指定

```
userid
password
```

还有浏览 -- 你只想在编辑文件前四处看看。在自己机器上浏览，只要 "编辑" 目录：

```
vim .
vim /home/userid/path
```

要在远端机器上浏览，也就是 "编辑" 目录（但要确保目录名后面跟上 "/"）：

```
vim scp://hostname/
vim ftp://hostname/path/to/dir/
```

详情可见 netrw-browse ！

除了 scp 和 ftp 以外，还支持更多其他的协议！关于如何在 netrw 和 vim 中使用这些外部应用，见下一节， netrw-externapp 。

禁 止 载 入 netrw-noload

如果你想使用插件，但由于某种原因不想用 netrw，那你需要禁止调入 netrw 插件本身和自动调入的部分。为此，在 <.vimrc> 里加入如下两行：

```
:let g:loaded_netrw       = 1
:let g:loaded_netrwPlugin = 1
```

---

3. netrw 参考手册 netrw-ref {{{1

除了 netrw-start 提到的 scp 和 ftp 以外，netrw 还支持几个协议。包括 dav、fetch、http、...。可以参阅 netrw-externapp 的列表。每个协议和一个变量关联，该变量指定支持协议的缺省命令。

外 部 程 序 及 协 议 netrw-externapp {{{2

协议	变量	缺省值	
dav:	g:netrw_dav_cmd	= "cadaver"	如果 cadaver 可执行
dav:	g:netrw_dav_cmd	= "curl -o"	否则，如果 curl 可执行
fetch:	g:netrw_fetch_cmd	= "fetch -o"	如果 fetch 可用
ftp:	g:netrw_ftp_cmd	= "ftp"	
http:	g:netrw_http_cmd	= "elinks"	如果 elinks 可用
http:	g:netrw_http_cmd	= "links"	否则，如果 links 可用
http:	g:netrw_http_cmd	= "curl"	否则，如果 curl 可用
http:	g:netrw_http_cmd	= "wget"	否则，如果 wget 可用
http:	g:netrw_http_cmd	= "fetch"	否则，如果 fetch 可用
http:	g:netrw_http_put_cmd	= "curl -T"	
rcp:	g:netrw_rcp_cmd	= "rcp"	
rsync:	g:netrw_rsync_cmd	= "rsync"	（见 g:netrw_rsync_sep ）
scp:	g:netrw_scp_cmd	= "scp -q"	
sftp:	g:netrw_sftp_cmd	= "sftp"	
file:	g:netrw_file_cmd	= "elinks" 或 "links"	

g:netrw_http_xcmd : http://... 协议的选项字符串由此字符串指定，而且可以独立修改。缺省地，http 处理命令的选项参数是

```
elinks : "-source >"
links  : "-dump >"
curl   : "-L -o"
wget   : "-q -O"
fetch  : "-o"
```

例如，如果你的系统有 elinks 而你想看渲染后的页面内容（而不是缺省的显示源代码），可以在 .vimrc 里加上

```
let g:netrw_http_xcmd= "-dump >"
```

g:netrw_http_put_cmd: 此选项指定可执行文件和任何必须的选项。此命令在 url 上执行 PUT 操作。

读 取 netrw-read netrw-nread {{{2

一般的，普通的编辑命令都可以使用 URL 形式，例如

```
:e ftp://[user@]machine/path
```

Netrw 也支持 Nread 命令:

```
:Nread ?                                    给出帮助
:Nread "machine:file"                       使用 rcp
:Nread "machine file"                       使用 ftp 并用 <.netrc>
:Nread "machine id password file"           使用 ftp
:Nread "dav://machine[:port]/file"          使用 cadaver
:Nread "fetch://[user@]machine/file"        使用 fetch
:Nread "ftp://[user@]machine[[:#]port]/file"  使用 ftp 并用 <.netrc>
:Nread "http://[user@]machine/file"         使用 http 用 wget 方法
:Nread "rcp://[user@]machine/file"          使用 rcp
:Nread "rsync://[user@]machine[:port]/file"  使用 rsync
:Nread "scp://[user@]machine[[:#]port]/file"  使用 scp
:Nread "sftp://[user@]machine/file"         使用 sftp
```

写 入 netrw-write netrw-nwrite {{{2

普通的保存命令都可以使用 URL 形式，例如

```
:w ftp://[user@]machine/path
```

Netrw 也支持 Nwrite 命令:

```
:Nwrite ?                                   给出帮助
:Nwrite "machine:file"                      使用 rcp
:Nwrite "machine file"                      使用 ftp 并用 <.netrc>
:Nwrite "machine id password file"          使用 ftp
:Nwrite "dav://machine[:port]/file"         使用 cadaver
:Nwrite "ftp://[user@]machine[[:#]port]/file"  使用 ftp 并用 <.netrc>
:Nwrite "rcp://[user@]machine/file"         使用 rcp
:Nwrite "rsync://[user@]machine[:port]/file"  使用 rsync
:Nwrite "scp://[user@]machine[[:#]port]/file"  使用 scp
:Nwrite "sftp://[user@]machine/file"        使用 sftp
```

不支持 http: 协议!

执 行 脚 本 netrw-source {{{2

普通的执行脚本命令都可以使用 URL 形式，例如

```
:so ftp://[user@]machine/path
```

Netrw 也支持 Nsource 命令:

```
:Nsource ?                                        给出帮助
:Nsource "dav://machine[:port]/path"              使用 cadaver
:Nsource "fetch://[user@]machine/path"            使用 fetch
:Nsource "ftp://[user@]machine[[:#]port]/path"    使用 ftp w/ <.netrc>
:Nsource "http://[user@]machine/path"             使用 http 用 wget 方法
:Nsource "rcp://[user@]machine/path"              使用 rcp
:Nsource "rsync://[user@]machine[:port]/path"     使用 rsync
:Nsource "scp://[user@]machine[[:#]port]/path"    使用 scp
:Nsource "sftp://[user@]machine/path"             使用 sftp
```

目 录 列 表 netrw-trailingslash netrw-dirlist {{{2

要浏览一个目录以得到文件列表，只需编辑该目录:

```
:e scp://[user]@hostname/path/
:e ftp://[user]@hostname/path/
```

对于远程目录列表 (也就是使用 scp 或 ftp 的那些)，结尾的 "/" 是必需的 (它告诉 netrw 对该参数使用目录方式浏览而不是去下载文件)。

Nread 命令也可完成 (再说一次，结尾的斜杠是必需的):

```
:Nread [protocol]://[user]@hostname/path/
```

netrw-login netrw-password
改 变 用 户 名 和 密 码 netrw-chgup netrw-userpass {{{2

使用 ftp 的尝试会提示你输入用户名字和密码。它们被存在全局变量 g:netrw_uid 和 s:netrw_passwd 里。下一次使用 ftp 就会再次使用这两个字符串来简化 ftp 将来的应用。不过，如果你要使用别的用户名和/或密码，先调用 NetUserPass() 。为了解决输入密码的问题，检查一下你的 ftp 是否支持在你的主目录下的 <.netrc> 文件。另见 netrw-passwd (如果你用 ssh/scp 但希望知道 scp 如何能不输入密码，见 netrw-ssh-hack)。

```
:NetUserPass [uid [password]]          -- 如果需要，提示
:call NetUserPass()                    -- 提示 uid 和 password
:call NetUserPass("uid")               -- 提示 password
:call NetUserPass("uid","password")    -- 设置全局 uid 和 password
```

(相关主题: ftp netrw-userpass netrw-start)

NETRW 变 量 和 设 置 netrw-variables {{{2
(另见:
netrw-browser-var : netrw 浏览器选项变量

```
    netrw-protocol          : 文件传输协议选项变量
    netrw-settings          : 附加文件传输选项
    netrw-browser-options   : 这些选项影响如何浏览目录
  )
```

netrw 提供很多变量，方便你根据自己的喜好定制 netrw。一个查阅这些变量的方法是 :NetrwSettings（见 netrw-settings），它显示你当前的 netrw 设置。大多数设置在下面描述，还有的在 netrw-browser-options 和 netrw-externapp 那里:

b:netrw_lastfile
最近一次通过网络读写的文件，该变量局部于缓冲区
(用于支持无参数的 :Nw)

g:netrw_bufsettings
netrw 缓冲区使用的设置
(缺省) noma nomod nonu nowrap ro nobl

g:netrw_chgwin
指定后续文件编辑使用的窗口号（另见 netrw-C ）
(缺省) -1

g:Netrw_funcref
指定 netrw 编辑文件时调用的函数。先编辑文件，然后调用此函数引用 (Funcref)。
该变量也可包含函数引用的列表。
(缺省) 无
(g:Netrw... 中的大写是必需的，因为它保存函函数引用)

```
示例: 放入 .vimrc; 影响所有文件打开
fun! MyFuncRef()
endfun
let g:Netrw_funcref= function("MyFuncRef")
```

g:Netrw_UserMaps
指定函数或函数的 List ，可用于设置用户指定的映射和功能。
见 netrw-usermaps

g:netrw_ftp
如果不存在，用缺省的 ftp
=0 使用缺省的 ftp (uid password)
=1 使用替代的 ftp 方法 (user uid password)
如果你的 ftp 使用有问题，尝试修改此值，看看另一个 ftp 方法是否适合你的设置。

g:netrw_ftp_options
缺省使用，这些选项的目的是关闭交互揭示，并且避免初始连接时自动重新登录。但是，看来不是所有 ftp 的实现都支持（例如 ncftp)。
="-i -n"

g:netrw_ftpextracmd
缺省: 不存在
如果此变量存在，那么它包含的任何字符串将被加入到发送给 ftp 客户的命令中。一个例子:
="passive"

g:netrw_ftpmode
="binary" (缺省)
="ascii"

g:netrw_ignorenetrc
=0 (linux、cygwin 缺省)

=1 如果你有 <.netrc> 文件但不能工作，设置此变量可以忽略该文件。(Windows + cmd.exe 缺省)

g:netrw_menu
=0 关闭 netrw 菜单
=1 (缺省) 打开 netrw 菜单

g:netrw_nogx
如果此变量存在，"gx" 映射不可用 (见 netrw-gx)

g:netrw_uid
(ftp) 用户名， 可用于同一 vim 会话中

s:netrw_passwd
(ftp) 密码， 可用于同一 vim 会话中

g:netrw_preview
=0 (缺省) 在横向分割的窗口中显示预览窗口
=1 在纵向分割的窗口中显示预览窗口以相同方式影响 "前次窗口" (见 netrw-P)。
g:netrw_alto 变量可用于提供额外的分割控制:

g:netrw_preview	g:netrw_alto	结果
0	0	:aboveleft
0	1	:belowright
1	0	:topleft
1	1	:botright

要控制大小，可见 g:netrw_winsize

g:netrw_scpport
= "-P" : 设置 scp 端口的选项

g:netrw_sshport
= "-p" : 设置 ssh 端口的选项

g:netrw_sepchr
=\0xff
=\0x01 如果 enc == euc-jp (也许也适用于其它编码，请让我知道)
内部用来分隔文件名和优先码。
见 netrw-p12 。

g:netrw_silent
=0 : 正常方式完成传输
=1 : 安静方式完成传输

g:netrw_use_errorwindow
=1 : netrw 的消息会使用分开的单行窗口。该窗口提供了消息的可靠传送。(缺省)
=0 : netrw 的消息使用 echoerr；这种方式下不是所有消息都保证能看到，但不需要退出窗口。

g:netrw_win95ftp
=1 如果使用 Win95，删除四个操作系统在传输时 "提供" 的拖尾行
=0 强制正常的 ftp 行为 (不删除拖尾的行)

g:netrw_cygwin
=1 假设在 windows 上的 scp 是 cygwin 的版本。同时允许用带时间和大小排序的 ls 进行网络浏览 (windows 的缺省值)
=0 假设在 windows 上的 scp 接受 windows 风格的路径网络浏览使用 dir 而不是 ls
如果使用 unix，忽略此选项

g:netrw_use_nt_rcp
=0 不要使用 WinNT、Win2000 和 WinXP 的 rcp
=1 使用 WinNT 二进制模式的 rcp (缺省)

路 径 netrw-path {{{2

对于多数协议，文件路径通常相对于用户目录。不过，有的协议里路径可以相对于一些相关的目录。

例如： vim scp://user@host/somefile
例如： vim scp://user@host/subdir1/subdir2/somefile

其中 "somefile" 在 "user" 的主目录。如果你想要文件相对于根目录的路径，使用完整路径：

例如： vim scp://user@host//somefile
例如： vim scp://user@host//subdir1/subdir2/somefile

4．基于网络的文件传输 netrw-xfer {{{1

Vim 下基于网络的文件传输是用基于 Vim 脚本的插件技术实现的 (<netrw.vim>)。目前，它支持基于网络的读写，并支持 rcp、scp、ftp 或者 ftp+<.netrc>、scp、fetch、dav/cadaver、rsync、 或者 sftp。

目前，http 只支持通过 wget 或者 fetch 读取。

<netrw.vim> 是一个标准插件，它起到 Vim 和若干文件传输程序的粘结剂作用。它使用自动命令事件 (BufReadCmd、FileReadCmd、BufWriteCmd) 来拦截貌似 url 的文件名的读写。

例如 vim ftp://hostname/path/to/file

冒号前面的字符序列指定所使用的协议；在这个例子里，就是 ftp。
<netrw.vim> 脚本然后构成一个或多个命令 (一般是 ftp)，然后用外部程序去执行 (ftp，scp 等) 来完成实际的文件传输/协议。文件被先读写到一个临时文件里 (在 Unix/Linux 上，/tmp/...)， <netrw.vim> 脚本会清理该文件。

现在，说说 Jan Minář 的 "FTP 用户名和密码披露问题": 首先，ftp 不是安全协议。用户名和密码在因特网上是明码传输的；任何网络嗅探器都可以截取；这不是 netrw 的问题，是 ftp 的问题。如果你很介意这点，请使用 scp 或 sftp。

只要远端宿主不变。netrw 会重用同一 vim 会话的用户名和密码。

Jan 也许不很清楚 netrw 对 ftp 的处理方式: 正常情况下，一个 "ftp 会话" 会执行多个命令，他也许认为用户名/密码信息应该只在单个 ftp 会话中保留。但是，netrw 的每个 ftp 操作都是在单独的 "ftp 会话" 中进行的: 只在所谓单个 "ftp 会话" 中记住用户名/密码，就跟完全不记住用户名/密码一样。依我愚见，如果这样去，譬如说，浏览远程的目录，用户很快就会厌烦的。

不过，很感谢 Jan M. 指出许多 netrw (包括 vim 本身) 处理 "特意构造" 文件名的漏洞。 shellescape() 和 fnameescape() 函数就是为此由 Bram Moolenaar 实现来处理这些问题的，netrw 也做了相应的修改，调用了这些函数。不过，我的建议还是，如果所谓的 "文件名" 看来像 vim 的命令而你不放心，不要打开这些文件。

netrw-putty netrw-pscp netrw-psftp
你可以通过修改变量修改任何实现协议的外部应用程序 (例如，scp 使用 g:netrw_scp_cmd 变量，缺省是 "scp -q")。 例如，考虑 PuTTY 的使用:

```
let g:netrw_scp_cmd = '"c:\Program Files\PuTTY\pscp.exe" -q -batch'
```

```
let g:netrw_sftp_cmd= '"c:\Program Files\PuTTY\psftp.exe"'
```

(备注：有报告说 windows 7 的 putty v0.6 的 "-batch" 选项不工作，那个系统最好不要那么改)

netrw-p8 介绍更多，如 putty、pscp、psftp 等。

ftp，一个古老的协议，看来被很多实现所支持。不幸的是，很多实现都有问题（例如，在文件尾部加入垃圾字符）。所以，如果有这样的问题，用户可以自己决定写一个 NetReadFixup() 函数来在从 ftp 读文件后进行清理。有些 Unix 系统（例如，FreeBSD）提供了一个叫 "fetch" 的工具，它使用 ftp 协议，但是没有这样的问题，而且更方便 <netrw.vim> 的使用。所以，只要 "fetch" 可用（也就是可以执行），最好用它来用来实现 ftp://... 的传输。

对于 rcp、scp、sftp 和 http 协议，你可能希望基于网络的文件传输尽可能透明；例如

```
vim rcp://[user@]machine/path
vim scp://[user@]machine/path
```

如果你的 ftp 支持 <.netrc>，那么也可以透明的使用 ftp，因为需要的机器名字，用户名和密码都定义在这个文件里。你的 ftp 必须自己能理解并使用 <.netrc> 文件。可以这样，

```
vim ftp://[user@]machine[[:#]portnumber]/path
```

Windows 提供的 ftp（通常在 c:\Windows\System32\ftp.exe）接受选项 -s:filename (filename 应该是也必须是完整路径)，该文件包含 ftp 启动时自动执行的 ftp 命令。用这个功能，你可以为一个网站输入用户名和密码：

```
userid
password
```

netrw-windows-netrc netrw-windows-s

如果 g:netrw_ftp_cmd 包含 -s:[path/]MACHINE，那么（只适用于 Windows），netrw 会用当前要 ftp 连接的机器名来替代 MACHINE，这样你就可以有多个 machine.ftp 文件，包含不同的登录名和密码。例如：

```
let g:netrw_ftp_cmd= 'c:\Windows\System32\ftp -s:C:\Users\Myself\MACHINE'
vim ftp://myhost.somewhere.net/
```

会使用文件

```
C:\Users\Myself\myhost.ftp
```

通常，ftp 需要查询用户需要的用户名和密码。后者通常不会被显示出来；也就是，输入的密码显示的是星号。netrw 会保留用户名和密码，以便以后从相同机器的读写不需要再提示输入。

netrw-urls

读	写	使用
DAV:		
dav://host/path		cadaver
:Nread dav://host/path	:Nwrite dav://host/path	cadaver

DAV + SSL:		
davs://host/path		cadaver
:Nread davs://host/path	:Nwrite davs://host/path	cadaver
FETCH:		
fetch://[user@]host/path		
fetch://[user@]host:http/path	不可用	fetch
:Nread fetch://[user@]host/path		
FILE:		
file:///*	file:///*	
file://localhost/*	file://localhost/*	
FTP: (*3)	(*3)	
ftp://[user@]host/path	ftp://[user@]host/path	ftp (*2)
:Nread ftp://host/path	:Nwrite ftp://host/path	ftp+.netrc
:Nread host path	:Nwrite host path	ftp+.netrc
:Nread host uid pass path	:Nwrite host uid pass path	ftp
HTTP: wget is executable: (*4)		
http://[user@]host/path	不可用	wget
HTTP: fetch is executable (*4)		
http://[user@]host/path	不可用	fetch
RCP:		
rcp://[user@]host/path	rcp://[user@]host/path	rcp
RSYNC:		
rsync://[user@]host/path	rsync://[user@]host/path	rsync
:Nread rsync://host/path	:Nwrite rsync://host/path	rsync
:Nread rcp://host/path	:Nwrite rcp://host/path	rcp
SCP:		
scp://[user@]host/path	scp://[user@]host/path	scp
:Nread scp://host/path	:Nwrite scp://host/path	scp (*1)
SFTP:		
sftp://[user@]host/path	sftp://[user@]host/path	sftp
:Nread sftp://host/path	:Nwrite sftp://host/path	sftp (*1)

(*1) 要指出绝对路径，用 scp://machine//path。

(*2) 如果 <.netrc> 存在，假设你的 ftp 客户端会使用之。不然，该脚本提示输入用户名和密码。

(*3) 对于 ftp，"machine" 可以是 machine#port 或者 machine:port，如果需要使用不同于标准 ftp 的端口的话。

(*4) 对于 http:...，如果 wget 可用，就使用之。不然，如果 fetch 可用，就使用之。

:Nread 和 :Nwrite 这几个 ex 命令都能接受多个文件名。

NETRC netrw-netrc

<.netrc> 文件通常位于你的主目录，它包含的内容是你想用的和机器名对应的用户名和密码。

<.netrc> 文件典型的语法如下显示。Unix 上的 ftp 通常支持 <.netrc>；而 Windows 上的 ftp 通常不支持。

```
machine {full machine name} login {user-id} password "{password}"
default login {user-id} password "{password}"
```

你的 ftp 客户端必须自己会使用 <.netrc>，如果 <.netrc> 文件存在，ftp 传输就不会询问用户名字和密码。

> 注意：
> 既然这个文件包含密码明码，要确保没人能够读到这个文件！许多程序拒绝使用能给别人读取的 .netrc 文件。当然不要忘记，系统管理员总是可以读到这个文件的！为此，对 Linux/Unix 而言：chmod 600 .netrc

虽然 Windows ftp 客户端不支持 .netrc，netrw 有一个解决方法：见 netrw-windows-s 。

密 码 netrw-passwd

脚本试图用内建的 Vim 函数 inputsecret() 不回显地读取 ftp 的密码。参见 netrw-userpass 了解如何在设置以后修改密码。

不幸的是，netrw 没有办法自动提供密码给 scp。所以每次通过 scp 传输文件都需要重新输入密码。不过， netrw-ssh-hack 有助于解决这个问题。

---

5. 激活 netrw-activate {{{1

基于网络的文件传输缺省在 'nocompatible' 模式打开时就可用。netrw 的脚本文件在你系统的 plugin、autoload 和 syntax 目录下，而在启动 vim 时，只自动加载 plugin/netrwPlugin.vim。netrw 的主要脚本定义在 autoload/netrw.vim 中，它只有在实际使用 netrw 时才会载入。我建议你至少在 <.vimrc> 定制文件里放上：

```
set nocp
if version >= 600
  filetype plugin indent on
endif
```

如果在 .vimrc 文件里也包括以下内容，不带文件名时启动 [g]vim 可以立即激活 netrw 以显示当前目录：

```
" Augroup VimStartup:
augroup VimStartup
  au!
  au VimEnter * if expand("%") == "" | e . | endif
augroup END
```

6. 透明远程文件编辑 netrw-transparent {{{1

透明文件传输指通过普通的文件读写命令进行的传输（通过 :autocmd 的 BufReadCmd 、 BufWriteCmd 或 SourceCmd 事件）。这样网络文件的读写或执行就能和本地文件一样容易。

```
vim ftp://[user@]machine/path
...
:wq
```

netrw-activate 进一步说明如何让你的 vim 更好地使用 netrw 这样的插件。

7. Ex 命令 netrw-ex {{{1

支持普通的读写命令。此外，还有一些额外的命令。通常你不需要使用 Nwrite 或 Nread，就像 netrw-transparent 演示的那样（也就是，用

```
:e URL
:r URL
:w URL
```

等即可）-- 见 netrw-urls 。下面的解释中，{netfile} 指代指向远程文件的 URL。

:Nwrite :Nw

:[range]Nw[rite] 把特定的行写到当前由 b:netrw_lastfile 指定的文件里。
(相关: netrw-write)

:[range]Nw[rite] {netfile} [{netfile}]...
把特定的行写到 {netfile} 所指定的网络文件里。

:Nread :Nr

:Nr[ead] 从 b:netrw_lastfile 指定的文件里读取行到当前的缓冲区。
(相关: netrw-read)

:Nr[ead] {netfile} {netfile}...
读取 {netfile} 到当前行之后。

:Nsource :Ns

:Ns[ource] {netfile}
执行 {netfile}。
启动 vim 时如要使用远程的 .vimrc，可以使用如下（都在同一行上）（多谢 Antoine Mechelynck）的命令

```
vim -u NORC -N
 --cmd "runtime plugin/netrwPlugin.vim"
 --cmd "source scp://HOSTNAME/.vimrc"
```

(相关: netrw-source)

:call NetUserPass() NetUserPass()
如果 g:netrw_uid 和 s:netrw_passwd 不存在，该函数向用户查询。
(相关: netrw-userpass)

:call NetUserPass("userid")
该调用设置 g:netrw_uid 并且，如果密码不存在的话，向用户查询。
(相关： netrw-userpass)

:call NetUserPass("userid","passwd")
该调用设置 g:netrw_uid 和 s:netrw_passwd。用户名和密码则被 ftp 传输使用。要删除用户名和密码，只用使用空字符串 (即 "") 便可。
(相关： netrw-userpass)

:NetrwSettings netrw-settings 里描述此命令 -- 用于显示 netrw 设置和修改 netrw 行为。

---

8. 变量和选项 netrw-var netrw-settings {{{1

(另见： netrw-options netrw-variables netrw-protocol
netrw-browser-settings netrw-browser-options)

<netrw.vim> 脚本提供若干变量，作为控制 <netrw.vim> 行为的选项。这些变量通常在用户的 <.vimrc> 里设置: (另见： netrw-settings netrw-protocol)
netrw-options

---

netrw 选项

选项	含义
b:netrw_col	保存当前的列号 (NetWrite 时)
g:netrw_cygwin	=1 假设在 windows 上的 scp 是 cygwin 的版本 (windows 的缺省值)
	=0 假设在 windows 上的 scp 接受 windows 风格的路径 (其他情况的缺省值)
g:netrw_ftp	=0 使用缺省的 ftp (uid password)
	=1 使用替代的 ftp (user uid password)
	(见 netrw-options)
g:netrw_ftpmode	="binary" (缺省)
	="ascii" (你的选择)
g:netrw_ignorenetrc	= 1 (缺省) 如果你有 <.netrc> 文件，但是你不想用它，就这么设置。它唯一的作用是使得 <.netrc> 被忽略。
b:netrw_lastfile	最近一次通过网络读写的方法/机器/路径。
b:netrw_line	保存当前的行号 (NetWrite 时)
g:netrw_silent	=0 正常方式的传输
	=1 安静方式的传输
g:netrw_uid	保存当前 ftp 的 user-id。
g:netrw_use_nt_rcp	=0 不要使用 WinNT/2K/XP 的 rcp (缺省)
	=1 使用 WinNT/2K/XP 二进制模式的 rcp
g:netrw_win95ftp	=0 使用 unix 风格连接 ftp，即使 win95/98/ME 等
	=1 使用缺省的方法连接 ftp

<

netrw-internal-variables

脚本里也可以使用以下的内部变量，尽管应该是暂时的。

临时变量

选项	含义
b:netrw_method	指明 rcp/ftp+.netrc/ftp 等的索引
w:netrw_method	(同 b:netrw_method)
g:netrw_machine	保存输入分析结果的机器名
b:netrw_fname	保存访问的文件名

<

netrw-protocol

netrw 支持若干协议。调用这些协议时使用下面的列出的变量，用户可以修改。

协议控制选项

选项	类型	设置	意义
netrw_ftp	变量	=不存在	userid 由 "user userid" 设置
		=0	userid 由 "user userid" 设置
		=1	userid 由 "userid" 设置
NetReadFixup	函数	=不存在	没有改变
		=存在	允许用户通过 ftp 自动传输的文件通过 NetReadFixUp() 进行转换
g:netrw_dav_cmd	变量	="cadaver"	如果 cadaver 可执行
g:netrw_dav_cmd	变量	="curl -o"	否则如果 curl 可执行
g:netrw_fetch_cmd	变量	="fetch -o"	如果 fetch 可以执行
g:netrw_ftp_cmd	变量	="ftp"	
g:netrw_http_cmd	变量	="fetch -o"	如果 fetch 可以执行
g:netrw_http_cmd	变量	="wget -O"	不然，如果 wget 可以执行
g:netrw_http_put_cmd	变量	="curl -T"	
g:netrw_list_cmd	变量	="ssh USEPORT HOSTNAME ls -Fa"	
g:netrw_rcp_cmd	变量	="rcp"	
g:netrw_rsync_cmd	变量	="rsync"	
g:netrw_rsync_sep	变量	="/"	用于分隔终端名和文件规格
g:netrw_scp_cmd	变量	="scp -q"	
g:netrw_sftp_cmd	变量	="sftp"	

<

netrw-ftp

g:netrw_...._cmd 选项 （ g:netrw_ftp_cmd 和 g:netrw_sftp_cmd ）指定如何处理特定的协议 (rcp、ftp 等) 的外部程序，也可以包含需要的命令行选项 (如指定 passive

模式的 -p)。例如:

```
let g:netrw_ftp_cmd= "ftp -p"
```

g:netrw_list_cmd 提供浏览支持: 其中的 HOSTNAME 子串会被替代为当前请求的机器名。

两个选项 (g:netrw_ftp 和 netrw-fixup) 都提供解决特定 ftp 问题的帮助。为了更好的理解如何在 ftp 有问题的时候使用这些选项, 以下是 netrw 如何进行 ftp 读取的一点讨论。

对于 ftp, netrw 通常用以下几种格式之一构造命令行, 并写到临时文件里:

如果 g:netrw_ftp 不存在或者不是 1	如果 g:netrw_ftp 存在且为 1
open machine [port]	open machine [port]
user userid password	userid password
[g:netrw_ftpmode]	password
[g:netrw_ftpextracmd]	[g:netrw_ftpmode]
get filename tempfile	[g:netrw_extracmd]
	get filename tempfile

<
g:netrw_ftpmode 和 g:netrw_ftpextracmd 是可选的。

netrw 然后通过过滤命令执行上述命令行:

```
:%! {g:netrw_ftp_cmd} -i [-n]
```

其中

```
g:netrw_ftp_cmd 通常是 "ftp",
-i 告诉 ftp 不要交互
-n 意味着不要用 netrc, 它用于方法 #3 (ftp 不用 <.netrc>)
```

如果 <.netrc> 存在, 就不需要想用户查询用户名和密码。传输完毕后, 保存文件到临时文件里。这时, 从临时文件里读取的内容被读到被请求的主编辑会话窗口中, 并删除临时文件。

如果你的 ftp 不支持 "user" 命令, 而是立即需要输入用户名, 那么试试把 "let netrw_ftp=1" 放到你的 <.vimrc> 里。

netrw-cadaver

要解决不受信任的服务器弹出 SSL 认证对话框的问题, 你可以下载该认证, 并把它放到 /usr/ssl/cert.pem 里。此操作使该服务器被认为是 "可信任的"。

netrw-fixup netreadfixup

如果你的 ftp 因某种原因产生不需要的行 (例如 AUTH 消息), 你需要写一个 NetReadFixup() 函数:

```
function! NetReadFixup(method,line1,line2)
  " a:line1: 当前文件的第一行新行
  " a:line2: 当前文件的最后一行新行
  if     a:method == 1 "rcp
```

```
    elseif a:method == 2 "ftp + <.netrc>
    elseif a:method == 3 "ftp + machine,uid,password,filename
    elseif a:method == 4 "scp
    elseif a:method == 5 "http/wget
    elseif a:method == 6 "dav/cadaver
    elseif a:method == 7 "rsync
    elseif a:method == 8 "fetch
    elseif a:method == 9 "sftp
    else                 " 报错
    endif
  endfunction
```

如果 NetReadFixup() 函数存在，它就会被调用，这样你就可以订制自己的读取进程。一个进一步的例子可以看看 <netrw.vim> 如何处理 Windows 95 ftp 的函数。不知何种原因，Windows 95 的 ftp 在传输结束以后加了四个空行，所以应该自动把它们删除。这里是 <netrw.vim> 自己的一些代码：

```
  if has("win95") && g:netrw_win95ftp
   fun! NetReadFixup(method, line1, line2)
     if method == 3    " ftp (无 <.netrc>)
      let fourblanklines= line2 - 3
      silent fourblanklines.",".line2."g/^\s*/d"
     endif
   endfunction
  endif
```

(相关主题： ftp netrw-userpass netrw-start)

---

9. 浏览 netrw-browsing netrw-browse netrw-help {{{1
netrw-browser netrw-dir netrw-list

浏 览 简 介 netrw-intro-browse {{{2
(快速参考： netrw-quickmaps netrw-quickcoms)

netrw 支持本地系统和远端机器上的目录浏览；浏览包括列出文件和目录、进入目录、编辑那里的文件、删除文件/目录、建立新目录和移动（换名）文件和目录，复制文件和目录，等等。还可以对文件进行标记，并对标记的文件执行任何系统命令！netrw 浏览器实现以前探索器里的映射和命令的更一般的版本以支持远程目录，不过具体细节（比如使用的全局变量名）肯定有所不同。要浏览一个目录，简单地 "编辑" 之便可！

```
vim /your/directory/
vim .
vim c:\your\directory\
```

(相关主题： netrw-cr netrw-o netrw-p netrw-P netrw-t
netrw-mf netrw-mx netrw-D netrw-R netrw-v)

netrw 远程文件和目录浏览器能处理两个协议：ssh 和 ftp。url 如果使用 ftp 协议，netrw 远程浏览时就同时使用 ftp。相反，如果指定其它协议，该协议将用于文件传输，而 ssh 协议会被用来进行远程浏览。

要使用 netrw 的远程目录浏览器，只要读入带拖尾斜杠的 "文件" 就可以，它被解释为

列出目录的请求：

```
vim [protocol]://[user@]hostname/path/
```

其中 [protocol] 一般是 scp 或 ftp。例如：

```
vim ftp://ftp.home.vim.org/pub/vim/
```

对于本地目录，拖尾的斜杠是不必要的。不过因为很容易忘记，这里不厌其烦再重复一遍：要浏览远程目录，URL 必须以斜杠结尾！

如果要避免 ssh 或 scp 在远程目录列表时反复输入密码的麻烦，见 netrw-ssh-hack 。要避免 ftp 密码的输入，见 netrw-netrc （如果你的 ftp 支持的话）。

改变浏览器显示文件的方式有以下几个途径：

* 要改变列表风格，按 "i" 键 (netrw-i)。
  目前支持四种风格：瘦、长、宽和树状。
  要使改变永久化，见 g:netrw_liststyle 。

* 要隐藏文件（不想再显示那些 xyz~ 文件？）见 netrw-ctrl-h 。

* 按 s 来对文件按名字、时间或大小排序。

要知道 netrw 可以做的一切事情，见 netrw-browse-cmds ！

netrw-getftype netrw-filigree netrw-ftype
在列出本地文件时， getftype() 函数被用来加上一个附，来表示文件类型：

```
directory  : /
executable : *
fifo       : |
links      : @
sockets    : =
```

附也影响 g:netrw_sort_sequence 。

快 速 帮 助 netrw-quickhelp {{{2

(按 ctrl-] 可选择主题)

浏览简介..... netrw-intro-browse
  快速参考：映射..... netrw-quickmap
  快速参考：命令..... netrw-browse-cmds
隐藏
  编辑隐藏列表..... netrw-ctrl-h
  隐藏文件或目录..... netrw-a
  按后缀隐藏或显示..... netrw-mh
  隐藏点文件..... netrw-gh
列表风格
  选择列表风格（瘦/长/宽/树状)..... netrw-i
  相关的设置变量..... g:netrw_liststyle
  执行列表的外壳命令..... g:netrw_list_cmd
  快速文件信息..... netrw-qf

如何排序
选择排序风格 (名字/时间/大小)..... netrw-s
编辑排序序列..... netrw-S
排序选项..... g:netrw_sort_options
相关的设置变量..... g:netrw_sort_sequence
反转排序顺序..... netrw-r

快 速 参 考 : 映 射

netrw-quickmap netrw-quickmaps netrw-browse-maps {{{2

映射	简要解释	链接
<F1>	netrw 给出帮助	
<cr>	netrw 进入目录或者打开文件	netrw-cr
<del>	netrw 试图删除文件/目录	netrw-del
<c-h>	编辑文件隐藏列表	netrw-ctrl-h
<c-l>	netrw 刷新目录列表	netrw-ctrl-l
<c-r>	使用 gvim 服务器浏览	netrw-ctrl-r
<c-tab>	收缩/打开 netrw/explore 窗口	netrw-c-tab
-	netrw 往上一层目录	netrw--
a	循环切换普通显示方式、隐藏方式 (不显示匹配 g:netrw_list_hide 的文件) 和显示方式 (只显示匹配 g:netrw_list_hide 的文件)	netrw-a
c	使浏览中的目录成为当前目录	netrw-c
C	设置编辑窗口	netrw-C
d	建立新目录	netrw-d
D	试图删除文件/目录	netrw-D
gb	切换到收入书签的目录	netrw-gb
gd	强制作目录处理	netrw-gd
gf	强制作文件处理	netrw-gf
gh	快速隐藏/显示点文件	netrw-gh
gn	置顶光标下的目录	netrw-gn
i	在瘦、长、宽和树状列表方式循环	netrw-i
I	切换横幅的显示	netrw-I
mb	把当前目录加入书签	netrw-mb
mc	把带标记文件复制到标记目标目录中	netrw-mc
md	对带标记文件进行比较 (不超过 3 个)	netrw-md
me	把带标记文件放到参数列表中并编辑之	netrw-me
mf	标记文件	netrw-mf
mF	撤销文件标记	netrw-mF
mg	给带标记文件应用 vimgrep	netrw-mg
mh	切换带标记文件的后缀在隐藏列表中的存在与否	netrw-mh
mm	把带标记文件移动到标记目标目录中	netrw-mm
mp	打印带标记文件	netrw-mp
mr	用外壳风格的 regexp 标记文件	netrw-mr
mt	使当前浏览目录成为标记文件的目标目录	netrw-mt
mT	对带标记文件应用 ctags	netrw-mT
mu	撤销所有带标记文件的标记	netrw-mu
mv	对带标记文件应用任意 vim 命令	netrw-mv
mx	对带标记文件应用任意 外壳 命令	netrw-mx
mx	对带标记文件整块应用任意外壳命令	netrw-mX
mz	对带标记文件压缩/解压缩	netrw-mz

o 用水平分割在新浏览窗口中进入光标所在的文件/目录 netrw-o
O 获取光标指定的文件 netrw-O
p 预览文件 netrw-p
P 在前次使用的窗口中浏览 netrw-P
qb 列出书签内的目录和历史 netrw-qb
qf 显示文件信息 netrw-qf
qF 用 quickfix 列表标记文件 netrw-qF
qL 用 location-list 标记文件 netrw-qL
r 反转排序顺序 netrw-r
R 给指定的文件或目录换名 netrw-R
s 选择排序风格：按名字、时间或文件大小 netrw-s
S 指定按名排序时的后缀优先级 netrw-S
t 在新标签页里进入光标所在的文件/目录 netrw-t
u 切换到较早访问的目录 netrw-u
U 切换到较迟访问的目录 netrw-U
v 用垂直分割在新浏览窗口中进入光标所在的文件/目录 netrw-v
x 用指定程序阅读文件 netrw-x
X 用 system() 执行光标所在的文件 netrw-X

% 在 netrw 当前目录打开新文件 netrw-%

netrw-mouse netrw-leftmouse netrw-middlemouse netrw-rightmouse

<leftmouse> (仅限于 gvim) 选择鼠标所在的单词，就像按了 <cr> 一样 (即编辑文件、改变目录)

<middlemouse> (仅限于 gvim) 等价于鼠标所在的单词上按 P；见 netrw-P

<rightmouse> (仅限于 gvim) 删除鼠标所在的文件/目录

<2-leftmouse> (仅限于 gvim) 当：
* 在 netrw 选择的文件中， 且
* g:netrw_retmap == 1 且
* 用户没有在 netrw 自动载入之前已经定义 <2-leftmouse> 映射，
那么双击鼠标左键返回到 netrw 浏览器窗口。见 g:netrw_retmap 。

<s-leftmouse> (仅限于 gvim) 等价于 mf，标记文件。拖动按住 shift 的鼠标左键可以标记多个文件。(见 netrw-mf)

(浏览时要禁用鼠标键： g:netrw_mousemaps)

netrw-quickcom netrw-quickcoms
快速参考：命令 netrw-explore-cmds netrw-browse-cmds {{{2

:NetrwClean[!] ..... netrw-clean
:NetrwSettings ..... netrw-settings
:Explore[!] [dir] 探索当前文件所在的目录 ..... netrw-explore
:Hexplore[!] [dir] 水平分割并探索 ..... netrw-explore
:Lexplore[!] [dir] 切换左浏览器 ..... netrw-explore
:Nexplore[!] [dir] 垂直分割并探索
(译者注：应为转到下一个匹配的文件) ..... netrw-explore
:Pexplore[!] [dir] 垂直分割并探索
(译者注：应为转到前一个匹配的文件) ..... netrw-explore
:Rexplore 返回浏览器 ..... netrw-explore
:Sexplore[!] [dir] 分割并探索目录 ..... netrw-explore
:Texplore[!] [dir] 建立新标签页并探索 ..... netrw-explore
:Vexplore[!] [dir] 垂直分割并探索 ..... netrw-explore

显 示 横 幅 netrw-I

按 "I" 可以切换横幅显示的打开与关闭。

另见: g:netrw_banner

把 目 录 加 入 书 签 netrw-mb netrw-bookmark netrw-bookmarks {{{2

用下面的命令可以很方便地把当前浏览目录加入 "书签"

mb

.netrwbook

书签可以跨 vim 会话使用，因为它们被作为 List 保存在叫 .netrwbook 的文件里，该文件通常存在用户的 ’ runtimepath ’ 中的第一个目录；所保存的项目已经排序。

如果有带标记的文件和/或目录，mb 会把它们加入书签列表。

netrw-:NetrwMB

另外，可用 :NetrwMB 来把文件或目录加入书签。

:NetrwMB[!] [files/directories]

没有 !: 把文件/目录加入 netrw 的书签系统

没有参数并且在 netrw 缓冲区里:
如果有带标记的文件 : 把带标记的文件加入书答
否则 : 把光标下的文件加入书答
没有参数并且不在 netrw 缓冲区里 : 把当前打开的文件加入书答
有参数 : 对每个参数用 glob() 展开并加入书签

带有 !: 从 netrw 的书签系统里删除文件/目录

:NetrwMB 命令可在 netrw 缓冲区之外使用 (一旦在会话中激活了 netrw)。

".netrwbook" 文件用来在 netrw (和 vim) 未激活时记录书签。缺省，它保存在用户 ’runtimepath’ 的第一个目录里。

相关主题:
netrw-gb 如果回到 (go) 书签
netrw-mB 如何删除书签
netrw-qb 如何列出书签
g:netrw_home 控制 .netrwbook 保存的位置

浏 览 netrw-enter netrw-cr {{{2

浏览很简单：移动光标到感兴趣的文件或目录上，按 <cr> (回车键) 就可以选择该文件或目录。如果还是目录，列出其内容，如果是文件，使用原来读请求中给出的协议来打开该文件。

警 告: 有四种列表的形式 (见 netrw-i)。netrw 假设两个或更多的空格定界长列表和宽列表格式下的文件和目录名。所以如果你的文件或目录名本身内嵌或拖尾两个或更多的空格，用 "瘦" 格式来选择文件。

g:netrw_browse_split 选项缺省为零。可以修改它，使文件的打开在新窗口或标签页里进行。如果该选项为一，分割会以水平方式进行，如果为二，分割以垂直方式进行。如果为三，<cr> 使得文件在新标签页里出现。

如果是用图形界面 (gvim)，可以按 <leftmouse> 选择文件。此外，如果

* g:netrw_retmap == 1 且 (其缺省值为 0)
* 在 netrw 选择的文件中， 且
* 用户没有在 netrw 载入之前已经定义 <2-leftmouse> 映射

那么双击鼠标左键返回到 netrw 浏览器窗口。

为了试图加快浏览速度，尤其是可能需要输入密码的远程浏览速度，netrw 会保留和重用上次得到的目录列表缓冲区。 g:netrw_fastbrowse 变量控制此行为；可以选择慢速浏览 (没有目录缓冲区列表重用)，中速浏览 (只对远程目录重用目录缓冲区列表) 和快速浏览 (尽可能重用目录缓冲区列表)。重用的代价是如果发生了改动 (例如在目录中出现了新文件)，列表就可能不是最新的。此时，按 ctrl-L 可以刷新目录列表缓冲区 (见 netrw-ctrl-l)。

netrw-s-cr

压缩当前树列表目录

如果打开树列表风格 (见 netrw-i) 并且使用 gvim，<s-cr> 映射可用来压缩 (关闭) 光标所在的目录。

否则，可以重新映射用户自己选择的键组合来达到同样的效果:

```
nmap <buffer> <silent> <nowait> YOURKEYCOMBO <Plug>NetrwTreeSqueeze
```

把此行放在$HOME/ftplugin/netrw/netrw.vim 里；必须只为 netrw 缓冲区而生成。

相关主题:

netrw-ctrl-r netrw-o netrw-p
netrw-P netrw-t netrw-v

相关设置变量:

g:netrw_browse_split g:netrw_fastbrowse
g:netrw_ftp_list_cmd g:netrw_ftp_sizelist_cmd
g:netrw_ftp_timelist_cmd g:netrw_ssh_browse_reject
g:netrw_ssh_cmd g:netrw_use_noswf

水 平 分 割 窗 口 方 式 浏 览 netrw-o netrw-horiz {{{2

通常用 <cr> 方式进入文件或目录。不过，"o" 映射可以打开新窗口来浏览新目录列表或文件。这里使用水平分割。(垂直分割可见 netrw-v)

通常，o 键水平分割窗口并使新窗口和光标出现在上方。

相关设置变量: g:netrw_alto g:netrw_winsize

相关主题:
netrw-ctrl-r netrw-o netrw-p
netrw-P netrw-t netrw-v
相关设置变量:
g:netrw_alto 控制上/下分割
g:netrw_winsize 控制初始大小

在 新 标 签 页 里 浏 览 netrw-t {{{2

通常你用 <cr> 进入文件或目录。"t" 映射允许你在新标签页里打开新的目录列表或文件。

如果你希望在背景标签页里显示新列表，用 gT 。

相关主题:
netrw-ctrl-r netrw-o netrw-p
netrw-P netrw-t netrw-v
相关设置变量:
g:netrw_winsize 控制初始大小

垂 直 分 割 窗 口 方 式 浏 览 netrw-v {{{2

通常用 <cr> 方式进入文件或目录。不过，"v" 映射可以打开新窗口来浏览新目录列表或文件。这里使用垂直分割。(水平分割可见 netrw-o)

通常，v 键垂直分割窗口并使新窗口和光标出现在左方。

只能有一个树状列表的缓冲区；因而，在子目录上按 "v" 会分割屏幕，但显示相同的缓冲区两遍。

相关主题:
netrw-ctrl-r netrw-o netrw-p
netrw-P netrw-t netrw-v
相关设置变量:
g:netrw_altv 控制右/左分割
g:netrw_winsize 控制初始大小

用 GVIM 服 务 器 浏 览 netrw-ctrl-r {{{2

可以建立浏览专用的 gvim，和用来编辑的 gvim 区隔。netrw 浏览器上对文件 (不是目录) 用 <c-r> 映射，会使用 gvim 服务器 (见 g:netrw_servername)。后续的 <cr> (见 netrw-cr) 会重用该服务器来编辑文件。

相关主题:
netrw-ctrl-r netrw-o netrw-p
netrw-P netrw-t netrw-v
相关设置变量:
g:netrw_servername : 设置服务器名
g:netrw_browse_split : 控制 <cr> 如何打开文件

改 变 列 表 方 式 (瘦 长 宽 树) netrw-i {{{2

"i" 映射在瘦、长、宽和树形列表格式之间循环。

瘦列表格式只给出文件和目录名。

对于远程目录，长格式基于通过 ssh 执行的 "ls" 命令结果，对于本地目录，则显示文件名、文件大小（字节计）和最近修改日期和时间。使用长格式时，netrw 不能识别带拖尾空格的文件名。这样的文件只能用瘦列表格式。

宽列表格式使用两个或多个连续的空格来给文件名定界；使用此格式时，netrw 不能识别内嵌或拖尾带有两个或多个连续的空格的文件名。不过，瘦格式可以处理这些文件。这种宽列表方式最为紧凑。

树形列表格式列出顶层目录，然后是文件和目录，它们的前面有一个或多个 "|"，指示目录深度。可以在目录名上按 <cr> 来打开或关闭目录。

可以设置缺省的列表方式：见 g:netrw_liststyle 。例如可在 .vimrc 中加入

```
let g:netrw_liststyle= 3
```

树形列表就成为缺省方式了。

一个使用 netrw 树形显示的典型方式是：

```
vim .
(反复按 i 直到树形显示出现)
找到一个文件
v （垂直分割窗口并编辑文件）
ctrl-w h （回到 netrw 列表）
P (在前次窗口编辑新选文件)
ctrl-w h （回到 netrw 列表）
P (在前次窗口编辑新选文件)
...等等...
```

相关设置变量： g:netrw_liststyle g:netrw_maxfilenamelen
g:netrw_timefmt g:netrw_list_cmd

改 变 文 件 权 限 netrw-gp {{{2

"gp" 会询问光标所在文件的新权限。目前，只适用于本地文件。

相关设置变量： g:netrw_chgperm

切 换 到 收 入 书 签 的 目 录 netrw-gb {{{2

要切换回收入书签的目录，用

```
{cnt}gb
```

可以使用计数来引用任何书签项目。
注意 netrw-qb 同时显示书签和历史；要转到历史的某个位置，见 netrw-u 和 netrw-U 。

相关主题：

netrw-mB 如何删除书签
netrw-mb 如何建立书签
netrw-qb 如何列出书签

切换到较早访问的目录 netrw-u netrw-updir {{{2

每次你切换到新目录时（当前会话里新出现），netrw 会把该目录保存在最近访问过的目录历史列表里（除非 g:netrw_dirhistmax 为零；缺省该值为 10 个项目）。用 "u" 映射可以切换到较早访问的目录（前任）。相反的操作见 netrw-U 。

"u" 映射也接受计数，回到若干个位置前的历史。为你方便起见，qb（见 netrw-qb ）列出历史编号，可用于此处的计数。

.netrwhist

参见 g:netrw_dirhistmax ，说明如何控制保存的历史总数。".netrwhist" 文件在 netrw（和 vim）未激活时记录历史。它缺省存放在用户 'runtimepath' 的第一个目录里。

相关主题:
netrw-U 切换到较迟访问的目录
g:netrw_home 控制 .netrwhist 何处存放

切换到较迟访问的目录 netrw-U netrw-downdir {{{2

用 "U" 映射可以切换到较迟访问的目录（后任）。和 "u" 映射相反。（见 netrw-u ）用 qb 映射可以同时列出书签和历史。（见 netrw-qb ）

"U" 映射也接受计数，回到若干个位置后的历史。

参见 g:netrw_dirhistmax ，说明如何控制保存的历史总数。

更改树置顶目录 netrw-ntree :Ntree netrw-gn {{{2

可以为树列表指定新的树置顶目录

```
:Ntree [dirname]
```

如果没有 "dirname"，使用当前行（跳过开头的深度信息）。
如果有 "dirname"，使用指定的目录名。

"gn" 映射获取光标下的单词并用之改变树列表的置顶。

NETRW 清理 netrw-clean :NetrwClean {{{2

使用 :NetrwClean 可以很容易地从用户主目录中删除 netrw；更准确的说，从 'runtimepath' 的第一个目录中删除之。

使用 :NetrwClean! 则会从 'runtimepath' 的所有目录中删除。当然，你必须要有正确的写入/删除权限才能这么做。

不管哪种形式，netrw 都会先确认你是否真的要这么做。如果 netrw 没有权限删除某文

件，它会报错。

通过特殊处理程序定制浏览 netrw-gx netrw-x netrw-handler {{{2
(另见 netrw_filehandler)

html、gif、jpeg、(word/office) 文档等若干类型的文件最好用特殊处理程序（也即，你机器的操作系统上的特定工具程序）来察看。netrw 允许你调用这些特殊的处理程序：

* 探索时，按 "x" 键
* 编辑时，在光标所在的特殊文件名上按 gx
  (g:netrw_nogx 变量存在时后者不可用)

netrw 用下面的方法决定使用哪个特殊处理程序：

* 如果存在 g:netrw_browsex_viewer ，尝试用它来察看文件。包含可用设置的示例（把它放在 <.vimrc> 里）：

  ```
  :let g:netrw_browsex_viewer= "kfmclient exec"
  ```
  或
  ```
  :let g:netrw_browsex_viewer= "xdg-open"
  ```

  如果 g:netrw_browsex_viewer == '-'，那么取而代之的是调用 netrwFileHandlers#Invoke() (见 netrw_filehandler)。

* 对 Windows 32 或 64 来说，使用 URL 和 FileProtocolHandler dll。
* 对 Gnome (带 gnome-open) 来说：使用 gnome-open。
* 对 KDE (带 kfmclient) 来说：使用 kfmclient。
* 对 Mac OS X 来说：使用 open。
* 否则，使用 netrwFileHandler 插件。

这些方法使用文件的后缀名来决定 "处理" 这些文件的合适的应用程序。这样就可以处理 OpenOffice (*.sfx)、可视文件 (*.jpg、*.gif 等) 和 PostScript (*.ps、 *.eps)。

gx 映射延伸到所有缓冲区；在单词上应用 "gx" 使用 netrw 在其上使用特殊处理程序（就像在 netrw 缓冲区里使用 "x" 一样)。也可用可视模式 (见 visual-start) 选择文本来应用特殊处理程序。通常，gx 使用 expand("<cfile>") 来获取光标下的文本；通过 g:netrw_gx 变量 (选项包括 "<cword>"、"<cWORD>") 可以改变 expand() 使用的参数。注意 expand("<cfile>") 取决于 'isfname' 设置。

此外，也可用可视选择 (见 visual-block) 然后按 gx 来选择 gx 使用的文本。

相关设置变量：

```
g:netrw_gx          控制 gx 如何获取光标下的文本
g:netrw_nogx        编辑时屏蔽 gx 映射
g:netrw_suppress_gx_mesg  控制 gx 对浏览器消息的抑制
```

netrw_filehandler

g:netrw_browsex_viewer 如果存在且为 "-"，netrw 试图通过 vim 函数来处理特殊文件。"x" 映射根据文件扩展名来对文件应用一个函数。当然能够调用的前提是该处理函数本身必须已经存在！

例如 mypgm.html x -> NFH_html("scp://user@host/some/path/mypgm.html")

用户可以编写他们自己的 netrw 文件处理函数来对更多后缀进行特殊处理。参见 <autoload/netrwFileHandlers.vim> 的例子来了解如何建立文件处理函数。例如:

```
" NFH_suffix(filename)
fun! NFH_suffix(filename)
..对文件名进行特殊处理..
endfun
```

这些函数需要在你的 .vim/plugin (vimfiles\plugin) 目录下的某个文件里定义。Vim 的函数名不能包含标点符号 (除了下划线以外)。要支持包含这类字符的后缀，netrw 会先按照下表对后缀进行转换:

```
@ -> AT        ! -> EXCLAMATION    % -> PERCENT
: -> COLON     = -> EQUAL          ? -> QUESTION
, -> COMMA     - -> MINUS          ; -> SEMICOLON
$ -> DOLLAR    + -> PLUS           ~ -> TILDE
```

因而，对于下例:

```
file.rcs,v  ->  NFH_rcsCOMMAv()
```

如果你需要更多的转换，请给我发电子邮件:
NdrOchip at ScampbellPfamily.AbizM - NOSPAM
提出要求。

相关设置变量: g:netrw_browsex_viewer

删 除 书 签 netrw-mB {{{2

要删除书签，用

```
{cnt}mB
```

如果有带标记的文件，mB 会从书签列表中删除之 。

此外，还可用 :NetrwMB! (见 netrw-:NetrwMB)。

```
:NetrwMB! [文件/目录]
```

相关主题:
netrw-gb 如何回到 (go) 书签
netrw-mb 如何建立书签
netrw-qb 如何列出书签

删 除 文 件 或 目 录 netrw-delete netrw-D netrw-del {{{2

如果没有用 netrw-mf 标记过文件: (局部标记文件列表)

文件和目录的删除涉及如下操作: 移动光标到要删除的文件/目录，然后按 "D"。目录必须为空才能成功删除。如果所谓的目录是指向目录的软链接。要成功删除该目录，netrw 需要发出两个请求。删除前 netrw 会给出确认。用 "V" 命令 (可视选

择）然后按 "D" 可以选择行范围。

如果用 netrw-mf 标记过文件：（局部标记文件列表）

带标记文件（含空目录）会被删除；还是一样，在实际操作之前，你会被询问以确认删除动作。

进一步，可以删除匹配模式的文件。

* 用 :MF 模式（见 netrw-:MF ）；然后按 "D"。

* 用 mr（见 netrw-mr ）提示你输入模式。使得匹配文件被标记。然后按 "D"。

如果你的 vim 版本有 7.4 patch#1107 补丁， g:netrw_localrmdir 不再用于删除目录；而使用 vim 带 "d" 选项的 delete() 代替。请注意 "D" 映射只能删除空目录。而 delete() 也能删除普通文件。

g:netrw_rm_cmd 、 g:netrw_rmf_cmd 和 g:netrw_rmdir_cmd 变量用以控制如何删除远程的文件和目录。g:netrw_rm_cmd 用于文件，缺省值为：

```
g:netrw_rm_cmd: ssh HOSTNAME rm
```

:netrw_rmdir_cmd 变量用于删除目录。缺省值为：

```
g:netrw_rmdir_cmd : ssh HOSTNAME rmdir
```

如果用 g:netrw_rmdir_cmd 删除目录失败，netrw 会再尝试一次用 g:netrw_rmf_cmd 变量进行删除。缺省值为：

```
g:netrw_rmf_cmd : ssh HOSTNAME rm -f
```

相关主题： netrw-d
相关设置变量： g:netrw_localrmdir g:netrw_rm_cmd
g:netrw_rmdir_cmd g:netrw_ssh_cmd

netrw-explore netrw-hexplore netrw-nexplore netrw-pexplore
netrw-rexplore netrw-sexplore netrw-texplore netrw-vexplore netrw-lexplore
目 录 探 索 命 令 {{{2

```
:[N]Explore[!]  [dir]... 探索当前文件所在的目录                  :Explore
:[N]Hexplore[!] [dir]... 水平分割  并探索                        :Hexplore
:[N]Lexplore[!] [dir]... 切换左浏览器                            :Lexplore
:[N]Sexplore[!] [dir]... 分割并探索当前文件所在的目录            :Sexplore
:[N]Vexplore[!] [dir]... 垂直分割  并探索                        :Vexplore
:Texplore       [dir]... 建立新标签页并探索                      :Texplore
:Rexplore            ... 返回到浏览器                            :Rexplore

用于 :Explore **/模式 :（另见 netrw-starstar ）
:Nexplore.............. 转到下一个匹配的文件                  :Nexplore
:Pexplore.............. 转到前一个匹配的文件                  :Pexplore
```

netrw-:Explore
:Explore 会打开当前文件目录上的本地目录浏览器（或者如果指定，[dir] 目录）。

只有该文件被修改且 'hidden' 未置位时才会分割窗口，否则浏览窗口使用该窗口。通常使用水平分割。

:Explore! 类似于 :Explore，但使用垂直分割。

netrw-:Hexplore

:Hexplore [dir] 执行 :Explore，用 :belowright 水平分割。
:Hexplore! [dir] 执行 :Explore，用 :aboveleft 水平分割。

netrw-:Lexplore

:[N]Lexplore [dir] 在当前标签页上左侧切换一个全高的浏览器窗口。如果 [dir] 省略，在当前目录上打开 netrw 窗口；不管从哪个窗口执行本命令，:Lexplore [dir] 会在左侧浏览器上显示指定目录。

缺省 :Lexplore 会将未初始化的 g:netrw_chgwin 值设为 2；从而，编辑会首选在 window#2 进行。

[N] 指定 g:netrw_winsize ，只适用于新建的 :Lexplore 窗口。

喜欢此方法的常常也喜欢树风格的显示；见 g:netrw_liststyle 。

:[N]Lexplore! [dir] 类似于 :Lexplore，但全高的探索器窗口开在右侧，未初始化的 g:netrw_chgwin 设为 1（例如，编辑会首选在最左侧进行）。

另见: netrw-C g:netrw_browse_split g:netrw_wiw
netrw-p netrw-P g:netrw_chgwin
netrw-c-tab g:netrw_winsize

netrw-:Sexplore

:Sexplore 调用本地目录浏览器前总分割窗口。和 Explore 相同，通常使用水平分割。
:Sexplore! [dir] 类似于 :Sexplore，但使用垂直分割。

netrw-:Texplore

:Texplore [dir] 在生成浏览窗口前先执行 :tabnew

netrw-:Vexplore

:Vexplore [dir] 执行 :Explore，用 :leftabove 垂直分割。
:Vexplore! [dir] 执行 :Explore，用 :rightbelow 垂直分割。

可选的参数是:

[N]: 此参数会代替 g:netrw_winsize 来指定新浏览器窗口应有的行数和列数。否则，如果用户指定 g:netrw_winsize 变量，用来设置新浏览器窗口应有的行数和列数。

[dir]: 缺省这些浏览器命令使用当前文件的目录。不过你可以显式提供使用的目录（路径）。

```
:Explore /some/path
```

netrw-:Rexplore

:Rexplore 此命令和其它的有所不同，它不一定会打开浏览器窗口。

返回浏览器

当用在 netrw 浏览器窗口的文件上按 <cr> 等方式编辑文件时，在编辑文件

时调用的 :Rexplore 会重新显示上次在此窗口上显示的 netrw 浏览器窗口。

从浏览器上返回
相反地，编辑目录时调用 :Rexplore 会返回到使用此窗口的上次的文件编辑状态。

<2-leftmouse> 映射（只有在 gvim 和有相应支持的终端才存在）的行为和 :Rexplore 相同。

另见： g:netrw_alto g:netrw_altv g:netrw_winsize

netrw-star netrw-starpat netrw-starstar netrw-starstarpat netrw-grep
使用星号和模式进行探索 {{{2

Explore、Sexplore、Hexplore 或 Vexplore 使用以下四种风格的路径时，Explore 生成本地文件系统生成满足要求的的文件列表。这些探索模式不适用于远程文件浏览。

```
*/filepat    当前目录满足文件模式 filepat 的文件
**/filepat   当前目录或之下满足文件模式的文件
*//pattern   当前目录内容包含模式 pattern 的文件（使用 vimgrep）
**//pattern  当前目录或之下内容包含模式 pattern 的文件（使用 vimgrep）
```

光标会被放在列表的第一个文件上。用 :Nexplore 可以访问该列表后面的文件，也可用 :Pexplore 访问列表前面的文件。Explore 会更新目录并把光标放在合适的位置。

简单的

```
:Explore
```

会清除探索列表。

如果控制台或 GUI 产生的 shift-up 或 shift-down 序列能够被识别，也许你会发现 shift-下箭头 和 shift-上箭头 更方便。netrw 提供它们的映射，如下：

```
<s-down>  == Nexplore，而
<s-up>    == Pexplore。
```

考虑下例

```
:Explore */*.c
:Nexplore
:Nexplore
:Pexplore
```

状态行的右侧会显示 "Match 3 of 20" 这样的消息。

相关设置变量：

```
g:netrw_keepdir            g:netrw_browse_split
g:netrw_fastbrowse         g:netrw_ftp_browse_reject
g:netrw_ftp_list_cmd       g:netrw_ftp_sizelist_cmd
g:netrw_ftp_timelist_cmd   g:netrw_list_cmd
g:netrw_liststyle
```

显示文件信息 netrw-qf {{{2

把光标放在文件名上，按 "qf" 会显示文件的大小和最近的修改时间。目前，此功能只适用于本地文件。

编辑文件或目录的隐藏列表 netrw-ctrl-h netrw-edithide {{{2

"<ctrl-h>" 映射显示请求以允许用户改变 g:netrw_list_hide 包含的文件/目录隐藏列表。隐藏列表包含一个或多个逗号分隔的模式列表。可以隐藏（也就是不显示）或只显示满足这些模式的文件和/或目录（见 netrw-a ）。

"gh" 映射（见 netrw-gh ）在普通的隐藏列表和隐藏 "." 开始的文件或目录间快速切换。

示例，

```
let g:netrw_list_hide= '\(^\|\s\s\)\zs\.\S\+'
```

效果上，它使得 netrw-gh 命令的结果成为初始的设置。具体来说：

```
\(^\|\s\s\)    : 如果下面的模式开始本行，-或-出现在两个连续的空白之后
\zs            : 开始隐藏匹配
\.             : 以句号开始
\S\+           : 后跟一个或多个非空白字符
```

相关设置变量： g:netrw_hide g:netrw_list_hide
相关主题： netrw-a netrw-gh netrw-mh

编辑排序序列 netrw-sort-sequence netrw-S netrw-sortsequence {{{2

如果 "排序风格" 为 name，用此排序序列可决定优先次序 (g:netrw_sort_sequence)。排序序列里，通常按后缀名安排名字列表的优先级，但其实任何模式都可以用。用逗号定界模式。缺省的排序序列是（在一行上）：

Unix 上：

```
'[\/]$,\<core\%(\.\d\+\)\=,\.[a-np-z]$,\.h$,\.c$,\.cpp$,*,\.o$,\.obj$,
\.info$,\.swp$,\.bak$,\~$'
```

其它：

```
'[\/]$,\.[a-np-z]$,\.h$,\.c$,\.cpp$,*,\.o$,\.obj$,\.info$,
\.swp$,\.bak$,\~$'
```

单独的 * 包含其它模式没有覆盖的所有文件名。用 g:netrw_sort_sequence 变量（手动或在 <.vimrc> 里设置）可以修改排序序列，"S" 映射也可以完成。

相关主题： netrw-s netrw-S
相关设置变量： g:netrw_sort_sequence g:netrw_sort_options

用 SYSTEM() 执行光标所在的文件 netrw-X {{{2

光标停留在可执行文件上时按 X 时，会用文件名给出提示，要求输入参数。按 [return] 后 netrw 会调用 system() 执行该命令和给定参数。结果用 :echomsg 显示，:messages 可以重新显示结果。Ansi 转义序列会被去掉。

见 cmdline-window 如何编辑参数的指示。

强制作为文件或目录对待 netrw-gd netrw-gf {{{2

远端符号链接（即通过 ssh 或 ftp 列出的）有一个问题，不容易知道它们实际链接到文件还是目录。

强制作为文件：用
        gf

强制作为目录：用
        gd

向上一层 netrw-- {{{2

要到上一层的目录，按 "-" 或在列表的 ../ 目录项上按 <cr>。

netrw 用 g:netrw_list_cmd 命令执行目录列表操作，其中 HOSTNAME 被用户提供的 url 的宿主名替换。netrw 提供的缺省命令是：

        ssh HOSTNAME ls -FLa

其中的 HOSTNAME 将根据读取操作的请求设为 [user@]hostname。很自然地，用户可以自行设置以覆盖此命令。NetList 函数实现远程浏览，它期待拖尾斜杠能标记目录名。

隐藏文件或目录 netrw-a netrw-hiding {{{2

netrw 的浏览机制提供三种使用隐藏列表的方法：忽略此列表、隐藏匹配的文件、还有只显示匹配的文件。

如果没有用 netrw-mf 标记过文件：

用户可以用 "a" 映射在这三种方式间切换。

g:netrw_list_hide 变量保存逗号分隔的正规表达式列表（例如 ^.*\.obj$,^\.），以指定隐藏列表。（另见 netrw-ctrl-h ）要设置隐藏列表，可以用 <c-h> 映射。例如，要隐藏 "." 开头的文件，可以用 <c-h> 映射来设置隐藏列表为 '^\..*'（你也可以在 <.vimrc> 里设置 g:netrw_list_hide= '^\..*'）。然后用 "a" 键就可以显示所有文件、隐藏匹配文件或只显示匹配文件。

        示例：\.[ch]$
                这个隐藏列表命令会隐藏/显示所有的 *.c 和 *.h 文件。

        示例：\.c$,\.h$
                这个隐藏列表命令也会隐藏/显示所有的 *.c 和 *.h 文件。

不要忘记用 "a" 映射来选择你想要的（普通/隐藏/显示）模式！

如果用 netrw-mf 标记过文件，那么此命令会：

如果显示所有文件或非隐藏文件：
 修改 g:netrw_list_hide 列表，加入带标记文件，并只显示非隐藏文件。

否则，如果只显示隐藏文件：
 修改 g:netrw_list_hide 列表，删除带标记文件，并只显示非隐藏文件。

netrw-gh netrw-hide

有一个快捷键，按

gh

可以切换是否隐藏句号（点）开始的文件。

相关设置变量： g:netrw_list_hide g:netrw_hide
相关主题： netrw-a netrw-ctrl-h netrw-mh

netrw-gitignore

Netrw 提供帮助函数 'netrw_gitignore#Hide()'，和 g:netrw_list_hide 一起使用时自动隐藏 git-ignore 的文件。

'netrw_gitignore#Hide' 搜索下面文件中的模式：

'./.gitignore'
'./.git/info/exclude'
全局 gitignore 文件：`git config --global core.excludesfile`
系统 gitignore 文件：`git config --system core.excludesfile`

忽略不存在的文件。
从存在的文件中提取 git-ignore 模式，转换为隐藏文件的模式。例如， '.gitignore' 文件中 '*.log' 转换为 '.*\.log'。

要使用此函数，简单地把其输出赋值给 g:netrw_list_hide 选项。

例如：let g:netrw_list_hide= netrw_gitignore#Hide()
  在 netrw 里隐藏 git-ignore 文件。

例如：let g:netrw_list_hide= netrw_gitignore#Hide('my_gitignore_file')
  函数可以接受使用 git-ignore 模式的额外文件。

例如：g:netrw_list_hide= netrw_gitignore#Hide() . '.*\.swp$'
  混合 'netrw_gitignore#Hide' 和定制的模式。

改 进 目 录 浏 览 netrw-listhack netrw-ssh-hack {{{2

不断要求输入密码很讨厌，远程目录的浏览尤其受这个问题影响。

对 Linux/Unix 系统而言，Rob Flickenger 所著的 "Linux Server Hacks - 100 industrial strength tips & tools" (O'Reilly, ISBN 0-596-00461-3) 一书给出如何设置无需密码的 ssh 和 scp 的使用方式的提示，也包括相关安全问题的讨论。该书曾经可从 http://hacks.oreilly.com/pub/h/66 得到，但现在该地址已经被重定向到某个 "hackzine" 网站。基于该文章和我与 Ben Schmidt 的通信，我试图给一个小结：

1. 在本地机器（ssh 客户端）上生成 公钥/私钥 对：
   ssh-keygen -t rsa
   (按照提示，把该文件存到 ~/.ssh/id_rsa)

2. 如果不要 passphrase，提示 passphrase 时只要按 <CR> 即可（两次）。如果需要 passphrase，你要使用 ssh-agent，从而每个会话期间只需输入一次 passphrase。如果不用 passphrase，只要简单地登录到你的本地机器上或者用任何方式得到密钥文件就足够可以访问任何使用该密钥提供授权登录的 ssh 服务器。

3. 这会建立两个文件：
```
~/.ssh/id_rsa
~/.ssh/id_rsa.pub
```

4. 在目标机器（ssh 服务器）上：
```
cd
mkdir -p .ssh
chmod 0700 .ssh
```

5. 在你的本地机器（ssh 客户端）上：（在一行上输入）
```
ssh {serverhostname}
  cat '>>' '~/.ssh/authorized_keys2' < ~/.ssh/id_rsa.pub
```

   或者，对于 OpenSSH 而言，（在一行上输入）
```
ssh {serverhostname}
  cat '>>' '~/.ssh/authorized_keys' < ~/.ssh/id_rsa.pub
```

你可以这样测试
```
ssh {serverhostname}
```
你应该可以登录到服务器机器而无须任何进一步的输入。

如果你决定使用 passphrase：
```
ssh-agent $SHELL
ssh-add
ssh {serverhostname}
```
ssh-add 时，你会被提示输入密钥 passphrase，但之后 ssh 就无须再输入了。要用于 vim，你可以用
```
ssh-agent vim
```
然后在 vim 中，用
```
:!ssh-add
```
另一个方案是在要执行 vim 的终端上应用 ssh-agent：
```
ssh-agent xterm &
```
然后需要的时候执行 ssh-add。

vim 邮件列表上也有人提到 Windows 上的 Pageant 有助于避免不断要求输入密码的麻烦。

Kingston Fung 写下另一个避免不断要求输入密码的方法：

> 要避免每次使用 scp 都输入密码的需要，你在文档里给出了一个设置无密码 ssh 帐号的技巧。我有个更好的方法：我可以使用普通有密码的 ssh 帐号来访问内容，而无须每次都输入密码。既安全又方便。我试了 ssh 公钥验证和 ssh-agent，这么做就可以用了！这里是具体操作的链接：
>
> http://www.ibm.com/developerworks/library/l-keyc2/
> http://sial.org/howto/openssh/publickey-auth/

Ssh 小提示:

Thomer Gil 提供的加速 netrw+ssh 的提示:
http://thomer.com/howtos/netrw_ssh.html

Alex Young 关于加速ssh 的若干提示:
http://usevim.com/2012/03/16/editing-remote-files/

列出书签和历史 netrw-qb netrw-listbookmark {{{2

按 "qb" (query bookmarks，查询书签) 会列出书签里记录的目录和目录遍历的历史。

相关主题:
netrw-gb 如何回到 (go) 书签
netrw-mb 如何建立书签
netrw-mB 如何删除书签
netrw-u 如何切换到历史堆栈里先前的目录
netrw-U 如何切换到历史堆栈里后来的目录

建立新目录 netrw-d {{{2

用 "d" 映射可以在远端 (取决于全局变量 g:netrw_mkdir_cmd) 或本地 (取决于全局变量 g:netrw_localmkdir) 建立目录。netrw 会要求新目录的名字。这时单输入 <CR> 可以中止目录建立过程。试图建立已经存在的本地目录 (可以是文件或目录) 会被检测到并进行报告，然后忽略。

相关主题: netrw-D
相关设置变量: g:netrw_localmkdir g:netrw_mkdir_cmd
g:netrw_remote_mkdir netrw-%

使浏览目录成为当前目录 netrw-cd netrw-curdir {{{2

g:netrw_keepdir 缺省为 1。该设置意味着当前目录不会跟踪浏览目录。(这么做是为于和 v6 版本的文件探索器后向兼容)

设置 g:netrw_keepdir 为 0 告诉 netrw 使 vim 的当前目录跟踪 netrw 的浏览目录。

不过，保持缺省为 1 的 g:netrw_keepdir 设置意味着 netrw 维护它自己的当前目录的概念。要想使两个目录相同，可用 "cd" 映射 (按 cd)。该映射会把 Vim 概念的当前目录设为当前的浏览目录。

netrw-c : 本映射名从 "c" 改成了 cd (见 netrw-cd)。
这个改动使 netrw-cb 和 netrw-cB 映射成为可能。

相关设置变量: g:netrw_keepdir

标记文件 netrw-:MF netrw-mf {{{2
(另见 netrw-mr)

netrw 提供了几种标记文件的方式:

* 把光标移到文件名上按 "mf" 就可以标记文件。

* gvim 中，更可以用 <s-leftmouse> 来标记文件。(见 netrw-mouse)

* :MF 命令接受文件列表 (针对本地目录，列表可能包含通配符 -- 见 glob())

```
:MF *.c
```

(备注 :MF 用 <f-args> 以便在空白内断开行)

* 用 argument-list 标记文件 (netrw-mA)

* 基于 location-list 标记文件 (netrw-qL)

* 基于 quickfix (netrw-qF) (quickfix-error-lists)

下面的 netrw 映射使用带标记文件:

netrw-a	隐藏标记文件/目录
netrw-D	删除标记文件/目录
netrw-ma	移动标记文件名到 arglist
netrw-mA	移动 arglist 文件名到标记文件列表
netrw-mb	附加标记文件到书签
netrw-mB	从书签中删除标记文件
netrw-mc	复制标记文件到目标
netrw-md	给标记文件应用 vimdiff
netrw-me	编辑标记文件
netrw-mF	撤销文件标记
netrw-mg	在标记文件上应用 vimgrep
netrw-mm	移动标记文件到目标
netrw-mp	打印标记文件
netrw-ms	Netrw 会执行标记文件
netrw-mt	为 netrw-mm 和 netrw-mc 设定目标
netrw-mT	用标记文件生成标签
netrw-mv	在标记文件上应用 vim 命令
netrw-mx	在标记文件上应用外壳命令
netrw-mX	在标记文件上整块应用外壳命令
netrw-mz	压缩/解压标记文件
netrw-O	获取标记文件
netrw-R	为标记文件换名

用标记文件相同的方式可以撤销文件的标记，也就是，把光标移到带标记的文件上按 "mf"。这也适用于 gvim 的 <s-leftmouse>。按 "mu" 可以一次撤销所有标记 (见 netrw-mu)。

标记的文件使用 "netrwMarkFile" 高亮组高亮，该组缺省链接到 "Identifier" (见 group-name 下的 Identifier)。可以把以下代码

```
highlight clear netrwMarkFile
hi link netrwMarkFile ..随便什么..
```

加入 $HOME/.vim/after/syntax/netrw.vim 来改变高亮组设置。

如果鼠标功能打开且在 Vim 里可用，可用 <s-leftmouse> 来标记一个或多个文件。多个文件的标记可通过按住 shift 拖动鼠标左键实现。(见 netrw-mouse)

markfilelist global_markfilelist local_markfilelist
所有的标记文件都进入全局标记文件列表；只有唯一的这样的列表。此外，每个 netrw 缓冲区都有自己缓冲区局部的标记文件列表；因为 netrw 缓冲区和特定的目录相关联，这意味着每个目录都有自己的局部标记文件列表。操作标记文件的命令分别使用两个标记文件列表中的一个。

已知问题：如果使用树模式 (g:netrw_liststyle) 而多个目录上有相同的文件名，标记其中一个文件会高亮所有这样的文件，就像它们都被标记了一样。不过，markfilelist 只包含你实际选择的文件。这个问题不太可能得到修正。

撤销文件标记 netrw-mF {{{2
(另见 netrw-mf 、 netrw-mu)

"mF" 命令撤销当前缓冲区所有文件的标记。也可用 mf (netrw-mf) 来撤销特定的已标记文件的标记。

用位置列表标记文件 netrw-qL {{{2
(另见 netrw-mf)

"qL" 可以把 location-list 转换为标记文件的列表。然后用 me (netrw-me) 这样的命令就可以编辑它们。

用 QUICKFIX 列表标记文件 netrw-qF {{{2
(另见 netrw-mf)

"qF" 可以把 quickfix-error-lists 转换为标记文件的列表。然后用 me (netrw-me) 这样的命令就可以编辑它们。例如，可以调用 :vimgrep 来生成 Quickfix 错误列表。

用正规表达式标记文件 netrw-mr {{{2
(另见 netrw-mf)

也可以用 "mr" 来标记文件；netrw 会给出提示，"Enter regexp: "。此时，你可以输入一个外壳风格的正规表达式，比如 *.c$ (见 glob())。glob() 不能用于远程文件，所以 netrw 把 "*" 转换成 ".*" (见 regexp) 并依此匹配文件。将来，我可能会考虑提供选项来使用 regexp 而不是 glob()-风格的表达式 (另一个选项)。

见 cmdline-window 关于更多如何编辑正规表达式的指示。

带标记文件，任意 VIM 命令 netrw-mv {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

"mv" 映射会使 netrw 在本地标记文件列表的每个文件上分别执行任意的 vim 命令：

```
* 1split
* sil! keepalt e file
* 执行 vim 命令
* sil! keepalt wq!
```

给出提示 "Enter vim command: " 让用户给出要使用的 vim 命令。
见 cmdline-window 关于更多如何编辑命令的指示。

带 标 记 文 件，任 意 外 壳 命 令 netrw-mx {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

激活 "mx" 映射后，netrw 会询问用户应用于所有带标记文件的（外部）命令。命令中所有的 "%" 被带标记文件的每个名字依次替代。如果命令中没有 "%"，那么命令后附加一个空格和一个带标记文件名。

示例：

```
(标记文件)
mx
Enter command: cat

结果是一系列外壳命令:
cat 'file1'
cat 'file2'
...
```

带 标 记 文 件，任 意 外 壳 命 令，整 块 netrw-mX {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

激活 'mX' 映射后，netrw 会请用户给出某个（外部）命令，应用在全局标记文件列表上的所有带标记文件。"整块" 的意思是一个命令一次执行在所有文件上：

```
command files
```

此方法可用于，例如，选择所有文件打个 tarball 包：

```
(标记文件)
mX
Enter command: tar cf mynewtarball.tar
```

此例中会执行的命令是：

```
tar cf mynewtarball.tar 'file1' 'file2' ...
```

带 标 记 文 件 : 参 数 列 表 netrw-ma netrw-mA
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

ma 可以从标记文件列表移动文件名到参数列表。
mA 可以从参数列表移动文件名到标记文件列表。

另见: netrw-cb netrw-cB netrw-qF argument-list :args

带 标 记 文 件 : 缓 冲 区 列 表 netrw-cb netrw-cB
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

cb 可以从标记文件列表移动文件名到缓冲区列表。
cB 可以从缓冲区列表移动文件名到标记文件列表。

另见: netrw-ma netrw-mA netrw-qF buffer-list :buffers

带 标 记 文 件 : 压 缩 与 解 压 缩 netrw-mz {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

对任何压缩过的带标记文件，"mz" 会对它们解压。
对任何被解压的带标记文件，"mz" 会用 g:netrw_compress 指定的命令对它们进行压缩。缺省，使用 "gzip"。

对于解压，netrw 使用一个后缀到相应解压工具的 Dictionary ；见 g:netrw_decompress 。

记住可以用正规表达式来标记多个文件（见 netrw-mr)；这对压缩与解压缩大量文件特别有用。

相关设置变量: g:netrw_compress g:netrw_decompress

带 标 记 文 件 : 复 制 netrw-mc {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

用 mt 选择目标目录 (netrw-mt)。然后改变目录，选择文件（见 netrw-mf)，然后按 "mc"。从当前窗口（执行 mf 的地方）复制到目标。

如果没有用 netrw-mt 设置目标目录，netrw 会让你输入要复制到的目录。

也可以复制目录及其内容（只限本地）来目标目录。

相关设置变量:
g:netrw_localcopycmd g:netrw_localcopycmdopt
g:netrw_localcopydircmd g:netrw_localcopydircmdopt
g:netrw_ssh_cmd

带 标 记 文 件 : 比 较 netrw-md {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

用 vimdiff 来显示选择文件的差异（为此，可以选择两到三个文件)。

带 标 记 文 件 : 编 辑 netrw-me {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)

(使用全局标记文件列表)

"me" 命令把带标记文件放在 arglist 上，然后开始编辑它们。用 :Rexplore 可以回到浏览器窗口。
(:n 和 :p 可以编辑 arglist 的下一个和上一个文件)

带 标 记 文 件: GREP netrw-mg {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

"mg" 命令在带标记文件上应用 :vimgrep 。
该命令会询问需要的模式；你可以输入:

```
/pattern/[g][j]
! /pattern/[g][j]
pattern
```

用 /pattern/，从 vimgrep 产生的 quickfix 列表 (见 :copen 、 :cnext 、 :cprevious 、 :cclose) 的首个项目开始编辑。使用的是 :vimgrep ，因而不带 'g' 时每行只加到 quickfix 列表上一次，带 'g' 时加入每个匹配。

用 /pattern/j，"mg" 可以缩减当前的标记文件列表，只包含给定模式的那些标记文件。因此可以用

```
mr ...文件模式...
mg /内容模式/j
```

来得到同时满足文件名匹配文件模式且内容限定在指定内容模式的标记文件列表。

带 标 记 文 件 : 按 后 缀 隐 藏 或 显 示 netrw-mh {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

"mh" 命令提取带标记文件的后缀，然后在隐藏列表中切换它们的存在与否。请 注意 据此，带有相同后缀的文件多次出现会使得每个文件都切换一次此后缀的存在与否 (所以偶数个相同后缀的带标记文件和不选任何文件的作用是一样的)。

相关主题: netrw-a g:netrw_list_hide

带 标 记 文 件 : 移 动 netrw-mm {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

> 警 告: 移动文件比复制文件要危险。被移动的文件先被复制然后被删除；如果复制操作失败但删除操作成功，该文件就丢失了。你要么用不太重要的文件先试试，要么用 mc 和 D 先进行复制然后再删除。请自行负责!

用 mt 选择目标目录 (netrw-mt)。然后改变目录，选择文件 (见 netrw-mf)，然后按 "mm"。从当前窗口 (执行 mf 的地方) 移动到目标。

相关设置变量: g:netrw_localmovecmd g:netrw_ssh_cmd

带 标 记 文 件 : 打 印 netrw-mp {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

"mp" 命令使 netrw 对带标记文件应用 :hardcopy 命令。具体地，它在一个单行的窗口上打开每个文件，执行 hardcopy，然后关闭这个单行窗口。

带 标 记 文 件 : 执 行 netrw-ms {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用局部标记文件列表)

"ms" 命令使 netrw 执行带标记文件 (使用 vim 的 :source 命令)

带 标 记 文 件 : 设 置 目 标 目 录 netrw-mt {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)

设置带标记文件复制/移动的目标 (见 netrw-mc 和 netrw-mm):

* 如果光标在文件名上，使用 netrw 窗口当前显示的目录作为复制/移动目标。
* 同样，如果光标在横幅处，使用 netrw 窗口当前显示的目录作为复制/移动目标。
  除非目标已经是当前目录，此时，键入 "mf" 撤销该目标。
* 但如果光标在一个目录名上，使用该目录作为复制/移动目标。
* 可用 :MT [directory] 命令来设置目标 netrw-:MT
  此命令使用 <q-args> ，所以接受里带空格的目录名，不需转义。
* 在打开鼠标的 vim 或 gvim 里，可用 <c-leftmouse> 选择目标

每个 vim 会话只有一个复制/移动目标；也就是说，该目标是一个脚本变量 (见 s:var)，并且由所有 (同一 vim 实例中的) netrw 窗口共享。

使用菜单和 gvim 时，netrw 提供 "Targets" 菜单项，可用来从书签和历史的选项中选取目标。

相关主题:
标记文件..... netrw-mf
用正规表达式标记文件..... netrw-mr
带标记文件: 用书签作为目标目录..... netrw-Tb
带标记文件: 用历史作为目标目录..... netrw-Th

带 标 记 文 件 : 建 立 标 签 netrw-mT {{{2
(关于如何标记文件，见 netrw-mf 和 netrw-mr)
(使用全局标记文件列表)

"mT" 映射为带标记文件应用 g:netrw_ctags 定义的命令 (缺省是 "ctags")。对于远程浏览，为了建立一个标签文件，netrw 会使用 ssh (见 g:netrw_ssh_cmd)，所以要能在远程系统使用此命令，ssh 必须存在。对于本地系统，见 ctags 说明如何得到可用的版本。我自己用 hdrtags，目前可从
http://www.drchip.org/astronaut/src/index.html 获得，并在我的 <.vimrc> 中有

```
let g:netrw_ctags= "hdrtag"
```

如果为一组远程的文件建立了标签，产生的 tags 文件被 "获取"；也即，一个备份被传输到本地系统的目录。然后这个本地的 tags 文件被修改，使得它可以在网上使用。具体地，tags 里的文件名被修改，每个路径前面加上了用于获取的 netrw 兼容的 URL。如果用户随后用某个转到标签的动作 (tags)，netrw 就可以用该 URL 来编辑想要的文件，并转到对应的标签上。

相关设置变量： g:netrw_ctags g:netrw_ssh_cmd

带标记文件：用书签作为目标目录 netrw-Tb {{{2

设置带标记文件复制/移动的目标。

netrw-qb 映射给出书签（和历史）的列表。可以用 [count]Tb（缺省计数：1）从中选取一个书签作为标记文件的目标。

相关主题：
复制文件到目标..... netrw-mc
列出书签和历史..... netrw-qb
带标记文件：设置目标目录..... netrw-mt
带标记文件：用历史作为目标目录..... netrw-Th
标记文件..... netrw-mf
用正规表达式标记文件..... netrw-mr
移动文件到目标..... netrw-mm

带标记文件：用历史作为目标目录 netrw-Th {{{2

设置带标记文件复制/移动的目标。

netrw-qb 映射给出历史（和书签）的列表。可以用 [count]Th（缺省计数：0，即当前目录）从中选取一个历史项作为标记文件的目标。

相关主题：
复制文件到目标..... netrw-mc
列出书签和历史..... netrw-qb
带标记文件：设置目标目录..... netrw-mt
带标记文件：用书签作为目标目录..... netrw-Tb
标记文件..... netrw-mf
用正规表达式标记文件..... netrw-mr
移动文件到目标..... netrw-mm

带标记文件：撤销标记 netrw-mu {{{2
（见 netrw-mf 、 netrw-mF ）

"mu" 映射撤销当前所有带标记文件的标记。此命令和 "mF" 不同之处在于后者只撤销当前目录下的文件，而 "mu" 撤销全局和所有缓冲区局部的标记文件的标记。
（见 netrw-mF ）

NETRW 浏览器变量 netrw-browser-settings netrw-browser-options netrw-browser-var {{{2

(如果你对 netrw 文件传输设置感兴趣，见 netrw-options 和 netrw-protocol)

<netrw.vim> 浏览器提供你可以修改的以变量形式出现的设置。把这些设置放在 <.vimrc> 里，你就可以定制浏览的偏好。(另见： netrw-settings)

变量	解释
g:netrw_altfile	有些人喜欢 CTRL-^ 返回最返的编辑文件。要选择此行为，设置此参数为 1。 其他的人喜欢 CTRL-^ 返回 netrw 浏览缓冲区。要选择此行为，设置此参数为 0。 缺省: =0
g:netrw_alto	置位此变量后，分割后的新窗口出现在下方而不是上方 (见 netrw-o) 缺省: =&sb (见 'sb')
g:netrw_altv	置位此变量后，分割后的新窗口出现在右方而不是左方 (见 netrw-v) 缺省: =&spr (见 'spr')
g:netrw_banner	显示/关闭横幅 =0: 关闭横幅 =1: 显示横幅1 (缺省) 注意: 关闭横幅是新功能，可能会引起问题。
g:netrw_bannerbackslash	如果此变量存在且非零，横幅显示时使用反斜杠而不是斜杠。
g:netrw_browse_split	浏览时，<cr> 这样打开文件: =0: 重用同一个窗口 (缺省) =1: 先水平分割窗口 =2: 先垂直分割窗口 =3: 在新标签页中打开文件 =4: 等同于 "P" (即打开前次窗口) 注意 可用 g:netrw_preview 来进行垂直分割而不是水平分割。 =[servername,标签页号,窗口号] 给出 List 就像这样，用名为 "servername" 的远程服务器进行编辑。使用给定的标签页号和窗口号来执行编辑 (见 clientserver 、 netrw-ctrl-r) 此选项不影响 :Lexplore 窗口的生成。 相关主题: g:netrw_alto g:netrw_altv netrw-C netrw-cr netrw-ctrl-r
g:netrw_browsex_viewer	指定查阅器的用户偏好: "kfmclient exec" "gnome-open"

如果使用了
"_"
那么 netrwFileHandler() 将寻找处理该扩展名的脚本/函数（见 netrw_filehandler ）。

g:netrw_chgperm

Unix/Linux: "chmod PERM FILENAME"
Windows: "cacls FILENAME /e /p PERM"
用以改变文件的访问权限。

g:netrw_compress

="gzip"
用此命令压缩带标记文件

g:Netrw_corehandler

可用来指定 netrw 浏览器的 "x" 命令处理 <core> 文件的额外处理。如果定义，g:Netrw_corehandler 指定一个或多个函数引用（见 Funcref ）。
(g:Netrw... 中的大写是必需的，因为它保存函函数引用)

g:netrw_ctags

="ctags"
建立标签使用的缺省外部程序

g:netrw_cursor

= 2 (缺省)
本选项控制 netrw 使用的 'cursorline' (cul) 和 'cursorcolumn' (cuc) 设置:

值	瘦-长-树	宽
=0	u-cul u-cuc	u-cul u-cuc
=1	u-cul u-cuc	cul u-cuc
=2	cul u-cuc	cul u-cuc
=3	cul u-cuc	cul cuc
=4	cul cuc	cul cuc

其中
u-cul : 用户使用的 'cursorline'
u-cuc : 用户使用的 'cursorcolumn'
cul : 局部设置的 'cursorline'
cuc : 局部设置的 'cursorcolumn'

g:netrw_decompress

= { ".gz" : "gunzip" ,
".bz2" : "bunzip2" ,
".zip" : "unzip" ,
".tar" : "tar -xf"}
映射后缀到解压程序的字典。

g:netrw_dirhistmax

=10: 控制过去历史最大数量。零关闭之。
（相关: netrw-qb netrw-u netrw-U ）

g:netrw_dynamic_maxfilenamelen

=32: 支持动态决定 g:netrw_maxfilenamelen 的值，它影响本地文件的长列表。

g:netrw_errorlvl

=0: 最低显示的错误级别

0: 备注
1: 警告
2: 错误

g:netrw_fastbrowse

=0: 慢速浏览;
不会重用目录列表;
总是重新获取目录列表。
=1: 中速浏览;
只有远程浏览才重用目录列表。
(缺省值)
=2: 快速目录浏览;
只获取没有浏览过的目录列表
(或在用 netrw-ctrl-l 时)。
快速浏览意味着保留旧目录列表的缓冲区，这样就不需要重新获取了。该特性对远程浏览特别有意义。不过，如果加入或者删除文件，旧的目录列表就不再是最新的。此时可用 netrw-ctrl-l 刷新目录列表。本选项提供用户机会来选择准确性（即最新的列表）和速度的权衡。

g:netrw_ffkeep

(缺省：不存在)
如果此变量存在且为零，netrw 不会保存和恢复 'fileformat' 。

g:netrw_fname_escape

=' ?&;%'
远程读取/写入前用于文件名。

g:netrw_ftp_browse_reject

ftp 可能会生成一些错误和警告信息，它们会作为 "目录" 和 "文件" 出现在列表里。使用本模式来删除这些内嵌的消息。缺省值为:

```
'^total\s\+\d\+$\|
^Trying\s\+\d\+.*$\|
^KERBEROS_V\d rejected\|
^Security extensions not\|
No such file\|
: connect to address [0-9a-fA-F:]*
: No route to host$'
```

g:netrw_ftp_list_cmd

传递给 ftp 用于列出目录的选项。缺省为:

```
unix 或置位 g:netrw_cygwin 时: "ls -lF"
否则                            "dir"
```

g:netrw_ftp_sizelist_cmd

传递给 ftp 用于列出目录并按文件大小排序的选项。
缺省:

```
unix 或置位 g:netrw_cygwin 时: "ls -slF"
否则                            "dir"
```

g:netrw_ftp_timelist_cmd

传递给 ftp 用于列出目录并按文件最后修改时间排序的选项。
缺省:

```
unix 或置位 g:netrw_cygwin 时: "ls -tlF"
否则                            "dir"
```

g:netrw_glob_escape
=’[]*?‘{~$’ (unix)
=’[]*?‘{$’ (windows
应用 glob() 前，目录名中的这些字符被转义

g:netrw_gx
="<cfile>"
此选项控制 gx (netrw-gx) 如果选择光标下的文本。可用值见 expand() 。

g:netrw_hide
受 "a" 映射控制（见 netrw-a ）
=0 : 显示全部
=1 : 显示非隐藏文件
=2 : 只显示隐藏文件
缺省: =1

g:netrw_home
书签和历史保存的主目录（分别存为 .netrwbook 和 .netrwhist)。
Netrw 在该字符串上使用 expand() 。
缺省: ’runtimepath’ 的第一个目录

g:netrw_keepdir
=1 (缺省) 浏览目录时不影响当前目录。
=0 浏览目录时设置当前目录为该目录。
当前的浏览目录保存在 b:netrw_curdir 里（另见 netrw-c ）

g:netrw_keepj
="keepj" (缺省) netrw 试图使 :jumps 表不受影响。
="" netrw 不用 :keepjumps ，但位置的保存/恢复是例外。

g:netrw_list_cmd
列出远程目录的命令
缺省: (如果 ssh 可执行)
"ssh HOSTNAME ls -FLa"

g:netrw_list_cmd_options
如果此变量存在，其内容被附加到 g:netrw_list_cmd 里。例如，
"2>/dev/null" 在 unix 系统上可用来去掉横幅。

g:netrw_liststyle
设置缺省的列表风格:
= 0: 瘦列表（每个文件一行）
= 1: 长列表（每个文件一行，带有时间信息和文件长度）
= 2: 宽列表（多个文件，分列显示）
= 3: 树型风格列表

g:netrw_list_hide
逗号分隔的隐藏文件的模式列表
模式是正规表达式（见 regexp ）
对 git-ignore 文件有特殊支持: 可加入帮助函数 ’netrw_gitignore#Hide()’ 的输出来自动隐藏所有的 git-ignore 文件。
更多详情可见 netrw-gitignore 。

示例:
let g:netrw_list_hide= ’.*\.swp$’

let g:netrw_list_hide= netrw_gitignore#Hide().'.*\.swp$'
缺省: ""

g:netrw_localcopycmd
="cp" Linux/Unix/MacOS/Cygwin
=expand("$COMSPEC") Windows
把带标记文件 (netrw-mf) 复制到目标目录
(netrw-mt 、 netrw-mc)

g:netrw_localcopycmdopt
='' Linux/Unix/MacOS/Cygwin
=' \c copy' Windows
g:netrw_localcopycmd 的选项

g:netrw_localcopydircmd
="cp" Linux/Unix/MacOS/Cygwin
=expand("$COMSPEC") Windows
复制目录到目标目录。
(netrw-mc 、 netrw-mt)

g:netrw_localcopydircmdopt
=" -R" Linux/Unix/MacOS/Cygwin
=" /c xcopy /e /c /h/ /i /k" Windows
g:netrw_localcopydircmd 的选项

g:netrw_localmkdir
="mkdir" Linux/Unix/MacOS/Cygwin
=expand("$COMSPEC") Windows
建立本地目录的命令

g:netrw_localmkdiropt
="" Linux/Unix/MacOS/Cygwin
=" /c mkdir" Windows
g:netrw_localmkdir 的选项

g:netrw_localmovecmd
="mv" Linux/Unix/MacOS/Cygwin
=expand("$COMSPEC") Windows
把带标记文件 (netrw-mf) 移动到目标目录
(netrw-mt 、 netrw-mm)

g:netrw_localmovecmdopt
="" Linux/Unix/MacOS/Cygwin
=" /c move" Windows
g:netrw_localmovecmd 的选项

g:netrw_localrmdir
="rmdir" Linux/Unix/MacOS/Cygwin
=expand("$COMSPEC") Windows
删除目录的命令 (rmdir)
此选项只用于 7.4 之前或不带 patch#1107 补丁的 vim 版本。否则，使用带 "d" 选项的 delete() 。

g:netrw_localrmdiropt
="" Linux/Unix/MacOS/Cygwin
=" /c rmdir" Windows
g:netrw_localrmdir 的选项

g:netrw_maxfilenamelen
=32 缺省之所以选为该值，使为了使长列表能在 80 列内显示。如果你的屏幕更宽或者文件或目录名可能长于 32 个字节，可以设置此选项使列表按列排列

g:netrw_mkdir_cmd
用 ssh 建立远程目录的命令 (另见

g:netrw_remote_mkdir)
缺省: "ssh USEPORT HOSTNAME mkdir"

g:netrw_mousemaps

=1 (缺省) 打开鼠标按钮的功能
浏览时:
leftmouse : 打开文件/目录
shift-leftmouse : 标记文件
middlemouse : 同 P
rightmouse : 删除文件/目录
=0: 关闭鼠标映射

g:netrw_nobeval

不存在 (缺省)
如果此变量存在, 不进行气泡计算
(见 'balooneval')

g:netrw_sizestyle

未定义: 实际字节 (缺省)
="b" : 实际字节 (缺省)
="h" : 人类可读 (例如 5k、4m、3g)
以 1000 为底
="H" : 人类可读 (例如 5K、4M、3G)
以 1024 为底
长列表 (netrw-i) 和查询文件的映射
(netrw-qf) 会使用指定风格显示文件大小。

g:netrw_usetab

如果此变量存在且非零, <tab> 映射支持 Lexplore 的缩/放或打开 netrw 窗口。
(见 netrw-c-tab)

g:netrw_remote_mkdir

用 ftp 建立本地目录的命令 (另见
g:netrw_mkdir_cmd)
缺省: "mkdir"

g:netrw_retmap

如果存在且为一, 那么:
* 如果在 netrw 选择的文件中, 且
* 没有定义过普通模式的 <2-leftmouse> 映射,
那么 <2-leftmouse> 会被映射以方便地回到 netrw 浏览器窗口。
示例: 点击一次, 选择并打开文件, 双击返回

备注: 你也可以选择这么做:
* let g:netrw_retmap= 1 (译者注: 0?), 且
* nmap <silent> 你的选择 <Plug>NetrwReturn
来让其它的映射, 而不是 <2-leftmouse> 执行返回的任务。

也可以用 :Rexplore 命令完成同样的功能。

缺省: =0

g:netrw_rm_cmd

删除远程文件的命令
缺省: "ssh USEPORT HOSTNAME rm"

g:netrw_rmdir_cmd

删除远程目录的命令
缺省: "ssh USEPORT HOSTNAME rmdir"

g:netrw_rmf_cmd

删除远程软链接的命令
缺省: "ssh USEPORT HOSTNAME rm -f"

g:netrw_servername

此变量提供 netrw-ctrl-r 所用的服务器名。
缺省: "NETRWSERVER"

g:netrw_sort_by

按 "name" (名字)、"time" (时间)、"size" (大小) 或 "exten" (扩展名) 排序。
缺省: "name"

g:netrw_sort_direction

排序方向: "normal" (普通) 或 "reverse" (反转) 缺省: "normal"

g:netrw_sort_options

排序是通过 :sort 命令进行的; 此变量的值会附加到排序命令之后。例如, 在 .vimrc 里加上以下内容就可以忽略大小写:

```
let g:netrw_sort_options="i"
```

缺省: ""

g:netrw_sort_sequence

按名字排序时, 先按逗号分隔的此模式序列排序。
注意 模式须考虑表示文件类型的任何附。
缺省: '[\/]$,*,\.bak$,\.o$,\.h$,
\.info$,\.swp$,\.obj$'

g:netrw_special_syntax

如果为真, 特定文件会以特殊语法出现在浏览器中:

```
netrwBak     : *.bak
netrwCompress: *.gz *.bz2 *.Z *.zip
netrwCoreDump: core.\d\+
netrwData    : *.dat
netrwDoc     : *.doc,*.txt,*.pdf,
               *.pdf,*.docx
netrwHdr     : *.h
netrwLex     : *.l *.lex
netrwLib     : *.a *.so *.lib *.dll
netrwMakefile: [mM]akefile *.mak
netrwObj     : *.o *.obj
netrwPix     : *.bmp,*.fit,*.fits,*.gif,
               *.jpg,*.jpeg,*.pcx,*.ppc
               *.pgm,*.png,*.psd,*.rgb
               *.tif,*.xbm,*.xcf
netrwTags    : tags ANmenu ANtags
netrwTilde   : *
netrwTmp     : tmp* *tmp
netrwYacc    : *.y
```

此外, 'suffixes' 提到的组也会加入特殊文件高亮组。
这些语法高亮组缺省被链接到 netrwGray 或 Folded (见 hl-Folded), 但你可以在 <.vimrc> 放上这样的行

```
hi link netrwCompress Visual
```

来定义自己的偏好。此外，还可以把设定存放在
.vim/after/syntax/netrw.vim.
以下情况下 netrw 设置 netrwGray 高亮模式
* netrwGray 之前没有定义过
* gui 在运行
例如，我自己使用深色背景的色彩方案，在
.vim/after/syntax/netrw.vim 里有以下设置:

```
hi netrwCompress term=NONE cterm=NONE gui=NONE ctermfg=10 guifg=green  ctermbg=0 guibg=black
hi netrwData     term=NONE cterm=NONE gui=NONE ctermfg=9 guifg=blue ctermbg=0 guibg=black
hi netrwHdr      term=NONE cterm=NONE,italic gui=NONE guifg=SeaGreen1
hi netrwLex      term=NONE cterm=NONE,italic gui=NONE guifg=SeaGreen1
hi netrwYacc     term=NONE cterm=NONE,italic gui=NONE guifg=SeaGreen1
hi netrwLib      term=NONE cterm=NONE gui=NONE ctermfg=14 guifg=yellow
hi netrwObj      term=NONE cterm=NONE gui=NONE ctermfg=12 guifg=red
hi netrwTilde    term=NONE cterm=NONE gui=NONE ctermfg=12 guifg=red
hi netrwTmp      term=NONE cterm=NONE gui=NONE ctermfg=12 guifg=red
hi netrwTags     term=NONE cterm=NONE gui=NONE ctermfg=12 guifg=red
hi netrwDoc      term=NONE cterm=NONE gui=NONE ctermfg=220 ctermbg=27 guifg=yellow2 guibg=Blue3
hi netrwSymLink  term=NONE cterm=NONE gui=NONE ctermfg=220 ctermbg=27 guifg=grey60
```

g:netrw_ssh_browse_reject

ssh 有时产生你不想要的行、消息、标题和任何你不希望看作是 "目录" 和 "文件" 的内容。
此模式可用于删除这些内嵌的信息。缺省值为:
'^total\s\+\d\+$'

g:netrw_ssh_cmd

可以用此值代替 ssh，用于执行列表，删除文件等这些远程动作的可执行命令。
缺省: ssh

g:netrw_suppress_gx_mesg

=1 : 浏览器有时生成通常不需要的消息，混杂在页面之中。
不过，比如使用 links 时，这些消息正是浏览器产生的。如果设置此选项为 0，netrw 不会抑制浏览器消息。

g:netrw_tmpfile_escape

=' &;'
对所有的临时文件使用 escape() 来转义这些字符。

g:netrw_timefmt

指定代替 %c 的 strftime() 的格式字符串
缺省值 "%c"，根据我的 strftime() 项目的 man 页面，是 "建议使用的本地 locale 日期时间的表现方式"；不过，不是所有人都满意。一些选择:
"%a %d %b %Y %T"、
" %a %Y-%m-%d %I-%M-%S %p"
缺省: "%c"

g:netrw_use_noswf

netrw 通常避免给浏览器缓冲区写入交换文件。不过，在有些系统上，看来这么做引起讨厌的 ml_get 错误；如果你看到 ml_get 错误，尝试在 .vimrc 中放入
let g:netrw_use_noswf= 0
缺省: 1

g:netrw_winsize — 指定 "o" (见 netrw-o)、"v" (见 netrw-v)、:Hexplore 或 :Vexplore 建立的新窗口的初始大小。g:netrw_winsize 是整数，指定当前 netrw 缓冲区窗口大小的百分比，来设定新窗口的大小。
如果 g:netrw_winsize 小于零，其绝对值用来指定新窗口的绝对行数和列数。
如果 g:netrw_winsize 为零，使用正常分割方式 (如 'equalalways' 生效，等等)。
缺省: "50" (代表 50%)

g:netrw_wiw — =1 指定收缩 netw/Lexplore 窗口使用的最小窗口宽度
(见 netrw-c-tab)。

g:netrw_xstrlen — 控制 netrw 如何计算包含多字节字符的字符串的长度 (来自 N Weibull、T Mechelynck)
=0: 使用 Vim 的内建 strlen()
=1: 代码点的个数 (拉丁 a + 组合用扬抑符是两个代码点) (缺省)
=2: 占空间的代码点个数 (拉丁 a + 组合用扬抑符是一个占空间的代码点; 硬制表符是一个; 宽和窄的 CJK 都是一个; 等等)
=3: 虚拟长度 (制表符在 1 到 'tabstop' 之间, 宽 CJK 是 2 而不是 1, 阿拉伯 alif 在 lam 之前是零, 否则为 1, 等等)

g:NetrwTopLvlMenu — 指定顶层子菜单名; 缺省为 "Netrw."。
如果你想改变，在 .vimrc 里进行。

NETRW 浏览和选项不兼容性 netrw-incompatible {{{2

netrw 的设计中，对用户选项的处理是先保存它们，设置和 netrw 需要兼容的选项值，然后恢复原有的值。但 autochdir 选项:

```
:set acd
```

是有问题的。autochdir 设置当前目录为编辑文件所在的目录; 这显然也适用于目录。换而言之，autochdir 设置当前目录为包含该 "文件" 的目录 (即使该 "文件" 本身也是目录)。

NETRW 设置窗口 netrw-settings-window {{{2

使用 NetrwSettings.vim 插件，

```
:NetrwSettings
```

会弹出一个窗口来显示用作 netrw 设置的许多变量。你可以修改这里列出的所有值; 保存文件时，设置的值就会被应用。在任何行上按 "?" 就可以得到每个变量功能的帮助。

(另见: netrw-browser-var netrw-protocol netrw-variables)

获取文件 netrw-obtain netrw-O {{{2

如果没有带标记文件:

浏览远程目录时，用户可用 O 键获取光标所在的文件 (即在本地机器上保存备份，但不编辑)。

如果有带标记文件:

    获取带标记文件 (即传输一个备份到本地机器, 但不用来编辑)。

此操作只支持 ftp 和 scp (既然只有它们能用作浏览, 这应该不是问题)。这时
状态行的右侧会显示 "Obtaining filename" 这样的消息。传输完成后, 状态行复原。

netrw 也可以用本地浏览器 "获得" 文件。netrw 显示的目录不一定和 Vim 的 "当前目录" 一致, 除非用户的 <.vimrc> 里把 g:netrw_keepdir 设为 0。在本地浏览器里选择文件 (光标移动到该文件名上) 然后按 "O" 就可以 "获得" 该文件; 也就是把它复制到 Vim 的当前目录。

相关主题:
 * 察看当前目录可用 :pwd
 * 使得当前浏览目录成为当前目录可见 netrw-c
 * 要自动使得当前浏览目录总成为当前目录可见 g:netrw_keepdir 。

在 NETRW 当 前 目 录 打 开 新 文 件 netrw-newfile netrw-createfile netrw-% {{{2

要在 netrw 当前目录打开一个新文件, 按 "%"。此映射询问用户新的文件名; 然后在
netrw 当前目录 (即 b:netrw_curdir) 就会建立以此命名的的空文件。

相关主题: netrw-d

预 览 窗 口 netrw-p netrw-preview {{{2

用 "p" 键可以使用预览窗口来预览光标所在的文件名。屏幕会被分割以同时显示浏览器
(光标还在那里) 和文件 (见 :pedit)。缺省, 进行水平分割; 如果先设置好
g:netrw_preview 就可以进行垂直分割。

一组有意思的 netrw 设置是:

```
let g:netrw_preview   = 1
let g:netrw_liststyle = 3
let g:netrw_winsize   = 30
```

它们会:

1. 使垂直分割成为缺省的文件预览方式
2. 使 "树形" 成为缺省的列表方式
3. 打开垂直预览窗口时, 目录列表只用 30% 列; 窗口其余部分用于预览窗口。

相关: 如果你喜欢这个想法, 你可能对 :Lexplore (netrw-:Lexplore) 或
g:netrw_chgwin 也会感兴趣

另见: g:netrw_chgwin netrw-P 'previewwindow' CTRL-W_z :pclose

前 次 窗 口 netrw-P netrw-prvwin {{{2

要使用上次使用 (最近访问) 的窗口编辑光标下的文件或目录 (见 :he CTRL-W_P), 按

"P"。如果只有一个窗口，该窗口被水平分割（缺省）。

如果多于一个窗口，重用上次使用的窗口以显示选择的文件/目录。如果前次窗口相关联的缓冲区被修改而且该缓冲区只有这个窗口，用户会被询问是否要先保存该缓冲区（yes（是）、no（否）或 cancel（取消））。

相关动作 netrw-cr netrw-o netrw-t netrw-v
相关设置变量:

g:netrw_alto	控制上/下分割
g:netrw_altv	控制右/左分割
g:netrw_preview	控制水平/垂直分割
g:netrw_winsize	控制初始大小

另见: g:netrw_chgwin netrw-p

刷 新 列 表 netrw-refresh netrw-ctrl-l netrw-ctrl_l {{{2

要刷新本地或远程目录列表，按 ctrl-l (<c-l>) 或在列表的 ./ 目录项上按 <cr>。用 ":e ." 也可以刷新本地目录。

反 转 排 序 顺 序 netrw-r netrw-reverse {{{2

用 "r" 键可以在正常和反转两种排序顺序之间切换。

相关主题: netrw-s
相关设置变量: g:netrw_sort_direction

给 文 件 或 目 录 换 名 netrw-move netrw-rename netrw-R {{{2

如果没有用 netrw-mf 标记过文件:

> 换名文件和目录涉及以下操作：移动光标到要移动（换名）的文件/目录，然后按 "R"。你会被询问要把文件/目录换什么名字。用 "V" 命令（可视选择）可以选择行范围，然后按 "R"；每个文件会依次询问要换成什么名字。

如果用 netrw-mf 标记过文件:

> 带标记文件会被移动（换名）；和上面一样，你会被询问文件/目录要移动到的位置。
>
> 如果你对换名询问的回答是 "s/frompattern/topattern/"，则会对标记文件列表的每个后续文件通过取得名字，执行替换，把替换结果作为要换的名字来进行换名。作为一个例子:
>
> ```
> mr  [query: reply with *.c]
> R   [query: reply with s/^\(.*\)\.c$/\1.cpp/]
> ```
>
> 此例会标记所有的 *.c 文件便执行换名为 *.cpp 文件。
>
> ctrl-X 字符对文件换名有特殊意义:
>
> ```
> <c-x>      : 单个 ctrl-x 告知 netrw 忽略响应在最后的 '/' 和 ctrl-x 之
> ```

间的部分。

<c-x><c-x> : 一对连续 ctrl-x 告知 netrw 忽略字符串在两个 ctrl-x 之前的所有部分。

警 告:

注意 移动文件是一个危险的操作；复制更安全。因为远程文件的 "移动" 实际上是复制 + 删除 -- 如果复制失败但删除成功，该文件就会丢失。
后果自负。

g:netrw_rename_cmd 变量实现远程换名。缺省值为:

```
ssh HOSTNAME mv
```

使用瘦风格时，用 V (linewise-visual) 可以给若干文件和目录同时换名。

见 cmdline-editing 关于更多如何编辑命令行的指示；尤其是 <ctrl-f> (开始命令行窗口编辑) 和 <ctrl-c> (使用光标下的命令行) 可以和 R 命令配合使用。

选 择 排 序 风 格 netrw-s netrw-sort {{{2

可以选择排序风格，使之按 name (名字)、time (时间) 或 size (文件大小) 排序。
"s" 映射允许你在这三个选择之间循环；目录列表会自动更新以反映选择的风格。

相关主题: netrw-r netrw-S
相关设置变量: g:netrw_sort_by g:netrw_sort_sequence

设 置 编 辑 窗 口 netrw-editwindow netrw-C netrw-:NetrwC {{{2

用 "C" 映射选择一个 netrw 窗口用以编辑，此映射使用 :NetrwC [win#] 命令，也可以设置 g:netrw_chgwin 为要选择的窗口号。此后，选择文件以进行编辑 (netrw-cr) 时都用该窗口。

* C : 单独使用，选择当前 netrw 缓冲区所在的窗口用于后续 netrw-cr 的编辑。C 映射只可用于 netrw 缓冲区。
* [count]C : 计数用作窗口号，用于后继的 netrw-cr 的编辑。
* :NetrwC 设置 g:netrw_chgwin 为当前窗口
* :NetrwC win# 设置 g:netrw_chgwin 为指定窗口号

用

```
let g:netrw_chgwin= -1
```

可以恢复缺省的编辑行为 (即使用当前窗口进行后续的编辑)。

相关主题: netrw-cr g:netrw_browse_split
相关设置变量: g:netrw_chgwin

缩 放 NETRW 或 LEXPLORE 窗 口 netrw-c-tab {{{2

<c-tab> 键切换 netrw 或 :Lexplore 窗口的宽度，但仅当 :netrw_usetab 存在且非零时才有效（当然，它还需要终端支持能区别 <c-tab> 和普通的 <tab>）。

* 如果当前窗口是 netrw 窗口，切换宽度（在 g:netrw_wiw 和它的原始宽度之间）
* 否则如果当前标签页有 :Lexplore 窗口，切换其宽度
* 否则打开新 :Lexplore 窗口

如果 g:netrw_usetab 存在且为零，或已有 <c-tab> 映射存在，不重新映射 <c-tab>。也可以使用 <c-tab> 以外的映射：（但 g:netrw_usetab 还必须设置）

```
nmap <unique> (随便什么)        <Plug>NetrwShrink
```

相关主题：　　:Lexplore
相关设置变量：　　g:netrw_usetab

用 户 定 义 映 射　　netrw-usermaps　　{{{1

可以自定义用户映射。指定变量， g:Netrw_UserMaps ，来保存键映射字符串和函数名的列表的 List ：

```
[["键映射序列","示例用户映射函数"],...]
```

netrw 为 netrw 缓冲区设置映射时，如果 g:Netrw_UserMaps 存在，调用内部函数 netrw#UserMaps(islocal)。此函数遍历 g:Netrw_UserMaps 列表的所有项目：

```
* 设置映射：
        nno <buffer> <silent> 键映射序列
        :call s:UserMaps(islocal,"示例用户映射函数")
* 如果函数调用返回字符串 "refresh"，刷新
* 如果返回的字符串不是 ""，执行该字符串（:exe 返回结果）
* 如果返回的是列表，对返回列表里的每个字符串执行上述两项
```

用户函数接受一个参数，就像这样

```
fun! ExampleUserMapFunc(islocal)
```

其中，如果是本地目录的系统调用，a:islocal 为 1，如果是远程目录的系统调用，为 0。

netrw-call　　netrw-expose　　netrw-modify

用 netrw#Expose("varname")　　来访问 netrw 内部（局部于脚本）变量。
用 netrw#Modify("varname",newvalue)　　来改变 netrw 内部变量。
用 netrw#Call("funcname"[,args])　　来调用 netrw 内部函数和指定所需参数。

示例：得到 netrw 标记文件列表的备份：

```
let netrwmarkfilelist= netrw#Expose("netrwmarkfilelist")
```

示例：修改 netrw 标记文件列表的值：

```
    call netrw#Modify("netrwmarkfilelist",[])
```

示例：通过映射 gu 来清除 netrw 标记文件列表

```
" ExampleUserMap: {{{2
fun! ExampleUserMap(islocal)
  call netrw#Modify("netrwmarkfilelist",[])
  call netrw#Modify('netrwmarkfilemtch_{bufnr("%")}','")
  let retval= ["refresh"]
  return retval
endfun
let g:Netrw_UserMaps= [["gu","ExampleUserMap"]]
```

---

10. 问题和修正 netrw-problems {{{1

(本节可能会不断增长，视乎我得到的反馈而定)
(另见 netrw-debug)

P1. 我用 windows 95，我的 ftp 每次读取之后多了额外的四个空行。 netrw-p1 {{{2

见 netrw-fixup ，在 <.vimrc> 文件里加入下行：

```
let g:netrw_win95ftp= 1
```

P2. 我用 windows，用 ftp 进行网络浏览不能按时间或大小排序 -或- 远程系统是 Windows 服务器；为什么我不能按时间或大小排序? netrw-p2 {{{2

Windows 的 ftp 只有基本的 ls 的支持 (即，不支持排序的选项)。它也不支持 -F 以给出类型字符 (ABC/ 代表 "ABC 是目录")。
netrw 使用 dir 得到的瘦长两种列表。如果你认为自己的 ftp 支持完整功能的 ls，在 <.vimrc> 加入下行：

```
let g:netrw_ftp_list_cmd= "ls -lF"
let g:netrw_ftp_timelist_cmd= "ls -tlF"
let g:netrw_ftp_sizelist_cmd= "ls -slF"
```

还有一个办法，如果你在 Windows 上安装了 cygwin，在 <.vimrc> 加入下行：

```
let g:netrw_cygwin= 1
```

如果远端系统使用 Windows，这个问题也会出现。在此情形下，g:netrw_ftp_[time|size]list_cmds 诸命令如上所示，但远端系统不会因而正确地修改它的列表方式。

P3. 我尝试 rcp://user@host/ (或任何不是 ftp 的协议) 但 netrw 使用 ssh! 我不要这样... netrw-p3 {{{2

netrw 有两个方法进行远程目录浏览：ssh 和 ftp。除非你指定 ftp，使用的只能是 ssh。
如果你需要下载文件 (不是目录列表)，netrw 会使用你要求的任何协

议。

netrw-p4

P4. 我喜欢缺省使用长格式的列表。 {{{2

在 .vimrc 里放入:

```
let g:netrw_liststyle=1
```

netrw-browser-var 说明你可以改变的更多设置。

netrw-p5

P5. 本地浏览时，时间显示不正常 {{{2

你系统的 strftime() 是否接受 "%c" 以产生类似于下面的格式: "Sun Apr 27 11:49:23 1997"? 如果不是，执行 "man strftime" 寻找什么选项可以完成类似的功能。然后在 .vimrc 里放入:

```
let g:netrw_timefmt= "%X"  (其中 X 是找到的选项)
```

netrw-p6

P6. 我想让当前目录和我的浏览同步。如何进行? {{{2

在 .vimrc 里放入:

```
let g:netrw_keepdir= 0
```

netrw-p7

P7. 我在文件名中使用中文 (或其它非 ascii) 字符，netrw (Explore、Sexplore、Hexplore 等) 却不能显示这些文件名! {{{2

(取自 Wu Yongwei 在 Vim 邮件列表上的回答)
我现在明白你的问题了。你的代码页不是 936，对么? Vim 看来只能打开在当前代码页中合法的文件名，这和许多其它不使用 Unicode 版本的 Windows API 的应用程序一样。这是操作系统上的问题。如果系统 locale 使用 UTF-8 就不应该有这个问题，例如现代的 Linux 的各种发布就是如此。

(...又多一个理由鼓励大家使用 utf-8!)

netrw-p8

P8. 我看到错误 "你的系统无法执行 ssh" -- 我该怎么办? {{{2

(Dudley Fox) 我知道的多数人在 windows 上 用 putty 的 ssh。这是一个免费的 ssh/telnet 应用。更多的情况可见:

http://www.chiark.greenend.org.uk/~sgtatham/putty/ 还有:

(Marlin Unruh) 我也用这个程序。它是一个单独的可执行文件，所以可以复制到 Windows\System32 文件夹并建立一个快捷方式。

(Dudley Fox) 你也可以考虑 plink，听起来它和你要找的很接近。plink 是 putty 程序包中的一个应用。

http://the.earth.li/~sgtatham/putty/0.58/htmldoc/Chapter7.html#plink

(Vissale Neang) 你可以试试 OpenSSH 的 windows 版本，可以在这里得到：

http://sshwindows.sourceforge.net/

它并不需要 Cygwin 包。

(Antoine Mechelynck) 对于单独需要在直接的 Windows 环境工作的 Unix 风格的程序，我推荐从 sourceforge 的 GnuWin32 项目获取，如果有的话：

    http://gnuwin32.sourceforge.net/

和 Cygwin 不同，后者建立一个在 Windows 之上的 Unix 风格的模拟机，而 GnuWin32 使用 Windows 系统调用来重写 Unix 工具，它的程序也能在 cmd.exe "Dos 窗口" 中正常运行。

(dave) 下载 WinSCP并用它连到服务器上。Preferences > Editors 选项里设置 gvim 为你的编辑器：

- 点击 "Add..."
- 设置外部编辑器（如果需要，调整实际的路径并加上引号和最后的 !.!）：
  "c:\Program Files\Vim\vim81\gvim.exe" !.!
- 确认下面框中的文件类型为 {asterisk}.{asterisk}（即所有文件），或任何你期望的类型（cec：把 {asterisk} 换成 * ；我这么写是因为不然的话，helptags 系统认为它是一个标签）
- 确认它在列表框的顶端（如若不然，点击它再点击 "Up"）

如果使用 Norton Commander 风格，你只要按 <F4> 来使用本地的 gvim 来编辑文件。

(Vit Gottwald) 如何生成公钥/私钥并把公钥存在服务器上：
http://www.chiark.greenend.org.uk/~sgtatham/putty/0.60/htmldoc/Chapter8.html#pubkey-gettingready
(8.3 Getting ready for public key authentication)

'pscp' 如何使用私钥：

http://www.chiark.greenend.org.uk/~sgtatham/putty/0.60/htmldoc/Chapter5.html
(5.2.4 Using public key authentication with PSCP)

(Ben Schmidt) 我发现 cwRsync 所带的 ssh 太好了，现在我在自己用的大多数 Windows 系统上都安装了 cwRsync 或 cwRsyncServer。对只需要在 Windows 上 ssh 的人来说，我推测同一作者提供的 COPSSH 可能更好，而且它还带有 cwRsync 没有的 sftp 等等工具。虽然它可能 ... (译者注：下文缺)

(cec) 要正确地使用的以上的建议，你需要在 .vimrc 中修改以下用户可设的变量：

g:netrw_ssh_cmd g:netrw_list_cmd g:netrw_mkdir_cmd
g:netrw_rm_cmd g:netrw_rmdir_cmd g:netrw_rmf_cmd

第一个 (g:netrw_ssh_cmd) 是最重要的；多数其它的变量在缺省情

况下使用和 g:netrw_ssh_cmd 相同的值。

netrw-p9 netrw-ml_get

P9. 我在浏览，切换目录，哦！出现了 ml_get 错误，我不得不杀掉 vim。 {{{2
有什么解决方法?

netrw 通常避免给临时目录缓冲区写入交换文件。不过，在有些系统上，看来这么做引起 ml_get 错误；尝试在 <.vimrc> 中设置 g:netrw_use_noswf 为零:

```
let g:netrw_use_noswf= 0
```

netrw-p10

P10. "[something] is a directory" 和 {{{2
"Press ENTER or type command to continue" 这些提示烦死我了...

"[something] is a directory" 提示是 Vim 发出的，不是 netrw，看来没有办法绕过它。再加上缺省的 cmdHeight 为 1，此消息导致了 "Press ENTER..." 提示。所以: 阅读 hit-enter ; 我也建议你在 <.vimrc> 文件里设置 'cmdheight' 为 2 (或更高)。

netrw-p11

P11. 我想要有两个窗口；一个瘦的在左边，我的编辑窗口在右边。应该怎么 {{{2
做?

你可能想在一个边窗运行 netrw。如果这样，":[N]Lexplore" 可能就是你所想要的。可选的 "[N]" 可以让你指定想要的 :Lexplore 开始的列数 (关于此参数如何工作，见 g:netrw_winsize)。

之前的方法:

* 在 <.vimrc> 放入:
  ```
  let g:netrw_altv = 1
  ```
* 编辑当前目录: :e .
* 选择某个文件，按 v
* 按照自己需要调整窗口大小 (见 CTRL-W_< 和 CTRL-W_>)。如果使用 gvim，可用鼠标拖动分割栏。
* 想编辑新文件时，用 ctrl-w h 回到 netrw 浏览器，选择文件，然后按 P (见 CTRL-W_h 和 netrw-P)。如果使用 gvim，在浏览器窗口按 <leftmouse> 然后用 <middlemouse> 来选择文件。

netrw-p12

P12. 我的目录排序不对，或者有不想要的字符出现在列出的文件名里，或者 {{{2
宽列表中名字的对齐有问题...

这可能是编码的问题，我自己通常使用 utf-8，但其实只用 ascii (也就是 32-126 的字符)。多字节编码每个字符使用两个 (或更多) 字节。你可能需要改变 g:netrw_sepchr 和/或 g:netrw_xstrlen 。

netrw-p13

P13. 我是 Windows + putty + ssh 的用户，每次我试图浏览时，目录没有 {{{2
结尾的 "/"，所以 netrw 当成文件传输而不试图当成子目录浏览。怎么办?

(mikeyao) 如果你在 Windows 上用 ssh 和 putty 访问 vim，尝试 pscp/psftp 加上 plink 的组合。pscp/psftp 用于连接，而 plink 用于执行服务器端的命令。例如：列出文件和 'ls' 目录。

以下是我使用的设置：

```
" 列出文件。这是关键设置。如果没有设置好，只能得到空白缓冲区。
let g:netrw_list_cmd = "plink HOSTNAME ls -Fa"
" 如果没有把 putty 目录加到系统目录中，必须指定 scp/sftp 命令的位
" 置。例如：
"let g:netrw_sftp_cmd = "d:\\dev\\putty\\PSFTP.exe"
"let g:netrw_scp_cmd = "d:\\dev\\putty\\PSCP.exe"
```

netrw-p14

P14. 我想加快 Nwrite 和 scp/ssh 风格连接的写操作。怎么做？(Thomer M. Gil) {{{2

ssh 的 ControlMaster 和 ControlPath（见 ssh_config man 帮助页）可用来使多个 ssh 连接共享同一个网络连接。这可减少每个文件写入的加密握手协议，有时这可提升速度一个数量级以上。
(见 http://thomer.com/howtos/netrw_ssh.html)
(此段文字得到许可)

在你的 ~/.ssh/config 中加入：

```
# 可把 "*" 改为你关心的终端名
Host *
  ControlMaster auto
  ControlPath /tmp/%r@%h:%p
```

然后建立一个到此终端的 ssh 连接，并保持之：

```
ssh -N host.domain.com
```

现在用 Vim 的 Netrw 远程打开文件，享受极速吧：

```
vim scp://host.domain.com//home/user/.bashrc
```

netrw-p15

P15. 如何用双击而不是 netrw 通常的单击来打开文件或目录？(Ben Fritz) {{{2

首先，关闭 netrw 的映射

```
let g:netrw_mousemaps= 0
```

然后在 $HOME/.vim/after/ftplugin/netrw.vim 中创建只适用 netrw 缓冲区的映射：

```
nmap <buffer> <2-leftmouse> <CR>
```

注意 设置 g:netrw_mousemaps 为零关闭所有 netrw 的鼠标映射，不只是 <leftmouse> 的。
(见 g:netrw_mousemaps)

netrw-p16

P16. Windows 里编辑远程文件（如 :e ftp://hostname/path/file）时，得到 E303 信息，抱怨不能打开交换文件。 {{{2

(romainl) 看来你从一个受保护的目录下启动 Vim。从你的 $HOME 或其它可写的目录下启动 netrw。

netrw-p17

P17. Netrw 自己关闭了缓冲区。 {{{2

如何重现此问题?

1. :Explore，访问目录，打开文件
2. :Explore，打开其它文件
3. 第一步打开的缓冲区被关闭。

期待结果是什么? 实际看到了什么?

我期待两个缓冲区同时存在，但实际只看见了最后那个。

(Lance) 问题出在 .vimrc 里的 "set autochdir"。
(drchip) 置位 'acd' 时我能重现此问题。不过，缓冲区不是真的关闭; ":ls!" 还能显示它们 (但 ":ls" 不会)。

netrw-P18

P18. 如何本地编辑只能通过 ssh 访问的其它服务器上的文件? {{{2

见 http://stackoverflow.com/questions/12469645/ "Using Vim to Remotely Edit A File on ServerB Only Accessible From ServerA"

netrw-P19

P19. 目录列表中如何得到行号? {{{2

通过 g:netrw_bufsettings 可以控制 netrw 缓冲区设置; 试试在 .vimrc 里放上

```
let g:netrw_bufsettings="noma nomod nu nobl nowrap ro nornu"
```

如果喜欢相对行号，试试

```
let g:netrw_bufsettings="noma nomod nonu nobl nowrap ro rnu"
```

netrw-P20

P20. 如何使 gvim 启动时显示目录列表? {{{2

在 .vimrc 里放上以下代码片断:

```
augroup VimStartup
  au!
  au VimEnter * if expand("%") == "" && argc() == 0 &&
  \ (v:servername =~ 'GVIM\d*' || v:servername == "")
  \ | e . | endif
augroup END
```

如果你特别想要，可以用 Lexplore 来代替 "e"。
此代码片断假定你打开客户-服务器功能 (即 "huge" vim 版本)。

netrw-P21

P21. 我建立的目录 (或文件) 使用了带重音的字符，但 netrw 不让我进入该目录/读取文件: {{{2

可能是外壳或操作系统使用了和 vim (netrw) 不同的编码。将来 vim 支持 "systemencoding" 的补丁也许可以解决这个问题; 现在，只要让 netrw 使用合适的编码就可以了。例如:

```
au FileType netrw set enc=latin1
```

netrw-P22

P22. 试图复制或移动文件时出错: {{{2

```
**error** (netrw) tried using g:netrw_localcopycmd<cp>; it doesn't work!
```

怎么回事?

Netrw 使用若干系统级的命令来完成任务（见

g:netrw_localcopycmd , g:netrw_localmovecmd , g:netrw_localrmdir , g:netrw_mkdir_cmd)。

可能需要在 .vimrc 里适当地调整部分这些命令。另一个困难源于这些命令使用了 vim 的本地目录，但它和 netrw 显示的浏览目录可能有所不同（见 g:netrw_keepdir)。

---

11. 调试 netrw 自身 netrw-debug {{{1

第一步：确定你遇到的问题是否所以通过获取最新的（通常是开发版本）netrw 得到解决，最新版本在：

http://www.drchip.org/astronaut/vim/index.html#NETRW

<netrw.vim> 脚本通常安装的系统位置是：

```
/usr/local/share/vim/vim8x/plugin/netrwPlugin.vim
/usr/local/share/vim/vim8x/autoload/netrw.vim
        (见 :echo &rtp 的输出)
```

在启动时自动被加载（假设 :set nocp)。如果你案头了新版本的 netrw，它位于

```
$HOME/.vim/plugin/netrwPlugin.vim
$HOME/.vim/autoload/netrw.vim
```

第二步：假定你安装了最新版本的 netrw，确定你的问题是否真的由 netrw 引起。新建一个叫 netrw.vimrc 的文件，包含以下内容：

```
set nocp
so $HOME/.vim/plugin/netrwPlugin.vim
```

然后这样执行 netrw:

```
vim -u netrw.vimrc --noplugins -i NONE [some path here]
```

执行任何你想要的 netrw 命令，然后检查问题是否存在。此过程回避了由于个人 .vimrc 设置，.viminfo 文件和其它插件引起的问题。如果问题不再出现，你需要自己找到什么 .vimrc 设置引起了和 netrw 或其它相关插件的冲突。

第三步：如果问题仍然存在，这样获取 netrw 的调试跟踪：

1. 先从这里取得 <Decho.vim> 脚本：

   http://www.drchip.org/astronaut/vim/index.html#DECHO
   或者

http://vim.sourceforge.net/scripts/script.php?script_id=120

Decho.vim 以 "vimball" 形式出现；见 vimball-intro 。

2. 修改 <netrw.vim> 文件如下：

```
vim netrw.vim
:DechoOn
:wq
```

要恢复到正常的非调试行为，修改 <netrw.vim> 如下：

```
vim netrw.vim
:DechoOff
:wq
```

此命令由 <Decho.vim> 提供，它会注释掉所有 Decho 调试语句 (Dfunc()、Dret()、Decho()、Dredir())。

3. 然后启动 vim 进行文件传输或者进行浏览以重现错误。一系列消息应该在一个单独的标签页或服务器 vim 窗口出现，提示 <netrw.vim> 进行网络文件读写的一系列步骤。

要保存文件，用

```
:tabnext
:set bt=
:w! DBG
```

进一步，最好你能键入

```
:Dsep <command>
```

其中的 <command> 是你将要键入的命令，这样就容易把调试跟踪的哪部分和哪条命令对应赶来。

请把该信息发送到 <netrw.vim> 的维护者，还请包括使用的操作系统和 vim 版本 (见 :version)

```
NdrOchip at ScampbellPfamily.AbizM - NOSPAM
```

---

12. 历史（英文） netrw-history {{{1

```
v167:   Nov 29, 2019    * netrw does a save&restore on @* and @+.
                          That causes problems with the clipboard.
                          Now restores occurs only if @* or @+ have
                          been changed.
                        * netrw will change @* or @+ less often.
                          Never if I happen to have caught all the
                          operations that modify the unnamed
                          register (which also writes @*).
                        * Modified hiding behavior so that "s"
                          will not ignore hiding.
v166:   Nov 06, 2019    * Removed a space from a nmap for "-"
                        * Numerous debugging statement changes
v163:   Dec 05, 2017    * (Cristi Balan) reported that a setting ('sel')
                          was left changed
```

* (Holger Mitschke) reported a problem with saving and restoring history. Fixed.
* Hopefully I fixed a nasty bug that caused a file rename to wipe out a buffer that it should not have wiped out.
* (Holger Mitschke) amended this help file with additional g:netrw_special_syntax items
* Prioritized wget over curl for g:netrw_http_cmd

v162: Sep 19, 2016
* (haya14busa) pointed out two syntax errors with a patch; these are now fixed.

Oct 26, 2016
* I started using mate-terminal and found that x and gx (netrw-x and netrw-gx) were no longer working. Fixed (using atril when $DESKTOP_SESSION is "mate").

Nov 04, 2016
* (Martin Vuille) pointed out that @+ was being restored with keepregstar rather than keepregplus.

Nov 09, 2016
* Broke apart the command from the options, mostly for Windows. Introduced new netrw settings: g:netrw_localcopycmdopt g:netrw_localcopydircmdopt g:netrw_localmkdiropt g:netrw_localmovecmdopt g:netrw_localrmdiropt

Nov 21, 2016
* (mattn) provided a patch for preview; swapped winwidth() with winheight()

Nov 22, 2016
* (glacambre) reported that files containing spaces weren't being obtained properly via scp. Fix: apparently using single quotes such as with 'file name' wasn't enough; the spaces inside the quotes also had to be escaped (ie. 'file\ name').
* Also fixed obtain (netrw-O) to be able to obtain files with spaces in their names

Dec 20, 2016
* (xc1427) Reported that using "I" (netrw-I) when atop "Hiding" in the banner also caused the active-banner hiding control to occur

Jan 03, 2017
* (Enno Nagel) reported that attempting to apply netrw to a directory that was without read permission caused a syntax error.

Jan 13, 2017
* (Ingo Karkat) provided a patch which makes using netrw#Call() better. Now returns value of internal routines return, for example.

Jan 13, 2017
* (Ingo Karkat) changed netrw#FileUrlRead to use :edit instead of :read . I also changed the routine name to netrw#FileUrlEdit.

Jan 16, 2017
* (Sayem) reported a problem where :Lexplore could generate a new listing buffer and window instead of toggling the netrw display. Unfortunately, the directions for eliciting the problem weren't complete, so I may or may not have fixed that issue.

Feb 06, 2017
* Implemented cb and cB. Changed "c" to "cd". (see netrw-cb , netrw-cB , and netrw-cd)

```
                Mar 21, 2017    * previously, netrw would specify (safe) settings
                                  even when the setting was already safe for
                                  netrw.  Netrw now attempts to leave such
                                  already-netrw-safe settings alone.
                                  (affects s:NetrwOptionRestore() and
                                  s:NetrwSafeOptions(); also introduced
                                  s:NetrwRestoreSetting())
                Jun 26, 2017    * (Christian Brabandt) provided a patch to
                                  allow curl to follow redirects (ie. -L
                                  option)
                Jun 26, 2017    * (Callum Howard) reported a problem with
                                  :Lexpore not removing the Lexplore window
                                  after a change-directory
                Aug 30, 2017    * (Ingo Karkat) one cannot switch to the
                                  previously edited file (e.g. with CTRL-^)
                                  after editing a file:// URL.  Patch to
                                  have a "keepalt" included.
                Oct 17, 2017    * (Adam Faryna) reported that gn ( netrw-gn )
                                  did not work on directories in the current
                                  tree
v157:           Apr 20, 2016    * (Nicola) had set up a "nmap <expr> ..." with
                                  a function that returned a 0 while silently
                                  invoking a shell command.  The shell command
                                  activated a ShellCmdPost event which in turn
                                  called s:LocalBrowseRefresh().  That looks
                                  over all netrw buffers for changes needing
                                  refreshes.  However, inside a  :map-<expr> ,
                                  tab and window changes are disallowed.  Fixed.
                                  (affects netrw's s:LocalBrowseRefresh())
                                * g:netrw_localrmdir  not used any more, but
                                  the relevant patch that causes  delete()  to
                                  take over was #1107 (not #1109).
                                * expand()  is now used on  g:netrw_home ;
                                  consequently, g:netrw_home may now use
                                  environment variables
                                * s:NetrwLeftmouse and s:NetrwCLeftmouse will
                                  return without doing anything if invoked
                                  when inside a non-netrw window
                Jun 15, 2016    * gx now calls netrw#GX() which returns
                                  the word under the cursor.  The new
                                  wrinkle: if one is in a netrw buffer,
                                  then netrw's s:NetrwGetWord().
                Jun 22, 2016    * Netrw was executing all its associated
                                  Filetype commands silently; I'm going
                                  to try doing that "noisily" and see if
                                  folks have a problem with that.
                Aug 12, 2016    * Changed order of tool selection for
                                  handling http://... viewing.
                                  (Nikolay Aleksandrovich Pavlov)
                Aug 21, 2016    * Included hiding/showing/all for tree
                                  listings
                                * Fixed refresh (^L) for tree listings
v156:           Feb 18, 2016    * Changed =~ to =~# where appropriate
                Feb 23, 2016    * s:ComposePath(base,subdir) now uses
```

fnameescape() on the base portion
Mar 01, 2016 * (gt_macki) reported where :Explore would make file unlisted. Fixed (tst943)
Apr 04, 2016 * (reported by John Little) netrw normally suppresses browser messages, but sometimes those "messages" are what is wanted. See g:netrw_suppress_gx_mesg
Apr 06, 2016 * (reported by Carlos Pita) deleting a remote file was giving an error message. Fixed.
Apr 08, 2016 * (Charles Cooper) had a problem with an undefined b:netrw_curdir. He also provided a fix.
Apr 20, 2016 * Changed s:NetrwGetBuffer(); now uses dictionaries. Also fixed the "No Name" buffer problem.
v155: Oct 29, 2015 * (Timur Fayzrakhmanov) reported that netrw's mapping of ctrl-l was not allowing refresh of other windows when it was done in a netrw window.
Nov 05, 2015 * Improved s:TreeSqueezeDir() to use search() instead of a loop
* NetrwBrowse() will return line to w:netrw_bannercnt if cursor ended up in banner
Nov 16, 2015 * Added a <Plug>NetrwTreeSqueeze (netrw-s-cr)
Nov 17, 2015 * Commented out imaps -- perhaps someone can tell me how they're useful and should be retained?
Nov 20, 2015 * Added netrw-ma and netrw-mA support
Nov 20, 2015 * gx (netrw-gx) on a URL downloaded the file in addition to simply bringing up the URL in a browser. Fixed.
Nov 23, 2015 * Added g:netrw_sizestyle support
Nov 27, 2015 * Inserted a lot of <c-u>s into various netrw maps.
Jan 05, 2016 * netrw-qL implemented to mark files based upon location-list s; similar to netrw-qF .
Jan 19, 2016 * using - call delete(directoryname,"d") - instead of using g:netrw_localrmdir if v7.4 + patch#1107 is available
Jan 28, 2016 * changed to using winsaveview() and winrestview()
Jan 28, 2016 * s:NetrwTreePath() now does a save and restore of view
Feb 08, 2016 * Fixed a tree-listing problem with remote directories
v154: Feb 26, 2015 * (Yuri Kanivetsky) reported a situation where a file was not treated properly as a file due to g:netrw_keepdir == 1
Mar 25, 2015 * (requested by Ben Friz) one may now sort by extension
Mar 28, 2015 * (requested by Matt Brooks) netrw has a lot of buffer-local mappings; however, some plugins (such as vim-surround) set up

```
                        conflicting mappings that cause vim to wait.
                        The "<nowait>" modifier has been included
                        with most of netrw's mappings to avoid that
                        delay.
                Jun 26, 2015    * netrw-gn mapping implemted
                                * :Ntree NotADir resulted in having
                                  the tree listing expand in the error messages
                                  window.  Fixed.
                Jun 29, 2015    * Attempting to delete a file remotely caused
                                  an error with "keepsol" mentioned; fixed.
                Jul 08, 2015    * Several changes to keep the :jumps table
                                  correct when working with
                                  g:netrw_fastbrowse set to 2
                                * wide listing with accented characters fixed
                                  (using %-S instead of %-s with a printf()
                Jul 13, 2015    * (Daniel Hahler) CheckIfKde() could be true
                                  but kfmclient not installed.  Changed order
                                  in netrw#BrowseX(): checks if kde and
                                  kfmclient, then will use xdg-open on a unix
                                  system (if xdg-open is executable)
                Aug 11, 2015    * (McDonnell) tree listing mode wouldn't
                                  select a file in a open subdirectory.
                                * (McDonnell) when multiple subdirectories
                                  were concurrently open in tree listing
                                  mode, a ctrl-L wouldn't refresh properly.
                                * The netrw:target menu showed duplicate
                                  entries
                Oct 13, 2015    * (mattn) provided an exception to handle
                                  windows with shellslash set but no shell
                Oct 23, 2015    * if g:netrw_usetab and <c-tab> now used
                                  to control whether NetrwShrink is used
                                  (see netrw-c-tab )
v153:   May 13, 2014    * added another g:netrw_ffkeep usage {{{2
                May 14, 2014    * changed s:PerformListing() so that it
                                  always sets ft=netrw for netrw buffers
                                  (ie. even when syntax highlighting is
                                  off, not available, etc)
                May 16, 2014    * introduced the netrw-ctrl-r functionality
                May 17, 2014    * introduced the netrw-:NetrwMB functionality
                                * mb and mB ( netrw-mb , netrw-mB ) will
                                  add/remove marked files from bookmark list
                May 20, 2014    * (Enno Nagel) reported that :Lex <dirname>
                                  wasn't working.  Fixed.
                May 26, 2014    * restored test to prevent leftmouse window
                                  resizing from causing refresh.
                                  (see s:NetrwLeftmouse())
                                * fixed problem where a refresh caused cursor
                                  to go just under the banner instead of
                                  staying put
                May 28, 2014    * (László Bimba) provided a patch for opening
                                  the :Lexplore window 100% high, optionally
                                  on the right, and will work with remote
                                  files.
                May 29, 2014    * implemented :NetrwC (see netrw-:NetrwC )
```

Jun 01, 2014 * Removed some "silent"s from commands used
to implemented scp://... and pscp://...
directory listing. Permits request for
password to appear.
Jun 05, 2014 * (Enno Nagel) reported that user maps "/"
caused problems with "b" and "w", which
are mapped (for wide listings only) to
skip over files rather than just words.
Jun 10, 2014 * g:netrw_gx introduced to allow users to
override default "<cfile>" with the gx
(netrw-gx) map
Jun 11, 2014 * gx (netrw-gx), with 'autowrite' set,
will write modified files. s:NetrwBrowseX()
will now save, turn off, and restore the
'autowrite' setting.
Jun 13, 2014 * added visual map for gx use
Jun 15, 2014 * (Enno Nagel) reported that with having hls
set and wide listing style in use, that the
b and w maps caused unwanted highlighting.
Jul 05, 2014 * netrw-mv and netrw-mX commands included
Jul 09, 2014 * g:netrw_keepj included, allowing optional
keepj
Jul 09, 2014 * fixing bugs due to previous update
Jul 21, 2014 * (Bruno Sutic) provided an updated
netrw_gitignore.vim
Jul 30, 2014 * (Yavuz Yetim) reported that editing two
remote files of the same name caused the
second instance to have a "temporary"
name. Fixed: now they use the same buffer.
Sep 18, 2014 * (Yasuhiro Matsumoto) provided a patch which
allows scp and windows local paths to work.
Oct 07, 2014 * gx (see netrw-gx) when atop a directory,
will now do gf instead
Nov 06, 2014 * For cygwin: cygstart will be available for
netrw#BrowseX() to use if its executable.
Nov 07, 2014 * Began support for file://... urls. Will use
g:netrw_file_cmd (typically elinks or links)
Dec 02, 2014 * began work on having mc (netrw-mc) copy
directories. Works for linux machines,
cygwin+vim, but not for windows+gvim.
Dec 02, 2014 * in tree mode, netrw was not opening
directories via symbolic links.
Dec 02, 2014 * added resolved link information to
thin and tree modes
Dec 30, 2014 * (issue#231) :ls was not showing
remote-file buffers reliably. Fixed.
v152: Apr 08, 2014 * uses the 'noswapfile' option (requires {{{2
vim 7.4 with patch 213)
* (Enno Nagel) turn 'rnu' off in netrw
buffers.
* (Quinn Strahl) suggested that netrw
allow regular window splitting to occur,
thereby allowing 'equalalways' to take
effect.

```
                    * (qingtian zhao) normally, netrw will
                      save and restore the  'fileformat' ;
                      however, sometimes that isn't wanted
                Apr 14, 2014    * whenever netrw marks a buffer as ro,
                      it will also mark it as nomod.
                Apr 16, 2014    * sftp protocol now supported by
                      netrw#Obtain(); this means that one
                      may use "mc" to copy a remote file
                      to a local file using sftp, and that
                      the  netrw-O  command can obtain remote
                      files via sftp.
                    * added [count]C support (see  netrw-C )
                Apr 18, 2014    * when  g:netrw_chgwin  is one more than
                      the last window, then vertically split
                      the last window and use it as the
                      chgwin window.
                May 09, 2014    * SavePosn was "saving filename under cursor"
                      from a non-netrw window when using :Rex.
v151:           Jan 22, 2014    * extended :Rexplore to return to buffer {{{2
                      prior to Explore or editing a directory
                    * (Ken Takata) netrw gave error when
                      clipboard was disabled.  Sol'n: Placed
                      several if has("clipboard") tests in.
                    * Fixed ftp://X@Y@Z// problem; X@Y now
                      part of user id, and only Z is part of
                      hostname.
                    * (A Loumiotis) reported that completion
                      using a directory name containing spaces
                      did not work.  Fixed with a retry in
                      netrw#Explore() which removes the
                      backslashes vim inserted.
                Feb 26, 2014    * :Rexplore now records the current file
                       using w:netrw_rexfile when returning via
                       :Rexplore
                Mar 08, 2014    * (David Kotchan) provided some patches
                      allowing netrw to work properly with
                      windows shares.
                    * Multiple one-liner help messages available
                      by pressing <cr> while atop the "Quick
                      Help" line
                    * worked on ShellCmdPost, FocusGained event
                      handling.
                    *  :Lexplore  path: will be used to update
                      a left-side netrw browsing directory.
                Mar 12, 2014    *  netrw-s-cr : use <s-cr>  to close
                      tree directory implemented
                Mar 13, 2014    * (Tony Mechylynck) reported that using
                      the browser with ftp on a directory,
                      and selecting a gzipped txt file, that
                      an E19 occurred (which was issued by
                      gzip.vim).  Fixed.
                Mar 14, 2014    * Implemented :MF and :MT (see  netrw-:MF
                      and  netrw-:MT , respectively)
                Mar 17, 2014    *  :Ntree  [dir] wasn't working properly; fixed
```

```
                Mar 18, 2014    * Changed all uses of set to setl
                Mar 18, 2014    * Commented the netrw_btkeep line in
                                  s:NetrwOptionSave(); the effect is that
                                  netrw buffers will remain as  'bt' =nofile.
                                  This should prevent swapfiles being created
                                  for netrw buffers.
                Mar 20, 2014    * Changed all uses of lcd to use s:NetrwLcd()
                                  instead.  Consistent error handling results
                                  and it also handles Window's shares
                                * Fixed  netrw-d  command when applied with ftp
                                * https: support included for netrw#NetRead()
v150:           Jul 12, 2013    * removed a "keepalt" to allow ":e #" to {{{2
                                  return to the netrw directory listing
                Jul 13, 2013    * (Jonas Diemer) suggested changing
                                  a <cWORD> to <cfile>.
                Jul 21, 2013    * (Yuri Kanivetsky) reported that netrw's
                                  use of mkdir did not produce directories
                                  following the user's umask.
                Aug 27, 2013    * introduced  g:netrw_altfile  option
                Sep 05, 2013    * s:Strlen() now uses  strdisplaywidth()
                                  when available, by default
                Sep 12, 2013    * (Selyano Baldo) reported that netrw wasn't
                                  opening some directories properly from the
                                  command line.
                Nov 09, 2013    *  :Lexplore  introduced
                                * (Ondrej Platek) reported an issue with
                                  netrw's trees (P15).  Fixed.
                                * (Jorge Solis) reported that "t" in
                                  tree mode caused netrw to forget its
                                  line position.
                Dec 05, 2013    * Added <s-leftmouse> file marking
                                  (see  netrw-mf )
                Dec 05, 2013    * (Yasuhiro Matsumoto) Explore should use
                                  strlen() instead s:Strlen() when handling
                                  multibyte chars with strpart()
                                  (ie. strpart() is byte oriented, not
                                  display-width oriented).
                Dec 09, 2013    * (Ken Takata) Provided a patch; File sizes
                                  and a portion of timestamps were wrongly
                                  highlighted with the directory color when
                                  setting `:let g:netrw_liststyle=1` on Windows.
                                * (Paul Domaskis) noted that sometimes
                                  cursorline was activating in non-netrw
                                  windows.  All but one setting of cursorline
                                  was done via setl; there was one that was
                                  overlooked.  Fixed.
                Dec 24, 2013    * (esquifit) asked that netrw allow the
                                  /cygdrive prefix be a user-alterable
                                  parameter.
                Jan 02, 2014    * Fixed a problem with netrw-based ballon
                                  evaluation (ie. netrw#NetrwBaloonHelp()
                                  not having been loaded error messages)
                Jan 03, 2014    * Fixed a problem with tree listings
                                * New command installed:  :Ntree
```

```
                Jan 06, 2014    * (Ivan Brennan) reported a problem with
                                  netrw-P .  Fixed.
                Jan 06, 2014    * Fixed a problem with  netrw-P  when the
                                  modified file was to be abandoned.
                Jan 15, 2014    * (Matteo Cavalleri) reported that when the
                                  banner is suppressed and tree listing is
                                  used, a blank line was left at the top of
                                  the display.  Fixed.
                Jan 20, 2014    * (Gideon Go) reported that, in tree listing
                                  style, with a previous window open, that
                                  the wrong directory was being used to open
                                  a file.  Fixed. (P21)
        v149:   Apr 18, 2013    * in wide listing format, now have maps for {{{2
                                  w and b to move to next/previous file
                Apr 26, 2013    * one may now copy files in the same
                                  directory; netrw will issue requests for
                                  what names the files should be copied under
                Apr 29, 2013    * Trying Benzinger's problem again.  Seems
                                  that commenting out the BufEnter and
                                  installing VimEnter (only) works.  Weird
                                  problem!  (tree listing, vim -O Dir1 Dir2)
                May 01, 2013    * :Explore ftp://... wasn't working.  Fixed.
                May 02, 2013    * introduced  g:netrw_bannerbackslash  as
                                  requested by Paul Domaskis.
                Jul 03, 2013    * Explore now avoids splitting when a buffer
                                  will be hidden.
        v148:   Apr 16, 2013    * changed Netrw's Style menu to allow direct {{{2
                                  choice of listing style, hiding style, and
                                  sorting style
```

---

13. Todo (英文) netrw-todo {{{1

```
07/29/09 : banner       : g:netrw_banner  can be used to suppress the
           suppression    banner.  This feature is new and experimental,
                          so its in the process of being debugged.
09/04/09 : "gp"         : See if it can be made to work for remote systems.
                        : See if it can be made to work with marked files.
```

---

14. 鸣谢 netrw-credits {{{1

```
        Vim editor      by Bram Moolenaar (谢了, Bram!)
        dav             提供支持: C Campbell
        fetch           提供支持: Bram Moolenaar 和 C Campbell
        ftp             提供支持: C Campbell <NdrOchip@ScampbellPfamily.AbizM>
        http            提供支持: Bram Moolenaar <bram@moolenaar.net>
        rcp
        rsync           提供支持: C Campbell (Erik Warendorph 提供建议)
        scp             提供支持: raf <raf@comdyn.com.au>
        sftp            提供支持: C Campbell

        inputsecret()、BufReadCmd、BufWriteCmd 由 C Campbell 提供
```

Jérme Augé	-- 也使用新的缓冲区方法操作 ftp+.netrc
Bram Moolenaar	-- 显然写了 vim 本身，还有 :e 和 v:cmdarg 的使用， fetch 等
Yasuhiro Matsumoto	-- 指出 undo+0r 的问题和解决方法
Erik Warendorph	-- 提供若干建议 (g:netrw_..._cmd 变量，rsync 等)
Doug Claar	-- 提供能成功通过 ftp 操作的测试的修改

模式行: {{{1
vim:tw=78:ts=8:noet:ft=help:norl:fdm=marker

VIM 参考手册 by Bram Moolenaar
译者: Willis
http://vimcdoc.sf.net

高亮匹配括号 matchparen

这里提到的功能是一个标准插件 standard-plugin 。
只有 'compatible' 不置位时才能使用此插件。

设置 "loaded_matchparen" 变量可以避免载入此插件:

```
:let loaded_matchparen = 1
```

此插件安装 CursorMoved、CursorMovedI 和 WinEnter 自动命令来重新定义匹配高亮。

:NoMatchParen :DoMatchParen

要在载入此插件后关闭它，用:

```
:NoMatchParen
```

要再次打开:

```
:DoMatchParen
```

使用的高亮是 MatchParen。":highlight" 命令可用来为它指定不同的颜色。例如:

```
:hi MatchParen ctermbg=blue guibg=lightblue
```

匹配的字符组来自 'matchpairs' 选项。你可以改变该值来高亮不同的匹配。注意 不是什么值都可以。比如，你不能高亮单引号或双引号，因为它们的开始和结束字符相同。

使用语法高亮的属性。如果光标目前不在字符串或注释语法项目里，则忽略字符串和注释里的匹配。任何名字里包含 "string" 或 "comment" 的语法项目都被认为是字符串或注释项目。

移动光标时，限制搜索范围以避免延迟。这些限制包括:
- 窗口的可见部分。
- 光标上下 100 行，避免关闭折叠存在时的延迟过长。
- 光标前后 'synmaxcol' 乘以 2 个字节，避免存在带语法高亮的长行时延迟过长。
- 300 毫秒（插入模式下是 60 毫秒）的超时。可用 g:matchparen_timeout 和 g:matchparen_insert_timeout 变量或局部于缓冲区的 b:matchparen_timeout 和 b:matchparen_insert_timeout 来改变。

如果你希望 % 命令更好的工作，可以使用 matchit 插件，见 matchit-install 。该插件也可用于跳过注释里的匹配。这不同于 matchparen 高亮，两者使用不同的机制。

---

pi_spec.txt For Vim version 8.2. 最近更新：2006年7月

作者：Gustavo Niemeyer
译者：Willis
http://vimcdoc.sf.net

这是一个针对 rpm 规格文件的文件类型插件。

此 Vim 插件让你轻松更新 RPM 规格（spec）文件的更新记录段（%changelog section）的内容。如果该段还不存在，它可以为你新建段。如果你今天还没有更新，它会帮你建立新的说明入口（entry）。如果你还没有提供格式串（ spec_chglog_format ），它会询问你的 email 地址，然后自动为你提供该格式串。

1. 如何使用 spec-how-to-use-it
2. 自定义 spec-customizing

---

1. 如何使用 spec-how-to-use-it

spec_chglog 插件提供如下映射：

```
:map <buffer> <LocalLeader>c <Plug>SpecChangelog
```

这意味着在规格文件里你可以通过按 maplocalleader 键（缺省是 '\'）加 'c' 来执行这个插件。如果你还没有设置 spec_chglog_format ，此时插件会要求你提供 email 地址，编辑过程里会用到这个地址。

每次你运行此插件，它检查更新记录，看看最近的说明入口是否今天由你写入。如果是，在此入口中插入新建更新记录，如果不是，它会建立一个新的更新记录入口。如果你打开 spec_chglog_release_info ，它同时还会检查文件名，版本和发布版本是不是匹配。如果你自己还没有更新该软件包的发布号的话。这个插件能够智能地询问你是否要这么做。

设置键盘映射 spec-setting-a-map

正如你应该已经知道的，可以设置键盘映射执行任何 Vim 命令（对此而言，其实还有别的）。如果你不喜欢缺省的映射 <LocalLeader>c，可以自己定义。下面说明在 .vimrc 文件里如何作此设置。比如，要把本插件映射到 <F5> 键：

```
au FileType spec map <buffer> <F5> <Plug>SpecChangelog
```

备注：本插件会尊重你的本人意愿，不会再次把它的缺省设置强加于你。

该命令只会在规格文件的缓冲区中定义这个映射。

---

2. 自定义 spec-customizing

格式串 spec_chglog_format

你可以轻松地定制自己的规格文件入口的样式。为此，只须在 .vimrc 文件中这样设置 "spec_chglog_format" 变量：

```
let spec_chglog_format = "%a %b %d %Y My Name <my@email.com>"
```

注意 "%a %b %d %Y" 是最常用的格式。如果你不提供格式串，第一次运行 SpecChanglog 命令时，它会询问 email 地址，然后为你构造 spec_chglog_format 变量。这种方式下，你只要提供 email 地址一次。

要知道可用的格式串选项，察看 strftime() 函数的 man 手册页。

何处插入新的更新记录 spec_chglog_prepend

本插件通常把新的 %changelog 更新记录（注意：不是入口本身）加在现有的记录之后。如果你设置了 spec_chglog_prepend 变量

```
let spec_chglog_prepend = 1
```

那么新记录将加在已有记录之前。

加入发布信息 spec_chglog_release_info

如果你愿意，本插件自动在每个更新记录入口中加入发布信息。这么做的一个好处是能控制软件包每次更改之后是否更新发布号。如果包的版本或发行号没有更新，它询问你是否要这么做。要打开这个功能，在 .vimrc 中加入以下代码：

```
let spec_chglog_release_info = 1
```

这样，更新记录入口的首个记录就会像这样：

```
+ name-1.0-1cl
```

如果你不喜欢发布更新功能，也不想每次检测到旧的发布号时都回答 "No"，可以关闭此功能

```
let spec_chglog_never_increase_release = 1
```

一切顺利！！

vim:tw=78:ts=8:noet:ft=help:norl:

pi_tar.txt For Vim version 8.2. 最近更新: 2013年1月

+====================+
| Tar 文件接口 |
+====================+

译者: Willis
http://vimcdoc.sf.net

Author: Charles E. Campbell <NdrOchip@ScampbellPfamily.AbizM>
(remove NOSPAM from Campbell's email first)
Copyright 2005-2012: tar-copyright
The VIM LICENSE (see copyright) applies to the files in this
package, including tarPlugin.vim, tar.vim, and pi_tar.txt. Like
anything else that's except use "tar.vim" instead of "VIM". Like
anything else that's free, tar.vim and its associated files are
provided *as is* and comes with no warranty of any kind, either
expressed or implied. No guarantees of merchantability. No
guarantees of suitability for any purpose. By using this plugin, you
agree that in no event will the copyright holder be liable for any
damages resulting from the use of this software. Use at your own risk!

---

1. 内容 tar tar-contents

1. 内容 ..... tar-contents
2. 用法 ..... tar-usage
3. 选项 ..... tar-options
4. 历史 ..... tar-history

---

2. 用法 tar-usage tar-manual

编辑 *.tar 文件时，此插件进行处理，显示内容页面。移动光标到文件上，然后按 <return> 键，就可以选择该文件进行编辑。编辑后，可以写回该文件。目前，不能用该插件建立新的 tar 归档文件。

:Vimuntar
VIMUNTAR

:Vimuntar [vimhome]

本命令，如有需要，把 tar 包文件复制到以下目录中的 .vim 或 vimfiles 目录: 若 [vimhome] 未指定， 'runtimepath' 的第一个可写目录。不然，[vimhome] 参数允许用户自定义该目录。

复制动作使用 g:tar_copycmd 定义的命令，即
cp 用于 cygwin、unix、macunix
copy 用于 windows (32、95、64、16)
提取动作使用 g:tar_extractcmd 定义的命令，缺省是
"tar -xf"

:TarDiff
支 持 比 较

:TarDiff [filename]

本命令会试图比较文件的 tar 包版本和系统上相对应的文件。为了找到系统上的对应文件，脚本会用到 tar 包中的文件相关联的路径。如果当前目录下没有该路径，:TarDiff 就无法找到相关的文件。

如果给出 [filename]，该文件名（和其路径）会被用来指定相关文件。

禁 止 载 入

如果由于某种原因你不想 vim 检查 tar 文件，在 <.vimrc> 中定义两个变量就可以不载入 tar 插件：

```
let g:loaded_tarPlugin= 1
let g:loaded_tar      = 1
```

---

3. 选项 tar-options

以下是用户可以改变的变量，通常在 <.vimrc> 文件里设置。

变量	缺省值	解释
g:tar_browseoptions	"Ptf"	用于得到内容列表
g:tar_readoptions	"OPxf"	用于从 tar 包里提取文件
g:tar_cmd	"tar"	tar 程序名
g:tar_nomax	0	如果为真，不最大化文件窗口
g:tar_secure	未定义	如果存在： 使用 "--" 来避免 tar 命令行上不想要的选项扩展。 请确定你用的 tar 命令接受 "--"；Posix 兼容的 tar 工具都接受。 如果不存在： tar 插件拒绝接受任何 "-" 开头的 tar 文件或成员文件。 不是所有的 tar 都支持 "--"，所以这不是缺省行为。
g:tar_writeoptions	"uf"	用于更新/替换文件

---

4. 历史（英文） tar-history

```
v28 Jun 23, 2011 * a few more decompression options (tbz tb2 txz)
v27 May 31, 2011 * moved cygwin detection before g:tar_copycmd handling
                 * inserted additional  :keepj  modifiers
                 * changed silent  to  sil!  ( :silent )
v26 Aug 09, 2010 * uses buffer-local instead of window variables to hold
                   tarfile name
                 * inserted keepj before 0d to protect jump list
v25 Jun 19, 2010 * (Jan Steffens) added support for xz compression
v24 Apr 07, 2009 * :Untarvim command implemented
    Sep 28, 2009 * Added lzma support
```

```
v22 Aug 08, 2008 * security fixes
v16 Jun 06, 2008 * tarfile:: used instead of tarfile: when editing files
                   inside tarballs.  Fixes a problem with tarballs called
                   things like c:\abc.tar. (tnx to Bill McCarthy)
v14 May 09, 2008 * arno caught a security bug
    May 28, 2008 * various security improvements.  Now requires patch 299
                   which provides the fnameescape() function
    May 30, 2008 * allows one to view *.gz and *.bz2 files that are in
                   *.tar files.
v12 Sep 07, 2007 * &shq now used if not the empty string for g:tar_shq
v10 May 02, 2006 * now using "redraw then echo" to show messages, instead
                   of "echo and prompt user"
v9 May 02, 2006 * improved detection of masquerading as tar file
v8 May 02, 2006 * allows editing of files that merely masquerade as tar
                  files
v7 Mar 22, 2006 * work on making tar plugin work across network
   Mar 27, 2006 * g:tar_cmd now available for users to change the name
                  of the tar program to be used.  By default, of course,
                  it's "tar".
v6 Dec 21, 2005 * writing to files not in directories caused problems -
                  fixed (pointed out by Christian Robinson)
v5 Nov 22, 2005 * report option workaround installed
v3 Sep 16, 2005 * handles writing files in an archive back to the
                  archive
   Oct 18, 2005 * <amatch> used instead of <afile> in autocmds
   Oct 18, 2005 * handles writing to compressed archives
   Nov 03, 2005 * handles writing tarfiles across a network using
                  netrw#NetWrite()
v2              * converted to use Vim7's new autoload feature by
                  Bram Moolenaar
v1 (original)   * Michael Toren (see http://michael.toren.net/code/)
```

---

vim:tw=78:ts=8:noet:ft=help

Vimball 归档处理程序

译者：Willis
http://vimcdoc.sf.net

Author: Charles E. Campbell <NdrOchip@ScampbellPfamily.AbizM>
  (remove NOSPAM from Campbell's email first)
Copyright: (c) 2004-2015 by Charles E. Campbell Vimball-copyright
  The VIM LICENSE (see copyright) applies to the files in this
  package, including vimballPlugin.vim, vimball.vim, and pi_vimball.txt.
  except use "vimball" instead of "VIM". Like anything else that's free,
  vimball.vim and its associated files are provided *as is* and comes with
  no warranty of any kind, either expressed or implied. No guarantees
  of merchantability. No guarantees of suitability for any purpose. By
  using this plugin, you agree that in no event will the copyright
  holder be liable for any damages resulting from the use of this
  software. Use at your own risk!

1. 内容 vba vimball vimball-contents

1. 内容.....: vimball-contents
2. Vimball 简介.....: vimball-intro
3. Vimball 手册.....: vimball-manual
   MkVimball.....: :MkVimball
   UseVimball.....: :UseVimball
   RmVimball.....: :RmVimball
4. Vimball 历史.....: vimball-history

2. Vimball 简介 vimball-intro

Vimball 的目标就是使插件用户的使用更方便。有了 vimball 以后，用户只需要：

```
vim someplugin.vba
:so %
:q
```

然后插件和它的所有部件都会被安装在合适的目录里。注意 用户无须刻意进到某个特定的目录来执行此命令。另外，插件的帮助也会被自动安装。

如果用户决定使用 AsNeeded 插件，vimball 会聪明地把本来要放到 .vim/plugin/ 中的脚本放到 .vim/AsNeeded/ 里去。

删除 vimball 安装的插件很容易：

```
vim
:RmVimball someplugin
```

举例说吧，zip 和 tarball 对应的操作就不是那么简单了。

Vimball 检视用户的 'runtimepath' 来确定把脚本放在哪里。如果可能，通常使用 runtimepath 提到的第一个目录。用

```
:echo &rtp
```

来查看该目录。

---

3. Vimball 手册 vimball-manual

创建 VIMBALL :MkVimball

```
:[range]MkVimball[!] filename [path]
```

行范围内指定要保存在 vimball 里的文件的路径，但不包含通常在 runtimepath ('rtp') 指定的那部分。例如:

```
plugin/something.vim
doc/something.txt
```

在此范围的行上使用

```
:[range]MkVimball filename
```

建立文件 "filename.vba"，Vimball.vim 随后可以用它来还原那些文件。如果文件 "filename.vba" 已经存在，MKVimball 会给出警告而不建立文件。注意这些路径都相对于你的 .vim (vimfiles) 目录，而所有文件都应该在该目录下。更准确地说，vimball 插件通常使用 'runtimepath' 里第一个存在的目录作为前缀；它不使用绝对路径，除非用户指定的就是如此。

如果使用感叹号 (!)，MkVimball 总是建立 "filename.vba" 文件。如果已经存在则覆盖之。此行为和 :w 类似。

如要强制允许文件名中包含斜杠，也可用感叹号版本实现 (即 :MkVimball! path/filename)。

http://vim.wikia.com/wiki/Using_VimBall_with_%27Make%27 给出的技巧提供了用 make 自动生成 vimball 的好方法。

通过 VIMBALL 建立目录 g:vimball_mkdir

首先尝试 mkdir() 命令 (并非所有系统都支持)。

如果该命令不存在，而且 g:vimball_mkdir 也不存在，它被设为:

```
|g:netrw_localmkdir|, 如果存在的话
"mkdir"           , 如果可执行的话
"makedir"         , 如果可执行的话
否则              , 未定义
```

用户可以自定义 g:vimball_mkdir 来直接指定建立目录的命令。vimball 如有需要，会通过该命令来建立目录。

控制 VIMBALL 提取目录 g:vimball_home

通过指定变量 g:vimball_home，你可以覆盖 'runtimepath' 的使用。

vimball-extract

```
vim filename.vba
```

简单编辑一个 vimball 文件，Vimball.vim 会告诉用户必须执行 (source) 该归档的内容才能提取其中的文件。

只有首行是 "Vimball Archiver by Charles E. Campbell" 的文件才会被认为是 vimball 文件，从而才会继续提取操作。

列出 VIMBALL 中的文件 :VimballList

```
:VimballList
```

本命令告诉 Vim 列出归档里的所有文件，包括每个文件的行数。

手动执行 VIMABALL 提取 :UseVimball

```
:UseVimball [path]
```

vimball 文件本身包含了该命令；它调用 vimball#Vimball() 例程，后者负责对 vimball 解包。用户可以手动执行该命令，而无须执行 vimball；用户还可以指定安装的路径来覆盖自动的选择，即 'runtimepath' 里的第一个存在的目录。

删除 VIMBALL :RmVimball

```
:RmVimball vimballfile [path]
```

本命令删除制定 vimball 产生的所有文件（但不包括它可能建立的任何目录）。用户也选择卸载文件的路径（见 'runtimepath'）；否则，缺省是 'runtimepath' 里的第一个存在的目录。
为了实现这一点，该目录下包含一个文件 (.VimballRecord)，它包含了目前为止所有使用过的 vimball 中要删除的文件的记录。

禁止载入

如果由于某种原因你不想使用 vimball 提取插件，在 <.vimrc> 中加入以下两个变量就可以不载入 vimball.vim:

```
let g:loaded_vimballPlugin= 1
let g:loaded_vimball      = 1
```

WINDDOWS vimball-windows

许多 vimball 文件使用 gzip 压缩。Windows 不幸地没有自带解压 gzip 文件的工具。幸运的是，Windows 用户可以找到不少解压 gzip 的工具:

项目	工具/套件	免费	网站
7zip	工具	是	http://www.7-zip.org/
Winzip	工具	否	http://www.winzip.com/downwz.htm
unxutils	套件	是	http://unxutils.sourceforge.net/
cygwin	套件	是	http://www.cygwin.com/
GnuWin32	套件	是	http://gnuwin32.sourceforge.net/
MinGW	套件	是	http://www.mingw.org/

```
      37 : Jul 18, 2014 * (by request of T. Miedema) added augroup around
                          the autocmds in vimballPlugin.vim
           Jul 06, 2015 * there are two uses of tabc; changed to tabc!
      34 : Sep 22, 2011 * "UseVimball path" now supports a non-full path by
                          prepending the current directory to it.
      33 : Apr 02, 2011 * Gave priority to *.vmb over *.vba
                        * Changed silent! to sil! (shorter)
                        * Safed 'swf' setting (during vimball extraction,
                          its now turned off)
      32 : May 19, 2010 * (Christian Brabrandt) :so someplugin.vba and
                          :so someplugin.vba.gz (and the other supported
                          compression types) now works
                        * (Jan Steffens) added support for xz compression
                        * fenc extraction was erroneously picking up the
                          end of the line number when no file encoding
                          was present.  Fixed.
                        * By request, beginning the switchover from the vba
                          extension to vmb.  Currently both are supported;
                          MkVimball, however, now will create *.vmb files.
           Feb 11, 2011 * motoyakurotsu reported an error with vimball's
                          handling of zero-length files
           Feb 18, 2016 * Changed =~ to =~# where appropriate
      30 : Dec 08, 2008 * fnameescape() inserted to protect error
                          messaging using corrupted filenames from
                          causing problems
                        * RmVimball supports filenames that would
                          otherwise be considered to have "magic"
                          characters (ie. Abc[1].vba)
           Feb 18, 2009 * s:Escape(), g:vimball_shq, and g:netrw_shq
                          removed (shellescape() used directly)
           Oct 05, 2009 * (Nikolai Weibull) suggested that MkVimball
                          be allowed to use slashes in the filename.
      26 : May 27, 2008 * g:vimball_mkdir usage installed.  Makes the
                          $HOME/.vim (or $HOME\vimfiles) directory if
                          necessary.
           May 30, 2008 * (tnx to Bill McCarthy) found and fixed a bug:
                          vimball wasn't updating plugins to AsNeeded/
                          when it should
      25 : Mar 24, 2008 * changed vimball#Vimball() to recognize doc/*.??x
                          files as help files, too.
           Apr 18, 2008 * RmVimball command is now protected by saving and
                          restoring settings -- in particular, acd was
                          causing problems as reported by Zhang Shuhan
      24 : Nov 15, 2007 * g:vimball_path_escape used by s:Path() to
                          prevent certain characters from causing trouble
                          (defunct: fnameescape() and shellescape()
                          now used instead)
      22 : Mar 21, 2007 * uses setlocal instead of set during BufEnter
      21 : Nov 27, 2006 * (tnx to Bill McCarthy) vimball had a header
                          handling problem and it now changes \s to /s
```

```
20 : Nov 20, 2006 * substitute() calls have all had the 'e' flag
                    removed.
18 : Aug 01, 2006 * vimballs now use folding to easily display their
                    contents.
                  * if a user has AsNeeded/somefile, then vimball
                    will extract plugin/somefile to the AsNeeded/
                    directory
17 : Jun 28, 2006 * changes all \s to /s internally for Windows
16 : Jun 15, 2006 * A. Mechelynck's idea to allow users to specify
                    installation root paths implemented for
                    UseVimball, MkVimball, and RmVimball.
                  * RmVimball implemented
15 : Jun 13, 2006 * bugfix
14 : May 26, 2006 * bugfixes
13 : May 01, 2006 * exists("&acd") used to determine if the acd
                    option exists
12 : May 01, 2006 * bugfix - the  'acd'  option is not always defined
11 : Apr 27, 2006 * VimballList would create missing subdirectories that
                    the vimball specified were needed.  Fixed.
10 : Apr 27, 2006 * moved all setting saving/restoration to a pair of
                    functions.  Included some more settings in them
                    which frequently cause trouble.
9  : Apr 26, 2006 * various changes to support Windows' prediction
                    for backslashes and spaces in file and directory
                    names.
7  : Apr 25, 2006 * bypasses foldenable
                  * uses more exe and less norm! (:yank :put etc)
                  * does better at insuring a "Press ENTER" prompt
                    appears to keep its messages visible
4  : Mar 31, 2006 * BufReadPost seems to fire twice; BufReadEnter
                    only fires once, so the "Source this file..."
                    message is now issued only once.
3  : Mar 20, 2006 * removed query, now requires sourcing to be
                    extracted (:so %).  Message to that effect
                    included.
                  * :VimballList  now shows files that would be
                    extracted.
2  : Mar 20, 2006 * query, :UseVimball included
1  : Mar 20, 2006 * initial release
```

```
vim:tw=78:ts=8:noet:ft=help:fdm=marker
```

pi_zip.txt For Vim version 8.2. 最近更新: 2017年8月

```
+====================+
|    Zip 文件接口    |
+====================+
```

译者: Willis
http://vimcdoc.sf.net

Author: Charles E. Campbell <NdrOchip@ScampbellPfamily.AbizM>
(remove NOSPAM from Campbell's email first)
Copyright: Copyright (C) 2005-2015 Charles E Campbell zip-copyright
The VIM LICENSE (see copyright) applies to the files in this
package, including zipPlugin.vim, zip.vim, and pi_zip.vim. except use
"zip.vim" instead of "VIM". Like anything else that's free, zip.vim
and its associated files are provided *as is* and comes with no
warranty of any kind, either expressed or implied. No guarantees of
merchantability. No guarantees of suitability for any purpose. By
using this plugin, you agree that in no event will the copyright
holder be liable for any damages resulting from the use of this
software. Use at your own risk!

---

1. 内容 zip zip-contents

1. 内容 ..... zip-contents
2. 用法 ..... zip-usage
3. 附加扩展名 ..... zip-extension
4. 历史 ..... zip-history

---

2. 用法 zip-usage zip-manual

编辑 *.zip 文件时，此插件进行处理，显示内容页面。移动光标到文件上，然后按 <return> 键，就可以选择该文件进行编辑。编辑后，可以写回该文件。目前，不能用该插件建立新的 zip 归档文件。

zip-x

x : 光标在列出文件名上时可以进行提取

选 项

g:zip_nomax

如果此变量存在且为真，文件窗口在打开时不会自动最大化。

g:zip_shq

不同的操作系统可能使用一个或多个外壳来执行命令。Zip 试图猜测正确的引号使用机制，使得文件名中可以包含空格和其它；如果猜错了，可用
g:zip_shq
修正。它在 Unix 上缺省是单引号 ('), 而 Windows 上缺省是双引号 (")。如果你希望不用引号，简单地在 <.vimrc> 里把 g:zip_shq 置为空串 (let g:zip_shq= "")。

g:zip_unzipcmd

用此选项指定执行 "unzip" 任务的程序，用于浏览。缺省:

```
let g:zip_unzipcmd= "unzip"
```

g:zip_zipcmd

用此选项指定执行 "zip" 任务的程序，用于写入（更新）已在 zip 包中的文件；缺省：

```
let g:zip_zipcmd= "zip"
```

g:zip_extractcmd

用此选项指定程序（和所需的选项），用于从 zip 归档文件提取文件。缺省

```
let g:zip_extractcmd= g:zip_unzipcmd
```

禁 止 载 入

如果由于某种原因你不想 vim 检查 zip 文件，在 <.vimrc> 中定义两个变量就可以不载入 zip 插件：

```
let g:loaded_zipPlugin= 1
let g:loaded_zip      = 1
```

---

3. 附加扩展名 zip-extension

看来有不少归档程序生成 zip 文件但不使用 .zip 扩展名（.jar、.xpi 等）。要处理这些文件，在 <.vimrc> 文件中放上：

```
au BufReadCmd *.jar,*.xpi call zip#Browse(expand("<amatch>"))
```

你可以自由扩展该行以包含更多应被当作 zip 文件的扩展名。

此外，也可以在 .vimrc 里改变 g:zipPlugin_ext 。当前（11/30/15）包含了：

```
let g:zipPlugin_ext= '*.zip,*.jar,*.xpi,*.ja,*.war,*.ear,*.celzip,
\ *.oxt,*.kmz,*.wsz,*.xap,*.docx,*.docm,*.dotx,*.dotm,*.potx,*.potm,
\ *.ppsx,*.ppsm,*.pptx,*.pptm,*.ppam,*.sldx,*.thmx,*.xlam,*.xlsx,*.xlsm,
\ *.xlsb,*.xltx,*.xltm,*.xlam,*.crtx,*.vdw,*.glox,*.gcsx,*.gqsx,*.epub'
```

---

4. 历史（英文） zip-history {{{1

```
v28 Oct 08, 2014 * changed the sanity checks for executables to reflect
                   the command actually to be attempted in zip#Read()
                   and zip#Write()
                 * added the extraction of a file capability
    Nov 30, 2015 * added *.epub to the  g:zipPlugin_ext  list
    Sep 13, 2016 * added *.apk to the  g:zipPlugin_ext  list and
                   sorted the suffices.
v27 Jul 02, 2013 * sanity check: zipfile must have "PK" as its first
                   two bytes.
                 * modified to allow zipfile: entries in quickfix lists
v26 Nov 15, 2012 * (Jason Spiro) provided a lot of new extensions that
                   are synonyms for .zip
v25 Jun 27, 2011 * using keepj with unzip -Z
                   (consistent with the -p variant)
```

```
                  * (Ben Staniford) now uses
                    has("win32unix") && executable("cygpath")
                    before converting to cygwin-style paths
   v24 Jun 21, 2010 * (Cdric Bosdonnat) unzip seems to need its filenames
                    fnameescape'd as well as shellquote'd
                  * (Motoya Kurotsu) inserted keepj before 0d to protect
                    jump list
   v17 May 09, 2008 * arno caught a security bug
   v15 Sep 07, 2007 * &shq now used if not the empty string for g:zip_shq
   v14 May 07, 2007 * using b:zipfile instead of w:zipfile to avoid problem
                    when editing alternate file to bring up a zipfile
   v10 May 02, 2006 * now using "redraw then echo" to show messages, instead
                    of "echo and prompt user"
                  * g:zip_shq provided to allow for quoting control for the
                    command being passed via :r! ... commands.
   v8 Apr 10, 2006 * Bram Moolenaar reported that he received an error message
                    due to "Pattern not found: ^.*\%0c"; this was caused by
                    stridx finding a Name... at the beginning of the line;
                    zip.vim tried 4,$s/^.*\%0c//, but that doesn't work.
                    Fixed.
   v7 Mar 22, 2006 * escaped some characters that can cause filename handling
                    problems.
   v6 Dec 21, 2005 * writing to files not in directories caused problems -
                    fixed (pointed out by Christian Robinson)
   v5 Nov 22, 2005 * report option workaround installed
   v3 Oct 18, 2005 * <amatch> used instead of <afile> in autocmds
   v2 Sep 16, 2005 * silenced some commands (avoiding hit-enter prompt)
                  * began testing under Windows; works thus far
                  * filetype detection fixed
      Nov 03, 2005 * handles writing zipfiles across a network using
                    netrw#NetWrite()
   v1 Sep 15, 2005 * Initial release, had browsing, reading, and writing
```

```
vim:tw=78:ts=8:noet:ft=help:fdm=marker
```

```
!_TAG_FILE_ENCODING                utf-8                //
!                      change.txt                      /*!*
!!                     change.txt                      /*!!*
#                      pattern.txt                     /*#*
#{}                     eval.txt                      /*#{}*
$                      motion.txt                      /*$*
$HOME                   options.txt                   /*$HOME*
$HOME-use               version5.txt                 /*$HOME-use*
$HOME-windows             options.txt               /*$HOME-windows*
$MYGVIMRC                 gui.txt                   /*$MYGVIMRC*
$MYVIMRC                starting.txt                  /*$MYVIMRC*
$VIM                   starting.txt                     /*$VIM*
$VIM-use                version5.txt                  /*$VIM-use*
$VIMRUNTIME              starting.txt                /*$VIMRUNTIME*
$VIM_POSIX               vi_diff.txt                 /*$VIM_POSIX*
%                      motion.txt                      /*%*
```

```
%:.                     cmdline.txt                 /*%:.*
%:8                     cmdline.txt                 /*%:8*
%:S                     cmdline.txt                 /*%:S*
%:e                     cmdline.txt                 /*%:e*
%:gs                    cmdline.txt                 /*%:gs*
%:h                     cmdline.txt                 /*%:h*
%:p                     cmdline.txt                 /*%:p*
%:r                     cmdline.txt                 /*%:r*
%:s                     cmdline.txt                 /*%:s*
%:t                     cmdline.txt                 /*%:t*
%:~                     cmdline.txt                 /*%:~*
&                       change.txt                  /*&*
'                       motion.txt                  /*'*
''                      motion.txt                  /*''*
'(                      motion.txt                  /*'(*
')                      motion.txt                  /*')*
'.                      motion.txt                  /*'.*
'0                      motion.txt                  /*'0*
'<                      motion.txt                  /*'<*
'>                      motion.txt                  /*'>*
'A                      motion.txt                  /*'A*
'[                      motion.txt                  /*'[*
']                      motion.txt                  /*']*
'^                      motion.txt                  /*'^*
'a                      motion.txt                  /*'a*
'acd'                   options.txt                 /*'acd'*
'ai'                    options.txt                 /*'ai'*
'akm'                   options.txt                 /*'akm'*
'al'                    options.txt                 /*'al'*
'aleph'                 options.txt                 /*'aleph'*
'allowrevins'           options.txt                 /*'allowrevins'*
'altkeymap'             options.txt                 /*'altkeymap'*
'ambiwidth'             options.txt                 /*'ambiwidth'*
'ambw'                  options.txt                 /*'ambw'*
'anti'                  options.txt                 /*'anti'*
'antialias'             options.txt                 /*'antialias'*
'ap'                    vi_diff.txt                 /*'ap'*
'ar'                    options.txt                 /*'ar'*
'arab'                  options.txt                 /*'arab'*
'arabic'                options.txt                 /*'arabic'*
'arabicshape'           options.txt                 /*'arabicshape'*
'ari'                   options.txt                 /*'ari'*
'arshape'               options.txt                 /*'arshape'*
'as'                    todo.txt                    /*'as'*
'autochdir'             options.txt                 /*'autochdir'*
'autoindent'            options.txt                 /*'autoindent'*
'autoprint'             vi_diff.txt                 /*'autoprint'*
'autoread'              options.txt                 /*'autoread'*
'autosave'              todo.txt                    /*'autosave'*
'autowrite'             options.txt                 /*'autowrite'*
'autowriteall'          options.txt                 /*'autowriteall'*
'aw'                    options.txt                 /*'aw'*
'awa'                   options.txt                 /*'awa'*
'background'            options.txt                 /*'background'*
```

```
'backspace'             options.txt             /*'backspace'*
'backup'               options.txt              /*'backup'*
'backupcopy'            options.txt            /*'backupcopy'*
'backupdir'             options.txt            /*'backupdir'*
'backupext'             options.txt            /*'backupext'*
'backupskip'            options.txt            /*'backupskip'*
'balloondelay'           options.txt          /*'balloondelay'*
'ballooneval'           options.txt            /*'ballooneval'*
'balloonevalterm'         options.txt        /*'balloonevalterm'*
'balloonexpr'           options.txt            /*'balloonexpr'*
'bdir'                 options.txt               /*'bdir'*
'bdlay'                options.txt               /*'bdlay'*
'beautify'              vi_diff.txt             /*'beautify'*
'belloff'              options.txt              /*'belloff'*
'beval'                options.txt              /*'beval'*
'bevalterm'             options.txt            /*'bevalterm'*
'bex'                  options.txt               /*'bex'*
'bexpr'                options.txt              /*'bexpr'*
'bf'                  vi_diff.txt               /*'bf'*
'bg'                  options.txt               /*'bg'*
'bh'                  options.txt               /*'bh'*
'bin'                 options.txt               /*'bin'*
'binary'               options.txt             /*'binary'*
'biosk'                options.txt             /*'biosk'*
'bioskey'               options.txt            /*'bioskey'*
'bk'                  options.txt               /*'bk'*
'bkc'                 options.txt               /*'bkc'*
'bl'                  options.txt               /*'bl'*
'bo'                  options.txt               /*'bo'*
'bomb'                options.txt              /*'bomb'*
'breakat'              options.txt             /*'breakat'*
'breakindent'           options.txt           /*'breakindent'*
'breakindentopt'         options.txt        /*'breakindentopt'*
'bri'                 options.txt               /*'bri'*
'briopt'               options.txt             /*'briopt'*
'brk'                 options.txt               /*'brk'*
'browsedir'             options.txt           /*'browsedir'*
'bs'                  options.txt               /*'bs'*
'bsdir'                options.txt              /*'bsdir'*
'bsk'                 options.txt               /*'bsk'*
'bt'                  options.txt               /*'bt'*
'bufhidden'             options.txt           /*'bufhidden'*
'buflisted'             options.txt           /*'buflisted'*
'buftype'              options.txt            /*'buftype'*
'casemap'              options.txt            /*'casemap'*
'cb'                  options.txt               /*'cb'*
'cc'                  options.txt               /*'cc'*
'ccv'                 options.txt               /*'ccv'*
'cd'                  options.txt               /*'cd'*
'cdpath'               options.txt             /*'cdpath'*
'cedit'                options.txt             /*'cedit'*
'cf'                  options.txt               /*'cf'*
'cfu'                 options.txt               /*'cfu'*
'ch'                  options.txt               /*'ch'*
```

'character' intro.txt /*'character'*
'charconvert' options.txt /*'charconvert'*
'ci' options.txt /*'ci'*
'cin' options.txt /*'cin'*
'cindent' options.txt /*'cindent'*
'cink' options.txt /*'cink'*
'cinkeys' options.txt /*'cinkeys'*
'cino' options.txt /*'cino'*
'cinoptions' options.txt /*'cinoptions'*
'cinw' options.txt /*'cinw'*
'cinwords' options.txt /*'cinwords'*
'clipboard' options.txt /*'clipboard'*
'cm' options.txt /*'cm'*
'cmdheight' options.txt /*'cmdheight'*
'cmdwinheight' options.txt /*'cmdwinheight'*
'cmp' options.txt /*'cmp'*
'cms' options.txt /*'cms'*
'co' options.txt /*'co'*
'cocu' options.txt /*'cocu'*
'cole' options.txt /*'cole'*
'colorcolumn' options.txt /*'colorcolumn'*
'columns' options.txt /*'columns'*
'com' options.txt /*'com'*
'comments' options.txt /*'comments'*
'commentstring' options.txt /*'commentstring'*
'compatible' options.txt /*'compatible'*
'complete' options.txt /*'complete'*
'completefunc' options.txt /*'completefunc'*
'completeopt' options.txt /*'completeopt'*
'completepopup' options.txt /*'completepopup'*
'completeslash' options.txt /*'completeslash'*
'concealcursor' options.txt /*'concealcursor'*
'conceallevel' options.txt /*'conceallevel'*
'confirm' options.txt /*'confirm'*
'consk' options.txt /*'consk'*
'conskey' options.txt /*'conskey'*
'copyindent' options.txt /*'copyindent'*
'cot' options.txt /*'cot'*
'cp' options.txt /*'cp'*
'cpo' options.txt /*'cpo'*
'cpoptions' options.txt /*'cpoptions'*
'cpp' options.txt /*'cpp'*
'cpt' options.txt /*'cpt'*
'crb' options.txt /*'crb'*
'cryptmethod' options.txt /*'cryptmethod'*
'cscopepathcomp' options.txt /*'cscopepathcomp'*
'cscopeprg' options.txt /*'cscopeprg'*
'cscopequickfix' options.txt /*'cscopequickfix'*
'cscoperelative' options.txt /*'cscoperelative'*
'cscopetag' options.txt /*'cscopetag'*
'cscopetagorder' options.txt /*'cscopetagorder'*
'cscopeverbose' options.txt /*'cscopeverbose'*
'csl' options.txt /*'csl'*
'cspc' options.txt /*'cspc'*

```
'csprg'              options.txt              /*'csprg'*
'csqf'               options.txt              /*'csqf'*
'csre'               options.txt              /*'csre'*
'cst'                options.txt              /*'cst'*
'csto'               options.txt              /*'csto'*
'csverb'             options.txt              /*'csverb'*
'cuc'                options.txt              /*'cuc'*
'cul'                options.txt              /*'cul'*
'culopt'             options.txt              /*'culopt'*
'cursorbind'         options.txt              /*'cursorbind'*
'cursorcolumn'       options.txt              /*'cursorcolumn'*
'cursorline'         options.txt              /*'cursorline'*
'cursorlineopt'      options.txt              /*'cursorlineopt'*
'cwh'                options.txt              /*'cwh'*
'debug'              options.txt              /*'debug'*
'deco'               options.txt              /*'deco'*
'def'                options.txt              /*'def'*
'define'             options.txt              /*'define'*
'delcombine'         options.txt              /*'delcombine'*
'dex'                options.txt              /*'dex'*
'dg'                 options.txt              /*'dg'*
'dict'               options.txt              /*'dict'*
'dictionary'         options.txt              /*'dictionary'*
'diff'               options.txt              /*'diff'*
'diffexpr'           options.txt              /*'diffexpr'*
'diffopt'            options.txt              /*'diffopt'*
'digraph'            options.txt              /*'digraph'*
'dip'                options.txt              /*'dip'*
'dir'                options.txt              /*'dir'*
'directory'          options.txt              /*'directory'*
'display'            options.txt              /*'display'*
'dy'                 options.txt              /*'dy'*
'ea'                 options.txt              /*'ea'*
'ead'                options.txt              /*'ead'*
'eadirection'        options.txt              /*'eadirection'*
'eb'                 options.txt              /*'eb'*
'ed'                 options.txt              /*'ed'*
'edcompatible'       options.txt              /*'edcompatible'*
'ef'                 options.txt              /*'ef'*
'efm'                options.txt              /*'efm'*
'ei'                 options.txt              /*'ei'*
'ek'                 options.txt              /*'ek'*
'emo'                options.txt              /*'emo'*
'emoji'              options.txt              /*'emoji'*
'enc'                options.txt              /*'enc'*
'encoding'           options.txt              /*'encoding'*
'endofline'          options.txt              /*'endofline'*
'eol'                options.txt              /*'eol'*
'ep'                 options.txt              /*'ep'*
'equalalways'        options.txt              /*'equalalways'*
'equalprg'           options.txt              /*'equalprg'*
'errorbells'         options.txt              /*'errorbells'*
'errorfile'          options.txt              /*'errorfile'*
'errorformat'        options.txt              /*'errorformat'*
```

```
'esckeys'              options.txt              /*'esckeys'*
'et'                  options.txt                /*'et'*
'eventignore'           options.txt            /*'eventignore'*
'ex'                  options.txt                /*'ex'*
'expandtab'             options.txt            /*'expandtab'*
'exrc'                options.txt                /*'exrc'*
'fcl'                 options.txt                /*'fcl'*
'fcs'                 options.txt                /*'fcs'*
'fdc'                 options.txt                /*'fdc'*
'fde'                 options.txt                /*'fde'*
'fdi'                 options.txt                /*'fdi'*
'fdl'                 options.txt                /*'fdl'*
'fdls'                options.txt                /*'fdls'*
'fdm'                 options.txt                /*'fdm'*
'fdn'                 options.txt                /*'fdn'*
'fdo'                 options.txt                /*'fdo'*
'fdt'                 options.txt                /*'fdt'*
'fe'                  options.txt                /*'fe'*
'fen'                 options.txt                /*'fen'*
'fenc'                options.txt                /*'fenc'*
'fencs'               options.txt                /*'fencs'*
'fex'                 options.txt                /*'fex'*
'ff'                  options.txt                /*'ff'*
'ffs'                 options.txt                /*'ffs'*
'fic'                 options.txt                /*'fic'*
'fileencoding'          options.txt            /*'fileencoding'*
'fileencodings'         options.txt            /*'fileencodings'*
'fileformat'            options.txt            /*'fileformat'*
'fileformats'           options.txt            /*'fileformats'*
'fileignorecase'        options.txt            /*'fileignorecase'*
'filetype'            options.txt                /*'filetype'*
'fillchars'           options.txt                /*'fillchars'*
'fixendofline'          options.txt            /*'fixendofline'*
'fixeol'              options.txt                /*'fixeol'*
'fk'                  options.txt                /*'fk'*
'fkmap'               options.txt                /*'fkmap'*
'fl'                  vi_diff.txt                /*'fl'*
'flash'               vi_diff.txt                /*'flash'*
'flp'                 options.txt                /*'flp'*
'fml'                 options.txt                /*'fml'*
'fmr'                 options.txt                /*'fmr'*
'fo'                  options.txt                /*'fo'*
'foldclose'             options.txt            /*'foldclose'*
'foldcolumn'            options.txt            /*'foldcolumn'*
'foldenable'            options.txt            /*'foldenable'*
'foldexpr'            options.txt                /*'foldexpr'*
'foldignore'            options.txt            /*'foldignore'*
'foldlevel'             options.txt            /*'foldlevel'*
'foldlevelstart'          options.txt          /*'foldlevelstart'*
'foldmarker'            options.txt            /*'foldmarker'*
'foldmethod'            options.txt            /*'foldmethod'*
'foldminlines'          options.txt            /*'foldminlines'*
'foldnestmax'           options.txt            /*'foldnestmax'*
'foldopen'            options.txt                /*'foldopen'*
```

'foldtext' options.txt /*'foldtext'*
'formatexpr' options.txt /*'formatexpr'*
'formatlistpat' options.txt /*'formatlistpat'*
'formatoptions' options.txt /*'formatoptions'*
'formatprg' options.txt /*'formatprg'*
'fp' options.txt /*'fp'*
'fs' options.txt /*'fs'*
'fsync' options.txt /*'fsync'*
'ft' options.txt /*'ft'*
'gcr' options.txt /*'gcr'*
'gd' options.txt /*'gd'*
'gdefault' options.txt /*'gdefault'*
'gfm' options.txt /*'gfm'*
'gfn' options.txt /*'gfn'*
'gfs' options.txt /*'gfs'*
'gfw' options.txt /*'gfw'*
'ghr' options.txt /*'ghr'*
'go' options.txt /*'go'*
'go-!' options.txt /*'go-!'*
'go-A' options.txt /*'go-A'*
'go-F' options.txt /*'go-F'*
'go-L' options.txt /*'go-L'*
'go-M' options.txt /*'go-M'*
'go-P' options.txt /*'go-P'*
'go-R' options.txt /*'go-R'*
'go-T' options.txt /*'go-T'*
'go-a' options.txt /*'go-a'*
'go-b' options.txt /*'go-b'*
'go-c' options.txt /*'go-c'*
'go-d' options.txt /*'go-d'*
'go-e' options.txt /*'go-e'*
'go-f' options.txt /*'go-f'*
'go-g' options.txt /*'go-g'*
'go-h' options.txt /*'go-h'*
'go-i' options.txt /*'go-i'*
'go-k' options.txt /*'go-k'*
'go-l' options.txt /*'go-l'*
'go-m' options.txt /*'go-m'*
'go-p' options.txt /*'go-p'*
'go-r' options.txt /*'go-r'*
'go-t' options.txt /*'go-t'*
'go-v' options.txt /*'go-v'*
'gp' options.txt /*'gp'*
'gr' vi_diff.txt /*'gr'*
'graphic' vi_diff.txt /*'graphic'*
'grepformat' options.txt /*'grepformat'*
'grepprg' options.txt /*'grepprg'*
'gtl' options.txt /*'gtl'*
'gtt' options.txt /*'gtt'*
'guicursor' options.txt /*'guicursor'*
'guifont' options.txt /*'guifont'*
'guifontset' options.txt /*'guifontset'*
'guifontwide' options.txt /*'guifontwide'*
'guiheadroom' options.txt /*'guiheadroom'*

```
'guioptions'            options.txt          /*'guioptions'*
'guipty'               options.txt            /*'guipty'*
'guitablabel'           options.txt         /*'guitablabel'*
'guitabtooltip'          options.txt       /*'guitabtooltip'*
'hardtabs'             vi_diff.txt           /*'hardtabs'*
'helpfile'             options.txt           /*'helpfile'*
'helpheight'            options.txt          /*'helpheight'*
'helplang'             options.txt           /*'helplang'*
'hf'                  options.txt              /*'hf'*
'hh'                  options.txt              /*'hh'*
'hi'                  options.txt              /*'hi'*
'hid'                 options.txt              /*'hid'*
'hidden'               options.txt            /*'hidden'*
'highlight'             options.txt          /*'highlight'*
'history'              options.txt           /*'history'*
'hk'                  options.txt              /*'hk'*
'hkmap'                options.txt           /*'hkmap'*
'hkmapp'               options.txt           /*'hkmapp'*
'hkp'                 options.txt              /*'hkp'*
'hl'                  options.txt              /*'hl'*
'hlg'                 options.txt              /*'hlg'*
'hls'                 options.txt              /*'hls'*
'hlsearch'              options.txt          /*'hlsearch'*
'ht'                  vi_diff.txt              /*'ht'*
'ic'                  options.txt              /*'ic'*
'icon'                 options.txt            /*'icon'*
'iconstring'             options.txt        /*'iconstring'*
'ignorecase'             options.txt        /*'ignorecase'*
'im'                  options.txt               /*'im'*
'imactivatefunc'          options.txt       /*'imactivatefunc'*
'imactivatekey'           options.txt       /*'imactivatekey'*
'imaf'                 options.txt             /*'imaf'*
'imak'                 options.txt             /*'imak'*
'imc'                  options.txt             /*'imc'*
'imcmdline'              options.txt         /*'imcmdline'*
'imd'                  options.txt             /*'imd'*
'imdisable'              options.txt         /*'imdisable'*
'imi'                  options.txt             /*'imi'*
'iminsert'               options.txt         /*'iminsert'*
'ims'                  options.txt             /*'ims'*
'imsearch'               options.txt         /*'imsearch'*
'imsf'                 options.txt             /*'imsf'*
'imst'                 options.txt             /*'imst'*
'imstatusfunc'            options.txt       /*'imstatusfunc'*
'imstyle'               options.txt          /*'imstyle'*
'inc'                  options.txt             /*'inc'*
'include'                options.txt         /*'include'*
'includeexpr'             options.txt       /*'includeexpr'*
'incsearch'              options.txt         /*'incsearch'*
'inde'                 options.txt             /*'inde'*
'indentexpr'             options.txt         /*'indentexpr'*
'indentkeys'             options.txt         /*'indentkeys'*
'indk'                 options.txt             /*'indk'*
'inex'                 options.txt             /*'inex'*
```

```
'inf'                options.txt                /*'inf'*
'infercase'            options.txt              /*'infercase'*
'insertmode'           options.txt              /*'insertmode'*
'is'                 options.txt                  /*'is'*
'isf'                options.txt                  /*'isf'*
'isfname'             options.txt               /*'isfname'*
'isi'                options.txt                  /*'isi'*
'isident'             options.txt               /*'isident'*
'isk'                options.txt                  /*'isk'*
'iskeyword'            options.txt              /*'iskeyword'*
'isp'                options.txt                  /*'isp'*
'isprint'             options.txt               /*'isprint'*
'joinspaces'           options.txt             /*'joinspaces'*
'js'                 options.txt                  /*'js'*
'key'                options.txt                  /*'key'*
'keymap'              options.txt               /*'keymap'*
'keymodel'             options.txt              /*'keymodel'*
'keywordprg'           options.txt              /*'keywordprg'*
'km'                 options.txt                  /*'km'*
'kmp'                options.txt                  /*'kmp'*
'kp'                 options.txt                  /*'kp'*
'langmap'             options.txt               /*'langmap'*
'langmenu'             options.txt              /*'langmenu'*
'langnoremap'           options.txt            /*'langnoremap'*
'langremap'            options.txt              /*'langremap'*
'laststatus'           options.txt              /*'laststatus'*
'lazyredraw'           options.txt              /*'lazyredraw'*
'lbr'                options.txt                  /*'lbr'*
'lcs'                options.txt                  /*'lcs'*
'linebreak'            options.txt              /*'linebreak'*
'lines'               options.txt                /*'lines'*
'linespace'            options.txt              /*'linespace'*
'lisp'               options.txt                  /*'lisp'*
'lispwords'            options.txt              /*'lispwords'*
'list'               options.txt                  /*'list'*
'listchars'            options.txt              /*'listchars'*
'lm'                 options.txt                  /*'lm'*
'lmap'               options.txt                  /*'lmap'*
'lnr'                options.txt                  /*'lnr'*
'loadplugins'           options.txt             /*'loadplugins'*
'lpl'                options.txt                  /*'lpl'*
'lrm'                options.txt                  /*'lrm'*
'ls'                 options.txt                  /*'ls'*
'lsp'                options.txt                  /*'lsp'*
'luadll'              options.txt               /*'luadll'*
'lw'                 options.txt                  /*'lw'*
'lz'                 options.txt                  /*'lz'*
'ma'                 options.txt                  /*'ma'*
'macatsui'             options.txt              /*'macatsui'*
'magic'               options.txt               /*'magic'*
'makeef'              options.txt               /*'makeef'*
'makeencoding'          options.txt            /*'makeencoding'*
'makeprg'             options.txt               /*'makeprg'*
'mat'                options.txt                  /*'mat'*
```

```
'matchpairs'          options.txt          /*'matchpairs'*
'matchtime'           options.txt          /*'matchtime'*
'maxcombine'          options.txt          /*'maxcombine'*
'maxfuncdepth'        options.txt          /*'maxfuncdepth'*
'maxmapdepth'         options.txt          /*'maxmapdepth'*
'maxmem'              options.txt          /*'maxmem'*
'maxmempattern'       options.txt          /*'maxmempattern'*
'maxmemtot'           options.txt          /*'maxmemtot'*
'mco'                 options.txt          /*'mco'*
'mef'                 options.txt          /*'mef'*
'menc'                options.txt          /*'menc'*
'menuitems'           options.txt          /*'menuitems'*
'mesg'                vi_diff.txt          /*'mesg'*
'mfd'                 options.txt          /*'mfd'*
'mh'                  options.txt          /*'mh'*
'mis'                 options.txt          /*'mis'*
'mkspellmem'          options.txt          /*'mkspellmem'*
'ml'                  options.txt          /*'ml'*
'mle'                 options.txt          /*'mle'*
'mls'                 options.txt          /*'mls'*
'mm'                  options.txt          /*'mm'*
'mmd'                 options.txt          /*'mmd'*
'mmp'                 options.txt          /*'mmp'*
'mmt'                 options.txt          /*'mmt'*
'mod'                 options.txt          /*'mod'*
'modeline'            options.txt          /*'modeline'*
'modelineexpr'        options.txt          /*'modelineexpr'*
'modelines'           options.txt          /*'modelines'*
'modifiable'          options.txt          /*'modifiable'*
'modified'            options.txt          /*'modified'*
'more'                options.txt          /*'more'*
'mouse'               options.txt          /*'mouse'*
'mousef'              options.txt          /*'mousef'*
'mousefocus'          options.txt          /*'mousefocus'*
'mousehide'           options.txt          /*'mousehide'*
'mousem'              options.txt          /*'mousem'*
'mousemodel'          options.txt          /*'mousemodel'*
'mouses'              options.txt          /*'mouses'*
'mouseshape'          options.txt          /*'mouseshape'*
'mouset'              options.txt          /*'mouset'*
'mousetime'           options.txt          /*'mousetime'*
'mp'                  options.txt          /*'mp'*
'mps'                 options.txt          /*'mps'*
'msm'                 options.txt          /*'msm'*
'mzq'                 options.txt          /*'mzq'*
'mzquantum'           options.txt          /*'mzquantum'*
'mzschemedll'         options.txt          /*'mzschemedll'*
'mzschemegcdll'       options.txt          /*'mzschemegcdll'*
'nf'                  options.txt          /*'nf'*
'noacd'               options.txt          /*'noacd'*
'noai'                options.txt          /*'noai'*
'noakm'               options.txt          /*'noakm'*
'noallowrevins'       options.txt          /*'noallowrevins'*
'noaltkeymap'         options.txt          /*'noaltkeymap'*
```

'noanti' options.txt /*'noanti'*
'noantialias' options.txt /*'noantialias'*
'noar' options.txt /*'noar'*
'noarab' options.txt /*'noarab'*
'noarabic' options.txt /*'noarabic'*
'noarabicshape' options.txt /*'noarabicshape'*
'noari' options.txt /*'noari'*
'noarshape' options.txt /*'noarshape'*
'noas' todo.txt /*'noas'*
'noautochdir' options.txt /*'noautochdir'*
'noautoindent' options.txt /*'noautoindent'*
'noautoread' options.txt /*'noautoread'*
'noautosave' todo.txt /*'noautosave'*
'noautowrite' options.txt /*'noautowrite'*
'noautowriteall' options.txt /*'noautowriteall'*
'noaw' options.txt /*'noaw'*
'noawa' options.txt /*'noawa'*
'nobackup' options.txt /*'nobackup'*
'noballooneval' options.txt /*'noballooneval'*
'noballoonevalterm' options.txt /*'noballoonevalterm'*
'nobeval' options.txt /*'nobeval'*
'nobevalterm' options.txt /*'nobevalterm'*
'nobin' options.txt /*'nobin'*
'nobinary' options.txt /*'nobinary'*
'nobiosk' options.txt /*'nobiosk'*
'nobioskey' options.txt /*'nobioskey'*
'nobk' options.txt /*'nobk'*
'nobl' options.txt /*'nobl'*
'nobomb' options.txt /*'nobomb'*
'nobreakindent' options.txt /*'nobreakindent'*
'nobri' options.txt /*'nobri'*
'nobuflisted' options.txt /*'nobuflisted'*
'nocf' options.txt /*'nocf'*
'noci' options.txt /*'noci'*
'nocin' options.txt /*'nocin'*
'nocindent' options.txt /*'nocindent'*
'nocompatible' options.txt /*'nocompatible'*
'noconfirm' options.txt /*'noconfirm'*
'noconsk' options.txt /*'noconsk'*
'noconskey' options.txt /*'noconskey'*
'nocopyindent' options.txt /*'nocopyindent'*
'nocp' options.txt /*'nocp'*
'nocrb' options.txt /*'nocrb'*
'nocscoperelative' options.txt /*'nocscoperelative'*
'nocscopetag' options.txt /*'nocscopetag'*
'nocscopeverbose' options.txt /*'nocscopeverbose'*
'nocsre' options.txt /*'nocsre'*
'nocst' options.txt /*'nocst'*
'nocsverb' options.txt /*'nocsverb'*
'nocuc' options.txt /*'nocuc'*
'nocul' options.txt /*'nocul'*
'nocursorbind' options.txt /*'nocursorbind'*
'nocursorcolumn' options.txt /*'nocursorcolumn'*
'nocursorline' options.txt /*'nocursorline'*

```
'nodeco'                options.txt                /*'nodeco'*
'nodelcombine'            options.txt            /*'nodelcombine'*
'nodg'                  options.txt                /*'nodg'*
'nodiff'                options.txt                /*'nodiff'*
'nodigraph'              options.txt              /*'nodigraph'*
'noea'                  options.txt                /*'noea'*
'noeb'                  options.txt                /*'noeb'*
'noed'                  options.txt                /*'noed'*
'noedcompatible'           options.txt           /*'noedcompatible'*
'noek'                  options.txt                /*'noek'*
'noemo'                 options.txt                /*'noemo'*
'noemoji'                options.txt               /*'noemoji'*
'noendofline'             options.txt             /*'noendofline'*
'noeol'                 options.txt                /*'noeol'*
'noequalalways'            options.txt            /*'noequalalways'*
'noerrorbells'            options.txt             /*'noerrorbells'*
'noesckeys'              options.txt              /*'noesckeys'*
'noet'                  options.txt                /*'noet'*
'noex'                  options.txt                /*'noex'*
'noexpandtab'             options.txt             /*'noexpandtab'*
'noexrc'                options.txt                /*'noexrc'*
'nofen'                 options.txt                /*'nofen'*
'nofic'                 options.txt                /*'nofic'*
'nofileignorecase'         options.txt         /*'nofileignorecase'*
'nofixendofline'           options.txt          /*'nofixendofline'*
'nofixeol'               options.txt              /*'nofixeol'*
'nofk'                  options.txt                /*'nofk'*
'nofkmap'                options.txt               /*'nofkmap'*
'nofoldenable'            options.txt            /*'nofoldenable'*
'nofs'                  options.txt                /*'nofs'*
'nofsync'                options.txt               /*'nofsync'*
'nogd'                  options.txt                /*'nogd'*
'nogdefault'              options.txt             /*'nogdefault'*
'noguipty'               options.txt              /*'noguipty'*
'nohid'                 options.txt                /*'nohid'*
'nohidden'               options.txt              /*'nohidden'*
'nohk'                  options.txt                /*'nohk'*
'nohkmap'                options.txt               /*'nohkmap'*
'nohkmapp'               options.txt               /*'nohkmapp'*
'nohkp'                 options.txt                /*'nohkp'*
'nohls'                 options.txt                /*'nohls'*
'nohlsearch'              options.txt             /*'nohlsearch'*
'noic'                  options.txt                /*'noic'*
'noicon'                options.txt                /*'noicon'*
'noignorecase'             options.txt           /*'noignorecase'*
'noim'                  options.txt                /*'noim'*
'noimc'                 options.txt                /*'noimc'*
'noimcmdline'             options.txt             /*'noimcmdline'*
'noimd'                 options.txt                /*'noimd'*
'noimdisable'             options.txt             /*'noimdisable'*
'noincsearch'             options.txt             /*'noincsearch'*
'noinf'                 options.txt                /*'noinf'*
'noinfercase'             options.txt             /*'noinfercase'*
'noinsertmode'            options.txt            /*'noinsertmode'*
```

```
'nois'                options.txt              /*'nois'*
'nojoinspaces'          options.txt          /*'nojoinspaces'*
'nojs'                options.txt              /*'nojs'*
'nolangnoremap'         options.txt          /*'nolangnoremap'*
'nolangremap'           options.txt          /*'nolangremap'*
'nolazyredraw'          options.txt          /*'nolazyredraw'*
'nolbr'               options.txt              /*'nolbr'*
'nolinebreak'           options.txt          /*'nolinebreak'*
'nolisp'              options.txt              /*'nolisp'*
'nolist'              options.txt              /*'nolist'*
'nolnr'               options.txt              /*'nolnr'*
'noloadplugins'         options.txt          /*'noloadplugins'*
'nolpl'               options.txt              /*'nolpl'*
'nolrm'               options.txt              /*'nolrm'*
'nolz'                options.txt              /*'nolz'*
'noma'                options.txt              /*'noma'*
'nomacatsui'            options.txt          /*'nomacatsui'*
'nomagic'             options.txt              /*'nomagic'*
'nomh'                options.txt              /*'nomh'*
'noml'                options.txt              /*'noml'*
'nomle'               options.txt              /*'nomle'*
'nomod'               options.txt              /*'nomod'*
'nomodeline'            options.txt          /*'nomodeline'*
'nomodelineexpr'          options.txt        /*'nomodelineexpr'*
'nomodifiable'          options.txt          /*'nomodifiable'*
'nomodified'            options.txt          /*'nomodified'*
'nomore'              options.txt              /*'nomore'*
'nomousef'            options.txt              /*'nomousef'*
'nomousefocus'          options.txt          /*'nomousefocus'*
'nomousehide'           options.txt          /*'nomousehide'*
'nonu'                options.txt              /*'nonu'*
'nonumber'            options.txt              /*'nonumber'*
'noodev'              options.txt              /*'noodev'*
'noopendevice'          options.txt          /*'noopendevice'*
'nopaste'             options.txt              /*'nopaste'*
'nopi'                options.txt              /*'nopi'*
'nopreserveindent'        options.txt      /*'nopreserveindent'*
'nopreviewwindow'         options.txt      /*'nopreviewwindow'*
'noprompt'            options.txt              /*'noprompt'*
'nopvw'               options.txt              /*'nopvw'*
'noreadonly'            options.txt          /*'noreadonly'*
'norelativenumber'        options.txt      /*'norelativenumber'*
'noremap'             options.txt              /*'noremap'*
'norestorescreen'         options.txt      /*'norestorescreen'*
'norevins'            options.txt              /*'norevins'*
'nori'                options.txt              /*'nori'*
'norightleft'           options.txt          /*'norightleft'*
'norl'                options.txt              /*'norl'*
'nornu'               options.txt              /*'nornu'*
'noro'                options.txt              /*'noro'*
'nors'                options.txt              /*'nors'*
'noru'                options.txt              /*'noru'*
'noruler'             options.txt              /*'noruler'*
'nosb'                options.txt              /*'nosb'*
```

'nosc' options.txt /*'nosc'*
'noscb' options.txt /*'noscb'*
'noscf' options.txt /*'noscf'*
'noscrollbind' options.txt /*'noscrollbind'*
'noscrollfocus' options.txt /*'noscrollfocus'*
'noscs' options.txt /*'noscs'*
'nosecure' options.txt /*'nosecure'*
'nosft' options.txt /*'nosft'*
'noshellslash' options.txt /*'noshellslash'*
'noshelltemp' options.txt /*'noshelltemp'*
'noshiftround' options.txt /*'noshiftround'*
'noshortname' options.txt /*'noshortname'*
'noshowcmd' options.txt /*'noshowcmd'*
'noshowfulltag' options.txt /*'noshowfulltag'*
'noshowmatch' options.txt /*'noshowmatch'*
'noshowmode' options.txt /*'noshowmode'*
'nosi' options.txt /*'nosi'*
'nosm' options.txt /*'nosm'*
'nosmartcase' options.txt /*'nosmartcase'*
'nosmartindent' options.txt /*'nosmartindent'*
'nosmarttab' options.txt /*'nosmarttab'*
'nosmd' options.txt /*'nosmd'*
'nosn' options.txt /*'nosn'*
'nosol' options.txt /*'nosol'*
'nospell' options.txt /*'nospell'*
'nosplitbelow' options.txt /*'nosplitbelow'*
'nosplitright' options.txt /*'nosplitright'*
'nospr' options.txt /*'nospr'*
'nosr' options.txt /*'nosr'*
'nossl' options.txt /*'nossl'*
'nosta' options.txt /*'nosta'*
'nostartofline' options.txt /*'nostartofline'*
'nostmp' options.txt /*'nostmp'*
'noswapfile' options.txt /*'noswapfile'*
'noswf' options.txt /*'noswf'*
'nota' options.txt /*'nota'*
'notagbsearch' options.txt /*'notagbsearch'*
'notagrelative' options.txt /*'notagrelative'*
'notagstack' options.txt /*'notagstack'*
'notbi' options.txt /*'notbi'*
'notbidi' options.txt /*'notbidi'*
'notbs' options.txt /*'notbs'*
'notermbidi' options.txt /*'notermbidi'*
'noterse' options.txt /*'noterse'*
'notextauto' options.txt /*'notextauto'*
'notextmode' options.txt /*'notextmode'*
'notf' options.txt /*'notf'*
'notgst' options.txt /*'notgst'*
'notildeop' options.txt /*'notildeop'*
'notimeout' options.txt /*'notimeout'*
'notitle' options.txt /*'notitle'*
'noto' options.txt /*'noto'*
'notop' options.txt /*'notop'*
'notr' options.txt /*'notr'*

'nottimeout' options.txt /*'nottimeout'*
'nottybuiltin' options.txt /*'nottybuiltin'*
'nottyfast' options.txt /*'nottyfast'*
'notx' options.txt /*'notx'*
'noudf' options.txt /*'noudf'*
'noundofile' options.txt /*'noundofile'*
'novb' options.txt /*'novb'*
'novice' vi_diff.txt /*'novice'*
'novisualbell' options.txt /*'novisualbell'*
'nowa' options.txt /*'nowa'*
'nowarn' options.txt /*'nowarn'*
'nowb' options.txt /*'nowb'*
'noweirdinvert' options.txt /*'noweirdinvert'*
'nowfh' options.txt /*'nowfh'*
'nowfw' options.txt /*'nowfw'*
'nowic' options.txt /*'nowic'*
'nowildignorecase' options.txt /*'nowildignorecase'*
'nowildmenu' options.txt /*'nowildmenu'*
'nowinfixheight' options.txt /*'nowinfixheight'*
'nowinfixwidth' options.txt /*'nowinfixwidth'*
'nowiv' options.txt /*'nowiv'*
'nowmnu' options.txt /*'nowmnu'*
'nowrap' options.txt /*'nowrap'*
'nowrapscan' options.txt /*'nowrapscan'*
'nowrite' options.txt /*'nowrite'*
'nowriteany' options.txt /*'nowriteany'*
'nowritebackup' options.txt /*'nowritebackup'*
'nows' options.txt /*'nows'*
'nrformats' options.txt /*'nrformats'*
'nu' options.txt /*'nu'*
'number' options.txt /*'number'*
'numberwidth' options.txt /*'numberwidth'*
'nuw' options.txt /*'nuw'*
'odev' options.txt /*'odev'*
'oft' options.txt /*'oft'*
'ofu' options.txt /*'ofu'*
'omnifunc' options.txt /*'omnifunc'*
'op' vi_diff.txt /*'op'*
'open' vi_diff.txt /*'open'*
'opendevice' options.txt /*'opendevice'*
'operatorfunc' options.txt /*'operatorfunc'*
'opfunc' options.txt /*'opfunc'*
'optimize' vi_diff.txt /*'optimize'*
'option' intro.txt /*'option'*
'osfiletype' options.txt /*'osfiletype'*
'pa' options.txt /*'pa'*
'packpath' options.txt /*'packpath'*
'para' options.txt /*'para'*
'paragraphs' options.txt /*'paragraphs'*
'paste' options.txt /*'paste'*
'pastetoggle' options.txt /*'pastetoggle'*
'patchexpr' options.txt /*'patchexpr'*
'patchmode' options.txt /*'patchmode'*
'path' options.txt /*'path'*

```
'pdev'                options.txt              /*'pdev'*
'penc'                options.txt              /*'penc'*
'perldll'              options.txt            /*'perldll'*
'pex'                options.txt              /*'pex'*
'pexpr'               options.txt             /*'pexpr'*
'pfn'                options.txt              /*'pfn'*
'ph'                options.txt              /*'ph'*
'pheader'              options.txt            /*'pheader'*
'pi'                options.txt              /*'pi'*
'pm'                options.txt              /*'pm'*
'pmbcs'               options.txt             /*'pmbcs'*
'pmbfn'               options.txt             /*'pmbfn'*
'popt'                options.txt             /*'popt'*
'pp'                options.txt              /*'pp'*
'preserveindent'          options.txt        /*'preserveindent'*
'previewheight'           options.txt        /*'previewheight'*
'previewpopup'            options.txt        /*'previewpopup'*
'previewwindow'           options.txt        /*'previewwindow'*
'printdevice'             options.txt        /*'printdevice'*
'printencoding'           options.txt        /*'printencoding'*
'printexpr'             options.txt            /*'printexpr'*
'printfont'             options.txt            /*'printfont'*
'printheader'             options.txt          /*'printheader'*
'printmbcharset'           options.txt         /*'printmbcharset'*
'printmbfont'             options.txt          /*'printmbfont'*
'printoptions'            options.txt          /*'printoptions'*
'prompt'               options.txt               /*'prompt'*
'pt'                options.txt                 /*'pt'*
'pumheight'             options.txt            /*'pumheight'*
'pumwidth'              options.txt            /*'pumwidth'*
'pvh'                options.txt                /*'pvh'*
'pvp'                options.txt                /*'pvp'*
'pvw'                options.txt                /*'pvw'*
'pw'                options.txt                 /*'pw'*
'pythondll'             options.txt            /*'pythondll'*
'pythonhome'            options.txt            /*'pythonhome'*
'pythonthreedll'          options.txt          /*'pythonthreedll'*
'pythonthreehome'         options.txt          /*'pythonthreehome'*
'pyx'                options.txt                /*'pyx'*
'pyxversion'            options.txt            /*'pyxversion'*
'qe'                options.txt                 /*'qe'*
'quote                motion.txt                /*'quote*
'quoteescape'           options.txt            /*'quoteescape'*
'rdt'                options.txt                /*'rdt'*
're'                options.txt                 /*'re'*
'readonly'              options.txt            /*'readonly'*
'redraw'               vi_diff.txt              /*'redraw'*
'redrawtime'            options.txt            /*'redrawtime'*
'regexpengine'          options.txt            /*'regexpengine'*
'relativenumber'          options.txt          /*'relativenumber'*
'remap'               options.txt              /*'remap'*
'renderoptions'           options.txt          /*'renderoptions'*
'report'               options.txt             /*'report'*
'restorescreen'           options.txt          /*'restorescreen'*
```

```
'revins'              options.txt              /*'revins'*
'ri'                 options.txt               /*'ri'*
'rightleft'           options.txt             /*'rightleft'*
'rightleftcmd'         options.txt           /*'rightleftcmd'*
'rl'                 options.txt               /*'rl'*
'rlc'                options.txt               /*'rlc'*
'rnu'                options.txt               /*'rnu'*
'ro'                 options.txt               /*'ro'*
'rop'                options.txt               /*'rop'*
'rs'                 options.txt               /*'rs'*
'rtp'                options.txt               /*'rtp'*
'ru'                 options.txt               /*'ru'*
'rubydll'             options.txt             /*'rubydll'*
'ruf'                options.txt               /*'ruf'*
'ruler'               options.txt             /*'ruler'*
'rulerformat'          options.txt           /*'rulerformat'*
'runtimepath'          options.txt           /*'runtimepath'*
'sb'                 options.txt               /*'sb'*
'sbo'                options.txt               /*'sbo'*
'sbr'                options.txt               /*'sbr'*
'sc'                 options.txt               /*'sc'*
'scb'                options.txt               /*'scb'*
'scf'                options.txt               /*'scf'*
'scl'                options.txt               /*'scl'*
'scr'                options.txt               /*'scr'*
'scroll'              options.txt             /*'scroll'*
'scrollbind'           options.txt          /*'scrollbind'*
'scrollfocus'          options.txt          /*'scrollfocus'*
'scrolljump'           options.txt          /*'scrolljump'*
'scrolloff'            options.txt          /*'scrolloff'*
'scrollopt'            options.txt          /*'scrollopt'*
'scs'                options.txt               /*'scs'*
'sect'               options.txt               /*'sect'*
'sections'            options.txt            /*'sections'*
'secure'             options.txt              /*'secure'*
'sel'                options.txt               /*'sel'*
'selection'           options.txt            /*'selection'*
'selectmode'          options.txt            /*'selectmode'*
'sessionoptions'       options.txt         /*'sessionoptions'*
'sft'                options.txt               /*'sft'*
'sh'                 options.txt               /*'sh'*
'shcf'               options.txt               /*'shcf'*
'shell'              options.txt               /*'shell'*
'shellcmdflag'         options.txt           /*'shellcmdflag'*
'shellpipe'           options.txt            /*'shellpipe'*
'shellquote'          options.txt            /*'shellquote'*
'shellredir'          options.txt            /*'shellredir'*
'shellslash'          options.txt            /*'shellslash'*
'shelltemp'           options.txt            /*'shelltemp'*
'shelltype'           options.txt            /*'shelltype'*
'shellxescape'         options.txt           /*'shellxescape'*
'shellxquote'          options.txt           /*'shellxquote'*
'shiftround'          options.txt            /*'shiftround'*
'shiftwidth'          options.txt            /*'shiftwidth'*
```

```
'shm'                options.txt              /*'shm'*
'shortmess'           options.txt             /*'shortmess'*
'shortname'           options.txt             /*'shortname'*
'showbreak'           options.txt             /*'showbreak'*
'showcmd'            options.txt              /*'showcmd'*
'showfulltag'         options.txt            /*'showfulltag'*
'showmatch'           options.txt             /*'showmatch'*
'showmode'           options.txt              /*'showmode'*
'showtabline'         options.txt            /*'showtabline'*
'shq'                options.txt              /*'shq'*
'si'                 options.txt              /*'si'*
'sidescroll'          options.txt            /*'sidescroll'*
'sidescrolloff'        options.txt          /*'sidescrolloff'*
'signcolumn'          options.txt            /*'signcolumn'*
'siso'               options.txt              /*'siso'*
'sj'                 options.txt              /*'sj'*
'slm'                options.txt              /*'slm'*
'slow'               vi_diff.txt              /*'slow'*
'slowopen'            vi_diff.txt            /*'slowopen'*
'sm'                 options.txt              /*'sm'*
'smartcase'           options.txt            /*'smartcase'*
'smartindent'         options.txt            /*'smartindent'*
'smarttab'           options.txt             /*'smarttab'*
'smc'                options.txt              /*'smc'*
'smd'                options.txt              /*'smd'*
'sn'                 options.txt              /*'sn'*
'so'                 options.txt              /*'so'*
'softtabstop'         options.txt            /*'softtabstop'*
'sol'                options.txt              /*'sol'*
'sourceany'           vi_diff.txt            /*'sourceany'*
'sp'                 options.txt              /*'sp'*
'spc'                options.txt              /*'spc'*
'spell'              options.txt             /*'spell'*
'spellcapcheck'        options.txt          /*'spellcapcheck'*
'spellfile'           options.txt            /*'spellfile'*
'spelllang'           options.txt            /*'spelllang'*
'spellsuggest'         options.txt          /*'spellsuggest'*
'spf'                options.txt              /*'spf'*
'spl'                options.txt              /*'spl'*
'splitbelow'          options.txt            /*'splitbelow'*
'splitright'          options.txt            /*'splitright'*
'spr'                options.txt              /*'spr'*
'sps'                options.txt              /*'sps'*
'sr'                 options.txt              /*'sr'*
'srr'                options.txt              /*'srr'*
'ss'                 options.txt              /*'ss'*
'ssl'                options.txt              /*'ssl'*
'ssop'               options.txt             /*'ssop'*
'st'                 options.txt              /*'st'*
'sta'                options.txt              /*'sta'*
'stal'               options.txt              /*'stal'*
'startofline'         options.txt            /*'startofline'*
'statusline'          options.txt            /*'statusline'*
'stl'                options.txt              /*'stl'*
```

```
'stmp'              options.txt              /*'stmp'*
'sts'               options.txt              /*'sts'*
'su'                options.txt              /*'su'*
'sua'               options.txt              /*'sua'*
'suffixes'            options.txt            /*'suffixes'*
'suffixesadd'          options.txt           /*'suffixesadd'*
'sw'                options.txt              /*'sw'*
'swapfile'            options.txt            /*'swapfile'*
'swapsync'            options.txt            /*'swapsync'*
'swb'               options.txt              /*'swb'*
'swf'               options.txt              /*'swf'*
'switchbuf'            options.txt           /*'switchbuf'*
'sws'               options.txt              /*'sws'*
'sxe'               options.txt              /*'sxe'*
'sxq'               options.txt              /*'sxq'*
'syn'               options.txt              /*'syn'*
'synmaxcol'            options.txt           /*'synmaxcol'*
'syntax'             options.txt             /*'syntax'*
't_#2'               term.txt                /*'t_#2'*
't_#4'               term.txt                /*'t_#4'*
't_%1'               term.txt                /*'t_%1'*
't_%i'               term.txt                /*'t_%i'*
't_&8'               term.txt                /*'t_&8'*
't_8b'               term.txt                /*'t_8b'*
't_8f'               term.txt                /*'t_8f'*
't_@7'               term.txt                /*'t_@7'*
't_AB'               term.txt                /*'t_AB'*
't_AF'               term.txt                /*'t_AF'*
't_AL'               term.txt                /*'t_AL'*
't_BD'               term.txt                /*'t_BD'*
't_BE'               term.txt                /*'t_BE'*
't_CS'               term.txt                /*'t_CS'*
't_CV'               term.txt                /*'t_CV'*
't_Ce'               term.txt                /*'t_Ce'*
't_Co'               term.txt                /*'t_Co'*
't_Cs'               term.txt                /*'t_Cs'*
't_DL'               term.txt                /*'t_DL'*
't_EC'               term.txt                /*'t_EC'*
't_EI'               term.txt                /*'t_EI'*
't_F1'               term.txt                /*'t_F1'*
't_F2'               term.txt                /*'t_F2'*
't_F3'               term.txt                /*'t_F3'*
't_F4'               term.txt                /*'t_F4'*
't_F5'               term.txt                /*'t_F5'*
't_F6'               term.txt                /*'t_F6'*
't_F7'               term.txt                /*'t_F7'*
't_F8'               term.txt                /*'t_F8'*
't_F9'               term.txt                /*'t_F9'*
't_GP'               term.txt                /*'t_GP'*
't_IE'               term.txt                /*'t_IE'*
't_IS'               term.txt                /*'t_IS'*
't_K1'               term.txt                /*'t_K1'*
't_K3'               term.txt                /*'t_K3'*
't_K4'               term.txt                /*'t_K4'*
```

```
't_K5'          term.txt          /*'t_K5'*
't_K6'          term.txt          /*'t_K6'*
't_K7'          term.txt          /*'t_K7'*
't_K8'          term.txt          /*'t_K8'*
't_K9'          term.txt          /*'t_K9'*
't_KA'          term.txt          /*'t_KA'*
't_KB'          term.txt          /*'t_KB'*
't_KC'          term.txt          /*'t_KC'*
't_KD'          term.txt          /*'t_KD'*
't_KE'          term.txt          /*'t_KE'*
't_KF'          term.txt          /*'t_KF'*
't_KG'          term.txt          /*'t_KG'*
't_KH'          term.txt          /*'t_KH'*
't_KI'          term.txt          /*'t_KI'*
't_KJ'          term.txt          /*'t_KJ'*
't_KK'          term.txt          /*'t_KK'*
't_KL'          term.txt          /*'t_KL'*
't_PE'          term.txt          /*'t_PE'*
't_PS'          term.txt          /*'t_PS'*
't_RB'          term.txt          /*'t_RB'*
't_RC'          term.txt          /*'t_RC'*
't_RF'          term.txt          /*'t_RF'*
't_RI'          term.txt          /*'t_RI'*
't_RS'          term.txt          /*'t_RS'*
't_RT'          term.txt          /*'t_RT'*
't_RV'          term.txt          /*'t_RV'*
't_Ri'          term.txt          /*'t_Ri'*
't_SC'          term.txt          /*'t_SC'*
't_SH'          term.txt          /*'t_SH'*
't_SI'          term.txt          /*'t_SI'*
't_SR'          term.txt          /*'t_SR'*
't_ST'          term.txt          /*'t_ST'*
't_Sb'          term.txt          /*'t_Sb'*
't_Sf'          term.txt          /*'t_Sf'*
't_Si'          term.txt          /*'t_Si'*
't_TE'          term.txt          /*'t_TE'*
't_TI'          term.txt          /*'t_TI'*
't_Te'          term.txt          /*'t_Te'*
't_Ts'          term.txt          /*'t_Ts'*
't_VS'          term.txt          /*'t_VS'*
't_WP'          term.txt          /*'t_WP'*
't_WS'          term.txt          /*'t_WS'*
't_ZH'          term.txt          /*'t_ZH'*
't_ZR'          term.txt          /*'t_ZR'*
't_al'          term.txt          /*'t_al'*
't_bc'          term.txt          /*'t_bc'*
't_cd'          term.txt          /*'t_cd'*
't_ce'          term.txt          /*'t_ce'*
't_cl'          term.txt          /*'t_cl'*
't_cm'          term.txt          /*'t_cm'*
't_cs'          term.txt          /*'t_cs'*
't_da'          term.txt          /*'t_da'*
't_db'          term.txt          /*'t_db'*
't_dl'          term.txt          /*'t_dl'*
```

```
't_fs'                term.txt                /*'t_fs'*
't_k1'                term.txt                /*'t_k1'*
't_k2'                term.txt                /*'t_k2'*
't_k3'                term.txt                /*'t_k3'*
't_k4'                term.txt                /*'t_k4'*
't_k5'                term.txt                /*'t_k5'*
't_k6'                term.txt                /*'t_k6'*
't_k7'                term.txt                /*'t_k7'*
't_k8'                term.txt                /*'t_k8'*
't_k9'                term.txt                /*'t_k9'*
't_k;'                term.txt                /*'t_k;'*
't_kB'                term.txt                /*'t_kB'*
't_kD'                term.txt                /*'t_kD'*
't_kI'                term.txt                /*'t_kI'*
't_kN'                term.txt                /*'t_kN'*
't_kP'                term.txt                /*'t_kP'*
't_kb'                term.txt                /*'t_kb'*
't_kd'                term.txt                /*'t_kd'*
't_ke'                term.txt                /*'t_ke'*
't_kh'                term.txt                /*'t_kh'*
't_kl'                term.txt                /*'t_kl'*
't_kr'                term.txt                /*'t_kr'*
't_ks'                term.txt                /*'t_ks'*
't_ku'                term.txt                /*'t_ku'*
't_le'                term.txt                /*'t_le'*
't_mb'                term.txt                /*'t_mb'*
't_md'                term.txt                /*'t_md'*
't_me'                term.txt                /*'t_me'*
't_mr'                term.txt                /*'t_mr'*
't_ms'                term.txt                /*'t_ms'*
't_nd'                term.txt                /*'t_nd'*
't_op'                term.txt                /*'t_op'*
't_se'                term.txt                /*'t_se'*
't_so'                term.txt                /*'t_so'*
't_sr'                term.txt                /*'t_sr'*
't_star7'              term.txt              /*'t_star7'*
't_te'                term.txt                /*'t_te'*
't_ti'                term.txt                /*'t_ti'*
't_ts'                term.txt                /*'t_ts'*
't_u7'                term.txt                /*'t_u7'*
't_ue'                term.txt                /*'t_ue'*
't_us'                term.txt                /*'t_us'*
't_ut'                term.txt                /*'t_ut'*
't_vb'                term.txt                /*'t_vb'*
't_ve'                term.txt                /*'t_ve'*
't_vi'                term.txt                /*'t_vi'*
't_vs'                term.txt                /*'t_vs'*
't_xn'                term.txt                /*'t_xn'*
't_xs'                term.txt                /*'t_xs'*
'ta'                options.txt                 /*'ta'*
'tabline'             options.txt              /*'tabline'*
'tabpagemax'           options.txt           /*'tabpagemax'*
'tabstop'             options.txt              /*'tabstop'*
'tag'                 options.txt               /*'tag'*
```

```
'tagbsearch'            options.txt          /*'tagbsearch'*
'tagcase'              options.txt           /*'tagcase'*
'tagfunc'              options.txt           /*'tagfunc'*
'taglength'             options.txt          /*'taglength'*
'tagrelative'            options.txt         /*'tagrelative'*
'tags'                 options.txt             /*'tags'*
'tagstack'              options.txt          /*'tagstack'*
'tal'                  options.txt             /*'tal'*
'tb'                   options.txt              /*'tb'*
'tbi'                  options.txt             /*'tbi'*
'tbidi'                 options.txt            /*'tbidi'*
'tbis'                 options.txt             /*'tbis'*
'tbs'                  options.txt              /*'tbs'*
'tc'                   options.txt              /*'tc'*
'tcldll'                options.txt           /*'tcldll'*
'tenc'                 options.txt             /*'tenc'*
'term'                 options.txt             /*'term'*
'termbidi'              options.txt          /*'termbidi'*
'termencoding'           options.txt        /*'termencoding'*
'termguicolors'          options.txt        /*'termguicolors'*
'termwinkey'            options.txt          /*'termwinkey'*
'termwinscroll'          options.txt        /*'termwinscroll'*
'termwinsize'           options.txt          /*'termwinsize'*
'termwintype'           options.txt          /*'termwintype'*
'terse'                options.txt             /*'terse'*
'textauto'              options.txt          /*'textauto'*
'textmode'              options.txt          /*'textmode'*
'textwidth'             options.txt          /*'textwidth'*
'tf'                   options.txt              /*'tf'*
'tfu'                  options.txt              /*'tfu'*
'tgc'                  options.txt              /*'tgc'*
'tgst'                 options.txt              /*'tgst'*
'thesaurus'             options.txt          /*'thesaurus'*
'tildeop'              options.txt           /*'tildeop'*
'timeout'              options.txt           /*'timeout'*
'timeoutlen'            options.txt          /*'timeoutlen'*
'title'                options.txt             /*'title'*
'titlelen'              options.txt          /*'titlelen'*
'titleold'              options.txt          /*'titleold'*
'titlestring'            options.txt         /*'titlestring'*
'tl'                   options.txt              /*'tl'*
'tm'                   options.txt              /*'tm'*
'to'                   options.txt              /*'to'*
'toolbar'               options.txt           /*'toolbar'*
'toolbariconsize'          options.txt      /*'toolbariconsize'*
'top'                  options.txt              /*'top'*
'tpm'                  options.txt              /*'tpm'*
'tr'                   options.txt              /*'tr'*
'ts'                   options.txt              /*'ts'*
'tsl'                  options.txt              /*'tsl'*
'tsr'                  options.txt              /*'tsr'*
'ttimeout'              options.txt          /*'ttimeout'*
'ttimeoutlen'            options.txt        /*'ttimeoutlen'*
'ttm'                  options.txt              /*'ttm'*
```

```
'tty'                options.txt              /*'tty'*
'ttybuiltin'           options.txt           /*'ttybuiltin'*
'ttyfast'            options.txt             /*'ttyfast'*
'ttym'               options.txt              /*'ttym'*
'ttymouse'             options.txt           /*'ttymouse'*
'ttyscroll'            options.txt           /*'ttyscroll'*
'ttytype'            options.txt             /*'ttytype'*
'tw'                options.txt              /*'tw'*
'twk'               options.txt              /*'twk'*
'tws'               options.txt              /*'tws'*
'twsl'               options.txt             /*'twsl'*
'twt'               options.txt              /*'twt'*
'tx'                options.txt              /*'tx'*
'uc'                options.txt              /*'uc'*
'udf'               options.txt              /*'udf'*
'udir'               options.txt             /*'udir'*
'ul'                options.txt              /*'ul'*
'undodir'              options.txt           /*'undodir'*
'undofile'             options.txt           /*'undofile'*
'undolevels'            options.txt          /*'undolevels'*
'undoreload'            options.txt          /*'undoreload'*
'updatecount'            options.txt         /*'updatecount'*
'updatetime'            options.txt          /*'updatetime'*
'ur'                options.txt              /*'ur'*
'ut'                options.txt              /*'ut'*
'varsofttabstop'         options.txt       /*'varsofttabstop'*
'vartabstop'           options.txt           /*'vartabstop'*
'vb'                options.txt              /*'vb'*
'vbs'               options.txt              /*'vbs'*
'vdir'               options.txt             /*'vdir'*
've'                options.txt              /*'ve'*
'verbose'              options.txt           /*'verbose'*
'verbosefile'            options.txt         /*'verbosefile'*
'vfile'              options.txt              /*'vfile'*
'vi'                options.txt              /*'vi'*
'viewdir'              options.txt           /*'viewdir'*
'viewoptions'            options.txt         /*'viewoptions'*
'vif'               options.txt              /*'vif'*
'viminfo'              options.txt           /*'viminfo'*
'viminfofile'            options.txt         /*'viminfofile'*
'virtualedit'            options.txt         /*'virtualedit'*
'visualbell'             options.txt         /*'visualbell'*
'vop'               options.txt              /*'vop'*
'vsts'              options.txt              /*'vsts'*
'vts'               options.txt              /*'vts'*
'w1200'              vi_diff.txt             /*'w1200'*
'w300'               vi_diff.txt             /*'w300'*
'w9600'              vi_diff.txt             /*'w9600'*
'wa'                options.txt              /*'wa'*
'wak'               options.txt              /*'wak'*
'warn'               options.txt             /*'warn'*
'wb'                options.txt              /*'wb'*
'wc'                options.txt              /*'wc'*
'wcm'               options.txt              /*'wcm'*
```

'wcr'	options.txt	/*'wcr'*
'wd'	options.txt	/*'wd'*
'weirdinvert'	options.txt	/*'weirdinvert'*
'wfh'	options.txt	/*'wfh'*
'wfw'	options.txt	/*'wfw'*
'wh'	options.txt	/*'wh'*
'whichwrap'	options.txt	/*'whichwrap'*
'wi'	options.txt	/*'wi'*
'wic'	options.txt	/*'wic'*
'wig'	options.txt	/*'wig'*
'wildchar'	options.txt	/*'wildchar'*
'wildcharm'	options.txt	/*'wildcharm'*
'wildignore'	options.txt	/*'wildignore'*
'wildignorecase'	options.txt	/*'wildignorecase'*
'wildmenu'	options.txt	/*'wildmenu'*
'wildmode'	options.txt	/*'wildmode'*
'wildoptions'	options.txt	/*'wildoptions'*
'wim'	options.txt	/*'wim'*
'winaltkeys'	options.txt	/*'winaltkeys'*
'wincolor'	options.txt	/*'wincolor'*
'window'	options.txt	/*'window'*
'winfixheight'	options.txt	/*'winfixheight'*
'winfixwidth'	options.txt	/*'winfixwidth'*
'winheight'	options.txt	/*'winheight'*
'winminheight'	options.txt	/*'winminheight'*
'winminwidth'	options.txt	/*'winminwidth'*
'winptydll'	options.txt	/*'winptydll'*
'winwidth'	options.txt	/*'winwidth'*
'wiv'	options.txt	/*'wiv'*
'wiw'	options.txt	/*'wiw'*
'wm'	options.txt	/*'wm'*
'wmh'	options.txt	/*'wmh'*
'wmnu'	options.txt	/*'wmnu'*
'wmw'	options.txt	/*'wmw'*
'wop'	options.txt	/*'wop'*
'wrap'	options.txt	/*'wrap'*
'wrapmargin'	options.txt	/*'wrapmargin'*
'wrapscan'	options.txt	/*'wrapscan'*
'write'	options.txt	/*'write'*
'writeany'	options.txt	/*'writeany'*
'writebackup'	options.txt	/*'writebackup'*
'writedelay'	options.txt	/*'writedelay'*
'ws'	options.txt	/*'ws'*
'ww'	options.txt	/*'ww'*
'{	motion.txt	/*'{*
'}	motion.txt	/*'}*
(	motion.txt	/*(*
)	motion.txt	/*)*
+	motion.txt	/*+*
++bad	editing.txt	/*++bad*
++bin	editing.txt	/*++bin*
++builtin_terms	various.txt	/*++builtin_terms*
++edit	editing.txt	/*++edit*
++enc	editing.txt	/*++enc*

++ff	editing.txt	/*++ff*
++nobin	editing.txt	/*++nobin*
++opt	editing.txt	/*++opt*
+ARP	various.txt	/*+ARP*
+GUI_Athena	various.txt	/*+GUI_Athena*
+GUI_GTK	various.txt	/*+GUI_GTK*
+GUI_Motif	various.txt	/*+GUI_Motif*
+GUI_Photon	various.txt	/*+GUI_Photon*
+GUI_neXtaw	various.txt	/*+GUI_neXtaw*
+X11	various.txt	/*+X11*
+acl	various.txt	/*+acl*
+arabic	various.txt	/*+arabic*
+autocmd	various.txt	/*+autocmd*
+autoservername	various.txt	/*+autoservername*
+balloon_eval	various.txt	/*+balloon_eval*
+balloon_eval_term	various.txt	/*+balloon_eval_term*
+browse	various.txt	/*+browse*
+builtin_terms	various.txt	/*+builtin_terms*
+byte_offset	various.txt	/*+byte_offset*
+channel	various.txt	/*+channel*
+cindent	various.txt	/*+cindent*
+clientserver	various.txt	/*+clientserver*
+clipboard	various.txt	/*+clipboard*
+clipboard_working	various.txt	/*+clipboard_working*
+cmd	editing.txt	/*+cmd*
+cmdline_compl	various.txt	/*+cmdline_compl*
+cmdline_hist	various.txt	/*+cmdline_hist*
+cmdline_info	various.txt	/*+cmdline_info*
+comments	various.txt	/*+comments*
+conceal	various.txt	/*+conceal*
+cryptv	various.txt	/*+cryptv*
+cscope	various.txt	/*+cscope*
+cursorbind	various.txt	/*+cursorbind*
+cursorshape	various.txt	/*+cursorshape*
+debug	various.txt	/*+debug*
+dialog_con	various.txt	/*+dialog_con*
+dialog_con_gui	various.txt	/*+dialog_con_gui*
+dialog_gui	various.txt	/*+dialog_gui*
+diff	various.txt	/*+diff*
+digraphs	various.txt	/*+digraphs*
+directx	various.txt	/*+directx*
+dnd	various.txt	/*+dnd*
+emacs_tags	various.txt	/*+emacs_tags*
+eval	various.txt	/*+eval*
+ex_extra	various.txt	/*+ex_extra*
+extra_search	various.txt	/*+extra_search*
+farsi	various.txt	/*+farsi*
+feature-list	various.txt	/*+feature-list*
+file_in_path	various.txt	/*+file_in_path*
+find_in_path	various.txt	/*+find_in_path*
+float	various.txt	/*+float*
+folding	various.txt	/*+folding*
+footer	various.txt	/*+footer*
+fork	various.txt	/*+fork*

+gettext various.txt /*+gettext*
+hangul_input various.txt /*+hangul_input*
+iconv various.txt /*+iconv*
+iconv/dyn various.txt /*+iconv\/dyn*
+insert_expand various.txt /*+insert_expand*
+job various.txt /*+job*
+jumplist various.txt /*+jumplist*
+keymap various.txt /*+keymap*
+lambda various.txt /*+lambda*
+langmap various.txt /*+langmap*
+libcall various.txt /*+libcall*
+linebreak various.txt /*+linebreak*
+lispindent various.txt /*+lispindent*
+listcmds various.txt /*+listcmds*
+localmap various.txt /*+localmap*
+lua various.txt /*+lua*
+lua/dyn various.txt /*+lua\/dyn*
+menu various.txt /*+menu*
+mksession various.txt /*+mksession*
+modify_fname various.txt /*+modify_fname*
+mouse various.txt /*+mouse*
+mouse_dec various.txt /*+mouse_dec*
+mouse_gpm various.txt /*+mouse_gpm*
+mouse_jsbterm various.txt /*+mouse_jsbterm*
+mouse_netterm various.txt /*+mouse_netterm*
+mouse_pterm various.txt /*+mouse_pterm*
+mouse_sgr various.txt /*+mouse_sgr*
+mouse_sysmouse various.txt /*+mouse_sysmouse*
+mouse_urxvt various.txt /*+mouse_urxvt*
+mouse_xterm various.txt /*+mouse_xterm*
+mouseshape various.txt /*+mouseshape*
+multi_byte various.txt /*+multi_byte*
+multi_byte_ime various.txt /*+multi_byte_ime*
+multi_lang various.txt /*+multi_lang*
+mzscheme various.txt /*+mzscheme*
+mzscheme/dyn various.txt /*+mzscheme\/dyn*
+netbeans_intg various.txt /*+netbeans_intg*
+num64 various.txt /*+num64*
+ole various.txt /*+ole*
+packages various.txt /*+packages*
+path_extra various.txt /*+path_extra*
+perl various.txt /*+perl*
+perl/dyn various.txt /*+perl\/dyn*
+persistent_undo various.txt /*+persistent_undo*
+popupwin various.txt /*+popupwin*
+postscript various.txt /*+postscript*
+printer various.txt /*+printer*
+profile various.txt /*+profile*
+python various.txt /*+python*
+python/dyn various.txt /*+python\/dyn*
+python3 various.txt /*+python3*
+python3/dyn various.txt /*+python3\/dyn*
+quickfix various.txt /*+quickfix*
+reltime various.txt /*+reltime*

+rightleft various.txt /*+rightleft*
+ruby various.txt /*+ruby*
+ruby/dyn various.txt /*+ruby\/dyn*
+scrollbind various.txt /*+scrollbind*
+signs various.txt /*+signs*
+smartindent various.txt /*+smartindent*
+sound various.txt /*+sound*
+spell various.txt /*+spell*
+startuptime various.txt /*+startuptime*
+statusline various.txt /*+statusline*
+sun_workshop various.txt /*+sun_workshop*
+syntax various.txt /*+syntax*
+system() various.txt /*+system()*
+tag_any_white various.txt /*+tag_any_white*
+tag_binary various.txt /*+tag_binary*
+tag_old_static various.txt /*+tag_old_static*
+tcl various.txt /*+tcl*
+tcl/dyn various.txt /*+tcl\/dyn*
+termguicolors various.txt /*+termguicolors*
+terminal various.txt /*+terminal*
+terminfo various.txt /*+terminfo*
+termresponse various.txt /*+termresponse*
+textobjects various.txt /*+textobjects*
+textprop various.txt /*+textprop*
+tgetent various.txt /*+tgetent*
+timers various.txt /*+timers*
+title various.txt /*+title*
+toolbar various.txt /*+toolbar*
+unix eval.txt /*+unix*
+user_commands various.txt /*+user_commands*
+vartabs various.txt /*+vartabs*
+vertsplit various.txt /*+vertsplit*
+viminfo various.txt /*+viminfo*
+virtualedit various.txt /*+virtualedit*
+visual various.txt /*+visual*
+visualextra various.txt /*+visualextra*
+vreplace various.txt /*+vreplace*
+vtp various.txt /*+vtp*
+wildignore various.txt /*+wildignore*
+wildmenu various.txt /*+wildmenu*
+windows various.txt /*+windows*
+writebackup various.txt /*+writebackup*
+xfontset various.txt /*+xfontset*
+xim various.txt /*+xim*
+xpm various.txt /*+xpm*
+xpm_w32 various.txt /*+xpm_w32*
+xsmp various.txt /*+xsmp*
+xsmp_interact various.txt /*+xsmp_interact*
+xterm_clipboard various.txt /*+xterm_clipboard*
+xterm_save various.txt /*+xterm_save*
, motion.txt /*,*
- motion.txt /*-*
-+ starting.txt /*-+*
-+/ starting.txt /*-+\/*

-+c starting.txt /*-+c*
-+reverse gui_x11.txt /*-+reverse*
-+rv gui_x11.txt /*-+rv*
-- starting.txt /*--*
--- starting.txt /*---*
--clean starting.txt /*--clean*
--cmd starting.txt /*--cmd*
--echo-wid starting.txt /*--echo-wid*
--help starting.txt /*--help*
--literal starting.txt /*--literal*
--nofork starting.txt /*--nofork*
--noplugin starting.txt /*--noplugin*
--not-a-term starting.txt /*--not-a-term*
--remote remote.txt /*--remote*
--remote-expr remote.txt /*--remote-expr*
--remote-send remote.txt /*--remote-send*
--remote-silent remote.txt /*--remote-silent*
--remote-tab remote.txt /*--remote-tab*
--remote-tab-silent remote.txt /*--remote-tab-silent*
--remote-tab-wait remote.txt /*--remote-tab-wait*
--remote-tab-wait-silent remote.txt /*--remote-tab-wait-silent*
--remote-wait remote.txt /*--remote-wait*
--remote-wait-silent remote.txt /*--remote-wait-silent*
--role starting.txt /*--role*
--serverlist remote.txt /*--serverlist*
--servername remote.txt /*--servername*
--socketid starting.txt /*--socketid*
--startuptime starting.txt /*--startuptime*
--ttyfail starting.txt /*--ttyfail*
--version starting.txt /*--version*
--windowid starting.txt /*--windowid*
-> eval.txt /*->*
-? starting.txt /*-?*
-A starting.txt /*-A*
-C starting.txt /*-C*
-D starting.txt /*-D*
-E starting.txt /*-E*
-F starting.txt /*-F*
-H starting.txt /*-H*
-L starting.txt /*-L*
-M starting.txt /*-M*
-N starting.txt /*-N*
-O starting.txt /*-O*
-P starting.txt /*-P*
-R starting.txt /*-R*
-S starting.txt /*-S*
-T starting.txt /*-T*
-U starting.txt /*-U*
-V starting.txt /*-V*
-W starting.txt /*-W*
-X starting.txt /*-X*
-Z starting.txt /*-Z*
-b starting.txt /*-b*
-background gui_x11.txt /*-background*

-bg gui_x11.txt /*-bg*
-boldfont gui_x11.txt /*-boldfont*
-borderwidth gui_x11.txt /*-borderwidth*
-bw gui_x11.txt /*-bw*
-c starting.txt /*-c*
-d starting.txt /*-d*
-dev starting.txt /*-dev*
-display gui_x11.txt /*-display*
-e starting.txt /*-e*
-f starting.txt /*-f*
-fg gui_x11.txt /*-fg*
-file starting.txt /*-file*
-fn gui_x11.txt /*-fn*
-font gui_x11.txt /*-font*
-foreground gui_x11.txt /*-foreground*
-g starting.txt /*-g*
-geom gui_x11.txt /*-geom*
-geometry gui_x11.txt /*-geometry*
-geometry-example gui_x11.txt /*-geometry-example*
-gui gui_x11.txt /*-gui*
-h starting.txt /*-h*
-i starting.txt /*-i*
-iconic gui_x11.txt /*-iconic*
-italicfont gui_x11.txt /*-italicfont*
-l starting.txt /*-l*
-m starting.txt /*-m*
-menufont gui_x11.txt /*-menufont*
-menufontset gui_x11.txt /*-menufontset*
-menuheight gui_x11.txt /*-menuheight*
-mf gui_x11.txt /*-mf*
-mh gui_x11.txt /*-mh*
-n starting.txt /*-n*
-nb starting.txt /*-nb*
-o starting.txt /*-o*
-p starting.txt /*-p*
-q starting.txt /*-q*
-qf starting.txt /*-qf*
-r starting.txt /*-r*
-register if_ole.txt /*-register*
-reverse gui_x11.txt /*-reverse*
-rv gui_x11.txt /*-rv*
-s starting.txt /*-s*
-s-ex starting.txt /*-s-ex*
-scrollbarwidth gui_x11.txt /*-scrollbarwidth*
-silent if_ole.txt /*-silent*
-sw gui_x11.txt /*-sw*
-t starting.txt /*-t*
-tag starting.txt /*-tag*
-u starting.txt /*-u*
-ul gui_x11.txt /*-ul*
-unregister if_ole.txt /*-unregister*
-v starting.txt /*-v*
-vim starting.txt /*-vim*
-w starting.txt /*-w*

-w_nr	starting.txt	/*-w_nr*
-x	starting.txt	/*-x*
-xrm	gui_x11.txt	/*-xrm*
-y	starting.txt	/*-y*
.	repeat.txt	/*.*
...	eval.txt	/*...*
.Xdefaults	gui_x11.txt	/*.Xdefaults*
.aff	spell.txt	/*.aff*
.dic	spell.txt	/*.dic*
.exrc	starting.txt	/*.exrc*
.gvimrc	gui.txt	/*.gvimrc*
.netrwbook	pi_netrw.txt	/*.netrwbook*
.netrwhist	pi_netrw.txt	/*.netrwhist*
.vimrc	starting.txt	/*.vimrc*
/	pattern.txt	/*\/*
/$	pattern.txt	/*\/$*
/.	pattern.txt	/*\/.*
//	version7.txt	/*\/\/*
//;	pattern.txt	/*\/\/;*
/<CR>	pattern.txt	/*\/<CR>*
/[[.	pattern.txt	/*\/[[.*
/[[=	pattern.txt	/*\/[[=*
/[\n]	pattern.txt	/*\/[\\n]*
/[]	pattern.txt	/*\/[]*
/\	pattern.txt	/*\/*
/\$	pattern.txt	/*\/\\$*
/\%#	pattern.txt	/*\/\\%#*
/\%#=	pattern.txt	/*\/\\%#=*
/\%$	pattern.txt	/*\/\\%$*
/\%'m	pattern.txt	/*\/\\%'m*
/\%(	pattern.txt	/*\/\\%(*
/\%(\)	pattern.txt	/*\/\\%(\\)*
/\%<'m	pattern.txt	/*\/\\%<'m*
/\%<c	pattern.txt	/*\/\\%<c*
/\%<l	pattern.txt	/*\/\\%<l*
/\%<v	pattern.txt	/*\/\\%<v*
/\%>'m	pattern.txt	/*\/\\%>'m*
/\%>c	pattern.txt	/*\/\\%>c*
/\%>l	pattern.txt	/*\/\\%>l*
/\%>v	pattern.txt	/*\/\\%>v*
/\%C	pattern.txt	/*\/\\%C*
/\%U	pattern.txt	/*\/\\%U*
/\%V	pattern.txt	/*\/\\%V*
/\%[]	pattern.txt	/*\/\\%[]*
/\%^	pattern.txt	/*\/\\%^*
/\%c	pattern.txt	/*\/\\%c*
/\%d	pattern.txt	/*\/\\%d*
/\%l	pattern.txt	/*\/\\%l*
/\%o	pattern.txt	/*\/\\%o*
/\%u	pattern.txt	/*\/\\%u*
/\%v	pattern.txt	/*\/\\%v*
/\%x	pattern.txt	/*\/\\%x*
/\&	pattern.txt	/*\/\\&*
/\(	pattern.txt	/*\/\\(*

```
/\(\)                pattern.txt              /*\/\\(\\)*
/\)                  pattern.txt               /*\/\\)*
/\+                  pattern.txt               /*\/\\+*
/\.                  pattern.txt               /*\/\\.*
/\1                  pattern.txt               /*\/\\1*
/\2                  pattern.txt               /*\/\\2*
/\3                  pattern.txt               /*\/\\3*
/\9                  pattern.txt               /*\/\\9*
/\<                  pattern.txt               /*\/\\<*
/\=                  pattern.txt               /*\/\\=*
/\>                  pattern.txt               /*\/\\>*
/\?                  pattern.txt               /*\/\\?*
/\@!                 pattern.txt              /*\/\\@!*
/\@<!                pattern.txt             /*\/\\@<!*
/\@<=                pattern.txt             /*\/\\@<=*
/\@=                 pattern.txt              /*\/\\@=*
/\@>                 pattern.txt              /*\/\\@>*
/\A                  pattern.txt               /*\/\\A*
/\C                  pattern.txt               /*\/\\C*
/\D                  pattern.txt               /*\/\\D*
/\F                  pattern.txt               /*\/\\F*
/\H                  pattern.txt               /*\/\\H*
/\I                  pattern.txt               /*\/\\I*
/\K                  pattern.txt               /*\/\\K*
/\L                  pattern.txt               /*\/\\L*
/\M                  pattern.txt               /*\/\\M*
/\O                  pattern.txt               /*\/\\O*
/\P                  pattern.txt               /*\/\\P*
/\S                  pattern.txt               /*\/\\S*
/\U                  pattern.txt               /*\/\\U*
/\V                  pattern.txt               /*\/\\V*
/\W                  pattern.txt               /*\/\\W*
/\X                  pattern.txt               /*\/\\X*
/\Z                  pattern.txt               /*\/\\Z*
/\[]                 pattern.txt              /*\/\\[]*
/\\                 pattern.txt              /*\/\\\*
/\]                  pattern.txt               /*\/\\]*
/\^                  pattern.txt               /*\/\\^*
/\_                  pattern.txt               /*\/\\_*
/\_$                 pattern.txt              /*\/\\_$*
/\_.                 pattern.txt              /*\/\\_.*
/\_A                 pattern.txt              /*\/\\_A*
/\_D                 pattern.txt              /*\/\\_D*
/\_F                 pattern.txt              /*\/\\_F*
/\_H                 pattern.txt              /*\/\\_H*
/\_I                 pattern.txt              /*\/\\_I*
/\_K                 pattern.txt              /*\/\\_K*
/\_L                 pattern.txt              /*\/\\_L*
/\_O                 pattern.txt              /*\/\\_O*
/\_P                 pattern.txt              /*\/\\_P*
/\_S                 pattern.txt              /*\/\\_S*
/\_U                 pattern.txt              /*\/\\_U*
/\_W                 pattern.txt              /*\/\\_W*
/\_X                 pattern.txt              /*\/\\_X*
```

```
/\_[]                pattern.txt              /*\/\\_[]*
/\_^                 pattern.txt              /*\/\\_^*
/\_a                 pattern.txt              /*\/\\_a*
/\_d                 pattern.txt              /*\/\\_d*
/\_f                 pattern.txt              /*\/\\_f*
/\_h                 pattern.txt              /*\/\\_h*
/\_i                 pattern.txt              /*\/\\_i*
/\_k                 pattern.txt              /*\/\\_k*
/\_l                 pattern.txt              /*\/\\_l*
/\_o                 pattern.txt              /*\/\\_o*
/\_p                 pattern.txt              /*\/\\_p*
/\_s                 pattern.txt              /*\/\\_s*
/\_u                 pattern.txt              /*\/\\_u*
/\_w                 pattern.txt              /*\/\\_w*
/\_x                 pattern.txt              /*\/\\_x*
/\a                  pattern.txt              /*\/\\a*
/\b                  pattern.txt              /*\/\\b*
/\bar                pattern.txt              /*\/\\bar*
/\c                  pattern.txt              /*\/\\c*
/\d                  pattern.txt              /*\/\\d*
/\e                  pattern.txt              /*\/\\e*
/\f                  pattern.txt              /*\/\\f*
/\h                  pattern.txt              /*\/\\h*
/\i                  pattern.txt              /*\/\\i*
/\k                  pattern.txt              /*\/\\k*
/\l                  pattern.txt              /*\/\\l*
/\m                  pattern.txt              /*\/\\m*
/\n                  pattern.txt              /*\/\\n*
/\o                  pattern.txt              /*\/\\o*
/\p                  pattern.txt              /*\/\\p*
/\r                  pattern.txt              /*\/\\r*
/\s                  pattern.txt              /*\/\\s*
/\star               pattern.txt              /*\/\\star*
/\t                  pattern.txt              /*\/\\t*
/\u                  pattern.txt              /*\/\\u*
/\v                  pattern.txt              /*\/\\v*
/\w                  pattern.txt              /*\/\\w*
/\x                  pattern.txt              /*\/\\x*
/\z(                 syntax.txt               /*\/\\z(*
/\z(\)               syntax.txt               /*\/\\z(\\)*
/\z1                 syntax.txt               /*\/\\z1*
/\z2                 syntax.txt               /*\/\\z2*
/\z3                 syntax.txt               /*\/\\z3*
/\z4                 syntax.txt               /*\/\\z4*
/\z5                 syntax.txt               /*\/\\z5*
/\z6                 syntax.txt               /*\/\\z6*
/\z7                 syntax.txt               /*\/\\z7*
/\z8                 syntax.txt               /*\/\\z8*
/\z9                 syntax.txt               /*\/\\z9*
/\ze                 pattern.txt              /*\/\\ze*
/\zs                 pattern.txt              /*\/\\zs*
/\{                  pattern.txt              /*\/\\{*
/\{-                 pattern.txt              /*\/\\{-*
/\~                  pattern.txt              /*\/\\~*
```

/^	pattern.txt	/*\/^*
/_CTRL-G	cmdline.txt	/*\/_CTRL-G*
/_CTRL-L	cmdline.txt	/*\/_CTRL-L*
/_CTRL-T	cmdline.txt	/*\/_CTRL-T*
/atom	pattern.txt	/*\/atom*
/bar	pattern.txt	/*\/bar*
/branch	pattern.txt	/*\/branch*
/character-classes	pattern.txt	/*\/character-classes*
/collection	pattern.txt	/*\/collection*
/concat	pattern.txt	/*\/concat*
/dyn	various.txt	/*\/dyn*
/ignorecase	pattern.txt	/*\/ignorecase*
/magic	pattern.txt	/*\/magic*
/multi	pattern.txt	/*\/multi*
/ordinary-atom	pattern.txt	/*\/ordinary-atom*
/pattern	pattern.txt	/*\/pattern*
/piece	pattern.txt	/*\/piece*
/star	pattern.txt	/*\/star*
/zero-width	pattern.txt	/*\/zero-width*
/~	pattern.txt	/*\/~*
0	motion.txt	/*0*
01.1	usr_01.txt	/*01.1*
01.2	usr_01.txt	/*01.2*
01.3	usr_01.txt	/*01.3*
01.4	usr_01.txt	/*01.4*
02.1	usr_02.txt	/*02.1*
02.2	usr_02.txt	/*02.2*
02.3	usr_02.txt	/*02.3*
02.4	usr_02.txt	/*02.4*
02.5	usr_02.txt	/*02.5*
02.6	usr_02.txt	/*02.6*
02.7	usr_02.txt	/*02.7*
02.8	usr_02.txt	/*02.8*
03.1	usr_03.txt	/*03.1*
03.10	usr_03.txt	/*03.10*
03.2	usr_03.txt	/*03.2*
03.3	usr_03.txt	/*03.3*
03.4	usr_03.txt	/*03.4*
03.5	usr_03.txt	/*03.5*
03.6	usr_03.txt	/*03.6*
03.7	usr_03.txt	/*03.7*
03.8	usr_03.txt	/*03.8*
03.9	usr_03.txt	/*03.9*
04.1	usr_04.txt	/*04.1*
04.10	usr_04.txt	/*04.10*
04.2	usr_04.txt	/*04.2*
04.3	usr_04.txt	/*04.3*
04.4	usr_04.txt	/*04.4*
04.5	usr_04.txt	/*04.5*
04.6	usr_04.txt	/*04.6*
04.7	usr_04.txt	/*04.7*
04.8	usr_04.txt	/*04.8*
04.9	usr_04.txt	/*04.9*
05.1	usr_05.txt	/*05.1*

05.2	usr_05.txt	/*05.2*
05.3	usr_05.txt	/*05.3*
05.4	usr_05.txt	/*05.4*
05.5	usr_05.txt	/*05.5*
05.6	usr_05.txt	/*05.6*
05.7	usr_05.txt	/*05.7*
05.8	usr_05.txt	/*05.8*
05.9	usr_05.txt	/*05.9*
06.1	usr_06.txt	/*06.1*
06.2	usr_06.txt	/*06.2*
06.3	usr_06.txt	/*06.3*
06.4	usr_06.txt	/*06.4*
06.5	usr_06.txt	/*06.5*
06.6	usr_06.txt	/*06.6*
07.1	usr_07.txt	/*07.1*
07.2	usr_07.txt	/*07.2*
07.3	usr_07.txt	/*07.3*
07.4	usr_07.txt	/*07.4*
07.5	usr_07.txt	/*07.5*
07.6	usr_07.txt	/*07.6*
07.7	usr_07.txt	/*07.7*
08.1	usr_08.txt	/*08.1*
08.2	usr_08.txt	/*08.2*
08.3	usr_08.txt	/*08.3*
08.4	usr_08.txt	/*08.4*
08.5	usr_08.txt	/*08.5*
08.6	usr_08.txt	/*08.6*
08.7	usr_08.txt	/*08.7*
08.8	usr_08.txt	/*08.8*
08.9	usr_08.txt	/*08.9*
09.1	usr_09.txt	/*09.1*
09.2	usr_09.txt	/*09.2*
09.3	usr_09.txt	/*09.3*
09.4	usr_09.txt	/*09.4*
10.1	usr_10.txt	/*10.1*
10.2	usr_10.txt	/*10.2*
10.3	usr_10.txt	/*10.3*
10.4	usr_10.txt	/*10.4*
10.5	usr_10.txt	/*10.5*
10.6	usr_10.txt	/*10.6*
10.7	usr_10.txt	/*10.7*
10.8	usr_10.txt	/*10.8*
10.9	usr_10.txt	/*10.9*
11.1	usr_11.txt	/*11.1*
11.2	usr_11.txt	/*11.2*
11.3	usr_11.txt	/*11.3*
11.4	usr_11.txt	/*11.4*
12.1	usr_12.txt	/*12.1*
12.2	usr_12.txt	/*12.2*
12.3	usr_12.txt	/*12.3*
12.4	usr_12.txt	/*12.4*
12.5	usr_12.txt	/*12.5*
12.6	usr_12.txt	/*12.6*
12.7	usr_12.txt	/*12.7*

12.8	usr_12.txt	/*12.8*
1gD	pattern.txt	/*1gD*
1gd	pattern.txt	/*1gd*
20.1	usr_20.txt	/*20.1*
20.2	usr_20.txt	/*20.2*
20.3	usr_20.txt	/*20.3*
20.4	usr_20.txt	/*20.4*
20.5	usr_20.txt	/*20.5*
21.1	usr_21.txt	/*21.1*
21.2	usr_21.txt	/*21.2*
21.3	usr_21.txt	/*21.3*
21.4	usr_21.txt	/*21.4*
21.5	usr_21.txt	/*21.5*
21.6	usr_21.txt	/*21.6*
22.1	usr_22.txt	/*22.1*
22.2	usr_22.txt	/*22.2*
22.3	usr_22.txt	/*22.3*
22.4	usr_22.txt	/*22.4*
23.1	usr_23.txt	/*23.1*
23.2	usr_23.txt	/*23.2*
23.3	usr_23.txt	/*23.3*
23.4	usr_23.txt	/*23.4*
23.5	usr_23.txt	/*23.5*
24.1	usr_24.txt	/*24.1*
24.10	usr_24.txt	/*24.10*
24.2	usr_24.txt	/*24.2*
24.3	usr_24.txt	/*24.3*
24.4	usr_24.txt	/*24.4*
24.5	usr_24.txt	/*24.5*
24.6	usr_24.txt	/*24.6*
24.7	usr_24.txt	/*24.7*
24.8	usr_24.txt	/*24.8*
24.9	usr_24.txt	/*24.9*
25.1	usr_25.txt	/*25.1*
25.2	usr_25.txt	/*25.2*
25.3	usr_25.txt	/*25.3*
25.4	usr_25.txt	/*25.4*
25.5	usr_25.txt	/*25.5*
26.1	usr_26.txt	/*26.1*
26.2	usr_26.txt	/*26.2*
26.3	usr_26.txt	/*26.3*
26.4	usr_26.txt	/*26.4*
27.1	usr_27.txt	/*27.1*
27.2	usr_27.txt	/*27.2*
27.3	usr_27.txt	/*27.3*
27.4	usr_27.txt	/*27.4*
27.5	usr_27.txt	/*27.5*
27.6	usr_27.txt	/*27.6*
27.7	usr_27.txt	/*27.7*
27.8	usr_27.txt	/*27.8*
27.9	usr_27.txt	/*27.9*
28.1	usr_28.txt	/*28.1*
28.10	usr_28.txt	/*28.10*
28.2	usr_28.txt	/*28.2*

28.3	usr_28.txt	/*28.3*
28.4	usr_28.txt	/*28.4*
28.5	usr_28.txt	/*28.5*
28.6	usr_28.txt	/*28.6*
28.7	usr_28.txt	/*28.7*
28.8	usr_28.txt	/*28.8*
28.9	usr_28.txt	/*28.9*
29.1	usr_29.txt	/*29.1*
29.2	usr_29.txt	/*29.2*
29.3	usr_29.txt	/*29.3*
29.4	usr_29.txt	/*29.4*
29.5	usr_29.txt	/*29.5*
2html.vim	syntax.txt	/*2html.vim*
30.1	usr_30.txt	/*30.1*
30.2	usr_30.txt	/*30.2*
30.3	usr_30.txt	/*30.3*
30.4	usr_30.txt	/*30.4*
30.5	usr_30.txt	/*30.5*
30.6	usr_30.txt	/*30.6*
31.1	usr_31.txt	/*31.1*
31.2	usr_31.txt	/*31.2*
31.3	usr_31.txt	/*31.3*
31.4	usr_31.txt	/*31.4*
31.5	usr_31.txt	/*31.5*
32.1	usr_32.txt	/*32.1*
32.2	usr_32.txt	/*32.2*
32.3	usr_32.txt	/*32.3*
32.4	usr_32.txt	/*32.4*
40.1	usr_40.txt	/*40.1*
40.2	usr_40.txt	/*40.2*
40.3	usr_40.txt	/*40.3*
41.1	usr_41.txt	/*41.1*
41.10	usr_41.txt	/*41.10*
41.11	usr_41.txt	/*41.11*
41.12	usr_41.txt	/*41.12*
41.13	usr_41.txt	/*41.13*
41.14	usr_41.txt	/*41.14*
41.15	usr_41.txt	/*41.15*
41.16	usr_41.txt	/*41.16*
41.2	usr_41.txt	/*41.2*
41.3	usr_41.txt	/*41.3*
41.4	usr_41.txt	/*41.4*
41.5	usr_41.txt	/*41.5*
41.6	usr_41.txt	/*41.6*
41.7	usr_41.txt	/*41.7*
41.8	usr_41.txt	/*41.8*
41.9	usr_41.txt	/*41.9*
42	usr_42.txt	/*42*
42.1	usr_42.txt	/*42.1*
42.2	usr_42.txt	/*42.2*
42.3	usr_42.txt	/*42.3*
42.4	usr_42.txt	/*42.4*
43.1	usr_43.txt	/*43.1*
43.2	usr_43.txt	/*43.2*

44.1	usr_44.txt	/*44.1*
44.10	usr_44.txt	/*44.10*
44.11	usr_44.txt	/*44.11*
44.12	usr_44.txt	/*44.12*
44.2	usr_44.txt	/*44.2*
44.3	usr_44.txt	/*44.3*
44.4	usr_44.txt	/*44.4*
44.5	usr_44.txt	/*44.5*
44.6	usr_44.txt	/*44.6*
44.7	usr_44.txt	/*44.7*
44.8	usr_44.txt	/*44.8*
44.9	usr_44.txt	/*44.9*
45.1	usr_45.txt	/*45.1*
45.2	usr_45.txt	/*45.2*
45.3	usr_45.txt	/*45.3*
45.4	usr_45.txt	/*45.4*
45.5	usr_45.txt	/*45.5*
8g8	various.txt	/*8g8*
90.1	usr_90.txt	/*90.1*
90.2	usr_90.txt	/*90.2*
90.3	usr_90.txt	/*90.3*
90.4	usr_90.txt	/*90.4*
90.5	usr_90.txt	/*90.5*
:	cmdline.txt	/*:*
:!	various.txt	/*:!*
:!!	various.txt	/*:!!*
:!cmd	various.txt	/*:!cmd*
:!start	os_win32.txt	/*:!start*
:#	various.txt	/*:#*
:#!	various.txt	/*:#!*
:$	cmdline.txt	/*:$*
:%	cmdline.txt	/*:%*
:&	change.txt	/*:&*
:&&	change.txt	/*:&&*
:'	cmdline.txt	/*:'*
:,	cmdline.txt	/*:,*
:.	cmdline.txt	/*:.*
:/	cmdline.txt	/*:\/*
:0file	editing.txt	/*:0file*
:2match	pattern.txt	/*:2match*
:3match	pattern.txt	/*:3match*
::.	cmdline.txt	/*::.*
::8	cmdline.txt	/*::8*
::S	cmdline.txt	/*::S*
::e	cmdline.txt	/*::e*
::gs	cmdline.txt	/*::gs*
::h	cmdline.txt	/*::h*
::p	cmdline.txt	/*::p*
::r	cmdline.txt	/*::r*
::s	cmdline.txt	/*::s*
::t	cmdline.txt	/*::t*
::~	cmdline.txt	/*::~*
:;	cmdline.txt	/*:;*
:<	change.txt	/*:<*

:<abuf>	cmdline.txt	/*:<abuf>*
:<afile>	cmdline.txt	/*:<afile>*
:<amatch>	cmdline.txt	/*:<amatch>*
:<cWORD>	cmdline.txt	/*:<cWORD>*
:<cexpr>	cmdline.txt	/*:<cexpr>*
:<cfile>	cmdline.txt	/*:<cfile>*
:<cword>	cmdline.txt	/*:<cword>*
:<sfile>	cmdline.txt	/*:<sfile>*
:<sflnum>	cmdline.txt	/*:<sflnum>*
:<slnum>	cmdline.txt	/*:<slnum>*
:=	various.txt	/*:=*
:>	change.txt	/*:>*
:?	cmdline.txt	/*:?*
:@	repeat.txt	/*:@*
:@:	repeat.txt	/*:@:*
:@@	repeat.txt	/*:@@*
:AdaLines	ft_ada.txt	/*:AdaLines*
:AdaRainbow	ft_ada.txt	/*:AdaRainbow*
:AdaSpaces	ft_ada.txt	/*:AdaSpaces*
:AdaTagDir	ft_ada.txt	/*:AdaTagDir*
:AdaTagFile	ft_ada.txt	/*:AdaTagFile*
:AdaTypes	ft_ada.txt	/*:AdaTypes*
:Arguments	terminal.txt	/*:Arguments*
:Break	terminal.txt	/*:Break*
:Cfilter	quickfix.txt	/*:Cfilter*
:Clear	terminal.txt	/*:Clear*
:CompilerSet	usr_41.txt	/*:CompilerSet*
:Continue	terminal.txt	/*:Continue*
:DiffOrig	diff.txt	/*:DiffOrig*
:DoMatchParen	pi_paren.txt	/*:DoMatchParen*
:Evaluate	terminal.txt	/*:Evaluate*
:Explore	pi_netrw.txt	/*:Explore*
:Finish	terminal.txt	/*:Finish*
:GLVS	pi_getscript.txt	/*:GLVS*
:Gdb	terminal.txt	/*:Gdb*
:GetLatestVimScripts_dat	pi_getscript.txt	/*:GetLatestVimScripts_dat*
:GnatFind	ft_ada.txt	/*:GnatFind*
:GnatPretty	ft_ada.txt	/*:GnatPretty*
:GnatTags	ft_ada.txt	/*:GnatTags*
:Hexplore	pi_netrw.txt	/*:Hexplore*
:LP	pi_logipat.txt	/*:LP*
:LPE	pi_logipat.txt	/*:LPE*
:LPF	pi_logipat.txt	/*:LPF*
:Lexplore	pi_netrw.txt	/*:Lexplore*
:Lfilter	quickfix.txt	/*:Lfilter*
:LogiPat	pi_logipat.txt	/*:LogiPat*
:Man	filetype.txt	/*:Man*
:MkVimball	pi_vimball.txt	/*:MkVimball*
:N	editing.txt	/*:N*
:NetrwClean	pi_netrw.txt	/*:NetrwClean*
:Nexplore	pi_netrw.txt	/*:Nexplore*
:Next	editing.txt	/*:Next*
:NoMatchParen	pi_paren.txt	/*:NoMatchParen*
:Nr	pi_netrw.txt	/*:Nr*

:Nread	pi_netrw.txt	/*:Nread*
:Ns	pi_netrw.txt	/*:Ns*
:Nsource	pi_netrw.txt	/*:Nsource*
:Ntree	pi_netrw.txt	/*:Ntree*
:Nw	pi_netrw.txt	/*:Nw*
:Nwrite	pi_netrw.txt	/*:Nwrite*
:Over	terminal.txt	/*:Over*
:P	various.txt	/*:P*
:Pexplore	pi_netrw.txt	/*:Pexplore*
:Print	various.txt	/*:Print*
:Program	terminal.txt	/*:Program*
:Rexplore	pi_netrw.txt	/*:Rexplore*
:RmVimball	pi_vimball.txt	/*:RmVimball*
:Run	terminal.txt	/*:Run*
:RustEmitAsm	ft_rust.txt	/*:RustEmitAsm*
:RustEmitIr	ft_rust.txt	/*:RustEmitIr*
:RustExpand	ft_rust.txt	/*:RustExpand*
:RustFmt	ft_rust.txt	/*:RustFmt*
:RustFmtRange	ft_rust.txt	/*:RustFmtRange*
:RustPlay	ft_rust.txt	/*:RustPlay*
:RustRun	ft_rust.txt	/*:RustRun*
:Sexplore	pi_netrw.txt	/*:Sexplore*
:Source	terminal.txt	/*:Source*
:Step	terminal.txt	/*:Step*
:Stop	terminal.txt	/*:Stop*
:TOhtml	syntax.txt	/*:TOhtml*
:TarDiff	pi_tar.txt	/*:TarDiff*
:Termdebug	terminal.txt	/*:Termdebug*
:TermdebugCommand	terminal.txt	/*:TermdebugCommand*
:Texplore	pi_netrw.txt	/*:Texplore*
:UseVimball	pi_vimball.txt	/*:UseVimball*
:Vexplore	pi_netrw.txt	/*:Vexplore*
:VimballList	pi_vimball.txt	/*:VimballList*
:Vimuntar	pi_tar.txt	/*:Vimuntar*
:Winbar	terminal.txt	/*:Winbar*
:X	editing.txt	/*:X*
:XMLent	insert.txt	/*:XMLent*
:XMLns	insert.txt	/*:XMLns*
:[range]	motion.txt	/*:[range]*
:\bar	cmdline.txt	/*:\\bar*
:_!	cmdline.txt	/*:_!*
:_#	cmdline.txt	/*:_#*
:_##	cmdline.txt	/*:_##*
:_#0	cmdline.txt	/*:_#0*
:_#<	cmdline.txt	/*:_#<*
:_#n	cmdline.txt	/*:_#n*
:_%	cmdline.txt	/*:_%*
:_%:	cmdline.txt	/*:_%:*
:_%<	cmdline.txt	/*:_%<*
:a	insert.txt	/*:a*
:ab	map.txt	/*:ab*
:abbreviate	map.txt	/*:abbreviate*
:abbreviate-<buffer>	map.txt	/*:abbreviate-<buffer>*
:abbreviate-local	map.txt	/*:abbreviate-local*

:abbreviate-verbose	map.txt	/*:abbreviate-verbose*
:abc	map.txt	/*:abc*
:abclear	map.txt	/*:abclear*
:abo	windows.txt	/*:abo*
:aboveleft	windows.txt	/*:aboveleft*
:addd	quickfix.txt	/*:addd*
:al	windows.txt	/*:al*
:all	windows.txt	/*:all*
:am	gui.txt	/*:am*
:amenu	gui.txt	/*:amenu*
:an	gui.txt	/*:an*
:anoremenu	gui.txt	/*:anoremenu*
:append	insert.txt	/*:append*
:ar	editing.txt	/*:ar*
:arg	editing.txt	/*:arg*
:arga	editing.txt	/*:arga*
:argadd	editing.txt	/*:argadd*
:argd	editing.txt	/*:argd*
:argdelete	editing.txt	/*:argdelete*
:argdo	editing.txt	/*:argdo*
:arge	editing.txt	/*:arge*
:argedit	editing.txt	/*:argedit*
:argglobal	editing.txt	/*:argglobal*
:arglocal	editing.txt	/*:arglocal*
:args	editing.txt	/*:args*
:args_f	editing.txt	/*:args_f*
:args_f!	editing.txt	/*:args_f!*
:argu	editing.txt	/*:argu*
:argument	editing.txt	/*:argument*
:as	various.txt	/*:as*
:ascii	various.txt	/*:ascii*
:au	autocmd.txt	/*:au*
:aug	autocmd.txt	/*:aug*
:augroup	autocmd.txt	/*:augroup*
:augroup-delete	autocmd.txt	/*:augroup-delete*
:aun	gui.txt	/*:aun*
:aunmenu	gui.txt	/*:aunmenu*
:autocmd	autocmd.txt	/*:autocmd*
:autocmd-verbose	autocmd.txt	/*:autocmd-verbose*
:b	windows.txt	/*:b*
:bN	windows.txt	/*:bN*
:bNext	windows.txt	/*:bNext*
:ba	windows.txt	/*:ba*
:bad	windows.txt	/*:bad*
:badd	windows.txt	/*:badd*
:ball	windows.txt	/*:ball*
:bar	cmdline.txt	/*:bar*
:bd	windows.txt	/*:bd*
:bdel	windows.txt	/*:bdel*
:bdelete	windows.txt	/*:bdelete*
:be	gui.txt	/*:be*
:behave	gui.txt	/*:behave*
:bel	windows.txt	/*:bel*
:belowright	windows.txt	/*:belowright*

:bf	windows.txt	/*:bf*
:bfirst	windows.txt	/*:bfirst*
:bl	windows.txt	/*:bl*
:blast	windows.txt	/*:blast*
:bm	windows.txt	/*:bm*
:bmodified	windows.txt	/*:bmodified*
:bn	windows.txt	/*:bn*
:bnext	windows.txt	/*:bnext*
:bo	windows.txt	/*:bo*
:botright	windows.txt	/*:botright*
:bp	windows.txt	/*:bp*
:bprevious	windows.txt	/*:bprevious*
:br	windows.txt	/*:br*
:bre	windows.txt	/*:bre*
:brea	eval.txt	/*:brea*
:break	eval.txt	/*:break*
:breaka	repeat.txt	/*:breaka*
:breakadd	repeat.txt	/*:breakadd*
:breakd	repeat.txt	/*:breakd*
:breakdel	repeat.txt	/*:breakdel*
:breakl	repeat.txt	/*:breakl*
:breaklist	repeat.txt	/*:breaklist*
:brewind	windows.txt	/*:brewind*
:bro	editing.txt	/*:bro*
:browse	editing.txt	/*:browse*
:browse-set	options.txt	/*:browse-set*
:bu	windows.txt	/*:bu*
:buf	windows.txt	/*:buf*
:bufdo	windows.txt	/*:bufdo*
:buffer	windows.txt	/*:buffer*
:buffer-!	windows.txt	/*:buffer-!*
:buffers	windows.txt	/*:buffers*
:bun	windows.txt	/*:bun*
:bunload	windows.txt	/*:bunload*
:bw	windows.txt	/*:bw*
:bwipe	windows.txt	/*:bwipe*
:bwipeout	windows.txt	/*:bwipeout*
:c	change.txt	/*:c*
:cN	quickfix.txt	/*:cN*
:cNext	quickfix.txt	/*:cNext*
:cNf	quickfix.txt	/*:cNf*
:cNfile	quickfix.txt	/*:cNfile*
:ca	map.txt	/*:ca*
:cab	map.txt	/*:cab*
:cabbrev	map.txt	/*:cabbrev*
:cabc	map.txt	/*:cabc*
:cabclear	map.txt	/*:cabclear*
:cabo	quickfix.txt	/*:cabo*
:cabove	quickfix.txt	/*:cabove*
:cad	quickfix.txt	/*:cad*
:cadd	quickfix.txt	/*:cadd*
:caddbuffer	quickfix.txt	/*:caddbuffer*
:cadde	quickfix.txt	/*:cadde*
:caddexpr	quickfix.txt	/*:caddexpr*

:caddf	quickfix.txt	/*:caddf*
:caddfile	quickfix.txt	/*:caddfile*
:caf	quickfix.txt	/*:caf*
:cafter	quickfix.txt	/*:cafter*
:cal	eval.txt	/*:cal*
:call	eval.txt	/*:call*
:cat	eval.txt	/*:cat*
:catch	eval.txt	/*:catch*
:cb	quickfix.txt	/*:cb*
:cbe	quickfix.txt	/*:cbe*
:cbefore	quickfix.txt	/*:cbefore*
:cbel	quickfix.txt	/*:cbel*
:cbelow	quickfix.txt	/*:cbelow*
:cbo	quickfix.txt	/*:cbo*
:cbottom	quickfix.txt	/*:cbottom*
:cbuffer	quickfix.txt	/*:cbuffer*
:cc	quickfix.txt	/*:cc*
:ccl	quickfix.txt	/*:ccl*
:cclose	quickfix.txt	/*:cclose*
:cd	editing.txt	/*:cd*
:cd-	editing.txt	/*:cd-*
:cdo	quickfix.txt	/*:cdo*
:ce	change.txt	/*:ce*
:center	change.txt	/*:center*
:cex	quickfix.txt	/*:cex*
:cexpr	quickfix.txt	/*:cexpr*
:cf	quickfix.txt	/*:cf*
:cfdo	quickfix.txt	/*:cfdo*
:cfile	quickfix.txt	/*:cfile*
:cfir	quickfix.txt	/*:cfir*
:cfirst	quickfix.txt	/*:cfirst*
:cg	quickfix.txt	/*:cg*
:cgetb	quickfix.txt	/*:cgetb*
:cgetbuffer	quickfix.txt	/*:cgetbuffer*
:cgete	quickfix.txt	/*:cgete*
:cgetexpr	quickfix.txt	/*:cgetexpr*
:cgetfile	quickfix.txt	/*:cgetfile*
:ch	change.txt	/*:ch*
:change	change.txt	/*:change*
:changes	motion.txt	/*:changes*
:chd	editing.txt	/*:chd*
:chdir	editing.txt	/*:chdir*
:che	tagsrch.txt	/*:che*
:chec	tagsrch.txt	/*:chec*
:check	tagsrch.txt	/*:check*
:checkpath	tagsrch.txt	/*:checkpath*
:checkt	editing.txt	/*:checkt*
:checktime	editing.txt	/*:checktime*
:chi	quickfix.txt	/*:chi*
:chistory	quickfix.txt	/*:chistory*
:cl	quickfix.txt	/*:cl*
:cla	quickfix.txt	/*:cla*
:clast	quickfix.txt	/*:clast*
:cle	motion.txt	/*:cle*

:clearjumps motion.txt /*:clearjumps*
:clist quickfix.txt /*:clist*
:clo windows.txt /*:clo*
:close windows.txt /*:close*
:cm map.txt /*:cm*
:cmap map.txt /*:cmap*
:cmap_l map.txt /*:cmap_l*
:cmapc map.txt /*:cmapc*
:cmapclear map.txt /*:cmapclear*
:cme gui.txt /*:cme*
:cmenu gui.txt /*:cmenu*
:cn quickfix.txt /*:cn*
:cne quickfix.txt /*:cne*
:cnew quickfix.txt /*:cnew*
:cnewer quickfix.txt /*:cnewer*
:cnext quickfix.txt /*:cnext*
:cnf quickfix.txt /*:cnf*
:cnfile quickfix.txt /*:cnfile*
:cno map.txt /*:cno*
:cnor map.txt /*:cnor*
:cnorea map.txt /*:cnorea*
:cnoreabbrev map.txt /*:cnoreabbrev*
:cnoremap map.txt /*:cnoremap*
:cnoreme gui.txt /*:cnoreme*
:cnoremenu gui.txt /*:cnoremenu*
:co change.txt /*:co*
:col quickfix.txt /*:col*
:colder quickfix.txt /*:colder*
:colo syntax.txt /*:colo*
:colorscheme syntax.txt /*:colorscheme*
:com map.txt /*:com*
:comc map.txt /*:comc*
:comclear map.txt /*:comclear*
:command map.txt /*:command*
:command-addr map.txt /*:command-addr*
:command-bang map.txt /*:command-bang*
:command-bar map.txt /*:command-bar*
:command-buffer map.txt /*:command-buffer*
:command-complete map.txt /*:command-complete*
:command-completion map.txt /*:command-completion*
:command-completion-custom map.txt /*:command-completion-custom*
:command-completion-customlist map.txt /*:command-completion-customlist*
:command-count map.txt /*:command-count*
:command-nargs map.txt /*:command-nargs*
:command-range map.txt /*:command-range*
:command-register map.txt /*:command-register*
:command-verbose map.txt /*:command-verbose*
:comment cmdline.txt /*:comment*
:comp quickfix.txt /*:comp*
:compiler quickfix.txt /*:compiler*
:con eval.txt /*:con*
:conf editing.txt /*:conf*
:confirm editing.txt /*:confirm*
:cons eval.txt /*:cons*

:const	eval.txt	/*:const*
:continue	eval.txt	/*:continue*
:cope	quickfix.txt	/*:cope*
:copen	quickfix.txt	/*:copen*
:copy	change.txt	/*:copy*
:cp	quickfix.txt	/*:cp*
:cpf	quickfix.txt	/*:cpf*
:cpfile	quickfix.txt	/*:cpfile*
:cprev	quickfix.txt	/*:cprev*
:cprevious	quickfix.txt	/*:cprevious*
:cq	quickfix.txt	/*:cq*
:cquit	quickfix.txt	/*:cquit*
:cr	quickfix.txt	/*:cr*
:crewind	quickfix.txt	/*:crewind*
:cs	if_cscop.txt	/*:cs*
:cscope	if_cscop.txt	/*:cscope*
:cstag	if_cscop.txt	/*:cstag*
:cu	map.txt	/*:cu*
:cun	map.txt	/*:cun*
:cuna	map.txt	/*:cuna*
:cunabbrev	map.txt	/*:cunabbrev*
:cunmap	map.txt	/*:cunmap*
:cunme	gui.txt	/*:cunme*
:cunmenu	gui.txt	/*:cunmenu*
:cw	quickfix.txt	/*:cw*
:cwindow	quickfix.txt	/*:cwindow*
:d	change.txt	/*:d*
:de	change.txt	/*:de*
:debug	repeat.txt	/*:debug*
:debug-name	repeat.txt	/*:debug-name*
:debugg	repeat.txt	/*:debugg*
:debuggreedy	repeat.txt	/*:debuggreedy*
:del	change.txt	/*:del*
:delc	map.txt	/*:delc*
:delcommand	map.txt	/*:delcommand*
:delcr	todo.txt	/*:delcr*
:delete	change.txt	/*:delete*
:delf	eval.txt	/*:delf*
:delfunction	eval.txt	/*:delfunction*
:delm	motion.txt	/*:delm*
:delmarks	motion.txt	/*:delmarks*
:di	change.txt	/*:di*
:dif	diff.txt	/*:dif*
:diff	diff.txt	/*:diff*
:diffg	diff.txt	/*:diffg*
:diffget	diff.txt	/*:diffget*
:diffo	diff.txt	/*:diffo*
:diffoff	diff.txt	/*:diffoff*
:diffp	diff.txt	/*:diffp*
:diffpatch	diff.txt	/*:diffpatch*
:diffpu	diff.txt	/*:diffpu*
:diffput	diff.txt	/*:diffput*
:diffs	diff.txt	/*:diffs*
:diffsplit	diff.txt	/*:diffsplit*

:difft	diff.txt	/*:difft*
:diffthis	diff.txt	/*:diffthis*
:diffupdate	diff.txt	/*:diffupdate*
:dig	digraph.txt	/*:dig*
:digraphs	digraph.txt	/*:digraphs*
:display	change.txt	/*:display*
:dj	tagsrch.txt	/*:dj*
:djump	tagsrch.txt	/*:djump*
:dl	change.txt	/*:dl*
:dli	tagsrch.txt	/*:dli*
:dlist	tagsrch.txt	/*:dlist*
:do	autocmd.txt	/*:do*
:doau	autocmd.txt	/*:doau*
:doaut	autocmd.txt	/*:doaut*
:doautoa	autocmd.txt	/*:doautoa*
:doautoall	autocmd.txt	/*:doautoall*
:doautocmd	autocmd.txt	/*:doautocmd*
:dp	change.txt	/*:dp*
:dr	windows.txt	/*:dr*
:drop	windows.txt	/*:drop*
:ds	tagsrch.txt	/*:ds*
:dsearch	tagsrch.txt	/*:dsearch*
:dsp	tagsrch.txt	/*:dsp*
:dsplit	tagsrch.txt	/*:dsplit*
:e	editing.txt	/*:e*
:ea	undo.txt	/*:ea*
:earlier	undo.txt	/*:earlier*
:ec	eval.txt	/*:ec*
:echo	eval.txt	/*:echo*
:echo-redraw	eval.txt	/*:echo-redraw*
:echoe	eval.txt	/*:echoe*
:echoerr	eval.txt	/*:echoerr*
:echoh	eval.txt	/*:echoh*
:echohl	eval.txt	/*:echohl*
:echom	eval.txt	/*:echom*
:echomsg	eval.txt	/*:echomsg*
:echon	eval.txt	/*:echon*
:edit	editing.txt	/*:edit*
:edit!	editing.txt	/*:edit!*
:edit!_f	editing.txt	/*:edit!_f*
:edit_f	editing.txt	/*:edit_f*
:el	eval.txt	/*:el*
:else	eval.txt	/*:else*
:elsei	eval.txt	/*:elsei*
:elseif	eval.txt	/*:elseif*
:em	gui.txt	/*:em*
:emenu	gui.txt	/*:emenu*
:en	eval.txt	/*:en*
:end	eval.txt	/*:end*
:endf	eval.txt	/*:endf*
:endfo	eval.txt	/*:endfo*
:endfor	eval.txt	/*:endfor*
:endfunction	eval.txt	/*:endfunction*
:endif	eval.txt	/*:endif*

```
:endt                  eval.txt                  /*:endt*
:endtry                eval.txt                  /*:endtry*
:endw                  eval.txt                  /*:endw*
:endwhile              eval.txt                  /*:endwhile*
:ene                   editing.txt               /*:ene*
:ene!                  editing.txt               /*:ene!*
:enew                  editing.txt               /*:enew*
:enew!                 editing.txt               /*:enew!*
:eval                  eval.txt                  /*:eval*
:ex                    editing.txt               /*:ex*
:exe                   eval.txt                  /*:exe*
:exe-comment           eval.txt                  /*:exe-comment*
:execute               eval.txt                  /*:execute*
:exi                   editing.txt               /*:exi*
:exit                  editing.txt               /*:exit*
:exu                   helphelp.txt              /*:exu*
:exusage               helphelp.txt              /*:exusage*
:f                     editing.txt               /*:f*
:fi                    editing.txt               /*:fi*
:file                  editing.txt               /*:file*
:file_f                editing.txt               /*:file_f*
:filename              editing.txt               /*:filename*
:files                 windows.txt               /*:files*
:filet                 filetype.txt              /*:filet*
:filetype              filetype.txt              /*:filetype*
:filetype-indent-off   filetype.txt              /*:filetype-indent-off*
:filetype-indent-on    filetype.txt              /*:filetype-indent-on*
:filetype-off          filetype.txt              /*:filetype-off*
:filetype-overview     filetype.txt              /*:filetype-overview*
:filetype-plugin-off   filetype.txt              /*:filetype-plugin-off*
:filetype-plugin-on    filetype.txt              /*:filetype-plugin-on*
:filt                  various.txt               /*:filt*
:filter                various.txt               /*:filter*
:fin                   editing.txt               /*:fin*
:fina                  eval.txt                  /*:fina*
:finally               eval.txt                  /*:finally*
:find                  editing.txt               /*:find*
:fini                  repeat.txt                /*:fini*
:finish                repeat.txt                /*:finish*
:fir                   editing.txt               /*:fir*
:first                 editing.txt               /*:first*
:fix                   options.txt               /*:fix*
:fixdel                options.txt               /*:fixdel*
:fo                    fold.txt                  /*:fo*
:fold                  fold.txt                  /*:fold*
:foldc                 fold.txt                  /*:foldc*
:foldclose             fold.txt                  /*:foldclose*
:foldd                 fold.txt                  /*:foldd*
:folddo                fold.txt                  /*:folddo*
:folddoc               fold.txt                  /*:folddoc*
:folddoclosed          fold.txt                  /*:folddoclosed*
:folddoopen            fold.txt                  /*:folddoopen*
:foldo                 fold.txt                  /*:foldo*
:foldopen              fold.txt                  /*:foldopen*
```

:for	eval.txt	/*:for*
:fu	eval.txt	/*:fu*
:func-abort	eval.txt	/*:func-abort*
:func-closure	eval.txt	/*:func-closure*
:func-dict	eval.txt	/*:func-dict*
:func-range	eval.txt	/*:func-range*
:function	eval.txt	/*:function*
:function-verbose	eval.txt	/*:function-verbose*
:g	repeat.txt	/*:g*
:global	repeat.txt	/*:global*
:go	motion.txt	/*:go*
:goto	motion.txt	/*:goto*
:gr	quickfix.txt	/*:gr*
:grep	quickfix.txt	/*:grep*
:grepa	quickfix.txt	/*:grepa*
:grepadd	quickfix.txt	/*:grepadd*
:gu	gui_x11.txt	/*:gu*
:gui	gui_x11.txt	/*:gui*
:gv	gui_x11.txt	/*:gv*
:gvim	gui_x11.txt	/*:gvim*
:h	helphelp.txt	/*:h*
:ha	print.txt	/*:ha*
:hardcopy	print.txt	/*:hardcopy*
:help	helphelp.txt	/*:help*
:helpc	helphelp.txt	/*:helpc*
:helpclose	helphelp.txt	/*:helpclose*
:helpf	helphelp.txt	/*:helpf*
:helpfind	helphelp.txt	/*:helpfind*
:helpg	helphelp.txt	/*:helpg*
:helpgrep	helphelp.txt	/*:helpgrep*
:helpt	helphelp.txt	/*:helpt*
:helptags	helphelp.txt	/*:helptags*
:hi	syntax.txt	/*:hi*
:hi-default	syntax.txt	/*:hi-default*
:hi-link	syntax.txt	/*:hi-link*
:hi-normal	syntax.txt	/*:hi-normal*
:hi-normal-cterm	syntax.txt	/*:hi-normal-cterm*
:hide	windows.txt	/*:hide*
:highlight	syntax.txt	/*:highlight*
:highlight-default	syntax.txt	/*:highlight-default*
:highlight-link	syntax.txt	/*:highlight-link*
:highlight-normal	syntax.txt	/*:highlight-normal*
:highlight-verbose	syntax.txt	/*:highlight-verbose*
:his	cmdline.txt	/*:his*
:history	cmdline.txt	/*:history*
:history-indexing	cmdline.txt	/*:history-indexing*
:i	insert.txt	/*:i*
:ia	map.txt	/*:ia*
:iabbrev	map.txt	/*:iabbrev*
:iabc	map.txt	/*:iabc*
:iabclear	map.txt	/*:iabclear*
:if	eval.txt	/*:if*
:ij	tagsrch.txt	/*:ij*
:ijump	tagsrch.txt	/*:ijump*

:il	tagsrch.txt	/*:il*
:ilist	tagsrch.txt	/*:ilist*
:im	map.txt	/*:im*
:imap	map.txt	/*:imap*
:imap_l	map.txt	/*:imap_l*
:imapc	map.txt	/*:imapc*
:imapclear	map.txt	/*:imapclear*
:ime	gui.txt	/*:ime*
:imenu	gui.txt	/*:imenu*
:in	insert.txt	/*:in*
:index	index.txt	/*:index*
:ino	map.txt	/*:ino*
:inor	map.txt	/*:inor*
:inorea	map.txt	/*:inorea*
:inoreabbrev	map.txt	/*:inoreabbrev*
:inoremap	map.txt	/*:inoremap*
:inoreme	gui.txt	/*:inoreme*
:inoremenu	gui.txt	/*:inoremenu*
:insert	insert.txt	/*:insert*
:intro	starting.txt	/*:intro*
:is	tagsrch.txt	/*:is*
:isearch	tagsrch.txt	/*:isearch*
:isp	tagsrch.txt	/*:isp*
:isplit	tagsrch.txt	/*:isplit*
:iu	map.txt	/*:iu*
:iuna	map.txt	/*:iuna*
:iunabbrev	map.txt	/*:iunabbrev*
:iunmap	map.txt	/*:iunmap*
:iunme	gui.txt	/*:iunme*
:iunmenu	gui.txt	/*:iunmenu*
:j	change.txt	/*:j*
:join	change.txt	/*:join*
:ju	motion.txt	/*:ju*
:jumps	motion.txt	/*:jumps*
:k	motion.txt	/*:k*
:kee	motion.txt	/*:kee*
:keep	motion.txt	/*:keep*
:keepa	editing.txt	/*:keepa*
:keepalt	editing.txt	/*:keepalt*
:keepj	motion.txt	/*:keepj*
:keepjumps	motion.txt	/*:keepjumps*
:keepmarks	motion.txt	/*:keepmarks*
:keepp	cmdline.txt	/*:keepp*
:keeppatterns	cmdline.txt	/*:keeppatterns*
:l	various.txt	/*:l*
:lN	quickfix.txt	/*:lN*
:lNext	quickfix.txt	/*:lNext*
:lNf	quickfix.txt	/*:lNf*
:lNfile	quickfix.txt	/*:lNfile*
:la	editing.txt	/*:la*
:lab	quickfix.txt	/*:lab*
:labove	quickfix.txt	/*:labove*
:lad	quickfix.txt	/*:lad*
:laddb	quickfix.txt	/*:laddb*

:laddbuffer	quickfix.txt	/*:laddbuffer*
:laddexpr	quickfix.txt	/*:laddexpr*
:laddf	quickfix.txt	/*:laddf*
:laddfile	quickfix.txt	/*:laddfile*
:laf	quickfix.txt	/*:laf*
:lafter	quickfix.txt	/*:lafter*
:lan	mlang.txt	/*:lan*
:lang	mlang.txt	/*:lang*
:language	mlang.txt	/*:language*
:last	editing.txt	/*:last*
:lat	undo.txt	/*:lat*
:later	undo.txt	/*:later*
:lb	quickfix.txt	/*:lb*
:lbe	quickfix.txt	/*:lbe*
:lbefore	quickfix.txt	/*:lbefore*
:lbel	quickfix.txt	/*:lbel*
:lbelow	quickfix.txt	/*:lbelow*
:lbo	quickfix.txt	/*:lbo*
:lbottom	quickfix.txt	/*:lbottom*
:lbuffer	quickfix.txt	/*:lbuffer*
:lc	editing.txt	/*:lc*
:lcd	editing.txt	/*:lcd*
:lch	editing.txt	/*:lch*
:lchdir	editing.txt	/*:lchdir*
:lcl	quickfix.txt	/*:lcl*
:lclose	quickfix.txt	/*:lclose*
:lcs	if_cscop.txt	/*:lcs*
:lcscope	if_cscop.txt	/*:lcscope*
:ldo	quickfix.txt	/*:ldo*
:le	change.txt	/*:le*
:left	change.txt	/*:left*
:lefta	windows.txt	/*:lefta*
:leftabove	windows.txt	/*:leftabove*
:let	eval.txt	/*:let*
:let%=	eval.txt	/*:let%=*
:let+=	eval.txt	/*:let+=*
:let-$	eval.txt	/*:let-$*
:let-&	eval.txt	/*:let-&*
:let-=	eval.txt	/*:let-=*
:let-@	eval.txt	/*:let-@*
:let-environment	eval.txt	/*:let-environment*
:let-heredoc	eval.txt	/*:let-heredoc*
:let-option	eval.txt	/*:let-option*
:let-register	eval.txt	/*:let-register*
:let-unpack	eval.txt	/*:let-unpack*
:let..=	eval.txt	/*:let..=*
:let.=	eval.txt	/*:let.=*
:let/=	eval.txt	/*:let\/=*
:let=<<	eval.txt	/*:let=<<*
:letstar=	eval.txt	/*:letstar=*
:lex	quickfix.txt	/*:lex*
:lexpr	quickfix.txt	/*:lexpr*
:lf	quickfix.txt	/*:lf*
:lfdo	quickfix.txt	/*:lfdo*

:lfi	quickfix.txt	/*:lfi*
:lfile	quickfix.txt	/*:lfile*
:lfir	quickfix.txt	/*:lfir*
:lfirst	quickfix.txt	/*:lfirst*
:lg	quickfix.txt	/*:lg*
:lge	quickfix.txt	/*:lge*
:lgetb	quickfix.txt	/*:lgetb*
:lgetbuffer	quickfix.txt	/*:lgetbuffer*
:lgete	quickfix.txt	/*:lgete*
:lgetexpr	quickfix.txt	/*:lgetexpr*
:lgetfile	quickfix.txt	/*:lgetfile*
:lgr	quickfix.txt	/*:lgr*
:lgrep	quickfix.txt	/*:lgrep*
:lgrepa	quickfix.txt	/*:lgrepa*
:lgrepadd	quickfix.txt	/*:lgrepadd*
:lh	helphelp.txt	/*:lh*
:lhelpgrep	helphelp.txt	/*:lhelpgrep*
:lhi	quickfix.txt	/*:lhi*
:lhistory	quickfix.txt	/*:lhistory*
:list	various.txt	/*:list*
:ll	quickfix.txt	/*:ll*
:lla	quickfix.txt	/*:lla*
:llast	quickfix.txt	/*:llast*
:lli	quickfix.txt	/*:lli*
:llist	quickfix.txt	/*:llist*
:lm	map.txt	/*:lm*
:lma	map.txt	/*:lma*
:lmak	quickfix.txt	/*:lmak*
:lmake	quickfix.txt	/*:lmake*
:lmap	map.txt	/*:lmap*
:lmap_l	map.txt	/*:lmap_l*
:lmapc	map.txt	/*:lmapc*
:lmapclear	map.txt	/*:lmapclear*
:ln	map.txt	/*:ln*
:lne	quickfix.txt	/*:lne*
:lnew	quickfix.txt	/*:lnew*
:lnewer	quickfix.txt	/*:lnewer*
:lnext	quickfix.txt	/*:lnext*
:lnf	quickfix.txt	/*:lnf*
:lnfile	quickfix.txt	/*:lnfile*
:lnoremap	map.txt	/*:lnoremap*
:lo	starting.txt	/*:lo*
:loadk	mbyte.txt	/*:loadk*
:loadkeymap	mbyte.txt	/*:loadkeymap*
:loadview	starting.txt	/*:loadview*
:loc	motion.txt	/*:loc*
:lock	motion.txt	/*:lock*
:lockmarks	motion.txt	/*:lockmarks*
:lockv	eval.txt	/*:lockv*
:lockvar	eval.txt	/*:lockvar*
:lol	quickfix.txt	/*:lol*
:lolder	quickfix.txt	/*:lolder*
:lop	quickfix.txt	/*:lop*
:lopen	quickfix.txt	/*:lopen*

:lp	quickfix.txt	/*:lp*
:lpf	quickfix.txt	/*:lpf*
:lpfile	quickfix.txt	/*:lpfile*
:lprev	quickfix.txt	/*:lprev*
:lprevious	quickfix.txt	/*:lprevious*
:lr	quickfix.txt	/*:lr*
:lrewind	quickfix.txt	/*:lrewind*
:ls	windows.txt	/*:ls*
:lt	tagsrch.txt	/*:lt*
:ltag	tagsrch.txt	/*:ltag*
:lu	map.txt	/*:lu*
:lua	if_lua.txt	/*:lua*
:luado	if_lua.txt	/*:luado*
:luafile	if_lua.txt	/*:luafile*
:lunmap	map.txt	/*:lunmap*
:lv	quickfix.txt	/*:lv*
:lvimgrep	quickfix.txt	/*:lvimgrep*
:lvimgrepa	quickfix.txt	/*:lvimgrepa*
:lvimgrepadd	quickfix.txt	/*:lvimgrepadd*
:lw	quickfix.txt	/*:lw*
:lwindow	quickfix.txt	/*:lwindow*
:m	change.txt	/*:m*
:ma	motion.txt	/*:ma*
:mak	quickfix.txt	/*:mak*
:make	quickfix.txt	/*:make*
:make_makeprg	quickfix.txt	/*:make_makeprg*
:map	map.txt	/*:map*
:map!	map.txt	/*:map!*
:map-<buffer>	map.txt	/*:map-<buffer>*
:map-<expr>	map.txt	/*:map-<expr>*
:map-<nowait>	map.txt	/*:map-<nowait>*
:map-<script>	map.txt	/*:map-<script>*
:map-<silent>	map.txt	/*:map-<silent>*
:map-<special>	map.txt	/*:map-<special>*
:map-<unique>	map.txt	/*:map-<unique>*
:map-alt-keys	map.txt	/*:map-alt-keys*
:map-arguments	map.txt	/*:map-arguments*
:map-commands	map.txt	/*:map-commands*
:map-expression	map.txt	/*:map-expression*
:map-local	map.txt	/*:map-local*
:map-modes	map.txt	/*:map-modes*
:map-nowait	map.txt	/*:map-nowait*
:map-operator	map.txt	/*:map-operator*
:map-script	map.txt	/*:map-script*
:map-silent	map.txt	/*:map-silent*
:map-special	map.txt	/*:map-special*
:map-special-chars	map.txt	/*:map-special-chars*
:map-special-keys	map.txt	/*:map-special-keys*
:map-undo	map.txt	/*:map-undo*
:map-verbose	map.txt	/*:map-verbose*
:map_l	map.txt	/*:map_l*
:map_l!	map.txt	/*:map_l!*
:mapc	map.txt	/*:mapc*
:mapc!	map.txt	/*:mapc!*

:mapclear map.txt /*:mapclear*
:mapclear! map.txt /*:mapclear!*
:mark motion.txt /*:mark*
:marks motion.txt /*:marks*
:mat pattern.txt /*:mat*
:match pattern.txt /*:match*
:me gui.txt /*:me*
:menu gui.txt /*:menu*
:menu-<script> gui.txt /*:menu-<script>*
:menu-<silent> gui.txt /*:menu-<silent>*
:menu-<special> gui.txt /*:menu-<special>*
:menu-disable gui.txt /*:menu-disable*
:menu-enable gui.txt /*:menu-enable*
:menu-script gui.txt /*:menu-script*
:menu-silent gui.txt /*:menu-silent*
:menu-special gui.txt /*:menu-special*
:menut mlang.txt /*:menut*
:menutrans mlang.txt /*:menutrans*
:menutranslate mlang.txt /*:menutranslate*
:mes message.txt /*:mes*
:messages message.txt /*:messages*
:mk starting.txt /*:mk*
:mkexrc starting.txt /*:mkexrc*
:mks starting.txt /*:mks*
:mksession starting.txt /*:mksession*
:mksp spell.txt /*:mksp*
:mkspell spell.txt /*:mkspell*
:mkv starting.txt /*:mkv*
:mkvi starting.txt /*:mkvi*
:mkvie starting.txt /*:mkvie*
:mkview starting.txt /*:mkview*
:mkvimrc starting.txt /*:mkvimrc*
:mo change.txt /*:mo*
:mod term.txt /*:mod*
:mode term.txt /*:mode*
:move change.txt /*:move*
:mz if_mzsch.txt /*:mz*
:mzf if_mzsch.txt /*:mzf*
:mzfile if_mzsch.txt /*:mzfile*
:mzscheme if_mzsch.txt /*:mzscheme*
:n editing.txt /*:n*
:nbclose netbeans.txt /*:nbclose*
:nbkey netbeans.txt /*:nbkey*
:nbstart netbeans.txt /*:nbstart*
:ne editing.txt /*:ne*
:new windows.txt /*:new*
:next editing.txt /*:next*
:next_f editing.txt /*:next_f*
:nm map.txt /*:nm*
:nmap map.txt /*:nmap*
:nmap_l map.txt /*:nmap_l*
:nmapc map.txt /*:nmapc*
:nmapclear map.txt /*:nmapclear*
:nme gui.txt /*:nme*

:nmenu gui.txt /*:nmenu*
:nn map.txt /*:nn*
:nnoremap map.txt /*:nnoremap*
:nnoreme gui.txt /*:nnoreme*
:nnoremenu gui.txt /*:nnoremenu*
:no map.txt /*:no*
:no! map.txt /*:no!*
:noa autocmd.txt /*:noa*
:noautocmd autocmd.txt /*:noautocmd*
:noh pattern.txt /*:noh*
:nohlsearch pattern.txt /*:nohlsearch*
:nor map.txt /*:nor*
:nore map.txt /*:nore*
:norea map.txt /*:norea*
:noreabbrev map.txt /*:noreabbrev*
:norem map.txt /*:norem*
:noremap map.txt /*:noremap*
:noremap! map.txt /*:noremap!*
:noreme gui.txt /*:noreme*
:noremenu gui.txt /*:noremenu*
:norm various.txt /*:norm*
:normal various.txt /*:normal*
:normal-range various.txt /*:normal-range*
:nos recover.txt /*:nos*
:noswapfile recover.txt /*:noswapfile*
:nu various.txt /*:nu*
:number various.txt /*:number*
:nun map.txt /*:nun*
:nunmap map.txt /*:nunmap*
:nunme gui.txt /*:nunme*
:nunmenu gui.txt /*:nunmenu*
:o vi_diff.txt /*:o*
:ol starting.txt /*:ol*
:oldfiles starting.txt /*:oldfiles*
:om map.txt /*:om*
:omap map.txt /*:omap*
:omap_l map.txt /*:omap_l*
:omapc map.txt /*:omapc*
:omapclear map.txt /*:omapclear*
:ome gui.txt /*:ome*
:omenu gui.txt /*:omenu*
:on windows.txt /*:on*
:only windows.txt /*:only*
:ono map.txt /*:ono*
:onoremap map.txt /*:onoremap*
:onoreme gui.txt /*:onoreme*
:onoremenu gui.txt /*:onoremenu*
:op vi_diff.txt /*:op*
:open vi_diff.txt /*:open*
:opt options.txt /*:opt*
:options options.txt /*:options*
:ou map.txt /*:ou*
:ounmap map.txt /*:ounmap*
:ounme gui.txt /*:ounme*

:ounmenu	gui.txt	/*:ounmenu*
:ownsyntax	syntax.txt	/*:ownsyntax*
:p	various.txt	/*:p*
:pa	repeat.txt	/*:pa*
:packadd	repeat.txt	/*:packadd*
:packl	repeat.txt	/*:packl*
:packloadall	repeat.txt	/*:packloadall*
:pc	windows.txt	/*:pc*
:pclose	windows.txt	/*:pclose*
:pe	if_perl.txt	/*:pe*
:ped	windows.txt	/*:ped*
:pedit	windows.txt	/*:pedit*
:perl	if_perl.txt	/*:perl*
:perld	if_perl.txt	/*:perld*
:perldo	if_perl.txt	/*:perldo*
:po	tagsrch.txt	/*:po*
:pop	tagsrch.txt	/*:pop*
:popu	gui.txt	/*:popu*
:popup	gui.txt	/*:popup*
:pp	windows.txt	/*:pp*
:ppop	windows.txt	/*:ppop*
:pr	various.txt	/*:pr*
:pre	recover.txt	/*:pre*
:preserve	recover.txt	/*:preserve*
:prev	editing.txt	/*:prev*
:previous	editing.txt	/*:previous*
:print	various.txt	/*:print*
:pro	change.txt	/*:pro*
:prof	repeat.txt	/*:prof*
:profd	repeat.txt	/*:profd*
:profdel	repeat.txt	/*:profdel*
:profile	repeat.txt	/*:profile*
:promptfind	change.txt	/*:promptfind*
:promptr	change.txt	/*:promptr*
:promptrepl	change.txt	/*:promptrepl*
:ps	windows.txt	/*:ps*
:psearch	windows.txt	/*:psearch*
:ptN	tagsrch.txt	/*:ptN*
:ptNext	tagsrch.txt	/*:ptNext*
:pta	windows.txt	/*:pta*
:ptag	windows.txt	/*:ptag*
:ptf	tagsrch.txt	/*:ptf*
:ptfirst	tagsrch.txt	/*:ptfirst*
:ptj	tagsrch.txt	/*:ptj*
:ptjump	tagsrch.txt	/*:ptjump*
:ptl	tagsrch.txt	/*:ptl*
:ptlast	tagsrch.txt	/*:ptlast*
:ptn	tagsrch.txt	/*:ptn*
:ptnext	tagsrch.txt	/*:ptnext*
:ptp	tagsrch.txt	/*:ptp*
:ptprevious	tagsrch.txt	/*:ptprevious*
:ptr	tagsrch.txt	/*:ptr*
:ptrewind	tagsrch.txt	/*:ptrewind*
:pts	tagsrch.txt	/*:pts*

:ptselect	tagsrch.txt	/*:ptselect*
:pu	change.txt	/*:pu*
:put	change.txt	/*:put*
:pw	editing.txt	/*:pw*
:pwd	editing.txt	/*:pwd*
:py	if_pyth.txt	/*:py*
:py3	if_pyth.txt	/*:py3*
:py3do	if_pyth.txt	/*:py3do*
:py3file	if_pyth.txt	/*:py3file*
:pydo	if_pyth.txt	/*:pydo*
:pyf	if_pyth.txt	/*:pyf*
:pyfile	if_pyth.txt	/*:pyfile*
:python	if_pyth.txt	/*:python*
:python3	if_pyth.txt	/*:python3*
:pythonx	if_pyth.txt	/*:pythonx*
:pyx	if_pyth.txt	/*:pyx*
:pyxdo	if_pyth.txt	/*:pyxdo*
:pyxfile	if_pyth.txt	/*:pyxfile*
:q	editing.txt	/*:q*
:qa	editing.txt	/*:qa*
:qall	editing.txt	/*:qall*
:quit	editing.txt	/*:quit*
:quita	editing.txt	/*:quita*
:quitall	editing.txt	/*:quitall*
:quote	cmdline.txt	/*:quote*
:r	insert.txt	/*:r*
:r!	insert.txt	/*:r!*
:range	cmdline.txt	/*:range*
:range!	change.txt	/*:range!*
:re	insert.txt	/*:re*
:read	insert.txt	/*:read*
:read!	insert.txt	/*:read!*
:rec	recover.txt	/*:rec*
:recover	recover.txt	/*:recover*
:recover-crypt	recover.txt	/*:recover-crypt*
:red	undo.txt	/*:red*
:redi	various.txt	/*:redi*
:redir	various.txt	/*:redir*
:redo	undo.txt	/*:redo*
:redr	various.txt	/*:redr*
:redraw	various.txt	/*:redraw*
:redraws	various.txt	/*:redraws*
:redrawstatus	various.txt	/*:redrawstatus*
:redrawt	various.txt	/*:redrawt*
:redrawtabline	various.txt	/*:redrawtabline*
:reg	change.txt	/*:reg*
:registers	change.txt	/*:registers*
:res	windows.txt	/*:res*
:resize	windows.txt	/*:resize*
:ret	change.txt	/*:ret*
:retab	change.txt	/*:retab*
:retab!	change.txt	/*:retab!*
:retu	eval.txt	/*:retu*
:return	eval.txt	/*:return*

```
:rew                    editing.txt             /*:rew*
:rewind                 editing.txt             /*:rewind*
:ri                     change.txt              /*:ri*
:right                  change.txt              /*:right*
:rightb                 windows.txt             /*:rightb*
:rightbelow             windows.txt             /*:rightbelow*
:ru                     repeat.txt              /*:ru*
:rub                    if_ruby.txt             /*:rub*
:ruby                   if_ruby.txt             /*:ruby*
:rubyd                  if_ruby.txt             /*:rubyd*
:rubydo                 if_ruby.txt             /*:rubydo*
:rubyf                  if_ruby.txt             /*:rubyf*
:rubyfile               if_ruby.txt             /*:rubyfile*
:rundo                  undo.txt                /*:rundo*
:runtime                repeat.txt              /*:runtime*
:rv                     starting.txt            /*:rv*
:rviminfo               starting.txt            /*:rviminfo*
:s                      change.txt              /*:s*
:s%                     change.txt              /*:s%*
:sI                     change.txt              /*:sI*
:sIc                    change.txt              /*:sIc*
:sIe                    change.txt              /*:sIe*
:sIg                    change.txt              /*:sIg*
:sIl                    change.txt              /*:sIl*
:sIn                    change.txt              /*:sIn*
:sIp                    change.txt              /*:sIp*
:sIr                    change.txt              /*:sIr*
:sN                     windows.txt             /*:sN*
:sNext                  windows.txt             /*:sNext*
:s\=                    change.txt              /*:s\\=*
:s_#                    change.txt              /*:s_#*
:s_I                    change.txt              /*:s_I*
:s_c                    change.txt              /*:s_c*
:s_e                    change.txt              /*:s_e*
:s_flags                change.txt              /*:s_flags*
:s_g                    change.txt              /*:s_g*
:s_i                    change.txt              /*:s_i*
:s_l                    change.txt              /*:s_l*
:s_n                    change.txt              /*:s_n*
:s_p                    change.txt              /*:s_p*
:s_r                    change.txt              /*:s_r*
:sa                     windows.txt             /*:sa*
:sal                    windows.txt             /*:sal*
:sall                   windows.txt             /*:sall*
:san                    eval.txt                /*:san*
:sandbox                eval.txt                /*:sandbox*
:sargument              windows.txt             /*:sargument*
:sav                    editing.txt             /*:sav*
:saveas                 editing.txt             /*:saveas*
:sb                     windows.txt             /*:sb*
:sbN                    windows.txt             /*:sbN*
:sbNext                 windows.txt             /*:sbNext*
:sba                    windows.txt             /*:sba*
:sball                  windows.txt             /*:sball*
```

:sbf	windows.txt	/*:sbf*
:sbfirst	windows.txt	/*:sbfirst*
:sbl	windows.txt	/*:sbl*
:sblast	windows.txt	/*:sblast*
:sbm	windows.txt	/*:sbm*
:sbmodified	windows.txt	/*:sbmodified*
:sbn	windows.txt	/*:sbn*
:sbnext	windows.txt	/*:sbnext*
:sbp	windows.txt	/*:sbp*
:sbprevious	windows.txt	/*:sbprevious*
:sbr	windows.txt	/*:sbr*
:sbrewind	windows.txt	/*:sbrewind*
:sbuffer	windows.txt	/*:sbuffer*
:sc	change.txt	/*:sc*
:scI	change.txt	/*:scI*
:sce	change.txt	/*:sce*
:scg	change.txt	/*:scg*
:sci	change.txt	/*:sci*
:scl	change.txt	/*:scl*
:scp	change.txt	/*:scp*
:scr	repeat.txt	/*:scr*
:script	repeat.txt	/*:script*
:scripte	repeat.txt	/*:scripte*
:scriptencoding	repeat.txt	/*:scriptencoding*
:scriptnames	repeat.txt	/*:scriptnames*
:scriptv	repeat.txt	/*:scriptv*
:scriptversion	repeat.txt	/*:scriptversion*
:scs	if_cscop.txt	/*:scs*
:scscope	if_cscop.txt	/*:scscope*
:se	options.txt	/*:se*
:search-args	tagsrch.txt	/*:search-args*
:set	options.txt	/*:set*
:set+=	options.txt	/*:set+=*
:set-!	options.txt	/*:set-!*
:set-&	options.txt	/*:set-&*
:set-&vi	options.txt	/*:set-&vi*
:set-&vim	options.txt	/*:set-&vim*
:set-=	options.txt	/*:set-=*
:set-args	options.txt	/*:set-args*
:set-browse	options.txt	/*:set-browse*
:set-default	options.txt	/*:set-default*
:set-inv	options.txt	/*:set-inv*
:set-termcap	options.txt	/*:set-termcap*
:set-verbose	options.txt	/*:set-verbose*
:set^=	options.txt	/*:set^=*
:set_env	options.txt	/*:set_env*
:setf	options.txt	/*:setf*
:setfiletype	options.txt	/*:setfiletype*
:setg	options.txt	/*:setg*
:setglobal	options.txt	/*:setglobal*
:setl	options.txt	/*:setl*
:setlocal	options.txt	/*:setlocal*
:sf	windows.txt	/*:sf*
:sfi	windows.txt	/*:sfi*

:sfind	windows.txt	/*:sfind*
:sfir	windows.txt	/*:sfir*
:sfirst	windows.txt	/*:sfirst*
:sg	change.txt	/*:sg*
:sgI	change.txt	/*:sgI*
:sgc	change.txt	/*:sgc*
:sge	change.txt	/*:sge*
:sgi	change.txt	/*:sgi*
:sgl	change.txt	/*:sgl*
:sgn	change.txt	/*:sgn*
:sgp	change.txt	/*:sgp*
:sgr	change.txt	/*:sgr*
:sh	various.txt	/*:sh*
:shell	various.txt	/*:shell*
:si	change.txt	/*:si*
:sic	change.txt	/*:sic*
:sie	change.txt	/*:sie*
:sig	sign.txt	/*:sig*
:sign	sign.txt	/*:sign*
:sign-define	sign.txt	/*:sign-define*
:sign-fname	sign.txt	/*:sign-fname*
:sign-jump	sign.txt	/*:sign-jump*
:sign-list	sign.txt	/*:sign-list*
:sign-place	sign.txt	/*:sign-place*
:sign-place-list	sign.txt	/*:sign-place-list*
:sign-undefine	sign.txt	/*:sign-undefine*
:sign-unplace	sign.txt	/*:sign-unplace*
:sil	various.txt	/*:sil*
:silent	various.txt	/*:silent*
:silent!	various.txt	/*:silent!*
:sim	gui_w32.txt	/*:sim*
:simalt	gui_w32.txt	/*:simalt*
:sin	change.txt	/*:sin*
:sip	change.txt	/*:sip*
:sir	change.txt	/*:sir*
:sl	various.txt	/*:sl*
:sla	windows.txt	/*:sla*
:slast	windows.txt	/*:slast*
:sleep	various.txt	/*:sleep*
:sm	change.txt	/*:sm*
:smagic	change.txt	/*:smagic*
:smap	map.txt	/*:smap*
:smap_l	map.txt	/*:smap_l*
:smapc	map.txt	/*:smapc*
:smapclear	map.txt	/*:smapclear*
:sme	gui.txt	/*:sme*
:smenu	gui.txt	/*:smenu*
:smile	index.txt	/*:smile*
:sn	windows.txt	/*:sn*
:snext	windows.txt	/*:snext*
:sno	change.txt	/*:sno*
:snomagic	change.txt	/*:snomagic*
:snor	map.txt	/*:snor*
:snore	map.txt	/*:snore*

:snoremap	map.txt	/*:snoremap*
:snoreme	gui.txt	/*:snoreme*
:snoremenu	gui.txt	/*:snoremenu*
:so	repeat.txt	/*:so*
:sor	change.txt	/*:sor*
:sort	change.txt	/*:sort*
:source	repeat.txt	/*:source*
:source!	repeat.txt	/*:source!*
:source_crnl	repeat.txt	/*:source_crnl*
:sp	windows.txt	/*:sp*
:spe	spell.txt	/*:spe*
:spelld	spell.txt	/*:spelld*
:spelldump	spell.txt	/*:spelldump*
:spellgood	spell.txt	/*:spellgood*
:spelli	spell.txt	/*:spelli*
:spellinfo	spell.txt	/*:spellinfo*
:spellr	spell.txt	/*:spellr*
:spellra	spell.txt	/*:spellra*
:spellrare	spell.txt	/*:spellrare*
:spellrepall	spell.txt	/*:spellrepall*
:spellu	spell.txt	/*:spellu*
:spellundo	spell.txt	/*:spellundo*
:spellw	spell.txt	/*:spellw*
:spellwrong	spell.txt	/*:spellwrong*
:split	windows.txt	/*:split*
:split_f	windows.txt	/*:split_f*
:spr	windows.txt	/*:spr*
:sprevious	windows.txt	/*:sprevious*
:sr	change.txt	/*:sr*
:srI	change.txt	/*:srI*
:src	change.txt	/*:src*
:sre	windows.txt	/*:sre*
:srewind	windows.txt	/*:srewind*
:srg	change.txt	/*:srg*
:sri	change.txt	/*:sri*
:srl	change.txt	/*:srl*
:srn	change.txt	/*:srn*
:srp	change.txt	/*:srp*
:st	starting.txt	/*:st*
:sta	windows.txt	/*:sta*
:stag	windows.txt	/*:stag*
:star	repeat.txt	/*:star*
:start	insert.txt	/*:start*
:startgreplace	insert.txt	/*:startgreplace*
:startinsert	insert.txt	/*:startinsert*
:startreplace	insert.txt	/*:startreplace*
:stj	tagsrch.txt	/*:stj*
:stjump	tagsrch.txt	/*:stjump*
:stop	starting.txt	/*:stop*
:stopi	insert.txt	/*:stopi*
:stopinsert	insert.txt	/*:stopinsert*
:sts	tagsrch.txt	/*:sts*
:stselect	tagsrch.txt	/*:stselect*
:su	change.txt	/*:su*

:substitute	change.txt	/*:substitute*
:sun	windows.txt	/*:sun*
:sunhide	windows.txt	/*:sunhide*
:sunm	map.txt	/*:sunm*
:sunmap	map.txt	/*:sunmap*
:sunme	gui.txt	/*:sunme*
:sunmenu	gui.txt	/*:sunmenu*
:sus	starting.txt	/*:sus*
:suspend	starting.txt	/*:suspend*
:sv	windows.txt	/*:sv*
:sview	windows.txt	/*:sview*
:sw	recover.txt	/*:sw*
:swapname	recover.txt	/*:swapname*
:sy	syntax.txt	/*:sy*
:syn	syntax.txt	/*:syn*
:syn-arguments	syntax.txt	/*:syn-arguments*
:syn-case	syntax.txt	/*:syn-case*
:syn-cchar	syntax.txt	/*:syn-cchar*
:syn-clear	syntax.txt	/*:syn-clear*
:syn-cluster	syntax.txt	/*:syn-cluster*
:syn-conceal	syntax.txt	/*:syn-conceal*
:syn-conceal-implicit	syntax.txt	/*:syn-conceal-implicit*
:syn-concealends	syntax.txt	/*:syn-concealends*
:syn-contained	syntax.txt	/*:syn-contained*
:syn-containedin	syntax.txt	/*:syn-containedin*
:syn-contains	syntax.txt	/*:syn-contains*
:syn-context	syntax.txt	/*:syn-context*
:syn-default-override	usr_06.txt	/*:syn-default-override*
:syn-define	syntax.txt	/*:syn-define*
:syn-display	syntax.txt	/*:syn-display*
:syn-enable	syntax.txt	/*:syn-enable*
:syn-end	syntax.txt	/*:syn-end*
:syn-excludenl	syntax.txt	/*:syn-excludenl*
:syn-ext-match	syntax.txt	/*:syn-ext-match*
:syn-extend	syntax.txt	/*:syn-extend*
:syn-file-remarks	syntax.txt	/*:syn-file-remarks*
:syn-files	syntax.txt	/*:syn-files*
:syn-fold	syntax.txt	/*:syn-fold*
:syn-include	syntax.txt	/*:syn-include*
:syn-iskeyword	syntax.txt	/*:syn-iskeyword*
:syn-keepend	syntax.txt	/*:syn-keepend*
:syn-keyword	syntax.txt	/*:syn-keyword*
:syn-lc	syntax.txt	/*:syn-lc*
:syn-leading	syntax.txt	/*:syn-leading*
:syn-list	syntax.txt	/*:syn-list*
:syn-manual	usr_06.txt	/*:syn-manual*
:syn-match	syntax.txt	/*:syn-match*
:syn-matchgroup	syntax.txt	/*:syn-matchgroup*
:syn-multi-line	syntax.txt	/*:syn-multi-line*
:syn-nextgroup	syntax.txt	/*:syn-nextgroup*
:syn-off	syntax.txt	/*:syn-off*
:syn-on	syntax.txt	/*:syn-on*
:syn-oneline	syntax.txt	/*:syn-oneline*
:syn-pattern	syntax.txt	/*:syn-pattern*

:syn-pattern-offset	syntax.txt	/*:syn-pattern-offset*
:syn-priority	syntax.txt	/*:syn-priority*
:syn-qstart	syntax.txt	/*:syn-qstart*
:syn-region	syntax.txt	/*:syn-region*
:syn-reset	syntax.txt	/*:syn-reset*
:syn-skip	syntax.txt	/*:syn-skip*
:syn-skipempty	syntax.txt	/*:syn-skipempty*
:syn-skipnl	syntax.txt	/*:syn-skipnl*
:syn-skipwhite	syntax.txt	/*:syn-skipwhite*
:syn-spell	syntax.txt	/*:syn-spell*
:syn-start	syntax.txt	/*:syn-start*
:syn-sync	syntax.txt	/*:syn-sync*
:syn-sync-ccomment	syntax.txt	/*:syn-sync-ccomment*
:syn-sync-first	syntax.txt	/*:syn-sync-first*
:syn-sync-fourth	syntax.txt	/*:syn-sync-fourth*
:syn-sync-linebreaks	syntax.txt	/*:syn-sync-linebreaks*
:syn-sync-maxlines	syntax.txt	/*:syn-sync-maxlines*
:syn-sync-minlines	syntax.txt	/*:syn-sync-minlines*
:syn-sync-second	syntax.txt	/*:syn-sync-second*
:syn-sync-third	syntax.txt	/*:syn-sync-third*
:syn-transparent	syntax.txt	/*:syn-transparent*
:sync	scroll.txt	/*:sync*
:syncbind	scroll.txt	/*:syncbind*
:syntax	syntax.txt	/*:syntax*
:syntax-enable	syntax.txt	/*:syntax-enable*
:syntax-off	syntax.txt	/*:syntax-off*
:syntax-on	syntax.txt	/*:syntax-on*
:syntax-reset	syntax.txt	/*:syntax-reset*
:syntime	syntax.txt	/*:syntime*
:t	change.txt	/*:t*
:tN	tagsrch.txt	/*:tN*
:tNext	tagsrch.txt	/*:tNext*
:ta	tagsrch.txt	/*:ta*
:tab	tabpage.txt	/*:tab*
:tabN	tabpage.txt	/*:tabN*
:tabNext	tabpage.txt	/*:tabNext*
:tabc	tabpage.txt	/*:tabc*
:tabclose	tabpage.txt	/*:tabclose*
:tabd	tabpage.txt	/*:tabd*
:tabdo	tabpage.txt	/*:tabdo*
:tabe	tabpage.txt	/*:tabe*
:tabedit	tabpage.txt	/*:tabedit*
:tabf	tabpage.txt	/*:tabf*
:tabfind	tabpage.txt	/*:tabfind*
:tabfir	tabpage.txt	/*:tabfir*
:tabfirst	tabpage.txt	/*:tabfirst*
:tabl	tabpage.txt	/*:tabl*
:tablast	tabpage.txt	/*:tablast*
:tabm	tabpage.txt	/*:tabm*
:tabmove	tabpage.txt	/*:tabmove*
:tabn	tabpage.txt	/*:tabn*
:tabnew	tabpage.txt	/*:tabnew*
:tabnext	tabpage.txt	/*:tabnext*
:tabo	tabpage.txt	/*:tabo*

:tabonly	tabpage.txt	/*:tabonly*
:tabp	tabpage.txt	/*:tabp*
:tabprevious	tabpage.txt	/*:tabprevious*
:tabr	tabpage.txt	/*:tabr*
:tabrewind	tabpage.txt	/*:tabrewind*
:tabs	tabpage.txt	/*:tabs*
:tag	tagsrch.txt	/*:tag*
:tags	tagsrch.txt	/*:tags*
:tc	if_tcl.txt	/*:tc*
:tcd	editing.txt	/*:tcd*
:tch	editing.txt	/*:tch*
:tchdir	editing.txt	/*:tchdir*
:tcl	if_tcl.txt	/*:tcl*
:tcld	if_tcl.txt	/*:tcld*
:tcldo	if_tcl.txt	/*:tcldo*
:tclf	if_tcl.txt	/*:tclf*
:tclfile	if_tcl.txt	/*:tclfile*
:te	gui_w32.txt	/*:te*
:tearoff	gui_w32.txt	/*:tearoff*
:ter	terminal.txt	/*:ter*
:terminal	terminal.txt	/*:terminal*
:tf	tagsrch.txt	/*:tf*
:tfirst	tagsrch.txt	/*:tfirst*
:th	eval.txt	/*:th*
:throw	eval.txt	/*:throw*
:tj	tagsrch.txt	/*:tj*
:tjump	tagsrch.txt	/*:tjump*
:tl	tagsrch.txt	/*:tl*
:tlast	tagsrch.txt	/*:tlast*
:tlm	gui.txt	/*:tlm*
:tlmenu	gui.txt	/*:tlmenu*
:tln	gui.txt	/*:tln*
:tlnoremenu	gui.txt	/*:tlnoremenu*
:tlu	gui.txt	/*:tlu*
:tlunmenu	gui.txt	/*:tlunmenu*
:tm	gui.txt	/*:tm*
:tma	map.txt	/*:tma*
:tmap	map.txt	/*:tmap*
:tmap_l	map.txt	/*:tmap_l*
:tmapc	map.txt	/*:tmapc*
:tmapclear	map.txt	/*:tmapclear*
:tmenu	gui.txt	/*:tmenu*
:tn	tagsrch.txt	/*:tn*
:tnext	tagsrch.txt	/*:tnext*
:tno	map.txt	/*:tno*
:tnoremap	map.txt	/*:tnoremap*
:topleft	windows.txt	/*:topleft*
:tp	tagsrch.txt	/*:tp*
:tprevious	tagsrch.txt	/*:tprevious*
:tr	tagsrch.txt	/*:tr*
:trewind	tagsrch.txt	/*:trewind*
:try	eval.txt	/*:try*
:ts	tagsrch.txt	/*:ts*
:tselect	tagsrch.txt	/*:tselect*

:tu	gui.txt	/*:tu*
:tunma	map.txt	/*:tunma*
:tunmap	map.txt	/*:tunmap*
:tunmenu	gui.txt	/*:tunmenu*
:u	undo.txt	/*:u*
:un	undo.txt	/*:un*
:una	map.txt	/*:una*
:unabbreviate	map.txt	/*:unabbreviate*
:undo	undo.txt	/*:undo*
:undoj	undo.txt	/*:undoj*
:undojoin	undo.txt	/*:undojoin*
:undol	undo.txt	/*:undol*
:undolist	undo.txt	/*:undolist*
:unh	windows.txt	/*:unh*
:unhide	windows.txt	/*:unhide*
:unl	eval.txt	/*:unl*
:unlet	eval.txt	/*:unlet*
:unlet-$	eval.txt	/*:unlet-$*
:unlet-environment	eval.txt	/*:unlet-environment*
:unlo	eval.txt	/*:unlo*
:unlockvar	eval.txt	/*:unlockvar*
:unm	map.txt	/*:unm*
:unm!	map.txt	/*:unm!*
:unmap	map.txt	/*:unmap*
:unmap!	map.txt	/*:unmap!*
:unme	gui.txt	/*:unme*
:unmenu	gui.txt	/*:unmenu*
:unmenu-all	gui.txt	/*:unmenu-all*
:uns	various.txt	/*:uns*
:unsilent	various.txt	/*:unsilent*
:up	editing.txt	/*:up*
:update	editing.txt	/*:update*
:v	repeat.txt	/*:v*
:ve	various.txt	/*:ve*
:ver	various.txt	/*:ver*
:verb	various.txt	/*:verb*
:verbose	various.txt	/*:verbose*
:verbose-cmd	various.txt	/*:verbose-cmd*
:version	various.txt	/*:version*
:vert	windows.txt	/*:vert*
:vertical	windows.txt	/*:vertical*
:vertical-resize	windows.txt	/*:vertical-resize*
:vglobal	repeat.txt	/*:vglobal*
:vi	editing.txt	/*:vi*
:vie	editing.txt	/*:vie*
:view	editing.txt	/*:view*
:vim	quickfix.txt	/*:vim*
:vimgrep	quickfix.txt	/*:vimgrep*
:vimgrepa	quickfix.txt	/*:vimgrepa*
:vimgrepadd	quickfix.txt	/*:vimgrepadd*
:visual	editing.txt	/*:visual*
:visual_example	visual.txt	/*:visual_example*
:viu	helphelp.txt	/*:viu*
:viusage	helphelp.txt	/*:viusage*

:vm	map.txt	/*:vm*
:vmap	map.txt	/*:vmap*
:vmap_l	map.txt	/*:vmap_l*
:vmapc	map.txt	/*:vmapc*
:vmapclear	map.txt	/*:vmapclear*
:vme	gui.txt	/*:vme*
:vmenu	gui.txt	/*:vmenu*
:vn	map.txt	/*:vn*
:vne	windows.txt	/*:vne*
:vnew	windows.txt	/*:vnew*
:vnoremap	map.txt	/*:vnoremap*
:vnoreme	gui.txt	/*:vnoreme*
:vnoremenu	gui.txt	/*:vnoremenu*
:vs	windows.txt	/*:vs*
:vsplit	windows.txt	/*:vsplit*
:vu	map.txt	/*:vu*
:vunmap	map.txt	/*:vunmap*
:vunme	gui.txt	/*:vunme*
:vunmenu	gui.txt	/*:vunmenu*
:w	editing.txt	/*:w*
:w!	editing.txt	/*:w!*
:wN	editing.txt	/*:wN*
:wNext	editing.txt	/*:wNext*
:w_a	editing.txt	/*:w_a*
:w_c	editing.txt	/*:w_c*
:w_f	editing.txt	/*:w_f*
:wa	editing.txt	/*:wa*
:wall	editing.txt	/*:wall*
:wh	eval.txt	/*:wh*
:while	eval.txt	/*:while*
:win	gui.txt	/*:win*
:winc	windows.txt	/*:winc*
:wincmd	windows.txt	/*:wincmd*
:windo	windows.txt	/*:windo*
:winp	gui.txt	/*:winp*
:winpos	gui.txt	/*:winpos*
:winsize	gui.txt	/*:winsize*
:wn	editing.txt	/*:wn*
:wnext	editing.txt	/*:wnext*
:wp	editing.txt	/*:wp*
:wprevious	editing.txt	/*:wprevious*
:wq	editing.txt	/*:wq*
:wqa	editing.txt	/*:wqa*
:wqall	editing.txt	/*:wqall*
:write	editing.txt	/*:write*
:write_a	editing.txt	/*:write_a*
:write_c	editing.txt	/*:write_c*
:write_f	editing.txt	/*:write_f*
:wundo	undo.txt	/*:wundo*
:wv	starting.txt	/*:wv*
:wviminfo	starting.txt	/*:wviminfo*
:x	editing.txt	/*:x*
:xa	editing.txt	/*:xa*
:xall	editing.txt	/*:xall*

:xit	editing.txt	/*:xit*
:xm	map.txt	/*:xm*
:xmap	map.txt	/*:xmap*
:xmap_l	map.txt	/*:xmap_l*
:xmapc	map.txt	/*:xmapc*
:xmapclear	map.txt	/*:xmapclear*
:xme	gui.txt	/*:xme*
:xmenu	gui.txt	/*:xmenu*
:xn	map.txt	/*:xn*
:xnoremap	map.txt	/*:xnoremap*
:xnoreme	gui.txt	/*:xnoreme*
:xnoremenu	gui.txt	/*:xnoremenu*
:xr	various.txt	/*:xr*
:xrestore	various.txt	/*:xrestore*
:xu	map.txt	/*:xu*
:xunmap	map.txt	/*:xunmap*
:xunme	gui.txt	/*:xunme*
:xunmenu	gui.txt	/*:xunmenu*
:y	change.txt	/*:y*
:yank	change.txt	/*:yank*
:z	various.txt	/*:z*
:z#	various.txt	/*:z#*
:~	change.txt	/*:~*
;	motion.txt	/*;*
<	change.txt	/*<*
<2-LeftMouse>	term.txt	/*<2-LeftMouse>*
<3-LeftMouse>	term.txt	/*<3-LeftMouse>*
<4-LeftMouse>	term.txt	/*<4-LeftMouse>*
<<	change.txt	/*<<*
<>	intro.txt	/*<>*
<A-	intro.txt	/*<A-*
<A-LeftMouse>	term.txt	/*<A-LeftMouse>*
<A-RightMouse>	term.txt	/*<A-RightMouse>*
<BS>	motion.txt	/*<BS>*
<Bar>	intro.txt	/*<Bar>*
<Bslash>	intro.txt	/*<Bslash>*
<C-	intro.txt	/*<C-*
<C-Del>	os_dos.txt	/*<C-Del>*
<C-End>	motion.txt	/*<C-End>*
<C-Home>	motion.txt	/*<C-Home>*
<C-Insert>	os_dos.txt	/*<C-Insert>*
<C-Left>	motion.txt	/*<C-Left>*
<C-LeftMouse>	tagsrch.txt	/*<C-LeftMouse>*
<C-PageDown>	tabpage.txt	/*<C-PageDown>*
<C-PageUp>	tabpage.txt	/*<C-PageUp>*
<C-Right>	motion.txt	/*<C-Right>*
<C-RightMouse>	tagsrch.txt	/*<C-RightMouse>*
<C-ScrollWheelDown>	scroll.txt	/*<C-ScrollWheelDown>*
<C-ScrollWheelLeft>	scroll.txt	/*<C-ScrollWheelLeft>*
<C-ScrollWheelRight>	scroll.txt	/*<C-ScrollWheelRight>*
<C-ScrollWheelUp>	scroll.txt	/*<C-ScrollWheelUp>*
<CR>	motion.txt	/*<CR>*
<CSI>	intro.txt	/*<CSI>*
<Char->	map.txt	/*<Char->*

<Char> map.txt /*<Char>*
<CursorHold> autocmd.txt /*<CursorHold>*
<D- intro.txt /*<D-*
<D-c> os_mac.txt /*<D-c>*
<D-v> os_mac.txt /*<D-v>*
<D-x> os_mac.txt /*<D-x>*
<Del> change.txt /*<Del>*
<Down> motion.txt /*<Down>*
<Drop> change.txt /*<Drop>*
<EOL> intro.txt /*<EOL>*
<End> motion.txt /*<End>*
<Enter> intro.txt /*<Enter>*
<Esc> intro.txt /*<Esc>*
<F10> term.txt /*<F10>*
<F11> term.txt /*<F11>*
<F12> term.txt /*<F12>*
<F13> term.txt /*<F13>*
<F14> term.txt /*<F14>*
<F15> term.txt /*<F15>*
<F16> term.txt /*<F16>*
<F17> term.txt /*<F17>*
<F18> term.txt /*<F18>*
<F19> term.txt /*<F19>*
<F1> helphelp.txt /*<F1>*
<F2> term.txt /*<F2>*
<F3> term.txt /*<F3>*
<F4> term.txt /*<F4>*
<F5> term.txt /*<F5>*
<F6> term.txt /*<F6>*
<F7> term.txt /*<F7>*
<F8> term.txt /*<F8>*
<F9> term.txt /*<F9>*
<Help> helphelp.txt /*<Help>*
<Home> motion.txt /*<Home>*
<Ignore> eval.txt /*<Ignore>*
<Insert> insert.txt /*<Insert>*
<Leader> map.txt /*<Leader>*
<Left> motion.txt /*<Left>*
<LeftDrag> term.txt /*<LeftDrag>*
<LeftMouse> visual.txt /*<LeftMouse>*
<LeftRelease> visual.txt /*<LeftRelease>*
<LocalLeader> map.txt /*<LocalLeader>*
<M- intro.txt /*<M-*
<MiddleDrag> term.txt /*<MiddleDrag>*
<MiddleMouse> change.txt /*<MiddleMouse>*
<MiddleRelease> term.txt /*<MiddleRelease>*
<Mouse> term.txt /*<Mouse>*
<MouseDown> scroll.txt /*<MouseDown>*
<MouseUp> scroll.txt /*<MouseUp>*
<NL> motion.txt /*<NL>*
<Nop> map.txt /*<Nop>*
<Nul> intro.txt /*<Nul>*
<PageDown> scroll.txt /*<PageDown>*
<PageUp> scroll.txt /*<PageUp>*

<Plug> map.txt /*<Plug>*
<Return> intro.txt /*<Return>*
<Right> motion.txt /*<Right>*
<RightDrag> term.txt /*<RightDrag>*
<RightMouse> visual.txt /*<RightMouse>*
<RightRelease> term.txt /*<RightRelease>*
<S- intro.txt /*<S-*
<S-Del> os_dos.txt /*<S-Del>*
<S-Down> scroll.txt /*<S-Down>*
<S-End> term.txt /*<S-End>*
<S-F10> term.txt /*<S-F10>*
<S-F11> term.txt /*<S-F11>*
<S-F12> term.txt /*<S-F12>*
<S-F1> intro.txt /*<S-F1>*
<S-F2> term.txt /*<S-F2>*
<S-F3> term.txt /*<S-F3>*
<S-F4> term.txt /*<S-F4>*
<S-F5> term.txt /*<S-F5>*
<S-F6> term.txt /*<S-F6>*
<S-F7> term.txt /*<S-F7>*
<S-F8> term.txt /*<S-F8>*
<S-F9> term.txt /*<S-F9>*
<S-Home> term.txt /*<S-Home>*
<S-Insert> os_dos.txt /*<S-Insert>*
<S-Left> motion.txt /*<S-Left>*
<S-LeftMouse> term.txt /*<S-LeftMouse>*
<S-Right> motion.txt /*<S-Right>*
<S-RightMouse> term.txt /*<S-RightMouse>*
<S-ScrollWheelDown> scroll.txt /*<S-ScrollWheelDown>*
<S-ScrollWheelLeft> scroll.txt /*<S-ScrollWheelLeft>*
<S-ScrollWheelRight> scroll.txt /*<S-ScrollWheelRight>*
<S-ScrollWheelUp> scroll.txt /*<S-ScrollWheelUp>*
<S-Tab> term.txt /*<S-Tab>*
<S-Up> scroll.txt /*<S-Up>*
<S-xF1> term.txt /*<S-xF1>*
<S-xF2> term.txt /*<S-xF2>*
<S-xF3> term.txt /*<S-xF3>*
<S-xF4> term.txt /*<S-xF4>*
<SID> map.txt /*<SID>*
<SNR> map.txt /*<SNR>*
<ScrollWheelDown> scroll.txt /*<ScrollWheelDown>*
<ScrollWheelLeft> scroll.txt /*<ScrollWheelLeft>*
<ScrollWheelRight> scroll.txt /*<ScrollWheelRight>*
<ScrollWheelUp> scroll.txt /*<ScrollWheelUp>*
<Space> motion.txt /*<Space>*
<Tab> motion.txt /*<Tab>*
<Undo> undo.txt /*<Undo>*
<Up> motion.txt /*<Up>*
<abuf> cmdline.txt /*<abuf>*
<afile> cmdline.txt /*<afile>*
<amatch> cmdline.txt /*<amatch>*
<args> map.txt /*<args>*
<bang> map.txt /*<bang>*
<buffer=N> autocmd.txt /*<buffer=N>*

<buffer=abuf> autocmd.txt /*<buffer=abuf>*
<cWORD> cmdline.txt /*<cWORD>*
<cexpr> cmdline.txt /*<cexpr>*
<cfile> cmdline.txt /*<cfile>*
<character> intro.txt /*<character>*
<count> map.txt /*<count>*
<cword> cmdline.txt /*<cword>*
<f-args> map.txt /*<f-args>*
<k0> term.txt /*<k0>*
<k1> term.txt /*<k1>*
<k2> term.txt /*<k2>*
<k3> term.txt /*<k3>*
<k4> term.txt /*<k4>*
<k5> term.txt /*<k5>*
<k6> term.txt /*<k6>*
<k7> term.txt /*<k7>*
<k8> term.txt /*<k8>*
<k9> term.txt /*<k9>*
<kDivide> term.txt /*<kDivide>*
<kEnd> motion.txt /*<kEnd>*
<kEnter> term.txt /*<kEnter>*
<kHome> motion.txt /*<kHome>*
<kMinus> term.txt /*<kMinus>*
<kMultiply> term.txt /*<kMultiply>*
<kPageDown> scroll.txt /*<kPageDown>*
<kPageUp> scroll.txt /*<kPageUp>*
<kPlus> term.txt /*<kPlus>*
<kPoint> term.txt /*<kPoint>*
<line1> map.txt /*<line1>*
<line2> map.txt /*<line2>*
<lt> intro.txt /*<lt>*
<mods> map.txt /*<mods>*
<nomodeline> autocmd.txt /*<nomodeline>*
<q-args> map.txt /*<q-args>*
<range> map.txt /*<range>*
<reg> map.txt /*<reg>*
<register> map.txt /*<register>*
<sfile> cmdline.txt /*<sfile>*
<sflnum> cmdline.txt /*<sflnum>*
<slnum> cmdline.txt /*<slnum>*
<xCSI> intro.txt /*<xCSI>*
<xDown> term.txt /*<xDown>*
<xEnd> term.txt /*<xEnd>*
<xEnd>-xterm term.txt /*<xEnd>-xterm*
<xF1> term.txt /*<xF1>*
<xF1>-xterm term.txt /*<xF1>-xterm*
<xF2> term.txt /*<xF2>*
<xF2>-xterm term.txt /*<xF2>-xterm*
<xF3> term.txt /*<xF3>*
<xF3>-xterm term.txt /*<xF3>-xterm*
<xF4> term.txt /*<xF4>*
<xF4>-xterm term.txt /*<xF4>-xterm*
<xHome> term.txt /*<xHome>*
<xHome>-xterm term.txt /*<xHome>-xterm*

<xLeft>	term.txt	/*<xLeft>*
<xRight>	term.txt	/*<xRight>*
<xUp>	term.txt	/*<xUp>*
=	change.txt	/*=*
==	change.txt	/*==*
>	change.txt	/*>*
>>	change.txt	/*>>*
>backtrace	repeat.txt	/*>backtrace*
>bt	repeat.txt	/*>bt*
>cont	repeat.txt	/*>cont*
>down	repeat.txt	/*>down*
>finish	repeat.txt	/*>finish*
>frame	repeat.txt	/*>frame*
>interrupt	repeat.txt	/*>interrupt*
>next	repeat.txt	/*>next*
>quit	repeat.txt	/*>quit*
>step	repeat.txt	/*>step*
>up	repeat.txt	/*>up*
>where	repeat.txt	/*>where*
?	pattern.txt	/*?*
?<CR>	pattern.txt	/*?<CR>*
@	repeat.txt	/*@*
@/	change.txt	/*@\/*
@:	repeat.txt	/*@:*
@=	change.txt	/*@=*
@@	repeat.txt	/*@@*
@r	eval.txt	/*@r*
A	insert.txt	/*A*
ACL	editing.txt	/*ACL*
ANSI-C	develop.txt	/*ANSI-C*
ATTENTION	usr_11.txt	/*ATTENTION*
Abbreviations	map.txt	/*Abbreviations*
Aleph	options.txt	/*Aleph*
Amiga	os_amiga.txt	/*Amiga*
Arabic	arabic.txt	/*Arabic*
Atari	os_mint.txt	/*Atari*
Athena	gui_x11.txt	/*Athena*
B	motion.txt	/*B*
BeBox	os_beos.txt	/*BeBox*
BeOS	os_beos.txt	/*BeOS*
Blob	eval.txt	/*Blob*
Blobs	eval.txt	/*Blobs*
Boolean	eval.txt	/*Boolean*
Bram	intro.txt	/*Bram*
BufAdd	autocmd.txt	/*BufAdd*
BufCreate	autocmd.txt	/*BufCreate*
BufDelete	autocmd.txt	/*BufDelete*
BufEnter	autocmd.txt	/*BufEnter*
BufFilePost	autocmd.txt	/*BufFilePost*
BufFilePre	autocmd.txt	/*BufFilePre*
BufHidden	autocmd.txt	/*BufHidden*
BufLeave	autocmd.txt	/*BufLeave*
BufNew	autocmd.txt	/*BufNew*
BufNewFile	autocmd.txt	/*BufNewFile*

```
BufRead                 autocmd.txt              /*BufRead*
BufReadCmd              autocmd.txt              /*BufReadCmd*
BufReadPost             autocmd.txt              /*BufReadPost*
BufReadPre              autocmd.txt              /*BufReadPre*
BufUnload               autocmd.txt              /*BufUnload*
BufWinEnter             autocmd.txt              /*BufWinEnter*
BufWinLeave             autocmd.txt              /*BufWinLeave*
BufWipeout              autocmd.txt              /*BufWipeout*
BufWrite                autocmd.txt              /*BufWrite*
BufWriteCmd             autocmd.txt              /*BufWriteCmd*
BufWritePost            autocmd.txt              /*BufWritePost*
BufWritePre             autocmd.txt              /*BufWritePre*
C                       change.txt                  /*C*
C-editing               tips.txt                 /*C-editing*
C-indenting             indent.txt               /*C-indenting*
C89                     develop.txt                 /*C89*
C99                     develop.txt                 /*C99*
COMSPEC                 starting.txt             /*COMSPEC*
CR-used-for-NL          pattern.txt              /*CR-used-for-NL*
CTRL-6                  editing.txt                 /*CTRL-6*
CTRL-<PageDown>         tabpage.txt              /*CTRL-<PageDown>*
CTRL-<PageUp>           tabpage.txt              /*CTRL-<PageUp>*
CTRL-A                  change.txt                  /*CTRL-A*
CTRL-B                  scroll.txt                  /*CTRL-B*
CTRL-C                  pattern.txt                 /*CTRL-C*
CTRL-D                  scroll.txt                  /*CTRL-D*
CTRL-E                  scroll.txt                  /*CTRL-E*
CTRL-F                  scroll.txt                  /*CTRL-F*
CTRL-G                  editing.txt                 /*CTRL-G*
CTRL-H                  motion.txt                  /*CTRL-H*
CTRL-I                  motion.txt                  /*CTRL-I*
CTRL-J                  motion.txt                  /*CTRL-J*
CTRL-L                  various.txt                 /*CTRL-L*
CTRL-M                  motion.txt                  /*CTRL-M*
CTRL-N                  motion.txt                  /*CTRL-N*
CTRL-O                  motion.txt                  /*CTRL-O*
CTRL-P                  motion.txt                  /*CTRL-P*
CTRL-Q                  gui_w32.txt                 /*CTRL-Q*
CTRL-R                  undo.txt                 /*CTRL-R*
CTRL-T                  tagsrch.txt                 /*CTRL-T*
CTRL-U                  scroll.txt                  /*CTRL-U*
CTRL-U-changed          version6.txt             /*CTRL-U-changed*
CTRL-V                  visual.txt                  /*CTRL-V*
CTRL-V-alternative          gui_w32.txt          /*CTRL-V-alternative*
CTRL-W                  index.txt                   /*CTRL-W*
CTRL-W_+                windows.txt                 /*CTRL-W_+*
CTRL-W_-                windows.txt                 /*CTRL-W_-*
CTRL-W_:                windows.txt                 /*CTRL-W_:*
CTRL-W_<                windows.txt                 /*CTRL-W_<*
CTRL-W_<BS>              windows.txt             /*CTRL-W_<BS>*
CTRL-W_<CR>              quickfix.txt             /*CTRL-W_<CR>*
CTRL-W_<Down>            windows.txt             /*CTRL-W_<Down>*
CTRL-W_<Enter>           quickfix.txt            /*CTRL-W_<Enter>*
CTRL-W_<Left>            windows.txt             /*CTRL-W_<Left>*
```

CTRL-W_<Right> windows.txt /*CTRL-W_<Right>*
CTRL-W_<Up> windows.txt /*CTRL-W_<Up>*
CTRL-W_= windows.txt /*CTRL-W_=*
CTRL-W_> windows.txt /*CTRL-W_>*
CTRL-W_CTRL-B windows.txt /*CTRL-W_CTRL-B*
CTRL-W_CTRL-C windows.txt /*CTRL-W_CTRL-C*
CTRL-W_CTRL-D tagsrch.txt /*CTRL-W_CTRL-D*
CTRL-W_CTRL-F windows.txt /*CTRL-W_CTRL-F*
CTRL-W_CTRL-H windows.txt /*CTRL-W_CTRL-H*
CTRL-W_CTRL-I tagsrch.txt /*CTRL-W_CTRL-I*
CTRL-W_CTRL-J windows.txt /*CTRL-W_CTRL-J*
CTRL-W_CTRL-K windows.txt /*CTRL-W_CTRL-K*
CTRL-W_CTRL-L windows.txt /*CTRL-W_CTRL-L*
CTRL-W_CTRL-N windows.txt /*CTRL-W_CTRL-N*
CTRL-W_CTRL-O windows.txt /*CTRL-W_CTRL-O*
CTRL-W_CTRL-P windows.txt /*CTRL-W_CTRL-P*
CTRL-W_CTRL-Q windows.txt /*CTRL-W_CTRL-Q*
CTRL-W_CTRL-R windows.txt /*CTRL-W_CTRL-R*
CTRL-W_CTRL-S windows.txt /*CTRL-W_CTRL-S*
CTRL-W_CTRL-T windows.txt /*CTRL-W_CTRL-T*
CTRL-W_CTRL-V windows.txt /*CTRL-W_CTRL-V*
CTRL-W_CTRL-W windows.txt /*CTRL-W_CTRL-W*
CTRL-W_CTRL-X windows.txt /*CTRL-W_CTRL-X*
CTRL-W_CTRL-Z windows.txt /*CTRL-W_CTRL-Z*
CTRL-W_CTRL-] windows.txt /*CTRL-W_CTRL-]*
CTRL-W_CTRL-^ windows.txt /*CTRL-W_CTRL-^*
CTRL-W_CTRL-_ windows.txt /*CTRL-W_CTRL-_*
CTRL-W_F windows.txt /*CTRL-W_F*
CTRL-W_H windows.txt /*CTRL-W_H*
CTRL-W_J windows.txt /*CTRL-W_J*
CTRL-W_K windows.txt /*CTRL-W_K*
CTRL-W_L windows.txt /*CTRL-W_L*
CTRL-W_P windows.txt /*CTRL-W_P*
CTRL-W_R windows.txt /*CTRL-W_R*
CTRL-W_S windows.txt /*CTRL-W_S*
CTRL-W_T windows.txt /*CTRL-W_T*
CTRL-W_W windows.txt /*CTRL-W_W*
CTRL-W_] windows.txt /*CTRL-W_]*
CTRL-W_^ windows.txt /*CTRL-W_^*
CTRL-W__ windows.txt /*CTRL-W__*
CTRL-W_b windows.txt /*CTRL-W_b*
CTRL-W_bar windows.txt /*CTRL-W_bar*
CTRL-W_c windows.txt /*CTRL-W_c*
CTRL-W_d tagsrch.txt /*CTRL-W_d*
CTRL-W_f windows.txt /*CTRL-W_f*
CTRL-W_gF windows.txt /*CTRL-W_gF*
CTRL-W_gT windows.txt /*CTRL-W_gT*
CTRL-W_g] windows.txt /*CTRL-W_g]*
CTRL-W_g_CTRL-] windows.txt /*CTRL-W_g_CTRL-]*
CTRL-W_gf windows.txt /*CTRL-W_gf*
CTRL-W_gt windows.txt /*CTRL-W_gt*
CTRL-W_g} windows.txt /*CTRL-W_g}*
CTRL-W_h windows.txt /*CTRL-W_h*
CTRL-W_i tagsrch.txt /*CTRL-W_i*

CTRL-W_j windows.txt /*CTRL-W_j*
CTRL-W_k windows.txt /*CTRL-W_k*
CTRL-W_l windows.txt /*CTRL-W_l*
CTRL-W_n windows.txt /*CTRL-W_n*
CTRL-W_o windows.txt /*CTRL-W_o*
CTRL-W_p windows.txt /*CTRL-W_p*
CTRL-W_q windows.txt /*CTRL-W_q*
CTRL-W_r windows.txt /*CTRL-W_r*
CTRL-W_s windows.txt /*CTRL-W_s*
CTRL-W_t windows.txt /*CTRL-W_t*
CTRL-W_v windows.txt /*CTRL-W_v*
CTRL-W_w windows.txt /*CTRL-W_w*
CTRL-W_x windows.txt /*CTRL-W_x*
CTRL-W_z windows.txt /*CTRL-W_z*
CTRL-W_} windows.txt /*CTRL-W_}*
CTRL-X change.txt /*CTRL-X*
CTRL-Y scroll.txt /*CTRL-Y*
CTRL-Z starting.txt /*CTRL-Z*
CTRL-_CTRL-G intro.txt /*CTRL-_CTRL-G*
CTRL-_CTRL-N intro.txt /*CTRL-_CTRL-N*
CTRL-] tagsrch.txt /*CTRL-]*
CTRL-^ editing.txt /*CTRL-^*
CTRL-{char} intro.txt /*CTRL-{char}*
Channel eval.txt /*Channel*
Channels eval.txt /*Channels*
Chinese mbyte.txt /*Chinese*
Cmd-event autocmd.txt /*Cmd-event*
CmdUndefined autocmd.txt /*CmdUndefined*
Cmdline cmdline.txt /*Cmdline*
Cmdline-mode cmdline.txt /*Cmdline-mode*
CmdlineChanged autocmd.txt /*CmdlineChanged*
CmdlineEnter autocmd.txt /*CmdlineEnter*
CmdlineLeave autocmd.txt /*CmdlineLeave*
CmdwinEnter autocmd.txt /*CmdwinEnter*
CmdwinLeave autocmd.txt /*CmdwinLeave*
ColorScheme autocmd.txt /*ColorScheme*
ColorSchemePre autocmd.txt /*ColorSchemePre*
Command-line cmdline.txt /*Command-line*
Command-line-mode cmdline.txt /*Command-line-mode*
CompleteChanged autocmd.txt /*CompleteChanged*
CompleteDone autocmd.txt /*CompleteDone*
ConPTY terminal.txt /*ConPTY*
Contents quickref.txt /*Contents*
Cscope if_cscop.txt /*Cscope*
CursorHold autocmd.txt /*CursorHold*
CursorHold-example windows.txt /*CursorHold-example*
CursorHoldI autocmd.txt /*CursorHoldI*
CursorIM mbyte.txt /*CursorIM*
CursorMoved autocmd.txt /*CursorMoved*
CursorMovedI autocmd.txt /*CursorMovedI*
D change.txt /*D*
DOS os_dos.txt /*DOS*
DOS-format editing.txt /*DOS-format*
DOS-format-write editing.txt /*DOS-format-write*

Dict eval.txt /*Dict*
Dictionaries eval.txt /*Dictionaries*
Dictionary eval.txt /*Dictionary*
Dictionary-function eval.txt /*Dictionary-function*
DiffUpdated autocmd.txt /*DiffUpdated*
Digraphs digraph.txt /*Digraphs*
DirChanged autocmd.txt /*DirChanged*
E motion.txt /*E*
E10 message.txt /*E10*
E100 diff.txt /*E100*
E101 diff.txt /*E101*
E102 diff.txt /*E102*
E103 diff.txt /*E103*
E104 digraph.txt /*E104*
E105 mbyte.txt /*E105*
E107 eval.txt /*E107*
E108 eval.txt /*E108*
E109 eval.txt /*E109*
E11 cmdline.txt /*E11*
E110 eval.txt /*E110*
E111 eval.txt /*E111*
E112 eval.txt /*E112*
E113 eval.txt /*E113*
E114 eval.txt /*E114*
E115 eval.txt /*E115*
E116 eval.txt /*E116*
E117 eval.txt /*E117*
E118 eval.txt /*E118*
E119 eval.txt /*E119*
E12 message.txt /*E12*
E120 eval.txt /*E120*
E121 eval.txt /*E121*
E122 eval.txt /*E122*
E123 eval.txt /*E123*
E124 eval.txt /*E124*
E125 eval.txt /*E125*
E126 eval.txt /*E126*
E127 eval.txt /*E127*
E128 eval.txt /*E128*
E129 eval.txt /*E129*
E13 message.txt /*E13*
E130 eval.txt /*E130*
E131 eval.txt /*E131*
E132 eval.txt /*E132*
E133 eval.txt /*E133*
E134 change.txt /*E134*
E135 autocmd.txt /*E135*
E136 starting.txt /*E136*
E137 starting.txt /*E137*
E138 starting.txt /*E138*
E139 message.txt /*E139*
E140 message.txt /*E140*
E141 message.txt /*E141*
E142 message.txt /*E142*

E143	autocmd.txt	/*E143*
E144	various.txt	/*E144*
E145	starting.txt	/*E145*
E146	change.txt	/*E146*
E147	repeat.txt	/*E147*
E148	repeat.txt	/*E148*
E149	helphelp.txt	/*E149*
E15	eval.txt	/*E15*
E150	helphelp.txt	/*E150*
E151	helphelp.txt	/*E151*
E152	helphelp.txt	/*E152*
E153	helphelp.txt	/*E153*
E154	helphelp.txt	/*E154*
E155	sign.txt	/*E155*
E156	sign.txt	/*E156*
E157	sign.txt	/*E157*
E158	sign.txt	/*E158*
E159	sign.txt	/*E159*
E16	cmdline.txt	/*E16*
E160	sign.txt	/*E160*
E161	repeat.txt	/*E161*
E162	message.txt	/*E162*
E163	editing.txt	/*E163*
E164	editing.txt	/*E164*
E165	editing.txt	/*E165*
E166	message.txt	/*E166*
E167	repeat.txt	/*E167*
E168	repeat.txt	/*E168*
E169	message.txt	/*E169*
E17	message.txt	/*E17*
E170	eval.txt	/*E170*
E171	eval.txt	/*E171*
E172	eval.txt	/*E172*
E173	message.txt	/*E173*
E174	map.txt	/*E174*
E175	map.txt	/*E175*
E176	map.txt	/*E176*
E177	map.txt	/*E177*
E178	map.txt	/*E178*
E179	map.txt	/*E179*
E18	eval.txt	/*E18*
E180	map.txt	/*E180*
E181	map.txt	/*E181*
E182	map.txt	/*E182*
E183	map.txt	/*E183*
E184	map.txt	/*E184*
E185	syntax.txt	/*E185*
E186	editing.txt	/*E186*
E187	editing.txt	/*E187*
E188	gui.txt	/*E188*
E189	message.txt	/*E189*
E19	message.txt	/*E19*
E190	message.txt	/*E190*
E191	motion.txt	/*E191*

E192	message.txt	/*E192*
E193	eval.txt	/*E193*
E194	message.txt	/*E194*
E195	starting.txt	/*E195*
E196	various.txt	/*E196*
E197	mlang.txt	/*E197*
E198	options.txt	/*E198*
E199	cmdline.txt	/*E199*
E20	motion.txt	/*E20*
E200	autocmd.txt	/*E200*
E201	autocmd.txt	/*E201*
E202	options.txt	/*E202*
E203	autocmd.txt	/*E203*
E204	autocmd.txt	/*E204*
E205	options.txt	/*E205*
E206	options.txt	/*E206*
E207	editing.txt	/*E207*
E208	message.txt	/*E208*
E209	message.txt	/*E209*
E21	options.txt	/*E21*
E210	message.txt	/*E210*
E211	message.txt	/*E211*
E212	message.txt	/*E212*
E213	options.txt	/*E213*
E214	options.txt	/*E214*
E215	autocmd.txt	/*E215*
E216	autocmd.txt	/*E216*
E217	autocmd.txt	/*E217*
E218	autocmd.txt	/*E218*
E219	message.txt	/*E219*
E22	message.txt	/*E22*
E220	message.txt	/*E220*
E221	eval.txt	/*E221*
E222	message.txt	/*E222*
E223	options.txt	/*E223*
E224	map.txt	/*E224*
E225	map.txt	/*E225*
E226	map.txt	/*E226*
E227	map.txt	/*E227*
E228	message.txt	/*E228*
E229	gui.txt	/*E229*
E23	message.txt	/*E23*
E230	starting.txt	/*E230*
E231	options.txt	/*E231*
E232	message.txt	/*E232*
E233	gui.txt	/*E233*
E234	options.txt	/*E234*
E235	options.txt	/*E235*
E236	gui.txt	/*E236*
E237	print.txt	/*E237*
E238	print.txt	/*E238*
E239	sign.txt	/*E239*
E24	message.txt	/*E24*
E240	remote.txt	/*E240*

E241 eval.txt /*E241*
E242 windows.txt /*E242*
E243 if_ole.txt /*E243*
E244 gui.txt /*E244*
E245 gui.txt /*E245*
E246 autocmd.txt /*E246*
E247 remote.txt /*E247*
E248 remote.txt /*E248*
E249 windows.txt /*E249*
E25 message.txt /*E25*
E250 options.txt /*E250*
E251 remote.txt /*E251*
E252 options.txt /*E252*
E253 mbyte.txt /*E253*
E254 message.txt /*E254*
E255 sign.txt /*E255*
E256 message.txt /*E256*
E257 if_cscop.txt /*E257*
E258 remote.txt /*E258*
E259 if_cscop.txt /*E259*
E26 rileft.txt /*E26*
E261 if_cscop.txt /*E261*
E262 if_cscop.txt /*E262*
E263 if_pyth.txt /*E263*
E264 if_pyth.txt /*E264*
E265 if_ruby.txt /*E265*
E266 if_ruby.txt /*E266*
E267 if_ruby.txt /*E267*
E268 if_ruby.txt /*E268*
E269 if_ruby.txt /*E269*
E27 farsi.txt /*E27*
E270 if_ruby.txt /*E270*
E271 if_ruby.txt /*E271*
E272 if_ruby.txt /*E272*
E273 if_ruby.txt /*E273*
E274 eval.txt /*E274*
E275 textprop.txt /*E275*
E276 eval.txt /*E276*
E277 remote.txt /*E277*
E278 popup.txt /*E278*
E279 terminal.txt /*E279*
E28 syntax.txt /*E28*
E280 if_tcl.txt /*E280*
E282 starting.txt /*E282*
E283 motion.txt /*E283*
E284 mbyte.txt /*E284*
E285 mbyte.txt /*E285*
E286 mbyte.txt /*E286*
E287 mbyte.txt /*E287*
E288 mbyte.txt /*E288*
E289 mbyte.txt /*E289*
E29 change.txt /*E29*
E293 message.txt /*E293*
E294 message.txt /*E294*

E295 message.txt /*E295*
E296 message.txt /*E296*
E297 message.txt /*E297*
E298 message.txt /*E298*
E299 if_perl.txt /*E299*
E30 change.txt /*E30*
E300 message.txt /*E300*
E301 message.txt /*E301*
E302 message.txt /*E302*
E303 message.txt /*E303*
E304 message.txt /*E304*
E305 recover.txt /*E305*
E306 recover.txt /*E306*
E307 recover.txt /*E307*
E308 recover.txt /*E308*
E309 recover.txt /*E309*
E31 message.txt /*E31*
E310 recover.txt /*E310*
E311 recover.txt /*E311*
E312 recover.txt /*E312*
E313 recover.txt /*E313*
E314 recover.txt /*E314*
E315 message.txt /*E315*
E316 message.txt /*E316*
E317 message.txt /*E317*
E318 message.txt /*E318*
E319 message.txt /*E319*
E32 message.txt /*E32*
E320 message.txt /*E320*
E321 editing.txt /*E321*
E322 message.txt /*E322*
E323 message.txt /*E323*
E324 print.txt /*E324*
E325 usr_11.txt /*E325*
E326 recover.txt /*E326*
E327 gui.txt /*E327*
E328 gui.txt /*E328*
E329 gui.txt /*E329*
E33 message.txt /*E33*
E330 gui.txt /*E330*
E331 gui.txt /*E331*
E332 gui.txt /*E332*
E333 gui.txt /*E333*
E334 gui.txt /*E334*
E335 gui.txt /*E335*
E336 gui.txt /*E336*
E337 gui.txt /*E337*
E338 editing.txt /*E338*
E339 message.txt /*E339*
E34 various.txt /*E34*
E340 vi_diff.txt /*E340*
E341 message.txt /*E341*
E342 message.txt /*E342*
E343 options.txt /*E343*

E344	options.txt	/*E344*
E345	options.txt	/*E345*
E346	options.txt	/*E346*
E347	options.txt	/*E347*
E348	pattern.txt	/*E348*
E349	pattern.txt	/*E349*
E35	message.txt	/*E35*
E350	fold.txt	/*E350*
E351	fold.txt	/*E351*
E352	fold.txt	/*E352*
E353	change.txt	/*E353*
E354	change.txt	/*E354*
E355	options.txt	/*E355*
E356	message.txt	/*E356*
E357	options.txt	/*E357*
E358	options.txt	/*E358*
E359	term.txt	/*E359*
E36	windows.txt	/*E36*
E360	various.txt	/*E360*
E363	options.txt	/*E363*
E364	eval.txt	/*E364*
E365	print.txt	/*E365*
E367	autocmd.txt	/*E367*
E368	eval.txt	/*E368*
E369	pattern.txt	/*E369*
E37	message.txt	/*E37*
E370	various.txt	/*E370*
E371	various.txt	/*E371*
E372	quickfix.txt	/*E372*
E373	quickfix.txt	/*E373*
E374	quickfix.txt	/*E374*
E375	quickfix.txt	/*E375*
E376	quickfix.txt	/*E376*
E377	quickfix.txt	/*E377*
E378	quickfix.txt	/*E378*
E379	quickfix.txt	/*E379*
E38	message.txt	/*E38*
E380	quickfix.txt	/*E380*
E381	quickfix.txt	/*E381*
E382	options.txt	/*E382*
E383	pattern.txt	/*E383*
E384	options.txt	/*E384*
E385	options.txt	/*E385*
E386	pattern.txt	/*E386*
E387	tagsrch.txt	/*E387*
E388	tagsrch.txt	/*E388*
E389	tagsrch.txt	/*E389*
E39	digraph.txt	/*E39*
E390	syntax.txt	/*E390*
E391	syntax.txt	/*E391*
E392	syntax.txt	/*E392*
E393	syntax.txt	/*E393*
E394	syntax.txt	/*E394*
E395	syntax.txt	/*E395*

E397	syntax.txt	/*E397*
E398	syntax.txt	/*E398*
E399	syntax.txt	/*E399*
E40	message.txt	/*E40*
E400	syntax.txt	/*E400*
E401	syntax.txt	/*E401*
E402	syntax.txt	/*E402*
E403	syntax.txt	/*E403*
E404	syntax.txt	/*E404*
E405	syntax.txt	/*E405*
E406	syntax.txt	/*E406*
E407	syntax.txt	/*E407*
E408	syntax.txt	/*E408*
E409	syntax.txt	/*E409*
E41	message.txt	/*E41*
E410	syntax.txt	/*E410*
E411	syntax.txt	/*E411*
E412	syntax.txt	/*E412*
E413	syntax.txt	/*E413*
E414	syntax.txt	/*E414*
E415	syntax.txt	/*E415*
E416	syntax.txt	/*E416*
E417	syntax.txt	/*E417*
E418	syntax.txt	/*E418*
E419	syntax.txt	/*E419*
E42	quickfix.txt	/*E42*
E420	syntax.txt	/*E420*
E421	syntax.txt	/*E421*
E422	syntax.txt	/*E422*
E423	syntax.txt	/*E423*
E424	message.txt	/*E424*
E425	tagsrch.txt	/*E425*
E426	tagsrch.txt	/*E426*
E427	tagsrch.txt	/*E427*
E428	tagsrch.txt	/*E428*
E429	tagsrch.txt	/*E429*
E43	message.txt	/*E43*
E430	tagsrch.txt	/*E430*
E431	tagsrch.txt	/*E431*
E432	message.txt	/*E432*
E433	options.txt	/*E433*
E434	tagsrch.txt	/*E434*
E435	tagsrch.txt	/*E435*
E436	term.txt	/*E436*
E437	term.txt	/*E437*
E438	message.txt	/*E438*
E439	message.txt	/*E439*
E44	message.txt	/*E44*
E440	message.txt	/*E440*
E441	windows.txt	/*E441*
E442	windows.txt	/*E442*
E443	windows.txt	/*E443*
E444	windows.txt	/*E444*
E445	windows.txt	/*E445*

E446 editing.txt /*E446*
E447 editing.txt /*E447*
E448 various.txt /*E448*
E449 eval.txt /*E449*
E45 message.txt /*E45*
E455 print.txt /*E455*
E456 print.txt /*E456*
E457 print.txt /*E457*
E458 message.txt /*E458*
E459 message.txt /*E459*
E46 message.txt /*E46*
E461 eval.txt /*E461*
E462 editing.txt /*E462*
E463 netbeans.txt /*E463*
E464 message.txt /*E464*
E465 gui.txt /*E465*
E466 gui.txt /*E466*
E467 map.txt /*E467*
E468 map.txt /*E468*
E469 if_cscop.txt /*E469*
E47 message.txt /*E47*
E470 change.txt /*E470*
E471 message.txt /*E471*
E472 editing.txt /*E472*
E473 message.txt /*E473*
E474 message.txt /*E474*
E475 message.txt /*E475*
E476 pattern.txt /*E476*
E477 message.txt /*E477*
E478 message.txt /*E478*
E479 editing.txt /*E479*
E48 eval.txt /*E48*
E480 editing.txt /*E480*
E481 message.txt /*E481*
E482 message.txt /*E482*
E483 message.txt /*E483*
E484 message.txt /*E484*
E485 message.txt /*E485*
E486 pattern.txt /*E486*
E487 options.txt /*E487*
E488 message.txt /*E488*
E49 message.txt /*E49*
E490 fold.txt /*E490*
E492 message.txt /*E492*
E493 cmdline.txt /*E493*
E494 editing.txt /*E494*
E495 cmdline.txt /*E495*
E496 cmdline.txt /*E496*
E497 cmdline.txt /*E497*
E498 cmdline.txt /*E498*
E499 cmdline.txt /*E499*
E50 syntax.txt /*E50*
E500 cmdline.txt /*E500*
E501 intro.txt /*E501*

E502 editing.txt /*E502*
E503 editing.txt /*E503*
E504 editing.txt /*E504*
E505 editing.txt /*E505*
E506 editing.txt /*E506*
E507 editing.txt /*E507*
E508 editing.txt /*E508*
E509 editing.txt /*E509*
E51 pattern.txt /*E51*
E510 editing.txt /*E510*
E511 netbeans.txt /*E511*
E512 editing.txt /*E512*
E513 options.txt /*E513*
E514 editing.txt /*E514*
E515 windows.txt /*E515*
E516 windows.txt /*E516*
E517 windows.txt /*E517*
E518 options.txt /*E518*
E519 options.txt /*E519*
E52 syntax.txt /*E52*
E520 options.txt /*E520*
E521 options.txt /*E521*
E522 options.txt /*E522*
E523 options.txt /*E523*
E524 options.txt /*E524*
E525 options.txt /*E525*
E526 options.txt /*E526*
E527 options.txt /*E527*
E528 options.txt /*E528*
E529 options.txt /*E529*
E53 pattern.txt /*E53*
E530 options.txt /*E530*
E531 options.txt /*E531*
E532 netbeans.txt /*E532*
E533 options.txt /*E533*
E534 options.txt /*E534*
E535 options.txt /*E535*
E536 options.txt /*E536*
E537 options.txt /*E537*
E538 options.txt /*E538*
E539 options.txt /*E539*
E54 pattern.txt /*E54*
E540 options.txt /*E540*
E541 options.txt /*E541*
E542 options.txt /*E542*
E543 options.txt /*E543*
E544 options.txt /*E544*
E545 options.txt /*E545*
E546 options.txt /*E546*
E547 options.txt /*E547*
E548 options.txt /*E548*
E549 options.txt /*E549*
E55 pattern.txt /*E55*
E550 options.txt /*E550*

E551	options.txt	/*E551*
E552	options.txt	/*E552*
E553	quickfix.txt	/*E553*
E554	pattern.txt	/*E554*
E555	tagsrch.txt	/*E555*
E556	tagsrch.txt	/*E556*
E557	term.txt	/*E557*
E558	term.txt	/*E558*
E559	term.txt	/*E559*
E560	if_cscop.txt	/*E560*
E561	if_cscop.txt	/*E561*
E562	if_cscop.txt	/*E562*
E563	if_cscop.txt	/*E563*
E564	if_cscop.txt	/*E564*
E566	if_cscop.txt	/*E566*
E567	if_cscop.txt	/*E567*
E568	if_cscop.txt	/*E568*
E570	message.txt	/*E570*
E571	if_tcl.txt	/*E571*
E572	if_tcl.txt	/*E572*
E573	remote.txt	/*E573*
E574	starting.txt	/*E574*
E575	starting.txt	/*E575*
E576	starting.txt	/*E576*
E577	starting.txt	/*E577*
E579	eval.txt	/*E579*
E580	eval.txt	/*E580*
E581	eval.txt	/*E581*
E582	eval.txt	/*E582*
E583	eval.txt	/*E583*
E584	eval.txt	/*E584*
E585	eval.txt	/*E585*
E586	eval.txt	/*E586*
E587	eval.txt	/*E587*
E588	eval.txt	/*E588*
E589	options.txt	/*E589*
E59	pattern.txt	/*E59*
E590	options.txt	/*E590*
E591	options.txt	/*E591*
E592	options.txt	/*E592*
E593	options.txt	/*E593*
E594	options.txt	/*E594*
E595	options.txt	/*E595*
E596	options.txt	/*E596*
E597	options.txt	/*E597*
E598	options.txt	/*E598*
E599	options.txt	/*E599*
E60	pattern.txt	/*E60*
E600	eval.txt	/*E600*
E601	eval.txt	/*E601*
E602	eval.txt	/*E602*
E603	eval.txt	/*E603*
E604	eval.txt	/*E604*
E605	eval.txt	/*E605*

E606 eval.txt /*E606*
E607 eval.txt /*E607*
E608 eval.txt /*E608*
E609 if_cscop.txt /*E609*
E61 pattern.txt /*E61*
E612 sign.txt /*E612*
E613 print.txt /*E613*
E614 editing.txt /*E614*
E615 editing.txt /*E615*
E616 editing.txt /*E616*
E617 options.txt /*E617*
E618 print.txt /*E618*
E619 print.txt /*E619*
E62 pattern.txt /*E62*
E620 print.txt /*E620*
E621 print.txt /*E621*
E622 if_cscop.txt /*E622*
E623 if_cscop.txt /*E623*
E624 print.txt /*E624*
E625 if_cscop.txt /*E625*
E626 if_cscop.txt /*E626*
E627 netbeans.txt /*E627*
E628 netbeans.txt /*E628*
E629 netbeans.txt /*E629*
E63 pattern.txt /*E63*
E630 channel.txt /*E630*
E631 channel.txt /*E631*
E632 netbeans.txt /*E632*
E633 netbeans.txt /*E633*
E634 netbeans.txt /*E634*
E635 netbeans.txt /*E635*
E636 netbeans.txt /*E636*
E637 netbeans.txt /*E637*
E638 netbeans.txt /*E638*
E639 netbeans.txt /*E639*
E64 pattern.txt /*E64*
E640 netbeans.txt /*E640*
E641 netbeans.txt /*E641*
E642 netbeans.txt /*E642*
E643 netbeans.txt /*E643*
E644 netbeans.txt /*E644*
E645 netbeans.txt /*E645*
E646 netbeans.txt /*E646*
E647 netbeans.txt /*E647*
E648 netbeans.txt /*E648*
E649 netbeans.txt /*E649*
E65 pattern.txt /*E65*
E650 netbeans.txt /*E650*
E651 netbeans.txt /*E651*
E652 netbeans.txt /*E652*
E655 eval.txt /*E655*
E656 netbeans.txt /*E656*
E657 netbeans.txt /*E657*
E658 netbeans.txt /*E658*

E659	if_pyth.txt	/*E659*
E66	syntax.txt	/*E66*
E660	netbeans.txt	/*E660*
E661	helphelp.txt	/*E661*
E662	motion.txt	/*E662*
E663	motion.txt	/*E663*
E664	motion.txt	/*E664*
E665	gui_x11.txt	/*E665*
E666	quickfix.txt	/*E666*
E667	editing.txt	/*E667*
E668	netbeans.txt	/*E668*
E669	syntax.txt	/*E669*
E67	syntax.txt	/*E67*
E670	helphelp.txt	/*E670*
E671	starting.txt	/*E671*
E672	starting.txt	/*E672*
E673	print.txt	/*E673*
E674	print.txt	/*E674*
E675	print.txt	/*E675*
E676	options.txt	/*E676*
E677	eval.txt	/*E677*
E678	pattern.txt	/*E678*
E679	syntax.txt	/*E679*
E68	pattern.txt	/*E68*
E680	autocmd.txt	/*E680*
E681	quickfix.txt	/*E681*
E682	quickfix.txt	/*E682*
E683	quickfix.txt	/*E683*
E684	eval.txt	/*E684*
E685	message.txt	/*E685*
E686	eval.txt	/*E686*
E687	eval.txt	/*E687*
E688	eval.txt	/*E688*
E689	eval.txt	/*E689*
E69	pattern.txt	/*E69*
E690	eval.txt	/*E690*
E691	eval.txt	/*E691*
E692	eval.txt	/*E692*
E694	eval.txt	/*E694*
E695	eval.txt	/*E695*
E696	eval.txt	/*E696*
E697	eval.txt	/*E697*
E698	eval.txt	/*E698*
E699	eval.txt	/*E699*
E70	pattern.txt	/*E70*
E700	eval.txt	/*E700*
E701	eval.txt	/*E701*
E702	eval.txt	/*E702*
E703	eval.txt	/*E703*
E704	eval.txt	/*E704*
E705	eval.txt	/*E705*
E707	eval.txt	/*E707*
E708	eval.txt	/*E708*
E709	eval.txt	/*E709*

E71 pattern.txt /*E71*
E710 eval.txt /*E710*
E711 eval.txt /*E711*
E712 eval.txt /*E712*
E713 eval.txt /*E713*
E714 eval.txt /*E714*
E715 eval.txt /*E715*
E716 eval.txt /*E716*
E717 eval.txt /*E717*
E718 eval.txt /*E718*
E719 eval.txt /*E719*
E72 message.txt /*E72*
E720 eval.txt /*E720*
E721 eval.txt /*E721*
E722 eval.txt /*E722*
E723 eval.txt /*E723*
E724 eval.txt /*E724*
E725 eval.txt /*E725*
E726 eval.txt /*E726*
E727 eval.txt /*E727*
E728 eval.txt /*E728*
E729 eval.txt /*E729*
E73 tagsrch.txt /*E73*
E730 eval.txt /*E730*
E731 eval.txt /*E731*
E732 eval.txt /*E732*
E733 eval.txt /*E733*
E734 eval.txt /*E734*
E735 eval.txt /*E735*
E736 eval.txt /*E736*
E737 eval.txt /*E737*
E738 eval.txt /*E738*
E739 eval.txt /*E739*
E74 message.txt /*E74*
E740 eval.txt /*E740*
E741 eval.txt /*E741*
E742 eval.txt /*E742*
E743 eval.txt /*E743*
E744 netbeans.txt /*E744*
E745 eval.txt /*E745*
E746 eval.txt /*E746*
E747 editing.txt /*E747*
E748 repeat.txt /*E748*
E749 various.txt /*E749*
E75 vi_diff.txt /*E75*
E750 repeat.txt /*E750*
E751 spell.txt /*E751*
E752 spell.txt /*E752*
E753 spell.txt /*E753*
E754 spell.txt /*E754*
E755 spell.txt /*E755*
E756 spell.txt /*E756*
E757 options.txt /*E757*
E758 spell.txt /*E758*

E759 spell.txt /*E759*
E76 pattern.txt /*E76*
E760 spell.txt /*E760*
E761 spell.txt /*E761*
E762 spell.txt /*E762*
E763 spell.txt /*E763*
E764 options.txt /*E764*
E765 options.txt /*E765*
E766 eval.txt /*E766*
E767 eval.txt /*E767*
E768 message.txt /*E768*
E769 pattern.txt /*E769*
E77 message.txt /*E77*
E770 spell.txt /*E770*
E771 spell.txt /*E771*
E772 spell.txt /*E772*
E773 recover.txt /*E773*
E774 map.txt /*E774*
E775 map.txt /*E775*
E776 quickfix.txt /*E776*
E777 quickfix.txt /*E777*
E778 spell.txt /*E778*
E779 spell.txt /*E779*
E78 motion.txt /*E78*
E780 spell.txt /*E780*
E781 spell.txt /*E781*
E782 spell.txt /*E782*
E783 spell.txt /*E783*
E784 tabpage.txt /*E784*
E785 eval.txt /*E785*
E786 eval.txt /*E786*
E787 diff.txt /*E787*
E788 autocmd.txt /*E788*
E789 syntax.txt /*E789*
E79 message.txt /*E79*
E790 undo.txt /*E790*
E791 mbyte.txt /*E791*
E792 gui.txt /*E792*
E793 diff.txt /*E793*
E794 eval.txt /*E794*
E795 eval.txt /*E795*
E796 editing.txt /*E796*
E797 spell.txt /*E797*
E798 eval.txt /*E798*
E799 eval.txt /*E799*
E80 message.txt /*E80*
E800 arabic.txt /*E800*
E801 eval.txt /*E801*
E802 eval.txt /*E802*
E803 eval.txt /*E803*
E804 eval.txt /*E804*
E805 eval.txt /*E805*
E806 eval.txt /*E806*
E807 eval.txt /*E807*

E808 eval.txt /*E808*
E809 cmdline.txt /*E809*
E81 map.txt /*E81*
E810 diff.txt /*E810*
E811 autocmd.txt /*E811*
E812 autocmd.txt /*E812*
E813 editing.txt /*E813*
E814 editing.txt /*E814*
E815 if_mzsch.txt /*E815*
E816 diff.txt /*E816*
E817 editing.txt /*E817*
E818 editing.txt /*E818*
E819 editing.txt /*E819*
E82 message.txt /*E82*
E820 editing.txt /*E820*
E821 options.txt /*E821*
E822 undo.txt /*E822*
E823 undo.txt /*E823*
E824 undo.txt /*E824*
E825 undo.txt /*E825*
E826 undo.txt /*E826*
E827 undo.txt /*E827*
E828 undo.txt /*E828*
E829 undo.txt /*E829*
E83 message.txt /*E83*
E830 undo.txt /*E830*
E831 editing.txt /*E831*
E832 undo.txt /*E832*
E833 editing.txt /*E833*
E834 options.txt /*E834*
E835 options.txt /*E835*
E836 if_pyth.txt /*E836*
E837 if_pyth.txt /*E837*
E838 netbeans.txt /*E838*
E839 insert.txt /*E839*
E84 windows.txt /*E84*
E840 insert.txt /*E840*
E841 map.txt /*E841*
E842 cmdline.txt /*E842*
E843 editing.txt /*E843*
E844 syntax.txt /*E844*
E845 spell.txt /*E845*
E846 options.txt /*E846*
E847 syntax.txt /*E847*
E848 syntax.txt /*E848*
E849 syntax.txt /*E849*
E85 options.txt /*E85*
E850 change.txt /*E850*
E851 gui_x11.txt /*E851*
E852 gui_x11.txt /*E852*
E853 eval.txt /*E853*
E854 options.txt /*E854*
E855 autocmd.txt /*E855*
E858 eval.txt /*E858*

E859 eval.txt /*E859*
E86 windows.txt /*E86*
E862 eval.txt /*E862*
E864 pattern.txt /*E864*
E865 pattern.txt /*E865*
E866 pattern.txt /*E866*
E867 pattern.txt /*E867*
E868 pattern.txt /*E868*
E869 pattern.txt /*E869*
E87 windows.txt /*E87*
E870 pattern.txt /*E870*
E871 pattern.txt /*E871*
E872 pattern.txt /*E872*
E873 pattern.txt /*E873*
E874 pattern.txt /*E874*
E875 pattern.txt /*E875*
E876 pattern.txt /*E876*
E877 pattern.txt /*E877*
E878 pattern.txt /*E878*
E879 syntax.txt /*E879*
E88 windows.txt /*E88*
E880 if_pyth.txt /*E880*
E881 autocmd.txt /*E881*
E882 eval.txt /*E882*
E883 eval.txt /*E883*
E884 eval.txt /*E884*
E885 sign.txt /*E885*
E886 starting.txt /*E886*
E887 if_pyth.txt /*E887*
E888 pattern.txt /*E888*
E89 message.txt /*E89*
E890 syntax.txt /*E890*
E891 eval.txt /*E891*
E892 eval.txt /*E892*
E893 eval.txt /*E893*
E894 eval.txt /*E894*
E895 if_mzsch.txt /*E895*
E896 eval.txt /*E896*
E897 eval.txt /*E897*
E898 channel.txt /*E898*
E899 eval.txt /*E899*
E90 message.txt /*E90*
E901 channel.txt /*E901*
E902 channel.txt /*E902*
E903 channel.txt /*E903*
E904 channel.txt /*E904*
E905 channel.txt /*E905*
E906 channel.txt /*E906*
E907 eval.txt /*E907*
E908 eval.txt /*E908*
E909 eval.txt /*E909*
E91 options.txt /*E91*
E910 eval.txt /*E910*
E911 eval.txt /*E911*

E912	channel.txt	/*E912*
E913	eval.txt	/*E913*
E914	eval.txt	/*E914*
E915	channel.txt	/*E915*
E916	channel.txt	/*E916*
E917	channel.txt	/*E917*
E918	channel.txt	/*E918*
E919	repeat.txt	/*E919*
E92	message.txt	/*E92*
E920	channel.txt	/*E920*
E921	channel.txt	/*E921*
E922	eval.txt	/*E922*
E923	eval.txt	/*E923*
E924	quickfix.txt	/*E924*
E925	quickfix.txt	/*E925*
E926	quickfix.txt	/*E926*
E927	eval.txt	/*E927*
E928	eval.txt	/*E928*
E929	starting.txt	/*E929*
E93	windows.txt	/*E93*
E930	eval.txt	/*E930*
E931	message.txt	/*E931*
E932	eval.txt	/*E932*
E933	eval.txt	/*E933*
E934	sign.txt	/*E934*
E935	eval.txt	/*E935*
E936	autocmd.txt	/*E936*
E937	autocmd.txt	/*E937*
E938	eval.txt	/*E938*
E939	change.txt	/*E939*
E94	windows.txt	/*E94*
E940	eval.txt	/*E940*
E941	eval.txt	/*E941*
E942	eval.txt	/*E942*
E943	message.txt	/*E943*
E944	pattern.txt	/*E944*
E945	pattern.txt	/*E945*
E946	terminal.txt	/*E946*
E947	terminal.txt	/*E947*
E948	terminal.txt	/*E948*
E949	editing.txt	/*E949*
E95	message.txt	/*E95*
E950	message.txt	/*E950*
E951	pattern.txt	/*E951*
E952	autocmd.txt	/*E952*
E953	terminal.txt	/*E953*
E954	options.txt	/*E954*
E955	terminal.txt	/*E955*
E956	pattern.txt	/*E956*
E957	eval.txt	/*E957*
E958	terminal.txt	/*E958*
E959	diff.txt	/*E959*
E96	diff.txt	/*E96*
E960	options.txt	/*E960*

E961 cmdline.txt /*E961*
E962 eval.txt /*E962*
E963 eval.txt /*E963*
E964 textprop.txt /*E964*
E965 textprop.txt /*E965*
E966 textprop.txt /*E966*
E967 textprop.txt /*E967*
E968 textprop.txt /*E968*
E969 textprop.txt /*E969*
E97 diff.txt /*E97*
E970 textprop.txt /*E970*
E971 textprop.txt /*E971*
E972 eval.txt /*E972*
E973 eval.txt /*E973*
E974 eval.txt /*E974*
E975 eval.txt /*E975*
E976 eval.txt /*E976*
E977 eval.txt /*E977*
E978 eval.txt /*E978*
E979 eval.txt /*E979*
E98 diff.txt /*E98*
E980 eval.txt /*E980*
E981 starting.txt /*E981*
E982 terminal.txt /*E982*
E983 message.txt /*E983*
E984 repeat.txt /*E984*
E985 eval.txt /*E985*
E986 tagsrch.txt /*E986*
E987 tagsrch.txt /*E987*
E988 gui_w32.txt /*E988*
E989 eval.txt /*E989*
E99 diff.txt /*E99*
E990 eval.txt /*E990*
E991 eval.txt /*E991*
E992 options.txt /*E992*
E993 popup.txt /*E993*
E994 eval.txt /*E994*
E995 eval.txt /*E995*
E996 eval.txt /*E996*
E997 popup.txt /*E997*
E999 repeat.txt /*E999*
EX intro.txt /*EX*
EXINIT starting.txt /*EXINIT*
Elvis intro.txt /*Elvis*
EncodingChanged autocmd.txt /*EncodingChanged*
Eterm syntax.txt /*Eterm*
Ex intro.txt /*Ex*
Ex-mode intro.txt /*Ex-mode*
ExitPre autocmd.txt /*ExitPre*
Exuberant_ctags tagsrch.txt /*Exuberant_ctags*
F motion.txt /*F*
FALSE eval.txt /*FALSE*
FAQ intro.txt /*FAQ*
Farsi farsi.txt /*Farsi*

FileAppendCmd autocmd.txt /*FileAppendCmd*
FileAppendPost autocmd.txt /*FileAppendPost*
FileAppendPre autocmd.txt /*FileAppendPre*
FileChangedRO autocmd.txt /*FileChangedRO*
FileChangedShell autocmd.txt /*FileChangedShell*
FileChangedShellPost autocmd.txt /*FileChangedShellPost*
FileEncoding autocmd.txt /*FileEncoding*
FileReadCmd autocmd.txt /*FileReadCmd*
FileReadPost autocmd.txt /*FileReadPost*
FileReadPre autocmd.txt /*FileReadPre*
FileType autocmd.txt /*FileType*
FileWriteCmd autocmd.txt /*FileWriteCmd*
FileWritePost autocmd.txt /*FileWritePost*
FileWritePre autocmd.txt /*FileWritePre*
FilterReadPost autocmd.txt /*FilterReadPost*
FilterReadPre autocmd.txt /*FilterReadPre*
FilterWritePost autocmd.txt /*FilterWritePost*
FilterWritePre autocmd.txt /*FilterWritePre*
Float eval.txt /*Float*
FocusGained autocmd.txt /*FocusGained*
FocusLost autocmd.txt /*FocusLost*
Folding fold.txt /*Folding*
FuncUndefined autocmd.txt /*FuncUndefined*
Funcref eval.txt /*Funcref*
G motion.txt /*G*
GNOME gui_x11.txt /*GNOME*
GTK gui_x11.txt /*GTK*
GTK+ gui_x11.txt /*GTK+*
GTK3 gui_x11.txt /*GTK3*
GUI gui.txt /*GUI*
GUI-X11 gui_x11.txt /*GUI-X11*
GUIEnter autocmd.txt /*GUIEnter*
GUIFailed autocmd.txt /*GUIFailed*
GetLatestVimScripts pi_getscript.txt /*GetLatestVimScripts*
GetLatestVimScripts-copyright pi_getscript.txt
/*GetLatestVimScripts-copyright*
GetLatestVimScripts_dat pi_getscript.txt /*GetLatestVimScripts_dat*
Gnome gui_x11.txt /*Gnome*
H motion.txt /*H*
I insert.txt /*I*
ICCF uganda.txt /*ICCF*
IM-server mbyte.txt /*IM-server*
IME mbyte.txt /*IME*
Insert insert.txt /*Insert*
Insert-mode insert.txt /*Insert-mode*
InsertChange autocmd.txt /*InsertChange*
InsertCharPre autocmd.txt /*InsertCharPre*
InsertEnter autocmd.txt /*InsertEnter*
InsertLeave autocmd.txt /*InsertLeave*
Integer eval.txt /*Integer*
J change.txt /*J*
Japanese mbyte.txt /*Japanese*
Job eval.txt /*Job*
Jobs eval.txt /*Jobs*

K	various.txt	/*K*
KDE	gui_x11.txt	/*KDE*
KVim	gui_x11.txt	/*KVim*
Kibaale	uganda.txt	/*Kibaale*
Korean	mbyte.txt	/*Korean*
L	motion.txt	/*L*
Linux-backspace	options.txt	/*Linux-backspace*
List	eval.txt	/*List*
Lists	eval.txt	/*Lists*
LogiPat()	pi_logipat.txt	/*LogiPat()*
LogiPat-flags	pi_logipat.txt	/*LogiPat-flags*
Lua	if_lua.txt	/*Lua*
M	motion.txt	/*M*
MDI	starting.txt	/*MDI*
MS-DOS	os_msdos.txt	/*MS-DOS*
MS-Windows	os_win32.txt	/*MS-Windows*
MSDOS	os_msdos.txt	/*MSDOS*
MSVisualStudio	if_ole.txt	/*MSVisualStudio*
MVS	os_390.txt	/*MVS*
Mac	os_mac.txt	/*Mac*
Mac-format	editing.txt	/*Mac-format*
Mac-format-write	editing.txt	/*Mac-format-write*
Macintosh	os_mac.txt	/*Macintosh*
Mark	motion.txt	/*Mark*
MenuPopup	autocmd.txt	/*MenuPopup*
MiNT	os_mint.txt	/*MiNT*
Moolenaar	intro.txt	/*Moolenaar*
MorphOS	os_amiga.txt	/*MorphOS*
Motif	gui_x11.txt	/*Motif*
Myspell	spell.txt	/*Myspell*
MzScheme	if_mzsch.txt	/*MzScheme*
N	pattern.txt	/*N*
N%	motion.txt	/*N%*
N:	cmdline.txt	/*N:*
N<Del>	various.txt	/*N<Del>*
NFA	pattern.txt	/*NFA*
NL-used-for-Nul	pattern.txt	/*NL-used-for-Nul*
Neovim	intro.txt	/*Neovim*
NetBSD-backspace	options.txt	/*NetBSD-backspace*
NetBeans	netbeans.txt	/*NetBeans*
NetUserPass()	pi_netrw.txt	/*NetUserPass()*
None	eval.txt	/*None*
Normal	intro.txt	/*Normal*
Normal-mode	intro.txt	/*Normal-mode*
Number	eval.txt	/*Number*
Nvi	intro.txt	/*Nvi*
O	insert.txt	/*O*
OS/2	os_os2.txt	/*OS\/2*
OS2	os_os2.txt	/*OS2*
OS390	os_390.txt	/*OS390*
OS390-Motif	os_390.txt	/*OS390-Motif*
OS390-PuTTY	os_390.txt	/*OS390-PuTTY*
OS390-bugs	os_390.txt	/*OS390-bugs*
OS390-has-ebcdic	os_390.txt	/*OS390-has-ebcdic*

```
OS390-limitations           os_390.txt          /*OS390-limitations*
OS390-open-source           os_390.txt          /*OS390-open-source*
OffTheSpot                 mbyte.txt              /*OffTheSpot*
OnTheSpot                  mbyte.txt               /*OnTheSpot*
Operator-pending             intro.txt           /*Operator-pending*
Operator-pending-mode         intro.txt        /*Operator-pending-mode*
OptionSet                 autocmd.txt              /*OptionSet*
OverTheSpot                mbyte.txt             /*OverTheSpot*
P                         change.txt                  /*P*
PATHEXT                      eval.txt              /*PATHEXT*
PEP8                      filetype.txt                /*PEP8*
PHP_BracesAtCodeLevel        indent.txt        /*PHP_BracesAtCodeLevel*
PHP_IndentFunctionCallParameters                              indent.txt
/*PHP_IndentFunctionCallParameters*
PHP_IndentFunctionDeclarationParameters                       indent.txt
/*PHP_IndentFunctionDeclarationParameters*
PHP_autoformatcomment        indent.txt        /*PHP_autoformatcomment*
PHP_default_indenting        indent.txt        /*PHP_default_indenting*
PHP_noArrowMatching          indent.txt         /*PHP_noArrowMatching*
PHP_outdentSLComments        indent.txt        /*PHP_outdentSLComments*
PHP_outdentphpescape         indent.txt         /*PHP_outdentphpescape*
PHP_removeCRwhenUnix         indent.txt         /*PHP_removeCRwhenUnix*
PHP_vintage_case_default_indent  indent.txt  /*PHP_vintage_case_default_indent*
Partial                      eval.txt                /*Partial*
Pattern                    pattern.txt               /*Pattern*
Perl                      if_perl.txt                /*Perl*
Posix                      intro.txt                 /*Posix*
Python                    if_pyth.txt                /*Python*
Q                         intro.txt                  /*Q*
Q-command-changed            version5.txt          /*Q-command-changed*
QNX                       os_qnx.txt                 /*QNX*
Q_ab                      quickref.txt               /*Q_ab*
Q_ac                      quickref.txt               /*Q_ac*
Q_ai                      quickref.txt               /*Q_ai*
Q_bu                      quickref.txt               /*Q_bu*
Q_ce                      quickref.txt               /*Q_ce*
Q_ch                      quickref.txt               /*Q_ch*
Q_cm                      quickref.txt               /*Q_cm*
Q_co                      quickref.txt               /*Q_co*
Q_ct                       help.txt                 /*Q_ct*
Q_de                      quickref.txt               /*Q_de*
Q_di                      quickref.txt               /*Q_di*
Q_ed                      quickref.txt               /*Q_ed*
Q_et                      quickref.txt               /*Q_et*
Q_ex                      quickref.txt               /*Q_ex*
Q_fl                      quickref.txt               /*Q_fl*
Q_fo                      quickref.txt               /*Q_fo*
Q_gu                      quickref.txt               /*Q_gu*
Q_in                      quickref.txt               /*Q_in*
Q_km                      quickref.txt               /*Q_km*
Q_lr                      quickref.txt               /*Q_lr*
Q_ma                      quickref.txt               /*Q_ma*
Q_op                      quickref.txt               /*Q_op*
Q_pa                      quickref.txt               /*Q_pa*
```

```
Q_qf                    quickref.txt                /*Q_qf*
Q_ra                    quickref.txt                /*Q_ra*
Q_re                    quickref.txt                /*Q_re*
Q_sc                    quickref.txt                /*Q_sc*
Q_si                    quickref.txt                /*Q_si*
Q_ss                    quickref.txt                /*Q_ss*
Q_st                    quickref.txt                /*Q_st*
Q_sy                    quickref.txt                /*Q_sy*
Q_ta                    quickref.txt                /*Q_ta*
Q_tm                    quickref.txt                /*Q_tm*
Q_to                    quickref.txt                /*Q_to*
Q_ud                    quickref.txt                /*Q_ud*
Q_ur                    quickref.txt                /*Q_ur*
Q_vc                    quickref.txt                /*Q_vc*
Q_vi                    quickref.txt                /*Q_vi*
Q_vm                    quickref.txt                /*Q_vm*
Q_wi                    quickref.txt                /*Q_wi*
Q_wq                    quickref.txt                /*Q_wq*
QuickFixCmdPost         autocmd.txt              /*QuickFixCmdPost*
QuickFixCmdPost-example    quickfix.txt       /*QuickFixCmdPost-example*
QuickFixCmdPre          autocmd.txt              /*QuickFixCmdPre*
Quickfix                quickfix.txt                /*Quickfix*
QuitPre                 autocmd.txt                 /*QuitPre*
R                       change.txt                     /*R*
RISC-OS                 os_risc.txt                 /*RISC-OS*
RISCOS                  os_risc.txt                  /*RISCOS*
RemoteReply             autocmd.txt              /*RemoteReply*
Replace                 insert.txt                  /*Replace*
Replace-mode            insert.txt               /*Replace-mode*
Root                    mbyte.txt                      /*Root*
Ruby                    if_ruby.txt                    /*Ruby*
Russian                 russian.txt                 /*Russian*
S                       change.txt                      /*S*
SHELL                   starting.txt                 /*SHELL*
SQLGetType              ft_sql.txt               /*SQLGetType*
SQLSetType              ft_sql.txt               /*SQLSetType*
SafeState               autocmd.txt                /*SafeState*
SafeStateAgain          autocmd.txt             /*SafeStateAgain*
Select                  visual.txt                     /*Select*
Select-mode             visual.txt                /*Select-mode*
Select-mode-mapping        visual.txt          /*Select-mode-mapping*
Session                 starting.txt                    /*Session*
SessionLoad-variable       starting.txt        /*SessionLoad-variable*
SessionLoadPost         autocmd.txt              /*SessionLoadPost*
ShellCmdPost            autocmd.txt               /*ShellCmdPost*
ShellFilterPost         autocmd.txt              /*ShellFilterPost*
SourceCmd               autocmd.txt                 /*SourceCmd*
SourcePost              autocmd.txt                /*SourcePost*
SourcePre               autocmd.txt                 /*SourcePre*
Special                 eval.txt                    /*Special*
SpellFileMissing        autocmd.txt              /*SpellFileMissing*
StdinReadPost           autocmd.txt               /*StdinReadPost*
StdinReadPre            autocmd.txt               /*StdinReadPre*
String                  eval.txt                    /*String*
```

SwapExists	autocmd.txt	/*SwapExists*
Syntax	autocmd.txt	/*Syntax*
T	motion.txt	/*T*
TCL	if_tcl.txt	/*TCL*
TERM	starting.txt	/*TERM*
TOhtml-encoding	syntax.txt	/*TOhtml-encoding*
TOhtml-encoding-detect	syntax.txt	/*TOhtml-encoding-detect*
TOhtml-performance	syntax.txt	/*TOhtml-performance*
TOhtml-progress-color	syntax.txt	/*TOhtml-progress-color*
TOhtml-uncopyable-text	syntax.txt	/*TOhtml-uncopyable-text*
TOhtml-wrap-text	syntax.txt	/*TOhtml-wrap-text*
TRUE	eval.txt	/*TRUE*
TSQL	ft_sql.txt	/*TSQL*
TTpro-telnet	syntax.txt	/*TTpro-telnet*
Tab	intro.txt	/*Tab*
TabClosed	autocmd.txt	/*TabClosed*
TabEnter	autocmd.txt	/*TabEnter*
TabLeave	autocmd.txt	/*TabLeave*
TabNew	autocmd.txt	/*TabNew*
Tcl	if_tcl.txt	/*Tcl*
TermChanged	autocmd.txt	/*TermChanged*
TermResponse	autocmd.txt	/*TermResponse*
Terminal-Job	terminal.txt	/*Terminal-Job*
Terminal-Normal	terminal.txt	/*Terminal-Normal*
Terminal-mode	terminal.txt	/*Terminal-mode*
TerminalOpen	autocmd.txt	/*TerminalOpen*
TerminalWinOpen	autocmd.txt	/*TerminalWinOpen*
TextChanged	autocmd.txt	/*TextChanged*
TextChangedI	autocmd.txt	/*TextChangedI*
TextChangedP	autocmd.txt	/*TextChangedP*
TextYankPost	autocmd.txt	/*TextYankPost*
Transact-SQL	ft_sql.txt	/*Transact-SQL*
U	undo.txt	/*U*
UTF-8	mbyte.txt	/*UTF-8*
UTF8-xterm	mbyte.txt	/*UTF8-xterm*
Uganda	uganda.txt	/*Uganda*
Unicode	mbyte.txt	/*Unicode*
Unix	os_unix.txt	/*Unix*
Unix-format	editing.txt	/*Unix-format*
Unix-format-write	editing.txt	/*Unix-format-write*
User	autocmd.txt	/*User*
UserGettingBored	autocmd.txt	/*UserGettingBored*
V	visual.txt	/*V*
VIMINIT	starting.txt	/*VIMINIT*
VMS	os_vms.txt	/*VMS*
Vi	intro.txt	/*Vi*
View	starting.txt	/*View*
VimEnter	autocmd.txt	/*VimEnter*
VimLeave	autocmd.txt	/*VimLeave*
VimLeavePre	autocmd.txt	/*VimLeavePre*
VimResized	autocmd.txt	/*VimResized*
Vimball-copyright	pi_vimball.txt	/*Vimball-copyright*
Virtual-Replace-mode	insert.txt	/*Virtual-Replace-mode*
VisVim	if_ole.txt	/*VisVim*

Visual	visual.txt	/*Visual*
Visual-mode	visual.txt	/*Visual-mode*
W	motion.txt	/*W*
W10	message.txt	/*W10*
W11	message.txt	/*W11*
W12	message.txt	/*W12*
W13	message.txt	/*W13*
W14	message.txt	/*W14*
W15	repeat.txt	/*W15*
W16	message.txt	/*W16*
W17	arabic.txt	/*W17*
W18	syntax.txt	/*W18*
W19	autocmd.txt	/*W19*
W20	if_pyth.txt	/*W20*
W21	if_pyth.txt	/*W21*
W22	eval.txt	/*W22*
WORD	motion.txt	/*WORD*
WWW	intro.txt	/*WWW*
Win32	os_win32.txt	/*Win32*
WinBar	gui.txt	/*WinBar*
WinEnter	autocmd.txt	/*WinEnter*
WinLeave	autocmd.txt	/*WinLeave*
WinNew	autocmd.txt	/*WinNew*
X	change.txt	/*X*
X11	options.txt	/*X11*
X11-icon	gui_x11.txt	/*X11-icon*
X11_mouse_shapes	gui_x11.txt	/*X11_mouse_shapes*
X1Drag	term.txt	/*X1Drag*
X1Mouse	term.txt	/*X1Mouse*
X1Release	term.txt	/*X1Release*
X2Drag	term.txt	/*X2Drag*
X2Mouse	term.txt	/*X2Mouse*
X2Release	term.txt	/*X2Release*
XIM	mbyte.txt	/*XIM*
XLFD	mbyte.txt	/*XLFD*
Y	change.txt	/*Y*
Y2K	intro.txt	/*Y2K*
ZQ	editing.txt	/*ZQ*
ZZ	editing.txt	/*ZZ*
[	index.txt	/*[*
[#	motion.txt	/*[#*
[’	motion.txt	/*[’*
[(	motion.txt	/*[(*
[++opt]	editing.txt	/*[++opt]*
[+cmd]	editing.txt	/*[+cmd]*
[..]	pattern.txt	/*[..]*
[/	motion.txt	/*[\/*
[:alnum:]	pattern.txt	/*[:alnum:]*
[:alpha:]	pattern.txt	/*[:alpha:]*
[:backspace:]	pattern.txt	/*[:backspace:]*
[:blank:]	pattern.txt	/*[:blank:]*
[:cntrl:]	pattern.txt	/*[:cntrl:]*
[:digit:]	pattern.txt	/*[:digit:]*
[:escape:]	pattern.txt	/*[:escape:]*

```
[:fname:]               pattern.txt             /*[:fname:]*
[:graph:]               pattern.txt             /*[:graph:]*
[:ident:]               pattern.txt             /*[:ident:]*
[:keyword:]              pattern.txt            /*[:keyword:]*
[:lower:]               pattern.txt             /*[:lower:]*
[:print:]               pattern.txt             /*[:print:]*
[:punct:]               pattern.txt             /*[:punct:]*
[:return:]               pattern.txt            /*[:return:]*
[:space:]               pattern.txt             /*[:space:]*
[:tab:]                 pattern.txt              /*[:tab:]*
[:upper:]               pattern.txt             /*[:upper:]*
[:xdigit:]               pattern.txt            /*[:xdigit:]*
[<MiddleMouse>            change.txt           /*[<MiddleMouse>*
[==]                   pattern.txt                /*[==]*
[D                    tagsrch.txt                 /*[D*
[I                    tagsrch.txt                 /*[I*
[M                    motion.txt                  /*[M*
[P                    change.txt                  /*[P*
[S                     spell.txt                  /*[S*
[[                    motion.txt                  /*[[*
[]                    motion.txt                  /*[]*
[_CTRL-D                tagsrch.txt              /*[_CTRL-D*
[_CTRL-I                tagsrch.txt              /*[_CTRL-I*
[‘                    motion.txt                  /*[‘*
[c                     diff.txt                   /*[c*
[count]                 intro.txt                /*[count]*
[d                    tagsrch.txt                 /*[d*
[f                    editing.txt                 /*[f*
[i                    tagsrch.txt                 /*[i*
[m                    motion.txt                  /*[m*
[p                    change.txt                  /*[p*
[pattern]               pattern.txt              /*[pattern]*
[quotex]                 intro.txt               /*[quotex]*
[range]                 cmdline.txt              /*[range]*
[s                     spell.txt                  /*[s*
[star                   motion.txt                /*[star*
[z                     fold.txt                   /*[z*
[{                    motion.txt                  /*[{*
\0                    change.txt                  /*\\0*
]                     index.txt                   /*]*
]#                    motion.txt                  /*]#*
]’                    motion.txt                  /*]’*
])                    motion.txt                  /*])*
]/                    motion.txt                  /*]\/*
]<MiddleMouse>            change.txt           /*]<MiddleMouse>*
]D                    tagsrch.txt                 /*]D*
]I                    tagsrch.txt                 /*]I*
]M                    motion.txt                  /*]M*
]P                    change.txt                  /*]P*
]S                     spell.txt                  /*]S*
][                    motion.txt                  /*][*
]]                    motion.txt                  /*]]*
]_CTRL-D                tagsrch.txt              /*]_CTRL-D*
]_CTRL-I                tagsrch.txt              /*]_CTRL-I*
```

]‘	motion.txt	/*]‘*
]c	diff.txt	/*]c*
]d	tagsrch.txt	/*]d*
]f	editing.txt	/*]f*
]i	tagsrch.txt	/*]i*
]m	motion.txt	/*]m*
]p	change.txt	/*]p*
]s	spell.txt	/*]s*
]star	motion.txt	/*]star*
]z	fold.txt	/*]z*
]}	motion.txt	/*]}*
^	motion.txt	/*^*
_	motion.txt	/*_*
_exrc	starting.txt	/*_exrc*
_gvimrc	gui.txt	/*_gvimrc*
_vimrc	starting.txt	/*_vimrc*
‘	motion.txt	/*‘*
‘(	motion.txt	/*‘(*
‘)	motion.txt	/*‘)*
‘-expansion	editing.txt	/*‘-expansion*
‘.	motion.txt	/*‘.*
‘0	motion.txt	/*‘0*
‘<	motion.txt	/*‘<*
‘=	editing.txt	/*‘=*
‘>	motion.txt	/*‘>*
‘A	motion.txt	/*‘A*
‘[	motion.txt	/*‘[*
‘]	motion.txt	/*‘]*
‘^	motion.txt	/*‘^*
‘‘	motion.txt	/*‘‘*
‘a	motion.txt	/*‘a*
‘quote	motion.txt	/*‘quote*
‘{	motion.txt	/*‘{*
‘}	motion.txt	/*‘}*
a	insert.txt	/*a*
a’	motion.txt	/*a’*
a(	motion.txt	/*a(*
a)	motion.txt	/*a)*
a4	print.txt	/*a4*
a:0	eval.txt	/*a:0*
a:000	eval.txt	/*a:000*
a:1	eval.txt	/*a:1*
a:firstline	eval.txt	/*a:firstline*
a:lastline	eval.txt	/*a:lastline*
a:var	eval.txt	/*a:var*
a<	motion.txt	/*a<*
a>	motion.txt	/*a>*
aB	motion.txt	/*aB*
aW	motion.txt	/*aW*
a[	motion.txt	/*a[*
a]	motion.txt	/*a]*
a‘	motion.txt	/*a‘*
ab	motion.txt	/*ab*
abandon	editing.txt	/*abandon*

```
abbreviations                map.txt              /*abbreviations*
abel.vim                    syntax.txt               /*abel.vim*
abs()                       eval.txt                 /*abs()*
acos()                      eval.txt                 /*acos()*
active-buffer                windows.txt            /*active-buffer*
ada#Create_Tags()             ft_ada.txt          /*ada#Create_Tags()*
ada#Jump_Tag()               ft_ada.txt            /*ada#Jump_Tag()*
ada#Listtags()               ft_ada.txt            /*ada#Listtags()*
ada#Switch_Syntax_Option()      ft_ada.txt      /*ada#Switch_Syntax_Option()*
ada#Word()                  ft_ada.txt               /*ada#Word()*
ada-compiler                 ft_ada.txt            /*ada-compiler*
ada-ctags                   ft_ada.txt               /*ada-ctags*
ada-extra-plugins             ft_ada.txt          /*ada-extra-plugins*
ada-reference                ft_ada.txt            /*ada-reference*
ada.vim                     ft_ada.txt               /*ada.vim*
add()                       eval.txt                 /*add()*
add-filetype-plugin           usr_05.txt          /*add-filetype-plugin*
add-global-plugin             usr_05.txt          /*add-global-plugin*
add-local-help               usr_05.txt            /*add-local-help*
add-option-flags             options.txt           /*add-option-flags*
add-package                 usr_05.txt               /*add-package*
add-plugin                  usr_05.txt               /*add-plugin*
added-5.1                   version5.txt             /*added-5.1*
added-5.2                   version5.txt             /*added-5.2*
added-5.3                   version5.txt             /*added-5.3*
added-5.4                   version5.txt             /*added-5.4*
added-5.5                   version5.txt             /*added-5.5*
added-5.6                   version5.txt             /*added-5.6*
added-5.7                   version5.txt             /*added-5.7*
added-5.8                   version5.txt             /*added-5.8*
added-6.1                   version6.txt             /*added-6.1*
added-6.2                   version6.txt             /*added-6.2*
added-6.3                   version6.txt             /*added-6.3*
added-6.4                   version6.txt             /*added-6.4*
added-7.1                   version7.txt             /*added-7.1*
added-7.2                   version7.txt             /*added-7.2*
added-7.3                   version7.txt             /*added-7.3*
added-7.4                   version7.txt             /*added-7.4*
added-8.1                   version8.txt             /*added-8.1*
added-8.2                   version8.txt             /*added-8.2*
added-BeOS                  version5.txt             /*added-BeOS*
added-Mac                   version5.txt             /*added-Mac*
added-VMS                   version5.txt             /*added-VMS*
added-cmdline-args             version5.txt       /*added-cmdline-args*
added-options                version5.txt           /*added-options*
added-regexp                 version5.txt           /*added-regexp*
added-various                version5.txt           /*added-various*
added-win32-GUI               version5.txt          /*added-win32-GUI*
aff-dic-format                spell.txt            /*aff-dic-format*
after-directory              options.txt           /*after-directory*
aleph                       options.txt                 /*aleph*
alt                         intro.txt                   /*alt*
alt-input                   debugger.txt              /*alt-input*
alternate-file                editing.txt           /*alternate-file*
```

amiga-window	starting.txt	/*amiga-window*
and()	eval.txt	/*and()*
anonymous-function	eval.txt	/*anonymous-function*
ant.vim	syntax.txt	/*ant.vim*
ap	motion.txt	/*ap*
apache.vim	syntax.txt	/*apache.vim*
append()	eval.txt	/*append()*
appendbufline()	eval.txt	/*appendbufline()*
aquote	motion.txt	/*aquote*
arabic.txt	arabic.txt	/*arabic.txt*
arabicfonts	arabic.txt	/*arabicfonts*
arabickeymap	arabic.txt	/*arabickeymap*
arg-functions	usr_41.txt	/*arg-functions*
argc()	eval.txt	/*argc()*
argidx()	eval.txt	/*argidx()*
arglist	editing.txt	/*arglist*
arglist-position	editing.txt	/*arglist-position*
arglist-quit	usr_07.txt	/*arglist-quit*
arglistid()	eval.txt	/*arglistid()*
argument-list	editing.txt	/*argument-list*
argv()	eval.txt	/*argv()*
argv-variable	eval.txt	/*argv-variable*
as	motion.txt	/*as*
asin()	eval.txt	/*asin()*
asm.vim	syntax.txt	/*asm.vim*
asm68k	syntax.txt	/*asm68k*
asmh8300.vim	syntax.txt	/*asmh8300.vim*
assert-functions-details	testing.txt	/*assert-functions-details*
assert-return	eval.txt	/*assert-return*
assert_beeps()	testing.txt	/*assert_beeps()*
assert_equal()	testing.txt	/*assert_equal()*
assert_equalfile()	testing.txt	/*assert_equalfile()*
assert_exception()	testing.txt	/*assert_exception()*
assert_fails()	testing.txt	/*assert_fails()*
assert_false()	testing.txt	/*assert_false()*
assert_inrange()	testing.txt	/*assert_inrange()*
assert_match()	testing.txt	/*assert_match()*
assert_notequal()	testing.txt	/*assert_notequal()*
assert_notmatch()	testing.txt	/*assert_notmatch()*
assert_report()	testing.txt	/*assert_report()*
assert_true()	testing.txt	/*assert_true()*
at	motion.txt	/*at*
atan()	eval.txt	/*atan()*
atan2()	eval.txt	/*atan2()*
athena-intellimouse	gui.txt	/*athena-intellimouse*
attr-list	syntax.txt	/*attr-list*
author	intro.txt	/*author*
auto-format	change.txt	/*auto-format*
auto-setting	options.txt	/*auto-setting*
auto-shortname	editing.txt	/*auto-shortname*
autocmd-<>	tips.txt	/*autocmd-<>*
autocmd-buffer-local	autocmd.txt	/*autocmd-buffer-local*
autocmd-buflocal	autocmd.txt	/*autocmd-buflocal*
autocmd-changes	autocmd.txt	/*autocmd-changes*

```
autocmd-define              autocmd.txt             /*autocmd-define*
autocmd-disable             autocmd.txt             /*autocmd-disable*
autocmd-events              autocmd.txt             /*autocmd-events*
autocmd-events-abc          autocmd.txt           /*autocmd-events-abc*
autocmd-execute             autocmd.txt             /*autocmd-execute*
autocmd-groups              autocmd.txt             /*autocmd-groups*
autocmd-intro               autocmd.txt              /*autocmd-intro*
autocmd-list                autocmd.txt              /*autocmd-list*
autocmd-nested              autocmd.txt             /*autocmd-nested*
autocmd-once                autocmd.txt              /*autocmd-once*
autocmd-osfiletypes         filetype.txt          /*autocmd-osfiletypes*
autocmd-patterns            autocmd.txt            /*autocmd-patterns*
autocmd-remove              autocmd.txt             /*autocmd-remove*
autocmd-searchpat           autocmd.txt           /*autocmd-searchpat*
autocmd-use                 autocmd.txt               /*autocmd-use*
autocmd.txt                 autocmd.txt               /*autocmd.txt*
autocmds-kept               version5.txt              /*autocmds-kept*
autocommand                 autocmd.txt               /*autocommand*
autocommand-events          autocmd.txt           /*autocommand-events*
autocommand-pattern         autocmd.txt          /*autocommand-pattern*
autocommands                autocmd.txt               /*autocommands*
autoformat                  change.txt                /*autoformat*
autoload                    eval.txt                  /*autoload*
autoload-functions          eval.txt             /*autoload-functions*
avoid-hit-enter             version5.txt              /*avoid-hit-enter*
aw                          motion.txt                    /*aw*
a{                          motion.txt                    /*a{*
a}                          motion.txt                    /*a}*
b                           motion.txt                    /*b*
b:                          eval.txt                     /*b:*
b:changedtick               eval.txt                /*b:changedtick*
b:changelog_name            filetype.txt            /*b:changelog_name*
b:clojure_syntax_keywords   syntax.txt      /*b:clojure_syntax_keywords*
b:clojure_syntax_without_core_keywords                        syntax.txt
/*b:clojure_syntax_without_core_keywords*
b:current_syntax-variable   syntax.txt      /*b:current_syntax-variable*
b:netrw_lastfile            pi_netrw.txt            /*b:netrw_lastfile*
b:tex_stylish               syntax.txt              /*b:tex_stylish*
b:var                       eval.txt                   /*b:var*
b:yaml_schema               syntax.txt              /*b:yaml_schema*
baan-folding                syntax.txt              /*baan-folding*
baan-syntax                 syntax.txt              /*baan-syntax*
baan.vim                    syntax.txt               /*baan.vim*
backslash                   intro.txt               /*backslash*
backspace                   intro.txt               /*backspace*
backspace-delete            version4.txt          /*backspace-delete*
backtick-expansion          editing.txt           /*backtick-expansion*
backup                      editing.txt                 /*backup*
backup-changed              version4.txt            /*backup-changed*
backup-extension            version4.txt           /*backup-extension*
backup-table                editing.txt              /*backup-table*
balloon-eval                debugger.txt             /*balloon-eval*
balloon_gettext()           eval.txt              /*balloon_gettext()*
balloon_show()              eval.txt               /*balloon_show()*
```

balloon_split() eval.txt /*balloon_split()*
bar motion.txt /*bar*
bars help.txt /*bars*
base_font_name_list mbyte.txt /*base_font_name_list*
basic.vim syntax.txt /*basic.vim*
beep options.txt /*beep*
beos-colors os_beos.txt /*beos-colors*
beos-compiling os_beos.txt /*beos-compiling*
beos-dragndrop os_beos.txt /*beos-dragndrop*
beos-fonts os_beos.txt /*beos-fonts*
beos-general os_beos.txt /*beos-general*
beos-gui os_beos.txt /*beos-gui*
beos-launch os_beos.txt /*beos-launch*
beos-meta os_beos.txt /*beos-meta*
beos-mouse os_beos.txt /*beos-mouse*
beos-perl os_beos.txt /*beos-perl*
beos-timeout os_beos.txt /*beos-timeout*
beos-unicode os_beos.txt /*beos-unicode*
beos-utf8 os_beos.txt /*beos-utf8*
beos-vimdir os_beos.txt /*beos-vimdir*
better-python-interface version7.txt /*better-python-interface*
beval_bufnr-variable eval.txt /*beval_bufnr-variable*
beval_col-variable eval.txt /*beval_col-variable*
beval_lnum-variable eval.txt /*beval_lnum-variable*
beval_text-variable eval.txt /*beval_text-variable*
beval_winid-variable eval.txt /*beval_winid-variable*
beval_winnr-variable eval.txt /*beval_winnr-variable*
binary-number eval.txt /*binary-number*
bitwise-function usr_41.txt /*bitwise-function*
blob eval.txt /*blob*
blob-identity eval.txt /*blob-identity*
blob-index eval.txt /*blob-index*
blob-literal eval.txt /*blob-literal*
blob-modification eval.txt /*blob-modification*
blockwise-examples visual.txt /*blockwise-examples*
blockwise-operators visual.txt /*blockwise-operators*
blockwise-register change.txt /*blockwise-register*
blockwise-visual visual.txt /*blockwise-visual*
blowfish options.txt /*blowfish*
blowfish2 options.txt /*blowfish2*
bold syntax.txt /*bold*
bom-bytes mbyte.txt /*bom-bytes*
book intro.txt /*book*
bookmark usr_03.txt /*bookmark*
books intro.txt /*books*
boolean options.txt /*boolean*
break-finally eval.txt /*break-finally*
browse() eval.txt /*browse()*
browsedir() eval.txt /*browsedir()*
browsefilter editing.txt /*browsefilter*
bufadd() eval.txt /*bufadd()*
bufexists() eval.txt /*bufexists()*
buffer-functions usr_41.txt /*buffer-functions*
buffer-hidden windows.txt /*buffer-hidden*

buffer-list	windows.txt	/*buffer-list*
buffer-variable	eval.txt	/*buffer-variable*
buffer-write	editing.txt	/*buffer-write*
buffer_exists()	eval.txt	/*buffer_exists()*
buffer_name()	eval.txt	/*buffer_name()*
buffer_number()	eval.txt	/*buffer_number()*
buffers	windows.txt	/*buffers*
buffers-menu	gui.txt	/*buffers-menu*
buflisted()	eval.txt	/*buflisted()*
bufload()	eval.txt	/*bufload()*
bufloaded()	eval.txt	/*bufloaded()*
bufname()	eval.txt	/*bufname()*
bufnr()	eval.txt	/*bufnr()*
bufwinid()	eval.txt	/*bufwinid()*
bufwinnr()	eval.txt	/*bufwinnr()*
bug-fixes-5	version5.txt	/*bug-fixes-5*
bug-fixes-6	version6.txt	/*bug-fixes-6*
bug-fixes-7	version7.txt	/*bug-fixes-7*
bug-fixes-8	version8.txt	/*bug-fixes-8*
bug-reports	intro.txt	/*bug-reports*
bugreport.vim	intro.txt	/*bugreport.vim*
bugs	intro.txt	/*bugs*
builtin-terms	term.txt	/*builtin-terms*
builtin-tools	gui.txt	/*builtin-tools*
builtin_terms	term.txt	/*builtin_terms*
byte-count	editing.txt	/*byte-count*
byte2line()	eval.txt	/*byte2line()*
byteidx()	eval.txt	/*byteidx()*
byteidxcomp()	eval.txt	/*byteidxcomp()*
bzip2	pi_gzip.txt	/*bzip2*
c	change.txt	/*c*
c.vim	syntax.txt	/*c.vim*
cW	change.txt	/*cW*
c_#	cmdline.txt	/*c_#*
c_##	cmdline.txt	/*c_##*
c_#<	cmdline.txt	/*c_#<*
c_#n	cmdline.txt	/*c_#n*
c_%	cmdline.txt	/*c_%*
c_<BS>	cmdline.txt	/*c_<BS>*
c_<C-Left>	cmdline.txt	/*c_<C-Left>*
c_<C-R>	cmdline.txt	/*c_<C-R>*
c_<C-R>_<C-A>	cmdline.txt	/*c_<C-R>_<C-A>*
c_<C-R>_<C-F>	cmdline.txt	/*c_<C-R>_<C-F>*
c_<C-R>_<C-L>	cmdline.txt	/*c_<C-R>_<C-L>*
c_<C-R>_<C-O>	cmdline.txt	/*c_<C-R>_<C-O>*
c_<C-R>_<C-P>	cmdline.txt	/*c_<C-R>_<C-P>*
c_<C-R>_<C-R>	cmdline.txt	/*c_<C-R>_<C-R>*
c_<C-R>_<C-W>	cmdline.txt	/*c_<C-R>_<C-W>*
c_<C-Right>	cmdline.txt	/*c_<C-Right>*
c_<CR>	cmdline.txt	/*c_<CR>*
c_<Del>	cmdline.txt	/*c_<Del>*
c_<Down>	cmdline.txt	/*c_<Down>*
c_<End>	cmdline.txt	/*c_<End>*
c_<Esc>	cmdline.txt	/*c_<Esc>*

c_<Home> cmdline.txt /*c_<Home>*
c_<Insert> cmdline.txt /*c_<Insert>*
c_<Left> cmdline.txt /*c_<Left>*
c_<LeftMouse> cmdline.txt /*c_<LeftMouse>*
c_<MiddleMouse> cmdline.txt /*c_<MiddleMouse>*
c_<NL> cmdline.txt /*c_<NL>*
c_<PageDown> cmdline.txt /*c_<PageDown>*
c_<PageUp> cmdline.txt /*c_<PageUp>*
c_<Right> cmdline.txt /*c_<Right>*
c_<S-Down> cmdline.txt /*c_<S-Down>*
c_<S-Left> cmdline.txt /*c_<S-Left>*
c_<S-Right> cmdline.txt /*c_<S-Right>*
c_<S-Tab> cmdline.txt /*c_<S-Tab>*
c_<S-Up> cmdline.txt /*c_<S-Up>*
c_<Tab> cmdline.txt /*c_<Tab>*
c_<Up> cmdline.txt /*c_<Up>*
c_BS cmdline.txt /*c_BS*
c_CR cmdline.txt /*c_CR*
c_CTRL-A cmdline.txt /*c_CTRL-A*
c_CTRL-B cmdline.txt /*c_CTRL-B*
c_CTRL-C cmdline.txt /*c_CTRL-C*
c_CTRL-D cmdline.txt /*c_CTRL-D*
c_CTRL-E cmdline.txt /*c_CTRL-E*
c_CTRL-F cmdline.txt /*c_CTRL-F*
c_CTRL-G cmdline.txt /*c_CTRL-G*
c_CTRL-H cmdline.txt /*c_CTRL-H*
c_CTRL-I cmdline.txt /*c_CTRL-I*
c_CTRL-J cmdline.txt /*c_CTRL-J*
c_CTRL-K cmdline.txt /*c_CTRL-K*
c_CTRL-L cmdline.txt /*c_CTRL-L*
c_CTRL-M cmdline.txt /*c_CTRL-M*
c_CTRL-N cmdline.txt /*c_CTRL-N*
c_CTRL-P cmdline.txt /*c_CTRL-P*
c_CTRL-Q cmdline.txt /*c_CTRL-Q*
c_CTRL-R cmdline.txt /*c_CTRL-R*
c_CTRL-R_= cmdline.txt /*c_CTRL-R_=*
c_CTRL-R_CTRL-A cmdline.txt /*c_CTRL-R_CTRL-A*
c_CTRL-R_CTRL-F cmdline.txt /*c_CTRL-R_CTRL-F*
c_CTRL-R_CTRL-L cmdline.txt /*c_CTRL-R_CTRL-L*
c_CTRL-R_CTRL-O cmdline.txt /*c_CTRL-R_CTRL-O*
c_CTRL-R_CTRL-P cmdline.txt /*c_CTRL-R_CTRL-P*
c_CTRL-R_CTRL-R cmdline.txt /*c_CTRL-R_CTRL-R*
c_CTRL-R_CTRL-W cmdline.txt /*c_CTRL-R_CTRL-W*
c_CTRL-SHIFT-Q cmdline.txt /*c_CTRL-SHIFT-Q*
c_CTRL-SHIFT-V cmdline.txt /*c_CTRL-SHIFT-V*
c_CTRL-T cmdline.txt /*c_CTRL-T*
c_CTRL-U cmdline.txt /*c_CTRL-U*
c_CTRL-V cmdline.txt /*c_CTRL-V*
c_CTRL-W cmdline.txt /*c_CTRL-W*
c_CTRL-Y cmdline.txt /*c_CTRL-Y*
c_CTRL-[cmdline.txt /*c_CTRL-[*
c_CTRL-_CTRL-G intro.txt /*c_CTRL-_CTRL-G*
c_CTRL-_CTRL-N intro.txt /*c_CTRL-_CTRL-N*
c_CTRL-_e cmdline.txt /*c_CTRL-_e*

c_CTRL-] cmdline.txt /*c_CTRL-]*
c_CTRL-^ cmdline.txt /*c_CTRL-^*
c_CTRL-_ cmdline.txt /*c_CTRL-_*
c_Del cmdline.txt /*c_Del*
c_Down cmdline.txt /*c_Down*
c_End cmdline.txt /*c_End*
c_Esc cmdline.txt /*c_Esc*
c_Home cmdline.txt /*c_Home*
c_Insert cmdline.txt /*c_Insert*
c_Left cmdline.txt /*c_Left*
c_Right cmdline.txt /*c_Right*
c_Up cmdline.txt /*c_Up*
c_ansi_constants syntax.txt /*c_ansi_constants*
c_ansi_typedefs syntax.txt /*c_ansi_typedefs*
c_comment_strings syntax.txt /*c_comment_strings*
c_curly_error syntax.txt /*c_curly_error*
c_digraph cmdline.txt /*c_digraph*
c_gnu syntax.txt /*c_gnu*
c_no_ansi syntax.txt /*c_no_ansi*
c_no_bracket_error syntax.txt /*c_no_bracket_error*
c_no_bsd syntax.txt /*c_no_bsd*
c_no_c11 syntax.txt /*c_no_c11*
c_no_c99 syntax.txt /*c_no_c99*
c_no_cformat syntax.txt /*c_no_cformat*
c_no_curly_error syntax.txt /*c_no_curly_error*
c_no_if0 syntax.txt /*c_no_if0*
c_no_tab_space_error syntax.txt /*c_no_tab_space_error*
c_no_trail_space_error syntax.txt /*c_no_trail_space_error*
c_no_utf syntax.txt /*c_no_utf*
c_space_errors syntax.txt /*c_space_errors*
c_syntax_for_h syntax.txt /*c_syntax_for_h*
c_wildchar cmdline.txt /*c_wildchar*
call() eval.txt /*call()*
carriage-return intro.txt /*carriage-return*
case change.txt /*case*
catch-all eval.txt /*catch-all*
catch-errors eval.txt /*catch-errors*
catch-interrupt eval.txt /*catch-interrupt*
catch-order eval.txt /*catch-order*
catch-text eval.txt /*catch-text*
cc change.txt /*cc*
ceil() eval.txt /*ceil()*
cfilter-plugin quickfix.txt /*cfilter-plugin*
ch.vim syntax.txt /*ch.vim*
ch_canread() channel.txt /*ch_canread()*
ch_close() channel.txt /*ch_close()*
ch_close_in() channel.txt /*ch_close_in()*
ch_evalexpr() channel.txt /*ch_evalexpr()*
ch_evalraw() channel.txt /*ch_evalraw()*
ch_getbufnr() channel.txt /*ch_getbufnr()*
ch_getjob() channel.txt /*ch_getjob()*
ch_info() channel.txt /*ch_info()*
ch_log() channel.txt /*ch_log()*
ch_logfile() channel.txt /*ch_logfile()*

ch_open() channel.txt /*ch_open()*
ch_read() channel.txt /*ch_read()*
ch_readblob() channel.txt /*ch_readblob()*
ch_readraw() channel.txt /*ch_readraw()*
ch_sendexpr() channel.txt /*ch_sendexpr()*
ch_sendraw() channel.txt /*ch_sendraw()*
ch_setoptions() channel.txt /*ch_setoptions()*
ch_status() channel.txt /*ch_status()*
change-list-jumps motion.txt /*change-list-jumps*
change-name tips.txt /*change-name*
change-tabs change.txt /*change-tabs*
change.txt change.txt /*change.txt*
changed-5.1 version5.txt /*changed-5.1*
changed-5.2 version5.txt /*changed-5.2*
changed-5.3 version5.txt /*changed-5.3*
changed-5.4 version5.txt /*changed-5.4*
changed-5.5 version5.txt /*changed-5.5*
changed-5.6 version5.txt /*changed-5.6*
changed-5.7 version5.txt /*changed-5.7*
changed-5.8 version5.txt /*changed-5.8*
changed-6.1 version6.txt /*changed-6.1*
changed-6.2 version6.txt /*changed-6.2*
changed-6.3 version6.txt /*changed-6.3*
changed-6.4 version6.txt /*changed-6.4*
changed-7.1 version7.txt /*changed-7.1*
changed-7.2 version7.txt /*changed-7.2*
changed-7.3 version7.txt /*changed-7.3*
changed-7.4 version7.txt /*changed-7.4*
changed-8.1 version8.txt /*changed-8.1*
changed-8.2 version8.txt /*changed-8.2*
changelist motion.txt /*changelist*
changelog.vim syntax.txt /*changelog.vim*
changenr() eval.txt /*changenr()*
changetick eval.txt /*changetick*
changing change.txt /*changing*
channel channel.txt /*channel*
channel-callback channel.txt /*channel-callback*
channel-close channel.txt /*channel-close*
channel-close-in channel.txt /*channel-close-in*
channel-commands channel.txt /*channel-commands*
channel-demo channel.txt /*channel-demo*
channel-drop channel.txt /*channel-drop*
channel-functions usr_41.txt /*channel-functions*
channel-functions-details channel.txt /*channel-functions-details*
channel-mode channel.txt /*channel-mode*
channel-more channel.txt /*channel-more*
channel-noblock channel.txt /*channel-noblock*
channel-open channel.txt /*channel-open*
channel-open-options channel.txt /*channel-open-options*
channel-raw channel.txt /*channel-raw*
channel-timeout channel.txt /*channel-timeout*
channel-use channel.txt /*channel-use*
channel.txt channel.txt /*channel.txt*
char-variable eval.txt /*char-variable*

```
char2nr()                   eval.txt              /*char2nr()*
characterwise               motion.txt            /*characterwise*
characterwise-register      change.txt            /*characterwise-register*
characterwise-visual        visual.txt            /*characterwise-visual*
charconvert_from-variable   eval.txt              /*charconvert_from-variable*
charconvert_to-variable     eval.txt              /*charconvert_to-variable*
charity                     uganda.txt            /*charity*
charset                     mbyte.txt             /*charset*
charset-conversion          mbyte.txt             /*charset-conversion*
chdir()                     eval.txt              /*chdir()*
chill.vim                   syntax.txt            /*chill.vim*
chmod                       eval.txt              /*chmod*
cindent()                   eval.txt              /*cindent()*
cinkeys-format              indent.txt            /*cinkeys-format*
cino-#                      indent.txt            /*cino-#*
cino-(                      indent.txt            /*cino-(*
cino-)                      indent.txt            /*cino-)*
cino-+                      indent.txt            /*cino-+*
cino-/                      indent.txt            /*cino-\/*
cino-:                      indent.txt            /*cino-:*
cino-=                      indent.txt            /*cino-=*
cino->                      indent.txt            /*cino->*
cino-C                      indent.txt            /*cino-C*
cino-E                      indent.txt            /*cino-E*
cino-J                      indent.txt            /*cino-J*
cino-L                      indent.txt            /*cino-L*
cino-M                      indent.txt            /*cino-M*
cino-N                      indent.txt            /*cino-N*
cino-U                      indent.txt            /*cino-U*
cino-W                      indent.txt            /*cino-W*
cino-^                      indent.txt            /*cino-^*
cino-b                      indent.txt            /*cino-b*
cino-c                      indent.txt            /*cino-c*
cino-e                      indent.txt            /*cino-e*
cino-f                      indent.txt            /*cino-f*
cino-g                      indent.txt            /*cino-g*
cino-h                      indent.txt            /*cino-h*
cino-i                      indent.txt            /*cino-i*
cino-j                      indent.txt            /*cino-j*
cino-k                      indent.txt            /*cino-k*
cino-l                      indent.txt            /*cino-l*
cino-m                      indent.txt            /*cino-m*
cino-n                      indent.txt            /*cino-n*
cino-p                      indent.txt            /*cino-p*
cino-star                   indent.txt            /*cino-star*
cino-t                      indent.txt            /*cino-t*
cino-u                      indent.txt            /*cino-u*
cino-w                      indent.txt            /*cino-w*
cino-{                      indent.txt            /*cino-{*
cino-}                      indent.txt            /*cino-}*
cinoptions-values           indent.txt            /*cinoptions-values*
clear-undo                  undo.txt              /*clear-undo*
clearmatches()              eval.txt              /*clearmatches()*
client-server               remote.txt            /*client-server*
```

client-server-name remote.txt /*client-server-name*
clientserver remote.txt /*clientserver*
clipboard gui.txt /*clipboard*
clipboard-autoselect options.txt /*clipboard-autoselect*
clipboard-autoselectml options.txt /*clipboard-autoselectml*
clipboard-autoselectplus options.txt /*clipboard-autoselectplus*
clipboard-exclude options.txt /*clipboard-exclude*
clipboard-html options.txt /*clipboard-html*
clipboard-unnamed options.txt /*clipboard-unnamed*
clipboard-unnamedplus options.txt /*clipboard-unnamedplus*
clojure-indent indent.txt /*clojure-indent*
close_cb channel.txt /*close_cb*
closure eval.txt /*closure*
cmdarg-variable eval.txt /*cmdarg-variable*
cmdbang-variable eval.txt /*cmdbang-variable*
cmdline-arguments vi_diff.txt /*cmdline-arguments*
cmdline-changed version5.txt /*cmdline-changed*
cmdline-completion cmdline.txt /*cmdline-completion*
cmdline-editing cmdline.txt /*cmdline-editing*
cmdline-history cmdline.txt /*cmdline-history*
cmdline-lines cmdline.txt /*cmdline-lines*
cmdline-ranges cmdline.txt /*cmdline-ranges*
cmdline-special cmdline.txt /*cmdline-special*
cmdline-too-long cmdline.txt /*cmdline-too-long*
cmdline-window cmdline.txt /*cmdline-window*
cmdline.txt cmdline.txt /*cmdline.txt*
cmdwin cmdline.txt /*cmdwin*
cmdwin-char cmdline.txt /*cmdwin-char*
cobol.vim syntax.txt /*cobol.vim*
codeset mbyte.txt /*codeset*
coding-style develop.txt /*coding-style*
col() eval.txt /*col()*
coldfusion.vim syntax.txt /*coldfusion.vim*
collapse tips.txt /*collapse*
color-xterm syntax.txt /*color-xterm*
coloring syntax.txt /*coloring*
colortest.vim syntax.txt /*colortest.vim*
command-line-functions usr_41.txt /*command-line-functions*
command-line-window cmdline.txt /*command-line-window*
command-mode intro.txt /*command-mode*
compatible-default starting.txt /*compatible-default*
compile-changes-5 version5.txt /*compile-changes-5*
compile-changes-6 version6.txt /*compile-changes-6*
compile-changes-7 version7.txt /*compile-changes-7*
compile-changes-8 version8.txt /*compile-changes-8*
compiler-compaqada ft_ada.txt /*compiler-compaqada*
compiler-decada ft_ada.txt /*compiler-decada*
compiler-gcc quickfix.txt /*compiler-gcc*
compiler-gnat ft_ada.txt /*compiler-gnat*
compiler-hpada ft_ada.txt /*compiler-hpada*
compiler-manx quickfix.txt /*compiler-manx*
compiler-perl quickfix.txt /*compiler-perl*
compiler-pyunit quickfix.txt /*compiler-pyunit*
compiler-select quickfix.txt /*compiler-select*

compiler-tex quickfix.txt /*compiler-tex*
compiler-vaxada ft_ada.txt /*compiler-vaxada*
compl-current insert.txt /*compl-current*
compl-define insert.txt /*compl-define*
compl-dictionary insert.txt /*compl-dictionary*
compl-filename insert.txt /*compl-filename*
compl-function insert.txt /*compl-function*
compl-generic insert.txt /*compl-generic*
compl-keyword insert.txt /*compl-keyword*
compl-omni insert.txt /*compl-omni*
compl-omni-filetypes insert.txt /*compl-omni-filetypes*
compl-spelling insert.txt /*compl-spelling*
compl-tag insert.txt /*compl-tag*
compl-vim insert.txt /*compl-vim*
compl-whole-line insert.txt /*compl-whole-line*
complete() eval.txt /*complete()*
complete-functions insert.txt /*complete-functions*
complete-item-kind insert.txt /*complete-item-kind*
complete-items insert.txt /*complete-items*
complete-popup insert.txt /*complete-popup*
complete-popuphidden insert.txt /*complete-popuphidden*
complete_CTRL-E insert.txt /*complete_CTRL-E*
complete_CTRL-Y insert.txt /*complete_CTRL-Y*
complete_add() eval.txt /*complete_add()*
complete_check() eval.txt /*complete_check()*
complete_info() eval.txt /*complete_info()*
complete_info_mode eval.txt /*complete_info_mode*
completed_item-variable eval.txt /*completed_item-variable*
completion-functions usr_41.txt /*completion-functions*
complex-change change.txt /*complex-change*
complex-repeat repeat.txt /*complex-repeat*
compress pi_gzip.txt /*compress*
conceal syntax.txt /*conceal*
confirm() eval.txt /*confirm()*
connection-refused message.txt /*connection-refused*
console-menus gui.txt /*console-menus*
control intro.txt /*control*
conversion-server mbyte.txt /*conversion-server*
convert-to-HTML syntax.txt /*convert-to-HTML*
convert-to-XHTML syntax.txt /*convert-to-XHTML*
convert-to-XML syntax.txt /*convert-to-XML*
copy() eval.txt /*copy()*
copy-diffs diff.txt /*copy-diffs*
copy-move change.txt /*copy-move*
copying uganda.txt /*copying*
copyright uganda.txt /*copyright*
cos() eval.txt /*cos()*
cosh() eval.txt /*cosh()*
count intro.txt /*count*
count() eval.txt /*count()*
count-bytes tips.txt /*count-bytes*
count-items tips.txt /*count-items*
count-variable eval.txt /*count-variable*
count1-variable eval.txt /*count1-variable*

cp-default version5.txt /*cp-default*
cpo options.txt /*cpo*
cpo-! options.txt /*cpo-!*
cpo-# options.txt /*cpo-#*
cpo-$ options.txt /*cpo-$*
cpo-% options.txt /*cpo-%*
cpo-& options.txt /*cpo-&*
cpo-+ options.txt /*cpo-+*
cpo-- options.txt /*cpo--*
cpo-. options.txt /*cpo-.*
cpo-/ options.txt /*cpo-\/*
cpo-; options.txt /*cpo-;*
cpo-< options.txt /*cpo-<*
cpo-> options.txt /*cpo->*
cpo-A options.txt /*cpo-A*
cpo-B options.txt /*cpo-B*
cpo-C options.txt /*cpo-C*
cpo-D options.txt /*cpo-D*
cpo-E options.txt /*cpo-E*
cpo-F options.txt /*cpo-F*
cpo-H options.txt /*cpo-H*
cpo-I options.txt /*cpo-I*
cpo-J options.txt /*cpo-J*
cpo-K options.txt /*cpo-K*
cpo-L options.txt /*cpo-L*
cpo-M options.txt /*cpo-M*
cpo-O options.txt /*cpo-O*
cpo-P options.txt /*cpo-P*
cpo-R options.txt /*cpo-R*
cpo-S options.txt /*cpo-S*
cpo-W options.txt /*cpo-W*
cpo-X options.txt /*cpo-X*
cpo-Z options.txt /*cpo-Z*
cpo-\ options.txt /*cpo-*
cpo-a options.txt /*cpo-a*
cpo-b options.txt /*cpo-b*
cpo-bar options.txt /*cpo-bar*
cpo-c options.txt /*cpo-c*
cpo-d options.txt /*cpo-d*
cpo-e options.txt /*cpo-e*
cpo-f options.txt /*cpo-f*
cpo-g options.txt /*cpo-g*
cpo-i options.txt /*cpo-i*
cpo-j options.txt /*cpo-j*
cpo-k options.txt /*cpo-k*
cpo-l options.txt /*cpo-l*
cpo-m options.txt /*cpo-m*
cpo-n options.txt /*cpo-n*
cpo-o options.txt /*cpo-o*
cpo-p options.txt /*cpo-p*
cpo-q options.txt /*cpo-q*
cpo-r options.txt /*cpo-r*
cpo-s options.txt /*cpo-s*
cpo-star options.txt /*cpo-star*

cpo-t options.txt /*cpo-t*
cpo-u options.txt /*cpo-u*
cpo-v options.txt /*cpo-v*
cpo-w options.txt /*cpo-w*
cpo-x options.txt /*cpo-x*
cpo-y options.txt /*cpo-y*
cpo-{ options.txt /*cpo-{*
cpp.vim syntax.txt /*cpp.vim*
crash-recovery recover.txt /*crash-recovery*
creating-menus gui.txt /*creating-menus*
credits intro.txt /*credits*
crontab options.txt /*crontab*
cs-find if_cscop.txt /*cs-find*
cs7-problem term.txt /*cs7-problem*
cscope if_cscop.txt /*cscope*
cscope-commands if_cscop.txt /*cscope-commands*
cscope-find if_cscop.txt /*cscope-find*
cscope-howtouse if_cscop.txt /*cscope-howtouse*
cscope-info if_cscop.txt /*cscope-info*
cscope-intro if_cscop.txt /*cscope-intro*
cscope-limitations if_cscop.txt /*cscope-limitations*
cscope-options if_cscop.txt /*cscope-options*
cscope-suggestions if_cscop.txt /*cscope-suggestions*
cscope-win32 if_cscop.txt /*cscope-win32*
cscope_connection() eval.txt /*cscope_connection()*
cscopepathcomp if_cscop.txt /*cscopepathcomp*
cscopeprg if_cscop.txt /*cscopeprg*
cscopequickfix if_cscop.txt /*cscopequickfix*
cscoperelative if_cscop.txt /*cscoperelative*
cscopetag if_cscop.txt /*cscopetag*
cscopetagorder if_cscop.txt /*cscopetagorder*
cscopeverbose if_cscop.txt /*cscopeverbose*
csh.vim syntax.txt /*csh.vim*
cspc if_cscop.txt /*cspc*
csprg if_cscop.txt /*csprg*
csqf if_cscop.txt /*csqf*
csre if_cscop.txt /*csre*
cst if_cscop.txt /*cst*
csto if_cscop.txt /*csto*
csverb if_cscop.txt /*csverb*
ctags tagsrch.txt /*ctags*
ctags-gone version6.txt /*ctags-gone*
cterm-colors syntax.txt /*cterm-colors*
ctrl intro.txt /*ctrl*
ctype-variable eval.txt /*ctype-variable*
curly-braces-function-names eval.txt /*curly-braces-function-names*
curly-braces-names eval.txt /*curly-braces-names*
curpos-visual version6.txt /*curpos-visual*
current-directory editing.txt /*current-directory*
current-file editing.txt /*current-file*
current_compiler quickfix.txt /*current_compiler*
cursor() eval.txt /*cursor()*
cursor-blinking options.txt /*cursor-blinking*
cursor-down intro.txt /*cursor-down*

cursor-functions	usr_41.txt	/*cursor-functions*
cursor-left	intro.txt	/*cursor-left*
cursor-motions	motion.txt	/*cursor-motions*
cursor-position	pattern.txt	/*cursor-position*
cursor-right	intro.txt	/*cursor-right*
cursor-up	intro.txt	/*cursor-up*
cursor_down	intro.txt	/*cursor_down*
cursor_left	intro.txt	/*cursor_left*
cursor_right	intro.txt	/*cursor_right*
cursor_up	intro.txt	/*cursor_up*
cw	change.txt	/*cw*
cweb.vim	syntax.txt	/*cweb.vim*
cynlib.vim	syntax.txt	/*cynlib.vim*
d	change.txt	/*d*
daB	motion.txt	/*daB*
daW	motion.txt	/*daW*
dab	motion.txt	/*dab*
dap	motion.txt	/*dap*
dart.vim	syntax.txt	/*dart.vim*
das	motion.txt	/*das*
date-functions	usr_41.txt	/*date-functions*
dav	pi_netrw.txt	/*dav*
davs	pi_netrw.txt	/*davs*
daw	motion.txt	/*daw*
dd	change.txt	/*dd*
debug-gcc	debug.txt	/*debug-gcc*
debug-highlight	debugger.txt	/*debug-highlight*
debug-leaks	debug.txt	/*debug-leaks*
debug-minidump	debug.txt	/*debug-minidump*
debug-mode	repeat.txt	/*debug-mode*
debug-scripts	repeat.txt	/*debug-scripts*
debug-signs	debugger.txt	/*debug-signs*
debug-vim	debug.txt	/*debug-vim*
debug-vs2005	debug.txt	/*debug-vs2005*
debug-win32	debug.txt	/*debug-win32*
debug-windbg	debug.txt	/*debug-windbg*
debug.txt	debug.txt	/*debug.txt*
debugbreak()	eval.txt	/*debugbreak()*
debugger-compilation	debugger.txt	/*debugger-compilation*
debugger-features	debugger.txt	/*debugger-features*
debugger-integration	debugger.txt	/*debugger-integration*
debugger-support	debugger.txt	/*debugger-support*
debugger.txt	debugger.txt	/*debugger.txt*
dec-mouse	options.txt	/*dec-mouse*
decada_members	ft_ada.txt	/*decada_members*
deepcopy()	eval.txt	/*deepcopy()*
defaults.vim	starting.txt	/*defaults.vim*
defaults.vim-explained	usr_05.txt	/*defaults.vim-explained*
definition-search	tagsrch.txt	/*definition-search*
definitions	intro.txt	/*definitions*
delete()	eval.txt	/*delete()*
delete-insert	change.txt	/*delete-insert*
delete-menus	gui.txt	/*delete-menus*
deletebufline()	eval.txt	/*deletebufline()*

deleting change.txt /*deleting*
demoserver.py channel.txt /*demoserver.py*
design-assumptions develop.txt /*design-assumptions*
design-compatible develop.txt /*design-compatible*
design-decisions develop.txt /*design-decisions*
design-documented develop.txt /*design-documented*
design-flexible develop.txt /*design-flexible*
design-goals develop.txt /*design-goals*
design-improved develop.txt /*design-improved*
design-maintain develop.txt /*design-maintain*
design-multi-platform develop.txt /*design-multi-platform*
design-not develop.txt /*design-not*
design-speed-size develop.txt /*design-speed-size*
desktop.vim syntax.txt /*desktop.vim*
develop-spell develop.txt /*develop-spell*
develop-spell-suggestions develop.txt /*develop-spell-suggestions*
develop.txt develop.txt /*develop.txt*
development develop.txt /*development*
dgn motion.txt /*dgn*
dh change.txt /*dh*
diB motion.txt /*diB*
diW motion.txt /*diW*
dialog gui_w32.txt /*dialog*
dialogs-added version5.txt /*dialogs-added*
dib motion.txt /*dib*
dict eval.txt /*dict*
dict-functions usr_41.txt /*dict-functions*
dict-identity eval.txt /*dict-identity*
dict-modification eval.txt /*dict-modification*
did_filetype() eval.txt /*did_filetype()*
diff diff.txt /*diff*
diff-diffexpr diff.txt /*diff-diffexpr*
diff-mode diff.txt /*diff-mode*
diff-options diff.txt /*diff-options*
diff-original-file diff.txt /*diff-original-file*
diff-patchexpr diff.txt /*diff-patchexpr*
diff-slow diff.txt /*diff-slow*
diff.txt diff.txt /*diff.txt*
diff.vim syntax.txt /*diff.vim*
diff_filler() eval.txt /*diff_filler()*
diff_hlID() eval.txt /*diff_hlID()*
diff_translations diff.txt /*diff_translations*
digraph digraph.txt /*digraph*
digraph-arg change.txt /*digraph-arg*
digraph-encoding digraph.txt /*digraph-encoding*
digraph-table digraph.txt /*digraph-table*
digraph-table-mbyte digraph.txt /*digraph-table-mbyte*
digraph.txt digraph.txt /*digraph.txt*
digraphs digraph.txt /*digraphs*
digraphs-changed version6.txt /*digraphs-changed*
digraphs-default digraph.txt /*digraphs-default*
digraphs-define digraph.txt /*digraphs-define*
digraphs-use digraph.txt /*digraphs-use*
dip motion.txt /*dip*

dircolors.vim syntax.txt /*dircolors.vim*
dis motion.txt /*dis*
disable-menus gui.txt /*disable-menus*
discard editing.txt /*discard*
distribute-script usr_41.txt /*distribute-script*
distribution intro.txt /*distribution*
diw motion.txt /*diw*
dl change.txt /*dl*
do diff.txt /*do*
doc-file-list help.txt /*doc-file-list*
docbk.vim syntax.txt /*docbk.vim*
docbksgml.vim syntax.txt /*docbksgml.vim*
docbkxml.vim syntax.txt /*docbkxml.vim*
docbook syntax.txt /*docbook*
documentation-6 version6.txt /*documentation-6*
donate uganda.txt /*donate*
dos os_dos.txt /*dos*
dos-:cd os_dos.txt /*dos-:cd*
dos-CTRL-Break os_dos.txt /*dos-CTRL-Break*
dos-backslash os_dos.txt /*dos-backslash*
dos-colors os_dos.txt /*dos-colors*
dos-file-formats os_dos.txt /*dos-file-formats*
dos-locations os_dos.txt /*dos-locations*
dos-shell os_dos.txt /*dos-shell*
dos-standard-mappings os_dos.txt /*dos-standard-mappings*
dos-temp-files os_dos.txt /*dos-temp-files*
dosbatch.vim syntax.txt /*dosbatch.vim*
double-click term.txt /*double-click*
download intro.txt /*download*
doxygen-syntax syntax.txt /*doxygen-syntax*
doxygen.vim syntax.txt /*doxygen.vim*
dp diff.txt /*dp*
drag-n-drop gui.txt /*drag-n-drop*
drag-n-drop-win32 gui_w32.txt /*drag-n-drop-win32*
drag-status-line term.txt /*drag-status-line*
dtd.vim syntax.txt /*dtd.vim*
dtd2vim insert.txt /*dtd2vim*
dying-variable eval.txt /*dying-variable*
e motion.txt /*e*
easy starting.txt /*easy*
echospace-variable eval.txt /*echospace-variable*
edit-a-file editing.txt /*edit-a-file*
edit-binary editing.txt /*edit-binary*
edit-dialogs editing.txt /*edit-dialogs*
edit-files editing.txt /*edit-files*
edit-intro editing.txt /*edit-intro*
edit-no-break usr_25.txt /*edit-no-break*
edit-paragraph-join usr_25.txt /*edit-paragraph-join*
editing.txt editing.txt /*editing.txt*
efm-%> quickfix.txt /*efm-%>*
efm-entries quickfix.txt /*efm-entries*
efm-ignore quickfix.txt /*efm-ignore*
eiffel.vim syntax.txt /*eiffel.vim*
emacs-keys tips.txt /*emacs-keys*

emacs-tags tagsrch.txt /*emacs-tags*
emacs_tags tagsrch.txt /*emacs_tags*
empty() eval.txt /*empty()*
encoding-names mbyte.txt /*encoding-names*
encoding-table mbyte.txt /*encoding-table*
encoding-values mbyte.txt /*encoding-values*
encryption editing.txt /*encryption*
end intro.txt /*end*
end-of-file pattern.txt /*end-of-file*
enlightened-terminal syntax.txt /*enlightened-terminal*
environ() eval.txt /*environ()*
erlang.vim syntax.txt /*erlang.vim*
err_buf channel.txt /*err_buf*
err_cb channel.txt /*err_cb*
err_mode channel.txt /*err_mode*
err_modifiable channel.txt /*err_modifiable*
err_msg channel.txt /*err_msg*
err_name channel.txt /*err_name*
err_timeout channel.txt /*err_timeout*
errmsg-variable eval.txt /*errmsg-variable*
error-file-format quickfix.txt /*error-file-format*
error-messages message.txt /*error-messages*
errorformat quickfix.txt /*errorformat*
errorformat-Jikes quickfix.txt /*errorformat-Jikes*
errorformat-LaTeX quickfix.txt /*errorformat-LaTeX*
errorformat-Perl quickfix.txt /*errorformat-Perl*
errorformat-ant quickfix.txt /*errorformat-ant*
errorformat-changed version4.txt /*errorformat-changed*
errorformat-jade quickfix.txt /*errorformat-jade*
errorformat-javac quickfix.txt /*errorformat-javac*
errorformat-multi-line quickfix.txt /*errorformat-multi-line*
errorformat-separate-filename quickfix.txt /*errorformat-separate-filename*
errorformats quickfix.txt /*errorformats*
errors message.txt /*errors*
errors-variable eval.txt /*errors-variable*
escape intro.txt /*escape*
escape() eval.txt /*escape()*
escape-bar version4.txt /*escape-bar*
euphoria3.vim syntax.txt /*euphoria3.vim*
euphoria4.vim syntax.txt /*euphoria4.vim*
eval eval.txt /*eval*
eval() eval.txt /*eval()*
eval-examples eval.txt /*eval-examples*
eval-sandbox eval.txt /*eval-sandbox*
eval.txt eval.txt /*eval.txt*
event-variable eval.txt /*event-variable*
eventhandler() eval.txt /*eventhandler()*
eview starting.txt /*eview*
evim starting.txt /*evim*
evim-keys starting.txt /*evim-keys*
evim.vim starting.txt /*evim.vim*
ex starting.txt /*ex*
ex-cmd-index index.txt /*ex-cmd-index*
ex-edit-index index.txt /*ex-edit-index*

ex-flags cmdline.txt /*ex-flags*
ex: options.txt /*ex:*
except-autocmd eval.txt /*except-autocmd*
except-autocmd-Cmd eval.txt /*except-autocmd-Cmd*
except-autocmd-Post eval.txt /*except-autocmd-Post*
except-autocmd-Pre eval.txt /*except-autocmd-Pre*
except-autocmd-ill eval.txt /*except-autocmd-ill*
except-compat eval.txt /*except-compat*
except-examine eval.txt /*except-examine*
except-from-finally eval.txt /*except-from-finally*
except-hier-param eval.txt /*except-hier-param*
except-several-errors eval.txt /*except-several-errors*
except-single-line eval.txt /*except-single-line*
except-syntax-err eval.txt /*except-syntax-err*
except-syntax-error eval.txt /*except-syntax-error*
exception-handling eval.txt /*exception-handling*
exception-variable eval.txt /*exception-variable*
exclusive motion.txt /*exclusive*
exclusive-linewise motion.txt /*exclusive-linewise*
executable() eval.txt /*executable()*
execute() eval.txt /*execute()*
execute-menus gui.txt /*execute-menus*
exepath() eval.txt /*exepath()*
exim starting.txt /*exim*
exists() eval.txt /*exists()*
exiting starting.txt /*exiting*
exp() eval.txt /*exp()*
expand() eval.txt /*expand()*
expand-env options.txt /*expand-env*
expand-environment-var options.txt /*expand-environment-var*
expandcmd() eval.txt /*expandcmd()*
expr eval.txt /*expr*
expr-! eval.txt /*expr-!*
expr-!= eval.txt /*expr-!=*
expr-!=# eval.txt /*expr-!=#*
expr-!=? eval.txt /*expr-!=?*
expr-!~ eval.txt /*expr-!~*
expr-!~# eval.txt /*expr-!~#*
expr-!~? eval.txt /*expr-!~?*
expr-% eval.txt /*expr-%*
expr-&& eval.txt /*expr-&&*
expr-' eval.txt /*expr-'*
expr-+ eval.txt /*expr-+*
expr-- eval.txt /*expr--*
expr-. eval.txt /*expr-.*
expr-.. eval.txt /*expr-..*
expr-/ eval.txt /*expr-\/*
expr-< eval.txt /*expr-<*
expr-<# eval.txt /*expr-<#*
expr-<= eval.txt /*expr-<=*
expr-<=# eval.txt /*expr-<=#*
expr-<=? eval.txt /*expr-<=?*
expr-<? eval.txt /*expr-<?*
expr-== eval.txt /*expr-==*

expr-==# eval.txt /*expr-==#*
expr-==? eval.txt /*expr-==?*
expr-=~ eval.txt /*expr-=~*
expr-=~# eval.txt /*expr-=~#*
expr-=~? eval.txt /*expr-=~?*
expr-> eval.txt /*expr->*
expr-># eval.txt /*expr->#*
expr->= eval.txt /*expr->=*
expr->=# eval.txt /*expr->=#*
expr->=? eval.txt /*expr->=?*
expr->? eval.txt /*expr->?*
expr-[:] eval.txt /*expr-[:]*
expr-[] eval.txt /*expr-[]*
expr-barbar eval.txt /*expr-barbar*
expr-entry eval.txt /*expr-entry*
expr-env eval.txt /*expr-env*
expr-env-expand eval.txt /*expr-env-expand*
expr-function eval.txt /*expr-function*
expr-is eval.txt /*expr-is*
expr-is# eval.txt /*expr-is#*
expr-is? eval.txt /*expr-is?*
expr-isnot eval.txt /*expr-isnot*
expr-isnot# eval.txt /*expr-isnot#*
expr-isnot? eval.txt /*expr-isnot?*
expr-lambda eval.txt /*expr-lambda*
expr-nesting eval.txt /*expr-nesting*
expr-number eval.txt /*expr-number*
expr-option eval.txt /*expr-option*
expr-quote eval.txt /*expr-quote*
expr-register eval.txt /*expr-register*
expr-star eval.txt /*expr-star*
expr-string eval.txt /*expr-string*
expr-unary-+ eval.txt /*expr-unary-+*
expr-unary-- eval.txt /*expr-unary--*
expr-variable eval.txt /*expr-variable*
expr1 eval.txt /*expr1*
expr2 eval.txt /*expr2*
expr3 eval.txt /*expr3*
expr4 eval.txt /*expr4*
expr5 eval.txt /*expr5*
expr6 eval.txt /*expr6*
expr7 eval.txt /*expr7*
expr8 eval.txt /*expr8*
expr9 eval.txt /*expr9*
expression eval.txt /*expression*
expression-commands eval.txt /*expression-commands*
expression-syntax eval.txt /*expression-syntax*
exrc starting.txt /*exrc*
extend() eval.txt /*extend()*
extension-removal cmdline.txt /*extension-removal*
extensions-improvements todo.txt /*extensions-improvements*
f motion.txt /*f*
false-variable eval.txt /*false-variable*
faq intro.txt /*faq*

farsi farsi.txt /*farsi*
farsi.txt farsi.txt /*farsi.txt*
fasm.vim syntax.txt /*fasm.vim*
fcs_choice-variable eval.txt /*fcs_choice-variable*
fcs_reason-variable eval.txt /*fcs_reason-variable*
feature-list eval.txt /*feature-list*
feedkeys() eval.txt /*feedkeys()*
fetch pi_netrw.txt /*fetch*
file-browser-5.2 version5.txt /*file-browser-5.2*
file-formats editing.txt /*file-formats*
file-functions usr_41.txt /*file-functions*
file-pattern autocmd.txt /*file-pattern*
file-read insert.txt /*file-read*
file-searching editing.txt /*file-searching*
file-type filetype.txt /*file-type*
file-types filetype.txt /*file-types*
file_readable() eval.txt /*file_readable()*
fileencoding-changed version6.txt /*fileencoding-changed*
filename-backslash cmdline.txt /*filename-backslash*
filename-modifiers cmdline.txt /*filename-modifiers*
filereadable() eval.txt /*filereadable()*
filetype filetype.txt /*filetype*
filetype-detect filetype.txt /*filetype-detect*
filetype-ignore filetype.txt /*filetype-ignore*
filetype-overrule filetype.txt /*filetype-overrule*
filetype-plugin usr_43.txt /*filetype-plugin*
filetype-plugins filetype.txt /*filetype-plugins*
filetype.txt filetype.txt /*filetype.txt*
filetypedetect-changed version6.txt /*filetypedetect-changed*
filetypes filetype.txt /*filetypes*
filewritable() eval.txt /*filewritable()*
filter change.txt /*filter*
filter() eval.txt /*filter()*
find-manpage usr_12.txt /*find-manpage*
find-replace usr_10.txt /*find-replace*
finddir() eval.txt /*finddir()*
findfile() eval.txt /*findfile()*
fixed-5.1 version5.txt /*fixed-5.1*
fixed-5.2 version5.txt /*fixed-5.2*
fixed-5.3 version5.txt /*fixed-5.3*
fixed-5.4 version5.txt /*fixed-5.4*
fixed-5.5 version5.txt /*fixed-5.5*
fixed-5.6 version5.txt /*fixed-5.6*
fixed-5.7 version5.txt /*fixed-5.7*
fixed-5.8 version5.txt /*fixed-5.8*
fixed-6.1 version6.txt /*fixed-6.1*
fixed-6.2 version6.txt /*fixed-6.2*
fixed-6.3 version6.txt /*fixed-6.3*
fixed-6.4 version6.txt /*fixed-6.4*
fixed-7.1 version7.txt /*fixed-7.1*
fixed-7.2 version7.txt /*fixed-7.2*
fixed-7.3 version7.txt /*fixed-7.3*
fixed-7.4 version7.txt /*fixed-7.4*
flexwiki.vim syntax.txt /*flexwiki.vim*

float-e eval.txt /*float-e*
float-functions usr_41.txt /*float-functions*
float-pi eval.txt /*float-pi*
float2nr() eval.txt /*float2nr()*
floating-point-format eval.txt /*floating-point-format*
floating-point-precision eval.txt /*floating-point-precision*
floor() eval.txt /*floor()*
fmod() eval.txt /*fmod()*
fname_diff-variable eval.txt /*fname_diff-variable*
fname_in-variable eval.txt /*fname_in-variable*
fname_new-variable eval.txt /*fname_new-variable*
fname_out-variable eval.txt /*fname_out-variable*
fnameescape() eval.txt /*fnameescape()*
fnamemodify() eval.txt /*fnamemodify()*
fo-table change.txt /*fo-table*
fold-behavior fold.txt /*fold-behavior*
fold-colors fold.txt /*fold-colors*
fold-commands fold.txt /*fold-commands*
fold-create-marker fold.txt /*fold-create-marker*
fold-delete-marker fold.txt /*fold-delete-marker*
fold-diff fold.txt /*fold-diff*
fold-expr fold.txt /*fold-expr*
fold-foldcolumn fold.txt /*fold-foldcolumn*
fold-foldlevel fold.txt /*fold-foldlevel*
fold-foldtext fold.txt /*fold-foldtext*
fold-indent fold.txt /*fold-indent*
fold-manual fold.txt /*fold-manual*
fold-marker fold.txt /*fold-marker*
fold-methods fold.txt /*fold-methods*
fold-options fold.txt /*fold-options*
fold-syntax fold.txt /*fold-syntax*
fold.txt fold.txt /*fold.txt*
foldclosed() eval.txt /*foldclosed()*
foldclosedend() eval.txt /*foldclosedend()*
folddashes-variable eval.txt /*folddashes-variable*
foldend-variable eval.txt /*foldend-variable*
folding fold.txt /*folding*
folding-functions usr_41.txt /*folding-functions*
foldlevel() eval.txt /*foldlevel()*
foldlevel-variable eval.txt /*foldlevel-variable*
folds fold.txt /*folds*
foldstart-variable eval.txt /*foldstart-variable*
foldtext() eval.txt /*foldtext()*
foldtextresult() eval.txt /*foldtextresult()*
font-sizes gui_x11.txt /*font-sizes*
fontset mbyte.txt /*fontset*
forced-motion motion.txt /*forced-motion*
foreground() eval.txt /*foreground()*
fork os_unix.txt /*fork*
form.vim syntax.txt /*form.vim*
format-bullet-list tips.txt /*format-bullet-list*
format-comments change.txt /*format-comments*
format-formatexpr change.txt /*format-formatexpr*
formatting change.txt /*formatting*

formfeed intro.txt /*formfeed*
fortran.vim syntax.txt /*fortran.vim*
friendship intro.txt /*friendship*
frombook usr_01.txt /*frombook*
ft-abel-syntax syntax.txt /*ft-abel-syntax*
ft-ada-commands ft_ada.txt /*ft-ada-commands*
ft-ada-constants ft_ada.txt /*ft-ada-constants*
ft-ada-functions ft_ada.txt /*ft-ada-functions*
ft-ada-indent ft_ada.txt /*ft-ada-indent*
ft-ada-omni ft_ada.txt /*ft-ada-omni*
ft-ada-options ft_ada.txt /*ft-ada-options*
ft-ada-plugin ft_ada.txt /*ft-ada-plugin*
ft-ada-syntax ft_ada.txt /*ft-ada-syntax*
ft-ada-variables ft_ada.txt /*ft-ada-variables*
ft-ant-syntax syntax.txt /*ft-ant-syntax*
ft-apache-syntax syntax.txt /*ft-apache-syntax*
ft-asm-syntax syntax.txt /*ft-asm-syntax*
ft-asm68k-syntax syntax.txt /*ft-asm68k-syntax*
ft-asmh8300-syntax syntax.txt /*ft-asmh8300-syntax*
ft-aspperl-syntax syntax.txt /*ft-aspperl-syntax*
ft-aspvbs-syntax syntax.txt /*ft-aspvbs-syntax*
ft-bash-syntax syntax.txt /*ft-bash-syntax*
ft-basic-syntax syntax.txt /*ft-basic-syntax*
ft-c-omni insert.txt /*ft-c-omni*
ft-c-syntax syntax.txt /*ft-c-syntax*
ft-ch-syntax syntax.txt /*ft-ch-syntax*
ft-changelog-plugin filetype.txt /*ft-changelog-plugin*
ft-changelog-syntax syntax.txt /*ft-changelog-syntax*
ft-chill-syntax syntax.txt /*ft-chill-syntax*
ft-clojure-indent indent.txt /*ft-clojure-indent*
ft-clojure-syntax syntax.txt /*ft-clojure-syntax*
ft-cobol-syntax syntax.txt /*ft-cobol-syntax*
ft-coldfusion-syntax syntax.txt /*ft-coldfusion-syntax*
ft-cpp-syntax syntax.txt /*ft-cpp-syntax*
ft-csh-syntax syntax.txt /*ft-csh-syntax*
ft-css-omni insert.txt /*ft-css-omni*
ft-cweb-syntax syntax.txt /*ft-cweb-syntax*
ft-cynlib-syntax syntax.txt /*ft-cynlib-syntax*
ft-dart-syntax syntax.txt /*ft-dart-syntax*
ft-dash-syntax syntax.txt /*ft-dash-syntax*
ft-desktop-syntax syntax.txt /*ft-desktop-syntax*
ft-dircolors-syntax syntax.txt /*ft-dircolors-syntax*
ft-docbk-syntax syntax.txt /*ft-docbk-syntax*
ft-docbksgml-syntax syntax.txt /*ft-docbksgml-syntax*
ft-docbkxml-syntax syntax.txt /*ft-docbkxml-syntax*
ft-dosbatch-syntax syntax.txt /*ft-dosbatch-syntax*
ft-dtd-syntax syntax.txt /*ft-dtd-syntax*
ft-eiffel-syntax syntax.txt /*ft-eiffel-syntax*
ft-erlang-syntax syntax.txt /*ft-erlang-syntax*
ft-euphoria-syntax syntax.txt /*ft-euphoria-syntax*
ft-flexwiki-syntax syntax.txt /*ft-flexwiki-syntax*
ft-form-syntax syntax.txt /*ft-form-syntax*
ft-fortran-indent indent.txt /*ft-fortran-indent*
ft-fortran-plugin filetype.txt /*ft-fortran-plugin*

ft-fortran-syntax	syntax.txt	/*ft-fortran-syntax*
ft-fvwm-syntax	syntax.txt	/*ft-fvwm-syntax*
ft-gitcommit-plugin	filetype.txt	/*ft-gitcommit-plugin*
ft-groff-syntax	syntax.txt	/*ft-groff-syntax*
ft-gsp-syntax	syntax.txt	/*ft-gsp-syntax*
ft-haskell-syntax	syntax.txt	/*ft-haskell-syntax*
ft-html-indent	indent.txt	/*ft-html-indent*
ft-html-omni	insert.txt	/*ft-html-omni*
ft-html-syntax	syntax.txt	/*ft-html-syntax*
ft-htmlos-syntax	syntax.txt	/*ft-htmlos-syntax*
ft-ia64-syntax	syntax.txt	/*ft-ia64-syntax*
ft-inform-syntax	syntax.txt	/*ft-inform-syntax*
ft-java-syntax	syntax.txt	/*ft-java-syntax*
ft-javascript-omni	insert.txt	/*ft-javascript-omni*
ft-json-syntax	syntax.txt	/*ft-json-syntax*
ft-ksh-syntax	syntax.txt	/*ft-ksh-syntax*
ft-lace-syntax	syntax.txt	/*ft-lace-syntax*
ft-lex-syntax	syntax.txt	/*ft-lex-syntax*
ft-lifelines-syntax	syntax.txt	/*ft-lifelines-syntax*
ft-lisp-syntax	syntax.txt	/*ft-lisp-syntax*
ft-lite-syntax	syntax.txt	/*ft-lite-syntax*
ft-lpc-syntax	syntax.txt	/*ft-lpc-syntax*
ft-lua-syntax	syntax.txt	/*ft-lua-syntax*
ft-mail-plugin	filetype.txt	/*ft-mail-plugin*
ft-mail.vim	syntax.txt	/*ft-mail.vim*
ft-make-syntax	syntax.txt	/*ft-make-syntax*
ft-man-plugin	filetype.txt	/*ft-man-plugin*
ft-maple-syntax	syntax.txt	/*ft-maple-syntax*
ft-masm-syntax	syntax.txt	/*ft-masm-syntax*
ft-mathematica-syntax	syntax.txt	/*ft-mathematica-syntax*
ft-matlab-indent	indent.txt	/*ft-matlab-indent*
ft-mma-syntax	syntax.txt	/*ft-mma-syntax*
ft-moo-syntax	syntax.txt	/*ft-moo-syntax*
ft-msql-syntax	syntax.txt	/*ft-msql-syntax*
ft-n1ql-syntax	syntax.txt	/*ft-n1ql-syntax*
ft-nasm-syntax	syntax.txt	/*ft-nasm-syntax*
ft-ncf-syntax	syntax.txt	/*ft-ncf-syntax*
ft-nroff-syntax	syntax.txt	/*ft-nroff-syntax*
ft-ocaml-syntax	syntax.txt	/*ft-ocaml-syntax*
ft-papp-syntax	syntax.txt	/*ft-papp-syntax*
ft-pascal-syntax	syntax.txt	/*ft-pascal-syntax*
ft-pdf-plugin	filetype.txt	/*ft-pdf-plugin*
ft-perl-syntax	syntax.txt	/*ft-perl-syntax*
ft-php-indent	indent.txt	/*ft-php-indent*
ft-php-omni	insert.txt	/*ft-php-omni*
ft-php-syntax	syntax.txt	/*ft-php-syntax*
ft-php3-syntax	syntax.txt	/*ft-php3-syntax*
ft-phtml-syntax	syntax.txt	/*ft-phtml-syntax*
ft-plaintex-syntax	syntax.txt	/*ft-plaintex-syntax*
ft-posix-synax	syntax.txt	/*ft-posix-synax*
ft-postscr-syntax	syntax.txt	/*ft-postscr-syntax*
ft-ppwiz-syntax	syntax.txt	/*ft-ppwiz-syntax*
ft-printcap-syntax	syntax.txt	/*ft-printcap-syntax*
ft-progress-syntax	syntax.txt	/*ft-progress-syntax*

ft-ptcap-syntax	syntax.txt	/*ft-ptcap-syntax*
ft-python-indent	indent.txt	/*ft-python-indent*
ft-python-plugin	filetype.txt	/*ft-python-plugin*
ft-python-syntax	syntax.txt	/*ft-python-syntax*
ft-qf-plugin	filetype.txt	/*ft-qf-plugin*
ft-quake-syntax	syntax.txt	/*ft-quake-syntax*
ft-r-indent	indent.txt	/*ft-r-indent*
ft-r-syntax	syntax.txt	/*ft-r-syntax*
ft-readline-syntax	syntax.txt	/*ft-readline-syntax*
ft-rego-syntax	syntax.txt	/*ft-rego-syntax*
ft-rexx-syntax	syntax.txt	/*ft-rexx-syntax*
ft-rmd-plugin	filetype.txt	/*ft-rmd-plugin*
ft-rmd-syntax	syntax.txt	/*ft-rmd-syntax*
ft-rrst-plugin	filetype.txt	/*ft-rrst-plugin*
ft-rrst-syntax	syntax.txt	/*ft-rrst-syntax*
ft-rst-plugin	filetype.txt	/*ft-rst-plugin*
ft-rst-syntax	syntax.txt	/*ft-rst-syntax*
ft-ruby-omni	insert.txt	/*ft-ruby-omni*
ft-ruby-syntax	syntax.txt	/*ft-ruby-syntax*
ft-rust	filetype.txt	/*ft-rust*
ft-scheme-syntax	syntax.txt	/*ft-scheme-syntax*
ft-sdl-syntax	syntax.txt	/*ft-sdl-syntax*
ft-sed-syntax	syntax.txt	/*ft-sed-syntax*
ft-sgml-syntax	syntax.txt	/*ft-sgml-syntax*
ft-sh-indent	indent.txt	/*ft-sh-indent*
ft-sh-syntax	syntax.txt	/*ft-sh-syntax*
ft-spec-plugin	filetype.txt	/*ft-spec-plugin*
ft-spup-syntax	syntax.txt	/*ft-spup-syntax*
ft-sql	filetype.txt	/*ft-sql*
ft-sql-omni	insert.txt	/*ft-sql-omni*
ft-sql-syntax	syntax.txt	/*ft-sql-syntax*
ft-sqlanywhere-syntax	syntax.txt	/*ft-sqlanywhere-syntax*
ft-sqlinformix-syntax	syntax.txt	/*ft-sqlinformix-syntax*
ft-syntax-omni	insert.txt	/*ft-syntax-omni*
ft-tcsh-syntax	syntax.txt	/*ft-tcsh-syntax*
ft-termcap-syntax	syntax.txt	/*ft-termcap-syntax*
ft-tex-plugin	filetype.txt	/*ft-tex-plugin*
ft-tex-syntax	syntax.txt	/*ft-tex-syntax*
ft-tf-syntax	syntax.txt	/*ft-tf-syntax*
ft-vb-syntax	syntax.txt	/*ft-vb-syntax*
ft-verilog-indent	indent.txt	/*ft-verilog-indent*
ft-vhdl-indent	indent.txt	/*ft-vhdl-indent*
ft-vim-indent	indent.txt	/*ft-vim-indent*
ft-vim-plugin	filetype.txt	/*ft-vim-plugin*
ft-vim-syntax	syntax.txt	/*ft-vim-syntax*
ft-xf86conf-syntax	syntax.txt	/*ft-xf86conf-syntax*
ft-xhtml-omni	insert.txt	/*ft-xhtml-omni*
ft-xml-omni	insert.txt	/*ft-xml-omni*
ft-xml-syntax	syntax.txt	/*ft-xml-syntax*
ft-xpm-syntax	syntax.txt	/*ft-xpm-syntax*
ft-yaml-syntax	syntax.txt	/*ft-yaml-syntax*
ft-zimbu-plugin	filetype.txt	/*ft-zimbu-plugin*
ft-zsh-syntax	syntax.txt	/*ft-zsh-syntax*
ft_ada.txt	ft_ada.txt	/*ft_ada.txt*

```
ft_rust.txt                ft_rust.txt                /*ft_rust.txt*
ft_sql.txt                 ft_sql.txt                 /*ft_sql.txt*
ftdetect                   filetype.txt                 /*ftdetect*
ftp                        pi_netrw.txt                   /*ftp*
ftplugin                   usr_41.txt                   /*ftplugin*
ftplugin-docs              filetype.txt               /*ftplugin-docs*
ftplugin-name              usr_05.txt                 /*ftplugin-name*
ftplugin-overrule          filetype.txt             /*ftplugin-overrule*
ftplugin-special           usr_41.txt               /*ftplugin-special*
ftplugins                  usr_05.txt                   /*ftplugins*
funcref()                  eval.txt                     /*funcref()*
function()                 eval.txt                     /*function()*
function-argument          eval.txt                 /*function-argument*
function-key               intro.txt                  /*function-key*
function-list              usr_41.txt                 /*function-list*
function-range-example     eval.txt             /*function-range-example*
function-search-undo       eval.txt              /*function-search-undo*
function_key               intro.txt                  /*function_key*
functions                  eval.txt                   /*functions*
fvwm.vim                   syntax.txt                   /*fvwm.vim*
fvwm2rc                    syntax.txt                   /*fvwm2rc*
fvwmrc                     syntax.txt                   /*fvwmrc*
g                          index.txt                      /*g*
g#                         pattern.txt                    /*g#*
g$                         motion.txt                     /*g$*
g&                         change.txt                     /*g&*
g'                         motion.txt                     /*g'*
g'a                        motion.txt                    /*g'a*
g+                         undo.txt                       /*g+*
g,                         motion.txt                     /*g,*
g-                         undo.txt                       /*g-*
g0                         motion.txt                     /*g0*
g8                         various.txt                    /*g8*
g:                         eval.txt                       /*g:*
g:NetrwTopLvlMenu          pi_netrw.txt             /*g:NetrwTopLvlMenu*
g:Netrw_UserMaps           pi_netrw.txt               /*g:Netrw_UserMaps*
g:Netrw_corehandler        pi_netrw.txt             /*g:Netrw_corehandler*
g:Netrw_funcref            pi_netrw.txt               /*g:Netrw_funcref*
g:actual_curbuf            options.txt                /*g:actual_curbuf*
g:actual_curwin            options.txt                /*g:actual_curwin*
g:ada#Comment              ft_ada.txt                 /*g:ada#Comment*
g:ada#Ctags_Kinds          ft_ada.txt               /*g:ada#Ctags_Kinds*
g:ada#DotWordRegex         ft_ada.txt               /*g:ada#DotWordRegex*
g:ada#Keywords             ft_ada.txt                 /*g:ada#Keywords*
g:ada#WordRegex            ft_ada.txt                 /*g:ada#WordRegex*
g:ada_abbrev               ft_ada.txt                  /*g:ada_abbrev*
g:ada_all_tab_usage        ft_ada.txt               /*g:ada_all_tab_usage*
g:ada_begin_preproc        ft_ada.txt               /*g:ada_begin_preproc*
g:ada_default_compiler     ft_ada.txt            /*g:ada_default_compiler*
g:ada_extended_completion  ft_ada.txt         /*g:ada_extended_completion*
g:ada_extended_tagging     ft_ada.txt            /*g:ada_extended_tagging*
g:ada_folding              ft_ada.txt                  /*g:ada_folding*
g:ada_gnat_extensions      ft_ada.txt            /*g:ada_gnat_extensions*
g:ada_line_errors          ft_ada.txt              /*g:ada_line_errors*
```

g:ada_no_tab_space_error ft_ada.txt /*g:ada_no_tab_space_error*
g:ada_no_trail_space_error ft_ada.txt /*g:ada_no_trail_space_error*
g:ada_omni_with_keywords ft_ada.txt /*g:ada_omni_with_keywords*
g:ada_rainbow_color ft_ada.txt /*g:ada_rainbow_color*
g:ada_space_errors ft_ada.txt /*g:ada_space_errors*
g:ada_standard_types ft_ada.txt /*g:ada_standard_types*
g:ada_with_gnat_project_files ft_ada.txt /*g:ada_with_gnat_project_files*
g:ada_withuse_ordinary ft_ada.txt /*g:ada_withuse_ordinary*
g:clojure_align_multiline_strings indent.txt
/*g:clojure_align_multiline_strings*
g:clojure_align_subforms indent.txt /*g:clojure_align_subforms*
g:clojure_fold syntax.txt /*g:clojure_fold*
g:clojure_fuzzy_indent indent.txt /*g:clojure_fuzzy_indent*
g:clojure_fuzzy_indent_blacklist indent.txt
/*g:clojure_fuzzy_indent_blacklist*
g:clojure_fuzzy_indent_patterns indent.txt /*g:clojure_fuzzy_indent_patterns*
g:clojure_maxlines indent.txt /*g:clojure_maxlines*
g:clojure_special_indent_words indent.txt /*g:clojure_special_indent_words*
g:clojure_syntax_keywords syntax.txt /*g:clojure_syntax_keywords*
g:colors_name options.txt /*g:colors_name*
g:decada ft_ada.txt /*g:decada*
g:decada.Error_Format ft_ada.txt /*g:decada.Error_Format*
g:decada.Make() ft_ada.txt /*g:decada.Make()*
g:decada.Make_Command ft_ada.txt /*g:decada.Make_Command*
g:decada.Unit_Name() ft_ada.txt /*g:decada.Unit_Name()*
g:filetype_csh syntax.txt /*g:filetype_csh*
g:filetype_r syntax.txt /*g:filetype_r*
g:ftplugin_rust_source_path ft_rust.txt /*g:ftplugin_rust_source_path*
g:gnat ft_ada.txt /*g:gnat*
g:gnat.Error_Format ft_ada.txt /*g:gnat.Error_Format*
g:gnat.Find() ft_ada.txt /*g:gnat.Find()*
g:gnat.Find_Program ft_ada.txt /*g:gnat.Find_Program*
g:gnat.Make() ft_ada.txt /*g:gnat.Make()*
g:gnat.Make_Command ft_ada.txt /*g:gnat.Make_Command*
g:gnat.Pretty() ft_ada.txt /*g:gnat.Pretty()*
g:gnat.Pretty_Program ft_ada.txt /*g:gnat.Pretty_Program*
g:gnat.Project_File ft_ada.txt /*g:gnat.Project_File*
g:gnat.Set_Project_File() ft_ada.txt /*g:gnat.Set_Project_File()*
g:gnat.Tags() ft_ada.txt /*g:gnat.Tags()*
g:gnat.Tags_Command ft_ada.txt /*g:gnat.Tags_Command*
g:html_charset_override syntax.txt /*g:html_charset_override*
g:html_diff_one_file syntax.txt /*g:html_diff_one_file*
g:html_dynamic_folds syntax.txt /*g:html_dynamic_folds*
g:html_encoding_override syntax.txt /*g:html_encoding_override*
g:html_end_line syntax.txt /*g:html_end_line*
g:html_expand_tabs syntax.txt /*g:html_expand_tabs*
g:html_font syntax.txt /*g:html_font*
g:html_hover_unfold syntax.txt /*g:html_hover_unfold*
g:html_id_expr syntax.txt /*g:html_id_expr*
g:html_ignore_conceal syntax.txt /*g:html_ignore_conceal*
g:html_ignore_folding syntax.txt /*g:html_ignore_folding*
g:html_line_ids syntax.txt /*g:html_line_ids*
g:html_no_foldcolumn syntax.txt /*g:html_no_foldcolumn*
g:html_no_invalid syntax.txt /*g:html_no_invalid*

g:html_no_pre syntax.txt /*g:html_no_pre*
g:html_no_progress syntax.txt /*g:html_no_progress*
g:html_number_lines syntax.txt /*g:html_number_lines*
g:html_pre_wrap syntax.txt /*g:html_pre_wrap*
g:html_prevent_copy syntax.txt /*g:html_prevent_copy*
g:html_start_line syntax.txt /*g:html_start_line*
g:html_use_css syntax.txt /*g:html_use_css*
g:html_use_encoding syntax.txt /*g:html_use_encoding*
g:html_use_input_for_pc syntax.txt /*g:html_use_input_for_pc*
g:html_use_xhtml syntax.txt /*g:html_use_xhtml*
g:html_whole_filler syntax.txt /*g:html_whole_filler*
g:netrw_altfile pi_netrw.txt /*g:netrw_altfile*
g:netrw_alto pi_netrw.txt /*g:netrw_alto*
g:netrw_altv pi_netrw.txt /*g:netrw_altv*
g:netrw_banner pi_netrw.txt /*g:netrw_banner*
g:netrw_bannerbackslash pi_netrw.txt /*g:netrw_bannerbackslash*
g:netrw_browse_split pi_netrw.txt /*g:netrw_browse_split*
g:netrw_browsex_viewer pi_netrw.txt /*g:netrw_browsex_viewer*
g:netrw_bufsettings pi_netrw.txt /*g:netrw_bufsettings*
g:netrw_chgperm pi_netrw.txt /*g:netrw_chgperm*
g:netrw_chgwin pi_netrw.txt /*g:netrw_chgwin*
g:netrw_compress pi_netrw.txt /*g:netrw_compress*
g:netrw_ctags pi_netrw.txt /*g:netrw_ctags*
g:netrw_cursor pi_netrw.txt /*g:netrw_cursor*
g:netrw_cygwin pi_netrw.txt /*g:netrw_cygwin*
g:netrw_dav_cmd pi_netrw.txt /*g:netrw_dav_cmd*
g:netrw_decompress pi_netrw.txt /*g:netrw_decompress*
g:netrw_dirhistmax pi_netrw.txt /*g:netrw_dirhistmax*
g:netrw_dynamic_maxfilenamelen pi_netrw.txt /*g:netrw_dynamic_maxfilenamelen*
g:netrw_errorlvl pi_netrw.txt /*g:netrw_errorlvl*
g:netrw_fastbrowse pi_netrw.txt /*g:netrw_fastbrowse*
g:netrw_fetch_cmd pi_netrw.txt /*g:netrw_fetch_cmd*
g:netrw_ffkeep pi_netrw.txt /*g:netrw_ffkeep*
g:netrw_file_cmd pi_netrw.txt /*g:netrw_file_cmd*
g:netrw_fname_escape pi_netrw.txt /*g:netrw_fname_escape*
g:netrw_ftp pi_netrw.txt /*g:netrw_ftp*
g:netrw_ftp_browse_reject pi_netrw.txt /*g:netrw_ftp_browse_reject*
g:netrw_ftp_cmd pi_netrw.txt /*g:netrw_ftp_cmd*
g:netrw_ftp_list_cmd pi_netrw.txt /*g:netrw_ftp_list_cmd*
g:netrw_ftp_options pi_netrw.txt /*g:netrw_ftp_options*
g:netrw_ftp_sizelist_cmd pi_netrw.txt /*g:netrw_ftp_sizelist_cmd*
g:netrw_ftp_timelist_cmd pi_netrw.txt /*g:netrw_ftp_timelist_cmd*
g:netrw_ftpextracmd pi_netrw.txt /*g:netrw_ftpextracmd*
g:netrw_ftpmode pi_netrw.txt /*g:netrw_ftpmode*
g:netrw_glob_escape pi_netrw.txt /*g:netrw_glob_escape*
g:netrw_gx pi_netrw.txt /*g:netrw_gx*
g:netrw_hide pi_netrw.txt /*g:netrw_hide*
g:netrw_home pi_netrw.txt /*g:netrw_home*
g:netrw_http_cmd pi_netrw.txt /*g:netrw_http_cmd*
g:netrw_http_put_cmd pi_netrw.txt /*g:netrw_http_put_cmd*
g:netrw_http_xcmd pi_netrw.txt /*g:netrw_http_xcmd*
g:netrw_ignorenetrc pi_netrw.txt /*g:netrw_ignorenetrc*
g:netrw_keepdir pi_netrw.txt /*g:netrw_keepdir*
g:netrw_keepj pi_netrw.txt /*g:netrw_keepj*

g:netrw_list_cmd pi_netrw.txt /*g:netrw_list_cmd*
g:netrw_list_cmd_options pi_netrw.txt /*g:netrw_list_cmd_options*
g:netrw_list_hide pi_netrw.txt /*g:netrw_list_hide*
g:netrw_liststyle pi_netrw.txt /*g:netrw_liststyle*
g:netrw_localcopycmd pi_netrw.txt /*g:netrw_localcopycmd*
g:netrw_localcopycmdopt pi_netrw.txt /*g:netrw_localcopycmdopt*
g:netrw_localcopydircmd pi_netrw.txt /*g:netrw_localcopydircmd*
g:netrw_localcopydircmdopt pi_netrw.txt /*g:netrw_localcopydircmdopt*
g:netrw_localmkdir pi_netrw.txt /*g:netrw_localmkdir*
g:netrw_localmkdiropt pi_netrw.txt /*g:netrw_localmkdiropt*
g:netrw_localmovecmd pi_netrw.txt /*g:netrw_localmovecmd*
g:netrw_localmovecmdopt pi_netrw.txt /*g:netrw_localmovecmdopt*
g:netrw_localrmdir pi_netrw.txt /*g:netrw_localrmdir*
g:netrw_localrmdiropt pi_netrw.txt /*g:netrw_localrmdiropt*
g:netrw_maxfilenamelen pi_netrw.txt /*g:netrw_maxfilenamelen*
g:netrw_menu pi_netrw.txt /*g:netrw_menu*
g:netrw_mkdir_cmd pi_netrw.txt /*g:netrw_mkdir_cmd*
g:netrw_mousemaps pi_netrw.txt /*g:netrw_mousemaps*
g:netrw_nobeval pi_netrw.txt /*g:netrw_nobeval*
g:netrw_nogx pi_netrw.txt /*g:netrw_nogx*
g:netrw_preview pi_netrw.txt /*g:netrw_preview*
g:netrw_rcp_cmd pi_netrw.txt /*g:netrw_rcp_cmd*
g:netrw_remote_mkdir pi_netrw.txt /*g:netrw_remote_mkdir*
g:netrw_rename_cmd pi_netrw.txt /*g:netrw_rename_cmd*
g:netrw_retmap pi_netrw.txt /*g:netrw_retmap*
g:netrw_rm_cmd pi_netrw.txt /*g:netrw_rm_cmd*
g:netrw_rmdir_cmd pi_netrw.txt /*g:netrw_rmdir_cmd*
g:netrw_rmf_cmd pi_netrw.txt /*g:netrw_rmf_cmd*
g:netrw_rsync_cmd pi_netrw.txt /*g:netrw_rsync_cmd*
g:netrw_rsync_sep pi_netrw.txt /*g:netrw_rsync_sep*
g:netrw_scp_cmd pi_netrw.txt /*g:netrw_scp_cmd*
g:netrw_scpport pi_netrw.txt /*g:netrw_scpport*
g:netrw_sepchr pi_netrw.txt /*g:netrw_sepchr*
g:netrw_servername pi_netrw.txt /*g:netrw_servername*
g:netrw_sftp_cmd pi_netrw.txt /*g:netrw_sftp_cmd*
g:netrw_silent pi_netrw.txt /*g:netrw_silent*
g:netrw_sizestyle pi_netrw.txt /*g:netrw_sizestyle*
g:netrw_sort_by pi_netrw.txt /*g:netrw_sort_by*
g:netrw_sort_direction pi_netrw.txt /*g:netrw_sort_direction*
g:netrw_sort_options pi_netrw.txt /*g:netrw_sort_options*
g:netrw_sort_sequence pi_netrw.txt /*g:netrw_sort_sequence*
g:netrw_special_syntax pi_netrw.txt /*g:netrw_special_syntax*
g:netrw_ssh_browse_reject pi_netrw.txt /*g:netrw_ssh_browse_reject*
g:netrw_ssh_cmd pi_netrw.txt /*g:netrw_ssh_cmd*
g:netrw_sshport pi_netrw.txt /*g:netrw_sshport*
g:netrw_suppress_gx_mesg pi_netrw.txt /*g:netrw_suppress_gx_mesg*
g:netrw_timefmt pi_netrw.txt /*g:netrw_timefmt*
g:netrw_tmpfile_escape pi_netrw.txt /*g:netrw_tmpfile_escape*
g:netrw_uid pi_netrw.txt /*g:netrw_uid*
g:netrw_use_errorwindow pi_netrw.txt /*g:netrw_use_errorwindow*
g:netrw_use_noswf pi_netrw.txt /*g:netrw_use_noswf*
g:netrw_use_nt_rcp pi_netrw.txt /*g:netrw_use_nt_rcp*
g:netrw_usetab pi_netrw.txt /*g:netrw_usetab*
g:netrw_win95ftp pi_netrw.txt /*g:netrw_win95ftp*

g:netrw_winsize pi_netrw.txt /*g:netrw_winsize*
g:netrw_wiw pi_netrw.txt /*g:netrw_wiw*
g:netrw_xstrlen pi_netrw.txt /*g:netrw_xstrlen*
g:rust_bang_comment_leader ft_rust.txt /*g:rust_bang_comment_leader*
g:rust_conceal ft_rust.txt /*g:rust_conceal*
g:rust_conceal_mod_path ft_rust.txt /*g:rust_conceal_mod_path*
g:rust_conceal_pub ft_rust.txt /*g:rust_conceal_pub*
g:rust_fold ft_rust.txt /*g:rust_fold*
g:rust_playpen_url ft_rust.txt /*g:rust_playpen_url*
g:rust_recommended_style ft_rust.txt /*g:rust_recommended_style*
g:rust_shortener_url ft_rust.txt /*g:rust_shortener_url*
g:rustc_makeprg_no_percent ft_rust.txt /*g:rustc_makeprg_no_percent*
g:rustc_path ft_rust.txt /*g:rustc_path*
g:rustfmt_autosave ft_rust.txt /*g:rustfmt_autosave*
g:rustfmt_command ft_rust.txt /*g:rustfmt_command*
g:rustfmt_fail_silently ft_rust.txt /*g:rustfmt_fail_silently*
g:rustfmt_options ft_rust.txt /*g:rustfmt_options*
g:statusline_winid options.txt /*g:statusline_winid*
g:syntax_on syntax.txt /*g:syntax_on*
g:tar_browseoptions pi_tar.txt /*g:tar_browseoptions*
g:tar_cmd pi_tar.txt /*g:tar_cmd*
g:tar_copycmd pi_tar.txt /*g:tar_copycmd*
g:tar_extractcmd pi_tar.txt /*g:tar_extractcmd*
g:tar_nomax pi_tar.txt /*g:tar_nomax*
g:tar_readoptions pi_tar.txt /*g:tar_readoptions*
g:tar_secure pi_tar.txt /*g:tar_secure*
g:tar_writeoptions pi_tar.txt /*g:tar_writeoptions*
g:terminal_ansi_colors terminal.txt /*g:terminal_ansi_colors*
g:tex_comment_nospell syntax.txt /*g:tex_comment_nospell*
g:tex_conceal syntax.txt /*g:tex_conceal*
g:tex_fast syntax.txt /*g:tex_fast*
g:tex_flavor filetype.txt /*g:tex_flavor*
g:tex_fold_enabled syntax.txt /*g:tex_fold_enabled*
g:tex_isk syntax.txt /*g:tex_isk*
g:tex_no_error syntax.txt /*g:tex_no_error*
g:tex_nospell syntax.txt /*g:tex_nospell*
g:tex_stylish syntax.txt /*g:tex_stylish*
g:tex_subscripts syntax.txt /*g:tex_subscripts*
g:tex_superscripts syntax.txt /*g:tex_superscripts*
g:tex_verbspell syntax.txt /*g:tex_verbspell*
g:var eval.txt /*g:var*
g:vimball_home pi_vimball.txt /*g:vimball_home*
g:vimball_mkdir pi_vimball.txt /*g:vimball_mkdir*
g:vimsyn_embed syntax.txt /*g:vimsyn_embed*
g:vimsyn_folding syntax.txt /*g:vimsyn_folding*
g:vimsyn_maxlines syntax.txt /*g:vimsyn_maxlines*
g:vimsyn_minlines syntax.txt /*g:vimsyn_minlines*
g:vimsyn_noerror syntax.txt /*g:vimsyn_noerror*
g:yaml_schema syntax.txt /*g:yaml_schema*
g:zipPlugin_ext pi_zip.txt /*g:zipPlugin_ext*
g:zip_extractcmd pi_zip.txt /*g:zip_extractcmd*
g:zip_nomax pi_zip.txt /*g:zip_nomax*
g:zip_shq pi_zip.txt /*g:zip_shq*
g:zip_unzipcmd pi_zip.txt /*g:zip_unzipcmd*

g:zip_zipcmd	pi_zip.txt	/*g:zip_zipcmd*
g;	motion.txt	/*g;*
g<	message.txt	/*g<*
g<Down>	motion.txt	/*g<Down>*
g<End>	motion.txt	/*g<End>*
g<Home>	motion.txt	/*g<Home>*
g<LeftMouse>	tagsrch.txt	/*g<LeftMouse>*
g<RightMouse>	tagsrch.txt	/*g<RightMouse>*
g<Up>	motion.txt	/*g<Up>*
g?	change.txt	/*g?*
g??	change.txt	/*g??*
g?g?	change.txt	/*g?g?*
g@	map.txt	/*g@*
gD	pattern.txt	/*gD*
gE	motion.txt	/*gE*
gF	editing.txt	/*gF*
gH	visual.txt	/*gH*
gI	insert.txt	/*gI*
gJ	change.txt	/*gJ*
gM	motion.txt	/*gM*
gN	visual.txt	/*gN*
gP	change.txt	/*gP*
gQ	intro.txt	/*gQ*
gR	change.txt	/*gR*
gT	tabpage.txt	/*gT*
gU	change.txt	/*gU*
gUU	change.txt	/*gUU*
gUgU	change.txt	/*gUgU*
gV	visual.txt	/*gV*
g]	tagsrch.txt	/*g]*
g^	motion.txt	/*g^*
g_	motion.txt	/*g_*
g_CTRL-A	various.txt	/*g_CTRL-A*
g_CTRL-G	editing.txt	/*g_CTRL-G*
g_CTRL-H	visual.txt	/*g_CTRL-H*
g_CTRL-]	tagsrch.txt	/*g_CTRL-]*
g`	motion.txt	/*g`*
g`a	motion.txt	/*g`a*
ga	various.txt	/*ga*
garbagecollect()	eval.txt	/*garbagecollect()*
gd	pattern.txt	/*gd*
gdb	debug.txt	/*gdb*
gdb-version	terminal.txt	/*gdb-version*
ge	motion.txt	/*ge*
get()	eval.txt	/*get()*
get-ms-debuggers	debug.txt	/*get-ms-debuggers*
getbufinfo()	eval.txt	/*getbufinfo()*
getbufline()	eval.txt	/*getbufline()*
getbufvar()	eval.txt	/*getbufvar()*
getchangelist()	eval.txt	/*getchangelist()*
getchar()	eval.txt	/*getchar()*
getcharmod()	eval.txt	/*getcharmod()*
getcharsearch()	eval.txt	/*getcharsearch()*
getcmdline()	eval.txt	/*getcmdline()*

getcmdpos() eval.txt /*getcmdpos()*
getcmdtype() eval.txt /*getcmdtype()*
getcmdwintype() eval.txt /*getcmdwintype()*
getcompletion() eval.txt /*getcompletion()*
getcurpos() eval.txt /*getcurpos()*
getcwd() eval.txt /*getcwd()*
getenv() eval.txt /*getenv()*
getfontname() eval.txt /*getfontname()*
getfperm() eval.txt /*getfperm()*
getfsize() eval.txt /*getfsize()*
getftime() eval.txt /*getftime()*
getftype() eval.txt /*getftype()*
getimstatus() eval.txt /*getimstatus()*
getjumplist() eval.txt /*getjumplist()*
getlatestvimscripts-install pi_getscript.txt /*getlatestvimscripts-install*
getline() eval.txt /*getline()*
getloclist() eval.txt /*getloclist()*
getmatches() eval.txt /*getmatches()*
getmousepos() eval.txt /*getmousepos()*
getpid() eval.txt /*getpid()*
getpos() eval.txt /*getpos()*
getqflist() eval.txt /*getqflist()*
getqflist-examples quickfix.txt /*getqflist-examples*
getreg() eval.txt /*getreg()*
getregtype() eval.txt /*getregtype()*
getscript pi_getscript.txt /*getscript*
getscript-autoinstall pi_getscript.txt /*getscript-autoinstall*
getscript-data pi_getscript.txt /*getscript-data*
getscript-history pi_getscript.txt /*getscript-history*
getscript-plugins pi_getscript.txt /*getscript-plugins*
getscript-start pi_getscript.txt /*getscript-start*
gettabinfo() eval.txt /*gettabinfo()*
gettabvar() eval.txt /*gettabvar()*
gettabwinvar() eval.txt /*gettabwinvar()*
gettagstack() eval.txt /*gettagstack()*
getwininfo() eval.txt /*getwininfo()*
getwinpos() eval.txt /*getwinpos()*
getwinposx() eval.txt /*getwinposx()*
getwinposy() eval.txt /*getwinposy()*
getwinvar() eval.txt /*getwinvar()*
gex starting.txt /*gex*
gf editing.txt /*gf*
gg motion.txt /*gg*
gh visual.txt /*gh*
gi insert.txt /*gi*
gj motion.txt /*gj*
gk motion.txt /*gk*
glob() eval.txt /*glob()*
glob2regpat() eval.txt /*glob2regpat()*
global-ime mbyte.txt /*global-ime*
global-local options.txt /*global-local*
global-variable eval.txt /*global-variable*
global_markfilelist pi_netrw.txt /*global_markfilelist*
globpath() eval.txt /*globpath()*

glvs pi_getscript.txt /*glvs*
glvs-alg pi_getscript.txt /*glvs-alg*
glvs-algorithm pi_getscript.txt /*glvs-algorithm*
glvs-autoinstall pi_getscript.txt /*glvs-autoinstall*
glvs-contents pi_getscript.txt /*glvs-contents*
glvs-copyright pi_getscript.txt /*glvs-copyright*
glvs-data pi_getscript.txt /*glvs-data*
glvs-dist-install pi_getscript.txt /*glvs-dist-install*
glvs-hist pi_getscript.txt /*glvs-hist*
glvs-install pi_getscript.txt /*glvs-install*
glvs-options pi_getscript.txt /*glvs-options*
glvs-plugins pi_getscript.txt /*glvs-plugins*
glvs-usage pi_getscript.txt /*glvs-usage*
gm motion.txt /*gm*
gn visual.txt /*gn*
gnat#Insert_Tags_Header() ft_ada.txt /*gnat#Insert_Tags_Header()*
gnat#New() ft_ada.txt /*gnat#New()*
gnat-xref ft_ada.txt /*gnat-xref*
gnat_members ft_ada.txt /*gnat_members*
gnome-session gui_x11.txt /*gnome-session*
go motion.txt /*go*
gp change.txt /*gp*
gpm-mouse term.txt /*gpm-mouse*
gq change.txt /*gq*
gqap change.txt /*gqap*
gqgq change.txt /*gqgq*
gqq change.txt /*gqq*
gr change.txt /*gr*
graphic-option-gone version4.txt /*graphic-option-gone*
greek options.txt /*greek*
grep quickfix.txt /*grep*
groff.vim syntax.txt /*groff.vim*
gross-national-happiness intro.txt /*gross-national-happiness*
group-name syntax.txt /*group-name*
gs various.txt /*gs*
gsp.vim syntax.txt /*gsp.vim*
gstar pattern.txt /*gstar*
gt tabpage.txt /*gt*
gtk-css gui_x11.txt /*gtk-css*
gtk-tooltip-colors gui_x11.txt /*gtk-tooltip-colors*
gu change.txt /*gu*
gugu change.txt /*gugu*
gui gui.txt /*gui*
gui-IME gui.txt /*gui-IME*
gui-clipboard gui_w32.txt /*gui-clipboard*
gui-colors syntax.txt /*gui-colors*
gui-extras gui.txt /*gui-extras*
gui-font gui.txt /*gui-font*
gui-fontwide gui.txt /*gui-fontwide*
gui-footer debugger.txt /*gui-footer*
gui-fork gui_x11.txt /*gui-fork*
gui-functions usr_41.txt /*gui-functions*
gui-gnome gui_x11.txt /*gui-gnome*
gui-gnome-session gui_x11.txt /*gui-gnome-session*

gui-gtk	gui_x11.txt	/*gui-gtk*
gui-gtk-socketid	gui_x11.txt	/*gui-gtk-socketid*
gui-horiz-scroll	gui.txt	/*gui-horiz-scroll*
gui-init	gui.txt	/*gui-init*
gui-kde	gui_x11.txt	/*gui-kde*
gui-mouse	gui.txt	/*gui-mouse*
gui-mouse-focus	gui.txt	/*gui-mouse-focus*
gui-mouse-mapping	gui.txt	/*gui-mouse-mapping*
gui-mouse-modeless	gui.txt	/*gui-mouse-modeless*
gui-mouse-move	gui.txt	/*gui-mouse-move*
gui-mouse-select	gui.txt	/*gui-mouse-select*
gui-mouse-status	gui.txt	/*gui-mouse-status*
gui-mouse-various	gui.txt	/*gui-mouse-various*
gui-pty	gui_x11.txt	/*gui-pty*
gui-pty-erase	gui_x11.txt	/*gui-pty-erase*
gui-resources	gui_x11.txt	/*gui-resources*
gui-scrollbars	gui.txt	/*gui-scrollbars*
gui-selections	gui.txt	/*gui-selections*
gui-shell	gui.txt	/*gui-shell*
gui-shell-win32	gui_w32.txt	/*gui-shell-win32*
gui-start	gui.txt	/*gui-start*
gui-toolbar	gui.txt	/*gui-toolbar*
gui-vert-scroll	gui.txt	/*gui-vert-scroll*
gui-w32	gui_w32.txt	/*gui-w32*
gui-w32-cmdargs	gui_w32.txt	/*gui-w32-cmdargs*
gui-w32-dialogs	gui_w32.txt	/*gui-w32-dialogs*
gui-w32-printing	gui_w32.txt	/*gui-w32-printing*
gui-w32-start	gui_w32.txt	/*gui-w32-start*
gui-w32-various	gui_w32.txt	/*gui-w32-various*
gui-w32-windowid	gui_w32.txt	/*gui-w32-windowid*
gui-w32s	os_win32.txt	/*gui-w32s*
gui-win32-maximized	gui_w32.txt	/*gui-win32-maximized*
gui-x11	gui_x11.txt	/*gui-x11*
gui-x11-athena	gui_x11.txt	/*gui-x11-athena*
gui-x11-compiling	gui_x11.txt	/*gui-x11-compiling*
gui-x11-gtk	gui_x11.txt	/*gui-x11-gtk*
gui-x11-kde	gui_x11.txt	/*gui-x11-kde*
gui-x11-misc	gui_x11.txt	/*gui-x11-misc*
gui-x11-motif	gui_x11.txt	/*gui-x11-motif*
gui-x11-neXtaw	gui_x11.txt	/*gui-x11-neXtaw*
gui-x11-printing	gui_x11.txt	/*gui-x11-printing*
gui-x11-start	gui_x11.txt	/*gui-x11-start*
gui-x11-various	gui_x11.txt	/*gui-x11-various*
gui.txt	gui.txt	/*gui.txt*
gui_w32.txt	gui_w32.txt	/*gui_w32.txt*
gui_x11.txt	gui_x11.txt	/*gui_x11.txt*
guifontwide_gtk	gui.txt	/*guifontwide_gtk*
guifontwide_win_mbyte	gui.txt	/*guifontwide_win_mbyte*
guu	change.txt	/*guu*
gv	visual.txt	/*gv*
gview	starting.txt	/*gview*
gvim	starting.txt	/*gvim*
gvimdiff	diff.txt	/*gvimdiff*
gvimrc	gui.txt	/*gvimrc*

gw change.txt /*gw*
gwgw change.txt /*gwgw*
gww change.txt /*gww*
gzip pi_gzip.txt /*gzip*
gzip-autocmd pi_gzip.txt /*gzip-autocmd*
gzip-example autocmd.txt /*gzip-example*
gzip-helpfile tips.txt /*gzip-helpfile*
g~ change.txt /*g~*
g~g~ change.txt /*g~g~*
g~~ change.txt /*g~~*
h motion.txt /*h*
hangul hangulin.txt /*hangul*
hangulin.txt hangulin.txt /*hangulin.txt*
has() eval.txt /*has()*
has-patch eval.txt /*has-patch*
has-python if_pyth.txt /*has-python*
has-pythonx if_pyth.txt /*has-pythonx*
has_key() eval.txt /*has_key()*
haskell.vim syntax.txt /*haskell.vim*
haslocaldir() eval.txt /*haslocaldir()*
hasmapto() eval.txt /*hasmapto()*
hebrew hebrew.txt /*hebrew*
hebrew.txt hebrew.txt /*hebrew.txt*
help helphelp.txt /*help*
help-context help.txt /*help-context*
help-summary usr_02.txt /*help-summary*
help-translated helphelp.txt /*help-translated*
help-writing helphelp.txt /*help-writing*
help-xterm-window helphelp.txt /*help-xterm-window*
help.txt help.txt /*help.txt*
helpfile_name.txt helphelp.txt /*helpfile_name.txt*
helphelp helphelp.txt /*helphelp*
helphelp.txt helphelp.txt /*helphelp.txt*
hex-editing tips.txt /*hex-editing*
hex-number eval.txt /*hex-number*
hidden-buffer windows.txt /*hidden-buffer*
hidden-changed version5.txt /*hidden-changed*
hidden-menus gui.txt /*hidden-menus*
hidden-options options.txt /*hidden-options*
hidden-quit windows.txt /*hidden-quit*
highlight-args syntax.txt /*highlight-args*
highlight-changed version4.txt /*highlight-changed*
highlight-cterm syntax.txt /*highlight-cterm*
highlight-ctermbg syntax.txt /*highlight-ctermbg*
highlight-ctermfg syntax.txt /*highlight-ctermfg*
highlight-default syntax.txt /*highlight-default*
highlight-font syntax.txt /*highlight-font*
highlight-groups syntax.txt /*highlight-groups*
highlight-gui syntax.txt /*highlight-gui*
highlight-guibg syntax.txt /*highlight-guibg*
highlight-guifg syntax.txt /*highlight-guifg*
highlight-guisp syntax.txt /*highlight-guisp*
highlight-start syntax.txt /*highlight-start*
highlight-stop syntax.txt /*highlight-stop*

highlight-term syntax.txt /*highlight-term*
highlightID() eval.txt /*highlightID()*
highlight_exists() eval.txt /*highlight_exists()*
highlighting-functions usr_41.txt /*highlighting-functions*
hist-names eval.txt /*hist-names*
histadd() eval.txt /*histadd()*
histdel() eval.txt /*histdel()*
histget() eval.txt /*histget()*
histnr() eval.txt /*histnr()*
history cmdline.txt /*history*
history-functions usr_41.txt /*history-functions*
hit-enter message.txt /*hit-enter*
hit-enter-prompt message.txt /*hit-enter-prompt*
hit-return message.txt /*hit-return*
hitest.vim syntax.txt /*hitest.vim*
hjkl usr_02.txt /*hjkl*
hl-ColorColumn syntax.txt /*hl-ColorColumn*
hl-Conceal syntax.txt /*hl-Conceal*
hl-Cursor syntax.txt /*hl-Cursor*
hl-CursorColumn syntax.txt /*hl-CursorColumn*
hl-CursorIM syntax.txt /*hl-CursorIM*
hl-CursorLine syntax.txt /*hl-CursorLine*
hl-CursorLineNr syntax.txt /*hl-CursorLineNr*
hl-DiffAdd syntax.txt /*hl-DiffAdd*
hl-DiffChange syntax.txt /*hl-DiffChange*
hl-DiffDelete syntax.txt /*hl-DiffDelete*
hl-DiffText syntax.txt /*hl-DiffText*
hl-Directory syntax.txt /*hl-Directory*
hl-EndOfBuffer syntax.txt /*hl-EndOfBuffer*
hl-ErrorMsg syntax.txt /*hl-ErrorMsg*
hl-FoldColumn syntax.txt /*hl-FoldColumn*
hl-Folded syntax.txt /*hl-Folded*
hl-Ignore syntax.txt /*hl-Ignore*
hl-IncSearch syntax.txt /*hl-IncSearch*
hl-LineNr syntax.txt /*hl-LineNr*
hl-LineNrAbove syntax.txt /*hl-LineNrAbove*
hl-LineNrBelow syntax.txt /*hl-LineNrBelow*
hl-MatchParen syntax.txt /*hl-MatchParen*
hl-Menu syntax.txt /*hl-Menu*
hl-ModeMsg syntax.txt /*hl-ModeMsg*
hl-MoreMsg syntax.txt /*hl-MoreMsg*
hl-NonText syntax.txt /*hl-NonText*
hl-Normal syntax.txt /*hl-Normal*
hl-Pmenu syntax.txt /*hl-Pmenu*
hl-PmenuSbar syntax.txt /*hl-PmenuSbar*
hl-PmenuSel syntax.txt /*hl-PmenuSel*
hl-PmenuThumb syntax.txt /*hl-PmenuThumb*
hl-Question syntax.txt /*hl-Question*
hl-QuickFixLine syntax.txt /*hl-QuickFixLine*
hl-Scrollbar syntax.txt /*hl-Scrollbar*
hl-Search syntax.txt /*hl-Search*
hl-SignColumn syntax.txt /*hl-SignColumn*
hl-SpecialKey syntax.txt /*hl-SpecialKey*
hl-SpellBad syntax.txt /*hl-SpellBad*

hl-SpellCap	syntax.txt	/*hl-SpellCap*
hl-SpellLocal	syntax.txt	/*hl-SpellLocal*
hl-SpellRare	syntax.txt	/*hl-SpellRare*
hl-StatusLine	syntax.txt	/*hl-StatusLine*
hl-StatusLineNC	syntax.txt	/*hl-StatusLineNC*
hl-StatusLineTerm	syntax.txt	/*hl-StatusLineTerm*
hl-StatusLineTermNC	syntax.txt	/*hl-StatusLineTermNC*
hl-TOhtmlProgress	syntax.txt	/*hl-TOhtmlProgress*
hl-TabLine	syntax.txt	/*hl-TabLine*
hl-TabLineFill	syntax.txt	/*hl-TabLineFill*
hl-TabLineSel	syntax.txt	/*hl-TabLineSel*
hl-Terminal	syntax.txt	/*hl-Terminal*
hl-Title	syntax.txt	/*hl-Title*
hl-Tooltip	syntax.txt	/*hl-Tooltip*
hl-User1	syntax.txt	/*hl-User1*
hl-User1..9	syntax.txt	/*hl-User1..9*
hl-User9	syntax.txt	/*hl-User9*
hl-VertSplit	syntax.txt	/*hl-VertSplit*
hl-Visual	syntax.txt	/*hl-Visual*
hl-VisualNOS	syntax.txt	/*hl-VisualNOS*
hl-WarningMsg	syntax.txt	/*hl-WarningMsg*
hl-WildMenu	syntax.txt	/*hl-WildMenu*
hl-debugBreakpoint	terminal.txt	/*hl-debugBreakpoint*
hl-debugPC	terminal.txt	/*hl-debugPC*
hlID()	eval.txt	/*hlID()*
hlexists()	eval.txt	/*hlexists()*
hlsearch-variable	eval.txt	/*hlsearch-variable*
holy-grail	index.txt	/*holy-grail*
home	intro.txt	/*home*
home-replace	editing.txt	/*home-replace*
hostname()	eval.txt	/*hostname()*
how-do-i	howto.txt	/*how-do-i*
how-to	howto.txt	/*how-to*
howdoi	howto.txt	/*howdoi*
howto	howto.txt	/*howto*
howto.txt	howto.txt	/*howto.txt*
hpterm	term.txt	/*hpterm*
hpterm-color	syntax.txt	/*hpterm-color*
html-flavor	insert.txt	/*html-flavor*
html-indent	indent.txt	/*html-indent*
html-indenting	indent.txt	/*html-indenting*
html.vim	syntax.txt	/*html.vim*
htmlos.vim	syntax.txt	/*htmlos.vim*
http	pi_netrw.txt	/*http*
i	insert.txt	/*i*
i'	motion.txt	/*i'*
i(	motion.txt	/*i(*
i)	motion.txt	/*i)*
i<	motion.txt	/*i<*
i>	motion.txt	/*i>*
iB	motion.txt	/*iB*
iBus	gui.txt	/*iBus*
iW	motion.txt	/*iW*
i[	motion.txt	/*i[*

i]	motion.txt	/*i]*
i_0_CTRL-D	insert.txt	/*i_0_CTRL-D*
i_<BS>	insert.txt	/*i_<BS>*
i_<C-End>	insert.txt	/*i_<C-End>*
i_<C-Home>	insert.txt	/*i_<C-Home>*
i_<C-Left>	insert.txt	/*i_<C-Left>*
i_<C-PageDown>	tabpage.txt	/*i_<C-PageDown>*
i_<C-PageUp>	tabpage.txt	/*i_<C-PageUp>*
i_<C-Right>	insert.txt	/*i_<C-Right>*
i_<CR>	insert.txt	/*i_<CR>*
i_<Del>	insert.txt	/*i_<Del>*
i_<Down>	insert.txt	/*i_<Down>*
i_<End>	insert.txt	/*i_<End>*
i_<Esc>	insert.txt	/*i_<Esc>*
i_<F1>	helphelp.txt	/*i_<F1>*
i_<Help>	helphelp.txt	/*i_<Help>*
i_<Home>	insert.txt	/*i_<Home>*
i_<Insert>	insert.txt	/*i_<Insert>*
i_<Left>	insert.txt	/*i_<Left>*
i_<LeftMouse>	insert.txt	/*i_<LeftMouse>*
i_<NL>	insert.txt	/*i_<NL>*
i_<PageDown>	insert.txt	/*i_<PageDown>*
i_<PageUp>	insert.txt	/*i_<PageUp>*
i_<Right>	insert.txt	/*i_<Right>*
i_<S-Down>	insert.txt	/*i_<S-Down>*
i_<S-Left>	insert.txt	/*i_<S-Left>*
i_<S-Right>	insert.txt	/*i_<S-Right>*
i_<S-ScrollWheelDown>	insert.txt	/*i_<S-ScrollWheelDown>*
i_<S-ScrollWheelLeft>	insert.txt	/*i_<S-ScrollWheelLeft>*
i_<S-ScrollWheelRight>	insert.txt	/*i_<S-ScrollWheelRight>*
i_<S-ScrollWheelUp>	insert.txt	/*i_<S-ScrollWheelUp>*
i_<S-Up>	insert.txt	/*i_<S-Up>*
i_<ScrollWheelDown>	insert.txt	/*i_<ScrollWheelDown>*
i_<ScrollWheelLeft>	insert.txt	/*i_<ScrollWheelLeft>*
i_<ScrollWheelRight>	insert.txt	/*i_<ScrollWheelRight>*
i_<ScrollWheelUp>	insert.txt	/*i_<ScrollWheelUp>*
i_<Tab>	insert.txt	/*i_<Tab>*
i_<Up>	insert.txt	/*i_<Up>*
i_BS	insert.txt	/*i_BS*
i_CTRL-<PageDown>	tabpage.txt	/*i_CTRL-<PageDown>*
i_CTRL-<PageUp>	tabpage.txt	/*i_CTRL-<PageUp>*
i_CTRL-@	insert.txt	/*i_CTRL-@*
i_CTRL-A	insert.txt	/*i_CTRL-A*
i_CTRL-B-gone	version5.txt	/*i_CTRL-B-gone*
i_CTRL-C	insert.txt	/*i_CTRL-C*
i_CTRL-D	insert.txt	/*i_CTRL-D*
i_CTRL-E	insert.txt	/*i_CTRL-E*
i_CTRL-F	indent.txt	/*i_CTRL-F*
i_CTRL-G_<Down>	insert.txt	/*i_CTRL-G_<Down>*
i_CTRL-G_<Up>	insert.txt	/*i_CTRL-G_<Up>*
i_CTRL-G_CTRL-J	insert.txt	/*i_CTRL-G_CTRL-J*
i_CTRL-G_CTRL-K	insert.txt	/*i_CTRL-G_CTRL-K*
i_CTRL-G_U	insert.txt	/*i_CTRL-G_U*
i_CTRL-G_j	insert.txt	/*i_CTRL-G_j*

```
i_CTRL-G_k              insert.txt              /*i_CTRL-G_k*
i_CTRL-G_u              insert.txt              /*i_CTRL-G_u*
i_CTRL-H                insert.txt               /*i_CTRL-H*
i_CTRL-I                insert.txt               /*i_CTRL-I*
i_CTRL-J                insert.txt               /*i_CTRL-J*
i_CTRL-K                insert.txt               /*i_CTRL-K*
i_CTRL-L                insert.txt               /*i_CTRL-L*
i_CTRL-M                insert.txt               /*i_CTRL-M*
i_CTRL-N                insert.txt               /*i_CTRL-N*
i_CTRL-O                insert.txt               /*i_CTRL-O*
i_CTRL-P                insert.txt               /*i_CTRL-P*
i_CTRL-Q                insert.txt               /*i_CTRL-Q*
i_CTRL-R                insert.txt               /*i_CTRL-R*
i_CTRL-R_=              insert.txt              /*i_CTRL-R_=*
i_CTRL-R_CTRL-O           insert.txt           /*i_CTRL-R_CTRL-O*
i_CTRL-R_CTRL-P           insert.txt           /*i_CTRL-R_CTRL-P*
i_CTRL-R_CTRL-R           insert.txt           /*i_CTRL-R_CTRL-R*
i_CTRL-SHIFT-V            insert.txt            /*i_CTRL-SHIFT-V*
i_CTRL-T                insert.txt               /*i_CTRL-T*
i_CTRL-U                insert.txt               /*i_CTRL-U*
i_CTRL-V                insert.txt               /*i_CTRL-V*
i_CTRL-V_digit            insert.txt            /*i_CTRL-V_digit*
i_CTRL-W                insert.txt               /*i_CTRL-W*
i_CTRL-X                insert.txt               /*i_CTRL-X*
i_CTRL-X_CTRL-D           insert.txt           /*i_CTRL-X_CTRL-D*
i_CTRL-X_CTRL-E           insert.txt           /*i_CTRL-X_CTRL-E*
i_CTRL-X_CTRL-F           insert.txt           /*i_CTRL-X_CTRL-F*
i_CTRL-X_CTRL-I           insert.txt           /*i_CTRL-X_CTRL-I*
i_CTRL-X_CTRL-K           insert.txt           /*i_CTRL-X_CTRL-K*
i_CTRL-X_CTRL-L           insert.txt           /*i_CTRL-X_CTRL-L*
i_CTRL-X_CTRL-N           insert.txt           /*i_CTRL-X_CTRL-N*
i_CTRL-X_CTRL-O           insert.txt           /*i_CTRL-X_CTRL-O*
i_CTRL-X_CTRL-P           insert.txt           /*i_CTRL-X_CTRL-P*
i_CTRL-X_CTRL-S           insert.txt           /*i_CTRL-X_CTRL-S*
i_CTRL-X_CTRL-T           insert.txt           /*i_CTRL-X_CTRL-T*
i_CTRL-X_CTRL-U           insert.txt           /*i_CTRL-X_CTRL-U*
i_CTRL-X_CTRL-V           insert.txt           /*i_CTRL-X_CTRL-V*
i_CTRL-X_CTRL-Y           insert.txt           /*i_CTRL-X_CTRL-Y*
i_CTRL-X_CTRL-]           insert.txt           /*i_CTRL-X_CTRL-]*
i_CTRL-X_index            index.txt            /*i_CTRL-X_index*
i_CTRL-X_s              insert.txt               /*i_CTRL-X_s*
i_CTRL-Y                insert.txt               /*i_CTRL-Y*
i_CTRL-Z                options.txt              /*i_CTRL-Z*
i_CTRL-[                insert.txt               /*i_CTRL-[*
i_CTRL-\_CTRL-G           intro.txt            /*i_CTRL-\\_CTRL-G*
i_CTRL-\_CTRL-N           intro.txt            /*i_CTRL-\\_CTRL-N*
i_CTRL-\_CTRL-O           insert.txt           /*i_CTRL-\\_CTRL-O*
i_CTRL-]                insert.txt               /*i_CTRL-]*
i_CTRL-^                insert.txt               /*i_CTRL-^*
i_CTRL-_                insert.txt               /*i_CTRL-_*
i_DEL                  insert.txt                /*i_DEL*
i_Tab                  insert.txt                /*i_Tab*
i_^_CTRL-D              insert.txt              /*i_^_CTRL-D*
i_backspacing            insert.txt            /*i_backspacing*
```

i_digraph	digraph.txt	/*i_digraph*
i_esc	intro.txt	/*i_esc*
i‘	motion.txt	/*i‘*
ia64.vim	syntax.txt	/*ia64.vim*
ib	motion.txt	/*ib*
iccf	uganda.txt	/*iccf*
iccf-donations	uganda.txt	/*iccf-donations*
icon-changed	version4.txt	/*icon-changed*
iconise	starting.txt	/*iconise*
iconize	starting.txt	/*iconize*
iconv()	eval.txt	/*iconv()*
iconv-dynamic	mbyte.txt	/*iconv-dynamic*
ident-search	tips.txt	/*ident-search*
idl-syntax	syntax.txt	/*idl-syntax*
idl.vim	syntax.txt	/*idl.vim*
if_cscop.txt	if_cscop.txt	/*if_cscop.txt*
if_lua.txt	if_lua.txt	/*if_lua.txt*
if_mzsch.txt	if_mzsch.txt	/*if_mzsch.txt*
if_ole.txt	if_ole.txt	/*if_ole.txt*
if_perl.txt	if_perl.txt	/*if_perl.txt*
if_pyth.txt	if_pyth.txt	/*if_pyth.txt*
if_ruby.txt	if_ruby.txt	/*if_ruby.txt*
if_sniff.txt	if_sniff.txt	/*if_sniff.txt*
if_tcl.txt	if_tcl.txt	/*if_tcl.txt*
ignore-errors	eval.txt	/*ignore-errors*
ignore-timestamp	editing.txt	/*ignore-timestamp*
improved-autocmds-5.4	version5.txt	/*improved-autocmds-5.4*
improved-quickfix	version5.txt	/*improved-quickfix*
improved-sessions	version5.txt	/*improved-sessions*
improved-viminfo	version5.txt	/*improved-viminfo*
improvements-5	version5.txt	/*improvements-5*
improvements-6	version6.txt	/*improvements-6*
improvements-7	version7.txt	/*improvements-7*
improvements-8	version8.txt	/*improvements-8*
in_bot	channel.txt	/*in_bot*
in_buf	channel.txt	/*in_buf*
in_io-buffer	channel.txt	/*in_io-buffer*
in_mode	channel.txt	/*in_mode*
in_name	channel.txt	/*in_name*
in_top	channel.txt	/*in_top*
inactive-buffer	windows.txt	/*inactive-buffer*
include-search	tagsrch.txt	/*include-search*
inclusive	motion.txt	/*inclusive*
incomp-small-6	version6.txt	/*incomp-small-6*
incompatible-5	version5.txt	/*incompatible-5*
incompatible-6	version6.txt	/*incompatible-6*
incompatible-7	version7.txt	/*incompatible-7*
incompatible-8	version8.txt	/*incompatible-8*
indent()	eval.txt	/*indent()*
indent-expression	indent.txt	/*indent-expression*
indent.txt	indent.txt	/*indent.txt*
indentkeys-format	indent.txt	/*indentkeys-format*
index	index.txt	/*index*
index()	eval.txt	/*index()*

index.txt	index.txt	/*index.txt*
info-message	starting.txt	/*info-message*
inform.vim	syntax.txt	/*inform.vim*
informix	ft_sql.txt	/*informix*
initialization	starting.txt	/*initialization*
input()	eval.txt	/*input()*
inputdialog()	eval.txt	/*inputdialog()*
inputlist()	eval.txt	/*inputlist()*
inputrestore()	eval.txt	/*inputrestore()*
inputsave()	eval.txt	/*inputsave()*
inputsecret()	eval.txt	/*inputsecret()*
ins-completion	insert.txt	/*ins-completion*
ins-completion-menu	insert.txt	/*ins-completion-menu*
ins-expandtab	insert.txt	/*ins-expandtab*
ins-reverse	rileft.txt	/*ins-reverse*
ins-smarttab	insert.txt	/*ins-smarttab*
ins-softtabstop	insert.txt	/*ins-softtabstop*
ins-special-keys	insert.txt	/*ins-special-keys*
ins-special-special	insert.txt	/*ins-special-special*
ins-textwidth	insert.txt	/*ins-textwidth*
insert	insert.txt	/*insert*
insert()	eval.txt	/*insert()*
insert-index	index.txt	/*insert-index*
insert.txt	insert.txt	/*insert.txt*
insert_expand	insert.txt	/*insert_expand*
inserting	insert.txt	/*inserting*
inserting-ex	insert.txt	/*inserting-ex*
inserting-file	insert.txt	/*inserting-file*
insertmode-variable	eval.txt	/*insertmode-variable*
install	usr_90.txt	/*install*
install-home	usr_90.txt	/*install-home*
install-registry	gui_w32.txt	/*install-registry*
intel-itanium	syntax.txt	/*intel-itanium*
intellimouse-wheel-problems	gui_w32.txt	/*intellimouse-wheel-problems*
interactive-functions	usr_41.txt	/*interactive-functions*
interfaces-5.2	version5.txt	/*interfaces-5.2*
internal-variables	eval.txt	/*internal-variables*
internal-wordlist	spell.txt	/*internal-wordlist*
internet	intro.txt	/*internet*
interrupt()	eval.txt	/*interrupt()*
intro	intro.txt	/*intro*
intro.txt	intro.txt	/*intro.txt*
inverse	syntax.txt	/*inverse*
invert()	eval.txt	/*invert()*
ip	motion.txt	/*ip*
iquote	motion.txt	/*iquote*
is	motion.txt	/*is*
isdirectory()	eval.txt	/*isdirectory()*
isinf()	eval.txt	/*isinf()*
islocked()	eval.txt	/*islocked()*
isnan()	eval.txt	/*isnan()*
it	motion.txt	/*it*
italic	syntax.txt	/*italic*
items()	eval.txt	/*items()*

iw	motion.txt	/*iw*
i{	motion.txt	/*i{*
i}	motion.txt	/*i}*
j	motion.txt	/*j*
java-cinoptions	indent.txt	/*java-cinoptions*
java-indenting	indent.txt	/*java-indenting*
java.vim	syntax.txt	/*java.vim*
javascript-cinoptions	indent.txt	/*javascript-cinoptions*
javascript-indenting	indent.txt	/*javascript-indenting*
job	channel.txt	/*job*
job-callback	channel.txt	/*job-callback*
job-channel-overview	channel.txt	/*job-channel-overview*
job-close_cb	channel.txt	/*job-close_cb*
job-control	channel.txt	/*job-control*
job-drop	channel.txt	/*job-drop*
job-err_cb	channel.txt	/*job-err_cb*
job-err_io	channel.txt	/*job-err_io*
job-exit_cb	channel.txt	/*job-exit_cb*
job-functions	usr_41.txt	/*job-functions*
job-functions-details	channel.txt	/*job-functions-details*
job-in_io	channel.txt	/*job-in_io*
job-noblock	channel.txt	/*job-noblock*
job-options	channel.txt	/*job-options*
job-out_cb	channel.txt	/*job-out_cb*
job-out_io	channel.txt	/*job-out_io*
job-start	channel.txt	/*job-start*
job-start-if-needed	channel.txt	/*job-start-if-needed*
job-start-nochannel	channel.txt	/*job-start-nochannel*
job-stoponexit	channel.txt	/*job-stoponexit*
job-term	channel.txt	/*job-term*
job-timeout	channel.txt	/*job-timeout*
job_getchannel()	channel.txt	/*job_getchannel()*
job_info()	channel.txt	/*job_info()*
job_setoptions()	channel.txt	/*job_setoptions()*
job_start()	channel.txt	/*job_start()*
job_status()	channel.txt	/*job_status()*
job_stop()	channel.txt	/*job_stop()*
join()	eval.txt	/*join()*
js_decode()	eval.txt	/*js_decode()*
js_encode()	eval.txt	/*js_encode()*
jsbterm-mouse	options.txt	/*jsbterm-mouse*
json.vim	syntax.txt	/*json.vim*
json_decode()	eval.txt	/*json_decode()*
json_encode()	eval.txt	/*json_encode()*
jtags	tagsrch.txt	/*jtags*
jump-motions	motion.txt	/*jump-motions*
jumplist	motion.txt	/*jumplist*
jumpto-diffs	diff.txt	/*jumpto-diffs*
k	motion.txt	/*k*
kcc	uganda.txt	/*kcc*
kde	gui_x11.txt	/*kde*
key-codes	intro.txt	/*key-codes*
key-codes-changed	version4.txt	/*key-codes-changed*
key-mapping	map.txt	/*key-mapping*

key-notation	intro.txt	/*key-notation*
key-variable	eval.txt	/*key-variable*
keycodes	intro.txt	/*keycodes*
keymap-accents	mbyte.txt	/*keymap-accents*
keymap-file-format	mbyte.txt	/*keymap-file-format*
keymap-hebrew	mbyte.txt	/*keymap-hebrew*
keypad-0	intro.txt	/*keypad-0*
keypad-9	intro.txt	/*keypad-9*
keypad-comma	term.txt	/*keypad-comma*
keypad-divide	intro.txt	/*keypad-divide*
keypad-end	intro.txt	/*keypad-end*
keypad-enter	intro.txt	/*keypad-enter*
keypad-home	intro.txt	/*keypad-home*
keypad-minus	intro.txt	/*keypad-minus*
keypad-multiply	intro.txt	/*keypad-multiply*
keypad-page-down	intro.txt	/*keypad-page-down*
keypad-page-up	intro.txt	/*keypad-page-up*
keypad-plus	intro.txt	/*keypad-plus*
keypad-point	intro.txt	/*keypad-point*
keys()	eval.txt	/*keys()*
known-bugs	todo.txt	/*known-bugs*
l	motion.txt	/*l*
l:	eval.txt	/*l:*
l:var	eval.txt	/*l:var*
lCursor	mbyte.txt	/*lCursor*
lace.vim	syntax.txt	/*lace.vim*
lambda	eval.txt	/*lambda*
lang-variable	eval.txt	/*lang-variable*
language-mapping	map.txt	/*language-mapping*
last-pattern	pattern.txt	/*last-pattern*
last-position-jump	usr_05.txt	/*last-position-jump*
last_buffer_nr()	eval.txt	/*last_buffer_nr()*
latex-syntax	syntax.txt	/*latex-syntax*
lc_time-variable	eval.txt	/*lc_time-variable*
lcs-conceal	options.txt	/*lcs-conceal*
lcs-eol	options.txt	/*lcs-eol*
lcs-extends	options.txt	/*lcs-extends*
lcs-nbsp	options.txt	/*lcs-nbsp*
lcs-precedes	options.txt	/*lcs-precedes*
lcs-space	options.txt	/*lcs-space*
lcs-tab	options.txt	/*lcs-tab*
lcs-trail	options.txt	/*lcs-trail*
left-right-motions	motion.txt	/*left-right-motions*
len()	eval.txt	/*len()*
less	various.txt	/*less*
letter	print.txt	/*letter*
lex.vim	syntax.txt	/*lex.vim*
lhaskell.vim	syntax.txt	/*lhaskell.vim*
libcall()	eval.txt	/*libcall()*
libcallnr()	eval.txt	/*libcallnr()*
license	uganda.txt	/*license*
lid	quickfix.txt	/*lid*
lifelines.vim	syntax.txt	/*lifelines.vim*
limits	vi_diff.txt	/*limits*

line() eval.txt /*line()*
line-continuation repeat.txt /*line-continuation*
line-continuation-comment repeat.txt /*line-continuation-comment*
line2byte() eval.txt /*line2byte()*
linefeed intro.txt /*linefeed*
linewise motion.txt /*linewise*
linewise-register change.txt /*linewise-register*
linewise-visual visual.txt /*linewise-visual*
lisp.vim syntax.txt /*lisp.vim*
lispindent() eval.txt /*lispindent()*
list eval.txt /*list*
list-functions usr_41.txt /*list-functions*
list-identity eval.txt /*list-identity*
list-index eval.txt /*list-index*
list-modification eval.txt /*list-modification*
list-repeat windows.txt /*list-repeat*
list2str() eval.txt /*list2str()*
listener_add() eval.txt /*listener_add()*
listener_flush() eval.txt /*listener_flush()*
listener_remove() eval.txt /*listener_remove()*
lite.vim syntax.txt /*lite.vim*
literal-Dict eval.txt /*literal-Dict*
literal-string eval.txt /*literal-string*
lnum-variable eval.txt /*lnum-variable*
load-plugins starting.txt /*load-plugins*
load-vim-script repeat.txt /*load-vim-script*
local-function eval.txt /*local-function*
local-options options.txt /*local-options*
local-variable eval.txt /*local-variable*
local-variables eval.txt /*local-variables*
local_markfilelist pi_netrw.txt /*local_markfilelist*
locale mbyte.txt /*locale*
locale-name mbyte.txt /*locale-name*
localtime() eval.txt /*localtime()*
location-list quickfix.txt /*location-list*
location-list-file-window quickfix.txt /*location-list-file-window*
location-list-window quickfix.txt /*location-list-window*
log() eval.txt /*log()*
log10() eval.txt /*log10()*
logiPat pi_logipat.txt /*logiPat*
logiPat-arg pi_logipat.txt /*logiPat-arg*
logiPat-caveat pi_logipat.txt /*logiPat-caveat*
logiPat-contents pi_logipat.txt /*logiPat-contents*
logiPat-copyright pi_logipat.txt /*logiPat-copyright*
logiPat-examples pi_logipat.txt /*logiPat-examples*
logiPat-history pi_logipat.txt /*logiPat-history*
logiPat-input pi_logipat.txt /*logiPat-input*
logiPat-man pi_logipat.txt /*logiPat-man*
logiPat-manual pi_logipat.txt /*logiPat-manual*
logiPat-operators pi_logipat.txt /*logiPat-operators*
logiPat-pattern pi_logipat.txt /*logiPat-pattern*
long-lines version5.txt /*long-lines*
love intro.txt /*love*
lowercase change.txt /*lowercase*

lpc.vim	syntax.txt	/*lpc.vim*
lua	if_lua.txt	/*lua*
lua-blob	if_lua.txt	/*lua-blob*
lua-buffer	if_lua.txt	/*lua-buffer*
lua-commands	if_lua.txt	/*lua-commands*
lua-dict	if_lua.txt	/*lua-dict*
lua-dynamic	if_lua.txt	/*lua-dynamic*
lua-eval	if_lua.txt	/*lua-eval*
lua-funcref	if_lua.txt	/*lua-funcref*
lua-list	if_lua.txt	/*lua-list*
lua-luaeval	if_lua.txt	/*lua-luaeval*
lua-vim	if_lua.txt	/*lua-vim*
lua-window	if_lua.txt	/*lua-window*
lua.vim	syntax.txt	/*lua.vim*
luaeval()	eval.txt	/*luaeval()*
m	motion.txt	/*m*
m'	motion.txt	/*m'*
m<	motion.txt	/*m<*
m>	motion.txt	/*m>*
m[	motion.txt	/*m[*
m]	motion.txt	/*m]*
m`	motion.txt	/*m`*
mac	os_mac.txt	/*mac*
mac-bug	os_mac.txt	/*mac-bug*
mac-compile	os_mac.txt	/*mac-compile*
mac-darwin-feature	os_mac.txt	/*mac-darwin-feature*
mac-faq	os_mac.txt	/*mac-faq*
mac-filename	os_mac.txt	/*mac-filename*
mac-lack	os_mac.txt	/*mac-lack*
mac-standard-mappings	os_mac.txt	/*mac-standard-mappings*
mac-vimfile	os_mac.txt	/*mac-vimfile*
macintosh	os_mac.txt	/*macintosh*
macro	map.txt	/*macro*
mail-list	intro.txt	/*mail-list*
mail.vim	syntax.txt	/*mail.vim*
maillist	intro.txt	/*maillist*
maillist-archive	intro.txt	/*maillist-archive*
make.vim	syntax.txt	/*make.vim*
man.vim	filetype.txt	/*man.vim*
manpager.vim	filetype.txt	/*manpager.vim*
manual-copyright	usr_01.txt	/*manual-copyright*
map()	eval.txt	/*map()*
map-<SID>	map.txt	/*map-<SID>*
map-CTRL-C	map.txt	/*map-CTRL-C*
map-ambiguous	map.txt	/*map-ambiguous*
map-backslash	map.txt	/*map-backslash*
map-backtick	tips.txt	/*map-backtick*
map-bar	map.txt	/*map-bar*
map-comments	map.txt	/*map-comments*
map-empty-rhs	map.txt	/*map-empty-rhs*
map-error	map.txt	/*map-error*
map-examples	map.txt	/*map-examples*
map-keys-fails	map.txt	/*map-keys-fails*
map-listing	map.txt	/*map-listing*

map-modes	map.txt	/*map-modes*
map-multibyte	map.txt	/*map-multibyte*
map-overview	map.txt	/*map-overview*
map-precedence	map.txt	/*map-precedence*
map-return	map.txt	/*map-return*
map-self-destroy	tips.txt	/*map-self-destroy*
map-space_in_lhs	map.txt	/*map-space_in_lhs*
map-space_in_rhs	map.txt	/*map-space_in_rhs*
map-typing	map.txt	/*map-typing*
map-which-keys	map.txt	/*map-which-keys*
map.txt	map.txt	/*map.txt*
map_CTRL-C	map.txt	/*map_CTRL-C*
map_backslash	map.txt	/*map_backslash*
map_bar	map.txt	/*map_bar*
map_empty_rhs	map.txt	/*map_empty_rhs*
map_return	map.txt	/*map_return*
map_space_in_lhs	map.txt	/*map_space_in_lhs*
map_space_in_rhs	map.txt	/*map_space_in_rhs*
maparg()	eval.txt	/*maparg()*
mapcheck()	eval.txt	/*mapcheck()*
maple.vim	syntax.txt	/*maple.vim*
mapleader	map.txt	/*mapleader*
maplocalleader	map.txt	/*maplocalleader*
mapmode-c	map.txt	/*mapmode-c*
mapmode-i	map.txt	/*mapmode-i*
mapmode-ic	map.txt	/*mapmode-ic*
mapmode-l	map.txt	/*mapmode-l*
mapmode-n	map.txt	/*mapmode-n*
mapmode-nvo	map.txt	/*mapmode-nvo*
mapmode-o	map.txt	/*mapmode-o*
mapmode-s	map.txt	/*mapmode-s*
mapmode-t	map.txt	/*mapmode-t*
mapmode-v	map.txt	/*mapmode-v*
mapmode-x	map.txt	/*mapmode-x*
mapping	map.txt	/*mapping*
mapping-functions	usr_41.txt	/*mapping-functions*
mark	motion.txt	/*mark*
mark-functions	usr_41.txt	/*mark-functions*
mark-motions	motion.txt	/*mark-motions*
markfilelist	pi_netrw.txt	/*markfilelist*
masm.vim	syntax.txt	/*masm.vim*
match()	eval.txt	/*match()*
match-highlight	pattern.txt	/*match-highlight*
match-parens	tips.txt	/*match-parens*
matchadd()	eval.txt	/*matchadd()*
matchaddpos()	eval.txt	/*matchaddpos()*
matcharg()	eval.txt	/*matcharg()*
matchdelete()	eval.txt	/*matchdelete()*
matchend()	eval.txt	/*matchend()*
matchit-install	usr_05.txt	/*matchit-install*
matchlist()	eval.txt	/*matchlist()*
matchparen	pi_paren.txt	/*matchparen*
matchstr()	eval.txt	/*matchstr()*
matchstrpos()	eval.txt	/*matchstrpos()*

matlab-indent indent.txt /*matlab-indent*
matlab-indenting indent.txt /*matlab-indenting*
max() eval.txt /*max()*
mbyte-IME mbyte.txt /*mbyte-IME*
mbyte-XIM mbyte.txt /*mbyte-XIM*
mbyte-combining mbyte.txt /*mbyte-combining*
mbyte-composing mbyte.txt /*mbyte-composing*
mbyte-conversion mbyte.txt /*mbyte-conversion*
mbyte-encoding mbyte.txt /*mbyte-encoding*
mbyte-first mbyte.txt /*mbyte-first*
mbyte-fonts-MSwin mbyte.txt /*mbyte-fonts-MSwin*
mbyte-fonts-X11 mbyte.txt /*mbyte-fonts-X11*
mbyte-func mbyte.txt /*mbyte-func*
mbyte-keymap mbyte.txt /*mbyte-keymap*
mbyte-locale mbyte.txt /*mbyte-locale*
mbyte-options mbyte.txt /*mbyte-options*
mbyte-terminal mbyte.txt /*mbyte-terminal*
mbyte-utf8 mbyte.txt /*mbyte-utf8*
mbyte.txt mbyte.txt /*mbyte.txt*
menu-changes-5.4 version5.txt /*menu-changes-5.4*
menu-examples gui.txt /*menu-examples*
menu-priority gui.txt /*menu-priority*
menu-separator gui.txt /*menu-separator*
menu.vim gui.txt /*menu.vim*
menus gui.txt /*menus*
merge diff.txt /*merge*
message-history message.txt /*message-history*
message.txt message.txt /*message.txt*
messages message.txt /*messages*
meta intro.txt /*meta*
method eval.txt /*method*
min() eval.txt /*min()*
missing-options vi_diff.txt /*missing-options*
mkdir() eval.txt /*mkdir()*
mlang.txt mlang.txt /*mlang.txt*
mma.vim syntax.txt /*mma.vim*
mode() eval.txt /*mode()*
mode-Ex intro.txt /*mode-Ex*
mode-cmdline cmdline.txt /*mode-cmdline*
mode-ins-repl insert.txt /*mode-ins-repl*
mode-replace insert.txt /*mode-replace*
mode-switching intro.txt /*mode-switching*
modeless-and-clipboard version6.txt /*modeless-and-clipboard*
modeless-selection gui.txt /*modeless-selection*
modeline options.txt /*modeline*
modeline-local options.txt /*modeline-local*
modeline-version options.txt /*modeline-version*
modifyOtherKeys map.txt /*modifyOtherKeys*
moo.vim syntax.txt /*moo.vim*
more-compatible version5.txt /*more-compatible*
more-prompt message.txt /*more-prompt*
more-variables eval.txt /*more-variables*
motion.txt motion.txt /*motion.txt*
mouse-mode-table term.txt /*mouse-mode-table*

mouse-overview term.txt /*mouse-overview*
mouse-swap-buttons term.txt /*mouse-swap-buttons*
mouse-using term.txt /*mouse-using*
mouse_col-variable eval.txt /*mouse_col-variable*
mouse_lnum-variable eval.txt /*mouse_lnum-variable*
mouse_win-variable eval.txt /*mouse_win-variable*
mouse_winid-variable eval.txt /*mouse_winid-variable*
movement intro.txt /*movement*
ms-dos os_msdos.txt /*ms-dos*
msdos os_msdos.txt /*msdos*
msql.vim syntax.txt /*msql.vim*
mswin.vim gui_w32.txt /*mswin.vim*
multi-byte mbyte.txt /*multi-byte*
multi-lang mlang.txt /*multi-lang*
multi-repeat repeat.txt /*multi-repeat*
multibyte mbyte.txt /*multibyte*
multibyte-ime mbyte.txt /*multibyte-ime*
multibyte-input mbyte.txt /*multibyte-input*
multilang mlang.txt /*multilang*
multilang-menus mlang.txt /*multilang-menus*
multilang-messages mlang.txt /*multilang-messages*
multilang-scripts mlang.txt /*multilang-scripts*
myfiletypefile syntax.txt /*myfiletypefile*
myscriptsfile syntax.txt /*myscriptsfile*
mysql ft_sql.txt /*mysql*
mysyntaxfile syntax.txt /*mysyntaxfile*
mysyntaxfile-add syntax.txt /*mysyntaxfile-add*
mysyntaxfile-replace syntax.txt /*mysyntaxfile-replace*
mzeval() eval.txt /*mzeval()*
mzscheme if_mzsch.txt /*mzscheme*
mzscheme-buffer if_mzsch.txt /*mzscheme-buffer*
mzscheme-commands if_mzsch.txt /*mzscheme-commands*
mzscheme-dynamic if_mzsch.txt /*mzscheme-dynamic*
mzscheme-examples if_mzsch.txt /*mzscheme-examples*
mzscheme-funcref if_mzsch.txt /*mzscheme-funcref*
mzscheme-mzeval if_mzsch.txt /*mzscheme-mzeval*
mzscheme-sandbox if_mzsch.txt /*mzscheme-sandbox*
mzscheme-setup if_mzsch.txt /*mzscheme-setup*
mzscheme-threads if_mzsch.txt /*mzscheme-threads*
mzscheme-vim if_mzsch.txt /*mzscheme-vim*
mzscheme-vimext if_mzsch.txt /*mzscheme-vimext*
mzscheme-window if_mzsch.txt /*mzscheme-window*
n pattern.txt /*n*
n1ql.vim syntax.txt /*n1ql.vim*
nasm.vim syntax.txt /*nasm.vim*
navigation motion.txt /*navigation*
nb-commands netbeans.txt /*nb-commands*
nb-events netbeans.txt /*nb-events*
nb-functions netbeans.txt /*nb-functions*
nb-messages netbeans.txt /*nb-messages*
nb-protocol_errors netbeans.txt /*nb-protocol_errors*
nb-special netbeans.txt /*nb-special*
nb-terms netbeans.txt /*nb-terms*
ncf.vim syntax.txt /*ncf.vim*

netbeans	netbeans.txt	/*netbeans*
netbeans-commands	netbeans.txt	/*netbeans-commands*
netbeans-configure	netbeans.txt	/*netbeans-configure*
netbeans-debugging	netbeans.txt	/*netbeans-debugging*
netbeans-download	netbeans.txt	/*netbeans-download*
netbeans-integration	netbeans.txt	/*netbeans-integration*
netbeans-intro	netbeans.txt	/*netbeans-intro*
netbeans-keybindings	netbeans.txt	/*netbeans-keybindings*
netbeans-messages	netbeans.txt	/*netbeans-messages*
netbeans-parameters	netbeans.txt	/*netbeans-parameters*
netbeans-preparation	netbeans.txt	/*netbeans-preparation*
netbeans-problems	netbeans.txt	/*netbeans-problems*
netbeans-protocol	netbeans.txt	/*netbeans-protocol*
netbeans-run	netbeans.txt	/*netbeans-run*
netbeans-setup	netbeans.txt	/*netbeans-setup*
netbeans-support	netbeans.txt	/*netbeans-support*
netbeans-xpm	netbeans.txt	/*netbeans-xpm*
netbeans.txt	netbeans.txt	/*netbeans.txt*
netreadfixup	pi_netrw.txt	/*netreadfixup*
netrw	pi_netrw.txt	/*netrw*
netrw-%	pi_netrw.txt	/*netrw-%*
netrw--	pi_netrw.txt	/*netrw--*
netrw-:Explore	pi_netrw.txt	/*netrw-:Explore*
netrw-:Hexplore	pi_netrw.txt	/*netrw-:Hexplore*
netrw-:Lexplore	pi_netrw.txt	/*netrw-:Lexplore*
netrw-:MF	pi_netrw.txt	/*netrw-:MF*
netrw-:MT	pi_netrw.txt	/*netrw-:MT*
netrw-:NetrwC	pi_netrw.txt	/*netrw-:NetrwC*
netrw-:NetrwMB	pi_netrw.txt	/*netrw-:NetrwMB*
netrw-:Rexplore	pi_netrw.txt	/*netrw-:Rexplore*
netrw-:Sexplore	pi_netrw.txt	/*netrw-:Sexplore*
netrw-:Texplore	pi_netrw.txt	/*netrw-:Texplore*
netrw-:Vexplore	pi_netrw.txt	/*netrw-:Vexplore*
netrw-C	pi_netrw.txt	/*netrw-C*
netrw-D	pi_netrw.txt	/*netrw-D*
netrw-I	pi_netrw.txt	/*netrw-I*
netrw-O	pi_netrw.txt	/*netrw-O*
netrw-P	pi_netrw.txt	/*netrw-P*
netrw-P18	pi_netrw.txt	/*netrw-P18*
netrw-P19	pi_netrw.txt	/*netrw-P19*
netrw-P20	pi_netrw.txt	/*netrw-P20*
netrw-P21	pi_netrw.txt	/*netrw-P21*
netrw-P22	pi_netrw.txt	/*netrw-P22*
netrw-R	pi_netrw.txt	/*netrw-R*
netrw-S	pi_netrw.txt	/*netrw-S*
netrw-Tb	pi_netrw.txt	/*netrw-Tb*
netrw-Th	pi_netrw.txt	/*netrw-Th*
netrw-U	pi_netrw.txt	/*netrw-U*
netrw-X	pi_netrw.txt	/*netrw-X*
netrw-a	pi_netrw.txt	/*netrw-a*
netrw-activate	pi_netrw.txt	/*netrw-activate*
netrw-bookmark	pi_netrw.txt	/*netrw-bookmark*
netrw-bookmarks	pi_netrw.txt	/*netrw-bookmarks*
netrw-browse	pi_netrw.txt	/*netrw-browse*

netrw-browse-cmds	pi_netrw.txt	/*netrw-browse-cmds*
netrw-browse-maps	pi_netrw.txt	/*netrw-browse-maps*
netrw-browser	pi_netrw.txt	/*netrw-browser*
netrw-browser-options	pi_netrw.txt	/*netrw-browser-options*
netrw-browser-settings	pi_netrw.txt	/*netrw-browser-settings*
netrw-browser-var	pi_netrw.txt	/*netrw-browser-var*
netrw-browsing	pi_netrw.txt	/*netrw-browsing*
netrw-c	pi_netrw.txt	/*netrw-c*
netrw-c-tab	pi_netrw.txt	/*netrw-c-tab*
netrw-cB	pi_netrw.txt	/*netrw-cB*
netrw-cadaver	pi_netrw.txt	/*netrw-cadaver*
netrw-call	pi_netrw.txt	/*netrw-call*
netrw-cb	pi_netrw.txt	/*netrw-cb*
netrw-cd	pi_netrw.txt	/*netrw-cd*
netrw-chgup	pi_netrw.txt	/*netrw-chgup*
netrw-clean	pi_netrw.txt	/*netrw-clean*
netrw-contents	pi_netrw.txt	/*netrw-contents*
netrw-copyright	pi_netrw.txt	/*netrw-copyright*
netrw-cr	pi_netrw.txt	/*netrw-cr*
netrw-createfile	pi_netrw.txt	/*netrw-createfile*
netrw-credits	pi_netrw.txt	/*netrw-credits*
netrw-ctrl-h	pi_netrw.txt	/*netrw-ctrl-h*
netrw-ctrl-l	pi_netrw.txt	/*netrw-ctrl-l*
netrw-ctrl-r	pi_netrw.txt	/*netrw-ctrl-r*
netrw-ctrl_l	pi_netrw.txt	/*netrw-ctrl_l*
netrw-curdir	pi_netrw.txt	/*netrw-curdir*
netrw-d	pi_netrw.txt	/*netrw-d*
netrw-debug	pi_netrw.txt	/*netrw-debug*
netrw-del	pi_netrw.txt	/*netrw-del*
netrw-delete	pi_netrw.txt	/*netrw-delete*
netrw-dir	pi_netrw.txt	/*netrw-dir*
netrw-dirlist	pi_netrw.txt	/*netrw-dirlist*
netrw-downdir	pi_netrw.txt	/*netrw-downdir*
netrw-edithide	pi_netrw.txt	/*netrw-edithide*
netrw-editwindow	pi_netrw.txt	/*netrw-editwindow*
netrw-enter	pi_netrw.txt	/*netrw-enter*
netrw-ex	pi_netrw.txt	/*netrw-ex*
netrw-explore	pi_netrw.txt	/*netrw-explore*
netrw-explore-cmds	pi_netrw.txt	/*netrw-explore-cmds*
netrw-expose	pi_netrw.txt	/*netrw-expose*
netrw-externapp	pi_netrw.txt	/*netrw-externapp*
netrw-file	pi_netrw.txt	/*netrw-file*
netrw-filigree	pi_netrw.txt	/*netrw-filigree*
netrw-fixup	pi_netrw.txt	/*netrw-fixup*
netrw-ftp	pi_netrw.txt	/*netrw-ftp*
netrw-ftype	pi_netrw.txt	/*netrw-ftype*
netrw-gb	pi_netrw.txt	/*netrw-gb*
netrw-gd	pi_netrw.txt	/*netrw-gd*
netrw-getftype	pi_netrw.txt	/*netrw-getftype*
netrw-gf	pi_netrw.txt	/*netrw-gf*
netrw-gh	pi_netrw.txt	/*netrw-gh*
netrw-gitignore	pi_netrw.txt	/*netrw-gitignore*
netrw-gn	pi_netrw.txt	/*netrw-gn*
netrw-gp	pi_netrw.txt	/*netrw-gp*

netrw-grep pi_netrw.txt /*netrw-grep*
netrw-gx pi_netrw.txt /*netrw-gx*
netrw-handler pi_netrw.txt /*netrw-handler*
netrw-help pi_netrw.txt /*netrw-help*
netrw-hexplore pi_netrw.txt /*netrw-hexplore*
netrw-hide pi_netrw.txt /*netrw-hide*
netrw-hiding pi_netrw.txt /*netrw-hiding*
netrw-history pi_netrw.txt /*netrw-history*
netrw-horiz pi_netrw.txt /*netrw-horiz*
netrw-i pi_netrw.txt /*netrw-i*
netrw-incompatible pi_netrw.txt /*netrw-incompatible*
netrw-internal-variables pi_netrw.txt /*netrw-internal-variables*
netrw-intro-browse pi_netrw.txt /*netrw-intro-browse*
netrw-leftmouse pi_netrw.txt /*netrw-leftmouse*
netrw-lexplore pi_netrw.txt /*netrw-lexplore*
netrw-list pi_netrw.txt /*netrw-list*
netrw-listbookmark pi_netrw.txt /*netrw-listbookmark*
netrw-listhack pi_netrw.txt /*netrw-listhack*
netrw-login pi_netrw.txt /*netrw-login*
netrw-mA pi_netrw.txt /*netrw-mA*
netrw-mB pi_netrw.txt /*netrw-mB*
netrw-mF pi_netrw.txt /*netrw-mF*
netrw-mT pi_netrw.txt /*netrw-mT*
netrw-mX pi_netrw.txt /*netrw-mX*
netrw-ma pi_netrw.txt /*netrw-ma*
netrw-mb pi_netrw.txt /*netrw-mb*
netrw-mc pi_netrw.txt /*netrw-mc*
netrw-md pi_netrw.txt /*netrw-md*
netrw-me pi_netrw.txt /*netrw-me*
netrw-mf pi_netrw.txt /*netrw-mf*
netrw-mg pi_netrw.txt /*netrw-mg*
netrw-mh pi_netrw.txt /*netrw-mh*
netrw-middlemouse pi_netrw.txt /*netrw-middlemouse*
netrw-ml_get pi_netrw.txt /*netrw-ml_get*
netrw-mm pi_netrw.txt /*netrw-mm*
netrw-modify pi_netrw.txt /*netrw-modify*
netrw-mouse pi_netrw.txt /*netrw-mouse*
netrw-move pi_netrw.txt /*netrw-move*
netrw-mp pi_netrw.txt /*netrw-mp*
netrw-mr pi_netrw.txt /*netrw-mr*
netrw-ms pi_netrw.txt /*netrw-ms*
netrw-mt pi_netrw.txt /*netrw-mt*
netrw-mu pi_netrw.txt /*netrw-mu*
netrw-mv pi_netrw.txt /*netrw-mv*
netrw-mx pi_netrw.txt /*netrw-mx*
netrw-mz pi_netrw.txt /*netrw-mz*
netrw-netrc pi_netrw.txt /*netrw-netrc*
netrw-newfile pi_netrw.txt /*netrw-newfile*
netrw-nexplore pi_netrw.txt /*netrw-nexplore*
netrw-noload pi_netrw.txt /*netrw-noload*
netrw-nread pi_netrw.txt /*netrw-nread*
netrw-ntree pi_netrw.txt /*netrw-ntree*
netrw-nwrite pi_netrw.txt /*netrw-nwrite*
netrw-o pi_netrw.txt /*netrw-o*

netrw-obtain	pi_netrw.txt	/*netrw-obtain*
netrw-options	pi_netrw.txt	/*netrw-options*
netrw-p	pi_netrw.txt	/*netrw-p*
netrw-p1	pi_netrw.txt	/*netrw-p1*
netrw-p10	pi_netrw.txt	/*netrw-p10*
netrw-p11	pi_netrw.txt	/*netrw-p11*
netrw-p12	pi_netrw.txt	/*netrw-p12*
netrw-p13	pi_netrw.txt	/*netrw-p13*
netrw-p14	pi_netrw.txt	/*netrw-p14*
netrw-p15	pi_netrw.txt	/*netrw-p15*
netrw-p16	pi_netrw.txt	/*netrw-p16*
netrw-p17	pi_netrw.txt	/*netrw-p17*
netrw-p2	pi_netrw.txt	/*netrw-p2*
netrw-p3	pi_netrw.txt	/*netrw-p3*
netrw-p4	pi_netrw.txt	/*netrw-p4*
netrw-p5	pi_netrw.txt	/*netrw-p5*
netrw-p6	pi_netrw.txt	/*netrw-p6*
netrw-p7	pi_netrw.txt	/*netrw-p7*
netrw-p8	pi_netrw.txt	/*netrw-p8*
netrw-p9	pi_netrw.txt	/*netrw-p9*
netrw-passwd	pi_netrw.txt	/*netrw-passwd*
netrw-password	pi_netrw.txt	/*netrw-password*
netrw-path	pi_netrw.txt	/*netrw-path*
netrw-pexplore	pi_netrw.txt	/*netrw-pexplore*
netrw-preview	pi_netrw.txt	/*netrw-preview*
netrw-problems	pi_netrw.txt	/*netrw-problems*
netrw-protocol	pi_netrw.txt	/*netrw-protocol*
netrw-prvwin	pi_netrw.txt	/*netrw-prvwin*
netrw-pscp	pi_netrw.txt	/*netrw-pscp*
netrw-psftp	pi_netrw.txt	/*netrw-psftp*
netrw-putty	pi_netrw.txt	/*netrw-putty*
netrw-qF	pi_netrw.txt	/*netrw-qF*
netrw-qL	pi_netrw.txt	/*netrw-qL*
netrw-qb	pi_netrw.txt	/*netrw-qb*
netrw-qf	pi_netrw.txt	/*netrw-qf*
netrw-quickcom	pi_netrw.txt	/*netrw-quickcom*
netrw-quickcoms	pi_netrw.txt	/*netrw-quickcoms*
netrw-quickhelp	pi_netrw.txt	/*netrw-quickhelp*
netrw-quickmap	pi_netrw.txt	/*netrw-quickmap*
netrw-quickmaps	pi_netrw.txt	/*netrw-quickmaps*
netrw-r	pi_netrw.txt	/*netrw-r*
netrw-read	pi_netrw.txt	/*netrw-read*
netrw-ref	pi_netrw.txt	/*netrw-ref*
netrw-refresh	pi_netrw.txt	/*netrw-refresh*
netrw-rename	pi_netrw.txt	/*netrw-rename*
netrw-reverse	pi_netrw.txt	/*netrw-reverse*
netrw-rexplore	pi_netrw.txt	/*netrw-rexplore*
netrw-rightmouse	pi_netrw.txt	/*netrw-rightmouse*
netrw-s	pi_netrw.txt	/*netrw-s*
netrw-s-cr	pi_netrw.txt	/*netrw-s-cr*
netrw-settings	pi_netrw.txt	/*netrw-settings*
netrw-settings-window	pi_netrw.txt	/*netrw-settings-window*
netrw-sexplore	pi_netrw.txt	/*netrw-sexplore*
netrw-sort	pi_netrw.txt	/*netrw-sort*

netrw-sort-sequence	pi_netrw.txt	/*netrw-sort-sequence*
netrw-sortsequence	pi_netrw.txt	/*netrw-sortsequence*
netrw-source	pi_netrw.txt	/*netrw-source*
netrw-ssh-hack	pi_netrw.txt	/*netrw-ssh-hack*
netrw-star	pi_netrw.txt	/*netrw-star*
netrw-starpat	pi_netrw.txt	/*netrw-starpat*
netrw-starstar	pi_netrw.txt	/*netrw-starstar*
netrw-starstarpat	pi_netrw.txt	/*netrw-starstarpat*
netrw-start	pi_netrw.txt	/*netrw-start*
netrw-t	pi_netrw.txt	/*netrw-t*
netrw-texplore	pi_netrw.txt	/*netrw-texplore*
netrw-todo	pi_netrw.txt	/*netrw-todo*
netrw-trailingslash	pi_netrw.txt	/*netrw-trailingslash*
netrw-transparent	pi_netrw.txt	/*netrw-transparent*
netrw-u	pi_netrw.txt	/*netrw-u*
netrw-updir	pi_netrw.txt	/*netrw-updir*
netrw-urls	pi_netrw.txt	/*netrw-urls*
netrw-usermaps	pi_netrw.txt	/*netrw-usermaps*
netrw-userpass	pi_netrw.txt	/*netrw-userpass*
netrw-v	pi_netrw.txt	/*netrw-v*
netrw-var	pi_netrw.txt	/*netrw-var*
netrw-variables	pi_netrw.txt	/*netrw-variables*
netrw-vexplore	pi_netrw.txt	/*netrw-vexplore*
netrw-windows-netrc	pi_netrw.txt	/*netrw-windows-netrc*
netrw-windows-s	pi_netrw.txt	/*netrw-windows-s*
netrw-write	pi_netrw.txt	/*netrw-write*
netrw-x	pi_netrw.txt	/*netrw-x*
netrw-xfer	pi_netrw.txt	/*netrw-xfer*
netrw.vim	pi_netrw.txt	/*netrw.vim*
netrw_filehandler	pi_netrw.txt	/*netrw_filehandler*
netterm-mouse	options.txt	/*netterm-mouse*
network	pi_netrw.txt	/*network*
new-5	version5.txt	/*new-5*
new-6	version6.txt	/*new-6*
new-7	version7.txt	/*new-7*
new-8	version8.txt	/*new-8*
new-GTK-GUI	version5.txt	/*new-GTK-GUI*
new-MzScheme	version7.txt	/*new-MzScheme*
new-Select-mode	version5.txt	/*new-Select-mode*
new-View	version6.txt	/*new-View*
new-argument-list	version6.txt	/*new-argument-list*
new-buftype	version6.txt	/*new-buftype*
new-cmdwin	version6.txt	/*new-cmdwin*
new-color-schemes	version6.txt	/*new-color-schemes*
new-commands	version5.txt	/*new-commands*
new-commands-5.4	version5.txt	/*new-commands-5.4*
new-conceal	version7.txt	/*new-conceal*
new-debug-itf	version6.txt	/*new-debug-itf*
new-debug-mode	version6.txt	/*new-debug-mode*
new-debug-support	version7.txt	/*new-debug-support*
new-define-operator	version7.txt	/*new-define-operator*
new-diff-mode	version6.txt	/*new-diff-mode*
new-encryption	version5.txt	/*new-encryption*
new-evim	version6.txt	/*new-evim*

new-ex-commands-5.2 version5.txt /*new-ex-commands-5.2*
new-file-browser version6.txt /*new-file-browser*
new-file-writing version6.txt /*new-file-writing*
new-filetype filetype.txt /*new-filetype*
new-filetype-5.4 version5.txt /*new-filetype-5.4*
new-filetype-plugins version6.txt /*new-filetype-plugins*
new-filetype-scripts filetype.txt /*new-filetype-scripts*
new-folding version6.txt /*new-folding*
new-functions-5.2 version5.txt /*new-functions-5.2*
new-global-values version6.txt /*new-global-values*
new-highlighting version5.txt /*new-highlighting*
new-indent-flex version6.txt /*new-indent-flex*
new-items-6 version6.txt /*new-items-6*
new-items-7 version7.txt /*new-items-7*
new-items-8 version8.txt /*new-items-8*
new-line-continuation version5.txt /*new-line-continuation*
new-location-list version7.txt /*new-location-list*
new-lua version7.txt /*new-lua*
new-manpage-trans version7.txt /*new-manpage-trans*
new-map-expression version7.txt /*new-map-expression*
new-map-select version7.txt /*new-map-select*
new-more-encryption version7.txt /*new-more-encryption*
new-more-highlighting version7.txt /*new-more-highlighting*
new-more-unicode version7.txt /*new-more-unicode*
new-multi-byte version5.txt /*new-multi-byte*
new-multi-lang version6.txt /*new-multi-lang*
new-netrw-explore version7.txt /*new-netrw-explore*
new-network-files version6.txt /*new-network-files*
new-omni-completion version7.txt /*new-omni-completion*
new-onemore version7.txt /*new-onemore*
new-operator-mod version6.txt /*new-operator-mod*
new-options-5.2 version5.txt /*new-options-5.2*
new-options-5.4 version5.txt /*new-options-5.4*
new-other-8.2 version8.txt /*new-other-8.2*
new-perl-python version5.txt /*new-perl-python*
new-persistent-undo version7.txt /*new-persistent-undo*
new-plugins version6.txt /*new-plugins*
new-popup-window version8.txt /*new-popup-window*
new-posix version7.txt /*new-posix*
new-print-multi-byte version7.txt /*new-print-multi-byte*
new-printing version6.txt /*new-printing*
new-python3 version7.txt /*new-python3*
new-regexp-engine version7.txt /*new-regexp-engine*
new-runtime-dir version5.txt /*new-runtime-dir*
new-script version5.txt /*new-script*
new-script-5.4 version5.txt /*new-script-5.4*
new-scroll-back version7.txt /*new-scroll-back*
new-search-path version6.txt /*new-search-path*
new-searchpat version6.txt /*new-searchpat*
new-session-files version5.txt /*new-session-files*
new-spell version7.txt /*new-spell*
new-style-testing testing.txt /*new-style-testing*
new-tab-pages version7.txt /*new-tab-pages*
new-terminal-window version8.txt /*new-terminal-window*

new-text-properties version8.txt /*new-text-properties*
new-undo-branches version7.txt /*new-undo-branches*
new-unlisted-buffers version6.txt /*new-unlisted-buffers*
new-user-defined version5.txt /*new-user-defined*
new-user-manual version6.txt /*new-user-manual*
new-utf-8 version6.txt /*new-utf-8*
new-vertsplit version6.txt /*new-vertsplit*
new-vim-script version7.txt /*new-vim-script*
new-vim-script-8 version8.txt /*new-vim-script-8*
new-vim-server version6.txt /*new-vim-server*
new-vimgrep version7.txt /*new-vimgrep*
new-vimscript-8.2 version8.txt /*new-vimscript-8.2*
new-virtedit version6.txt /*new-virtedit*
news intro.txt /*news*
nextnonblank() eval.txt /*nextnonblank()*
no-eval-feature eval.txt /*no-eval-feature*
no-type-checking eval.txt /*no-type-checking*
no_buffers_menu gui.txt /*no_buffers_menu*
no_mail_maps filetype.txt /*no_mail_maps*
no_plugin_maps filetype.txt /*no_plugin_maps*
nocombine syntax.txt /*nocombine*
non-greedy pattern.txt /*non-greedy*
non-zero-arg eval.txt /*non-zero-arg*
none-variable eval.txt /*none-variable*
normal-index index.txt /*normal-index*
not-compatible usr_01.txt /*not-compatible*
not-edited editing.txt /*not-edited*
notation intro.txt /*notation*
notepad gui_w32.txt /*notepad*
nr2char() eval.txt /*nr2char()*
nroff.vim syntax.txt /*nroff.vim*
null-variable eval.txt /*null-variable*
number_relativenumber options.txt /*number_relativenumber*
numbered-function eval.txt /*numbered-function*
o insert.txt /*o*
o_CTRL-V motion.txt /*o_CTRL-V*
o_V motion.txt /*o_V*
o_v motion.txt /*o_v*
object-motions motion.txt /*object-motions*
object-select motion.txt /*object-select*
objects index.txt /*objects*
obtaining-exted netbeans.txt /*obtaining-exted*
ocaml.vim syntax.txt /*ocaml.vim*
octal eval.txt /*octal*
octal-nrformats options.txt /*octal-nrformats*
octal-number eval.txt /*octal-number*
old-style-testing testing.txt /*old-style-testing*
oldfiles-variable eval.txt /*oldfiles-variable*
ole-activation if_ole.txt /*ole-activation*
ole-eval if_ole.txt /*ole-eval*
ole-gethwnd if_ole.txt /*ole-gethwnd*
ole-interface if_ole.txt /*ole-interface*
ole-methods if_ole.txt /*ole-methods*
ole-normal if_ole.txt /*ole-normal*

ole-registration if_ole.txt /*ole-registration*
ole-sendkeys if_ole.txt /*ole-sendkeys*
ole-setforeground if_ole.txt /*ole-setforeground*
omap-info map.txt /*omap-info*
omni-sql-completion ft_sql.txt /*omni-sql-completion*
online-help helphelp.txt /*online-help*
opening-window windows.txt /*opening-window*
operator motion.txt /*operator*
operator-pending-index index.txt /*operator-pending-index*
operator-variable eval.txt /*operator-variable*
option-backslash options.txt /*option-backslash*
option-list quickref.txt /*option-list*
option-summary options.txt /*option-summary*
option-window options.txt /*option-window*
option_restore() todo.txt /*option_restore()*
option_save() todo.txt /*option_save()*
optional-function-argument eval.txt /*optional-function-argument*
options options.txt /*options*
options-changed version5.txt /*options-changed*
options-in-terminal terminal.txt /*options-in-terminal*
options.txt options.txt /*options.txt*
optwin options.txt /*optwin*
or() eval.txt /*or()*
oracle ft_sql.txt /*oracle*
os2 os_os2.txt /*os2*
os390 os_390.txt /*os390*
os_390.txt os_390.txt /*os_390.txt*
os_amiga.txt os_amiga.txt /*os_amiga.txt*
os_beos.txt os_beos.txt /*os_beos.txt*
os_dos.txt os_dos.txt /*os_dos.txt*
os_mac.txt os_mac.txt /*os_mac.txt*
os_mint.txt os_mint.txt /*os_mint.txt*
os_msdos.txt os_msdos.txt /*os_msdos.txt*
os_os2.txt os_os2.txt /*os_os2.txt*
os_qnx.txt os_qnx.txt /*os_qnx.txt*
os_risc.txt os_risc.txt /*os_risc.txt*
os_unix.txt os_unix.txt /*os_unix.txt*
os_vms.txt os_vms.txt /*os_vms.txt*
os_win32.txt os_win32.txt /*os_win32.txt*
other-features vi_diff.txt /*other-features*
out_buf channel.txt /*out_buf*
out_cb channel.txt /*out_cb*
out_io-buffer channel.txt /*out_io-buffer*
out_mode channel.txt /*out_mode*
out_modifiable channel.txt /*out_modifiable*
out_msg channel.txt /*out_msg*
out_name channel.txt /*out_name*
out_timeout channel.txt /*out_timeout*
p change.txt /*p*
pack-add repeat.txt /*pack-add*
package-create repeat.txt /*package-create*
packages repeat.txt /*packages*
packload-two-steps repeat.txt /*packload-two-steps*
page-down intro.txt /*page-down*

page-up intro.txt /*page-up*
page_down intro.txt /*page_down*
page_up intro.txt /*page_up*
pager message.txt /*pager*
papp.vim syntax.txt /*papp.vim*
paragraph motion.txt /*paragraph*
pascal.vim syntax.txt /*pascal.vim*
patches-8 version8.txt /*patches-8*
patches-8.1 version8.txt /*patches-8.1*
patches-8.2 version8.txt /*patches-8.2*
pathshorten() eval.txt /*pathshorten()*
pattern pattern.txt /*pattern*
pattern-atoms pattern.txt /*pattern-atoms*
pattern-multi-byte pattern.txt /*pattern-multi-byte*
pattern-multi-items pattern.txt /*pattern-multi-items*
pattern-overview pattern.txt /*pattern-overview*
pattern-searches pattern.txt /*pattern-searches*
pattern.txt pattern.txt /*pattern.txt*
patterns-composing pattern.txt /*patterns-composing*
pdev-option print.txt /*pdev-option*
peace intro.txt /*peace*
penc-option print.txt /*penc-option*
perl if_perl.txt /*perl*
perl-Append if_perl.txt /*perl-Append*
perl-Blob if_perl.txt /*perl-Blob*
perl-Buffer if_perl.txt /*perl-Buffer*
perl-Buffers if_perl.txt /*perl-Buffers*
perl-Count if_perl.txt /*perl-Count*
perl-Delete if_perl.txt /*perl-Delete*
perl-DoCommand if_perl.txt /*perl-DoCommand*
perl-Eval if_perl.txt /*perl-Eval*
perl-Get if_perl.txt /*perl-Get*
perl-GetCursor if_perl.txt /*perl-GetCursor*
perl-Msg if_perl.txt /*perl-Msg*
perl-Name if_perl.txt /*perl-Name*
perl-Number if_perl.txt /*perl-Number*
perl-Set if_perl.txt /*perl-Set*
perl-SetHeight if_perl.txt /*perl-SetHeight*
perl-SetOption if_perl.txt /*perl-SetOption*
perl-Windows if_perl.txt /*perl-Windows*
perl-compiling if_perl.txt /*perl-compiling*
perl-dynamic if_perl.txt /*perl-dynamic*
perl-editing if_perl.txt /*perl-editing*
perl-overview if_perl.txt /*perl-overview*
perl-patterns pattern.txt /*perl-patterns*
perl-using if_perl.txt /*perl-using*
perl.vim syntax.txt /*perl.vim*
perleval() eval.txt /*perleval()*
persistent-undo undo.txt /*persistent-undo*
pexpr-option print.txt /*pexpr-option*
pfn-option print.txt /*pfn-option*
pheader-option print.txt /*pheader-option*
photon-fonts os_qnx.txt /*photon-fonts*
photon-gui os_qnx.txt /*photon-gui*

php-comment	indent.txt	/*php-comment*
php-indent	indent.txt	/*php-indent*
php-indenting	indent.txt	/*php-indenting*
php.vim	syntax.txt	/*php.vim*
php3.vim	syntax.txt	/*php3.vim*
phtml.vim	syntax.txt	/*phtml.vim*
pi_getscript.txt	pi_getscript.txt	/*pi_getscript.txt*
pi_gzip.txt	pi_gzip.txt	/*pi_gzip.txt*
pi_logipat.txt	pi_logipat.txt	/*pi_logipat.txt*
pi_netrw.txt	pi_netrw.txt	/*pi_netrw.txt*
pi_paren.txt	pi_paren.txt	/*pi_paren.txt*
pi_spec.txt	pi_spec.txt	/*pi_spec.txt*
pi_tar.txt	pi_tar.txt	/*pi_tar.txt*
pi_vimball.txt	pi_vimball.txt	/*pi_vimball.txt*
pi_zip.txt	pi_zip.txt	/*pi_zip.txt*
pkzip	options.txt	/*pkzip*
plaintex.vim	syntax.txt	/*plaintex.vim*
plsql	ft_sql.txt	/*plsql*
plugin	usr_05.txt	/*plugin*
plugin-details	filetype.txt	/*plugin-details*
plugin-filetype	usr_41.txt	/*plugin-filetype*
plugin-special	usr_41.txt	/*plugin-special*
pmbcs-option	print.txt	/*pmbcs-option*
pmbfn-option	print.txt	/*pmbfn-option*
popt-option	print.txt	/*popt-option*
popup	popup.txt	/*popup*
popup-buffer	popup.txt	/*popup-buffer*
popup-callback	popup.txt	/*popup-callback*
popup-close	popup.txt	/*popup-close*
popup-examples	popup.txt	/*popup-examples*
popup-filter	popup.txt	/*popup-filter*
popup-filter-mode	popup.txt	/*popup-filter-mode*
popup-function-details	popup.txt	/*popup-function-details*
popup-functions	popup.txt	/*popup-functions*
popup-intro	popup.txt	/*popup-intro*
popup-mapping	popup.txt	/*popup-mapping*
popup-mask	popup.txt	/*popup-mask*
popup-menu	gui.txt	/*popup-menu*
popup-menu-added	version5.txt	/*popup-menu-added*
popup-position	popup.txt	/*popup-position*
popup-props	popup.txt	/*popup-props*
popup-scrollbar	popup.txt	/*popup-scrollbar*
popup-textprop-pos	popup.txt	/*popup-textprop-pos*
popup-usage	popup.txt	/*popup-usage*
popup-window	popup.txt	/*popup-window*
popup-window-functions	usr_41.txt	/*popup-window-functions*
popup.txt	popup.txt	/*popup.txt*
popup_atcursor()	popup.txt	/*popup_atcursor()*
popup_beval()	popup.txt	/*popup_beval()*
popup_beval_example	popup.txt	/*popup_beval_example*
popup_clear()	popup.txt	/*popup_clear()*
popup_close()	popup.txt	/*popup_close()*
popup_create()	popup.txt	/*popup_create()*
popup_create-arguments	popup.txt	/*popup_create-arguments*

popup_dialog() popup.txt /*popup_dialog()*
popup_dialog-example popup.txt /*popup_dialog-example*
popup_filter_menu() popup.txt /*popup_filter_menu()*
popup_filter_yesno() popup.txt /*popup_filter_yesno()*
popup_findinfo() popup.txt /*popup_findinfo()*
popup_findpreview() popup.txt /*popup_findpreview()*
popup_getoptions() popup.txt /*popup_getoptions()*
popup_getpos() popup.txt /*popup_getpos()*
popup_hide() popup.txt /*popup_hide()*
popup_locate() popup.txt /*popup_locate()*
popup_menu() popup.txt /*popup_menu()*
popup_menu-shortcut-example popup.txt /*popup_menu-shortcut-example*
popup_move() popup.txt /*popup_move()*
popup_notification() popup.txt /*popup_notification()*
popup_setoptions() popup.txt /*popup_setoptions()*
popup_settext() popup.txt /*popup_settext()*
popup_show() popup.txt /*popup_show()*
popupmenu-completion insert.txt /*popupmenu-completion*
popupmenu-keys insert.txt /*popupmenu-keys*
popupwin popup.txt /*popupwin*
ports-5.2 version5.txt /*ports-5.2*
ports-6 version6.txt /*ports-6*
posix vi_diff.txt /*posix*
posix-compliance vi_diff.txt /*posix-compliance*
posix-screen-size vi_diff.txt /*posix-screen-size*
postgresql ft_sql.txt /*postgresql*
postscr.vim syntax.txt /*postscr.vim*
postscript-cjk-printing print.txt /*postscript-cjk-printing*
postscript-print-encoding print.txt /*postscript-print-encoding*
postscript-print-trouble print.txt /*postscript-print-trouble*
postscript-print-util print.txt /*postscript-print-util*
postscript-printing print.txt /*postscript-printing*
pow() eval.txt /*pow()*
ppwiz.vim syntax.txt /*ppwiz.vim*
press-enter message.txt /*press-enter*
press-return message.txt /*press-return*
prevcount-variable eval.txt /*prevcount-variable*
preview-popup windows.txt /*preview-popup*
preview-window windows.txt /*preview-window*
prevnonblank() eval.txt /*prevnonblank()*
print-intro print.txt /*print-intro*
print-options print.txt /*print-options*
print.txt print.txt /*print.txt*
printf() eval.txt /*printf()*
printf-% eval.txt /*printf-%*
printf-B eval.txt /*printf-B*
printf-E eval.txt /*printf-E*
printf-G eval.txt /*printf-G*
printf-S eval.txt /*printf-S*
printf-X eval.txt /*printf-X*
printf-b eval.txt /*printf-b*
printf-c eval.txt /*printf-c*
printf-d eval.txt /*printf-d*
printf-e eval.txt /*printf-e*

```
printf-f                    eval.txt                /*printf-f*
printf-g                    eval.txt                /*printf-g*
printf-o                    eval.txt                /*printf-o*
printf-s                    eval.txt                /*printf-s*
printf-x                    eval.txt                /*printf-x*
printing                    print.txt               /*printing*
printing-formfeed           print.txt               /*printing-formfeed*
profile                     repeat.txt              /*profile*
profiling                   repeat.txt              /*profiling*
profiling-variable          eval.txt                /*profiling-variable*
progname-variable           eval.txt                /*progname-variable*
progpath-variable           eval.txt                /*progpath-variable*
progress.vim                syntax.txt              /*progress.vim*
prompt-buffer               channel.txt             /*prompt-buffer*
prompt_setcallback()        eval.txt                /*prompt_setcallback()*
prompt_setinterrupt()       eval.txt                /*prompt_setinterrupt()*
prompt_setprompt()          eval.txt                /*prompt_setprompt()*
promptbuffer-functions      usr_41.txt              /*promptbuffer-functions*
pronounce                   intro.txt               /*pronounce*
prop_add()                  textprop.txt            /*prop_add()*
prop_clear()                textprop.txt            /*prop_clear()*
prop_find()                 textprop.txt            /*prop_find()*
prop_list()                 textprop.txt            /*prop_list()*
prop_remove()               textprop.txt            /*prop_remove()*
prop_type_add()             textprop.txt            /*prop_type_add()*
prop_type_change()          textprop.txt            /*prop_type_change()*
prop_type_delete()          textprop.txt            /*prop_type_delete()*
prop_type_get()             textprop.txt            /*prop_type_get()*
prop_type_list()            textprop.txt            /*prop_type_list()*
psql                        ft_sql.txt              /*psql*
ptcap.vim                   syntax.txt              /*ptcap.vim*
pterm-mouse                 options.txt             /*pterm-mouse*
pum_getpos()                eval.txt                /*pum_getpos()*
pumvisible()                eval.txt                /*pumvisible()*
put                         change.txt              /*put*
put-Visual-mode             change.txt              /*put-Visual-mode*
py3eval()                   eval.txt                /*py3eval()*
pyeval()                    eval.txt                /*pyeval()*
python                      if_pyth.txt             /*python*
python-.locked              if_pyth.txt             /*python-.locked*
python-2-and-3              if_pyth.txt             /*python-2-and-3*
python-Dictionary           if_pyth.txt             /*python-Dictionary*
python-Function             if_pyth.txt             /*python-Function*
python-List                 if_pyth.txt             /*python-List*
python-VIM_SPECIAL_PATH     if_pyth.txt             /*python-VIM_SPECIAL_PATH*
python-_get_paths           if_pyth.txt             /*python-_get_paths*
python-bindeval             if_pyth.txt             /*python-bindeval*
python-bindeval-objects     if_pyth.txt             /*python-bindeval-objects*
python-buffer               if_pyth.txt             /*python-buffer*
python-buffers              if_pyth.txt             /*python-buffers*
python-building             if_pyth.txt             /*python-building*
python-chdir                if_pyth.txt             /*python-chdir*
python-command              if_pyth.txt             /*python-command*
python-commands             if_pyth.txt             /*python-commands*
```

python-current	if_pyth.txt	/*python-current*
python-dynamic	if_pyth.txt	/*python-dynamic*
python-error	if_pyth.txt	/*python-error*
python-eval	if_pyth.txt	/*python-eval*
python-examples	if_pyth.txt	/*python-examples*
python-fchdir	if_pyth.txt	/*python-fchdir*
python-find_module	if_pyth.txt	/*python-find_module*
python-foreach_rtp	if_pyth.txt	/*python-foreach_rtp*
python-input	if_pyth.txt	/*python-input*
python-options	if_pyth.txt	/*python-options*
python-output	if_pyth.txt	/*python-output*
python-path_hook	if_pyth.txt	/*python-path_hook*
python-pyeval	if_pyth.txt	/*python-pyeval*
python-range	if_pyth.txt	/*python-range*
python-special-path	if_pyth.txt	/*python-special-path*
python-strwidth	if_pyth.txt	/*python-strwidth*
python-tabpage	if_pyth.txt	/*python-tabpage*
python-tabpages	if_pyth.txt	/*python-tabpages*
python-vars	if_pyth.txt	/*python-vars*
python-vim	if_pyth.txt	/*python-vim*
python-vvars	if_pyth.txt	/*python-vvars*
python-window	if_pyth.txt	/*python-window*
python-windows	if_pyth.txt	/*python-windows*
python.vim	syntax.txt	/*python.vim*
python2-directory	if_pyth.txt	/*python2-directory*
python3	if_pyth.txt	/*python3*
python3-directory	if_pyth.txt	/*python3-directory*
python_x	if_pyth.txt	/*python_x*
python_x-special-comments	if_pyth.txt	/*python_x-special-comments*
pythonx	if_pyth.txt	/*pythonx*
pythonx-directory	if_pyth.txt	/*pythonx-directory*
pyxeval()	eval.txt	/*pyxeval()*
q	repeat.txt	/*q*
q/	cmdline.txt	/*q\/*
q:	cmdline.txt	/*q:*
q?	cmdline.txt	/*q?*
qf.vim	filetype.txt	/*qf.vim*
qnx	os_qnx.txt	/*qnx*
qnx-compiling	os_qnx.txt	/*qnx-compiling*
qnx-general	os_qnx.txt	/*qnx-general*
qnx-terminal	os_qnx.txt	/*qnx-terminal*
quake.vim	syntax.txt	/*quake.vim*
quickfix	quickfix.txt	/*quickfix*
quickfix-6	version6.txt	/*quickfix-6*
quickfix-ID	quickfix.txt	/*quickfix-ID*
quickfix-buffer	quickfix.txt	/*quickfix-buffer*
quickfix-changedtick	quickfix.txt	/*quickfix-changedtick*
quickfix-context	quickfix.txt	/*quickfix-context*
quickfix-directory-stack	quickfix.txt	/*quickfix-directory-stack*
quickfix-error-lists	quickfix.txt	/*quickfix-error-lists*
quickfix-functions	usr_41.txt	/*quickfix-functions*
quickfix-gcc	quickfix.txt	/*quickfix-gcc*
quickfix-index	quickfix.txt	/*quickfix-index*
quickfix-manx	quickfix.txt	/*quickfix-manx*

quickfix-parse quickfix.txt /*quickfix-parse*
quickfix-perl quickfix.txt /*quickfix-perl*
quickfix-size quickfix.txt /*quickfix-size*
quickfix-title quickfix.txt /*quickfix-title*
quickfix-valid quickfix.txt /*quickfix-valid*
quickfix-window quickfix.txt /*quickfix-window*
quickfix-window-ID quickfix.txt /*quickfix-window-ID*
quickfix.txt quickfix.txt /*quickfix.txt*
quickref quickref.txt /*quickref*
quickref.txt quickref.txt /*quickref.txt*
quote change.txt /*quote*
quote# change.txt /*quote#*
quote% change.txt /*quote%*
quote+ gui_x11.txt /*quote+*
quote- change.txt /*quote-*
quote. change.txt /*quote.*
quote/ change.txt /*quote\/*
quote0 change.txt /*quote0*
quote1 change.txt /*quote1*
quote2 change.txt /*quote2*
quote3 change.txt /*quote3*
quote4 change.txt /*quote4*
quote9 change.txt /*quote9*
quote: change.txt /*quote:*
quote= change.txt /*quote=*
quote_ change.txt /*quote_*
quote_# change.txt /*quote_#*
quote_% change.txt /*quote_%*
quote_- change.txt /*quote_-*
quote_. change.txt /*quote_.*
quote_/ change.txt /*quote_\/*
quote_: change.txt /*quote_:*
quote_= change.txt /*quote_=*
quote_alpha change.txt /*quote_alpha*
quote_number change.txt /*quote_number*
quote_quote change.txt /*quote_quote*
quote_~ change.txt /*quote_~*
quotea change.txt /*quotea*
quotecommandquote intro.txt /*quotecommandquote*
quoteplus gui_x11.txt /*quoteplus*
quotequote change.txt /*quotequote*
quotes quotes.txt /*quotes*
quotes.txt quotes.txt /*quotes.txt*
quotestar gui.txt /*quotestar*
quote~ change.txt /*quote~*
r change.txt /*r*
r.vim syntax.txt /*r.vim*
rand() eval.txt /*rand()*
range() eval.txt /*range()*
raw-terminal-mode term.txt /*raw-terminal-mode*
rcp pi_netrw.txt /*rcp*
read-in-close-cb channel.txt /*read-in-close-cb*
read-messages insert.txt /*read-messages*
read-only-share editing.txt /*read-only-share*

read-stdin	version5.txt	/*read-stdin*
readdir()	eval.txt	/*readdir()*
readfile()	eval.txt	/*readfile()*
readline.vim	syntax.txt	/*readline.vim*
recording	repeat.txt	/*recording*
recover.txt	recover.txt	/*recover.txt*
recovery	recover.txt	/*recovery*
recursive_mapping	map.txt	/*recursive_mapping*
redo	undo.txt	/*redo*
redo-register	undo.txt	/*redo-register*
ref	intro.txt	/*ref*
reference	intro.txt	/*reference*
reference_toc	help.txt	/*reference_toc*
reg_executing()	eval.txt	/*reg_executing()*
reg_recording()	eval.txt	/*reg_recording()*
regexp	pattern.txt	/*regexp*
regexp-changes-5.4	version5.txt	/*regexp-changes-5.4*
register	sponsor.txt	/*register*
register-faq	sponsor.txt	/*register-faq*
register-variable	eval.txt	/*register-variable*
registers	change.txt	/*registers*
rego.vim	syntax.txt	/*rego.vim*
regular-expression	pattern.txt	/*regular-expression*
reload	editing.txt	/*reload*
reltime()	eval.txt	/*reltime()*
reltimefloat()	eval.txt	/*reltimefloat()*
reltimestr()	eval.txt	/*reltimestr()*
remote.txt	remote.txt	/*remote.txt*
remote_expr()	eval.txt	/*remote_expr()*
remote_foreground()	eval.txt	/*remote_foreground()*
remote_peek()	eval.txt	/*remote_peek()*
remote_read()	eval.txt	/*remote_read()*
remote_send()	eval.txt	/*remote_send()*
remote_startserver()	eval.txt	/*remote_startserver()*
remove()	eval.txt	/*remove()*
remove-filetype	filetype.txt	/*remove-filetype*
remove-option-flags	options.txt	/*remove-option-flags*
rename()	eval.txt	/*rename()*
rename-files	tips.txt	/*rename-files*
repeat()	eval.txt	/*repeat()*
repeat.txt	repeat.txt	/*repeat.txt*
repeating	repeat.txt	/*repeating*
replacing	change.txt	/*replacing*
replacing-ex	insert.txt	/*replacing-ex*
reselect-Visual	visual.txt	/*reselect-Visual*
resolve()	eval.txt	/*resolve()*
restore-cursor	usr_05.txt	/*restore-cursor*
restore-position	tips.txt	/*restore-position*
restricted-mode	starting.txt	/*restricted-mode*
retab-example	change.txt	/*retab-example*
rethrow	eval.txt	/*rethrow*
reverse()	eval.txt	/*reverse()*
rexx.vim	syntax.txt	/*rexx.vim*
rgb.txt	gui_w32.txt	/*rgb.txt*

rgview starting.txt /*rgview*
rgvim starting.txt /*rgvim*
right-justify change.txt /*right-justify*
rileft rileft.txt /*rileft*
rileft.txt rileft.txt /*rileft.txt*
riscos os_risc.txt /*riscos*
rmd.vim syntax.txt /*rmd.vim*
rot13 change.txt /*rot13*
round() eval.txt /*round()*
rrst.vim syntax.txt /*rrst.vim*
rst.vim syntax.txt /*rst.vim*
rsync pi_netrw.txt /*rsync*
ruby if_ruby.txt /*ruby*
ruby-blob if_ruby.txt /*ruby-blob*
ruby-buffer if_ruby.txt /*ruby-buffer*
ruby-command if_ruby.txt /*ruby-command*
ruby-commands if_ruby.txt /*ruby-commands*
ruby-dynamic if_ruby.txt /*ruby-dynamic*
ruby-evaluate if_ruby.txt /*ruby-evaluate*
ruby-globals if_ruby.txt /*ruby-globals*
ruby-message if_ruby.txt /*ruby-message*
ruby-rubyeval if_ruby.txt /*ruby-rubyeval*
ruby-set_option if_ruby.txt /*ruby-set_option*
ruby-vim if_ruby.txt /*ruby-vim*
ruby-window if_ruby.txt /*ruby-window*
ruby.vim syntax.txt /*ruby.vim*
ruby_fold syntax.txt /*ruby_fold*
ruby_foldable_groups syntax.txt /*ruby_foldable_groups*
ruby_minlines syntax.txt /*ruby_minlines*
ruby_no_expensive syntax.txt /*ruby_no_expensive*
ruby_operators syntax.txt /*ruby_operators*
ruby_space_errors syntax.txt /*ruby_space_errors*
ruby_spellcheck_strings syntax.txt /*ruby_spellcheck_strings*
rubyeval() eval.txt /*rubyeval()*
russian russian.txt /*russian*
russian-intro russian.txt /*russian-intro*
russian-issues russian.txt /*russian-issues*
russian-keymap russian.txt /*russian-keymap*
russian-l18n russian.txt /*russian-l18n*
russian.txt russian.txt /*russian.txt*
rust ft_rust.txt /*rust*
rust-commands ft_rust.txt /*rust-commands*
rust-intro ft_rust.txt /*rust-intro*
rust-mappings ft_rust.txt /*rust-mappings*
rust-settings ft_rust.txt /*rust-settings*
rust_<D-R> ft_rust.txt /*rust_<D-R>*
rust_<D-r> ft_rust.txt /*rust_<D-r>*
rview starting.txt /*rview*
rvim starting.txt /*rvim*
rxvt syntax.txt /*rxvt*
s change.txt /*s*
s/\& change.txt /*s/\&*
s/\0 change.txt /*s/\0*
s/\1 change.txt /*s/\1*

s/\2	change.txt	/*s/\\2*
s/\3	change.txt	/*s/\\3*
s/\9	change.txt	/*s/\\9*
s/\<CR>	change.txt	/*s/\\<CR>*
s/\=	change.txt	/*s/\\=*
s/\E	change.txt	/*s/\\E*
s/\L	change.txt	/*s/\\L*
s/\U	change.txt	/*s/\\U*
s/\\	change.txt	/*s/*
s/\b	change.txt	/*s/\\b*
s/\e	change.txt	/*s/\\e*
s/\l	change.txt	/*s/\\l*
s/\n	change.txt	/*s/\\n*
s/\r	change.txt	/*s/\\r*
s/\t	change.txt	/*s/\\t*
s/\u	change.txt	/*s/\\u*
s/\~	change.txt	/*s/\\~*
s:netrw_passwd	pi_netrw.txt	/*s:netrw_passwd*
s:var	eval.txt	/*s:var*
s<CR>	change.txt	/*s<CR>*
sandbox	eval.txt	/*sandbox*
sandbox-option	eval.txt	/*sandbox-option*
save-file	editing.txt	/*save-file*
save-settings	starting.txt	/*save-settings*
scheme.vim	syntax.txt	/*scheme.vim*
scp	pi_netrw.txt	/*scp*
screenattr()	eval.txt	/*screenattr()*
screenchar()	eval.txt	/*screenchar()*
screenchars()	eval.txt	/*screenchars()*
screencol()	eval.txt	/*screencol()*
screenpos()	eval.txt	/*screenpos()*
screenrow()	eval.txt	/*screenrow()*
screenstring()	eval.txt	/*screenstring()*
script	usr_41.txt	/*script*
script-here	if_perl.txt	/*script-here*
script-local	map.txt	/*script-local*
script-variable	eval.txt	/*script-variable*
scriptnames-dictionary	eval.txt	/*scriptnames-dictionary*
scriptout-changed	version4.txt	/*scriptout-changed*
scriptversion	eval.txt	/*scriptversion*
scriptversion-1	eval.txt	/*scriptversion-1*
scriptversion-2	eval.txt	/*scriptversion-2*
scriptversion-3	eval.txt	/*scriptversion-3*
scriptversion-4	eval.txt	/*scriptversion-4*
scroll-binding	scroll.txt	/*scroll-binding*
scroll-cursor	scroll.txt	/*scroll-cursor*
scroll-down	scroll.txt	/*scroll-down*
scroll-horizontal	scroll.txt	/*scroll-horizontal*
scroll-insert	tips.txt	/*scroll-insert*
scroll-mouse-wheel	scroll.txt	/*scroll-mouse-wheel*
scroll-region	term.txt	/*scroll-region*
scroll-smooth	tips.txt	/*scroll-smooth*
scroll-up	scroll.txt	/*scroll-up*
scroll.txt	scroll.txt	/*scroll.txt*

scrollbind-quickadj	scroll.txt	/*scrollbind-quickadj*
scrollbind-relative	scroll.txt	/*scrollbind-relative*
scrolling	scroll.txt	/*scrolling*
scrollstart-variable	eval.txt	/*scrollstart-variable*
sdl.vim	syntax.txt	/*sdl.vim*
search()	eval.txt	/*search()*
search()-sub-match	eval.txt	/*search()-sub-match*
search-commands	pattern.txt	/*search-commands*
search-offset	pattern.txt	/*search-offset*
search-pattern	pattern.txt	/*search-pattern*
search-range	pattern.txt	/*search-range*
search-replace	change.txt	/*search-replace*
searchdecl()	eval.txt	/*searchdecl()*
searchforward-variable	eval.txt	/*searchforward-variable*
searchpair()	eval.txt	/*searchpair()*
searchpairpos()	eval.txt	/*searchpairpos()*
searchpos()	eval.txt	/*searchpos()*
section	motion.txt	/*section*
sed.vim	syntax.txt	/*sed.vim*
self	eval.txt	/*self*
send-money	sponsor.txt	/*send-money*
send-to-menu	gui_w32.txt	/*send-to-menu*
sendto	gui_w32.txt	/*sendto*
sentence	motion.txt	/*sentence*
server-functions	usr_41.txt	/*server-functions*
server2client()	eval.txt	/*server2client()*
serverlist()	eval.txt	/*serverlist()*
servername-variable	eval.txt	/*servername-variable*
session-file	starting.txt	/*session-file*
set-option	options.txt	/*set-option*
set-spc-auto	spell.txt	/*set-spc-auto*
setbufline()	eval.txt	/*setbufline()*
setbufvar()	eval.txt	/*setbufvar()*
setcharsearch()	eval.txt	/*setcharsearch()*
setcmdpos()	eval.txt	/*setcmdpos()*
setenv()	eval.txt	/*setenv()*
setfperm()	eval.txt	/*setfperm()*
setline()	eval.txt	/*setline()*
setloclist()	eval.txt	/*setloclist()*
setmatches()	eval.txt	/*setmatches()*
setpos()	eval.txt	/*setpos()*
setqflist()	eval.txt	/*setqflist()*
setqflist-examples	quickfix.txt	/*setqflist-examples*
setreg()	eval.txt	/*setreg()*
settabvar()	eval.txt	/*settabvar()*
settabwinvar()	eval.txt	/*settabwinvar()*
settagstack()	eval.txt	/*settagstack()*
setting-guifont	gui.txt	/*setting-guifont*
setting-guitablabel	tabpage.txt	/*setting-guitablabel*
setting-tabline	tabpage.txt	/*setting-tabline*
setuid	change.txt	/*setuid*
setwinvar()	eval.txt	/*setwinvar()*
sftp	pi_netrw.txt	/*sftp*
sgml.vim	syntax.txt	/*sgml.vim*

sgr-mouse options.txt /*sgr-mouse*
sh-awk syntax.txt /*sh-awk*
sh-embed syntax.txt /*sh-embed*
sh.vim syntax.txt /*sh.vim*
sha256() eval.txt /*sha256()*
shell-window tips.txt /*shell-window*
shell_error-variable eval.txt /*shell_error-variable*
shellescape() eval.txt /*shellescape()*
shift intro.txt /*shift*
shift-left-right change.txt /*shift-left-right*
shiftwidth() eval.txt /*shiftwidth()*
short-name-changed version4.txt /*short-name-changed*
showing-menus gui.txt /*showing-menus*
sign-column sign.txt /*sign-column*
sign-commands sign.txt /*sign-commands*
sign-functions usr_41.txt /*sign-functions*
sign-functions-details sign.txt /*sign-functions-details*
sign-group sign.txt /*sign-group*
sign-identifier sign.txt /*sign-identifier*
sign-intro sign.txt /*sign-intro*
sign-priority sign.txt /*sign-priority*
sign-support sign.txt /*sign-support*
sign.txt sign.txt /*sign.txt*
sign_define() sign.txt /*sign_define()*
sign_getdefined() sign.txt /*sign_getdefined()*
sign_getplaced() sign.txt /*sign_getplaced()*
sign_jump() sign.txt /*sign_jump()*
sign_place() sign.txt /*sign_place()*
sign_placelist() sign.txt /*sign_placelist()*
sign_undefine() sign.txt /*sign_undefine()*
sign_unplace() sign.txt /*sign_unplace()*
sign_unplacelist() sign.txt /*sign_unplacelist()*
signs sign.txt /*signs*
simple-change change.txt /*simple-change*
simplify() eval.txt /*simplify()*
simulated-command vi_diff.txt /*simulated-command*
sin() eval.txt /*sin()*
single-repeat repeat.txt /*single-repeat*
sinh() eval.txt /*sinh()*
skeleton autocmd.txt /*skeleton*
skip_defaults_vim starting.txt /*skip_defaults_vim*
slice eval.txt /*slice*
slow-fast-terminal term.txt /*slow-fast-terminal*
slow-start starting.txt /*slow-start*
slow-terminal term.txt /*slow-terminal*
socket-interface channel.txt /*socket-interface*
sort() eval.txt /*sort()*
sorting change.txt /*sorting*
sound_clear() eval.txt /*sound_clear()*
sound_playevent() eval.txt /*sound_playevent()*
sound_playfile() eval.txt /*sound_playfile()*
sound_stop() eval.txt /*sound_stop()*
soundfold() eval.txt /*soundfold()*
space intro.txt /*space*

spec-customizing pi_spec.txt /*spec-customizing*
spec-how-to-use-it pi_spec.txt /*spec-how-to-use-it*
spec-setting-a-map pi_spec.txt /*spec-setting-a-map*
spec_chglog_format pi_spec.txt /*spec_chglog_format*
spec_chglog_prepend pi_spec.txt /*spec_chglog_prepend*
spec_chglog_release_info pi_spec.txt /*spec_chglog_release_info*
special-buffers windows.txt /*special-buffers*
speed-up tips.txt /*speed-up*
spell spell.txt /*spell*
spell-ACCENT spell.txt /*spell-ACCENT*
spell-AUTHOR spell.txt /*spell-AUTHOR*
spell-BAD spell.txt /*spell-BAD*
spell-BREAK spell.txt /*spell-BREAK*
spell-CHECKCOMPOUNDCASE spell.txt /*spell-CHECKCOMPOUNDCASE*
spell-CHECKCOMPOUNDDUP spell.txt /*spell-CHECKCOMPOUNDDUP*
spell-CHECKCOMPOUNDPATTERN spell.txt /*spell-CHECKCOMPOUNDPATTERN*
spell-CHECKCOMPOUNDREP spell.txt /*spell-CHECKCOMPOUNDREP*
spell-CHECKCOMPOUNDTRIPLE spell.txt /*spell-CHECKCOMPOUNDTRIPLE*
spell-CHECKSHARPS spell.txt /*spell-CHECKSHARPS*
spell-CIRCUMFIX spell.txt /*spell-CIRCUMFIX*
spell-COMMON spell.txt /*spell-COMMON*
spell-COMPLEXPREFIXES spell.txt /*spell-COMPLEXPREFIXES*
spell-COMPOUND spell.txt /*spell-COMPOUND*
spell-COMPOUNDBEGIN spell.txt /*spell-COMPOUNDBEGIN*
spell-COMPOUNDEND spell.txt /*spell-COMPOUNDEND*
spell-COMPOUNDFIRST spell.txt /*spell-COMPOUNDFIRST*
spell-COMPOUNDFLAG spell.txt /*spell-COMPOUNDFLAG*
spell-COMPOUNDFORBIDFLAG spell.txt /*spell-COMPOUNDFORBIDFLAG*
spell-COMPOUNDLAST spell.txt /*spell-COMPOUNDLAST*
spell-COMPOUNDMIDDLE spell.txt /*spell-COMPOUNDMIDDLE*
spell-COMPOUNDMIN spell.txt /*spell-COMPOUNDMIN*
spell-COMPOUNDPERMITFLAG spell.txt /*spell-COMPOUNDPERMITFLAG*
spell-COMPOUNDROOT spell.txt /*spell-COMPOUNDROOT*
spell-COMPOUNDRULE spell.txt /*spell-COMPOUNDRULE*
spell-COMPOUNDRULES spell.txt /*spell-COMPOUNDRULES*
spell-COMPOUNDSYLLABLE spell.txt /*spell-COMPOUNDSYLLABLE*
spell-COMPOUNDSYLMAX spell.txt /*spell-COMPOUNDSYLMAX*
spell-COMPOUNDWORDMAX spell.txt /*spell-COMPOUNDWORDMAX*
spell-COPYRIGHT spell.txt /*spell-COPYRIGHT*
spell-EMAIL spell.txt /*spell-EMAIL*
spell-FLAG spell.txt /*spell-FLAG*
spell-FOL spell.txt /*spell-FOL*
spell-FORBIDDENWORD spell.txt /*spell-FORBIDDENWORD*
spell-HOME spell.txt /*spell-HOME*
spell-IGNOREEXTRA spell.txt /*spell-IGNOREEXTRA*
spell-KEEPCASE spell.txt /*spell-KEEPCASE*
spell-KEY spell.txt /*spell-KEY*
spell-LANG spell.txt /*spell-LANG*
spell-LEMMA_PRESENT spell.txt /*spell-LEMMA_PRESENT*
spell-LOW spell.txt /*spell-LOW*
spell-MAP spell.txt /*spell-MAP*
spell-MAXNGRAMSUGS spell.txt /*spell-MAXNGRAMSUGS*
spell-NAME spell.txt /*spell-NAME*
spell-NEEDAFFIX spell.txt /*spell-NEEDAFFIX*

spell-NEEDCOMPOUND spell.txt /*spell-NEEDCOMPOUND*
spell-NOBREAK spell.txt /*spell-NOBREAK*
spell-NOCOMPOUNDSUGS spell.txt /*spell-NOCOMPOUNDSUGS*
spell-NOSPLITSUGS spell.txt /*spell-NOSPLITSUGS*
spell-NOSUGFILE spell.txt /*spell-NOSUGFILE*
spell-NOSUGGEST spell.txt /*spell-NOSUGGEST*
spell-ONLYINCOMPOUND spell.txt /*spell-ONLYINCOMPOUND*
spell-PFX spell.txt /*spell-PFX*
spell-PFXPOSTPONE spell.txt /*spell-PFXPOSTPONE*
spell-PSEUDOROOT spell.txt /*spell-PSEUDOROOT*
spell-RARE spell.txt /*spell-RARE*
spell-REP spell.txt /*spell-REP*
spell-SAL spell.txt /*spell-SAL*
spell-SET spell.txt /*spell-SET*
spell-SFX spell.txt /*spell-SFX*
spell-SLASH spell.txt /*spell-SLASH*
spell-SOFOFROM spell.txt /*spell-SOFOFROM*
spell-SOFOTO spell.txt /*spell-SOFOTO*
spell-SUGSWITHDOTS spell.txt /*spell-SUGSWITHDOTS*
spell-SYLLABLE spell.txt /*spell-SYLLABLE*
spell-SYLLABLENUM spell.txt /*spell-SYLLABLENUM*
spell-SpellFileMissing spell.txt /*spell-SpellFileMissing*
spell-TRY spell.txt /*spell-TRY*
spell-UPP spell.txt /*spell-UPP*
spell-VERSION spell.txt /*spell-VERSION*
spell-WORDCHARS spell.txt /*spell-WORDCHARS*
spell-aff-format spell.txt /*spell-aff-format*
spell-affix-chars spell.txt /*spell-affix-chars*
spell-affix-comment spell.txt /*spell-affix-comment*
spell-affix-flags spell.txt /*spell-affix-flags*
spell-affix-mbyte spell.txt /*spell-affix-mbyte*
spell-affix-not-supported spell.txt /*spell-affix-not-supported*
spell-affix-vim spell.txt /*spell-affix-vim*
spell-cjk spell.txt /*spell-cjk*
spell-compound spell.txt /*spell-compound*
spell-dic-format spell.txt /*spell-dic-format*
spell-double-scoring spell.txt /*spell-double-scoring*
spell-file-format spell.txt /*spell-file-format*
spell-functions usr_41.txt /*spell-functions*
spell-german spell.txt /*spell-german*
spell-load spell.txt /*spell-load*
spell-midword spell.txt /*spell-midword*
spell-mkspell spell.txt /*spell-mkspell*
spell-quickstart spell.txt /*spell-quickstart*
spell-remarks spell.txt /*spell-remarks*
spell-russian spell.txt /*spell-russian*
spell-sug-file spell.txt /*spell-sug-file*
spell-syntax spell.txt /*spell-syntax*
spell-wordlist-format spell.txt /*spell-wordlist-format*
spell-yiddish spell.txt /*spell-yiddish*
spell.txt spell.txt /*spell.txt*
spellbadword() eval.txt /*spellbadword()*
spellfile-cleanup spell.txt /*spellfile-cleanup*
spellfile.vim spell.txt /*spellfile.vim*

spellsuggest() eval.txt /*spellsuggest()*
split() eval.txt /*split()*
splitfind windows.txt /*splitfind*
splitview windows.txt /*splitview*
sponsor sponsor.txt /*sponsor*
sponsor-faq sponsor.txt /*sponsor-faq*
sponsor.txt sponsor.txt /*sponsor.txt*
spoon os_unix.txt /*spoon*
spup.vim syntax.txt /*spup.vim*
sql-adding-dialects ft_sql.txt /*sql-adding-dialects*
sql-completion ft_sql.txt /*sql-completion*
sql-completion-columns ft_sql.txt /*sql-completion-columns*
sql-completion-customization ft_sql.txt /*sql-completion-customization*
sql-completion-dynamic ft_sql.txt /*sql-completion-dynamic*
sql-completion-filetypes ft_sql.txt /*sql-completion-filetypes*
sql-completion-maps ft_sql.txt /*sql-completion-maps*
sql-completion-procedures ft_sql.txt /*sql-completion-procedures*
sql-completion-static ft_sql.txt /*sql-completion-static*
sql-completion-tables ft_sql.txt /*sql-completion-tables*
sql-completion-tutorial ft_sql.txt /*sql-completion-tutorial*
sql-completion-views ft_sql.txt /*sql-completion-views*
sql-dialects ft_sql.txt /*sql-dialects*
sql-macros ft_sql.txt /*sql-macros*
sql-matchit ft_sql.txt /*sql-matchit*
sql-navigation ft_sql.txt /*sql-navigation*
sql-object-motions ft_sql.txt /*sql-object-motions*
sql-predefined-objects ft_sql.txt /*sql-predefined-objects*
sql-type-default ft_sql.txt /*sql-type-default*
sql-types ft_sql.txt /*sql-types*
sql.vim syntax.txt /*sql.vim*
sqlanywhere ft_sql.txt /*sqlanywhere*
sqlanywhere.vim syntax.txt /*sqlanywhere.vim*
sqlgettype ft_sql.txt /*sqlgettype*
sqlinformix.vim syntax.txt /*sqlinformix.vim*
sqlj ft_sql.txt /*sqlj*
sqlserver ft_sql.txt /*sqlserver*
sqlsettype ft_sql.txt /*sqlsettype*
sqrt() eval.txt /*sqrt()*
srand() eval.txt /*srand()*
sscanf eval.txt /*sscanf*
standard-plugin usr_05.txt /*standard-plugin*
standard-plugin-list help.txt /*standard-plugin-list*
standout syntax.txt /*standout*
star pattern.txt /*star*
starstar editing.txt /*starstar*
starstar-wildcard editing.txt /*starstar-wildcard*
start-of-file pattern.txt /*start-of-file*
start-vimdiff diff.txt /*start-vimdiff*
starting starting.txt /*starting*
starting-amiga starting.txt /*starting-amiga*
starting.txt starting.txt /*starting.txt*
startup starting.txt /*startup*
startup-options starting.txt /*startup-options*
startup-terminal term.txt /*startup-terminal*

state() eval.txt /*state()*
static-tag tagsrch.txt /*static-tag*
status-line windows.txt /*status-line*
statusmsg-variable eval.txt /*statusmsg-variable*
stl-%{ options.txt /*stl-%{*
str2float() eval.txt /*str2float()*
str2list() eval.txt /*str2list()*
str2nr() eval.txt /*str2nr()*
strcasestr() eval.txt /*strcasestr()*
strcharpart() eval.txt /*strcharpart()*
strchars() eval.txt /*strchars()*
strchr() eval.txt /*strchr()*
strcspn() eval.txt /*strcspn()*
strdisplaywidth() eval.txt /*strdisplaywidth()*
strftime() eval.txt /*strftime()*
strgetchar() eval.txt /*strgetchar()*
stridx() eval.txt /*stridx()*
strikethrough syntax.txt /*strikethrough*
string eval.txt /*string*
string() eval.txt /*string()*
string-functions usr_41.txt /*string-functions*
string-match eval.txt /*string-match*
strlen() eval.txt /*strlen()*
strpart() eval.txt /*strpart()*
strpbrk() eval.txt /*strpbrk()*
strptime() eval.txt /*strptime()*
strrchr() eval.txt /*strrchr()*
strridx() eval.txt /*strridx()*
strspn() eval.txt /*strspn()*
strstr() eval.txt /*strstr()*
strtrans() eval.txt /*strtrans()*
strwidth() eval.txt /*strwidth()*
style-changes develop.txt /*style-changes*
style-compiler develop.txt /*style-compiler*
style-examples develop.txt /*style-examples*
style-functions develop.txt /*style-functions*
style-names develop.txt /*style-names*
style-spaces develop.txt /*style-spaces*
style-various develop.txt /*style-various*
sub-menu-priority gui.txt /*sub-menu-priority*
sub-replace-\= change.txt /*sub-replace-\\=*
sub-replace-expression change.txt /*sub-replace-expression*
sub-replace-special change.txt /*sub-replace-special*
sublist eval.txt /*sublist*
submatch() eval.txt /*submatch()*
subscribe-maillist intro.txt /*subscribe-maillist*
subscript eval.txt /*subscript*
substitute() eval.txt /*substitute()*
substitute-CR version6.txt /*substitute-CR*
suffixes cmdline.txt /*suffixes*
suspend starting.txt /*suspend*
swap-exists-choices usr_11.txt /*swap-exists-choices*
swap-file recover.txt /*swap-file*
swapchoice-variable eval.txt /*swapchoice-variable*

swapcommand-variable	eval.txt	/*swapcommand-variable*
swapfile-changed	version4.txt	/*swapfile-changed*
swapinfo()	eval.txt	/*swapinfo()*
swapname()	eval.txt	/*swapname()*
swapname-variable	eval.txt	/*swapname-variable*
sybase	ft_sql.txt	/*sybase*
syn-sync-grouphere	syntax.txt	/*syn-sync-grouphere*
syn-sync-groupthere	syntax.txt	/*syn-sync-groupthere*
syn-sync-linecont	syntax.txt	/*syn-sync-linecont*
synID()	eval.txt	/*synID()*
synIDattr()	eval.txt	/*synIDattr()*
synIDtrans()	eval.txt	/*synIDtrans()*
syncbind	scroll.txt	/*syncbind*
syncolor	syntax.txt	/*syncolor*
synconcealed()	eval.txt	/*synconcealed()*
synload-1	syntax.txt	/*synload-1*
synload-2	syntax.txt	/*synload-2*
synload-3	syntax.txt	/*synload-3*
synload-4	syntax.txt	/*synload-4*
synload-5	syntax.txt	/*synload-5*
synload-6	syntax.txt	/*synload-6*
synstack()	eval.txt	/*synstack()*
syntax	syntax.txt	/*syntax*
syntax-functions	usr_41.txt	/*syntax-functions*
syntax-highlighting	syntax.txt	/*syntax-highlighting*
syntax-loading	syntax.txt	/*syntax-loading*
syntax-printing	usr_06.txt	/*syntax-printing*
syntax.txt	syntax.txt	/*syntax.txt*
syntax_cmd	syntax.txt	/*syntax_cmd*
sys-file-list	help.txt	/*sys-file-list*
sysmouse	term.txt	/*sysmouse*
system()	eval.txt	/*system()*
system-functions	usr_41.txt	/*system-functions*
system-vimrc	starting.txt	/*system-vimrc*
systemlist()	eval.txt	/*systemlist()*
s~	change.txt	/*s~*
t	motion.txt	/*t*
t:	eval.txt	/*t:*
t:var	eval.txt	/*t:var*
t_#2	term.txt	/*t_#2*
t_#4	term.txt	/*t_#4*
t_%1	term.txt	/*t_%1*
t_%i	term.txt	/*t_%i*
t_&8	term.txt	/*t_&8*
t_8b	term.txt	/*t_8b*
t_8f	term.txt	/*t_8f*
t_@7	term.txt	/*t_@7*
t_AB	term.txt	/*t_AB*
t_AF	term.txt	/*t_AF*
t_AL	term.txt	/*t_AL*
t_BD	term.txt	/*t_BD*
t_BE	term.txt	/*t_BE*
t_CS	term.txt	/*t_CS*
t_CTRL-W_.	terminal.txt	/*t_CTRL-W_.*

t_CTRL-W_: terminal.txt /*t_CTRL-W_:*
t_CTRL-W_CTRL-C terminal.txt /*t_CTRL-W_CTRL-C*
t_CTRL-W_CTRL-W terminal.txt /*t_CTRL-W_CTRL-W*
t_CTRL-W_N terminal.txt /*t_CTRL-W_N*
t_CTRL-W_gT terminal.txt /*t_CTRL-W_gT*
t_CTRL-W_gt terminal.txt /*t_CTRL-W_gt*
t_CTRL-W_quote terminal.txt /*t_CTRL-W_quote*
t_CTRL-_CTRL-N terminal.txt /*t_CTRL-_CTRL-N*
t_CV term.txt /*t_CV*
t_Ce term.txt /*t_Ce*
t_Co term.txt /*t_Co*
t_Cs term.txt /*t_Cs*
t_DL term.txt /*t_DL*
t_EC term.txt /*t_EC*
t_EI term.txt /*t_EI*
t_F1 term.txt /*t_F1*
t_F2 term.txt /*t_F2*
t_F3 term.txt /*t_F3*
t_F4 term.txt /*t_F4*
t_F5 term.txt /*t_F5*
t_F6 term.txt /*t_F6*
t_F7 term.txt /*t_F7*
t_F8 term.txt /*t_F8*
t_F9 term.txt /*t_F9*
t_GP term.txt /*t_GP*
t_IE term.txt /*t_IE*
t_IS term.txt /*t_IS*
t_K1 term.txt /*t_K1*
t_K3 term.txt /*t_K3*
t_K4 term.txt /*t_K4*
t_K5 term.txt /*t_K5*
t_K6 term.txt /*t_K6*
t_K7 term.txt /*t_K7*
t_K8 term.txt /*t_K8*
t_K9 term.txt /*t_K9*
t_KA term.txt /*t_KA*
t_KB term.txt /*t_KB*
t_KC term.txt /*t_KC*
t_KD term.txt /*t_KD*
t_KE term.txt /*t_KE*
t_KF term.txt /*t_KF*
t_KG term.txt /*t_KG*
t_KH term.txt /*t_KH*
t_KI term.txt /*t_KI*
t_KJ term.txt /*t_KJ*
t_KK term.txt /*t_KK*
t_KL term.txt /*t_KL*
t_PE term.txt /*t_PE*
t_PS term.txt /*t_PS*
t_RB term.txt /*t_RB*
t_RC term.txt /*t_RC*
t_RF term.txt /*t_RF*
t_RI term.txt /*t_RI*
t_RS term.txt /*t_RS*

t_RT term.txt /*t_RT*
t_RV term.txt /*t_RV*
t_Ri term.txt /*t_Ri*
t_SC term.txt /*t_SC*
t_SH term.txt /*t_SH*
t_SI term.txt /*t_SI*
t_SR term.txt /*t_SR*
t_ST term.txt /*t_ST*
t_Sb term.txt /*t_Sb*
t_Sf term.txt /*t_Sf*
t_Si term.txt /*t_Si*
t_TE term.txt /*t_TE*
t_TI term.txt /*t_TI*
t_Te term.txt /*t_Te*
t_Ts term.txt /*t_Ts*
t_VS term.txt /*t_VS*
t_WP term.txt /*t_WP*
t_WS term.txt /*t_WS*
t_ZH term.txt /*t_ZH*
t_ZR term.txt /*t_ZR*
t_al term.txt /*t_al*
t_bc term.txt /*t_bc*
t_blob-variable eval.txt /*t_blob-variable*
t_bool-variable eval.txt /*t_bool-variable*
t_cd term.txt /*t_cd*
t_cdl version4.txt /*t_cdl*
t_ce term.txt /*t_ce*
t_channel-variable eval.txt /*t_channel-variable*
t_ci version4.txt /*t_ci*
t_cil version4.txt /*t_cil*
t_cl term.txt /*t_cl*
t_cm term.txt /*t_cm*
t_cri version4.txt /*t_cri*
t_cs term.txt /*t_cs*
t_csc version4.txt /*t_csc*
t_cv version4.txt /*t_cv*
t_cvv version4.txt /*t_cvv*
t_da term.txt /*t_da*
t_db term.txt /*t_db*
t_dict-variable eval.txt /*t_dict-variable*
t_dl term.txt /*t_dl*
t_ed version4.txt /*t_ed*
t_el version4.txt /*t_el*
t_f1 version4.txt /*t_f1*
t_f10 version4.txt /*t_f10*
t_f2 version4.txt /*t_f2*
t_f3 version4.txt /*t_f3*
t_f4 version4.txt /*t_f4*
t_f5 version4.txt /*t_f5*
t_f6 version4.txt /*t_f6*
t_f7 version4.txt /*t_f7*
t_f8 version4.txt /*t_f8*
t_f9 version4.txt /*t_f9*
t_float-variable eval.txt /*t_float-variable*

t_fs term.txt /*t_fs*
t_func-variable eval.txt /*t_func-variable*
t_help version4.txt /*t_help*
t_il version4.txt /*t_il*
t_job-variable eval.txt /*t_job-variable*
t_k1 term.txt /*t_k1*
t_k2 term.txt /*t_k2*
t_k3 term.txt /*t_k3*
t_k4 term.txt /*t_k4*
t_k5 term.txt /*t_k5*
t_k6 term.txt /*t_k6*
t_k7 term.txt /*t_k7*
t_k8 term.txt /*t_k8*
t_k9 term.txt /*t_k9*
t_k; term.txt /*t_k;*
t_kB term.txt /*t_kB*
t_kD term.txt /*t_kD*
t_kI term.txt /*t_kI*
t_kN term.txt /*t_kN*
t_kP term.txt /*t_kP*
t_kb term.txt /*t_kb*
t_kd term.txt /*t_kd*
t_ke term.txt /*t_ke*
t_kh term.txt /*t_kh*
t_kl term.txt /*t_kl*
t_kr term.txt /*t_kr*
t_ks term.txt /*t_ks*
t_ku term.txt /*t_ku*
t_le term.txt /*t_le*
t_list-variable eval.txt /*t_list-variable*
t_mb term.txt /*t_mb*
t_md term.txt /*t_md*
t_me term.txt /*t_me*
t_mr term.txt /*t_mr*
t_ms term.txt /*t_ms*
t_nd term.txt /*t_nd*
t_none-variable eval.txt /*t_none-variable*
t_number-variable eval.txt /*t_number-variable*
t_op term.txt /*t_op*
t_se term.txt /*t_se*
t_sf1 version4.txt /*t_sf1*
t_sf10 version4.txt /*t_sf10*
t_sf2 version4.txt /*t_sf2*
t_sf3 version4.txt /*t_sf3*
t_sf4 version4.txt /*t_sf4*
t_sf5 version4.txt /*t_sf5*
t_sf6 version4.txt /*t_sf6*
t_sf7 version4.txt /*t_sf7*
t_sf8 version4.txt /*t_sf8*
t_sf9 version4.txt /*t_sf9*
t_skd version4.txt /*t_skd*
t_skl version4.txt /*t_skl*
t_skr version4.txt /*t_skr*
t_sku version4.txt /*t_sku*

t_so	term.txt	/*t_so*
t_sr	term.txt	/*t_sr*
t_star7	term.txt	/*t_star7*
t_string-variable	eval.txt	/*t_string-variable*
t_tb	version4.txt	/*t_tb*
t_te	term.txt	/*t_te*
t_ti	term.txt	/*t_ti*
t_tp	version4.txt	/*t_tp*
t_ts	term.txt	/*t_ts*
t_ts_old	version4.txt	/*t_ts_old*
t_u7	term.txt	/*t_u7*
t_ue	term.txt	/*t_ue*
t_undo	version4.txt	/*t_undo*
t_us	term.txt	/*t_us*
t_ut	term.txt	/*t_ut*
t_vb	term.txt	/*t_vb*
t_ve	term.txt	/*t_ve*
t_vi	term.txt	/*t_vi*
t_vs	term.txt	/*t_vs*
t_xn	term.txt	/*t_xn*
t_xs	term.txt	/*t_xs*
tab	intro.txt	/*tab*
tab-page	tabpage.txt	/*tab-page*
tab-page-commands	tabpage.txt	/*tab-page-commands*
tab-page-intro	tabpage.txt	/*tab-page-intro*
tab-page-other	tabpage.txt	/*tab-page-other*
tabline-menu	tabpage.txt	/*tabline-menu*
tabnew-autocmd	tabpage.txt	/*tabnew-autocmd*
tabpage	tabpage.txt	/*tabpage*
tabpage-variable	eval.txt	/*tabpage-variable*
tabpage.txt	tabpage.txt	/*tabpage.txt*
tabpagebuflist()	eval.txt	/*tabpagebuflist()*
tabpagenr()	eval.txt	/*tabpagenr()*
tabpagewinnr()	eval.txt	/*tabpagewinnr()*
tag	tagsrch.txt	/*tag*
tag-!	tagsrch.txt	/*tag-!*
tag-binary-search	tagsrch.txt	/*tag-binary-search*
tag-blocks	motion.txt	/*tag-blocks*
tag-commands	tagsrch.txt	/*tag-commands*
tag-details	tagsrch.txt	/*tag-details*
tag-function	tagsrch.txt	/*tag-function*
tag-functions	usr_41.txt	/*tag-functions*
tag-highlight	syntax.txt	/*tag-highlight*
tag-matchlist	tagsrch.txt	/*tag-matchlist*
tag-old-static	tagsrch.txt	/*tag-old-static*
tag-overloaded	version5.txt	/*tag-overloaded*
tag-preview	tagsrch.txt	/*tag-preview*
tag-priority	tagsrch.txt	/*tag-priority*
tag-regexp	tagsrch.txt	/*tag-regexp*
tag-search	tagsrch.txt	/*tag-search*
tag-security	tagsrch.txt	/*tag-security*
tag-skip-file	tagsrch.txt	/*tag-skip-file*
tag-stack	tagsrch.txt	/*tag-stack*
tagfiles()	eval.txt	/*tagfiles()*

taglist()	eval.txt	/*taglist()*
tags	tagsrch.txt	/*tags*
tags-and-searches	tagsrch.txt	/*tags-and-searches*
tags-file-changed	version5.txt	/*tags-file-changed*
tags-file-format	tagsrch.txt	/*tags-file-format*
tags-option	tagsrch.txt	/*tags-option*
tagsrch.txt	tagsrch.txt	/*tagsrch.txt*
tagstack	tagsrch.txt	/*tagstack*
tan()	eval.txt	/*tan()*
tanh()	eval.txt	/*tanh()*
tar	pi_tar.txt	/*tar*
tar-contents	pi_tar.txt	/*tar-contents*
tar-copyright	pi_tar.txt	/*tar-copyright*
tar-history	pi_tar.txt	/*tar-history*
tar-manual	pi_tar.txt	/*tar-manual*
tar-options	pi_tar.txt	/*tar-options*
tar-usage	pi_tar.txt	/*tar-usage*
tcl	if_tcl.txt	/*tcl*
tcl-beep	if_tcl.txt	/*tcl-beep*
tcl-buffer	if_tcl.txt	/*tcl-buffer*
tcl-buffer-append	if_tcl.txt	/*tcl-buffer-append*
tcl-buffer-cmds	if_tcl.txt	/*tcl-buffer-cmds*
tcl-buffer-command	if_tcl.txt	/*tcl-buffer-command*
tcl-buffer-count	if_tcl.txt	/*tcl-buffer-count*
tcl-buffer-delcmd	if_tcl.txt	/*tcl-buffer-delcmd*
tcl-buffer-delete	if_tcl.txt	/*tcl-buffer-delete*
tcl-buffer-expr	if_tcl.txt	/*tcl-buffer-expr*
tcl-buffer-get	if_tcl.txt	/*tcl-buffer-get*
tcl-buffer-insert	if_tcl.txt	/*tcl-buffer-insert*
tcl-buffer-last	if_tcl.txt	/*tcl-buffer-last*
tcl-buffer-mark	if_tcl.txt	/*tcl-buffer-mark*
tcl-buffer-option	if_tcl.txt	/*tcl-buffer-option*
tcl-buffer-set	if_tcl.txt	/*tcl-buffer-set*
tcl-buffer-windows	if_tcl.txt	/*tcl-buffer-windows*
tcl-bugs	if_tcl.txt	/*tcl-bugs*
tcl-command	if_tcl.txt	/*tcl-command*
tcl-commands	if_tcl.txt	/*tcl-commands*
tcl-dynamic	if_tcl.txt	/*tcl-dynamic*
tcl-ex-commands	if_tcl.txt	/*tcl-ex-commands*
tcl-examples	if_tcl.txt	/*tcl-examples*
tcl-expr	if_tcl.txt	/*tcl-expr*
tcl-linenumbers	if_tcl.txt	/*tcl-linenumbers*
tcl-misc	if_tcl.txt	/*tcl-misc*
tcl-option	if_tcl.txt	/*tcl-option*
tcl-output	if_tcl.txt	/*tcl-output*
tcl-var-current	if_tcl.txt	/*tcl-var-current*
tcl-var-lbase	if_tcl.txt	/*tcl-var-lbase*
tcl-var-line	if_tcl.txt	/*tcl-var-line*
tcl-var-lnum	if_tcl.txt	/*tcl-var-lnum*
tcl-var-range	if_tcl.txt	/*tcl-var-range*
tcl-variables	if_tcl.txt	/*tcl-variables*
tcl-window	if_tcl.txt	/*tcl-window*
tcl-window-buffer	if_tcl.txt	/*tcl-window-buffer*
tcl-window-cmds	if_tcl.txt	/*tcl-window-cmds*

tcl-window-command if_tcl.txt /*tcl-window-command*
tcl-window-cursor if_tcl.txt /*tcl-window-cursor*
tcl-window-delcmd if_tcl.txt /*tcl-window-delcmd*
tcl-window-expr if_tcl.txt /*tcl-window-expr*
tcl-window-height if_tcl.txt /*tcl-window-height*
tcl-window-option if_tcl.txt /*tcl-window-option*
tcsh-style cmdline.txt /*tcsh-style*
tcsh.vim syntax.txt /*tcsh.vim*
tear-off-menus gui.txt /*tear-off-menus*
telnet-CTRL-] tagsrch.txt /*telnet-CTRL-]*
temp-file-name eval.txt /*temp-file-name*
tempfile change.txt /*tempfile*
template autocmd.txt /*template*
tempname() eval.txt /*tempname()*
term++close terminal.txt /*term++close*
term++open terminal.txt /*term++open*
term-dependent-settings term.txt /*term-dependent-settings*
term-list syntax.txt /*term-list*
term.txt term.txt /*term.txt*
term_dumpdiff() terminal.txt /*term_dumpdiff()*
term_dumpload() terminal.txt /*term_dumpload()*
term_dumpwrite() terminal.txt /*term_dumpwrite()*
term_getaltscreen() terminal.txt /*term_getaltscreen()*
term_getansicolors() terminal.txt /*term_getansicolors()*
term_getattr() terminal.txt /*term_getattr()*
term_getcursor() terminal.txt /*term_getcursor()*
term_getjob() terminal.txt /*term_getjob()*
term_getline() terminal.txt /*term_getline()*
term_getscrolled() terminal.txt /*term_getscrolled()*
term_getsize() terminal.txt /*term_getsize()*
term_getstatus() terminal.txt /*term_getstatus()*
term_gettitle() terminal.txt /*term_gettitle()*
term_gettty() terminal.txt /*term_gettty()*
term_list() terminal.txt /*term_list()*
term_scrape() terminal.txt /*term_scrape()*
term_sendkeys() terminal.txt /*term_sendkeys()*
term_setansicolors() terminal.txt /*term_setansicolors()*
term_setapi() terminal.txt /*term_setapi()*
term_setkill() terminal.txt /*term_setkill()*
term_setrestore() terminal.txt /*term_setrestore()*
term_setsize() terminal.txt /*term_setsize()*
term_start() terminal.txt /*term_start()*
term_wait() terminal.txt /*term_wait()*
termcap term.txt /*termcap*
termcap-changed version4.txt /*termcap-changed*
termcap-colors term.txt /*termcap-colors*
termcap-cursor-color term.txt /*termcap-cursor-color*
termcap-cursor-shape term.txt /*termcap-cursor-shape*
termcap-options term.txt /*termcap-options*
termcap-title term.txt /*termcap-title*
termdebug-commands terminal.txt /*termdebug-commands*
termdebug-communication terminal.txt /*termdebug-communication*
termdebug-customizing terminal.txt /*termdebug-customizing*
termdebug-example terminal.txt /*termdebug-example*

termdebug-prompt terminal.txt /*termdebug-prompt*
termdebug-starting terminal.txt /*termdebug-starting*
termdebug-stepping terminal.txt /*termdebug-stepping*
termdebug-variables terminal.txt /*termdebug-variables*
termdebug_popup terminal.txt /*termdebug_popup*
termdebug_shortcuts terminal.txt /*termdebug_shortcuts*
termdebug_use_prompt terminal.txt /*termdebug_use_prompt*
termdebug_wide terminal.txt /*termdebug_wide*
terminal terminal.txt /*terminal*
terminal-api terminal.txt /*terminal-api*
terminal-client-server terminal.txt /*terminal-client-server*
terminal-close terminal.txt /*terminal-close*
terminal-colors os_unix.txt /*terminal-colors*
terminal-communication terminal.txt /*terminal-communication*
terminal-cursor-style terminal.txt /*terminal-cursor-style*
terminal-debug terminal.txt /*terminal-debug*
terminal-debugger terminal.txt /*terminal-debugger*
terminal-diff terminal.txt /*terminal-diff*
terminal-diffscreendump terminal.txt /*terminal-diffscreendump*
terminal-dumptest terminal.txt /*terminal-dumptest*
terminal-function-details terminal.txt /*terminal-function-details*
terminal-functions usr_41.txt /*terminal-functions*
terminal-info term.txt /*terminal-info*
terminal-job-index index.txt /*terminal-job-index*
terminal-key-codes term.txt /*terminal-key-codes*
terminal-mouse term.txt /*terminal-mouse*
terminal-ms-windows terminal.txt /*terminal-ms-windows*
terminal-options term.txt /*terminal-options*
terminal-output-codes term.txt /*terminal-output-codes*
terminal-resizing terminal.txt /*terminal-resizing*
terminal-screendump terminal.txt /*terminal-screendump*
terminal-session terminal.txt /*terminal-session*
terminal-size-color terminal.txt /*terminal-size-color*
terminal-special-keys terminal.txt /*terminal-special-keys*
terminal-testing terminal.txt /*terminal-testing*
terminal-to-job terminal.txt /*terminal-to-job*
terminal-typing terminal.txt /*terminal-typing*
terminal-unix terminal.txt /*terminal-unix*
terminal-use terminal.txt /*terminal-use*
terminal-window terminal.txt /*terminal-window*
terminal.txt terminal.txt /*terminal.txt*
terminfo term.txt /*terminfo*
termresponse-variable eval.txt /*termresponse-variable*
test-functions usr_41.txt /*test-functions*
test-functions-details testing.txt /*test-functions-details*
test_alloc_fail() testing.txt /*test_alloc_fail()*
test_autochdir() testing.txt /*test_autochdir()*
test_feedinput() testing.txt /*test_feedinput()*
test_garbagecollect_now() testing.txt /*test_garbagecollect_now()*
test_garbagecollect_soon() testing.txt /*test_garbagecollect_soon()*
test_getvalue() testing.txt /*test_getvalue()*
test_ignore_error() testing.txt /*test_ignore_error()*
test_null_blob() testing.txt /*test_null_blob()*
test_null_channel() testing.txt /*test_null_channel()*

test_null_dict()	testing.txt	/*test_null_dict()*
test_null_job()	testing.txt	/*test_null_job()*
test_null_list()	testing.txt	/*test_null_list()*
test_null_partial()	testing.txt	/*test_null_partial()*
test_null_string()	testing.txt	/*test_null_string()*
test_option_not_set()	testing.txt	/*test_option_not_set()*
test_override()	testing.txt	/*test_override()*
test_refcount()	testing.txt	/*test_refcount()*
test_scrollbar()	testing.txt	/*test_scrollbar()*
test_setmouse()	testing.txt	/*test_setmouse()*
test_settime()	testing.txt	/*test_settime()*
testing	testing.txt	/*testing*
testing-support	testing.txt	/*testing-support*
testing-variable	eval.txt	/*testing-variable*
testing.txt	testing.txt	/*testing.txt*
tex-cchar	syntax.txt	/*tex-cchar*
tex-cole	syntax.txt	/*tex-cole*
tex-conceal	syntax.txt	/*tex-conceal*
tex-error	syntax.txt	/*tex-error*
tex-folding	syntax.txt	/*tex-folding*
tex-math	syntax.txt	/*tex-math*
tex-morecommands	syntax.txt	/*tex-morecommands*
tex-nospell	syntax.txt	/*tex-nospell*
tex-package	syntax.txt	/*tex-package*
tex-runon	syntax.txt	/*tex-runon*
tex-slow	syntax.txt	/*tex-slow*
tex-stopzone	syntax.txt	/*tex-stopzone*
tex-style	syntax.txt	/*tex-style*
tex-supersub	syntax.txt	/*tex-supersub*
tex-sync	syntax.txt	/*tex-sync*
tex-verb	syntax.txt	/*tex-verb*
tex.vim	syntax.txt	/*tex.vim*
text-functions	usr_41.txt	/*text-functions*
text-objects	motion.txt	/*text-objects*
text-objects-changed	version5.txt	/*text-objects-changed*
text-prop-changes	textprop.txt	/*text-prop-changes*
text-prop-functions	textprop.txt	/*text-prop-functions*
text-prop-intro	textprop.txt	/*text-prop-intro*
text-properties	textprop.txt	/*text-properties*
textlock	eval.txt	/*textlock*
textprop	textprop.txt	/*textprop*
textprop.txt	textprop.txt	/*textprop.txt*
tf.vim	syntax.txt	/*tf.vim*
this_session-variable	eval.txt	/*this_session-variable*
throw-catch	eval.txt	/*throw-catch*
throw-expression	eval.txt	/*throw-expression*
throw-from-catch	eval.txt	/*throw-from-catch*
throw-variables	eval.txt	/*throw-variables*
throwpoint-variable	eval.txt	/*throwpoint-variable*
time-functions	usr_41.txt	/*time-functions*
timer	eval.txt	/*timer*
timer-functions	usr_41.txt	/*timer-functions*
timer_info()	eval.txt	/*timer_info()*
timer_pause()	eval.txt	/*timer_pause()*

timer_start()	eval.txt	/*timer_start()*
timer_stop()	eval.txt	/*timer_stop()*
timer_stopall()	eval.txt	/*timer_stopall()*
timers	eval.txt	/*timers*
timestamp	editing.txt	/*timestamp*
timestamps	editing.txt	/*timestamps*
tips	tips.txt	/*tips*
tips.txt	tips.txt	/*tips.txt*
todo	todo.txt	/*todo*
todo.txt	todo.txt	/*todo.txt*
toggle	options.txt	/*toggle*
toggle-revins	version4.txt	/*toggle-revins*
tolower()	eval.txt	/*tolower()*
toolbar-icon	gui.txt	/*toolbar-icon*
toupper()	eval.txt	/*toupper()*
tr()	eval.txt	/*tr()*
trim()	eval.txt	/*trim()*
trojan-horse	starting.txt	/*trojan-horse*
true-variable	eval.txt	/*true-variable*
trunc()	eval.txt	/*trunc()*
try-conditionals	eval.txt	/*try-conditionals*
try-echoerr	eval.txt	/*try-echoerr*
try-finally	eval.txt	/*try-finally*
try-nested	eval.txt	/*try-nested*
try-nesting	eval.txt	/*try-nesting*
tutor	usr_01.txt	/*tutor*
twice	if_cscop.txt	/*twice*
two-engines	pattern.txt	/*two-engines*
type()	eval.txt	/*type()*
type-mistakes	tips.txt	/*type-mistakes*
typecorr-settings	usr_41.txt	/*typecorr-settings*
typecorr.txt	usr_41.txt	/*typecorr.txt*
u	undo.txt	/*u*
uganda	uganda.txt	/*uganda*
uganda.txt	uganda.txt	/*uganda.txt*
undercurl	syntax.txt	/*undercurl*
underline	syntax.txt	/*underline*
undo	undo.txt	/*undo*
undo-blocks	undo.txt	/*undo-blocks*
undo-branches	undo.txt	/*undo-branches*
undo-commands	undo.txt	/*undo-commands*
undo-persistence	undo.txt	/*undo-persistence*
undo-redo	undo.txt	/*undo-redo*
undo-remarks	undo.txt	/*undo-remarks*
undo-tree	undo.txt	/*undo-tree*
undo-two-ways	undo.txt	/*undo-two-ways*
undo.txt	undo.txt	/*undo.txt*
undo_ftplugin	usr_41.txt	/*undo_ftplugin*
undo_indent	usr_41.txt	/*undo_indent*
undofile()	eval.txt	/*undofile()*
undotree()	eval.txt	/*undotree()*
unicode	mbyte.txt	/*unicode*
uniq()	eval.txt	/*uniq()*
unix	os_unix.txt	/*unix*

unlisted-buffer	windows.txt	/*unlisted-buffer*
up-down-motions	motion.txt	/*up-down-motions*
uppercase	change.txt	/*uppercase*
urxvt-mouse	options.txt	/*urxvt-mouse*
use-cpo-save	usr_41.txt	/*use-cpo-save*
use-visual-cmds	version4.txt	/*use-visual-cmds*
useful-mappings	tips.txt	/*useful-mappings*
usenet	intro.txt	/*usenet*
user-cmd-ambiguous	map.txt	/*user-cmd-ambiguous*
user-commands	map.txt	/*user-commands*
user-functions	eval.txt	/*user-functions*
user-manual	usr_toc.txt	/*user-manual*
using-<Plug>	usr_41.txt	/*using-<Plug>*
using-menus	gui.txt	/*using-menus*
using-scripts	repeat.txt	/*using-scripts*
using-xxd	tips.txt	/*using-xxd*
using_CTRL-V	map.txt	/*using_CTRL-V*
usr_01.txt	usr_01.txt	/*usr_01.txt*
usr_02.txt	usr_02.txt	/*usr_02.txt*
usr_03.txt	usr_03.txt	/*usr_03.txt*
usr_04.txt	usr_04.txt	/*usr_04.txt*
usr_05.txt	usr_05.txt	/*usr_05.txt*
usr_06.txt	usr_06.txt	/*usr_06.txt*
usr_07.txt	usr_07.txt	/*usr_07.txt*
usr_08.txt	usr_08.txt	/*usr_08.txt*
usr_09.txt	usr_09.txt	/*usr_09.txt*
usr_10.txt	usr_10.txt	/*usr_10.txt*
usr_11.txt	usr_11.txt	/*usr_11.txt*
usr_12.txt	usr_12.txt	/*usr_12.txt*
usr_20.txt	usr_20.txt	/*usr_20.txt*
usr_21.txt	usr_21.txt	/*usr_21.txt*
usr_22.txt	usr_22.txt	/*usr_22.txt*
usr_23.txt	usr_23.txt	/*usr_23.txt*
usr_24.txt	usr_24.txt	/*usr_24.txt*
usr_25.txt	usr_25.txt	/*usr_25.txt*
usr_26.txt	usr_26.txt	/*usr_26.txt*
usr_27.txt	usr_27.txt	/*usr_27.txt*
usr_28.txt	usr_28.txt	/*usr_28.txt*
usr_29.txt	usr_29.txt	/*usr_29.txt*
usr_30.txt	usr_30.txt	/*usr_30.txt*
usr_31.txt	usr_31.txt	/*usr_31.txt*
usr_32.txt	usr_32.txt	/*usr_32.txt*
usr_40.txt	usr_40.txt	/*usr_40.txt*
usr_41.txt	usr_41.txt	/*usr_41.txt*
usr_42.txt	usr_42.txt	/*usr_42.txt*
usr_43.txt	usr_43.txt	/*usr_43.txt*
usr_44.txt	usr_44.txt	/*usr_44.txt*
usr_45.txt	usr_45.txt	/*usr_45.txt*
usr_90.txt	usr_90.txt	/*usr_90.txt*
usr_toc.txt	usr_toc.txt	/*usr_toc.txt*
utf-8	mbyte.txt	/*utf-8*
utf-8-char-arg	mbyte.txt	/*utf-8-char-arg*
utf-8-in-xwindows	mbyte.txt	/*utf-8-in-xwindows*
utf-8-typing	mbyte.txt	/*utf-8-typing*

utf8 mbyte.txt /*utf8*
v visual.txt /*v*
v: eval.txt /*v:*
v:argv eval.txt /*v:argv*
v:beval_bufnr eval.txt /*v:beval_bufnr*
v:beval_col eval.txt /*v:beval_col*
v:beval_lnum eval.txt /*v:beval_lnum*
v:beval_text eval.txt /*v:beval_text*
v:beval_winid eval.txt /*v:beval_winid*
v:beval_winnr eval.txt /*v:beval_winnr*
v:char eval.txt /*v:char*
v:charconvert_from eval.txt /*v:charconvert_from*
v:charconvert_to eval.txt /*v:charconvert_to*
v:cmdarg eval.txt /*v:cmdarg*
v:cmdbang eval.txt /*v:cmdbang*
v:completed_item eval.txt /*v:completed_item*
v:count eval.txt /*v:count*
v:count1 eval.txt /*v:count1*
v:ctype eval.txt /*v:ctype*
v:dying eval.txt /*v:dying*
v:echospace eval.txt /*v:echospace*
v:errmsg eval.txt /*v:errmsg*
v:errors eval.txt /*v:errors*
v:event eval.txt /*v:event*
v:exception eval.txt /*v:exception*
v:false eval.txt /*v:false*
v:fcs_choice eval.txt /*v:fcs_choice*
v:fcs_reason eval.txt /*v:fcs_reason*
v:fname_diff eval.txt /*v:fname_diff*
v:fname_in eval.txt /*v:fname_in*
v:fname_new eval.txt /*v:fname_new*
v:fname_out eval.txt /*v:fname_out*
v:folddashes eval.txt /*v:folddashes*
v:foldend eval.txt /*v:foldend*
v:foldlevel eval.txt /*v:foldlevel*
v:foldstart eval.txt /*v:foldstart*
v:hlsearch eval.txt /*v:hlsearch*
v:insertmode eval.txt /*v:insertmode*
v:key eval.txt /*v:key*
v:lang eval.txt /*v:lang*
v:lc_time eval.txt /*v:lc_time*
v:lnum eval.txt /*v:lnum*
v:mouse_col eval.txt /*v:mouse_col*
v:mouse_lnum eval.txt /*v:mouse_lnum*
v:mouse_win eval.txt /*v:mouse_win*
v:mouse_winid eval.txt /*v:mouse_winid*
v:none eval.txt /*v:none*
v:null eval.txt /*v:null*
v:oldfiles eval.txt /*v:oldfiles*
v:operator eval.txt /*v:operator*
v:option_command eval.txt /*v:option_command*
v:option_new eval.txt /*v:option_new*
v:option_old eval.txt /*v:option_old*
v:option_oldglobal eval.txt /*v:option_oldglobal*

v:option_oldlocal eval.txt /*v:option_oldlocal*
v:option_type eval.txt /*v:option_type*
v:prevcount eval.txt /*v:prevcount*
v:profiling eval.txt /*v:profiling*
v:progname eval.txt /*v:progname*
v:progpath eval.txt /*v:progpath*
v:register eval.txt /*v:register*
v:scrollstart eval.txt /*v:scrollstart*
v:searchforward eval.txt /*v:searchforward*
v:servername eval.txt /*v:servername*
v:shell_error eval.txt /*v:shell_error*
v:statusmsg eval.txt /*v:statusmsg*
v:swapchoice eval.txt /*v:swapchoice*
v:swapcommand eval.txt /*v:swapcommand*
v:swapname eval.txt /*v:swapname*
v:t_TYPE eval.txt /*v:t_TYPE*
v:t_blob eval.txt /*v:t_blob*
v:t_bool eval.txt /*v:t_bool*
v:t_channel eval.txt /*v:t_channel*
v:t_dict eval.txt /*v:t_dict*
v:t_float eval.txt /*v:t_float*
v:t_func eval.txt /*v:t_func*
v:t_job eval.txt /*v:t_job*
v:t_list eval.txt /*v:t_list*
v:t_none eval.txt /*v:t_none*
v:t_number eval.txt /*v:t_number*
v:t_string eval.txt /*v:t_string*
v:termblinkresp eval.txt /*v:termblinkresp*
v:termrbgresp eval.txt /*v:termrbgresp*
v:termresponse eval.txt /*v:termresponse*
v:termrfgresp eval.txt /*v:termrfgresp*
v:termstyleresp eval.txt /*v:termstyleresp*
v:termu7resp eval.txt /*v:termu7resp*
v:testing eval.txt /*v:testing*
v:this_session eval.txt /*v:this_session*
v:throwpoint eval.txt /*v:throwpoint*
v:true eval.txt /*v:true*
v:val eval.txt /*v:val*
v:var eval.txt /*v:var*
v:version eval.txt /*v:version*
v:versionlong eval.txt /*v:versionlong*
v:vim_did_enter eval.txt /*v:vim_did_enter*
v:warningmsg eval.txt /*v:warningmsg*
v:windowid eval.txt /*v:windowid*
v_! change.txt /*v_!*
v_$ visual.txt /*v_$*
v_: cmdline.txt /*v_:*
v_< change.txt /*v_<*
v_<BS> change.txt /*v_<BS>*
v_<Del> change.txt /*v_<Del>*
v_<Esc> visual.txt /*v_<Esc>*
v_= change.txt /*v_=*
v_> change.txt /*v_>*
v_C change.txt /*v_C*

v_CTRL-A change.txt /*v_CTRL-A*
v_CTRL-C visual.txt /*v_CTRL-C*
v_CTRL-G visual.txt /*v_CTRL-G*
v_CTRL-H change.txt /*v_CTRL-H*
v_CTRL-O visual.txt /*v_CTRL-O*
v_CTRL-V visual.txt /*v_CTRL-V*
v_CTRL-X change.txt /*v_CTRL-X*
v_CTRL-Z starting.txt /*v_CTRL-Z*
v_CTRL-_CTRL-G intro.txt /*v_CTRL-_CTRL-G*
v_CTRL-_CTRL-N intro.txt /*v_CTRL-_CTRL-N*
v_CTRL-] tagsrch.txt /*v_CTRL-]*
v_D change.txt /*v_D*
v_J change.txt /*v_J*
v_K various.txt /*v_K*
v_O visual.txt /*v_O*
v_P change.txt /*v_P*
v_R change.txt /*v_R*
v_S change.txt /*v_S*
v_U change.txt /*v_U*
v_V visual.txt /*v_V*
v_X change.txt /*v_X*
v_Y change.txt /*v_Y*
v_a motion.txt /*v_a*
v_a' motion.txt /*v_a'*
v_a(motion.txt /*v_a(*
v_a) motion.txt /*v_a)*
v_a< motion.txt /*v_a<*
v_a> motion.txt /*v_a>*
v_aB motion.txt /*v_aB*
v_aW motion.txt /*v_aW*
v_a[motion.txt /*v_a[*
v_a] motion.txt /*v_a]*
v_a` motion.txt /*v_a`*
v_ab motion.txt /*v_ab*
v_ap motion.txt /*v_ap*
v_aquote motion.txt /*v_aquote*
v_as motion.txt /*v_as*
v_at motion.txt /*v_at*
v_aw motion.txt /*v_aw*
v_a{ motion.txt /*v_a{*
v_a} motion.txt /*v_a}*
v_b_< visual.txt /*v_b_<*
v_b_<_example visual.txt /*v_b_<_example*
v_b_> visual.txt /*v_b_>*
v_b_>_example visual.txt /*v_b_>_example*
v_b_A visual.txt /*v_b_A*
v_b_A_example visual.txt /*v_b_A_example*
v_b_C visual.txt /*v_b_C*
v_b_D change.txt /*v_b_D*
v_b_I visual.txt /*v_b_I*
v_b_I_example visual.txt /*v_b_I_example*
v_b_c visual.txt /*v_b_c*
v_b_r visual.txt /*v_b_r*
v_b_r_example visual.txt /*v_b_r_example*

v_c change.txt /*v_c*
v_d change.txt /*v_d*
v_g? change.txt /*v_g?*
v_gF editing.txt /*v_gF*
v_gJ change.txt /*v_gJ*
v_gN visual.txt /*v_gN*
v_gV visual.txt /*v_gV*
v_g] tagsrch.txt /*v_g]*
v_g_CTRL-A change.txt /*v_g_CTRL-A*
v_g_CTRL-G editing.txt /*v_g_CTRL-G*
v_g_CTRL-X change.txt /*v_g_CTRL-X*
v_g_CTRL-] tagsrch.txt /*v_g_CTRL-]*
v_gf editing.txt /*v_gf*
v_gn visual.txt /*v_gn*
v_gq change.txt /*v_gq*
v_gv visual.txt /*v_gv*
v_gw change.txt /*v_gw*
v_i motion.txt /*v_i*
v_i' motion.txt /*v_i'*
v_i(motion.txt /*v_i(*
v_i) motion.txt /*v_i)*
v_i< motion.txt /*v_i<*
v_i> motion.txt /*v_i>*
v_iB motion.txt /*v_iB*
v_iW motion.txt /*v_iW*
v_i[motion.txt /*v_i[*
v_i] motion.txt /*v_i]*
v_i` motion.txt /*v_i`*
v_ib motion.txt /*v_ib*
v_ip motion.txt /*v_ip*
v_iquote motion.txt /*v_iquote*
v_is motion.txt /*v_is*
v_it motion.txt /*v_it*
v_iw motion.txt /*v_iw*
v_i{ motion.txt /*v_i{*
v_i} motion.txt /*v_i}*
v_o visual.txt /*v_o*
v_p change.txt /*v_p*
v_r change.txt /*v_r*
v_s change.txt /*v_s*
v_u change.txt /*v_u*
v_v visual.txt /*v_v*
v_x change.txt /*v_x*
v_y change.txt /*v_y*
v_~ change.txt /*v_~*
vab motion.txt /*vab*
val-variable eval.txt /*val-variable*
valgrind debug.txt /*valgrind*
values() eval.txt /*values()*
var-functions usr_41.txt /*var-functions*
variables eval.txt /*variables*
various various.txt /*various*
various-cmds various.txt /*various-cmds*
various-functions usr_41.txt /*various-functions*

various-motions	motion.txt	/*various-motions*
various.txt	various.txt	/*various.txt*
vb.vim	syntax.txt	/*vb.vim*
vba	pi_vimball.txt	/*vba*
verbose	starting.txt	/*verbose*
version-5.1	version5.txt	/*version-5.1*
version-5.2	version5.txt	/*version-5.2*
version-5.3	version5.txt	/*version-5.3*
version-5.4	version5.txt	/*version-5.4*
version-5.5	version5.txt	/*version-5.5*
version-5.6	version5.txt	/*version-5.6*
version-5.7	version5.txt	/*version-5.7*
version-5.8	version5.txt	/*version-5.8*
version-6.1	version6.txt	/*version-6.1*
version-6.2	version6.txt	/*version-6.2*
version-6.3	version6.txt	/*version-6.3*
version-6.4	version6.txt	/*version-6.4*
version-7.0	version7.txt	/*version-7.0*
version-7.1	version7.txt	/*version-7.1*
version-7.2	version7.txt	/*version-7.2*
version-7.3	version7.txt	/*version-7.3*
version-7.4	version7.txt	/*version-7.4*
version-8.0	version8.txt	/*version-8.0*
version-8.1	version8.txt	/*version-8.1*
version-8.2	version8.txt	/*version-8.2*
version-variable	eval.txt	/*version-variable*
version4.txt	version4.txt	/*version4.txt*
version5.txt	version5.txt	/*version5.txt*
version6.txt	version6.txt	/*version6.txt*
version7.0	version7.txt	/*version7.0*
version7.1	version7.txt	/*version7.1*
version7.2	version7.txt	/*version7.2*
version7.3	version7.txt	/*version7.3*
version7.4	version7.txt	/*version7.4*
version7.txt	version7.txt	/*version7.txt*
version8.0	version8.txt	/*version8.0*
version8.1	version8.txt	/*version8.1*
version8.2	version8.txt	/*version8.2*
version8.txt	version8.txt	/*version8.txt*
versionlong-variable	eval.txt	/*versionlong-variable*
vi	intro.txt	/*vi*
vi-differences	vi_diff.txt	/*vi-differences*
vi-features	vi_diff.txt	/*vi-features*
vi:	options.txt	/*vi:*
vi_diff.txt	vi_diff.txt	/*vi_diff.txt*
vib	motion.txt	/*vib*
view	starting.txt	/*view*
view-diffs	diff.txt	/*view-diffs*
view-file	starting.txt	/*view-file*
views-sessions	starting.txt	/*views-sessions*
vim-7.4	version7.txt	/*vim-7.4*
vim-8	version8.txt	/*vim-8*
vim-8.1	version8.txt	/*vim-8.1*
vim-8.2	version8.txt	/*vim-8.2*

vim-additions vi_diff.txt /*vim-additions*
vim-announce intro.txt /*vim-announce*
vim-arguments starting.txt /*vim-arguments*
vim-default-editor gui_w32.txt /*vim-default-editor*
vim-dev intro.txt /*vim-dev*
vim-mac intro.txt /*vim-mac*
vim-modes intro.txt /*vim-modes*
vim-modes-intro intro.txt /*vim-modes-intro*
vim-script-intro usr_41.txt /*vim-script-intro*
vim-use intro.txt /*vim-use*
vim-variable eval.txt /*vim-variable*
vim.vim syntax.txt /*vim.vim*
vim7 version7.txt /*vim7*
vim8 version8.txt /*vim8*
vim: options.txt /*vim:*
vim_announce intro.txt /*vim_announce*
vim_dev intro.txt /*vim_dev*
vim_did_enter-variable eval.txt /*vim_did_enter-variable*
vim_mac intro.txt /*vim_mac*
vim_starting eval.txt /*vim_starting*
vim_use intro.txt /*vim_use*
vimball pi_vimball.txt /*vimball*
vimball-contents pi_vimball.txt /*vimball-contents*
vimball-extract pi_vimball.txt /*vimball-extract*
vimball-history pi_vimball.txt /*vimball-history*
vimball-intro pi_vimball.txt /*vimball-intro*
vimball-manual pi_vimball.txt /*vimball-manual*
vimball-windows pi_vimball.txt /*vimball-windows*
vimdev intro.txt /*vimdev*
vimdiff diff.txt /*vimdiff*
vimfiles options.txt /*vimfiles*
viminfo starting.txt /*viminfo*
viminfo-! options.txt /*viminfo-!*
viminfo-% options.txt /*viminfo-%*
viminfo-’ options.txt /*viminfo-’*
viminfo-/ options.txt /*viminfo-\/*
viminfo-: options.txt /*viminfo-:*
viminfo-< options.txt /*viminfo-<*
viminfo-@ options.txt /*viminfo-@*
viminfo-c options.txt /*viminfo-c*
viminfo-encoding starting.txt /*viminfo-encoding*
viminfo-errors starting.txt /*viminfo-errors*
viminfo-f options.txt /*viminfo-f*
viminfo-file starting.txt /*viminfo-file*
viminfo-file-marks starting.txt /*viminfo-file-marks*
viminfo-file-name starting.txt /*viminfo-file-name*
viminfo-h options.txt /*viminfo-h*
viminfo-n options.txt /*viminfo-n*
viminfo-quote options.txt /*viminfo-quote*
viminfo-r options.txt /*viminfo-r*
viminfo-read starting.txt /*viminfo-read*
viminfo-read-write starting.txt /*viminfo-read-write*
viminfo-s options.txt /*viminfo-s*
viminfo-timestamp starting.txt /*viminfo-timestamp*

viminfo-write starting.txt /*viminfo-write*
vimrc starting.txt /*vimrc*
vimrc-filetype usr_05.txt /*vimrc-filetype*
vimrc-intro usr_05.txt /*vimrc-intro*
vimrc-option-example starting.txt /*vimrc-option-example*
vimrc_example.vim usr_05.txt /*vimrc_example.vim*
vimscript-version eval.txt /*vimscript-version*
vimscript-versions eval.txt /*vimscript-versions*
vimtutor usr_01.txt /*vimtutor*
virtcol() eval.txt /*virtcol()*
visual-block visual.txt /*visual-block*
visual-change visual.txt /*visual-change*
visual-examples visual.txt /*visual-examples*
visual-index index.txt /*visual-index*
visual-mode visual.txt /*visual-mode*
visual-operators visual.txt /*visual-operators*
visual-repeat visual.txt /*visual-repeat*
visual-search visual.txt /*visual-search*
visual-start visual.txt /*visual-start*
visual-use visual.txt /*visual-use*
visual.txt visual.txt /*visual.txt*
visualmode() eval.txt /*visualmode()*
vms os_vms.txt /*vms*
vms-authors os_vms.txt /*vms-authors*
vms-changes os_vms.txt /*vms-changes*
vms-compiling os_vms.txt /*vms-compiling*
vms-deploy os_vms.txt /*vms-deploy*
vms-download os_vms.txt /*vms-download*
vms-gui os_vms.txt /*vms-gui*
vms-notes os_vms.txt /*vms-notes*
vms-problems os_vms.txt /*vms-problems*
vms-started os_vms.txt /*vms-started*
vms-usage os_vms.txt /*vms-usage*
vote-for-features sponsor.txt /*vote-for-features*
votes-counted sponsor.txt /*votes-counted*
votes-for-changes todo.txt /*votes-for-changes*
vreplace-mode insert.txt /*vreplace-mode*
vt100-cursor-keys term.txt /*vt100-cursor-keys*
vt100-function-keys term.txt /*vt100-function-keys*
w motion.txt /*w*
w32-clientserver remote.txt /*w32-clientserver*
w32-xpm-support gui_w32.txt /*w32-xpm-support*
w: eval.txt /*w:*
w:current_syntax syntax.txt /*w:current_syntax*
w:quickfix_title quickfix.txt /*w:quickfix_title*
w:var eval.txt /*w:var*
waittime channel.txt /*waittime*
warningmsg-variable eval.txt /*warningmsg-variable*
white-space pattern.txt /*white-space*
whitespace pattern.txt /*whitespace*
wildcard editing.txt /*wildcard*
wildcards editing.txt /*wildcards*
wildmenumode() eval.txt /*wildmenumode()*
win16 os_win32.txt /*win16*

win32 os_win32.txt /*win32*
win32-!start gui_w32.txt /*win32-!start*
win32-PATH os_win32.txt /*win32-PATH*
win32-backslashes os_win32.txt /*win32-backslashes*
win32-cmdargs os_win32.txt /*win32-cmdargs*
win32-colors gui_w32.txt /*win32-colors*
win32-compiling os_win32.txt /*win32-compiling*
win32-curdir os_win32.txt /*win32-curdir*
win32-faq os_win32.txt /*win32-faq*
win32-gettext mlang.txt /*win32-gettext*
win32-gui gui_w32.txt /*win32-gui*
win32-hidden-menus gui.txt /*win32-hidden-menus*
win32-mouse os_win32.txt /*win32-mouse*
win32-open-with-menu gui_w32.txt /*win32-open-with-menu*
win32-popup-menu gui_w32.txt /*win32-popup-menu*
win32-problems os_win32.txt /*win32-problems*
win32-quotes os_win32.txt /*win32-quotes*
win32-restore os_win32.txt /*win32-restore*
win32-startup os_win32.txt /*win32-startup*
win32-term os_win32.txt /*win32-term*
win32-vimrun gui_w32.txt /*win32-vimrun*
win32-win3.1 os_win32.txt /*win32-win3.1*
win32-win95 os_win32.txt /*win32-win95*
win32s os_win32.txt /*win32s*
win_execute() eval.txt /*win_execute()*
win_findbuf() eval.txt /*win_findbuf()*
win_getid() eval.txt /*win_getid()*
win_gotoid() eval.txt /*win_gotoid()*
win_id2tabwin() eval.txt /*win_id2tabwin()*
win_id2win() eval.txt /*win_id2win()*
win_screenpos() eval.txt /*win_screenpos()*
win_splitmove() eval.txt /*win_splitmove()*
winbufnr() eval.txt /*winbufnr()*
wincol() eval.txt /*wincol()*
window windows.txt /*window*
window-ID windows.txt /*window-ID*
window-contents intro.txt /*window-contents*
window-exit editing.txt /*window-exit*
window-functions usr_41.txt /*window-functions*
window-move-cursor windows.txt /*window-move-cursor*
window-moving windows.txt /*window-moving*
window-resize windows.txt /*window-resize*
window-size term.txt /*window-size*
window-size-functions usr_41.txt /*window-size-functions*
window-tag windows.txt /*window-tag*
window-toolbar gui.txt /*window-toolbar*
window-variable eval.txt /*window-variable*
windowid windows.txt /*windowid*
windowid-variable eval.txt /*windowid-variable*
windows windows.txt /*windows*
windows-3.1 os_win32.txt /*windows-3.1*
windows-icon os_win32.txt /*windows-icon*
windows-intro windows.txt /*windows-intro*
windows-starting windows.txt /*windows-starting*

windows.txt	windows.txt	/*windows.txt*
windows95	os_win32.txt	/*windows95*
windows98	os_win32.txt	/*windows98*
windowsme	os_win32.txt	/*windowsme*
winheight()	eval.txt	/*winheight()*
winid	windows.txt	/*winid*
winlayout()	eval.txt	/*winlayout()*
winline()	eval.txt	/*winline()*
winnr()	eval.txt	/*winnr()*
winrestcmd()	eval.txt	/*winrestcmd()*
winrestview()	eval.txt	/*winrestview()*
winsaveview()	eval.txt	/*winsaveview()*
winwidth()	eval.txt	/*winwidth()*
word	motion.txt	/*word*
word-count	editing.txt	/*word-count*
word-motions	motion.txt	/*word-motions*
wordcount()	eval.txt	/*wordcount()*
workbench	starting.txt	/*workbench*
workshop	workshop.txt	/*workshop*
workshop-support	workshop.txt	/*workshop-support*
workshop.txt	workshop.txt	/*workshop.txt*
wrap-off	intro.txt	/*wrap-off*
write-compiler-plugin	usr_41.txt	/*write-compiler-plugin*
write-device	editing.txt	/*write-device*
write-fail	editing.txt	/*write-fail*
write-filetype-plugin	usr_41.txt	/*write-filetype-plugin*
write-library-script	usr_41.txt	/*write-library-script*
write-local-help	usr_41.txt	/*write-local-help*
write-permissions	editing.txt	/*write-permissions*
write-plugin	usr_41.txt	/*write-plugin*
write-plugin-quickload	usr_41.txt	/*write-plugin-quickload*
write-quit	editing.txt	/*write-quit*
write-readonly	editing.txt	/*write-readonly*
writefile()	eval.txt	/*writefile()*
writing	editing.txt	/*writing*
www	intro.txt	/*www*
x	change.txt	/*x*
x-input-method	mbyte.txt	/*x-input-method*
x-resources	version5.txt	/*x-resources*
x11-clientserver	remote.txt	/*x11-clientserver*
x11-cut-buffer	gui_x11.txt	/*x11-cut-buffer*
x11-selection	gui_x11.txt	/*x11-selection*
xf86conf.vim	syntax.txt	/*xf86conf.vim*
xfontset	mbyte.txt	/*xfontset*
xfree-xterm	syntax.txt	/*xfree-xterm*
xim	mbyte.txt	/*xim*
xim-input-style	mbyte.txt	/*xim-input-style*
xiterm	syntax.txt	/*xiterm*
xml-folding	syntax.txt	/*xml-folding*
xml-omni-datafile	insert.txt	/*xml-omni-datafile*
xml.vim	syntax.txt	/*xml.vim*
xor()	eval.txt	/*xor()*
xpm.vim	syntax.txt	/*xpm.vim*
xterm-8-bit	term.txt	/*xterm-8-bit*

xterm-8bit term.txt /*xterm-8bit*
xterm-blink syntax.txt /*xterm-blink*
xterm-blinking-cursor syntax.txt /*xterm-blinking-cursor*
xterm-bracketed-paste term.txt /*xterm-bracketed-paste*
xterm-clipboard term.txt /*xterm-clipboard*
xterm-codes term.txt /*xterm-codes*
xterm-color syntax.txt /*xterm-color*
xterm-command-server term.txt /*xterm-command-server*
xterm-copy-paste term.txt /*xterm-copy-paste*
xterm-cursor-keys term.txt /*xterm-cursor-keys*
xterm-end-home-keys term.txt /*xterm-end-home-keys*
xterm-function-keys term.txt /*xterm-function-keys*
xterm-modifier-keys term.txt /*xterm-modifier-keys*
xterm-mouse options.txt /*xterm-mouse*
xterm-mouse-wheel scroll.txt /*xterm-mouse-wheel*
xterm-resize term.txt /*xterm-resize*
xterm-save-screen tips.txt /*xterm-save-screen*
xterm-screens tips.txt /*xterm-screens*
xterm-scroll-region term.txt /*xterm-scroll-region*
xterm-shifted-keys term.txt /*xterm-shifted-keys*
xterm-true-color term.txt /*xterm-true-color*
y change.txt /*y*
yaml.vim syntax.txt /*yaml.vim*
yank change.txt /*yank*
ye-option-gone version4.txt /*ye-option-gone*
year-2000 intro.txt /*year-2000*
your-runtime-dir usr_43.txt /*your-runtime-dir*
yy change.txt /*yy*
z index.txt /*z*
z+ scroll.txt /*z+*
z- scroll.txt /*z-*
z. scroll.txt /*z.*
z/OS os_390.txt /*z\/OS*
z<CR> scroll.txt /*z<CR>*
z<Left> scroll.txt /*z<Left>*
z<Right> scroll.txt /*z<Right>*
z= spell.txt /*z=*
zA fold.txt /*zA*
zC fold.txt /*zC*
zD fold.txt /*zD*
zE fold.txt /*zE*
zF fold.txt /*zF*
zG spell.txt /*zG*
zH scroll.txt /*zH*
zL scroll.txt /*zL*
zM fold.txt /*zM*
zN fold.txt /*zN*
zN<CR> scroll.txt /*zN<CR>*
zO fold.txt /*zO*
zOS os_390.txt /*zOS*
zOS-Bugs os_390.txt /*zOS-Bugs*
zOS-Motif os_390.txt /*zOS-Motif*
zOS-PuTTY os_390.txt /*zOS-PuTTY*
zOS-has-ebcdic os_390.txt /*zOS-has-ebcdic*

zOS-limitations	os_390.txt	/*zOS-limitations*
zOS-open-source	os_390.txt	/*zOS-open-source*
zR	fold.txt	/*zR*
zW	spell.txt	/*zW*
zX	fold.txt	/*zX*
z^	scroll.txt	/*z^*
za	fold.txt	/*za*
zb	scroll.txt	/*zb*
zc	fold.txt	/*zc*
zd	fold.txt	/*zd*
ze	scroll.txt	/*ze*
zf	fold.txt	/*zf*
zg	spell.txt	/*zg*
zh	scroll.txt	/*zh*
zi	fold.txt	/*zi*
zip	pi_zip.txt	/*zip*
zip-contents	pi_zip.txt	/*zip-contents*
zip-copyright	pi_zip.txt	/*zip-copyright*
zip-extension	pi_zip.txt	/*zip-extension*
zip-history	pi_zip.txt	/*zip-history*
zip-manual	pi_zip.txt	/*zip-manual*
zip-usage	pi_zip.txt	/*zip-usage*
zip-x	pi_zip.txt	/*zip-x*
zj	fold.txt	/*zj*
zk	fold.txt	/*zk*
zl	scroll.txt	/*zl*
zm	fold.txt	/*zm*
zn	fold.txt	/*zn*
zo	fold.txt	/*zo*
zr	fold.txt	/*zr*
zs	scroll.txt	/*zs*
zsh.vim	syntax.txt	/*zsh.vim*
zt	scroll.txt	/*zt*
zuG	spell.txt	/*zuG*
zuW	spell.txt	/*zuW*
zug	spell.txt	/*zug*
zuw	spell.txt	/*zuw*
zv	fold.txt	/*zv*
zw	spell.txt	/*zw*
zx	fold.txt	/*zx*
zz	scroll.txt	/*zz*
{	motion.txt	/*{*
{Visual}	intro.txt	/*{Visual}*
{address}	cmdline.txt	/*{address}*
{arglist}	editing.txt	/*{arglist}*
{bufname}	windows.txt	/*{bufname}*
{char1-char2}	intro.txt	/*{char1-char2}*
{event}	autocmd.txt	/*{event}*
{file}	editing.txt	/*{file}*
{group-name}	syntax.txt	/*{group-name}*
{lhs}	map.txt	/*{lhs}*
{motion}	intro.txt	/*{motion}*
{move-around}	visual.txt	/*{move-around}*
{offset}	pattern.txt	/*{offset}*

{pat}	autocmd.txt	/*{pat}*
{register}	change.txt	/*{register}*
{rhs}	map.txt	/*{rhs}*
{subject}	helphelp.txt	/*{subject}*
{}	intro.txt	/*{}*
}	motion.txt	/*}*
~	change.txt	/*~*